The caOS Operating System Manual

Angelo Coppi

October 27, 2026

Contents

1 Introduction: Bare Metal and RTOS **79**
1.1 Chapter 1 - Introduction: Bare Metal and RTOS 79
1.1.1 1.1 What are embedded systems? 79
1.1.2 1.2 “Bare Metal” Programming 79
1.1.3 1.3 What is an RTOS? 80
1.1.4 1.4 When to use Bare Metal or RTOS? 81
1.1.5 1.5 RTOS on ARM Cortex-M microcontrollers 82
1.1.6 1.6 Available RTOS 83

2 The Kernel: Tasks, Scheduler and Stack Management **85**
2.1 Chapter 2 - The Kernel: Tasks, Scheduling, and Memory Management 85
2.1.1 2.1 The Task 85
2.1.2 2.2 The Scheduler 87
2.1.3 2.3 The Context Switch 88
2.1.4 2.4 Working Memory Management (Stack) 89
2.1.5 2.5 The System Timer 90
2.1.6 2.6 System startup 90

3 Resources: Synchronization and Mutexes **93**
3.1 Chapter 3 - Resources: Sharing and Synchronization 93
3.1.1 3.1 The concurrency problem 93
3.1.2 3.2 Safe (Atomic) Operations 95
3.1.3 3.3 Mutex 95
3.1.4 3.4 Semaphore 97
3.1.5 3.5 Condition Variable 97
3.1.6 3.6 Spinlock 98
3.1.7 3.7 Message Queue 98
3.1.8 3.8 Event Groups 99
3.1.9 3.9 Task Notification 99
3.1.10 3.10 Deadlock 99

4 Memory: Partitioning and Protection **101**

5 Embedded I/O: Drivers, Sensors, Timers **103**
5.1 Chapter 5 - Embedded I/O: Drivers, Sensors, and Timers 103
5.1.1 5.1 GPIO (General Purpose Inputs/Outputs) 103
5.1.2 5.2 Driver model: Weak Functions 104
5.1.3 5.3 UART (Universal Asynchronous Receiver/Transmitter) 105
5.1.4 5.4 Communication buses: SPI and I2C 106
5.1.5 5.5 Hardware Timers 106
5.1.6 5.6 Time measurement and performance analysis 107
5.1.7 5.7 Interrupts and I/O: the RTOS challenge 107

6 Reliability: Watchdog, Errors, OTA **109**
6.1 Chapter 6 - Reliability: Watchdog, Error Handling and Updates . . . 109
6.1.1 6.1 Types of errors in embedded systems . . . 109
6.1.2 6.2 Cortex-M errors . . . 109
6.1.3 6.3 Fault handler in an RTOS . . . 110
6.1.4 6.4 Watchdog Timer . . . 111
6.1.5 6.5 Checks and Safety . . . 112
6.1.6 6.6 Protection against corruption . . . 112
6.1.7 6.7 Reset and Recovery . . . 113
6.1.8 6.8 Firmware Update (OTA) . . . 114

I caOS Implementation **117**

7 caOS Overview **119**
7.1 Chapter 7 - Overview of caOS . . . 119
7.1.1 7.1 What is caOS? . . . 119
7.1.2 7.2 Project Structure . . . 120
7.1.3 7.3 Supported Microcontrollers . . . 121
7.1.4 7.4 caOS Modules . . . 122
7.1.5 7.5 Configuration . . . 124
7.1.6 7.6 Build System . . . 124
7.1.7 7.7 Entry point: ca_os_main() . . . 125
7.1.8 7.8 Configurable Tick . . . 125

8 caOS Features **127**
8.1 caOS Features . . . 127
8.1.1 1. Scheduler and Kernel . . . 127
8.1.2 2. Hardware Drivers . . . 129
8.1.3 3. Network Stack . . . 131
8.1.4 4. Cryptography . . . 133

9 caOS Kernel: Scheduler and Tasks **135**
9.1 Chapter 8 - The caOS Kernel: Scheduling and Task Management . . . 135
9.1.1 8.1 The Task in caOS . . . 135
9.1.2 8.2 Task Creation . . . 137
9.1.3 8.3 Stack Allocation . . . 140
9.1.4 8.4 Scheduling Algorithm . . . 141
9.1.5 8.5 Context Switching . . . 145
9.1.6 8.6 Idle task and maintenance . . . 149
9.1.7 8.7 Stack protection . . . 150
9.1.8 8.8 Suspension, Blocking and Wake-up . . . 150
9.1.9 8.9 System Startup . . . 153

10 Synchronization in caOS **157**
10.1 Chapter 9 - Synchronization in caOS . . . 157
10.1.1 9.1 Synchronization Architecture . . . 157
10.1.2 9.2 Mutex . . . 157
10.1.3 9.3 Recursive Mutex . . . 159
10.1.4 9.4 Semaphore . . . 159
10.1.5 9.5 Condition variable . . . 161
10.1.6 9.6 Critical Sections . . . 161

10.1.7 9.7 Code . . . 162
10.1.8 9.8 Event Groups . . . 163
10.1.9 9.9 Task Notification . . . 164
10.1.10 9.10 Stream Buffer . . . 165
10.1.11 9.11 Message Buffer . . . 165
10.1.12 9.12 Read-Write Lock (`ca_rwlock_t`) . . . 166
10.1.13 9.13 Barrier (`ca_barrier_t`) . . . 167
10.1.14 9.14 Block Pool (`ca_block_pool_t`) . . . 168

11 Memory Management in caOS **171**
11.1 Chapter 10 - Memory Management in caOS . . . 171
11.1.1 10.1 Memory Layout . . . 171
11.1.2 10.2 Heap Allocator . . . 173
11.1.3 10.3 MPU Protection . . . 174
11.1.4 10.4 MemAux: newlib replacement . . . 177
11.1.5 10.5 Summary: who allocates what and where . . . 177

12 Drivers and I/O in caOS **179**
12.1 Chapter 11 - Drivers and I/O in caOS . . . 179
12.1.1 11.1 The Extern pattern . . . 179
12.1.2 11.2 GPIO Driver . . . 181
12.1.3 11.3 Debug UART . . . 181
12.1.4 11.4 Stdout and repeater . . . 183
12.1.5 11.5 CLI (Command Line Interface) . . . 184
12.1.6 11.6 Hardware timers . . . 189
12.1.7 11.7 Software Timer . . . 191
12.1.8 11.8 Profiling with Measure . . . 192
12.1.9 11.9 ITM (Instrumentation Trace Macrocell) . . . 192
12.1.10 11.10 Hardware Watchdog (wdt) . . . 193
12.1.11 11.11 Backup registers (bkp) . . . 194
12.1.12 11.12 CORDIC Coprocessor (cordic) . . . 195
12.1.13 11.13 FMAC filter accelerator (fmac) . . . 196
12.1.14 11.14 Operational amplifiers (opamp) . . . 197

13 SafeGuard and Fault Recovery in caOS **199**
13.1 Chapter 12 - SafeGuard and Fault Recovery in caOS . . . 199
13.1.1 12.1 SafeGuard: control with recovery . . . 199
13.1.2 12.2 CPU Fault Handling: dedicated handlers and per-task recovery . . . 203
13.1.3 12.3 Stack overflow and corruption detection: defense levels . . . 204
13.1.4 12.4 Kernel protections: overall framework . . . 206
13.1.5 12.5 CRC32 . . . 208
13.1.6 12.6 Keys Store . . . 209
13.1.7 12.7 Diagnostica: dump e statistiche . . . 210

14 Thread Pool in caOS **213**
14.1 Thread Pool in caOS . . . 213
14.1.1 Motivation . . . 213
14.1.2 Pool Architecture . . . 213
14.1.3 Synchronization mechanism . . . 214
14.1.4 Public API . . . 214
14.1.5 Configuration in `os_config.h` . . . 215
14.1.6 Memory Usage . . . 216

14.1.7 Use Cases . . . 216
14.1.8 Limitations . . . 216
14.1.9 Internal Implementation . . . 216

15 Cryptography in caOS **219**
15.1 Chapter 13 - Cryptography in caOS . . . 219
15.1.1 13.1 Hardware driver overview . . . 220
15.1.2 13.2 CRC32 . . . 220
15.1.3 13.3 HASH (SHA-256) . . . 220
15.1.4 13.4 AES . . . 223
15.1.5 13.5 RNG . . . 225
15.1.6 13.6 Initialization in `ca_os_configure()` . . . 226
15.1.7 13.7 Pattern: Key Derivation . . . 226
15.1.8 13.8 Crypto Infrastructure (`caos/crypto/`) . . . 227
15.1.9 13.9 Keys Store (`keys_store.h`) . . . 228
15.1.10 13.10 AES Channel Service (`aes_channel_service.h`) . . . 230
15.1.11 13.11 Crypto levels: architecture summary . . . 236
15.1.12 13.12 Availability by MCU family . . . 236
15.1.13 13.13 Keystore Service (`keystore_service.h`) . . . 236
15.1.14 13.14 ECDSA P-256 (`ecdsa_p256.h`) . . . 241
15.1.15 13.15 Crypto module status summary . . . 242

16 caOS Network Stack: Low Level **245**
16.1 Chapter 14 - caOS Network Stack: Low Level . . . 245
16.1.1 14.0 Zero-copy Architecture . . . 245
16.1.2 14.1 MPU Configuration — NCacheable Regions . . . 246
16.1.3 14.2 Network stack modules . . . 247
16.1.4 14.2.1 Interface abstraction — `ca_net_iface_t` . . . 248
16.1.5 14.3 Initialization Sequence . . . 250
16.1.6 14.4 Configuration — `ca_net_cfg_t` . . . 251
16.1.7 14.5 Link control . . . 251
16.1.8 14.6 Source files . . . 251

17 caOS Network Stack: net_proto **253**
17.1 Chapter 14 - caOS Network Stack: net_proto (pure decoder/builder) . . . 253
17.1.1 14.7 Overview . . . 253
17.1.2 14.8 Decoding pipeline . . . 253
17.1.3 14.9 L2 — Ethernet and VLAN 802.1Q . . . 254
17.1.4 14.10 Proto — ARP . . . 255
17.1.5 14.11 Proto — IPv4 . . . 256
17.1.6 14.12 Proto — UDP . . . 256
17.1.7 14.13 Proto — ICMP . . . 257
17.1.8 14.14 Proto — TCP (wire-format and builder) . . . 257
17.1.9 14.15 Proto — DHCP (pure functions) . . . 258
17.1.10 14.16 Proto — NTP, Syslog, DNS, mDNS (Pure Functions) . . . 259
17.1.11 14.17 Route lookup — `ca_net_resolve_mac` . . . 260

18 caOS Network Stack: Network Services **261**
18.1 ARP Service (`EN_ARP_SERVICE`) . . . 261
18.1.1 Architecture . . . 261
18.1.2 API . . . 261
18.1.3 Parameters `os_config.h` . . . 262

18.1.4 Typical pattern . . . 262
18.2 Route Service (EN_ROUTE_SERVICE) . . . 262
18.2.1 Configuration . . . 263
18.2.2 Public API . . . 263
18.2.3 Integration with DHCP . . . 263
18.2.4 Static configuration (without DHCP) . . . 263
18.2.5 CLI . . . 263
18.3 DHCP Service (EN_DHCP_SERVICE) . . . 264
18.3.1 Configurable parameters . . . 264
18.3.2 State machine . . . 264
18.3.3 Public API . . . 264
18.3.4 Callbacks . . . 265
18.3.5 Adapter dhcp_udp_handler . . . 265
18.3.6 Critical Warnings . . . 265
18.4 NTP Service (EN_NTP_SERVICE) . . . 265
18.4.1 API . . . 265
18.4.2 Parameters os_config.h . . . 266
18.4.3 Typical pattern . . . 266
18.5 Syslog Service (EN_SYSLOG_SERVICE) . . . 266
18.5.1 API . . . 266
18.5.2 Parameters os_config.h . . . 266
18.5.3 Typical pattern . . . 267
18.6 DNS Service (EN_DNS_SERVICE) . . . 267
18.6.1 API . . . 267
18.6.2 os_config.h Parameters . . . 267
18.6.3 Typical Pattern . . . 267
18.7 mDNS Service (EN_MDNS_SERVICE) . . . 267
18.7.1 API . . . 267
18.7.2 Embedded network scenario without router . . . 268
18.8 TCP Service (EN_TCP_SERVICE) . . . 268
18.8.1 Configuration . . . 268
18.8.2 State machine . . . 268
18.8.3 Public API — client . . . 269
18.8.4 Configuration structure — client . . . 269
18.8.5 Public API — server . . . 269
18.8.6 Configuration structure — server . . . 269
18.8.7 CLI . . . 270
18.8.8 Warnings . . . 270
18.9 UDP Server (EN_UDP_SERVER) . . . 270
18.9.1 API . . . 270
18.9.2 Configuration Structure . . . 270
18.10 UDP Client (EN_UDP_CLIENT) . . . 271
18.10.1 API . . . 271
18.11 IP Defrag Service (EN_IP_DEFRAG) . . . 271
18.11.1 Configuration parameters . . . 271
18.11.2 Operational flow . . . 272
18.11.3 Initialization . . . 272
18.11.4 Public API . . . 272
18.12 TFTP Service (EN_TFTP_SERVICE) . . . 272
18.12.1 Configuration parameters . . . 272
18.12.2 Public API . . . 273

18.12.3 `on_block` Callbacks . . . 273
18.12.4 Server side: `ca_tftp_receive` . . . 273
18.12.5 CLI . . . 273
18.13 Packet Tracer Service (`EN_PACKET_TRACER`) . . . 274
18.13.1 Architecture . . . 274
18.13.2 Configuration . . . 274
18.13.3 API . . . 274
18.13.4 CLI . . . 274
18.13.5 Output format . . . 274
18.14 HTTP Service . . . 275
18.14.1 Configuration `os_config.h` . . . 275
18.14.2 Architecture . . . 275
18.14.3 Data structures . . . 276
18.14.4 Public API . . . 277
18.14.5 DHCP Integration . . . 277
18.14.6 Usage patterns . . . 278
18.14.7 CLI Capture Integration . . . 278
18.14.8 Implementation details . . . 278
18.14.9 CLI: `http` command . . . 279
18.14.10 Warnings . . . 279
18.15 MQTT Service . . . 280
18.15.1 Features . . . 280
18.15.2 Configuration `os_config.h` . . . 280
18.15.3 Architecture . . . 280
18.15.4 Public API . . . 281
18.15.5 Initialization . . . 281
18.15.6 Pattern — periodic publish + reconnect . . . 282
18.15.7 CLI: command `mqtt` . . . 283
18.15.8 Warnings . . . 283
18.16 Modbus/TCP Service . . . 283
18.16.1 `os_config.h` Configuration . . . 283
18.16.2 Architecture . . . 283
18.16.3 Initialization . . . 284
18.16.4 Public API . . . 284
18.16.5 Usage patterns . . . 285
18.16.6 CLI: command `modbus` . . . 285
18.16.7 Warnings . . . 286
18.17 iperf Service . . . 286
18.17.1 `os_config.h` Configuration . . . 286
18.17.2 Architettura . . . 286
18.17.3 Initialization . . . 287
18.17.4 Public API . . . 287
18.17.5 CLI: command `iperf` . . . 288
18.17.6 Typical Performance (NUCLEO-H753ZI, 100BASE-TX) . . . 288

19 caOS USB Driver: DWC2 Core **289**
19.1 Low-level USB Driver . . . 289
19.1.1 Overview . . . 289
19.1.2 The Two USB IPs . . . 290
19.1.3 Two transfer modes (DWC2) . . . 290
19.1.4 Per-EP Code (DWC2) . . . 291
19.1.5 Public API . . . 291

19.1.6 Source layout . . . 293
19.1.7 Notes per family . . . 293

20 caOS USB Stack: Management and Classes 295
20.1 USB device stack: management and classes . . . 295
20.1.1 Overview . . . 295
20.1.2 Two-level model (D1) . . . 295
20.1.3 API management (D18 — 5 functions) . . . 295
20.1.4 Class framework — `ca_usb_class_ops_t` . . . 296
20.1.5 Class services . . . 296
20.1.6 Endpoint allocation . . . 297
20.1.7 Composite PID (D14) . . . 298
20.1.8 Device identity (os_config.h) . . . 298
20.1.9 Composite devices . . . 299
20.1.10 Source file . . . 299
20.1.11 Reference Examples . . . 300

21 FatFS on caOS: SDIO Driver 301
21.1 FatFS on caOS: SDIO driver . . . 301
21.1.1 Overview . . . 301
21.1.2 Layer 1 — FatFS and the ffconf.h redirect . . . 301
21.1.3 Layer 2 — glue diskio (`fatfs_diskio.c`) . . . 302
21.1.4 Layer 3 — `ca_sdio` driver . . . 303
21.1.5 Status by family . . . 305
21.1.6 File structure . . . 306

22 Centralized Power Manager 307
22.1 Centralized power manager . . . 307
22.1.1 Overview . . . 307
22.1.2 `pwr info` — power state snapshot . . . 307
22.1.3 Frequency scaling . . . 308
22.1.4 Low-power modes . . . 309
22.1.5 Wake-up sources . . . 310
22.1.6 Status by family . . . 311
22.1.7 Implementation notes . . . 312
22.1.8 Configuration . . . 312
22.1.9 File structure . . . 313

23 Migrating from FreeRTOS, Zephyr and ThreadX to caOS 315
23.1 Migrating from FreeRTOS, Zephyr and ThreadX to caOS . . . 315
23.1.1 Overview . . . 315
23.1.2 1. Application structure . . . 315
23.1.3 2. Tasks . . . 316
23.1.4 3. Time . . . 318
23.1.5 4. Critical sections and scheduler lock . . . 319
23.1.6 5. Synchronization and communication . . . 320
23.1.7 6. Heap . . . 322
23.1.8 7. Semantic differences . . . 323
23.1.9 8. Summary table . . . 323
23.1.10 9. Porting checklist . . . 324
23.1.11 10. From Zephyr to caOS . . . 324
23.1.12 11. From ThreadX to caOS . . . 327

II Practical Examples **331**

24 Hello World: the First Program on caOS **333**
24.1 Example 1 — Hello World: the first program on caOS . . . 333
24.1.1 Introduction . . . 333
24.1.2 The source code . . . 333
24.1.3 Line-by-line analysis . . . 334
24.1.4 What this example teaches us . . . 335
24.1.5 Example output . . . 335

25 Scheduling and Memory Allocation **337**
25.1 Example 2 — Scheduling and memory allocation . . . 337
25.1.1 Introduction . . . 337
25.1.2 What is scheduling? . . . 337
25.1.3 What is dynamic memory allocation? . . . 337
25.1.4 The source code . . . 338
25.1.5 Detailed analysis . . . 338
25.1.6 What this example teaches us . . . 340
25.1.7 Example output . . . 340

26 Mutex and Shared Resource Access **341**
26.1 Example 3 — LED and GPIO CLI . . . 341
26.1.1 Introduction . . . 341
26.1.2 Architecture . . . 342
26.1.3 Source code . . . 342
26.1.4 Detailed analysis . . . 343
26.1.5 Expected output . . . 344
26.1.6 Required configuration in `os_config.h` . . . 344
26.1.7 Summary . . . 344

27 Queues: Inter-Task Communication **345**
27.1 Example 4 — Code Queue: task-to-task communication via queues . . . 345
27.1.1 Introduction . . . 345
27.1.2 What is a queue? . . . 345
27.1.3 The source code . . . 346
27.1.4 Architecture . . . 347
27.1.5 Detailed analysis . . . 348
27.1.6 Typical execution scenario . . . 350
27.1.7 What this example teaches us . . . 351
27.1.8 Example output . . . 351

28 Recursive Mutex: Handling Recursion in Multitasking **353**
28.1 Example 5 — Recursive mutex: managing recursion in multitasking . . . 353
28.1.1 Introduction . . . 353
28.1.2 What is a recursive mutex? . . . 353
28.1.3 The source code . . . 353
28.1.4 Architecture . . . 356
28.1.5 Detailed analysis . . . 356
28.1.6 When to use recursive mutexes . . . 358
28.1.7 Beware of potential issues . . . 358
28.1.8 Important considerations for the embedded world . . . 358
28.1.9 What this example teaches us . . . 359

28.1.10 Example output . . . 359
28.1.11 Verification of waits with timeout (`twait`) . . . 359

29 Semaphore: Controlling Access to Shared Resources 361
29.1 Example 6 — Semaphore: controlling access to shared resources . . . 361
29.1.1 Introduction . . . 361
29.1.2 What is a semaphore? . . . 361
29.1.3 The source code . . . 361
29.1.4 Architecture . . . 366
29.1.5 Detailed analysis . . . 367
29.1.6 When to use semaphores . . . 369
29.1.7 What this example teaches us . . . 369
29.1.8 Example output . . . 369
29.1.9 Verification of waits with timeout (`twait`) . . . 370

30 Condition Variable: Producer-Consumer Coordination 371
30.1 Example 7 — Condition Variable: producer-consumer coordination . . . 371
30.1.1 Introduction . . . 371
30.1.2 What is a condition variable? . . . 371
30.1.3 The source code . . . 372
30.1.4 Architecture . . . 374
30.1.5 Detailed analysis . . . 374
30.1.6 API . . . 377
30.1.7 What this example teaches us . . . 377
30.1.8 Example output . . . 378
30.1.9 Verification of waits with timeout (`twait`) . . . 378

31 Critical Sections: Protecting Data by Disabling Interrupts 381
31.1 Example 8 — Critical sections: data protection with interrupt disabling . . . 381
31.1.1 Introduction . . . 381
31.1.2 What is a critical section? . . . 381
31.1.3 The source code . . . 382
31.1.4 Architecture . . . 383
31.1.5 Detailed analysis . . . 384
31.1.6 API . . . 385
31.1.7 Critical sections vs other primitives . . . 386
31.1.8 Warning: fundamental rules . . . 386
31.1.9 What this example teaches us . . . 386
31.1.10 Example output . . . 386

32 Task Notification: Lightweight Inter-Task Signaling 389
32.1 Example 9 — Task Notification: lightweight signaling between tasks . . . 389
32.1.1 Introduction . . . 389
32.1.2 What is a task notification? . . . 389
32.1.3 The source code . . . 389
32.1.4 Architecture . . . 391
32.1.5 Detailed analysis . . . 392
32.1.6 API . . . 394
32.1.7 Notifications vs other primitives . . . 394
32.1.8 When not to use notifications . . . 394
32.1.9 What this example teaches us . . . 394
32.1.10 Verification of waits with timeout (`twait`) . . . 395

33 Watchdog **397**
33.1 Example 10 — Watchdog Timer (WDT) . . . 397
33.1.1 Introduction . . . 397
33.1.2 What is the Watchdog Timer? . . . 397
33.1.3 Driver API (`wdt.h`) . . . 398
33.1.4 The source code . . . 398
33.1.5 Architecture . . . 399
33.1.6 CLI Commands . . . 399
33.1.7 Expected Output . . . 400
33.1.8 The bench test . . . 401
33.1.9 Differences between Families . . . 401
33.1.10 Best Practices . . . 402
33.1.11 Summary . . . 402

34 CPU Temperature **403**
34.1 Example 11 — Core Temperature Monitor with Watchdog Protection . . . 403
34.1.1 Introduction . . . 403
34.1.2 Thermal Protection Architecture . . . 403
34.1.3 Source Code . . . 404
34.1.4 CLI Commands . . . 406
34.1.5 The digital DTS sensor (H563) . . . 406
34.1.6 Detailed analysis . . . 407
34.1.7 Expected Output . . . 408
34.1.8 Test Variants . . . 409
34.1.9 Threshold Considerations . . . 410
34.1.10 Thermal System Debugging . . . 410
34.1.11 Best Practices . . . 410
34.1.12 Summary . . . 411

35 Extending the System Command Line **413**
35.1 Example 12 — Extending the CLI with user commands . . . 413
35.1.1 Introduction . . . 413
35.1.2 The system CLI . . . 413
35.1.3 Source code . . . 414
35.1.4 Detailed analysis . . . 416
35.1.5 Adding more commands . . . 418
35.1.6 Expected output . . . 418
35.1.7 Required configuration in `os_config.h` . . . 419
35.1.8 Summary . . . 419

36 Execution Time Profiling with DWT CYCCNT **421**
36.1 Example 13 — Profiling and task watchdog . . . 421
36.1.1 Introduction . . . 421
36.1.2 The `measure` module . . . 421
36.1.3 Task watchdog . . . 422
36.1.4 Source code . . . 423
36.1.5 Detailed analysis . . . 424
36.1.6 Expected output . . . 425
36.1.7 Required configuration in `os_config.h` . . . 425
36.1.8 Summary . . . 425

37 Streams **427**
37.1 Example 14 — Stream Buffer with interactive CLI . . . 427
37.1.1 Introduction . . . 427
37.1.2 Example architecture . . . 428
37.1.3 Source code . . . 428
37.1.4 Detailed analysis . . . 432
37.1.5 Expected output . . . 433
37.1.6 Stream vs Message Buffer . . . 434
37.1.7 Required configuration in `os_config.h` . . . 434
37.1.8 Summary . . . 435
37.1.9 Verification of waits with timeout (`twait`) . . . 435

38 Messages **437**
38.1 Example 15 — Message Buffer with heap allocation . . . 437
38.1.1 Introduction . . . 437
38.1.2 Example architecture . . . 438
38.1.3 Source code . . . 438
38.1.4 Detailed analysis . . . 442
38.1.5 Expected output . . . 444
38.1.6 Stream vs Message Buffer . . . 445
38.1.7 Required configuration in `os_config.h` . . . 446
38.1.8 Heap memory management . . . 446
38.1.9 Summary . . . 446
38.1.10 Verification of waits with timeout (`twait`) . . . 447

39 Thread Pool with RNG and SHA-256 **449**
39.1 Example 16 — Thread Pool with RNG and SHA-256 . . . 449
39.1.1 Introduction . . . 449
39.1.2 Example Architecture . . . 452
39.1.3 Source Code . . . 453
39.1.4 Detailed analysis . . . 455
39.1.5 Expected output . . . 457
39.1.6 Required configuration in `os_config.h` . . . 458
39.1.7 Created threads . . . 458
39.1.8 Summary . . . 459

40 Event Group: Synchronization on Bit Patterns **461**
40.1 Example 17 — Event Group: bit pattern synchronization . . . 461
40.1.1 Introduction . . . 461
40.1.2 Example architecture . . . 462
40.1.3 Source code . . . 462
40.1.4 Detailed analysis . . . 464
40.1.5 Expected output . . . 465
40.1.6 Required configuration in `os_config.h` . . . 466
40.1.7 Created threads . . . 466
40.1.8 Summary . . . 466
40.1.9 Verification of waits with timeout (`twait`) . . . 467

41 SW Timer: One-Shot and Periodic **469**
41.1 Example 18 — SW Timer: one-shot and periodic . . . 469
41.1.1 Introduction . . . 469
41.1.2 Example architecture . . . 470
41.1.3 Source code . . . 470
41.1.4 Detailed analysis . . . 472
41.1.5 Expected output . . . 473
41.1.6 Created tasks . . . 475
41.1.7 Configuration required in `os_config.h` . . . 475
41.1.8 Summary . . . 475

42 RW Lock and Barrier **477**
42.1 Example 19 — RW Lock and Barrier . . . 477
42.1.1 Introduction . . . 477
42.1.2 Example Architecture . . . 478
42.1.3 Source Code . . . 478
42.1.4 Detailed analysis . . . 480
42.1.5 Expected output . . . 482
42.1.6 Created threads . . . 482
42.1.7 Summary . . . 483

43 Block Pool, sleep_until and Timed Primitives **485**
43.1 Example 20 — Block Pool, sleep_until and Timed Primitives . . . 485
43.1.1 Introduction . . . 485
43.1.2 Example Architecture . . . 486
43.1.3 Source Code . . . 486
43.1.4 Detailed analysis . . . 488
43.1.5 Expected output . . . 490
43.1.6 Created threads . . . 491
43.1.7 Required configuration in `os_config.h` . . . 491
43.1.8 Summary . . . 491
43.1.9 Verification of waits with timeout (`twait`) . . . 492
43.1.10 Waiter removed while waiting (`twkill`) . . . 492

44 Multi-Instance UART with Binary Protocol **493**
44.1 Example 21 — Multi-instance UART with binary protocol . . . 493
44.1.1 Introduction . . . 493
44.1.2 Required Hardware . . . 494
44.1.3 The binary protocol . . . 494
44.1.4 Example architecture . . . 496
44.1.5 The source code . . . 497
44.1.6 Detailed analysis . . . 499
44.1.7 Expected output . . . 500
44.1.8 Required configuration in `os_config.h` . . . 501
44.1.9 Bridge VCP UART . . . 501
44.1.10 Created Threads . . . 502
44.1.11 Summary . . . 503

45 I2C Master/Slave Intra-MCU **505**
45.1 Example 22 — Intra-MCU I2C master/slave . . . 505
45.1.1 Introduction . . . 505
45.1.2 Required hardware . . . 506
45.1.3 The protocol . . . 507
45.1.4 Example Architecture . . . 508
45.1.5 The source code . . . 509
45.1.6 Detailed analysis . . . 511
45.1.7 Expected output . . . 513
45.1.8 Required configuration in `os_config.h` . . . 514
45.1.9 Created threads . . . 514
45.1.10 Summary . . . 515

46 Internal Flash: Program / Verify / WDT Reset **517**
46.1 Example 23 — Internal Flash: CLI integrated into the driver . . . 517
46.1.1 Introduction . . . 517
46.1.2 Required hardware . . . 518
46.1.3 Sector map by family . . . 518
46.1.4 Flash alignment constraints . . . 518
46.1.5 Example architecture . . . 519
46.1.6 Flash IRQ driver architecture . . . 520
46.1.7 Source code . . . 520
46.1.8 Command analysis . . . 521
46.1.9 Expected output . . . 522
46.1.10 Required configuration in `os_config.h` . . . 523
46.1.11 Created threads . . . 523
46.1.12 Summary . . . 523

47 Timer: Periodic + Counter **525**
47.1 Example 24 — Timer IRQ: 10 kHz generator + external countdown counter . . . 525
47.1.1 Introduction . . . 525
47.1.2 Required Hardware . . . 526
47.1.3 Timer configuration . . . 526
47.1.4 System architecture . . . 528
47.1.5 Source code . . . 529
47.1.6 Detailed analysis . . . 530
47.1.7 Expected output . . . 531
47.1.8 Required configuration in `os_config.h` . . . 532
47.1.9 Created threads . . . 532
47.1.10 Possible extensions . . . 532
47.1.11 Summary . . . 533

48 Timer: Periodic + Encoder Counter **535**
48.1 Example 25 — Quadrature Signal Generator + Timer Encoder . . . 535
48.1.1 Introduction . . . 535
48.1.2 System Architecture . . . 537
48.1.3 Source Code . . . 537
48.1.4 Detailed Analysis . . . 543
48.1.5 Expected output . . . 544
48.1.6 Required Configuration in `os_config.h` . . . 544
48.1.7 Important Notes . . . 544
48.1.8 Summary . . . 545

49 Cryptography: CRC32, HASH, AES, RNG and Key Derivation 547
49.1 Example 26 — Cryptography: CRC32, HASH, AES, RNG and Key Derivation . 547
49.1.1 Introduction . . . 547
49.1.2 System architecture . . . 549
49.1.3 Source code . . . 549
49.1.4 Detailed analysis . . . 550
49.1.5 Expected output . . . 558
49.1.6 Required configuration in `os_config.h` . . . 559
49.1.7 Important notes . . . 559
49.1.8 Summary . . . 561

50 DAC Output and ADC Readback (Polling and ISR) 563
50.1 Example 27 — DAC output and ADC readback (polling and ISR) . . . 563
50.1.1 Introduction . . . 563
50.1.2 System architecture . . . 565
50.1.3 Source code . . . 565
50.1.4 Detailed analysis . . . 566
50.1.5 Expected output . . . 568
50.1.6 ADC driver architecture — ISR mode . . . 568
50.1.7 Required configuration in `os_config.h` . . . 570
50.1.8 Technical notes . . . 570
50.1.9 Summary . . . 572

51 Real-Time Clock (Polling and ISR) 573
51.1 Example 28 — RTC: setting, reading, and alarms with callbacks . . . 573
51.1.1 Introduction . . . 573
51.1.2 System architecture . . . 575
51.1.3 Source code . . . 575
51.1.4 Detailed analysis . . . 576
51.1.5 Tasks and priorities . . . 580
51.1.6 Expected output . . . 580
51.1.7 RTC driver architecture . . . 581
51.1.8 Technical notes . . . 583
51.1.9 Required configuration in `os_config.h` . . . 584
51.1.10 Summary . . . 585

52 SPI Master and Slave (Polling and ISR) 587
52.1 Example 29 — SPI master-slave on the same MCU . . . 587
52.1.1 Introduction . . . 587
52.1.2 Source configuration . . . 587
52.1.3 System architecture . . . 588
52.1.4 Source code . . . 588
52.1.5 Technical notes — MODF on STM32H7 SPI v3 . . . 591
52.1.6 Expected output . . . 592
52.1.7 CLI Commands . . . 592
52.1.8 Tasks and priorities . . . 593
52.1.9 SPI APIs used . . . 593
52.1.10 Required configuration in `os_config.h` . . . 593
52.1.11 Summary . . . 593

53 EXTI: Interrupt from the User Button **595**
53.1 Example 30 — EXTI: interrupt from user button . . . 595
53.1.1 Introduction . . . 595
53.1.2 Required configuration in `os_config.h` . . . 595
53.1.3 System architecture . . . 596
53.1.4 Software debounce . . . 596
53.1.5 Source code . . . 597
53.1.6 Technical notes . . . 600
53.1.7 Expected output . . . 601
53.1.8 Tamper on the same button (`EN_TAMP`) . . . 602
53.1.9 Tasks and priorities . . . 602
53.1.10 EXTI API used . . . 602
53.1.11 Summary . . . 603

54 DMA: Memory-to-Memory and Memory-to-Peripheral (DAC1) **605**
54.1 Example 31 — DMA: memory-to-memory and memory-to-peripheral (DAC1) . . . 605
54.1.1 Introduction . . . 605
54.1.2 Configuration required in `os_config.h` . . . 605
54.1.3 CLI Commands . . . 606
54.1.4 System architecture . . . 606
54.1.5 Source code . . . 607
54.1.6 Technical notes . . . 609
54.1.7 Expected output . . . 610
54.1.8 Tasks and priorities . . . 610
54.1.9 DMA APIs used . . . 610
54.1.10 DAC APIs used . . . 611
54.1.11 Summary . . . 611

55 ETH: Link UP / Link DOWN Detection **613**
55.1 Example 32 — ETH: link UP/DOWN, packet reception and transmission . . . 613
55.1.1 Introduction . . . 613
55.1.2 Required configuration in `os_config.h` . . . 614
55.1.3 System architecture . . . 614
55.1.4 The source code . . . 619
55.1.5 Technical notes . . . 623
55.1.6 Expected output . . . 625
55.1.7 Tasks and priorities . . . 627
55.1.8 ETH APIs used . . . 627
55.1.9 MCU family support . . . 628
55.1.10 Summary . . . 628

56 caOS Net Stack: Buffer Pool, ARP Reply, ICMP Echo **629**
56.1 Example 33 — caOS Net Stack: buffer pool, ARP reply, ICMP echo, UDP . . . 629
56.1.1 Introduction . . . 629
56.1.2 System architecture . . . 630
56.1.3 Source code . . . 633
56.1.4 Expected output . . . 636
56.1.5 Technical notes . . . 638

57 Multi-Client UDP Server on Port 7961 **641**
57.1 Example 34 — Multi-client UDP Server on port 7961 . . . 641
57.1.1 Introduction . . . 641
57.1.2 System architecture . . . 642
57.1.3 Source code . . . 644
57.1.4 Multi-client model — UDP Server API . . . 647
57.1.5 Expected output . . . 647
57.1.6 Technical notes . . . 648

58 DHCP Client + ARP + ICMP Ping + UDP Server with 2 Clients **651**
58.1 Example 35 — DHCP client + ARP + ICMP ping + UDP server 2 client . . . 651
58.1.1 Introduction . . . 651
58.1.2 System architecture . . . 652
58.1.3 Source code . . . 654
58.1.4 Test procedure from PC . . . 657
58.1.5 Expected output . . . 658
58.1.6 Technical notes . . . 659

59 DHCP + NTP + ARP + ICMP Ping + UDP Server **661**
59.1 Example 36 — DHCP + NTP + ARP + ICMP ping + UDP server . . . 661
59.1.1 Introduction . . . 661
59.1.2 System architecture . . . 662
59.1.3 The source code . . . 663
59.1.4 Test procedure from PC . . . 664
59.1.5 Expected output . . . 665
59.1.6 Technical notes . . . 665

60 DHCP + NTP + DNS + ARP + ICMP Ping + UDP Server **667**
60.1 Example 37 — DHCP + NTP + DNS + ARP + ICMP ping + UDP server . . . 667
60.1.1 Introduction . . . 667
60.1.2 System Architecture . . . 668
60.1.3 Source code . . . 669
60.1.4 Testing procedure from PC . . . 670
60.1.5 Expected Output . . . 671
60.1.6 Technical Notes . . . 671

61 DHCP + mDNS Responder + ARP + ICMP Ping + UDP Server **673**
61.1 Example 38 — DHCP + mDNS responder + ARP + ICMP ping + UDP server 673
61.1.1 Introduction . . . 673
61.1.2 System Architecture . . . 674
61.1.3 Source code . . . 675
61.1.4 PC Test Procedure . . . 676
61.1.5 Expected Output . . . 677
61.1.6 Technical Notes . . . 678

62 UDP Syslog Client (RFC 3164) **679**
62.1 Example 39 — DHCP + Syslog Service . . . 679
62.1.1 Introduction . . . 679
62.1.2 System Architecture . . . 680
62.1.3 The source code . . . 682
62.1.4 Test procedure from PC . . . 684
62.1.5 Expected output (verified on NUCLEO-H753ZI) . . . 685

62.1.6 Technical Notes 686

63 TCP Client HTTP GET **687**
63.1 Example 40 — TCP Client HTTP GET 687
63.1.1 Introduction 687
63.1.2 Hardware 687
63.1.3 os_config.h Configuration 688
63.1.4 Architettura 688
63.1.5 Code — key points 691
63.1.6 Test from PC (verified on NUCLEO-H753ZI) 692
63.1.7 Technical notes 693

64 TCP Server (Echo Server, RFC 862) **695**
64.1 Example 41 — TCP Server (echo server RFC 862) 695
64.1.1 Introduction 695
64.1.2 Hardware 695
64.1.3 Configuration os_config.h 695
64.1.4 Architecture 696
64.1.5 Code — key points 699
64.1.6 Test from PC (verified on NUCLEO-H753ZI) 700
64.1.7 Technical notes 701

65 MQTT Client **703**
65.1 Example 42 — MQTT Client 703
65.1.1 Introduction 703
65.1.2 Hardware 703
65.1.3 os_config.h Configuration 704
65.1.4 Architecture 704
65.1.5 Code — key points 705
65.1.6 Test from PC 707
65.1.7 Technical Notes 708
65.1.8 Fixes implemented during HW testing 709

66 Modbus TCP Client **711**
66.1 Example 43 — Modbus TCP Client 711
66.1.1 Introduction 711
66.1.2 Hardware 711
66.1.3 os_config.h Configuration 712
66.1.4 Architecture 712
66.1.5 task_modbus flow 713
66.1.6 Interactive CLI 713
66.1.7 Test PC — modbus_server.py 714
66.1.8 Expected UART Output 715
66.1.9 Technical Notes 715
66.1.10 TCP CLI command sequence during testing 716

67 iperf TCP Client/Server **717**
67.1 Example 44 — iperf TCP client/server 717
67.1.1 Introduction 717
67.1.2 Hardware 717
67.1.3 os_config.h Configuration 718
67.1.4 Architecture 718

67.1.5 iperf client and server flow . . . 719
67.1.6 Interactive CLI . . . 720
67.1.7 Test PC — iperf_peer.py . . . 720
67.1.8 Expected UART Output . . . 720
67.1.9 Measured Performance . . . 721
67.1.10 Memory . . . 722
67.1.11 Technical Notes . . . 722

68 HTTP Server with CLI over HTTP 723
68.1 Example 45 — HTTP and HTTPS Server with CLI via HTTP . . . 723
68.1.1 Introduction . . . 723
68.1.2 Hardware . . . 723
68.1.3 os_config.h configuration . . . 724
68.1.4 Architecture . . . 725
68.1.5 Route Table . . . 727
68.1.6 PC Tests . . . 727
68.1.7 Parsing HTTP POST — “from the bottom” body . . . 728
68.1.8 `ca_cli_exec_capture` flow . . . 729
68.1.9 Content-Length in the HTTP response . . . 731
68.1.10 Packet Tracer . . . 731
68.1.11 Expected UART output . . . 731
68.1.12 Technical notes . . . 732
68.1.13 Bench verification . . . 732

69 AES Channel Service: Encrypted Ping-Pong 733
69.1 Example 46 — Secure pipe: encrypted ping-pong between two tasks . . . 733
69.1.1 Introduction . . . 733
69.1.2 Hardware . . . 733
69.1.3 os_config.h Configuration . . . 734
69.1.4 Architecture . . . 734
69.1.5 Wire format GCM . . . 734
69.1.6 Keys Store — MPU protection . . . 735
69.1.7 Heap management — mandatory wipe . . . 736
69.1.8 Task_alice flow (100 iterations) . . . 736
69.1.9 Task_bob flow (infinite loop) . . . 736
69.1.10 Output UART . . . 736
69.1.11 Task and priority . . . 736
69.1.12 Tamper detection GCM . . . 737
69.1.13 Implementation notes — CRYP H7 and partial blocks GCM . . . 737

70 Keystore Service: makekey + Encrypted Ping-Pong 739
70.1 Example 47 — Keystore service: makekey + ping-pong encrypted . . . 739
70.1.1 Introduction . . . 739
70.1.2 Hardware . . . 739
70.1.3 os_config.h Configuration . . . 740
70.1.4 Architecture . . . 741
70.1.5 Phase 1 — makekey . . . 742
70.1.6 Phase 2 — start . . . 742
70.1.7 Encrypted ping-pong (same as example 46) . . . 743
70.1.8 CLI — state machine . . . 743
70.1.9 UART Output . . . 743
70.1.10 Security — notes . . . 744

71 IP Defrag Service: DHCP + UDP Server + Fragments 745
71.1 Example 48 — IP defrag service: DHCP + UDP server + fragment reassembly . 745
71.1.1 Introduction . . . 745
71.1.2 Hardware . . . 745
71.1.3 os_config.h Configuration . . . 746
71.1.4 Architecture . . . 746
71.1.5 ip_defrag_service API . . . 746
71.1.6 Internal structure — session . . . 747
71.1.7 Tasks and priorities . . . 747
71.1.8 PC Tests . . . 747
71.1.9 Implementation notes . . . 748

72 TFTP Client: GET and PUT over UDP (RFC 1350) 749
72.1 Example 49 — TFTP Client (RFC 1350) . . . 749
72.1.1 Description . . . 749
72.1.2 TFTP Protocol (RFC 1350) . . . 749
72.1.3 Architecture `tftp_service` . . . 750
72.1.4 API . . . 751
72.1.5 `ca_tftp_cfg_t` Configuration . . . 751
72.1.6 Parameters `os_config.h` . . . 751
72.1.7 CLI Commands . . . 752
72.1.8 PC Testing . . . 752
72.1.9 Application flow (example 49) . . . 754
72.1.10 File Structure . . . 755

73 USB Vendor Class: Multi-EP Bulk Loopback 757
73.1 Example 50 — USB Vendor-Class Bulk Loopback . . . 757
73.1.1 Description . . . 757
73.1.2 Example Architecture . . . 757
73.1.3 Descriptors . . . 758
73.1.4 SETUP handler (EP0) . . . 758
73.1.5 CLI `usb` . . . 759
73.1.6 Parameters (per-MCU, in `ca_os_ex50.c`) . . . 759
73.1.7 Test from PC . . . 759
73.1.8 File structure . . . 760
73.1.9 F407 Notes — slave mode driver . . . 760

74 USB Management Stack: Custom Class 761
74.1 Example 51 — USB Management Stack: custom class . . . 761
74.1.1 Description . . . 761
74.1.2 Hardware . . . 761
74.1.3 os_config.h Configuration . . . 762
74.1.4 Architecture . . . 762
74.1.5 Class config (D20: persistent lifetime) . . . 763
74.1.6 CLI `usb` . . . 764
74.1.7 Mount/unmount state . . . 764
74.1.8 Parameters . . . 764
74.1.9 Expected UART output . . . 764
74.1.10 Test from PC . . . 764
74.1.11 File Structure . . . 765

75 USB CDC-ACM: Virtual Serial Port **767**
75.1 Example 52 — USB CDC-ACM: virtual serial port . . . 767
75.1.1 Description . . . 767
75.1.2 Hardware . . . 767
75.1.3 os_config.h Configuration . . . 768
75.1.4 CDC ACM Structure . . . 768
75.1.5 SETUP class-level . . . 768
75.1.6 Architecture . . . 769
75.1.7 Application API . . . 770
75.1.8 Parameters . . . 770
75.1.9 Expected UART output . . . 770
75.1.10 Test from PC . . . 770
75.1.11 File Structure . . . 771

76 USB MSC: Mass Storage Device (Ramdisk) **773**
76.1 Example 53 — USB MSC: Mass Storage Device (ramdisk) . . . 773
76.1.1 Introduction . . . 773
76.1.2 Hardware . . . 774
76.1.3 os_config.h Configuration . . . 774
76.1.4 Architecture . . . 775
76.1.5 Back-end block device . . . 775
76.1.6 Bulk-Only Transport + SCSI . . . 776
76.1.7 Parameters . . . 776
76.1.8 Expected UART Output . . . 777
76.1.9 Test from PC . . . 777
76.1.10 Technical Notes . . . 778
76.1.11 File structure . . . 778

77 USB NCM: USB Ethernet (L2 Loopback) **781**
77.1 Example 54 — USB NCM: USB Ethernet (L2 loopback) . . . 781
77.1.1 Introduction . . . 781
77.1.2 PC Prerequisite: Remove the interface from NetworkManager . . . 782
77.1.3 Hardware . . . 782
77.1.4 os_config.h Configuration . . . 782
77.1.5 Architecture . . . 783
77.1.6 Loopback L2 with SADA swap . . . 784
77.1.7 Wire NTB (1 datagram per block) . . . 784
77.1.8 Parameters . . . 785
77.1.9 Expected UART Output . . . 785
77.1.10 Test from PC . . . 785
77.1.11 Technical notes . . . 786
77.1.12 File structure . . . 786

78 Net Stack over USB-NCM: Real Ping **787**
78.1 Example 55 — caOS Net Stack on USB-NCM (real ping) . . . 787
78.1.1 Introduction . . . 787
78.1.2 PC Prerequisite: Remove the interface from NetworkManager . . . 787
78.1.3 Hardware . . . 788
78.1.4 os_config.h Configuration . . . 788
78.1.5 Architecture . . . 789
78.1.6 Interface Abstraction: ETH vs NCM . . . 790
78.1.7 Expected UART Output . . . 790

78.1.8 Test from PC . . . 790
78.1.9 Technical notes . . . 791
78.1.10 File structure . . . 791

79 Composite USB: CDC + MSC + NCM 793
79.1 Example 56 — Composite USB CDC + MSC + NCM . . . 793
79.1.1 Introduction . . . 793
79.1.2 PC Prerequisite: Remove the interface from NetworkManager . . . 794
79.1.3 Hardware . . . 794
79.1.4 os_config.h Configuration . . . 795
79.1.5 Endpoint allocation (maximum case) . . . 795
79.1.6 Architecture . . . 796
79.1.7 The three application functions . . . 797
79.1.8 Expected UART Output . . . 797
79.1.9 Test from PC . . . 797
79.1.10 Technical notes . . . 798
79.1.11 File structure . . . 798

80 USB DFU: Runtime Mode 799
80.1 Example 57 — USB Management Stack: DFU runtime . . . 799
80.1.1 Description . . . 799
80.1.2 Hardware . . . 799
80.1.3 os_config.h Configuration . . . 800
80.1.4 Architecture . . . 800
80.1.5 Class configuration (D20: persistent lifetime) . . . 801
80.1.6 Handled DFU Requests (DFU 1.1) . . . 802
80.1.7 CLI `usb` . . . 802
80.1.8 Parameters . . . 802
80.1.9 Expected UART Output . . . 802
80.1.10 Test from PC . . . 803
80.1.11 File structure . . . 803

81 FatFS on SD Card: SDIO Driver with DMA 805
81.1 Example 58 — FatFS on SD card: SDIO driver with DMA . . . 805
81.1.1 Description . . . 805
81.1.2 Hardware . . . 806
81.1.3 os_config.h Configuration . . . 806
81.1.4 Architecture . . . 807
81.1.5 stdout log to file . . . 807
81.1.6 `fat cat` — the trip through the heap . . . 809
81.1.7 Expected output (CLI, HW F4VE) . . . 809
81.1.8 Fault injection: `inject` . . . 810
81.1.9 State per family HW . . . 810
81.1.10 File structure . . . 811

82 Power Manager: Frequency, Stop and Standby 813
82.1 Example 59 — Power manager (pwr) . . . 813
82.1.1 What it demonstrates . . . 813
82.1.2 Requirements . . . 814
82.1.3 Structure . . . 814
82.1.4 Notes . . . 814
82.1.5 HW Status . . . 814

82.1.6 File structure . . . 815

83 PWM: LED Fade with User Button 817
83.1 Example 60 — PWM: LED fade with user button . . . 817
83.1.1 Behavior . . . 817
83.1.2 Thresholds . . . 817
83.1.3 Board mapping . . . 817
83.1.4 Key points of the code . . . 818
83.1.5 Test from CLI . . . 818
83.1.6 Requirements . . . 818

84 LPTIM: Low-Power Blink 819
84.1 Example 61 — LPTIM: blink low-power (periodic + one-shot) . . . 819
84.1.1 Behavior . . . 819
84.1.2 LPTIM Driver . . . 819
84.1.3 Board mapping . . . 820
84.1.4 Key points of the code . . . 820
84.1.5 Test from CLI . . . 820
84.1.6 Requirements . . . 820

85 LPUART: Low-Power Console with Wake from Stop 821
85.1 Example 62 — LPUART: low-power console with wake-up from Stop . . . 821
85.1.1 Configuration (os_config.h) . . . 821
85.1.2 Behavior . . . 821
85.1.3 LPUART Driver . . . 821
85.1.4 Board mapping (external USB-UART adapter, 3.3V) . . . 822
85.1.5 Test without adapter: ST-LINK VCP via jumpers . . . 822
85.1.6 Requirements . . . 822

86 UART DMA: Variable-Length Protocol 823
86.1 Example 63 — UART DMA: variable-length protocol . . . 823
86.1.1 Configuration (os_config.h) . . . 823
86.1.2 The example protocol . . . 823
86.1.3 Header-to-payload flow . . . 823
86.1.4 Driver API (EN_UART_DMA) . . . 824
86.1.5 Host test . . . 824

87 SPI DMA: Full-Duplex Loopback on a Single Board 825
87.1 Example 64 — SPI DMA: loopback full-duplex on single board . . . 825
87.1.1 Configuration (os_config.h) . . . 825
87.1.2 Hardware — 2-wire loopback . . . 825
87.1.3 The synchronous transfer . . . 826
87.1.4 Driver API (EN_SPI_DMA) . . . 826
87.1.5 Behavior . . . 827
87.1.6 DMA Portability by Family . . . 827
87.1.7 SPI v3 Erratum — CRCSIZE (H5/H7/U5) . . . 828
87.1.8 Test Status (Hardware) . . . 828

88 SPI DMA: Master + Slave IRQ on Two Boards 829
88.1 Example 65 — SPI DMA master (with the slave from example 93) . . . 829
88.1.1 Connections: same position on both boards . . . 829
88.1.2 Hardware — reference pair H563 + H753 . . . 829
88.1.3 Hardware — alternative pair U575 + F407 . . . 830

88.1.4 Why two examples (master 65, slave 93) . . . 830
88.1.5 Flow . . . 830
88.1.6 Speed: why the default is DIV256 . . . 831
88.1.7 Why master DMA + slave IRQ . . . 832
88.1.8 Notes per family . . . 832
88.1.9 Test status (hardware) . . . 832

89 Crypto Self-Test: SHA-256, SHA-512, X25519, Ed25519 833
89.1 Example 66 — Cryptographic self-test (SHA-512, X25519, Ed25519, P-256) . . . 833
89.1.1 Purpose . . . 833
89.1.2 Configuration . . . 833
89.1.3 Behavior . . . 834
89.1.4 CLI . . . 835
89.1.5 The tests that matter . . . 836
89.1.6 Note on constant time . . . 836
89.1.7 An instructive bug: `fe25519_sq2` . . . 836
89.1.8 API . . . 836
89.1.9 Implementation notes . . . 837
89.1.10 P-256: MCU timings (10/23/2026) . . . 837
89.1.11 Next step . . . 837

90 SSH Server: Transport, Ed25519 Userauth and Shell 839
90.1 Example 67 — SSH Server: transport, userauth Ed25519 and interactive shell . . . 839
90.1.1 Introduction . . . 839
90.1.2 Hardware and families . . . 840
90.1.3 Handshake flow . . . 841
90.1.4 The shell: channel CLI . . . 842
90.1.5 Keys: volatile host key and authorized key . . . 843
90.1.6 Required configuration in `os_config.h` . . . 843
90.1.7 CLI . . . 843
90.1.8 Linking procedure . . . 844
90.1.9 Expected output . . . 844
90.1.10 Lesson: crypto HW always known-answer . . . 844
90.1.11 Summary . . . 845

91 Fault Test Bench: MPU, Bus, Usage and HardFault 847
91.1 Example 68 — Fault Test Bench: MPU, Bus, Usage and HardFault . . . 847
91.1.1 Introduction . . . 847
91.1.2 Hardware and Families . . . 847
91.1.3 The 19 Cases, by Fault Category . . . 848
91.1.4 How it runs . . . 851
91.1.5 What the dump shows . . . 852
91.1.6 Actual output (STM32H753, Cortex-M7 @ 480 MHz) . . . 852
91.1.7 Comparison: the same bank on STM32H563 (Cortex-M33 @ 250 MHz) . . . 854
91.1.8 Summary . . . 857

92 UDP Server over USB-NCM 859
92.1 Example 69 — UDP Server on USB-NCM . . . 859
92.1.1 Introduction . . . 859
92.1.2 PC Prerequisite: Remove the interface from NetworkManager . . . 860
92.1.3 Hardware . . . 860
92.1.4 os_config.h Configuration . . . 860

92.1.5 Architecture . . . 861
92.1.6 Expected UART output . . . 862
92.1.7 Test from PC . . . 862
92.1.8 Technical Notes . . . 862
92.1.9 File Structure . . . 863

93 DHCP Client over USB-NCM 865
93.1 Example 70 — DHCP Client over USB-NCM . . . 865
93.1.1 Introduction . . . 865
93.1.2 PC Prerequisite: detach the interface from NetworkManager . . . 865
93.1.3 The critical point: there is no DHCP server . . . 866
93.1.4 Hardware . . . 866
93.1.5 os_config.h configuration . . . 866
93.1.6 Startup sequence . . . 867
93.1.7 Expected UART output . . . 868
93.1.8 Technical notes . . . 868
93.1.9 File structure . . . 868

94 DHCP and NTP Client over USB-NCM 869
94.1 Example 71 — DHCP Client + NTP over USB-NCM . . . 869
94.1.1 Introduction . . . 869
94.1.2 PC Prerequisite: Remove the interface from NetworkManager . . . 869
94.1.3 The critical point: the PC must act as a server twice . . . 870
94.1.4 Hardware . . . 870
94.1.5 os_config.h configuration . . . 871
94.1.6 Boot Sequence . . . 871
94.1.7 Expected UART output . . . 872
94.1.8 Technical notes . . . 872
94.1.9 File structure . . . 873

95 DHCP, NTP and DNS Client over USB-NCM 875
95.1 Example 72 — DHCP, NTP and DNS client over USB-NCM . . . 875
95.1.1 Introduction . . . 875
95.1.2 PC Prerequisite: detach the interface from NetworkManager . . . 875
95.1.3 The critical point: the PC must act as a server three times . . . 876
95.1.4 Hardware . . . 876
95.1.5 os_config.h configuration . . . 877
95.1.6 Boot sequence . . . 877
95.1.7 Expected UART output . . . 878
95.1.8 Technical notes . . . 878
95.1.9 File structure . . . 879

96 mDNS Responder over USB-NCM 881
96.1 Example 73 — mDNS Responder over USB-NCM . . . 881
96.1.1 Introduction . . . 881
96.1.2 PC Prerequisite: Remove the interface from NetworkManager . . . 881
96.1.3 mDNS does not need a server . . . 882
96.1.4 Hardware . . . 882
96.1.5 os_config.h Configuration . . . 882
96.1.6 Boot Sequence . . . 883
96.1.7 Note: `task_stats` without `held` . . . 884
96.1.8 Expected UART Output . . . 884

96.1.9 Technical notes . . . 884
96.1.10 File structure . . . 885

97 Syslog Service over USB-NCM 887
97.1 Example 74 — Syslog service over USB-NCM . . . 887
97.1.1 Introduction . . . 887
97.1.2 PC Prerequisite: Remove the interface from NetworkManager . . . 887
97.1.3 The first “link UP” waits in queue, and arrives together with the bound . 888
97.1.4 The PC acts as syslog server and as DHCP server . . . 889
97.1.5 Hardware . . . 889
97.1.6 os_config.h Configuration . . . 889
97.1.7 Boot Sequence . . . 890
97.1.8 Expected output . . . 890
97.1.9 Technical notes . . . 891
97.1.10 File structure . . . 891

98 TCP Client over USB-NCM 893
98.1 Example 75 — TCP Client (HTTP GET) over USB-NCM . . . 893
98.1.1 Introduction . . . 893
98.1.2 PC Prerequisite: detach the interface from NetworkManager . . . 893
98.1.3 What changes compared to UDP clones . . . 894
98.1.4 The PC acts as a DHCP server and a TCP server . . . 894
98.1.5 Hardware . . . 894
98.1.6 os_config.h configuration . . . 895
98.1.7 Startup sequence . . . 895
98.1.8 Expected output . . . 896
98.1.9 Technical notes . . . 896
98.1.10 File structure . . . 897

99 TCP Echo Server over USB-NCM 899
99.1 Example 76 — TCP echo server over USB-NCM . . . 899
99.1.1 Introduction . . . 899
99.1.2 PC prerequisite: remove the interface from NetworkManager . . . 899
99.1.3 The real difference: where `ca_tcp_listen()` is located . . . 900
99.1.4 The PC is the client . . . 901
99.1.5 Hardware . . . 901
99.1.6 os_config.h configuration . . . 901
99.1.7 Boot Sequence . . . 901
99.1.8 Expected output . . . 902
99.1.9 Technical Notes . . . 902
99.1.10 File structure . . . 903

100 MQTT Client over USB-NCM 905
100.1 Example 77 — MQTT Client over USB-NCM . . . 905
100.1.1 Introduction . . . 905
100.1.2 PC Prerequisite: Remove the interface from NetworkManager . . . 905
100.1.3 The broker is the gateway . . . 906
100.1.4 The test that this example carries with it . . . 906
100.1.5 Hardware . . . 906
100.1.6 os_config.h Configuration . . . 907
100.1.7 The PC: MQTT broker + DHCP . . . 908
100.1.8 Boot Sequence . . . 908

100.1.9 Expected output . . . 909
100.1.10 Technical notes . . . 909
100.1.11 File structure . . . 910

101 Modbus TCP Client over USB-NCM 911
101.1 Example 78 — Modbus TCP Client over USB-NCM . . . 911
101.1.1 Introduction . . . 911
101.1.2 PC Prerequisite: Remove the interface from NetworkManager . . . 911
101.1.3 The server is the gateway . . . 912
101.1.4 The confirmation on the first connect . . . 912
101.1.5 Hardware . . . 912
101.1.6 os_config.h Configuration . . . 913
101.1.7 The PC: Modbus server + DHCP . . . 913
101.1.8 Boot Sequence . . . 913
101.1.9 Expected output . . . 914
101.1.10 Technical Notes . . . 915
101.1.11 File structure . . . 915

102 iperf TCP over USB-NCM 917
102.1 Example 79 — iperf TCP over USB-NCM . . . 917
102.1.1 Introduction . . . 917
102.1.2 PC Prerequisite: Remove the interface from NetworkManager . . . 917
102.1.3 The numbers: what to expect and what not to believe . . . 918
102.1.4 Priority: `net` paired with the service . . . 919
102.1.5 Hardware . . . 919
102.1.6 os_config.h Configuration . . . 919
102.1.7 Usage . . . 920
102.1.8 Boot sequence . . . 921
102.1.9 Expected output . . . 921
102.1.10 Technical notes . . . 922
102.1.11 File structure . . . 922

103 HTTP Server with CLI over USB-NCM 923
103.1 Example 80 — HTTP and HTTPS server with CLI via HTTP over USB-NCM . 923
103.1.1 Introduction . . . 923
103.1.2 PC Prerequisite: detach the interface from NetworkManager . . . 923
103.1.3 The three differences compared to ex45 . . . 924
103.1.4 Test on PC . . . 924
103.1.5 Unlimited CLI output: chunked response . . . 925
103.1.6 HTTPS on port 443 . . . 925
103.1.7 Status . . . 926

104 IP Defragmentation and UDP Server over USB-NCM 927
104.1 Example 81 — IP Defragmentation + UDP server on USB-NCM . . . 927
104.1.1 Introduction . . . 927
104.1.2 PC Prerequisite: Remove the interface from NetworkManager . . . 927
104.1.3 Why the test is identical . . . 928
104.1.4 The CRC32 is not a detail . . . 928
104.1.5 The three differences from ex48 . . . 928
104.1.6 Test on PC . . . 928
104.1.7 Status . . . 929

105 TFTP Client over USB-NCM **931**
105.1 Example 82 — TFTP Client over USB-NCM . . . 931
105.1.1 Introduction . . . 931
105.1.2 PC Prerequisite: detach the interface from NetworkManager . . . 931
105.1.3 Why TFTP does not need a gate . . . 932
105.1.4 Test on PC . . . 932
105.1.5 Status . . . 932

106 SSH Server over USB-NCM **935**
106.1 Example 83 — SSH Server over USB-NCM . . . 935
106.1.1 Introduction . . . 935
106.1.2 PC Prerequisite: detach the interface from NetworkManager . . . 935
106.1.3 Static IP: the simplest clone in the series, not the most complex . . . 936
106.1.4 Differences compared to ex67 . . . 936
106.1.5 PC Test . . . 936
106.1.6 Status . . . 937

107 L2 Echo: the Driver Measured Without the Transport **939**
107.1 Example 84 — L2 Echo: the driver measured without the transport . . . 939
107.1.1 Introduction . . . 939
107.1.2 How it works . . . 939
107.1.3 Why static IP . . . 940
107.1.4 The network filter (`EN_NET_FILTER`) . . . 940
107.1.5 The two queues, and why they are short . . . 940
107.1.6 Usage . . . 941
107.1.7 The offered rate (`--mbps`), and why it is needed . . . 941
107.1.8 Searching for the limit (phase `echo_rate_limit`) . . . 942
107.1.9 Reading `eth dump` . . . 942
107.1.10 Configuration . . . 943
107.1.11 Hardware notes . . . 943

108 Several Connections at Once: How the Buffer Pool Is Shared **945**
108.1 Example 85 — Multiple connections together: how the buffer pool is shared . . . 945
108.1.1 Introduction . . . 945
108.1.2 Why it doesn't resolve by shrinking the rings . . . 945
108.1.3 The quota, calculated when needed . . . 946
108.1.4 What this example answers . . . 946
108.1.5 How it works . . . 947
108.1.6 Commands . . . 947
108.1.7 How to read the report . . . 948
108.1.8 Requirements . . . 948

109 callto: Calling an Address from the Console **949**
109.1 Example 86 — `callto`: calling an address from the console . . . 949
109.1.1 Introduction . . . 949
109.1.2 The call runs in a task of its own . . . 949
109.1.3 Syntax . . . 950
109.1.4 The three targets . . . 950
109.1.5 Typical session . . . 950
109.1.6 Portability . . . 951
109.1.7 Automated test . . . 951

110 Lost Wake-up Bench **953**
110.1 Example 87 — Lost wakeup bench . . . 953
110.1.1 Introduction . . . 953
110.1.2 How it works . . . 953
110.1.3 The phases . . . 954
110.1.4 Commands . . . 956
110.1.5 Configuration . . . 956
110.1.6 Test . . . 956

111 CLI Echo on the USB CDC Console **957**
111.1 Example 90 — Echo of CLI on USB CDC console . . . 957
111.1.1 Description . . . 957
111.1.2 Why this example . . . 957
111.1.3 os_config.h configuration . . . 958
111.1.4 Commands . . . 958
111.1.5 How to test . . . 958
111.1.6 Results (10/04, NUCLEO-H753ZI, Debug) . . . 959

112 Kernel Semantics Bench **961**
112.1 Example 91 — Kernel Semantic Testbench . . . 961
112.1.1 Description . . . 961
112.1.2 Why This Example . . . 961
112.1.3 The Mechanism: Actors with a Script . . . 961
112.1.4 Configuration os_config.h . . . 962
112.1.5 Commands . . . 962
112.1.6 How to test . . . 963
112.1.7 Results (05/10, NUCLEO-H753ZI, Debug) . . . 963

113 External SPI NOR Flash on the QSPI **965**
113.1 Example 92 — External SPI NOR Flash on QSPI/OCTOSPI . . . 965
113.1.1 Configuration (os_config.h) . . . 965
113.1.2 Hardware — 6-pin module . . . 965
113.1.3 Commands . . . 966

III caOS Reference **967**

114 Reference API caOS **969**
`ca_adc_hw_init_periph` . . . 969
`ca_adc_hw_init_periph_isr` . . . 970
`ca_adc_hw_irq_handler` . . . 972
`ca_adc_hw_read` . . . 974
`ca_adc_hw_start_isr` . . . 976
`ca_adc_hw_stop` . . . 977
`ca_adc_hw_stop_isr` . . . 979
`ca_adc_init` . . . 980
`ca_adc_init_isr` . . . 982
`ca_adc_irq_handler` . . . 983
`ca_adc_isr_done` . . . 984
`ca_adc_pin_init` . . . 986
`ca_adc_read` . . . 987
`ca_adc_start_isr` . . . 989

ca_adc_stop . . . 990
ca_aes_channel_decrypt . . . 991
ca_aes_channel_deinit . . . 993
ca_aes_channel_encrypt . . . 994
ca_aes_channel_free . . . 996
ca_aes_channel_init . . . 998
ca_aes_channel_rotate . . . 1000
ca_aes_ctx_set_iv . . . 1001
ca_aes_ctx_setup . . . 1002
ca_aes_decrypt . . . 1004
ca_aes_ecb_selftest . . . 1006
ca_aes_encrypt . . . 1007
ca_aes_gcm_decrypt . . . 1009
ca_aes_gcm_encrypt . . . 1011
ca_aes_gcm_selftest . . . 1013
ca_aes_hw_gcm . . . 1014
ca_aes_hw_init . . . 1016
ca_aes_hw_process . . . 1017
ca_aes_hw_setup . . . 1019
ca_aes_init . . . 1021
ca_arp_avail . . . 1022
ca_arp_del . . . 1023
ca_arp_del_ip . . . 1025
ca_arp_dump . . . 1026
ca_arp_find . . . 1027
ca_arp_flush . . . 1029
ca_arp_resolve . . . 1030
ca_arp_send_garp . . . 1031
ca_arp_service_init . . . 1033
ca_arp_service_start . . . 1035
ca_arp_service_update_local_ip . . . 1036
ca_arp_set . . . 1037
ca_barrier_init . . . 1038
ca_barrier_wait . . . 1040
ca_bkp_count . . . 1042
ca_bkp_hw_count . . . 1043
ca_bkp_hw_read . . . 1044
ca_bkp_hw_sram_open . . . 1046
ca_bkp_hw_sram_retain . . . 1047
ca_bkp_hw_sram_size . . . 1048
ca_bkp_hw_write . . . 1049
ca_bkp_read . . . 1051
ca_bkp_sram_read . . . 1052
ca_bkp_sram_retain . . . 1054
ca_bkp_sram_size . . . 1055
ca_bkp_sram_write . . . 1056
ca_bkp_write . . . 1058
ca_block_alloc . . . 1059
ca_block_free . . . 1060
ca_block_pool_free_count . . . 1062
ca_block_pool_init . . . 1063

ca_block_pool_total 1065
ca_boot_confirm 1066
ca_boot_image_slot 1068
ca_boot_image_start 1069
ca_boot_is_trial 1070
ca_boot_receive_update 1071
ca_boot_request_update 1073
ca_bswap16 1074
ca_bswap16_signed 1075
ca_bswap32 1076
ca_bswap32_signed 1078
ca_bswap64 1079
ca_bswap64_signed 1080
ca_bus_error_init 1082
ca_can_deinit 1083
ca_can_hw_deinit 1084
ca_can_hw_init 1086
ca_can_hw_recv 1088
ca_can_hw_send 1089
ca_can_init 1091
ca_can_recv 1092
ca_can_retune 1094
ca_can_send 1095
ca_caos_config_hash 1096
ca_caos_get_config 1098
ca_ccm_get_free 1099
ca_ccm_get_used 1100
ca_cli_callto_init 1101
ca_cli_char_class 1102
ca_cli_create_thread 1104
ca_cli_data_from_isr 1105
ca_cli_exec_capture 1106
ca_cli_exec_capture_stream 1108
ca_cli_filtered_count 1110
ca_cli_get_hook 1111
ca_cli_get_sender 1112
ca_cli_hist_add 1113
ca_cli_hist_count 1114
ca_cli_hist_forget 1116
ca_cli_hist_get 1117
ca_cli_init 1119
ca_cli_is_enabled 1120
ca_cli_register 1121
ca_cli_rx_byte 1123
ca_cli_set_hook 1124
ca_cli_set_secret_prefix 1126
ca_cli_set_sender 1127
ca_cli_stats 1128
ca_cli_typo_count 1130
ca_clock_hw_adc_init 1131
ca_clock_hw_dac_init 1133

ca_clock_hw_deinit . . . 1135
ca_clock_hw_dma_init . . . 1136
ca_clock_hw_i2c_init . . . 1138
ca_clock_hw_lptim_init . . . 1139
ca_clock_hw_lse_lock . . . 1141
ca_clock_hw_spi_init . . . 1142
ca_clock_hw_timer_init . . . 1144
ca_cond_broadcast . . . 1145
ca_cond_init . . . 1147
ca_cond_signal . . . 1148
ca_cond_timedwait . . . 1150
ca_cond_wait . . . 1151
ca_cordic_calc . . . 1153
ca_cordic_hw_calc . . . 1155
ca_cordic_hw_init . . . 1157
ca_cordic_init . . . 1158
ca_cordic_polar . . . 1159
ca_cordic_sincos . . . 1160
ca_crc32_get . . . 1162
ca_crc32_hw_init . . . 1164
ca_crc32_init . . . 1165
ca_critical_nesting_level . . . 1166
ca_dac_comp_init . . . 1167
ca_dac_comp_level . . . 1169
ca_dac_comp_stop . . . 1170
ca_dac_hw_comp_init . . . 1171
ca_dac_hw_comp_level . . . 1171
ca_dac_hw_comp_stop . . . 1173
ca_dac_hw_init_periph . . . 1174
ca_dac_hw_stop . . . 1175
ca_dac_hw_write . . . 1177
ca_dac_init . . . 1178
ca_dac_stop . . . 1180
ca_dac_write . . . 1181
ca_dhcp_service_bind . . . 1182
ca_dhcp_service_bound . . . 1184
ca_dhcp_service_get_dns . . . 1185
ca_dhcp_service_get_gw . . . 1186
ca_dhcp_service_get_ip . . . 1187
ca_dhcp_service_get_mask . . . 1188
ca_dhcp_service_get_ntp_server . . . 1189
ca_dhcp_service_get_state . . . 1190
ca_dhcp_service_init . . . 1192
ca_dhcp_service_net_ready . . . 1193
ca_dhcp_service_renew . . . 1195
ca_dhcp_service_rx_handler . . . 1196
ca_dhcp_service_start . . . 1198
ca_dma2d_abort . . . 1199
ca_dma2d_blend . . . 1200
ca_dma2d_bpp . . . 1201
ca_dma2d_busy . . . 1203

ca_dma2d_convert 1204
ca_dma2d_copy 1206
ca_dma2d_errors 1207
ca_dma2d_fill 1209
ca_dma2d_hw_abort 1210
ca_dma2d_hw_busy 1211
ca_dma2d_hw_clut 1212
ca_dma2d_hw_dst 1214
ca_dma2d_hw_init 1216
ca_dma2d_hw_irq 1217
ca_dma2d_hw_src 1218
ca_dma2d_hw_start 1220
ca_dma2d_init 1221
ca_dma2d_wait 1222
ca_dma_abort 1224
ca_dma_copy_inc 1225
ca_dma_copy_target 1227
ca_dma_free_channel 1230
ca_dma_free_channel_index 1231
ca_dma_free_channel_m2m 1233
ca_dma_get_channel_count 1235
ca_dma_get_remaining 1236
ca_dma_hw_abort 1237
ca_dma_hw_ch_to_flat 1239
ca_dma_hw_copy_inc 1240
ca_dma_hw_copy_target 1242
ca_dma_hw_flat_to_ch 1245
ca_dma_hw_get_channel_count 1246
ca_dma_hw_get_remaining 1248
ca_dma_hw_irq_disable 1249
ca_dma_hw_irq_enable 1251
ca_dma_hw_m2m_first_flat 1253
ca_dma_hw_release 1254
ca_dma_hw_set_request 1256
ca_dma_hw_start_circular 1257
ca_dma_init 1260
ca_dma_isr_complete 1261
ca_dma_release 1263
ca_dma_set_callback 1264
ca_dma_set_request 1266
ca_dma_start_circular 1267
ca_dma_stop_circular 1269
ca_dma_wait 1270
ca_dns_resolve 1272
ca_dns_service_bind 1273
ca_dns_service_init 1275
ca_dns_service_net_ready 1276
ca_dns_service_rx_handler 1277
ca_dns_service_set_server 1279
ca_dns_service_start 1280
ca_ecdsa_p256_keygen 1281

ca_ecdsa_p256_public_from_private 1283
ca_ecdsa_p256_selftest 1285
ca_ecdsa_p256_sign 1286
ca_ecdsa_p256_sign_hash 1288
ca_ecdsa_p256_test_init 1289
ca_ecdsa_p256_verify 1290
ca_ecdsa_p256_verify_hash 1291
ca_ed25519_keygen 1293
ca_ed25519_public_from_seed 1294
ca_ed25519_selftest 1296
ca_ed25519_sign 1297
ca_ed25519_test_init 1299
ca_ed25519_verify 1300
ca_enter_critical 1302
ca_eth_dbg_dump 1303
ca_eth_deinit 1304
ca_eth_hw_dbg_dump 1305
ca_eth_hw_deinit 1307
ca_eth_hw_dma_sync 1308
ca_eth_hw_init 1309
ca_eth_hw_isr 1311
ca_eth_hw_link_poll 1313
ca_eth_hw_phy_read 1314
ca_eth_hw_phy_write 1316
ca_eth_hw_pool_held_count 1318
ca_eth_hw_recv 1319
ca_eth_hw_send 1321
ca_eth_iface_get 1323
ca_eth_init 1324
ca_eth_link_poll 1326
ca_eth_phy_read 1327
ca_eth_phy_write 1329
ca_eth_pool_held_count 1330
ca_eth_recv 1331
ca_eth_restart 1333
ca_eth_send 1334
ca_event_clear 1336
ca_event_clear_isr 1337
ca_event_get 1339
ca_event_init 1340
ca_event_reset 1341
ca_event_set 1343
ca_event_set_isr 1345
ca_event_wait 1346
ca_event_wait_all 1348
ca_event_wait_any 1350
ca_exit_critical 1351
ca_exti_disable 1352
ca_exti_enable 1354
ca_exti_hw_clock_init 1355
ca_exti_hw_imr_disable 1356

ca_exti_hw_imr_enable . . . 1358
ca_exti_hw_nvic_disable . . . 1360
ca_exti_hw_nvic_enable . . . 1361
ca_exti_hw_select_source . . . 1363
ca_exti_hw_set_edge . . . 1364
ca_exti_hw_tamper_disable . . . 1366
ca_exti_hw_tamper_enable . . . 1367
ca_exti_init . . . 1369
ca_exti_tamper_disable . . . 1370
ca_exti_tamper_dispatch . . . 1371
ca_exti_tamper_enable . . . 1372
ca_fat_cat . . . 1374
ca_fat_cd . . . 1375
ca_fat_fdisk . . . 1376
ca_fat_free . . . 1378
ca_fat_is_mounted . . . 1379
ca_fat_lock . . . 1380
ca_fat_ls . . . 1381
ca_fat_mkdir . . . 1383
ca_fat_mkfs . . . 1384
ca_fat_mount . . . 1385
ca_fat_pwd . . . 1386
ca_fat_read_enc . . . 1388
ca_fat_rm . . . 1389
ca_fat_rmdir . . . 1391
ca_fat_service_init . . . 1392
ca_fat_service_set_event_cb . . . 1393
ca_fat_strerror . . . 1394
ca_fat_touch . . . 1396
ca_fat_trylock . . . 1397
ca_fat_umount . . . 1398
ca_fat_unlock . . . 1399
ca_fat_work_buf . . . 1400
ca_fat_write_enc . . . 1402
ca_flash_erase_range . . . 1403
ca_flash_erase_sector . . . 1405
ca_flash_erase_sector_irq . . . 1406
ca_flash_erase_sector_num . . . 1408
ca_flash_get_error . . . 1409
ca_flash_get_num_sectors . . . 1410
ca_flash_get_sector_addr . . . 1412
ca_flash_get_sector_size . . . 1413
ca_flash_hw_erase_sector . . . 1415
ca_flash_hw_irq_disable . . . 1416
ca_flash_hw_irq_enable . . . 1418
ca_flash_hw_irq_handler . . . 1419
ca_flash_hw_lock . . . 1421
ca_flash_hw_num_sectors . . . 1422
ca_flash_hw_nvic_enable . . . 1423
ca_flash_hw_sector_addr . . . 1425
ca_flash_hw_sector_size . . . 1426

ca_flash_hw_start_erase_irq 1428
ca_flash_hw_start_write_irq 1429
ca_flash_hw_unlock 1432
ca_flash_hw_write 1433
ca_flash_init 1435
ca_flash_irq_init 1436
ca_flash_lock 1437
ca_flash_unlock 1438
ca_flash_write 1439
ca_flash_write_irq 1441
ca_fmac_flags 1442
ca_fmac_hw_flags 1444
ca_fmac_hw_init 1445
ca_fmac_hw_param 1446
ca_fmac_hw_run 1448
ca_fmac_hw_setup 1450
ca_fmac_hw_stop 1451
ca_fmac_hw_write 1453
ca_fmac_init 1454
ca_fmac_run 1455
ca_fmac_start 1457
ca_fmac_stop 1458
ca_gpio_configure 1459
ca_gpio_get_io_port 1461
ca_gpio_hw_adc_pin_init 1462
ca_gpio_hw_configure 1464
ca_gpio_hw_dac_init 1466
ca_gpio_hw_i2c_init 1467
ca_gpio_hw_init_port 1469
ca_gpio_hw_spi_init 1471
ca_gpio_init_port 1473
ca_gpio_read 1475
ca_gpio_toggle 1476
ca_gpio_write 1478
ca_hash_compute 1479
ca_hash_ctx_final 1481
ca_hash_ctx_free 1483
ca_hash_ctx_init 1484
ca_hash_ctx_update 1485
ca_hash_hw_csr_count 1487
ca_hash_hw_digest 1489
ca_hash_hw_feed 1490
ca_hash_hw_init 1492
ca_hash_hw_restore_ctx 1493
ca_hash_hw_save_ctx 1495
ca_hash_hw_set_nblw 1496
ca_hash_hw_start 1498
ca_hash_init 1499
ca_hexdump 1500
ca_hexdump_u32 1502
ca_hkdf_sha256 1503

ca_hmac_sha256 1505
ca_hmac_sha256_final 1506
ca_hmac_sha256_init 1508
ca_hmac_sha256_update 1509
ca_http_resp_bad_request 1511
ca_http_resp_begin 1512
ca_http_resp_begin_chunked 1513
ca_http_resp_flush 1515
ca_http_resp_not_found 1516
ca_http_resp_puti 1518
ca_http_resp_puts 1519
ca_http_resp_write 1520
ca_http_serve_connection 1522
ca_http_service_start 1523
ca_http_update_local_ip 1525
ca_https_service_start 1526
ca_hw_led_count 1528
ca_hw_led_get 1529
ca_hw_led_init 1530
ca_hw_led_off 1531
ca_hw_led_on 1533
ca_hw_led_toggle 1534
ca_hw_memmap_get 1535
ca_i2c_bus_recovery 1536
ca_i2c_err_handler 1538
ca_i2c_hw_bus_recovery 1539
ca_i2c_hw_enable_irq 1541
ca_i2c_hw_err_handler 1542
ca_i2c_hw_init_periph 1544
ca_i2c_hw_recv_handler 1545
ca_i2c_hw_slave_init 1547
ca_i2c_hw_transfer7 1549
ca_i2c_init 1550
ca_i2c_read 1552
ca_i2c_recover_pins 1553
ca_i2c_recv_handler 1555
ca_i2c_slave_init 1556
ca_i2c_slave_set_tx 1558
ca_i2c_write 1559
ca_i2c_write_read 1560
ca_ip_defrag_init 1562
ca_ip_defrag_service_start 1563
ca_ip_defrag_submit 1565
ca_ip_fragment_send 1566
ca_iperf_client 1568
ca_iperf_server 1570
ca_iperf_service_start 1571
ca_iperf_update_local_ip 1573
ca_itm_init 1574
ca_itm_printf 1575
ca_itm_put 1577

ca_jpeg_decode 1578
ca_jpeg_encode 1580
ca_jpeg_errors 1582
ca_jpeg_geometry 1583
ca_jpeg_hw_decode 1584
ca_jpeg_hw_encode 1586
ca_jpeg_hw_get 1587
ca_jpeg_hw_header 1589
ca_jpeg_hw_init 1590
ca_jpeg_hw_put 1591
ca_jpeg_hw_reset 1593
ca_jpeg_hw_status 1594
ca_jpeg_hw_tables 1595
ca_jpeg_init 1597
ca_jpeg_mcu_to_rgb 1598
ca_jpeg_rgb_to_mcu 1599
ca_kdf_selftest 1601
ca_keys_store_allocate 1603
ca_keys_store_copy 1604
ca_keys_store_free 1606
ca_keys_store_get_access_mode 1608
ca_keys_store_get_keys_info 1609
ca_keys_store_init 1611
ca_keys_store_is_key_expired 1612
ca_keys_store_lock_completely 1614
ca_keys_store_lock_mpu_config 1615
ca_keys_store_set_access_mode 1616
ca_keys_store_set_expiry 1618
ca_keys_store_temp_disable_access 1620
ca_keys_store_temp_enable_access 1621
ca_keystore_service_init 1623
ca_keystore_service_is_active 1625
ca_keystore_service_lock 1626
ca_keystore_service_unlock 1627
ca_led_count 1629
ca_led_get 1630
ca_led_init 1631
ca_led_off 1632
ca_led_on 1633
ca_led_toggle 1635
ca_lptim_deinit 1636
ca_lptim_dispatch 1637
ca_lptim_hw_deinit 1639
ca_lptim_hw_get_cnt 1640
ca_lptim_hw_init 1642
ca_lptim_hw_start 1643
ca_lptim_hw_stop 1645
ca_lptim_init 1646
ca_lptim_init_oneshot 1648
ca_lptim_init_periodic 1649
ca_lptim_is_busy 1651

ca_lptim_start 1652
ca_lptim_stop 1654
ca_lptim_wake_active 1655
ca_lpuart_configure_gpio 1656
ca_lpuart_deinit 1658
ca_lpuart_dispatch 1659
ca_lpuart_hw_deinit 1660
ca_lpuart_hw_gpio_init 1661
ca_lpuart_hw_init 1662
ca_lpuart_hw_put 1664
ca_lpuart_hw_recv 1666
ca_lpuart_init 1667
ca_lpuart_is_busy 1669
ca_lpuart_put 1670
ca_lpuart_std_cli 1671
ca_lpuart_wake_active 1672
ca_mdns_service_bind 1674
ca_mdns_service_init 1675
ca_mdns_service_net_ready 1676
ca_mdns_service_rx_handler 1677
ca_mdns_service_start 1679
ca_measure_start 1680
ca_measure_stop 1681
ca_memcmp 1683
ca_memcpy 1684
ca_memcpy_u16 1686
ca_memcpy_u32 1688
ca_memmap_get 1689
ca_memory_adopt 1690
ca_memory_allocate 1692
ca_memory_allocate_shared 1693
ca_memory_canary_high 1695
ca_memory_canary_low 1696
ca_memory_check 1697
ca_memory_check_canary 1698
ca_memory_dump 1700
ca_memory_dump_avail 1701
ca_memory_free 1702
ca_memory_free_all_owned_by 1704
ca_memory_get_avail_memory 1706
ca_memory_get_end_address 1707
ca_memory_get_start_address 1708
ca_memory_get_task_usage 1710
ca_memory_get_total_memory 1711
ca_memory_get_total_size 1713
ca_memory_info 1714
ca_memory_init 1715
ca_memory_list 1717
ca_memory_reserve_secret 1718
ca_memory_restore_canary 1720
ca_memory_secret 1721

ca_memset 1722
ca_memset_u16 1723
ca_memset_u32 1725
ca_memzero 1726
ca_message_count 1728
ca_message_destroy 1729
ca_message_init 1730
ca_message_init_static 1732
ca_message_is_empty 1733
ca_message_peek_size 1735
ca_message_receive 1736
ca_message_receive_isr 1738
ca_message_reset 1740
ca_message_send 1741
ca_message_send_isr 1742
ca_message_space_available 1744
ca_modbus_disconnect 1745
ca_modbus_is_connected 1746
ca_modbus_read_coils 1748
ca_modbus_read_holding_regs 1750
ca_modbus_service_init 1751
ca_modbus_service_net_ready 1753
ca_modbus_write_multiple_regs 1754
ca_modbus_write_single_reg 1756
ca_mpu_add_region 1757
ca_mpu_dump_regions 1759
ca_mpu_first_free_region 1761
ca_mpu_get_available_regions 1762
ca_mpu_get_busy_regions 1764
ca_mpu_get_total_regions 1765
ca_mpu_init 1766
ca_mpu_protect_canary_msp 1768
ca_mpu_protect_canary_psp_end 1769
ca_mpu_protect_canary_psp_start 1771
ca_mpu_region_covers 1773
ca_mpu_region_is_reserved 1775
ca_mpu_update_region 1776
ca_mqtt_disconnect 1778
ca_mqtt_is_connected 1779
ca_mqtt_ping 1780
ca_mqtt_publish 1781
ca_mqtt_service_init 1783
ca_mqtt_service_net_ready 1784
ca_mqtt_subscribe 1786
ca_mutex_init 1787
ca_mutex_is_owner 1789
ca_mutex_lock 1790
ca_mutex_pi_refresh 1792
ca_mutex_pi_waiter_gone 1793
ca_mutex_timedlock 1794
ca_mutex_trylock 1796

ca_mutex_unlock . . . 1798
ca_ncm_iface_get . . . 1799
ca_net_buf_alloc . . . 1801
ca_net_buf_double_free_caller . . . 1802
ca_net_buf_double_free_count . . . 1803
ca_net_buf_free . . . 1805
ca_net_buf_free_count . . . 1806
ca_net_buf_pool_init . . . 1807
ca_net_buf_pressure . . . 1808
ca_net_buf_pressure_reset . . . 1810
ca_net_buf_ptr . . . 1811
ca_net_buf_ref . . . 1813
ca_net_buf_shared_count . . . 1814
ca_net_cksum . . . 1815
ca_net_count_err . . . 1817
ca_net_filter_clear . . . 1818
ca_net_filter_count . . . 1819
ca_net_filter_mac_set . . . 1820
ca_net_filter_set . . . 1822
ca_net_iface_at . . . 1823
ca_net_iface_count . . . 1825
ca_net_iface_link . . . 1826
ca_net_init . . . 1828
ca_net_link_down . . . 1829
ca_net_link_status . . . 1830
ca_net_link_up . . . 1831
ca_net_process . . . 1832
ca_net_proto_arp . . . 1834
ca_net_proto_arp_build_garp . . . 1835
ca_net_proto_arp_build_reply . . . 1837
ca_net_proto_bind_tcp . . . 1839
ca_net_proto_bind_udp . . . 1840
ca_net_proto_csum_fill . . . 1842
ca_net_proto_dhcp_build_discover . . . 1843
ca_net_proto_dhcp_build_request . . . 1845
ca_net_proto_dhcp_parse_offer . . . 1847
ca_net_proto_dns_build_query . . . 1849
ca_net_proto_dns_parse_response . . . 1851
ca_net_proto_get_local_ip . . . 1852
ca_net_proto_icmp_build_echo_reply . . . 1854
ca_net_proto_icmp_echo . . . 1855
ca_net_proto_init . . . 1857
ca_net_proto_ip4 . . . 1859
ca_net_proto_l2 . . . 1860
ca_net_proto_mdns_build_announce . . . 1862
ca_net_proto_mdns_parse_query . . . 1864
ca_net_proto_ntp_build_request . . . 1866
ca_net_proto_ntp_parse_response . . . 1868
ca_net_proto_set_local_ip . . . 1869
ca_net_proto_syslog_build . . . 1870
ca_net_proto_tcp . . . 1872

ca_net_proto_tcp_build_ack 1874
ca_net_proto_tcp_build_data 1875
ca_net_proto_tcp_build_fin 1877
ca_net_proto_tcp_build_rst 1879
ca_net_proto_tcp_build_syn 1881
ca_net_proto_tcp_build_synack 1883
ca_net_proto_udp 1884
ca_net_queue_empty 1886
ca_net_queue_full 1887
ca_net_queue_init 1889
ca_net_queue_pop 1890
ca_net_queue_push 1891
ca_net_register 1893
ca_net_register_iface 1894
ca_net_resolve_mac 1896
ca_net_rx_push 1897
ca_net_send 1899
ca_net_send_on 1900
ca_net_send_raw 1902
ca_net_set_phy_nrst 1903
ca_net_stats 1905
ca_net_tcp_cksum 1906
ca_net_tx_csum_offload 1908
ca_net_tx_mtu 1909
ca_notify_clear 1910
ca_notify_get 1912
ca_notify_give 1913
ca_notify_give_inc 1915
ca_notify_give_isr 1916
ca_notify_peek 1918
ca_notify_send_handle 1919
ca_notify_send_handle_isr 1921
ca_notify_take 1922
ca_notify_wait 1924
ca_ntp_service_bind 1925
ca_ntp_service_get_unix_time 1926
ca_ntp_service_init 1928
ca_ntp_service_net_ready 1929
ca_ntp_service_rx_handler 1930
ca_ntp_service_set_server 1932
ca_ntp_service_set_server_n 1933
ca_ntp_service_start 1934
ca_ntp_service_sync_now 1936
ca_ntp_service_synced 1937
ca_opamp_adc_channel 1938
ca_opamp_count 1939
ca_opamp_hw_adc_channel 1941
ca_opamp_hw_count 1942
ca_opamp_hw_init 1943
ca_opamp_hw_stop 1945
ca_opamp_init 1946

ca_opamp_stop . . . 1948
ca_os_banner . . . 1949
ca_os_clock_config . . . 1950
ca_os_clock_dump . . . 1952
ca_os_clock_get_freq . . . 1953
ca_os_clock_reduce_freq . . . 1954
ca_os_clock_restore_freq . . . 1955
ca_os_clocks_per_tick . . . 1957
ca_os_configure . . . 1958
ca_os_context_from_slot . . . 1959
ca_os_delay . . . 1961
ca_os_delay_ms . . . 1962
ca_os_destroy . . . 1963
ca_os_end . . . 1965
ca_os_handle_from_thid . . . 1966
ca_os_hw_clock_config . . . 1967
ca_os_hw_clock_dump . . . 1969
ca_os_hw_clock_reduce_freq . . . 1971
ca_os_hw_clock_restore_freq . . . 1972
ca_os_hw_periph_reset . . . 1974
ca_os_job . . . 1975
ca_os_job_suspend . . . 1977
ca_os_main . . . 1979
ca_os_remove_all_jobs . . . 1980
ca_os_remove_job_handle . . . 1982
ca_os_reset . . . 1983
ca_os_resume_handle . . . 1984
ca_os_run . . . 1986
ca_os_running . . . 1987
ca_os_sched_lock . . . 1989
ca_os_sched_unlock . . . 1990
ca_os_self_handle . . . 1991
ca_os_set_priority_handle . . . 1992
ca_os_signal_handle . . . 1993
ca_os_sleep . . . 1995
ca_os_sleep_prepare . . . 1996
ca_os_sleep_ticks . . . 1998
ca_os_sleep_until . . . 1999
ca_os_stack_from_handle . . . 2000
ca_os_suspend_handle . . . 2002
ca_os_task_handle . . . 2003
ca_os_task_slot . . . 2004
ca_os_task_snapshot . . . 2006
ca_os_thid_from_handle . . . 2008
ca_os_thread_reset_watchdog . . . 2009
ca_os_thread_set_watchdog . . . 2010
ca_os_tick . . . 2012
ca_os_tick_64 . . . 2013
ca_os_tick_64_ms . . . 2014
ca_os_tick_calibrate . . . 2015
ca_os_tick_compensate_us . . . 2017

ca_os_tick_disable 2018
ca_os_tick_drift_cycles 2019
ca_os_tick_hw_counter 2020
ca_os_tick_init 2022
ca_os_tick_ms 2023
ca_os_tick_restart_gap 2025
ca_os_tick_restart_quantum 2026
ca_os_tick_resume 2027
ca_os_tick_retune 2028
ca_os_tick_suspend 2029
ca_os_tick_x_ms 2031
ca_os_timeout 2032
ca_os_wait 2034
ca_os_wait_prepare 2035
ca_os_wake_up_handle 2037
ca_os_yield 2038
ca_parse_ipv4 2039
ca_parse_mac 2041
ca_pbkdf2_sha256 2042
ca_pktrace_enable 2044
ca_pktrace_filter_dst 2045
ca_pktrace_filter_src 2046
ca_pktrace_push_rx 2048
ca_pktrace_push_tx 2049
ca_pktrace_set_sender 2050
ca_pktrace_start 2052
ca_pool_init 2053
ca_pool_stats 2054
ca_pool_submit 2056
ca_printf 2057
ca_printf_n 2059
ca_put 2061
ca_put_n 2062
ca_pwr_enter 2064
ca_pwr_get_info 2065
ca_pwr_hw_dbg_dump 2067
ca_pwr_hw_enter_standby 2068
ca_pwr_hw_enter_stop 2070
ca_pwr_hw_get_vos 2071
ca_pwr_hw_init 2073
ca_pwr_hw_lptim_arm 2074
ca_pwr_hw_lptim_disarm 2075
ca_pwr_hw_max_freq 2076
ca_pwr_hw_periph_table 2077
ca_pwr_hw_standby_resumed 2079
ca_pwr_hw_stop_wake_nvic 2081
ca_pwr_hw_wake_pin_enable 2082
ca_pwr_init 2083
ca_pwr_periph_table 2084
ca_pwr_restore_freq 2086
ca_pwr_set_freq 2087

ca_qspi_command 2088
ca_qspi_erase_sector 2089
ca_qspi_get_pin 2091
ca_qspi_hw_abort 2092
ca_qspi_hw_get_pin 2093
ca_qspi_hw_init 2095
ca_qspi_hw_mmap 2097
ca_qspi_hw_start_autopoll 2099
ca_qspi_hw_start_xfer 2100
ca_qspi_init 2102
ca_qspi_isr_done 2103
ca_qspi_mmap_disable 2104
ca_qspi_mmap_enable 2106
ca_qspi_read 2107
ca_qspi_read_id 2108
ca_qspi_write 2110
ca_queue_heap_available 2111
ca_queue_heap_count 2113
ca_queue_heap_destroy 2114
ca_queue_heap_init 2115
ca_queue_heap_is_empty 2117
ca_queue_heap_is_full 2118
ca_queue_heap_overwrite 2120
ca_queue_heap_peek 2121
ca_queue_heap_receive 2123
ca_queue_heap_receive_isr 2124
ca_queue_heap_reset 2126
ca_queue_heap_send 2127
ca_queue_heap_send_front 2129
ca_queue_heap_send_isr 2131
ca_queue_heap_tryreceive 2132
ca_queue_heap_trysend 2134
ca_queue_static_available 2135
ca_queue_static_count 2137
ca_queue_static_init 2138
ca_queue_static_is_empty 2140
ca_queue_static_is_full 2141
ca_queue_static_overwrite 2143
ca_queue_static_peek 2144
ca_queue_static_receive 2146
ca_queue_static_receive_isr 2148
ca_queue_static_reset 2149
ca_queue_static_send 2151
ca_queue_static_send_front 2152
ca_queue_static_send_isr 2154
ca_queue_static_tryreceive 2155
ca_queue_static_trysend 2157
ca_rmutex_get_count 2159
ca_rmutex_init 2160
ca_rmutex_is_owner 2162
ca_rmutex_lock 2163

ca_rmutex_timedlock 2165
ca_rmutex_trylock 2166
ca_rmutex_unlock 2168
ca_rng_fill 2169
ca_rng_get 2171
ca_rng_hw_get 2172
ca_rng_hw_init 2174
ca_rng_init 2175
ca_rng_is_hardware 2176
ca_route_add 2177
ca_route_del 2179
ca_route_flush 2180
ca_route_get 2181
ca_route_init 2183
ca_route_lookup 2184
ca_route_set_gw 2186
ca_route_update_local 2187
ca_rtc_calibrate 2188
ca_rtc_clear_alarm 2190
ca_rtc_clock_init 2191
ca_rtc_from_net 2193
ca_rtc_get 2194
ca_rtc_get_clk_src 2195
ca_rtc_get_stamp_ms 2197
ca_rtc_get_timestamp 2198
ca_rtc_hw_calibrate 2200
ca_rtc_hw_clear_alarm 2201
ca_rtc_hw_get 2203
ca_rtc_hw_get_src 2205
ca_rtc_hw_get_sub 2206
ca_rtc_hw_init 2208
ca_rtc_hw_lse_on 2209
ca_rtc_hw_set 2211
ca_rtc_hw_set_alarm 2212
ca_rtc_hw_set_prescaler 2214
ca_rtc_init 2216
ca_rtc_lse_start_ms 2217
ca_rtc_lsi_hz 2219
ca_rtc_set 2220
ca_rtc_set_alarm 2221
ca_rtc_to_net 2223
ca_rwlock_init 2224
ca_rwlock_read_lock 2225
ca_rwlock_read_unlock 2227
ca_rwlock_write_lock 2228
ca_rwlock_write_unlock 2230
ca_safeguard_failed 2231
ca_safeguard_reset 2233
ca_sai_errors 2234
ca_sai_fs 2235
ca_sai_hw_dma_bind 2236

ca_sai_hw_dma_count_div 2238
ca_sai_hw_dr 2239
ca_sai_hw_errors 2241
ca_sai_hw_get 2242
ca_sai_hw_init 2243
ca_sai_hw_put 2245
ca_sai_hw_start 2247
ca_sai_hw_stop 2248
ca_sai_init 2249
ca_sai_stop 2251
ca_sai_stream_pos 2252
ca_sai_stream_start 2253
ca_sai_xfer 2255
ca_sai_xfer_dma 2256
ca_scheduler_check_all_canaries 2258
ca_scheduler_dump_all_tasks 2259
ca_scheduler_dump_task 2261
ca_scheduler_get_context_from_idx 2262
ca_scheduler_get_current_context 2264
ca_scheduler_get_current_task_id 2265
ca_scheduler_get_next_wait 2267
ca_scheduler_get_runtime_total 2268
ca_scheduler_get_task_name 2269
ca_scheduler_init 2270
ca_scheduler_is_valid_context 2272
ca_scheduler_manage_fault 2273
ca_scheduler_sched_info 2274
ca_scheduler_set_eff_priority 2275
ca_scheduler_set_next_wait 2277
ca_scheduler_signal_wake_up 2278
ca_scheduler_size 2279
ca_scheduler_tasks_info 2280
ca_scheduler_tick 2282
ca_scheduler_update_all_stack_usage 2283
ca_scheduler_wake_waiter 2284
ca_scheduler_wake_waiter_handle 2286
ca_sdio_get_block_count 2288
ca_sdio_hw_get_block_count 2289
ca_sdio_hw_init 2290
ca_sdio_hw_read_block 2292
ca_sdio_hw_write_block 2293
ca_sdio_hw_write_blocks 2295
ca_sdio_init 2297
ca_sdio_read_block 2298
ca_sdio_write_block 2300
ca_sdio_write_blocks 2301
ca_secure_pipe_deinit 2303
ca_secure_pipe_init 2304
ca_secure_pipe_recv 2306
ca_secure_pipe_send 2307
ca_sem_getvalue 2309

ca_sem_init 2310
ca_sem_post 2312
ca_sem_post_capped 2314
ca_sem_timedwait 2315
ca_sem_trywait 2317
ca_sem_wait 2318
ca_sha256 2320
ca_sha256_final 2321
ca_sha256_init 2323
ca_sha256_selftest 2324
ca_sha256_test_init 2325
ca_sha256_update 2326
ca_sha512 2328
ca_sha512_final 2329
ca_sha512_init 2330
ca_sha512_selftest 2332
ca_sha512_test_init 2333
ca_sha512_update 2334
ca_skip_sp 2336
ca_snprintf 2337
ca_sp_link_accept 2338
ca_sp_link_deinit 2340
ca_sp_link_hello 2341
ca_sp_link_init 2343
ca_sp_link_is_up 2345
ca_sp_link_need_rekey 2346
ca_sp_link_open 2347
ca_sp_link_seal 2349
ca_spdifrx_channel_status 2351
ca_spdifrx_errors 2353
ca_spdifrx_fs 2354
ca_spdifrx_hw_dma_bind 2355
ca_spdifrx_hw_dr 2357
ca_spdifrx_hw_flags 2358
ca_spdifrx_hw_get 2360
ca_spdifrx_hw_get_cs 2361
ca_spdifrx_hw_init 2362
ca_spdifrx_hw_state 2364
ca_spdifrx_hw_width5 2365
ca_spdifrx_init 2366
ca_spdifrx_read 2368
ca_spdifrx_read_dma 2369
ca_spdifrx_stop 2371
ca_spdifrx_sync 2372
ca_spi_cs_assert 2373
ca_spi_cs_deassert 2374
ca_spi_dma_read 2376
ca_spi_dma_write 2377
ca_spi_dma_xfer 2379
ca_spi_hw_dma_bind 2380
ca_spi_hw_dma_finish 2382

ca_spi_hw_dma_kick 2384
ca_spi_hw_dma_prepare 2385
ca_spi_hw_dma_rx_addr 2387
ca_spi_hw_dma_tx_addr 2388
ca_spi_hw_init_periph 2390
ca_spi_hw_init_periph_slave 2392
ca_spi_hw_recv_handler 2394
ca_spi_hw_slave_sync 2395
ca_spi_hw_slave_sync_pending 2397
ca_spi_hw_slave_tx_fill 2398
ca_spi_hw_stop 2399
ca_spi_hw_xfer 2401
ca_spi_init 2403
ca_spi_init_dma 2404
ca_spi_read_buf 2406
ca_spi_recv_handler 2407
ca_spi_slave_sync 2409
ca_spi_stop 2410
ca_spi_write_buf 2411
ca_spi_xfer 2413
ca_spinlock_cooperative_init 2414
ca_spinlock_get 2416
ca_spinlock_irq_init 2417
ca_spinlock_lock_adaptive 2418
ca_spinlock_lock_cooperative 2420
ca_spinlock_lock_irq 2422
ca_spinlock_trylock_cooperative 2423
ca_spinlock_trylock_irq 2425
ca_spinlock_unlock_cooperative 2426
ca_spinlock_unlock_irq 2428
ca_ssh_bpp_selftest 2429
ca_ssh_service_init 2431
ca_ssh_service_stop 2432
ca_ssh_test_init 2433
ca_stdout_create_thread 2434
ca_stdout_echo 2436
ca_stdout_echo_dropped 2437
ca_stdout_flush 2438
ca_stdout_get_repeater 2439
ca_stdout_get_sender 2440
ca_stdout_init 2441
ca_stdout_printf 2442
ca_stdout_printf_n 2444
ca_stdout_put 2446
ca_stdout_put_n 2447
ca_stdout_put_str 2449
ca_stdout_put_str_n 2451
ca_stdout_set_repeater 2452
ca_stdout_set_sender 2454
ca_stdout_stats 2455
ca_strcmp 2456

ca_strcpy 2458
ca_stream_bytes_available 2459
ca_stream_destroy 2461
ca_stream_init 2462
ca_stream_init_static 2463
ca_stream_is_empty 2465
ca_stream_is_full 2466
ca_stream_receive 2468
ca_stream_receive_isr 2469
ca_stream_reset 2471
ca_stream_send 2472
ca_stream_send_isr 2474
ca_stream_set_trigger 2476
ca_stream_space_available 2477
ca_stricmp 2478
ca_strlen 2480
ca_strncmp 2481
ca_strncpy 2483
ca_strtol 2484
ca_strtol_index 2485
ca_sw_timer_count_active 2487
ca_sw_timer_create 2489
ca_sw_timer_delete 2490
ca_sw_timer_get_remaining 2492
ca_sw_timer_get_state 2493
ca_sw_timer_init 2495
ca_sw_timer_reset 2496
ca_sw_timer_set_period 2498
ca_sw_timer_start 2499
ca_sw_timer_stop 2501
ca_syslog_dropped 2502
ca_syslog_send 2504
ca_syslog_service_init 2505
ca_syslog_service_net_ready 2507
ca_syslog_service_set_server 2508
ca_syslog_service_start 2509
ca_system_banner 2510
ca_task_set_emergency_exec 2512
ca_task_swap_param 2513
ca_tcp_abort 2515
ca_tcp_accept 2516
ca_tcp_close 2517
ca_tcp_connect 2519
ca_tcp_is_connected 2521
ca_tcp_listen 2522
ca_tcp_recv 2524
ca_tcp_send 2525
ca_tcp_service_start 2527
ca_tcp_unlisten 2528
ca_tcp_wait_writable 2529
ca_temp_dts_present 2531

ca_temp_hw_convert 2532
ca_temp_hw_dts_alarm 2534
ca_temp_hw_dts_centi 2535
ca_temp_hw_dts_init 2537
ca_temp_hw_dts_raw 2538
ca_temp_hw_dts_sample 2540
ca_temp_hw_init 2541
ca_temp_hw_read_raw 2543
ca_temp_hw_read_vrefint 2544
ca_temp_init 2546
ca_temp_read 2547
ca_temp_read_dts 2548
ca_temp_read_raw 2549
ca_temp_retune 2550
ca_temp_set_alarm 2552
ca_tftp_get 2553
ca_tftp_put 2555
ca_tftp_receive 2556
ca_tftp_service_init 2558
ca_timer_capture_event 2559
ca_timer_clock_hz 2560
ca_timer_counter_read 2562
ca_timer_deinit 2563
ca_timer_dispatch 2564
ca_timer_encoder_read 2566
ca_timer_hw_clock_hz 2567
ca_timer_hw_deinit 2569
ca_timer_hw_get_cnt 2570
ca_timer_hw_init_capture 2572
ca_timer_hw_init_counter 2573
ca_timer_hw_init_counter_irq 2575
ca_timer_hw_init_encoder 2577
ca_timer_hw_init_oneshot 2579
ca_timer_hw_init_periodic 2581
ca_timer_hw_init_pwm 2582
ca_timer_hw_set_pwm 2584
ca_timer_hw_start 2586
ca_timer_hw_stop 2587
ca_timer_init 2589
ca_timer_init_capture 2590
ca_timer_init_counter 2592
ca_timer_init_counter_irq 2593
ca_timer_init_encoder 2595
ca_timer_init_oneshot 2596
ca_timer_init_periodic 2598
ca_timer_init_pwm 2600
ca_timer_is_busy 2601
ca_timer_set_pwm 2603
ca_timer_start 2604
ca_timer_stop 2606
ca_tls_accept 2607

ca_tls_close 2608
ca_tls_read 2610
ca_tls_write 2611
ca_top 2613
ca_uart_configure_gpio 2614
ca_uart_dbg_put 2615
ca_uart_dma_abort 2617
ca_uart_dma_recv 2618
ca_uart_dma_remaining 2620
ca_uart_dma_send 2621
ca_uart_dma_wait 2623
ca_uart_hw_configure_gpio 2624
ca_uart_hw_dma_bind 2626
ca_uart_hw_dma_init 2628
ca_uart_hw_dma_rx_addr 2629
ca_uart_hw_dma_rx_off 2630
ca_uart_hw_dma_rx_on 2632
ca_uart_hw_dma_tx_addr 2633
ca_uart_hw_init 2635
ca_uart_hw_put 2636
ca_uart_hw_recv_handler 2638
ca_uart_init 2640
ca_uart_init_dma 2641
ca_uart_put 2643
ca_uart_recv_handler 2645
ca_uart_retune 2646
ca_udp_client_connect 2647
ca_udp_client_disconnect 2649
ca_udp_client_recv 2650
ca_udp_client_refresh_mac 2652
ca_udp_client_send 2653
ca_udp_server_bind 2655
ca_udp_server_client_count 2656
ca_udp_server_disconnect 2657
ca_udp_server_get_client 2659
ca_udp_server_send 2660
ca_udp_server_set_local_ip 2662
ca_udp_server_start 2663
ca_usb_cdc_connected 2665
ca_usb_cdc_get_line 2666
ca_usb_cdc_wait_connected 2667
ca_usb_cdc_write 2669
ca_usb_connect 2671
ca_usb_dbg_fs_reset 2672
ca_usb_dbg_fs_stat 2673
ca_usb_dbg_hs_reset 2675
ca_usb_dbg_hs_stat 2676
ca_usb_dbg_print 2678
ca_usb_dbg_reset 2679
ca_usb_dbg_stat 2680
ca_usb_dev_mng_add 2682

ca_usb_dev_mng_cdc_cli 2683
ca_usb_dev_mng_get_num 2685
ca_usb_dev_mng_init 2686
ca_usb_dev_mng_remove 2687
ca_usb_dev_mng_start 2689
ca_usb_dev_mng_stop 2690
ca_usb_dfu_poll 2691
ca_usb_disconnect 2693
ca_usb_ep_clear_stall 2694
ca_usb_ep_close 2695
ca_usb_ep_is_stalled 2697
ca_usb_ep_open 2698
ca_usb_ep_read 2700
ca_usb_ep_stall 2701
ca_usb_ep_write 2703
ca_usb_fs_dev_clock_init 2704
ca_usb_fs_dev_connect 2706
ca_usb_fs_dev_disconnect 2707
ca_usb_fs_dev_ep_clear_stall 2708
ca_usb_fs_dev_ep_close 2710
ca_usb_fs_dev_ep_is_stalled 2712
ca_usb_fs_dev_ep_open 2713
ca_usb_fs_dev_ep_read 2715
ca_usb_fs_dev_ep_stall 2716
ca_usb_fs_dev_ep_write 2718
ca_usb_fs_dev_get_speed 2719
ca_usb_fs_dev_gpio_init 2721
ca_usb_fs_dev_init 2722
ca_usb_fs_dev_isr 2724
ca_usb_fs_dev_set_address 2725
ca_usb_get_speed 2726
ca_usb_hs_dev_clock_init 2728
ca_usb_hs_dev_connect 2729
ca_usb_hs_dev_disconnect 2731
ca_usb_hs_dev_ep_clear_stall 2732
ca_usb_hs_dev_ep_close 2733
ca_usb_hs_dev_ep_is_stalled 2735
ca_usb_hs_dev_ep_open 2737
ca_usb_hs_dev_ep_read 2738
ca_usb_hs_dev_ep_stall 2740
ca_usb_hs_dev_ep_write 2741
ca_usb_hs_dev_get_speed 2743
ca_usb_hs_dev_gpio_init 2744
ca_usb_hs_dev_init 2746
ca_usb_hs_dev_isr 2747
ca_usb_hs_dev_set_address 2749
ca_usb_init 2750
ca_usb_isr 2752
ca_usb_ncm_is_up 2753
ca_usb_ncm_send_frame 2754
ca_usb_set_address 2756

ca_valid_pointer 2757
ca_vsnprintf 2759
ca_wait_queue_dequeue 2760
ca_wait_queue_enqueue 2762
ca_wait_queue_init 2763
ca_wait_queue_is_empty 2765
ca_wait_queue_remove 2766
ca_wdt_early_supported 2768
ca_wdt_enable 2769
ca_wdt_get_timeout_ms 2770
ca_wdt_hw_early_supported 2772
ca_wdt_hw_enable 2773
ca_wdt_hw_get_timeout_ms 2775
ca_wdt_hw_init 2776
ca_wdt_hw_is_enabled 2778
ca_wdt_hw_max_timeout_ms 2779
ca_wdt_hw_on_early 2781
ca_wdt_hw_reload 2782
ca_wdt_hw_reset_cause 2783
ca_wdt_init 2785
ca_wdt_is_enabled 2786
ca_wdt_max_timeout_ms 2787
ca_wdt_on_early 2789
ca_wdt_reload 2790
ca_wdt_reset_cause 2791
ca_x25519 2792
ca_x25519_base 2794
ca_x25519_selftest 2795
ca_x25519_test_init 2797

115 Reference Tipi di dato caOS **2799**

ca_adc_config_t 2799
ca_adc_isr_cb_t 2800
ca_aes_channel_t 2800
ca_aes_ctx_t 2801
ca_aes_iv_gen_cb_t 2802
ca_aes_key_ratchet_cb_t 2803
ca_aes_keysize_t 2804
ca_aes_mode_t 2805
ca_arp_cfg_t 2806
ca_arp_frame_t 2806
ca_barrier_t 2807
ca_block_pool_t 2808
ca_can_cfg_t 2810
ca_can_frame_t 2810
ca_caos_config_status 2811
ca_cli_hook_t 2815
ca_cli_node_t 2816
ca_condvar_t 2816
ca_cordic_func_t 2817
ca_dac_comp_hyst_t 2818
ca_dac_config_t 2819

ca_dhcp_bound_cb_t 2820
ca_dhcp_bound_t 2820
ca_dhcp_cfg_t 2821
ca_dhcp_expire_cb_t 2822
ca_dhcp_offer_t 2823
ca_dma2d_am_t 2824
ca_dma2d_dst_t 2825
ca_dma2d_fmt_t 2825
ca_dma2d_src_t 2827
ca_dma_cb_t 2828
ca_dma_ch_t 2828
ca_dma_dir_t 2829
ca_dma_width_t 2830
ca_dns_hdr_t 2830
ca_dns_qtail_t 2831
ca_dns_response_t 2832
ca_dns_rr_hdr_t 2833
ca_emergency_fn_t 2834
ca_eth_cfg_t 2834
ca_eth_error_cb_t 2835
ca_eth_link_cb_t 2836
ca_eth_rx_cb_t 2837
ca_eth_tx_done_cb_t 2837
ca_event_group_t 2838
ca_event_waiter_meta_t 2839
ca_exti_cb_t 2840
ca_exti_edge_t 2840
ca_fat_event_cb_t 2841
ca_fat_event_t 2842
ca_fat_read_cb_t 2843
ca_flash_key_block_hdr_t 2843
ca_flash_key_entry_t 2844
ca_fmac_filter_t 2845
ca_gpio_mode_t 2846
ca_gpio_otype_t 2847
ca_gpio_pin_t 2848
ca_gpio_port_t 2849
ca_gpio_pull_t 2850
ca_gpio_speed_t 2850
ca_hash_algo_t 2851
ca_hash_ctx_t 2852
ca_hmac_sha256_ctx_t 2853
ca_http_conn_t 2854
ca_http_handler_fn 2854
ca_http_io_t 2855
ca_http_request_t 2856
ca_http_response_t 2857
ca_http_route_t 2859
ca_i2c_config_t 2860
ca_i2c_slave_ctx_t 2860
ca_i2c_slave_rx_cb 2861

ca_i2c_speed_t 2862
ca_icmp_cfg_t 2863
ca_icmp_echo_t 2863
ca_ip4_frame_t 2864
ca_iperf_result_t 2865
ca_ipv4_t 2866
ca_job_type 2867
ca_jpeg_cfg_t 2868
ca_jpeg_css_t 2869
ca_jpeg_info_t 2870
ca_keys_store_access_mode_t 2871
ca_l2_frame_t 2872
ca_l2_result_t 2872
ca_l2_vlan_frame_t 2873
ca_lptim_cb_t 2874
ca_lptim_mode_t 2875
ca_lpuart_rx_cb_t 2875
ca_mac_t 2876
ca_mem_range 2877
ca_memory_check_t 2878
ca_message_buffer_t 2879
ca_mqtt_connected_cb_t 2881
ca_mqtt_disconnected_cb_t 2881
ca_mqtt_message_cb_t 2882
ca_mutex_t 2883
ca_net_buf_meta_t 2883
ca_net_buf_pressure_t 2884
ca_net_buf_rx_t 2885
ca_net_buf_t 2887
ca_net_buf_tx_t 2888
ca_net_cfg_t 2889
ca_net_filter_fn_t 2890
ca_net_handler_t 2891
ca_net_mac_filter_fn_t 2892
ca_net_queue_t 2892
ca_net_stats_t 2893
ca_ntp_cfg_t 2894
ca_ntp_fail_cb_t 2895
ca_ntp_pkt_t 2896
ca_ntp_sync_cb_t 2897
ca_ntp_synced_t 2898
ca_opamp_input_t 2899
ca_pool_cfg_t 2899
ca_pwr_info_t 2900
ca_pwr_mode_t 2901
ca_pwr_periph_t 2902
ca_qspi_cfg_t 2903
ca_qspi_pin_t 2904
ca_qspi_xfer_t 2905
ca_queue_static_t 2906
ca_queue_t 2908

ca_recursive_mutex_t 2909
ca_route_entry_t 2910
ca_rtc_alarm_cb_t 2911
ca_rtc_alarm_t 2912
ca_rtc_clk_src_t 2912
ca_rtc_dt_t 2913
ca_rwlock_t 2914
ca_sai_config_t 2915
ca_sai_proto_t 2916
ca_scheduler_tasks_info_mode 2917
ca_secure_key_entry_t 2917
ca_secure_pipe_end_t 2918
ca_secure_pipe_t 2919
ca_sem_name_t 2920
ca_semaphore_t 2921
ca_sha256_ctx_t 2922
ca_sha512_ctx_t 2923
ca_sp_link_stats_t 2924
ca_sp_link_t 2925
ca_spi_br_t 2928
ca_spi_config_t 2928
ca_spi_slave_cb_t 2930
ca_spinlock_irq_t 2930
ca_spinlock_t 2931
ca_stream_buffer_t 2932
ca_sw_timer_handle_t 2933
ca_sw_timer_mode_t 2934
ca_sw_timer_state_t 2935
ca_sw_timer_t 2936
ca_syslog_cfg_t 2937
ca_task_fn_t 2937
ca_task_handle_t 2938
ca_task_list_t 2939
ca_tcp_close_t 2940
ca_tcp_conn_t 2940
ca_tcp_connect_cfg_t 2941
ca_tcp_frame_t 2942
ca_tcp_listen_cfg_t 2943
ca_tcp_rx_raw_t 2944
ca_tcp_rx_t 2945
ca_temp_alarm_cb_t 2946
ca_tftp_cfg_t 2947
ca_tftp_err_t 2948
ca_tftp_rx_block_t 2948
ca_tftp_srv_cfg_t 2949
ca_tftp_wrq_t 2950
ca_thread_context_t 2951
ca_tick_speed_t 2953
ca_timer_callback_t 2954
ca_timer_cb_t 2954
ca_timer_ic_cb_t 2955

ca_timer_ic_src_t . . . 2955
ca_timer_mode_t . . . 2956
ca_tls_config_t . . . 2957
ca_tls_dir_t . . . 2958
ca_tls_io_t . . . 2959
ca_tls_t . . . 2960
ca_uart_baud_rate_t . . . 2961
ca_uart_rx_cb_t . . . 2962
ca_udp_binding_t . . . 2963
ca_udp_cfg_t . . . 2964
ca_udp_cli_rx_t . . . 2964
ca_udp_client_cfg_t . . . 2965
ca_udp_frame_t . . . 2966
ca_udp_rx_t . . . 2967
ca_udp_server_cfg_t . . . 2968
ca_udp_srv_client_info_t . . . 2969
ca_udp_srv_event_t . . . 2970
ca_udp_srv_rx_t . . . 2970
ca_usb_cb_t . . . 2971
ca_usb_cdc_cfg_t . . . 2972
ca_usb_cdc_line_t . . . 2973
ca_usb_class_ops_t . . . 2974
ca_usb_custom_cfg_t . . . 2977
ca_usb_dbg_t . . . 2978
ca_usb_dev_ops_t . . . 2980
ca_usb_dfu_cfg_t . . . 2982
ca_usb_ep_type_t . . . 2983
ca_usb_mounted_cb_t . . . 2984
ca_usb_msc_cfg_t . . . 2985
ca_usb_ncm_cfg_t . . . 2986
ca_usb_ncm_rx_cb_t . . . 2987
ca_usb_reset_cb_t . . . 2988
ca_usb_rx_cb_t . . . 2988
ca_usb_setup_cb_t . . . 2989
ca_usb_speed_t . . . 2990
ca_usb_tx_done_cb_t . . . 2990
ca_usb_type_t . . . 2991
ca_usb_unmounted_cb_t . . . 2992
ca_wait_queue_t . . . 2993
ca_wdt_early_cb_t . . . 2994
ca_wdt_mode_t . . . 2994
ca_wdt_reset_t . . . 2995
ca_wdt_type_t . . . 2996

116 Reference Enumerazioni caOS **2997**

ca_aes_keysize_t . . . 2997
ca_aes_mode_t . . . 2997
ca_cordic_func_t . . . 2997
ca_dac_comp_hyst_t . . . 2998
ca_dma2d_am_t . . . 2998
ca_dma2d_fmt_t . . . 2998
ca_dma_dir_t . . . 2999

ca_dma_width_t . . . 2999
ca_exti_edge_t . . . 2999
ca_fat_event_t . . . 3000
ca_gpio_mode_t . . . 3000
ca_gpio_otype_t . . . 3000
ca_gpio_pin_t . . . 3001
ca_gpio_port_t . . . 3001
ca_gpio_pull_t . . . 3002
ca_gpio_speed_t . . . 3002
ca_hash_algo_t . . . 3003
ca_i2c_speed_t . . . 3003
ca_job_type . . . 3004
ca_jpeg_css_t . . . 3004
ca_keys_store_access_mode_t . . . 3004
ca_l2_result_t . . . 3005
ca_lptim_mode_t . . . 3005
ca_opamp_input_t . . . 3006
ca_pwr_mode_t . . . 3006
ca_qspi_pin_t . . . 3006
ca_rtc_clk_src_t . . . 3006
ca_sai_proto_t . . . 3007
ca_scheduler_tasks_info_mode . . . 3007
ca_secure_pipe_end_t . . . 3007
ca_spi_br_t . . . 3008
ca_stack_size_t . . . 3008
ca_sw_timer_mode_t . . . 3009
ca_sw_timer_state_t . . . 3009
ca_tick_speed_t . . . 3009
ca_timer_ic_src_t . . . 3010
ca_timer_mode_t . . . 3010
ca_uart_baud_rate_t . . . 3011
ca_usb_ep_type_t . . . 3011
ca_usb_speed_t . . . 3011
ca_usb_type_t . . . 3012
ca_wdt_mode_t . . . 3012
ca_wdt_reset_t . . . 3013
ca_wdt_type_t . . . 3013

117 Reference Definizioni caOS **3015**

AES_CH_HDR_SIZE . . . 3015
AES_CH_IV_SIZE . . . 3015
AES_CH_OVERHEAD . . . 3016
AES_CH_TAG_SIZE . . . 3016
AES_GCM_TAG_SIZE . . . 3016
ARP_CACHE_SIZE . . . 3017
ARP_ENTRY_TTL . . . 3017
ARP_HTYPE_ETH . . . 3017
ARP_OP_REPLY . . . 3018
ARP_OP_REQUEST . . . 3018
ARP_PTYPE_IP . . . 3018
ARP_RETRY_MS . . . 3019
ARP_RETRY_N . . . 3019

ARRAY_SIZE 3020
CAOS_ENDIAN_BIG 3020
CAOS_ENDIAN_LITTLE 3020
CA_BOOT_BKP_MSG 3021
CA_BOOT_BKP_STATE 3021
CA_BOOT_CONFIRMED 3021
CA_BOOT_HDR_SIZE 3022
CA_BOOT_IWDG_MS 3022
CA_BOOT_LOADER_LEN 3023
CA_BOOT_MAGIC 3023
CA_BOOT_MAGIC_MASK 3023
CA_BOOT_MAX_TRIES 3024
CA_BOOT_MSG 3024
CA_BOOT_MSG_UPDATE 3024
CA_BOOT_SLOT_A_BASE 3025
CA_BOOT_SLOT_B_BASE 3025
CA_BOOT_SLOT_LEN 3026
CA_BOOT_STATE 3026
CA_BOOT_SUPPORTED 3026
CA_BOOT_TRIAL_MS 3027
CA_CONFIG_HASH_LEN 3027
CA_DMA2D_AI 3028
CA_DMA2D_CLUT_CPU 3028
CA_DMA2D_ERR_ARG 3028
CA_DMA2D_ERR_CAE 3029
CA_DMA2D_ERR_CE 3029
CA_DMA2D_ERR_TE 3030
CA_DMA2D_ERR_TMO 3030
CA_DMA2D_HW_TC 3030
CA_DMA2D_OUT_FMTS 3031
CA_DMA2D_RBS 3031
CA_ETH_ERR_BUS 3031
CA_ETH_ERR_RBU 3032
CA_ETH_ERR_RPS 3032
CA_ETH_ERR_TPS 3033
CA_ETH_SEND_ERR_LINK 3033
CA_ETH_SEND_ERR_TX 3033
CA_ETH_SEND_OK 3034
CA_EVENT_BIT 3034
CA_EVENT_WAIT_ALL 3034
CA_EVENT_WAIT_ANY 3035
CA_FAULT_ASSERT 3035
CA_FAULT_CHECK 3036
CA_FAULT_EXEC_EMERGENCY 3036
CA_FAULT_RESTART_THREAD 3036
CA_FMAC_FIR_MAX 3037
CA_FMAC_IIR_MAX 3037
CA_FMAC_OVFL 3037
CA_FMAC_SAT 3038
CA_FMAC_UNFL 3038
CA_ICMP_PAYLOAD 3039

CA_IP4_IHL 3039
CA_IP4_PAYLOAD 3039
CA_JPEG_ERR_ARG 3040
CA_JPEG_ERR_HDR 3040
CA_JPEG_ERR_OVF 3040
CA_JPEG_ERR_TMO 3041
CA_JPEG_HW_EOC 3041
CA_JPEG_HW_HDR 3042
CA_JPEG_HW_IN_FREE 3042
CA_JPEG_HW_OUT_DATA 3042
CA_L2_PAYLOAD 3043
CA_L2_VLAN_PAYLOAD 3043
CA_LED_COUNT 3043
CA_LED_TABLE 3044
CA_MEMMAP_COUNT 3045
CA_MEMMAP_MAX 3046
CA_MEMMAP_TABLE 3046
CA_MSG_HEADER_SIZE 3047
CA_NOTIFY_ACTION_CLEAR_BITS 3047
CA_NOTIFY_ACTION_INCREMENT 3047
CA_NOTIFY_ACTION_NONE 3048
CA_NOTIFY_ACTION_OVERWRITE 3048
CA_NOTIFY_ACTION_SET_BITS 3049
CA_NOTIFY_ACTION_SET_IF_EMPTY 3049
CA_NO_WAIT_TIME 3049
CA_PI_MAX_CHAIN 3050
CA_PWR_WAKE_LPTIM 3050
CA_PWR_WAKE_LPUART 3051
CA_PWR_WAKE_PIN 3051
CA_PWR_WAKE_RTC 3051
CA_QSPI_DEV_NOR 3052
CA_QSPI_DEV_RAM 3052
CA_QSPI_DEV_RAW 3053
CA_QSPI_MMAP_BASE 3053
CA_QSPI_SSHIFT_AUTO 3053
CA_QSPI_SSHIFT_AUTO_HZ 3054
CA_QSPI_SSHIFT_OFF 3054
CA_QSPI_SSHIFT_ON 3055
CA_RTC_ALARM_A 3055
CA_RTC_ALARM_B 3055
CA_SAFEGUARD 3056
CA_SAFE_FAIL 3056
CA_SAI_ERR_AFS 3056
CA_SAI_ERR_LFS 3057
CA_SAI_ERR_OVR 3057
CA_SAI_ERR_UDR 3058
CA_SAI_SPDIF_C 3058
CA_SAI_SPDIF_U 3058
CA_SAI_SPDIF_V 3059
CA_SDIO_READ_TIMEOUT_MS 3059
CA_SDIO_WRITE_TIMEOUT_MS 3059

CA_SECRET_AREA_BASE 3060
CA_SECRET_AREA_SIZE 3060
CA_SECURE_PIPE_DECRYPT 3061
CA_SECURE_PIPE_ERR 3061
CA_SECURE_PIPE_MAX_MSG 3062
CA_SECURE_PIPE_NOT_UP 3062
CA_SECURE_PIPE_OK 3062
CA_SECURE_PIPE_REPLAY 3063
CA_SECURE_PIPE_REPLY 3063
CA_SECURE_PIPE_SIZE 3063
CA_SECURE_PIPE_TIMEOUT 3064
CA_SLICE_MIN_TICKS 3064
CA_SPDIFRX_C 3065
CA_SPDIFRX_CS_BYTES 3065
CA_SPDIFRX_ERR_FRM 3066
CA_SPDIFRX_ERR_OVR 3066
CA_SPDIFRX_ERR_PAR 3067
CA_SPDIFRX_ERR_SYNC 3067
CA_SPDIFRX_ERR_TMO 3067
CA_SPDIFRX_HW_SBD 3068
CA_SPDIFRX_HW_STOP 3068
CA_SPDIFRX_HW_SYNCD 3068
CA_SPDIFRX_PE 3069
CA_SPDIFRX_PT 3069
CA_SPDIFRX_PT_B 3070
CA_SPDIFRX_PT_M 3070
CA_SPDIFRX_PT_W 3070
CA_SPDIFRX_SAMPLE 3071
CA_SPDIFRX_U 3071
CA_SPDIFRX_V 3071
CA_SPINLOCK_COOP_INIT 3072
CA_SPINLOCK_IRQ_INIT 3072
CA_SPI_XFER_TIMEOUT_MS 3072
CA_SP_LINK_AAD_MAX 3073
CA_SP_LINK_DATA_HDR 3073
CA_SP_LINK_HELLO_SIZE 3074
CA_SP_LINK_OVERHEAD_MAX 3074
CA_SP_LINK_PBKDF2_ITER 3074
CA_SP_LINK_REKEY_MS 3075
CA_SP_LINK_REKEY_MSGS 3075
CA_SP_LINK_SECRET_MAX 3076
CA_SP_LINK_SECRET_MIN 3076
CA_ST_TH_BLOCKED 3076
CA_ST_TH_DIGEST_ERROR 3077
CA_ST_TH_INIT 3077
CA_ST_TH_REMOVE 3077
CA_ST_TH_RUN 3078
CA_ST_TH_RUNNING 3078
CA_ST_TH_SIG_WDT 3079
CA_ST_TH_SLEEP 3079
CA_ST_TH_STACK_OVERFLOW 3079

CA_ST_TH_STOP 3080
CA_ST_TH_SUSPENDED 3080
CA_ST_TH_UNKNOW 3080
CA_TASK_HANDLE_NONE 3081
CA_TCP_GONE 3081
CA_TCP_HDR_LEN 3082
CA_TCP_PAYLOAD 3082
CA_TCP_RETRY 3082
CA_TCP_SENT 3083
CA_TEMP_NO_DTS 3083
CA_TFTP_NO_ERROR 3083
CA_THID_API 3084
CA_TH_LEVEL0 3084
CA_TH_LEVEL1 3085
CA_TH_LEVEL2 3085
CA_TH_LEVEL3 3085
CA_TH_LEVEL4 3086
CA_TH_LEVEL5 3086
CA_TH_LEVEL6 3086
CA_TIMER_INVALID 3087
CA_TIMER_MAX 3087
CA_TIMER_TASK_PRIO 3088
CA_TIMER_TASK_STACK 3088
CA_UDP_MAX_PORTS 3088
CA_UDP_PAYLOAD 3089
CA_VLAN_PCP 3089
CA_VLAN_VID 3089
CA_WAIT_FOREVER 3090
CA_WDT_TIMEOUT_MIN_MS 3090
CH_CMD 3091
CH_ERASE 3091
CH_ESC 3091
CH_NONE 3092
CH_SUBMIT 3092
CH_TYPO 3092
CLI_HISTORY_SIZE 3093
CLI_PRINTF_N 3093
CLI_PUT_N 3093
CLI_QUEUE_DEPTH 3094
DEEP_GUARD_MAX 3094
DHCP_BIND_TIMEOUT_MS 3095
DHCP_CLIENT_PORT 3095
DHCP_DISCOVER_RETRY_N 3096
DHCP_MSG_ACK 3096
DHCP_MSG_DISCOVER 3097
DHCP_MSG_NAK 3097
DHCP_MSG_OFFER 3097
DHCP_MSG_REQUEST 3098
DHCP_OFFER_TIMEOUT_MS 3098
DHCP_RENEW_TIMEOUT_MS 3098
DHCP_REQUEST_TIMEOUT_MS 3099

DHCP_RETRY_BACKOFF_MS . . . 3099
DHCP_SERVER_PORT . . . 3100
DHCP_TASK_STACK_MIN . . . 3100
DMA_CH_INVALID . . . 3100
DMA_MAX_CHANNELS . . . 3101
DMA_REQ_NONE . . . 3101
DNS_CLASS_IN . . . 3101
DNS_FLAGS_QR . . . 3102
DNS_FLAGS_RCODE . . . 3102
DNS_FLAGS_RD . . . 3102
DNS_PENDING_MAX . . . 3103
DNS_PORT . . . 3103
DNS_RCODE_NXDOMAIN . . . 3104
DNS_RCODE_OK . . . 3104
DNS_RESPONSE_TIMEOUT_MS . . . 3104
DNS_RETRY_N . . . 3105
DNS_TYPE_A . . . 3105
DUMP_BUFFER_SIZE . . . 3105
DUMP_TASK_ADDR . . . 3106
DUMP_TASK_ALL . . . 3106
DUMP_TASK_HDR . . . 3107
DUMP_TASK_REG . . . 3107
DUMP_TASK_STACK . . . 3108
DUMP_TASK_STATS . . . 3108
ECDSA_P256_HASH_SIZE . . . 3108
ECDSA_P256_PRIVATE_SIZE . . . 3109
ECDSA_P256_PUBLIC_SIZE . . . 3109
ECDSA_P256_SIGNATURE_SIZE . . . 3109
ED25519_PUBLIC_SIZE . . . 3110
ED25519_SEED_SIZE . . . 3110
ED25519_SIGNATURE_SIZE . . . 3110
ENABLE_CLI . . . 3111
ENABLE_CLI_STDOUT . . . 3111
ENABLE_DBG_UART . . . 3111
ENABLE_ITM . . . 3112
ENABLE_SENSOR . . . 3113
ENABLE_STDOUT . . . 3113
ENABLE_UART . . . 3113
EN_ADC . . . 3114
EN_AES . . . 3114
EN_AES_CHANNEL_SERVICE . . . 3114
EN_AES_GCM_SW_TEST . . . 3115
EN_ARP_SERVICE . . . 3115
EN_BARRIER . . . 3116
EN_BIG_E . . . 3116
EN_BLOCK_POOL . . . 3116
EN_BOOT_IMAGE . . . 3117
EN_CAN . . . 3117
EN_CLI_ADC . . . 3117
EN_CLI_ARP . . . 3118
EN_CLI_CALLTO . . . 3118

EN_CLI_CAN 3119
EN_CLI_CAPTURE 3119
EN_CLI_CONFIG 3119
EN_CLI_CRC 3120
EN_CLI_DAC 3120
EN_CLI_DHCP 3120
EN_CLI_DMA 3121
EN_CLI_DNS 3121
EN_CLI_ECDSA_P256 3121
EN_CLI_ED25519 3122
EN_CLI_ETH 3122
EN_CLI_FAT 3122
EN_CLI_FLASH 3123
EN_CLI_GPIO 3123
EN_CLI_HTTP 3123
EN_CLI_HTTPS 3124
EN_CLI_I2C 3124
EN_CLI_IPERF 3125
EN_CLI_KSTORE 3125
EN_CLI_KSVC 3125
EN_CLI_LED 3126
EN_CLI_LPTIM 3126
EN_CLI_MDNS 3126
EN_CLI_MODBUS 3127
EN_CLI_MQTT 3127
EN_CLI_NET 3127
EN_CLI_NTP 3128
EN_CLI_PWR 3128
EN_CLI_QSPI 3128
EN_CLI_RNG 3129
EN_CLI_ROUTE 3129
EN_CLI_RTC 3129
EN_CLI_SDIO 3130
EN_CLI_SHA256 3130
EN_CLI_SHA512 3131
EN_CLI_SPI 3131
EN_CLI_SSH 3131
EN_CLI_SYSLOG 3132
EN_CLI_TCP 3132
EN_CLI_TFTP 3132
EN_CLI_TIMER 3133
EN_CLI_TRACE 3133
EN_CLI_UART 3133
EN_CLI_UDP_SRV 3134
EN_CLI_WDT 3134
EN_CLI_X25519 3134
EN_CRC 3135
EN_DAC 3135
EN_DHCP_SERVICE 3136
EN_DMA 3136
EN_DMA2D 3136

EN_DNS_SERVICE 3137
EN_ECDSA_P256 3137
EN_ECDSA_P256_TEST 3137
EN_ED25519 3138
EN_ED25519_TEST 3138
EN_ETH 3139
EN_ETH_CHK_HW 3139
EN_ETH_DBG 3139
EN_EVENT 3140
EN_EXTI 3140
EN_FAT 3140
EN_FAT_CRYPT 3141
EN_FAT_SERVICE 3141
EN_FLASH 3141
EN_FMAC 3142
EN_HASH 3142
EN_HTTPS_SERVICE 3142
EN_HTTP_SERVICE 3143
EN_I2C 3143
EN_IPERF_SERVICE 3143
EN_IP_DEFRAG 3144
EN_JPEG 3144
EN_KEYSTORE_SERVICE 3145
EN_KEYS_STORE 3145
EN_LED 3145
EN_LITTLE_E 3146
EN_LPTIM 3146
EN_LPUART 3146
EN_MDNS_SERVICE 3147
EN_MEASURE 3147
EN_MESSAGE 3147
EN_MODBUS_SERVICE 3148
EN_MQTT_SERVICE 3148
EN_NET 3149
EN_NET_DHCP 3149
EN_NET_DNS 3149
EN_NET_FILTER 3150
EN_NET_MDNS 3150
EN_NET_NTP 3150
EN_NET_SYSLOG 3151
EN_NOTIFY 3151
EN_NTP_SERVICE 3152
EN_PACKET_TRACER 3152
EN_POOL 3152
EN_PWR 3153
EN_PWR_DBG 3153
EN_QSPI 3153
EN_RNG 3154
EN_ROUTE_SERVICE 3154
EN_RTC 3154
EN_RWLOCK 3155

EN_SAI 3155
EN_SDIO 3156
EN_SHA256 3156
EN_SHA256_TEST 3156
EN_SHA512 3157
EN_SHA512_TEST 3157
EN_SPDIFRX 3157
EN_SPI 3158
EN_SPI_DMA 3158
EN_SSH_SERVICE 3158
EN_SSH_TEST 3159
EN_SSH_TRACE 3159
EN_STLINK_WAIT_READY 3159
EN_STREAM 3160
EN_SW_TIMER 3160
EN_SYSLOG_SERVICE 3161
EN_TAMP 3161
EN_TCP_SERVICE 3161
EN_TCP_TRACE 3162
EN_TFTP_SERVICE 3162
EN_TIMER 3162
EN_TLS 3163
EN_UART_DMA 3163
EN_UDP_CLIENT 3163
EN_UDP_SERVER 3164
EN_USB 3164
EN_USB_CDC 3164
EN_USB_CUSTOM 3165
EN_USB_DBG 3165
EN_USB_DFU 3166
EN_USB_FS 3166
EN_USB_HS 3166
EN_USB_MSC 3167
EN_USB_NCM 3167
EN_WDT 3167
EN_X25519 3168
EN_X25519_TEST 3168
ETH_MPU_REGION 3169
ETH_P_ARP 3169
ETH_P_IP 3170
ETH_P_IPV6 3170
ETH_P_VLAN 3170
FAT_CRYPT_CHUNK 3171
HASH_CTX_INIT_CAP 3171
HASH_MAX_DIGEST_SIZE 3171
HASH_SHA1_DIGEST_SIZE 3172
HASH_SHA256_DIGEST_SIZE 3172
HCLK_FREQ_HZ 3172
HKDF_SHA256_MAX_OUT 3173
HMAC_SHA256_SIZE 3173
HTTPS_DEFAULT_PORT 3174

HTTP_DEFAULT_PORT 3174
HTTP_METHOD_GET 3175
HTTP_METHOD_HEAD 3175
HTTP_METHOD_OPTIONS 3175
HTTP_METHOD_POST 3176
HTTP_QUERY_MAX 3176
HTTP_RECV_TIMEOUT_MS 3176
HTTP_REQ_BUF_SIZE 3177
HTTP_RESP_BUF_SIZE 3177
HTTP_URI_MAX 3178
I2C_EN_IRQ 3178
IPERF_DEFAULT_DURATION_MS 3178
IPERF_DEFAULT_PORT 3179
IPERF_IDLE_TIMEOUT_MS 3179
IP_DEFRAG_MAX_PAYLOAD 3179
IP_DEFRAG_MAX_SESSIONS 3180
IP_DEFRAG_TIMEOUT_MS 3180
IP_FLAG_DF 3181
IP_FLAG_MF 3181
IP_FRAG_OFF_MASK 3181
IP_PROTO_ICMP 3182
IP_PROTO_TCP 3182
IP_PROTO_UDP 3182
IP_TX_MTU_DEFAULT 3183
ITM_BAUDRATE 3183
KSTORE_DATE_DAY 3183
KSTORE_DATE_MONTH 3184
KSTORE_DATE_PACK 3184
KSTORE_DATE_YEAR 3185
KSTORE_SVC_IV_SIZE 3185
KSTORE_SVC_KEY_SIZE 3185
KSTORE_SVC_LABEL_SIZE 3186
KSTORE_SVC_NONCE_SIZE 3186
KSTORE_SVC_P1_SIZE 3186
KSTORE_SVC_P2_SIZE 3187
KSTORE_SVC_TAG_SIZE 3187
LPUART_STD_BAUD 3188
LPUART_STD_WAKE 3188
MAX 3188
MAX_SECURE_KEYS 3189
MAX_TASK 3189
MAX_TIMER 3190
MAX_TOTAL_KEY_STORAGE 3190
MCU_HAS_ETH 3191
MCU_HAS_GPDMA 3192
MCU_HAS_QSPI 3192
MCU_HAS_RNG 3193
MCU_HAS_USB 3194
MCU_NO_ETH 3194
MDNS_CACHE_FLUSH 3195
MDNS_DST_IP 3195

MDNS_DST_MAC 3196
MDNS_PORT 3196
MDNS_TTL 3196
MIN 3197
MIN_KEY_SIZE 3197
MODBUS_RX_BUF_SIZE 3197
MODBUS_TIMEOUT_MS 3198
MPU_ACCESS_NONE 3198
MPU_ACCESS_RDONLY 3199
MPU_ACCESS_RDWR 3199
MPU_CANARY_REGION_MSP 3199
MPU_CANARY_REGION_PSP_END 3200
MPU_CANARY_REGION_PSP_START 3200
MPU_REGION_COUNT 3200
MQTT_KEEPALIVE_SEC 3201
MQTT_RX_BUF_SIZE 3201
NET_BUF_COUNT 3202
NET_BUF_DATA_SIZE 3203
NET_BUF_F_IN_USE 3203
NET_BUF_F_RAW 3203
NET_BUF_F_TX 3204
NET_BUF_HDR_SIZE 3204
NET_BUF_HEADROOM 3204
NET_BUF_INVALID 3205
NET_BUF_SIZE 3205
NET_HANDLER_MAX 3206
NET_IFACE_INVALID 3206
NET_IFACE_MAX 3206
NET_MAC_FILTER_MAX 3207
NET_MPU_REGION 3207
NET_PROTO_ARP 3207
NET_PROTO_IP4 3208
NET_PROTO_IP6 3208
NET_PROTO_NONE 3209
NET_QUEUE_DEPTH 3209
NET_ROUTE_MAX 3209
NTP_EPOCH_DELTA 3210
NTP_POLL_MS 3210
NTP_PORT 3210
NTP_RESPONSE_TIMEOUT_MS 3211
NTP_RETRY_BACKOFF_MS 3211
NTP_RETRY_N 3212
NTP_SERVER_N 3212
OS_TICK_US 3212
PACKET_TRACER_RING_SIZE 3213
PCLK1_FREQ_HZ 3213
PCLK2_FREQ_HZ 3214
PCLK3_FREQ_HZ 3215
PCLK4_FREQ_HZ 3215
POOL_QUEUE_SIZE 3216
PRINTF 3216

PRINTF_N 3216
PSP_CHECK_TOP 3217
PUT 3217
PUT_N 3218
RO_MPU_REGION 3218
SHA256_BLOCK_SIZE 3218
SHA256_DIGEST_SIZE 3219
SHA512_BLOCK_SIZE 3219
SHA512_DIGEST_SIZE 3219
SPINLOCK_FREE 3220
SPINLOCK_TAKEN 3220
SPI_DEF_PKT_SIZE 3221
SSH_LISTEN_PORT 3221
STACK_CANARY_SIZE 3221
STACK_CANARY_VALUE 3222
STACK_FILL_PATTERN 3222
STACK_MIN 3222
STDOUT_ON_ITM 3223
STDOUT_ON_LPUART 3223
STDOUT_ON_UART 3224
STDOUT_ON_USB_CDC 3224
STDOUT_QUEUE_DEPTH 3225
SVC_OS_END 3225
SVC_SAFEGUARD 3225
SYSCLK_FREQ_HZ 3226
SYSLOG_FAC_KERN 3226
SYSLOG_FAC_LOCAL0 3227
SYSLOG_FAC_LOCAL7 3227
SYSLOG_FAC_USER 3227
SYSLOG_MSG_MAX 3228
SYSLOG_PORT 3228
SYSLOG_QUEUE_SIZE 3229
SYSLOG_SEV_ALERT 3229
SYSLOG_SEV_CRIT 3229
SYSLOG_SEV_DEBUG 3230
SYSLOG_SEV_EMERG 3230
SYSLOG_SEV_ERR 3230
SYSLOG_SEV_INFO 3231
SYSLOG_SEV_NOTICE 3231
SYSLOG_SEV_WARN 3231
SYSTICK_ERROR 3232
SYSTICK_OK 3232
TASK_CANARY_VALUE_SAFE 3233
TASK_CANARY_VALUE_SAFE_FP 3233
TASK_CANARY_VALUE_STD 3233
TASK_CANARY_VALUE_STD_FP 3234
TASK_NAME_LEN 3234
TCP_ARP_TIMEOUT_MS 3234
TCP_CLOSE_WAIT_MS 3235
TCP_CONNECT_TIMEOUT_MS 3235
TCP_FLAG_ACK 3236

TCP_FLAG_FIN 3236
TCP_FLAG_PSH 3236
TCP_FLAG_RST 3237
TCP_FLAG_SYN 3237
TCP_FLAG_URG 3237
TCP_MAX_CONN 3238
TCP_MAX_INFLIGHT 3238
TCP_MAX_LISTEN 3239
TCP_MAX_RETRIES 3239
TCP_MSS_DEFAULT 3239
TCP_RTO_INITIAL_MS 3240
TCP_RTO_MAX_MS 3240
TCP_RX_RING_DEPTH 3241
TCP_TCP_ACCEPT_TIMEOUT_MS 3241
TCP_TIMER_TICK_MS 3241
TCP_TIME_WAIT_MS 3242
TFTP_BLOCK_SIZE 3242
TFTP_BLOCK_TIMEOUT_MS 3242
TFTP_ERR_ACCESS 3243
TFTP_ERR_DISKFULL 3243
TFTP_ERR_EXISTS 3243
TFTP_ERR_ILLEGAL 3244
TFTP_ERR_NOTFOUND 3244
TFTP_ERR_NOUSER 3245
TFTP_ERR_TID 3245
TFTP_ERR_UNDEF 3245
TFTP_MAX_XFER 3246
TFTP_PORT 3246
TFTP_RETRY_N 3246
TLS_HS_BUF_SIZE 3247
TLS_MAX_PLAINTEXT 3247
TLS_READ_CLOSED 3247
TLS_READ_ERROR 3248
TLS_READ_TIMEOUT 3248
TLS_RX_BUF_SIZE 3249
TLS_TX_BUF_SIZE 3249
TLS_TX_RECORD 3249
UART_DBG_BAUDRATE 3250
UART_DBG_NUM 3250
UART_DBG_PINMAP 3251
UART_STDOUT_NUM 3251
UDPCLI_MAX_HANDLES 3252
UDPCLI_RING_DEPTH 3252
UDPSRV_MAX_CLIENTS 3252
UDPSRV_RING_DEPTH 3253
UDPSRV_WORKER_STACK_MIN 3253
UNUSED 3253
USB_BCD_DEVICE 3254
USB_CDC_EP_DATA 3254
USB_CDC_EP_NOTIF 3255
USB_CDC_FIFO_WORDS_DATA 3256

USB_CDC_FIFO_WORDS_NOTIF . . . 3256
USB_DFU_ATTR_CAN_DNLOAD . . . 3256
USB_DFU_ATTR_CAN_UPLOAD . . . 3257
USB_DFU_ATTR_MANIFEST_TOL . . . 3257
USB_DFU_ATTR_WILL_DETACH . . . 3258
USB_DFU_STR_IFACE . . . 3258
USB_DFU_XFER_MAX . . . 3258
USB_EP0_MPS . . . 3259
USB_EP0_TX_BUF . . . 3259
USB_EP0_XFR_MAX . . . 3260
USB_EPBIT . . . 3260
USB_EPMASK_CDC . . . 3261
USB_EPMASK_MSC . . . 3261
USB_EPMASK_NCM . . . 3262
USB_EP_MAX . . . 3262
USB_MPU_REGION . . . 3263
USB_MSC_EP_DATA . . . 3263
USB_MSC_FIFO_WORDS_DATA . . . 3264
USB_NCM_EP_DATA . . . 3264
USB_NCM_EP_NOTIF . . . 3265
USB_NCM_FIFO_WORDS_DATA . . . 3266
USB_NCM_FIFO_WORDS_NOTIF . . . 3266
USB_NCM_MAX_FRAME . . . 3267
USB_PID_BASE_CDC . . . 3267
USB_PID_BASE_CUSTOM . . . 3267
USB_PID_BASE_DFU . . . 3268
USB_PID_BASE_MSC . . . 3268
USB_PID_BASE_NCM . . . 3269
USB_STR_MANUFACTURER . . . 3269
USB_STR_PRODUCT . . . 3269
USB_STR_SERIAL . . . 3270
USB_VID . . . 3270
X25519_KEY_SIZE . . . 3270
_SAFEGUARD_DISPATCH . . . 3271
_SAFEGUARD_DISPATCH_ . . . 3271
_SAFEGUARD_NARG . . . 3272
_SAFEGUARD_NARG_ . . . 3272
_SAFEGUARD_RSEQ . . . 3272
__disable_irq . . . 3273
__enable_irq . . . 3273
ca_be16toh . . . 3273
ca_be32toh . . . 3274
ca_be64toh . . . 3274
ca_cli_char_class . . . 3275
ca_cli_exec_capture . . . 3275
ca_cli_exec_capture_stream . . . 3275
ca_cli_filtered_count . . . 3276
ca_cli_hist_count . . . 3276
ca_cli_hist_get . . . 3276
ca_cli_typo_count . . . 3277
ca_htobe16 . . . 3277

ca_htobe32 . . . 3278
ca_htobe64 . . . 3278
ca_htole16 . . . 3278
ca_htole32 . . . 3279
ca_htole64 . . . 3279
ca_htonl . . . 3279
ca_htonl_signed . . . 3280
ca_htonll . . . 3280
ca_htonll_signed . . . 3280
ca_htons . . . 3281
ca_htons_signed . . . 3281
ca_le16toh . . . 3281
ca_le32toh . . . 3282
ca_le64toh . . . 3282
ca_mpu_add_region . . . 3283
ca_mpu_first_free_region . . . 3283
ca_mpu_get_available_regions . . . 3283
ca_mpu_get_busy_regions . . . 3284
ca_mpu_get_system_regions . . . 3284
ca_mpu_get_total_regions . . . 3284
ca_mpu_region_covers . . . 3285
ca_mpu_update_region . . . 3285
ca_ntohl . . . 3286
ca_ntohl_signed . . . 3286
ca_ntohll . . . 3286
ca_ntohll_signed . . . 3287
ca_ntohs . . . 3287
ca_ntohs_signed . . . 3287
ca_stdout_create_thread . . . 3288
ca_stdout_echo . . . 3288
ca_stdout_echo_dropped . . . 3289
ca_stdout_get_repeater . . . 3289
ca_stdout_printf . . . 3289
ca_stdout_printf_n . . . 3290
ca_stdout_put . . . 3290
ca_stdout_put_n . . . 3290
ca_stdout_put_str . . . 3291
ca_stdout_put_str_n . . . 3291
ca_usb_dbg_print . . . 3291
ca_usb_dbg_reset . . . 3292
ca_usb_dbg_stat . . . 3292
ca_usb_dev_mng_add . . . 3293
ca_usb_dev_mng_init . . . 3293
ca_usb_dev_mng_remove . . . 3293
ca_usb_dev_mng_start . . . 3294
ca_usb_dev_mng_stop . . . 3294
ca_usb_ep_clear_stall . . . 3294
ca_usb_ep_close . . . 3295
ca_usb_ep_is_stalled . . . 3295
ca_usb_ep_open . . . 3296
ca_usb_ep_read . . . 3296

ca_usb_ep_write . . . 3296
ca_usb_get_speed . . . 3297
ca_usb_init . . . 3297
ms_to_ticks . . . 3297
seconds_to_ticks . . . 3298

118 Call Graph API caOS 3299

118.1 adc . . . 3300
118.2 aes . . . 3302
118.3 aes_channel_service . . . 3304
118.4 arp_service . . . 3305
118.5 atomic . . . 3307
118.6 bkp . . . 3308
118.7 block_pool . . . 3310
118.8 boot_image . . . 3311
118.9 bus . . . 3312
118.10 ca_glue_hw . . . 3313
118.11 callto . . . 3314
118.12 can . . . 3315
118.13 caos . . . 3316
118.14 cli . . . 3317
118.15 clock . . . 3320
118.16 cordic . . . 3322
118.17 crc32 . . . 3323
118.18 dac . . . 3324
118.19 dhcp_service . . . 3326
118.20 dma . . . 3328
118.21 dma2d . . . 3331
118.22 dns_service . . . 3333
118.23 ecdsa_p256 . . . 3334
118.24 ed25519 . . . 3335
118.25 endian . . . 3336
118.26 eth . . . 3337
118.27 eth_iface . . . 3340
118.28 event . . . 3341
118.29 exti . . . 3342
118.30 fat_service . . . 3344
118.31 flash . . . 3347
118.32 fmac . . . 3350
118.33 gpio_driver . . . 3352
118.34 hash . . . 3353
118.35 http_service . . . 3355
118.36 https_service . . . 3357
118.37 i2c . . . 3358
118.38 i2c_recover . . . 3360
118.39 ip_defrag . . . 3361
118.40 iperf_service . . . 3362
118.41 itm . . . 3363
118.42 jpeg . . . 3364
118.43 kdf . . . 3366
118.44 keys_store . . . 3367
118.45 keystore_service . . . 3369

118.46 led . . . 3370
118.47 lptim . . . 3372
118.48 lpuart . . . 3374
118.49 mdns_service . . . 3376
118.50 measure . . . 3377
118.51 memaux . . . 3378
118.52 memmap . . . 3380
118.53 memory . . . 3381
118.54 message . . . 3384
118.55 modbus_service . . . 3386
118.56 mpu . . . 3387
118.57 mqtt_service . . . 3389
118.58 ncm_iface . . . 3390
118.59 net . . . 3391
118.60 net_buf . . . 3394
118.61 net_proto . . . 3396
118.62 net_proto_tcp . . . 3399
118.63 net_queue . . . 3400
118.64 net_utils . . . 3401
118.65 notify . . . 3402
118.66 ntp_service . . . 3403
118.67 opamp . . . 3404
118.68 packet_tracer_service . . . 3405
118.69 pool . . . 3406
118.70 pwr . . . 3407
118.71 qspi . . . 3409
118.72 queue . . . 3411
118.73 rng . . . 3415
118.74 route_service . . . 3416
118.75 rtc . . . 3417
118.76 safeguard . . . 3420
118.77 sai . . . 3421
118.78 scheduler . . . 3423
118.79 sdio . . . 3429
118.80 secure_pipe_service . . . 3430
118.81 sha256 . . . 3432
118.82 sha512 . . . 3433
118.83 spdifrx . . . 3434
118.84 spi . . . 3436
118.85 ssh_bpp . . . 3439
118.86 ssh_transport . . . 3440
118.87 stdout . . . 3441
118.88 stream . . . 3443
118.89 sw_timer . . . 3445
118.90 sync . . . 3446
118.91 syslog_service . . . 3451
118.92 systick . . . 3452
118.93 tcp_service . . . 3455
118.94 temp_sensor . . . 3457
118.95 tftp_service . . . 3459
118.96 timer . . . 3460

118.97 tls 3464
118.98 uart 3465
118.99 udp_client 3468
118.100 udp_server 3469
118.101 usb 3470
118.102 usb_cdc 3472
118.103 usb_dfu 3473
118.104 usb_fs 3474
118.105 usb_hs 3476
118.106 usb_mgmt 3478
118.107 usb_ncm 3479
118.108 utils 3480
118.109 wdt 3482
118.110 x25519 3484

Chapter 1

Introduction: Bare Metal and RTOS

1.1 Chapter 1 - Introduction: Bare Metal and RTOS

1.1.1 1.1 What are embedded systems?

Imagine you want to control an electronic device such as a smart thermostat, a car, or a robot. An **embedded system** is a specialized computer designed to perform a specific function within a larger device. Unlike a regular computer, an embedded microcontroller (MCU) has very limited resources: a few kilobytes of RAM, relatively low processing speed, and no operating system like Windows or Linux.

When you turn on a microcontroller, it does not happen like when you turn on your computer. There is no operating system that boots up slowly. The microcontroller immediately starts executing your program, directly from the hardware. This is both an advantage (speed) and a challenge (you have to manage everything yourself).

1.1.2 1.2 "Bare Metal" Programming

What is Bare Metal?

Bare Metal means programming directly on the hardware, without any operating system. It is like driving a car without an onboard computer: you have full control but you must manage everything personally.

The typical structure of a bare metal program is an **infinite loop**:

```
int main(void) {
    // Inizializzazione del microcontrollore
    init_clock();     // Imposta la velocit  del processore
    init_gpio();      // Configura i pin di ingresso/uscita
    init_uart();      // Configura la comunicazione seriale

    // Loop che si ripete all'infinito
    while (1) {
        leggi_sensore();
        elabora_dati();
        aggiorna_uscite();
        gestisci_comunicazione();
    }
}
```

Everything happens in sequence, one operation after another. If `leggi_sensore()` takes 10 milliseconds, nothing else is executed during those 10 ms.

Advantages of Bare Metal

- **Simplicity**: the program is linear and easy to understand.

- **Efficiency**: no extra code slowing down the system.
- **Total control**: you know exactly what your program is doing.
- **Low memory consumption**: all space is used for your code.

Limitations of Bare Metal

When your project becomes more complex, the infinite loop shows its limitations:

Problem 1 - Response time: if the loop takes 50 ms to complete a cycle, a critical event might have to wait up to 50 ms before being handled.

Problem 2 - No priority: in the loop, all operations have the same importance. You cannot say "this sensor reading is urgent, the display update can wait".

Problem 3 - System blocking: if a function has to wait for data (for example, a response from a sensor), it blocks the entire system.

Problem 4 - Difficult to expand: adding new features means modifying the main loop, with the risk of breaking what is already working.

Interrupts: a first solution

Interrupts partially solve the response time problem. When something important happens (for example, you press a button or data arrives), the microcontroller temporarily interrupts the main loop to handle it immediately:

```
// Questa funzione viene eseguita immediatamente quando arriva un dato
void USART1_IRQHandler(void) {
    uint8_t byte = USART1->DR;  // Legge il dato ricevuto
    buffer[idx++] = byte;       // Lo salva in un buffer
}

// Il loop principale continua normalmente
while (1) {
    if (idx > 0) {
        elabora_buffer();
    }
    aggiorna_display();
}
```

Interrupts are very useful, but they have limitations: - They must be short, otherwise they block other functions. - They can create data concurrency issues if not handled correctly. - They do not solve the problem of managing different activities with different priorities.

1.1.3 1.3 What is an RTOS?

What is an RTOS?

An **RTOS** (Real-Time Operating System) is a lightweight operating system designed to manage multiple activities simultaneously. Unlike Windows or macOS, an RTOS is:

- **Very small**: it occupies a few kilobytes of memory.
- **Punctual**: it responds to events within guaranteed times.
- **Capable of preempting**: it can stop an activity to perform a more urgent one.

The concept of Task

The heart of an RTOS is the **task** (or process). Instead of a single loop, the work is divided into multiple independent tasks, each with its own job:

```c
// Task 1: lettura sensore (importante)
void task_sensore(void) {
    while (1) {
        leggi_sensore();
        elabora_dati();
        sleep(100);  // Attende 100 ms, permettendo ad altri task di lavorare
    }
}

// Task 2: aggiornamento display (meno importante)
void task_display(void) {
    while (1) {
        aggiorna_schermo();
        sleep(500);  // Aggiorna ogni 500 ms
    }
}

// Task 3: comunicazione (media importanza)
void task_comunicazione(void) {
    while (1) {
        if (dati_disponibili()) {
            invia_dati();
        }
        sleep(50);
    }
}
```

Each task operates as if it had the microcontroller all to itself. It is the RTOS **scheduler** that decides which task should run at any given moment, switching rapidly from one to another.

Advantages of an RTOS

- **Organization**: each task is a separate module, easier to develop and debug.
- **Priority**: important tasks are executed before less important ones.
- **Responsiveness**: an important event immediately interrupts the current task.
- **Efficiency**: when a task is waiting for something, other tasks can work.
- **Safe communication**: specific tools (queues, semaphores) allow tasks to exchange information safely.

Disadvantages of an RTOS

Everything has a cost. An RTOS introduces:

- **Memory consumption**: each task requires its own personal memory area (stack).
- **Context switch**: switching from one task to another takes some CPU time.
- **Complexity**: errors like race conditions are harder to find when there are multiple simultaneous activities.
- **Sizing**: you must accurately decide how much memory to allocate to each task.

1.1.4 1.4 When to use Bare Metal or RTOS?

The choice depends exclusively on the needs of your project:

Use Bare Metal when:

- The system does only one thing (or a few simple things).
- It is not critical to respond instantly to events.
- You have very little memory available on the MCU.
- The project is stable and will not become very complex.

Use an RTOS when:

- There are many independent and different activities to manage.
- You need to respond quickly and deterministically to external events.
- The project will grow and receive many new features.
- You need to communicate in a complex way between different subsystems.
- You want to facilitate porting code to different microcontrollers.

Hybrid approach

In practice, many systems use a mixed approach: a main loop with interrupts for simple cases, and an RTOS when complexity increases. The advantage of a good RTOS is that your code remains linear, while the complexity of simultaneous management is delegated to the operating system.

1.1.5 1.5 RTOS on ARM Cortex-M microcontrollers

ARM Cortex-M microcontrollers are particularly suitable for running an RTOS thanks to dedicated hardware features:

Two separate work areas

The Cortex-M has two separate stack pointers: - **MSP** (Main Stack Pointer): used by the system (kernel) and interrupt service routines (ISR). - **PSP** (Process Stack Pointer): used by individual user tasks.

This separation protects the system: if a task crashes or runs out of stack, it does not damage the kernel's central stack.

Special functions for multitasking

- **SysTick**: a hardware timer integrated into the core that generates the time interrupt (tick) for the scheduler.
- **PendSV**: a low-priority software interrupt used specifically to perform the context switch without interfering with urgent hardware interrupts.
- **SVC** (Supervisor Call): an instruction that allows tasks to call system functions in privileged mode.

Memory protection

In more advanced models (Cortex-M4, M7, M23, M33), there is a **Memory Protection Unit (MPU)** that allows you to: - Isolate a task's memory to prevent it from overwriting that of other tasks. - Protect kernel code from application errors. - Define restricted-access areas (e.g., read-only).

System security

The Cortex-M supports two code execution levels: - **Privileged**: full access to all registers and peripherals (used by the kernel). - **Unprivileged**: limited access, controlled by the system (used by tasks).

This separation makes the entire firmware extremely robust and secure.

1.1.6 1.6 Available RTOS

There are several established RTOS in the microcontroller world:

RTOS	License	Features
FreeRTOS	Free (MIT)	The most widely used, supported by Amazon.
Zephyr	Free	Linux Foundation project, very modular and rich in network stacks.
RTEMS	Free	Highly reliable, used in space and avionics (NASA, ESA).
ChibiOS	Free/commercial	Extremely efficient, natively optimized for STM32.
ThreadX (Azure RTOS)	Free (Eclipse)	Certified for safety-critical environments, now open source.
caOS	Free	The subject of this manual.

Each RTOS has its own strengths. In the following chapters, we will see the fundamental concepts common to all real-time architectures and how caOS implements them specifically.

Chapter 2

The Kernel: Tasks, Scheduler and Stack Management

2.1 Chapter 2 - The Kernel: Tasks, Scheduling, and Memory Management

The kernel is the heart of every RTOS. It is the software component that manages task execution, decides who works and when, and handles saving and restoring the state of each task during context switches. In this chapter, we examine the fundamental concepts that every RTOS must implement.

2.1.1 2.1 The Task

What is a Task

A task (also called a process in some RTOSs) is an independent unit of work. From the programmer's perspective, each task is a function with its own infinite loop:

```
void task_example(void) {
    // Initialization (run only once)
    configure_peripheral();

    // Main loop
    while (1) {
        do_work();
        event_wait();
    }
}
```

From the kernel's perspective, each task is like a small program with its own information:

- **State**: running, waiting, dormant, suspended
- **Workspace**: where the task saves its temporary data
- **Priority**: importance of the task (higher or lower)
- **Name**: identifier to understand which task it is
- **Context**: information about the processor state

Task States

A task can be in different states during its lifetime:

```
                    +----------+
                    |  CREATO  |
                    +----+-----+
```

```
                          |
                          v
   +---------+     +-----------+     +----------+
   | BLOCKED|<--->|   READY   |<--->|  RUNNING  |
   +---------+     +-----------+     +----------+
        |                |                |
        |                v                |
        |          +-----------+          |
        +--------->| SOSPESO   |<---------+
                   +-----------+
                          |
                          v
                   +-----------+
                   |    END    |
                   +-----------+
```

- **Created**: the task has been created but not yet started
- **Ready**: the task is ready to work, waiting for its turn
- **Running**: the task is working (only one at a time)
- **Blocked**: the task is waiting for an event (time, communication, etc.)
- **Suspended**: the task has been manually stopped, it will not work
- **Finished**: the task has finished or has been removed

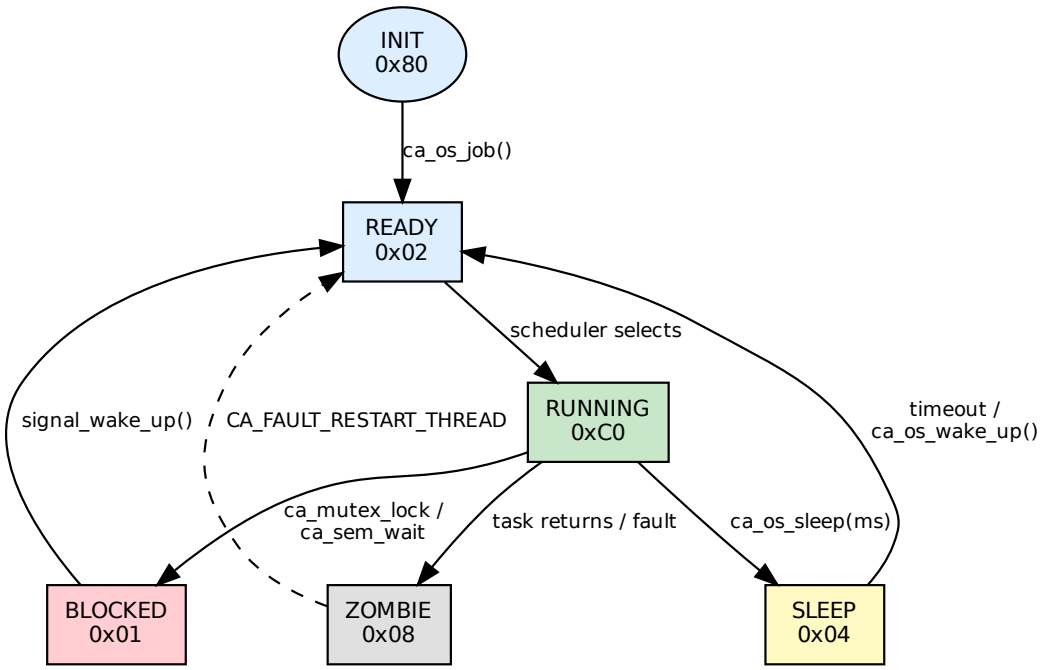


Figure 2.1: caOS task states

Creating a Task

When creating a task, the kernel must:

1. **Create a container** to store the task information
2. **Assign a workspace** (stack) dedicated to the task
3. **Prepare the workspace** so that when the task starts, it executes its function
4. **Add the task to the list** of tasks ready to work

The workspace (stack) is important: when the task starts working, the system prepares the area as if the task had just been interrupted, so when it resumes, it starts exactly from its function.

Idle task

Every RTOS has a special task called the **idle task**: a very low priority task that works only when no other task is ready. The idle task:

- Puts the processor into power-saving mode
- Performs system maintenance operations
- Must never block or stop

2.1.2 2.2 The Scheduler

What is the Scheduler

The scheduler is like an orchestra conductor: it decides which task must work at any given moment. It is called:

- At regular intervals (like a metronome)
- When a task stops voluntarily (to sleep or wait)
- When a new task is ready
- When a task voluntarily yields its place

Scheduling algorithms

Round-Robin The simplest algorithm: each task receives the same amount of working time. When the time expires, the task yields its place to the next one.

```
Time -->
Task A: [====]       [====]       [====]       ...
Task B:       [====]       [====]       [====] ...
Task C:             [====]             [====] ...
         t0   t1    t2    t3    t4    t5
```

Advantages: all tasks receive the same treatment **Disadvantages**: an important task must wait for its turn like all the others

Priority-Based Each task has a priority. The scheduler always runs the most important task among those that are ready. If a very important task arrives while a less important one is running, the former interrupts the latter.

```
Time -->
Task A (high):  [==]   [========]         [===]
Task B (media):     [==]          [======]
Task C (low):                                  [====...]
```

Advantages: important tasks are served immediately **Disadvantages**: less important tasks might never run if the important ones are always active

By priority with sharing A more advanced version: important tasks receive more time, but everyone receives at least some time.

```
CPU time assigned:
  High priority:                                                    (much time)
  Medium priority:                                  (medium time)
  Low priority:                           (little time)
```

Advantages: everyone receives time, but the most important ones receive more **Disadvantages**: more complex to manage

The system heartbeat

The scheduler needs a "heartbeat" to function: a timer that ticks at regular intervals (usually every millisecond).

At each tick, the scheduler: 1. Checks if any task needs to wake up 2. Checks time limits 3. Decides if it is time to switch tasks

Choosing the frequency is a compromise:

Frequency	Advantages	Disadvantages
Fast	Very fast responses	System busy checking
Slow	Little checking work	Less precise responses

2.1.3 2.3 The Context Switch

What is the Context Switch

The context switch is the operation by which the system saves the state of the current task and resumes the state of the next task to be executed. It is the most important operation of the entire RTOS: it must be fast and correct.

What is saved

When switching from one task to another, the system must remember:

- Where the previous task was working
- What data it was processing
- What information it had in memory

How the switch works

The switch does not happen immediately. The system schedules the switch so that it occurs when it is safe to do so, that is, when all important operations are finished.

```
System check:
  1. Checks who must run
  2. If it differs from the current one, schedules the switch
  3. At the end of the important operations:
     - Saves the state of the current task
     - Loads the state of the next task
     - Continues with the new task
```

This two-phase system ensures that the switch does not interfere with critical operations.

Context switch time

On a modern microcontroller, the context switch requires very little time (less than a millisecond). This includes: - Saving and restoring processor information - Updating task lists - Switching to the new task

2.1.4 2.4 Working Memory Management (Stack)

Working memory management (called stack) is very important in an RTOS. Errors here cause problems that are difficult to understand.

A working area for each Task

Each task has its own personal working area. Unlike a normal computer (where memory can grow), on a microcontroller the working area is fixed, assigned when the task is created.

```
Memory

      System area              For the kernel and the important operations
      (4 KB)

      Task 1 area              Where Task 1 works
      (1 KB)

      Task 2 area              Where Task 2 works
      (1 KB)

      Task 3 area              Where Task 3 works
      (512 B)

         ...

      Free memory
      (variabile)

```

How much memory is needed?

How much workspace does a task need? It depends on:

- **How deep the work is**: every function that calls other functions uses memory
- **How much temporary data it uses**: a large array uses a lot of memory
- **If interrupts occur**: when an event arrives, it uses some extra memory
- **Complex functions**: some system functions can use a lot of memory

The risk is **stack overflow**: when a task uses more memory than allocated and starts overwriting other data. The symptoms are unpredictable: other tasks behave strangely, the system crashes at unexpected points.

How to protect memory

Software check A special value is placed at the end of the workspace. At regular intervals, the system checks that the value is still there:

```
Task 1 work area:
                                                  area start
      ... data ...          memory fills from the bottom up

      ... data ...

     SPECIAL VALUE           if found changed, there was an overflow!
                                                  area end
      Other task            the next task's area starts here

```

Advantages: simple to implement **Disadvantages**: discovers the problem after it has happened

Hardware control Some microcontrollers have a special feature that protects memory areas. If a task tries to write outside its area, the system notices immediately.

Advantages: immediate discovery of the problem **Disadvantages**: not all microcontrollers have this feature

Usage monitoring At the beginning, the entire workspace is filled with a special value. During operation, you can see how much of the area has been used by looking at how many special values have been overwritten.

This technique is useful for understanding how much area each task needs.

Memory tips

1. **Do not use large temporary areas**: use fixed or shared areas
2. **Avoid recursive functions**: they can use a lot of memory
3. **Calculate needs carefully**: account for the worst-case scenario
4. **Use hardware protection if available**: it is the safest
5. **Check usage during development**: see how much the tasks use

2.1.5 2.5 The System Timer

How the rhythm works

Most RTOS use a fixed rhythm (e.g., every millisecond). Some advanced systems use a tickless approach: the system wakes up only when it needs to do something, saving energy.

For an RTOS on a microcontroller, the choice of rhythm is a compromise:

- **Fast rhythm**: very precise responses, but the system checks frequently
- **Slow rhythm**: fewer checks, but pauses might be less precise

Converting times

System functions use times in milliseconds because they are easy to understand:

```
os_sleep(100);  // The task wants to sleep 100 milliseconds
```

But inside the system, everything works with the rhythm:

```
// If the tick is every 500 microseconds, then 100ms = 200 ticks
task->pause_counter = ms_to_ticks(100);  // = 200
```

At each tick, the system decrements the counter:

```
// In the periodic check:
for each sleeping task:
    task->pause_counter--;
    if (task->pause_counter == 0)
        wake_up(task);
```

2.1.6 2.6 System startup

The startup sequence of an RTOS follows these steps:

1. **Power-on**: the microcontroller starts from the entry point
2. **Preparation**: copies necessary data, clears memory
3. **main()**: initializes hardware and the system
4. **Task creation**: the program creates all initial tasks
5. **System startup**: the kernel starts running the first task
6. **First boot**: the kernel prepares the first task and starts running

From this moment on, the code in `main()` is no longer executed. The system alternates between tasks, managed entirely by the scheduler.

```
System start

    v
  main()

              prepara hardware (orologio, porte, comunicazioni)
              prepare system
              crea task A
              crea task B
              crea task C
              start system       > Task A (first job)

                                   Ritmo       > Cambio       > Task B

                                                                Ritmo       > ...
```

Previous chapter: Introduction Next chapter: Resources - Sharing and Synchronization

Chapter 3

Resources: Synchronization and Mutexes

3.1 Chapter 3 - Resources: Sharing and Synchronization

When multiple tasks share resources (such as variables, hardware devices, or memory areas), mechanisms are needed to ensure that access occurs in an orderly manner. Without these precautions, **concurrency problems** can occur: bugs that are difficult to reproduce, which corrupt data or cause strange behaviors.

3.1.1 3.1 The concurrency problem

Race Condition

Let's imagine two tasks that need to increment a shared counter:

```
u32 counter = 0;  // Variable shared between the tasks

// Task A                    // Task B
counter++;                  counter++;
```

The `contatore++` operation seems simple, but in reality it requires three steps:

1. Read the current value
2. Increment it by 1
3. Write it back

If the system switches from one task to another at the wrong moment:

```
Task A: reads counter (finds 0)
--- system switches to Task B ---
Task B: reads counter (still finds 0!)
Task B: adds 1 (becomes 1)
Task B: writes the value (counter = 1)
--- system returns to Task A ---
Task A: adds 1 (it started from 0, becomes 1)
Task A: writes the value (counter = 1)
```

We performed two increments, but the counter equals 1 instead of 2. This is a classic concurrency bug.

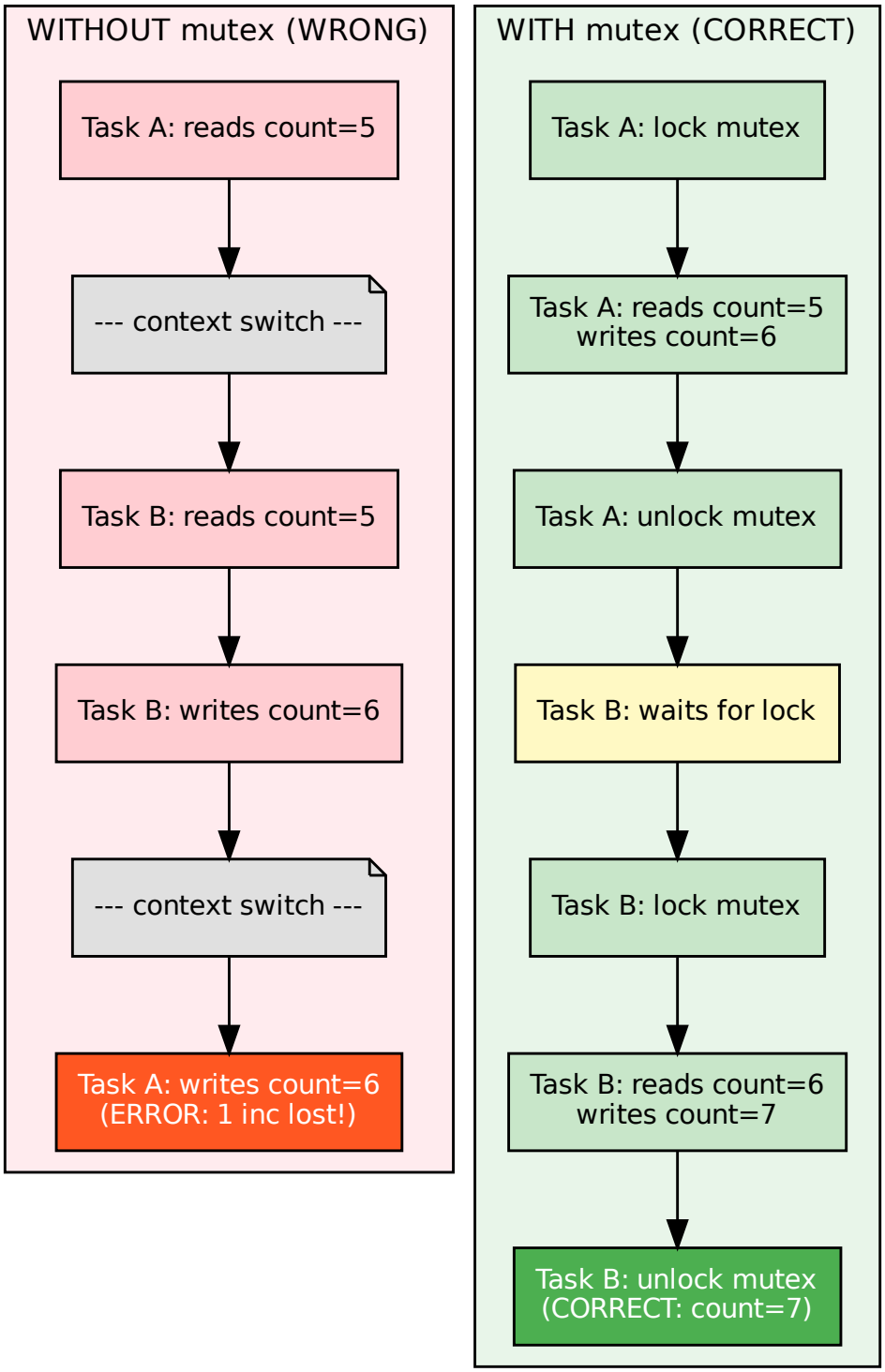


Figure 3.1: Race condition with and without mutex

Critical Section

A **critical section** is a part of code that accesses a shared resource and must be executed without interruptions. The simplest way to protect it is to temporarily disable interrupts:

```
disable_interrupts();   // No task switch possible
counter++;                 // Critical section
enable_interrupts();      // Back to normal
```

It works, but it has a problem: while interrupts are disabled, **no external event** can be handled. If the critical section lasts too long, important information is lost.

Nesting of critical sections

A common problem: if a function disables interrupts and calls another function that does the same thing, when the inner function finishes it might re-enable interrupts too soon:

```
void outer_function(void) {
    disable_interrupts();
    inner_function();       // This one also disables and re-enables
    // HERE interrupts are already re-enabled! Error!
    important_operation();
    enable_interrupts();
}
```

The solution is to keep a **count of how many times** interrupts have been disabled, and re-enable them only when the count returns to zero.

3.1.2 3.2 Safe (Atomic) Operations

Microcontrollers offer special operations that are "atomic" (they cannot be interrupted):

Exclusive Read and Write

There are special operations that allow reading a value, modifying it, and writing it back safely:

```
Riprova:
    Read value (with a "reservation")
    Modify the value
    Write value, but only if nobody else changed it
    If someone interfered, retry
```

If another task modifies the value between the read and the write, the operation fails and is repeated. This is the principle on which many synchronization tools work.

3.1.3 3.3 Mutex

What is a Mutex

The mutex is the most used synchronization tool. It ensures that only one task at a time can access a shared resource:

```
mutex_take(&my_mutex);     // If the mutex is taken, the task waits
// ... use of the resource ...
mutex_release(&my_mutex);   // Release the mutex, wake any waiting tasks
```

The important difference: a task waiting for a mutex **does not consume computing time**. The system switches to other tasks and wakes the first one when the mutex is free.

Properties

A mutex has an **owner**: the task that acquired it. Only that task can release it. This prevents serious errors.

Recursive Mutex

A normal mutex creates problems if the same task tries to acquire it twice:

```
mutex_take(&my_mutex);      // Preso
mutex_take(&my_mutex);      // PROBLEM: the task waits for itself!
```

A **recursive mutex** allows the same task to acquire it multiple times, but it must release it the same number of times:

```
mutex_take(&my_recursive); // counter = 1
mutex_take(&my_recursive); // counter = 2 (same task, OK)
mutex_release(&my_recursive); // counter = 1
mutex_release(&my_recursive); // counter = 0, mutex free
```

Priority Inversion

A common problem in systems with priorities. Imagine three tasks:

- **Task A** (high priority): needs a resource
- **Task B** (medium priority): does not use the resource
- **Task C** (low priority): owns the resource

```
1. Task C takes the resource
2. Task A becomes ready, interrupts Task C
3. Task A looks for the resource     blocked (Task C owns it)
4. Task B becomes ready     runs instead of Task C!
5. Task C cannot free the resource because Task B is running
6. Task A stays blocked     PRIORITY INVERSION
```

Task A (important) is blocked by Task B (less important) which has nothing to do with the resource. This problem has caused serious issues in space missions.

Priority Inheritance

The solution: when an important task is blocked on a resource owned by a less important task, the system **temporarily increases** the priority of the less important task:

```
1. Task C takes the resource (low priority)
2. Task A looks for the resource     blocked
3. System: raises Task C's priority to Task A's level
4. Task B becomes ready, but Task C now has higher priority     Task C goes on
5. Task C frees the resource     priority restored, Task A unblocks
```

Timeout

A task should never wait indefinitely. Mutexes with a timeout allow you to specify the maximum wait time:

```
if (mutex_take_timeout(&my_mutex, 100) == OK) {
    // Mutex taken within 100ms
    mutex_release(&my_mutex);
} else {
    // Time out: handle the error
}
```

3.1.4 3.4 Semaphore

Difference from Mutex

The semaphore is often confused with the mutex, but they have different purposes:

	Mutex	Semaphore
Purpose	Protect a resource	Signal an event or count resources
Ownership	Only the one who takes it releases it	Anyone can signal
Value	0 or 1	From 0 to N
Typical use	Critical section	Inter-task communication

Binary Semaphore (for signaling)

Used to communicate between an interrupt routine and a task:

```
// Interrupt routine: signals that the data are ready
void UART_handler(void) {
    buffer[indice++] = UART->DR;
    sem_signal(&data_ready);      // Wake the task
}

// Task: waits for the data
void task_processing(void) {
    while (1) {
        sem_wait(&data_ready);  // Sleeps until the signal arrives
        process(buffer);
    }
}
```

Counting Semaphore (for multiple resources)

Used to manage a limited set of resources:

```
// 3 buffer disponibili
sem_init(&buffer_free, 3, 3);  // 3 available, maximum 3

// Task: request a buffer
sem_wait(&buffer_free);        // Decreases (if 0, blocks)
buffer = get_DMA_buffer();
// ... use the buffer ...
release_DMA_buffer(buffer);
sem_signal(&buffer_free);        // Aumenta
```

3.1.5 3.5 Condition Variable

The condition variable allows a task to wait for a complex condition to become true:

```
my_mutex mtx;
condition_variable cv;
u32 data_item_available = 0;

// Task produttore
mutex_take(&mtx);
data_item_available = 1;
cond_signal(&cv);       // Wakes ONE waiting task
mutex_release(&mtx);

// Task consumatore
mutex_take(&mtx);
```

```
while (!data_item_available) {    // ALWAYS in a while loop, never an if
    cond_wait(&cv, &mtx);  // Releases mtx, waits, takes mtx back
}
process_item();
data_item_available = 0;
mutex_release(&mtx);
```

The `while` loop is important: a task can be woken up without a reason (spurious wakeup), so it must always check the condition again.

3.1.6 3.6 Spinlock

The spinlock is a mechanism that performs "active waiting": the task that fails to obtain access keeps trying in a loop, using computing time:

```
while (!try_lock(&my_spinlock)) {
    // Busy-wait
}
// Critical section
sblocca(&my_spinlock);
```

When to use Spinlock

- **Very short** critical sections (a few instructions)
- Where waiting is not possible (interrupt routines)
- When a context switch would cost more than the short wait

When NOT to use it

- Long critical sections: it wastes computing time
- In low-priority tasks: priority inversion with no solution
- When the lock is free: a mutex is almost always better

3.1.7 3.7 Message Queue

Queues are the primary method for communication between tasks. Instead of sharing variables (which require protection), tasks exchange messages through a secure queue:

```
// Task produttore
u32 item = read_sensor();
queue_send(&my_queue, &item, timeout);

// Task consumatore
u32 item;
queue_receive(&my_queue, &item, timeout);
process(item);
```

Processing order

- **FIFO** (First In, First Out): messages are processed in the order of arrival. Standard behavior.
- **LIFO** (Last In, First Out): the most recent messages have priority. Useful when only the latest data is relevant.

Queue from Interrupts

Interrupt routines cannot wait. Queues offer special versions for use from interrupts:

```
void UART_handler(void) {
    u8 byte = UART->DR;
    queue_send_from_isr(&rx_queue_isr, &byte);  // Never waits, error if full
}
```

3.1.8 3.8 Event Groups

Event Groups are a set of 32 flags that allow a task to wait for combinations of events:

```
// Task A: signals that the sensor is ready
event_set(&group, BIT_SENSOR_OK);

// Task B: signals that the display is ready
event_set(&group, BIT_DISPLAY_OK);

// Task C: waits until BOTH are ready
event_wait_all(&group, BIT_SENSOR_OK | BIT_DISPLAY_OK, timeout);
// Or: wait until AT LEAST ONE is ready
event_wait_any(&group, BIT_SENSOR_OK | BIT_DISPLAY_OK, timeout);
```

Event Groups are lightweight and allow for complex synchronization without many separate structures.

3.1.9 3.9 Task Notification

Task Notification is a direct method for communicating between tasks, without intermediate data structures. Each task has a built-in notification value:

```
// Task A sends a notification to Task B
notify_send(task_b_id, value, NOTIFY_SET_BITS);

// Task B waits for the notification
u32 val = notify_wait(true, timeout);  // true = clear on wake-up
```

Advantages over semaphore

- **No extra memory consumption**: uses a field already present
- **Faster**: no data structures to manage
- **More flexible**: can carry a value

Limitations

- **Point-to-point**: only one task can wait (no waiting “queue”)
- **Risk of loss**: if the task is not waiting when the notification arrives, it might not see it

3.1.10 3.10 Deadlock

What is Deadlock

Deadlock is a situation in which two or more tasks block each other, each waiting for a resource held by the other:

```
Task A: takes resource_1      OK
Task B: takes resource_2      OK
Task A: looks for resource_2      BLOCKED (Task B owns it)
Task B: looks for resource_1      BLOCKED (Task A owns it)
// Both blocked forever!
```

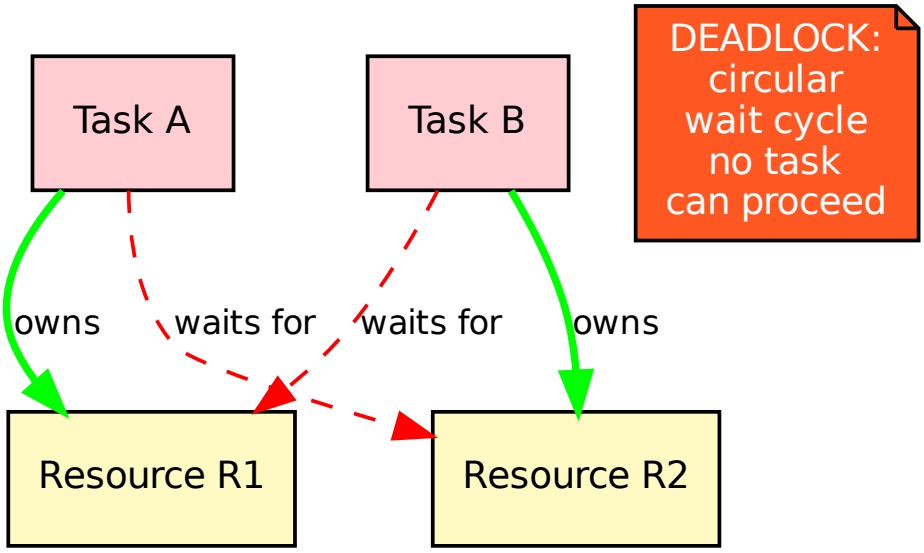


Figure 3.2: Deadlock between two tasks

How to avoid it

- **Fixed order**: always acquire resources in the same order in all tasks
- **Timeout**: use versions with a timeout to avoid infinite waiting
- **Acquire all at once**: acquire all necessary resources at once
- **Minimize sharing**: reduce the number of shared resources

---

Previous chapter: The Kernel Next chapter: Memory - Partitioning and Protection Next chapter: Memory - Partitioning and Protection

Chapter 4

Memory: Partitioning and Protection

Previous chapter: Resources

Previous chapter: Resources Next chapter: Embedded I/O - Drivers, Sensors and Timers

Chapter 5

Embedded I/O: Drivers, Sensors, Timers

5.1 Chapter 5 - Embedded I/O: Drivers, Sensors, and Timers

A microcontroller without peripherals is useless. Its raison d'être is to interact with the outside world: reading sensors, controlling actuators, and communicating with other devices. In this chapter, we see how an RTOS handles I/O and timers.

5.1.1 5.1 GPIO (General Purpose Inputs/Outputs)

What is a GPIO

Every pin of a microcontroller can be configured as a digital input or output. This is the simplest form of I/O:

- **Output**: turning on an LED, controlling a relay, generating a signal
- **Input**: reading a button, a digital sensor, an external signal

GPIO Registers on STM32

Each GPIO port (A, B, C, ...) is controlled by special registers:

Register	Function
MODER	Mode: input, output, alternate function, analog
OTYPER	Output type: push-pull or open-drain
OSPEEDR	Speed: low, medium, high, very high
PUPDR	Internal pull-up/pull-down resistor
IDR	Input data register (read)
ODR	Output data register (write)
BSRR	Bit set/reset register (safe write)
AFR[2]	Alternate function (selects peripheral: UART, SPI, I2C...)

Basic configuration

```
// Example: configure PA5 as a push-pull output, high speed
GPIOA->MODER   &= ~(3 << (5*2));     // Clear the field
GPIOA->MODER   |=  (1 << (5*2));     // Output mode
GPIOA->OTYPER  &= ~(1 << 5);         // Push-pull
GPIOA->OSPEEDR |=  (3 << (5*2));     // Very high speed

```

```
// Safe write (no concurrency problem)
GPIOA->BSRR = (1 << 5);             // Turn PA5 on (LED)
GPIOA->BSRR = (1 << (5 + 16));      // Turn PA5 off (LED)
```

The BSRR register is particularly important: it allows setting or resetting a single pin with a single safe write, without having to perform a read-modify-write operation as with ODR.

The compatibility problem

GPIO registers change from one STM32 model to another: - **F1xx**: uses CRL/CRH with different encoding (4 bits per pin) - **F4xx/F7xx/H5xx/H7xx**: uses MODER/OTYPER-/OSPEEDR (2 bits per pin in MODER) - **Clock enabling**: different RCC registers for each model

A portable GPIO driver must hide these differences behind a common interface.

HAL vs Direct Code

ST provides the HAL (Hardware Abstraction Layer), a library that hides the registers. However:

	HAL	Direct Code
Portability	High (between STM32)	Low (requires abstraction)
Overhead	Significant	Zero
Code size	Large	Minimal
Control	Limited	Total
Debug	Difficult (many layers)	Direct (registers visible)

For a bare-metal RTOS, the natural choice is to write your own drivers with a minimal abstraction layer: a generic interface that calls model-specific functions.

5.1.2 5.2 Driver model: Weak Functions

The problem

A driver must work on different microcontrollers. The general logic (e.g., “configure this pin as output”) is the same, but the hardware details change.

The solution: Weak Functions

The **weak functions** model leverages a feature of the GNU linker:

```
// driver_generico.c - Logica comune
__attribute__((weak))
GPIO_TypeDef* get_port(u8 port_id) {
    return NULL;  // Default: no implementation
}

void gpio_configure(u8 port, u8 pin, u8 mode) {
    GPIO_TypeDef *hw = get_port(port);  // Call the specific version
    if (hw == NULL) return;
    hw->MODER = ...;  // Logic common to all families
}
```

```
// driver_f4xx.c - Replacement for F4xx
GPIO_TypeDef* get_port(u8 port_id) {
    // F4xx-specific table
    GPIO_TypeDef *ports[] = {GPIOA, GPIOB, GPIOC, ...};
    return ports[port_id];
}
```

The linker automatically chooses the "strong" (non-weak) version if present, otherwise it uses the default weak one. When compiling for F4xx, `driver_f4xx.c` is included; when compiling for H7xx, `driver_h7xx.c` is included. The generic code does not change.

Advantages of the model

- **Shared generic code**: common logic is written only once
- **Expandability**: adding a new model only requires a new replacement file
- **No #ifdef**: the code is clean, without complex preprocessor directives
- **Safe default**: if a weak function is not replaced, the default (no action or error) prevents crashes

5.1.3 5.3 UART (Universal Asynchronous Receiver/Transmitter)

What is UART

UART is the simplest and most widespread serial communication peripheral. Two wires (TX, RX), no shared clock, bidirectional communication. Used for:

- Debugging (text on PC terminal)
- Communication with modules (GPS, Bluetooth, WiFi)
- Communication between microcontrollers

Configuration parameters

- **Baudrate**: speed in bits/second (9600, 115200, 921600...)
- **Data bits**: typically 8
- **Parity**: None, Even, Odd
- **Stop bits**: 1 or 2

UART and RTOS

UART presents specific challenges in a multitask environment:

Transmission: if multiple tasks write to the same UART, messages get mixed up. A mutex or a transmission queue is needed:

```
// Approach 1: mutex
mutex_take(&uart_mtx);
uart_send("complete message\r\n");
mutex_release(&uart_mtx);

// Approach 2: message queue
queue_send(&uart_tx_queue, &message, timeout);
// A dedicated task drains the queue and transmits
```

Reception: bytes arrive at any time, regardless of who is waiting for them. Typical handling:

```
// Interrupt routine: saves every byte received
void UART_IRQHandler(void) {
    u8 byte = UART->DR;
    queue_send_from_isr(&rx_queue, &byte);
}

// Task: process the data
void task_rx(void) {
    while (1) {
        u8 byte;
        queue_receive(&rx_queue, &byte, WAIT_FOREVER);
```

```
        process_byte(byte);
    }
}
```

5.1.4 5.4 Communication buses: SPI and I2C

SPI (Serial Peripheral Interface)

Synchronous, bidirectional, master-slave bus. Four wires: - **MOSI**: Master Out, Slave In - **MISO**: Master In, Slave Out - **SCK**: Clock (generated by the master) - **CS**: Chip Select (one wire for each slave)

Fast (up to tens of MHz), but requires many pins. Used for: displays, external flash, fast ADCs.

I2C (Inter-Integrated Circuit)

Synchronous, half-duplex, multi-master bus. Two wires: - **SDA**: Data - **SCL**: Clock

Slow (100KHz-1MHz standard), but requires only 2 pins and supports many devices on the same bus. Used for: sensors (temperature, accelerometer), EEPROM, RTC.

Bus management in an RTOS

A shared bus (SPI or I2C) requires mutual exclusion:

```
// Access to the SPI bus shared by sensor and display
mutex_take(&spi_bus_mtx);
cs_select(SENSORE);
spi_transfer(tx_buf, rx_buf, len);
cs_deselect(SENSORE);
mutex_release(&spi_bus_mtx);
```

Without the mutex, a task switch during the SPI transfer would leave the chip select active and the bus in an inconsistent state.

5.1.5 5.5 Hardware Timers

Timers as time bases

Hardware timers are counters that increment (or decrement) at every clock cycle (or at a divided frequency). When they reach a programmed value, they generate an interrupt.

Main uses: - **SysTick**: the RTOS heartbeat (see Chap. 2) - **Timeout**: measuring time intervals - **PWM**: generating variable duty-cycle signals (e.g., motor control, LED brightness) - **Input Capture**: measuring the duration of an external signal - **Performance analysis**: measuring code execution time

Software Timers in an RTOS

Software timers are built on top of the RTOS heartbeat. A “timer daemon” task runs at high priority and manages a list of active timers:

```
// Periodic timer creation
timer_create(&t, "heartbeat", 1000, PERIODICO, callback);
timer_start(&t);

// The callback is called in the timer daemon context
void callback(timer_t *t) {
    toggle_led();  // Every 1000ms
}
```

Advantages over hardware timers: - Unlimited number (limited only by RAM) - Callbacks executed in a task context (can use mutexes, queues, etc.) - Portable across microcontrollers

Disadvantages: - Resolution limited to the RTOS tick rate - Jitter: the callback is executed when the timer daemon is scheduled, not exactly at the millisecond

One-shot vs Periodic

- **One-shot**: expires only once. Useful for timeouts (e.g., "if I don't receive a response within 500ms, signal an error")
- **Periodic**: reloads automatically. Useful for repetitive actions (e.g., sensor sampling every 100ms)

5.1.6 5.6 Time measurement and performance analysis

DWT Cycle Counter

The Cortex-M3/M4/M7 has a hardware cycle counter (DWT CYCCNT) that counts every clock cycle:

```
CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
DWT->CYCCNT = 0;
DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;

// Misura
u32 start = DWT->CYCCNT;
function_to_measure();
u32 end = DWT->CYCCNT;
u32 cycles = end - start;
// Time = cycles / clock_frequency
```

Precision: 1 clock cycle (e.g., ~6ns at 168MHz). Ideal for precise performance analysis.

Hardware timers for analysis

As an alternative to the DWT, a hardware timer (e.g., TIM2, 32-bit) can be configured to count freely:

```
// TIM2 a 1MHz (prescaler = clock/1000000)
u32 start = TIM2->CNT;
function_to_measure();
u32 end = TIM2->CNT;
u32 microsecondi = end - start;
```

5.1.7 5.7 Interrupts and I/O: the RTOS challenge

Deferred Interrupt Processing

Interrupt routines must be short. The recommended practice is **deferred processing**:

1. The routine does the bare minimum (reads the register, clears the flag)
2. It signals a task via semaphore, notification, or queue
3. The task performs the heavy processing

```
// Routine: fast (< 1us)
void ADC_IRQHandler(void) {
    u16 value = ADC1->DR;        // Read data
    ADC1->SR &= ~ADC_SR_EOC;      // Clear the flag
    queue_send_from_isr(&adc_q, &value);  // Hand over to the task
```

```
}

// Task: elaborazione pesante
void task_adc(void) {
    while (1) {
        u16 value;
        queue_receive(&adc_q, &value, WAIT_FOREVER);
        // Filter, moving average, unit conversion...
        risultato = filter_and_convert(value);
    }
}
```

Interrupt priorities and RTOS

On Cortex-M, interrupt levels are divided into two groups:

- **Above BASEPRI**: ultra-high priority interrupts that are never disabled by the RTOS. They cannot use RTOS APIs (no queues, semaphores, etc.)
- **Below BASEPRI**: interrupts managed by the RTOS. They can use `_isr` APIs (queue_send_isr, sem_signal_isr, etc.)

```
Interrupt priority (0 = highest on Cortex-M):
  0-3: NOT managed by the RTOS ("bare" routines, no OS API)
  4-15: Gestite dall'RTOS (possono usare API _isr)

SysTick: typically low priority (e.g. 15)
PendSV:  lowest possible priority (255)
```

Previous chapter: Memory Next chapter: Reliability - Watchdog, Errors and Updates

---

Previous chapter: Memory Next chapter: Reliability - Watchdog, Errors and Updates

Chapter 6

Reliability: Watchdog, Errors, OTA

6.1 Chapter 6 - Reliability: Watchdog, Error Handling and Updates

An embedded system must function even when things go wrong. Unlike a PC application where the user can close and reopen the program, an embedded microcontroller is often installed in inaccessible places and must operate for months or years without human intervention. Reliability is not optional: it is a requirement.

6.1.1 6.1 Types of errors in embedded systems

Hardware errors

- **Brown-out**: the supply voltage drops below the minimum. The processor may execute incorrect instructions before resetting
- **Bit flip**: cosmic radiation or electromagnetic interference can flip a bit in RAM (rare, but real in space or high-altitude systems)
- **Peripheral locked**: an I2C bus in a stall, a sensor that does not respond
- **Clock failure**: the external crystal stops oscillating

Software errors

- **Stack overflow**: the task exceeds its stack (see Ch. 2)
- **Null pointer access**: access to address 0x00000000
- **Division by zero**: causes an error on Cortex-M
- **Memory corruption**: a task overwrites data of another task
- **Deadlock**: two tasks blocked by each other (see Ch. 3)
- **Starvation**: a task never receives CPU time

Logical errors

- **Watchdog timeout**: a task does not complete its cycle in time
- **Check failure**: a condition that "should never be false" is false
- **Arithmetic overflow**: a calculation produces a result out of range

6.1.2 6.2 Cortex-M errors

The ARM Cortex-M processor has a hierarchical error system to handle hardware problems:

General error (HardFault)

The “catch-all” error. If a more specific error is not enabled, the processor enters HardFault. Common causes: - Access to an invalid address - Execution of an invalid instruction - Error during the handling of another error (double fault)

Memory management error (MemManage Fault)

Generated by the protection unit when a protected region is violated: - Writing to a read-only region (e.g., stack canary) - Access to a region with “No access” permission - Execution of code from a non-executable region

Bus error (BusFault)

Error on the system bus: - Access to a non-existent address - Unaligned access to a peripheral that requires alignment - Error during instruction fetch

Usage error (UsageFault)

Error in the use of the processor: - Undefined instruction - Unaligned access (if not permitted) - Division by zero (if checking is enabled) - Attempt to switch to ARM state (Cortex-M only supports Thumb)

Error status registers

The Cortex-M provides registers that indicate the precise cause of the error: | Register | Information | |———-|————-| | CFSR (Configurable Fault Status Register) | Combines MMFSR, BFSR, UFSR | | MMFSR | Memory management fault: which MPU violation | | BFSR | Bus fault: type of bus error | | UFSR | Usage fault: type of usage error | | HFSR | Hard fault: escalation or forced | | MMFAR | Address that caused the Memory management fault | | BFAR | Address that caused the Bus fault |

Reading these registers in the fault handler allows for precise diagnosis of the cause of the problem.

6.1.3 6.3 Fault handler in an RTOS

The problem

When a fault occurs in a task, what should the RTOS do?

- **Total reset**: simple but drastic. All tasks restart from scratch
- **Remove the task**: isolate the problem, other tasks continue
- **Restart the task**: if the task is repeatable, restart from the beginning
- **Emergency handler**: call a cleanup function before taking action

Saving registers

A good fault handler saves diagnostic information before taking action:

```
=== ERRORE GENERALE ===
Task: "sensore" (ID: 3)
PC:  0x08001234  (indirizzo dell'istruzione che ha causato l'errore)
LR:  0x08001200  (chi ha chiamato la funzione)
SP:  0x2001E7F0  (puntatore stack al momento dell'errore)
CFSR: 0x00020000 (INVSTATE - tentativo di eseguire istruzione invalida)
```

This information is fundamental for debugging. The PC indicates where the problem occurred, the LR indicates who called the function, and the CFSR indicates the type of error.

Recovery policies for tasks

A robust RTOS allows configuring the recovery policy for each task:

```
// Task critico: riavvia automaticamente
os_job("sensore", ..., FAULT_RESTART);

// Task non critico: rimuovi e basta
os_job("display", ..., FAULT_REMOVE);

// Task con pulizia: chiama gestore poi riavvia
os_job("comunicazione", ..., FAULT_EMERGENCY | FAULT_RESTART);
```

6.1.4 6.4 Watchdog Timer

Concept

The watchdog timer (WDT) is a hardware timer independent of the processor. If it is not "kicked" within a programmed time, it resets the processor. It is the last line of defense against software hangs.

```
// Configurazione
watchdog_init(tempo_limite_ms);
watchdog_start();

// Nel loop principale (o nel task dedicato)
while (1) {
    esegui_operazioni();
    watchdog_kick();    // "Sono ancora vivo"
}
```

If the software hangs (infinite loop, deadlock, crash), the watchdog is not updated and resets the entire system after the timeout period.

IWDG vs WWDG on STM32

STM32 has two watchdog timers:

IWDG (Independent Watchdog): - Independent clock (LSI ~32KHz) - Works even if the main clock fails - Configurable timeout from ~125us to ~32s - Once enabled, it cannot be disabled (until reset)

WWDG (Window Watchdog): - Clock derived from the APB bus - Has a "window": the update must occur within a precise interval, not too early and not too late - Generates an interrupt before the reset (allows for save operations)

Watchdog and RTOS

In an RTOS, the watchdog must monitor **all** tasks, not just one. The strategies:

Approach 1: Watchdog task

```
Task watchdog (alta priorit ):
  while (1) {
    if (task_1_vivo && task_2_vivo && task_3_vivo) {
      watchdog_kick();
      resetta_flag;
    }
    sleep(tempo_limite / 2);
  }

Ogni task: imposta il proprio flag vivo ad ogni ciclo
```

Approach 2: Notification chain

```
Task 1     notifica     Task 2     notifica     Task 3     notifica     aggiorna
    watchdog
```

If any task hangs, the chain is broken and the watchdog expires.

6.1.5 6.5 Checks and Safety

Classic check

The check is the simplest verification mechanism:

```
assert(ptr != NULL);         // Se falso, interruzione()
assert(indice < DIM_ARRAY); // Controllo limiti
```

On a microcontroller, `interruzione()` typically causes an infinite loop or a reset. The information is minimal: it is only known that a check has failed.

Safety: advanced control

A Safety system goes beyond simple control:

1. **Identify the task**: which task caused the error
2. **Record information**: message, file, line, variable values
3. **Apply the policy**: remove, restart, or emergency handler
4. **Continue execution**: other tasks are not affected

```
// Controllo con messaggio
SAFEGUARD(ptr != NULL, "Buffer non allocato");

// Se ptr    NULL:
// 1. Log: "SAFEGUARD FAIL: Buffer non allocato [task: sensore, riga: 42]"
// 2. Chiama gestore emergenza se configurato
// 3. Rimuove o riavvia il task secondo la politica
// 4. Lo scheduler continua con gli altri task
```

Severity levels

A robust system distinguishes between different levels:

- **Warning**: anomalous but non-critical situation, the task continues
- **Error**: the task cannot continue, it is removed/restarted
- **Panic**: the system cannot continue, total reset

6.1.6 6.6 Protection against corruption

Stack Canary + MPU

As seen in Chap. 2, the combination of software canary and hardware MPU protection is the best defense against stack overflow.

Data structure integrity

Critical kernel structures can be protected with:

- **Magic number**: a constant value at the beginning of the structure. If corrupted, the structure is invalid
- **Checksum**: a CRC calculated on the state of the structure, verified periodically

- **Redundancy**: duplication of critical variables (counter a + counter b, they must be equal)

```c
typedef struct {
    u32 magic;          // Deve essere 0xCAFEBABE
    u32 stato;
    u32 contatore;
    u32 checksum;       // CRC32 dei campi precedenti
} struttura_critica;
```

ECC (Error Correcting Code)

Some microcontrollers (such as the H5xx) have ECC on RAM: they can correct single-bit errors and detect double-bit errors. This protects against bit flips caused by interference.

6.1.7 6.7 Reset and Recovery

Reset types on STM32

Type	Cause	Effect
Power-On Reset	Power-up	Full reset
NRST Pin	External pin	Full reset
IWDG Reset	Watchdog expired	Full reset
WWDG Reset	Window watchdog	Full reset
Software Reset	`NVIC_SystemReset()`	Full reset
Brown-Out Reset	Low voltage	Full reset

Determining the reset cause

After a reset, the RCC_CSR register indicates the cause:

```c
if (RCC->CSR & RCC_CSR_IWDGRSTF) {
    // Reset causato dal watchdog
    log("IWDG reset detected");
}
if (RCC->CSR & RCC_CSR_BORRSTF) {
    // Brown-out
    log("Power issue detected");
}
RCC->CSR |= RCC_CSR_RMVF;  // Cancella flag
```

This information is valuable for diagnostics: if the system repeatedly resets due to a watchdog, there is a blocking bug in the software.

Persistent state across resets

RAM is not cleared by a software or watchdog reset (only by a power-on reset). It is possible to use a RAM region marked `noinit` to maintain information between resets:

```c
__attribute__((section(".noinit")))
static u32 contatore_crash;

__attribute__((section(".noinit")))
static u32 ultimo_errore_pc;
```

This allows counting crashes and identifying error patterns.

6.1.8 6.8 Firmware Update (OTA)

The problem

An embedded device installed in the field needs software updates for bug fixes or new features. However, a failed update can “brick” the device (render it unusable).

Dual-bank bootloader

The standard solution is the **dual-bank firmware update**:

```
Flash:

    Bootloader (16KB)        Sempre presente, non aggiornabile

    Bank A: Firmware             Versione corrente (in esecuzione)
    (es. 512KB)

    Bank B: Firmware             Nuova versione (in download)
    (es. 512KB)

```

The process: 1. The current firmware (Bank A) downloads the new firmware into Bank B 2. Verifies integrity (CRC, digital signature) 3. Signals the bootloader to start Bank B at the next reset 4. Reset 5. The bootloader verifies Bank B and starts it 6. If Bank B does not work (watchdog reset), the bootloader returns to Bank A

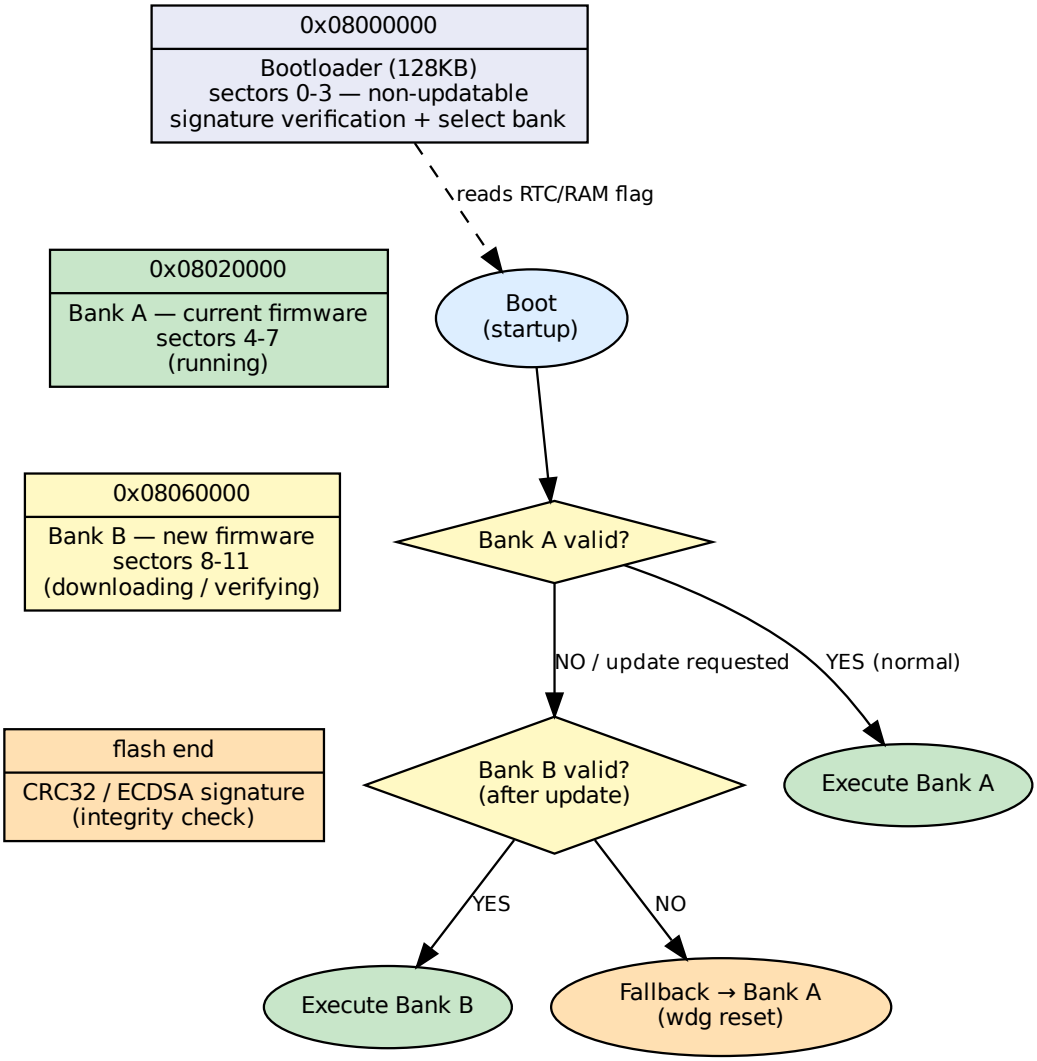


Figure 6.1: Dual-bank OTA flash layout

Firmware digital signature

To prevent the installation of unauthorized firmware:

```
Firmware = image + header
The header holds:
  - version
  - size
  - hash of the code (integrity: SHA-256)
  - signature (identity: e.g. Ed25519, ECDSA), checked with the public key in
    the bootloader
```

The bootloader verifies the signature before executing the new firmware. Without the private key (which remains on the build server), it is impossible to create fraudulent firmware. CRC is sufficient against transmission errors, but not against someone intentionally altering the image: for this reason, integrity relies on a cryptographic hash and identity on a signature.

In caOS

caOS provides a complete bootloader as a workspace application, `workspace/app_2`, for NUCLEO-H563ZI and NUCLEO-H753ZI. It follows the scheme just described:

- default map 512 + 512 + 512 KB: bootloader and slot A in bank 1, slot B in bank 2; the map is in `caos/inc/boot_image.h` and can be customized according to needs (bootloader size versus image size, guide §1.1);
- `.cimg` images with 1 KB header, mandatory SHA-256 and Ed25519 signature (`tools/caos_image.py`);
- network update: the PC pushes the image via TFTP (`tools/caos_tftp`), both to the bootloader in recovery and to the running image;
- the new image starts **in trial mode**, with the IWDG armed and attempts counted in a backup register; if it is not confirmed within three boots, the bootloader reverts to the previous one (rollback).

The transition from the bootloader to the image occurs with `ca_os_end()`, which shuts down caOS and resets the hardware as if after a reset. The image side is in the kernel (`caos/inc/boot_image.h`, flag `EN_BOOT_IMAGE`). Complete guide: `workspace/app_2/doc/app_2_user_guide.md`.

Previous chapter: Embedded I/O

End of Section 1 - General Concepts
Section 2: How caOS implements these concepts

Part I

caOS Implementation

Chapter 7

caOS Overview

7.1 Chapter 7 - Overview of caOS

7.1.1 7.1 What is caOS?

Coppi **A**ngelo **O**perating **S**ystem (caOS) is a lightweight operating system designed for ARM Cortex-M microcontrollers. Born as an adapted version of a project for Raspberry Pi (Cortex-A), it has been completely rewritten to work on the Cortex-M architecture, supporting 9 different STM32 microcontrollers distributed across 6 different families.

Basic Principles

- **Pure bare metal**: no dependency on external libraries such as ST HAL. Direct access to hardware registers
- **Easy to port**: general logic is separated from specific implementation for each microcontroller family
- **Single include**: all `.c` files in the project include only `caos.h`, which in turn includes all necessary modules
- **Simplified types**: `u8`, `u16`, `u32`, `i8`, `i16`, `i32`, `cstr` instead of standard types

Main Features

Category	Features
Scheduling	weight-based proportional (similar to CFS), 7 priority levels, 1 tick per switch, context switch only on task change
Synchronization	Mutex, Recursive Mutex, Semaphore, Condition Variable, Interrupt-safe Spinlock
Communication	Queue (FIFO/LIFO), Event Groups (32-bit), Task Notification, Stream Buffer, Message Buffer
Memory	Dynamic allocation (SRAM) with owner tracking, stack allocation in CCM/DTCM
Protection	Memory Protection Unit (MPU) to protect stacks and heap boundaries, error detection
Timers	Software Timer (one-shot and periodic), up to 16 simultaneously

Category	Features
Debug	Asynchronous two-channel audit log, UART command-line interface, detailed task information
Reliability	Safety checks with recovery policies for tasks, error handler with diagnostic information

7.1.2 7.2 Project Structure

Three-Level Organization

caOS is organized in three distinct levels:

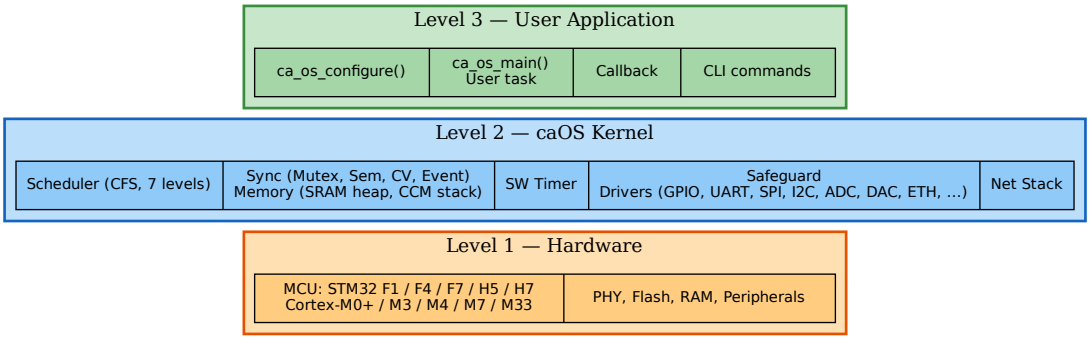


Figure 7.1: Three-level organization

Folder Structure

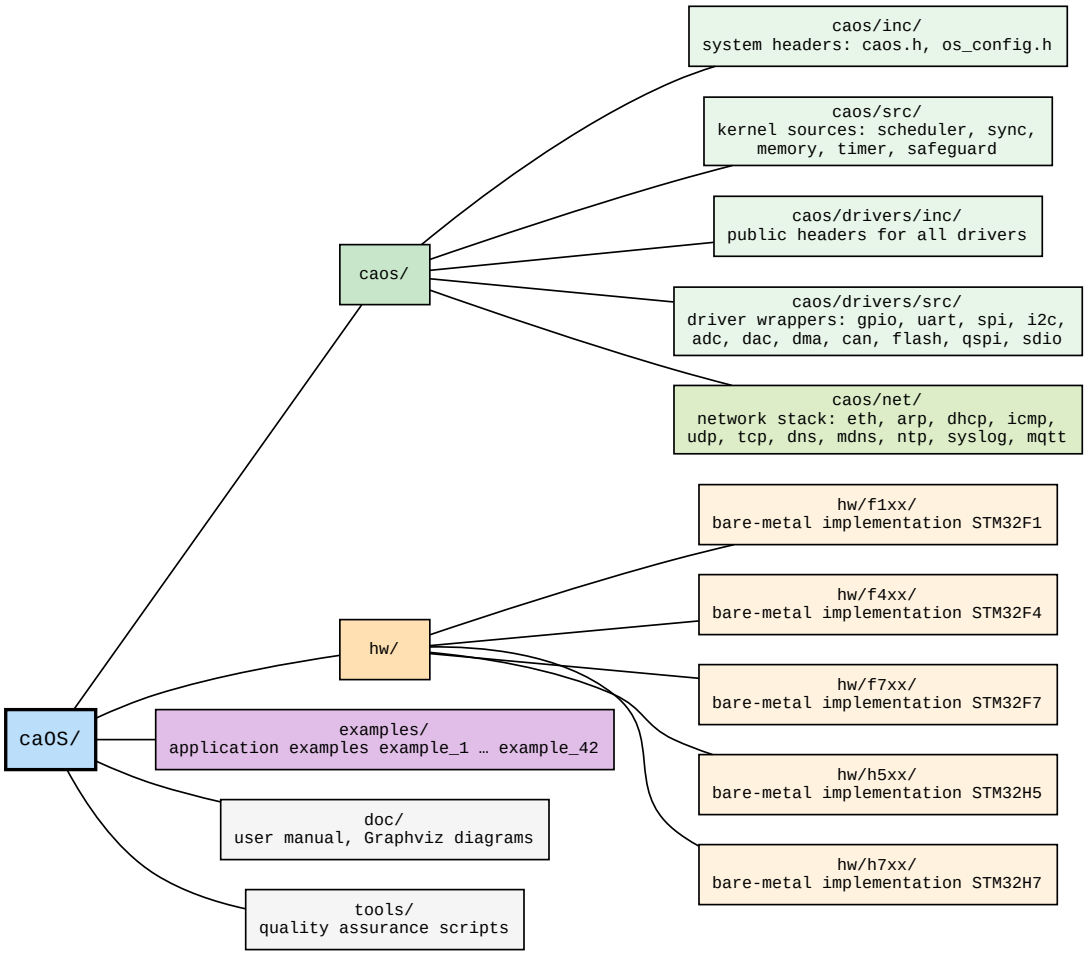


Figure 7.2: Folder structure

Include Schema

All `.c` files in the project use a single include:

```
1 #include "caos.h"
```

`caos.h` includes all modules automatically: `common.h`, `scheduler.h`, `sync.h`, `log.h`, `queue.h`, `event.h`, `notify.h`, `timer.h`, `memory.h`, `mpu.h`, `safeguard.h`, `stream.h`, `message.h`, `cli.h`, `gpio_driver.h`, `uart.h`, `atomic.h`, `memaux.h`, etc.

This approach simplifies the management of includes and avoids dependency issues between headers.

7.1.3 7.3 Supported Microcontrollers

MCU	Core	Clock	Flash	SRAM	TCM	Status
STM32F103C8T6	Cortex-M3	72MHz	64KB	20KB	-	Base (no MPU, no FPU)
STM32F407VET6	Cortex-M4F	168MHz	512KB	128KB	64KB CCM	Complete
STM32F429ZIT6	Cortex-M4F	180MHz	2MB	256KB	64KB CCM	Complete
STM32F446ZET6	Cortex-M4F	180MHz	512KB	128KB	-	Complete
STM32F746ZGT6	Cortex-M7	216MHz	1MB	320KB	64KB DTCM	Complete
STM32H563ZIT6	Cortex-M33	250MHz	2MB	640KB	-	Complete
STM32H747XIH6	Cortex-M7+M4	480MHz	2MB	1MB	128KB DTCM	In development (clock issue)
STM32H753ZIT6	Cortex-M7	480MHz	2MB	512KB	128KB DTCM	Complete
STM32U575ZIT6	Cortex-M33	160MHz	2MB	784KB	-	Complete (no ETH)

Differences between families

Feature	F1xx (M3)	F4xx (M4F)	F7xx (M7)	H5xx (M33)	H7xx (M7)	U5xx (M33)
FPU (Floating Point Unit)	No	Yes	Yes (double)	Yes	Yes (double)	Yes
MPU (Memory Protection Unit)	No	Yes (8 regions)	Yes (8/16)	Yes (16)	Yes (16)	Yes (16)
TCM (TCM - Tightly Coupled Memory)	No	CCM 64KB	DTCM 64KB	No	DTCM 128KB	No
GPIO Registers	CRL/CRH	MODER/OTYPER	MODER/OTYPER	MODER/OTYPER	MODER/OTYPER	MODER/OTYPER
GPIO Clock RCC Registers	APB2ENR	AHB1ENR	AHB1ENR	AHB2ENR	AHB4ENR	AHB2ENR1

7.1.4 7.4 caOS Modules

Complete Overview

Module	Header	Description
Kernel		
Scheduling	scheduler.h	RTOS core: tasks, context switch, suspend/resume
Atomic	atomic.h	Interrupt-safe spinlocks, LDREX/STREX atomic operations
SysTick	systick.h	System timer configuration
Synchronization		
Sync	sync.h	Mutex, Recursive Mutex, Semaphores, Condition Variables, Critical Sections
Communication		
Queue	queue.h	FIFO/LIFO queues (static and dynamic) with timeouts
Event	event.h	32-bit Event Groups (wait for any/all)
Notify	notify.h	Direct task notifications (zero overhead)
Stream	stream.h	Circular byte stream buffer with activation level
Message	message.h	Message buffers with atomic framing
Memory		
Memory	memory.h	Dynamic allocation with proprietary tracking
MPU	mpu.h	Memory protection (conditional on `__MPU_PRESENT`)
MemAux	memaux.h	Memory/string functions (newlib replacement)
Debug and diagnostics		
Log	log.h	Asynchronous two-channel audit log (audit + stdout)
CLI	cli.h	UART command-line interface
ITM	itm.h	SWO/ITM output for debugging
Measure	measure.h	Profiling via TIM2
Reliability		
SafeGuard	safeguard.h	Task monitoring with recovery policies
Panic	panic.c	Critical error handler with diagnostic info
Bus	bus.h	BusFault exception handler
Drivers		
GPIO	gpio_driver.h	GPIO abstraction with weak functions
UART	uart.h	UART debug with weak functions
Cryptography		
CRC32	crc32.h	Hardware CRC32 calculation (data integrity)

Module	Header	Description
HASH	hash.h	Hardware SHA-1 / SHA-256 with multithreading context swap
AES	aes.h	Hardware AES-256 ECB/CBC/CTR/GCM with context swap
RNG	rng.h	Hardware True Random Number Generator
Keys Store	keys_store.h	Secure storage of cryptographic keys via MPU
Utilities		
Utils	utils.h	Utility types and functions

7.1.5 7.5 Configuration

caOS configuration is centralized in file `os_config.h` (automatically included by `common.h` via `config.h`):

```
// Output debug
ENABLE_ITM          0|1     // Debug via SWO (Instrumentation Trace Macrocell)
ENABLE_DBG_UART     0|1     // Debug via UART
ENABLE_STDOUT       0|1     // stdout channel (ca_stdout_*)
ENABLE_CLI          0|1     // Command-line interface

// UART debug
UART_DBG_NUM        1-12    // UART number for debug
UART_DBG_BAUDRATE   ...     // Baudrate (es. 115200)
```

Disabled features become empty macros with no overhead:

```
// Se ENABLE_STDOUT == 0:
#define ca_stdout_printf(...)       0   // Compila in nulla
#define ca_stdout_set_repeater(hook)    // idem
```

7.1.6 7.6 Build System

Toolchain

caOS uses arm-none-eabi-gcc (included in STM32CubeIDE) and CMake.

Main Commands

```
source ./source_this        # Set up the toolchain PATH (once per session)
./bootstrap Debug           # Configure CMake (Debug or Release)
./do                        # Compila
./distclean                 # Clean everything
./rebuild                   # distclean + bootstrap + do
```

Microcontroller Selection

In `cpu.cmake`:

```
set(CURR_MCU "F407")  # F103|F407|F429|F446|F746|H563|H747|H753|U575
```

Changing this line and executing `./rebuild` is sufficient to compile for a different microcontroller. The build system automatically selects the correct startup files, linker scripts, drivers and configuration.

Programming

```
./prog.sh -j -f elfs/f407.elf     # J-Link
./prog.sh -s -f elfs/h753.elf     # ST-Link
```

7.1.7 7.7 Entry point: ca_os_main()

The user application does not write `main()` — this function is managed by the kernel. The user implements `ca_os_main()`, which is called after system initialization:

```
#include "caos.h"

void ca_os_main(void) {
    // The hardware is already initialized (clock, GPIO, UART, MPU...)
    // The scheduler is already initialized
    // The log is already active

    // Crea i task dell'applicazione
    ca_os_job("task_a", CA_TH_LEVEL3, my_task_a, NULL,
              STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);

    ca_os_job("task_b", CA_TH_LEVEL2, my_task_b, NULL,
              STACK_SMALL, CA_JOB_STD, CA_FAULT_RESTART_THREAD);

    // Start the scheduler (never returns)
    ca_os_run(true);
}
```

7.1.8 7.8 Configurable Tick

Unlike most RTOS that use a fixed tick (typically 1ms), caOS uses a configurable tick per family of microcontrollers via `OS_TICK_US` in `objmap.h`:

MCU	OS_TICK_US	Clocks/tick
F103	2 ms	144K
F4xx	250 us	42K
F746/F767	200 us	43.2K
H563	100 us	25K
H747/H753	100 us	40K/48K
U575	250 us	40K

The tick only costs the SysTick: context switches occur when tasks change, not at every tick (S2_02 §8.4, *Fette*). Therefore, the scheduler cost depends on task switches and the number of tasks, because each context switch checks the canary of all stacks: measurements and table in S2_02 §8.5, *Scheduler Cost*.

The suspension APIs automatically convert milliseconds to ticks:

```
ca_os_sleep(100);  // Sleeps 100ms regardless of the configured tick
// Internally: tick = (100 * 1000) / OS_TICK_US
```

Next chapter: The caOS Kernel

Chapter 8

caOS Features

8.1 caOS Features

Overview of the features offered by caOS, organized by functional area.

8.1.1 1. Scheduler and Kernel

Execution model

Feature	Detail
Scheduling model	weighted proportional, virtual time (inspired by Linux CFS), 1 tick per slice, switch only on task change
Priority levels	7 (from `CA_TH_LEVEL0` to `CA_TH_LEVEL6`)
Task creation	`ca_os_job()` — allocates stack + control block (TCB)
Thread pool	`EN_POOL=1` — pre-allocated reusable pool for dynamic jobs
Block pool	`EN_BLOCK_POOL=1` — fixed-size block allocation O(1)
Context switch	PendSV at end of slice (SysTick); tick configurable per MCU via `OS_TICK_US`
Stack per family	CCM/DTCM for MCUs that have it; SRAM otherwise
MAX_TASK	Configurable per MCU (e.g., H753: 93, U575: 93, F103: limited by SRAM)

System tick

MCU	OS_TICK_US	Clocks/tick
F103	2 ms	144K
F4xx	250 µs	42K
F746/F767	200 µs	43.2K
H563	100 µs	25K
H747/H753	100 µs	40K/48K

MCU	OS_TICK_US	Clocks/tick
U575	250 µs	40K

The tick only costs the SysTick; context switching occurs when the task changes (see S2_02 §8.4, *Slices*).

`ca_os_sleep(ms)` uses an **absolute deadline in ticks** (immune to `yield`); the limit for a single call is ~24.8 days (2^{31} ticks). Details in the Kernel chapter → *Timed suspension.*

Synchronization

Primitive	Header	Notes
Mutex	sync.h	With priority inheritance
Recursive mutex	sync.h	Reentrant from the same task
Semaphore	sync.h	Binary and counting
Condition variable	sync.h	`ca_cond_wait` / `ca_cond_signal`
Spinlock	atomic.h	ISR-safe, LDREX/STREX
RWLock	sync.h	`EN_RWLOCK` — multiple readers / exclusive writer
Barrier	sync.h	`EN_BARRIER` — N-task synchronization
Critical section	atomic.h	`ca_enter_critical` / `ca_exit_critical`

Inter-task communication

Primitive	Header	Features
Queue	queue.h	FIFO/LIFO, static and dynamic, timeout; `CAQUEUE_STATIC_SIZE`
Event Group	event.h	32-bit, wait ANY or ALL
Task Notify	notify.h	Zero overhead, direct between tasks
Stream Buffer	stream.h	Circular bytes with trigger level
Message Buffer	message.h	Messages with atomic framing

Memory

Module	Features
Dynamic allocation	`ca_memory_allocate` — heap with proprietary tracking (task)
MPU	Stack and heap protection (`__MPU_PRESENT`); 8 or 16 regions per MCU
MemAux	`ca_memcpy`, `ca_strlen`, `ca_strncmp` — newlib replacement
Profiling	`ca_measure_start/stop` via DWT CYCCNT (no HW timer occupied)

Reliability

Module	Function
SafeGuard	Recovery policy for tasks: `CA_FAULT_ASSERT`, `CA_FAULT_RESTART_THREAD`, `CA_FAULT_EXEC_EMERGENCY`
Panic	Critical error handler with diagnostic dump via UART/ITM
WDT	Hardware Watchdog IWDG/WWDG, reconfigurable IWDG enabled, reset cause (`ca_wdt_reset_cause()`)
BusFault	Handler `bus.h` with fault address logging

Debug and diagnostics

Tool	Description
CLI (`cli.h`)	UART command-line interface; handlers registerable via `ca_cli_register`
Stdout	Asynchronous channel `ca_stdout_printf` (one task, one queue); `ca_stdout_set_repeater` forwards every line to syslog, flash, file
ITM/SWO	SWO output for debugging with Segger J-Link
Software timer	`EN_SW_TIMER` — up to 16 one-shot and periodic software timers

---

8.1.2 2. Hardware Drivers

Each driver follows a two-level scheme:

```
caos/drivers/src/<periph>.c              extern declarations + API pubblica
hw/<family>/drivers/<periph>_<family>.c      bare-metal CMSIS, no HAL
```

Supported Peripherals

Driver	Header	Families	Notes
GPIO	gpio_driver.h	F1/F4/F7/H5/H7/U5	CRL/CRH on F1xx; MODER on all others
UART	uart.h	F1/F4/F7/H5/H7/U5	Debug + application; RX IRQ
SPI	spi.h	F1/F4/F7/H5/H7/U5	Polling; SPI v3 on H5/U5
I2C	i2c.h	F1/F4/F7/H5/H7/U5	Polling
ADC	adc.h	F1/F4/F7/H5/H7/U5	Polling; F1xx: EXTTRIG+EXTSEL=7 for SWSTART

Driver	Header	Families	Notes
DAC	dac.h	F4/F7/H5/H7/U5	F1xx stub (F103C8T6 no DAC)
DMA	dma.h	F1/F4/F7/H5/H7/U5	Completion semaphore; `in_use_dma` bitmask
EXTI	exti.h	F1/F4/F7/H5/H7/U5	`callbacks[16]`; pending clear differs by family
FLASH	flash.h	F1/F4/F7/H5/H7/U5	F1=16bit, F4/F7=32bit, H7=256bit, H5/U5=128bit
QSPI	qspi.h	F4/F7/H7/H5	F1=stub; F4 MCU_ID guard; H5=OCTOSPI (XSPI)
SDIO	sdio.h	F4/F7/H7/H5/U5	SDHC polling; CLKDIV formula differs by family
CAN	can.h	F1/F4/F7/H7/H5	bxCAN (F1/F4/F7), FDCAN (H7/H5)
TIMER	timer.h	F1/F4/F7/H5/H7/U5	General TIM; U575: no TIM12/13/14
RTC	rtc.h	F1/F4/F7/H5/H7/U5	BCD calendar; access via PWR unlock
WDT	wdt.h	All	IWDG (max 32.7 s; 131 s on H5/U5) + WWDG; reset cause
Temp Sensor	temp_sensor.h	H5/H7/U5	Factory calibration TS_CAL1/CAL2
ETH	eth.h	F4/F7/H7	RMII; HW checksum offload; not on H5/U5
CRC32	crc32.h	All	Hardware CRC
HASH	hash.h	F4/F7/H5/H7/U5	Hardware SHA-1/SHA-256; multithreading context swap

Driver	Header	Families	Notes
AES	aes.h	All	AES-128/256 ECB/CBC/CTR/GCM; HW where available (CRYP H7), otherwise software; context swap
RNG	rng.h	F4/F7/H5/H7/U5	Hardware True RNG
USB CDC	usb_cdc.h	—	(EN_USB_CDC, in development)
USB MSC	usb_msc.h	—	(EN_USB_MSC, in development)
USB NCM	usb_ncm.h	—	(EN_USB_NCM, in development)
USB DFU	usb_dfu.h	—	(EN_USB_DFU, in development)

GPIO RCC differences by family

Family	Register	Bus
F1xx	RCC->APB2ENR	APB2
F4xx	RCC->AHB1ENR	AHB1
F7xx	RCC->AHB1ENR	AHB1
H5xx	RCC->AHB2ENR	AHB2
H7xx	RCC->AHB4ENR	AHB4
U5xx	RCC->AHB2ENR1	AHB2

8.1.3 3. Network Stack

The stack is active with `EN_NET=1` (requires `EN_ETH=1` on MCUs with physical ETH).

Architecture

```
Applicazione


     Servizi applicativi
     MQTT    Modbus    iperf    HTTP    TFTP


     TCP/IP
     tcp_service    udp_server/client
     ip_defrag    route_service
```

```
Infrastruttura L2/L3
ARP    DHCP    DNS    mDNS
NTP    Syslog    Packet Tracer


Buffer pool 64 2048B   |  Ring queue


ETH driver (RMII, HW offload)
```

Modules and status

Module	File	Capacity	HW Status
Buffer pool	net_buf.c	64 × 2048B, O(1), metadata L2/L3/L4	
Ring queue	net_queue.c	Lock-free SPSC ISR/task	
L2 dispatcher	net.c	EtherType → handler table	
Parser/Builder	net_proto.c	ARP, IPv4, UDP, ICMP, DHCP, NTP, DNS, mDNS, Syslog	
ARP service	arp_service.c	Cache + blocking resolve	
Routing	route_service.c	Longest-prefix + GW + CLI	
UDP server	udp_server.c	Max 8 slots	
UDP client	udp_client.c	Max 16 handles, ephemeral ports	
DHCP service	dhcp_service.c	DORA + renew T1/T2	
NTP service	ntp_service.c	3 servers, callback on_sync/on_fail	
DNS service	dns_service.c	Query A record, blocking with timeout	
mDNS service	mdns_service.c	RFC 6762 responder	
Syslog service	syslog_service.c	Queue 8×128B, fire-and-forget	
TCP client	tcp_service.c	PCB pool 4 connections, retransmit	
TCP server	tcp_service.c	Listen table 2, `ca_tcp_accept` blocking	
IP defrag	ip_defrag.c	RFC 791; max 4 sessions, timeout 15s, payload 1480B	

Module	File	Capacity	HW Status
MQTT client	mqtt_service.c	QoS 0; CONNECT/PUB/SUB/PING	
Modbus TCP	modbus_service.c	Client FC01/FC03/FC06/FC16	
iperf	iperf_service.c	TCP client/server; ~25 Mbps avg, 39 Mbps peak	
HTTP server	http_service.c	GET/POST/HEAD/OPTIONS; route table; CLI via HTTP	
TLS 1.3 server	tls.c	TLS_AES_128_GCM_SHA256, X25519, ECDSA P-256 certificate; transport independent	
HTTPS server	https_service.c	Port 443, same routes as HTTP over TLS 1.3	5 Core
Packet tracer	packet_tracer_service.c	Ring 8×20B; hook RX/TX; CLI `trace on/off/filter`	
TFTP	tftp_service.c	RFC 1350; GET/PUT; CLI; OTA-ready	

Network examples

Example	Functionality
39	UDP client/server, DHCP, ARP, NTP, DNS, mDNS, Syslog
40	TCP client — blocking HTTP GET
41	TCP server — echo RFC 862
42	MQTT client — pub/sub
43	Modbus TCP client — CLI `modbus`
44	iperf TCP — CLI `iperf`
45	HTTP and HTTPS server + CLI via HTTP
48	IP defrag service
49	TFTP client/server — CLI `tftp`

8.1.4 4. Cryptography

Hardware accelerators (CRYP, HASH, RNG) are used without HAL where available; where missing, AES, SHA-256 and curves (X25519, Ed25519, ECDSA P-256) run in software.

Hardware accelerators

Module	Header	Algorithms	Notes
CRC32	crc32.h	CRC-32	HW on all families

Module	Header	Algorithms	Notes
HASH	hash.h	SHA-1, SHA-256	Context swap for multithreading
AES	aes.h	AES-128/256 ECB/CBC/CTR/GCM	HW or software; context swap; GCM IV: `[nonce:12][0x00000002:4]`
RNG	rng.h	Hardware TRNG	`ca_rng_get()`

Application infrastructure (`caos/crypto/`)

Module	File	Function	Status
Keys Store	keys_store.c	2048B MPU-protected area, max 20 keys; access `temp_enable`/`lock_completely`	HW tested
AES Channel Service	aes_channel_service.c	Synchronous redblack AES-256-GCM pipe; wire `[IV:12][cipher][tag:16]`; mandatory wipe with `ca_aes_channel_free()`	HW tested 100/100
Keystore Service	keystore_service.c	KDF from password, CLI `kstore`, automatic key ratchet	HW tested
Software primitives	sha256 / sha512 / kdf / x25519 / ed25519 / ecdsa_p256	SSH and TLS 1.3; ECDSA P-256 with RFC 6979 nonce	HW tested (ex66)

Crypto examples

Example	Functionality
46	Encrypted ping-pong between two tasks via `ca_queue_static_t` + AES-256-GCM
47	Keystore CLI: `kstore makekey`, `kstore start`, persistent key management

References: caOS Overview · Kernel · I/O and Drivers · Crypto

Chapter 9

caOS Kernel: Scheduler and Tasks

9.1 Chapter 8 - The caOS Kernel: Scheduling and Task Management

This chapter describes how caOS implements the concepts of kernel, scheduling and stack management.

9.1.1 8.1 The Task in caOS

Task Container (ca_thread_context_t)

Every task in caOS is represented by a `ca_thread_context_t` structure of exactly **180 bytes** (verified with `_Static_assert`, `packed` and aligned to 4). This structure contains everything necessary to describe a task:

```
typedef struct __attribute__((packed, aligned(4))) ca_thread_context {
    u32 digest;             //   0 Validity signature (0xCACAFEF8/FC/F9/FD)
    u32 r[13];              //   4 Registers R0-R12
    u32 lr;                 //  56 Link Register
    u32 pc;                 //  60 Program Counter
    u32 xpsr;               //  64 Program Status Register
    u32 psp;                //  68 Process Stack Pointer
    u32 stack_start;        //  72 Stack start (high address)
    u32 stack_end;          //  76 Stack end (above the canary)
    u16 index;              //  80 Position in the task list (changes!)
    u16 priority;           //  82 EFFECTIVE priority (1,2,4..64): prio_base
   raised by PI
    u16 num_alloc;          //  84 Allocated heap blocks
    volatile u16 status;    //  86 Current state
    volatile u32 notify_value; // 88 Task notification value
    u32 nswitch;            //  92 Context switch counter
    u32 running;            //  96 Tick totali in esecuzione
    u32 sleep;              // 100 Absolute wake-up deadline (ticks)
    void *wait_list;        // 104 Wait list the task is linked into, NULL if
   none
    u32 result;             // 108 Return value
    struct ca_thread_context *next_task_wait; // 112 Link in the wait list
    ca_task_fn_t thread_exec;    // 116 Entry function
    u32 p1;                 // 120 Parametro task
    u32 emerg_param;        // 124 Emergency handler parameter
    ca_emergency_fn_t emergency_exec; // 128 Emergency handler
    u32 runtime_enter;      // 132 CYCCNT at the last entry
    u32 runtime_peak_max;   // 136 Durata massima (WCET) in cicli
    u32 runtime_total;      // 140 Total cycles consumed
    u32 runtime_wdt;        // 144 Task watchdog threshold (0 = off)
    u16 n_assert;           // 148 Safeguard counter
```

```
    u16 n_restart;          // 150 Restart counter
    u16 n_memfault;         // 152 Memory error counter
    u16 prio_base;          // 154 Priority given at creation / by set_priority
    u16 stack_usage_pct;    // 156 0-1000 (0.0%-100.0%)
    u16 fault_type;         // 158 Recovery policy
    struct ca_mutex_s *pi_held; // 160 Owned mutexes (list, priority inheritance
    )
    struct ca_mutex_s *pi_wait; // 164 Mutex the task is blocked on
    char name[12];          // 168 Nome task
} ca_thread_context_t;          // Totale: 180 byte
```

`wait_list` **(starting from 09/27) replaced** `blocked`, a step counter for refresh waiting for nobody to use it anymore. Same offset and same size: the structure thus remained 172 bytes. It is used to remove a task from its wait list while it is waiting — see §8.8.

180 bytes starting from 10/05: `pi_held` and `pi_wait` for priority inheritance (§8.8). The context lives in the TLB, so `MAX_TASK` = TLB size / 180: **22** on 4 KB TLBs (F407, F429, F446, F746), **90** on 16 KB TLBs (H753, H747, U575, H563); the F767 stays at 25 (arena PSP constraint). `ca_scheduler_init()` stops with a message if `MAX_TASK * sizeof(ca_thread_context_t)` exceeds `_tlb_area_size`.

Digest: validity signature

The `digest` field identifies the task type and validity:

Value	Meaning
`0xCACAFEF8`	Standard task (CA_JOB_STD)
`0xCACAFEFC`	Safe task — PSP bounds checking (JOB_SAFE)
`0xCACAFEF9`	Task with floating point (JOB_FLOAT_POINT)
`0xCACAFEFD`	Safe task + floating point (CA_JOB_SAFE_FLOAT_POINT)
`0`	Removed/invalid task

The digest is periodically checked by the idle task. If corrupted → task marked `CA_ST_TH_DIGEST_ERROR` and removed from the scheduler.

Task States

```
CA_ST_TH_INIT           0x80    // Just created
CA_ST_TH_RUN            0x40    // Ready/running
CA_ST_TH_RUNNING        0xC0    // INIT | RUN (active)
CA_ST_TH_SLEEP          0x04    // In sospensione (sveglio a deadline 'sleep')
CA_ST_TH_BLOCKED        0x01    // Blocked on mutex/semaphore/queue
CA_ST_TH_SUSPENDED      0x100   // Suspended manually
CA_ST_TH_STOP           0x02    // Task terminato
CA_ST_TH_REMOVE         0x08    // Marked for removal
CA_ST_TH_STACK_OVERFLOW 0x10    // Stack overflow detected
CA_ST_TH_DIGEST_ERROR   0x20    // Corrupted digest
```

The state `CA_ST_TH_RUNNING` (0xC0) is an OR of INIT and RUN: the scheduler checks `status & CA_ST_TH_RUNNING` to determine if a task is eligible.

Task Types

Type	Digest	Stack Check	FPU	Minimum Stack
CA_JOB_STD	0xCACAFEF8	Canary only	No	STACK_SMALL
JOB_SAFE	0xCACAFEFC	Canary + PSP limits check	No	STACK_SMALL
JOB_FLOAT_POINT	0xCACAFEF9	Canary only	Yes (S16-S31)	STACK_MEDIUM
CA_JOB_SAFE_FLOAT_POINT	0xCACAFEFD	Canary + PSP limits + FP	Yes (S16-S31)	STACK_MEDIUM

FP tasks require at least `STACK_MEDIUM` because saving the S16-S31 registers adds 64 bytes to the stack frame.

9.1.2 8.2 Task Creation

API

```
ca_task_handle_t ca_os_job(const char *name,      // Name (max 10 chars)
              u16 priority,           // CA_TH_LEVEL0..CA_TH_LEVEL6
              ca_task_fn_t func,            // Funzione task: u32 func(void *arg)
              u32 par1,               // Parametro
              ca_stack_size_t stack_size,// Stack size
              ca_job_type type,       // CA_JOB_STD, JOB_SAFE, JOB_FLOAT_POINT ...
              u8 fault_type);         // CA_FAULT_ASSERT, CA_FAULT_RESTART_THREAD...
ca_task_handle_t ca_os_job_suspend(...);   // same signature, created SUSPENDED

bool ca_os_remove_job_handle(ca_task_handle_t h);              // Removes the task
bool ca_os_set_priority_handle(ca_task_handle_t h, u16 prio);  // Cambia priorit  a runtime
bool ca_os_suspend_handle(ca_task_handle_t h);
bool ca_os_resume_handle(ca_task_handle_t h);
bool ca_os_remove_all_jobs(void);     // Rimuove tutti i task
```

Handles, not indices

`ca_os_job()` returns a **handle** (`ca_task_handle_t`), `CA_TASK_HANDLE_NONE` if the creation fails. The handle names the container slot (fixed for the entire lifetime of the task) plus a **generation**: when the task ends, its handle no longer refers to anything, even when the slot is reused. The `_handle` forms resolve and act at masked IRQ, and return `false` if the task is finished.

The **thid** (`ctx->index`, the one printed by `top`) is instead a **position** in the list: when a task is removed, the last one takes its place (swap-with-last), so a thid preserved over time can name **a different task**. The APIs acting on another task by thid (`ca_os_remove_job`, `ca_os_suspend`, `ca_os_resume`, `ca_os_wake_up`, `ca_os_signal`, `ca_os_set_priority`, `ca_os_block`, `ca_notify_send`) are **deprecated**: with `-Werror` their use does not compile. A thid is valid only at the instant it is read; to convert it immediately (e.g. a number typed in CLI) there is `ca_os_handle_from_thid()`.

What happens internally

1. **Allocate task container**: from the TLB area (fixed zone in RAM, defined by the linker)
2. **Allocate stack**: from CCM/DTCM, aligned to 32 bytes (MPU requirement)

3. **Fill stack**: pattern `0xDEADC0DE` for the watermark, but only on the deep part: from `stack_end` up to the address that the task reaches at 65% usage (`STACK_FILL_FROM_PCT`), i.e. 35% of the stack. Count the alarm (usage over 75%), not the exact figure of a task using little: under threshold `top` prints `Stack= <65%`. The filling is done in 1 KB blocks, each at masked IRQ with MPU guard suspended: in a single shot it kept IRQs off for 84.8 µs (16 KB, U575 Release), equal to the margin of a UART without FIFO, and each detached CLI command creates such a task
4. **Set exception frame**: prepares the stack as if the task had been interrupted:

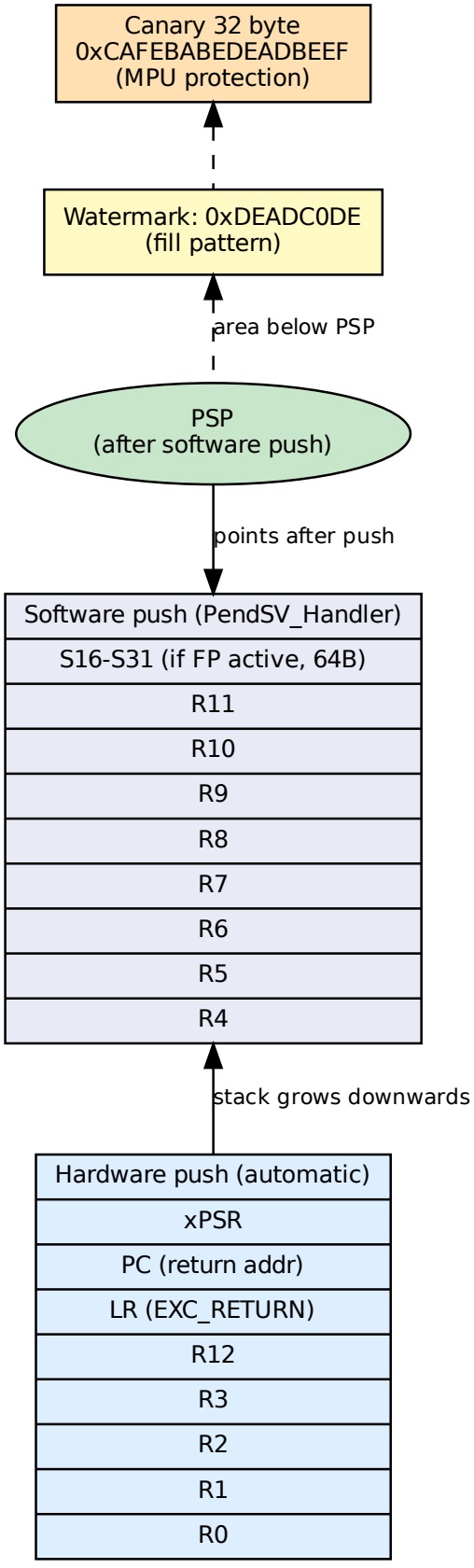
```
Stack (grows downward):
                                                          inizio_stack
      xPSR               0x01000000 (bit Thumb)
      PC                     ca_thread_launch_thread
      LR                 0xFFFFFFFD (EXC_RETURN)
      R12                0
      R3                 0
      R2                 0 (riservato)
      R1                 par1
      R0                 func

     [S16-S31]          (only if JOB_FLOAT_POINT, 64 bytes)

      EXC_RETURN         0xFFFFFFFD (0xFFFFFFED with the FP slot)

      R11..R4            0
                                                             PSP (saved in the
   task container)
        ...              contenuto precedente (non toccato)
                                                             65% used
    RIEMPI_STACK         0xDEADC0DE (watermark, 35% profondo)

    CANARY (32B)         0xCAFEBABEDEADBEEF (protezione MPU)
                                                          fine_stack
```



exception stack frame

5. **Write canary**: 32 bytes of `0xCAFEBABEDEADBEEF` pattern at the base of the stack
6. **Check canary**: immediate check post-write
7. **Add to task list**: digest and initial state (`INIT`, or `SUSPENDED` for `ca_os_job_suspend`) are written **before**; the insertion and reading of the handle occurs with IRQ masked. The two allocations (container and stack) are also masked, because PendSV returns the resources of removed tasks to the same free-list; if the stack is missing, the container is returned.

Launch wrapper

The PC points to `ca_thread_launch_thread`, a wrapper that:

```
static void ca_thread_launch_thread(ca_task_fn_t f, u32 p1) {
    (void)ca_thread_start_task();      // Mark as RUN
    u32 res = f((void *)(uintptr_t)p1);  // Run the task function
    ca_thread_end_task(res);           // Marca come STOP
    while(1) { __WFI(); }
}
```

9.1.3 8.3 Stack Allocation

CCM/DTCM Allocator

Stacks are not allocated from the heap, but from a dedicated allocator on CCM (F4xx) or DTCM (F7xx/H7xx). Advantages:

1. **Zero wait states**: CCM/DTCM is directly connected to the core
2. **Physical isolation**: Stack and heap in separate memories cannot overlap

The allocator uses a bump pointer with exact-fit free-list to recycle the stacks of removed tasks:

```
CCM/DTCM:
                                                  _psp_stack_top (top)
     Stack Task 0          primo stack allocato

     Stack Task 1

     Stack Task 2

        ...               ccm.stack_current
      (free)
                                                  _psp_stack_base (limite)
      Area TLB             Array contenitori task (ca_thread_context_t[MAX_TASK])

```

Why you cannot pass your own stack

Other kernels allow creating a task with an application-supplied stack (in FreeRTOS `xTaskCreateStatic`), to know from the linker where each stack lives. caOS does not, by design:

- **the location is already known**: every stack is described in the task container (`stack_start` , `stack_end`), readable at run time and with `task/stack` from the CLI;
- **stacks are protected**: MPU guard and canary on each one (§8.7), which an arbitrary application stack would not have in a guaranteed way;
- **stacks live in fast memory**: CCM/DTCM, the same as MSP and PSP. An application buffer would usually end up in ordinary SRAM, where the heap also lives.

Available Stack Sizes

```
typedef enum {
    STACK_TINY   = 256,
    STACK_SMALL  = 512,
    STACK_MEDIUM = 1024,
    STACK_LARGE  = 2048,
    STACK_XLARGE = 4096
} ca_stack_size_t;
```

The idle task uses `STACK_MEDIUM`. For application tasks, `STACK_MEDIUM` is a good starting point; tasks that use `sprintf` or large local structures require `STACK_LARGE`.

9.1.4 8.4 Scheduling Algorithm

caOS has **a single policy**: proportional, similar to Linux's CFS. Each task receives the CPU in proportion to its weight; tasks at the same level divide it equally and no task ever starves. The scheduler tick happens at every tick, but context switching only occurs when the scheduler chooses a different task from the currently running one.

Virtual Time

```
vtime = nswitch << log2(64 / priority)      /* u32, compared by signed
    difference */
```

`nswitch` counts the steps in which the task was running and still ready. The levels are powers of two, so the weight is a shift (`__CLZ`), never a division. A task LEVEL6 (priority 64) accumulates vtime slowly and is chosen often, a LEVEL0 (priority 1) quickly and is chosen rarely.

Selection: the task with the lowest vtime wins. In case of tie:

1. the highest actual priority;
2. the one that consumed less CPU (lower `runtime_total`).

A vtime tie is almost always a simultaneous wake-up: two tasks woken up in the same tick both fall back to the same value (see below). The winner then accumulates vtime, and the next round goes to the other.

Level | Weight | Approximate CPU |
|——-|– ——|– ————–| | CA_TH_LEVEL0 | 1 | ~0.78% | | CA_TH_LEVEL1 | 2 | ~1.57% | | CA_TH_LEVEL2 | 4 | ~3.15% | | CA_TH_LEVEL3 | 8 | ~6.28% | | CA_TH_LEVEL4 | 16 | ~12.56% | | CA_TH_LEVEL5 | 32 | ~25.12% | | CA_TH_LEVEL6 | 64 | ~50.24% |

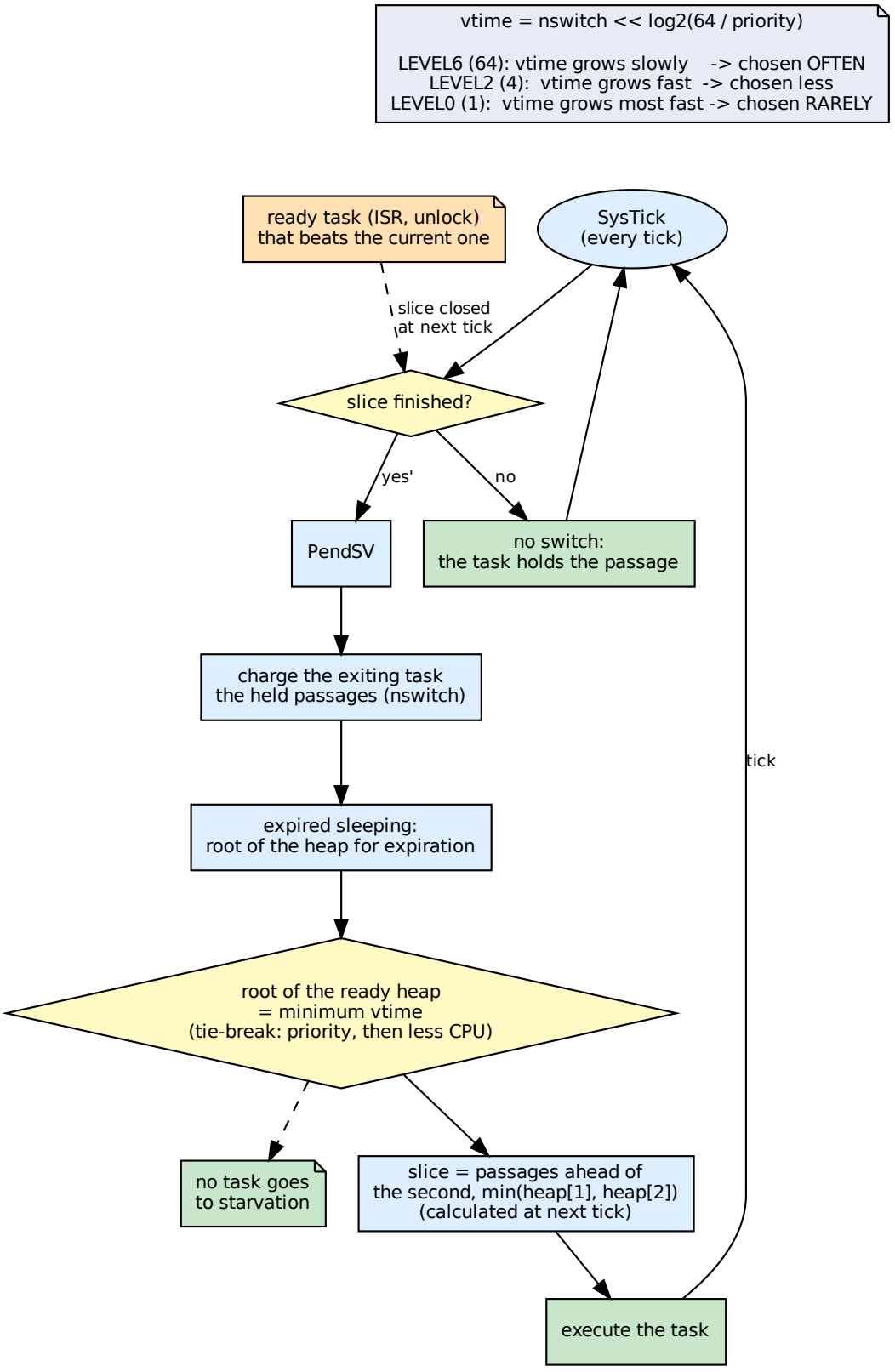


Figure 9.1: caOS CFS scheduling algorithm

Two heaps: ready and sleeping

The choice does not scan the list of tasks. Ready tasks are in a **min-heap** ordered by the selection criterion (vtime, priority, `runtime_total`): the next task is the root. Sleeping tasks are in a second min-heap ordered by **deadline**: the scheduler only looks at the root and wakes up expired sleepers.

- The cost is in **state changes**: a task that becomes ready, blocks, sleeps, is suspended or removed enters or exits a heap in O(log n). All pass through a single point, `task_set_status()`.
- The running task remains in the heap: at every switch only `runtime_total` and `nswitch` change, and it gets repositioned.
- The idle task is not in the heap: it is chosen when the ready heap is empty.
- A task's position in the two heaps is in an array indexed by **slot**, not in the TCB (which remains 180 bytes).

Slices: only context switch when task changes

From the CFS result is derived for how many steps the chosen task will stay ahead of the others, i.e. its **slice**:

```
rivale = min(heap[1], heap[2])          /* the two children of the root are not ordered */
fetta  = 1 + ceil((vtime(rivale) - vtime(task)) / peso(task))     /* 1 in parity */
```

A LEVEL4 ready next to a LEVEL0 pays 4 vtime per tick instead of 64: its slice is 16 ticks, and within those 16 ticks there is only one context switch instead of 16.

The SysTick counts every tick but raises PendSV only when the slice is finished (`ca_scheduler_tick()`). The ticks kept without switching are **charged** to the task at the end of the slice (`nswitch`, `running`): the vtime is what it would have with a full passage at every tick, and thus the order of choices remains the same.

The slice ends early when something changes that the count didn't see:

- **a task becomes ready** (waking up from ISR, unlock, new task): if it beats the current task, the switch occurs at the next tick;
- **a sleeper expires**: the slice never goes beyond the expiration of the first sleeper;
- **a priority changes** (priority inheritance) or **a task is released**: the decision is made at the next tick.

Two limits, in `objmap.h` of the board (default in `scheduler.c`):

define	value	meaning
`CA_SLICE_MAX_TICKS`	32	longest slice: limits the delay of the canary and watchdog checks for tasks
`CA_SLICE_MIN_TICKS`	2	shortest slice: tasks of the same level alternate every 2 ticks

The slice calculation is not in PendSV, whose masked interrupt section is the floor of IRQ latency: PendSV only arms the next tick, and it is that SysTick which calculates the length. The result does not change, because between the choice and that tick the keys remain the same and a task that entered in the meantime is already in the heap.

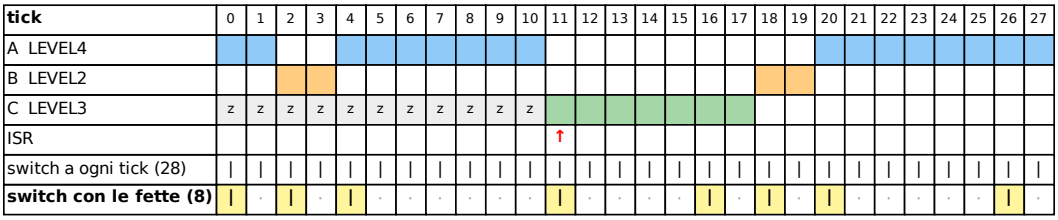


Figure 9.2: Slices: A LEVEL4 and B LEVEL2 always ready, C LEVEL3 made ready by an ISR

Line `slices` of `sched` counts the ticks kept without switching.

The minimum and task reentry (`s_min_vtime`)

`s_min_vtime` (static) is the minimum vtime among ready tasks, updated at every choice and **never lowered**. A task that becomes ready below `s_min_vtime` `64` — just created, restarted, or returning from sleep or wait — is **raised** there:

- reenters next to the least served task, **without** the credit of all the time it did not run: a sleeper, or a new task with `nswitch = 0`, otherwise it would win every passage until it recovers, starving others;
- the credit of **64** (one passage of the lowest level) is placed just **under** the task at the minimum, so it runs before him, like Linux with `min_vruntime`. Placed exactly at the minimum would tie with the current task, and the tie decided by `runtime_total` would sometimes give it the wrong.

During a slice the minimum includes the passages already held by the current task: they are charged before calculating the reentry.

When the minimum exceeds 2^{31} all vtimes are lowered by the same amount (multiple of 64, exact for every weight): distances do not change and `nswitch` (`u32`) never overflows.

Priority Change

The **actual** priority changes with priority inheritance and with `ca_os_set_priority`. It always passes from `ca_scheduler_set_eff_priority()`, which **scales** `nswitch` because the vtime must remain continuous: otherwise, the end of a boost (LEVEL6 → LEVEL2) would multiply the vtime of the task by 16, leaving it without CPU until the others catch up.

The Idle

The idle **does not participate** in the CFS: it is chosen only if no other task is ready. Its turn:

1. periodic time-based checks: digest and PSP every 5 ticks, stack watermark and heap canary every 100 ticks;
2. `idle_wait()`, at masked IRQs: if another task is ready → `ca_os_yield()`, otherwise `WFI`.

The idle slice goes up to the first sleeping task expiring (at most `CA_SLICE_MAX_TICKS`): the SysTick wakes it from WFI at every tick, but without context switch. So the switches at rest depend on the sleeping tasks, not on the tick.

Wake-up from ISR

An ISR that makes a task ready (`ca_scheduler_wake_waiter`, signals, wake-up):

- if it interrupted a **real task**: the awakened task starts at the next tick (latency at most one tick: 100–250 µs, 2 ms on F103). An interrupt never shortens the tick of the current task;
- if it interrupted the **idle**: immediate switch (PendSV) and SysTick restarted from zero, so the incoming task gets a full tick. The already elapsed fraction accumulates (`ca_os_tick_restart_quantum()`) and becomes a tick when it completes one: the clock remains exact. On H563, a QSPI wake-up drops from ~262 µs to ~6 µs.

Line `clock` of `sched` compares ticks with `CYCCNT` (where it counts in WFI): on H563 the deviation stays within ~15 ppm even under hundreds of wake-ups per second.

Why an ISR does not preempt a switch of a real task RTOSes with fixed priority (FreeRTOS, Zephyr, ThreadX) switch at the exit of the ISR if the awakened task is more important than the interrupted one. caOS does not, by design:

- **the tick is a contract.** Preempting the switch would shorten the tick of the interrupted task and lengthen that of the incoming one: `running`, `nswitch` and thus the vtime would count transitions that do not correspond to ticks, and fairness — the very reason of the scheduler — would depend on the interrupt rate;
- **ticks are tight.** caOS runs at 100–250 µs (others typically at 1 ms or more), and with slices a switch only costs when the task changes. So the worst latency of an awakened task is one tick, **bounded and known**;
- **the case that really needs it is covered.** When the CPU is free (idle is running) the switch happens immediately. When it is occupied by a real task, waiting for the tick wastes nothing: one job is finished to start another;
- **what cannot wait a tick must go in the ISR.** Refilling a buffer, responding to hardware, taking a timestamp: short work in the ISR, processing at the task. Those needing task latencies under a tick reduce `OS_TICK_US` on their own board. Briefly: preemption from ISR buys latency under the tick at the price of fairness; with tight ticks that latency is worth little, fairness is not.

9.1.5 8.5 Context Switching

PendSV Mechanism

Context switching follows the classic Cortex-M pattern with two phases:

Phase 1 — SysTick/yield: requests context switching by setting PendSV. SysTick does it at the end of timeslice; a task that yields, blocks or sleeps does it immediately:

```
void ca_os_yield(void) {
    if (SCB->ICSR & SCB_ICSR_PENDSVSET_Msk) return; /* already pending */
    SCB->ICSR |= SCB_ICSR_PENDSVSET_Msk;
    __DSB();
    __ISB();   /* no WFI: whoever yields is ready when it resumes; only the idle
     sleeps */
}
```

Phase 2 — PendSV Handler: performs the actual context switch (in assembly):

1. Saves R4-R11 and EXC_RETURN to the current stack (PSP)
2. If FP task, also saves S16-S31
3. Calls `ca_scheduler_switch_context(current_psp)`:
   - Saves PSP in the current task container
   - Reads the exception frame from the stack to update PC, LR, registers
   - Calls `ca_task_manager_priority_next()`: charges the slice, checks canaries of all stacks, wakes up expired sleepers, chooses the root, arms the tick
   - Updates MPU protection for the new task's canary
   - Returns the PSP of the new task
4. Restores R4-R11 from the new stack
5. If FP task, restores S16-S31
6. Returns (hardware restores R0-R3, R12, LR, PC, xPSR)

Context Switch Statistics

At every context switch, caOS updates, for the outgoing task:

- `running` — execution ticks
- `nswitch` — steps (the base of vtime)

and `g_scheduler_tick`, the number of context switches. At the end of a slice, `running` and `nswitch` also receive the steps held without switching (`slice_settle()`), so that they count ticks as if switching occurred at every tick. The charging is limited to `CA_SLICE_MAX_TICKS`: a clock jump (return from Stop) is not time charged to the task.

```
cpu_permille = (task->running * 1000) / g_scheduler_tick;
```

Scheduler cost

Measured with `sched` on ex87, Release build. `pick` is the time of choosing the next task, inside the context switch; `slices` the steps per second held without switch.

Three loads: *at rest* (the tasks of the examples sleep or wait), `e87 start queue` (ISR every 50 µs, one item at a time towards a consumer) and `e87 start mv_queue` (as `queue`, with the consumer changing index: more tasks, more state changes).

MCU	clock	tick	load	switch/s	cycles per switch	of which `pick`	slices/s	CPU in scheduler
H753	480 MHz	100 µs	at rest	1 504	1 748	1 448	123	0.54%
			`queue`	9 547	1 311	1 001	5 767	2.60%
			`mv_queue`	12 013	1 763	1 456	5 040	4.41%
H563	250 MHz	100 µs	at rest	1 504	862	752	122	0.51%
			`queue`	8 039	736	618	6 211	2.36%
			`mv_queue`	10 645	1 037	921	5 429	4.41%
F767	216 MHz	200 µs	at rest	1 502	1 484	1 232	59	1.03%
			`queue`	8 103	1 159	898	2 518	4.35%

MCU	clock	tick	load	switch/s	cycles per switch	of which `pick`	slices/s	CPU in scheduler
			`mv_queue`	9 961	1 534	1 276	2 323	7.07%
F446	168 MHz	250 µs	at rest	1 503	1 132	886	45	1.01%
			`queue`	5 585	962	715	2 219	3.19%
			`mv_queue`	8 331	1 394	1 148	1 203	6.91%
U575	160 MHz	250 µs	at rest	1 504	744	628	44	0.69%
			`queue`	7 280	635	496	1 863	2.89%
			`mv_queue`	9 797	943	811	1 068	5.77%

At rest, context switches are ~1 500 per second on all MCUs, regardless of the tick: they are made by the sleepers (stdout, cli, example tasks), not by SysTick. Under load `slices` shows how many steps the slices hold without switch: where the tick is finer (100 µs) there are more context switches themselves.

The cost of a context switch has a wanted linear part: **at every context switch PendSV checks the canary of all stacks**, not only that of the exiting task. A task that has not run cannot have damaged its own canary, but another could have; safety comes before cycles, and the check remains complete.

The number of context switches per second does not depend on the tick but on task changes: at rest from sleepers, under load from alternating tasks. The tick (`OS_TICK_US` in `objmap.h`) costs only the SysTick:

MCU	tick
H753, H747, H563	100 µs
F746, F767	200 µs
F407, F429, F446, U575	250 µs
F103	2 ms

The figures apply to the MCUs in the table: on another platform with different clock and tick, they must be remeasured. `sched` prints the distribution from the last call (context switch, task, idle woken up, CPU free in WFI) relative to the real tick time; two calls a few seconds apart give the window.

Interrupt Latency

How long after its event does an ISR start, in the worst case. The measurement is of ex87: TIM3 at the **same NVIC priority as UART RX** and its ISR reads its own counter at the input, so the value read *is* the service delay of a received byte. When the delay falls above another handler the probe records which one (PendSV, SysTick, SVC or the IRQ number).

```
1 e87 lat [sec] [churn|heap|spin]
```

opens a free-running measurement window, to be filled with the load you want; `churn` adds tasks that are constantly born and die (~1000 per second), so that task creation and removal also end up in the measurement. `heap` adds a task that keeps the heap fragmented (64 blocks from 16 to 512 bytes, one freed and one allocated at random at every step, without pauses); `spin`

an always-ready task that touches nothing, to separate the cost of a specific load from that of a task that never sleeps. Besides the maximum, the probe reports the **maximum per source** (thread mode with the interrupted PC, PendSV, SysTick, SVC, IRQ).

The probe period must be incommensurable with the tick. With TIM3 at a submultiple of the tick, the phase between TIM3 and context switching remains fixed for the entire window: TIM3 always hits the same point of the switch, and the maximum depends on the phase picked at opening. For `e87 lat` TIM3 therefore runs at **53 µs** (first): the phase scrolls and in 10 s passes through all points of the switch. The phases of `e87 start` remain at 50 µs.

Build **Release** (in Debug the numbers are much higher and should not be declared), 10 s windows (~189,000 samples), maximum over three repeated runs.

Tasks alive during measurement. Boards with ETH (F767, H753, H563) run 10 tasks: `idle`, `stdout`, `cli`, `example_87`, `e87_rx`, `e87_rxt`, the three network tasks (`net`, `arp`, `dhcp`) and `cli_cmd` which runs `e87 lat`. Those without ETH (F446, U575) run 7: the same minus the three network tasks. The *churn* adds `e87_churn` and the current child: 12 and 9 tasks.

Loads. *at rest*: example tasks sleep or wait, only `stdout` and the tick work. *churn*: as above. *flood*: `examples/example_87/stress_peer.py` sends a ping every 2 ms (ETH interrupt, task `net` busy, ICMP replies).

MCU	clock	tick	tasks alive	at rest	churn	flood	flood + churn	worst
F767	216 MHz	200 µs	10 / 12	9.8 µs	12.6 µs	15.2 µs	16.1 µs	**16.1 µs**
F446	168 MHz	250 µs	7 / 9	9.6 µs	12.4 µs	—	—	**12.4 µs**
U575	160 MHz	250 µs	7 / 9	6.2 µs	8.8 µs	—	—	**8.8 µs**
H753	480 MHz	100 µs	10 / 12	5.1 µs	7.0 µs	7.8 µs	8.7 µs	**8.7 µs**
H563	250 MHz	100 µs	10 / 12	4.2 µs	6.3 µs	7.3 µs	6.4 µs	**7.3 µs**

(*tasks alive*: without / with churn.)

In all measurements, slow samples fall into **PendSV**: context switching runs entirely at masked interrupts (`CPSID I`), and an ISR arriving during the switch waits for it to finish. No peripheral handler appears; critical sections in thread mode (task creation) are light. Therefore, the latency floor is the duration of the longest context switches — those that, by switching tasks, also wake up some sleepers — which depends on clock, core and number of tasks (canary checking goes through all of them). The slices do not shorten those switches, but remove all those that pick the same task: collisions between an IRQ and PendSV become much less frequent.

The margin to compare is that of a UART character at 115200 baud without FIFO, ~87 µs: the worst case measured (F767, flood + churn, 12 tasks) uses less than a fifth. **What these numbers do not cover.** They apply to the kernel with the number of tasks in the table. With more tasks, the switch lengthens, because the canary checks go through them all. Outside the measurement are also the critical sections of the application and drivers (IRQs disabled, spinlocks), the time spent in ISRs with priority higher than UART, Debug builds, and untested MCUs (F103, F407, F429, F746, H747). Latency does not depend on the tick.

Scheduler lock (`ca_os_sched_lock`)

```
void ca_os_sched_lock(void);
void ca_os_sched_unlock(void);
```

Prevents context switches away from the task that takes it until the matching unlock. Interrupts **stay enabled** and are served; if a switch is requested in the meantime (end of slice, wake-up from ISR) the PendSV stays on the task, remembers it, and the switch happens at the last `ca_os_sched_unlock()`. It can be nested. It must not be used from an ISR.

Rule: the holder of the lock **must not block** (sleep, waits on semaphores, queues, events): the PendSV would not switch task and the caller would continue without having waited. If the task holding it is removed because of a fault, the lock goes with it.

What it is for. It is the third way between two tools that cost more than needed when only the other tasks have to stay out:

tool	excludes	cost
`ca_enter_critical()` / IRQ spinlock	tasks and ISRs	interrupt latency for the whole section
mutex	only those who take it	waiting, PI, everyone must take it
`ca_os_sched_lock()`	all other tasks	no IRQ delayed

First use: the heap. `ca_memory_allocate` and `ca_memory_free` are not called from ISRs, but they searched for the free block (next-fit on the chain) with IRQs disabled. The only code outside the tasks that touches the heap is the cleanup of removed tasks in the PendSV, and that is exactly what the lock defers. Measurement of 18/10 with `e87 lat 10 heap` (Release, 10 s windows):

	heap: max thread mode		samples 2 µs thread mode	
MCU	before	after	before	after
F767	15.32 µs	4.00 µs	84,281	423
F446	14.79 µs	4.01 µs	86,537	427
U575	10.34 µs	2.65 µs	17,075	115
H753	7.28 µs	2.17 µs	9,641	11
H563	7.03 µs	< 2 µs	658	0

The **declared maximum does not change**: under load it is set by the PendSV, and the PendSV rises with any always-ready task, not with the heap (`spin`, which does not touch the heap, brings the F767 to 15.2 µs; with the lock, `heap` and `spin` give the same numbers). What changes is the **frequency**: an ISR serving the UART or the network, while a task works on the heap, sees ~4 µs instead of ~15 for all that time. ex87 (the lost wake-up bench) 19/19 on the five boards with the lock active.

Why not `ca_os_run(false)`. It also stops the scheduler, but it is the pre-start "scheduler not active" state: a `CA_SAFEGUARD` triggered in that state takes the fatal branch and halts the board (`[SAFEGUARD FATAL] ... Mode: HALT (IRQ/no-sched)`, tried on H753 with ex68), instead of removing just the task. With `ca_os_sched_lock()` the safeguard stays per-task.

9.1.6 8.6 Idle task and maintenance

The idle task at every iteration:

1. **Every 5 ticks** (`IDLE_CHECK_TICKS`): `update_check_task_validity()` verifies digest and saved PSP of all tasks; if corrupted → `CA_ST_TH_DIGEST_ERROR` / `CA_ST_TH_STACK_OVERFLOW`. The check and verdict occur at masked IRQs, one task at a time, so that a task removed and replaced mid-check is not mistaken for another
2. **Every 100 cycles**: `ca_scheduler_update_all_stack_usage()` — recalculates the watermark with binary search O(log n)
3. **DSB + WFI**: synchronizes memory and puts the core to sleep

9.1.7 8.7 Stack protection

Triple protection

1. **Software canary (32 bytes)**: at the base of every stack, pattern `0xCAFEBABEDEADBEEF`. Verified at every tick
2. **MPU region**: the same area is protected in read-only. A write generates an immediate MemManage Fault
3. **PSP limits check** (SAFE task): the idle task verifies that the PSP of each task is within limits `stack_end..stack_start`

MPU protection is updated at every context switch to point to the base of the stack of the task just loaded.

Without MPU (F103 Cortex-M3)

- MPU functions become empty macros (`#define ca_mpu_init()`)
- Only software canary check remains
- Overflow is detected at the next tick (no immediate protection)

9.1.8 8.8 Suspension, Blocking and Wake-up

Timed Suspension

```
ca_os_sleep(100);  // 100ms      convertito in tick
```

Internally: 1. Converts ms to tick: `tick = ca_os_tick_x_ms(ms)` 2. Marks the task as `CA_ST_TH_SLEEP` and sets `sleep` as **absolute deadline in tick**: `sleep = ca_os_tick() + tick` 3. Calls `ca_os_yield()` 4. The scheduler wakes up the task when the current tick reaches the deadline, with wraparound-safe comparison: `(i32)(ca_os_tick() - sleep) >= 0` 5. Upon reaching the deadline, the task returns to `CA_ST_TH_RUN`

> **Note — time base.** The `sleep` field is an absolute deadline expressed in ticks of `systick_counter`, which advances **only** in `SysTick_Handler` (one increment per real tick) and is therefore immune to voluntary `yield`. Previous versions decremented a counter (`sleep--`) at every context switch (PendSV), including every `ca_os_yield()`: with many tasks yielding sleep resulted in being much shorter than required (measured ~50× on F4VE). The comparison via `(i32)(now - sleep)` handles the wraparound of the 32-bit counter (~49.7 days @ 1 ms/tick).
>
> **Limit.** The `ca_os_tick_x_ms()` conversion works on 64-bit (until 27/09 `ms * 1000` overflowed beyond ~71 minutes) and limits the result to $2^{31}1$ ticks, the maximum that the signed deadline comparison can represent: approximately 24.8 days at 1 ms/tick, 4.9 days at 200 µs/tick. Longer sleeps must be implemented in application loops, not with a single `ca_os_sleep`.

Periodic anti-drift suspension

```
void ca_os_sleep_until(u32 *last_wake_ms, u32 period_ms);
```

Calculate the remaining time to the next wake-up instant and sleep for exactly that duration. `*last_wake_ms` is advanced by `period_ms` on each call, maintaining synchronization with absolute time.

```
static u32 task_periodic(void *arg) {
    (void)arg;
    u32 last_wake = ca_os_tick_ms();
```

```
4
5       while (1) {
6           ca_os_sleep_until(&last_wake, 100);   /* woken every exact 100ms */
7           do_work();
8       }
9       return 0;
10  }
```

Function	Synchronization	Cumulative Drift	Typical Use
`ca_os_sleep(ms)`	Relative (from now)	Yes	Simple delays
`ca_os_sleep_until(&last, ms)`	Absolute (from initial tick)	No	Precise periodic tasks

Block and Wake-up

```
1 ca_os_wait_prepare();        // The current task marks itself BLOCKED (inside the
      lock)
2 ca_os_sleep_prepare(ticks); // ...or SLEEP with a deadline (wait with timeout)
3 ca_os_yield();               // ...and yields the CPU after releasing it
4 ca_os_wake_up_handle(h);     // Wakes from SLEEP (CA_ST_TH_SLEEP)
5 ca_os_signal_handle(h);      // Wakes from BLOCKED (CA_ST_TH_BLOCKED)
```

`wake_up` and `signal` operate on different states: using the wrong function has no effect. Kernel primitives do not use either of them: they wake up with `ca_scheduler_wake_waiter(ctx)`, which applies to both states (below).

Timed waits: in list, like those without timeout

All blocking primitives (code, semaphores, mutexes, recursive mutexes, condition variables, event groups, stream and message buffers, notifications) follow **the same scheme**, with or without timeout:

1. under the primitive's lock, the task rechecks the condition, **registers in the wait list**, and **parks**: `BLOCKED` without timeout, `SLEEP` until expiration with timeout (`ca_os_wait_prepare()` / `ca_os_sleep_prepare()`);
2. releases the lock and yields the CPU;
3. upon awakening, under the lock, **exits the list** and checks what happened.

The timeout value is in milliseconds and means the same for all of them: `CA_NO_WAIT_TIME` (0) does not wait, `CA_WAIT_FOREVER` (`0xFFFFFFFF`) waits with no deadline, any other value is a deadline (`common.h`). Until 19/10 queues, notifications and event groups used `0` for "forever", the opposite of semaphores, mutexes and streams: the same number meant two opposite things depending on the object.

Parking occurs *inside* the critical section: a wakeup arriving between unlock and parking would find the task still `RUNNING` and would be lost (the "lost wakeup" measured by the ex87 bench). The expiration is an absolute tick calculated on entry: subsequent loops do not extend it. At expiration, it is the scheduler refresh that puts the task back into `RUN` (§8.8, timed suspension): no timer is needed.

Until 09/27, waits **with** timeout did not register: they polled with `ca_os_sleep(10)`. Two consequences, both measured:

	polling (before)	in list (now)
delay from signal to awakening	up to 10 ms (45-50 ticks on H753)	0 ticks
to whom the element goes	to whoever checks first (ex4: 107 vs 15)	to whoever waited longer (ex4: 42/40)

Who wakes up. `ca_scheduler_wake_waiter(ctx)` brings a task `RUN` in `BLOCKED` or `SLEEP`, for **context** (never by index: the index is a position and changes when a task below terminates). Returns `false` **only** if the task is terminating (`REMOVE`, overflow, digest, watchdog).

Direct handover. Mutexes, semaphores and rwlocks *deliver* the resource to the waiter that is removed from the list: the mutex changes owner, the semaphore counter is not incremented. For the waiter, therefore, *being removed from the list* means *being served*, even if it has expired in the meantime. The one releasing skips waiters that are terminating: if delivered to a task that will not run again, the mutex would remain theirs forever and the count would be lost.

Mutex owner. The mutex (even recursive) records the owner with its **handle** (`owner`), not with the index: a moved owner in the list from the removal of another task would no longer pass the `ca_mutex_unlock()` check (safeguard, then HardFault). **Priority inheritance (Zephyr schema, from 05/10).** `priority` is the *actual* priority that the scheduler uses as weight, `prio_base` is the one assigned at creation or by `ca_os_set_priority_handle()`. Actual = maximum between `prio_base` and the priority of every task waiting on a mutex owned (`pi_held`, linked list by `pi_next` in the mutex). It is recalculated at every change in the set — a waiter arrives or leaves (lock, timeout, removal), a mutex is passed (unlock), `prio_base` changes — and the change continues on the owner of the mutex the task itself is waiting on (`pi_wait`), for at most `CA_PI_MAX_CHAIN` steps (default 4, `sync.h`): each step runs with IRQ masked, so the chain is limited by construction. The previous schema kept only one `prio_bkup` and restored it at any unlock: releasing the second of two mutexes removed the boost due to the other, a waiter in timeout left the owner boosted, the boost did not follow the chain, and a `set_priority` on an already boosted owner was lost. Recursive mutexes do not have priority inheritance. Verification: ex91 (`pi_*`).

Task removed while waiting. A terminated task (kill, fault, return) while in a wait list remained linked: the list pointed to a freed and reused context from a task created after. Now the `wait_list` field indicates in which list it is, and the scheduler detaches it before freeing it (and at task restart).

Condition variable. `ca_cond_wait()` releases the mutex *before* parking and parks only if it is still in the list. Marked `BLOCKED` with still code to execute, the task was removed from the CPU at the next tick **keeping the mutex** that the notifier needs.

Notifications. The task in `ca_notify_wait()` subscribes to a dedicated wait list, and `ca_notify_send*()` wakes up the recipient only if it is really waiting for a notification. Previously it would wake any task `BLOCKED`, even one blocked on a semaphore.

> **Wake-up order: FIFO.** Priority does not enter the wait list order: the scheduler is a weighted CFS, not strict priority, and a priority-ordered list would starve low-priority tasks on a very busy queue. Priority acts after wake-up, in the scheduler. A timeout expiry does not depend on priority; priority determines how soon a task, already woken, resumes execution.

> **Polling remains only** for `send` with stream and message buffer timeouts when the buffer is full (1 ms).

Verification. The ex87 bank (phases `fair`, `tmo`, `kill`) and the `twait` command in examples 5, 6, 7, 9, 14, 15, 17, 20 (`twkill` in ex20) — see respective S03 chapters.

9.1.9 8.9 System Startup

The complete caOS startup sequence:

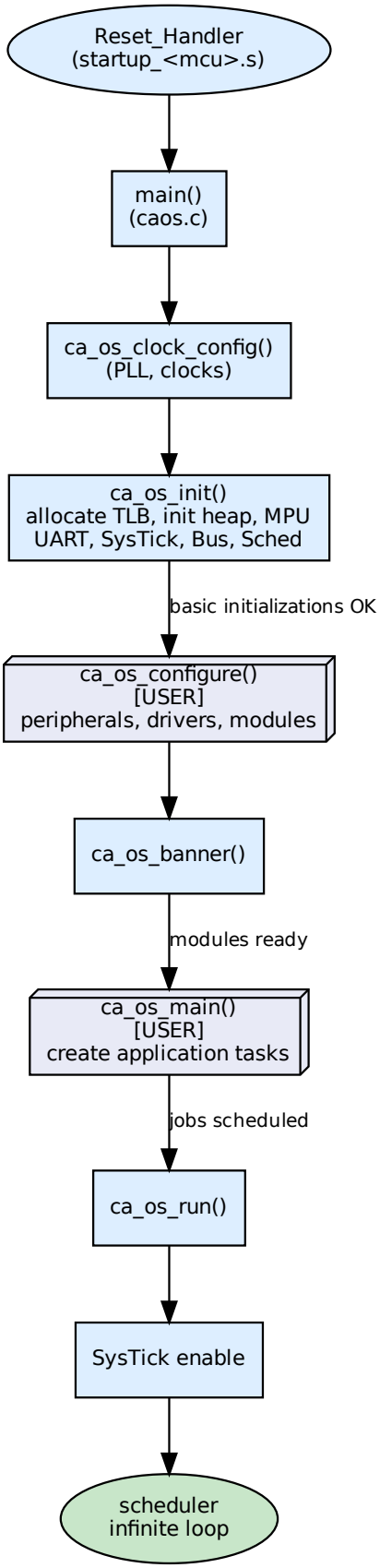


Figure 9.3: caOS system startup sequence

```
Reset_Handler (startup_<mcu>.s)


main() (caos.c)
              ca_os_init()
                        ca_os_clock_config()            clock, PLL
                        ca_itm_init()                   ITM (se ENABLE_ITM)
                        ca_uart_init()                  UART debug (se
    ENABLE_DBG_UART)
                        ca_os_tick_init(OS_TICK_US)     SysTick
                        ca_mpu_init()                   regioni MPU base
                        ca_mpu_protect_canary_msp()     protezione MSP
                        ca_bus_error_init()             abilita BusFault
                        ca_memory_init()                allocatore heap
                        ca_scheduler_init()             allocatore CCM + task
    idle
                                  ca_os_job("idle", ...)
                        ca_os_configure()               UTENTE: periferiche,
    driver, task sistema
                                ca_stdout_init() / ca_cli_init() (opzionali)
                                ...

              ca_os_main()                              UTENTE: crea i task
    applicativi
                        ca_os_job("task_a", ...)
                        ca_os_job("task_b", ...)
                        ca_os_run(true)                   abilita lo scheduler

              ca_os_dead()                              WFI infinito (non
    raggiunto)

SysTick_Handler     SCB->ICSR |= PENDSVSET     PendSV_Handler
        primo cambio contesto     idle task     task_a     ...
```

Previous chapter: caOS Overview Next chapter: Synchronization in caOS

Chapter 10

Synchronization in caOS

10.1 Chapter 9 - Synchronization in caOS

This chapter describes how caOS implements the synchronization primitives presented in Chapter 3.

10.1.1 9.1 Synchronization Architecture

SpinlockIrq as base

All synchronization primitives in caOS (mutex, semaphore, condition variable, queue) internally use a `ca_spinlock_irq_t` to protect their data structures. The interrupt-safe spinlock is implemented with LDREX/STREX and disables interrupts locally:

```
typedef struct {
    volatile u32 lock;   // SPINLOCK_FREE (0) o SPINLOCK_LOCKED (1)
    volatile u32 irq_state; // Saved PRIMASK state
} ca_spinlock_irq_t;
```

This ensures that modifications to synchronization structures are atomic even in the presence of interrupts.

Wait Queue

When a task blocks on a primitive (mutex busy, semaphore at zero, queue empty), it is inserted into a **wait queue**: a linked list of waiting tasks, using the `next_task_wait` field of the task container.

```
typedef struct {
    ca_thread_context_t *head;  // First waiting task
    ca_thread_context_t *tail;  // Last waiting task
} ca_wait_queue_t;
```

The wait queue is FIFO: the first task to block is the first to be woken up. Operations: - `ca_wait_queue_enqueue()` — adds a task to the queue - `ca_wait_queue_dequeue()` — removes and returns the first task - `ca_wait_queue_remove()` — removes a specific task (for timeout)

10.1.2 9.2 Mutex

Structure

```
typedef struct {
    volatile u32 owner;          // owner's handle (ca_task_handle_t)
    volatile u8 locked;          // 0 = free, 1 = locked
    ca_wait_queue_t wait_queue;      // Waiting tasks
    ca_spinlock_irq_t internal_lock; // Internal protection
```

```
} ca_mutex_t;
```

API

```
ca_mutex_t mtx;
ca_mutex_init(&mtx);              // Inizializza

ca_mutex_lock(&mtx);              // Blocks if taken
ca_mutex_trylock(&mtx);           // Does not block, returns true/false
ca_mutex_timedlock(&mtx, 100);    // Blocks at most 100ms
ca_mutex_unlock(&mtx);            // Release (owner only)
ca_mutex_is_owner(&mtx);          // Check ownership
```

Lock Behavior

1. Acquires `internal_lock` (spinlock IRQ)
2. If `locked == 0`: acquires, sets `owner` to the task handle, releases spinlock
3. If `locked == 1`: priority inheritance on the owner, then inserts the task into the wait queue and parks it (BLOCKED, or SLEEP until expiry for `timedlock`) **before** releasing the spinlock, yield
4. On wake-up: if no longer in list, `unlock` has already passed the mutex and returns; otherwise (expiry) retry or return `false`

The owner is an **handle**, not the task index: the index is a position in the list and changes when a task that is below it terminates. With the index, a moved owner would no longer pass the `ca_mutex_unlock()` check (safeguard, then HardFault) — found on 27/09 by the `twkill` command of ex20.

Unlock Behavior

1. Acquires `internal_lock`
2. Checks ownership: only the owner can release
3. If wait queue not empty: removes the first task **that will still run** (skips those that are terminating), transfers ownership to it and wakes it up, from BLOCKED or from SLEEP
4. Otherwise: `locked = 0`
5. Releases spinlock

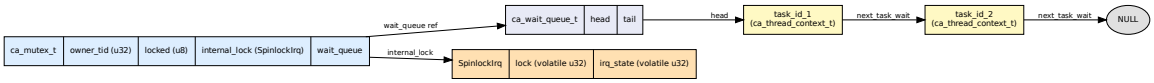


Figure 10.1: Internal structure of ca_mutex_t

Timedlock

The task waits in the same wait list as `ca_mutex_lock()`, in `SLEEP` until expiry: `unlock` wakes it up immediately and in arrival order (FIFO), expiry wakes it up from the scheduler refresh. `timeout_ms = 0` is equivalent to `ca_mutex_trylock()`.

Until 27/09 it was polling (sleep 10 ms + retry): the mutex went to whoever tried again first, up to 10 ms after release. The common scheme for all primitives is described in S2_02 §8.8, “Timeout waits”.

10.1.3 9.3 Recursive Mutex

```
typedef struct {
    volatile u32 owner;          // owner's handle
    volatile u16 lock_count;     // Re-entry counter
    ca_wait_queue_t wait_queue;
    ca_spinlock_irq_t internal_lock;
} ca_recursive_mutex_t;
```

Difference from standard mutex

If the calling task is already the owner, `ca_rmutex_lock()` increments `lock_count` without blocking. `ca_rmutex_unlock()` decrements the counter and releases only when `lock_count == 0`.

```
ca_recursive_mutex_t rmtx;
ca_rmutex_init(&rmtx);

ca_rmutex_lock(&rmtx);    // count = 1
ca_rmutex_lock(&rmtx);    // count = 2 (stesso task, OK)
ca_rmutex_unlock(&rmtx);  // count = 1
ca_rmutex_unlock(&rmtx);  // count = 0     mutex free
```

10.1.4 9.4 Semaphore

Structure

```
typedef struct {
    volatile u32 count;          // Available resources
    u32 max_count;               // Maximum (0 = unlimited)
    ca_wait_queue_t wait_queue;
    ca_spinlock_irq_t internal_lock;
} ca_semaphore_t;
```

API

```
ca_semaphore_t sem;
ca_sem_init(&sem, "SEM_ABC", 0, 10); // nome (8 byte), iniziale=0, max=10

ca_sem_wait(&sem);                 // Decrements, blocks if 0
ca_sem_trywait(&sem);              // Does not block
ca_sem_timedwait(&sem, 100);       // Max 100ms (0 = like trywait)
ca_sem_post(&sem);                 // Hand over to a waiter, or increment
ca_sem_getvalue(&sem);             // Reads count
```

Differences from mutex

	Mutex	Semaphore
Ownership	Yes	No
Value	0/1	0..N
Release	Only owner	Any task/ISR
Typical use	Critical section	Producer/consumer

`ca_sem_post()` with waiting tasks **delivers** the unit directly to the first waiter that runs (FIFO) and does not increment `count`; without waiters it increments. `ca_sem_wait()` and `ca_sem_timedwait()` wait in the same list, BLOCKED or SLEEP until timeout (S2_02 §8.8).

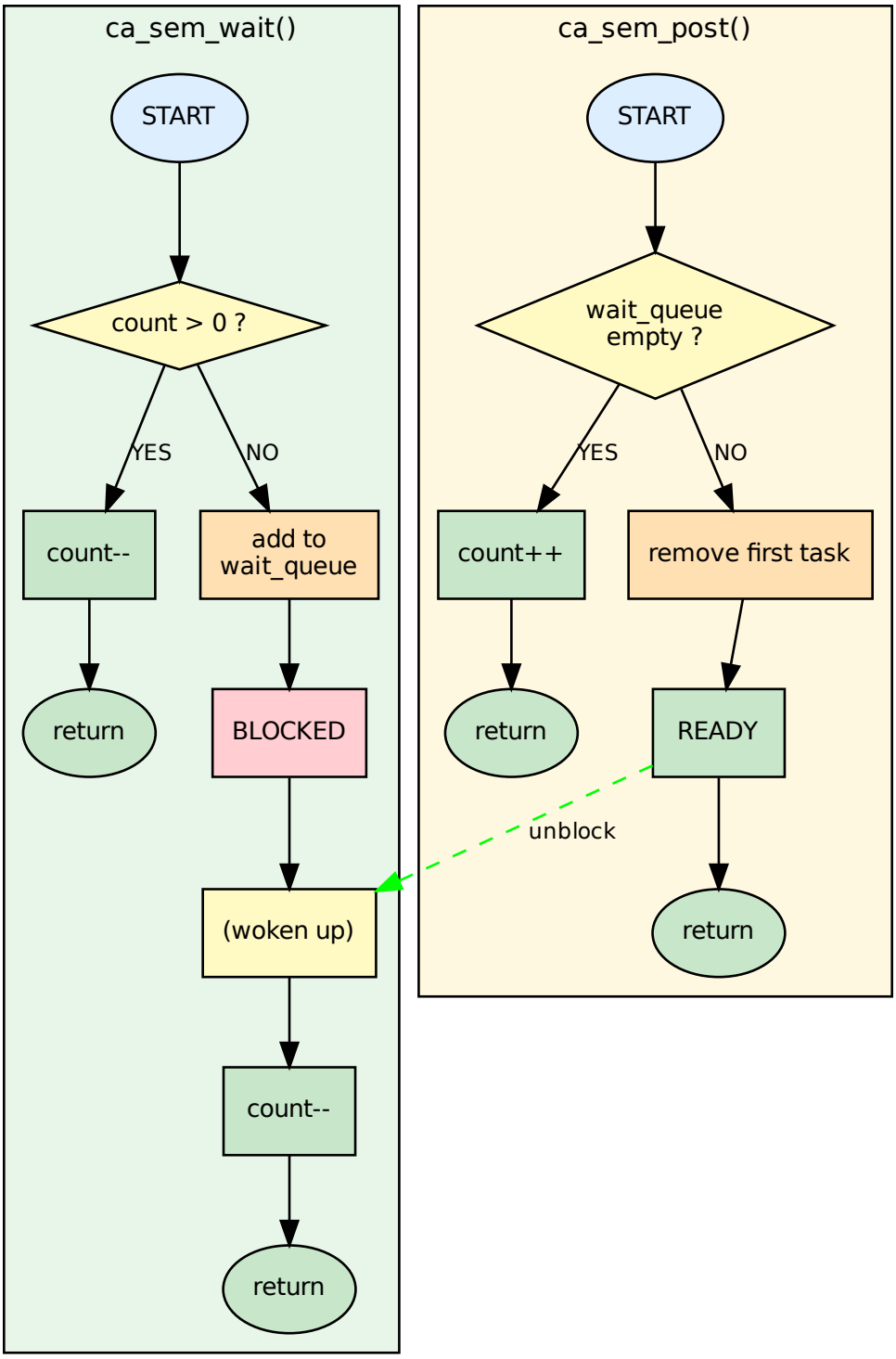


Figure 10.2: Flowchart semaphore wait/post

10.1.5 9.5 Condition variable

Structure

```
typedef struct {
    ca_wait_queue_t wait_queue;
    ca_spinlock_irq_t internal_lock;
} ca_condvar_t;
```

API

```
ca_condvar_t cv;
ca_mutex_t mtx;
ca_cond_init(&cv);

// Attendente
ca_mutex_lock(&mtx);
while (!condizione) {
    ca_cond_wait(&cv, &mtx);      // Releases mtx, blocks, takes mtx back
}
ca_mutex_unlock(&mtx);

// Semaphore
ca_mutex_lock(&mtx);
condizione = true;
ca_cond_signal(&cv);              // Wakes ONE task
ca_cond_broadcast(&cv);           // Wakes ALL
ca_mutex_unlock(&mtx);
```

`ca_cond_wait` executes: 1. Inserts the task into the wait queue (under the cv lock) 2. Releases the mutex 3. If it is still in the list (no signal in the meantime), parks itself and yields the CPU 4. Upon waking up, exits the list and reacquires the mutex before returning

The order matters. Parked (BLOCKED) **before** releasing the mutex, the task was removed from the CPU at the next tick while holding the mutex, and the one who had to signal couldn't acquire it. And a signal arriving between release and parking would not be lost: the task is no longer in the list and won't park.

Timeout: `ca_cond_timedwait(&cv, &mtx, timeout_ms)` returns `false` if it expires (or for a spurious wakeup: the caller always rechecks its condition).

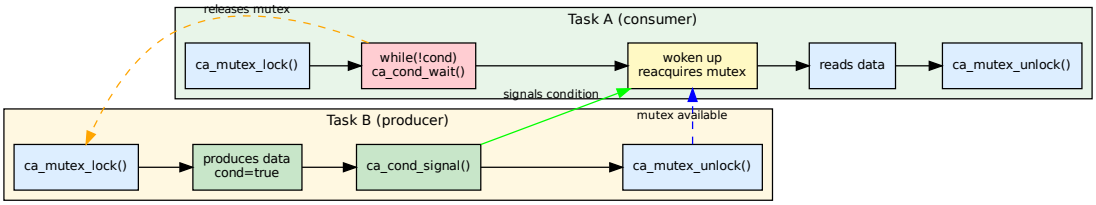


Figure 10.3: Sequence diagram CondVar producer-consumer

10.1.6 9.6 Critical Sections

API

```
ca_enter_critical();     // Disable IRQs with nesting
// ... critical code ...
ca_exit_critical();      // Re-enables only at nesting 0

u32 level = ca_critical_nesting_level();
```

Implementation with nesting counter: the first `ca_enter_critical()` disables interrupts (`CPSID I`), the subsequent ones only increment the counter. `ca_exit_critical()` decrements and re-enables only when the counter returns to zero.

10.1.7 9.7 Code

caOS offers two implementations: **static queue** (inline buffer) and **heap queue** (buffer allocated from the heap).

Static Queue

Definition and usage:

```
// Global definition: 10 slots of sizeof(u32)
CAQUEUE_STATIC_DEFINE(my_queue, 10, sizeof(u32));

// Produttore
u32 dato = 42;
ca_queue_static_send(&my_queue, &dato, 100);        // Timeout 100ms
ca_queue_static_send_front(&my_queue, &dato, CA_WAIT_FOREVER);   // LIFO

// Consumatore
u32 ricevuto;
ca_queue_static_receive(&my_queue, &ricevuto, 100);

// Da ISR
ca_queue_static_send_isr(&my_queue, &dato);          // Never blocks
ca_queue_static_receive_isr(&my_queue, &ricevuto);

// Utilities
ca_queue_static_peek(&my_queue, &dato);               // Reads without removing
ca_queue_static_is_empty(&my_queue);
ca_queue_static_count(&my_queue);
ca_queue_static_reset(&my_queue);                     // Empty
```

Mailbox (one-item queue). `ca_queue_static_overwrite()` and `ca_queue_heap_overwrite()` write the only slot of a queue with `max_items == 1`, whether it is empty or holds data not yet read: the receiver always finds the latest value, never an old one. They never wait and can also be called from an ISR; on a queue longer than one item they refuse (`false`) without touching it. It is the equivalent of FreeRTOS `xQueueOverwrite`. Verification: `overwrite` command of ex4.

```
CAQUEUE_STATIC_DEFINE(state, 1, sizeof(u32));        // a single slot
u32 v = measure();
ca_queue_static_overwrite(&state.q, &v);             // replaces, does not wait
```

Heap Queue

```
ca_queue_t q;
ca_queue_heap_init(&q, sizeof(MyStruct), 20);  // 20 slots of sizeof(MyStruct)
// ... same API as static (send, receive, isr, peek) ...
ca_queue_heap_destroy(&q);                     // Libera buffer heap
```

Internal Implementation

The queues use a circular buffer with indices `head` (read) and `tail` (write). Two separate wait queues: - `wait_send`: tasks blocked because the queue is full - `wait_recv`: tasks blocked because the queue is empty

A send operation that finds the queue full inserts the task into `wait_send`. A receive operation that empties the queue wakes up the first task in `wait_send` (if present).

10.1.8 9.8 Event Groups

Structure

```
typedef struct {
    volatile u32 bits;                      // 32 flag
    ca_wait_queue_t wait_queue;
    ca_spinlock_irq_t lock;
    ca_event_waiter_meta_t waiters[MAX_TASK];   // Metadati per task
} ca_event_group_t;
```

Every waiting task has its own criteria saved in `ca_event_waiter_meta_t`: - `wait_bits`: which bits it waits for - `wait_mode`: ANY or ALL - `auto_clear`: clear bits after match

API

```
ca_event_group_t eg;
ca_event_init(&eg);

// Setter (from task or ISR)
ca_event_set(&eg, CA_EVENT_BIT(0) | CA_EVENT_BIT(1));
ca_event_set_isr(&eg, CA_EVENT_BIT(2));
ca_event_clear(&eg, CA_EVENT_BIT(0));

// Attendente
u32 bits = ca_event_wait_any(&eg, 0x03, true, 1000);   // ANY, auto-clear, 1s
u32 bits = ca_event_wait_all(&eg, 0x03, false, 1000);   // ALL, no clear, 1s

// Query
u32 current = ca_event_get(&eg);
ca_event_reset(&eg);  // Clear all bits
```

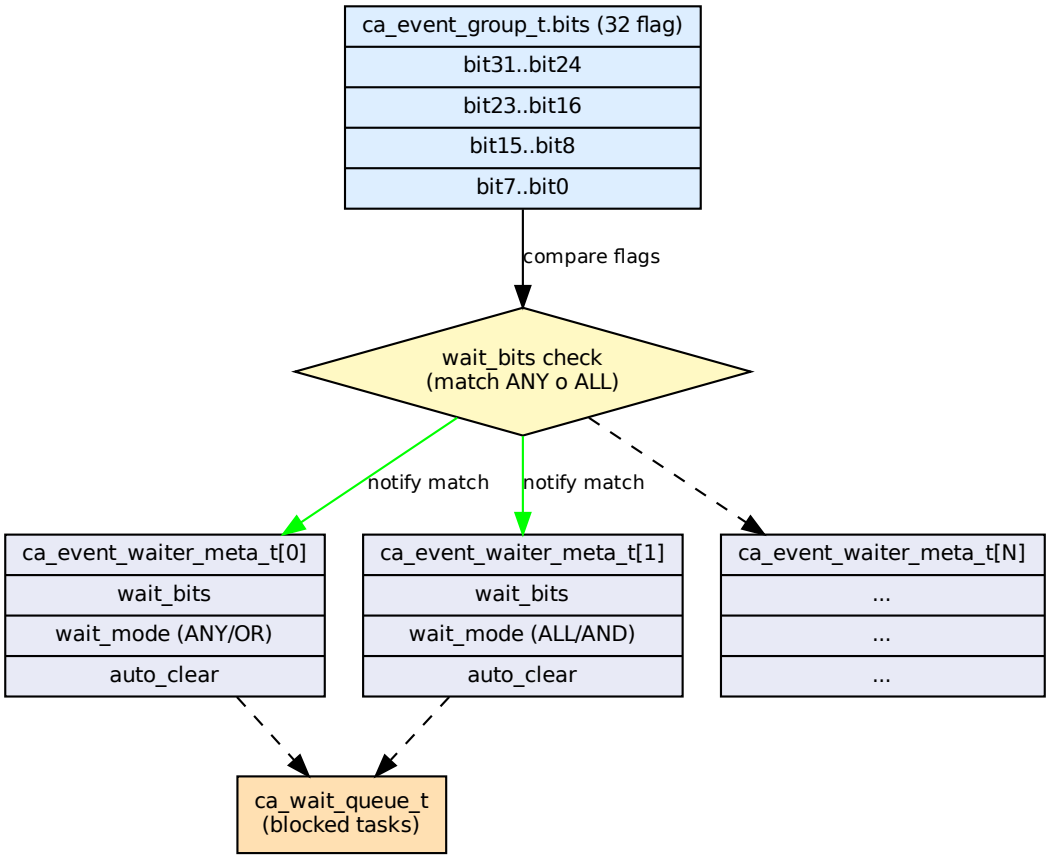


Figure 10.4: Event Group Structure 32-bit

10.1.9 9.9 Task Notification

Concept

Each task container has an integrated `notify_value` (u32) field. No external structure required.

API

```
// Send (6 possible actions)
ca_notify_send(task_idx, value, CA_NOTIFY_ACTION_SET_BITS);     // OR
ca_notify_send(task_idx, value, CA_NOTIFY_ACTION_CLEAR_BITS);   // AND NOT
ca_notify_send(task_idx, value, CA_NOTIFY_ACTION_OVERWRITE);    // Sostituisci
ca_notify_send(task_idx, value, CA_NOTIFY_ACTION_INCREMENT);    // ++
ca_notify_send(task_idx, value, CA_NOTIFY_ACTION_SET_IF_EMPTY);// Write only if
    empty
ca_notify_send_isr(task_idx, value, action);                    // Da ISR

// Ricezione
u32 val = ca_notify_wait(true, 1000);   // Blocks, clear on wake-up
u32 val = ca_notify_wait(false, 1000);  // Blocca, no clear

// Helper
ca_notify_give(task_idx, value);        // = send with OVERWRITE
```

```
ca_notify_give_inc(task_idx);           // = send with INCREMENT
ca_notify_take(timeout_ms);             // = wait con clear
ca_notify_peek(timeout_ms);             // = wait without clear
ca_notify_get(task_idx);                // Non-blocking read
ca_notify_clear(task_idx);              // Azzera
```

`CA_NOTIFY_ACTION_SET_IF_EMPTY` writes the value only if no notification is pending, i.e. if `notify_value` is 0 (the same test as `ca_notify_wait`). If one is already there it touches nothing, wakes nobody and `ca_notify_send` returns `false`: the sender knows the previous data has not been read yet. It is the equivalent of FreeRTOS `eSetValueWithoutOverwrite`. A value of 0 is not a notification: sending it with this action leaves nothing pending.

10.1.10 9.10 Stream Buffer

Circular byte buffer for continuous data streams (e.g. UART reception):

```
// Static definition
CA_STREAM_STATIC_DEFINE(uart_rx, 256);

// Or dynamic
ca_stream_buffer_t sb;
ca_stream_init(&sb, 256);

// Configurazione
ca_stream_set_trigger(&sb, 10);  // Wake the receiver after 10 bytes

// Producer (also from ISR)
ca_stream_send(&sb, data, len, timeout_ms);
ca_stream_send_isr(&sb, data, len);

// Consumatore
u32 letti = ca_stream_receive(&sb, buf, max, timeout_ms);

// Utilities
ca_stream_bytes_available(&sb);
ca_stream_space_available(&sb);
```

The **activation level** is a useful feature: the receiver is not woken up on every byte, but only when at least N bytes are available.

> The two waits are not symmetric. The **receiver** parks itself and is woken up to handle by the producer (`ca_scheduler_wake_waiter_handle()`) when the activation level is reached. The **sender** with a full buffer instead still does **polling**: `ca_os_sleep(1)` and retries, until timeout. It resumes therefore up to 1 ms after space has been freed. The same applies for `ca_message_send()`. With `timeout = 0` no waiting occurs: the stream writes as much as it can and returns the number of bytes written, the message (atomic) writes nothing and returns `false`. In both cases the return value must be checked.

10.1.11 9.11 Message Buffer

Like the stream, but with framing: each message is preceded by a 2-byte header (length). Send and receive are atomic.

```
CA_MESSAGE_STATIC_DEFINE(msg_buf, 512);

// Produttore
ca_message_send(&msg_buf, &my_struct, sizeof(my_struct), timeout);
ca_message_send_isr(&msg_buf, &my_struct, sizeof(my_struct));

// Consumatore
```

```
u16 len = ca_message_receive(&msg_buf, buf, max_len, timeout);

// Query
u16 next_size = ca_message_peek_size(&msg_buf);
u16 count = ca_message_count(&msg_buf);
```

10.1.12 9.12 Read-Write Lock (`ca_rwlock_t`)

Concept

The **Read-Write Lock** allows concurrent read access to N simultaneous tasks, while maintaining exclusive write access. It is more efficient than a mutex when reads are frequent and writes are rare (configuration data, tables, cache).

Write-preferring strategy: a waiting writer blocks new readers, preventing write starvation in the presence of continuous readers.

Structure

```
typedef struct {
    volatile u16  readers;       /* active readers */
    volatile u8   writer_active; /* scrittore in corso */
    ca_wait_queue_t   read_queue;    /* waiting readers */
    ca_wait_queue_t   write_queue;   /* waiting writers */
    ca_spinlock_irq_t internal_lock;
} ca_rwlock_t;
```

API

```
ca_rwlock_t rw;
ca_rwlock_init(&rw);

/* Read (N simultaneous tasks) */
ca_rwlock_read_lock(&rw);     /* block if a writer is active or waiting */
/* ... read shared data ... */
ca_rwlock_read_unlock(&rw);

/* Write (exclusive) */
ca_rwlock_write_lock(&rw);    /* block while readers > 0 or another writer */
/* ... modify shared data ... */
ca_rwlock_write_unlock(&rw);  /* wake waiting writers (priority) or the readers
   */
```

Wake-up Policy

On write release: 1. If there are writers waiting (`write_queue` not empty) → wake up the first writer 2. Otherwise → wake up all readers waiting (`read_queue`)

This ensures that the writer will not wait indefinitely even in the presence of continuous readers.

Comparison with Mutex

	Mutex	RWLock
Concurrent readers	1	N unlimited
Writing	Exclusive	Exclusive
Overhead	Lower	Higher

	Mutex	RWLock
Ideal use	Short critical sections	Read-heavy data structures

Configuration

```
#define EN_RWLOCK  1   /* in os_config.h */
```

10.1.13 9.13 Barrier (`ca_barrier_t`)

Concept

The **Barrier** is a synchronization point: N tasks block until all have arrived, then they all resume simultaneously. It is the classic pattern for coordinating parallel tasks that must proceed in synchronous phases.

```
task_A
task_B
task_C

           ca_barrier_wait()       all blocked here

                                        only one returns true (the trigger)
          ripartono tutti simultaneamente
```

The barrier is **auto-reset**: after release it automatically clears and is reusable in the next cycle without re-initialization.

Structure

```
typedef struct {
    volatile u32  count;         /* numero task da sincronizzare */
    volatile u32  arrived;       /* tasks arrived in this phase */
    volatile u32  phase;         /* current phase (auto-incremented) */
    ca_wait_queue_t   wait_queue;
    ca_spinlock_irq_t internal_lock;
} ca_barrier_t;
```

The `phase` field allows to distinguish successive cycles: a task that has just crossed the barrier sees `phase == N` while tasks from the previous cycle saw `phase == N-1`.

API

```
ca_barrier_t barrier;
ca_barrier_init(&barrier, 3);   /* 3 tasks must arrive */

/* in every participating task: */
bool is_trigger = ca_barrier_wait(&barrier);
/* returns true only for the last task to arrive (the "trigger") */
/* all the others return false */
```

Typical usage

```c
#define N_WORKERS 4
static ca_barrier_t phase_barrier;

static u32 worker(void *arg) {
    const u32 idx = ca_os_thid_from_handle(ca_os_self_handle());
    (void)arg;
    while (1) {
        /* PHASE 1: parallel processing */
        do_phase1_work(idx);

        /* synchronization: waits for everyone */
        ca_barrier_wait(&phase_barrier);

        /* PHASE 2: all phase-1 data are ready */
        do_phase2_work(idx);

        ca_barrier_wait(&phase_barrier);
    }
    return 0;
}

void ca_os_main(void) {
    ca_barrier_init(&phase_barrier, N_WORKERS);
    for (u32 i = 0; i < N_WORKERS; i++)
        ca_os_job("worker", CA_TH_LEVEL2, worker, (void *)(uintptr_t)i,
    STACK_SMALL, CA_JOB_STD, CA_FAULT_ASSERT);
    ca_os_run(true);
}
```

Difference with Event Group

	Event Group (WAIT_ALL)	Barrier
Producers/consumers	Distinct	Same tasks
Auto-reset	Configurable	Always
Number of phases	Single	Unlimited cyclic
Typical use	Data aggregation	Synchronization of parallel phases

Configuration

```c
#define EN_BARRIER  1   /* in os_config.h */
```

10.1.14 9.14 Block Pool (`ca_block_pool_t`)

Concept

The **Block Pool** is a fixed-size block allocator. Compared to the generic heap (`ca_memory_allocate`), it offers:

- **O(1)** for both allocation and deallocation
- **Zero fragmentation**: all blocks have the same size
- **ISR-safe**: usable from interrupt handlers
- **Determinism**: worst-case predictable, no scanning

It is the ideal mechanism for embedded systems with hard real-time requirements or certification constraints (DO-178C, IEC 61508).

Structure

```
typedef struct {
    void         *free_list;   /* first free block (in-place linked list) */
    u32           block_size;  /* size of each block (>= 4, multiple of 4) */
    u32           total;       /* total number of blocks */
    volatile u32  free_count;  /* currently free blocks */
    ca_spinlock_irq_t lock;        /* ISR-safe spinlock */
} ca_block_pool_t;
```

Free list in-place

The free block list does not use external structures: the first 4 bytes of each free block contain the pointer to the next free block. When the block is allocated, the user overwrites those 4 bytes with their own data — no memory overhead.

```
pool_buf (N    block_size byte):


    *next              *next              *next                          NULL

     (free)       (free)       (free)               (free)


  free_list (pool->free_list)
```

API

```
/* Dichiarazione */
static u8 __attribute__((aligned(4))) buf[8 * 64];
static ca_block_pool_t bp;

/* Inizializzazione */
ca_block_pool_init(&bp, buf, sizeof(buf), 64);
/*     8 blocchi da 64 byte */

/* Allocation (O(1), ISR-safe) */
void *blk = ca_block_alloc(&bp);    /* NULL if the pool is exhausted */

/* Release (O(1), ISR-safe) */
ca_block_free(&bp, blk);

/* Query */
u32 free  = ca_block_pool_free_count(&bp);  /* free blocks */
u32 total = ca_block_pool_total(&bp);       /* total blocks */
```

Task switching via queue

The typical pattern is: producer allocates, inserts pointer into a queue, consumer receives and frees:

```
/* Produttore */
MyData *blk = (MyData *)ca_block_alloc(&bp);
if (blk == NULL) { alloc_fail++; continue; }
blk->value = read_sensor();
```

```
u32 ptr = (u32)(uintptr_t)blk;
ca_queue_static_send(&queue.q, &ptr, CA_WAIT_FOREVER);

/* Consumatore */
u32 ptr = 0;
ca_queue_static_receive(&queue.q, &ptr, 500);
MyData *blk = (MyData *)(uintptr_t)ptr;
process(blk);
ca_block_free(&bp, blk);
```

The cast `(u32)(uintptr_t)` is valid on Cortex-M where all pointers are 32 bits.

Sizing

```
N_BLOCKS = sizeof(buf) / block_size

buffer richiesto = N_BLOCKS    block_size byte
(must be 4-byte aligned)
```

The pool must be sized to contain the maximum number of blocks simultaneously in flight (allocated but not yet freed). If the pool is exhausted, `ca_block_alloc` returns NULL without blocking — the caller must handle this case.

Comparison with heap

	`ca_memory_allocate` (heap)	`ca_block_pool_t`
Block size	Variable	Fixed
Fragmentation	Possible	None
Allocation time	O(N) worst	O(1)
ISR-safe	No	Yes
Overhead	Per block	Only pool header

Configuration

```
#define EN_BLOCK_POOL  1   /* in os_config.h */
```

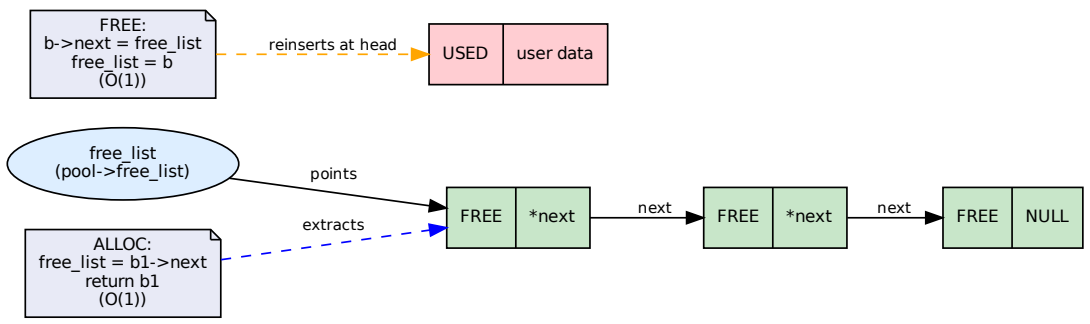


Figure 10.5: Block Pool O(1) Architecture

Previous chapter: caOS Kernel Next chapter: Memory Management in caOS

Chapter 11

Memory Management in caOS

11.1 Chapter 10 - Memory Management in caOS

This chapter describes how caOS implements memory management and protection via MPU, introduced in Chapter 4.

11.1.1 10.1 Memory Layout

Two memories, two uses

caOS physically separates stacks from the heap by leveraging TCM memory:

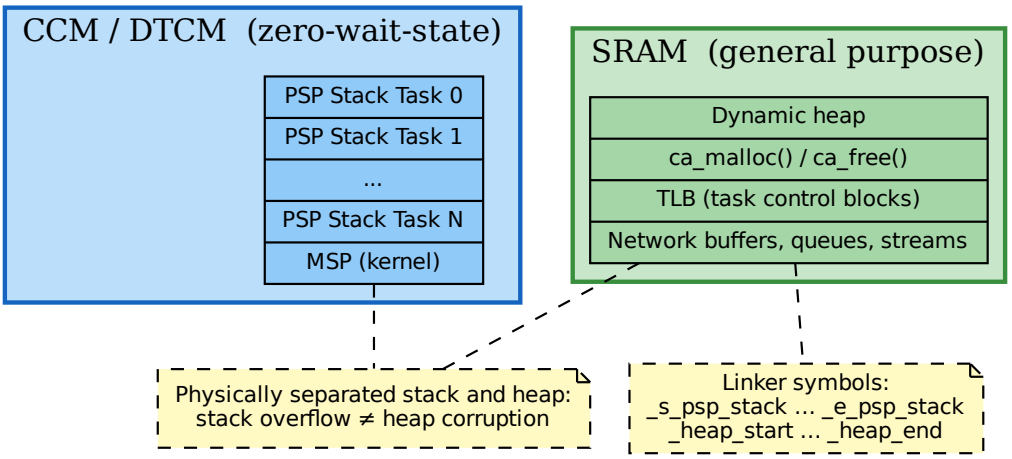


Figure 11.1: Two memories, two uses

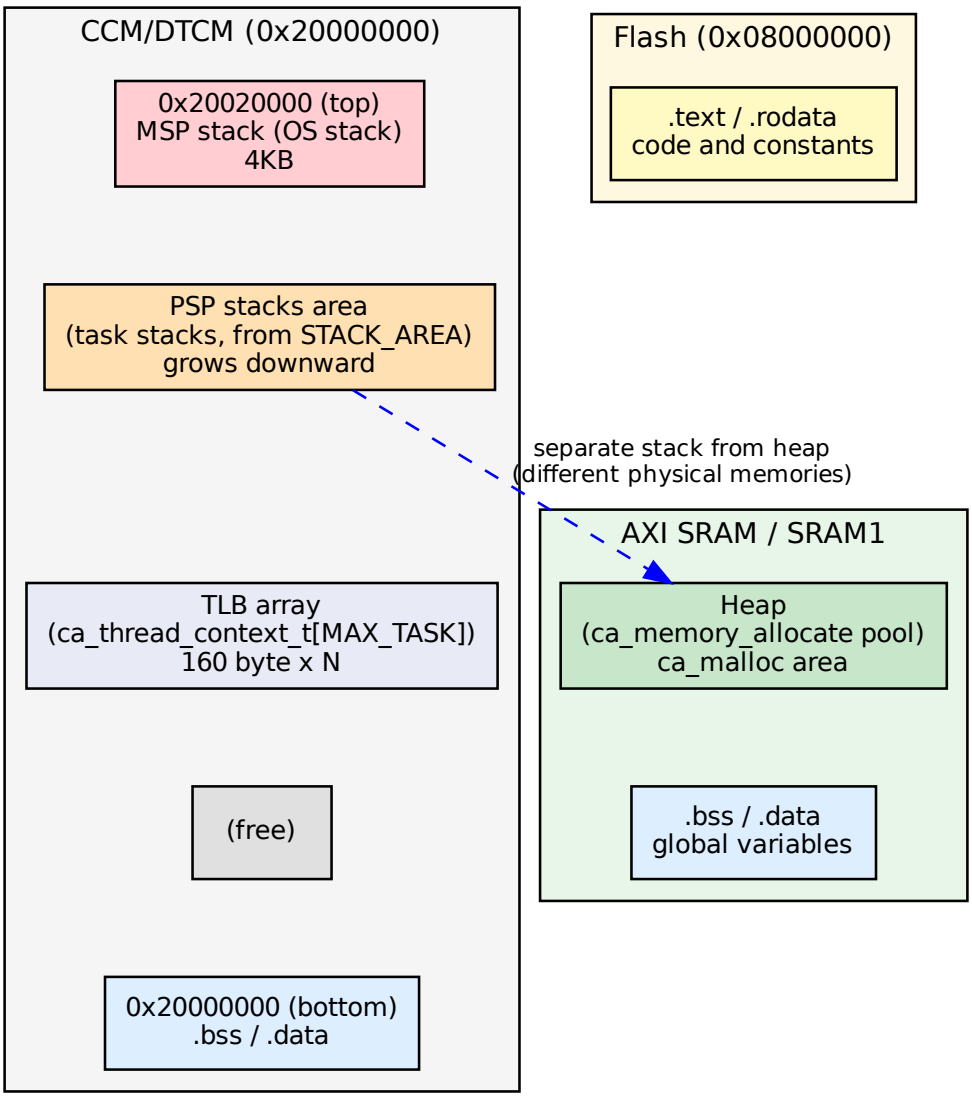


Figure 11.2: STM32H753 memory layout

This separation has important advantages: - **Stacks and heap cannot overlap** (physically different memories) - **Faster stacks** (CCM/DTCM has zero wait states) - **Isolated corruption**: a stack overflow does not corrupt the heap

Linker symbols

The boundaries of the areas are defined in the linker script and accessible as symbols:

```
extern u32 _tlb_area_base;      // Base TLB (array contenitori task)
extern u32 _s_psp_stack;        // Limite basso stack PSP
extern u32 _e_psp_stack;        // Top stack PSP
extern u32 _msp_stack_base;     // Base MSP
extern u32 _msp_stack_top;      // Top MSP
```

```
extern u32 _heap_start;         // Inizio heap (SRAM)
extern u32 _heap_end;           // Fine heap (SRAM)
```

11.1.2 10.2 Heap Allocator

Initialization

```
ca_memory_init(&_heap_start, &_heap_end);
```

The heap occupies the SRAM between the symbols `_heap_start` and `_heap_end`. The size varies by MCU (from a few KB on the F103 to hundreds of KB on the H753).

API

```
// Allocazione
void *ptr = ca_memory_allocate(256);           // Alloca 256 byte (del task chiamante)
void *shr = ca_memory_allocate_shared(256);    // Alloca 256 byte senza proprietario

// Liberazione
u32 freed_size;
ca_memory_free(ptr, &freed_size, true);        // Libera e azzera

// Query
u32 avail = ca_memory_get_avail_memory();      // Byte liberi
u32 total = ca_memory_get_total_size();         // Dimensione totale
u32 usage = ca_memory_get_task_usage(task_idx); // Byte usati da un task
ca_memory_check_t chk; ca_memory_check(&chk);  // Integrita' della catena + spazio (largest_free)

// Debug
ca_memory_list();                               // Stampa lista blocchi
ca_memory_dump(ptr, lines);                     // Hexdump
ca_memory_dump_avail();                         // Mostra blocchi liberi
ca_memory_info(ptr);                            // Info su un blocco
```

Proprietary tracking

The most important feature of the caOS allocator is **proprietary tracking**: every allocated block records the **slot** of the task that allocated it (`owner_slot`).

The slot is the fixed cell of the task container, not its position in the task list: that changes when a task below it terminates, and an owner registered by position would end up naming another task (and causing it to free memory that is not its own).

When a task dies (removed, crash, safeguard), the scheduler automatically calls:

```
ca_memory_free_all_owned_by(task_slot);
```

This frees all blocks allocated by the task, preventing memory leaks. The `num_alloc` field in the task container keeps track of active blocks for each task.

Memory without an owner: `ca_memory_allocate_shared`

Tracking has a downside: an object created by a task and passed to others (a queue, a delivered buffer) dies with the task that created it. For these cases, `ca_memory_allocate_shared()` is identical to `ca_memory_allocate()` but **does not register the owner** (`owner_slot = 0xFFFF`, as with allocations made before the scheduler): the block survives the task's exit and is only freed with `ca_memory_free()`, which is common to both. The responsibility for freeing it lies with the code that uses it: no automatic cleanup collects it.

Concurrency: scheduler lock, not IRQ disabled

Allocation and deallocation exclude other tasks with `ca_os_sched_lock()` (§8.5), not by disabling interrupts: no ISR touches the heap, and the cleanup of tasks removed in PendSV cannot start while a task holds the lock. The chain scan, which on a fragmented heap reached ~15 µs on F446/F767, no longer delays ISRs. Diagnostic functions (`ca_memory_check`, `list`, `dump`, ...) remain with IRQs disabled. `HEAP_SCHED_LOCK 0` in `memory.c` restores the previous behavior for comparison.

Cleanup on free

`ca_memory_free(ptr, &size, true)` zeroes out data before freeing the block. This prevents accidental reuse of sensitive data and facilitates debugging.

Secret area (`EN_TAMP`, paid version only)

On U575 and H563, the **SRAM2** (64 KB) is in the list of hardware tamper "secrets": a tamper in confirmed mode erases it, one in potential mode blocks access to it. However, in the linker script, SRAM2 is inside the single RAM region, and therefore inside the heap: without precautions, a tamper would erase live blocks. On the H563, SRAM2 is just 32 KB below the top of the heap, and the allocator carves blocks right from the top.

With `EN_TAMP = 1`, caOS removes it from the heap **on its own, at startup**, immediately after `ca_memory_init` and before any allocation: it does not depend on the application remembering to do so.

```
1 void *ca_memory_reserve_secret(u32 *size);   /* chiamata da caos.c al boot;
    idempotente */
2 void *ca_memory_secret(u32 *size);           /* l'area, NULL (*size = 0) se la
    MCU non ce l'ha */
```

- The area is determined by the hardware: `CA_SECRET_AREA_BASE` / `CA_SECRET_AREA_SIZE` in the objmap (U575 `0x20030000`, H563 `0x20040000`, 64 KB). On other MCUs it is not defined and `ca_memory_secret()` returns `NULL`.
- It is a heap block marked as occupied and without an owner, which no one frees (`ca_memory_free` rejects it). The block **headers** remain outside the SRAM2: the one for the reserved block is on an 8-byte boundary just below (U575 `0x2002FFE0`), the one for the next free block at the beginning of the SRAM3. The tamper erases only the content, which the allocator does not check, and the chain remains intact (verified with `memlist` after a real tamper on the U575).
- The heap loses 64 KB.
- `keys_store` takes its 2 KB from here (§13.9); the remaining **62 KB are available to the application** for other data to be erased in case of intrusion. The area is 64 KB anyway even if `keys_store` is not compiled.
- With the FREE package, the heap is limited and may not reach the SRAM2: in that case, the area is RAM that no one manages and it is provided as is. However, the tamper is only in the paid version.

The actual tamper is described in the chapter Backup SRAM and tamper.

11.1.3 10.3 MPU Protection

Initialization

```
1 ca_mpu_init();  // Chiamata da main() prima dello scheduler
```

Configure the MPU with the base regions.

System-reserved regions

caOS reserves **3 MPU regions** for the kernel (`OS_REGIONS`):

Region	Usage	Type
0	Canary MSP (Main Stack)	Read-only, 32 bytes
1	Current task stack guard (PSP canary)	Read-only, 32 bytes
2	Extended guard (*deep guard* on ARMv7-M) / top PSP canary if `PSP_CHECK_TOP`	Read-only

Regions 1-2 are updated at every context switch to protect the current task's stack.

Above these, three regions are reserved **only if the corresponding peripheral is active** — therefore they are not contiguous and must be read from the `objmap.h`:

Region	Usage	Condition
3	`.usb_dma` — USB descriptors and buffers	`EN_USB`
4	`.net_pool` — Network buffer zero-copy pool	`EN_NET`
5	`.eth_dma` — ETH descriptor rings	`EN_ETH`

> Precisely because these regions **are not contiguous**, the number of *occupied* regions is not a valid index for the first free one: with NET enabled and USB disabled, the count is 4, while region 4 is occupied by NET. `ca_mpu_add_region()` uses `ca_mpu_first_free_region()`, which scans the regions looking for the first one that is truly disabled (`RASR.ENABLE` on ARMv7-M, `RLAR.EN` on ARMv8-M) **and not reserved** (`ca_mpu_region_is_reserved()`): it always skips kernel regions (0..`OS_REGIONS`-1) and those peripheral regions `USB/NET/ETH_MPU_REGION` defined in `objmap.h`, even if they are currently turned off.
>
> Checking only the enable bit was not enough: before `ca_os_run()` the canaries are not yet armed, so a region requested by `ca_os_configure()` would receive region 1, which the stack guard reprograms at every context switch. The protection would disappear at the first switch without any error (ex26/ex46/ex47, key store; fixed on 09/23). For protection that must last, verify with `mpu` **after** the system is running.

User regions

The remaining regions (from 6 upwards, up to 8 or 16 depending on the part) are available for the application:

```
// Proteggi una regione custom
int region = ca_mpu_add_region(addr, size, MPU_ACCESS_RDONLY);

// Aggiorna permessi di una regione
ca_mpu_update_region(region, addr, size, MPU_ACCESS_RDWR);

// Query
u32 total = ca_mpu_get_total_regions();       // 8 o 16, vedi MPU_REGION_COUNT
u32 free = ca_mpu_get_available_regions();    // Libere per l'utente
u32 busy = ca_mpu_get_busy_regions();         // Occupate

```

```
// Debug
ca_mpu_dump_regions();  // Stampa tutte le regioni attive
```

The number of regions **does not** follow the core: it is an implementation choice of the part, declared in `MPU_REGION_COUNT` in its `objmap.h`.

Part	Core	HW Regions	Free with ETH active
F103	M3	8	— (no ETH)
F407 / F429 / F446	M4	8	2 (F446: 3, no ETH)
F746	M7	8	2
U575	M33	8	3 (no ETH)
H563 / H747 / H753	M33 / M7	16	10

> Be careful not to deduce the count from the core: **U575 is an M33 but has 8 regions**, not 16 like the H563. The tight budget of the 8-region parts is the reason why the two heap boundaries are switched to software canaries.

Canary protection: flow at every context switch

In `ca_scheduler_switch_context()`:

```
// Aggiorna protezione canary per il nuovo task
ca_mpu_protect_canary_psp_start(GET_CANARY_LOWER(g_current_task));
```

This reconfigures MPU region 1 to point to the base of the new task's stack. If the task writes into the canary zone (overflow), the MPU generates an immediate MemManage Fault.

Heap boundary protection: software canary, not MPU

The heap boundaries are protected by a **software canary**, not by MPU regions: 32 bytes at each end (one hexdump line) filled with the `0xAABBCCDD` pattern, placed **outside** the block chain and therefore not allocatable.

```
bool ok = ca_memory_check_canary();   // verificato dall'idle ogni 100 cicli
ca_memory_restore_canary();           // ripristina il marcatore dopo la
    segnalazione
```

The pattern is deliberately different from `START_CHECK_CANARY` (the block checksum seed) and stack canaries, so that an hexdump can distinguish them.

> **Why not the MPU.** Until 08/20/2026, two MPU regions protected the heap boundaries. They were removed: the allocator is **next-fit** and cannot exceed START/END anyway, so those regions covered the least likely case, while an overrun *inside* the heap — which is what actually happens — was not covered in any way. The software canary offers the same coverage and returns **2 MPU regions** to the budget on all families: on F407/F429/F746 with ETH active, it goes from 0 to 2 free regions.

The reaction to a corrupted canary is **message + immediate reset**, not `CA_SAFE_FAIL`: heap corruption is a *global* state, not a fault of the task that detects it, and with `RESTART` policy, the safeguard would enter a loop that cyclically destroys the idle.

Conditionality: `__MPU_PRESENT`

All MPU functions are conditional:

```
#if __MPU_PRESENT
void ca_mpu_init(void) { /* implementazione reale */ }
#else
#define ca_mpu_init()   /* no-op */
#endif
```

On Cortex-M3 (F103) without MPU, all functions become empty macros. Application code does not need `#ifdef`: it compiles on all targets.

11.1.4 10.4 MemAux: newlib replacement

caOS does not use the standard C library (newlib) for memory and string functions. The `memaux` module provides optimized implementations:

```
// Memoria
ca_memset_u32(dst, value, n_words);  // Azzeramento/fill a 32 bit
ca_memcpy(dst, src, n_bytes);        // Copia
ca_memcmp(a, b, n_bytes);            // Confronto

// Stringhe
ca_strlen(str);
ca_strncpy(dst, src, max);
ca_strcmp(a, b);
ca_strncmp(a, b, n);
```

Advantages: - **No dependency on newlib**: reduces binary size by tens of KB - **No calls to `_sbrk`**: newlib requires `_sbrk` for `malloc`, which conflicts with the custom allocator - **Total control**: no unverified external code

11.1.5 10.5 Summary: who allocates what and where

What	Where	Allocator
Task stack (PSP)	CCM/DTCM	CCM incremental allocator
Task containers	TLB area (SRAM/DTCM)	Static array
Main stack (MSP)	Top CCM/DTCM	Defined by linker
Heap (malloc)	SRAM	ca_memory_allocate
Static queue buffer	.bss (SRAM)	Compile-time (macro)
Heap queue buffer	SRAM heap	ca_memory_allocate
Stream/Message buffer	SRAM heap or .bss	User choice
Global variables	.data/.bss (SRAM)	Compile-time

Previous chapter: caOS Synchronization Next chapter: Drivers and I/O in caOS

Chapter 12

Drivers and I/O in caOS

12.1 Chapter 11 - Drivers and I/O in caOS

This chapter describes how caOS implements hardware drivers and the I/O system, introduced in Chapter 5.

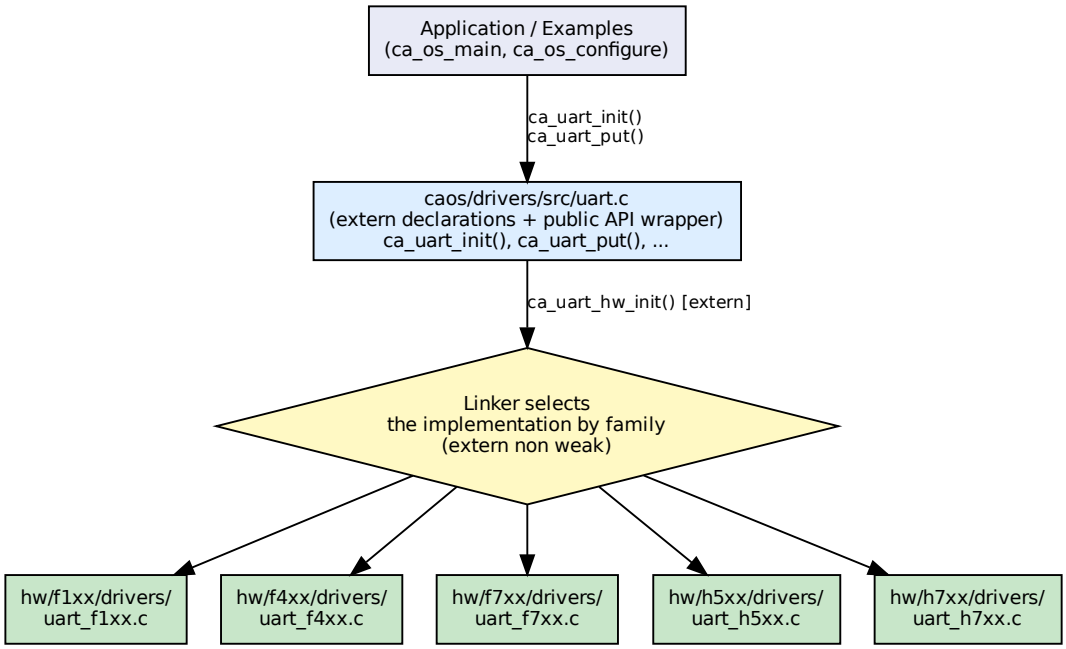


Figure 12.1: caOS driver architecture

12.1.1 11.1 The Extern pattern

Two-level architecture

Each driver in caOS consists of two files:

```
caos/drivers/src/gpio_driver.c          Logica generica + extern declarations
hw/f4xx/drivers/gpio_driver_f4xx.c      Implementazione HW bare-metal (CMSIS)
```

The generic file declares HW-specific functions with `extern` (without implementation). The family-specific file provides the concrete implementations. The linker resolves the references during the link phase: if a family does not implement a function, the linker reports an error.

HW functions **are not** `weak`: every family must implement all of them.

Concrete example: GPIO

Generic file (`caos/drivers/src/gpio_driver.c`):

```
extern GPIO_TypeDef* ca_gpio_get_io_port(ca_gpio_port_t port);
extern bool          ca_gpio_hw_init_port(ca_gpio_port_t port);
extern bool          ca_gpio_hw_configure(ca_gpio_port_t port, ca_gpio_pin_t pin
    ,
                                          ca_gpio_mode_t mode, ...);

// Logica comune: opera sui registri GPIO via le extern
void ca_gpio_configure(u8 port, u8 pin, u8 mode, u8 otype, u8 speed, u8 pull, u8
     af) {
    GPIO_TypeDef *hw = ca_gpio_get_io_port(port);
    if (hw == NULL) return;
    hw->MODER  &= ~(3U << (pin * 2));
    hw->MODER  |=  (mode << (pin * 2));
    // ... ecc
}
```

F4xx specific file (`hw/f4xx/drivers/gpio_driver_f4xx.c`):

```
// Implementazione: mappa ID porta     registro hardware
GPIO_TypeDef* ca_gpio_get_io_port(ca_gpio_port_t port) {
    static GPIO_TypeDef * const ports[] = {GPIOA, GPIOB, GPIOC, GPIOD, GPIOE,
    ...};
    if (port >= ARRAY_SIZE(ports)) return NULL;
    return ports[port];
}
```

HW functions declared extern for drivers

Driver	Extern function	Responsibility
GPIO	`ca_gpio_get_io_port(port)`	Maps port ID → GPIO_TypeDef*
GPIO	`ca_gpio_hw_init_port(port)`	Enables RCC clock for port
UART	`ca_uart_hw_configure_gpio(num)`	Configures pins and AF for UART
UART	`ca_uart_hw_init(num, baud, inst)`	Configures UART registers
UART	`ca_uart_hw_put(inst, msg)`	Transmits string
UART	`ca_uart_hw_recv_handler(inst)`	Handles ISR reception

GPIO clock per family

GPIO clock enabling changes for each STM32 family:

Family	Register	Bit
F1xx	RCC->APB2ENR	RCC_APB2ENR_IOPxEN
F4xx	RCC->AHB1ENR	RCC_AHB1ENR_GPIOxEN
F7xx	RCC->AHB1ENR	RCC_AHB1ENR_GPIOxEN
H5xx	RCC->AHB2ENR	RCC_AHB2ENR_GPIOxEN
H7xx	RCC->AHB4ENR	RCC_AHB4ENR_GPIOxEN

Family	Register	Bit
U5xx	RCC->AHB2ENR1	RCC_AHB2ENR1_GPIOxEN

Each `gpio_driver_<famiglia>.c` implements `ca_gpio_init_port()` with the correct register.

Special case: F1xx

The F1xx has completely different GPIO registers (CRL/CRH with 4 bits per pin). The `gpio_driver_f1xx.c` driver also replaces the basic configuration functions, not just the port mapping.

12.1.2 11.2 GPIO Driver

API

```
// Configurazione
ca_gpio_configure(port, pin, mode, otype, speed, pull, af);
ca_uart_configure_gpio(uart_num);  // Configura pin per UART

// I/O
ca_gpio_write(port, pin, state);   // Usa BSRR (atomico)
ca_gpio_read(port, pin);           // Legge IDR
ca_gpio_toggle(port, pin);         // Toggle via ODR

// Inizializzazione
ca_gpio_init_port(port);           // Abilita clock RCC
```

Atomic writing via BSRR

caOS uses the BSRR (Bit Set/Reset Register) register for GPIO writes. A single write instruction sets or resets a pin, without the read-modify-write sequence that would be necessary with ODR. This is inherently thread-safe.

12.1.3 11.3 Debug UART

Architecture

The debug UART in caOS is a textual output channel, the sender of **stdout**:

```
Task                ca_stdout_printf() / put()     coda     task "stdout"
    sender()

    repeater() (opzionale)
ISR                 ca_stdout_echo() / flush()   (eco della CLI, svuotamento: ISR
    -safe)
Fault handler       PUT() diretto sul sender, senza coda
```

The `sender` is the function that physically writes the message; the `STDOUT` macro selects it according to the configuration: - `ca_uart_dbg_put`: Debug UART (the ST-LINK VCP on Nucleo boards) - `ca_itm_put`: SWO (debug probe) - `ca_lpuart_put`: LPUART, low-power console (see below)

Configuration

In `os_config.h` the channel is selected; the UART instance depends on the board and is located in `hw/<fam>/<mcu>/objmap.h` (USART3 on Nucleo-144, i.e., the ST-LINK VCP; USART1 on F103, F407, and H747):

```
ENABLE_ITM          0                   // os_config.h: output ITM
UART_DBG_BAUDRATE   CA_UART_BAUD_115200   // os_config.h
ENABLE_DBG_UART     1                   // objmap.h
UART_DBG_NUM        3                   // objmap.h: USART3 sulle Nucleo-144
```

UART initialization and transmission

The console does not have a separate driver: it is the generic UART `UART_DBG_NUM`, opened by the kernel in `ca_os_init()` (with `ENABLE_DBG_UART`):

```
ca_uart_init(UART_DBG_NUM, UART_DBG_BAUDRATE,
             STDIN_ON_UART ? ca_cli_rx_byte : NULL,   // RX -> CLI
             UART_DBG_PINMAP);                        // GPIO dentro
    ca_uart_init
```

The stdout sender is instead chosen by the application in `ca_os_configure()`, with the `STDOUT` macro (which equals `ca_uart_dbg_put`, `ca_itm_put`, ... according to the config):

```
ca_stdout_init();
ca_stdout_set_sender(STDOUT);
ca_stdout_create_thread();
```

Internally: 1. `ca_uart_init()` configures the pins according to `UART_DBG_PINMAP`, then the registers with `ca_uart_hw_init()` 2. `ca_uart_dbg_put()` is the sender-compatible wrapper: it calls `ca_uart_put(UART_DBG_NUM, ...)`, busy-waits on TXE 3. Reception is interrupt-driven: the ISR calls `ca_uart_recv_handler()`, which passes each byte to `ca_cli_rx_byte()`

RX: FIFO and cyclic emptying

Without a FIFO, the UART holds **one** byte: at 115200 baud, the ISR has ~87 us to read it before an overrun (ORE), and the lost byte never reaches the parser. This was the cause of the "CLI paste losing characters" (U575: `cli_stat` overrun=4).

On **U5, H5, and H7**, the 8-level RX FIFO is enabled (`USART_CR1_FIFOEN`, writable only with `UE=0`, therefore set before enable). Consequently, `ca_uart_recv_handler()` **loops** as long as `ca_uart_hw_recv_handler()` returns data (cap of 16 per interrupt, so that a line receiving continuously does not hold the ISR forever); a byte discarded due to a line error (FE/NE/PE, return 2) does not interrupt the loop. The margin increases from 1 to 8 characters (~700 us). F1/F4/F7 do not have a FIFO on the USART: there, only ISR latency matters (measured with ex87, see S2_02). Verified on 09/29 on U575: 20 examples with the bench typing at 0 ms between characters, overrun 0. The LPUART is not affected.

LPUART — low-power console

On U5xx/H5xx/H7xx, the LPUART driver (`caos/drivers/src/lpuart.c`) offers an alternative console designed for battery-powered systems:

- **kernel clock HSI16 + HSIKERON**: the baudrate is independent of the sysclk (no retune after `pwr freq` or wake from Stop) and reception remains functional in Stop;
- with `wake=true`, the reception of a character **wakes the MCU from Sleep/Stop** and the byte is delivered to the CLI (it is not lost);
- TX polling (`ca_lpuart_put`, sender-compatible), RX interrupt-driven.

Console configuration in `os_config.h`:

```
STDOUT_ON_LPUART   1        // STDOUT = ca_lpuart_put
STDIN_ON_LPUART    1        // STDIN  = ca_cli_data_from_isr
LPUART_STD_BAUD    115200U
LPUART_STD_WAKE    1        // console armata come wake da Sleep/Stop
```

With both active, `EVAL_LPUART_STD` expands to `ca_lpuart_std_cli()` called by `ca_os_init` (on_rx → `ca_cli_rx_byte`). **Note**: `EN_LPUART` must be defined **before** the STDOUT block in `os_config.h`.

Warning (test with jumpers on the ST-LINK VCP): the board's debug UART (`ENABLE_DBG_UART` in `objmap.h`) must be disabled if its pins are bridged with the LPUART pins — the USART push-pull TX would be in contention. Pins per board and jumper method: see example 62.

12.1.4 11.4 Stdout and repeater

One channel, one task

caOS has **one** textual output channel, the stdout. Tasks write to a static queue of `STDOUT_QUEUE_DEPTH` lines of `DEFAULT_MSG_SIZE` bytes (the depth is in `hw/<fam>/<mcu>/objmap.h`); a dedicated task, "stdout", empties it to the sender. It is the **only** console writer once started: no line will be mixed with another halfway through. The application starts it in `ca_os_configure()`:

```
EVAL_STDOUT(
    ca_stdout_init();
    ca_stdout_set_sender(STDOUT);
    ca_stdout_create_thread();
)
```

API

```
// Da task
ca_stdout_printf("Debug: %d\r\n", val);
ca_stdout_printf_n("Con newline: %d", val);  // aggiunge \r\n
ca_stdout_put("Stringa raw");
ca_stdout_put_n("Con newline");
ca_stdout_put_str(pstr);                     // p_str
ca_stdout_put_str_n(pstr);

// ISR-safe
ca_stdout_echo(buf, len);   // eco della CLI: lo scrive il task stdout fra due righe
ca_stdout_flush();          // scarta le righe in coda (es. all'apertura di una console CDC)

// Statistiche
u32 queued, dropped;
ca_stdout_stats(&queued, &dropped);          // righe in coda e perse (coda piena)
u32 echo_lost = ca_stdout_echo_dropped();    // byte di eco persi
```

Do not print from an ISR. Printing functions format into a task buffer and may block on the queue: an ISR raises a flag or a notification (`ca_notify_give_isr`) and the line is written by the task that wakes up.

Repeater: where to send lines besides the console

The old two-channel audit log is gone: the operating system provides the **mechanism**, the application decides the **policy**. `ca_stdout_set_repeater()` installs a hook called with every line, after the sender:

```
static u16 to_syslog(p_str line)
{
    ca_syslog_send(6U /* informational */, line->data);   // o flash, file, ...
    return 0;
}

void ca_os_main(void) {
```

```
    ca_stdout_set_repeater(to_syslog);
    /* ... */
}
```

The repeater runs in the stdout task — a normal task, scheduler active — so it **can block**, and it is subject to the same backpressure as the sender's queue. The string belongs to the stdout task and is valid only during the call: it must be copied if it needs to persist. It is **never called from a fault handler**: `panic.c` writes directly to the sender with `PUT()`, without a queue, by design (a flash write from handler mode could remain hung on an ISR that will never be serviced).

Conditional compilation

With `ENABLE_STDOUT == 0` all functions become empty macros (those that return a value evaluate to `0`, `false` or `NULL`):

```
#define ca_stdout_printf(...)        0
#define ca_stdout_set_repeater(hook)
```

Application code does not need `#ifdef`. The overhead without stdout is zero.

12.1.5 11.5 CLI (Command Line Interface)

Concept

The CLI is an interactive text interface via UART. It allows querying the system in real-time: task status, memory, MPU, drivers, network peripherals.

Architecture

```
UART RX ISR      ca_cli_rx_byte()      buffer circolare      Coda      Thread CLI
        parser      risposta
```

1. The UART ISR receives a byte and calls `ca_cli_rx_byte()`
2. Bytes are accumulated in a buffer until `\r` or `\n`
3. The complete line is inserted into the CLI queue
4. The CLI thread traverses the handler chain and calls the first one that recognizes the command

Echo. The echo of typed characters no longer exits from the ISR: it goes into a dedicated ISR-safe queue (`ca_stdout_echo()`, 128 one-byte slots) and the stdout drain thread, the sole console writer, empties it before each line. Previously, the echo written by the ISR could end up **inside** a line that a task was outputting byte by byte (`19788t37` in an app_2 capture). Before the drain starts, or without a sender, the CLI still writes directly. Echo bytes that do not fit into the queue are counted in `cli_stat`. A line from a task that arrives while typing still breaks the input line, just like in a PC shell.

Chain of Responsibility

CLI commands are managed via a chain of dynamically registered handlers. Each driver or service adds its own node to the chain via `ca_cli_register()`:

```
static bool my_cli_handler(c_str *pcmd) {
    c_str cmd = *pcmd;
    const char *p = ca_skip_sp(cmd->data);
    if (ca_strncmp(p, "help", 4) == 0) {
        CLI_PUT_N("mydrv info      mostra stato\r\n");
```

```
        return false;   // false = passa al prossimo handler
    }
    if (ca_strncmp(p, "mydrv", 5) == 0) {
        CLI_PRINTF_N("stato: ok");
        return true;    // true = comando gestito
    }
    return false;
}

static ca_cli_node_t s_cli_node = { my_cli_handler, NULL };

// Registrazione (una sola volta, es. in init)
static bool s_registered = false;
if (!s_registered) { ca_cli_register(&s_cli_node); s_registered = true; }
```

Rules: - `return true` — command handled, chain stops - `return false` — proceed to the next node - `help` always returns `false` to show all handlers in the chain - **DO NOT hold mutexes while calling** `CLI_PRINTF_N`/`CLI_PUT_N` (they contain `ca_os_sleep`)

Commands carrying a secret

Typed lines end up in the **history** (up/down arrows, `cli_stat` lists them): a command with a password would remain in RAM and be recallable. Two functions, introduced for `link pass <password> <nome>` of `workspace/app_3`:

Function	Effect
`ca_cli_set_secret_prefix("link pass ")`	as soon as the line being typed starts with the prefix, the following characters return as echo as `*` and the line **does not enter** the history
`ca_cli_hist_forget("link pass")`	clears history entries starting with the prefix, and the ISR line buffer if not currently typing

The handler also clears its own copy of the command after extracting the secret. The bench test of app_3 (`workspace/app_3/test_app3.py`) verifies that the password never appears either in the console or in `cli_stat`.

Built-in driver CLI commands

Driver	Flag `os_config.h`	Commands
`gpio_driver.c`	`EN_CLI_GPIO`	`gpio info <P> <n>`, `gpio all <P>`, `gpio get <P> <n>`, `gpio set <P> <n> <0|1>`, `gpio tog <P> <n>`
`uart.c`	`EN_CLI_UART`	`uart info`
`i2c.c`	`EN_CLI_I2C`	`i2c info`, `i2c scan <1..4>` (with timeout — does not block without devices)

Driver	Flag `os_config.h`	Commands
`adc.c`	`EN_CLI_ADC`	`adc info`, `adc read <1..3> <ch>`, `adc volt <1..3> <ch>`, `adc isr <1..3> <ch>`, `adc stop <1..3>`
`dac.c`	`EN_CLI_DAC`	`dac info`, `dac set <1|2> <0-4095>`, `dac run <1|2>`, `dac stop <1|2>`
`eth.c`	`EN_CLI_ETH`	`eth link`, `eth stats`, `eth dump`
`rtc.c`	`EN_CLI_RTC`	`rtc info`, `rtc set YYYY-MM-DD HH:MM:SS`, `rtc alarm <a|b> HH:MM:SS`, `rtc cal <ppm>`, `rtc unix [<sec>]`
`flash.c`	`EN_CLI_FLASH`	`flash info [s]`, `flash dump [addr]`, `flash erase <addr> <n>`, `flash erase_s <s>`, `flash test <s>`
`timer.c`	`EN_CLI_TIMER`	`timer info [n]`, `timer start <n>`, `timer stop <n>`, `timer free <n>`, `timer cnt <n>`, `timer pwm <n> <ch> <duty>`
`dma.c`	`EN_CLI_DMA`	`dma info`
`spi.c`	`EN_CLI_SPI`	`spi info`, `spi read <n> <addr>`, `spi write <n> <addr> <val>`
`wdt.c`	`EN_CLI_WDT`	`wdt info`
`rng.c`	`EN_CLI_RNG`	`rng get`
`crc32.c`	`EN_CLI_CRC`	`crc32 calc <addr> <len>`
`sdio.c`	`EN_CLI_SDIO`	`sdio info`, `sdio read <block>`
`keys_store.c`	`EN_CLI_KSTORE`	`kstore info`, `kstore list`
`qspi.c`	`EN_CLI_QSPI`	`qspi info`, `qspi read <addr> [len]`, `qspi erase <addr>`, `qspi test <addr>`
`can.c`	`EN_CLI_CAN`	`can info`, `can recv <n>`, `can send <n> <id> [b0..b7]`

Driver	Flag `os_config.h`	Commands
`led.c`	`EN_CLI_LED`	`led info`, `led on <n>`, `led off <n>`, `led tog <n>` (n=1..number of board LEDs)
`lptim.c`	`EN_CLI_LPTIM`	`lptim info [n]`, `lptim start <n>`, `lptim stop <n>`, `lptim free <n>`
`pwr.c`	`EN_CLI_PWR`	`pwr info`, `pwr freq <1-99>`, `pwr restore`, `pwr alarm <s>`, `pwr sleep <s>`, `pwr stop <s>`, `pwr stopms <ms>`, `pwr standby <s>` (`pwr dbg` where the family supports it)
`fat_service.c`	`EN_CLI_FAT`	`fat fdisk`, `fat mkfs`, `fat mount`, `fat umount`, `fat free`, `fat pwd`, `fat cd <dir>`, `fat ls [dir]`, `fat mkdir <dir>`, `fat rmdir <dir>`, `fat touch <file>`, `fat rm <file>`, `fat cat <file>`

Network service CLI commands

Service	Flag `os_config.h`	Commands
`net_proto.c`	`EN_CLI_NET`	`ip info`, `ip set <ip>`, `ip flush`, `ip link up|down`, `ip mtu <N>`, `ping <ip> [count=N] [time=T]`
`arp_service.c`	`EN_CLI_ARP`	`arp info`, `arp flush`, `arp del <n|ip>`, `arp add <mac> <ip>`, `arp req <ip>`
`route_service.c`	`EN_CLI_ROUTE`	`route info`, `route gw <ip>`, `route add <ip/prefix> gw <ip>`, `route del <n>`, `route flush`

Service	Flag `os_config.h`	Commands
`dhcp_service.c`	`EN_CLI_DHCP`	`dhcp info`, `dhcp renew`
`ntp_service.c`	`EN_CLI_NTP`	`ntp info`, `ntp sync`, `ntp server <n> <ip>`
`dns_service.c`	`EN_CLI_DNS`	`dns info`, `dns resolve <host>`, `dns server <ip>`
`mdns_service.c`	`EN_CLI_MDNS`	`mdns info`, `mdns announce`
`syslog_service.c`	`EN_CLI_SYSLOG`	`syslog info`, `syslog put [sev=0..7] <msg>`
`udp_server.c`	`EN_CLI_UDP_SRV`	`udp info`, `udp list`, `udp kick <id>`, `udp send <id> <msg>`
`tcp_service.c`	`EN_CLI_TCP`	`tcp list`, `tcp info <h>`, `tcp slist`, `tcp buf`, `tcp bufreset`, `tcp rxdrop`
`tftp_service.c`	`EN_CLI_TFTP`	`tftp get <ip> <file>`, `tftp put <ip> <file> [size]`, `tftp info`
`http_service.c`	`EN_CLI_HTTP`	`http info`, `http routes`
`mqtt_service.c`	`EN_CLI_MQTT`	`mqtt stat`, `mqtt pub <topic> <msg>` (QoS 0), `mqtt disc`
`modbus_service.c`	`EN_CLI_MODBUS`	`modbus rdreg <addr> [count]` (FC03), `modbus wrreg <addr> <val>` (FC06), `modbus rdcoil <addr> [count]` (FC01), `modbus stat`
`iperf_service.c`	`EN_CLI_IPERF`	`iperf client <ip> [port=5001] [dur_s=10]`, `iperf server [port=5001] [idle_s=3]`
`packet_tracer_service.c`	`EN_CLI_TRACE`	`trace on`, `trace off`, `trace info`, `trace filter src|dst|clear`
`ssh_transport.c`	`EN_CLI_SSH`	`ssh test`

Crypto CLI commands

Module	Flag `os_config.h`	Commands
`sha256.c`	`EN_CLI_SHA256`	`sha256 test`, `sha256 kdf [N]` (HMAC/HKDF/PBKDF2 vectors and time for a PBKDF2 with N iterations)
`sha512.c`	`EN_CLI_SHA512`	`sha512 test`
`ed25519.c`	`EN_CLI_ED25519`	`ed25519 test`, `ed25519 keygen`
`x25519.c`	`EN_CLI_X25519`	`x25519 test`
`keystore_service.c`	`EN_CLI_KSVC`	`ksvc info`, `ksvc lock`

Optional system CLI commands

Flag `os_config.h`	Commands / effect
`EN_CLI_CONFIG`	`config`: build identity (MCU, version) and value of all configuration flags
`EN_CLI_CALLTO`	`callto <addr hex> [param hex]`: calls an address in a one-shot task (**debug only**)
`EN_CLI_CAPTURE`	no command: captures CLI output in a buffer. Required for the remote SSH shell, which otherwise does not execute commands (`ssh_transport.c` emits a `#warning`)

Configuration

```
ENABLE_CLI      1  // In os_config.h     abilita il thread CLI
EN_CLI_GPIO     1  // Abilita CLI per GPIO
EN_CLI_UART     1  // Abilita CLI per UART
// ... ecc. per ogni driver
```

If `ENABLE_CLI == 0`, the entire CLI becomes empty macros.

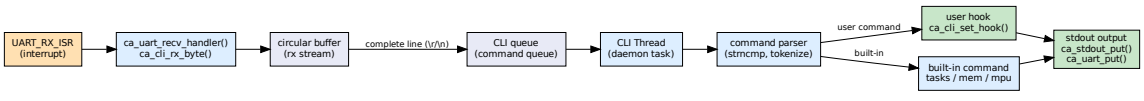


Figure 12.2: UART RX Flow - CLI

12.1.6 11.6 Hardware timers

Architecture

The `timer.c` driver manages hardware timers TIM1..TIM17 (1-based numbering, availability per family) with a bitmask `in_use` and a timer slot. Six modes:

Mode	Init	Usage
PERIODIC	`ca_timer_init_periodic(n, period_us, cb)`	periodic callback from ISR
ONESHOT	`ca_timer_init_oneshot(n, delay_us, cb)`	single callback, then the timer is released
COUNTER	`ca_timer_init_counter(n, &count_u64)`	counts external pulses (ETR), incremental reading
ENCODER	`ca_timer_init_encoder(n, &pos_i64)`	quadrature encoder, signed position
COUNTER_IRQ	`ca_timer_init_counter_irq (n, preset, cb)`	down-counter on ETR, IRQ every `preset` pulses
PWM	`ca_timer_init_pwm(n, ch, freq_hz)`	PWM output, up to 4 channels per timer

After init: `ca_timer_start(n)` / `ca_timer_stop(n)` / `ca_timer_deinit(n)`. For COUNTER/ENCODER, the reading is incremental (`ca_timer_counter_read` / `ca_timer_encoder_read`): the HW delta is accumulated in a caller's `u64`/`i64`, wrap-around safe.

PWM

```
ca_timer_init_pwm(3, 1, 1000);    /* TIM3 ch1 a 1 kHz */
ca_timer_start(3);
ca_timer_set_pwm(3, 1, 250);      /* duty 250/1000 = 25% */
```

- **Duty in permille (0–1000)**: 0.1% resolution, no float. 0 = fixed low output, 1000 = fixed high.
- **Glitch-free**: PWM mode 1 with preload (`OCxPE` + `ARPE`) — the new duty is loaded at the next period, `ca_timer_set_pwm` can be called at any rate.
- **Multi-channel**: frequency is per-timer (shared ARR), duty per-channel. Additional channels: further `ca_timer_init_pwm` on the same timer with the **same** frequency (different → `false`).
- **Automatic PSC/ARR**: maximizes duty resolution (largest possible ARR 65535).
- **Advanced timers** (TIM1/8, plus 15/16/17 on H5/H7/U5): the driver sets `BDTR.MOE` automatically.
- **Pins are the caller's responsibility** (caOS pattern, like the encoder): `ca_gpio_configure (port, pin, CA_GPIO_MODE_AF, ..., af)` before init.

Available channels by family: F1 TIM1-4 (4ch); F4/F7 TIM1-5/8 (4ch), TIM9/12 (2ch), TIM10/11/13/14 (1ch); H5/H7 also TIM12/15 (2ch), TIM13/14/16/17 (1ch); U5 also TIM15 (2ch), TIM16/17 (1ch). TIM6/7 are basic timers without outputs.

Examples: 24 (periodic/oneshot), 25 (counter/encoder), 60 (PWM LED fade).

Input capture

`ca_timer_init_capture` measures the time between rising edges on a channel: at each capture (one every 1, 2, 4, or 8 edges, parameter `div`) the callback receives, from the interrupt, the ticks elapsed since the previous capture. `ca_timer_clock_hz` indicates how many ticks make up one second.

```
bool ca_timer_init_capture(u8 num, u8 ch, ca_timer_ic_src_t src, u8 div,
    ca_timer_ic_cb_t cb);
u32  ca_timer_clock_hz(u8 num);
typedef void (*ca_timer_ic_cb_t)(u32 ticks);    /* timer interrupt */
```

- **Source**: `CA_TIMER_IC_PIN` is the channel pin, which the application must first set to its alternate function (as for PWM). `CA_TIMER_IC_LSE` and `CA_TIMER_IC_LSI` are the internal connections intended for calibration, present on only one channel per family: TIM16/TIM17 CH1 on U5, TIM16 CH1 on H5 and H7, TIM5 CH4 on F4 and F7. Elsewhere, the init rejects the internal source instead of capturing a floating pin.
- **Limits**: the counter runs at 16 bits as in the other modes, so less than 65536 ticks must pass between two captures; TIM1 and TIM8 are not supported, because their capture interrupt is a separate vector.
- The timer handlers only service the sources enabled in `DIER` and clear the flags one by one (`SR = ~flag`): a read-modify-write of `SR` would also clear a capture that arrived between the read and the write.

Example 100 captures the LSE wirelessly and measures its frequency.

LPTIM — low-power timer

The LPTIM driver (`caos/drivers/src/lptim.c`) exposes LPTIM1 on **LSI 32 kHz**: times in milliseconds (1..262144), automatic prescaler /1../128, precision limited by the LSI tolerance (±5%).

```
ca_lptim_init_periodic(1, 500, cb, wake);   /* callback a ogni ARR match */
ca_lptim_init_oneshot (1, 2000, cb, wake);  /* una volta, poi si ferma */
ca_lptim_start(1);  ca_lptim_stop(1);  ca_lptim_deinit(1);
```

- The counter remains active in **Stop** (LSI kernel); with `wake=true` the instance wakes the MCU from Stop/Sleep of the power manager at every ARR match — `ca_pwr_enter(CA_PWR_MODE_STOP, 0, 0)` accepts the input even without other sources ("blink in STOP" pattern of example 61).
- Two IP variants in family files: v1 on F7/H7 (`IER`, +EXTI 23/47 armed by the driver), v2 on H5/U5 (`DIER`/`DIEROK`). F1/F4: absent.
- LPTIM1 is shared with `pwr stopms`: with an active instance the pwr rejects stopms (guard in `ca_pwr_enter`).
- CLI: `lptim info [n] | start | stop | free`.

Examples: 61 (blink periodic/oneshot + phase C in STOP).

12.1.7 11.7 Software Timer

Architecture

caOS implements up to 16 software timers managed by a daemon task:

```
bool ca_sw_timer_init(void);  // Crea il task timer daemon
```

The timer daemon wakes up at every tick and decrements the counters of active timers. When a timer expires, it calls the callback in the context of the daemon task (not in ISR).

API

```c
// Crea timer
ca_sw_timer_handle_t t = ca_sw_timer_create(1000, CA_TIMER_PERIODIC, callback,
    arg);

// Controllo
ca_sw_timer_start(t);
ca_sw_timer_stop(t);
ca_sw_timer_reset(t);
ca_sw_timer_set_period(t, 500);  // Cambia periodo
ca_sw_timer_delete(t);           // Distrugge

// Query
u32 remaining = ca_sw_timer_get_remaining(t);
ca_sw_timer_state_t state = ca_sw_timer_get_state(t);
u8 active = ca_sw_timer_count_active();
```

The callback runs in the daemon task and can stop or rearm **its own** timer: a periodic one that calls `ca_sw_timer_stop()` from its callback remains stopped, while a one-shot that calls `ca_sw_timer_start()` restarts. The daemon rearms or stops the timer only if the callback has not changed its state (corrected on 05/10: previously the periodic one would restart anyway; see ex91 `tmr_stopcb`, `tmr_rearm`).

Callback

```c
void my_callback(void *arg) {
    // Eseguita nel contesto del timer daemon (non ISR)
    // PU  usare mutex, coda, sospensione, ecc.
    ca_gpio_toggle(PORT_A, 5);  // Toggle LED
}
```

12.1.8 11.8 Profiling with Measure

The `measure.h` module measures the clock cycles consumed by the task using `runtime_total` (cycles accumulated by the current task), not the raw DWT CYCCNT. The measurement automatically excludes preemption and IRQs: it only counts the time during which the task is actually running.

```c
u32 snap;
ca_measure_start(&snap);          // salva snapshot di runtime_total
funzione_da_misurare();
ca_measure_stop(&snap);           // snap = cicli task consumati
u32 ms = snap / (ca_os_clock_get_freq() / 1000);
```

The `snap` variable is task-local: start/stop can be nested on different variables. It does not require initialization: caOS enables DWT at startup.

Aspect	Direct DWT→CYCCNT	`ca_measure` (runtime_total)
Count IRQ/preemption	Yes	No
Nested measurements	No (single register)	Yes (local variable)
Initialization	`ca_measure_init()` (deprecated)	automatic
Result	wall-clock cycles	pure task cycles

Available on all Cortex-M3/M4/M7/M33/M55. TIM2 remains free for the application.

12.1.9 11.9 ITM (Instrumentation Trace Macrocell)

ITM is a hardware debug interface present in Cortex-M3/M4/M7 that allows sending text via the debug probe's SWO pin, without occupying a UART:

```
// Configurazione
ENABLE_ITM    1  // In os_config.h

// Uso come sender (STDOUT vale ca_itm_put con ENABLE_ITM)
ca_stdout_set_sender(ca_itm_put);
```

Advantages: no physical pins occupied, no UART peripherals used. Disadvantage: requires a connected debugger.

12.1.10 11.10 Hardware Watchdog (wdt)

The `wdt.h` driver (`EN_WDT` flag) manages the two STM32 watchdogs with the same API: the **IWDG**, which relies on the LSI and is independent of the system clock, and the **WWDG**, which counts on PCLK. The extern pattern (§11.1) follows: `caos/drivers/src/wdt.c` plus `hw/<fam>/drivers/wdt_<fam>.c` for all six families.

API

```
bool ca_wdt_init(u32 timeout_ms, ca_wdt_mode_t mode, ca_wdt_type_t type);
bool ca_wdt_enable(void);                  /* starts and reloads */
void ca_wdt_reload(void);
u32  ca_wdt_get_timeout_ms(void);          /* effective timeout, after HW
    rounding */
bool ca_wdt_is_enabled(void);
u32  ca_wdt_max_timeout_ms(ca_wdt_type_t type);
ca_wdt_reset_t ca_wdt_reset_cause(bool clear);   /* CA_WDT_RESET_NONE | _IWDG |
    _WWDG */
void ca_wdt_on_early(ca_wdt_early_cb_t cb, void *arg);   /*
    CA_WDT_MODE_INTERRUPT */
bool ca_wdt_early_supported(ca_wdt_type_t type);
```

```
ca_wdt_init(2000U, CA_WDT_MODE_RESET, CA_WDT_TYPE_IWDG);
ca_wdt_enable();
/* ... in a dedicated task, every 200-500 ms: */
ca_wdt_reload();
```

Rules

- **Timeout**: from `CA_WDT_TIMEOUT_MIN_MS` (1 ms) to `ca_wdt_max_timeout_ms(type)`, the actual maximum for the family. Values outside this range are rejected.
- **An enabled IWDG cannot be turned off**, but it can be **reconfigured**: a new `ca_wdt_init()` changes the prescaler and reload, and the subsequent `ca_wdt_enable()` reloads the counter with the new value. This is how `ca_os_reset()` and the `reboot` command arm it at 1 ms, and it is what allows an image started by a bootloader to take over an IWDG left enabled. An enabled WWDG, or a change of type, are rejected.
- **Reset cause**: `ca_wdt_reset_cause(false)` reads the RCC flags; `ca_wdt_reset_cause(true)` clears them all (`RMVF`), including those not related to watchdogs (power-on, pin, software). The flags remain at 1 across subsequent resets until someone clears them: whoever reads them at boot clears them, otherwise a watchdog reset from yesterday will be reported again today.
- **Debug**: with a debugger connected (`DHCSR.C_DEBUGEN`) the driver freezes the watchdogs while the core is in halt. Without a debugger, the DBGMCU is not touched.
- **Nominal timeouts**: the LSI is not a crystal and on some parts it deviates from the nominal value by tens of percentage points. The reload period must be chosen with a wide margin (4-10 times).

Maximums by family

Family	IWDG (prescaler)	IWDG max	WWDG max measured (PCLK1)
F1xx	/4 .. /256, LSI 40 kHz	~26 s	— (not on the bench)
F4xx	/4 .. /256	~32.7 s	49 ms (F446, 42 MHz)
F7xx	/4 .. /256	~32.7 s	38 ms (F767, 54 MHz)
H7xx	/4 .. /256	~32.7 s	279 ms (H753, 120 MHz)
H5xx	/4 .. /1024	~131 s	134 ms (H563, 250 MHz)
U5xx	/4 .. /1024	~131 s	419 ms (U575, 80 MHz)

Register differences that the driver absorbs: reset flags in `RCC->CSR` on F1/F4/F7/U5 and in `RCC->RSR` on H5/H7; instances `IWDG1`/`WWDG1` on the H7, where the WWDG1 resets the system only with `RCC_GCR.WW1RSC = 1` (RM0433 §7.7.35), set by the driver; `WDGTB` field at bit 7 on F1/F4/F7 and at 11 on H5/H7/U5.

Early warning (`CA_WDT_MODE_INTERRUPT`)

With `CA_WDT_MODE_INTERRUPT` the watchdog calls, **in interrupt context**, the callback registered with `ca_wdt_on_early()` shortly before the reset:

Watchdog	When	Where
WWDG	counter at `0x40`, one tick before the reset (EWI)	all families
IWDG	with one quarter of the timeout still to elapse (`IWDG_EWCR`, RM0481 §44.6)	H5 and U5 only

On the IWDG of F1, F4, F7 and H7 the interrupt does not exist: `ca_wdt_init()` **rejects** the mode instead of silently arming a simple reset. `ca_wdt_early_supported(type)` indicates in advance if it is available. The reset **happens anyway**: the callback is the last opportunity to save the state (for example, in a backup register, §11.11, to be read at the next boot), not a way to stay alive. Reloading the watchdog from the callback would hide the very fault that left it without a reload. No blocking calls.

```
static void last_gasp(void *arg) { ca_bkp_write(ca_bkp_count() - 1U, 0xE1u); }

ca_wdt_on_early(last_gasp, NULL);
ca_wdt_init(2000U, CA_WDT_MODE_INTERRUPT, CA_WDT_TYPE_IWDG);   /* false on F4/F7
    /H7 */
ca_wdt_enable();
```

CLI

`wdt info` (`EN_CLI_WDT=1`), registered by `ca_wdt_init()`: status, type, mode, effective timeout, maximums, and cause of the last reset. The complete example, with resets triggered and verified upon restart, is example 10.

12.1.11 11.11 Backup registers (bkp)

`bkp.h` provides access to backup registers: a few words in the backup domain that survive a system reset (software, watchdog, pin) and standby. With VBAT powered, they also survive power-off; on Nucleo boards, VBAT is connected to VDD, so a power cycle resets them.

```
u32  ca_bkp_count(void);                /* 10 (F1), 20 (F4), 32 (F7/H7/H5/U5) */
u32  ca_bkp_read(u32 idx);              /* 0 out of range */
bool ca_bkp_write(u32 idx, u32 value);  /* unlocks the backup domain (DBP) itself */
```

On the F1, each register is 16 bits. It does not have a `EN_` flag: they are just a few bytes, and they are used for the transition between the bootloader `workspace/app_2` and the images (registers 16 and 17, `caos/inc/boot_image.h`).

Backup SRAM

In the same backup domain, there is actual RAM, much larger than the registers:

MCU	Size	Address
F446, F767	4 KB	`0x40024000`
H753	4 KB	`0x38800000` (D3 domain)
H563	4 KB	`0x40036400`
U575	2 KB	`0x40036400`
F103	none	—

It is not in the linker script: it is used in blocks, like the registers.

```
u32  ca_bkp_sram_size(void);                                  /* bytes, 0 = none */
bool ca_bkp_sram_read(u32 off, void *dst, u32 len);           /* false when out of range */
bool ca_bkp_sram_write(u32 off, const void *src, u32 len);
bool ca_bkp_sram_retain(void);                                /* backup regulator on */
```

The first call enables the clock and unlocks the backup domain write (`DBP`). It survives system resets, watchdog resets, and Stop mode. For **Standby** and **VBAT**, the backup regulator is required, which `ca_bkp_sram_retain()` enables and waits for (`BRE`/`BREN`). On the U575, that bit can only be written while the regulator is the LDO: with the SMPS already active, it remains as is, and the function indicates this. On Nucleo boards, VBAT is connected to VDD, so a power-off clears it.

The driver only writes **full 32-bit words** (avoiding read-modify-write on unaligned boundaries): on memories with ECC, a shorter write remains pending until the word is completed, and a reset in between causes it to be lost. On the H753, with the first version that copied byte by byte, the upper half of the last written word was missing after a reset.

On the U575 and H563, the backup SRAM has ECC when the option byte `BKPRAM_ECC` enables it: in that case, after power-up, an area must be written before being read. Example 101 fills it, resets the MCU twice, and reads it back. The chapter Backup SRAM and tamper gathers the rules of the backup domain and describes the hardware tamper that clears it.

12.1.12 11.12 CORDIC Coprocessor (cordic)

`cordic.h` uses the CORDIC coprocessor of the **H563 and U575** for trigonometric and hyperbolic functions in q1.31 fixed-point (from -1 to 1-2^{31}). On other families, the coprocessor is not present: `ca_cordic_init()` returns `false` and every call fails.

```
bool ca_cordic_init(void);                          /* false: no CORDIC on this part */
bool ca_cordic_calc(ca_cordic_func_t func, u32 scale, i32 arg1, i32 arg2,
                    i32 *res1, i32 *res2);          /* res2 may be NULL */
```

```
bool ca_cordic_sincos(i32 angle, i32 *sin_q31, i32 *cos_q31);   /* angle = rad/pi */
bool ca_cordic_polar(i32 x, i32 y, i32 *angle, i32 *modulus);   /* angle = rad/pi */
```

`ca_cordic_calc` covers the ten coprocessor functions with the conventions of the reference manual (RM0456 §25.3.2, RM0481 §20.3.2): angles are in radians divided by , and functions that exit the q1.31 range are scaled by 2 with the `scale` parameter.

`func`	arg1	arg2	res1	res2	`scale`
`CA_CORDIC_COS, _SIN`	angle/	modulus m	m · cos or m · sin	the other	0
`CA_CORDIC_PHASE`	x	y	atan2(y,x)/	modulus	0
`CA_CORDIC_MODULUS`	x	y	modulus	phase/	0
`CA_CORDIC_ATAN`	x · 2	—	atan(x)/(· 2)	—	0..7
`CA_CORDIC_COSH, _SINH`	x/2	—	cosh/2 or sinh/2	the other	1
`CA_CORDIC_ATANH`	x/2	—	atanh(x)/2	—	1
`CA_CORDIC_LN`	x · 2	—	ln(x)/2^1	—	1..4
`CA_CORDIC_SQRT`	x · 2	—	x/2	—	0..2

The driver always performs 24 iterations, the maximum useful in q1.31: residual error is below 2^1 on circular functions and below 2^1 on hyperbolic ones, plus 2^2 rounding. A higher scale loses precision proportionally.

Each operation (configuration, arguments, results) runs in a critical section of a few dozen cycles, so multiple tasks can use the coprocessor. It is called from a task, not from an ISR. A `ca_cordic_sincos` costs 136 cycles on the U575 at 160 MHz and 213 on the H563 at 250 MHz, including checks and the critical section: the actual calculation is 6 cycles.

Like `bkp`, it does not have a `EN_` flag: code that is not called is discarded by the linker. Example 98 verifies all functions against the PC's libm.

12.1.13 11.13 FMAC filter accelerator (fmac)

`fmac.h` uses the FMAC (Filter Math Accelerator) of **H563 and U575** for FIR and IIR filters in q1.15 fixed-point. It is only in the paid version (`EN_FMAC`); on other families the unit is not present and `ca_fmac_init()` returns `false`.

```
typedef struct {
    const i16 *b;  u32 nb;     /* feed-forward b[0..nb-1] */
    const i16 *a;  u32 na;     /* feedback a[1..na], NULL for a FIR */
    u32  gain;                 /* output x 2^gain, 0..7 */
    bool clip;                 /* saturate instead of wrapping */
} ca_fmac_filter_t;

bool ca_fmac_init(void);                                  /* false: no FMAC on this part */
bool ca_fmac_start(const ca_fmac_filter_t *f);            /* coefficients, zero state, run */
u32  ca_fmac_run(const i16 *in, i16 *out, u32 len);       /* samples filtered */
void ca_fmac_stop(void);
u32  ca_fmac_flags(void);                                 /* CA_FMAC_SAT | _OVFL | _UNFL */
```

For each sample, the unit calculates (RM0456 §26.3.6, RM0481 §21.3.6)

```
y[n] = 2^gain    (    b[k] x [n-k]  +     a[k] y [n-k] )
```

with the **plus** on the feedback: `a[]` is the opposite of the denominator in the usual convention (`a[1..]` of scipy). A FIR has from 2 to 127 coefficients; an IIR (direct form 1) from 2 to 64 forward and from 1 to `nb`-1 in feedback. The coefficients are in q1.15: a filter that has coefficients greater

than 1 is scaled by 2 and `gain` returns the factor. The accumulator is q4.22 and overflows without warning (the `CA_FMAC_SAT` flag remains); with `clip` the output saturates at ±1 instead of wrapping around. An IIR must be used with `clip`: an output that wraps around re-enters the feedback and the filter can oscillate at full scale without ever recovering.

`ca_fmac_start()` loads the coefficients and clears the delay lines, so the first output sample sees null past inputs and outputs, as `lfilter`. From there, the state remains inside the unit: subsequent calls to `ca_fmac_run()` are a single stream, one output sample for every input sample, and the signal can be filtered in blocks. The 256-word local memory is divided by the driver between coefficients, input line, and output line; what remains goes to the two circular buffers as margin.

The driver works in polling, from a task: one filter at a time, used by a single task. Interrupts and DMA (the continuous ADC → FMAC → DAC flow without CPU) are not yet available.

How to design the coefficients, how to scale an IIR, and when it is better to leave the work to the CPU is explained in the chapter Digital filters with the FMAC. Example 102 verifies the sample-by-sample arithmetic and measures the frequency response of each filter on a DAC → ADC chain.

12.1.14 11.14 Operational amplifiers (opamp)

`opamp.h` uses the two OPAMPs of **U575 and H753** as a follower (gain 1) or as an internal programmable gain amplifier (PGA: ×2, ×4, ×8, ×16). On other families `ca_opamp_count()` is 0 and initialization fails.

```
u32  ca_opamp_count(void);                                  /* 0: none on
    this part */
bool ca_opamp_init(u32 num, ca_opamp_input_t in, u32 gain);  /* 1 = follower
    , 2/4/8/16 = PGA */
void ca_opamp_stop(u32 num);
u8   ca_opamp_adc_channel(u32 num, u8 *adc_num);             /* 0: not
    routed to an ADC */
```

The non-inverting input is pin `OPAMPx_VINP` (`CA_OPAMP_IN_PIN`) or, internally, the DAC channel with the same number (`CA_OPAMP_IN_DAC`: OPAMP1 ← DAC CH1, OPAMP2 ← DAC CH2). For this reason, the U5 and H7 DAC driver sets the output to *pin + internal peripherals* mode (MCR MODE 001): for those using the pin, nothing changes. The output is always on its pin and, where the MCU connects it, on an ADC channel:

OPAMP	U575: output / ADC	H753: output / ADC
1	PA3 / ADC1 ch8	PC4 / ADC1 ch4
2	PB0 / ADC1 ch15	PE7 / none

On the Nucleo-H753, PC4 is the Ethernet PHY's RMII_RXD0 (SB36 closed): with OPAMP1 enabled, the two outputs conflict. On the H753, the driver exists but has not been tested on the board.

The offset is the factory-calibrated one (U575: within ±7 mV), and the PGA gain error is ±1% (DS13737). The input must be kept below VDDA divided by the gain, otherwise the output saturates. Like `cordic`, it does not have a `EN_` flag. Example 99 verifies the follower and PGA.

The **comparators (COMP)** of the U575 and H753 do not have a driver yet: their positive input is only a pin, which the comparator does not see when the pin is a GPIO output (verified on the U575). A real test requires a wire from the DAC or OPAMP output to a comparator input.

Previous chapter: caOS Memory Next chapter: SafeGuard and Fault Recovery in caOS

Chapter 13

SafeGuard and Fault Recovery in caOS

13.1 Chapter 12 - SafeGuard and Fault Recovery in caOS

This chapter describes how caOS implements reliability, error handling and recovery, as presented in Chapter 6.

13.1.1 12.1 SafeGuard: control with recovery

The problem of traditional assertions

A standard `assert()` on an MCU typically causes a `while(1)` or a reset. The entire system stops for an error in a single task. caOS does better: the failing task is handled according to a configurable policy, and the rest of the system continues.

Macro CA_SAFEGUARD

```
// Condition only
CA_SAFEGUARD(ptr != NULL);

// Condition + message
CA_SAFEGUARD(idx < MAX, "indice fuori range");
```

When the condition is false:

1. `ca_safeguard_failed(cond_str, msg, file, line)` is called
2. The error message is printed with condition, file, line and task name
3. If the scheduler is active and we are not in ISR:
   - `ca_scheduler_manage_fault()` is called which applies the recovery policy
   - Context switch occurs via SVC to switch to the next task
4. If the scheduler is not active or we are in ISR:
   - `while(1) { __WFI(); }` loop - halt

Recovery policy

The policy is configured at task creation:

```
// Policy 1: ASSERT -- remove the task (default)
ca_os_job("task", prio, func, NULL, STACK_MEDIUM, CA_JOB_STD,
          CA_FAULT_ASSERT);

```

```
// Policy 2: RESTART -- restart the task with the same parameters
ca_os_job("task", prio, func, NULL, STACK_MEDIUM, CA_JOB_STD,
          CA_FAULT_RESTART_THREAD);

// Policy 3: EMERGENCY -- call the handler, then remove
ca_os_job("task", prio, func, NULL, STACK_MEDIUM, CA_JOB_STD,
          CA_FAULT_EXEC_EMERGENCY);

// Policy 4: EMERGENCY + RESTART -- call the handler, then restart
ca_os_job("task", prio, func, NULL, STACK_MEDIUM, CA_JOB_STD,
          CA_FAULT_EXEC_EMERGENCY | CA_FAULT_RESTART_THREAD);
```

Emergency handler

```
void my_emergency(ca_task_handle_t h, u32 param) {
    // Cleanup: close connections, save data, notify other tasks
    ca_gpio_write(PORT_A, 5, 0);  // Turn the actuator off for safety
}

ca_task_handle_t task = ca_os_job("control", ..., CA_FAULT_EXEC_EMERGENCY);
ca_task_set_emergency_exec(task, my_emergency, 0);
```

The emergency handler is called **before** restart or remove. It allows putting the system in a safe state.

SafeGuard failure flow

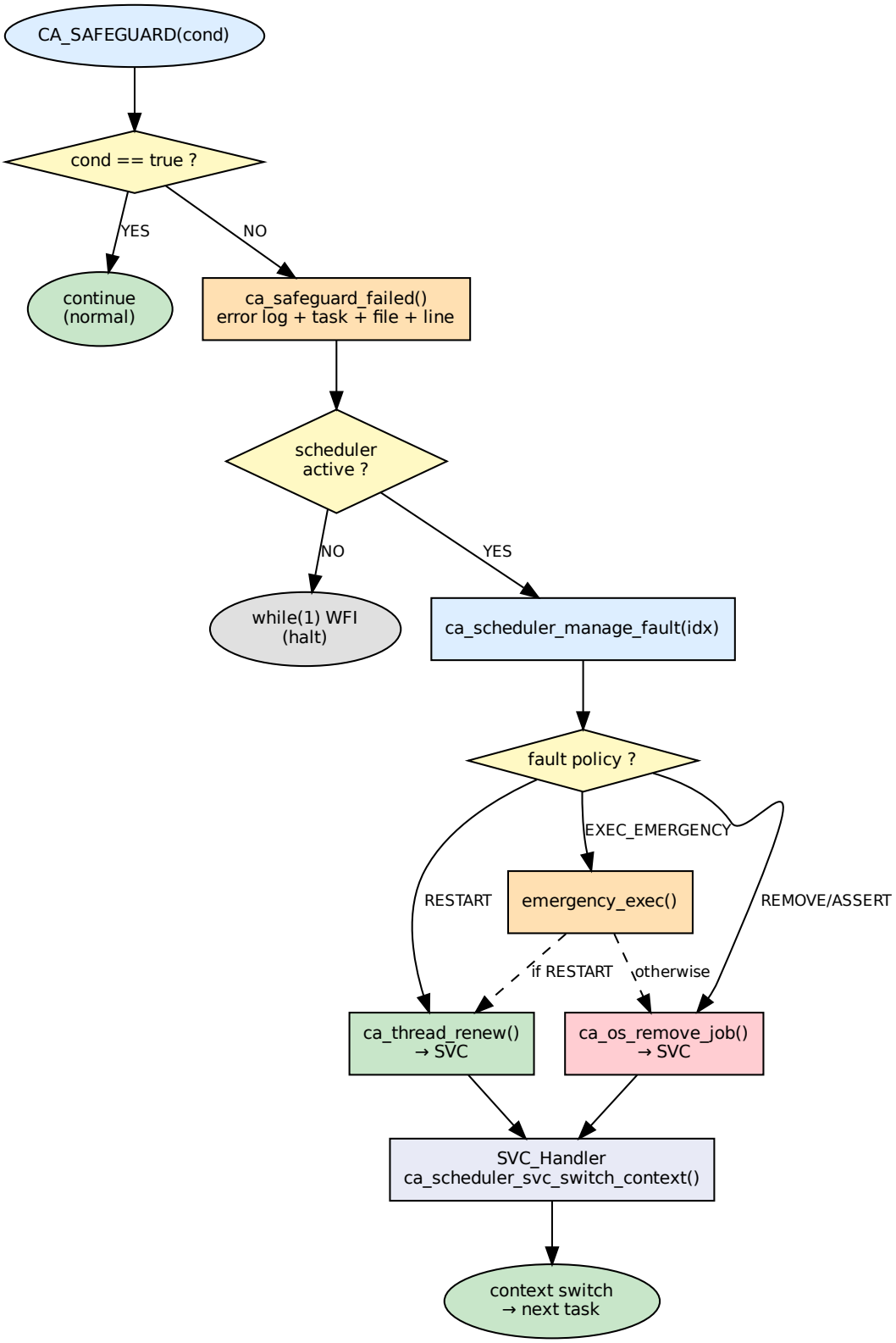


Figure 13.1: Flowchart CA_SAFEGUARD fault recovery

```
CA_SAFEGUARD(cond) fallisce
    |
    v
ca_safeguard_failed()
    |- Log: "SAFEGUARD: condizione in file:line [task: nome]"
    |
    v
ca_scheduler_manage_fault()
    |- Se FAULT_EXEC_EMERGENCY && emergency_exec != NULL:
    |       chiama emergency_exec(handle, emerg_param)
    |
    |- Se FAULT_RESTART_THREAD:
    |       Log "Mode: RESTART"
    |       (the restart happens in the SVC handler)
    |
    `- Altrimenti:
            Log "Mode: RECOVER (task removed)"
            ca_os_remove_job(task_idx)
    |
    v
SVC #0xFE -> SVC_Handler
    |
    v
ca_scheduler_svc_switch_context()
    |- Se RESTART: ca_memory_free_all_owned_by() + ca_thread_renew()
    |- If REMOVE: next READY task (like PendSV), idle if none
    `- In ENTRAMBI i casi: ca_safeguard_reset()
    |
    v
Context switch -> next task (or idle)
```

Task Renew (restart)

When a task is restarted, `ca_thread_renew()`:

1. Fills with the watermark pattern `0xDEADC0DE` only the deep part of the stack (from `stack_end` up to 65% usage, `STACK_FILL_FROM_PCT`), with IRQ masked
2. Prepares a new exception frame with PC -> `ca_thread_launch_thread`, without FP context (the task reacquires it at the first FP instruction)
3. Resets registers R4-R11
4. Rewrites the canary (temporarily disabling MPU on the region)
5. Resets counters and state: `stack_usage_pct`, `num_alloc`, `notify_value`, `nswitch`, `running`, `sleep`, runtime; detaches it from any wait list (`task_unlink_wait`): restarts from zero, has no pending wait to resume
6. The task restarts from the original function with the same parameters

The memory allocated by the task is freed before the renew with `ca_memory_free_all_owned_by()`.

> The anti-recursion flag of `ca_safeguard_failed()` is cleared on **both** branches (restart and removal). Until 09/23, it was only cleared on restart: after the first safeguard handled with removal, every subsequent safeguard, of any task, ended up with IRQ disabled in `WFI` without printing anything. It exercises ex68 (`fault safeguard` then `fault sg_irq`, without reflashing).

Diagnostic Counters

Each task container maintains diagnostic counters:

```
ctx->n_assert;    // Number of failed safeguards
ctx->n_restart;   // Number of restarts done
ctx->n_memfault;  // Number of MemManage faults
```

These values are visible with `ca_scheduler_dump_task()`.

13.1.2 12.2 CPU Fault Handling: dedicated handlers and per-task recovery

The `panic.c` module together with `bus.c` and the scheduler implements a **per-task** fault management: instead of the classic global stop (`while(1)`), the task that generates the fault is handled according to its recovery policy and **the rest of the system continues**. This applies to all three Cortex-M fault exceptions — MemManage, BusFault, UsageFault — and, from the most recent version, also for the escalated HardFault.

Enabling dedicated handlers

By default, an unhandled fault **escalates** to HardFault. caOS instead enables the three dedicated handlers, so that each category is captured and decoded:

```
ca_bus_error_init();
//  SCB->SHCSR |= BUSFAULTENA | USGFAULTENA;   (MEMFAULTENA is enabled by the
    MPU init)
//  SCB->CCR   |= DIV_0_TRP;                    (division by zero     UsageFault
    )
```

`UNALIGN_TRP` is **not** enabled globally: it would break initialization and libraries that perform legitimate unaligned accesses. It is only activated locally where needed to trap unaligned access (see Example 68).

Complete decode of subtypes (CFSR)

The dump decodes **all** subtypes of the Configurable Fault Status Register:

- **MemManage** (MMFSR): IACCVIOL, DACCVIOL, MUNSTKERR, MSTKERR, MLSPERR, MMARVALID
- **BusFault** (BFSR): IBUSERR, PRECISERR, IMPRECISERR, UNSTKERR, STKERR, LSPERR, BFARVALID
- **UsageFault** (UFSR): UNDEFINSTR, INVSTATE, INVPC, NOCP, UNALIGNED, DIVBYZERO

In addition to the subtypes, the dump prints the raw **HFSR** and **CFSR** registers, the fault addresses (MMFAR/BFAR when valid), the **PC** and **LR** stacked, and the **name/index of the offending task**.

Actual output (UsageFault from division by zero on STM32H753, retrieved):

```
[USAGE FAULT]
  DIVBYZERO: divide by zero
  UFSR = 0x0200   CFSR = 0x02000000
  Stacked LR = 0x080097BF
  Stacked PC = 0x0800072E
  Task: div0 [4]
  Mode: RECOVER (task removed)
```

The complete sequence of 16 faults and their respective real dumps are in Example 68.

Recovery per-task

When the handler has identified a valid task, it calls `ca_scheduler_manage_fault()`, which applies the recovery policy configured at task creation (see §12.1: ASSERT/remove, RESTART, EMERGENCY) and forces a **direct context switch** to the next task or idle. The handler never returns to the offending code.

HardFault escalated → declassified to per-task policy

A HardFault with `HFSR.FORCED` bit is a fault that is **escalated** (fault bank disabled, or fault-in-fault). caOS declassifies it to the same remove-policy of the three dedicated handlers **if** the fault satisfies a conservative gate:

- it is `FORCED` and not from vector table (`!VECTTBL`) nor debug (`!DEBUGEVT`);
- the exception frame is on **PSP** (so task code, not kernel on MSP);
- the **CFSR is non-zero** (the original sub-fault left a trace);
- the **task is valid** and the **PSP is in range** `[stack_end .. stack_start]`.

In that case the *recover* branch clears W1C the status registers, re-enables ENA (the kernel never leaves BUS/USG/MEMFAULTENA disabled), applies the remove-policy and does the context switch. If even one condition is missing (HardFault from kernel, opaque CFSR, dead task) it goes to **controlled reset** — see §12.2.1.

12.2.1 Controlled reboot instead of halt

On an unrecoverable HardFault, caOS does not hang in `while(1)`: it executes an orderly reset.

```
// panic.c
#ifndef PANIC_REBOOT_ON_HARDFAULT
#define PANIC_REBOOT_ON_HARDFAULT 1     // 1 = reboot in campo
#endif
// ... NVIC_SystemReset();  se PANIC_REBOOT_ON_HARDFAULT
```

Set `PANIC_REBOOT_ON_HARDFAULT 0` in a debug build to **stay asleep** on WFI and inspect the state with the debugger.

With `EN_WDT=1`, `ca_os_reset()` has a second path, independent of the core: it arms the IWDG at 1 ms and stops reloading it, then requests the system reset; if that does not happen, the watchdog fires anyway. The CLI command `reboot` uses the IWDG only (`reset` uses the system reset). At restart `ca_wdt_reset_cause()` tells whether the reset came from a watchdog (see §11.10).

> **Open issue — persistent crash log.** Today the dump goes only to UART and disappears on reset. It is planned (TODO in `hard_fault_reset`) a sink that survives the reset — buffered RAM / RTC backup registers / serial EEPROM / SD — to persist CFSR/HFSR + frame + task-id. It depends on the completion of SDIO/FAT support on H/U series.

13.1.3 12.3 Stack overflow and corruption detection: defense levels

Stack overflow is the most common cause of crashes in embedded systems. caOS addresses it with multiple complementary defenses, both **hardware** and **software**, that cover each other:

Level	Mechanism	When detects	Direction
1	**MPU guard** (RO canary band under `stack_end`)	Immediate (MemManage DACCVIOL)	downward

Level	Mechanism	When detects	Direction
2	**Extended guard under** `stack_end` (deep jumps over 32B band)	Immediate (MemManage)	downward
3	**PSP range-check** at context switch (exit **and** entry)	At context switch	both
4	**Predictive watermark** (pattern at init + idle scan)	Warning **before** overflow	growth
5	**Floor check** (8 word at `stack_end`, idle scan)	At next idle tick	upward (cross-task)
6	Canary check + CRC32 digest (idle scan)	At next idle tick	downward

The key difference from a single canary: the PSP range-check also catches corruption of the saved PSP *at a distance* (while the task is not running), which no hardware-based access guard would see. The watermark, finally, warns **before** the overflow becomes a fault (STACK WARNING $> 75\%$ via idle scan).

The asymmetry of the MPU guard

The MPU guard **is not symmetric**, and it's important to state this precisely: it only protects what is **under** the task's `stack_end`. Since stacks are allocated from top to bottom, the first-created tasks — system tasks, `idle` / `stdout` / `cli` — are located **higher** in memory, and are exactly those **exposed** to erroneous writes from subsequently created application tasks. A cross-task write upward does not generate any fault: the MPU does not see it.

On ARMv7-M (F4/F7/H7) there is a second limitation. The MPU region must have size as a power of 2 and natural alignment, so the extended guard cannot cover the entire PSP arena: its depth is the largest aligned block that *ends* exactly at the canary, limited by `DEEP_GUARD_MAX` (8 KB on F4xx/F7xx, 32 KB on H7xx) and still contained in the arena. In practice, it ranges from 32 bytes to several KB, depending on where the canary address falls. On ARMv8-M (H5/U5) this constraint does not exist: base and limit are arbitrary and the guard covers the exact `[arena_base, stack_end)` span.

The two gaps are covered by software mechanisms, not by the MPU: - **upward**: the *floor check* (level 5), linear fixed-cost check on 8 word at `stack_end`, executed by the idle task. It is needed precisely because the watermark is not enough: `calc_stack_watermark` performs a **binary** search and assumes monotonic stack, so a cross-task write in the middle of the intact area either gets skipped or gets mistaken for legitimate use; - **diagnosis**: the panic handler traces back the **owner task** of the hit address (`stack_owner_of()`) instead of attributing every write in the PSP arena to the current task as a false overflow. It distinguishes the **canary** of the victim (under its `stack_end`, where the offset would be an underflow and is not printed) from the **internal offset** `+N` within the stack; only when no task owns the address does the dump fall back to the overflow message.

The floor check, unlike the heap canary, **does not restore** the pattern: at the floor a broken pattern can mean a real overflow, and deleting it would lose the trace.

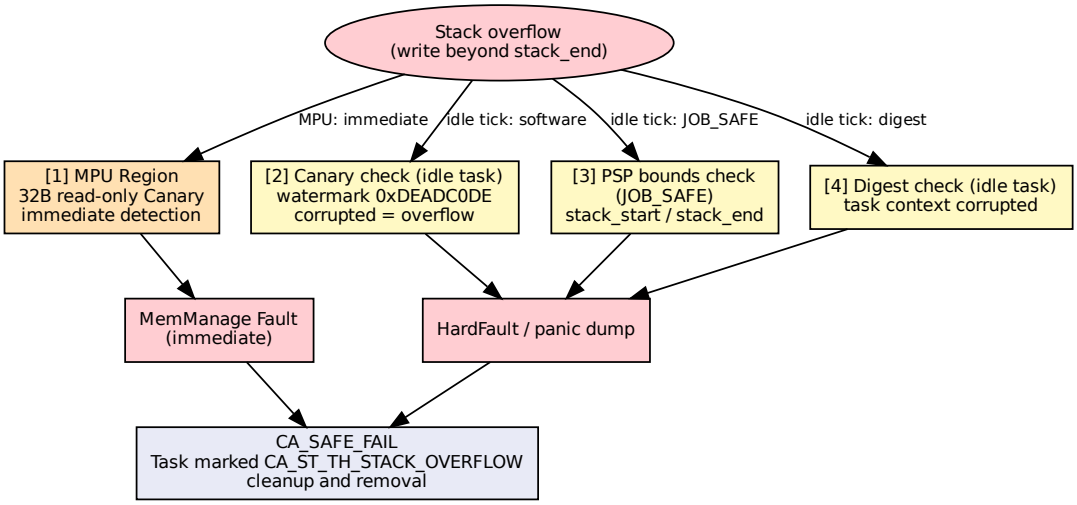


Figure 13.2: Stack MPU protection levels

MPU AP encoding on ARMv8-M (M33). On Cortex-M33 (U5/H5) the encoding of access permissions is **inverted** compared to ARMv7-M: read-write is `AP=1` and read-only is `AP=3` (on M4/M7 `AP=3` is full access). An error in this leaves RO regions *writable* — a real bug fixed in the canary guards and key store.

13.1.4 12.4 Kernel protections: overall framework

The previous sections describe the individual mechanisms. Here is the complete overview of the caOS kernel **safety protections**, from CPU faults to reboot. 1. **Full Cortex-M trap coverage** — dedicated handlers for MemManage / BusFault / UsageFault with decode of *all* CFSR subtypes (§12.2). Enabled by `ca_bus_error_init` (BUSFAULTENA | USGFAULTENA | DIV_0_TRP); MEMFAULTENA from MPU init. 2. **Recovery per-task** — `ca_scheduler_manage_fault()` applies the task policy (remove / restart / emergency) and performs direct context switch to next/idle, without global halt (§12.1). 3. **HardFault escalated de-escalated** — a HardFault `FORCED` from a task, with valid PSP in range, is de-escalated to the same remove-policy instead of killing the RTOS; conservative gate for the rest → reset (§12.2). 4. **MPU stack guard (downward)** — RO canary band under `stack_end`, extended to catch deep jumps that overshoot it; the scheduler suspends/resumes the guard on kernel writes to other tasks' stacks. Covers the exact span on ARMv8-M, limited depth from alignment on ARMv7-M; **does not** protect upward (§12.3). 5. **PSP range-check at context switch** — `stack_end` `psp` `stack_start` verified both on exit and entry; refuses to mount a corrupted saved PSP at distance (§12.3). 6. **Predictive stack watermark** — only the deep part of the stack (from 65% usage down) is filled with pattern at creation and restart; below that threshold `top` prints `Stack= <65%`. STACK WARNING > 75% via idle scan, **before** it becomes a fault (§12.3). 7. **Watchdog per-task on CPU cycles** — `ca_os_thread_set_watchdog(cicli)` accumulates runtime at every context switch; upon budget overrun marks the task `CA_ST_TH_SIG_WDT` ("task watchdog on!"). Captures cumulative CPU overrun. 8. **Stack floor check** — 8 word at `stack_end` checked by idle; intercepts cross-task writes **upward**, which the MPU guard misses and the binary watermark may overshoot (§12.3). 9. **Software canary on heap** — 32-byte pattern `0xAABBCCDD` at extreme, outside the block chain, verified by idle every 100 cycles. Replaces

the two MPU regions at heap boundaries, returning them to the budget: the allocator is next-fit and cannot exceed START/END anyway, while heap overrun *within* the heap — the real case — was not covered by MPU in any way. 10. **MPU on key store** — dedicated MPU region for crypto key store (2048 B), accessible only upon explicit unlock (DACCVIOL otherwise) — see §12.6. 11. **Controlled reboot** — `NVIC_SystemReset()` gated by `PANIC_REBOOT_ON_HARDFAULT` instead of halt (§12.2.1). 12. **Hardware watchdog** — IWDG/WWDG (`wdt.h`, §11.10): resets the system if the software stops reloading it, even with the core stuck; `ca_os_reset()` uses it as a second reset path and `ca_wdt_reset_cause()` reports it at restart. It is global: complementary to, not a replacement for, the per-task watchdog (7).

> On heap canary, the reaction is **message + immediate marker restoration**, deliberately **not** `CA_SAFE_FAIL`: heap corruption is a *global* state, not a task fault that notices it. With policy `RESTART`, the safeguard would trigger an infinite loop that cyclically destroys the idle. All of these protections are exercised on real hardware by **Example 68** (fault test bench): 16 cases that each trigger one of the categories and show their recovery.

Comparison with other embedded RTOSes

The table compares the protections of caOS with those typical of the main RTOSes for Cortex-M. "Optional" indicates a function that is available but not active by default or that requires configuration/additional modules.

Protection	caOS	FreeRTOS	Zephyr	ThreadX	Bare-metal
Dedicated Mem/Bus/Usage handlers with CFSR decode		(HardFault hook only)		opz.	
Per-task recovery after fault (no global halt)			thread abort (userspace)		
HardFault escalated downgraded to per-task policy					
Stack guard via **MPU** (downwards)	exact span on ARMv8-M; limited depth on ARMv7-M	opz. (MPU port)	(`HW_STACK_PROTECTION`)	Modules	
Cross-task write **upwards** detection	SW floor check				
Diagnosis of **owning task** of the hit address					

Protection	caOS	FreeRTOS	Zephyr	ThreadX	Bare-metal
Stack watermark / high-water mark	predictive	(`uxTaskGetStackHighWaterMark`)			
Range-check of saved PSP at context switch					
Per-task watchdog on CPU budget		(global HW WDT only)	opz. (thread analyzer)		
Canary on heap checked periodically	SW		opz. (heap listener)		
MPU on crypto key area			mem domains	Modules	
Controlled reboot on unrecoverable fault	gated	user burden	(`CONFIG_RESET_ON_FATAL`)	Timer	
Restart recovery of failed task					

The distinctive feature of caOS is not the individual mechanism (for example, the stack watermark exists everywhere), but the **per-task recovery**: a fault in a task does not stop the entire system, and depending on the policy the task can be removed, restarted, or pass through an emergency handler. The downgrading of escalated HardFault and the range-check of saved PSP are, as far as we know, specific to caOS.

The limit, stated in full. caOS's MPU guard is *not* stronger than that of other RTOSes in the direction that all cover: it is a downward guard, and on ARMv7-M it covers a depth that depends on the canary alignment, not the entire arena. What caOS adds compared to others is not a better MPU, but **software** defenses that cover what no Cortex-M MPU can see: cross-task write upwards (floor check) and attribution of the hit address to the task that actually owns that memory, instead of the false "overflow of current task".

13.1.5 12.5 CRC32

The `crc32.h` module provides a CRC32 calculation system for thread context integrity verification:

```
// Hardware CRC32 module initialization
void ca_crc32_init(void);
```

Mainly used for: - Verification of thread context integrity - Detection of corruption in context data - Error diagnostics support

The module enables and uses the STM32 microcontroller's hardware CRC32 peripheral to perform efficient calculations.

13.1.6 12.6 Keys Store

The `keys_store.h` module provides an advanced protection system for cryptographic keys using the MPU (Memory Protection Unit) of the microcontroller. Unlike a simple data storage, this module offers a protection mechanism based on dynamic access policies.

System Architecture

The module creates a protected common memory area through a single MPU region that contains all secure keys. This approach optimizes the use of the limited hardware resources of the microcontroller:

- Only one MPU region is used to protect the entire storage area
- All keys share this common area
- Protection is controlled at the common area level, not at the individual key level

Access Modes

Keys can have different access modes:

- `CA_KEYS_STORE_NO_ACCESS`: No access allowed (region disabled)
- `CA_KEYS_STORE_READ_ONLY`: Read-only access allowed
- `CA_KEYS_STORE_READ_WRITE`: Read and write access allowed
- `CA_KEYS_STORE_EXEC_ONLY`: Execute-only access allowed (rarely used)

Typical Usage Sequence

1. **Storage**: Keys are allocated and immediately protected with full access disabled
2. **Temporary Access**: When a key is needed, it is temporarily made accessible (read/write)
3. **Usage**: The key is used for cryptographic operations
4. **Protection**: After use, the key is fully protected again

Main API

```
// System initialization
int ca_keys_store_init(u8 max_keys);

// Allocation of a secure key
void* ca_keys_store_allocate(u32 size);

// Setting the access mode
int ca_keys_store_set_access_mode(void* ptr, ca_keys_store_access_mode_t mode);

// Temporary access (enable/disable)
int ca_keys_store_temp_enable_access(void* ptr, ca_keys_store_access_mode_t mode
    );
int ca_keys_store_temp_disable_access(void* ptr);

// Full protection (no access)
int ca_keys_store_lock_completely(void* ptr);

// Safe copy of data into a protected area
int ca_keys_store_copy(void* dest, const void* src, u32 size);

// Deallocazione
int ca_keys_store_free(void* ptr);
```

Advanced Security

The module implements several security measures:

- **Memory wipe**: Upon deallocation, memory is cleaned with zeros
- **Immediate protection**: Keys are protected immediately after allocation
- **Access control**: Functions verify that operations are permitted
- **Configuration lock**: Possibility to lock MPU configuration to prevent future modifications

This system enables secure handling of cryptographic keys, minimizing the time they are accessible and protecting them from unauthorized access during normal system operation.

13.1.7 12.7 Diagnostica: dump e statistiche

Dump task

```
// Dump of a single task
ca_scheduler_dump_task(index, DUMP_TASK_ALL);

// Dump of all tasks
ca_scheduler_dump_all_tasks(DUMP_TASK_ALL);

// Compact info
ca_scheduler_tasks_info();
```

Selectable flags for the dump:

Flag	Output
DUMP_TASK_HDR	Name, index, state, priority
DUMP_TASK_ADDR	Addresses: TLB, PSP, stack range
DUMP_TASK_REG	Registers R0-R12, LR, PC, xPSR
DUMP_TASK_STACK	Stack usage %, PSP validity
DUMP_TASK_STATS	Running time, sleep, nswitch, fault counters
DUMP_TASK_ALL	All

Compact information

`ca_scheduler_tasks_info()` prints one line per task:

```
task 'idle      ' CPU= 85.3% Stack= 12.5% SW=  84521 SL=      0 Mem=      0
task 'sensore   ' CPU=  8.2% Stack= 45.2% SW=   8420 SL=   4210 Mem=    256
task 'display   ' CPU=  4.1% Stack= 23.8% SW=   4210 SL=   2105 Mem=   1024
```

Canary Check

```
ca_scheduler_check_all_canaries();
```

Output:

```
[0] idle     canary: low:0x1000EFE0 OK
[1] sensore  canary: low:0x1000EBE0 OK
[2] display  canary: low:0x1000E7E0 OK
```

Stdout Statistics

```
u32 queued, dropped;
ca_stdout_stats(&queued, &dropped);
// queued  = lines in the queue
// dropped = lines lost (queue full)
```

Available with `ENABLE_STDOUT 1` in `os_config.h`; with `ENABLE_STDOUT 0` becomes empty macro.

Memory Statistics

```
ca_memory_list();          // List all blocks (used and free)
ca_memory_dump_avail();    // Free blocks only
u32 avail = ca_memory_get_avail_memory();
```

MPU

```
ca_mpu_dump_regions();     // All active MPU regions
```

Previous chapter: caOS Drivers and I/O

End of Section 2 - caOS Implementation
Section 3: Advanced Concepts

Chapter 14

Thread Pool in caOS

14.1 Thread Pool in caOS

The **thread pool** is a concurrent programming pattern in which a fixed set of threads is pre-allocated at system startup and reused to execute jobs sent dynamically during execution. Instead of creating and destroying a thread for each operation, the workers remain active and sleep waiting for work.

14.1.1 Motivation

In a bare-metal RTOS, every `ca_os_job()` allocates stack and control structure. On MCUs with limited memory, repeating this operation at high frequency causes:

- **heap fragmentation** in CCM (if the stack is released and reallocated)
- **variable task creation latency**
- **bursts of occupancy** when many jobs are created simultaneously

The pool pre-allocates everything at boot and maintains a constant and predictable number of concurrent tasks.

> **Availability**: the pool is enabled with `EN_POOL=1` in `os_config.h` and is operational only on MCUs with `MAX_TASK >= 50` (H5xx, H7xx, U5xx). On smaller platforms the code is excluded at compile time.

14.1.2 Pool Architecture

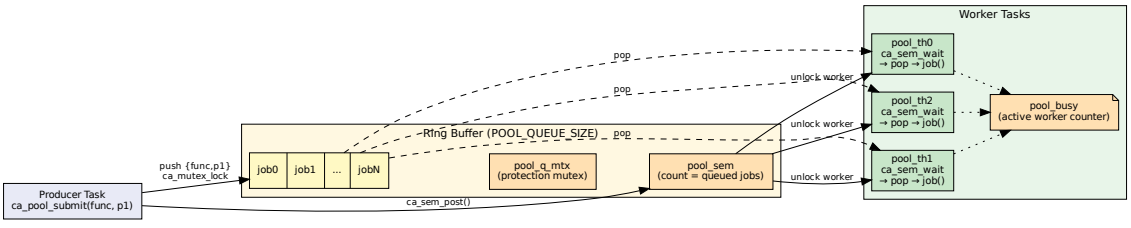


Figure 14.1: caOS Thread Pool Architecture

```
ca_pool_init(n, stack, prio)

                  ca_sem_init(&pool_sem, 0, POOL_QUEUE_SIZE)
                  ca_mutex_init(&pool_q_mtx)

                  pool_th0
                  pool_th1                    loop: ca_sem_wait     pop     job()
        repeat
                  pool_thN

ca_pool_submit(func, arg)

                  push {func, arg} into the ring buffer  (mutex protected)
                  ca_sem_post(&pool_sem)                        wakes a worker
```

The workers never terminate: they execute an infinite loop on `ca_sem_wait()`. The semaphore counts the available jobs; when it is zero, the workers sleep without consuming CPU.

14.1.3 Synchronization mechanism

Counting semaphore

```
ca_sem_init(&pool_sem, 0, POOL_QUEUE_SIZE);
```

- Initial value = **0**: no job available, all workers blocked.
- `ca_pool_submit()` calls `ca_sem_post()` → increments → wakes up **one** worker.
- The worker calls `ca_sem_wait()` → decrements → takes ownership of the job.
- If all workers are busy and a new job arrives, `ca_sem_post()` increments the semaphore above 1: the next worker that becomes free will immediately find work.

Mutex for the ring buffer

```
ca_mutex_init(&pool_q_mtx);
```

The queue is a static ring buffer shared between the caller of `ca_pool_submit()` and the worker threads. The mutex protects push and pop operations from concurrent accesses.

Worker body

```
static u32 pool_worker(void *arg) {
    (void)arg;
    while (1) {
        ca_sem_wait(&pool_sem);          /* sleeps until there is work */
        ca_pool_job_t job = pool_queue_pop();
        pool_busy++;
        job.func(job.arg);               /* runs the job */
        pool_busy--;
    }
    return 0;
}
```

The job receives only its argument. To know which worker runs it, use `ca_os_thid_from_handle(ca_os_self_handle())`, as Example 16 does (`w=5`, `w=6`, `w=7`).

14.1.4 Public API

Initialization

```c
typedef struct {
    u8           n_threads;   /* number of workers (max 99) */
    ca_stack_size_t stack_size;  /* stack of each worker */
    u16          priority;    /* scheduler priority */
} ca_pool_cfg_t;

i32 ca_pool_init(const ca_pool_cfg_t *cfg);
```

Create `n_threads` worker with names `pool_th0 ... pool_thN`. All use the same priority and the same stack size. Returns `0` on success, `-1` if `n_threads == 0` or MAX_TASK exhausted.

Must be called before `ca_os_run()`.

Sending a job

```c
i32 ca_pool_submit(ca_task_fn_t func, void *arg);
```

Return	Meaning
`0`	Job executed (in worker queue or temporary fallback one-shot)
`-1`	Queue full **and** MAX_TASK exhausted: job discarded

The signature of `func` is identical to that of a regular caOS task:

```c
u32 my_job(void *arg);
```

`arg` is the argument passed to `ca_pool_submit()`.

Note: `ca_pool_submit()` is **not safe from ISR** because it uses `ca_mutex_lock()`. Call only from tasks.

Statistics

```c
void ca_pool_stats(u8 *busy, u8 *free_th, u8 *queued);
```

Parameter	Meaning
`busy`	Worker that are executing a job
`free_th`	Worker waiting on the semaphore
`queued`	Job in queue not yet assigned

`busy` is a `volatile u8` counter updated by the worker before and after `job.func()`. It is accurate for display; a delta of ± 1 during a context switch is acceptable.

14.1.5 Configuration in `os_config.h`

```c
/* Enable the pool */
#define EN_POOL         1

/* Depth of the job queue (static ring buffer) */
#define POOL_QUEUE_SIZE 8
```

`POOL_QUEUE_SIZE` determines the maximum number of waiting jobs when all workers are busy. If the queue fills up, `ca_pool_submit()` attempts a fallback with a one-shot thread.

14.1.6 Memory Usage

For a pool of N workers with stack S:

Resource	Size
`ca_thread_context_t` × N	N × 160 bytes (in CCM)
Worker stack	N × S bytes (in CCM)
Queue ring buffer	`POOL_QUEUE_SIZE` `8` bytes (in .bss)
Semaphore	~16 bytes (in .bss)
Mutex	~12 bytes (in .bss)

Example: 3 workers with `STACK_MEDIUM` (512 bytes), `POOL_QUEUE_SIZE=8`:

- Structures: 3 × 160 = 480 bytes in CCM
- Stack: 3 × 512 = 1536 bytes in CCM
- Queue: 8 × 8 = 64 bytes in .bss

Total: approximately 2080 bytes, known and fixed at boot.

14.1.7 Use Cases

Scenario	Pool Useful?	Why
Asynchronous cryptographic (SHA, AES) computing	Yes	Limits concurrent threads, protects the HASH peripheral
Ethernet/CAN packet processing	Yes	Packet bursts, predictable latency
Asynchronous QSPI logging/storage	Yes	Slow operations, does not block the producer
Periodic tasks with fixed frequency	No	Better with a dedicated task using `ca_os_sleep()`
Operations with highly variable stack usage	No	Fixed stack for all jobs is a waste
Fast interrupt response	No	`ca_pool_submit()` is not ISR-safe

14.1.8 Limitations

- **Fixed stack**: all jobs share the same stack size. Size for the most demanding job.
- **Fire-and-forget**: no built-in mechanism to retrieve the job result. If the result is needed, use a global variable, an event (`ca_event_*`) or a task notification (`ca_notify_send`).
- **Single priority**: all workers have the same priority. Urgent jobs and slow jobs compete at the same level.
- **ISR**: `ca_pool_submit()` is not callable from interrupt handlers.

14.1.9 Internal Implementation

The pool is implemented in `caos/src/pool.c`, included at the end of `scheduler.c` with `#include "pool.c"`. This technique allows `pool.c` to access the scheduler's static variables (`g_work_index`, `g_task_array`) without exposing them publicly, maintaining encapsulation.

```
caos/inc/pool.h          API pubblica (ca_pool_init, ca_pool_submit,
    ca_pool_stats)
caos/src/pool.c          Implementazione (inclusa da scheduler.c)
```

No modification to the scheduler core is necessary: workers are normal caOS tasks that never terminate.

Next chapter: Cryptographic Modules in caOS

Chapter 15

Cryptography in caOS

15.1 Chapter 13 - Cryptography in caOS

Cryptography in caOS is organized into **two levels**:

- **Hardware drivers** (`caos/drivers/`): CRC32, HASH, RNG — direct access to STM32 peripherals, protected by mutexes for multithreading.
- **Cryptography** (`caos/crypto/`): AES (driver with HW/SW dispatch), Keys Store, AES Channel Service, Secure Pipe (between tasks and, in link mode, between two boards), key derivation (HMAC / HKDF / PBKDF2 on SHA-256), plus software primitives SHA-256 / SHA-512 / X25519 / Ed25519 / ECDSA P-256 — abstraction that hides key management, IV, wire format, heap, and wipe.

Above these primitives, the network stack builds SSH and the TLS 1.3 server (`caos/net/tls.c`, used by HTTPS): see the HTTPS / TLS 1.3 chapter.

```
Applicazione
           ca_secure_pipe_* / ca_sp_link_* / ca_aes_channel_encrypt / decrypt

caos/crypto/             aes.c, keys_store, aes_channel_service,
   secure_pipe_service,
                         kdf (HMAC/HKDF/PBKDF2), sha256/sha512/x25519/ed25519,
                         ecdsa_p256
           ca_aes_gcm_*, ca_keys_store_*, ca_rng_fill

  dispatch AES                  HW: hw/*/drivers/aes_*.c  (CRYP su H7, AES su
    U5xx con IP)
                           SW: percorso software interno ad aes.c (board senza
    AES HW)

caos/drivers/            rng.c, hash.c, crc32.c
           HASH / RNG / CRC periferici

hw/*/drivers/            implementazioni bare-metal per famiglia
```

> **Note (2026-07-21):** `aes.c`/`aes.h` have been moved from `caos/drivers/` to `caos/crypto/`. The AES driver automatically chooses **hardware or software** based on the peripheral present on the device (detected via CMSIS macros `CRYP_BASE` / `AES_BASE`): where an AES/CRYP IP exists, hardware is used, otherwise a software AES implementation (FIPS-197 + NIST SP 800-38D) internal to the file, at 128 or 256 bits, is used. The public API `ca_aes_*` is identical in both cases.

15.1.1 13.1 Hardware driver overview

Module	Header	Flag	HW Peripheral	Families
CRC32	`crc32.h`	`EN_CRC`	CRC (RCC)	All
HASH (SHA-256)	`hash.h`	`EN_HASH`	HASH	H5xx, H7xx
AES	`aes.h`	`EN_AES`	CRYP / AES **or software**	**All** (HW where present, otherwise SW)
RNG	`rng.h`	`EN_RNG`	RNG	F4xx, F7xx, H5xx, H7xx

For RNG and HASH, on families lacking the peripheral, the functions are **stubs** that return `false`. **AES is an exception**: if the device does not have an AES crypto IP, the driver falls back to a software implementation (128 and 256-bit ECB/CTR/GCM), so `ca_aes_*` works on all families — including Nucleo-U575 and H563, which lack HW AES.

Enablement in `os_config.h`:

```
#define EN_CRC                  1   /* CRC32 hardware */
#define EN_HASH                 1   /* SHA-1 / SHA-256 hardware */
#define EN_AES                  1   /* AES ECB/CBC/CTR/GCM, HW o SW */
#define EN_RNG                  1   /* RNG hardware (True Random Number
    Generator) */
#define EN_KEYS_STORE           1   /* area MPU-protetta per chiavi */
#define EN_AES_CHANNEL_SERVICE  1   /* pipe cifrata applicativa */
```

15.1.2 13.2 CRC32

The CRC32 module uses the microcontroller's hardware CRC peripheral to calculate 32-bit checksums efficiently. It is available on all STM32 families supported by caOS.

Typical uses: buffer integrity verification, packet validation, corruption detection.

API

```
void ca_crc32_init(void);           /* chiamare una volta in ca_os_configure */
u32  ca_crc32_get(u8 *buff, u32 size);   /* protetta da mutex */
```

Typical usage

```
u8 data[256];
u32 crc = ca_crc32_get(data, 256);

data[100] ^= 0xFF;                   /* simula corruzione */
u32 crc2  = ca_crc32_get(data, 256);
if (crc != crc2)
    ca_stdout_put_n("Corruzione rilevata!");
```

`ca_crc32_get()` is internally protected by a mutex: multiple tasks can call it concurrently.

15.1.3 13.3 HASH (SHA-256)

The problem: a single hardware engine, N tasks

The STM32 HASH peripheral is **unique**. The SHA-256 calculation is incremental — the intermediate state (CSR registers) must be preserved between blocks. Two tasks calculating SHA-256 in parallel without protection would corrupt each other's hardware registers.

The solution: context swap

caOS solves the problem with a **cryptographic context swap**: each task possesses a local `ca_hash_ctx_t` structure that contains a copy of the hardware registers (CSR[]). Before using the peripheral, the driver restores the registers from the task's context; after the operation, it saves them again.

```
Task A: update("block1")
         lock mutex HASH
         ripristina registri da ctx_A.csr[]
         elabora "block1" sull'hardware
         salva registri in ctx_A.csr[]
         unlock mutex
  [context switch     Task B]

Task B: update("blockX")
         lock mutex HASH
         ripristina registri da ctx_B.csr[]
         elabora "blockX"
         salva registri in ctx_B.csr[]
         unlock mutex
  [context switch     Task A]

Task A: update("block2")
         ripristina ctx_A.csr[]       riprende esattamente dov'era
         digest A corretto
```

Structure `ca_hash_ctx_t`

```c
typedef enum {
    CA_HASH_ALGO_SHA1   = 0,
    CA_HASH_ALGO_SHA256 = 1,
} ca_hash_algo_t;

#define HASH_SHA1_DIGEST_SIZE    20
#define HASH_SHA256_DIGEST_SIZE  32

typedef struct {
    ca_hash_algo_t algo;
    u32 csr[HASH_MAX_CSR];   /* copia registri HW (H5=103 CSR, H7=54 CSR) */
    u32 cr_save;
    u32 csr_count;
    bool started;
} ca_hash_ctx_t;
```

API

```c
bool ca_hash_init(void);   /* una volta in ca_os_configure */

/* One-shot */
bool ca_hash_compute(ca_hash_algo_t algo,
                     const u8 *data, u32 len,
                     u8 *digest, u32 *digest_len);

/* Incrementale con context swap */
bool ca_hash_ctx_init(ca_hash_ctx_t *ctx, ca_hash_algo_t algo);
bool ca_hash_ctx_update(ca_hash_ctx_t *ctx, const u8 *data, u32 len);
bool ca_hash_ctx_final(ca_hash_ctx_t *ctx, u8 *digest, u32 *digest_len);
```

One-shot

```
u8 digest[HASH_SHA256_DIGEST_SIZE];
u32 digest_len;
ca_hash_compute(CA_HASH_ALGO_SHA256, data, data_len, digest, &digest_len);
```

Incremental (block data)

```
ca_hash_ctx_t ctx;   /* LOCALE al task     non condivisa */
u8 digest[HASH_SHA256_DIGEST_SIZE];
u32 digest_len;

ca_hash_ctx_init(&ctx, CA_HASH_ALGO_SHA256);
ca_hash_ctx_update(&ctx, header,  header_len);
ca_hash_ctx_update(&ctx, payload, payload_len);
ca_hash_ctx_update(&ctx, trailer, trailer_len);
ca_hash_ctx_final(&ctx, digest, &digest_len);
/* digest = SHA-256(header || payload || trailer) */
```

Two tasks SHA-256 in parallel

```
static u32 task_hash_channel_a(void *arg) {
    (void)arg;
    ca_hash_ctx_t ctx;
    u8 digest[HASH_SHA256_DIGEST_SIZE]; u32 digest_len;
    while (1) {
        ca_hash_ctx_init(&ctx, CA_HASH_ALGO_SHA256);
        ca_hash_ctx_update(&ctx, (const u8*)"CanalA-block1", 13);
        ca_os_sleep(10);   /* Task B pu  usare l'HW HASH qui */
        ca_hash_ctx_update(&ctx, (const u8*)"CanalA-block2", 13);
        ca_hash_ctx_update(&ctx, (const u8*)"CanalA-block3", 13);
        ca_hash_ctx_final(&ctx, digest, &digest_len);
        ca_stdout_printf_n("[A] %02X..%02X", digest[0], digest[31]);
        ca_os_sleep(200);
    }
    return 0;
}

static u32 task_hash_channel_b(void *arg) {
    (void)arg;
    ca_hash_ctx_t ctx;
    u8 digest[HASH_SHA256_DIGEST_SIZE]; u32 digest_len;
    while (1) {
        ca_hash_ctx_init(&ctx, CA_HASH_ALGO_SHA256);
        ca_hash_ctx_update(&ctx, (const u8*)"CanalB-X", 8);
        ca_os_sleep(15);
        ca_hash_ctx_update(&ctx, (const u8*)"CanalB-Y", 8);
        ca_hash_ctx_final(&ctx, digest, &digest_len);
        ca_stdout_printf_n("[B] %02X..%02X", digest[0], digest[31]);
        ca_os_sleep(200);
    }
    return 0;
}
```

The two digests are **independent and correct** despite having shared the same physical peripheral.

> **Rule:** `ca_hash_ctx_t` must never be shared between tasks. The physical peripheral is shared; the context is private.

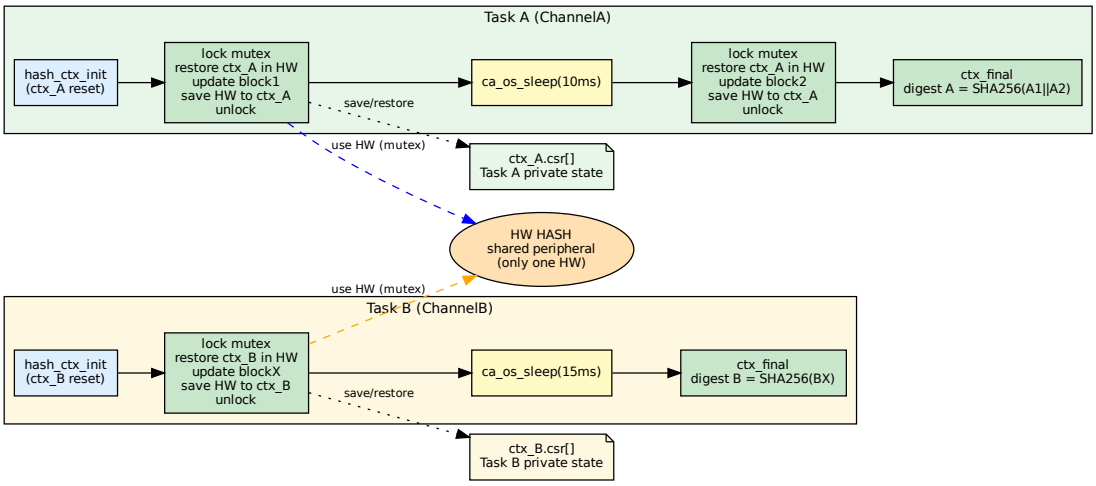


Figure 15.1: Timeline context swap HASH multithread

15.1.4 13.4 AES

Automatic hardware / software dispatch

The AES peripheral is not uniform across the STM32 family: the **CRYP** block (AES+GCM) exists on H753, the **AES** IP on some U5 variants (U585/U5A5...), while Nucleo-U575 and H563 have no AES accelerator at all. To ensure `ca_aes_*` everywhere, the driver chooses the path only once in `ca_aes_init()`:

```
#if defined(CRYP_BASE) || defined(AES_BASE)
    aes_hw_ready = true;    /* IP crypto presente     hardware */
#else
    aes_hw_ready = false;   /* nessun HW     AES software interno */
#endif
```

Detection uses the **device** CMSIS macros (`CRYP_BASE`, `AES_BASE`), not `MCU`/`MCU_ID` (which do not reliably discriminate families in the preprocessor). The software path is a pure AES (FIPS-197) with GCM/GHASH over GF(2^128) (NIST SP 800-38D), verified against NIST vectors (McGrew/Viega) — CLI command `aes_nist test` (example 26). It covers ECB, CTR, and GCM at **128 and 256 bits**: 256 for SSH and crypto services, 128 for TLS 1.3 (`TLS_AES_128_GCM_SHA256` is the mandatory suite). The number of rounds (10 or 14) is fixed when the key is expanded, the first time the context is used. AES-192 is not supported in software. The public API is identical in both paths.

The problem: key and IV to be preserved per session

The AES peripheral contains the loaded key, the initialization vector (IV), and the GCM state. Two tasks with different keys sharing the peripheral without protection would find the other's key loaded. In CTR/GCM mode, the internal counter advances with every encrypted block: an interruption produces corrupted ciphertext without any visible error.

Context swap AES: state in `ca_aes_ctx_t`

```c
typedef struct {
    ca_aes_mode_t    mode;       /* ECB / CBC / CTR / GCM */
    ca_aes_keysize_t keysize;    /* 128 / 192 / 256 bit */
    u32 key[8];                  /* chiave (fino a 256 bit = 8 word) */
    u32 iv[4];                   /* IV corrente (aggiornato dopo ogni blocco CTR
   /GCM) */
    bool configured;
} ca_aes_ctx_t;

#define AES_GCM_TAG_SIZE  16
```

Before each encrypt/decrypt operation, the driver: acquires the CRYP mutex → loads `ctx.key` and `ctx.iv` → performs the operation → updates `ctx.iv` → releases the mutex.

Operating modes

Mode	Constant	IV	Authentication	Typical use
ECB	`CA_AES_MODE_ECB`	No	No	Independent single blocks
CBC	`CA_AES_MODE_CBC`	Yes	No	Chaining streams
CTR	`CA_AES_MODE_CTR`	Yes	No	High-speed streams
GCM	`CA_AES_MODE_GCM`	Yes	Yes (16B tag)	Authenticated protocols

API

```c
bool ca_aes_init(void);   /* una volta in ca_os_configure */

bool ca_aes_ctx_setup(ca_aes_ctx_t *ctx, ca_aes_mode_t mode,
                      ca_aes_keysize_t keysize,
                      const u8 *key, const u8 *iv);

/* Solo un IV nuovo: chiave e key schedule SW restano (niente ri-espansione).
 * Per una sessione che tiene il contesto per molti messaggi. */
bool ca_aes_ctx_set_iv(ca_aes_ctx_t *ctx, const u8 *iv);

bool ca_aes_encrypt(ca_aes_ctx_t *ctx, const u8 *in, u8 *out, u32 len);
bool ca_aes_decrypt(ca_aes_ctx_t *ctx, const u8 *in, u8 *out, u32 len);

bool ca_aes_gcm_encrypt(ca_aes_ctx_t *ctx,
                        const u8 *aad, u32 aad_len,
                        const u8 *in, u8 *out, u32 len,
                        u8 tag[AES_GCM_TAG_SIZE]);

bool ca_aes_gcm_decrypt(ca_aes_ctx_t *ctx,
                        const u8 *aad, u32 aad_len,
                        const u8 *in, u8 *out, u32 len,
                        const u8 tag[AES_GCM_TAG_SIZE]);
```

AES-256 ECB — basic usage

```c
ca_aes_ctx_t ctx;
u8 key[32], plain[16] = "Hello AES World!", cipher[16], decrypted[16];

ca_rng_fill(key, 32);
ca_aes_ctx_setup(&ctx, CA_AES_MODE_ECB, CA_AES_KEY_256, key, NULL);
ca_aes_encrypt(&ctx, plain, cipher, 16);
ca_aes_decrypt(&ctx, cipher, decrypted, 16);
```

AES-256 GCM — authenticated encryption

GCM authenticates both the ciphertext and the AAD (*Additional Authenticated Data*) left in plaintext. The tag (16 bytes) binds key + IV + AAD + ciphertext: any modification is detected during decryption.

```
ca_aes_ctx_t ctx;
u8 key[32], iv[16];
u8 aad[16]   = "header-in-chiaro";
u8 plain[32] = "payload segreto!!!";
u8 cipher[32], decrypted[32], tag[AES_GCM_TAG_SIZE];

ca_rng_fill(key, 32); ca_rng_fill(iv, 16);
ca_aes_ctx_setup(&ctx, CA_AES_MODE_GCM, CA_AES_KEY_256, key, iv);

ca_aes_gcm_encrypt(&ctx, aad, 16, plain, cipher, 32, tag);

if (ca_aes_gcm_decrypt(&ctx, aad, 16, cipher, decrypted, 32, tag))
    ca_stdout_put_n("GCM OK     dati autentici");
else
    ca_stdout_put_n("GCM FAIL     dati manomessi!");
```

> **Rule:** `ca_aes_ctx_t` must never be shared between tasks. The physical CRYP peripheral is shared and protected by an internal mutex.

HASH vs AES context swap comparison

	HASH	AES
Saved state	CSR[] (up to 103 regs.)	key[8] + iv[4] (12 words)
Atomic operation	one block (update)	entire encrypt/decrypt call
Interruptible calculation between blocks	Yes	No (ECB/CBC); Yes (CTR/GCM)
Parallel sessions	unlimited	unlimited

15.1.5 13.5 RNG

The RNG module uses the STM32 hardware True Random Number Generator peripheral to generate random numbers based on thermal noise. It is not a PRNG: every value is unpredictable and non-reproducible.

Availability: F4xx, F7xx, H5xx, H7xx. On F1xx, the functions are stubs (`false`).

API

```
bool ca_rng_init(void);
bool ca_rng_get(u32 *val);          /* singolo u32, blocking, mutex */
bool ca_rng_fill(u8 *buf, u32 len); /* buffer di byte casuali */
```

Typical usage

```
u32 random_val;
ca_rng_get(&random_val);

u8 aes_key[32], aes_iv[16];
ca_rng_fill(aes_key, 32);
ca_rng_fill(aes_iv, 16);
```

15.1.6 13.6 Initialization in `ca_os_configure()`

Cryptographic modules are initialized using `EVAL_*` macros. Recommended order: CRC → HASH → AES → RNG (no dependencies between them):

```
void ca_os_configure(void) {
    EVAL_CRC(  ca_crc32_init(); )
    EVAL_HASH( ca_hash_init();  )
    EVAL_AES(  ca_aes_init();   )
    EVAL_RNG(  ca_rng_init();   )
    /* ... */
    EVAL_STDOUT( ca_stdout_init(); ca_stdout_set_sender(STDOUT);
    ca_stdout_create_thread(); )
}
```

If a module is disabled (`EN_xxx 0`), the `EVAL_xxx(...)` macro expands to nothing.

15.1.7 13.7 Pattern: Key Derivation

A common pattern is to derive a cryptographic key by combining RNG, HASH, and AES:

```
IV_SECRET (16 B, RNG) + KEY_SECRET (32 B, RNG)

                        Concatenazione 48 B     SHA-256     Chiave derivata 32 B
```

```
u8 iv_secret[16], key_secret[32];
u8 hash_input[48], digest[32]; u32 digest_len;

ca_rng_fill(iv_secret, 16);
ca_rng_fill(key_secret, 32);
ca_memcpy(hash_input,      iv_secret,  16);
ca_memcpy(hash_input + 16, key_secret, 32);
ca_hash_compute(CA_HASH_ALGO_SHA256, hash_input, 48, digest, &digest_len);
/* digest = chiave derivata (32 byte per AES-256) */
```

For the secure storage of the derived key, see the **Keys Store** module (§13.9).

Standard functions: HMAC, HKDF, PBKDF2 (`kdf.h`)

As of 10/15/2026, derivation has its own standard functions, in `caos/crypto/kdf.{h,c}`, built on the **software** SHA-256: they yield the same result on every family, with or without the HASH accelerator. They are enabled with `EN_SHA256`; known-answer tests with `EN_SHA256_TEST`.

Function	Standard	Purpose
`ca_hmac_sha256` (+ `_init/_update/_final`)	RFC 2104 / FIPS 198-1	Keyed MAC, the building block for the other two
`ca_hkdf_sha256(salt, ikm, info, out, n)`	RFC 5869	from key material to independent keys (one per use, with different `info`)
`ca_pbkdf2_sha256(pwd, salt, iter, out, n)`	RFC 8018	from a **password** to a key, intentionally slow

```
u8 master[32], k12[32], k21[32];
ca_pbkdf2_sha256((const u8 *)pwd, ca_strlen(pwd), (const u8 *)name, ca_strlen(
    name),
                 100000U, master, 32U);
ca_hkdf_sha256(NULL, 0U, master, 32U, (const u8 *)"link 1->2", 9U, k12, 32U);
ca_hkdf_sha256(NULL, 0U, master, 32U, (const u8 *)"link 2->1", 9U, k21, 32U);
ca_memzero(master, sizeof(master));
```

- Each intermediate buffer (HMAC pads, PRK, U blocks) is zeroed before exiting.
- PBKDF2 prepares the password HMAC pads **once** and copies the context at each round, instead of re-hashing them 2 times per iteration.
- **Measured cost** (`sha256 kdf [N]` of ex66): ~22 µs per iteration on H753 → 100000 iterations = 2.25 s; F767 and H563 about double. One derivation per boot, after the password: every attempt by an attacker costs the same time.
- `ca_kdf_selftest()`: RFC 4231 #1/#2/#6, RFC 5869 #1/#3, RFC 7914 c=1 and a c=4096 compared with `hashlib`. 7/7 on H753.

The "SHA-256 of a concatenation" scheme above remains valid for simple cases (used by `keystore_service`); to derive **multiple keys** from the same secret or start from a **password**, HKDF and PBKDF2 are used.

15.1.8 13.8 Crypto Infrastructure (`caos/crypto/`)

Application services in `caos/crypto/` build an abstraction layer on top of hardware drivers that hides key management, IVs, wire format, heap, and wiping.

```
caos/crypto/
          inc/
                aes.h                       driver AES, dispatch HW/SW
                keys_store.h                area MPU-protetta per chiavi
                aes_channel_service.h       canale cifrato sincrono  r e d black
                secure_pipe_service.h       pipe fra task + modalit  link fra
    board
                keystore_service.h          chiave divisa in flash
                kdf.h                       HMAC / HKDF / PBKDF2 (SHA-256)
                sha256.h sha512.h x25519.h ed25519.h ecdsa_p256.h
          src/
              aes.c  keys_store.c  aes_channel_service.c  secure_pipe_service.c
              keystore_service.c  kdf.c
              sha256.c  sha512.c  x25519.c  ed25519.c  ecdsa_p256.c
```

Dependencies

```
Driver HW (caos/drivers/)          caos/crypto/


  aes.h  (EN_AES)                                  aes_channel_service.h
  rng.h  (EN_RNG)                                  aes_channel_service.h
  hash.h (EN_HASH)                                 (keystore_service     futuro)
  mpu.h                                            keys_store.h
  memory.h                                         aes_channel_service.h
  sync.h (ca_mutex_t)                                 aes_channel_service.h
```

15.1.9 13.9 Keys Store (`keys_store.h`)

Problem

Cryptographic keys in RAM are vulnerable: a buggy task, a buffer overflow, or access via debugger can read arbitrary memory areas. On Cortex-M, the **MPU** allows marking regions as inaccessible until they are explicitly opened.

Design

`keys_store` manages **a single contiguous area** of 2048 bytes protected by a single MPU region. The protection granularity is the entire region (not the individual key).

- Keys are allocated sequentially (**bump allocator**) and never move. Each allocation is rounded to **multiples of 4 bytes**: a slot can also contain a structure with `u32` fields, for example, the link mode AES session context (§13.10).
- **First-fit allocation**: a new key goes into the first free interval large enough among those already allocated, starting from the beginning of the area. `ca_keys_store_free` zeroes the key and its space immediately becomes reusable, even in the middle of the area. Until 10/15/2026, the area was a bump allocator and `free` only freed the table entry: every allocate/free cycle consumed the area forever. Harmless with a few 32-byte keys, fatal for a link that changes passwords (app_3 keeps two sets of keys and frees the *lower* one, contexts of ~300 bytes).

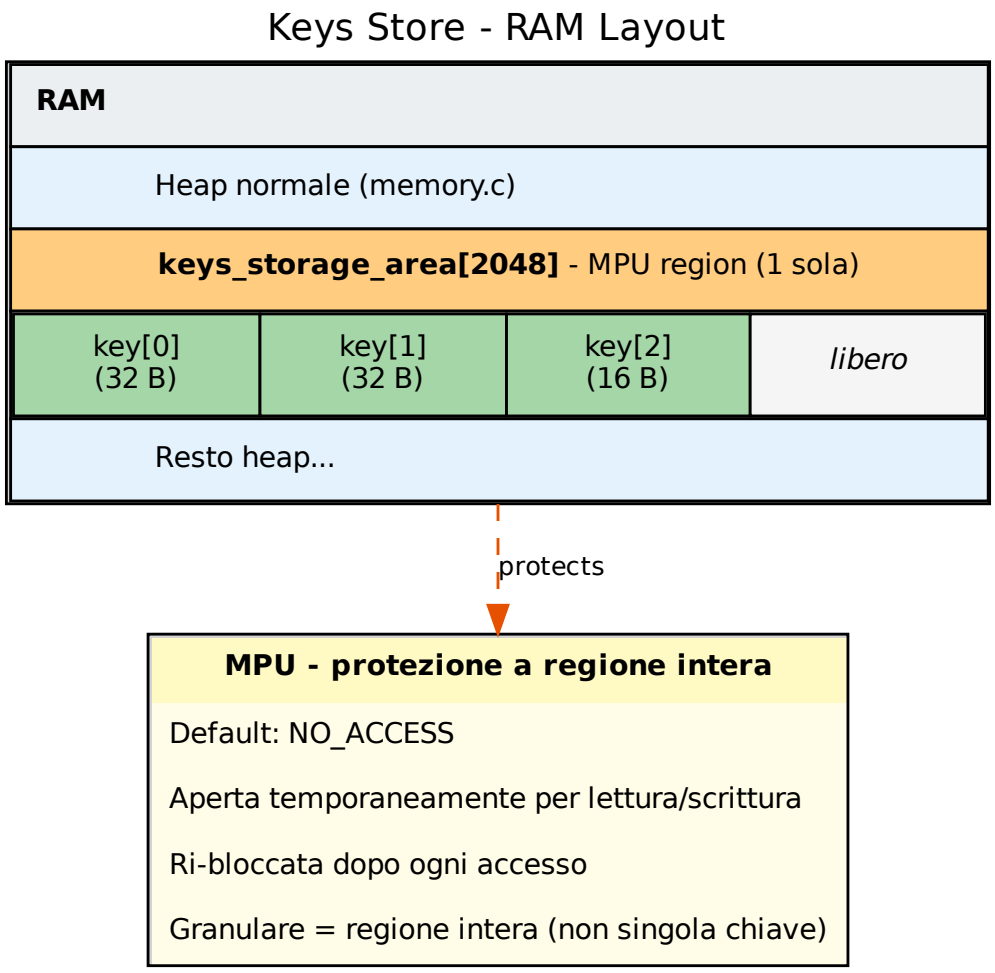


Figure 15.2: Layout RAM keys_store

Key lifecycle

```
ca_keys_store_allocate(32)
      ptr nell'area protetta
ca_keys_store_temp_enable_access(ptr, READ_WRITE)
ca_keys_store_copy(ptr, raw_key, 32)
ca_keys_store_lock_completely(ptr)              MPU: NO_ACCESS
  [chiave inaccessibile finch  non serve]
ca_keys_store_temp_enable_access(ptr, READ_ONLY)
  [leggi, usa, azzera dallo stack]
ca_keys_store_lock_completely(ptr)              ri-proteggi
```

Access modes

```
typedef enum {
    CA_KEYS_STORE_NO_ACCESS  = 0,
```

```
    CA_KEYS_STORE_READ_ONLY  = 1,
    CA_KEYS_STORE_READ_WRITE = 2,
    CA_KEYS_STORE_EXEC_ONLY  = 3,
} ca_keys_store_access_mode_t;
```

API

```
int  ca_keys_store_init(u8 max_keys);
void *ca_keys_store_allocate(u32 size);
int  ca_keys_store_free(void *ptr);
int  ca_keys_store_temp_enable_access(void *ptr, ca_keys_store_access_mode_t
    mode);
int  ca_keys_store_lock_completely(void *ptr);
int  ca_keys_store_copy(void *dest, const void *src, u32 size);
bool ca_keys_store_is_key_expired(void *ptr);
int  ca_keys_store_set_expiry(u8 key_idx, u32 start, u32 stop);
```

Packed date

```
/* 2026-03-15     0x001A030F */
#define KSTORE_DATE_PACK(y, m, d)  (((u32)(y) << 16) | ((u32)(m) << 8) | (u32)(d
    ))

ca_keys_store_set_expiry(0,
    KSTORE_DATE_PACK(26, 1, 1),
    KSTORE_DATE_PACK(26, 12, 31));
```

If the RTC is disabled and the expiration is configured, `is_key_expired()` returns `true` for security reasons (key not usable without a time reference).

Tamper (`EN_TAMP`, paid version). On U575 and H563, with `EN_TAMP = 1`, the `keys_store` area resides in the **secret area** (SRAM2) that caOS reserves at startup (§10.2): a tamper on the B1 button clears the keys in hardware, before any code runs. Elsewhere, or without `EN_TAMP`, it remains a static array in the main SRAM. The MPU region is the same: the start of SRAM2 is 64 KB aligned. See Backup SRAM and tamper.

15.1.10 13.10 AES Channel Service (`aes_channel_service.h`)

Problem

Using `ca_aes_gcm_encrypt/decrypt` directly requires the application to handle: IV generation, wire format, temporary access to the key in the keys_store, output buffer allocation, wipe after use, thread safety — at least 10 lines of boilerplate for every encrypt/decrypt point.

`aes_channel_service` encapsulates all this complexity:

```
ca_aes_channel_encrypt(&ch, plain, plen, &out, &out_len);
ca_aes_channel_decrypt(&ch, cipher, clen, &out, &out_len);
```

Red/black terminology

Side	Content	Direction
Red	plaintext	→ encrypt → black

Side	Content	Direction
Black	ciphertext	→ decrypt → red

The terminology is borrowed from military cryptography (TEMPEST, NATO SDIP-27).

Wire format

```
     nonce          orig_len          ciphertext (paddato)          GCM tag
    12 byte          2 byte           ceil(plain_len, 16) B          16 byte

overhead fisso = AES_CH_OVERHEAD = 30 byte
```

- **nonce**: 12 random bytes from HW RNG, freshness for each message
- **orig_len**: original length of the plaintext (big-endian, 2 bytes)
- **ciphertext**: always a multiple of 16 bytes (plaintext padded with zeros)
- **tag**: 16 bytes GCM — end-to-end authentication

The header `[nonce|orig_len]` (14 bytes) is the GCM AAD (as of 10/15/2026, `AES_CH_HDR_SIZE`). Previously, GCM ran with an empty AAD: `orig_len` traveled outside the tag, and a value lowered by an attacker would **truncate the plaintext under a valid tag**, without an error. The AAD is copied into an aligned buffer because the H7 CRYP reads it in 32-bit words.

This is a format change: a ciphertext produced previously (including encrypted files from `fat_service`) can no longer be decrypted. There is no fallback to the old format: that would be the vulnerability itself. Verified with ex46/ex47 on H753 (CRYP), F767, and H563 (software GCM).

The GCM nonce passed to the H7 CRYP peripheral is `[nonce:12][0x00000002:4]`: the final 4 bytes are the initial counter required by the GCM specification (RFC 5116 §3) for the Init phase of the H753 CRYP peripheral.

Note: padding and H7 CRYP The STM32H7 CRYP peripheral has an asymmetry in the GHASH calculation for partial blocks in GCM: in encrypt, GHASH covers the ciphertext with non-zero keystream bytes; in decrypt, it covers the ciphertext padded with zeros — the two GHASH values diverge → tag mismatch. The `NPBLB` field of the CR should correct this, but it proved unreliable on the H753 CRYP.

Solution: always pad the plaintext to a multiple of 16 before encrypting. `orig_len` in the wire allows the receiver to extract the actual bytes.

Structure `ca_aes_channel_t`

```
typedef void (*ca_aes_iv_gen_cb_t)(u8 iv[AES_CH_IV_SIZE], void *user);
typedef bool (*ca_aes_key_ratchet_cb_t)(u8 *key, u32 key_len, void *user);

typedef struct {
    ca_aes_mode_t            mode;
    ca_aes_keysize_t         keysize;
    void                    *ks_handle;   /* ptr keys_store */
    ca_aes_iv_gen_cb_t       iv_gen_cb;   /* NULL     ca_rng_fill() automatico
   */
    ca_aes_key_ratchet_cb_t  ratchet_cb;  /* NULL     rotate disabilitato */
```

```
    void                    *cb_user;
    ca_aes_ctx_t            *red_ctx;     /* heap */
    ca_aes_ctx_t            *black_ctx;   /* heap */
    ca_mutex_t                  mutex;
} ca_aes_channel_t;
```

Generatore di callback IV

```
/* Default (NULL): usa HW RNG */

/* Deterministico per test */
static void test_iv_gen(u8 iv[12], void *user) {
    u32 *counter = (u32 *)user;
    ca_memzero(iv, 12);
    ca_memcpy(iv, counter, 4);
    (*counter)++;
}
```

Using a deterministic IV is only safe for testing. In production, always use NULL (HW RNG).

Callback key ratchet

Updates the key in-place: it receives the current key and must overwrite it with the new one.

```
static bool sha256_ratchet(u8 *key, u32 key_len, void *user) {
    u32 session_id = *(u32 *)user;
    u8 input[36], digest[32]; u32 digest_len;

    ca_memcpy(input,      key,         key_len);
    ca_memcpy(input + 32, &session_id, 4);
    ca_hash_compute(CA_HASH_ALGO_SHA256, input, 36, digest, &digest_len);
    ca_memcpy(key, digest, key_len);
    ca_memzero(input, sizeof(input));
    return true;
}
```

Complete API

```
i32  ca_aes_channel_init(ca_aes_channel_t *ch,
                         ca_aes_mode_t mode, ca_aes_keysize_t keysize,
                         void *ks_handle,
                         ca_aes_iv_gen_cb_t iv_cb,
                         ca_aes_key_ratchet_cb_t ratchet_cb,
                         void *cb_user);

/*  R e d Black : alloca *out su heap */
i32  ca_aes_channel_encrypt(ca_aes_channel_t *ch,
                            const u8 *plain, u32 plain_len,
                            u8 **out, u32 *out_len);

/*  B l a c k Red : alloca *out su heap; -1 se tag mismatch */
i32  ca_aes_channel_decrypt(ca_aes_channel_t *ch,
                            const u8 *cipher, u32 cipher_len,
                            u8 **out, u32 *out_len);

void ca_aes_channel_free(void *p);      /* wipe + free     OBBLIGATORIO */
i32  ca_aes_channel_rotate(ca_aes_channel_t *ch);
void ca_aes_channel_deinit(ca_aes_channel_t *ch);
```

Thread safety

Level	Protection	Granularity
AES Driver	Internal `aes_mutex`	Single HW operation
Channel service	`ch->mutex`	load_key → setup_ctx → encrypt/decrypt

Heap management and wipe

```
/* CORRETTO */
u8 *out; u32 out_len;
ca_aes_channel_encrypt(&ch, plain, plen, &out, &out_len);
/* ... usa out ... */
ca_aes_channel_free(out);           /* wipe + free */

/* SBAGLIATO */
ca_memory_free(out, NULL, false);   /* non azzera il ciphertext */
```

Wiping is mandatory: it prevents ciphertext or plaintext from remaining readable in "free" heap areas that other tasks might subsequently allocate.

Complete typical usage

```
/* In ca_os_configure() */
ca_aes_init();
ca_rng_init();
ca_keys_store_init(4);

/* Carica chiave */
void *key_ptr = ca_keys_store_allocate(32);
ca_keys_store_temp_enable_access(key_ptr, CA_KEYS_STORE_READ_WRITE);
ca_keys_store_copy(key_ptr, my_key_32bytes, 32);
ca_keys_store_lock_completely(key_ptr);

/* Init canale */
static ca_aes_channel_t ch;
ca_aes_channel_init(&ch, CA_AES_MODE_GCM, CA_AES_KEY_256,
                    key_ptr, NULL, NULL, NULL);

/* Encrypt (task mittente) */
u8 *enc; u32 enc_len;
if (ca_aes_channel_encrypt(&ch, plain, plen, &enc, &enc_len) == 0) {
    send_to_transport(enc, enc_len);
    ca_aes_channel_free(enc);
}

/* Decrypt (task ricevente) */
u8 *dec; u32 dec_len;
u8 rx[MAX_RX]; u32 rx_len = recv_from_transport(rx, sizeof(rx));
if (ca_aes_channel_decrypt(&ch, rx, rx_len, &dec, &dec_len) == 0) {
    process_plaintext(dec, dec_len);
    ca_aes_channel_free(dec);
} else {
    /* tag mismatch: messaggio corrotto o manomesso */
}

/* Deinit */
ca_aes_channel_deinit(&ch);
ca_keys_store_free(key_ptr);
```

Secure pipe (`secure_pipe_service.h`)

The channel encrypts one message at a time, but the transport remains the caller's responsibility: code, packet struct, and heap buffer properties (who frees them, even when sending fails). The **secure pipe** closes the loop between two tasks: a shared AES channel + one heap queue per direction, with encrypted and plaintext buffers never leaving the service. It is enabled with the same flag `EN_AES_CHANNEL_SERVICE`.

```
static ca_secure_pipe_t pipe;                    /* deve partire azzerata */
ca_secure_pipe_init(&pipe, CA_AES_MODE_GCM, CA_AES_KEY_256, key_ptr, 4);

/* capo A */
ca_secure_pipe_send(&pipe, CA_SECURE_PIPE_A, msg, len, 2000);

/* capo B */
u8 buf[64]; u32 n;
if (ca_secure_pipe_recv(&pipe, CA_SECURE_PIPE_B, buf, sizeof(buf), &n, 5000)
        == CA_SECURE_PIPE_OK)
    process_plaintext(buf, n);

ca_secure_pipe_deinit(&pipe);
```

Code	Meaning
`CA_SECURE_PIPE_OK`	message sent / received
`CA_SECURE_PIPE_ERR`	arguments, pipe not ready, heap full, encryption failed
`CA_SECURE_PIPE_TIMEOUT`	queue full (send) or empty (recv) until timeout
`CA_SECURE_PIPE_DECRYPT`	invalid GCM tag: message discarded
`CA_SECURE_PIPE_SIZE`	message larger than buffer: discarded, `*out_len` = size

- `send` encrypts and queues towards the **other** end; `recv` decrypts and **copies** into the caller's buffer, then zeroes and frees internal buffers.
- Maximum message `CA_SECURE_PIPE_MAX_MSG` = 65535 bytes (16-bit length in the channel wire format).
- `deinit` flushes queues with wipe, but must be called **while the pipe is stopped**: a task still inside `send`/`recv` will wait again on the destroyed queue. Examples: ex46 (fixed pipe) and ex47 (pipe recreated at every `start`, closed by Bob).

Secure pipe in link mode (`ca_sp_link_*`)

The pipe connects two tasks on the **same** board. **Link mode** connects two **boards**: it seals and opens **buffers**, and the application handles transport (UDP in `workspace/app_3`). Since the service does not touch the network, the two ends can be tested on a single board (ex97). Requires `EN_AES_CHANNEL_SERVICE` + `EN_SHA256`.

Keys

```
password + nome del link (8..16 caratteri, uguali sulle due board)
          PBKDF2-HMAC-SHA256(password, nome, 100000)      master
               HKDF(master, "caos-sp-link 1->2")      K12
               HKDF(master, "caos-sp-link 2->1")      K21
```

Role 1 encrypts with K12 and decrypts with K21, role 2 does the opposite (no reflection). In `keys_store`, K12/K21 and the two **AES session contexts** remain; password, name, and master are zeroed out.

Handshake (at every startup, peer restart, rekey)

```
HELLO = [tipo][ruolo][iv:12][nonce|eco cifrati:32][tag:16]     62 byte
  tipo 1 = apre, tipo 3 = risposta, che ripete il nonce a cui risponde
sessione: S12 = HKDF(K12, n1|n2)   S21 = HKDF(K21, n1|n2)
```

- New session keys for every handshake: the data counter restarts from 1 without reusing a GCM nonce, and a registered session is not valid for the next one.
- A response is **never responded to** (otherwise the two boards would respond to each other infinitely: found in app_3, ~4000 handshakes in 4 s).
- The initiator only accepts the response with the **echo of its current nonce**: a response to an old HELLO, queued while the peer was waiting for the password, does not pair with the new one (without the echo, the two boards ended up with different keys).
- If they start at the same time, the role is decided: 2 responds, 1 waits.

Data

```
[tipo=2][contatore:8][len:2][cifrato, padding a 16][tag:16]   overhead     42
    byte
```

- GCM Nonce = `0000|contatore`; AAD = header + caller's AAD (in app_3: IP and UDP ports), therefore length and stream are authenticated.
- Anti-replay with a window of 64, updated only after a valid tag; an incorrect tag clears the output buffer.
- The **session AES context** is prepared once per session in the `keys_store` slot; for each message, only the nonce changes (`ca_aes_ctx_set_iv`), without re-expanding the key. Overwritten upon rekey, cleared and freed upon deinit.
- Suggested rekey (`ca_sp_link_need_rekey`) after 1 h or 2^20 messages.

```c
ca_sp_link_init(&l, role, password, name);        /* secondi: PBKDF2 */
ca_sp_link_hello(&l, false, out, cap, &n);        /* apre: spedire al peer */
rc = ca_sp_link_accept(&l, in, len);              /* OK | REPLY | REPLAY |
    DECRYPT */
if (rc == CA_SECURE_PIPE_REPLY) ca_sp_link_hello(&l, true, out, cap, &n);
ca_sp_link_seal(&l, aad, alen, plain, len, out, cap, &n);
ca_sp_link_open(&l, aad, alen, in, len, out, cap, &n);   /* REPLAY se ripetuto
    */
ca_sp_link_deinit(&l);
```

Code	Meaning (in addition to pipe codes)
`CA_SECURE_PIPE_REPLY`	`accept`: peer opened, respond with `hello(reply = true)`
`CA_SECURE_PIPE_REPLAY`	counter already seen or out of window; HELLO response exceeded
`CA_SECURE_PIPE_NOT_UP`	no session: handshake first

Verified: ex97 13/13 on H753 (CRYP), F767 and H563 (software GCM); in app_3 between H753 and H563, encrypted UDP in both directions with no failed tags, peer reboot with new handshake.

15.1.11 13.11 Crypto levels: architecture summary

```
Livello applicazione:
  ca_secure_pipe_send/recv          (trasporto fra task: code + buffer)
  ca_sp_link_hello/accept/seal/open (fra board: handshake, sessione, anti-replay
    )
                                                ca_pbkdf2/ca_hkdf/ca_hmac_sha256 (
    kdf.c)

  ca_aes_channel_encrypt/decrypt
               (nasconde: IV gen, wire format, key access, heap, wipe, mutex)

Livello servizio (caos/crypto/):
  aes_channel_service.c
               (chiama: ca_aes_gcm_*, ca_keys_store_*, ca_rng_fill)

Livello driver AES (caos/crypto/aes.c):
  ca_aes_*     dispatch HW/SW (aes_hw_ready)
                                           SW: AES-128/256 ECB/CTR/GCM interno
              (HW)
Livello hardware (hw/*/drivers/):
  aes_h7xx.c (CRYP)   aes_u5xx.c (AES)   ...

Altri driver (caos/drivers/):  rng.c   hash.c   crc32.c
```

15.1.12 13.12 Availability by MCU family

Module	F1xx	F4xx	F7xx	H5xx	H7xx	U5xx
CRC32	Yes	Yes	Yes	Yes	Yes	Yes
HASH SHA-256	Stub	Stub	Stub	Yes (HASH)	Yes (HASH)	Yes (HASH)
AES (ECB/CTR/GCM)	SW	SW	SW	SW[3]	Yes (CRYP)	SW / HW
RNG	Stub	Yes	Yes	Yes	Yes	Yes
`keys_store`	Yes[1]	Yes[1]	Yes[1]	Yes	Yes	Yes
`aes_channel_service`	Yes[2]	Yes[2]	Yes[2]	Yes	Yes	Yes[2]
`keystore_service`	Yes[2]	Yes[2]	Yes[2]	Yes	Yes	Yes[2]
`kdf` (HMAC/HKDF/PBKDF2)	Yes	Yes	Yes	Yes	Yes	Yes
ECDSA P-256 (software)	No	Yes	Yes	Yes	Yes	Yes
secure pipe link	Yes[2]	Yes[2]	Yes[2]	Yes	Yes	Yes[2]

[1] `keys_store` works on all families with Cortex-M MPU (M3+).
[2] Now works everywhere: `ca_aes_gcm_*` falls back to the software path where HW AES is missing.
[3] H563 does not have AES/SAES → software. Other H5 variants with AES would use the HW. U575 does not have AES → software; U585/U5A5 (AES IP) → hardware, auto-detected via `AES_BASE`.
Excluded by choice: no hardware RNG and too little RAM for the TLS that uses it. The H563 PKA is not used: it is in limited mode, it only knows how to verify (RM0481 §36.3).

15.1.13 13.13 Keystore Service (`keystore_service.h`)

Problem

`keys_store` protects keys in RAM but does not solve the **provisioning** problem: how to securely bring keys onto the device? Writing them to flash in cleartext exposes them to anyone with

physical access or access via a debugger.

`keystore_service` solves this: keys are pre-provisioned in flash **encrypted with AES-256-GCM** and are decrypted and loaded into `keys_store` only at runtime, providing the second half of the secret.

Split-key security model

The decryption secret is built from **three components**:

Component	Where it resides	Who knows it
`IV` (16 B)	Flash (cleartext header)	anyone who reads the flash
`p1` (16 B)	Flash (cleartext header)	anyone who reads the flash
`p2` (16 B)	Provided at runtime	only those who need to unlock

The decryption key is derived with:

```
decrypt_key = SHA256( SHA256( ... SHA256(IV || p1 || p2) ... ) )
                                                  n_evolve volte
```

`IV` and `p1` in flash are inherently useless without `p2`. `n_evolve` makes brute-forcing `p2` computationally expensive (key stretching).

Flash block layout

Keystore Service - FLASH block layout

base_addr	HEADER - cleartext (AAD per GCM) - 54 byte
+0	size (u32, 4 B) - dimensione totale blocco
+4	IV[16] (16 B) - sale per KDF
+20	n_evolve (u32, 4 B) - iterazioni SHA256
+24	p1[16] (16 B) - prima meta segreto (in flash)
+40	n_key (u16, 2 B) - numero di chiavi
+42	gcm_nonce[12] (12 B) - nonce AES-GCM
+54	**PAYLOAD - AES-256-GCM cifrato - ceil(n_key*48, 16) B**
key[0]	ca_flash_key_entry_t - 48 byte
key[1]	ca_flash_key_entry_t - 48 byte
...	...
-16	**TAG GCM (16 B) - autentica header + payload**

detail

input KDF

ca_flash_key_entry_t - 48 byte	
+0 (2 B)	key_id (u16)
+2 (6 B)	label[6]
+8 (32 B)	**key[32] - chiave AES-256**
+40 (4 B)	start (KSTORE_DATE_PACK)
+44 (4 B)	stop (KSTORE_DATE_PACK)

KDF - chiave di decifratura

decrypt_key =

SHA256^n_evolve(IV || p1 || p2)

p1: da flash (header)

p2: fornito a runtime (segreto)

Figure 15.3: Flash block layout keystore_service

GCM authenticates both the encrypted payload and the plaintext header (AAD): any modification to the header or the payload is detected during decryption.

Data structures

```
/* Singola entry chiave     48 byte */
typedef struct {
    u16 key_id;
    u8  label[KSTORE_SVC_LABEL_SIZE];   /* 6 byte */
    u8  key[KSTORE_SVC_KEY_SIZE];       /* 32 byte */
    u32 start;      /* KSTORE_DATE_PACK validity start (0 = no limit) */
    u32 stop;       /* KSTORE_DATE_PACK validity stop  (0 = no limit) */
} ca_flash_key_entry_t;
```

```

/* Header del blocco flash     cleartext, AAD per GCM     54 byte */
typedef struct {
    u32 size;
    u8  IV[16];
    u32 n_evolve;
    u8  p1[16];
    u16 n_key;
    u8  gcm_nonce[12];
} ca_flash_key_block_hdr_t;
```

API

```
/* Legge e valida l'header dal blocco flash a base_addr.
 * Ritorna 0 se header valido, -1 altrimenti. */
int ca_keystore_service_init(u32 base_addr);

/* Fornisce p2 (16 byte), deriva la chiave, decifra il payload,
 * carica le chiavi in keys_store.
 * Ritorna 0 se successo, -1 se p2 errato o flash corrotta.
 * In caso di errore chiama automaticamente ca_keystore_service_lock(). */
int ca_keystore_service_unlock(const u8 *p2);

/* Wipe di tutte le chiavi caricate in keys_store     stato INACTIVE. */
void ca_keystore_service_lock(void);

/* Ritorna true se le chiavi sono caricate (stato ACTIVE). */
bool ca_keystore_service_is_active(void);
```

State machine

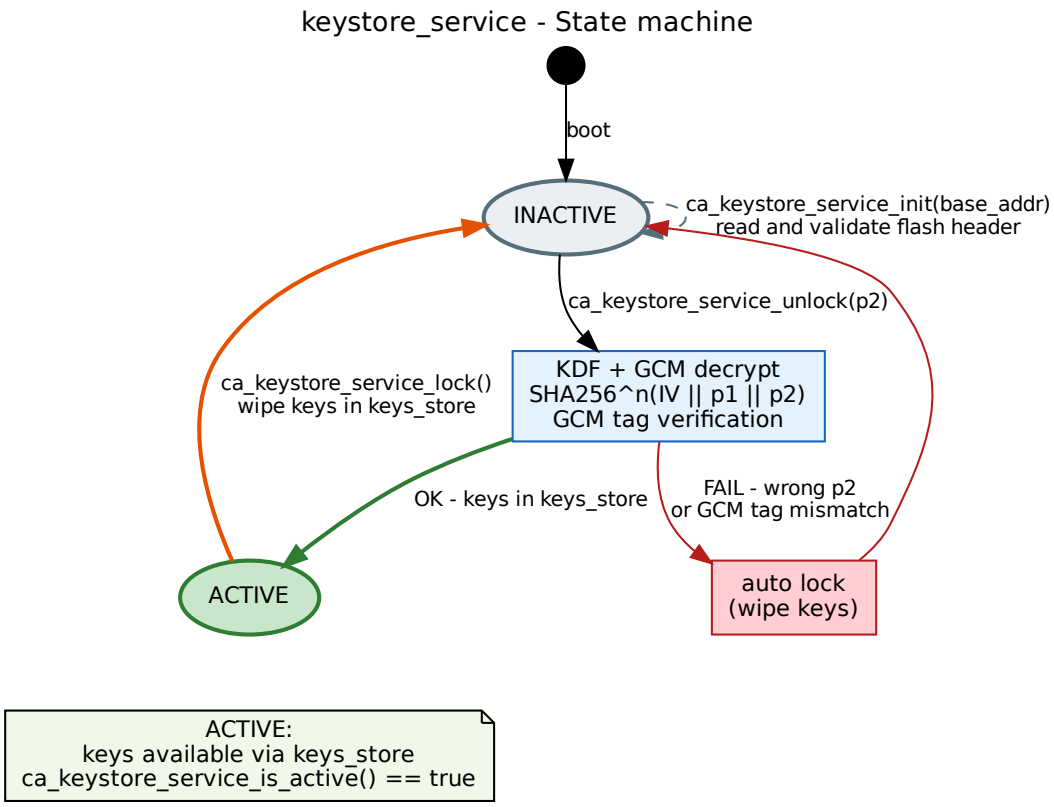


Figure 15.4: State machine keystore_service

Any error in `unlock` (incorrect p2, corrupted flash, GCM tag mismatch, allocation failed) calls `lock()` automatically: keys are never partially loaded.

Internal unlock sequence

```
ca_keystore_service_unlock(p2)

        derive_key(hdr.IV, hdr.p1, p2, hdr.n_evolve, decrypt_key)
               SHA256^n_evolve(IV || p1 || p2)     32 byte

        ca_aes_ctx_setup(ctx, GCM, 256, decrypt_key, hdr.gcm_nonce)
        ca_memzero(decrypt_key)                 wipe immediato

        ca_aes_gcm_decrypt(ctx,
           AAD=hdr, payload_enc, payload_dec, tag)
               tag mismatch?     lock() + return -1

        per ogni ca_flash_key_entry_t in payload_dec:
      ptr = ca_keys_store_allocate(32)
      ca_keys_store_copy(ptr, entry.key, 32)
      ca_keys_store_lock_completely(ptr)
      [imposta scadenza se entry.start/stop != 0]
```

```
        s_key_ptrs[i] = ptr

          ca_memory_free(payload_dec, wipe=true)
          s_state = KSS_ACTIVE
```

Typical usage

```
/* 1. In ca_os_configure(): init FLASH keystore */
if (ca_keystore_service_init(FLASH_KEYSTORE_BASE) != 0) {
    /* flash non provisionata o header corrotto */
    error_handler();
}

/* 2. Al login dell'utente / dopo ricezione p2 da canale sicuro */
u8 p2[16];
receive_p2_from_secure_channel(p2);     /* es. via UART cifrata, NFC, ecc. */

if (ca_keystore_service_unlock(p2) != 0) {
    ca_stdout_put_n("Unlock fallito     p2 errato o flash corrotta");
    return;
}
ca_memzero(p2, sizeof(p2));             /* wipe p2 dallo stack */

/* 3. Le chiavi sono ora in keys_store, accessibili via ptr */
/* ... usa le chiavi con aes_channel_service o direttamente ... */

/* 4. Al logout o fine sessione */
ca_keystore_service_lock();
```

Note: key provisioning in flash

The flash block must be written during production/provisioning with an external tool that knows `IV`, `p1`, `p2` and the keys to be protected. The device never writes the block itself — it only receives `p2` at runtime.

15.1.14 13.14 ECDSA P-256 (`ecdsa_p256.h`)

Why

The TLS 1.3 (HTTPS) server signs the exchange with its certificate key, and browsers do not accept Ed25519 certificates: **ECDSA on P-256** (`ecdsa_secp256r1_sha256`) is required. None of our boards do this in hardware. The H563 has a PKA, but the H523/562/563 series uses it in **limited mode**: only signature verification (RM0481 §36.3). A server must sign, so ECDSA is software on all MCUs, and it is the same code everywhere.

Formats

Data	Bytes	Format
private key	32	integer `d` in [1, n-1], big-endian
public key	64	`X` `Y` (without the `0x04` prefix of the SEC1 form)
signature	64	raw `r` `s` (TLS wants it in DER: conversion is done by `tls.c`)

Deterministic Nonce (RFC 6979)

The `k` value of each signature does not come from the RNG but from HMAC-SHA256 on the private key and message hash. A weak or repeated RNG cannot leak the key (with ECDSA, two signatures with the same `k` reveal it), and the signature is verifiable against published vectors.

Implementation

- 8 32-bit limbs, a single Montgomery multiplier (CIOS) for the `p` field and for the `n` order; inverses using Fermat's Little Theorem.
- Points in projective coordinates with the **complete** addition formulas by Renes-Costello-Batina (2016): a single formula for addition, doubling, and point at infinity, without special cases.
- Signing (secret data `k`): "always add and select with a mask", without branches or memory accesses that depend on `k`.
- Verification (public data): Shamir's trick `u1G + u2Q`, variable time. Rejects `r/s` outside [1, n-1] and a public key that does not lie on the curve.

API

```
bool ca_ecdsa_p256_public_from_private(u8 *public_key, const u8 *private_key);
bool ca_ecdsa_p256_keygen(u8 *public_key, u8 *private_key);       /* RNG */
bool ca_ecdsa_p256_sign_hash(u8 *signature, const u8 *hash, const u8 *
    private_key);
bool ca_ecdsa_p256_sign(u8 *signature, const u8 *message, u32 len, const u8 *
    private_key);
bool ca_ecdsa_p256_verify_hash(const u8 *signature, const u8 *hash, const u8 *
    public_key);
bool ca_ecdsa_p256_verify(const u8 *signature, const u8 *message, u32 len, const
     u8 *public_key);
```

Flag: `EN_ECDSA_P256` (requires `EN_SHA256`), `EN_ECDSA_P256_TEST` for self-test, `EN_CLI_ECDSA_P256` for the `p256 test | keygen` command. The self-test is in example 66.

Measured timings (23/10/2026)

MCU	Signature	Verification
H753	17 ms	13 ms
F767	36 ms	29 ms
H563 (250 MHz)	43 ms	36–41 ms
U575	67 ms	132 ms
F446	70 ms	140 ms

On U575 and F446, verification is slower than signing, unlike the others: not yet investigated. For a TLS server, only the signature counts (one per handshake).

15.1.15 13.15 Crypto module status summary

Module	Status	Notes
CRC32/HASH/RNG Driver	**HW Tested**	All families

Module	Status	Notes
AES (HW CRYP)	**HW Tested**	H753 via CRYP
AES (software)	**HW Tested**	U575+F407: ECB/CTR/GCM 256; NIST vectors 4/4 (`aes_nist test`), ex26 aes/aes_gcm PASS. AES-128 (23/10): 184/184 on PC against `cryptography`, in use in TLS on H563/F767/U575
AES-128 on CRYP	**HW Tested**	H753, via TLS 1.3 (ex45/ex80, 23/10)
AES (HW AES U5xx)	**Code**	`aes_u5xx.c` driver for U585/U5A5; not tested (U575 board = SW)
`keys_store`	**HW Tested**	MPU region, expiration, max 20 keys
`aes_channel_service`	**HW Tested**	GCM-256, IV from HW RNG, 16B padding
`keystore_service`	**HW Tested**	KDF SHA256ˆn, AES-GCM, key split p1/p2
`kdf` HMAC/HKDF/PBKDF2	**HW Tested**	7/7 RFC vectors on H753 (ex66 `sha256 kdf`); PBKDF2 100000 it = 2.25 s
`aes_channel_service` AAD header	**HW Tested**	ex46/ex47 on H753, F767, H563 (15/10)
`secure_pipe` link	**HW Tested**	ex97 13/13 on H753, F767, H563; app_3 between H753 and H563
`keys_store` space recovery	**HW Tested**	ex46/47/97 on H753 and H563 after `free` fix
ECDSA P-256	**HW Tested**	RFC 6979 A.2.5 + negative cases, ex66 10/10 on the 5 Nucleo boards (23/10)

Previous chapter: caOS Thread Pool
Next: caOS Network Stack - Low Level
Practical examples: Example 26 — CRC/HASH/AES/RNG · Example 46 — AES ping-pong · Example 47 — Keystore Service · Example 66 — SSH primitives + KDF + P-256 · Example 97 — Secure pipe link

Chapter 16

caOS Network Stack: Low Level

16.1 Chapter 14 - caOS Network Stack: Low Level

This section describes the physical and transport layers of the caOS network stack: zero-copy ETH driver, DMA-safe buffer pool, MPU configuration, initialization sequence and link control.

16.1.1 14.0 Zero-copy Architecture

The caOS ETH driver uses a **DMA-safe buffer pool** (NCacheable) to eliminate copies between CPU and DMA in the RX/TX hot path:

```
[ DMA ETH ]          [ Pool NCacheable ]          [ CPU / task_net ]
```

No `memcpy`: the DMA writes directly to the pool, the processor reads directly from the pool. The only necessary condition is that the pool be in an **NCacheable** region (MPU) to avoid inconsistency between CPU D-cache and DMA.

Buffer pool — `ca_net_buf_t`

```c
/* caos/net/inc/net_buf.h */
#define NET_BUF_SIZE       2048U   /* total structure size */
#define NET_BUF_HDR_SIZE     64U   /* metadata header */
#define NET_BUF_DATA_SIZE  1984U   /* data area (2048 - 64) */
#define NET_BUF_HEADROOM    128U   /* prepend room for protocol headers */
#define NET_BUF_COUNT        64U   /* number of buffers in the pool */
#define NET_BUF_INVALID  0xFFFFU   /* sentinel index "no buffer" */

/* RX parse cache -- filled step by step by the L2->L3->L4 pipeline */
typedef struct {
    u16    l3_off;    /* L3 header offset from frame start */
    u16    l4_off;    /* L4 header offset from frame start; 0 if absent */
    ca_ipv4_t src_ip;    /* source IPv4 (spa for ARP) */
    ca_ipv4_t dst_ip;    /* destination IPv4 (tpa for ARP) */
    u16    src_port;  /* source UDP/TCP port */
    u16    dst_port;  /* destination UDP/TCP port */
    u16    vlan_id;   /* 802.1Q VID (0 = untagged) */
    u8     vlan_pcp;  /* 802.1Q PCP */
    u8     ip_proto;  /* IP_PROTO_ICMP/UDP/TCP; 0 if not IP */
} ca_net_buf_rx_t;                                          /* 20 byte */

typedef struct {
    u16                next;       /* next buffer index (chain) */
    u16                data_off;   /* offset payload in data[] */
    u16                data_len;   /* valid data length */
```

```
    u8                flags;      /* NET_BUF_F_IN_USE, NET_BUF_F_TX */
    u8                proto;      /* NET_PROTO_ARP/IP4/NONE */
    u32               timestamp;  /* ca_os_tick() at reception */
    ca_net_buf_rx_t   meta_rx;    /* cache parse RX */
    u8                data[NET_BUF_DATA_SIZE];  /* 1984 byte DMA-safe */
} ca_net_buf_t;                                              /* 2048 byte */
```

`data_off = NET_BUF_HEADROOM = 128` leaves room for header prepend (e.g. adding IP/UDP before the payload without reallocation). DMA descriptors point to `b->data + b->data_off`.

Buffer pool API

Function	Description
`ca_net_buf_pool_init()`	Initialize pool in D2 SRAM1 NCacheable; call before `ca_eth_init()`
`ca_net_buf_alloc()`	Allocate a buffer; returns `NET_BUF_INVALID` if pool is exhausted
`ca_net_buf_free(idx)`	Free a buffer; DO NOT call if driver owns it
`ca_net_buf_ptr(idx)`	Pointer to `ca_net_buf_t` from index
`ca_net_buf_free_count()`	Available buffers in pool

ETH zero-copy API

```
/* Transmission -- no copy */
i32 ca_eth_send(u16 buf_idx);
/* CA_ETH_SEND_OK (0):          the driver owns buf_idx (deferred free)
   CA_ETH_SEND_ERR_LINK (-102): link down (state seen by the last
                                ca_eth_link_poll(), DOWN until the first poll)
   CA_ETH_SEND_ERR_TX   (-101): TX ring full or MAC not initialized
   In EVERY case the driver owns buf_idx: the caller never frees it
   (freeing it on error was a double free: the driver already gave it back) */

/* Reception -- zero-copy swap */
bool ca_eth_recv(u16 *out_idx, u16 new_idx);
/* true:  *out_idx holds the frame (to process and then free)
          new_idx is now in DMA (do not touch)
   false: new_idx not consumed -> ca_net_buf_free(new_idx) */

/* Buffers held by the driver (for a correct leak check) */
u16 ca_eth_pool_held_count(void);
/* Typical steady-state value: 4 (fixed RX) + 0-4 (deferred TX) */
```

16.1.2 14.1 MPU Configuration — NCacheable Regions

```
/* MPU region 3: pool D2 SRAM1 -- configurata in ca_eth_hw_init() */
MPU->RNR  = 3U;
MPU->RBAR = 0x30000000U;                        /* base pool */
MPU->RASR = MPU_RASR_ENABLE_Msk
          | (16U << MPU_RASR_SIZE_Pos)          /* 128 KB */
          | (3U  << MPU_RASR_AP_Pos)            /* RW/RW */
          | (1U  << MPU_RASR_TEX_Pos)           /* TEX=001 */
          | MPU_RASR_S_Msk;                     /* Shareable */
/* C=0, B=0 -> Non-cacheable */

/* MPU region 4: .eth_dma D2 SRAM3 */
MPU->RNR  = 4U;
MPU->RBAR = (u32)tx_desc;                       /* base .eth_dma */
```

```
MPU->RASR = MPU_RASR_ENABLE_Msk
          | (14U << MPU_RASR_SIZE_Pos)          /* 32 KB */
          | (3U  << MPU_RASR_AP_Pos)
          | (1U  << MPU_RASR_TEX_Pos)
          | MPU_RASR_S_Msk;
```

IMPRECISERR: if `.eth_dma` is in Write-back cacheable region, the CPU write buffer may delay the write of the OWN bit to the physical RAM. The DMA reads the old value -> asynchronous bus fault (CFSR bit 10). The MPU NCacheable resolves the issue at the root.

16.1.3 14.2 Network stack modules

Module	File	EN_ flag	Description
ETH driver	`caos/drivers/src/eth.c` + hw/	`EN_ETH`	DMA TX/RX, IRQ, buffer pool
Net proto	`caos/net/src/net_proto.c`	`EN_NET`	pure decoder/builder (ARP/IP/ICMP/TCP/UDP)
ARP service	`caos/net/src/arp_service.c`	`EN_ARP_SERVICE`	ARP cache, GARP, MAC resolution
Route service	`caos/net/src/route_service.c`	`EN_ROUTE_SERVICE`	route table + longest-prefix match
DHCP service	`caos/net/src/dhcp_service.c`	`EN_DHCP_SERVICE`	DORA client, lease renewal
NTP service	`caos/net/src/ntp_service.c`	`EN_NTP_SERVICE`	NTP client with automatic polling
DNS service	`caos/net/src/dns_service.c`	`EN_DNS_SERVICE`	DNS client with cache
mDNS service	`caos/net/src/mdns_service.c`	`EN_MDNS_SERVICE`	mDNS .local responder
Syslog svc	`caos/net/src/syslog_service.c`	`EN_SYSLOG_SERVICE`	remote UDP logging on port 514
TCP service	`caos/net/src/tcp_service.c`	`EN_TCP_SERVICE`	PCB, RTO timer, listen/connect
UDP server	`caos/net/src/udp_server.c`	`EN_UDP_SERVER`	multi-client with worker task
UDP client	`caos/net/src/udp_client.c`	`EN_UDP_CLIENT`	async or blocking send/recv
IP defrag	`caos/net/src/ip_defrag.c`	`EN_IP_DEFRAG`	fragment reassembly RFC 791
TFTP service	`caos/net/src/tftp_service.c`	`EN_TFTP_SERVICE`	TFTP client RFC 1350 GET/PUT, OTA
HTTP service	`caos/net/src/http_service.c`	`EN_HTTP_SERVICE`	HTTP/1.1 server with route table
MQTT service	`caos/net/src/mqtt_service.c`	`EN_MQTT_SERVICE`	MQTT 3.1.1 QoS 0 client

Module	File	EN_ flag	Description
Modbus svc	caos/net/src/modbus_service.c	EN_MODBUS_SERVICE	Modbus/TCP client
iperf svc	caos/net/src/iperf_service.c	EN_IPERF_SERVICE	TCP throughput measurement
Packet tracer	caos/net/src/packet_tracer_service.c	EN_PACKET_TRACER	bare-metal tcpdump

Basic enable in `os_config.h`:

```
#define EN_ETH    1   /* driver Ethernet HW */
#define EN_NET    1   /* stack net_proto */
```

`EN_NET` requires **at least one L2 interface**: ETH (`EN_ETH=1`) or USB-NCM (`EN_USB && EN_USB_NCM`). If neither is active, `EN_NET` is forced to 0 (and with it all sub-protocols and services).

16.1.4 14.2.1 Interface abstraction — `ca_net_iface_t`

The `net.c` core does not communicate directly with the driver: it uses an **abstract interface** (`ca_net_iface_t`, function pointer structure). This way the same IP stack runs interchangeably on **Ethernet** or on **USB-NCM** (USB Ethernet), and in the future on WiFi/PPP, changing only the registered interface.

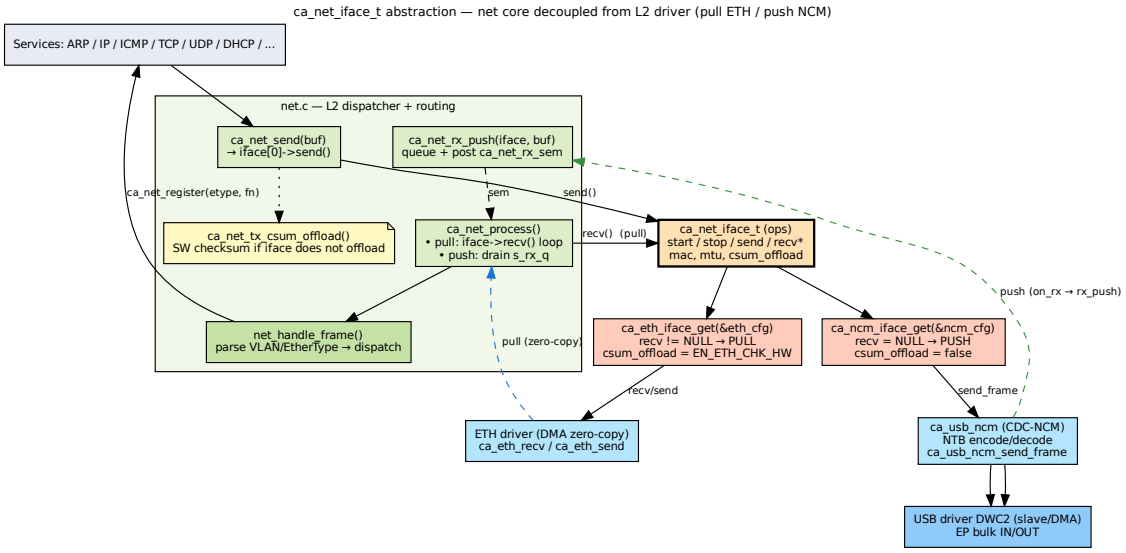


Figure 16.1: net_iface

Source: `doc/diagrams/net_iface_arch.dot` *(regenerated from* `make diagrams`*).*

```
/* caos/net/inc/net_iface.h */
struct ca_net_iface {
    cstr   name;              /* "eth0", "usb0" -- diagnostics */
    u8     id;                /* assigned by ca_net_register_iface */
```

```
    ca_mac_t mac;                /* L2 MAC of the interface */
    u16   mtu;                /* MTU L2 (payload max) */
    bool  up;                 /* state managed by the core via start/stop */
    bool  csum_offload;       /* true = HW calcola checksum IP/L4 (ETH DMA);
                                 false = the SW computes them (NCM) */
    void *priv;               /* ctx driver (ca_eth_cfg_t*, ctx NCM) */
    bool (*start)(ca_net_iface_t *self);
    bool (*stop) (ca_net_iface_t *self);
    bool (*send) (ca_net_iface_t *self, u16 buf_idx);
    bool (*recv) (ca_net_iface_t *self, u16 *out_idx, u16 new_idx); /* opz. */
};
```

Hybrid RX model — pull / push

The abstraction supports **two RX models** to cover drivers with different characteristics without rewriting the core:

Model	`ops.recv`	Who delivers the frames	Used by
pull (zero-copy)	!= NULL	`ca_net_process()` calls `recv()` in loop	ETH (DMA)
push	NULL	the driver calls `ca_net_rx_push()`	USB-NCM

`ca_net_process()` for each iface with `recv` executes a zero-copy pull-loop; then drains the push queue (`ca_net_rx_push`). The two paths converge in the same L2 parser (`net_handle_frame`).

Adapter

Two factories wrap the concrete drivers in a `ca_net_iface_t`:

```
/* ETH: pull zero-copy, checksum HW (caos/net/inc/eth_iface.h) */
ca_net_iface_t *ca_eth_iface_get(ca_eth_cfg_t *cfg);   /* csum_offload =
    EN_ETH_CHK_HW */

/* USB-NCM: push, checksum SW (caos/net/inc/ncm_iface.h) */
ca_net_iface_t *ca_ncm_iface_get(ca_usb_ncm_cfg_t *cfg); /* csum_offload = false
    */
```

Per-interface Checksum

`net_proto` builders leave the IP/ICMP/TCP/UDP checksum fields to 0 **only if the TX interface does HW offload** (`ca_net_tx_csum_offload()`). ETH delegates to the MAC, which calculates them and **inserts** them by overwriting the field: CIC=11 in the TX descriptor, valid because the MTL is in store-and-forward (TSF). The NCM, which lacks offload, has the software calculate the checksums. The choice is runtime, per-interface:

```
if (!ca_net_tx_csum_offload())
    ip4->cksum = ca_htons(ca_net_cksum((const u8 *)ip4, 20U));
```

> The check is **at 13 call-sites**, at runtime. `EN_ETH_CHK_HW` no longer has (a) `EVAL_` macro: the old `EVAL_NO_ETH_CHK_HW` was generated by `caos_model.py` in the generic form (`__VA_ARGS__` dry). A previous version contained the runtime test written by hand, and a regen of the config silently reverted it to the generic form: since then the SW checksum has always been calculated — wasted work on ETH — and with `EN_ETH_CHK_HW=0` it is **never** calculated **by anyone**. Having no users left, it was removed from the model (03/10) because nobody should put the test back in.

API interface

```
bool ca_net_init(ca_net_iface_t *iface);          /* registra iface default + start */
bool ca_net_register_iface(ca_net_iface_t *iface);/* additional iface (multi-iface) */
bool ca_net_rx_push(ca_net_iface_t *iface, u16 buf_idx); /* RX push (driver -> core) */
bool ca_net_tx_csum_offload(void);                /* true if the TX iface does HW offload */
```

16.1.5 14.3 Initialization Sequence

```
/* ca_os_configure() -- hardware init and core services */
eth_gpio_init();

ca_eth_cfg_t eth_cfg = { .link_cb = on_link_change, /* ... */ };
ca_net_init(ca_eth_iface_get(&eth_cfg));   /* ETH; per USB-NCM: ca_ncm_iface_get(&ncm) */

ca_ipv4_t zero = { .addr = 0U };
ca_arp_service_init(s_mac, zero);
ca_dhcp_service_init(s_mac, on_dhcp_bound, on_dhcp_expire);
/* other services: ca_ntp_service_init(), ca_dns_service_init(), ... */

/* ca_os_main() -- avvio task */
ca_arp_service_start();
ca_dhcp_service_start();
ca_tcp_service_start();
ca_os_job("net",     CA_TH_LEVEL2, task_net,     NULL, STACK_SMALL, CA_JOB_STD, CA_FAULT_RESTART_THREAD);
ca_os_run(true);
```

task_net — type structure

```
static u32 task_net(void *arg)
{
    ca_net_proto_init(&net_cfg);
    ca_net_proto_bind_udp(DHCP_CLIENT_PORT, dhcp_udp_handler);
    /* bind other services: ca_ntp_service_bind(), ca_dns_service_bind(), ... */

    ca_eth_link_poll();
    ca_os_delay_ms(200U);
    ca_dhcp_service_net_ready();   /* unblocks the DHCP task */

    while (1) {
        ca_sem_wait(&ca_net_rx_sem);
        ca_net_process();
    }
    return 0U;
}
```

Callback DHCP bound

```
static void on_dhcp_bound(ca_ipv4_t ip, ca_ipv4_t mask, ca_ipv4_t gw, ca_ipv4_t dns)
{
    (void)mask; (void)dns;
    ca_net_proto_set_local_ip(ip);
```

```
    ca_arp_service_update_local_ip(ip);
    ca_arp_send_garp();
    /* notify the application services: */
    /* ca_http_update_local_ip(ip); */
    /* ca_iperf_update_local_ip(ip); */
    /* ca_sem_post(&s_dhcp_sem); */
}
```

16.1.6 14.4 Configuration — `ca_net_cfg_t`

All configuration is collected in `caos/net/inc/net_cfg.h`.

Attention to types `ca_ipv4_t` and `ca_mac_t`: always use `.v = {}` for initialization:

```
ca_net_cfg_t net_cfg = {
    .ip   = { .v = { 192U, 168U, 10U, 33U } },
    .mask = { .v = { 255U, 255U, 255U,  0U } },
    .mac  = { .v = { 0x02U, 0x00U, 0x00U, 0x00U, 0x00U, 0x33U } },
    .arp  = { .renew_ms = 30000U },
    .icmp = { .echo_reply = true },
#if EN_NET_DHCP
    .dhcp = { .timeout_ms = 5000U, .on_bound = my_dhcp_callback },
#endif
#if EN_NET_NTP
    .ntp  = { .server_ip = { .v = {192U,168U,10U,1U} }, .poll_ms = 64000U },
#endif
#if EN_NET_SYSLOG
    .syslog = { .server_ip = { .v = {192U,168U,10U,200U} },
                .facility = SYSLOG_FAC_LOCAL0 },
#endif
#if EN_NET_DNS
    .dns  = { .server_ip = { .v = {8U,8U,8U,8U} } },
#endif
#if EN_NET_MDNS
    .mdns = { .hostname = "caos-board" },
#endif
};

ca_net_proto_init(&net_cfg);
```

16.1.7 14.5 Link control

Software to bring down and restore the Ethernet link, with optional PHY nRST pin control:

```
/* Takes the link down: disables the RCC clock, stops the ETH DMA */
void ca_net_link_down(void);

/* Restores the link: re-enables the RCC clock, restarts the ETH DMA */
void ca_net_link_up(void);

/* Configures the PHY nRST GPIO (optional) */
void ca_net_set_phy_nrst(u8 port, u8 pin);
```

`port = 0xFF` means "not wired" (no GPIO control). The structure `ca_net_cfg_t` also exposes `mtu`, `phy_nrst_port`, `phy_nrst_pin` for static configuration.

16.1.8 14.6 Source files

File	Role
caos/net/inc/net_buf.h	ca_net_buf_t, constants pool
caos/net/src/net_buf.c	ca_net_buf_pool_init, alloc, free, ptr, free_count
caos/net/src/net.c	ca_net_init, ca_net_process, ca_net_send, ca_net_stats
caos/net/inc/net_proto.h	Pure net_proto API (decode/encode)
caos/net/src/net_proto.c	ARP, ICMP, UDP, DHCP, NTP, DNS, mDNS decoder/builder
caos/net/src/net_proto_tcp.c	TCP builder: SYN/ACK/DATA/FIN/RST
caos/net/inc/net_cfg.h	configuration structures, ca_ipv4_t, ca_mac_t
caos/drivers/inc/eth.h	ETH driver API (zero-copy)
caos/drivers/src/eth.c	MCU-independent wrapper
hw/h7xx/drivers/eth_h7xx.c	GMAC 4.x implementation, MPU, ISR, zero-copy

Chapter 17

caOS Network Stack: net_proto

17.1 Chapter 14 - caOS Network Stack: net_proto (pure decoder/builder)

`net_proto` is the bare-metal frame decoding and building layer of caOS. All functions are **pure** (no side effects, no global state), **zero-copy** (pointers to original raw buffer), and identical on PC and MCU.

17.1.1 14.7 Overview

```
Applicazione / task caOS
  (ARP cache, timer DHCP/DNS, state machine)
-------------------------------------------------
net_proto  -- decoder/builder puri
  L2: Ethernet, VLAN 802.1Q, ARP
  L3: IPv4, ICMP echo
  L4: UDP, TCP
  App: DHCP, NTP, Syslog, DNS, mDNS
-------------------------------------------------
net.c / net_buf.c -- dispatcher RX, buffer pool
-------------------------------------------------
Driver ETH bare-metal (CMSIS)
```

Fundamental properties: - **Pure functions**: no side effects, no global state - **Zero-copy**: output pointers directly into the original raw buffer - Return `false` on malformed input leaving `*out` unchanged - Identical on PC (test) and MCU (production)

17.1.2 14.8 Decoding pipeline

Each received frame goes through a pipeline of pure functions:

```
raw frame (u8 *, len)
  +-- ca_net_proto_l2()    -> const ca_l2_frame_t *   (filtro MAC, VLAN strip)
       +-- EtherType ARP   -> ca_net_proto_arp()        -> const ca_arp_frame_t
    *
       +-- EtherType IPv4  -> ca_net_proto_ip4()        -> const ca_ip4_frame_t
    *
                              +-- proto UDP  -> ca_net_proto_udp()        ->
    const ca_udp_frame_t *
                              +-- proto ICMP -> ca_net_proto_icmp_echo()  ->
    const ca_icmp_echo_t *
```

```
                                +-- proto TCP  -> ca_net_proto_tcp()        ->
    const ca_tcp_frame_t *
```

Complete example — UDP reception:

```
const ca_l2_frame_t  *l2;
const ca_ip4_frame_t *ip4;
const ca_udp_frame_t *udp;

ca_l2_result_t r = ca_net_proto_l2(frame, len, local_mac, &l2);
if (r == CA_L2_DROP) return;

if (ca_ntohs(l2->ethertype) != ETH_P_IP) return;
if (!ca_net_proto_ip4(l2, len, &ip4)) return;
if (!ca_net_proto_udp(ip4, &udp)) return;

const u8 *payload     = CA_UDP_PAYLOAD(udp);
u16       payload_len = (u16)(ca_ntohs(udp->length) - 8U);
process_udp(ca_ntohs(udp->src_port), ca_ntohs(udp->dst_port),
            payload, payload_len);
```

17.1.3 14.9 L2 — Ethernet and VLAN 802.1Q

Wire-format

```
/** Ethernet II header -- 14 byte (IEEE 802.3) */
typedef struct {
    ca_mac_t  dst;       /* destination MAC */
    ca_mac_t  src;       /* source MAC */
    u16    ethertype; /* ETH_P_IP / ETH_P_ARP / ETH_P_VLAN (big-endian) */
} __attribute__((packed)) ca_l2_frame_t;

/** 802.1Q VLAN header -- 18 byte */
typedef struct {
    ca_mac_t  dst;
    ca_mac_t  src;
    u16    tpid;      /* 0x8100 big-endian */
    u16    tci;       /* PCP[15:13]|DEI[12]|VID[11:0] big-endian */
    u16    ethertype; /* inner EtherType big-endian */
} __attribute__((packed)) ca_l2_vlan_frame_t;
```

Decoder

```
ca_l2_result_t ca_net_proto_l2(const u8 *frame, u16 len,
                               ca_mac_t local_mac,
                               const ca_l2_frame_t **out);
```

Result	Condition
`CA_L2_ACCEPT_UNICAST`	`dst == local_mac`
`CA_L2_ACCEPT_BROADCAST`	`dst == FF:FF:FF:FF:FF:FF`
`CA_L2_ACCEPT_MULTICAST`	first byte dst I/G bit == 1
`CA_L2_DROP`	frame not for us or too short

`local_mac = {0}` enables promiscuous mode.

VLAN 802.1Q: if `ca_ntohs((*out)->ethertype) == ETH_P_VLAN` the pointer is cast to `const ca_l2_vlan_frame_t *`. Read `VID` and `PCP` with the macros:

```
u16 vid = CA_VLAN_VID(vl2->tci);   /* 0-4094 */
u8  pcp = CA_VLAN_PCP(vl2->tci);   /* 0-7    */
```

Payload Navigation Macros

```
CA_L2_PAYLOAD(l2)        /* first byte after Ethernet header (14 bytes) */
CA_L2_VLAN_PAYLOAD(l2v)  /* first byte after 802.1Q header  (18 bytes) */
CA_IP4_PAYLOAD(ip4)      /* first byte after IPv4 header (IHL-aware)   */
CA_UDP_PAYLOAD(udp)      /* first byte after UDP header     ( 8 bytes) */
```

17.1.4 14.10 Proto — ARP

Wire-format

```
/** ARP per IPv4/Ethernet -- 28 byte (RFC 826) */
typedef struct {
    u16    htype;  /* ARP_HTYPE_ETH = 1 (big-endian) */
    u16    ptype;  /* ARP_PTYPE_IP = 0x0800 (big-endian) */
    u8     hlen;   /* 6 */
    u8     plen;   /* 4 */
    u16    oper;   /* ARP_OP_REQUEST=1 / ARP_OP_REPLY=2 (big-endian) */
    ca_mac_t  sha;    /* sender hardware address */
    ca_ipv4_t spa;    /* sender protocol address */
    ca_mac_t  tha;    /* target hardware address */
    ca_ipv4_t tpa;    /* target protocol address */
} __attribute__((packed)) ca_arp_frame_t;
```

Decoder

```
bool ca_net_proto_arp(const ca_l2_frame_t *l2, u16 len,
                      const ca_arp_frame_t **out);
```

Returns `false` if `htype != Ethernet`, `ptype != IPv4`, `hlen != 6`, `plen != 4` or frame too short.

ARP Reply Builder

```
u16 ca_net_proto_arp_build_reply(const ca_arp_frame_t *req,
                                 ca_mac_t  local_mac,
                                 ca_ipv4_t local_ip,
                                 u8    *out_frame);  /* buffer >= 42 byte */
```

Returns 42 (14 Ethernet + 28 ARP) or 0 if `req->oper != ARP_OP_REQUEST`.

GARP Builder

```
u16 ca_net_proto_arp_build_garp(ca_mac_t  local_mac,
                                ca_ipv4_t local_ip,
                                u8    *out_frame);  /* buffer >= 60 byte */
```

Builds an ARP REQUEST with `spa == tpa == local_ip` (Gratuitous ARP, RFC 5227). Resulting frame: 60 bytes. Pure function (stateless).

Typical flow

```
RX frame -> ca_net_proto_l2() -> ca_net_proto_arp()
  oper == ARP_OP_REQUEST && tpa == local_ip ?
    -> ca_net_proto_arp_build_reply()
    -> ca_net_send()
```

17.1.5 14.11 Proto — IPv4

Wire-format

```
/** IPv4 header -- 20 byte minimo (RFC 791) */
typedef struct {
    u8      ver_ihl;     /* version[7:4]=4 | IHL[3:0] in word a 32-bit */
    u8      dscp_ecn;
    u16     total_len;   /* total length incl. header (big-endian) */
    u16     id;
    u16     flags_frag;  /* flags[15:13] | fragment offset[12:0] (big-endian) */
    u8      ttl;
    u8      proto;       /* IP_PROTO_ICMP=1 / TCP=6 / UDP=17 */
    u16     cksum;       /* header checksum (big-endian) */
    ca_ipv4_t src;
    ca_ipv4_t dst;
} __attribute__((packed)) ca_ip4_frame_t;
```

Decoder

```
bool ca_net_proto_ip4(const ca_l2_frame_t *l2, u16 len,
                      const ca_ip4_frame_t **out);
```

Verification: version == 4, IHL >= 20, `total_len <= payload_len`, header checksum RFC 1071.

Internet checksum

```
u16 ca_net_cksum(const u8 *buf, u16 len);
```

Implementation of RFC 1071. Returns 0 if the data is correct (RX verification).

> **TX Note**: `ca_net_cksum()` returns the checksum in host format (little-endian on ARM). For TX, **always** assign with `ca_htons()`:
>
> ```
> ip4->cksum = ca_htons(ca_net_cksum((const u8 *)ip4, 20U));
> ```

17.1.6 14.12 Proto — UDP

Wire-format

```
/** UDP header -- 8 byte (RFC 768) */
typedef struct {
    u16     src_port;  /* (big-endian) */
    u16     dst_port;  /* (big-endian) */
    u16     length;    /* header + data (big-endian) */
    u16     checksum;  /* 0 = absent (RFC 768) */
} __attribute__((packed)) ca_udp_frame_t;
```

Decoder

```
bool ca_net_proto_udp(const ca_ip4_frame_t *ip4,
                      const ca_udp_frame_t **out);
```

Check `proto == IP_PROTO_UDP`, `udp_len >= 8`, `udp_len <= ip4_payload_len`. The UDP checksum is optional (RFC 768): if 0 it is accepted without verification.

17.1.7 14.13 Proto — ICMP

Wire-format

```
/** ICMP echo request/reply -- 8 byte (RFC 792) */
typedef struct {
    u8     type;      /* 8 = echo request, 0 = echo reply */
    u8     code;      /* 0 */
    u16    checksum;  /* (big-endian) */
    u16    id;        /* identifier (big-endian) */
    u16    seq;       /* sequence number (big-endian) */
} __attribute__((packed)) ca_icmp_echo_t;
```

Decoder

```
bool ca_net_proto_icmp_echo(const ca_ip4_frame_t *ip4,
                            const ca_icmp_echo_t **out);
```

Accept only `type == 8` (request) or `type == 0` (reply).

Builder Echo Reply

```
u16 ca_net_proto_icmp_build_echo_reply(
    const ca_l2_frame_t  *req_l2,
    const ca_ip4_frame_t *req_ip4,
    const ca_icmp_echo_t *req_icmp,
    ca_mac_t  local_mac,
    ca_ipv4_t local_ip,
    u8    *out_frame,
    u16    out_max);
```

Produce complete Ethernet+IPv4+ICMP frame with IPv4 and ICMP checksums.

17.1.8 14.14 Proto — TCP (wire-format and builder)

Wire-format

```
typedef struct {
    u16 src_port;
    u16 dst_port;
    u32 seq;
    u32 ack_seq;
    u8  data_off;   /* [7:4] = header len in parole a 32-bit */
    u8  flags;      /* TCP_FLAG_* */
    u16 window;
    u16 checksum;
    u16 urg_ptr;
} __attribute__((packed)) ca_tcp_frame_t;

#define TCP_FLAG_FIN  0x01U
#define TCP_FLAG_SYN  0x02U
#define TCP_FLAG_RST  0x04U
#define TCP_FLAG_PSH  0x08U
#define TCP_FLAG_ACK  0x10U

#define CA_TCP_HDR_LEN(tcp)   ((u16)(((tcp)->data_off >> 4U) << 2U))
#define CA_TCP_PAYLOAD(tcp)   ((u8*)(tcp) + CA_TCP_HDR_LEN(tcp))
#define TCP_MSS_DEFAULT       1460U
```

Decoder

```
bool ca_net_proto_tcp(const ca_ip4_frame_t *ip4,
                      const ca_tcp_frame_t **out);
```

Builder TCP (`net_proto_tcp.c`)

All builders produce a complete Ethernet+IPv4+TCP frame with pseudo-header checksum RFC 793. They return the frame length or 0.

```
/* SYN con opzione MSS (header TCP 24 byte) */
u16 ca_net_proto_tcp_build_syn(ca_mac_t dst_mac, ca_ipv4_t dst_ip,
                               ca_mac_t src_mac, ca_ipv4_t src_ip,
                               u16 src_port, u16 dst_port,
                               u32 seq, u16 mss,
                               u8 *out_frame, u16 out_max);

/* ACK puro (header 20 byte) */
u16 ca_net_proto_tcp_build_ack(ca_mac_t dst_mac, ca_ipv4_t dst_ip,
                               ca_mac_t src_mac, ca_ipv4_t src_ip,
                               u16 src_port, u16 dst_port,
                               u32 seq, u32 ack_seq, u16 window,
                               u8 *out_frame, u16 out_max);

/* PSH+ACK with data (20-byte header + payload) */
u16 ca_net_proto_tcp_build_data(ca_mac_t dst_mac, ca_ipv4_t dst_ip,
                                ca_mac_t src_mac, ca_ipv4_t src_ip,
                                u16 src_port, u16 dst_port,
                                u32 seq, u32 ack_seq, u16 window,
                                const u8 *data, u16 data_len,
                                u8 *out_frame, u16 out_max);

/* FIN+ACK (header 20 byte) */
u16 ca_net_proto_tcp_build_fin(ca_mac_t dst_mac, ca_ipv4_t dst_ip,
                               ca_mac_t src_mac, ca_ipv4_t src_ip,
                               u16 src_port, u16 dst_port,
                               u32 seq, u32 ack_seq, u16 window,
                               u8 *out_frame, u16 out_max);

/* RST (header 20 byte, ack=0) */
u16 ca_net_proto_tcp_build_rst(ca_mac_t dst_mac, ca_ipv4_t dst_ip,
                               ca_mac_t src_mac, ca_ipv4_t src_ip,
                               u16 src_port, u16 dst_port,
                               u32 seq,
                               u8 *out_frame, u16 out_max);

/* SYN+ACK with MSS option -- server reply to client SYN (24-byte header) */
u16 ca_net_proto_tcp_build_synack(ca_mac_t dst_mac, ca_ipv4_t dst_ip,
                                  ca_mac_t src_mac, ca_ipv4_t src_ip,
                                  u16 src_port, u16 dst_port,
                                  u32 seq, u32 ack_seq, u16 mss,
                                  u8 *out_frame, u16 out_max);
```

17.1.9 14.15 Proto — DHCP (pure functions)

This section describes the **pure** DHCP decoding/construction functions. The complete service with DORA state machine, tasks and callbacks is in section §14.17 DHCP Service.

Constants

```
#define DHCP_SERVER_PORT   67U
#define DHCP_CLIENT_PORT   68U
#define DHCP_MSG_DISCOVER  1U
#define DHCP_MSG_OFFER     2U
#define DHCP_MSG_REQUEST   3U
#define DHCP_MSG_ACK       5U
```

Builder Discover

```
u16 ca_net_proto_dhcp_build_discover(ca_mac_t local_mac, u32 xid,
                                     u8 *out_frame, u16 out_max);
```

Produce a **286 byte** frame: Ethernet(14)+IP(20)+UDP(8)+BOOTP(244).

Offer/ACK Parser

```
bool ca_net_proto_dhcp_parse_offer(const ca_udp_frame_t *udp,
                                   ca_dhcp_offer_t *out);
```

```
typedef struct {
    ca_ipv4_t offered_ip;   /* yiaddr -- offered IP */
    ca_ipv4_t server_ip;    /* siaddr -- DHCP server IP */
    ca_ipv4_t subnet_mask;  /* option 1 */
    ca_ipv4_t gateway;      /* option 3 (first router) */
    ca_ipv4_t dns;          /* option 6 (first DNS) */
    u32    lease_time;   /* option 51, seconds */
    u8     msg_type;     /* DHCP_MSG_OFFER o DHCP_MSG_ACK */
} ca_dhcp_offer_t;
```

17.1.10 14.16 Proto — NTP, Syslog, DNS, mDNS (Pure Functions)

NTP (RFC 5905)

```
#define NTP_PORT         123U
#define NTP_EPOCH_DELTA  2208988800UL  /* NTP epoch (1900) a UNIX (1970) */

u16 ca_net_proto_ntp_build_request(ca_mac_t server_mac, ca_ipv4_t server_ip,
                                   ca_mac_t local_mac,  ca_ipv4_t local_ip,
                                   u8 *out_frame, u16 out_max);
/* Produce 90 byte: ETH(14)+IP(20)+UDP(8)+NTP(48) */

bool ca_net_proto_ntp_parse_response(const ca_udp_frame_t *udp,
                                     ca_ntp_pkt_t *out);
```

Syslog (RFC 3164)

```
#define SYSLOG_PORT       514U
#define SYSLOG_FAC_LOCAL0 16U
#define SYSLOG_SEV_INFO   6U

u16 ca_net_proto_syslog_build(ca_mac_t server_mac, ca_ipv4_t server_ip,
                              ca_mac_t local_mac,  ca_ipv4_t local_ip,
                              u8 facility, u8 severity,
                              const char *msg,
                              u8 *out_frame, u16 out_max);
```

The `PRI` field = `facility * 8 + severity`. UDP payload: `<PRI>message`.

Severity	Value
SYSLOG_SEV_EMERG	0
SYSLOG_SEV_ERR	3
SYSLOG_SEV_WARN	4
SYSLOG_SEV_INFO	6
SYSLOG_SEV_DEBUG	7

DNS (RFC 1035)

```
#define DNS_PORT     53U
#define DNS_TYPE_A   1U    /* record A -- IPv4 */
#define DNS_CLASS_IN 1U

u16 ca_net_proto_dns_build_query(ca_mac_t server_mac, ca_ipv4_t server_ip,
                                 ca_mac_t local_mac,  ca_ipv4_t local_ip,
                                 u16 id, const char *hostname,
                                 u8 *out_frame, u16 out_max);

bool ca_net_proto_dns_parse_response(const ca_udp_frame_t *udp,
                                     ca_dns_response_t *out);
```

The parser handles DNS **pointer compression** (0xC0xx) and the wrapping of multi-level labels (e.g. "pool.ntp.org" -> \x04pool\x03ntp\x03org\x00).

mDNS (RFC 6762)

```
#define MDNS_PORT    5353U
#define MDNS_TTL     120U

u16 ca_net_proto_mdns_build_announce(ca_mac_t local_mac, ca_ipv4_t local_ip,
                                     const char *hostname,
                                     u8 *out_frame, u16 out_max);

bool ca_net_proto_mdns_parse_query(const ca_udp_frame_t *udp,
                                   const char *our_hostname,
                                   bool *is_for_us);
```

Typical flow:

```
Boot:
  ca_net_proto_mdns_build_announce(...) -> ca_net_send()

RX su porta UDP 5353 (multicast 01:00:5E:00:00:FB):
  ca_net_proto_mdns_parse_query(&udp, "caos-board", &is_for_us)
  is_for_us == true?
    -> ca_net_proto_mdns_build_announce(...) -> ca_net_send()
```

17.1.11 14.17 Route lookup — ca_net_resolve_mac

All services use ca_net_resolve_mac (with routing) instead of ca_arp_resolve (without):

```
bool ca_net_resolve_mac(ca_ipv4_t dst, ca_mac_t *mac_out, u32 timeout_ms);
/* equivale a: ca_arp_resolve(ca_route_lookup(dst), mac_out, timeout_ms) */
```

ca_route_lookup(dst) executes in order: 1. On-link: if (dst & local_mask) == (local_ip & local_mask) -> returns dst 2. Longest prefix match: scans route table, selects longest match 3. Default gateway: if configured -> returns default_gw 4. Fallback: returns dst (ARP will likely fail)

When EN_ROUTE_SERVICE=0, ca_net_resolve_mac is a direct alias of ca_arp_resolve (fallback without overhead).

Chapter 18

caOS Network Stack: Network Services

This chapter describes network services with dedicated tasks and state machines: ARP, Route, DHCP, NTP, DNS, mDNS, Syslog, TCP, UDP, IP defrag, TFTP, Packet Tracer, HTTP, MQTT, Modbus/TCP, iperf.

18.1 ARP Service (`EN_ARP_SERVICE`)

The ARP service manages a local **IP-MAC mapping cache** and provides **blocking** resolution with fast-path: if the MAC is already in cache, the function returns immediately.

The ARP task runs at **CA_TH_LEVEL0** (maximum priority).

18.1.1 Architecture

```
Caller (qualsiasi task)
  ca_arp_resolve(ip, mac_out, timeout_ms)
    +-- ca_arp_avail(ip) == true  -> MAC immediato  (fast path)
    +-- cache miss                -> post s_request_sem
                                     wait s_done_sem (con timeout)

ARP task (CA_TH_LEVEL0)
  wait s_request_sem
  loop retry:
    build ARP REQUEST -> broadcast FF:FF:FF:FF:FF:FF
    wait s_reply_sem (ARP_RETRY_MS)
  post s_done_sem

net task
  every ARP frame received -> ca_arp_set(spa, sha)
  se ip == pending -> post s_reply_sem
```

18.1.2 API

Function	Description
`ca_arp_service_init(local_mac, local_ip)`	Init cache; call in `ca_os_configure()`
`ca_arp_service_start()`	Create ARP task (CA_TH_LEVEL0); call in `ca_os_main()`

Function	Description
`ca_arp_service_update_local_ip(local_ip)`	Update local IP after DHCP bind (without reinit)
`ca_arp_resolve(ip, mac_out, timeout_ms)`	Blocking resolution; fast-path on cache hit
`ca_arp_avail(ip)`	Non-blocking cache lookup (only valid non-expired entries)
`ca_arp_find(ip, mac_out)`	Cache lookup without TTL check
`ca_arp_set(ip, mac)`	Insert/update entry; called by net task and DHCP
`ca_arp_send_garp()`	Send Gratuitous ARP (GARP) with configured local MAC/IP
`ca_arp_dump()`	Print cache to stdout
`ca_arp_flush()`	Invalidate all entries

18.1.3 Parameters `os_config.h`

Parameter	Default	Meaning
`ARP_CACHE_SIZE`	8	Maximum entries in cache
`ARP_RETRY_N`	3	Attempts before failure
`ARP_RETRY_MS`	500	Timeout for single response
`ARP_ENTRY_TTL_MS`	60000	Entry expiration (1 min)

`timeout_ms` in `ca_arp_resolve` must be `>= ARP_RETRY_N x ARP_RETRY_MS` (default: 1500 ms).

18.1.4 Typical pattern

```c
/* ca_os_configure() */
ca_arp_service_init(local_mac, local_ip);

/* ca_os_main() */
ca_arp_service_start();

/* after DHCP bound */
ca_arp_service_update_local_ip(new_ip);
ca_arp_send_garp();    /* announce the new IP on the LAN */

/* MAC resolution before sending UDP */
ca_mac_t server_mac;
if (!ca_arp_resolve(server_ip, &server_mac, 1500U)) {
    /* server not reachable */
}
```

> **DO NOT call `ca_arp_service_init()` from `on_dhcp_bound`:** reinitialize internal semaphores while the ARP task is running -> SAFEGUARD overflow. Always use `ca_arp_service_update_local_ip()`.

18.2 Route Service (`EN_ROUTE_SERVICE`)

`route_service` maintains a static route table with **longest prefix match** and a default gateway. It resolves routing towards different subnets.

18.2.1 Configuration

```
/* os_config.h */
#define EN_ROUTE_SERVICE  1
#ifndef NET_ROUTE_MAX
#define NET_ROUTE_MAX  8U   /* max entries in the table */
#endif
```

18.2.2 Public API

```
/* Initialization -- called automatically by ca_net_proto_init() */
void ca_route_init(ca_ipv4_t local_ip, ca_ipv4_t local_mask);

/* Updates the local IP/mask at runtime (e.g. after DHCP bind) */
void ca_route_update_local(ca_ipv4_t local_ip, ca_ipv4_t local_mask);

/* Sets/replaces the default gateway */
bool ca_route_set_gw(ca_ipv4_t gw);

/* Adds a route: dst/prefix_len via gw */
bool ca_route_add(ca_ipv4_t dst, u8 prefix_len, ca_ipv4_t gw);

/* Removes a route by index */
bool ca_route_del(u8 idx);

/* Empties the table (the default gateway stays) */
void ca_route_flush(void);
```

18.2.3 Integration with DHCP

The DHCP service automatically updates the route service after binding: the DHCP default gateway becomes operational without manual actions.

18.2.4 Static configuration (without DHCP)

```
void ca_os_main(void) {
    /* ca_route_init() already called by ca_net_proto_init() with ip+mask */
    ca_ipv4_t gw = { .v = { 192U, 168U, 10U, 1U } };
    ca_route_set_gw(gw);
    /* optional additional routes: */
    ca_ipv4_t dst = { .v = { 10U, 0U, 0U, 0U } };
    ca_ipv4_t via = { .v = { 192U, 168U, 10U, 254U } };
    ca_route_add(dst, 8U, via);   /* 10.0.0.0/8 via 192.168.10.254 */
    ca_os_run(true);
}
```

18.2.5 CLI

```
route info                          -- mostra local, default gw e tabella rotte
route gw <ip>                       -- sets/replaces the default gateway
route add <ip/prefix> gw <ip>       -- adds a route
route del <n>                       -- removes the route at index n
route flush                         -- svuota la tabella (default gw invariato)
```

18.3 DHCP Service (`EN_DHCP_SERVICE`)

`ca_dhcp_service` manages the entire life cycle of the DHCP lease: acquisition (DORA), unicast renewal (RENEWING), broadcast rebinding (REBINDING) and reacquisition at expiration.

Enable: `EN_DHCP_SERVICE 1` (requires `EN_NET=1` and `EN_NET_DHCP=1`).

18.3.1 Configurable parameters

Macro	Default	Meaning
`DHCP_DISCOVER_RETRY_N`	3	DISCOVER attempts before backoff
`DHCP_OFFER_TIMEOUT_MS`	4000	WAIT for OFFER per attempt (ms)
`DHCP_REQUEST_TIMEOUT_MS`	4000	WAIT for ACK after REQUEST (ms)
`DHCP_BIND_TIMEOUT_MS`	60000	Maximum `ca_dhcp_service_bind()` timeout

18.3.2 State machine

```
INIT -> DISCOVERING
  send DISCOVER (broadcast, xid)
  timedwait OFFER_TIMEOUT (retry -> backoff -> xid++)
REQUESTING
  send REQUEST (broadcast, ciaddr=0, opt50+opt54)
  timedwait REQUEST_TIMEOUT (NAK/timeout -> DISCOVERING)
BOUND
  apply_lease(): calcola T1, T2, lease_deadline
  fire on_bound(ip, mask, gw, dns)
  timedwait ms_until(T1) (interrompibile da force-renew)
RENEWING (unicast al server, finche' T2 non scade)
  REQUEST unicast: ciaddr=ip
  ACK -> apply_lease() -> BOUND | timeout -> REBINDING
REBINDING (broadcast, finche' lease non scade)
  REQUEST broadcast: ciaddr=ip
  ACK -> BOUND | lease scaduta -> on_expire() -> DISCOVERING
```

18.3.3 Public API

```
/* Inizializza strutture e semafori. Chiamare in ca_os_configure(). */
void ca_dhcp_service_init(ca_mac_t             local_mac,
                          ca_dhcp_bound_cb_t   on_bound,
                          ca_dhcp_expire_cb_t  on_expire);

/* Creates the DHCP task (LEVEL1). Call in ca_os_main(). */
void ca_dhcp_service_start(void);

/* Unblocks the DHCP task -- port 68 ready.
 * Call from task_net AFTER ca_net_proto_bind_udp(68, ...). */
void ca_dhcp_service_net_ready(void);

/* RX handler: call from task_net for every UDP frame on port 68. */
void ca_dhcp_service_rx_handler(const ca_udp_frame_t *udp, u16 len);

/* Blocks until BOUND or timeout. Do NOT call from task_net (deadlock). */
bool ca_dhcp_service_bind(void);

/* Checks the BOUND state without blocking. */
static inline bool ca_dhcp_service_bound(void);

/* Forces an early renew. */
```

```
void ca_dhcp_service_renew(void);

/* Parameters of the current lease (valid only if bound()==true). */
ca_ipv4_t ca_dhcp_service_get_ip(void);
ca_ipv4_t ca_dhcp_service_get_mask(void);
ca_ipv4_t ca_dhcp_service_get_gw(void);
ca_ipv4_t ca_dhcp_service_get_dns(void);
```

18.3.4 Callbacks

```
typedef void (*ca_dhcp_bound_cb_t)(ca_ipv4_t ip, ca_ipv4_t mask,
                                   ca_ipv4_t gw,  ca_ipv4_t dns);
typedef void (*ca_dhcp_expire_cb_t)(void);
```

18.3.5 Adapter dhcp_udp_handler

```
static void dhcp_udp_handler(u16 buf_idx)
{
    ca_net_buf_t *b  = ca_net_buf_ptr(buf_idx);
    u8 *frame        = b->data + b->data_off;
    u16 l4_off       = b->meta.rx.l4_off;
    const ca_udp_frame_t *udp = (const ca_udp_frame_t *)(frame + l4_off);
    ca_dhcp_service_rx_handler(udp, (u16)(b->data_len - l4_off));
    ca_net_buf_free(buf_idx);
}
```

18.3.6 Critical Warnings

Deadlock: do not call `ca_dhcp_service_bind()` from task_net: the net task is blocked and OFFER/ACK frames are never processed.

`on_dhcp_bound` **must explicitly update:** - `ca_net_proto_set_local_ip(ip)` – for ICMP/ARP reply with correct IP - `ca_udp_server_set_local_ip(ip)` – for UDP replies with valid `src_ip` - `ca_arp_service_update_local_ip(ip)` – updates local_ip in the ARP service

18.4 NTP Service (EN_NTP_SERVICE)

NTP client (RFC 5905) with automatic polling and multi-server fallback.

18.4.1 API

Function	Description
`ca_ntp_service_init(server_ip, local_mac, on_sync, on_fail)`	Init; in `ca_os_configure()`
`ca_ntp_service_start()`	Create NTP task (CA_TH_LEVEL3)
`ca_ntp_service_bind()`	Register port 123 in UDP dispatcher
`ca_ntp_service_net_ready()`	Signal stack ready (post-DHCP)
`ca_ntp_service_rx_handler(udp, len)`	RX handler for port 123
`ca_ntp_service_synced()`	`true` if at least one sync has occurred
`ca_ntp_service_get_unix_time()`	Unix timestamp of last sync
`ca_ntp_service_set_server(server_ip)`	Update primary server
`ca_ntp_service_sync_now()`	Force immediate sync cycle

18.4.2 Parameters `os_config.h`

Parameter	Default	Meaning
`NTP_SERVER_N`	3	Configurable servers (multi-server fallback)
`NTP_RETRY_N`	3	Attempts per server before `on_fail`
`NTP_RESPONSE_TIMEOUT_MS`	5000	Timeout for single response
`NTP_POLL_MS`	64000	Normal polling interval (~1 min)

18.4.3 Typical pattern

```
/* ca_os_configure() */
ca_ntp_service_init(gw_ip, local_mac, on_ntp_sync, NULL);
/* ca_os_main() */
ca_ntp_service_start();
/* task_net, after a valid IP */
ca_ntp_service_bind();
ca_ntp_service_net_ready();
/* callback */
static void on_ntp_sync(u32 unix_time) {
    ca_rtc_from_net(unix_time);
}
```

`ca_rtc_from_net()` converts the unix time (UTC, since 1970) to the RTC format and writes it to the hardware RTC; the opposite is `ca_rtc_to_net()`, which reads the RTC and returns the unix time. Valid range 2000-2099, that of the RTC.

18.5 Syslog Service (`EN_SYSLOG_SERVICE`)

Syslog client (RFC 3164) with message queue. `ca_syslog_send` is non-blocking and safe from any task.

18.5.1 API

Function	Description
`ca_syslog_service_init(server_ip, local_mac, facility)`	Init; in `ca_os_configure()`
`ca_syslog_service_start()`	Create syslog task (CA_TH_LEVEL4)
`ca_syslog_service_net_ready()`	Signal stack ready
`ca_syslog_service_set_server(server_ip)`	Update server IP at runtime
`ca_syslog_send(severity, msg)`	Queue message – fire-and-forget
`ca_syslog_dropped()`	Messages discarded due to full queue

18.5.2 Parameters `os_config.h`

Parameter	Default	Meaning
`SYSLOG_QUEUE_SIZE`	8	Message queue depth
`SYSLOG_MSG_MAX`	128	Maximum message length (bytes)

18.5.3 Typical pattern

```
ca_syslog_service_init(server_ip, local_mac, SYSLOG_FAC_LOCAL0);
ca_syslog_service_start();
/* task_net, after a valid IP */
ca_syslog_service_net_ready();
/* qualsiasi task */
ca_syslog_send(SYSLOG_SEV_INFO,  "system started");
ca_syslog_send(SYSLOG_SEV_ERR,   "sensor timeout");
```

18.6 DNS Service (EN_DNS_SERVICE)

Blocking DNS client (RFC 1035) with concurrent slots.

18.6.1 API

Function	Description
`ca_dns_service_init(server_ip, local_mac)`	Init; in `ca_os_configure()`
`ca_dns_service_start()`	Create DNS task (CA_TH_LEVEL3)
`ca_dns_service_bind()`	Register port 53 in UDP dispatcher
`ca_dns_service_net_ready()`	Signal stack ready
`ca_dns_service_set_server(server_ip)`	Update DNS server
`ca_dns_resolve(hostname, &out, timeout_ms)`	Blocking resolution

18.6.2 os_config.h Parameters

Parameter	Default	Meaning
`DNS_PENDING_MAX`	4	Concurrent resolution slots
`DNS_RESPONSE_TIMEOUT_MS`	3000	Timeout for single response
`DNS_RETRY_N`	2	Attempts before failure

18.6.3 Typical Pattern

```
ca_ipv4_t ip;
if (ca_dns_resolve("pool.ntp.org", &ip, 10000U)) {
    ca_ntp_service_set_server(ip);
}
```

18.7 mDNS Service (EN_MDNS_SERVICE)

mDNS responder (RFC 6762) without a DNS server. The device responds to multicast queries on `224.0.0.251:5353` for `hostname.local`.

18.7.1 API

Function	Description
`ca_mdns_service_init(local_mac, hostname)`	Init; in `ca_os_configure()`
`ca_mdns_service_start()`	Create mDNS responder task (CA_TH_LEVEL3)

Function	Description
`ca_mdns_service_bind()`	Register port 5353 in the UDP dispatcher
`ca_mdns_service_net_ready(local_ip)`	Signal stack ready + send announce
`ca_mdns_service_rx_handler(udp, len)`	RX handler for port 5353

18.7.2 Embedded network scenario without router

```
[PC 192.168.10.1] -- gateway + DNS esterno
      |
   [Switch]
      +-- caos-board1  192.168.10.10
      +-- caos-board2  192.168.10.11

Risoluzione locale:
  caos-board1 -> "caos-board2.local" -> mDNS query multicast
  caos-board2 -> answers with its own IP
```

At the startup of each device: 1. Gratuitous ARP -> everyone learns MAC/IP 2. mDNS announce -> everyone learns hostname.local

18.8 TCP Service (`EN_TCP_SERVICE`)

`ca_tcp_service` implements a bare-metal TCP stack with a pool of concurrent connections, retransmission timers, and a complete state machine. It supports both **active open** (client) and **passive open** (server).

18.8.1 Configuration

```
#define EN_TCP_SERVICE   1
#define TCP_MAX_CONN          8U     /* connessioni dati concorrenti */
#define TCP_MAX_INFLIGHT      16U    /* segmenti TX in-volo per connessione */
#define TCP_RX_RING_DEPTH     8U     /* buffer RX per connessione */
#define TCP_MAX_LISTEN        2U     /* slot listen concorrenti */
#define TCP_RTO_INITIAL_MS    1000U
#define TCP_RTO_MAX_MS        16000U
#define TCP_CONNECT_TIMEOUT_MS  5000U
```

`TCP_RX_RING_DEPTH` is the **ceiling** for connection, not a reserve: the actual space is decided at runtime by `rx_quota()`, which divides the pool among active connections keeping 4 reserve buffers (the sum cannot exceed the pool by design — ex85). Default 8 since 10/05, measured on ex44/H563 Release in reception: 4 → 59.7 Mbps, **8 → 93.9**, 16 → 94.7. The ring contains indices, not buffers: increasing it costs 2 bytes per slot.

18.8.2 State machine

```
CLOSED -> (ca_tcp_connect) -> SYN_SENT
  SYN+ACK ricevuto -> ACK inviato -> ESTABLISHED
  ca_tcp_close() -> FIN_WAIT_1 -> FIN_WAIT_2 -> TIME_WAIT(4s) -> CLOSED

Passive open:
  ca_tcp_listen() -> attende SYN -> SYN_RECEIVED -> ESTABLISHED
  peer chiude (on_rx, callback):   ESTABLISHED -> LAST_ACK -> CLOSED
  peer chiude (ca_tcp_recv):       ESTABLISHED -> CLOSE_WAIT
      ca_tcp_recv() drena i segmenti arrivati prima del FIN, poi false (fine
    flusso)
      ca_tcp_close() del proprietario -> LAST_ACK -> CLOSED
      TCP_CLOSE_WAIT_MS (10 s) senza close -> il FIN lo manda lo stack
```

Half-close (from 10/02/2026). A reader connection no longer loses bytes that arrived with or before the peer's FIN: it remains in CLOSE_WAIT until the owner has read them and calls `ca_tcp_close()`. In CLOSE_WAIT, `ca_tcp_send()` is still allowed (RFC 793) and `ca_tcp_accept()` also returns a connection that is already half-closed, because its data is still to be read. Previously, the board would respond with FIN+ACK immediately and, upon the final ACK, would discard the queue ("DISCARDING n queued rx segment(s)").

18.8.3 Public API — client

```
/* Avvia il timer task TCP (CA_TH_LEVEL3, 100ms tick). */
void ca_tcp_service_start(void);

/* Active open. Ritorna handle 1-based oppure 0 su errore. */
u8 ca_tcp_connect(const ca_tcp_connect_cfg_t *cfg);

/* Invia dati su ESTABLISHED. Segmenta in chunk MSS. */
bool ca_tcp_send(u8 handle, const u8 *data, u16 len);

/* Ricezione bloccante (solo se on_rx==NULL).
 * Consegna UN segmento per chiamata: i byte oltre buf_len vengono SCARTATI. */
bool ca_tcp_recv(u8 handle, u8 *buf, u16 buf_len,
                 u16 *out_len, u32 timeout_ms);

/* Graceful close (FIN). */
void ca_tcp_close(u8 handle);

bool ca_tcp_is_connected(u8 handle);
```

18.8.4 Configuration structure — client

```
typedef struct {
    ca_ipv4_t         remote_ip;
    u16               remote_port;
    ca_ipv4_t         local_ip;
    ca_mac_t          local_mac;
    u32               arp_timeout_ms;      /* 0 -> TCP_ARP_TIMEOUT_MS */
    u32               connect_timeout_ms;  /* 0 -> TCP_CONNECT_TIMEOUT_MS */
    ca_tcp_rx_t       on_rx;               /* NULL = blocking ca_tcp_recv() */
    ca_tcp_conn_t     on_connected;        /* NULL = ca_tcp_connect() bloccante */
    ca_tcp_close_t    on_closed;
} ca_tcp_connect_cfg_t;
```

18.8.5 Public API — server

```
u8   ca_tcp_listen(const ca_tcp_listen_cfg_t *cfg);
void ca_tcp_unlisten(u8 listen_handle);
u8   ca_tcp_accept(u8 listen_handle, u32 timeout_ms);
```

18.8.6 Configuration structure — server

```
typedef struct {
    u16               local_port;
    ca_ipv4_t         local_ip;    /* l'IP della board: obbligatorio, vedi
    Avvertenze */
    ca_mac_t          local_mac;
    ca_tcp_conn_t     on_accept;   /* NULL = blocking ca_tcp_accept() */
    ca_tcp_rx_t       on_rx;       /* NULL = blocking ca_tcp_recv() */
    ca_tcp_close_t    on_closed;
```

```
} ca_tcp_listen_cfg_t;
```

18.8.7 CLI

```
tcp list          -- handle, stato, porte, IP remoto
tcp info <h>      -- dettaglio PCB: seq, ack, window, MSS, inflight, rx_ring
tcp slist         -- slot listen attivi
tcp buf           -- pool liberi/held/leak (diagnosi perdite buffer)
```

18.8.8 Warnings

Buffer pool: 4 conn x 4 inflight = 16 store buffers in the worst-case. Pool 64 buffers; leave at least 20 buffers free for RX and other services.

`ca_tcp_recv()` **from task_net**: deadlock. Use only from separate application tasks or the callback mode (`on_rx`).

`ca_tcp_recv()` **delivers one segment at a time and truncates**: if the segment is longer than `buf_len`, the rest is lost. Anyone reading in small pieces (for example, the 5-byte header of a TLS record) must read an entire segment into a `TCP_MSS_DEFAULT`-byte support buffer and consume it from there, as `https_service` does.

`local_ip` **of the listener**: must be set with the board's IP (`ca_net_proto_get_local_ip()` or the one provided by DHCP). With 0.0.0.0, the SYN is accepted but the SYN-ACK is not sent, and connections remain in `SYN_RECEIVED` (found on 10/23/2026 with `https_service`). For this reason, services open the listener after the DHCP bind.

RST on a server connection (fixed on 10/23/2026): the branch handling an RST was incrementing `conn_sem` without a ceiling, but that semaphore is only consumed by a blocking `ca_tcp_connect()`. On an accepted connection, no one was consuming it: on the second RST on the same reused slot, it would overflow, triggering a safeguard in the `net` task while holding the TCP mutex, and from there, every task restart would hang on the same mutex. Now the post has a ceiling. TLS clients often close with an RST: with HTTPS, it happened on the third request.

18.9 UDP Server (`EN_UDP_SERVER`)

The UDP Server service manages multi-client UDP connections, assigning each `src_ip:src_port` combination to a dedicated slot with worker task.

18.9.1 API

Function	Description
`ca_udp_server_start(&cfg)`	Init structures, spawn workers; in `ca_os_main()`
`ca_udp_server_bind()`	Register UDP port in dispatcher; from net task
`ca_udp_server_send(client_id, payload, len)`	Send reply to client
`ca_udp_server_disconnect(client_id)`	Force disconnection
`ca_udp_server_set_local_ip(ip)`	Update local IP (after DHCP bound)

18.9.2 Configuration Structure

```
typedef struct {
    u16                 port;
    u8                  max_clients;        /* <= UDPSRV_MAX_CLIENTS */
    u32                 client_timeout_ms;
```

```
    ca_mac_t              local_mac;
    ca_ipv4_t             local_ip;
    ca_udp_srv_rx_t    on_rx;             /* obbligatorio */
    ca_udp_srv_event_t on_connect;        /* NULL ok */
    ca_udp_srv_event_t on_disconnect;     /* NULL ok */
} ca_udp_server_cfg_t;
```

Parameter `os_config.h`: `UDPSRV_MAX_CLIENTS` (default 4).

18.10 UDP Client (EN_UDP_CLIENT)

Two modes: **async** (`on_rx != NULL`, called in the net task) and **blocking** (`on_rx == NULL`, `ca_udp_client_recv()` unblocks the task).

18.10.1 API

Function	Description
`ca_udp_client_connect(&cfg)`	Resolves server MAC, binds port; returns handle or 0
`ca_udp_client_send(handle, payload, len)`	Sends UDP datagram
`ca_udp_client_recv(handle, buf, len, &out_len, timeout)`	Blocking RX
`ca_udp_client_disconnect(handle)`	Closes connection
`ca_udp_client_refresh_mac(handle, arp_timeout_ms)`	Renews MAC via ARP

Parameter `os_config.h`: `UDPCLI_MAX_HANDLES` (default 4).

18.11 IP Defrag Service (EN_IP_DEFRAG)

Transparent IP fragment reassembly (RFC 791) for the upper stack.

18.11.1 Configuration parameters

Define	Default	Description
`IP_DEFRAG_MAX_SESSIONS`	4	Parallel reassembly sessions
`IP_DEFRAG_TIMEOUT_MS`	15000	ms before an incomplete session is discarded
`IP_DEFRAG_MAX_PAYLOAD`	1480	Maximum reassemblable payload (bytes, excluding IP header)

Heap allocated at init: `IP_DEFRAG_MAX_SESSIONS x IP_DEFRAG_MAX_PAYLOAD` bytes (default 4 x 1480 = 5920 B).

18.11.2 Operational flow

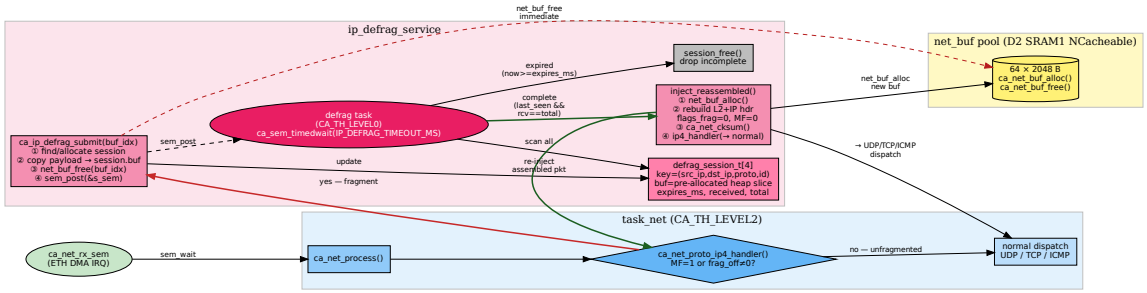


Figure 18.1: IP Defrag flow

18.11.3 Initialization

```
/* ca_os_configure() */
EVAL_IP_DEFRAG(
    if (!ca_ip_defrag_init(IP_DEFRAG_MAX_PAYLOAD))
        CA_SAFE_FAIL("ca_ip_defrag_init");
)
/* ca_os_main() */
EVAL_IP_DEFRAG(ca_ip_defrag_service_start();)
```

18.11.4 Public API

Function	Called by	Description
`ca_ip_defrag_init(max_payload)`	`ca_os_configure()`	allocates heap + initializes sessions
`ca_ip_defrag_service_start()`	`ca_os_main()`	creates task `defrag`
`ca_ip_defrag_submit(buf_idx)`	`ca_net_proto_ip4_handler()`	receives a fragment (internal use)

Usage example: Example 48.

18.12 TFTP Service (`EN_TFTP_SERVICE`)

TFTP (RFC 1350) over UDP. As a client it supports GET (download from server) and PUT (upload to server); as a server it receives **one file pushed by a client** (WRQ), one per call. Main use case: firmware update (the `workspace/app_2` bootloader receives its images this way). One transfer at a time.

18.12.1 Configuration parameters

Define	Default	Description
`EN_TFTP_SERVICE`	`1`	Enables the TFTP service

Define	Default	Description
TFTP_BLOCK_TIMEOUT_MS	5000	Timeout per block (ms)
TFTP_RETRY_N	3	Retries per block before failure
EN_CLI_TFTP	1	Enables tftp CLI commands

18.12.2 Public API

Function	Description
ca_tftp_service_init(mac)	Initializes service and registers CLI
ca_tftp_get(cfg, on_block, arg)	Downloads file from server (blocking)
ca_tftp_put(cfg, data, len)	Uploads buffer to server (blocking)
ca_tftp_receive(cfg, on_wrq, on_block, arg, wait_ms)	Waits for a WRQ and receives that file (blocking, server side)

18.12.3 on_block Callbacks

```
typedef bool (*ca_tftp_rx_block_t)(u16 block_num,
                                   const u8 *data, u16 len,
                                   void *arg);
```

Returning false aborts the transfer.

18.12.4 Server side: ca_tftp_receive

```
typedef u16 (*ca_tftp_wrq_t)(const char *filename, ca_ipv4_t client, void *arg);

ca_tftp_srv_cfg_t cfg = { .local_mac = mac };      /* port 0 = 69 */
bool ok = ca_tftp_receive(&cfg, on_wrq, on_block, &ctx, CA_WAIT_FOREVER);
```

- The port is listened to **only while a task waits** in ca_tftp_receive(); a WRQ arriving outside that window, or during a transfer, gets no answer and the client times out. One waiting task at a time. RRQ is not served.
- The listener runs in the net task and does not block: it records the WRQ (name, IP, port, MAC from the frame, so no ARP) and wakes the waiting task, which runs the transfer.
- on_wrq decides before block 0 is acknowledged: it returns 0 to accept or a TFTP_ERR_* code to refuse (the client gets an ERROR). It may take its time, for instance to erase the destination flash.
- Blocks reach on_block as with ca_tftp_get(): the same receive loop serves both.

From the PC the file is pushed with the tools/caos_tftp package (standard library, no sudo: port 69 is the board's):

```
python3 tools/caos_tftp put <board-ip> <file> [--name NAME]
```

18.12.5 CLI

```
tftp get <ip> <filename>    -- downloads a file from the TFTP server
tftp put <ip> <filename>    -- uploads a file to the TFTP server
tftp recv [sec]             -- receives ONE file pushed by a client (default 30
   s), data discarded
tftp info                   -- shows the service state
```

Usage example: Example 49.

18.13 Packet Tracer Service (`EN_PACKET_TRACER`)

Lightweight bare-metal tcpdump. Captures metadata of every RX/TX frame without copying the payload. A dedicated task formats and sends on configurable output channel (UART, USB CDC, ITM).

18.13.1 Architecture

```
net_proto.c:
  ARP/IP/UDP/TCP handler -> ca_pktrace_push_rx()    ring[8]
net.c:
  ca_net_send           -> ca_pktrace_push_tx() ---> pktrace_task
                                                       filter_pass()
                                                       s_out(msg)
```

Every entry in the ring is a 20-byte snapshot of `ca_net_buf_rx_t`: **zero-copy of the payload**.

18.13.2 Configuration

```
#define EN_PACKET_TRACER        1
#define PACKET_TRACER_RING_SIZE 8U
#define EN_CLI_TRACE            1
```

18.13.3 API

```
void ca_pktrace_start(void);
void ca_pktrace_set_sender(sender fn);
void ca_pktrace_enable(bool en);
void ca_pktrace_filter_src(ca_ipv4_t ip);
void ca_pktrace_filter_dst(ca_ipv4_t ip);
```

18.13.4 CLI

Command	Description
`trace on`	Enable frame capture
`trace off`	Disable capture
`trace filter src <ip>`	Only frames with src IP = `<ip>`
`trace filter dst <ip>`	Only frames with dst IP = `<ip>`
`trace filter clear`	Remove all filters

18.13.5 Output format

```
[RX] ARP  192.168.10.1 -> 192.168.10.158 len=60
[TX] ARP  192.168.10.158 -> 192.168.10.1 len=42
[RX] ICMP 192.168.10.1 -> 192.168.10.158 len=98
[RX] UDP  192.168.10.1:67 -> 255.255.255.255:68 len=342
[RX] TCP  192.168.10.1:48282 -> 192.168.10.158:80 len=74
```

The tracer uses its own `sender` – **not** `ca_stdout_printf_n` – to avoid conflicts with CLI capture and audit_log.

Resource	Value
RAM ring	8 x 20 B = 160 B

Resource	Value
Overhead per frame (push)	~12 instructions
Overhead when disabled	1 test on `s_enabled` -> zero

18.14 HTTP Service

`ca_http_service` implements a sequential bare-metal HTTP/1.1 server on `ca_tcp_service`. It supports GET, POST, HEAD, OPTIONS with a static route table defined by the application.

The same routes can also be served via **HTTPS** on port 443: the parser, the route table, and the response builder are the same; only the transport changes (TLS 1.3 instead of the TCP socket). See HTTPS and TLS 1.3.

File: `caos/net/src/http_service.c` / `caos/net/inc/http_service.h`

Dependencies: `EN_ETH 1`, `EN_NET 1`, `EN_ARP_SERVICE 1`, `EN_TCP_SERVICE 1`.

18.14.1 Configuration `os_config.h`

```
#define EN_HTTP_SERVICE     1    /* richiede EN_NET=1 + EN_TCP_SERVICE=1 */

/* Dimensioni buffer -- critiche per browser reali */
#ifndef HTTP_REQ_BUF_SIZE
#define HTTP_REQ_BUF_SIZE   2048U   /* default 512 insufficiente per Chrome */
#endif
#ifndef HTTP_RESP_BUF_SIZE
#define HTTP_RESP_BUF_SIZE  4096U   /* default 1024 -- aumentare per CLI verbose
    */
#endif
#ifndef HTTP_URI_MAX
#define HTTP_URI_MAX        128U
#endif
#ifndef HTTP_QUERY_MAX
#define HTTP_QUERY_MAX      128U
#endif
#ifndef HTTP_DEFAULT_PORT
#define HTTP_DEFAULT_PORT   80U
#endif
#ifndef HTTP_RECV_TIMEOUT_MS
#define HTTP_RECV_TIMEOUT_MS  1500U
#endif
```

> **Note**: the default values (512/1024) are sufficient only for `curl` with minimal headers. Modern browsers (Chrome, Firefox) send ~610 bytes of headers in POST requests. With `HTTP_REQ_BUF_SIZE=512` the body falls into the headers -> 400 Bad Request. **Always** set >= 2048 in `os_config.h`.

18.14.2 Architecture

```
ca_http_service_start(routes, count, port)
        |
        +-- task "http" (CA_TH_LEVEL6)
                |
                +-- ca_tcp_listen(port)
                +-- loop: accept -> ca_http_serve_connection(io TCP) -> close
```

The task handles one connection at a time (sequential). The connection is closed after sending the response.

Abstract transport (23/10/2026)

`ca_http_serve_connection()` reads the request, passes it to the route table, and writes the response through a **transport** (`ca_http_io_t`), without knowing what lies underneath. Task `http` passes it the TCP socket; task `https` a TLS session. Each service has its own buffers (`ca_http_conn_t`), so the two tasks never share them.

```
typedef struct {
    bool  (*send)(void *io, const u8 *data, u16 len);           /* true = tutto scritto */
    i32   (*recv)(void *io, u8 *buf, u16 len, u32 timeout_ms);  /* >0 byte, 0 timeout, <0 chiuso */
    void   *io;
} ca_http_io_t;

typedef struct {
    u8                rx[HTTP_REQ_BUF_SIZE];
    u8                tx[HTTP_RESP_BUF_SIZE];
    ca_http_response_t resp;
} ca_http_conn_t;

void ca_http_serve_connection(ca_http_conn_t *c, const ca_http_io_t *io);
```

The transport is not closed by `ca_http_serve_connection()`: closing it (TCP FIN, `close_notify` TLS) is up to the caller. `ca_http_conn_t` and the `io` descriptor must be in **static** memory, never on the task stack: in chunked mode, a handler can pass the response to another task (the `/cli` route in example 80).

18.14.3 Data structures

HTTP Methods (bitmask)

Macro	Value
`HTTP_METHOD_GET`	`0x01`
`HTTP_METHOD_POST`	`0x02`
`HTTP_METHOD_HEAD`	`0x04`
`HTTP_METHOD_OPTIONS`	`0x08`

`ca_http_request_t`

Field	Type	Description
`method`	`u8`	received method bitmask
`uri`	`char[]`	path, e.g., “/status”
`query`	`char[]`	query string after `?`, e.g., “state=on”
`body`	`const u8 *`	POST body (pointer in the rx buffer)
`body_len`	`u16`	body length

ca_http_response_t

```
typedef struct {
    u8  *buf;           /* buffer output (fornito dal service) */
    u16  size;          /* dimensione buffer */
    u16  len;           /* byte scritti finora */
    u16  cl_pos;        /* offset placeholder Content-Length */
    u16  hdr_end;       /* offset inizio body (dopo \r\n\r\n) */
    bool head_only;     /* true se metodo HEAD */
    bool chunked;       /* risposta a chunk (ca_http_resp_begin_chunked) */
    const ca_http_io_t *io;  /* trasporto su cui vanno i chunk */
    bool failed;        /* un invio    fallito: la risposta    abbandonata */
} ca_http_response_t;
```

Route table

```
typedef void (*ca_http_handler_fn)(const ca_http_request_t *req,
                                   ca_http_response_t      *resp);

typedef struct {
    const char        *uri;      /* es. "/status" -- confronto esatto */
    u8                 methods;  /* bitmask HTTP_METHOD_* accettati */
    ca_http_handler_fn handler;
} ca_http_route_t;
```

18.14.4 Public API

```
/* Avvia il task HTTP e registra CLI.
 * routes/count: tabella route -- il puntatore deve restare valido.
 * port: 0 -> usa HTTP_DEFAULT_PORT (80). */
void ca_http_service_start(const ca_http_route_t *routes, u8 count, u16 port);

/* Aggiorna local IP dopo DHCP bind. */
void ca_http_update_local_ip(ca_ipv4_t ip);

/* Costruisce header HTTP + Content-Type. Chiamare prima del body. */
void ca_http_resp_begin(ca_http_response_t *r, u16 status,
                        const char *content_type);

/* Aggiunge dati al body (no-op se head_only). */
void ca_http_resp_write(ca_http_response_t *r, const char *s, u16 len);
void ca_http_resp_puts(ca_http_response_t *r, const char *s);
void ca_http_resp_puti(ca_http_response_t *r, u32 v);

/* Risposte di errore pre-costruite. */
void ca_http_resp_not_found(ca_http_response_t *r);    /* 404 */
void ca_http_resp_bad_request(ca_http_response_t *r);  /* 400 */
```

18.14.5 DHCP Integration

```
static void on_dhcp_bound(ca_ipv4_t ip, ca_ipv4_t mask, ca_ipv4_t gw, ca_ipv4_t
    dns)
{
    (void)mask; (void)gw; (void)dns;
    ca_net_proto_set_local_ip(ip);
```

```
    ca_arp_service_update_local_ip(ip);
    ca_arp_send_garp();
    ca_http_update_local_ip(ip);   /* notifica l'IP al servizio HTTP */
}
```

18.14.6 Usage patterns

```
/* Route table (flash) */
static const ca_http_route_t routes[] = {
    { "/",       HTTP_METHOD_GET | HTTP_METHOD_HEAD, handler_root   },
    { "/status", HTTP_METHOD_GET | HTTP_METHOD_HEAD, handler_status },
    { "/cli",    HTTP_METHOD_POST,                   handler_cli    },
};

/* Handler di esempio */
static void handler_status(const ca_http_request_t *req,
                           ca_http_response_t      *resp)
{
    (void)req;
    ca_http_resp_begin(resp, 200U, "application/json");
    ca_http_resp_puts(resp, "{\"ip\":\"");
    /* ... */
    ca_http_resp_puts(resp, "\"}");
}

/* Avvio (da on_dhcp_bound o ca_os_main) */
ca_http_update_local_ip(ip);
ca_http_service_start(routes, 3U, 0U);   /* porta 80 */
```

18.14.7 CLI Capture Integration

With `EN_CLI_CAPTURE 1`, every CLI command is reachable via HTTP:

```
static void handler_cli(const ca_http_request_t *req,
                        ca_http_response_t *resp)
{
    char cmd[128];
    extract_cmd(req->body, req->body_len, cmd, sizeof(cmd));
    ca_http_resp_begin(resp, 200U, "text/plain");
    if (!ca_cli_exec_capture(cmd, g_buf, sizeof(g_buf), 1500U))
        ca_http_resp_puts(resp, "timeout");
    else
        ca_http_resp_puts(resp, g_buf);
}
```

`ca_cli_exec_capture` redirects stdout to a buffer for the duration of the command.

18.14.8 Implementation details

Content-Length in the response

`ca_http_resp_begin` writes a 10-space placeholder. Upon completion of the body construction, `ca_http_serve_connection` patches the actual value in place:

```
/* In ca_http_resp_begin() */
resp_append(r, "Content-Length: ", 16U);
r->cl_pos = r->len;
resp_append(r, "          ", 10U);  /* placeholder 10 spazi */
r->hdr_end = r->len + 4U;

/* In ca_http_serve_connection() dopo route_and_respond() */
u16 body_len = rp->len - rp->hdr_end;
ca_memcpy(rp->buf + rp->cl_pos, itoa_left(body_len, 10), 10U);
```

Multi-segment reception

If a browser splits the request into multiple TCP segments, `ca_http_serve_connection` accumulates the data with a loop before calling `parse_request`:

```
for (;;) {
    body_off = cerca_rnrn(rx_buf, rx_len);
    if (body_off > 0 || rx_len >= HTTP_REQ_BUF_SIZE) break;
    u16 more = io_read(io, rx_buf + rx_len, HTTP_REQ_BUF_SIZE - rx_len);
    if (more == 0) break;
    rx_len += more;
}
```

Waiting for the body: only if announced

After the headers, the server waits for the body **only if `Content-Length` is greater than zero**, and in that case, it reads until the body is complete. Until 10/23/2026 it always waited, even for a GET without a body: every request cost the entire `HTTP_RECV_TIMEOUT_MS`. Measured on example 45: a GET went from 1.5 s to 0.8 ms.

18.14.9 CLI: `http` command

With `EN_CLI_HTTP 1`:

```
http            -> stato, porta, numero route, IP locale
http info       -> uguale
http routes     -> URI + bitmask metodi per ogni route
```

Example:

```
c.a.O.S. > http info
HTTP  : running
port  : 80
routes: 3
IP    : 192.168.10.158

c.a.O.S. > http routes
  [0] /         methods=0x0D
  [1] /status   methods=0x0D
  [2] /cli      methods=0x02
```

18.14.10 Warnings

`HTTP_REQ_BUF_SIZE`: set >= 2048. Real browsers send ~610 bytes of headers in POST requests -> 400 Bad Request with buffers that are too small.

`HTTP_RESP_BUF_SIZE`: set >= 4096 for verbose CLI responses (output of `top`, `help`, `tcp list` can exceed 1 kB).

Simultaneous connections: the server is sequential. The TCP accept queue handles multiple browser connections without dropping them, but serves them one at a time.

OPTIONS: handled automatically by the router with the `Allow:` header generated from the method bitmask of the corresponding route.

URI not found: -> `404`. Method not allowed: -> `405` with `Allow:`.

See **Example 45** for a complete application (HTTP and HTTPS over Ethernet) and **Example 80** for the same application over USB-NCM.

18.15 MQTT Service

`mqtt_service` implements an MQTT 3.1.1 client over TCP for caOS. Fully callback-driven: no internal MQTT task, everything is processed by `on_tcp_rx()` within the net task context.

File: `caos/net/src/mqtt_service.c` / `caos/net/inc/mqtt_service.h`

Dependencies: `EN_ETH 1`, `EN_NET 1`, `EN_TCP_SERVICE 1`, `EN_ARP_SERVICE 1`.

18.15.1 Features

Property	Value
MQTT Version	3.1.1
Supported QoS	0 (fire-and-forget)
TX Packets	CONNECT, PUBLISH, SUBSCRIBE, PINGREQ, DISCONNECT
RX Packets	CONNACK, PUBLISH, SUBACK, PINGRESP
Dedicated Task	none – callback-driven in net task
Concurrent Connections	1 (single internal state)

18.15.2 Configuration `os_config.h`

Define	Default	Description
`EN_MQTT_SERVICE`	`1`	enables the service
`MQTT_RX_BUF_SIZE`	`256`	max RX payload in bytes
`MQTT_KEEPALIVE_SEC`	`60`	keepalive advertised to broker
`EN_CLI_MQTT`	`1`	`mqtt` command from CLI

18.15.3 Architecture

```
ca_mqtt_service_init()   <- sets up state, registers callbacks
ca_mqtt_service_net_ready() <- apre connessione TCP (asincrona)

TCP ESTABLISHED -> on_tcp_connected() -> MQTT CONNECT inviato
CONNACK OK      -> on_connected() dell'applicazione
PUBLISH RX      -> on_message(topic, tlen, payload, plen)
TCP chiuso      -> on_disconnected() dell'applicazione
```

RX Parser a stati

```
state 0 -> waiting for fixed header (1 byte: packet type)
state 1 -> decodes remaining length (1-4 bytes, 7-bit varlen encoding)
state 2 -> accumulating payload in rx_buf up to rem_len bytes
           -> process_packet()
```

Packets with payload > `MQTT_RX_BUF_SIZE` are silently discarded. The parser correctly handles TCP fragmentation.

18.15.4 Public API

```
void ca_mqtt_service_init(ca_ipv4_t broker_ip, u16 broker_port,
                          ca_ipv4_t local_ip,  ca_mac_t local_mac,
                          const char *client_id,
                          ca_mqtt_connected_cb_t    on_connected,
                          ca_mqtt_message_cb_t      on_message,
                          ca_mqtt_disconnected_cb_t on_disconnected);

bool ca_mqtt_service_net_ready(void);   /* starts TCP connect */
bool ca_mqtt_publish(const char *topic, const u8 *payload, u16 len);
bool ca_mqtt_subscribe(const char *topic);
void ca_mqtt_ping(void);
bool ca_mqtt_is_connected(void);
void ca_mqtt_disconnect(void);
```

Callbacks

```
typedef void (*ca_mqtt_connected_cb_t)(void);

typedef void (*ca_mqtt_message_cb_t)(const char *topic,  u16 topic_len,
                                     const char *payload, u16 payload_len);

typedef void (*ca_mqtt_disconnected_cb_t)(void);
```

> **Warning**: `topic` and `payload` point to `rx_buf` inside the service. They are valid **only for the duration of the callback**. Copy the data if necessary after the return.

18.15.5 Initialization

```
/* ca_os_configure() -- before the network is available */
ca_mqtt_service_init(broker_ip, 1883,
                     local_ip, local_mac,
                     "my-client",
                     on_mqtt_connected,
                     on_mqtt_message,
                     on_mqtt_disconnected);
```

After the DHCP bind, notify the service:

```
static void on_dhcp_bound(ca_ipv4_t ip, ...)
{
    ca_net_proto_set_local_ip(ip);
    ca_arp_service_update_local_ip(ip);
    ca_mqtt_service_net_ready();   /* starts the connection to the broker */
}
```

18.15.6 Pattern — periodic publish + reconnect

```
static u32 task_mqtt(void *arg)
{
    (void)a; (void)b;
    ca_sem_wait(&s_dhcp_sem);

    ca_ipv4_t broker_ip = s_gw;          /* DHCP gateway = broker */
    ca_mqtt_service_init(broker_ip, 1883, s_ip, s_mac,
                         "caos-h753",
                         on_connected, on_message, on_disconnected);
    ca_mqtt_service_net_ready();

    u32 last_pub = 0, last_ping = 0, last_reconnect = 0;

    while (1) {
        u32 now = ca_os_tick_ms();

        if (!ca_mqtt_is_connected()) {
            if ((now - last_reconnect) >= MQTT_RECONNECT_MS) {
                last_reconnect = now;
                ca_mqtt_service_init(broker_ip, 1883, s_ip, s_mac,
                                     "caos-h753",
                                     on_connected, on_message, on_disconnected);
                ca_mqtt_service_net_ready();
            }
            ca_os_delay_ms(500U);
            continue;
        }

        if ((now - last_pub) >= PUB_PERIOD_MS) {
            last_pub = now;
            char payload[64];
            ca_snprintf(payload, sizeof(payload), "{\"up\":%u}", now / 1000U);
            ca_mqtt_publish("caos/status", (const u8 *)payload,
                            (u16)ca_strlen(payload));
        }
        if ((now - last_ping) >= PING_PERIOD_MS) {
            last_ping = now;
            ca_mqtt_ping();
        }
        ca_os_delay_ms(500U);
    }
    return 0U;
}
```

Broker IP from DHCP gateway

The recommended pattern is to use the DHCP gateway as the broker address. Remove the hardcoded PC IP from the firmware.

```
static void on_dhcp_bound(ca_ipv4_t ip, ca_ipv4_t mask, ca_ipv4_t gw, ca_ipv4_t
    dns)
{
    (void)mask; (void)dns;
    s_ip = ip;
    s_gw = gw;                    /* broker = gateway */
    ca_net_proto_set_local_ip(ip);
    ca_arp_service_update_local_ip(ip);
    ca_arp_send_garp();
    ca_sem_post(&s_dhcp_sem);
}
```

18.15.7 CLI: command `mqtt`

With `EN_CLI_MQTT 1`:

```
mqtt                          -> connection state, broker IP
mqtt pub <topic> <payload> -> pubblica
mqtt sub <topic>           -> sottoscrive
mqtt ping                  -> PINGREQ manuale
mqtt disconnect            -> chiude connessione
```

18.15.8 Warnings

Single internal state: `ca_mqtt_service_init` clears the state including `tcp_handle`. Do not do it while a connection is active: use `ca_mqtt_disconnect()` first.

`ca_mqtt_ping()` **is application responsibility**: the service does not send automatic keepalives. The task must call `ca_mqtt_ping()` every `< MQTT_KEEPALIVE_SEC` seconds to prevent the broker from closing the session.

`ca_mqtt_subscribe()` **inside** `on_connected`: correct and recommended pattern.

TX packets as single buffer: each `ca_mqtt_publish` / `ca_mqtt_subscribe` builds the complete packet in a local buffer (128 bytes) and calls `ca_tcp_send` only once. Respects the `TCP_MAX_INFLIGHT =4` limit.

See **Example 41** for a complete application.

18.16 Modbus/TCP Service

`modbus_service` implements a Modbus/TCP client for caOS. Blocking API with timeout: the calling task blocks until the response arrives or the timeout expires.

File: `caos/net/src/modbus_service.c` / `caos/net/inc/modbus_service.h`

Dependencies: `EN_ETH 1`, `EN_NET 1`, `EN_TCP_SERVICE 1`.

18.16.1 `os_config.h` Configuration

Define	Default	Description
`EN_MODBUS_SERVICE`	`1`	enables the service
`MODBUS_TIMEOUT_MS`	`1000`	response timeout (ms)
`MODBUS_RX_BUF_SIZE`	`270`	RX buffer in bytes (>= max expected PDU)
`EN_CLI_MODBUS`	`1`	`modbus` command from CLI

18.16.2 Architecture

```
applicazione
    | ca_modbus_read_holding_regs() / write_single_reg() / ...
    v
modbus_service.c   -- MBAP framing, RX state machine, serializzazione tx_mtx
    | ca_tcp_send() / callbacks on_rx / on_connected / on_closed
    v
tcp_service.c      -- PCB, retransmit, finestra
    |
    v
rete fisica
```

MBAP Framing

Every request/response is encapsulated in an ADU (Application Data Unit):

```
+--------+--------+--------+--------+--------+--------+--------+--------+---
| TID(H) | TID(L) |  0x00  |  0x00  | Len(H) | Len(L) | UnitID |  FC    | data...
+--------+--------+--------+--------+--------+--------+--------+--------+---
 <---------------- MBAP header (6 byte) ----------------> <---- PDU ---->
```

`Len` = `1 (UnitID) + len(PDU)`. The RX parser uses two states: - **state 0**: accumulates 6 bytes of header, extracts `rx_pdu_len` - **state 1**: accumulates exactly `rx_pdu_len` bytes in the buffer, sets `rx_sem`

Supported function codes

FC	Name	Description
`0x01`	Read Coils	reads N coils (max 2000)
`0x03`	Read Holding Registers	reads N 16-bit registers (max 125)
`0x06`	Write Single Register	writes a register
`0x10`	Write Multiple Registers	writes N registers (max 123)

18.16.3 Initialization

```c
/* ca_os_configure() */
ca_modbus_service_init(server_ip, server_port, local_ip, mac, unit_id);
```

Parameter	Type	Description
`server_ip`	`ca_ipv4_t`	Modbus server (PLC/RTU gateway) IP
`server_port`	`u16`	TCP port, normally 502
`local_ip`	`ca_ipv4_t`	Local IP (0.0.0.0 before DHCP)
`local_mac`	`ca_mac_t`	Local MAC
`unit_id`	`u8`	Modbus Unit ID (slave address)

After DHCP bind:

```c
static void on_dhcp_bound(ca_ipv4_t ip, ...)
{
    ca_net_proto_set_local_ip(ip);
    ca_modbus_service_init(server_ip, 502U, ip, mac, unit_id);
    ca_modbus_service_net_ready();   /* opens a TCP connection to the server */
}
```

18.16.4 Public API

```c
/* Starts a TCP connection to the server (asynchronous via callback) */
bool ca_modbus_service_net_ready(void);

/* Connection state */
bool ca_modbus_is_connected(void);

/* Closes the connection */
void ca_modbus_disconnect(void);
```

```

/* FC01 -- Read Coils */
bool ca_modbus_read_coils(u16 addr, u16 count, u8 *out);

/* FC03 -- Read Holding Registers */
bool ca_modbus_read_holding_regs(u16 addr, u16 count, u16 *out);

/* FC06 -- Write Single Register */
bool ca_modbus_write_single_reg(u16 addr, u16 value);

/* FC16 -- Write Multiple Registers */
bool ca_modbus_write_multiple_regs(u16 addr, const u16 *values, u16 count);
```

All blocking functions are serialized by `tx_mtx`.

18.16.5 Usage patterns

```
static u32 task_modbus(void *arg)
{
    (void)a; (void)b;
    ca_sem_wait(&s_dhcp_sem);

    ca_ipv4_t server = s_gw;
    ca_modbus_service_init(server, 502U, s_ip, s_mac, 1U);
    ca_modbus_service_net_ready();

    u32 last_poll = 0U;
    u32 last_reconnect = ca_os_tick_ms();

    while (1) {
        u32 now = ca_os_tick_ms();

        if (!ca_modbus_is_connected()) {
            if ((now - last_reconnect) >= 5000U) {
                last_reconnect = now;
                ca_modbus_service_init(server, 502U, s_ip, s_mac, 1U);
                ca_modbus_service_net_ready();
            }
            ca_os_delay_ms(500U);
            continue;
        }

        if ((now - last_poll) >= 3000U) {
            last_poll = now;
            u16 regs[4];
            if (ca_modbus_read_holding_regs(0U, 4U, regs))
                ca_stdout_printf_n("reg[0]=%u", (u32)regs[0]);
            ca_modbus_write_single_reg(4U, 1234U);
        }
        ca_os_delay_ms(200U);
    }
    return 0U;
}
```

18.16.6 CLI: command `modbus`

With `EN_CLI_MODBUS 1`:

```
modbus                      -> connection state, server IP, unit id
modbus read <addr> <count> -> FC03 Read Holding Registers
```

```
modbus write <addr> <val>  -> FC06 Write Single Register
modbus coils <addr> <cnt>  -> FC01 Read Coils
```

18.16.7 Warnings

`ca_modbus_service_init` **closes the previous connection** (if active) before resetting the state, avoiding zombie PCBs in the TCP service.

Serialized concurrent accesses: `tx_mtx` ensures that the automatic task and CLI do not overlap. The response waiting occurs with the mutex acquired but with the task in SLEEP state.

Stale callback management: each TCP callback verifies that `handle == s.tcp_handle` before acting. Prevents events from a already closed connection from corrupting the state of the new one.

```
static void on_tcp_closed(u8 handle)
{
    if (handle != s.tcp_handle) return;   /* ignora TIME_WAIT stantio */
    s.tcp_up = false;
    ca_sem_post(&s.rx_sem);              /* unblocks the waiting transact() */
}
```

See **Example 43** for a complete application.

18.17 iperf Service

`iperf_service` measures the TCP throughput between the board and a network peer compatible with iperf2 (default port 5001).

Supports two modes: - **client**: the board opens the connection and transmits bulk data - **server**: the board accepts the connection and measures received data

File: `caos/net/src/iperf_service.c` / `caos/net/inc/iperf_service.h`

Dependencies: `EN_ETH 1`, `EN_NET 1`, `EN_ARP_SERVICE 1`, `EN_TCP_SERVICE 1`.

18.17.1 `os_config.h` Configuration

Define	Default	Description
`EN_IPERF_SERVICE`	`1`	enables the service
`EN_CLI_IPERF`	`1`	`iperf` command from CLI
`NET_BUF_COUNT`	—	required; 64 recommended (128 KB NET_SRAM1)

18.17.2 Architettura

```
CLI / applicazione
    | ca_sem_post(&s.sem)   <- non bloccante: iperf_task esegue in background
    v
iperf_task (CA_TH_LEVEL3)
    | ca_iperf_client()  o  ca_iperf_server()
    v
iperf_service.c  -- TX/RX loop, misura bytes/ms/bps
    | ca_tcp_connect / listen / accept / send / recv / close
    v
tcp_service.c    -- PCB, retransmit, window
```

Client Mode — `ca_iperf_client()`

```
ca_tcp_connect(remote_ip, port)       <- SYN -> attende ESTABLISHED
loop finche' (now - t0) < duration_ms:
    ca_tcp_wait_writable(h, 200 ms)   <- yields the CPU if the window is full
    ca_tcp_send(h, buf, 1460 B)       <- segmento a MSS
ca_tcp_close(h)                       <- FIN + drain
-> RESULT: bytes_total / duration_ms / bps
```

The fixed chunk of 1460 bytes corresponds to the standard Ethernet MSS (MTU 1500 - 20 IP - 20 TCP).

Server Mode — `ca_iperf_server()`

```
ca_tcp_listen(port)                   <- apre slot passivo
ca_tcp_accept(lh, idle_timeout_ms)    <- BLOCKED: waits for SYN from the PC
ca_tcp_unlisten(lh)                   <- libera slot ascolto
loop:
    ca_tcp_recv(h, buf, 1460, &got, idle_ms)
    received += got
    [break if got == 0 or timeout]   <- FIN/RST from the PC
ca_tcp_close(h)
-> RESULT: bytes_total / duration_ms / bps
```

18.17.3 Initialization

```c
/* ca_os_main() -- after ca_tcp_service_start() */
{
    ca_ipv4_t zero = { .addr = 0U };
    ca_iperf_service_start(zero, s_mac);
}

/* update the IP at DHCP bind */
static void on_dhcp_bound(ca_ipv4_t ip, ...)
{
    ca_net_proto_set_local_ip(ip);
    ca_arp_service_update_local_ip(ip);
    ca_iperf_update_local_ip(ip);
}
```

18.17.4 Public API

```c
/* Starts the service (registers the CLI handler). Call from ca_os_main(). */
void ca_iperf_service_start(ca_ipv4_t local_ip, ca_mac_t local_mac);

/* Updates the local IP after DHCP bind. */
void ca_iperf_update_local_ip(ca_ipv4_t ip);

/* Client mode: transmits bulk to remote_ip:port for duration_ms ms.
 * Blocking. Writes the result in *out. Returns false if connect fails. */
bool ca_iperf_client(ca_ipv4_t remote_ip, u16 port,
                     ca_ipv4_t local_ip, ca_mac_t local_mac,
                     u32 duration_ms, ca_iperf_result_t *out);

/* Server mode: accepts one connection on port.
```

```
 * Waits up to idle_timeout_ms for a client.
 * Blocking. Returns false if no client connects. */
bool ca_iperf_server(u16 port, ca_ipv4_t local_ip, ca_mac_t local_mac,
                     u32 idle_timeout_ms, ca_iperf_result_t *out);
```

Result Structure

```
typedef struct {
    u32 bytes_total;   /* byte TX (client) o RX (server) */
    u32 duration_ms;   /* actual duration */
    u32 bps;           /* bit/s = bytes_total / (duration_ms/1000) * 8 */
} ca_iperf_result_t;
```

18.17.5 CLI: command `iperf`

With `EN_CLI_IPERF 1`:

```
iperf help
iperf client <ip> [port=5001] [dur_s=10]
iperf server [port=5001] [idle_s=30]
tcp buf         -- pool free/held/leak (buffer leak diagnosis)
```

The CLI does not block: post the semaphore and return immediately. The result appears on the output when `iperf_task` terminates.

Client session (board -> PC)

```
# On the PC:
nc -l 5001 | pv > /dev/null

# On the board:
> iperf client 192.168.10.1 5001 10
  iperf RESULT: 39906910 bytes  10036 ms  31925528 bps
```

Server session (PC -> board)

```
# On the PC:
dd if=/dev/zero bs=64k | nc <board_ip> 5001

# On the board:
> iperf server 5001 30
  iperf RESULT: 38675400 bytes  10000 ms  30940320 bps
```

18.17.6 Typical Performance (NUCLEO-H753ZI, 100BASE-TX)

Mode	Average Throughput	Peak Throughput
Client (board TX -> PC RX)	~25 Mbps	~32 Mbps
Server (PC TX -> board RX)	~25 Mbps	~39 Mbps

Main limitations: - **CPU copying**: two `memcpy` per each TCP segment between DMA buffers and application buffers - **TCP window**: limited by the number of free buffers in the pool (64 x 2 KB) - **scheduler latency**: SysTick ~104 us; `__WFI()` in idle task indispensable

See **Example 44** for a complete application.

Chapter 19

caOS USB Driver: DWC2 Core

19.1 Low-level USB Driver

19.1.1 Overview

The caOS USB driver is a bare-metal **device controller** Full-Speed that exposes a generic endpoint API — USB descriptors, enumeration and classes are **not** in the driver: they live in the application (example 50) or in the `caos/usb/` services stack (CDC, MSC, NCM, ...).

The STM32 families handled by the project **do not all use the same USB controller**: under the same public API, two completely different hardware IPs coexist.

- **DWC2 OTG** (Synopsys DesignWare USB 2.0 OTG) — F4xx, F7xx, H7xx, U5xx. Architecture based on **FIFO** (slave or DMA), full OTG core.
- **USB_DRD_FS** — H5xx (H563). ST proprietary IP based on **PMA** (Packet Memory Area), **not** DWC2: no shared FIFO, dedicated HW buffers for endpoints. See the "The Two USB IPs" paragraph.

The rest of this chapter (per-EP code, public API, source layout) is common to both IPs; hardware differences are isolated in `hw/<family>/drivers/`.

The driver handles only:

1. **Init / clock / GPIO** of the OTG core for the family
2. **EP0 control**: SETUP reception, status stage, `SET_ADDRESS`
3. **Data endpoints** (bulk/interrupt): open/close, read/write, stall
4. **DWC2 ISR**: bus reset, RXFLVL, IEPINT/OEPINT → callback to the application

The driver class separation is the same as in `net` and `crypto`: the low-level HW stays in `caos/drivers/` + `hw/<family>/`, class services in `caos/usb/`.

Tested HW (all in **slave mode**, CPU copies the FIFO): - **NUCLEO-H753ZI** — OTG_FS CN14, 6 EP @ 0.579 MB/s - **NUCLEO-U575ZI-Q** — internal OTG_FS, 5 EP @ 0.466 MB/s - **STM32F4VE (black board)** — OTG_FS PA11/PA12, 3 EP @ 0.548 MB/s

The H7 driver in **DMA mode** (previous) is kept as `usb_hs_h7xx.c` for the future **OTG_HS** (480 Mbit/s), where bandwidth justifies the DMA. At Full-Speed the slave has no bottlenecks (CPU at 480 MHz copies the FIFO faster than the 12 Mbit/s bus) and on H7 it actually **outperforms** DMA (0.579 vs 0.491 MB/s) thanks to deeper per-EP queues; it also removes the fragility of the EP0 control path in DMA (shared RX FIFO for control+bulk).

19.1.2 The Two USB IPs

caOS supports two physically different USB device controllers behind the same endpoint API. The choice depends on the family, it is not configurable. | Aspect | **DWC2 OTG** | **USB_DRD_FS** | |———|———————|———————-| | Families | F4xx, F7xx, H7xx, U5xx | H5xx (H563) | | Origin | Synopsys DesignWare USB 2.0 OTG | ST proprietary (FS host/device) | | Data buffers | **FIFO** (RX shared among OUT EPs) | **PMA** — dedicated HW buffers per EP | | Mode | slave (CPU) or internal DMA | always PMA, CPU copies from/to PMA | | Buffer descriptors | EP registers + FIFO | BTABLE (2 words/EP) at start of PMA | | EP registers | `DIEPCTLn`/`DOEPCTLn` separate IN/OUT | `CHEPnR` single (bit toggle STAT/DTOG) | | USB clock | PLLQ 48 MHz (F4) / HSI48 (U5) | HSI48 (`USBSEL=11`) + CRS | | IRQ | `OTG_FS_IRQn` | `USB_DRD_FS_IRQn` | | HW reference | Family RM, OTG chapter | rm0481-h5, ch. 55 |

DWC2 is a full OTG core: OUT endpoints share a single RX FIFO and the driver dispatches packets from ISR `RXFLVL` to per-EP software rings (see the following two sections). It supports slave mode and internal DMA.

USB_DRD_FS (H5xx) is instead based on **PMA**: a linear 2 KB memory (512 32-bit words) where each endpoint has dedicated **hardware** TX and RX buffers, described by the Buffer Description Table (BTABLE) at the beginning of the PMA. There is no shared FIFO or software ring: the driver directly copies payload between PMA and application RAM. EP registers `CHEPnR` use *toggle* bit semantics (`STAT_TX`/`STAT_RX`/`DTOG` XOR on write) and *rc_w0* (`VTRX`/`VTTX`/`SETUP`), different from DWC2's `DIEPCTL`/`DOEPCTL`. Under the public API (`ca_usb_ep_open/read/write`, SETUP, enumeration), behavior is identical to other drivers.

The following two sections ("Two transfer modes" and "Per-EP code") describe the **DWC2** implementation. The H5 USB_DRD_FS IP has no FIFO or ring: the related PMA management is documented inline in `hw/h5xx/drivers/usb_fs_h5xx.c`.

19.1.3 Two transfer modes (DWC2)

The DWC2 core has two modes to move data between FIFO and RAM. caOS supports both; all **active** drivers use slave mode, DMA is reserved for future OTG_HS:

Mode	Families	Mechanism
Slave (CPU)	H7xx (FS), U5xx, F4xx	CPU reads/writes FIFO word-by-word in ISR (RXFLVL)
Internal DMA	H7xx (HS, future)	Core copies from/to RAM autonomously; driver `usb_hs_h7xx.c` parked

Both modes share the same public API; only the implementation in `hw/<family>/drivers/usb_<family>.c` changes.

Slave mode — the RX FIFO is shared among all OUT EPs: the ISR `RXFLVL` extracts the packet header (`GRXSTSP`), reads the payload word-by-word and dispatches it to the **per-EP ring** of the destination endpoint. **Flow-control lossless**: every OUT EP is armed with `PKTCNT=1`, so after each packet it automatically returns to NAK. The ISR `OEPINT` (XFRC) **re-arms the EP only if the RX ring has space**; if it is full it does NOT re-arm → the host gets NAKed and retransmits (no packet lost). `ca_usb_ep_read` re-arms the EP when the task frees a slot. This eliminates the drop-on-full from the previous version (which drained and discarded the already ACKed packet) — essential under load and in composite devices with saturated EPs.

The driver `usb_f4xx.c` is structurally twin to `usb_u5xx.c`; the differences from the U5 concern the hardware capabilities of the core:

Aspect	F4xx OTG_FS	U5xx OTG_FS
DMAEN	hardwired 0 (RM0090 §34.3.2)	0 (slave mode chosen)
Total EPs	4 (EP0..3), `USB_EP_MAX=4`	6 (EP0..5), `USB_EP_MAX=6`
Bulk data EPs	3 (EP1..3), `USB_NQEP=3`	5 (EP1..5), `USB_NQEP=5`
Total FIFO	320 word (1280 B)	320 word (1280 B)
VBUS sensing	`GCCFG.NOVBUSSENS` (GUID < 0x2000)	`GOTGCTL.BVALOEN/BVALOVAL`
USB Clock	PLLQ = 48 MHz	HSI48
IRQ	`OTG_FS_IRQn` (single)	`OTG_FS_IRQn` (single)

`GCCFG.NOVBUSSENS` is required on F4xx because the DWC2 core GUID is `< 0x2000` (F2/F4 generation): the `GOTGCTL.BVALOEN` field does not exist. On F4 boards, OTG_FS does not sense VBUS → suspend/resume (USBSUSP/WKUPINT) is the only reliable proxy for disconnect detection, as already on U5.

19.1.4 Per-EP Code (DWC2)

In slave mode, each data endpoint has **independent RX and TX queues** (ring SPSC, head-/tail per-EP, flat pool): no contention between endpoints, each can be served by a dedicated task. This allows composite devices (multiple parallel bulk EPs) as in example 50.

- **H7xx**: `USB_NQEP=6` data EPs, ring depth **uniform 128 slots per EP**. Pools (`s_qrx_pool` /`s_qtx_pool`) are located in `.usb_dma` (USB_SRAM D2, 128 KB) and are allocated **only for active cores** (`EN_USB_HS`/`EN_USB_FS` in `os_config.h`): with only FS, the depth doubles compared to dual-core (all RAM allocated to the active core). `USB_PIDX(n)` maps `usb_num` → pool instance.
- **U5xx**: `USB_NQEP=5` data EPs, variable ring depth (EP1/2=64, EP3=16, EP4/5=8 slots), flat pool.
- **F4xx**: `USB_NQEP=3` data EPs (EP1..3), uniform ring depth 64 slots per EP, flat pool of 192 slots.

Ringbuffer data copy uses `ca_memcpy_u32`/`ca_memcpy_u16` (fallback byte) depending on length alignment; bulk MPS packets (64 B) always go on the 32-bit path. FIFO writes/reads remain word-per-word loops (it is a fixed-address MMIO register, not a memory-to-memory copy).

> **Note on uniform depth**: the multi-EP loopback (ex50) automatically limits packets in flight to the firmware queue depth; with heterogeneous depths, shorter EPs would saturate. With lossless flow-control (above), drops no longer occur, but uniform depth keeps the pipelining balanced across EPs. **EP0 IN multi-packet** (slave U5): if the response to a `GET_DESCRIPTOR` exceeds `MPS` (e.g. config descriptor of 88 B with `MPS=64`), the driver sends the subsequent chunks on the **TXFE** interrupt (FIFO empty, `DIEPEMPMSK` bit 0). Without this chunking, the enumeration of a device with 5 EPs would fail.

19.1.5 Public API

All functions take `usb_num` as the first parameter (on H7 there are two OTG cores: `0`=HS, `1`=FS).

Initialization

```
bool ca_usb_init(u32 usb_num, const ca_usb_cb_t *cb);
```

Initialize clock, GPIO and DWC2 core, register callbacks and wait for the first bus reset from the host. Returns `false` if clock/GPIO fails.

```
typedef struct {
    ca_usb_reset_cb_t     on_reset;      /* bus reset */
    ca_usb_mounted_cb_t   on_mounted;    /* enumerated (with speed) */
    ca_usb_unmounted_cb_t on_unmounted;  /* scollegato */
    ca_usb_setup_cb_t     on_setup;      /* SETUP 8B su EP0 */
    ca_usb_rx_cb_t        on_rx;         /* OUT data ready on EPn */
    ca_usb_tx_done_cb_t   on_tx_done;    /* TX IN completata */
} ca_usb_cb_t;
```

The callbacks are called **by ISR**: they must only signal a task (`ca_notify_send_isr`), not block. Unused pointers go to `NULL`.

Endpoint

```
bool ca_usb_ep_open (u32 usb_num, u8 ep_addr, ca_usb_ep_type_t type, u16 mps);
bool ca_usb_ep_close(u32 usb_num, u8 ep_addr);
u16  ca_usb_ep_read (u32 usb_num, u8 ep_addr, u8 *buf, u16 max_len);
bool ca_usb_ep_write(u32 usb_num, u8 ep_addr, const u8 *buf, u16 len);
```

- `ep_addr` follows the USB convention: bit 7 = direction (`0x81` = EP1 IN, `0x01` = EP1 OUT). EP0 IN = `0x80`.
- `ca_usb_ep_read` returns the bytes extracted from the EP RX queue (0 if empty); does not block.
- `ca_usb_ep_write` **copies** the buffer to the TX queue before returning → the caller can reuse `buf` immediately. Returns `false` only when TX queue is full (backpressure): retry after `ca_os_yield()`.

```
typedef enum {
    CA_USB_EP_TYPE_CTRL = 0,
    CA_USB_EP_TYPE_ISOC = 1,
    CA_USB_EP_TYPE_BULK = 2,
    CA_USB_EP_TYPE_INTR = 3,
} ca_usb_ep_type_t;
```

Stall (Patch 1)

```
void ca_usb_ep_stall      (u32 usb_num, u8 ep_addr);
bool ca_usb_ep_clear_stall(u32 usb_num, u8 ep_addr);
bool ca_usb_ep_is_stalled (u32 usb_num, u8 ep_addr);
```

`ep_close` / `clear_stall` / `is_stalled` (Patch 1) follow, on **DWC2** cores, the known sequence A.1 (SNAK→EPDIS→flush for IN; SGONAK→GONAKEFF→EPDIS for OUT). Implemented on **H7, U5 and F4**. On **H5** (USB_DRD_FS) stall uses instead the `STAT_TX`/`STAT_RX = STALL` fields of register `CHEPnR`. F1/F7 remain stub (porting step 13).

Bus control

```
void ca_usb_set_address(u32 usb_num, u8 addr);   /* applies DCFG.DAD */
void ca_usb_connect    (u32 usb_num);            /* pull-up D+ */
void ca_usb_disconnect (u32 usb_num);            /* soft disconnect */
```

`ca_usb_set_address`: apply **immediately** after SETUP `SET_ADDRESS`, before the ZLP status (quirk DWC2 / ST HAL pattern). Deferred apply in `on_tx_done` is race-prone because the ZLP XFRC fires in a few µs.

19.1.6 Source layout

File	Content
`caos/drivers/inc/usb.h`	public types + API wrapper
`caos/drivers/src/usb.c`	extern HW + generic wrapper
`caos/drivers/inc/ca_glue_hw.h`	extern `ca_usb_fs_dev_*` / `ca_usb_hs_dev_*` declaration
`hw/h7xx/drivers/usb_h7xx.c`	DWC2 **slave** FIFO — H753/H747 OTG_FS (HW tested)
`hw/h7xx/drivers/usb_hs_h7xx.c`	DWC2 **DMA** — parked for future OTG_HS (not in build)
`hw/u5xx/drivers/usb_u5xx.c`	DWC2 **slave** FIFO — U575 (HW tested)
`hw/f4xx/drivers/usb_f4xx.c`	DWC2 **slave** FIFO — F407/F429/F446, no DMA (HW tested F407)
`hw/h5xx/drivers/usb_fs_h5xx.c`	**USB_DRD_FS** (PMA) — H563, non-DWC2 IP (HW tested)
`hw/f1xx,f7xx/drivers/usb_*.c`	stub (porting step 13)

`EN_USB` in `os_config.h` enables the driver.

19.1.7 Notes per family

- **CMSIS bit naming**: the same bit 7 of `GINTSTS` is called `USB_OTG_GINTSTS_BOUTNAKEFF` on H7 and `USB_OTG_GINTSTS_GONAKEFF` on U5/F4.
- **H7 OTG**: dual-core — `usb_num=0` → OTG_HS (ULPI, MPS up to 512); `usb_num=1` → OTG_FS (MPS 64). `EN_USB_HS`/`EN_USB_FS` in `os_config.h` select active cores and pool allocation (currently only FS, slave mode). The DMA EP0 control path on this core was fragile (shared RX FIFO for control+bulk, subsequent control transfers random, not hot-recoverable): rewriting in slave mode eliminated it. ex52 CDC: enum + echo + open/close repeated terminal **without HW reset** + replug → re-enumeration, all OK on H753.
- **U5 OTG_FS**: `USB_EP_MAX=6` → at most 5 data EPs + EP0.
- **F4 OTG_FS**: `USB_EP_MAX=4` → at most 3 data EPs + EP0. On F407/F429/F446 the bit `DMAEN` is hardwired to 0 (RM0090 §34.3.2): the driver always operates in slave mode regardless of configuration.
- **STUPCNT EP0 (F4)**: during bring-up on F407 it was found that `DOEPCTL0` requires `STUPCNT=3` (not 1) to correctly receive multiple consecutive SETUPs during enumeration. Value 1 caused `-110` on the host at second bus reset.
- **H5 USB_DRD_FS** (H563): non-DWC2 IP, 2 KB PMA with per-EP dedicated buffers (no FIFO, no ring). Clock 48 MHz from HSI48 with `USBSEL=11` and correction via CRS. Single endpoint register `CHEPnR` with *toggle* bits (`STAT_TX`/`STAT_RX`/`DTOG`) and *rc_w0* (`VTRX`/`VTTX`/`SETUP`): write 1 to preserve, 0 to clear. PMA management documented inline in `hw/h5xx/drivers/usb_fs_h5xx.c` (ref. rm0481-h5 ch. 55).

See also: example 50 (vendor-class bulk loopback) for full API usage in a multi-EP composite device.

Chapter 20

caOS USB Stack: Management and Classes

20.1 USB device stack: management and classes

20.1.1 Overview

Above the low-level DWC2 driver (S02_11_00), caOS builds a **composable device stack**: a *management* layer (`ca_usb_dev_mng`) that assembles configuration descriptors and dispatches SETUP requests, and a set of interchangeable **class services** (CDC, MSC, NCM, DFU, custom) at runtime. It lives in `caos/usb/`, in symmetry with `caos/net/` and `caos/crypto/`.

```
applicazione
        ca_usb_dev_mng_init / add / start

ca_usb_dev_mng (caos/usb/src/usb_mgmt.c)
        ca_usb_class_ops_t  (fan-out per servizio attivo)
             ca_usb_cdc    (CDC-ACM, seriale virtuale)
             ca_usb_msc    (Mass Storage, BOT/SCSI)
             ca_usb_ncm    (USB Ethernet, CDC-NCM)
             ca_usb_dfu    (DFU runtime, EP0 only)
             ca_usb_custom (vendor loopback, test)
        ca_usb_ep_open/read/write/...

driver DWC2 (caos/drivers/ + hw/<family>)
             hw/h7xx/drivers/usb_h7xx.c   slave FIFO (H753/H747 OTG_FS)
             hw/u5xx/drivers/usb_u5xx.c   slave FIFO (U575)
             hw/f4xx/drivers/usb_f4xx.c   slave FIFO (F407/F429/F446     OTG_FS
   no DMA)
```

20.1.2 Two-level model (D1)

- **compile-time**: `EN_USB_CDC`, `EN_USB_MSC`, `EN_USB_NCM`, `EN_USB_DFU`, `EN_USB_CUSTOM` in `os_config.h` → which classes are *available* in the binary.
- **runtime**: `ca_usb_dev_mng_add(&cfg)` / `remove(type)` → which classes are *active* at a given moment. Allows modal firmware.

20.1.3 API management (D18 — 5 functions)

```
bool ca_usb_dev_mng_init(ca_usb_speed_t speed, u32 usb_num);
    /* registers the driver cb */
bool ca_usb_dev_mng_add   (const ca_usb_device_type_t *cfg); /* adds a class */
```

```
bool ca_usb_dev_mng_remove(ca_usb_type_t type);
bool ca_usb_dev_mng_start (void);           /* build descriptors + connect */
bool ca_usb_dev_mng_stop  (void);
```

Typical usage (three lines are enough):

```
ca_usb_dev_mng_init(CA_USB_SPEED_FS, USB_NUM);
ca_usb_dev_mng_add(&s_ncm.header);   /* s_ncm static, persistent lifetime (D20) */
ca_usb_dev_mng_start();
```

20.1.4 Class framework — `ca_usb_class_ops_t`

Every service defines a singleton `const ca_usb_class_ops_t` with its handlers; the management invokes them via ops, without `switch` on the type:

Callback	Role
`build_iface_desc`	appends interface (+IAD) + endpoint to config descriptor
`pid_contribution`	composite PID (`PID_BASE_x + EP_DATA`)
`on_reset`	bus reset
`on_mounted` / `on_unmounted`	enumeration / disconnection (on U5 and H7: `USBSUSP` as proxy for disconnect, VBUS not wired on NUCLEO)
`on_configure`	opens EPs (`SET_CONFIGURATION`)
`on_set_interface`	alt setting (NCM: alt0=idle / alt1=active)
`on_setup`	class-level SETUP request
`on_rx` / `on_tx_done`	fan-out EP I/O
`get_string`	string descriptor index > 3 (e.g. `iMACAddress` NCM)

The cfg of each class starts with the `ca_usb_device_type_t {type, ops}` tag (D19), allowing direct style `sockaddr` cast. The cfg must be `static` (ideally `static const`): the management keeps the pointer without copying.

20.1.5 Class services

CDC-ACM — virtual serial port (example_52)

Enumerates with IAD + comm-interface + data-interface. The PC mounts the device as `/dev/ttyACMx`. Handles `SET/GET_LINE_CODING`, `SET_CONTROL_LINE_STATE` (DTR). `SET_LINE_CODING` is a control-OUT: the data-stage (7 B) arrives via `on_rx(0)`; only after the service emits the ZLP IN.

EP	Type	Define
`USB_CDC_EP_DATA` (default 3)	bulk IN+OUT	serial data
`USB_CDC_EP_NOTIF` (default 4)	interrupt IN	line status notifications

Application API: `ca_usb_cdc_write`, `ca_usb_cdc_connected`, `ca_usb_cdc_get_line`.

MSC — block device (example_53)

Implements BOT (Bulk-Only Transport) + subset SCSI (INQUIRY, READ/WRITE_CAPACITY, READ10/WRITE10, REQUEST_SENSE, MODE_SENSE6). The host manages the filesystem: caOS exposes only an array of 512 B blocks. The back-end is abstracted via callback `read(lba, buf, count)` / `write(lba, buf, count)` in `ca_usb_msc_cfg_t` — changing only these the service works on ramdisk, FLASH or SD card.

EP	Type	Define
`USB_MSC_EP_DATA` (default 2)	bulk IN+OUT	CBW/data/CSW

NCM — USB Ethernet (example_54 loopback, example_55 stack IP)

CDC-NCM 1.0: the PC sees the device as a network card (`usb0` / `enxXX`). The Ethernet frames travel encapsulated in NTB (1 datagram per block). Handles `SET_INTERFACE` alt0=idle / alt1=active (link up/down) and `iMACAddress` via `get_string`. Integrates with `caos/net` through the `ca_ncm_iface_get` (push model) adapter: `on_rx` → `ca_net_rx_push` → net task semaphore.

EP	Type	Define
`USB_NCM_EP_DATA` (default 1)	bulk IN+OUT	NTB Ethernet
`USB_NCM_EP_NOTIF` (default 5)	interrupt IN	link/speed notify

DFU runtime — Device Firmware Upgrade (example_57)

Exposes a **DFU interface** (class `0xFE` / subclass `0x01` / protocol `0x01`) **without data endpoint**: all dialog occurs on EP0. It does not contribute to the endpoint collision bitmap and can coexist with any other class without consuming data EPs.

Handled requests (DFU 1.1): `DFU_DETACH`, `DFU_GETSTATUS`, `DFU_GETSTATE`, `DFU_CLRSTATUS`, `DFU_ABORT` . `DFU_DNLOAD`/`DFU_UPLOAD` are not handled at runtime (require DFU mode) → STALL on EP0 is correct.

The application receives `on_detach()` and decides how to enter the bootloader (flag RAM no-init + reset to system or application bootloader).

EP	Type	Notes
—	EP0 only	`PID = USB_PID_BASE_DFU = 0x0040`

custom — vendor interface (example_51)

Reference implementation of `ca_usb_class_ops_t`: the minimal model to copy when writing a new class. Exposes a vendor interface (`0xFF`) with EP1 bulk IN+OUT and loopback.

20.1.6 Endpoint allocation

Endpoints are assigned by **queue depth / expected bandwidth**, without collision in the composite (validated at compile-time by `_Static_assert`, D16):

Class	EP DATA	EP NOTIF	Bandwidth / queue
NCM	1 (bulk)	5 (intr)	high / deep

Class	EP DATA	EP NOTIF	Bandwidth / queue
MSC	2 (bulk)	—	high / deep
CDC	3 (bulk)	4 (intr)	low / short
DFU	— (EP0 only)	—	—
custom	1 (bulk)	—	test / variable

Limits per family

Family	Max bulk data EPs	Notes
H753/H747	6 (EP1..6)	OTG_FS (slave FIFO)
U575	5 (EP1..5)	OTG_FS (slave FIFO, RM0456 §72.2.3)
F407/F429/F446	3 (EP1..3)	OTG_FS (slave FIFO, `USB_EP_MAX=4`, no DMA)

The composite CDC+MSC+NCM uses **5 EPs** (EP1..5): fits exactly in the U575, has margin on the H7, but **does not fit** in the F407 (3 data EPs). On F407, example 56 uses the bi-modal **NCM+MSC** mode (3 EPs, `EX56_HAS_CDC=0` is automatically set for `STM32F4`).

20.1.7 Composite PID (D14)

The device PID is the **sum** of the `pid_contribution` of all active classes:

Class	`USB_PID_BASE_x`	typical contribution
CDC	`0x0010`	`0x0010 + EP_DATA (3) = 0x0013`
MSC	`0x0020`	`0x0020 + EP_DATA (2) = 0x0022`
NCM	`0x0030`	`0x0030 + EP_DATA (1) = 0x0031`
DFU	`0x0040`	`0x0040` (fixed, no data EPs)
custom	`0x0000`	`0x0000 + EP_DATA (1) = 0x0001`

Examples: single CDC class → `PID = 0x0013`; composite CDC+MSC+NCM → `PID = 0x0013 + 0x0022 + 0x0031 = 0x0066`.

20.1.8 Device identity (os_config.h)

VID, release, and device strings are project configuration parameters, `usb` group, as the `USB_PID_BASE_x` in the table above — not constants in the code: | Symbol | Default | Descriptor | |—|—|—| | `USB_VID` | `0xCA01U` | `idVendor` | | `USB_BCD_DEVICE` | `0x0100U` | `bcdDevice` | | `USB_STR_MANUFACTURER` | `"caOS"` | manufacturer string | | `USB_STR_PRODUCT` | `"caOS USB Device"` | product string | | `USB_STR_SERIAL` | `"0001"` | serial number | | `USB_PID_BASE_x` | see above | contribution to composite PID |

`0xCA01` is the **development** VID, unregistered: caOS is software, and whoever builds a product sets their own VID and PID — `caos_config.py --set USB_VID=0xNNNNU <progetto>`, without touching the sources.

20.1.9 Composite devices

Multiple classes can coexist on the same device: just register more than one before `start`. The management builds **a single config descriptor** that concatenates the interfaces/IAD/endpoints of all active classes.

```
ca_usb_dev_mng_init(CA_USB_SPEED_FS, USB_NUM);
ca_usb_dev_mng_add(&s_cdc.header);   /* /dev/ttyACMx              */
ca_usb_dev_mng_add(&s_msc.header);   /* removable disk          */
ca_usb_dev_mng_add(&s_ncm.header);   /* usb0 (+ ca_net_init)      */
ca_usb_dev_mng_start();              /* single composite descriptor */
```

The device presents `bDeviceClass = 0xEF` (Miscellaneous) + `bDeviceProtocol = 1` (IAD), and the host loads a driver for each function.

2-class combinations: these are obtained by turning off a `EN_USB_X` — the EP allocation remains valid because it is a subset:

Combination	EPs used	Enters F407?
CDC+MSC+NCM	5	no (max 3 on F407)
MSC+NCM	3	yes
CDC+NCM	4	no
CDC+MSC	3	yes
single class	1–3	yes

Multi-round EP0 data-stage (descriptors > 127 B)

The config descriptor of a 3-class composite is ~183 B. The DWC2 register `DIEPTSIZ0.XFRSIZ` is **7-bit** (max 127 B per *single* EP0 transfer), so the descriptor must be sent in multiple **rounds** by reprogramming `DIEPTSIZ0`/`DIEPDMA` at each `XFRC` on EP0.

> **Critical rule**: every non-final round must be an **exact multiple of MPS** (64 B → 1 full packet). A *short* packet in the middle of the descriptor would close the control transfer from the host's perspective (→ timeout). Only the final round may end with a short packet.

Implementation: `ep0_tx_start_round()` in slave drivers `hw/u5xx`, `hw/h7xx` and `hw/f4xx`, with a 512 B EP0 staging buffer. Single classes (descriptor 127 B) use a single round, unchanged behavior.

20.1.10 Source file

```
caos/usb/inc/usb_mgmt.h    caos/usb/src/usb_mgmt.c     # management
caos/usb/inc/usb_cdc.h     caos/usb/src/usb_cdc.c      # CDC-ACM
caos/usb/inc/usb_msc.h     caos/usb/src/usb_msc.c      # MSC
caos/usb/inc/usb_ncm.h     caos/usb/src/usb_ncm.c      # NCM
caos/usb/inc/usb_dfu.h     caos/usb/src/usb_dfu.c      # DFU runtime
caos/usb/inc/usb_custom.h  caos/usb/src/usb_custom.c   # custom (vendor test)

hw/h7xx/drivers/usb_h7xx.c     # DWC2 slave FIFO (H753/H747 OTG_FS)
hw/h7xx/drivers/usb_hs_h7xx.c  # DWC2 DMA, parked for a future OTG_HS (not in
    the build)
hw/u5xx/drivers/usb_u5xx.c     # DWC2 slave FIFO (U575)
hw/f4xx/drivers/usb_f4xx.c     # DWC2 slave FIFO (F407/F429/F446, OTG_FS no DMA)
```

20.1.11 Reference Examples

Example	Class	Tested Boards	HW Result
50	vendor bulk (direct driver, no mgmt)	H753, U575, F407	0.579 / 0.466 / 0.548 MB/s
51	custom (mgmt)	H753, U575, F407	10000/10000 loopback EP1
52	CDC-ACM	H753, U575, F407	echo 100/100 ttyACM[1]
53	MSC ramdisk	H753, U575, F407[2]	FAT12 mount/write/persist
54	NCM loopback L2	H753, U575, F407	arping 5/5 swap SADA
55	NCM + net stack (ping)	H753, U575, F407	ping 5/5 ~0.8 ms
56	composite CDC+MSC+NCM	U575, H753	ping+echo+FAT simultaneous
	composite NCM+MSC (F407)	F407[2]	ping+FAT (no CDC, 3 EP)
57	DFU runtime	H753, U575, F407	dfu-util -l OK + on_detach

[1] H753 ex52 CDC validated HW (slave driver): repeated echo + open/close of the terminal without HW reset + replug → re-enumerates. This was the case where the old DMA driver was not up to the task.
[2] F407 ramdisk 64 KB (`BLK_COUNT=128`), minimal FAT12; U575/H7 = 128 KB.

Chapter 21

FatFS on caOS: SDIO Driver

21.1 FatFS on caOS: SDIO driver

21.1.1 Overview

The caOS storage subsystem exposes a **FAT12/16/32 filesystem** (third-party library **FatFS**, `external/fatfs`) over a **bare-metal SDIO driver** for SDHC/SDXC SD cards. As with USB, net, and crypto, there is a clear separation between generic library/glue (`caos/`) and family-specific hardware implementation (`hw/<family>/drivers/`).

The subsystem is organized into three layers:

```
FatFS (external/fatfs, source/ff.c)
        configurazione: source/ffconf.h     redirect a caos_ffconf.h

fatfs_diskio.c (caos/drivers/src/fatfs_diskio.c)
        disk_initialize/status/read/write/ioctl     ca_sdio_*

ca_sdio (caos/drivers/inc/sdio.h + src/sdio.c)
        wrapper pubblico     extern ca_sdio_hw_* per famiglia

hw/<family>/drivers/sdio_<family>.c
```

Tested HW: **STM32F4VE (F407)** via generic DMA, **Nucleo-H563** and **Nucleo-H753** via SDMMC v2 with IDMA; microSD via 4-bit SDIO. See the complete application example (CLI `fat`) in the chapter "FatFS on SD card: SDIO driver with DMA" (example 58).

21.1.2 Layer 1 — FatFS and the ffconf.h redirect

FatFS resides as a third-party source in `external/fatfs/` (submodule). The only local modification to the submodule is precisely `source/ffconf.h`: the configuration (`FF_FS_*`, `FF_USE_*`, `FF_LFN_BUF`, ...) is normally written directly there, but in caOS that file **does not contain the configuration**: it only does

```
#include "../../caos_fatfs/caos_ffconf.h"
```

pointing to `external/caos_fatfs/caos_ffconf.h`, where the actual caOS-specific configuration resides.

Rationale — library/consumer ODR mismatch. `libfatfs.a` (build of `ff.c`) and the application consuming it (e.g., `example_58`) must see **exactly the same** `ffconf.h`, because macros like `FF_FS_EXFAT`, `FF_USE_LFN`, `FF_MAX_SS` change `sizeof(FATFS)`, `sizeof(FIL)`, and internal buffer layouts. If the library were compiled with one configuration and the application firmware included another (One Definition Rule violated at the binary struct level, not just symbols), the symptom is subtle: `f_mount()` returns `FR_OK` but subsequent operations see `"non montato"` or inconsistent data,

because the two sides read/write different offsets of the same structure (`FATFS::win[]` overflowing into subsequent fields like `s_mounted`, for example). The redirect to a single file (`caos_ffconf.h`, included in the same way by `fatfs.cmake` and by those using FatFS) eliminates the dual source of truth.

Library build (`external/caos_fatfs/fatfs.cmake`):

```
add_library(fatfs STATIC
    ${FATFS_DIR}/source/ff.c
    ${FATFS_DIR}/source/ffunicode.c
    ${CAOS_FATFS_DIR}/caos_ffsystem.c
)
target_include_directories(fatfs PUBLIC
    ${FATFS_DIR}/source
    ${CAOS_FATFS_DIR}
)
```

`caos_ffsystem.c` provides the FatFS-to-OS hooks required by `ff.c` when `FF_FS_REENTRANT`/dynamic allocation are enabled (mutex, malloc) adapted to caOS primitives.

21.1.3 Layer 2 — glue diskio (`fatfs_diskio.c`)

FatFS talks to the physical layer via the standard `diskio.h` interface (five functions, single drive `pdrv=0` = the SD card via `ca_sdio`):

```
DSTATUS disk_initialize(BYTE pdrv);
DSTATUS disk_status    (BYTE pdrv);
DRESULT disk_read      (BYTE pdrv, BYTE *buff, LBA_t sector, UINT count);
DRESULT disk_write     (BYTE pdrv, const BYTE *buff, LBA_t sector, UINT count);
DRESULT disk_ioctl     (BYTE pdrv, BYTE cmd, void *buff);
```

Mapping to the `ca_sdio` driver:

diskio function	Implementation
`disk_initialize`	`ca_sdio_init()`; if already initialized (`s_status==0`) no-op
`disk_status`	returns cache status (`STA_NOINIT` or 0)
`disk_read`	`ca_sdio_read_block()` loop for `count` sectors
`disk_write`	single `ca_sdio_write_blocks()` call (multi-block if `count>1`)
`disk_ioctl(CTRL_SYNC)`	`RES_OK` (no write cache to flush)
`disk_ioctl(GET_SECTOR_COUNT)`	`ca_sdio_get_block_count()`
`disk_ioctl(GET_SECTOR_SIZE)`	fixed at 512
`disk_ioctl(GET_BLOCK_SIZE)`	fixed at 1 (no erase-block hint)

`disk_write` **always** delegates to `ca_sdio_write_blocks()`, even for a single sector: it is the HW driver that decides whether to issue `CMD24` (single) or `CMD25` multi-block (see Layer 3).

An auxiliary function `disk_deinitialize(BYTE pdrv)` (declared `extern` by the caller, not part of the standard diskio interface) forces `STA_NOINIT`: used by the example after `f_fdisk()` to invalidate the cache state before the next `f_mkfs()`.

`get_fattime()` (required by FatFS for timestamps) reads the RTC via `ca_rtc_get()`; returns `0` (FatFS epoch date) if the RTC is not available.

21.1.4 Layer 3 — `ca_sdio` driver

Public API (`caos/drivers/inc/sdio.h`)

```
bool ca_sdio_init(void);
bool ca_sdio_read_block (u32 block, u8 *buf);
bool ca_sdio_write_block(u32 block, const u8 *buf);
bool ca_sdio_write_blocks(u32 block, const u8 *buf, u32 count);
u32  ca_sdio_get_block_count(void);
```

`buf` must be 4-byte aligned (HW drivers internally copy to an aligned bounce buffer when needed, but the public API requires it for efficiency). Addressing is always in **512-byte blocks** (SDHC).

Timeouts. No call waits forever: every wait on data, D0 busy and card state (CMD13) has a deadline in milliseconds, taken from the SD specification and defined per block in `sdio.h`:

Constant	Value	Reference
`CA_SDIO_READ_TIMEOUT_MS`	100 ms	maximum read access time
`CA_SDIO_WRITE_TIMEOUT_MS`	500 ms	write busy: 250 ms SDHC, 500 ms SDXC (the larger is used)

A write of `n` blocks waits at most `n  500 ms`. When the deadline passes the function returns `false`; the card is never abandoned before the time the specification grants it.

The wrapper in `caos/drivers/src/sdio.c` delegates to the `extern` functions `ca_sdio_hw_*` (one implementation per family, declared in `ca_glue_hw.h` according to the standard caOS driver pattern) and optionally registers a CLI node (`sdio info`, `sdio read <block>`) if `EN_CLI_SDIO` is enabled.

Enable defines in `os_config.h`:

```
#define EN_SDIO 1   /* driver SDIO/SDMMC */
#define EN_FAT  1   /* FatFS library + diskio glue */
```

The SDIO driver on F4xx also requires `EN_DMA=1` (the READ/WRITE data path passes through DMA2 Stream3); generic DMA initialization (`ca_dma_init()`) must be performed **before** SDIO init, which reserves the channel for the allocator.

F4xx HW Implementation (`hw/f4xx/drivers/sdio_f4xx.c`)

Pins (AF12):

Signal	Pin	Pull
D0..D3	PC8, PC9, PC10, PC11	pull-up
CLK	PC12	no pull
CMD	PD2	pull-up

Clock: 400kHz identification is mandatory per SD specification (`CLKDIV=118` → 48MHz/(118+2)), 24MHz 4-bit transfer after card selection (`CLKDIV=0` → 48MHz/2).

Data path — DMA2 Stream3 Channel4 (fixed SDIO mapping on F4xx), because pure polling at 24MHz results in `RXOVERR`:

- **READ**: *direct mode*, DMA as flow controller (`NDTR=128` word); the SDIO does not generate DMA requests below the remaining 8 words (threshold `RXFIFOHF`), so the last handful of words is drained via polling after `DBCKEND`.

- **WRITE single-block** (CMD24): *FIFO mode*, threshold=full, PFCTRL (the **SDIO** is the flow controller, not the DMA — RM0090 §31.3.2 point 4.d), burst INCR4 on peripheral and memory.
- **WRITE multi-block** (CMD25): in chunks of SDIO_BOUNCE_BLOCKS=8 blocks (4KB), with ACMD23 (pre-erase hint) before each chunk and CMD12 (STOP_TRANSMISSION) at the end — always, even on error, to bring the card out of the data state.

All transfers pass through a 32-byte aligned bounce buffer (s_bounce), because the FatFS buffer may not be 4-byte aligned while the DMA requires word alignment.

Only **SDHC/SDXC**: ca_sdio_hw_init() verifies CCS=1 in the OCR response of ACMD41 (bit 30 of RESP1) and explicitly fails if absent (init_fail("no-sdhc")).

Robustness points (portable to other families)

- **Time-based timeouts, not loop counts**: the waits were while (... && t--) loops with a fixed t, which lasted a few milliseconds and a different time on every MCU and clock. A card that took longer to program a block was abandoned in the middle of a write, and stayed in an abnormal state for minutes, even after an MCU reset (the card stays powered). Measured on H753 with inject (example 58): CK removed for 500 ms during a write gave WRITE FAIL and a wedged card; with the specification's deadlines the same write completes without errors, about 1 s longer, and the card stays healthy.

- PBURST **is not an "enable burst" flag**: it is a 2-bit field, where PBURST_0=bit0=01=INCR4 and PBURST_1=bit1=10=INCR8. INCR8 × 4B/word = 32B exceeds the DMA FIFO (16B), forbidden by RM0090 — the stream remains inert without obvious errors. Always use PBURST_0.

- **EN synchronization before reading NDTR**: after CR &= ~DMA_SxCR_EN one must wait for EN to read 0 before reading NDTR (RM0090 §10.3.14, the in-flight transfer must be completed) — otherwise the drain restarts from a wrong index and the sector results shifted by one word.

- **Clear** ICR **cross-clock-domain**: clearing STA flags crosses the SDIO_CK domain; at 400kHz these are microseconds, and an optimized binary (-O2) may re-read STA before the clear has arrived. Fix: wait_flags_clear() explicitly waits for zero after every write ICR. Bug reproducible only in **release** builds, not in debug.

H5xx / H7xx HW Implementation (SDMMC v2 with IDMA)

The H families use the **SDMMC version 2** controller, which integrates a dedicated **internal DMA (IDMA)**: a generic DMA channel is not needed (as on F4xx), the data path is entirely within the peripheral. The driver model is common to H5xx and H7xx; they differ only in two details:

	H5xx (H563)	H7xx (H753)
RCC Clock	AHB4ENR	AHB3ENR
IDMA base register	IDMABASER	IDMABASE0

Single-buffer IDMA sequence (aligned with the ST HAL order HAL_SD_ReadBlocks_DMA to avoid arming races):

1. DLEN + DCTRL (block size 512, direction; **without** DTEN: on SD, it is CMDTRANS that starts the transfer, not DTEN);

2. preset of only the CMDTRANS bit in CMD;
3. arm the IDMA: IDMABASER/IDMABASE0 = buffer, then IDMACTRL = IDMAEN;
4. command transmission (CPSMEN) — the DPSM starts with the IDMA already armed.

Completion on DATAEND. The **multi-block** (CMD25) is a single native IDMA transfer (no loop per block); CMD12 (STOP) closes the open-ended transmission after the transfer. The write then waits for the card to exit the programming state via BUSYD0 (hardware poll, no CMD13 per iteration) + a final CMD13 confirmation.

32-byte alignment quirk (fundamental). The IDMA transfers in **8-beat bursts (32 bytes)** and requires the buffer address to be 32-byte aligned; the caOS allocator only guarantees 8 bytes and FatFS passes buffers as such. With a buffer not aligned to 32, **the last half-word of the block is not committed to RAM** (last 2 bytes lost, but with valid CRC — the data arrives from the card, it is the IDMA→RAM write that loses it). The driver resolves this with a **static bounce buffer** aligned(32) **of 8 blocks (4KB)**: if the caller's buffer is 32-byte aligned, it uses direct IDMA, otherwise it goes through the bounce buffer (with memcpy for reading, copy-then-IDMA for writing; misaligned multi-block uses CMD25 in 8-block chunks).

D-cache (H7xx). caOS runs the H7 with **D-cache disabled** (the zero-copy ETH/USB stack uses non-cacheable MPU regions, so the cache has never been turned on). Consequently, IDMA buffers **do not** require cache maintenance. Warning: calling SCB_CleanDCache_by_Addr with the D-cache off generates a bus fault (imprecise write to SCB->DCCMVAC); if the D-cache were to be enabled in the future, maintenance would need to be reintroduced and conditioned on SCB->CCR & SCB_CCR_DC_Msk (cache actually active), not on __DCACHE_PRESENT. On H5xx (Cortex-M33), the D-cache is not present: no problem.

MPU heap guard (ARMv8-M, H5xx). On the H563 linker, the .heap section must be 32-byte aligned (ALIGN(32)): the heap-start MPU guard aligns _heap_start downwards to 32 bytes and, with an 8-byte heap, the RO region would encroach on the last bytes of .bss (driver state variables) → DACCVIOL. Latent on all ARMv8-M. The H753 linker (Cortex-M7) already has .heap ALIGN(32).

21.1.5 Status by family

Family	Status
F4xx (F407 tested STM32F4VE @24MHz, F446)	full driver with generic DMA (DMA2 **Stream3**; **Stream6 on the F446**, where Stream3 gets no requests): read DMA, write single CMD24 (FIFO+PFCTRL), write multi CMD25 (DMA in 8-block chunks)
H5xx (H563, tested Nucleo-H563 @25MHz)	SDMMC v2 **IDMA** + bounce aligned-32; read 1.6 MB/s, write single-block ~220 KB/s (card physical limit), mkfs ~0.5s
H7xx (H753, tested Nucleo-H753 @24MHz)	same as H5xx (IDMA+bounce); AHB3ENR/IDMABASE0; D-cache disabled → no cache maintenance; read 1.6 MB/s, write ~235 KB/s
U5xx (U575, tested Nucleo-U575 @24MHz)	like H5xx: SDMMC v2 **IDMA** + aligned-32 bounce, multi-block CMD25 in chunks; kernel clock from ICLK (CCIPR2.SDMMCSEL)

Family	Status
F7xx (F746/F767)	same IP as F4 (SDMMC1 v1): DMA2 **Stream3** channel 4 with the FIFO tail drained by hand; multi-block write as a single-block loop. No cache maintenance: the D-cache is off (`clock_f7xx.c` enables the I-cache only). No bench round recorded
F1xx	stub (no SDIO controller on the supported family)

21.1.6 File structure

File	Content
`caos/drivers/inc/sdio.h`	Public API
`caos/drivers/src/sdio.c`	public wrapper + extern HW + CLI `sdio`
`caos/drivers/inc/ca_glue_hw.h`	extern declaration `ca_sdio_hw_*`
`hw/f4xx/drivers/sdio_f4xx.c`	F4xx HW implementation, generic DMA (HW tested)
`hw/h5xx/drivers/sdio_h5xx.c`	SDMMC v2 IDMA + bounce aligned-32 (H563 HW tested)
`hw/h7xx/drivers/sdio_h7xx.c`	same as H5xx; `AHB3ENR`/`IDMABASE0`, D-cache off (H753 HW tested)
`hw/u5xx/drivers/sdio_u5xx.c`	like H5xx, IDMA + bounce (HW tested U575)
`hw/f7xx/drivers/sdio_f7xx.c`	SDMMC1 v1 with DMA2 Stream3, like F4xx
`hw/f1xx/drivers/sdio_f1xx.c`	stub
`caos/drivers/src/fatfs_diskio.c`	FatFS diskio glue → `ca_sdio`
`external/fatfs/`	FatFS library submodule (only local modification: redirect in `source/ffconf.h`)
`external/caos_fatfs/caos_ffconf.h`	caOS FatFS configuration (redirect from `source/ffconf.h`)
`external/caos_fatfs/caos_ffsystem.c`	FatFS→OS hooks (mutex/malloc)
`external/caos_fatfs/fatfs.cmake`	build `libfatfs.a`

See example chapter 58 (“FatFS on SD card: SDIO driver with DMA”) for complete practical usage: CLI `fat` (fdisk/mount/ls/blob/...), FAT context on heap, test output on real hardware.

Chapter 22

Centralized Power Manager

22.1 Centralized power manager

22.1.1 Overview

The `pwr` driver handles the **power management** of caOS: low-power modes (SLEEP/STOP/-STANDBY), CPU frequency scaling, and wakeup sources (RTC, external pin, LPTIM). It is a **centralized** power manager: the state of each peripheral is not tracked by an internal driver flag, but read **on-the-fly** directly from the hardware registers — RCC for clock gating, PWR block for voltage scaling, and wakeup flags.

```
ca_pwr_get_info()              reads RCC_xxxENR of every listed peripheral
ca_pwr_periph_table()             legge PWR->CR/CSR (VOS, standby flag)
```

No peripheral driver (`uart.c`, `spi.c`, `can.c`, ...) contains hooks to `pwr`: the information "this peripheral is powered on" is already written in the RCC enable bit since the peripheral itself was initialized, and `pwr` reads it from there. This avoids the classic (and fragile) pattern of keeping a duplicate counter/flag for each driver that must be manually kept in sync.

The subsystem follows the standard caOS driver architectural pattern, on three layers:

```
caos/drivers/inc/pwr.h          # API pubblica: ca_pwr_mode_t, ca_pwr_info_t,
    ca_pwr_enter()...
caos/drivers/src/pwr.c          # public wrapper + CLI "pwr" + retune
    coordination
        extern ca_pwr_hw_*

hw/<family>/drivers/pwr_<family>.c   # bare-metal CMSIS per family
```

Tested hardware: **U575, F407, H563, H753** (STOP and STANDBY; `stopms` on U575, H563 and H753). Reference application example: **example 59** (CLI `pwr`).

22.1.2 `pwr info` — power state snapshot

`ca_pwr_get_info(ca_pwr_info_t *info)` populates:

```
typedef struct {
    u32  sysclk_hz;         /* current SYSCLK */
    u32  sysclk_max_hz;     /* nominal SYSCLK of the board */
    u8   vos;               /* current voltage scaling; 0xFF = not available (F1)
     */
    bool standby_resume;    /* true if the boot comes from Standby */
    u16  periph_on;         /* peripherals with RCC clock active */
    u16  periph_total;      /* peripherals listed for the family */
} ca_pwr_info_t;
```

- `sysclk_hz` / `sysclk_max_hz`: useful for checking at a glance if frequency scaling (`ca_pwr_set_freq`) is active compared to the nominal value.
- `vos`: voltage scaling regulator output — on F1xx this concept does not exist (there is no VOS), so the value is conventionally `0xFF`.
- `standby_resume`: the current reset comes from a STANDBY wakeup (read once in `ca_pwr_init ()` and cached, because the HW flag is consumed/cleared at startup).
- `periph_on` / `periph_total`: count of peripherals with RCC enable bit active, based on a total census per family.

The peripheral table below is also exposed directly:

```
const ca_pwr_periph_t *ca_pwr_periph_table(u32 *count);
```

```
typedef struct {
    const char   *name;   /* es. "USART1" */
    const char   *bus;    /* es. "APB2" */
    volatile u32 *enr;    /* RCC xxxENR register */
    u32           mask;   /* clock enable bit */
} ca_pwr_periph_t;
```

Each entry points **directly** to the actual RCC register: it is not a static snapshot; the caller re-reads `*enr & mask` whenever it wants the updated state.

List of peripherals and delta vs baseline

`pwr info` **always** prints the list of peripherals with active clock, one line per bus (`AHB1:  / APB1 :  / APB2:  `). The count alone (`periph=14/81`) says little; the explicit list says *which* ones are on.

To make the information actionable, `ca_pwr_init()` captures a **read-only baseline**: a snapshot of the RCC bits that were on at the time of init ("cold"). Each subsequent `pwr info` compares the current state with that baseline and marks the deviations:

Marker	Meaning
`NOME`	on, consistent with the baseline
`NOME+`	turned on **after** init, was not in the baseline → **residual clock**
`off: NOME-`	was on at cold start, **now off**

```
pwr: sysclk=64000000/64000000 Hz  vos=1  standby_resume=no  periph=14/81
AHB1: GPIOA GPIOB DMA1+
APB1: USART2 PWR
APB2: USART1 SYSCFG RNG+
off: SPI3-
```

In the example `DMA1` and `RNG` are clocks enabled after init (candidates for shutdown before a stop), `SPI3` was turned off compared to cold boot. The baseline is **purely diagnostic**: no writes to RCC, no automatic restoration — active power gating, if needed, should be explicitly declared in the code, not derived from the hardware state.

22.1.3 Frequency scaling

```
u32 ca_pwr_set_freq(u16 scale);     /* scale = 1..99 (%) */
u32 ca_pwr_restore_freq(void);
```

`ca_pwr_set_freq` reduces SYSCLK by acting on the AHB prescaler (`HPRE`) instead of recalculating the PLL from scratch — it is the most efficient and reversible change. The delicate point is that **reducing the CPU frequency is not an isolated operation**: anyone deriving a time base from SYSCLK/HCLK must be recalibrated in a coordinated manner, otherwise it becomes silently incorrect (UART baud rate wrong, CAN bit-timing out of sync, SysTick no longer ticking at 1ms):

```
u32 ca_pwr_set_freq(u16 scale) {
    u32 f = ca_os_clock_reduce_freq(scale);
    ca_os_tick_retune();      /* SysTick: recompute the reload for 1 ms at the
    new HCLK */
    ca_uart_retune();         /* recompute BRR of every active UART */
#if EN_CAN
    ca_can_retune();          /* recomputes bxCAN/FDCAN bit timing */
#endif
    return f;
}
```

`ca_pwr_restore_freq()` performs the same coordinated retune to return to the nominal value. Peripherals with **kernel clock independent of HCLK** (USB at 48MHz, RTC on LSE/LSI) do not require retune: they are already decoupled from SYSCLK upstream.

The CLI (`pwr freq <1-99>`) also warns if the new HCLK violates a minimum constraint of currently active peripherals (e.g., Ethernet RMII requires HCLK 25MHz, USB FS DWC2 requires AHB 14.2MHz, WWDG has a window that scales with PCLK) — this is only an informative warning, it does not block the operation.

22.1.4 Low-power modes

```
int ca_pwr_enter(ca_pwr_mode_t mode, u32 wake_flags, u32 duration);
```

Mode	Behavior	Wake
`CA_PWR_MODE_SLEEP`	`WFI` simple, clock and RAM remain active	any IRQ (including SysTick)
`CA_PWR_MODE_STOP`	deepsleep, regulator in low-power, PLL off	returns to caller after clock reinit
`PWR_MODE_STANDBY`	RAM lost, almost everything off	**reset** (detectable via `standby_resume`)

`CA_PWR_MODE_SLEEP`

The simplest case: `__disable_irq()` + OS tick suspension + `WFI` + restoration. The clock remains active, so **any IRQ** (not just the declared wake source) exits the WFI. It does not require reinit of anything upon return.

`CA_PWR_MODE_STOP`

Deepsleep with regulator in low-power mode: PLL and high-frequency oscillators are turned off by the hardware upon entry. Upon wake-up, the CPU restarts on a slow default clock (MSI/HSI depending on the family), and it is necessary to **explicitly reinitialize the PLL** before doing anything else:

```
__disable_irq();
ca_os_tick_suspend();
ca_pwr_hw_enter_stop();          /* WFI in deepsleep, HW-specifico */

/* on wake-up: reinit right away, IRQs still disabled */
int rc = ca_os_clock_config();
SystemCoreClockUpdate();
ca_os_tick_retune();
ca_os_tick_resume();
ca_uart_retune();
#if EN_CAN
ca_can_retune();
#endif
__enable_irq();
```

The reinit occurs **before** re-enabling IRQs and before any `printf`/sleep: a scheduler cycle at the slow post-wake clock would cost real seconds (tasks are calibrated for the full clock) and would also skew the measurement of time spent in stop.

Critical point — the OS tick remains stopped during STOP. The time that passes in deepsleep is not counted by the scheduler: the sleeps of ongoing tasks (`ca_os_sleep`) **shift** forward by the entire duration of the stop. If an independent watchdog is active (IWDG, which runs on LSI and does not stop in stop mode), its timeout must be sized to exceed the maximum expected duration of the stop, otherwise the system will reset itself during the sleep period. The limit is `ca_wdt_max_timeout_ms(CA_WDT_TYPE_IWDG)`: ~32.7 s on F4/F7/H7, ~131 s on H5/U5 (§11.10).

`PWR_MODE_STANDBY`

The deepest mode: RAM is not retained, wake-up **is a core reset**, not a "return" from `ca_pwr_enter`. The application distinguishes a normal boot from a post-standby boot by reading `ca_pwr_info_t::standby_resume` immediately after `ca_pwr_init()`.

```
__disable_irq();
ca_os_tick_suspend();
for (u32 i = 0; i < 8U; i++) {
    NVIC->ICER[i] = 0xFFFFFFFFU;   /* disable */
    NVIC->ICPR[i] = 0xFFFFFFFFU;   /* clear pending */
}
SCB->ICSR = SCB_ICSR_PENDSVCLR_Msk | SCB_ICSR_PENDSTCLR_Msk;
ca_pwr_hw_enter_standby();   /* does not return */
```

A full purge of the NVIC (disable + clear pending on all 8 registers) before the internal `WFI` is **necessary**, not defensive: if any interrupt remains pending (typically OTG with the USB console active, or SysTick just suspended but not yet consumed), the `WFI` behaves like a nop and execution continues beyond it — meaning STANDBY is never reached, and the code hangs at the next point because there is nothing coherent left to execute (peripherals already half-powered down).

22.1.5 Wake-up sources

`wake_flags` is a bitmask that can be combined; **at least one** is mandatory (otherwise `ca_pwr_enter` rejects with `-1`, to avoid entering a low-power mode from which there is no return):

Flag	Source	Requires	duration
CA_PWR_WAKE_RTC	RTC Alarm (h:m:s match)	ca_rtc_init	**seconds**
CA_PWR_WAKE_PIN	WKUP1 pin (PA0 on most families)	—	ignored
CA_PWR_WAKE_LPTIM	LPTIM1 one-shot on LSI	**U5xx/H5xx/H7xx/F7xx**	**milliseconds** (max 262144, resolution ~31 µs)

Semantics of duration

The duration field changes **unit of measurement** depending on the flag used, because they are different hardware counters:

- CA_PWR_WAKE_RTC: duration in **SECONDS**, the absolute time now + duration is calculated (24h wrap) and programmed as a second/minute/hour match on the RTC alarm A.
- CA_PWR_WAKE_LPTIM: duration in **MILLISECONDS**, LPTIM1 counts on the LSI clock with fine resolution (up to ~31 µs), useful for rapid tests or short duty-cycle cycles where one second of RTC granularity would be too coarse.
- CA_PWR_WAKE_PIN: duration is not used (the pin is a pure external edge).

If CA_PWR_WAKE_RTC | CA_PWR_WAKE_LPTIM are combined in a single call, duration is applied to **both** with their respective units (same numerical value, interpreted twice) — in typical usage, only one timer is combined, possibly with the pin.

Wake from a driver LPTIM (EN_LPTIM)

An LPTIM driver instance initialized with wake=true (ca_lptim_init_periodic/oneshot) is an **implicit** wake-up source: ca_pwr_enter(CA_PWR_MODE_STOP, 0, 0) accepts entering Stop even with wake_flags=0 — the pwr queries ca_lptim_wake_active() and re-enables the LPTIM NVIC after the purge (on F7/H7 the EXTI 23/47 line is already armed by the driver at start). The periodic wake-armed is not touched upon wake: it continues to count and can wake subsequent Stops (ex61 phase C pattern).

Mutual exclusion guard: CA_PWR_WAKE_LPTIM (stopms) arms LPTIM1 directly, so if the driver owns LPTIM1 (ca_lptim_is_busy(1)), ca_pwr_enter with that flag returns -1.

Wake from LPUART (EN_LPUART)

Same pattern as LPTIM: an LPUART initialized with wake=true (ca_lpuart_init) is an implicit wake-up source — **receiving a character wakes the MCU** from Sleep/Stop, the byte is not lost (kernel clock HSI16+HSIKERON active in Stop) and is delivered to the callback after wake-up. The pwr queries ca_lpuart_wake_active() and re-enables the LPUART NVIC (on H7, the EXTI 34 line is armed by the driver). Typical scenario: CLI console on LPUART that responds to the first key press with the MCU in Stop (ex62).

22.1.6 Status by family

Family	STOP	STANDBY	`stopms` (LPTIM)	VOS
U575	HW tested	HW tested	HW tested	available
F407	HW tested	HW tested	not available (no LPTIM)	available
H563	HW tested	HW tested	HW tested	available
H753	HW tested	HW tested	HW tested	available
F7xx	build-only	build-only	build-only	available
F1xx	build-only	build-only	not available (no LPTIM)	`0xFF` (not applicable)

Wake-up via LPTIM (`stopms`, sub-millisecond resolution) is implemented on **U5xx, H5xx, H7xx and F7xx** and validated on U575, H563 and H753 hardware (F7xx build-only pending test board). The implementation handles the two variants of the LPTIM IP: v2 on U5/H5 (register `DIER`), v1 on H7/F7 (register `IER`, writable only when the timer is disabled); on H7 and F7, the LPTIM1 interrupt also passes through an EXTI line that must be unmasked so that the event wakes the system from STOP. F4xx and F1xx do not have an LPTIM block usable in this configuration and therefore remain limited to RTC wake-up (1s granularity) or pins.

22.1.7 Implementation notes

- **"Stale" wakeup flags**: PWR/RTC status flags signaling the last wakeup cause are stored in the backup domain, powered by the backup battery (VBAT) even when the board is powered off. A flag can therefore remain set from a previous wakeup cycle and must be **explicitly cleared before re-arming** a new wake (e.g., `ca_rtc_clear_alarm` after each use), otherwise the subsequent reading of `standby_resume` or the alarm will be ambiguous.
- **NVIC purge before STANDBY**: see above — a pending IRQ makes the `WFI` a nop. This applies to any peripheral with an active IRQ at the time of the call (typically OTG/USB or SysTick).
- **NVIC mask during STOP**: as with standby, a pending IRQ (typically OTG/USB with active CDC console) makes the `WFI` a nop and STOP is never reached. However, in the STOP path, the NVIC is still needed for wakeup: `ca_pwr_enter` therefore saves the `ISER` state, masks everything, clears pending bits, re-enables **only** the requested wake sources (hook for `ca_pwr_hw_stop_wake_nvic` family), and restores the saved state upon return.
- **HSI48 after STOP on U5xx/H5xx/H7xx**: if the CDC (USB) console is active, HSI48 (the dedicated USB kernel clock) must be **explicitly turned back on** upon wakeup from STOP, because STOP turns it off along with the rest of the high-frequency oscillators, and it is not part of the generic system PLL reinit. Without this step, the USB remains silently dead after wake, even though there is no visible error. Handled in `enter_stop` on U5, H5, and H7 (F4 is immune: the 48MHz derives from PLLQ, restored by the clock reinit).
- **IWDG and STOP duration**: as mentioned above, the OS tick is stopped during stop but IWDG (on LSI) is not — size the timeout accordingly.

22.1.8 Configuration

```
#define EN_PWR      1   /* power manager driver */
#define EN_PWR_DBG  1   /* low-power register dump via CLI "pwr dbg"     U5xx/
    F4xx only */
#define EN_CLI_PWR  1   /* CLI node "pwr" */
```

EN_PWR_DBG enables ca_pwr_hw_dbg_dump() (raw dump of relevant low-power registers) and is only available for the U5xx and F4xx families at this stage. The list of peripherals with delta vs baseline (above) is instead **always** in pwr info and does not depend on EN_PWR_DBG.

22.1.9 File structure

File	Content
caos/drivers/inc/pwr.h	Public API: ca_pwr_mode_t, ca_pwr_info_t, ca_pwr_periph_t, CA_PWR_WAKE_* flags
caos/drivers/src/pwr.c	Public wrapper + retune coordination + pwr CLI
caos/drivers/inc/ca_glue_hw.h	ca_pwr_hw_* extern declaration
hw/<family>/drivers/pwr_<family>.c	HW implementation per family (init, enter_stop, enter_standby, get_vos, max_freq, periph_table, standby_resumed, wake_pin_enable, lptim_arm/disarm, dbg_dump)

See chapter example 59 for complete practical usage: pwr CLI (info, freq, restore, sleep, stop, stopms, standby, alarm, dbg) with tests on real hardware.

Chapter 23

Migrating from FreeRTOS, Zephyr and ThreadX to caOS

23.1 Migrating from FreeRTOS, Zephyr and ThreadX to caOS

23.1.1 Overview

This chapter is for anyone who has an application written on FreeRTOS, Zephyr or ThreadX and wants to port it to caOS. Sections 1-9 cover FreeRTOS; sections 10 and 11 cover Zephyr and ThreadX and refer back to the earlier ones for everything the ports have in common. There is no compatibility layer: the code has to be rewritten on the `ca_*` API. The translation is almost always one to one, because the two kernels offer the same objects (tasks, semaphores, mutexes, queues, notifications, event groups, stream buffers, software timers). The real work is not renaming calls, but recognising the places where **the same call behaves differently**.

The chapter follows this order:

1. application structure: startup, `main`, clock;
2. tasks, priorities, stacks;
3. time and tick;
4. critical sections and scheduler lock;
5. synchronization and communication objects;
6. heap;
7. semantic differences to know before porting;
8. summary table and checklist;
9. from Zephyr to caOS (section 10);
10. from ThreadX to caOS (section 11).

caOS timeouts are in milliseconds, not ticks (section 5.1): every `xTicksToWait` must be converted, together with `portMAX_DELAY`.

23.1.2 1. Application structure

1.1 No `main`, no `vTaskStartScheduler`

In FreeRTOS the application owns `main()`: it initializes the hardware, creates the tasks and calls `vTaskStartScheduler()`, which never returns. In caOS `main()` belongs to the system: it initializes clock, memory, tick and console, then calls two functions that the application must provide.

FreeRTOS	caOS
`main()`: hardware init	`ca_os_configure()`: peripherals and services (stdout, CLI, drivers)
`main()`: `xTaskCreate(...)`	`ca_os_main()`: `ca_os_job(...)`
`vTaskStartScheduler()`	`ca_os_run(true)` at the end of `ca_os_main()`

```c
#include "caos.h"

void ca_os_configure(void) {
    EVAL_STDOUT(
        ca_stdout_init();
        ca_stdout_set_sender(STDOUT);
        ca_stdout_create_thread();
    )
}

void ca_os_main(void) {
    ca_os_job("worker", CA_TH_LEVEL3, worker, NULL,
              STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);
    ca_os_run(true);
}
```

A task created with `ca_os_job()` is **immediately schedulable**: there is no separate "start". `ca_os_run(true)` activates the scheduler but does not start anything new. To choose the start order, create the task with `ca_os_job_suspend()` (same signature, created suspended) and release it later with `ca_os_resume_handle()`.

1.2 Clock and HAL tick

caOS configures the MCU clock by itself and uses the SysTick as the system tick. From a CubeMX-generated project you must therefore **remove**:

- `SystemClock_Config()`: it would reprogram the clocks after caOS and skew the tick;
- `HAL_InitTick()` / `HAL_IncTick()`: the SysTick belongs to caOS.

caOS is bare-metal on CMSIS and does not use the HAL. Application code that calls the HAL can be kept, but the port concerns the **kernel API**, not an entire CubeMX project. Where the HAL asks for `HAL_GetTick()`, the equivalent value is `ca_os_tick_ms()`.

1.3 Configuration

`FreeRTOSConfig.h` corresponds to the project config `os_config_<project>.h`. It is not written by hand: it is generated and modified with `tools/caos_config.py` (see the configuration chapter). Optional objects have their own flag: `EN_NOTIFY`, `EN_EVENT`, `EN_STREAM`, `EN_SW_TIMER`, `EN_BARRIER`. In the FREE version the configuration is fixed and all these objects are already enabled.

23.1.3 2. Tasks

2.1 Function signature

	FreeRTOS	caOS
signature	`void f(void *arg)`	`u32 f(void *arg)`

	FreeRTOS	caOS
task end	`vTaskDelete(NULL)`, never `return`	`return 0`

The argument is the same: the pointer passed at creation. The only difference is the return type. A caOS task can **return**: the kernel removes it, frees its resources and keeps the returned value as the exit code. In FreeRTOS a `return` from a task is an error.

```
/* FreeRTOS */
static void blink(void *arg) {
    const u32 period = (u32)arg;
    for (;;) {
        toggle_led();
        vTaskDelay(pdMS_TO_TICKS(period));
    }
}
xTaskCreate(blink, "blink", 256, (void *)500, 2, NULL);

/* caOS */
static u32 blink(void *arg) {
    const u32 period = (u32)(uintptr_t)arg;
    for (;;) {
        toggle_led();
        ca_os_sleep(period);
    }
    return 0;
}
ca_os_job("blink", CA_TH_LEVEL2, blink, (void *)500,
          STACK_SMALL, CA_JOB_STD, CA_FAULT_ASSERT);
```

2.2 Creation

```
ca_task_handle_t ca_os_job(const char *name, u16 priority, ca_task_fn_t func,
                           void *arg, ca_stack_size_t stack_size,
                           ca_job_type type, u8 fault_type);
```

Compared with `xTaskCreate()` there are two extra parameters:

- `type`: `CA_JOB_STD`, or the variants with FPU context (`JOB_FLOAT_POINT`, `CA_JOB_SAFE_FLOAT_POINT`) for tasks that use floating point. In FreeRTOS the FPU context is handled by the port; in caOS it is declared.
- `fault_type`: what the kernel does if the task faults (`CA_FAULT_ASSERT`, `CA_FAULT_RESTART_THREAD`, `CA_FAULT_EXEC_EMERGENCY`, `CA_FAULT_CHECK`). In FreeRTOS a fault stops the whole system; in caOS the policy is per task.

2.3 Stack: size and placement

FreeRTOS asks for the stack in **words**; caOS in classes: `STACK_SMALL`, `STACK_MEDIUM`, `STACK_LARGE`, `STACK_KERNEL`.

The classes are **ratios** of `STACK_MIN`, which changes with the family:

MCU	STACK_MIN	SMALL	MEDIUM	LARGE	KERNEL
F103	256	512 B	1 KB	1 KB	1 KB
F407, F429, F446, F746	512	1 KB	2 KB	4 KB	8 KB
F767	1024	2 KB	4 KB	8 KB	16 KB
U575, H563, H753	2048	4 KB	8 KB	16 KB	32 KB

The same `STACK_SMALL` is therefore 1 KB on an F446 and 4 KB on a U575. A task sized on a large board must be rechecked on the small ones. Actual usage can be read at runtime from the CLI.

`xTaskCreateStatic()` **has no equivalent**, and that is by design: in caOS the stack is always allocated by the kernel, in fast memory (CCM/DTCM where available), with an MPU guard and a canary. A user-supplied stack would usually end up in slower RAM and without protection. The static buffer of the FreeRTOS code is simply removed.

2.4 Handles

`TaskHandle_t` corresponds to `ca_task_handle_t`. A caOS handle contains a **generation number**: if the task has terminated and its slot has been reused, the `*_handle` functions fail instead of acting on the new task. In FreeRTOS a stale handle points to freed memory.

FreeRTOS	caOS
`xTaskGetCurrentTaskHandle()`	`ca_os_self_handle()`
`vTaskDelete(h)`	`ca_os_remove_job_handle(h)`
`vTaskDelete(NULL)`	`return 0` from the task function
`vTaskSuspend(h)` / `vTaskResume(h)`	`ca_os_suspend_handle(h)` / `ca_os_resume_handle(h)`
`vTaskPrioritySet(h, p)`	`ca_os_set_priority_handle(h, p)`

Do not store the `thid` (the numeric index of the task) instead of the handle: it is a **position** in the task list and changes when another task is removed.

2.5 Priorities: CFS levels, not fixed priorities

This is the deepest difference between the two kernels.

FreeRTOS is **preemptive with fixed priorities**: the ready task with the highest priority always runs. A high-priority task that never blocks **starves** all the ones below it.

caOS uses a **weighted CFS** scheduler. The seven levels `CA_TH_LEVEL0 ... CA_TH_LEVEL6` are **weights** (1, 2, 4, 8, 16, 32, 64) that share the CPU among the ready tasks in proportion:

Two always-ready tasks	FreeRTOS (prio 3 vs 1)	caOS (LEVEL5 vs LEVEL3)
“high” task	100 %	32 / (32 + 8) = 80 %
“low” task	0 % (starved)	8 / (32 + 8) = 20 %

The idle task is outside the CFS and takes no share from anyone.

How to map: FreeRTOS priorities are spread over the levels in the same order (`tskIDLE_PRIORITY + 1` → `CA_TH_LEVEL0` or `LEVEL1`, the highest → `LEVEL6`). Code that uses priority only as “this is more important than that” ports this way. Code that relies on strict preemption for mutual exclusion must instead be **rethought**, of the kind “the high task is never interrupted by the low one, so no lock is needed”: in caOS the low task runs anyway, for its share. There a mutex or the scheduler lock is needed (section 4).

23.1.4 3. Time

3.1 The tick depends on the MCU

In FreeRTOS `configTICK_RATE_HZ` is chosen by the application, usually 1000 Hz. In caOS the tick is fixed per board (`OS_TICK_US` in `objmap.h`):

MCU	tick
F103	2 ms
F407, F429, F446, U575	250 µs
F746, F767	200 µs
H563, H747, H753	100 µs

The caOS API works almost entirely in **milliseconds**, not ticks: the `pdMS_TO_TICKS()` conversion disappears. `ca_os_tick()` returns raw ticks, `ca_os_tick_ms()` milliseconds.

3.2 Waiting

FreeRTOS	caOS
`vTaskDelay(pdMS_TO_TICKS(ms))`	`ca_os_sleep(ms)`
`vTaskDelayUntil(&last, period)`	`ca_os_sleep_until(&last_ms, period_ms)`
`xTaskGetTickCount()`	`ca_os_tick()` (ticks) / `ca_os_tick_ms()` (ms)
`taskYIELD()`	`ca_os_yield()`
`xTaskAbortDelay(h)`	`ca_os_wake_up_handle(h)` (only on a task in `ca_os_sleep`)

There is also `ca_os_delay_ms()`, but it does **not** correspond to `vTaskDelay`: it is a busy wait (with WFI) to be used **before** the scheduler, in `ca_os_configure()`. Called from a task it triggers a safeguard. From a task always use `ca_os_sleep()`.

`ca_os_sleep_until()` wants `last_ms` initialized with `ca_os_tick_ms()`:

```
u32 last = ca_os_tick_ms();
for (;;) {
    sample();
    ca_os_sleep_until(&last, 10);   /* every 10 ms, no drift */
}
```

In caOS `xTaskAbortDelay()` applies only to a task stopped in `ca_os_sleep()`. There is no way to interrupt from outside a task blocked on a semaphore, a queue or a mutex: this is a design choice. Whoever needs it uses a timeout and rechecks a condition on wake-up.

23.1.5 4. Critical sections and scheduler lock

FreeRTOS	caOS	what it blocks
`taskENTER_CRITICAL()` / `taskEXIT_CRITICAL()`	`ca_enter_critical()` / `ca_exit_critical()`	all IRQs (PRIMASK), nestable
`vTaskSuspendAll()` / `xTaskResumeAll()`	`ca_os_sched_lock()` / `ca_os_sched_unlock()`	only the context switch, IRQs enabled

`ca_os_sched_lock()` nests. While it is held, interrupts are served but the kernel does not take the CPU away from the task: a switch requested in the meantime happens at the last `ca_os_sched_unlock()`. The same rules as FreeRTOS apply: do not block (sleep, waits) while holding the lock, and do not use it from an ISR. If the task faults while holding the lock, the kernel releases it.

23.1.6 5. Synchronization and communication

5.1 Timeouts

In FreeRTOS every wait takes `TickType_t xTicksToWait`, with `0` = do not wait and `portMAX_DELAY` = forever. caOS follows the same rule for **all** blocking objects (semaphores, mutexes, condvars, queues, notifications, event groups, streams, message buffers), but the value is in **milliseconds**:

FreeRTOS	caOS
`0`	`CA_NO_WAIT_TIME` (equals 0)
`pdMS_TO_TICKS(ms)`	`ms`
`portMAX_DELAY`	`CA_WAIT_FOREVER`

The two constants are in `common.h`. There are also calls without a timeout, equivalent to `CA_WAIT_FOREVER` (`ca_sem_wait`, `ca_mutex_lock`, `ca_cond_wait`) and to `CA_NO_WAIT_TIME` (`ca_sem_trywait`, `ca_mutex_trylock`, `ca_queue_*_trysend` / `*_tryreceive`).

5.2 Semaphores and mutexes

FreeRTOS	caOS
`xSemaphoreCreateBinary()`	`ca_sem_init(&s, "name", 0, 1)`
`xSemaphoreCreateCounting(max, init)`	`ca_sem_init(&s, "name", init, max)` (note the order)
`xSemaphoreTake(s, t)`	`ca_sem_timedwait(&s, ms)` / `ca_sem_wait(&s)`
`xSemaphoreGive(s)` / `...FromISR`	`ca_sem_post(&s)` (also valid from ISR)
`xSemaphoreCreateMutex()`	`ca_mutex_init(&m)`
`xSemaphoreTake(m, t)` / `Give(m)`	`ca_mutex_timedlock(&m, ms)` / `ca_mutex_lock(&m)` / `ca_mutex_unlock(&m)`
`xSemaphoreCreateRecursiveMutex()`	`ca_rmutex_init(&r)`, then `ca_rmutex_lock/unlock`

There is no `Create` that allocates and returns a handle: the object's memory is supplied by the caller and the `*_init` initializes it. It can be a static variable, a field of another struct or a heap block. For an object created at runtime and shared between tasks, the counterpart of `xSemaphoreCreate*()` is `ca_memory_allocate_shared()` followed by the `*_init`. With `ca_memory_allocate()`, instead, the object would disappear when the task that created it exits (section 6).

```
ca_semaphore_t *s = ca_memory_allocate_shared(sizeof(ca_semaphore_t));
if (s != NULL) {
    ca_sem_init(s, "rx", 0, 1);     /* = xSemaphoreCreateBinary() */
}
```

In FreeRTOS `xSemaphoreGive()` on a semaphore already at its maximum fails silently. In caOS the same case is written `ca_sem_post_capped()`, designed for ISRs that signal an event and need not care whether the task is lagging behind.

caOS mutexes have **priority inheritance** (Zephyr scheme): the holder of the mutex inherits the level of the highest waiter and gives it back on unlock.

5.3 Queues

caOS has two variants with the same API: `ca_queue_static_*` (buffer declared with `CAQUEUE_STATIC_DEFINE`) and `ca_queue_heap_*` (buffer from the heap, `ca_queue_heap_init` / `ca_queue_heap_destroy`).

FreeRTOS	caOS (static variant)
`xQueueCreateStatic(n, size, ...)`	`CAQUEUE_STATIC_DEFINE(q, n, size)`
`xQueueCreate(n, size)`	`ca_queue_heap_init(&q, size, n)`
`xQueueSend` / `xQueueSendToBack`	`ca_queue_static_send(&q.q, &x, ms)`
`xQueueSendToFront`	`ca_queue_static_send_front(&q.q, &x, ms)`
`xQueueReceive`	`ca_queue_static_receive(&q.q, &x, ms)`
`xQueuePeek`	`ca_queue_static_peek(&q.q, &x)`
`xQueueOverwrite`	`ca_queue_static_overwrite(&q.q, &x)` (single-element queues only)
`xQueueReset`	`ca_queue_static_reset(&q.q)`
`uxQueueMessagesWaiting` / `uxQueueSpacesAvailable`	`ca_queue_static_count` / `ca_queue_static_available`
`xQueueSendFromISR` / `xQueueReceiveFromISR`	`ca_queue_static_send_isr` / `ca_queue_static_receive_isr`

```c
/* FreeRTOS */
QueueHandle_t q = xQueueCreate(8, sizeof(msg_t));
xQueueSend(q, &m, portMAX_DELAY);
if (xQueueReceive(q, &m, 0) == pdTRUE) { ... }       /* poll */

/* caOS */
CAQUEUE_STATIC_DEFINE(q, 8, sizeof(msg_t));
ca_queue_static_send(&q.q, &m, CA_WAIT_FOREVER);
if (ca_queue_static_receive(&q.q, &m, CA_NO_WAIT_TIME)) { ... }  /* poll */
```

Queues are always used through the `.q` member of the union defined by the macro.

5.4 Task notifications

FreeRTOS	caOS
`xTaskNotify(h, v, eSetBits)`	`ca_notify_send_handle(h, v, CA_NOTIFY_ACTION_SET_BITS)`
`xTaskNotify(h, v, eIncrement)`	`... CA_NOTIFY_ACTION_INCREMENT`
`xTaskNotify(h, v, eSetValueWithOverwrite)`	`... CA_NOTIFY_ACTION_OVERWRITE`
`xTaskNotify(h, v, eSetValueWithoutOverwrite)`	`... CA_NOTIFY_ACTION_SET_IF_EMPTY`
`xTaskNotify(h, 0, eNoAction)`	`... CA_NOTIFY_ACTION_NONE`
`xTaskNotifyGive(h)`	`ca_notify_give_inc(h)`
`ulTaskNotifyTake(pdTRUE, t)`	`ca_notify_take(ms)`
`xTaskNotifyWait(0, clrExit, &v, t)`	`v = ca_notify_wait(clear_on_exit, ms)`
`...FromISR`	`ca_notify_send_handle_isr(h, v, action)`

`ca_notify_give(h, v)` is **not** `xTaskNotifyGive`: it writes the value (`OVERWRITE`), it does not increment. The counterpart of `xTaskNotifyGive` is `ca_notify_give_inc(h)`.

`ca_notify_wait()` returns the received value, and `0` both on timeout and when the notification value was zero: a protocol that uses the value zero must be rethought. `ulBitsToClearOnEntry` of `xTaskNotifyWait` has no parameter: it is obtained with `ca_notify_clear(0xFFFFFFFF)` before the wait.

5.5 Event groups

FreeRTOS	caOS
`xEventGroupCreate()`	`ca_event_init(&eg)`
`xEventGroupSetBits` / `...FromISR`	`ca_event_set(&eg, bits)` / `ca_event_set_isr`
`xEventGroupClearBits`	`ca_event_clear(&eg, bits)`
`xEventGroupWaitBits(eg, b, clr, pdTRUE, t)`	`ca_event_wait_all(&eg, b, clr, ms)`
`xEventGroupWaitBits(eg, b, clr, pdFALSE, t)`	`ca_event_wait_any(&eg, b, clr, ms)`
`xEventGroupGetBits`	`ca_event_get(&eg)`
`xEventGroupSync`	no direct equivalent; for a rendezvous of N tasks: `ca_barrier_init` / `ca_barrier_wait`

5.6 Stream and message buffers

FreeRTOS	caOS
`xStreamBufferCreate(size, trig)`	`ca_stream_init(&sb, size)`, then `ca_stream_set_trigger`
`xStreamBufferCreateStatic`	`ca_stream_init_static(&sb, buf, size)`
`xStreamBufferSend` / `Receive`	`ca_stream_send` / `ca_stream_receive`
`...FromISR`	`ca_stream_send_isr` / `ca_stream_receive_isr`
`xMessageBuffer*`	`ca_stream_*` with the message length written at the head by the application

5.7 Software timers

FreeRTOS	caOS
(daemon task, `configUSE_TIMERS`)	`ca_sw_timer_init()` once, before creating timers
`xTimerCreate(name, period, autoReload, id, cb)`	`ca_sw_timer_create(period_ms, CA_TIMER_PERIODIC / CA_TIMER_ONE_SHOT, cb, arg)`
`xTimerStart` / `Stop` / `Reset`	`ca_sw_timer_start` / `ca_sw_timer_stop` / `ca_sw_timer_reset`
`xTimerChangePeriod`	`ca_sw_timer_set_period`
`xTimerDelete`	`ca_sw_timer_delete`
`...FromISR`	`_isr` variants

The callback has signature `void cb(void *arg)` and, as in FreeRTOS, runs in the timer service task: it must not block.

23.1.7 6. Heap

FreeRTOS	caOS
`pvPortMalloc(n)`	`ca_memory_allocate_shared(n)`
`vPortFree(p)`	`ca_memory_free(p, NULL, false)`
`xPortGetFreeHeapSize()`	`ca_memory_get_avail_memory()`

In caOS every block allocated with `ca_memory_allocate()` **belongs to the task** that requested it, and the kernel frees it when the task terminates or is removed: no memory leaks from finished tasks. In FreeRTOS memory from `pvPortMalloc()` has no owner.

FreeRTOS code often allocates in one task and passes the block to another (a message, a buffer, a shared object). If the allocating task terminates first, with `ca_memory_allocate()` the block would be freed from under the recipient. This is why the counterpart of `pvPortMalloc` is `ca_memory_allocate_shared()`: same heap, no owner, freed only by `ca_memory_free()`. Use `ca_memory_allocate()` only where the block really stays with the task.

The third argument of `ca_memory_free()` (`wiped`) zeroes the block before freeing it: use it for keys and sensitive data.

23.1.8 7. Semantic differences

The translated code compiles and works; what can change is **timing** and **ordering**. These points must be known before the port, not discovered after.

Weighted scheduler (section 2.5). A high-level task does not starve the low ones. Code that uses priority as an implicit lock must be protected with a mutex or with `ca_os_sched_lock()`.

An ISR does not bring the switch forward. In FreeRTOS `portYIELD_FROM_ISR(pdTRUE)` makes the woken task run as soon as the ISR ends. In caOS the task woken by an ISR starts at the next scheduler pass: at the tick, or earlier if the running task yields by itself. The exception is when the ISR interrupts the idle task: then the switch is immediate. The ISR→task latency is therefore at most one tick (100–250 µs on the current MCUs). This is a choice for predictability: no interrupt lengthens or shortens a task's slice. Work with a tight deadline goes **in the ISR**, the task keeps the processing. `portYIELD_FROM_ISR` is removed from the code.

FIFO wake-up order. In FreeRTOS, on a contended queue or semaphore, the highest-priority waiter is woken. In caOS **the first to arrive** is woken; the priority only decides how much CPU it gets afterwards. Code that relies on "the most important task always takes the semaphore first" must be reviewed.

No interruption of waits. See section 3.2: use a timeout.

Handles with generation. A stale handle fails, it does not hit another task (section 2.4).

Per-task faults. A task fault does not stop the system if the task is created with `CA_FAULT_RESTART_THREAD`: the kernel removes it, frees its resources and restarts it. `configASSERT` corresponds to `CA_SAFEGUARD(cond, msg)`, which uses the same mechanism.

Stacks always protected. MPU guard and canary on every stack replace `configCHECK_FOR_STACK_OVERFLOW` and `vApplicationStackOverflowHook`, with no configuration.

Per-task watchdog. `ca_os_thread_set_watchdog(cycles)` sets a CPU-cycle budget for the calling task; if it is exceeded, the kernel removes the task (`task watchdog on!`). `ca_os_thread_reset_watchdog()` resets the count. Only running time is counted: it detects a task that consumes too much CPU, not one stuck in a wait.

23.1.9 8. Summary table

FreeRTOS	caOS
`main` + `vTaskStartScheduler`	`ca_os_configure` + `ca_os_main` + `ca_os_run(true)`
`xTaskCreate`	`ca_os_job` (`ca_os_job_suspend` to start suspended)
`xTaskCreateStatic`	`ca_os_job` (stack always owned by the kernel)
`vTaskDelete`	`ca_os_remove_job_handle` / `return 0`

FreeRTOS	caOS
`vTaskDelay` / `vTaskDelayUntil`	`ca_os_sleep` / `ca_os_sleep_until`
`vTaskSuspend` / `vTaskResume`	`ca_os_suspend_handle` / `ca_os_resume_handle`
`vTaskPrioritySet`	`ca_os_set_priority_handle`
`taskYIELD`	`ca_os_yield`
`portYIELD_FROM_ISR`	(removed)
`taskENTER/EXIT_CRITICAL`	`ca_enter_critical` / `ca_exit_critical`
`vTaskSuspendAll` / `xTaskResumeAll`	`ca_os_sched_lock` / `ca_os_sched_unlock`
semaphores, mutexes	`ca_sem_*`, `ca_mutex_*`, `ca_rmutex_*`
queues	`ca_queue_static_*`, `ca_queue_heap_*`
notifications	`ca_notify_*`
event groups	`ca_event_*`
stream / message buffers	`ca_stream_*`
software timers	`ca_sw_timer_*`
`pvPortMalloc` / `vPortFree`	`ca_memory_allocate_shared` / `ca_memory_free`
`configASSERT`	`CA_SAFEGUARD`

Left out, with no equivalent: queue sets, co-routines, FreeRTOS MPU wrappers, SMP.

23.1.10 9. Porting checklist

1. `main()` becomes `ca_os_configure()` + `ca_os_main()`; remove `SystemClock_Config()` and the HAL tick.
2. Task signatures from `void f(void *arg)` to `u32 f(void *arg)`, with `return 0` instead of `vTaskDelete (NULL)`.
3. Stacks from words to `STACK_*` classes, rechecked on the smallest board.
4. Priorities on the `CA_TH_LEVEL0..6` levels; look for code that uses priority as mutual exclusion.
5. `vTaskDelay` → `ca_os_sleep`, never `ca_os_delay_ms` from a task.
6. Timeouts from ticks to milliseconds; `portMAX_DELAY` → `CA_WAIT_FOREVER`.
7. `xTaskNotifyGive` → `ca_notify_give_inc`, not `ca_notify_give`.
8. `pvPortMalloc` → `ca_memory_allocate_shared` where the block passes between tasks.
9. Remove `portYIELD_FROM_ISR`; move work with a tight deadline into the ISR.
10. Check code that relies on priority-ordered wake-up on IPC.

23.1.11 10. From Zephyr to caOS

Zephyr is closer to caOS than FreeRTOS: timeouts have the same form, the scheduler lock has the same semantics, and the priority inheritance of caOS mutexes follows the Zephyr scheme. This section reports only what is specific to Zephyr; for startup, stacks, tick, heap and scheduling differences, sections 1-7 apply.

10.1 Direct correspondences

Zephyr	caOS	Note
`K_NO_WAIT` / `K_FOREVER` / `K_MSEC(ms)`	`CA_NO_WAIT_TIME` / `CA_WAIT_FOREVER` / `ms`	same semantics on all objects
`k_msleep(ms)`, `k_sleep(K_MSEC(ms))`	`ca_os_sleep(ms)`	

Zephyr	caOS	Note
`k_yield()`	`ca_os_yield()`	
`k_wakeup(t)`	`ca_os_wake_up_handle(h)`	in both it only wakes a sleeping thread
`k_current_get()`	`ca_os_self_handle()`	
`k_thread_suspend` / `k_thread_resume`	`ca_os_suspend_handle` / `ca_os_resume_handle`	
`k_thread_abort(t)`	`ca_os_remove_job_handle(h)`	
`k_thread_priority_set(t, p)`	`ca_os_set_priority_handle(h, p)`	different scale, section 10.2
`k_sched_lock()` / `k_sched_unlock()`	`ca_os_sched_lock()` / `ca_os_sched_unlock()`	nestable, IRQs enabled
`irq_lock()` / `irq_unlock(key)`	`ca_enter_critical()` / `ca_exit_critical()`	
`k_spin_lock` / `k_spin_unlock`	`ca_spinlock_lock_irq` / `ca_spinlock_unlock_irq`	
`k_sem_init(s, init, limit)`	`ca_sem_init(&s, "name", init, limit)`	
`k_sem_take(s, t)` / `k_sem_give(s)`	`ca_sem_timedwait(&s, ms)` / `ca_sem_post(&s)`	`give` beyond the limit: `ca_sem_post_capped`
`k_sem_count_get(s)`	`ca_sem_getvalue(&s)`	
`k_condvar_wait(cv, m, t)` / `signal` / `broadcast`	`ca_cond_timedwait(&cv, &m, ms)` / `ca_cond_signal` / `ca_cond_broadcast`	
`K_MSGQ_DEFINE(q, size, n, align)`	`CAQUEUE_STATIC_DEFINE(q, n, size)`	
`k_msgq_put` / `k_msgq_get` / `k_msgq_peek` / `k_msgq_purge`	`ca_queue_static_send` / `_receive` / `_peek` / `_reset`	
`k_event_post` (OR of the bits)	`ca_event_set`	
`k_event_set` (replaces the bits)	`ca_event_clear` + `ca_event_set`	
`k_event_wait` / `k_event_wait_all`	`ca_event_wait_any` / `ca_event_wait_all`	
`k_pipe_put` / `k_pipe_get`	`ca_stream_send` / `ca_stream_receive`	
`k_malloc` / `k_free`	`ca_memory_allocate_shared` / `ca_memory_free`	section 6

10.2 Correspondences with adaptation

Thread function. In Zephyr the signature is `void f(void *p1, void *p2, void *p3)`, in caOS `u32 f(void *arg)`. When more parameters are needed they are gathered in a struct and its address is passed; it must stay valid for the whole life of the thread (static, or allocated with `ca_memory_allocate_shared`).

Stack and static definition. `K_THREAD_STACK_DEFINE` and `K_THREAD_DEFINE` have no equivalent: the stack is always allocated by the kernel, in fast, protected memory (section 2.3). `k_thread_create(...)` becomes `ca_os_job(...)`; a thread created with a start delay is created with `ca_os_job_suspend()` and released with `ca_os_resume_handle()` at the right moment.

Priorities and cooperative threads. In Zephyr a lower number means a higher priority, and negative values denote **cooperative** threads, which are never interrupted by another thread. caOS has seven weighted CFS levels (section 2.5), with the highest at `CA_TH_LEVEL6`, and has no

cooperative threads. A cooperative thread is ported as a normal thread; the sections that rely on not being interrupted are enclosed between `ca_os_sched_lock()` and `ca_os_sched_unlock()`.

Recursive mutex. `k_mutex` is **recursive** and has priority inheritance. In caOS the two properties live in different objects: `ca_mutex_*` has priority inheritance but is not recursive, `ca_rmutex_*` is recursive but without priority inheritance. Code that re-takes a mutex it already owns must be ported to `ca_rmutex_*` and loses priority inheritance; where priority inheritance is needed, the re-entry must be removed. This is the most insidious difference of the port, because a `ca_mutex_lock` repeated by the same task gives no compilation error.

Timers. In Zephyr the expiry function of `k_timer` runs in **ISR** context; in caOS the one of `ca_sw_timer_*` runs in the timer service task (section 5.7). Code written for Zephyr is already ISR-safe and works as is; `k_timer_status_sync()` has no equivalent and is replaced by a semaphore given by the callback.

Workqueues. `k_work_submit` corresponds to `ca_pool_submit` (thread pool, dedicated chapter). Delayed work (`k_work_schedule`) is obtained with a one-shot `ca_sw_timer` whose callback calls `ca_pool_submit`.

FIFOs, LIFOs, queues and pointer stacks. `k_fifo`, `k_lifo`, `k_queue` and `k_stack` are pointer queues with no size limit. In caOS they become a fixed-capacity queue holding pointers (`CAQUEUE_STATIC_DEFINE(q, n, sizeof(void *))`); LIFO behaviour is obtained with `ca_queue_static_send_front`. The capacity must be sized: where Zephyr always enqueues, caOS can answer "queue full".

10.3 Semantic differences

They are the same as for the FreeRTOS port (section 7):

- **Wake-up order**: Zephyr wakes by priority, caOS in order of arrival (FIFO).
- **Wake-up from ISR**: Zephyr switches thread on ISR exit if the woken one has higher priority; caOS at the next tick, unless the ISR interrupted the idle task.
- **Faults**: instead of `k_sys_fatal_error_handler`, caOS applies a per-task policy (`fault_type`) and can call an emergency handler (`ca_task_set_emergency_exec`). MPU guard and canary are active on every stack with no configuration, unlike `CONFIG_MPU_STACK_GUARD`.

10.4 Out of scope

The port concerns the kernel API. The following are left out and must be rewritten on the caOS counterparts:

Zephyr	caOS
Kconfig, `prj.conf`, devicetree	project config (`os_config_*.h`) and the board's `objmap.h`
driver model (`DEVICE_DT_GET`, `gpio_pin_configure_dt`, ...)	`ca_*` drivers indexed by peripheral number
`LOG_INF` / `LOG_ERR`	`ca_stdout_printf_n`
shell	CLI (`ca_cli_register`)
BSD sockets	`ca_tcp_*`, `ca_udp_*`

No equivalent: userspace, SMP, `k_poll`, `k_mbox`.

23.1.12 11. From ThreadX to caOS

ThreadX (Azure RTOS, now Eclipse ThreadX) has a small, regular API: every object has `tx_<object>_create`, operations return a code (`TX_SUCCESS`, `TX_NO_INSTANCE`, ...) and all waits take a timeout in ticks. Two fundamental choices coincide with caOS: wait lists are **FIFO** by default, and the values `TX_NO_WAIT` (0) and `TX_WAIT_FOREVER` (`0xFFFFFFFF`) are the same as `CA_NO_WAIT_TIME` and `CA_WAIT_FOREVER`. The unit changes: ThreadX counts in ticks, caOS in milliseconds.

11.1 Startup

`main()` calls `tx_kernel_enter()`, which calls `tx_application_define()` where the objects are created, and then never returns. In caOS the content of `tx_application_define()` goes into `ca_os_main()`, followed by `ca_os_run(true)` (section 1.1). The `first_unused_memory` parameter, used to carve out memory pools, has no equivalent: the caOS heap is already initialized by the system.

11.2 Direct correspondences

ThreadX	caOS	Note
`TX_NO_WAIT` / `TX_WAIT_FOREVER` / ticks	`CA_NO_WAIT_TIME` / `CA_WAIT_FOREVER` / ms	same values, different unit
`tx_thread_sleep(ticks)`	`ca_os_sleep(ms)`	
`tx_thread_relinquish()`	`ca_os_yield()`	
`tx_thread_identify()`	`ca_os_self_handle()`	
`tx_thread_suspend` / `tx_thread_resume`	`ca_os_suspend_handle` / `ca_os_resume_handle`	
`tx_thread_terminate` + `tx_thread_delete`	`ca_os_remove_job_handle`	
`tx_thread_priority_change`	`ca_os_set_priority_handle`	different scale, section 11.3
`tx_thread_wait_abort`	`ca_os_wake_up_handle`	only on a task in `ca_os_sleep` (section 3.2)
`tx_interrupt_control(TX_INT_DISABLE / restore)`	`ca_enter_critical()` / `ca_exit_critical()`	
`tx_semaphore_get(&s, t)` / `tx_semaphore_put(&s)`	`ca_sem_timedwait(&s, ms)` / `ca_sem_post(&s)`	
`tx_semaphore_ceiling_put(&s, max)`	`ca_sem_post_capped(&s)`	the ceiling is the one given to `ca_sem_init`
`tx_queue_send` / `tx_queue_receive`	`ca_queue_static_send` / `ca_queue_static_receive`	
`tx_queue_front_send` / `tx_queue_flush`	`ca_queue_static_send_front` / `ca_queue_static_reset`	
`tx_event_flags_set(&g, f, TX_OR)`	`ca_event_set(&g, f)`	
`tx_event_flags_set(&g, f, TX_AND)`	`ca_event_clear(&g, ~f)`	in ThreadX `TX_AND` clears the bits outside `f`
`tx_event_flags_get(..., TX_OR / TX_AND, ...)`	`ca_event_wait_any` / `ca_event_wait_all`	
`TX_OR_CLEAR` / `TX_AND_CLEAR`	`auto_clear = true`	
`tx_byte_allocate` / `tx_byte_release`	`ca_memory_allocate_shared` / `ca_memory_free`	section 6
`tx_block_allocate` / `tx_block_release`	`ca_block_alloc` / `ca_block_free`	`tx_block_pool_create` → `ca_block_pool_init`

11.3 Correspondences with adaptation

Thread function. ThreadX uses `void f(ULONG input)`, caOS `u32 f(void *arg)`: an integer is passed as `(void *)(uintptr_t)n` and read back with `(u32)(uintptr_t)arg`.

`tx_thread_create`**.** It takes name, entry, input, **caller-supplied stack**, priority, preemption threshold, time slice and `auto_start`. In caOS it becomes `ca_os_job()`:

- the stack is allocated by the kernel (section 2.3): the ThreadX buffer is removed and the size is converted to a `STACK_*` class;
- `TX_DONT_START` corresponds to `ca_os_job_suspend()`;
- the time slice has no parameter: CPU sharing is decided by the CFS.

Priorities and preemption threshold. In ThreadX `0` is the highest priority and the value grows towards the lower ones (32 levels by default). caOS has seven weighted levels, with the highest at `CA_TH_LEVEL6` (section 2.5). The **preemption threshold** (a thread that can be interrupted only by those above the threshold) has no equivalent: sections that use it to avoid being interrupted are enclosed between `ca_os_sched_lock()` and `ca_os_sched_unlock()`.

Mutexes. As in Zephyr, `TX_MUTEX` is **recursive**, and priority inheritance is chosen at creation (`TX_INHERIT` / `TX_NO_INHERIT`). In caOS: `TX_INHERIT` without re-entry → `ca_mutex_*`; re-entry without `TX_INHERIT` → `ca_rmutex_*`; the two together do not exist (section 10.2).

Semaphores. `tx_semaphore_create` has no maximum; `ca_sem_init` requires one. For a pure counting semaphore pass the maximum value the program can reach.

Queues. ThreadX messages are 1, 2, 4, 8 or 16 32-bit words and the buffer is supplied by the caller. In caOS the size is free, in bytes: `CAQUEUE_STATIC_DEFINE(q, n, words * 4)`.

Block pools. `ca_block_alloc` does not wait: with the pool empty it returns `NULL`. The `wait_option` of `tx_block_allocate` is obtained by pairing the pool with a counting semaphore initialized to the number of blocks.

Timers. `tx_timer_create` takes separate initial delay and reschedule period; `ca_sw_timer_create` has only one. If the two values differ, create a one-shot timer on the initial delay and have the callback reprogram it with `ca_sw_timer_set_period` and restart it. In both systems the callback runs in the timer service task, except with `TX_TIMER_PROCESS_IN_ISR`.

`tx_thread_wait_abort`**.** In ThreadX it interrupts any wait; in caOS only sleep (section 3.2). For the other waits use a timeout and a condition rechecked on wake-up.

11.4 Semantic differences

- **Wake-up order**: both FIFO by default. `tx_*_prioritize`, which moves the highest-priority waiter to the head, has no equivalent.
- **Wake-up from ISR**: ThreadX switches thread on ISR exit, caOS at the next tick (section 7).
- **Return codes**: ThreadX returns `UINT` with distinct codes (`TX_NO_INSTANCE`, `TX_QUEUE_FULL`, `TX_DELETED`, ...); caOS returns `bool` (or the data). Code that distinguishes between error cases must be rewritten.
- **Faults and stacks**: `tx_thread_stack_error_notify` and stack checking are replaced by the always-active MPU guard and canary and by the per-task fault policy (section 7).

11.5 Out of scope

Azure RTOS / ThreadX	caOS
NetX Duo	network stack `ca_net_*`, `ca_tcp_*`, `ca_udp_*`
FileX	FatFS on SDIO

Azure RTOS / ThreadX	caOS
USBX	USB device stack `caos/usb/`
TraceX	CLI command `sched` and task statistics

No equivalent: GUIX, LevelX, ThreadX Modules, SMP.

Part II

Practical Examples

Chapter 24

Hello World: the First Program on caOS

24.1 Example 1 — Hello World: the first program on caOS

24.1.1 Introduction

Every journey begins with a first step. In the world of programming, that first step is traditionally called “Hello World”: a very simple program whose sole purpose is to demonstrate that the development environment works and that our code can “talk” to the outside world.

In the case of a microcontroller, “talking to the outside world” means sending a message through a serial port (UART) or a debug channel (ITM/SWO). There is no monitor connected, there is no graphical terminal: our program lives inside a chip the size of a fingernail, and the proof that it works is a line of text that appears on a terminal connected via USB cable.

This first example is intentionally minimal. **It does not use the scheduler**, it does not create tasks, it does not leverage any advanced operating system mechanisms. It is pure sequential code, like a traditional “bare metal” program, but executed within the caOS infrastructure.

Why start so simple? Because before understanding how a multitasking system works, you must ensure that the foundations are working: the processor clock is configured correctly, the debug UART transmits, the `ca_printf` function produces readable output. If something goes wrong at this level, adding tasks and a scheduler would only complicate the diagnosis.

24.1.2 The source code

The file is located in `examples/example_1/ca_os_ex1.c`:

```
1  #include "caos.h"
2
3  /**
4   * @brief User module initialization
5   * @details Function called by the system before the scheduler starts.
6   *          In this example nothing needs initializing.
7   */
8  void ca_os_configure(void) {
9      // No extra initialization
10 }
11
12 /**
13  * @brief Main task
14  * @details Endless loop that prints a message every second.
15  *          Does not use the scheduler (no call to ca_os_run).
16  */
17 void ca_os_main(void) {
18     while(1) {
```

```
19          u32 E = 0x123;
20          u16 V = 0x456;
21          u8  T = 0x78;
22          char buffer[128];
23          ca_printf(STDOUT, buffer, sizeof(buffer),
24                    "Hello from CAOS ! E=%08X V=%04x T=%02x\r\n", E, V, T);
25          ca_os_delay_ms(1000);
26      }
27  }
```

24.1.3 Line-by-line analysis

`#include "caos.h"`

All caOS programs include a single header: `caos.h`. This file in turn includes everything necessary: type definitions (`u8`, `u16`, `u32`), kernel function prototypes, drivers, logs, and so on. There is no need to search for and include dozens of separate files.

`void ca_os_configure(void)`

This function **must be defined by the user** (even if empty). It is called by the system during initialization, before the scheduler starts. It is used to initialize additional hardware modules (I2C, SPI, USB, log, CLI, etc.). In this example, nothing is needed, so the function is empty.

`void ca_os_main(void)`

This is the entry function of the caOS application. It is the equivalent of the `main()` function in a traditional C program, but with one important difference: when `ca_os_main` is called, the system has already completed basic hardware initialization (clocks, debug peripherals, MPU, log system). The programmer finds an environment that is already ready.

The `while(1)` loop

Since in this example we **do not activate the scheduler** (we do not call `ca_os_run`), the code runs in a classic infinite loop, exactly as it would in a bare metal program. The processor executes instructions one after the other, without interruptions, without context switches, without other tasks.

The `E`, `V`, `T` variables

These are three variables of different sizes: 32-bit, 16-bit, and 8-bit. They serve as an example to demonstrate that the `ca_printf` correctly handles different hexadecimal print formats (`%08X`, `%04x`, `%02x`). In a real project, they could represent an error code, a value read from a sensor, and a status flag.

`ca_printf(STDOUT, buffer, sizeof(buffer), ...)`

This is the caOS formatted print function. It works similarly to the standard C `printf`, but with some differences designed for an embedded system:

- `STDOUT`: indicates the output channel (the debug UART or ITM, depending on the configuration in `os_config.h`).
- `buffer`: a temporary buffer provided by the caller. Unlike the library `printf`, which uses internal buffers (and is therefore not thread-safe without precautions), here the buffer is explicit and local to the function stack.

- `sizeof(buffer)`: the maximum size, to avoid buffer overflows.
- The rest is the classic `printf` format with variable arguments.

`ca_os_delay_ms(1000)`

Introduces a delay of 1000 milliseconds (1 second). This is an **active** wait function (busy-wait): the processor counts clock cycles without doing anything useful. In a system with an active scheduler, one would use `ca_os_sleep()`, which instead "puts the task to sleep" and leaves the CPU free for other tasks. But here, without a scheduler, the active delay is the only option.

24.1.4 What this example teaches us

1. **The minimum structure of a caOS program**: include `caos.h`, implement `ca_os_main`, and the system takes care of the rest.
2. **Debug output works**: if we see the message on the terminal, we know that clock, UART, and `ca_printf` are configured correctly.
3. **Without a scheduler, the behavior is sequential**: the processor executes the while loop continuously, monopolizing the CPU. This is exactly the starting "bare metal" behavior.

The next step will be to activate the scheduler and create actual tasks: that is where caOS shows its value compared to bare metal.

24.1.5 Example output

Below is the output produced by the example on the debug serial port:

```
================================================
 -- c.a.O.S --
   build : b8eeed75+dev - 2026-02-14 17:45:54
   MCU : STM 32 F:H7xx T:753
================================================
   System Clock: 480000000 Hz
   SysTick: 47999 clocks per tick
   Debug UART3 Baudrate: 4000000 bps
================================================
Hello from CAOS ! E=00000123 V=0456 T=78
Hello from CAOS ! E=00000123 V=0456 T=78
```

Chapter 25

Scheduling and Memory Allocation

25.1 Example 2 — Scheduling and memory allocation

25.1.1 Introduction

In the first example, we saw a program running in an infinite loop, by itself, without any other task. It is the simplest way to use a microcontroller, but also the most limited: if the program needs to do two things "simultaneously" (for example, reading a sensor and blinking an LED), the programmer must devise state machines, flags, and manual timing.

This second example takes a leap forward: **it activates the caOS scheduler and creates two independent tasks** that run in parallel. Furthermore, each task uses **dynamic memory allocation** to request a block of RAM from the heap, use it, and then return it to the system.

25.1.2 What is scheduling?

Imagine an office with only one desk (the CPU) and two employees (the tasks). They cannot both sit at the desk at the same time, but a supervisor (the scheduler) makes them take turns: "You work for a while, then you get up and it is the other one's turn." If an employee has nothing to do (they are sleeping with `ca_os_sleep`), the supervisor gives the desk to whoever has work to do.

This mechanism is called **cooperative/preemptive multitasking**: each task believes it has the CPU all to itself, but in reality, the scheduler alternates them so quickly (every fraction of a millisecond) that it seems they are running in parallel.

25.1.3 What is dynamic memory allocation?

In a traditional C program on a PC, when memory is needed, `malloc` is used. On a microcontroller, the situation is more delicate: RAM is scarce (tens or hundreds of KB), there is no operating system to manage virtual memory, and an error can crash the entire system.

caOS provides its own memory allocator (`ca_memory_allocate` / `ca_memory_free`) which offers important guarantees:

- **Owner tracking**: every allocated block knows which task owns it. If a task is terminated, the system can automatically free its memory.
- **MPU protection**: heap boundaries are protected by the Memory Protection Unit (MPU). If a bug causes a write beyond the heap, the processor generates a fault instead of silently corrupting other data.
- **Thread-safe**: multiple tasks can allocate and free memory simultaneously without corrupting the allocator's internal structures.

25.1.4 The source code

The file is located in `examples/example_2/ca_os_ex2.c`:

```c
#include "caos.h"

static u32 terminate;

static u32 task_1_exec(void *arg) {
    const u32 p1 = (u32)(uintptr_t)arg;
    char *p = ca_memory_allocate(1024);
    int count = 0;
    ca_stdout_printf_n("TASK 1: par1=%u", p1);
    while(terminate) {
        count++;
        ca_memset(p, 0, 1024);
        ca_stdout_printf_n("TASK 1: Hi!");
        ca_os_sleep(391);
    }
    ca_memory_free(p, NULL, false);
    return 0;
}

static u32 task_2_exec(void *arg) {
    const u32 p1 = (u32)(uintptr_t)arg;
    char *p = ca_memory_allocate(512);
    int count = 0;
    ca_stdout_printf_n("TASK 2: par1=%u", p1);
    while(terminate) {
        count++;
        ca_memset(p, 0, 50);
        ca_stdout_printf_n("TASK 2: hello!");
        ca_os_sleep(457);
    }
    ca_memory_free(p, NULL, false);
    return 0;
}

void ca_os_main(void) {
    terminate = 1;
    ca_os_job("t1_w1", CA_TH_LEVEL3, task_1_exec, (void *)(uintptr_t)1,
                       STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);
    ca_os_job("t1_w2", CA_TH_LEVEL3, task_2_exec, (void *)(uintptr_t)3,
                       STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);
    ca_os_run(true);
}
```

25.1.5 Detailed analysis

The `terminate` variable

```c
static u32 terminate;
```

It is a global variable shared between tasks. As long as it is 1, the tasks continue to run in their `while` loop. If someone were to set it to 0, both tasks would exit the loop, free the allocated memory, and terminate with `return 0`. It is a simple mechanism to control the life of tasks from the outside.

The task function

```c
static u32 task_1_exec(void *arg) {
    (void)arg;
```

Each task in caOS is a function with this signature: - `idx`: task index assigned by the scheduler (useful for debugging). - `p1`: parameter passed during task creation. It can be used for any purpose: an address, a configuration value, an ID. - **Returns** `u32`: the return value (0 = success). caOS can use it for diagnostics.

Memory allocation

```
char *p = ca_memory_allocate(1024);
```

At the beginning of the task, before entering the loop, we allocate 1024 bytes from the heap. This is an operation that must be performed with awareness:

- The allocator aligns the requested size to 8 bytes.
- Internally, it adds a management header (28 bytes) to track the block.
- The block is marked as "BUSY" and associated with the current task.
- If there is not enough memory, it returns `NULL`.

Task 2 instead allocates 512 bytes: different sizes to demonstrate that the allocator correctly handles multiple blocks of different sizes.

The work loop

```
while(terminate) {
    count++;
    ca_memset(p, 0, 1024);
    ca_stdout_printf_n("TASK 1: Hi!");
    ca_os_sleep(391);
}
```

At each iteration, the task:

1. **Increments a counter** (`count++`).
2. **Clears the allocated buffer** with `ca_memset`. In a real program, useful data would be written and read here. The zero-fill serves to demonstrate that the buffer is accessible throughout its entire length without generating faults.
3. **Prints a message** to the debug port.
4. **Sleeps** for 391 ms with `ca_os_sleep(391)`.

The crucial difference compared to the `ca_os_delay_ms` of example 1 is that `ca_os_sleep` **does not waste CPU cycles**. When a task calls `ca_os_sleep`, the scheduler puts it into the "sleeping" state and passes the CPU to another task (or to the idle task). When the time expires, the scheduler wakes it up and puts it back in the queue for execution.

The two tasks have different sleep times (391 ms and 457 ms) precisely to show that the scheduler manages them independently: they do not march in lockstep, each has its own rhythm.

Memory deallocation

```
ca_memory_free(p, NULL, false);
```

When the task exits the loop (if `terminate` becomes 0), it frees the allocated block: - `p`: the pointer to be freed. - `NULL`: we do not need to know the freed size. - `false`: we do not want a secure "wipe" of the data (useful for sensitive data such as passwords or cryptographic keys; it is not needed here).

Task creation

```
ca_os_job("t1_w1", CA_TH_LEVEL3, task_1_exec, (void *)(uintptr_t)1,
                   STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);
```

The `ca_os_job` function registers a new task in the scheduler. The parameters are:

Parameter	Value	Meaning
name	`"t1_w1"`	Readable name for debugging and logs
priority	`CA_TH_LEVEL3`	Priority level (higher = more important)
function	`task_1_exec`	The function that the task will execute
p1	`1`	Parameter passed to the function
stack	`STACK_MEDIUM`	Task stack size
type	`CA_JOB_STD`	Standard task (not daemon, not one-shot)
fault	`CA_FAULT_ASSERT`	Behavior in case of error (assert)

Starting the scheduler

```
ca_os_run(true);
```

This call starts the scheduler and never returns. From this moment on, the CPU is managed by the scheduler, which alternates between the registered tasks. The `true` parameter enables system tasks (audit log, debug CLI).

25.1.6 What this example teaches us

1. **The transition from bare metal to multitasking**: simply create the task functions, register them with `ca_os_job`, and call `ca_os_run`.
2. **`ca_os_sleep` vs `ca_os_delay_ms`**: the former is cooperative (it releases the CPU), the latter is busy-wait (it wastes cycles). In a multitasking system, always use `ca_os_sleep`.
3. **Dynamic memory is protected**: the allocator tracks who owns what, and the MPU monitors the heap boundaries.
4. **Tasks are independent**: each has its own stack, its own rhythm, and its own allocated memory. The scheduler takes care of making them coexist.

25.1.7 Example output

Below is the output produced by the example on the debug serial port:

```
================================================
 -- c.a.O.S --
   build : b8eeed75+dev - 2026-02-14 17:45:54
   MCU : STM 32 F:H7xx T:753
================================================
   System Clock: 480000000 Hz
   SysTick: 47999 clocks per tick
   Debug UART3 Baudrate: 4000000 bps
================================================
TASK 1: par1=1
TASK 2: par1=3
TASK 2: hello!
TASK 1: Hi!
TASK 1: Hi!
TASK 2: hello
```

Chapter 26

Mutex and Shared Resource Access

26.1 Example 3 — LED and GPIO CLI

26.1.1 Introduction

This example demonstrates two driver CLIs that complement each other: the one for `led.c`, which reasons in terms of the board ("turn on LED 2"), and the one for `gpio_driver.c`, which reaches any pin ("read PA0"). A monitor task blinks LD1 every second. **The example does not register any custom CLI hook**: both command sets come from their respective drivers.

The two CLIs are intentionally present together. `led` is portable and does not require knowing the pinout; `gpio` is not portable but reaches everything. On a newly arrived board, ex3 is usually the first thing to be programmed, so raw pin access is useful in the simplest example and not just in ex21/67/83.

Supported Hardware

The example **contains no pins**: how many LEDs the board has and which pins they are on is stated by `CA_LED_TABLE` in the board's `objmap.h`, and the example reaches it only through the `ca_led_*` API. It therefore runs on any part without modifications, and on a board with only one LED (F407, F103) `ca_led_count()` responds 1 without the need for invented pins to make the CLI compile.

Board	Available LEDs
Nucleo-H753ZI	LD1 PB0, LD2 PE1, LD3 PB14
NUCLEO-U575ZI-Q	LD1 PC7, LD2 PB7, LD3 PG2
NUCLEO-H563ZI	LD1 PB0, LD2 PF4, LD3 PG4
NUCLEO-F446ZE	LD1 PB0, LD2 PB7, LD3 PB14
STM32F4VE (F407)	D2 PA6 (only one)

CLI Commands (driver `led.c`, `EN_CLI_LED=1`)

Command	Description
`led info`	How many LEDs the board has and status of each
`led on <n>`	Turns on LED n (1..`ca_led_count()`)
`led off <n>`	Turns off LED n
`led tog <n>`	Toggle and print new status

`n` = LED index, from 1 to `ca_led_count()`. An out-of-range index fails instead of writing to a pin that does not carry an LED.

CLI Commands (driver `gpio_driver.c`, `EN_CLI_GPIO=1`)

Command	Description
`gpio info <P> <n>`	Mode, type, pull, AF and level of a pin
`gpio all <P>`	Decoding of all 16 pins of the port
`gpio get <P> <n>`	Reads the pin status
`gpio set <P> <n> <0|1>`	Writes the pin
`gpio tog <P> <n>`	Toggles the pin

`P` = A..K, `n` = 0..15.

`gpio set` and `gpio tog` write to **any** pin, LED or not. On a pin configured as an alternate function — a UART, the SWD — an active peripheral is overwritten. `info`, `all` and `get` are read-only and do not carry this risk. ### STDOUT Log (every 1 s)

```
EX3: tick=NNNNN  LED1=N
```

26.1.2 Architecture

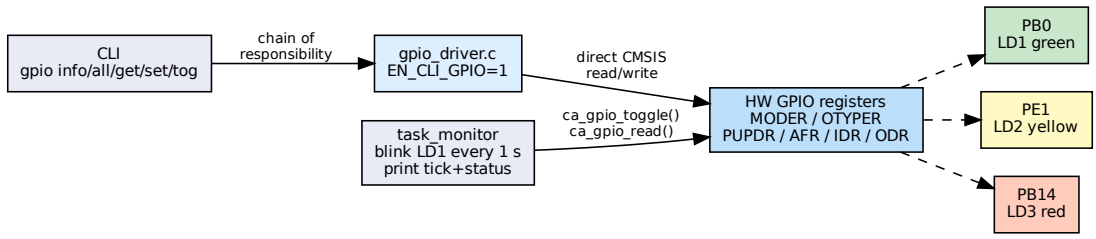


Figure 26.1: LED and GPIO CLI Architecture — Example 3

26.1.3 Source code

The file is located in `examples/example_3/ca_os_ex3.c`.

The example does not contain a pin table: `ca_led_init()` reads the `CA_LED_TABLE` of the board and configures everything itself.

```
void ca_os_configure(void)
{
    /* Clock, pin configuration and "all off" for every LED of the board */
    ca_led_init();
    ...
}
```

26.1.4 Detailed analysis

1. How the two CLIs are registered

`led.c` registers its own node within `ca_led_init()`. The GPIO node, on the other hand, is registered **only within** `ca_gpio_init_port()` and not in `ca_gpio_configure()`: this way, an application that does not touch the GPIOs does not carry that CLI along with it.

This example never calls `ca_gpio_init_port()` directly — it reaches it through the LED driver, which invokes it for the port of every declared LED:

```
/* hw/common/led.c */
ca_gpio_init_port(s_leds[i].port);
ca_gpio_configure(s_leds[i].port, s_leds[i].pin, ...);
```

Registration occurs only once thanks to the internal guard, even if the board's LEDs are on three different ports:

```
static bool s_cli_registered = false;
if (!s_cli_registered) { ca_cli_register(&s_gpio_cli_node); s_cli_registered =
    true; }
```

A practical consequence follows: `EN_CLI_GPIO=1` alone **is not enough**. If an application calls neither `ca_gpio_init_port()` nor anything that does so on its behalf, the `gpio` commands are not registered and the console responds `huh!` even though the flag is enabled.

2. Task monitor

Single task. Toggles LD1 every second with `ca_led_toggle(1)` and prints the read-back status with `ca_led_get(1)`. The other LEDs remain off at startup and are controlled only via CLI.

3. Command `gpio info <P> <n>`

Reads the MODER, OTYPER, PUPDR, AFR, and IDR registers and decodes:

Field	Register	Bit/pin	Values
Mode	MODER	2 bits per pin	IN / OUT / ALT / ANALOG
Type	OTYPER	1 bit per pin	PP (push-pull) / OD (open-drain)
Pull	PUPDR	2 bits per pin	NOPULL / PU / PD
AF	AFR[0/1]	4 bits per pin	AF0..AF15 (only if ALT)
State	IDR	1 bit per pin	0 / 1

Example:

```
> gpio info C 7
GPIO PC7: OUT  PP  NOPULL  IDR=1

> gpio info A 9
GPIO PA9: ALT  PP  NOPULL  AF7  IDR=1
```

The second line is instructive: PA9 is the AF7 of USART1, which is the console itself. It is the quickest way to verify that a pin has actually been switched to the expected peripheral.

4. `gpio all <P>` command

It iterates over the 16 pins of the port and prints the decoding of each one, preceded by a line with raw MODER and IDR for quick reference:

```
> gpio all C
GPIO PC  (MODER=A8000280 IDR=0080)
  PC7: OUT    PP NOPULL =1
  ...
```

26.1.5 Expected output

At startup

The banner has no branches for boards: the number of LEDs is declared by the board.

```
EX3: LED and GPIO CLI - this board has 3 LED(s)
Commands: led info | led on 1 | led off 1 | led tog 1
          gpio info A 0 | gpio all A | gpio get A 0   (P=A..K, n=0..15)
```

The suggestions `gpio` name only port A and only read commands: which port drives an LED changes with the board and `ca_led_*` does not expose it, so a `set` suggested on a guessed pin could drive something real.

Periodic log (every 1 s)

```
EX3: tick=1000  LED1=1
EX3: tick=2000  LED1=0
EX3: tick=3000  LED1=1
```

Typical test sequence

```
> led info
--- LEDs (3) ---
  1: ON
  2: off
  3: off

> led on 2
led 2 ON

> gpio all A
GPIO PA  (MODER=... IDR=...)
  PA0: IN     PP NOPULL =0
  ...

> gpio info A 0
GPIO PA0: IN  PP  NOPULL  IDR=0
```

26.1.6 Required configuration in `os_config.h`

```
#define ENABLE_CLI    1
#define EN_LED        1
#define EN_CLI_LED    1
#define EN_CLI_GPIO   1
#define ENABLE_STDOUT 1
```

26.1.7 Summary

1. `ca_led_init()` — clock, pins, and "all off" from `CA_LED_TABLE`, without `#if`
2. `led info|on|off|tog <n>` — LEDs in board terms, portable
3. `gpio info <P> <n>` — complete decoding of a pin from HW registers
4. `gpio all <P>` — overview of the 16 port pins
5. `gpio get/set/tog` — raw real-time I/O, on any pin
6. **No local CLI hook** — both CLIs live in their respective drivers
7. **The GPIO CLI registers in `ca_gpio_init_port()`** — you get here via `ca_led_init()`, not with a direct call

Chapter 27

Queues: Inter-Task Communication

27.1 Example 4 — Code Queue: task-to-task communication via queues

27.1.1 Introduction

In the previous examples, we saw how to create tasks, how to manage dynamic memory, and how to protect access to shared resources using mutexes. However, one fundamental element for building complex systems is still missing: **task-to-task communication**.

So far, each task has worked in isolation, or at most shared global variables protected by mutexes. But in many real-world cases, a task must **send data** to another: a sensor that acquires data and sends it to a processing task, a task that receives commands from a serial port and forwards them to a control task, or a data producer feeding a consumer.

This example introduces caOS **queues**: special data structures that allow a task to **send messages** to another in a safe, synchronized, and conflict-free manner. A queue is like a box with two openings: on one side, **producers** insert messages, and on the other, **consumers** retrieve them in the order they arrived (FIFO: First In, First Out).

In this example, we have **two producers** and **two consumers** sharing a single queue. The producers generate messages containing an ID, a sequence number, and a calculated value; the consumers receive and print them. We will see how the queue automatically handles synchronization: if the queue is full, producers wait; if it is empty, consumers wait.

27.1.2 What is a queue?

A queue in caOS is a thread-safe data structure that enables communication between tasks:

- **Producer**: a task that **inserts** messages into the queue using `ca_queue_static_send`.
- **Consumer**: a task that **retrieves** messages from the queue using `ca_queue_static_receive`.
- **Circular buffer**: the queue uses a circular array of slots, where each slot contains a message.
- **Automatic synchronization**: if the queue is full, `send` waits (or fails if a timeout occurs); if it is empty, `receive` waits (or fails if a timeout occurs).
- **FIFO order**: messages exit in the same order they entered.

Queues are particularly useful for: - Smoothing out production spikes: a fast task produces data that a slow task consumes. - Decoupling different logic: the producer does not need to know who receives the data. - Handling asynchronous events: a task receives commands or notifications from other tasks or interrupts.

27.1.3 The source code

The file is located in `examples/example_4/ca_os_ex4.c`:

```c
#include "caos.h"

/* Message exchanged between producers and consumers */
typedef struct {
    u32 producer_id;
    u32 sequence;
    u32 value;
} Message;

/* Static queue: 8 slots of Message (12 bytes each) */
#define MSG_SIZE  12
CAQUEUE_STATIC_DEFINE(msg_queue, 8, MSG_SIZE);

static u32 running;

static u32 producer_1(void *arg) {
    (void)arg;
    u32 seq = 0;
    ca_stdout_printf_n("Producer 1 started");
    while(running) {
        Message msg = { .producer_id = 1, .sequence = seq, .value = seq * 10 };
        if (ca_queue_static_send(&msg_queue.q, &msg, 500)) {
            ca_stdout_printf_n("P1 sent: seq=%u val=%u", msg.sequence, msg.value
    );
            seq++;
        } else {
            ca_stdout_printf_n("P1: queue full (timeout)");
        }
        ca_os_sleep(300);
    }
    return 0;
}

static u32 producer_2(void *arg) {
    (void)arg;
    u32 seq = 0;
    ca_stdout_printf_n("Producer 2 started");
    while(running) {
        Message msg = { .producer_id = 2, .sequence = seq, .value = seq * 100 };
        if (ca_queue_static_send(&msg_queue.q, &msg, 500)) {
            ca_stdout_printf_n("P2 sent: seq=%u val=%u", msg.sequence, msg.value
    );
            seq++;
        } else {
            ca_stdout_printf_n("P2: queue full (timeout)");
        }
        ca_os_sleep(500);
    }
    return 0;
}

static u32 consumer_1(void *arg) {
    (void)arg;
    Message msg;
    ca_stdout_printf_n("Consumer 1 started");
    while(running) {
        if (ca_queue_static_receive(&msg_queue.q, &msg, 1000)) {
            ca_stdout_printf_n("C1 recv: from P%u seq=%u val=%u",
                               msg.producer_id, msg.sequence, msg.value);
        } else {
```

```
59              ca_stdout_printf_n("C1: queue empty (timeout)");
60          }
61      }
62      return 0;
63  }
64
65  static u32 consumer_2(void *arg) {
66      (void)arg;
67      Message msg;
68      ca_stdout_printf_n("Consumer 2 started");
69      while(running) {
70          if (ca_queue_static_receive(&msg_queue.q, &msg, 1000)) {
71              ca_stdout_printf_n("C2 recv: from P%u seq=%u val=%u",
72                                 msg.producer_id, msg.sequence, msg.value);
73          } else {
74              ca_stdout_printf_n("C2: queue empty (timeout)");
75          }
76      }
77      return 0;
78  }
79
80  void ca_os_main(void) {
81      running = 1;
82
83      ca_os_job("prod_1", CA_TH_LEVEL3, producer_1, NULL,
84                          STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);
85      ca_os_job("prod_2", CA_TH_LEVEL3, producer_2, NULL,
86                          STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);
87      ca_os_job("cons_1", CA_TH_LEVEL3, consumer_1, NULL,
88                          STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);
89      ca_os_job("cons_2", CA_TH_LEVEL3, consumer_2, NULL,
90                          STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);
91
92      ca_os_run(true);
93  }
```

27.1.4 Architecture

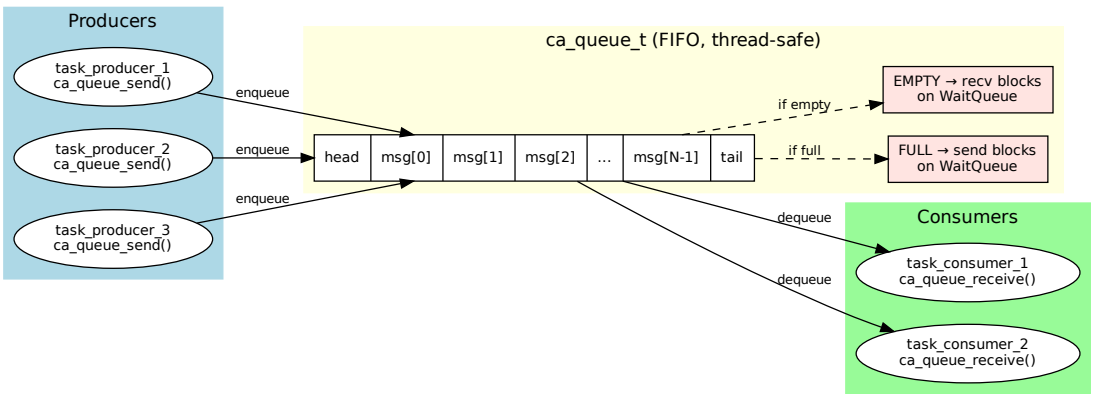


Figure 27.1: Message queue flow — producers and consumers

The diagram shows the flow of messages from the shared queue: the three producer tasks insert messages into the queue (`ca_queue_static_send`); the two consumers extract the messages (`ca_queue_static_receive`). If the queue is full, the producer blocks; if it is empty, the consumer blocks. Both wake up automatically when the state of the queue changes.

27.1.5 Detailed analysis

The message structure

```
typedef struct {
    u32 producer_id;
    u32 sequence;
    u32 value;
} Message;
```

This is the data structure that will be exchanged between producers and consumers:

- `producer_id`: identifies which producer generated the message (1 or 2).
- `sequence`: sequential message number (0, 1, 2, ...).
- `value`: value calculated by the producer (depends on the producer).

This structure represents a simple example of a useful payload: in a real system, it could contain sensor data, commands, events, processing results, etc.

Static queue definition

```
#define MSG_SIZE  12
CAQUEUE_STATIC_DEFINE(msg_queue, 8, MSG_SIZE);
```

Here we define a **static queue** (allocated in fixed RAM, not dynamic):

- `msg_queue`: name of the queue variable (it will be of type `ca_queue_static_t`).
- `8`: number of slots available in the queue (maximum capacity).
- `MSG_SIZE`: size in bytes of each message (12 bytes for the `Message` structure).

The `CAQUEUE_STATIC_DEFINE` macro automatically creates the queue data structure and the internal buffer to store the 12-byte messages. Being static, the memory is allocated at compile time and does not change during execution.

The control variable `running`

```
static u32 running;
```

As in the previous examples, this global variable controls the life of the tasks. As long as it is 1, all tasks continue to run in their `while` loop. When someone sets it to 0, all tasks will exit the loops and terminate.

The producers: `producer_1` and `producer_2`

```
static u32 producer_1(void *arg) {
    (void)arg;
    u32 seq = 0;
    ca_stdout_printf_n("Producer 1 started");
    while(running) {
        Message msg = { .producer_id = 1, .sequence = seq, .value = seq * 10 };
        if (ca_queue_static_send(&msg_queue.q, &msg, 500)) {
            ca_stdout_printf_n("P1 sent: seq=%u val=%u", msg.sequence, msg.value
    );
```

```
            seq++;
        } else {
            ca_stdout_printf_n("P1: queue full (timeout)");
        }
        ca_os_sleep(300);
    }
    return 0;
}
```

Let's analyze the operation of producer 1:

1. **Initialization**: starts with `seq = 0` and prints a startup message.
2. **Message creation**: at each iteration, it creates a new message with:
   - `producer_id = 1` (identifies the producer)
   - `sequence = seq` (progressive number)
   - `value = seq * 10` (calculated value)
3. **Message sending**: `ca_queue_static_send` attempts to insert the message into the queue:
   - **First parameter**: pointer to the queue (`&msg_queue`)
   - **Second parameter**: pointer to the message to be sent (`&msg`)
   - **Third parameter**: timeout in milliseconds (`500`)
   - Returns `true` if the sending is successful, `false` if the queue was full and the timeout expired
4. **Result handling**: if the sending is successful, it prints a message and increments `seq`; otherwise, it signals that the queue was full.
5. **Waiting**: `ca_os_sleep(300)` causes a 300 ms wait before the next message.

Producer 2 is identical, but: - It has `producer_id = 2` - It calculates `value = seq * 100` (grows more rapidly) - It waits 500 ms between one message and the next (less frequent)

This difference in frequency and value allows one to clearly see in the output which producer generated each message.

The consumers: `consumer_1` and `consumer_2`

```
static u32 consumer_1(void *arg) {
    (void)arg;
    Message msg;
    ca_stdout_printf_n("Consumer 1 started");
    while(running) {
        if (ca_queue_static_receive(&msg_queue.q, &msg, 1000)) {
            ca_stdout_printf_n("C1 recv: from P%u seq=%u val=%u",
                               msg.producer_id, msg.sequence, msg.value);
        } else {
            ca_stdout_printf_n("C1: queue empty (timeout)");
        }
    }
    return 0;
}
```

Consumers operate symmetrically to producers:

1. **Message declaration**: `Message msg` is used to receive the message.
2. **Message reception**: `ca_queue_static_receive` attempts to retrieve a message:
   - **First parameter**: pointer to the queue (`&msg_queue`)
   - **Second parameter**: pointer to the message where data should be copied (`&msg`)

- **Third parameter**: timeout in milliseconds (`1000`)
- Returns `true` if a message was received, `false` if the queue was empty and the timeout expired

3. **Result handling**: if a message is received, it prints its fields; otherwise, it signals that the queue was empty.
4. **Infinite loop**: continues as long as `running` is true.

Consumers do not have fixed waits: when the queue is full, they receive immediately; when it is empty, they wait up to 1 second for a new message.

Timeouts in queue operations

The functions `ca_queue_static_send` and `ca_queue_static_receive` accept a timeout parameter:

- `ca_queue_static_send(queue, msg, 500)`: if the queue is full, it waits for a maximum of 500 ms. If a slot becomes free within that time, it inserts the message and returns `true`. Otherwise, it returns `false`.
- `ca_queue_static_receive(queue, msg, 1000)`: if the queue is empty, it waits for a maximum of 1000 ms. If a message arrives within that time, it receives it and returns `true`. Otherwise, it returns `false`.

These timeouts are important because: - They prevent a task from blocking indefinitely if the queue is full or empty - They allow handling anomalous situations (for example, if a producer is much faster than all consumers) - They allow performing other operations while waiting (even if we do not do so in this example)

Task registration and startup

```
void ca_os_main(void) {
    running = 1;

    ca_os_job("prod_1", CA_TH_LEVEL3, producer_1, NULL,
                        STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);
    ca_os_job("prod_2", CA_TH_LEVEL3, producer_2, NULL,
                        STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);
    ca_os_job("cons_1", CA_TH_LEVEL3, consumer_1, NULL,
                        STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);
    ca_os_job("cons_2", CA_TH_LEVEL3, consumer_2, NULL,
                        STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);

    ca_os_run(true);
}
```

Here, 4 tasks are created: - Two producers (1 and 2) with different frequencies - Two consumers (1 and 2) that receive messages in FIFO order - All have the same priority (`CA_TH_LEVEL3`) and stack size (`STACK_MEDIUM`)

27.1.6 Typical execution scenario

With this configuration, an interesting communication scenario occurs:

1. **The two producers** start sending messages simultaneously:

   - Producer 1: every 300 ms, with `value = seq * 10`
   - Producer 2: every 500 ms, with `value = seq * 100`

2. **The queue** accumulates messages up to a maximum of 8 slots.
3. **The two consumers** retrieve messages in FIFO order:
   - Whoever reaches the queue first receives the oldest message
   - If the queue is empty, they wait for up to 1 second
4. **Result**: in the output, you will see messages coming from both producers, received by both consumers, in the order in which they were sent.

27.1.7 What this example teaches us

1. **Queues facilitate communication between tasks**: there is no need to share variables or use complex mutexes. Simply send and receive messages.
2. **Synchronization is automatic**: the queue takes care of managing concurrency between producers and consumers.
3. **Timeouts prevent indefinite blocking**: if the queue is full or empty, tasks can handle the situation instead of hanging.
4. **Data maintains integrity and order**: thanks to the FIFO mechanism, messages arrive in the same order they were sent.
5. **Producer-consumer architectures**: this pattern is very common in real-time systems and allows for decoupling different logic.

27.1.8 Example output

Below is the output produced by the example on the debug serial port:

```
================================================
 -- c.a.O.S --
   build : 51ada1e4+dev - 2026-02-14 18:46:10
   MCU : STM 32 F:H7xx T:753
================================================
   System Clock: 480000000 Hz
   SysTick: 47999 clocks per tick
   Debug UART3 Baudrate: 4000000 bps
================================================
Producer 1 started
P1 sent: seq=0 val=0
Producer 2 started
P2 sent: seq=0 val=0
Consumer 2 started
C2 recv: from P1 seq=0 val=0
C2 recv: from P2 seq=0 val=0
Consumer 1 started
P1 sent: seq=1 val=10
C2 recv: from P1 seq=1 val=10
P2 sent: seq=1 val=100
C1 recv: from P2 seq=1 val=100
P1 sent: seq=2 val=20
C2 recv: from P1 seq=2 val=20
P1 sent: seq=3 val=30
C1 recv: from P1 seq=3 val=30
P2 sent: seq=2 val=200
```

Chapter 28

Recursive Mutex: Handling Recursion in Multitasking

28.1 Example 5 — Recursive mutex: managing recursion in multitasking

28.1.1 Introduction

In the previous examples, we saw how standard mutexes protect access to shared resources between tasks. However, there is a specific scenario where a standard mutex is not sufficient: when a task that **already owns a mutex** attempts to acquire it again, for example due to a recursive call or a nested function.

In a standard mutex, this secondary acquisition attempt by the same task would cause a **deadlock**: the task would block itself, because it would wait for the mutex to be released, but only the task itself can do so, and it will not be able to do so until it finishes the function it is waiting for.

The **recursive mutex** solves this problem by allowing a task that is already an owner to acquire the mutex again. Each acquisition increments an **internal counter**, and only when the number of releases equals the number of acquisitions does the mutex actually become free for other tasks.

28.1.2 What is a recursive mutex?

A recursive mutex (or reentrant mutex) is an advanced version of the standard mutex that:

- Maintains an **acquisition count** (lock count)
- Tracks which task is the **current owner**
- Allows the owner to acquire the mutex additional times
- Requires an equal number of releases to actually free the mutex
- Prevents deadlock in case of multiple acquisitions by the same task

28.1.3 The source code

The file is located in `examples/example_5/ca_os_ex5.c`:

```
#include "caos.h"

static u32 running;
static ca_recursive_mutex_t rmtx;

// Recursive function that uses the recursive mutex
static void recursive_function(const char* task_name, int depth) {
```

```
8       // Try to take the recursive mutex
9       ca_rmutex_lock(&rmtx);
10
11      // Print the recursion level and the task name
12      ca_stdout_printf_n("%s: entered recursion level %d, lock count: %u",
13                         task_name, depth, ca_rmutex_get_count(&rmtx));
14
15      // If we are at a given recursion level, recurse
16      if (depth > 0) {
17          recursive_function(task_name, depth - 1);
18      }
19
20      // Print when leaving the recursion level
21      ca_stdout_printf_n("%s: exiting recursion level %d, lock count: %u",
22                         task_name, depth, ca_rmutex_get_count(&rmtx));
23
24      // Release the mutex (note: since it is recursive,
25      // we must release as many times as the locks taken)
26      ca_rmutex_unlock(&rmtx);
27  }
28
29  static u32 task_1_exec(void *arg) {
30      const u32 p1 = (u32)(uintptr_t)arg;
31      const u32 idx = ca_os_thid_from_handle(ca_os_self_handle());
32      ca_stdout_printf_n("TASK recursive_1 started: idx=%u p1=%u", idx, p1);
33
34      while(running) {
35          // Take the recursive mutex
36          ca_rmutex_lock(&rmtx);
37          ca_stdout_printf_n("TASK recursive_1 acquired mutex, lock count: %u",
38                             ca_rmutex_get_count(&rmtx));
39
40          // Call the recursive function with a given level
41          recursive_function("TASK recursive_1", 2);
42
43          ca_stdout_printf_n("TASK recursive_1 about to release mutex, lock count:
        %u",
44                             ca_rmutex_get_count(&rmtx));
45
46          // Release the recursive mutex (it is actually released
47          // only when the lock count goes back to 0)
48          ca_rmutex_unlock(&rmtx);
49
50          ca_os_sleep(1000);
51      }
52      return 0;
53  }
54
55  static u32 task_2_exec(void *arg) {
56      const u32 p1 = (u32)(uintptr_t)arg;
57      const u32 idx = ca_os_thid_from_handle(ca_os_self_handle());
58      ca_stdout_printf_n("TASK recursive_2 started: idx=%u p1=%u", idx, p1);
59
60      while(running) {
61          // Short pause to let the first task get the mutex
62          ca_os_sleep(300);
63
64          // Try to take the recursive mutex
65          if (ca_rmutex_trylock(&rmtx)) {
66              ca_stdout_printf_n("TASK recursive_2 acquired mutex, lock count: %u"
        ,
67                                 ca_rmutex_get_count(&rmtx));
68
```

```
69              // Simulate some work
70              ca_stdout_printf_n("TASK recursive_2 doing some work...");
71
72              // Release the recursive mutex
73              ca_rmutex_unlock(&rmtx);
74          } else {
75              ca_stdout_printf_n("TASK recursive_2: mutex busy (owner may be in
    recursion)");
76          }
77
78          ca_os_sleep(800);
79      }
80      return 0;
81  }
82
83  void ca_os_main(void) {
84      running = 1;
85
86      // Initialize the recursive mutex
87      ca_rmutex_init(&rmtx);
88
89      // Crea i task
90      ca_os_job("rec_1", CA_TH_LEVEL2, task_1_exec, (void *)(uintptr_t)1,
91                        STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);
92      ca_os_job("rec_2", CA_TH_LEVEL2, task_2_exec, (void *)(uintptr_t)3,
93                        STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);
94
95      ca_os_run(true);
96  }
```

28.1.4 Architecture

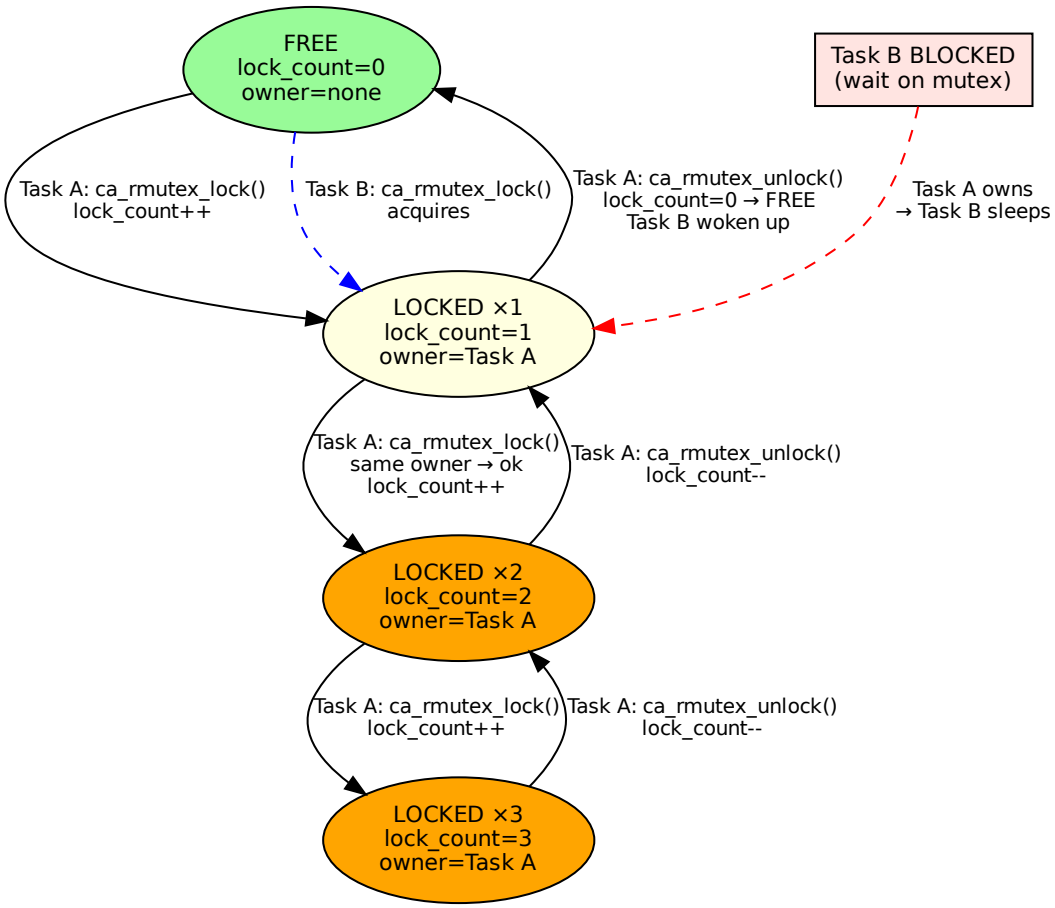


Figure 28.1: Recursive mutex states — lock count transitions

The diagram shows the state transitions of the recursive mutex. Unlike the simple mutex, the same task can acquire the lock multiple times: each call to `ca_rmutex_lock` increments the counter; only when the counter returns to zero is the mutex free and another task can acquire it. A different task that calls `ca_rmutex_lock` when the mutex is owned blocks until it is fully released.

28.1.5 Detailed analysis

Shared variables

```
1 static u32 running;
2 static ca_recursive_mutex_t rmtx;
```

- `running`: control variable to keep tasks active.
- `rmtx`: the recursive mutex shared between tasks.

Recursive mutex initialization

```
ca_rmutex_init(&rmtx);
```

As with a standard mutex, the recursive mutex must be initialized before use with the dedicated function `ca_rmutex_init`.

Recursive function

```
static void recursive_function(const char* task_name, int depth) {
    ca_rmutex_lock(&rmtx);
    ca_stdout_printf_n("%s: entered recursion level %d, lock count: %u",
                       task_name, depth, ca_rmutex_get_count(&rmtx));

    if (depth > 0) {
        recursive_function(task_name, depth - 1);
    }

    ca_stdout_printf_n("%s: exiting recursion level %d, lock count: %u",
                       task_name, depth, ca_rmutex_get_count(&rmtx));

    ca_rmutex_unlock(&rmtx);
}
```

This function demonstrates the core concept of a recursive mutex:

1. **Acquisition**: Each call to `ca_rmutex_lock` increments the internal count, even if the task is already the owner.
2. **Counting**: The `ca_rmutex_get_count` function shows how many locks are currently active.
3. **Recursion**: The function calls itself recursively, further increasing the count.
4. **Release**: Each `ca_rmutex_unlock` decrements the count, but the mutex remains occupied until the count returns to zero.

Main task (recursive_1)

The first task demonstrates the typical use of a recursive mutex:

1. Acquires the mutex with `ca_rmutex_lock`.
2. Calls the recursive function which acquires the mutex again.
3. Releases the mutex with `ca_rmutex_unlock`.

Thanks to the recursive mutex, the task does not deadlock even if it acquires the mutex multiple times.

Second task (recursive_2)

The second task shows how the recursive mutex behaves with respect to other tasks:

1. Uses `ca_rmutex_trylock` to try to acquire the mutex without blocking.
2. If the mutex is occupied (by the first task in recursion), it prints an appropriate message.
3. If it succeeds in acquiring it, it performs some work and then releases it.

Key differences from a standard mutex

- **Recursion allowed**: A task can acquire a recursive mutex multiple times without deadlock.
- **Internal counting**: The mutex keeps track of how many times it has been acquired by the same task.
- **Balanced release**: An equal number of releases is required to actually free the mutex.

28.1.6 When to use recursive mutexes

Recursive mutexes are useful in these scenarios:

1. **Recursive functions**: When a function that acquires a mutex can call itself.
2. **Nested calls**: When a function protected by a mutex calls other functions that are also protected by the same mutex.
3. **Legacy code**: When it is not possible to restructure the code to avoid multiple acquisitions.

28.1.7 Beware of potential issues

- **More complex debugging**: Since the internal count is hidden, it is more difficult to diagnose unbalanced acquisition/release issues.
- **Performance**: Recursive mutexes have a slightly higher overhead compared to standard mutexes.
- **Better design**: Often, good code design can avoid the need for recursive mutexes.

28.1.8 Important considerations for the embedded world

As indicated in the `hw/h7xx/h753/objmap.h` file, the STM32H753 microcontroller used in the Nucleo-H753ZI board has limited resources:

- **Limited stack**: Although the average stack is 8KB (`STACK_MEDIUM = 8192` bytes) and the large one is 16KB (`STACK_LARGE = 16384` bytes), recursion can quickly consume this memory.
- **Defined stack size**: As can be seen from `objmap.h`, the stack sizes are predefined and limited:

```
#define STACK_MIN       2048
typedef enum {
    STACK_SMALL  = (STACK_MIN*2),       /**< 4096 byte */
    STACK_MEDIUM = (STACK_SMALL*2),     /**< 8192 byte */
    STACK_LARGE  = (STACK_MEDIUM*2),    /**< 16384 byte */
    STACK_XLARGE = (STACK_LARGE*2),     /**< 32768 byte */
    STACK_KERNEL = (STACK_LARGE*2)      /**< 32768 byte */
} ca_stack_size_t;
```

  In the embedded world, **the use of recursion is strongly discouraged** for the following reasons:

1. **Unpredictable stack consumption**: Each recursive call adds a new frame to the stack, and in resource-constrained systems, this can very easily lead to a stack overflow.
2. **Debugging difficulty**: Stack overflow errors in embedded systems can be very difficult to diagnose.
3. **Unpredictable execution times**: The depth of recursion can vary based on input data, making it difficult to guarantee deterministic execution times.
4. **Inefficient memory usage**: Recursion requires additional stack space for each call, whereas iterative solutions are generally more efficient.

For these reasons, whenever possible, it is always better to replace recursion with an iterative solution, even if it requires restructuring the code.

28.1.9 What this example teaches us

1. **The difference between standard and recursive mutexes**: How the recursive mutex allows multiple acquisitions by the same task.
2. **Internal counting**: How the mutex keeps track of how many times it has been acquired.
3. **Deadlock prevention**: How the recursive mutex avoids deadlock in recursion scenarios.
4. **Behavior towards other tasks**: How the mutex remains occupied for other tasks as long as the count has not returned to zero.

28.1.10 Example output

Below is the output produced by the example on the debug serial port:

```
================================================
 -- c.a.O.S --
   build : 77af553c+dev - 2026-02-15 20:02:40
   MCU : STM 32 F:H7xx T:753
================================================
   System Clock: 480000000 Hz
   SysTick: 47999 clocks per tick
   Debug UART3 Baudrate: 4000000 bps
================================================
TASK recursive_1 started: idx=3 p1=1
TASK recursive_1 acquired mutex, lock count: 1
TASK recursive_2 started: idx=4 p1=3
TASK recursive_1: entered recursion level 2, lock count: 2
TASK recursive_1: entered recursion level 1, lock count: 3
TASK recursive_1: entered recursion level 0, lock count: 4
TASK recursive_1: exiting recursion level 0, lock count: 4
TASK recursive_1: exiting recursion level 1, lock count: 3
TASK recursive_1: exiting recursion level 2, lock count: 2
TASK recursive_1 about to release mutex, lock count: 1
TASK recursive_2 acquired mutex, lock count: 1
TASK recursive_2 doing some work...
```

28.1.11 Verification of waits with timeout (`twait`)

Since 09/27, a task waiting for a recursive mutex **with timeout** (`ca_rmutex_timedlock()`) is added to the same wait list as those waiting without a timeout, parked in `SLEEP` until expiration, and the signaler wakes it up immediately (S2_02 §8.8). Previously, it performed polling every 10 ms outside the list: it would wake up up to 10 ms after the signal.

The command `twait [sec]` (default 3 s) measures it with a peer task created at the first call. The wait runs in the command task:

1. **latency** — 300 cycles: the peer holds the mutex and releases it after a random 1..8 ms; the waiter, with a 20 ms timeout, must obtain it (`lost=0`), not before the signal (`spur=0`), and be running within 2 ms (`lat_max` limit, in ticks);
2. **timeout** — the peer holds the mutex for the entire test; for `sec` seconds, T = 1..20 ms: the return never arrives **before** T (`early=0`), at most 2 ms later (`late=0`), and always with `false` (`got=0`).

One line, terminated by `PASS` or `FAIL`. On H753 (tick 200 µs):

```
TW rmutex : rounds=300 lat_max=0/10 lost=0 spur=0 | tmo n=291 early=0 late=0 got
   =0 over_max=0/10  PASS
```

On the previous kernel: FAIL, `lat_max=50/10`: the 10 ms polling.

In the bank is case `timed_wait` of `test_ex05.cpp` (helper `test/hw_test/inc/caosTimedWait.h`). Manually:

```
caos_shell -e 5 -q -t 40 -u "TW .*(PASS|FAIL)" -g "^TW" "twait 3"
```

Chapter 29

Semaphore: Controlling Access to Shared Resources

29.1 Example 6 — Semaphore: controlling access to shared resources

29.1.1 Introduction

In the previous examples, we saw how mutexes protect exclusive access to shared resources between tasks. However, there is a different scenario where we have **a pool of identical resources** and we want to limit the number of tasks that can access these resources simultaneously. In this case, mutexes are not the ideal solution, because they allow access to only one instance of the resource.

The **semaphore** is the right synchronization primitive for this scenario. Unlike the mutex, which is binary (it can only be free or occupied), a semaphore maintains a **count** of available resources. When a task requests a resource, the count decreases; when it releases it, the count increases. If the count is zero, subsequent tasks are blocked while waiting.

In this example, we simulate a shared resource control system using a semaphore to limit the number of tasks that can simultaneously access a pool of resources.

29.1.2 What is a semaphore?

A semaphore is a synchronization primitive that:

- Maintains a **count of available resources**
- Allows multiple tasks to access simultaneously up to a maximum limit
- Blocks tasks when all resources are occupied
- Unblocks tasks when a resource becomes available

Semaphores are particularly useful for:

1. **Traffic control**: Limiting the number of tasks that can access a resource
2. **Resource pools**: Managing a limited set of identical resources
3. **Producer-consumer synchronization**: Coordinating tasks that produce and consume data

29.1.3 The source code

The file is located in `examples/example_6/ca_os_ex6.c`:

```c
#include "caos.h"

/*
 * Example of semaphore usage
 *
 * In this example we simulate a system that controls shared resources
 * using semaphores. We have a pool of resources (for example, database connections,
 * accesses to a hardware device, etc.) and several tasks that request them.
 *
 * The semaphore is used to limit the number of tasks that can access
 * the shared resources at the same time.
 */

#define NUM_RESOURCES 3  // Number of available resources
#define NUM_PRODUCERS 2  // Number of producer tasks
#define NUM_CONSUMERS 4  // Number of consumer tasks

static u32 running;
static ca_semaphore_t resource_semaphore;  // Semaphore to control access to the resources

// Array to track the state of the resources
static u32 resource_status[NUM_RESOURCES];
static ca_mutex_t resource_mutex;  // Mutex to protect access to the array

static u32 producer_task(void *arg) {
    const u32 p1 = (u32)(uintptr_t)arg;
    const u32 idx = ca_os_thid_from_handle(ca_os_self_handle());
    char task_name[32];
    snprintf(task_name, sizeof(task_name), "PRODUCER_%u", idx);

    ca_stdout_printf_n("%s started: idx=%u p1=%u", task_name, idx, p1);

    while(running) {
        // Wait a bit before requesting a resource
        ca_os_sleep(500 + (idx * 200));  // Different delay for each producer

        // Try to take a resource (waiting at most 1000ms)
        if (ca_sem_timedwait(&resource_semaphore, 1000)) {
            // We got access to a resource
            ca_stdout_printf_n("%s: acquired resource", task_name);

            // Look for a free resource to use
            u32 resource_idx = (u32)-1;
            ca_mutex_lock(&resource_mutex);
            for (int i = 0; i < NUM_RESOURCES; i++) {
                if (resource_status[i] == 0) {  // Resource free
                    resource_status[i] = idx + 1;  // Assign it to this producer
                    resource_idx = i;
                    break;
                }
            }
            ca_mutex_unlock(&resource_mutex);

            if (resource_idx != (u32)-1) {
                // Simulate using the resource
                ca_stdout_printf_n("%s: using resource %u", task_name, resource_idx);

                // Use the resource for a while
                ca_os_sleep(800);

```

```
61                  // Free the resource
62                  ca_mutex_lock(&resource_mutex);
63                  resource_status[resource_idx] = 0;  // Mark as free
64                  ca_mutex_unlock(&resource_mutex);
65
66                  ca_stdout_printf_n("%s: released resource %u", task_name,
    resource_idx);
67              } else {
68                  ca_stdout_printf_n("%s: ERROR - no free resource found!",
    task_name);
69              }
70
71              // Release the semaphore to let other tasks access it
72              ca_sem_post(&resource_semaphore);
73          } else {
74              // Timeout: we could not get a resource within the time limit
75              ca_stdout_printf_n("%s: timeout waiting for resource", task_name);
76          }
77      }
78
79      return 0;
80  }
81
82  static u32 consumer_task(void *arg) {
83      const u32 p1 = (u32)(uintptr_t)arg;
84      const u32 idx = ca_os_thid_from_handle(ca_os_self_handle());
85      char task_name[32];
86      snprintf(task_name, sizeof(task_name), "CONSUMER_%u", idx);
87
88      ca_stdout_printf_n("%s started: idx=%u p1=%u", task_name, idx, p1);
89
90      while(running) {
91          // Wait a bit before checking the state
92          ca_os_sleep(700 + (idx * 100));
93
94          // Check the state of the resources
95          ca_mutex_lock(&resource_mutex);
96          u32 busy_resources = 0;
97          for (int i = 0; i < NUM_RESOURCES; i++) {
98              if (resource_status[i] != 0) {
99                  busy_resources++;
100             }
101         }
102         ca_mutex_unlock(&resource_mutex);
103
104         ca_stdout_printf_n("%s: %u/%u resources busy", task_name, busy_resources
    , NUM_RESOURCES);
105
106         // Check the semaphore value
107         u32 sem_value = ca_sem_getvalue(&resource_semaphore);
108         ca_stdout_printf_n("%s: semaphore value: %u", task_name, sem_value);
109     }
110
111     return 0;
112 }
113
114 void ca_os_main(void) {
115     running = 1;
116
117     // Initialize the semaphore with NUM_RESOURCES available
118     ca_sem_init(&resource_semaphore, NUM_RESOURCES, NUM_RESOURCES);
119
120     // Initialize the mutex that protects access to the resource array
```

```
121     ca_mutex_init(&resource_mutex);
122 
123     // Initialize the state of the resources
124     for (int i = 0; i < NUM_RESOURCES; i++) {
125         resource_status[i] = 0;  // All resources are free at the start
126     }
127 
128     // Creiamo i task produttori
129     for (int i = 0; i < NUM_PRODUCERS; i++) {
130         char task_name[16];
131         snprintf(task_name, sizeof(task_name), "prod_%d", i+1);
132         ca_os_job(task_name, CA_TH_LEVEL2, producer_task, (void *)(uintptr_t)i,
133                           STACK_LARGE, CA_JOB_STD, CA_FAULT_ASSERT);
134     }
135 
136     // Creiamo i task consumatori
137     for (int i = 0; i < NUM_CONSUMERS; i++) {
138         char task_name[16];
139         snprintf(task_name, sizeof(task_name), "cons_%d", i+1);
140         ca_os_job(task_name, CA_TH_LEVEL2, consumer_task, (void *)(uintptr_t)i,
141                           STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);
142     }
143 
144     ca_os_run(true);
145 }
```

29.1.4 Architecture

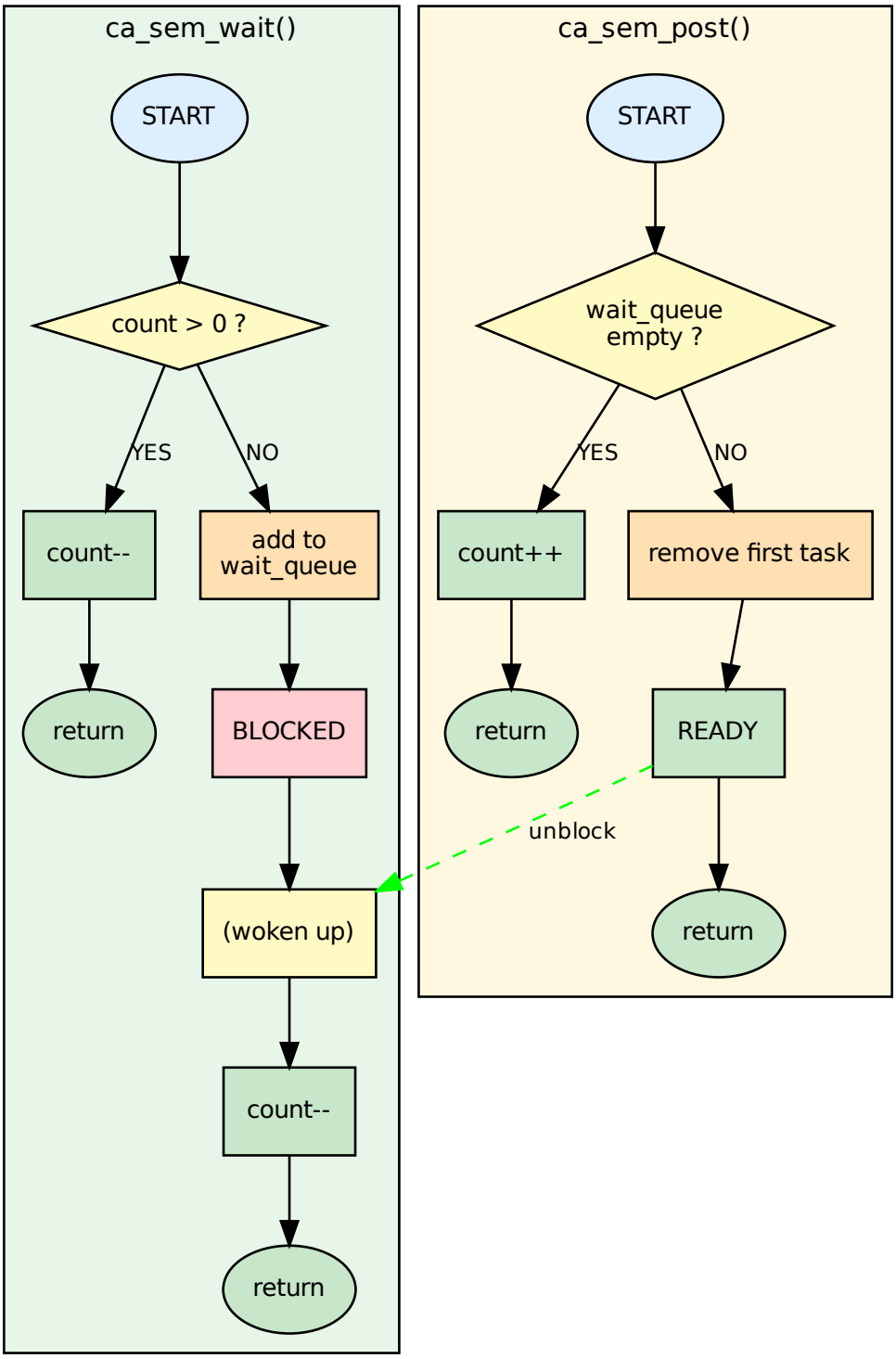


Figure 29.1: Semaphore flow — shared resource access control

The diagram shows the behavior of the counting semaphore: the initial value represents the available resources. Each `ca_sem_wait` decrements the counter; if the value is already zero, the task blocks. Each `ca_sem_post` increments the counter and wakes up a waiting task. In this example, the semaphore controls access to a pool of `NUM_RESOURCES=3` resources shared between producers and consumers.

29.1.5 Detailed analysis

Configuration constants

```
#define NUM_RESOURCES 3  // Number of available resources
#define NUM_PRODUCERS 2  // Number of producer tasks
#define NUM_CONSUMERS 4  // Number of consumer tasks
```

These constants define the system configuration: - **NUM_RESOURCES**: The total number of resources in the pool (3 in this case) - **NUM_PRODUCERS**: The number of tasks that will request access to the resources - **NUM_CONSUMERS**: The number of tasks that will monitor the system status

Shared variables

```
static u32 running;
static ca_semaphore_t resource_semaphore;  // Semaphore to control access to the
    resources

// Array to track the state of the resources
static u32 resource_status[NUM_RESOURCES];
static ca_mutex_t resource_mutex;  // Mutex to protect access to the array
```

- `running`: Control variable to keep tasks active.
- `resource_semaphore`: The semaphore that controls access to resources.
- `resource_status`: Array that tracks the status of each resource.
- `resource_mutex`: Mutex to protect access to the shared resource array.

Initialization

```
void ca_os_main(void) {
    running = 1;

    // Initialize the semaphore with NUM_RESOURCES available
    ca_sem_init(&resource_semaphore, NUM_RESOURCES, NUM_RESOURCES);

    // Initialize the mutex that protects access to the resource array
    ca_mutex_init(&resource_mutex);

    // Initialize the state of the resources
    for (int i = 0; i < NUM_RESOURCES; i++) {
        resource_status[i] = 0;  // All resources are free at the start
    }
    ...
}
```

The semaphore is initialized with an initial value equal to the number of available resources. This means that initially, all resources are available for use.

Producer task

```
static u32 producer_task(void *arg) {
    const u32 idx = ca_os_thid_from_handle(ca_os_self_handle());
    (void)arg;
    ...
    if (ca_sem_timedwait(&resource_semaphore, 1000)) {
        // We got access to a resource
        ca_stdout_printf_n("%s: acquired resource", task_name);

        // Look for a free resource to use
        u32 resource_idx = (u32)-1;
        ca_mutex_lock(&resource_mutex);
        for (int i = 0; i < NUM_RESOURCES; i++) {
            if (resource_status[i] == 0) {  // Resource free
                resource_status[i] = idx + 1;  // Assign it to this producer
                resource_idx = i;
                break;
            }
        }
        ca_mutex_unlock(&resource_mutex);

        if (resource_idx != (u32)-1) {
            // Simulate using the resource
            ca_stdout_printf_n("%s: using resource %u", task_name, resource_idx)
    ;

            // Use the resource for a while
            ca_os_sleep(800);

            // Free the resource
            ca_mutex_lock(&resource_mutex);
            resource_status[resource_idx] = 0;  // Mark as free
            ca_mutex_unlock(&resource_mutex);

            ca_stdout_printf_n("%s: released resource %u", task_name,
    resource_idx);
        }

        // Release the semaphore to let other tasks access it
        ca_sem_post(&resource_semaphore);
    } else {
        // Timeout: we could not get a resource within the time limit
        ca_stdout_printf_n("%s: timeout waiting for resource", task_name);
    }
    ...
}
```

The producer task:

1. **Requests a resource** using `ca_sem_timedwait` with a timeout of 1000ms
2. **If it succeeds in obtaining the resource**, it searches for an actually free resource in the array
3. **Uses the resource** for a period of time (simulated with `ca_os_sleep`)
4. **Releases the resource** by updating the state and incrementing the semaphore count with `ca_sem_post`

Semaphore operations

- `ca_sem_timedwait(&resource_semaphore, 1000)`: Decrements the semaphore count. If the count is already zero, the task waits up to 1000ms for an available resource.

- `ca_sem_post(&resource_semaphore)`: Increments the semaphore count, signaling that a resource has been released and is now available for other tasks.

Consumer task

Consumer tasks monitor the system state:

1. **Check the status of resources** in the `resource_status` array
2. **Read the semaphore value** using `ca_sem_getvalue`
3. **Print information** about the system state

Key differences from the mutex

- **Multiple count**: The semaphore maintains a count of available resources, while the mutex is binary (free/busy).
- **Multiple access**: Multiple tasks can obtain the semaphore simultaneously up to the set limit, while only one task at a time can obtain the mutex.
- **Different purpose**: Semaphores are used to control access to a pool of resources, mutexes to protect critical sections.

29.1.6 When to use semaphores

Semaphores are useful in these scenarios:

1. **Resource pool**: When you have a limited number of identical resources to share among tasks.
2. **Traffic throttling**: When you want to limit the number of tasks that can perform a certain operation simultaneously.
3. **Producer-consumer synchronization**: To coordinate tasks that produce and consume data through a limited buffer.

29.1.7 What this example teaches us

1. **Semaphore operation**: How the count controls access to resources.
2. **The difference from the mutex**: How semaphores allow multiple accesses up to a limit.
3. **Managing a resource pool**: How to limit access to a fixed number of shared resources.
4. **Synchronization operations**: How to use `timedwait`, `post` and `getvalue` to control and monitor the semaphore.

29.1.8 Example output

Below is the output produced by the example on the debug serial port:

```
================================================
 -- c.a.O.S --
   build : 77af553c+dev - 2026-02-15 20:27:24
   MCU : STM 32 F:H7xx T:753
================================================
   System Clock: 480000000 Hz
   SysTick: 47999 clocks per tick
   Debug UART3 Baudrate: 4000000 bps
================================================
PRODUCER_3 started: idx=3 p1=0
PRODUCER_4 started: idx=4 p1=1
CONSUMER_6 started: idx=6 p1=1
CONSUMER_7 started: idx=7 p1=2
CONSUMER_8 started: idx=8 p1=3
```

```
CONSUMER_5 started: idx=5 p1=0
PRODUCER_3: acquired resource
PRODUCER_3: using resource 0
CONSUMER_5: 1/3 resources busy
CONSUMER_5: semaphore value: 2
PRODUCER_4: acquired resource
PRODUCER_4: using resource 1
CONSUMER_6: 2/3 resources busy
CONSUMER_6: semaphore value: 1
CONSUMER_7: 2/3 resources busy
CONSUMER_7: semaphore value: 1
CONSUMER_8: 2/3 resources busy
CONSUMER_8: semaphore value: 1
PRODUCER_3: released resource 0
PRODUCER_4: released resource 1
CONSUMER_5: 0/3 resources busy
```

29.1.9 Verification of waits with timeout (`twait`)

Since 09/27, a task waiting for a semaphore **with timeout** (`ca_sem_timedwait()`) registers in the same wait list as those waiting without a timeout, parked in `SLEEP` until expiration, and the signaler wakes it up immediately (S2_02 §8.8). Previously, it performed polling every 10 ms outside the list: it would wake up up to 10 ms after the signal.

The command `twait [sec]` (default 3 s) measures it with a peer task created at the first call. The wait runs in the command task:

1. **latency** — 300 loops: the peer performs `ca_sem_post()` after a random 1..8 ms; the waiter, with a 20 ms timeout, must obtain it (`lost=0`), not before the signal (`spur=0`), and be running within 2 ms (`lat_max` limit, in ticks);
2. **timeout** — no one performs `post`; for `sec` seconds, T = 1..20 ms: the return never arrives **before** T (`early=0`), at most 2 ms after (`late=0`), and always with `false` (`got=0`).

One line, closed by `PASS` or `FAIL`. On H753 (tick 200 µs):

```
TW sem : rounds=300 lat_max=0/10 lost=0 spur=0 | tmo n=291 early=0 late=0 got=0
    over_max=0/10  PASS
```

On the previous kernel: FAIL, `lat_max=45/10`.

In the bank, it is case `timed_wait` of `test_ex06.cpp` (helper `test/hw_test/inc/caosTimedWait.h`). Manually:

```
caos_shell -e 6 -q -t 40 -u "TW .*(PASS|FAIL)" -g "^TW" "twait 3"
```

Chapter 30

Condition Variable: Producer-Consumer Coordination

30.1 Example 7 — Condition Variable: producer-consumer coordination

30.1.1 Introduction

In the previous examples, we saw how mutexes protect access to shared resources and how semaphores control access to a pool of resources. However, there is an even more complex scenario where a task must **wait for a specific condition** before proceeding.

For example, in a producer-consumer system, the consumer must wait for the buffer to be non-empty before extracting data, while the producer must wait for the buffer to be non-full before inserting data. Simple mutexes are not sufficient for this type of coordination: **condition variables** are needed.

A condition variable allows a task to suspend itself until a certain condition becomes true. When another task modifies the condition, it can wake up the waiting tasks. This mechanism is particularly useful for:

1. **Producer-consumer coordination**: The consumer waits for the buffer to be non-empty, the producer waits for it to be non-full
2. **Event notification**: A task waits for another to perform an action
3. **Complex synchronization**: When simple mutual exclusion is not enough

30.1.2 What is a condition variable?

A condition variable is a synchronization primitive that:

- Allows a task to **wait for a condition** while temporarily releasing a mutex
- Allows other tasks to **signal** when the condition is met
- Supports both **single signals** (`signal`) and **broadcasts** (`broadcast`)
- Can include **timeouts** to avoid infinite waits

Condition variables are always used **together with a mutex**. The typical sequence is:

1. Acquire the mutex
2. Check the condition
3. If the condition is not met, call `ca_cond_wait` (releases the mutex and waits)
4. When it wakes up, re-check the condition
5. Perform the required work
6. Release the mutex

30.1.3 The source code

The file is located in `examples/example_7/ca_os_ex7.c`:

```c
#include "caos.h"

/*
 * Example of condition variable usage
 *
 * In this example we simulate a producer-consumer system that uses
 * a condition variable to coordinate access to a shared resource.
 * We have a shared buffer and two tasks: a producer that puts data
 * into the buffer and a consumer that takes them out. The condition variable
    lets
 * the consumer be suspended when the buffer is empty and the producer
 * when the buffer is full.
 */

#define BUFFER_SIZE 5
static u32 buffer[BUFFER_SIZE];
static u32 head = 0;  // Head index (next item to read)
static u32 tail = 0;  // Tail index (next item to write)
static u32 count = 0; // Number of items in the buffer

static u32 running;
static ca_mutex_t buffer_mutex;      // Mutex per proteggere l'accesso al buffer
static ca_condvar_t buffer_not_full;  // Condition variable for buffer not full
static ca_condvar_t buffer_not_empty; // Condition variable for buffer not empty

// Function to check whether the buffer is full
static bool is_buffer_full(void) {
    return count == BUFFER_SIZE;
}

// Function to check whether the buffer is empty
static bool is_buffer_empty(void) {
    return count == 0;
}

// Function to put an item into the buffer
static void put_item(u32 item) {
    if (is_buffer_full()) {
        return; // Buffer full
    }
    buffer[tail] = item;
    tail = (tail + 1) % BUFFER_SIZE;
    count++;
}

// Function to take an item out of the buffer
static u32 get_item(void) {
    if (is_buffer_empty()) {
        return 0; // Buffer empty
    }
    u32 item = buffer[head];
    head = (head + 1) % BUFFER_SIZE;
    count--;
    return item;
}

static u32 producer_task(void *arg) {
    const u32 p1 = (u32)(uintptr_t)arg;
    const u32 idx = ca_os_thid_from_handle(ca_os_self_handle());
    char task_name[32];
```

```
60      snprintf(task_name, sizeof(task_name), "PRODUCER_%u", idx);
61
62      ca_stdout_printf_n("%s started: idx=%u p1=%u", task_name, idx, p1);
63
64      u32 item = 100; // Inizia a produrre da 100
65
66      while(running) {
67          // Take the mutex to access the buffer
68          ca_mutex_lock(&buffer_mutex);
69
70          // Wait while the buffer is full
71          while(is_buffer_full()) {
72              ca_stdout_printf_n("%s: buffer full, waiting...", task_name);
73              // Use cond_wait to release the mutex and wait until someone
74              // signals that the buffer is no longer full
75              ca_cond_wait(&buffer_not_full, &buffer_mutex);
76          }
77
78          // Now we can put an item into the buffer
79          put_item(item);
80          ca_stdout_printf_n("%s: put item %u, count=%u", task_name, item, count);
81
82          // Signal that the buffer is no longer empty
83          ca_cond_signal(&buffer_not_empty);
84
85          // Release the mutex
86          ca_mutex_unlock(&buffer_mutex);
87
88          item++; // Next item to produce
89
90          // Wait a bit before producing the next item
91          ca_os_sleep(500);
92      }
93
94      return 0;
95  }
96
97  static u32 consumer_task(void *arg) {
98      const u32 p1 = (u32)(uintptr_t)arg;
99      const u32 idx = ca_os_thid_from_handle(ca_os_self_handle());
100     char task_name[32];
101     snprintf(task_name, sizeof(task_name), "CONSUMER_%u", idx);
102
103     ca_stdout_printf_n("%s started: idx=%u p1=%u", task_name, idx, p1);
104
105     while(running) {
106         // Take the mutex to access the buffer
107         ca_mutex_lock(&buffer_mutex);
108
109         // Wait while the buffer is empty
110         while(is_buffer_empty()) {
111             ca_stdout_printf_n("%s: buffer empty, waiting...", task_name);
112             // Use cond_wait to release the mutex and wait until someone
113             // signals that the buffer is no longer empty
114             ca_cond_wait(&buffer_not_empty, &buffer_mutex);
115         }
116
117         // Now we can take an item out of the buffer
118         u32 item = get_item();
119         ca_stdout_printf_n("%s: got item %u, count=%u", task_name, item, count);
120
121         // Signal that the buffer is no longer full
122         ca_cond_signal(&buffer_not_full);
```

```
123
124         // Release the mutex
125         ca_mutex_unlock(&buffer_mutex);
126
127         // Simulate some processing of the item
128         ca_os_sleep(800);
129     }
130
131     return 0;
132 }
133
134 void ca_os_main(void) {
135     running = 1;
136
137     // Initialize the mutex and the condition variables
138     ca_mutex_init(&buffer_mutex);
139     ca_cond_init(&buffer_not_full);
140     ca_cond_init(&buffer_not_empty);
141
142     // Crea i task
143     ca_os_job("prod_1", CA_TH_LEVEL2, producer_task, (void *)(uintptr_t)1,
144                         STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);
145     ca_os_job("cons_1", CA_TH_LEVEL2, consumer_task, (void *)(uintptr_t)2,
146                         STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);
147
148     ca_os_run(true);
149 }
```

30.1.4 Architecture

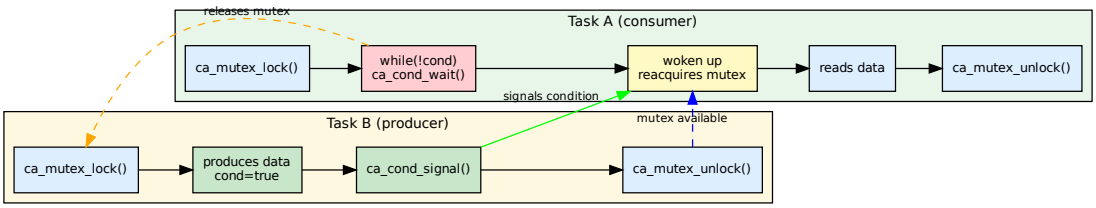


Figure 30.1: Condition variable sequence — producer-consumer coordination

The diagram shows the interaction sequence between producer and consumer via condition variable. The consumer waits (`ca_cond_wait`) until the buffer contains data; the producer inserts an element and signals (`ca_cond_signal`) waking up the consumer. The condition variable is always used together with a mutex that protects the shared buffer.

30.1.5 Detailed analysis

Shared variables

```
1 static u32 buffer[BUFFER_SIZE];
2 static u32 head = 0;  // Head index (next item to read)
3 static u32 tail = 0;  // Tail index (next item to write)
4 static u32 count = 0; // Number of items in the buffer
```

These variables represent a circular buffer shared between producer and consumer:

- `buffer`: The array containing the data
- `head`: Index of the next element to read
- `tail`: Index of the next element to write
- `count`: Number of elements currently in the buffer

All these variables are protected by the mutex `buffer_mutex`.

Condition variables

```
static ca_mutex_t buffer_mutex;      // Mutex per proteggere l'accesso al buffer
static ca_condvar_t buffer_not_full;  // Condition variable for buffer not full
static ca_condvar_t buffer_not_empty; // Condition variable for buffer not empty
```

We have three synchronization primitives:

- `buffer_mutex`: Protects access to shared variables
- `buffer_not_full`: Signals when the buffer is not full (producer can insert)
- `buffer_not_empty`: Signals when the buffer is not empty (consumer can extract)

Utility functions

The functions `is_buffer_full()`, `is_buffer_empty()`, `put_item()` and `get_item()` are utility functions that operate on the buffer. They are simple functions that do not require synchronization because they will only be called when the mutex is acquired.

Producer task

```
static u32 producer_task(void *arg) {
    (void)arg;
    // ...
    while(running) {
        // Take the mutex to access the buffer
        ca_mutex_lock(&buffer_mutex);

        // Wait while the buffer is full
        while(is_buffer_full()) {
            ca_stdout_printf_n("%s: buffer full, waiting...", task_name);
            // Use cond_wait to release the mutex and wait until someone
            // signals that the buffer is no longer full
            ca_cond_wait(&buffer_not_full, &buffer_mutex);
        }

        // Now we can put an item into the buffer
        put_item(item);
        ca_stdout_printf_n("%s: put item %u, count=%u", task_name, item, count);

        // Signal that the buffer is no longer empty
        ca_cond_signal(&buffer_not_empty);

        // Release the mutex
        ca_mutex_unlock(&buffer_mutex);
        // ...
    }
}
```

The producer task:

1. **Acquires the mutex** to safely access shared variables
2. **Checks if the buffer is full** using `is_buffer_full()`

3. **If it is full**, it calls `ca_cond_wait(&buffer_not_full, &buffer_mutex)`:
    - Automatically releases the mutex
    - Suspends itself until someone calls `ca_cond_signal(&buffer_not_full)`
    - When it wakes up, it automatically re-acquires the mutex
4. **Inserts an element** into the buffer using `put_item()`
5. **Signals that the buffer is no longer empty** with `ca_cond_signal(&buffer_not_empty)`
6. **Releases the mutex** with `ca_mutex_unlock()`

Consumer task

```
static u32 consumer_task(void *arg) {
    (void)arg;
    // ...
    while(running) {
        // Take the mutex to access the buffer
        ca_mutex_lock(&buffer_mutex);

        // Wait while the buffer is empty
        while(is_buffer_empty()) {
            ca_stdout_printf_n("%s: buffer empty, waiting...", task_name);
            // Use cond_wait to release the mutex and wait until someone
            // signals that the buffer is no longer empty
            ca_cond_wait(&buffer_not_empty, &buffer_mutex);
        }

        // Now we can take an item out of the buffer
        u32 item = get_item();
        ca_stdout_printf_n("%s: got item %u, count=%u", task_name, item, count);

        // Signal that the buffer is no longer full
        ca_cond_signal(&buffer_not_full);

        // Release the mutex
        ca_mutex_unlock(&buffer_mutex);
        // ...
    }
}
```

The consumer task:

1. **Acquires the mutex** to safely access shared variables
2. **Checks if the buffer is empty** using `is_buffer_empty()`
3. **If it is empty**, it calls `ca_cond_wait(&buffer_not_empty, &buffer_mutex)`:
    - Automatically releases the mutex
    - Suspends itself until someone calls `ca_cond_signal(&buffer_not_empty)`
    - When it wakes up, it automatically re-acquires the mutex
4. **Extracts an element** from the buffer using `get_item()`
5. **Signals that the buffer is no longer full** with `ca_cond_signal(&buffer_not_full)`
6. **Releases the mutex** with `ca_mutex_unlock()`

Coordination sequence

Coordination between producer and consumer works as follows:

1. The **producer** inserts data and signals `buffer_not_empty` when it adds something
2. The **consumer** waits for `buffer_not_empty` when the buffer is empty

3. The **consumer** extracts data and signals `buffer_not_full` when it removes something
4. The **producer** waits for `buffer_not_full` when the buffer is full

This bidirectional coordination allows for an efficient flow of data between the two tasks.

Using `while` instead of `if`

It is essential to use `while` instead of `if` when checking the condition:

```
while(is_buffer_full()) {
    ca_cond_wait(&buffer_not_full, &buffer_mutex);
}
```

This is because:

1. **Spurious wakeups**: A task might wake up without the condition actually having changed
2. **Multiplexing**: There could be multiple tasks waiting on the same condition variable
3. **Safety**: Re-checking the condition ensures that the logic is correct

Initialization

```
void ca_os_main(void) {
    running = 1;

    // Initialize the mutex and the condition variables
    ca_mutex_init(&buffer_mutex);
    ca_cond_init(&buffer_not_full);
    ca_cond_init(&buffer_not_empty);
    // ...
}
```

As with all synchronization primitives, mutexes and condition variables must be initialized before use.

30.1.6 API

The main functions for condition variables are:

- `ca_cond_init(cv)`: Initializes a condition variable
- `ca_cond_wait(cv, mutex)`: Waits for the condition to become true, temporarily releasing the mutex
- `ca_cond_signal(cv)`: Wakes up a task waiting on this condition variable
- `ca_cond_broadcast(cv)`: Wakes up all tasks waiting on this condition variable
- `ca_cond_timedwait(cv, mutex, timeout_ms)`: Like `wait` but with a timeout

30.1.7 What this example teaches us

1. **Complex coordination**: How to use condition variables to coordinate tasks that must wait for specific conditions
2. **Producer-consumer pattern**: A fundamental concurrent programming model
3. **Correct use with mutexes**: How to combine mutexes and condition variables to protect shared data
4. **Conditional waits**: How to suspend a task until a condition is met
5. **Mutual signaling**: How tasks wake each other up when conditions change

30.1.8 Example output

Below is the output produced by the example on the debug serial port:

```
================================================
 -- c.a.O.S --
   build : 77af553c+dev - 2026-02-15 21:14:22
   MCU : STM 32 F:H7xx T:753
================================================
   System Clock: 480000000 Hz
   SysTick: 47999 clocks per tick
   Debug UART3 Baudrate: 4000000 bps
================================================
PRODUCER_3 started: idx=3 p1=1
PRODUCER_3: inserito item 100, count=1
CONSUMER_4 started: idx=4 p1=2
CONSUMER_4: estratto item 100, count=0
PRODUCER_3: inserito item 101, count=1
CONSUMER_4: estratto item 101, count=0
PRODUCER_3: inserito item 102, count=1
PRODUCER_3: inserito item 103, count=2
CONSUMER_4: estratto item 102, count=1
PRODUCER_3: inserito item 104, count=2
CONSUMER_4: estratto item 103, count=1
PRODUCER_3: inserito item 105, count=2
PRODUCER_3: inserito item 106, count=3
CONSUMER_4: estratto item 104, count=2
PRODUCER_3: inserito item 107, count=3
CONSUMER_4: estratto item 105, count=2
PRODUCER_3: inserito item 108, count=3
PRODUCER_3: inserito item 109, count=4
CONSUMER_4: estratto item 106, count=3
PRODUCER_3: inserito item 110, count=4
PRODUCER_3: inserito item 111, count=5
CONSUMER_4: estratto item 107, count=4
PRODUCER_3: inserito item 112, count=5
CONSUMER_4: estratto item 108, count=4
PRODUCER_3: inserito item 113, count=5
PRODUCER_3: buffer full, waiting...
CONSUMER_4: estratto item 109, count=4
PRODUCER_3: inserito item 114, count=5
PRODUCER_3: buffer full, waiting...
CONSUMER_4: estratto item 110, count=4
PRODUCER_3: inserito item 115, count=5
PRODUCER_3: buffer full, waiting...
CONSUMER_4: estratto item 111, count=4
PRODUCER_3: inserito item 116, count=5
PRODUCER_3: buffer full, waiting...
CONSUMER_4: estratto item 112, count=4
PRODUCER_3: inserito item 117, count=5
PRODUCER_3: buffer full, waiting...
CONSUMER_4: estratto item 113, count=4
PRODUCER_3: inserito item 118, count=5
PRODUCER_3: buffer full, waiting...
CONSUMER_4: estratto item 114, count=4
```

30.1.9 Verification of waits with timeout (`twait`)

Since 09/27, a task waiting for a condition variable **with timeout** (`ca_cond_timedwait()`) registers in the same wait list as those waiting without a timeout, parked in `SLEEP` until expiration, and the signaler wakes it up immediately (S2_02 §8.8). Previously, it performed polling every 10 ms outside the list: it would wake up up to 10 ms after the signal.

The command `twait [sec]` (default 3 s) measures it with a peer task created at the first call. The wait runs in the command task:

1. **latency** — 300 cycles: the peer sets the flag and performs `ca_cond_signal()` after a random 1..8 ms; the waiter, with a timeout of 20 ms, must obtain it (`lost=0`), not before the signal (`spur=0`), and be running within 2 ms (`lat_max` limit, in ticks);
2. **timeout** — no one signals; for `sec` seconds, T = 1..20 ms: the return never arrives **before** T (`early=0`), at most 2 ms after (`late=0`), and always with `false` (`got=0`).

One line, closed by `PASS` or `FAIL`. On H753 (tick 200 µs):

```
TW cond : rounds=300 lat_max=0/10 lost=0 spur=0 | tmo n=291 early=0 late=0 got=0
    over_max=0/10  PASS
```

On the previous kernel: PASS: the waiter with timeout was already in the list (in `SLEEP`). Here `twait` is a regression; the two fixes (mutex released before parking, parking inside the lock) have a window that is too narrow for this test.

In the bench, it is case `timed_wait` of `test_ex07.cpp` (helper `test/hw_test/inc/caosTimedWait.h`). Manually:

```
caos_shell -e 7 -q -t 40 -u "TW .*(PASS|FAIL)" -g "^TW" "twait 3"
```

Chapter 31

Critical Sections: Protecting Data by Disabling Interrupts

31.1 Example 8 — Critical sections: data protection with interrupt disabling

31.1.1 Introduction

In the previous examples, we saw how mutexes, semaphores, and condition variables protect access to resources shared between tasks. All these primitives have a cost: they require data structures, wait queues, and can block waiting tasks. However, there is a scenario where a lighter and more immediate protection is needed: when the block of code to be protected is **very short** and cannot tolerate even the overhead of a mutex.

Critical sections are the simplest and most direct mechanism for protecting shared data. They work by temporarily disabling interrupts, ensuring that no context switch occurs during the execution of the protected code. This makes them ideal for atomic operations on a few variables, where the execution time is minimal.

Critical sections in caOS support **nesting**: it is possible to nest multiple `ca_enter_critical` / `ca_exit_critical` without issues. Only the first call disables interrupts, and only the last one re-enables them. This is fundamental when functions protected by a critical section call other functions that are also protected.

31.1.2 What is a critical section?

A critical section is a protection mechanism that:

- **Disables interrupts** to prevent context switching
- Supports **nesting** via an internal counter
- Does not require additional data structures (no mutexes, no wait queues)
- Is the **lightest** protection mechanism available

Critical sections are particularly useful for:

1. **Short atomic operations**: Updating one or a few shared variables safely
2. **Consistent reads**: Reading a set of related variables without them being modified halfway through
3. **Task-ISR protection**: Ensuring consistency between code executed by tasks and interrupt handlers

31.1.3 The source code

The file is located in `examples/example_8/ca_os_ex8.c`:

```c
#include "caos.h"

/*
 * Example of critical section usage
 *
 * Critical sections temporarily disable interrupts to guarantee
 * that a block of code runs without interruptions.
 * They support nesting: only the first ca_enter_critical() disables the IRQs,
 * only the last ca_exit_critical() re-enables them.
 *
 * A writer task updates a shared variable and a reader task reads it.
 * Both use critical sections to keep the data consistent.
 */

static u32 running;
static u32 shared_counter = 0;
static u32 write_count = 0;
static u32 read_count = 0;

// Task that updates the shared variable
static u32 task_writer(void *arg) {
    const u32 p1 = (u32)(uintptr_t)arg;
    const u32 idx = ca_os_thid_from_handle(ca_os_self_handle());

    ca_stdout_printf_n("WRITER started: idx=%u p1=%u", idx, p1);

    while(running) {
        ca_enter_critical();
        shared_counter++;
        write_count++;
        u32 sc = shared_counter;
        u32 wc = write_count;
        u32 nesting = ca_critical_nesting_level();
        ca_exit_critical();

        ca_stdout_printf_n("WRITER: counter=%u writes=%u nesting_was=%u", sc, wc
    , nesting);
        ca_os_sleep(300);
    }

    return 0;
}

// Task that reads the shared variable
static u32 task_reader(void *arg) {
    const u32 p1 = (u32)(uintptr_t)arg;
    const u32 idx = ca_os_thid_from_handle(ca_os_self_handle());

    ca_stdout_printf_n("READER started: idx=%u p1=%u", idx, p1);

    while(running) {
        ca_enter_critical();
        u32 sc = shared_counter;
        u32 wc = write_count;
        u32 rc = ++read_count;
        ca_exit_critical();

        ca_stdout_printf_n("READER: counter=%u writes=%u reads=%u", sc, wc, rc);
        ca_os_sleep(500);
    }
```

```
60
61      return 0;
62  }
63
64  void ca_os_main(void) {
65      running = 1;
66
67      ca_os_job("writer", CA_TH_LEVEL2, task_writer, (void *)(uintptr_t)1,
68                          STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);
69      ca_os_job("reader", CA_TH_LEVEL2, task_reader, (void *)(uintptr_t)2,
70                          STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);
71
72      ca_os_run(true);
73  }
```

31.1.4 Architecture

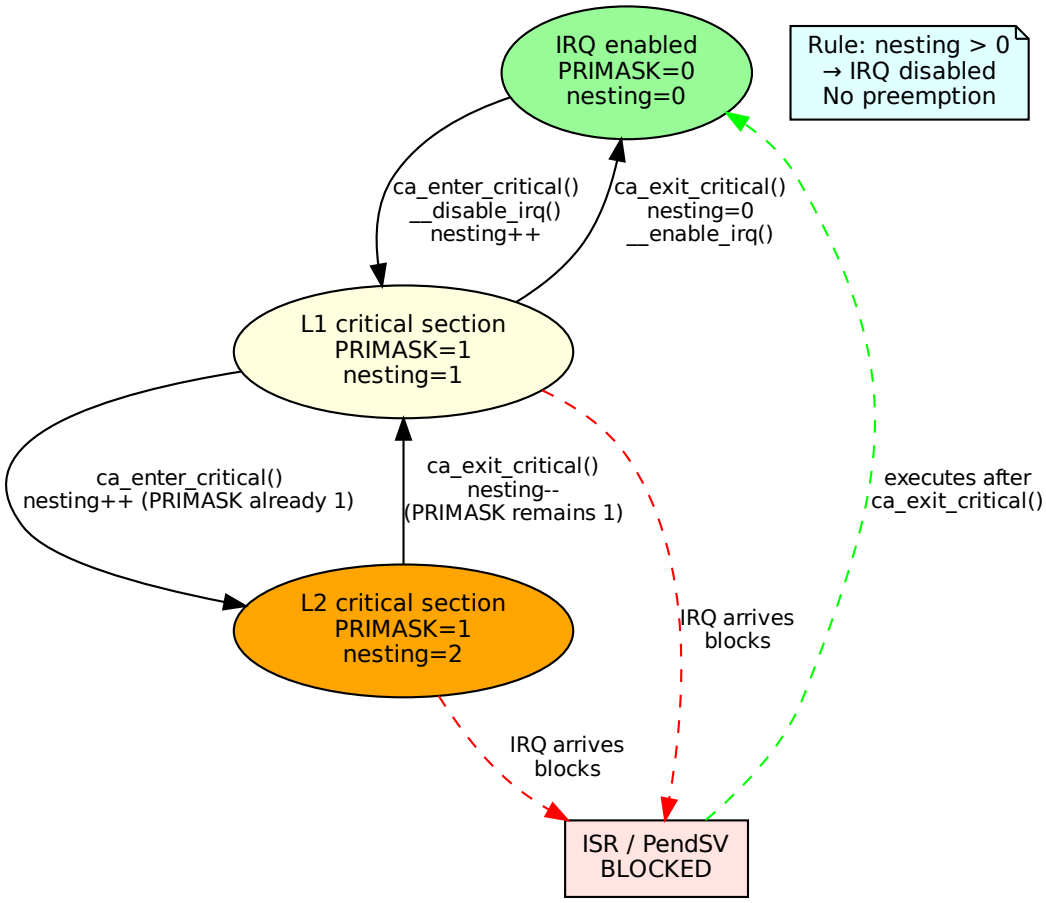


Figure 31.1: Critical section states — PRIMASK nesting

The diagram shows the management of critical section nesting. Each `ca_enter_critical` increments the nesting counter and disables interrupts (PRIMASK=1). Each `ca_exit_critical` decrements the counter; only when the counter returns to zero are interrupts re-enabled. This allows

nesting critical sections without the risk of re-enabling interrupts prematurely.

31.1.5 Detailed analysis

Shared variables

```
static u32 shared_counter = 0;
static u32 write_count = 0;
static u32 read_count = 0;
```

These three variables are shared between the two tasks:

- `shared_counter`: Counter incremented by the writer and read by the reader
- `write_count`: Total number of writes performed
- `read_count`: Total number of reads performed

Note that, unlike the previous examples, **there is no need to declare or initialize any synchronization structure**. The critical sections do not require objects: they operate directly on the processor's PRIMASK register.

Writer task

```
static u32 task_writer(void *arg) {
    (void)arg;
    // ...
    while(running) {
        ca_enter_critical();
        shared_counter++;
        write_count++;
        u32 sc = shared_counter;
        u32 wc = write_count;
        u32 nesting = ca_critical_nesting_level();
        ca_exit_critical();

        ca_stdout_printf_n("WRITER: counter=%u writes=%u nesting_was=%u", sc, wc
    , nesting);
        ca_os_sleep(300);
    }
    // ...
}
```

The writer task:

1. **Enters the critical section** with `ca_enter_critical()`: interrupts are disabled
2. **Updates shared variables** safely: no other task can interrupt this sequence
3. **Copies values into local variables** to be able to print them after releasing the critical section
4. **Queries the nesting level** with `ca_critical_nesting_level()` to show the internal counter
5. **Exits the critical section** with `ca_exit_critical()`: interrupts are re-enabled
6. **Prints the results** outside the critical section, because I/O is a slow operation

Reader task

```
static u32 task_reader(void *arg) {
    (void)arg;
    // ...
    while(running) {
        ca_enter_critical();
        u32 sc = shared_counter;
```

```
        u32 wc = write_count;
        u32 rc = ++read_count;
        ca_exit_critical();

        ca_stdout_printf_n("READER: counter=%u writes=%u reads=%u", sc, wc, rc);
        ca_os_sleep(500);
    }
    // ...
}
```

The reader task:

1. **Enters a critical section** to ensure a consistent read
2. **Reads all related variables** in a single atomic operation: `shared_counter`, `write_count` and `read_count` will always be consistent with each other
3. **Increments the read counter** inside the critical section
4. **Exits the critical section** and prints the results

Consistent reading is the key point: without a critical section, the writer could modify `shared_counter` between the reading of `shared_counter` and that of `write_count`, producing inconsistent values.

Critical section nesting

The implementation in caOS supports nesting via a global counter:

```
static volatile u32 g_critical_nesting = 0;
static volatile u32 g_saved_primask = 0;
```

- The first `ca_enter_critical()` saves the PRIMASK state and disables IRQs
- Subsequent ones only increment the counter
- Each `ca_exit_critical()` decrements the counter
- Only when the counter returns to zero are IRQs re-enabled

This means it is safe to call `ca_enter_critical()` from a function already inside a critical section, without the risk of prematurely re-enabling interrupts.

Initialization

```
void ca_os_main(void) {
    running = 1;

    ca_os_job("writer", CA_TH_LEVEL2, task_writer, (void *)(uintptr_t)1,
                        STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);
    ca_os_job("reader", CA_TH_LEVEL2, task_reader, (void *)(uintptr_t)2,
                        STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);

    ca_os_run(true);
}
```

We note the absence of any initialization call for critical sections. Unlike mutexes (`ca_mutex_init`), semaphores (`ca_sem_init`), and condition variables (`ca_cond_init`), critical sections do not require initialization: they are ready for use immediately.

31.1.6 API

The functions for critical sections are:

- `ca_enter_critical()`: Enters a critical section (disables IRQs, supports nesting)
- `ca_exit_critical()`: Exits a critical section (re-enables IRQs only when the nesting count returns to 0)
- `ca_critical_nesting_level()`: Returns the current nesting level

31.1.7 Critical sections vs other primitives

Feature	Critical section	Mutex	IRQ Spinlock
Initialization	None	`ca_mutex_init`	`ca_spinlock_irq_init`
Blocks other tasks	No (disables IRQs)	Yes (wait queue)	No (disables IRQs)
Overhead	Minimal	Medium	Low
Nesting	Yes (global counter)	No (deadlock)	No
Recommended duration	Very short	Any	Short (<100µs)
Use from ISR	Yes	No	Yes

Critical sections are the right choice when:

1. The code to be protected is **very short** (a few instructions)
2. There is no need to **block** other tasks, only to prevent context switching
3. Protection from **interrupt handlers** is also required
4. The **lightest mechanism possible** is desired

31.1.8 Warning: fundamental rules

- **Never perform slow operations** inside a critical section: no I/O, no sleep, no allocations. Interrupts are disabled, so the entire system is halted.
- **Always balance** calls: every `ca_enter_critical` must have its own `ca_exit_critical`. A failure to release leaves interrupts permanently disabled.
- **Prefer mutexes** when the protected section is long or when it is necessary to block other tasks while waiting.

31.1.9 What this example teaches us

1. **Minimal protection**: How to use critical sections for short atomic operations without synchronization overhead
2. **Consistent reading**: How to ensure that a set of related variables is read consistently
3. **Safe nesting**: How critical sections can be nested without the risk of premature interrupt re-enabling
4. **Printing outside the critical section**: How to copy data into local variables and print only after releasing the protection
5. **Zero initialization**: Unlike other primitives, critical sections are ready for use without setup

31.1.10 Example output

Below is the output produced by the example on the debug serial port:

```
================================================
 -- c.a.O.S --
   build : 77af553c+dev - 2026-02-15 22:28:56
   MCU : STM 32 F:H7xx T:753
================================================
   System Clock: 480000000 Hz
```

```
   SysTick: 47999 clocks per tick
   Debug UART3 Baudrate: 4000000 bps
================================================
WRITER started: idx=3 p1=1
WRITER: counter=1 writes=1 nesting_was=1
READER started: idx=4 p1=2
READER: counter=1 writes=1 reads=1
WRITER: counter=2 writes=2 nesting_was=1
READER: counter=2 writes=2 reads=2
WRITER: counter=3 writes=3 nesting_was=1
WRITER: counter=4 writes=4 nesting_was=1
```

Chapter 32

Task Notification: Lightweight Inter-Task Signaling

32.1 Example 9 — Task Notification: lightweight signaling between tasks

32.1.1 Introduction

In the previous examples, we have seen various synchronization primitives: mutexes for mutual exclusion, semaphores for resource control, condition variables for producer-consumer coordination, and critical sections for fast data protection. All these primitives require the declaration and initialization of dedicated data structures.

However, there is a very common scenario where a task simply needs to **signal something to another task**: "data is ready", "work is completed", "an event has arrived". For these cases, caOS offers **task notifications**, the lightest signaling mechanism available.

Task notifications do not require any additional data structures: they use a `notify_value` field already present in the context of each task. This makes them faster than queues and semaphores, with zero memory overhead. The constraint is that each task has only one `u32` value for notifications.

32.1.2 What is a task notification?

A task notification is a direct task-to-task signaling mechanism that:

- Uses a **single u32 value** already present in the task context
- Requires **no additional data structures** (no mutexes, semaphores, or queues)
- Is **faster** than any other synchronization primitive
- Supports different **actions** on the value: overwrite, increment, bit set/clear
- Can also be used by **ISRs** (interrupt handlers)

Task notifications are particularly useful for:

1. **"Data ready" signaling**: A task notifies another that data is ready
2. **Event counters**: Incrementing a counter for every event
3. **Status flags**: Setting specific bits to signal different conditions

32.1.3 The source code

The file is located in `examples/example_9/ca_os_ex9.c`:

```c
#include "caos.h"

/*
 * Example of task notification usage
 *
 * Notifications are the lightest signalling mechanism in caOS:
 * zero memory overhead (they use the notify_value field of the task context)
 * and they are faster than queues and semaphores for simple signalling.
 *
 * In this example a sender task sends notifications to a receiver task,
 * showing the different actions available: overwrite, increment and set bits.
 */

static u32 running;
static u32 receiver_idx;  // receiver task index, saved for the sender

// Receiver task: waits for notifications from the sender
static u32 task_receiver(void *arg) {
    const u32 p1 = (u32)(uintptr_t)arg;
    const u32 idx = ca_os_thid_from_handle(ca_os_self_handle());

    ca_stdout_printf_n("RECEIVER started: idx=%u p1=%u", idx, p1);

    // save its own index for the sender
    receiver_idx = idx;

    while(running) {
        // attende notification con clear automatico, timeout 2000ms
        u32 value = ca_notify_take(2000);

        if (value != 0) {
            ca_stdout_printf_n("RECEIVER: notification received value=0x%08X",
    value);
        } else {
            ca_stdout_printf_n("RECEIVER: timeout, no notification");
        }
    }

    return 0;
}

// Sender task: sends notifications to the receiver with different actions
static u32 task_sender(void *arg) {
    const u32 p1 = (u32)(uintptr_t)arg;
    const u32 idx = ca_os_thid_from_handle(ca_os_self_handle());

    ca_stdout_printf_n("SENDER started: idx=%u p1=%u", idx, p1);

    u32 cycle = 0;

    while(running) {
        ca_os_sleep(500);

        switch(cycle % 3) {
        case 0:
            // overwrite: writes a value directly
            ca_notify_give(receiver_idx, 0xAA00 + cycle);
            ca_stdout_printf_n("SENDER: overwrite value=0x%08X", 0xAA00 + cycle)
    ;
            break;

        case 1:
            // increment: increments the receiver's counter
```

```
            ca_notify_give_inc(receiver_idx);
            ca_stdout_printf_n("SENDER: increment");
            break;

        case 2:
            // set bits: imposta bit specifici
            ca_notify_send(receiver_idx, (1 << 0) | (1 << 4),
    CA_NOTIFY_ACTION_SET_BITS);
            ca_stdout_printf_n("SENDER: set bits 0x%08X", (1 << 0) | (1 << 4));
            break;
        }

        cycle++;
    }

    return 0;
}

void ca_os_main(void) {
    running = 1;

    // receiver created first so that its idx is available
    ca_os_job("receiver", CA_TH_LEVEL2, task_receiver, NULL,
                          STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);
    ca_os_job("sender",   CA_TH_LEVEL3, task_sender, NULL,
                          STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);

    ca_os_run(true);
}
```

32.1.4 Architecture

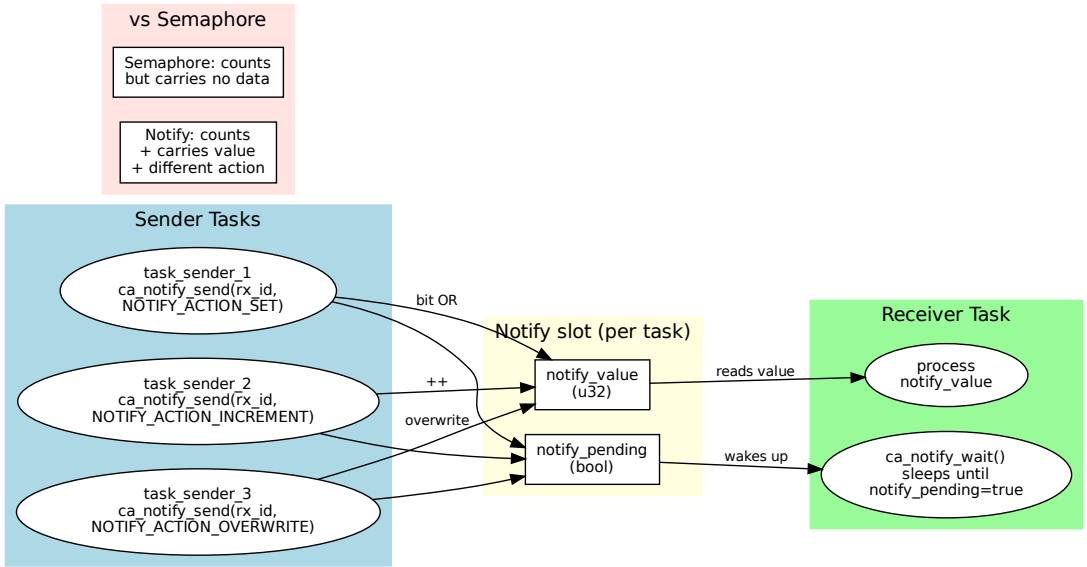


Figure 32.1: Task notification sequence — direct signaling between tasks

The diagram shows how sender tasks send notifications to the receiver with different actions: `SET` (OR bits), `INCREMENT` (counter), `OVERWRITE` (direct value). The receiver wakes up automatically when a notification arrives. Unlike a semaphore, the notification also carries a u32 value and allows encoding additional information within the action itself.

32.1.5 Detailed analysis

Destination task addressing

```
static u32 receiver_idx;  // receiver task index, saved for the sender
```

Unlike mutexes and semaphores, where tasks access a shared structure, notifications are **direct**: the sender must know the index of the destination task. In this example, the receiver saves its own index (received as parameter `idx`) in a global variable, so the sender can use it.

The index `idx` is the one assigned by the scheduler at the time of task creation with `ca_os_job`. It is the first parameter of the task function.

Task receiver

```
static u32 task_receiver(void *arg) {
    const u32 idx = ca_os_thid_from_handle(ca_os_self_handle());
    (void)arg;
    // ...
    receiver_idx = idx;

    while(running) {
        u32 value = ca_notify_take(2000);

        if (value != 0) {
            ca_stdout_printf_n("RECEIVER: notification received value=0x%08X",
    value);
        } else {
            ca_stdout_printf_n("RECEIVER: timeout, no notification");
        }
    }
    // ...
}
```

The receiver task:

1. **Saves its own index** in `receiver_idx` to make it accessible to the sender
2. **Waits for a notification** with `ca_notify_take(2000)`: it blocks for up to 2000ms
3. **Checks the received value**: if it is not 0, the notification has arrived; if it is 0, the timeout has expired
4. **The clear is automatic**: `ca_notify_take` resets the `notify_value` after reading, so the receiver is ready for the next notification

Sender task

```
static u32 task_sender(void *arg) {
    (void)arg;
    // ...
    u32 cycle = 0;

    while(running) {
        ca_os_sleep(500);

        switch(cycle % 3) {
        case 0:
```

```
            ca_notify_give(receiver_idx, 0xAA00 + cycle);
            break;
        case 1:
            ca_notify_give_inc(receiver_idx);
            break;
        case 2:
            ca_notify_send(receiver_idx, (1 << 0) | (1 << 4),
    CA_NOTIFY_ACTION_SET_BITS);
            break;
        }
        cycle++;
    }
    // ...
}
```

The sender task demonstrates three different notification actions, cycling through them:

1. **Overwrite** (`ca_notify_give`): Overwrites the receiver's `notify_value` with a direct value. This is the most common action, ideal for passing simple data.
2. **Increment** (`ca_notify_give_inc`): Increments the receiver's `notify_value` by 1. Useful as an event counter: the receiver reads how many events have accumulated.
3. **Set bits** (`ca_notify_send` with `CA_NOTIFY_ACTION_SET_BITS`): Performs an OR operation between the current value and the provided bitmask. It allows signaling multiple conditions: each bit represents a different event.

The five available actions

Action	Constant	Operation	Typical use
None	`CA_NOTIFY_ACTION_NONE`	No modification, wake-up only	Pure wake-up
Set bits	`CA_NOTIFY_ACTION_SET_BITS`	`value |= param`	Multiple flags
Clear bits	`CA_NOTIFY_ACTION_CLEAR_BITS`	`value &= ~param`	Reset specific flags
Overwrite	`CA_NOTIFY_ACTION_OVERWRITE`	`value = param`	Data passing
Increment	`CA_NOTIFY_ACTION_INCREMENT`	`value++`	Event counter
Set if empty	`CA_NOTIFY_ACTION_SET_IF_EMPTY`	`value = param` only if `value == 0`, otherwise `false`	Data that must not overwrite an unread one

Priority and creation order

```
ca_os_job("receiver", CA_TH_LEVEL2, task_receiver, NULL,
                      STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);
ca_os_job("sender",   CA_TH_LEVEL3, task_sender, NULL,
                      STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);
```

The receiver is created with a higher priority (`CA_TH_LEVEL2`) than the sender (`CA_TH_LEVEL3`). This ensures that the receiver is scheduled first and can save its `idx` in `receiver_idx` before the sender starts sending notifications.

No initialization

As with critical sections (example 8), task notifications do not require any initialization call. The `notify_value` field in the task context is initialized to zero by the scheduler at the time of task creation.

32.1.6 API

The main functions for task notifications are:

- `ca_notify_send(task_idx, value, action)`: Send notification with specified action
- `ca_notify_give(task_idx, value)`: Send notification with overwrite (inline helper)
- `ca_notify_give_inc(task_idx)`: Send notification with increment (inline helper)
- `ca_notify_take(timeout_ms)`: Wait for notification with automatic clear (inline helper)
- `ca_notify_peek(timeout_ms)`: Wait for notification without clear (inline helper)
- `ca_notify_wait(clear_on_exit, timeout_ms)`: Wait for notification (base function)
- `ca_notify_get(task_idx)`: Read value without blocking
- `ca_notify_clear(task_idx)`: Clear value
- `ca_notify_send_isr(task_idx, value, action)`: Send notification from ISR
- `ca_notify_give_isr(task_idx, value)`: Send notification from ISR with overwrite (inline helper)

32.1.7 Notifications vs other primitives

Feature	Notification	Queue	Semaphore
Additional memory	Zero	Buffer + structure	Structure
Initialization	None	`ca_queue_static_init`	`ca_sem_init`
Transferable values	1 x u32	N messages	Counter
Directionality	Task-to-task	Any	Any
Use from ISR	Yes (`_isr`)	Limited	Yes (`_post`)
Overhead	Minimal	Medium	Low

Notifications are the right choice when:

1. You need to signal a **single event** or pass a **single value**
2. Communication is **direct** between two specific tasks
3. You want the **fastest and lightest** mechanism possible
4. You are operating from an **ISR** and need to signal a task with minimal overhead

32.1.8 When not to use notifications

- If you need **multiple values** in a queue: use queues
- If **multiple tasks** need to wait for the same signal: use semaphores or condition variables
- If you need **mutual exclusion**: use mutexes
- If the receiver is not a specific task but **anyone interested**: use other mechanisms

32.1.9 What this example teaches us

1. **Direct signaling**: How to send notifications to a specific task via its handle
2. **Multiple actions**: How to use overwrite, increment, and set bits for different scenarios
3. **Zero setup**: How notifications require no data structures or initialization
4. **Waiting with timeout**: How the receiver blocks waiting with a timeout and automatic clear

5. **Informed trade-offs**: Notifications are extremely fast but limited to a single u32 value per task ## Example output

Below is the output produced by the example on the debug serial port:

```
================================================
 -- c.a.O.S --
   build : 77af553c+dev - 2026-02-15 22:34:58
   MCU : STM 32 F:H7xx T:753
================================================
   System Clock: 480000000 Hz
   SysTick: 47999 clocks per tick
   Debug UART3 Baudrate: 4000000 bps
================================================
SENDER started: idx=4 p1=0
RECEIVER started: idx=3 p1=0
SENDER: overwrite value=0x0000AA00
RECEIVER: notifica ricevuta value=0x0000AA00
SENDER: increment
RECEIVER: notifica ricevuta value=0x00000001
SENDER: set bits 0x00000011
RECEIVER: notifica ricevuta value=0x00000011
SENDER: overwrite value=0x0000AA03
RECEIVER: notifica ricevuta value=0x0000AA03
SENDER: increment
RECEIVER: notifica ricevuta value=0x00000001
SENDER: set bits 0x00000011
RECEIVER: notifica ricevuta value=0x00000011
SENDER: overwrite value=0x0000AA06
RECEIVER: notifica ricevuta value=0x0000AA06
```

32.1.10 Verification of waits with timeout (`twait`)

Since 09/27, a task waiting for a notification **with timeout** (`ca_notify_take()`) registers in the same wait list as those waiting without a timeout, parked in `SLEEP` until expiration, and the signaler wakes it up immediately (S2_02 §8.8). Previously, it performed polling every 10 ms outside the list: it would wake up up to 10 ms after the signal.

The command `twait [sec]` (default 3 s) measures it with a peer task created at the first call. The wait runs in the command task:

1. **latency** — 300 loops: the peer performs `ca_notify_send_handle()` to the waiting task after a random 1..8 ms; the waiter, with a 20 ms timeout, must obtain it (`lost=0`), not before the signal (`spur=0`), and be running within 2 ms (`lat_max` limit, in ticks);
2. **timeout** — no one notifies; for `sec` seconds, T = 1..20 ms: the return never arrives **before** T (`early=0`), at most 2 ms after (`late=0`), and always with `false` (`got=0`).

One line, closed by `PASS` or `FAIL`. On H753 (tick 200 µs):

```
TW notify : rounds=300 lat_max=0/10 lost=0 spur=0 | tmo n=291 early=0 late=0 got
   =0 over_max=0/10  PASS
```

On the previous kernel: FAIL, `lat_max=50/10`.

In the bank, it is case `timed_wait` of `test_ex09.cpp` (helper `test/hw_test/inc/caosTimedWait.h`). Manually:

```
caos_shell -e 9 -q -t 40 -u "TW .*(PASS|FAIL)" -g "^TW" "twait 3"
```

Chapter 33

Watchdog

33.1 Example 10 — Watchdog Timer (WDT)

33.1.1 Introduction

In the previous examples, we saw how tasks can be protected from software faults using the Safeguard module. However, there is another fundamental protection in embedded systems: the **Watchdog Timer (WDT)**.

The watchdog is a hardware timer that, if not periodically "reloaded" by the software, **resets the MCU**. It protects the system from hangs, infinite loops, and tasks that stop performing their duties.

A watchdog that is configured but never triggers is indistinguishable from one that is working, until the day it is needed: for this reason, the example does not just configure it, but **actually triggers it** and, upon reboot, reports what caused the reset.

In this example, we will see how to: 1. configure the watchdog (IWDG or WWDG) and know its maximum timeout; 2. reload it from a dedicated task; 3. reconfigure an **already running** IWDG (new timeout, the watchdog remains active); 4. read the **cause of the last reset** with `ca_wdt_reset_cause()`.

33.1.2 What is the Watchdog Timer?

IWDG (Independent Watchdog)

Feature	Description
Clock	LSI (~32 kHz, ~40 kHz on F1) - independent of the system clock
Timeout	from 1 ms to the maximum of the family (table below)
Once started	**CANNOT be stopped**, only reconfigured

The LSI is not a crystal: on some parts, it deviates from the nominal value by tens of percentage points, so IWDG timeouts are **nominal**.

WWDG (Window Watchdog)

Feature	Description
Clock	PCLK divided by 4096 × 2^WDGTB
Timeout	from about 0.1 ms to a few tens/hundreds of ms

Feature	Description
Window	the reload must occur within a window (in caOS the window is open: 0x7F)

33.1.3 Driver API (`wdt.h`)

```
bool ca_wdt_init(u32 timeout_ms, ca_wdt_mode_t mode, ca_wdt_type_t type);
bool ca_wdt_enable(void);              /* starts and reloads */
void ca_wdt_reload(void);
u32  ca_wdt_get_timeout_ms(void);      /* effective timeout after HW rounding */
bool ca_wdt_is_enabled(void);
u32  ca_wdt_max_timeout_ms(ca_wdt_type_t type);
ca_wdt_reset_t ca_wdt_reset_cause(bool clear);   /* NONE | IWDG | WWDG */
```

- The timeout ranges from `CA_WDT_TIMEOUT_MIN_MS` (1 ms) to `ca_wdt_max_timeout_ms(type)`, the **actual** maximum for the family.
- A **running IWDG** accepts a new `ca_wdt_init()`: prescaler and reload change, and the subsequent `ca_wdt_enable()` reloads the counter with the new value. This is what `ca_os_reset()` and the `reboot` command (1 ms timeout) do, and it is what is needed for an image started by a bootloader that left the IWDG enabled. A running WWDG, or a change of type, is rejected.
- With a **debugger connected**, the watchdogs stop while the core is in halt (freeze in DBGMCU, written only if `DHCSR.C_DEBUGEN` is active: without a debugger, the DBGMCU is not touched).
- RCC reset flags remain at 1 until they are cleared: `ca_wdt_reset_cause(true)` clears **all** of them (power-on, pin, software…), so those who need the others must read them first.

Maximums per family

Values printed by the boards at boot (`E10: max IWDG ... WWDG ...`), 10/21/2026:

Board	IWDG max	WWDG max	PCLK1
NUCLEO-F446ZE	32768 ms	49 ms	42 MHz, WDGTB up to /8
NUCLEO-F767ZI	32768 ms	38 ms	54 MHz, /8
NUCLEO-H753ZI	32768 ms	279 ms	120 MHz, /128
NUCLEO-H563ZI	131072 ms	134 ms	250 MHz, /128
NUCLEO-U575ZI-Q	131072 ms	419 ms	80 MHz, /128

F1xx (LSI 40 kHz) reaches ~26 s of IWDG; it is not on the test bench.

33.1.4 The source code

The file is located in `examples/example_10/ca_os_ex10.c`. The example remains **stopped at boot** and works on command.

Only one task reloads the watchdog, and only when told to do so:

```
static u32 task_feeder(void *arg)
{
    (void)arg;
    while (1) {
        if (s_feed) ca_wdt_reload();
        ca_os_sleep(E10_FEED_MS);          /* 200 ms */
    }
    return 0;
}
```

At boot, first of all, the cause of the last reset is read and the flags are cleared:

```
s_boot_cause = ca_wdt_reset_cause(true);
PRINTF_N(buf, sizeof(buf), "E10: last reset by watchdog: %s", s_cause_name[
    s_boot_cause]);
```

The heart of `e10 start` is phases 3 and 4: an IWDG at 20 s, then the **same IWDG in operation** brought to 5 s. Surviving 6 s after reconfiguration proves that the new value has been taken and that the task is keeping it alive:

```
bool p3 = ca_wdt_init(E10_LONG_MS, CA_WDT_MODE_RESET, CA_WDT_TYPE_IWDG);
u32 t3 = ca_wdt_get_timeout_ms();
s_feed = true;
p3 = p3 && near(t3, E10_LONG_MS) && ca_wdt_enable() && ca_wdt_is_enabled();
ca_os_sleep(3000);

bool p4 = ca_wdt_init(E10_RECONF_MS, CA_WDT_MODE_RESET, CA_WDT_TYPE_IWDG);
u32 t4 = ca_wdt_get_timeout_ms();
p4 = p4 && near(t4, E10_RECONF_MS) && ca_wdt_enable();
ca_os_sleep(E10_RECONF_MS + 1000U);
```

`e10 bite` configures the watchdog and stops reloading it:

```
static void bite(ca_wdt_type_t type, u32 ms)
{
    if (!ca_wdt_init(ms, CA_WDT_MODE_RESET, type)) { ... }
    s_feed = false;
    if (!ca_wdt_enable()) { ... }
    CLI_PRINTF_N("E10: starving %s, %u ms - a reset must follow", ...);
}
```

33.1.5 Architecture

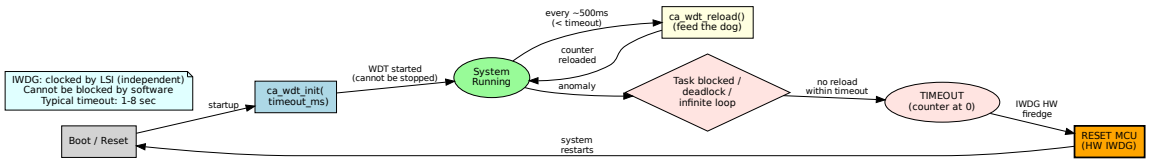


Figure 33.1: Watchdog Timer state machine

The diagram shows the watchdog lifecycle: after initialization, the system must periodically reload the counter (`ca_wdt_reload`) within the configured timeout. If the task hangs or enters an infinite loop, the reload does not occur and, upon timeout expiration, the IWDG generates a hardware reset independent of the software. The watchdog cannot be stopped after it has been started.

33.1.6 CLI Commands

Command	Description
`e10 start`	single pass, no reset, closed by `RUN RESULT : OK (4/4)`
`e10 bite iwdg [ms]`	IWDG at `ms` (default 2000), reload task stopped: reset

Command	Description
`e10 bite wwdg`	WWDG at its maximum, never reloaded: reset. Rejected if `start` has already enabled the IWDG
`e10 early wwdg`	WWDG with early warning: the callback leaves a mark in the last backup register, then reset; the next boot prints `E10: early warning before the reset: WWDG, 1 call(s)`
`e10 early iwdg [ms]`	same with IWDG on H5/U5; on other families the mode must be rejected: `E10: no early warning for IWDG here, mode refused`
`e10 info`	reset cause at boot, maximums, driver status
`wdt info`	driver node (`EN_CLI_WDT=1`), registered by `ca_wdt_init()`
`reboot`	`cli.c` command: arms the IWDG at 1 ms and stops reloading it

The four phases of `e10 start`:

1. `ca_wdt_init(0)` and `ca_wdt_init(max + 1)` rejected;
2. IWDG maximum reaches at least 20 s on every family;
3. IWDG at 20 s: effective timeout within 5%, started, kept alive for 3 s;
4. IWDG **running** set to 5 s: accepted, and the board survives for 6 s.

From the end of the pass, the IWDG remains on (it cannot be turned off) and the task reloads it.

33.1.7 Expected Output

On the NUCLEO-H563ZI:

```
================================================
       TEST 10: Watchdog Timer (WDT)
================================================
E10: last reset by watchdog: none
E10: max IWDG 131072 ms, WWDG 134 ms
E10: early warning before the reset: none
E10: idle - e10 start | bite | early | info

c.a.O.S > e10 start
E10: start - 4 phases
WDT: timeout out of range
WDT: timeout out of range
E10: phase 1 OK - 0 ms and max+1 ms refused
E10: max IWDG 131072 ms, WWDG 134 ms
E10: phase 2 OK - IWDG reaches 20 s
E10: IWDG 20000 ms -> effective 20000 ms, running
E10: phase 3 OK - IWDG at 20 s enabled and fed
E10: running IWDG set to 5000 ms -> effective 5000 ms
E10: phase 4 OK - running IWDG reconfigured, still alive
  RUN RESULT : OK  (4/4)

c.a.O.S > e10 bite iwdg 2000
E10: starving IWDG, 2000 ms - a reset must follow
[RESET AFTER ~2 SECONDS]
...
E10: last reset by watchdog: IWDG
```

The two `WDT: timeout out of range` are the driver rejecting the phase 1 values: they are expected. The driver writes them directly to the UART, not from the stdout queue, so they may appear broken or mixed with the task lines.

A `e10 bite wwdg` on the NUCLEO-H753ZI (after a `reset`, because the IWDG enabled by `start` rejects it):

```
c.a.O.S > e10 bite wwdg
E10: starving WWDG, 275 ms - a reset must follow
...
E10: last reset by watchdog: WWDG
```

33.1.8 The bench test

`test/hw_test/src/test_ex10.cpp` executes, in this order:

Phase	Expected
boot	cause, maximums, `E10: idle`
`e10 start`	`RUN RESULT : OK (4/4)`, no reset during the pass
`wdt info`	`wdt: ENABLED type=IWDG`
`e10 bite iwdg 2000`	no banner in the first 600 ms, banner within 8 s, cause `IWDG`
`e10 bite wwdg`	banner within 4 s, cause `WWDG`
`e10 early wwdg`	banner within 4 s, cause `WWDG`, early warning sign
`e10 early iwdg 2000`	H5/U5: no banner in the first 600 ms, banner within 8 s, cause `IWDG` and sign; elsewhere `mode refused` and no reset
`reboot`	banner within 4 s, cause `IWDG`
`reset`	cause `none`: the previous boot cleared the flags

The order matters: the WWDG comes before `reboot` because an enabled IWDG (left by `start`) rejects the WWDG, and only a reset turns off the IWDG.

Bench result, 21/10/2026: 7/7 on NUCLEO-H563ZI, H753ZI, F446ZE, F767ZI and U575ZI-Q (Release). **22/10/2026**, with the two early warning phases: 9/9 on NUCLEO-H563ZI, H753ZI, F767ZI, U575ZI-Q and F446ZE.

33.1.9 Differences between Families

Family	Reset flag register	Notes
F1xx / F4xx / F7xx / U5xx	`RCC->CSR` (`IWDGRSTF`, `WWDGRSTF`, `RMVF`)	
H5xx	`RCC->RSR` (`IWDGRSTF`, `WWDGRSTF`, `RMVF`)	IWDG with 4-bit PR (/1024)

Family	Reset flag register	Notes
H7xx	`RCC->RSR` (`IWDG1RSTF`, `WWDG1RSTF`, `RMVF`)	instances `IWDG1`/`WWDG1`; WWDG1 resets the system only with `RCC_GCR.WW1RSC = 1` (RM0433 §7.7.35), which the driver sets

33.1.10 Best Practices

1. Only one task reloads

Reloading scattered across multiple tasks hides the blocked task: it is enough for just one to continue reloading. It is better to have a dedicated task that reloads only if the others have shown signs of life.

2. Timeout well above the reload period

```
ca_wdt_init(2000, CA_WDT_MODE_RESET, CA_WDT_TYPE_IWDG);  /* 2 s */
/* reload every 200-500 ms */
```

With the LSI potentially deviating significantly from the nominal value, a margin of 4-10 times is reasonable.

3. Reading the reset cause at boot

```
ca_wdt_reset_t why = ca_wdt_reset_cause(true);
```

A watchdog reset in the field is information: it must be recorded, not lost at the next reset.

33.1.11 Summary

1. **The watchdog protects against software hangs** by resetting the MCU if it is not reloaded in time.
2. **IWDG** is independent of the system clock and can reach tens of seconds (131 s on H5/U5); **WWDG** has millisecond timeouts.
3. **An enabled IWDG cannot be turned off**, but it can be reconfigured: `ca_wdt_init()` + `ca_wdt_enable()`.
4. `ca_wdt_reset_cause()` indicates whether the last reset was caused by a watchdog; use `clear = true` to clear the flags.
5. **The real test is the reset**: the example triggers it and verifies the cause upon reboot.

Chapter 34

CPU Temperature

34.1 Example 11 — Core Temperature Monitor with Watchdog Protection

34.1.1 Introduction

In embedded systems, MCU overheating can cause malfunctions, performance degradation, or permanent damage. This example demonstrates how to use the **internal temperature sensor** to monitor the core temperature and trigger automatic protections.

In this example, we will see how to:

1. Initialize the internal temperature sensor
2. Create a temperature monitoring task
3. Implement a two-threshold protection:
   - **75°C**: core clock reduction
   - **85°C**: stop watchdog reload → system reset
4. Manage recovery when the temperature drops

34.1.2 Thermal Protection Architecture

The system implements a **two-threshold** protection with hysteresis:

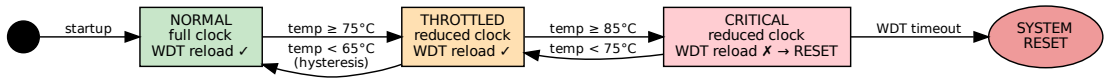


Figure 34.1: Thermal protection state machine

Why Two Thresholds?

1. **75°C Threshold (Clock Reduction)**: First level of protection. By reducing the clock, power consumption and thermal dissipation are decreased, allowing the system to cool down without interrupting operations.
2. **85°C Threshold (WDT Reset)**: Second level of protection. If the temperature continues to rise despite the clock reduction, the system is forced to reset to prevent permanent damage.

3. **Hysteresis (65°C)**: The recovery temperature is lower than the activation temperature to avoid frequent state oscillations when the temperature is near the threshold.

34.1.3 Source Code

The file is located in `examples/example_11/ca_os_ex11.c`:

```
#include "caos.h"
#include "temp_sensor.h"

/* Configurazione */
#define TEMP_CLOCK_THRESHOLD      75      // 75 C  - soglia riduzione clock
#define TEMP_WDT_RESET_THRESHOLD  85      // 85 C  - soglia reset WDT
#define TEMP_RECOVERY_THRESHOLD   65      // 65 C  - soglia di ripristino
#define TEMP_READ_PERIOD_MS       500     // 500ms
#define WDT_TIMEOUT_MS            2000    // 2 secondi

/* Stati del sistema termico */
typedef enum {
    THERM_STATE_NORMAL = 0,       // T < 65 C
    THERM_STATE_CLOCK_REDUCE,     // 75 C  <= T < 85 C
    THERM_STATE_WDT_RESET,        // T >= 85 C      reset inevitabile
} thermal_state_t;

static volatile thermal_state_t g_thermal_state = THERM_STATE_NORMAL;
static volatile bool g_wdt_stopped = false;
static volatile bool g_clock_reduced = false;

/* Task di monitoraggio temperatura */
static u32 task_temp_monitor(void *arg) {
    (void)arg;
    while (1) {
        i32 current_temp = ca_temp_read();

        /* Determina stato in base alla temperatura */
        if (current_temp >= TEMP_WDT_RESET_THRESHOLD) {
            new_state = THERM_STATE_WDT_RESET;
        } else if (current_temp >= TEMP_CLOCK_THRESHOLD) {
            new_state = THERM_STATE_CLOCK_REDUCE;
        } else if (current_temp <= TEMP_RECOVERY_THRESHOLD) {
            new_state = THERM_STATE_NORMAL;
        } else {
            new_state = g_thermal_state;
        }

        /* Gestisci transizione di stato */
        if (new_state != g_thermal_state) {
            g_thermal_state = new_state;

            switch (g_thermal_state) {
                case THERM_STATE_WDT_RESET:
                    g_wdt_stopped = true;
                    /* Riduci anche il clock se non ancora ridotto */
                    if (!g_clock_reduced) {
                        ca_os_clock_reduce_freq(50);
                        g_clock_reduced = true;
                    }
                    break;

                case THERM_STATE_CLOCK_REDUCE:
                    if (!g_clock_reduced) {
                        ca_os_clock_reduce_freq(50);
                        g_clock_reduced = true;
```

```
                    }
                    break;

                case THERM_STATE_NORMAL:
                    if (g_wdt_stopped)
                        g_wdt_stopped = false;
                    if (g_clock_reduced) {
                        ca_os_clock_restore_freq();
                        g_clock_reduced = false;
                    }
                    break;
            }
        }

        ca_os_sleep(TEMP_READ_PERIOD_MS);
    }
}

/* Task di reload watchdog */
static u32 task_wdt_reload(void *arg) {
    (void)arg;
    while (1) {
        /* Se g_wdt_stopped    true, NON ricaricare     reset sistema */
        if (!g_wdt_stopped) {
            ca_wdt_reload();
        }
        ca_os_sleep(WDT_TIMEOUT_MS / 2);
    }
}

char buf[DEFAULT_MSG_SIZE];
void ca_os_main(void) {
    /* 1. Inizializza sensore temperatura */
    if (!ca_temp_init()) {
        PRINTF_N(buf,sizeof(buf),"ERROR: Temperature sensor init FAILED!");
        return;
    }

    /* 2. Inizializza WDT */
    if (!ca_wdt_init(WDT_TIMEOUT_MS, CA_WDT_MODE_RESET, CA_WDT_TYPE_IWDG)) {
        PRINTF_N(buf,sizeof(buf),"ERROR: WDT initialization FAILED!");
        return;
    }

    /* 3. Crea task */
    ca_os_job("temp_mon", CA_TH_LEVEL3, task_temp_monitor, NULL,
              STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);
    ca_os_job("wdt_reload", CA_TH_LEVEL2, task_wdt_reload, NULL,
              STACK_SMALL, CA_JOB_STD, CA_FAULT_ASSERT);

    /* 4. Abilita WDT */
    if (!ca_wdt_enable()) {
        PRINTF_N(buf,sizeof(buf),"ERROR: WDT enable FAILED!");
        return;
    }

    /* 5. Avvia scheduler */
    ca_os_run(true);
}
```

34.1.4 CLI Commands

`ca_wdt_init()` automatically registers the WDT driver CLI node (if `EN_CLI_WDT=1`). In this example, the watchdog is initialized in `ca_os_main`, so the command is available as soon as the scheduler starts.

Command	Description
`wdt info`	Shows status, type, mode, and configured timeout
`sensor`	Shows the current core temperature (built-in `cli.c`); on the H563, it also shows the DTS temperature
`dts`	H563: DTS versus ADC sensor, with `DTS CMP PASS/FAIL` verdict
`dts alarm`	H563: DTS threshold alarm test, `DTS ALARM PASS/FAIL` verdict

Typical output during operation:

```
c.a.O.S > wdt info
wdt: ENABLED  type=IWDG  mode=RESET  timeout=2000 ms

c.a.O.S > sensor
Core temperature : 47 Celsius
```

Useful for monitoring the running system without interrupting the thermal protection cycle.

34.1.5 The digital DTS sensor (H563)

The H563 has a second sensor, the **DTS** (RM0481 ch. 27), independent of the ADC: an oscillator whose frequency increases with temperature, measured by counting PCLK cycles. The calibration is factory-written in the block registers (frequency at 30 °C and slope in Hz/°C), so it does not depend on VREF+ or VREFINT and does not occupy ADC1. It is not present on other families: the functions indicate this and the rest of the example remains unchanged.

```
bool ca_temp_dts_present(void);
i32  ca_temp_read_dts(void);     /* centi-degC, average of 16 measurements */
bool ca_temp_set_alarm(i32 cold_c, i32 hot_c, ca_temp_alarm_cb_t cb);
void ca_temp_retune(void);       /* after a core clock change */
```

- **Reading**: a measurement oscillates by about 1 °C; `ca_temp_read_dts` averages 16 of them in about 2 ms (each measurement recalibrates itself). On the bench, the DTS reads 1-2 °C above the ADC sensor, within the sum of the two datasheet errors (DS14258: DTS from -7 to +2 °C, ADC 3 °C linearity).
- **Alarm**: the DTS compares each measurement with two hardware thresholds and generates an interrupt, without a task reading it. The callback runs **in the interrupt** and receives the side exceeded (`hot` true above `hot_c`, false below `cold_c`). Each side triggers **once** and then remains off until the next `ca_temp_set_alarm`; `cb` to `NULL` turns off the alarm.
- **Clock**: the thresholds are PCLK counts. By halving the clock, the same heat would give half the count, i.e., a "hotter" die, and the alarm would trigger falsely. `ca_pwr_set_freq` /`ca_pwr_restore_freq` call `ca_temp_retune()`, which reconstructs the thresholds; `therm_set_clock()` of this example also does this.

The reference is PCLK and not the LSE because, over 15 LSE periods, the count moves by about 1 unit per °C against the ~15 of PCLK. Calibration would require a clock under 1 MHz

and the divider stops at 127, so at 250 MHz it works at about 2 MHz: measured on 24/10/2026, the readings remain within 1-2 °C of the ADC sensor.

`dts alarm` tests three cases: a high threshold below the current temperature must trigger (hot side, only once); a window around the temperature must remain silent; the same window must also remain silent with the clock halved.

```
c.a.O.S > dts
DTS temp   29.87 C
DTS adc   28 C   read 2106 us
DTS CMP PASS (DTS-ADC 1 C, window -10..5)

c.a.O.S > dts alarm
DTS alarm -1..26 C at 29 C: hot 1 cold 0 (want 1 0)
DTS alarm -1..59 C at 29 C: hot 0 cold 0 (want 0 0)
DTS alarm across half clock: hot 0 cold 0 (want 0 0)
DTS ALARM PASS
```

34.1.6 Detailed analysis

Threshold Configuration

```
#define TEMP_CLOCK_THRESHOLD      75      // 75 C  - soglia riduzione clock
#define TEMP_WDT_RESET_THRESHOLD  85      // 85 C  - soglia reset WDT
#define TEMP_RECOVERY_THRESHOLD   65      // 65 C  - soglia di ripristino
```

- `TEMP_CLOCK_THRESHOLD`: When the temperature reaches 75°C, the core clock is halved to reduce thermal dissipation.
- `TEMP_WDT_RESET_THRESHOLD`: When the temperature reaches 85°C, the watchdog reload is stopped, causing a system reset after ~2 seconds.
- `TEMP_RECOVERY_THRESHOLD`: When the temperature drops below 65°C, all protections are deactivated and the system returns to normal configuration.

Thermal System States

```
typedef enum {
    THERM_STATE_NORMAL = 0,       // T < 65 C  - funzionamento normale
    THERM_STATE_CLOCK_REDUCE,     // 75 C  <= T < 85 C  - clock ridotto
    THERM_STATE_WDT_RESET,        // T >= 85 C  - reset inevitabile
} thermal_state_t;
```

Note: once the 85°C threshold is exceeded, a reset is inevitable. The WDT is no longer reloaded and the system resets after the timeout (~2s). There is no recovery from this state — the protection is intentionally irreversible.

Temperature Monitoring Task

```
static u32 task_temp_monitor(void *arg) {
    (void)arg;
    while (1) {
        i32 current_temp = ca_temp_read();

        /* Determina stato in base alla temperatura */
        if (current_temp >= TEMP_WDT_RESET_THRESHOLD) {
            new_state = THERM_STATE_WDT_RESET;
        } else if (current_temp >= TEMP_CLOCK_THRESHOLD) {
            new_state = THERM_STATE_CLOCK_REDUCE;
        } else if (current_temp <= TEMP_RECOVERY_THRESHOLD) {
            new_state = THERM_STATE_NORMAL;
```

```
        } else {
            new_state = g_thermal_state;
        }

        /* Gestisci transizione di stato */
        if (new_state != g_thermal_state) {
            // ... applica protezioni ...
        }

        ca_os_sleep(TEMP_READ_PERIOD_MS);
    }
}
```

The task: 1. Reads the temperature every 500ms via `ca_temp_read()` 2. Determines the new state based on the thresholds 3. Manages state transitions by activating/deactivating protections

Watchdog Reload Task

```
static u32 task_wdt_reload(void *arg) {
    (void)arg;
    while (1) {
        /* Se g_wdt_stopped    true, NON ricaricare     reset sistema */
        if (!g_wdt_stopped) {
            ca_wdt_reload();
        }
        ca_os_sleep(WDT_TIMEOUT_MS / 2);
    }
}
```

This task checks the `g_wdt_stopped` variable: - If **false**: reloads the watchdog normally - If **true** (T >= 85°C): does **NOT** reload the watchdog → the system is reset after `WDT_TIMEOUT_MS`

Clock Reduction Functions

```
/* Da hw/<famiglia>/common/clock.c */
u32 ca_os_clock_reduce_freq(u16 scale);    // Riduce clock al X% (es: 50 = 50%)
u32 ca_os_clock_restore_freq(void);        // Ripristina clock originale
```

These functions are implemented in the `clock.c` file specific to the MCU family.

The functions are implemented in `hw/<famiglia>/common/clock.c` for each MCU family. They operate on the AHB prescaler (HPRE) for F1xx/F4xx/F7xx, HPRE in CFGR2 for H5xx, and D1CPRE for H7xx. They select the hardware divisor closest to the requested percentage among those available (1, 2, 4, 8, 16, 64, 128, 256, 512).

`ca_os_clock_reduce_freq(50)` reduces the clock to 50% of the nominal frequency (defined by `SYSCLK_FREQ_HZ` in `objmap.h`). The PLL remains unchanged — no clock glitches.

34.1.7 Expected Output

```
================================================
  TEST 11: Core Temperature Monitor + WDT
================================================

Initializing temperature sensor...
Temperature sensor initialized

Initializing WDT:
  - Timeout: 2000 ms
  - Type: IWDG
  - Mode: RESET
WDT initialized successfully
```

```
13
14 TEMP Monitor started
15   - Clock reduce threshold: 75 C
16   - WDT reset threshold: 85 C
17   - Recovery threshold: 65 C
18   - Read period: 500 ms
19
20 WDT Reload task started
21 WDT ENABLED - system protected
22 Temperature monitoring active
23 Clock reduce threshold: 75 C
24 WDT reset threshold: 85 C
25 Recovery threshold: 65 C
26
27 NORMAL: Temp=45 C  - System OK
28 Temp=46 C , State=0, Clk=0, WDT=0
29 Temp=47 C , State=0, Clk=0, WDT=0
30 ...
31 WARNING: Temp=75 C  - Reducing clock
32   -> Core clock: REDUCED (50%)
33 Temp=76 C , State=1, Clk=1, WDT=0
34 ...
35 NORMAL: Temp=64 C  - System OK          <-- recupero sotto 65 C
36   -> Core clock: RESTORED
37 ...
38 !!! WDT RESET: Temp=85 C  - Stopping WDT reload!
39   -> Core clock: REDUCED (50%)
40   -> WDT reload: STOPPED (reset incoming!)
41 [RESET DELLA MCU DOPO ~2 SECONDI]
```

34.1.8 Test Variants

Test 1: Normal Operation

Keep the temperature below 65°C (controlled environment or cold MCU).

Expected result: No protection activated, system stable.

Test 2: Clock Reduction Activation

Heat the MCU up to 75-84°C (e.g., execute intensive code, increase clock, hot environment).

Expected result: - Print “WARNING: Temp=XX°C - Reducing clock” - Clock halved (if implemented) - System continues to operate

Test 3: WDT Reset Activation

Heat the MCU above 85°C.

Expected result: - Print “!!! WDT RESET: Temp=XX°C - Stopping WDT reload!” - MCU reset after ~2 seconds - Flag `RCC_CSR.IWDGRSTF` set after reset

Test 4: Recovery from Clock Reduce

After clock reduction activation (75-84°C), cool the MCU below 65°C.

Expected result: - Print “NORMAL: Temp=XX°C - System OK” - Original clock restored - System returns to full frequency operation

Note: If the temperature has exceeded 85°C (WDT_RESET), recovery is not possible — the system is reset by the watchdog.

34.1.9 Threshold Considerations

MCU Family	Max Operating Temp	Max Junction Temp	Recommended Threshold
STM32F1xx	85°C	150°C	70°C / 80°C
STM32F4xx	105°C	150°C	75°C / 85°C
STM32F7xx	105°C	150°C	75°C / 85°C
STM32H5xx	125°C	150°C	80°C / 90°C
STM32H7xx	125°C	150°C	80°C / 90°C

Note: Thresholds can be adjusted based on the specific application and expected operating conditions.

34.1.10 Thermal System Debugging

Variable Monitoring

In STM32CubeIDE, add the following variables to the "Expressions" view:

- `g_thermal_state`: Current state (0=normal, 1=clock, 2=wdt)
- `g_wdt_stopped`: true if WDT is not reloaded
- `g_clock_reduced`: true if clock is reduced
- `g_clock_events`: Number of clock reduction activations
- `g_wdt_events`: Number of WDT reset activations

Post-Reset Diagnostics

After a reset caused by overheating:

```
if (RCC->CSR & RCC_CSR_IWDGRSTF) {           /* F1, F4, F7, U5 */
    ca_stdout_put_n("Reset caused by IWDG (possible overheating)");
}
```

The example does not do this by itself; the register changes with the family (`RCC->RSR` on the H5 and on the H7), as explained in example 10.

34.1.11 Best Practices

1. Choosing Appropriate Thresholds

```
/* Per applicazioni automotive (temperature elevate) */
#define TEMP_CLOCK_THRESHOLD      80      // 80 C
#define TEMP_WDT_RESET_THRESHOLD  95      // 95 C

/* Per applicazioni consumer (temperature moderate) */
#define TEMP_CLOCK_THRESHOLD      70      // 70 C
#define TEMP_WDT_RESET_THRESHOLD  80      // 80 C
```

2. Verify Clock Implementation

The `ca_os_clock_reduce_freq()` and `ca_os_clock_restore_freq()` functions are already implemented in `hw/<famiglia>/common/clock.c`.

For families that do not support them, implement the functions in the specific clock file.

3. Logging Thermal Events

```
case THERM_STATE_WDT_RESET:
    g_wdt_events++;
    ca_stdout_printf_n("!!! WDT RESET: Temp=%dC - Stopping WDT reload!",
                       current_temp);
```

4. Testing in Real-World Conditions

- Perform tests in a thermal chamber
- Verify system response times
- Validate thresholds with real workloads

34.1.12 Summary

1. **The internal temperature sensor** provides accurate core temperature readings (±1°C after calibration).

2. **The two-threshold protection** allows for gradual overheating management:

   - 75°C: clock reduction (first cooling attempt)
   - 85°C: system reset (extreme protection)

3. **Hysteresis** (65°C recovery) prevents frequent oscillations when the temperature is near the thresholds.

4. **The watchdog timer** is used as a controlled reset mechanism: when the temperature is critical, the reload is stopped, causing a reset after the configured timeout.

5. **The `ca_os_clock_reduce_freq()` and `ca_os_clock_restore_freq()` functions** are implemented in `hw/<famiglia>/common/clock.c` and manage the MCU clock reduction/restoration.

6. **Integrated CLI**: `wdt info` shows the watchdog status; `sensor` shows the current temperature. Both are available without additional application code.

Chapter 35

Extending the System Command Line

35.1 Example 12 — Extending the CLI with user commands

35.1.1 Introduction

caOS includes a system CLI (Command Line Interface) accessible via UART/USB_CDC. Built-in commands provide generic diagnostic tools: task status, memory usage, stack, MPU. Each application can add its own commands by registering a node in the CLI's **chain of responsibility**.

In this example, we will see how to:

1. Define a CLI handler with the correct signature
2. Register it in the chain with `ca_cli_register()`
3. Respond with application output (task statistics and shared counters)
4. Coexist correctly with built-in commands and other handlers in the chain

The application context is intentionally simple: three concurrent tasks contending for a mutex with `ca_mutex_trylock`. The educational focus is entirely on the CLI.

35.1.2 The system CLI

The caOS CLI is a dedicated task that receives characters via STDIN (uart, usb_cdc, etc.) byte by byte, accumulates the line, and, upon receiving `\r` or `\n`, sends the command to the internal queue. The CLI thread processes each command through the chain of registered handlers.

Built-in commands

Command	Description
`help`	List commands from all handlers in the chain
`top`	CPU + memory + stack usage
`sched`	CPU breakdown since last call: context switch, task, idle, free
`taskall`	All tasks (status, priority, stack)
`task <n>`	Specific task detail
`free`	Available heap memory
`canary`	Stack canary status (if MPU is present)
`mpu`	Configured MPU regions
`stack <n>`	Address and stack map of a task

Command	Description
`memlist`	Allocated heap blocks
`hexdump <addr> [lines]`	Memory dump in hexadecimal

Chain of Responsibility

The CLI uses the **chain of responsibility** pattern: each registered handler receives the command and decides whether to handle it or pass it to the next one.

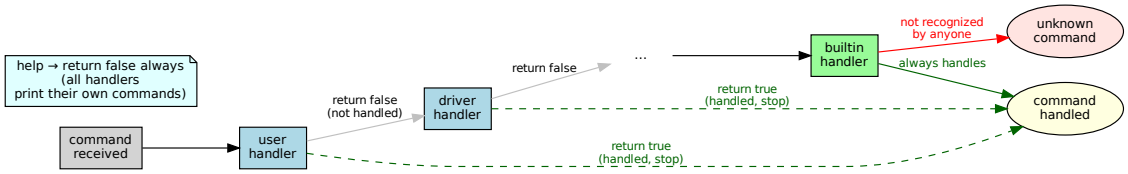


Figure 35.1: CLI Chain of Responsibility

Rules: - `return true` — command handled, the chain stops - `return false` — command not recognized, pass to the next handler - `help` — must always return `false` to allow all handlers to show their commands

`ca_cli_register()` inserts the node **at the head** of the chain (prepend), so the last one registered is queried first.

API

```
typedef struct ca_cli_node {
    bool (*handler)(c_str *cmd);   /* true = command handled */
    struct ca_cli_node *next;
} ca_cli_node_t;

void ca_cli_register(ca_cli_node_t *node);  /* prepend to the chain */
```

The node must be **static** (duration for the entire execution). Registration must be done **before** `ca_os_run()`, typically in `ca_os_main()`.

35.1.3 Source code

The file is located in `examples/example_12/ca_os_ex12.c`:

```
#include "caos.h"

static u32 killer;
static u32 shared_counter;
static u32 counter_t1;
static u32 counter_t2;
static u32 counter_t3;
static ca_mutex_t mtx;

static u32 task_1_exec(void *arg) {
    const u32 p1 = (u32)(uintptr_t)arg;
    const u32 idx = ca_os_thid_from_handle(ca_os_self_handle());
    ca_os_sleep(700);
    ca_stdout_printf_n("TASK t1_w1 started: idx=%u p1=%u", idx, p1);
```

```
    while(killer) {
        counter_t1++;
        if (ca_mutex_trylock(&mtx)) {
            shared_counter++;
            ca_mutex_unlock(&mtx);
        }
        ca_os_sleep(500);
    }
    return 0;
}

static u32 task_2_exec(void *arg) {
    const u32 p1 = (u32)(uintptr_t)arg;
    const u32 idx = ca_os_thid_from_handle(ca_os_self_handle());
    ca_os_sleep(700);
    ca_stdout_printf_n("TASK t1_w2 started: idx=%u p1=%u", idx, p1);
    while(killer) {
        counter_t2++;
        if (ca_mutex_trylock(&mtx)) {
            shared_counter++;
            ca_mutex_unlock(&mtx);
        }
        ca_os_sleep(700);
    }
    return 0;
}

static u32 task_3_exec(void *arg) {
    const u32 p1 = (u32)(uintptr_t)arg;
    const u32 idx = ca_os_thid_from_handle(ca_os_self_handle());
    ca_stdout_printf_n("TASK t1_w3 started: idx=%u p1=%u", idx, p1);
    while(killer) {
        counter_t3++;
        if (ca_mutex_trylock(&mtx)) {
            shared_counter++;
            ca_mutex_unlock(&mtx);
        }
        ca_os_sleep(900);
    }
    return 0;
}

static bool user_cli_handler(c_str *pcmd)
{
    c_str cmd = *pcmd;
    const char *p = ca_skip_sp(cmd->data);

    if (ca_strncmp(p, "help", 4) == 0) {
        CLI_PUT_N("show : show task counters");
        return false;   /* continue the chain: show the other help too */
    }
    if (ca_strncmp(p, "show", 4) == 0) {
        CLI_PUT_N("--- task counters ---");
        CLI_PRINTF_N("shared  : %u", (u32)shared_counter);
        CLI_PRINTF_N("t1_w1   : %u", (u32)counter_t1);
        CLI_PRINTF_N("t1_w2   : %u", (u32)counter_t2);
        CLI_PRINTF_N("t1_w3   : %u", (u32)counter_t3);
        return true;
    }
    return false;
}

static ca_cli_node_t s_user_cli_node = { user_cli_handler, NULL };
```

```
78
79 void ca_os_configure(void) {
80     /* Initialize temperature sensor */
81
82     /* Log system */
83     EVAL_STDOUT(
84         ca_stdout_init();
85         ca_stdout_set_sender(STDOUT);
86         ca_stdout_create_thread();
87     )
88     /* CLI */
89     EVAL_CLI(
90         ca_cli_init(4096);
91         ca_cli_set_sender(STDOUT);
92         ca_cli_create_thread();
93     )
94     /* Onboard LEDs: registers the "led" CLI node, so it has to run after
95      * ca_cli_init() above - the flags alone leave the command unknown. */
96     EVAL_LED( ca_led_init(); )
97     /* DBG */
98     EVAL_USB_CDC_STD(ca_usb_dev_mng_cdc_cli();)
99 }
100
101 void ca_os_main(void) {
102     killer = 1;
103     shared_counter = counter_t1 = counter_t2 = counter_t3 = 0;
104
105 #if ( MCU==F1xx && MCU_ID==103 )
106     ca_os_job("t1_w1", CA_TH_LEVEL2, task_1_exec, (void *)(uintptr_t)1,
    STACK_SMALL,  CA_JOB_STD, CA_FAULT_ASSERT);
107     ca_os_job("t1_w2", CA_TH_LEVEL2, task_2_exec, (void *)(uintptr_t)3,
    STACK_SMALL,  CA_JOB_STD, CA_FAULT_ASSERT);
108     ca_os_job("t1_w3", CA_TH_LEVEL2, task_3_exec, (void *)(uintptr_t)5,
    STACK_SMALL,  CA_JOB_STD, CA_FAULT_ASSERT);
109 #else
110     ca_os_job("t1_w1", CA_TH_LEVEL2, task_1_exec, (void *)(uintptr_t)1,
    STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);
111     ca_os_job("t1_w2", CA_TH_LEVEL2, task_2_exec, (void *)(uintptr_t)3,
    STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);
112     ca_os_job("t1_w3", CA_TH_LEVEL2, task_3_exec, (void *)(uintptr_t)5,
    STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);
113 #endif
114
115     EVAL_CLI( ca_cli_register(&s_user_cli_node); )
116     ca_os_run(true);
117 }
```

Note: the `ca_os_configure()` function is documented in detail in chapter `S2_07_00_ca_os_configure.md`.

35.1.4 Detailed analysis

CLI Handler `user_cli_handler`

```
1 static bool user_cli_handler(c_str *pcmd)
2 {
3     c_str cmd = *pcmd;
4     const char *p = ca_skip_sp(cmd->data);
5
6     if (ca_strncmp(p, "help", 4) == 0) {
7         CLI_PUT_N("show : show task counters");
8         return false;   /* let the other handlers add their help */
```

```
    }
    if (ca_strncmp(p, "show", 4) == 0) {
        CLI_PUT_N("--- task counters ---");
        CLI_PRINTF_N("shared  : %u", (u32)shared_counter);
        CLI_PRINTF_N("t1_w1   : %u", (u32)counter_t1);
        CLI_PRINTF_N("t1_w2   : %u", (u32)counter_t2);
        CLI_PRINTF_N("t1_w3   : %u", (u32)counter_t3);
        return true;    /* command handled, stop the chain */
    }
    return false;       /* not recognized, pass to the next handler */
}
```

Key points:

- **Signature**: `bool (*handler)(c_str *pcmd)` — pointer to pointer, no copy
- `ca_skip_sp`: skips any leading spaces before comparing
- `help` → `return false`: mandatory to allow all handlers in the chain to print their commands
- **Output via `CLI_PUT_N` / `CLI_PRINTF_N`**: these macros include `ca_os_yield()` and `ca_os_sleep(1)` — **never call them while holding a mutex**
- `return true`: stops the chain; the command is not propagated to subsequent handlers

Static node and registration

```
static ca_cli_node_t s_user_cli_node = { user_cli_handler, NULL };

EVAL_CLI( ca_cli_register(&s_user_cli_node); )
```

- The node must be **static**: `ca_cli_register` inserts it into the linked list without copying it
- `NULL` in the `next` field is updated by `ca_cli_register` at the time of insertion
- `EVAL_CLI(...)` compiles the code only if `ENABLE_CLI=1`, making it a no-op otherwise

Application context: mutex trylock

The three tasks contend for a mutex with `ca_mutex_trylock`, which is **non-blocking**: if the mutex is occupied, the task does not suspend but continues the loop.

```
counter_t1++;                       /* always incremented */
if (ca_mutex_trylock(&mtx)) {       /* only if the mutex is free */
    shared_counter++;
    ca_mutex_unlock(&mtx);
}
ca_os_sleep(500);
```

The `show` command allows you to observe at runtime how many times each task has executed its loop and how many times the mutex has been acquired.

Stack per MCU family

```
#if ( MCU==F1xx && MCU_ID==103 )
    ca_os_job(..., STACK_SMALL, ...);   /* F103: 20 KB SRAM in total */
#else
    ca_os_job(..., STACK_MEDIUM, ...);  /* H753 and others */
#endif
```

35.1.5 Adding more commands

To add commands, extend the handler with additional comparisons:

```
static bool user_cli_handler(c_str *pcmd)
{
    c_str cmd = *pcmd;
    const char *p = ca_skip_sp(cmd->data);

    if (ca_strncmp(p, "help", 4) == 0) {
        CLI_PUT_N("show : show task counters");
        CLI_PUT_N("reset : stop the tasks");
        return false;
    }
    if (ca_strncmp(p, "show", 4) == 0) {
        /* ... print statistics ... */
        return true;
    }
    if (ca_strncmp(p, "reset", 5) == 0) {
        killer = 0;
        CLI_PUT_N("tasks stopped");
        return true;
    }
    return false;
}
```

For applications with many separate commands on different modules, you can register **multiple nodes** in the chain — each one handles its own commands independently of the others:

```
static ca_cli_node_t s_net_cli_node   = { net_cli_handler,   NULL };
static ca_cli_node_t s_sensor_cli_node = { sensor_cli_handler, NULL };

ca_cli_register(&s_net_cli_node);
ca_cli_register(&s_sensor_cli_node);
```

35.1.6 Expected output

Startup:

```
TASK t1_w3 started: idx=0 p1=5
TASK t1_w1 started: idx=0 p1=1
TASK t1_w2 started: idx=0 p1=3
```

By typing `help` on the UART (all handlers in the chain contribute):

```
c.a.O.S > help
show : show task counters
top        : cpu + mem + stack usage
sched      : cpu split since last call: switch, tasks, idle, free
banner     : show os banner
taskall    : show all tasks
task <n>   : show task N
free       : Show global memory usage
...
```

By typing `show`:

```
c.a.O.S > show
--- task counters ---
shared  : 47
t1_w1   : 22
t1_w2   : 16
t1_w3   : 12
```

By typing an unknown command (no handler recognizes it):

```
c.a.O.S > xyz
xyz:3 huh!
```

35.1.7 Required configuration in `os_config.h`

```
#define ENABLE_CLI       1    /* the CLI must be enabled */
#define ENABLE_DBG_UART 1    /* UART needed as the I/O channel */
```

Without `ENABLE_CLI 1`, `EVAL_CLI(...)` becomes a no-op and `ca_cli_register` does nothing.

35.1.8 Summary

1. `ca_cli_register(&node)` inserts a static node with handler `bool (*)(c_str *pcmd)` at the head of the chain.
2. `return true` stops the chain — the command has been handled.
3. `return false` propagates to the next handler — command not recognized or `help`.
4. `help` **must always return** `false` to allow all handlers to print their commands.
5. **Output via** `CLI_PUT_N` / `CLI_PRINTF_N` — never under mutex, they include `yield` + `sleep`.
6. **Multiple nodes** can be registered independently: each module handles its own commands without knowing about the others.

Chapter 36

Execution Time Profiling with DWT CYCCNT

36.1 Example 13 — Profiling and task watchdog

36.1.1 Introduction

This example demonstrates two features of the caOS scheduler module:

1. **Execution time profiling** via `ca_measure_start`/`ca_measure_stop`, based on clock cycles consumed by the task (not real time).
2. **Individual task watchdog**: a task that does not call `ca_os_thread_reset_watchdog` within the set limit is automatically terminated by the scheduler.

36.1.2 The `measure` module

API (`caos/inc/measure.h`)

```
void ca_measure_start(u32 *p_snap);
void ca_measure_stop(volatile u32 *p);
```

Behavior

`ca_measure_start` saves the current value of the task's `runtime_total` (clock cycles accumulated since the task started). `ca_measure_stop` calculates the difference and writes it to the same pointer. The result is the number of clock cycles spent by the task between `start` and `stop`.

```
// Implementazione
void ca_measure_start(u32 *p) {
    if (p) *p = ca_scheduler_get_runtime_total();
}

void ca_measure_stop(volatile u32 *p) {
    u32 res = 0;
    if (p) res = ca_scheduler_get_runtime_total() - (*p);
    *p = res;
}
```

Difference compared to the direct DWT approach

Aspect	Direct DWT→CYCCNT	`ca_measure` (runtime_total)
Unit	wall-clock cycles	task-only cycles
Preemption	included in the measurement	excluded (task cycles only)
Interrupts	included	excluded
Initialization	`ca_measure_init()` (deprecated)	automatic (caOS init)
Nested measurements	no	yes (independent local variables)

The main advantage: the measurement is **deterministic** and does not depend on how much other tasks or IRQs have interrupted the task during the profiled section.

Conversion to microseconds

The result of `ca_measure_stop` is in clock cycles. Conversion to ms:

```
u32 time_ms = (u32)((elapsed * 1000) / ca_os_clock_get_freq());
```

`ca_os_clock_get_freq()` returns `SystemCoreClock` in Hz.

Nested measurements

Since `p_snap` is a local variable, independent measurements can also be performed in recursive calls or in different functions:

```
u32 snap_outer, snap_inner;
ca_measure_start(&snap_outer);
    ca_measure_start(&snap_inner);
    inner_function();
    ca_measure_stop(&snap_inner);   // inner_function cycles
outer_function();
ca_measure_stop(&snap_outer);       // total cycles (inner + outer)
```

36.1.3 Task watchdog

API

```
void ca_os_thread_set_watchdog(u32 clock_cycles);   // set the threshold
void ca_os_thread_reset_watchdog(void);              // reset runtime_total
```

Mechanism

The scheduler accumulates the task's active clock cycles in `runtime_total`. At each context switch, it verifies:

```
if (runtime_wdt > 0 && runtime_total > runtime_wdt)
    task->status = CA_ST_TH_SIG_WDT;  // the task is terminated
```

`ca_os_thread_set_watchdog(N)` sets `runtime_wdt = N`. The task must call `ca_os_thread_reset_watchdog()` periodically (which resets `runtime_total`) to avoid termination.

Note: the limit is in active task clock cycles (not real time). A task that sleeps with `ca_os_sleep` does not consume cycles and does not advance towards the threshold. The value must be calibrated experimentally, taking into account the expected load.

36.1.4 Source code

```c
#include "caos.h"

#if EN_MEASURE

static u32 g_elapsed_us;
static volatile u32 g_run_count;

static void work_function(void)
{
    volatile u32 x = 1U;
    for (u32 i = 0U; i < 50000U; i++) {
        x = x * 3U + 1U;
    }
    (void)x;
}

static u32 task_measure_exec(void *arg)
{
    (void)arg;
    u32 elapsed;
    while (1) {
        ca_measure_start(&elapsed);
        work_function();
        ca_measure_stop(&elapsed);
        g_elapsed_us = elapsed;
        g_run_count++;
        ca_os_sleep(1000);
    }
    return 0U;
}

static u32 task_loop_fail(void *arg)
{
    const u32 p1 = (u32)(uintptr_t)arg;
    ca_os_thread_set_watchdog(p1);   // p1 = 16000000 cycles
    while (1) {
        while (1) {
            ca_os_sleep(10);         // never resets the watchdog
        }
        ca_os_thread_reset_watchdog();
    }
    return 0U;
}

static bool user_cli_handler(c_str *pcmd)
{
    const char *p = ca_skip_sp((*pcmd)->data);
    if (ca_strncmp(p, "help", 4) == 0) {
        CLI_PUT_N("show : elapsed time by example_13");
        return false;
    }
    if (ca_strncmp(p, "show", 4) == 0) {
        u32 time_ms = (u32)((g_elapsed_us * 1000) / ca_os_clock_get_freq());
        CLI_PRINTF_N("run count : %u", g_run_count);
        CLI_PRINTF_N("last us   : %u clocks %u ms", g_elapsed_us, time_ms);
        return true;
    }
    return false;
}
static ca_cli_node_t s_user_cli_node = { user_cli_handler, NULL };

```

```
#endif /* EN_MEASURE */

void ca_os_configure(void)
{


    EVAL_STDOUT(
        ca_stdout_init();
        ca_stdout_set_sender(STDOUT);
        ca_stdout_create_thread();
    )
    EVAL_CLI(
        ca_cli_init(4096);
        ca_cli_set_sender(STDOUT);
        ca_cli_create_thread();
    )
    EVAL_USB_CDC_STD(ca_usb_dev_mng_cdc_cli();)


}

void ca_os_main(void)
{
    EVAL_MEASURE(
        g_elapsed_us = 0U;
        g_run_count  = 0U;
        ca_os_job("measure", CA_TH_LEVEL2, task_measure_exec, NULL,
                  STACK_SMALL, CA_JOB_STD, CA_FAULT_ASSERT);
        ca_os_job("loop", CA_TH_LEVEL4, task_loop_fail, (void *)(uintptr_t)
    16000000,
                  STACK_SMALL, CA_JOB_STD, CA_FAULT_ASSERT);
        EVAL_CLI( ca_cli_register(&s_user_cli_node); )
    )
    ca_os_run(true);
}
```

36.1.5 Detailed analysis

`task_measure_exec`

The task measures the clock cycles consumed by `work_function()` (50000 iterations of a volatile calculation). The variable `elapsed` is local: `ca_measure_start` saves the snapshot of `runtime_total` into it, and `ca_measure_stop` writes the difference to it. The result is directly in clock cycles (not microseconds). The CLI converts to ms via `ca_os_clock_get_freq()`.

`task_loop_fail` — watchdog demonstration

```
ca_os_thread_set_watchdog(16000000);   // 16M cycles     ~95ms on H753@480MHz
while (1) {
    while (1) { ca_os_sleep(10); }     // internal endless loop
    ca_os_thread_reset_watchdog();     // never reached
}
```

The internal loop never terminates, so `ca_os_thread_reset_watchdog()` is never called. When the task accumulates 16 million active cycles, the scheduler terminates it with `CA_ST_TH_SIG_WDT` and prints an error message.

Note: `ca_os_sleep` does not consume active cycles — the task sleeps and the scheduler does not increment `runtime_total`. The watchdog triggers only when active cycles exceed the threshold, not the elapsed real time.

Watchdog trigger time estimation

The real time before triggering depends on how many cycles the task consumes per iteration of the `ca_os_sleep(10)`. The sleep is negligible; active cycles are consumed mainly in the scheduler overhead during context switches. The 16M cycle threshold must therefore be calibrated experimentally.

EVAL_MEASURE

The `EVAL_MEASURE` guard enables both tasks and state variables only if `EN_MEASURE` is defined in `os_config.h`. Unlike older examples with `EVAL_F103`/`EVAL_F407`, here the selection is by functionality, not by family — `measure` works identically on all supported MCUs.

36.1.6 Expected output

show command via CLI

```
show
run count : 47
last us   : 284521 clocks  0 ms
```

The value in cycles depends on the frequency and the core pipeline. The conversion to ms will be 0 for short `work_function`; for longer functions, the correct value will appear.

top -r command

```
top -r
task[00] 'idle      ' RUNNING  L1   CPU= 0.0% Clk:      0 Stack= 3.4%  total
    :12345678 wdt=0
task[01] 'stdout    ' BLOCKED  L2   CPU= 0.0% Clk:      0 Stack=12.0%  total
    :456789 wdt=0
task[02] 'measure   ' BLOCKED  L2   CPU= 0.1% Clk: 284521 Stack=18.0%  total
    :9876543 wdt=0
task[03] 'loop      ' REMOVED  L4   CPU= 0.0% Clk:      0 Stack= 0.0%  total
    :16000123 wdt=16000000
```

The task `loop` appears as REMOVED with `total` having exceeded `wdt`.

Watchdog message

```
ERROR : Task[3] 'loop':task watchdog on! 12345
```

36.1.7 Required configuration in os_config.h

```
#define EN_MEASURE  1    // abilita modulo measure + task watchdog
#define ENABLE_STDOUT   1
#define ENABLE_CLI      1    // optional: 'show' command
```

36.1.8 Summary

1. `ca_measure_start(u32 *p)` / `ca_measure_stop(volatile u32 *p)`: the measurement uses `runtime_total` of the task — it counts only active cycles, immune to preemption and IRQs.
2. **Nested measurements**: each local variable is an independent snapshot.
3. `ca_os_thread_set_watchdog(N)`: sets the threshold in active clock cycles. The task must call `ca_os_thread_reset_watchdog()` periodically.

4. **Watchdog vs sleep**: `ca_os_sleep` does not consume active cycles — the watchdog measures only the actual work of the task.

5. `EVAL_MEASURE`: portable guard, works on all supported MCU families.

Chapter 37

Streams

37.1 Example 14 — Stream Buffer with interactive CLI

37.1.1 Introduction

The **Stream Buffer** is a FIFO circular buffer for byte streams. Unlike the Message Buffer (which preserves message boundaries), the Stream treats data as a continuous flow of bytes, similar to a pipe.

In this example: - A **writer** task sends data into the stream (production) - A **reader** task receives data from the stream (consumption) - A **monitor** task displays the stream status in real-time - The **CLI** allows controlling prints and viewing statistics

Demonstrated features

Function	Description
`CA_STREAM_STATIC_DEFINE()`	Static buffer declaration in .bss
`ca_stream_send()` / `ca_stream_receive()`	Send/receive with timeout
`ca_stream_bytes_available()`	Bytes available for reading
`ca_stream_space_available()`	Free space for writing
`ca_stream_set_trigger()`	Minimum threshold for receiver wake-up
`ca_stream_reset()`	Clear the buffer
`ca_stream_is_empty()` / `ca_stream_is_full()`	Buffer status query

CLI Commands

Command	Description
`start`	Enable task prints
`stop`	Disable task prints
`show`	Show stream statistics
`clear`	Reset stream buffer
`help`	List available commands

37.1.2 Example architecture

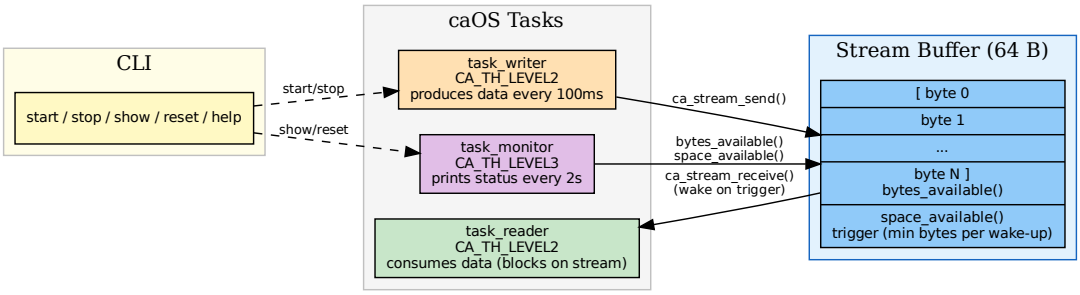


Figure 37.1: Stream Buffer architecture

37.1.3 Source code

The file is located in `examples/example_14/ca_os_ex14.c`:

```c
#include "caos.h"

#define STREAM_SIZE 64  // Dimensione buffer stream in byte

/* Stream buffer statico (allocazione in .bss) */
CA_STREAM_STATIC_DEFINE(data_stream, STREAM_SIZE);

static u32 running;
static u32 bytes_sent;
static u32 bytes_received;
static u32 writer_cycles;
static u32 reader_cycles;
static volatile u8 print_enabled;  // 0 = prints disabled, 1 = enabled

/* ============================================================================
 * TASK: Writer - sends data into the stream
 * ============================================================================
    */

static u32 task_writer(void *arg) {
    const u32 p1 = (u32)(uintptr_t)arg;
    const u32 idx = ca_os_thid_from_handle(ca_os_self_handle());

    if (print_enabled) {
        ca_stdout_printf_n("WRITER started: idx=%u p1=%u", idx, p1);
    }

    u8 sequence = 0;
    char tx_buffer[16];
    int tx_len;

    while (running) {
        // Prepare a block of data to send
        tx_len = ca_snprintf(tx_buffer, sizeof(tx_buffer), "DATA_%03d", sequence
   );

        // Send data into the stream (timeout 500ms)
        u32 written = ca_stream_send(&data_stream, (u8*)tx_buffer, tx_len, 500);
```

```
37
38          if (written > 0) {
39              bytes_sent += written;
40              writer_cycles++;
41              sequence++;
42
43              if (print_enabled) {
44                  ca_stdout_printf_n("WRITER: sent %u bytes (seq=%d)", written,
    sequence - 1);
45              }
46          } else {
47              if (print_enabled) {
48                  ca_stdout_printf_n("WRITER: timeout (stream full?)");
49              }
50          }
51
52          // Wait before the next send
53          ca_os_sleep(200);
54      }
55
56      return 0;
57  }
58
59  /* ============================================================================
60   * TASK: Reader - receives data from the stream
61   * ============================================================================
         */
62
63  static u32 task_reader(void *arg) {
64      const u32 p1 = (u32)(uintptr_t)arg;
65      const u32 idx = ca_os_thid_from_handle(ca_os_self_handle());
66
67      if (print_enabled) {
68          ca_stdout_printf_n("READER started: idx=%u p1=%u", idx, p1);
69      }
70
71      u8 rx_buffer[32];
72      u32 bytes_to_read;
73
74      while (running) {
75          // Determina quanti byte leggere (max 32 o quelli disponibili)
76          bytes_to_read = ca_stream_bytes_available(&data_stream);
77          if (bytes_to_read > sizeof(rx_buffer)) {
78              bytes_to_read = sizeof(rx_buffer);
79          }
80
81          if (bytes_to_read > 0) {
82              // Receive data from the stream (timeout 100ms)
83              u32 read = ca_stream_receive(&data_stream, rx_buffer, bytes_to_read,
     100);
84
85              if (read > 0) {
86                  bytes_received += read;
87                  reader_cycles++;
88
89                  // Null-terminate for printing
90                  rx_buffer[read] = '\0';
91
92                  if (print_enabled) {
93                      ca_stdout_printf_n("READER: recv %u bytes: '%s'", read,
    rx_buffer);
94                  }
95              }
```

```
96          } else {
97              // Stream empty, wait
98              ca_os_sleep(50);
99          }
100     }
101
102     return 0;
103 }
104
105 /* ============================================================================
106  * TASK: Monitor - mostra statistiche stream
107  * ============================================================================
        */
108
109 static u32 task_monitor(void *arg) {
110     const u32 p1 = (u32)(uintptr_t)arg;
111     const u32 idx = ca_os_thid_from_handle(ca_os_self_handle());
112
113     ca_os_sleep(1000);  // Wait for it to settle
114
115     if (print_enabled) {
116         ca_stdout_printf_n("MONITOR started: idx=%u p1=%u", idx, p1);
117     }
118
119     while (running) {
120         u32 available = ca_stream_bytes_available(&data_stream);
121         u32 free_space = ca_stream_space_available(&data_stream);
122         bool is_empty = ca_stream_is_empty(&data_stream);
123         bool is_full = ca_stream_is_full(&data_stream);
124
125         if (print_enabled) {
126             ca_stdout_printf_n("=== STREAM STATUS ===");
127             ca_stdout_printf_n("Bytes available : %u / %u", available,
    STREAM_SIZE);
128             ca_stdout_printf_n("Free space      : %u / %u", free_space,
    STREAM_SIZE);
129             ca_stdout_printf_n("Empty           : %s", is_empty ? "YES" : "NO ")
    ;
130             ca_stdout_printf_n("Full            : %s", is_full ? "YES" : "NO ");
131             ca_stdout_printf_n("Total sent      : %u bytes (%u cycles)",
    bytes_sent, writer_cycles);
132             ca_stdout_printf_n("Total recv      : %u bytes (%u cycles)",
    bytes_received, reader_cycles);
133             ca_stdout_printf_n("====================");
134         }
135
136         ca_os_sleep(3000);  // Print every 3 seconds
137     }
138
139     return 0;
140 }
141
142 /* ============================================================================
143  * CLI HOOK - user commands for stream
144  * ============================================================================
        */
145
146 static bool user_cli_handler(c_str *pcmd)
147 {
148     const char *p = ca_skip_sp((*pcmd)->data);
149     if (ca_strncmp(p, "help", 4) == 0) {
150         CLI_PUT_N("start | stop | show | clear");
151         return false;
```

```
152     }
153     if (ca_strncmp(p, "start", 5) == 0) {
154         print_enabled = 1;
155         CLI_PUT_N("Print ENABLED");
156         return true;
157     }
158     if (ca_strncmp(p, "stop", 4) == 0) {
159         print_enabled = 0;
160         CLI_PUT_N("Print DISABLED");
161         return true;
162     }
163     if (ca_strncmp(p, "show", 4) == 0) {
164         CLI_PUT_N("--- Stream Buffer Statistics:");
165         CLI_PRINTF_N("   Bytes sent    : %u", bytes_sent);
166         CLI_PRINTF_N("   Bytes received: %u", bytes_received);
167         CLI_PRINTF_N("   Writer cycles : %u", writer_cycles);
168         CLI_PRINTF_N("   Reader cycles : %u", reader_cycles);
169         CLI_PRINTF_N("   Available     : %u", ca_stream_bytes_available(&
    data_stream));
170         CLI_PRINTF_N("   Free space    : %u", ca_stream_space_available(&
    data_stream));
171         CLI_PRINTF_N("   Print status  : %s", print_enabled ? "ENABLED" : "
    DISABLED");
172         return true;
173     }
174     if (ca_strncmp(p, "clear", 5) == 0) {
175         ca_stream_reset(&data_stream);
176         CLI_PUT_N("Stream buffer cleared!");
177         return true;
178     }
179     return false;
180 }
181 static ca_cli_node_t s_user_cli_node = { user_cli_handler, NULL };
182 
183 /* ============================================================================
184  * ca_os_configure - module initialization
185  * ============================================================================
     */
186 
187 void ca_os_configure(void) {
188     /* Initialize temperature sensor */
189 
190     /* Log system */
191     EVAL_STDOUT(
192         ca_stdout_init();
193         ca_stdout_set_sender(STDOUT);
194         ca_stdout_create_thread();
195     )
196     /* CLI */
197     EVAL_CLI(
198         ca_cli_init(4096);
199         ca_cli_set_sender(STDOUT);
200         ca_cli_create_thread();
201     )
202 
203     /* USB CDC */
204     EVAL_USB_CDC_STD(ca_usb_dev_mng_cdc_cli();)
205 }
206 
207 /* ============================================================================
208  * ca_os_main - entry point
209  * ============================================================================
     */
```

```

void ca_os_main(void) {
    running = 1;
    bytes_sent = 0;
    bytes_received = 0;
    writer_cycles = 0;
    reader_cycles = 0;
    print_enabled = 0;  // Prints DISABLED by default

    /* Set the trigger level to 8 bytes to wake the receiver */
    ca_stream_set_trigger(&data_stream, 8);

    /* Create the writer task (medium priority) */
    ca_os_job("writer", CA_TH_LEVEL2, task_writer, NULL,
              STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);

    /* Create the reader task (medium priority) */
    ca_os_job("reader", CA_TH_LEVEL2, task_reader, NULL,
              STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);

    /* Create the monitor task (low priority) */
    ca_os_job("monitor", CA_TH_LEVEL1, task_monitor, NULL,
              STACK_SMALL, CA_JOB_STD, CA_FAULT_ASSERT);

    /* Register CLI hook */
    EVAL_CLI( ca_cli_register(&s_user_cli_node); )

    /* Start the scheduler */
    ca_os_run(true);
}
```

37.1.4 Detailed analysis

Static Stream Buffer

```
#define STREAM_SIZE 64
CA_STREAM_STATIC_DEFINE(data_stream, STREAM_SIZE);
```

The macro expands to:

```
static u8 data_stream_buf[64];
static ca_stream_buffer_t data_stream = {
    .buffer = data_stream_buf,
    .size = 64,
    .head = 0, .tail = 0, .count = 0,
    .lock = CA_SPINLOCK_IRQ_INIT,
    .waiting_task = 0xFFFFFFFF,
    .trigger_level = 1
};
```

Advantages: - Allocation in .bss (zero heap overhead) - Compile-time initialization - No risk of fragmentation

Trigger level

```
ca_stream_set_trigger(&data_stream, 8);
```

The **trigger level** sets the minimum byte threshold to wake up the waiting receiver task. Default: 1 byte.

When to use it: - If the receiver needs to process multiple blocks of bytes - To reduce the number of context switches - To optimize throughput vs latency

Print control with `print_enabled`

```
static volatile u8 print_enabled;  // 0 = disabled, 1 = enabled

// Nei task:
if (print_enabled) {
    ca_stdout_printf_n("...");
}
```

Prints are **disabled by default** (`print_enabled = 0` in `ca_os_main()`). They can be enabled with the `start` command from the CLI.

> **Note**: It is not possible to send prints with `ca_stdout_*` before `ca_os_run(true)`, because the output system (stdout thread) is not yet active.

CLI Hook

The CLI node handles 5 commands: - `start`: enable prints (`print_enabled = 1`) - `stop`: disable prints (`print_enabled = 0`) - `show`: statistics + print status - `clear`: flush stream buffer - `help`: adds the line `start | stop | show | clear` to the system help and returns `false`, so the chain goes on to the other nodes

The handler returns `true` if it handled the command, `false` to pass it to the next node. The node is registered with `ca_cli_register()`.

37.1.5 Expected output

At startup

The scheduler is started with `ca_os_run(true)`. Task prints are disabled by default. Type `help` to view the available commands.

Command `help`

```
c.a.O.S > help
internal commands:
top       : cpu + mem + stack usage
sched     : cpu split since last call: switch, tasks, idle, free
banner    : show os banner
taskall   : show all tasks
task <n>  : show task N
free      : Show global memory usage
canary    : show stack canary status
mpu       : show mpu regions
sensor    : show core temperature
stack <n> : show stack x task N usage
memlist   : show all memory allocated blocks
hexdump <addr hex> [lines] : classic hexdump

c.a.O.S > user commands:
start | stop | show | clear
```

`show` command (with prints disabled)

```
c.a.O.S > show

--- Stream Buffer Statistics:
   Bytes sent    : 1840
   Bytes received: 1840
```

```
   Writer cycles : 230
   Reader cycles : 230
   Available     : 0
   Free space    : 64
   Print status  : DISABLED
```

Enabling prints

```
c.a.O.S > start

Print ENABLED - task output active
READER: recv 8 bytes: 'DATA_045'
WRITER: sent 8 bytes (seq=46)
READER: recv 8 bytes: 'DATA_046'
WRITER: sent 8 bytes (seq=47)
...
```

Disabling prints

```
c.a.O.S > stop

Print DISABLED - task output silenced
```

Tasks continue to execute (they write/read from the stream), but they do not print.

37.1.6 Stream vs Message Buffer

Feature	Stream Buffer	Message Buffer
Data unit	Byte	Message
Boundaries	Not preserved	Preserved
Use case	Continuous stream (UART, TCP)	Distinct packets
Overhead	Minimal (head/tail only)	Length per message

Example:

```
// Stream: "ABC" + "DEF"      "ABCDEF" (merged)
ca_stream_send(sb, "ABC", 3, 100);
ca_stream_send(sb, "DEF", 3, 100);
ca_stream_receive(sb, buf, 6, 100);  // buf = "ABCDEF"

// Message: "ABC" + "DEF"      two separate messages
ca_message_send(mb, "ABC", 3, 100);
ca_message_send(mb, "DEF", 3, 100);
ca_message_receive(mb, buf, &len, 100);  // buf = "ABC", len = 3
ca_message_receive(mb, buf, &len, 100);  // buf = "DEF", len = 3
```

37.1.7 Required configuration in `os_config.h`

```
#define ENABLE_CLI      1    // CLI needed for user commands
#define ENABLE_DBG_UART 1    // UART for output
#define ENABLE_STDOUT      1    // Log system for ca_stdout_*
```

37.1.8 Summary

1. **Stream Buffer** is a FIFO circular buffer for byte streams, ideal for continuous communication between tasks.
2. **Main APIs**:
   - `ca_stream_send()` / `ca_stream_receive()`: blocking with timeout
   - `ca_stream_bytes_available()` / `ca_stream_space_available()`: status query
   - `ca_stream_set_trigger()`: receiver wake-up threshold
   - `ca_stream_reset()`: flush buffer
3. **CLI hook** allows adding user commands (`start`, `stop`, `show`, `clear`, `help`).
4. **Print control** with variable `print_enabled`: prints are disabled by default to avoid saturating the UART.
5. **Static vs Heap**: `CA_STREAM_STATIC_DEFINE()` allocates the buffer in .bss, zero heap overhead.
6. **Trigger level**: optimizes the trade-off between latency (low trigger) and throughput (high trigger).

37.1.9 Verification of waits with timeout (`twait`)

Since 09/27, a task waiting for the stream buffer **with timeout** (`ca_stream_receive()`) registers itself in the same wait list as those waiting without a timeout, parked in `SLEEP` until expiration, and the signaler wakes it up immediately (S2_02 §8.8). Previously, it performed polling every 10 ms outside the list: it would wake up up to 10 ms after the signal.

The command `twait [sec]` (default 3 s) measures this with a peer task created at the first call. The wait runs in the command task:

1. **latency** — 300 cycles: the peer sends a byte after 1..8 ms randomly; the waiter, with a 20 ms timeout, must obtain it (`lost=0`), not before the signal (`spur=0`), and be running within 2 ms (`lat_max` limit, in ticks);
2. **timeout** — no one sends; for `sec` seconds, T = 1..20 ms: the return never arrives **before** T (`early=0`), at most 2 ms after (`late=0`), and always with `false` (`got=0`).

One line, closed by `PASS` or `FAIL`. On H753 (tick 200 µs):

```
TW stream : rounds=300 lat_max=0/10 lost=0 spur=0 | tmo n=291 early=0 late=0 got
   =0 over_max=0/10  PASS
```

On the previous kernel: FAIL, `lat_max=45/10` and `late=140` out of 200: the old code compared ticks with milliseconds.

In the test bench, it is case `timed_wait` of `test_ex14.cpp` (helper `test/hw_test/inc/caosTimedWait.h`). Manually:

```
caos_shell -e 14 -q -t 40 -u "TW .*(PASS|FAIL)" -g "^TW" "twait 3"
```

Chapter 38

Messages

38.1 Example 15 — Message Buffer with heap allocation

38.1.1 Introduction

The **Message Buffer** is a message buffer with framing that preserves message boundaries. Unlike the Stream Buffer (which treats data as a continuous stream of bytes), every send/receive operation is atomic:

- **Send**: writes `[lunghezza][payload]` as an indivisible unit
- **Receive**: reads a complete message or nothing at all

This ensures message integrity during communication between tasks.

In this example: - A **writer** task sends structured messages - A **reader** task receives the messages - A **monitor** task displays the buffer status in real-time - The **CLI** allows controlling prints and viewing statistics

Demonstrated features

Function	Description
`ca_message_init()`	Initialization with heap allocation
`ca_message_destroy()`	Free memory (not used in this example)
`ca_message_send()` / `ca_message_receive()`	Atomic send/receive with timeout
`ca_message_peek_size()`	Reads size of next message (without removing)
`ca_message_count()`	Number of messages present in the buffer
`ca_message_space_available()`	Free space in the buffer
`ca_message_reset()`	Resets the buffer
`ca_message_is_empty()`	Query buffer status

CLI Commands

Command	Description
`start`	Enables task prints
`stop`	Disables task prints
`show`	Shows message buffer statistics
`clear`	Resets message buffer
`help`	List of available commands

38.1.2 Example architecture

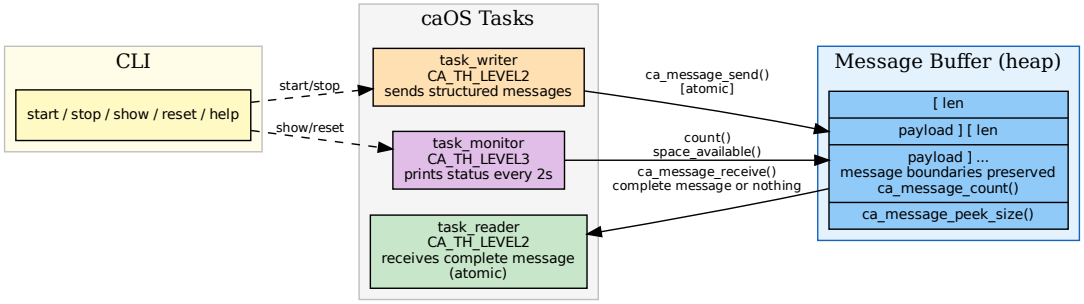


Figure 38.1: Message Buffer architecture

38.1.3 Source code

The file is located in `examples/example_15/ca_os_ex15.c`:

```c
#include "caos.h"

/* Maximum payload size per message */
#define MSG_PAYLOAD_SIZE    64U

/* Buffer size: payload + header overhead (2 bytes per message) */
/* With at most 10 messages: 10 * (2 + 64) = 660 bytes */
#define MSG_BUFFER_SIZE     660U

/* Maximum number of messages in the buffer */
#define MSG_MAX_COUNT       10U

/* Message structure */
typedef struct {
    u32 id;             /* Sender ID */
    u32 sequence;       /* Sequence number */
    char payload[MSG_PAYLOAD_SIZE];  /* Data (string) */
} Message_t;

/* Message buffer (allocazione heap) */
static ca_message_buffer_t msg_buffer;

static u32 running;
static u32 messages_sent;
static u32 messages_received;
static u32 writer_cycles;
static u32 reader_cycles;
static volatile u8 print_enabled;  // 0 = prints disabled, 1 = enabled

/* ============================================================================
 * TASK: Writer - sends structured messages
 * ============================================================================
    */

static u32 task_writer(void *arg) {
    const u32 p1 = (u32)(uintptr_t)arg;
    const u32 idx = ca_os_thid_from_handle(ca_os_self_handle());

```

```
38      if (print_enabled) {
39          ca_stdout_printf_n("WRITER started: idx=%u p1=%u", idx, p1);
40      }
41
42      Message_t msg;
43      u32 sequence = 0;
44
45      while (running) {
46          // Prepare a structured message
47          msg.id = 1;  // ID writer
48          msg.sequence = sequence;
49          ca_snprintf(msg.payload, MSG_PAYLOAD_SIZE, "Hello from Writer #%u",
    sequence);
50
51          // Send a message (timeout 500ms)
52          bool ok = ca_message_send(&msg_buffer, &msg, sizeof(Message_t), 500);
53
54          if (ok) {
55              messages_sent++;
56              writer_cycles++;
57              sequence++;
58
59              if (print_enabled) {
60                  ca_stdout_printf_n("WRITER: sent msg seq=%u (size=%u)",
61                                     sequence - 1, sizeof(Message_t));
62              }
63          } else {
64              if (print_enabled) {
65                  ca_stdout_printf_n("WRITER: timeout (buffer full?)");
66              }
67          }
68
69          // Wait before the next send
70          ca_os_sleep(300);
71      }
72
73      return 0;
74  }
75
76  /* ============================================================================
77   * TASK: Reader - receives structured messages
78   * ============================================================================
        */
79
80  static u32 task_reader(void *arg) {
81      const u32 p1 = (u32)(uintptr_t)arg;
82      const u32 idx = ca_os_thid_from_handle(ca_os_self_handle());
83
84      if (print_enabled) {
85          ca_stdout_printf_n("READER started: idx=%u p1=%u", idx, p1);
86      }
87
88      Message_t msg;
89      u16 received_len;
90
91      while (running) {
92          // Peek the size of the next message
93          u16 peek_size = ca_message_peek_size(&msg_buffer);
94
95          if (peek_size > 0) {
96              if (print_enabled) {
97                  ca_stdout_printf_n("READER: peek size=%u", peek_size);
98              }
```

```
99          }
100
101         // Receive a message (timeout 200ms)
102         received_len = ca_message_receive(&msg_buffer, &msg, sizeof(Message_t),
    200);
103
104         if (received_len > 0) {
105             messages_received++;
106             reader_cycles++;
107
108             if (print_enabled) {
109                 ca_stdout_printf_n("READER: recv msg from ID=%u seq=%u payload
    ='%s'",
110                                    msg.id, msg.sequence, msg.payload);
111             }
112         }
113     }
114
115     return 0;
116 }
117
118 /* ============================================================================
119  * TASK: Monitor - mostra statistiche message buffer
120  * ============================================================================
     */
121
122 static u32 task_monitor(void *arg) {
123     const u32 p1 = (u32)(uintptr_t)arg;
124     const u32 idx = ca_os_thid_from_handle(ca_os_self_handle());
125
126     ca_os_sleep(1000);  // Wait for it to settle
127
128     if (print_enabled) {
129         ca_stdout_printf_n("MONITOR started: idx=%u p1=%u", idx, p1);
130     }
131
132     while (running) {
133         u16 msg_count = ca_message_count(&msg_buffer);
134         u32 free_space = ca_message_space_available(&msg_buffer);
135         bool is_empty = ca_message_is_empty(&msg_buffer);
136         u16 next_size = ca_message_peek_size(&msg_buffer);
137
138         if (print_enabled) {
139             ca_stdout_printf_n("=== MESSAGE BUFFER STATUS ===");
140             ca_stdout_printf_n("Messages in queue : %u / %u", msg_count,
    MSG_MAX_COUNT);
141             ca_stdout_printf_n("Free space        : %u / %u bytes", free_space,
    MSG_BUFFER_SIZE);
142             ca_stdout_printf_n("Empty             : %s", is_empty ? "YES" : "NO
    ");
143             ca_stdout_printf_n("Next message size : %u bytes", next_size);
144             ca_stdout_printf_n("Total sent        : %u messages (%u cycles)",
    messages_sent, writer_cycles);
145             ca_stdout_printf_n("Total recv        : %u messages (%u cycles)",
    messages_received, reader_cycles);
146             ca_stdout_printf_n("==============================");
147         }
148
149         ca_os_sleep(3000);  // Print every 3 seconds
150     }
151
152     return 0;
153 }
```

```
154
155 /* ============================================================================
156  * CLI HOOK - user commands for message buffer
157  * ============================================================================
        */
158
159 static bool user_cli_handler(c_str *pcmd)
160 {
161     const char *p = ca_skip_sp((*pcmd)->data);
162     if (ca_strncmp(p, "help", 4) == 0) {
163         CLI_PUT_N("start | stop | show | clear");
164         return false;
165     }
166     if (ca_strncmp(p, "start", 5) == 0) {
167         print_enabled = 1;
168         CLI_PUT_N("Print ENABLED");
169         return true;
170     }
171     if (ca_strncmp(p, "stop", 4) == 0) {
172         print_enabled = 0;
173         CLI_PUT_N("Print DISABLED");
174         return true;
175     }
176     if (ca_strncmp(p, "show", 4) == 0) {
177         CLI_PUT_N("--- Message Buffer Statistics:");
178         CLI_PRINTF_N("   Messages sent    : %u", messages_sent);
179         CLI_PRINTF_N("   Messages received: %u", messages_received);
180         CLI_PRINTF_N("   Writer cycles    : %u", writer_cycles);
181         CLI_PRINTF_N("   Reader cycles    : %u", reader_cycles);
182         CLI_PRINTF_N("   Messages in queue: %u", ca_message_count(&msg_buffer));
183         CLI_PRINTF_N("   Free space       : %u bytes",
    ca_message_space_available(&msg_buffer));
184         CLI_PRINTF_N("   Print status     : %s", print_enabled ? "ENABLED" : "
    DISABLED");
185         return true;
186     }
187     if (ca_strncmp(p, "clear", 5) == 0) {
188         ca_message_reset(&msg_buffer);
189         CLI_PUT_N("Message buffer cleared!");
190         return true;
191     }
192     return false;
193 }
194 static ca_cli_node_t s_user_cli_node = { user_cli_handler, NULL };
195
196 /* ============================================================================
197  * ca_os_configure - module initialization
198  * ============================================================================
        */
199
200 void ca_os_configure(void) {
201     /* Initialize temperature sensor */
202
203     /* Log system */
204     EVAL_STDOUT(
205         ca_stdout_init();
206         ca_stdout_set_sender(STDOUT);
207         ca_stdout_create_thread();
208     )
209     /* CLI */
210     EVAL_CLI(
211         ca_cli_init(4096);
212         ca_cli_set_sender(STDOUT);
```

```
        ca_cli_create_thread();
    )

    /* USB CDC */
    EVAL_USB_CDC_STD(ca_usb_dev_mng_cdc_cli();)
}

/* ============================================================================
 * ca_os_main - entry point
 * ============================================================================
    */

void ca_os_main(void) {
    running = 1;
    messages_sent = 0;
    messages_received = 0;
    writer_cycles = 0;
    reader_cycles = 0;
    print_enabled = 0;  // Prints DISABLED by default

    /* Inizializza message buffer con allocazione heap */
    if (!ca_message_init(&msg_buffer, MSG_BUFFER_SIZE)) {
        // Error handling: we cannot print before ca_os_run()
        while (1) {
            __NOP();
        }
    }

    /* Create the writer task (medium priority) */
    ca_os_job("writer", CA_TH_LEVEL2, task_writer, NULL,
              STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);

    /* Create the reader task (medium priority) */
    ca_os_job("reader", CA_TH_LEVEL2, task_reader, NULL,
              STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);

    /* Create the monitor task (low priority) */
    ca_os_job("monitor", CA_TH_LEVEL1, task_monitor, NULL,
              STACK_SMALL, CA_JOB_STD, CA_FAULT_ASSERT);

    /* Register CLI hook */
    EVAL_CLI( ca_cli_register(&s_user_cli_node); )

    /* Start the scheduler */
    ca_os_run(true);
}
```

38.1.4 Detailed analysis

Initialization with heap allocation

```
#define MSG_BUFFER_SIZE     660U  // 10 messages    (2-byte header + 64-byte
    payload)

ca_message_buffer_t msg_buffer;  // Dichiarazione

void ca_os_main(void) {
    // Initialization with heap allocation
    if (!ca_message_init(&msg_buffer, MSG_BUFFER_SIZE)) {
        // Error handling
        while (1) { __NOP(); }
    }
```

```
    // ...
}
```

Heap advantages: - Flexibility in size - Memory allocated only when needed
Disadvantages: - Possible fragmentation - Allocation overhead - Need to handle errors

Message structure

```
typedef struct {
    u32 id;              /* Sender ID */
    u32 sequence;        /* Sequence number */
    char payload[64];    /* Application data */
} Message_t;
```

Total size: `4 + 4 + 64 = 72 byte`
Message Buffer Overhead: 2 bytes per header (length)
Occupancy per message: `72 + 2 = 74 byte`

Buffer size calculation

```
#define MSG_PAYLOAD_SIZE    64U
#define MSG_MAX_COUNT       10U
#define MSG_BUFFER_SIZE     (MSG_MAX_COUNT * (CA_MSG_HEADER_SIZE +
    MSG_PAYLOAD_SIZE))
                          // = 10 * (2 + 64) = 660 byte
```

Message size peek

```
u16 peek_size = ca_message_peek_size(&msg_buffer);

if (peek_size > 0) {
    ca_stdout_printf_n("READER: peek size=%u", peek_size);
}
```

`ca_message_peek_size()` reads the size of the next message **without removing it** from the buffer. Useful for: - Allocating buffers of the correct size - Debugging and statistics - Size-based decision making

Print control with `print_enabled`

```
static volatile u8 print_enabled;  // 0 = disabled, 1 = enabled

// Nei task:
if (print_enabled) {
    ca_stdout_printf_n("...");
}
```

Prints are **disabled by default** (`print_enabled = 0` in `ca_os_main()`). They can be enabled with the `start` command from the CLI.

> **Note**: It is not possible to send prints with `ca_stdout_*` before `ca_os_run(true)`, because the output system (stdout thread) is not yet active.

CLI Hook

The CLI node handles 5 commands: - `start`: enable prints (`print_enabled = 1`) - `stop`: disable prints (`print_enabled = 0`) - `show`: statistics + print status - `clear`: flush message buffer - `help`: adds the line `start | stop | show | clear` to the system help and returns `false`, so the chain goes on to the other nodes

The handler returns `true` if it handled the command, `false` to pass it to the next node. The node is registered with `ca_cli_register()`.

38.1.5 Expected output

At startup

The scheduler is started with `ca_os_run(true)`. Task prints are disabled by default. Type `help` to view the available commands.

Command `help`

```
c.a.O.S > help
internal commands:
top       : cpu + mem + stack usage
sched     : cpu split since last call: switch, tasks, idle, free
banner    : show os banner
taskall   : show all tasks
task <n>  : show task N
free      : Show global memory usage
canary    : show stack canary status
mpu       : show mpu regions
sensor    : show core temperature
stack <n> : show stack x task N usage
memlist   : show all memory allocated blocks
hexdump <addr hex> [lines] : classic hexdump

c.a.O.S > user commands:
start | stop | show | clear
```

`clear` command

```
c.a.O.S > clear

Message buffer cleared!
```

`show` command (with prints disabled)

```
c.a.O.S > show

--- Message Buffer Statistics:
   Messages sent    : 192
   Messages received: 192
   Writer cycles    : 192
   Reader cycles    : 192
   Messages in queue: 0
   Free space       : 660 bytes
   Print status     : DISABLED
```

Enabling prints

```
c.a.O.S > start

Print ENABLED - task output active
WRITER: sent msg seq=214 (size=72)
READER: recv msg from ID=1 seq=214 payload='Hello from Writer #214'
WRITER: sent msg seq=215 (size=72)
READER: recv msg from ID=1 seq=215 payload='Hello from Writer #215'
WRITER: sent msg seq=216 (size=72)
READER: recv msg from ID=1 seq=216 payload='Hello from Writer #216'
WRITER: sent msg seq=217 (size=72)
READER: recv msg from ID=1 seq=217 payload='Hello from Writer #217'
WRITER: sent msg seq=218 (size=72)
READER: recv msg from ID=1 seq=218 payload='Hello from Writer #218'
WRITER: sent msg seq=219 (size=72)
READER: recv msg from ID=1 seq=219 payload='Hello from Writer #219'
WRITER: sent msg seq=220 (size=72)
READER: recv msg from ID=1 seq=220 payload='Hello from Writer #220'
WRITER: sent msg seq=221 (size=72)
READER: recv msg from ID=1 seq=221 payload='Hello from Writer #221'
WRITER: sent msg seq=222 (size=72)
READER: recv msg from ID=1 seq=222 payload='Hello from Writer #222'
WRITER: sent msg seq=223 (size=72)
READER: recv msg from ID=1 seq=223 payload='Hello from Writer #223'
=== MESSAGE BUFFER STATUS ===
Messages in queue : 0 / 10
Free space        : 660 / 660 bytes
Empty             : YES
Next message size : 0 bytes
Total sent        : 224 messages (224 cycles)
Total recv        : 224 messages (224 cycles)
============================
WRITER: sent msg seq=224 (size=72)
READER: recv msg from ID=1 seq=224 payload='Hello from Writer #224'
```

Disabling prints

```
c.a.O.S > stop

Print DISABLED - task output silenced
```

Tasks continue to execute (send/receive messages), but do not print.

38.1.6 Stream vs Message Buffer

Feature	Stream Buffer	Message Buffer
Data unit	Byte	Message
Boundaries	Not preserved	Preserved
Use case	Continuous stream (UART, TCP)	Distinct packets
Overhead	Minimal (head/tail only)	2-byte header per message
API count	Available bytes	Number of messages
API peek	No	`ca_message_peek_size()`
Atomicity	No	Yes (atomic send/receive)

Practical example:

```
// Stream: "ABC" + "DEF"      "ABCDEF" (fusi in un unico flusso)
ca_stream_send(sb, "ABC", 3, 100);
ca_stream_send(sb, "DEF", 3, 100);
ca_stream_receive(sb, buf, 6, 100);  // buf = "ABCDEF"

// Message: "ABC" + "DEF"      two separate messages
ca_message_send(mb, "ABC", 3, 100);
ca_message_send(mb, "DEF", 3, 100);

// First receive
len = ca_message_receive(mb, buf1, max, 100);  // buf1 = "ABC", len = 3

// Second receive
len = ca_message_receive(mb, buf2, max, 100);  // buf2 = "DEF", len = 3
```

38.1.7 Required configuration in `os_config.h`

```
#define ENABLE_CLI       1    // CLI needed for user commands
#define ENABLE_DBG_UART 1    // UART for output
#define ENABLE_STDOUT      1    // Log system for ca_stdout_*
```

38.1.8 Heap memory management

Allocation

```
bool ca_message_init(ca_message_buffer_t *mb, u32 size);
```

- Allocate `size` bytes from the heap
- Initialize internal structure
- Return `true` on success, `false` if out of memory

Destruction (not used in this example)

```
void ca_message_destroy(ca_message_buffer_t *mb);
```

- Free allocated memory
- To be called if the buffer is no longer needed

Best practice

```
// In ca_os_main():
if (!ca_message_init(&mb, SIZE)) {
    // Error handling: not enough memory
    while (1) { __NOP(); }
}

// In a cleanup task (optional):
ca_message_destroy(&mb);
```

38.1.9 Summary

1. **Message Buffer** preserves message boundaries, ideal for communication of distinct packets between tasks.
2. **Heap allocation**: `ca_message_init()` dynamically allocates memory, offering flexibility but requiring error management.

3. **Main API**:
   - `ca_message_send()` / `ca_message_receive()`: atomic operations with timeout
   - `ca_message_peek_size()`: reads size of next message without removing it
   - `ca_message_count()`: number of messages in queue
   - `ca_message_space_available()`: free space
   - `ca_message_reset()`: clears buffer
4. **Overhead**: 2 bytes per message (length header).
5. **CLI hook** allows adding user commands (`start`, `stop`, `show`, `clear`, `help`).
6. **Print control** with variable `print_enabled`: prints are disabled by default to avoid saturating the UART.
7. **Message structure**: the example uses a `{id, sequence, payload}` struct, but the application can define any format.
8. **Atomicity**: every `send` writes a complete message, every `receive` reads a complete message. No partial or merged messages.

38.1.10 Verification of waits with timeout (`twait`)

Since 09/27, a task waiting for the message buffer **with timeout** (`ca_message_receive()`) registers in the same wait list as those waiting without timeout, parked in `SLEEP` until expiration, and the signaler wakes it up immediately (S2_02 §8.8). Previously, it performed polling every 10 ms outside the list: it would wake up up to 10 ms after the signal.

The `twait [sec]` command (default 3 s) measures it with a peer task created at the first call. The wait runs in the command task:

1. **latency** — 300 cycles: the peer sends a message after 1..8 ms randomly; the waiter, with a timeout of 20 ms, must obtain it (`lost=0`), not before the signal (`spur=0`), and be running within 2 ms (`lat_max` limit, in ticks);
2. **timeout** — no one sends; for `sec` seconds, T = 1..20 ms: the return never arrives **before** T (`early=0`), at most 2 ms later (`late=0`), and always with `false` (`got=0`).

One line, closed by `PASS` or `FAIL`. On H753 (tick 200 µs):

```
TW message : rounds=300 lat_max=0/10 lost=0 spur=0 | tmo n=291 early=0 late=0
    got=0 over_max=0/10  PASS
```

On the previous kernel: FAIL, `lat_max=50/10` and `late=140` out of 200: the old code compared ticks with milliseconds.

In the test bench, it is case `timed_wait` of `test_ex15.cpp` (helper `test/hw_test/inc/caosTimedWait.h`). Manually:

```
caos_shell -e 15 -q -t 40 -u "TW .*(PASS|FAIL)" -g "^TW" "twait 3"
```

Chapter 39

Thread Pool with RNG and SHA-256

39.1 Example 16 — Thread Pool with RNG and SHA-256

39.1.1 Introduction

The **Thread Pool** is a set of pre-allocated threads that remain active for the entire life of the system, ready to execute dynamically assigned jobs. Compared to creating one-shot threads for every operation, the pool eliminates allocation overhead and ensures a fixed number of concurrent tasks.

In this example, a **producer** task periodically generates random byte buffers via the hardware RNG and sends the SHA-256 calculation to the pool. The **worker threads** perform the cryptographic calculation asynchronously, without blocking the producer.

The pattern is **fire-and-forget**: the producer does not wait for the result; the calculation happens in the background.

> **Availability**: this example requires `EN_RNG=1`, `EN_HASH=1`, `EN_POOL=1`. On platforms that do not support these features (F1xx, F4xx, F7xx with default configuration) the code compiles but only displays an informative message.

Demonstrated Features

Function	Description
`ca_pool_init()`	Creates N pre-allocated worker threads
`ca_pool_submit()`	Sends a job to the pool (fire-and-forget)
`ca_pool_stats()`	Real-time statistics: busy/free/queued
`ca_rng_fill()`	Fills a buffer with HW random data
`ca_hash_compute()`	Calculates SHA-256 on buffer (one-shot)

CLI Commands

Command	Source	Description
`start`	application	Enables SHA-256 digest printing
`stop`	application	Disables printing
`show`	application	Shows pool statistics and counters
`rng get`	driver `rng.c`	Generates and prints a single 32-bit random value

Command	Source	Description
`help`	chain	Lists all commands (driver + application)

The CLI uses the **chain of responsibility** pattern: `ca_rng_init()` registers the driver node, the application code registers its own node with `ca_cli_register()`. The `rng get` commands are handled by the driver without code in the application.

39.1.2 Example Architecture

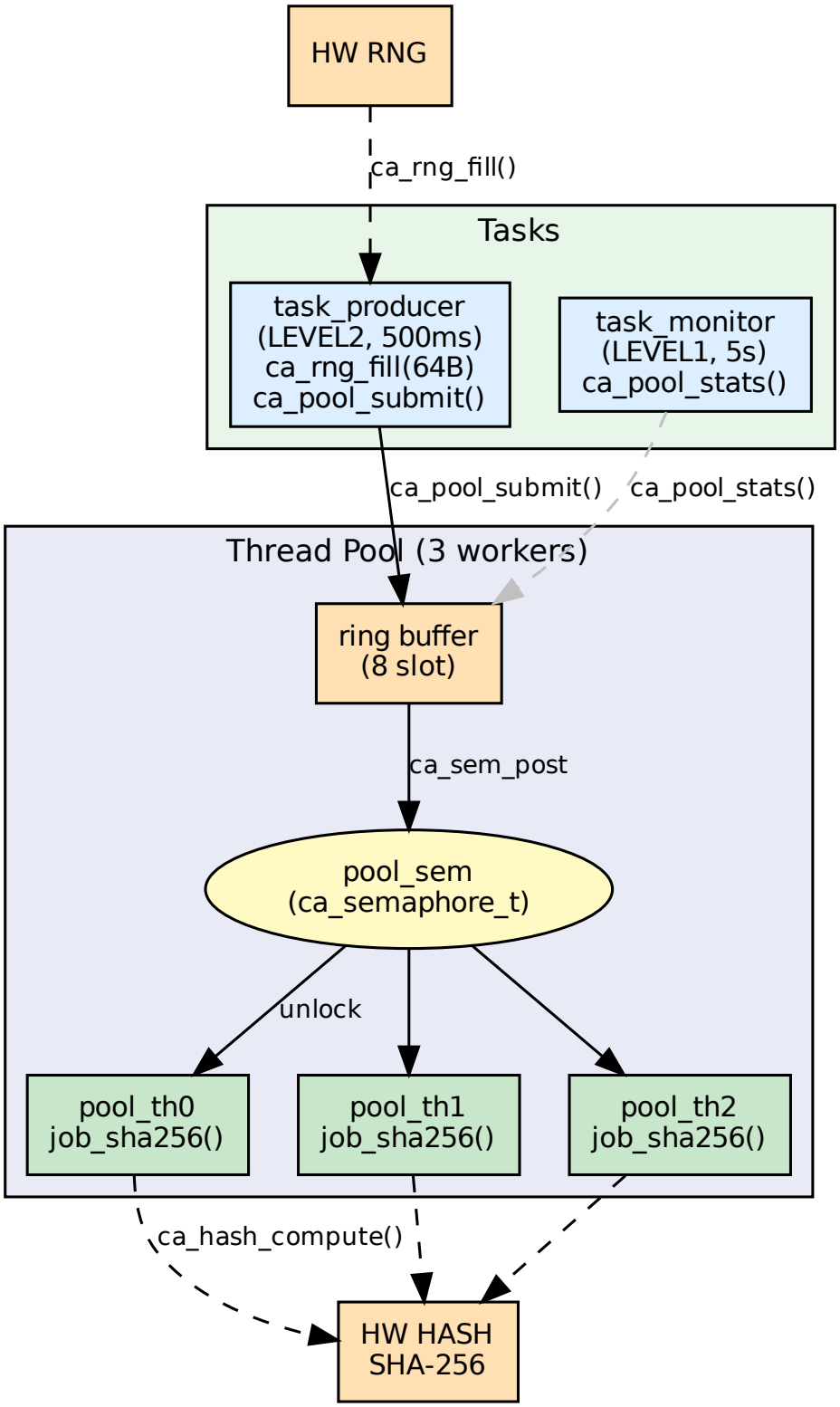


Figure 39.1: Example 16 Architecture — Thread Pool + RNG + SHA-256

39.1.3 Source Code

The file is located in `examples/example_16/ca_os_ex16.c`:

```c
#include "caos.h"

#if EN_RNG == 1 && EN_HASH == 1 && EN_POOL == 1

#define BUF_SIZE       64    /* byte per buffer RNG */
#define POOL_THREADS   3
#define PRODUCE_MS     500   /* production interval */

static volatile u32 jobs_submitted; /* accepted by the pool */
static volatile u32 jobs_done;      /* submitted, hash succeeded */
static volatile u32 jobs_failed;    /* submitted, hash failed */
static volatile u32 jobs_rejected;  /* never entered: RNG failed or queue full */
static volatile u8  print_digest;

/* Buffer pool statico in round-robin (evita allocazioni runtime) */
#define N_BUFS  (POOL_THREADS + 2)

typedef struct {
    u8   data[BUF_SIZE];
    u32  seq;
} ex16_buf_t;

static ex16_buf_t buf_pool[N_BUFS];
static volatile u8 buf_idx;

/* Job run by the workers: computes SHA-256 and prints the digest */
static u32 job_sha256(void *arg) {
    const u32 p1 = (u32)(uintptr_t)arg;
    const u32 worker_idx = ca_os_thid_from_handle(ca_os_self_handle());
    ex16_buf_t *b = (ex16_buf_t *)(uintptr_t)p1;
    u8  digest[HASH_SHA256_DIGEST_SIZE];
    u32 dlen = 0;

    bool ok = ca_hash_compute(CA_HASH_ALGO_SHA256, b->data, BUF_SIZE, digest, &dlen);

    if (ok) {
        jobs_done++;
        if (print_digest) {
            ca_stdout_printf_n("SHA256 seq=%u w=%u : %02x%02x%02x%02x%02x%02x%02x%02x...",
                b->seq, worker_idx,
                digest[0], digest[1], digest[2], digest[3],
                digest[4], digest[5], digest[6], digest[7]);
        }
    } else {
        jobs_failed++;
    }
    return 0;
}

/* Producer task: generates RNG buffers and submits jobs to the pool */
static u32 task_producer(void *arg) {
    (void)arg;
    u32 seq = 0;

    while (1) {
        u8 bi = buf_idx;
        buf_idx = (u8)((bi + 1u) % N_BUFS);
```

```
58
59          ex16_buf_t *b = &buf_pool[bi];
60          b->seq = seq++;
61
62          if (!ca_rng_fill(b->data, BUF_SIZE)) {
63              jobs_rejected++;    /* never reached the pool */
64              ca_os_sleep(PRODUCE_MS);
65              continue;
66          }
67
68          i32 res = ca_pool_submit(job_sha256, b);
69          if (res == 0) {
70              jobs_submitted++;
71          } else {
72              jobs_rejected++;    /* queue full: never reached the pool */
73          }
74
75          ca_os_sleep(PRODUCE_MS);
76      }
77      return 0;
78  }
79
80  /* Monitor task: statistics every 5s */
81  static u32 task_monitor(void *arg) {
82      (void)arg;
83      ca_os_sleep(2000);
84
85      while (1) {
86          u8 busy, free_th, queued;
87          ca_pool_stats(&busy, &free_th, &queued);
88          ca_stdout_printf_n("=== POOL STATS ===");
89          ca_stdout_printf_n("  submitted : %u", jobs_submitted);
90          ca_stdout_printf_n("  done      : %u", jobs_done);
91          ca_stdout_printf_n("  failed    : %u", jobs_failed);
92          ca_stdout_printf_n("  rejected  : %u", jobs_rejected);
93          ca_stdout_printf_n("  busy      : %u / %u", busy, (u8)POOL_THREADS);
94          ca_stdout_printf_n("  free      : %u", free_th);
95          ca_stdout_printf_n("  queued    : %u / %u", queued, (u8)POOL_QUEUE_SIZE)
    ;
96          ca_stdout_printf_n("==================");
97          ca_os_sleep(5000);
98      }
99      return 0;
100 }
101
102 void ca_os_main(void) {
103     jobs_submitted = 0;
104     jobs_done      = 0;
105     jobs_failed    = 0;
106     jobs_rejected  = 0;
107     print_digest   = 0;
108     buf_idx        = 0;
109
110     ca_pool_cfg_t pool_cfg = {
111         .n_threads  = POOL_THREADS,
112         .stack_size = STACK_MEDIUM,
113         .priority   = CA_TH_LEVEL1,
114     };
115     ca_pool_init(&pool_cfg);
116
117     ca_os_job("producer", CA_TH_LEVEL2, task_producer, NULL,
118               STACK_SMALL, CA_JOB_STD, CA_FAULT_ASSERT);
119
```

```
120     ca_os_job("monitor", CA_TH_LEVEL1, task_monitor, NULL,
121               STACK_SMALL, CA_JOB_STD, CA_FAULT_ASSERT);
122
123     EVAL_CLI( ca_cli_register(&s_user_cli_node); )
124
125     ca_os_run(true);
126 }
```

39.1.4 Detailed analysis

Pool initialization

```
1 ca_pool_cfg_t pool_cfg = {
2     .n_threads  = 3,
3     .stack_size = STACK_MEDIUM,
4     .priority   = CA_TH_LEVEL1,
5 };
6 ca_pool_init(&pool_cfg);
```

`ca_pool_init()` creates 3 permanent threads with names `pool_th0`, `pool_th1`, `pool_th2`. Each thread executes a loop:

```
1 while (1) {
2     ca_sem_wait(&pool_sem);        // sleeps until work arrives
3     ca_pool_job_t job = pop();     // takes the job from the queue
4     pool_busy++;
5     job.func(idx, job.p1);         // esegue
6     pool_busy--;
7 }
```

The semaphore `pool_sem` is the synchronization mechanism: it counts the available jobs. When it is zero, threads sleep without consuming CPU.

Sending a job

```
1 i32 res = ca_pool_submit(job_sha256, b);
```

`ca_pool_submit()`: 1. Inserts `{func, p1}` into the queue's ring buffer (mutex-protected) 2. Calls `ca_sem_post()` to wake up a worker 3. If the queue is full, it creates a one-shot thread as a fallback

The buffer pointer is passed as `u32` — valid on Cortex-M33/M7 where pointers are 32-bit.

Static round-robin buffer pool

```
1 #define N_BUFS  (POOL_THREADS + 2)   /* = 5 buffer */
2
3 static ex16_buf_t buf_pool[N_BUFS];
4 static volatile u8 buf_idx;
5
6 /* In task_producer: */
7 u8 bi = buf_idx;
8 buf_idx = (u8)((bi + 1u) % N_BUFS);
9 ex16_buf_t *b = &buf_pool[bi];
```

Buffers are static in `.bss`, reused in round-robin. With `N_BUFS = POOL_THREADS + 2 = 5` buffers and a producer sending a job every 500ms while workers complete in a few µs, there is no risk of overwriting while a worker is reading the buffer.

> **Rule**: `N_BUFS` must be at least `POOL_THREADS + 1` to ensure that the producer does not overwrite a buffer that is still being processed.

Hardware SHA-256 calculation

```
bool ok = ca_hash_compute(CA_HASH_ALGO_SHA256, b->data, BUF_SIZE, digest, &dlen)
    ;
```

`ca_hash_compute()` uses the **hardware HASH peripheral** present on H5/H7: - On H563: HASH with context swap (CSR), protected by a mutex for concurrent use - On H753: identical, with automatic register save/restore

The digest is 32 bytes (SHA-256). The example prints only the first 8 bytes as a visual identifier.

Pool statistics

```
void ca_pool_stats(u8 *busy, u8 *free_th, u8 *queued);
```

Field	Meaning
`busy`	Thread currently executing a job
`free_th`	Thread waiting (`pool_n - busy`)
`queued`	Job in queue yet to be assigned

`busy` is a `volatile u8` counter, incremented/decremented before/after `job.func()`. It is accurate for visualization; a delta of ±1 during context switching is acceptable.

CLI — Chain of Responsibility

The CLI follows the **chain of responsibility** pattern: each node receives the command and can handle it (returns `true`) or pass it to the next node (returns `false`). `help` always returns `false` to show the output of all nodes in the chain.

```
static bool user_cli_handler(c_str *pcmd)
{
    c_str cmd = *pcmd;
    const char *p = ca_skip_sp(cmd->data);

    if (ca_strncmp(p, "help", 4) == 0) {
        CLI_PUT_N("ex16: start | stop | show  (rng get - driver)");
        return false;           /* propagate: show the other nodes' help too */
    }
    if (ca_strncmp(p, "start", 5) == 0) {
        print_digest = 1;
        CLI_PUT_N("verbose ON");
        return true;
    }
    if (ca_strncmp(p, "stop", 4) == 0) {
        print_digest = 0;
        CLI_PUT_N("verbose OFF");
        return true;
    }
    if (ca_strncmp(p, "show", 4) == 0) {
        u8 busy, free_th, queued;
        ca_pool_stats(&busy, &free_th, &queued);
        CLI_PRINTF_N("submitted=%u  done=%u  failed=%u  rejected=%u  "
                     "busy=%u/%u  queued=%u/%u",
                     jobs_submitted, jobs_done, jobs_failed, jobs_rejected,
                     (u32)busy, (u32)POOL_THREADS,
                     (u32)queued, (u32)POOL_QUEUE_SIZE);
        return true;
    }
```

```
    return false;
}

static ca_cli_node_t s_user_cli_node = { user_cli_handler, NULL };
```

Registration in `ca_os_main`:

```
EVAL_CLI( ca_cli_register(&s_user_cli_node); )
```

The RNG driver node (`rng get`) is automatically registered by `ca_rng_init()` in `ca_os_configure`. The application must not handle `rng`.

`print_digest` is `volatile u8` written by the CLI and read by the workers — atomic read/write on 32-bit ARM, no need for mutexes.

39.1.5 Expected output

At startup — automatic statistics every 5s

The monitor wakes up after 2 seconds and then every 5 seconds. Digest prints are disabled by default.

```
=== POOL STATS ===
  submitted : 106
  done      : 106
  failed    : 0
  rejected  : 0
  busy      : 0 / 3
  free      : 3
  queued    : 0 / 8
==================
=== POOL STATS ===
  submitted : 114
  done      : 114
  failed    : 0
  rejected  : 0
  busy      : 0 / 3
  free      : 3
  queued    : 0 / 8
==================
```

`submitted == done` and `failed == 0` confirm that all jobs have been executed correctly. `busy = 0` indicates that the workers are waiting on the semaphore (no jobs in flight at the time of the snapshot).

The four counters

The distinction between `failed` and `rejected` is the didactic point of the example, not a printing detail:

counter	meaning
`submitted`	jobs **accepted** by the pool
`done`	of those, hash successful
`failed`	of those, hash failed
`rejected`	**never entered** the pool: RNG failed, or queue full

`rejected` is deliberately **outside** of `submitted`: the pool cannot be held responsible for a job that was never delivered to it. Hence the invariant that holds at every real instant:

```
submitted == done + failed + busy + queued
```

Keeping the two categories in a single counter makes the accounting unreadable: `submitted - done - failed` would go negative as soon as the queue fills up.

`show` Command

```
c.a.O.S > show
submitted=106  done=106  failed=0  rejected=0  busy=0/3  queued=0/8
```

`rng get` command (driver)

```
c.a.O.S > rng get
rng: 0xA3F12C7B  (2751987835)
```

Command `start` — digest enablement

```
c.a.O.S > start

Verbose ON     digest stampati
SHA256 seq=131 w=6 : 4c275ad7e07eba44...
SHA256 seq=132 w=7 : 32b0e47b92d63ddd...
SHA256 seq=133 w=5 : 5f552c9aafae4838...
=== POOL STATS ===
  submitted : 134
  done      : 134
  ...
SHA256 seq=134 w=6 : 095a9baa872a1d8a...
SHA256 seq=135 w=7 : 57eec8a2dfa3b7ca...
```

The worker IDs (`w=5`, `w=6`, `w=7`) are the scheduler indices of the 3 pool threads, assigned at the time of `ca_pool_init()`. The sequence number is strictly increasing and assigned by the producer; the completion order may vary slightly between workers.

Command `stop` — with CLI/output interleaving

```
stSHA256 seq=136 w=5 : 61c5f7cd6ad674b0...
SHA256 seq=137 w=6 : 18f7252d0c1329e1...
opSHA256 seq=138 w=7 : a19d55be77cae44f...


Verbose OFF
```

The characters `st` and `op` appear in the middle of SHA256 prints. This is normal behavior: the CLI receives typed characters (`s`, `t`, `o`, `p`) asynchronously, while the log system sends worker strings over the same output channel. This is not an error.

39.1.6 Required configuration in `os_config.h`

```
#define EN_RNG          1    /* Hardware RNG     F4/F7/H5/H7 */
#define EN_HASH         1    /* Hardware SHA-256     H5/H7 only */
#define EN_POOL         1    /* Thread pool     MAX_TASK >= 50 */
#define POOL_QUEUE_SIZE 8    /* Job queue depth (default) */

#define ENABLE_CLI      1    /* CLI for user commands */
#define ENABLE_STDOUT      1    /* Log system for ca_stdout_* */
```

39.1.7 Created threads

With the configuration of this example, the system creates:

Thread	Name	Priority	Role
idle	`idle`	0	Scheduler idle task
log	`log`	—	Asynchronous output
CLI	`cli`	—	Command reception
USB CDC	`usb_cdc`	—	I/O channel
pool_th0	`pool_th0`	LEVEL1	SHA-256 worker
pool_th1	`pool_th1`	LEVEL1	SHA-256 worker
pool_th2	`pool_th2`	LEVEL1	SHA-256 worker
producer	`producer`	LEVEL2	Generates RNG buffers
monitor	`monitor`	LEVEL1	Prints statistics

39.1.8 Summary

1. **Thread Pool** pre-allocates N permanent threads. No runtime creation/destruction, minimal overhead.

2. **Synchronization mechanism**: counting semaphore `pool_sem`. Workers sleep on `ca_sem_wait()`, the producer wakes them up with `ca_sem_post()` after each `ca_pool_submit()`.

3. **Fire-and-forget**: the producer does not wait for the SHA-256 calculation result. If the result is needed, the application must use a separate channel (event, notify, global variable).

4. **Round-robin buffer**: `N_BUFS = POOL_THREADS + 2` static buffers ensure that the producer does not overwrite a buffer still being processed, without dynamic allocations.

5. **One-shot fallback**: if the queue is full (all workers busy and `POOL_QUEUE_SIZE` jobs waiting), `ca_pool_submit()` creates a temporary one-shot thread. In this example, this never happens (slow producer, fast workers).

6. **CLI/output interleaving**: normal with an asynchronous log system. Typed characters may appear in the middle of task prints on the shared channel.

7. **Hardware HASH**: the SHA-256 calculation uses the dedicated H5/H7 peripheral. Multiple workers can use it simultaneously thanks to the CSR register context swap, managed internally by `ca_hash_compute()` with a mutex.

Chapter 40

Event Group: Synchronization on Bit Patterns

40.1 Example 17 — Event Group: bit pattern synchronization

40.1.1 Introduction

The **Event Group** is a synchronization primitive based on a set of 32 boolean flags (bits). Multiple tasks can wait for one or more bits to be set, using two modes:

- **WAIT_ANY**: the task wakes up when *at least one* of the requested bits is set.
- **WAIT_ALL**: the task wakes up when *all* requested bits are set simultaneously.

Compared to semaphores, event groups allow expressing composite conditions without introducing auxiliary state variables. The `ca_event_set()` function is ISR-safe and can be called from interrupt handlers.

In this example, three simulated sensors (temperature, pressure, humidity) produce samples asynchronously. An **aggregator** task waits for all three to have published their data (WAIT_ALL) before processing the result. A **watchdog** task instead waits for any error bit (WAIT_ANY) with a 3-second timeout.

Demonstrated features

Function	Description
`ca_event_init()`	Initializes an event group
`ca_event_set()`	Sets one or more bits (OR) — ISR-safe
`ca_event_wait_all()`	Waits for ALL bits to be set
`ca_event_wait_any()`	Waits for AT LEAST ONE bit to be set, with timeout
`ca_event_get()`	Reads the current value of the bits without blocking
`CA_EVENT_BIT(n)`	Helper for the n-th bit mask

CLI Commands

Command	Description
`err_t`	Injects a temperature error (sets BIT_ERR_TEMP)
`err_p`	Injects a pressure error (sets BIT_ERR_PRES)
`show`	Shows aggregation counters, watchdog alerts, and current bits

Command	Description
`help`	Lists available commands

40.1.2 Example architecture

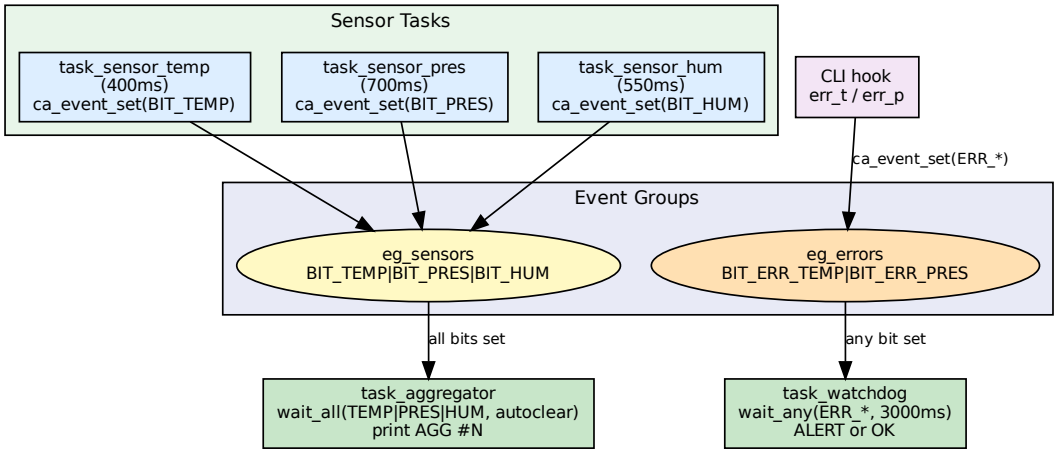


Figure 40.1: Example 17 Architecture — Event Groups

40.1.3 Source code

The file is located in `examples/example_17/ca_os_ex17.c`:

```c
#include "caos.h"

/* Sensor bit map */
#define BIT_TEMP    CA_EVENT_BIT(0)
#define BIT_PRES    CA_EVENT_BIT(1)
#define BIT_HUM     CA_EVENT_BIT(2)
#define BITS_ALL    (BIT_TEMP | BIT_PRES | BIT_HUM)

/* Error bit map */
#define BIT_ERR_TEMP  CA_EVENT_BIT(8)
#define BIT_ERR_PRES  CA_EVENT_BIT(9)
#define BITS_ERR      (BIT_ERR_TEMP | BIT_ERR_PRES)

static ca_event_group_t eg_sensors;
static ca_event_group_t eg_errors;

static volatile i32 g_temp;
static volatile u32 g_pres;
static volatile u32 g_hum;

/* Temperature sensor task: sets BIT_TEMP every 400ms */
static u32 task_sensor_temp(void *arg) {
    (void)arg;
    i32 base = 200;
    while (1) {
        ca_os_sleep(400);
        base += (i32)(ca_os_tick_ms() & 3) - 1;
```

```
        g_temp = base;
        ca_event_set(&eg_sensors, BIT_TEMP);
    }
    return 0;
}

/* Aggregator task: WAIT_ALL     wakes up when all 3 bits are ready */
static u32 task_aggregator(void *arg) {
    (void)arg;
    while (1) {
        u32 bits = ca_event_wait_all(&eg_sensors, BITS_ALL, true,
    CA_WAIT_FOREVER);
        if (bits != 0) {
            aggregations++;
            ca_stdout_printf_n("AGG #%u  T=%d.%d  P=%u  H=%u.%u",
                aggregations, g_temp / 10, g_temp % 10,
                g_pres, g_hum / 10, g_hum % 10);
        }
    }
    return 0;
}

/* Watchdog task: WAIT_ANY on error bits with a 3s timeout */
static u32 task_watchdog(void *arg) {
    (void)arg;
    while (1) {
        u32 bits = ca_event_wait_any(&eg_errors, BITS_ERR, true, 3000);
        if (bits == 0) {
            ca_stdout_printf_n("WDG: OK (no errors in 3s)");
        } else {
            watchdog_alerts++;
            if (bits & BIT_ERR_TEMP)
                ca_stdout_printf_n("WDG: ALERT - temperature error! (#%u)",
    watchdog_alerts);
            if (bits & BIT_ERR_PRES)
                ca_stdout_printf_n("WDG: ALERT - pressure error! (#%u)",
    watchdog_alerts);
        }
    }
    return 0;
}

void ca_os_main(void) {
    ca_event_init(&eg_sensors);
    ca_event_init(&eg_errors);

    ca_os_job("temp",  CA_TH_LEVEL2, task_sensor_temp, NULL, STACK_SMALL,
    CA_JOB_STD, CA_FAULT_ASSERT);
    ca_os_job("pres",  CA_TH_LEVEL2, task_sensor_pres, NULL, STACK_SMALL,
    CA_JOB_STD, CA_FAULT_ASSERT);
    ca_os_job("hum",   CA_TH_LEVEL2, task_sensor_hum,  NULL, STACK_SMALL,
    CA_JOB_STD, CA_FAULT_ASSERT);
    ca_os_job("aggr",  CA_TH_LEVEL1, task_aggregator,  NULL, STACK_SMALL,
    CA_JOB_STD, CA_FAULT_ASSERT);
    ca_os_job("wdog",  CA_TH_LEVEL1, task_watchdog,    NULL, STACK_SMALL,
    CA_JOB_STD, CA_FAULT_ASSERT);

    EVAL_CLI( ca_cli_register(&s_user_cli_node); )
    ca_os_run(true);
}
```

40.1.4 Detailed analysis

Two independent event groups

The example uses **two** separate `ca_event_group_t` structures:

```
static ca_event_group_t eg_sensors;   /* bits 0-2: data ready */
static ca_event_group_t eg_errors;    /* bits 8-9: errors */
```

Separating the two groups is a design choice: it avoids ambiguity in the bitmask and allows for different waiting policies (WAIT_ALL for sensors, WAIT_ANY for errors). A single event group with 32 bits would be technically sufficient, but less readable.

WAIT_ALL with auto-clear

```
u32 bits = ca_event_wait_all(&eg_sensors, BITS_ALL, true, CA_WAIT_FOREVER);
```

Parameter	Value	Meaning
`bits_to_wait`	`BITS_ALL` = 0x07	Wait for bits 0, 1, and 2 simultaneously
`auto_clear`	`true`	Clear bits 0, 1, 2 after match
`timeout_ms`	`0`	Infinite wait — only WAIT_ALL makes sense without timeout

Auto-clear ensures that each sample is processed *only once*: after aggregation, the bits return to zero and the sensors must set them again in the next cycle.

Why `CA_WAIT_FOREVER`? It makes no sense for the aggregator to have a timeout: if data is missing, it simply waits. The timeout is, however, fundamental for the watchdog.

WAIT_ANY with timeout

```
u32 bits = ca_event_wait_any(&eg_errors, BITS_ERR, true, 3000);
```

The watchdog wakes up in two cases:

1. **Injected error** (bits != 0): `ca_event_set` immediately wakes up the task, the bits are read and cleared. The watchdog prints the alert and returns to waiting.
2. **3s timeout** (bits == 0): no error in 3 seconds — the watchdog prints "OK" and re-arms the timeout.

This pattern is equivalent to a *software watchdog*: periodic verification of the system's health status with immediate reaction to events.

Internal wake-up mechanism

Internally, `ca_event_wait` with `timeout_ms > 0` uses the task's **SLEEP** state:

```
ca_event_wait_any(timeout_ms=3000)
           ca_os_sleep(3000)      SLEEP state, svegliato da SysTick countdown
                                or by ca_os_wake_up() called by ca_event_set()
```

When `ca_event_set(&eg_errors, BIT_ERR_TEMP)` is called from the CLI:

1. Sets the bit in `eg_errors.bits`
2. `wakeup_waiters()` checks the queue: the watchdog has a satisfied condition
3. Saves `meta->result = BIT_ERR_TEMP` *before* the auto-clear
4. Calls `ca_scheduler_signal_wake_up()` (for BLOCKED tasks) and `ca_os_wake_up()` (for SLEEP tasks)

5. The watchdog wakes up from the `ca_os_sleep()` and reads `meta->result`

With `timeout_ms = 0` (aggregator), the state used is **BLOCKED**, woken up exclusively by `ca_scheduler_signal_wake_up`.

auto-clear and race condition

The `result` field in the internal structure `ca_event_waiter_meta_t` resolves a subtle race condition: the auto-clear clears the bits *before* the woken-up task can read them, so the matched value is saved in `meta->result` and returned directly to the task upon wake-up.

Error injection from CLI

```
static bool user_cli_handler(c_str *pcmd)
{
    const char *p = ca_skip_sp((*pcmd)->data);
    ...
    if (ca_strncmp(p, "err_t", 5) == 0) {
        ca_event_set(&eg_errors, BIT_ERR_TEMP);
        CLI_PUT_N("Injected ERR_TEMP");
        return true;
    }
```

`ca_event_set()` is thread-safe (it uses `ca_spinlock_irq_t`): it can be called from the CLI task, from user tasks, or from an ISR without risk of corruption. The watchdog will be woken up immediately, regardless of how much time is left before the 3s expiration.

40.1.5 Expected output

Normal operation

```
AGG #142  T=28.6  P=101813  H=71.9
AGG #143  T=28.9  P=101818  H=71.9
AGG #144  T=28.8  P=101820  H=72.4
AGG #145  T=28.9  P=101827  H=72.5
WDG: OK (no errors in 3s)
AGG #146  T=29.1  P=101831  H=72.8
AGG #147  T=29.4  P=101832  H=73.0
AGG #148  T=29.3  P=101838  H=73.3
WDG: OK (no errors in 3s)
```

Aggregations trigger ~7 times per watchdog cycle (3s). The rate is determined by the slowest sensor: pressure at 700ms is the bottleneck. All 3 bits must be set *simultaneously*, so they aggregate approximately `3000 / 700  4.3` times every 3 seconds (in practice 6-8 due to sensor phase shifting).

The sensor bits shown by `show` are always 0x00 because they are reset by the auto-clear immediately after each aggregation.

With error injection

```
c.a.O.S > err_t

Iniettato ERR_TEMP
WDG: ALERT - temperature error! (#1)

c.a.O.S > err_p

Iniettato ERR_PRES
WDG: ALERT - pressure error! (#2)
```

The watchdog wakes up immediately after `ca_event_set()`, well before the 3s expiration. After the alert, it returns to waiting with a 3s timeout restarted from zero.

`show` command output

```
c.a.O.S > show

--- Example 17 ---
  Aggregazioni   : 181
  WDG alerts     : 2
  Sensor bits    : 0x00 (T=0 P=0 H=1)
  Error bits     : 0x00
```

`Sensor bits` may show 0x00 (all cleared after aggregation) or a partial value if read between one set and another. `Error bits` is 0x00 after the watchdog has consumed the alerts with auto-clear.

40.1.6 Required configuration in `os_config.h`

```
#define ENABLE_CLI       1    /* CLI for user commands */
#define ENABLE_STDOUT      1    /* Log system for ca_stdout_* */
```

No special flags are required: `ca_event_group_t` is always available (not conditioned by the `EN_` flag).

40.1.7 Created threads

Thread	Name	Priority	Role
idle	`idle`	0	Scheduler idle task
log	`log`	—	Asynchronous output
CLI	`cli`	—	Command reception
USB CDC	`usb_cdc`	—	I/O channel
`temp`	`temp`	LEVEL2	Temperature sensor (400ms)
`pres`	`pres`	LEVEL2	Pressure sensor (700ms)
`hum`	`hum`	LEVEL2	Humidity sensor (550ms)
`aggr`	`aggr`	LEVEL1	WAIT_ALL aggregator
`wdog`	`wdog`	LEVEL1	Watchdog WAIT_ANY + timeout

40.1.8 Summary

1. **Two wait modes**: WAIT_ALL synchronizes on a set of events that must all occur; WAIT_ANY reacts to the first event in a set, with an optional timeout.

2. **auto_clear**: matched bits are atomically cleared after the match, ensuring that each event is processed only once without an explicit reset.

3. **SLEEP timeout vs BLOCKED infinite wait**: `ca_event_wait` uses distinct task states. With `timeout_ms=0` the task is BLOCKED (only a signal wakes it up); with `timeout_ms>0` it is SLEEP (it wakes up due to a countdown or an early signal).

4. **ISR-safe**: `ca_event_set()` uses spinlocks with IRQ disabling — safe from interrupt handlers and any task.

5. **Software watchdog pattern**: WAIT_ANY with a periodic timeout is the standard pattern for supervision tasks: it reacts to errors instantly but also verifies the absence of errors every N seconds.

6. **Scalability**: with 32 bits available, a single event group can coordinate up to 32 independent conditions without additional data structures.

40.1.9 Verification of waits with timeout (`twait`)

Since 09/27, a task waiting for the event group **with timeout** (`ca_event_wait_any()`) registers in the same wait list as those waiting without a timeout, parked in `SLEEP` until expiration, and the signaler wakes it up immediately (S2_02 §8.8). Previously, it performed polling every 10 ms outside the list: it would wake up up to 10 ms after the signal.

The command `twait [sec]` (default 3 s) measures it with a peer task created at the first call. The wait runs in the command task:

1. **latency** — 300 cycles: the peer sets the bit after 1..8 ms randomly; the waiter, with a 20 ms timeout, must obtain it (`lost=0`), not before the signal (`spur=0`), and be running within 2 ms (`lat_max` limit, in ticks);
2. **timeout** — no one sets it; for `sec` seconds, T = 1..20 ms: the return never arrives **before** T (`early=0`), at most 2 ms after (`late=0`), and always with `false` (`got=0`).

One line, closed by `PASS` or `FAIL`. On H753 (tick 200 µs):

```
TW event : rounds=300 lat_max=0/10 lost=0 spur=0 | tmo n=291 early=0 late=0 got
   =0 over_max=0/10  PASS
```

On the previous kernel: PASS: the waiter with timeout was already in the list. Here `twait` is a regression; the fix (parking inside the lock) has a window that is too narrow for this test.

In the bench it is case `timed_wait` of `test_ex17.cpp` (helper `test/hw_test/inc/caosTimedWait.h`). Manually:

```
caos_shell -e 17 -q -t 40 -u "TW .*(PASS|FAIL)" -g "^TW" "twait 3"
```

Chapter 41

SW Timer: One-Shot and Periodic

41.1 Example 18 — SW Timer: one-shot and periodic

41.1.1 Introduction

The **Software Timer** is a mechanism that allows executing a callback function after a predefined delay, without occupying a dedicated task for each timing. The caOS timer system manages up to 16 simultaneous timers through a single daemon task (`tmr`), which updates their counters at every tick and calls the callbacks upon expiration.

There are two modes:

- **One-shot**: the callback is executed only once, then the timer enters the EXPIRED/STOP state.
- **Periodic**: the callback is executed periodically, restarting automatically after each expiration.

Callbacks are executed in the context of the daemon task `tmr`, **not** from an ISR. They must be short and must not block with sleep or wait calls.

In this example, three timers are created: - **LED** periodic at 500ms — simulates a LED toggle - **Stats** periodic at 2000ms — prints counters - **One-shot** at 3000ms — triggers once, rearmable via CLI

Demonstrated features

Function	Description
`ca_sw_timer_init()`	Initializes the timer system and creates the daemon task `tmr`
`ca_sw_timer_create()`	Creates a timer (one-shot or periodic) with a callback
`ca_sw_timer_start()`	Starts the counting
`ca_sw_timer_stop()`	Stops the timer (STOP state)
`ca_sw_timer_reset()`	Resets the counter and restarts
`ca_sw_timer_get_state()`	Reads the state: RUNNING / STOPPED / EXPIRED
`ca_sw_timer_get_remaining()`	Remaining milliseconds before expiration
`ca_sw_timer_count_active()`	Number of timers currently running

CLI Commands

Command	Description
`arm`	(Re)arms the one-shot timer (3s) — restart from zero
`stop`	Stops the periodic stats timer
`start`	Restarts the periodic stats timer
`show`	Shows fire counters and timer status
`help`	List of available commands

41.1.2 Example architecture

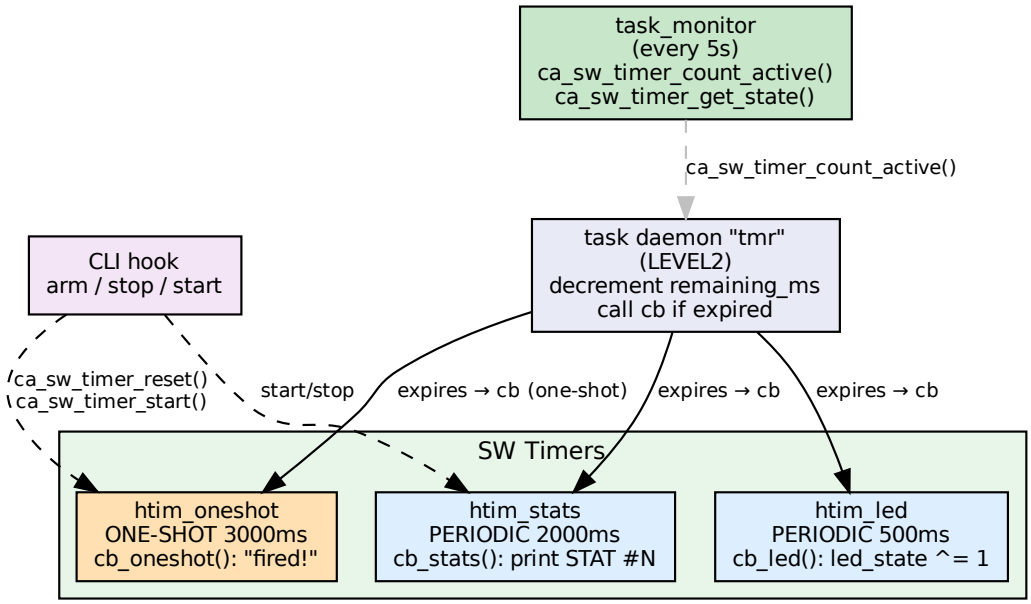


Figure 41.1: Example 18 Architecture — SW Timers

41.1.3 Source code

The file is located in `examples/example_18/ca_os_ex18.c`:

```
#include "caos.h"

#if EN_SW_TIMER == 1

static ca_sw_timer_handle_t htim_led;
static ca_sw_timer_handle_t htim_stats;
static ca_sw_timer_handle_t htim_oneshot;

static volatile u32 led_ticks;
static volatile u32 stats_ticks;
static volatile u32 oneshot_fired;
static volatile u8  led_state;

```

```
14 /* Callbacks run by the "tmr" daemon task */
15 static void cb_led(void *arg) {
16     (void)arg;
17     led_state ^= 1u;
18     led_ticks++;
19 }
20
21 static void cb_stats(void *arg) {
22     (void)arg;
23     stats_ticks++;
24     ca_stdout_printf_n("STAT #%u  LED=%u  OS=%u",
25         stats_ticks, led_ticks, oneshot_fired);
26 }
27
28 static void cb_oneshot(void *arg) {
29     (void)arg;
30     oneshot_fired++;
31     ca_stdout_printf_n("ONE-SHOT fired! (#%u)", oneshot_fired);
32 }
33
34 /* Monitor task: prints the timer state every 5s */
35 static u32 task_monitor(void *arg) {
36     (void)arg;
37     ca_os_sleep(1000);
38     while (1) {
39         u8 active = ca_sw_timer_count_active();
40         ca_sw_timer_state_t s1 = ca_sw_timer_get_state(htim_led);
41         ca_sw_timer_state_t s2 = ca_sw_timer_get_state(htim_stats);
42         ca_sw_timer_state_t s3 = ca_sw_timer_get_state(htim_oneshot);
43         u32 rem3 = ca_sw_timer_get_remaining(htim_oneshot);
44
45         ca_stdout_printf_n("=== TIMER STATUS (active=%u) ===", active);
46         ca_stdout_printf_n("  led     : %s (fires=%u)",
47             s1 == CA_TIMER_RUNNING ? "RUN" : "STOP", led_ticks);
48         ca_stdout_printf_n("  stats   : %s (fires=%u)",
49             s2 == CA_TIMER_RUNNING ? "RUN" : "STOP", stats_ticks);
50         ca_stdout_printf_n("  oneshot: %s rem=%ums (fires=%u)",
51             s3 == CA_TIMER_RUNNING ? "RUN" :
52             s3 == CA_TIMER_EXPIRED ? "EXP" : "STOP",
53             rem3, oneshot_fired);
54         ca_stdout_printf_n("================================");
55         ca_os_sleep(5000);
56     }
57     return 0;
58 }
59
60 void ca_os_main(void) {
61     ca_sw_timer_init();
62
63     htim_led     = ca_sw_timer_create(500,  CA_TIMER_PERIODIC,  cb_led,     NULL
       );
64     htim_stats   = ca_sw_timer_create(2000, CA_TIMER_PERIODIC,  cb_stats,   NULL
       );
65     htim_oneshot = ca_sw_timer_create(3000, CA_TIMER_ONE_SHOT,  cb_oneshot, NULL
       );
66
67     ca_sw_timer_start(htim_led);
68     ca_sw_timer_start(htim_stats);
69     ca_sw_timer_start(htim_oneshot);
70
71     ca_os_job("monitor", CA_TH_LEVEL1, task_monitor, NULL,
72               STACK_SMALL, CA_JOB_STD, CA_FAULT_ASSERT);
73
```

```
    EVAL_CLI( ca_cli_register(&s_user_cli_node); )
    ca_os_run(true);
}
```

41.1.4 Detailed analysis

System timer initialization

```
ca_sw_timer_init();
```

Create the daemon task `tmr` with stack `CA_TIMER_TASK_STACK` (512B default) and priority `CA_TIMER_TASK_PRIO` (2 = LEVEL2). The daemon executes a loop:

```
while (1) {
    ca_os_sleep(1);                    /* 1ms resolution */
    for (each active timer) {
        timer.remaining_ms -= elapsed;
        if (timer.remaining_ms == 0) {
            timer.cb(timer.cb_arg);    /* calls the callback */
            if (PERIODIC) rimpiazza; else EXPIRED;
        }
    }
}
```

The minimum resolution is **1ms** (one SysTick tick). Overhead: the daemon is active once per tick, but it sleeps almost immediately if no timer expires.

Timer creation and start

```
ca_sw_timer_handle_t htim_led = ca_sw_timer_create(500, CA_TIMER_PERIODIC,
    cb_led, NULL);
ca_sw_timer_start(htim_led);
```

`ca_sw_timer_create()` allocates a slot in the internal array `ca_sw_timer_t[CA_TIMER_MAX]` and returns a handle (index + 1, 0 = `CA_TIMER_INVALID`). The timer does not start until `ca_sw_timer_start()` is called.

`ca_sw_timer_handle_t` is a `u8` — simple index, zero overhead.

Difference between ONE-SHOT and PERIODIC

Type	Behavior after expiration
`CA_TIMER_ONE_SHOT`	Enters the EXPIRED/STOP state, does not restart
`CA_TIMER_PERIODIC`	Automatically restarts with the same period

To rearm the one-shot:

```
ca_sw_timer_reset(htim_oneshot);   /* reset remaining_ms = period_ms */
ca_sw_timer_start(htim_oneshot);   /* rimette in RUNNING */
```

Golden rule of callbacks

Callbacks are called **in the context of the daemon task** `tmr`. Rules:

1. **Do not call** `ca_os_sleep()` — it blocks the daemon and delays all other timers
2. **Do not call blocking** `ca_mutex_lock()` — same issue
3. **Fast operations**: incrementing counters, GPIO toggling, `ca_sem_post()`, `ca_event_set()` — OK

4. **The callback itself must not create or modify timers** — risk of internal deadlock

If you need to perform a long operation upon timer expiration, the callback must *signal* a task (using a semaphore or event group) that will perform the work.

```
/* Correct pattern for long operations */
static ca_semaphore_t sem_timer_work;

static void cb_trigger(void *arg) {
    ca_sem_post(&sem_timer_work);   /* signals, does not block */
}

static u32 task_worker(void *arg) {
    (void)arg;
    while (1) {
        ca_sem_wait(&sem_timer_work);
        /* long work here: OK, we are in a normal task */
    }
}
```

Timer states

```
                  ca_sw_timer_start()
    STOPPED

    RUNNING

           ca_sw_timer_stop()                scadenza


                                                 (PERIODIC: torna a RUNNING)

                                      EXPIRED (ONE-SHOT only)
           ca_sw_timer_start()
```

`ca_sw_timer_reset()` reports `remaining_ms = period_ms` but does not change the state — if it was STOPPED, it remains STOPPED until `start()` is called.

`ca_sw_timer_get_remaining()`

```
u32 rem3 = ca_sw_timer_get_remaining(htim_oneshot);
```

Returns the remaining milliseconds before expiration. It is 0 if the timer is STOPPED or EXPIRED. Useful for diagnostics and visualization (such as in monitor printing).

41.1.5 Expected output

At startup — automatic statistics

The stats timer triggers every 2s, the monitor every 5s. At startup, the one-shot starts and triggers after 3s.

```
STAT #1  LED=4  OS=0
STAT #2  LED=8  OS=0
ONE-SHOT fired! (#1)
STAT #3  LED=12  OS=1
=== TIMER STATUS (active=1) ===
```

```
  led    : RUN (fires=10)
  stats  : RUN (fires=5)
  oneshot: STOP rem=0ms (fires=1)
================================
```

`active=1` after the one-shot triggers (only the LED remains active because the monitor shows the state *before* stats triggers again in the monitor loop — the two periodicities of 2s and 5s overlap randomly).

Command `arm` — one-shot rearm

```
c.a.O.S > arm

One-shot armed (3s)
=== TIMER STATUS (active=2) ===
  led    : RUN (fires=394)
  stats  : STOP (fires=32)
  oneshot: RUN rem=2679ms (fires=3)
================================
ONE-SHOT fired! (#4)
=== TIMER STATUS (active=1) ===
  led    : RUN (fires=399)
  stats  : STOP (fires=32)
  oneshot: STOP rem=0ms (fires=4)
================================
```

The monitor shows the one-shot in RUN with 2679ms remaining — the print occurred ~321ms after the rearm. Three seconds later, "ONE-SHOT fired! (#4)" confirms the expiration; the timer returns to STOP.

Commands `stop` / `start`

```
c.a.O.S > stop

Timer stats stopped

c.a.O.S > start

Timer stats avviato
=== TIMER STATUS (active=2) ===
  led    : RUN (fires=410)
  stats  : RUN (fires=32)
  oneshot: STOP rem=0ms (fires=4)
================================
```

After `stop` the stats timer enters STOPPED and no longer calls `cb_stats`. After `start` it resumes; `fires=32` remains unchanged until the next 2s expiration.

Interleaving CLI/output

```
st=== TIMER STATUS (active=1) ===
  led    : RUN (fires=405)
op
Timer stats stopped
```

The characters `st` and `op` appear in the middle of the monitor output — this is normal behavior for asynchronous logging on a shared USB CDC channel (the same observed in example 16).

41.1.6 Created tasks

Thread	Name	Priority	Role
idle	`idle`	0	Scheduler idle task
log	`audit`	—	Asynchronous output
CLI	`cli`	—	Command reception
USB CDC	`usb cdc`	—	I/O channel
timer daemon	`tmr`	LEVEL2	Timer expiration management
monitor	`monitor`	LEVEL1	Print timer status every 5s

41.1.7 Configuration required in `os_config.h`

```
#define EN_SW_TIMER      1    /* Software timer system */
#define CA_TIMER_MAX     16   /* Maximum number of timers (default) */

#define ENABLE_CLI       1
#define ENABLE_STDOUT       1
```

41.1.8 Summary

1. **Single daemon task**: a single `tmr` task manages all timers. The overhead is proportional to the number of active timers, not the number of callbacks.

2. **One-shot vs Periodic**: one-shot triggers once and stops; periodic restarts automatically. Re-arming a one-shot requires `reset()` + `start()`.

3. **Lightweight callbacks**: callbacks run in the daemon task — they must be very short. For heavy work, use the signal-task pattern: the callback performs `ca_sem_post()`, and a normal task executes the work.

4. **1ms resolution**: the minimum granularity is the SysTick tick (`OS_TICK_US` in `objmap.h`: 200 µs on H753). Very short periods (< 5ms) may accumulate jitter due to daemon overhead.

5. **Handle as index**: `ca_sw_timer_handle_t` is a `u8`. Slots exhausted → `CA_TIMER_INVALID` (0). Always verify the return of `ca_sw_timer_create()` in production code.

6. `ca_sw_timer_get_remaining()`: useful for debugging and implementing software watchdogs: if a periodic timer does not trigger within `remaining + tolerance`, the system is locked.

Chapter 42

RW Lock and Barrier

42.1 Example 19 — RW Lock and Barrier

42.1.1 Introduction

This example presents two complementary synchronization primitives:

Read-Write Lock (`ca_rwlock_t`): allows multiple tasks to read a shared resource *simultaneously*, while ensuring exclusive access only during writing. It is the optimal solution for data that is read frequently and written rarely: it avoids the bottleneck of a simple mutex (which would block readers from each other as well).

Barrier (`ca_barrier_t`): an N-task synchronization point. All participants call `ca_barrier_wait()` and block until the last one arrives. The system resets automatically, allowing for repeated use in a loop (auto-reset barrier).

Demonstrated Features

Function	Description
`ca_rwlock_init()`	Initializes the RW lock
`ca_rwlock_read_lock()`	Acquires read lock (shared)
`ca_rwlock_read_unlock()`	Releases read lock
`ca_rwlock_write_lock()`	Acquires write lock (exclusive)
`ca_rwlock_write_unlock()`	Releases write lock, wakes up waiting tasks
`ca_barrier_init()`	Initializes barrier with N participants
`ca_barrier_wait()`	Waits at the barrier, returns `true` only to the last one

CLI Commands

Command	Description
`show`	Statistics: reads, writes, barrier phases, CFG version
`help`	List commands

42.1.2 Example Architecture

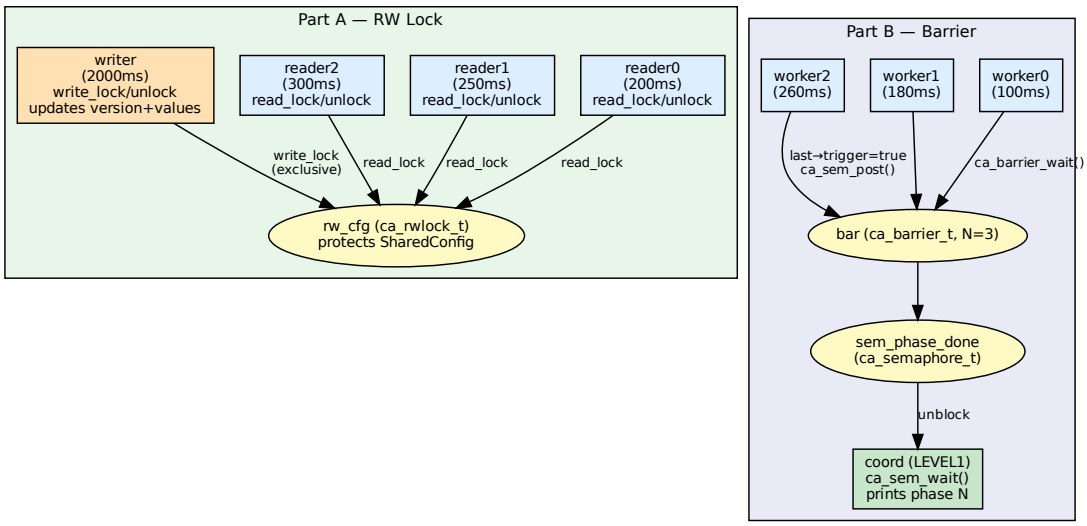


Figure 42.1: Example 19 Architecture — RW Lock + Barrier

42.1.3 Source Code

The file is located in `examples/example_19/ca_os_ex19.c`:

```c
#include "caos.h"

#if EN_RWLOCK == 1 && EN_BARRIER == 1

/* === PARTE A: RW Lock === */

#define CFG_SIZE  8

typedef struct {
    u32 value[CFG_SIZE];
    u32 version;
} SharedConfig;

static ca_rwlock_t    rw_cfg;
static SharedConfig cfg;

/* 3 reader task: leggono simultaneamente */
static u32 task_reader(void *arg) {
    const u32 idx = ca_os_thid_from_handle(ca_os_self_handle());
    (void)arg;
    while (1) {
        ca_os_sleep(200 + idx * 50u);

        ca_rwlock_read_lock(&rw_cfg);
        /* Consistent read     other readers can enter in parallel */
        u32 ver = cfg.version;
        u32 sum = 0;
        for (u32 i = 0; i < CFG_SIZE; i++) sum += cfg.value[i];
        ca_rwlock_read_unlock(&rw_cfg);

```

```
        reads_ok++;
        (void)ver; (void)sum;
    }
    return 0;
}

/* 1 writer task: updates every 2s with exclusive access */
static u32 task_writer(void *arg) {
    (void)arg;
    u32 seq = 0;
    while (1) {
        ca_os_sleep(2000);

        ca_rwlock_write_lock(&rw_cfg);
        cfg.version = ++seq;
        for (u32 i = 0; i < CFG_SIZE; i++)
            cfg.value[i] = seq * (i + 1u);
        ca_rwlock_write_unlock(&rw_cfg);

        writes_ok++;
        ca_stdout_printf_n("CFG update v%u (reads so far: %u)", seq, reads_ok);
    }
    return 0;
}

/* === PART B: Barrier === */

#define N_WORKERS  3

static ca_barrier_t   bar;
static ca_semaphore_t sem_phase_done;

/* 3 worker tasks: variable work, then barrier */
static u32 task_worker(void *arg) {
    const u32 idx = ca_os_thid_from_handle(ca_os_self_handle());
    (void)arg;
    while (1) {
        ca_os_sleep(100u + idx * 80u);   /* 100, 180, 260ms */

        bool trigger = ca_barrier_wait(&bar);
        if (trigger) {
            barrier_phases++;
            ca_stdout_printf_n("BARRIER phase %u - trigger=worker%u",
                barrier_phases, idx);
            ca_sem_post(&sem_phase_done);
        }
    }
    return 0;
}

/* Coordinator: receives a notification from the trigger worker */
static u32 task_coordinator(void *arg) {
    (void)arg;
    while (1) {
        ca_sem_wait(&sem_phase_done);
        ca_stdout_printf_n("COORD: phase %u done, reads=%u writes=%u",
            barrier_phases, reads_ok, writes_ok);
    }
    return 0;
}

void ca_os_main(void) {
    ca_rwlock_init(&rw_cfg);
```

```
94      ca_barrier_init(&bar, N_WORKERS);
95      ca_sem_init(&sem_phase_done, 0, 16);
96
97      /* RW Lock */
98      ca_os_job("reader0", CA_TH_LEVEL2, task_reader, NULL, STACK_SMALL, ...);
99      ca_os_job("reader1", CA_TH_LEVEL2, task_reader, (void *)(uintptr_t)1,
        STACK_SMALL, ...);
100     ca_os_job("reader2", CA_TH_LEVEL2, task_reader, (void *)(uintptr_t)2,
        STACK_SMALL, ...);
101     ca_os_job("writer",  CA_TH_LEVEL3, task_writer, NULL, STACK_SMALL, ...);
102
103     /* Barrier */
104     ca_os_job("worker0", CA_TH_LEVEL2, task_worker, NULL, STACK_SMALL, ...);
105     ca_os_job("worker1", CA_TH_LEVEL2, task_worker, (void *)(uintptr_t)1,
        STACK_SMALL, ...);
106     ca_os_job("worker2", CA_TH_LEVEL2, task_worker, (void *)(uintptr_t)2,
        STACK_SMALL, ...);
107     ca_os_job("coord",   CA_TH_LEVEL1, task_coordinator, NULL, STACK_SMALL, ...)
        ;
108
109     EVAL_CLI( ca_cli_register(&s_user_cli_node); )
110     ca_os_run(true);
111 }
```

42.1.4 Detailed analysis

RW Lock — write-preferring strategy

`ca_rwlock_t` implements the **write-preferring** policy: a waiting writer blocks the entry of *new* readers, even if there are already active readers. This prevents writer starvation in scenarios with very frequent readers.

Typical sequence:

```
1 Time:    0ms     200ms   400ms   600ms   800ms   1000ms  ...  2000ms
2 reader0:      R     D                   R     D                    R     D
            R  D
3 reader1:           R      D                        R      D
          R     D
4 reader2:                R        D
            R        D                                 R  D
5 writer:                                                        W     R     WR
6                                                      (esclusivo)
```

During the reading phase, multiple readers enter in parallel (`readers` counter > 0). When the writer calls `ca_rwlock_write_lock()`, it waits for `readers == 0` before proceeding. In the meantime, new readers find the writer waiting and block themselves in turn.

Read/write ratio

With 3 readers every 200-300ms and 1 writer every 2000ms:

```
1 Rate lettori      3    (1/250ms) = 12 letture/s
2 Rate writer   = 1 write every 2s = 0.5 writes/s
3 Ratio atteso      24:1
```

The output shows ~12-14 readings for each CFG update (2s / ~150ms average per reader), confirming the behavior:

```
1 CFG update v8  (reads so far: 95)
2 CFG update v9  (reads so far: 106)        11 letture in 2s
3 CFG update v10 (reads so far: 119)        13 letture in 2s
4 CFG update v11 (reads so far: 130)        11 letture in 2s
```

Barrier — trigger determinism

```
BARRIER phase 17     trigger=worker11
BARRIER phase 18     trigger=worker11
...
BARRIER phase 30     trigger=worker11
```

The trigger is **always** `worker11` (the scheduler index assigned to `worker2`). This is deterministic: the sleep times are fixed (100ms, 180ms, 260ms), so worker2 *always* arrives last at the barrier.

The number "11" is not the `idx=2` parameter passed to the task, but the **global scheduler index** assigned to that task at the moment of `ca_os_job()`. The index depends on the order of task creation in the example.

```
worker0: sleep 100ms     arrives FIRST
worker1: sleep 180ms     arrives SECOND
worker2: sleep 260ms     arrives LAST = trigger     true
```

The barrier resets automatically (`phase` is incremented) after each cycle, so workers can execute infinite phases without reinitialization.

Coordinator via semaphore

The coordinator does not participate in the barrier — it is an *external* task that receives notification via semaphore:

```
/* Trigger worker (last to arrive) */
bool trigger = ca_barrier_wait(&bar);
if (trigger) {
    barrier_phases++;
    ca_sem_post(&sem_phase_done);   /* notify the coordinator */
}

/* Coordinatore */
ca_sem_wait(&sem_phase_done);   /* waits for every phase */
```

This pattern decouples the internal synchronization of workers from the coordination logic. The coordinator can start the next processing phase, save data, or communicate with peripherals — without blocking the workers.

Comparison: RW Lock vs Simple Mutex

Scenario	Mutex (ca_mutex_t)	RW Lock (ca_rwlock_t)
3 readers, 1 writer	Readers block each other	Parallel readers
Read throughput	~1/3 (serialized)	~1 (parallel)
Write latency	Low	Can increase with many readers
Implementation	Simple	More complex
When to use	Frequently modified data	Frequently read, rarely written data

Required configuration in `os_config.h`

```
#define EN_RWLOCK    1    /* Read-Write Lock */
#define EN_BARRIER   1    /* Barrier N task */

#define ENABLE_CLI   1
#define ENABLE_STDOUT  1
```

42.1.5 Expected output

Normal operation

```
CFG update v8 (reads so far: 95)
BARRIER phase 17     trigger=worker11
COORD: fase 17 completata, reads=100 writes=8
BARRIER phase 18     trigger=worker11
COORD: fase 18 completata, reads=106 writes=8
CFG update v9 (reads so far: 106)
BARRIER phase 19     trigger=worker11
COORD: fase 19 completata, reads=111 writes=9
```

The barriers trigger every ~260ms (the rate of the slowest worker). CFG updates occur every 2s. The two patterns are independent and interleave in the output.

Command `show`

```
c.a.O.S > show

--- Example 19 ---
  Reads ok        : 153
  Writes ok       : 12
  Barrier phases : 26
  CFG version     : 12
```

`CFG version == Writes ok` confirms that every write increments the version by 1. `Barrier phases` grows approximately 8 times faster than `Writes ok` (2000ms / 260ms 7.7).

Interleaving CLI/output

```
sBARRIER phase 25     trigger=worker11
howBARRIER phase 26     trigger=worker11
```

Characters `s`, `how` of command `show` appear in the middle of task output — normal with asynchronous logging on a shared channel.

42.1.6 Created threads

Thread	Name	Priority	Role
idle	`idle`	0	Scheduler idle
log	`audit`	—	Asynchronous output
CLI	`cli`	—	Command reception
USB CDC	`usb cdc`	—	I/O channel
`reader0`	`reader0`	LEVEL2	RW reader (200ms)
`reader1`	`reader1`	LEVEL2	RW reader (250ms)
`reader2`	`reader2`	LEVEL2	RW reader (300ms)
`writer`	`writer`	LEVEL3	RW writer (2000ms)
`worker0`	`worker0`	LEVEL2	Barrier worker (100ms)
`worker1`	`worker1`	LEVEL2	Barrier worker (180ms)
`worker2`	`worker2`	LEVEL2	Barrier worker (260ms)
`coord`	`coord`	LEVEL1	Barrier coordinator

42.1.7 Summary

1. **RW Lock**: allows parallel reads between tasks with shared access. The writer blocks only when writing, not during all reads. Ideal for configuration structures or frequently read caches.

2. **Write-preferring**: a waiting writer blocks new readers, preventing its starvation in high read-traffic systems.

3. **Barrier auto-reset**: after each synchronization, the barrier is ready for the next cycle without reinitialization. The internal `phase` field ensures that delayed wake-ups do not interfere with the next cycle.

4. **Unique trigger**: only the last task to arrive at the barrier receives `true`. Using this return value for post-synchronization actions (starting the next phase, signaling the coordinator) avoids duplication.

5. **Coordinator pattern**: the trigger must not perform long operations — use `ca_sem_post()` to delegate to the coordinator. Workers restart immediately in the next cycle.

6. **Trigger determinism**: with fixed sleeps, the same worker always arrives last. In real algorithms, the workload varies and the trigger changes at each phase — the barrier handles both cases correctly.

Chapter 43

Block Pool, sleep_until and Timed Primitives

43.1 Example 20 — Block Pool, sleep_until and Timed Primitives

43.1.1 Introduction

This example integrates three advanced caOS mechanisms into a single program:

Block Pool (`ca_block_pool_t`): fixed-size block allocator. No heap, zero fragmentation, O(1) allocation time, and ISR-safe. Ideal for embedded systems with deterministic memory requirements.

`ca_os_sleep_until`: anti-drift variant of `ca_os_sleep`. A periodic task maintains the exact frequency even if the loop body takes variable time, by accumulating the delay and compensating autonomously.

Timed primitives (`ca_mutex_timedlock`, `ca_sem_timedwait`): mutex and semaphore acquisition with explicit timeout. The caller can handle the "resource not available within N milliseconds" case without blocking indefinitely.

Demonstrated Features

Function	Description
`ca_block_pool_init()`	Initializes the pool on a static buffer
`ca_block_alloc()`	Allocates a block in O(1)
`ca_block_free()`	Returns a block to the pool in O(1)
`ca_block_pool_free_count()`	Number of free blocks
`ca_os_sleep_until()`	Periodic sleep with cumulative anti-drift
`ca_mutex_timedlock()`	Acquires mutex with timeout; `true`=OK, `false`=timeout
`ca_sem_timedwait()`	Acquires semaphore with timeout; `true`=OK, `false`=timeout

CLI Commands

Command	Description
`show`	Full statistics: alloc/consumed, drift, timedlock/timedwait
`twait [sec]`	Mutex verification with timeout: wake-up latency and deadline precision
`twkill [sec]`	Verification of direct mutex and semaphore handover to a waiter removed while waiting
`help`	Command list

43.1.2 Example Architecture

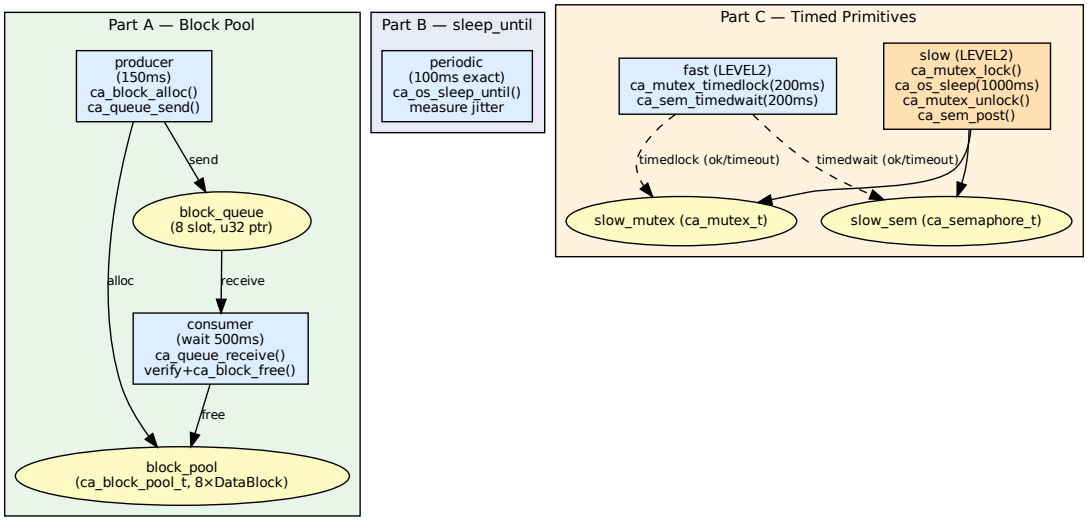


Figure 43.1: Example 20 Architecture — Block Pool + Timed Primitives

43.1.3 Source Code

The file is located in `examples/example_20/ca_os_ex20.c`:

```
#include "caos.h"

#if EN_BLOCK_POOL == 1

/* === PARTE A: Block Pool === */

#define BLOCK_DATA_SIZE  32
#define POOL_N_BLOCKS    8

typedef struct {
    u32 seq;
    u8  data[BLOCK_DATA_SIZE];
    u32 checksum;
} DataBlock;

static u8 pool_buf[POOL_N_BLOCKS * sizeof(DataBlock)];
static ca_block_pool_t  block_pool;

```

```
CAQUEUE_STATIC_DEFINE(block_queue, POOL_N_BLOCKS, sizeof(u32));

static volatile u32 alloc_ok, alloc_fail, consumed;

static u32 task_producer(void *arg) {
    (void)arg;
    u32 seq = 0;
    while (1) {
        ca_os_sleep(150);
        DataBlock *blk = (DataBlock *)ca_block_alloc(&block_pool);
        if (blk == NULL) { alloc_fail++; continue; }
        blk->seq = seq++;
        for (u32 i = 0; i < BLOCK_DATA_SIZE; i++) blk->data[i] = (u8)(seq + i);
        blk->checksum = blk->seq ^ 0xDEADBEEFu;
        alloc_ok++;
        u32 ptr_val = (u32)(uintptr_t)blk;
        bool sent = ca_queue_static_send(&block_queue.q, &ptr_val,
    CA_WAIT_FOREVER);
        if (!sent) { ca_block_free(&block_pool, blk); alloc_fail++; }
    }
    return 0;
}

static u32 task_consumer(void *arg) {
    (void)arg;
    while (1) {
        u32 ptr = 0;
        if (!ca_queue_static_receive(&block_queue.q, &ptr, 500)) continue;
        DataBlock *blk = (DataBlock *)(uintptr_t)ptr;
        consumed++;
        ca_block_free(&block_pool, blk);
    }
    return 0;
}

/* === PARTE B: sleep_until === */

static volatile u32 periodic_count;
static volatile u32 drift_max_ms;

static u32 task_periodic(void *arg) {
    (void)arg;
    u32 last_wake = ca_os_tick_ms();
    u32 prev_wake = last_wake;
    while (1) {
        ca_os_sleep_until(&last_wake, 100);
        periodic_count++;
        u32 now = ca_os_tick_ms();
        u32 err = (now - prev_wake > 100u) ? (now - prev_wake - 100u) : (100u -
    (now - prev_wake));
        if (err > drift_max_ms) drift_max_ms = err;
        prev_wake = now;
    }
    return 0;
}

/* === PART C: Timed Primitives === */

static ca_mutex_t      slow_mutex;
static ca_semaphore_t  slow_sem;
static volatile u32 timedlock_ok, timedlock_timeout;
static volatile u32 timedwait_ok, timedwait_timeout;

```

```c
/* Slow: holds the mutex 1s, posts sem, pauses 500ms */
static u32 task_slow(void *arg) {
    (void)arg;
    while (1) {
        ca_mutex_lock(&slow_mutex);
        ca_os_sleep(1000);
        ca_mutex_unlock(&slow_mutex);
        ca_sem_post(&slow_sem);
        ca_os_sleep(500);
    }
    return 0;
}

/* Fast: timedlock 200ms + timedwait 200ms every 50ms */
static u32 task_fast(void *arg) {
    (void)arg;
    while (1) {
        bool got = ca_mutex_timedlock(&slow_mutex, 200);
        if (got) { timedlock_ok++; ca_mutex_unlock(&slow_mutex); }
        else      { timedlock_timeout++; }

        bool sem_ok = ca_sem_timedwait(&slow_sem, 200);
        if (sem_ok) timedwait_ok++;
        else        timedwait_timeout++;

        ca_os_sleep(50);
    }
    return 0;
}

void ca_os_main(void) {
    ca_block_pool_init(&block_pool, pool_buf, sizeof(pool_buf), sizeof(DataBlock
    ));
    ca_mutex_init(&slow_mutex);
    ca_sem_init(&slow_sem, 0, 16);

    ca_os_job("producer", CA_TH_LEVEL2, task_producer, NULL, STACK_SMALL, ...);
    ca_os_job("consumer", CA_TH_LEVEL2, task_consumer, NULL, STACK_SMALL, ...);
    ca_os_job("periodic", CA_TH_LEVEL2, task_periodic, NULL, STACK_SMALL, ...);
    ca_os_job("slow",     CA_TH_LEVEL2, task_slow,     NULL, STACK_SMALL, ...);
    ca_os_job("fast",     CA_TH_LEVEL2, task_fast,     NULL, STACK_SMALL, ...);
    ca_os_job("stats",    CA_TH_LEVEL1, task_stats,    NULL, STACK_SMALL, ...);

    EVAL_CLI( ca_cli_register(&s_user_cli_node); )
    ca_os_run(true);
}
```

43.1.4 Detailed analysis

Block Pool — structure and allocation

```c
ca_block_pool_init(&block_pool, pool_buf, sizeof(pool_buf), sizeof(DataBlock));
```

`ca_block_pool_init()` divides `pool_buf` into `N_BLOCKS = sizeof(pool_buf)/sizeof(DataBlock)` slots and builds an **in-place free list**: the first bytes of each free block contain the pointer to the next block.

```
pool_buf (8    40 byte = 320 byte):


    *next              *next              *next                        NULL

     (free)       (free)       (free)              (free)
```

```
5
6
7   head (pool->free_list)
```

`ca_block_alloc()` detaches the first block from the list (`head = head->next`) — O(1) operation. `ca_block_free()` reinserts the block at the head — O(1). Access is protected by `ca_spinlock_irq_t`, making both operations ISR-safe.

Passing pointers via u32 queue

```
1 u32 ptr_val = (u32)(uintptr_t)blk;
2 ca_queue_static_send(&block_queue.q, &ptr_val, CA_WAIT_FOREVER);
```

The queue is defined as a slot of `sizeof(u32)`. The pointer to the `DataBlock` is truncated to 32 bits — valid on Cortex-M33/M7 where all pointers are 32 bits. The consumer reconstructs the pointer with the reverse cast `(DataBlock *)(uintptr_t)ptr`.

Static buffer vs heap

Approach	Fragmentation	Alloc time	Worst case
`malloc`	Yes	O(N)	Indeterminate
Block Pool	No	O(1)	Deterministic

With 8 blocks and producer every 150ms: capacity $8 \times 150\text{ms} = 1.2$ seconds of buffering before overflow. If the consumer is slow, `alloc_fail` increments but the system does not hang.

`ca_os_sleep_until` — anti-drift

```
1 u32 last_wake = ca_os_tick_ms();   /* record the next expected wake-up */
2 while (1) {
3     ca_os_sleep_until(&last_wake, 100);  /* sleeps until last_wake + 100ms */
4     /* work of the cycle */
5     last_wake += 100;  /* internal to the function */
6 }
```

`ca_os_sleep_until` compares the current tick with the next expected wake-up instant. If the loop body took T ms, the subsequent sleep lasts (100 - T) ms instead of always 100ms. The cumulative drift is reset.

Comparison with `ca_os_sleep(100)`:

```
1 sleep classico:   | body(5ms) | sleep(100ms) | body(5ms) | sleep(100ms) | ...
2                   periodo effettivo = 105ms     deriva +5ms/ciclo
3
4 sleep_until:      | body(5ms) | sleep(95ms)  | body(5ms) | sleep(95ms)  | ...
5                   periodo effettivo = 100ms     deriva = 0
```

The output shows `drift_max=2ms` — residual due to the SysTick tick granularity (100µs for H753) and context switch latency.

Timed primitives — waiting in list with expiration

`ca_mutex_timedlock` and `ca_sem_timedwait` wait **in the same wait list** as `ca_mutex_lock` / `ca_sem_wait`, parked in `SLEEP` until expiration (S2_02 §8.8, S2_03 §9.2):

- whoever releases (`ca_mutex_unlock`, `ca_sem_post`) hands the resource to the first waiter in the list and wakes it up immediately, even from `SLEEP`;

- if no one releases it, the expiration wakes the task from the scheduler refresh, and the function returns `false`;
- with `timeout_ms = 0` they behave like `trylock` / `trywait`.

Until 09/27 they were a **sleep-poll** (`trylock` + `ca_os_sleep(10)` + retry): up to 10 ms delay from the release, and the resource went to whoever retried first instead of whoever had been waiting the longest. The `twait` command below measures this.

Ratio timedlock ok/timeout

`slow` cycle: 1000ms hold + 500ms free = 1500ms total. `fast` cycle: 200ms timedlock + 200ms timedwait + 50ms sleep = ~450ms total.

During the free window (500ms), `fast` can acquire the mutex multiple times: - First acquisition: trylock → OK → unlock → go to timedwait - Second: trylock → OK again (mutex free) - These occur until slow re-acquires

During the hold window (1000ms), every timedlock call does: - wait in list for 200 ms (SLEEP), then timeout

The observed ratio (`ok=66, timeout=44` 3:2) reflects the multiple acquisitions per free window against the frequent timeouts during the hold.

Ratio timedwait ok/timeout

`ca_sem_post` is called by `slow` every ~1500ms. `fast` calls `ca_sem_timedwait(200ms)` every ~450ms: - 1500ms / 450ms 3.3 calls per post → expected ratio 1 ok : 3 timeout - Output: `ok=35, timeout =141` 1:4 — `fast` is not always waiting when `slow` performs `post`: the semaphore accumulates and the next `timedwait` takes it immediately

43.1.5 Expected output

Normal operation

```
=== EX20 STATS ===
  BlockPool: alloc=153 fail=0 consumed=153 free=8/8
  Periodic : count=227 drift_max=2ms
  Timedlock: ok=45 timeout=31
  Timedwait: ok=15 timeout=61
==================
```

`alloc==consumed` and `free=8/8` confirm that no blocks were lost. `drift_max=2ms` with a 100ms period = 2% maximum jitter. `Timedlock ok > 0` confirms that `fast` acquires the mutex during the free windows of `slow`.

`show` Command

```
--- Example 20 ---
  BlockPool  alloc    : 135
  BlockPool  fail     : 0
  BlockPool  consumed : 135
  BlockPool  free     : 8 / 8
  Periodic   count    : 200
  Periodic   drift_max: 2 ms
  Timedlock  ok       : 39
  Timedlock  timeout  : 28
  Timedwait  ok       : 13
  Timedwait  timeout  : 53
```

Profile `top`

```
task 'producer  ' CPU= 0.1% Stack= 4.6% SW=   175 SL=  1381 BL=1
task 'consumer  ' CPU= 0.1% Stack= 4.6% SW=   175 SL=     0 BL=1
task 'periodic  ' CPU= 0.1% Stack= 5.0% SW=   261 SL=   903 BL=1
task 'slow      ' CPU= 0.0% Stack= 3.6% SW=    34 SL=  2037 BL=1
task 'fast      ' CPU= 1.7% Stack= 4.6% SW=  2250 SL=     0 BL=1
```

`fast` has `SW=2250` (many context switches for polling) and `CPU=1.7%` — the cost of sleep-poll every 10ms. `slow` has `SL=2037` (many sleeps: 1000ms + 500ms per cycle). `idle=97.1%` confirms that the system is well-sized.

43.1.6 Created threads

Thread	Name	Priority	Role
idle	`idle`	0	Scheduler idle task
log	`audit`	—	Asynchronous output
CLI	`cli`	—	Command reception
`producer`	`producer`	LEVEL2	Allocates blocks, sends to queue
`consumer`	`consumer`	LEVEL2	Receives blocks, verifies, frees
`periodic`	`periodic`	LEVEL2	Exact 100ms via sleep_until
`slow`	`slow`	LEVEL2	Holds mutex 1s, posts sem
`fast`	`fast`	LEVEL2	Timedlock + timedwait every 450ms
`stats`	`stats`	LEVEL1	Prints statistics every 5s

43.1.7 Required configuration in `os_config.h`

```
#define EN_BLOCK_POOL    1    /* Block Pool allocator */

#define ENABLE_CLI       1
#define ENABLE_STDOUT      1
```

43.1.8 Summary

1. **Block Pool**: O(1) allocator on a static buffer. Free list in-place within free blocks. ISR-safe via spinlock. `alloc_ok == consumed` guarantees no memory leaks.

2. **No heap**: `DataBlock` are never `malloc`-ed. The pool is statically sized with `POOL_N_BLOCKS`. In case of overflow, `alloc_fail` increments and the producer skips the cycle — no crash.

3. **Anti-drift** `ca_os_sleep_until`: automatically compensates for body duration. After N iterations, `last_wake = start + Nperiod` is exact. Essential for ADC sampling tasks, PID control, and timed protocols.

4. **Timed primitives sleep-poll**: `ca_mutex_timedlock` and `ca_sem_timedwait` use the trylock-/sleep(10ms)/retry pattern. No permanent blocking, no implicit ownership transfer. Maximum latency 10ms from resource availability.

5. **Ownership-safe**: the sleep-poll pattern avoids the "ownership transfer + timeout" bug that would occur with direct `ca_os_block()` + `ca_mutex_unlock()`: the task never receives ownership without knowing it.

6. **Producer/consumer pattern**: static queue of `u32` pointers as a channel between producer and consumer. The pool ensures that blocks are not freed while in transit in the queue.

43.1.9 Verification of waits with timeout (`twait`)

Since 09/27, a task waiting for a mutex **with timeout** (`ca_mutex_timedlock()`) registers in the same wait list as those waiting without a timeout, parked in `SLEEP` until expiration, and the signaler wakes it up immediately (S2_02 §8.8). Previously, it polled every 10 ms outside the list: it would wake up up to 10 ms after the signal.

The `twait [sec]` command (default 3 s) measures it with a peer task created at the first call. The wait runs in the command task:

1. **latency** — 300 cycles: the peer holds the mutex and releases it after 1..8 ms randomly; the waiter, with a 20 ms timeout, must obtain it (`lost=0`), not before the signal (`spur=0`), and be running within 2 ms (`lat_max` limit, in ticks);
2. **timeout** — the peer holds the mutex for the entire test; for `sec` seconds, T = 1..20 ms: the return never arrives **before** T (`early=0`), at most 2 ms after (`late=0`), and always with `false` (`got=0`).

One line, closed by `PASS` or `FAIL`. On H753 (tick 200 µs):

```
TW mutex : rounds=300 lat_max=0/10 lost=0 spur=0 | tmo n=291 early=0 late=0 got
   =0 over_max=0/10  PASS
```

On the previous kernel: FAIL, `lat_max=50/10`.

In the bank, it is case `timed_wait` of `test_ex20.cpp` (helper `test/hw_test/inc/caosTimedWait.h`). Manually:

```
caos_shell -e 20 -q -t 40 -u "TW .*(PASS|FAIL)" -g "^TW" "twait 3"
```

43.1.10 Waiter removed while waiting (`twkill`)

Mutex and semaphore **hand over** the resource to the waiter they remove from the list. A task killed while waiting is marked `REMOVE` immediately, but only freed at the next scheduler refresh: a release arriving in between would have handed it the mutex (owner dead forever) or the semaphore unit (lost). Now, the releaser skips waiters that are terminating.

`twkill [sec]` performs 10 cycles per second for the mutex and as many for the semaphore: a victim (alternating with and without timeout) waits, a survivor waits behind it, the victim is removed and the resource is released **immediately**, before the refresh. The survivor must obtain it within 100 ms (`lost=0`) and in the end, no one must possess it anymore (`owner=0`).

```
TK mutex : rounds=30/30 lost=0 owner=0  PASS
TK sem   : rounds=30/30 lost=0 owner=0  PASS
```

The first version of this test revealed a defect that was already present: the mutex remembered the owner by **index**. The survivor received the mutex, then the removal of the victim shifted it in the list, and its `ca_mutex_unlock()` no longer passed the ownership check (safeguard, then HardFault). Now the owner is a handle (S2_03 §9.2).

On the previous kernel: `lost=1` on the first round (the mutex delivered to the victim), then `SETUP_FAILED` (the victim with timeout never entered the list). In the test bench: case `kill_handoff` of `test_ex20.cpp`.

Chapter 44

Multi-Instance UART with Binary Protocol

44.1 Example 21 — Multi-instance UART with binary protocol

44.1.1 Introduction

Up to example 20, the UART has been used exclusively as a debug channel and CLI input (UART3 connected to the ST-Link VCP). This example introduces the **multi-instance** UART driver: multiple serial ports active simultaneously, each with its own reception callback and its own application role.

The practical case is communication between two nodes on the same microcontroller via a **structured binary protocol**: UART7 acts as *master* (sends commands), UART2 as *slave* (receives, processes, and responds). The CLI to monitor traffic in real-time is available on USB CDC (or UART3 ST-Link VCP depending on the configuration).

The chosen binary protocol is minimal but robust: fixed start-of-packet field (SOP), command, payload length, optional payload, and 8-bit checksum. This scheme is directly applicable to real-world communications with sensors, actuators, radio modules, or secondary microcontrollers.

Demonstrated features

Function / Concept	Description
`ca_uart_init(num, baud, cb)`	Multi-instance initialization with RX callback
`ca_uart_put(num, msg)`	Transmission on a specific instance
`ca_uart_rx_cb_t`	Byte reception callback type (ISR-safe)
RX state machine	Byte-by-byte protocol decoding in ISR context
Binary protocol SOP/CMD/LEN/PAYLOAD/CHK	Message encapsulation with integrity check
LED from ISR	GPIO toggle in RX callback for visual debugging
`ENABLE_UART`	Flag to enable the generic UART driver (independent of debug UART)

CLI Commands

Command	Description
`hello`	Manually send the HELLO command (0x10) on UART7
`status`	Manually send the STATUS command (0x20) on UART7
`show`	Master statistics (TX/RX) and slave (processed cmds, uptime)
`wire`	Continuity of jumpers on pure GPIO, verdict `WIRE RESULT : OK|FAIL (n/2)`; must be run first
`bridge`	VCP UART slave repeater: `echo`, `conv`, `baud <n>` (see VCP UART Bridge)
`help`	List of available commands and wiring diagram

44.1.2 Required Hardware

Board: Nucleo-H753ZI (STM32H753ZIT6)

Jumpers (2 wires):

```
PE8  (UART7 TX)                          PD6  (UART2 RX)
PD5  (UART2 TX)                          PE7  (UART7 RX)
```

Signal	MCU Pin	AF	Nucleo Connector
UART7 TX	PE8	7	CN10 pin 18
UART7 RX	PE7	7	CN10 pin 20
UART2 TX	PD5	7	CN9 pin 6
UART2 RX	PD6	7	CN9 pin 4

UART3 (TX: PD8, RX: PD9, AF7) is already wired to the Nucleo's ST-Link VCP and does not require jumpers.

Nucleo-H753ZI LEDs:

LED	Pin	Toggle when
LD1 green	PB0	Byte received on UART7 RX (master RX callback)
LD2 yellow	PE1	Byte received on UART2 RX (slave RX callback)
LD3 red	PB14	Checksum error detected by `rx_feed()`

> **Note on hardware conflicts:** UART4 and UART5 present conflicts on the Nucleo-H753ZI: - `PA1` (UART4 RX, AF8) is `ETH_RMII_REF_CLK`: the LAN8742A PHY generates a **continuous 50 MHz clock** on this line — UART reception is impossible. - `PC12` (UART5 TX, AF8) and `PD2` (UART5 RX, AF8) are `SDMMC1_CK` and `SDMMC1_CMD`. UART7 (PE8/PE7, AF7) and UART2 (PD5/PD6, AF7) are free pins with no conflicts with onboard peripherals.

44.1.3 The binary protocol

Each packet has the structure:

```
1
2      SOP          CMD          LEN           PAYLOAD               CHK
3     1 byte      1 byte       1 byte        0 .. 60 byte         1 byte
4
5   0xAA     command   n. bytes   optional data          all
6                      payload                        i byte
7                                                    precedenti
```

Field	Size	Description
SOP	1 byte	Always `0xAA` — marks the start of the packet
CMD	1 byte	Command identifier (see table below)
LEN	1 byte	Payload length in bytes (0 if absent)
PAYLOAD	0–60 bytes	Parameters or response data
CHK	1 byte	Checksum = mod-256 sum of SOP+CMD+LEN+PAYLOAD

Maximum packet size: 64 bytes (4 header/checksum + 60 payload).
Response: the CMD field of the response has bit 7 set (`CMD | 0x80`).

Implemented commands

Command	CMD	Response	Response CMD	Response payload
HELLO	0x10	0x90	(0x10|0x80)	none
STATUS	0x20	0xA0	(0x20|0x80)	`[cmds_lo, tick_lo, tick_hi, 0x01]` (4 bytes)

44.1.4 Example architecture

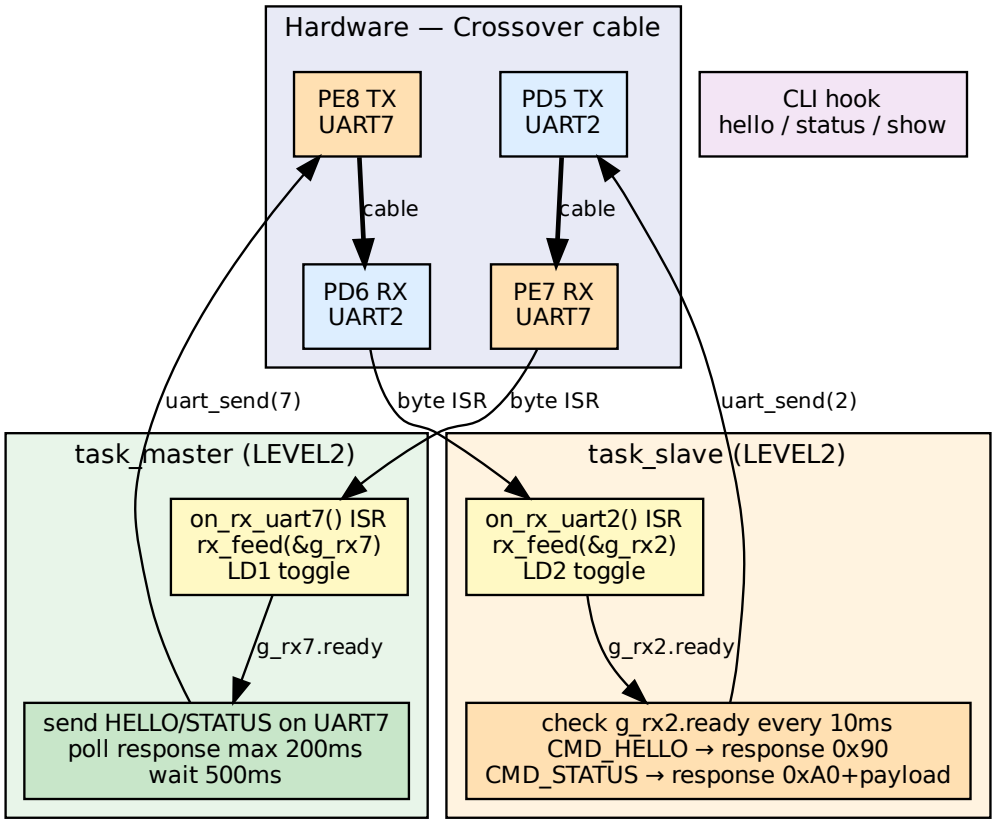


Figure 44.1: Example 21 architecture — UART master/slave

Complete data flow:

```
[task_master]                                   [task_slave]

uart_send(7, HELLO)
          PE8 TX                                                  PD6 RX

                      on_rx_uart2(byte)
                      LD2 toggle (ISR)
                      rx_feed(&g_rx2, byte)
                      g_rx2.ready = true
                                                 elabora CMD_HELLO
                                                 uart_send(2, 0x90)
          PE7 RX                                                  PD5 TX

on_rx_uart7(byte)
LD1 toggle (ISR)
rx_feed(&g_rx7, byte)
g_rx7.ready = true
poll: g_rx_hello++
```

44.1.5 The source code

The file is located in `examples/example_21/ca_os_ex21.c`.

Protocol and constants

```
#define PKT_SOP     0xAAu
#define PKT_MAX     64u
#define PKT_HDR     4u                      /* SOP + CMD + LEN + CHK */
#define PKT_MAX_PL  (PKT_MAX - PKT_HDR)  /* 60 byte payload max */

#define CMD_HELLO   0x10u
#define CMD_STATUS  0x20u
#define RESP_FLAG   0x80u
```

Reception state machine

Reception is byte-by-byte via the ISR callback. A `rx_ctx_t` structure maintains the parser state for each UART:

```
typedef enum { RX_SOP, RX_CMD, RX_LEN, RX_PAYLOAD, RX_CHK } rx_state_t;

typedef struct {
    rx_state_t   state;
    u8           cmd, len, pi, chk;
    u8           payload[PKT_MAX_PL];
    u8           pkt[PKT_MAX];
    u8           pkt_size;
    volatile bool ready;   /* true = complete packet, waiting to be consumed */
    u32          err;      /* checksum error counter */
} rx_ctx_t;
```

The `rx_feed()` function advances the state machine byte by byte. In case of an incorrect checksum, it increments `ctx->err` and toggles LD3 (red):

```
case RX_CHK:
    if (byte == ctx->chk) {
        /* valid packet */
        ctx->pkt[0] = PKT_SOP; ctx->pkt[1] = ctx->cmd;
        ctx->pkt[2] = ctx->len;
        for (u8 i = 0; i < ctx->len; i++) ctx->pkt[3+i] = ctx->payload[i];
        ctx->pkt[3 + ctx->len] = byte;
        ctx->pkt_size = PKT_HDR + ctx->len;
        ctx->ready = true;
    } else {
        ctx->err++;
        ca_gpio_toggle(CA_GPIO_PORT_B, CA_GPIO_PIN_14); /* LD3 red: chk error */
    }
    ctx->state = RX_SOP;
    break;
```

ISR Callback — one per UART

```
static rx_ctx_t g_rx7;   /* slave replies     received on UART7 RX (PE7) */
static rx_ctx_t g_rx2;   /* master commands     received on UART2 RX (PD6) */

static void on_rx_uart7(u32 byte) {
    ca_gpio_toggle(CA_GPIO_PORT_B, CA_GPIO_PIN_0);   /* LD1 green: master RX */
    rx_feed(&g_rx7, (u8)byte);
}
static void on_rx_uart2(u32 byte) {
```

```
    ca_gpio_toggle(CA_GPIO_PORT_E, CA_GPIO_PIN_1);   /* LD2 yellow: slave RX */
    rx_feed(&g_rx2, (u8)byte);
}
```

Registered at init time:

```
ca_uart_init(7, CA_UART_BAUD_115200, on_rx_uart7);
ca_uart_init(2, CA_UART_BAUD_115200, on_rx_uart2);
```

Building and sending a packet

```
static void uart_send(u32 num, u8 cmd, const u8 *pl, u8 plen) {
    string raw_s;
    p_str msg = &raw_s;
    u8   *buf = (u8 *)msg->data;

    buf[0] = PKT_SOP; buf[1] = cmd; buf[2] = plen;
    u8 chk = buf[0] + buf[1] + buf[2];
    for (u8 i = 0; i < plen; i++) { buf[3+i] = pl[i]; chk += pl[i]; }
    buf[3 + plen] = chk;
    msg->size = PKT_HDR + plen;
    ca_uart_put(num, msg);
}
```

The buffer is allocated on the stack (not on the heap): zero fragmentation, safe in task context.

Task master

The master sends each command and waits for the response (max 200 ms, poll every 10 ms) before proceeding with the next command. This avoids the race condition on the single buffer `g_rx7`:

```
static u32 task_master(void *arg) {
    (void)arg;
    ca_os_sleep(200);   /* waits for slave initialization */
    while (1) {
        /* HELLO: send and wait for the reply */
        uart_send(7, CMD_HELLO, NULL, 0);
        g_tx_hello++;
        for (u32 t = 0; t < 20; t++) {
            ca_os_sleep(10);
            if (g_rx7.ready) {
                if (g_rx7.pkt[1] == (CMD_HELLO | RESP_FLAG))
                    g_rx_hello++;
                g_rx7.ready = false;
                break;
            }
        }
        ca_os_sleep(500);

        /* STATUS: send and wait for the reply */
        uart_send(7, CMD_STATUS, NULL, 0);
        g_tx_status++;
        for (u32 t = 0; t < 20; t++) {
            ca_os_sleep(10);
            if (g_rx7.ready) {
                if (g_rx7.pkt[1] == (CMD_STATUS | RESP_FLAG))
                    g_rx_status++;
                g_rx7.ready = false;
                break;
            }
        }
```

```
        ca_os_sleep(500);
    }
    return 0;
}
```

Task slave

```
static u32 task_slave(void *arg) {
    (void)arg;
    while (1) {
        g_slave_ticks++;
        if (g_rx2.ready) {
            u8 cmd = g_rx2.pkt[1]; g_rx2.ready = false; g_slave_cmds++;
            if (cmd == CMD_HELLO) {
                uart_send(2, CMD_HELLO | RESP_FLAG, NULL, 0);
            } else if (cmd == CMD_STATUS) {
                u8 pl[4] = { (u8)(g_slave_cmds & 0xFF),
                             (u8)(g_slave_ticks & 0xFF),
                             (u8)((g_slave_ticks >> 8) & 0xFF),
                             0x01 };
                uart_send(2, CMD_STATUS | RESP_FLAG, pl, 4);
            }
        }
        ca_os_sleep(10);
    }
    return 0;
}
```

44.1.6 Detailed analysis

UART multi-instance: how it works

The `uart.c` driver maintains two static arrays indexed by UART number (1..12):

```
static USART_TypeDef* g_uart_instances[CA_UART_MAX] = {NULL};
static ca_uart_rx_cb_t     hook_on_rx[CA_UART_MAX]       = {NULL};
```

`ca_uart_init(num, baud, cb)` populates both. Each ISR `USARTx_IRQHandler` calls `ca_uart_recv_handler(N)` with the corresponding UART number, which in turn invokes `hook_on_rx[N](rx_data)`. In this way, any number of UARTs can be active simultaneously with independent callbacks.

State machine: why byte by byte

The callback `on_rx_uartN` is called **by the ISR** for every single byte received. It is not possible to block, allocate heap, or access shared structures with locks in this context. The state machine `rx_feed()` is designed exactly for this:

- No locks: a single producer (ISR) and a single consumer (task) for each `rx_ctx_t`
- No dynamic allocation: all buffers are static within the structure
- O(1) operation per byte: the switch case is deterministic

The flag `ready` is `volatile bool`: the ISR sets it to `true`, the task sets it to `false`. On Cortex-M, writing a bool is atomic — the race condition is handled correctly without mutexes.

> **Note**: the structure buffers only one complete packet. The master consumes `g_rx7.ready` within 200 ms after each transmission, well before the next packet arrives (500 ms pause). For higher rates, a circular queue of `rx_ctx_t` is used.

Race condition: single buffer and polling

A naïve implementation of the master task that sends HELLO and STATUS in sequence without waiting for the responses creates a race condition:

```
master sends HELLO       HELLO reply arrives, g_rx7.ready=true
master sends STATUS      STATUS reply arrives, overwrites pkt[], g_rx7.ready
    still true
master reads pkt[1]       sees 0xA0 (STATUS), not 0x90 (HELLO)     g_rx_hello
    stays 0
```

The solution is **blocking polling after each transmission**: the master waits up to 200 ms for the response, consumes it, and resets `ready` before sending the next command. With a 500 ms pause between commands, the timing is generous even considering the jitter of the task_slave (10 ms).

LED: visual debugging of interrupts

The three LEDs on the Nucleo provide immediate feedback without the need for a CLI or debugger:

- **Green LD1** (PB0): blinks on every byte on UART7 RX → confirms that the master is receiving
- **Yellow LD2** (PE1): blinks on every byte on UART2 RX → confirms that the slave is receiving
- **Red LD3** (PB14): turns on only on checksum error → indicator of HW/wiring issues

GPIO toggling is safe in ISR: `ca_gpio_toggle()` writes directly to the BSRR/BRR register without locks.

Checksum and error detection

```
CHK = (SOP + CMD + LEN +    PAYLOAD) mod 256
```

When the checksum does not match, `rx_ctx_t.err` is incremented, LD3 flashes, and the machine returns to `RX_SOP`. The error is visible with the `show` command:

```
RX HELLO resp : 42  (chk_err=0)
Slave cmds rx : 42  (chk_err=0)
```

STATUS response: structured payload

```
u8 pl[4] = {
    (u8)(g_slave_cmds  & 0xFF),         /* commands processed (low byte) */
    (u8)(g_slave_ticks & 0xFF),         /* uptime ticks 10ms (low byte) */
    (u8)((g_slave_ticks >> 8) & 0xFF),  /* uptime ticks 10ms (high byte) */
    0x01                                 /* status: 1=OK */
};
```

The master receives the 4 bytes in `g_rx7.pkt[3..6]` and can reconstruct the values.

44.1.7 Expected output

Startup

```
c.a.O.S >
```

The CLI appears after boot (via USB CDC). The master task waits 200 ms before sending the first HELLO. LEDs LD1 and LD2 immediately start blinking alternately with each byte exchanged.

Command `show` after ~10 seconds

```
c.a.O.S > show

--- Example 21 UART multi-istanza ---
  [MASTER  UART7 TX:PE8]
    TX HELLO      : 5
    TX STATUS     : 5
  [MASTER  UART7 RX:PE7  <-- slave UART2]
    RX HELLO resp : 5  (chk_err=0)
    RX STATUS resp: 5
  [SLAVE   UART2 RX:PD6  <-- master UART7]
    Cmds elaborati: 10  (chk_err=0)
    Slave ticks   : 987  (x10ms)
```

`TX HELLO + TX STATUS = Cmds elaborati` confirms that every command sent has been received and processed by the slave. `chk_err=0` indicates the absence of corruption on the channel.

Simulated checksum error (by disconnecting a wire)

```
c.a.O.S > show

  [MASTER  UART7 RX:PE7  <-- slave UART2]
    RX HELLO resp : 12  (chk_err=3)
```

The 3 errors indicate corrupted packets. The red LD3 flashes on every error. The state machine automatically resynchronizes at the next valid SOP (0xAA).

44.1.8 Required configuration in `os_config.h`

```
#define ENABLE_UART        1    /* generic multi-instance UART driver active */
#define STDOUT_ON_UART     1    /* STDOUT and CLI on the debug UART (VCP) */
#define STDIN_ON_UART      1
#define EN_USB             0    /* console on UART, not on USB CDC */
#define ENABLE_CLI         1    /* CLI for user commands */
#define ENABLE_STDOUT          1    /* log system for ca_stdout_* */
```

44.1.9 Bridge VCP UART

The ST-LINK VCP is the only serial port that the bench PC already has available. The `bridge` command repeats it on the **slave** UART of the example, which on the five mapped Nucleo-144 boards is always the same pair:

Board	UART	TX	RX
H753, H563, U575, F446, F767	USART2 (`USART_B` on Zio)	PD5 = **CN9-6**	PD6 = **CN9-4**

This way, the console of another board (for example the LPUART of ex62, which the VCP does not reach) can be read without a USB-to-serial adapter. The bridge uses the slave UART: the two crossed master/slave jumpers must be **removed** first.

Command	Effect
`bridge echo`	Autotest with **a single CN9-6 CN9-4 jumper**: 4 × 56 bytes are sent from the TX and must return from the RX. Prints `bridge echo: PASS|FAIL (ok/totale, sbagliati, persi)` and the `WIRE RESULT : OK|FAIL (n/1)` line. At the end of the test, the UART returns to the example's slave.
`bridge conv`	Converter: from that moment on, every byte from the VCP is sent out of the UART TX and every byte received returns to the VCP. CLI and stdout are turned off: **you can only exit by resetting**.
`bridge baud <n>`	Bridge UART speed (1200 ... 4000000, default 115200 8N1).
`bridge`	Summary of pins, speed, and rules.

Use as a converter to another board

```
board A (ex21, bridge conv)          board B (es. ex62, console LPUART)
  CN9-6  PD5  TX  ------------------>  RX of B's console
  CN9-4  PD6  RX  <------------------  TX of B's console
  GND             -------------------  GND
```

On the PC, open the VCP of board A at 115200 (or at the speed set with `bridge baud` before `bridge conv`) to communicate with the console of board B.

Implementation

Both directions pass through two 1 KB rings: the RX callbacks (in ISR) fill them, and a task empties them with `ca_uart_put()`, which waits for the UART and therefore must never be called from an ISR. In converter mode, the VCP UART (`UART_DBG_NUM`) is reinitialized with the bridge callback instead of `ca_cli_rx_byte`. This requires the driver to call the registered callback also on the console UART (`uart.c`, `ca_uart_recv_handler`): before 10/13, the console went directly to the CLI and the callback was ignored.

Measurements 10/13 on H753 (jumper CN9-6 CN9-4): `bridge echo` PASS 224/224 at 9600, 115200, and 921600; `bridge conv` in loopback, 3 lines of 105 bytes returned identical. The test bench (`test_homo_ex21`) adds 4 phases: bridge jumper, echo at the three speeds, converter in loopback, jumper removed.

44.1.10 Created Threads

Thread	Name	Priority	Role
idle	`idle`	0	Scheduler idle task
log	`log`	—	Asynchronous output on USB CDC
CLI	`cli`	—	Command reception on USB CDC
USB CDC	`usb`	—	USB Device CDC stack
`master`	`master`	LEVEL2	Sends HELLO/STATUS on UART7, processes responses
`slave`	`slave`	LEVEL2	Processes commands from UART2, responds on UART2

44.1.11 Summary

1. **UART Multi-instance**: `ca_uart_init(num, baud, cb)` registers independent callbacks for each port. The driver handles up to 12 simultaneous UARTs with arrays indexed by number.

2. **Channel separation**: USB CDC for debug/CLI, UART7/UART2 for application protocol. The callback architecture makes the channels completely independent.

3. **Pin selection**: on Nucleo-H753ZI use UART7 (PE8/PE7, AF7) and UART2 (PD5/PD6, AF7) to avoid conflicts with ETH (PA1) and SDMMC (PC12/PD2).

4. **ISR-safe state machine**: protocol parsing occurs byte-by-byte in the callback, without locks, without heap, in O(1) per byte. The `volatile bool ready` flag ensures zero-cost synchronization between ISR and task.

5. **Blocking polling post-send**: the master waits for the response (max 200 ms) before sending the next command. This prevents overwriting the single buffer `g_rx7` when response times are in the 10–20 ms range.

6. **LED from ISR**: GPIO toggle in `on_rx_uart7/2` and in `rx_feed` for checksum error. Immediate visual confirmation of interrupt operation without additional tools.

7. **Robust binary protocol**: SOP as a synchronization marker, LEN for variable payload (0–60 bytes), CHK for error detection. Response with `CMD | 0x80` allows distinguishing commands from responses on the same link.

8. **Scalability**: adding a new command only requires a `case` in `task_slave` and the corresponding constant `CMD_xxx`. The protocol framework (SOP/LEN/CHK) remains unchanged.

Chapter 45

I2C Master/Slave Intra-MCU

45.1 Example 22 — Intra-MCU I2C master/slave

45.1.1 Introduction

Up to example 21, the I2C driver has been used exclusively in **master polling** mode: the task transmits a command, waits for the response, and proceeds. This example introduces **slave IRQ-driven** mode: two I2C peripherals on the same microcontroller communicate with each other, I2C1 as master and I2C2 as slave, physically connected with two wires and pull-up resistors.

The practical case demonstrates the entire chain of a combined I2C transaction (*write_read* with *repeated START*): the master sends a command and reads the response in a single sequence without releasing the bus. The slave receives data via interrupt, prepares the response in the ISR callback with `ca_i2c_slave_set_tx()`, and the master reads it in the same bus cycle.

This architecture is directly applicable to systems with a "controller" MCU (master) and a "sensor" or "coprocessor" MCU (slave) connected via I2C.

Demonstrated features

Function / Concept	Description
`ca_i2c_init()`	Master polling initialization on I2C1
`ca_i2c_slave_init(num, addr, cb)`	Slave IRQ-driven initialization on I2C2
`ca_i2c_write_read()`	Combined transaction: write + repeated START + read
`ca_i2c_slave_set_tx()`	Prepare response buffer for the master (from ISR)
`ca_i2c_slave_rx_cb`	Callback invoked by EV-IRQ upon receipt of a complete packet
Slave state machine IRQ	Handling of ADDR/RXNE/TXIS/STOPF/NACKF flags in `ca_i2c_hw_recv_handler`
Repeated START	No STOP between write and read — the bus remains held by the master

Function / Concept	Description
External 4.7 k pull-ups	GPIO configured `CA_GPIO_NOPULL`, I2C lines pulled to VCC externally

CLI Commands

Command	Description
`ping`	Send CMD_PING (0x01) and print the 4 response bytes
`echo`	Send CMD_ECHO (0x02) with `0x12 0x34`, verify inversion
`show`	Master statistics (TX/RX/errors) and slave (packets received)
`stuck`	Block the slave mid-response with SDA low, then release it with `ca_i2c_bus_recovery()`
`help`	List commands and wiring diagram

45.1.2 Required hardware

Reference board: Nucleo-H753ZI (STM32H753ZIT6)

Jumpers (2 wires) + pull-ups (2 x 4.7 k resistors):

```
PB8  (I2C1 SCL)                    PB10 (I2C2 SCL)
PB9  (I2C1 SDA)                    PB11 (I2C2 SDA)
```

Pins change from board to board: the reference manual states which pins *can* carry I2C, the board user manual states which ones are *reachable* from the connectors. On U575, H563, and F446, the driver defaults end up on an LED or on a pin that does not reach the Zio. Run `wire` **first** on a new board: it is a continuity test on pure GPIOs, independent of I2C.

Board	Master (CN7)	Slave (CN9 / CN10)
Nucleo-H753ZI	I2C1 PB8-2 / PB9-4	I2C2 PB10 CN10-32 / PB11 CN10-34
NUCLEO-U575ZI-Q	I2C4 PF14 CN10-8 / PF15 CN10-12	I2C2 PF1 CN9-19 / PF0 CN9-21
NUCLEO-H563ZI	I2C1 PB8-2 / PB9-4	I2C2 PF1 CN9-19 / PF0 CN9-21
NUCLEO-F446ZE	I2C1 PB8-2 / PB9-4	I2C2 PF1 CN9-19 / PF0 CN9-21

On the **F446**, the driver defaults (`i2c_f4xx.c`: I2C1 on PB6/PB7, I2C2 on PB10/PB11) **are not usable**: PB7 is the blue LED LD2, PB6 is QSPI_BK1/D27, and PB11 does not reach any connector. UM1974 Table 18 routes I2C1 to PB8/PB9 (CN7-2 `I2C_A_SCL`, CN7-4 `I2C_A_SDA`) and leaves the PF1/PF0 pair free on CN9 (`I2C_B_SCL` / `I2C_B_SDA`).

The two lines must be pulled up to 3.3 V with external **4.7 k** resistors (one for SCL, one for SDA). The GPIOs are configured `CA_GPIO_NOPULL` (no internal pull-up): the internal resistors of the Cortex-M (typically 40–50 k) are too weak for I2C at 100 kHz and cause slow edges that generate arbitration-loss errors.

Signal	MCU Pin	AF	Nucleo Connector
I2C1 SCL	PB8	4	CN10 pin 3 (D15)
I2C1 SDA	PB9	4	CN10 pin 5 (D14)
I2C2 SCL	PB10	4	CN10 pin 32 (D36)
I2C2 SDA	PB11	4	CN10 pin 34 (D35)

Pull-up schematic:

```
3.3V                    4.7 k              SCL              PB8 /
    PB10
                      4.7 k                SDA            PB9 / PB11
```

UART3 (TX: PD8, RX: PD9, AF7) is already wired to the ST-Link VCP and provides the CLI (or USB CDC depending on the configuration).

45.1.3 The protocol

Each transaction uses a **fixed 4-byte packet**:

```
Master     Slave:


      CMD           D0           D1           D2
     1 byte       1 byte       1 byte       1 byte


Slave     Master (reply):


  CMD  |0x80        ~D0          ~D1         cnt_lo
     1 byte       1 byte       1 byte       1 byte
```

Field	Direction	Description
CMD	M→S	Command identifier (0x01=PING, 0x02=ECHO)
D0, D1	M→S	Command data
D2	M→S	Reserved (zero)
CMD|0x80	S→M	Response: bit 7 set to distinguish it from the command
~D0, ~D1	S→M	Bitwise inversion of received data (round-trip verification)
cnt_lo	S→M	Low byte of the slave packet counter (uptime)

Implemented commands

Command	CMD	CMD Response	Slave behavior
PING	0x01	0x81	Reflects inverted data, confirms connection
ECHO	0x02	0x82	Inverts D0/D1 — verifies data integrity

The master verification is:

```
bool ok = (rx[0] == (cmd | RESP_FLAG)) &&
          (rx[1] == (u8)(~tx[1]))      &&
          (rx[2] == (u8)(~tx[2]));
```

45.1.4 Example Architecture

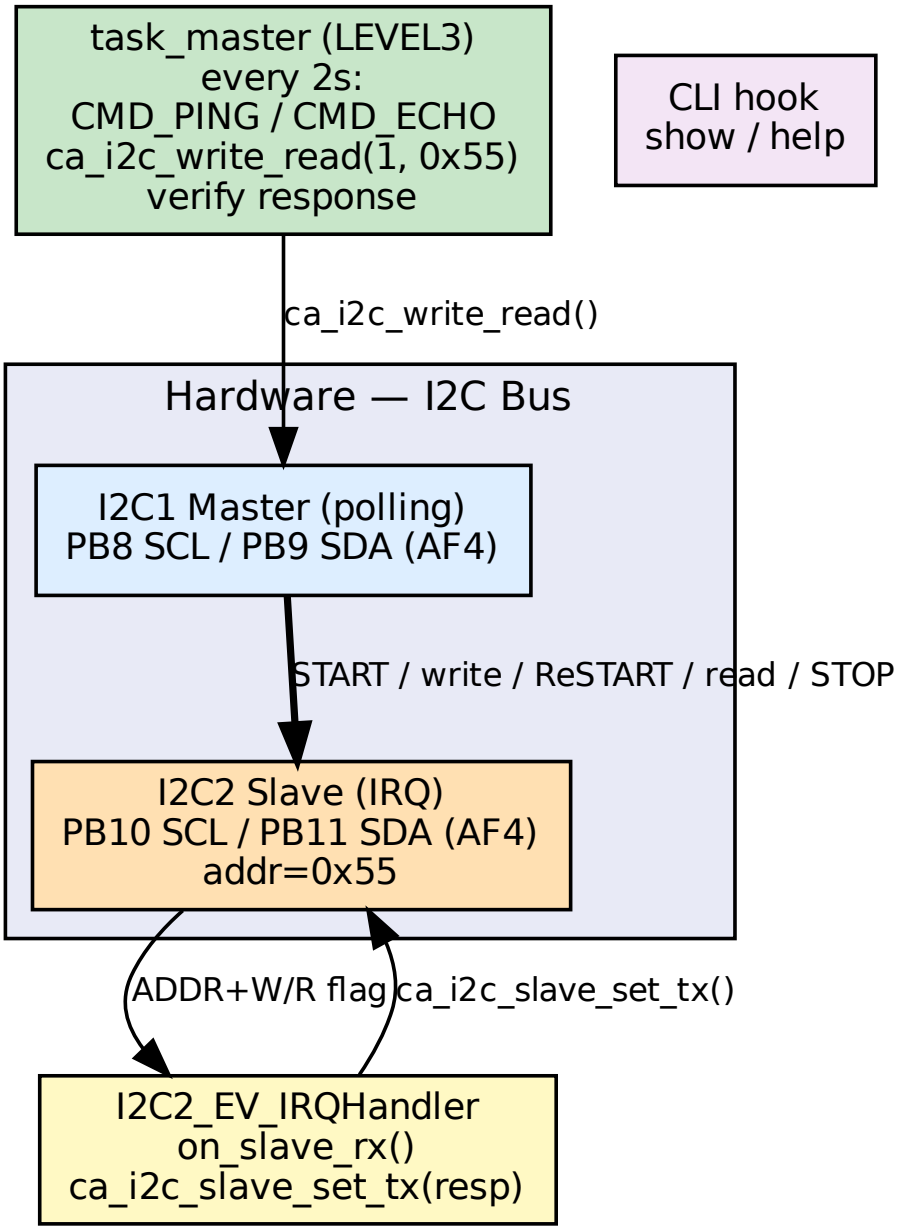


Figure 45.1: Example Architecture 22 — I2C master/slave

Complete data flow (combined transaction):

```
[task_master]                                  [I2C2 EV-IRQ     on_slave_rx]

ca_i2c_write_read(...)

          START
          ADDR+W 0x55
                                                                ADDR flag (DIR=0)
      reset idx
          byte 0: CMD
                                                                RXNE     rx_buf
  [0]
          byte 1: D0
                                                                RXNE     rx_buf
  [1]
          byte 2: D1
                                                                RXNE     rx_buf
  [2]
          byte 3: D2
                                                                RXNE     rx_buf
  [3]

          ReSTART (no STOP)
          ADDR+R 0x55
                                                                ADDR flag (DIR=1)
  :
                                               rx_cb(2, rx_buf, 4)
  on_slave_rx!
                                               ca_i2c_slave_set_tx(resp, 4)
                                               TXDR     resp[0]
          byte 0: resp[0]                                                  TXIS
      TXDR     resp[1]
          byte 1: resp[1]                                                  TXIS
      TXDR     resp[2]
          byte 2: resp[2]                                                  TXIS
      TXDR     resp[3]
          byte 3: resp[3]                                                  NACKF
   (master NACK ultimo byte)
          STOP                                 NACKF     reset tx_idx

verifica rx[]: ok/err
```

45.1.5 The source code

The file is located in `examples/example_22/ca_os_ex22.c`.

Constants and global state

```c
#define SLAVE_ADDR   0x55u
#define PKT_SIZE     4u
#define CMD_PING     0x01u
#define CMD_ECHO     0x02u
#define RESP_FLAG    0x80u

static volatile u32  g_slave_rx_count  = 0;   /* packets received by the slave */
static volatile u32  g_master_tx_count = 0;   /* transactions started by the master */
static volatile u32  g_master_rx_count = 0;   /* valid replies received */
static volatile u32  g_err_count       = 0;   /* risposte errate o timeout */
```

The counters are `volatile` because `g_slave_rx_count` is written by the ISR and read by the CLI task.

Slave callbacks (ISR context)

```
static void on_slave_rx(u8 num, const u8 *buf, u8 len)
{
    if (len < PKT_SIZE) return;

    u8 resp[PKT_SIZE];
    resp[0] = buf[0] | RESP_FLAG;      /* CMD con bit 7 */
    resp[1] = (u8)(~buf[1]);           /* D0 inversion */
    resp[2] = (u8)(~buf[2]);           /* D1 inversion */
    resp[3] = (u8)(g_slave_rx_count & 0xFFu);  /* low counter */

    ca_i2c_slave_set_tx(num, resp, PKT_SIZE);
    g_slave_rx_count++;
}
```

The callback is invoked by the driver when the ISR detects ADDR with DIR=1 (master reads) and `rx_idx > 0` — that is, exactly at the ReSTART of the combined transaction. At that moment, the 4 bytes of the command are already in `buf[]` and the response must be loaded into `tx_buf[]` before the bus controller starts transmitting.

`ca_i2c_slave_set_tx()` copies the buffer into the `g_i2c_slave_ctx[num-1].tx_buf[]` structure and sets `tx_len`. The driver then sends byte by byte on the subsequent TXIS.

Master task

```
static u32 task_master(void *arg)
{
    u8 tx[PKT_SIZE], rx[PKT_SIZE];
    u8 seq = 0;
    ca_os_sleep(500);   /* gives the slave init some time */

    while (1) {
        u8 cmd = ((seq & 1u) == 0u) ? CMD_PING : CMD_ECHO;
        tx[0] = cmd;
        tx[1] = (u8)(seq + 1u);
        tx[2] = (u8)(seq + 2u);
        tx[3] = 0u;

        ca_i2c_write_read(1, SLAVE_ADDR, tx, PKT_SIZE, rx, PKT_SIZE);
        g_master_tx_count++;

        bool ok = (rx[0] == (cmd | RESP_FLAG)) &&
                  (rx[1] == (u8)(~tx[1]))      &&
                  (rx[2] == (u8)(~tx[2]));
        if (ok) g_master_rx_count++;
        else    g_err_count++;

        seq++;
        ca_os_sleep(2000);
    }
    return 0;
}
```

The task alternates between PING and ECHO at each iteration (`seq` even → PING, odd → ECHO). Bytes D0/D1 change at every cycle (`seq+1`, `seq+2`), allowing verification that the slave reflects exactly the received data and not residual values from the previous transaction.

Initialization

```
void ca_os_configure(void)
{
    /* Master: I2C1 polling */
    ca_i2c_init(&(ca_i2c_config_t){ .i2c_num = 1, .speed = CA_I2C_SPEED_STANDARD
    });

    /* Slave: I2C2 IRQ-driven */
    ca_i2c_slave_init(2, SLAVE_ADDR, on_slave_rx);
    ...
}

void ca_os_main(void)
{
    ca_os_job("master", CA_TH_LEVEL3, task_master, NULL, STACK_SMALL, CA_JOB_STD
    , CA_FAULT_RESTART_THREAD);
    EVAL_CLI( ca_cli_register(&s_user_cli_node); )
    ca_os_run(true);
}
```

There is no `task_slave`: the slave is completely managed by I2C2 interrupts. The master task is the only application thread.

45.1.6 Detailed analysis

Combined transaction: write_read with repeated START

The I2C protocol provides two variants for reading data from a slave:

Variant 1 — Separate STOP + START:

```
Master: START     ADDR+W     [write bytes]     STOP     START     ADDR+R     [
    read bytes]     STOP
```

Variant 2 — Repeated START (used in this example):

```
Master: START     ADDR+W     [write bytes]     ReSTART     ADDR+R     [read
    bytes]     STOP
```

With variant 2, the master never releases the bus between write and read: no other master can intervene, and the slave knows for certain that the following read is the response to the previous write. This eliminates the race condition present if `ca_i2c_write` + `ca_os_sleep` + `ca_i2c_read` were used in sequence.

`ca_i2c_write_read()` implements variant 2 for the v2 families (H7xx, H5xx, F7xx):

```
/* write phase: CR2 with NBYTES, START; no AUTOEND */
i2c->CR2 = (addr << 1) | (wn << NBYTES_Pos);
i2c->CR2 |= I2C_CR2_START;
/* waits for TC (Transfer Complete)     the bus stays held */
while (!(i2c->ISR & I2C_ISR_TC)) {}

/* read phase: CR2 con RD_WRN, START (ReSTART), AUTOEND */
i2c->CR2 = (addr << 1) | (rn << NBYTES_Pos) | I2C_CR2_RD_WRN;
i2c->CR2 |= I2C_CR2_START | I2C_CR2_AUTOEND;
```

The `TC` (Transfer Complete) flag is raised by the peripheral when all write bytes have been transmitted **and** the bus is on hold (clock stretching): only then can the master issue the ReSTART.

Slave IRQ state machine (I2C v2)

The H7xx driver handles I2C2 interrupts in `ca_i2c_hw_recv_handler()`. The flags of the ISR register are processed **all in sequence** without premature `return`, with TXIS **before** ADDR:

```
ISR call     leggi ISR register snapshot

           BERR|OVR|ARLO ?      ICR clear, reset idx

           TXIS ?               TXDR     tx_buf[tx_idx++]  (o 0xFF se esaurito)
          FIRST: consume any leftover TXIS BEFORE processing ADDR

           ADDR ?
                 DIR=1 (master legge)      rx_cb(buf, rx_idx)        on_slave_rx
    !
                                          reset idx
                                          TXDR     tx_buf[0]          first reply
    byte
                                          ICR |= ADDRCF
                 DIR=0 (master scrive)     reset idx, ISR|=TXE, ICR|=ADDRCF

           RXNE ?               rx_buf[rx_idx++]     RXDR

           STOPF ?              ICR|=STOPCF, rx_cb se rx_idx>0, reset idx, ISR|=
    TXE

           NACKF ?              ICR|=NACKCF, reset tx_idx, ISR|=TXE
```

Why TXIS before ADDR is critical: when the master issues the ReSTART, the slave peripheral can simultaneously raise TXIS (residue from the previous TX cycle) and ADDR (new address). If ADDR were processed first and loaded `tx_buf[0]` into TXDR, the subsequent handling of TXIS would overwrite TXDR with `tx_buf[1]` — the first byte of the response would be lost. By processing TXIS first, the residual byte is consumed, then ADDR loads the actual `tx_buf[0]`.

This bug was present in the original version of the driver (structure `if/else if` with `return`) and is the cause of the symptom "correct response on the first cycle, then 0xFF as byte 0 on all subsequent cycles".

Pull-up: internal vs external

I2C requires open-drain lines pulled to VCC. The choice of pull-up value influences the edges:

Pull-up	Typical resistance	Rise time at 100 kHz	Notes
Internal Cortex-M GPIO	40–50 k	~2–5 µs	Too slow for 100 kHz (1 bit = 10 µs)
External 4.7 k	4.7 k	~0.3 µs	Correct for SM 100 kHz
External 1 k	1 k	~0.07 µs	Correct for FM 400 kHz, more current

With internal 40 k pull-ups, the RC time constant (R × C_bus 40k × 30pF = 1.2 µs) produces edges so slow that the controller interprets the level as indeterminate and generates `ARLO` (arbitration lost) or `BERR`. The GPIOs are therefore configured `CA_GPIO_NOPULL` in `i2c_h7xx.c`.

ca_i2c_slave_set_tx: slave buffer access from ISR

```
void ca_i2c_slave_set_tx(u8 num, const u8 *buf, u8 len)
{
    ca_i2c_slave_ctx_t *ctx = &g_i2c_slave_ctx[num - 1];
    if (len > sizeof(ctx->tx_buf)) len = sizeof(ctx->tx_buf);
    for (u8 i = 0; i < len; i++) ctx->tx_buf[i] = buf[i];
    ctx->tx_len = len;
    ctx->tx_idx = 0;
}
```

Called from the ISR (inside `on_slave_rx`), it must not use locks: the ISR is the only producer of `tx_buf[]` and the driver (also in ISR) is the only consumer. Writing occurs between ADDR (which has already reset `tx_idx`) and the first TXIS — the bytes are ready in time.

Why only one application task

In this example, the slave has no logic of its own that requires a dedicated thread: all processing (swapping two bytes, incrementing a counter) is O(1) and ISR-safe. A separate slave task would unnecessarily add latency (context switch + `ca_os_sleep`).

The appropriate pattern for a slave with complex processing is: 1. ISR copies data into a volatile buffer and sets a flag 2. Slave task consumes the flag, processes, calls `ca_i2c_slave_set_tx` 3. Upon the next ReSTART from the master, the buffer is already ready

In this example, the master waits 2 seconds between transactions: even a slave task with a 10 ms sleep would have plenty of time.

Bus recovery: slave stuck on SDA

An MCU reset while a slave is transmitting leaves it halfway through a byte with **SDA low**, waiting for clock pulses that will not arrive: the master can no longer generate a START and every transfer fails until the slave loses power. The standard remedy (UM10204 §3.1.16, *bus clear*) is to manually provide up to 9 pulses on SCL until the slave releases SDA, then a STOP.

The driver does this automatically in `ca_i2c_init()` (free on a healthy bus: with SCL and SDA high it does nothing) and exposes `ca_i2c_bus_recovery(num)` to call it after a timeout; from CLI `i2c recover <n>`. Each family uses the pins chosen in init (`ca_i2c_hw_bus_recovery(num, alternate)`), the sequence is unique (`hw/common/i2c_recover.c`).

`stuck` reproduces the real case: it takes the master pins as GPIO, manually performs START, address+R and ACK, then provides clock pulses until the hardware slave holds SDA low on a 0 bit of the response (a `ping` beforehand makes the response known, `0x81`), and releases the pins like an MCU that resets. The driver must return the bus:

```
stuck
  stuck      : ping before=OK, addr ACK=yes, SDA held low by the slave after 1
    data bit(s)=yes
  recovery   : bus idle=yes, ping after=OK
RUN RESULT : OK  (1/1)
```

45.1.7 Expected output

Startup

```
c.a.O.S >
```

The CLI appears after boot. The master task waits 500 ms before the first transaction. There are no dedicated LEDs for this example (no debug GPIO configured).

Command `ping`

```
c.a.O.S > ping
ping: 81 FE AA 00  OK
```

Response: `0x81` = `CMD_PING | 0x80` , `0xFE` = `~0xAA` , `0xAA` = `~0x55` , `0x00` = cnt_lo (first packet).

`echo` command

```
c.a.O.S > echo
echo: 82 ED CB 02  OK
```

Response: `0x82` = `CMD_ECHO | 0x80` , `0xED` = `~0x12` , `0xCB` = `~0x34` . `0x02` = counter (3rd packet if ping + echo have already been executed from the previous CLI).

`show` command after ~20 seconds

```
c.a.O.S > show

--- Example 22 I2C master/slave ---
  master tx   : 10
  master rx   : 10
  errors      : 0
  slave rx    : 10
```

`master tx == master rx == slave rx` and `errors = 0` confirm that every transaction has been completed successfully without data loss.

Wiring error (disconnected wire)

```
c.a.O.S > show

--- Example 22 I2C master/slave ---
  master tx   : 15
  master rx   : 3
  errors      : 12
  slave rx    : 3
```

Errors indicate that the master is not receiving the expected response (`rx[0] != CMD | 0x80`). `i2c_transfer7_v2` includes a counter-based timeout on every busy-wait (50,000 iterations) and checks `NACKF` in each loop: if the bus does not respond within the limit, it sends a STOP, clears the flags, and returns `false`. The master task increments `g_err_count` and continues.

45.1.8 Required configuration in `os_config.h`

```
#define EN_I2C            1    /* I2C driver enabled */
#define STDOUT_ON_UART    1    /* STDOUT and CLI on the debug UART (VCP) */
#define STDIN_ON_UART     1
#define EN_USB            0    /* console on UART, not on USB CDC */
#define ENABLE_CLI        1
#define ENABLE_STDOUT        1
```

In `cpu.cmake`:

```
set(CURR_MCU "H753")
```

45.1.9 Created threads

Thread	Name	Priority	Role
idle	`idle`	0	Scheduler idle task
log	`log`	—	Asynchronous output via USB CDC
CLI	`cli`	—	Command reception via USB CDC
USB CDC	`usb`	—	USB Device CDC stack
`master`	`master`	LEVEL3	I2C transactions every 2 s, response verification

The slave does not have its own thread: it is managed entirely by the I2C2 IRQ.

45.1.10 Summary

1. **IRQ-driven slave**: `ca_i2c_slave_init(num, addr, cb)` configures I2C2 as a slave, enables NVIC EV+ER, and registers the callback. From that moment on, the driver autonomously handles all I2C2 events without task intervention.

2. **Combined write_read transaction**: `ca_i2c_write_read()` uses ReSTART to keep the bus held between write and read. It eliminates the race condition of a separate sequence `write + sleep + read` and ensures that the read retrieves exactly the response to the previous command.

3. **ISR callback**: `on_slave_rx` is called directly by the interrupt handler when ReSTART arrives (ADDR with DIR=1 and rx_idx>0). It must be O(1), without locks, without sleep. `ca_i2c_slave_set_tx()` is safe in this context.

4. **ISR flag order**: the slave handler processes TXIS **before** ADDR in the same ISR call. This ensures that a residual TXIS does not overwrite the first response byte loaded by the ADDR block. Violating this order produced the "byte 0 = 0xFF from the second transaction onwards" bug.

5. **External pull-ups**: GPIO configured `CA_GPIO_NOPULL`. The internal Cortex-M pull-up resistors (40–50 k) are too weak for I2C at 100 kHz and cause ARLO/BERR. 4.7 k pull-ups to 3.3 V are the standard value for SM (100 kHz).

6. **Fixed-size protocol**: 4 bytes in both directions simplify parsing on the slave side (no length state machine) and make timing predictable. For variable payloads, a LEN field is introduced as in example 21.

7. **Scalability**: adding a new command only requires a `case` in `on_slave_rx`. The master/slave framework (write_read + slave_set_tx) remains unchanged regardless of the packet content.

Chapter 46

Internal Flash: Program / Verify / WDT Reset

46.1 Example 23 — Internal Flash: CLI integrated into the driver

46.1.1 Introduction

This example shows the built-in CLI registered by the `flash.c` driver of caOS. There is no additional application logic: after initialization, the firmware is completely driven by the `flash` commands available from the terminal.

The example also enables the IRQ-driven infrastructure (`ca_flash_irq_init`) which prepares the semaphore and the NVIC for the `ca_flash_erase_sector_irq` / `ca_flash_write_irq` APIs, usable by application code.

Demonstrated features

Function / Concept	Description
`ca_flash_init()`	Registers the driver CLI node in the CLI chain
`ca_flash_irq_init()`	Initializes semaphore and enables NVIC FLASH_IRQn
`ca_flash_get_num_sectors()`	Number of physical sectors of the device
`ca_flash_get_sector_addr(s)`	Base address of sector `s`
`ca_flash_get_sector_size(s)`	Size in bytes of sector `s`
`ca_flash_erase_sector_num(s)`	Polling erase of the sector by number
`ca_flash_erase_range(addr, n)`	Polling erase of all sectors in a range
`ca_flash_hw_write()`	Polling write (HW unit per family)
`ca_hexdump()`	Hexadecimal dump to stdout
`EN_FLASH` / `EN_CLI_FLASH`	Compilation guards for the driver and its CLI

Driver CLI commands

Command	Description
`flash help`	List all available flash commands
`flash info`	Number of sectors, total/used/free KB, grouped sector map
`flash info <s>`	Base address, size, and end address of sector `<s>`

Command	Description
`flash dump <addr>`	Hexdump 256 bytes from `<addr>`; saves position for the next call
`flash dump`	Hexdump 256 bytes from the point following the last `dump` (↑ Enter to advance)
`flash erase <addr> <n>`	Erase range `[addr, addr+n)`: fully covered sectors → direct erase; partial sectors → read-modify-erase-write (allocates heap)
`flash erase_s <s>`	Erase of sector number `<s>`
`flash test <s>`	Erase + write pattern `0xCAA55ACA` on the entire sector `<s>` + verify (polling)
`flash test_irq <s>`	Erase IRQ + write pattern `0xCAA55ACA` IRQ (32 B chunk) + verify + dump first lines on sector `<s>`

46.1.2 Required hardware

Board: Nucleo-H753ZI (STM32H753ZIT6) — no external components required.
CLI and stdout via **USB CDC**.

46.1.3 Sector map by family

MCU	Sectors	Size	Total
F103C8	~64 pages	2 KB	128 KB
F407VE	12 (s0-3=16KB, s4=64KB, s5-11=128KB)	mixed	512 KB
F429ZI	12 per bank	mixed	1 MB
F446ZE	8	mixed	512 KB
F746ZG	8 (s0-3=32KB, s4=128KB, s5-7=256KB)	mixed	1 MB
H747/H753	8+8 × 128 KB	128 KB	2 MB
H563	128+128 × 8 KB	8 KB	2 MB

46.1.4 Flash alignment constraints

Family	Write unit	Notes
F1xx	2 bytes (halfword)	PSIZE=0
F4xx	4 bytes (word)	PSIZE=2
F7xx	4 bytes (word)	PSIZE=2, invalidate cache after write
H7xx	32 bytes (256 bit)	dual bank, write in 8-word blocks
H5xx	16 bytes (128 bit)	BKSEL for bank2, FW force-flush

`flash test` writes in 32-byte chunks, compatible with all families.

46.1.5 Example architecture

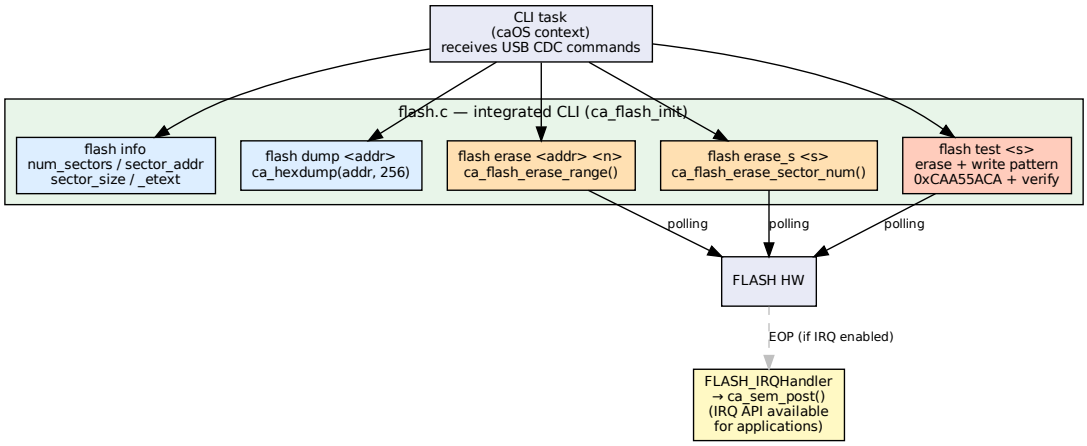


Figure 46.1: Example 23 Architecture — Flash CLI

No dedicated application task: all flash operations are executed in the context of the **CLI task** (regular caOS task, can block on `ca_sem_wait`).

46.1.6 Flash IRQ driver architecture

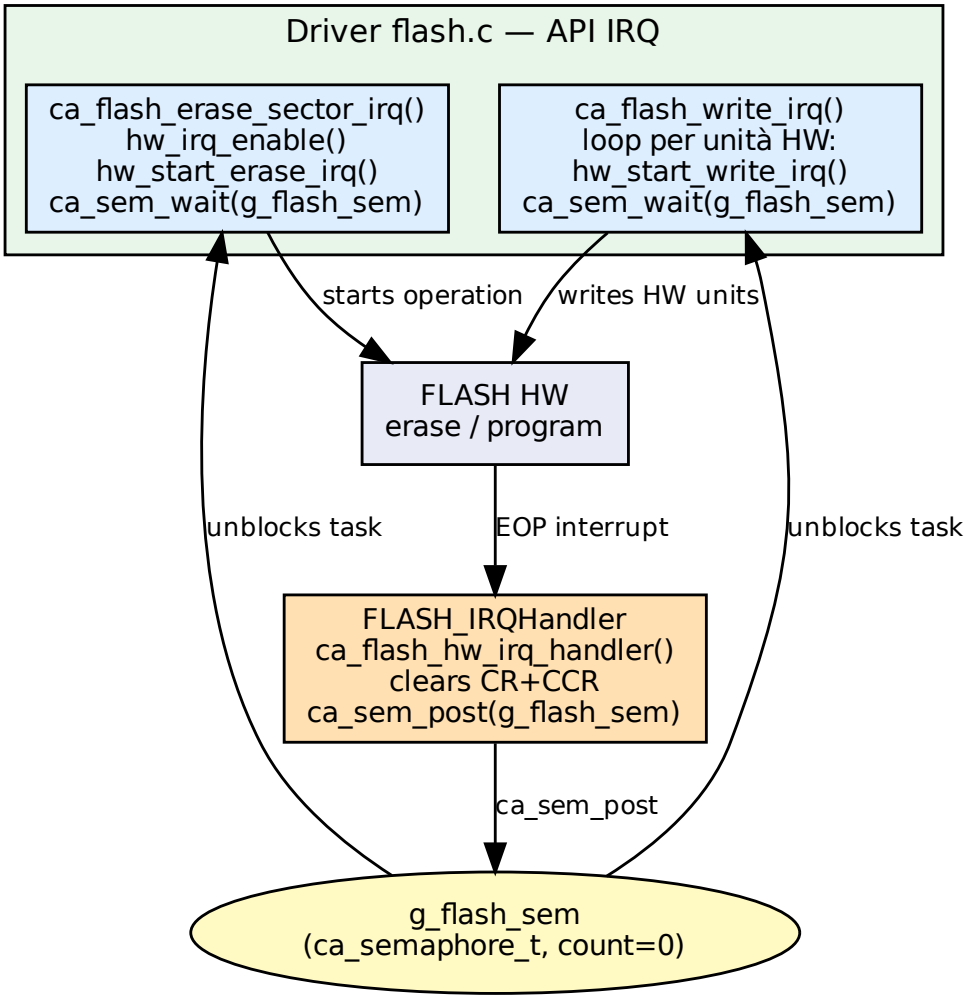


Figure 46.2: Flash IRQ driver architecture

46.1.7 Source code

The file is located in `examples/example_23/ca_os_ex23.c`.

Initialization

```
void ca_os_configure(void)
{
    ca_flash_irq_init();   /* semaphore + NVIC     enables the IRQ-driven API */
    /* stdout / CLI / USB CDC ... */
}

void ca_os_main(void)
```

```
{
    ca_flash_init();   /* registers the driver's CLI node */
    ca_os_run(true);
}
```

`ca_flash_init()` registers a `ca_cli_node_t` node in the CLI chain; from that moment on, all `flash` commands are available without additional hooks.

`ca_flash_irq_init()` is idempotent and must be called before using the `_irq` APIs. The driver CLI internally uses polling APIs; IRQ APIs are available for application code that requires them.

46.1.8 Command analysis

`flash info` / `flash info <s>`

Without an argument, it queries `ca_flash_hw_num_sectors()`, `ca_flash_hw_sector_addr()`, and `ca_flash_hw_sector_size()` to build the map. It uses the `_etext` linker symbol to calculate the flash used by the firmware. Sectors with the same size are grouped:

```
flash: 16 sectors  total=2048KB  used=58KB  free=1989KB
  s0-15: 128KB each @ 0x08000000
```

With the sector number, it resolves the base address, size, and end:

```
flash info 15
s15: 128KB @ 0x081E0000  end=0x081FFFFF
```

`flash dump [<addr>]`

Accepts the address in hexadecimal with or without the `0x` prefix. Calls `ca_hexdump((u8*)addr, 256U)` — the flash is memory-mapped, readable directly via pointer. After each dump, it saves `addr + 256` internally:

```
flash dump 0x081E0000            prima pagina, salva next=0x081E0100
flash dump                       dump 0x081E0100, salva next=0x081E0200
```

Out-of-range address is rejected before touching memory:

```
flash dump 0x80000000
dump: 0x80000000 out of flash range [0x08000000-0x08200000)
```

Typical usage: `flash dump <addr>` the first time, then ↑ **Enter** to scroll.

`flash erase_s <s>`

Call `ca_flash_erase_sector_num(s)`: unlock → `ca_flash_hw_erase_sector` → lock.

```
flash erase_s 15
s15 erased OK
```

`flash erase <addr> <n>`

Calls `ca_flash_erase_range(addr, n)`. For each sector overlapping the range:

- **Completely covered** (`addr` `sa` and `sa+ss` `addr+n`): direct erase, no allocation.
- **Partially covered**: allocates `ss` bytes with `ca_memory_allocate`, copies the entire sector, zeroes out the bytes in the range at `0xFF`, erases, and rewrites. If the heap does not have sufficient memory → `erase: error` without touching the flash.

Prints `wait...` before operating (on H753, a 128 KB sector takes a few seconds).

```
flash erase 0x08100000 0x20000
wait...
erase 0x08100000 +131072 OK
```

`flash test <s>`

Complete sequence on sector `<s>` in **polling** mode: 1. Erase via `ca_flash_erase_sector_num` 2. Write pattern `0xCAA55ACA` repeated across the entire sector (32-byte chunks) 3. Byte-by-byte verification via direct flash pointer 4. Print PASS or number of errors

```
flash test 15
flash test s15 @ 0x081E0000  128KB
wait...
PASS: s15 pattern OK
```

The command is useful for verifying the integrity of the erase/write cycle on a non-critical sector before using it for persistent data.

`flash test_irq <s>`

Complete sequence on sector `<s>` in **IRQ-driven** mode — exercises the `ca_flash_erase_sector_irq` / `ca_flash_write_irq` path with task→ISR semaphore:

1. Erase via `ca_flash_erase_sector_irq(addr)` — the task blocks on `ca_sem_wait` until the ISR signals EOP
2. Write pattern `0xCAA55ACA` in 32-byte chunks via `ca_flash_write_irq(addr+off, wbuf, 32)` — loop for the entire sector, one chunk per iteration
3. Byte-by-byte verification via direct flash pointer
4. Hexdump of the first lines of the sector
5. Print PASS or number of errors

Requires that `ca_flash_irq_init()` has already been called (`g_flash_irq_ready` flag).

```
flash test_irq 13
flash test_irq s13 @ 0x081A0000  128KB
erase (IRQ)...
write (IRQ)...
PASS: s13 pattern OK (IRQ)
dump 0x081A0000:
0x081A0000  ca a5 5a ca ca a5 5a ca |..Z...Z...Z...Z.|
```

46.1.9 Expected output

Startup

```
c.a.O.S >
```

No automatic operation. The flash is not touched at boot.

Full session on H753

```
c.a.O.S > flash info
flash: 16 sectors  total=2048KB  used=58KB  free=1989KB
  s0-15: 128KB each @ 0x08000000

c.a.O.S > flash info 15
s15: 128KB @ 0x081E0000  end=0x081FFFFF

c.a.O.S > flash test 15
flash test s15 @ 0x081E0000  128KB
wait...
PASS: s15 pattern OK

c.a.O.S > flash dump 0x081E0000
```

```
081E0000: CA A5 5A CA CA A5 5A CA CA A5 5A CA CA A5 5A CA  ..Z...Z...Z...Z.
081E0010: CA A5 5A CA CA A5 5A CA CA A5 5A CA CA A5 5A CA  ..Z...Z...Z...Z.
...

c.a.O.S > flash dump
081E0100: CA A5 5A CA CA A5 5A CA CA A5 5A CA CA A5 5A CA  ..Z...Z...Z...Z.
...

c.a.O.S > flash erase_s 15
s15 erased OK

c.a.O.S > flash test_irq 13
flash test_irq s13 @ 0x081A0000  128KB
erase (IRQ)...
write (IRQ)...
PASS: s13 pattern OK (IRQ)
dump 0x081A0000:
0x081A0000  ca a5 5a ca ca a5 5a ca |..Z...Z...Z...Z.|

c.a.O.S > flash dump 0x080000000
dump: 0x80000000 out of flash range [0x08000000-0x08200000)
```

46.1.10 Required configuration in `os_config.h`

```
#define EN_FLASH          1
#define EN_CLI_FLASH      1
#define ENABLE_STDOUT     1
#define STDOUT_ON_UART    1    /* STDOUT and CLI on the debug UART (VCP) */
#define STDIN_ON_UART     1
#define EN_USB            0    /* console on UART, not on USB CDC */
#define ENABLE_CLI        1
```

46.1.11 Created threads

Thread	Role
idle	Scheduler idle task
log	Asynchronous output via USB CDC
CLI	Command reception — also executes flash operations
USB CDC	USB Device CDC stack

46.1.12 Summary

1. **Zero-config CLI**: `ca_flash_init()` is enough to make all `flash` commands available. No application hooks required.

2. **Polling operations in CLI**: driver commands use polling APIs (unlock → hw_erase/write → lock). Simple, predictable, no NVIC requirements.

3. **IRQ-driven available**: `ca_flash_irq_init()` enables the semaphore and the NVIC for `ca_flash_erase_sector_irq` / `ca_flash_write_irq` APIs, usable by application tasks that do not want to occupy the CPU while waiting. CLI commands use **polling** APIs — IRQ APIs are for application code that calls them directly (not from the CLI).

   Bug fix note (2026-05-22): `ca_flash_hw_start_erase_irq` on H7/F4/F7/H5 used direct assignment `FLASH->CR = ...` which cleared EOPIE as soon as it was set

by `ca_flash_hw_irq_enable`. Fix: `(CR & IE_MASK) | operazione`. Corrected families: H7xx, F4xx, F7xx, H5xx. F1xx was already correct (`|=`).

4. **Portability**: the same source code works on all families. The write unit and sector geometry are encapsulated in `flash_<family>.c` files.

5. **Quick polling test**: `flash test <s>` validates the erase/write polling cycle on a sector. The fixed pattern `0xCAA55ACA` covers all bits and makes failures easily detectable.

6. **IRQ-driven test**: `flash test_irq <s>` exercises the entire IRQ path — erase + write + verify — blocking the CLI task on the semaphore and unblocking it from the ISR on EOP. Requires `ca_flash_irq_init()` in `ca_os_configure()`. HW verified on NUCLEO-H753ZI (s13 → PASS).

Chapter 47

Timer: Periodic + Counter

47.1 Example 24 — Timer IRQ: 10 kHz generator + external countdown counter

47.1.1 Introduction

This example shows how to use two STM32 timers in a master-slave configuration to generate a precise periodic signal and count it via an external input (ETR - External Trigger).

The system demonstrates: - **TIM3** as a 10 kHz signal generator (100 µs period) - **TIM2** in counter-irq mode with external clock on ETR (PA0) - Physical connection between the two timers via a single wire - Pulse counting and interrupt generation every N edges

Demonstrated features

Function / Concept	Description
`ca_timer_init_periodic()`	Initializes TIM3 in 100 µs periodic mode
`ca_timer_init_counter_irq()`	Initializes TIM2 in counter mode with IRQ on preset
`ca_gpio_configure()`	Configures GPIO for push-pull output and AF (Alternate Function)
`CA_GPIO_MODE_AF`	Alternate Function mode for TIM2_ETR
`CA_GPIO_PULLUP`	Pull-up on ETR input for signal stability
`EN_TIMER` guard	The entire example is conditioned on the timer driver being enabled

CLI Commands

Application commands:

Command	Description
`info`	Expected wiring (PD0 TIM3 out → PA0 TIM2_ETR), generator and counter

Command	Description
`wire`	Continuity of jumpers on pure GPIO, verdict `WIRE RESULT : OK|FAIL (n/1)`; must be launched first

The example also registers the timer driver CLI via `ca_timer_init()`.

Command	Description
`timer info`	Shows all acquired timers (TIM2, TIM3), mode and current CNT
`timer info <n>`	Single timer detail: mode and CNT
`timer stop <n>`	Stops TIMn
`timer start <n>`	Restarts TIMn
`timer free <n>`	Deinitializes and releases TIMn
`timer cnt <n>`	For encoder/counter: reads the accumulated value; for periodic: raw CNT

STDOUT log output every 5 seconds with statistics.

47.1.2 Required Hardware

Board: Nucleo-H753ZI (STM32H753ZIT6)

Wiring

A single jumper connects the generator to the counter:

Pin	Signal	Description
PD0 (CN9 pin 25)	TIM3 output	10 kHz signal generated
PA0 (CN10 pin 29)	TIM2_ETR	External counter input

```
PD0 (generatore)                    PA0 (ETR counter)
```

Status LEDs

LED	Pin	Color	Function
LD2	PE1	Yellow	Blink 20 Hz — confirms TIM3 active
LD1	PB0	Green	Blink 1 Hz — confirms ETR counting

47.1.3 Timer configuration

TIM3 — 10 kHz generator

Parameter	Value	Notes
Frequency	10 kHz	Period 100 µs
Output	PD0	Toggle in ISR
ISR	`cb_10khz()`	Toggle PD0 + LD2 + increment counter

TIM2 — External counter

Parameter	Value	Notes
Clock	ETR (PA0)	Rising edges
Preset	5000	IRQ every 5000 pulses
ETR Frequency	5 kHz	TIM3_Hz / 2 (toggle = 2 edges/cycle)
IRQ Period	1 s	5000 pulses / 5000 Hz = 1 s
ISR	`cb_1sec()`	Toggle LD1 + increment counter

47.1.4 System architecture

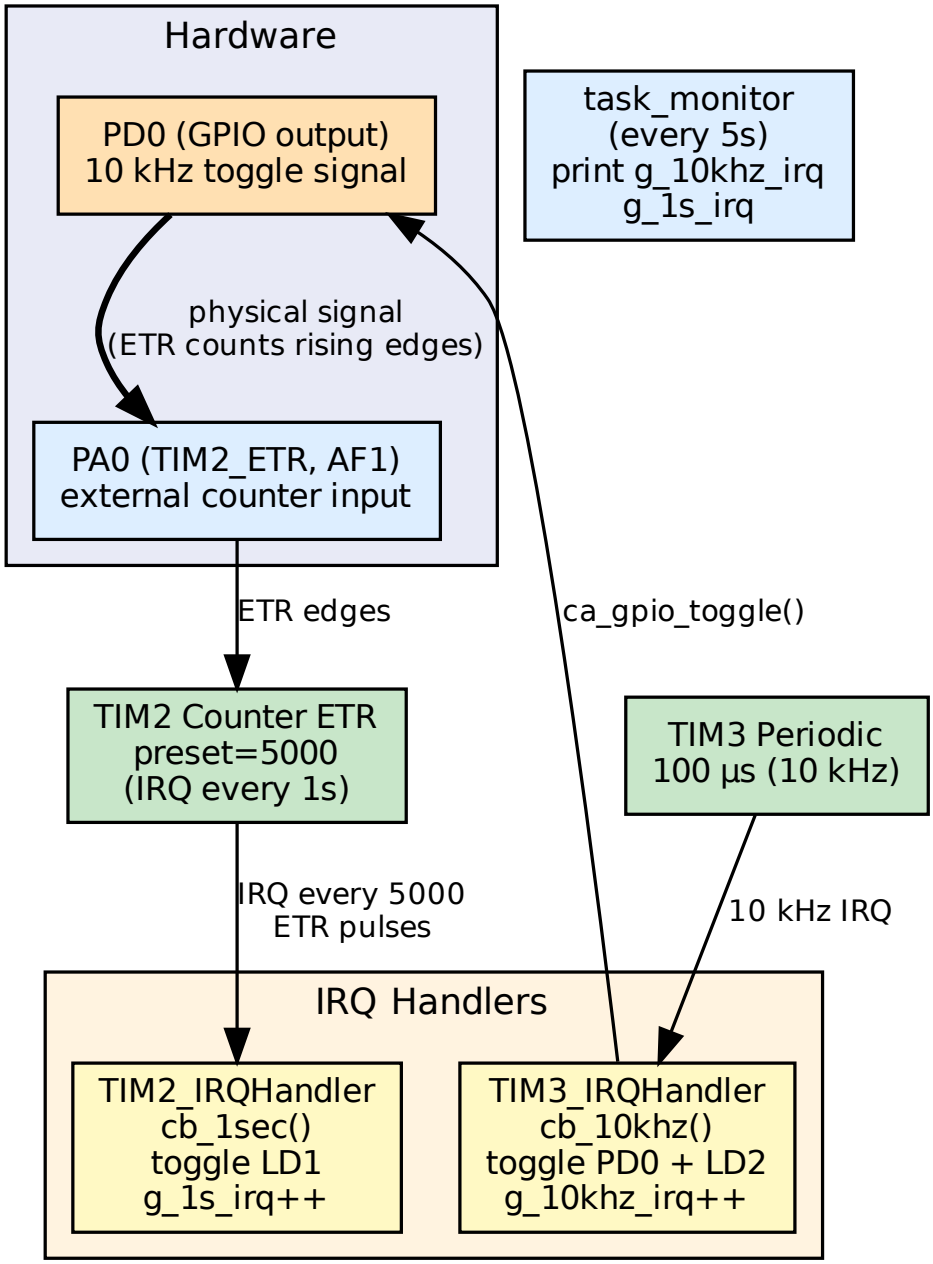


Figure 47.1: Architecture Example 24 — Timer counter ETR

47.1.5 Source code

The file is located in `examples/example_24/ca_os_ex24.c`.

Hardware configuration

```
#define TIM_GEN       3U          /* TIM3     generator */
#define TIM_CNT       2U          /* TIM2     external countdown counter */

#define PERIOD_GEN    100U        /* 100  s  = 10 kHz */
#define PRESET_CNT    5000U       /* 5000 pulses = 1 s @ 5 kHz */

/* signal GPIO output */
#define GPIO_OUT_PORT  CA_GPIO_PORT_D
#define GPIO_OUT_PIN   CA_GPIO_PIN_0

/* GPIO ETR input TIM2: PA0, AF1 = TIM2_ETR (H7xx) */
#define GPIO_ETR_PORT  CA_GPIO_PORT_A
#define GPIO_ETR_PIN   CA_GPIO_PIN_0
#define GPIO_ETR_AF    1U
```

Frequency relationship

```
/* ETR counts only RISING edges of PD0: pulses_ETR = TIM3_Hz / 2
 * DEBUG:      20 Hz TIM3     10 edges/s, preset=10      IRQ every 1 s
 * PRODUCTION: 10 kHz TIM3     5000 edges/s, preset=5000     IRQ every 1 s  */
```

Explanation: TIM3 toggles PD0 every interrupt. A full cycle (HIGH→LOW→HIGH) requires 2 interrupts, so the signal frequency on PD0 is `TIM3_Hz / 2`. ETR counts rising edges, therefore: - TIM3 at 10 kHz → 5000 edges/s on ETR - Preset 5000 → IRQ every 1 second

ISR Callback

```
static volatile u32 g_10khz_irq = 0;
static volatile u32 g_1s_irq    = 0;

static void cb_10khz(void)
{
    ca_gpio_toggle(GPIO_OUT_PORT, GPIO_OUT_PIN);
    ca_gpio_toggle(GPIO_LED2_PORT, GPIO_LED2_PIN);
    g_10khz_irq++;
}

static void cb_1sec(void)
{
    g_1s_irq++;
    ca_gpio_toggle(GPIO_LED_PORT, GPIO_LED_PIN);
}
```

Note: - `g_10khz_irq` counts TIM3 interrupts (actual 10 kHz) - `g_1s_irq` counts TIM2 interrupts (every 5000 ETR pulses) - Both variables are `volatile` because they are accessible from ISRs

Task monitor

```
static u32 task_monitor(void *arg)
{
    (void)arg;

```

```
    while (1) {
        ca_os_sleep(5000);
        u32 hz  = g_10khz_irq;
        u32 sec = g_1s_irq;
        EVAL_STDOUT( ca_stdout_printf_n("EX24: os_ms=%u  10khz_irq=%u  1s_irq=%u
    ",
                                        (unsigned)ca_os_tick_ms(), (unsigned)hz, (
    unsigned)sec); )
    }
    return 0;
}
```

The task reads the global counters and prints the statistics every 5 seconds.

Initialization in `ca_os_configure`

```
void ca_os_configure(void)
{
    /* PD0: fast push-pull output     10 kHz signal */
    ca_gpio_init_port(GPIO_OUT_PORT);
    ca_gpio_configure(GPIO_OUT_PORT, GPIO_OUT_PIN,
                      CA_GPIO_MODE_OUTPUT, CA_GPIO_OTYPE_PP,
                      CA_GPIO_SPEED_VERY_HIGH, CA_GPIO_NOPULL, 0U);

    /* PA0: AF1 (TIM2_ETR)     external clock input TIM2 */
    ca_gpio_init_port(GPIO_ETR_PORT);
    ca_gpio_configure(GPIO_ETR_PORT, GPIO_ETR_PIN,
                      CA_GPIO_MODE_AF, CA_GPIO_OTYPE_PP,
                      CA_GPIO_SPEED_HIGH, CA_GPIO_PULLUP, GPIO_ETR_AF);

    /* LED 1 blinks at 1 Hz from TIM2, LED 2 at 20 Hz from TIM3 (visual
     * confirmation that the generator runs). Pins come from objmap.h. */
    ca_led_init();

    /* TIM3: 20 Hz generator (debug) */
    ca_timer_init_periodic(TIM_GEN, PERIOD_GEN, cb_10khz);
    ca_timer_start(TIM_GEN);

    /* TIM2: countdown 10 ETR rising edges @ 20Hz/2=10Hz = 1 s (debug) */
    ca_timer_init_counter_irq(TIM_CNT, PRESET_CNT, cb_1sec);
    ca_timer_start(TIM_CNT);

    EVAL_STDOUT(
        ca_stdout_init();
        ca_stdout_set_sender(STDOUT);
        ca_stdout_create_thread();
    )
    EVAL_CLI(
        ca_cli_init(2048);
        ca_cli_set_sender(STDOUT);
        ca_cli_create_thread();
    )
    EVAL_USB_CDC_STD(ca_usb_dev_mng_cdc_cli();)
}
```

47.1.6 Detailed analysis

Timer modes used

TIM3 — Periodic mode TIM3 is configured in periodic mode with an interrupt every 100 µs. The ISR executes: 1. Toggle of PD0 (generates effective 5 kHz square wave) 2. Toggle

of LD2 (visual confirmation 20 Hz — every 2 interrupts = 1 visible toggle) 3. Global counter increment

TIM2 — External clock mode 1 TIM2 uses the ETR (External Trigger) input as a clock source. The counter: 1. Counts rising edges on PA0 2. When CTR (counter register) reaches the preset, it generates an interrupt 3. The ISR toggles LD1 and increments the 1s counter

Why PA0 requires pull-up

The ETR input must be stable when there are no transitions. The pull-up: - Keeps the signal HIGH at rest - Prevents false triggers due to noise - Ensures sharp edges when PD0 switches LOW

Debug vs Production

The code includes comments for two configurations:

Mode	TIM3 Hz	ETR edges/s	Preset	IRQ/s
DEBUG	20 Hz	10	10	1 Hz
PRODUCTION	10 kHz	5000	5000	1 Hz

The production configuration (10 kHz) is more accurate and demonstrates the maximum timer resolution.

Exact timing

```
TIM3 ISR (every 100  s ):
         t=0  s :   PD0 = HIGH
         t=100  s : PD0 = LOW
         t=200  s : PD0 = HIGH      1 fronte salita ETR
         t=300  s : PD0 = LOW
         t=400  s : PD0 = HIGH      1 fronte salita ETR

Periodo segnale PD0: 200  s  (5 kHz)
Fronti salita/s: 5000
Preset TIM2: 5000
IRQ TIM2: every exact 1 s
```

47.1.7 Expected output

Startup

```
c.a.O.S >
```

No automatic output at startup. Timers start immediately.

Periodic log (every 5 s)

```
EX24: os_ms=5000  10khz_irq=50000  1s_irq=5
EX24: os_ms=10000  10khz_irq=100000  1s_irq=10
EX24: os_ms=15000  10khz_irq=150000  1s_irq=15
```

Verification: - `10khz_irq` should increment by ~50000 every 5 s (10 kHz × 5 s) - `1s_irq` should increment by 5 every 5 s (1 Hz × 5 s)

Oscilloscope/logic analyzer signals

Channel	Pin	Frequency	Waveform
CH1	PD0	5 kHz	50% duty square wave
CH2	PA0	5 kHz	Identical to PD0
CH3	LD2	20 Hz	Square wave (ISR toggle)
CH4	LD1	1 Hz	Square wave (ETR IRQ)

47.1.8 Required configuration in `os_config.h`

```
#define EN_TIMER          1
#define EN_CLI_TIMER      1
#define ENABLE_STDOUT     1
#define STDOUT_ON_UART    1    /* STDOUT and CLI on the debug UART (VCP) */
#define STDIN_ON_UART     1
#define EN_USB            0    /* console on UART, not on USB CDC */
#define ENABLE_CLI        1
```

47.1.9 Created threads

Thread	Priority	Role
idle	0	Scheduler idle task
monitor	3	Log statistics every 5 s
log	—	Asynchronous output via USB CDC
CLI	—	Command reception
USB CDC	—	USB Device CDC stack

47.1.10 Possible extensions

1. ETR precision measurement

Add a CLI command to measure the ETR count drift compared to the internal timer:

```
static void cmd_measure(void)
{
    u32 start_10k = g_10khz_irq;
    u32 start_1s  = g_1s_irq;
    ca_os_sleep(10000);  /* 10 secondi */
    u32 delta_10k = g_10khz_irq - start_10k;
    u32 delta_1s  = g_1s_irq - start_1s;

    ca_stdout_printf_n("10s measure: 10k_irq=%u (expected 100000), 1s_irq=%u (
    expected 10)",
                       delta_10k, delta_1s);
}
```

2. Variable frequency generator

Add a CLI command to change the TIM3 frequency dynamically:

```
static void cmd_freq(u32 freq_hz)
{
    u32 period_us = 1000000 / freq_hz;
    ca_timer_stop(TIM_GEN);
    ca_timer_init_periodic(TIM_GEN, period_us, cb_10khz);
```

```
    ca_timer_start(TIM_GEN);
    ca_stdout_printf_n("TIM3 set to %u Hz (period %u us)", freq_hz, period_us);
}
```

3. Variable preset

Allow changing the TIM2 preset to obtain IRQs at different intervals:

```
static void cmd_preset(u32 preset)
{
    ca_timer_stop(TIM_CNT);
    ca_timer_init_counter_irq(TIM_CNT, preset, cb_1sec);
    ca_timer_start(TIM_CNT);
    ca_stdout_printf_n("TIM2 preset: %u (IRQ every %u ms)",
                       preset, preset / 5);  /* @ 5 kHz ETR */
}
```

47.1.11 Summary

1. **Dual-timer configuration**: TIM3 generates a signal, TIM2 counts it via ETR — demonstrates the combined use of independent timers.
2. **External trigger mode**: TIM2 uses PA0 as an external clock, showing how to count external events without CPU intervention.
3. **Hardware connection**: A single wire (PD0→PA0) connects the two timers — minimal configuration for precision testing.
4. **Visual feedback**: Two LEDs show the activity of both timers (LD2 at 20 Hz, LD1 at 1 Hz).
5. **Statistics logging**: STDOUT output every 5 seconds allows verifying the correctness of the count and the stability of the system.
6. **Scalability**: The same architecture can be used to:
   - Count pulses from rotary encoders
   - Measure the frequency of external signals
   - Generate precise baud rates for UART
   - Create multiple timebases derived from a single timer

Chapter 48

Timer: Periodic + Encoder Counter

48.1 Example 25 — Quadrature Signal Generator + Timer Encoder

48.1.1 Introduction

This example demonstrates how to generate quadrature signals (CW/CCW) using a timer and how to count them using a second timer in encoder mode.

Hardware (Nucleo-H753ZI, MB1364)

- **TIM3**: 100 µs (10 kHz) generator
  - ISR: every 4 pulses generates a quadrature cycle on PB8/PB9 (2.5 kHz)
- **PB8** (GPIO output): Signal A (quadrature) - CN7 pin 2
- **PB9** (GPIO output): Signal B (quadrature, 90° phase shifted) - CN7 pin 4
- **TIM4**: encoder mode
  - **PB6** (TIM4_CH1, AF2): Channel A input - CN10 pin 13
  - **PB7** (TIM4_CH2, AF2): Channel B input - CN10 pin 15

Direction

- **CW (ClockWise)**: A leads B → counter increments
- **CCW (CounterClockWise)**: B leads A → counter decrements

Wiring (2 jumpers)

```
PB8 (CN7 pin 2)                  PB6 (CN10 pin 13)  [Segnale A]
PB9 (CN7 pin 4)                  PB7 (CN10 pin 15)  [Segnale B]
```

Application CLI Commands

Command	Description
`setcw`	Sets CW (clockwise) direction
`setccw`	Sets CCW (counterclockwise) direction
`show`	Shows encoder position and statistics
`wire`	Bridge continuity on pure GPIO, verdict `WIRE RESULT : OK|FAIL (n/2)`; must be run first

CLI driver timer commands (`ca_timer_init`)

Command	Description
`timer info`	Shows TIM3 (periodic) and TIM4 (encoder) with current CNT
`timer info <n>`	Single timer details
`timer stop <n>`	Stops the timer
`timer start <n>`	Restarts the timer
`timer cnt 4`	Reads accumulated encoder position TIM4

48.1.2 System Architecture

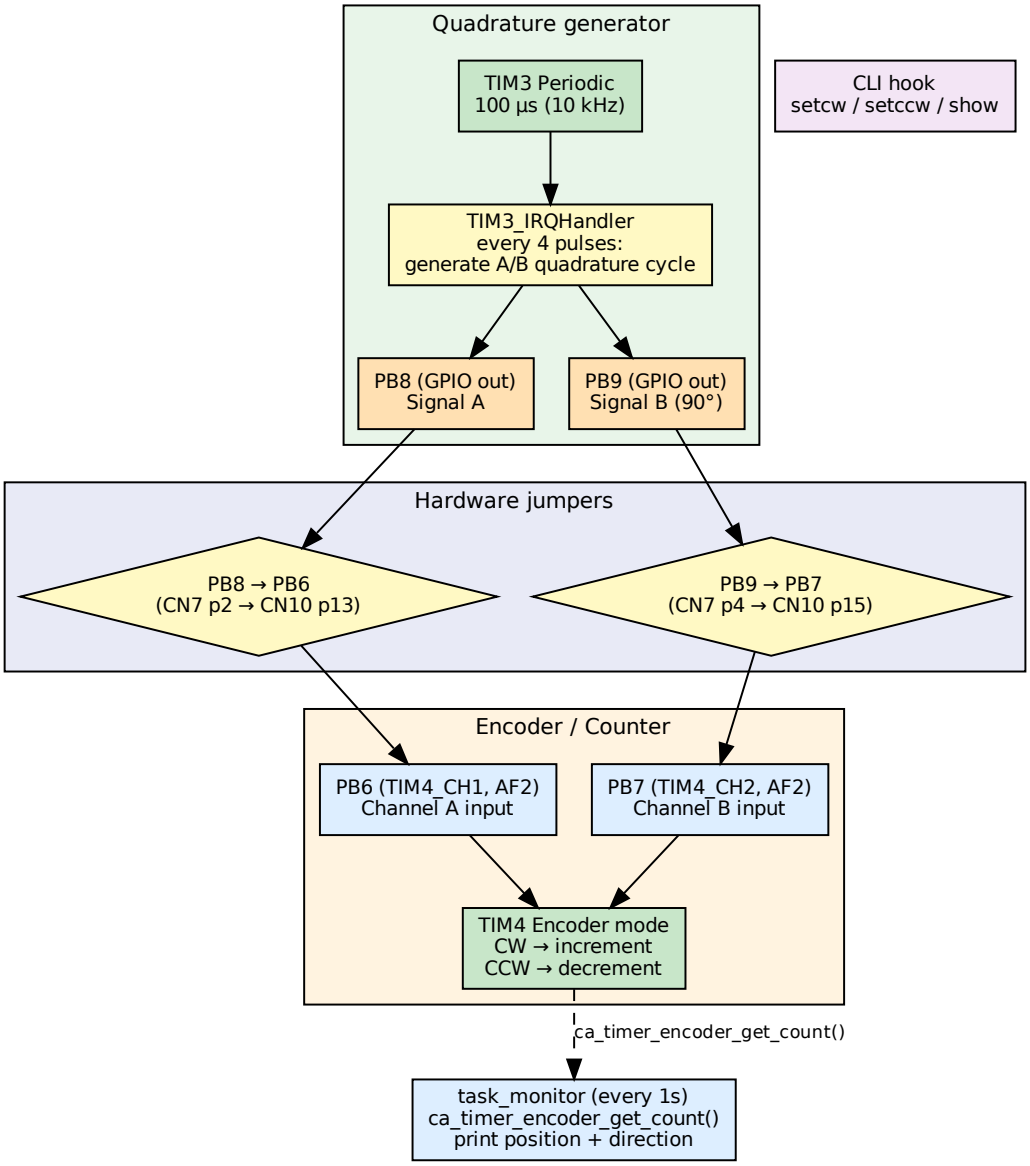


Figure 48.1: Example 25 — Quadrature Encoder Architecture

48.1.3 Source Code

The file is located in `examples/example_25/ca_os_ex25.c`:

```
1  // Proprietary     All rights reserved
2  // Copyright (c) 2026 Angelo Coppi
3
4  /*
5   * Example 25     Timer IRQ: quadrature generator + Timer Encoder
6   *
7   * Hardware (Nucleo-H753ZI, MB1364):
```

```
 *
 *   TIM3  periodic 100  s  (10 kHz)
 *         ISR: every 4 pulses generates one quadrature cycle on PB8/PB9
 *
 *   PB8   (GPIO output)     Signal A (quadrature) - CN7 pin 2
 *   PB9   (GPIO output)     Signal B (quadrature, 90   shifted) - CN7 pin 4
 *
 *   TIM4  Encoder mode     Counts pulses on PB6/PB7
 *   PB6   (TIM4_CH1, AF2)     Channel A input (jumper from PB8) - CN10 pin 13
 *   PB7   (TIM4_CH2, AF2)     Channel B input (jumper from PB9) - CN10 pin 15
 *
 *   Direction:
 *     - CW  (ClockWise)          A leads B     counter increments
 *     - CCW (CounterClockWise)     B leads A     counter decrements
 *
 * Wiring (2 jumpers):
 *   PB8 (CN7 pin 2)                  PB6 (CN10 pin 13)  [Signal A]
 *   PB9 (CN7 pin 4)                  PB7 (CN10 pin 15)  [Signal B]
 */

#include "caos.h"

#if EN_TIMER

#define TIM_GEN            3U           /* TIM3     10 kHz generator */
#define TIM_QUAD_A_PORT    CA_GPIO_PORT_B  /* PB8     GPIO output (segnale A) */
#define TIM_QUAD_A_PIN     CA_GPIO_PIN_8
#define TIM_QUAD_B_PORT    CA_GPIO_PORT_B  /* PB9     GPIO output (signal B) */
#define TIM_QUAD_B_PIN     CA_GPIO_PIN_9

#define TIM_ENC            4U           /* TIM4     Encoder mode */
#define TIM_ENC_A_PORT     CA_GPIO_PORT_B  /* PB6     TIM4_CH1 (input A) */
#define TIM_ENC_A_PIN      CA_GPIO_PIN_6
#define TIM_ENC_A_AF       2U           /* AF2 = TIM4_CH1 (H7xx) */
#define TIM_ENC_B_PORT     CA_GPIO_PORT_B  /* PB7     TIM4_CH2 (input B) */
#define TIM_ENC_B_PIN      CA_GPIO_PIN_7
#define TIM_ENC_B_AF       2U           /* AF2 = TIM4_CH2 (H7xx) */

#define GPIO_LED2_PORT     CA_GPIO_PORT_E
#define GPIO_LED2_PIN      CA_GPIO_PIN_1

#define TIM3_PERIOD_US     100U
#define QUAD_DIVISOR       4U

static volatile u32 g_tim3_irq_count = 0;
static volatile u32 g_quad_cycles = 0;
static volatile u8  g_req_cw = 1;
static volatile i64 g_encoder_pos = 0;
static i64 encoder_pos_accum = 0;
static volatile u32 g_cw_cycles = 0;
static volatile u32 g_ccw_cycles = 0;

static void cb_tim3_generator(void)
{
    g_tim3_irq_count++;
    ca_gpio_toggle(GPIO_LED2_PORT, GPIO_LED2_PIN);

    static u8 quad_state = 0;

    if (g_tim3_irq_count % QUAD_DIVISOR == 0) {
        g_quad_cycles++;

        if (g_req_cw) {
```

```
71              /* CW: 00     10     11     01     00 */
72              switch (quad_state) {
73                  case 0: ca_gpio_write(TIM_QUAD_A_PORT, TIM_QUAD_A_PIN, 0);
74                          ca_gpio_write(TIM_QUAD_B_PORT, TIM_QUAD_B_PIN, 0); break
    ;
75                  case 1: ca_gpio_write(TIM_QUAD_A_PORT, TIM_QUAD_A_PIN, 1);
76                          ca_gpio_write(TIM_QUAD_B_PORT, TIM_QUAD_B_PIN, 0); break
    ;
77                  case 2: ca_gpio_write(TIM_QUAD_A_PORT, TIM_QUAD_A_PIN, 1);
78                          ca_gpio_write(TIM_QUAD_B_PORT, TIM_QUAD_B_PIN, 1); break
    ;
79                  case 3: ca_gpio_write(TIM_QUAD_A_PORT, TIM_QUAD_A_PIN, 0);
80                          ca_gpio_write(TIM_QUAD_B_PORT, TIM_QUAD_B_PIN, 1); break
    ;
81              }
82              g_cw_cycles++;
83          } else {
84              /* CCW: 00     01     11     10     00 */
85              switch (quad_state) {
86                  case 0: ca_gpio_write(TIM_QUAD_A_PORT, TIM_QUAD_A_PIN, 0);
87                          ca_gpio_write(TIM_QUAD_B_PORT, TIM_QUAD_B_PIN, 0); break
    ;
88                  case 1: ca_gpio_write(TIM_QUAD_A_PORT, TIM_QUAD_A_PIN, 0);
89                          ca_gpio_write(TIM_QUAD_B_PORT, TIM_QUAD_B_PIN, 1); break
    ;
90                  case 2: ca_gpio_write(TIM_QUAD_A_PORT, TIM_QUAD_A_PIN, 1);
91                          ca_gpio_write(TIM_QUAD_B_PORT, TIM_QUAD_B_PIN, 1); break
    ;
92                  case 3: ca_gpio_write(TIM_QUAD_A_PORT, TIM_QUAD_A_PIN, 1);
93                          ca_gpio_write(TIM_QUAD_B_PORT, TIM_QUAD_B_PIN, 0); break
    ;
94              }
95              g_ccw_cycles++;
96          }
97          quad_state = (quad_state + 1) % 4;
98      }
99  }
100
101 static u32 task_monitor(void *arg)
102 {
103     (void)arg;
104     while (1) {
105         ca_os_sleep(5000);
106         ca_timer_encoder_read(TIM_ENC);
107         g_encoder_pos = encoder_pos_accum;
108         EVAL_STDOUT(
109             ca_stdout_printf_n("EX25: pos=%d  dir=%s  tim3_irq=%u  quad_cycles=%
    u  cw=%u  ccw=%u",
110                                (long)g_encoder_pos, g_req_cw ? "CW" : "CCW",
111                                (unsigned)g_tim3_irq_count, (unsigned)
    g_quad_cycles,
112                                (unsigned)g_cw_cycles, (unsigned)g_ccw_cycles);
113         )
114     }
115     return 0;
116 }
117
118 static bool user_cli_handler(c_str *pcmd)
119 {
120     c_str cmd = *pcmd;
121     const char *p = ca_skip_sp(cmd->data);
122
123     if (ca_strncmp(p, "help", 4) == 0) {
```

```
124         /* One line listing the bare commands, like the other examples that
125          * register theirs bare (ex21 "hello | status | show | wire", ex22,
126          * ex24). Four descriptive lines said the same thing in a shape of
127          * their own; ex64 keeps its "s64: ..." prefix because there the
128          * prefix is real - it dispatches on strncmp(p, "s64", 3). */
129         CLI_PUT_N("setcw | setccw | show | wire");
130         return false;
131     }
132     if (ca_strncmp(p, "wire", 4) == 0) {
133         /* Continuity test of the two jumpers, on plain GPIO: no timer, no
134          * encoder, so a FAIL here means the wire, not the configuration.
135          * TIM3 must be STOPPED first - its ISR drives these very pins 10000
136          * times a second and would overwrite every level before the read.
137          * The encoder timer goes too: it owns the input pins as AF. */
138         ca_timer_stop(TIM_GEN);
139         ca_timer_stop(TIM_ENC);
140 
141         static const struct { ca_gpio_port_t op; ca_gpio_pin_t opin;
142                               ca_gpio_port_t ip; ca_gpio_pin_t ipin; const char
    *name; }
143         L[] = {
144             { TIM_QUAD_A_PORT, TIM_QUAD_A_PIN, TIM_ENC_A_PORT, TIM_ENC_A_PIN,
145               WIRE_A_TXT },
146             { TIM_QUAD_B_PORT, TIM_QUAD_B_PIN, TIM_ENC_B_PORT, TIM_ENC_B_PIN,
147               WIRE_B_TXT },
148         };
149         u32 good = 0U;
150         for (u32 i = 0; i < 2U; i++) {
151             ca_gpio_configure(L[i].op, L[i].opin, CA_GPIO_MODE_OUTPUT,
152                               CA_GPIO_OTYPE_PP, CA_GPIO_SPEED_LOW,
    CA_GPIO_NOPULL, 0);
153             ca_gpio_configure(L[i].ip, L[i].ipin, CA_GPIO_MODE_INPUT,
154                               CA_GPIO_OTYPE_PP, CA_GPIO_SPEED_LOW,
    CA_GPIO_NOPULL, 0);
155             ca_gpio_write(L[i].op, L[i].opin, 0);
156             for (volatile u32 d = 0; d < 20000U; d++) { }
157             u8 lo = ca_gpio_read(L[i].ip, L[i].ipin);
158             ca_gpio_write(L[i].op, L[i].opin, 1);
159             for (volatile u32 d = 0; d < 20000U; d++) { }
160             u8 hi = ca_gpio_read(L[i].ip, L[i].ipin);
161             CLI_PRINTF_N("  %s : drive0->read%u  drive1->read%u  %s",
162                          L[i].name, lo, hi,
163                          (lo == 0 && hi == 1) ? "OK (wire present)"
164                                               : "FAIL (wire missing or swapped)"
    );
165             if (lo == 0 && hi == 1) good++;
166         }
167         /* One line the bench can read, identical in every example that has a
168          * `wire` command - see the note in ex64. */
169         CLI_PRINTF_N("  WIRE RESULT : %s  (%u/2)",
170                      (good == 2U) ? "OK" : "FAIL", good);
171         /* The pins are plain GPIO now, not timer AF: reboot to put the
172          * generator and the encoder back the way ca_os_configure() set them. */
173         CLI_PUT_N("  rebooting in 2.5 s to restore the timers...");
174         ca_os_sleep(2500);
175         ca_os_reset();
176         return true;
177     }
178     if (ca_strncmp(p, "setcw", 5) == 0 && (p[5] == '\0' || p[5] == ' ')) {
179         g_req_cw = 1;
180         CLI_PUT_N("direction: CW");
181         return true;
182     }
```

```
183     if (ca_strncmp(p, "setccw", 6) == 0 && (p[6] == '\0' || p[6] == ' ')) {
184         g_req_cw = 0;
185         CLI_PUT_N("direction: CCW");
186         return true;
187     }
188     if (ca_strncmp(p, "show", 4) == 0 && (p[4] == '\0' || p[4] == ' ')) {
189         ca_timer_encoder_read(TIM_ENC);
190         i32 pos      = (i32)encoder_pos_accum;
191         u32 tim3_cnt = g_tim3_irq_count;
192         u32 quad_cnt = g_quad_cycles;
193         u8  dir      = g_req_cw;
194         u32 cw_cnt   = g_cw_cycles;
195         u32 ccw_cnt  = g_ccw_cycles;
196         i32 expected = dir ? (i32)quad_cnt : -(i32)quad_cnt;
197 
198         CLI_PUT_N("=== QUADRATURE ENCODER STATUS ===");
199         CLI_PUT_N("  wiring    : " WIRE_A_TXT);
200         CLI_PUT_N("              " WIRE_B_TXT);
201         CLI_PRINTF_N("  pos       : %d", pos);
202         CLI_PRINTF_N("  dir       : %s", dir ? "CW" : "CCW");
203         CLI_PRINTF_N("  tim3_irq  : %u", tim3_cnt);
204         CLI_PRINTF_N("  quad_cyc  : %u", quad_cnt);
205         CLI_PRINTF_N("  cw_cyc    : %u", cw_cnt);
206         CLI_PRINTF_N("  ccw_cyc   : %u", ccw_cnt);
207         CLI_PRINTF_N("  expected  : %d", expected);
208         return true;
209     }
210     return false;
211 }
212 
213 static ca_cli_node_t s_user_cli_node = { user_cli_handler, NULL };
214 
215 void ca_os_configure(void)
216 {
217     /* ========================================================================
218      * 1. Initialize GPIO for quadrature signal (output)
219      * ========================================================================
    */
220 
221     /* PB8: GPIO output     Signal A - CN7 pin 2 */
222     ca_gpio_init_port(TIM_QUAD_A_PORT);
223     ca_gpio_configure(TIM_QUAD_A_PORT, TIM_QUAD_A_PIN,
224                       CA_GPIO_MODE_OUTPUT, CA_GPIO_OTYPE_PP,
225                       CA_GPIO_SPEED_VERY_HIGH, CA_GPIO_NOPULL, 0U);
226     ca_gpio_write(TIM_QUAD_A_PORT, TIM_QUAD_A_PIN, 0);
227 
228     /* PB9: GPIO output     Signal B - CN7 pin 4 */
229     ca_gpio_init_port(TIM_QUAD_B_PORT);
230     ca_gpio_configure(TIM_QUAD_B_PORT, TIM_QUAD_B_PIN,
231                       CA_GPIO_MODE_OUTPUT, CA_GPIO_OTYPE_PP,
232                       CA_GPIO_SPEED_VERY_HIGH, CA_GPIO_NOPULL, 0U);
233     ca_gpio_write(TIM_QUAD_B_PORT, TIM_QUAD_B_PIN, 0);
234 
235     /* ========================================================================
236      * 2. Initialize GPIO for encoder input (TIM4 CH1/CH2)
237      * ========================================================================
    */
238 
239     /* PB6: AF2 (TIM4_CH1)     Encoder channel A input - CN10 pin 14 */
240     ca_gpio_init_port(TIM_ENC_A_PORT);
241     ca_gpio_configure(TIM_ENC_A_PORT, TIM_ENC_A_PIN,
242                       CA_GPIO_MODE_AF, CA_GPIO_OTYPE_PP,
243                       CA_GPIO_SPEED_HIGH, CA_GPIO_PULLUP, TIM_ENC_A_AF);
```

```
244
245     /* PB7: AF2 (TIM4_CH2)     Encoder channel B input - CN10 pin 16 */
246     ca_gpio_init_port(TIM_ENC_B_PORT);
247     ca_gpio_configure(TIM_ENC_B_PORT, TIM_ENC_B_PIN,
248                       CA_GPIO_MODE_AF, CA_GPIO_OTYPE_PP,
249                       CA_GPIO_SPEED_HIGH, CA_GPIO_PULLUP, TIM_ENC_B_AF);
250
251     /* ========================================================================
252      * 3. Initialize yellow LED LD2 (visual confirmation of TIM3)
253      * ========================================================================
    */
254
255     ca_led_init();
256     ca_led_off(2U);
257
258     /* ========================================================================
259      * 4. Initialize TIM3: 10 kHz generator (100  s )
260      * ========================================================================
    */
261
262     ca_timer_init_periodic(TIM_GEN, TIM3_PERIOD_US, cb_tim3_generator);
263     ca_timer_start(TIM_GEN);
264
265     /* ========================================================================
266      * 5. Initialize TIM8: Encoder mode (counting on both edges)
267      * ========================================================================
    */
268
269     ca_timer_init_encoder(TIM_ENC, &encoder_pos_accum);
270     ca_timer_start(TIM_ENC);
271
272     /* ========================================================================
273      * 6. Initialize system modules (sensors, log, CLI, USB)
274      * ========================================================================
    */
275
276
277     EVAL_STDOUT(
278         ca_stdout_init();
279         ca_stdout_set_sender(STDOUT);
280         ca_stdout_create_thread();
281     )
282
283     EVAL_CLI(
284         ca_cli_init(2048);
285         ca_cli_set_sender(STDOUT);
286         ca_cli_create_thread();
287     )
288
289     EVAL_USB_CDC_STD(ca_usb_dev_mng_cdc_cli();)
290
291 }
292
293 void ca_os_main(void)
294 {
295     ca_timer_init();   /* registers the timer driver CLI */
296     ca_os_job("monitor", CA_TH_LEVEL3, task_monitor, NULL, STACK_SMALL,
    CA_JOB_STD, CA_FAULT_RESTART_THREAD);
297     EVAL_CLI( ca_cli_register(&s_user_cli_node); )
298     ca_os_run(true);
299 }
300
301 #else
```

```
302
303 void ca_os_configure(void) {}
304 void ca_os_main(void) { ca_os_run(true); }
305
306 #endif /* EN_TIMER */
```

48.1.4 Detailed Analysis

Quadrature Signal Generation

The TIM3 generates an interrupt every 100 µs (10 kHz). In the ISR, every 4 interrupts a complete quadrature cycle is generated:

```
1 #define TIM3_PERIOD_US      100U      /* 10 kHz */
2 #define QUAD_DIVISOR        4U        /* 4 interrupts = 1 quadrature cycle */
3                                     /* Result: 2.5 kHz */
```

PB8 and PB9 are configured as pure **GPIO output**: the ISR drives them directly with `ca_gpio_write()`. No timer alternate function is used.

Quadrature sequences

CW (ClockWise):

```
1 State  A  B
2   0    0  0
3   1    1  0      A leads
4   2    1  1
5   3    0  1
```

CCW (CounterClockWise):

```
1 State  A  B
2   0    0  0
3   1    0  1      B leads
4   2    1  1
5   3    1  0
```

TIM4 Encoder mode

TIM4 is configured in encoder mode, which automatically counts the edges on channels CH1 (PB6) and CH2 (PB7):

```
1 ca_timer_init_encoder(TIM_ENC, &encoder_pos_accum);
2 ca_timer_start(TIM_ENC);
```

The timer increments/decrements the counter based on the relative phase between the two signals.

Note: The STM32 encoder mode requires the main channels CH1 and CH2 — the complementary channels (CH1N, CH2N) cannot be used as encoder inputs.

Variable `g_req_cw`

```
1 static volatile u8 g_req_cw = 1;  /* Default CW */
```

- `g_req_cw = 1`: generate CW sequence
- `g_req_cw = 0`: generate CCW sequence

Modifiable via CLI commands `setcw` and `setccw`.

48.1.5 Expected output

At startup

```
=== Example 25: Quadrature Encoder ===
TIM3: 10 kHz generator (100  s )
Quadrature: 2.5 kHz on PB8/PB9
Encoder: TIM4 on PB6/PB7
Connect: PB8(CN7-2)->PB6(CN10-13), PB9(CN7-4)->PB7(CN10-15)
Type 'help' for commands
Default direction: CW
```

Command `show`

```
c.a.O.S > show
=== QUADRATURE ENCODER STATUS ===
  pos       : 5000
  dir       : CW
  tim3_irq  : 50000
  quad_cyc  : 12500
  cw_cyc    : 12500
  ccw_cyc   : 0
  expected  : 12500
```

Command `timer info`

```
c.a.O.S > timer info
timer: in_use=0x0000000A
  TIM3  periodic   cnt=0x00001F4A
  TIM4  encoder    cnt=0x00001388
```

Periodic log (every 5 seconds)

```
EX25: pos=125  dir=CW  tim3_irq=50000  quad_cycles=12500  cw=12500  ccw=0
```

48.1.6 Required Configuration in `os_config.h`

```
#define EN_TIMER          1
#define EN_CLI_TIMER      1
#define ENABLE_STDOUT     1
#define STDOUT_ON_UART    1    /* STDOUT and CLI on the debug UART (VCP) */
#define STDIN_ON_UART     1
#define EN_USB            0    /* console on UART, not on USB CDC */
#define ENABLE_CLI        1
```

48.1.7 Important Notes

1. Frequencies

Signal	Frequency	Period
TIM3 ISR	10 kHz	100 µs
Quadrature	2.5 kHz	400 µs
LED toggle	5 kHz	200 µs

2. Encoder Resolution

TIM4 in encoder mode x4 counts: - 4 edges per quadrature cycle - Effective resolution: 4× the quadrature frequency

3. Required Jumpers

2 jumpers are required: 1. PB8 (CN7 pin 2) → PB6 (CN10 pin 13) [Signal A] 2. PB9 (CN7 pin 4) → PB7 (CN10 pin 15) [Signal B]

4. Pins and Alternate Functions

Pin	Function	AF	Connector
PB8	GPIO output (signal A)	—	CN7 pin 2
PB9	GPIO output (signal B)	—	CN7 pin 4
PB6	TIM4_CH1 (encoder A)	AF2	CN10 pin 13
PB7	TIM4_CH2 (encoder B)	AF2	CN10 pin 15

5. GPIO Speed

- **Quadrature output**: `CA_GPIO_MODE_OUTPUT` + `CA_GPIO_SPEED_VERY_HIGH` for sharp edges
- **Encoder input**: `CA_GPIO_MODE_AF` + `CA_GPIO_SPEED_HIGH` with pull-up

48.1.8 Summary

1. **TIM3** generates interrupt at 10 kHz for precise quadrature timing.
2. **Quadrature signal** generated on PB8/PB9 (GPIO output) with CW or CCW sequence.
3. **TIM4** in encoder mode automatically counts edges on CH1/CH2 (PB6/PB7).
4. **Application CLI** (`setcw`, `setccw`, `show`, `wire`) in chain of responsibility. **CLI driver** (`timer info/stop/start/cnt`) registered by `ca_timer_init()`.
5. **2 jumpers** connect quadrature output to encoder input.
6. **Default direction**: CW (changeable via CLI).
7. **Statistics**: CW/CCW cycle count, TIM3 interrupts, encoder position.

Chapter 49

Cryptography: CRC32, HASH, AES, RNG and Key Derivation

49.1 Example 26 — Cryptography: CRC32, HASH, AES, RNG and Key Derivation

49.1.1 Introduction

This example demonstrates the use of STM32 cryptographic modules:

1. **CRC32** — Checksum calculation for data integrity verification
2. **HASH** — SHA-256 calculation (one-shot and concurrent contextual)
3. **AES** — AES-256 encryption/decryption (ECB, CTR, GCM)
4. **RNG** — Hardware random number generation
5. **Multithreading** — HASH and AES support context swap with mutex
6. **Key Derivation** — Key derivation: IV+KEY_SECRET → SHA256 → 32-byte key
7. **Key-Store** — Secure storage with MPU protection, validity period for data and auto-lock

> **Hardware or software AES (2026-07-21):** the AES driver automatically selects the path based on the device (see chap. 13.4). On Nucleo-H753 it uses the hardware CRYP; on Nucleo-U575 — which lacks HW AES — it uses the software AES-256 implementation (FIPS-197 + SP 800-38D). The example runs **identically** on both: the commands `aes`, `aes_gcm`, `aes_mt` do not change. The command `aes_nist test` verifies the NIST vectors (absolute correctness of the software path).
>
> **AES-128 (23/10/2026):** the software path now also supports 128-bit keys, required by the mandatory TLS 1.3 suite. This example continues to test only AES-256; AES-128 is verified on the PC (184/184 against the `cryptography` library) and on the boards via HTTPS (examples 45 and 80), including on the H753 CRYP.

Hardware (Nucleo-H753ZI or Nucleo-U575ZI-Q)

- **Console UART** for STDOUT/CLI: the ST-LINK VCP (`UART_DBG_NUM` in `objmap.h`: USART3 on H753, USART1 on U575)
- **USB CDC** for alternative output

CLI Commands

Command	Source	Description
`crc32`	application	Calculate CRC32 on test buffer
`hash`	application	Calculate SHA-256 one-shot
`hash_mt`	application	Concurrent HASH test (2 synchronized threads)
`aes`	application	AES-256 ECB encrypt/decrypt test
`aes_gcm`	application	AES-256 GCM (authenticated encryption) test
`aes_mt`	application	Concurrent AES test (2 synchronized threads)
`aes_nist`	application	Known-answer test: GCM 4/4 (McGrew/Viega) + ECB 2/2 (FIPS-197)
`rng get`	driver `rng.c`	Generate and print a single 32-bit random value
`derive`	application	Derive key: IV+KEY → SHA256 → 32-byte key
`kstore`	application	Key-store test: 3 keys with different validity
`test`	application	Run all tests (crc32, hash, hash_mt, aes_ecb, aes_gcm, aes_mt, aes_nist, derive, kstore) and print PASS/FAIL for each, with final verdict
`full`	application	Run entire sequence: derive + kstore + use
`kstore info`	driver `keys_store.c`	Global status: slots used, occupied area
`kstore list`	driver `keys_store.c`	Key list: id, size, access, validity, status
`crc32 calc <addr> <len>`	driver `crc32.c`	Calculate CRC32 on memory range
`help`	chain	Command list (driver + application)

The CLI uses the **chain of responsibility** pattern. Commands `rng get`, `kstore info/list` and `crc32 calc` are handled by driver nodes registered automatically during driver init.

STDOUT Log (every 5 s)

```
EX26: rng=0xXXXXXXXX  tick=NNNNN
```

49.1.2 System architecture

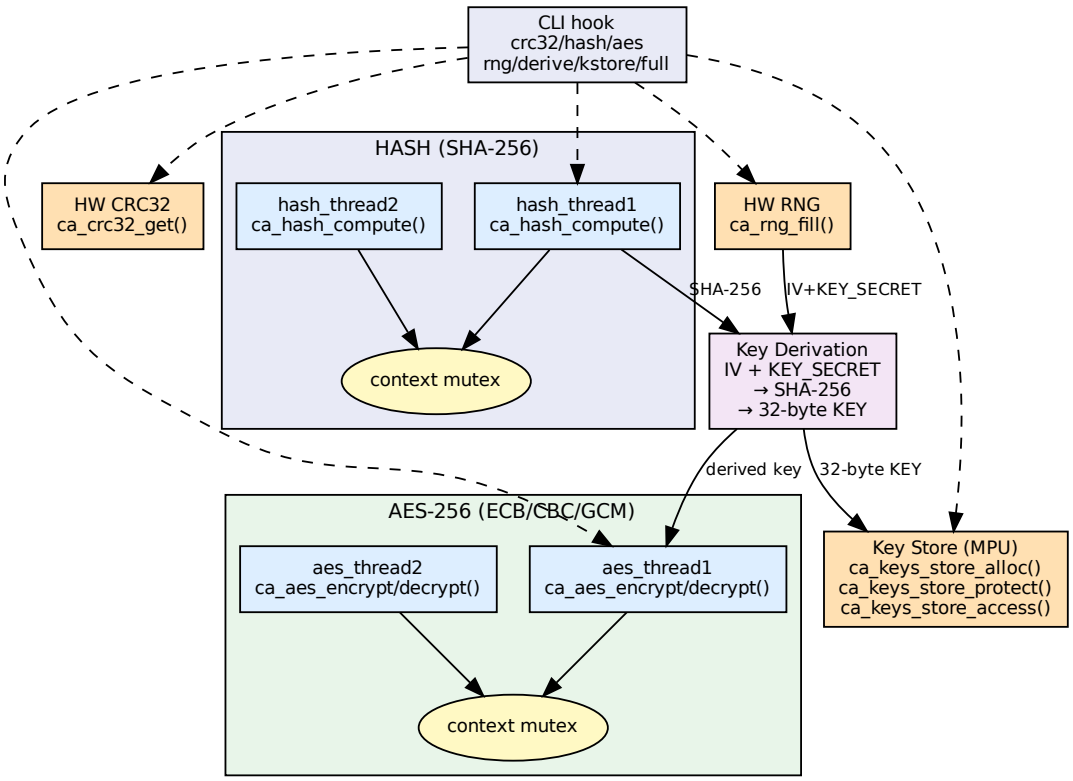


Figure 49.1: Architecture Example 26 — Crypto pipeline

49.1.3 Source code

The file is located in `examples/example_26/ca_os_ex26.c`:

```
// Proprietary     All rights reserved
// Copyright (c) 2026 Angelo Coppi

/*
 * Esempio 26     Crittografia: CRC32, HASH, AES, RNG e Key Derivation
 *
 * Hardware (Nucleo-H753ZI):
 *   - UART2 per STDOUT/CLI (CN5: TX=PA2, RX=PA3)
 *   - USB CDC per output alternativo
 *
 * Comandi CLI:
 *   crc32          Calcola CRC32 su buffer di test
 *   hash           Calcola SHA-256 one-shot
 *   hash_mt        Test HASH concorrente (2 thread)
 *   aes            Test AES-256 ECB encrypt/decrypt
 *   aes_gcm        Test AES-256 GCM (authenticated encryption)
 *   aes_mt         Test AES concorrente (2 thread)
 *   rng get        Genera un valore casuale 32-bit (driver rng.c)
 *   derive         Deriva chiave: IV+KEY -> SHA256 -> AES256 -> 32-byte key
 *   kstore         Test key-store: salva/recupera chiave protetta
 *   full           Esegue tutta la sequenza: derive + kstore + aes_gcm
```

```
 *   help           Lista comandi disponibili
 */

#include "caos.h"

#if EN_CRC && EN_HASH && EN_AES && EN_RNG

#define TEST_BUFFER_SIZE    256
#define AES_256_KEY_SIZE    32
#define AES_IV_SIZE         16
#define GCM_TAG_SIZE        16
#define KEY_SECRET_SIZE     32
#define IV_SECRET_SIZE      16

/* Stato condiviso */
static ca_mutex_t g_sync_mutex;
static ca_semaphore_t g_start_sem;
static volatile u8 g_hash_threads_started = 0;
static volatile u8 g_aes_threads_started = 0;
static u8 g_test_data[TEST_BUFFER_SIZE];
static u8 g_aes_key[AES_256_KEY_SIZE];
static u8 g_aes_iv[AES_IV_SIZE];
static u8 g_derived_key[32];
static u8 g_derived_key_ready = 0;
static void* g_secure_key_area = NULL;
static volatile u32 g_hash_thread1_count = 0;
static volatile u32 g_hash_thread2_count = 0;
static volatile u32 g_aes_thread1_count = 0;
static volatile u32 g_aes_thread2_count = 0;

/* Stampa buffer esadecimale con etichetta */
static void print_hex_label(const char* label, const u8* data, u32 len) {
    ca_stdout_printf("\n%s (%u bytes):", label, len);
    u16 lines = (u16)((len + 15) / 16);
    ca_hexdump((u8*)data, lines);
}

/* Inizializza buffer di test */
static void init_test_buffer(void) {
    for (u32 i = 0; i < TEST_BUFFER_SIZE; i++) {
        g_test_data[i] = (u8)(i & 0xFF);
    }
}

/* Genera chiave e IV casuali */
static void generate_random_key_iv(void) {
    ca_rng_fill(g_aes_key, AES_256_KEY_SIZE);
    ca_rng_fill(g_aes_iv, AES_IV_SIZE);
}

/* ... (continua nel file completo) ... */
```

49.1.4 Detailed analysis

1. CRC32 Example

The hardware CRC32 module calculates a 32-bit checksum to verify data integrity:

```
static void example_crc32(void) {
    init_test_buffer();

    u32 crc = ca_crc32_get(g_test_data, TEST_BUFFER_SIZE);
    ca_stdout_put_n("=== CRC32 Example ===");
```

```
    ca_stdout_printf_n("Buffer size: %u bytes", TEST_BUFFER_SIZE);
    ca_stdout_printf_n("CRC32: 0x%08X", crc);

    /* Modifica un byte e ricalcola */
    g_test_data[100] ^= 0xFF;
    u32 crc_modified = ca_crc32_get(g_test_data, TEST_BUFFER_SIZE);
    ca_stdout_printf_n("CRC32 (modified): 0x%08X (different!)", crc_modified);
}
```

Note on output functions: - `ca_stdout_put_n()` and `ca_stdout_printf_n()` automatically add a trailing `\n` - `ca_stdout_put()` and `ca_stdout_printf()` do NOT add a trailing `\n` - Use `_n` for messages that automatically wrap to a new line

Expected output:

```
=== CRC32 Example ===
Buffer size: 256 bytes
CRC32: 0x12345678
CRC32 (modified): 0xABCDEF01 (different!)
```

2. HASH Example (SHA-256)

```
static void example_hash_oneshot(void) {
    u8 digest[HASH_SHA256_DIGEST_SIZE];
    u32 digest_len;

    init_test_buffer();

    if (ca_hash_compute(CA_HASH_ALGO_SHA256, g_test_data, TEST_BUFFER_SIZE,
                        digest, &digest_len)) {
        ca_stdout_put_n("=== HASH SHA-256 One-Shot ===");
        print_hex_label("SHA-256 digest", digest, digest_len);
    }
}
```

Multithread Hash (Context Swap) HASH supports context swap via mutex, allowing multiple threads to calculate hashes concurrently:

```
static u32 task_hash_thread1(void *arg) {
    (void)arg;
    ca_hash_ctx_t ctx;
    u8 digest[HASH_SHA256_DIGEST_SIZE];
    u32 digest_len;

    g_hash_threads_started++;
    ca_sem_wait(&g_start_sem);  /* Attendi start sincronizzato */

    while (1) {
        ca_hash_ctx_init(&ctx, CA_HASH_ALGO_SHA256);

        /* Calcolo incrementale in 3 step */
        ca_hash_ctx_update(&ctx, (const u8*)"Hello ", 6);
        ca_os_sleep(10);
        ca_hash_ctx_update(&ctx, (const u8*)"concurrent ", 11);
        ca_os_sleep(10);
        ca_hash_ctx_update(&ctx, (const u8*)"world!", 6);

        ca_hash_ctx_final(&ctx, digest, &digest_len);
        g_hash_thread1_count++;

        ca_os_sleep(100);
```

```
    }
    return 0;
}
```

Synchronization:

```
static void example_hash_multithread(void) {
    ca_sem_init(&g_start_sem, 0, 10);  /* Semaphore a 0 */

    /* Crea 2 thread */
    ca_os_job("hash_t1", CA_TH_LEVEL2, task_hash_thread1, NULL, STACK_MEDIUM,
    CA_JOB_STD, 0);
    ca_os_job("hash_t2", CA_TH_LEVEL2, task_hash_thread2, NULL, STACK_MEDIUM,
    CA_JOB_STD, 0);

    /* Attendi che entrambi i thread siano in ca_sem_wait */
    while (g_hash_threads_started < 2) {
        ca_os_sleep(10);
    }

    /* Start sincronizzato: 2 post per 2 thread */
    ca_sem_post(&g_start_sem);
    ca_sem_post(&g_start_sem);
}
```

3. AES Example

```
static void example_aes_ecb(void) {
    u8 plaintext[16] = "Hello AES World!";
    u8 ciphertext[16];
    u8 decrypted[16];
    ca_aes_ctx_t ctx;

    generate_random_key_iv();
    ca_aes_ctx_setup(&ctx, CA_AES_MODE_ECB, CA_AES_KEY_256, g_aes_key, NULL);

    ca_aes_encrypt(&ctx, plaintext, ciphertext, 16);
    ca_aes_decrypt(&ctx, ciphertext, decrypted, 16);

    if (ca_memcmp(plaintext, decrypted, 16) == 0) {
        ca_stdout_put_n("AES-256 ECB: SUCCESS (plaintext recovered)");
    }
}
```

```
static void example_aes_gcm(void) {
    u8 plaintext[48] = "AES-GCM authenticated encrypt!";
    u8 ciphertext[48];
    u8 decrypted[48];
    u8 tag[GCM_TAG_SIZE];
    u8 aad[16] = "authenticated!";
    ca_aes_ctx_t ctx;

    generate_random_key_iv();
    ca_aes_ctx_setup(&ctx, CA_AES_MODE_GCM, CA_AES_KEY_256, g_aes_key, g_aes_iv)
    ;

    /* Encrypt con GCM */
    ca_aes_gcm_encrypt(&ctx, aad, 16, plaintext, ciphertext, 32, tag);

    /* Decrypt con verifica tag */
```

```
    if (ca_aes_gcm_decrypt(&ctx, aad, 16, ciphertext, decrypted, 32, tag)) {
        ca_stdout_put_n("AES-256 GCM Decrypt: SUCCESS");
    }

    /* Test tamper detection */
    tag[0] ^= 0xFF;
    if (!ca_aes_gcm_decrypt(&ctx, aad, 16, ciphertext, decrypted, 32, tag)) {
        ca_stdout_put_n("AES-256 GCM Tamper Detection: SUCCESS");
    }
}
```

Verification against known vectors (aes_nist) `aes`/`aes_gcm` test the **round-trip** (encrypt→decrypt with the same key), but a round-trip *hides* symmetric bugs: if the hardware path incorrectly orders the bytes during encryption **and** decryption in the same way, the plaintext still returns correctly. A **known-answer test** (KAT) does not: it compares the output against a known external value. The `aes_nist` command — from the example — executes two KAT batteries through the **public dispatch** (`ca_aes_encrypt`/`ca_aes_gcm_*`), thereby validating the **active** backend, HW on H753 or SW on U575:

- `ca_aes_gcm_selftest()` — AES-256-GCM vectors (McGrew/Viega, the same as the NIST suites): 4 cases;
- `ca_aes_ecb_selftest()` — AES-256-ECB single-block vector FIPS-197 App. C.3.

```
c.a.O.S > aes_nist
aes-gcm: NIST AES-256-GCM known-answer tests (dispatch: HW/SW)
  case13 empty                PASS
  case14 one block            PASS
  case15 payload              PASS
  case16 aad+payload          PASS
aes-gcm: ALL PASS (4/4)
aes-ecb: FIPS-197 AES-256-ECB known-answer (dispatch: HW/SW)
  ecb256 encrypt              PASS
  ecb256 decrypt              PASS
aes-ecb: ALL PASS (2/2)
```

The four GCM cases cover: no data, a single block, long payload (60 B), and AAD+payload; each case also verifies the decryption round-trip and the rejection of a tampered tag (constant-time). ECB verifies encryption and decryption of a block against the FIPS-197 vector. The KATs run **in-place** (same input and output buffer), as in the actual SSH record path.

> **Why KAT and not round-trip:** this distinction unmasked three real bugs in the H753 hardware CRYP during SSH bring-up (IV counter, data endianness, missing `ca_aes_init()`) that a simple encrypt→decrypt would have let pass. General rule: **HW crypto always validated with known-answer, never with round-trip.**

4. RNG

Random number generation is used internally by `generate_random_key_iv()` and `example_key_derivation()` via `ca_rng_fill()`.

The CLI command `rng get` is provided directly by the `rng.c` driver (automatically registered by `ca_rng_init()`):

```
c.a.O.S > rng get
rng: 0xA3F12C7B  (2751987835)
```

The periodic log every 5 seconds also shows an RNG sample:

```
EX26: rng=0x1A2B3C4D  tick=5000
```

The application command `rng` (which generated 8 values) has been removed. Use `rng get` from the driver for interactive RNG testing.

5. Key Derivation: IV+KEY_SECRET → SHA256 → 32-BYTE KEY

```
static void example_key_derivation(void) {
    u8 iv_secret[IV_SECRET_SIZE];
    u8 key_secret[KEY_SECRET_SIZE];
    u8 hash_input[IV_SECRET_SIZE + KEY_SECRET_SIZE];
    u8 sha256_digest[HASH_SHA256_DIGEST_SIZE];
    u32 digest_len;

    /* Genera IV e KEY_SECRET casuali */
    ca_rng_fill(iv_secret, IV_SECRET_SIZE);
    ca_rng_fill(key_secret, KEY_SECRET_SIZE);

    print_hex_label("IV_SECRET", iv_secret, IV_SECRET_SIZE);
    print_hex_label("KEY_SECRET", key_secret, KEY_SECRET_SIZE);

    /* Step 1: Concatena IV + KEY_SECRET */
    ca_memcpy(hash_input, iv_secret, IV_SECRET_SIZE);
    ca_memcpy(hash_input + IV_SECRET_SIZE, key_secret, KEY_SECRET_SIZE);
    ca_stdout_printf_n("Step 1: IV + KEY_SECRET concatenated (%u bytes)",
            IV_SECRET_SIZE + KEY_SECRET_SIZE);

    /* Step 2: SHA-256 hash */
    ca_hash_compute(CA_HASH_ALGO_SHA256, hash_input,
                    IV_SECRET_SIZE + KEY_SECRET_SIZE,
                    sha256_digest, &digest_len);

    /* Step 3: Usa SHA-256 digest come chiave AES-256 (32 byte) */
    ca_memcpy(g_derived_key, sha256_digest, 32);
    g_derived_key_ready = 1;

    print_hex_label("Derived 32-BYTE SECRET KEY", g_derived_key, 32);
}
```

Derivation flow:

```


     IV_SECRET            KEY_SECRET
     (16 byte)             (32 byte)



                   Concatenazione

            IV || KEY          (48 byte)



             SHA-256
              (HASH)



            32-byte KEY
```

```
            (chiave AES-256)

```

Note: The derivation uses only SHA-256. The SHA-256 digest (32 bytes) is the derived key itself, without further AES steps.

6. Key-Store Example (Secure Storage with MPU + Validity)

The key-store protects cryptographic keys via the **MPU (Memory Protection Unit)**: all keys reside in a single memory area covered by a single MPU region. Each key can have a **validity period by date** (start/stop); if the key has expired, it is automatically locked (`NO_ACCESS`) upon the first verification.

```c
void ca_os_configure(void) {
    EVAL_CRC(ca_crc32_init();)
    EVAL_HASH(ca_hash_init();)
    EVAL_AES(ca_aes_init();)
    EVAL_RNG(ca_rng_init();)
#if EN_KEYS_STORE
    ca_keys_store_init(10);   /* max 10 chiavi     CLI kstore disponibile subito
    */
#endif
    /* ... stdout, CLI ... */
}
```

Early init ensures that `kstore info` and `kstore list` respond immediately without waiting for the execution of the `kstore` command.

6.2 Packed date — validity format The date is encoded in a `u32` with the `KSTORE_DATE_PACK` macro:

```c
/* Macro definite in keys_store.h */
#define KSTORE_DATE_PACK(y, m, d)  (((u32)(y) << 16) | ((u32)(m) << 8) | (u32)(d
    ))
#define KSTORE_DATE_YEAR(v)        (((v) >> 16) & 0xFFU)
#define KSTORE_DATE_MONTH(v)       (((v) >>  8) & 0xFFU)
#define KSTORE_DATE_DAY(v)         ((v) & 0xFFU)

/* Esempio: 2026-03-15 */
u32 d = KSTORE_DATE_PACK(26, 3, 15);   /* year    relativo al 2000 */
```

The packed u32 is **directly comparable** as an integer: larger values = more recent dates.

```c
/* Imposta periodo di validit  per chiave all'indice key_idx */
int  ca_keys_store_set_expiry(u8 key_idx, u32 start, u32 stop);

/* Verifica se chiave    scaduta; se s  , la blocca automaticamente */
bool ca_keys_store_is_key_expired(void *ptr);
```

Rules:

`validity_start`	`validity_stop`	Behavior
0	0	no validity → always valid
date	0	valid from `start` onwards, no expiration
0	date	valid until `stop` (immediately operational)

validity_start	validity_stop	Behavior
date	date	valid within the window [start, stop]

RTC Dependency:

- RTC disabled (EN_RTC=0) and validity set → key **not usable** (auto-lock)
- RTC enabled but date invalid (year < 2024 or > 2300, month > 12, day > 31) → key **not usable**
- RTC enabled and date valid → comparison against current date

```
static u8 get_key_id(void *ptr) {
    ca_secure_key_entry_t info[MAX_SECURE_KEYS];
    u8 cnt = MAX_SECURE_KEYS;
    if (!ptr || ca_keys_store_get_keys_info(info, &cnt) != 0) return 0xFF;
    for (u8 i = 0; i < cnt; i++) {
        if (info[i].base_address == (u32)ptr) return (u8)info[i].key_id;
    }
    return 0xFF;
}
```

ca_keys_store_set_expiry requires the slot index (key_id). get_key_id derives it from the pointer via get_keys_info, without touching the area protected by the MPU.

```
static void example_key_store(void) {
    u8 key_data[32];

    /* --- KEY 0: permanente (AES-256, derived o random) --- */
    g_secure_key_area = ca_keys_store_allocate(32);

    if (g_derived_key_ready)
        ca_memcpy(key_data, g_derived_key, 32);
    else
        ca_rng_fill(key_data, 32);

    ca_keys_store_temp_enable_access(g_secure_key_area, CA_KEYS_STORE_READ_WRITE
    );
    ca_keys_store_copy(g_secure_key_area, key_data, 32);
    ca_keys_store_lock_completely(g_secure_key_area);
    /* nessuna set_expiry     validity_start=0, validity_stop=0     permanente
    */

    /* --- KEY 1: valida 2026-01-01     2027-12-31 --- */
    void *kv = ca_keys_store_allocate(16);
    ca_keys_store_temp_enable_access(kv, CA_KEYS_STORE_READ_WRITE);
    /* ... copia e lock ... */
    ca_keys_store_set_expiry(get_key_id(kv),
        KSTORE_DATE_PACK(26, 1,  1),
        KSTORE_DATE_PACK(27, 12, 31));
    bool exp = ca_keys_store_is_key_expired(kv);
    /* exp = false se RTC indica data 2026-xx-xx .. 2027-xx-xx */

    /* --- KEY 2: scaduta 2024-01-01     2025-12-31 --- */
    void *ke = ca_keys_store_allocate(16);
    ca_keys_store_temp_enable_access(ke, CA_KEYS_STORE_READ_WRITE);
    /* ... copia e lock ... */
```

```
    ca_keys_store_set_expiry(get_key_id(ke),
        KSTORE_DATE_PACK(24, 1,  1),
        KSTORE_DATE_PACK(25, 12, 31));
    bool exp2 = ca_keys_store_is_key_expired(ke);
    /* exp2 = true     ca_keys_store_is_key_expired chiama lock_completely
    automaticamente */

    /* --- Uso della chiave permanente --- */
    if (!ca_keys_store_is_key_expired(g_secure_key_area)) {
        ca_keys_store_temp_enable_access(g_secure_key_area,
    CA_KEYS_STORE_READ_ONLY);
        u8 retrieved[32];
        ca_memcpy(retrieved, g_secure_key_area, 32);
        ca_keys_store_temp_disable_access(g_secure_key_area);
    }
}
```

```
CA_KEYS_STORE_NO_ACCESS   = 0   (default     nessun accesso)
CA_KEYS_STORE_READ_ONLY   = 1   (sola lettura)
CA_KEYS_STORE_READ_WRITE  = 2   (lettura + scrittura)
CA_KEYS_STORE_EXEC_ONLY   = 3   (solo esecuzione     uso raro)
```

6.7 CLI driver kstore The CLI driver is automatically registered by `ca_keys_store_init()` (guard `EN_CLI_KSTORE=1`):

```
c.a.O.S > kstore info
kstore: slots=3/10  area=64/2048 bytes  mpu_rgn=2

c.a.O.S > kstore list
id  size  access  start       stop        status
 0    32  LOCKED  *           *           permanent
 1    16  LOCKED  2026-1-1    2027-12-31  valid
 2    16  LOCKED  2024-1-1    2025-12-31  EXPIRED
```

- `*` = no limit (validity_start=0 or validity_stop=0)
- `EXPIRED` = key already locked by `is_key_expired()`
- `permanent` = no validity set

7. Full Demo: Derive → Store → Use

```
static void example_full_demo(void) {
    ca_stdout_put_n("=========================================================");
    ca_stdout_put_n("         FULL CRYPTO DEMO: Derive -> Store -> Use        ");
    ca_stdout_put_n("=========================================================");

    /* Step 1: Key derivation */
    example_key_derivation();
    ca_os_sleep(500);

    /* Step 2: Store in key-store */
    example_key_store();
    ca_os_sleep(500);

    /* Step 3: Usa la chiave dal key-store per AES-GCM */
    if (g_secure_key_area && g_derived_key_ready) {
        ca_keys_store_temp_enable_access(g_secure_key_area,
    CA_KEYS_STORE_READ_ONLY);
```

```

        u8 stored_key[32];
        ca_memcpy(stored_key, g_secure_key_area, 32);
        ca_keys_store_temp_disable_access(g_secure_key_area);

        ca_aes_ctx_t ctx;
        u8 iv[16];
        ca_rng_fill(iv, 16);

        u8 plaintext[64] = "Msg encrypted with derived stored key!";
        u8 ciphertext[64];
        u8 tag[16];

        ca_aes_ctx_setup(&ctx, CA_AES_MODE_GCM, CA_AES_KEY_256, stored_key, iv);
        ca_aes_gcm_encrypt(&ctx, NULL, 0, plaintext, ciphertext, 48, tag);
    }
}
```

49.1.5 Expected output

At startup

```
=== Example 26: Crypto + Key-Store ===
Hardware: CRC32 | SHA-256 | AES-256 | RNG
Multithread: HASH & AES with context swap
Key derivation: IV+KEY -> SHA256 -> 32-byte key
Key-store: MPU protected, validity per date, auto-lock
CLI driver: crc32 calc | rng get | kstore info | kstore list
Type 'help' for app commands
```

kstore command

```
=== Key-Store Example (MPU protected) ===
Key[0]: 32B  AES-256  validity=permanent
Key[1]: 16B  start=2026-01-01  stop=2027-12-31  -> valid
Key[2]: 16B  start=2024-01-01  stop=2025-12-31  -> EXPIRED (auto-locked)
--- Using key[0] for AES (simulate) ---
Key[0] verification: OK
Type 'kstore list' to see live validity status
Type 'kstore info' for storage statistics
```

kstore list command (CLI driver)

```
c.a.O.S > kstore list
id  size  access  start       stop        status
 0    32  LOCKED  *           *           permanent
 1    16  LOCKED  2026-1-1    2027-12-31  valid
 2    16  LOCKED  2024-1-1    2025-12-31  EXPIRED
```

full command

```
=========================================================
         FULL CRYPTO DEMO: Derive -> Store -> Use
=========================================================
=== Key Derivation ===
...
Step 3: Derived 32-BYTE SECRET KEY (32 bytes): ...
=== Key-Store Example (MPU protected) ===
```

```
Key[0]: 32B  AES-256  validity=permanent
...
=== Using stored key for AES-256 GCM ===
AES-256 GCM with stored key: SUCCESS
============================================================
```

Periodic log (every 5 seconds)

```
EX26: rng=0x1A2B3C4D  tick=5000
EX26: rng=0x5E6F7A8B  tick=10000
```

49.1.6 Required configuration in `os_config.h`

```
#define EN_CRC         1    /* Driver CRC32 */
#define EN_HASH        1    /* Driver HASH (SHA-256) */
#define EN_AES         1    /* Driver AES */
#define EN_RNG         1    /* Driver RNG */
#define EN_KEYS_STORE  1    /* Secure key-store con MPU */
#define EN_RTC         1    /* RTC     obbligatorio per validit  chiavi */
#define ENABLE_CLI     1    /* CLI per comandi utente */
#define ENABLE_STDOUT  1    /* STDOUT per output */
/* CLI driver */
#define EN_CLI_CRC     1    /* crc32 calc <addr> <len> */
#define EN_CLI_RNG     1    /* rng get */
#define EN_CLI_KSTORE  1    /* kstore info | kstore list */
```

Note: if `EN_RTC=0`, keys with set validity are always **unusable** (auto-locked). If `EN_RTC=1` but the RTC is not yet initialized with a valid date (year < 2024 or > 2300), keys with validity are likewise locked.

49.1.7 Important notes

1. Context Swap for Concurrency

Both HASH and AES support **context swap**, allowing multiple threads to use the hardware modules concurrently:

- **HASH**: `ca_hash_ctx_t` saves/restores the CSR[] registers
- **AES**: `ca_aes_ctx_t` preserves key and IV; the driver reloads them into the peripheral (HW) or uses them directly (SW) at each operation

Both use internal mutexes to protect access.

2. Thread Synchronization

Multithreaded threads use a **semaphore** to start in sync:

```
ca_sem_init(&g_start_sem, 0, 10);  /* Inizializza a 0 */

/* Ogni thread incrementa il contatore e si blocca */
g_hash_threads_started++;          /* thread 1 */
g_hash_threads_started++;          /* thread 2 */
ca_sem_wait(&g_start_sem);         /* entrambi si bloccano */

/* Chiamante attende che ENTRAMBI siano pronti, poi d  il via */
while (g_hash_threads_started < 2) { ca_os_sleep(10); }
ca_sem_post(&g_start_sem);         /* Sblocca primo thread */
ca_sem_post(&g_start_sem);         /* Sblocca secondo thread */
```

3. Key-Store, MPU and Validity

The key-store uses the **MPU** to protect keys in a single shared area:

- A single MPU region covers all keys (1 region = max 10 keys)
- Default: `NO_ACCESS` — no access, any read/write generates a Hard Fault
- Temporary access: `temp_enable_access` → use the key → `temp_disable_access`

Validity by date:

```
/* Imposta finestra di validit  */
ca_keys_store_set_expiry(key_idx,
    KSTORE_DATE_PACK(26, 1, 1),   /* start: 2026-01-01 */
    KSTORE_DATE_PACK(27, 12, 31)); /* stop:  2027-12-31 */

/* Verifica (auto-lock se scaduta) */
if (ca_keys_store_is_key_expired(ptr)) {
    /* chiave bloccata     non usabile */
}
```

`is_key_expired()` **Behavior:**

Condition	Result
`validity_start=0` and `validity_stop=0`	`false` (permanent)
RTC disabled, validity set	`true` + auto-lock
RTC enabled, invalid date	`true` + auto-lock
Current date outside window	`true` + auto-lock
Current date inside window	`false`

4. AES-GCM Tag Verification

AES-GCM provides **authenticated encryption**:

- The tag (16 bytes) authenticates the ciphertext
- Decryption automatically verifies the tag
- If the tag is tampered with, decryption fails

5. CLI — Chain of Responsibility

The CLI uses the **chain of responsibility** pattern. The application node handles cryptographic commands; the driver node `rng.c` handles `rng get` (automatically registered by `ca_rng_init()`).

```
static bool user_cli_handler(c_str *pcmd)
{
    c_str cmd = *pcmd;
    const char *p = ca_skip_sp(cmd->data);

    if (ca_strncmp(p, "help", 4) == 0) {
        CLI_PUT_N("ex26      : test (all, with verdict) | full (derive+kstore+
    gcm)");
        CLI_PUT_N("            crc32 | hash | hash_mt | aes | aes_gcm | aes_mt")
    ;
        CLI_PUT_N("            aes_nist (GCM+ECB KAT) | derive | kstore");
        CLI_PUT_N("            rng get  (driver command)");
        return false;
    }
    if (ca_strncmp(p, "crc32", 5) == 0)   { example_crc32();              return
     true; }
    if (ca_strncmp(p, "hash_mt", 7) == 0) { example_hash_multithread();   return
     true; }
```

```
    if (ca_strncmp(p, "hash", 4) == 0)    { example_hash_oneshot();        return true; }
    if (ca_strncmp(p, "aes_nist", 8) == 0) { EVAL_AES_GCM_SW_TEST((void)ca_aes_gcm_selftest(); (void)ca_aes_ecb_selftest();) return true; }
    if (ca_strncmp(p, "aes_gcm", 7) == 0) { example_aes_gcm();             return true; }
    if (ca_strncmp(p, "aes_mt", 6) == 0)  { example_aes_multithread();     return true; }
    if (ca_strncmp(p, "aes", 3) == 0 && (p[3] == 0 || p[3] == ' ')) { example_aes_ecb(); return true; }
    if (ca_strncmp(p, "derive", 6) == 0)  { example_key_derivation();      return true; }
    if (ca_strncmp(p, "kstore", 6) == 0)  { example_key_store();          return true; }
    if (ca_strncmp(p, "test", 4) == 0)    { run_all_tests();              return true; }
    if (ca_strncmp(p, "full", 4) == 0)    { example_full_demo();          return true; }
    return false;
}
```

Registration:

```
EVAL_CLI( ca_cli_register(&s_user_cli_node); )
```

6. Hardware RNG

The hardware RNG module generates true random numbers:

- `ca_rng_get()`: single 32-bit value
- `ca_rng_fill()`: fills buffer with random bytes
- Protected by mutex for concurrent access
- CLI: `rng get` automatically registered by `ca_rng_init()`

6. STDOUT Output Functions

STDOUT output functions have different behaviors:

Function	Adds trailing `\n`	Typical use
`ca_stdout_put_n()`	Yes	Complete messages
`ca_stdout_printf_n()`	Yes	Formatted string with newline
`ca_stdout_put()`	No	Messages without newline
`ca_stdout_printf()`	No	Formatted string without newline

Example:

```
/* Corretto: _n aggiunge newline automaticamente */
ca_stdout_put_n("=== CRC32 Example ===");
ca_stdout_printf_n("Buffer size: %u bytes", TEST_BUFFER_SIZE);

/* Corretto: senza _n, newline va specificato se serve */
ca_stdout_printf("\n%s (%u bytes):", label, len);  /* Solo \n iniziale */
ca_hexdump((u8*)data, lines);  /* hexdump gestisce il resto */
```

49.1.8 Summary

1. **CRC32**: checksum for data integrity; `crc32 calc <addr> <len>` via CLI driver
2. **HASH SHA-256**: one-shot and incremental with context swap

3. **AES-256**: ECB, CTR, GCM with context swap; automatic dispatch HW (CRYP/AES) or software; `aes_nist` = known-answer test of the active backend (GCM 4/4 + ECB 2/2), not round-trip
4. **RNG**: hardware random number generation; `rng get` via CLI driver
5. **Multithreading**: 2 threads synchronized with semaphore
6. **Key Derivation**: IV+KEY → SHA256 → 32-byte key
7. **Key-Store**: MPU + date-based validity; `ca_keys_store_set_expiry()` sets the window; `is_key_expired()` verifies and auto-locks; `kstore info/list` via CLI driver
8. **RTC-dependent validity**: if RTC is disabled or date is invalid, keys with validity are automatically locked (fail-secure)
9. **STDOUT**: use `_n` for automatic newline, without `_n` for manual control
10. **CLI chain of responsibility**: driver nodes (`crc32.c`, `rng.c`, `keys_store.c`) automatically registered at init; application node handles cryptographic commands; `help` propagates the entire chain

Chapter 50

DAC Output and ADC Readback (Polling and ISR)

50.1 Example 27 — DAC output and ADC readback (polling and ISR)

50.1.1 Introduction

This example demonstrates the combined use of the **DAC** and **ADC** on the Nucleo-H753ZI, connecting the DAC analog output directly to an ADC input via a simple hardware jumper.

The CLI commands are provided directly by the `dac.c` and `adc.c` drivers via the **chain of responsibility** — the example does not register any CLI handler of its own.

The coexistence of DAC, ADC, and USB CDC in the same system requires correct configuration of the ADC kernel clock, a topic covered in detail in the technical notes at the end of the chapter.

Hardware (Nucleo-H753ZI, MB1364)

Signal	MCU Pin	Nucleo Connector	Function
DAC CH1	PA4	CN7 pin 17 (D24)	Analog output
ADC1 CH10	PC0	CN9 pin 3 (A1)	Analog input

Required jumper:

```
CN7 pin 17 (PA4, DAC CH1)                    CN9 pin 3 (PC0, ADC1 CH10)
```

Without the jumper, the DAC works correctly (verifiable with a multimeter), but the ADC reads an indeterminate value.

Application CLI Commands

Command	Description
`test`	DAC→ADC sweep with readback, 5% full-scale tolerance (205 counts), final verdict
`info`	Expected jumper, DAC channel and ADC channel on this board
`wire`	Jumper continuity on pure GPIO, verdict `WIRE RESULT : OK|FAIL (n/1)`; must be run first

Command	Description
`adcmap`	**U575 only**, no wire: checks on silicon the ADC1 channel→pin map CH15=PB0, CH16=PB1, CH17=PB2. The pin is driven high/low as an output, then made analog: the charge left on the floating pad moves the reading only on the right channel (CH1-19 scan). Verdict `ADCMAP RESULT: PASS|FAIL`. CH13/14 (PC4/PC5) left out: they reach no Nucleo connector

Commands are registered **without prefix** (`test`, not `ex27 test`).

CLI Commands (`dac.c` driver)

Command	Description
`dac info`	Show active channels and current values
`dac set <1|2> <0-4095>`	Set DAC value (0 = 0 V, 4095 = 3.3 V)
`dac run <1|2>`	Start automatic sawtooth; needs `EN_SW_TIMER`, off in ex27: here it answers `dac run: requires EN_SW_TIMER=1`
`dac stop <1|2>`	Stop sawtooth and set to 0 V

CLI Commands (`adc.c` driver)

Command	Description
`adc info`	Show active ADCs, ISR mode, counters
`adc read <1..3> <ch>`	Single polling read
`adc volt <1..3> <ch>`	Polling read with output in Volts
`adc isr <1..3> <ch>`	Start continuous conversion via ISR
`adc stop <1..3>`	Stop ISR mode

STDOUT Log (every 5 s)

```
EX27: adc=NNNN  N.NNNv  tick=NNNNN
```

50.1.2 System architecture

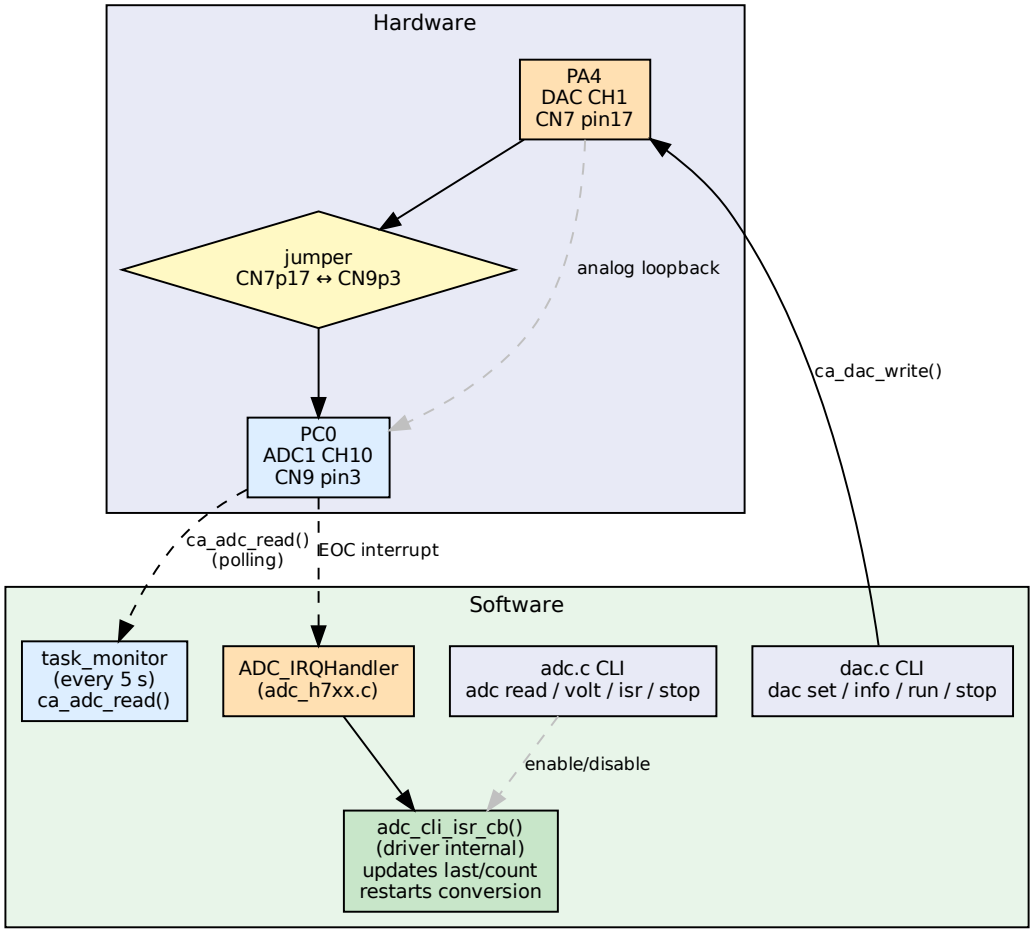


Figure 50.1: DAC+ADC Example architecture 27

50.1.3 Source code

The file is located in `examples/example_27/ca_os_ex27.c`:

```
// Proprietary     All rights reserved
// Copyright (c) 2026 Angelo Coppi

/*
 * Example 27     DAC output + ADC readback (polling and ISR)
 *
 * Hardware (Nucleo-H753ZI, MB1364):
 *   DAC CH1      PA4 (analog out)      CN7 pin 17 (D24)
 *   ADC1 CH10     PC0 (analog in)     CN9 pin 3   (A1)
 *
 *   JUMPER REQUIRED:
 *     CN7 pin 17               CN9 pin 3
 *     (PA4 DAC)        (PC0 ADC)
 *
```

```
 * CLI commands (driver dac.c + adc.c):
 *   dac set 1 <0-4095>       sets DAC CH1
 *   adc read 1 10            single polling read
 *   adc isr 1 10             starts continuous ISR
 *   adc stop 1               stops ISR
 *   adc volt 1 10            reading in Volts
 */

#include "caos.h"

#if EN_DAC && EN_ADC
```

50.1.4 Detailed analysis

1. Configuration constants

```
#define ADC_NUM           1U        /* ADC1 */

#if MCU_ID == 575
#define DAC_CHANNEL       1U        /* DAC1_OUT1     PA4, CN7-17 */
#define ADC_CHANNEL       1U        /* CH1       PC0 = ADC12_INP1  (U575) */
#elif MCU_ID == 563
#define DAC_CHANNEL       2U        /* DAC1_OUT2     PA5, CN7-10 */
#define ADC_CHANNEL       10U       /* CH10      PC0 = ADC12_INP10 */
#else
#define DAC_CHANNEL       1U        /* DAC1_OUT1     PA4, CN7-17 */
#define ADC_CHANNEL       10U       /* CH10      PC0 = ADC12_INP10 (H7xx) */
#endif
/* each branch also defines DAC_TXT, ADC_TXT, WIRE_TXT and WIRE_OUT_PIN */

#define ADC_SAMPLE_TIME   4U        /* 64.5 cycles     good accuracy compromise
    */
#define VREF_MV           3300U     /* 3.3 V reference */
```

Channels and pins depend on the board, not on the family: on the U575 PC0 is channel 1 (CN9-11); on the H563 PA4 reaches only the morpho header, so DAC channel 2 on PA5 is used (CN7-10, D13). On the H753 (`#else` branch), ADC channel 10 corresponds to pin PC0 both in the GPIO driver mapping (`set_channel_gpio: ch >= 10` → `CA_GPIO_PORT_C`, pin `ch-10`) and in the hardware connection of ADC1/2 (ADC12_INP10 = PC0). This correspondence is the only one guaranteed for ADC1/2 on H753; other channels present misalignments between the driver mapping and the physical pinout.

2. Shared state

The example no longer has its own shared state — ISR and counter management is internal to the `adc.c` driver (`s_cli_isr_*` variables). The monitor task reads directly via `ca_adc_read()` every 5 seconds.

3. Raw → millivolt conversion

The conversion is inline in the monitor task:

```
u32 mv = ((u32)adc * VREF_MV) / 4095U;
```

The multiplicand is promoted to `u32` to avoid overflow ($4095 \times 3300 = 13\,513\,500$, which exceeds the range of `u16`).

4. Task monitor

```
static u32 task_monitor(void *arg)
{
    (void)arg;

    ca_os_sleep(2000);
    ca_stdout_put_n("EX27: " DAC_TXT);
    ca_stdout_put_n("      " ADC_TXT);
    ca_stdout_put_n("Jumper: " WIRE_TXT);
    ca_stdout_put_n("Commands: test | info | wire   (+ the driver's dac/adc)");

    while (1) {
        ca_os_sleep(5000);
        u16 adc = ca_adc_read(ADC_NUM, ADC_CHANNEL);
        u32 mv  = ((u32)adc * VREF_MV) / 4095U;
        ca_stdout_printf_n("EX27: adc=%u  %u.%03uV  tick=%u",
            (unsigned)adc, mv/1000U, mv%1000U, (unsigned)ca_os_tick_ms());
    }
    return 0;
}
```

The example's unique task. It reads ADC1 ch10 in polling mode every 5 seconds and prints the raw value and voltage. DAC and ADC ISR control is performed via the driver's CLI commands.

5. ca_os_configure

```
void ca_os_configure(void)
{
    /* DAC CH1: PA4 analog output */
    ca_dac_config_t dac_cfg = { DAC_CHANNEL };
    ca_dac_init(&dac_cfg);
    ca_dac_write(DAC_CHANNEL, 0U);

    /* ADC CH10: PC0 analog input */
    ca_adc_config_t adc_cfg = { ADC_NUM, ADC_SAMPLE_TIME };
    ca_adc_init(&adc_cfg);
    ca_adc_pin_init(ADC_NUM, ADC_CHANNEL);


    EVAL_STDOUT(
        ca_stdout_init();
        ca_stdout_set_sender(STDOUT);
        ca_stdout_create_thread();
    )
    EVAL_CLI(
        ca_cli_init(2048);
        ca_cli_set_sender(STDOUT);
        ca_cli_create_thread();
    )
    EVAL_USB_CDC_STD(ca_usb_dev_mng_cdc_cli();)
}
```

The order is significant: DAC and ADC must be initialized **before** USB CDC (technical note on ADC clock). `ca_dac_init()` and `ca_adc_init()` automatically register their CLI nodes in the chain of responsibility.

6. ca_os_main

```
void ca_os_main(void)
```

```
{
    ca_os_job("monitor", CA_TH_LEVEL3, task_monitor, NULL, STACK_SMALL, ...);
    ca_os_run(true);
}
```

A single task. No local CLI hook — commands arrive from the drivers.

Task	Priority	Function
`monitor`	CA_TH_LEVEL3	Periodic log every 5 s

50.1.5 Expected output

At startup

```
EX27: DAC  CH1  -> PA4  CN7-17 (D24)
      ADC1 CH10 <- PC0  CN9-3  (A1)
Jumper: PA4 CN7-17 (DAC out) ---- PC0 CN9-3  (ADC in)
Commands: test | info | wire   (+ the driver's dac/adc)
```

Typical test sequence

```
> dac set 1 2048
DAC1 = 2048  1.650 V

> adc read 1 10
ADC1 ch10 = 2041  1.644 V

> adc volt 1 10
ADC1 ch10 = 1.644 V (raw=2041)

> adc isr 1 10
ADC1 ch10 ISR started

EX27: adc=2041  1.644V  tick=5000
EX27: adc=2039  1.643V  tick=10000

> adc stop 1
ADC1 stopped  4092116 samples
```

Periodic log (every 5 s)

```
EX27: adc=2041  1.644V  tick=5000
```

50.1.6 ADC driver architecture — ISR mode

ISR API added to the `adc.c` driver

```
/* Type of the callback called by the IRQ handler */
typedef void (*ca_adc_isr_cb_t)(u8 adc_num, u16 value);

/* Initialize the ADC with ISR support and register the callback */
bool ca_adc_init_isr(const ca_adc_config_t *cfg, ca_adc_isr_cb_t cb);

/* Starts a single conversion in ISR mode */
void ca_adc_start_isr(u8 adc_num, u8 channel);

/* Called by the IRQ handler of the hardware family */
void ca_adc_irq_handler(u8 adc_num);
```

Full ISR mode flow

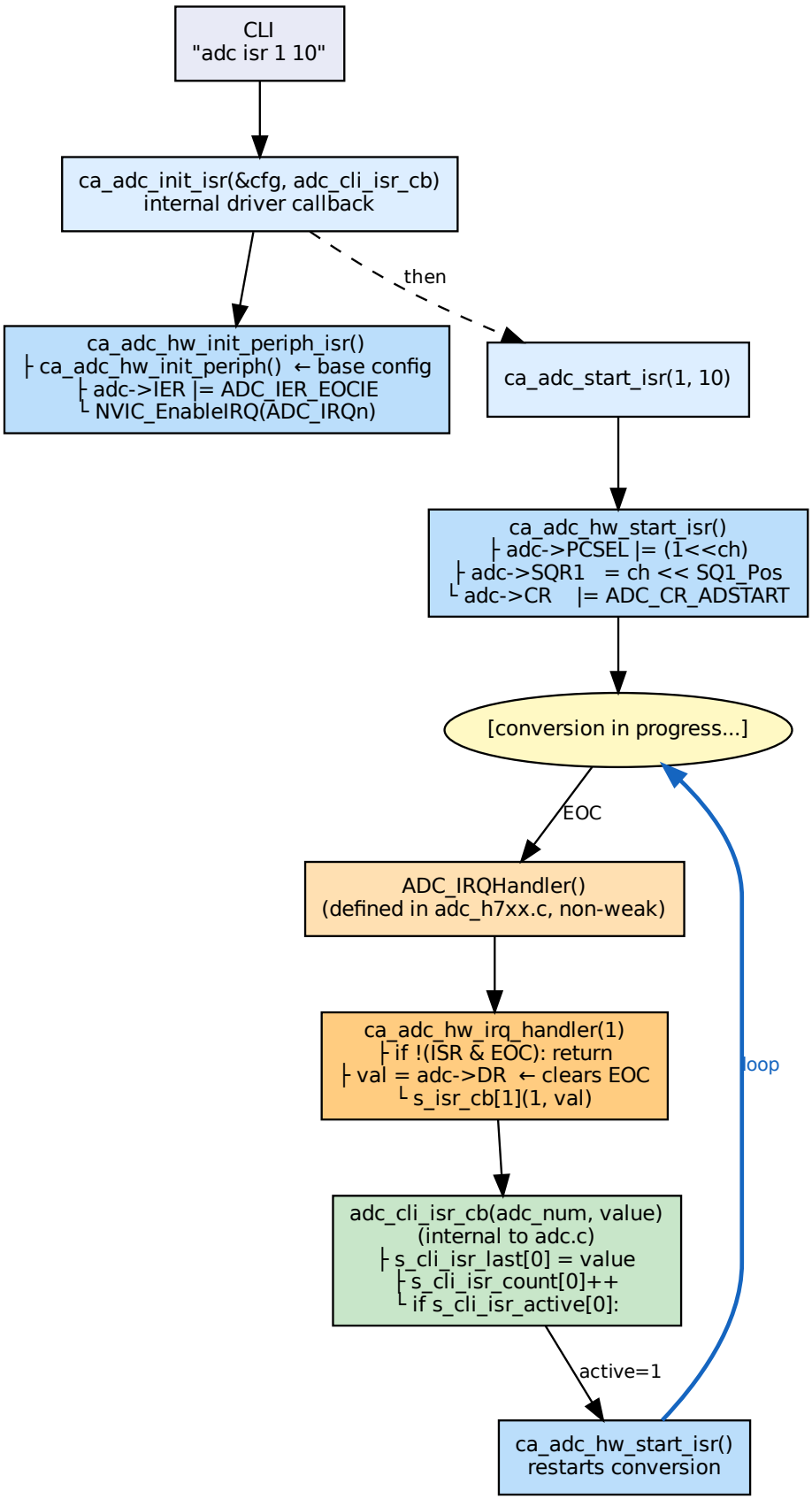


Figure 50.2: ADC ISR flow — init and loop

IRQ handlers by family

ADC IRQ handlers are protected by `#if EN_ADC`. On **F4xx/F7xx/H5xx** they are located in `hw/<famiglia>/common/handlers.c`; on **H7xx** they are defined directly in `hw/h7xx/drivers/adc_h7xx.c` (non-weak functions) to prevent the linker from using the `weak` handler of `handlers.c` (which would execute `CA_SAFE_FAIL`):

Family	IRQ handler	Source	Call
F1xx	`ADC1_2_IRQHandler`	`handlers.c`	`ca_adc_irq_handler(1)`
F4xx	`ADC_IRQHandler`	`handlers.c`	`ca_adc_irq_handler(1)`
F7xx	`ADC_IRQHandler`	`handlers.c`	`ca_adc_irq_handler(1)`
H5xx	`ADC1_IRQHandler`	`handlers.c`	`ca_adc_irq_handler(1)`
H5xx	`ADC2_IRQHandler`	`handlers.c`	`ca_adc_irq_handler(2)`
H7xx	`ADC_IRQHandler`	`adc_h7xx.c` (non-weak)	`ca_adc_hw_irq_handler(1)`
H7xx	`ADC3_IRQHandler`	`adc_h7xx.c` (non-weak)	`ca_adc_hw_irq_handler(3)`

50.1.7 Required configuration in `os_config.h`

```
#define EN_DAC         1     /* Driver DAC */
#define EN_ADC         1     /* Driver ADC */
#define ENABLE_CLI     1     /* CLI for user commands */
#define ENABLE_STDOUT  1     /* STDOUT for output */
```

To also enable USB CDC (alternative output to UART):

```
#define EN_USB_CDC     1
```

50.1.8 Technical notes

1. ADC kernel clock on H7xx — issue and solution

On STM32H7xx, the ADC kernel clock is selected by `RCC->D3CCIPR.ADCSEL` (it applies to ADC1, ADC2, and ADC3). The reset value `00` selects `pll2_p_ck`, which is not configured in the caOS `clock.c` (only PLL1 is started).

If the clock is not available, the calibration loop in the ADC init:

```
adc->CR |= ADC_CR_ADCAL;
while (adc->CR & ADC_CR_ADCAL) {}   /*     never ends */
```

never terminates, blocking the entire `ca_os_configure()`. Consequently, any peripheral initialized after the ADC (such as USB CDC) is never enabled.

Implemented solution in `hw/h7xx/drivers/adc_h7xx.c`, function `ca_clock_hw_adc_init()`:

```
/* D3CCIPR.ADCSEL = 10     per_ck (HSI 64 MHz, always available) */
RCC->D3CCIPR = (RCC->D3CCIPR & ~RCC_D3CCIPR_ADCSEL_Msk)
             | RCC_D3CCIPR_ADCSEL_1;

/* CKMODE=00 (async) + PRESC=0001 ( 2 )      ADC_CLK = 32 MHz      50 MHz      */
ADC_Common_TypeDef *cmn = (adc_num <= 2U) ? ADC12_COMMON : ADC3_COMMON;
cmn->CCR = (cmn->CCR & ~(ADC_CCR_CKMODE_Msk | ADC_CCR_PRESC_Msk))
         | ADC_CCR_PRESC_0;
```

With `per_ck = HSI = 64 MHz` (default at reset, always active) and divider `/2`, the resulting ADC clock is **32 MHz**, within the 50 MHz limit for the H753 @ VOS0.

```
HSI (64 MHz)              ADCSEL=per_ck              CCR PRESC= 2
    ADC_CLK = 32 MHz
```

2. Difference between DAC and ADC value

It is normal to observe a small difference between the value set on the DAC and the one read by the ADC:

- The DAC has 12-bit resolution over 3.3 V → 1 LSB 0.8 mV
- The ADC has 12-bit resolution over 3.3 V → 1 LSB 0.8 mV
- Both introduce an offset and non-linearity of a few LSBs

A difference of ±10 LSB (±8 mV) is typical and acceptable. The monitor task shows the raw ADC value; the comparison with the set DAC value is done via `dac info`.

3. Polling / ISR coexistence

The monitor task calls `ca_adc_read()` every 5 seconds (blocking polling). If ISR mode is active via CLI in the meantime (`adc isr 1 10`), the two modes use the same ADC: the monitor's single polling interferes with the ISR sequence. In practice, during an ISR test, one can stop the periodic log by increasing the period or simply read `adc volt 1 10` (which the driver handles by drawing from the last accumulated ISR value).

4. Pin PC0 as ADC12_INP10 channel

On STM32H7xx, the ADC channel mapping is not sequential with respect to the port pins. Channel 10 of ADC1/ADC2 physically corresponds to PC0 (ADC12_INP10), which is also pin A1 of the Nucleo-144 CN9 connector (the Arduino A1 equivalent).

The driver `adc_h7xx.c` automatically configures the pin in ANALOG mode with:

```
/* ch >= 10 && ch <= 15      CA_GPIO_PORT_C, pin (ch-10) */
ca_gpio_configure(CA_GPIO_PORT_C, (u8)(CA_GPIO_PIN_0 + (ch - 10)),
                  CA_GPIO_MODE_ANALOG, CA_GPIO_OTYPE_PP,
                  CA_GPIO_SPEED_LOW, CA_GPIO_NOPULL, 0);
```

For CH10: port C, pin 0 → PC0 .

5. ISR mode: cascaded one-shot

The ISR mode does not use the hardware automatic trigger (continuous mode) but implements a chain of single-shots. The internal callback in the `adc.c` driver immediately restarts the next conversion:

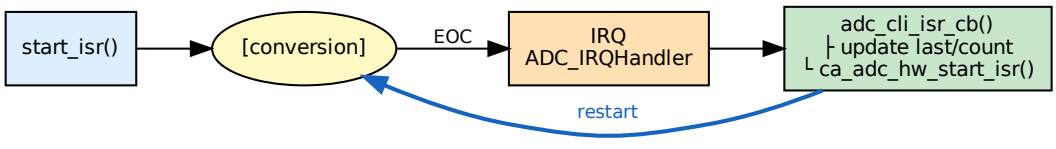


Figure 50.3: ISR mode — cascaded one-shot

This approach maximizes the sampling rate (270k samples/s on H753 at 32 MHz ADC_CLK) without requiring dedicated tasks or semaphores.

50.1.9 Summary

1. **DAC** — generates 0–3.3 V on PA4; configurable with `dac set 1 <0-4095>` (CLI driver)
2. **ADC polling** — blocking `ca_adc_read()`, `adc read 1 10` from CLI; monitor task every 5 s
3. **ADC ISR** — interrupt-driven cascaded one-shot, 270k sps; `adc isr 1 10` / `adc stop 1`
4. **ISR state in driver** — no dedicated task; `s_cli_isr_last/count` updated in IRQ
5. **H7xx IRQ fix** — non-weak `ADC_IRQHandler` in `adc_h7xx.c` (avoids SAFEGUARD FATAL)
6. **H7xx ADC clock** — `D3CCIPR.ADCSEL = per_ck`, `CCR PRESC=2` $\rightarrow$ 32 MHz essential
7. **Hardware jumper** — CN7 pin 17 (PA4) CN9 pin 3 (PC0) mandatory for loopback

Chapter 51

Real-Time Clock (Polling and ISR)

51.1 Example 28 — RTC: setting, reading, and alarms with callbacks

51.1.1 Introduction

This example demonstrates the use of the **Real Time Clock (RTC)**, showing the four main features of the caOS driver:

1. **Initialization** — clock source selection (LSI/LSE), date/time setting
2. **Reading** — acquisition of current date/time from a periodic task
3. **Alarms** — periodic Alarm A (every minute) and configurable one-shot Alarm B via CLI
4. **Calibration** — drift correction in ppm via the CALR register

The clock source depends on the board, and it is the only difference between the two:

board	32.768 kHz crystal	RTC clock	precision
NUCLEO-U575ZI-Q	mounted	`CA_RTC_CLK_LSE`	< 5 ppm
NUCLEO-H753ZI	absent (X2 not populated)	`CA_RTC_CLK_LSI`	$\pm 5\%$, temperature dependent

On the H753, the drift is therefore visible even over short periods; the `cal` command allows for partial compensation. The rest of the example is identical on both boards.

The output is transmitted via **UART** (`STDOUT_ON_UART`) and commands are sent using any serial terminal (e.g., minicom, PuTTY, screen).

Hardware

No additional connections: the RTC uses only internal resources.

Signal	Description
console UART	Board debug UART (ST-Link VCP)
RTC clock	LSE on U575, LSI on H753 — no external pin

No additional hardware connections are necessary: the RTC uses only internal resources.

CLI Commands (`rtc.c` driver)

Commands are handled directly by the driver via the CLI chain of responsibility.

Command	Description
`rtc info`	Read current date/time
`rtc set YYYY-MM-DD HH:MM:SS`	Set date/time
`rtc alarm <a|b> HH:MM:SS`	Set Alarm A or B to the specified time (one-shot)
`rtc cal <ppm>`	Calibrate RTC (e.g., `rtc cal -10`); range ±487 ppm
`rtc unix [<sec>]`	With value: set RTC from a Unix UTC time (`ca_rtc_from_net`); without: read it back (`ca_rtc_to_net`)
`help`	List commands (propagates to the chain)

STDOUT Log (every 5 s)

```
EX28: Ven 21 Mar 2026  12:00:05
```

The day of the week and the month are printed in Italian (Lun. . . Dom, Gen. . . Dic). Alarms produce additional messages when they trigger.

51.1.2 System architecture

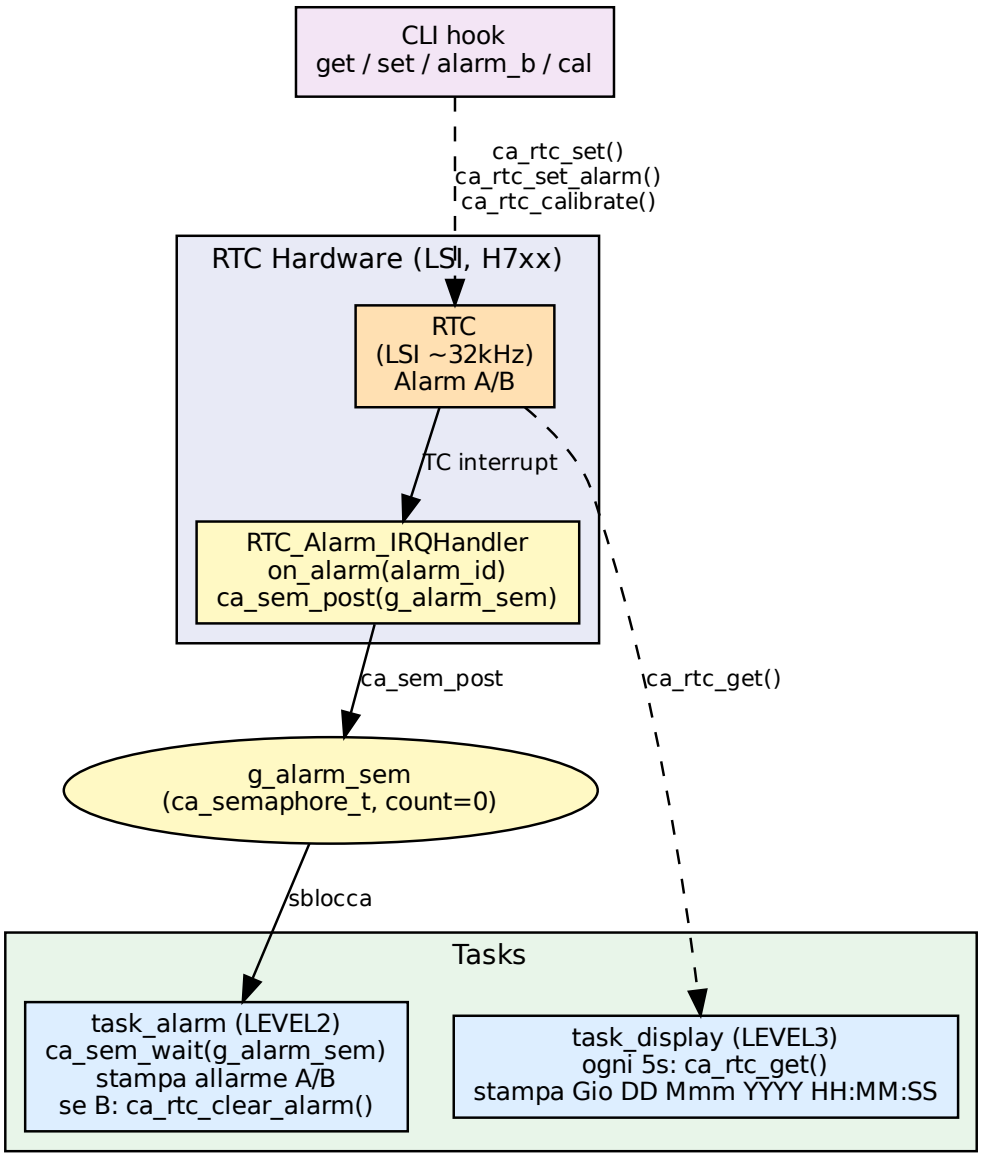


Figure 51.1: Architecture Example 28 — RTC

ISR → semaphore → task pattern: the `on_alarm()` callback is invoked directly by the `RTC_Alarm_IRQHandler` (H7xx) and only performs `ca_sem_post()`. The `task_alarm` task, awakened by the semaphore, performs all the processing (reading date/time, printing, optional disable Alarm B) outside the IRQ context.

51.1.3 Source code

The file is located in `examples/example_28/ca_os_ex28.c`:

```
// Proprietary     All rights reserved
// Copyright (c) 2026 Angelo Coppi

/*
 * Example 28     RTC: setting, reading, alarms with callbacks
 *
 * Hardware: NUCLEO-H753ZI and NUCLEO-U575ZI-Q
 * RTC Clock: CA_RTC_CLK_AUTO - the 32.768 kHz LSE crystal (every Nucleo-144
 *            carries one, H753ZI included) when it is found running at boot,
 *            else the LSI calibrated against the SysTick (less accurate,
 *            temperature-dependent: expect drift over long runs).
 *
 * Tasks:
 *   task_display      prints date/time every 5 seconds via STDOUT
 *   task_alarm        responds to alarm callbacks (via semaphore)
 *
 * Alarms:
 *   Alarm A     periodic: triggers every minute when seconds return to 0
 *   Alarm B     one-shot:  triggers at the time set via CLI, then disables
   itself
 *
 * CLI Commands (rtc.c driver):
 *   rtc info                          read current date/time
 *   rtc set YYYY-MM-DD HH:MM:SS       set date/time
 *   rtc alarm <a|b> HH:MM:SS          set alarm A or B
 *   rtc cal <ppm>                     calibrate RTC (e.g. rtc cal -10)
 *
 * STDOUT Log (every 5 s):
 *   EX28: Mon 21 Mar 2026  12:00:05
 *
 * REQUIREMENTS os_config.h:
 *   EN_RTC 1
 */

#include "caos.h"

#if EN_RTC
```

51.1.4 Detailed analysis

1. Configuration constants

```
#define INIT_YEAR   26u   /* 2026 */
#define INIT_MONTH   3u
#define INIT_DAY    21u
#define INIT_WDAY    5u   /* 1=Lun     7=Dom */
#define INIT_HOUR   12u
#define INIT_MIN     0u
#define INIT_SEC     0u
```

The initial date/time is set directly in the source code. In a real application, it should be saved in backup registers or retrieved from an external source. The year is stored as 2 digits (26 = 2026) as in the RTC BCD register.

2. Shared state

```
static ca_semaphore_t   g_alarm_sem;
static volatile u8   g_alarm_last = 0;
```

Only two variables shared between tasks and IRQs: - `g_alarm_sem` — semaphore initialized to 0; the ISR performs `post`, the task performs `wait` - `g_alarm_last` — ID of the last triggered alarm (`CA_RTC_ALARM_A` or `CA_RTC_ALARM_B`)

`g_alarm_last` is `volatile` because it is written by the IRQ and read by the task.

3. Helper: day and month names

```
static const char * const wday_str[]  = { "", "Lun", "Mar", "Mer",
                                          "Gio", "Ven", "Sab", "Dom" };
static const char * const month_str[] = { "", "Gen", "Feb", "Mar", "Apr",
                                          "Mag", "Giu", "Lug", "Ago",
                                          "Set", "Ott", "Nov", "Dic" };
```

Arrays are 1-indexed, with an empty string at position 0 as a guard against out-of-range values. `wday_str` uses the `dt.wday & 7u` index to limit access to [0..7]; `month_str` uses `dt.month <= 12u ? dt.month : 0u`.

4. Helper: `parse_u8`

```
static u32 parse_u8(const char **pp)
{
    const char *p = *pp;
    u32 v = 0;
    while (*p >= '0' && *p <= '9')
        v = v * 10u + (u32)(*p++ - '0');
    *pp = p;
    return v;
}
```

Parses a decimal number from the string pointed to by `*pp`, advancing the pointer beyond the consumed digits. Used by the `set` and `alarm_b` commands to extract HH, MM, SS, DD, MM, YY from a string with mixed separators (`:` and `/`).

5. Alarm callback

```
static void on_alarm(u8 alarm_id)
{
    g_alarm_last = alarm_id;
    ca_sem_post(&g_alarm_sem);
}
```

Invoked directly by the `RTC_Alarm_IRQHandler`. It must be very brief: it saves the alarm ID and performs a `post` on the semaphore. `ca_sem_post()` is ISR-safe in caOS (uses `BASEPRI` for the critical section, not `__disable_irq`).

6. Task display

```
static u32 task_display(void *arg)
{
    ca_rtc_dt_t dt;

    while (1) {
        if (ca_rtc_get(&dt)) {
            ca_stdout_printf_n("EX28: %s %02u %s 20%02u  %02u:%02u:%02u",
                wday_str[dt.wday & 7u],
                (unsigned)dt.day,
                month_str[dt.month <= 12u ? dt.month : 0u],
                (unsigned)dt.year,
                (unsigned)dt.hour, (unsigned)dt.min, (unsigned)dt.sec);
        } else {
            ca_stdout_put_n("EX28: RTC read error");
        }
        ca_os_sleep(5000);
```

```
    }
    return 0;
}
```

Simple loop: every 5 seconds it reads the date/time and prints it with readable names. `ca_rtc_get()` returns `false` only if the RTC is not synchronized (RSF/INITS flags not set), a condition that should not occur after `ca_rtc_set()`.

7. Task alarm

```
static u32 task_alarm(void *arg)
{
    ca_rtc_dt_t dt;

    while (1) {
        ca_sem_wait(&g_alarm_sem);

        u8 id = g_alarm_last;
        ca_rtc_get(&dt);

        if (id == CA_RTC_ALARM_A) {
            ca_stdout_printf_n("*** ALARM A ***  %02u:%02u:%02u  (every minute)"
    ,
                (unsigned)dt.hour, (unsigned)dt.min, (unsigned)dt.sec);
        } else {
            ca_stdout_printf_n("*** ALARM B ***  %02u:%02u:%02u  (one-shot,
    disabled)",
                (unsigned)dt.hour, (unsigned)dt.min, (unsigned)dt.sec);
            ca_rtc_clear_alarm(CA_RTC_ALARM_B);
        }
    }
    return 0;
}
```

The task remains blocked on `ca_sem_wait()` until an alarm triggers. Upon receiving the signal:
- **Alarm A**: prints the event; the alarm remains active (it will trigger again at the next minute)
- **Alarm B**: prints the event and calls `ca_rtc_clear_alarm()` to disable it

Reading `id` before `ca_rtc_get()` ensures that the timestamp is consistent with the alarm even if two alarms trigger in rapid succession.

8. `rtc.c` driver CLI

The RTC commands are implemented directly in the driver via the CLI **chain of responsibility**. The example does not register any custom CLI handler: `ca_rtc_init()` automatically registers the driver's CLI node.

```
ca_rtc_dt_t dt = { .year=26, .month=4, .day=21, .hour=12, .min=0, .sec=0, .wday
    =1 };
ca_rtc_set(&dt);
```

The parsing in the driver uses `ca_strtol()` on year-month-day and hour-minute-second. `dt.wday` is set to 1 as a placeholder — the RTC does not automatically calculate the day of the week.

```
ca_rtc_alarm_t al = {
    .sec = s, .min = m, .hour = h, .day = 0,
    .match_sec = true, .match_min = true, .match_hour = true, .match_day = false
};
```

```
ca_rtc_set_alarm(alarm_id, &al, rtc_cli_alarm_cb);
```

With all hour/min/sec masks at `true` and `match_day = false`, the alarm triggers when the time matches HH:MM:SS regardless of the day. The driver's default callback prints `RTC Alarm ON!`.

```
ca_rtc_calibrate(ppm);
```

Use `ca_strtol()` to parse a signed integer. Range ±487 ppm (~0.954 ppm/step resolution).

9. ca_os_configure

```
void ca_os_configure(void)
{

    EVAL_STDOUT(
        ca_stdout_init();
        ca_stdout_set_sender(STDOUT);
        ca_stdout_create_thread();
    )
    EVAL_CLI(
        ca_cli_init(2048);
        ca_cli_set_sender(STDOUT);
        ca_cli_create_thread();
    )
    EVAL_USB_CDC_STD(ca_usb_dev_mng_cdc_cli();)

    /* LSE if the 32.768 kHz crystal is found running, else LSI */
    ca_rtc_init(CA_RTC_CLK_AUTO);

    /* Set initial date/time */
    ca_rtc_dt_t dt = {
        .sec   = INIT_SEC,   .min   = INIT_MIN,   .hour  = INIT_HOUR,
        .wday  = INIT_WDAY,  .day   = INIT_DAY,
        .month = INIT_MONTH, .year  = INIT_YEAR
    };
    ca_rtc_set(&dt);

    /* Alarm A     periodic: every minute (match only seconds == 0) */
    ca_rtc_alarm_t alarm_a = {
        .sec        = 0,
        .match_sec  = true,
        .match_min  = false,
        .match_hour = false,
        .match_day  = false,
    };
    ca_rtc_set_alarm(CA_RTC_ALARM_A, &alarm_a, on_alarm);

    /* Semaphore for alarm callbacks */
    ca_sem_init(&g_alarm_sem,"EX28_S1", 0, 8);
}

void ca_os_main(void)
{
    ca_os_job("display", CA_TH_LEVEL3, task_display, NULL, STACK_SMALL,
    CA_JOB_STD, CA_FAULT_RESTART_THREAD);
    ca_os_job("alarm",   CA_TH_LEVEL2, task_alarm,   NULL, STACK_SMALL,
    CA_JOB_STD, CA_FAULT_RESTART_THREAD);


    ca_os_run(true);
```

```
}
```

The order is significant: 1. `ca_rtc_init(CA_RTC_CLK_AUTO)` probes the LSE crystal: if it oscillates, it uses it, otherwise it falls back to the (calibrated) LSI and resets the backup domain 2. `ca_rtc_set()` loads the initial date/time 3. `ca_rtc_set_alarm()` configures Alarm A **before** creating the tasks 4. `ca_sem_init()` with name `"EX28_S1"`, count=0 (blocking) and max=8 (buffer for alarm bursts) 5. tasks `display` and `alarm` are created in `ca_os_main()`, after the configuration

The tasks are created with `CA_FAULT_RESTART_THREAD` for automatic recovery in case of an exception.

51.1.5 Tasks and priorities

Task	Priority	Function
`display`	CA_TH_LEVEL3	Print date/time every 5 s
`alarm`	CA_TH_LEVEL2	Alarm semaphore consumer (high relative priority)

`task_alarm` has a higher priority than `task_display` to ensure that the alarm message is printed immediately when the semaphore is signaled, without waiting for the display task sleep.

51.1.6 Expected output

At startup

```
EX28: Ven 21 Mar 2026  12:00:05
EX28: Ven 21 Mar 2026  12:00:10
EX28: Ven 21 Mar 2026  12:00:15
...
```

`rtc info`

```
RTC  2026-03-21  12:00:42
```

`rtc set 2026-03-22 14:30:00`

```
RTC set 2026-03-22 14:30:00
```

`rtc alarm b 14:31:00`

```
rtc alarm B set 14:31:00
```

When the alarm triggers:

```
RTC Alarm ON!
*** ALARM B ***  14:31:00  (one-shot, disabled)
```

Alarm A (every minute)

```
*** ALARM A ***  14:32:00  (every minute)
*** ALARM A ***  14:33:00  (every minute)
```

cal -20

```
Calibrazione: -20 ppm applicata.
```

Typical test sequence

```
> rtc info
RTC  2026-03-21  12:05:33

> rtc set 2026-03-21 12:06:00
RTC set 2026-03-21 12:06:00

> rtc alarm b 12:07:00
rtc alarm B set 12:07:00

EX28: Ven 21 Mar 2026  12:06:05
EX28: Ven 21 Mar 2026  12:06:10
...
RTC Alarm ON!
*** ALARM B ***  12:07:00  (one-shot, disabled)
EX28: Ven 21 Mar 2026  12:07:05
...
*** ALARM A ***  12:08:00  (every minute)
*** ALARM A ***  12:09:00  (every minute)
```

51.1.7 RTC driver architecture

Public API (caos/drivers/inc/rtc.h)

```c
/* Clock source */
typedef enum { CA_RTC_CLK_LSE = 1, CA_RTC_CLK_LSI, CA_RTC_CLK_HSE }
    ca_rtc_clk_src_t;

/* Date/time */
typedef struct {
    u8 sec, min, hour;    /* 0-59, 0-59, 0-23 */
    u8 wday;              /* 1= L u n 7 =Dom       */
    u8 day, month;        /* 1-31, 1-12        */
    u8 year;              /* 0-99 (offset 2000)*/
} ca_rtc_dt_t;

/* Alarm structure */
typedef struct {
    u8   sec, min, hour, day;
    bool match_sec, match_min, match_hour, match_day;
} ca_rtc_alarm_t;

typedef void (*ca_rtc_alarm_cb_t)(u8 alarm_id);

```

```
/* Alarm constants */
#define CA_RTC_ALARM_A   1u
#define CA_RTC_ALARM_B   2u

/* API */
bool ca_rtc_init(ca_rtc_clk_src_t src);
bool ca_rtc_set(const ca_rtc_dt_t *dt);
bool ca_rtc_get(ca_rtc_dt_t *dt);
bool ca_rtc_calibrate(i32 ppm);
bool ca_rtc_set_alarm(u8 alarm_id, const ca_rtc_alarm_t *al, ca_rtc_alarm_cb_t
    cb);
bool ca_rtc_clear_alarm(u8 alarm_id);
```

IRQ → semaphore → task flow

```
RTC_Alarm_IRQHandler()        [hw/h7xx/common/rtc_h7xx.c]
         ISR & ALRAF?      chiama g_alarm_cb[0](CA_RTC_ALARM_A)
                               on_alarm(CA_RTC_ALARM_A)
                                     g_alarm_last = CA_RTC_ALARM_A
                                     ca_sem_post(&g_alarm_sem)

         ISR & ALRBF?      chiama g_alarm_cb[1](CA_RTC_ALARM_B)
                             on_alarm(CA_RTC_ALARM_B)
                                   g_alarm_last = CA_RTC_ALARM_B
                                   ca_sem_post(&g_alarm_sem)

task_alarm (risvegliato da sem_wait)
         legge g_alarm_last e ca_rtc_get()
         prints the alarm message
         se Alarm B: ca_rtc_clear_alarm(CA_RTC_ALARM_B)
```

ALRMAR register configuration (Alarm A — every minute)

```
ALRMAR bits:
  MSK4 [31]   = 1      ignora giorno
  MSK3 [23]   = 1      ignore hours
  MSK2 [15]   = 1      ignora minuti
  MSK1 [ 7]   = 0      confronta secondi
  ST   [6:4] = 0       secondi decine BCD = 0
  SU   [3:0] = 0       secondi unit  BCD = 0
```

With all masks set to 1 except MSK1, the alarm triggers every time the seconds are 0 — that is, at the beginning of every minute, regardless of the hour and day.

Calibration — CALR register

The RTC CALR register (common to F4/F7/H5/H7) allows for correcting the RTC clock frequency within a range of ±487 ppm:

```
CALP [15]    = 1      adds 512 pulses every 32 s  (+15.9 ppm)
CALM [8:0]   = N      subtracts N pulses every 32 s     (step ~0.954 ppm)
```

The `ca_rtc_calibrate(ppm)` driver automatically calculates CALP and CALM from the value in ppm:

requested ppm	CALP	CALM	actual ppm
0	0	0	0.000
-10	0	10	-9.537
+5	1	11	+5.126
-487	0	511	-487.137
+15	1	0	+15.954

51.1.8 Technical notes

1. LSI vs LSE — depends on the board

It is not true that "there is no crystal on Nucleo boards": it depends on the board, and that is the only reason why this example has a `#if MCU_ID` block. The NUCLEO-H753ZI (MB1364) does not mount the X2 footprint (32.768 kHz), the **NUCLEO-U575ZI-Q does**. The driver supports both sources:

Source	Frequency	Precision	PRER (async/sync)	Notes
LSE	32.768 kHz	<5 ppm	127/255	External crystal — **present on U575ZI-Q**
LSI	~32 kHz	±5%	127/249	Internal, always available — used on H753ZI
HSE/128	configurable	same as HSE	variable	Rare, for special applications

With LSI and `PRER = (127, 249)`, the resulting frequency is:

```
f_RTC = 32000 / (128    250) = 1.000 Hz (ideale)
```

In practice, the LSI varies between ~28 and ~34 kHz depending on the temperature and the chip batch, causing an accumulated drift of several seconds per day. The `cal` command allows for partial compensation of this drift.

2. Differences between families — RTC registers

The caOS driver abstracts the differences between families through the implementations in `hw/<family>/common/rtc_<family>.c`:

Family	Init/status reg	Alarm status	Alarm clear	IRQ handler	EXTI line 17
F4xx	`RTC->ISR`	`ISR.ALRAF`	`ISR &= ~flag`	`RTC_Alarm_IRQHandler`	`EXTI->IMR/RTSR`
F7xx	`RTC->ISR`	`ISR.ALRAF`	`ISR &= ~flag`	`RTC_Alarm_IRQHandler`	`EXTI->IMR/RTSR`
H7xx	`RTC->ISR`	`ISR.ALRAF`	`ISR &= ~flag`	`RTC_Alarm_IRQHandler`	`EXTI_D1->IMR1 / EXTI->RTSR1`
H5xx	`RTC->ICSR`	`RTC->SR`	`RTC->SCR`	`RTC_IRQHandler`	Not necessary (direct IRQ)

The most relevant differences: - **H5xx**: uses `RTC->ICSR` (Init Control/Status Register) instead of `RTC->ISR`; alarm clearing is performed by writing to the write-only register `RTC->SCR` - **H7xx**:

requires `EXTI_D1->IMR1` (CPU1 domain) instead of the simple `EXTI->IMR` - **F4/F7**: use `PWR->CR |= PWR_CR_DBP` to unlock the backup domain; H7 uses `PWR->CR1 |= PWR_CR1_DBP`; H5 uses `PWR->DBPCR |= PWR_DBPCR_DBP`

3. RTC Write Protection (Write Protection Register)

Before any write to the RTC registers (PRER, TR, DR, ALRMAR, ALRMBR, CALR), it is necessary to disable the hardware protection with the sequence:

```
RTC->WPR = 0xCA;   /* key 1 */
RTC->WPR = 0x53;   /* key 2 */
/* ... register writes ... */
RTC->WPR = 0xFF;   /* re-enable protection */
```

An error in the sequence leaves the register protected and the write is silently ignored. The driver handles this protection internally.

4. INIT mode

Modifying TR, DR, and PRER requires entering INIT mode:

```
RTC->ISR |= RTC_ISR_INIT;              /* request INIT */
while (!(RTC->ISR & RTC_ISR_INITF)) {} /* wait for INITF */
/* ... write PRER, TR, DR ... */
RTC->ISR &= ~RTC_ISR_INIT;            /* leave INIT */
```

In INIT mode, the RTC counter stops; upon completion, it resumes from the set value. Synchronization with the calendar is immediate.

5. Alarm B one-shot — behavior upon restart

Alarm B is configured via CLI and is not saved in backup registers. If the board is restarted after setting Alarm B but before it triggers, the alarm is lost. In a real-world application, the configuration should be saved in one of the 4 RTC backup registers (RTC->BKPxR) and restored at startup.

51.1.9 Required configuration in `os_config.h`

```
#define EN_RTC          1    /* Driver RTC */
#define EN_CLI_RTC      1    /* Driver 'rtc *' commands */
#define ENABLE_CLI      1    /* CLI for user commands */
#define ENABLE_STDOUT   1    /* STDOUT for output */
#define STDOUT_ON_UART  1    /* Console on UART, not on USB CDC */
#define STDIN_ON_UART   1
```

The example is conditioned by the `EN_RTC` flag:

```
#if EN_RTC
/* ... all the example code ... */
#else /* EN_RTC == 0 */

/* Pre-scheduler diagnostic: ca_stdout_* needs a task context (TLB=NULL before
 * ca_os_run), so print straight to the console backend with PUT_N. */
PRIVATE_STRING(m28a, "=== Example 28 ===");
PRIVATE_STRING(m28b, "EN_RTC == 0, skipping RTC example");
void ca_os_configure(void)
```

```
{
    /* The line the bench matches to know this example cannot run on
       this part. The reason follows it. */
    CA_EXAMPLE_UNSUPPORTED();
    PUT_N(&priv_m28a);
    PUT_N(&priv_m28b);
}
void ca_os_main(void) { ca_os_run(true); }

#endif /* EN_RTC */
```

51.1.10 Summary

1. **Initialization** — `ca_rtc_init()` with the board source (LSE on U575, LSI on H753) resets the backup domain and sets the prescalers for 1 Hz
2. **Set date/time** — `ca_rtc_set(&dt)` enters INIT mode, writes TR/DR in BCD, exits
3. **Get date/time** — `ca_rtc_get(&dt)` reads TR/DR and converts from BCD; waits for RSF if necessary
4. **Periodic Alarm A** — match only on seconds (=0): triggers at the beginning of every minute
5. **One-shot Alarm B** — match on HH:MM:SS; the task disables it via `ca_rtc_clear_alarm()` after the first trigger
6. **ISR→sem→task pattern** — the callback is minimal (post semaphore); all processing occurs in the task
7. **Calibration** — `ca_rtc_calibrate(ppm)` writes CALR; range ±487 ppm; useful for compensating LSI drift
8. **H5/H7/F4/F7 differences** — completely hidden in the hardware-specific implementation

Chapter 52

SPI Master and Slave (Polling and ISR)

52.1 Example 29 — SPI master-slave on the same MCU

52.1.1 Introduction

This example demonstrates the caOS **SPI** driver on a NUCLEO-H753ZI with two SPI peripherals on the same MCU connected via wires: **SPI1 as master** and **SPI3 as slave**. The example covers three operating modes:

1. **Loopback** (`EN_SPI_LOOPBACK`) — verifies the master in isolation with a single MOSI→MISO wire
2. **Polling** (`EN_SPI_EX_POLL`) — master and slave on separate tasks, handshake via volatile flags
3. **IRQ** — the slave accumulates bytes in interrupt and calls a callback upon frame completion

Hardware (NUCLEO-H753ZI, MB1364)

SPI	SCK Pin	MISO Pin	MOSI Pin	Bus	Role
SPI1	PA5 (CN7-10)	PA6 (CN7-12)	PB5 (CN7-14)	APB2	Master
SPI3	PC10 (CN7-1)	PC11 (CN7-5)	PC12 (CN7-3)	APB1L	Slave

PB5 Note: PA7 is shared with RMII_CRS_DV (Ethernet, SB31 ON on H753ZI) and is not available. PB5 is used as an alternative MOSI for SPI1 (AF5, CN7-14).

Wiring for polling/IRQ mode (3 wires):

```
PA5 (CN7-10) SCK              PC10 (CN7-1)  SCK  SPI3
PB5 (CN7-14) MOSI             PC12 (CN7-3)  MOSI SPI3
PA6 (CN7-12) MISO             PC11 (CN7-5)  MISO SPI3
```

Wiring for loopback test (1 wire):

```
PB5 (CN7-14) MOSI             PA6 (CN7-12) MISO   (adiacenti su CN7)
```

NSS software on both peripherals (SSM=1): no CS pin required.

52.1.2 Source configuration

The three modes are selected with `#define` at the top of the file `examples/example_29/ca_os_ex29.c`:

```
/* Uncomment to switch to polling mode */
/* #define EN_SPI_EX_POLL */

/* Define for the master loopback test: PB5(MOSI)     PA6(MISO). */
/* #define EN_SPI_LOOPBACK */
```

These are **example-local** macros, not `os_config.h` flags: they are enabled by uncommenting them in the source code. In the file, both are commented out, so the default mode is **IRQ**.

Active define	Mode	Tasks started
`EN_SPI_LOOPBACK`	Loopback master	only `task_master`
`EN_SPI_EX_POLL`	Polling master+slave	`task_master` + `task_slave_poll`
none (default)	IRQ slave callback	only `task_master` + SPI3 ISR

In `os_config.h` it is necessary to:

```
#define EN_SPI    1
```

52.1.3 System architecture

Polling mode

```
task_master                          task_slave_poll

      g_slave_ready = false
                                  g_slave_ready = true
                                  ca_spi_read_buf(SPI3, ...)      blocca
      while (!g_slave_ready) yield()
      ca_spi_write_buf(SPI1, g_tx_buf, 8)
                8    ca_spi_xfer(SPI1, byte)
                      SCK genera clock     SPI3 riceve
                                              ritorna con 8 byte
                                  g_slave_ready = false
                                  verifica pattern
      ca_os_sleep(500)            ca_os_sleep(200)
```

IRQ Mode

```
task_master                          SPI3_IRQHandler

      ca_spi_write_buf(SPI1, g_tx_buf, N)
                N    ca_spi_xfer                  RXPIE     byte     s_rx_buf
                                               s_rx_count == frame_size?
                                                 slave_irq_cb(num, buf, len)
      ca_os_sleep(1000)                              verifica + stampa
```

52.1.4 Source code

The file is located in `examples/example_29/ca_os_ex29.c`.

1. Configuration constants

```
#define POLL_FRAME_SIZE    8U          /* 8 bytes: fits the H7 FIFO (16 bytes) */
#define IRQ_FRAME_SIZE     SPI_DEF_PKT_SIZE   /* 128 byte */
#define MASTER_SPI         1U          /* SPI1 */
#define SLAVE_SPI          3U          /* SPI3 */
```

`POLL_FRAME_SIZE` is intentionally small: the H753 has a 16-byte TX FIFO, so 8 bytes fit entirely into the FIFO before the master stalls waiting for TXP. In this way, byte-level synchronization between master and slave is not required.

2. Shared state

```
/* Polling mode only */
static volatile bool g_slave_ready = false;
static u8 g_rx_poll[POLL_FRAME_SIZE];

/* Loopback mode only */
static u8 g_rx_loopback[POLL_FRAME_SIZE];

static u8 g_tx_buf[SPI_DEF_PKT_SIZE];  /* buffer TX master */
```

The `g_slave_ready` flag is `volatile` because it is written by the slave task and read by the master task without explicit synchronization mechanisms. The correct sequence is: the slave sets the flag **before** entering `ca_spi_read_buf`; the master waits for the flag with `ca_os_yield()` before calling `ca_spi_write_buf`.

3. Slave IRQ callback

```
static void slave_irq_cb(u8 spi_num, const u8 *data, u16 len)
{
    u16 errors = 0U;
    for (u16 i = 0; i < len; i++)
        if (data[i] != (u8)(i & 0xFFU)) errors++;
    g_frame_count++;
    if (errors == 0U)
        ca_stdout_printf_n("EX29 IRQ: frame %u OK (%u byte)", ...);
    else
        ca_stdout_printf_n("EX29 IRQ: frame %u ERRORS=%u", ...);
}
```

The callback is invoked by the SPI3 IRQ handler after accumulating `frame_size` bytes in the driver's internal buffer. Execution occurs in an IRQ context: it must be brief and non-blocking.

4. Master task — loopback mode

```
#ifdef EN_SPI_LOOPBACK
    while (1) {
        u8 errors = 0U;
        for (u8 i = 0; i < POLL_FRAME_SIZE; i++) {
            u8 rx = ca_spi_xfer(MASTER_SPI, i);
            g_rx_loopback[i] = rx;
            if (rx != i) errors++;
        }
        frame++;
        if (errors == 0U)
            ca_stdout_printf_n("EX29 LOOPBACK: frame %u OK", (unsigned)frame);
        else
            ca_stdout_printf_n("EX29 LOOPBACK: frame %u ERR=%u", ...);
        ca_os_sleep(500);
    }
```

In loopback, `ca_spi_xfer` sends the byte `i` on MOSI (PB5) and reads the byte received on MISO (PA6). With the wire PB5→PA6, the received byte is identical to the transmitted one: `rx == i` for all bytes.

5. Master task — polling mode

```
#elif defined(EN_SPI_EX_POLL)
    while (1) {
        while (!g_slave_ready)
            ca_os_yield();          /* yields the CPU until the slave is ready
    */

        for (u8 i = 0; i < POLL_FRAME_SIZE; i++)
            g_tx_buf[i] = i;

        ca_spi_write_buf(MASTER_SPI, g_tx_buf, POLL_FRAME_SIZE);
        frame++;
        ca_stdout_printf_n("EX29 POLL M: frame %u TX OK", (unsigned)frame);
        ca_os_sleep(500);
    }
```

6. Task slave polling

```
    while (1) {
        g_slave_ready = true;                                   /* signal ready */
        ca_spi_read_buf(SLAVE_SPI, g_rx_poll, POLL_FRAME_SIZE); /* blocca */
        g_slave_ready = false;

        u8 errors = 0U;
        for (u8 i = 0; i < POLL_FRAME_SIZE; i++)
            if (g_rx_poll[i] != i) errors++;
        frame++;
        if (errors == 0U)
            ca_stdout_printf_n("EX29 POLL S: frame %u OK", ...);
        else
            ca_stdout_printf_n("EX29 POLL S: frame %u ERR=%u", ...);
        ca_os_sleep(200);
    }
```

7. ca_os_configure

```
void ca_os_configure(void)
{
    /* ... stdout, log, cli, usb ... */

    /* Init SPI1     MASTER (no external CS: software NSS) */
    ca_spi_config_t m_cfg = {
        .spi_num    = MASTER_SPI,
        .br         = CA_SPI_BR_DIV64,   /* APB2/64     1.87 MHz su H753 @480MHz
     */
        .cpol       = 0, .cpha = 0,
        .master     = true,
        .frame_size = 0, .on_rx = NULL,
        .manage_cs  = false,          /* CS not managed: slave always selected (
    SSM=1) */
    };
    ca_spi_init(&m_cfg);

#ifndef EN_SPI_LOOPBACK
    /* Init SPI3     SLAVE */
    ca_spi_config_t s_cfg = {
        .spi_num    = SLAVE_SPI,
        .br         = 0,              /* slave: baud ignored */
        .cpol       = 0, .cpha = 0,
        .master     = false,
```

```
        .manage_cs  = false,
#ifdef EN_SPI_EX_POLL
        .frame_size = POLL_FRAME_SIZE,
        .on_rx      = NULL,           /* polling */
#else
        .frame_size = IRQ_FRAME_SIZE,
        .on_rx      = slave_irq_cb,   /* IRQ */
#endif
    };
    ca_spi_init(&s_cfg);
#endif
}
```

In loopback mode, SPI3 is not initialized. In IRQ mode, passing `on_rx != NULL` automatically enables `RXPIE` and the NVIC in the driver.

52.1.5 Technical notes — MODF on STM32H7 SPI v3

The STM32H7 uses version 3 of the SPI controller (CFG1/CFG2 registers, separate TXDR/RXDR, endless mode TSIZE=0). This peripheral exhibits critical behavior regarding the **MODF (Mode Fault)**:

Cause

In master mode with SSM=1, MODF is triggered when SSI=0. It can be generated by a previous run that left the flag dirty in the SR register, or by a race condition during peripheral reinitialization.

Effect

MODF clears **both** `CR1.SPE` **and** `CFG2.MASTER`. If not handled before the master/slave dispatch in the driver, the `ca_spi_hw_xfer` function reads `CFG2.MASTER == 0` and enters the slave branch, where it hangs waiting for `SPI_SR_RXP` which will never arrive (deadlock).

Solution implemented in caOS

```
/* ca_spi_hw_xfer     spi_h7xx.c / spi_h5xx.c */

/* 1. MODF recovery BEFORE the master/slave dispatch */
if ((spi->SR & SPI_SR_MODF) || !(spi->CR1 & SPI_CR1_SPE)) {
    (void)spi->SR;               /* step 1: read SR (unlock) */
    spi->IFCR  = (1U << 9U);   /* step 2: clear MODFC      */
    spi->CFG2 |= SPI_CFG2_MASTER; /* step 3: restore MASTER */
    spi->CR1   = SPI_CR1_SSI | SPI_CR1_SPE;
}

/* 2. Master/slave dispatch after the recovery */
if (spi->CFG2 & SPI_CFG2_MASTER) {
    spi->CR1 |= SPI_CR1_CSTART;  /* avvia burst (non settato in init) */
    /* ... TXP wait     write TXDR     RXP wait con SPE check ... */
}
```

CSTART is not set during init: it is set in every call to `ca_spi_hw_xfer`. This prevents the peripheral in endless mode from generating MODF during the idle period between frames (master task sleep).

MODF-clear procedure in init (H7/H5)

```
spi->CR1 = 0;
(void)spi->SR;              /* step 1 */
spi->IFCR = (1U << 9U);  /* step 2: MODFC */
spi->CFG1 = ...;
spi->CFG2 = SPI_CFG2_MASTER | SPI_CFG2_SSM | ...;  /* step 3 */
spi->CR2  = 0;
spi->CR1  = SPI_CR1_SSI | SPI_CR1_SPE;   /* no CSTART */
```

Differences by family

Family	SPI ver	MODF effect	Recovery
F1/F4xx	v1 (DR/SR)	clears only SPE	simple: SPE=1
F7xx	v2 (CR2 DS+FRXTH)	clears only SPE	simple: SPE=1
H5/H7xx	v3 (CFG1/CFG2)	clears SPE **and** MASTER	read SR + IFCR.MODFC + write CFG2

52.1.6 Expected output

Loopback (`EN_SPI_LOOPBACK` + wire PB5→PA6)

```
EX29 LOOPBACK: frame 1 OK
EX29 LOOPBACK: frame 2 OK
EX29 LOOPBACK: frame 3 OK
```

Polling (`EN_SPI_EX_POLL` + 3 wires)

```
EX29 POLL M: frame 1 TX OK
EX29 POLL S: frame 1 OK
EX29 POLL M: frame 2 TX OK
EX29 POLL S: frame 2 OK
```

IRQ (no define + 3 wires)

```
EX29 IRQ: frame 1 OK (128 byte)
EX29 IRQ M: frame 1 TX OK
EX29 IRQ: frame 2 OK (128 byte)
EX29 IRQ M: frame 2 TX OK
```

52.1.7 CLI Commands

```
ex29 wire       continuity of the 3 jumpers (SCK, MOSI, MISO), verdict WIRE
    RESULT : OK|FAIL (n/3)
ex29 info       the three links expected on this board and the LED pin
```

`wire` first **parks** the SPI tasks (waits ~1.2 s for them to finish their cycle) and stops the two peripherals, then tests each line on pure GPIO, driving it from the side that drives it during normal operation: master → slave for SCK and MOSI, slave → master for MISO. Driving the same side always resulted in a MISO FAIL with the wires connected correctly (two drivers on the same line). If the peripheral does not stop, `wire` reports it (`WARNING: spi_stop ...`) instead of making it look like a missing wire. At the end, the pins remain GPIO, so the board **restarts itself** after 2.5 s.

A heartbeat task blinks **LED 1** (pin from `objmap.h`), on a port already used by the example: if it blinks, the port clock, GPIO, and scheduler are working, and a `wire` FAIL concerns the pins, not the GPIO.

52.1.8 Tasks and priorities

Task	Priority	Mode	Function
`master`	CA_TH_LEVEL3	all	Sends SPI frame
`slave`	CA_TH_LEVEL3	polling	Receives and verifies frame
—	ISR SPI3	IRQ	Accumulates bytes + callback

In polling mode, master and slave have the same priority: the `g_slave_ready` + `ca_os_yield()` mechanism ensures that the slave enters reception before the master starts transmitting.

52.1.9 SPI APIs used

```
/* Inizializzazione */
bool ca_spi_init(const ca_spi_config_t *cfg);

/* CS manuale (no-op se manage_cs == false) */
void ca_spi_cs_assert(u8 spi_num);
void ca_spi_cs_deassert(u8 spi_num);

/* Trasferimento singolo byte full-duplex */
u8 ca_spi_xfer(u8 spi_num, u8 data);

/* Invio buffer (scarta RX) */
u16 ca_spi_write_buf(u8 spi_num, const u8 *buf, u16 len);  /* bytes sent; < len
    on timeout */

/* Ricezione buffer polling (invia 0x00) */
u16 ca_spi_read_buf(u8 spi_num, u8 *buf, u16 len);   /* bytes received; < len on
     timeout */

/* Called by the IRQ handler (slave IRQ mode) */
void ca_spi_recv_handler(u8 spi_num);
```

52.1.10 Required configuration in `os_config.h`

```
#define EN_SPI    1    /* Driver SPI */
```

The example is conditioned by `EN_SPI`:

```
#if EN_SPI
/* ... all the code ... */
#else
void ca_os_configure(void) {}
void ca_os_main(void) { ca_os_run(true); }
#endif
```

52.1.11 Summary

1. **Loopback** — a PB5→PA6 wire verifies the master in isolation; `ca_spi_xfer` returns the sent byte
2. **Polling** — `g_slave_ready` handshake ensures the slave is waiting before the master transmits

3. **IRQ** — `on_rx != NULL` in `ca_spi_config_t` automatically enables RXPIE and NVIC; the callback receives complete frames
4. **MODF H7/H5** — recovery before master/slave dispatch; per-xfer CSTART avoids idle MODF
5. **Non-DMA Pattern** — suitable for frames up to ~32 bytes at reduced baud rate; for large frames or high clock speeds, consider DMA

Chapter 53

EXTI: Interrupt from the User Button

53.1 Example 30 — EXTI: interrupt from user button

53.1.1 Introduction

This example demonstrates the caOS **EXTI** driver on NUCLEO-H753ZI. The goal is to handle the **B1** user button press via interrupt on the RISING edge, showing the interaction between ISR, tasks, and CLI.

The example adopts a **progressive debugging** strategy:

1. At startup, EXTI is disabled: first, it is verified that GPIO polling reads the button correctly
2. The `exti_on` command from CLI enables the interrupt
3. The ISR callback filters bounces and counts presses; `task_monitor` reports them

Hardware (NUCLEO-H753ZI, MB1364)

Signal	MCU Pin	Connector	Description
B1 USER	PC13	—	Blue button, external pull-down, press = HIGH
LED1 (green)	PB0	—	Toggle on every press via EXTI
LED2 (yellow)	PE1	—	Toggle every 5 presses
LED3 (red)	PB14	—	Heartbeat 500 ms

The LEDs are driven with `ca_led_toggle(1..3)`: the pins come from `objmap.h` of the board, not from the example. The button changes with the board (`BTN_TXT`): PC13 on H753 and U575, PA0 (WK_UP) on the F407.

> **Polarity note**: the B1 button on the Nucleo-H753ZI has an external pull-down resistor. At rest PC13=0; when pressed PC13=1. The interrupt is configured on **RISING edge**.

53.1.2 Required configuration in `os_config.h`

```
#define EN_EXTI       1    /* Driver EXTI */
#define ENABLE_CLI    1    /* Comando exti_on / exti_off / btn */
#define EN_TAMP       1    /* Tamper sullo stesso pulsante (solo versione a
    pagamento) */
#define EN_RTC        1    /* Clock dell'RTC: serve al tamper su F446, F767, H753
    */
```

With `EN_TAMP = 0` (FREE package) the example is the usual one: the `tamper` commands are not compiled.

53.1.3 System architecture

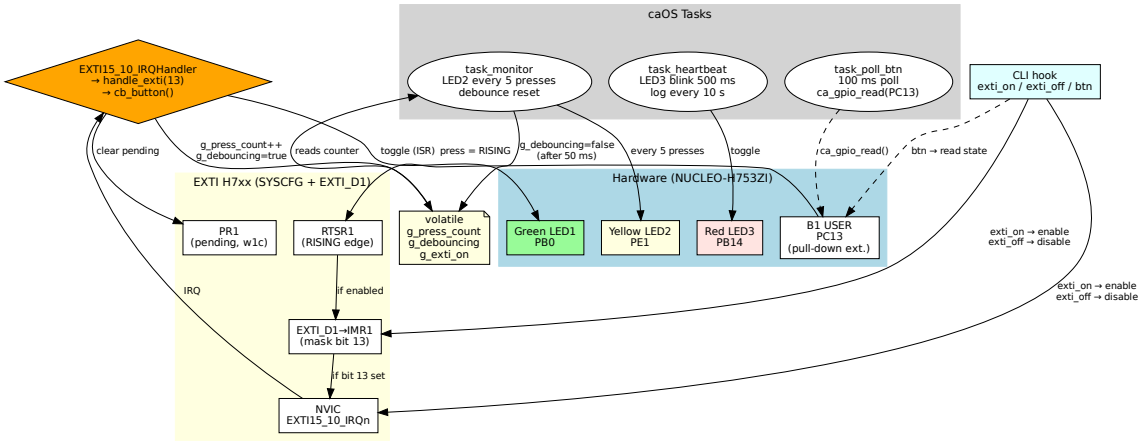


Figure 53.1: Example architecture 30

The flow consists of three levels:

```
B1 (PC13)        RISING       > EXTI15_10_IRQHandler     cb_button()
                                   scarta l'impulso se < DEBOUNCE_MS dall'
    ultimo (ISR)
                                   ca_led_toggle(1)
       (ISR)
                                   g_press_count++
       (ISR)

task_poll_btn         100ms       > init LED/B1/EXTI, poi ca_gpio_read(B1)     log
     cambi stato
task_monitor            10ms        > legge g_press_count, ogni 5 press
    ca_led_toggle(2)
task_heartbeat        500ms       > ca_led_toggle(3) + log ogni 10 s

nodo CLI             exti_on       ca_exti_enable(BTN_PIN)  (IMR + NVIC)
                     exti_off      ca_exti_disable(BTN_PIN)
                     btn           ca_gpio_read(B1) + stampa
```

53.1.4 Software debounce

Debouncing is performed in the callback, **by timestamp**: an edge arriving less than `DEBOUNCE_MS` (50 ms) after the last accepted one is discarded. The window starts from the pulse itself, without a flag that a task must then reset.

```
press                                                              rimbalzi <
    50 ms dal primo fronte: scartati

B1

                       g_last_edge_ms = now   (ISR, fronte accettato)
```

The previous version raised a flag and left the task of resetting it after 50 ms to `task_monitor`; however, the task checks every 10 ms, so the window started late and ended early. Longer bounces were counted as actual presses: on the H563, a single press registered two or three.

Why not use `ca_exti_disable()` for debouncing? `ca_exti_disable()` also disables the NVIC, and the subsequent `ca_exti_enable()` must re-enable both the IMR and the NVIC. The time comparison in the callback is simpler and does not touch the hardware.

53.1.5 Source code

The file is located in `examples/example_30/ca_os_ex30.c`.

1. Constants and shared state

```
#define DEBOUNCE_MS     50U
#define LED2_THRESHOLD  5U

static volatile u32  g_press_count = 0U;
static volatile u32  g_last_edge_ms = (u32)0 - DEBOUNCE_MS;
static volatile bool g_exti_on     = false;   /* EXTI enabled? */
static volatile bool g_hw_ready    = false;
```

Variables shared between ISRs and tasks are `volatile`. A mutex is not needed: the counter is only incremented by the ISR and only read by tasks. `g_last_edge_ms` starts a window in the past, so the first press is accepted immediately. `g_hw_ready` raises it `task_poll_btn` when the button, LED, and EXTI line are ready: other tasks touch the LEDs and wait for it, because the task creation order does not guarantee the execution order.

2. EXTI Callback (ISR context)

```
static void cb_button(void)
{
    /* Debounce by timestamp, not by a flag another task has to clear: the
     * bounce window is measured from THIS edge. The previous version raised a
     * flag and let task_monitor clear it DEBOUNCE_MS after it noticed - and it
     * only looks every 10 ms, so the window started late and closed early.
     * Contact bounces outliving it were counted as real presses: on the H563
     * a single press could log two or three. */
    u32 now = ca_os_tick_ms();

    if ((u32)(now - g_last_edge_ms) < DEBOUNCE_MS)
        return;                 /* still inside the bounce window */

    g_last_edge_ms = now;
    ca_led_toggle(1U);
    g_press_count++;
}
```

The callback is invoked by `EXTI15_10_IRQHandler` via the driver dispatch. It must be brief: no blocking or sleep calls.

3. CLI Node — user commands

```
static bool user_cli_handler(c_str *pcmd)
{
    const char *p = ca_skip_sp((*pcmd)->data);
    if (ca_strncmp(p, "help", 4) == 0) {
        CLI_PUT_N("exti_on | exti_off | btn");
        return false;
    }
```

```
    if (ca_strncmp(p, "exti_on", 7) == 0) {
        /* task_poll_btn owns the EXTI setup: enabling the line before it ran
         * would arm a callback the driver has not configured yet. */
        if (!g_hw_ready) {
            CLI_PUT_N("EX30: hardware not ready yet, retry in a moment");
            return true;
        }
        g_last_edge_ms = ca_os_tick_ms() - DEBOUNCE_MS;  /* accept the next edge */
        ca_exti_enable(BTN_PIN);
        g_exti_on = true;
        CLI_PUT_N("EX30: EXTI enabled (RISING edge on the B1 pin)");
        return true;
    }
    if (ca_strncmp(p, "exti_off", 8) == 0) {
        ca_exti_disable(BTN_PIN);
        g_exti_on = false;
        CLI_PUT_N("EX30: EXTI disabled");
        return true;
    }
    if (ca_strncmp(p, "btn", 3) == 0) {
        u32 v = ca_gpio_read(BTN_PORT, BTN_PIN);
        CLI_PRINTF_N("EX30: " BTN_TXT "=%u (%s)", v, v ? "HIGH(pressed)" : "LOW(released)");
        return true;
    }
    return false;
}
static ca_cli_node_t s_user_cli_node = { user_cli_handler, NULL };
```

The handler returns `true` if it has handled the command, `false` to pass it to the next node. On `help` it adds its own line and returns `false`, so the chain continues. `exti_on` refuses to start until `task_poll_btn` has configured the line: enabling it beforehand would arm a callback that the driver has not yet set.

4. GPIO polling task (and hardware init)

```
static u32 task_poll_btn(void *arg)
{
    (void)arg;

    u32 prev = 2U;   // impossible value     force log on first cycle

    /* Hardware init lives here rather than in ca_os_configure: with a task
     * context ca_stdout_* works, so the setup and the messages that describe it
     * stay together. LEDs come from objmap.h through the driver. */
    ca_led_init();

    /* B1: input, no internal pull - the board provides the network.
     * On the U575 the button pin must NEVER be driven as OUTPUT/LOW: UM2861
     * warns it shorts the button network. Input is what we want anyway. */
    ca_gpio_configure(BTN_PORT, BTN_PIN,
                      CA_GPIO_MODE_INPUT, CA_GPIO_OTYPE_PP,
                      CA_GPIO_SPEED_LOW, CA_GPIO_NOPULL, 0U);

    /* RISING edge, but the line starts DISABLED: phase 1 is polling only. */
    ca_exti_init(BTN_PORT, BTN_PIN, CA_EXTI_EDGE_RISING, cb_button);
    ca_exti_disable(BTN_PIN);
    g_exti_on = false;

    g_hw_ready = true;

```

```
    /* Let ca_os_banner() finish first: it is a task too, at CA_TH_LEVEL6, and it
     * paces itself with sleeps to let the UART drain. This task runs at LEVEL3
     * and would otherwise print straight through the middle of it. The hardware
     * above is already up, so the other tasks are not held back by this wait.
     */
    ca_os_sleep(1000);

    ca_stdout_printf_n("EX30: started - EXTI OFF");

    /* Phase 1 is only half the example: polling proves the pin is read, it says
     * nothing about the EXTI path. Say so out loud, because a run that stops
     * here looks like a pass - it is how the EXTICR field-width bug survived a
     * "passed" record on the U575 (see the driver comment in exti_u5xx.c). */
    ca_stdout_printf_n("EX30: polling active on " BTN_TXT
                       " - type 'exti_on' to enable the EXTI interrupt");

    while (1) {
        u32 cur = ca_gpio_read(BTN_PORT, BTN_PIN);
        if (cur != prev) {
            prev = cur;
            if (cur == 1U)
                ca_stdout_printf_n("EX30 POLL: B1 PRESSED  (" BTN_TXT "=1)");
            else
                ca_stdout_printf_n("EX30 POLL: B1 RELEASED (" BTN_TXT "=0)");
        }
        ca_os_sleep(100);
    }
    return 0;
}
```

Hardware initialization is here and not in `ca_os_configure()`: in a task `ca_stdout_*` it works, so the configuration and the messages describing it remain together. Then the task reads the pin every 100 ms, independently of the EXTI: it is used to verify the pin before enabling the interrupt. Polling alone, however, does not test the EXTI path, and the startup message explicitly states this.

5. Task monitor — LED2 threshold

```
static u32 task_monitor(void *arg)
{
    (void)arg;

    u32 last_count = 0U;
    u32 led2_group = 0U;

    while (!g_hw_ready) ca_os_sleep(10);

    while (1) {
        u32 cur = g_press_count;
        if (cur != last_count) {
            u32 delta = cur - last_count;
            last_count = cur;
            ca_stdout_printf_n("EX30 EXTI: total presses=%u (delta=%u)",
                               (unsigned)cur, (unsigned)delta);

            u32 grp = cur / LED2_THRESHOLD;
            if (grp != led2_group) {
                led2_group = grp;
                ca_led_toggle(2U);
                ca_stdout_printf_n("EX30: LED2 toggle (group %u)", (unsigned)grp
    );
```

```
            }
        }
        ca_os_sleep(10);
    }
    return 0;
}
```

The task sleeps for 10 ms per cycle; the debounce no longer concerns it, as it is entirely handled in the callback.

6. ca_os_configure and ca_os_main

```
void ca_os_configure(void)
{
    EVAL_STDOUT(
        ca_stdout_init();
        ca_stdout_set_sender(STDOUT);
        ca_stdout_create_thread();
    )
    EVAL_CLI(
        ca_cli_init(4096);
        ca_cli_set_sender(STDOUT);
        ca_cli_create_thread();
        ca_cli_register(&s_user_cli_node);
    )

    EVAL_USB_CDC_STD(ca_usb_dev_mng_cdc_cli();)

    /* The button, the LEDs and the EXTI line are set up by task_poll_btn, not
     * here: this hook runs before ca_os_run, where ca_stdout_* has no task
     * context and refuses to print. Doing the init inside the task keeps the
     * hardware setup next to the messages that describe it. */
}

void ca_os_main(void)
{

    ca_os_job("poll_btn",   CA_TH_LEVEL3, task_poll_btn,   NULL,
              STACK_MEDIUM, CA_JOB_STD, CA_FAULT_RESTART_THREAD);

    ca_os_job("monitor",    CA_TH_LEVEL3, task_monitor,    NULL,
              STACK_MEDIUM, CA_JOB_STD, CA_FAULT_RESTART_THREAD);

    ca_os_job("heartbeat",  CA_TH_LEVEL3, task_heartbeat,  NULL,
              STACK_MEDIUM, CA_JOB_STD, CA_FAULT_RESTART_THREAD);

    ca_os_run(true);
}
```

`ca_exti_init` (called in `task_poll_btn`) configures SYSCFG, RTSR1, IMR and NVIC; the subsequent `ca_exti_disable` disables IMR and NVIC, leaving the configuration ready.

53.1.6 Technical notes

EXTI on STM32H7xx

On STM32H7 (dual-core), the interrupt mask for the M7 core is located in `EXTI_D1->IMR1` (not in `EXTI->IMR1`). The source selection uses `SYSCFG->EXTICR[3]` (pin 13 → register 3, bits [7:4]):

```
/* pin=13     reg=3, shift=4 */
SYSCFG->EXTICR[3] = (SYSCFG->EXTICR[3] & ~0xF0U) | ((u32)CA_GPIO_PORT_C << 4U);
```

The pending register `EXTI->PR1` is cleared by writing 1 to the corresponding bit (write-1-to-clear). The pending status is read **before** calling the callback to avoid double dispatching on fast bounces.

`ca_exti_enable` — IMR and NVIC

The function `ca_exti_enable` re-enables both IMR and NVIC, because `ca_exti_disable` disables them both:

```
void ca_exti_enable(u8 pin)
{
    if (pin < 16U) {
        ca_exti_hw_imr_enable(pin);    /* EXTI_D1->IMR1 |= bit */
        ca_exti_hw_nvic_enable(pin);   /* NVIC_EnableIRQ */
    }
}
```

> **Warning**: in previous versions of the driver `ca_exti_enable` only re-enabled IMR, leaving the NVIC disabled. The fix was introduced in `caos/drivers/src/exti.c` along with this example.

EXTI15_10 pin sharing

Pins 10–15 share `EXTI15_10_IRQn`. The handler loops through all pins in the group and calls `ca_exti_dispatch(p)` only if the pending bit is active:

```
void EXTI15_10_IRQHandler(void)
{
    for (u8 p = 10U; p <= 15U; p++) handle_exti(p);
}
```

The central dispatcher in `exti.c` calls the registered callback for the pin:

```
void ca_exti_dispatch(u8 pin)
{
    if (pin < 16U && callbacks[pin])
        callbacks[pin]();
}
```

53.1.7 Expected output

Phase 1 — polling (EXTI disabled)

```
EX30: started - EXTI OFF
EX30: polling active on PC13 - type 'exti_on' to enable the EXTI interrupt
EX30 POLL: B1 PRESSED  (PC13=1)
EX30 POLL: B1 RELEASED (PC13=0)
EX30: uptime=10s  press=0  exti=OFF
```

Phase 2 — EXTI enabled via CLI

```
exti_on
EX30: EXTI enabled (RISING edge on the B1 pin)
EX30 EXTI: total presses=1 (delta=1)
EX30 POLL: B1 PRESSED  (PC13=1)
EX30 POLL: B1 RELEASED (PC13=0)
EX30 EXTI: total presses=2 (delta=1)
...
EX30 EXTI: total presses=5 (delta=1)
EX30: LED2 toggle (group 1)
```

53.1.8 Tamper on the same button (`EN_TAMP`)

The B1 pin, PC13, is also **tamper 1** on all five Nucleo boards. With `EN_TAMP` the example adds the `tamper` command, which arms the tamper on the same button and verifies what the hardware erases (Backup SRAM and tamper).

Command	What it does
`tamper arm`	writes a pattern in the backup registers, in the first 256 bytes of the backup SRAM and in the secret area (first 64 words and last word), then arms the tamper
`tamper check`	number of events, status of each area (`erased`, `kept`, `MIXED`) against the family expectation, verdict
`tamper off`	disarms

The expectation by family: registers always erased; backup SRAM erased on U575, H563 and H753, **preserved** on F446 and F767; secret area (SRAM2) erased on U575 and H563, absent on the others.

```
c.a.O.S > tamper arm
EX30 TAMPER ARMED: 32 regs, 2048 B backup SRAM, 64 KB secret - press B1
   (pressione di B1)
c.a.O.S > tamper check
EX30 TAMPER events=1
EX30 TAMPER backup registers: erased (want erased)
EX30 TAMPER backup SRAM: erased (want erased)
EX30 TAMPER secret area 64 KB at 0x20030000: erased (want erased)
EX30 TAMPER CHECK PASS
```

(U575 output). The fact that the CLI responds after the tamper is already a verification: on the U575 and H563, the erased SRAM2 is outside the heap, and caOS continues to run.

Bench: the `test_homo_ex30` class has a sixth phase, `tamper_erases_backup_domain`, which arms the tamper, asks the operator to press B1, and expects `TAMPER CHECK PASS`. Passed on all five boards on 10/24/2026.

53.1.9 Tasks and priorities

Task	Priority	Period	Function
`poll_btn`	CA_TH_LEVEL3	100 ms	Hardware init, then button GPIO polling
`monitor`	CA_TH_LEVEL3	10 ms	Press count + LED2 threshold
`heartbeat`	CA_TH_LEVEL3	500 ms	Blink LED3 + log
—	ISR EXTI15_10	on-demand	Debounce, toggle LED1 + counter

53.1.10 EXTI API used

```c
/* Inizializzazione: configura SYSCFG, edge, IMR, NVIC e registra callback */
bool ca_exti_init(u8 port, u8 pin, ca_exti_edge_t edge, ca_exti_cb_t cb);

/* Abilita interrupt (IMR + NVIC) */
void ca_exti_enable(u8 pin);

/* Disabilita interrupt (IMR + NVIC) */
void ca_exti_disable(u8 pin);
```

```
#if EN_TAMP
/* Tamper 1 sul pin di B1 (PC13), fronte di salita, modo confermato:
 * l'hardware cancella, poi cb gira nell'interrupt */
bool ca_exti_tamper_enable(ca_exti_cb_t cb);
void ca_exti_tamper_disable(void);
#endif
```

```
/* Edge selezionabile */
typedef enum {
    CA_EXTI_EDGE_RISING  = 0,
    CA_EXTI_EDGE_FALLING = 1,
    CA_EXTI_EDGE_BOTH    = 2,
} ca_exti_edge_t;
```

53.1.11 Summary

1. **Polling first, EXTI later** — the GPIO polling phase allows verifying the correct pin configuration before enabling the interrupt
2. **Debounce for timestamp in ISR** — the window starts from the edge itself; a flag cleared by a task caused it to start late and allowed bounces to pass through
3. `ca_exti_enable` **re-enables NVIC** — fix introduced in this example: `ca_exti_enable` must re-enable both IMR and NVIC to function after a `ca_exti_disable`
4. **CLI node** — `ca_cli_register()` adds a node to the CLI chain; the handler returns `true` if it handled the command, `false` to pass it on
5. **EXTI_D1 on H7xx** — use `EXTI_D1->IMR1` (CPU1/M7), not `EXTI->IMR1`

Chapter 54

DMA: Memory-to-Memory and Memory-to-Peripheral (DAC1)

54.1 Example 31 — DMA: memory-to-memory and memory-to-peripheral (DAC1)

54.1.1 Introduction

This example demonstrates the caOS **DMA** driver on NUCLEO-H753ZI and NUCLEO-U575ZI-Q, with two distinct operating modes:

1. **M2M (Memory-to-Memory)**: copying a buffer from SRAM to SRAM, with content verification.
2. **M2P (Memory-to-Peripheral)**: writing a 12-bit sine table to `DAC1->DHR12R1` via DMA, producing an analog wave on **PA4**.

The two transfers use **independent DMA channels** dynamically allocated via `ca_dma_free_channel()`. The NVIC for each channel is automatically enabled upon allocation and disabled upon release.

Hardware — two boards

The two pins do not coincide: the LED is the only difference between the boards.

Signal	H753 (MB1364)	U575 (MB1549)	Description
LED3 (red)	PB14	**PG2**	Heartbeat 1 Hz
DAC1_OUT1	PA4	PA4	Sine analog output ~1.56 Hz (0–3.3 V)

The code selects the pin with a `#if MCU_ID == 575` block. **PB14 is not an LED on the U575** (LD1 PC7, LD2 PB7, LD3 PG2): as long as the block was missing, the board ran correctly but with the heartbeat invisible — and the tests looked at the console, not the board.

> **Verification**: connect an oscilloscope or voltmeter to **PA4** (CN10 pin 17 on Nucleo-H753ZI) to observe the sine wave.

54.1.2 Configuration required in `os_config.h`

```
#define EN_DMA        1    /* Driver DMA */
#define EN_DAC        1    /* Driver DAC */
#define EN_CLI_DMA    1    /* dma info */
#define ENABLE_CLI    1
```

54.1.3 CLI Commands

Command	Description
`dma info`	Total channels, in-use bitmask, list of allocated channels

Typical output with the two channels allocated from the example:

```
dma: total=24  in_use=0x00000003
  flat=0  DMA1_s0  in-use
  flat=1  DMA1_s1  in-use
```

Channel `flat=0` is allocated by `task_dma_m2m`, channel `flat=1` by `task_dma_dac`.

54.1.4 System architecture

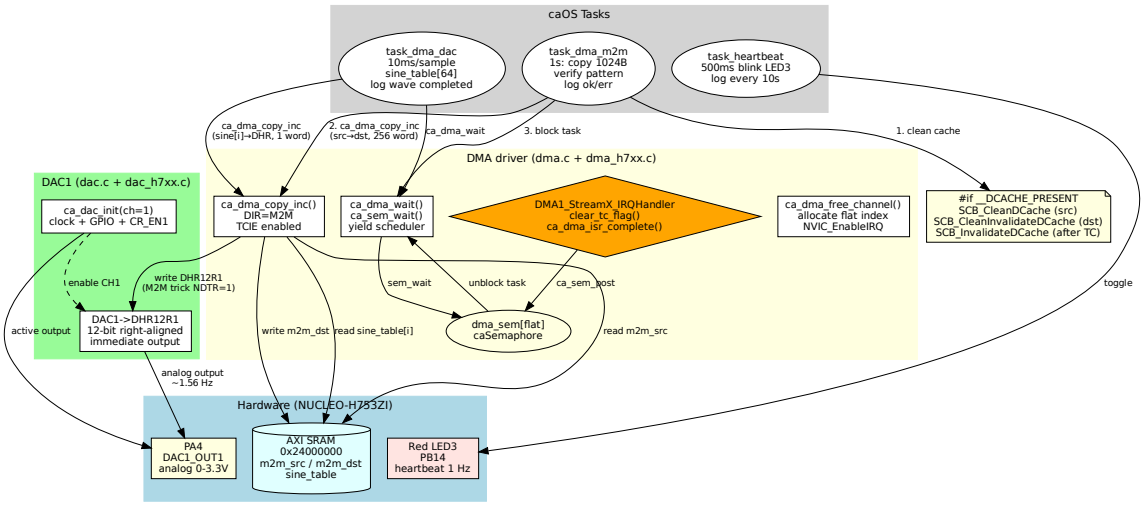


Figure 54.1: Example architecture 31

DMA M2M flow

```
ca_os_configure()
              ca_dma_init()           inizializza semafori e bitmask
              ca_dac_init(ch=1)       clock APB1L + PA4 analog + DAC_CR_EN1

task_dma_m2m:
    1. ca_dma_free_channel()    alloca flat index, NVIC_EnableIRQ automatico
    2. [every 1 s]
       __DSB()                  retires the writes before the DMA reads
       ca_dma_copy_inc(ch, src, dst, 256, CA_DMA_WIDTH_32)
       ca_dma_wait(ch, ms)      blocca task     yield scheduler
       checks the pattern     logs ok/errors
```

DMA M2P Flow (DAC)

```
task_dma_dac:
    1. ca_dma_free_channel()    canale separato da M2M
    2. SCB_CleanDCache(sine_table)  once at startup
```

```
    3. [every 10 ms, for 64 samples]
       ca_dma_copy_inc(ch, &sine_table[i], &DAC1->DHR12R1, 1, CA_DMA_WIDTH_32)
       ca_dma_wait(ch, ms)
       i = (i+1) % 64
       se i==0: logga "onda #N completata"

DAC1->DHR12R1           PA4 (output analogico immediato, no trigger)
```

ISR → semaphore mechanism

```
DMA trasferimento completo

              DMA1_StreamX_IRQHandler()   [definito in dma_h7xx.c, non-weak]
            clear_tc_flag(flat)
            ca_dma_isr_complete(controller, stream)
                         ca_sem_post(&dma_sem[flat])
                                 sblocca task in ca_dma_wait()
```

The task remains suspended (zero CPU) for the entire duration of the transfer. The scheduler executes other tasks in the meantime.

54.1.5 Source code

The file is located in `examples/example_31/ca_os_ex31.c`.

1. DMA buffer — cache alignment

```
#define BUF_WORDS    256U
#define CACHE_LINE   32U    /* Cortex-M7: cache line = 32 byte */

__attribute__((aligned(CACHE_LINE))) static u32 m2m_src[BUF_WORDS];
__attribute__((aligned(CACHE_LINE))) static u32 m2m_dst[BUF_WORDS];

__attribute__((aligned(CACHE_LINE))) static u32 sine_table[64] = { ... };
```

The buffers are declared `static` (they go into BSS → AXI SRAM at `0x24000000` on the H753, SRAM1 at `0x20000000` on the U575), accessible by the DMA. The 32-byte alignment is maintained as a precautionary measure — it is the cache line granularity of the Cortex-M7 — so that the buffers remain suitable for cache maintenance should the D-cache be enabled in the future.

> **Why not in TCM?** On STM32H7xx, the ITCM (`0x00000000`) and DTCM (`0x20000000`) regions are accessible only by the CPU, not by the DMA. The driver verifies this with `CA_SAFEGUARD` in `dma_h7xx.c`.

2. M2M Task — copy and verify

```
static u32 task_dma_m2m(void *arg)
{
    ca_dma_ch_t ch;
    i16 idx = ca_dma_free_channel(&ch);   /* allocate + NVIC_EnableIRQ */

    /* pattern fisso in src */
    for (u32 i = 0U; i < BUF_WORDS; i++)
        m2m_src[i] = (i * 0x55AA0011U) ^ 0xDEADBEEFU;

    while (1) {
        /* clear dst */
        ...

```

```
        __DSB();   /* no cache maintenance: D-cache off */

        ca_dma_copy_inc(ch, m2m_src, m2m_dst, BUF_WORDS, CA_DMA_WIDTH_32);
        ca_dma_wait(ch, 100U);   /* block     yield     wake-up from ISR */

        /* verifica */
        u32 errs = 0U;
        for (u32 i = 0U; i < BUF_WORDS; i++)
            if (m2m_dst[i] != m2m_src[i]) errs++;

        if (errs == 0U)
            ca_stdout_printf_n("EX31 M2M: ok #%u (1024B)", m2m_ok_count++);
        else
            ca_stdout_printf_n("EX31 M2M: ERRORS=%u  #%u", errs, m2m_err_count
    ++);

        ca_os_sleep(1000);
    }
}
```

3. Task M2P — Sine DAC

```
static u32 task_dma_dac(void *arg)
{
    ca_dma_ch_t ch;
    i16 idx = ca_dma_free_channel(&ch);   /* channel other than M2M */

    __DSB();   /* sine_table is read-only from here on */

    u32 si = 0U, waves = 0U;

    while (1) {
        /* M2M NDTR=1 trick: writes 1 word from SRAM to the DAC register */
        ca_dma_copy_inc(ch,
                        (const void *)&sine_table[si],
                        (void *)&DAC1->DHR12R1,
                        1U, CA_DMA_WIDTH_32);
        ca_dma_wait(ch, 100U);

        si++;
        if (si >= SINE_SAMPLES) {
            si = 0U;
            waves++;
            ca_stdout_printf_n("EX31 DAC: wave #%u done", waves);
        }
        ca_os_sleep(10);   /* 10 ms/sample     640 ms period */
    }
}
```

4. ca_os_configure

```
void ca_os_configure(void)
{
    /* ... stdout, log, cli ... */

    ca_dma_init();                                    /* semaphores + DMA clock
    */
    ca_dac_init(&(ca_dac_config_t){ .channel = 1 }); /* APB1L + PA4 + CR_EN1 */
}
```

54.1.6 Technical notes

DMA M2M trick for peripheral register access

The `ca_dma_copy_inc` driver uses **DIR=M2M** (bit 10 of `DMA_SxCR`), which does not require a DREQ handshake from the peripheral and does not need DMAMUX configuration. With `NDTR =1`, a single word is transferred from the source address to the destination address — in this case `&DAC1->DHR12R1` — treated as a normal memory address.

With `NDTR=1`, the `PINC` flag (destination increment) is irrelevant since only one transfer is performed.

```
M2M con NDTR=1:
  PAR  = &sine_table[i]        sorgente (SRAM)
  M0AR = &DAC1->DHR12R1        destinazione (APB1, trattato come memoria)
  NDTR = 1
  DIR  = 10 (M2M)
      1 trasferimento da 32 bit
  DAC1->DHR12R1 = sine_table[i]
      DAC CH1 (no trigger, TEN=0)
  PA4 = (sine_table[i] / 4096.0)    3.3V
```

Cache coherence: why it is not needed here

The STM32H753 **has** a 16KB data D-cache, but **caOS leaves it off**: in the entire `hw/h7xx/` there is not a single call to `SCB_EnableDCache()`. With the cache disabled, the CPU and DMA see the same physical RAM and there is no incoherence to resolve — a simple `__DSB()` is enough to flush pending writes before arming the transfer.

Calling `SCB_CleanDCache_by_Addr()` with the cache off **is not useless: it is harmful**. It is a write to `SCB->DCCMVAC` with the cache disabled, which is undefined behavior on the Cortex-M7, and on the H753 it produces an **IMPRECISERR BusFault**. This is exactly the defect removed by `eth_h7xx.c` (ETH bug of 08/19) and even earlier by `sdio_h7xx.c`: this example was the last point in the project that still did it.

> **The trap was the guard.** The calls were protected by `#if __DCACHE_PRESENT`, which seems like the right condition but is not: that macro says that the **core possesses** a cache, not that it is **on**. On the H753 it was 1 and the block was compiled anyway. The correct condition cannot be expressed at compile time with CMSIS — it depends on a system choice of the project.

On the U575, the issue does not arise at all: the Cortex-M33 has no D-cache.

If one day the D-cache were to be turned on on the H7, maintenance must be reintroduced **together** with that choice, in three points: `Clean` on `src` before the DMA reads, `CleanInvalidate` on `dst` before it writes, `Invalidate` on `dst` after. The 32-byte alignment of the buffers is already prepared for this.

Automatic ISRs

Starting from this example, the DMA driver directly defines the `DMA1_StreamX_IRQHandler` handlers in `dma_h7xx.c` (non-weak), eliminating the need to redefine them in the application. The NVIC is enabled automatically by `ca_dma_free_channel()` and disabled by `ca_dma_release()`.

DAC1 on STM32H7xx

Register	Value	Effect
`RCC->APB1LENR`	bit `DAC12EN`	DAC1 clock
`GPIOA->MODER[PA4]`	`11` (analog)	analog pin, no AF
`DAC1->CR`	`DAC_CR_EN1`	CH1 enabled, no trigger (`TEN1=0`)
`DAC1->DHR12R1`	`[11:0]`	12-bit value → immediate output

With `TEN1=0` (no trigger), the DAC updates the output immediately after every write to `DHR12R1`, without waiting for timer events or software triggers.

54.1.7 Expected output

```
EX31: DMA M2M + M2P DAC1 avviato
EX31: PA4=DAC1_OUT1     misura con oscilloscopio (seno ~1.56Hz)
EX31: M2M check 1024B every second
EX31 M2M: canale flat=0 (ctrl=1 str=0)
EX31 DAC: canale flat=1     PA4 DAC1_OUT1 seno 1.56Hz
EX31 M2M: ok #1 (1024B)
EX31 DAC: onda #1 completata
EX31 M2M: ok #2 (1024B)
EX31 DAC: onda #2 completata
...
EX31: uptime=10s  m2m_ok=10  m2m_err=0
```

54.1.8 Tasks and priorities

Task	Priority	Period	Function
`dma_m2m`	CA_TH_LEVEL3	1 s	M2M copy 1024B + verification
`dma_dac`	CA_TH_LEVEL3	10 ms/sample	Sine on DAC1 via DMA
`heartbeat`	CA_TH_LEVEL3	500 ms	Blink LED3 + uptime log

54.1.9 DMA APIs used

```c
/* Initializes the driver: semaphores + bitmask + controller clock */
void ca_dma_init(void);

/* Allocates the first free channel; enables the NVIC automatically */
i16 ca_dma_free_channel(ca_dma_ch_t *out);

/* Allocates a specific channel (flat index) */
i16 ca_dma_free_channel_index(i16 index, ca_dma_ch_t *out);

/* M2M transfer: src and dst increment, TCIE enabled */
bool ca_dma_copy_inc(ca_dma_ch_t ch, const void *src, void *dst,
                     u32 size, ca_dma_width_t width);

/* Attende completamento (sem_wait     yield scheduler) */
bool ca_dma_wait(ca_dma_ch_t ch, u32 timeout_ms);   /* false: no TC in time */

/* Releases the channel and disables the NVIC */
void ca_dma_release(ca_dma_ch_t ch);
```

```c
/* Transfer width */
typedef enum {
    CA_DMA_WIDTH_8  = 0,   /* byte */
    CA_DMA_WIDTH_16 = 1,   /* halfword */
```

```
    CA_DMA_WIDTH_32 = 2,   /* word */
} ca_dma_width_t;
```

54.1.10 DAC APIs used

```
/* Initializes the DAC channel: clock + analog GPIO + CR enable */
bool ca_dac_init(const ca_dac_config_t *cfg);   /* cfg.channel = 1|2 */

/* Direct 12-bit write (used by the CPU; DMA writes straight to DHR12R1) */
void ca_dac_write(u8 channel, u16 val);
```

54.1.11 Summary

1. **Two independent DMA channels** allocated with `ca_dma_free_channel()`; NVIC automatically enabled — no additional lines of code in the application
2. **M2M NDTR=1 trick** to write to a peripheral register (DAC) without configuring DMAMUX; works because DIR=M2M does not require DREQ from the device
3. **Cache maintenance** required on Cortex-M7: Clean before DMA reads, Invalidate after DMA writes; protected by `#if __DCACHE_PRESENT` for portability
4. **ca__dma__wait** is a true yield: the task blocks on the semaphore and the scheduler executes other tasks for the entire duration of the transfer
5. **Automatic ISRs**: `dma_h7xx.c` defines all non-weak handlers; the application no longer needs to redefine them

Chapter 55

ETH: Link UP / Link DOWN Detection

55.1 Example 32 — ETH: link UP/DOWN, packet reception and transmission

55.1.1 Introduction

This example demonstrates the caOS **ETH** driver on the NUCLEO-H753ZI with a **LAN8742A** PHY in **RMII** mode. The objectives are:

1. Automatic detection of Ethernet link state transitions (link UP / link DOWN) with notification via **callback**.
2. **Polling** reception of incoming Ethernet frames and dump to **USB CDC** via `ca_hexdump`.
3. Transmission of a **Gratuitous ARP** upon link connection to announce its MAC+IP to the network.
4. **L2 Echo**: every received frame is sent back to the sender with the source/destination MACs swapped (`swap_and_send`).

Traffic is generated by a PC connected directly to the board via an Ethernet cable. No external traffic generator is needed: the PC itself automatically generates ARP and IGMP traffic as soon as the network port becomes active.

At each link transition, a message is sent via **USB CDC** and the state of **LED1** (green, PB0) is updated. Each received frame is printed with the Ethernet header (DST/SRC/EtherType) and a hexadecimal dump.

Hardware (NUCLEO-H753ZI, MB1364)

Signal	MCU Pin	Description
LED1 (green)	PB0	On = link UP, off = link DOWN
LED3 (red)	PB14	Heartbeat 1 Hz
ETH_RMII_REF_CLK	PA1	RMII 50 MHz clock from PHY
ETH_MDIO	PA2	MDIO PHY management data
ETH_RMII_CRS_DV	PA7	Carrier Sense / Data Valid
ETH_MDC	PC1	MDIO clock
ETH_RMII_RXD0	PC4	RX data bit 0
ETH_RMII_RXD1	PC5	RX data bit 1
ETH_RMII_TX_EN	PG11	Transmission enabled

Signal	MCU Pin	Description
ETH_RMII_TXD1	PB13	TX data bit 1
ETH_RMII_TXD0	PG13	TX data bit 0

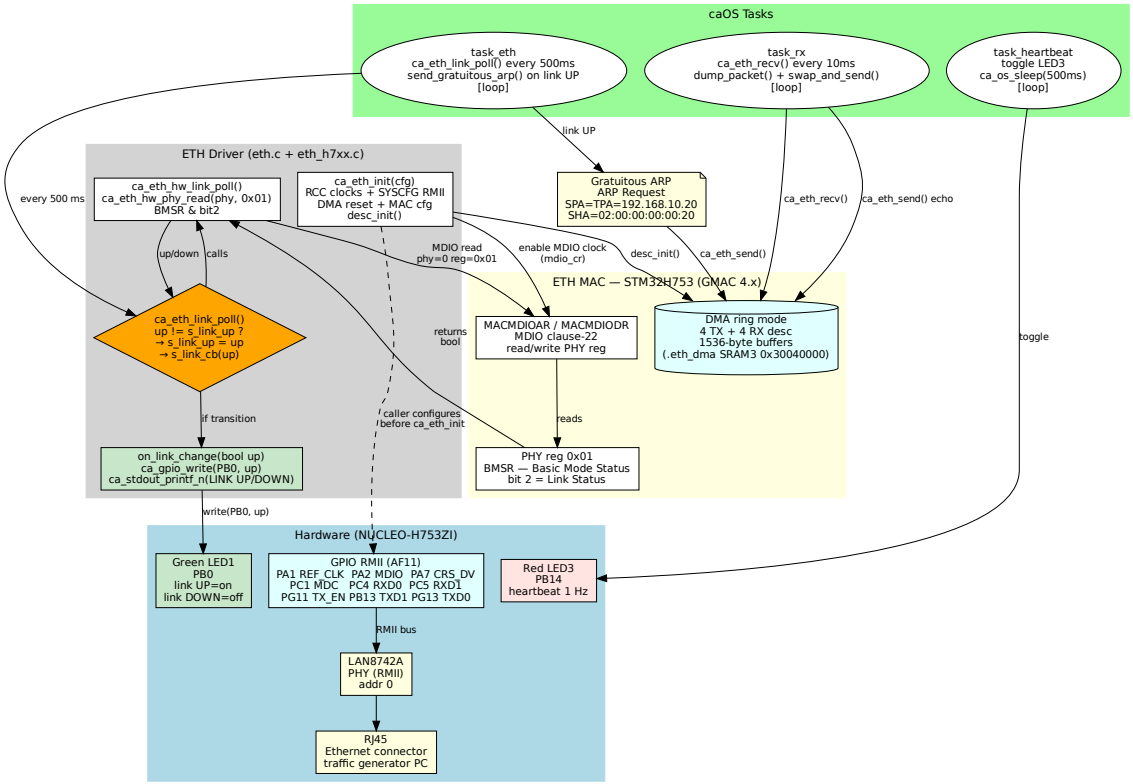
All RMII pins are configured with **AF11**, **CA_GPIO_SPEED_VERY_HIGH**, **CA_GPIO_NOPU** **CA_GPIO_OTYPE_PP**.

> **Connection**: use a straight-through Ethernet cable between the NUCLEO-H753ZI RJ45 connector (CN1) and a PC port. The traffic generator can be any tool capable of sending L2 Ethernet frames (e.g., `pktgen`, `scapy`, or simply ping).

55.1.2 Required configuration in `os_config.h`

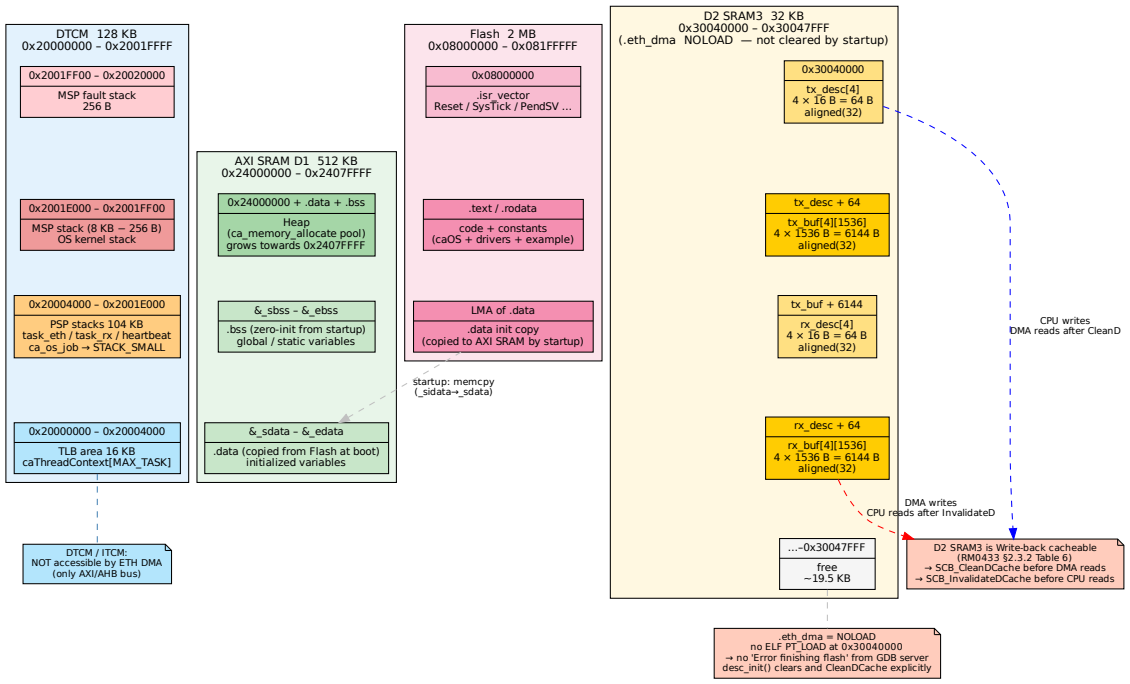
```
#define EN_ETH          1     /* Driver ETH */
#define EVAL_USB_CDC    ...   /* USB CDC for message output */
```

55.1.3 System architecture



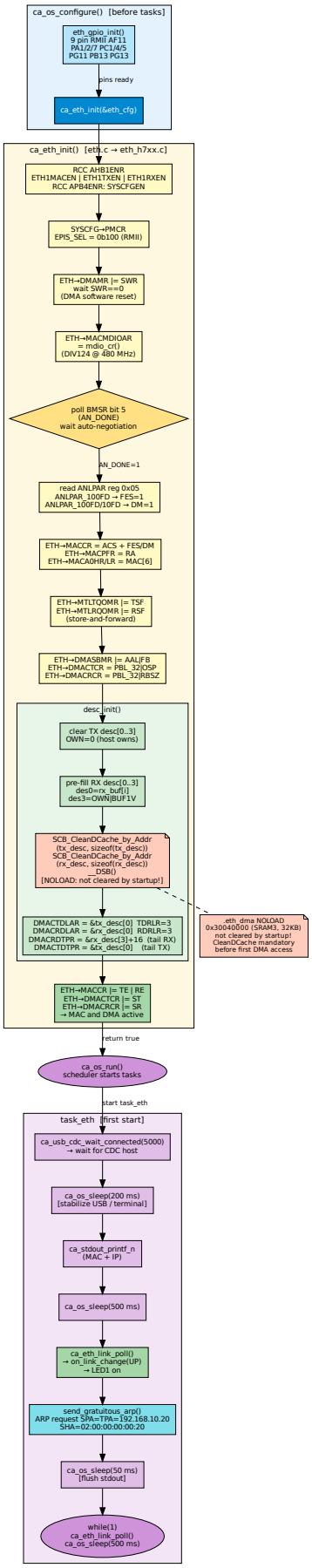
(source: `doc/diagrams/ex32_arch.dot`)

Memory map



(source: `doc/diagrams/ex32_memory.dot`)

Init flow

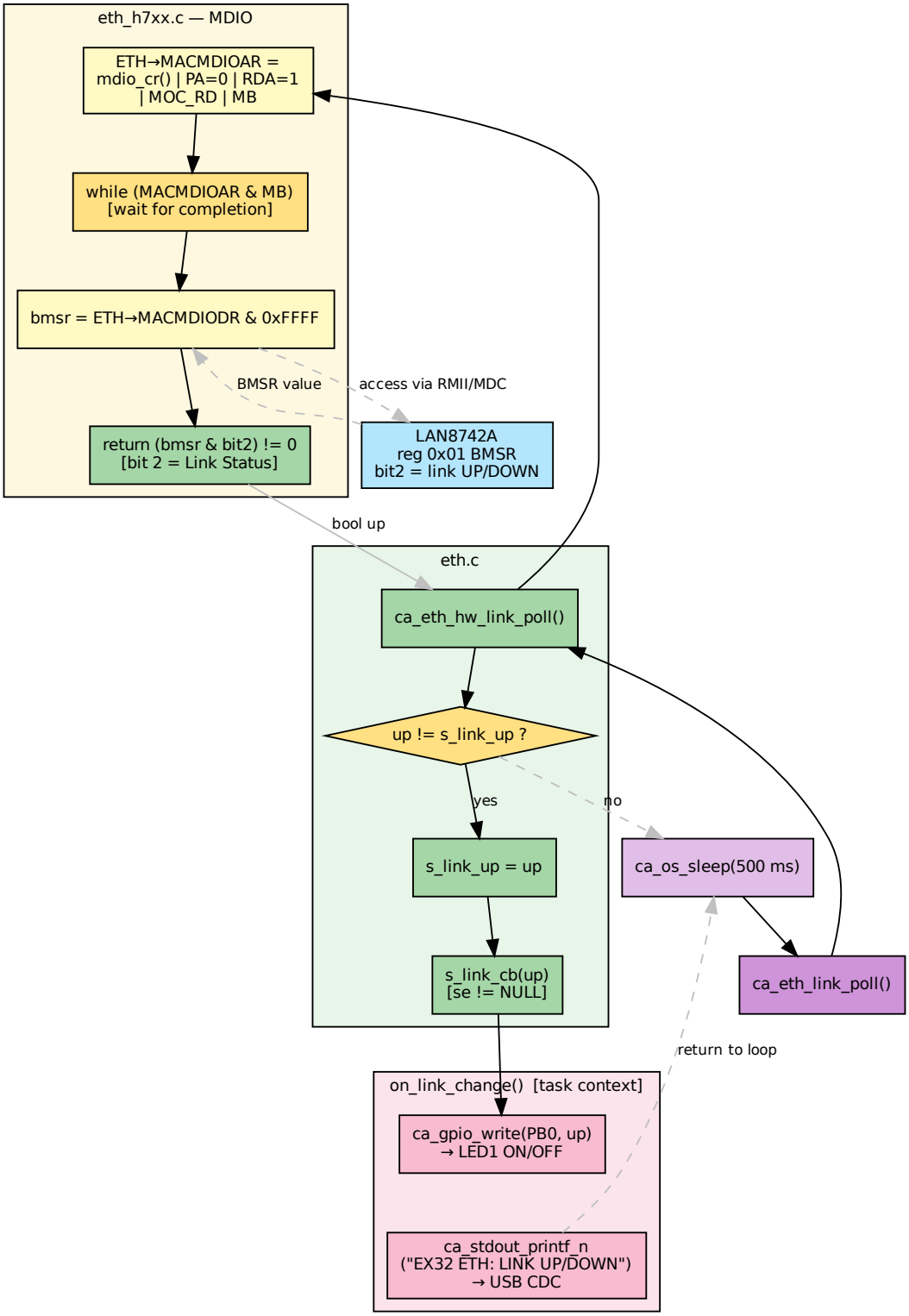


(source: `doc/diagrams/ex32_init_flow.dot`)

Why is `ca_eth_init` in `ca_os_configure` and not in the task? `ca_eth_init()` is a purely HW operation (RCC, SYSCFG, busy-wait MDIO) that does not use stdout. Executing it in `ca_os_configure` ensures that the MAC is already ready when the tasks start. Status messages are printed subsequently by `task_eth` when stdout is already active.

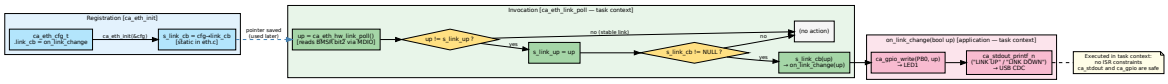
Why `ca_usb_cdc_wait_connected` + sleep in `task_eth`? The USB CDC sender silently discards messages if `tud_cdc_connected()` is false. With `wait_connected` the task waits for the host connection. The subsequent `ca_os_sleep(200)` is necessary because the busy-wait in `ca_eth_init()` (DMA reset, MDIO polling) blocks the scheduler during `ca_os_configure`: the first message would arrive too soon after the USB connection. The `ca_os_sleep(50)` before the loop gives the stdout task time to flush the message before `ca_eth_link_poll()` queues any “LINK UP”.

Link poll flow



(source: `doc/diagrams/ex32_link_poll.dot`)

Callback mechanism



(source: doc/diagrams/ex32_callback.dot)

The callback is called in **task context** (not ISR), so it is safe to invoke `ca_stdout_printf_n` and `ca_gpio_write` directly inside it.

55.1.4 The source code

The file is located in `examples/example_32/ca_os_ex32.c`.

1. `ca_eth_cfg_t` configuration and IP address

```
static void on_link_change(bool up);   /* forward declaration */

static const u8 board_ip[4] = { 192U, 168U, 10U, 20U };

static const ca_eth_cfg_t eth_cfg = {
    .phy_addr = 0U,                                          /* LAN8742A addr 0
    */
    .rmii     = true,                                        /* RMII     9 pin
    */
    .mac      = { 0x02U, 0x00U, 0x00U, 0x00U, 0x00U, 0x20U }, /* locally
    administered */
    .link_cb  = on_link_change,
};
```

The `eth_cfg` is `static const` at the file level. The forward declaration is necessary because `on_link_change` is defined after the initializer that references it as a function pointer.

The MAC address uses the `02:xx` prefix (U/L bit=1) which indicates a **locally administered** address, not assigned by the IEEE. Sufficient for L2 testing.

`board_ip` is the static IPv4 address of the board, used only for Gratuitous ARP; there is no full IP stack in this example.

2. GPIO init RMII

```
static void eth_gpio_init(void)
{
    /* PA1 REF_CLK, PA2 MDIO, PA7 CRS_DV */
    ca_gpio_configure(CA_GPIO_PORT_A, 1U,
                      CA_GPIO_MODE_AF, CA_GPIO_OTYPE_PP, CA_GPIO_SPEED_VERY_HIGH
    ,
                      CA_GPIO_NOPULL, ETH_AF);   /* ETH_AF = 11 */
    /* ... same configuration for PA2, PA7, PC1, PC4, PC5, PG11, PG12, PG13 */
}
```

> **Note**: RMII pins must be configured **before** calling `ca_eth_init()`. The ETH driver does not handle GPIOs (caOS pattern: the caller is responsible for pin configuration).

3. Link state callback

```
static void on_link_change(bool up)
{
    if (up) {
        ca_gpio_write(LED_LINK_PORT, LED_LINK_PIN, 1U);
```

```
        ca_stdout_printf_n("EX32 ETH: LINK UP");
    } else {
        ca_gpio_write(LED_LINK_PORT, LED_LINK_PIN, 0U);
        ca_stdout_printf_n("EX32 ETH: LINK DOWN");
    }
}
```

4. Task monitor link and Gratuitous ARP

```
static u32 task_eth(void *arg)
{
    (void)arg; (void)arg2;

    ca_usb_cdc_wait_connected(5000);  /* attende host; timeout 5 s */
    ca_os_sleep(200);                 /* lets USB and the host terminal settle */

    ca_stdout_printf_n("\n\nEX32: ETH INIT OK");
    ca_stdout_printf_n("PHY addr=%d RMII MAC %02X:%02X:%02X:%02X:%02X:%02X IP %u.%u.%u.%u",
        eth_cfg.phy_addr,
        eth_cfg.mac[0], eth_cfg.mac[1], eth_cfg.mac[2],
        eth_cfg.mac[3], eth_cfg.mac[4], eth_cfg.mac[5],
        board_ip[0], board_ip[1], board_ip[2], board_ip[3]);

    ca_os_sleep(500);
    ca_eth_link_poll();               /* first read; GARP from on_link_change(up) */
    ca_os_sleep(50);                  /* flush before the loop */

    while (1) {
        ca_eth_link_poll();
        ca_os_sleep(500);
    }
    return 0;
}
```

The Gratuitous ARP starts at `on_link_change(true)`, i.e., on the **rising edge** of the link and at every cable reconnection: it is used to update the ARP cache of the peers with the board's MAC. Sent blindly 500 ms after init, as in the early versions, it started ~0.9 s before the PC had the carrier and was lost, while `ca_eth_send()` responded OK anyway (measured on H563, 02/10/2026). The 500 ms period for the link poll is a good compromise between responsiveness and load on the MDIO bus.

5. Gratuitous ARP

```
static void send_gratuitous_arp(void)
{
    u16 idx = ca_net_buf_alloc();
    if (idx == NET_BUF_INVALID) { ca_stdout_printf_n("GARP: no buf"); return; }

    ca_net_buf_t *b = ca_net_buf_ptr(idx);
    u8  *arp = b->data + NET_BUF_HEADROOM;
    const u8 *mac = eth_cfg.mac.v;
    const u8 *ip  = board_ip;

    /* Ethernet header */
    for (u32 i = 0U; i < 6U; i++) arp[i]      = 0xFFU;
    for (u32 i = 0U; i < 6U; i++) arp[6U+i]   = mac[i];
    arp[12] = 0x08U; arp[13] = 0x06U;

```

```
    /* ARP payload */
    arp[14] = 0x00U; arp[15] = 0x01U;
    arp[16] = 0x08U; arp[17] = 0x00U;
    arp[18] = 6U;    arp[19] = 4U;
    arp[20] = 0x00U; arp[21] = 0x01U;

    for (u32 i = 0U; i < 6U; i++) arp[22U+i] = mac[i];
    for (u32 i = 0U; i < 4U; i++) arp[28U+i] = ip[i];
    for (u32 i = 0U; i < 6U; i++) arp[32U+i] = 0x00U;
    for (u32 i = 0U; i < 4U; i++) arp[38U+i] = ip[i];
    for (u32 i = 42U; i < 60U; i++) arp[i]   = 0x00U;

    b->data_off = NET_BUF_HEADROOM;
    b->data_len = 60U;
    i32 rc = ca_eth_send(idx);    /* driver owns buf, whatever rc says */
    if (rc == CA_ETH_SEND_OK)
        ca_stdout_printf_n("GARP %u.%u.%u.%u %02X:%02X:%02X:%02X:%02X:%02X OK",
            ip[0], ip[1], ip[2], ip[3],
            mac[0], mac[1], mac[2], mac[3], mac[4], mac[5]);
    else
        ca_stdout_printf_n("GARP FAIL %d%s", rc,
            rc == CA_ETH_SEND_ERR_LINK ? " (link down)" : " (tx ring)");
}
```

The frame is built directly in the buffer pool (NCacheable D2 SRAM1) — no copy to the DMA. The field OPER=0x0001 (Request) is correct for a Gratuitous ARP: RFC 5227 defines the GARP as an ARP Request where SPA=TPA=own IP. In case of success `ca_eth_send` owns the buffer and frees it in a deferred manner; with a negative code, it has already freed it: the caller never touches it. When the link is down `ca_eth_send` rejects the frame with `CA_ETH_SEND_ERR_LINK` (-102) instead of accepting it and losing it on the wire.

Verification with tcpdump: `sudo tcpdump -e -i eth0 arp` Expected output: `02:00:00:00:00:20 > Broadcast, ARP, length 60, ARP Request who-has 192.168.10.20 tell 192.168.10.20`

6. L2 reception and echo: `task_rx`, `dump_packet`, `swap_and_send`

```
#define DUMP_MAX  32U   /* max bytes hex-dumped per packet */

static void dump_packet(const u8 *pkt, u32 len, u32 n)
{
    ca_stdout_printf_n("PKT#%u len=%u", n, len);
    if (len < 14U) return;
    ca_stdout_printf_n(
        "  D=%02X:%02X:%02X:%02X:%02X:%02X"
        "  S=%02X:%02X:%02X:%02X:%02X:%02X"
        "  t=0x%04X",
        pkt[0],pkt[1],pkt[2],pkt[3],pkt[4],pkt[5],
        pkt[6],pkt[7],pkt[8],pkt[9],pkt[10],pkt[11],
        ((u32)pkt[12] << 8) | pkt[13]);
    u32 d = len < DUMP_MAX ? len : DUMP_MAX;
    ca_hexdump(pkt, (u16)d);
}

static void swap_and_send(u16 buf_idx)
{
    ca_net_buf_t *b = ca_net_buf_ptr(buf_idx);
    u8 *pkt = b->data + b->data_off;
    u32 len = b->data_len;
    if (len < 14U) { ca_net_buf_free(buf_idx); return; }
    /* DST = original source (unicast to the sender) */
    for (u32 i = 0U; i < 6U; i++) pkt[i] = pkt[6U + i];
```

```
    /* SRC = MAC of the board */
    for (u32 i = 0U; i < 6U; i++) pkt[6U + i] = eth_cfg.mac[i];
    i32 rc = ca_eth_send(buf_idx);   /* the driver owns buf_idx, whatever the rc
     */
    if (rc == CA_ETH_SEND_OK) ca_stdout_printf_n("TX len=%u OK", len);
    else                      ca_stdout_printf_n("TX len=%u FAIL %d", len, rc);
}

static u32 task_rx(void *arg)
{
    (void)arg; (void)arg2;
    u32 count = 0U;
    while (1) {
        u16 new_idx = ca_net_buf_alloc();
        if (new_idx == NET_BUF_INVALID) { ca_os_sleep(10); continue; }
        u16 out_idx;
        if (!ca_eth_recv(&out_idx, new_idx)) {
            ca_net_buf_free(new_idx);
            ca_os_sleep(10);
            continue;
        }
        ca_net_buf_t *b = ca_net_buf_ptr(out_idx);
        dump_packet(b->data + b->data_off, b->data_len, ++count);
        swap_and_send(out_idx);   /* trasferisce ownership a send o free */
    }
    return 0;
}
```

The static buffer `rx_pkt[1536]` no longer exists: every frame arrives already in the buffer pool (NCacheable D2 SRAM1) via **zero-copy swap**. `task_rx` allocates a replacement buffer (`new_idx`) for the RX ring, obtains the received frame (`out_idx`), and transfers its ownership to `swap_and_send`.

`swap_and_send` receives ownership of the buffer and passes it to `ca_eth_send`, which keeps it regardless of what it returns: in case of an error, the buffer has already been freed by the driver, and `swap_and_send` no longer touches it.

`ca_hexdump(pkt, d)` (from `caos/src/utils.c`) prints 16-byte lines with address, hex, and ASCII column — `hexdump -C` style format.

7. ca_os_configure and ca_os_main

```
void ca_os_configure(void)
{
    /* stdout / log / cli / USB CDC ... */
    EVAL_SENSOR(ca_temp_init();)

    ca_gpio_configure(LED_LINK_PORT, LED_LINK_PIN, CA_GPIO_MODE_OUTPUT, ...);
    ca_gpio_configure(LED_HB_PORT,   LED_HB_PIN,   CA_GPIO_MODE_OUTPUT, ...);

    /* RMII GPIOs     must be ready before ca_eth_init */
    eth_gpio_init();
    /* Zero-copy buffer pool: must be initialized BEFORE ca_eth_init
     * because desc_init() pre-allocates ETH_RX_N buffers from the pool */
    ca_net_buf_pool_init();
    if (!ca_eth_init(&eth_cfg))
        CA_SAFE_FAIL("ca_eth_init");
}

void ca_os_main(void)
{
    ca_os_job("eth_mon",   CA_TH_LEVEL3, task_eth,       NULL, STACK_SMALL, ...)
    ;
```

```
    ca_os_job("eth_rx",     CA_TH_LEVEL3, task_rx,        NULL, STACK_SMALL, ...)
    ;
    ca_os_job("heartbeat", CA_TH_LEVEL3, task_heartbeat, NULL, STACK_SMALL, ...)
    ;
    ca_os_run(true);
}
```

> **Critical order**: `ca_net_buf_pool_init()` before `ca_eth_init()`. `desc_init()` (called inside `ca_eth_hw_init`) executes `ca_net_buf_alloc()` for the 4 RX descriptors. If the pool is not yet initialized, `alloc` always returns `NET_BUF_INVALID` and the RX descriptors are loaded with NULL buffers → hard fault on the first received frame.

55.1.5 Technical notes

PHY BMSR — register 0x01

The **BMSR** (Basic Mode Status Register) register is defined by the IEEE 802.3 standard and is present on any clause-22 PHY. The bit of interest is:

Bit	Name	Description
2	Link Status	`1` = link established, `0` = link absent

Reading is performed via **MDIO** (Management Data Input/Output), a 2-wire serial protocol (MDC clock, MDIO data) managed by the `MACMDIOAR` register of the GMAC 4.x.

```
ca_eth_hw_phy_read(phy=0, reg=0x01):
    ETH->MACMDIOAR = mdio_cr()          /* divisore clock: DIV124 a 480MHz */
                   | (0 << PA_Pos)      /* phy addr = 0 */
                   | (1 << RDA_Pos)     /* reg addr = BMSR */
                   | MDIO_MOC_RD        /* clause-22 read: MOC = 0b11 */
                   | MB;                /* MB=1: starts the transaction */
    while (MACMDIOAR & MB);             /* wait for completion */
    return MACMDIODR & 0xFFFF;
```

MDIO clock on STM32H753 at 480 MHz

The MDC clock must be 2.5 MHz to be compatible with all PHYs. The `CR` field of `MACMDIOAR` selects the divider:

HCLK	Divider	Resulting MDC
< 35 MHz	DIV16	~2.2 MHz
35–60 MHz	DIV26	~2.3 MHz
60–100 MHz	DIV42	~2.4 MHz
100–150 MHz	DIV62	~2.4 MHz
150–250 MHz	DIV102	~2.4 MHz
250 MHz	DIV124	~3.9 MHz @ 480 MHz → **< 2.5 MHz**

> At 480 MHz: 480 / 124 3.9 MHz. The `DIV124` value is the maximum divider available on GMAC 4.x; the LAN8742A tolerates MDC up to 25 MHz, so it is compatible even with this frequency.

Auto-negotiation — ANLPAR

After the DMA reset, the driver waits for the PHY auto-negotiation to complete (BMSR bit 5 `AN_DONE`), then reads **ANLPAR** (reg 0x05, Link Partner Ability):

```
u16 anlpar  = ca_eth_hw_phy_read(s_phy_addr, PHY_ANLPAR);
bool speed100 = (anlpar & (PHY_ANLPAR_100FD | PHY_ANLPAR_100HD)) != 0U;
bool fulldpx  = (anlpar & (PHY_ANLPAR_100FD | PHY_ANLPAR_10FD))  != 0U;

u32 maccr = ETH_MACCR_ACS;
if (speed100) maccr |= ETH_MACCR_FES;   /* Fast Ethernet 100M */
if (fulldpx)  maccr |= ETH_MACCR_DM;    /* Duplex Mode full */
ETH->MACCR = maccr;
```

This allows the driver to operate correctly with both switches/routers and direct PC–board cables (crossover or auto-MDI/MDIX), without having to assume a fixed 100FD.

SYSCFG PMCR — RMII selection

Before the DMA reset, the `EPIS_SEL` field of `SYSCFG->PMCR` must be configured to select the physical interface:

```
/* RMII: EPIS_SEL = 0b100 = bit 23 */
SYSCFG->PMCR = (SYSCFG->PMCR & ~SYSCFG_PMCR_EPIS_SEL)
             | SYSCFG_PMCR_EPIS_SEL_2;
```

This configuration must take place **before the ETH DMA reset** (`ETH->DMAMR |= SWR`), otherwise the MAC will not synchronize correctly with the PHY.

Zero-copy architecture — memory map

With the zero-copy driver, data buffers are in the **pool** (D2 SRAM1 NCacheable) and only the DMA descriptors (128 bytes total, 4+4 × 16 bytes) remain in SRAM3:

```
D2 SRAM1  0x30000000  128 KB  NCacheable (MPU region 3)
            pool: NET_BUF_COUNT    ca_net_buf_t (2 KB ciascuno)
                  [0..3]       puntati da rx_desc[0..3].des0  (driver owned)
                  [4..7]       waiting for deferred TX free  (driver owned)
                  [8..]        free in the pool

D2 SRAM3  0x30040000  32 KB   NCacheable (MPU region 4)
            .eth_dma: tx_desc[4] + rx_desc[4]  (only 128 bytes, NOLOAD)
```

```
/* eth_h7xx.c */
__attribute__((section(".eth_dma"), aligned(32))) static eth_desc_t tx_desc[
    ETH_TX_N];
__attribute__((section(".eth_dma"), aligned(32))) static eth_desc_t rx_desc[
    ETH_RX_N];
/* There are NO more tx_buf/rx_buf: the data live in the pool */
```

In the linker script `STM32H753ZITX_FLASH.ld`:

```
MEMORY {
  ETH_SRAM3 (rw) : ORIGIN = 0x30040000, LENGTH = 32K
}

.eth_dma (NOLOAD) : { *(.eth_dma) } >ETH_SRAM3
```

> **Why NOLOAD?** The section has no initializer in flash. The driver explicitly zeroes it in `desc_init()`. It avoids a PT_LOAD ELF at 0x30040000 that the GDB server tries to verify before RCC enables the D2 SRAM clock.

Cache coherency — why the pool is NCacheable

Both the descriptors (SRAM3 `.eth_dma`) and the data buffers (SRAM1 pool) are configured as **Non-cacheable** via MPU, eliminating the need for `SCB_CleanDCache_by_Addr` / `SCB_InvalidateDCache_by_Addr` operations:

Region	MPU	Reason
Pool D2 SRAM1 (0x30000000)	Region 3, TEX=001, C=0, B=0	The ETH DMA reads/writes frames; the CPU accesses the payload. NCacheable = always coherent
Desc D2 SRAM3 (0x30040000)	Region 4, TEX=001, C=0, B=0	The ETH DMA reads/writes descriptor OWN bit; without NCacheable → IMPRECISERR (write buffer)

IMPRECISERR root cause: if the descriptors or the pool are in a Write-back cacheable region, the CPU may write the OWN bit to the D-cache without immediate flushing to physical RAM. The DMA reads the old value from RAM → failed buffer handover → asynchronous bus fault (CFSR bit 10).

The `ACTLR.DISDEFWBUF` flag (bit 1, @0xE000E008) can be used as a diagnostic tool: setting it to 1 makes all bus faults PRECISERR; if the faults disappear entirely, it means the only cause was the write buffer on a cacheable region. In production, MPU NCacheable is used instead of DISDEFWBUF (which degrades core performance).

GMAC 4.x ring details

The GMAC 4.x (present on H7xx and H5xx) uses **ring mode** with 4-word (128-bit) descriptors:

RX descriptor (pre-fill):

Word	Field	Value
DES0	Buffer 1 address	`(u32)(b->data + NET_BUF_HEADROOM)` (buffer pool)
DES1	—	0
DES2	—	0
DES3	OWN=1, BUF1V=1	DMA owned, buf1 valid

RX write-back (after reception):

DES3 Field	Bit	Description
OWN	31	0 = host can read
PL	[14:0]	Packet Length (includes 4-byte FCS)

The driver subtracts 4 bytes before returning the length to the caller.

55.1.6 Expected output

```
EX32: ETH INIT OK
PHY addr=0 RMII MAC 02:00:00:00:00:20 IP 192.168.10.20

EX32 ETH: LINK UP
GARP 192.168.10.20 02:00:00:00:00:20 OK
```

```
6
7  PKT#1 len=60
8    D=FF:FF:FF:FF:FF:FF  S=C0:25:A5:51:6C:13  t=0x0806
9  0x30040...  0xff 0xff 0xff 0xff 0xff 0xff 0xc0 0x25 ...
10 TX len=60 OK
11
12 PKT#2 len=60
13   D=01:00:5E:00:00:16  S=C0:25:A5:51:6C:13  t=0x0800
14 ...
15 TX len=60 OK
16
17 [scollega cavo]
18 EX32 ETH: LINK DOWN
19 [ricollega]
20 EX32 ETH: LINK UP
21 GARP ... OK
```

The init message appears after the 200 ms USB stabilization period. The Gratuitous ARP is sent once as soon as the link is UP. Link state messages appear within **500 ms** of the physical signal change. For every received frame, the dump and the TX confirmation (`TX len=N OK`) are printed.

> **TX LED does not blink**: at 100 Mbps RMII, a 60-byte frame lasts ~5 s — the physical ETH port LED cannot keep up with the transition.

Analysis of received packets

The board receives ordinary network traffic automatically generated by the PC — no external tool is required.

```
1  DST = FF:FF:FF:FF:FF:FF   broadcast L2
2  SRC = C0:25:A5:51:6C:13   PC Ethernet interface
3  type = 0x0806              ARP
4
5  Payload ARP (IEEE 802.3     RFC 826):
6    HTYPE = 0x0001  Ethernet
7    PTYPE = 0x0800  IPv4
8    HLEN  = 6       lunghezza MAC
9    PLEN  = 4       lunghezza IP
10   OPER  = 0x0001  ARP Request ("chi ha IP X? dimmelo a C0:25:...")
11   SHA   = C0:25:A5:51:6C:13
12   SPA   = C0:A8:0A:96       192.168.10.150  (PC IP)
```

The PC does not know the MAC associated with an IP address on the network and sends a broadcast request. The board (with filter `MACPFR = RA`, receive-all) receives it but does not respond because it does not have an IP stack.

```
1  DST = 01:00:5E:00:00:16   multicast L2     gruppo IP 224.0.0.22
2  SRC = C0:25:A5:51:6C:13   PC
3  type = 0x0800              IPv4
4
5  Header IPv4 (24 byte, IHL=6, con Router Alert option):
6    DSCP/ECN = 0xC0
7    Length   = 40
8    TTL      = 1       does not leave the local network
9    Protocol = 0x02    IGMP
10   SRC IP   = 192.168.10.150
11   DST IP   = 224.0.0.22   "All IGMPv3-capable multicast routers"
```

The PC (Windows/Linux) periodically sends **IGMP v3 Membership Reports** to the group `224.0.0.22` to notify the local router of the multicast groups it is subscribed to (e.g., mDNS `224.0.0.251`, SSDP `239.255.255.250`, etc.). With TTL=1, the packet does not cross the local segment. The board receives it because its multicast MAC `01:00:5E:xx:xx:xx` is accepted by the filter `RA`.

`len=60` for short frames Ethernet imposes a minimum length of **64 bytes** per frame (60 data + 4 FCS). Shorter frames (ARP = 42 actual bytes, IGMP = ~54 bytes) are **padded** to 60 bytes by the PC's NIC before transmission. The H7xx driver subtracts the 4 bytes of FCS → `len=60`.

55.1.7 Tasks and priorities

Task	Priority	Period	Function
`eth_mon`	CA_TH_LEVEL3	500 ms	Poll PHY BMSR, callback on link transition; send GARP on link UP
`eth_rx`	CA_TH_LEVEL3	10 ms	Drain DMA ring, dump packets, L2 echo (`swap_and_send`)
`heartbeat`	CA_TH_LEVEL3	500 ms	Blink red LED3

55.1.8 ETH APIs used

```
/* Initializes MAC + DMA, configures RMII/MII via SYSCFG.
 * GPIO pins already configured by the caller before this call.
 * Saves cfg->link_cb for link state notifications. */
bool ca_eth_init(const ca_eth_cfg_t *cfg);

/* Polls PHY BMSR; calls link_cb on an UP/DOWN transition.
 * Call periodically from a task (e.g. every 500 ms).
 * Returns true if the link is currently up. */
bool ca_eth_link_poll(void);

/* Transmits an Ethernet frame     zero-copy pool.
 * CA_ETH_SEND_OK (0): the driver owns buf_idx (deferred free).
 * CA_ETH_SEND_ERR_LINK (-102): link down.  CA_ETH_SEND_ERR_TX (-101): TX ring
 * full.  In every case the driver owns buf_idx: the caller does not free it. */
i32 ca_eth_send(u16 buf_idx);

/* Receives an Ethernet frame     zero-copy swap.
 * true:  *out_idx holds the frame (to process and then free);
 *        new_idx is now in the DMA ring (do not touch).
 * false: new_idx not consumed     ca_net_buf_free(new_idx). */
bool ca_eth_recv(u16 *out_idx, u16 new_idx);

/* Buffers legitimately held by the driver (4 fixed RX + deferred TX).
 * Use fc + held == NET_BUF_COUNT for the correct leak check. */
u16 ca_eth_pool_held_count(void);

/* Direct access to the PHY registers via MDIO (after ca_eth_init). */
u16  ca_eth_phy_read (u8 phy, u8 reg);
void ca_eth_phy_write(u8 phy, u8 reg, u16 val);
```

```c
typedef void (*ca_eth_link_cb_t)(bool up);

typedef struct {
    u8               phy_addr;  /* PHY MDIO address, 0  31   */
    bool             rmii;      /* true = RMII (9 pin), false = MII (17 pin) */
    ca_mac_t            mac;       /* MAC address (ca_mac_t with field .v[6]) */
    ca_eth_link_cb_t link_cb;   /* link state callback; NULL = disabled */
} ca_eth_cfg_t;
```

55.1.9 MCU family support

Family	ETH IP	Notes
F1xx	none	stub — always returns false
F4xx (F407/F429)	legacy GMAC	`MACMIIAR`/`MACMIIDR`, DMA TCH chained
F4xx (F446)	none	stub MCU_ID guard
F7xx	legacy GMAC	identical to F4xx, HCLK up to 216 MHz
H7xx	GMAC 4.x	ring mode, `MACMDIOAR`/`MACMDIODR`, MTL layer
H5xx	GMAC 4.x	identical to H7xx, different RCC and SBS

The application code (`ca_os_ex32.c`) is identical for all families thanks to the driver abstraction; only `os_config.h` and cmake select the correct HW implementation.

55.1.10 Summary

1. **GPIO + `ca_eth_init` in `ca_os_configure`** — synchronous HW init without stdout dependencies; tasks find the MAC ready at startup
2. **Dynamic auto-negotiation** — ANLPAR read after AN_DONE: speed/duplex configured automatically; works with switches and direct cables
3. **Gratuitous ARP on link UP** — updates peer ARP cache without requiring a ping from the PC; very useful for immediate testing with tcpdump
4. **L2 Echo (`swap_and_send`)** — end-to-end TX/RX verification at Ethernet level without IP stack; tcpdump on the PC confirms both RX and TX
5. **Descriptors in SRAM3 (.eth_dma NOLOAD)** — dedicated region 0x30040000; no problematic PT_LOAD for GDB server; no conflicts with D1 heap
6. **Cache coherency via MPU NCacheable** — pool (SRAM1) and descriptors (SRAM3) configured as Non-cacheable: no `SCB_CleanDCache` / `SCB_InvalidateDCache` in the hot path. `aligned(32)` mandatory for DMA descriptors.
7. **`ca_eth_link_poll()` in task** — polling every 500 ms on PHY BMSR bit 2; no ETH interrupt required, minimal and portable approach
8. **Synchronous callback in task context** — safe for `ca_stdout_printf_n` and GPIO; avoids ISR complexity
9. **`task_rx` total drain for burst** — loop `for(;;)` allocates `new_idx`, calls `ca_eth_recv(&out_idx , new_idx)`; if no frame, frees `new_idx` and exits; ensures that closely spaced packets are not lost
10. **TX LED does not blink at 100 Mbps** — 60-byte frame lasts ~5 s; transmission must be verified with tcpdump, not by eye

Chapter 56

caOS Net Stack: Buffer Pool, ARP Reply, ICMP Echo

56.1 Example 33 — caOS Net Stack: buffer pool, ARP reply, ICMP echo, UDP

56.1.1 Introduction

Example 33 integrates the **complete net layer** of caOS on NUCLEO-H753ZI (STM32H753ZI, PHY LAN8742A RMII). Objectives:

1. **Pool self-test**: verifies that the NCacheable DMA pool is correctly initialized (address range, exhaustion, absence of duplicates).
2. **Automatic ARP reply**: response to PC ARP requests via `ca_net_proto_init()`.
3. **ICMP echo reply**: response to pings without any additional application code.
4. **Diagnostics on request**: `eth stats` and `eth pool` from the CLI, with the leak check on `ca_eth_pool_held_count`.

Unlike example 32 (raw L2 echo), here the net stack transparently handles ARP and ICMP; the application only deals with its own UDP traffic (zero binding in this basic configuration).

Hardware (NUCLEO-H753ZI, MB1364)

Signal	MCU Pin	Description
LED1 (green)	PB0	ON = link UP
LED3 (red)	PB14	Heartbeat 500 ms
ETH_RMII_REF_CLK	PA1	RMII clock 50 MHz from PHY
ETH_MDIO	PA2	MDIO PHY management data
ETH_RMII_CRS_DV	PA7	Carrier Sense / Data Valid
ETH_MDC	PC1	MDIO clock
ETH_RMII_RXD0	PC4	RX data bit 0
ETH_RMII_RXD1	PC5	RX data bit 1
ETH_RMII_TX_EN	PG11	Transmission enabled
ETH_RMII_TXD1	PB13	TX data bit 1
ETH_RMII_TXD0	PG13	TX data bit 0

All RMII pins: **AF11**, **CA_GPIO_SPEED_VERY_HIGH**, **CA_GPIO_NOPULL**, **CA_GPIO_OTYPE_PP**.

Required configuration in `os_config.h`

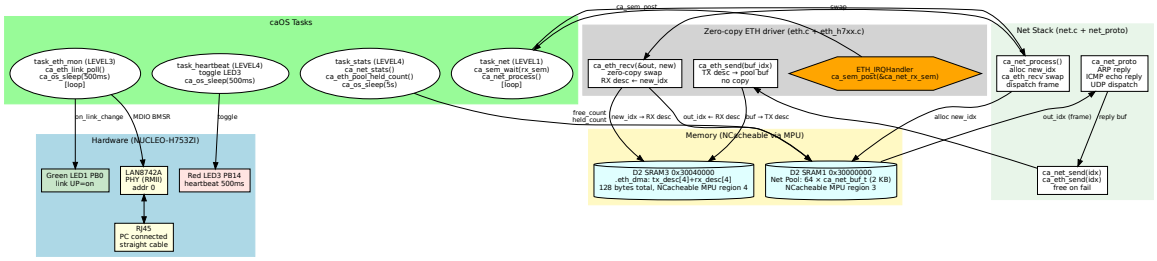
```
#define EN_ETH    1   /* driver Ethernet HW */
#define EN_NET    1   /* net_proto stack (requires EN_ETH) */
```

PC test procedure

```
sudo ip addr add 192.168.10.1/24 dev eth0
arping  -c 10  -I eth0  192.168.10.33
ping    -c 5   192.168.10.33
```

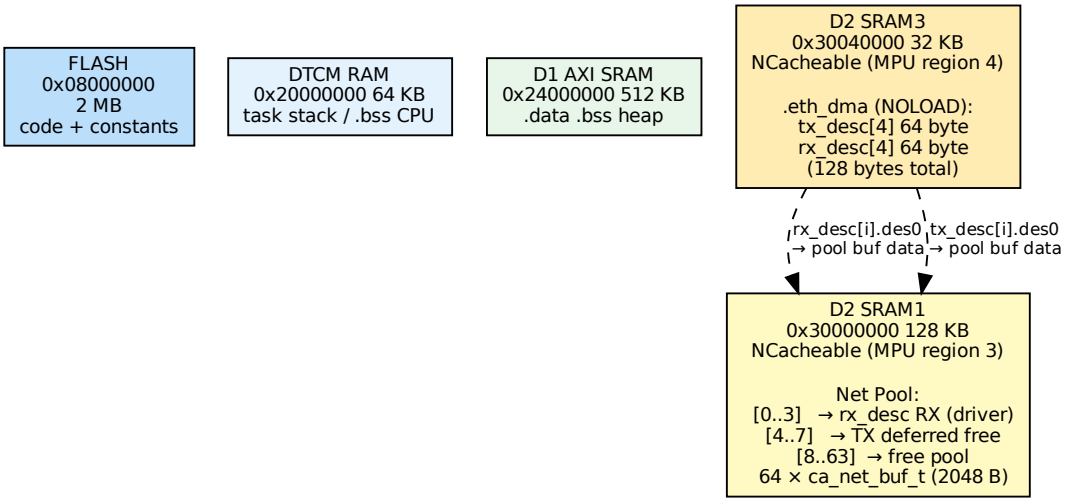
The PC must be connected directly to the board with an Ethernet cable (straight-through or crossover — LAN8742A supports Auto-MDI/MDIX).

56.1.2 System architecture



(source: `doc/diagrams/ex33_arch.dot`)

Memory map

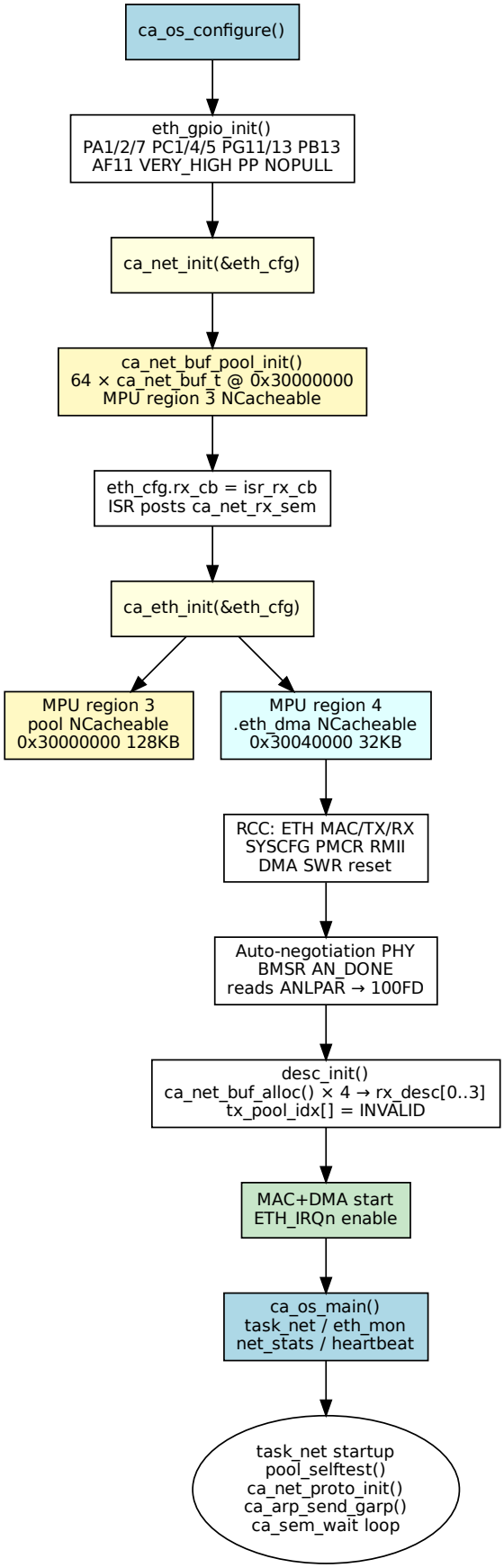


(source: `doc/diagrams/ex33_memory.dot`)

Region	Address	Size	Usage	MPU
D2 SRAM1	0x30000000	128 KB	Net pool (64 × ca_net_buf_t, 2 KB)	Region **4** NCacheable
D2 SRAM3	0x30040000	32 KB	.eth_dma: tx_desc[16] + rx_desc[8]	Region **5** NCacheable

The pool is in D2 SRAM1 NCacheable: the ETH DMA accesses the buffers directly without going through the D-cache. No cache flush/invalidate operations are necessary in the RX/TX hot path.

Initialization flow



(source: `doc/diagrams/ex33_init_flow.dot`)

56.1.3 Source code

The file is located in `examples/example_33/ca_os_ex33.c`.

1. Network configuration

```
static const ca_mac_t  s_mac = { .v = { 0x02U, 0x00U, 0x00U, 0x00U, 0x00U, 0x33U } };
static const ca_ipv4_t s_ip  = { .v = { 192U, 168U, 10U, 33U } };

static const ca_net_cfg_t net_cfg = {
    .ip  = { .v = { 192U, 168U, 10U, 33U } },
    .mac = { .v = { 0x02U, 0x00U, 0x00U, 0x00U, 0x00U, 0x33U } },
    .arp  = { .renew_ms = 30000U, .cache_size = 0U },
    .udp  = { .bindings = {{0}}, .count = 0U },
    .icmp = { .echo_reply = true },
};

static ca_eth_cfg_t eth_cfg = {
    .phy_addr = 0U,
    .rmii     = true,
    .mac      = { .v = { 0x02U, 0x00U, 0x00U, 0x00U, 0x00U, 0x33U } },
    .link_cb  = on_link_change,
    /* rx_cb / error_cb: sovrascritti da ca_net_init() */
};
```

`ca_net_cfg_t` configures the net layer: static IP, MAC, ARP parameters (no cache, renew every 30 s), no UDP binding in this demo, ICMP echo reply enabled. `ca_eth_cfg_t` has only `link_cb` set — `rx_cb` is overwritten by `ca_net_init()` which points it to its own ISR handler.

2. Pool self-test

The `pool_selftest()` function performs five checks before starting traffic:

```
static void pool_selftest(void)
{
    /* Step 1: check the physical range (must be in D2 SRAM1) */
    u32 addr = (u32)ca_net_buf_ptr(0);
    /* addr must be in [0x30000000, 0x30020000) */

    /* Step 2: allocate everything the pool can still give.
     * ca_eth_init() has already loaded ETH_RX_N buffers into the RX descriptors,
     * so the pool is NOT pristine: the expected value is NET_BUF_COUNT - held.
     */
    u16 held     = ca_eth_pool_held_count();
    u16 expected = (u16)(NET_BUF_COUNT - held);
    u16 saved[NET_BUF_COUNT];
    u16 n = 0U;
    for (; n < NET_BUF_COUNT; n++) {
        saved[n] = ca_net_buf_alloc();
        if (saved[n] == NET_BUF_INVALID) break;
    }
    /* n == expected     PASS */

    /* Step 3: the next alloc must fail (pool exhausted) */
    u16 extra = ca_net_buf_alloc();
    /* extra == NET_BUF_INVALID     PASS */

    /* Step 4: free everything, check the count */
```

```
    for (u16 i = 0U; i < n; i++) ca_net_buf_free(saved[i]);
    u16 after = ca_net_buf_free_count();
    /* after + held == NET_BUF_COUNT     PASS (no leak) */

    /* Step 5: check index uniqueness (bitmap) */
    u8 seen[NET_BUF_COUNT / 8U + 1U] = {0};
    /* allocate all, check no duplicate index     PASS */
}
```

Typical output on USB CDC:

```
--- pool selftest ---
  pool[0] @ 0x30000000  D2-SRAM1 OK
  alloc 56/56 (driver holds 8): PASS
  alloc when full: INVALID PASS
  free_count after restore: 56/56 (+8 held)  PASS
  unique indices: PASS
--- pool selftest done  free=56 ---
```

The pool is in D2 SRAM1 (`0x30000000`) — the physical address verification confirms that MPU NCacheable region 4 is active and the pool did not end up in DTCM or AXI SRAM.

3. `task_net` — RX loop

```
static u32 task_net(void *arg)
{
    (void)arg; (void)arg2;

    ca_usb_cdc_wait_connected(5000);
    ca_os_sleep(300);

    /* ... print banner, pool_selftest ... */

    ca_net_proto_init(&net_cfg);   /* registers the ARP/ICMP/UDP handlers */

    ca_os_sleep(500);
    ca_eth_link_poll();
    ca_arp_send_garp();
    ca_os_sleep(50);

    ca_stdout_printf_n("Net task ready -- waiting frames");

    while (1) {
        ca_sem_wait(&ca_net_rx_sem);   /* unblocked by the ETH ISR */
        ca_net_process();              /* swap RX, dispatch, reply */
    }
    return 0;
}
```

The task waits for the semaphore `ca_net_rx_sem` posted by the ETH ISR (connected via `rx_cb` in `ca_net_init`). `ca_net_process()` executes the RX swap loop and dispatches to the protocols: for every received frame, it allocates a replacement buffer, calls `ca_eth_recv`, delivers the frame to `net_proto` which constructs the response (ARP reply or ICMP echo), and transmits it via `ca_net_send()`.

> **Why a semaphore instead of polling sleep?** With the semaphore, the task consumes 0% CPU when no frames arrive. With a `ca_os_sleep(10)` + polling, frames would be lost during bursts and CPU would be wasted.

4. Gratuitous ARP

```
ca_arp_send_garp();
```

`ca_arp_send_garp()` (in `caos/net/src/arp_service.c`) allocates a buffer pool, constructs an ARP REQUEST frame with SPA==TPA=local IP and broadcasts it. It forces all peers on the LAN to update their ARP cache with the new MAC→IP binding of the board.

5. Pool diagnostics (on request, via CLI)

The example no longer has a periodic statistics task: counters, pool and leak check are asked of the ETH driver CLI (`caos/drivers/src/eth.c`):

Command	What it prints
`eth stats`	`rx`, `tx`, `rx_drop`, `tx_fail`, `no_hdl`, `err_bus`, `err_rbu` from `ca_net_stats()`
`eth pool`	`pool_free`, `driver_held`, historical minimum (`low_water`) and refused allocations (`alloc_fail`)
`eth dump`	registers and DMA rings (`ca_eth_dbg_dump()`)

The leak check is still the relation `pool_free + driver_held == NET_BUF_COUNT`; `eth pool` also reports a pool that ran dry and refused double frees.

6. Auxiliary tasks

```
static void on_link_change(bool up)
{
    if (up) ca_led_on(1U); else ca_led_off(1U);
    ca_stdout_printf_n("ETH LINK %s", up ? "UP" : "DOWN");
}

static u32 task_eth_mon(void *arg)
{
    (void)arg;
    while (1) {
        ca_eth_link_poll();
        ca_os_sleep(500);
    }
    return 0;
}

static u32 task_heartbeat(void *arg)
{
    (void)arg;
    while (1) {
        ca_led_toggle(3U);
        ca_os_sleep(500);
    }
    return 0;
}
```

7. ca_os_configure and ca_os_main

```
void ca_os_configure(void)
{
    EVAL_STDOUT(
        ca_stdout_init();
        ca_stdout_set_sender(STDOUT);
```

```
        ca_stdout_create_thread();
    )
    EVAL_CLI(
        ca_cli_init(2048);
        ca_cli_set_sender(STDOUT);
        ca_cli_create_thread();
    )
    EVAL_USB_CDC_STD(ca_usb_dev_mng_cdc_cli();)


    ca_led_off(1U);
    ca_led_init();
    ca_led_off(3U);

    eth_gpio_init();
    if (!ca_net_init(ca_eth_iface_get(&eth_cfg))) {
        CA_SAFE_FAIL("ca_net_init");
    }

    /* ARP replies come from net_proto's own handler, so the wire works even
     * without this - which is why its absence went unnoticed. What was missing
     * is the SERVICE: the cache, and the node that puts "arp" in the CLI
     * (arp_service.c registers it here, under EN_CLI_ARP). ca_arp_send_garp()
     * in task_net was also being called on an uninitialised service.
     * Unlike the DHCP examples, the address is known at compile time here, so
     * it is passed straight in rather than as 0.0.0.0 to be filled in later. */
    ca_arp_service_init(s_mac, s_ip);
}

void ca_os_main(void)
{
    ca_os_job("net",       CA_TH_LEVEL4, task_net,       NULL,
              STACK_SMALL, CA_JOB_STD, CA_FAULT_RESTART_THREAD);
    ca_os_job("eth_mon",   CA_TH_LEVEL3, task_eth_mon,   NULL,
              STACK_SMALL, CA_JOB_STD, CA_FAULT_RESTART_THREAD);
    ca_os_job("heartbeat", CA_TH_LEVEL4, task_heartbeat, NULL,
              STACK_SMALL, CA_JOB_STD, CA_FAULT_RESTART_THREAD);
    ca_os_run(true);
}
```

`ca_net_init()` internally executes `ca_net_buf_pool_init()` before `ca_eth_init()`: a mandatory order because `desc_init()` pre-allocates `ETH_RX_N` RX buffers (8 on H753, 4 elsewhere) from the pool. In example 32, the pool is explicitly initialized by the application; here `ca_net_init` does it transparently.

`task_net` has **CA__TH__LEVEL4**: in the weighted CFS it gets twice the CPU share of the link monitor (LEVEL3), which only does an MDIO poll every 500 ms.

56.1.4 Expected output

Startup

```
=== EX33: caOS net stack ===
Pool: 64 bufs x 2048 B = 128 KB
MAC  02:00:00:00:00:33
IP   192.168.10.33

--- pool selftest ---
  pool[0] @ 0x30000000  D2-SRAM1 OK
  alloc 56/56 (driver holds 8): PASS
  alloc when full: INVALID PASS
```

```
  free_count after restore: 56/56 (+8 held)  PASS
  unique indices: PASS
--- pool selftest done  free=56 ---

ETH LINK UP
Net task ready -- waiting frames
```

Warm restart (reset without power cycling). The PHY is not reset along with the MCU, so the link is already UP and the PC continues to send broadcasts. Two normal consequences, not to be mistaken for defects: `no_handler` counts some frames that arrived **before** `ca_net_proto_init()` registered the handlers, and `ETH LINK UP` is printed very early — during `ca_os_configure()`, therefore **above** the `=== EX33: ===` banner. The `on_link_change` callback triggers only on a state **change**: to see it again, simply unplug and replug the cable.

arping from PC

```
$ sudo arping -c 5 -I eth0 192.168.10.33
ARPING 192.168.10.33 from 192.168.10.1 eth0
Unicast reply from 192.168.10.33 [02:00:00:00:00:33]  1.23ms
Unicast reply from 192.168.10.33 [02:00:00:00:00:33]  0.98ms
...
5 packets transmitted, 5 packets received,   0% unanswered
```

ping from PC

```
$ ping -c 5 192.168.10.33
64 bytes from 192.168.10.33: icmp_seq=1 ttl=255 time=0.85 ms
64 bytes from 192.168.10.33: icmp_seq=2 ttl=255 time=0.82 ms
...
5 packets transmitted, 5 received, 0% packet loss
rtt min/avg/max = 0.82/0.85/0.89 ms
```

Statistics (`eth stats`, `eth pool`)

```
> eth stats
rx=331 tx=331 rx_drop=0 tx_fail=0 no_hdl=0 err_bus=0 err_rbu=2
> eth pool
pool_free=55/64  driver_held=9
```

`pool_free=55`: 8 fixed buffers in RX descriptors (`ETH_RX_N`) + 1 TX buffer in deferred free = 9 "held" buffers → 64 55 = 9, **no actual leak**.

The number of held buffers is not constant and depends on the part. `ETH_RX_N` is **8 on H753** and 4 on other families; `ETH_TX_N` is **16 on H753** and 4 elsewhere. The leak criterion is never the absolute value, but the relationship `pool_free + driver_held == NET_BUF_COUNT` — the same one verified by the self-test.

Moment	`pool_free`	`driver_held`	Composition
immediately after init	56	8	8 preloaded RX
at rest, after the first TX	55	9	8 RX + 1 deferred TX
after sustained traffic	**40**	**24**	8 RX + **16 TX** (full ring)

The growth up to 24 is **correct**: the *deferred free* releases the buffer of a TX slot only when that slot is **reused**, therefore at idle traffic each slot holds its own last

buffer. On H753 this immobilizes 16 2 KB buffers (**32 KB**, 25% of the pool) — with `NET_BUF_COUNT=64` there is margin, but it must be kept in mind when sizing the pool on smaller parts.

A real leak is recognized because `pool_free + driver_held` **drops below 64**, not because `pool_free` decreases.

Verified under real load (Example 44, iperf at 33 Mbps sustained): the sum remains **exactly 64** and `driver_held` stops at 24 without exceeding it. 24 is therefore a **structural ceiling**, not a progressive accumulation.

`err_rbu=2`: Ring Buffer Unavailable — the DMA had exhausted the RX buffers during an initial burst; the ISR restarts the DMA automatically.

56.1.5 Technical notes

Network init sequence

```
ca_os_configure()
            ca_net_init(&eth_cfg)
                  ca_net_buf_pool_init()            pool in D2 SRAM1 NCacheable
                  eth_cfg.rx_cb = isr_rx_cb         semaforo in ISR
                  ca_eth_init(&eth_cfg)
                        ca_eth_hw_init()
                                MPU region 3: pool NCacheable
                                MPU region 4: .eth_dma NCacheable
                                RCC, SYSCFG RMII, reset DMA
                                auto-neg PHY (BMSR AN_DONE)
                                desc_init():
                                        ca_net_buf_alloc()    4      rx_desc
    [0..3].des0
                                        SCB_CleanDCache (descriptor, NCacheable
        no-op)
                                MAC+DMA start, ETH_IRQn enable
                        return true

ca_os_main()
            task_net: ca_net_proto_init()     registra ARP/ICMP handler
```

RX semaphore and throughput

The `ca_net_rx_sem` semaphore is posted by the ETH ISR (interrupt context). `ca_net_process()` empties the RX ring in a loop (it does not stop at the first frame): a single `ca_sem_post` can unblock the processing of multiple frames accumulated between one wake-up and the next.

With a 1 ms inter-packet ping (`ping -i 0.001 -s 1472`), typical RTTs of 0.8–1.2 ms are obtained, limited primarily by the USB CDC latency of the log terminal.

Handling `err_rbu` (RX Buffer Unavailable)

`err_rbu` increments when the DMA reaches the end of the RX ring before `ca_net_process()` has swapped the buffer. The driver detects the stall via the `RBU` bit in `DMACSR` and restarts the RX DMA channel by writing `DMACRDTPR`. No frames are lost (the DMA stops, it does not overwrite): upon restart, it resumes from the next free descriptor.

To reduce `err_rbu`: increase `ETH_RX_N` (from 4 to 8) or increase the priority of `task_net`.

`ca_net_proto_init` vs `ca_net_init`

Function	Role
`ca_net_init(cfg)`	Initializes pool, ETH driver, RX ISR. Call in `ca_os_configure`.
`ca_net_proto_init(cfg)`	Registers ARP/ICMP/UDP handlers. Call at the start of the net task (after stdout is active).

The two functions are separated to allow hardware initialization (pool + MAC + DMA) in `ca_os_configure` (without the OS active) and to register protocols after the task has stdout available for debug prints.

Chapter 57

Multi-Client UDP Server on Port 7961

57.1 Example 34 — Multi-client UDP Server on port 7961

57.1.1 Introduction

Example 34 adds a **multi-client UDP server** to the caOS net layer, again on NUCLEO-H753ZI (STM32H753ZI, PHY LAN8742A RMII). Objectives:

1. **UDP server** listening on port 7961 with up to 4 simultaneous clients.
2. **Echo** of received data with prefix `"ECHO: "`.
3. **Automatic disconnection** after 10 s of client inactivity.
4. **Integrated ARP service**: automatic MAC resolution for UDP responses.
5. **CLI** integrated into the `ca_udp_server` service — commands `udp stats`, `udp list`, `udp kick`, `udp send` available automatically.

Unlike example 33 (passive ARP + ICMP), here a UDP client sends data to the server which responds actively; each client gets a dedicated worker thread managed internally by `ca_udp_server` .

Hardware (NUCLEO-H753ZI, MB1364)

Identical to example 33 — same RMII pinout, same LEDs.

Signal	MCU Pin	Description
LED1 (green)	PB0	ON = link UP
LED3 (red)	PB14	Heartbeat 500 ms
ETH_RMII_REF_CLK	PA1	RMII 50 MHz clock from PHY
ETH_MDIO	PA2	MDIO PHY management data
ETH_RMII_CRS_DV	PA7	Carrier Sense / Data Valid
ETH_MDC	PC1	MDIO clock
ETH_RMII_RXD0	PC4	RX data bit 0
ETH_RMII_RXD1	PC5	RX data bit 1
ETH_RMII_TX_EN	PG11	Transmission enabled
ETH_RMII_TXD1	PB13	TX data bit 1
ETH_RMII_TXD0	PG13	TX data bit 0

Required configuration in `os_config.h`

```
#define EN_ETH           1   /* driver Ethernet HW */
#define EN_NET           1   /* net_proto stack (requires EN_ETH) */
#define EN_ARP_SERVICE   1   /* ARP cache + automatic resolution */
#define EN_UDP_SERVER    1   /* UDP server multi-client */
```

PC test procedure

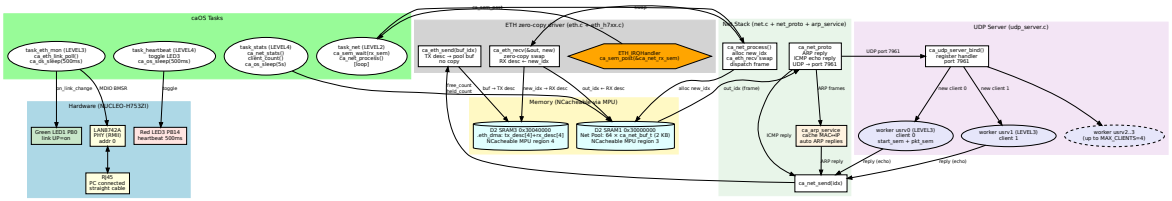
```
sudo ip addr add 192.168.10.1/24 dev eth0

# Single packet test:
echo "hello" | nc -u -q1 192.168.10.34 7961

# Interactive session (CTRL+C to quit):
nc -u 192.168.10.34 7961
```

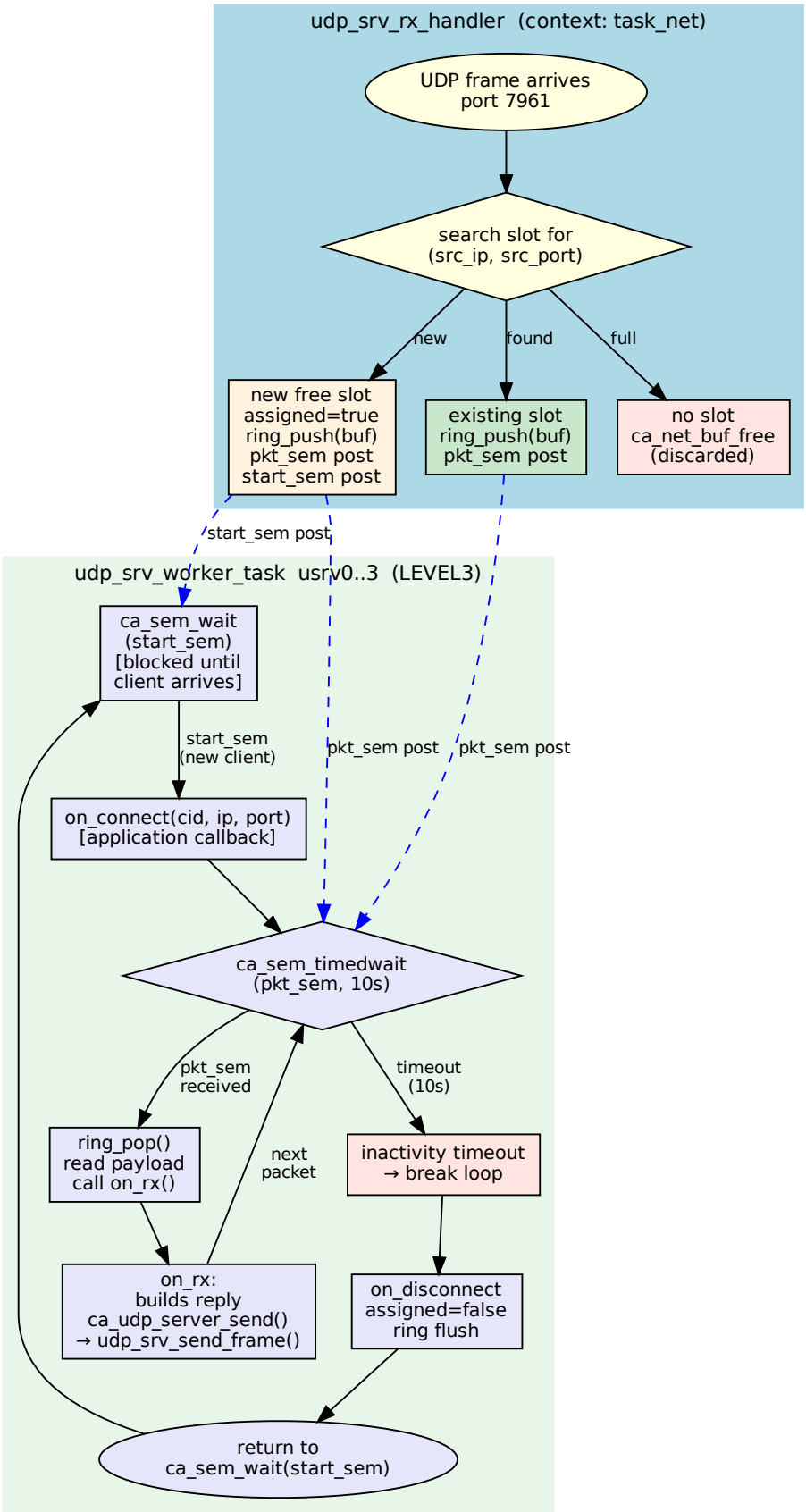
The server responds with `ECHO: <message>`. After 10 s without packets, the connection is closed (on_disconnect printed to console).

57.1.2 System architecture



(source: `doc/diagrams/ex34_arch.dot`)

UDP server worker flow



(source: `doc/diagrams/ex34_worker_flow.dot`)

Memory map

Identical to example 33:

Region	Address	Size	Usage	MPU
D2 SRAM1	0x30000000	128 KB	Net pool (64 × ca_net_buf_t, 2 KB)	Region 3 NCacheable
D2 SRAM3	0x30040000	32 KB	.eth_dma: tx_desc[4]+rx_desc[4]	Region 4 NCacheable

Differences compared to example 33

Aspect	Ex 33	Ex 34
Additional dependencies	—	EN_ARP_SERVICE, EN_UDP_SERVER
UDP port	none (no binding)	7961 (server)
Simultaneous clients	—	up to 4
ARP service	no	yes (`ca_arp_service`)
task_net priority	LEVEL1	LEVEL2
Worker thread	—	usrv0..3 (LEVEL3)
CLI	—	via integrated `ca_udp_server`

`task_net` is at LEVEL2 (not LEVEL1 as in ex33) because UDP workers must be able to respond to clients without having to wait for RX processing.

57.1.3 Source code

The file is located in `examples/example_34/ca_os_ex34.c`.

1. Network configuration

```
static const ca_mac_t  s_mac = { .v = { 0x02U, 0x00U, 0x00U, 0x00U, 0x00U, 0x34U } };
static const ca_ipv4_t s_ip  = { .v = { 192U, 168U, 10U, 34U } };

static ca_eth_cfg_t eth_cfg = {
    .phy_addr = 0U,
    .rmii     = true,
    .mac      = { .v = { 0x02U, 0x00U, 0x00U, 0x00U, 0x00U, 0x34U } },
    .link_cb  = on_link_change,
};

static const ca_net_cfg_t net_cfg = {
    .ip   = { .v = { 192U, 168U, 10U, 34U } },
    .mac  = { .v = { 0x02U, 0x00U, 0x00U, 0x00U, 0x00U, 0x34U } },
    .arp  = { .renew_ms = 30000U },
    .udp  = { .bindings = { {0U, NULL} }, .count = 0U },
    .icmp = { .echo_reply = true },
};
```

`s_mac` and `s_ip` are passed to `ca_arp_service_init()` and used in the construction of the GARP. Initialization with `.v = {}` is mandatory: the first member of `ca_ipv4_t` is `u32 addr` and the first of `ca_mac_t` is the struct `raw` — without `.v` only the first field would be initialized.

2. UDP server configuration

```
#define UDP_SRV_PORT       7961U
#define MAX_CLIENTS        4U
#define CLIENT_TIMEOUT_MS  10000U

static const ca_udp_server_cfg_t srv_cfg = {
    .port              = UDP_SRV_PORT,
    .max_clients       = MAX_CLIENTS,
    .client_timeout_ms = CLIENT_TIMEOUT_MS,
    .local_mac         = { .v = { 0x02U, 0x00U, 0x00U, 0x00U, 0x00U, 0x34U } },
    .local_ip          = { .v = { 192U, 168U, 10U, 34U } },
    .on_rx             = on_rx,
    .on_connect        = on_connect,
    .on_disconnect     = on_disconnect,
};
```

`local_mac` and `local_ip` in the server configuration are used to build the TX frames of UDP responses (Ethernet + IP header).

3. Callbacks

```
static void on_connect(u8 cid, ca_ipv4_t ip, u16 port)
{
    ca_stdout_printf_n("UDP[7961] client %u: %u.%u.%u.%u:%u connected",
        (u32)cid, ip.v[0], ip.v[1], ip.v[2], ip.v[3], (u32)port);
}

static void on_disconnect(u8 cid, ca_ipv4_t ip, u16 port)
{
    ca_stdout_printf_n("UDP[7961] client %u: %u.%u.%u.%u:%u disconnected",
        (u32)cid, ip.v[0], ip.v[1], ip.v[2], ip.v[3], (u32)port);
}

static void on_rx(u8 cid, ca_ipv4_t ip, u16 port,
                  const u8 *pld, u16 len)
{
    u8 reply[256U];
    static const u8 pfx[] = "ECHO: ";
    const u16 pfxlen = (u16)(sizeof(pfx) - 1U);
    u16 rlen = pfxlen + len;
    if (rlen > (u16)sizeof(reply)) rlen = (u16)sizeof(reply);
    for (u16 i = 0U; i < pfxlen; i++) reply[i] = pfx[i];
    u16 dlen = rlen - pfxlen;
    for (u16 i = 0U; i < dlen; i++) reply[pfxlen + i] = pld[i];

    ca_udp_server_send(cid, reply, rlen);
}
```

Callbacks are called in the context of the **worker task** `usrv<N>` — not the task_net. This allows performing blocking operations (e.g., wait on a resource) without blocking the RX processing of other clients.

`on_rx` builds the response on the stack (local buffer `reply[256]`) and sends it with `ca_udp_server_send(cid, ...)` which allocates a buffer pool and calls `ca_net_send()`.

4. `task_net` — RX loop

```
static u32 task_net(void *arg)
{
    ca_net_proto_init(&net_cfg);   /* registers the ARP/ICMP handler */
    ca_udp_server_bind();          /* registers the port 7961 handler */
```

```
5
6       ca_eth_link_poll();
7       ca_arp_send_garp();
8
9       while (1) {
10          ca_sem_wait(&ca_net_rx_sem);
11          ca_net_process();
12      }
13  }
```

`ca_udp_server_bind()` must be called **after** `ca_net_proto_init()`: it registers its own handler in the protocol stack's UDP table. Packets on port 7961 are delivered directly to the `udp_srv_rx_handler` which handles demultiplexing for clients.

5. ca_os_configure and ca_os_main

```
1   void ca_os_configure(void)
2   {
3       /* ... stdout, CLI, USB CDC, GPIO, ETH ... */
4       if (!ca_net_init(&eth_cfg))
5           CA_SAFE_FAIL("ca_net_init");
6
7       ca_arp_service_init(s_mac, s_ip);  /* init ARP cache */
8       /* CLI udp* commands registered automatically by ca_udp_server_start() */
9   }
10
11  void ca_os_main(void)
12  {
13      ca_arp_service_start();             /* avvia task ARP */
14      ca_udp_server_start(&srv_cfg);      /* starts workers usrv0..3 */
15
16      ca_os_job("net",       CA_TH_LEVEL2, task_net,    ...);
17      ca_os_job("eth_mon",   CA_TH_LEVEL3, task_eth_mon, ...);
18      ca_os_job("net_stats", CA_TH_LEVEL4, task_stats,  ...);
19      ca_os_job("heartbeat", CA_TH_LEVEL4, task_heartbeat, ...);
20      ca_os_run(true);
21  }
```

Mandatory order: 1. `ca_net_init()` (in `ca_os_configure`) — initialize pool + ETH + ISR 2. `ca_arp_service_init()` (in `ca_os_configure`) — init ARP cache (no OS yet) 3. `ca_arp_service_start()` (in `ca_os_main`) — start ARP task (OS active) 4. `ca_udp_server_start()` (in `ca_os_main`) — create worker thread (OS active) 5. `ca_net_proto_init()` + `ca_udp_server_bind()` (in `task_net`) — register handler

6. UDP CLI commands

CLI commands are automatically registered by `ca_udp_server_start()` via `ca_cli_register()` — they do not require hooks in the example code.

Command	caOS function	Description
`udp stats`	`ca_net_stats()`, `ca_udp_server_client_count()`	Network counters + active clients
`udp list`	`ca_udp_server_get_client(i, &info)`	IP/port/ring for each slot
`udp kick <id>`	`ca_udp_server_disconnect(id)`	Force slot closure (0–3)
`udp send <id> <msg>`	`ca_udp_server_send(id, ...)`	Send data to active client

57.1.4 Multi-client model — UDP Server API

Opening a connection (server-side)

A "client" is identified by the `(src_ip, src_port)` pair of the first UDP packet received. There is no TCP handshake: any packet from a new source opens a slot. Free slots: `MAX_CLIENTS - ca_udp_server_client_count()`.

Worker thread model

Each `[0..MAX_CLIENTS-1]` slot has its own `usrv<N>` task created by `ca_udp_server_start()`. Workers are **always running**: blocked on `start_sem` when free, active in the `pkt_sem` loop when they have a client.

```
free slot:
  usrv<N>       ca_sem_wait(start_sem)  [blocked]

new packet from an unknown IP:
  udp_srv_rx_handler:
    assegna slot N
    ring_push(buf)
    ca_sem_post(&pkt_sem)
    ca_sem_post(&start_sem)       unblocks the worker

  usrv<N>       on_connect(N, ip, port)
                loop: ca_sem_timedwait(pkt_sem, 10s)
                     ring_pop()     on_rx()     ca_udp_server_send()

timeout 10s:
  usrv<N>       on_disconnect(N, ip, port)
                assigned=false
                ca_sem_wait(start_sem)  [blocked, ready for a new client]
```

Public API

```
/* Avvia i worker thread (chiamare in ca_os_main, OS attivo) */
bool ca_udp_server_start(const ca_udp_server_cfg_t *cfg);

/* Registers the UDP handler (call in task_net after ca_net_proto_init) */
bool ca_udp_server_bind(void);

/* Sends data to the active client (call from the on_rx callback or another task
   ) */
bool ca_udp_server_send(u8 client_id, const u8 *payload, u16 len);

/* Forces a client to disconnect */
void ca_udp_server_disconnect(u8 client_id);

/* Number of clients currently assigned */
u8 ca_udp_server_client_count(void);

/* Info on a slot (returns false if not assigned) */
bool ca_udp_server_get_client(u8 client_id, ca_udp_srv_client_info_t *info);
```

57.1.5 Expected output

Startup

```
=== EX34: UDP server porta 7961 ===
MAC  02:00:00:00:00:34
IP   192.168.10.34
ETH LINK UP
GARP 192.168.10.34  pool_free=59  OK
UDP server ready
```

Client connection

```
$ echo "hello caos" | nc -u -q1 192.168.10.34 7961
ECHO: hello caos
```

MCU Console:

```
UDP[7961] client 0: 192.168.10.1:54321 connesso
UDP[7961] client 0: rx 10 bytes from 192.168.10.1:54321
UDP[7961] client 0: 192.168.10.1:54321 disconnesso
```

Multi-client session (CLI)

```
> udp stats
NET  rx=47 drop=0 tx=23
POOL free=59/64 held=5
UDP  port=7961 clients=2/4

> udp list
  [0] 192.168.10.1:54321  ring=0
  [1] 192.168.10.2:48102  ring=1

> udp kick 1
client 1 disconnesso

> udp send 0 hello from the server
send client 0: ok
```

Statistics (every 5 s)

```
STATS pool_free=59/64 rx=134 drop_pool=0 tx=67 clients=1
```

57.1.6 Technical notes

ARP service vs ARP in net_proto

`ca_net_proto_init()` already handles unicast ARP responses directed to the board. `ca_arp_service` adds a **MACIP cache**: when the server needs to send a UDP response to a client, it already has the resolving MAC in the cache without having to perform a blocking ARP request.

Without `EN_ARP_SERVICE`, the server might use the broadcast MAC as a fallback (less efficient, discards unnecessary traffic on the LAN).

Client timeout and concurrency

The 10 s timeout is implemented with `ca_sem_timedwait` in the worker: if no packets arrive within the timeout, the loop exits, calls `on_disconnect`, and the slot is freed for a new client.

Concurrency between `udp_srv_rx_handler` (task_net context) and the worker (task usrv context) is protected by `s_srv_mtx`. The ring buffer is the only shared data structure and is always accessed under a mutex.

Task priorities

```
LEVEL1: (not used in ex34     reserved for more urgent tasks)
LEVEL2: task_net            processing frame RX
LEVEL3: task_eth_mon        link PHY poll
         usrv0..3           worker UDP (creati da ca_udp_server_start)
LEVEL4: task_stats          statistiche
         task_heartbeat     LED
```

`task_net` at LEVEL2 yields the CPU to workers at LEVEL3 as soon as it posts `pkt_sem`: workers respond to the client with minimal latency.

Reply sizes

The local buffer `reply[256]` in the callback `on_rx` is on the worker's `usrv<N>` **stack**. The stack is `STACK_SMALL` (configurable in `os_config.h`). For UDP payloads > 250 bytes, increase the reply buffer size or use a direct buffer pool.

Conditional compilation

```
#if EN_ETH && EN_NET && EN_UDP_SERVER
/* ... all the ex34 code ... */
#endif
```

`EVAL_ARP_SVC(expr)` and `EVAL_UDP_SRV(expr)` are macros that execute `expr` only if `EN_ARP_SERVICE` and `EN_UDP_SERVER` are both enabled.

Chapter 58

DHCP Client + ARP + ICMP Ping + UDP Server with 2 Clients

58.1 Example 35 — DHCP client + ARP + ICMP ping + UDP server 2 client

58.1.1 Introduction

Example 35 adds the **DHCP client** to the caOS net layer, still on NUCLEO-H753ZI (STM32H753ZI, PHY LAN8742A RMII). Objectives:

1. **DHCP client**: obtains IP, mask, gateway, and DNS automatically via DORA protocol (Discover → Offer → Request → Ack).
2. **Lease renewal**: T1/T2/rebinding state machine compliant with RFC 2131.
3. **ARP service**: MACIP cache, updated with the new IP after every BOUND.
4. **ICMP echo reply** (ping) — functional as soon as DHCP is BOUND.
5. **UDP server** on port 7961 with up to 2 simultaneous clients — echo with prefix `"ECHO: "`.
6. **GARP** (Gratuitous ARP) sent after every DHCP bound/renew.
7. **CLI** integrated into services — commands `dhcp info/renew` from `ca_dhcp_service`, `udp stats /list/kick/send` from `ca_udp_server`, available automatically.

Unlike example 34 (static IP), here the network address is not known a priori: the stack starts with `0.0.0.0` and updates all layers (net_proto, udp_server, arp_service) in the `on_dhcp_bound` callback.

Hardware (NUCLEO-H753ZI, MB1364)

Identical to examples 33 and 34 — same RMII pinout, same LEDs.

Signal	MCU Pin	Description
LED1 (green)	PB0	ON = link UP
LED3 (red)	PB14	Heartbeat 500 ms
ETH_RMII_REF_CLK	PA1	RMII 50 MHz clock from PHY
ETH_MDIO	PA2	MDIO PHY management data
ETH_RMII_CRS_DV	PA7	Carrier Sense / Data Valid
ETH_MDC	PC1	MDIO clock
ETH_RMII_RXD0	PC4	RX data bit 0
ETH_RMII_RXD1	PC5	RX data bit 1
ETH_RMII_TX_EN	PG11	Transmission enabled

Signal	MCU Pin	Description
ETH_RMII_TXD1	PB13	TX data bit 1
ETH_RMII_TXD0	PG13	TX data bit 0

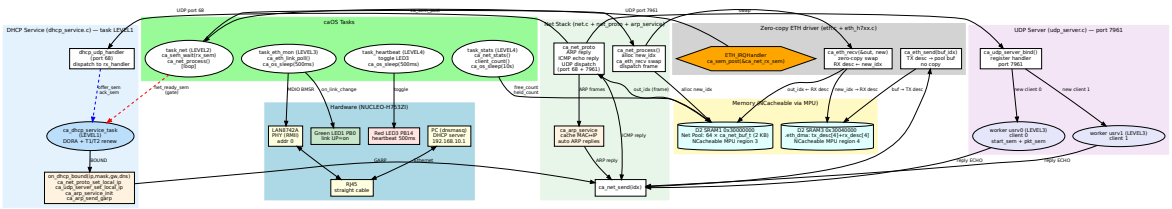
Required configuration in `os_config.h`

```
#define EN_ETH           1   /* driver Ethernet HW */
#define EN_NET           1   /* net_proto stack (requires EN_ETH) */
#define EN_NET_DHCP      1   /* DHCP parser in the proto stack */
#define EN_ARP_SERVICE   1   /* ARP cache + automatic resolution */
#define EN_UDP_SERVER    1   /* UDP server multi-client */
#define EN_DHCP_SERVICE  1   /* DHCP client DORA + T1/T2 renew */
```

DHCP parameters (modifiable in `os_config.h`):

Macro	Default	Meaning
`DHCP_DISCOVER_RETRY_N`	3	DISCOVER attempts before backoff
`DHCP_OFFER_TIMEOUT_MS`	4000	Wait for OFFER per attempt
`DHCP_REQUEST_TIMEOUT_MS`	4000	Wait for ACK after REQUEST
`DHCP_RENEW_TIMEOUT_MS`	10000	Unicast interval in RENEWING/REBINDING
`DHCP_RETRY_BACKOFF_MS`	5000	Pause before restarting DORA
`DHCP_BIND_TIMEOUT_MS`	60000	Max timeout of `ca_dhcp_service_bind()`

58.1.2 System architecture

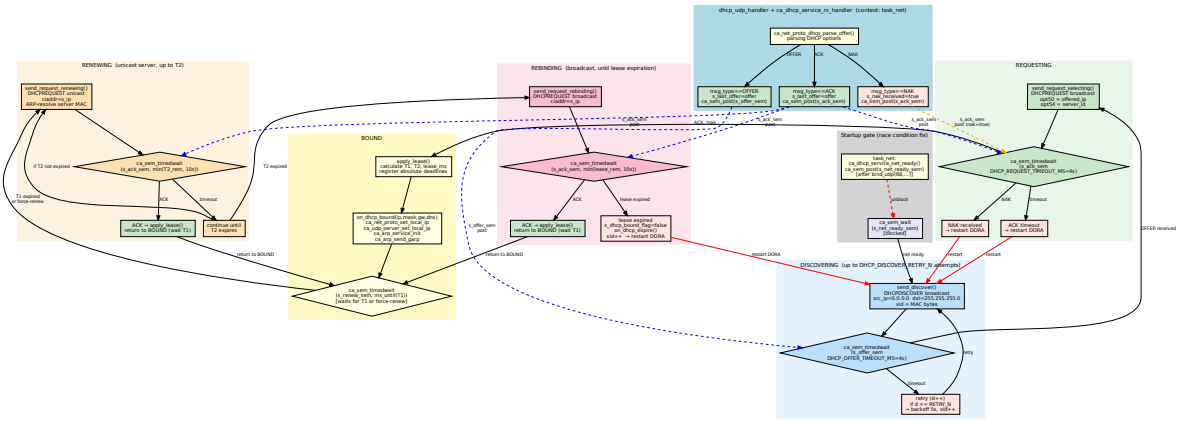


(source: doc/diagrams/ex35_arch.dot)

Initialization flow

(source: doc/diagrams/ex35_init_flow.dot)

DHCP DORA + renew state machine



(source: `doc/diagrams/ex35_dhcp_flow.dot`)

Memory map

Identical to examples 33 and 34:

Region	Address	Size	Usage	MPU
D2 SRAM1	0x30000000	128 KB	Net pool (64 × ca_net_buf_t, 2 KB)	Region 3 NCacheable
D2 SRAM3	0x30040000	32 KB	.eth_dma: tx_desc[4]+rx_desc[4]	Region 4 NCacheable

Differences compared to example 34

Aspect	Ex 34	Ex 35
IP address	static (192.168.10.34)	dynamic DHCP
Additional dependencies	EN_ARP_SERVICE, EN_UDP_SERVER	+ EN_NET_DHCP, EN_DHCP_SERVICE
LEVEL1 task	—	task_dhcp (DORA + renew)
Concurrent clients	4	2 (`MAX_CLIENTS`)
ARP init	static ip	ip=0, updated in `on_dhcp_bound`
local_ip udp server	static	0, updated in `on_dhcp_bound`
CLI	via `ca_udp_server`	+ integrated `ca_dhcp_service`
GARP	at startup	after every DHCP BOUND/renew

`task_net` remains at LEVEL2; `task_dhcp` is LEVEL1 to react to the OFFER before another task consumes the semaphore.

58.1.3 Source code

The file is located in `examples/example_35/ca_os_ex35.c`.

1. Network configuration — Dynamic IP

```
static const ca_mac_t s_mac = { .v = { 0x02U,0x00U,0x00U,0x00U,0x00U,0x35U } };

/* Dynamic IP     written by on_bound, read by task_net and the CLI */
static volatile ca_ipv4_t s_ip   = { .addr = 0U };
static volatile ca_ipv4_t s_mask = { .addr = 0U };
static volatile ca_ipv4_t s_gw   = { .addr = 0U };
static volatile ca_ipv4_t s_dns  = { .addr = 0U };
static volatile bool   s_dhcp_bound = false;

static const ca_net_cfg_t net_cfg = {
    .ip   = { .addr = 0U },   /* 0.0.0.0     updated after DHCP */
    .mac  = { .v = { 0x02U,0x00U,0x00U,0x00U,0x00U,0x35U } },
    .arp  = { .renew_ms = 30000U },
    .udp  = { .bindings = { {0U, NULL} }, .count = 0U },
    .icmp = { .echo_reply = true },
};
```

The stack starts with `IP=0.0.0.0`; the field is updated by `ca_net_proto_set_local_ip()` inside `on_dhcp_bound`.

2. UDP server configuration — dynamic local_ip

```
static ca_udp_server_cfg_t srv_cfg = {
    .port              = UDP_SRV_PORT,      /* 7961 */
    .max_clients       = MAX_CLIENTS,       /* 2 */
    .client_timeout_ms = CLIENT_TIMEOUT_MS, /* 15 s */
    .local_mac         = { .v = { 0x02U,0x00U,0x00U,0x00U,0x00U,0x35U } },
    .local_ip          = { .addr = 0U },    /* updated after DHCP */
    .on_rx             = on_rx,
    .on_connect        = on_connect,
    .on_disconnect     = on_disconnect,
};
```

`local_ip` is `0` at the time of the `ca_udp_server_start()` call; it is updated with `ca_udp_server_set_local_ip(ip)` in the DHCP callback. Without this update, UDP response frames would have `src_ip=0.0.0.0` and would be discarded by the PC.

3. Adapter for the UDP dispatcher (port 68)

```
static void dhcp_udp_handler(u16 buf_idx)
{
    ca_net_buf_t *b = ca_net_buf_ptr(buf_idx);
    u8 *frame = b->data + b->data_off;
    u16 l4_off = b->meta.rx.l4_off;
    const ca_udp_frame_t *udp = (const ca_udp_frame_t *)(frame + l4_off);
    u16 udp_total = (u16)(b->data_len - l4_off);
    ca_dhcp_service_rx_handler(udp, udp_total);
    ca_net_buf_free(buf_idx);
}
```

The dispatcher `ca_net_proto` calls UDP handlers with the signature `void handler(u16 buf_idx)`. The DHCP service instead exposes `ca_dhcp_service_rx_handler(const ca_udp_frame_t *udp, u16 len)`: this small adapter performs the conversion and frees the buffer.

4. DHCP Callback — multilayer update

```
static void on_dhcp_bound(ca_ipv4_t ip, ca_ipv4_t mask, ca_ipv4_t gw, ca_ipv4_t
    dns)
{
    s_ip = ip;  s_mask = mask;  s_gw = gw;  s_dns = dns;
    s_dhcp_bound = true;

    ca_net_proto_set_local_ip(ip);            /* proto stack */
    EVAL_UDP_SRV(ca_udp_server_set_local_ip(ip);)  /* UDP server TX */

#if EN_ARP_SERVICE
    ca_arp_service_update_local_ip(ip);       /* aggiorna local_ip ARP (no
    reinit) */
#endif

    ca_stdout_printf_n("\nDHCP BOUND  IP=%u.%u.%u.%u  mask=%u.%u.%u.%u",
        ip.v[0],ip.v[1],ip.v[2],ip.v[3],
        mask.v[0],mask.v[1],mask.v[2],mask.v[3]);
    ca_stdout_printf_n("            GW=%u.%u.%u.%u  DNS=%u.%u.%u.%u",
        gw.v[0],gw.v[1],gw.v[2],gw.v[3],
        dns.v[0],dns.v[1],dns.v[2],dns.v[3]);

    ca_arp_send_garp();
}
```

Three layers are updated in sequence: proto stack, UDP server, ARP service. The order is important: the GARP must start with the IP already set in the proto stack (otherwise, ARP response frames would still use `0.0.0.0`).

> **Note**: `ca_arp_service_update_local_ip(ip)` is used and **not** `ca_arp_service_init(s_mac, ip)`. The latter reinitializes the ARP service's internal semaphores while the task is running, causing a SAFEGUARD semaphore overflow upon the second DHCP bound.

5. task_net — gate and RX loop

```
static u32 task_net(void *arg)
{
    ca_usb_cdc_wait_connected(5000);
    ca_os_sleep(300);

    ca_net_proto_init(&net_cfg);
    ca_net_proto_bind_udp(DHCP_CLIENT_PORT, dhcp_udp_handler);
    EVAL_UDP_SRV(ca_udp_server_bind();)

    ca_eth_link_poll();
    ca_os_sleep(200);

    /* Unblocks the DHCP task     from here port 68 is ready */
    EVAL_DHCP_SVC(ca_dhcp_service_net_ready();)

    while (1) {
        ca_sem_wait(&ca_net_rx_sem);
        ca_net_process();
    }
}
```

Critical point: `ca_dhcp_service_net_ready()` must be called **after** `ca_net_proto_bind_udp(68, ...)`. Before this call, the DHCP task is blocked on `s_net_ready_sem` and cannot send a DISCOVER. Without this gate (race condition), the DHCP task (LEVEL1, highest priority) would

wake up first, send the DISCOVER, but port 68 would not yet be registered in the dispatcher → the OFFER would be silently discarded.

The task_net **does not call** `ca_dhcp_service_bind()`: that function is blocking and would freeze the net task → the `ca_net_process()` loop would not run → the OFFER/ACK packets would never be processed → deadlock. The IP arrives via the `on_dhcp_bound` callback from the DHCP task.

6. ca_os_configure and ca_os_main

```
void ca_os_configure(void)
{
    /* stdout, CLI, USB CDC, GPIO, ETH ... */
    if (!ca_net_init(&eth_cfg)) CA_SAFE_FAIL("ca_net_init");

    /* ARP service with a provisional IP; updated in on_dhcp_bound */
    EVAL_ARP_SVC({
        ca_ipv4_t zero = { .addr = 0U };
        ca_arp_service_init(s_mac, zero);
    })

    /* DHCP service: registers callbacks, inits semaphores */
    EVAL_DHCP_SVC(ca_dhcp_service_init(s_mac, on_dhcp_bound, on_dhcp_expire);)

    /* CLI dhcp commands */renew, udp*/list/kick/send registrati automaticamente
     * da ca_dhcp_service_init() e ca_udp_server_start() */
}

void ca_os_main(void)
{
    EVAL_ARP_SVC(ca_arp_service_start();)       /* task ARP (LEVEL1) */
    EVAL_DHCP_SVC(ca_dhcp_service_start();)     /* task dhcp (LEVEL1) */
    EVAL_UDP_SRV(ca_udp_server_start(&srv_cfg);) /* usrv0, usrv1 (LEVEL3) */

    ca_os_job("net",       CA_TH_LEVEL2, task_net,       ...);
    ca_os_job("eth_mon",   CA_TH_LEVEL3, task_eth_mon,   ...);
    ca_os_job("net_stats", CA_TH_LEVEL4, task_stats,     ...);
    ca_os_job("heartbeat", CA_TH_LEVEL4, task_heartbeat, ...);
    ca_os_run(true);
}
```

Mandatory order:

Step	Function	Where	Notes
1	`ca_net_init()`	`ca_os_configure`	pool + ETH + ISR
2	`ca_arp_service_init()`	`ca_os_configure`	init cache, no OS
3	`ca_dhcp_service_init()`	`ca_os_configure`	inits semaphores, no OS
4	`ca_arp_service_start()`	`ca_os_main`	create task (OS active)
5	`ca_dhcp_service_start()`	`ca_os_main`	create LEVEL1 task (blocked)
6	`ca_udp_server_start()`	`ca_os_main`	create usrv0..1 (blocked)
7	`ca_net_proto_init()`	`task_net`	register ARP/ICMP/UDP handler
8	`bind_udp(68)` + `udp_server_bind()`	`task_net`	register ports
9	`ca_dhcp_service_net_ready()`	`task_net`	unblock DHCP task

7. DHCP Service — internal state

```
typedef enum {
    DHCP_ST_INIT, DHCP_ST_DISCOVERING, DHCP_ST_REQUESTING,
```

```
    DHCP_ST_BOUND, DHCP_ST_RENEWING, DHCP_ST_REBINDING,
} dhcp_state_t;
```

Internal semaphores:

Semaphore	Producer	Consumer	Purpose
`s_net_ready_sem`	`task_net`	`task_dhcp`	gate: port 68 ready
`s_offer_sem`	`rx_handler`	`task_dhcp`	OFFER received
`s_ack_sem`	`rx_handler`	`task_dhcp`	ACK or NAK received
`s_renew_sem`	CLI (`dhcp renew`)	`task_dhcp`	force-renew
`s_dhcp_bound_flag_sem`	`task_dhcp`	`ca_dhcp_service_bind()`	BOUND event

8. Available CLI commands

Commands are automatically registered by services via `ca_cli_register()`. No hooks in the example.

Command	Service	Description
`dhcp info`	`ca_dhcp_service`	Current lease status (IP, mask, GW, DNS, T1/T2 timer)
`dhcp renew`	`ca_dhcp_service`	Force early renew
`udp stats`	`ca_udp_server`	Pool + active clients
`udp list`	`ca_udp_server`	IP:port + ring of each slot
`udp kick <id>`	`ca_udp_server`	Force slot closure
`udp send <id> <msg>`	`ca_udp_server`	Send data to active client

58.1.4 Test procedure from PC

The `examples/example_35/test_dhcp_pc.sh` script automates the entire procedure. It requires `dnsmasq`, `ip`, `ping`, `nc` (netcat-openbsd).

```
sudo ./examples/example_35/test_dhcp_pc.sh enp7s0
```

The script: 1. Assigns `192.168.10.1/24` to the specified interface. 2. Starts `dnsmasq` with pool `192.168.10.100 200`, 1 h lease. 3. Waits up to 60 s for `DHCPACK` to appear in the dnsmasq log. 4. Executes `ping -c4` to the board. 5. Sends `hello caOS` on UDP port 7961 and verifies the `ECHO:` response. 6. Launches 2 simultaneous UDP clients. 7. Prints the complete dnsmasq DHCP log. 8. Upon pressing Ctrl+C, performs cleanup (stop dnsmasq, remove IP).

Equivalent manual test:

```
# Setup PC
sudo ip addr add 192.168.10.1/24 dev eth0
# (starts dnsmasq or uses an existing DHCP server)

# Ping
ping <ip_assegnato>

# Echo UDP
echo "hello caOS" | nc -u -w2 <ip_assegnato> 7961

# 2 simultaneous clients
echo -n "client_one" | nc -u -w2 <ip_assegnato> 7961 &
echo -n "client_two" | nc -u -w2 <ip_assegnato> 7961 &
wait
```

58.1.5 Expected output

MCU Console (USB CDC)

```
=== EX35: DHCP + ARP + ping + UDP server ===
MAC  02:00:00:00:00:35
IP   --- (waiting for DHCP)
DHCP: avvio DORA...

DHCP BOUND  IP=192.168.10.135  mask=255.255.255.0
            GW=192.168.10.1  DNS=8.8.8.8




================================================
 -- C.A.O.S --
   build : 5cbd0d57+dev - 2026-04-18 08:49:04
   MCU : STM 32 F:H7xx T:753
================================================
   System Clock: 480000000 Hz
   Total memory: 512064 B
   SysTick: 47999 clocks per tick
   Core Temperature 35 Celsius
================================================
UDP[7961] client 0: 192.168.10.1:44203 connesso
UDP[7961] cid=0 rx=18 bytes from 192.168.10.1:44203
UDP[7961] client 1: 192.168.10.1:57790 connesso
UDP[7961] cid=1 rx=18 bytes from 192.168.10.1:57790
STATS dhcp=BOUND pool=56/64 held=8 rx=17 drop=0 tx=10 cli=2
STATS dhcp=BOUND pool=56/64 held=8 rx=20 drop=0 tx=10 cli=2
UDP[7961] client 0: 192.168.10.1:44203 disconnesso
UDP[7961] client 1: 192.168.10.1:57790 disconnesso
STATS dhcp=BOUND pool=56/64 held=8 rx=20 drop=0 tx=10 cli=0
```

Script test_dhcp_pc.sh (PC side)

```
============================================================
 caOS EX35 - DHCP test  (interface: enp7s0)
============================================================
[*] Configure enp7s0  ->  192.168.10.1/255.255.255.0
[*] Link UP
[*] Starting dnsmasq  (pool 192.168.10.100 - 192.168.10.200)
[*] dnsmasq running (pid: 24186)
[*] Waiting for DHCP ACK...
[*] Waiting... 3s
[+] DHCP ACK sent to: 192.168.10.135

============================================================
 Test ping
============================================================
PING 192.168.10.135 (192.168.10.135) 56(84) bytes of data.
64 bytes from 192.168.10.135: icmp_seq=1 ttl=64 time=0.660 ms
64 bytes from 192.168.10.135: icmp_seq=2 ttl=64 time=0.407 ms
64 bytes from 192.168.10.135: icmp_seq=3 ttl=64 time=0.386 ms
64 bytes from 192.168.10.135: icmp_seq=4 ttl=64 time=0.418 ms
4 packets transmitted, 4 received, 0% packet loss
[OK] ping

============================================================
 UDP echo test (port 7961)
```

```
============================================================
[*] Sending 'hello caOS' to 192.168.10.135:7961 ...
[OK] Echo received: ECHO: hello caOS

============================================================
 Test 2 simultaneous UDP clients
============================================================
[*] Client 1...
[*] Client 2...
ECHO: client_one
[*] Both clients done (check the USB console for ECHO)
```

58.1.6 Technical notes

Race conditions and gate `s_net_ready_sem`

The DHCP task is at LEVEL1 (more urgent than the task_net at LEVEL2). Without explicit synchronization, the DHCP task would wake up before the task_net and send the DISCOVER when port 68 is not yet registered in the dispatcher. The OFFER from the DHCP server would be silently discarded.

The solution is a gate semaphore (`s_net_ready_sem`) with an initial value of 0: the DHCP task blocks on `ca_sem_wait(&s_net_ready_sem)` and resumes only when the task_net calls `ca_dhcp_service_net_ready()` after completing `ca_net_proto_bind_udp(68, ...)`.

Deadlock and prohibition of `ca_dhcp_service_bind()` in task_net

`ca_dhcp_service_bind()` blocks the caller until the BOUND arrives or `DHCP_BIND_TIMEOUT_MS` expires. If called by `task_net`, it freezes the `ca_net_process()` loop: the OFFER and ACK packets are never processed → the BOUND semaphore is never posted → permanent deadlock.

The function can only be used by application tasks that are not part of the RX path (e.g., a configuration task that waits for the IP before opening a TCP connection).

Updating `local_ip` of the UDP server

`ca_udp_server_start()` copies the passed configuration. The `s_srv_cfg.local_ip` field is used in every TX frame to construct the source IP header. If it remains `0.0.0.0`, the PC receives responses with an invalid source IP and discards them (or the kernel fails to deliver them to the UDP socket).

`ca_udp_server_set_local_ip()` is a `static inline` that updates `s_udp_srv_cfg.local_ip` directly via the extern variable exposed by the header. It is not necessary to reinitialize the server.

Task priorities

```
LEVEL1: task_dhcp       DORA + T1/T2 renew (alta priorit      reagisce subito
    all'OFFER)
         task_arp       ARP cache (same priority, does not interfere)
LEVEL2: task_net        processing frame RX
LEVEL3: task_eth_mon     link PHY poll
         usrv0, usrv1     worker UDP
LEVEL4: task_stats      statistics every 10 s
         task_heartbeat     LED
```

T1/T2 and lease renewal

By default, the dnsmasq server sends T1 = lease/2, T2 = lease×7/8. If the optional fields are absent, the DHCP service calculates them autonomously:

```
u32 t1_s = (s_last_offer.t1 != 0U) ? s_last_offer.t1 : (lease_s / 2U);
u32 t2_s = (s_last_offer.t2 != 0U) ? s_last_offer.t2
                                   : (lease_s - lease_s / 8U);
```

The force-renew from CLI (`dhcp renew`) posts `s_renew_sem` which interrupts the `ca_sem_timedwait` of the T1 wait → the task immediately transitions to RENEWING.

Conditional compilation

```
#if EN_ETH && EN_NET && EN_NET_DHCP && EN_DHCP_SERVICE && EN_UDP_SERVER
/* ... all the ex35 code ... */
#endif
```

`EVAL_DHCP_SVC(expr)`, `EVAL_ARP_SVC(expr)`, `EVAL_UDP_SRV(expr)` are macros defined in `os_config.h` that execute `expr` only if the respective service is enabled.

Chapter 59

DHCP + NTP + ARP + ICMP Ping + UDP Server

59.1 Example 36 — DHCP + NTP + ARP + ICMP ping + UDP server

59.1.1 Introduction

Example 36 adds the **NTP client** to the caOS network stack, still on the NUCLEO-H753ZI (STM32H753ZI, PHY LAN8742A RMII). Objectives:

1. **DHCP client**: obtains IP automatically (DORA + T1/T2 renew).
2. **NTP client**: synchronizes unix time with the server indicated by DHCP (option 42) or with the gateway as a fallback.
3. **ARP service**: MACIP cache, updated with the new IP after every BOUND.
4. **ICMP echo reply** (ping) — functional as soon as DHCP is BOUND.
5. **UDP server** on port 7961 with up to 2 simultaneous clients.
6. **GARP** sent after every DHCP bound/renew.
7. **CLI** integrated into the services — commands automatically registered by `ca_dhcp_service`, `ca_ntp_service`, `ca_arp_service`, `ca_udp_server`.

Compared to example 35, the new element is the NTP task synchronized via a second gate: `ca_ntp_service_net_ready()` is called by `on_dhcp_bound` after the IP has been assigned, ensuring that the NTP task does not send requests with IP=0.

Hardware (NUCLEO-H753ZI, MB1364)

Identical to examples 33–35 — same RMII pinout, same LEDs.

Signal	MCU Pin	Description
LED1 (green)	PB0	On = link UP
LED3 (red)	PB14	Heartbeat 500 ms
ETH_RMII_REF_CLK	PA1	RMII 50 MHz clock from PHY
ETH_MDIO	PA2	MDIO PHY management data
ETH_RMII_CRS_DV	PA7	Carrier Sense / Data Valid
ETH_MDC	PC1	MDIO clock
ETH_RMII_RXD0	PC4	RX data bit 0
ETH_RMII_RXD1	PC5	RX data bit 1

Signal	MCU Pin	Description
ETH_RMII_TX_EN	PG11	Transmission enabled
ETH_RMII_TXD1	PB13	TX data bit 1
ETH_RMII_TXD0	PG13	TX data bit 0

Configuration required in `os_config.h`

```
#define EN_ETH          1   /* driver Ethernet HW */
#define EN_NET          1   /* stack net_proto */
#define EN_NET_DHCP     1   /* parser DHCP */
#define EN_NET_NTP      1   /* parser NTP */
#define EN_ARP_SERVICE  1   /* ARP cache + resolve */
#define EN_UDP_SERVER   1   /* UDP server multi-client */
#define EN_DHCP_SERVICE 1   /* DHCP client DORA + T1/T2 renew */
#define EN_NTP_SERVICE  1   /* NTP client with polling */
```

59.1.2 System architecture

Differences compared to example 35

Aspect	Ex 35	Ex 36
NTP client	no	yes (`ca_ntp_service`)
Additional dependencies	—	+`EN_NET_NTP`, `EN_NTP_SERVICE`
Additional tasks	—	`task_ntp` (LEVEL1, blocked on `s_ntp_ev_q`)
NTP gate	—	`ca_ntp_service_net_ready()` in `on_dhcp_bound`
CLI	integrated dhcp+udp	+`ntp info/sync`, `arp info` integrated
STATS	`dhcp=%s`	`dhcp=%s ntp=%s`

Init flow

```
ca_os_configure():
  ca_net_init()
  ca_arp_service_init(mac, 0.0.0.0)
  ca_dhcp_service_init(mac, on_bound, on_expire)
  ca_ntp_service_init(fallback_ip, mac, on_sync, on_fail)

ca_os_main():
  ca_arp_service_start()    -- task LEVEL0 (blocked on req_q)
  ca_dhcp_service_start()   -- task LEVEL1 (blocked on net_ready_sem)
  ca_ntp_service_start()    -- task LEVEL1 (blocked on ev_q)
  ca_udp_server_start()     -- usrv0..1 (LEVEL3, blocked)

task_net:
  ca_net_proto_init()
  ca_net_proto_bind_udp(68)     -- DHCP
  ca_udp_server_bind()          -- porta 7961
  ca_ntp_service_bind()         -- porta 123
  ca_dhcp_service_net_ready()   -- sblocca DHCP
  [loop: ca_sem_wait + ca_net_process]

on_dhcp_bound():
  ca_net_proto_set_local_ip(ip)
  ca_udp_server_set_local_ip(ip)
```

```
  ca_arp_service_update_local_ip(ip)
  ca_arp_send_garp()
  ca_ntp_service_set_server(opt42 o gw)
  ca_ntp_service_net_ready()    -- sblocca NTP
```

Task priorities

```
LEVEL0: task_arp         cache ARP (slot pool + queue)
LEVEL1: task_dhcp        DORA + T1/T2 renew
         task_ntp         NTP polling (blocked on ev_q between polls)
LEVEL2: task_net         processing frame RX
LEVEL3: task_eth_mon      link PHY poll
         usrv0, usrv1      worker UDP
LEVEL4: task_stats       statistics every 10 s
         task_heartbeat      LED
```

59.1.3 The source code

The file is located in `examples/example_36/ca_os_ex36.c`.

1. NTP server — from DHCP option 42

```c
static void on_dhcp_bound(ca_ipv4_t ip, ca_ipv4_t mask, ca_ipv4_t gw, ca_ipv4_t
    dns)
{
    /* update the proto stack, UDP server, ARP service */
    ca_net_proto_set_local_ip(ip);
    EVAL_UDP_SRV(ca_udp_server_set_local_ip(ip);)
    EVAL_ARP_SVC(ca_arp_service_update_local_ip(ip);)

    ca_arp_send_garp();

    /* Update the NTP IP: use opt42 from DHCP, fall back to the gateway */
    EVAL_NTP_SVC({
        ca_ipv4_t ntp_ip = ca_dhcp_service_get_ntp_server();
        if (ntp_ip.addr == 0U) ntp_ip = gw;
        ca_ntp_service_set_server(ntp_ip);
        ca_ntp_service_net_ready();   /* sblocca task NTP */
    })
}
```

`ca_dhcp_service_get_ntp_server()` returns the IP from the option 42 field of the ACK. If the DHCP server does not provide option 42 (common with simple dnsmasq), the gateway is used as a fallback NTP server.

`ca_ntp_service_net_ready()` is the gate for the NTP task: without this call, the task would remain blocked on the event queue and would never send requests.

2. NTP Callbacks

```c
static void on_ntp_sync(u32 unix_time)
{
    u16 yr; u8 mo, dy, hh, mm, ss;
    unix_to_dt(unix_time, &yr, &mo, &dy, &hh, &mm, &ss);
    ca_stdout_printf_n(
        "NTP SYNC  unix=%u  %04u-%02u-%02u %02u:%02u:%02u UTC", ...);
}

```

```
static void on_ntp_fail(void)
{
    ca_stdout_put_n("NTP FAIL  server unreachable");
}
```

Callbacks are called in the context of the **NTP task** (LEVEL1), not in `task_net`. They can perform blocking operations without impacting the RX path.

`unix_to_dt()` converts the UNIX timestamp into year/month/day/hour/minute/second, handling leap years and months with 28/30/31 days.

3. NTP Configuration

```
/* ca_os_configure(): server placeholder, aggiornato da on_dhcp_bound */
EVAL_NTP_SVC(ca_ntp_service_init(
    NTP_SERVER_IP_FALLBACK,   /* { 192, 168, 10, 1 } */
    s_mac,
    on_ntp_sync,
    on_ntp_fail
);)
```

The fallback `192.168.10.1` is the test PC gateway — usually the PC has an active NTP server (either ntpd or systemd-timesyncd).

4. `task_net` — port binding

```
ca_net_proto_bind_udp(DHCP_CLIENT_PORT, dhcp_udp_handler);  /* 68 */
EVAL_UDP_SRV(ca_udp_server_bind();)                          /* 7961 */
EVAL_NTP_SVC(ca_ntp_service_bind();)                         /* 123 */
```

The NTP port (123) must be registered before unlocking DHCP, otherwise the NTP task might send a request before the port is in the dispatcher and lose the response.

5. Available CLI commands

Automatically registered by services via `ca_cli_register()`. No hooks in the example.

Command	Service	Description
`dhcp info`	`ca_dhcp_service`	Lease status: IP, MASK, GW, DNS, T1/T2 timers
`dhcp renew`	`ca_dhcp_service`	Force early renew
`ntp info`	`ca_ntp_service`	Unix timestamp + formatted date/time
`ntp sync`	`ca_ntp_service`	Force immediate sync
`arp info`	`ca_arp_service`	Print ARP cache with age
`udp stats`	`ca_udp_server`	Pool + active clients
`udp list`	`ca_udp_server`	IP:port + ring usage for each slot
`udp kick <id>`	`ca_udp_server`	Disconnect client
`udp send <id> <msg>`	`ca_udp_server`	Send data to active client

59.1.4 Test procedure from PC

The `examples/example_36/test_dhcp_ntp_pc.sh` script executes the complete procedure. It requires `dnsmasq`, `ntp` (or `ntpdate`).

```
sudo ./examples/example_36/test_dhcp_ntp_pc.sh enp7s0
```

Manual test:

```
# Setup PC
sudo ip addr add 192.168.10.1/24 dev eth0
# start dnsmasq
sudo dnsmasq --no-daemon --interface=eth0 \
  --dhcp-range=192.168.10.100,192.168.10.200,1h \
  --dhcp-option=option:ntp-server,192.168.10.1

# Ping (after DHCP bound)
ping <ip_assegnato>

# NTP check: ntpd or chronyd must be running on the PC
# The board queries it on port 123 of the PC
```

59.1.5 Expected output

```
=== EX36: DHCP + NTP + ARP + ping + UDP server ===
MAC  02:00:00:00:00:36
IP   --- (waiting for DHCP)
DHCP: avvio DORA...

DHCP BOUND  IP=192.168.10.136  mask=255.255.255.0
            GW=192.168.10.1  DNS=192.168.10.1

NTP SYNC  unix=1745000000  2025-04-18 12:33:20 UTC

STATS dhcp=BOUND ntp=SYNC pool=56/64 held=8 rx=34 drop=0 tx=12 cli=0
```

CLI

```
> ntp info
NTP SYNCED  unix=1745000000
            2025-04-18 12:33:20 UTC

> ntp sync
NTP sync forzato

> arp cache
ARP cache:
  192.168.10.1  C0:25:A5:51:6C:13  age=4500ms
```

59.1.6 Technical notes

Two-level gate: DHCP then NTP

The NTP task must wait for two events in sequence: 1. **DHCP BOUND** — to have a valid source IP in the NTP request frames 2. **NTP server available** — to know the server IP

The gate `s_net_ready_sem` (inside `ca_ntp_service_net_ready()`) covers both: it is called only by `on_dhcp_bound`, which has already updated the IP and the NTP server. Before this call, the NTP task is blocked on the event queue.

Port 123 and `ca_ntp_service_bind()`

Port 123 (UDP) must be registered in the dispatcher **before** the NTP task sends the first request. If the binding were to happen later, the server response would arrive on port 123 but there would be no handler → `rx_no_handler++` and the response is discarded. The NTP task would time out and retry at the next poll.

Conditional compilation

```
#if EN_ETH && EN_NET && EN_NET_DHCP && EN_NET_NTP && \
    EN_DHCP_SERVICE && EN_NTP_SERVICE && EN_UDP_SERVER
```

`EVAL_NTP_SVC(expr)` is a macro defined in `os_config.h` that executes `expr` only if `EN_NTP_SERVICE=1`.

Chapter 60

DHCP + NTP + DNS + ARP + ICMP Ping + UDP Server

60.1 Example 37 — DHCP + NTP + DNS + ARP + ICMP ping + UDP server

60.1.1 Introduction

Example 37 adds the **DNS client** to the network stack of caOS, on NUCLEO-H753ZI (STM32H753ZI, PHY LAN8742A RMII). Objectives:

1. **DHCP client**: automatically obtains IP, gateway and DNS server.
2. **NTP client**: synchronizes the clock via DHCP option 42 or fallback to gateway.
3. **DNS client**: resolves hostnames to IPv4 addresses using the DNS server from DHCP (opt 6).
4. **ARP service**: MACIP cache with pool slots and queue — queue-based version.
5. **ICMP echo reply** (ping).
6. **UDP server** on port 7961 with up to 2 simultaneous clients.
7. **task_dns_demo**: at startup resolves a list of demo hostnames; the built-in CLI in `ca_dns_service` allows interactive resolutions with `dns resolve <hostname>`.

Compared to example 36, the new element is the DNS service. The task `task_dns_demo` demonstrates how to use `ca_dns_resolve()` in a blocking manner from an application task.

Hardware (NUCLEO-H753ZI, MB1364)

Identical to examples 33–36 — same RMII pinout, same LEDs.

Required configuration in `os_config.h`

```
#define EN_ETH          1
#define EN_NET          1
#define EN_NET_DHCP     1
#define EN_NET_NTP      1
#define EN_NET_DNS      1
#define EN_ARP_SERVICE  1
#define EN_UDP_SERVER   1
#define EN_DHCP_SERVICE 1
#define EN_NTP_SERVICE  1
#define EN_DNS_SERVICE  1
```

60.1.2 System Architecture

Differences from example 36

Aspect	Ex 36	Ex 37
DNS client	no	yes (`ca_dns_service`)
Additional dependencies	—	+`EN_NET_DNS`, `EN_DNS_SERVICE`
Additional tasks	—	`task_dns_demo` (LEVEL3), `task_dns` (LEVEL1)
DNS Gate	—	`ca_dns_service_net_ready()` in `on_dhcp_bound`
Demo semaphore	—	`s_dns_demo_sem`: DHCP→task_dns_demo
CLI	integrated dhcp/ntp/arp/udp	+`dns resolve`, `dns server` integrated

Flow `on_dhcp_bound`

```
static void on_dhcp_bound(ca_ipv4_t ip, ca_ipv4_t mask, ca_ipv4_t gw, ca_ipv4_t
    dns)
{
    ca_net_proto_set_local_ip(ip);
    EVAL_UDP_SRV(ca_udp_server_set_local_ip(ip);)
    EVAL_ARP_SVC(ca_arp_service_update_local_ip(ip);)
    ca_arp_send_garp();

    /* NTP: option 42 or gateway fallback */
    EVAL_NTP_SVC({
        ca_ipv4_t ntp_ip = ca_dhcp_service_get_ntp_server();
        if (ntp_ip.addr == 0U) ntp_ip = gw;
        ca_ntp_service_set_server(ntp_ip);
        ca_ntp_service_net_ready();
    })

    /* DNS: option 6 or gateway fallback */
    EVAL_DNS_SVC({
        ca_ipv4_t dns_ip = dns;
        if (dns_ip.addr == 0U) dns_ip = gw;
        ca_dns_service_set_server(dns_ip);
        ca_dns_service_net_ready();
        ca_sem_trywait(&s_dns_demo_sem);   /* drain */
        ca_sem_post(&s_dns_demo_sem);      /* sblocca task_dns_demo */
    })
}
```

Two independent gates: NTP and DNS are both unlocked by `on_dhcp_bound`, but remain separate tasks that operate concurrently.

Task Priority

```
LEVEL0: task_arp          cache ARP
LEVEL1: task_dhcp         DORA + T1/T2
         task_ntp          polling NTP
         task_dns          gestione query DNS
LEVEL2: task_net          processing frame RX
LEVEL3: task_eth_mon      link PHY poll
         usrv0..1         worker UDP
         task_dns_demo     risoluzioni demo all'avvio
LEVEL4: task_stats        statistiche
         task_heartbeat     LED
```

60.1.3 Source code

The file is located in `examples/example_37/ca_os_ex37.c`.

1. DNS demo semaphore

```
static ca_semaphore_t s_dns_demo_sem;   /* on_dhcp_bound     task_dns_demo */
```

Initialized in `ca_os_configure()` with value 0 (blocking). It is posted by `on_dhcp_bound` at the time of BOUND. `task_dns_demo` waits for this semaphore before starting the resolutions.

The `trywait + post` pattern ensures that if DHCP makes multiple renews, the semaphore does not accumulate posts: `trywait` drains the previous value before posting again.

2. task_dns_demo

```
static const char * const s_demo_hosts[] = {
    "test.caos.local",
    "gateway.caos.local",
    "pool.ntp.org",
};

static u32 task_dns_demo(void *arg)
{
    ca_sem_wait(&s_dns_demo_sem);   /* waits for DHCP BOUND */

    for (u32 h = 0U; h < DEMO_HOSTS_N; h++) {
        ca_ipv4_t ip = { .addr = 0U };
        bool ok = ca_dns_resolve(s_demo_hosts[h], &ip, 10000U);
        if (ok) {
            ca_stdout_printf_n("DNS  %-24s  ->  %u.%u.%u.%u", ...);
        } else {
            ca_stdout_printf_n("DNS  %-24s  ->  FAILED", s_demo_hosts[h]);
        }
        ca_os_sleep(500U);
    }
    ca_stdout_put_n("DNS: demo done - use the CLI 'dns resolve <hostname>'");
    return 0U;   /* task termina */
}
```

`ca_dns_resolve()` is blocking: the caller suspends until the response arrives or the timeout expires. It does not block `task_net` because it runs on a separate task (LEVEL3).

The hostname `.caos.local` usually fails on external DNS servers (NXDOMAIN) — this is to test error handling. `pool.ntp.org` requires that the PC has internet access or a DNS server that responds.

3. Available CLI commands

Automatically registered by services via `ca_cli_register()`. No hook in the example.

Command	Service	Description
`dhcp info`	`ca_dhcp_service`	Current lease status
`dhcp renew`	`ca_dhcp_service`	Force early renew
`ntp info`	`ca_ntp_service`	Unix timestamp + date/time
`ntp sync`	`ca_ntp_service`	Force immediate sync
`dns resolve <host>`	`ca_dns_service`	Resolve hostname to IPv4 (blocking max 10 s)
`dns server`	`ca_dns_service`	Show current DNS server IP

Command	Service	Description
`arp info`	`ca_arp_service`	Print ARP cache with age
`ip info`	`ca_net_proto`	IP, mask, GW, MTU

Warning: `dns resolve` blocks the CLI task until response (max 10 s).

4. `ca_os_configure` and `ca_os_main`

```c
void ca_os_configure(void)
{
    /* ... stdout, CLI, USB CDC, GPIO, ETH ... */
    ca_sem_init(&s_dns_demo_sem, "DNSDMO", 0U, 1U);

    ca_arp_service_init(s_mac, zero);          /* IP=0, aggiornato in on_bound
    */
    ca_dhcp_service_init(s_mac, on_bound, on_expire);
    ca_ntp_service_init(NTP_FALLBACK, s_mac, on_sync, on_fail);
    ca_dns_service_init(DNS_FALLBACK, s_mac);
}

void ca_os_main(void)
{
    ca_arp_service_start();
    ca_dhcp_service_start();
    ca_ntp_service_start();
    ca_dns_service_start();
    ca_udp_server_start(&srv_cfg);

    ca_os_job("net",        CA_TH_LEVEL2, task_net,       ...);
    ca_os_job("dns_demo",   CA_TH_LEVEL3, task_dns_demo,  ...);
    ca_os_job("eth_mon",    CA_TH_LEVEL3, task_eth_mon,   ...);
    ca_os_job("net_stats",  CA_TH_LEVEL4, task_stats,     ...);
    ca_os_job("heartbeat",  CA_TH_LEVEL4, task_heartbeat,...);
    ca_os_run(true);
}
```

5. Port binding in `task_net`

```c
ca_net_proto_bind_udp(DHCP_CLIENT_PORT, dhcp_udp_handler);  /* 68 */
EVAL_UDP_SRV(ca_udp_server_bind();)                          /* 7961 */
EVAL_NTP_SVC(ca_ntp_service_bind();)                         /* 123 */
EVAL_DNS_SVC(ca_dns_service_bind();)                         /* 53 */
```

All bindings occur **before** `ca_dhcp_service_net_ready()`: ensuring that ports 123 and 53 are registered in the dispatcher before the NTP/DNS tasks begin sending requests.

60.1.4 Testing procedure from PC

```bash
# Interface setup
sudo ip addr add 192.168.10.1/24 dev eth0

# DHCP server with NTP and DNS
sudo dnsmasq --no-daemon --interface=eth0 \
  --dhcp-range=192.168.10.100,192.168.10.200,1h \
  --dhcp-option=option:ntp-server,192.168.10.1 \
  --dhcp-option=option:dns-server,192.168.10.1
```

```

# Simple local DNS     answers for *.caos.local
# (aggiungere in /etc/hosts o usare dnsmasq address=/.caos.local/192.168.10.1)
```

60.1.5 Expected Output

```
=== EX37: DHCP + NTP + DNS + ARP + ping + UDP server ===
MAC  02:00:00:00:00:37
IP   --- (waiting for DHCP)
DHCP: avvio DORA...

DHCP BOUND  IP=192.168.10.137  mask=255.255.255.0
            GW=192.168.10.1  DNS=192.168.10.1

NTP SYNC  unix=1745000000  2025-04-18 12:33:20 UTC

DNS: avvio risoluzioni demo...
DNS  test.caos.local               FAILED
DNS  gateway.caos.local            FAILED
DNS  pool.ntp.org                  185.125.190.58
DNS: demo done - use the CLI 'dns resolve <hostname>'

STATS dhcp=BOUND ntp=SYNC pool=56/64 held=8 rx=41 drop=0 tx=18
```

CLI

```
> dns resolve pool.ntp.org
DNS: risolvo 'pool.ntp.org'...
DNS  pool.ntp.org       185.125.190.58

> dns server
DNS server: 192.168.10.1

> arp cache
ARP cache:
  192.168.10.1  C0:25:A5:51:6C:13  age=2300ms
```

60.1.6 Technical Notes

`ca_dns_resolve()` — Internal operation

`ca_dns_resolve(hostname, &ip, timeout_ms)` is blocking:

1. Searches internal DNS cache (hit → returns immediately)
2. Cache miss: allocates a request slot, sends UDP query to DNS server
3. Waits for response on slot semaphore (max `timeout_ms`)
4. Response received: updates cache, writes `*ip`, returns `true`
5. Timeout: frees slot, returns `false`

The DNS service uses the same slot-pool architecture as the ARP service: up to `DNS_PENDING_MAX` concurrent callers, each with dedicated semaphores.

Conditional compilation

```
#if EN_ETH && EN_NET && EN_NET_DHCP && EN_NET_NTP && EN_NET_DNS && \
    EN_DHCP_SERVICE && EN_NTP_SERVICE && EN_DNS_SERVICE && EN_UDP_SERVER
```

Chapter 61

DHCP + mDNS Responder + ARP + ICMP Ping + UDP Server

61.1 Example 38 — DHCP + mDNS responder + ARP + ICMP ping + UDP server

61.1.1 Introduction

Example 38 introduces the **mDNS responder** (Multicast DNS, RFC 6762) in caOS, on NUCLEO-H753ZI (STM32H753ZI, PHY LAN8742A RMII). Objectives:

1. **DHCP client**: automatically obtains IP (DORA + T1/T2 renew).
2. **mDNS responder**: the board announces itself as `caos-board.local` and responds to multicast queries on `224.0.0.251:5353` without any DNS server.
3. **ARP service**: MACIP cache for UDP replies.
4. **ICMP echo reply** (ping).
5. **UDP server** on port 7961 with up to 2 simultaneous clients.
6. **GARP + mDNS announce** sent after every DHCP bound.
7. **Integrated CLI** in services — commands `dhcp info/renew`, `mdns info`, `arp info` automatically registered by `ca_dhcp_service`, `ca_mdns_service`, `ca_arp_service`.

With mDNS active, it is not necessary to know the dynamic IP assigned by DHCP: `ping caos-board.local` and `nc -u caos-board.local 7961` work directly from the PC as soon as the board is BOUND.

Hardware (NUCLEO-H753ZI, MB1364)

Identical to examples 33–37 — same RMII pinout, same LEDs.

Required Configuration in `os_config.h`

```
#define EN_ETH           1
#define EN_NET           1
#define EN_NET_DHCP      1
#define EN_NET_MDNS      1
#define EN_ARP_SERVICE   1
#define EN_UDP_SERVER    1
#define EN_DHCP_SERVICE  1
#define EN_MDNS_SERVICE  1
```

61.1.2 System Architecture

Differences from example 35

Aspect	Ex 35	Ex 38
mDNS responder	no	yes (`ca_mdns_service`)
Hostname .local	—	`caos-board.local`
Additional dependencies	—	+`EN_NET_MDNS`, `EN_MDNS_SERVICE`
Additional tasks	—	`task_mdns` (LEVEL1)
mDNS Gate	—	`ca_mdns_service_net_ready(ip)` in `on_dhcp_bound`
Additional UDP port	—	5353 (multicast 224.0.0.251)
NTP/DNS	no	no
CLI	integrated dhcp+udp	`dhcp info/renew`, `mdns info`, `arp info` integrated

---

mDNS Service Architecture

```
task_mdns (LEVEL1)
         ca_sem_wait(s_announce_sem)               sbloccato da
    ca_mdns_service_net_ready()
         send_announce()                           gratuitous announce all'avvio
         loop: ca_sem_timedwait(s_query_sem)
                   unblocked by ca_mdns_service_rx_handler() for each query

task_net (LEVEL2)
         ca_net_process():
       porta 5353     mdns_udp_rx()
             ca_net_proto_mdns_parse_query()
             is_for_us?     ca_sem_post(s_query_sem)
```

Two semaphores separate the startup flow (announce one-shot) from the continuing flow (query response): - `s_announce_sem` (max=1): `ca_mdns_service_net_ready()` → task - `s_query_sem` (max=8): `ca_mdns_service_rx_handler()` → task

Multicast Flow

```
Ethernet: dst=01:00:5E:00:00:FB (gruppo mDNS)
IPv4:     dst=224.0.0.251  TTL=255
UDP:      src=5353  dst=5353
DNS:      ID=0  QR=1  AA=1  ANCOUNT=1
Answer:   caos-board.local     A     <ip_board>  (CLASS|CACHE_FLUSH)
```

The ETH `MACPFR_RA` MAC filter (Receive All) is required to receive multicast mDNS frames; it is already configured in `eth_h7xx.c`.

Task Priority

```
LEVEL0: task_arp         cache ARP
LEVEL1: task_dhcp        DORA + T1/T2
         task_mdns       announce + query reply
LEVEL2: task_net         processing frame RX (include porta 5353)
LEVEL3: task_eth_mon     link PHY poll
         usrv0..1        worker UDP
LEVEL4: task_stats       statistiche
         task_heartbeat     LED
```

61.1.3 Source code

The file is located in `examples/example_38/ca_os_ex38.c`.

1. Hostname mDNS

```
#define MDNS_HOSTNAME  "caos-board"   /* risponde a caos-board.local */
```

Must match the name used by the PC tests. Editable without recompiling the service: it is a string passed to `ca_mdns_service_init()`.

2. `on_dhcp_bound` — update sequence

The `on_dhcp_bound` update sequence is a critical process in the caOS real-time operating system, designed to ensure safe and reliable firmware updates for STM32 microcontroller-based systems. This sequence is implemented to prevent system crashes or corrupted states during the update process, which could otherwise lead to device lockups or data loss.

The update sequence begins with the initialization of the update mechanism, where the system verifies the integrity of the new firmware image. This verification includes checking the image's checksum and ensuring it is compatible with the target hardware. Once the verification is complete, the system prepares the necessary resources for the update, including allocating memory and configuring the required peripherals.

The actual update process involves writing the new firmware to the device's flash memory. This is done in a manner that ensures atomicity, meaning that either the entire update succeeds or it fails completely, preventing partial updates that could corrupt the system. The system also implements a rollback mechanism, which allows the device to revert to the previous firmware version if the update fails or is deemed unstable.

During the update, the system maintains a watchdog timer to monitor the progress and ensure that the update does not hang or get stuck. If the watchdog timer expires, the system will trigger a reset to prevent indefinite hanging.

Finally, after a successful update, the system performs a verification step to confirm that the new firmware is correctly installed and operational. This step includes running a basic self-test to ensure that all system components are functioning as expected. If all tests pass, the system proceeds to normal operation with the updated firmware.

The `on_dhcp_bound` update sequence is designed to be robust and fault-tolerant, ensuring that even in the event of unexpected errors or power failures during the update process, the system remains stable and recoverable. This is particularly important in embedded systems where reliability and uptime are critical factors.

```
static void on_dhcp_bound(ca_ipv4_t ip, ca_ipv4_t mask, ca_ipv4_t gw, ca_ipv4_t
    dns)
{
    (void)dns;   /* not used in ex38 */
    s_ip = ip;  s_mask = mask;  s_gw = gw;
    s_dhcp_bound = true;

    ca_net_proto_set_local_ip(ip);
    EVAL_UDP_SRV(ca_udp_server_set_local_ip(ip);)
    EVAL_ARP_SVC(ca_arp_service_update_local_ip(ip);)

    ca_arp_send_garp();

    /* unblocks the mDNS task and sends the gratuitous announce */
    EVAL_MDNS_SVC(ca_mdns_service_net_ready(ip);)
}
```

The order is critical: `ca_net_proto_set_local_ip(ip)` must precede `ca_mdns_service_net_ready(ip)` because the mDNS task (LEVEL1, more urgent than `task_net` LEVEL2) will be scheduled immediately and use `build_announce()` which reads the IP from the parameter passed, not from the proto stack.

3. Init mDNS in `ca_os_configure`

```
EVAL_MDNS_SVC(ca_mdns_service_init(s_mac, MDNS_HOSTNAME);)
```

`ca_mdns_service_init()` saves MAC and hostname, initializes the semaphores. It does not create the task — that is done by `ca_mdns_service_start()` in `ca_os_main()`.

4. Binding port 5353 in `task_net`

```
ca_net_proto_bind_udp(DHCP_CLIENT_PORT, dhcp_udp_handler);  /* 68 */
EVAL_UDP_SRV(ca_udp_server_bind();)                          /* 7961 */
EVAL_MDNS_SVC(ca_mdns_service_bind();)                       /* 5353 */
```

`ca_mdns_service_bind()` registers the handler `mdns_udp_rx` on port 5353. The dispatcher accepts the frame because the MAC `01:00:5E:00:00:FB` is multicast (I/G bit=1) and is accepted by the `CA_L2_ACCEPT_MULTICAST` filter.

5. `ca_os_main`

```
void ca_os_main(void)
{
    EVAL_ARP_SVC(ca_arp_service_start();)
    EVAL_DHCP_SVC(ca_dhcp_service_start();)
    EVAL_MDNS_SVC(ca_mdns_service_start();)
    EVAL_UDP_SRV(ca_udp_server_start(&srv_cfg);)

    ca_os_job("net",       CA_TH_LEVEL2, task_net,       ...);
    ca_os_job("eth_mon",   CA_TH_LEVEL3, task_eth_mon,   ...);
    ca_os_job("net_stats", CA_TH_LEVEL4, task_stats,     ...);
    ca_os_job("heartbeat", CA_TH_LEVEL4, task_heartbeat, ...);
    ca_os_run(true);
}
```

61.1.4 PC Test Procedure

The script `examples/example_38/test_mdns_pc.sh` automates the procedure. Requires: `dnsmasq`, `avahi-utils`, `python3`.

```
sudo ./examples/example_38/test_mdns_pc.sh
# or with a specific interface:
sudo IFACE=enp7s0 ./examples/example_38/test_mdns_pc.sh
```

The script runs three tests:

1. **avahi-resolve** — uses the PC's `avahi-daemon` to resolve `caos-board.local`
2. **ping .local** — `ping -c3 caos-board.local`
3. **mDNS raw** — Python script that sends a UDP multicast query on 224.0.0.251:5353 bypassing avahi, verifies that the board responds directly

Equivalent manual test:

```
# Setup PC
sudo ip addr add 192.168.10.1/24 dev eth0
sudo dnsmasq --no-daemon --interface=eth0 \
  --dhcp-range=192.168.10.100,192.168.10.200,1h

# After the board's DHCP bound:
avahi-resolve -n caos-board.local
ping -c3 caos-board.local
echo "hello" | nc -u caos-board.local 7961

# Monitor mDNS
sudo tcpdump -i eth0 udp port 5353 -vvv
```

61.1.5 Expected Output

Console MCU (USB CDC)

```
=== EX38: DHCP + mDNS + ARP + ping + UDP server ===
MAC          02:00:00:00:00:38
mDNS         caos-board.local
IP           --- (waiting for DHCP)
DHCP: avvio DORA...

DHCP BOUND  IP=192.168.10.138  mask=255.255.255.0
            GW=192.168.10.1

STATS dhcp=BOUND pool=56/64 held=8 rx=23 drop=0 tx=9
```

Test script test_mdns_pc.sh (PC side)

```
============================================================
  caOS Example 38     mDNS responder test
  Interfaccia : enp7s0
  PC IP       : 192.168.10.1
  mDNS target : caos-board.local
============================================================

=== Test 1: avahi-resolve ===
[ OK ]  avahi-resolve: caos-board.local 192.168.10.138

=== Test 2: ping .local ===
[ OK ]  ping caos-board.local OK

=== Test 3: query mDNS raw ===
[mDNS] query per caos-board.local ...
[mDNS] caos-board.local       192.168.10.138  (da 192.168.10.138)
[ OK ]  mDNS raw OK

========================================
 Test completati
 Hostname: caos-board.local
========================================
```

61.1.6 Technical Notes

Gratuitous announce vs query response

The mDNS task distinguishes two behaviors:

- **Gratuitous announce** (at startup): sends an unsolicited `QR=1` response with `QDCOUNT=0` and `ANCOUNT=1`. All peers on the LAN learn the hostname→IP association without having to perform a query. Avahi receives this immediately.
- **Query response** (loop): when `ca_net_proto_mdns_parse_query()` returns `is_for_us=true`, the task posts `s_query_sem` and the mDNS task sends an identical response. Typical latency: ~500 µs from query to response.

RFC 6762 — Transaction ID always 0

mDNS requires `txid=0` in all messages (request and response). A different value causes the response to be rejected by avahi and other stacks. `ca_net_proto_mdns_build_announce()` sets `ID=0` correctly.

Multicast reception — `MACPFR_RA`

The mDNS frame has `dst_mac=01:00:5E:00:00:FB`. The GMAC 4.x (H7xx) must be configured with the `ETH_MACPFR_RA` bit (Receive All) or with the multicast hash filter to accept this group. `eth_h7xx.c` sets `RA` for simplicity — the board receives all multicast, including IGMP and mDNS.

Conditional Compilation

```
#if EN_ETH && EN_NET && EN_NET_DHCP && EN_NET_MDNS && \
    EN_DHCP_SERVICE && EN_MDNS_SERVICE && EN_UDP_SERVER
```

`EVAL_MDNS_SVC(expr)` executes `expr` only if `EN_MDNS_SERVICE=1`.

Chapter 62

UDP Syslog Client (RFC 3164)

62.1 Example 39 — DHCP + Syslog Service

62.1.1 Introduction

Example 39 shows how to integrate the **Syslog service** of caOS into a real networked application on Ethernet. The service sends log messages to a remote server via UDP port 514 (RFC 3164), with a fire-and-forget architecture: any task calling `ca_syslog_send()` does not block, delivery occurs in the background by the lowest priority task.

Objectives:

1. **DHCP client**: automatically obtains IP (DORA + T1/T2 renew).
2. **Syslog service**: sends messages with variable severity (DEBUG, INFO, WARN) to the syslog server on the PC.
3. **ARP service**: resolves the syslog server's MAC address (only the first time, then cached).
4. **ICMP echo reply** (ping from the board).
5. **Integrated CLI**: `syslog info`, `arp info`, `dhcp info/renew`.

Hardware (NUCLEO-H753ZI, MB1364)

Identical to examples 33–38 — same RMII pinout, same LEDs.

Required configuration in `os_config.h`

```
#define EN_ETH           1
#define EN_NET           1
#define EN_NET_SYSLOG    1
#define EN_NET_DHCP      1
#define EN_ARP_SERVICE   1
#define EN_DHCP_SERVICE  1
#define EN_SYSLOG_SERVICE 1
```

62.1.2 System Architecture

Architecture Diagram

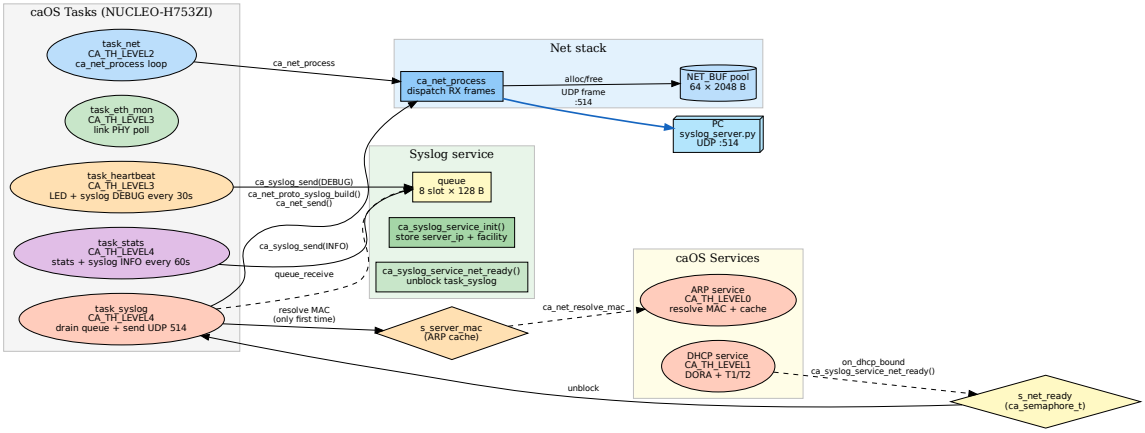


Figure 62.1: Example 39 architecture

Syslog task flow

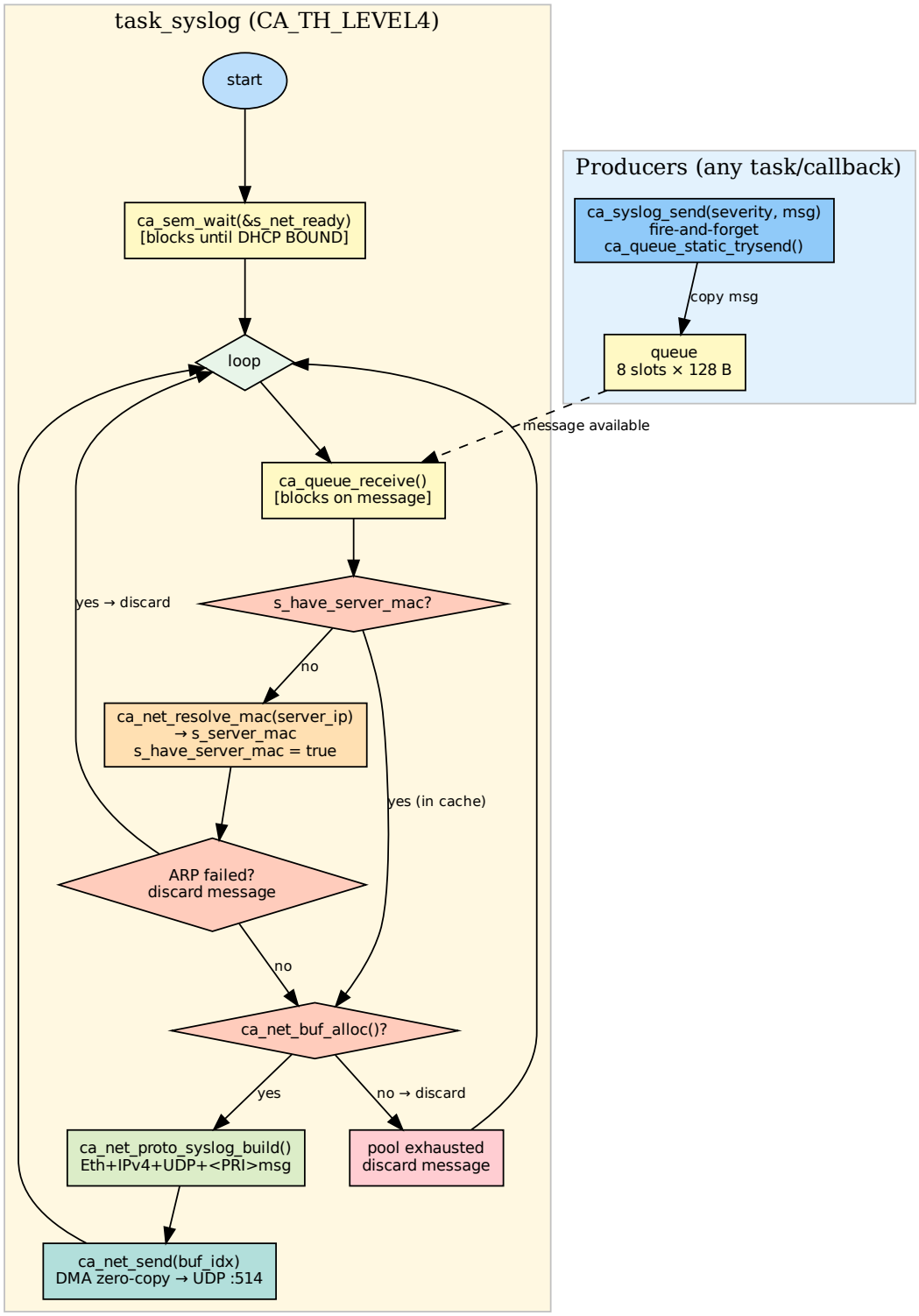


Figure 62.2: Syslog task flow

Message sources in the example

Function / task	Severity	Text
`on_dhcp_bound`	INFO	`EX39 BOUND IP=x.x.x.x`
`on_dhcp_expire`	WARN	`EX39 DHCP lease expired  rebinding`
`on_link_change(true)`	INFO	`EX39 ETH link UP`
`on_link_change(false)`	WARN	`EX39 ETH link DOWN`
`task_heartbeat`	DEBUG	`EX39 heartbeat seq=N pool=M/64`
`task_stats`	INFO	`EX39 stats rx=N tx=N drop=N slg_drop=N`

Differences from example 38

Aspect	Ex 38	Ex 39
mDNS responder	yes	no
UDP server port 7961	yes	no
Syslog service	no	**yes**
Log sources	—	4 (hb, stats, dhcp, link)
Severity levels used	—	DEBUG, INFO, WARN
PC test script	`test_mdns_pc.sh`	`syslog_server.py`
Additional ENs	`EN_NET_MDNS`, `EN_MDNS_SERVICE`	`EN_NET_SYSLOG`, `EN_SYSLOG_SERVICE`

Task Priorities

```
LEVEL0: task_arp            cache ARP + resolve
LEVEL1: task_dhcp           DORA + T1/T2
LEVEL2: task_net            processing frame RX
LEVEL3: task_eth_mon        link PHY poll
         task_heartbeat      LED + syslog DEBUG every 30s
LEVEL4: task_stats          statistics + syslog INFO every 60s
         task_syslog        drena queue + invia UDP 514
```

The syslog task is at LEVEL4 (the lowest priority) in order not to interfere with the critical RX/TX path. Messages are delivered in idle windows.

62.1.3 The source code

The file is located in `examples/example_39/ca_os_ex39.c`.

1. Syslog server IP

```
#define SYSLOG_SERVER_IP    { 192U, 168U, 10U, 1U }
```

Modify with the IP of the PC where `syslog_server.py` is running. The IP is passed to `ca_syslog_service_init()` in `ca_os_configure()`. It can be updated at runtime with `ca_syslog_service_set_server(new_ip)`.

2. ca_os_configure — initialization

```
void ca_os_configure(void)
{
    /* ... GPIO, ETH, ARP, DHCP init ... */
    {
        ca_ipv4_t zero = { .addr = 0U };
        ca_arp_service_init(s_mac, zero);      /* provisional IP 0 */
    }
    ca_dhcp_service_init(s_mac, on_dhcp_bound, on_dhcp_expire);

    {
        ca_ipv4_t srv = { .v = SYSLOG_SERVER_IP };
        ca_syslog_service_init(srv, s_mac, SYSLOG_FAC_LOCAL0);
    }
}
```

`ca_syslog_service_init()` initializes the queue, registers the CLI (`syslog info`), and saves the server's MAC and IP address. The task has not yet been created.

3. ca_os_main — task start

```
void ca_os_main(void)
{
    ca_arp_service_start();
    ca_dhcp_service_start();
    ca_syslog_service_start();   /* crea task LEVEL4 */
    /* ... ca_os_job for net, eth_mon, heartbeat, stats ... */
    ca_os_run(true);
}
```

The syslog task blocks immediately on `s_net_ready` — waits for the gate.

4. on_dhcp_bound — unlock and first message

```
static void on_dhcp_bound(ca_ipv4_t ip, ca_ipv4_t mask, ca_ipv4_t gw, ca_ipv4_t
    dns)
{
    s_ip = ip; s_mask = mask; s_gw = gw; s_dhcp_bound = true;

    ca_net_proto_set_local_ip(ip);         /* update the proto stack */
    ca_arp_service_update_local_ip(ip);    /* update ARP */

    ca_arp_send_garp();

    ca_syslog_service_net_ready();         /* unblocks the syslog task */

    char msg[64];
    ca_snprintf(msg, sizeof(msg),
                "EX39 BOUND IP=%u.%u.%u.%u",
                (u32)ip.v[0], (u32)ip.v[1], (u32)ip.v[2], (u32)ip.v[3]);
    ca_syslog_send(SYSLOG_SEV_INFO, msg);
}
```

Critical order: `ca_net_proto_set_local_ip(ip)` before `ca_syslog_service_net_ready()`. The syslog task uses `s_net_proto_local_ip` as `src_ip` of the UDP frame: if it is still 0.0.0.0 the frame will be discarded by the server.

5. task_heartbeat — periodic log

```c
static u32 task_heartbeat(void *arg)
{
    u32 seq = 0U;
    while (1) {
        ca_os_delay_ms(HEARTBEAT_PERIOD_MS);   /* 30 s */
        ca_gpio_toggle(LED_HB_PORT, LED_HB_PIN);
        if (!s_dhcp_bound) continue;
        char msg[64];
        ca_snprintf(msg, sizeof(msg),
                    "EX39 heartbeat seq=%u pool=%u/%u",
                    seq, (u32)ca_net_buf_free_count(), (u32)NET_BUF_COUNT);
        ca_syslog_send(SYSLOG_SEV_DEBUG, msg);
        seq++;
    }
    return 0U;
}
```

Check `s_dhcp_bound` before sending: if DHCP is not yet bound, the syslog does not have a valid IP and the messages would be discarded.

6. `ca_syslog_send` — fire-and-forget semantics

```c
ca_syslog_send(SYSLOG_SEV_WARN, "EX39 ETH link DOWN");
```

- **Non blocking**: copies the message to the queue and returns immediately.
- **Silent drop**: if the queue is full (8 slots) the message is lost and `ca_syslog_dropped()` increments the counter.
- **Safe from ISR**: does not acquire mutex — uses `ca_queue_static_trysend()`.

62.1.4 Test procedure from PC

1. Start the syslog server (DHCP included)

`syslog_server.py` automatically starts the DHCP server (`dnsmasq`) and configures the interface to `192.168.10.1/24`. At completion it restores the initial conditions.

```
# Needs sudo (port 514 + dnsmasq + ip)
sudo python3 examples/example_39/syslog_server.py --iface enp7s0
```

Main options:

Option	Default	Description
`--iface <eth>`	—	Network interface (required for DHCP)
`--pc-ip <ip>`	`192.168.10.1`	PC IP (must match `SYSLOG_SERVER_IP`)
`--port <n>`	`514`	UDP syslog port
`--no-dhcp`	off	Syslog only, do not start dnsmasq

Output on startup:

```
[*] enp7s0 configurato     192.168.10.1/24
[*] dnsmasq avviato (pid 12358)
[*] DHCP log: tail -f /tmp/dnsmasq-caos39.log
============================================================
 caOS EX39     syslog server  192.168.10.1:514
 DHCP pool  192.168.10.100     192.168.10.200
============================================================
 Ctrl+C per terminare (cleanup automatico).
------------------------------------------------------------
```

2. Programming the board and monitoring

On the serial monitor (USB CDC):

```
=== EX39: DHCP + Syslog service ===
MAC           02:00:00:00:00:39
IP            --- (waiting for DHCP)
Syslog        -> 192.168.10.1:514
DHCP: avvio DORA...

DHCP BOUND  IP=192.168.10.139  mask=255.255.255.0
            GW=192.168.10.1
```

On the syslog server (PC), the messages appear immediately after the bind:

```
[18:51:35.738] 192.168.10.139:514  [INFO  ]  EX39 ETH link UP
[18:51:35.738] 192.168.10.139:514  [INFO  ]  EX39 BOUND IP=192.168.10.139
[18:52:05.040] 192.168.10.139:514  [DEBUG ]  EX39 heartbeat seq=0 pool=56/64
[18:52:34.936] 192.168.10.139:514  [INFO  ]  EX39 stats rx=3 tx=6 drop=0
    slg_drop=0
[18:52:34.943] 192.168.10.139:514  [DEBUG ]  EX39 heartbeat seq=1 pool=56/64
```

3. Simulate a link change

Unplug and replug the Ethernet cable:

```
[HH:MM:SS.mmm] 192.168.10.139:514  [WARN  ]  EX39 ETH link DOWN
[HH:MM:SS.mmm] 192.168.10.139:514  [INFO  ]  EX39 ETH link UP
[HH:MM:SS.mmm] 192.168.10.139:514  [INFO  ]  EX39 BOUND IP=192.168.10.139
```

4. CLI from the board

```
c.a.O.S > syslog info
syslog server : 192.168.10.1:514
facility      : 16
net ready     : yes
dropped msgs  : 0

c.a.O.S > arp info
slot  IP               MAC                age(ms)
[0]   192.168.10.1     AA:BB:CC:11:22:33  3241
```

62.1.5 Expected output (verified on NUCLEO-H753ZI)

MCU Console (USB CDC)

```
=== EX39: DHCP + Syslog service ===
MAC           02:00:00:00:00:39
IP            --- (waiting for DHCP)
Syslog        -> 192.168.10.1:514
DHCP: avvio DORA...

DHCP BOUND  IP=192.168.10.139  mask=255.255.255.0
            GW=192.168.10.1

STATS dhcp=BOUND pool=56/64 held=8 rx=3 drop=0 tx=6 syslog_drop=0
```

Syslog Server PC (`syslog_server.py`)

```
[18:51:35.738] 192.168.10.139:514  [INFO  ]  EX39 ETH link UP
[18:51:35.738] 192.168.10.139:514  [INFO  ]  EX39 BOUND IP=192.168.10.139
[18:52:05.040] 192.168.10.139:514  [DEBUG ]  EX39 heartbeat seq=0 pool=56/64
[18:52:34.936] 192.168.10.139:514  [INFO  ]  EX39 stats rx=3 tx=6 drop=0
    slg_drop=0
[18:52:34.943] 192.168.10.139:514  [DEBUG ]  EX39 heartbeat seq=1 pool=56/64
```

62.1.6 Technical Notes

Gate `s_net_ready` — why it is necessary

The syslog task runs at LEVEL4 (the lowest priority). At system creation (before DHCP binds), there is no valid local IP yet: the UDP frame would have `src_ip=0.0.0.0` and would be dropped by the server. The gate prevents wasting ARP requests and buffers before the network is operational.

ARP — resolve and cache

The first `ca_syslog_send()` after the gate triggers an `ca_net_resolve_mac()` to obtain the MAC address of the syslog server. The result is kept in `s_server_mac` (`s_have_server_mac = true`): all subsequent messages do not re-do ARP.

If the server changes IP (`ca_syslog_service_set_server(new_ip)`), the cache is invalidated (`s_have_server_mac = false`) and ARP is re-done.

`ca_syslog_send()` from ISR

`ca_syslog_send()` uses `ca_queue_static_trysend()` which is ISR-safe (no mutexes, no suspension). It can be called from interrupt handlers, provided the string `msg` is in the data segment (not in the ephemeral ISR stack).

Dropped messages

If tasks produce messages faster than the syslog task delivers them (e.g. burst of high-priority events), the queue becomes saturated. `ca_syslog_dropped()` counts the lost messages. Increase `SYSLOG_QUEUE_SIZE` in `os_config.h` if the value is > 0 at steady state.

Wire format (RFC 3164)

```
UDP payload: <PRI>message
PRI = facility * 8 + severity
```

With `facility=SYSLOG_FAC_LOCAL0=16` and `severity=SYSLOG_SEV_INFO=6`: `PRI = 16*8+6 = 134` → `<134>EX39 BOUND IP=192.168.10.139`

The format does not include timestamp (no RTC in the example): the PC server adds the local timestamp upon receipt.

Chapter 63

TCP Client HTTP GET

63.1 Example 40 — TCP Client HTTP GET

63.1.1 Introduction

Application example of the caOS TCP client service: the NUCLEO-H753ZI board obtains an IP via DHCP, establishes a TCP connection to a Python server on the LAN, sends an HTTP GET request and reads the response.

The example **does not start by itself**: after the DHCP lease it remains idle and announces (`ex40: idle`). The `e40 start` command executes **one** request and closes it with a verdict that the test bench reads:

```
RUN RESULT : OK   (1/1)       connected, GET sent, non-empty reply read
RUN RESULT : FAIL  (0/1)      one of the three things did not happen
```

`e40 info` indicates whether the client is ready and how many requests have succeeded.

> **Why not a loop anymore.** Until 10/04, the client repeated the request every 30 s from boot. The test bench could not ask it to connect: it had to wait for the loop to arrive, on average almost a full period (~29 s for the connection phase alone), and count events within time windows twice the period wide. With `e40 start`, the same test lasts about a second and the criterion is a verdict line, not a count.

Demonstrative objectives:

Feature	Notes
DHCP client	DORA + T1/T2 renew, gate semaphore
ARP service	resolve server MAC, GARP at bind
Blocking TCP client	`ca_tcp_connect()` + `ca_tcp_send()` + `ca_tcp_recv()`
HTTP GET / HTTP 1.0	minimal request, connection close
ICMP echo reply	ping from the board during the test
Task stats	pool buffer, rx/tx frame

63.1.2 Hardware

Component	Value
Board	NUCLEO-H753ZI (STM32H753ZI)
PHY	LAN8742A (RMII, addr 0)

Component	Value
LED link	PB0
LED heartbeat	PB14
MAC	02:00:00:00:00:40
IP	assigned by DHCP
TCP Server	192.168.10.1 : 8080 (configurable)

63.1.3 os_config.h Configuration

```
#define EN_ETH            1
#define EN_NET            1
#define EN_ARP_SERVICE    1
#define EN_DHCP_SERVICE   1
#define EN_NET_DHCP       1
#define EN_TCP_SERVICE    1
/* TCP_MAX_CONN 8, TCP_MAX_INFLIGHT 16, TCP_RX_RING_DEPTH 8: model defaults */
```

63.1.4 Architettura

Tasks and Priorities

```
LEVEL0: ARP service           cache + blocking resolve
LEVEL1: DHCP service          DORA + T1/T2
LEVEL2: task_net              ca_net_process loop
LEVEL3: task_eth_mon          link PHY poll
         task_tcp              HTTP GET client loop
         TCP timer task        RTO retransmit
LEVEL4: task_stats            statistics every 20 s
```

Architecture diagram

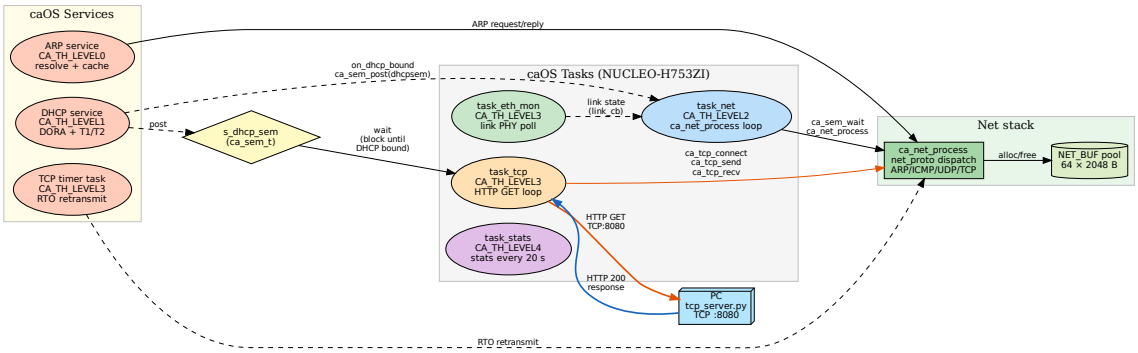


Figure 63.1: Example architecture 40

Task_tcp flow

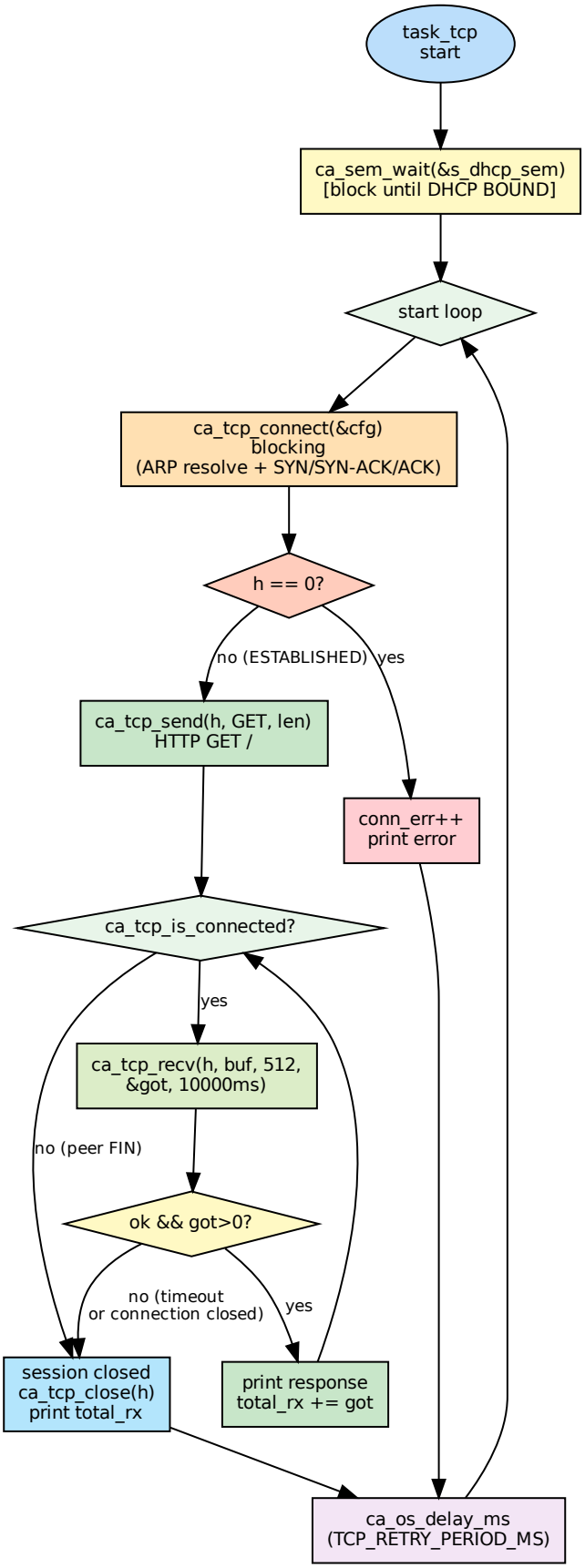


Figure 63.2: TCP client flow

63.1.5 Code — key points

Gate DHCP → TCP

The `task_tcp` must wait for the IP to be valid before attempting any connection. Synchronization is done via a semaphore:

```
/* ca_os_configure: */
ca_sem_init(&s_dhcp_sem, "DHCPBND", 0U, 1U);

/* on_dhcp_bound: */
ca_net_proto_set_local_ip(ip);
ca_arp_service_update_local_ip(ip);
ca_arp_send_garp();
ca_sem_post(&s_dhcp_sem);    /* sblocca task_tcp */

/* task_tcp: */
ca_sem_wait(&s_dhcp_sem);    /* waits for the bind */
```

Blocking Connection

```
ca_tcp_connect_cfg_t cfg = {
    .remote_ip          = srv_ip,
    .remote_port        = TCP_SERVER_PORT,
    .local_ip           = s_ip,
    .local_mac          = s_mac,
    .arp_timeout_ms     = 0U,   /* default TCP_ARP_TIMEOUT_MS  = 2000 ms */
    .connect_timeout_ms = 0U,   /* default TCP_CONNECT_TIMEOUT_MS = 5000 ms */
    .on_rx              = NULL, /* blocking mode ca_tcp_recv() */
    .on_connected       = NULL, /* blocks on connect */
    .on_closed          = NULL,
};

u8 h = ca_tcp_connect(&cfg);
/* h == 0     error (ARP timeout, SYN timeout, pool full) */
/* h >= 1     valid handle, connection ESTABLISHED */
```

Internally `ca_tcp_connect()` executes: 1. ARP resolution of the server's MAC (from cache or request) 2. Allocates a PCB from the pool (4 slots) 3. Sends SYN, waits for SYN-ACK, sends ACK

Send HTTP GET

```
static const char http_get[] =
    "GET / HTTP/1.0\r\n"
    "Host: caOS-board\r\n"
    "Connection: close\r\n"
    "\r\n";

bool ok = ca_tcp_send(h, (const u8 *)http_get, (u16)(sizeof(http_get) - 1U));
```

`ca_tcp_send()` segments the payload into chunks of MSS size; each segment is kept in a buffer pool until the peer's ACK is received (retransmit store-and-send).

Response Reception

```
u16  got = 0U;
bool ok  = ca_tcp_recv(h, rx_buf, sizeof(rx_buf) - 1U, &got, 10000U);
```

The loop continues while `ca_tcp_is_connected(h)` is true and `ca_tcp_recv()` returns data. When the peer closes (FIN), `ca_tcp_is_connected()` becomes `false` and the loop terminates.

63.1.6 Test from PC (verified on NUCLEO-H753ZI)

Start the server (DHCP included)

`tcp_server.py` automatically starts `dnsmasq` and configures the interface to `192.168.10.1/24`. At completion it restores the initial conditions.

```
cd examples/example_40
sudo python3 tcp_server.py --iface enp7s0
```

Main options:

Option	Default	Description
`--iface <eth>`	—	Network interface (required for DHCP)
`--pc-ip <ip>`	`192.168.10.1`	PC IP (must match `TCP_SERVER_IP`)
`--port <n>`	8080	TCP port
`--no-dhcp`	off	TCP server only, without dnsmasq

PC side output:

```
[*] enp7s0 already configured with 192.168.10.1
[*] dnsmasq avviato (pid 20147)
[*] DHCP log: tail -f /tmp/dnsmasq-caos40.log
============================================================
 caOS EX40     TCP server  192.168.10.1:8080
 DHCP pool  192.168.10.100     192.168.10.200
============================================================
 Ctrl+C per terminare (cleanup automatico).
------------------------------------------------------------
[19:11:41] connect from 192.168.10.146:49200
  >> GET / HTTP/1.0
  << 167 byte sent
[19:12:21] connect from 192.168.10.146:49200
  >> GET / HTTP/1.0
  << 167 byte sent
```

Output board (stdout/USB) — verified on NUCLEO-H753ZI

```
=== EX40: TCP client HTTP GET ===
MAC          02:00:00:00:00:40
TCP server   192.168.10.1:8080
DHCP: avvio DORA...

DHCP BOUND  IP=192.168.10.146
ex40: idle - "e40 start" fetches once from the server
e40 start

TCP connect -> 192.168.10.1:8080
TCP ESTABLISHED  h=1  ok=1
TCP send GET  OK
TCP rx 167 byte:
HTTP/1.0 200 OK
Content-Type: text/plain
Content-Length: 83
```

```
Connection: close

caOS EX40 TCP server reply
time=2026-0
TCP session done  total_rx=167 byte
  RUN RESULT : OK  (1/1)
```

Display note — `time=2026-0`

The HTTP response uses `\r\n` as line terminator (RFC 2616). The board prints the raw buffer with `ca_stdout_put_n()`, which sends the bytes as they are to the USB CDC terminal. The `\r` character (CR, ASCII 13) orders the terminal to return to **column 0 of the current line** without advancing to the next line. The next line of the body (`client=192.168.10.146:...`) is then printed starting from column 0 of the same line, overwriting the end part of the data. Only `time=2026-0` remains visible because the string `client=` (7 characters) covers the first 7 characters of what was there.

The data **is received correctly** (`total_rx=167`): this is a terminal display artifact, not a caOS bug.

63.1.7 Technical notes

Blocking vs callback mode

With `on_connected=NULL` and `on_rx=NULL` (blocking mode), the calling task blocks during connect and recv. This is the simplest way for sequential code. To handle multiple concurrent connections, use callbacks (`on_connected`, `on_rx`) in separate tasks.

Ephemeral ports

The implementation automatically assigns the local port starting from `49200 + slot` (one per PCB). It is not necessary to configure it.

Reconnect and PCB

After `ca_tcp_close()` the PCB enters TIME_WAIT (`TCP_TIME_WAIT_MS` = 4 s) before being released. A second `e40 start` immediately after the first does not wait: with `TCP_MAX_CONN` = 4 the new connection takes another PCB. While a request is in progress, `e40 start` responds `a fetch is already running` instead of queuing another one.

Buffer pool and TCP

Each in-flight TX segment occupies a buffer pool until ACK. With `TCP_MAX_INFLIGHT=4` and `TCP_MAX_CONN=4`, TCP can keep at most 16 buffers occupied. Check `NET_BUF_COUNT >= 16 + margine_rx` .

Garanzia sequenza on_dhcp_bound

```
ca_net_proto_set_local_ip(ip)          proto usa IP corretto nei frame TCP
ca_arp_service_update_local_ip(ip)      ARP usa IP corretto per richieste
ca_arp_send_garp()                      updates the peers' caches on the LAN
ca_sem_post(&s_dhcp_sem)                unblocks task_tcp only after all of the
    above
```

Chapter 64

TCP Server (Echo Server, RFC 862)

64.1 Example 41 — TCP Server (echo server RFC 862)

64.1.1 Introduction

Example of the caOS TCP server in **passive open** mode: the NUCLEO-H753ZI board listens on TCP port 7 (echo, RFC 862), accepts one connection at a time, and returns any received data to the client.

Demonstration objectives:

Feature	Notes
TCP server passive open	`ca_tcp_listen` + `ca_tcp_accept` blocking
3-way handshake	SYN → SYN-ACK → ACK handled automatically
Echo loop	`ca_tcp_recv` → `ca_tcp_send`
DHCP client	IP assigned dynamically, listen updated post-bind
ARP service	client MAC resolution + GARP
ICMP echo reply	ping from the board during the test

64.1.2 Hardware

Component	Value
Board	NUCLEO-H753ZI (STM32H753ZI)
PHY	LAN8742A (RMII, addr 0)
Link LED	PB0
Heartbeat LED	PB14 (toggled at each closed connection)
MAC	02:00:00:00:00:41
IP	assigned by DHCP
Server port	TCP 7 (echo RFC 862)

64.1.3 Configuration os_config.h

```
#define EN_ETH           1
#define EN_NET           1
#define EN_ARP_SERVICE   1
```

```
#define EN_DHCP_SERVICE  1
#define EN_NET_DHCP      1
#define EN_TCP_SERVICE   1
/* TCP_MAX_LISTEN default 2     1 is enough for this example */
```

64.1.4 Architecture

Tasks and priorities

```
LEVEL0: ARP service            cache + blocking resolve
LEVEL1: DHCP service           DORA + T1/T2
LEVEL2: task_net               ca_net_process loop
LEVEL3: task_eth_mon           link PHY poll
         task_server            echo server loop
         TCP timer task         RTO + SYN-ACK retransmit
LEVEL4: task_stats             statistics every 20 s
```

Architecture diagram

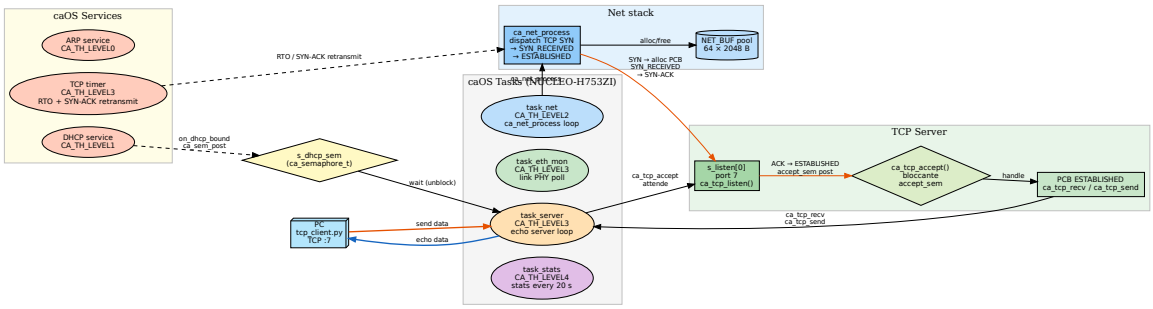


Figure 64.1: Architecture example 41

TCP server flow

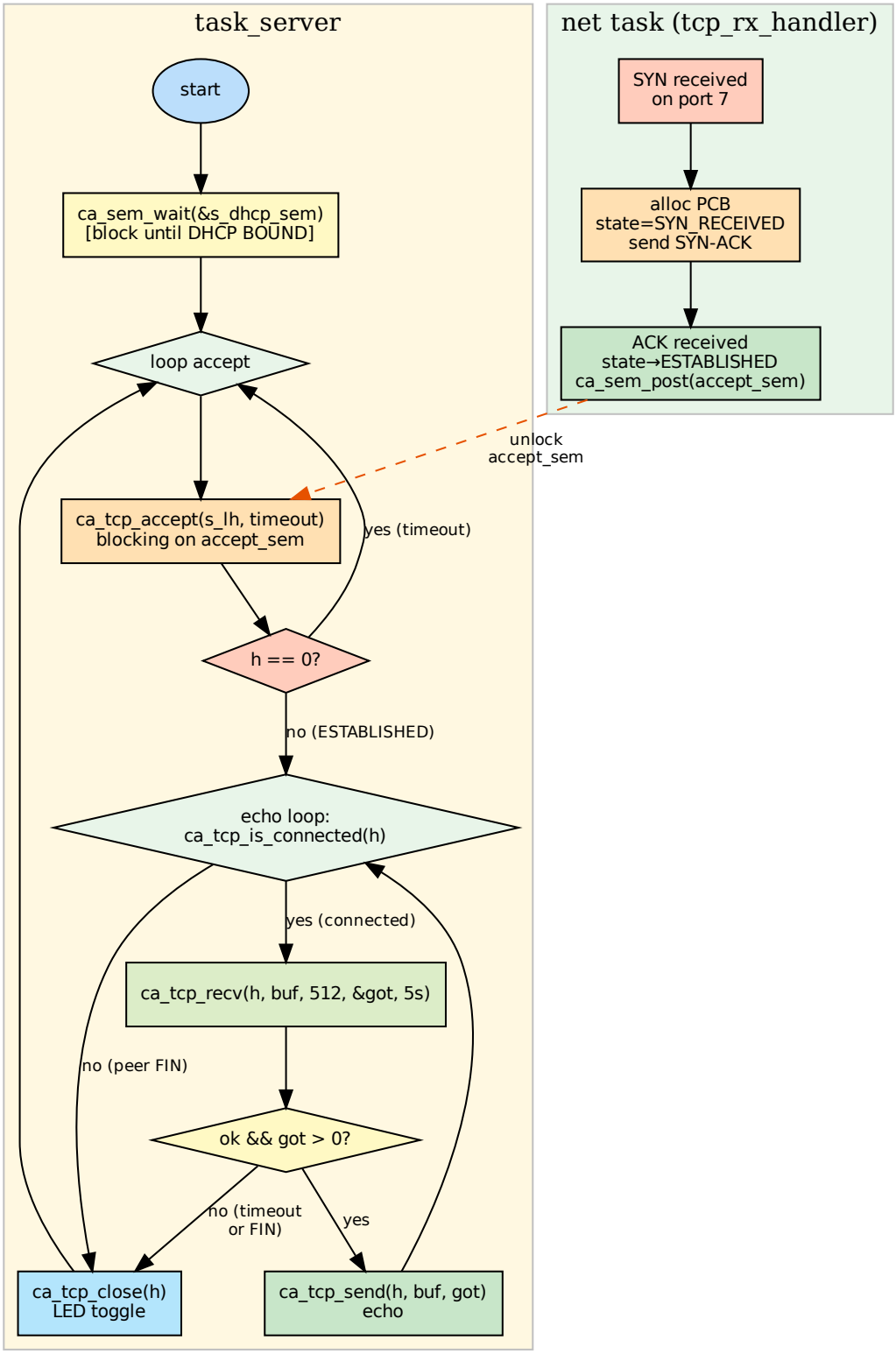


Figure 64.2: TCP server flow

64.1.5 Code — key points

Opening the listen

The listen is created in `ca_os_main()` with `local_ip=0.0.0.0` before DHCP has assigned an IP. When `on_dhcp_bound` arrives, it is updated with the real IP:

```
/* ca_os_main: */
ca_tcp_listen_cfg_t lcfg = {
    .local_port = TCP_ECHO_PORT,
    .local_ip   = { .addr = 0U },   /* provisional IP */
    .local_mac  = s_mac,
    .on_accept  = NULL,             /* blocking mode */
    .on_rx      = NULL,
    .on_closed  = NULL,
};
s_lh = ca_tcp_listen(&lcfg);

/* on_dhcp_bound: */
ca_tcp_unlisten(s_lh);     /* removes the listen with IP=0 */
lcfg.local_ip = ip;
s_lh = ca_tcp_listen(&lcfg);   /* re-registers with the real IP */
ca_sem_post(&s_dhcp_sem);       /* sblocca task_server */
```

Blocking accept

```
u8 h = ca_tcp_accept(s_lh, TCP_TCP_ACCEPT_TIMEOUT_MS);
/* h == 0     timeout, riprova */
/* h >= 1     connection ESTABLISHED */
```

`ca_tcp_accept` blocks on `accept_sem` until the 3-way handshake completes. The flow in the net task is:

```
SYN ricevuto su porta 7
      alloc PCB, state = SYN_RECEIVED
      send SYN-ACK
ACK ricevuto
      state = ESTABLISHED
      ls->last_handle = h
      ca_sem_post(&ls->accept_sem)       sblocca ca_tcp_accept()
```

Echo loop

```
while (ca_tcp_is_connected(h)) {
    u16  got = 0U;
    bool ok  = ca_tcp_recv(h, buf, sizeof(buf)-1U, &got, ECHO_RECV_TIMEOUT);
    if (!ok) {
        if (ca_tcp_is_connected(h)) continue;  /* timeout: riprova */
        break;                                  /* peer has closed */
    }
    if (got == 0U) continue;
    ca_tcp_send(h, buf, got);
}
ca_tcp_close(h);
```

`ca_tcp_recv` can return `false` due to timeout (no data within `ECHO_RECV_TIMEOUT`) or disconnection. Distinguishing between the two cases with `ca_tcp_is_connected()` allows the echo server to wait for the peer indefinitely without closing due to inactivity.

64.1.6 Test from PC (verified on NUCLEO-H753ZI)

Start the client (DHCP included)

`tcp_client.py` starts `dnsmasq`, configures the interface to `192.168.10.1/24`, automatically detects the IP assigned to the board from the DHCP log, and connects.

```
cd examples/example_41
sudo python3 tcp_client.py --iface enp7s0
```

Main options:

Option	Default	Description
`--iface <eth>`	—	Network interface (mandatory for DHCP)
`--pc-ip <ip>`	`192.168.10.1`	PC IP on the interface
`--port <n>`	`7`	TCP echo port on the board
`--host <ip>`	—	Direct board IP (skips DHCP auto-detect)
`--no-dhcp`	off	Client only, without dnsmasq (requires `--host`)

PC-side output (verified):

```
[*] enp7s0 already configured with 192.168.10.1
[*] dnsmasq avviato (pid 26173)
[*] DHCP log: tail -f /tmp/dnsmasq-caos41.log
[*] Waiting for DHCP ACK from the board (max 60s)...
============================================================
 caOS EX41     TCP echo client       192.168.10.147:7
============================================================
[*] Connessione a 192.168.10.147:7 ...
[*] Connesso. Digita testo + Invio. 'quit' per uscire.

> hello world
[19:34:22] echo (12B): hello world
> quit
[*] Connessione chiusa.
```

Output board (stdout/USB) — verified

```
=== EX41: TCP echo server ===
MAC     02:00:00:00:00:41
Port    TCP:7  (RFC 862 echo)
DHCP: avvio DORA...

DHCP BOUND  IP=192.168.10.147
TCP echo server on port 7
TCP server: listening on port 7
TCP accept  h=1  conn#1
  echo 12 byte: hello world

STATS dhcp=BOUND pool=56/64 held=8 rx=7 drop=0 tx=7
```

Test with netcat (alternative)

```
echo -n "Hello from nc" | nc 192.168.10.147 7
```

64.1.7 Technical notes

SYN_RECEIVED and SYN-ACK retransmit

The timer task handles the SYN-ACK retransmit exactly like the SYN retransmit for the client: exponential backoff up to `TCP_MAX_RETRIES`, then the PCB is freed.

One connection at a time

The `task_server` uses `ca_tcp_accept` in blocking mode: while an echo session is in progress, a new SYN connection is accepted at the IP level (PCB allocated, SYN-ACK sent), but `ca_tcp_accept` will only return when the current session ends. If the PCB pool is full, the new SYN is discarded with RST.

To handle concurrent connections, use `on_accept` callback + separate tasks for data handling.

IP listen update after DHCP

The SYN-ACK frame uses `local_ip` taken from the PCB at the time of acceptance. Since `on_dhcp_bound` updates the listen slot with the real IP before unblocking `task_server`, all subsequent SYNs will use the correct IP.

`ca_tcp_listen` before `ca_os_run`

`ca_tcp_listen` and `ca_tcp_unlisten` use a dedicated IRQ spinlock (`s_listen_lock`) instead of the blocking mutex `s_mtx`. This allows calling them in `ca_os_main()` before the scheduler is started, without causing the SAFEGUARD `current != NULL` of `ca_mutex_lock`.

Recv timeout in the echo loop

`ca_tcp_recv` with `ECHO_RECV_TIMEOUT` can expire before the peer sends data (e.g., slow user in interactive mode). The loop distinguishes timeout (`continue`) from disconnection (`break`) via `ca_tcp_is_connected()`.

Chapter 65

MQTT Client

65.1 Example 42 — MQTT Client

65.1.1 Introduction

Application example of the caOS MQTT client service: the NUCLEO-H753ZI board obtains an IP via DHCP, connects to an MQTT broker on the LAN and establishes an MQTT v3.1.1 session. The board periodically publishes its status on the topic `caos/status` and receives LED commands on the topic `caos/led`. All network management is performed via callbacks — no blocking polling in the MQTT task.

Demonstration objectives:

Feature	Notes
DHCP client	DORA + T1/T2 renew, gate semaphore
ARP service	resolve broker MAC, GARP at bind
MQTT v3.1.1 client	QoS 0, CONNECT/PUBLISH/SUBSCRIBE/PING
Asynchronous mode	all callback-driven on TCP layer, no dedicated MQTT task
Periodic publish	`caos/status` every 5 s with JSON payload
Subscribe + command	`caos/led` → `on`/`off` → GPIO LED green
Application keepalive	`ca_mqtt_ping()` every 30 s (< MQTT_KEEPALIVE_SEC=60)
Automatic reconnection	retry TCP+MQTT every 8 s after disconnection
ICMP echo reply	ping from board during test
Task stats	pool buffer, MQTT up/down status, session counter

65.1.2 Hardware

Component	Value
Board	NUCLEO-H753ZI (STM32H753ZI)
PHY	LAN8742A (RMII, addr 0)
Link LED	PB0 (red)
Green LED	PB0 — controlled by `caos/led`

Component	Value
Heartbeat LED	PB14 (blue — toggle stats)
MAC	02:00:00:00:00:42
IP	assigned by DHCP
MQTT Broker	DHCP gateway : 1883 — PC IP automatically detected

65.1.3 os_config.h Configuration

```
#define EN_ETH            1
#define EN_NET            1
#define EN_ARP_SERVICE    1
#define EN_DHCP_SERVICE   1
#define EN_NET_DHCP       1
#define EN_TCP_SERVICE    1
#define EN_MQTT_SERVICE   1
/* MQTT_RX_BUF_SIZE    default 256      max payload RX */
/* MQTT_KEEPALIVE_SEC  default 60       advertised to the broker */
```

65.1.4 Architecture

Tasks and priorities

```
LEVEL0: ARP service             cache + blocking resolve
LEVEL1: DHCP service            DORA + T1/T2
LEVEL2: task_net                ca_net_process loop
LEVEL3: task_eth_mon            link PHY poll
         task_mqtt               publish + ping + reconnect
         TCP timer task          RTO retransmit
LEVEL4: task_stats              statistics every 20 s
```

Architecture diagram

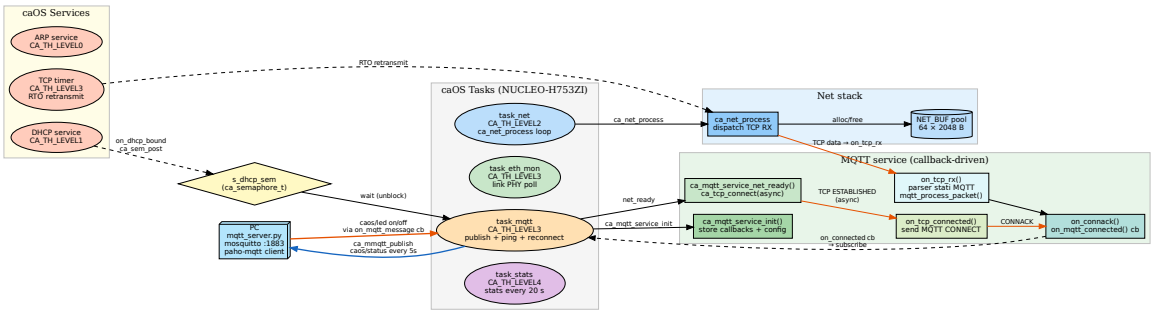


Figure 65.1: Example architecture 42

Task_mqtt flow

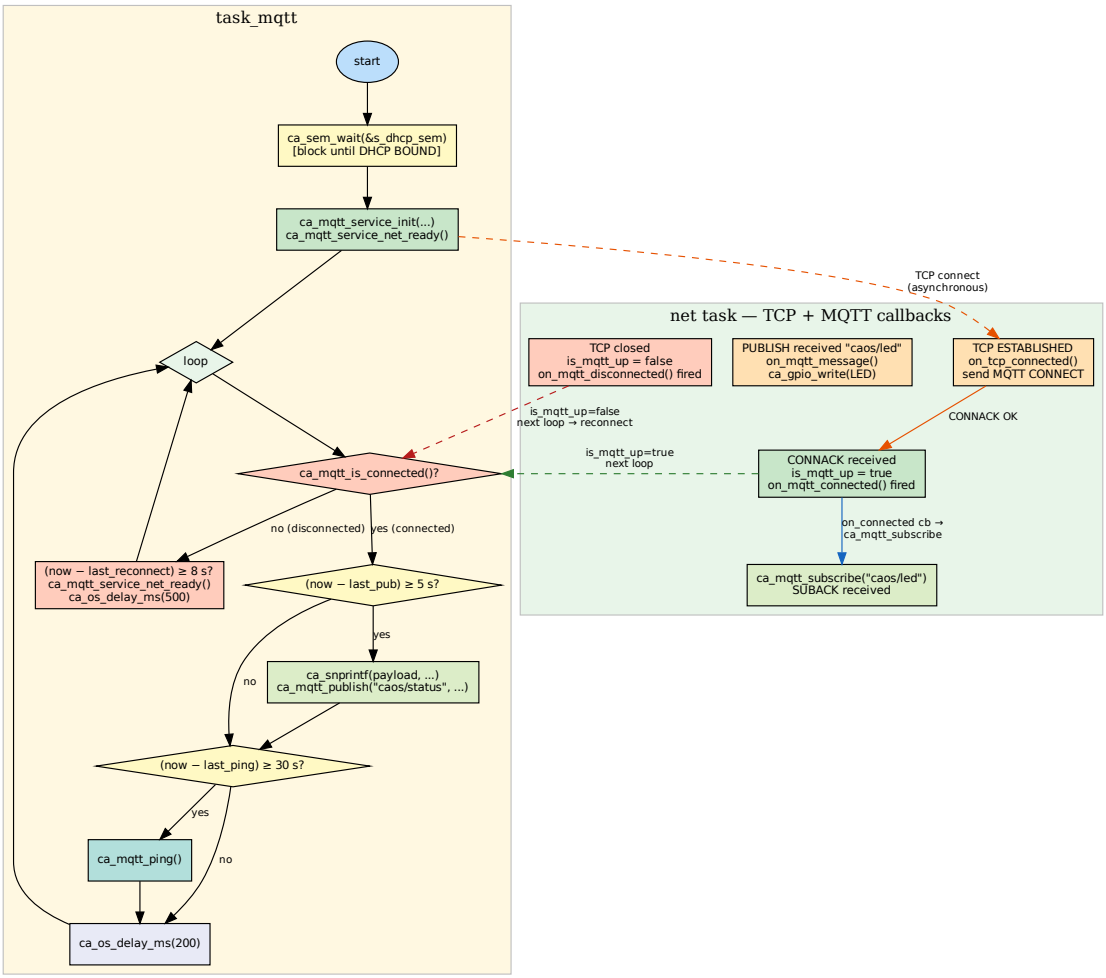


Figure 65.2: MQTT client flow

65.1.5 Code — key points

MQTT service initialization

The service is configured by `ca_os_configure()` before the network is available. At this stage the three application callbacks are registered and the broker configuration is saved. The TCP connection is not opened yet.

```
ca_mqtt_service_init(broker_ip, MQTT_BROKER_PORT,
                     local_ip, s_mac,
                     MQTT_CLIENT_ID,
                     on_mqtt_connected,
                     on_mqtt_message,
                     on_mqtt_disconnected);
```

Gate DHCP → MQTT

`task_mqtt` blocks on `s_dhcp_sem` until `on_dhcp_bound` signals that the IP is valid. Only at that point the TCP connection to the broker is opened:

```
/* on_dhcp_bound: */
ca_net_proto_set_local_ip(ip);
ca_arp_service_update_local_ip(ip);
ca_arp_send_garp();
ca_sem_post(&s_dhcp_sem);    /* sblocca task_mqtt */

/* task_mqtt: */
ca_sem_wait(&s_dhcp_sem);
ca_mqtt_service_net_ready(); /* opens asynchronous TCP     automatic CONNECT */
```

Automatic CONNECT

When the TCP 3-way handshake completes, `on_tcp_connected()` in the service automatically sends the MQTT CONNECT packet. The application does not need to handle this step: it only receives the `on_mqtt_connected()` callback when the CONNACK with return code 0 arrives.

```
static void on_mqtt_connected(void)
{
    s_mqtt_conn++;
    ca_mqtt_subscribe(MQTT_TOPIC_LED);   /* subscribe in reply to CONNACK */
}
```

Periodic publish

The `task_mqtt` loop checks the time elapsed since the last publication and builds a JSON payload with sequence number, uptime in seconds and current IP address:

```
ca_snprintf(payload, sizeof(payload),
    "{\"seq\":%u,\"up\":%u,\"ip\":\"%u.%u.%u.%u\"}",
    pub_seq++,
    now / 1000U,
    (u32)ip.v[0], (u32)ip.v[1], (u32)ip.v[2], (u32)ip.v[3]);

ca_mqtt_publish(MQTT_TOPIC_STATUS, (const u8 *)payload, (u16)ca_strlen(payload))
    ;
```

LED Command Reception

The callback `on_mqtt_message` is invoked in the context of the net task. The topic and payload are **not** null-terminated: it is necessary to use the explicit lengths.

```
static void on_mqtt_message(const char *topic, u16 topic_len,
                            const char *payload, u16 payload_len)
{
    if (topic_len == (u16)(sizeof(MQTT_TOPIC_LED) - 1U) &&
        ca_memcmp(topic, MQTT_TOPIC_LED, topic_len) == 0)
    {
        bool on = (payload_len >= 2U &&
                   payload[0] == 'o' && payload[1] == 'n');
        ca_gpio_write(LED_GREEN_PORT, LED_GREEN_PIN, on ? 1U : 0U);
    }
}
```

Keepalive and Reconnection

The keepalive PINGREQ is the responsibility of the application. The task sends `ca_mqtt_ping()` every `MQTT_PING_PERIOD_MS` (30 s), well below the limit of `MQTT_KEEPALIVE_SEC` (60 s) advertised to the broker.

If the connection fails (`on_mqtt_disconnected` fired → `mqtt_up=false`), the task attempts a new connection every `MQTT_RECONNECT_MS` (8 s). Before reopening the connection, it calls `ca_mqtt_service_init()` again to update the local IP (useful after a DHCP renewal).

```
if (!ca_mqtt_is_connected()) {
    if ((now - last_reconnect) >= MQTT_RECONNECT_MS) {
        last_reconnect = now;
        local_ip = s_ip;            /* IP updated by DHCP */
        ca_mqtt_service_init(...);
        ca_mqtt_service_net_ready();
    }
    ca_os_delay_ms(500U);
    continue;
}
```

65.1.6 Test from PC

Start the server (broker + DHCP)

`mqtt_server.py` starts `dnsmasq` for DHCP, `mosquitto` as MQTT broker and a client `paho-mqtt` that prints incoming messages. At the end it restores the initial conditions of the interface.

```
cd examples/example_42
sudo python3 mqtt_server.py --iface enp7s0
```

Main options:

Option	Default	Description
`--iface <eth>`	—	Network interface (required for DHCP)
`--pc-ip <ip>`	`192.168.10.1`	PC IP — becomes both DHCP gateway and MQTT broker
`--no-dhcp`	off	Broker only, without dnsmasq

> **Note**: the IP broker is not hardcoded. The board reads the gateway from the DHCP ACK (`on_dhcp_bound`) and uses it as the broker address. It is sufficient that `--pc-ip` matches the physical IP of the PC on the interface.

PC Dependencies:

```
sudo apt install mosquitto dnsmasq
pip3 install paho-mqtt
```

PC side output:

```
[*] enp7s0 configurato     192.168.10.70/24
[*] dnsmasq avviato (pid 28959)
[*] DHCP pool 192.168.10.100     192.168.10.200
[*] DHCP log: tail -f /tmp/dnsmasq-caos42.log
[*] Waiting for DHCP ACK from the board (max 60s)...
[*] Board IP: 192.168.10.148
[*] mosquitto avviato (pid 28965)  192.168.10.70:1883
[*] Paho connesso al broker
[*] Sottoscritto a caos/status
============================================================
```

```
 caOS EX42     MQTT broker  192.168.10.70:1883
 Board IP:  192.168.10.148
 Subscribe: caos/status
 Publish:   caos/led
 DHCP pool  192.168.10.100     192.168.10.200
============================================================
 Commands: on | off | q
------------------------------------------------------------
>   [caos/status] {"seq":0,"up":8,"ip":"192.168.10.148"}
> off
  -> [caos/led] off
> on
  -> [caos/led] on
>   [caos/status] {"seq":2,"up":18,"ip":"192.168.10.148"}
```

> **Note**: The script waits for the DHCP ACK from the board (max 60 s) before starting mosquitto. This ensures that the board has a valid IP before the broker is reachable.

Board output (stdout/USB)

```
=== EX42: MQTT client ===
MAC     02:00:00:00:00:42
Broker  via DHCP GW :1883
Topics  pub=caos/status  sub=caos/led
DHCP: avvio DORA...

DHCP BOUND  IP=192.168.10.148
MQTT broker 192.168.10.70:1883
MQTT: avvio connessione...
MQTT CONNECTED  (#1)
MQTT subscribed -> caos/led
MQTT pub -> {"seq":0,"up":8,"ip":"192.168.10.148"}
MQTT pub -> {"seq":1,"up":13,"ip":"192.168.10.148"}
MQTT LED -> OFF
MQTT LED -> ON
STATS dhcp=BOUND mqtt=UP conn=1 pool=56/64 held=8 rx=35 tx=29
```

65.1.7 Technical Notes

No dedicated MQTT task

Unlike NTP or Syslog (which have a dedicated task for polling or draining the queue), the MQTT service is entirely callback-driven: RX packets are processed by `on_tcp_rx()` in the context of the net task, without additional buffering. `task_mqtt` exists only for application timers (publish, ping, reconnect) and has no synchronization dependencies with the net task.

Stateful RX parser

TCP is a stream: an MQTT packet can arrive in multiple segments, or multiple packets can arrive in the same TCP segment. The service uses a three-state machine to correctly reassemble each packet before processing it:

```
state 0: waiting for fixed header (1 byte)
state 1: decodes remaining length (1-4 bytes, 7-bit varlen encoding)
state 2: accumulating payload in rx_buf up to rem_len bytes
             mqtt_process_packet()
```

Packets with payload > `MQTT_RX_BUF_SIZE` (256 B) are silently discarded, resetting the state parser to 0.

Automatic CONNECT on TCP established

When the 3-way handshake completes, `on_tcp_connected()` is called from the TCP layer in the context of the net task and immediately sends MQTT CONNECT. The application does not need to handle the timing between TCP connect and MQTT connect: just wait for `on_mqtt_connected()`.

Topics and payload in on_mqtt_message callback

The pointers `topic` and `payload` passed to `on_mqtt_message` point directly to `rx_buf` inside the service. They are valid **only during the callback**. If the data must survive the callback (e.g. passing to another task), copy them before returning.

on_dhcp_bound sequence

```
ca_net_proto_set_local_ip(ip)          proto usa IP corretto nei frame TCP
ca_arp_service_update_local_ip(ip)     ARP usa IP corretto per resolve broker
ca_arp_send_garp()                     updates the peers' caches on the LAN
ca_sem_post(&s_dhcp_sem)               unblocks task_mqtt only after all of the
    above
```

Pool buffer and persistent connection

The TCP connection remains open for the entire duration of the MQTT session. The pool buffers are only occupied during TX (`ca_mqtt_publish`) and released on ACK. In idle, the connection does not hold buffers.

Broker IP from DHCP gateway

The broker IP is not hardcoded: `on_dhcp_bound` saves the `gw` field and passes it to `ca_mqtt_service_init`. This makes the code independent from the PC's IP: just make sure the PC is configured as a DHCP gateway (`--pc-ip` option).

```
static void on_dhcp_bound(ca_ipv4_t ip, ca_ipv4_t mask, ca_ipv4_t gw, ca_ipv4_t
    dns)
{
    s_ip = ip;
    s_gw = gw;   /* broker = gateway */
    ca_sem_post(&s_dhcp_sem);
}
/* task_mqtt: */
ca_ipv4_t broker_ip = s_gw;
ca_mqtt_service_init(broker_ip, MQTT_BROKER_PORT, ...);
```

65.1.8 Fixes implemented during HW testing

The following fixes were introduced during testing on NUCLEO-H753ZI and are permanent part of the TCP/MQTT layer.

1 — Mutex deadlock in `tcp_service.c` (callback `on_connected`)

Problem: `on_tcp_connected` was called from the net task while `s_mtx` was still acquired. The callback called `ca_tcp_send` → `ca_mutex_lock(&s_mtx)` → deadlock detected by SAFEGUARD with `RESTART`.

Fix: in `TCP_STATE_SYN_SENT`, the mutex is released before the callback, using the same pattern as `on_rx` and `on_accept`.

2 — SEM OVERFLOW on conn_sem (reused PCB slot)

Problem: conn_sem (max=1) was not reset between one connection and the next on the same slot. In async mode (callback on_connected set), nobody consumes the semaphore after the post. On the second connection the post caused overflow (cnt=1 2).

Fix: ca_sem_init(&p->conn_sem, 0, 1) reinitialized in ca_tcp_connect before SYN. The post of conn_sem now happens only in blocking mode (on_connected == NULL).

3 — TIME_WAIT on Linux (fixed ephemeral port)

Problem: on every reconnection the board used the same local port (49200 + slot). The Linux server, with the old 4-tuple in TIME_WAIT (60 s), responded with challenge ACK instead of SYN-ACK.

Fix: s_ephemeral_ctr counter rotates ports on 49200..49399. Every ca_tcp_connect assigns a different port from the previous one.

4 — MQTT CONNECT truncated (inflight slots exhausted)

Problem: send_connect() called ca_tcp_send 4 separate times (fixed, rem_len, proto, vhdr), saturating the 4 inflight slots (TCP_MAX_INFLIGHT=4). The fifth call for client_id failed silently; mosquitto did not respond with CONNACK.

Fix: send_connect, ca_mqtt_publish and ca_mqtt_subscribe now build the complete packet in a local buffer (MQTT_TX_BUF_SIZE=128) and call ca_tcp_send only once.

5 — %lX in ca_vsnprintf

Problem: length modifiers l/h (%lX, %lu) were not supported in ca_vsnprintf and caused literal printing of the format.

Fix: l and h are now ignored in the format options loop (ARM32: u32/i32 = 4 bytes, no functional difference).

Chapter 66

Modbus TCP Client

66.1 Example 43 — Modbus TCP Client

66.1.1 Introduction

Application example of the caOS Modbus TCP client on NUCLEO-H753ZI (STM32H753ZI).

The board obtains an IP address via DHCP, opens a TCP connection to a Modbus server on port 502 and performs a periodic cycle of reading and writing using function codes FC01, FC03, FC06, FC16. An interactive CLI allows executing on-demand transactions without waiting for the automatic cycle.

Feature	Notes
DHCP client	DORA + synchronization semaphore towards task_modbus
ARP service	GARP at bind, gateway MAC resolution
Modbus TCP client	MBAP framing, serialization with mutex
FC03 Read Holding Registers	4 registers (addr 0..3)
FC06 Write Single Register	setpoint (addr 4)
FC16 Write Multiple Registers	2 config registers (addr 5..6)
FC01 Read Coils	8 coils (addr 0..7)
Interactive CLI	`rdreg`, `wrreg`, `rdcoil`, `stat`
Automatic reconnection	retry every 5 s
ICMP echo reply	ping from board during test
Task stats	pool, OK/ERR counters, connection status

66.1.2 Hardware

Board: NUCLEO-H753ZI (STM32H753ZI, Cortex-M7 @ 480 MHz)
PHY: LAN8742A (RMII, phy_addr=0)

Pin	Function
PB0	Red LED — ETH link up/down
PE1	Green LED — Modbus cycle OK
PB14	Blue LED — heartbeat stats
PA1, PA2, PA7, PC1, PC4, PC5, PG11, PB13, PG13	ETH RMII (AF11)

66.1.3 os_config.h Configuration

```
#define EN_ETH                1
#define EN_NET                1
#define EN_ARP_SERVICE        1
#define EN_DHCP_SERVICE       1
#define EN_NET_DHCP           1
#define EN_TCP_SERVICE        1
#define EN_MODBUS_SERVICE     1
#define MODBUS_TIMEOUT_MS     1000
#define MODBUS_RX_BUF_SIZE    270
```

For the CLI add `ENABLE_CLI 1` and `EN_CLI_MODBUS 1`.

66.1.4 Architecture

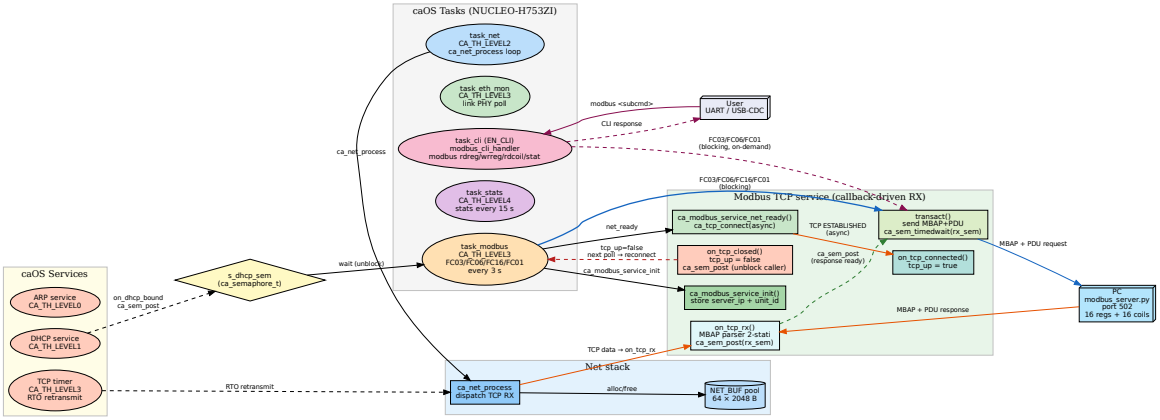


Figure 66.1: Example architecture 43

The system starts 4 tasks:

Task	Priority	Function
`task_net`	LEVEL2	RX loop: `ca_sem_wait` + `ca_net_process`
`task_eth_mon`	LEVEL3	ETH link polling every 500 ms
`task_modbus`	LEVEL3	Modbus loop + reconnection
`task_stats`	LEVEL4	print statistics every 15 s

The DHCP gateway is used as the Modbus server address: `s_gw` is captured in the callback `on_dhcp_bound` and passed to `ca_modbus_service_init`.

66.1.5 task_modbus flow

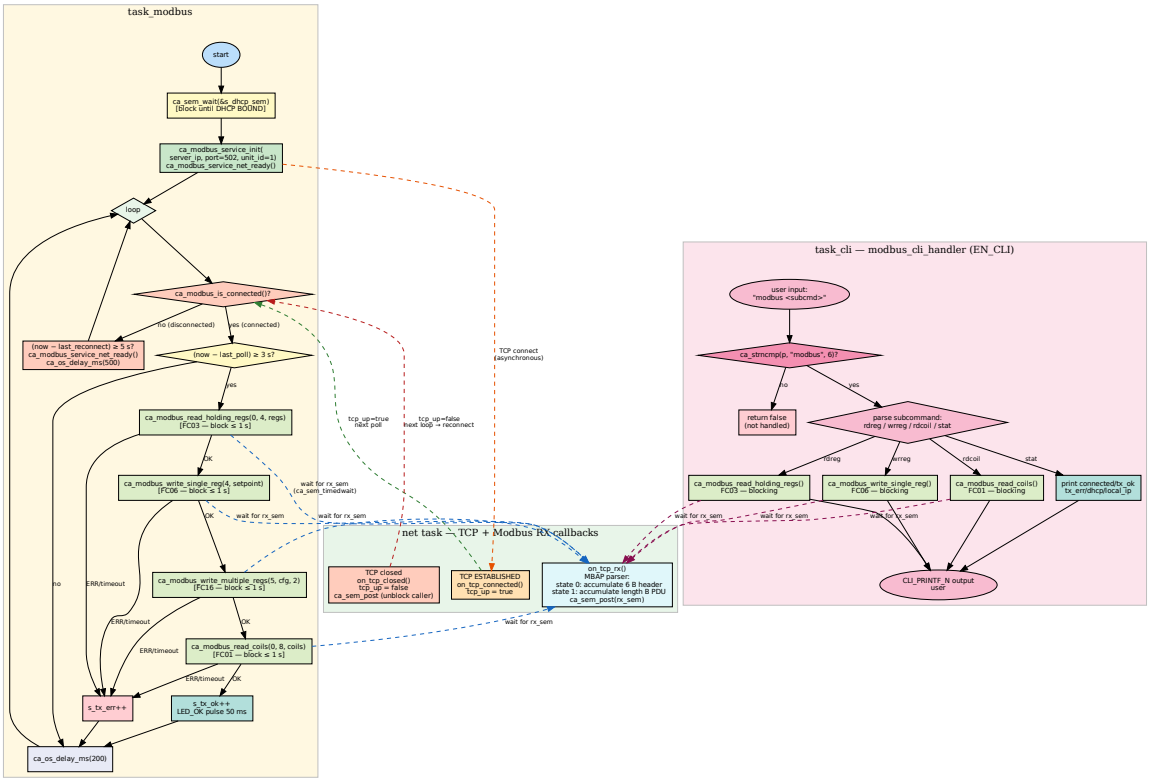


Figure 66.2: task_modbus flow

```
ca_sem_wait(&s_dhcp_sem)              sblocca on_dhcp_bound
ca_modbus_service_init(...)
ca_modbus_service_net_ready()         invia SYN

loop:
  if !connected:
    if timeout 5s: reinit + reconnect
    delay 500ms; continue

  if poll timeout 3s:
    FC03 leggi 4 registri
    FC06 scrivi setpoint
    FC16 scrivi 2 config
    FC01 leggi 8 coil
        ok++  o  err++

  delay 200ms
```

66.1.6 Interactive CLI

With `EN_CLI_MODBUS 1` and `ENABLE_CLI 1` the `modbus_cli_handler` node is available.

Command	Description
`modbus help`	lists sub-commands

Command	Description
`modbus rdreg <addr> [count]`	FC03 — reads 1..4 holding registers
`modbus wrreg <addr> <val>`	FC06 — writes a register
`modbus rdcoil <addr> [count]`	FC01 — reads 1..8 coil
`modbus stat`	connection status, local IP, OK/ERR counters

CLI and automatic task are serialized by the internal `tx_mtx` in the Modbus service: if the automatic task is waiting for a response, the CLI command blocks until the current transaction completes (at most `MODBUS_TIMEOUT_MS`).

Sample session

```
> modbus stat
  connected : YES
  tx_ok     : 3
  tx_err    : 0
  dhcp      : BOUND
  local_ip  : 192.168.10.149

> modbus rdreg 0 4
  reg[0] = 101 (0x0065)
  reg[1] = 201 (0x00C9)
  reg[2] = 301 (0x012D)
  reg[3] = 401 (0x0191)

> modbus wrreg 4 9999
  reg[4] <- 9999 OK

> modbus rdcoil 0 8
  coil[0] = 1
  coil[1] = 0
  coil[2] = 1
  coil[3] = 0
  coil[4] = 1
  coil[5] = 0
  coil[6] = 1
  coil[7] = 0
```

66.1.7 Test PC — modbus_server.py

The Python server in the example directory simulates a Modbus TCP device with 16 holding registers and 16 coils, automatically updated every 5 s.

```
sudo python3 examples/example_43/modbus_server.py --iface enp7s0
```

The server: 1. Configures `enp7s0` with IP `192.168.10.1/24` 2. Starts `dnsmasq` as DHCP server (pool `192.168.10.100..200`) 3. Waits for the board's DHCP ACK (max 60 s) 4. Listens for Modbus TCP server on `192.168.10.1:502`

Optional arguments:

Argument	Default	Description
`--iface`	`enp7s0`	Ethernet interface
`--pc-ip`	`192.168.10.1`	PC IP (Modbus server)
`--port`	`502`	TCP port
`--no-dhcp`	—	skip IP/DHCP setup (IP already configured)

Expected output from the server:

```
[+] client 192.168.10.149:49201
  RX tid=1 unit=1 FC=0x03
  RX tid=2 unit=1 FC=0x06
  [WR] reg[4] = 100
  RX tid=3 unit=1 FC=0x10
  [WR] reg[5..6] = [1, 2]
  RX tid=4 unit=1 FC=0x01
  RX tid=5 unit=1 FC=0x03
  ...
```

66.1.8 Expected UART Output

```
=== EX43: Modbus TCP client ===
MAC     02:00:00:00:00:43
Server  192.168.10.1:502  unit_id=1
DHCP: avvio DORA...
DHCP BOUND  IP=192.168.10.149
MODBUS server 192.168.10.1:502
MODBUS: avvio connessione TCP...
FC03 regs[0..3]= 100 200 300 400
FC06 setpoint=100 OK
FC16 config OK
FC01 coils=0x55
FC03 regs[0..3]= 100 200 300 400
FC06 setpoint=200 OK
FC16 config OK
FC01 coils=0x55
STATS dhcp=BOUND modbus=UP ok=2 err=0 pool=56/64 held=8 rx=21 tx=38
```

`held=8` is the normal value in steady state: 4 RX DMA slots + 4 TX DMA slots occupied by the ETH H7xx driver.

66.1.9 Technical Notes

Three-segment TCP Framing

`send_request` sends the Modbus ADU with three separate calls to `ca_tcp_send`: MBAP header (6 bytes), unit_id (1 byte), PDU body (N bytes). Each call occupies a slot inflight in the TCP service. The server receives the segments in order and reassembles them in the socket buffer before passing them to the application.

CLI Serialization + automatic task

The internal mutex `tx_mtx` prevents overlaps between the automatic task and the CLI. The pattern in `transact()`:

```
ca_mutex_lock(&s.tx_mtx);
/* drain stale rx_sem */
while (ca_sem_trywait(&s.rx_sem)) {}
send_request(tid, ...);
ok = ca_sem_timedwait(&s.rx_sem, MODBUS_TIMEOUT_MS);
ca_mutex_unlock(&s.tx_mtx);
```

Callback stale after reconnection

The old connection (TIME_WAIT) can invoke `on_tcp_closed` after the new one is already ESTABLISHED. The check `handle != s.tcp_handle` at the entrance of each callback ignores these events:

```
static void on_tcp_closed(u8 handle)
{
    if (handle != s.tcp_handle) return;
    s.tcp_up = false;
    ca_sem_post(&s.rx_sem);
}
```

Timeout and Reconnection

If the server does not respond within `MODBUS_TIMEOUT_MS` (default 1000 ms), `transact()` returns `false`. The TCP connection remains open; the task retries on the next cycle. If `ca_modbus_is_connected()` returns `false` (TCP closed by RST or FIN), the task reinitializes and reconnects after 5 s.

66.1.10 TCP CLI command sequence during testing

```
tcp list              1  ESTABLISHED  49201  502  192.168.10.1
tcp info 1            snd_wnd=64240 inflight=0/4
modbus stat           connected=YES ok=N err=0
modbus rdreg 0 4      reg[0..3] valori aggiornati
```

Chapter 67

iperf TCP Client/Server

67.1 Example 44 — iperf TCP client/server

67.1.1 Introduction

Example of TCP throughput measurement on NUCLEO-H753ZI (STM32H753ZI) using the caOS iperf service.

The board obtains an IP address via DHCP and provides two operational modes selectable from CLI:

- **client**: the board opens a TCP connection to the PC and transmits data in bulk for the specified duration;
- **server**: the board accepts an incoming TCP connection and measures the throughput of received data.

Feature	Notes
DHCP client	DORA + IP update at `ca_iperf_update_local_ip`
ARP service	GARP on bind, gateway MAC resolution
TCP service	connection management, RTO timers
iperf service	TX/RX loop, measure bytes/ms/bps
Interactive CLI	`iperf client <ip> [port] [dur_s]`, `iperf server [port] [idle_s]`
Pool diagnostics	`tcp buf` — free, held, leak pool
ICMP echo reply	ping from board during test

67.1.2 Hardware

Board: NUCLEO-H753ZI (STM32H753ZI, Cortex-M7 @ 480 MHz)
PHY: LAN8742A (RMII, phy_addr=0)

Pin	Function
PB0	Red LED — ETH link up/down
PB14	Blue LED — heartbeat stats
PA1, PA2, PA7, PC1, PC4, PC5, PG11, PB13, PG13	ETH RMII (AF11)

67.1.3 os_config.h Configuration

```
#define EN_ETH              1
#define EN_NET              1
#define EN_ARP_SERVICE      1
#define EN_DHCP_SERVICE     1
#define EN_NET_DHCP         1
#define EN_TCP_SERVICE      1
#define EN_IPERF_SERVICE    1
#define TCP_RX_RING_DEPTH   16U
```

`NET_BUF_COUNT` **is not here**: it is passed by CMake, in `examples/<fam>/<board>/CMakeLists.txt`, and it is a per-part value — 64 on H753, F7xx and H563, 32 on U575, 16 on F4xx, 0 on F103 (which has no MAC). On the H753 there are 64 × 2 KB = 128 KB in NET_SRAM1.

67.1.4 Architecture

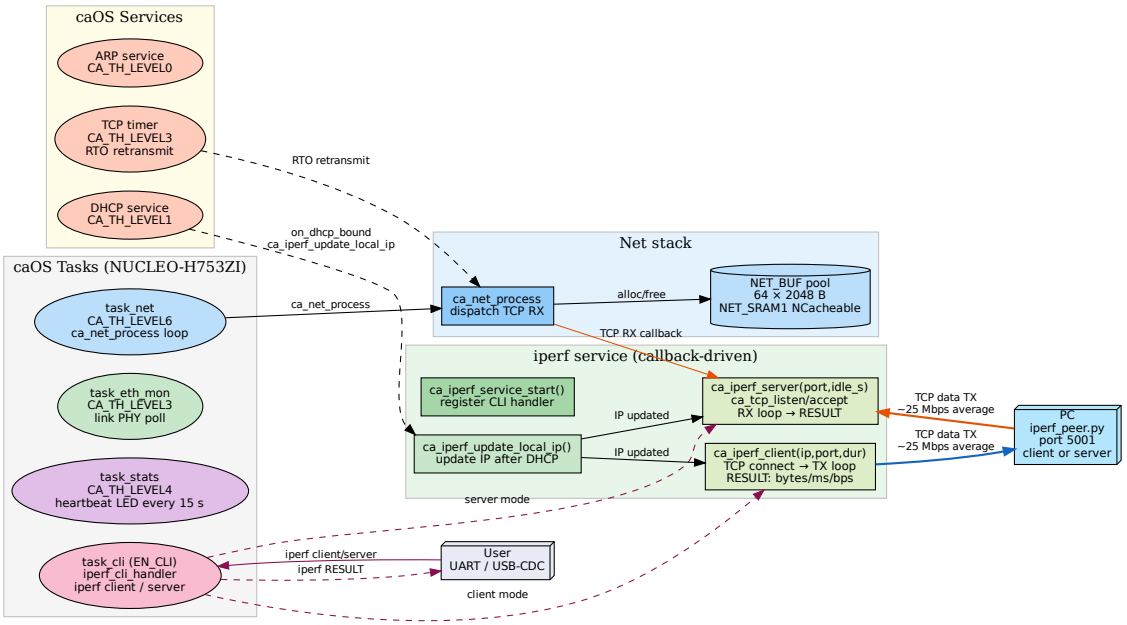


Figure 67.1: Example architecture 44

The system starts 3 application tasks; the iperf service is executed in the context of the task that invokes the CLI command.

Task	Priority	Function
`task_net`	LEVEL6	RX loop: `ca_sem_wait` + `ca_net_process`
`task_eth_mon`	LEVEL3	ETH link polling every 500 ms
`task_stats`	LEVEL4	LED heartbeat toggle every 15 s
`task_cli` (EN_CLI_IPERF)	LEVEL5	iperf client / server on-demand

67.1.5 iperf client and server flow

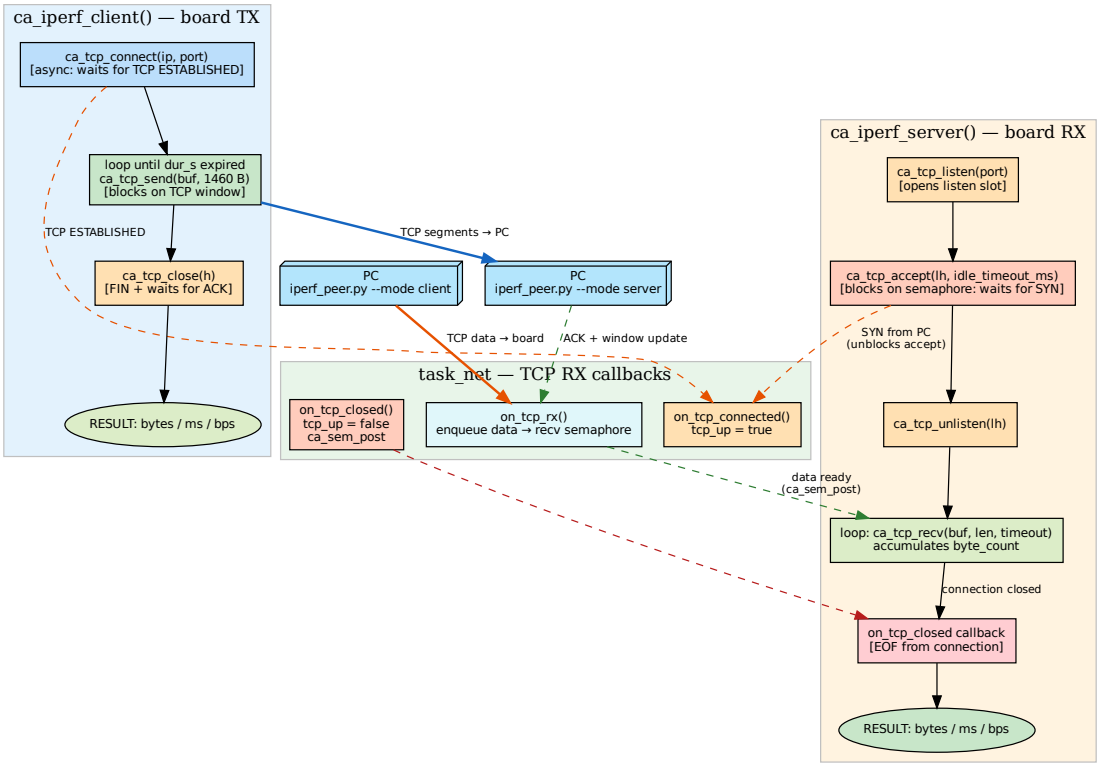


Figure 67.2: iperf flow

Client Mode (board → PC)

```
ca_iperf_client(ip, port=5001, dur_s=10)
      ca_tcp_connect(ip, port)          # TCP SYN
      loop dur_s secondi:
      ca_tcp_send(buf, 1460)          # MSS segments, blocking on window
      ca_tcp_close(h)                   # FIN
      stampa RESULT: bytes / ms / bps
```

Server Mode (PC → board)

```
ca_iperf_server(port=5001, idle_s=30)
      ca_tcp_listen(port)               # opens a listen slot
      ca_tcp_accept(lh, idle_timeout_ms)# attende SYN (bloccante)
      ca_tcp_unlisten(lh)
      loop:
      ca_tcp_recv(buf, len, timeout)  # accumula byte_count
      [on_tcp_closed     EOF]
      stampa RESULT: bytes / ms / bps
```

67.1.6 Interactive CLI

With `EN_CLI_IPERF 1` and `ENABLE_CLI 1` the service automatically registers `iperf_cli_handler`.

Command	Description
`iperf help`	list sub-commands
`iperf client <ip> [port] [dur_s]`	start TX bulk towards PC
`iperf server [port] [idle_s]`	listen, measure RX
`tcp buf`	free pool / held / leak

Example session — client

```
> iperf client 192.168.10.1 5001 10
  iperf RESULT: 39906910 bytes  10036 ms  31925528 bps

> tcp buf
  pool=40/64  held=24  leak=0
```

Example session — server

```
> iperf server 5001 30
  iperf RESULT: 38675400 bytes  10000 ms  30940320 bps

> tcp buf
  pool=40/64  held=24  leak=0
```

67.1.7 Test PC — iperf_peer.py

The Python peer in the example directory acts as the opposite role to the board.

```
# board as client, PC as server
python3 examples/example_44/iperf_peer.py --mode server [--port 5001]

# board as server, PC as client
python3 examples/example_44/iperf_peer.py --mode client --ip <board_ip> \
        [--port 5001] [--dur 10]
```

Alternatively with netcat:

```
# PC as server
nc -l 5001 | pv > /dev/null

# PC as client
dd if=/dev/zero bs=64k | nc <board_ip> 5001
```

67.1.8 Expected UART Output

```
=== EX44: iperf TCP client/server ===
MAC  02:00:00:00:00:44
DHCP: avvio DORA...
ETH LINK UP
DHCP BOUND  IP=192.168.10.150

```

```
> iperf client 192.168.10.1 5001 10
  iperf RESULT: 39906910 bytes  10036 ms  31925528 bps

> iperf server 5001 30
  iperf RESULT: 38675400 bytes  10000 ms  30940320 bps
```

67.1.9 Measured Performance

Tests on direct board–PC link (Ethernet 100BASE-TX, cat.5e cable):

Mode	Average Throughput	Peak Throughput
Client (board TX → PC RX)	~25 Mbps	~32 Mbps
Server (PC TX → board RX)	~25 Mbps	~39 Mbps

The average throughput of 25 Mbps on a 100 Mbps link reflects:

- **Cortex-M7 overhead**: CPU copy of payloads between DMA buffers and application buffers — the lack of scatter-gather DMA at L4 level forces two memcpy operations per TCP segment;
- **caOS scheduling**: the SysTick at ~104 µs introduces wakeup latency in the task_net; the presence of `__WFI()` in the idle task is essential to immediately yield the CPU to the ETH interrupt without burning cycles.

What is NOT the bottleneck

Not the TCP window. The assumption is natural, and with an example using `TCP_RX_RING_DEPTH=4` the math even works out too well ($4 \times \text{MSS} = 5840$ bytes, which at 32 Mbps implies an RTT of 1.45 ms). But it is false: the cost is **per packet**, and a wider window only queues more packets that cost the same. Here the ring is already **16** and the throughput is the same as ex79, which had 4 — which settles the matter by itself.

Not the driver. On the same silicon, example 84 — L2 echo, with the transport layer removed — achieves **78 Mbps with 5000 frames out of 5000** and a 1.00 transmission-to-reception ratio. Here the ratio is 1.86.

The meaningful comparison is in **events per second**, not bits: a 1500-byte frame and a 60-byte ACK cost almost the same, so Mbps can be misleading.

	Mbps	events/s
ex84, L2 echo (RX **and** TX per frame)	62.3	~10 200
ex44, iperf (RX + 1 ACK every 2)	23–32	~3 000–4 100

Where the traffic really goes

A run of iperf on the H563, captured with `eth dump`:

```
rx frames=156484   tx calls=290841   rbu=648
```

34.8 MB divided by the MSS make **23,866 data frames** out of 156,484 received: the **15.3%**. The other 132,618 are ACKs and retransmissions, and the board transmits 1.86 for each one it receives. `rbu=648` says the receive ring runs dry: frames are lost, the PC retransmits, the load increases, the ring runs dry again.

It's a self-sustaining cycle, and there's much more margin here than in any TCP parameter.

67.1.10 Memory

Region	Address	Size	Content
NET_SRAM1	0x30000000	128 KB	pool 64 × 2 KB (MPU NCacheable)
USB_SRAM	0x30020000	128 KB	reserved for future USB DMA
ETH_SRAM3	0x30040000	32 KB	ETH DMA descriptors + ring buf

`held=24` under load: 8 RX DMA slots + 16 TX DMA slots occupied by the ETH H7xx driver.

67.1.11 Technical Notes

NCacheable Pool and DMA Coherency

The pool `.net_pool` is mapped in D2 SRAM1 and the MPU region 3 is configured as Normal NCacheable. CPU and ETH DMA share the buffers without explicit cache invalidation.

> **The D-cache is disabled on this part**, as on all M7 in caOS (the I-cache remains enabled). The NCacheable region is therefore not what makes CPU and DMA coherent — it would be if the cache were active, and it remains the correct configuration if the cache is ever re-enabled. With the cache disabled **no cache maintenance is performed**: `clean`/`invalidate` on a disabled D-cache are not a harmless redundancy.
>
> Barriers are another matter: `__DSB()`/`__DMB()` in the ETH driver order bus accesses and are also needed with a disabled cache, because Normal Non-cacheable memory is not Strongly-ordered and the DMA writes the same memory as an independent master.

Importance of `__WFI()` in the idle task

Removing `__WFI()` from the idle task causes an increase from 31% to 57% of CPU idle time with 286,000 context switches/s: the idle task competing actively with task_net degrades throughput. With `__WFI()` the CPU stops until the ETH interrupt and restarts immediately, minimizing receive latency.

TCP CLI Commands during testing

```
tcp list            1  ESTABLISHED  <sport>  5001  192.168.10.1
tcp info 1          snd_wnd=65535  inflight=N/4
tcp buf             pool=40/64  held=24  leak=0
```

Chapter 68

HTTP Server with CLI over HTTP

68.1 Example 45 — HTTP and HTTPS Server with CLI via HTTP

68.1.1 Introduction

Example of an HTTP server on NUCLEO-H753ZI (STM32H753ZI) that exposes the entire caOS CLI via browser or `curl`.

The board obtains an IP address via DHCP and starts an HTTP server on port 80 and, with `EN_HTTPS_SERVICE`, an HTTPS server (TLS 1.3) on port 443 with the same routes. From any browser on the network, it is possible to execute CLI commands and view their output in text format.

Feature	Notes
DHCP client	DORA + IP update on bind
ARP service	GARP on bind, MAC resolution
TCP service	connection management
HTTP service	GET/POST/HEAD/OPTIONS, route table
HTTPS service	same routes on TLS 1.3, port 443 (HTTPS and TLS 1.3)
CLI capture	`ca_cli_exec_capture` — redirect stdout to HTTP buffer
ICMP echo reply	ping from the board
Packet tracer	`trace on/off/filter` — dump RX/TX frames on UART

68.1.2 Hardware

Board: NUCLEO-H753ZI (STM32H753ZI, Cortex-M7 @ 480 MHz)
PHY: LAN8742A (RMII, phy_addr=0)

Pin	Function
PB0	Red LED — ETH link up/down
PE1	Green LED — heartbeat
PA1, PA2, PA7, PC1, PC4, PC5, PG11, PB13, PG13	ETH RMII (AF11)

68.1.3 os_config.h configuration

```
#define EN_ETH              1
#define EN_NET              1
#define EN_ARP_SERVICE      1
#define EN_DHCP_SERVICE     1
#define EN_NET_DHCP         1
#define EN_TCP_SERVICE      1
#define EN_HTTP_SERVICE     1
#define EN_CLI_CAPTURE      1
#define EN_PACKET_TRACER    0    /* opzionale: CLI trace on/off/filter */
#define EN_CLI_TRACE        0    /* accendere entrambi per usare 'trace' */
#define NET_BUF_COUNT       64U

/* Dimensioni buffer HTTP     critiche per browser reali */
#define HTTP_REQ_BUF_SIZE   2048U   /* Chrome invia ~610 byte di headers POST */
#define HTTP_RESP_BUF_SIZE  4096U   /* copre output CLI lunghi: top, help, ecc.
    */

/* HTTPS (23/10/2026) */
#define EN_HTTPS_SERVICE    1
#define EN_TLS              1
#define EN_ECDSA_P256       1    /* firma del certificato */
#define EN_X25519           1    /* scambio chiavi */
#define EN_AES              1    /* AES-128-GCM del record layer */
#define EN_SHA256           1
#define EN_RNG              1
#define EN_KEYS_STORE       1    /* la chiave privata sta nel keys_store */
#define EN_CLI_HTTPS        1    /* comando 'https info' */
```

Flags are set using `caos_config.py --on ...`, never manually.

> **Warning**: values smaller than `2048U` and `4096U` (the current defaults in `http_service.h`) are insufficient for real browsers. Chrome sends POST headers of ~610 bytes (User-Agent, Accept, Referer, Origin, Accept-Language, etc.); with a 512-byte buffer, the body falls into the headers → 400 Bad Request. Always set `HTTP_REQ_BUF_SIZE 2048` in `os_config.h`.

68.1.4 Architecture

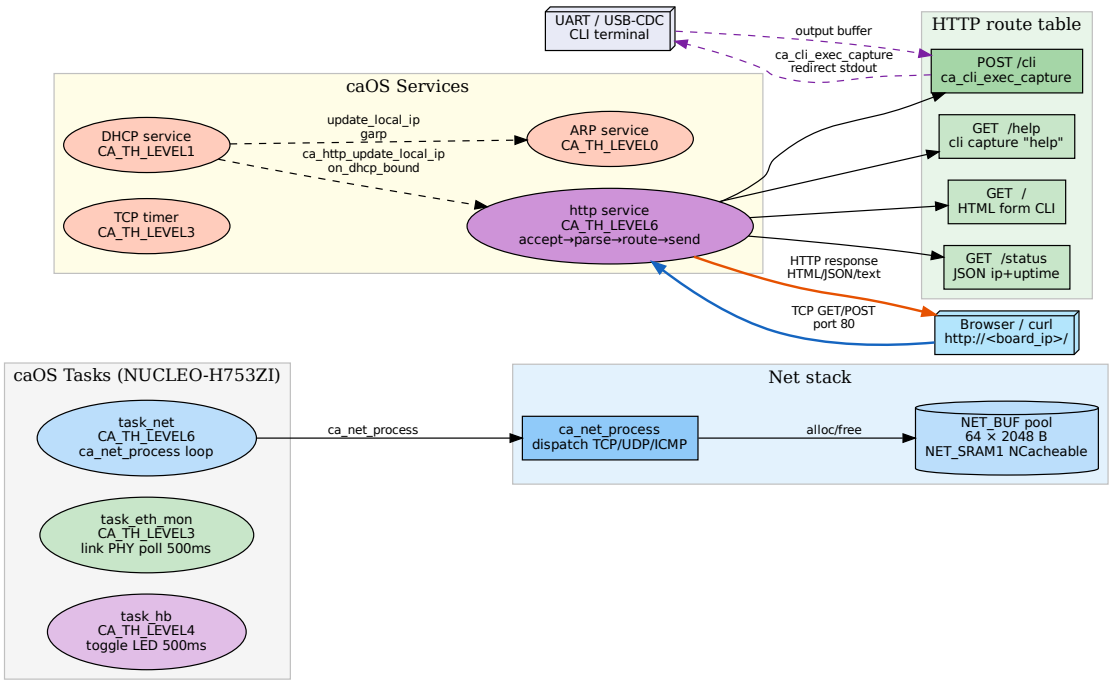


Figure 68.1: Task and service architecture

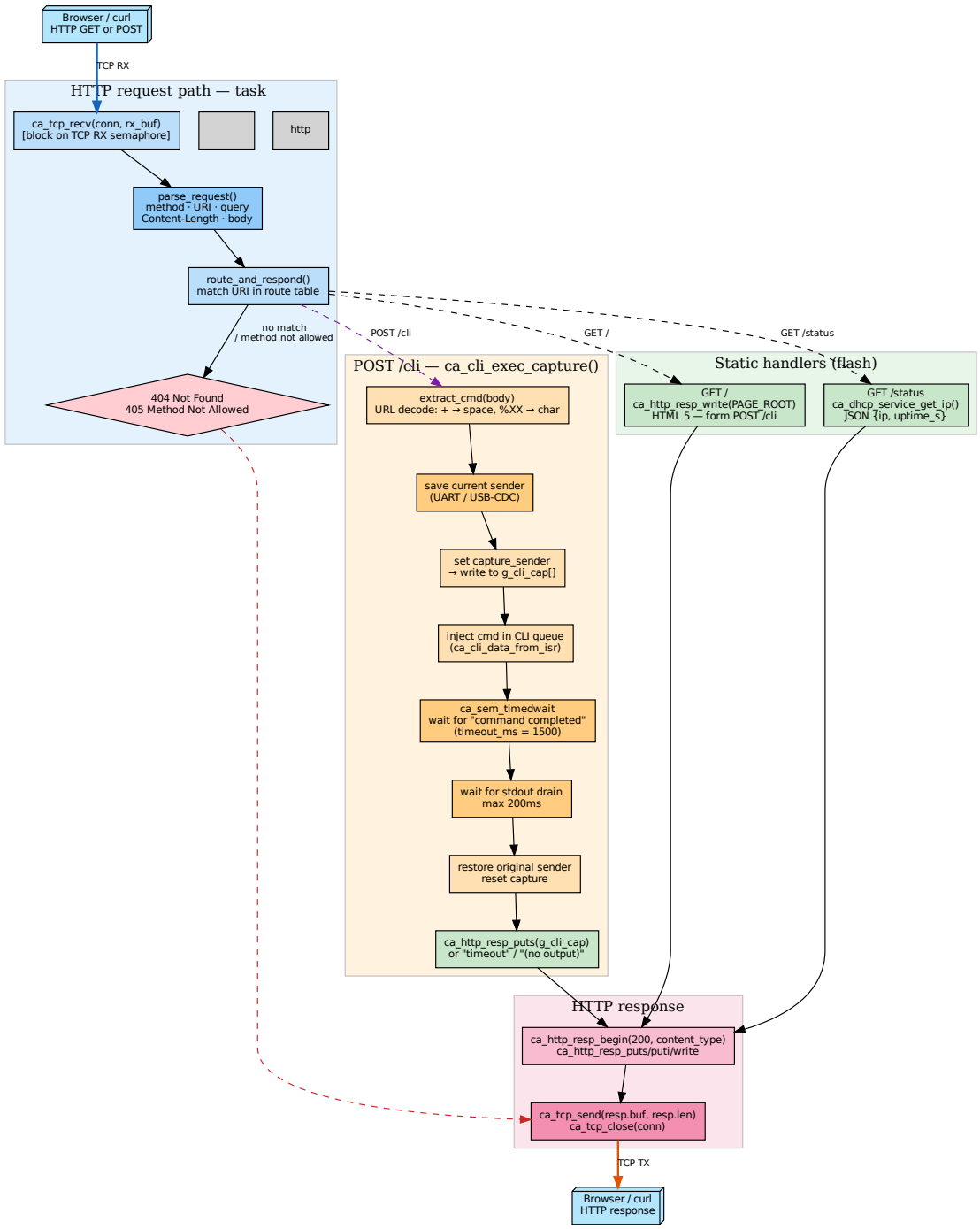


Figure 68.2: HTTP request flow — parse, route, CLI capture

Task	Priority	Function
`task_net`	LEVEL6	RX loop: `ca_sem_wait` + `ca_net_process`
`task_eth_mon`	LEVEL3	ETH link polling every 500 ms

Task	Priority	Function
`task_hb`	LEVEL4	heartbeat LED toggle every 500 ms
`http` (internal)	LEVEL6	accept → parse → route → response → close
`https` (internal)	LEVEL6	accept → TLS handshake → same path as `http` → close_notify → close
`ptrace` (internal)	LEVEL3	drain tracer ring, filter, print to sender

---

68.1.5 Route Table

URI	Methods	Description
`/`	GET, HEAD, OPTIONS	HTML page with command form
`/status`	GET, HEAD, OPTIONS	JSON: `{"ip":"...","uptime_s":N}`
`/cli`	POST, OPTIONS	Executes CLI command, responds with text
`/help`	GET, HEAD, OPTIONS	Output of `help` from the CLI

68.1.6 PC Tests

Browser

Open `http://<board_ip>/` — a text form with a command field appears. Type `top`, `free`, `eth`, `arp`, `dhcp info`, `tcp list`, etc.

curl

```
# IP assegnato da DHCP
curl http://192.168.10.150/status
#     {"ip":"192.168.10.150","uptime_s":42}

curl -X POST http://192.168.10.150/cli -d "cmd=top"
#     output del comando "top" dalla CLI caOS

curl -X POST http://192.168.10.150/cli -d "cmd=free"
#     blocchi memoria liberi

curl http://192.168.10.150/help
#     lista comandi CLI disponibili
```

HTTPS

The server identity (P-256 key and self-signed certificate for `caos.local`) is in `examples/example_45/tls_identity.h`, generated with:

```
python3 tools/gen_tls_identity.py examples/example_45
```

The certificate carries a name and not the IP, because the IP comes from DHCP:

```
curl --cacert examples/example_45/tls_identity_cert.pem \
     --resolve caos.local:443:192.168.10.84 https://caos.local/status
#     {"ip":"192.168.10.84","uptime_s":53}

curl --cacert examples/example_45/tls_identity_cert.pem \
     --resolve caos.local:443:192.168.10.84 -X POST https://caos.local/cli -d "
    cmd=free"
```

Browser: add `192.168.10.84 caos.local` to `/etc/hosts` and open `https://caos.local/`. The page opens with an "insecure" warning, which is normal for a self-signed certificate (verified on H563 on 10/23/2026).

Python script (automated test)

```
# Con DHCP automatico (richiede root per dnsmasq)
sudo python3 examples/example_45/http_test.py --iface enp7s0 --pc-ip
    192.168.10.1

# Senza DHCP, IP noto
python3 examples/example_45/http_test.py --no-dhcp --board-ip 192.168.10.158

# Singolo comando CLI, no browser
python3 examples/example_45/http_test.py --no-dhcp --board-ip 192.168.10.158 --
    cmd "top"
```

68.1.7 Parsing HTTP POST — "from the bottom" body

The body of a POST (`cmd=top`, `cmd=free`, etc.) is the last `Content-Length` bytes of the TCP payload. `parse_request` finds it with a direct O(1) calculation:

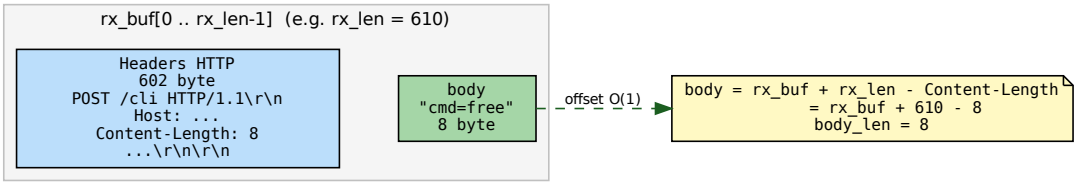


Figure 68.3: POST buffer layout — body offset

```
/* parse_request()      caos/net/src/http_service.c */
req->body     = buf + buf_len - content_length;   /* puntatore diretto */
req->body_len = (u16)content_length;              /* nessun loop */
```

The approach works because HTTP guarantees that the body is exactly the last `Content-Length` bytes of the received payload.

Multi-segment reception

If Chrome splits the request into multiple TCP segments (headers > 512 bytes in one segment, body in the next), `ca_http_serve_connection` accumulates the data:

```
/* Loop recv finch  \r\n\r\n non    nel buffer (header completo) */
for (;;) {
    body_off = cerca_rnrn(rx_buf, rx_len);
    if (body_off > 0 || rx_len >= HTTP_REQ_BUF_SIZE) break;
    more = io_read(io, rx_buf + rx_len, ...);   /* segmento successivo */
    if (more == 0) break;
    rx_len += more;
}
/* Il body si aspetta solo se Content-Length > 0, fino a riceverlo tutto */
want = header_content_length(rx_buf, body_off);
while (want > 0 && rx_len - body_off < want && rx_len < HTTP_REQ_BUF_SIZE) {
    more = io_read(io, rx_buf + rx_len, ...);
    if (more == 0) break;
    rx_len += more;
}
```

Before 10/23/2026, the body always expected: every GET paid a 1.5 s timeout (`HTTP_RECV_TIMEOUT_MS`). Now a GET responds in less than a millisecond.

68.1.8 `ca_cli_exec_capture` flow

The detailed flow is shown in diagram `ex45_http_flow` (section “POST /cli — ca_cli_exec_capture()”):

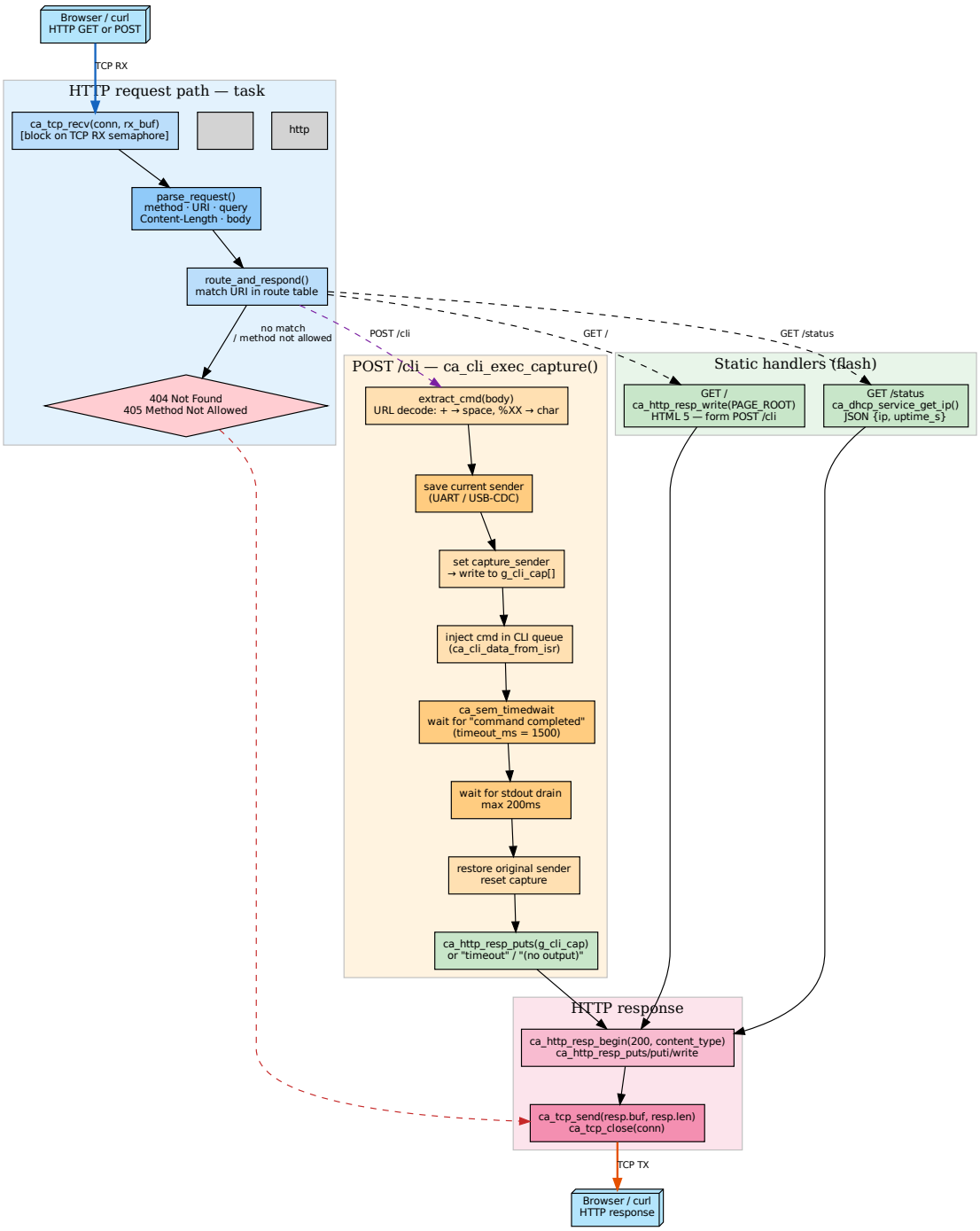


Figure 68.4: CLI capture flow

The mechanism uses a semaphore internal to the CLI thread to synchronize the HTTP caller with the completion of the asynchronous command.

68.1.9 Content-Length in the HTTP response

`ca_http_resp_begin` writes a 10-space placeholder for `Content-Length` and records its position in the buffer. Upon completion of the body construction, `ca_http_serve_connection` patches the actual value in place:

```
/* placeholder in ca_http_resp_begin() */
resp_append(r, "Content-Length: ", 16U);
r->cl_pos = r->len;
resp_append(r, "          ", 10U);   /* 10 spazi     patchati dopo */
r->hdr_end = r->len + 4U;           /* inizio body (dopo \r\n\r\n) */

/* patch in ca_http_serve_connection() dopo route_and_respond() */
u16 body_len = rp->len - rp->hdr_end;
ca_memcpy(resp.buf + resp.cl_pos, itoa10(body_len), 10U);
```

This avoids the double-pass on the response and is compatible with `HEAD` (body not written, `Content-Length` = 0 patched correctly).

68.1.10 Packet Tracer

The packet tracer is enabled by `ca_pktrace_start()` in `ca_os_main()` with sender set to `STDOUT`. At startup it is **disabled** — zero overhead until it is activated via CLI.

Terminal usage

```
c.a.O.S. > trace on
trace: on
[RX] ARP  192.168.10.1 -> 192.168.10.158 len=60
[TX] ARP  192.168.10.158 -> 192.168.10.1 len=42
[RX] ICMP 192.168.10.1 -> 192.168.10.158 len=98
[TX] ICMP 192.168.10.158 -> 192.168.10.1 len=98
[RX] TCP  192.168.10.1:48282 -> 192.168.10.158:80 len=74
[TX] TCP  192.168.10.158:80 -> 192.168.10.1:48282 len=58
[RX] TCP  192.168.10.1:48282 -> 192.168.10.158:80 len=531
[TX] TCP  192.168.10.158:80 -> 192.168.10.1:48282 len=1059

c.a.O.S. > trace filter src 192.168.10.1
src filter: 192.168.10.1

c.a.O.S. > trace off
trace: off
```

> **Note**: `trace filter` is used via serial terminal, **not** via HTTP POST `/cli`. The tracer uses an independent sender (not stdout); calling it during an HTTP capture session is safe because the two channels do not overlap.

68.1.11 Expected UART output

```
=== EX45: HTTP server + CLI via HTTP ===
MAC  02:00:00:01:02:45
DHCP: avvio DORA...
ETH LINK UP
DHCP BOUND  IP=192.168.10.158  GW=192.168.10.1
HTTP  http://192.168.10.158/
HTTPS https://192.168.10.158/
```

68.1.12 Technical notes

HTTP buffer — critical sizing

Parameter	Recommended value	Notes
`HTTP_REQ_BUF_SIZE`	2048	Chrome POST: ~610 byte headers + body
`HTTP_RESP_BUF_SIZE`	4096	output `top`/`help` can exceed 1 kB
`CLI_CAP_SIZE`	2048	capture CLI output in the handler

The old 512/1024 defaults were only sufficient for curl with minimal headers; now `http_service.h` uses 2048/4096. Real browsers send User-Agent, Accept, Accept-Language, Accept-Encoding, Referer, Origin, Cache-Control → ~610 bytes of headers alone.

Browser: simultaneous connections

Modern browsers (Chrome, Firefox) open **2–3 simultaneous TCP connections** for each loaded page (e.g., GET + prefetch favicon). The caOS TCP service uses a **FIFO queue** (`hq[]`) in the `accept_sem` to correctly handle multiple incoming connections without losing any.

Fix details in `tcp_service.c`: - `accept_sem` initialized with `max = TCP_MAX_CONN` (not 1) - `hq[TCP_MAX_CONN]` in every listen slot with head/tail → FIFO - `rx_sem` re-initialized upon PCB reuse (SEM OVERFLOW fix) - `pcb_close` uses `ca_sem_post_capped` to unblock pending recv without overflow

CLI capture serialization

`ca_cli_exec_capture` is not reentrant. With a single HTTP task (current configuration) this is not an issue. In a future multi-task version, use a protection mutex on `g_cli_cap`.

URL decode in POST body

The browser sends the body as `cmd=top` or `cmd=dhcp+info` (spaces → `+`). `extract_cmd` handles: - `+` → space - `%XX` → corresponding hex character - Terminator: `&` or end of buffer

```
/* extract_cmd     ricerca "cmd=" nel body con flag found */
bool found = false;
while (p + 4U <= end) {
    if (p[0]=='c' && p[1]=='m' && p[2]=='d' && p[3]=='=') {
        p += 4U; found = true; break;
    }
    p++;
}
if (!found || p >= end) return 0U;  /*     400 Bad Request */
```

68.1.13 Bench verification

The `test_ex45` class (10 phases) also tests HTTPS: `/status` and `POST /cli` over TLS 1.3 with the certificate verified by curl (`--cacert`, never `-k`), the rejection of a TLS 1.2-only client, and the `https info` counters. Passed on 10/23/2026 on H563, H753, and F767. On boards without Ethernet (U575, F446), the example declares itself unsupported.

Chapter 69

AES Channel Service: Encrypted Ping-Pong

69.1 Example 46 — Secure pipe: encrypted ping-pong between two tasks

69.1.1 Introduction

Example of **synchronous** encrypted communication between two tasks on NUCLEO-H753ZI. The two tasks exchange messages through a **secure pipe** (`ca_secure_pipe_t`, in `secure_pipe_service.h`): a `aes_channel_service` channel based on **AES-256-GCM** with integrated authentication, plus a queue for direction. Encryption, transport and buffer wipe are all handled by the service.

The objective is to verify end-to-end the entire crypto chain:

```
keys_store (MPU)     secure pipe (aes_channel + code)     encrypt/decrypt GCM
       verifica
```

Item	Value
Algorithm	AES-256-GCM (encryption + authenticity)
Wire format	`[nonce:12B][orig_len:2B][ciphertext_padded: ceil(N,16)B][tag:16B]`
Fixed overhead	30 bytes (`AES_CH_OVERHEAD`)
Key	32 bytes in `keys_store` (MPU-protected area)
IV/nonce	Generated by HW RNG (`ca_rng_fill`) for each message
Heap	internal buffers within the service, always freed with wipe
Communication	`ca_secure_pipe_t` — peers `CA_SECURE_PIPE_A` (Alice) and `_B` (Bob), depth 4
Iterations	100 ping-pong (`PING_COUNT`) with 1 s interval
HW Result	**100/100 ping OK**

69.1.2 Hardware

Board: NUCLEO-H753ZI (STM32H753ZI, Cortex-M7 @ 480 MHz)

Pin	Function
PB7	Green LED — heartbeat (toggle every 500 ms)

69.1.3 os_config.h Configuration

```
#define EN_AES                    1   /* H7xx CRYP peripheral */
#define EN_RNG                    1   /* TRNG for IV generation */
#define EN_KEYS_STORE             1   /* MPU-protected area for keys */
#define EN_AES_CHANNEL_SERVICE    1   /*  r e d black  encrypted pipe */
```

69.1.4 Architecture

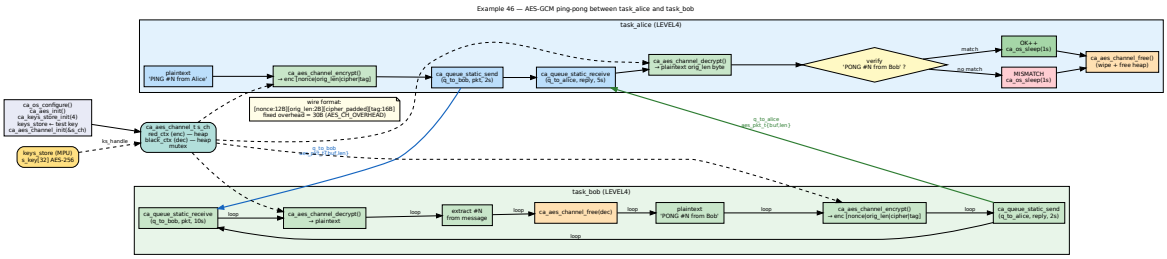


Figure 69.1: Ping-pong AES-GCM flow — task_alice, task_bob, keys_store, wire format

The pipe

```
static ca_secure_pipe_t s_pipe;

/* ca_os_configure() */
ca_secure_pipe_init(&s_pipe, CA_AES_MODE_GCM, CA_AES_KEY_256, s_key_ptr, 4);
```

Inside the pipe, only pointers to the ciphertext (8 bytes per item) travel: the encrypted buffer is allocated on the heap by the channel and transferred without copying; `recv` decrypts it, **copies** the plaintext into the task's buffer and clears both internal buffers. Before the service, this code (two code paths, the packet struct, who frees what) was manually written in ex46 and ex47.

69.1.5 Wire format GCM

Every encrypted message has the form (also visible as node `WF` in the architecture diagram):

Field	Size	Notes
`nonce`	12 byte	random from `ca_rng_fill()` for each encrypt

Field	Size	Notes
`orig_len`	2 byte	original plaintext length, big-endian
`ciphertext`	`ceil(plain_len, 16)` byte	always multiple of 16 (padding with zeros)
`GCM tag`	16 byte	authenticity — verified during decrypt

Fixed overhead = `AES_CH_OVERHEAD` = **30 byte** (nonce + orig_len + tag).

- **nonce**: 12 random bytes from `ca_rng_fill()` for each call to `ca_aes_channel_encrypt()`
- **orig_len**: original plaintext length in big-endian 2-byte format
- **ciphertext**: always multiple of 16 bytes (padding with zeros to plaintext before encryption)
- **tag**: 16 byte GCM — authenticity verification in `ca_aes_channel_decrypt()`; if incorrect, the function returns `-1` and the buffer is not delivered

The plaintext is always padded to a multiple of 16 before encryption (→ HW CRYP Notes section).

The GCM IV passed to the CRYP H7 peripheral is `[nonce:12][0x00000002:4]` — the initial counter required by the hardware for the GCM Init phase.

enc_len calculation

Plaintext	ceil(N,16)	enc_len = 30 + ceil(N,16)
“PING #0 from Alice” = 18 B	32	**62**
“PONG #0 from Bob” = 16 B	16	**46**
“PONG #99 from Bob” = 17 B	32	**62**
17 B	32	62

69.1.6 Keys Store — MPU protection

The AES key resides in a memory area protected by MPU via `keys_store`:

```
/* ca_os_configure() */
ca_keys_store_init(4);                                  /* max 4 keys */
s_key_ptr = ca_keys_store_allocate(32);                 /* 32 byte = AES-256 */

ca_keys_store_temp_enable_access(s_key_ptr, CA_KEYS_STORE_READ_WRITE);
ca_keys_store_copy(s_key_ptr, s_test_key, 32);          /* copy the key into the
    protected area */
ca_keys_store_lock_completely(s_key_ptr);               /* no CPU access */
```

During each encrypt/decrypt operation, `aes_channel_service` temporarily opens the region in read-only mode, copies the key to a local buffer on the stack, closes the protection, uses the key and clears it:

```
ca_keys_store_temp_enable_access(ks_handle, CA_KEYS_STORE_READ_ONLY);
ca_memcpy(key_buf, ks_handle, key_bytes);
ca_keys_store_lock_completely(ks_handle);
/* ... usa key_buf ... */
ca_memzero(key_buf, sizeof(key_buf));     /*     wipe the stack after use */
```

69.1.7 Heap management — mandatory wipe

The `ca_aes_channel_encrypt/decrypt` buffers are allocated on the heap and must be freed with `ca_aes_channel_free` (`ca_memory_free(p, NULL, true)`: zero before returning to the pool). With the pipe, the service does it in every branch: send failure, wrong tag, message too large, `deinit` with messages still in queue. The task only sees its own clear buffer.

69.1.8 Task_alice flow (100 iterations)

```
per ping = 0..99:
  1. formato plaintext: "PING #N from Alice"
  2. ca_secure_pipe_send(A)      cifra e accoda verso B   (timeout 2 s)
  3. ca_secure_pipe_recv(A)      decrypts the reply       (timeout 5 s)
  4. verifica: "PONG #N from Bob" ?
  5. sleep 1 s
stampa contatore OK/PING_COUNT
```

69.1.9 Task_bob flow (infinite loop)

```
loop:
  1. ca_secure_pipe_recv(B)      decrypts the ping        (timeout 10 s)
  2. extracts the ping number from the message (#N)
  3. formato pong: "PONG #N from Bob"
  4. ca_secure_pipe_send(B)      cifra e accoda verso A   (timeout 2 s)
```

69.1.10 Output UART

```
=== EX46: AES-GCM ping-pong (secure pipe) ===
Alice TX: 'PING #0 from Alice'
Bob RX: 'PING #0 from Alice'
Bob TX: 'PONG #0 from Bob'
Alice RX: 'PONG #0 from Bob' - OK
...
Alice: 100/100 ping OK
Alice: test done.
```

The encrypted length is no longer printed: it remains inside the pipe (see the table "Calcolo enc_len" above). The order between `Bob TX` and `Alice RX` can be inverted, because Bob prints after having appended.

69.1.11 Task and priority

Task	Priority	Stack	Function
`alice`	LEVEL4	STACK_MEDIUM	initiator — encrypt/send/receive/decrypt
`bob`	LEVEL4	STACK_MEDIUM	responder — receive/decrypt/encrypt/send
`hb`	LEVEL3	STACK_SMALL	toggle LED every 500 ms

Alice and Bob have the same priority — the scheduler alternates them for time-slicing. The queues block the task waiting without busy-wait.

69.1.12 Tamper detection GCM

If a byte of the ciphertext in transit is corrupted (attack or transmission error), `ca_aes_gcm_decrypt` recognizes the mismatch of the tag and returns `false`. The service frees the buffer with wipe and returns `-1` to the caller:

```
ciphertext corrotto     ca_aes_gcm_decrypt     tag mismatch     -1

                                              ca_memory_free(buf, NULL, true)
                                              (buffer plaintext azzerato)
```

The ping continues with the next one without crashing.

69.1.13 Implementation notes — CRYP H7 and partial blocks GCM

The **STM32H7 CRYP peripheral** has an asymmetry in GHASH calculation for partial blocks in GCM mode:

- In **encrypt**: GHASH covers the OUTPUT (ciphertext), including the keystream bytes produced by the padding bits — non-zero bytes.
- In **decrypt**: GHASH covers the INPUT (ciphertext), padded with zeros.

If a message is not a multiple of 16 bytes, the two GHASH values diverge → **tag mismatch**.

The `NPBLB` field of the CR register (bits [23:20]) should correct this behavior, but it has proven unreliable on the CRYP H753 in the caOS context.

Solution adopted: the plaintext is always padded to a multiple of 16 in `ca_aes_channel_encrypt` before being passed to the driver. The hardware CRYP sees only complete blocks — no partial blocks, no NPBLB. The original length is stored in the wire at offset 12 (2 byte big-endian) and used in decrypt to return exactly `orig_len` bytes.

Previous chapter: Example 45 — HTTP Server with CLI via HTTP See also: Encryption in caOS

Chapter 70

Keystore Service: makekey + Encrypted Ping-Pong

70.1 Example 47 — Keystore service: makekey + ping-pong encrypted

70.1.1 Introduction

Example of key lifecycle management through `keystore_service` of caOS, combined with encrypted communication of `aes_channel_service`.

The flow is divided into two distinct phases, both controlled by CLI:

1. **makekey** — generates a block of 16 random AES-256 keys, encrypts it with a key derived from two passwords, and writes it to flash (sector 6, bank1).
2. **start** — decrypts the block in flash using only `pwd2`, loads the chosen key into `keys_store` (MPU-protected area), and starts the encrypted ping-pong between two tasks.

Element	Value
Block encryption algorithm	AES-256-GCM
KDF	`SHA256^N(IV || pad16(pwd))[:16]` — N configurable evolutions
Number of keys in the block	16 (`NUM_KEYS`)
Flash sector	6 bank1 — `0x080C0000`
Ping-pong wire format	`[nonce:12B][orig_len:2B][ciphertext_padded][tag:16B]`
Fixed overhead	30 bytes (`AES_CH_OVERHEAD`)
Ping-pong iterations	20 (`PING_COUNT`)
HW result	**20/20 ping OK**

70.1.2 Hardware

Board: NUCLEO-H753ZI (STM32H753ZI, Cortex-M7 @ 480 MHz)

Pin	Function
PB7	Green LED — heartbeat (toggle every 500 ms)

70.1.3 os_config.h Configuration

```
#define EN_AES                  1   /* H7xx CRYP peripheral */
#define EN_RNG                  1   /* TRNG for IV and key generation */
#define EN_HASH                 1   /* SHA256 per KDF */
#define EN_FLASH                1   /* key block write */
#define EN_KEYS_STORE           1   /* MPU-protected area for keys */
#define EN_AES_CHANNEL_SERVICE  1   /*  r e d black  encrypted pipe */
#define EN_KEYSTORE_SERVICE     1   /* management of the key block in flash */
```

70.1.4 Architecture

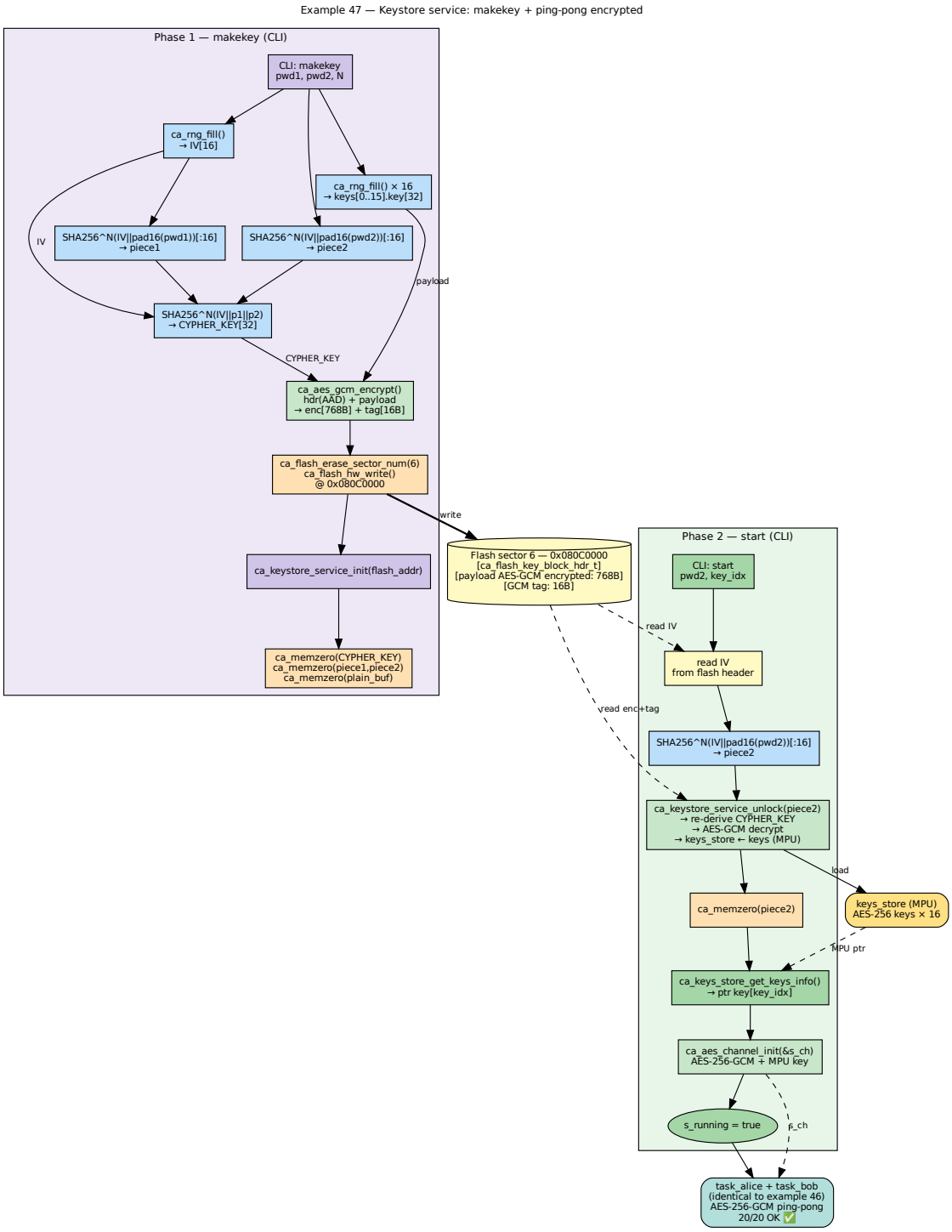


Figure 70.1: makekey + start + ping-pong flow — keystore service

70.1.5 Phase 1 — makekey

KDF (Key Derivation Function)

```
piece1 = SHA256^N(IV || pad16(pwd1))[:16]
piece2 = SHA256^N(IV || pad16(pwd2))[:16]

CYPHER_KEY = SHA256^N(IV || piece1 || piece2)   [32 byte]
```

- `IV` = 16 random bytes from `ca_rng_fill()`
- `pad16(pwd)` = password truncated/padded to 16 bytes with zeros
- `N` = number of hash iterations (1..16), configurable via CLI

Separating `pwd1` and `pwd2` allows distributing the two passwords to different entities (e.g. provisioner and operator): to decrypt the block only `pwd2` is needed, but to recreate it both are required.

Flash block structure

```
typedef struct {
    u32 size;           /* dimensione totale blocco in byte */
    u8  IV[16];         /* random IV     needed to derive piece2 in do_start()
    */
    u32 n_evolve;       /* number of SHA256 rounds */
    u8  p1[16];         /* piece1 salvato in chiaro nell'header */
    u16 n_key;          /* number of keys (16) */
    u8  gcm_nonce[12];  /* AES-GCM nonce for the payload */
} ca_flash_key_block_hdr_t;
```

The encrypted payload contains an array of `ca_flash_key_entry_t` (16×48 bytes $= 768$ bytes), each with `key_id`, `label`, `key[32]`, `start`, `stop`.

Operation sequence

```
1. RNG     IV[16]
2. derive_piece(IV, pwd1, N)     piece1[16]
3. derive_piece(IV, pwd2, N)     piece2[16]
4. RNG     keys[0..15].key[32]   (16 random AES-256 keys)
5. KDF     CYPHER_KEY[32]
6. RNG     gcm_nonce[12]
7. costruisce header (IV, n_evolve, p1, n_key, gcm_nonce)
8. AES-GCM encrypt(CYPHER_KEY, gcm_nonce, header_AAD, payload)     enc + tag
9. ca_flash_erase_sector_num(6)
10. ca_flash_hw_write(addr, [hdr | enc | tag], len)
11. ca_keystore_service_init(flash_addr)
```

70.1.6 Phase 2 — start

```
1. ca_keystore_service_init(flash_addr)   if not already active
2. legge IV dall'header in flash
3. derive_piece(IV, pwd2, n_evolve)     piece2
4. ca_keystore_service_unlock(piece2)
       re-derives CYPHER_KEY = SHA256^N(IV || p1_dal_hdr || piece2)
       AES-GCM decrypt payload
       loads the keys into keys_store (MPU-protected area)
5. ca_memzero(piece2)
6. ca_keys_store_get_keys_info()     reads the ptr to key[key_idx]
7. ca_secure_pipe_init(&s_pipe, CA_AES_MODE_GCM, CA_AES_KEY_256, key_ptr, 4)
8. s_running = true     sblocca task_alice e task_bob
```

The password `pwd1` is never needed after `makekey`. Only `pwd2` + IV in flash allows to reconstruct `piece2` and decrypt the block.

70.1.7 Encrypted ping-pong (same as example 46)

Once started (`s_running = true`), the tasks `alice` and `bob` execute the same encrypted ping-pong of example 46 through the secure pipe `ca_secure_pipe_t s_pipe`.

The pipe is recreated at every `start` (the key changes with index N), therefore it must also be closed: at the end of the loop Alice raises `s_done`, and **Bob** closes it — the only task that can still be inside `ca_secure_pipe_recv` — `deinit` must be called on a closed pipe. Until Bob closes it, `start` responds "already running".

Task	Priority	Stack	Role
`alice`	LEVEL4	STACK_MEDIUM	initiator — encrypt/send/receive/decrypt
`bob`	LEVEL4	STACK_MEDIUM	responder — receive/decrypt/encrypt/send
`hb`	LEVEL3	STACK_SMALL	heartbeat LED

For details on the GCM wire format and heap management see Example 46.

70.1.8 CLI — state machine

The CLI uses a state hook to collect parameters in sequence:

```
makekey:
  prompt: password1     password2     n_evoluzioni     esegue do_makekey()

start:
  prompt: password2     key index (0..15)     runs do_start()
```

Both reset the passwords from RAM with `ca_memzero` after use.

70.1.9 UART Output

```
makekey: password1:
> admin
makekey: password2:
> operator
makekey: n_evoluzioni (1..16):
> 3
[makekey] IV: 9a f3 12 ...
[piece1] SHA256 iter 1: ...
[piece1] piece[:16]: 5c 2b ...
[piece2] piece[:16]: a1 d7 ...
[makekey] CYPHER_KEY iter 3: 44 8e ...
[makekey] write 880 byte @ 0x080C0000
makekey: OK     852 bytes on flash @ 0x080C0000, 16 keys

start: password2:
> operator
start: key index (0..15):
> 0
[start] IV da flash: 9a f3 12 ...
```

```
[start/p2] piece[:16]: a1 d7 ...
start: OK     key[0] loaded, starting ping-pong
=== EX47: AES-GCM ping-pong (keystore, secure pipe) ===
Alice TX: 'PING #0 from Alice'
Bob RX: 'PING #0 from Alice'
Bob TX: 'PONG #0 from Bob'
Alice RX: 'PONG #0 from Bob'     OK
...
Alice: 20/20 ping OK
Alice: test completato.
```

70.1.10 Security — notes

Aspect	Solution
Keys in RAM	`keys_store` with MPU — CPU access blocked except during use
Password in RAM	`ca_memzero` immediately after derivation
Encryption key block	calculated on-the-fly, zeroed (`ca_memzero`) after encrypt
GCM nonce reuse	impossible — new random nonce for each session `makekey`
Block reconstruction	requires `pwd1` + `pwd2` + new `makekey`
Block decryption	requires only `pwd2` + `IV` saved in flash header

---

Previous chapter: Example 46 — AES channel service See also: Encryption in caOS

Chapter 71

IP Defrag Service: DHCP + UDP Server + Fragments

71.1 Example 48 — IP defrag service: DHCP + UDP server + fragment reassembly

71.1.1 Introduction

This example demonstrates the IP fragment reassembly service (`ip_defrag_service`) of caOS, according to RFC 791.

The system receives UDP datagrams that the sender transmits split into multiple IP fragments. The `defrag task` collects the fragments, reassembles them in order, and injects the reconstructed packet into the normal IP dispatch chain, making the reassembly completely transparent to the UDP application.

Element	Value
Parallel sessions	`IP_DEFRAG_MAX_SESSIONS` (default 4)
Session timeout	`IP_DEFRAG_TIMEOUT_MS` (default 15 000 ms)
Maximum payload	`IP_DEFRAG_MAX_PAYLOAD` (default 1480 bytes)
Allocated heap	`IP_DEFRAG_MAX_SESSIONS IP_DEFRAG_MAX_PAYLOAD`
Session identification	`(src_ip, dst_ip, proto, ip_id)`
Buffer pool release	immediately upon fragment arrival
HW Test	**NUCLEO-H753ZI — tested OK**

71.1.2 Hardware

Board: NUCLEO-H753ZI (STM32H753ZI, Cortex-M7 @ 480 MHz)

Pin	Function
PB0	Green LED — ETH link (on = link UP)
PB14	Red LED — heartbeat (toggle every 500 ms)
PHY LAN8742A	RMII, addr 0 — Ethernet connection

71.1.3 os_config.h Configuration

```
#define EN_ETH                1   /* driver Ethernet H7xx */
#define EN_NET                1   /* caOS network stack */
#define EN_NET_DHCP           1   /* parser DHCP in net_proto */
#define EN_ARP_SERVICE        1   /* cache ARP + GARP */
#define EN_DHCP_SERVICE       1   /* DHCP client DORA + renew */
#define EN_UDP_SERVER         1   /* multi-client UDP server */
#define EN_IP_DEFRAG          1   /* IP fragment reassembly */
#define EN_CRC                1   /* HW CRC (optional     used by the tests) */

/* Defrag parameters (default values) */
#define IP_DEFRAG_MAX_SESSIONS   4U
#define IP_DEFRAG_TIMEOUT_MS     15000U
#define IP_DEFRAG_MAX_PAYLOAD    1480U
```

71.1.4 Architecture

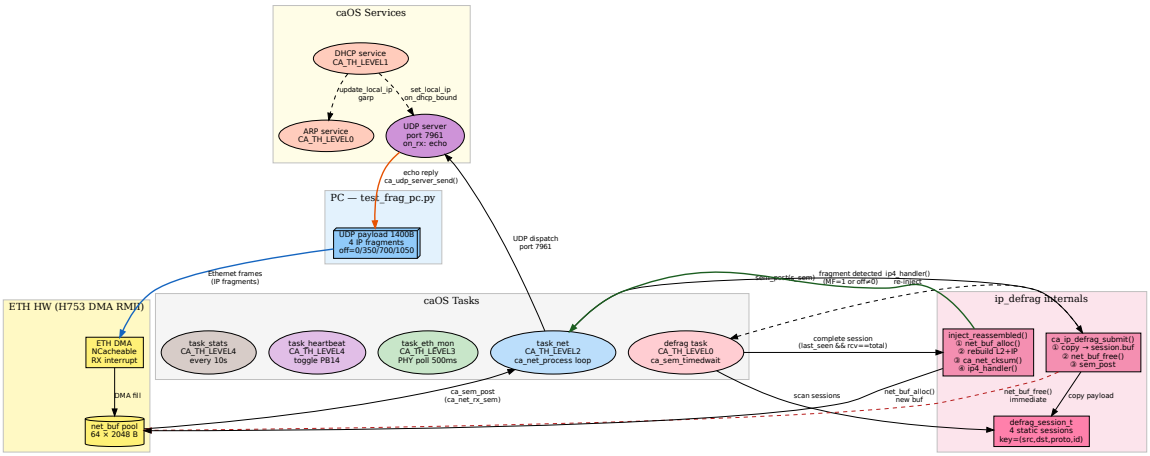


Figure 71.1: IP defrag flow — fragments, submit, defrag task, inject, UDP server

---

71.1.5 ip_defrag_service API

Initialization

```
bool ca_ip_defrag_init(u32 max_payload_size);
```

Allocate `IP_DEFRAG_MAX_SESSIONS` `max_payload_size` bytes from caOS heap. Initialize semaphore `s_sem` and mutex `s_mtx`. To be called in `ca_os_configure()`.

Parameter	Description
`max_payload_size`	Maximum IP payload size (without header). Typically 1480 (MTU 1500 20 IP hdr).
Return	`true` if allocation successful, `false` if heap exhausted or parameter out of range.

Task startup

```
void ca_ip_defrag_service_start(void);
```

Create task `defrag` with priority `CA_TH_LEVEL0`. To be called in `ca_os_main()`.

Submit fragment (internal use)

```
void ca_ip_defrag_submit(u16 buf_idx);
```

Automatically called by `ca_net_proto_ip4_handler()` when a fragmented packet is detected (`MF=1` or `fragment offset  0`).

71.1.6 Internal structure — session

```
typedef struct {
    bool     active;
    u32      src_ip, dst_ip;
    u8       proto;
    u16      ip_id;           /* identification key */
    u8       src_mac[6], dst_mac[6];
    u32      expires_ms;      /* ca_os_tick_ms() + IP_DEFRAG_TIMEOUT_MS */
    u32      total_payload;   /* set when the last fragment arrives */
    u32      received;        /* byte accumulati */
    bool     last_seen;       /* true when MF=0 received */
    u8      *buf;             /* slice of s_mem: reassembled payload */
} defrag_session_t;
```

The session is identified by the quadruple `(src_ip, dst_ip, proto, ip_id)`. Each session has a fixed-size `s_max_payload` pre-allocated buffer. The net_buf are freed **immediately** after copying, avoiding pool starvation.

71.1.7 Tasks and priorities

Task	Priority	Stack	Function
`net`	CA_TH_LEVEL2	STACK_SMALL	rx loop + dispatch
`defrag`	CA_TH_LEVEL0	STACK_SMALL	reassembly + inject
`eth_mon`	CA_TH_LEVEL3	STACK_SMALL	poll PHY every 500 ms
`stats`	CA_TH_LEVEL4	STACK_SMALL	statistics every 10 s
`heartbeat`	CA_TH_LEVEL4	STACK_SMALL	toggle LED PB14

71.1.8 PC Tests

```
sudo python3 examples/example_48/test_frag_pc.py --ip <ip_board> --port 7961
```

The script sends a 1400-byte UDP datagram (incremental fill `i % 256`) split into 4 IP fragments of 350 bytes each, with offset multiples of 8. It waits for the echo and verifies that `len` and `checksum` match those sent.

Expected output on board:

```
=== EX48: DHCP + UDP server + IP defrag ===
MAC  02:00:00:00:00:48
defrag: sessions=4 timeout=15000ms payload_max=1480B
DHCP: avvio DORA...
DHCP BOUND  IP=192.168.1.42  mask=255.255.255.0
            GW=192.168.1.1  DNS=192.168.1.1
DEFRAG init OK: sessions=4 max_payload=1480 heap=5920 B
DEFRAG frag: off=0   len=350 MF=1 rcv=350
DEFRAG frag: off=350 len=350 MF=1 rcv=700
DEFRAG frag: off=700 len=350 MF=1 rcv=1050
DEFRAG frag: off=1050 len=350 MF=0 rcv=1400 [LAST]
DEFRAG COMPLETE: id=0x1234 total=1400 B     inject
UDP[7961] cid=0  rx=1400 B  sum=0x0006DC00  from 192.168.1.10:12345
```

71.1.9 Implementation notes

Reconstructed IP checksum: `inject_reassembled` clears `ip4->cksum`, then calls `ca_htons(ca_net_cksum(...))` to recalculate the correct checksum. Without this fix the packet would be dropped by `ca_net_proto_ip4`.

Fragment order: the service supports out-of-order fragments — each fragment is written to `buf` at the correct `byte_off` position before `last_seen` becomes true.

Pool starvation: net_bufs are freed in `ca_ip_defrag_submit()`, not in the defrag task. This ensures that under burst of fragments the pool does not get exhausted before the task can process them.

Previous chapter: Example 47 — Keystore service See also: caOS network stack

Chapter 72

TFTP Client: GET and PUT over UDP (RFC 1350)

72.1 Example 49 — TFTP Client (RFC 1350)

72.1.1 Description

Example 49 demonstrates the **TFTP client** of caOS: a UDP-based file transfer service conforming to RFC 1350, with support for GET (download from server) and PUT (upload to server).

Main use case: **OTA firmware update** — the board downloads a binary image from a PC TFTP server and writes it to flash.

Example features: 1. **DHCP** — obtains IP address automatically 2. **ARP** — resolves server MAC address 3. **ICMP echo reply** — responds to ping requests 4. **CLI** — `tftp get <ip> <file>`, `tftp put <ip> <file> [size]`, `tftp info` 5. **TFTP client** — blocking GET and PUT with retry

Hardware: NUCLEO-H753ZI (STM32H753ZI), PHY LAN8742A RMII.

> **HW test status (H753, 2026-06-04):** GET and PUT verified on NUCLEO-H753ZI. The network stack has no regressions after introduction of the `ca_net_iface_t` abstraction (NCM): ARP/ICMP (ex33) and DHCP+TFTP (ex49) OK.

72.1.2 TFTP Protocol (RFC 1350)

TFTP uses UDP port 69 for the initial request; the server replies from its own TID (Transfer ID = ephemeral port). PDU types are:

Opcode	Name	Description
1	RRQ	Read Request (GET)
2	WRQ	Write Request (PUT)
3	DATA	Data block (max 512 bytes)
4	ACK	Acknowledgement (block number)
5	ERROR	Error (code + message)

GET flow:

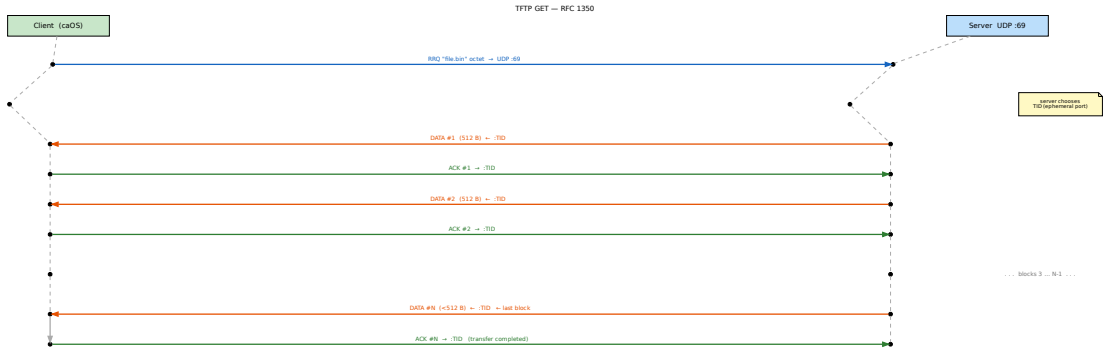


Figure 72.1: TFTP GET sequence — RFC 1350

PUT flow and PDU table:

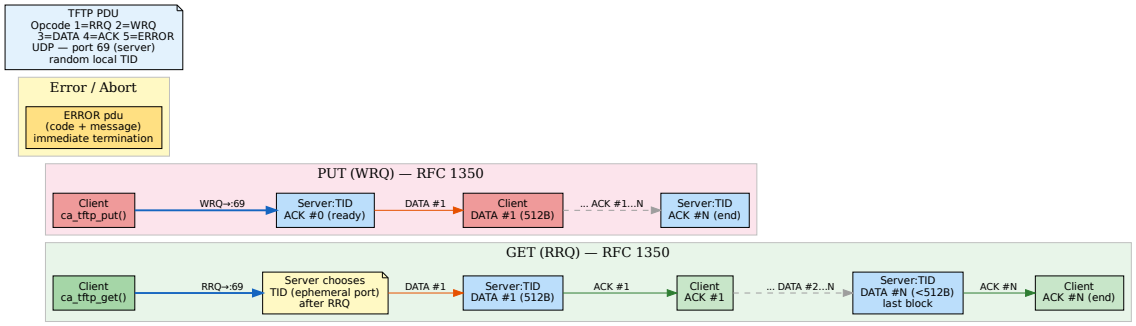


Figure 72.2: TFTP PUT sequence / PDU table — RFC 1350

72.1.3 Architecture `tftp_service`

```
Caller task (LEVEL3+)
  ca_tftp_get(cfg, on_block, arg)
            ca_net_resolve_mac()            ARP bloccante
            ca_net_proto_bind_udp()         porta TID locale
            tftp_send_pkt(port=69)          RRQ
            loop:
          ca_sem_timedwait(rx_sem)
          on_block(blk, data, len, arg)
          tftp_send_pkt(server_tid)       ACK

Net task (LEVEL2)
  tftp_rx_handler(buf_idx)
            verifica server_ip + TID
            ring_push(buf_idx)
            ca_sem_post(rx_sem)
```

72.1.4 API

ca_tftp_get

```
bool ca_tftp_get(const ca_tftp_cfg_t *cfg,
                 ca_tftp_rx_block_t on_block,
                 void *arg);
```

Download a file from the TFTP server. Call `on_block` for each received block (max 512 byte); the last block has `len < 512`. Returns `true` if the transfer is completed successfully.

Callback:

```
typedef bool (*ca_tftp_rx_block_t)(u16 block_num,
                                   const u8 *data, u16 len,
                                   void *arg);
```

Returning `false` from the abort callback aborts the transfer (the service sends an ERROR PDU to the server).

ca_tftp_put

```
bool ca_tftp_put(const ca_tftp_cfg_t *cfg,
                 const u8 *data, u32 total_len);
```

Load a memory buffer on the TFTP server as a remote file. Returns `true` if all blocks have been ACK-ed.

> **EOF (RFC 1350):** if `total_len` is a multiple of 512, the client sends a final DATA block of **0 bytes** to signal end of transfer. Without this block the server would remain waiting (timeout).

ca_tftp_service_init

```
void ca_tftp_service_init(ca_mac_t local_mac);
```

Initialize the service and register the CLI. Call from `ca_os_configure()`.

72.1.5 ca_tftp_cfg_t Configuration

```
typedef struct {
    ca_ipv4_t      server_ip;
    const char *filename;
    ca_mac_t       local_mac;
    ca_ipv4_t      local_ip;
    u16         local_port;         /* 0 = auto (TFTP_LOCAL_PORT_BASE) */
    u32         arp_timeout_ms;     /* 0 = 2000 ms */
    u32         block_timeout_ms;   /* 0 = TFTP_BLOCK_TIMEOUT_MS */
    u8          retry_n;            /* 0 = TFTP_RETRY_N */
} ca_tftp_cfg_t;
```

72.1.6 Parameters os_config.h

Define	Default	Description
`EN_TFTP_SERVICE`	1	Enable TFTP service
`TFTP_BLOCK_TIMEOUT_MS`	5000	Block timeout (ms)
`TFTP_RETRY_N`	3	Retries per block before fail
`EN_CLI_TFTP`	1	Enable CLI `tftp`

72.1.7 CLI Commands

Command	Description
`tftp get <ip> <file>`	Download `<file>` from `<ip>` server (PC→board download)
`tftp put <ip> <file> [size]`	Upload a pattern buffer of `size` bytes (default 1024, max 64 KB) as `<file>` to the server (board→PC upload)
`tftp info`	Show port, timeout, retries, block size

The command `put` allocates the buffer in RAM with `ca_memory_allocate(size)`, fills it with a known pattern (`0x00..0xFF` cyclic) and frees it with `ca_memory_free()` at the end.

72.1.8 PC Testing

The script `test_tftp_pc.py` runs **DHCP server + TFTP server** in one single command. With `--iface` it configures the interface (IP `192.168.10.1/24` + dnsmasq, pool `.100 .200`) as in example 35. Then TFTP is served. Requires root (port 69 + network setup) and automatic cleanup on exit.

```
# DHCP + TFTP server in one go (replace enp7s0 with your own interface)
sudo python3 examples/example_49/test_tftp_pc.py --iface enp7s0 \
    --file firmware.bin --size 16384

# TFTP only (board with an IP already assigned): legacy behaviour
sudo python3 examples/example_49/test_tftp_pc.py --bind 0.0.0.0 \
    --file firmware.bin --size 16384
```

The board, having obtained the IP via DHCP, prints `DHCP BOUND IP=192.168.10.1xx`.

GET (download from PC):

```
> tftp get 192.168.10.1 firmware.bin
TFTP GET firmware.bin from 192.168.10.1 ...
TFTP GET OK: 32 blocks, 16384 bytes
```

PUT (upload to PC — requires `--put` on server side, saves to `<file>_recv.bin`):

```
> tftp put 192.168.10.1 up.bin 4096
TFTP PUT up.bin (4096 B) to 192.168.10.1 ...
TFTP PUT OK: 4096 bytes
```

```
sudo python3 examples/example_49/test_tftp_pc.py --iface enp7s0 --put --file up.
    bin
# ... [OK] received 4096B     up.bin_recv.bin
```

72.1.9 Application flow (example 49)

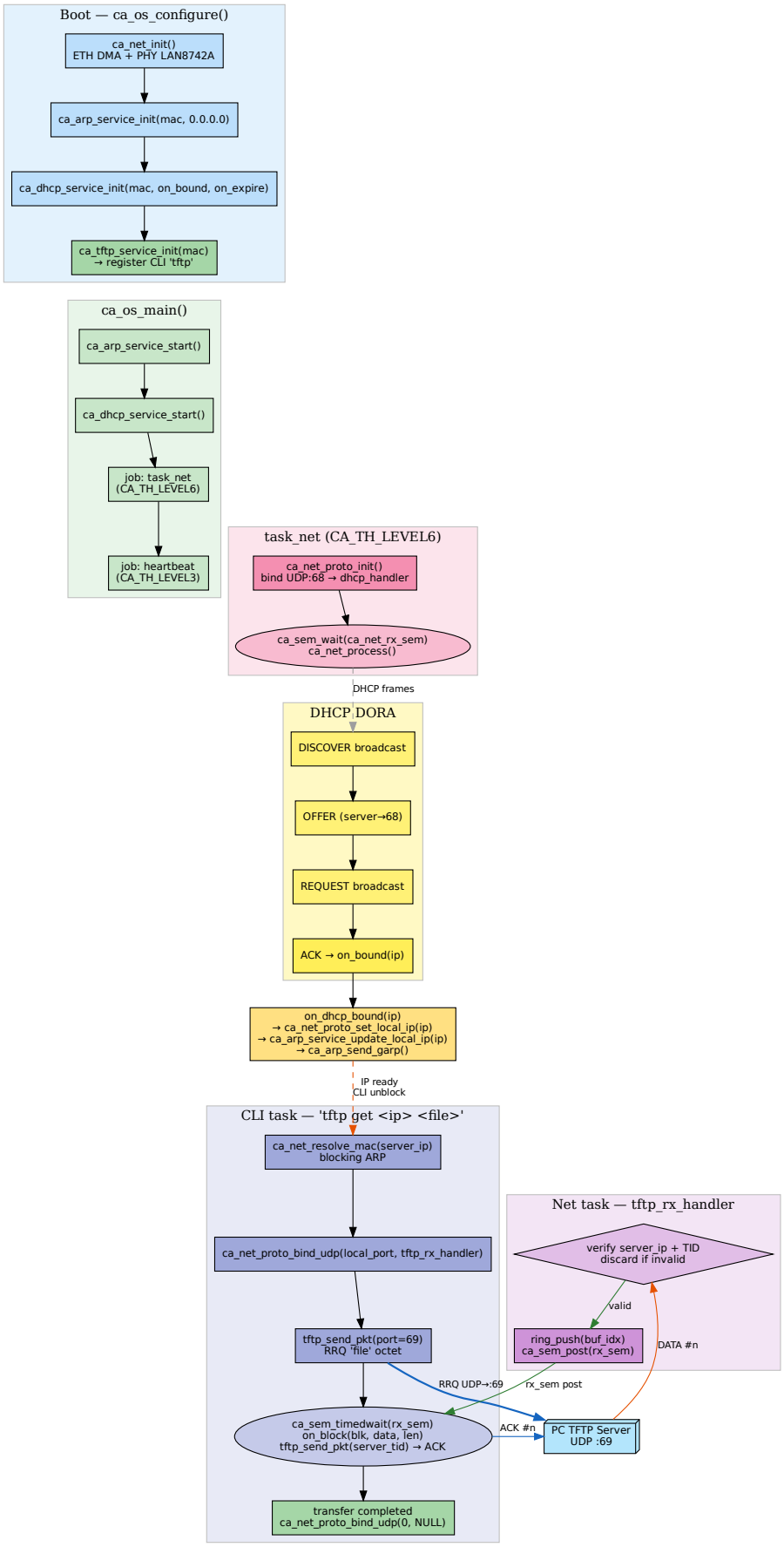


Figure 72.3: Complete DHCP → TFTP GET flow — example 49

```
ca_os_configure()
         ca_arp_service_init()
         ca_dhcp_service_init()
         ca_tftp_service_init(mac)        registra CLI

ca_os_main()
         ca_arp_service_start()
         ca_dhcp_service_start()
         jobs: net / eth_mon / heartbeat

task_net:
  ca_net_proto_init()
  ca_net_proto_bind_udp(68, dhcp_handler)
  ca_dhcp_service_net_ready()
  loop: ca_net_process()

CLI (after DHCP):
  tftp get <ip> <file>
           ca_tftp_get()     blocking in the CLI task
```

72.1.10 File Structure

File	Description
`caos/net/inc/tftp_service.h`	Public API
`caos/net/src/tftp_service.c`	Implementation + CLI
`examples/example_49/ca_os_ex49.c`	DHCP + TFTP CLI Example
`examples/example_49/test_tftp_pc.py`	TFTP server Python for testing

Chapter 73

USB Vendor Class: Multi-EP Bulk Loopback

73.1 Example 50 — USB Vendor-Class Bulk Loopback

73.1.1 Description

Example 50 demonstrates the **bare-metal USB driver** of caOS (DWC2 core) used directly, without a class stack: the board enumerates as a **vendor-class** device and exposes N pairs of bulk IN/OUT endpoints that perform **loopback** (every byte received on OUT is sent back on the corresponding IN).

It is the test bench for the low-level driver and the **per-EP queue** mechanism: each bulk endpoint has a dedicated loopback task, so all EPs run in parallel (composite device).

Example features: 1. **Enumeration** as a vendor device (VID `0xCA01`, PID `0x0001`) 2. **EP0 control**: `GET_DESCRIPTOR`, `SET_ADDRESS`, `SET_CONFIGURATION`, `CLEAR/SET_FEATURE(ENDPOINT_HALT)`, `GET_STATUS` 3. **EP1..EPn bulk IN/OUT**: loopback, one task per EP 4. **CLI** `usb`: status, halt/resume, ep, open/close/stall/unstall, errs

Supported hardware:

Board	OTG	`USB_NUM`	MPS	Data EP
NUCLEO-H753ZI	FS CN14 (or HS ULPI)	1 (or 0)	64 (512 HS)	6
NUCLEO-U575ZI-Q	Internal FS	0	64	5
STM32F4VE (black board)	Internal FS	0	64	3

Measured HW throughput: **H753 6 EP @ 0.491 MB/s**, **U575 5 EP @ 0.466 MB/s**, **F407 3 EP @ 0.548 MB/s** (aggregated across parallel EPs).

> The low-level USB driver (API, DMA vs slave mode, per-EP queues) is documented in `S02_11_00_usb_driver.md`. This chapter only describes the application usage in example 50.

73.1.2 Example Architecture

```
ISR DWC2 (driver)
  on_reset       s_dev_state=ATTACHED, notify task_usb
  on_setup       copia SETUP 8B, notify task_usb (NOTIFY_SETUP)
  on_rx(epN)     notify task_loop[N] (NOTIFY_BULK_RX)
  on_tx_done     applica SET_ADDRESS deferred su EP0
```

```
6
7  task_usb (LEVEL6)     controllo
8    handle_setup()       EP0: descriptor, address, config, feature, status
9    handle_status()      logs the state change
10
11 task_loop[1..N] (LEVEL5)     one per bulk EP
12   loop_drain(ep): ca_usb_ep_read(OUT)     ca_usb_ep_write(IN)
13
14 task_hb (LEVEL4)     heartbeat LED 500 ms
```

Each bulk EP has a **per-EP driver queue + dedicated task** → no contention, fair round-robin between endpoints with equal priority.

73.1.3 Descriptors

Minimal vendor-specific device:

- **Device**: class/subclass/protocol = `0xFF`, `bMaxPacketSize0 = 64`, 1 configuration.
- **Config**: 1 vendor interface with `NUM_DATA_EP * 2` bulk endpoints (`wTotalLength = 9 + 9 + NUM_DATA_EP*2*7`).
- **String**: LANGID US-EN, “caOS”, “USB Vendor Test”.

`NUM_DATA_EP` = 6 (H7/F4) or 5 (U5, for `USB_EP_MAX=6`). The cap stems from the `XFRSIZ` limit of EP0: the config descriptor (18 + N*14 bytes) must fit in 127 B to avoid requiring data-stage chunking across all families.

73.1.4 SETUP handler (EP0)

`handle_setup()` runs in task context (woken by `on_setup`) and implements the standard subset required for enumeration:

bmRequest	bRequest	Action
`0x80`	`GET_DESCRIPTOR`	send device/config/string
`0x00`	`SET_ADDRESS`	`ca_usb_set_address` immediately + ZLP status
`0x00`	`SET_CONFIGURATION`	open EP1..N IN+OUT, ZLP status
`0x02/0x82`	`CLEAR/SET_FEATURE(HALT)`	`ep_clear_stall`/`ep_stall` + ZLP
`0x82`	`GET_STATUS` (endpoint)	return HALT bit from `ep_is_stalled`
`0x80/0x81`	`GET_STATUS` (dev/iface)	return `0x0000` (bus-powered)
other	—	STALL EP0 + log in debug ring

`CLEAR/SET_FEATURE(ENDPOINT_HALT)` and `GET_STATUS` are **necessary**: pyusb and the Linux kernel issue them at `set_configuration`. Without a handler, the device STALLs and the host reacts with a bus reset → infinite enumeration loop (bug encountered on U575).

73.1.5 CLI `usb`

```
usb status                  # state, speed, rx/tx/err counters
usb ep                      # list EPs and their state (open/closed/STALL)
usb halt | resume           # soft disconnect / reconnection (re-enumerates)
usb open|close   <ep>       # apre/chiude EP a runtime (Patch 1)
usb stall|unstall <ep>      # forces/clears STALL on an EP (Patch 1)
usb errs                    # ring of the last 16 SETUPs that went to STALL
usb dbg | dbg reset         # H753 only: ISR latencies via the DWT cycle counter
```

`usb open/close/stall/unstall` exercise the API endpoint at runtime while the device is configured — useful for testing flow-control and host-side STALL recovery.

`usb errs` decodes the ring of unhandled SETUPs (`bmRequestType / bRequest / wValue / wIndex / wLength`). These commands are the debug template to be replicated in the next USB examples (51–56).

73.1.6 Parameters (per-MCU, in `ca_os_ex50.c`)

Define	H753	U575	F407
`USB_NUM`	1	0	0
`EP_BULK_MPS`	64	64	64
`NUM_DATA_EP`	6	5	3
LED heartbeat	PB7	PC7	PA6

F407 (STM32F4VE black board): `OTG_FS` has only 4 total endpoints (`USB_EP_MAX=4`), so data EP = EP1..3 = 3 bulk pairs. The LED is **PA6** (D2, user on the black board) — not PD12 as on NUCLEO F4 boards. Measured throughput: **0.548 MB/s** aggregated over 3 EPs. The EP3 ring queue was increased from 16 to 64 slots to eliminate drops that caused RX sequence misalignment during loopback.

`os_config.h` requirements: `EN_USB = 1`; all standard classes at 0 (`EN_USB_CDC`, `EN_USB_MSC`, `EN_USB_NCM`, `EN_USB_CUSTOM`): the example builds its own vendor-class descriptors directly in the source code. The USB stack is the native caOS one: TinyUSB has been removed from the project.

73.1.7 Test from PC

Verify enumeration:

```
lsusb                       # VID=0xCA01 PID=0x0001  "USB Vendor Test"
```

Perform a loopback on all EPs (the number is auto-detected from the config descriptor — H753=6, U575=5):

```
sudo python3 examples/example_50/test_usb_loopback.py --ep 6
```

Expected output (H753, 6 EP):

```
EP pairs detected from config descriptor: 6
loopback EP1..EP6  500 frames each ...
OK: 3000/3000 frames  @ 0.491 MB/s aggregate
```

The `count_device_ep_pairs()` script reads the config descriptor and auto-detects the number of EPs, so there are no magic numbers on the test side.

73.1.8 File structure

File	Description
caos/drivers/inc/usb.h	USB driver API (see S02_11)
caos/drivers/src/usb.c	generic wrapper
hw/h7xx/drivers/usb_h7xx.c	DWC2 DMA (H753)
hw/u5xx/drivers/usb_u5xx.c	DWC2 slave + per-EP code (U575)
hw/f4xx/drivers/usb_f4xx.c	DWC2 slave FIFO (F407/F429/F446, no DMA)
examples/example_50/ca_os_ex50.c	vendor loopback example + CLI
examples/example_50/test_usb_loopback.py	PC loopback test (auto-EP)

73.1.9 F407 Notes — slave mode driver

The DWC2 OTG_FS of the STM32F407 (RM0090 §34.3.2) **does not have DMA**: the DMAEN bit in GAHBCFG is *hardwired* to 0. The usb_f4xx.c driver is therefore written in **slave mode** (CPU push/pop on FIFOs), identical in structure to usb_u5xx.c. Hardware differences compared to the U575:

Aspect	F407 OTG_FS	U575 OTG_FS
DMA	no (DMAEN=0)	no (slave Phase 1b)
Total EPs	4 (EP0..3)	6 (EP0..5)
Bulk data EPs	3 (EP1..3)	5 (EP1..5)
Total FIFO	320 words (1280 B)	320 words (1280 B)
VBUS sensing	GCCFG.NOVBUSSENS	GOTGCTL.BVALOEN
Clock	PLLQ = 48 MHz	HSI48
GPIO	PA11=DM, PA12=DP AF10	PA11=DM, PA12=DP AF10
IRQ	OTG_FS_IRQn (single)	OTG_FS_IRQn (single)

GCCFG.NOVBUSSENS is necessary on F407 because the DWC2 GUID is < 0x2000 (F2/F4 generation): it does not expose GOTGCTL.BVALOEN/BVALOVAL used by the U575 to force VBUS sensing on the software side.

Chapter 74

USB Management Stack: Custom Class

74.1 Example 51 — USB Management Stack: custom class

74.1.1 Description

Example 51 demonstrates the **high-level USB stack** of caOS (`caos/usb/usb_mgmt`) using the `ca_usb_custom` service as reference implementation of `ca_usb_class_ops_t`.

Unlike example 50 — where descriptors and EP0 were entirely in the application — here the app **does not touch** EP0 nor the descriptors: the management builds composite device/config descriptors and dispatches standard SETUP; the custom service provides only the interface descriptor, opens its EP on `SET_CONFIGURATION` and does fan-out of `on_rx`/`on_tx_done`.

Three calls are enough to bring up the device:

```
ca_usb_dev_mng_init(CA_USB_SPEED_FS, USB_NUM);          /* registers the mgmt driver callbacks */
ca_usb_dev_mng_add(&s_custom.header);   /* registers the custom class */
ca_usb_dev_mng_start();                /* build descriptors + connect enumerates */
```

Features: 1. **Enumeration** as device vendor (VID `0xCA01`, PID `0x0001`) — descriptors generated by mgmt, not hard-coded in app 2. **1 bulk IN/OUT EP** (EP1) with loopback in task context 3. **CLI** `usb`: status, start/stop, add/remove (runtime lifecycle)

Supported hardware: NUCLEO-H753ZI (`USB_NUM=1`), NUCLEO-U575ZI-Q (`USB_NUM=0`), NUCLEO-F407ZG (`USB_NUM=0`), all OTG_FS, MPS 64.

> The `ca_usb_dev_mng` stack is documented in the `doc/usb_stack_workplan.md` plan (decisions D1–D20). The low-level driver is in `S02_11_00_usb_driver.md`.

74.1.2 Hardware

All boards operate in OTG_FS. No external wiring: connect the device USB connector (OTG_FS, not ST-LINK).

Board	MCU	`USB_NUM`	USB Connector	Heartbeat LEI
NUCLEO-H753ZI	STM32H753ZI	1	OTG_FS (CN14, “USB user”)	PB7 (LD2 gre
NUCLEO-U575ZI-Q	STM32U575ZI	0	Internal OTG_FS	PC7 (LD1 gre
STM32F4VE (black board)	STM32F407VET6	0	Internal OTG_FS (PA11/PA12)	PA6 (D2)

F407 (Chinese black board): the LED is PA6; user USB connector on-board micro-USB connected to PA11/PA12. Tested HW: 10000/10000 EP1 loopback OK.

74.1.3 os_config.h Configuration

```
#define EN_USB        1     /* driver USB DWC2 + stack mgmt */
#define EN_USB_CUSTOM 1     /* enables the custom class (reference class_ops) */
/* Test isolato: le altre classi a 0 */
#define EN_USB_CDC    0
#define EN_USB_MSC    0
#define EN_USB_NCM    0
```

`ca_usb_custom` is the **reference implementation** of `ca_usb_class_ops_t`: the minimal model to copy when writing a new USB class.

74.1.4 Architecture

```
ca_usb_dev_mng_init(CA_USB_SPEED_FS, USB_NUM)
         ca_usb_init(USB_NUM, &s_mgmt_cb)       mgmt registra i SUOI callback
    driver

ISR DWC2     callback mgmt     fan-out via ops ai servizi attivi:
  mng_on_setup     dispatch_standard (EP0: descriptor/address/config/...)
                  dispatch_class    (SETUP class-level     ops->on_setup)
  mng_on_rx        ops->on_rx          custom_on_rx     cfg->on_rx (app)
  mng_on_tx_done     ops->on_tx_done     custom_on_tx_done     cfg->on_tx_done (
    app)

ca_usb_custom_ops (in caos/usb/usb_custom.c):
  build_iface_desc     interface vendor 0xFF + 2 EP bulk (23B)
  on_configure         ca_usb_ep_open EP1 IN+OUT
  on_rx / on_tx_done     fan-out a cfg->on_rx / cfg->on_tx_done

App (example_51):
  custom_rx(ep,len)      da ISR     notify task_loop
  task_loop: loop_drain()     ca_usb_ep_read(OUT)     ca_usb_ep_write(IN)
```

The management does not switch on `type`: it iterates the registered services and invokes their `ops`. Adding a class (CDC/MSC/NCM) means only registering a new `cfg` with its `ops` table — the core remains unchanged.

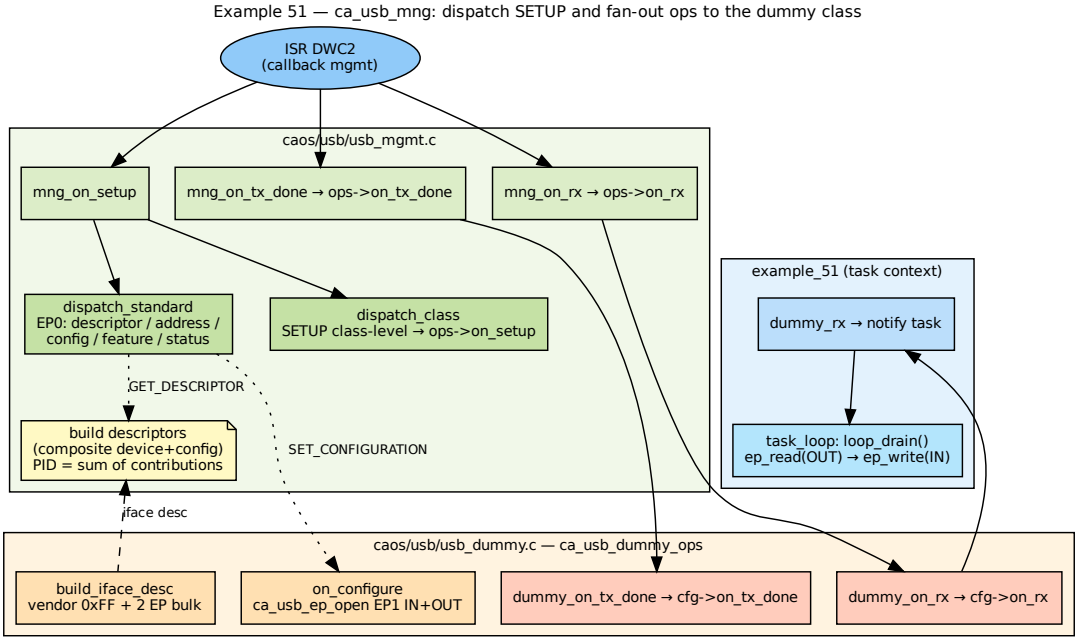


Figure 74.1: Dispatch SETUP and fan-out ops (ex51)

Source: `doc/diagrams/ex51_mgmt_dispatch.dot` *(regenerated from* `make diagrams`*).*

Task	Priority	Function
`lp1`	LEVEL5	loopback bulk: woken by `custom_rx`, `ep_read(OUT)` → `ep_write(IN)`
`usb`	LEVEL6	init stack + polling mount/unmount (print state changes)
`hb`	LEVEL4	toggle LED heartbeat every 500 ms

The task `lp1` is created **before** `usb` so `s_loop_task_id` is valid when the first `on_rx` arrive (the ISR notifications point to an already existing task).

74.1.5 Class config (D20: persistent lifetime)

```
static const ca_usb_custom_cfg_t s_custom = {
    .header     = { CA_USB_TYPE_CUSTOM, &ca_usb_custom_ops },
    .ep_data    = 1,                /* EP1 IN + EP1 OUT */
    .mps        = 64,
    .on_rx      = custom_rx,
    .on_tx_done = custom_tx_done,
};
```

The cfg **must** be `static` (ideally `static const`): the management preserves the pointer without copying it. The first field `header` (tag + ops) must remain at the head of the struct (D19).

`PID = USB_PID_BASE_CUSTOM + ep_data = 0 + 1 = 0x0001` — the management sums the PID contributions of all active services (composite PID, D14).

74.1.6 CLI `usb`

```
usb status                    # mount state + rx/tx counters
usb start | stop              # connect / disconnect (the host re-enumerates)
usb add | remove              # registra/rimuove la classe custom a runtime
```

The `remove stop add start` loop allows to check the re-attach: `ca_usb_dev_mng_start` reconstructs the descriptors based on the currently registered services.

74.1.7 Mount/unmount state

The custom cfg exposes `on_mounted`/`on_unmounted` (fan-out from ops via `ca_usb_dev_mng`). The example uses them to report state changes on the console:

- **mount** → `on_mounted` on `ENUMDNE` (host has enumerated the device)
- **unmount** → `on_unmounted` on `USBSUSP` (cable unplugged / host suspended)

The task polls a volatile state and prints `host enumerated (mounted)` / `host disconnected (unmounted)` on changes; `usb status` shows the current state.

74.1.8 Parameters

Define	H753	U575	F407
`USB_NUM`	1	0	0
`EP_BULK_MPS`	64	64	64
`CUSTOM_EP_DATA`	1	1	1
LED heartbeat	PB7	PC7	PA6

`os_config.h` requirements: `EN_USB = 1`, `EN_USB_CUSTOM = 1`.

74.1.9 Expected UART output

```
[USB] mng init...
[USB] D+ pull-up on     plug cable (custom vendor loopback EP1)
[USB] host enumerated device (mounted)
```

At cable disconnection (or host suspension):

```
[USB] host disconnected (unmounted)
```

74.1.10 Test from PC

```
lsusb                                       # VID=0xCA01 PID=0x0001
sudo python3 examples/example_51/test_usb_custom.py
```

The script prints the descriptors generated by mgmt and then performs loopback on EP1:

```
Descriptors (generati da ca_usb_dev_mng):
  idVendor=0xca01 idProduct=0x0001
  bInterfaceClass=0xff bNumEndpoints=2
    EP 0x81 IN  type=2 mps=64
    EP 0x01 OUT type=2 mps=64
Loopback EP1 x 10000 packets x 64 bytes ...
EP1: 10000/10000 OK  0 err
```

74.1.11 File Structure

File	Description
`caos/usb/inc/usb_mgmt.h`	API management (init/add/remove/start/stop)
`caos/usb/src/usb_mgmt.c`	composite descriptors + dispatch SETUP
`caos/usb/inc/usb_custom.h`	cfg `ca_usb_custom_cfg_t`
`caos/usb/src/usb_custom.c`	`ca_usb_custom_ops` (reference class_ops)
`hw/f4xx/drivers/usb_f4xx.c`	DWC2 slave FIFO (F407/F429/F446)
`examples/example_51/ca_os_ex51.c`	mgmt example + custom + loopback
`examples/example_51/test_usb_custom.py`	loopback test PC

Chapter 75

USB CDC-ACM: Virtual Serial Port

75.1 Example 52 — USB CDC-ACM: virtual serial port

75.1.1 Description

Example 52 demonstrates the **CDC-ACM** class (`ca_usb_cdc`): a virtual USB serial port. The device enumerates with IAD + comm-interface + data-interface; the PC mounts it as `/dev/ttyACM0` (Linux) or COM port (Windows).

It is the first "real" USB class above `ca_usb_dev_mng`: unlike the custom (vendor), it uses **SETUP class-level** (`SET/GET_LINE_CODING`, `SET_CONTROL_LINE_STATE`), **mixed endpoints** (interrupt NOTIF + bulk DATA) and an **IAD** to group the two interfaces.

Behavior: 1. on terminal opening (DTR asserted) sends a welcome banner 2. does **echo** of every received byte (serial loopback)

```
ca_usb_dev_mng_init(CA_USB_SPEED_FS, USB_NUM);
ca_usb_dev_mng_add(&s_cdc.header);
ca_usb_dev_mng_start();
```

Hardware: H753 (`USB_NUM=1`), U575 (`USB_NUM=0`), F407 (`USB_NUM=0`).

> The `ca_usb_cdc` service and `ca_usb_dev_mng` stack are described in `doc/usb_stack_workplan.md` and `S02_11_00_usb_driver.md` (low-level drivers).

75.1.2 Hardware

Board	MCU	`USB_NUM`	USB Connector	LED heartbeat
NUCLEO-H753ZI	STM32H753ZI	1	OTG_FS (CN14, "USB user")	PB7 (LD2 gree
NUCLEO-U575ZI-Q	STM32U575ZI	0	Internal OTG_FS	PC7 (LD1 gree
STM32F4VE (black board)	STM32F407VET6	0	Internal OTG_FS (PA11/PA12)	PA6 (D2)

No external wiring: connect the device USB connector (OTG_FS, not ST-LINK).

> **F407**: HW tested (black board STM32F4VE, OTG_FS slave mode). Echo 100/100 OK on `/dev/ttyACM0`. The LED is PA6 instead of PD12 (black board, not NUCLEO F4).

75.1.3 os_config.h Configuration

```
#define EN_USB        1     /* driver USB DWC2 + stack mgmt */
#define EN_USB_CDC    1     /* enables the CDC-ACM class */
/* Test isolato: le altre classi a 0 */
#define EN_USB_MSC    0
#define EN_USB_NCM    0
#define EN_USB_CUSTOM 0
```

Optional overrides for endpoints (the defaults do not collide with other classes):

```
#define USB_CDC_EP_NOTIF  2U     /* EP2: interrupt IN, notifiche (8 byte) */
#define USB_CDC_EP_DATA   3U     /* EP3: bulk IN+OUT, serial data */
/* PID = USB_PID_BASE_CDC + USB_CDC_EP_DATA = 0x0010 + 3 = 0x0013 */
```

75.1.4 CDC ACM Structure

The descriptor (66 bytes, TinyUSB `TUD_CDC_DESCRIPTOR` layout) is generated by `cdc_build_iface_desc` and contains:

Block	Bytes	Notes
IAD	8	groups the 2 interfaces (CDC class)
Comm interface	9	class 0x02, subclass ACM 0x02
Functional Header	5	bcdCDC 1.20
Functional Call Mgmt	5	data iface = comm+1
Functional ACM	4	bmCapabilities 0x06 (line coding + line state)
Functional Union	5	master=comm, slave=data
EP NOTIF	7	interrupt IN, 8B, bInterval 16
Data interface	9	class 0x0A (CDC-DATA)
EP OUT + EP IN	14	bulk, MPS

Endpoint numbers (override in `os_config.h`): `USB_CDC_EP_NOTIF=2` (interrupt IN), `USB_CDC_EP_DATA =3` (bulk IN+OUT). PID = `USB_PID_BASE_CDC + USB_CDC_EP_DATA` = `0x0010 + 3 = 0x0013`.

75.1.5 SETUP class-level

`cdc_on_setup` (invoked by `mng_dispatch_class`) handles:

bmRequestType	bRequest	Action
`0x21`	`SET_LINE_CODING` (0x20)	data-stage OUT 7B → applies baud/parity/stop
`0xA1`	`GET_LINE_CODING` (0x21)	returns current line coding (7B)
`0x21`	`SET_CONTROL_LINE_STATE` (0x22)	wValue bit0=DTR bit1=RTS → `on_line`

> `SET_LINE_CODING` is a **control OUT**: SETUP → data OUT (7B) → status IN. The service does not write the status in SETUP; it receives the 7 bytes via `on_rx(0)` and only then emits the ZLP IN (proper data-stage EP0 handling).

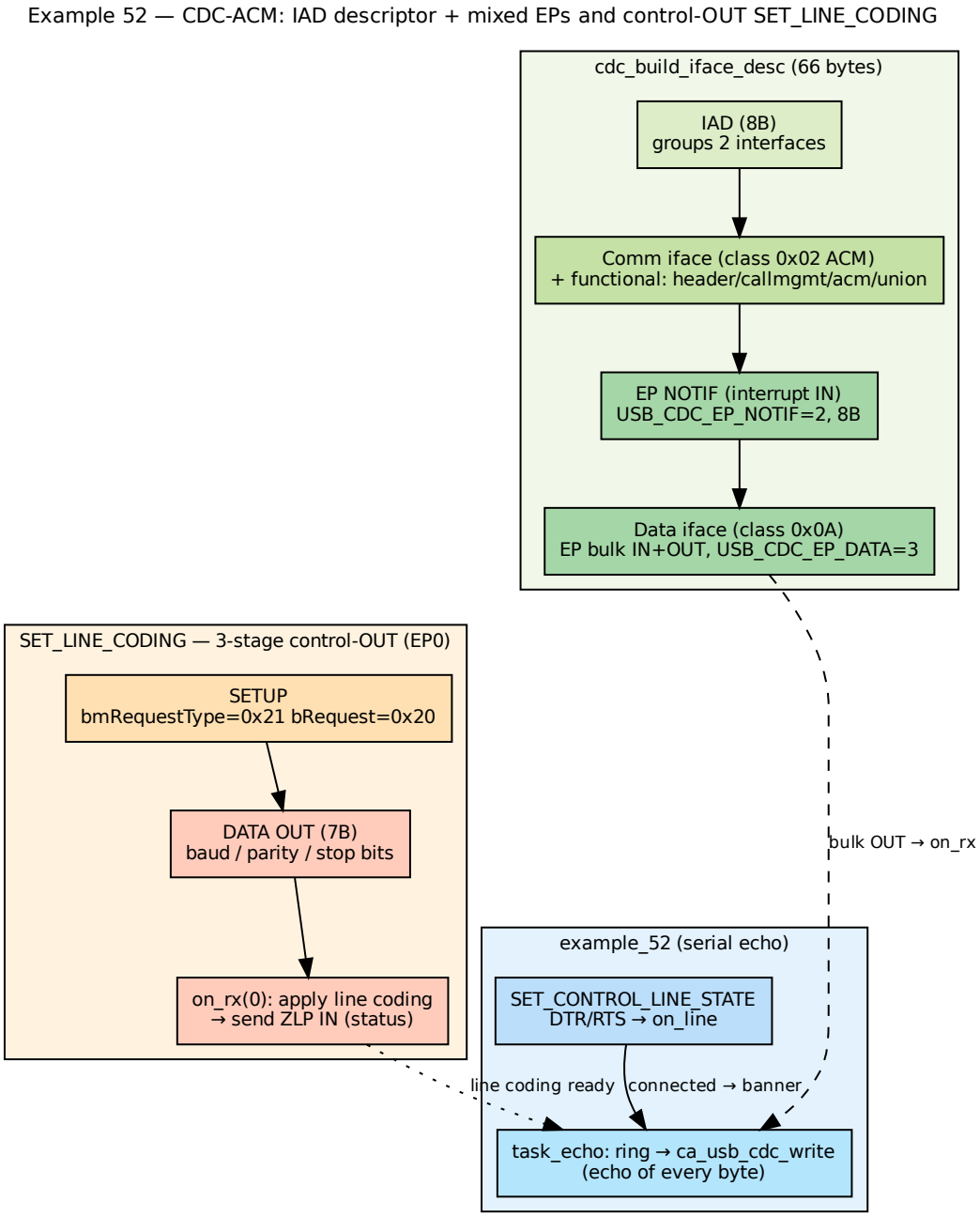


Figure 75.1: CDC descriptor and control-OUT SET_LINE_CODING (ex52)

Source: `doc/diagrams/ex52_cdc_setup.dot` *(regenerated from* `make diagrams`*).*

75.1.6 Architecture

Task	Priority	Function
`echo`	LEVEL5	drains the RX ring (filled by ISR) → echoes bytes to host
`usb`	LEVEL6	init stack + polling DTR status (prints terminal open/close)
`hb`	LEVEL4	toggle LED heartbeat every 500 ms

The CDC callbacks (`on_rx`, `on_line`) are called **from ISR**: `cdc_rx` accumulates bytes in a ring and wakes up `task_echo` with `ca_notify_send_isr`. Echo never occurs in ISR — only in the task, never blocking the interrupt.

75.1.7 Application API

```
bool ca_usb_cdc_write(const u8 *buf, u16 len);   /* send to the host */
bool ca_usb_cdc_connected(void);                 /* true if DTR asserted */
const ca_usb_cdc_line_t *ca_usb_cdc_get_line(void);  /* current baud/parity */
```

The callbacks in cfg (`on_rx`, `on_line`) are called **from ISR**: the example accumulates received bytes in a ring and echoes them from a dedicated task (`task_echo`), never blocking in ISR.

75.1.8 Parameters

Define	H753	U575	F407
`USB_NUM`	1	0	0
`CDC_MPS`	64	64	64
LED heartbeat	PB7	PC7	PA6

`os_config.h` requirements: `EN_USB=1`, `EN_USB_CDC=1`.

75.1.9 Expected UART output

```
[USB] CDC init...
[USB] D+ pull-up on     waiting host (plug cable)
```

At serial terminal open and close from PC:

```
[USB] host opened CDC terminal (DTR on)
[USB] host closed CDC terminal (DTR off)
```

75.1.10 Test from PC

Verify enumeration (must show IAD + CDC Communications + CDC Data):

```
lsusb -v -d ca01: | grep -A2 -iE "interface association|CDC"
```

Interactive echo:

```
sudo screen /dev/ttyACM0 115200      # type: it is echoed back
```

Automatic test (requires pyserial):

```
python3 examples/example_52/test_usb_cdc.py --lines 100
```

Expected output:

```
Porta: /dev/ttyACM0
Echo: 100/100 OK  0 err
```

75.1.11 File Structure

File	Description
`caos/usb/inc/usb_cdc.h`	cfg `ca_usb_cdc_cfg_t` + API write/connected/line
`caos/usb/src/usb_cdc.c`	`ca_usb_cdc_ops` (descriptors + SETUP class)
`hw/f4xx/drivers/usb_f4xx.c`	DWC2 slave FIFO (F407/F429/F446)
`examples/example_52/ca_os_ex52.c`	CDC echo example
`examples/example_52/test_usb_cdc.py`	PC echo test (pyserial)

Chapter 76

USB MSC: Mass Storage Device (Ramdisk)

76.1 Example 53 — USB MSC: Mass Storage Device (ramdisk)

76.1.1 Introduction

This example demonstrates the **MSC** (Mass Storage Class) of caOS: once the board is connected to the PC, the device appears as a **removable disk** — exactly like a USB flash drive. The host operating system recognizes it, assigns it a drive letter (`/dev/sdX` on Linux, a letter on Windows), and can create a filesystem on it.

An important point to understand: **caOS does not have a filesystem**. The device exposes only a **block device** — an array of 512-byte blocks, addressed by LBA (Logical Block Address). The filesystem (FAT, exFAT, ext4...) is created and managed **by the host**, not the board. The board simply acts as "block memory": it reads and writes 512-byte blocks upon request from the host. This is the standard model of all real USB flash drives.

The storage backend in this example is a **ramdisk on heap**: 128 KB allocated with `ca_memory_allocate`. The three calls that bring up the device are the same as in every example above the `ca_usb_dev_mng` stack:

```
1 ca_usb_dev_mng_init(CA_USB_SPEED_FS, USB_NUM);        /* registers the
    management driver callbacks */
2 ca_usb_dev_mng_add(&s_msc.header);   /* registers the MSC class with ramdisk
    read/write */
3 ca_usb_dev_mng_start();              /* builds the descriptors and connects (
    enumerates) */
```

Feature	Notes
MSC Enumeration	removable disk device, BOT (Bulk-Only Transport)
SCSI Subset	INQUIRY, READ_CAPACITY10, READ10, WRITE10, ...
Block device	512 B blocks, 256 blocks → 128 KB
Back-end ramdisk	`ca_memcpy` from/to heap array
Class SETUP	`GET_MAX_LUN` (1 LUN) + BOT `RESET`

The service `ca_usb_msc` (CBW/CSW state machine + SCSI subset) is in `caos/usb/src/usb_msc.c`; the management stack `ca_usb_dev_mng` is described in `S02_11` and in the plan `doc/usb_stack_workplan.md`.

76.1.2 Hardware

The example runs on three NUCLEO boards, all in OTG_FS mode (Full-Speed, 12 Mbit/s). No external wiring is required: just the USB cable to the board's device connector.

Board	MCU	USB_NUM	USB Connector	Heartbeat LEI
NUCLEO-H753ZI	STM32H753ZI	1	OTG_FS (CN14, "USB user")	PB7 (green LI
NUCLEO-U575ZI-Q	STM32U575ZI	0	Internal OTG_FS	PC7 (green LI
STM32F4VE (black board)	STM32F407VET6	0	Internal OTG_FS (PA11/PA12)	PA6 (D2)

F407 (black board): HW tested. The ramdisk is reduced to **64 KB** (`BLK_COUNT =128`) because the STM32F407VET6 has 192 KB of SRAM; 128 KB remain for the OS and stacks. 64 KB is the minimum for a valid FAT12. LED is PA6 (not PD12).

Attention to the connector: on NUCLEO boards, the "ST-LINK" connector is for flashing/debugging and is **not** the application's USB device connector. To make the disk appear, you must connect the OTG_FS connector (the second micro-USB / USB-C "USER" on the board).

76.1.3 os_config.h Configuration

```
#define EN_USB        1     /* enables the USB DWC2 driver + stack mgmt */
#define EN_USB_MSC    1     /* enables the Mass Storage class */
/* For the isolated test, leave the other classes at 0: */
#define EN_USB_CDC    0
#define EN_USB_NCM    0
#define EN_USB_CUSTOM 0
```

EP number and PID are optional overrides (defaults work):

```
#define USB_MSC_EP_DATA  4U      /* EP4: bulk IN + bulk OUT with the same number
    */
/* PID = USB_PID_BASE_MSC + USB_MSC_EP_DATA = 0x0020 + 4 = 0x0024 */
```

Note on EP allocation: MSC uses **a single endpoint number** for two directions (EP4 IN for responses, EP4 OUT for commands and write data). This is normal in USB: IN and OUT with the same number are distinct endpoints.

76.1.4 Architecture

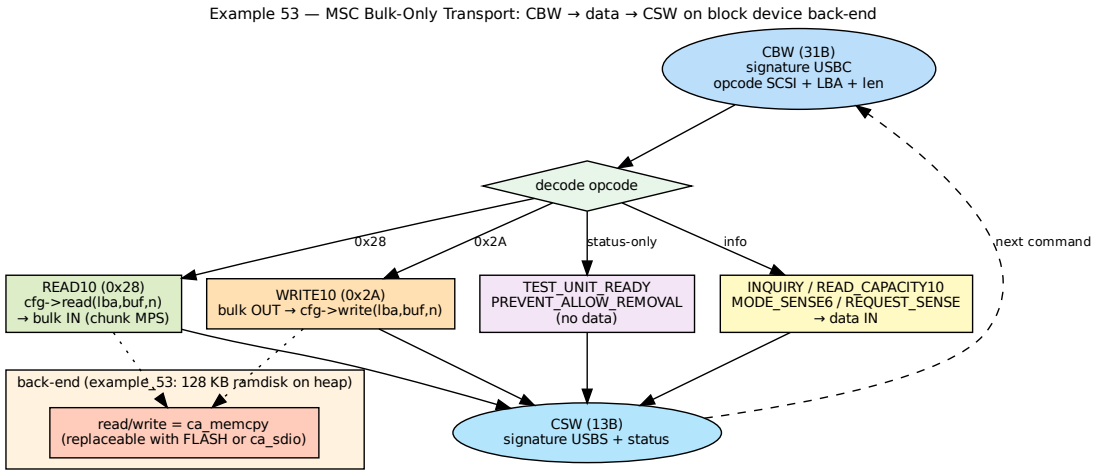


Figure 76.1: State machine BOT CBW → data → CSW (ex53)

Source: `doc/diagrams/ex53_msc_bot.dot` (regenerated from `make diagrams`).

Task	Priority	Function
`usb`	LEVEL6	`ca_usb_dev_mng_init/add/start` then idle (the service runs from ISR)
`hb`	LEVEL4	toggle LED heartbeat every 500 ms (system life)

Unlike the CDC/NCM examples, here **there is no application task** that drains the data: the MSC service is entirely driven by the host. Each SCSI command arrives as a bulk transfer, the BOT state machine processes it in the ISR/callback context and calls `read`/`write` of the ramdisk. The task `usb` only serves to initialize and then sleep.

76.1.5 Back-end block device

The MSC service is **storage agnostic**: the cfg provides two callbacks, one for reading and one for writing blocks. By changing only these two functions the same service works on FLASH, on SD card (`ca_sdio_*`) or on any other medium.

```
typedef struct {
    ca_usb_device_type_t header;       /* { tag, ops }     must stay first */
    u16  mps;                          /* max packet size bulk (64 in FS) */
    u32  block_size;                   /* 512 */
    u32  block_count;                  /* 256     128 KB */
    bool (*read) (u32 lba, u8 *buf, u32 count);        /* reads count blocks */
    bool (*write)(u32 lba, const u8 *buf, u32 count);  /* writes count blocks */
} ca_usb_msc_cfg_t;
```

The ramdisk implementation is trivial — `ca_memcpy` with boundary checking:

```c
static u8 *s_disk;   /* 128 KB allocati in ca_os_configure */

static bool ramdisk_read(u32 lba, u8 *buf, u32 count)
{
    if (s_disk == NULL || (lba + count) > BLK_COUNT) return false;  /* out of
    range */
    ca_memcpy(buf, &s_disk[lba * BLK_SIZE], count * BLK_SIZE);
    return true;
}

static bool ramdisk_write(u32 lba, const u8 *buf, u32 count)
{
    if (s_disk == NULL || (lba + count) > BLK_COUNT) return false;
    ca_memcpy(&s_disk[lba * BLK_SIZE], buf, count * BLK_SIZE);
    return true;
}
```

> The control `(lba + count) > BLK_COUNT` is a **security guard**: the host might (due to bugs or attack) request an LBA outside the declared capacity. Returning `false` causes the service to respond with a SCSI error status instead of writing outside the buffer.

76.1.6 Bulk-Only Transport + SCSI

`ca_usb_msc` implements the **BOT** (`CBW` `data` `CSW`) state machine and the minimal SCSI subset required for enumeration and I/O. The BOT protocol encapsulates each command in three phases:

1. **CBW** (Command Block Wrapper, 31 bytes) — the host sends the SCSI command
2. **data** — data transfer (read or write, optional)
3. **CSW** (Command Status Wrapper, 13 bytes) — the device reports the result

SCSI Command	Opcode	Usage
INQUIRY	0x12	device identity (vendor / product / revision)
TEST_UNIT_READY	0x00	is the media ready?
READ_CAPACITY10	0x25	capacity: last LBA + block size
REQUEST_SENSE	0x03	details of the last error
MODE_SENSE6	0x1A	parameters (here: not write-protected)
READ10	0x28	read N blocks starting from LBA
WRITE10	0x2A	write N blocks starting from LBA
PREVENT_ALLOW_REMOVAL	0x1E	lock/unlock ejection

At the **class SETUP** level (control transfer on EP0), the service handles `GET_MAX_LUN` (responds with 1 LUN, the number of "logical disks") and the **BOT RESET** (reset of the state machine after an error). Data travels on **1 bulk IN EP + 1 bulk OUT EP** with the same number (`USB_MSC_EP_DATA`).

76.1.7 Parameters

Define	H753 / U575	F407	Meaning
`BLK_SIZE`	512	512	block size (SCSI standard)
`BLK_COUNT`	256	128	blocks → 128 KB (H7/U5) / 64 KB (F407)
`DISK_BYTES`	131072	65536	total ramdisk capacity
`USB_MSC_EP_DATA`	4	4	bulk endpoint for data
PID	`0x0024`	`0x0024`	`USB_PID_BASE_MSC (0x20) + 4`

Reduced ramdisk on F407: 64 KB is the minimum for FAT12 with 1 primary partition (`mkfs.fat -F 12`). The STM32F407VET6 has only 192 KB of total SRAM; 128 KB remain for the OS, stacks, and code.

76.1.8 Expected UART Output

On the serial console (STDOUT) at startup:

```
[USB] MSC init...
[USB] ramdisk 128 KB ready     plug cable (mkfs.fat)
```

From this point on, the heartbeat LED blinks at 1 Hz and the device is ready: by connecting the OTG_FS cable, the host enumerates the disk.

76.1.9 Test from PC

The script `examples/example_53/test_fat.sh` automates the entire cycle (auto-detection of the device for VID caOS `0xCA01`, partitioning, `mkfs.fat`, mount, file copy, verification of persistence at remount):

```
sudo ./examples/example_53/test_fat.sh             # auto-detect the device
sudo ./examples/example_53/test_fat.sh /dev/sdb    # forced device
```

The script is destructive on the selected device: it asks for confirmation and uses `lsblk` to identify the correct unit (~128 KB) before formatting.

Equivalent manual procedure (`sdX` = letter assigned by the host):

```
lsblk                                  # find the device (capacity ~128 KB)

# 1) Partitioning: DOS table + 1 primary FAT partition     /dev/sdX1
sudo fdisk /dev/sdX <<'EOF'
o                                      # new DOS partition table
n                                      # new partition
p                                      # primary
1                                      # partizione #1
                                       # first sector: default (enter)
                                       # last sector: default = whole disk (enter)
t                                      # change partition type
b                                      # b = W95 FAT32 (LBA)
w                                      # write and quit
EOF
sudo partprobe /dev/sdX                # reread the partition table (creates /dev/
    sdX1)

# 2) Filesystem on the partition (NOT on the whole device)
sudo mkfs.fat -F 12 /dev/sdX1          # FAT12 (small capacity     FAT12)
```

```
sudo mount /dev/sdX1 /mnt
echo "ciao caOS" | sudo tee /mnt/test.txt
sync; sudo umount /mnt

# 3) Check persistence: remount and read back
sudo mount /dev/sdX1 /mnt && cat /mnt/test.txt && sudo umount /mnt
```

`/dev/sdX` **vs** `/dev/sdX1`: partitioning (`fdisk`) operates on the **entire device** (`/dev/sdX`); the filesystem (`mkfs.fat`, `mount`) operates on the **partition** (`/dev/sdX1`). Confusing them is the most common mistake: formatting `/dev/sdX` creates a "superfloppy" without a partition table, which some systems fail to mount. The script `test_fat.sh` uses the same flow with the partition.

The ramdisk is **volatile**: its contents persist as long as the board is powered, but are lost on reset. The tested persistence above is only between successive mounts, not between board resets.

76.1.10 Technical Notes

Transfer Chunking with respect to MPS

A SCSI read/write operation can involve many blocks (e.g., `READ10` of 8 blocks = 4096 bytes), but each USB Full-Speed bulk packet carries at most **64 bytes** (MPS). The MSC service automatically splits the data transfer into multiple packets of MPS size. This is transparent to the back-end: `read`/`write` always see the complete block count, and chunking occurs in the driver.

Single LUN

`GET_MAX_LUN` responds with 0 (meaning "1 LUN", numbered from 0). caOS exposes a single logical disk. For multi-LUN support, the cfg would need to be extended with an array of back-ends.

Reusing the back-end with real storage

To use an SD card instead of the ramdisk, simply replace the two callbacks:

```
static bool sd_read(u32 lba, u8 *buf, u32 count)  { return ca_sdio_read(lba, buf
    , count); }
static bool sd_write(u32 lba, const u8 *buf, u32 count) { return ca_sdio_write(
    lba, buf, count); }
```

The MSC service and descriptors remain identical: the block-device abstraction completely isolates the USB transport from the physical storage.

76.1.11 File structure

File	Description
`caos/usb/inc/usb_msc.h`	cfg block-device `ca_usb_msc_cfg_t` + ops
`caos/usb/src/usb_msc.c`	`ca_usb_msc_ops` (state machine BOT + subset SCSI)
`hw/f4xx/drivers/usb_f4xx.c`	DWC2 slave FIFO (F407/F429/F446)
`examples/example_53/ca_os_ex53.c`	ramdisk on heap + MSC registration

File	Description
`examples/example_53/test_fat.sh`	automatic FAT test (format/mount/persistence)

Chapter 77

USB NCM: USB Ethernet (L2 Loopback)

77.1 Example 54 — USB NCM: USB Ethernet (L2 loopback)

77.1.1 Introduction

This example demonstrates the **NCM** (CDC-NCM 1.0, *Network Control Model*) class of caOS: connecting the board to the PC, the device appears as a **USB network card** — `usb0` (or `enxXXXXXX`) on Linux, "USB Ethernet" / "Remote NDIS" on Windows. This is the same mechanism used by 4G USB modems and many devices that "provide internet" through the USB cable.

The transport is **Layer 2 Ethernet**: the device receives and transmits complete Ethernet frames (destination MAC, source MAC, ethertype, payload), encapsulated in **NTB** (NCM Transfer Block) — the wire format of the CDC-NCM specification.

This example deliberately stops at the **Layer 2** level and performs **loopback**: for every frame received from the host, it swaps source MAC destination MAC and sends it back. There is no IP stack on the device. The purpose is to validate in isolation the most sensitive part — the encode/decode of the wire NTB and the integration of the service into the `ca_usb_dev_mng` stack. The next example (55) adds the IP stack for actual ping.

```
ca_usb_dev_mng_init(CA_USB_SPEED_FS, USB_NUM);
ca_usb_dev_mng_add(&s_ncm.header);   /* cfg con MAC + callback on_rx */
ca_usb_dev_mng_start();
```

Feature	Notes
NCM Enumeration	device "USB Ethernet" (CDC-NCM 1.0)
Wire NTB	1 datagram per block (NTH16 + NDP16 + payload)
L2 Loopback	swap SADA, re-inject the frame
Link Control	`SET_INTERFACE` alt0=down / alt1=up
Notifications	`NETWORK_CONNECTION` + `CONNECTION_SPEED_CHANGE`
String descriptor	`iMACAddress` (MAC exposed to host)

The service `ca_usb_ncm` (CDC-NCM descriptors, NTB, SET_INTERFACE alt0/1, MAC string) is in `caos/usb/src/usb_ncm.c`; the stack `ca_usb_dev_mng` is in `S02_11`.

77.1.2 PC Prerequisite: Remove the interface from NetworkManager

When the board enumerates, a new network interface appears on the PC (`enx02ca0500xx` — the name udev derives from the MAC). NetworkManager sees it as any other newly connected card and **assigns it an address of its own**: it has been observed `172.21.10.100/24`. The board is on `192.168.10.0/24`, so the two end up on different subnets and the board is unreachable.

The symptom is misleading, because the interface **exists and is UP**: `ip link` shows it, and even `ping -I <iface>` responds — that flag bypasses the routing table. What doesn't work is everything else (`socat`, `nc`, the PC scripts from the examples), which follow the routing table.

The rule is versioned in the repository:

```
sudo cp tools/99-caos-ncm.conf /etc/NetworkManager/conf.d/
sudo systemctl reload NetworkManager
nmcli -t -f DEVICE,STATE dev status | grep enx    # it must say: unmanaged
```

> **The glob goes to `interface-name`, not to `mac:`.** The form `mac:02:ca:05:00:00:*` seems equivalent but it is not: it requires a full MAC and fails **silently** with a wildcard character — no error, and the interface remains managed.

The address of the bank does not set this rule: it is set by `sudo ./tools/bench_net.sh usb` or the example PC script.

77.1.3 Hardware

Board	MCU	`USB_NUM`	USB Connector	LED heartbeat
NUCLEO-H753ZI	STM32H753ZI	1	OTG_FS (CN14, "USB user")	PB7 (green LD
NUCLEO-U575ZI-Q	STM32U575ZI	0	internal OTG_FS	PC7 (green LD
STM32F4VE (black board)	STM32F407VET6	0	internal OTG_FS (PA11/PA12)	PA6 (D2)

No external wiring: the "network cable" is the USB cable itself. Connect the OTG_FS connector (not the ST-LINK one).

> **F407**: HW tested (black board STM32F4VE). Loopback L2 5/5 OK (`test_ncm.sh`). LED PA6.

77.1.4 os_config.h Configuration

```
#define EN_USB         1     /* driver USB DWC2 + stack mgmt */
#define EN_USB_NCM     1     /* enables the CDC-NCM class */
/* Test isolato: le altre classi a 0 */
#define EN_USB_CDC     0
#define EN_USB_MSC     0
#define EN_USB_CUSTOM  0
```

Optional overrides for endpoints (the defaults do not collide with other classes):

```
#define USB_NCM_EP_DATA   1U     /* EP1: bulk IN+OUT, trasporto NTB */
#define USB_NCM_EP_NOTIF  5U     /* EP5: interrupt IN, notifiche link */
/* PID = USB_PID_BASE_NCM + USB_NCM_EP_DATA */
```

77.1.5 Architecture

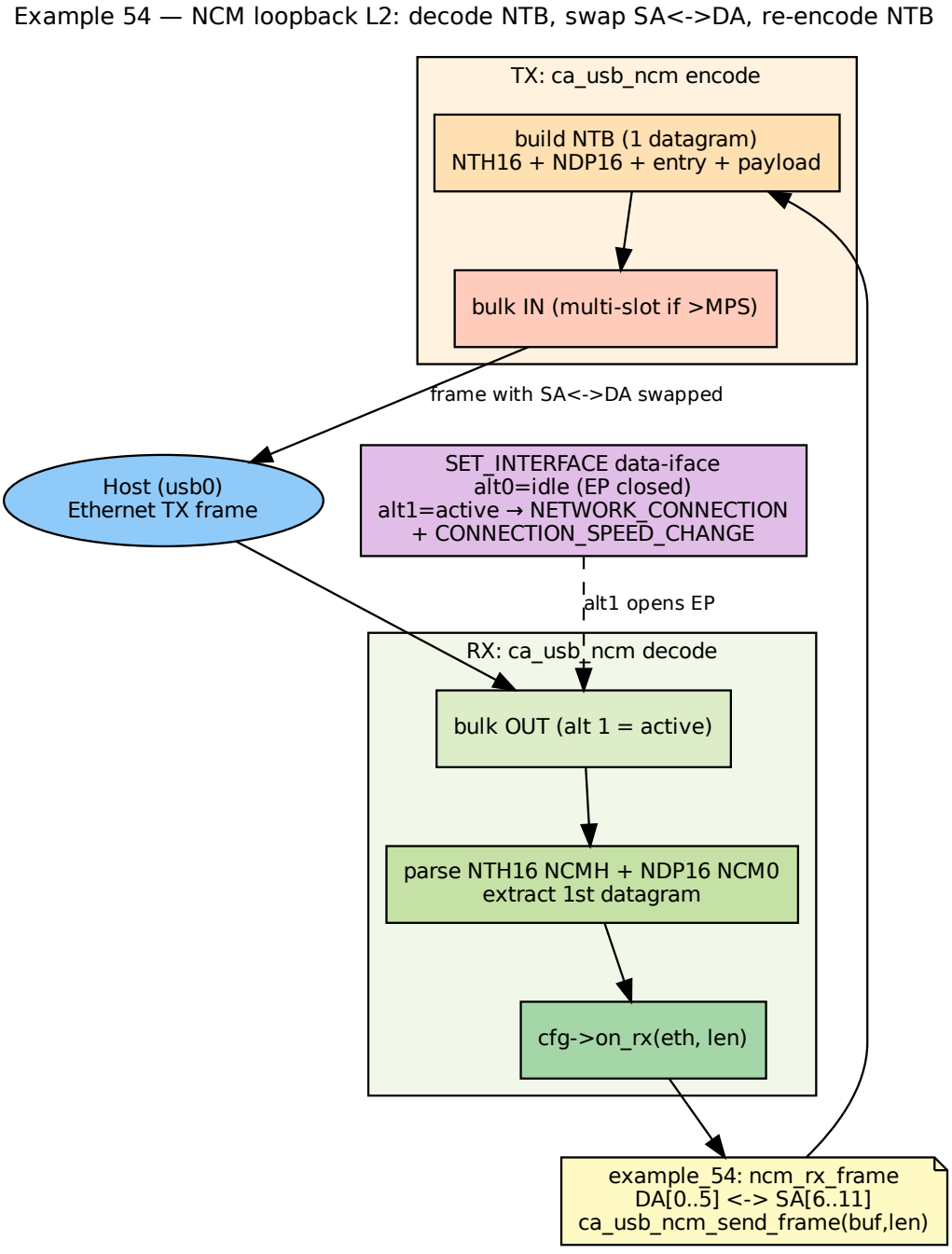


Figure 77.1: Loopback L2: decode NTB, swap SADA, re-encode (ex54)

Source: `doc/diagrams/ex54_ncm_loopback.dot` *(regenerated from* `make diagrams`*).*

Task	Priority	Function
`usb`	LEVEL6	init stack + polling link up/down (print status change)
`hb`	LEVEL4	toggle LED heartbeat every 500 ms

Frame reception **does not** go through a task: the NCM service delivers each decoded datagram directly to the `on_rx` callback, called in ISR context. The example performs the work (swap + retransmission) inside the callback because it is a short operation; in a real case with heavy processing, the frame would be queued and processed in a task (this is what example 55 does with the net core).

77.1.6 Loopback L2 with SADA swap

The NCM service is a **neutral L2 transport**: it delivers each received Ethernet frame to the `on_rx(eth, len)` callback, where `eth` points to the complete frame starting from the destination MAC address. The example swaps the first 12 bytes (DA[0..5] SA[6..11]) and reinjects the frame:

```
static void ncm_rx_frame(const u8 *eth, u16 len)
{
    if (len < 14U) return;                  /* at least the Ethernet header is
    needed */
    s_rx_count++;

    static u8 buf[USB_NCM_MAX_FRAME] __attribute__((aligned(4)));
    if (len > sizeof(buf)) return;
    ca_memcpy(buf, eth, len);
    for (u32 i = 0U; i < 6U; i++) {          /* DA <-> SA: scambia i 6+6 byte */
        u8 t = buf[i]; buf[i] = buf[i + 6U]; buf[i + 6U] = t;
    }
    if (ca_usb_ncm_send_frame(buf, len)) s_tx_count++;
}
```

By swapping SADA, the host sees a frame returning with **DA = its own MAC** (thus accepting it) and **SA = the device's MAC**. Without the swap, the host would discard the frame because it is addressed to itself (DA = own MAC, already in the outgoing frame is the case of a frame that the device echoes identically — the host ignores it).

> Copying to a local buffer is necessary: `eth` points to service memory that may be reused after the return of `on_rx`. `ca_usb_ncm_send_frame` copies to its own NTB, therefore `buf` (static) is reusable in the next iteration.

77.1.7 Wire NTB (1 datagram per block)

Each Ethernet frame is encapsulated in an **NTB** (NCM Transfer Block) containing a single datagram. The block structure:

```
NTH16 (12B) : signature "NCMH", wHeaderLength, wSequence, wBlockLength,
    wNdpIndex
NDP16  (8B) : signature "NCM0", wLength, wNextNdpIndex
entries     : (wDatagramIndex, wDatagramLength) + terminatore (0, 0)
payload     : full Ethernet frame (from DA up to the payload)
```

The NTH16 header declares the total length of the block and points to the NDP16 descriptor; the latter lists the entries (offset + length of each datagram) terminated by a (0,0) pair. With a single datagram per block, the logic remains simple; the support for aggregated multi-datagram is a future extension (`wNtbOutMaxDatagrams=1` for now).

Link Control via SET_INTERFACE

The NCM data-interface has two **alternate settings** that act as link switches:

- **alt 0** = idle → bulk endpoints closed, no traffic (`ca_usb_ncm_is_up()` returns `false`)
- **alt 1** = active → bulk endpoints open, the service sends the notifications `NETWORK_CONNECTION` + `CONNECTION_SPEED_CHANGE` (link up) on the interrupt EP

It is the host that decides when to bring up the link (`ip link set usb0 up` → sends `SET_INTERFACE alt 1`). As long as it remains at alt 0, the device does not transmit.

77.1.8 Parameters

Define	Value	Meaning
`USB_NCM_EP_DATA`	1	Bulk EP for NTB transport
`USB_NCM_EP_NOTIF`	5	Interrupt EP for link notifications
`mps`	64	max packet size bulk (Full-Speed)
Device MAC	`02:CA:05:00:00:01`	MAC of the device (locally administered)

77.1.9 Expected UART Output

```
[USB] NCM init...
[USB] NCM ready     bring up usb0 on host (loopback L2)
```

When the host asserts the interface (`ip link set usb0 up`):

```
[USB] link UP
```

And at shutdown or disconnection:

```
[USB] link DOWN
```

Diagnostic CLI — the `ncm` command reports the status:

```
c.a.O.S. > ncm
ncm: link=up rx=42 tx=42
```

77.1.10 Test from PC

```
ip link                                  # find the interface (usb0 / enxXX)
sudo ip addr add 192.168.7.1/24 dev usb0
sudo ip link set usb0 up                 #     SET_INTERFACE alt1     link UP on
    the device
sudo tcpdump -i usb0 -e                  # watch incoming/outgoing frames
arping -I usb0 192.168.7.2               # requests bounce back with  S A DA
    swapped
```

> **Success criterion**: (1) `usb0` correctly enumerated, (2) TX frames bounce back in RX with SADA swapped, visible in `tcpdump -e`. Without IP stack on the device, `ping` **does not** receive ICMP reply — expected: loopback is purely L2. Integration with `caos/net` (push via `ca_net_iface_t`) is example 55.

Tested HW on U575: 5/5 frame `arping` bounced correctly (`test_ncm.sh`).

77.1.11 Technical notes

Why loopback and not ping immediately?

Separating validation of the wire NTB from the net stack is a debug choice: if the ping of example 55 does not work, example 54 tells whether the problem is in the encode/decode NTB (low level) or in the integration with the net core (high level). The L2 loopback only exercises the first.

MAC locally administered

The MAC `02:CA:05:00:00:01` has bit 1 of the first byte set to 1 (`0x02`): it is a *locally administered* address, not assigned by IEEE. This is the correct practice for devices that do not have a registered OUI — avoids collisions with real hardware on the network.

Buffer alignment

`buf` is declared `__attribute__((aligned(4)))`: word alignment speeds up `ca_memcpy` and the swap, and is required by some driver implementations that access the buffer at 32 bit.

77.1.12 File structure

File	Description
`caos/usb/inc/usb_ncm.h`	cfg `ca_usb_ncm_cfg_t` + API `ca_usb_ncm_send_frame/is_up`
`caos/usb/src/usb_ncm.c`	`ca_usb_ncm_ops` (CDC-NCM descriptors + wire NTB)
`hw/f4xx/drivers/usb_f4xx.c`	DWC2 slave FIFO (F407/F429/F446)
`examples/example_54/ca_os_ex54.c`	L2 loopback + CLI `ncm`
`examples/example_54/test_ncm.sh`	automated test (arping + swap verification)

Chapter 78

Net Stack over USB-NCM: Real Ping

78.1 Example 55 — caOS Net Stack on USB-NCM (real ping)

78.1.1 Introduction

This example is the **USB variant of example 33** (net stack on Ethernet). The same caOS net core — ARP for address resolution, ICMP for ping — runs here on a **USB network adapter** (CDC-NCM) instead of wired Ethernet. The concrete result: after bringing up `usb0`, from the PC you can do `ping` to the board and receive a response.

The central pedagogical point is the `ca_net_iface_t` abstraction: the net core does not know and does not care whether frames arrive from the integrated Ethernet MAC or the USB port. It only sees a "network interface" — a struct of function pointers. By changing only the argument of `ca_net_init`, the exact same IP stack works on both transports. It's the same principle as the Linux/BSD driver model, scaled down for embedded use.

Unlike example 54 (pure L2 loopback), here the device has the **active IP stack** and therefore responds to a real ping:

```
ca_usb_dev_mng_init(CA_USB_SPEED_FS, USB_NUM);
ca_usb_dev_mng_add(&s_ncm.header);
ca_usb_dev_mng_start();
ca_net_init(ca_ncm_iface_get(&s_ncm));   /* registers NCM as a PUSH iface */
ca_net_proto_init(&net_cfg);             /* enables ARP + ICMP echo reply  */
```

Feature	Notes
USB Ethernet (NCM)	device `usb0` via CDC-NCM
`ca_net_iface_t` abstraction	same net core on ETH or NCM
RX push model	`ca_net_rx_push` (NCM delivery, no DMA pull)
ARP service	MAC IP resolution
ICMP echo reply	replies to `ping` from host
CLI `ping` / `ip`	reverse ping device → host (with `EN_CLI_NET`)

78.1.2 PC Prerequisite: Remove the interface from NetworkManager

When the board enumerates, a new network interface appears on the PC (`enx02ca0500xx` — the name that udev derives from the MAC). NetworkManager sees it as any other attached card

and **assigns it an address of its own**: it has been observed `172.21.10.100/24`. The board is on `192.168.10.0/24`, so the two end up on different subnets and the board becomes unreachable.

The symptom is misleading, because the interface **exists and is UP**: `ip link` shows it, and even `ping -I <iface>` responds — that flag bypasses the routing table. What fails is everything else (`socat`, `nc`, the PC scripts from the examples), which follow the routing table.

The rule is versioned in the repository:

```
sudo cp tools/99-caos-ncm.conf /etc/NetworkManager/conf.d/
sudo systemctl reload NetworkManager
nmcli -t -f DEVICE,STATE dev status | grep enx    # it must say: unmanaged
```

> **The glob goes on `interface-name`, not on `mac:`.** The form `mac:02:ca:05:00:00:*` seems equivalent but it is not: it requires a full MAC and fails **silently** with a wildcard character — no error, and the interface remains managed.

The bank address does not set this rule: it is set by `sudo ./tools/bench_net.sh usb` or the example PC script.

78.1.3 Hardware

Board	MCU	`USB_NUM`	USB Connector	LED heartbea
NUCLEO-U575ZI-Q	STM32U575ZI	0	Internal OTG_FS	PC7 (green LI
NUCLEO-H753ZI	STM32H753ZI	1	OTG_FS (CN14, “USB user”)	PB7 (green LI
STM32F4VE (black board)	STM32F407VET6	0	Internal OTG_FS (PA11/PA12)	PA6 (D2)

> **F407**: HW tested. PING 5/5 OK from PC → device; CLI `ping` reverse OK. LED PA6. `NET_BUF_COUNT=8` (reduced to leave RAM for ramdisk in ex56).

Device network identity (defined in the example):

Parameter	Value
IP device	192.168.10.55
MAC device (caOS side)	02:CA:05:00:00:55
MAC host (exposed by NCM)	00:CA:05:00:00:A5

78.1.4 os_config.h Configuration

```
#define EN_NET       1     /* net core: pool buffer, ARP, ICMP,     */
#define EN_USB       1     /* driver USB DWC2 + stack mgmt */
#define EN_USB_NCM   1     /* CDC-NCM class (the network transport) */
#define EN_CLI_NET   1     /* optional: CLI commands 'ping' / 'ip' */
/* Test isolato: altre classi USB a 0 */
#define EN_USB_CDC    0
#define EN_USB_MSC    0
#define EN_USB_CUSTOM 0

#define NET_BUF_COUNT  32U   /* number of net_buf in the RX/TX pool */
```

> **Note**: `EN_NET` enables the full net core. `ca_net_proto_init` automatically registers the CLI command `ping`/`ip` if `EN_CLI_NET=1`.

78.1.5 Architecture

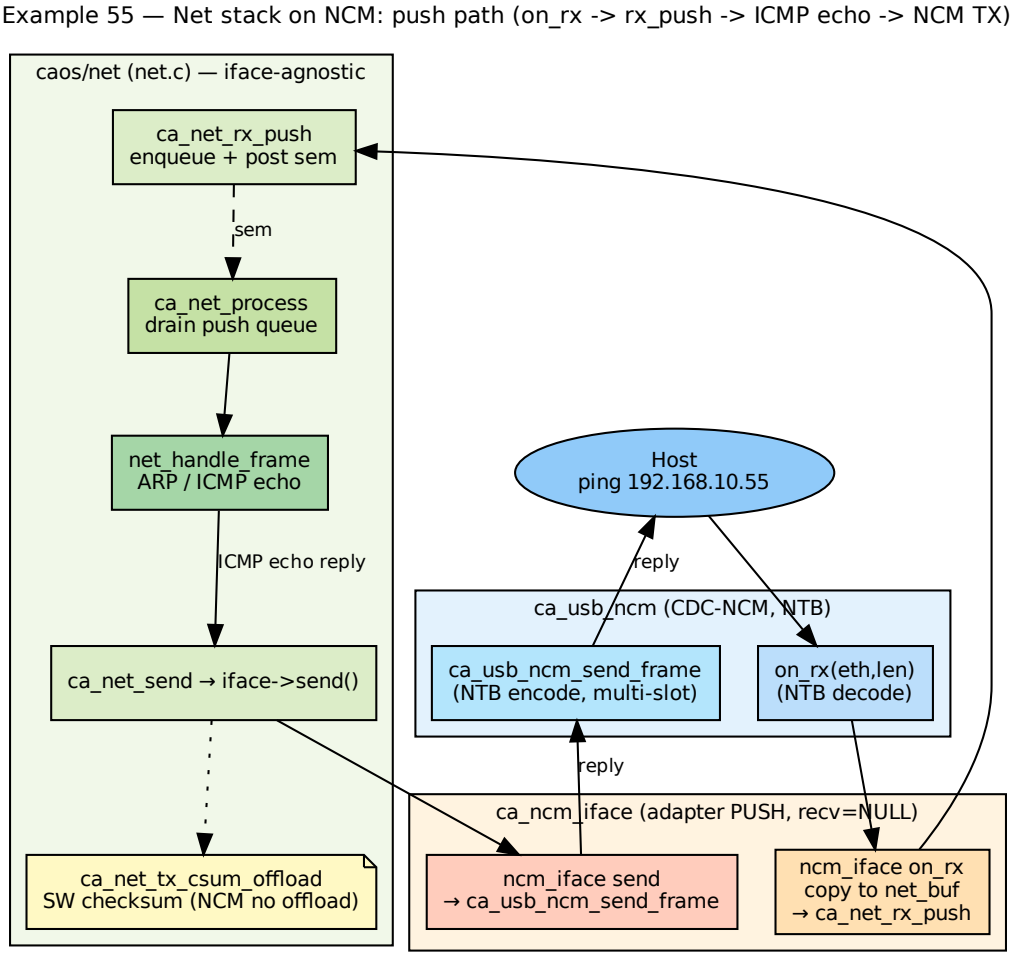


Figure 78.1: Path push: on_rx → rx_push → ICMP echo → NCM TX (ex55)

Source: `doc/diagrams/ex55_ncm_netstack.dot` *(regenerated from* `make diagrams`*).*

Task	Priority	Function
`net`	LEVEL1	init USB+NCM+net core, then loop: `ca_sem_wait(&ca_net_rx_sem)` → `ca_net_process`
`net_stats`	LEVEL4	print network counters every 5 s (pool, rx, tx, drop)
`heartbeat`	LEVEL4	toggle LED heartbeat every 500 ms

The task `net` has high priority (LEVEL1): it must drain incoming frames with low latency

to avoid saturating the `net_buf` pool. It blocks on `ca_net_rx_sem` and wakes up when the NCM adapter has queued a frame.

78.1.6 Interface Abstraction: ETH vs NCM

The net core (`net.c`) **never calls the driver directly**: it uses a `ca_net_iface_t`, a function pointer struct. Two adapters provide it, with different receive models:

Adapter	RX Model	TX
`ca_eth_iface_get(&eth_cfg)`	**pull** zero-copy (DMA, `ops.recv`)	`ca_eth_send`
`ca_ncm_iface_get(&ncm_cfg)`	**push** (`ca_net_rx_push`, `ops.recv=NULL`)	copy to NTB

Pull (ETH): the ETH driver has frames already in RAM via DMA; the net core "pulls" them when ready, without copies. **Push (NCM)**: the USB service delivers each datagram to the adapter's callback, which copies it into an `net_buf` and calls `ca_net_rx_push()` (queues + posts the semaphore that wakes up the net task). NCM TX copies the `net_buf` into an NTB and sends it with `ca_usb_ncm_send_frame()`.

The notable thing is that `ca_net_init` is **identical** to the one in example 33 — only the argument changes:

```
/* ex33 (Ethernet):  */ ca_net_init(ca_eth_iface_get(&eth_cfg));
/* ex55 (USB-NCM):   */ ca_net_init(ca_ncm_iface_get(&s_ncm));
```

All the rest (ARP, ICMP, the loop `ca_net_process`, the counters) is the same code.

78.1.7 Expected UART Output

```
=== EX55: caOS net stack su USB-NCM ===
MAC  02:CA:05:00:00:55
IP   192.168.10.55
Net task ready     bring up usb0 then ping 192.168.10.55
CLI: 'ping <ip> [count=N] [time=T]' to ping the host from here
```

Periodic statistics (every 5 s) from task `net_stats`:

```
STATS link=up pool_free=32/32 rx=5 drop_pool=0 no_handler=0 tx=5 tx_fail=0 proc
   =5
```

78.1.8 Test from PC

```
ip link                                    # find usb0 / enxNN
sudo ip addr add 192.168.10.1/24 dev usb0
sudo ip link set usb0 up
ping -c 5 192.168.10.55                     # ICMP echo reply from the device
arping -c 3 -I usb0 192.168.10.55           # ARP reply from the device
```

Success criterion: `ping` receives reply from device — entire path push works: NCM `on_rx` → adapter `ca_net_rx_push` → net task `ca_net_process` → ARP/ICMP → ICMP echo reply → NCM TX. Tested HW on U575: **ping 5/5 ~0.8 ms**.

Reverse ping (device → host) from serial CLI

`ca_net_proto_init()` automatically registers CLI commands `ping`/`ip` (requires `EN_CLI_NET=1`). From device's serial console:

```
ping 192.168.10.1 [count=N] [time=T]    # the device pings the host
ip info | set <ip> | flush | link up|down | mtu <N>
```

The command resolves the host's MAC via ARP, sends echo requests over the NCM interface and measures the RTT with DWT — **the exact same code** used on Ethernet, a tangible proof that the stack is interface-agnostic.

78.1.9 Technical notes

Push vs pull: why two models

Ethernet has a DMA that writes frames to RAM autonomously: pull zero-copy (no copy, the net core reads directly from the DMA buffer) is preferred. USB does not have this: data arrives "pushed" by a bulk transfer inside an ISR callback, and must be copied into a `net_buf` to decouple it from the volatile service memory. The `ca_net_iface_t` abstraction hides this difference from the net core.

Two MACs, one per side

NCM has **two** MACs: the device's (`...:55`, used by caOS's net core) and the "host side" (`...:A5`) that the device declares to the host via `iMACAddress`. This is the standard CDC-NCM model: the USB link is a point-to-point Ethernet segment with two nodes, each with its own MAC.

Pool sizing

`NET_BUF_COUNT` limits how many frames can be in flight. With the push path, if the net task doesn't drain fast enough, the pool gets exhausted and frames are dropped (`drop_pool` in `STATS`). For higher traffic, increase `NET_BUF_COUNT` or raise the priority of task `net`.

78.1.10 File structure

File	Description
`caos/net/src/net.c`	net core (pool, dispatch, `ca_net_process`)
`caos/net/src/ncm_iface.c`	push adapter `ca_ncm_iface_get` (NCM → net core)
`caos/usb/src/usb_ncm.c`	CDC-NCM service (transport)
`hw/f4xx/drivers/usb_f4xx.c`	DWC2 slave FIFO (F407/F429/F446)
`examples/example_55/ca_os_ex55.c`	net stack on NCM + ping + CLI

Chapter 79

Composite USB: CDC + MSC + NCM

79.1 Example 56 — Composite USB CDC + MSC + NCM

79.1.1 Introduction

This example puts it all together: a **composite USB device** with **three active classes simultaneously** on the same `ca_usb_dev_mng` stack. By connecting a single cable, the PC will see three devices appear at the same time:

- a **virtual serial port** (CDC-ACM) — `/dev/ttyACMx`, with character echo
- a **removable disk** (MSC) — a 128 KB ramdisk on which to create a FAT filesystem
- a **USB network card** (CDC-NCM) — `usb0`, with the caOS net stack on top that responds to real ping requests

The value of this example is to demonstrate that the `ca_usb_dev_mng` stack is truly composable: the application **registers the three classes** with three `ca_usb_dev_mng_add` and the management automatically builds a **single composite config descriptor** (with the necessary IAD for CDC) and sums up the PID contributions of the three classes. There is no special code "for the composite case": it is simply the sum of examples 52, 53 and 55.

```
ca_usb_dev_mng_init(CA_USB_SPEED_FS, USB_NUM);
ca_usb_dev_mng_add(&s_cdc.header);    /* /dev/ttyACMx     serial echo          */
ca_usb_dev_mng_add(&s_msc.header);    /* removable disk     128 KB ramdisk    */
ca_usb_dev_mng_add(&s_ncm.header);    /* usb0     caOS net stack (real ping)  */
ca_usb_dev_mng_start();
ca_net_init(ca_ncm_iface_get(&s_ncm));   /* NCM as a push iface */
ca_net_proto_init(&net_cfg);             /* ARP + ICMP echo     */
```

Feature	Class	Result on PC
Virtual serial port	CDC-ACM	`/dev/ttyACMx`, interactive echo
Removable disk	MSC	`/dev/sdX`, FAT mountable
Network card	CDC-NCM	`usb0`, real ping to the device
Single config descriptor	mgmt	single device, three interfaces
Composite PID	mgmt	sum of the contributions of the 3 classes

79.1.2 PC Prerequisite: Remove the interface from NetworkManager

When the board enumerates, a new network interface appears on the PC (`enx02ca0500xx` — the name that udev derives from the MAC). NetworkManager sees it as any other newly connected card and **assigns it an address of its own**: it has been observed `172.21.10.100/24`. The board is on `192.168.10.0/24`, so the two end up on different subnets and the board is unreachable.

The symptom is misleading, because the interface **exists and is UP**: `ip link` shows it, and even `ping -I <iface>` responds — that flag bypasses the routing table. What doesn't work is everything else (`socat`, `nc`, the PC scripts of the examples), which follow the routing table.

The rule is versioned in the repository:

```
sudo cp tools/99-caos-ncm.conf /etc/NetworkManager/conf.d/
sudo systemctl reload NetworkManager
nmcli -t -f DEVICE,STATE dev status | grep enx    # it must say: unmanaged
```

> **The glob goes to `interface-name`, not to `mac:`.** The form `mac:02:ca:05:00:00:*` seems equivalent but it is not: it requires a full MAC and fails **silently** with a wildcard character — no error, and the interface remains managed.

The bank address does not set this rule: it is set by `sudo ./tools/bench_net.sh usb` or the example PC script.

79.1.3 Hardware

Board	MCU	`USB_NUM`	USB Connector	LED heartbeat	Classes
NUCLEO-U575ZI-Q	STM32U575ZI	0	Internal OTG_FS	PC7 (green LD1)	CDC+MSC+NCM (5 EP)
NUCLEO-H753ZI	STM32H753ZI	1	OTG_FS (CN14, "USB user")	PB7 (green LD2)	CDC+MSC+NCM (5 EP, margin to 6)
STM32F4VE (black board)	STM32F407VET6	0	Internal OTG_FS (PA11/PA12)	PA6 (D2)	**NCM+MSC** (3 EP, no CDC)

> **F407 — bimodal configuration**: the F407's OTG_FS has only **3 data EPs** (EP1..3, `USB_EP_MAX=4`). The composite CDC+MSC+NCM requires 5 EPs → it does not fit. On F407 the example compiles in **NCM+MSC** mode (2 classes, 3 EPs total), controlled by `#define EX56_HAS_CDC 0` (automatically set for `STM32F4`). For U575 and H753 the CDC is included (`EX56_HAS_CDC=1`).

Device network identity (NCM side):

Parameter	Value
IP device	192.168.10.56
MAC device	02:CA:05:00:00:56
MAC host (NCM)	02:CA:05:00:00:A6

> **Endpoint limit**: the three classes together use **5 data endpoints**, which is exactly the maximum of OTG_FS on U575 and F407 (RM0456 §72.2.3: EP0 control + 5 IN + 5 OUT). The H7 has margin (up to 6 data EPs). It is not possible to add a fourth data class without freeing an endpoint.

79.1.4 os_config.h Configuration

```
#define EN_USB        1     /* driver USB DWC2 + stack mgmt */
#define EN_USB_CDC    1     /* virtual serial port (not on F407) */
#define EN_USB_MSC    1     /* removable disk */
#define EN_USB_NCM    1     /* USB network card */
#define EN_NET        1     /* net core for ping over NCM */
```

2-class combinations: to obtain CDC+MSC, MSC+NCM or CDC+NCM, just turn off one of `EN_USB_CDC` / `EN_USB_MSC` / `EN_USB_NCM`. No additional files or examples needed: the EP allocation remains valid (a subset). CDC+MSC = 3 EP, MSC+NCM = 3 EP, CDC+NCM = 4 EP.

F407: keep `EN_USB_CDC=1` in `os_config.h` but the source `ca_os_ex56.c` automatically sets `EX56_HAS_CDC=0` for `STM32F4` → the CDC class is not registered, the composite descriptor is NCM+MSC (3 EP, within the limit). Also reduce `NET_BUF_COUNT` from 16 to 8 to leave space for a 64 KB ramdisk.

```
/* F407 only (ridotto per RAM 192 KB): */
#define NET_BUF_COUNT   8U
```

79.1.5 Endpoint allocation (maximum case)

The three classes together **saturate** the OTG_FS. The default allocation in `os_config.h` is designed to avoid collisions:

EP	Class	Type	Define
EP1	NCM	bulk data (NTB)	`USB_NCM_EP_DATA`
EP2	MSC	bulk data (CBW/CSW)	`USB_MSC_EP_DATA`
EP3	CDC	bulk data (serial)	`USB_CDC_EP_DATA`
EP4	CDC	interrupt notif	`USB_CDC_EP_NOTIF`
EP5	NCM	interrupt notif	`USB_NCM_EP_NOTIF`

The high-bandwidth bulk EPs (NCM, MSC) have a deeper ring queue; the CDC (low bandwidth) has a short queue. A `_Static_assert` (D16 decision) in `usb_mgmt.h` checks at **compile-time** that the EP masks of active classes do not overlap — an allocation error never reaches runtime:

```
USB_POPCOUNT(USB_EPMASK_CDC) + USB_POPCOUNT(USB_EPMASK_MSC) +
USB_POPCOUNT(USB_EPMASK_NCM) == USB_POPCOUNT(USB_EPMASK_ALL)
```

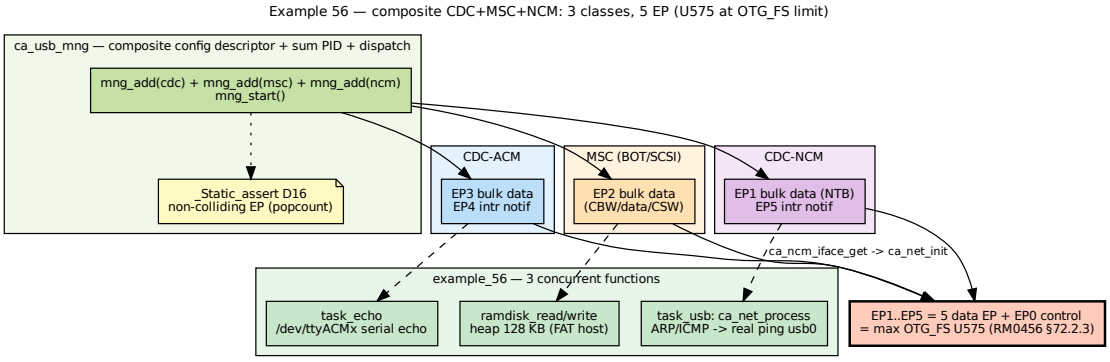


Figure 79.1: Composito CDC+MSC+NCM: 3 classes, 5 EP (ex56)

Source: `doc/diagrams/ex56_composite.dot` *(regenerated from* `make diagrams`*).*

79.1.6 Architecture

Task	Priority	Function
`echo`	LEVEL5	drains the CDC RX ring (filled by ISR) → echo to host
`usb`	LEVEL1	init 3 classes + net core; loop: drains `ca_net_rx_sem` + reports DTR changes
`stats`	LEVEL4	counters (NCM link, DTR, pool, rx/tx) every 5 s
`hb`	LEVEL4	toggle LED heartbeat every 500 ms

The task `usb` provides **dual service** in a non-blocking way: it uses `ca_sem_timedwait(&ca_net_rx_sem, 100ms)` to drain the net core (NCM frames incoming) and, when the timeout expires, checks if the CDC DTR status has changed and reports it to the console. One task, two responsibilities, no indefinite blocking.

```
u8 dtr_prev = 0U;
while (1) {
    if (ca_sem_timedwait(&ca_net_rx_sem, 100U)) {
        ca_net_process();                  /* frame NCM     ARP/ICMP */
    }
    u8 dtr = s_dtr;
    if (dtr != dtr_prev) {                   /* CDC terminal state change */
        ca_stdout_put_n(dtr ? "[USB] CDC terminal opened (DTR on)"
                            : "[USB] CDC terminal closed (DTR off)");
        dtr_prev = dtr;
    }
}
```

79.1.7 The three application functions

Class	Task / context	Behavior
CDC	`task_echo`	ring RX filled by ISR → echo of each byte toward the host
MSC	callback	`ramdisk_read`/`ramdisk_write` on 128 KB heap
NCM	`task_usb`	drains `ca_net_rx_sem` → `ca_net_process` (ARP/ICMP)

The three classes are completely independent: each has its own endpoints, its own callbacks and its own data flow. The management keeps them together only at the descriptor and enumeration level; at runtime each travels on its own EP without interfering.

79.1.8 Expected UART Output

```
=== EX56: USB composito CDC+MSC+NCM ===
[USB] composite up: /dev/ttyACMx + ramdisk 128 KB + usb0
[NET] ping 192.168.10.56 (bring up usb0 first)
```

When opening the CDC terminal from the PC:

```
[USB] CDC terminal opened (DTR on)
```

Periodic statistics (every 5 s):

```
STATS ncm=up cdc_dtr=1 pool=32/32 rx=5 tx=5 tx_fail=0
```

79.1.9 Test from PC

The three interfaces appear together under the same VID `0xCA01`:

```
lsusb -v -d ca01: | grep -iE "interface association|Mass Storage|CDC"
```

CDC — serial echo:

```
sudo screen /dev/ttyACM0 115200      # type     it is echoed back
```

MSC — FAT filesystem (reuses the ex53 script):

```
sudo ./examples/example_53/test_fat.sh    # auto-detect VID 0xCA01
```

NCM — actual ping to the device:

```
sudo ip addr add 192.168.10.1/24 dev usb0
sudo ip link set usb0 up
ping -c 5 192.168.10.56
```

Success criterion (U575/H753 — CDC+MSC+NCM): all three functions respond **simultaneously** without interference (serial echo + FAT + ping active at the same time). Tested HW on U575: NCM ping 5/5 + 3/3 reverse, CDC echo 100/100 on `ttyACM1`, MSC FAT persistence OK.

F407 (black board) — NCM+MSC without CDC:

```
lsusb -v -d ca01: | grep -iE "Mass Storage|NCM"
```

MSC — filesystem FAT (ramdisk 64 KB, minimum FAT12):

```
sudo ./examples/example_53/test_fat.sh
```

NCM — real ping to the device:

```
sudo ip addr add 192.168.10.1/24 dev usb0
sudo ip link set usb0 up
ping -c 5 192.168.10.56
```

Tested HW on F407: NCM ping 5/5 + MSC FAT persistence OK. Without CDC (no `/dev/ttyACMx` appears), the system log is visible only via UART ST-Link.

79.1.10 Technical notes

Composite descriptor over 127 bytes — EP0 multi-round

The config descriptor of the three classes put together is **183 bytes**, exceeding the 127 byte limit manageable in a single EP0 transfer on DWC2 (the `XFRSIZ` field in `DIEPTSIZ0` is 7 bit). The driver splits it into multiple "rounds" of transmission: intermediate rounds are **exactly 64 bytes** (a full packet), only the last round ends with a short packet. A short packet in the middle of the descriptor would terminate the control transfer and the host would timeout — for this reason intermediate rounds must be full. Single-class examples (descriptor 127 bytes) are unaffected. Detail of the fix in `hw/u5xx/drivers/usb_u5xx.c` (and symmetric in `hw/h7xx/drivers/usb_h7xx.c`, DMA mode).

IAD for CDC

The CDC-ACM uses two interfaces (comm + data) that must be grouped for the host with an **IAD** (Interface Association Descriptor). The management inserts it automatically when CDC is among the registered classes. MSC and NCM do not need it (single interface, or group managed differently).

Composite PID

`PID = sum of the contributions of the active classes`. With CDC+MSC+NCM the resulting PID is `0x0066` (`NCM 0x31 + MSC 0x22 + CDC 0x13`). This gives each class combination a distinct PID, useful for host-side udev rules.

79.1.11 File structure

File	Description
`examples/example_56/ca_os_ex56.c`	composite CDC+MSC+NCM (U5/H7) or NCM+MSC (F407)
`caos/usb/src/usb_cdc.c`	CDC-ACM class
`caos/usb/src/usb_msc.c`	MSC class (BOT/SCSI)
`caos/usb/src/usb_ncm.c`	CDC-NCM class
`caos/usb/src/usb_mgmt.c`	composite config descriptor + SETUP dispatch
`hw/f4xx/drivers/usb_f4xx.c`	DWC2 slave FIFO (F407/F429/F446)

Chapter 80

USB DFU: Runtime Mode

80.1 Example 57 — USB Management Stack: DFU runtime

80.1.1 Description

Example 57 demonstrates the `ca_usb_dfu` service (decision **D11** of plan `doc/usb_stack_workplan.md`) over the `caos/usb/usb_mgmt` stack. DFU *runtime* exposes a **DFU interface** (class `0xFE` / subclass `0x01` / protocol `0x01`) **without data endpoints**: all dialog passes through EP0.

In runtime mode the device continues to function normally; the DFU interface serves only to get recognized by the host (e.g. `dfu-util`), which can send `DFU_DETACH` to request switching to **DFU mode** (download/upload of the firmware). caOS notifies the application via `on_detach`: the typical usage is to set a flag in no-init RAM and reset to the system bootloader (ST-ROM or an application bootloader). The actual download is **out of scope** for this service.

Three calls are enough to bring up the device:

```
ca_usb_dev_mng_init(CA_USB_SPEED_FS, USB_NUM);        /* registers the mgmt
    driver callbacks */
ca_usb_dev_mng_add(&s_dfu.header);    /* registra la classe DFU (1 iface, no EP)
    */
ca_usb_dev_mng_start();              /* build descriptors + connect
    enumerates */
```

Features: 1. **Enumeration** as a device with DFU interface (VID `0xCA01`, PID `0x0040`) 2. **DFU runtime**: response to `DFU_DETACH`, `DFU_GETSTATUS`, `DFU_GETSTATE`, `DFU_CLRSTATUS`, `DFU_ABORT` (EP0 only, no data EP) 3. **CLI** `usb`: status, start/stop, add/remove (runtime lifecycle)

Supported hardware: NUCLEO-H753ZI (`USB_NUM=1`), NUCLEO-U575ZI-Q (`USB_NUM=0`), NUCLEO-F407ZG (`USB_NUM=0`), all OTG_FS.

> The `ca_usb_dev_mng` stack is documented in the `doc/usb_stack_workplan.md` plan (decisions D1–D20). The low-level driver is in `S02_11_00_usb_driver.md`.

80.1.2 Hardware

All boards operate in OTG_FS. No external wiring: connect the device USB connector (OTG_FS, not ST-LINK).

Board	MCU	`USB_NUM`	USB Connector	Heartbeat LEI
NUCLEO-H753ZI	STM32H753ZI	1	OTG_FS (CN14, “USB user”)	PB7 (LD2 gre
NUCLEO-U575ZI-Q	STM32U575ZI	0	Internal OTG_FS	PC7 (LD1 gre

Board	MCU	USB_NUM	USB Connector	Heartbeat LEI
STM32F4VE (black board)	STM32F407VET6	0	Internal OTG_FS (PA11/PA12)	PA6 (D2)

F407: HW tested. `dfu-util -l` → `Found Runtime: [ca01:0040]` OK; `dfu-util -e` → `DFU_DETACH` received and `on_detach` callback invoked. LED PA6.

80.1.3 os_config.h Configuration

```
#define EN_USB     1     /* driver USB DWC2 + stack mgmt */
#define EN_USB_DFU 1     /* enables the DFU runtime class */
/* CDC stays compiled but unused: the example does not open it and the console
 * does not go through it (STDOUT_ON_USB_CDC=0, STDIN_ON_USB_CDC=0). */
#define EN_USB_CDC    1
#define EN_USB_MSC    0
#define EN_USB_NCM    0
#define EN_USB_CUSTOM 0
```

DFU runtime uses **only EP0** (control): it does not contribute to the EP collision bitmap management (`USB_EPMASK_*`). It can therefore coexist with any other class without consuming data endpoints.

80.1.4 Architecture

```
ca_usb_dev_mng_init(CA_USB_SPEED_FS, USB_NUM)
         ca_usb_init(USB_NUM, &s_mgmt_cb)       mgmt registra i SUOI callback
    driver

ISR DWC2     callback mgmt     fan-out via ops ai servizi attivi:
  mng_on_setup     dispatch_standard (EP0: descriptor/address/config/...)
                  dispatch_class    (SETUP class-level     ops->on_setup)

ca_usb_dfu_ops (in caos/usb/usb_dfu.c):
  build_iface_desc     interface DFU (0xFE/0x01/0x01) + functional desc (18B)
  on_setup (class)     DFU_DETACH / GETSTATUS / GETSTATE / CLRSTATUS / ABORT
  on_configure         NULL (no data EP to open)

App (example_57):
  dfu_detached()      da on_setup(DFU_DETACH)     set s_detach_req
  task_usb: polling s_detach_req     stampa "DFU_DETACH ricevuto"
```

The DFU service does not open endpoint: `on_configure`, `on_rx`, `on_tx_done` they are `NULL`. The entire protocol lives in `on_setup` (control transfer on EP0).

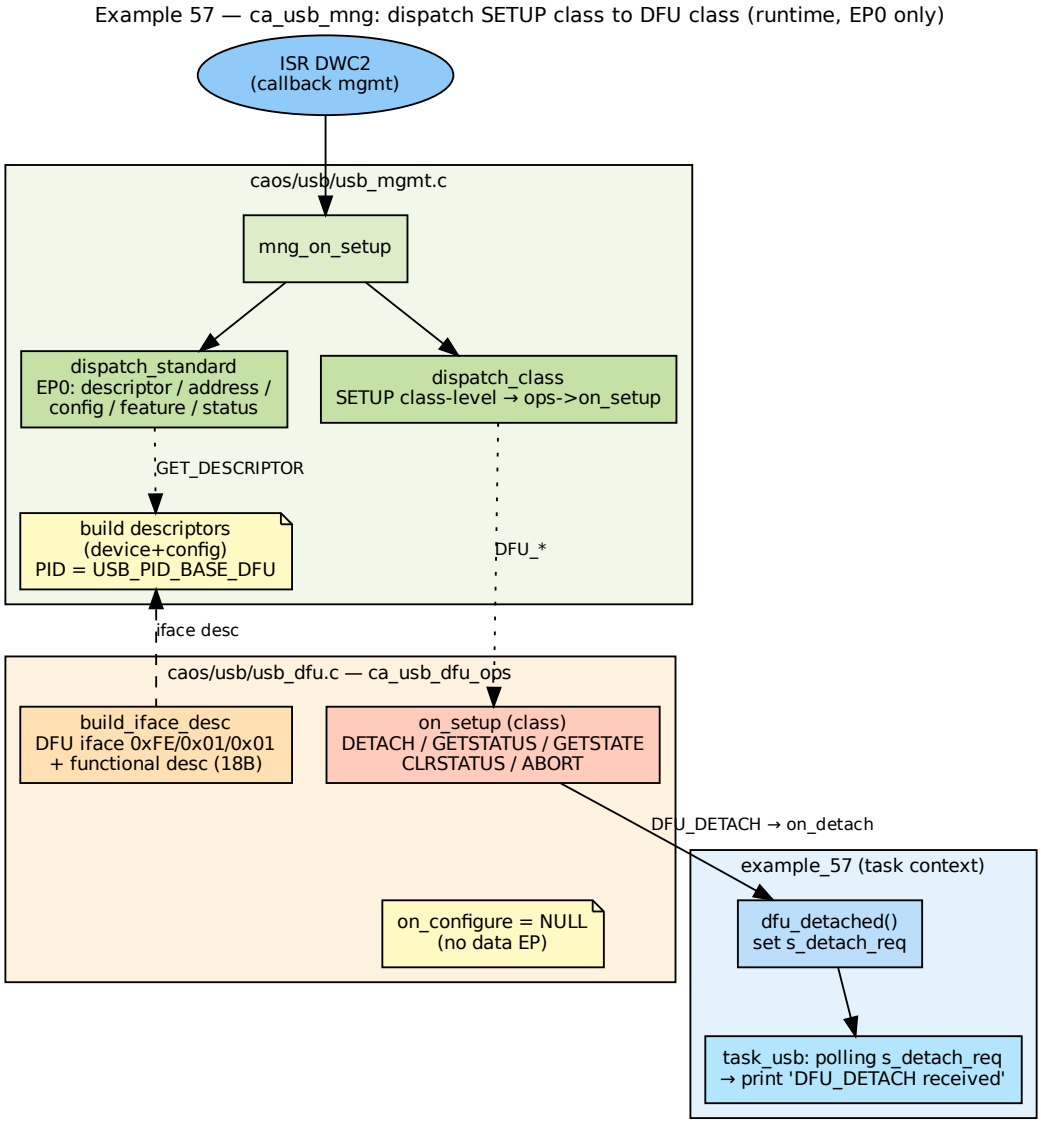


Figure 80.1: Dispatch SETUP class DFU (ex57)

Source: `doc/diagrams/ex57_dfu_runtime.dot` *(regenerated from* `make diagrams`*).*

Task	Priority	Function
`usb`	LEVEL6	init stack + polling `DFU_DETACH` (prints the host request)
`hb`	LEVEL4	toggle LED heartbeat every 500 ms

80.1.5 Class configuration (D20: persistent lifetime)

```
static const ca_usb_dfu_cfg_t s_dfu = {
    .header            = { CA_USB_TYPE_DFU, &ca_usb_dfu_ops },
```

```
    .detach_timeout_ms = 1000,
    .transfer_size     = 1024,
    .attributes        = USB_DFU_ATTR_CAN_DNLOAD | USB_DFU_ATTR_WILL_DETACH,
    .on_detach         = dfu_detached,
};
```

The cfg **must** be `static` (ideally `static const`): the management preserves the pointer without copying it. The first field `header` (tag + ops) must remain at the head of the struct (D19).

`detach_timeout_ms`/`transfer_size`/`attributes` end up in the **DFU functional descriptor** (9B). `PID = USB_PID_BASE_DFU = 0x0040` — the management sums the contributions of all active services (composite PID, D14).

80.1.6 Handled DFU Requests (DFU 1.1)

Request	Code	`bmRequestType`	Action
`DFU_DETACH`	0x00	0x21 (host→dev)	ACK ZLP + `on_detach()`; state → `appDETACH`
`DFU_GETSTATUS`	0x03	0xA1 (dev→host)	6B: `bStatus=OK`, `bwPollTimeout=0`, `bState`
`DFU_GETSTATE`	0x05	0xA1 (dev→host)	1B: `bState`
`DFU_CLRSTATUS`	0x04	0x21 (host→dev)	ACK ZLP; state → `appIDLE`
`DFU_ABORT`	0x06	0x21 (host→dev)	ACK ZLP; state → `appIDLE`

`DFU_DNLOAD`/`DFU_UPLOAD` are not handled at runtime (require switching to DFU mode) → the mgmt stalls on EP0.

80.1.7 CLI `usb`

```
usb status                  # state + detach_req flag
usb start | stop            # connect / disconnect (the host re-enumerates)
usb add | remove            # registra/rimuove la classe DFU a runtime
```

80.1.8 Parameters

Define	H753	U575	F407
`USB_NUM`	1	0	0
LED heartbeat	PB7	PC7	PA6

Requirements `os_config.h`: `EN_USB = 1`, `EN_USB_DFU = 1`.

80.1.9 Expected UART Output

```
[USB] mng init...
[USB] D+ pull-up on     plug cable (DFU runtime, dfu-util -l)
```

When the host sends `DFU_DETACH` (e.g., `dfu-util -e`):

```
[USB] DFU_DETACH received     the app would switch to DFU mode (here: report
    only)
```

80.1.10 Test from PC

First time on a new machine: the udev rule

Without it, udev leaves the device node owned by root and **not writable**:

```
crw-rw-r-- 1 root root 189, 77 /dev/bus/usb/001/078
```

Readable yes, writable no — and that is exactly the trap. `lsusb` reads and that's it, so the device **compares** and seems fine; but `dfu-util` must *open and claim* the interface, and fails one step later:

```
dfu-util: Cannot open DFU device ca01:0040
dfu-util: No DFU capable USB device available
```

The rule is versioned in the repository:

```
sudo cp tools/99-caos-usb.rules /etc/udev/rules.d/
sudo udevadm control --reload-rules && sudo udevadm trigger
```

A board that has already been enumerated retains the old permissions until it re-enumerates: unplug and reattach the cable, or let the next reflash handle it.

The commands

```
lsusb                          # VID=0xCA01 PID=0x0040
dfu-util -l                    # elenca l'interface DFU in runtime:
                               #   Found Runtime: [ca01:0040] ... name="DFU"
dfu-util -e                    # send DFU_DETACH     on_detach on the device
```

Nothing `sudo`: with the udev rule installed the example runs as a normal user, and this is what keeps ex57 in **class C** on the test bench.

`dfu-util -l` recognizes the interface (class 0xFE/sub 0x01); `dfu-util -e` emits `DFU_DETACH`, which on the device triggers `on_detach()` (here only UART signaling; in a real firmware: no-init flag + reset to bootloader).

80.1.11 File structure

File	Description
`caos/usb/inc/usb_mgmt.h`	API management (init/add/remove/start/stop)
`caos/usb/src/usb_mgmt.c`	composite descriptors + SETUP dispatch
`caos/usb/inc/usb_dfu.h`	cfg `ca_usb_dfu_cfg_t` + DFU attributes
`caos/usb/src/usb_dfu.c`	`ca_usb_dfu_ops` (DFU runtime class_ops)
`hw/f4xx/drivers/usb_f4xx.c`	DWC2 slave FIFO (F407/F429/F446)
`examples/example_57/ca_os_ex57.c`	mgmt example + DFU runtime

Chapter 81

FatFS on SD Card: SDIO Driver with DMA

81.1 Example 58 — FatFS on SD card: SDIO driver with DMA

81.1.1 Description

Example 58 brings a complete FAT32 filesystem on microSD through the `ca_sdio` driver and the **FatFS** library (`external/fatfs`), exposed by an interactive CLI over USB CDC. It is the first example with mass storage managed by a filesystem (in contrast to `sdtest`, which remains available as raw low-level test within the example itself).

CLI commands (prefix `fat`). Volume, file and directory commands come from the **FAT service** of the kernel (`caos/drivers/src/fat_service.c`, `EN_FAT_SERVICE` + `EN_CLI_FAT`), reusable by any application; `blob`, `sdtest`, `test_all` and `log` are test tools and remain in the example:

```
fat fdisk [<MB>]                    partiziona MBR soltanto (default 64MB)
fat mkfs                            formats as FAT32 the partition created by
    fdisk
fat mount                           monta volume, stampa fs_type/size/free
fat umount                          smonta
fat free                            total/free/used space
fat pwd                             current directory
fat cd <dir>                        cambia directory
fat ls [dir]                        lists the current directory or <dir>
fat mkdir <dir>                      crea directory
fat rmdir <dir>                      rimuove directory vuota
fat touch <file>                     creates an empty file (error if it exists)
fat rm <file>                        cancella file
fat cat <file>                       reads into dynamic memory, hexdump, frees
fat blob <file> <n> <B|K|M>          scrive n byte/KB/MB (pattern 0x00..0xFF
    ciclico)
fat sdtest <lba> [<n>]               test raw driver  w r i t e readverify  (
    DISTRUTTIVO)
fat test_all                         sequenza completa non presidiata (DISTRUTTIVA
    )
fat log                              logger counters
inject <Pn> <delay> <hold>           pin to input mid-transfer (C12 = CK)
```

From `mount` onward, **every line printed to stdout is also written to file** on the card (`<uptime>_cmd_log.log`): see "Log of stdout to file". The log is **for the example, not the service**: it hooks into the service events (`ca_fat_service_set_event_cb`: mounted, unmounting, destructive command) and shares its lock (`ca_fat_lock`/`ca_fat_trylock`).

Tested hardware: **STM32F4VE** (F407, SDIO onboard via generic DMA), **Nucleo-H563**, **Nucleo-H753** and **NUCLEO-U575ZI-Q** (SDMMC v2 with IDMA, microSD on Zio CN8 adapter) — 4-bit bus, console on UART, heartbeat LED.

81.1.2 Hardware

Signal	Pin	Notes
SDIO D0	PC8	AF12, pull-up
SDIO D1	PC9	AF12, pull-up
SDIO D2	PC10	AF12, pull-up
SDIO D3	PC11	AF12, pull-up
SDIO CLK	PC12	AF12, no pull
SDIO CMD	PD2	AF12, pull-up
Heartbeat LED	**board-dependent** (see below)	toggle 500 ms
Console	Debug UART	`STDOUT_ON_UART=1`, `STDIN_ON_UART=1`

The SDMMC pins above are for the F4VE. The **heartbeat LED**, however, varies by board, and the example selects it with a `#if MCU_ID` (same mapping as ex18):

Board	LED	Pin
STM32F4VE (F407)	D2 onboard	PA6
NUCLEO-U575ZI-Q	Green LD1	**PC7**
Nucleo-144 (H753, H563, F746, F429, F446, H747)	Green LD1	**PB0**

PA6 is **not an LED** on any Nucleo-144: it's just a Zio pin. While it was hardcoded, the heartbeat remained off on those boards while the console continued to respond — the same issue seen in ex30 and ex31. A board not mapped by `#error` at compile time instead runs with silent LED.

Data path on **DMA2 Stream3 Channel4** (fixed SDIO mapping on F4xx), reserved for the generic DMA allocator at driver startup.

> **Why `fdisk` and `mkfs` are separate commands.** Partitioning and formatting are two distinct operations, and keeping them in one command means an error can only be attributed to the whole: `fdisk` succeeded and `mkfs` failed, but the user saw only one failed command. Separated, each responds on its own — and one can reformat without repartitioning.
>
> Tested HW on STM32F4VE: `fat fdisk` + `mkfs` + `mount` + `blob` + `ls` + `sdtest` all OK.

81.1.3 os_config.h Configuration

```
#define EN_SDIO 1
#define EN_FAT  1
#define EN_FAT_SERVICE 1    /* FAT service: ca_fat_* + CLI node */
#define EN_CLI_FAT 1        /* the service's 'fat' command */
#define EN_DMA  1           /* required by the F4xx SDIO driver (DMA data path)
    */
```

81.1.4 Architecture

The example relies on the storage subsystem of caOS (FatFS + SDIO driver), documented in detail in the chapter "FatFS su caOS: SDIO driver" (three-level architecture, redirect `ffconf.h` →`caos_ffconf.h` and ODR motivation, glue `fatfs_diskio.c`, HW driver F4xx with DMA, robustness notes). Here only the specific aspects of the example application are reported: static CLI context and commands.

Static FAT Context

> **As of 10/02/2026** the context is split: `FATFS`, the 4 KB work buffer, and the command objects are in the service (`fat_svc_t` in `fat_service.c`, same alignment and same reasons); the example keeps only the read-back of `sdtest` in `ex58_ctx_t`, the `FIL` of `blob` and the logger status. `blob` and `sdtest` borrow the work buffer of the service with `ca_fat_work_buf()`, under `ca_fat_lock()`. The structure below is the original one, before the separation.

The caOS task stack lives in **CCM** (scheduler constraint: not resizable, not DMA-capable), while FatFS with LFN requires large buffers (`FIL`/`DIR`/`FILINFO` ~1.5KB, work buffer `mkfs` 4KB). The example collects everything in a `fat_ctx_t` structure (~7KB) **statically**, aligned to 32 bytes, zeroed in `ca_os_configure()`:

```
typedef struct {
    FATFS   fs;
    bool    mounted;
    u8      work_tx[4096];          /* mkfs/fdisk work buffer (TX) */
    u8      work_rx[512];           /* sdtest read-back (RX)       */
    FIL     fil;
    DIR     dj;
    FILINFO fno;
    char    path[FF_LFN_BUF + 4];
    u8      pattern[256];           /* blob pattern 0x00..0xFF */
    char    sub[32], a1[FF_LFN_BUF + 2], a2[16], a3[4];  /* token CLI */
    FIL     logf;                   /* log: FIL separate from fil */
    bool    logging;
    char    logname[32];
    u32     log_lines, log_lost;
} fat_ctx_t;
```

> **Why static and not on heap.** It's a unique context, of fixed size, that lives as long as the program: the heap brings no advantage and adds failure modes — a silent NULL on full heap, and especially a **placement that changes from board to board**. Static ends up in `.bss` at a known address, which is what the SDMMC IDMA needs: aligned and always reachable. 32-byte alignment avoids every transfer going through the driver's bounce buffer. `ex2` continues to use `ca_memory_allocate()` on purpose (heap is its argument), and `ex53`/`ex56` allocate ramdisk in MB, where a static wouldn't make sense.

CLI commands use `g_ctx->...` instead of local variables: the CLI task stack stays lightweight.

81.1.5 stdout log to file

`fat mount` installs a **repeater** on stdout (`ca_stdout_set_repeater()`): from that moment on, every line printed by anyone is also appended to `<uptime>_cmd_log.log` on the card. `fat umount` closes

the file and uninstalls the repeater, reporting how many lines were written and how many were lost.

The uptime-based name gives a unique file for each boot without needing an RTC, and the files sort chronologically by themselves.

The problem: FatFS is not reentrant

This is the part that makes the example interesting, and also the reason why the code is not trivial. The repeater runs in the **stdout drain thread**, while the commands `fat` run in the **CLI thread**: they are two different tasks. And caOS compiles FatFS with `FF_FS_REENTRANT=0` (`external/caos_fatfs/caos_ffconf.h`), i.e. **without any internal locking**.

Two threads inside FatFS at the same time do not give a clean error: **they corrupt the volume**. And it would have happened at the first `fat blob big 10 M` — 44 seconds into `f_write` — if in the meantime something had printed a single line.

Two **guards** are needed, and they cover different faults:

Guard	What it prevents	Why the other is not enough
mutex	the two threads inside FatFS running together	doesn't stop recursion
flag	the repeater re-entering itself	a mutex is recursive for tasks

The mutex serializes CLI and stdout threads. The repeater takes it with `ca_mutex_trylock`, **never** `ca_mutex_lock`: blocking there would freeze the console for the entire duration of a `blob`, the stdout queue would fill up and lines would be lost anyway — but with a frozen system instead of just one line missing. A rejected lock means "a fat command is in progress": the line is counted and discarded, which is the fair compromise. **The flag** stops the recursion, which the mutex cannot: a mutex is recursive for tasks, so the drain thread would happily re-enter itself. Anything printed *while* we are inside `f_write` — a FatFS error, a line of SDIO driver diagnostics — would re-enter the repeater and call `f_write`, infinitely. Those lines are lost by design: they are the log of the failing logger, and losing them is the price of not blocking the console.

Destructive commands

`fdisk`, `mkfs` and `sdtest` rewrite the sectors on which the open log file lives: they stop the logger **before** touching the card, and say so. Silently losing the log would be worse than a noisy line.

Single entry and exit

The command dispatcher is structured with **a single** `fat_enter()` **and a single** `fat_leave()`. With a `return` per branch — and there are ten branches — a forgotten unlock would silently block the logger **forever**: the defect would not cause a compilation error and would only manifest as "the log has stopped writing".

Declared limits

- During a `fat` command, lines are **discarded**, not queued
- `f_sync` every 16 lines: a reset loses at most the last
- CLI command output **goes to the log** when the console is on UART (`STDOUT_ON_UART=1`), because in that case `CLI_PRINTF_N` expands to `ca_stdout_printf_n` and goes through the same queue. On a separate transport (SSH, HTTP) it would use the CLI sender and not go through there

81.1.6 `fat cat` — the trip through the heap

`fat cat <file>` reads the file into **dynamic memory**, does a hexdump of it and frees it. It is the only example command that goes through the allocator: all others work on the static context on purpose (see above). It therefore also serves as a test that the heap can handle a real load, with allocation and release of blocks of arbitrary size.

The ceiling on size is calculated **at runtime** by `ca_memory_get_avail_memory()`, with a 2 KB margin, instead of being a constant: the relevant limit is the one the allocator can actually satisfy, and a fixed value in the code would go out of sync with the configuration. A file too large is rejected by reporting the requested size and available memory, instead of letting the allocation fail.

> **Beware of `ca_hexdump`**: the second argument is a **byte count**, despite being declared `u16 lines`. The name lies; callers agree on bytes (`cli.c` passes `lines*16`). It is precisely this command that also surfaced a latent defect in the function: the ASCII column was always reading 16 fixed bytes, so on a shorter final line it would go **beyond the caller's buffer**. Invisible until all callers passed multiples of 16; fixed on 08/24. Perform `cat` on the active log file works (uses a second handle), but returns the file as it was at the last `f_sync` — up to 16 lines back. And every line that the dump prints is then appended to that log: the dump describes a file that grows while it is being read.

81.1.7 Expected output (CLI, HW F4VE)

```
fat fdisk
[fat] fdisk 64 MB (131072 settori)...
[fat] fdisk OK - now 'fat mkfs' to format
fat mkfs
[fat] mkfs (attendere)...
[fat] mkfs OK     2370 ms     volume ready
fat mount
[fat] mount OK  fs=FAT32  total=63519 KB  free=63519 KB
fat blob big 2 M
[fat] blob big  2097152 byte scritti
fat sdtest 140000 64
[sdtest] 64 blocchi, 0 word errate     OK
```

The time of `mkfs` (2.37s on F407) reflects multi-block write via `CMD25`; the same formatting with only single-block writes (`CMD24` repeated) measured ~8.3s — chunk writes of 8 blocks with `ACMD23` pre-erase are about 3.5× faster. On H-family with IDMA (H563/H753) formatting drops to ~0.5s. The above output is from F4VE; on H563 the `mkfs` completes in ~478 ms and single-block write remains ~235 KB/s (physical limit of card programming, not the driver — valid for all families).

`fat sdtest` is **destructive**: writes and reads directly via `ca_sdio_write_block`/`ca_sdio_read_block`, bypassing FatFS. Should be used on LBA outside the mounted volume (e.g. beyond the end of the FAT partition) to avoid corrupting the filesystem.

> The source comment says *"use LBA outside the FAT volume"*, but the code **does not verify it**: `fat sdtest 0 2000` accepts and rewrites MBR and boot sector. The damage is **latent** — as long as the volume remains mounted, FatFS works on structures already in RAM and `fat ls` continues to function; on first remount (or reset) `FR_NO_FILESYSTEM` arrives. It is the same pattern as `flash test 0` before the `_etext` guard, with much milder consequences: a card to reformat, not a board to recover with a probe.

File log and cat (HW U575)

```
fat mount
[fat] mount OK  fs=FAT32  total=63519 KB  free=63496 KB
[log] stdout -> '8767_cmd_log.log'
fat ls
  [DIR]  test
       1272  8767_cmd_log.log
[fat] 2 elementi
fat cat 8767_cmd_log.log
[fat] cat '8767_cmd_log.log': 1272 byte @ 0x2009CAC8
0x2009CAC8  5b 66 61 74 5d 20 6d 6f 75 6e 74 20 4f 4b 20 20 |[fat] mount OK  |
0x2009CAD8  66 73 3d 46 41 54 33 32 20 20 74 6f 74 61 6c 3d |fs=FAT32  total=|
...
fat umount
[log] closed '8767_cmd_log.log' - 42 lines, 0 lost
[fat] smontato
```

The file can also be read from a PC by extracting the card: it is plain text terminated `\r\n`.

81.1.8 Fault injection: `inject`

`inject <Pn> <delay_ms> <hold_ms>` arms a task that, after the delay, puts the pin into input mode for the given time and then restores its mode (the alternate function registers are not touched). It tests what the driver does when a transfer does not finish, without pulling wires:

```
inject C12 2000 500           # SDIO CK removed after 2 s, for 500 ms
fat sdtest 5200000 2048       # meanwhile, a 2048-block write
```

Choose a pin the **MCU drives**: CK yes, D0-D3 and CMD no, since the card drives them too. The delay starts at the `inject` command, and the next command arrives a few hundred ms later: with a long write (2048 blocks, about 4 s) the fault surely lands inside it.

`sdtest` writes a fixed pattern per LBA: use it on sectors **never written** before, or old data reads back as if it were new.

A fault can leave the card in an abnormal state that only a power cycle clears: `inject` is a manual tool, not a bench phase.

Result on H753 (19/10/2026): with the specification's timeouts (FatFS chapter, "Timeouts") the write completes without errors, about 1 s longer, and the card stays healthy.

81.1.9 State per family HW

Tested HW on this example: **F407** (STM32F4VE, generic DMA), **H563** (@25MHz), **H753** (@24MHz) and **U575** (@24MHz) — the last three with SDMMC v2 and IDMA. The **F7** family (F746/F767) has no recorded bench round; its driver uses DMA2 Stream3 like the F4.

The porting of H753 and U575 (08/23) required fixing the same defect on both, with opposite symptoms: **the driver assumed a kernel clock that nobody configured**. On U575 the clock was completely **missing** (`SDMMCSEL`→ICLK, only turned on by the USB driver) and the state machine never completed; on H753 it was **240 MHz instead of 48** (PLL1Q with SYSCLK at 480), so the card was driven **5× over spec** — 2 MHz in init instead of the allowed 400 kHz, 120 MHz in transfer instead of 25. Short transactions survived, which is why `sdtest` always passed; `f_mkfs`, which writes the FAT, corrupted data. Now `CLKDIV` is **derived at runtime from the PLL read by RCC**, so a future change to the clock will not silently overclock the card again.

> **Diagnostic rule confirmed twice in a day**: active state machine and **zero error bits** means that the **clock** is missing or unstable, not that the wiring is wrong — incorrect wiring would instead cause a timeout.

Detailed driver status for all families: see chapter “FatFS su caOS: driver SDIO”.

81.1.10 File structure

File	Description
`examples/example_58/ca_os_ex58.c`	CLI `fat`, static context, heartbeat task

For the subsystem architecture (driver `ca_sdio`, glue diskio, build FatFS) see the chapter “FatFS su caOS: driver SDIO”.

Chapter 82

Power Manager: Frequency, Stop and Standby

82.1 Example 59 — Power manager (pwr)

Interactive example for validating the centralized power manager driver. **Silent by design**: no task prints autonomously, all output is on request via CLI — ideal for verifying low-power modes without console noise.

For the driver theory (low-power modes, wake sources, frequency scaling, family-specific quirks) see the "Centralized power manager" chapter (S02.13): this document only covers practical usage via CLI.

82.1.1 What it demonstrates

CLI commands (prefix `pwr`, plus `rtc info`):

```
pwr info           power state (sysclk, vos, standby_resume, peripherals)
pwr freq <1-99>    riduzione frequenza (HPRE) + retune tick/uart/can
pwr restore        ripristino frequenza nominale
pwr alarm <s>      test allarme RTC in RUN (nessuno stop)
pwr sleep <s>      WFI semplice, wake da allarme RTC
pwr stop <s>       STOP, wake da allarme RTC
pwr stopms <ms>    STOP, wake da LPTIM1 su LSI in millisecondi (U5xx/H5xx/H7xx/
    F7xx)
pwr dbg            dump of low-power registers (U5xx/F4xx only, requires
    EN_PWR_DBG)
pwr standby <s>    STANDBY, wake da allarme RTC = reset
rtc info           current RTC time
```

- `pwr info` — complete overview of the power state read from RCC/PWR: current/nominal SYSCLK, voltage scaling (VOS), standby source, list of peripherals with active clock grouped by bus. The list marks with `+` the peripherals turned on **after** init (residual clocks compared to the "cold" baseline) and with `off:  -` those turned off compared to it — useful for identifying residual clocks before/after a stop (see S02.13)
- `pwr freq <1-99>` / `pwr restore` — CPU frequency reduction/restoration via AHB prescaler with coordinated retuning of SysTick, UART, and bxCAN/FDCAN
- `pwr alarm <s>` — RTC alarm test in RUN (IRQ verification without stop)
- `pwr sleep <s>` — Simple WFI with wake from RTC alarm
- `pwr stop <s>` — STOP mode with wake from RTC alarm (seconds) and PLL re-init upon wakeup
- `pwr stopms <ms>` — STOP mode with wake from **LPTIM1** on LSI, in **milliseconds** (not seconds like `pwr stop`): fine resolution (down to ~31 µs), max 262144 ms. Useful for rapid

test cycles or short duty cycles where the 1s granularity of the RTC would be too coarse. Available on **U5xx/H5xx/H7xx/F7xx** (families with LPTIM block; F4xx and F1xx lack it → command rejected)
- `pwr dbg` — raw dump of relevant low-power registers (U5xx/F4xx only, requires `EN_PWR_DBG`)
- `pwr standby <s>` — STANDBY mode: wake from RTC alarm causes a reset; upon restart `pwr info` shows `standby_resume=yes`. With `<s>` at 0 (or omitted, depending on the build) the wake occurs instead from the **WKUP pin** (typically PA0) rather than the RTC alarm — convenient for manual push-button wake during tests, without having to wait for the timeout

82.1.2 Requirements

- Active CLI console (UART_DBG or USB CDC according to `os_config.h`)
- RTC initialized (the example uses `CA_RTC_CLK_LSI`, no crystal required)

82.1.3 Structure

A single silent `hbeat` task (sleep 1s in loop, toggle LED) verifies that the scheduler survives stop/wake cycles. All interaction takes place via the CLI.

Heartbeat task + per-MCU LED

Under `EN_PWR_DBG` the heartbeat task also drives an LED, one per board:

MCU	LED	Pin
F407 (F4VE)	D2	PA6, active low
U575 (Nucleo-U575ZI-Q)	Blue LD2	PB7
H563 / H753 / F746 (Nucleo-144)	Green LD1	PB0
F103 (Blue Pill)	—	PC13, active low

The blinking LED is the fastest way to distinguish "OS alive, only the console is dead" from "wake failed and the system is locked": if the LED continues to blink after a `pwr stop`/`pwr standby` but the console does not respond, the problem is in the console (e.g., USB re-enumeration in progress), not in the scheduler. The heartbeat task counter can still be inspected from the CLI with `task 1`.

82.1.4 Notes

- During STOP, the OS tick is stopped: task sleeps are delayed by the duration of the stop (documented driver behavior, see S02.13)
- With IWDG active, the timeout must exceed the duration of the stop
- **CDC console vs stop/standby**: on USB CDC console, STOP sends the USB link into suspend; upon wake, a host-side re-enumeration may be required before the wake-up print is visible. On **U5xx**, the USB session survives STOP (suspend→resume, no re-enumeration). For testing, the UART console remains cleaner, as the problem does not arise there. To avoid losing the wake-up print while the CDC is re-mounting, the CLI command inserts a `sleep 100ms` plus a newline immediately after returning from `ca_pwr_enter` .

82.1.5 HW Status

`pwr` has been validated via CDC console on **F407** and **U575** (14/07) and on **H563/H753** (13/07): STOP and STANDBY tested with positive results on all four families. It is the only

example dedicated to the power manager driver (removed from `example_2`/`example_4`, where it was initially drafted).

82.1.6 File structure

File	Description
`examples/example_59/ca_os_ex59.c`	CLI `pwr`/`rtc`, heartbeat task, per-MCU LED under `EN_PWR_DBG`

For the driver architecture (low-power modes, wake sources, coordinated frequency scaling, per-family state) see the “Centralized power manager” chapter (S02.13).

Chapter 83

PWM: LED Fade with User Button

83.1 Example 60 — PWM: LED fade with user button

Demonstration example of the **PWM** mode of the timer driver (`ca_timer_init_pwm` / `ca_timer_set_pwm`): the user LED on the board is controlled by a timer channel in PWM mode 1 and the brightness is controlled by the user button.

83.1.1 Behavior

- At boot the LED starts **on** (duty = `DUTY_MAX` = 1000‰).
- **While holding** the user button the duty decreases with a linear ramp; when the low threshold is reached the direction **inverts** and the duty increases: while the button is pressed the LED "breathes" between the two thresholds.
- **Releasing** the button freezes the brightness at the current value.
- Each inversion is printed to stdout (`ex60: min 150/1000, salgo`).

83.1.2 Thresholds

Below 15-20% duty the LED is already visually off: the ramp inversion would happen "in the dark" and the button would seem unresponsive. The thresholds are constants defined at the top of the file:

```
#define PWM_FREQ_HZ   1000U   /* 1 kHz: no visible flicker */
#define DUTY_MAX      1000U   /* high inversion threshold (   ) */
#define DUTY_MIN       150U   /* low threshold: below ~15% the LED is already
    off */
#define DUTY_STEP        5U   /* passo per tick     sweep completo in ~1.7 s */
#define TICK_MS         10U   /* button/ramp polling period */
```

83.1.3 Board mapping

Board	LED	PWM Channel	AF	Button
F4VE	D2 = PA6 (**active low**)	TIM3_CH1	AF2	K0 = PE4 (pressed = 0, pull-up)
NUCLEO-U575ZI-Q	LD2 blue = PB7	TIM4_CH2	AF2	B1 = PC13 (pressed = 1)
NUCLEO-H563ZI	LD1 green = PB0	TIM3_CH3	AF2	B1 = PC13 (pressed = 1)
NUCLEO-H753ZI	LD1 green = PB0	TIM3_CH3	AF2	B1 = PC13 (pressed = 1)

On the F4VE the LED is **active low** (anode at 3.3V): the example inverts the duty applied (`1000 - duty`) under `#define LED_ACTIVE_LOW`, so the logic remains identical on all boards.

83.1.4 Key points of the code

1. **Caller-owned pin** (caOS pattern, like the encoder in ex25):

```
ca_gpio_configure(LED_PORT, LED_PIN, CA_GPIO_MODE_AF,
                  CA_GPIO_OTYPE_PP, CA_GPIO_SPEED_LOW, CA_GPIO_NOPULL, LED_AF);
```

2. **Init and start** — the frequency is per-timer, the duty is per-channel:

```
ca_timer_init_pwm(PWM_TIM, PWM_CH, PWM_FREQ_HZ);
ca_timer_set_pwm (PWM_TIM, PWM_CH, duty);   /* 0-1000     */
ca_timer_start   (PWM_TIM);
```

3. **Duty change glitch-free**: `ca_timer_set_pwm` writes the CCR with preload enabled — the new duty is loaded at the next PWM period, without spurious pulses. It can be called at any rate (here every 10 ms).

4. **Polling, not EXTI**: the button is read every `TICK_MS` by the fade task — simpler than an EXTI ISR and the polling period acts as natural debounce.

83.1.5 Test from CLI

The CLI node `timer` is registered: the duty can be forced from console bypassing the ramp:

```
timer info                TIM3  pwm  cnt=0x...
timer pwm 3 1 500         TIM3 ch1 duty=500/1000
```

83.1.6 Requirements

```
#define EN_TIMER  1
```

No external connection: LED and button are on the board.

Chapter 84

LPTIM: Low-Power Blink

84.1 Example 61 — LPTIM: blink low-power (periodic + one-shot)

Demonstrative example of the **LPTIM** driver (`ca_lptim_init_periodic` / `ca_lptim_init_oneshot`): the user LED blinks on LPTIM1 with kernel clock **LSI 32 kHz**, independent from system clocks. The counter remains active even in Stop: it is the basis for the future tickless idle.

84.1.1 Behavior

The task alternates in loop the phases:

- **Phase A** — periodic 500 ms × 20 toggle (~10 s, slow blink);
- **pause** — one-shot 2000 ms with LED off: the timer generates a single interrupt and stops by itself (SNGSTRT);
- **Phase B** — periodic 100 ms × 50 toggle (~5 s, fast blink);
- **Phase C** (if `EN_PWR`) — periodic 500 ms with **wake=true**: the MCU enters **STOP** and the LPTIM wakes it up at every ARR match — the LED blinks with the CPU stopped (10 toggle), then execution resumes. During Phase C the CDC console is suspended.

The LED toggle occurs in the **ISR callback** (ARR match); the task counts the fires and orchestrates the transitions with `stop` + `deinit` + re-init. Each transition is printed to stdout (`ex61: fase A - periodic 500 ms x 20`).

84.1.2 LPTIM Driver

Times in **milliseconds** (1..262144), automatic prescaler /1../128 for maximum resolution; precision limited by LSI tolerance (±5%).

```
ca_lptim_init_periodic(1, 500, cb, wake);   /* callback on every ARR match */
ca_lptim_init_oneshot (1, 2000, cb, wake);  /* callback once, then stops */
ca_lptim_start (1);
ca_lptim_stop  (1);                         /* disable: CNT restarts from 0 */
ca_lptim_deinit(1);
```

With `wake=true` the instance becomes a wakeup source for the power manager: `ca_pwr_enter(CA_PWR_MODE_STOP, 0, 0)` accepts the input in Stop even without RTC/pin (the pwr interrogates `ca_lptim_wake_active()` and re-enables the LPTIM NVIC by itself; on F7/H7 the EXTI line 23/47 is armed by the driver at start). The kernel clock LSI keeps the counter alive during Stop.

Two IP variants handled in family files:

Family	IP	Notes
F7xx / H7xx	v1	`IER` is writable only when disabled
H5xx / U5xx	v2	`DIER` from enabled, sync on `DIEROK`
F1xx / F4xx	—	LPTIM not present: init → `false`

Note: LPTIM1 is shared with `ca_pwr_enter()` stopms, which arms it on its own: with an active driver instance the pwr **rejects** stopms (guard in `ca_pwr_enter`).

84.1.3 Board mapping

Board	LED
NUCLEO-U575ZI-Q	LD2 blue = PB7
NUCLEO-H563ZI / H753ZI / F746ZG	LD1 green = PB0
F4VE	LPTIM absent on F407: message and exit

84.1.4 Key points of the code

1. **Runtime probe** — on F1/F4 the init fails cleanly:

```
if (!ca_lptim_init_oneshot(1, 100U, NULL, false)) {
    ca_stdout_put_n("ex61: LPTIM not present on this MCU");
    return 1;
}
ca_lptim_deinit(1);
```

2. **One-shot autoterminante**: on fire the dispatch stops the HW and releases the slot *before* calling the callback — inside the callback you can already re-init the same LPTIM.

3. **Callback in contesto ISR**: only short operations (here `ca_gpio_toggle` + increment of a counter `volatile`); the phase logic is in the task.

4. **Blink in STOP** (phase C) — the task re-enters Stop after every wake:

```
ca_lptim_init_periodic(1, 500, cb_blink, true);   /* wake-armed */
ca_lptim_start(1);
while (s_fires < STOP_TOGGLES)
    ca_pwr_enter(CA_PWR_MODE_STOP, 0, 0);   /* the LPTIM wakes it */
```

When waking up, the pwr re-initializes the clock with IRQ disabled, then the LPTIM ISR executes the callback (toggle): the power consumption between one toggle and the other is that of Stop, not Run.

84.1.5 Test from CLI

```
lptim info              LPTIM1  periodic  cnt=0x1F40
lptim stop 1            LPTIM1 stopped (blink congelato)
lptim start 1           LPTIM1 started
```

84.1.6 Requirements

```
#define EN_LPTIM  1
```

No external link: the LED is onboard.

Chapter 85

LPUART: Low-Power Console with Wake from Stop

85.1 Example 62 — LPUART: low-power console with wake-up from Stop

Demonstration example of the **LPUART** driver (`ca_lpuart_init` / `ca_lpuart_put`): the system CLI console migrates to **LPUART1** (115200 8N1) and, thanks to `wake=true`, **receiving a character wakes the MCU from Stop** mode of the power manager. This is the typical battery-powered scenario: MCU almost always in Stop, console responding to the first key press.

85.1.1 Configuration (os_config.h)

The LPUART console is a first-class option, like UART dbg and CDC:

```
#define STDOUT_ON_LPUART   1     /* STDOUT = ca_lpuart_put */
#define STDIN_ON_LPUART    1     /* STDIN  = ca_cli_data_from_isr */
#define LPUART_STD_BAUD    115200U
#define LPUART_STD_WAKE    1     /* console armed as wake source from Sleep/Stop
    */
```

With both active, `EVAL_LPUART_STD` expands to `ca_lpuart_std_cli()` called by `ca_os_init` (before `ca_os_configure`): LPUART init with `on_rx` → `ca_cli_rx_byte` and `wake=LPUART_STD_WAKE`. The example uses the standard pattern (`ca_stdout_set_sender(STDOUT)`), no runtime switch. For testing, turn off the other consoles (`STDOUT_ON_USB_CDC 0`, `STDIN_ON_USB_CDC 0`, and possibly `EN_USB 0`).

85.1.2 Behavior

- The system console is the terminal connected to LPUART1.
- A task prints the uptime every 15 s.
- From the LPUART terminal:

```
pwr stop 60
```

The MCU enters STOP mode (RTC alarm set to a 60 s safety timeout) — by pressing **any key**, the MCU wakes up immediately: the byte is not lost, it is delivered to the CLI after wake-up (a RETURN is harmless).

85.1.3 LPUART Driver

```
ca_lpuart_init(115200, on_rx, true);   /* wake=true     wakes from Stop/Sleep */
ca_lpuart_put(msg);                    /* sender-compatible: STDOUT/CLI     */
ca_lpuart_deinit();
```

Driver choices:

- **Kernel clock HSI16 + HSIKERON**: the baudrate is independent of the sysclk — no retune after `pwr freq` or wake from Stop (the classic dbg UART instead dies with clock changes) — and the oscillator remains alive in Stop: reception is reliable at 115200 even as a wake event.
- **TX polling, RX interrupt-driven** byte-by-byte (FIFO off, RXNEIE).
- With `wake=true`, **UESM** is set and the pwr block automatically re-enables the LPUART NVIC at Stop (`ca_lpuart_wake_active()`, same pattern as LPTIM); on H7, the driver also arms the **EXTI 34** line.

Family	Notes
U5xx	Autonomous APB3 (SMEN+AMEN), direct NVIC wake
H5xx	APB3 (LPEN), direct NVIC wake
H7xx	D3/APB4 (LPEN+D3AMR), wake via EXTI 34 (IMR2)
F1xx / F4xx / F7xx	LPUART not present: init → `false`

85.1.4 Board mapping (external USB-UART adapter, 3.3V)

Board	TX	RX	AF
NUCLEO-U575ZI-Q	PA2	PA3	AF8
NUCLEO-H563ZI	PB6	PB7	AF8
NUCLEO-H753ZI	PA9	PA10	AF3 (do not use together with USB device)

85.1.5 Test without adapter: ST-LINK VCP via jumpers

You can reuse the onboard ST-LINK USB-UART converter: with USB off and the dbg UART **not** initialized, the MCU pins connected to the VCP remain in **high impedance** (GPIOs at reset are in analog mode), so two jumpers are enough — TX line to TX, RX line to RX (the TX/RX crossover is already done on the board between ST-LINK and MCU):

Board	VCP (MCU side)	Jumpers
U575	USART1 PA9/PA10	PA9–PA2 (TX), PA10–PA3 (RX)
H563	USART3 PD8/PD9	PD8–PB6 (TX), PD9–PB7 (RX)
H753	USART3 PD8/PD9	PD8–PA9 (TX), PD9–PA10 (RX)

Warning: disable the board's debug UART — `ENABLE_DBG_UART 0` in `hw/<family>/<board>/objmap.h` (it is a board property, not an os_config one): `ca_os_init` initializes it regardless of `STDOUT_ON_UART`, and its push-pull TX would conflict with the LPUART TX through the jumper. Verified on U575: with USART1 active on PA9/PA10, the LPUART console is silent.

85.1.6 Requirements

```
#define EN_LPUART 1      /* BEFORE the STDOUT block in os_config.h */
#define EN_PWR    1      /* CLI "pwr" for the stop/sleep demo */
#define EN_RTC    1      /* safety alarm for pwr stop <s> */
```

The example calls `ca_pwr_init()` and `ca_rtc_init(CA_RTC_CLK_LSI)` in `ca_os_configure`.

Terminal at 115200 8N1 (ST-LINK VCP with jumpers, or 3.3V USB-UART adapter with crossed TX/RX and GND).

Chapter 86

UART DMA: Variable-Length Protocol

86.1 Example 63 — UART DMA: variable-length protocol

Demonstration example of the **UART driver DMA mode** (`EN_UART_DMA`) in the most general and frequent case in practice: a protocol whose length **is not known a priori**. The header is decoded byte by byte in normal mode; once the length is known, the rest of the message arrives **entirely via DMA**, without any interrupt per byte. This is the reference scheme for framed protocols over serial (e.g., Ethernet-over-UART, extended Modbus RTU, inter-MCU link).

86.1.1 Configuration (os_config.h)

```
#define EN_UART_DMA    1      /* exclusive with ENABLE_UART (classic driver) */
#define EN_DMA         1      /* richiesto */
```

DMA mode is **mutually exclusive** with the classic UART driver (`#error` in os_config). The UART under test is the ST-LINK VCP one: in the board's objmap, `ENABLE_DBG_UART 0` must be set (TX contention) — the example monitor is on the USB CDC console.

86.1.2 The example protocol

```
[SOF 0xCA][type][len_lo][len_hi]   payload[len]   [crc_lo][crc_hi]
```

CRC16-CCITT (poly 0x1021, init 0xFFFF) on type+len+payload. The board responds with an echo containing `type|0x80` and the same payload (max 1024 bytes). Frames with incorrect CRC: counted, no echo.

86.1.3 Header-to-payload flow

1. **Header phase (normal mode)** — `ca_uart_init_dma()` leaves RX on interrupt: every byte arrives at the `on_rx` callback (ISR context), where a small state machine searches for the SOF and accumulates type+len. Bytes out of sync are discarded (counter `resync`).
2. **Switch to DMA — from the ISR itself** — upon `len`, `on_rx` calls `ca_uart_dma_recv( num, buf, len+2)`: RXNEIE off, DMA channel armed one-shot P2M on `buf`. Switching in the ISR of the last header byte guarantees staying within a byte-time (no overrun); any pending RXNE is drained by the DMA upon activation of DMAR.
3. **Payload phase** — the DMA fills the caller's buffer: zero CPU load until transfer-complete.

4. **Frame end (TC, counter at zero)** — the driver's TC callback restores header mode (DMAR off, RXNEIE on) and the channel semaphore wakes the task blocked in `ca_uart_dma_wait()`.
5. **Response** — the task verifies the CRC and transmits the echo with `ca_uart_dma_send()`: TX is also DMA one-shot, the task sleeps on the channel semaphore until TC (no busy waiting).

86.1.4 Driver API (EN_UART_DMA)

```
ca_uart_init_dma(num, baud, on_rx);   /* fase header: byte     on_rx (ISR) */
ca_uart_dma_recv(num, buf, len);      /* da ISR: arma RX one-shot        */
ca_uart_dma_wait(num, CA_WAIT_FOREVER); /* task: sleeps until end of frame */
ca_uart_dma_send(num, buf, len);      /* TX one-shot, task-blocking      */
ca_uart_dma_remaining(num);           /* byte mancanti (diagnosi)        */
ca_uart_dma_abort(num);               /* truncated frame recovery */
```

The **fixed-length** case is a trivial subset: `ca_uart_dma_recv()` is called immediately with the known size, without a header phase.

Under the hood, the driver uses the generic building blocks of the DMA framework: `ca_dma_set_request()` (peripheral request line, family bindings verified against reference manuals) and `ca_dma_set_callback()` (TC callback in ISR, new with this example).

86.1.5 Host test

```
python3 examples/example_63/host_test.py /dev/ttyACM0
```

Sequence: echo ping-pong on sizes 1→1024 B, frame with corrupted CRC (no echo expected), resync after 64 bytes of garbage, throughput on 50 frames of 1024 B. At 115200 8N1 the ping-pong saturates the line (~11 KiB/s round-trip measured on NUCLEO-U575).

CLI on CDC console:

```
u63 info        frame/byte/CRC err/resync + byte DMA mancanti
u63 abort       cancels a stuck reception (truncated frame)
```

Family	Notes
F1xx	fixed DMA mapping per channel (request ignored)
F4xx/F7xx	CHSEL per stream, bind RM0090/RM0385
H7xx	DMAMUX1; **frame buffer in RAM D2** (no DTCM)
H5xx/U5xx	GPDMA REQSEL; HW tested on NUCLEO-U575

Chapter 87

SPI DMA: Full-Duplex Loopback on a Single Board

87.1 Example 64 — SPI DMA: loopback full-duplex on single board

Demonstration example of the **SPI driver DMA mode** (`EN_SPI_DMA`). Unlike UART, SPI is **synchronous**: the master clocks exactly `len` bytes, so the **transfer size is known a priori**. No header phase is needed — the transfer of the desired length is armed directly and the task sleeps until completion, without byte interrupts nor busy waiting.

87.1.1 Configuration (os_config.h)

```
#define EN_SPI_DMA    1      /* exclusive with EN_SPI (classic driver) */
#define EN_DMA        1      /* richiesto */
```

`EN_SPI_DMA` is **mutually exclusive** with the SPI polling/IRQ driver (`#error` in os_config) and requires `EN_DMA`.

87.1.2 Hardware — 2-wire loopback

Single master, no slave: bridge MOSI to MISO so that every transmitted byte returns identical.

```
  SPI1:  PA7 (MOSI)              PA6 (MISO)      SCK PA5 free
```

> **NUCLEO-U575ZI-Q** — does not have Ethernet mounted, so PA5/PA6/PA7 are free and the **default** mapping (`.alternate_gpio = false`) is used. Loopback verified on the **Zio CN7**: **PA7 (MOSI, D11, CN7 pin 14) PA6 (MISO, D12, CN7 pin 12)**; SCK PA5 = D13 = CN7 pin 10 is not needed. Unlike the H563, here the Arduino pins of CN7 are actually wired (UM2861 Table 15).
>
> **STM32F4VE** (black board, F407) — no Ethernet: default PA5/PA6/PA7, loopback **PA7 (MOSI) PA6 (MISO)** on the side headers.

Boards with mounted Ethernet

On Nucleo-144, the PA7 pin is wired to the RMII (`RMII_CRS_DV`) and controlled by the PHY: it cannot be used as MOSI. On these boards, SPI must be opened with `.alternate_gpio = true`, which moves SPI1 to **PB3 (SCK) / PB4 (MISO) / PB5 (MOSI)**.

> **NUCLEO-H563ZI** — loopback verified: **CN12 pin 29 (PB5) CN12 pin 27 (PB4)**, on the morpho connector.

Do not use CN7: PB5 is not reachable from there. Pin 13 (D22) goes through solder bridge SB17, which is open by default; pin 14 (D11) does not carry the signal even though it is listed as PB5 in Table 21 of UM3115. In both cases, MISO reads `FF` constant and the test fails on all boards.

NUCLEO-H753ZI — here the driver already maps SPI1 to PA5 (SCK) / PA6 (MISO) / **PB5** (MOSI) in order to bypass PA7, so `.alternate_gpio` remains `false`. Loopback verified: **CN12 pin 29 (PB5) CN12 pin 13 (PA6)**.

Warning: MOSI falls on the same pin as the H563 (29), MISO **does not** — 13 and not 27, because the two boards use different mappings.

Also avoid CN7 here: pin 14 (D11) is switchable between PA7 and PB5 via SB33/SB35 and is wired to **PA7** by default (UM2407 Table 14, note 3). Also note that on this board `SPI_B` of the connector is **SPI3**, not SPI1.

In general, it is advisable to use the morpho connector on both boards: the Arduino pins go through solder bridges that the documentation does not always reliably report, and the symptom of an unconnected pin (`rx` constant at `FF`, errors on all boards) is indistinguishable from a driver fault.

87.1.3 The synchronous transfer

The core is a single blocking call:

```
u16 ca_spi_dma_xfer(u8 spi_num, const u8 *tx, u8 *rx, u16 len);
```

1. **prepare** — the peripheral is stopped (SPE=0 on IP v3), flags cleared and size set: on SPI v3 (H5/H7/U5) `len` goes to **TSIZE** (EOT after `len` frames); on v1/v2 (F1/F4/F7) only DMA channels count. `RXDMAEN` is enabled.
2. **arm** — both DMA channels are armed: RX in `P2M` (RXDR→buffer) and TX in `M2P` (buffer→TXDR), both at 8 bit.
3. **kick** — `TXDMAEN` is enabled and (on v3) `SPE+CSTART`: the transfer starts here. The DMA writes/reads at the pace of TXP/RXP requests.
4. **wait** — the task blocks on the channels' semaphores (first TX then RX: the last TX fetch always precedes the last RX store).
5. **finish** — the end of the last byte shift is awaited (`EOT` on v3, `BSY` on v1), flags are cleared and the idle master state is restored.

Mode bytes (`tx==NULL` → dummy 0xFF; `rx==NULL` → TX-only with discard of received) make the API useful also for pure write/read operations, in addition to full-duplex.

87.1.4 Driver API (EN_SPI_DMA)

```
ca_spi_init_dma(&cfg);                  /* like ca_spi_init + DMA channel bind */
ca_spi_dma_xfer(num, tx, rx, len);      /* synchronous full-duplex, blocking */
ca_spi_dma_write(num, buf, len);        /* TX-only (rx scartato)              */
ca_spi_dma_read(num, buf, len);         /* RX-only (tx dummy 0xFF)            */
```

Under the hood, the DMA framework bricks are used (`ca_dma_set_request()` with REQSEL/CHSEL checked against reference manuals, `ca_dma_copy_target()`, `ca_dma_wait()`).

87.1.5 Behavior

Nothing happens by itself. At boot, the card is idle; `e64 start` performs **a** pass through the sizes 1, 2, 16, 64, 128, 255 and 256 bytes, with pseudo-random payload (xorshift32), checks `rx == tx` bytes one by one and concludes with a verdict:

```
RUN RESULT : OK   (1/1)       every transfer came back identical
RUN RESULT : FAIL   (0/1)     almeno uno no
```

then it stops. Before it used to cycle from boot printing one line per transfer: there was no moment of silence to read the response of another command (the verdict of `wire`, on H563, did not appear at all) and no notion of "successful cycle", only counters always in motion.

Commands on the UART console:

```
e64 start        one round over all sizes, ends with RUN RESULT
e64 wire         MOSI-MISO jumper continuity on plain GPIO (no SPI, no DMA)
e64 info         trasferimenti OK/ERR + ultima taglia e byte errati
e64 regs         SPI registers (CR1/CR2/CFG1/CFG2/SR) and the two DMA channels'
e64 buf          first 16 bytes of tx and rx of the last transfer
```

`wire` drives MOSI as output and reads MISO with pull-down: a FAIL there means missing wire or wrong pins, not an SPI or DMA problem. It must be run first, with the board at rest.

`regs` and `buf` are diagnostic tools: the first shows whether the peripheral is really in master mode and if the channels have consumed the bytes (`BNDT`), the second distinguishes at a glance a wiring problem (constant `rx`) from a frame problem (`rx` misaligned with respect to `tx`).

87.1.6 DMA Portability by Family

Family	Notes
F1xx	fixed DMA mapping per channel (request ignored); SPI1/2, shared channels with USART
F4xx	SPI v1, CHSEL for stream (RM0090 tab.43/44)
F7xx	SPI v2 FIFO, 8-bit DMA on `&DR` with FRXTH=1 (RM0385)
H7xx	SPI v3 TSIZE/EOT, DMAMUX1; buffers in AXI-SRAM (`.bss`, no DTCM)

Family	Notes
H5xx/U5xx	SPI v3 TSIZE/EOT, GPDMA REQSEL (spi{1,2,3}_rx/tx_dma)

Note on H7 buffers: DMA1/DMA2 channels do not reach DTCM. On H753/H747, `.bss` is mapped in AXI-SRAM (0x24000000), reachable by DMA — thus, the static buffers in the example are valid without dedicated sections.

87.1.7 SPI v3 Erratum — CRCSIZE (H5/H7/U5)

On families with SPI v3, the last byte received in DMA can return **corrupted** even with CRC disabled. This is erratum **ES0499 §2.23.1** (STM32U575/585, with similar behavior on H5/H7): with `CRCEN=0`, `TSIZE > 0` and the `CRCSIZE < 7` field, the data of the last frame in RxFIFO is invalid. The reset value of `CRCSIZE` is 0, so this condition occurs by default.

The symptom observed on U575 was exactly one wrong byte **at the end of the transfer**, with a deterministic pattern `rx = tx & 0x03` (only the two lowest bits arrived). The fix, applied in the SPI init (master and slave) for the three v3 families, forces `CRCSIZE = 7` in `CFG1`. The DMA path does not rewrite `CFG1` from scratch, so the init setting is sufficient.

On H563/H753 the defect had not manifested: it is timing-dependent (PCLK/SCK ratio) and those clock pairs masked it. The fix `CRCSIZE = 7` covers them anyway.

87.1.8 Test Status (Hardware)

Board	SPI IP	Outcome
NUCLEO-U575ZI-Q	v3	all sizes (after CRCSIZE fix)
NUCLEO-H563ZI	v3	
NUCLEO-H753ZI	v3	
STM32F4VE (F407)	v1	(on first try)
NUCLEO-F746ZG	v2	not tested (hardware not available)

Chapter 88

SPI DMA: Master + Slave IRQ on Two Boards

88.1 Example 65 — SPI DMA master (with the slave from example 93)

Extends example 64 to the real case of **two boards**: this one acts as **DMA master**, the other as **IRQ slave** with example 93. The goal is to verify **interoperability** between the two paths on different silicon, under a continuous flow of data.

88.1.1 Connections: same position on both boards

With the default SPI1 mapping, every Nucleo-144 board has the three lines on the same **Arduino pins of CN7**, so the wires go straight from one board to the other:

Signal	Position (on both boards)	MCU Pin
SCK	**CN7-10** (D13)	PA5
MISO	**CN7-12** (D12)	PA6
MOSI	**CN7-14** (D11)	PA7, **PB5 on H7** (PA7 is Ethernet)
CS	**CN7-16** (D10)	PD14, on all (including H563)
GND	**CN7-8**	—

Exception: boards with **Ethernet on PA7** — **H563** (UM3115), **F767** and **F746** (UM1974 note 2: PA7 is D11 and, with JP6 closed, also `RMII_DV` of the PHY) — use the alternative mapping PB3/PB4/PB5, which arrives at the morpho in the same positions on both board families: **CN12-31/27/29**, GND **CN12-9**. The F446 (same board MB1137, without Ethernet) stays on CN7. CS remains on CN7-16 (PD14) even on the H563. `s65 wire` and `s93 wire` print this table, with the pins of the board they run on, before the measurements.

88.1.2 Hardware — reference pair H563 + H753

The reference configuration is **NUCLEO-H563ZI (master)** + **NUCLEO-H753ZI (slave)**, connected with 3 wires plus ground.

> **The wires do not connect matching pins.** The two boards use different mappings: the H563 has Ethernet on the default SPI1 pins and adopts the alternative mapping (`alternate_gpio = true` → PB3/PB4/PB5), while the H753 uses the default driver mapping for h7xx (PA5/PA6/PB5). Only MOSI happens to fall on the same pin by chance.

Signal	H563 — master	H753 — slave
SCK	PB3 → **CN12-31**	**CN12-11** ← PA5
MISO	PB4 → **CN12-27**	**CN12-13** ← PA6
MOSI	PB5 → **CN12-29**	**CN12-29** ← PB5
GND	**CN12-9**	**CN12-9**

All on the **morpho CN12** connector of both boards. Do not use the Arduino connectors: different pins are disconnected by open solder bridges from the factory, and the symptom (data constantly at `FF`) is indistinguishable from a driver fault — see the note in example 64. NSS software (`SSM=1` on both): the slave peripheral is always active. From 10/13, however, there is a **CS wire** (D10, PD14, CN7-16): the master pulls it low around every frame, and the slave, with an EXTI interrupt on the falling edge, calls `ca_spi_slave_sync()` at the beginning of every frame. Without it, a slave started while the master was transmitting would remain out of sync, or on SPI v3 (H5/H7/U5) blocked by an overrun never cleared: tested on H753, 0 frames received. CPOL=0/CPHA=0 (SPI mode 0) on both boards.

The role is the example flashed: **ex65** on the master board, **ex93** on the slave, on any pair of MCUs. It is advisable to put the slave on the faster board, because it receives **one byte per interrupt** and is the side that saturates first when the speed is increased.

88.1.3 Hardware — alternative pair U575 + F407

If you don't have the H563/H753 pair, the example has also been validated on **NUCLEO-U575ZI-Q (master) + STM32F4VE / F407 (slave)**. Here **neither** of the two boards has Ethernet which occupies PA5/PA6/PA7, so both use the **default** mapping (`alternate_gpio = false`) and — unlike the reference pair — **the pins are identical**.

Signal	U575 — master (Zio CN7)	F407 — slave
SCK	PA5 → **CN7-10** (D13)	PA5
MISO	PA6 → **CN7-12** (D12)	PA6
MOSI	PA7 → **CN7-14** (D11)	PA7
GND	GND	GND

All three wires plus ground are needed: from 10/13 the master **verifies the MISO** (see below), so without that wire every frame results in `miso=BAD`.

88.1.4 Why two examples (master 65, slave 93)

Until 10/13/2026, master and slave were a single example, with the role deduced from `MCU_ID` . However, the two roles require **mutually exclusive** configuration flags (`EN_SPI_DMA` for the master, `EN_SPI` for the slave, with `#error` if both are set), and one example has **a single** config: one of the two boards would always be compiled with the wrong flag and remain silent. For this reason, the example was split into two:

Role | Example | Config |
|———|– ——|———| | Master | **ex65** | `EN_SPI_DMA 1`, `EN_DMA 1` | | Slave | **ex93** | `EN_SPI 1` |

The role is now **which example is flashed**, on any pair of boards, and the configuration is no longer changed from one build to the next.

88.1.5 Flow

- **Master** — every 500 ms it composes a frame of `EX65_FRAME` (64) bytes: `byte[0]` = progressive counter, `byte[1..]` = pattern `(i + seq)`. It sends it with `ca_spi_dma_xfer()` (blocking until

TC) and **checks the MISO**: the example slave 93 responds to every byte with `0x5A` (`slave_tx_fill` of `ca_spi_config_t`), so every received byte must be `0x5A`. A disconnected MISO wire reads `0x00` or `0xFF` constant. The periodic line says `miso=ok|BAD`, and `s65 info` counts good and bad frames with the first wrong byte of the last bad frame.

- **Slave** (example 93) — receives the bytes in ISR (`ca_spi_recv_handler`, wired in the handlers of each family). When the frame is complete, the callback `slave_cb` verifies the pattern against `seq` and updates the counters. The callback **does not print**: it runs in interrupt context and `ca_stdout_printf_n()` enqueues with `ca_queue_static_send()` at timeout 0, which is *blocking* and does not drop — calling it from ISR means potentially blocking the interrupt when the queue is full, exactly under the fast flow that this example wants to measure. The prints are done by a dedicated task that observes the counters.

`s65 wire 0|1` (master) and `s93 wire 0|1` (slave) check the wires at static levels. A single board cannot judge a wire, because a pin is driven **or** read: the master drives SCK, MOSI and CS at the given level and reads MISO, the slave drives MISO and reads SCK, MOSI and CS, all inputs with pull-down. A level is set on one board and read on the other, then the inverse:

```
master: s65 wire 1      -> WIRE M: drive SCK=MOSI=CS=1  read MISO=..
slave : s93 wire 0      -> WIRE S: drive MISO=0  read SCK=1 MOSI=1 CS=1
master: s65 wire 1      -> ... read MISO=0
(then the same with the levels swapped: SCK=MOSI=0 on the slave, MISO=1 on the
    master)
```

A detached wire always reads 0 (pull-down): the 1 reading discovers it, while the 0 reading discovers a high-blocked line. `s65 wire end` / `s93 wire end` reboot the board and return the pins to SPI. Without argument, `wire` prints the link table. The bank (`test_homo_ex65`) executes the sequence by itself on the two consoles.

Each board has its own console: `s65 info` on the master (frames sent, MISO good/bad), `s93 info` and `s93 buf` on the slave (frames, errors, last `seq`, length, `gap`, first 16 bytes).

88.1.6 Speed: why the default is DIV256

The default is deliberately **cautious**:

```
#define EX65_BR      CA_SPI_BR_DIV256  /* ~625 kHz con PCLK ~160 MHz */
#define EX65_PERIOD  500U              /* ms between one frame and the next */
```

The **slave is the bottleneck**: it receives one byte per interrupt and must read `SPI->DR` before the next one arrives. If the master clocks too fast, the unread byte gets overwritten → **overrun (OVR)** → byte lost. With the U575/F407 pair at `DIV64` (~2.5 MHz, one byte every ~3 µs) the slave was losing about **one byte every 2–3**: `s93 buf` showed the ramp of the pattern `(i+seq)` with jumps (`4E 51 52 54 55` instead of `4E 4F 50 51`), all frames were `err` and the count was off. At `DIV256` (~625 kHz, one byte every ~13 µs) the ISR had plenty of margin: **`err=0, gap=1` stable**.

The `s93 buf` command on the slave is the tool that distinguishes the two failure modes:

- **ramp with jumps** (missing bytes) → **OVR**: the slave cannot keep up with the rate, reduce the baud (or lighten the ISR / switch the slave to DMA).
- **clean but shifted ramp** (correct pattern, shifted by k) → **frame shift**: the slave has locked onto the middle of a frame. Without a CS line to delimit frames, misalignment, once introduced, is permanent.

For high-speed testing, we proceed in steps — `DIV128`, `DIV64`, ... — monitoring `s93 info`/`s93 buf` on the slave. Two indicators:

- `err` increasing → corrupted bytes or (more often here) lost due to OVR.

- `gap` different from 1 → **lost** entire frames.

It is exactly the rate limit of the architecture with one byte per ISR that this example is meant to find.

88.1.7 Why master DMA + slave IRQ

The master knows the frame length in advance → **synchronous DMA** ideal (zero CPU load during transfer). The slave does not know when bytes will arrive → **IRQ per byte** reception with accumulation up to `frame_size` is the natural choice, reusing the already validated slave path (example 29). Any potential slave DMA would require circular RX and framing management on the receiver side, outside the scope of this example.

88.1.8 Notes per family

Family	Notes
F1xx	slave only SPI1/2 (no SPI3); master DMA with fixed channel mapping
F4xx/F7xx	CHSEL/FIFO as in example 64
H7xx	master buffer in AXI-SRAM (`.bss`, no DTCM)
H5xx/U5xx	GPDMA REQSEL

With a different pair than the reference one, **derive the pins from both sides** before wiring: they depend on whether the board uses `alternate_gpio` or not, and on Nucleo-144 boards with Ethernet mounted, the default SPI1 pins are not available.

88.1.9 Test status (hardware)

Pair	Master	Slave	Outcome
Alternative	U575 (v3 DMA)	F407 (v1 IRQ)	at DIV256: `err=0`, `gap=1` stable
Reference	H563 (v3 DMA)	H753 (v3 IRQ)	not tested (hardware not available)

The interoperability **master v3 (U575) slave v1 (F407)** confirms that the two paths are independent of the IP: the master generates the clock and frames, the slave samples them, and the verification protocol works regardless of the silicon generation. From 10/13/2026 the pair is tested also at the bench: `test_homo_ex65` runs on the master board and reads the slave's console (ex93, manually prepared) from a second port, `CAOS_TTY65S`. See example 93.

Chapter 89

Crypto Self-Test: SHA-256, SHA-512, X25519, Ed25519

89.1 Example 66 — Cryptographic self-test (SHA-512, X25519, Ed25519, P-256)

89.1.1 Purpose

Validate the cryptographic primitives that form the basis of the SSH server stack and the TLS 1.3 (HTTPS) server, each against the official vectors of its own standard.

Primitive	Standard	Test	Role in the SSH stack
SHA-256	NIST FIPS 180-4	6	Session KDF + fingerprint (SW fallback where IP is missing)
SHA-512	NIST FIPS 180-4	6	Ed25519 prerequisite
X25519	RFC 7748	8	key exchange `curve25519-sha256`
Ed25519	RFC 8032	8	host key + user authentication
HMAC / HKDF / PBKDF2 (SHA-256)	RFC 4231 / 5869 / 7914	7	key derivation from password (`kdf.c`, app_3 encrypted link)
ECDSA P-256 (SHA-256)	RFC 6979 A.2.5	10	TLS 1.3 (HTTPS) server certificate signature

The Edwards/Montgomery curves and SHA-512 do not have an accelerator on these families and therefore run in software. SHA-256 has the HASH IP on H5/H7 but not on F1/F4/F7: the software version validated here is the **fallback** that makes SSH/KDF portable even where the accelerator is missing (see ex26 for HW/SW dispatch). The self-test runs on the software path, which is the same across all families.

Validation is not a formality: an incorrect primitive still produces output that looks random, and the error would only emerge much further downstream as a handshake that fails without explanation.

89.1.2 Configuration

```
/* os_config.h */
#define EN_SHA256_TEST   1   /* opzionale: valida il SHA-256 software */
#define EN_SHA512        1
#define EN_SHA512_TEST   1
```

```
#define EN_X25519         1
#define EN_X25519_TEST    1
#define EN_ED25519        1   /* richiede EN_X25519 + EN_SHA512 */
#define EN_ED25519_TEST   1
#define EN_ECDSA_P256       1   /* richiede EN_SHA256 */
#define EN_ECDSA_P256_TEST  1
#define EN_CLI_ECDSA_P256   1   /* comando 'p256' */
```

The task is compiled if at least one of `EN_SHA512_TEST`, `EN_X25519_TEST`, or `EN_ED25519_TEST` is active; `EN_SHA256_TEST` adds the SHA-256 battery to the beginning of the sequence.

No specific hardware, except for `ed25519 keygen` and the `keygen + sign + verify` di P-256, che usano il RNG. L'esempio chiama `ca_rng_init()` test in `ca_os_main()`: until 10/23/2026 it did not do so, and `ed25519 keygen` always responded "RNG not available".

89.1.3 Behavior

At startup, the task executes the three batteries in sequence and prints the overall result with the time taken.

```
=== Example 66: crypto self-test ===
SHA-512 (FIPS 180-4), X25519 (RFC 7748), Ed25519 (RFC 8032)

sha256: NIST FIPS 180-4 known-answer tests
  empty                         PASS
  abc                           PASS
  two-block (56B)               PASS
  1M x 'a'                      PASS
  incremental == one-shot       PASS
  padding boundary 55..65       PASS
sha256: ALL PASS (6/6)

sha512: NIST FIPS 180-4 known-answer tests
  empty                         PASS
  abc                           PASS
  two-block (112B)              PASS
  1M x 'a'                      PASS
  incremental == one-shot       PASS
  padding boundary 111..129     PASS
sha512: ALL PASS (6/6)

x25519: RFC 7748 test vectors
  RFC7748 5.2 #1                PASS
  RFC7748 5.2 #2                PASS
  6.1 alice public              PASS
  6.1 bob public                PASS
  6.1 shared secret             PASS
  shared agreement              PASS
  100 iterations stable         PASS
  small-order rejected          PASS
x25519: ALL PASS (8/8)

ed25519: RFC 8032 test vectors
  RFC8032 TEST1 (0 byte)        PASS
  RFC8032 TEST2 (1 byte)        PASS
  RFC8032 TEST3 (2 byte)        PASS
  tampered signature            PASS
  tampered message              PASS
  wrong public key              PASS
  non-canonical S               PASS
  200B sign+verify              PASS
ed25519: ALL PASS (8/8)

p256: RFC 6979 A.2.5 test vectors
```

```
  public from private          PASS
  RFC6979 "sample"             PASS
  RFC6979 "test"               PASS
  tampered signature           PASS
  tampered message             PASS
  public key off the curve     PASS
  r = 0                        PASS
  s >= n                       PASS
  private key 0 rejected       PASS
  keygen + sign + verify       PASS
p256: sign 43 ms, verify 36 ms
p256: ALL PASS (10/10)

ex66: durata totale 2870 ms
ex66: primitive validate - pronte per SSH
```

Actual measurement **on NUCLEO-U575ZI-Q (Cortex-M33 @ 160 MHz)**: 2870 ms for the SHA-512, X25519, and Ed25519 batteries (the SHA-256 battery, added at the beginning, only adds its 1 MB test, in the order of ~1 s). About 1360 ms are from SHA-512 (dominated by the 1 MB test); the remaining ~1500 ms cover the curve operations of X25519 and Ed25519, which equate to ~**30-50 ms per scalar multiplication**. A complete SSH handshake (one X25519 plus one Ed25519 signature) is therefore around **100 ms** on this board, paid once at login.

This data is valid **only for U575**. Other families have different cores and clocks and must be measured separately: H753 (M7 @ 480 MHz, with I/D cache) will be significantly faster, F407 (M4 @ 168 MHz) slower. The code is pure 32-bit calculation without accelerators, so it essentially scales with the core's clock and IPC.

89.1.4 CLI

Command	Effect
`sha256 test`	re-runs NIST vectors (if `EN_SHA256_TEST`)
`sha256 kdf [N]`	HMAC/HKDF/PBKDF2 vectors (RFC 4231, 5869, 7914) and time for a PBKDF2 with N iterations (default 10000)
`sha512 test`	re-runs NIST vectors
`x25519 test`	re-runs RFC 7748 vectors
`ed25519 test`	re-runs RFC 8032 vectors
`ed25519 keygen`	generates a pair from the RNG, prints **only** the public key
`p256 test`	re-runs RFC 6979 vectors and prints the ms for signing and verification
`p256 keygen`	generates a P-256 pair from the RNG, prints **only** the public key (`04 XY `)

`sha256 kdf` measures how much a password costs on the MCU: on H753 (480 MHz) 10000 iterations = 227 ms, 100000 = 2.25 s (about 22 us each), keys equal to Python's `hashlib.pbkdf2_hmac`. The number of iterations for an app is chosen from here: every attempt by an attacker costs the same amount of time.

`ed25519 keygen` fails cleanly if the RNG is not available (F1xx, lacking the peripheral) instead of producing weak keys. The seed is never printed.

89.1.5 The tests that matter

Many tests are chosen because they expose errors that canonical vectors let pass:

- **SHA-512** `1M x 'a'` — multi-block path and 128-bit counter.
- **SHA-512 padding 111..129** — the boundary where an extra block is needed.
- **X25519** `small-order rejected` — a peer can send a small-order point to force a null shared secret; `ca_x25519()` returns `false` and the handshake must abort.
- **Ed25519** `non-canonical S` — with `S >= L` from a valid signature, others just as valid can be derived for the same message (malleability). Verification rejects them.
- **P-256** `public key off the curve` — a public key that is not on the curve must be rejected before any calculation; otherwise, an attacker can choose a point on a weak curve.
- **P-256** `RFC6979 "sample" / "test"` — compare the deterministic signature byte by byte: they test both the curve arithmetic and the nonce generation together.

89.1.6 Note on constant time

X25519 and Ed25519 operate on secret material, so execution times must not depend on the data:

- the Montgomery ladder performs **255 fixed iterations**, without branching on the scalar bit; swaps use arithmetic masks (`fe25519_cswap`) and not `if`;
- Ed25519 scalar multiplication *always* calculates the sum and adopts it with `fe25519_cmov`, without tables indexed by the secret (no observable cache patterns);
- the final signature comparison has no early-exit.

It is slower than a wNAF with a precomputed table, but it does not expose timing patterns.

89.1.7 An instructive bug: `fe25519_sq2`

During development, Ed25519 failed **all** RFC 8032 vectors while X25519 passed them all. The cause was in `fe25519_sq2` (shared):

```
fe25519_sq(h, f);
for (i = 0; i < 10; i++) h->v[i] += h->v[i];   /* limbs fuori range */
```

`fe25519_sq` leaves the limbs close to the 25/26-bit representation limit; doubling them without propagating the carries pushes them over, and `ge_p2_dbl` adds to them. X25519 did not expose this because the Montgomery ladder does not use `sq2` on the same path.

The decisive diagnostic symptom: **the same point, when doubled, gave different results depending on the projective representation** — which is impossible for a correct doubling, given that the Z coordinate is arbitrary. Fortunately, the values remained below the threshold up to bit 91 of the scalar; from bit 92 onwards, they did not.

Fix: carry propagation after doubling the limbs.

89.1.8 API

```
/* Key exchange */
u8 priv[32], pub[32], shared[32];
ca_rng_fill(priv, 32);
ca_x25519_base(pub, priv);
if (!ca_x25519(shared, priv, peer_pub)) { /* punto degenere: abortire */ }

/* Firma */
u8 seed[32], pk[32], sig[64];
ca_ed25519_keygen(pk, seed);
ca_ed25519_sign(sig, msg, len, seed, pk);
bool ok = ca_ed25519_verify(sig, msg, len, pk);
```

89.1.9 Implementation notes

- **Stack**: SHA-512 compression uses `W[80]` of `u64` = 640 bytes per block; curve operations use various `fe25519` (40 bytes each). The task uses `STACK_LARGE`.
- Secret material on the stack (clamped scalars, nonces, hash contexts) is zeroed out upon function exit.

89.1.10 P-256: MCU timings (10/23/2026)

MCU	Sign	Verify
H753	17 ms	13 ms
F767	36 ms	29 ms
H563	43 ms	36–41 ms
U575	67 ms	132 ms
F446	70 ms	140 ms

The test suite passes 10/10 on the 5 Nucleo boards (bench class `test_ex66`). Before writing it to the MCU, the implementation was tested on PC against the Python `cryptography` library (72/72, including 20 random keys). Details in the cryptography chapter.

89.1.11 Next step

These primitives are the building blocks of the **SSH server stack** (Example 67): X25519 for key exchange `curve25519-sha256`, Ed25519 for host keys and user authentication, SHA-256/SHA-512 for KDF and signatures. Once validated here against the official vectors, the complete stack — transport + AES-256-GCM record + userauth + channel/interactive shell — was finalized and HW-tested on U575, H563, and H753 (see Example 67). ECDSA P-256 is instead used for the HTTPS server (Examples 45 and 80).

Chapter 90

SSH Server: Transport, Ed25519 Userauth and Shell

90.1 Example 67 — SSH Server: transport, userauth Ed25519 and interactive shell

90.1.1 Introduction

This example brings a **complete SSH server** on top of the caOS stack: from the PC one connects with the standard `ssh` client, authenticates with an Ed25519 key, and gets an **interactive shell** which is the caOS CLI. It is the endpoint of the crypto journey that started with Example 66 (SHA/X25519/Ed25519 primitives validated against official vectors) and Example 26 (AES-256-GCM HW/SW).

The transport is **wired Ethernet** (RMII PHY): the board connects to the network with a cable and responds to the static IP `192.168.10.67`. The net core — ARP, ICMP, TCP — receives frames from the integrated MAC through a `ca_net_iface_t`, the same abstraction that decouples the stack from the physical medium.

> Therefore, the example requires a board with **ETH MAC and PHY**: it runs on H753 and H563, **not** on U575 nor F4VE (`MCU_HAS_ETH 0` in `objmap.h`). On boards not listed in the source, the file forces `EN_ETH 0` and the example compiles empty. This is a change from early versions, which used USB-NCM to run everywhere.

The three negotiated algorithm suites are one per category (the server offers **only** these):

Category	Algorithm	Standard	caOS Primitive
Key exchange	`curve25519-sha256`	RFC 8731	X25519 + SHA-256
Host key / signature	`ssh-ed25519`	RFC 8709	Ed25519
Cipher (record)	`aes256-gcm@openssh.com`	RFC 5647	AES-256-GCM

The sequence is the canonical SSH-2: **transport** (version exchange → KEXINIT → KEX X25519 → NEWKEYS), then the **GCM record layer**, then **userauth** publickey Ed25519, and finally the **connection layer** (session channel + shell). From NEWKEYS onward, every packet is encrypted and authenticated.

90.1.2 Hardware and families

The example adapts to the board via `MCU_ID`: LED heartbeat GPIO, RMII pins, and SSH task stack size. **An ETH MAC with PHY is required**: this is the requirement that distinguishes usable boards.

MCU	Board	ETH MAC	Heartbeat	Crypto	HW Status on ETH
H753	NUCLEO-H753ZI		PB7 (LD2)	AES **HW CRYP**	end-to-end (19/08)
H563	NUCLEO-H563ZI		PB0 (LD1)	AES **software**	branch ready (RMII_TXD1 PB15, JP6 ON), not retested after switching to ETH
U575	NUCLEO-U575ZI-Q		PC7 (LD1)	AES **software**	not executable: the part lacks ETH MAC
F407	STM32F4VE	(on the part)	PA6 (D2)	AES software	board lacks wired RMII PHY

U575 and F407 no longer run this example. While the transport was USB-NCM, both ran end-to-end; with wired ETH, U575 is excluded from the part (`MCU_HAS_ETH 0`) and F4VE lacks the PHY. Their `MCU_ID` branches remain in the source because they are needed for the equivalent NCM example. **AES hardware or software:** the AES driver chooses the path based on the device (see Example 26). On H753 it uses **CRYP hardware**; on the others, AES-256 software. The SSH record is identical on both paths. The bring-up on H753 required four fixes to the CRYP driver — see *Lesson* at the bottom.

The SSH task stack is `STACK_LARGE`, except on F4xx where extra headroom is needed (`STACK_XLARGE + STACK_MEDIUM`).

90.1.3 Handshake flow

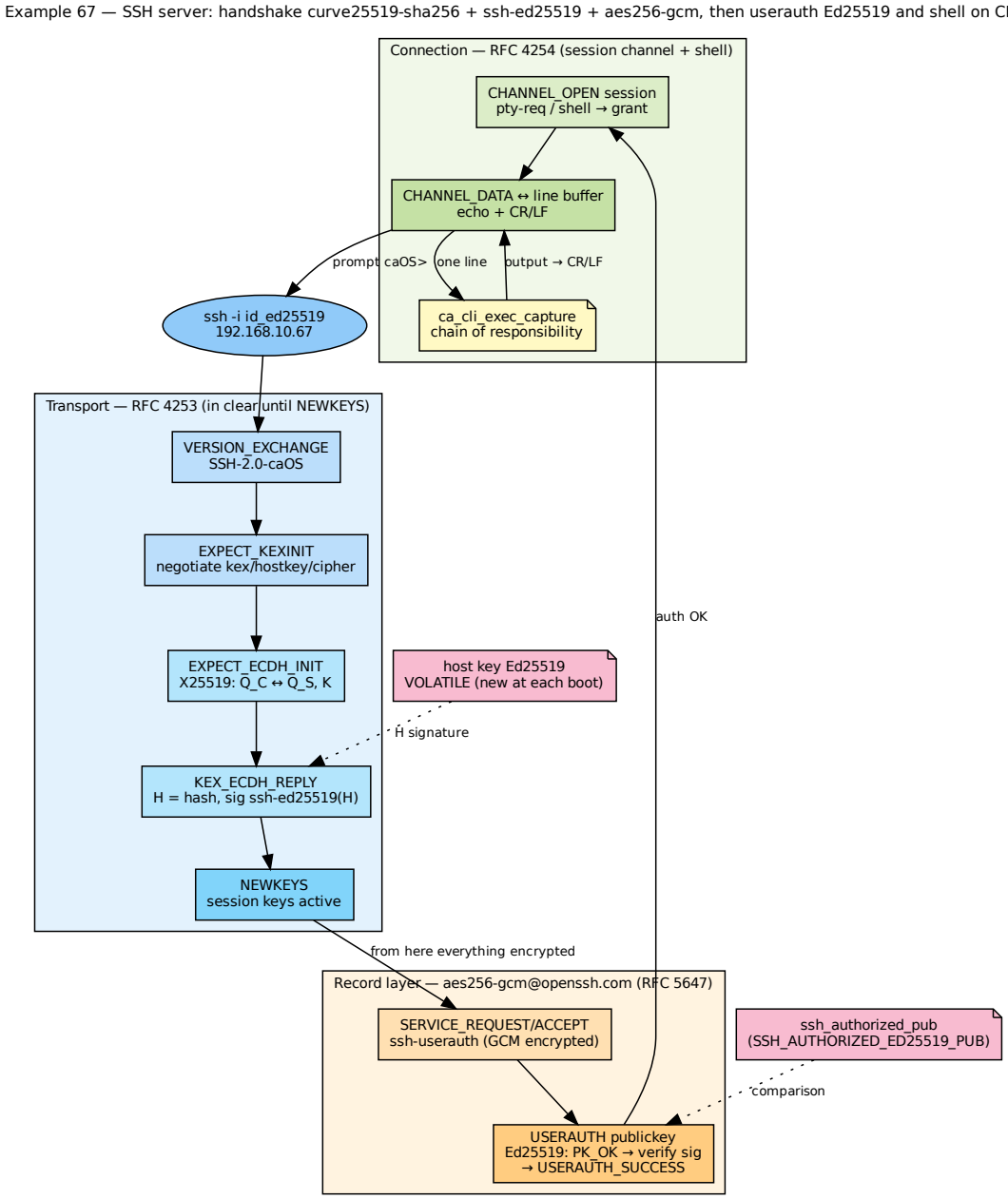


Figure 90.1: Example 67 — SSH handshake and shell

The transport FSM (`ssh_state_t`) goes through these states:

State	What happens
`VERSION_EXCHANGE`	exchange of `SSH-2.0-...` strings
`EXPECT_KEXINIT`	negotiation: kex, host key, cipher (one per category)

State	What happens
`EXPECT_ECDH_INIT`	X25519: the server receives `Q_C`, computes `Q_S` and the secret `K`
`KEX_REPLY_SENT`	sends `KEX_ECDH_REPLY` { host key, `Q_S`, signature of `H` }, then `NEWKEYS`
`NEWKEYS`	session keys derived, GCM record layer active
`ESTABLISHED`	transport ready; from here userauth and connection

After `ESTABLISHED` (all encrypted in GCM):

1. **SERVICE_REQUEST/ACCEPT** — the client requests `ssh-userauth`.
2. **Userauth publickey Ed25519** (RFC 4252 §7): OpenSSH first probes the key without signature (the server responds `USERAUTH_PK_OK`), then resends the signed request. The server reconstructs the signed blob (session id `H` + fields of the request) and verifies the signature with `ca_ed25519_verify` against the authorized key. Only `publickey` + `ssh-ed25519` are accepted; password is not.
3. **Connection** (RFC 4254): `CHANNEL_OPEN` of type `session`, grants `pty-req` and `shell`. The channel is then a bidirectional bridge to the CLI.

The single remote command (`exec`) is **not** supported: it is rejected, the shell is only interactive.

90.1.4 The shell: channel CLI

Once the `shell` is granted, the bytes of `CHANNEL_DATA` feed a line buffer with key echo and backspace (DEL/BS) handling; on CR/LF the complete line is passed to `ca_cli_exec_capture_stream`, which executes the command and forwards its output **to stream**, calling a callback `sender` as the command prints. The output is then re-emitted with `\n` → `\r\n` translation because the client terminal is in raw mode.

> The *streaming* variant was born from a real bug (08/17): the shell used `ca_cli_exec_capture`, which accumulates in a **fixed buffer of 1024 bytes**, and long-output commands — `config`, `top` — arrived **truncated** at the client. Without an intermediate buffer there is no ceiling on the response length. `ca_cli_exec_capture` remains available for those who want output in memory (used by Example 45).

```
caOS SSH shell     type 'help'
caOS> help
...
caOS>
```

> The remote shell requires `EN_CLI_CAPTURE=1`: it is the mechanism by which `ca_cli_exec_capture` intercepts the CLI output. Without it, the shell opens but no command produces output (a `#warning` reports this at compile time).

90.1.5 Keys: volatile host key and authorized key

Two distinct Ed25519 keys, not to be confused:

- **Host key** of the server: generated **at first init, new at every boot** (`hostkey_ready`, static volatile struct). It is not in the key-store, it is not persistent. It signs the `H` hash exchange during KEX.
- **Authorized user key** (`ssh_authorized_pub`): the **public** key of the allowed client, compiled from `SSH_AUTHORIZED_ED25519_PUB` into `os_config.h`. Authentication succeeds only if the offered public key matches this one.

  Since the host key changes at every reboot, the client will see a **warning known_hosts** ("host key changed") at every board restart. This is expected: use the `-o UserKnownHostsFile =/dev/null -o StrictHostKeyChecking=no` options to avoid polluting `~/.ssh/known_hosts`.

90.1.6 Required configuration in `os_config.h`

```
#define EN_NET           1
#define EN_ETH           1   /* transport: Ethernet RMII */
#define EN_TCP_SERVICE   1
#define EN_SSH_SERVICE   1   /* server SSH */
#define EN_HASH          1   /* SHA-256 (KDF)     HW or SW fallback */
#define EN_CLI_CAPTURE   1   /* the remote shell captures the CLI output */

#define SSH_LISTEN_PORT 22U
#define EN_SSH_TRACE     0   /* per-state handshake trace: turned on by hand when
    needed */
#define EN_SSH_TEST      1   /* 'ssh test' command     BPP framing self-test */

/* Public key of the authorized client (32 raw bytes) */
#define SSH_AUTHORIZED_ED25519_PUB { \
    0x.., 0x.., ... /* 32 byte */ }
```

`ca_ssh_service_init()` requires a working hardware RNG: if the RNG fails, the service **refuses to start** in order not to generate weak keys. It also depends on `ca_hash_init`, `ca_keys_store_init` and `ca_rng_init`.

90.1.7 CLI

Command	Effect
`ssh test`	re-executes the binary packet protocol framing vectors (offline, `ca_ssh_bpp_selftest`)

The BPP self-test runs at startup before opening the port, as a sanity check of the RFC 4253 §6 framing.

90.1.8 Linking procedure

1. **Generate a client key pair** (once), and insert the public one in `SSH_AUTHORIZED_ED25519_PUB`:

```
ssh-keygen -t ed25519 -f caos_ed25519/id_ed25519 -N ''
```

2. **Connect the Ethernet cable** and configure the PC interface on the same subnet (replace `<eth-iface>` with the actual name, e.g. `enp3s0`):

```
sudo ip addr add 192.168.10.1/24 dev <eth-iface>
sudo ip link set <eth-iface> up
```

   The board uses **static IP** `192.168.10.67`, therefore there is no DHCP binding to confirm that the cable is properly connected: instead, the board reports the PHY link status with `ETH LINK UP` / `ETH LINK DOWN` on the console.

3. **Connect** (script `run_ssh_client.sh` in `examples/example_67/`):

```
ssh -o IdentitiesOnly=yes -v \
    -o UserKnownHostsFile=/dev/null -o StrictHostKeyChecking=no \
    -i caos_ed25519/id_ed25519 192.168.10.67 -p 22
```

   The flag `-v` shows the client-side handshake; on the board side, the trace `EN_SSH_TRACE` shows the same sequence from the server's point of view — it is **off** in the example's config and turns on when needed (`python3 tools/caos_config.py --on EN_SSH_TRACE 67`).

90.1.9 Expected output

At startup:

```
=== Example 67: SSH transport bring-up ===
curve25519-sha256 + ssh-ed25519 + aes256-gcm over Ethernet

=== EX67: SSH server su ETH ===
IP  192.168.10.67  port 22
Set the PC iface to 192.168.10.1/24, then: ssh -v 192.168.10.67
ETH LINK UP
```

Trace of a session (with `EN_SSH_TRACE=1`), from `NEWKEYS` onward:

```
svc: SERVICE_REQUEST ssh-userauth (GCM record verified)
svc: sent SERVICE_ACCEPT     awaiting USERAUTH
auth: PK_OK (probe accepted), awaiting signature
auth: publickey verified     USERAUTH_SUCCESS (user 4 bytes)
chan: shell granted     bridging CLI
```

90.1.10 Lesson: crypto HW always known-answer

The bring-up of SSH on H753 (AES **hardware** path via CRYP) required four fixes to the driver, invisible until the AES validation remained a simple round-trip encrypt→decrypt:

1. **GCM IV counter**: the driver loaded `ctx->iv[3]` instead of the fixed value `0x00000002` required by the initial block.
2. **Data endianness**: with `DATATYPE=0` the CRYP expects word big-endian, but AAD and payload were little-endian → `__REV` on DIN/DOUT (GCM, ECB, CTR).
3. **Missing `ca_aes_init()` in `ca_ssh_service_init`**: without it, `aes_hw_ready` remained `false`, the GCM path returned without encrypting, and the AES software masked the defect. It was **the** blocker.

4. **Backpressure TCP**: `ca_tcp_send` returning `false` on inflight-full/window-0 is **transient**; it closed the session on long output (e.g. `help`). Fix: retry in `ssh_send_packet_gcm` (atomic because the record is MSS).

A round-trip hides symmetric bugs (wrong byte order in both directions still returns correct). Hence the rule, also applied to Example 26 (`aes_nist`): **crypto hardware always validated with known-answer vectors, never with round-trip.**

> **F407 in Release (fixed 23/07/2026):** on STM32F4VE in Release `-O3` the shell would hang on command `help`. It was neither crypto nor CLI capture: it was the **stack of thread `SSH_SVC` too small** in `ssh_transport.c`, now sized via `SSH_STACK_TASK`. An instructive case of misleading symptom — the hang appeared only in Release because `-O3` changes stack usage, and pointed to crypto only because it was the new code.

90.1.11 Summary

1. **Full SSH-2 server** on Ethernet: transport, record GCM, userauth, shell.
2. **Algorithms**: `curve25519-sha256`, `ssh-ed25519`, `aes256-gcm@openssh.com` — one per category, offered exclusively.
3. **Authentication**: only Ed25519 publickey against `SSH_AUTHORIZED_ED25519_PUB`; no password.
4. **Host key volatile**: new on every boot → expected warning in known_hosts.
5. **Shell = CLI**: the channel is a bridge to `ca_cli_exec_capture_stream` (requires `EN_CLI_CAPTURE=1`), forwarding output to stream instead of accumulating in a fixed buffer; `exec` single not supported.
6. **Tested HW on ETH** end-to-end on H753; H563 has the branch ready but not retested after switching from NCM to ETH. U575 and F407 are no longer executable on this transport (no ETH MAC / no PHY).
7. **AES HW on H753**: four driver CRYP fixes, lesson from known-answer tests.

Chapter 91

Fault Test Bench: MPU, Bus, Usage and HardFault

91.1 Example 68 — Fault Test Bench: MPU, Bus, Usage and HardFault

91.1.1 Introduction

This example is the **fault handling system test bench** of caOS. It deliberately triggers, one at a time, **19 different hardware and software faults** of various categories and shows that in every case the RTOS **survives**: the faulty task is handled according to its recovery policy and the rest of the system continues to run.

It is the practical counterpart of Chapter 12 (*SafeGuard and Fault Recovery*): there the mechanisms are described (dedicated handlers for MemManage/BusFault/UsageFault, task-specific recovery, MPU stack guards, PSP range-checks, escalated declassified HardFault); here they are all exercised on real hardware.

The test bench console is intentionally **direct UART/CDC** (`ca_stdout_*`): in a fault context, the log queue and buffered channels (ITM, CDC with queues) may truncate output, whereas synchronous stdout printing always arrives.

91.1.2 Hardware and Families

The example adapts to the board via `MCU_ID`: end of RAM and Flash are family-specific (they are needed to produce deterministic *out-of-bounds* accesses), because memory cuts differ (F407: 128K/512K, U575: ~628K/2MB, ...). RAM base `0x20000000` and Flash base `0x08000000` on all these parts.

MCU	Core	Board	HW Status
F407	Cortex-M4	STM32F4VE / DISCOVERY	
H753	Cortex-M7	NUCLEO-H753ZI	
H563	Cortex-M33	NUCLEO-H563ZI	
U575	Cortex-M33	NUCLEO-U575ZI-Q	16/16
F746	Cortex-M7		
F103	Cortex-M3 (no FPU)		

Architectural note. Some faults do not manifest the same way on all cores:

- `null_r` / `null_w` (access to `0x0`): on M4/M7 with BOOT0=0 Flash is aliased to `0x0`, so reading **does not** fault and writing is *absorbed* by the Flash interface (FPEC locked) without bus error. On M33 (U5/H5) instead, access to `0x0` does generate a fault. This is not a kernel defect: it is the memory map of the part.

91.1.3 The 19 Cases, by Fault Category

The cases are grouped by the type of exception they trigger.

BusFault (data fetch or instruction fetch out of bounds)

Case	Trigger	Expected Fault
`null_r`	read `*(u32*)0`	BusFault (or none on M4/M7, see note)
`null_w`	write `*(u32*)0`	BusFault/MemManage (or absorbed on M4/M7)
`ram`	access beyond end of RAM	BusFault
`flash`	read beyond end of Flash	BusFault (PRECISERR)
`ibuserr`	fetch code beyond end of Flash	BusFault **IBUSERR**

MemManage (MPU violation)

Case	Trigger	Expected Fault
`ro`	writing to own stack canary (RO region)	MemManage **DACCVIOL**
`stack`	writing within 32B canary band below `stack_end`	MemManage **DACCVIOL**
`stackjmp`	writing *well below* `stack_end` (jumping over the 32B band)	MemManage **DACCVIOL**
`cross`	deliberate write **into the stack of another task**	MemManage **DACCVIOL**, with the victim's name
`xn`	execution from a data buffer **covered by an XN region**	MemManage **IACCVIOL** at first fetch
`runaway`	execution from a data buffer **without region**: the twin of `xn`	a fault **further ahead** from the buffer, not on it (see §"Divergences")

The case `stackjmp` exists because the single 32-byte canary band can be bypassed by a deep stack jump: the MPU guard has been extended below `stack_end` precisely to catch this too (see §12.3).

The case `cross`: the victim is not chosen at random `cross` verifies **cross-task diagnosis**: that the panic handler knows *whose* memory was hit, instead of attributing the write to the current task as a false "stack overflow". The task writes into the stack of another task and the dump must name the victim.

However, there is a non-obvious constraint that the code respects explicitly: the victim **cannot be chosen freely**. The MPU guard only covers the window under the `stack_end` of the current task — on ARMv7-M limited by alignment (see §12.3) — and a victim whose stack falls **outside** that window receives the write **silently**, without any fault. `task_cross` therefore scans

the task list and chooses a victim whose stack **intersects** the guarded window; if none exists, the case is declared *skipped* instead of faking an outcome.

> The constraint is not theoretical: on the first run on H753, the chosen victim was 6 KB lower, outside the 4 KB guard, and the silent corruption surfaced five cases later disguised as "`f_victim` stack overflow" — a completely misleading diagnosis. This is exactly the kind of mistake this case exists to make visible.

The dump distinguishes where the hit address falls relative to the victim. Below its `stack_end` one is in the **canary** of the victim and the offset is not printed (it would be an underflow); from `stack_end` onwards the offset `+N` inside its stack is printed:

```
>> CROSS-TASK WRITE into 'idle' CANARY (end=0x2001C020)
>> CROSS-TASK WRITE into 'idle' +512 (end=0x2001C020)
```

If instead no task owns that address, the dump falls back to the classic `STACK GUARD: overflow N bytes below stk_end` message — which is the correct diagnosis only in that case.

PSP range-check (software protection, not a CPU fault)

Case	Provocation	Outcome
`stackup`	caused by **another** task, corrupting upwards the saved PSP of the victim	the victim task is removed at the **PSP check on entry** during context switch

This is not a hardware exception: it is the range-check `stack_end psp stack_start` that the scheduler applies on the saved PSP *before re-arming it*. It catches a corruption that occurred at a distance, while the victim was not running — a case that the MPU guard (which triggers upon access) would not see.

UsageFault

Case	Provocation	Expected Fault
`div0`	division by zero (with `DIV_0_TRP` enabled)	UsageFault **DIVBYZERO**
`unalign`	`u32` access to odd address (with `UNALIGN_TRP` local)	UsageFault **UNALIGNED**
`undef`	undefined instruction (`UDF.W`)	UsageFault **UNDEFINSTR**
`invstate`	branch to non-Thumb address	UsageFault **INVSTATE**

`DIV_0_TRP` is enabled globally in `ca_bus_error_init`. `UNALIGN_TRP` instead **is not** global (it would break init and libraries doing legitimate unaligned accesses): case `unalign` enables it locally, causes the fault and relies on recovery to bring the system back to a clean state.

HardFault escalated → downgraded to per-task policy

Case	Provocation	Outcome
`forced`	disables `USGFAULTENA` and performs a `div0` → the UsageFault **escalates** to HardFault (HFSR.FORCED)	downgraded to the **same remove-policy** as the three dedicated handlers

This is the most significant case: it shows that even a *forced* HardFault (fault bank disabled, or fault-in-fault) generated by a task, with a valid PSP frame and PSP in range, does not halt the entire RTOS. The gate is conservative: a HardFault from kernel code (MSP), from vector table, or with invalid task still goes to **controlled reset** (`NVIC_SystemReset`, gated by `PANIC_REBOOT_ON_HARDFAULT`).

Restart of a task (only from CLI, outside the normal sequence)

All other cases use the `CA_FAULT_ASSERT` policy (removal). This is the only one that goes through **restart**: a task `CA_FAULT_RESTART_THREAD` performs a division by zero **only on first execution**, the kernel renews it with `ca_thread_renew()` (stack cleaned, stack guard rewritten and rearmed) and the task restarts.

Case	Provocation	Outcome
`restart`	`div0` on first execution of a task with RESTART policy	`Mode: RESTART`, then `restart: run 2` and `alive after restart, canary OK`: one single fault, kernel alive

Added on 10/04: on M33, rearming the guard during restart caused **IACCVIOL**. The fault that triggered it had been removed, but the path had never been tested, because no test case in the bank traversed it.

Kernel safeguard (only from CLI, outside the sequence)

These are not CPU faults: `CA_SAFEGUARD` is a kernel assertion. When it fails, `ca_safeguard_failed()` prints condition, file, line and task and raises `SVC_SAFEGUARD`, which removes the task. None of the 19 cases go through here.

Case	Provocation	Outcome
`safeguard`	`CA_SAFE_FAIL()` from a task, IRQ enabled	line `[SAFEGUARD]` with message, SVC served, `Mode: RECOVER`, kernel alive
`sg_irq`	`ca_mutex_unlock()` on a mutex no one owns	line `[SAFEGUARD]` on `sync.c`, then HardFault FORCED with `CFSR=0` → **reset**

`sg_irq` resets the board **and it is correct this way**: `sync.c` checks the owner after `ca_spinlock_lock_irq()`, so with PRIMASK=1 the `svc` cannot be served and escalates to an opaque HardFault (`CFSR=0`), which the gate rejects. What the case checks is the **order**: line `[SAFEGUARD]`, which names the

condition, must come out **before** the dump. Without it, the dump would only say `HFSR=FORCED, CFSR=0`. For this reason it is not in the automatic sequence.

Fault inside an ISR (only CLI, requires `EN_TIMER`)

All previous cases cause faults in **thread mode**, where there is a task to blame. `fault irq` arms TIM3 and divides by zero **in the update callback**, i.e. in handler mode with the frame on MSP: there is no guilty task.

Case	Provocation	Outcome
`irq`	division by zero in TIM3 callback	`[USAGE FAULT] DIVBYZERO, Faulted in handler mode` with `EXC_RETURN` bit 2 = 0, `No task to blame: system halted` → **board halted**, no task removed

The kernel must **stop**, not remove the interrupted task (innocent) and return to thread mode with the ISR still active: on M4/M7 it would end up in INVPC, on the M33 in a **mute** block (H563 ex87, 29/09). Measured on F446: `EXC_RETURN=0xFFFFFFF1`, and after the halt the board does not even respond to echo. Also `forced` stays harmless: the callback re-enables `USGFAULTENA`, which that case leaves off, otherwise the division would escalate to HardFault.

91.1.4 How it runs

The example has two modes, both active simultaneously.

Automatic sequence

At boot, the task `monitor` (low priority, with policy `CA_FAULT_RESTART_THREAD`) runs **all** 19 cases in ascending order of severity, one every 2 seconds:

```
=== Example 68: Fault sequence (auto start) ===

--- [1/16] fault: null_r ---
[EX68] provoking: read from NULL (0x0)
...
--- [12/16] fault: div0 ---
[EX68] provoking: divide by zero
=== USAGE FAULT ===
  DIVBYZERO: Divide by zero
CFSR: 0x02000000
Task: "f_div0" [n]
Mode: RECOVER (task removed)
...
=== Fault sequence complete     system still alive ===
EX68 alive  tick=
```

At the end, the monitor periodically prints `EX68 alive`, as proof that the scheduler continues to run after having absorbed 19 faults.

Interactive CLI command

From the console, a single case can be launched at will:

```
fault div0
fault stackjmp
fault forced
fault safeguard       kernel safeguard, task removed
fault restart         div0 once in a RESTART task: renewed, restarts
fault sg_irq          safeguard under PRIMASK: the board RESETS
fault irq             div0 inside the TIM3 ISR: the board HALTS
help                  list of the names
```

Every case is executed by a one-shot task with policy `CA_FAULT_ASSERT` (remove): a *deterministic* fault would cause fault on restart, so removal is the correct policy for the bank.

91.1.5 What the dump shows

The panic handler prints, for each fault:

- the **category** and **sub-type** decoded from the CFSR (e.g., `DIVBYZERO`, `IACCVIOL`, `IBUSERR`);
- the raw **HFSR** and **CFSR** registers and, for address faults, **MMFAR**/**BFAR**;
- the stacked **PC** and **LR** (faulting instruction address);
- the **name and index of the faulty task**;
- the **recovery mode** applied (`RECOVER (task removed)` / `RESTART`).

Historical correction. The UsageFault decoding in the dump used the mask `0xFF0000` (bits 16–23 only): `DIVBYZERO` (bit 25) and `UNALIGNED` (bit 24) were **never** decoded, and the strings for these were inverted. The correct mask is `0xFFFF0000`. The bug was latent in the `panic.c` path (dedicated handlers worked); bank ex68 brought it to light.

91.1.6 Actual output (STM32H753, Cortex-M7 @ 480 MHz)

Complete automatic sequence, significant excerpts (dump with task name printed only once). Each case ends with `Mode: RECOVER (task removed)` and the system arrives alive at the end.

```
=== Example 68: Fault sequence (auto start) ===

--- [3/16] fault: ram ---
[BUS FAULT]
  IMPRECISERR: async write buffer
  CFSR = 0x400
  Stacked PC = 0x08001E7A
  Task: ram [4]
  Mode: RECOVER (task removed)

--- [4/16] fault: flash ---
[BUS FAULT]
  PRECISERR @ 0x8200000
  CFSR = 0x8200   BFAR = 0x8200000
  Task: flash [4]
  Mode: RECOVER (task removed)

--- [5/16] fault: ibuserr ---
[BUS FAULT]
```

```
  IBUSERR: instruction fetch error
  CFSR = 0x100
  Stacked PC = 0x08200000   (fetch oltre fine Flash)
  Task: ibuserr [4]
  Mode: RECOVER (task removed)

--- [6/16] fault: ro ---   (identico a [7] stack)
[MPU FAULT]
  DACCVIOL: data access violation
  MMFAR = 0x20017000  (valid)   CFSR = 0x00000082
  stk_start=0x20018000 stk_end=0x20017020 psp=0x20017FBC
  >> HIT LOWER CANARY @ 0x20017000
  Task: ro [4]
  Mode: RECOVER (task removed)

--- [8/16] fault: stackjmp ---
[MPU FAULT]
  DACCVIOL: data access violation
  MMFAR = 0x20016F00  (valid)   CFSR = 0x00000082
  >> STACK GUARD: overflow 288 bytes below stk_end (jumped the band)
  Task: stackjmp [4]
  Mode: RECOVER (task removed)

--- [9/16] fault: cross ---
[EX68] provoking: write into ANOTHER task's stack (cross-task)
[EX68] writing into 'idle' stack @ 0x2001C220
[MPU FAULT]
  DACCVIOL: data access violation
  MMFAR = 0x2001C220  (valid)   CFSR = 0x00000082
  >> CROSS-TASK WRITE into 'idle' +512 (end=0x2001C020)
  Task: cross [4]
  Mode: RECOVER (task removed)

--- [11/16] fault: stackup ---
ERROR : Task[4] 'f_victim': stack overflow! 22057
  victim 'f_victim' psp forced to 0x20015040 (stack_start=0x20015000)
  (removed by the PSP range check on entry     not a CPU fault)

--- [12/16] fault: div0 ---
[USAGE FAULT]
  DIVBYZERO: divide by zero
  UFSR = 0x0200   CFSR = 0x02000000
  Task: div0 [4]
  Mode: RECOVER (task removed)

--- [13/16] fault: unalign ---
[USAGE FAULT]
  UNALIGNED: unaligned access
  UFSR = 0x0100   CFSR = 0x01000000
  Task: unalign [4]
  Mode: RECOVER (task removed)

--- [16/16] fault: forced ---
*** HARD FAULT ENTRY ***
=== HARD FAULT ===
  UsageFault: DIVBYZERO: Divide by zero
  Forced HardFault
  HFSR = 0x40000000   CFSR = 0x2000000
  Stacked PC = 0x8000680   LR = 0x80097BF
=== END HARD FAULT ===
  Task: forced [4]
  Mode: RECOVER (task removed)

```

```
=== Fault sequence complete     system still alive ===
```

Note the following in the actual output:

- `stackjmp` shows the extended guard in action: *"overflow 288 bytes below stk_end (jumped the band)"* — the write has overrun the 32 B canary band and the guard has nevertheless intercepted it.
- `cross` shows cross-task diagnosis: the dump **names the victim** (`CROSS-TASK WRITE into 'idle' +512`) instead of attributing the write to the running task, as it used to do by reporting a "never occurred" overflow.
- `stackup` does not produce a CPU fault but the *"stack overflow!"* message from the range-check: the PSP of the victim, forced above `stack_start`, is rejected at assembly during the context switch.
- `forced` goes through the actual HardFault handler (`*** HARD FAULT ENTRY ***`), prints its complete dump (`Forced HardFault`, `HFSR=0x40000000`) and **then** applies the remove-policy like dedicated faults: `Mode: RECOVER`.

91.1.7 Comparison: the same bank on STM32H563 (Cortex-M33 @ 250 MHz)

Some cases behave differently on M33 compared to M4/M7, because the **memory map** and MPU configuration change, not the kernel. Here are the excerpts where the difference is visible (the rest of the dump is identical to H753):

```
--- [1/16] fault: null_r ---
[BUS FAULT]
  PRECISERR @ 0x0
  CFSR = 0x8200
  BFAR = 0x0
  Task: null_r [4]
  Mode: RECOVER (task removed)

--- [3/16] fault: ram ---
[EX68] provoking: access beyond
       RAM end   (no fault)
```

```
--- [2/16] fault: null_w ---
[BUS FAULT]
  PRECISERR @ 0x0
  CFSR = 0x8200
  BFAR = 0x0
  Task: null_w [4]
  Mode: RECOVER (task removed)

--- [10/16] fault: xn ---
[BUS FAULT]
PRECISERR @ 0x17
    BFAR = 0x17
    Stacked PC = 0x20000070  (in the RAM buffer)
    Task: xn [4]
    Mode: RECOVER (task removed)
```

Differences M33 vs M4/M7:

Case	H753 (M4/M7)	H563 (M33)	Why
`null_r` / `null_w`	UsageFault **UNDEFINSTR**	BusFault **PRECISERR @ 0x0**	On M4/M7 `0x0` is Flash aliased → access does not fault and traps the `udf` which the compiler inserts on null-deref (UB); on M33 `0x0` is not readable → immediate BusFault.
`ram`	BusFault IMPRECISERR	**no fault**	The address *beyond RAM* chosen falls, on H563, into a region that does not generate BusFault; on H753 it does. To be refined `ADDR_BEYOND_RAM` for H5.
`runaway`	UsageFault UNDEFINSTR	BusFault **PRECISERR @ 0x17**	Neither of the two buffers was Execute-Never: the `blx` **executes data** and traps later, where it happens.

Fixed on 09/18, and the case has been split. The line above was not a M33/M7 divergence to be explained: it was the case that was not behaving as stated, on **both** cores.

Measures taken on F446, four runs of the same binary:

run	fault	`Stacked PC`
1, 2, 4	UsageFault **UNDEFINSTR**	`0x2000F118`
3	UsageFault **UNALIGNED**	`0x2000F64C`

`g_data_buf` is at `0x2000F040`, 16 bytes of `.bss` all zeros. In Thumb `0x0000` is `LSLS r0,r0,#0` — **a valid instruction**. The `blx` therefore **does not fault**: the core executes eight NOPs and continues inside the `.bss`, stopping 200 bytes after the end of the buffer inside `g_cur_index`, a *live* variable of the scheduler (on run 3 it had advanced 1532 bytes further). The stopping point is decided by the kernel state at that moment: from here both the variability between runs, and the "divergence" between boards.

The fetch is legitimate, and it is not a bug: `MPU->CTRL` has `PRIVDEFENA`, tasks run privileged, and the default ARM map **does not** mark SRAM as execute-never. This also answers the question of why the case originated: yes, **the heap is executable** on these parts, unless a region says otherwise.

From here, the two twin cases, which differ in **only one thing**:

case	buffer	MPU region	outcome
`xn`	`g_xn_buf` (32 B, 32-aligned)	**yes**, `XN=1`	fault **on** the buffer: the first fetch is rejected
`runaway`	`g_data_buf`	**no**	fault **beyond** the buffer: the fetch is allowed and the core continues

The criterion is the PC, not the fault type — and this cost one run. The first version demanded IACCVIOL from `xn` and any-other-thing from `runaway`. The board said no: the two buffers are **28 bytes** apart in `.bss`, so as soon as the region exists the runaway exits its own buffer by NOPs and ends **against that same region** — same fault, same PC. It would have been the layout again deciding the verdict, which is the error from which this whole investigation started.

Measured on F446 after the fix:

case	buffer	`Stacked PC`	delta
`xn`	`0x2000F080`	`0x2000F080`	**+0** — did not execute anything
`runaway`	`0x2000F064`	`0x2000F080`	**+28** — executed, then hit the guard

The fault type is the same for both; the PC separates them unambiguously, and no memory layout can make one look like the other.

The region is armed **once** in `ca_os_configure()` and never disarmed: doing so inside the case would mean disarming it afterwards, and the case never returns — the fault removes the task. If there are no free regions, `xn` prints **SKIPPED** instead of looping: without the region it would be `runaway` under a false name, and would pass for the wrong reason.

Why the board didn't notice: `test_ex68.cpp` counted the `Mode: RECOVER` lines asking for at least 12 out of 18. The case did cause a fault, the line was there, the count added up. *Which* fault it was didn't matter to anyone. The `xn_region_is_what_stops_the_fetch` phase now asserts the **type** of fault on both twins. The rest (BusFault flash/ibuserr, MemManage ro/stack/stackjmp, PSP range-check stackup, UsageFault div0/unalign/undef/invstate, HardFault forced) is **identical** on both cores and recovers in all cases.

Note. In early versions the dump printed the task name **twice** in MemManage and UsageFault faults (one local print in the handler + one in `ca_scheduler_manage_fault`). Fixed: now the task appears **only once**, uniformly for all fault types, as seen in the outputs above.

91.1.8 Summary

1. **16 faults** in 5 categories (BusFault, MemManage, PSP range-check, UsageFault, HardFault escalated), all **handled and recovered** without stopping the RTOS.
2. **Full Cortex-M trap coverage**: all CFSR subtypes are decoded by the dump, even those not directly triggered by the bank.
3. **Two modes**: automatic sequence at boot + CLI command `fault <nome>`.
4. **Robust console in fault context**: synchronous stdout, no buffered queues.
5. **The `forced` case** demonstrates the escalation of HardFault to a per-task policy — a forced HardFault from a task no longer kills the system.
6. **The `cross` case** verifies cross-task diagnosis: that the dump names the **owner** of the affected memory, and that the victim is chosen within the actually watched window — outside of it, corruption passes silently and reemerges disguised as something else.
7. **HW tested** on F407 (M4), H753 (M7), H563 and U575 (M33), the latter two 16/16; F746 and F103 remain.

Chapter 92

UDP Server over USB-NCM

92.1 Example 69 — UDP Server on USB-NCM

92.1.1 Introduction

This example is the **USB variant of example 34** (UDP server on Ethernet). The service is identical — port 7961, a dedicated worker for each client, echo with prefix `ECHO:`, idle timeout — but the frames no longer pass through the Ethernet MAC: they travel on a **USB network adapter** (CDC-NCM), that is, on the cable that already powers and programs the board.

The educational point is the practical confirmation of the `ca_net_iface_t` abstraction introduced in example 55: between ex34 and ex69 **the application code does not change by a single line**. Callbacks, server configuration struct, `ca_net_process()` loop are the same. Only *which interface* is passed to `ca_net_init()` changes:

```
/* example 34     Ethernet */          /* example 69     USB-NCM */
eth_gpio_init();                      ca_usb_dev_mng_init(CA_USB_SPEED_FS,
    USB_NUM);
                                      ca_usb_dev_mng_add(&s_ncm.header);
                                      ca_usb_dev_mng_start();
ca_net_init(ca_eth_iface_get(&cfg)); ca_net_init(ca_ncm_iface_get(&s_ncm));
ca_net_proto_init(&net_cfg);          ca_net_proto_init(&net_cfg);   /* identico
    */
```

If ex55 demonstrates the abstraction on the absolute minimum (ARP + ICMP), ex69 demonstrates it where it really counts: **a full application service**, with threads for client and state management, running over two different transports without knowing it.

Feature	Notes
UDP Server on port 7961	`ca_udp_server_start()`, up to 4 simultaneous clients
Worker per client	a dedicated thread for each client (internal to the service)
Echo with prefix	replies with `ECHO: <data>`
Inactivity timeout	disconnects after 10 s of inactivity
USB Ethernet (NCM)	device `usb0` via CDC-NCM
`ca_net_iface_t` abstraction	same UDP service on ETH (ex34) or NCM (ex69)
CLI `ping` / `ip`	with `EN_CLI_NET`

92.1.2 PC Prerequisite: Remove the interface from NetworkManager

When the board enumerates, a new network interface appears on the PC (`enx02ca0500xx` — the name udev derives from the MAC). NetworkManager sees it as any ordinary plugged-in card and **assigns it an address of its own**: it has been observed `172.21.10.100/24`. The board is on `192.168.10.0/24`, so the two end up on different subnets and the board is unreachable.

The symptom is misleading, because the interface **exists and is UP**: `ip link` shows it, and even `ping -I <iface>` responds — that flag bypasses the routing table. What doesn't work is everything else (`socat`, `nc`, the PC scripts of the examples), which follow the routing table.

The rule is versioned in the repository:

```
sudo cp tools/99-caos-ncm.conf /etc/NetworkManager/conf.d/
sudo systemctl reload NetworkManager
nmcli -t -f DEVICE,STATE dev status | grep enx    # it must say: unmanaged
```

> **The glob goes on `interface-name`, not on `mac:`.** The form `mac:02:ca:05:00:00:*` seems equivalent but it is not: it requires a full MAC and fails **silently** with a wildcard character — no error, and the interface remains managed.

The bank address does not set this rule: it is set by `sudo ./tools/bench_net.sh usb` or the example PC script.

92.1.3 Hardware

No PHY, no Ethernet cable: only the USB device port of the board is needed.

MCU	`USB_NUM`	Connector	LED heartbeat
U575 (NUCLEO-U575ZI-Q)	0	USB user	PC7 (LD1)
H753 (NUCLEO-H753ZI)	1	CN14 (OTG_FS)	PB7 (LD2)
F407 (STM32F4VE)	0	USB device	PA6 (D2)
H563 (NUCLEO-H563ZI)	0	USB user	PB0 (LD1)

On other families the example disables itself (`#undef EN_USB`) and prints the message from the `#else` branch.

92.1.4 os_config.h Configuration

```
#define EN_NET          1     /* net core: pool buffer, ARP, ICMP,     */
#define EN_USB          1     /* driver USB DWC2 + stack mgmt */
#define EN_USB_NCM      1     /* CDC-NCM class (the network transport) */
#define EN_UDP_SERVER   1     /* the service being shown */
#define EN_ARP_SERVICE  1     /* MAC     IP resolution */
#define EN_CLI_NET      1     /* optional: CLI commands 'ping' / 'ip' */
```

Off relative to ex34:

```
#define EN_ETH        0     /* no Ethernet MAC */
#define EN_CLI_ETH    0     /* see note below */
#define EN_USB_CDC    0     /* the USB port acts as a network card, not a serial
    port */
```

Why `EN_CLI_ETH` should be turned off. The commands `eth link` / `eth stats` / `eth pool` are inside `#if EN_ETH` in `caos/drivers/src/eth.c`: with `EN_ETH=0` they are not compiled at all. Leaving `EN_CLI_ETH=1` would not cause a build error — it would remain an active flag but **inactive**, i.e. a configuration that declares something that does not exist in the firmware. This is the type of defect that `caos_config.py --find --inert` reports. The pool counters remain accessible by the `net` command.

The configuration is generated and modified **only** with the agent:

```
python3 tools/caos_config.py --new 69 --from ex34 \
        --set EN_ETH=0 --set EN_USB=1 --set EN_USB_NCM=1 --set EN_USB_CDC=0
python3 tools/caos_config.py --off EN_CLI_ETH 69
```

92.1.5 Architecture

```
        PC (usb0)                          Board

       nc -u ... 7961                      udp_server (7961)

    worker per client

                                                    ca_net_process()

                                        net core: ARP/ICMP

                                                    ca_net_iface_t      l'unico
    punto

        that changes
                        USB CDC-NCM                      ca_usb_ncm (push)
```

The UDP service and the net core are **the same logical binaries** as in ex34: the replacement happens underneath, at the interface level.

Where initialization lives

In ex34 `ca_net_init()` lives in `ca_os_configure()`, before the scheduler. In ex69 **it cannot**: the NCM interface exists only after the USB device has been enumerated, so USB bring-up and `ca_net_init()` are together inside `task_net`. It's the same choice as in ex55.

There is also no `eth_mon` task: without a PHY, there is no link to query with `ca_eth_link_poll()`. The "link" is the USB enumeration, readable with `ca_usb_ncm_is_up()`. For the same reason, GARP is not sent: the host learns the device's MAC from the NCM descriptors.

> **The NCM link is not up just when the device is enumerated.** `s_up` goes high only when the host selects the **alt setting 1** of the data interface, i.e., when the PC actually brings up `usb0`. Before that moment `ca_usb_ncm_send_frame()` discards every frame and returns `false`.
>
> However, this example is purely **reactive** — it responds, never speaks first — so it doesn't need to wait: when the first packet arrives, the link is by definition already up. **And the wait should not be added**: blocking the net task before the loop

would prevent the enumeration itself (the Linux kernel would report `device not accepting address`), because the stack needs this task to already be serving frames. For a service that instead *initiates* the conversation, the gate is really needed, but it must be placed **inside** the loop: see example 70.

92.1.6 Expected UART output

```
=== EX69: UDP server porta 7961 su USB-NCM ===
MAC  02:CA:05:00:00:69
IP   192.168.10.69
USB NCM UP
UDP server ready - bring up usb0, then: nc -u 192.168.10.69 7961
UDP[7961] client 0: 192.168.10.1:54321 connesso
UDP[7961] client 0: rx 6B ('hello.') da 192.168.10.1:54321
```

92.1.7 Test from PC

When the cable is connected, a new network interface appears (`usb0`, or `enxNN` with systemd's predictable names):

```
ip link | grep -i usb                          # find the real name
sudo ip addr add 192.168.10.1/24 dev usb0
sudo ip link set usb0 up

echo "hello" | nc -u 192.168.10.69 7961        # reply: ECHO: hello
nc -u 192.168.10.69 7961                       # interactive session
```

Two simultaneous `nc` terminals show the per-client workers: the log reports `client 0` and `client 1` with different source ports.

Inverse Ping (device → host) from serial CLI

```
ping 192.168.10.1 count=3
```

92.1.8 Technical Notes

Comparison with ex34: what really changes

	ex34 (ETH)	ex69 (NCM)
Bring-up	`eth_gpio_init()` (9 pin RMII)	`ca_usb_dev_mng_*()`
Interface	`ca_eth_iface_get(&eth_cfg)`	`ca_ncm_iface_get(&s_ncm)`
RX Model	pull (DMA descriptor)	push (`ca_net_rx_push`)
Link	`ca_eth_link_poll()` + task `eth_mon`	USB enumeration, `ca_usb_ncm_is_up()`
GARP	sent after bring-up	not needed
Init	in `ca_os_configure()`	in `task_net` (after enumeration)
UDP Callback, `net_cfg`, loop	— **identical** —	

Two MACs, one for each side

`s_mac` (`02:CA:05:00:00:69`) is the **device** MAC; `s_ncm.mac` (`02:CA:05:00:00:A9`) is the one announced to the **host side**. These are the two ends of the same cable and must be different. The first byte `0x02` marks the address as *locally administered*.

Bandwidth

USB Full Speed provides 12 Mbit/s raw bandwidth, compared to ~33 Mbps measured on Ethernet in ex44. For a UDP service with short packets it is irrelevant, but must be kept in mind if this scheme is reused for volume transfers.

92.1.9 File Structure

```
examples/example_69/
          ca_os_ex69.c          entry point
          os_config_ex_69.h     configurazione (generata da ex34)
```

The test does not require scripts: `nc -u` is sufficient, as in ex34.

Chapter 93

DHCP Client over USB-NCM

93.1 Example 70 — DHCP Client over USB-NCM

93.1.1 Introduction

USB-NCM variant of **example 35** (DHCP client over Ethernet). The services are the same — DHCP client with DORA and renewal, ARP, ICMP echo, UDP server on port 7961 — but the transport is the USB port instead of the Ethernet MAC.

Compared to example 69 (UDP server over NCM, static address), here the address is **dynamically assigned**, and this is precisely what makes the example interesting on a point-to-point link: it shows that the caOS DHCP client does not depend on the transport, but requires **someone to act as a server** on the other end of the cable.

Feature	Notes
DHCP Client	Full DORA + T1/T2 renewal
GARP	sent at every bound/renew
ARP service	Initial IP 0.0.0.0, updated at bound
ICMP echo reply	responds to `ping` on the obtained address
UDP server 7961	up to 2 simultaneous clients
USB Ethernet (NCM)	device `usb0` / `enxNN`

93.1.2 PC Prerequisite: detach the interface from NetworkManager

When the board enumerates, a new network interface appears on the PC (`enx02ca0500xx` — the name that udev derives from the MAC). NetworkManager sees it as any newly connected card and **assigns its own address to it**: `172.21.10.100/24` has been observed. The board is on `192.168.10.0/24`, so the two end up on different subnets and the board is unreachable.

The symptom is misleading, because the interface **exists and is UP**: `ip link` shows it, and even `ping -I <iface>` responds — that flag bypasses the routing table. What does not work is everything else (`socat`, `nc`, the PC scripts of the examples), which follows the routing table.

The rule is versioned in the repository:

```
sudo cp tools/99-caos-ncm.conf /etc/NetworkManager/conf.d/
sudo systemctl reload NetworkManager
nmcli -t -f DEVICE,STATE dev status | grep enx    # deve dire: unmanaged
```

> **The glob goes on `interface-name`, not on `mac:`.** The form `mac:02:ca:05:00:00:*` seems equivalent but it is not: it expects a full MAC and fails **silently** with a wildcard character — no error, and the interface remains managed.

The bank address is not set by this rule: it is set by `sudo ./tools/bench_net.sh usb` or the PC script in the example.

93.1.3 The critical point: there is no DHCP server

On Ethernet (ex35) the DHCP server is the LAN router. On a point-to-point USB cable **it does not exist**: on one side there is the board, on the other the PC, and neither of the two offers default addresses.

The PC must therefore act as a server. The script provided with the example configures `dnsmasq` on the NCM interface:

```
sudo examples/example_70/test_dhcp_ncm_pc.sh
```

The script **finds the interface on its own**, searching for the one whose MAC starts with `02:ca:05` (the host-side MAC announced by the NCM descriptors), and waits up to 30 s for it to appear — because unlike `eth0`, the USB interface **is created when the board enumerates**. You can still pass it manually:

```
sudo examples/example_70/test_dhcp_ncm_pc.sh enx02ca050000b0
```

Then the script: assigns `192.168.10.1/24` to the PC, starts `dnsmasq` with pool `.100 .200`, waits for the ACK by reading it from the log, and finally tests ping, UDP echo, and two simultaneous clients. Upon exit, it performs cleanup (stops dnsmasq, removes the IP).

93.1.4 Hardware

MCU	`USB_NUM`	Connector	Heartbeat LED
U575 (NUCLEO-U575ZI-Q)	0	USB user	PC7 (LD1)
H753 (NUCLEO-H753ZI)	1	CN14 (OTG_FS)	PB7 (LD2)
F407 (STM32F4VE)	0	USB device	PA6 (D2)
H563 (NUCLEO-H563ZI)	0	USB user	PB0 (LD1)

93.1.5 os_config.h configuration

```
#define EN_NET            1
#define EN_USB            1
#define EN_USB_NCM        1
#define EN_NET_DHCP       1     /* protocollo DHCP nel net core */
#define EN_DHCP_SERVICE   1     /* task client DHCP */
#define EN_UDP_SERVER     1
#define EN_ARP_SERVICE    1
```

Off compared to ex35: `EN_ETH 0`, `EN_CLI_ETH 0`, `EN_USB_CDC 0`.

```
python3 tools/caos_config.py --new 70 --from ex35 \
        --set EN_ETH=0 --set EN_USB=1 --set EN_USB_NCM=1 --set EN_USB_CDC=0
python3 tools/caos_config.py --off EN_CLI_ETH 70
```

93.1.6 Startup sequence

The order is more delicate than on Ethernet, because the "link" is not the PHY but the USB enumeration:

```
task_net:
  1. ca_usb_dev_mng_init/add/start      bring-up USB
  2. ca_net_init(ca_ncm_iface_get())    NCM diventa l'iface del net core
  3. ca_net_proto_init(&net_cfg)        stack con IP = 0.0.0.0
  4. ca_net_proto_bind_udp(68,    )       porta 68 per OFFER/ACK
  5. ca_udp_server_bind()               porta 7961
  6. loop: appena il link    su     ca_dhcp_service_net_ready()     parte il
    DORA
           poi ca_sem_timedwait() + ca_net_process()
```

The gate of step 6 is specific to NCM, and it is not a formality. In ex35 the equivalent is `ca_eth_link_poll()`: it expects the PHY to see the cable. Here it expects the host to **select alt setting 1** of the data interface, which is what raises `s_up` in `usb_ncm.c`. As long as it is zero, `ca_usb_ncm_send_frame()` **discards every frame and returns** `false`:

```
bool ca_usb_ncm_send_frame(const u8 *eth, u16 len)
{
    if (!s_up || eth == NULL || ...) return false;   /* silenziosamente perso */
```

A DISCOVER sent too early is therefore not *delayed*: it is **lost**. And the DHCP client would consume the entire retry budget (`DHCP_DISCOVER_RETRY_N` × `DHCP_OFFER_TIMEOUT_MS` = 3 × 4 s) talking into a closed pipe.

> **Where to place the wait — and where not to.** The gate **must be inside the loop**, not before it. Blocking the net task while waiting for the link *before* serving frames prevents enumeration itself: the host fails to complete the SET_ADDRESS and the Linux kernel reports `device not accepting address NN, error -62, unable to enumerate USB device`. It waits for an event that cannot happen as long as it is waiting. For the same reason, the loop uses `ca_sem_timedwait(&ca_net_rx_sem, 100)` and not `ca_sem_wait()`: before the link is up, no frames arrive, so a wait without a timeout would park the task and the gate would never be re-evaluated.
>
> **Power-on order**: first the board, then the script on the PC. The firmware waits as long as necessary; it is the script that needs the interface to be already present in order to configure it.

The link monitor

`task_ncm_mon` is the NCM counterpart to `task_eth_mon` in ex35: it samples `ca_usb_ncm_is_up()` every 500 ms and signals transitions. If the link comes back up **while a lease is already active** (cable reattached, `usb0` reconfigured), it requests a renew, so the address is revalidated instead of being taken for granted until T1. When there is no lease, however, it does nothing: the DORA cycle is already retrying on its own.

> **Why `ca_dhcp_service_bind()` must not be called in the net task** (this also applies to ETH): it is blocking, and it would block the task that needs to run `ca_net_process()` → the OFFER would never be processed → deadlock. The IP arrives via callback `on_dhcp_bound` from the DHCP task.

93.1.7 Expected UART output

```
=== EX70: DHCP + ARP + ping + UDP server su USB-NCM ===
MAC  02:CA:05:00:00:70
IP   --- (in attesa DHCP)
USB NCM DOWN - attendo che l'host porti su l'interfaccia...
USB NCM UP
DHCP: avvio DORA...
DBG DHCP DISCOVER attempt 1 xid=0x05000031

DHCP BOUND  IP=192.168.10.100  mask=255.255.255.0
            GW=192.168.10.1  DNS=8.8.8.8
UDP[7961] client 0: 192.168.10.1:51234 connesso
UDP[7961] cid=0 rx 10B ('hello caOS') from 192.168.10.1:51234
```

93.1.8 Technical notes

Two MACs, one for each side

`s_mac` (`02:CA:05:00:00:70`) is the device MAC; `s_ncm.mac` (`02:CA:05:00:00:B0`) is the one announced to the host side — this is what the `enx02ca050000b0` interface name on the PC is derived from, and it is what the script looks for.

The dynamic IP and the three mirrors

Upon binding, the address must be propagated to three subsystems, or they will remain misaligned:

```
ca_net_proto_set_local_ip(ip);        /* stack IP: filtro dei pacchetti in
    ingresso */
ca_udp_server_set_local_ip(ip);       /* server UDP: indirizzo sorgente delle
    risposte */
ca_arp_service_update_local_ip(ip);   /* ARP: a quale IP rispondere */
```

It is the same sequence as ex35: it does not change with transport.

93.1.9 File structure

```
examples/example_70/
          ca_os_ex70.c            entry point
          os_config_ex_70.h       configurazione (generata da ex35)
          test_dhcp_ncm_pc.sh     setup dnsmasq su usb0 + test automatico
```

Chapter 94

DHCP and NTP Client over USB-NCM

94.1 Example 71 — DHCP Client + NTP over USB-NCM

94.1.1 Introduction

Variant USB-NCM of **example 36** (DHCP + NTP over Ethernet). The services are the same — DHCP client with DORA and renewal, NTP client, ARP, ICMP echo, UDP server on port 7961 — but the transport is the USB port instead of the Ethernet MAC.

Compared to example 70 (DHCP over NCM), the novelty is only one, and it's worth isolating: the **NTP client is downstream of the DHCP**. It has no gate directly on the USB link, because it is already protected by the DHCP's gate: when a lease exists, the link is necessarily up. It is the first NCM clone where two services are chained, and it shows that the chaining remains identical to that of ex36 — the abstraction `ca_net_iface_t` is not visible.

Feature	Notes
DHCP Client	Full DORA + T1/T2 renewal
NTP Client	server from opt42, fallback to gateway
GARP	sent at every bound/renew
ARP service	initial IP 0.0.0.0, updated at bound
ICMP echo reply	replies to `ping` on the obtained address
UDP Server 7961	up to 2 simultaneous clients
USB Ethernet (NCM)	device `usb0` / `enxNN`

94.1.2 PC Prerequisite: Remove the interface from NetworkManager

When the board enumerates, a new network interface appears on the PC (`enx02ca0500xx` — the name udev derives from the MAC). NetworkManager sees it as any ordinary attached card and **assigns it an address of its own**: this has been observed `172.21.10.100/24`. The board is on `192.168.10.0/24`, so the two end up on different subnets and the board becomes unreachable.

The symptom is misleading, because the interface **exists and is UP**: `ip link` shows it, and even `ping -I <iface>` responds — that flag bypasses the routing table. What fails is everything else (`socat`, `nc`, the PC scripts of the examples), which follow the routing table.

The rule is versioned in the repository:

```
sudo cp tools/99-caos-ncm.conf /etc/NetworkManager/conf.d/
sudo systemctl reload NetworkManager
nmcli -t -f DEVICE,STATE dev status | grep enx    # it must say: unmanaged
```

The glob goes to `interface-name`, not to `mac:`. The form `mac:02:ca:05:00:00:*` seems equivalent but it is not: it requires a full MAC and fails **silently** with a wildcard character — no error, and the interface remains managed.

The address of the bank does not set this rule: it is set by `sudo ./tools/bench_net.sh usb` or the example PC script.

94.1.3 The critical point: the PC must act as a server twice

On Ethernet (ex36) the DHCP server is the LAN router, and the NTP server is typically a different host. On a point-to-point USB cable **there is nothing else but the two ends**: the PC must simultaneously act as gateway, DHCP server and NTP server.

This has a consequence that is not visible on ETH: the firmware fallback and DHCP option 42 **point to the same address**. The code chooses the NTP as (identical to ex36):

```
ca_ipv4_t ntp_ip = ca_dhcp_service_get_ntp_server();   /* opt42 */
if (ntp_ip.addr == 0U) ntp_ip = gw;                  /* fallback: the gateway */
ca_ntp_service_set_server(ntp_ip);
ca_ntp_service_net_ready();
```

On a point-to-point link the gateway **is** the PC, so both routes arrive at destination. The test remains valid, but does not distinguish the two paths: to verify that option 42 is really read it is necessary to check the log (`grep ntp /tmp/dnsmasq-caos.log`) — on ETH it would distinguish ex37, where the resolver received in the lease was different from the default.

The provided script configures both services:

```
sudo examples/example_71/test_dhcp_ntp_ncm_pc.sh
```

In ex70 **it finds the interface by itself**, searching for the one whose MAC starts with `02:ca:05` and waits up to 30 s for it to appear, since the USB interface is created during enumeration. Moreover, compared to ex70: stops the system NTP service (`ntpd`, `chrony`, `systemd-timesyncd` — it would occupy port 123), starts a minimal NTP responder in Python and adds `dhcp-option=option:ntp-server` to the dnsmasq configuration.

The Python responder **responds immediately**, without needing to be synchronized itself: here it is important that the board receives a timestamp and converts it correctly, not absolute accuracy. At exit, the script performs a complete cleanup, including restarting the system NTP service that it had stopped.

94.1.4 Hardware

MCU	`USB_NUM`	Connector	LED heartbeat
U575 (NUCLEO-U575ZI-Q)	0	USB user	PC7 (LD1)
H753 (NUCLEO-H753ZI)	1	CN14 (OTG_FS)	PB7 (LD2)
F407 (STM32F4VE)	0	USB device	PA6 (D2)
H563 (NUCLEO-H563ZI)	0	USB user	PB0 (LD1)

94.1.5 os_config.h configuration

```
#define EN_NET            1
#define EN_USB            1
#define EN_USB_NCM        1
#define EN_NET_DHCP       1     /* DHCP protocol in the net core */
#define EN_DHCP_SERVICE   1     /* task client DHCP */
#define EN_NET_NTP        1     /* NTP protocol in the net core */
#define EN_NTP_SERVICE    1     /* task client NTP */
#define EN_UDP_SERVER     1
#define EN_ARP_SERVICE    1
```

Off with respect to ex36: `EN_ETH 0`, `EN_CLI_ETH 0`, `EN_USB_CDC 0`.

```
python3 tools/caos_config.py --new 71 --from ex36 \
        --set EN_ETH=0 --set EN_USB=1 --set EN_USB_NCM=1 --set EN_USB_CDC=0
python3 tools/caos_config.py --off EN_CLI_ETH 71
```

Compared to ex70, the config differs by **exactly three flags** — `EN_NET_NTP`, `EN_NTP_SERVICE`, `EN_CLI_NTP` — verifiable with:

```
python3 tools/caos_config.py --diff 70 71
```

94.1.6 Boot Sequence

```
task_net:
  1. ca_usb_dev_mng_init/add/start      bring-up USB
  2. ca_net_init(ca_ncm_iface_get())    NCM becomes the net core iface
  3. ca_net_proto_init(&net_cfg)        stack con IP = 0.0.0.0
  4. ca_net_proto_bind_udp(68,    )       porta 68 per OFFER/ACK
  5. ca_udp_server_bind()               porta 7961
  6. ca_ntp_service_bind()              porta 123
  7. loop: as soon as the link is up     ca_dhcp_service_net_ready()     the
    DORA starts
           then ca_sem_timedwait() + ca_net_process()

on_dhcp_bound (callback, from the DHCP task):
      ca_ntp_service_set_server(opt42 | gw)
      ca_ntp_service_net_ready()          unblocks the NTP task
```

The gate at step 7 is specific to NCM, and not just a formality. In ex36 the equivalent is `ca_eth_link_poll()`: it expects the PHY to see the cable. Here we expect the host **to select alternate setting 1** of the data interface, which is what sets `s_up` in `usb_ncm.c`. As long as it is zero, `ca_usb_ncm_send_frame()` **discards every frame and returns** `false`: a DISCOVER sent too early is not *delayed*, it is **lost**.

> **Where the wait should go — and where it should not.** The gate **must be inside the loop**, not before. Blocking the net task waiting for the link *before* serving frames prevents enumeration itself: the host does not complete SET_ADDRESS and Linux reports `device not accepting address NN, error -62`. It expects an event that cannot happen until it waits. For the same reason the loop uses `ca_sem_timedwait(& ca_net_rx_sem, 100)` and not `ca_sem_wait()`: before the link is up no frames arrive, so a wait without timeout would block the task and the gate would never be re-evaluated.

Why NTP does not have its own gate

It is the difference this example adds to ex70. The NTP client **speaks first** — sends a request, does not respond to one — so in theory it would fall into the same trap as DHCP. In practice no gate is needed, for two independent reasons:

1. `ca_ntp_service_net_ready()` is called **inside** `on_dhcp_bound`: the NTP task remains blocked until a lease exists, and a lease cannot exist if the link is not up.
2. At the time of the first request the source IP is already valid — which is then the reason why the same chain exists also on ETH in ex36, where the link issue does not arise at all.

`ca_ntp_service_bind()` instead is at step 6, **before** the gate: registering port 123 does not send anything on the network, and must be done before a response can arrive.

The link monitor

`task_ncm_mon` is the NCM counterpart of `task_eth_mon` in ex36: it samples `ca_usb_ncm_is_up()` every 500 ms and reports transitions. If the link comes back up **while a lease is already active** it requests a renew. The renew also carries NTP: a new bound calls `on_dhcp_bound` and then `ca_ntp_service_net_ready()`.

94.1.7 Expected UART output

```
=== EX71: DHCP + NTP + ARP + ping + UDP server su USB-NCM ===
MAC  02:CA:05:00:00:71
IP   --- (waiting for DHCP)
waiting for the host to bring the interface up...
USB NCM UP
DHCP: avvio DORA...
DBG DHCP DISCOVER attempt 1 xid=0x05000031

DHCP BOUND  IP=192.168.10.100  mask=255.255.255.0
            GW=192.168.10.1  DNS=8.8.8.8
NTP SYNC  unix=1787299886  2026-08-21 08:11:26 UTC
UDP[7961] client 0: 192.168.10.1:51234 connesso
UDP[7961] cid=0 rx 10B ('hello caOS') da 192.168.10.1:51234
```

The order `DHCP BOUND` → `NTP SYNC` is not random and is the visual verification of the chain: if `NTP SYNC` appeared first, or without a bound, it would be a defect.

94.1.8 Technical notes

Two MACs, one per side

`s_mac` (`02:CA:05:00:00:71`) is the device MAC; `s_ncm.mac` (`02:CA:05:00:00:B1`) is the one announced to the host side — it is from this that the name `enx02ca050000b1` of the interface on the PC derives, and it is this that the script searches for. The two **must differ**.

The timestamp conversion

`unix_to_dt()` is copied identical from ex36: it converts unix time into a readable date handling leap years (complete rule, including the centennial exception). It does not use the RTC — it is pure arithmetic on the received value — therefore the comparison between the line `NTP SYNC` and the PC server log verifies end-to-end both the protocol and the conversion.

The dynamic IP and the three mirrors

At the bound, the address must be propagated to three subsystems, or they will remain misaligned:

```
ca_net_proto_set_local_ip(ip);        /* IP stack: filter for incoming packets */
ca_udp_server_set_local_ip(ip);       /* UDP server: source address of the replies */
ca_arp_service_update_local_ip(ip);   /* ARP: a quale IP rispondere */
```

It is the same sequence as in ex36 and ex70: it does not change with transport.

94.1.9 File structure

```
examples/example_71/
          ca_os_ex71.c                  entry point
          os_config_ex_71.h             configurazione (generata da ex36)
          test_dhcp_ntp_ncm_pc.sh       setup dnsmasq + NTP su usb0 + test automatico
```

Chapter 95

DHCP, NTP and DNS Client over USB-NCM

95.1 Example 72 — DHCP, NTP and DNS client over USB-NCM

95.1.1 Introduction

USB-NCM variant of **example 37** (DHCP + NTP + DNS over Ethernet). The services are the same — DHCP client with DORA and renewal, NTP client, DNS client, ARP, ICMP echo, UDP server on port 7961 — but the transport is the USB port instead of the Ethernet MAC.

Compared to example 71, a **DNS client** is added with a demo task that resolves three names as soon as the lease is active. It is the third service in the chain, and confirms the rule that emerged in ex71: **a gate on the NCM link is only needed for the first service that communicates**.

Feature	Notes
DHCP client	Full DORA + T1/T2 renewal
NTP client	server from opt42, fallback to gateway
DNS client	resolver from opt6 of the lease, fallback to gateway
DNS demo	3 resolutions at startup + CLI `dns resolve`
GARP	sent at every bound/renew
ARP service	initial IP 0.0.0.0, updated at bound
ICMP echo reply	responds to `ping` on the obtained address
UDP server 7961	up to 2 simultaneous clients
USB Ethernet (NCM)	device `usb0` / `enxNN`

95.1.2 PC Prerequisite: detach the interface from NetworkManager

When the board enumerates, a new network interface appears on the PC (`enx02ca0500xx` — the name that udev derives from the MAC). NetworkManager sees it as any other newly connected card and **assigns its own address to it**: `172.21.10.100/24` has been observed. The board is on `192.168.10.0/24`, so the two end up on different subnets and the board is unreachable.

The symptom is misleading, because the interface **exists and is UP**: `ip link` shows it, and even `ping -I <iface>` responds — that flag bypasses the routing table. Everything else (`socat`, `nc`, the PC scripts of the examples) does not work, because it follows the routing table.

The rule is versioned in the repository:

```
sudo cp tools/99-caos-ncm.conf /etc/NetworkManager/conf.d/
sudo systemctl reload NetworkManager
nmcli -t -f DEVICE,STATE dev status | grep enx    # it must say: unmanaged
```

> **The glob goes on `interface-name`, not on `mac:`.** The form `mac:02:ca:05:00:00:*` seems equivalent but it is not: it expects a complete MAC and fails **silently** with a wildcard character — no error, and the interface remains managed.

The bank address is not set by this rule: it is set by `sudo ./tools/bench_net.sh usb` or the PC script in the example.

95.1.3 The critical point: the PC must act as a server three times

On Ethernet (ex37) DHCP, NTP and DNS are typically distinct hosts on the LAN. On a point-to-point USB cable **there is nothing else but the two ends**: the PC is simultaneously gateway, DHCP server, NTP server and DNS resolver.

The provided script mounts all three services:

```
sudo examples/example_72/test_dhcp_dns_ncm_pc.sh
```

As with the other clones, it **automatically finds the interface** by searching for the one whose MAC starts with `02:ca:05` and waits up to 30 s for it to appear, because the USB interface is created upon enumeration. Then:

- stops `systemd-resolved` / `named` / `bind9` (they occupy port 53) and starts a **Python DNS resolver** that statically resolves the names `.caos.local` and forwards the rest to a public upstream;
- stops the system NTP service and starts the **Python NTP responder** from ex71;
- starts `dnsmasq` **only for DHCP** (`port=0` disables its DNS), announcing `option:dns-server` and `option:ntp-server` = PC address.

> **Beware of test ambiguity.** On point-to-point, the gateway *is* the PC, so the DHCP option and the firmware fallback point to the same address: the test passes through both paths and **does not distinguish between them**. To verify that the option is actually read, you need the log (`grep -i dns /tmp/dnsmasq-caos.log`). On ETH, this distinction was visible in ex37, where the lease resolver differed from the default.

95.1.4 Hardware

MCU	`USB_NUM`	Connector	Heartbeat LED
U575 (NUCLEO-U575ZI-Q)	0	USB user	PC7 (LD1)
H753 (NUCLEO-H753ZI)	1	CN14 (OTG_FS)	PB7 (LD2)
F407 (STM32F4VE)	0	USB device	PA6 (D2)
H563 (NUCLEO-H563ZI)	0	USB user	PB0 (LD1)

95.1.5 os_config.h configuration

```
#define EN_NET            1
#define EN_USB            1
#define EN_USB_NCM        1
#define EN_NET_DHCP       1     /* DHCP protocol in the net core */
#define EN_DHCP_SERVICE   1     /* task client DHCP */
#define EN_NET_NTP        1
#define EN_NTP_SERVICE    1
#define EN_NET_DNS        1     /* DNS protocol in the net core */
#define EN_DNS_SERVICE    1     /* task client DNS */
#define EN_UDP_SERVER     1
#define EN_ARP_SERVICE    1
```

Switched off compared to ex37: `EN_ETH 0`, `EN_CLI_ETH 0`, `EN_USB_CDC 0`.

```
python3 tools/caos_config.py --new 72 --from ex37 \
        --set EN_ETH=0 --set EN_USB=1 --set EN_USB_NCM=1 --set EN_USB_CDC=0
python3 tools/caos_config.py --off EN_CLI_ETH 72
```

Compared to ex71, the configuration differs by **exactly three flags** — `EN_NET_DNS`, `EN_DNS_SERVICE`, `EN_CLI_DNS`:

```
python3 tools/caos_config.py --diff 71 72
```

95.1.6 Boot sequence

```
task_net:
  1. ca_usb_dev_mng_init/add/start      bring-up USB
  2. ca_net_init(ca_ncm_iface_get())    NCM becomes the net core iface
  3. ca_net_proto_init(&net_cfg)        stack con IP = 0.0.0.0
  4. ca_net_proto_bind_udp(68,    )       porta 68 per OFFER/ACK
  5. ca_udp_server_bind()               porta 7961
  6. ca_ntp_service_bind()              porta 123
  7. ca_dns_service_bind()              porta 53
  8. loop: as soon as the link is up     ca_dhcp_service_net_ready()     the DORA starts
          then ca_sem_timedwait() + ca_net_process()

on_dhcp_bound (callback, from the DHCP task):
      ca_ntp_service_set_server(opt42 | gw) + net_ready()
      ca_dns_service_set_server(opt6  | gw) + net_ready()
      post su s_dns_demo_sem                 sblocca task_dns_demo
```

The step 8 gate is specific to NCM. The NCM link only comes up when the host selects alt setting 1: as long as it is down, `ca_usb_ncm_send_frame()` **discards every frame and returns** `false`, so a DISCOVER sent too early is not delayed, it is **lost**.

> **Where to place the wait — and where not to.** The gate **must be inside the loop**, not before: blocking the net task waiting for the link *before* serving frames prevents enumeration itself (the host reports `device not accepting address`). For the same reason, the loop uses `ca_sem_timedwait(&ca_net_rx_sem, 100)` and not `ca_sem_wait()`.

A single gate, for the first one to speak

This is the point that this example makes evident by having **three** services in a chain. NTP and DNS both speak first — they send a request, they do not respond to one — and in theory, they would fall into the same trap as DHCP. In practice, neither of them needs its own gate:

- `ca_ntp_service_net_ready()` and `ca_dns_service_net_ready()` are called **inside** `on_dhcp_bound`;

- `task_dns_demo` waits for `s_dns_demo_sem`, which is posted by the same callback;
- **a lease cannot exist if the link is down**, so the DHCP gate already protects the entire chain.

The `bind()` instead are all **before** the gate (steps 5–7): registering a port sends nothing over the network, and must be done before a response can arrive.

> **The `trywait` before the `post` is not a refinement.** In `on_dhcp_bound` the demo semaphore is first emptied and then reposted:
>
> ```
> ca_sem_trywait(&s_dns_demo_sem);   /* consumes any residual count */
> ca_sem_post(&s_dns_demo_sem);
> ```
>
> The semaphore is created with `max_count = 1`, and `ca_sem_post()` **triggers `CA_SAFEGUARD` if the count exceeds the maximum**. At the first bound, the demo consumes the token and everything returns to zero; but if a renewal were to arrive *before* `task_dns_demo` has started, the token would still be there and the second `post` would be an overflow — that is, a system halt, not a demo executed twice. The `trywait` makes the operation idempotent and the sequence safe at every rebind.

The link monitor

`task_ncm_mon` is the NCM counterpart to `task_eth_mon` of ex37: it samples `ca_usb_ncm_is_up()` every 500 ms and signals transitions. If the link comes back up while a lease is active, it requests a renew, which also propagates NTP and DNS by passing through `on_dhcp_bound` again.

95.1.7 Expected UART output

```
=== EX72: DHCP + NTP + DNS + ARP + ping + UDP server su USB-NCM ===
MAC  02:CA:05:00:00:72
IP   --- (waiting for DHCP)
waiting for the host to bring the interface up...
USB NCM UP
DHCP: avvio DORA...

DHCP BOUND  IP=192.168.10.100  mask=255.255.255.0
            GW=192.168.10.1  DNS=192.168.10.1
NTP SYNC  unix=1787316943  2026-08-21 12:55:43 UTC

DNS: avvio risoluzioni demo...
DNS  test.caos.local           ->  192.168.10.1
DNS  gateway.caos.local        ->  192.168.10.1
DNS  pool.ntp.org              ->  162.159.200.1
DNS: demo completato - usa CLI 'dns resolve <hostname>'
```

The first two names are resolved **statically** by the Python resolver; the third goes through **upstream forwarding**, so its success depends on the PC's Internet connectivity — if it is missing, `FAILED` is expected and is not a firmware defect.

95.1.8 Technical notes

Two MACs, one for each side

`s_mac` (`02:CA:05:00:00:72`) is the device MAC; `s_ncm.mac` (`02:CA:05:00:00:B2`) is the one announced to the host side — hence the name `enx02ca050000b2` of the interface on the PC, which is what the script looks for. The two **must differ**.

`ca_dns_resolve` is blocking

`task_dns_demo` runs on its own task (`dns_demo`, LEVEL3) and not in the net task, because `ca_dns_resolve()` **blocks** until the response or timeout (10 s here). Calling it from the net task would stop `ca_net_process()` and therefore the response itself — the same reason why `ca_dhcp_service_bind()` must not be called there.

The dynamic IP and the three mirrors

At bind, the address must be propagated to three subsystems, or they will remain misaligned:

```
ca_net_proto_set_local_ip(ip);        /* IP stack: filter for incoming packets */
ca_udp_server_set_local_ip(ip);       /* UDP server: source address of the replies */
ca_arp_service_update_local_ip(ip);   /* ARP: a quale IP rispondere */
```

It is the same sequence as ex37, ex70, and ex71: it does not change with transport.

95.1.9 File structure

```
examples/example_72/
          ca_os_ex72.c                entry point
          os_config_ex_72.h           configurazione (generata da ex37)
          test_dhcp_dns_ncm_pc.sh     setup dnsmasq + NTP + DNS su usb0 + test
```

Chapter 96

mDNS Responder over USB-NCM

96.1 Example 73 — mDNS Responder over USB-NCM

96.1.1 Introduction

Variant of **example 38** (mDNS responder over Ethernet) using USB-NCM. The services are the same — DHCP client with DORA and renewal, mDNS responder, ARP, ICMP echo, UDP server on port 7961 — but the transport is USB instead of Ethernet MAC.

The novelty compared to previous clones is the service type: mDNS is **multicast peer-to-peer**, not client-server. It is the only cloned service so far that **does not require a server on the PC**: any existing Bonjour/avahi resolver on the host interrogates it, and the board responds by itself.

Feature	Notes
DHCP Client	Full DORA + T1/T2 renewal
mDNS Responder	responds to `caos-board.local` on 224.0.0.251:5353
Gratuitous Announce	sent on every bound/renew
GARP	sent on every bound/renew
ARP Service	initial IP 0.0.0.0, updated on bound
ICMP echo reply	responds to `ping` on the obtained address
UDP Server 7961	up to 2 simultaneous clients
USB Ethernet (NCM)	device `usb0` / `enxNN`

96.1.2 PC Prerequisite: Remove the interface from NetworkManager

When the board enumerates, a new network interface (`enx02ca0500xx` — the name udev derives from the MAC) appears on the PC. NetworkManager sees it as any regular attached card and **assigns it an address**: it has been observed `172.21.10.100/24`. The board is on `192.168.10.0/24`, so the two end up on different subnets and the board becomes unreachable.

The symptom is misleading, because the interface **exists and is UP**: `ip link` shows it, and even `ping -I <iface>` responds — that flag bypasses the routing table. What fails is everything else (`socat`, `nc`, PC scripts from the examples), which follows the routing table.

The rule is versioned in the repository:

```
sudo cp tools/99-caos-ncm.conf /etc/NetworkManager/conf.d/
sudo systemctl reload NetworkManager
nmcli -t -f DEVICE,STATE dev status | grep enx    # it must say: unmanaged
```

The glob goes to `interface-name`, not to `mac:`. The form `mac:02:ca:05:00:00:*` seems equivalent but it is not: it requires a full MAC and fails **silently** with a wildcard character — no error, and the interface remains managed.

The address of the bank does not set this rule: it is set by `sudo ./tools/bench_net.sh usb` or the PC script of the example.

96.1.3 mDNS does not need a server

This is the difference with ex70/71/72. There, the PC had to act as DHCP, NTP and DNS server, because on a point-to-point cable there is no one else. mDNS instead **has no server** by definition: it is a multicast query to which the one who owns the name responds directly. The PC is only needed as *querying* device.

The script therefore mounts `dnsmasq` only for DHCP (which remains necessary to assign an address to the board) and uses avahi as **client**:

```
sudo examples/example_73/test_mdns_ncm_pc.sh
```

Find the interface from MAC `02:ca:05` by itself, then perform three independent tests:

1. `avahi-resolve -n caos-board.local` — passes through the system daemon;
2. `ping caos-board.local` — verifies that the resolution is usable by the entire PC stack, not just the tool;
3. a **raw mDNS query in Python**, which bypasses avahi entirely: this is the test that isolates the firmware from the PC configuration.

The bug that this test has already paid for once. The raw query sets `IP_MULTICAST_IF`, and that `setsockopt` **requires an address that is actually assigned to the interface**: if an IP that the NIC does not have is passed, it fails with `Errno 99` *before even sending*, and the error message would wrongly blame the card. The script therefore reads the real IP from `ip -4 -br addr show $IFACE` instead of assuming `192.168.10.1` — correction from 21/08, inherited intact from ex38.

96.1.4 Hardware

MCU	`USB_NUM`	Connector	LED heartbeat
U575 (NUCLEO-U575ZI-Q)	0	USB user	PC7 (LD1)
H753 (NUCLEO-H753ZI)	1	CN14 (OTG_FS)	PB7 (LD2)
F407 (STM32F4VE)	0	USB device	PA6 (D2)
H563 (NUCLEO-H563ZI)	0	USB user	PB0 (LD1)

96.1.5 os_config.h Configuration

```
#define EN_NET           1
#define EN_USB           1
#define EN_USB_NCM       1
#define EN_NET_DHCP      1     /* DHCP protocol in the net core */
#define EN_DHCP_SERVICE  1     /* task client DHCP */
```

```
#define EN_NET_MDNS       1     /* mDNS protocol in the net core */
#define EN_MDNS_SERVICE   1     /* task responder mDNS */
#define EN_UDP_SERVER     1
#define EN_ARP_SERVICE    1
```

Off with respect to ex38: `EN_ETH 0`, `EN_CLI_ETH 0`, `EN_USB_CDC 0`.

```
python3 tools/caos_config.py --new 73 --from ex38 \
        --set EN_ETH=0 --set EN_USB=1 --set EN_USB_NCM=1 --set EN_USB_CDC=0
python3 tools/caos_config.py --off EN_CLI_ETH 73
```

Compared to ex70, the config differs by **exactly three flags** — `EN_NET_MDNS`, `EN_MDNS_SERVICE`, `EN_CLI_MDNS`:

```
python3 tools/caos_config.py --diff 70 73
```

96.1.6 Boot Sequence

```
task_net:
  1. ca_usb_dev_mng_init/add/start      bring-up USB
  2. ca_net_init(ca_ncm_iface_get())    NCM becomes the net core iface
  3. ca_net_proto_init(&net_cfg)        stack con IP = 0.0.0.0
  4. ca_net_proto_bind_udp(68,    )       porta 68 per OFFER/ACK
  5. ca_udp_server_bind()               porta 7961
  6. ca_mdns_service_bind()             porta 5353 multicast
  7. loop: as soon as the link is up     ca_dhcp_service_net_ready()     the
    DORA starts
          then ca_sem_timedwait() + ca_net_process()

on_dhcp_bound (callback, from the DHCP task):
      ca_mdns_service_net_ready(ip)       impara l'IP e manda l'announce
    gratuito
```

The gate of step 7 is specific to NCM. The NCM link goes up only when the host selects the alt setting 1: as long as it is down, `ca_usb_ncm_send_frame()` **discards every frame and returns** `false`, so a DISCOVER sent too early is not delayed, it is **lost**.

> **Where to put the wait — and where not to.** The gate **must be inside the loop**, not before: blocking the net task in wait for the link *before* serving frames prevents enumeration itself (the host reports `device not accepting address`). For the same reason the loop uses `ca_sem_timedwait(&ca_net_rx_sem, 100)` and not `ca_sem_wait()`.

mDNS confirms the rule seen in ex71 and ex72: **only the first service that speaks needs a gate**. The gratuitous announce starts from `on_dhcp_bound`, and a lease cannot exist if the link is down. `ca_mdns_service_bind()` instead is placed before the gate, like all the `bind()`: registering port 5353 does not send anything on the network.

The link monitor

`task_ncm_mon` samples `ca_usb_ncm_is_up()` every 500 ms. If the link comes back up while a lease is active it asks for a renew — which here has an additional effect compared to other clones: **it re-announces mDNS as well**, because the announce is tied to `on_dhcp_bound`. This is the correct behavior after a cable reconnection, when the peers might have cached an stale binding.

96.1.7 Note: `task_stats` without `held`

This is the first clone that inherits a `task_stats`, and does not carry it identically. On ETH (ex38) the count is:

```
u16 fc   = ca_net_buf_free_count();
u16 held = ca_eth_pool_held_count();   /* buffers held by the descriptors */
if (fc + held != NET_BUF_COUNT)
```

`ca_eth_pool_held_count()` lives **inside** `#if EN_ETH` (`caos/drivers/src/eth.c:14`), so it is not compiled with `EN_ETH 0`. But the difference is not only compilation-related: on NCM **the concept that that function measures** does not exist. There is no DMA descriptor ring that holds buffers, so at rest the pool must simply be full:

```
u16 fc = ca_net_buf_free_count();
if (fc != NET_BUF_COUNT)       /* "in transit" */
```

On ETH the structural ceiling measured on H753 is `ETH_RX_N + ETH_TX_N` = 24 buffers held at steady state, and the leak criterion is `pool_free + held < NET_BUF_COUNT`. On NCM the criterion simplifies: **any permanent shortfall is a true leak**.

96.1.8 Expected UART Output

```
=== EX73: DHCP + mDNS + ARP + ping + UDP server su USB-NCM ===
MAC          02:CA:05:00:00:73
mDNS         caos-board.local
IP           --- (waiting for DHCP)
waiting for the host to bring the interface up...
USB NCM UP
DHCP: avvio DORA...

DHCP BOUND  IP=192.168.10.100  mask=255.255.255.0
            GW=192.168.10.1
STATS dhcp=BOUND pool=64/64 rx=12 drop=0 tx=8
UDP[7961] client 0: 192.168.10.1:51234 connesso
```

On the PC side, the success is read in the resolution:

```
[ OK ]  avahi-resolve: caos-board.local   192.168.10.100
[ OK ]  ping caos-board.local OK
[ OK ]  mDNS raw OK
```

The third one is the most significant: being independent from avahi, it demonstrates that the reply comes from the firmware and not from a PC cache.

96.1.9 Technical notes

Two MACs, one per side

`s_mac` (`02:CA:05:00:00:73`) is the device MAC; `s_ncm.mac` (`02:CA:05:00:00:B3`) is the one announced on the host side — from which the name `enx02ca050000b3` of the interface on the PC comes, the one the script looks for. The two **must differ**.

The name must match both ends

`MDNS_HOSTNAME` in the firmware and `MDNS_HOSTNAME` in the script must be the same (`caos-board`). Changing it in only one of the two causes the test to fail with an empty resolution — which is indistinguishable from a non-working responder.

The dynamic IP and the three mirrors

At binding, the address must be propagated to three subsystems, or they remain misaligned:

```
ca_net_proto_set_local_ip(ip);        /* IP stack: filter for incoming packets */
ca_udp_server_set_local_ip(ip);       /* UDP server: source address of the replies */
ca_arp_service_update_local_ip(ip);   /* ARP: a quale IP rispondere */
```

More the fourth, specific to this example: `ca_mdns_service_net_ready(ip)`, which is together address update and trigger of the announce.

96.1.10 File structure

```
examples/example_73/
          ca_os_ex73.c            entry point
          os_config_ex_73.h       configurazione (generata da ex38)
          test_mdns_ncm_pc.sh     dnsmasq (DHCP only) + avahi + raw Python query
```

Chapter 97

Syslog Service over USB-NCM

97.1 Example 74 — Syslog service over USB-NCM

97.1.1 Introduction

Variant of **example 39** (syslog client over Ethernet). The services are the same — DHCP client with DORA and renewal, sending RFC 3164 syslog messages over UDP 514, ARP, ICMP echo — but the transport is the USB port instead of the Ethernet MAC.

Compared to previous clones, this example has a feature that sets it apart: the syslog is **transmit-only**. It has no `bind()`, it listens to nothing, and it does not respond to anyone. It is the first cloned service that speaks and that's it, and this makes visible a consequence of the NCM gate that remained hidden in the others.

Feature	Notes
DHCP client	Full DORA + T1/T2 renewal
Syslog service	RFC 3164 to UDP 514, facility `local0`
ARP service	Resolves the MAC address of the syslog server
ICMP echo reply	Replies to `ping` on the acquired address
Drop counter	`ca_syslog_dropped()`, reported in stats
USB Ethernet (NCM)	Device `usb0` / `enxNN`

Message sources:

Source	Severity	Message
`on_dhcp_bound`	INFO	`EX74 BOUND IP=x.x.x.x`
`on_dhcp_expire`	WARN	`EX74 DHCP lease expired`
`task_ncm_mon`	INFO/WARN	`EX74 USB NCM link UP/DOWN`
`task_heartbeat`	DEBUG	`EX74 heartbeat seq=N pool=M` (30 s)
`task_stats`	INFO	`EX74 stats rx= tx= drop= slg_drop=` (60 s)

97.1.2 PC Prerequisite: Remove the interface from NetworkManager

When the board enumerates, a new network interface appears on the PC (`enx02ca0500xx` — the name udev derives from the MAC). NetworkManager sees it as any other newly connected card and **assigns it an address of its own**: it has been observed that `172.21.10.100/24`. The board is on `192.168.10.0/24`, so the two end up on different subnets and the board becomes unreachable.

The symptom is misleading, because the interface **exists and is UP**: `ip link` shows it, and even `ping -I <iface>` responds — that flag bypasses the routing table. What doesn't work is everything else (`socat`, `nc`, the PC scripts of the examples), which follow the routing table.

The rule is versioned in the repository:

```
sudo cp tools/99-caos-ncm.conf /etc/NetworkManager/conf.d/
sudo systemctl reload NetworkManager
nmcli -t -f DEVICE,STATE dev status | grep enx    # it must say: unmanaged
```

> **The glob goes to `interface-name`, not to `mac:`.** The form `mac:02:ca:05:00:00:*` seems equivalent but it is not: it requires a complete MAC and fails **silently** with a wildcard character — no error, and the interface remains managed.

The bank address does not set this rule: it is set by `sudo ./tools/bench_net.sh usb` or the example PC script.

97.1.3 The first "link UP" waits in queue, and arrives together with the bound

This is the point that this example makes visible. In `task_ncm_mon` the link transition sends a syslog message:

```
if (up)
    ca_syslog_send(SYSLOG_SEV_INFO, "EX74 USB NCM link UP");
```

But the **first** time the link goes up, DHCP does not yet have an address: the board is at `0.0.0.0` and the DORA is just starting. However, the message is **not lost**, and this is the difference from network frames: the syslog is decoupled from a queue.

```
void ca_syslog_send(u8 severity, const char *msg)
{

    if (!ca_queue_static_trysend(&s_syslog_q.q, &entry))
        s_dropped++;          /* only if the QUEUE is full */
}
```

`ca_syslog_send()` just queues and that's it; it is the syslog task that sends, and that task is blocked on its own gate until `ca_syslog_service_net_ready()` arrives:

```
while (!s_net_ready)
    ca_os_sleep(200U);
```

The "link UP" message therefore remains queued and **is sent as soon as the lease exists**. And since `EX74 BOUND` also goes through the same `ca_syslog_send()` — there is no synchronous shortcut for bound messages — the two are queued in the same order in which the events occurred:

```
t    task_ncm_mon sees the link come up      queues "link UP"
t    the lease arrives, on_dhcp_bound       queues "BOUND"
t    ca_syslog_service_net_ready()           the task passes the gate and drains
    the queue
```

The queue is FIFO, so on the server events arrive **in event order** — first `link UP`, then `BOUND` — but with the **same timestamp**, because they were sent in burst when the gate opened:

```
[15:21:27.971] 192.168.10.126:514  [INFO  ]  EX74 USB NCM link UP
[15:21:27.971] 192.168.10.126:514  [INFO  ]  EX74 BOUND IP=192.168.10.126
```

It is the same timestamp, not the order, the detail that reveals the decoupling: two separate events delivered at the same instant.

What `slg_drop` means. It does not count messages born without IP — those wait in the queue. It only counts those lost because **the queue was full**, that is because the task was unable to process them. A growing `slg_drop` indicates a real problem: unreachable server, ARP not resolving, or messages produced faster than the network can carry them away. It must remain at zero at steady state.

97.1.4 The PC acts as syslog server and as DHCP server

As for the other clones, on a point-to-point cable the PC must provide DHCP. In addition, here, it must also listen to syslog messages. The script does both:

```
sudo python3 examples/example_74/syslog_server_ncm.py
```

It finds the interface from MAC `02:ca:05` (waiting up to 30 s for it to appear with enumeration), assigns `192.168.10.1/24`, starts `dnsmasq` for DHCP and listens on UDP 514 decoding RFC 3164 priority and severity. On exit (Ctrl+C) it performs full cleanup and **restores the previous IP** of the interface.

Useful variants:

```
sudo python3 syslog_server_ncm.py --iface usb0    # explicit interface
sudo python3 syslog_server_ncm.py --no-dhcp       # syslog only, on 0.0.0.0
```

97.1.5 Hardware

MCU	`USB_NUM`	Connector	LED heartbeat
U575 (NUCLEO-U575ZI-Q)	0	USB user	PC7 (LD1)
H753 (NUCLEO-H753ZI)	1	CN14 (OTG_FS)	PB7 (LD2)
F407 (STM32F4VE)	0	USB device	PA6 (D2)
H563 (NUCLEO-H563ZI)	0	USB user	PB0 (LD1)

97.1.6 os_config.h Configuration

```
#define EN_NET             1
#define EN_USB             1
#define EN_USB_NCM         1
#define EN_NET_DHCP        1     /* DHCP protocol in the net core */
#define EN_DHCP_SERVICE    1     /* task client DHCP */
#define EN_NET_SYSLOG      1     /* syslog protocol in the net core */
#define EN_SYSLOG_SERVICE  1     /* task syslog */
#define EN_ARP_SERVICE     1
```

Off with respect to ex39: `EN_ETH 0`, `EN_CLI_ETH 0`, `EN_USB_CDC 0`.

```
python3 tools/caos_config.py --new 74 --from ex39 \
        --set EN_ETH=0 --set EN_USB=1 --set EN_USB_NCM=1 --set EN_USB_CDC=0
python3 tools/caos_config.py --off EN_CLI_ETH 74
```

Compared to ex70, the config differs by **five** flags, not three as in the previous clones — because ex39 does not have the UDP server that ex34-38 had:

```
EN_NET_SYSLOG      0 -> 1
EN_SYSLOG_SERVICE  0 -> 1
EN_CLI_SYSLOG      0 -> 1
EN_UDP_SERVER      1 -> 0
EN_CLI_UDP_SRV     1 -> 0
```

97.1.7 Boot Sequence

```
task_net:
  1. ca_usb_dev_mng_init/add/start      bring-up USB
  2. ca_net_init(ca_ncm_iface_get())    NCM becomes the net core iface
  3. ca_net_proto_init(&net_cfg)        stack con IP = 0.0.0.0
  4. ca_net_proto_bind_udp(68,    )       porta 68 per OFFER/ACK
  5. loop: as soon as the link is up     ca_dhcp_service_net_ready()     the
    DORA starts
          then ca_sem_timedwait() + ca_net_process()

on_dhcp_bound (callback, from the DHCP task):
      ca_syslog_service_net_ready()       unblocks the syslog task
      ca_syslog_send(INFO, "EX74 BOUND IP=   ")
```

Note the absence of a `bind()` for syslog at step 5: **there isn't one**, because the service receives nothing. It is the `bind()` list that is shorter than all the clones.

The gate of step 5 is specific to NCM. The NCM link only goes up when the host selects alternate setting 1: as long as it is down, `ca_usb_ncm_send_frame()` **discards every frame and returns** `false`, so a DISCOVER sent too early is not delayed, it is **lost**.

> **Where to put the wait — and where not to.** The gate **must be inside the loop**, not before: blocking the net task waiting for the link *before* serving frames prevents enumeration itself (the host reports `device not accepting address`). For the same reason the loop uses `ca_sem_timedwait(&ca_net_rx_sem, 100)` and not `ca_sem_wait()`.

The syslog confirms the rule from ex71/72/73: **only the first service that speaks needs a gate**. `ca_syslog_service_net_ready()` is called from `on_dhcp_bound`, and a lease cannot exist with the link down.

97.1.8 Expected output

Board console:

```
=== EX74: DHCP + Syslog service su USB-NCM ===
MAC          02:CA:05:00:00:74
IP           --- (waiting for DHCP)
Syslog       -> 192.168.10.1:514
waiting for the host to bring the interface up...
USB NCM LINK UP          <- syslog queued: no IP yet, sent after the bound
USB NCM UP
DHCP: avvio DORA...

DHCP BOUND  IP=192.168.10.100  mask=255.255.255.0
            GW=192.168.10.1
```

Server on PC (measured on H753):

```
[15:21:27.971] 192.168.10.126:514  [INFO  ]  EX74 USB NCM link UP
[15:21:27.971] 192.168.10.126:514  [INFO  ]  EX74 BOUND IP=192.168.10.126
[15:21:57.xxx] 192.168.10.126:514  [DEBUG ]  EX74 heartbeat seq=0 pool=64/64
[15:22:27.xxx] 192.168.10.126:514  [INFO  ]  EX74 stats rx=    tx=    drop=0
   slg_drop=0
```

The first two lines have the **same timestamp** even though they describe distinct events: it is the signature of the queue de-coupling, both were issued when the gate opened. `slg_drop` must remain **0**.

The times are long on purpose: heartbeat every 30 s, stats every 60 s. For a quick check, it's enough to look for the bound message, which arrives immediately.

97.1.9 Technical notes

Two MACs, one per side

`s_mac` (`02:CA:05:00:00:74`) is the device MAC; `s_ncm.mac` (`02:CA:05:00:00:B4`) is the one announced to the host side — from which the name `enx02ca050000b4` of the interface on the PC comes, the one the script looks for. The two **must be different**.

ARP is really needed here

In examples that just reply and that's it, ARP is almost implicit: one replies to whoever just wrote, and their MAC is already known. The syslog instead **starts the conversation** toward a configured address, so the server's MAC must be resolved before sending. This is the reason why `EN_ARP_SERVICE` is an explicit requirement of this example, not an accessory.

There is no UDP server

Unlike ex70–73, `EN_UDP_SERVER` is off: ex39 did not foresee it and the clone remains faithful. The UDP echo tests from the other examples here do not apply; the verification is the appearance of messages on the syslog server.

97.1.10 File structure

```
examples/example_74/
          ca_os_ex74.c            entry point
          os_config_ex_74.h       configurazione (generata da ex39)
          syslog_server_ncm.py    server syslog + DHCP con auto-detect NCM
```

Chapter 98

TCP Client over USB-NCM

98.1 Example 75 — TCP Client (HTTP GET) over USB-NCM

98.1.1 Introduction

USB-NCM variant of **example 40** (TCP client over Ethernet). The services are the same — DHCP client with DORA and renewal, ARP, TCP client that opens a connection, sends a `GET` and reads the response until closure — but the transport is the USB port instead of the Ethernet MAC.

It is the **first TCP clone** of the series: ex69–74 were all UDP. The leap is significant because TCP introduces a connection state — handshake, sequences, retransmissions, orderly closure — which must be exercised for the first time on a point-to-point USB pipe.

Feature	Notes
DHCP client	Full DORA + T1/T2 renewal
ARP service	resolves the TCP server MAC before connect
TCP client	connect → `GET` → recv until closure → close
Reconnection	every `TCP_RETRY_PERIOD_MS` (30 s) after closure or error
ICMP echo reply	responds to `ping` on the obtained address
USB Ethernet (NCM)	device `usb0` / `enxNN`

98.1.2 PC Prerequisite: detach the interface from NetworkManager

When the board enumerates, a new network interface appears on the PC (`enx02ca0500xx` — the name that udev derives from the MAC). NetworkManager sees it as any newly connected card and **assigns its own address to it**: `172.21.10.100/24` has been observed. The board is on `192.168.10.0/24`, so the two end up on different subnets and the board is unreachable.

The symptom is misleading, because the interface **exists and is UP**: `ip link` shows it, and even `ping -I <iface>` responds — that flag bypasses the routing table. Everything else fails (`socat`, `nc`, the PC scripts of the examples), which follows the routing table.

The rule is versioned in the repository:

```
sudo cp tools/99-caos-ncm.conf /etc/NetworkManager/conf.d/
sudo systemctl reload NetworkManager
nmcli -t -f DEVICE,STATE dev status | grep enx    # it must say: unmanaged
```

> **The glob goes on `interface-name`, not on `mac:`.** The form `mac:02:ca:05:00:00:*` seems equivalent but it is not: it expects a complete MAC and fails **silently** with a wildcard — no error, and the interface remains managed.

The bank address is not set by this rule: it is set by `sudo ./tools/bench_net.sh usb` or the PC script in the example.

98.1.3 What changes compared to UDP clones

With UDP, the packet either leaves or is lost, and that's it. With TCP, the connection has a state, and on the NCM path there are two points worth looking at.

ARP becomes a blocking prerequisite. `ca_tcp_connect()` cannot send the SYN before knowing the server's MAC: it resolves the ARP and only then opens. On a point-to-point link, the peer is always the PC, so the resolution is immediate — but it remains a step that, if it were to fail, would cause the connect to fail *before* any TCP traffic.

Retry is integrated into the design. The client loop retries every 30 s in case of error, and this also covers the case where the server on the PC has not yet started. There is no need to synchronize the power-on of the two sides precisely.

> **The first connect may fail — it is known and not part of this example.** On ETH it has been observed (ex42 MQTT, ex43 Modbus, 08/21) that the **first** `connect` fails and the subsequent retry succeeds. The current hypothesis is that the connect starts before the ARP has resolved the peer's MAC. If the line `TCP connect FAILED err=1` followed by a `TCP ESTABLISHED` on the next attempt also appears here, it is that known behavior of the TCP layer, not a defect of the NCM transport.

98.1.4 The PC acts as a DHCP server and a TCP server

As with the other clones, on a point-to-point cable the PC must provide DHCP; here, in addition, it must act as an HTTP server on port 8080. The script does both:

```
sudo python3 examples/example_75/tcp_server_ncm.py
```

It automatically finds the interface from the MAC `02:ca:05` (waiting up to 30 s for it to appear via enumeration), assigns it `192.168.10.1/24`, starts `dnsmasq` for DHCP, and listens on TCP 8080, responding to every `GET` with a text body. Upon exit, it performs cleanup and **restores the previous IP** of the interface.

```
sudo python3 tcp_server_ncm.py --iface usb0     # explicit interface
sudo python3 tcp_server_ncm.py --no-dhcp        # TCP only, on 0.0.0.0
sudo python3 tcp_server_ncm.py --port 9090      # different port (align the
    firmware)
```

98.1.5 Hardware

MCU	`USB_NUM`	Connector	Heartbeat LED
U575 (NUCLEO-U575ZI-Q)	0	USB user	PC7 (LD1)
H753 (NUCLEO-H753ZI)	1	CN14 (OTG_FS)	PB7 (LD2)
F407 (STM32F4VE)	0	USB device	PA6 (D2)
H563 (NUCLEO-H563ZI)	0	USB user	PB0 (LD1)

98.1.6 os_config.h configuration

```
#define EN_NET            1
#define EN_USB            1
#define EN_USB_NCM        1
#define EN_NET_DHCP       1     /* DHCP protocol in the net core */
#define EN_DHCP_SERVICE   1     /* task client DHCP */
#define EN_TCP_SERVICE    1     /* stack TCP */
#define EN_ARP_SERVICE    1
```

Off compared to ex40: `EN_ETH 0`, `EN_CLI_ETH 0`, `EN_USB_CDC 0`.

```
python3 tools/caos_config.py --new 75 --from ex40 \
        --set EN_ETH=0 --set EN_USB=1 --set EN_USB_NCM=1 --set EN_USB_CDC=0
python3 tools/caos_config.py --off EN_CLI_ETH 75
```

Compared to ex70, the configuration differs by **four** flags: `EN_TCP_SERVICE` and `EN_CLI_TCP` enabled, `EN_UDP_SERVER` and `EN_CLI_UDP_SRV` disabled (ex40 did not have the UDP server).

98.1.7 Startup sequence

```
task_net:
  1. ca_usb_dev_mng_init/add/start      bring-up USB
  2. ca_net_init(ca_ncm_iface_get())    NCM becomes the net core iface
  3. ca_net_proto_init(&net_cfg)        stack con IP = 0.0.0.0
  4. ca_net_proto_bind_udp(68,    )       porta 68 per OFFER/ACK
  5. loop: as soon as the link is up     ca_dhcp_service_net_ready()     the
    DORA starts
           then ca_sem_timedwait() + ca_net_process()

on_dhcp_bound (callback, from the DHCP task):
      ca_sem_trywait/post(&s_dhcp_sem)    sblocca task_tcp

task_tcp:
  ca_sem_wait(&s_dhcp_sem)
  loop: connect     send GET     recv until close     close     sleep 30 s
```

No `bind()` for TCP at step 5: the client **opens** the connection, it does not listen. This is the same reason why the syslog of ex74 did not have any.

The gate at step 5 is specific to NCM. The NCM link only comes up when the host selects alt setting 1: as long as it is down, `ca_usb_ncm_send_frame()` **discards every frame and returns** `false`, so a DISCOVER sent too early is not delayed, it is **lost**.

> **Where to place the wait — and where not to.** The gate **must be inside the loop**, not before: blocking the net task waiting for the link *before* serving frames prevents enumeration itself (the host reports `device not accepting address`). For the same reason, the loop uses `ca_sem_timedwait(&ca_net_rx_sem, 100)` and not `ca_sem_wait()`.

The TCP client confirms the rule: **a gate is only needed for the first service that speaks**. `task_tcp` waits for `s_dhcp_sem`, posted by `on_dhcp_bound`, and a lease cannot exist with the link down.

> **The `trywait` before the `post`.** The semaphore has `max_count = 1` and `ca_sem_post()` triggers `CA_SAFEGUARD` in overflow. If a lease renewal arrived before `task_tcp` had consumed the token, the second `post` would be a system halt. The `trywait` drains any residue and makes the pair idempotent — a precaution added in this clone, not present in ex40.

98.1.8 Expected output

Board console:

```
=== EX75: TCP client HTTP GET su USB-NCM ===
MAC          02:CA:05:00:00:75
TCP server   192.168.10.1:8080
waiting for the host to bring the interface up...
USB NCM UP
DHCP: avvio DORA...

DHCP BOUND  IP=192.168.10.100
TCP: DHCP bound, avvio client loop

TCP connect -> 192.168.10.1:8080
TCP ESTABLISHED  h=1  ok=1
TCP send GET  OK
TCP rx 167 byte:
HTTP/1.0 200 OK

TCP session done  total_rx=167 byte
```

On the PC side, the server records the connection with **source port** and bytes sent: comparing the two logs provides the end-to-end verification, just as on ex40 where the source port and count matched exactly.

98.1.9 Technical notes

Two MACs, one for each side

`s_mac` (`02:CA:05:00:00:75`) is the device MAC; `s_ncm.mac` (`02:CA:05:00:00:B5`) is the one announced to the host side — hence the name `enx02ca050000b5` of the interface on the PC, which is what the script looks for. The two **must differ**.

ca_tcp_recv is blocking, and resides on its own task

`task_tcp` is a separate task (LEVEL3) and not the net task, because both `ca_tcp_connect()` and `ca_tcp_recv()` **block**. Calling them from the net task would stop `ca_net_process()` — that is, exactly what needs to run to deliver incoming segments — and the connection would never progress. It is the same constraint as `ca_dns_resolve()` in ex72 and `ca_dhcp_service_bind()` everywhere else.

The read cycle

```
while (ca_tcp_is_connected(h)) {
    bool ok = ca_tcp_recv(h, rx_buf, sizeof(rx_buf) - 1, &got,
    TCP_RECV_TIMEOUT_MS);
    if (!ok || got == 0U) break;

}
```

The exit condition is twofold: the closure seen from the connection state *or* a `recv` that produces no data. Both are needed because the HTTP/1.0 response ends with the server closing the connection (`Connection: close`), and depending on when the FIN arrives, one of the two conditions triggers first.

98.1.10 File structure

```
examples/example_75/
          ca_os_ex75.c            entry point
          os_config_ex_75.h       configurazione (generata da ex40)
          tcp_server_ncm.py       server HTTP 8080 + DHCP con auto-detect NCM
```

Chapter 99

TCP Echo Server over USB-NCM

99.1 Example 76 — TCP echo server over USB-NCM

99.1.1 Introduction

Variant USB-NCM of **example 41** (TCP server over Ethernet). The services are the same — DHCP client, ARP, TCP echo server RFC 862 on port 7 — but the transport is the USB port instead of the Ethernet MAC.

It is the **reverse role of ex75**: there the board opened the connection to the PC, here the board stays in `accept` and the PC connects. Together the two examples cover TCP in both directions on NCM.

Feature	Notes
DHCP client	Full DORA + T1/T2 renewal
ARP service	resolves client MAC
TCP port 7 server	RFC 862 echo, one connection at a time
Blocking accept	`ca_tcp_accept()` with timeout, retry in loop
ICMP echo reply	replies to `ping` on the acquired address
USB Ethernet (NCM)	device `usb0` / `enxNN`

99.1.2 PC prerequisite: remove the interface from NetworkManager

When the board enumerates, a new network interface appears on the PC (`enx02ca0500xx` — the name udev derives from the MAC). NetworkManager sees it as any ordinary attached card and **assigns it an address of its own**: it has been observed `172.21.10.100/24`. The board is on `192.168.10.0/24`, so the two end up on different subnets and the board is unreachable.

The symptom is misleading, because the interface **exists and is UP**: `ip link` shows it, and even `ping -I <iface>` responds — that flag bypasses the routing table. What doesn't work is everything else (`socat`, `nc`, the PC scripts of the examples), which follow the routing table.

The rule is versioned in the repository:

```
sudo cp tools/99-caos-ncm.conf /etc/NetworkManager/conf.d/
sudo systemctl reload NetworkManager
nmcli -t -f DEVICE,STATE dev status | grep enx    # it must say: unmanaged
```

> **The glob goes to `interface-name`, not to `mac:`.** The form `mac:02:ca:05:00:00:*` seems equivalent but it is not: it requires a full MAC and fails **silently** with a wildcard character — no error, and the interface remains managed.

The bank address does not set this rule: it is set by `sudo ./tools/bench_net.sh usb` or the example PC script.

99.1.3 The real difference: where `ca_tcp_listen()` is located

This clone is not a mechanical transposition like the previous ones, because in ex41 the listen is in a location that on NCM **is not reachable**.

In ex41:

```
void ca_os_main(void)
{
    ca_tcp_service_start();
    /* listen with temporary IP 0.0.0.0 */
    s_lh = ca_tcp_listen(&lcfg);       /* <-- BEFORE ca_os_run() */
    ca_os_job("net",    );
    ca_os_run(true);
}
```

Here it works because on ETH `ca_net_init()` has already been called in `ca_os_configure()`: the network core exists before the scheduler starts.

On NCM **it cannot be so**. The NCM interface exists only after USB enumeration, so `ca_net_init()` lives inside `task_net` — and a `ca_tcp_listen()` in `ca_os_main()` would operate on a stack that has not yet been initialized. The listen is therefore moved to `task_net`, right after `ca_net_proto_init()`:

```
ca_net_proto_init(&net_cfg);
ca_net_proto_bind_udp(DHCP_CLIENT_PORT, dhcp_udp_handler);
s_lh = ca_tcp_listen(&lcfg);          /* 0.0.0.0, before the gate */
```

It is consistent with the rule that applies to all `bind()` of previous clones: **listening is passive, it puts nothing on the wire**, therefore it should be done before the link gate. What must come after is only what *transmits*.

The double listen

Listening is opened twice, and this is not a mistake:

```
/* in task_net: first listen, IP still 0.0.0.0 */
s_lh = ca_tcp_listen(&lcfg);

/* in on_dhcp_bound: re-armed with the real address */
ca_tcp_unlisten(s_lh);
lcfg.local_ip = ip;
s_lh = ca_tcp_listen(&lcfg);
```

The listening PCB carries with it the local address on which to accept. Open at `0.0.0.0` before the lease, it must be regenerated when the address arrives, or it would accept on an address that the board does not have. It is the same mechanism as in ex41 — on ETH as on NCM the problem is identical, because it stems from DHCP, not from the transport.

> The re-arm happens inside `on_dhcp_bound`, so **every renewal** regenerates the listening. It is intentional: if the lease were to change address, the server would follow.

99.1.4 The PC is the client

Reversal compared to the other clones: here the PC does not act as an application server, but connects to the board. It must still provide **DHCP**, because on a point-to-point cable there is no one else to provide it.

```
sudo python3 examples/example_76/tcp_client_ncm.py
```

The script finds the interface from MAC `02:ca:05`, assigns `192.168.10.1/24`, starts `dnsmasq`, **reads the assigned IP from the log** and connects to its port 7. Then every typed line is sent and read back from the echo.

```
sudo python3 tcp_client_ncm.py --iface usb0                  # explicit
    interface
sudo python3 tcp_client_ncm.py --host 192.168.10.128 --no-dhcp   # DHCP already
    running
```

Type `quit` to close the connection.

99.1.5 Hardware

MCU	`USB_NUM`	Connector	LED heartbeat
U575 (NUCLEO-U575ZI-Q)	0	USB user	PC7 (LD1)
H753 (NUCLEO-H753ZI)	1	CN14 (OTG_FS)	PB7 (LD2)
F407 (STM32F4VE)	0	USB device	PA6 (D2)
H563 (NUCLEO-H563ZI)	0	USB user	PB0 (LD1)

99.1.6 os_config.h configuration

```
#define EN_NET            1
#define EN_USB            1
#define EN_USB_NCM        1
#define EN_NET_DHCP       1
#define EN_DHCP_SERVICE   1
#define EN_TCP_SERVICE    1
#define EN_ARP_SERVICE    1
```

Off compared to ex41: `EN_ETH 0`, `EN_CLI_ETH 0`, `EN_USB_CDC 0`.

```
python3 tools/caos_config.py --new 76 --from ex41 \
        --set EN_ETH=0 --set EN_USB=1 --set EN_USB_NCM=1 --set EN_USB_CDC=0
python3 tools/caos_config.py --off EN_CLI_ETH 76
```

The config is **identical to that of ex75** (`caos_config.py --diff 75 76` does not report differences): client and server TCP turn on the same flags, the only thing that changes is the code. The accept timeout is a config parameter, not a source parameter:

```
#define TCP_TCP_ACCEPT_TIMEOUT_MS     10000U
```

99.1.7 Boot Sequence

```
task_net:
  1. ca_usb_dev_mng_init/add/start      bring-up USB
  2. ca_net_init(ca_ncm_iface_get())    NCM becomes the net core iface
  3. ca_net_proto_init(&net_cfg)        stack con IP = 0.0.0.0
  4. ca_net_proto_bind_udp(68,    )       porta 68 per OFFER/ACK
  5. ca_tcp_listen(0.0.0.0:7)           ascolto passivo, ri-armato al bound
  6. loop: as soon as the link is up     ca_dhcp_service_net_ready()     the
    DORA starts
          then ca_sem_timedwait() + ca_net_process()

on_dhcp_bound (callback, from the DHCP task):
      unlisten + listen again, with the real IP
      ca_sem_trywait/post(&s_dhcp_sem)    sblocca task_server

task_server:
  ca_sem_wait(&s_dhcp_sem)
  loop: accept (timeout 10 s)     recv/send in echo     close
```

The server does not need any link gate: **it only responds**, and when a client manages to reach it, the link is necessarily up. This is the same reason why ex69 (UDP server) did not need it.

> **The `trywait` before the `post`.** The semaphore has `max_count = 1` and `ca_sem_post()` triggers `CA_SAFEGUARD` on overflow. If a renewal arrives before `task_server` has consumed the token, the second `post` would be a halt. This precaution was added in this clone, as in ex75.

99.1.8 Expected output

Board console:

```
=== EX76: TCP echo server su USB-NCM ===
MAC     02:CA:05:00:00:76
Port    TCP:7  (RFC 862 echo)
waiting for the host to bring the interface up...
USB NCM UP
DHCP: avvio DORA...

DHCP BOUND  IP=192.168.10.100
TCP echo server on port 7
TCP server: listening on port 7
TCP accept  h=1  conn#1
  echo 5 byte: ciao
TCP close  h=1  session=5 byte  total=5 byte
```

On the PC side, each typed line returns unchanged. The comparison between the bytes counted by the board (`session=N`) and those sent by the client constitutes the end-to-end verification.

99.1.9 Technical Notes

Two MAC addresses, one for each side

`s_mac` (`02:CA:05:00:00:76`) is the device MAC; `s_ncm.mac` (`02:CA:05:00:00:B6`) is the one announced on the host side — from which the name `enx02ca050000b6` of the interface on the PC comes, which the script searches for. The two **must be different**.

The receive loop distinguishes between timeout and closure

```
bool ok = ca_tcp_recv(h, buf,    , &got, ECHO_RECV_TIMEOUT);
if (!ok) {
    if (ca_tcp_is_connected(h)) continue;   /* timeout only: retry */
    break;                                   /* the peer has closed */
}
```

A failed `recv` does not mean session end: with an interactive client typing slowly the 5 s timeout keeps firing continuously. The distinction is made by `ca_tcp_is_connected()`. Without this check the server would close the connection at every typing pause.

One connection at a time

`ca_tcp_accept()` is blocking and the loop serves one session at a time until closure. A second client waiting is accepted only after. This is the design of ex41, kept: to serve multiple connections in parallel a task per session would be needed, as the UDP server of ex69/70 does with its worker per-client.

99.1.10 File structure

```
examples/example_76/
          ca_os_ex76.c            entry point
          os_config_ex_76.h       configurazione (generata da ex41)
          tcp_client_ncm.py       client echo + DHCP con auto-detect NCM
```

Chapter 100

MQTT Client over USB-NCM

100.1 Example 77 — MQTT Client over USB-NCM

100.1.1 Introduction

Variant of **example 42** (MQTT client over Ethernet) using USB-NCM. The services are the same — DHCP client, ARP, MQTT v3.1.1 client that publishes status and receives commands — but the transport is USB instead of Ethernet MAC.

This is the first clone that implements an **application protocol over TCP**: ex75 and ex76 exercised raw TCP, here there is an MQTT session with CONNECT, SUBSCRIBE, PUBLISH and keepalive to be maintained over time.

Feature	Notes
DHCP Client	Full DORA + T1/T2 renewal
ARP Service	Resolves broker's MAC address
Publish	`caos/status` every 5 s, JSON payload with `seq`/`up`/`ip`
Subscribe	`caos/led` → `on`/`off` → LED
Keepalive	PING every 30 s (under `MQTT_KEEPALIVE_SEC` = 60)
Reconnection	Automatic every 8 s after disconnection
USB Ethernet (NCM)	Device `usb0` / `enxNN`

100.1.2 PC Prerequisite: Remove the interface from NetworkManager

When the board enumerates, a new network interface appears on the PC (`enx02ca0500xx` — the name udev derives from the MAC). NetworkManager sees it as any other newly connected card and **assigns it an address**: it has been observed `172.21.10.100/24`. The board is on `192.168.10.0/24`, so the two end up on different subnets and the board becomes unreachable.

The symptom is misleading, because the interface **exists and is UP**: `ip link` shows it, and even `ping -I <iface>` responds — that flag bypasses the routing table. What doesn't work is everything else (`socat`, `nc`, PC scripts from examples), which follow the routing table.

The rule is versioned in the repository:

```
sudo cp tools/99-caos-ncm.conf /etc/NetworkManager/conf.d/
sudo systemctl reload NetworkManager
nmcli -t -f DEVICE,STATE dev status | grep enx    # it must say: unmanaged
```

> **The glob goes to `interface-name`, not to `mac:`.** The form `mac:02:ca:05:00:00:*` seems equivalent but it is not: it requires a full MAC and fails **silently** with a wildcard character — no error, and the interface remains managed.

The bank address does not set this rule: it is set by `sudo ./tools/bench_net.sh usb` or the example PC script.

100.1.3 The broker is the gateway

On a point-to-point link the PC acts as gateway, DHCP server and MQTT broker. The firmware derives the broker address **from the lease**:

```
ca_ipv4_t broker_ip = s_gw;   /* the DHCP gateway: on p2p it is the PC */
```

It is not an NCM adaptation — it is what ex42 was already doing, where the test PC was also the gateway of the subnet. The advantage is that there is no hardcoded address in the firmware: changing the PC's subnet, the board follows suit.

This means that `ca_mqtt_service_init()` **cannot be in** `ca_os_configure()`: it needs the broker, which is only known after the bind. It therefore lives inside `task_mqtt`, after waiting on the semaphore — as in ex42.

100.1.4 The test that this example carries with it

On ETH it was observed (21/08, ex42 MQTT and ex43 Modbus) that **the first** `connect` **TCP always fails** and the automatic retry succeeds:

```
MQTT DISCONNECTED - riconnessione in corso...
MQTT: tentativo riconnessione...
MQTT CONNECTED  (#1)
```

The current assumption is that the connect starts before ARP has resolved the MAC address of the peer. On **ex75 (TCP client on NCM)**, this phenomenon did not occur: connection succeeded on the first attempt.

This example is therefore discriminant:

- if the first connect **fails here too**, the cause lies in the MQTT client or in the opening sequence, not in the transport layer;
- if it **does not fail**, it remains specific to the ETH path — and the difference with ex75 (which connects to a server already listening) must be sought elsewhere.

In neither case is this a defect of this clone: the automatic reconnection works by design.

100.1.5 Hardware

MCU	`USB_NUM`	Connector	LED MQTT (`caos/led`)	Heartbeat A	Heartbeat B
U575 (NUCLEO-U575ZI-Q)	0	USB user	PC7 (LD1)	PB7	PB14
H753 (NUCLEO-H753ZI)	1	CN14 (OTG_FS)	PB0 (LD1 green)	PB14 (LD3 red)	PB7 (LD2 blue)

MCU	USB_NUM	Connector	LED MQTT (caos/led)	Heartbeat A	Heartbeat B
F407 (STM32F4VE)	0	USB device	PA7 (D3)	PA6 (D2)	PA6 (single)
H563 (NUCLEO-H563ZI)	0	USB user	PB0 (LD1)	PF4 (LD2)	PG4 (LD3)

Three LEDs, three roles

The **green** follows `caos/led`. The **red** and **blue** perform heartbeat in **exclusive alternation**: when one is on, the other is off, and at each beat they swap.

The alternation is not achieved with two independent `toggle`, but by writing both pins together with opposite values:

```
hb = !hb;
ca_gpio_write(HB_A_GPIO_PORT, HB_A_GPIO_PIN, hb ? 1U : 0U);
ca_gpio_write(HB_B_GPIO_PORT, HB_B_GPIO_PIN, hb ? 0U : 1U);
```

With two separate `toggle`, the invariant "one on, one off" would depend on the initial state and could break (a fault, a task restart, a missed cycle): the two LEDs would end up on or off together, and from then on would remain in phase. By writing `hb` and `!hb`, the invariant holds **by construction at every cycle**, regardless of what happened before.

> **Why the heartbeat has its own dedicated pin.** If the heartbeat shared the pin with the MQTT command, every heartbeat write would overwrite the requested state and *on/off* would cease to be readable. On NUCLEO-H753ZI the LEDs are LD1 green PB0, LD2 blue PB7, LD3 red PB14; ex42 on ETH used PB0 for the command, and this clone maintains it.
>
> **On F4VE the alternation does not exist**: the board has only one free LED, so `HB_B` coincides with `HB_A` and the heartbeat remains a simple on/off.
>
> **Where the GPIO is configured.** All three pins are configured in `ca_os_configure()`, before any task starts. This is not a stylistic detail: the LEDs are driven by **different contexts** — the MQTT one from `on_mqtt_message()` (which runs in the MQTT service task), the heartbeat from `task_ncm_mon`. Leaving the `ca_gpio_configure()` inside one of the two tasks, the other could write to a pin not yet configured as output, and the first command would be lost. In other NCM clones this problem does not arise because only one task touches the LED and configures it by itself.
>
> Explicit port initialization is not needed instead: `ca_gpio_configure()` calls `ca_gpio_hw_init_port()` first, which enables the RCC clock of the port (with read-back of the register required by erratum RM0433 against BusFault IMPRECISERR) and is idempotent.

100.1.6 os_config.h Configuration

```
#define EN_NET            1
#define EN_USB            1
#define EN_USB_NCM        1
#define EN_NET_DHCP       1
#define EN_DHCP_SERVICE   1
#define EN_TCP_SERVICE    1     /* MQTT runs on top of TCP */
#define EN_MQTT_SERVICE   1
#define EN_ARP_SERVICE    1
```

Off with respect to ex42: `EN_ETH 0`, `EN_CLI_ETH 0`, `EN_USB_CDC 0`.

```
python3 tools/caos_config.py --new 77 --from ex42 \
        --set EN_ETH=0 --set EN_USB=1 --set EN_USB_NCM=1 --set EN_USB_CDC=0
python3 tools/caos_config.py --off EN_CLI_ETH 77
```

Compared to ex75 (TCP client), it differs by **only two flags**: `EN_MQTT_SERVICE` and `EN_CLI_MQTT`. The rest of the stack is identical, as is appropriate for a protocol that simply sits above TCP.

100.1.7 The PC: MQTT broker + DHCP

```
sudo python3 examples/example_77/mqtt_server_ncm.py
```

Find the interface from MAC `02:ca:05`, assign `192.168.10.1/24`, start `dnsmasq` for DHCP and `mosquitto` as broker, and connect a client `paho-mqtt` that prints messages to `caos/status` and allows sending commands.

Interactive commands after startup: `on` and `off` publish to `caos/led`.

```
apt install mosquitto dnsmasq
pip3 install paho-mqtt
```

100.1.8 Boot Sequence

```
task_net:
  1. ca_usb_dev_mng_init/add/start      bring-up USB
  2. ca_net_init(ca_ncm_iface_get())    NCM becomes the net core iface
  3. ca_net_proto_init(&net_cfg)        stack con IP = 0.0.0.0
  4. ca_net_proto_bind_udp(68,    )       porta 68 per OFFER/ACK
  5. loop: as soon as the link is up     ca_dhcp_service_net_ready()     the
    DORA starts
          then ca_sem_timedwait() + ca_net_process()

on_dhcp_bound (callback, from the DHCP task):
      s_gw stored (= broker address)
      ca_sem_trywait/post(&s_dhcp_sem)    sblocca task_mqtt

task_mqtt:
  ca_sem_wait(&s_dhcp_sem)
  ca_mqtt_service_init(broker = s_gw,    )
  ca_mqtt_service_net_ready()           apre TCP     CONNECT MQTT
  loop: publish every 5 s    ping every 30 s    reconnect every 8 s if dropped
```

No `bind()` for MQTT: it's a client, it opens its own TCP connection to the broker. The gate on the link is only needed for DHCP, which is the first to speak.

The gate of step 5 is specific to NCM. The NCM link goes up only when the host selects alternate setting 1: as long as it's down, `ca_usb_ncm_send_frame()` **discards every frame and returns** `false`, so a DISCOVER sent too early is not delayed, it's **lost**.

> **Where to put the wait — and where not to.** The gate **must be inside the loop**, not before: blocking the net task waiting for the link *before* serving frames prevents enumeration itself (the host reports `device not accepting address`). For the same reason the loop uses `ca_sem_timedwait(&ca_net_rx_sem, 100)` and not `ca_sem_wait()`.

100.1.9 Expected output

Board console:

```
=== EX77: MQTT client su USB-NCM ===
MAC     02:CA:05:00:00:77
Topics  pub 'caos/status'  sub 'caos/led'
waiting for the host to bring the interface up...
USB NCM UP
DHCP: avvio DORA...

DHCP BOUND  IP=192.168.10.100  GW=192.168.10.1
MQTT broker 192.168.10.1:1883
MQTT: avvio connessione...
MQTT CONNECTED  (#1)
MQTT subscribed -> caos/led
MQTT pub -> {"seq":0,"up":12,"ip":"192.168.10.100"}
MQTT LED -> ON
```

On the PC side, client `paho` shows the same payloads. The end-to-end verification is **bidirectional**: `seq` that advances without skipping from the board to the PC, and the command `on`/`off` that from the PC produces line `MQTT LED ->` on the board.

100.1.10 Technical notes

Two MACs, one for each side

`s_mac` (`02:CA:05:00:00:77`) is the device MAC; `s_ncm.mac` (`02:CA:05:00:00:B7`) is the one announced on the host side — from which the name `enx02ca050000b7` of the interface on the PC comes, the one that the script searches for. The two **must be different**.

Topics and payloads are not null-terminated

```
if (topic_len == (u16)(sizeof(MQTT_TOPIC_LED) - 1U) &&
    ca_memcmp(topic, MQTT_TOPIC_LED, topic_len) == 0)
```

The callback receives pointer **and length**, and the data points inside the network buffer: there is no trailing zero. Comparing with `ca_strcmp` would read past the end. It is the same constraint as in ex42, and applies to any callback that exposes raw data from the pool.

Reconnection reinitializes the service

In the reconnection cycle `ca_mqtt_service_init()` is called before `ca_mqtt_service_net_ready()`, with the local IP re-read from `s_ip`. This is needed because a DHCP renewal might have changed the address: starting with the old one would produce packets with wrong source.

Keepalive is not optional

The PING every 30 s is under the 60 s declared in CONNECT (`MQTT_KEEPALIVE_SEC`). If the ping period were to exceed the keepalive, the broker would close the session due to inactivity, and the reconnection cycle would restart continuously — a fault that manifests only after a minute of silence, hence easy to miss in a short test.

100.1.11 File structure

```
examples/example_77/
          ca_os_ex77.c            entry point
          os_config_ex_77.h       configurazione (generata da ex42)
          mqtt_server_ncm.py      mosquitto + paho + DHCP con auto-detect NCM
```

Chapter 101

Modbus TCP Client over USB-NCM

101.1 Example 78 — Modbus TCP Client over USB-NCM

101.1.1 Introduction

Variant USB-NCM of **example 43** (Modbus TCP client over Ethernet). The services are the same — DHCP client, ARP, Modbus TCP client that performs a cycle of four function codes — but the transport is the USB port instead of the Ethernet MAC.

It is the second application protocol over TCP after MQTT (ex77), and it is the most demanding in the series in terms of verification: each cycle performs **reads and writes**, and values must be compared on both sides.

Feature	Notes
DHCP Client	Full DORA + T1/T2 renewal
ARP service	resolves the MAC address of the Modbus server
FC03	reads 4 holding registers (addr 0..3)
FC06	writes a single register (addr 4, setpoint 100→1000)
FC16	writes 2 registers (addr 5..6)
FC01	reads 8 coils (addr 0..7)
Cycle	every `MODBUS_POLL_PERIOD_MS` (3 s)
Reconnection	automatic every 5 s if TCP drops
USB Ethernet (NCM)	device `usb0` / `enxNN`

101.1.2 PC Prerequisite: Remove the interface from NetworkManager

When the board enumerates, a new network interface appears on the PC (`enx02ca0500xx` — the name udev derives from the MAC). NetworkManager sees it as any other newly connected card and **assigns it an address of its own**: it has been observed `172.21.10.100/24`. The board is on `192.168.10.0/24`, so the two end up on different subnets and the board becomes unreachable.

The symptom is misleading, because the interface **exists and is UP**: `ip link` shows it, and even `ping -I <iface>` responds — that flag bypasses the routing table. What fails is everything else (`socat`, `nc`, PC scripts from the examples), which follow the routing table.

The rule is versioned in the repository:

```
sudo cp tools/99-caos-ncm.conf /etc/NetworkManager/conf.d/
sudo systemctl reload NetworkManager
nmcli -t -f DEVICE,STATE dev status | grep enx    # it must say: unmanaged
```

The glob goes to `interface-name`, not to `mac:`. The form `mac:02:ca:05:00:00:*` seems equivalent but it is not: it requires a full MAC and fails **silently** with a wildcard character — no error, and the interface remains managed.

The bank address does not set this rule: it is set by `sudo ./tools/bench_net.sh usb` or the example PC script.

101.1.3 The server is the gateway

As for MQTT in ex77, on a point-to-point link the PC acts as gateway, DHCP server and Modbus server. The firmware derives the server address **from the lease**:

```
ca_ipv4_t server_ip = s_gw;   /* the DHCP gateway: on p2p it is the PC */
```

Therefore, `ca_modbus_service_init()` cannot be in `ca_os_configure()`: it needs the server address, only known after the bind. It therefore lives in `task_modbus`, after waiting on the semaphore — as in ex43.

101.1.4 The confirmation on the first connect

On ETH it is documented (21/08) that **the first TCP `connect` always fails** and the retry succeeds: observed on ex42 (MQTT) *and* ex43 (Modbus). On NCM this phenomenon **has not occurred** neither in ex75 (TCP client) nor in ex77 (MQTT).

This example is the confirmation for the second protocol. If the first connection succeeds here too, the picture is:

	ETH	NCM
Raw TCP	—	first attempt (ex75)
MQTT	first fails (ex42)	first attempt (ex77)
Modbus	first fails (ex43)	← this example

Since the client code is **identical** between ex43 and ex78 (only the transport changes), a positive outcome here definitively excludes the client and the opening sequence, and narrows the cause to the ETH path — where the investigation will have to shift to ARP and driver.

The symptom to look for in the log is:

```
MODBUS: tentativo riconnessione...
```

If it appears **before** the first successful poll cycle, the first connect failed.

101.1.5 Hardware

MCU	`USB_NUM`	LED OK (transaction)	Heartbeat A	Heartbeat B
U575 (NUCLEO-U575ZI-Q)	0	PC7 (LD1 green)	PG2 (LD3 red)	PB7 (LD2 blue)
H753 (NUCLEO-H753ZI)	1	PB0 (LD1 green)	PB14 (LD3 red)	PE1 (LD2 yellow)
F407 (STM32F4VE)	0	PA7 (D3)	PA6 (D2)	PA6 (single)
H563 (NUCLEO-H563ZI)	0	PB0 (LD1 green)	PG4 (LD3 red)	PF4 (LD2 yellow)

The **LED OK** blinks 50 ms at each cycle completed without errors: it is an immediate visual feedback that the FC03→FC06→FC16→FC01 chain has passed entirely.

The two heartbeat LEDs are in **exclusive alternation** — one on, the other off, they swap at each beat. They are written together with opposite values, not with two independent `toggle` that could desynchronize.

> **Pinout verified on manuals** (UM2407 for H753, UM2861 for U575, UM3115 for H563), not deduced. Warning: **PB7 is the LD2 blue LED of the U575 and is not a LED on the H753**, where LD2 is yellow on PE1.

101.1.6 os_config.h Configuration

```
#define EN_NET             1
#define EN_USB             1
#define EN_USB_NCM         1
#define EN_NET_DHCP        1
#define EN_DHCP_SERVICE    1
#define EN_TCP_SERVICE     1     /* Modbus runs on top of TCP */
#define EN_MODBUS_SERVICE  1
#define EN_ARP_SERVICE     1
```

Off with respect to ex43: `EN_ETH 0`, `EN_CLI_ETH 0`, `EN_USB_CDC 0`.

```
python3 tools/caos_config.py --new 78 --from ex43 \
        --set EN_ETH=0 --set EN_USB=1 --set EN_USB_NCM=1 --set EN_USB_CDC=0
python3 tools/caos_config.py --off EN_CLI_ETH 78
```

Compared to ex75 (TCP client), it differs by **only two flags**: `EN_MODBUS_SERVICE` and `EN_CLI_MODBUS` — the same distance that ex77 has from ex75 for MQTT.

101.1.7 The PC: Modbus server + DHCP

```
sudo python3 examples/example_78/modbus_server_ncm.py
```

Find the interface from MAC `02:ca:05`, assign `192.168.10.1/24`, start `dnsmasq`, wait for the board to obtain the lease and open a **simulated Modbus TCP server** on port 502. The registers have values that change over time, so a FC03 that always returned the same numbers would reveal a cache instead of a real read.

```
sudo python3 modbus_server_ncm.py --iface usb0   # explicit interface
sudo python3 modbus_server_ncm.py --no-dhcp      # DHCP already running
    elsewhere
```

101.1.8 Boot Sequence

```
task_net:
  1. ca_usb_dev_mng_init/add/start      bring-up USB
  2. ca_net_init(ca_ncm_iface_get())    NCM becomes the net core iface
  3. ca_net_proto_init(&net_cfg)        stack con IP = 0.0.0.0
  4. ca_net_proto_bind_udp(68,    )       porta 68 per OFFER/ACK
  5. loop: as soon as the link is up     ca_dhcp_service_net_ready()     the
    DORA starts
          then ca_sem_timedwait() + ca_net_process()

on_dhcp_bound (callback, from the DHCP task):
```

```
      s_gw stored (= Modbus server address)
      ca_sem_trywait/post(&s_dhcp_sem)    sblocca task_modbus

task_modbus:
  ca_sem_wait(&s_dhcp_sem)
  ca_modbus_service_init(server = s_gw,    )
  ca_modbus_service_net_ready()         apre la connessione TCP
  loop: FC03/FC06/FC16/FC01 cycle every 3 s    reconnect every 5 s if dropped
```

No `bind()` for Modbus: it is a client, it opens its own TCP connection to the server. The gate on the link is only for DHCP, which is the first to speak.

The gate of step 5 is specific to NCM. The NCM link only comes up when the host selects alt setting 1: as long as it is down, `ca_usb_ncm_send_frame()` **discards every frame and returns** `false`, so a DISCOVER sent too early is not delayed, it is **lost**.

> **Where to put the wait — and where not to.** The gate **must be inside the loop**, not before: blocking the net task waiting for the link *before* serving the frames prevents enumeration itself (the host reports `device not accepting address`). For the same reason the loop uses `ca_sem_timedwait(&ca_net_rx_sem, 100)` and not `ca_sem_wait()`.

101.1.9 Expected output

Board console:

```
=== EX78: Modbus TCP client su USB-NCM ===
MAC     02:CA:05:00:00:78
Modbus  porta 502  unit_id 1
waiting for the host to bring the interface up...
USB NCM UP
DHCP: avvio DORA...

DHCP BOUND  IP=192.168.10.100  GW=192.168.10.1
MODBUS server 192.168.10.1:502
MODBUS: avvio connessione TCP...
FC03 regs[0..3]= 101 201 301 401
FC06 setpoint=100 OK
FC16 config OK
FC01 coils=0x55
cycle done  ok=1 err=0
```

What to look for, beyond the fact that it works:

- **FC03 values must advance over time** (100→101→102…), but **not at every cycle**: the simulator increments the registers every 5 s while the board reads every 3 s, so two consecutive reads often fall in the same window and return the same value. The expected sequence is of the type `100, 101, 102, 102, 103, 104, 104` — a repeated value every two or three cycles is **aliasing between the two periods, not a cache**. An alarm signal would be a value that never changes;
- `err=0` and `ok` advancing at every cycle;
- on the PC side, the FC06 setpoint (100, 200, 300…) and FC16 registers must appear with the same values written by the board;
- the **transaction ids** in the server log must be consecutive, without gaps.

101.1.10 Technical Notes

Two MACs, one for each side

`s_mac` (`02:CA:05:00:00:78`) is the device MAC; `s_ncm.mac` (`02:CA:05:00:00:B8`) is the one announced on the host side — from which the name `enx02ca050000b8` of the interface on the PC comes, the one the script looks for. The two **must differ**.

Modbus functions are blocking

`ca_modbus_read_holding_regs()` and the related functions wait for the server's response, so `task_modbus` is a task by itself (LEVEL3) and not the net task: calling them from there would block `ca_net_process()`, i.e. exactly what must run to deliver the response. This is the same constraint as `ca_tcp_recv()` in ex75 and `ca_dns_resolve()` in ex72.

The chain stops at the first error

```
if (ca_modbus_read_holding_regs(...)) {     } else { ok = false; }
if (ok) { /* FC06 */ }
if (ok) { /* FC16 */ }
```

Every subsequent function code is conditioned on the success of the previous ones. It is not sloth: if the connection has dropped in the middle of a cycle, insisting with the other three requests would produce three additional errors that hide which one was the first to fail.

`last_reconnect` initialized to the current tick

```
u32 last_reconnect = ca_os_tick_ms();   /* avoids immediate double init */
```

Without this, `now - last_reconnect` would immediately be `MODBUS_RECONNECT_MS` and the loop would redo `ca_modbus_service_init()` an instant after the one just executed, resetting the connection in the process of being opened. Detail inherited from ex43.

101.1.11 File structure

```
examples/example_78/
          ca_os_ex78.c            entry point
          os_config_ex_78.h       configurazione (generata da ex43)
          modbus_server_ncm.py    server Modbus simulato + DHCP con auto-detect
   NCM
```

Chapter 102

iperf TCP over USB-NCM

102.1 Example 79 — iperf TCP over USB-NCM

102.1.1 Introduction

Variant USB-NCM of **example 44** (iperf over Ethernet). The services are the same — DHCP client, ARP, iperf service which acts as client or server on CLI command — but the transport is the USB port instead of the Ethernet MAC.

It is the example that measures the **bandwidth ceiling** of the NCM transport, and for this reason it is also the one where it is easiest to misread the numbers.

Feature	Notes
DHCP client	Full DORA + T1/T2 renewal
ARP service	resolves the peer's MAC
`iperf server`	the board receives, the PC pushes
`iperf client <ip>`	the board pushes, the PC receives
ICMP echo reply	replies to `ping`
USB Ethernet (NCM)	device `usb0` / `enxNN`

102.1.2 PC Prerequisite: Remove the interface from NetworkManager

When the board enumerates, a new network interface appears on the PC (`enx02ca0500xx` — the name that udev derives from the MAC). NetworkManager sees it as any other attached card and **assigns it an address of its own**: it has been observed `172.21.10.100/24`. The board is on `192.168.10.0/24`, so the two end up on different subnets and the board is unreachable.

The symptom is misleading, because the interface **exists and is UP**: `ip link` shows it, and even `ping -I <iface>` responds — that flag bypasses the routing table. What doesn't work is everything else (`socat`, `nc`, the PC scripts of the examples), which follow the routing table.

The rule is versioned in the repository:

```
sudo cp tools/99-caos-ncm.conf /etc/NetworkManager/conf.d/
sudo systemctl reload NetworkManager
nmcli -t -f DEVICE,STATE dev status | grep enx    # it must say: unmanaged
```

> **The glob goes on `interface-name`, not on `mac:`.** The form `mac:02:ca:05:00:00:*` seems equivalent but it is not: it requires a full MAC and fails **silently** with a wildcard character — no error, and the interface remains managed.

The address of the bank does not set this rule: it is set by `sudo ./tools/bench_net.sh usb` or the example PC script.

102.1.3 The numbers: what to expect and what not to believe

USB Full Speed are 12 Mbit/s raw signaling rate. From there, USB overhead, NCM encapsulation and Ethernet/IP/TCP headers must be subtracted. The comparison with the ~27–33 Mbps measured on ETH in ex44 **does not make sense**: here the bottleneck is the bus, not the network stack.

> **The valid measurement is always on the receiving side.** The total printed by the transmitting side counts the bytes accepted by its own kernel with `send()`, not those that arrived at the destination: what remains in the queue at shutdown gets **discarded**. On ex44 this produced a 12× discrepancy (PC: 73 MB sent, board: 5.8 MB received) that seemed like a performance collapse but was actually an artifact of the sender-side measurement.

How to distinguish a real problem

If the two numbers diverge significantly, the board counters tell you which direction to look:

```
eth stats          rx_drop=0  tx_fail=0  err_rbu=0
net                pool_free + driver_held == NET_BUF_COUNT
```

- `rx_drop=0` **and intact pool** → the board has not lost anything: the missing bytes never arrived on the cable, the bottleneck is upstream (sender or link);
- `rx_drop>0` **or declining pool** → actual dropouts due to exhausted buffers, and there's something to investigate in the firmware.

On ETH the measured reference on 08/21 is `pool_free + driver_held = 40+24 = 64` under load, with `rx_drop=0`.

Reference measurement

On NUCLEO-H753ZI, board as server:

```
iperf RESULT: 5689222 bytes  10481 ms  4551376 bps
```

4.55 Mbps, about 38% of the raw 12 Mbit/s of the bus. It is a plausible performance: from the raw signaling bandwidth, the USB transaction overhead (tokens and handshakes for each packet), the NTB/NCM encapsulation, the Ethernet+IP+TCP headers and the return ACKs must be subtracted. On the same hardware ex44 over Ethernet measures ~27 Mbps: **these two numbers are not comparable**, because here the bottleneck is the bus and not the stack.

The trap of duplicate routes

Both the ETH example and this one use **both** `192.168.10.0/24`. If the wired NIC keeps its address from a previous test, the kernel finds two routes for the same subnet:

```
192.168.10.0  0.0.0.0  255.255.255.0  U  0    0 0 enp0s31f6        <-- vince
192.168.10.0  0.0.0.0  255.255.255.0  U  100  0 0 enx02ca050000b9
```

It wins the lowest metric — Ethernet, where the board **is not present**. The symptom is a `timed out` from the client, and the distinction is diagnostic:

- `connection refused` → board reached, port closed (not listening);
- `timed out` → packets leave from the wrong interface, nothing comes back.

Fix: `sudo ip addr flush dev <nic-ethernet>`. The script detects the collision and reports it before starting the test.

102.1.4 Priority: `net` paired with the service

This example is one of three where `task_net` runs at **LEVEL6** instead of the usual LEVEL4 — together with ex44 (iperf over ETH) and ex45 (HTTP).

caOS levels **are not fixed priorities but CPU shares** (`caos/inc/scheduler.h`): LEVEL4 12.6 %, LEVEL6 50 %, doubling at each step. The rule that follows:

> **`net` and the service that consumes what `net` produces must be at the same level.** Pairing them splits the same bandwidth equally. If `net` were above, it would fill queues that the service does not empty in time; if below, the service would run idle.

The iperf service is declared `CA_TH_LEVEL6` in `iperf_service.c`, so `net` follows it. In the other examples, the leading service is at LEVEL0..4 and `net` stays at LEVEL4.

> On NCM, **there is no USB task** under which to place it: the USB stack runs in ISR and delivers frames via `ncm_iface_rx()` → queue + post to `ca_net_rx_sem`. Pairing is always with the application service, never with the transport.

102.1.5 Hardware

MCU	`USB_NUM`	Connector	LED heartbeat
U575 (NUCLEO-U575ZI-Q)	0	USB user	PC7 (LD1 green)
H753 (NUCLEO-H753ZI)	1	CN14 (OTG_FS)	PB14 (LD3 red)
F407 (STM32F4VE)	0	USB device	PA6 (D2)
H563 (NUCLEO-H563ZI)	0	USB user	PB0 (LD1 green)

Pinout verified on board manuals (UM2407, UM2861, UM3115).

102.1.6 os_config.h Configuration

```
#define EN_NET            1
#define EN_USB            1
#define EN_USB_NCM        1
#define EN_NET_DHCP       1
#define EN_DHCP_SERVICE   1
#define EN_TCP_SERVICE    1     /* iperf runs on top of TCP */
#define EN_IPERF_SERVICE  1
#define EN_ARP_SERVICE    1
```

Off with respect to ex44: `EN_ETH 0`, `EN_CLI_ETH 0`, `EN_USB_CDC 0`.

```
python3 tools/caos_config.py --new 79 --from ex44 \
        --set EN_ETH=0 --set EN_USB=1 --set EN_USB_NCM=1 --set EN_USB_CDC=0
python3 tools/caos_config.py --off EN_CLI_ETH 79
```

Compared to ex75 (TCP client), it differs by **only two flags**: `EN_IPERF_SERVICE` and `EN_CLI_IPERF` .

102.1.7 Usage

The service registers the CLI commands by itself; the firmware has no application task.

> **The order is mandatory: first the PC, then the board.** This is not a preference but a circular dependency to break: the board can accept the `iperf` commands **only after obtaining the IP**, and the IP is provided by the DHCP running on the PC. Issuing the command earlier produces `iperf: local IP non disponibile` — the service receives the address from `ca_iperf_update_local_ip()` inside `on_dhcp_bound`, which has not yet occurred.
>
> The scripts are designed for this order: they wait for the board instead of failing immediately.

Board as server (the PC pushes):

```
# 1. PC: configures the interface, starts dnsmasq, then waits for the board
sudo python3 examples/example_79/iperf_peer_ncm.py --mode client

# 2. board: as soon as "DHCP BOUND" appears, on its console
iperf server 5001 30
```

The PC script retries the connection for 60 s and prints `type 'iperf server' on the board console`: there is all the time to give the command after seeing the bound.

Board as client (the board pushes):

```
# 1. PC
sudo python3 examples/example_79/iperf_peer_ncm.py --mode server

# 2. board, after "DHCP BOUND"
iperf client 192.168.10.1 5001
```

> **Be careful with the `accept` timeout: the default is 3 seconds, not 30.** `IPERF_IDLE_TIMEOUT_MS` is `3000U` in the config (the same on ex44), while the CLI help writes `[idle_s=30]` — the help is imprecise. With `iperf server` without arguments the board waits for a connection for **3 s** and then prints `iperf ERR`: it's not a failure, the accept timed out.
>
> Two ways to avoid this: - give an explicit timeout — `iperf server 5001 30` — and then launch the PC; - or launch **first** the peer on the PC, which stays waiting, and only after give the command on the board.
>
> The same timeout also governs inactivity *during* the transfer (`ca_tcp_recv`), so increasing it does not falsify the measurement: it only gives time for the connection to establish.
>
> Both arguments are **required**: the parser reads the port first, then the timeout. `iperf server 30` would interpret `30` as *port*.

If the board already has a lease, the script may not find it. In state `BOUND` the DHCP client sleeps until T1 (one hour): it sends nothing, and no new `DHCPACK` will appear in the log — which `start_dnsmasq()` clears at every boot. The script notices this and after 5 s looks for the address in the ARP table (`ip neigh`, filtering by MAC `02:ca:05`). Two manual options remain:

- `--board-ip 192.168.10.131` to skip the detection entirely;
- `dhcp renew` on the console **while the script is waiting** (not before: the log is cleared at the start of the script).

The script finds the interface by MAC `02:ca:05`, assigns it `192.168.10.1/24`, starts `dnsmasq` and waits for the board's lease.

Cleanup. SIGINT/SIGTERM handlers are installed **before** starting `dnsmasq`, and at startup the script collects any leftover instances. Without this, a Ctrl+C during the lease wait (up to 60 s) would leave `dnsmasq` alive, keeping the IP occupied, and the next test would fail with `failed to create listening socket: Address already in use`.

102.1.8 Boot sequence

```
task_net:
  1. ca_usb_dev_mng_init/add/start      bring-up USB
  2. ca_net_init(ca_ncm_iface_get())    NCM becomes the net core iface
  3. ca_net_proto_init(&net_cfg)        stack con IP = 0.0.0.0
  4. ca_net_proto_bind_udp(68,    )       porta 68 per OFFER/ACK
  5. loop: as soon as the link is up     ca_dhcp_service_net_ready()     the
    DORA starts
          then ca_sem_timedwait() + ca_net_process()

on_dhcp_bound:
      ca_iperf_update_local_ip(ip)        the service learns the address
```

iperf does not need any link gate: it is commanded from the CLI, so it comes into play only after the link is up. It does not even have a `bind()` at startup — it only opens listen or connect when it receives the command.

The gate of step 5 is specific to NCM. The NCM link comes up only when the host selects alternate setting 1: as long as it is down, `ca_usb_ncm_send_frame()` **discards every frame and returns** `false`, so a DISCOVER sent too early is not delayed, it is **lost**.

Where to put the wait — and where not to. The gate **must be inside the loop**, not before: blocking the net task in wait for the link *before* serving frames prevents enumeration itself (the host reports `device not accepting address`). For the same reason, the loop uses `ca_sem_timedwait(&ca_net_rx_sem, 100)` and not `ca_sem_wait()`. The timeout costs nothing under load: as soon as frames flow, the semaphore is posted and the wait returns immediately.

102.1.9 Expected output

```
=== EX79: iperf TCP client/server su USB-NCM ===
MAC  02:CA:05:00:00:79
waiting for the host to bring the interface up...
USB NCM UP
```

```
DHCP: avvio DORA...

DHCP BOUND  IP=192.168.10.100
ready: use 'iperf server' or 'iperf client <ip>'

> iperf server
iperf server listening on port 5001 (idle=3 s)...
iperf SERVER  port=5001  idle=3 s
iperf RESULT: <N> bytes  <T> ms  <bps> bps
```

`iperf RESULT` is **the right measure**: it counts the bytes actually delivered to the application on the board.

102.1.10 Technical notes

Two MACs, one for each side

`s_mac` (`02:CA:05:00:00:79`) is the device MAC; `s_ncm.mac` (`02:CA:05:00:00:B9`) is the one announced to the host side — from which the name `enx02ca050000b9` of the interface on the PC comes, the one the script looks for. The two **must differ**.

The service carries its own CLI

`ca_iperf_service_start()` registers the commands `iperf client` / `iperf server`. For this reason the example has no application task: everything is driven by the console. This is also the reason why the local address must be updated separately, with `ca_iperf_update_local_ip()` inside `on_dhcp_bound` — the service is started before the lease exists.

Boot order

First the board, then the script: the USB interface is created during enumeration. On the firmware side there is no rush — the net task waits for the link as long as needed.

102.1.11 File structure

```
examples/example_79/
          ca_os_ex79.c            entry point
          os_config_ex_79.h       configurazione (generata da ex44)
          iperf_peer_ncm.py       peer PC (client o server) + DHCP con auto-
   detect NCM
```

Chapter 103

HTTP Server with CLI over USB-NCM

103.1 Example 80 — HTTP and HTTPS server with CLI via HTTP over USB-NCM

103.1.1 Introduction

USB-NCM variant of **example 45** (HTTP server over Ethernet). The services are the same — DHCP client, ARP, TCP, HTTP server with the four routes — but the transport is the USB port instead of the Ethernet MAC.

The application logic (HTML page, handlers, route table, CLI capture) is **copied identically** from ex45: this is the point of the `ca_net_iface_t` abstraction. Only the interface passed to `ca_net_init()` changes.

Feature	Notes
DHCP client	Full DORA + T1/T2 renewal
ARP service	cache + blocking resolution
HTTP server port 80	GET / POST / HEAD / OPTIONS
HTTPS server port 443	same routes over TLS 1.3 (`EN_HTTPS_SERVICE`)
`GET /`	HTML page with command form
`GET /status`	JSON: IP, uptime, NCM link status
`POST /cli`	executes a CLI command, responds with text
`GET /help`	list of available commands
ICMP echo reply	responds to `ping`
USB Ethernet (NCM)	device `usb0` / `enxNN`

103.1.2 PC Prerequisite: detach the interface from NetworkManager

When the board enumerates, a new network interface appears on the PC (`enx02ca0500xx` — the name that udev derives from the MAC). NetworkManager sees it as any other newly connected card and **assigns its own address to it**: `172.21.10.100/24` has been observed. The board is on `192.168.10.0/24`, so the two end up on different subnets and the board is unreachable.

The symptom is misleading, because the interface **exists and is UP**: `ip link` shows it, and even `ping -I <iface>` responds — that flag bypasses the routing table. Everything else (`socat`, `nc`, the PC scripts of the examples) does not work, as it follows the routing table.

The rule is versioned in the repository:

```
sudo cp tools/99-caos-ncm.conf /etc/NetworkManager/conf.d/
sudo systemctl reload NetworkManager
nmcli -t -f DEVICE,STATE dev status | grep enx    # deve dire: unmanaged
```

> **The glob goes on `interface-name`, not on `mac:`.** The form `mac:02:ca:05:00:00:*` seems equivalent but it is not: it expects a full MAC and with a wildcard it fails **silently** — no error, and the interface remains managed.

The bank address is not set by this rule: it is set by `sudo ./tools/bench_net.sh usb` or the PC script in the example.

103.1.3 The three differences compared to ex45

1. `eth_gpio_init()` → `ca_usb_dev_mng_init()` + `ca_usb_dev_mng_add()` + `ca_usb_dev_mng_start()`
2. `ca_net_init(ca_eth_iface_get(&eth_cfg))` → `ca_net_init(ca_ncm_iface_get(&s_ncm))`
3. `ca_net_init()` moves from `ca_os_configure()` to `task_net`: the NCM interface exists only **after** USB enumeration.

The `eth_mon`, `ca_eth_link_poll()` task and the GARP to the switch disappear (the host learns the MAC from the NCM descriptors). The "link" is `ca_usb_ncm_is_up()`.

The HTTP listener does not move

Unlike ex76 (where `ca_tcp_listen()` had to be moved from `ca_os_main()` to `task_net`), here **no movement is needed**: `ca_http_service_start()` is already called inside `on_dhcp_bound` even in the ETH version, i.e., after the lease. The listening PCB carries the local IP, so opening it before the address would not make sense on either transport.

The gate on the link

The HTTP server **does not need its own gate**: it just responds, and when a browser manages to reach it, the link is necessarily up. The gate is used by DHCP, which speaks first, and is **inside** the `task_net` loop — never before, otherwise it would block the enumeration itself (see ex70).

103.1.4 Test on PC

Power-on order: first the board, then the script. The USB interface does not exist until the device has enumerated.

```
sudo python3 examples/example_80/http_test_ncm.py
```

The script finds the interface from the MAC `02:ca:05:*`, starts dnsmasq, waits for the lease, verifies the four routes, and opens the browser.

Useful variants:

```
sudo python3 examples/example_80/http_test_ncm.py --cmd "top"     # un comando
    CLI
sudo python3 examples/example_80/http_test_ncm.py --status        # solo /status
sudo python3 examples/example_80/http_test_ncm.py --no-browser    # DHCP e
    attesa
```

Expected outcome

```
[1] GET http://192.168.10.180/status
    -> 200  {"ip":"192.168.10.180","uptime_s":42,"link":"up"}
[2] GET http://192.168.10.180/
    -> 200  1024 byte  <!DOCTYPE html><html><head>
[3] GET http://192.168.10.180/help
    -> 200  28 righe di help
[4] POST http://192.168.10.180/cli  cmd=free
    -> 200
 RISULTATO: PASS
```

103.1.5 Unlimited CLI output: chunked response

Unlike ex45, this example **does not use** `ca_cli_exec_capture()` with a fixed buffer. `/cli` and `/help` open a **chunked** response (`Transfer-Encoding: chunked`, RFC 7230 §4.1) and stream the CLI output into it as it is produced, using `ca_cli_exec_capture_stream()` — the same API that on 08/17 solved the SSH shell truncation.

Why SSH streaming was not enough

SSH can send its records as they come. HTTP cannot: it must write `Content-Length` in the header **before** the body, and the service patches it afterwards via `cl_pos` — which requires having the entire body in memory. Chunked encoding breaks this constraint: no `Content-Length`, each piece carries its own length, and a final chunk of zero length closes the body.

Support was added to `http_service.c` in an **additive** way: normal routes do not change, chunked is opt-in per response (`ca_http_resp_begin_chunked()`).

The test

`config` is the longest command (~3 KB) and ends with the SHA-256 hash of the configuration — therefore that line is the perfect probe: if it is there, it did not truncate.

```
sudo python3 examples/example_80/http_test_ncm.py --cmd "config"
```

```
  usb_ncm_ep_data      = 1
  usb_ncm_ep_notif     = 5
--- caOS Config Hash (SHA-256, excl. build_ver) ---
  aa52946f1baeb31872aed6fab71a5ee4ca387b23db6ac857677992669d4144cb
```

Before the modification, the output stopped at `TCP_TCP_ACCEPT_TIMEOUT_MS`, in the middle of the values table.

Requires `EN_CLI_CONFIG=1` (the `config` command) and `EN_SHA256=1` (the hash line); both enabled in this example, disabled in ex45.

103.1.6 HTTPS on port 443

As in ex45, with `EN_HTTPS_SERVICE` the same routes are also served in TLS 1.3. The example has its own identity (`examples/example_80/tls_identity.h`, generated with `tools/gen_tls_identity.py examples/example_80`) and starts it in `on_dhcp_bound`, right after the HTTP server. The operation is described in HTTPS and TLS 1.3.

```
curl --cacert examples/example_80/tls_identity_cert.pem \
     --resolve caos.local:443:192.168.10.180 https://caos.local/status
curl --cacert examples/example_80/tls_identity_cert.pem \
     --resolve caos.local:443:192.168.10.180 -X POST https://caos.local/cli -d "
    cmd=config"
```

Chunked responses also work over HTTPS: each chunk becomes one or more TLS records. The header of each chunk (a few bytes) goes into its own record, with its 22 bytes of overhead; for CLI output, this is a negligible cost.

RAM: a TLS session occupies approximately 24 KB. On F407 and F446 (128 KB), the example was already excluded before HTTPS, because it exceeded the limit by about 4.7 KB (`EX80_RAM_TOO_SMALL`, see the comment at the top of the source code); with TLS, the gap increases.

103.1.7 Status

PASS on H753 hardware (08/22). DHCP, HTTP server, the four routes, and the CLI via HTTP with full output. First example to use chunked responses.

HTTPS (10/23/2026): the `test_ex80` class (10 phases, including the HTTPS one with verified certificate and rejection of TLS 1.2) passes on H563, H753, F767, and U575.

ex81/82/83, written together with this one, remain to be tested.

Chapter 104

IP Defragmentation and UDP Server over USB-NCM

104.1 Example 81 — IP Defragmentation + UDP server on USB-NCM

104.1.1 Introduction

Variant of **example 48** (IP defragmentation over Ethernet) using USB-NCM. The services are the same — DHCP client, ARP, IP defragmentation service, UDP server on port 7961 — but the transport is USB instead of Ethernet MAC.

Feature	Notes
DHCP client	Full DORA + T1/T2 renewal
ARP service	cache + blocking resolution
IP defrag	reassembles fragments (`EN_IP_DEFRAG=1`)
UDP server 7961	payload up to `IP_DEFRAG_MAX_PAYLOAD` (1480 B)
Hardware CRC32	ensures reassembly is byte-exact
ICMP echo reply	replies to `ping`
USB Ethernet (NCM)	device `usb0` / `enxNN`

104.1.2 PC Prerequisite: Remove the interface from NetworkManager

When the board enumerates, a new network interface appears on the PC (`enx02ca0500xx` — the name udev derives from the MAC). NetworkManager sees it as any ordinary plugged-in card and **assigns it an address of its own**: it has been observed `172.21.10.100/24`. The board is on `192.168.10.0/24`, so the two end up on different subnets and the board becomes unreachable.

The symptom is misleading, because the interface **exists and is UP**: `ip link` shows it, and even `ping -I <iface>` responds — that flag bypasses the routing table. What doesn't work is everything else (`socat`, `nc`, the PC scripts of the examples), which follow the routing table.

The rule is versioned in the repository:

```
sudo cp tools/99-caos-ncm.conf /etc/NetworkManager/conf.d/
sudo systemctl reload NetworkManager
nmcli -t -f DEVICE,STATE dev status | grep enx    # it must say: unmanaged
```

> **The glob goes to `interface-name`, not to `mac:`.** The form `mac:02:ca:05:00:00:*` seems equivalent but it is not: it requires a full MAC and fails **silently** with a wildcard character — no error, and the interface remains managed.

The bank address does not set this rule: it is set by `sudo ./tools/bench_net.sh usb` or the example PC script.

104.1.3 Why the test is identical

NCM announces `wMaxSegmentSize` **1514** (`USB_NCM_MAX_FRAME` in `usb_ncm.h`), i.e. the same L2 MTU as Ethernet: 1500 bytes of IP payload. A UDP datagram of 1472 bytes (1480 bytes of IP payload with UDP header) therefore exceeds the MTU **exactly as on ETH** and is fragmented by the sender in the same way.

This is the reason why the reassembly path exercised is identical and the test script did not require adaptations for fragmentation — only for interface detection.

104.1.4 The CRC32 is not a detail

The server calculates the **hardware** CRC32 of the received payload and prints it. Unlike a byte sum, the CRC32 changes even if the bytes are the same but in **different order** — which is precisely what must be demonstrated when reassembling: correct length is not enough, because fragments reassembled out of order would give the same length and wrong content.

`ca_crc32_get()` runs in the UDP worker task, never in ISR, so the internal mutex is safe.

104.1.5 The three differences from ex48

1. `eth_gpio_init()` → USB-NCM bring-up (`ca_usb_dev_mng_*`)
2. `ca_eth_iface_get()` → `ca_ncm_iface_get()`
3. `ca_net_init()` from `ca_os_configure()` to `task_net`

`ca_udp_server_bind()` stays **before** the link gate: registering a port does not put anything on the wire. The UDP server does not need a gate, because it only responds; the gate is needed for DHCP, which speaks first.

104.1.6 Test on PC

Power-on order: first the board, then the script. Requires `scapy` and comes with a minimal DHCP server (no dnsmasq needed).

```
sudo python3 examples/example_81/test_frag_ncm.py
```

Without `--iface`, the interface is searched for by MAC `02:ca:05:*` and waited for up to 30 s. Useful parameters:

```
--size 1472     # UDP data bytes (default; max 1472)
--frags 4       # number of IP fragments
--repeat 3      # repeats the test
--skip-dhcp     # the board already has an IP
```

Expected Outcome

On the board's console:

```
UDP[7961] cid=0  rx=1472 B  crc32=0x........  from 192.168.10.1:54321
  pld[0..7]= 00 01 02 03 04 05 06 07
```

The printed CRC32 must match the one calculated by the script on the sent payload, and the echo must return intact.

104.1.7 Status

PASS on hardware H753 (22/08). 4 fragments (offsets 0/376/752/1128, `MF` correct, last one 352 B) reassembled into 1472 bytes, **intact echo in content** and not only in length — which also excludes out-of-order reassembly.

The same exact offsets measured on ex48 over Ethernet: practical confirmation that the MTU matches and that the reassembly path exercised is the same across both transports.

ex82/83 remain to be tested, written together with this.

Chapter 105

TFTP Client over USB-NCM

105.1 Example 82 — TFTP Client over USB-NCM

105.1.1 Introduction

USB-NCM variant of **example 49** (TFTP over Ethernet). The services are the same — DHCP client, ARP, CLI-driven TFTP client — but the transport is the USB port instead of the Ethernet MAC.

Feature	Notes
DHCP client	Full DORA + T1/T2 renewal
ARP service	cache + blocking resolution
`tftp get <ip> <file>`	download from server on PC
`tftp put <ip> <file> [size]`	upload a pattern generated in RAM
`tftp info`	status of the last transfer
ICMP echo reply	responds to `ping`
USB Ethernet (NCM)	device `usb0` / `enxNN`

105.1.2 PC Prerequisite: detach the interface from NetworkManager

When the board enumerates, a new network interface appears on the PC (`enx02ca0500xx` — the name udev derives from the MAC). NetworkManager sees it as any newly connected card and **assigns it its own address**: `172.21.10.100/24` has been observed. The board is on `192.168.10.0/24`, so the two end up on different subnets and the board is unreachable.

The symptom is misleading, because the interface **is there and is UP**: `ip link` shows it, and even `ping -I <iface>` responds — that flag bypasses the routing table. Everything else fails (`socat`, `nc`, the PC scripts of the examples), which follows the routing table.

The rule is versioned in the repository:

```
sudo cp tools/99-caos-ncm.conf /etc/NetworkManager/conf.d/
sudo systemctl reload NetworkManager
nmcli -t -f DEVICE,STATE dev status | grep enx    # it must say: unmanaged
```

> **The glob goes on `interface-name`, not on `mac:`.** The form `mac:02:ca:05:00:00:*` seems equivalent but it is not: it expects a full MAC and fails **silently** with a wildcard — no error, and the interface remains managed.

The bank address is not set by this rule: it is set by `sudo ./tools/bench_net.sh usb` or the PC script in the example.

105.1.3 Why TFTP does not need a gate

The TFTP client **speaks first**, so in theory it falls into the NCM link trap that caused ex70 to fail. In practice, it does not: it only speaks **when the user types the command**, and by that point the link has been up for a long time.

The general rule for clones applies: a gate is only needed for the first service that speaks on its own initiative — here, DHCP.

`ca_tftp_service_init()` remains in `ca_os_configure()`

Unlike `ca_net_init()`, this function **does not touch the network**: it only registers the CLI node (`tftp_service.c:670`). It can therefore remain where it was on ETH.

The local IP does not need to be propagated manually: the CLI manager reads `s_net_proto_local_ip` at the moment the command is executed (lines 578 and 635 of `tftp_service.c`), so the DHCP lease is collected automatically.

105.1.4 Test on PC

Power-on order: first the board, then the script. The script brings with it both the DHCP server (dnsmasq) and the TFTP server.

```
sudo python3 examples/example_82/test_tftp_ncm.py --put
```

`--put` is required to accept board uploads (`tftp put`). Without `--iface`, the interface is searched for by the MAC `02:ca:05:*`.

Then, from the board console:

```
tftp get 192.168.10.1 test.bin
tftp put 192.168.10.1 upload.bin 4096
tftp info
```

Expected outcome

A GET of an 8192-byte file produces **17 blocks**, not 16: 16 full 512-byte blocks plus a final **empty** block. This is not a defect — RFC 1350 requires that a transfer whose size is an exact multiple of the block size must terminate with an empty block, otherwise the receiver would not know that the file has ended.

105.1.5 Status

PASS on H753 hardware (08/22). Lease `.140`, then both directions:

- `tftp put` → 1024 bytes, intact pattern verified on the PC side (`test.bin_recv.bin`)
- `tftp get` → **17 blocks / 8192 bytes**, i.e., the correct RFC 1350 termination, with the cyclic `00..ff` pattern intact across block boundaries

The same numbers as ex49 on Ethernet.

Pitfall seen in the field: the server IP

The first attempt failed with `TFTP GET FAIL` because it pointed to `192.168.10.150` — the address of the **wired NIC** used in ETH tests. On NCM, the TFTP server is on the IP that the script assigns to the USB interface, i.e., **192.168.10.1** (`--pc-ip`). The board instead takes a `.1xx` from the DHCP pool.

If TFTP times out, `ping 192.168.10.1` from the board immediately distinguishes between the two cases: ping succeeds and TFTP fails = service issue; ping fails = wrong address or interface.

Chapter 106

SSH Server over USB-NCM

106.1 Example 83 — SSH Server over USB-NCM

106.1.1 Introduction

USB-NCM variant of **example 67** (SSH over Ethernet), and the last of the fifteen NCM clones. Minimal SSH server — `curve25519-sha256` + `ssh-ed25519` + `aes256-gcm` — with the transport moved from the RMII PHY to the USB port.

It is no coincidence that it was left for last: it is the most complex service in the series, and ex67 retains the NCM bring-up **commented in the source** precisely as a base for this example.

Feature	Notes
SSH server port 22	KEX X25519, Ed25519 host key, AES-256-GCM record
Authentication	`publickey` Ed25519 (`SSH_AUTHORIZED_ED25519_PUB`)
Interactive shell	RFC 4254, the caOS CLI via SSH
`ssh test`	repeats BPP framing vectors (offline)
ICMP echo reply	responds to `ping`
USB Ethernet (NCM)	device `usb0` / `enxNN`

106.1.2 PC Prerequisite: detach the interface from NetworkManager

When the board enumerates, a new network interface appears on the PC (`enx02ca0500xx` — the name that udev derives from the MAC). NetworkManager sees it as just another connected card and **assigns its own address to it**: `172.21.10.100/24` has been observed. The board is on `192.168.10.0/24`, so the two end up on different subnets and the board is unreachable.

The symptom is misleading, because the interface **exists and is UP**: `ip link` shows it, and even `ping -I <iface>` responds — that flag bypasses the routing table. What does not work is everything else (`socat`, `nc`, the PC scripts of the examples), which follows the routing table.

The rule is versioned in the repository:

```
sudo cp tools/99-caos-ncm.conf /etc/NetworkManager/conf.d/
sudo systemctl reload NetworkManager
nmcli -t -f DEVICE,STATE dev status | grep enx    # it must say: unmanaged
```

> **The glob goes on `interface-name`, not on `mac:`.** The form `mac:02:ca:05:00:00:*` seems equivalent but it is not: it expects a complete MAC and fails **silently** with a wildcard — no error, and the interface remains managed.

The bank address is not set by this rule: it is set by `sudo ./tools/bench_net.sh usb` or the PC script in the example.

106.1.3 Static IP: the simplest clone in the series, not the most complex

Unlike all other clones, ex67 **does not use DHCP**: it has a static IP (192.168.10.67, here 192.168.10.83). This eliminates the DHCP gate and with it the lease monitoring task.

However, the link gate is still needed, for a different reason. `ca_ssh_service_init()` creates the SSH task, which calls `ca_tcp_listen()` using `ca_net_proto_get_local_ip()` (`ssh_transport.c:1367` — the comment in the source warns that otherwise the SYN-ACK would start from 0.0.0.0). With the static address, that value is valid immediately, so the listen itself would be safe even early on; the point is that **the first SYN-ACK must be able to go out**, and until the host has selected alt setting 1, every frame is discarded by `ca_usb_ncm_send_frame()`.

The service is therefore started when the link is up, **inside** the loop of `task_net`. As always in the clones, never before: blocking there would stop the enumeration itself.

106.1.4 Differences compared to ex67

1. `eth_gpio_init()` → USB-NCM bring-up (`ca_usb_dev_mng_*`)
2. `ca_eth_iface_get()` → `ca_ncm_iface_get()`
3. `ca_net_init()` from `ca_os_configure()` to `task_net`
4. `ca_ssh_service_init()` moved **inside the loop**, behind the link gate
5. `task_eth_mon` → `task_ncm_mon`, without lease renewal request (static IP)

`ca_tcp_service_start()` remains before the gate: it does not put anything on the wire until someone connects.

106.1.5 PC Test

Power-on order: first the board, then the script.

```
./examples/example_83/run_ssh_client_ncm.sh
```

The script searches for the interface by MAC `02:ca:05:*`, assigns it 192.168.10.1/24 (it asks for `sudo` only if needed), and launches the client. IP and interface can be forced:

```
./examples/example_83/run_ssh_client_ncm.sh 192.168.10.83
IFACE=usb0 ./examples/example_83/run_ssh_client_ncm.sh
```

The test key pair

`examples/caos_ed25519/` contains a **versioned disposable key in the repo**, only for the demo: it has no security value. `SSH_AUTHORIZED_ED25519_PUB` in `os_config` is its public part.

Why host key options are needed

```
-o UserKnownHostsFile=/dev/null -o StrictHostKeyChecking=no
```

The board **regenerates its own host key at every boot** (it is volatile, held in a static struct and not in the key store). Without these options, `ssh` would refuse the connection on the second boot, signaling a changed host key. `IdentitiesOnly=yes` is instead used to prevent `ssh` from offering agent keys first.

106.1.6 Status

PASS on H753 hardware (08/22), on the first try. Ed25519 publickey authentication, RFC 4254 interactive shell, `help` with full output, `ssh test` **11/11 PASS** on BPP framing vectors.

The result closes an unknown that no other example covered: SSH was only tested over Ethernet (U575, H563, H753) and NCM was tested up to iperf (ex79) — **never the two together**. The heaviest load of the series (crypto + TCP + shell) runs on the USB transport without modifications to the SSH service: only the interface passed to `ca_net_init()` changes, which is exactly what the `ca_net_iface_t` abstraction promised.

With ex83, the **series of NCM clones is complete: 15 out of 15 validated on hardware.**

Chapter 107

L2 Echo: the Driver Measured Without the Transport

107.1 Example 84 — L2 Echo: the driver measured without the transport

107.1.1 Introduction

This is not a tutorial example: it is a **measurement tool**, born from a number that did not add up.

Example 44 measures iperf over Ethernet. On the H753, the two ends of the same transfer disagreed by **13.6 times**: the PC claimed to have sent 76,021,760 bytes at 60.7 Mbps, while the board's iperf counted 5,596,180 at 4.4 Mbps.

The frames had **not** been lost on the wire. The driver counters recorded 52,096 received, with zero errors — and 76,021,760 ÷ 52,096 = **1459.3**, which is exactly the MSS. Every frame had arrived and had been delivered: the bytes were disappearing **above** the driver.

The problem with measuring that phenomenon with iperf is that iperf is TCP. The driver, network task, TCP receive ring, and application all end up in a single number, and a slow result does not say **which** of the four is slow.

This example removes everything above L2.

107.1.2 How it works

A raw EtherType — `0x88B5`, which IEEE reserves specifically for experimental use — is picked up from the receive path via the **network filter**, its source and destination MACs are swapped, and it is sent back immediately.

No IP, no TCP, no receive ring, no application to schedule: just input DMA and output DMA.

Feature	Notes
L2 Echo on `0x88B5`	network filter, src/dst MAC swap, zero-copy
Static IP `192.168.10.84`	no DHCP
ARP	the board responds to requests
ICMP echo reply	responds to `ping`
CLI `echo stats`	rx/tx/drop counters
CLI `eth dump`	ETH driver counters
CLI `eth sync`	rebuilds DMA rings and restarts engines

What the result says

Outcome	Interpretation
the echo **keeps up** with the sender	driver and network task are healthy: the 13.6× is in the transport above them
the echo **does not keep up**	the limit is down here, and iperf only showed it by reflection

Both answers are useful — and that is what makes it a measurement and not a fix.

107.1.3 Why static IP

No DHCP, no TCP, no UDP services: **the fewer things running, the fewer things can explain the number**.

ARP and ICMP echo remain enabled for one reason only: a `ping` from the PC proves that the board is on the network **before** giving credit to any throughput figure. A board that does not respond to a ping has not produced a slow measurement — it has produced no measurement at all.

The address `192.168.10.84` is outside the bench's DHCP pool (`.50`–`.99`), so it is reachable from `192.168.10.1` and cannot collide with a lease.

107.1.4 The network filter (`EN_NET_FILTER`)

The example introduced in `caos/net/` a general mechanism, inspired by the Linux netfilter but stripped to the bone: **a single slot**.

```
ca_net_filter_set(0x88B5U, echo_filter);   /* arma  */
ca_net_filter_clear();                      /* toglie */
u32 n = ca_net_filter_count();              /* how many frames it took */
```

The filter is consulted at the top of `net_handle_frame`, **before** VLAN/EtherType parsing, before `b->meta` population, and before scanning the handler table. The callback receives `buf_idx` and **takes ownership of it**: it either frees it or passes it to transmission.

> **It is a shorter path, not a more urgent one.** The filter runs in the same network task, on the same wakeup, inside the same `ca_net_process()` as every other frame: it does not preempt anything. What it removes is the per-frame work between the driver and the consumer — and, above all, everything above L2.

107.1.5 The two queues, and why they are short

Between the filter (which runs in the network task) and the echo task, there is a **16-slot** queue.

The filter must **never** block: behind it is the entire reception drainage. Therefore, it delivers the buffer and returns.

The queue is intentionally small: if the echo task cannot keep up, frames must be discarded **there and counted**, not silently accumulated until the pool is exhausted. A counted discard is a metric; a hidden discard is a lie about the driver.

The two tasks are at the **same level** (`CA_TH_LEVEL6`): the frame passes from one to the other and both are on the reception path, so giving one of the two a higher quota would shift the queue, not shorten it.

107.1.6 Usage

On the board

```
echo stats      contatori: rx, tx, scarti, "balanced" o "GAP: rx != tx"
echo clear      azzera i contatori
eth dump        ETH driver counters
```

`rx` and `tx` must match. A discrepancy is the responsibility of **this example** — full queue or `ca_net_send` rejecting — and both cases are counted separately, so the driver is never blamed for a frame discarded by this code.

On the PC

```
# first: is the board on the network?
ping -c3 192.168.10.84

# then the measurement (needs root: raw sockets always require it)
sudo python3 examples/example_84/eth_echo_peer.py --iface <if>
sudo python3 examples/example_84/eth_echo_peer.py --iface <if> \
     --count 20000 --size 1500 --json /tmp/echo.json
```

The script sends N frames and counts how many return. A frame that arrives has been resent by the board: nothing else on the network responds to `0x88B5`.

> Loss here **is a data point, not an error**. The board is by design the slowest endpoint, so the interesting figure is not "did they all return" but *how many* return and at what rate when the sender pushes harder than the receiver can send back. The script only considers it a failure if the board has not sent **anything** back.

107.1.7 The offered rate (`--mbps`), and why it is needed

Without `--mbps` the script fires at wire speed. However, that number **measures the PC**: it offers ~98 Mbps because that is what Fast Ethernet does, and the percentage that returns is as much a property of the generator as it is of the board. It cannot be compared between different boards, and it changes if the PC changes.

A rate that the board **can handle** is instead a property of the board alone.

```
sudo python3 examples/example_84/eth_echo_peer.py --iface <if> --mbps 40
```

The rate is calculated starting from the initial instant, rather than sleeping for a fixed interval at each frame: a per-frame wait accumulates its own overhead and the actual rate slips below the required one — which would be read as a board rejecting in advance. The frame time includes preamble, IFG, and CRC (24 bytes beyond the payload), so the requested Mbit/s are those seen on the wire.

The JSON reports `offered_mbps` next to `send_mbps`: if the two diverge, it is the **PC** that failed to keep up with the rate, and that step says nothing about the board.

`first_echoed_seq` and `last_echoed_seq` indicate **where** the losses are. Missing frames at the *beginning* mean that the link was not yet carrying — the receiver had not finished recovering —

while losses scattered throughout the run indicate the receiver is overwhelmed, which is what this example measures. By eye, the two cases are indistinguishable; the sequence numbers separate them exactly.

107.1.8 Searching for the limit (phase `echo_rate_limit`)

The test bench automates the measurement with a **binary search**: it doubles from a rate known to be sustainable until a step is rejected, then halves between the last successful one and the first rejected one. It converges in a handful of runs and prints a number that means the same thing on every family.

> **Each step restarts from a known state**, and this is not a precaution: it is the measurement. A DMA ring stops at the slot where the previous burst left it, and with 4 slots, each run would start from a different phase. Without this measure, the ramp resulted in being **non-monotonic** (100, 100, 3.7, 100, 49, 79, 70, 100 %): the steps were not comparable to each other. The test bench reprograms the board and then calls `eth sync`, which brings the DMA exactly where `ca_eth_hw_init()` leaves it.

A warm-up burst seems like the obvious solution and **it is not**: it warms up the path but leaves the ring on an arbitrary slot, i.e., it produces the problem it is supposed to remove.

The limit found is **recorded, not promoted to a criterion**: the only step that can cause the phase to fail is the starting one, because below it there is nothing to measure. Turning today's performance into a threshold would mean causing a healthy board to fail tomorrow.

107.1.9 Reading `eth dump`

The driver counters are what distinguish a **slow** ring from a **stalled** one, and they have the same format on H5xx, H7xx, and F7xx, so a single reader interprets them all.

```
tx: calls=7176 full=0 spin=0 kick=4115
rx: calls=14353 frames=7176 err=0 ctxt=0 partial=0 runt=0
rx: rbu=1 susp=1
```

Counter	Description
`err ctxt partial runt`	the ring delivered something that **is not a frame**: a fault at any rate, must remain 0
`full`	a transmission found the ring still owned by the DMA — **not** a fault: it is what a receiver does when overwhelmed by the sender
`spin`	cycles burned in active waiting for the DMA to free the slot
`kick`	the ring had suspended and was explicitly restarted
`rbu`	receive descriptors have run out: frames are being lost

> On a v1 MAC (F7xx) `ctxt` equals `none`, not `0`: context descriptors in that silicon **do not exist**, and saying "zero occurrences" would claim a check that cannot happen. The bank decodes the absence instead of judging it.

The fault that these counters found

On the NUCLEO-H563ZI, after a burst offered at 80 Mbps, the board stopped echoing **and would not recover**: 0% even when returning to 20 Mbps, a rate it had just sustained at 100%. Only reprogramming recovered it.

The dump showed `TPS=7` (transmission suspended) with `cur` **beyond** `tail`, `spin=99999`, and `err/ctxt/partial/runt` all at zero: the ring was not broken, it was **stopped**. The engine suspends when the current descriptor reaches the tail pointer, and restarts only with a write that **moves** it; with 4 slots, upon a full lap the tail resumed a value already written and moved nothing.

The condition to monitor is `TBU` in `ETH_DMACSR`, which RM0481 explicitly names in the description of bit `ST`. **Not** the `TPS` field: the value 6 is also the normal idle state of an engine without work, and monitoring that triggered a restart on every transmission, with a 13% cost on throughput.

After the fix, two consecutive bursts at 78 Mbps without touching the board in between:

	burst 1	burst 2	driver
before	40%	**0%** (until reflash)	`full=1 spin=99999 kick=0`
after	43%	**100%**	`full=0 spin=0 kick=4115`

`kick` on 57% of transmissions says that the engine suspends continuously under load and that the restart is what keeps the ring moving.

107.1.10 Configuration

```
EN_ETH 1, EN_NET 1, EN_NET_FILTER 1, EN_ARP_SERVICE 1, EN_ETH_DBG 1
EN_TCP_SERVICE 0, EN_UDP_SERVER 0, EN_DHCP_SERVICE 0
EN_NET_NTP 0, EN_NET_DNS 0, EN_NET_MDNS 0, EN_NET_SYSLOG 0, EN_SDIO 0
```

107.1.11 Hardware notes

`RMII_TXD1` pin changes depending on the board: **PB15** on the NUCLEO-H563ZI (UM3115, JP6 ON / SB10 OFF), **PB13** on H753 and F767. Wiring PB13 fixed leaves TXD1 disconnected on the H563 — and the fault is insidious: the MAC continues to count every frame as transmitted and `tx_fail` remains 0, but half of the data bits never reach the PHY.

Chapter 108

Several Connections at Once: How the Buffer Pool Is Shared

108.1 Example 85 — Multiple connections together: how the buffer pool is shared

108.1.1 Introduction

Like example 84, this is not a didactic example but a **measurement tool**. And like that one, it stems from a question that example 44 cannot answer: example 44 measures **one** connection, and a single connection alone does not show what we are interested in here.

The network buffer pool is **only one** (`NET_BUF_COUNT`). The receive ring of each TCP connection does not own its own buffers: it holds them, taken from that pool. On paper, four connections at full depth require

```
4    (16 rx + 16 inflight) + 8 (driver) = 136 buffer
```

where there are **64**. It is an over-subscription of more than 2:1, and the question is what happens when the connections are really there.

108.1.2 Why it doesn't resolve by shrinking the rings

The obvious answer is to lower `TCP_RX_RING_DEPTH` and `TCP_MAX_INFLIGHT` until the worst case fits. It was measured on example 44, on H563, on 16/09/2026:

configuration	RX	TX
16 / 16	35.9 Mbps	31.8 Mbps
8 / 4	25.9 Mbps **(28 %)**	4.5 Mbps **(86 %)**

Both directions pay, and transmission **plummets**: with four segments in flight the send window is around 5.8 KB, and the sender spends each round waiting for an ACK. The loss rate across the five sessions was **0.01 %**, which is the signature of a structural limit and not of noise.

The point is that a depth chosen to serve four connections badly serves **even worse** the single connection that usually exists.

108.1.3 The quota, calculated when needed

Therefore, the depth remains at maximum and the division moves to **run-time**. `rx_quota()` in `caos/net/src/tcp_service.c` assigns to each connection a portion of what the pool can grant:

```
quota = min( (free + already queued     reserve) / active connections,
             TCP_RX_RING_DEPTH,
             slots already mine + (free     reserve) )
```

Three terms, each for a different reason:

- **Division** redistributes what is available among those who are asking for it;
- `TCP_RX_RING_DEPTH` is a physical limit: the ring is an array of fixed size;
- **the third term** is what makes the reserve truly a reserve. Without it, three already full connections plus a new one would be assigned a quota of 13 slots, which growing would consume the reserve and leave the pool at zero — and the next SYN would fail due to exhaustion, which is exactly the fault the reserve must prevent.

With **one** connection, the quota is the entire ring and nothing changes: example 44 confirms this, 35.1 Mbps against 35.9 of the static version, a difference within the dispersion of measurements.

Buffers already occupied **are never removed** from a connection. They contain data that the peer has sent and that the application has not yet read: discarding them would mean losing bytes already confirmed, i.e., repeating the defect of `rcv_nxt` already corrected. The ceiling applies when one **asks** for a slot, and the return happens by itself as the application reads.

108.1.4 What this example answers

If that quota holds when connections really compete:

outcome	meaning
streams are **equally distributed**	the quota redistributes: this is the main result
one stream stops at **zero**	a connection has starved the others — exactly what the quota must prevent
a connection is **rejected**	the pool ran out before the quota limited anyone: the reserve is too small
the sum **drops** below the value for one connection	dividing costs more than it yields

The measured result, and why it's surprising

On H563, four streams of 10 s (16/09/2026): **4 × 23.7 Mbps, total 95.0 Mbps**, with the four within 0.2 % of each other.

It is not the division of 36 Mbps into four parts: it is almost **three times** what a single connection achieves. And it shows that the measurement for one connection never regarded how much work the board could actually perform.

95.0 Mbps of payload become **100.1 Mbps on the wire** once headers, preamble, and interframe intervals are counted: the Fast Ethernet link is **saturated**, and the bottleneck has moved outside the board.

What limited the single connection was the **round-trip time** — the time spent waiting for an ACK before being able to receive more — and four connections fill in those gaps with

each other. In events per second, the stack went from ~6100 to ~**16 300**, so it wasn't even the per-packet cost that was the ceiling.

From here on, this measurement is **blind upward**: it cannot tell how much the board could do, because the cable runs out first. To discover this, one must offer less than 100 Mbps — fewer streams, or a set rate — and find where it stops scaling.

The fairness among the four, however, remains what this example really verifies: it is the quota that works.

108.1.5 How it works

The server uses the **callback** mode of `ca_tcp_listen()`: `on_accept`, `on_rx` and `on_closed` run in the context of the network task, so N connections do not cost N stacks. This is a substantial point and not a detail: the RAM pool size is being measured, and spending as much in task stacks would falsify the measurement.

This is also the reason why the iperf service from example 44 cannot be reused here: it performs **a single** blocking `ca_tcp_accept()` and then a blocking receive loop, and is therefore mono-connection by design.

Each accepted connection occupies an entry that counts bytes, segments and the instant of the first and last byte. The data is **not stored**: it's a sink, it counts and discards it.

108.1.6 Commands

On the board:

```
msink start [porta]    accetta connessioni e conta i byte (default 5001)
msink stats            totali per connessione e somma
msink stop             closes everything and prints the report
```

On the PC, with the example counterpart:

```
sudo python3 examples/example_85/msink_peer.py --parallel 4 --dur 10
```

You do not need to type anything on the board. Two seconds after receiving the address, the example listens on its own and announces it:

```
msink: listening on 5001 (started automatically)
```

The reason is that the order is mandatory: the DHCP server provides it to the PC script, so the peer is **always** already running when the board discovers its own address — and is already retrying connections. Asking for a typed command at that point would add a person to a sequence that doesn't need one, and the console should write it with the timing implications that this entails.

The two seconds are needed to let the gratuitous ARP go out, so the peer's first SYN doesn't arrive at an address that the network hasn't learned yet.

The commands remain available for another loop or for a different port: `sink_start()` rejects if a listener is already open, so they cannot conflict with automatic startup.

If the board already has an address — for example on the bench, where DHCP handles it on its own — everything is skipped with `--no-dhcp --board-ip <ip>`.

The network interface is passed with `--iface`; if the name doesn't exist on that machine the script lists them and, when there is only one wired one, chooses it by itself.

108.1.7 How to read the report

```
conn 0: h=17 45088768 bytes 30883 segs 10012 ms 36024 kbps
...
accepted=4 refused=0  live=0  slots_used=4/8
TOTAL 180355072 bytes 123532 segs 10031 ms 143829 kbps
```

The sum is measured over the time during which the connections **overlapped**, not the sum of their individual durations: they run together, so adding the durations would count the bytes roughly N times too many and report a quarter of the actual rate.

To be watched alongside the report, on the board's console:

- `tcp buf` — the pool distribution by owner, the historic minimum of free buffers and the rejected allocations. `low_water` near zero or `fail` different from zero indicates that the pool was indeed the bottleneck;
- `tcp rxdrop` — discarded segments due to full ring, and the transmission accounting (`retrans` must stay at zero: if it rises, contention is causing retransmissions).

`refused > 0` is not a pool defect: it means that more connections arrived than the slots in this example, which are `TCP_MAX_CONN`.

108.1.8 Requirements

`os_config.h`: `EN_ETH 1`, `EN_NET 1`, `EN_TCP_SERVICE 1`, `EN_ARP_SERVICE 1`, `EN_NET_DHCP 1`.

Hardware: NUCLEO-H753ZI, NUCLEO-H563ZI or NUCLEO-F767ZI (RMII, LAN8742A).

Chapter 109

callto: Calling an Address from the Console

109.1 Example 86 — `callto`: calling an address from the console

109.1.1 Introduction

The console can already **read** any address: `mem` reads a word, `hexdump` dumps a block. This example serves the command that **calls** one.

The value is not in the call itself — with the probe attached, `gdb` already does `call`. It is in the case where the probe **does not connect**, which on this bench has happened often enough to deserve a memory of its own. There, `mem` and `hexdump` are the only way to look inside a live board, and `callto` extends that from reading to executing.

> `EN_CLI_CALLTO` **is a debug flag.** It is arbitrary execution from the console: a build that carries it hands over everything the firmware holds, `keys_store` included. It is off by default and `config` reports it, so a firmware can be asked whether it has it. It is on here because it is what the example is about.

109.1.2 The call runs in a task of its own

This is not a convenience, it is the point.

From the CLI thread, a wrong address brings down **the console** — and the console is the very channel the answer is supposed to come back on. In a separate task:

choice	what it buys
`JOB_SAFE`	the stack canary watches a callee that overruns
`CA_FAULT_ASSERT`	a fault **removes that task** and nothing else
name `"callto"`	`top` and `taskall` show it, `kill <n>` stops it

So a callee that never returns is **visible and killable** instead of fatal. The command does not wait: it launches, gives the prompt back, and the result arrives on a line of its own when the call finishes.

109.1.3 Syntax

```
callto <addr hex> [param hex]    call the address, with an optional u32
callto                           what the last call did
```

Without a parameter the target is called as `u32(void)`, not with a zero: under AAPCS the two forms produce the same instructions, but the prototype is the one that was asked for and the one that gets reported — printing `param 0x00000000` would claim an argument that nobody passed.

The command sets the Thumb bit. On Cortex-M a `BLX` to an even address is a UsageFault INVSTATE, and the even form is what a listing shows: it is the mistake everyone makes once. The effective address is printed, so the correction is visible rather than silent.

The address is validated with `ca_valid_pointer()`, i.e. against the board's `CA_MEMMAP_TABLE`. Mind what that means: it answers **mapped**, not **executable**. An address in RAM passes. It rules out the typo that would fault on the first fetch, not an address that simply holds no code.

109.1.4 The three targets

At boot the example prints the addresses of its three functions, so they can be called without a map file and without a debugger:

```
example_86: callto targets
ex86: u32  0x08004A11  f(x) = x*2+1
ex86: void 0x08004A19  f()  = 0xCA05CA05
ex86: spin 0x08004A21  never returns
example_86: ready
```

target	what it proves
`u32`	the parameter arrives in `r0` and the result comes back. `x*2+1`, not `x*2`: with `+1` a callee that was never entered, returning `r0` at zero, cannot look correct
`void`	the call without an argument, and the magic `0xCA05CA05` that nobody prints by chance
`spin`	never returns (it sleeps, it does not burn CPU): it proves the task stays findable in `taskall` and stops with `kill`

109.1.5 Typical session

```
> callto 08000209 15
callto: launching 0x08000209 (param 0x00000015) - 'callto' for its thread
> callto: 0x08000209 returned 0x0000002B

> callto
callto: last 0x08000209 (param 0x00000015) -> 0x0000002B

> callto FFFF0000
callto: 0xFFFF0000 is not in this board's memory map

> callto 08000239
```

```
callto: launching 0x08000239 (no param) - 'callto' for its thread
> callto
callto: 0x08000239 still running as thread[4] - 'kill 4' stops it
> kill 4
thread[04] named 'callto' killed
```

Why the thread number is not in the launch line

Because it would **already be wrong** by the time you read it.

A `thid` is a **position** in the task list, and `TaskList_Remove` (`scheduler.c:323`) does *swap-with-last*: it moves the last task into the freed slot and **rewrites** its `->index`. Every CLI command then runs in a throw-away task of its own (`cli_cmd`, `cli.c:653`), created **before** the `callto` task and destroyed as soon as the command returns — which slides `callto` right into its slot.

Measured on U575 at the first run: the launch line said `thread[5]`, `top` showed the task at `4`, and `kill 5` hit an **EMPTY** slot and answered nothing.

The stable handle is the **task handle** (`ca_task_handle_t`: slot + generation), fixed for the task's whole life: `ca_os_thid_from_handle()` turns it back into the index that is valid *now*. That is what `callto` keeps, and the index is resolved at the moment it is printed. It used to keep the stack base, which is not an identity: the PSP area of a finished task is reused by the next task created, and an old base could recognise the newcomer as the `callto` still running. The handle's generation prevents that. Same class of defect as the TCP handle that named the slot and outlived the connection.

The result outlives the task that produced it, and that is why `callto` without arguments exists: the kernel **saves** what a task returned, in `task->result`, but the very next line marks the task `REMOVE` and the scheduler frees struct and stack. There is no API to read the result of a finished task, and the slot is recycled. The value therefore has to land somewhere that is not the task — an input **and** output struct, passed as `p1`.

That single struct also serialises the command: one call at a time, and a second `callto` while the first is still running is refused instead of overwriting a result that nobody has read yet.

109.1.6 Portability

No peripheral, no LED, no jumper: the command is MCU-independent and so is the example. It runs on every board.

109.1.7 Automated test

`test/hw_test/src/test_ex86.cpp`, 9 phases. The addresses are taken **from the board** — from the boot transcript — not from a map file: what gets called is what is on the part, not what the host believes it built.

The phases that matter are not "does it call?", they are the two that would fail silently: the **even** address passed on purpose (if the command did not set the Thumb bit, the phase would not give a wrong value — it would bring the task down), and the callee that **never returns**, where the test checks that the console still answers, that `callto` reports it as running and that `kill` ends it.

Chapter 110

Lost Wake-up Bench

110.1 Example 87 — Lost wakeup bench

110.1.1 Introduction

Blocking kernel waits all follow the same pattern: under lock, the task registers itself to the wait list, releases the lock, and **only afterwards** calls `ca_os_wait()`, which marks it `BLOCKED`. A wakeup occurring between the unlock and `ca_os_wait()` finds the task still `RUNNING`, and `ca_os_signal()` acts only on a `BLOCKED` task: the signal is lost. Then the task blocks outside of any list and no one wakes it up anymore.

The defect emerged on 09/23 on U575 in the stdout drain, where it silenced the entire console. This example searches for it in **all** other primitives.

The window is only a few cycles wide. Waiting for it is useless: you must **sweep** it.

110.1.2 How it works

element	role
TIM3, every 50 µs	produces **one** element from the ISR, and the next one only after it has been processed
`e87_rx`	consumer waiting **without timeout** on the phase primitive
`e87_rxt`	second consumer, **with** timeout, only in phase `mixed`
random pause	after each element, the consumer spins for 0..1 period

The random pause shifts the next tick along the entire consumer path, including the window. Critical sections mask with `PRIMASK`: a tick that expires while the consumer holds the lock starts **exactly at the unlock**, i.e., inside the window.

The ping-pong transforms a lost wakeup from a delay into a **block**: the ISR does not produce anymore until the confirmation arrives. The ISR itself monitors the consumer: if it remains `BLOCKED` for 100 ms with pending work, or outside of any list, it counts it, classifies it, and wakes it up, so the pass reaches the end.

The three forms of the defect

counter	observed state	cause
`lost_in_list`	`BLOCKED`, in list, pending work	the element arrived between the “empty?” check and the registration, performed in two different critical sections
`lost_off_list`	`BLOCKED`, **not** in list, pending work	the wakeup arrived between the unlock and `ca_os_wait()`
`orphan`	`BLOCKED`, in no list, nothing pending	the task was detached from the list by someone else
`late`	element not taken for 200 ms, none of the forms above	an unclassified block

`orphan` concerns the timeout branches of `queue.c`: at each attempt, the task registers itself again without having removed itself, and registering an already connected task truncates the list behind it. Phase `mixed` puts a consumer with timeout and one without on the same queue on purpose to make it emerge.

For `notify` the in/out of list division is not observable (notify has no registration): every block is counted as `lost_off_list`.

110.1.3 The phases

phase	primitive	producer (ISR)
`queue`	`ca_queue_static_receive( , CA_WAIT_FOREVER)`	`ca_queue_static_send_isr`
`qheap`	`ca_queue_heap_receive( , CA_WAIT_FOREVER)`	`ca_queue_heap_send_isr`
`qsend`	`ca_queue_static_send( , CA_WAIT_FOREVER)` on a queue of 1, full	`ca_queue_static_receive_isr` frees the slot
`notify`	`ca_notify_wait(true, CA_WAIT_FOREVER)`	`ca_notify_send_isr`
`event`	`ca_event_wait_any( , CA_WAIT_FOREVER)`	`ca_event_set_isr`
`stream`	`ca_stream_receive( , 0 xFFFFFFFF)`	`ca_stream_send_isr`
`message`	`ca_message_receive( , 0 xFFFFFFFF)`	`ca_message_send_isr`
`mixed`	`queue`, plus a second receiver with 3 ms timeout	`ca_queue_static_send_isr`
`mv_queue`	like `queue`, with the consumer **changing index** while waiting	check: wait lists hold pointers
`mv_event`	like `event`, same	`waiters[]` was indexed by position
`mv_stream`	like `stream`, same	`waiting_task` was a position
`mv_msg`	like `message`, same	`waiting_task` was a position

phase	primitive	producer (ISR)
`mv_mem`	a task that **owns heap blocks** changes index 8 times, then terminates without freeing them	ownership was registered by index
`fair`	two receivers **with timeout** on the same queue	one task, one element every 2 ms (no ISR)
`tmo`	`ca_queue_static_receive( , T)` on a queue that no one feeds, T = 1..20 ms	none
`kill`	a waiter (with and without timeout) **removed** while in the list, another one behind	one task, one element after removal

The phases of waits with timeout: `fair`, `tmo`, `kill`

Since 09/27, a waiter with timeout registers in the wait list like one without, parked in `SLEEP` until expiration (S2_02 §8.8). Previously, it performed polling every 10 ms outside the list. These three phases do not use the ISR: each has its own runner.

- `fair` — two consumers with 20 ms timeout on the same queue; each element carries the tick in which it was sent. It passes if the elements alternate (the least served one has at least 40%), none are lost, and each arrives within 3 ms. On the kernel with polling: `lat_max=30/8 ticks` (H563), FAIL.
- `tmo` — the timeout must expire **never before** T and at most 2 ms after.
- `kill` — the victim puts itself in the list, the survivor behind it; the victim is removed. Its context must **exit the list** before being reused (`stale=0`: then a specific task is created that reuses it) and the survivor must receive the next element (`lost=0`). On the previous kernel: `stale=25` over 25 rounds without timeout, and the victim with timeout never entered the list (`unlisted=25`).

```
e87 fair    : a=1250 b=1250 sent=2500 lat_max=0/15 ticks  PASS
e87 tmo     : n=478 early=0 late=0 got=0 over_max=0/10 ticks  PASS
e87 kill    : rounds=50/50 stale=0 lost=0 unlisted=0  PASS
```

The other primitives are verified in examples 5, 6, 7, 9, 14, 15, 17, and 20 (command `twait`, and `twkill` in ex20).

The `mv_*` phases: the consumer changing index

A `thid` is a **position** in the task list, and removing a task moves the last one into the freed spot, overwriting its index. A primitive that remembers who is waiting by index wakes up the wrong position. In the `mv_*` phases, the consumer is created last, above 8 "plug" tasks: each plug that terminates makes it move down one spot **while it is waiting** (production is suspended during the move, so the consumer is definitely waiting), and it remains the last one, so the next plug moves it again. `moves=8/8` confirms that the moves have occurred; less than 8 is `TOO_FEW_MOVES`.

On the kernel prior to stable task handles, `mv_event`, `mv_stream`, and `mv_msg` lost one wakeup for every move (8 out of 8), while `mv_queue` passed. Now the primitives remember who is waiting by slot (event) or by task handle (stream, message), which do not move.

The console thread is **not** a phase. It is the channel on which the test bench responds, and putting it into execution meant driving the dispatching of CLI commands (command slots, inline execution) instead of its waiting. Its waiting is `ca_queue_static_receive( , CA_WAIT_FOREVER)`, already covered by the `queue` phase.

Consumers are held by **context**, not by index: every console command creates and destroys a task, and this reshuffles the indices (see example 86). The index is read at the moment it is needed.

110.1.4 Commands

```
e87 start [fase|all] [sec]   one pass, closed by RUN RESULT
e87 lat [sec]                only the IRQ latency probe, no phase
e87 info                     timer, periodo, soglia, jitter
```

A second `e87 start` sent while a pass is in progress is **rejected**. The CLI executes the command in its own thread when `cli_cmd` is busy: without the guard, two passes of the same phase would run together (seen on `fair`: two producers on a full queue whose consumers had already finished, the console busy for two minutes — it looked like a kernel hang).

By default `start` executes all phases, 3 seconds each. One line per phase:

```
e87 queue   : iter=41230 lost_in_list=0 lost_off_list=0 orphan=0 late=0  PASS
...
RUN RESULT : OK  (16/16)
```

Each phase using the ISR also prints the **interrupt latency** measured by the probe on TIM3 (maximum, PC interrupted, histogram):

```
e87 queue   : irq_lat max=16.25 us pc=0x00000000  <1:625 <2:40232 <5:1708
    <10:15570 <20:1965 <50:0 <87:0 >=87:0
```

`pc=0` means that the maximum occurred inside a handler (PendSV), not in a task code section with interrupts disabled.

A phase passes if all counters are zero, the consumer exits cleanly at the end of the phase, and the number of cycles is at least 50 per second.

A PASS means "not hit in N thousand attempts", not "impossible". For this reason, the criterion also requires the number of cycles. The true test of the bench is the other direction: **on code with the defect, it must return FAIL**. A bench that also passes on broken code does not prove the fix.

110.1.5 Configuration

flag	value
`EN_TIMER`	1
`EN_EVENT`, `EN_NOTIFY`	1
`EN_STREAM`, `EN_MESSAGE`	1
`ENABLE_CLI`	1

No wiring: TIM3 is only used as an interrupt source.

20 task slots are required (`MAX_TASK`): on F103 (5), the example does not run: it prints `EXAMPLE IS DISABLED FOR UNSUPPORTED HARDWARE` and the reason.

110.1.6 Test

`test/hw_test/src/test_ex87.cpp`: one test phase per primitive, so a regression names the primitive. `CAOS_E87_SEC` changes the duration of each phase.

```
./hw_test.sh -n ex87
CAOS_E87_SEC=10 ./hw_test.sh -n ex87
```

Chapter 111

CLI Echo on the USB CDC Console

111.1 Example 90 — Echo of CLI on USB CDC console

111.1.1 Description

Example 90 demonstrates that, with the console on **USB CDC**, a character typed by the user never ends **inside** a line printed by another task.

The example starts idle (`ex90: idle`). `e90 start` launches a task `talker` that prints a fixed-format line every `EX90_PERIOD_MS` (20 ms):

```
EX90 seq=<n> tick=<ms> END
```

While lines scroll, type on the CDC console. The typed characters may appear **between** a line and the next one, but never inside: every line that contains `EX90 seq=` must end exactly with the format above. The check is done on the format, not by eye, and consecutive `seq` also indicates if a line was lost.

Hardware: boards with USB device used by ex52 (U575, H753, F407, F446, F767, H563). Test target: **NUCLEO-H753ZI**, user USB port (CN13).

111.1.2 Why this example

The defect: echo inside lines

Until 09/23 the CLI echo started **from the receive ISR**: every byte received was immediately re-sent to the console, while the stdout task could be in the middle of a line from another task. The typed character ended up in the middle of a line:

```
19788t37          <- a 't' typed inside a number
lA   113 ...      <- corrupted table row
```

It wasn't just a visual annoyance: lines like this ended up in an **archived capture** (the app_2 guide, log from 08/31). They remain plausible and are false, and a capture is meant to be re-read months later.

The fix

Starting from 09/23, the echo is no longer written from the ISR: it goes into a dedicated queue (`ca_stdout_echo()`, 128 slots of one byte, ISR-safe) and the **stdout task is the only writer to the console**. The task empties the echo only **between** one line and the next. Lost echoes due to full queue are counted (`echo dropped` in `cli_stat`).

The case **between** lines (for example `upti` + one line from the monitor + `me`) remains, and it's a choice: this is the standard behavior even on PC (bash with a background job that prints), and the command still arrives intact.

What was missing: the CDC console

The fix had been tested on the **UART** console: U575 by hand (09/23), H563 (09/30), and U575 (10/02) Release with emulated typing on ex68. On CDC, bytes don't come from the UART ISR but from the USB callback, which calls the same `ca_cli_rx_byte()`: the echo path is the same, but it had never been tested. No existing example allowed it: ex63 has the console on CDC but no task that continuously prints. Hence ex90.

111.1.3 os_config.h configuration

Created with `caos_config.py --new 90 --from ex63 --set EN_UART_DMA=0 --set EN_DMA=0`: the console from ex63 (CDC) without its UART DMA protocol.

```
#define EN_USB              1
#define EN_USB_CDC          1
#define EN_USB_FS           1
#define STDOUT_ON_USB_CDC   1
#define STDIN_ON_USB_CDC    1
#define ENABLE_CLI          1
```

`ENABLE_DBG_UART` is off as in ex63: nothing comes out on the ST-LINK port. On MCUs without support, the *unsupported* marker still comes out on the debug UART, written directly.

111.1.4 Commands

```
e90 info         lines printed, period, stdout and CDC counters
e90 period N     talker period in ms (5..1000)
e90 stop|start   stops / restarts the talker (to read diagnostic commands)
e90 line N       a line of exactly N characters (1..127): with N=62 and 126 the
                 USB write is 64 and 128 bytes, the empty packet case
cli_stat         CLI state: 'echo dropped' must stay 0
```

`e90 info` print:

```
lines  : 24431
period : 20 ms
stdout : queued 0  dropped 0  echo dropped 0
cdc tx : refused 0
```

- `dropped` — lost strings from stdout: either due to a full queue (with the `stdout: warning queue is full!` marker in the stream) or because the sender has rejected them after `STDOUT_SENDER_RETRIES` attempts (silently).
- `cdc tx refused` — number of times `ca_usb_cdc_write()` has rejected a string because the IN endpoint ring was full. A rejection **is not** a loss: the drain retries the same string one tick later.

111.1.5 How to test

Connect **both** cables: ST-LINK (flash) and USB user. The console is the CDC port (VID `ca01`, for example `/dev/ttyACM1`), not the ST-LINK's VCP.

Human-speed simulated typing with `caos_shell`:

```
CAOS_TTY=/dev/ttyACM1 CAOS_CHAR_MS=120 \
    test/hw_test/build/caos_shell -t 1 -i < commands.txt > run.log
```

Criteria, from the log:

1. every line containing `EX90 seq=` ends with `tick=<n> END`;
2. consecutive `seq` (except for the port opening, see below);
3. one response per command;
4. `echo dropped = 0`.

111.1.6 Results (10/04, NUCLEO-H753ZI, Debug)

Test	Lines from talker	Lines with one character inside	Responses	echo dropped
95 commands, 120 ms/car, talker 20 and 5 ms	18,120	**0**	60/60 `uptime`, 32/32 `e90 info`	0
65 commands, 120 ms/car, talker 5 ms	24,579	**0**	62/62	0
203 commands in burst, talker 5 ms	40,810	**0**	202/202 (*)	0

(*) In 2 responses out of 202, the line `lines : N` is missing from the log but not from the board (`dropped` and `refused` are quiet). `caos_shell` removes the first line from the response if it starts with the typed command; here the echo of the CR arrived **after** the first line of the response (`e90 info + lines : N + \r\n`), so the strip removed also `lines : N`, and instead a blank line remains. The reversed order is a small race in the drain (echo controlled, then ISR + command, then the line): the echo remains **between** the lines.

The echo on CDC behaves like on UART: typed characters appear only at the beginning of lines, never in the middle.

Lost lines: they were from the bench, not the board

At 5 ms (200 lines per second) the first two rounds lost, each time and always on the same command (`e90 period 20`), about 100 bytes: three talker lines and the echo of `2` and `0`. The command arrived intact.

`e90 info` said where: `dropped 5`, `refused 259`, empty queue and no marker. Five strings discarded after 50 attempts, that is, the IN endpoint was stuck: **the PC was not reading**.

The cause is in `serialIface::sendLine()` (`test/common/inc/serialIface.h`): with `CAOS_CHAR_MS` it does `delay()` between one character and the next **without reading the port**. `e90 period 20` is the longest command, 14 characters at 120 ms, so about 1.7 s of PC silence while the board produces ~6 KB/s: the tty fills up, `cdc-acm` stops reading the endpoint and the board, after the attempts, discards. This is the intended behavior for a console that nobody empties: the drain must not hang.

Test: same load with `CAOS_CHAR_MS=0` (command sent in one go, the PC always reads), 200 commands at 5 ms: `dropped` **unchanged**, no lines lost.

Two other bench effects to know:

- **At port opening** `seq` is skipped: the DTR causes the banner to be reprinted and the stdout queue to be cleared (`ca_stdout_flush()`), and the PC can return old lines remaining in its buffer.
- **At port closed the board continues printing.** The bench removes `HUPCL` (on VCP the drop of DTR resets the board), so the DTR stays high even after closing, `ca_usb_cdc_connected()` stays true and the lines are discarded after the attempts: `dropped` grows from one round to the next.

Chapter 112

Kernel Semantics Bench

112.1 Example 91 — Kernel Semantic Testbench

112.1.1 Description

Example 91 tests the **semantics** of kernel primitives: not whether they work under normal conditions, but whether they do the right thing in edge cases. The cases come from reading Zephyr's catalog `tests/kernel` (1009 cases): for every situation that Zephyr thought to test and that caOS's testbench wasn't covering, ex91 has a case.

The example starts idle and prints nothing until it receives a command. Each case is deterministic and judges itself: it prints a line `[e91] <caso> PASS|FAIL <valori misurati>` and the passed case closes with `RUN RESULT : OK|FAIL (n/m)`.

No wiring required. Runs on all boards with at least 16 tasks and the FPU (the `fpu_share` case exists only where there is one).

112.1.2 Why This Example

A green test says that the tested cases pass, not that the function is correct: a conceptual hole remains invisible until someone names the case. Examples ex05-ex20 prove that mutexes, semaphores, condition variables, and timers work in normal flow; nobody asked what happens if the waiter that raised the priority times out, if a timer stops from its own callback, or if an FP task declared actually starts.

Two of these questions found **real bugs** on the first run (05/10):

- **Periodic timer stopped by its own callback**: kept running. The daemon, after the callback, re-armed the periodic (and stopped the one-shot) without checking if the callback had already decided: 12 expirations instead of 3. Now the daemon only decides if the state is still `EXPIRED`.
- **Task** `JOB_FLOAT_POINT`: every FP task declared started with PC=0 and dirtied the neighbor's stack. Two defects summed in `ca_thread_create`: the initial hardware frame was base (8 word) while EXC_RETURN said «extended» (26 word), and `ca_memset_u32` received 64 as count — which is in **word** — zeroing 256 bytes instead of 64, including the PC just written. Latent because no example declared an FP task.

112.1.3 The Mechanism: Actors with a Script

A case is played by up to eight **actors**. Each executes a small script of operations (lock, timed lock, unlock, wait for a gate, turn on a marker, record its own priority, receive from a queue...).

The command task acts as the conductor: it creates the actors, waits for markers, opens gates, and reads priorities. The script makes the order of events explicit and keeps a case in a few lines.

An actor that hasn't exited by the end of the case is blocked in an unexpected wait: it is removed and the case fails with `STUCK`.

Priority inheritance is judged on the **actual `priority` field**, not on who gets the CPU: caOS's scheduler is weight-based (CFS) and a low-priority task continues to run next to a medium one, so the classic «M starving L» doesn't appear.

112.1.4 Configuration os_config.h

Created with `caos_config.py --new 91 --from ex18`, then `--on EN_RWLOCK` and `--off EN_LED --off EN_CLI_LED`. Flags that count:

Flag	Value	Why
`ENABLE_CLI`	1	test cases are launched from CLI
`EN_SW_TIMER`	1	cases `tmr_*`
`EN_RWLOCK`	1	case `rw_excl`
`MAX_TASK`	>= 16	8 actors plus system tasks (F103: example not supported)

112.1.5 Commands

Command	Effect
`e91 list`	lists test cases with one line of description
`e91 start`	runs all test cases, then `RUN RESULT`
`e91 start <caso>`	runs a single test case

Case	What it checks
`pi_basic`	the owner of a mutex boosts to the priority of the waiter, and returns to its own after unlock
`pi_two_mtx`	with two held mutexes, releasing the one that H does not wait for does not remove the boost
`pi_timeout`	the waiter that times out regains the boost: the owner returns low without unlock
`pi_chain`	the boost follows the chain: L holds M1, M holds M0 and waits for M1, H waits for M0 → L boosts to H
`pi_setprio`	`set_priority` on a boosted owner: remains H during the boost, new value after
`pi_kill`	the waiter that raised the owner is removed while waiting: the owner returns to its own priority

Case	What it checks
`pi_hopcap`	chain of 6 owners: the boost stops after `CA_PI_MAX_CHAIN` steps, nothing blocks
`rmutex_cnt`	a recursive mutex is only free to another task when the count reaches zero
`wake_pend`	`ca_os_wake_up_handle` on a task waiting for a semaphore does not grant a count
`reset_wait`	resetting a queue does not let waiting receivers exit with a phantom data
`send_front`	`send_front` bypasses queued elements
`scr_tmo`	8 waiters with timeouts mixed exit in order of expiry, timely within 3 ms
`cv_signal`	a `signal` wakes exactly one waiter
`cv_bcast`	a `broadcast` wakes them all
`cv_relock`	a `timedwait` in timeout returns with the mutex owned
`rw_excl`	no reader sees a partially written block, the writer is not starved
`coop_spin`	the cooperative spinlock does not lose increments (with a yield inside the section)
`tmr_drift`	a 10 ms periodic for 2 s does not drift
`tmr_restart`	`reset` re-arms starting from reset
`tmr_stopcb`	a periodic stopped by its own callback stays stopped
`tmr_rearm`	a one-shot re-armed by its own callback restarts
`tmr_same`	8 armed timers all expire, within 2 ms of each other
`tmr_period`	`set_period` takes effect from the next cycle
`fpu_share`	3 FP preempted tasks for 2 s always give the same bits

caOS has no *scheduler lock*: the corresponding Zephyr case is now `coop_spin`, the cooperative spinlock mutual exclusion.

112.1.6 How to test

```
./do_examples 91 && ./prog.sh -s -e 91
./hw_test.sh -n ex91            # one phase per case
```

or manually from the console: `e91 start`.

112.1.7 Results (05/10, NUCLEO-H753ZI, Debug)

24 cases out of 24. On the first run, the four cases `pi_two_mtx`, `pi_timeout`, `pi_chain` and `pi_setprio` failed: these were the limits of the priority inheritance of that time (one `prio_bkup` per task, boost only on the direct owner, removed only on unlock), as foreseen by the plan before writing the case. Correcting them led to the current PI.

```
[e91] pi_two_mtx  PASS  L boosted=32, after unlock M0 (H still on M1)=32 (want 32), after unlock M1=2 (want 2)
[e91] pi_timeout  PASS  L boosted=32, after H timed out (L still owner)=2 (want 2)
[e91] pi_chain    PASS  M (owner of M0)=32 (want 32), L (owner of M1, M waits)=32 (want 32)
[e91] pi_setprio  PASS  set to 4 while boosted: during=32 (want 32), after unlock=4 (want 4)
[e91] pi_kill     PASS  L boosted=32, after H removed while waiting=2 (want 2)
[e91] pi_hopcap   PASS  chain of 6 owners, cap 4 hops: boosted 5 (want 5) - A0..A5 = 2 32 32 32 32 32
[e91] scr_tmo     PASS  8 timeouts 10..80 ms: early=0 late=0 (worst +0 ms, slack 3) out_of_order=0
[e91] tmr_drift   PASS  10 ms for 2 s: hits=200 (want 200 +-1), span drift=0 ms (max 2)
[e91] fpu_share   PASS  3 FP tasks + 1 integer task for 2000 ms: rounds=4552 wrong=0 (want 0)
RUN RESULT : OK  (24/24)
```

Priority inheritance (Zephyr schema)

- `priority` is the **actual** priority (the weight for the scheduler); `prio_base` is the one assigned at creation or by `ca_os_set_priority_handle`.
- Actual = maximum between `prio_base` and the priority of every task waiting on a mutex owned (`pi_held`, list in the context, `pi_next` in the mutex).
- It is recalculated at every change of the set: a waiter arrives or leaves (lock, timeout, removal), a mutex changes hand (unlock), `prio_base` changes.
- A change proceeds on the owner of the mutex on which the task itself is waiting (`pi_wait`), at most `CA_PI_MAX_CHAIN` steps (default 4, in `sync.h`, overridable): each step switches to masked IRQ, therefore the chain is limited by construction.
- Cost: +8 B per context (180 B), +4 B per mutex. `MAX_TASK` recalculated: 22 on 4 KB TLB (F407, F429, F446, F746), 90 on 16 KB TLB (H753, H747, U575, H563); F767 stays 25 (arena PSP constraint). `ca_scheduler_init()` now stops with a message if `MAX_TASK * sizeof(ca_thread_context_t)` exceeds `_tlb_area_size`.
- The recursive mutex (`ca_rmutex_*`) does not have priority inheritance.

Chapter 113

External SPI NOR Flash on the QSPI

113.1 Example 92 — External SPI NOR Flash on QSPI/OCTOSPI

Drives a W25Qxx-class serial NOR flash via the QSPI driver (QUADSPI on F446/F7/H7, OCTOSPI1 on H5/U5), in **single** and **dual** mode and from the **memory-mapped** window at `0x90000000`. The driver operates via **interrupt**: data on FIFO interrupt, erasure and programming waits on the controller's auto-polling, the task sleeps.

113.1.1 Configuration (os_config.h)

```
#define EN_QSPI      1
#define EN_CLI_QSPI  1     /* the driver's "qspi" command, optional */
```

The example only runs where `MCU_HAS_QSPI` (board objmap) is 1: on F103, F407, and F429 it is declared unsupported.

113.1.2 Hardware — 6-pin module

The module (VCC CS DO GND CLK DI) goes on the Zio QSPI positions, **the same on all Nucleo-144** boards; only the MCU pins behind them change, which come from the driver table (`pinmap` 0):

module	position	F446/F746/F767	H753/H563	U575
1 VCC	CN8-7 (3V3)	–	–	–
2 CS	CN10-13	PB6	PG6	PA2
DI	CN10-23	PD11	PD11	PE12
4 GND	CN10-17	–	–	–
5 CLK	CN10-15	PB2	PB2	PB10
DO	CN10-21	PD12	PD12	PB0

DO/DI are the flash OUTPUT/INPUT data, easy to read as D0/D1: DI goes to IO0, DO to IO1, and `e92 wire` also tests the swap. CN10 does not carry 3V3 (pin 1 is AVDD): power comes from CN8-7. With only DI/DO, the module works in single (1-1-1) or dual output (1-1-2, command `0x3B`), never in quad. WP and HOLD must be pulled to VCC on the module: a floating HOLD freezes the flash.

113.1.3 Commands

At boot, the board is stopped; each test is **one pass** that concludes with a verdict readable by the test bench.

command	what it does	verdict
`e92 wire`	prints the connections of the board under test and reads the JEDEC ID via **simple GPIO** (no QSPI): a FAIL blames the wires, not the driver	`WIRE RESULT`
`e92 start`	everything: id, init, erase, erase verification, write, single and dual readback, and the same data read from the memory-mapped window (0x90000000)	`RUN RESULT`
`e92 write`	erase + pattern write + single readback	`RUN RESULT`
`e92 read`	reads back the pattern (single, dual, memory-mapped) **without writing**: after a power cycle, it tests retention	`RUN RESULT`
`e92 bench`	read speed on 1 MB at the current clock: indirect (interrupt-driven, 512 B calls) versus memory-mapped (memcpy from the window)	`RUN RESULT`
`e92 sweep`	increases the clock (from 7.8 to 125 MHz on H563): for each divider, single write+readback and dual read, with shifting off and on, one line per frequency and the maximum it can handle	`RUN RESULT`
`e92 clk <MHz>`	clock for subsequent tests (default 8 MHz)	–
`e92 shift auto|on|off`	half-cycle sampling shift (auto: on above 40 MHz)	–
`e92 dump [addr]`	64 bytes in single	–
`e92 info`	last ID, prescaler, counters	–

Each step prints `STEP <nome> : OK|FAIL`. The test area is the 4 KB sector at `0x001000`; the pattern (512 B, fixed seed) starts at offset `0x80` so it crosses two page boundaries and also verifies the driver's page splitting. The clock is kept below 8 MHz for the flying wires.

`wire` leaves the pins as GPIO: no need to reboot, the next test reopens the driver and returns them to the alternate function.

Part III

caOS Reference

Chapter 114

Reference API caOS

Funzioni in ordine alfabetico. Per ogni funzione: firma, descrizione, parametri, valore di ritorno, note, esempio.

ca_adc_hw_init_periph

Header: caos/drivers/inc/ca_glue_hw.h:10
GUARD: EN_CA_GLUE_HW

```
1 extern bool ca_adc_hw_init_periph(u8 adc_num, u8 sample_time); /*
     configura il periferico ADC in modalit  polling */
```

Initializes and calibrates the specified ADC hardware peripheral for polling-based operations.

Parametri:

Nome	Tipo	Descrizione
adc_num	u8	The index of the ADC peripheral to initialize (typically 1, 2, or 3 depending on the STM32 model).
sample_time	u8	The sampling time configuration value to be applied to all channels; higher values increase accuracy but decrease conversion speed.

Ritorna:
Returns true if the ADC was successfully initialized and calibrated, false if the hardware failed to respond (e.g., timeout during calibration or power-up).

Note:

- This function is blocking because it performs hardware polling loops (waiting for voltage regulators or calibration cycles to complete). It is NOT thread-safe and must not be called from an Interrupt Service Routine (ISR) due to the potential for long-running loops. Before calling this, ensure the peripheral clock has been enabled via the appropriate clock control function. A common mistake is calling this function multiple times without resetting the peripheral, which may lead to undefined hardware states or calibration errors.

Uso:

- This function is intended to be used during the system startup phase or when a specific ADC peripheral is first requested by the application. It is typically called inside a higher-level driver initialization function (like ca_adc_init) that handles clock gating and configuration validation. It should be paired with subsequent functions that trigger conversions and read data registers, ensuring the hardware is in a ready state before any sampling begins.

Esempio:

```
/* Initialize ADC1 with a sample time of 7 (e.g., 47.5 cycles) */
if (ca_adc_hw_init_periph(1, 7)) {
    /* Hardware is ready, proceed to trigger conversions */
} else {
    /* Handle initialization failure: log error or enter safe state */
    handle_error();
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_adc_hw_init_periph() in my embedded application?
A: You should call ca_adc_hw_init_periph() once during system initialization, before attempting any ADC conversions. It configures the ADC peripheral for polling mode, sets the sampling time for all channels, and calibrates the ADC. This function is typically called after enabling the ADC clock with ca_clock_hw_adc_init().

Code example conf:0.95 **Q:** Can you show a minimal example of how to use ca_adc_hw_init_periph()?
A:

```
u8 adc_num = 1;
u8 sample_time = 5;
bool success = ca_adc_hw_init_periph(adc_num, sample_time);
/* Use ADC for conversions */
```

Parameters conf:0.95 **Q:** What do the parameters adc_num and sample_time represent in ca_adc_hw_init_periph()?
A: adc_num specifies which ADC peripheral to initialize (1, 2, or 3). sample_time sets the sampling time for all ADC channels, ranging from 0 to 7. Invalid adc_num values (0 or >3) or sample_time values outside 0-7 may cause undefined behavior or incorrect operation.

Return conf:0.95 **Q:** Does ca_adc_hw_init_periph() ever return false?
A: No, ca_adc_hw_init_periph() always returns true. The function is designed not to fail during initialization, although incorrect parameters may lead to malfunctioning ADC behavior.

Error/Edge case conf:0.90 **Q:** What are the most common mistakes when using ca_adc_hw_init_periph()?
A: Common mistakes include calling the function without first enabling the ADC clock using ca_clock_hw_adc_init(), using invalid adc_num values (0 or >3), or using sample_time values outside the valid range (0-7). These can lead to incorrect ADC configuration or initialization failure.

Behavior conf:0.95 **Q:** Is ca_adc_hw_init_periph() blocking or non-blocking, and can it be called from an ISR?
A: It is a blocking function because it waits during the calibration phase using a while loop. It cannot be called from an ISR because it contains blocking code that would prevent proper interrupt handling.

HW differences conf:0.90 **Q:** How does ca_adc_hw_init_periph() differ between STM32F1xx and STM32H7xx families?
A: On F1xx, the function uses ADC_CR2_ADON, ADC_CR2_EXTTRIG, and ADC_CR2_EXTSEL for triggering, and performs calibration by setting ADC_CR2_CAL and waiting for it to clear. On H7xx, it enables the voltage regulator via ADC_CR_ADVREGEN, performs calibration with ADC_CR_ADCAL, and waits for ADC_ISR_ADRDY to be set before enabling the ADC with ADC_CR_ADEN.

`ca_adc_hw_init_periph_isr`

Header: `caos/drivers/inc/ca_glue_hw.h:13`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_adc_hw_init_periph_isr(u8 adc_num, u8 sample_time,
    ca_adc_isr_cb_t cb); /* configura ADC in modalit  interrupt con
    callback */
```

Initializes the ADC hardware peripheral and configures it to trigger an interrupt upon conversion completion.

Parametri:

Nome	Tipo	Descrizione
`adc_num`	`u8`	The index of the ADC peripheral (typically 1, 2, or 3 depending on the specific STM32 model). Invalid values return false.
`sample_time`	`u8`	The sampling duration configuration for the ADC channels. Must be a valid value supported by the hardware registers.
`cb`	`ca_adc_isr_cb_t`	A function pointer to the callback function that will be executed inside the Interrupt Service Routine (ISR) when the conversion finishes.

Ritorna:
Returns true if the ADC was successfully initialized and the interrupt was enabled. Returns false if the adc_num is out of range, the underlying hardware initialization fails, or the provided callback is invalid.

Note:

- This function is non-blocking and performs hardware-level configuration, including enabling the End-Of-Conversion (EOC) interrupt and setting the NVIC priority. It is not thread-safe and should be called during the system initialization phase before starting any conversions. It must NOT be called from within an ISR itself, as it modifies NVIC settings and peripheral registers that could lead to race conditions or system instability. A common mistake is passing a NULL pointer as the callback, which will cause the system to crash or jump to an invalid memory address when the interrupt fires.

Uso:

- This function is used when you need to perform non-blocking ADC conversions, allowing the CPU to execute other tasks while waiting for the hardware to finish sampling. It is typically paired with ca_clock_hw_adc_init to ensure the peripheral clock is running, followed by ca_adc_hw_start_isr to trigger the actual conversion process. Once the conversion completes, the hardware automatically jumps to the provided callback function, where you can read the result using a function like ca_adc_get_value.

Esempio:

```
/* Define the callback function to handle the result */
void my_adc_callback(u16 value) {
    /* Process the ADC result here */
}

/* Initialize ADC1 with a specific sample time and the callback */
if (ca_adc_hw_init_periph_isr(1, 15, my_adc_callback)) {
    /* Start the conversion on channel 5 */
    ca_adc_hw_start_isr(1, 5);
} else {
    /* Handle initialization error */
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_adc_hw_init_periph_isr() in my embedded application?
A: You should call ca_adc_hw_init_periph_isr() when you want to configure an ADC in interrupt mode to handle conversion results asynchronously. This is useful when you need to process ADC data without blocking the main execution flow, such as in real-time systems or when handling multiple sensors.

Code example conf:0.90 **Q:** Can you provide a minimal working example of how to use ca_adc_hw_init_periph
A:

```
u8 adc_num = 1;
u8 sample_time = 13;
ca_adc_isr_cb_t callback = my_adc_callback;
bool success = ca_adc_hw_init_periph_isr(adc_num, sample_time, callback);
if (!success) {
    // Handle error
}
```

Note: my_adc_callback must be defined as a function of type ca_adc_isr_cb_t.

Parameters conf:0.95 **Q:** What are the valid values for the adc_num parameter in ca_adc_hw_init_periph_isr
A: The adc_num parameter must be 1, 2, or 3 depending on the MCU family. For F1xx and H5xx, valid values are 1 and 2. For F4xx, F7xx, and H7xx, valid values are 1, 2, and 3. Values outside this range will cause the function to return false.

Return conf:0.95 **Q:** What does it mean when ca_adc_hw_init_periph_isr() returns false?
A: A return value of false indicates one of the following failures: invalid adc_num (less than 1 or greater than 3), hardware initialization failure via ca_adc_hw_init_periph, or a NULL callback function was passed.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_adc_hw_init_periph_isr()?
A: Common mistakes include passing an invalid adc_num (outside 1-3 range), not properly initializing the ADC pin before calling this function, passing a NULL callback function, or failing to ensure that the ADC clock is enabled and the pin is configured correctly.

Behavior conf:0.95 **Q:** Is ca_adc_hw_init_periph_isr() thread-safe and can it be called from an ISR?
A: No, ca_adc_hw_init_periph_isr() is not thread-safe because it modifies a global variable (s_isr_cb). Additionally, it should not be called from an ISR as it's a hardware initialization function and not meant for interrupt context.

HW differences conf:0.85 **Q:** How does ca_adc_hw_init_periph_isr() differ between H5xx and H7xx MCUs in terms of hardware implementation?
A: On H5xx, only ADC1 is used regardless of the adc_num parameter, and the function uses ADC1_IRQn for ADC1 and ADC2_IRQn for ADC2. On H7xx, the function supports ADC1, ADC2, and ADC3, with ADC3 using ADC3_IRQn and others using ADC_IRQn. H7xx also uses ADC_TypeDef *adc = adc_inst(adc_num) to get the correct ADC instance.

`ca_adc_hw_irq_handler`

Header: `caos/drivers/inc/ca_glue_hw.h:15`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_adc_hw_irq_handler(u8 adc_num); /* gestore IRQ ADC: legge il risultato e invoca la callback */
```

Hardware-specific interrupt service routine handler for ADC peripheral completion events.

Parametri:

Nome	Tipo	Descrizione
`adc_num`	`u8`	The identifier of the ADC peripheral (e.g., 1, 2, or 3). Must match the hardware configuration limits of the specific STM32 series.

Ritorna:
void. This function does not return a value as it is an interrupt handler.

Note:

- This function is designed to be called exclusively from the hardware interrupt vector table (ISR context). Because it runs in an ISR, it must be extremely fast; performing heavy processing or blocking operations inside the callback will cause system instability or missed interrupts. It is not thread-safe in the sense that it modifies hardware registers directly, but it is re-entrant if the hardware supports multiple ADC instances. A common beginner mistake is to perform long calculations or use blocking RTOS primitives (like mutexes or delays) inside the callback function triggered by this handler, which will crash the system or cause a watchdog reset.

Uso:

- This function is the bridge between the hardware ADC peripheral and your application logic. It is typically invoked by the STM32 vector table when the 'End of Conversion' (EOC) flag is raised by the hardware. You must register a callback function using the appropriate driver initialization function before this handler is triggered, otherwise, the handler will simply exit without notifying your application. It is used in a sequence: initialize the ADC, enable the ADC interrupt in the NVIC, and then this handler will automatically execute every time a conversion finishes.

Esempio:

```c
/* Example of an ISR implementation in the vector table */
void ADC1_2_IRQHandler(void) {
    /* Call the caOS hardware handler for ADC 1 */
    ca_adc_hw_irq_handler(1);
}

/* Example of a callback function registered to handle the data */
void my_adc_callback(u8 adc_num, u16 value) {
    /* Process the 12-bit ADC result here */
    /* Keep this code short and fast! */
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:1.00 **Q:** When should ca_adc_hw_irq_handler be called, and why is it important to call it from an ISR?
A: ca_adc_hw_irq_handler should be called from an Interrupt Service Routine (ISR) to handle ADC conversion complete interrupts. It is important because the function reads hardware registers (like DR and SR/ISR) and clears the EOC flag, which must be done in the ISR context to avoid race conditions and ensure correct ADC operation. It must not be called from a task or thread context.

Code example conf:1.00 **Q:** Can you provide a minimal C code example showing how to use ca_adc_hw_irq_handler with caOS types?
A:

```c
u16 adc_callback(u8 adc_num, u16 value) {
    return value;
}

void my_adc_isr(void) {
```

```
6     ca_adc_hw_irq_handler(1);
7 }
```

Parameters conf:1.00 **Q:** What does the adc_num parameter represent, and what are the valid values for it?
A: The adc_num parameter specifies which ADC peripheral to handle. Valid values depend on the MCU family: 1 and 2 for F1xx, 1, 2, and 3 for F4xx and F7xx, 1 and 2 for H5xx, and all ADCs for H7xx. Any other value causes the function to return immediately without effect.

Return conf:1.00 **Q:** Does ca_adc_hw_irq_handler return any value, and what happens if it fails?
A: The function does not return any value. It cannot fail in a strict sense, but it may silently ignore invalid inputs such as an invalid adc_num or be called from a non-ISR context. It always attempts to read the ADC data register and invoke the callback if valid.

Error/Edge case conf:1.00 **Q:** What are the most common mistakes when using ca_adc_hw_irq_handler?
A: Common mistakes include calling the function with an invalid adc_num (outside the valid range for the MCU family), calling it from a task or thread context instead of an ISR, or calling it when the EOC flag is not set. Another mistake is not ensuring that the callback function is properly registered via the s_isr_cb array.

HW differences conf:1.00 **Q:** How does ca_adc_hw_irq_handler differ between H5xx and H7xx MCU families?
A: In H5xx, the function always uses ADC1 regardless of the adc_num parameter, and checks the ADC1->ISR register for EOC. In H7xx, the function uses the adc_inst(adc_num) macro to get the correct ADC register base and checks the ISR flag for that specific ADC. H7xx also supports up to 3 ADCs, while H5xx only supports 2.

`ca_adc_hw_read`

Header: `caos/drivers/inc/ca_glue_hw.h:12`
GUARD: `EN_CA_GLUE_HW`

```
1 extern u16 ca_adc_hw_read(u8 adc_num, u8 channel); /* avvia conversione
    e ritorna il valore grezzo a 12 bit */
```

Triggers a single ADC conversion on a specific channel and returns the 12-bit raw digital value.

Parametri:

Nome	Tipo	Descrizione
`adc_num`	`u8`	The identifier of the ADC peripheral (e.g., 1, 2, or 3). Must correspond to valid hardware instances defined in the BSP.
`channel`	`u8`	The analog input channel index. Must be a valid channel supported by the selected ADC peripheral.

Ritorna:
A 12-bit unsigned integer (0-4095) representing the converted voltage. Note that this function does not return error codes; it assumes valid hardware state.

Note:

- This function is strictly blocking, meaning the CPU will sit in a 'while' loop waiting for the hardware to finish the conversion. Because it uses a busy-wait loop, it is NOT thread-safe

if multiple tasks access the same ADC instance simultaneously, and it must NEVER be called from an Interrupt Service Routine (ISR) as it would cause a system hang. Before calling this, ensure the ADC peripheral has been properly initialized via the system clock and GPIO configuration functions. A common beginner mistake is calling this function at a high frequency in a low-priority task, which can starve other tasks of CPU time due to the blocking nature of the hardware wait.

Uso:

- This function is typically used for polling-based sensor reading where high-speed, non-blocking asynchronous operation is not required. It is usually paired with an initialization function like 'ca_adc_init_isr' or a generic hardware setup routine that configures the GPIO pins to analog mode. In a typical application, you would call this function periodically within a task loop to sample analog data, process the raw 12-bit value, and then perform control logic based on the result.

Esempio:

```
/* Example: Reading a potentiometer value from ADC1, Channel 5 */
u16 raw_value;

/* Ensure ADC is initialized before reading */
if (ca_adc_init_isr(&my_adc_config, my_callback)) {
    /* Blocking read: CPU waits here until conversion completes */
    raw_value = ca_adc_hw_read(1, 5);

    /* Process the 12-bit raw value (0 to 4095) */
    if (raw_value > 2048) {
        /* Perform action if voltage is above threshold */
    }
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_adc_hw_read() in my caOS project?
A: You should use ca_adc_hw_read() when you need to perform a blocking, non-thread-safe analog-to-digital conversion in a non-critical context. It is intended for use in non-ISR, non-concurrent code where you can tolerate the blocking behavior and don't require thread safety.

Code example conf:0.95 **Q:** Can you show a minimal example of how to read ADC channel 5 from ADC1 using ca_adc_hw_read()?
A: u16 adc_value = ca_adc_hw_read(1, 5);

Parameters conf:0.95 **Q:** What are the valid ranges for adc_num and channel parameters in ca_adc_hw_read()?
A: adc_num must be 1, 2, or 3. channel must be between 0 and 18. Invalid values for either parameter may cause undefined behavior or incorrect readings.

Return conf:0.95 **Q:** What does ca_adc_hw_read() return, and how can I tell if the read was successful?
A: It returns a 12-bit value representing the ADC conversion result. There is no error code returned, so it's impossible to distinguish a valid 0 value from a failed read.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_adc_hw_read()?
A: Common mistakes include calling it from an ISR, calling it from multiple threads simultaneously, not initializing the ADC or configuring the channel beforehand, and assuming that a returned value of 0 indicates an error.

Behavior conf:0.95 **Q:** Is ca_adc_hw_read() thread-safe and ISR-safe?
A: No, ca_adc_hw_read() is neither thread-safe nor ISR-safe. It is a blocking function and should not be called from an ISR or from multiple threads concurrently.

HW differences conf:0.85 **Q:** How does ca_adc_hw_read() differ between STM32H7xx and STM32F4xx families?
A: On STM32F4xx, it uses ADC_CR2_SWSTART to start conversion and ADC_SR_EOC to check completion. On STM32H7xx, it uses ADC_CR_ADSTART and ADC_ISR_EOC, and also sets ADC_PCSEL for channel pre-selection.

---

ca_adc_hw_start_isr

Header: `caos/drivers/inc/ca_glue_hw.h:14`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_adc_hw_start_isr(u8 adc_num, u8 channel); /* avvia
    conversione asincrona     il risultato arriva via callback */
```

Triggers an asynchronous ADC conversion on a specific channel without blocking the CPU.

Parametri:

Nome	Tipo	Descrizione
`adc_num`	`u8`	The index of the ADC peripheral to use (e.g., 1 for ADC1). Note that some hardware implementations ignore this and always default to ADC1.
`channel`	`u8`	The specific analog input channel to sample. Valid values are typically 0-19 depending on the MCU architecture.

Ritorna:
void. This function does not return a status code; it assumes the hardware is correctly initialized and ready to start.

Note:

- This function is non-blocking and designed to be called from within an Interrupt Service Routine (ISR) or a high-priority task. It is not thread-safe by itself, so ensure that no other process is modifying the ADC registers simultaneously. A common beginner mistake is calling this function before the ADC peripheral has been properly clocked and configured, which will result in a hard fault or silent failure. Because it is asynchronous, the function returns immediately after triggering the hardware; you must rely on a separate interrupt handler or callback mechanism to retrieve the conversion result once the hardware sets the End-Of-Conversion (EOC) flag.

Uso:

- This function is typically used in a continuous sampling loop or a triggered data acquisition system. It is usually paired with an ADC interrupt handler: you call this function to start the conversion, and when the hardware finishes, the ADC interrupt fires, allowing you to read the result and potentially call this function again to start the next conversion. It is essential for high-performance applications where you cannot afford to wait for the ADC to finish (blocking) while the CPU does other work. In a typical sequence, you initialize the ADC, enable the ADC interrupt, and then call this function to kick off the first conversion.

Esempio:

```
/* Example: Triggering a new conversion inside an ADC completion interrupt */
void ADC_IRQHandler(void) {
    /* Check if the conversion is complete */
    if (ADC1->ISR & ADC_ISR_EOC) {
```

```
        u16 result = (u16)ADC1->DR; /* Read the conversion result */

        /* Process the data... */

        /* Trigger the next conversion on the same channel */
        /* Assuming channel 5 was the one being sampled */
        ca_adc_hw_start_isr(1, 5);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_adc_hw_start_isr() and what is its main advantage over synchronous ADC conversion?
A: You should use ca_adc_hw_start_isr() when you need non-blocking ADC conversions, especially in real-time applications where blocking the execution is not acceptable. Its main advantage is that it initiates the conversion via ISR and returns immediately, allowing other tasks or ISRs to run while the ADC conversion is in progress. The result is handled through a callback function, making it suitable for time-critical systems.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to call ca_adc_hw_start_isr() with proper caOS types?
A:

```
u8 adc_num = 1;
u8 channel = 5;
ca_adc_hw_start_isr(adc_num, channel);
```

Parameters conf:0.90 **Q:** What are the valid ranges for adc_num and channel parameters in ca_adc_hw_start_isr(), and what happens if invalid values are passed?
A: The adc_num parameter must be 1 or 2; values 0 or greater than 2 are invalid and may cause memory access violations. The channel parameter must be between 0 and 15; values outside this range may result in incorrect channel conversion or undefined behavior. The function does not return an error code, so it's the caller's responsibility to ensure valid inputs.

HW differences conf:0.85 **Q:** How does ca_adc_hw_start_isr() implementation differ across STM32 families like F1, F4, F7, H5, and H7?
A: The F1, F4, and F7 families all use a similar approach by setting the channel in SQR3 and triggering SWSTART in CR2. The H5 family uses ADC1 directly, sets the channel in SQR1, clears EOC in ISR, and starts the ADC. The H7 family additionally sets the PCSEL register to enable the channel before starting the conversion, which is a more complex configuration for its ADC architecture.

`ca_adc_hw_stop`

Header: `caos/drivers/inc/ca_glue_hw.h:93`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_adc_hw_stop(u8 adc_num); /* stops conversions and
   disables the ADC (clears ADEN) */
```

Safely stops ADC conversions and powers down the hardware peripheral.

Parametri:

Nome	Tipo	Descrizione
`adc_num`	`u8`	The index of the ADC peripheral to stop (e.g., 1 for ADC1). Note: Current implementations primarily target ADC1, so this parameter is reserved for future multi-ADC support.

Ritorna:
Returns true if the ADC was successfully disabled or was already off. Returns false if the hardware failed to acknowledge the stop command within the internal timeout period.

Note:

- This function is blocking because it performs a busy-wait loop to ensure the hardware registers (like ADEN or ADSTART) have cleared. It is not thread-safe; if multiple tasks share the same ADC, you must wrap this call in a mutex to prevent race conditions. It is generally not safe to call this from an ISR because the blocking loops could stall the system if the hardware hangs. The most common beginner mistake is calling this while a DMA transfer is still active, which can cause the ADC to lock up or leave the DMA in an inconsistent state; always ensure DMA is stopped before calling this.

Uso:

- This function is used to cleanly shut down the ADC peripheral during system power-down, mode switching, or error recovery. It is the mandatory cleanup step after using functions like ca_adc_init or ca_adc_init_isr. In a typical sequence, you would first stop any active DMA or interrupts, call ca_adc_hw_stop to release the hardware, and then potentially re-initialize the peripheral or enter a low-power sleep mode.

Esempio:

```
/* Stop the ADC before reconfiguring or entering sleep */
if (ca_adc_hw_stop(1)) {
    /* ADC is now disabled and analog switches are released */
} else {
    /* Hardware failed to stop; handle as a critical system error */
    /* Log error using available system logging or diagnostic tools */
}
```

Q&A — fine-tuning review *7/7 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_adc_hw_stop in my caOS application?
A: You should call this function during system shutdown, power-saving transitions, or when you need to reconfigure the ADC for a different mode of operation. It acts as the counterpart to initialization functions like ca_adc_init.

Code example conf:1.00 **Q:** Provide a minimal example of how to safely stop the ADC using caOS types.
A: u8 adc_id = 1; if (!ca_adc_hw_stop(adc_id)) { /* handle hardware failure */ }

Parameters conf:1.00 **Q:** What does the adc_num parameter control, and are there specific values I should use?
A: The adc_num parameter is intended to specify the index of the ADC peripheral (e.g., 1 for ADC1). However, in the current implementation, this parameter is ignored as the driver uses a fixed ADC1 reference.

Return conf:1.00 **Q:** What do the boolean return values of ca_adc_hw_stop indicate?
A: It returns true if the ADC was successfully disabled or was already off. It returns false if the hardware failed to acknowledge the stop command within the internal timeout period.

Error/Edge case conf:1.00 **Q:** What is the most common mistake when calling ca_adc_hw_stop?
A: The most common mistake is calling this function while a DMA transfer is still active, which can lead to incomplete data buffers or unexpected hardware states.

Behavior conf:1.00 **Q:** Is ca_adc_hw_stop thread-safe and safe to call from an ISR?
A: The function is blocking because it waits for hardware status register updates. It is not thread-safe; if multiple tasks access the same ADC, you must use a mutex. It is generally not recommended to call it from an ISR due to the risk of an infinite loop if the hardware fails to respond.

HW differences conf:1.00 **Q:** How does the implementation of ca_adc_hw_stop differ between F1/F4/F7 and H5/H7/U5 families?
A: On F1, F4, and F7 families, the function simply clears the ADON bit in CR2. On H5, H7, and U5 families, the function performs a more complex handshake using ADSTP and ADDIS bits to ensure conversions are stopped before disabling the peripheral, and H7/U5 additionally clear the PCSEL register to release I/O pad switches.

`ca_adc_hw_stop_isr`

Header: `caos/drivers/inc/ca_glue_hw.h:94`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_adc_hw_stop_isr(u8 adc_num); /* leaves the ADC on but
   disables its end-of-conversion interrupt */
```

Disables the ADC end-of-conversion interrupt while keeping the peripheral powered and active.

Parametri:

Nome	Tipo	Descrizione
`adc_num`	`u8`	The ADC peripheral index (e.g., 1). Note: current implementation ignores this and defaults to ADC1.

Ritorna:
Always returns true.

Note:

- This function is non-blocking and safe to call from an ISR. It is designed to silence the interrupt mechanism without powering down the ADC hardware, which is critical if you intend to perform polled reads immediately after. If you were to call a full stop function instead, the ADC would lose its configuration and require a complete re-initialization, causing subsequent reads to return zero or invalid data. A common mistake is calling this when you actually want to power down the peripheral; use ca_adc_stop() for that purpose instead.

Uso:

- This function is typically used when transitioning from an interrupt-driven sampling mode to a polled mode, or when cleaning up after an ISR-based acquisition sequence. It is usually paired with ca_adc_isr_done() to signal that the interrupt-driven phase is complete. The typical sequence is: ca_adc_start_isr() to begin sampling, then ca_adc_hw_stop_isr() to silence the hardware interrupt, followed by ca_adc_isr_done() to finalize the driver state.

Esempio:

```
/* Stop the interrupt-driven sampling process */
if (ca_adc_hw_stop_isr(1)) {
    /* Signal the driver that the ISR phase is finished */
    ca_adc_isr_done(1);

    /* Now safe to perform a manual, polled read without ISR interference */
    u32 value = ca_adc_read(1);
    ca_printf("Polled ADC value: %u\n", value);
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_adc_hw_stop_isr instead of a standard stop function?
A: Use ca_adc_hw_stop_isr when you need to disable the end-of-conversion interrupt while keeping the ADC peripheral powered and active. This is typically done when transitioning from interrupt-driven sampling to polled mode, or when cleaning up after an ISR-based acquisition sequence.

Code example conf:1.00 **Q:** Provide a minimal example of how to use ca_adc_hw_stop_isr in a sequence.
A: u8 adc_idx = 1; ca_adc_start_isr(adc_idx); /* ... perform sampling ... */ ca_adc_hw_stop_isr(ad ca_adc_isr_done(adc_idx);

Parameters conf:1.00 **Q:** What does the u8 adc_num parameter control?
A: The u8 adc_num parameter represents the ADC peripheral index. However, the current implementation ignores this value and defaults to ADC1 across all supported hardware families.

Return conf:1.00 **Q:** What are the possible return values for ca_adc_hw_stop_isr?
A: The function always returns true. There are no failure conditions defined for this operation.

Error/Edge case conf:1.00 **Q:** What is a common mistake when using ca_adc_hw_stop_isr?
A: A common mistake is calling this function when you actually intend to power down the peripheral. If you want to power down the ADC, use ca_adc_stop() instead, as ca_adc_hw_stop_isr keeps the hardware active and configured.

HW differences conf:1.00 **Q:** How does the implementation of ca_adc_hw_stop_isr differ between F1/F4/F7 and H5/H7/U5 families?
A: On F1, F4, and F7 families, the function clears the EOCIE bit in the CR1 register and the EOC bit in the SR register. On H5, H7, and U5 families, it clears the EOCIE bit in the IER register, checks for active conversions in the CR register, performs a stop sequence if necessary, and clears the EOC bit in the ISR register.

`ca_adc_init`

Header: `caos/drivers/inc/adc.h:35`
Source: `caos/drivers/src/adc.c:132:143`
GUARD: `EN_ADC`

```
1 bool ca_adc_init(const ca_adc_config_t *cfg)
```

Initializes the hardware ADC peripheral and configures its sampling parameters.

Parametri:

Nome	Tipo	Descrizione
`cfg`	`const ca_adc_config_t *`	Pointer to a configuration structure containing the ADC index (1-3) and the desired sample time. Passing a NULL pointer or an invalid ADC index (outside 1-3) will cause the function to return false immediately.

Ritorna:
Returns true if the hardware clock and peripheral initialization were successful. Returns false if the input configuration is invalid, the hardware clock fails to initialize, or the peripheral configuration fails.

Note:

- This function is non-blocking and performs direct hardware register manipulation to set up the ADC clock and peripheral registers. It is not thread-safe, so you must ensure that

initialization happens during the system startup phase before any tasks attempt to read from the ADC. It is strictly forbidden to call this from an Interrupt Service Routine (ISR) because it performs hardware setup that may take time and is not re-entrant. A common beginner mistake is to call this function multiple times for the same ADC index without checking if it is already initialized, which can lead to unpredictable hardware states or clock configuration conflicts.

Uso:

- This function is the mandatory first step in the ADC lifecycle. It is typically called during the system initialization sequence, immediately after hardware clocks are enabled but before any pin-specific configuration (like ca_adc_pin_init) or data acquisition occurs. In a typical application, you define a configuration structure, pass it to this function, and then proceed to configure the specific GPIO pins associated with the ADC channels. If you skip this step, subsequent calls to read ADC values will fail or return garbage data because the peripheral clock will be disabled.

Esempio:

```
/* Define the configuration for ADC 1 with a specific sample time */
ca_adc_config_t adc_cfg = { .adc_num = 1, .sample_time = 15 };

/* Initialize the ADC peripheral */
if (ca_adc_init(&adc_cfg)) {
    /* Success: now configure the specific pin for the ADC channel */
    ca_adc_pin_init(1, 10);
} else {
    /* Handle initialization failure: log error or enter safe state */
    while(1);
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_adc_init() in my caOS application?
A: You should call ca_adc_init() before performing any ADC readings. It must be called once to configure the ADC peripheral with the correct channel number and sampling time. The function is blocking and must be called before any ADC operations.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_adc_init() with caOS types?
A: ca_adc_config_t config = { .adc_num = 1, .sampling_time = 15 }; bool result = ca_adc_init(&config); if (!result) { /* Handle error */ }

Parameters conf:0.90 **Q:** What does the adc_num parameter in ca_adc_config_t represent, and what are valid values?
A: The adc_num parameter specifies which ADC peripheral to initialize. Valid values are 1, 2, or 3. Any value outside this range will cause ca_adc_init() to return false.

Return conf:0.95 **Q:** What does ca_adc_init() return, and what causes it to return false?
A: ca_adc_init() returns true on successful initialization. It returns false if the cfg parameter is NULL or if adc_num is outside the valid range of 1 to 3.

Error/Edge case conf:0.85 **Q:** What are common mistakes when calling ca_adc_init()?
A: Common mistakes include passing a NULL pointer for cfg, or specifying an adc_num outside the valid range (1-3). Additionally, calling ca_adc_init() from an ISR or without enabling the ADC clock beforehand may lead to unexpected behavior, though these are not directly reflected in the return value.

ca_adc_init_isr

Header: caos/drivers/inc/adc.h:49
Source: caos/drivers/src/adc.c:157:162
GUARD: EN_ADC

```
bool ca_adc_init_isr(const ca_adc_config_t *cfg, ca_adc_isr_cb_t cb)
```

Initializes an ADC peripheral in interrupt-driven mode with a user-defined callback function.

Parametri:

Nome	Tipo	Descrizione
cfg	const ca_adc_config_t *	Pointer to a configuration structure containing the ADC number (1-3) and sample time. Must not be NULL.
cb	ca_adc_isr_cb_t	Function pointer to the callback that will be executed when the ADC conversion completes. Must not be NULL.

Ritorna:
Returns true if the ADC peripheral was successfully initialized and the interrupt enabled. Returns false if the configuration pointer is NULL, the ADC number is out of range (1-3), the callback is NULL, or the hardware initialization fails.

Note:

- This function is non-blocking and configures the hardware to trigger an interrupt upon conversion completion. It is not thread-safe by itself, so ensure it is called during system initialization or from a controlled task context. A common beginner mistake is passing a NULL callback or an invalid ADC index, which will cause the function to fail immediately. Before calling this, ensure the system clock for the ADC peripheral is managed correctly by the underlying hardware abstraction layer, as this function relies on ca_clock_hw_adc_init to prepare the peripheral.

Uso:

- This function is typically used when you need to perform non-blocking ADC sampling, allowing the CPU to perform other tasks while waiting for the hardware conversion to finish. It is usually called once during the setup phase of a specific module or via a CLI command to enable diagnostic monitoring. It is almost always paired with ca_adc_start_isr to trigger the actual conversion process after initialization is complete. Once the conversion finishes, the provided callback is executed, where you would typically read the result and potentially trigger the next conversion.

Esempio:

```
ca_adc_config_t my_adc_cfg = { .adc_num = 1, .sample_time = 4 };

// Define the callback function to handle ADC data
void my_adc_callback(u16 value) {
    // Process the ADC result here
}

// Initialize the ADC for interrupt mode
if (ca_adc_init_isr(&my_adc_cfg, my_adc_callback)) {
    // Start the conversion on channel 5
    ca_adc_start_isr(1, 5);
} else {
    // Handle initialization error
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_adc_init_isr() in my embedded application?
A: You should use ca_adc_init_isr() when you need to perform ADC conversions in the background without blocking the main thread. This function is ideal for acquiring sensor data or other analog signals asynchronously, allowing your application to continue processing other tasks while the ADC conversion is in progress.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_adc_init_isr() with caOS types?
A: ca_adc_config_t config = { .adc_num = 1, .sample_time = 15 }; ca_adc_isr_cb_t callback = my_adc_callback; bool success = ca_adc_init_isr(&config, callback);

Parameters conf:0.90 **Q:** What are the valid values for adc_num in the ca_adc_config_t structure passed to ca_adc_init_isr()?
A: The adc_num field in ca_adc_config_t must be an integer between 1 and 3 inclusive. Values outside this range, such as 0 or 4, are invalid and will cause ca_adc_init_isr() to return false.

Return conf:0.95 **Q:** What does it mean if ca_adc_init_isr() returns false?
A: If ca_adc_init_isr() returns false, it indicates that the initialization failed due to invalid parameters. This can happen if cfg is NULL, adc_num is not between 1 and 3, or cb is NULL.

`ca_adc_irq_handler`

Header: `caos/drivers/inc/adc.h:55`
Source: `caos/drivers/src/adc.c:170:173`
GUARD: `EN_ADC`

```
1 void ca_adc_irq_handler(u8 adc_num)
```

Dispatches the hardware ADC interrupt to the appropriate caOS driver handler for a specific ADC peripheral.

Parametri:

Nome	Tipo	Descrizione
`adc_num`	`u8`	The index of the ADC peripheral (e.g., 1 for ADC1, 2 for ADC2). Must match the hardware peripheral ID defined in your STM32 configuration.

Ritorna:
void

Note:

- This function is strictly intended to be called from within the STM32 hardware interrupt vector table (the ISR). Because it is an ISR handler, it must execute as quickly as possible to avoid blocking other system interrupts or causing timing jitter in the RTOS scheduler. It is not thread-safe in the sense that it should never be called from a standard task; it is the entry point for the hardware signal itself. A common beginner mistake is to perform heavy processing or blocking operations inside the function called by this handler, which will crash the system or cause missed ADC samples. Ensure that the ADC peripheral has been properly initialized via ca_adc_start_isr before this handler is triggered, otherwise, the hardware may trigger an interrupt for an unconfigured peripheral, leading to undefined behavior.

Uso:

- This function is used exclusively in the STM32 startup or HAL interrupt vector files (e.g., stm32f4xx_it.c). You place it inside the specific ADCx_IRQHandler function generated by the IDE. It acts as the bridge between the raw hardware interrupt and the caOS driver logic. It is typically paired with ca_adc_start_isr, which sets up the GPIO pins and triggers the hardware to begin sampling. Once the hardware completes a conversion, it fires the interrupt, which calls this handler to process the data or signal a waiting task.

Esempio:

```
/* Inside stm32f4xx_it.c - The hardware interrupt vector */
void ADC_IRQHandler(void) {
    /* Notify caOS that an ADC interrupt occurred for ADC1 */
    /* This handles the low-level register clearing and data retrieval */
    ca_adc_irq_handler(1);
}

/* Inside your application task */
void start_sampling(void) {
    /* Initialize pins and start the hardware conversion process */
    ca_adc_start_isr(1, 0);
    /* The system will now automatically call ca_adc_irq_handler when data is
    ready */
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_adc_irq_handler() and why is it important to invoke it from an ISR?
A: You should call ca_adc_irq_handler() from within an ADC interrupt service routine (ISR) that is registered in the interrupt vector table for the ADC peripheral. It is important to invoke it from an ISR because the function is designed to handle ADC interrupts and modifies ADC hardware state. Calling it from a non-ISR context may lead to undefined behavior. The function is used to process ADC conversion results and is typically invoked automatically by the system when an ADC interrupt occurs.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_adc_irq_handler() with caOS types?
A: void ADC1_IRQHandler(void) { ca_adc_irq_handler(0); }
// This example assumes ADC1 is being used and the interrupt is properly configured.
// The function ca_adc_irq_handler(0) handles the interrupt for ADC1.

Parameters conf:0.90 **Q:** What does the adc_num parameter in ca_adc_irq_handler() represent, and what are the valid values for this parameter?
A: The adc_num parameter in ca_adc_irq_handler() represents the number of the ADC peripheral to be handled, where 0 typically corresponds to ADC1, 1 to ADC2, and so on. Valid values are integers between 0 and the maximum number of ADCs available on the specific STM32 chip. Using invalid values (e.g., greater than the number of ADCs on the chip) can lead to memory access violations or undefined behavior.

`ca_adc_isr_done`

Header: `caos/drivers/inc/adc.h:63`
Source: `caos/drivers/src/adc.c:305:309`
GUARD: `EN_ADC`

```
void ca_adc_isr_done(u8 adc_num)
```

Signals that an ADC interrupt operation is complete, releasing the ADC peripheral for further use.

Parametri:

Nome	Tipo	Descrizione
adc_num	u8	The ADC peripheral index. Valid values are 1, 2, or 3. Values outside this range are ignored.

Ritorna:
void

Note:

- This function is non-blocking and thread-safe because it performs a simple memory write to a status array. It is specifically designed to be called from an Interrupt Service Routine (ISR) context, as it does not use mutexes or any OS primitives that could cause a deadlock or context switch. The most common mistake is passing an invalid ADC index, which results in the function returning immediately without clearing the busy flag, effectively locking the ADC peripheral permanently.

Uso:

- This function is used in the final stage of an asynchronous ADC conversion sequence. It is typically called inside the ADC interrupt handler after the conversion result has been retrieved or stored. It pairs with ca_adc_start_isr, which sets the busy flag, and acts as the 'unlock' mechanism so that subsequent calls to ADC functions from tasks or other interrupts can proceed without being blocked by a stale 'busy' state.

Esempio:

```c
/* Example: Handling an ADC conversion completion in an ISR */
void ADC1_IRQHandler(void) {
    u8 adc_num = 1;
    /* Check if conversion is complete and read the value */
    /* ca_adc_irq_handler(adc_num); // Internal hardware flag clearing */

    /* Signal that the ADC is now free for other tasks */
    ca_adc_isr_done(adc_num);
}

/* Example: Starting a conversion from a task */
void start_adc_task(void) {
    u8 adc_num = 1;
    u8 channel = 0;
    ca_adc_pin_init(adc_num, channel);
    ca_adc_start_isr(adc_num, channel);
    /* The ADC is now busy until the ISR calls ca_adc_isr_done */
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_adc_isr_done and why is it necessary?
A: You should call ca_adc_isr_done inside the ADC interrupt handler after the conversion result has been retrieved or stored. It is necessary to signal that the ADC operation is complete, acting as an 'unlock' mechanism to clear the busy flag so that subsequent ADC requests from tasks or other interrupts are not blocked by a stale state.

Code example conf:1.00 **Q:** Provide a minimal C example of how to use ca_adc_isr_done within an ISR.
A: void ADC1_IRQHandler(void) { u8 adc_id = 1; /* Retrieve conversion result here */ ca_adc_isr_done(adc_id); }

Parameters conf:1.00 **Q:** What are the valid values for the adc_num parameter, and what happens if I pass an invalid one?
A: The adc_num parameter must be 1, 2, or 3. If you pass a value outside this range, the function ignores it and returns immediately without clearing the busy flag, which will effectively lock the ADC peripheral permanently.

`ca_adc_pin_init`

Header: `caos/drivers/inc/adc.h:40`
Source: `caos/drivers/src/adc.c:145:148`
GUARD: `EN_ADC`

```
void ca_adc_pin_init(u8 adc_num, u8 channel)
```

Configures a specific microcontroller pin to function as an analog input for the ADC peripheral.

Parametri:

Nome	Tipo	Descrizione
`adc_num`	`u8`	The identifier of the ADC peripheral (e.g., 1, 2, or 3). Must match the hardware capabilities of your specific STM32 chip.
`channel`	`u8`	The specific ADC channel index to be mapped to the pin. Ensure this channel is physically connected to the pin you intend to use according to the datasheet.

Ritorna:
void

Note:

- This function is a hardware abstraction layer (HAL) wrapper that configures the GPIO pin mode to 'Analog'. It is non-blocking and executes quickly, but it is not thread-safe; it should be called during the system initialization phase before the ADC is used for sampling. It is not intended to be called from an Interrupt Service Routine (ISR) as it performs hardware register configuration which should be done in a controlled environment. A common beginner mistake is calling this function before initializing the ADC peripheral itself or using an invalid channel/pin combination, which will result in the pin remaining in a digital state or causing unexpected hardware behavior.

Uso:

- This function is typically used during the system startup sequence, immediately after calling ca_adc_init(). The standard workflow is: first, define the ADC configuration structure; second, initialize the ADC peripheral with ca_adc_init(); third, call ca_adc_pin_init() to map the physical pin to the ADC channel; and finally, start reading values using ca_adc_read(). It is essential to pair this with the correct hardware pin mapping found in your STM32 reference manual to avoid reading floating or incorrect voltages.

Esempio:

```
/* Define ADC parameters */
#define ADC_NUM 1
#define ADC_CHANNEL 10

/* Initialize ADC and pin during system setup */
```

```
ca_adc_config_t adc_cfg = { ADC_NUM, ADC_SAMPLE_TIME };
ca_adc_init(&adc_cfg);

/* Configure the physical pin for analog input */
ca_adc_pin_init(ADC_NUM, ADC_CHANNEL);

/* Now it is safe to read the sensor value */
u16 sensor_value = ca_adc_read(ADC_NUM, ADC_CHANNEL);
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_adc_pin_init() in my caOS application?
A: You should call ca_adc_pin_init() before attempting to read analog values using ca_adc_read(). This function initializes the GPIO pin corresponding to the specified ADC channel, making it ready for analog input. It must be called once per pin you intend to use, and it's typically called during system initialization or driver setup.

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_adc_pin_init() with caOS types?
A: u8 adc_num = 1; u8 channel = 5; ca_adc_pin_init(adc_num, channel);

Parameters conf:0.90 **Q:** What are the valid values for the adc_num and channel parameters in ca_adc_pin_init()?
A: The adc_num parameter must be 1, 2, or 3, depending on the STM32 chip's ADC peripherals. A value of 0 or greater than 3 is invalid. The channel parameter must be between 0 and 18, inclusive, based on the ADC channel capabilities of the specific STM32 chip. Values outside this range will cause an error.

Return conf:0.85 **Q:** Does ca_adc_pin_init() return any value, and what can cause it to fail?
A: The function does not return any value. It is non-blocking and not thread-safe. It cannot be called from an ISR. Failures may occur if invalid parameters are passed, such as an adc_num outside the valid range (1-3) or a channel outside the valid range (0-18). It also fails if the underlying GPIO or ADC initialization fails.

`ca_adc_read`

Header: `caos/drivers/inc/adc.h:43`
Source: `caos/drivers/src/adc.c:151:154`
GUARD: `EN_ADC`

```
u16 ca_adc_read(u8 adc_num, u8 channel)
```

Reads the raw digital value from a specific ADC peripheral and channel.

Parametri:

Nome	Tipo	Descrizione
`adc_num`	`u8`	The index of the ADC peripheral (e.g., 1, 2, or 3). Must be within the hardware-supported range defined by the MCU datasheet.
`channel`	`u8`	The specific input channel index to sample. Must correspond to a pin configured for analog input.

Ritorna:
Returns a 16-bit unsigned integer representing the raw ADC conversion result (typically 0 to 4095 for a 12-bit ADC).

Note:

- This function is a synchronous, blocking call that waits for the hardware conversion to complete before returning the value. It is generally not thread-safe if multiple tasks access the same ADC peripheral simultaneously, so ensure you have external synchronization if sharing resources. It is typically safe to call from an ISR if the underlying hardware driver supports it, but be aware that blocking in an ISR is bad practice as it stalls the entire system. A common beginner mistake is calling this function without first initializing the GPIO pins and the ADC peripheral using ca_adc_pin_init, which will result in reading garbage data or potentially triggering a hardware fault.

Uso:

- This function is used to retrieve analog sensor data, such as voltage levels, potentiometers, or temperature sensors. It is typically used in a periodic loop (e.g., inside a task using ca_os_sleep) to monitor environmental changes over time. The standard workflow involves calling ca_adc_pin_init once during system startup to configure the hardware, followed by repeated calls to ca_adc_read to fetch the current state. It is often paired with mathematical scaling logic to convert the raw 12-bit value into meaningful units like millivolts or physical measurements.

Esempio:

```
/* Initialize the pin once before reading */
ca_adc_pin_init(1, 4);

while (1) {
    /* Read raw value from ADC1, channel 4 */
    u16 raw_val = ca_adc_read(1, 4);

    /* Convert 12-bit raw value (0-4095) to millivolts (assuming 3.3V reference)
     */
    u32 mv = ((u32)raw_val * 3300U) / 4095U;

    /* Output the result to the console */
    ca_stdout_printf_n("Voltage: %u.%03u V", (unsigned)(mv / 1000U), (unsigned)(
    mv % 1000U));

    /* Wait 500ms before next sample to avoid flooding the CPU */
    ca_os_sleep(500);
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_adc_read() in a caOS project?
A: You should use ca_adc_read() when you need to perform a blocking analog-to-digital conversion on a specific ADC channel. It is intended for non-critical contexts where thread safety is not a concern and where you can afford the blocking behavior. It must not be used in ISRs or high-frequency loops due to its blocking nature and lack of thread safety.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_adc_read()?
A: u16 adc_value; ca_adc_pin_init(1, 5); adc_value = ca_adc_read(1, 5); // adc_value now holds the converted value from ADC1 channel 5

Parameters conf:0.90 **Q:** What are the valid ranges for adc_num and channel parameters in ca_adc_read()?
A: The adc_num parameter must be 1, 2, or 3. Any other value will cause an error. The channel parameter must be between 0 and 15 inclusive. Values outside this range will also cause an error. Both parameters are validated at runtime.

Return conf:0.85 **Q:** What does ca_adc_read() return, and what can cause it to fail?
A: ca_adc_read() returns a 12-bit value (0-4095) representing the analog reading from the specified ADC channel. If the function fails, it may return 0 or an invalid value.

Failures can occur if the pin was not initialized with ca_adc_pin_init(), or if invalid adc_num or channel values are passed. The function does not return explicit error codes.

---

ca_adc_start_isr

Header: caos/drivers/inc/adc.h:52
Source: caos/drivers/src/adc.c:164:168
GUARD: EN_ADC

```
void ca_adc_start_isr(u8 adc_num, u8 channel)
```

Initiates an ADC conversion sequence on a specific hardware peripheral and channel using interrupt-driven mode.

Parametri:

Nome	Tipo	Descrizione
adc_num	u8	The hardware ADC unit index (typically 1, 2, or 3). Must correspond to valid hardware peripherals on the STM32 chip.
channel	u8	The specific analog input channel index to be sampled. Must be a valid channel supported by the selected ADC unit.

Ritorna:
void. This function does not provide feedback on success or failure, assuming the hardware configuration is valid.

Note:

- This function is non-blocking, meaning it triggers the hardware and returns immediately while the conversion happens in the background. It is designed to be called after the ADC peripheral has been successfully initialized via ca_adc_init_isr. A common beginner mistake is calling this function before initializing the ADC or the GPIO pins, which will result in undefined behavior or hardware faults. Because it relies on interrupts, ensure that the global interrupt vector table is correctly configured and that the callback provided during initialization is ready to handle the incoming data.

Uso:

- This function is typically used in a setup sequence where you first initialize the ADC peripheral with a callback function, then configure the specific pin, and finally trigger the start. It is often paired with a stop function (not shown here) to manage power consumption or reconfigure the ADC for different channels. In a typical application, you call this once to begin a continuous sampling process or a single-shot conversion that will be processed asynchronously by your ISR callback.

Esempio:

```
/* Ensure the ADC is initialized first with a callback */
if (ca_adc_init_isr(&my_adc_cfg, my_data_ready_callback)) {
    /* Now start the hardware conversion on ADC1, Channel 5 */
    ca_adc_start_isr(1, 5);
} else {
    /* Handle initialization failure */
    log_error("ADC init failed");
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_adc_start_isr() in a caOS project, and what are its main benefits?
A: You should use ca_adc_start_isr() when you need to initiate a non-blocking ADC conversion on a specific channel. This function is ideal for embedded applications where you want to avoid blocking the execution flow during ADC acquisition. It allows the system to continue executing other tasks while the ADC conversion is in progress, which is particularly useful in real-time systems or when handling multiple peripherals concurrently. The function is designed to be called from a non-ISRs context and assumes that the ADC has already been initialized with an ISR setup.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_adc_start_isr() with caOS types?
A: u8 adc_num = 1; u8 channel = 5; ca_adc_start_isr(adc_num, channel);

Parameters conf:0.85 **Q:** What are the valid ranges for adc_num and channel parameters in ca_adc_start_isr(), and what happens if invalid values are passed?
A: The adc_num parameter must be 1, 2, or 3. If a value outside this range is passed, the function returns immediately without performing any action. The channel parameter should be a valid ADC channel number for the target STM32 microcontroller (e.g., 0-18 for STM32F4). Invalid channel values may lead to undefined behavior. The function does not validate these parameters internally, so it's the programmer's responsibility to ensure correct values are used.

`ca_adc_stop`

Header: `caos/drivers/inc/adc.h:54`
Source: `caos/drivers/src/adc.c:267:277`
GUARD: `EN_ADC`

```
1 bool ca_adc_stop(u8 adc_num)
```

Safely disables a specific ADC peripheral and releases associated internal resources.

Parametri:

Nome	Tipo	Descrizione
`adc_num`	`u8`	The ADC peripheral index to stop. Valid values are 1, 2, or 3. Values outside this range will cause the function to return false immediately.

Ritorna:
Returns true if the ADC was successfully stopped, or false if the provided adc_num is invalid or the hardware stop operation failed.

Note:

- This function is not thread-safe in isolation and relies on internal state management to prevent deadlocks. It is designed to be called from task context to shut down the ADC hardware, ensuring that any pending ISR-based conversions are cancelled before the peripheral is disabled. A common mistake is calling this function while an ADC conversion is still expected to complete, which can lead to inconsistent state if the hardware is powered down abruptly. It is not intended to be called directly from an ISR, as it performs logic that assumes a stable task environment.

Uso:

- Use this function when you need to power down an ADC peripheral to save energy or reconfigure the pins for a different function (like GPIO or DAC). It is typically paired with ca_adc_init or ca_adc_start_isr during the teardown phase of a driver or application

lifecycle. Always ensure that any tasks waiting on ADC data are notified or have timed out before calling this, as it effectively kills the active conversion process.

Esempio:

```c
/* Example: Safely shutting down ADC1 before reconfiguring pins */
#define ADC_NUM 1

void shutdown_adc_task(void *pvParameters) {
    /* Stop the ADC hardware to prevent further conversions */
    if (ca_adc_stop(ADC_NUM)) {
        /* ADC is now stopped, safe to reconfigure pins if necessary */
        /* ca_gpio_configure(...) would be called here to change pin mode */
    } else {
        /* Handle error: ADC_NUM was likely invalid */
    }
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_adc_stop and what is its primary purpose?
A: You should call ca_adc_stop when you need to power down an ADC peripheral to save energy or reconfigure its pins for other functions like GPIO or DAC. It is typically used during the teardown phase of a driver or application lifecycle.

Code example conf:1.00 **Q:** Provide a minimal example of how to stop ADC 2 using caOS types.
A: u8 adc_id = 2; if (ca_adc_stop(adc_id)) { /* ADC stopped successfully */ }

Parameters conf:1.00 **Q:** What are the valid values for the adc_num parameter in ca_adc_stop?
A: The adc_num parameter must be a u8 with a value of 1, 2, or 3. Providing any value outside this range will cause the function to return false immediately.

Return conf:1.00 **Q:** What do the boolean return values of ca_adc_stop indicate?
A: The function returns true if the ADC was successfully stopped. It returns false if the provided adc_num is invalid or if the hardware stop operation itself failed.

Error/Edge case conf:1.00 **Q:** What are the risks of calling ca_adc_stop while an ADC conversion is still in progress?
A: Calling this function while an ADC conversion is expected to complete can lead to an inconsistent state because the hardware is powered down abruptly, effectively killing the active conversion process.

`ca_aes_channel_decrypt`

Header: `caos/crypto/inc/aes_channel_service.h:74`
Source: `caos/crypto/src/aes_channel_service.c:164:233`
GUARD: `EN_AES_CHANNEL_SERVICE`

```c
i32 ca_aes_channel_decrypt(ca_aes_channel_t *ch, const u8 *cipher, u32
    cipher_len, u8 **out, u32 *out_len)
```

Decrypts an AES-GCM encrypted buffer using a secure channel context.

Parametri:

Nome	Tipo	Descrizione
ch	ca_aes_channel_t *	Pointer to the initialized AES channel structure. Must not be NULL.
cipher	const u8 *	Pointer to the raw encrypted data buffer, including nonce, length, ciphertext, and tag.
cipher_len	u32	Total length of the input cipher buffer in bytes. Must be greater than AES_CH_OVERHEAD.
out	u8 **	Pointer to a buffer pointer that will be allocated by this function to hold the decrypted data.
out_len	u32 *	Pointer to a variable that will store the length of the decrypted data.

Ritorna:
Returns 0 on success. Returns -1 if any parameter is NULL, if the cipher length is invalid, if memory allocation fails, or if the GCM tag verification fails (indicating data tampering or incorrect key).

Note:

- This function is thread-safe as it uses an internal mutex to protect the cryptographic context during the decryption process. It is a blocking function, meaning the calling task will wait if another task is currently using the same channel. It must NOT be called from an Interrupt Service Routine (ISR) because it performs dynamic memory allocation and mutex locking, both of which are forbidden in ISR context. A common mistake is failing to free the memory returned in the 'out' pointer using ca_aes_channel_free (or the appropriate memory deallocator) after processing the decrypted data, which leads to memory leaks.

Uso:

- This function is typically used in secure communication protocols where data is received over a network or serial interface. The standard workflow involves: 1) Receiving the encrypted packet, 2) Calling ca_aes_channel_decrypt to verify the integrity and decrypt the payload, 3) Processing the decrypted data, and 4) Freeing the allocated output buffer. It is usually paired with ca_aes_channel_free to prevent memory exhaustion in long-running applications.

Esempio:

```
u8 *decrypted_data = NULL;
u32 decrypted_len = 0;

// Attempt to decrypt the received buffer
if (ca_aes_channel_decrypt(&my_channel, rx_buffer, rx_len, &decrypted_data, &
    decrypted_len) == 0) {
    // Success: process the data
    process_data(decrypted_data, decrypted_len);

    // Mandatory: free the buffer allocated by the decryption function
    ca_aes_channel_free(decrypted_data);
} else {
    // Failure: log error or handle authentication failure
    ca_stdout_put_n("Decryption failed or data tampered");
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_aes_channel_decrypt in a caOS application?
A: You should use ca_aes_channel_decrypt when you need to decrypt an AES-GCM encrypted message using a pre-initialized AES channel. It is typically used in secure

communication protocols or data protection scenarios where you have a cipher buffer containing nonce, tag, and encrypted data, and you want to obtain the plaintext.

Code example conf:0.90 **Q:** Can you show a minimal example of how to call ca_aes_channel_decrypt?
A: u8 *plaintext = NULL; u32 plain_len = 0; ca_aes_channel_t channel; // Assume channel is initialized u8 cipher_data[100]; // Fill cipher_data with valid AES-GCM format data i32 result = ca_aes_channel_decrypt(&channel, cipher_data, 100, &plaintext, &plain_len); if (result == 0) { // Use plaintext and plain_len // Remember to free plaintext after use }

Parameters conf:0.95 **Q:** What are the valid values for the cipher_len parameter in ca_aes_channel_decrypt?
A: The cipher_len must be greater than 30, which is the AES_CH_OVERHEAD. Values less than or equal to 30 will cause the function to return -1. The cipher buffer must contain at least the nonce, tag, and some encrypted data to be valid for decryption.

Return conf:0.95 **Q:** What does a return value of -1 from ca_aes_channel_decrypt indicate?
A: A return value of -1 indicates an error occurred during decryption. This can be due to invalid input parameters, a buffer that is too short, a mismatched authentication tag, or a failure in memory allocation.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_aes_channel_decrypt?
A: Common mistakes include passing NULL pointers for cipher, out, or out_len, providing a cipher buffer shorter than 30 bytes, not allocating memory for *out after the call, or ignoring the return value of the function. Also, calling this function from an ISR is not allowed.

Behavior conf:0.95 **Q:** Is ca_aes_channel_decrypt thread-safe and can it be called from an ISR?
A: Yes, ca_aes_channel_decrypt is thread-safe because it uses a mutex to protect access to the AES channel. However, it is not ISR-safe and must not be called from interrupt service routines because it uses mutex locking and dynamic memory allocation.

`ca_aes_channel_deinit`

Header: `caos/crypto/inc/aes_channel_service.h:93`
Source: `caos/crypto/src/aes_channel_service.c:278:293`
GUARD: `EN_AES_CHANNEL_SERVICE`

```
1 void ca_aes_channel_deinit(ca_aes_channel_t *ch)
```

Safely releases and clears all memory resources associated with an AES cryptographic channel.

Parametri:

Nome	Tipo	Descrizione
`ch`	`ca_aes_channel_t *`	Pointer to the AES channel structure to be deinitialized. Must not be NULL (though the function handles NULL safely).

Ritorna:
void

Note:

- This function is non-blocking and performs memory cleanup by zeroing out sensitive cryptographic contexts before freeing them. It is not thread-safe; you must ensure no other threads are attempting to use the channel while this function is running, typically by holding a mutex or ensuring the task is shutting down. It is strictly forbidden to call this from an ISR because it performs dynamic memory deallocation, which can lead to heap corruption or system crashes. A common beginner mistake is to continue using the

channel pointer after calling this function; always set your local channel reference to NULL immediately after deinit to prevent use-after-free bugs.

Uso:

- This function is used during the teardown phase of a cryptographic service or when a specific secure session is terminated. It is typically called after the channel has been closed and the associated mutex has been released. It is the final step in the lifecycle of an AES channel, following the sequence: initialization (allocation), usage (encryption/decryption), and finally deinitialization to prevent memory leaks and ensure sensitive key material is wiped from RAM.

Esempio:

```
/* Assuming 'my_channel' was previously initialized and used */

// Ensure no other task is accessing the channel
// ca_mutex_lock(&my_channel->mutex);

// Perform cleanup of the AES channel resources
ca_aes_channel_deinit(my_channel);

// Prevent dangling pointer usage
my_channel = NULL;

// Now it is safe to free the channel structure itself if it was dynamically
    allocated
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_aes_channel_deinit and why is it important?
A: You should call ca_aes_channel_deinit when you are done using an AES channel and want to free the resources allocated for encryption. It is important to call this function to prevent memory leaks and ensure proper cleanup of the AES context.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_aes_channel_deinit with caOS types?
A: ca_aes_channel_t channel; ca_aes_channel_init(&channel); // ... use the channel ... ca_aes_channel_deinit(&channel);

Parameters conf:0.90 **Q:** What is the significance of the ca_aes_channel_t *ch parameter, and what happens if it is NULL?
A: The parameter 'ch' is a pointer to the AES channel to be deinitialized. If it is NULL, the function does nothing, as it checks for a valid pointer before proceeding.

Error/Edge case conf:0.85 **Q:** What are the common mistakes when using ca_aes_channel_deinit?
A: Common mistakes include calling this function on a channel that has already been deinitialized or passing a NULL pointer. The function does not check if the channel is already deinitialized, so calling it twice on the same channel may lead to undefined behavior.

Behavior conf:0.90 **Q:** Is ca_aes_channel_deinit blocking, ISR-safe, or thread-safe?
A: The function is non-blocking. It is not ISR-safe and should not be called from an ISR. It is also not thread-safe unless called within a protected context using a mutex.

`ca_aes_channel_encrypt`

Header: `caos/crypto/inc/aes_channel_service.h:65`
Source: `caos/crypto/src/aes_channel_service.c:85:162`
GUARD: `EN_AES_CHANNEL_SERVICE`

```
i32 ca_aes_channel_encrypt(ca_aes_channel_t *ch, const u8 *plain, u32
    plain_len, u8 **out, u32 *out_len)
```

Encrypts plaintext data using an AES-GCM channel, returning a dynamically allocated buffer containing the nonce, length, ciphertext, and tag.

Parametri:

Nome	Tipo	Descrizione
ch	ca_aes_channel_t *	Pointer to an initialized AES channel structure. Must not be NULL.
plain	const u8 *	Pointer to the raw plaintext data to be encrypted. Must not be NULL.
plain_len	u32	Length of the plaintext in bytes.
out	u8 **	Pointer to a buffer pointer that will be updated to point to the newly allocated encrypted data. Must not be NULL.
out_len	u32 *	Pointer to a variable that will store the total length of the allocated encrypted buffer. Must not be NULL.

Ritorna:
Returns 0 on success. Returns -1 if any input parameter is NULL, if memory allocation fails, if key loading fails, or if the encryption process encounters an error.

Note:

- This function is blocking because it uses a mutex to ensure thread-safe access to the AES hardware context. It must NOT be called from an Interrupt Service Routine (ISR) because mutex operations are not permitted in ISR context and the function performs dynamic memory allocation. The function allocates memory internally for the output buffer; the caller is responsible for freeing this memory using ca_memory_free once the encrypted data has been processed or transmitted. A common mistake is forgetting to free the returned buffer, which will lead to memory leaks in long-running embedded applications.

Uso:

- This function is typically used in secure communication tasks where data needs to be encrypted before being sent over a queue or network interface. The standard workflow involves initializing the channel with ca_mutex_init, calling ca_aes_channel_encrypt to prepare the packet, sending the packet via a queue (e.g., ca_queue_static_send), and finally freeing the allocated buffer. It is designed to handle padding automatically, making it easier for developers to encrypt arbitrary lengths of data without manually managing 16-byte alignment requirements.

Esempio:

```
/* Assuming s_ch is an initialized ca_aes_channel_t */
u8 *encrypted_data = NULL;
u32 encrypted_len = 0;
const char *message = "Secure Payload";

/* Encrypt the message */
if (ca_aes_channel_encrypt(&s_ch, (const u8 *)message, 14, &encrypted_data, &
    encrypted_len) == 0) {
    /* Send the encrypted packet to a queue */
    aes_pkt_t pkt = { .buf = encrypted_data, .len = encrypted_len };
    if (ca_queue_static_send(q_to_bob, &pkt, 2000)) {
        /* Success: buffer is now owned by the queue receiver */
    } else {
        /* Failed to send: free the buffer to prevent memory leak */
```

```
        ca_memory_free(encrypted_data, NULL, true);
    }
} else {
    /* Handle encryption error */
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use the ca_aes_channel_encrypt function in a caOS application?
A: You should use ca_aes_channel_encrypt when you need to encrypt data using AES-GCM in a thread-safe, blocking manner. It is ideal for applications requiring secure communication or data protection where you want automatic memory management and mutex handling. It is not suitable for interrupt service routines (ISR) due to its blocking nature.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_aes_channel_encrypt?
A: ca_aes_channel_t ch; ca_aes_channel_init(&ch); const u8 plaintext[] = "Hello, World!"; u8 *ciphertext; u32 cipher_len; i32 result = ca_aes_channel_encrypt(&ch, plaintext, sizeof(plaintext), &ciphertext, &cipher_len); if (result == 0) { // Use ciphertext and cipher_len ca_free(ciphertext); // Free allocated memory }

Parameters conf:0.90 **Q:** What are the valid and invalid values for the plain parameter in ca_aes_channel_encrypt?
A: The plain parameter must be a valid pointer to a buffer containing the plaintext to encrypt. It cannot be NULL. The plain_len must be greater than 0 and should not cause an overflow in padded_len calculation. If plain is NULL or plain_len is 0, the function will return -1.

Return conf:0.95 **Q:** What does a return value of -1 from ca_aes_channel_encrypt indicate?
A: A return value of -1 indicates that the encryption failed. This can happen due to invalid parameters (e.g., NULL pointers, zero length), memory allocation failure, or a failure in the underlying AES-GCM encryption process. It also occurs if the AES context is not properly initialized.

Error/Edge case conf:0.90 **Q:** What are common error conditions or edge cases when calling ca_aes_channel_encrypt?
A: Common error conditions include passing NULL pointers for plain, out, or out_len, providing a zero or excessively large plain_len that causes overflow, or calling the function before properly initializing the AES channel. Additionally, if the AES context is not initialized, the function will fail and return -1.

Behavior conf:0.95 **Q:** Is ca_aes_channel_encrypt blocking or non-blocking, and is it safe to call from an ISR?
A: ca_aes_channel_encrypt is a blocking function that waits for a mutex lock before proceeding. It is not safe to call from an ISR because it may block and could cause deadlocks or system instability. It is thread-safe due to internal mutex usage, but must be used only from task contexts.

`ca_aes_channel_free`

Header: `caos/crypto/inc/aes_channel_service.h:82`
Source: `caos/crypto/src/aes_channel_service.c:235:238`
GUARD: `EN_AES_CHANNEL_SERVICE`

```
void ca_aes_channel_free(void *p)
```

Deallocates memory previously allocated for an AES channel buffer.

Parametri:

Nome	Tipo	Descrizione
p	void *	A pointer to the memory block to be freed. Must be a pointer previously returned by a caOS memory allocation function. Passing NULL is safe and does nothing.

Ritorna:
void

Note:

- This function is a wrapper around the system memory manager. It is non-blocking and thread-safe, meaning it can be called from different tasks without corrupting the heap. However, it is NOT safe to call from an ISR (Interrupt Service Routine) because heap management operations often require mutexes that cannot be acquired in an interrupt context. The most common beginner mistake is 'double-freeing' memory—calling this function twice on the same pointer—which will lead to heap corruption and likely a system crash or hard fault. Always set your pointer to NULL immediately after calling this function to prevent accidental reuse.

Uso:

- This function is used to clean up dynamic memory buffers that were allocated to hold encrypted or decrypted data packets. It is typically used in error-handling paths where a transmission fails (e.g., if ca_queue_static_send fails, the buffer is no longer needed) or during the teardown phase of a communication session. It is paired with memory allocation functions; once the data has been successfully sent or processed, the buffer must be freed to prevent memory leaks, which would eventually cause the system to run out of RAM.

Esempio:

```
/* Example: Handling a failed transmission */
void *enc_buffer = /* pointer to allocated memory */;
aes_pkt_t pkt = { .buf = enc_buffer, .len = 128 };

/* Attempt to send the packet to a queue */
if (!ca_queue_static_send(q_to_bob, &pkt, 2000)) {
    ca_stdout_put_n("Transmission failed, cleaning up memory");

    /* Free the buffer because the queue failed to take ownership */
    ca_aes_channel_free(enc_buffer);

    /* Prevent dangling pointer usage */
    enc_buffer = NULL;
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_aes_channel_free() and why is it important?
A: You should call ca_aes_channel_free() after completing AES encryption or decryption operations to free the memory allocated for the AES channel. This prevents memory leaks in your embedded application. It's important because failing to free the memory can lead to memory exhaustion, especially in long-running systems with frequent cryptographic operations.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_aes_channel_free() with caOS types?
A: void example_aes_free() { void *channel = ca_memory_alloc(sizeof(u32) * 4); if (channel != NULL) { ca_aes_channel_free(channel); } }

Parameters conf:0.90 **Q:** What does the parameter 'p' in ca_aes_channel_free() represent, and what are valid and invalid values?
A: The parameter 'p' is a void pointer to memory previously allocated with ca_memory_alloc() or a similar allocator. Valid values are pointers to memory allocated by the same memory management system. Invalid values include NULL (which the function handles gracefully by doing nothing) and pointers to memory not allocated with ca_memory_alloc(), which will cause crashes or memory corruption.

`ca_aes_channel_init`

Header: `caos/crypto/inc/aes_channel_service.h:53`
Source: `caos/crypto/src/aes_channel_service.c:52:83`
GUARD: `EN_AES_CHANNEL_SERVICE`

```
i32 ca_aes_channel_init(ca_aes_channel_t *ch, ca_aes_mode_t mode,
    ca_aes_keysize_t keysize, void *ks_handle, ca_aes_iv_gen_cb_t iv_cb,
    ca_aes_key_ratchet_cb_t ratchet_cb, void *cb_user)
```

Initializes an AES cryptographic channel structure by allocating internal contexts and setting up synchronization primitives.

Parametri:

Nome	Tipo	Descrizione
`ch`	`ca_aes_channel_t *`	Pointer to the channel structure to initialize. Must be a valid, non-NULL memory address.
`mode`	`ca_aes_mode_t`	The AES mode of operation (e.g., GCM, CBC).
`keysize`	`ca_aes_keysize_t`	The size of the key (e.g., CA_AES_KEY_128, CA_AES_KEY_256).
`ks_handle`	`void *`	Pointer to the key storage handle or key material. Must not be NULL.
`iv_cb`	`ca_aes_iv_gen_cb_t`	Optional callback for IV generation. Pass NULL if not required.
`ratchet_cb`	`ca_aes_key_ratchet_cb_t`	Optional callback for key ratcheting. Pass NULL if not required.
`cb_user`	`void *`	User-defined pointer passed to callbacks.

Ritorna:
Returns 0 on success. Returns -1 if initialization fails, typically due to NULL pointers or memory allocation failure.

Note:

- This function is non-blocking but performs dynamic memory allocation via ca_memory_allocate, meaning it is NOT safe to call from an ISR. It initializes a mutex for thread safety, so it must be called before any other thread attempts to use the channel. A common beginner mistake is failing to check the return value; if memory allocation fails, the channel remains in an uninitialized state, and subsequent operations will cause a system crash or undefined behavior. Ensure the channel structure is not modified by other threads while this function is running.

Uso:

- This function is the mandatory first step in the AES channel lifecycle. It is typically called during system startup or when a new secure communication session is established. It pairs with subsequent encryption/decryption functions and should eventually be followed by a cleanup function (if provided by the API) to free the allocated contexts. It is often used

in conjunction with keystore services to link a specific key handle to a communication channel.

Esempio:

```
ca_aes_channel_t s_ch;
void *s_key_ptr = get_key_from_storage();

// Initialize the channel before any crypto operations
if (ca_aes_channel_init(&s_ch, CA_AES_MODE_GCM, CA_AES_KEY_256, s_key_ptr, NULL,
    NULL, NULL) != 0) {
    // Handle error: allocation failed or invalid parameters
    return;
}

// Now the channel is ready for use in tasks
// ca_aes_encrypt(&s_ch, ...);
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_aes_channel_init and what is its purpose in a caOS application?
A: You should call ca_aes_channel_init to initialize an AES channel for encryption or decryption operations. It sets up the channel with a specific mode, key size, and optional callbacks for IV generation and key ratcheting. This function is used when you need to perform AES operations in your application, such as securing data transmission or storage.

Code example conf:0.90 **Q:** Can you provide a minimal working C example using ca_aes_channel_init with caOS types?
A: ca_aes_channel_t ch; ca_aes_keysize_t keysize = CA_AES_KEY_256; ca_aes_mode_t mode = CA_AES_MODE_GCM; void *key_handle = (void *)0x12345678; int result = ca_aes_channel_init(&ch, mode, keysize, key_handle, NULL, NULL, NULL); if (result != 0) { // Handle error }

Parameters conf:0.95 **Q:** What are the valid values for the mode and keysize parameters in ca_aes_channel_init?
A: The mode parameter can be one of: CA_AES_MODE_ECB, CA_AES_MODE_CBC, CA_AES_MODE_CTR, CA_AES_MODE_GCM. The keysize parameter can be one of: CA_AES_KEY_128, CA_AES_KEY_192, CA_AES_KEY_256. These values define the encryption mode and key length used for the AES operations.

Return conf:0.95 **Q:** What does a return value of 0 or -1 indicate in ca_aes_channel_init?
A: A return value of 0 indicates successful initialization of the AES channel. A return value of -1 indicates an error, such as invalid parameters, allocation failure, or a NULL pointer being passed where a valid pointer is required.

Error/Edge case conf:0.90 **Q:** What are common error conditions or edge cases when calling ca_aes_channel_init?
A: Common error conditions include passing NULL to the 'ch' or 'ks_handle' parameters, invalid values for mode or keysize, or insufficient memory for internal allocations. Passing NULL to 'ch' will cause an immediate negative return. The function may also fail if memory is exhausted during internal allocation for AES contexts.

Behavior conf:0.85 **Q:** Is ca_aes_channel_init blocking or non-blocking, and is it safe to call from an ISR or in a multi-threaded context?
A: ca_aes_channel_init is a blocking function and cannot be called from an ISR. It is thread-safe if the underlying context allows it, but the function itself is not ISR-safe. It requires valid pointers and allocates memory for two AES contexts, so it must be called in a context where memory allocation is possible.

ca_aes_channel_rotate

Header: caos/crypto/inc/aes_channel_service.h:88
Source: caos/crypto/src/aes_channel_service.c:240:276
GUARD: EN_AES_CHANNEL_SERVICE

```
i32 ca_aes_channel_rotate(ca_aes_channel_t *ch)
```

Updates an AES channel's cryptographic key by executing a ratchet callback and storing the new key securely.

Parametri:

Nome	Tipo	Descrizione
ch	ca_aes_channel_t *	Pointer to the initialized AES channel structure. Must not be NULL and must have a valid ratchet_cb assigned.

Ritorna:
Returns 0 on success. Returns -1 if the channel pointer is invalid, the ratchet callback fails, the key storage access fails, or if the mutex cannot be acquired.

Note:

- This function is blocking because it uses a mutex to ensure thread safety during the key rotation process. It is NOT safe to call from an ISR (Interrupt Service Routine) because blocking operations are forbidden in ISR context. Before calling this, the channel must be fully initialized with a valid ratchet callback function. A common beginner mistake is failing to check the return value; if the function returns -1, the key was not updated, and the system might be using an outdated or compromised key, which could lead to decryption errors in subsequent operations.

Uso:

- This function is used in secure communication protocols where 'Perfect Forward Secrecy' is required, meaning keys must be rotated periodically. It is typically called after a specific number of messages have been encrypted or after a time interval has elapsed. It is usually paired with an initialization function that sets up the channel and the ratchet callback, and eventually with a cleanup function when the channel is no longer needed. The workflow is: Initialize channel -> Perform encryption/decryption -> Periodically call ca_aes_channel_rotate -> Destroy channel.

Esempio:

```
/* Assuming 'my_channel' is an initialized ca_aes_channel_t */

if (ca_aes_channel_rotate(&my_channel) != 0) {
    /* Handle rotation failure: log error or force re-authentication */
    log_error("Failed to rotate AES key, channel might be compromised");
} else {
    /* Rotation successful, proceed with encrypted communication */
    send_secure_data();
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_aes_channel_rotate() in a cryptographic application?
A: You should call ca_aes_channel_rotate() when you need to periodically update the encryption key of an AES channel to maintain security, typically as part of a key rotation strategy. This function is used to generate a new key via a callback and update it in protected memory.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_aes_channel_rotate()?
A: ca_aes_channel_t ch; i32 result = ca_aes_channel_rotate(&ch); if (result != 0) { /* handle error */ }

Parameters conf:0.90 **Q:** What does the ch parameter represent, and what are valid values for it?
A: The ch parameter is a pointer to a ca_aes_channel_t structure that must be valid and not NULL. It must have been initialized with a valid ratchet_cb callback. Passing NULL or an uninitialized channel will result in a failure.

Return conf:0.95 **Q:** What do the return values of ca_aes_channel_rotate() indicate?
A: A return value of 0 indicates success: the key was updated successfully. A return value of -1 indicates failure, which can be caused by a NULL pointer, an unset ratchet_cb, or an error during key loading or updating.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_aes_channel_rotate()?
A: Common mistakes include calling the function without initializing the channel, not setting the ratchet_cb callback, or not handling the negative return value, which can lead to unpredictable behavior.

Behavior conf:0.90 **Q:** Is ca_aes_channel_rotate() thread-safe and can it be called from an ISR?
A: Yes, the function is thread-safe due to internal mutex locking. However, it is not ISR-safe and must not be called from interrupt service routines. It is a blocking function that waits for the mutex before proceeding.

`ca_aes_ctx_set_iv`

Header: `caos/crypto/inc/aes.h:67`
Source: `caos/crypto/src/aes.c:453:462`
GUARD: `EN_AES`

```
1 bool ca_aes_ctx_set_iv(ca_aes_ctx_t *ctx, const u8 *iv)
```

Sets the Initialization Vector (IV) for an AES cryptographic context.

Parametri:

Nome	Tipo	Descrizione
`ctx`	`ca_aes_ctx_t *`	Pointer to the AES context structure. Must be initialized and configured before use.
`iv`	`const u8 *`	Pointer to a 16-byte buffer containing the Initialization Vector. Must not be NULL.

Ritorna:
Returns true if the IV was successfully copied into the context, false if the context is invalid, the IV pointer is NULL, or the context has not been configured.

Note:

- This function is non-blocking and thread-safe provided the context is not being accessed by another task simultaneously. It is safe to call from an ISR, but ensure the context pointer is valid. The most common mistake is passing an uninitialized or unconfigured context, which will cause the function to return false immediately. It does not perform any hardware-specific operations; it merely updates the internal software state of the context structure.

Uso:

- This function is used during the setup phase of an AES operation, typically after the context has been initialized via ca_aes_ctx_setup and before calling encryption or decryption

functions like ca_aes_gcm_encrypt. It is essential for maintaining security in modes like GCM or CBC where a unique IV is required for every operation. In a typical sequence, you would enable access to a key slot, set the IV, perform the cryptographic operation, and then lock the slot again.

Esempio:

```
ca_aes_ctx_t *ctx = (ca_aes_ctx_t *)slot;
u8 iv[16] = {0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08, 0x09, 0x0A, 0x0B, 0
    x0C, 0x0D, 0x0E, 0x0F, 0x10};

/* Ensure the context is ready for use */
if (ca_keys_store_temp_enable_access(slot, CA_KEYS_STORE_READ_WRITE) == 0) {
    if (ca_aes_ctx_set_iv(ctx, iv)) {
        /* Proceed with encryption using the configured context */
        bool success = ca_aes_gcm_encrypt(ctx, NULL, 0, input_data, output_data,
     len, tag);
    }
    ca_keys_store_lock_completely(slot);
    /* Clear sensitive data from stack */
    ca_memzero(iv, sizeof(iv));
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** At what point in the cryptographic workflow should I call ca_aes_ctx_set_iv?
A: You should call this function during the setup phase of an AES operation, specifically after the context has been initialized via ca_aes_ctx_setup and before executing encryption or decryption functions like ca_aes_gcm_encrypt. It is necessary for modes like GCM or CBC that require a unique IV for each operation.

Code example conf:0.90 **Q:** How do I correctly initialize the IV in a caOS AES context?
A: You must provide a 16-byte buffer for the IV. Example: ca_aes_ctx_t ctx; u8 my_iv[16] = {0x01, 0x02, ...}; if (ca_aes_ctx_set_iv(&ctx, my_iv)) { /* Success */ }

Parameters conf:1.00 **Q:** What are the requirements for the iv parameter in ca_aes_ctx_set_iv?
A: The iv parameter must be a pointer to a 16-byte buffer containing the Initialization Vector. It must not be NULL, as passing a NULL pointer will cause the function to return false.

Return conf:1.00 **Q:** Why might ca_aes_ctx_set_iv return false?
A: The function returns false if the provided context is invalid, the IV pointer is NULL, or the context has not been properly configured via ca_aes_ctx_setup prior to the call.

`ca_aes_ctx_setup`

Header: `caos/drivers/inc/aes.h:51`
Source: `caos/drivers/src/aes.c:30:65`
GUARD: `EN_AES`

```
bool ca_aes_ctx_setup(ca_aes_ctx_t *ctx, ca_aes_mode_t mode,
    ca_aes_keysize_t keysize, const u8 *key, const u8 *iv)
```

Initializes an AES context structure with the specified mode, key, and initialization vector (IV).

Parametri:

Nome	Tipo	Descrizione
`ctx`	`ca_aes_ctx_t *`	Pointer to the AES context structure to be initialized. Must not be NULL.
`mode`	`ca_aes_mode_t`	The AES operation mode (e.g., CA_AES_MODE_ECB, CA_AES_MODE_CBC, CA_AES_MODE_CTR, CA_AES_MODE_GCM).
`keysize`	`ca_aes_keysize_t`	The size of the key (CA_AES_KEY_128, CA_AES_KEY_192, or CA_AES_KEY_256).
`key`	`const u8 *`	Pointer to the raw key buffer. Must not be NULL.
`iv`	`const u8 *`	Pointer to the initialization vector buffer. Required for CBC, CTR, and GCM modes; can be NULL for ECB.

Ritorna:
Returns true if the context was successfully configured, false if parameters are invalid (e.g., NULL pointers, missing IV for required modes, or unsupported key size).

Note:

- This function is non-blocking and performs local memory operations to format the key and IV into the context structure. It is thread-safe in the sense that it only modifies the provided local context object, but it does not interact with the hardware AES peripheral directly. It is safe to call from an ISR, provided the context pointer is valid and not shared with other tasks without proper synchronization. The most common beginner mistake is passing a NULL pointer for the IV when using modes like CBC or GCM, which will cause the function to return false immediately.

Uso:

- This function is the mandatory first step before performing any encryption or decryption operations. It is typically used in a sequence: first, you define a local or heap-allocated ca_aes_ctx_t structure, then you call ca_aes_ctx_setup to populate it with your security parameters, and finally, you pass that context to functions like ca_aes_encrypt or ca_aes_decrypt. It is good practice to zero out the key buffer using ca_memzero immediately after calling this function to prevent sensitive data from lingering in memory.

Esempio:

```
ca_aes_ctx_t my_ctx;
u8 key[32] = { /* 32 bytes of secret key */ };
u8 iv[16] = { /* 16 bytes of IV */ };

// Initialize the context for AES-256-CBC
if (ca_aes_ctx_setup(&my_ctx, CA_AES_MODE_CBC, CA_AES_KEY_256, key, iv)) {
    // Context is ready, proceed to encrypt or decrypt
    // ca_aes_encrypt(&my_ctx, input, output, len);
} else {
    // Handle configuration error
}

// Clear sensitive key material from stack
ca_memzero(key, sizeof(key));
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_aes_ctx_setup in a caOS application?
A: You should call ca_aes_ctx_setup before performing any AES encryption or decryption operations to initialize the AES context with the required mode, key size, key, and IV. It is typically called once at the beginning of your application or before starting a new AES operation.

Code example conf:0.95 **Q:** Can you provide a minimal C example using ca_aes_ctx_setup with caOS types?
A: ca_aes_ctx_t ctx; ca_aes_mode_t mode = CA_AES_MODE_CBC; ca_aes_keysize_t keysize = CA_AES_KEY_128; u8 key[16] = {0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F}; u8 iv[16] = {0x10, 0x11, 0x12, 0x13, 0x14, 0x15, 0x16, 0x17, 0x18, 0x19, 0x1A, 0x1B, 0x1C, 0x1D, 0x1E, 0x1F}; bool result = ca_aes_ctx_setup(&ctx, mode, keysize, key, iv);

Parameters conf:0.95 **Q:** What is the meaning of the key parameter in ca_aes_ctx_setup and what are valid values?
A: The key parameter is a pointer to the AES key used for encryption or decryption. It must not be NULL and must contain a valid key of the specified keysize (128, 192, or 256 bits). For example, for CA_AES_KEY_128, the key must be 16 bytes long.

Return conf:0.95 **Q:** What does the return value of ca_aes_ctx_setup indicate and what causes it to return false?
A: The function returns true if the context is successfully initialized. It returns false if any parameter is invalid, such as NULL ctx or key, or if the IV is missing for modes that require it (e.g., CBC, CTR, GCM).

Error/Edge case conf:0.90 **Q:** What are common mistakes or edge cases when using ca_aes_ctx_setup?
A: Common mistakes include passing NULL for ctx or key, not providing an IV for modes that require it (like CBC), or using an invalid keysize or mode. Passing an incorrect key length for the specified keysize also causes failure.

Behavior conf:0.90 **Q:** Is ca_aes_ctx_setup blocking, ISR-safe, or thread-safe?
A: ca_aes_ctx_setup is not blocking but may take some time to copy key and IV data. It is not thread-safe and should be protected by a mutex if used in a multi-threaded environment. It is not recommended to call it from an ISR due to potential non-allowed operations in interrupt contexts.

`ca_aes_decrypt`

Header: `caos/drivers/inc/aes.h:59`
Source: `caos/drivers/src/aes.c:85:101`
GUARD: `EN_AES`

```
1 bool ca_aes_decrypt(ca_aes_ctx_t *ctx, const u8 *in, u8 *out, u32 len)
```

Decrypts a block of data using the hardware AES engine with thread-safe access.

Parametri:

Nome	Tipo	Descrizione
`ctx`	`ca_aes_ctx_t *`	Pointer to the AES context structure. Must be initialized and configured before use; passing NULL or an unconfigured context will cause the function to return false.
`in`	`const u8 *`	Pointer to the encrypted input buffer. Must not be NULL.
`out`	`u8 *`	Pointer to the destination buffer where decrypted data will be stored. Must not be NULL and must have enough allocated space for 'len' bytes.
`len`	`u32`	Length of the data in bytes. Must be greater than 0 and a multiple of 16 (AES block size). If not a multiple of 16, the function returns false.

Ritorna:
Returns true if decryption was successful, false otherwise. A false return indicates invalid parameters, unconfigured context, input length not aligned to 16 bytes, or a hardware setup failure.

Note:

- This function is blocking because it uses a mutex to ensure exclusive access to the hardware AES peripheral. It is NOT safe to call this function from an Interrupt Service Routine (ISR) because the mutex lock operation will cause a system hang or crash. You must ensure the context is properly initialized before calling this function. A common mistake is passing a length that is not a multiple of 16, which will cause the function to fail immediately; always ensure your buffers are padded correctly according to the AES standard.

Uso:

- This function is used when you need to recover original data from an encrypted buffer using the STM32 hardware acceleration. It is typically used in a sequence: first, you initialize the context, then you call this function to perform the decryption, and finally, you verify the output. It is often paired with ca_aes_encrypt to perform round-trip testing or to process incoming secure data packets. Because it uses a mutex, it is designed to be used in multi-threaded environments where multiple tasks might need to access the AES hardware without corrupting each other's operations.

Esempio:

```
ca_aes_ctx_t ctx;
// Assume ctx is already initialized and configured
u8 ciphertext[16] = { /* encrypted data */ };
u8 decrypted[16];

// Attempt to decrypt the 16-byte block
if (ca_aes_decrypt(&ctx, ciphertext, decrypted, 16)) {
    // Decryption successful, proceed with data
} else {
    // Handle error: check if length was multiple of 16 or context was valid
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_aes_decrypt in a caOS application?
A: You should call ca_aes_decrypt when you need to decrypt AES-encrypted data using a pre-configured AES context. It is typically used after initializing the AES hardware and setting up the decryption key and mode via functions like ca_aes_init(). This function is ideal for handling secure data transmission or storage where AES decryption is required.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_aes_decrypt with caOS types?
A: ca_aes_ctx_t ctx; ca_aes_init(&ctx, AES_MODE_DECRYPT, key, 16); const u8 ciphertext[16] = {0x00}; u8 plaintext[16]; bool result = ca_aes_decrypt(&ctx, ciphertext, plaintext, 16); if (result) { // Decryption successful }

Parameters conf:0.95 **Q:** What are the valid and invalid values for the parameters of ca_aes_decrypt?
A: The ctx parameter must be a valid pointer to a pre-configured AES context; NULL causes an error. The in and out parameters must be valid, non-NULL pointers to memory buffers. The len parameter must be a positive multiple of 16 bytes; otherwise, the function returns false. Passing invalid pointers or incorrect lengths will result in failure.

Return conf:0.95 **Q:** What does ca_aes_decrypt return, and what causes it to return false?
A: ca_aes_decrypt returns true if decryption is successful. It returns false if the context is NULL, input or output buffers are NULL, the length is not a multiple of 16, or if hardware configuration fails. It also returns false if the AES hardware is not properly initialized.

Error/Edge case conf:0.90 **Q:** What are common mistakes or edge cases when using ca_aes_decrypt?
A: Common mistakes include passing NULL pointers for ctx, in, or out; providing a length that is not a multiple of 16; or calling the function without initializing the AES context beforehand. Another frequent error is calling ca_aes_decrypt from an ISR, which is not allowed due to its blocking behavior.

Behavior conf:0.95 **Q:** Is ca_aes_decrypt blocking or non-blocking, and is it safe to call from an ISR?
A: ca_aes_decrypt is a blocking function because it waits for a mutex lock before accessing the AES hardware. It is not safe to call from an ISR because it uses mutex operations that are not ISR-safe. It is thread-safe due to the use of aes_mutex, allowing only one thread to access the AES hardware at a time.

`ca_aes_ecb_selftest`

Header: `caos/crypto/inc/aes.h:92`
Source: `caos/crypto/src/aes.c:751:784`
GUARD: `EN_AES`

```
u32 ca_aes_ecb_selftest(void)
```

Performs a FIPS-197 AES-256-ECB known-answer self-test to validate the cryptographic backend.

Ritorna:
Returns a u32 representing the number of failed tests. A return value of 0 indicates that all tests passed successfully.

Note:

- This function is a diagnostic tool used to verify the integrity of the AES implementation. It is not intended for high-frequency use in production loops, as it performs memory-intensive operations and blocking cryptographic calculations. It is thread-safe assuming the underlying AES hardware/software backend is properly initialized. It must NOT be called from an ISR because it performs blocking operations and uses standard output for reporting. The most common mistake is calling this before the system's cryptographic subsystem or memory management has been fully initialized, which will lead to undefined behavior or system crashes.

Uso:

- This function is typically used during the system boot sequence or as part of a diagnostic CLI command to ensure the hardware acceleration or software library is functioning correctly. It is often paired with other self-test functions like ca_aes_gcm_selftest or ca_sha256_selftest to provide a comprehensive health check of the security module. Developers should invoke this during the initialization phase of the application to catch hardware faults or configuration errors before the system starts processing sensitive data.

Esempio:

```c
/* Example: Running the AES self-test during system initialization */
#include "caos.h"

void run_security_diagnostics(void) {
    u32 failures = 0;

    /* Ensure the AES subsystem is ready before testing */
    ca_aes_init();

    /* Execute the ECB self-test */
    failures = ca_aes_ecb_selftest();

    if (failures > 0) {
        /* Log the failure to the system console */
        ca_stdout_printf_n("Security check failed with %u errors", failures);
        /* Halt or enter a safe state if crypto is broken */
        while(1) { ca_os_yield(); }
    } else {
        ca_stdout_put_n("Security subsystem verified.");
    }
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to call ca_aes_ecb_selftest in my caOS application?
A: It is typically used during the system boot sequence or as part of a diagnostic CLI command. It should be invoked during the initialization phase to ensure the cryptographic subsystem is functioning correctly before the system begins processing sensitive data.

Code example conf:1.00 **Q:** How do I implement a basic check using ca_aes_ecb_selftest?
A: u32 result = ca_aes_ecb_selftest(); if (result != 0) { /* Handle failure */ }

Return conf:1.00 **Q:** What does the u32 return value from ca_aes_ecb_selftest signify?
A: The return value represents the number of failed tests. A return value of 0 indicates that all FIPS-197 AES-256-ECB known-answer tests passed successfully.

Error/Edge case conf:1.00 **Q:** What is the most common mistake developers make when calling ca_aes_ecb_selftest?
A: The most common mistake is calling this function before the system's cryptographic subsystem or memory management has been fully initialized, which will lead to undefined behavior or system crashes.

Behavior conf:1.00 **Q:** Can I call ca_aes_ecb_selftest from an ISR or in a high-frequency loop?
A: No. It must not be called from an ISR because it performs blocking operations and uses standard output. It is also not intended for high-frequency use in production loops due to its memory-intensive nature and blocking cryptographic calculations.

`ca_aes_encrypt`

Header: `caos/drivers/inc/aes.h:56`
Source: `caos/drivers/src/aes.c:67:83`

GUARD: `EN_AES`

```
bool ca_aes_encrypt(ca_aes_ctx_t *ctx, const u8 *in, u8 *out, u32 len)
```

Encrypts a block of data using the hardware AES engine with thread-safe access.

Parametri:

Nome	Tipo	Descrizione
`ctx`	`ca_aes_ctx_t *`	Pointer to the AES context structure. Must be initialized and configured before use.
`in`	`const u8 *`	Pointer to the input plaintext buffer. Must not be NULL.
`out`	`u8 *`	Pointer to the output buffer where ciphertext will be stored. Must not be NULL.
`len`	`u32`	Length of the data in bytes. Must be a non-zero multiple of 16 (AES block size).

Ritorna:
Returns true if encryption was successful, false if parameters are invalid, the context is unconfigured, or the hardware processing failed.

Note:

- This function is blocking because it uses a mutex to ensure exclusive access to the hardware AES peripheral. It is thread-safe, meaning multiple tasks can call it, but they will be queued by the mutex. It must NOT be called from an ISR (Interrupt Service Routine) because mutexes cannot be locked in interrupt context. A common mistake is providing a length that is not a multiple of 16, which will cause the function to return false immediately. Ensure the context is fully initialized before calling this, otherwise the 'configured' check will fail.

Uso:

- This function is used when you need to secure data buffers before transmission or storage. It is typically used in a sequence: first, initialize the context with the key, then call ca_aes_encrypt to process the data, and finally use ca_aes_decrypt to verify or retrieve the original data. It is designed to be used in task-level code where you have enough stack space and can afford to wait for the mutex if another task is currently using the AES hardware.

Esempio:

```
ca_aes_ctx_t ctx;
u8 plaintext[16] = "Hello World 1234";
u8 ciphertext[16];

/* Assume context is initialized with a key elsewhere */
if (ca_aes_encrypt(&ctx, plaintext, ciphertext, 16)) {
    /* Encryption successful, proceed to send ciphertext */
} else {
    /* Handle error: check if length is multiple of 16 or context is valid */
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use the ca_aes_encrypt function in a caOS project?
A: You should use ca_aes_encrypt when you need to perform AES encryption on data in a thread-safe manner. It is ideal for securing sensitive data before storage or transmission, especially in embedded systems where hardware AES support is available. The function

ensures exclusive access to the AES hardware through a mutex, making it suitable for multi-threaded environments.

Code example conf:0.90 **Q:** Can you provide a minimal working example of how to use ca_aes_encrypt?
A: ca_aes_ctx_t ctx; bool result; u8 input[16] = {0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F}; u8 output[16]; ctx.configured = true; result = ca_aes_encrypt(&ctx, input, output, 16); // Check result for success or failure

Parameters conf:0.90 **Q:** What are the valid values for the input and output buffers in ca_aes_encrypt?
A: The input buffer (in) and output buffer (out) must be valid pointers to memory locations. They must not be NULL. The input buffer should contain the data to be encrypted, and the output buffer must be large enough to hold the encrypted result. Both buffers must be aligned properly for the hardware AES peripheral to access them.

Return conf:0.95 **Q:** What does it mean when ca_aes_encrypt returns false?
A: A return value of false indicates that the encryption failed. This can happen due to invalid parameters such as a NULL buffer, incorrect length (not a multiple of 16), or an unconfigured context. It may also indicate a hardware setup failure or a failure during the encryption process itself.

Error/Edge case conf:0.90 **Q:** What are common mistakes when calling ca_aes_encrypt?
A: Common mistakes include passing NULL pointers for input or output buffers, providing a length that is not a multiple of 16, using an unconfigured context (ctx->configured = false), or calling the function from an ISR. These errors will cause the function to return false and may lead to undefined behavior or system crashes.

Behavior conf:0.95 **Q:** Is ca_aes_encrypt blocking or non-blocking, and can it be called from an ISR?
A: ca_aes_encrypt is a blocking function. It waits for the AES hardware mutex to be available and does not return until the encryption is complete. It is not safe to call from an ISR because it uses mutexes, which could lead to deadlock situations in interrupt contexts.

`ca_aes_gcm_decrypt`

Header: `caos/drivers/inc/aes.h:71`
Source: `caos/drivers/src/aes.c:119:140`
GUARD: `EN_AES`

```
1 bool ca_aes_gcm_decrypt(ca_aes_ctx_t *ctx, const u8 *aad, u32 aad_len,
    const u8 *in, u8 *out, u32 len, const u8 tag[AES_GCM_TAG_SIZE])
```

Decrypts data using AES-GCM and verifies the integrity tag to ensure the data has not been tampered with.

Parametri:

Nome	Tipo	Descrizione
ctx	ca_aes_ctx_t *	Pointer to the initialized AES context. Must be configured before use.
aad	const u8 *	Pointer to Additional Authenticated Data. Can be NULL if no AAD is used.
aad_len	u32	Length of the AAD buffer in bytes.
in	const u8 *	Pointer to the encrypted input data buffer.
out	u8 *	Pointer to the buffer where decrypted data will be stored.
len	u32	Length of the input/output data in bytes.
tag	const u8[AES_GCM_TAG_SIZE]	The 16-byte authentication tag received with ciphertext to verify integrity.

Ritorna:
Returns true if decryption succeeded and the tag matches (integrity verified). Returns false if the context is invalid, pointers are null, or the tag verification fails.

Note:

- This function is blocking because it uses a mutex to ensure thread-safe access to the hardware AES peripheral. It must NOT be called from an ISR (Interrupt Service Routine) because mutexes cannot be acquired in interrupt context. The context must be initialized and configured prior to calling this function, or it will return false immediately. A common mistake is failing to check the return value; if the tag verification fails, the output buffer will contain untrusted data, so you must never process the 'out' buffer if the function returns false.

Uso:

- This function is typically used in secure communication channels or file storage systems where data integrity is as important as confidentiality. It is usually paired with a preceding 'ca_aes_init' and 'ca_aes_ctx_setup' to prepare the context. In a typical workflow, you receive a packet containing the ciphertext and the tag, then call this function to decrypt and verify. If it returns true, the data is safe to use; if false, the system should discard the data and potentially trigger a security alert.

Esempio:

```
ca_aes_ctx_t ctx;
/* Assume ca_aes_init and ca_aes_ctx_setup have been called successfully */

/* Allocate buffer for decrypted data */
u8 *decrypted_buffer = ca_memory_allocate(32);

if (decrypted_buffer && ca_aes_gcm_decrypt(&ctx, aad_buffer, 16, encrypted_data,
    decrypted_buffer, 32, received_tag)) {
    /* Decryption successful and data is authentic */
    ca_printf_n("Decryption successful");
    ca_memory_free(decrypted_buffer, NULL, true);
} else {
    /* Decryption failed or data was tampered with */
    ca_printf_n("Security alert: Tag mismatch or decryption failure");
    if (decrypted_buffer) {
        ca_memory_free(decrypted_buffer, NULL, true);
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_aes_gcm_decrypt in my embedded application?

A: You should use ca_aes_gcm_decrypt when you need to decrypt data that was encrypted using AES-GCM mode and verify its integrity using the authentication tag. This function is ideal for secure communication protocols or data storage where both confidentiality and authenticity are required.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_aes_gcm_decrypt with caOS types?
A: ca_aes_ctx_t ctx; bool result = ca_aes_gcm_decrypt(&ctx, NULL, 0, ciphertext, plaintext, 16, tag); if (result) { // Decryption successful } else { // Handle decryption failure }

Parameters conf:0.90 **Q:** What are the valid values for the 'aad' parameter in ca_aes_gcm_decrypt?
A: The 'aad' parameter must be a pointer to Additional Authenticated Data. It can be NULL if aad_len is 0. If aad_len > 0, 'aad' must point to valid memory containing the authenticated data. The data must remain valid until the function completes.

Return conf:0.95 **Q:** What does a return value of false from ca_aes_gcm_decrypt indicate?
A: A return value of false indicates that decryption failed. This can happen due to an invalid context, incorrect authentication tag, invalid input/output buffers, incorrect data length (not a multiple of 16), or hardware errors.

Error/Edge case conf:0.85 **Q:** What are common mistakes when calling ca_aes_gcm_decrypt?
A: Common mistakes include passing NULL pointers for input/output buffers when len > 0, providing a data length that is not a multiple of 16, not verifying the return value, or calling the function with an unconfigured AES context (ctx->configured = false).

Behavior conf:0.90 **Q:** Is ca_aes_gcm_decrypt thread-safe and can it be called from an ISR?
A: Yes, ca_aes_gcm_decrypt is thread-safe because it uses a mutex to protect access to the AES hardware. However, it is not ISR-safe and should not be called from an Interrupt Service Routine, as it may block while acquiring the mutex.

`ca_aes_gcm_encrypt`

Header: `caos/drivers/inc/aes.h:66`
Source: `caos/drivers/src/aes.c:105:117`
GUARD: `EN_AES`

```
bool ca_aes_gcm_encrypt(ca_aes_ctx_t *ctx, const u8 *aad, u32 aad_len,
    const u8 *in, u8 *out, u32 len, u8 tag[AES_GCM_TAG_SIZE])
```

Encrypts data using AES-GCM mode and generates an authentication tag.

Parametri:

Nome	Tipo	Descrizione
`ctx`	`ca_aes_ctx_t *`	Pointer to an initialized AES context. Must be configured for GCM mode.
`aad`	`const u8 *`	Pointer to Additional Authenticated Data. Can be NULL if aad_len is 0.
`aad_len`	`u32`	Length of the AAD in bytes.
`in`	`const u8 *`	Pointer to the plaintext buffer. Must be valid if len > 0.
`out`	`u8 *`	Pointer to the output buffer for ciphertext. Must be valid if len > 0.
`len`	`u32`	Length of the plaintext/ciphertext in bytes.
`tag`	`u8[AES_GCM_TAG_SIZE]`	Buffer to store the generated 16-byte authentication tag.

Ritorna:

Returns true if encryption and tag generation succeeded, false otherwise (e.g., invalid context, null pointers, or hardware failure).

Note:

- This function is thread-safe because it uses an internal mutex to protect the hardware AES peripheral. It is a blocking function, meaning the calling task will wait if another task is currently using the AES hardware. You must NEVER call this from an Interrupt Service Routine (ISR) because blocking inside an ISR will cause a system crash or deadlock. Ensure the context is fully initialized via ca_aes_ctx_setup before calling this. A common mistake is passing a NULL pointer for the tag buffer or failing to check the return value, which leads to using uninitialized or corrupted ciphertext.

Uso:

- This function is typically used in secure communication protocols to ensure both data confidentiality and integrity. The standard workflow is: first, initialize the context using ca_aes_ctx_setup with the correct key and nonce; second, call ca_aes_gcm_encrypt to process the data; finally, transmit the ciphertext and the generated tag to the receiver. It is often paired with ca_aes_gcm_decrypt on the receiving end to verify the tag and recover the original plaintext. Always ensure that sensitive buffers are cleared using ca_memzero after the operation is complete to prevent data leakage.

Esempio:

```
ca_aes_ctx_t ctx;
u8 tag[16];
/* Initialize context with GCM mode and key */
if (ca_aes_ctx_setup(&ctx, CA_AES_MODE_GCM, CA_AES_KEY_256, key, nonce)) {
    /* Encrypt plaintext into ciphertext buffer */
    if (ca_aes_gcm_encrypt(&ctx, aad, 16, plaintext, ciphertext, 32, tag)) {
        /* Success: ciphertext and tag are ready for transmission */
    } else {
        /* Handle encryption failure */
    }
    /* Clear sensitive data from memory */
    ca_memzero(key, sizeof(key));
}
```

Q&A — fine-tuning review

5/5 pairs · qwen3-coder:latest

Usage conf:0.95 **Q:** When should I use ca_aes_gcm_encrypt in a caOS project?
A: You should use ca_aes_gcm_encrypt when you need to encrypt data using AES in GCM mode, which provides both confidentiality and authenticity. This is useful in secure communication protocols or when protecting sensitive data where you also need to verify that the data has not been tampered with.

Code example conf:0.90 **Q:** Can you show a minimal example of how to call ca_aes_gcm_encrypt?
A: ca_aes_ctx_t ctx; ca_aes_init(&ctx, CA_AES_KEY_256, (u8*)"example_key_32_bytes_long"); const u8 plaintext[16] = {0x00}; u8 ciphertext[16]; u8 tag[AES_GCM_TAG_SIZE]; bool result = ca_aes_gcm_encrypt(&ctx, NULL, 0, plaintext, ciphertext, 16, tag); // Check result for success or failure

Parameters conf:0.95 **Q:** What are the valid values for the 'len' parameter in ca_aes_gcm_encrypt?
A: The 'len' parameter must be either 0 or a multiple of 16 bytes (128 bits). This is because AES operates on 128-bit blocks, and GCM mode requires data to be processed in these block sizes.

Return conf:0.90 **Q:** What does it mean if ca_aes_gcm_encrypt returns false?
A: A return value of false indicates that the encryption failed due to invalid parameters such as a NULL pointer for ctx, in, out, or tag; or if the input length is not a multiple of 16 bytes. It may also fail if the AES context is not properly initialized.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_aes_gcm_encrypt?
A: Common mistakes include passing NULL pointers for input/output buffers or the tag, failing to ensure the data length is a multiple of 16 bytes, using an uninitialized or improperly configured AES context, and calling this function from an ISR which is not allowed due to mutex acquisition.

`ca_aes_gcm_selftest`

Header: `caos/crypto/inc/aes.h:79`
Source: `caos/crypto/src/aes.c:655:700`
GUARD: `EN_AES`

```
u32 ca_aes_gcm_selftest(void)
```

Performs a self-test of the AES-GCM cryptographic implementation to verify software integrity.

Ritorna:
Returns 0 if all tests pass, or a non-zero integer representing the count of failed test cases if any errors occur.

Note:

- This function is a diagnostic tool used to ensure the cryptographic implementation is functioning correctly. It is not thread-safe and should not be called from an ISR, as it performs multiple operations and potentially heavy calculations that would block the system. A common mistake is to assume it returns a boolean true/false; it actually returns the number of failures, so a return value of 0 is the only indicator of success. Ensure the system has enough stack space before calling this, as it processes several cryptographic vectors.

Uso:

- This function is typically called during the system boot sequence or as part of a diagnostic CLI command to verify the integrity of the crypto library. It is often paired with other self-test functions like ca_sha512_selftest or ca_x25519_selftest to ensure the entire security stack is operational. It is used to validate the environment before the application starts processing real encrypted data, ensuring that the underlying math is correct.

Esempio:

```
/* Run the AES-GCM self-test during system startup */
u32 result;

/* Perform the self-test and check for any failures */
result = ca_aes_gcm_selftest();

if (result == 0) {
    /* All tests passed, proceed with secure operations */
    ca_printf("AES-GCM integrity verified.\n");
} else {
    /* Log the number of failures and halt or enter a safe state */
    ca_printf("AES-GCM self-test failed with %u errors!\n", result);
    /* Perform system reset or enter error loop */
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to execute ca_aes_gcm_selftest in a caOS application?
A: It is typically called during the system boot sequence or as part of a diagnostic CLI command to verify the integrity of the crypto library before the application begins processing real encrypted data.

Code example conf:1.00 **Q:** Provide a minimal C example showing how to check the result of ca_aes_gcm_selftest.
A: u32 result = ca_aes_gcm_selftest(); if (result == 0) { /* Success */ } else { /* Handle result failures */ }

Return conf:1.00 **Q:** How should I interpret the return value of ca_aes_gcm_selftest?
A: The function returns 0 if all tests pass. A non-zero return value represents the count of failed test cases, meaning any value other than 0 indicates an error.

Error/Edge case conf:1.00 **Q:** What is a common mistake developers make when checking the return value of this function?
A: A common mistake is assuming the function returns a boolean true/false. Developers must remember it returns the number of failures, so only a return value of 0 indicates success.

Behavior conf:1.00 **Q:** Can I call ca_aes_gcm_selftest from an ISR or a multi-threaded environment?
A: No. The function is not thread-safe and must not be called from an ISR because it performs heavy calculations that would block the system. Additionally, you must ensure sufficient stack space is available before calling it.

`ca_aes_hw_gcm`

Header: `caos/drivers/inc/ca_glue_hw.h:23`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_aes_hw_gcm(ca_aes_ctx_t *ctx, bool encrypt, const u8 *aad
    , u32 aad_len, const u8 *in, u8 *out, u32 len, u8 *tag); /* cifra/
    decifra in modalit  GCM con autenticazione AAD e tag */
```

Performs hardware-accelerated AES-GCM encryption or decryption with authentication tag generation.

Parametri:

Nome	Tipo	Descrizione
`ctx`	`ca_aes_ctx_t *`	Pointer to the initialized AES context structure containing keys and IV.
`encrypt`	`bool`	True for encryption, false for decryption.
`aad`	`const u8 *`	Pointer to Additional Authenticated Data (AAD). Can be NULL if aad_len is 0.
`aad_len`	`u32`	Length of AAD in bytes.
`in`	`const u8 *`	Pointer to input data buffer. Must be valid if len $>$ 0.
`out`	`u8 *`	Pointer to output data buffer. Must be valid if len $>$ 0.
`len`	`u32`	Length of payload data in bytes.
`tag`	`u8 *`	Pointer to a 16-byte buffer where the authentication tag will be stored.

Ritorna:

Returns true if the operation completed successfully, false if a hardware timeout occurred or parameters were invalid.

Note:

- This function is a blocking call that waits for the hardware peripheral to process data. It is NOT thread-safe on its own; it must be protected by a mutex (as seen in the driver layer) to prevent concurrent access to the shared AES hardware registers. It is NOT callable from an ISR because it relies on busy-waiting loops (cryp_wait_busy) which would hang the system if interrupts are disabled or if the hardware is busy. A common mistake is passing unaligned buffers or invalid pointers, which will cause the hardware to hang or the function to return false due to timeout.

Uso:

- This function is used in the high-level AES driver layer to perform authenticated encryption/decryption. It is typically called after the context has been initialized with a key and IV. The standard sequence is: 1) Lock the AES mutex, 2) Call ca_aes_hw_gcm, 3) Unlock the mutex. It is used when data integrity and confidentiality are both required, such as in secure network protocols or encrypted storage.

Esempio:

```
ca_aes_ctx_t my_ctx; /* Assume initialized with key/IV */
u8 tag[16];
u8 input[] = "SecretData";
u8 output[16];

/* Always protect hardware access with a mutex */
ca_mutex_lock(&aes_mutex);

/* Perform GCM encryption */
bool success = ca_aes_hw_gcm(&my_ctx, true, NULL, 0, input, output, sizeof(input), tag);

ca_mutex_unlock(&aes_mutex);

if (!success) {
    /* Handle hardware error or timeout */
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.90 **Q:** When should I use ca_aes_hw_gcm() function in my caOS project?
A: You should use ca_aes_hw_gcm() when you need to perform AES encryption or decryption in GCM mode with AAD authentication and tag generation using hardware AES acceleration. However, note that all hardware implementations currently return false, making this function non-functional in real deployments.

Code example conf:0.80 **Q:** Can you provide a minimal C code example showing how to call ca_aes_hw_gcm()?
A:

```
ca_aes_ctx_t ctx;
bool result = ca_aes_hw_gcm(&ctx, true, NULL, 0, input_data, output_data, 16, tag);
```

Parameters conf:0.90 **Q:** What are the valid values for the 'encrypt' parameter in ca_aes_hw_gcm()?
A: The 'encrypt' parameter must be either true (to perform encryption) or false (to perform decryption). Any other value is invalid and will lead to undefined behavior.

Return conf:0.95 **Q:** What does it mean when ca_aes_hw_gcm() returns false?

A: A return value of false indicates that the operation failed. This can happen due to invalid parameters, an improperly configured context, or hardware errors. However, since all hardware implementations return false, this function will always fail in practice.

HW differences conf:0.90 **Q:** Are there any differences in how ca_aes_hw_gcm() is implemented across different STM32 families?

A: No, there are no functional differences in the implementation of ca_aes_hw_gcm() across STM32 families (f1xx, f4xx, f7xx, h5xx, h7xx). All implementations simply return false without performing any actual hardware operations, indicating that the function is not functional on any supported hardware.

`ca_aes_hw_init`

Header: `caos/drivers/inc/ca_glue_hw.h:20`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_aes_hw_init(void); /* initializes the hardware AES/CRYP
   peripheral */
```

Initializes the hardware AES cryptographic peripheral by enabling its clock in the RCC register.

Ritorna:
Returns true if the hardware peripheral was successfully enabled, or false if the hardware is unsupported, disabled in configuration, or unavailable on the specific MCU model.

Note:

- This function is non-blocking and performs a direct register write to the RCC (Reset and Clock Control) unit. It is thread-safe in the sense that it only modifies a peripheral clock register, but it is not intended to be called repeatedly; it should be executed once during system startup. A common beginner mistake is calling this function multiple times or assuming it configures the AES engine's internal registers (like keys or IVs), which it does not. It must be called before any other AES operations are attempted, as accessing the AES peripheral registers without enabling the clock will cause a hard fault or bus error. It is safe to call from an ISR, though typically unnecessary as it is part of the system boot sequence.

Uso:

- This function is typically called during the system initialization phase, specifically within the higher-level ca_aes_init() function. It acts as the hardware-level prerequisite for the AES driver. You should pair this with other initialization routines to ensure that subsequent cryptographic operations are thread-safe when accessed by multiple tasks. Once initialized, the driver is ready to accept context setups via ca_aes_ctx_setup() and perform encryption or decryption tasks.

Esempio:

```
/* Initialize the AES driver during system startup */
if (ca_aes_init()) {
    /* AES hardware is ready for use */
} else {
    /* Handle error: AES hardware not supported or failed to initialize */
    while(1);
}

/* Later, in a task, setup the context for encryption */
/* Assuming my_ctx is a valid pointer to a structure defined by the driver */
```

```c
if (ca_aes_ctx_setup(my_ctx, 0, 0, key_buffer, iv_buffer)) {
    /* Context is ready, proceed with encryption */
} else {
    /* Handle configuration error */
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should ca_aes_hw_init() be called, and why is it important to call it only once?
A: ca_aes_hw_init() should be called once at system startup before any AES encryption or decryption operations are performed. It initializes the AES hardware peripheral and must be called only once because it is not designed to be re-initialized. Calling it multiple times could lead to undefined behavior or crashes.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_aes_hw_init() with caOS types?
A: bool result = ca_aes_hw_init(); if (result) { // AES hardware initialized successfully // Proceed with AES operations } else { // Handle initialization failure // AES hardware may be disabled or not functional }

Return conf:0.95 **Q:** What does it mean when ca_aes_hw_init() returns false, and what causes this failure?
A: ca_aes_hw_init() returns false when the AES hardware initialization fails. This can happen if the AES module is disabled in the firmware (EN_AES=0) or if the hardware implementation is not functional. According to the documentation, all current implementations return false, indicating that the function is not currently functional.

HW differences conf:0.85 **Q:** How do the AES hardware implementations differ across STM32 families like F1, F4, F7, H5, and H7?
A: All STM32 families (F1, F4, F7, H5, H7) currently implement ca_aes_hw_init() to return false, meaning none of them have functional AES hardware initialization. The F7 and H7 families have specific notes: F7xx has no CRYP peripheral, and H747 specifically lacks CRYP support. The F1, F4, and H5 families are noted to have AES hardware but are not functional in the current implementation.

`ca_aes_hw_process`

Header: `caos/drivers/inc/ca_glue_hw.h:22`
GUARD: `EN_CA_GLUE_HW`

```c
extern bool ca_aes_hw_process(const u8 *in, u8 *out, u32 len); /* cifra/
   decifra un blocco con la configurazione corrente */
```

Processes a buffer of data through the hardware AES engine using the currently configured key and mode.

Parametri:

Nome	Tipo	Descrizione
`in`	`const u8 *`	Pointer to the input data buffer. Must be aligned to 4-byte boundaries for optimal performance.
`out`	`u8 *`	Pointer to the output buffer where the encrypted/decrypted data will be stored. Must be at least 'len' bytes long.
`len`	`u32`	Length of the data in bytes. Must be a multiple of 16 (AES block size). If not a multiple, the function will fail to process the final partial block.

Ritorna:
Returns true if the entire buffer was processed successfully, or false if a hardware timeout occurred or the input parameters were invalid.

Note:

- This function is a blocking call that waits for the hardware FIFO to become ready for each 128-bit block. It is not thread-safe on its own; it relies on an external mutex (as seen in the usage context) to prevent multiple tasks from accessing the hardware simultaneously. It is not safe to call from an ISR because it uses a polling loop with a timeout, which could cause significant latency or system hangs if the hardware is busy. A common beginner mistake is passing a length that is not a multiple of 16, which will cause the function to process only complete blocks and potentially leave the output buffer partially unwritten or corrupted.

Uso:

- This function is typically used within a higher-level driver wrapper that manages resource locking. The standard sequence is: first, initialize the hardware with 'ca_aes_hw_init', then configure the key and mode using 'ca_aes_hw_setup', acquire a mutex to ensure exclusive access, call 'ca_aes_hw_process' to perform the actual transformation, and finally release the mutex. It is designed to be called repeatedly for large data streams, provided the mutex is held throughout the entire operation to maintain state consistency.

Esempio:

```
/* Assuming aes_mutex is initialized and hardware is configured */
if (ca_mutex_lock(&aes_mutex, 100)) {
    /* Process 32 bytes (2 blocks of 16) */
    if (ca_aes_hw_process(input_buffer, output_buffer, 32)) {
        /* Success: data is now encrypted/decrypted */
    } else {
        /* Handle hardware error or timeout */
    }
    ca_mutex_unlock(&aes_mutex);
} else {
    /* Could not acquire mutex, hardware busy */
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_aes_hw_process in my embedded application?
A: You should call ca_aes_hw_process when you need to encrypt or decrypt a block of data using the hardware AES peripheral. It is typically used in contexts where high-performance cryptographic operations are required, and it must be called within a mutex-protected section to ensure thread safety.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_aes_hw_process with caOS types?
A:

```
u8 input[16] = {0x00};
u8 output[16];
bool result = ca_aes_hw_process(input, output, 16);
```

Parameters conf:0.90 **Q:** What are the valid values for the length parameter in ca_aes_hw_process?
A: The length parameter must be a multiple of 16 bytes, which corresponds to one AES block. Passing zero or odd-length values can cause errors and undefined behavior. The input and output buffers must also be valid and non-NULL.

Return conf:0.95 **Q:** What does a return value of false from ca_aes_hw_process indicate?
A: A return value of false indicates that the operation failed. This can be caused by invalid parameters such as NULL buffers, incorrect length (not a multiple of 16), or an uninitialized or misconfigured hardware AES peripheral.

HW differences conf:0.80 **Q:** Are there any differences in how ca_aes_hw_process is implemented across different STM32 families?
A: Based on the provided documentation, all hardware implementations (f1xx, f4xx, f7xx, h5xx, h7xx) currently return false for all functions including ca_aes_hw_process, indicating that the actual hardware support is not implemented in these stubs. There are no specific register or peripheral differences mentioned in the stubs.

`ca_aes_hw_setup`

Header: `caos/drivers/inc/ca_glue_hw.h:21`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_aes_hw_setup(ca_aes_ctx_t *ctx, bool encrypt); /* carica
    chiave e IV nel periferico per ECB/CBC */
```

Configures the hardware AES peripheral with the provided key, IV, and operational mode.

Parametri:

Nome	Tipo	Descrizione
`ctx`	`ca_aes_ctx_t *`	Pointer to the AES context structure containing the key, IV, and mode settings. Must be initialized before calling this function.
`encrypt`	`bool`	Set to true for encryption, false for decryption. Note that decryption may trigger additional key preparation steps.

Ritorna:
Returns true if the hardware was successfully configured. Returns false if the mode is unsupported (e.g., GCM), if key preparation fails, or if the hardware is not available on the current platform.

Note:

- This function is non-blocking but interacts directly with hardware registers, so it is not thread-safe on its own; it must be protected by a mutex if multiple tasks access the AES peripheral. It is generally not safe to call from an ISR because it may perform complex operations like key preparation. A common beginner mistake is failing to check the return value, which leads to processing data with uninitialized or incorrect hardware settings. Ensure the AES peripheral is powered and initialized before calling this function, as it assumes the hardware is ready to accept configuration.

Uso:

- This function is the mandatory setup step before any data processing occurs. It is typically used within a wrapper function that handles mutual exclusion, ensuring that only one task configures and uses the AES engine at a time. The standard sequence is: lock the mutex, call ca_aes_hw_setup, perform the data processing using ca_aes_hw_process, and finally unlock the mutex. It is never used in isolation; it must always be paired with a processing function to actually perform the cryptographic operation.

Esempio:

```
ca_aes_ctx_t my_aes_ctx;
// ... initialize my_aes_ctx with key and IV ...

ca_mutex_lock(&aes_mutex);

// Configure hardware for encryption
if (ca_aes_hw_setup(&my_aes_ctx, true)) {
    // Proceed to process data if setup succeeded
    bool success = ca_aes_hw_process(input_buffer, output_buffer, data_len);
    if (!success) {
        // Handle processing error
    }
} else {
    // Handle setup failure (e.g., unsupported mode)
}

ca_mutex_unlock(&aes_mutex);
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_aes_hw_setup() in my AES encryption/decryption workflow?
A: You should call ca_aes_hw_setup() after initializing the AES context with the key and IV, but before starting actual encryption or decryption operations. It prepares the hardware AES peripheral with the key and IV for ECB/CBC modes. This function is typically called once per encryption/decryption session before processing data.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_aes_hw_setup() with caOS types?
A: ca_aes_ctx_t ctx; ctx.key[0] = 0x12345678; ctx.iv[0] = 0x87654321; bool success = ca_aes_hw_setup(&ctx, true); /* encrypt mode */ if (!success) { /* handle error */ }

Parameters conf:0.95 **Q:** What does the 'encrypt' parameter in ca_aes_hw_setup() control, and what are valid values?
A: The 'encrypt' parameter controls the operation mode: true enables encryption, false enables decryption. Valid values are only true or false. Any other value will result in undefined behavior. The parameter determines whether the hardware AES peripheral will perform encryption or decryption using the loaded key and IV.

Return conf:0.90 **Q:** What does ca_aes_hw_setup() return, and what causes it to return false?
A: It returns true on successful loading of key and IV into the hardware AES peripheral, or false on failure. Common causes of failure include: hardware not available, invalid context pointer, or hardware errors. The function is also not thread-safe and requires external mutex protection.

HW differences conf:0.70 **Q:** How does ca_aes_hw_setup() differ between STM32F1xx and STM32H7xx families in terms of hardware implementation?
A: The implementation is stubbed in all families (F1xx, F4xx, F7xx, H5xx, H7xx) and returns false. There are no functional differences in the API itself, but the underlying hardware peripherals (AES registers, clocking, and DMA support) vary between families.

The F1xx family lacks hardware AES support, while H7xx has full hardware acceleration with advanced features.

---

ca_aes_init

Header: caos/drivers/inc/aes.h:48
Source: caos/drivers/src/aes.c:24:28
GUARD: EN_AES

```
bool ca_aes_init(void)
```

Initializes the AES hardware peripheral and the internal mutex required for thread-safe cryptographic operations.

Ritorna:
Returns true if both the mutex and the AES hardware were initialized successfully; returns false if the hardware initialization failed.

Note:

- This function is not thread-safe and must be called exactly once during the system startup phase, typically before any tasks are created or before any other AES functions are invoked. It is strictly non-blocking and must NOT be called from an Interrupt Service Routine (ISR) because it initializes a mutex, which is a kernel object that requires a running scheduler context. A common mistake is calling this function multiple times or from different tasks; doing so will re-initialize the mutex, potentially causing race conditions or memory corruption if other tasks are already waiting on that mutex.

Uso:

- This function is the mandatory entry point for any application requiring AES encryption or decryption. It is typically called during the system initialization sequence, alongside other hardware drivers like RNG, HASH, or Flash. It must be called before any calls to ca_aes_ctx_setup or actual encryption/decryption functions, as those functions rely on the mutex initialized here to prevent concurrent access to the shared AES hardware peripheral.

Esempio:

```
/* System startup sequence */
void system_init(void) {
    /* Initialize the AES hardware and its internal mutex */
    if (!ca_aes_init()) {
        /* Handle failure: log error or enter safe state */
        CA_SAFE_FAIL("AES init failed");
    }

    /* Now it is safe to use other AES functions */
    ca_aes_ctx_t my_ctx;
    ca_aes_ctx_setup(&my_ctx, CA_AES_MODE_CBC, AES_KEYSIZE_128);
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should ca_aes_init() be called in a caOS application, and why is it important to call it only once?
A: ca_aes_init() should be called once at system startup before any other AES functions are used. It initializes the AES hardware module and creates an internal mutex for thread safety. Calling it more than once or forgetting to initialize it before using other AES

functions will lead to errors. It is a blocking function and must not be called from an ISR.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_aes_init() with caOS types?
A: bool result = ca_aes_init(); if (result) { // AES module initialized successfully } else { // Handle initialization error }

Return conf:0.90 **Q:** What do the return values of ca_aes_init() indicate, and what causes initialization to fail?
A: ca_aes_init() returns true if the AES hardware module was successfully initialized and the mutex was created. It returns false if there was an error during hardware initialization. The most common cause of failure is calling this function more than once or not initializing the mutex before using other AES functions.

`ca_arp_avail`

Header: `caos/net/inc/arp_service.h:74`
Source: `caos/net/src/arp_service.c:231:234`
GUARD: `EN_ARP_SERVICE`

```
bool ca_arp_avail(ca_ipv4_t ip)
```

Checks if a specific IPv4 address has a corresponding MAC address already resolved in the ARP cache.

Parametri:

Nome	Tipo	Descrizione
`ip`	`ca_ipv4_t`	The IPv4 address to search for in the ARP table. Must be a valid 32-bit representation of an IP address.

Ritorna:
Returns true if the IP address exists in the ARP cache and is associated with a valid MAC address; returns false if the IP is not found or the entry has expired.

Note:

- This function is non-blocking and performs a simple lookup in the internal ARP cache memory. It is thread-safe, meaning it can be called from multiple tasks simultaneously without corrupting the cache data. It is safe to call from an ISR (Interrupt Service Routine) because it does not perform any complex operations or wait for hardware responses. The most common mistake is assuming that a 'false' return means the device is offline; it simply means the ARP entry is not currently cached, so you may need to trigger a network request to resolve it.

Uso:

- This function is typically used as a pre-check before attempting to send data to a specific network node to avoid unnecessary ARP resolution delays. It is often paired with ca_arp_find() when you need to retrieve the actual MAC address after confirming its availability. Developers use this in the application layer to verify if a target device is 'known' to the stack before initiating a communication sequence, ensuring that the network stack is ready to transmit packets immediately.

Esempio:

```
ca_ipv4_t target_ip = {192, 168, 1, 50};

```

```c
// Check if we already have the MAC address for this IP
if (ca_arp_avail(target_ip)) {
    // We have the mapping, proceed with sending data
    // ca_mac_t mac;
    // ca_arp_find(target_ip, &mac);
} else {
    // Entry not in cache, we might need to trigger an ARP request
    // or wait for the stack to resolve it automatically.
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_arp_avail() in my network application?
A: You should call ca_arp_avail() when you need to check if an IPv4 address is already resolved in the ARP cache before attempting to send data to that address. This helps avoid unnecessary ARP requests and improves network efficiency. It's typically used before sending packets to ensure the MAC address is known.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_arp_avail() with caOS types?
A: ca_ipv4_t target_ip = {192, 168, 1, 10}; bool is_resolved = ca_arp_avail(target_ip); if (is_resolved) { // Proceed with sending data } else { // Handle unresolved IP }

Parameters conf:0.90 **Q:** What does the ca_ipv4_t parameter represent in ca_arp_avail() and what are valid values?
A: The ca_ipv4_t parameter represents the IPv4 address to check in the ARP cache. It must be a valid IPv4 address structure with four octets (u8[4]) representing the address. Invalid values include NULL or malformed addresses, which lead to undefined behavior.

Return conf:0.95 **Q:** What do the return values of ca_arp_avail() indicate and what causes it to return false?
A: ca_arp_avail() returns true if the IP address is present in the ARP cache and can be used for communication. It returns false if the IP address is not in the cache, meaning it has not yet been resolved to a MAC address. A return value of false does not indicate an error, but rather that the address is not yet resolved.

`ca_arp_del`

Header: `caos/net/inc/arp_service.h:117`
Source: `caos/net/src/arp_service.c:412:424`
GUARD: `EN_ARP_SERVICE`

```c
bool ca_arp_del(u8 idx)
```

Removes a specific entry from the ARP cache by its index.

Parametri:

Nome	Tipo	Descrizione
`idx`	`u8`	The zero-based index of the active ARP entry to remove. Valid values range from 0 to (ARP_CACHE_SIZE - 1). Providing an index higher than the number of currently active entries will result in a failure.

Ritorna:
Returns true if the entry at the specified index was found and successfully invalidated. Returns false if the index is out of range or if no active entry exists at that position.

Note:

- This function is thread-safe because it uses a mutex to protect the shared ARP cache array from concurrent access. It is a blocking function; if another thread currently holds the mutex, the calling thread will be suspended until the mutex is released. You must NEVER call this function from an Interrupt Service Routine (ISR) because blocking operations are strictly forbidden in ISR context and will cause system instability or hard faults. A common beginner mistake is assuming the index refers to a static memory address rather than the dynamic count of currently active entries.

Uso:

- This function is typically used in command-line interfaces (CLI) or network management tasks where a user or system process needs to clear stale or incorrect ARP mappings. It is usually paired with a display function that lists current entries so the user knows which index to target. The typical sequence is: first, call a function to list or iterate through the cache to identify the target index, then call ca_arp_del(idx) to remove it, and finally verify the result via the return boolean to provide feedback to the user.

Esempio:

```c
/* Example: Deleting an ARP entry based on user input from a CLI */
void handle_arp_delete_command(char *input_idx) {
    u8 idx = (u8)ca_strtol(input_idx, 10);

    /* Attempt to delete the entry at the provided index */
    if (ca_arp_del(idx)) {
        /* Success: entry was found and marked as invalid */
        ca_printf("Successfully deleted ARP entry at index %u\n", idx);
    } else {
        /* Failure: index was invalid or no entry existed at that position */
        ca_printf("Error: Could not delete ARP entry at index %u\n", idx);
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_arp_del() in a network application?
A: You should call ca_arp_del() when you need to remove a specific entry from the ARP cache, for example, when a device is no longer reachable or when you want to force a refresh of the ARP table. This function is typically used in network services that manage dynamic ARP entries.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_arp_del()?
A: u8 index = 5; bool result = ca_arp_del(index); if (result) { // Entry was successfully deleted } else { // Entry was not found or invalid index }

Parameters conf:0.95 **Q:** What is the significance of the idx parameter in ca_arp_del()?
A: The idx parameter specifies the index of the ARP cache entry to delete. It must be between 0 and ARP_CACHE_SIZE-1. Values outside this range can cause undefined behavior, such as memory access violations, because the function does not validate the input.

Return conf:0.90 **Q:** What does ca_arp_del() return when it fails to delete an ARP entry?
A: The function returns false if no valid entry corresponds to the given index idx. This typically happens when the index is out of bounds or the entry at that index is not currently in use. It returns true only if a valid entry was found and successfully deleted.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_arp_del()?
A: Common mistakes include passing an index outside the valid range [0, ARP_CACHE_SIZE-1], which can lead to memory corruption; not checking the return value, which may cause silent failures; and calling the function from an ISR, which is unsafe due to mutex acquisition and potential deadlocks.

ca_arp_del_ip

Header: caos/net/inc/arp_service.h:123
Source: caos/net/src/arp_service.c:196:209
GUARD: EN_ARP_SERVICE

```
bool ca_arp_del_ip(ca_ipv4_t ip)
```

Removes a specific IPv4 address entry from the internal ARP cache table.

Parametri:

Nome	Tipo	Descrizione
ip	ca_ipv4_t	The IPv4 address structure to search for and remove. Ensure the structure is correctly populated before calling.

Ritorna:
Returns true if the IP address was found and successfully removed from the cache, or false if the IP address did not exist in the cache.

Note:

- This function is thread-safe because it uses a mutex (s_cache_mtx) to protect the shared ARP cache array from concurrent access. It is a blocking function; if another thread currently holds the mutex, the calling thread will wait until it is released. You must NOT call this function from an Interrupt Service Routine (ISR) because blocking operations are strictly forbidden in ISRs and will cause system instability or deadlocks. A common mistake is assuming the cache is updated instantly across all network tasks; always check the return value to confirm if the entry was actually present before assuming it was deleted.

Uso:

- This function is typically used in network management interfaces, such as a Command Line Interface (CLI), to manually clear stale or incorrect ARP entries. It is usually paired with parsing functions like ca_parse_ipv4 to convert user input strings into the required ca_ipv4_t format. In a typical lifecycle, you would use this when a network administrator needs to force a re-discovery of a device's MAC address by clearing its existing ARP mapping, ensuring the system does not use outdated cached information.

Esempio:

```c
ca_ipv4_t target_ip;
// Parse a string input into the ca_ipv4_t structure
if (ca_parse_ipv4("192.168.1.50", &target_ip)) {
    // Attempt to remove the entry from the ARP cache
    if (ca_arp_del_ip(target_ip)) {
        // Successfully removed
        CLI_PRINTF_N("ARP entry deleted successfully.");
    } else {
        // Entry not found in cache
        CLI_PRINTF_N("ARP entry not found.");
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_arp_del_ip() in a network application?
A: You should call ca_arp_del_ip() when you need to remove a specific IPv4 address from the ARP cache, typically when the corresponding device is no longer reachable or has changed its network configuration. This helps maintain an accurate ARP table and prevents stale entries from causing communication issues.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_arp_del_ip() with caOS types?
A: ca_ipv4_t target_ip = {192, 168, 1, 10}; bool result = ca_arp_del_ip(target_ip); if (result) { // Entry was found and removed } else { // Entry not found in ARP cache }

Parameters conf:0.90 **Q:** What is the significance of the ca_ipv4_t parameter in ca_arp_del_ip() and what values are valid?
A: The ca_ipv4_t parameter represents the IPv4 address to be removed from the ARP cache. It must be a valid IPv4 address; otherwise, the behavior is undefined. Valid values are any 4-byte array representing a valid IPv4 address such as {192, 168, 1, 1}. Invalid values include malformed or out-of-range addresses.

Return conf:0.95 **Q:** What does it mean when ca_arp_del_ip() returns false?
A: When ca_arp_del_ip() returns false, it means that no entry matching the provided IPv4 address was found in the ARP cache. This does not indicate an error in execution, but rather that the specified IP address was not present in the cache.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_arp_del_ip()?
A: Common mistakes include calling the function with an uninitialized or invalid IPv4 address, which can lead to undefined behavior. Additionally, calling this function before the ARP cache has been initialized will result in accessing unallocated memory. It's also not recommended to call it from an ISR due to potential mutex blocking and scheduling issues.

`ca_arp_dump`

Header: `caos/net/inc/arp_service.h:106`
Source: `caos/net/src/arp_service.c:372:403`
GUARD: `EN_ARP_SERVICE`

```
1 void ca_arp_dump(void)
```

Prints the current contents of the ARP cache to the system CLI for debugging purposes.

Ritorna:
void

Note:

- This function is a blocking operation because it acquires a mutex (s_cache_mtx) to ensure data consistency while reading the cache. It is NOT safe to call from an Interrupt Service Routine (ISR) because mutexes are designed for thread synchronization and can cause the system to hang or trigger an assertion if locked in an ISR context. The function creates a local snapshot of the cache to minimize the time the mutex is held, which is a good practice to prevent blocking other network tasks for too long. A common beginner mistake is calling this function inside a high-frequency loop or an ISR, which will lead to system instability or deadlocks.

Uso:

- This function is intended for diagnostic use, typically triggered by a user command via the Command Line Interface (CLI). It is usually paired with other maintenance functions like ca_arp_flush() or specific entry deletion functions. Developers use this to verify that the network stack is correctly resolving IP addresses to MAC addresses and to check if entries are expiring as expected. It should be invoked only when the user explicitly requests status information, as it performs significant string formatting and output operations that consume CPU cycles.

Esempio:

```c
/* Example: CLI command handler to display ARP table */
void handle_cli_command(char *cmd) {
    if (ca_strncmp(cmd, "arp info", 8) == 0) {
        /* Safely print the ARP cache to the console */
        ca_arp_dump();
    } else if (ca_strncmp(cmd, "arp flush", 9) == 0) {
        /* Clear the cache if requested */
        ca_arp_flush();
        CLI_PUT_N("Cache cleared.");
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_arp_dump() in my caOS network application?
A: You should call ca_arp_dump() primarily for debugging and monitoring purposes to inspect the contents of the ARP cache. It is useful when you need to verify ARP table entries or diagnose network connectivity issues. The function is intended for use in non-critical sections of code where blocking is acceptable.

Code example conf:0.90 **Q:** Can you provide a minimal example of how to use ca_arp_dump() in a caOS application?
A: void example_function(void) { ca_arp_dump(); }

Error/Edge case conf:0.85 **Q:** What are the potential issues or edge cases when using ca_arp_dump()?
A: The main edge cases include calling ca_arp_dump() from an ISR, which is not thread-safe and may cause issues. Additionally, if CLI_PUT_N and CLI_PRINTF_N are not implemented or available, the function will not produce any output. If the ARP cache is empty, it will print a specific message indicating this. The function also blocks while acquiring a mutex, so it should not be used in time-critical sections.

Behavior conf:0.90 **Q:** Is ca_arp_dump() blocking or non-blocking, and is it safe to call from an ISR?
A: ca_arp_dump() is a blocking function because it acquires a mutex before reading the ARP cache. It is not safe to call from an ISR as it is not thread-safe in that context. The function must be used in contexts where waiting is acceptable, such as in task-level code, not in interrupt handlers.

`ca_arp_find`

Header: `caos/net/inc/arp_service.h:79`
Source: `caos/net/src/arp_service.c:236:249`
GUARD: `EN_ARP_SERVICE`

```c
bool ca_arp_find(ca_ipv4_t ip, ca_mac_t *mac_out)
```

Searches the local ARP cache for a MAC address associated with a specific IPv4 address.

Parametri:

Nome	Tipo	Descrizione
`ip`	`ca_ipv4_t`	The target IPv4 address to look up in the cache. Must be a valid ca_ipv4_t structure.
`mac_out`	`ca_mac_t *`	A pointer to a ca_mac_t structure where the result will be stored if found. Can be NULL if you only want to check existence.

Ritorna:

Returns true if the IP address exists in the cache and the MAC was successfully copied to mac_out. Returns false if the IP is not found in the cache or if the entry is marked as invalid.

Note:

- This function is blocking because it uses a mutex (ca_mutex_lock) to ensure thread safety while reading the shared ARP cache table. Because it uses a mutex, it must NEVER be called from an Interrupt Service Routine (ISR), as doing so will cause a system deadlock or kernel panic. Before calling this, ensure the network stack and ARP service have been initialized, otherwise the cache will be empty. A common beginner mistake is assuming this function will trigger a network request; it does not. It only checks the local memory table, so if it returns false, you must manually trigger a resolution process like ca_arp_resolve.

Uso:

- This function is typically used as the first step in a network communication sequence to avoid unnecessary ARP broadcast traffic. You call it before sending a packet to check if the destination hardware address is already known. If it returns true, you proceed with the transmission using the retrieved MAC. If it returns false, you typically pair it with ca_arp_resolve to perform a network-level lookup, wait for the response, and then proceed with your communication.

Esempio:

```
ca_ipv4_t target_ip = {{192, 168, 1, 10}}; ca_mac_t target_mac;

// Check if we already know the MAC address
if (ca_arp_find(target_ip, &target_mac)) {
    // MAC found, proceed to send data
    // send_packet(target_mac, data);
} else {
    // MAC not in cache, attempt to resolve it via network
    // Wait up to 1000ms for the ARP reply
    if (ca_arp_resolve(target_ip, &target_mac, 1000U)) {
        // Resolution successful, now we can send
    } else {
        // Handle error: host unreachable
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_arp_find to look up an IP address in the ARP cache?
A: You should use ca_arp_find when you need to resolve an IPv4 address to its corresponding MAC address from the local ARP cache. This is typically done before sending an Ethernet frame to ensure the correct destination MAC address is used. It is not used for dynamic ARP resolution; it only checks pre-populated cache entries.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_arp_find in C with caOS types?
A: ca_ipv4_t ip = {192, 168, 1, 10}; mac_t mac; bool found = ca_arp_find(ip, &mac); if (found) { // Use mac here }

Parameters conf:0.90 **Q:** What is the significance of the mac_out parameter in ca_arp_find, and what happens if it's NULL?
A: The mac_out parameter is a pointer to a buffer where the MAC address will be copied if the IP is found in the ARP cache. If mac_out is NULL, the function still returns true if the IP is found, but no MAC address is copied. This is a valid usage, but the caller must be aware that no result will be returned.

Return conf:0.95 **Q:** What does it mean when ca_arp_find returns false?
A: When ca_arp_find returns false, it means the specified IPv4 address was not found in the ARP cache. This does not indicate an error in the function itself, but rather that

no mapping exists for the given IP at that time. It is up to the caller to handle this case, possibly by initiating an ARP request.

Error/Edge case conf:0.85 **Q:** What are common mistakes when calling ca_arp_find?

A: Common mistakes include passing a NULL pointer for mac_out when expecting to retrieve the MAC address, which leads to no data being copied despite a successful lookup. Another mistake is calling ca_arp_find from an ISR, which is not allowed due to potential mutex deadlock issues. Also, calling the function before the ARP cache has been initialized or populated may lead to unexpected behavior.

---

ca_arp_flush

Header: `caos/net/inc/arp_service.h:111`
Source: `caos/net/src/arp_service.c:405:410`
GUARD: `EN_ARP_SERVICE`

```
void ca_arp_flush(void)
```

Clears all entries in the ARP cache by marking them as invalid.

Ritorna:
This function returns void and does not provide a status code.

Note:

- This function is thread-safe because it uses a mutex (s_cache_mtx) to protect the shared ARP cache array during the clearing process. It is a blocking function, meaning if another thread currently holds the mutex, the calling thread will be suspended until the mutex is released. You must NOT call this function from an Interrupt Service Routine (ISR) because blocking operations are strictly forbidden in ISRs and will cause system instability or hard faults. A common mistake is assuming this function resets the hardware network interface; it only clears the software-level mapping table, so the device will need to re-resolve IP-to-MAC addresses via ARP requests after this call.

Uso:

- This function is typically used during network stack resets, interface re-initialization, or when a user explicitly requests a cache clear via a CLI command to resolve connectivity issues caused by stale ARP entries. It is often paired with ca_arp_dump() to verify the state of the cache before and after the flush. In a typical lifecycle, it is called when the network link status changes or when the system detects a potential IP address conflict that requires forcing a fresh ARP discovery cycle.

Esempio:

```
/* Example: Clearing the ARP cache via a CLI command handler */
void handle_network_command(const char *cmd) {
    if (ca_strncmp(cmd, "flush", 5) == 0) {
        /* Clear the cache to force re-discovery of all network neighbors */
        ca_arp_flush();

        /* Notify the user that the operation is complete */
        CLI_PUT_N("ARP cache has been cleared successfully.");
    }
}
```

Q&A — fine-tuning review *2/2 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_arp_flush() and what is the intended use case for it?
A: You should call ca_arp_flush() when you need to invalidate all entries in the ARP cache, typically to force a renewal of network connections. This function is useful in scenarios where network topology changes or when you want to ensure that all MAC addresses are re-learned. It is a blocking function that acquires a mutex before clearing the cache, so it should be used carefully in high-frequency contexts to avoid temporary communication slowdowns.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_arp_flush() with caOS types?
A: void example_arp_flush(void) { ca_arp_flush(); }

`ca_arp_resolve`

Header: `caos/net/inc/arp_service.h:87`
Source: `caos/net/src/arp_service.c:294:331`
GUARD: `EN_ARP_SERVICE`

```
1 bool ca_arp_resolve(ca_ipv4_t ip, ca_mac_t *mac_out, u32 timeout_ms)
```

Resolves an IPv4 address to a MAC address using the ARP protocol with a blocking timeout.

Parametri:

Nome	Tipo	Descrizione
`ip`	`ca_ipv4_t`	The target IPv4 address to resolve.
`mac_out`	`ca_mac_t *`	Pointer to a buffer where the resolved MAC address will be stored. Must be a valid memory address.
`timeout_ms`	`u32`	Maximum time in milliseconds to wait for the ARP response. Must be greater than the internal ARP retry cycle to avoid race conditions.

Ritorna:
Returns true if the MAC address was successfully resolved and copied to mac_out. Returns false if the resolution failed, the timeout expired, or if no internal ARP slots were available.

Note:

- This function is blocking; it will suspend the calling thread until the ARP process completes or the timeout is reached. It is NOT safe to call from an ISR (Interrupt Service Routine) because it uses mutexes and semaphores that require thread context. A common mistake is providing a timeout that is too short; if the timeout is shorter than the time required for the ARP request/retry cycle, the function will return false even if the device is reachable. Ensure the system network stack is initialized before calling this, as it relies on an internal ARP task to process the request queue.

Uso:

- This function is typically used when preparing to send an Ethernet frame to a specific IP address. It is often paired with routing functions, such as checking a routing table to find the next hop IP, and then calling this function to resolve that IP into a hardware MAC address. In a typical application flow, you would call this before attempting to transmit data, ensuring the destination hardware address is known. If it returns false, the application should handle the failure by dropping the packet or notifying the user, as communication with that IP is currently impossible.

Esempio:

```
ca_ipv4_t target_ip = {{192, 168, 1, 10}}; ca_mac_t target_mac; // Buffer for
    result

// Attempt to resolve with a 2000ms timeout
// We use a generous timeout to allow for ARP retries
if (ca_arp_resolve(target_ip, &target_mac, 2000U)) {
    // Success: target_mac is now populated
    // Proceed to send Ethernet frame using target_mac
} else {
    // Failure: log error or inform the user that the host is unreachable
    // Do not attempt to send data using an uninitialized ca_mac_t
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_arp_resolve to resolve an IP address to a MAC address?

A: You should use ca_arp_resolve when you need to send an Ethernet frame to a specific IPv4 address and you don't already have its corresponding MAC address. This is typically required in network protocols like IP routing or when building an Ethernet driver that needs to resolve addresses before transmission.

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_arp_resolve with caOS types?

A: ca_ipv4_t target_ip = {192, 168, 1, 10}; mac_t resolved_mac; bool success = ca_arp_resolve(target_ip, &resolved_mac, 1000); if (success) { // Use resolved_mac to send Ethernet frame }

Parameters conf:0.90 **Q:** What are the valid and invalid values for the ip parameter in ca_arp_resolve?

A: The ip parameter must be a valid ca_ipv4_t structure representing an IPv4 address. Invalid values include any structure that does not represent a valid IPv4 address, such as all zeros or an address outside the valid range. Passing an invalid IP will cause the function to return false.

Return conf:0.95 **Q:** What does it mean when ca_arp_resolve returns false?

A: ca_arp_resolve returns false when the ARP resolution fails. This can happen due to a timeout, no free ARP slots available, or an internal error during the resolution process. The MAC address buffer is not modified in this case.

Error/Edge case conf:0.90 **Q:** What are common mistakes when calling ca_arp_resolve?

A: Common mistakes include calling the function from an ISR (which is not allowed), passing a timeout value less than ARP_RETRY_N * ARP_RETRY_MS (causing premature timeouts), passing a NULL pointer for mac_out, or using an invalid IPv4 address in the ip parameter.

Behavior conf:0.95 **Q:** Is ca_arp_resolve blocking or non-blocking, and can it be called from an ISR?

A: ca_arp_resolve is a blocking function. It suspends the calling task until either the MAC address is resolved or the timeout expires. It cannot be called from an ISR because it uses blocking operations like ca_sem_timedwait, and must be called from a task context.

`ca_arp_send_garp`

Header: `caos/net/inc/arp_service.h:131`
Source: `caos/net/src/arp_service.c:437:453`
GUARD: `EN_ARP_SERVICE`

```
bool ca_arp_send_garp(void)
```

Sends a Gratuitous ARP (GARP) packet to announce the device's IP address on the network.

Ritorna:
Returns true if the GARP packet was successfully queued for transmission, false if the IP is not set, buffer allocation failed, or the packet construction failed.

Note:

- This function is non-blocking and thread-safe, as it relies on the internal network buffer management system. It is not intended to be called from an ISR because it performs memory allocation via ca_net_buf_alloc. A common beginner mistake is calling this function before the network stack is fully initialized or before the link is confirmed to be up, which will result in the function returning false or the packet being dropped by the hardware. Ensure that the local IP address has been assigned (either via static configuration or DHCP) before calling this, otherwise, the function will immediately return false.

Uso:

- This function is typically used during the network initialization sequence to announce the device's presence to other nodes on the local network segment. It is often paired with ca_net_proto_init to set up the stack and ca_eth_link_poll to ensure the physical link is active. Developers call this after the IP address is assigned to prevent IP conflicts and to update the ARP tables of other devices on the network. It is also used when a DHCP lease is renewed or when the network interface is reset to ensure the network topology remains consistent.

Esempio:

```
/* Initialize the network stack first */
ca_net_proto_init(&net_cfg);

/* Wait for the link to stabilize */
ca_os_sleep(500);
ca_eth_link_poll();

/* Announce our IP to the network */
if (ca_arp_send_garp()) {
    ca_stdout_printf_n("GARP sent successfully");
} else {
    ca_stdout_printf_n("Failed to send GARP: check IP assignment or buffer
    availability");
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_arp_send_garp() and why is it important for network stability?
A: You should call ca_arp_send_garp() whenever the local IP address changes on the network interface. This function sends a Gratuitous ARP (GARP) packet to notify other nodes in the network about the new IP address. This ensures that the ARP tables of neighboring devices are updated correctly and prevents communication issues due to stale IP-to-MAC mappings.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_arp_send_garp() in a caOS application?
A: u32 ip_addr = 0xC0A80101; // 192.168.1.1 // Assume IP is configured here if (ca_arp_send_garp()) { // GARP packet initialized for sending } else { // Failed to send GARP, check IP configuration or buffer availability }

Return conf:0.95 **Q:** What does it mean when ca_arp_send_garp() returns false?
A: A return value of false indicates that the GARP packet could not be initialized for sending. This typically happens if the local IP address has not been configured yet or if

there is no available buffer to construct the packet. It's a failure condition that should be handled by checking IP setup or resource availability.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_arp_send_garp()?
A: Common mistakes include calling ca_arp_send_garp() before the local IP address is properly configured, which leads to a false return. Another mistake is calling it from an ISR or a high-priority context without proper synchronization, which can lead to race conditions or system instability. Also, not ensuring sufficient network buffers are available may cause the function to fail.

Behavior conf:0.90 **Q:** Is ca_arp_send_garp() blocking or non-blocking, and is it safe to use in an ISR?
A: ca_arp_send_garp() is a non-blocking function. It does not wait for the packet to be transmitted but prepares it for sending. It is NOT safe to call from an ISR or high-priority context unless proper synchronization mechanisms are in place. It must be called from a thread context and is not thread-safe unless protected by appropriate locking.

`ca_arp_service_init`

Header: `caos/net/inc/arp_service.h:63`
Source: `caos/net/src/arp_service.c:211:229`
GUARD: `EN_ARP_SERVICE`

```
1 void ca_arp_service_init(ca_mac_t local_mac, ca_ipv4_t local_ip)
```

Initializes the ARP service internal structures, mutexes, and semaphores required for network address resolution.

Parametri:

Nome	Tipo	Descrizione
`local_mac`	`ca_mac_t`	The 6-byte hardware MAC address assigned to the device interface. Must be a valid, unique MAC address.
`local_ip`	`ca_ipv4_t`	The initial IPv4 address for the interface. Can be set to 0.0.0.0 if using DHCP to obtain an address later.

Ritorna:
void

Note:

- This function is not thread-safe and must be called exactly once during the system initialization phase, typically before the RTOS scheduler starts or before any network tasks are spawned. It performs critical setup of mutexes and semaphores; calling it while the network stack is already active will corrupt the internal state, leading to unpredictable behavior or system crashes. It is strictly forbidden to call this function from an Interrupt Service Routine (ISR) because it performs blocking-style initialization operations that are not ISR-safe. A common mistake is to call this function multiple times or after the network stack has already begun processing packets, which will reset the ARP cache and cause active connections to drop.

Uso:

- This function is the mandatory entry point for the ARP subsystem. It is typically called within the main initialization sequence, often immediately after the Ethernet hardware driver is initialized and the network interface is registered via ca_net_init. It is frequently paired with ca_arp_service_start, which enables the background processing of

ARP requests. In dynamic network environments, it is common to initialize the service with a zeroed IP address (0.0.0.0) and later update the IP configuration once a DHCP server provides a valid lease.

Esempio:

```c
/* Typical initialization sequence in ca_os_main */
void ca_os_main(void) {
    ca_mac_t my_mac = { .addr = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55} };
    ca_ipv4_t my_ip = { .addr = 0x00000000 }; // Will be updated via DHCP later

    // 1. Initialize hardware and network interface
    eth_gpio_init();
    ca_net_init(ca_eth_iface_get(&eth_cfg));

    // 2. Initialize ARP service before starting network tasks
    ca_arp_service_init(my_mac, my_ip);

    // 3. Start the service and background tasks
    ca_arp_service_start();

    // 4. Proceed to spawn application tasks
    ca_os_job("net", CA_TH_LEVEL2, task_net, 0, STACK_SMALL, CA_JOB_STD,
    CA_FAULT_RESTART_THREAD);
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_arp_service_init() in my caOS project?
A: You should call ca_arp_service_init() early in your application, before using any other ARP-related functions. It initializes the ARP cache and semaphores needed for handling ARP requests and replies. It must be called once before any ARP operations, typically during system initialization.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_arp_service_init() with caOS types?
A: u8 local_mac[6] = {0x00, 0x08, 0x9B, 0x00, 0x00, 0x01}; ipv4_t local_ip = 0x01020304; ca_arp_service_init(local_mac, local_ip);

Parameters conf:0.90 **Q:** What are the valid values for local_mac and local_ip in ca_arp_service_init()?
A: The local_mac must be a valid 6-byte MAC address. It cannot be NULL or invalid, as it's used to build ARP packets. The local_ip must be a valid IPv4 address. If set to 0.0.0.0, it indicates that the IP has not yet been assigned to the device.

Error/Edge case conf:0.85 **Q:** What can go wrong when calling ca_arp_service_init()?
A: If invalid parameters are passed, such as a NULL mac address or an invalid IP, the function may cause communication errors. Additionally, calling this function multiple times without resetting previous state may overwrite existing data. If the internal mutex or semaphore initialization fails, the function may not behave correctly, though it does not return an error code directly.

Behavior conf:0.90 **Q:** Is ca_arp_service_init() blocking or non-blocking, and is it safe to call from an ISR?
A: ca_arp_service_init() is a non-blocking function. It is not safe to call from an ISR, as it may attempt to use system resources like mutexes or semaphores that are not ISR-safe. It is thread-safe if called with valid parameters, but concurrent calls from multiple threads without external protection are not recommended.

ca_arp_service_start

Header: caos/net/inc/arp_service.h:69
Source: caos/net/src/arp_service.c:431:435
GUARD: EN_ARP_SERVICE

```
void ca_arp_service_start(void)
```

Initializes and launches the background ARP service task within the caOS scheduler.

Ritorna:
void

Note:

- This function is non-blocking; it simply registers a new job with the caOS scheduler and returns immediately. It is not thread-safe to call this from an ISR, as it interacts with the OS task management system. You must ensure that the ARP service has been properly initialized (e.g., via ca_arp_service_init) before calling this, otherwise the task may attempt to operate on uninitialized memory or invalid network configurations. A common beginner mistake is calling this function multiple times; doing so will attempt to spawn multiple instances of the ARP task, which will lead to unpredictable network behavior and potential system instability.

Uso:

- This function is typically called once during the system startup phase inside ca_os_main, alongside other network service starters like DHCP or UDP servers. It acts as the 'enable' switch for the ARP protocol stack, allowing the device to resolve IP addresses to MAC addresses. It is almost always preceded by a configuration step where the local IP and MAC addresses are set, ensuring the ARP service has the necessary identity to communicate on the network. Once started, the ARP service runs as a background task, handling incoming ARP requests and maintaining the ARP cache automatically.

Esempio:

```
void ca_os_main(void) {
    // 1. Configure the network identity first
    // Assuming s_mac and s_ip are defined globally
    ca_arp_service_init(s_mac, s_ip);

    // 2. Start the ARP service task
    // This registers the task with the scheduler
    ca_arp_service_start();

    // 3. Start other dependent services
    ca_udp_server_start(&srv_cfg);

    // 4. Proceed to launch other application tasks
    ca_os_job("app_task", CA_TH_LEVEL5, my_app_task, 0, STACK_SMALL, CA_JOB_STD,
     CA_FAULT_RESTART_THREAD);
}
```

Q&A — fine-tuning review *2/2 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_arp_service_start() and what is its purpose in a caOS network application?
A: You should call ca_arp_service_start() after initializing the ARP service to begin handling ARP requests and responses in the network. This function starts a task that manages ARP operations, which is essential for IP address resolution in Ethernet networks. It is non-blocking and must be called from a main task context, not from an ISR.

The function is not thread-safe and should not be called multiple times without stopping the previous task first.

Code example conf:0.85 **Q:** Can you provide a minimal C example using caOS types that demonstrates how to start the ARP service?
A: u8 result = ca_arp_service_start(); if (result != 0) { // Handle error return; } // ARP service is now running

---

`ca_arp_service_update_local_ip`

Header: `caos/net/inc/arp_service.h:101`
Source: `caos/net/src/arp_service.c:426:429`
GUARD: `EN_ARP_SERVICE`

```
void ca_arp_service_update_local_ip(ca_ipv4_t local_ip)
```

Updates the local IPv4 address used by the ARP service to resolve hardware addresses.

Parametri:

Nome	Tipo	Descrizione
`local_ip`	`ca_ipv4_t`	A structure containing the 4-byte representation of the new local IP address. Must be a valid IPv4 address; passing an uninitialized or zeroed structure may cause ARP resolution failures.

Ritorna:
void

Note:

- This function is non-blocking and executes immediately as it performs a simple memory assignment. It is thread-safe in the sense that it performs an atomic write to a static variable, but it does not use mutexes, meaning it should be called when the network configuration is stable to avoid race conditions during packet processing. It is safe to call from an ISR, though this is rarely necessary. The most common beginner mistake is calling this function before the ARP service has been initialized or while the network stack is actively processing traffic, which could lead to a momentary inconsistency in ARP replies.

Uso:

- This function is typically used during the network initialization phase or when a DHCP client receives a new IP address lease. It is part of a sequence where the application sets the local IP across multiple layers: first in the core network protocol stack, then in the UDP/TCP server modules, and finally in the ARP service to ensure that outgoing ARP requests use the correct source IP. It is usually paired with functions that configure the network interface to ensure the entire stack is synchronized with the new network identity.

Esempio:

```
ca_ipv4_t new_ip = {{192, 168, 1, 50}};

/* Update the core network stack first using internal network protocol functions */
/* ca_net_proto_set_local_ip(new_ip); */

/* Update the ARP service so it knows which IP to claim in ARP requests */
#if EN_ARP_SERVICE
    ca_arp_service_update_local_ip(new_ip);
```

```
#endif

/* Verify the change via console output */
ca_stdout_printf_n("ARP service updated to: %u.%u.%u.%u",
                   new_ip.v[0], new_ip.v[1], new_ip.v[2], new_ip.v[3]);
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_arp_service_update_local_ip() and why is it important for network operations?
A: You should call ca_arp_service_update_local_ip() when you need to update or reset the local IPv4 address used by the ARP service. This function is important for maintaining correct ARP table entries and ensuring proper network communication. It can be called from any context, including ISRs, and is typically used when the local IP changes (e.g., after DHCP assignment or manual configuration).

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_arp_service_update_local_ip() with caOS types?
A: ca_ipv4_t local_ip = 0xC0A80164; // 192.168.1.100 in network byte order ca_arp_service_update_ // To reset the IP: ca_arp_service_update_local_ip(0);

Parameters conf:0.90 **Q:** What does the local_ip parameter represent in ca_arp_service_update_local_ip(), and what are valid and invalid values?
A: The local_ip parameter represents the IPv4 address to be set as the local IP for ARP service operations. A valid value is any IPv4 address in network byte order (e.g., 0xC0A80164 for 192.168.1.100). If 0 is passed, the IP is reset. The function does not validate the IP address, so passing an invalid or malformed value may cause communication issues, but no error is returned.

`ca_arp_set`

Header: `caos/net/inc/arp_service.h:95`
Source: `caos/net/src/arp_service.c:251:288`
GUARD: `EN_ARP_SERVICE`

```
void ca_arp_set(ca_ipv4_t ip, ca_mac_t mac)
```

Updates or adds an entry in the ARP cache and notifies any tasks waiting for that specific IP address.

Parametri:

Nome	Tipo	Descrizione
`ip`	`ca_ipv4_t`	The IPv4 address structure to associate with the MAC address. Must be a valid IPv4 representation.
`mac`	`ca_mac_t`	The 6-byte MAC address structure to store in the cache.

Ritorna:
void

Note:

- This function is thread-safe as it uses a mutex to protect the internal cache array. It is a blocking operation because it acquires a mutex; therefore, it must NEVER be called from an Interrupt Service Routine (ISR), as this could lead to system deadlocks or crashes. The function implements an LRU (Least Recently Used) replacement policy: if the cache is full and the IP is not found, it overwrites the oldest entry. A common mistake is calling this

function from a high-priority interrupt context when processing incoming network packets; ensure that the network stack architecture offloads this to a task context if necessary.

Uso:

- This function is the primary mechanism for the network stack to 'learn' the mapping between IP and MAC addresses. It is typically called when an ARP reply is received or when a manual entry is added via a CLI command. It is often paired with ca_sem_post to wake up tasks that were blocked waiting for an ARP resolution to complete. In the network stack lifecycle, it is used during the packet parsing phase to ensure the ARP table remains current, allowing subsequent outgoing packets to be routed correctly without repeated ARP requests.

Esempio:

```
/* Example: Manually updating the ARP cache from a configuration task */
ipv4_t target_ip = {{192, 168, 1, 50}};
mac_t target_mac = {{0x00, 0x11, 0x22, 0x33, 0x44, 0x55}};

/* Update the cache with the new mapping */
ca_arp_set(target_ip, target_mac);

/* The function handles internal mutex locking and notifies any tasks
   waiting on this IP via semaphores automatically. */
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_arp_set() in a network application?
A: You should call ca_arp_set() when you need to update or insert an entry in the ARP cache, typically after receiving an ARP reply or when you have resolved an IP address to a MAC address. It is commonly used in network protocol implementations to maintain up-to-date address resolution tables.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_arp_set() with caOS types?
A: u8 ip_addr[4] = {192, 168, 1, 10}; u8 mac_addr[6] = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55}; ca_arp_set(ip_addr, mac_addr);

Parameters conf:0.90 **Q:** What are the valid values for the ip and mac parameters in ca_arp_set()?
A: The ip parameter must be a valid IPv4 address represented as a 4-byte array. The mac parameter must be a valid MAC address represented as a 6-byte array. Passing invalid or NULL values leads to undefined behavior.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_arp_set()?
A: Common mistakes include calling ca_arp_set() with NULL or invalid parameters, which leads to undefined behavior. Additionally, calling it from an ISR is not thread-safe and may cause issues due to mutex usage. If the ARP cache is full, the function may silently ignore the operation.

Behavior conf:0.90 **Q:** Is ca_arp_set() blocking or non-blocking, and is it safe to call from an ISR?
A: ca_arp_set() is blocking if the internal mutex is already acquired by another thread, which can lead to deadlock if not handled properly. It is not safe to call from an ISR because it uses mutexes and is not ISR-safe. It is also not thread-safe if called from multiple threads without proper synchronization.

`ca_barrier_init`

Header: `caos/inc/sync.h:367`
Source: `caos/src/sync.c:838:844`

GUARD: `EN_SYNC`

```
void ca_barrier_init(ca_barrier_t *b, u32 count)
```

Initializes a synchronization barrier object that forces a specific number of threads to wait until all have reached a common point.

Parametri:

Nome	Tipo	Descrizione
`b`	`ca_barrier_t *`	Pointer to the barrier structure to be initialized. Must be allocated in memory accessible by the RTOS.
`count`	`u32`	The number of threads that must call the wait function before any are allowed to proceed. Must be greater than 0.

Ritorna:
void

Note:

- This function is non-blocking and performs the setup of internal structures like the wait queue and spinlock. It is not thread-safe if called while other threads are actively using the same barrier, so it must be called during the system initialization phase before any worker threads are spawned. It is not intended to be called from an ISR. A common beginner mistake is to initialize the barrier with a count of 0 or to re-initialize it while threads are already blocked on it, which will lead to undefined behavior or system deadlocks.

Uso:

- The barrier is typically used in parallel processing scenarios where multiple worker threads must synchronize their progress before moving to the next phase of a task. You initialize the barrier once during the system setup phase, then pass the pointer to the worker threads. It is usually paired with ca_barrier_wait that threads call to pause execution until the 'count' threshold is met. After the barrier is no longer needed, ensure no threads are still waiting on it before deallocating the memory.

Esempio:

```
ca_barrier_t bar;
#define N_WORKERS 3

void system_init(void) {
    /* Initialize the barrier for 3 workers before starting tasks */
    ca_barrier_init(&bar, N_WORKERS);

    /* Now it is safe to spawn threads that will use this barrier */
    ca_os_job("worker1", CA_TH_LEVEL2, worker_task, NULL, STACK_SMALL,
    CA_JOB_STD, CA_FAULT_ASSERT);
    ca_os_job("worker2", CA_TH_LEVEL2, worker_task, NULL, STACK_SMALL,
    CA_JOB_STD, CA_FAULT_ASSERT);
    ca_os_job("worker3", CA_TH_LEVEL2, worker_task, NULL, STACK_SMALL,
    CA_JOB_STD, CA_FAULT_ASSERT);
}

void worker_task(void *arg) {
    /* Each worker performs work and then waits for others */
    /* Call the barrier wait function to synchronize with other threads */
    ca_barrier_wait(&bar);
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_barrier_init and what is its purpose in a multi-threaded caOS application?
A: You should call ca_barrier_init to initialize a synchronization barrier before any threads attempt to use it. The purpose of the barrier is to synchronize a fixed number of threads, ensuring they all reach the same point in execution before any of them can proceed further. This is useful for coordinating tasks in a multi-threaded environment where certain operations must complete before others begin.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_barrier_init with caOS types?
A: ca_barrier_t my_barrier; ca_barrier_init(&my_barrier, 3); // The barrier is now initialized to wait for 3 threads before allowing them to proceed.

Parameters conf:0.90 **Q:** What is the significance of the 'count' parameter in ca_barrier_init, and what happens if it is set to zero?
A: The 'count' parameter specifies how many threads must reach the barrier before all threads can proceed. If count is set to zero, the barrier will not function correctly because it requires at least one thread to participate in the synchronization. The parameter must be greater than zero, otherwise the behavior is undefined and may cause the barrier to never release any threads.

`ca_barrier_wait`

Header: `caos/inc/sync.h:374`
Source: `caos/src/sync.c:846:872`
GUARD: `EN_SYNC`

```
bool ca_barrier_wait(ca_barrier_t *b)
```

Synchronizes multiple threads at a specific point by blocking them until a predefined number of threads have arrived.

Parametri:

Nome	Tipo	Descrizione
`b`	`ca_barrier_t *`	Pointer to an initialized ca_barrier_t structure. Must be properly allocated and initialized via ca_barrier_init before use.

Ritorna:
Returns true only for the last thread that reaches the barrier (the one that triggers the release). Returns false for all other threads that arrive earlier and are subsequently blocked.

Note:

- This function is blocking; threads that arrive before the count is met will be suspended and removed from the CPU scheduler until the barrier is released. It is NOT safe to call this function from an Interrupt Service Routine (ISR) because it performs thread context switching and blocking operations. The function uses an internal spinlock to ensure thread safety during the increment of the arrival counter. A common beginner mistake is to assume all threads continue execution simultaneously; in reality, the threads are woken up by the scheduler, and their order of resumption depends on the scheduler's priority policy. Ensure the barrier count is correctly set during initialization, otherwise, threads may block indefinitely if the count is never reached.

Uso:

- The barrier is typically used in parallel processing scenarios where multiple worker threads must complete a specific stage of computation before moving to the next phase. The typical sequence is: initialize the barrier with a specific count using ca_barrier_init, spawn worker threads, have each worker perform its task, and then call ca_barrier_wait. The thread that receives 'true' is often designated as the 'coordinator' to perform cleanup, log progress, or signal a main control thread that the phase is complete. After the barrier is released, all threads proceed to the next block of code, effectively synchronizing the execution flow across the system.

Esempio:

```c
/* Example: Synchronizing 3 worker threads */
ca_barrier_t bar;

void worker_task(void *arg) {
    /* Perform some work */
    ca_os_sleep(50u);

    /* Wait for all 3 workers to reach this point */
    bool is_last = ca_barrier_wait(&bar);

    if (is_last) {
        /* Only the last thread executes this cleanup */
        ca_stdout_printf_n("All workers finished, proceeding to next phase.");
    }

    /* All threads continue here after the barrier is released */
    /* Proceed to next processing stage */
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_barrier_wait() in a caOS application?
A: You should use ca_barrier_wait() when you need to synchronize multiple tasks at a specific point in your code, ensuring that all tasks reach that point before any of them proceed. This is useful for coordinating parallel execution paths, such as in multi-core or multi-threaded applications where certain operations must complete before others begin.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_barrier_wait() with caOS types?
A: ca_barrier_t barrier; ca_barrier_init(&barrier, 2); // Task 1 if (ca_barrier_wait(&barrier)) { // Only one task reaches here // Proceed with post-barrier logic } // Task 2 if (ca_barrier_wait(&barrier)) { // Only one task reaches here // Proceed with post-barrier logic }

Parameters conf:0.95 **Q:** What does the 'b' parameter in ca_barrier_wait() represent and what are valid values?
A: The 'b' parameter is a pointer to a pre-initialized ca_barrier_t structure. It must not be NULL and must point to a valid barrier that has been initialized with ca_barrier_init(). Passing a NULL pointer or an uninitialized barrier will result in undefined behavior or a crash.

Return conf:0.90 **Q:** What do the return values of ca_barrier_wait() indicate?
A: The function returns true if the calling task is the last one to reach the barrier, indicating that all other tasks have also arrived and synchronization is complete. It returns false for all other tasks, which will block until the last task arrives and returns true.

Error/Edge case conf:0.90 **Q:** What are common mistakes or error conditions when using ca_barrier_wait()?
A: Common errors include calling ca_barrier_wait() without first initializing the barrier

with ca_barrier_init(), passing a NULL pointer, or using an uninitialized or corrupted barrier structure. These conditions can lead to undefined behavior or system crashes.

Behavior conf:0.90 **Q:** Is ca_barrier_wait() thread-safe, ISR-safe, and blocking?

A: Yes, ca_barrier_wait() is thread-safe, using a spinlock to protect critical sections. It is also blocking for all tasks except the last one, which returns true. It is not safe to call from an ISR because it blocks and may cause deadlocks or undefined behavior.

---

ca_bkp_count

Header: `caos/drivers/inc/bkp.h:22`
Source: `caos/drivers/src/bkp.c:14:17`
GUARD: `EN_BKP`

```c
u32 ca_bkp_count(void); /* registers available on this part */
```

Returns the total number of available hardware backup registers on the current device.

Ritorna:
A u32 value representing the count of available backup registers. Returns 0 if the hardware abstraction layer is not initialized or if the device lacks backup register support.

Note:

- This function is non-blocking and thread-safe, as it typically performs a simple read of a hardware-defined constant or register. It is safe to call from an ISR (Interrupt Service Routine) because it does not involve complex OS primitives or blocking operations. The most common beginner mistake is assuming the returned count is zero-indexed; if the function returns 16, the valid register indices are 0 through 15. Always ensure the system hardware abstraction layer is initialized before relying on this value.

Uso:

- This function is used to determine the bounds for data storage in non-volatile backup registers, which persist across system resets. It is typically paired with ca_bkp_write and ca_bkp_read to implement a safe indexing strategy. Developers use it to prevent 'out-of-bounds' access errors when writing status flags or error codes that must survive a reboot. It is often called during system initialization or within an emergency callback (like a watchdog reset handler) to log diagnostic information.

Esempio:

```c
/* Example: Safely writing a status marker to the last available backup register */
#include "caos.h"

void log_system_status(u32 status_code) {
    u32 count = ca_bkp_count();

    /* Ensure we have at least one register to avoid underflow */
    if (count > 0) {
        /* Use the last register index (zero-based) */
        u32 last_idx = count - 1U;

        /* Write the status code to the hardware backup register */
        ca_bkp_write(last_idx, status_code);
    } else {
        /* Handle case where no backup registers are available */
        ca_printf("Error: No backup registers available to log status.\n");
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_bkp_count and what is its primary purpose?
A: You should call ca_bkp_count during system initialization or within emergency callbacks like watchdog reset handlers. Its primary purpose is to determine the bounds for non-volatile storage in backup registers, allowing you to implement safe indexing strategies for data that must persist across system resets.

Code example conf:1.00 **Q:** How do I safely iterate through all available backup registers using caOS types?
A: You can determine the limit using ca_bkp_count and iterate using a u32 index. Example: u32 count = ca_bkp_count(); for(u32 i = 0; i < count; i++) { /* perform operations with index i */ }

Return conf:1.00 **Q:** What does the return value of ca_bkp_count represent, and what causes it to return 0?
A: The return value is a u32 representing the total count of available hardware backup registers. It returns 0 if the hardware abstraction layer is not initialized or if the device lacks backup register support. Note that the count is not zero-indexed; if it returns 16, valid indices are 0 through 15.

`ca_bkp_hw_count`

Header: `caos/drivers/inc/ca_glue_hw.h:362`
GUARD: `EN_CA_GLUE_HW`

```
extern u32 ca_bkp_hw_count(void); /* backup registers on this part */
```

Returns the total number of hardware backup registers available on the current STM32 microcontroller.

Ritorna:
An unsigned 32-bit integer representing the count of available backup registers (e.g., 10, 20, or 32 depending on the specific STM32 series).

Note:

- This function is a hardware abstraction layer (HAL) utility that provides the limit for backup register indexing. It is non-blocking, thread-safe, and safe to call from an ISR because it simply returns a constant value defined at compile-time for the specific target hardware. The most common mistake is assuming a fixed number of registers across different STM32 families; always use this function to determine the limit dynamically rather than hardcoding values like 10 or 32, which will cause out-of-bounds access on different chips.

Uso:

- This function is typically used as a boundary check before performing read or write operations on backup registers. It is usually paired with ca_bkp_read and ca_bkp_write to ensure that the index provided by the application logic does not exceed the physical hardware capacity. By using this function, your code remains portable across different STM32 series (e.g., F1, F4, H7) without requiring manual adjustments to array bounds or loop limits.

Esempio:

```
/* Example: Safely reading all available backup registers */
#include "caos.h"

```

```c
void dump_backup_registers(void) {
    u32 total_regs = ca_bkp_count();

    for (u32 i = 0; i < total_regs; i++) {
        u32 val = ca_bkp_read(i);
        /* Use ca_printf to output the register value */
        ca_printf("BKP[%lu] = 0x%08lX\n", i, val);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_bkp_hw_count in my caOS application?
A: You should call ca_bkp_hw_count as a boundary check before performing any read or write operations on backup registers. It is used to ensure that the index provided by your application logic does not exceed the physical hardware capacity, preventing out-of-bounds access.

Code example conf:0.90 **Q:** How do I safely write to a backup register using ca_bkp_hw_count?
A: You should use the function to validate the index before calling the write function: u32 index = 5; if (index < ca_bkp_hw_count()) { ca_bkp_write(index, 0xDEADBEEF); }

Return conf:1.00 **Q:** What does the return value of ca_bkp_hw_count represent and can it fail?
A: The function returns an unsigned 32-bit integer representing the total number of hardware backup registers available on the specific STM32 microcontroller. It cannot fail, as it is a non-blocking, thread-safe utility that returns a constant value defined at compile-time for the target hardware.

HW differences conf:1.00 **Q:** How does the number of backup registers differ across STM32 families in caOS?
A: The number of registers is hardware-dependent: STM32F1xx provides 10 registers, STM32F4xx provides 20 registers, and the F7xx, H5xx, H7xx, and U5xx families provide 32 registers. You must use ca_bkp_hw_count to determine the limit dynamically rather than hardcoding values to ensure portability.

`ca_bkp_hw_read`

Header: `caos/drivers/inc/ca_glue_hw.h:363`
GUARD: `EN_CA_GLUE_HW`

```c
extern u32 ca_bkp_hw_read(u32 idx); /* backup register idx (in range,
    checked by the caller) */
```

Reads a 32-bit value directly from a hardware backup register at the specified index.

Parametri:

Nome	Tipo	Descrizione
`idx`	`u32`	The index of the backup register to read. Must be less than the value returned by ca_bkp_count().

Ritorna:
The 32-bit value stored in the specified hardware backup register. If the index is out of bounds, the behavior is undefined (typically resulting in a memory access fault).

Note:

- This function is a low-level hardware abstraction layer (HAL) call and does not perform any bounds checking; it is intended to be called by the higher-level ca_bkp_read() function. It

is non-blocking and executes in constant time, making it safe to call from ISRs or critical sections. Because it accesses hardware registers directly, ensure the backup domain is properly initialized and powered before use. A common beginner mistake is calling this function directly instead of using the safer ca_bkp_read() wrapper, which handles index validation.

Uso:

- This function is used to retrieve persistent data stored in the STM32 backup domain, which survives system resets. It is typically used in a sequence where the application first checks the number of available registers using ca_bkp_count(), then reads specific indices to restore system state or configuration flags after a reboot. It is paired with ca_bkp_write() for data persistence and is often used during the system initialization phase to check for 'magic numbers' that indicate if the device has been previously configured.

Esempio:

```c
/* Example: Restoring a system configuration flag from backup memory */
#include "caos.h"

void restore_system_state(void) {
    u32 magic_idx = 0U;
    u32 expected_magic = 0xDEADBEEFU;

    /* Always check bounds using the safe wrapper before accessing hardware */
    if (magic_idx < ca_bkp_count()) {
        u32 current_val = ca_bkp_read(magic_idx);

        if (current_val == expected_magic) {
            /* Configuration is valid, proceed with system boot */
        } else {
            /* Backup data is invalid or uninitialized, perform factory reset */
            ca_bkp_write(magic_idx, expected_magic);
        }
    }
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_bkp_hw_read() instead of ca_bkp_read()?
A: You should generally avoid calling ca_bkp_hw_read() directly. It is a low-level function intended for use by the higher-level ca_bkp_read() wrapper. The wrapper provides necessary index validation, whereas ca_bkp_hw_read() performs no bounds checking and can lead to undefined behavior or memory access faults if the index is invalid.

Code example conf:0.90 **Q:** How do I safely read a value from a backup register using caOS types?
A: To safely read a value, first verify the index is within bounds using ca_bkp_count(), then perform the read: u32 idx = 0; if (idx < ca_bkp_count()) { u32 val = ca_bkp_hw_read(idx); }

Parameters conf:1.00 **Q:** What are the constraints for the 'idx' parameter in ca_bkp_hw_read?
A: The 'idx' parameter represents the index of the hardware backup register. It must be a u32 value strictly less than the total number of available registers returned by ca_bkp_count(). Providing an index outside this range results in undefined behavior, typically a memory access fault.

Return conf:1.00 **Q:** What does the return value of ca_bkp_hw_read represent, and what happens if it fails?
A: The function returns the 32-bit value stored in the specified hardware backup register. If the function 'fails' due to an out-of-bounds index, the behavior is undefined, which

typically manifests as a memory access fault rather than a specific error return code.

HW differences conf:1.00 **Q:** Are there differences in how ca_bkp_hw_read is implemented across STM32 families like F1xx, F4xx, or H7xx?
A: Based on the provided implementation files for F1xx, F4xx, F7xx, H5xx, H7xx, and U5xx, the implementation is identical across all these families. Each version simply returns the value at the index provided by the bkp_regs() accessor function.

---

ca_bkp_hw_sram_open

Header: `caos/drivers/inc/ca_glue_hw.h:377`
GUARD: `EN_CA_GLUE_HW`

```
extern u8 *ca_bkp_hw_sram_open(void); /* clock + backup domain write
    access on, returns the base */
```

Initializes the hardware backup SRAM domain and returns a pointer to its base address.

Ritorna:
Returns a pointer (u8 *) to the start of the Backup SRAM memory region on success. Returns NULL if the hardware platform does not support Backup SRAM (e.g., F1xx series).

Note:

- This function is not thread-safe and should be called during system initialization before any tasks attempt to access the backup memory. It performs low-level register manipulation to enable the power and clock domains required for the SRAM to persist data across resets. It is not intended to be called from an ISR, as it performs hardware configuration that should be done in a controlled, synchronous manner. A common mistake is assuming the memory is ready immediately after power-on without calling this function; failing to call it will result in a hard fault or bus error when attempting to access the memory address.

Uso:

- This function is typically used by the internal backup driver to lazily initialize the memory region upon the first request. It is paired with ca_bkp_sram_read and ca_bkp_sram_write to manage persistent data. Developers should check the return value against NULL before dereferencing the pointer to ensure the current hardware target actually supports this feature.

Esempio:

```
u8 *sram_ptr;

/* Check if the hardware supports backup SRAM before attempting to use it */
if (ca_bkp_sram_size() > 0U) {
    /* Initialize the hardware domain and get the base address */
    sram_ptr = ca_bkp_hw_sram_open();

    if (sram_ptr != NULL) {
        /* Perform a write operation to the first byte of backup SRAM */
        sram_ptr[0] = 0xAA;
    } else {
        /* Handle error: hardware initialization failed */
    }
} else {
    /* Backup SRAM is not available on this device */
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_bkp_hw_sram_open in my caOS application?
A: You should call it during system initialization, before any tasks attempt to access the backup memory. It is not thread-safe and should not be called from an ISR. It is required to enable the power and clock domains for the SRAM to persist data across resets; failing to call it will result in a hard fault or bus error.

Code example conf:1.00 **Q:** Provide a minimal C example for initializing the backup SRAM.
A: u8 *bkp_ptr = ca_bkp_hw_sram_open(); if (bkp_ptr != NULL) { /* Safe to access backup memory */ }

Return conf:1.00 **Q:** What does the return value of ca_bkp_hw_sram_open indicate?
A: It returns a pointer (u8 *) to the base address of the Backup SRAM on success. It returns NULL if the hardware platform does not support Backup SRAM, such as on the F1xx series. You must check for NULL before dereferencing the pointer to avoid errors.

HW differences conf:1.00 **Q:** How does the implementation of ca_bkp_hw_sram_open differ across STM32 families?
A: Implementations vary by register access and peripheral clocking. F4xx and F7xx use RCC_APB1ENR_PWREN and RCC_AHB1ENR_BKPSRAMEN, while H5xx uses RCC_AHB1ENR_BKPRAMEN and H7xx uses RCC_AHB4ENR_BKPRAMEN. U5xx requires enabling RCC_AHB3ENR_PWREN. Additionally, the base address returned differs, such as D3_BKPSRAM_BASE for H7xx versus BKPSRAM_BASE for others. F1xx does not support this and returns NULL.

`ca_bkp_hw_sram_retain`

Header: `caos/drivers/inc/ca_glue_hw.h:378`
GUARD: `EN_CA_GLUE_HW`

```
1 extern bool ca_bkp_hw_sram_retain(void); /* backup regulator on (Standby
     /VBAT retention), false if not possible */
```

Enables the backup regulator to maintain power to the Backup SRAM during low-power modes (Standby/VBAT).

Ritorna:
Returns true if the backup regulator is successfully enabled and ready, false if the hardware does not support it or if the regulator failed to stabilize within the timeout period.

Note:

- This function is non-blocking in some architectures (U5) but uses a busy-wait loop in others (H7, H5, F4, F7) to verify the regulator's readiness. It is generally thread-safe as it performs atomic register modifications, but it should be called during system initialization before entering low-power states. A common beginner mistake is assuming that calling this function automatically preserves all RAM; it only preserves the specific Backup SRAM region. If the hardware does not support Backup SRAM (like F1), this function will always return false, so always check the return value before assuming data persistence.

Uso:

- This function is typically used during the system power-up sequence or before entering a deep sleep mode where the main power rail is cut. It is usually paired with ca_bkp_sram_write to store critical data before sleep and ca_bkp_sram_read to retrieve it after a wake-up event. Developers should call this early in the application lifecycle to ensure the hardware state is prepared for potential power loss scenarios.

Esempio:

```c
/* Check if Backup SRAM is available and enable retention */
if (ca_bkp_sram_retain()) {
    /* Backup regulator is active, safe to write critical data */
    uint32_t data = 0xDEADBEEF;
    ca_bkp_sram_write(0, (uint8_t *)&data, sizeof(data));
} else {
    /* Retention not possible, handle error or log warning */
    ca_printf("Backup SRAM retention failed!\n");
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_bkp_hw_sram_retain in my caOS application?
A: You should call this function during the system initialization sequence, ideally before entering low-power modes like Standby or VBAT. It is intended to prepare the hardware to maintain power to the Backup SRAM, and it is typically paired with ca_bkp_sram_write and ca_bkp_sram_read to manage critical data persistence across power cycles.

Code example conf:1.00 **Q:** Provide a minimal example of how to initialize Backup SRAM retention in caOS.
A: void init_backup_storage(void) { if (ca_bkp_hw_sram_retain()) { /* Backup SRAM is ready for use */ } else { /* Handle failure or lack of hardware support */ } }

Return conf:1.00 **Q:** What does the boolean return value of ca_bkp_hw_sram_retain indicate?
A: A return value of true indicates the backup regulator is successfully enabled and ready. A return value of false indicates that the hardware does not support Backup SRAM (e.g., F1 series), the regulator failed to stabilize within the timeout period, or the initialization failed.

HW differences conf:1.00 **Q:** How does the implementation of ca_bkp_hw_sram_retain differ across STM32 families?
A: Implementations vary significantly: F1 returns false as it lacks support. F4 and F7 use the PWR->CSR/CSR1 registers with a busy-wait loop. H5 and H7 use PWR->BDCR/CR2 registers and check ready flags (BRRDY/BRRDH). The U5 implementation is non-blocking, checks the LDO status via PWR->SVMSR, and does not use a ready flag.

`ca_bkp_hw_sram_size`

Header: `caos/drivers/inc/ca_glue_hw.h:376`
GUARD: `EN_CA_GLUE_HW`

```c
extern u32 ca_bkp_hw_sram_size(void); /* backup SRAM bytes, 0 = none */
```

Returns the total size in bytes of the hardware-specific Backup SRAM available on the current MCU.

Ritorna:
Returns a u32 representing the size in bytes. A return value of 0 indicates that the specific MCU model does not have Backup SRAM hardware support.

Note:

- This function is a hardware abstraction layer (HAL) utility that provides compile-time or platform-specific information about the available non-volatile memory region. It is non-blocking, thread-safe, and safe to call from an ISR because it simply returns a constant value defined by the hardware configuration. The most common mistake is assuming that all STM32 variants have the same amount of Backup SRAM; always check the return value

before attempting to allocate or access memory buffers to avoid out-of-bounds access or null pointer dereferences.

Uso:

- This function is typically used during the initialization phase of a driver or a persistent data storage module to determine if the Backup SRAM is available for use. It is often paired with memory management functions to verify if the system can support persistent data retention across resets. Developers use this to dynamically size buffers or to decide whether to fallback to other storage mechanisms like internal Flash or external EEPROM if the returned size is 0.

Esempio:

```c
/* Check if Backup SRAM is available before initializing a persistent storage buffer */
u32 bkp_size = ca_bkp_hw_sram_size();

if (bkp_size > 0U) {
    /* Backup SRAM is present, proceed to use it for critical data */
    /* ca_bkp_sram_write(offset, data, length); */
} else {
    /* No Backup SRAM available, log an error or use alternative storage */
    ca_printf("Warning: No Backup SRAM detected on this hardware.\n");
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_bkp_hw_sram_size in my caOS application?
A: You should call this function during the initialization phase of a driver or persistent data storage module. It is used to determine if Backup SRAM is available so you can decide whether to use it for data retention across resets or to fallback to other storage mechanisms like internal Flash or external EEPROM if the returned size is 0.

Code example conf:1.00 **Q:** How do I check for Backup SRAM availability and store a value in C?
A: You should check the return value before accessing memory. Example: u32 size = ca_bkp_hw_sram_size(); if (size > 0) { /* Proceed with memory operations */ } else { /* Fallback to alternative storage */ }

Return conf:1.00 **Q:** What does a return value of 0 from ca_bkp_hw_sram_size signify?
A: A return value of 0 indicates that the specific MCU model does not have Backup SRAM hardware support. It does not indicate an error, but rather a lack of hardware capability for that specific platform.

HW differences conf:1.00 **Q:** How does the Backup SRAM capacity vary across different STM32 families in caOS?
A: Capacity varies by family: F1xx has 0 bytes, F4xx, F7xx, and H7xx provide 4096 bytes, U5xx provides 2048 bytes, and H5xx returns a value defined by the BKPSRAM_SIZE constant. Memory locations also differ, such as 0x40024000 for F4xx/F7xx versus 0x38800000 for H7xx.

`ca_bkp_hw_write`

Header: `caos/drivers/inc/ca_glue_hw.h:364`
GUARD: `EN_CA_GLUE_HW`

```c
extern void ca_bkp_hw_write(u32 idx, u32 value); /* unlocks the backup domain (DBP) and writes register idx */
```

Unlocks the hardware backup domain and writes a value to a specific backup register index.

Parametri:

Nome	Tipo	Descrizione
`idx`	`u32`	The index of the backup register to write. Must be within the valid range defined by the hardware (check ca_bkp_count()).
`value`	`u32`	The 32-bit data value to be stored in the register.

Ritorna:
void

Note:

- This function is a low-level hardware access routine. It performs a 'write-unlock' sequence on the Power Control (PWR) register to allow modification of the backup domain, which is protected by default to prevent accidental corruption. It is not thread-safe; if multiple tasks attempt to write to backup registers simultaneously, you must wrap this call in a mutex or scheduler lock. It is generally safe to call from an ISR, but be aware that it performs a direct register write which is blocking in terms of CPU cycles. The most common mistake is passing an index that exceeds the hardware's available register count, which can lead to memory corruption or hard faults.

Uso:

- This function is typically used to persist small amounts of configuration data or state across system resets. It is usually paired with ca_bkp_read() to retrieve the data after a reboot. In a typical application, you would first verify the index using ca_bkp_count(), then use ca_bkp_write() to save your data. Because the backup domain is battery-backed, this is the standard way to store 'non-volatile' flags that must survive a power cycle without writing to the main Flash memory.

Esempio:

```c
/* Example: Saving a system state flag to the first backup register */
#include "caos.h"

void save_system_state(u32 state_flag) {
    /* Check if the index is valid before writing */
    if (ca_bkp_count() > 0) {
        /* Lock the scheduler to ensure atomic access to the backup domain */
        ca_os_sched_lock();

        /* Write the state to the first backup register (index 0) */
        ca_bkp_write(0, state_flag);

        ca_os_sched_unlock();
    } else {
        /* Handle error: no backup registers available */
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_bkp_hw_write in my caOS application?
A: You should use ca_bkp_hw_write to persist small amounts of configuration data or state that must survive a system reset. Since the backup domain is battery-backed, this function provides a way to store non-volatile flags without writing to main Flash memory. It is typically paired with ca_bkp_read() to retrieve the data after a reboot.

Code example conf:1.00 **Q:** How do I safely write a value to a backup register using caOS types?
A: You should first verify the index using ca_bkp_count() to avoid memory corruption, then call ca_bkp_hw_write. Example: u32 idx = 0; u32 val = 0xDEADBEEF; if (idx < ca_bkp_count()) { ca_bkp_hw_write(idx, val); }

Parameters conf:1.00 **Q:** What are the constraints on the 'idx' parameter for ca_bkp_hw_write?
A: The 'idx' parameter represents the index of the backup register to write. It must be within the valid range defined by the hardware, which you should verify by calling ca_bkp_count() before the write operation. Passing an index that exceeds the available register count can lead to memory corruption or hard faults.

HW differences conf:1.00 **Q:** How does the implementation of ca_bkp_hw_write differ across STM32 families?
A: The function handles the backup domain protection (DBP) differently based on the MCU family's Power Control (PWR) register structure. For example, f1xx and f4xx use PWR->CR, f7xx and h7xx use PWR->CR1, h5xx uses PWR->DBPCR, and u5xx uses PWR->DBPR to unlock the backup domain.

`ca_bkp_read`

Header: `caos/drivers/inc/bkp.h:23`
Source: `caos/drivers/src/bkp.c:19:22`
GUARD: `EN_BKP`

```
u32 ca_bkp_read(u32 idx); /* 0 when idx is out of range */
```

Reads a 32-bit value from the non-volatile backup register at the specified index.

Parametri:

Nome	Tipo	Descrizione
`idx`	`u32`	The index of the backup register to read. Must be less than the value returned by ca_bkp_count().

Ritorna:
The 32-bit value stored in the backup register if the index is valid; returns 0 if the index is out of range.

Note:

- This function is non-blocking and thread-safe, as it performs a direct hardware read. It is safe to call from an ISR because it does not involve complex OS primitives or waiting. The most common mistake is assuming that a return value of 0 means the register contains 0; since 0 is also returned for invalid indices, you must ensure your index is always within the bounds provided by ca_bkp_count() before calling this function.

Uso:

- This function is typically used to persist system state across device resets, such as boot flags, error codes, or trial status markers. It is often paired with ca_bkp_write() to update the state and ca_bkp_count() to verify index bounds. In a typical lifecycle, you read the value during system initialization to decide the boot path, and update it during runtime when a state transition occurs (e.g., confirming a firmware update).

Esempio:

```
/* Check if the system is in a trial state using a backup register */
#include "caos.h"

```

```c
void check_system_status(void) {
    u32 count = ca_bkp_count();
    u32 target_idx = 0; /* Assume index 0 is used for system state */

    if (target_idx < count) {
        u32 state = ca_bkp_read(target_idx);
        if (state == 0xDEADBEEF) {
            /* Perform recovery or specific logic for this state */
        }
    } else {
        /* Handle error: index out of range */
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_bkp_read in my application lifecycle?
A: You should use ca_bkp_read during system initialization to retrieve persisted state, such as boot flags or error codes, to determine the appropriate boot path. It is also used during runtime to check state markers, often in conjunction with ca_bkp_write to update system status across device resets.

Code example conf:1.00 **Q:** How do I safely read a backup register value using caOS types?
A: You must verify the index against ca_bkp_count() before reading. Example: u32 idx = 0; if (idx < ca_bkp_count()) { u32 val = ca_bkp_read(idx); }

Parameters conf:1.00 **Q:** What are the constraints for the idx parameter in ca_bkp_read?
A: The idx parameter represents the index of the non-volatile backup register. It must be a u32 value that is strictly less than the total count returned by ca_bkp_count(). Providing an index outside this range is considered invalid.

Return conf:1.00 **Q:** Does a return value of 0 always indicate that the backup register contains the value 0?
A: No. A return value of 0 is ambiguous because it is returned both when the register contains 0 and when the provided index is out of range. You must validate the index using ca_bkp_count() before calling the function to ensure the returned 0 is a valid data value.

`ca_bkp_sram_read`

Header: `caos/drivers/inc/bkp.h:32`
Source: `caos/drivers/src/bkp.c:45:53`
GUARD: `EN_BKP`

```c
bool ca_bkp_sram_read(u32 off, void *dst, u32 len); /* false when out of range */
```

Reads a block of data from the Backup SRAM memory area.

Parametri:

Nome	Tipo	Descrizione
`off`	`u32`	The starting byte offset within the Backup SRAM to begin reading from.
`dst`	`void *`	Pointer to the destination buffer where the read data will be stored. Must not be NULL.
`len`	`u32`	The number of bytes to read. The sum of off and len must not exceed the total Backup SRAM size.

Ritorna:
Returns true if the read operation was successful. Returns false if the parameters are invalid (e.g., NULL pointer, out-of-bounds access, or if the requested length exceeds the available memory).

Note:

- This function is a synchronous, blocking operation that copies data directly from the hardware Backup SRAM to your provided buffer. It is thread-safe in the sense that it performs a simple memory copy, but it does not implement internal locking for concurrent access to the same memory region; if multiple tasks access the same Backup SRAM offsets, you must manage synchronization externally. It is safe to call from an ISR, as it performs no blocking OS primitives, but keep in mind that large read operations will consume CPU cycles and increase interrupt latency. The most common mistake is failing to check the return value, which leads to processing uninitialized or garbage data when the read fails due to an out-of-bounds offset.

Uso:

- This function is typically used to retrieve persistent configuration data, state flags, or diagnostic logs that must survive a system reset. It is often paired with ca_bkp_sram_size to determine the memory boundaries before reading, and ca_bkp_sram_write to save data. In a typical lifecycle, you would call ca_bkp_sram_size to verify the hardware is available, then perform one or more ca_bkp_sram_read calls to restore the application state during the system initialization phase.

Esempio:

```c
/* Example: Restoring a configuration structure from Backup SRAM */
#include "caos.h"

void restore_system_state(void) {
    u32 bkp_size = ca_bkp_sram_size();
    u32 config_buffer[16]; /* Buffer for 64 bytes of data */

    /* Check if Backup SRAM is available and large enough */
    if (bkp_size >= sizeof(config_buffer)) {
        /* Attempt to read the configuration from the start of Backup SRAM */
        if (ca_bkp_sram_read(0U, config_buffer, sizeof(config_buffer))) {
            /* Data read successfully, proceed to use config_buffer */
        } else {
            /* Handle read failure: log error or use default values */
            ca_printf_n("Failed to read Backup SRAM");
        }
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_bkp_sram_read in my application?
A: You should use this function during system initialization to restore persistent configuration data, state flags, or diagnostic logs that need to survive a system reset. It is typically used after calling ca_bkp_sram_size to verify memory boundaries.

Code example conf:1.00 **Q:** How do I read 16 bytes from the start of the Backup SRAM into a buffer?
A: u8 buffer[16]; if (ca_bkp_sram_read(0, buffer, 16)) { /* Data read successfully */ }

Parameters conf:1.00 **Q:** What are the constraints on the 'off' and 'len' parameters?
A: The 'off' parameter is the starting byte offset. The sum of 'off' and 'len' must not exceed the total Backup SRAM size. If this sum exceeds the available memory, the function will return false.

Return conf:1.00 **Q:** What causes ca_bkp_sram_read to return false?
A: The function returns false if the parameters are invalid, which includes passing a NULL pointer for the destination, requesting an out-of-bounds offset, or requesting a length that exceeds the available Backup SRAM capacity.

---

ca_bkp_sram_retain

Header: caos/drivers/inc/bkp.h:34
Source: caos/drivers/src/bkp.c:85:89
GUARD: EN_BKP

```
bool ca_bkp_sram_retain(void); /* backup regulator on: kept in Standby/
    VBAT */
```

Checks if the Backup SRAM is configured to retain data during low-power Standby or VBAT modes.

Ritorna:
Returns true if the hardware supports and is configured for SRAM retention, false if the regulator is off or the hardware does not support this feature.

Note:

- This function is non-blocking and thread-safe. It does not require complex initialization, but it relies on the underlying hardware support for Backup SRAM. It is safe to call from any context, including ISRs, as it performs a simple register check. A common mistake is assuming that data in Backup SRAM is automatically preserved across all power states; you must verify this function returns true to ensure your data will survive a transition to Standby mode.

Uso:

- Use this function when your application needs to store critical state information that must persist even if the main power supply is removed or the device enters a deep sleep state. It is typically used in a diagnostic or initialization sequence, often paired with ca_bkp_sram_size to determine how much space is available, and ca_bkp_sram_write or ca_bkp_sram_read to manage the actual data storage.

Esempio:

```
if (ca_bkp_sram_retain()) {
    /* Backup SRAM is active, safe to store critical system state */
    unsigned char data[] = "SYSTEM_OK";
    ca_bkp_sram_write(0, data, 9);
} else {
    /* Retention not available, log error or use alternative storage */
    ca_printf_n("Warning: Backup SRAM retention is disabled!");
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_bkp_sram_retain in my application?
A: You should call this function when your application needs to store critical state information that must persist through deep sleep or power loss. It is typically used during initialization or diagnostic sequences to verify that the hardware is correctly configured to keep data in Backup SRAM before attempting to store persistent data.

Code example conf:1.00 **Q:** How do I check if Backup SRAM is ready for data storage using caOS types?

A: You can verify the retention status using a simple conditional check as shown: if (ca_bkp_sram_retain()) { /* Safe to proceed with data operations */ } else { /* Handle error: retention not supported or regulator off */ }

Return conf:1.00 **Q:** What do the return values of ca_bkp_sram_retain indicate?

A: The function returns true if the hardware supports and is configured for Backup SRAM retention, ensuring data will survive a transition to Standby mode. It returns false if the backup regulator is off or if the specific hardware does not support the retention feature.

---

ca_bkp_sram_size

Header: `caos/drivers/inc/bkp.h:31`
Source: `caos/drivers/src/bkp.c:40:43`
GUARD: `EN_BKP`

```
u32 ca_bkp_sram_size(void); /* bytes, 0 = none */
```

Returns the total size in bytes of the available Backup SRAM memory.

Ritorna:
Returns a u32 representing the size in bytes. Returns 0 if no Backup SRAM is available or if the hardware feature is disabled.

Note:

- This function is thread-safe and non-blocking. It can be safely called from an ISR, as it simply queries the hardware configuration or a cached size value. The most common mistake is assuming that a non-zero return value guarantees that the memory is currently writable; you must still ensure the Backup SRAM domain is powered and unlocked (via hardware-specific registers) before attempting write operations. It does not perform any initialization itself, but it is often used as a guard condition before calling ca_bkp_sram_write.

Uso:

- Use this function to determine the memory capacity before performing bulk operations like saving system state, logs, or calibration data to the Backup SRAM. It is typically used in a sequence: first check the size with ca_bkp_sram_size, then allocate a buffer if necessary, and finally use ca_bkp_sram_write to commit data. It is essential for writing robust code that adapts to different STM32 variants, some of which may have different Backup SRAM sizes or none at all.

Esempio:

```
#include "caos.h"

void save_system_state(u32 *data, u32 len) {
    /* Check if Backup SRAM is available on this hardware */
    u32 bkp_size = ca_bkp_sram_size();

    if (bkp_size >= len) {
        /* Write data to Backup SRAM if it fits */
        (void)ca_bkp_sram_write(0U, data, len);
    } else {
        /* Handle case where memory is too small or unavailable */
        ca_printf("Backup SRAM too small or missing: %u bytes available\n",
    bkp_size);
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_bkp_sram_size and why is it important for my application?
A: You should call ca_bkp_sram_size before performing bulk operations like saving system state, logs, or calibration data. It is important because it allows your code to adapt to different STM32 variants that may have varying Backup SRAM sizes or none at all, serving as a guard condition before attempting write operations.

Code example conf:1.00 **Q:** How do I check if Backup SRAM is available before proceeding with a write operation?
A: You can use the following pattern: u32 size = ca_bkp_sram_size(); if (size > 0) { /* Proceed with memory operations */ }

Return conf:1.00 **Q:** What does the return value of ca_bkp_sram_size indicate, and does a non-zero value guarantee I can write to the memory?
A: The function returns a u32 representing the size in bytes. A return value of 0 indicates that no Backup SRAM is available or the feature is disabled. A non-zero return value does not guarantee the memory is writable; you must still ensure the Backup SRAM domain is powered and unlocked via hardware-specific registers before attempting writes.

`ca_bkp_sram_write`

Header: `caos/drivers/inc/bkp.h:33`
Source: `caos/drivers/src/bkp.c:59:83`
GUARD: `EN_BKP`

```
1 bool ca_bkp_sram_write(u32 off, const void *src, u32 len)
```

Writes data to the Backup SRAM, handling word-alignment and ECC requirements automatically.

Parametri:

Nome	Tipo	Descrizione
`off`	`u32`	The byte offset within the Backup SRAM where writing begins. Must be within [0, ca_bkp_sram_size() - 1].
`src`	`const void *`	Pointer to the source buffer containing the data to be written. Must not be NULL.
`len`	`u32`	Number of bytes to write. The sum of off + len must not exceed the total Backup SRAM size.

Ritorna:
Returns true if the write operation was successful. Returns false if parameters are invalid (e.g., NULL pointer, out-of-bounds access, or if Backup SRAM is unavailable).

Note:

- This function is not thread-safe and should not be called from an ISR. It performs a read-modify-write operation to handle unaligned access, which is critical because Backup SRAM on many STM32 chips (like the H7) uses ECC logic that requires full 32-bit word writes to maintain data integrity. If you attempt to write partial words without this function, a system reset could corrupt the surrounding bytes in the same word. The function is blocking, meaning it will not return until the entire memory copy is complete.

Uso:

- Use this function when you need to persist small amounts of configuration data or state across system resets. It is typically used in conjunction with ca_bkp_sram_size() to verify memory bounds before writing. In a typical lifecycle, you would initialize your data, call ca_bkp_sram_write() to save it, and later use ca_bkp_sram_read() to retrieve it after a reboot. Always ensure the offset and length are validated against the total size to prevent memory corruption.

Esempio:

```c
/* Example: Saving a system configuration structure to Backup SRAM */
#include "caos.h"

void save_system_state(const u32 *data, u32 count) {
    u32 size = ca_bkp_sram_size();
    u32 bytes_to_write = count * sizeof(u32);

    /* Ensure we do not overflow the Backup SRAM */
    if (size >= bytes_to_write) {
        /* Write the data starting at offset 0 */
        if (!ca_bkp_sram_write(0, data, bytes_to_write)) {
            /* Handle write error: log or trigger recovery */
        }
    }
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_bkp_sram_write in my caOS application?
A: You should use this function when you need to persist small amounts of configuration data or system state across system resets. It is typically used to save data that you intend to retrieve later using ca_bkp_sram_read after a reboot.

Code example conf:0.90 **Q:** Provide a minimal C example of how to write a u32 value to the Backup SRAM.
A: u32 my_data = 0xDEADBEEF; if (ca_bkp_sram_write(0, &my_data, sizeof(u32))) { /* Success */ }

Parameters conf:1.00 **Q:** What are the constraints on the 'off' and 'len' parameters for ca_bkp_sram_write?
A: The 'off' parameter must be within [0, ca_bkp_sram_size() - 1]. The sum of 'off' and 'len' must not exceed the total Backup SRAM size to prevent out-of-bounds access.

Return conf:1.00 **Q:** What does a return value of false indicate when calling ca_bkp_sram_write?
A: A return value of false indicates that the operation failed. This occurs if parameters are invalid, such as passing a NULL pointer, requesting an out-of-bounds memory access, or if the Backup SRAM is currently unavailable.

Error/Edge case conf:1.00 **Q:** What happens if I try to write partial words to the Backup SRAM without using this function?
A: On STM32 chips like the H7, the Backup SRAM uses ECC logic that requires full 32-bit word writes. Attempting to write partial words without this function could lead to a system reset or corruption of surrounding bytes within the same word.

Behavior conf:1.00 **Q:** Is ca_bkp_sram_write safe to call from an interrupt service routine (ISR) or in a multi-threaded environment?
A: No, this function is not thread-safe and must not be called from an ISR. Additionally, it is a blocking function, meaning it will not return until the entire memory copy operation is complete.

---

ca_bkp_write

Header: caos/drivers/inc/bkp.h:24
Source: caos/drivers/src/bkp.c:24:29
GUARD: EN_BKP

```
bool ca_bkp_write(u32 idx, u32 value); /* false when idx is out of range
    */
```

Writes a 32-bit value to a specific index in the non-volatile backup memory.

Parametri:

Nome	Tipo	Descrizione
idx	u32	The index of the backup register to write. Must be less than the value returned by ca_bkp_count().
value	u32	The 32-bit data to store in the backup register.

Ritorna:
Returns true if the write operation was successful (index was valid). Returns false if the index is out of range, indicating the write was ignored.

Note:

- This function is a wrapper for hardware-specific backup registers, which are typically preserved across system resets. It is not thread-safe by default, so ensure that multiple tasks do not attempt to write to the same index simultaneously without external synchronization. It is generally safe to call from an ISR if the underlying hardware access is atomic, but you should verify your specific hardware implementation. The most common mistake is passing an index that exceeds the hardware limit, which silently fails; always validate your index or use constants defined by the system.

Uso:

- This function is primarily used to persist system state, boot flags, or diagnostic markers that must survive a reboot. It is often paired with ca_bkp_read() to retrieve the stored value upon system startup. In bootloader scenarios, it is used to confirm successful image updates or to signal state transitions between the application and the bootloader. Always check the return value if your application logic depends on the persistence of the data.

Esempio:

```
/* Example: Persisting a boot state marker */
#define MY_BKP_INDEX 0U

void update_system_status(u32 new_status) {
    /* Attempt to write the status to backup memory */
    if (ca_bkp_write(MY_BKP_INDEX, new_status)) {
        /* Success: status is now in non-volatile storage */
    } else {
        /* Error: index was invalid, handle accordingly */
        ca_printf("Failed to write to backup register!\n");
    }
}

/* Example: Reading back the value later */
void check_status(void) {
    u32 status = ca_bkp_read(MY_BKP_INDEX);
    ca_printf("Current status: %u\n", status);
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_bkp_write in my caOS application?
A: You should use ca_bkp_write to persist system state, boot flags, or diagnostic markers that need to survive a system reset. It is commonly used to signal state transitions between an application and a bootloader or to confirm successful image updates.

Code example conf:1.00 **Q:** How do I write a value to the first backup register using caOS types?
A: u32 index = 0; u32 data = 0xDEADBEEF; if (ca_bkp_write(index, data)) { /* Success */ }

Parameters conf:1.00 **Q:** What are the constraints for the idx parameter in ca_bkp_write?
A: The idx parameter represents the index of the backup register to write. It must be a u32 value that is strictly less than the value returned by ca_bkp_count(). Passing an index that exceeds this hardware limit will cause the function to fail silently.

Return conf:1.00 **Q:** What does the boolean return value of ca_bkp_write indicate?
A: The function returns true if the write operation was successful, meaning the provided index was valid. It returns false if the index is out of range, which indicates that the write operation was ignored.

`ca_block_alloc`

Header: `caos/inc/block_pool.h:64`
Source: `caos/src/block_pool.c:39:50`
GUARD: `EN_BLOCK_POOL`

```
1 void *ca_block_alloc(ca_block_pool_t *bp)
```

Allocates a fixed-size memory block from a pre-allocated memory pool.

Parametri:

Nome	Tipo	Descrizione
`bp`	`ca_block_pool_t *`	Pointer to the initialized block pool structure. Must not be NULL and must have been initialized via the appropriate pool creation function.

Ritorna:
Returns a pointer to the allocated memory block if successful, or NULL if the pool is empty (no free blocks available).

Note:

- This function is thread-safe because it uses a spinlock to protect the free list during the allocation process. It is safe to call from an Interrupt Service Routine (ISR) because it uses a spinlock with interrupt disabling (ca_spinlock_lock_irq). The function is non-blocking; it returns immediately with NULL if no memory is available, rather than waiting for a block to be freed. A common beginner mistake is failing to check if the returned pointer is NULL before dereferencing it, which will cause a hard fault or undefined behavior if the pool is exhausted.

Uso:

- This function is used in scenarios where dynamic memory allocation (like standard malloc) is discouraged due to fragmentation or non-deterministic timing. It is typically used in a producer-consumer pattern where a task requests a block, fills it with data, and then passes it to a queue or another task. It is usually paired with ca_block_free, which returns the block to the pool once the task is finished with it. Before calling this, the pool must be fully initialized to ensure the free list pointers are correctly linked.

Esempio:

```
/* Assuming block_pool is globally initialized */
DataBlock *blk = (DataBlock *)ca_block_alloc(&block_pool);

if (blk != NULL) {
    /* Successfully allocated, safe to write data */
    blk->seq = 123;
    /* ... process data ... */

    /* Always return the block to the pool when done */
    /* ca_block_free(&block_pool, blk); */
} else {
    /* Handle allocation failure: log error or retry later */
    handle_alloc_error();
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_block_alloc() in a real-time embedded system?
A: You should use ca_block_alloc() when you need to dynamically allocate fixed-size memory blocks from a pre-defined pool, typically for managing data structures or buffers in a deterministic way. It's ideal for scenarios where you want to avoid heap fragmentation and ensure predictable allocation times.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_block_alloc() with caOS types?
A: ca_block_pool_t pool; ca_block_pool_init(&pool, 10, sizeof(u32)); void *block = ca_block_alloc(&pool); if (block != NULL) { *(u32*)block = 0x12345678; }

Parameters conf:0.95 **Q:** What is the significance of the ca_block_pool_t pointer passed to ca_block_alloc()?
A: The ca_block_pool_t pointer must point to a valid and properly initialized pool. It defines the memory pool from which the block is allocated. If the pool is not initialized using ca_block_pool_init(), the behavior is undefined and may cause crashes or corruption.

Return conf:0.90 **Q:** What does it mean if ca_block_alloc() returns NULL?
A: If ca_block_alloc() returns NULL, it means the pool is exhausted and no more blocks are available for allocation. This is not an error condition per se, but the caller must handle this case, for example by retrying later or using a fallback mechanism.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_block_alloc()?
A: Common mistakes include not checking the return value for NULL, which can lead to accessing unallocated memory; and calling ca_block_free() on memory not allocated via ca_block_alloc(). Additionally, calling ca_block_alloc() without prior initialization of the pool leads to undefined behavior.

`ca_block_free`

Header: `caos/inc/block_pool.h:70`
Source: `caos/src/block_pool.c:52:64`
GUARD: `EN_BLOCK_POOL`

```
void ca_block_free(ca_block_pool_t *bp, void *ptr)
```

Returns a previously allocated memory block back to the specified block pool for future reuse.

Parametri:

Nome	Tipo	Descrizione
bp	ca_block_pool_t *	Pointer to the initialized block pool structure. Must not be NULL.
ptr	void *	Pointer to the memory block to be freed. If NULL, the function safely returns without action.

Ritorna:
This function returns void; it does not provide a status code because it is designed to always succeed as long as the pointer belongs to the pool.

Note:

- This function is thread-safe because it uses a spinlock to protect the internal free list from concurrent access by multiple tasks or interrupts. It is safe to call from an ISR because it uses the 'irq' variant of the spinlock, which disables local interrupts to prevent deadlocks. A common beginner mistake is passing a pointer that was not originally allocated from this specific pool, which will corrupt the free list and cause the system to crash or behave unpredictably during the next allocation. Always ensure the block pool has been properly initialized before calling this function, as using an uninitialized pool will lead to a hard fault.

Uso:

- This function is used to recycle memory blocks once a task has finished processing data, preventing memory leaks in long-running embedded applications. It is typically paired with a block allocation function (like ca_block_alloc) in a cycle: allocate a block, fill it with data, send it to a queue for processing, and then call ca_block_free once the consumer task is done with the data. It is also used as a fallback mechanism when a message queue is full, allowing the producer to release the memory back to the pool instead of losing the block.

Esempio:

```c
/* Assuming block_pool is initialized and blk is a pointer to a block */
void process_data(void *blk) {
    if (blk == NULL) return;

    /* Perform operations on the block */
    // ... data processing logic ...

    /* Return the block to the pool so it can be reused by other tasks */
    ca_block_free(&block_pool, blk);
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_block_free and why is it important for memory management in caOS?
A: You should call ca_block_free when you are done using a memory block that was previously allocated from a ca_block_pool_t. It is important because it returns the block to the pool's free list, making it available for future allocations. This ensures efficient reuse of memory and prevents memory leaks in a bare-metal environment where manual memory management is required.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_block_free with caOS types?
A: ca_block_pool_t pool; void *block = ca_block_alloc(&pool, 32); if (block != NULL) { // Use the block ca_block_free(&pool, block); }

Parameters conf:0.90 **Q:** What is the significance of the 'bp' parameter in ca_block_free, and what happens if it's invalid?
A: The 'bp' parameter is a pointer to the ca_block_pool_t from which the memory block was originally allocated. It must be a valid and initialized pool structure. If it is invalid or uninitialized, calling ca_block_free will likely cause a crash because the function relies on the pool's internal structures to manage the free list.

Return conf:0.95 **Q:** Does ca_block_free return any value, and what does it mean if it does not fail?
A: ca_block_free does not return any value. It is a void function. It cannot fail in the traditional sense, but it may silently ignore invalid inputs such as a NULL pointer. The function is designed to be robust and will not crash even if given invalid parameters, though incorrect usage can still lead to memory corruption.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_block_free that could lead to memory corruption?
A: Common mistakes include calling ca_block_free with a pointer that was not allocated from the specified pool, or calling it with a pointer that has been modified after allocation. Additionally, calling ca_block_free with a NULL pointer is safe but does nothing. Using the same block pointer more than once or freeing memory allocated by malloc or another system will corrupt the free list and lead to unpredictable behavior.

`ca_block_pool_free_count`

Header: `caos/inc/block_pool.h:75`
Source: `caos/src/block_pool.c:66:68`
GUARD: `EN_BLOCK_POOL`

```
1 u32 ca_block_pool_free_count(ca_block_pool_t *bp)
```

Returns the current number of available (free) memory blocks in a specific block pool.

Parametri:

Nome	Tipo	Descrizione
`bp`	`ca_block_pool_t *`	A pointer to the initialized block pool structure. Must not be NULL.

Ritorna:
Returns an unsigned 32-bit integer representing the count of currently free blocks. If the pool is empty, it returns 0.

Note:

- This function is non-blocking and executes in constant time, making it safe to call from both tasks and Interrupt Service Routines (ISRs). It is thread-safe in the sense that it performs a simple read of a variable, but note that the value returned is a 'snapshot'—it may change immediately after the function returns if another task allocates or frees a block. A common mistake is assuming this value remains constant for subsequent logic; always re-read the count if you need to make decisions based on the current availability of memory. Ensure the block pool pointer is valid and initialized before calling, as passing a NULL or uninitialized pointer will cause a hard fault.

Uso:

- This function is primarily used for system monitoring, diagnostics, and debugging purposes rather than for flow control. It is typically paired with 'ca_block_pool_total' to calculate the percentage of memory usage or to log system health statistics to a console or telemetry

interface. Developers use it in background tasks or periodic reporting threads to detect memory exhaustion trends before they lead to critical allocation failures in time-sensitive tasks.

Esempio:

```c
/* Example: Monitoring block pool health */
#include "caos.h"

void monitor_task(void *arg) {
    while (1) {
        /* Get current status of the pool */
        u32 free_blocks = ca_block_pool_free_count(&block_pool);
        u32 total_blocks = ca_block_pool_total(&block_pool);

        /* Log status if memory is running low */
        if (free_blocks < 5) {
            ca_stdout_printf_n("Warning: Low memory! Free: %u/%u", free_blocks,
    total_blocks);
        }

        /* Wait before next check */
        ca_os_sleep(1000);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should a developer call ca_block_pool_free_count() and why is it useful?
A: A developer should call ca_block_pool_free_count() to monitor memory usage in a block pool, which is useful for debugging and ensuring sufficient free memory is available. It returns the number of free blocks in the pool, allowing developers to track memory consumption and prevent potential memory exhaustion issues.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_block_pool_free_count() with caOS types?
A: ca_block_pool_t my_pool; ca_status_t status = ca_block_pool_create(&my_pool, 10, sizeof(u32)); if (status == CA_SUCCESS) { u32 free_count = ca_block_pool_free_count(&my_po // Use free_count for monitoring or debugging }

Parameters conf:0.95 **Q:** What does the 'bp' parameter represent in ca_block_pool_free_count() and what are valid/invalid values?
A: The 'bp' parameter is a pointer to a ca_block_pool_t structure that must be valid and properly initialized. If passed a NULL pointer or a pointer to an uninitialized pool, the behavior is undefined and will likely cause a crash. Only a valid, initialized ca_block_pool_t pointer should be passed.

Return conf:0.90 **Q:** What does ca_block_pool_free_count() return and what causes failures in its execution?
A: The function returns a u32 representing the number of free blocks in the pool. It does not return error codes; failures occur only due to invalid input parameters like NULL pointer or uninitialized pool, which cause undefined behavior and potential crashes. The function itself is not designed to signal errors.

`ca_block_pool_init`

Header: `caos/inc/block_pool.h:58`
Source: `caos/src/block_pool.c:16:37`
GUARD: `EN_BLOCK_POOL`

```
void ca_block_pool_init(ca_block_pool_t *bp, void *buf, u32 buf_size,
    u32 block_size)
```

Initializes a memory pool to manage fixed-size memory blocks for efficient, deterministic allocation.

Parametri:

Nome	Tipo	Descrizione
bp	ca_block_pool_t *	Pointer to the block pool control structure. Must be allocated in static memory or global scope.
buf	void *	Pointer to the raw memory buffer used for the pool. Must be aligned to 4 bytes for optimal performance.
buf_size	u32	Total size of the buffer in bytes. Determines the maximum number of blocks available.
block_size	u32	Size of each individual block. If smaller than 4 bytes, it is automatically padded to 4 bytes.

Ritorna:
void. This function does not return a status code; it assumes valid pointers are provided.

Note:

- This function is non-blocking and thread-safe as it initializes the internal spinlock. It is safe to call from an ISR, provided the buffer memory is already accessible. The function performs 'in-place' initialization, meaning it writes pointers into the provided buffer to link free blocks together; therefore, the buffer must not be used by other parts of the application before this initialization is complete. A common mistake is providing a buffer that is too small to hold even one block, or failing to ensure the buffer is properly aligned, which can lead to hard faults on some ARM Cortex-M architectures.

Uso:

- This function is typically called during the system startup phase or within a task initialization routine before any memory allocation requests are made. It is the mandatory first step in the lifecycle of a block pool, followed by calls to ca_block_alloc and ca_block_free. It is often paired with static buffer definitions to ensure memory is reserved at compile-time. Once initialized, the pool is ready to be used by other tasks to safely manage fixed-size data structures without the fragmentation risks associated with standard heap allocators.

Esempio:

```
/* Define a structure and a static buffer for the pool */
typedef struct { u32 id; u32 value; } DataBlock;
static u8 pool_buf[128];
static ca_block_pool_t my_pool;

void system_init(void) {
    /* Initialize the pool with the buffer and block size */
    ca_block_pool_init(&my_pool, pool_buf, sizeof(pool_buf), sizeof(DataBlock));

    /* Now the pool is ready for use with other caOS functions */
    /* void *ptr = ca_block_alloc(&my_pool); */
    /* if (ptr != NULL) { ca_block_free(&my_pool, ptr); } */
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_block_pool_init() in a caOS project?
A: You should call ca_block_pool_init() when you need to set up a fixed-size memory pool for efficient allocation and deallocation of blocks of memory. This is useful for managing resources like task stacks, message buffers, or any scenario where you repeatedly allocate and free fixed-size objects.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_block_pool_init() with caOS types?
A: ca_block_pool_t pool; static u8 buffer[1024]; ca_block_pool_init(&pool, buffer, 1024, 32);

Parameters conf:0.85 **Q:** What are the valid ranges for block_size and buf_size in ca_block_pool_init()?
A: The block_size must be greater than or equal to sizeof(void*) and must be aligned to 4-byte boundaries. The buf_size must be at least as large as block_size and should be sufficient to hold multiple blocks of block_size.

Return conf:0.90 **Q:** Does ca_block_pool_init() return any value, and what does it indicate?
A: The function does not return any value. It initializes the ca_block_pool_t structure in-place and prepares the memory pool for use. Failures are not indicated through return values but may manifest as incorrect behavior if parameters are invalid.

Error/Edge case conf:0.80 **Q:** What are common mistakes or edge cases when calling ca_block_pool_init()?
A: Common mistakes include passing an unaligned buffer, using a block_size smaller than sizeof(void*), or providing a buf_size that is too small to accommodate even one block. Additionally, calling this function from an ISR or without proper alignment can lead to undefined behavior.

Behavior conf:0.90 **Q:** Is ca_block_pool_init() thread-safe or ISR-safe, and does it block?
A: The function is blocking and not inherently thread-safe unless an external lock is used. It is also not safe to call from an ISR, as it is designed for use in task contexts where it can safely manage memory without interruption.

`ca_block_pool_total`

Header: `caos/inc/block_pool.h:80`
Source: `caos/src/block_pool.c:70:72`
GUARD: `EN_BLOCK_POOL`

```
u32 ca_block_pool_total(ca_block_pool_t *bp)
```

Returns the total number of memory blocks available in a specific block pool.

Parametri:

Nome	Tipo	Descrizione
`bp`	`ca_block_pool_t *`	A pointer to the initialized block pool structure. Must not be NULL; passing a NULL pointer will cause a hard fault or undefined behavior.

Ritorna:
Returns a u32 value representing the total capacity of the pool (the number of blocks defined at creation time). This value is constant after the pool is initialized.

Note:

- This function is non-blocking and executes in constant time, making it extremely lightweight. It is thread-safe because it only performs a read operation on a structure member that does not change after initialization. It is safe to call from an Interrupt Service Routine (ISR) or any task context. The most common mistake is passing an uninitialized or NULL pointer to the function, which will lead to a memory access violation. Ensure the block pool has been successfully created before calling this function.

Uso:

- This function is primarily used for system monitoring, diagnostics, and telemetry. It is typically paired with ca_block_pool_free_count to calculate the percentage of memory usage or to detect potential memory leaks in the application. Developers use it inside logging tasks or status reporting loops to provide visibility into the system's resource consumption. It is usually called after the system initialization phase, once the block pool has been allocated and is ready for use.

Esempio:

```c
/* Assuming block_pool is a global or accessible ca_block_pool_t instance */
caBlockPool block_pool;

void monitor_task(void *p) {
    while (1) {
        /* Get total capacity and current free blocks */
        u32 total = ca_block_pool_total(&block_pool);
        u32 free = ca_block_pool_free_count(&block_pool);

        /* Calculate usage percentage */
        u32 used = total - free;
        ca_stdout_printf_n("Pool Usage: %u/%u blocks", used, total);

        /* Wait before next update */
        ca_os_sleep(1000);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_block_pool_total, and what is its purpose in memory monitoring?
A: You should call ca_block_pool_total to monitor the total number of blocks available in a block pool, which helps in assessing memory utilization. It is useful for debugging memory leaks or ensuring sufficient memory is available for allocation. The function is non-blocking and returns immediately, making it suitable for real-time systems.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_block_pool_total with caOS types?
A: ca_block_pool_t my_pool; ca_status status = ca_block_pool_create(&my_pool, 10, sizeof(u32)); if (status == CA_SUCCESS) { u32 total_blocks = ca_block_pool_total(&my_pool); // Use total_blocks for monitoring }

Parameters conf:0.95 **Q:** What does the 'bp' parameter represent, and what happens if it is NULL or invalid?
A: The 'bp' parameter is a pointer to a valid and initialized ca_block_pool_t structure. If bp is NULL or points to an uninitialized structure, the behavior is undefined and may cause a crash. It is essential to ensure the pool is properly initialized before calling this function.

Return conf:0.90 **Q:** What does ca_block_pool_total return, and what causes it to fail?
A: ca_block_pool_total returns a u32 representing the total number of blocks in the block pool. It does not have failure modes in itself; however, calling it with a NULL or invalid pointer results in undefined behavior, which may lead to a crash or incorrect data.

`ca_boot_confirm`

Header: `caos/inc/boot_image.h:80`
Source: `caos/src/boot_image.c:60:66`

GUARD: `EN_BOOT_IMAGE`

```
void ca_boot_confirm(void)
```

Marks the current firmware image as successfully booted and stable in the bootloader's configuration.

Ritorna:
This function returns void and does not provide a status code.

Note:

- This function is used to finalize a 'trial' boot process. If the system is not currently in a trial state (checked via ca_boot_is_trial), the function returns immediately without doing anything. It is not thread-safe in the sense that it should only be called once the application has verified its own health (e.g., network connectivity, sensor initialization). It is not intended to be called from an ISR. The most common mistake is calling this too early, before the application has actually verified that it can perform its primary tasks, which could lead to 'bricking' the device if the new firmware is faulty but marked as confirmed.

Uso:

- This function is typically used in the main application loop or an initialization task after the system has successfully performed a self-test. It is usually paired with ca_boot_is_trial to ensure the confirmation logic only runs when a new update is being tested. Once ca_boot_confirm is called, the bootloader will consider this image as the 'golden' image and will not attempt to revert to the previous version on the next power cycle.

Esempio:

```
/* Check if we are running in a trial mode after an update */
if (ca_boot_is_trial()) {
    /* Perform critical system health checks */
    bool system_healthy = true; /* Logic to verify hardware/network */

    if (system_healthy) {
        /* Confirm the image so the bootloader keeps it permanently */
        ca_boot_confirm();
        ca_printf("Firmware confirmed successfully.\n");
    } else {
        /* If unhealthy, we do not confirm. On next reboot, the bootloader
           will detect the lack of confirmation and revert to the old image. */
        ca_printf("System unhealthy, not confirming image.\n");
    }
}
```

Q&A — fine-tuning review *2/2 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to call ca_boot_confirm in my application?
A: You should call ca_boot_confirm only after the application has verified its own health, such as confirming network connectivity or successful sensor initialization. It is typically called in the main application loop or an initialization task after a successful self-test. Calling it too early before verifying primary tasks can lead to 'bricking' the device if the firmware is faulty.

Code example conf:0.90 **Q:** How do I implement a check to confirm the firmware update using caOS types?
A: You should check the trial status before confirming the boot. Example: if (ca_boot_is_trial()) { /* Perform self-test logic here */ ca_boot_confirm(); }

ca_boot_image_slot

Header: caos/inc/boot_image.h:72
Source: caos/src/boot_image.c:18:26
GUARD: EN_BOOT_IMAGE

```
char ca_boot_image_slot(void)
```

Returns the identifier ('A' or 'B') of the memory slot from which the current application is executing.

Ritorna:
'A' if running from slot A, 'B' if running from slot B, or 0 if the execution context does not match a known boot slot.

Note:

- This function is non-blocking and thread-safe as it performs a simple read of the SCB->VTOR register. It is safe to call from an ISR because it does not involve any OS primitives, locks, or complex logic. The function relies on the bootloader correctly setting the Vector Table Offset Register (VTOR) before jumping to the application. If the application is started via a debugger or a non-standard bootloader that does not configure VTOR as expected, this function will return 0, which may cause logic errors in applications that assume they are always running from a valid slot.

Uso:

- This function is typically used during application initialization to determine the current execution context, allowing the firmware to distinguish between slot A and slot B. It is frequently paired with ca_boot_is_trial to verify if the current image is a 'trial' version or a confirmed production image. Developers use it to dynamically locate headers, verify versioning, or decide which backup registers to read for state management.

Esempio:

```
void check_my_slot(void) {
    char slot = ca_boot_image_slot();
    if (slot == 'A') {
        ca_printf("Running from Slot A\n");
    } else if (slot == 'B') {
        ca_printf("Running from Slot B\n");
    } else {
        ca_printf("Unknown execution slot!\n");
        return;
    }

    // Check if this is a trial image using the slot identifier
    if (ca_boot_is_trial()) {
        ca_printf("This is a trial image in slot %c\n", slot);
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_boot_image_slot in my application?
A: You should call it during application initialization. It is used to determine the current execution context, allowing the firmware to distinguish between slot A and slot B, which helps in dynamically locating headers, verifying versioning, or managing state via backup registers.

Code example conf:1.00 **Q:** How do I implement a check for the current boot slot using caOS types?
A: You can store the result in a u8 variable as shown: u8 slot = ca_boot_image_slot(); if (slot == 'A') { /* Logic for slot A */ } else if (slot == 'B') { /* Logic for slot B */ }.

Return conf:1.00 **Q:** What does it mean if ca_boot_image_slot returns 0?
A: A return value of 0 indicates that the execution context does not match a known boot slot. This typically happens if the application is started via a debugger or a non-standard bootloader that fails to correctly configure the SCB->VTOR register before jumping to the application.

`ca_boot_image_start`

Header: `caos/inc/boot_image.h:78`
Source: `caos/src/boot_image.c:50:58`
GUARD: `EN_BOOT_IMAGE`

```
void ca_boot_image_start(void)
```

Initializes the watchdog and starts a background task to monitor system health during trial mode.

Ritorna:
This function returns void and does not provide a status code.

Note:

- This function is intended to be called during the system startup sequence. It checks if the device is in 'trial' mode; if so, it initializes the Independent Watchdog (IWDG) and spawns a background thread to periodically feed it. It is not thread-safe in the sense that it should only be called once during initialization, typically within ca_os_main. A common mistake is calling this function multiple times, which could lead to multiple watchdog-feeding threads competing for resources or re-initializing hardware peripherals incorrectly.

Uso:

- This function is used in the early stages of the application lifecycle, specifically inside ca_os_main, to ensure the system remains responsive during trial periods. It is typically paired with other service startup functions like ca_arp_service_start or ca_dhcp_service_start. By spawning a dedicated job, it ensures that even if the main application logic hangs, the watchdog is serviced, preventing an unwanted system reset while the trial logic is active.

Esempio:

```
void ca_os_main(void) {
    /* Initialize system watchdog and trial monitoring */
    ca_boot_image_start();

    /* Start network services */
    ca_arp_service_start();
    ca_dhcp_service_start();

    /* Start main application task */
    if (ca_os_job("app", CA_TH_LEVEL3, task_app, NULL, STACK_MEDIUM, CA_JOB_STD,
     CA_FAULT_RESTART_THREAD) == 0) {
        /* Handle error: job creation failed */
    }

    /* Enter the OS scheduler */
    ca_os_run();
```

```
}
```

Q&A — fine-tuning review *2/2 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_boot_image_start and what is its primary purpose?
A: You should call ca_boot_image_start during the early system startup sequence, specifically within ca_os_main. Its purpose is to initialize the Independent Watchdog (IWDG) and spawn a background thread to feed it if the device is in 'trial' mode, ensuring the system remains responsive even if the main application logic hangs.

Code example conf:0.95 **Q:** How do I correctly implement ca_boot_image_start in my ca_os_main function?
A: It should be called once during initialization. Example: void ca_os_main(void) { ca_boot_image_start(); /* Other service startups like ca_arp_service_start() */ }

`ca_boot_is_trial`

Header: `caos/inc/boot_image.h:74`
Source: `caos/src/boot_image.c:28:34`
GUARD: `EN_BOOT_IMAGE`

```
bool ca_boot_is_trial(void)
```

Checks if the current firmware is running in a temporary 'trial' state, typically after an update.

Ritorna:
Returns true (1) if the system is currently running in a trial mode (waiting for confirmation), and false (0) otherwise.

Note:

- This function is thread-safe and reads from the system's backup registers to determine the boot state. It is non-blocking and can be safely called from any task or ISR. The function relies on the bootloader having correctly set the magic numbers in the backup registers during the boot process. A common mistake is to assume that a 'false' return means the firmware is invalid; it simply means the system is not in a trial state (it is either fully confirmed or not using the trial mechanism).

Uso:

- This function is primarily used by the application to decide whether to initialize safety mechanisms like the Independent Watchdog (IWDG). In a typical workflow, the application calls ca_boot_is_trial() at startup; if true, it initializes the watchdog and starts a feeder task. If the application fails to call ca_boot_confirm() before the watchdog expires, the system will reset, allowing the bootloader to revert to the previous stable firmware slot. It is often paired with ca_boot_image_start() to manage the transition from the bootloader to the main application.

Esempio:

```
void app_init_check(void) {
    /* Check if we are in a trial period after an update */
    if (ca_boot_is_trial()) {
        /* Initialize watchdog to ensure we confirm the update */
        if (ca_wdt_init(5000, 1, 0)) {
            ca_wdt_enable();
        }
        /* Log the status to the system console */
```

```
        ca_printf_n("System running in TRIAL mode. Confirm update soon.");
    } else {
        ca_printf_n("System running in NORMAL mode.");
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_boot_is_trial() and what is its primary purpose in my application?
A: You should call ca_boot_is_trial() at application startup. Its primary purpose is to determine if the system is in a temporary 'trial' state after an update. If it returns true, you should initialize safety mechanisms like the Independent Watchdog (IWDG) and start a feeder task to ensure the system can revert to a stable firmware slot if the application fails to confirm the boot.

Code example conf:0.90 **Q:** Provide a minimal C example showing how to check the trial state and conditionally initialize the watchdog.
A: bool is_trial = ca_boot_is_trial(); if (is_trial) { iwdg_init(); iwdg_start_feeder(); }

Return conf:1.00 **Q:** What do the return values of ca_boot_is_trial() signify, and does a false return indicate a firmware error?
A: A return of true (1) indicates the system is in a trial mode waiting for confirmation. A return of false (0) means the system is not in a trial state; it does not imply the firmware is invalid, only that the system is either fully confirmed or not utilizing the trial mechanism. The function reads this state from the system's backup registers.

`ca_boot_receive_update`

Header: `caos/inc/boot_image.h:87`
Source: `caos/src/boot_image.c:120:143`
GUARD: `EN_BOOT_IMAGE`

```
bool ca_boot_receive_update(ca_mac_t local_mac, u32 wait_ms)
```

Downloads a firmware update via TFTP into the inactive flash slot and triggers a system reset to apply it.

Parametri:

Nome	Tipo	Descrizione
`local_mac`	`ca_mac_t`	The MAC address to assign to the TFTP service for receiving the update.
`wait_ms`	`u32`	Timeout in milliseconds to wait for the TFTP transfer to complete. Use CA_WAIT_FOREVER to block indefinitely.

Ritorna:
Returns true if the update was successfully received, verified, and the boot request was queued. Returns false if the device is in a trial state, the TFTP transfer failed, the image header is invalid, or the boot request could not be initiated.

Note:

- This function is a blocking call that manages the entire lifecycle of a firmware update, including network reception and flash writing. It is not thread-safe and must be called from a task context, never from an ISR. Before calling this, ensure the system is not in an unconfirmed trial state, as the function will explicitly refuse to overwrite the current image.

A common mistake is calling this function without sufficient free flash space or while the network stack is not yet initialized, which will cause the underlying TFTP service to fail immediately.

Uso:

- This function is typically used in a dedicated background task responsible for system maintenance or remote updates. It is often paired with a loop that checks for updates periodically. Once the function returns true, the system will automatically reset into the bootloader to finalize the installation. It is the high-level entry point for the 'A/B' slot update mechanism, abstracting the complexity of TFTP, flash memory management, and bootloader communication.

Esempio:

```c
/* Example: Background task to handle remote firmware updates */
void update_task(void *pvParameters) {
    ca_mac_t my_mac = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55};

    while (1) {
        /* Attempt to receive update; blocks until transfer finishes or timeout */
        if (ca_boot_receive_update(my_mac, 60000)) {
            /* If successful, the system will reset into the bootloader */
            ca_printf("Update received, rebooting...\n");
        } else {
            /* Log failure and wait before retrying */
            ca_printf("Update failed or timed out, retrying in 5s...\n");
            ca_os_sleep(5000);
        }
    }
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_boot_receive_update in my application?
A: It should be called from a dedicated background task responsible for system maintenance or remote updates, typically within a loop that checks for updates periodically. It serves as the high-level entry point for the A/B slot update mechanism.

Code example conf:1.00 **Q:** Provide a minimal C example of how to invoke the update function.
A: ca_mac_t my_mac = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55}; u32 timeout = 5000; bool success = ca_boot_receive_update(my_mac, timeout);

Parameters conf:1.00 **Q:** What is the purpose of the wait_ms parameter and what value should I use to wait indefinitely?
A: The wait_ms parameter defines the timeout in milliseconds for the TFTP transfer to complete. You should use the constant CA_WAIT_FOREVER to block indefinitely.

Return conf:1.00 **Q:** What does a return value of false indicate?
A: A return value of false indicates that the update failed. This can occur if the device is in a trial state, the TFTP transfer failed, the image header is invalid, or the boot request could not be initiated.

Error/Edge case conf:1.00 **Q:** What are common mistakes that cause ca_boot_receive_update to fail?
A: Common mistakes include calling the function while the system is in an unconfirmed trial state, having insufficient free flash space, or calling it before the network stack is initialized.

Behavior conf:1.00 **Q:** Is ca_boot_receive_update thread-safe or safe to call from an ISR?
A: No, it is not thread-safe and must never be called from an ISR. It is a blocking call that must be executed from a task context.

ca_boot_request_update

Header: caos/inc/boot_image.h:83
Source: caos/src/boot_image.c:75:82
GUARD: EN_BOOT_IMAGE

```
bool ca_boot_request_update(char slot)
```

Requests a system reboot into the bootloader to perform a firmware update on a specific flash slot.

Parametri:

Nome	Tipo	Descrizione
slot	char	The target flash slot for the update. Must be either 'A' or 'B'. Any other character will cause the function to return false.

Ritorna:
Returns false if the slot is invalid, if the system is currently in an unconfirmed trial state, or if the backup register write fails. If successful, the function triggers a system reset and does not return.

Note:

- This function is a high-level control routine that triggers a system reset; it is not thread-safe in the sense that it alters global system state (backup registers) and halts execution. It must not be called from an ISR because it performs a system reset, which is destructive to the current execution context. A common beginner mistake is calling this function without verifying the integrity of the received update or ignoring the return value, which could lead to a boot loop if the backup register write fails. Ensure that the system is not in a 'trial' state that requires confirmation before attempting an update, as the function will block the request to prevent bricking an unverified installation.

Uso:

- This function is typically used at the end of a firmware download process (e.g., via TFTP or UART). The typical sequence is: 1) Receive the update data, 2) Verify the data integrity, 3) Call ca_boot_request_update to signal the bootloader to perform the swap/install upon the next reset. It is often paired with ca_os_sleep to ensure communication buffers are flushed before the reset occurs.

Esempio:

```
/* Example: Triggering an update after receiving a valid image */
char target_slot = 'B';

/* Ensure we are not in a critical state before requesting update */
if (ca_boot_is_trial() == false) {
    /* Wait briefly to allow communication buffers to clear */
    ca_os_delay_ms(100);

    /* Request the update; if this returns, the request failed */
    if (ca_boot_request_update(target_slot) == false) {
        ca_printf("Failed to initiate update request for slot %c\n", target_slot
    );
    }
} else {
    /* Handle case where trial must be confirmed first */
```

```
    ca_printf("Cannot update: current trial not confirmed.\n");
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to call ca_boot_request_update in my firmware update routine?
A: It should be called at the end of a firmware download process, specifically after receiving the update data and verifying its integrity. It is recommended to pair it with ca_os_sleep to ensure communication buffers are flushed before the system reset occurs.

Code example conf:0.90 **Q:** How do I implement a call to ca_boot_request_update in C using caOS types?
A: You should verify the return value to ensure the request was accepted before the system resets. Example: bool status = ca_boot_request_update('A'); if (!status) { /* Handle error */ }

Parameters conf:1.00 **Q:** What are the valid values for the 'slot' parameter in ca_boot_request_update?
A: The 'slot' parameter must be either 'A' or 'B'. Providing any other character will cause the function to return false and the update request will not be processed.

Return conf:1.00 **Q:** Why might ca_boot_request_update return false instead of triggering a reset?
A: The function returns false if the provided slot is invalid, if the system is currently in an unconfirmed trial state, or if the backup register write fails. If successful, the function triggers a system reset and does not return.

`ca_bswap16`

Header: `caos/inc/endian.h:63`
GUARD: `EN_ENDIAN`

```
static inline u16 ca_bswap16(u16 value)
```

Swaps the byte order of a 16-bit unsigned integer to convert between little-endian and big-endian formats.

Parametri:

Nome	Tipo	Descrizione
`value`	`u16`	The 16-bit value to be byte-swapped. Any valid u16 value is accepted.

Ritorna:
The byte-swapped 16-bit value (e.g., 0x1234 becomes 0x3412).

Note:

- This function is a highly efficient, non-blocking operation that typically maps to a single CPU instruction (REV16) on ARM Cortex-M processors. It is thread-safe and can be safely called from an ISR because it performs no memory access or OS-level blocking. The most common mistake is using this function on data that is already in the correct host endianness, which will corrupt the data; always verify the source format (e.g., network packets are usually big-endian) before applying the swap.

Uso:

- This function is essential when parsing binary protocols or reading hardware registers that do not match the CPU's native endianness. It is typically used in the data-link or application layer when processing raw buffers received from peripherals like SPI, I2C,

or Ethernet. You will often see it paired with manual buffer indexing or memory copy operations to ensure that multi-byte fields are correctly interpreted by the application logic.

Esempio:

```
/* Example: Reading a 16-bit big-endian value from a buffer */
#include "caos.h"

u16 process_network_data(u8 *buffer) {
    /* Combine two bytes into a 16-bit value (little-endian construction) */
    u16 raw_val = (u16)(((u16)buffer[1] << 8) | (u16)buffer[0]);

    /* If the network protocol is big-endian, swap the bytes to host order */
    u16 host_val = ca_bswap16(raw_val);

    return host_val;
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_bswap16 in my caOS application?
A: You should use ca_bswap16 when parsing binary protocols or reading hardware registers that do not match the CPU's native endianness. It is essential for converting between little-endian and big-endian formats, such as when processing raw buffers received from peripherals like SPI, I2C, or Ethernet.

Code example conf:1.00 **Q:** How do I implement a byte swap for a 16-bit value using caOS types?
A: You can use the function directly as shown: u16 raw_val = 0xAA55; u16 swapped = ca_bswap16(raw_val); // swapped becomes 0x55AA.

Parameters conf:1.00 **Q:** What are the constraints on the 'value' parameter for ca_bswap16?
A: The parameter 'value' is a u16 type representing the 16-bit value to be byte-swapped. Any valid u16 value is accepted as an input.

Return conf:1.00 **Q:** What does the return value of ca_bswap16 represent, and can it fail?
A: The function returns the byte-swapped 16-bit value (e.g., 0x1234 becomes 0x3412). The function cannot fail as it is a non-blocking, thread-safe operation that maps to a single CPU instruction; however, the result will be logically incorrect if the input data was already in the correct host endianness.

`ca_bswap16_signed`

Header: `caos/inc/endian.h:134`
GUARD: `EN_ENDIAN`

```
static inline i16 ca_bswap16_signed(i16 value)
```

Swaps the byte order of a 16-bit signed integer to convert between Little-Endian and Big-Endian formats.

Parametri:

Nome	Tipo	Descrizione
`value`	`i16`	The 16-bit signed integer to be byte-swapped. Any valid i16 value is accepted.

Ritorna:
The input value with its two bytes swapped. The return type is i16, preserving the signed representation.

Note:

- This function is a static inline helper that performs a byte-swap operation. It is thread-safe and can be safely called from an ISR because it performs a simple register-level manipulation without blocking or accessing shared resources. It is primarily used when communicating with external hardware or network protocols that use a different endianness than the STM32 (which is natively Little-Endian). A common mistake is assuming this function performs a mathematical sign inversion; it only swaps the memory layout of the bytes, not the value's sign.

Uso:

- Use this function when receiving 16-bit data from external sensors, network packets, or storage devices that transmit data in Big-Endian format. It is typically used in the data processing stage after receiving raw bytes from a communication peripheral like SPI or UART. It is often paired with ca_bswap32_signed or ca_bswap64_signed when handling larger data structures, and is essential for ensuring data integrity when parsing protocol headers or sensor registers.

Esempio:

```c
/* Example: Processing a 16-bit signed temperature value received from a sensor */
#include "caos.h"

void process_sensor_data(void) {
    u16 raw_data = 0x0A00; // Example: 0x0A00 received from a Big-Endian sensor

    /* Convert Big-Endian raw data to local Little-Endian format */
    i16 temperature = ca_bswap16_signed((i16)raw_data);

    /* Log the result using caOS logging facilities */
    ca_printf("Processed temperature: %d\n", temperature);
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_bswap16_signed in my caOS project?
A: You should use this function when communicating with external hardware, network protocols, or storage devices that transmit 16-bit data in Big-Endian format, as the STM32 is natively Little-Endian. It is typically used after receiving raw bytes from peripherals like SPI or UART.

Code example conf:1.00 **Q:** How do I implement a byte swap for a signed 16-bit sensor reading?
A: You can use the function as follows: i16 raw_data = 0x1234; i16 swapped_data = ca_bswap16_signed(raw_data);

Parameters conf:1.00 **Q:** Are there any restrictions on the i16 value passed to ca_bswap16_signed?
A: No, any valid i16 value is accepted. The parameter represents the 16-bit signed integer whose byte order you wish to swap.

Return conf:1.00 **Q:** Does the return value of ca_bswap16_signed indicate an error if the sign bit changes?
A: No, the function does not perform mathematical sign inversion or error checking. It returns the input value with its two bytes swapped, preserving the signed representation. There are no failure states as it performs simple register-level manipulation.

`ca_bswap32`

Header: `caos/inc/endian.h:85`
GUARD: `EN_ENDIAN`

```
static inline u32 ca_bswap32(u32 value)
```

Swaps the byte order of a 32-bit unsigned integer (Endianness conversion).

Parametri:

Nome	Tipo	Descrizione
value	u32	The 32-bit integer to be byte-swapped. Any 32-bit value is valid.

Ritorna:
The byte-swapped 32-bit integer (e.g., 0x11223344 becomes 0x44332211).

Note:

- This function is a high-performance utility that typically maps to the hardware 'REV' instruction on ARM Cortex-M processors. It is non-blocking, thread-safe, and can be safely called from an ISR because it performs a simple register-level operation without accessing shared resources or OS primitives. The most common mistake is using this on data that is already in the correct host endianness, which will corrupt the value; always verify if your data source is Big-Endian or Little-Endian before calling this.

Uso:

- This function is essential when parsing network protocols or reading binary files from external storage where data is stored in Big-Endian format, but your STM32 is running in Little-Endian mode. It is typically used in a sequence where you read raw bytes from a buffer, assemble them into a u32, and then pass the result through ca_bswap32 to ensure the CPU interprets the bytes correctly. It is often paired with ca_bswap16 or ca_bswap64 depending on the data width being processed, and is frequently used alongside memory management functions like ca_memory_allocate to prepare data structures for network transmission.

Esempio:

```c
#include "caos.h"

// Example: Reading a 32-bit Big-Endian value from a buffer
void process_network_data(u8 *buffer) {
    u32 raw_val;
    u32 host_val;

    // Assemble 4 bytes from buffer into a 32-bit integer (Big-Endian order)
    raw_val = ((u32)buffer[0] << 24) | ((u32)buffer[1] << 16) |
              ((u32)buffer[2] << 8)  | ((u32)buffer[3]);

    // Convert from Big-Endian (network) to Little-Endian (host)
    host_val = ca_bswap32(raw_val);

    // Log the result using caOS logging
    ca_printf("Converted value: %u\n", host_val);
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_bswap32 in my caOS project?
A: You should use ca_bswap32 when parsing network protocols or reading binary data from external storage like SD cards or Flash. It is necessary when the data source is in Big-Endian format and your STM32 is operating in Little-Endian mode, ensuring the CPU interprets the bytes correctly.

Code example conf:1.00 **Q:** How do I implement a simple byte swap for a 32-bit value using caOS types?
A: You can use the function directly as follows: u32 raw_data = 0x11223344; u32 swapped = ca_bswap32(raw_data); // swapped becomes 0x44332211.

Parameters conf:1.00 **Q:** What are the constraints on the 'value' parameter for ca_bswap32?
A: The 'value' parameter is a u32 type representing the 32-bit integer to be byte-swapped. Any 32-bit value is valid for this function.

Return conf:1.00 **Q:** What does the return value of ca_bswap32 represent, and can it fail?
A: The function returns the byte-swapped 32-bit integer. It cannot fail because it is a non-blocking, high-performance utility that maps to the hardware 'REV' instruction on ARM Cortex-M processors; it does not access shared resources or OS primitives.

`ca_bswap32_signed`

Header: `caos/inc/endian.h:143`
GUARD: `EN_ENDIAN`

```
static inline i32 ca_bswap32_signed(i32 value)
```

Swaps the byte order of a 32-bit signed integer to convert between Little-Endian and Big-Endian formats.

Parametri:

Nome	Tipo	Descrizione
`value`	`i32`	The 32-bit signed integer value to be byte-swapped. Any 32-bit value is valid.

Ritorna:
The input value with its bytes reversed (e.g., 0x12345678 becomes 0x78563412).

Note:

- This function is a static inline helper that performs a byte-swap on the underlying bit representation of a signed integer. It is thread-safe and non-blocking because it performs a simple CPU register manipulation without accessing shared resources or OS primitives. It is safe to call from an ISR (Interrupt Service Routine) because it does not trigger context switches or wait for locks. The most common mistake is assuming this function performs a mathematical sign inversion; it only affects the memory layout of the bytes, not the numerical sign of the value.

Uso:

- This function is typically used when communicating with external hardware or network protocols that use a different endianness than the STM32 (which is Little-Endian). You will often use it when preparing data to be sent via ca_uart_dma_send or ca_net_send, or when parsing incoming data from a buffer received via ca_queue_static_receive. It is essential for ensuring that multi-byte integers are interpreted correctly by both the sender and the receiver.

Esempio:

```
/* Example: Preparing a signed 32-bit value for a Big-Endian network protocol */

void send_signed_data(i32 raw_value) {
    /* Convert from local Little-Endian to Big-Endian for network transmission
    */
    i32 network_value = ca_bswap32_signed(raw_value);

```

```
    /* Send the swapped value over a network interface */
    /* ca_net_send(interface_handle, &network_value, sizeof(network_value)); */

    ca_printf("Original: %d, Swapped: %d\n", raw_value, network_value);
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_bswap32_signed in my caOS project?
A: You should use this function when communicating with external hardware or network protocols that use a different endianness than the STM32's native Little-Endian format. It is typically used when preparing data for transmission via ca_uart_dma_send or ca_net_send, or when parsing data received from ca_queue_static_receive.

Code example conf:1.00 **Q:** How do I implement a byte swap for a signed 32-bit integer in caOS?
A: You can use the function directly as shown here: i32 raw_data = 0x12345678; i32 swapped_data = ca_bswap32_signed(raw_data);

Parameters conf:1.00 **Q:** Are there any restrictions on the i32 value passed to ca_bswap32_signed?
A: No, any 32-bit signed integer value is valid. The function operates on the underlying bit representation of the integer, so there are no restricted or invalid input ranges.

Return conf:1.00 **Q:** What does the return value of ca_bswap32_signed represent, and can it fail?
A: The return value is the input integer with its bytes reversed (e.g., 0x12345678 becomes 0x78563412). The function cannot fail as it performs a simple CPU register manipulation; it is thread-safe, non-blocking, and safe to call from an ISR.

`ca_bswap64`

Header: `caos/inc/endian.h:110`
GUARD: `EN_ENDIAN`

```
static inline u64 ca_bswap64(u64 value)
```

Swaps the byte order of a 64-bit unsigned integer to convert between little-endian and big-endian formats.

Parametri:

Nome	Tipo	Descrizione
`value`	`u64`	The 64-bit integer value to be byte-swapped. Any 64-bit value is valid.

Ritorna:
The byte-swapped 64-bit integer. There are no error conditions as this is a pure mathematical transformation.

Note:

- This function is a non-blocking, thread-safe, and CPU-cycle-efficient operation that performs a direct register manipulation. It is safe to call from an ISR because it does not interact with the OS scheduler, memory, or peripherals. The most common mistake is using this function when the data is already in the correct host endianness, which will result in corrupted data values. Always verify the endianness of your data source (e.g., network packets or external sensors) before applying this swap.

Uso:

- This function is typically used when parsing binary protocols or reading raw data from external hardware that uses a different endianness than the STM32 (which is little-endian).

It is often paired with memory access functions like ca_memcpy or when reading from communication buffers. You use it to normalize incoming data into the host's native format so that subsequent arithmetic or logic operations work correctly. It is a fundamental building block for network stacks and file system drivers.

Esempio:

```c
/* Example: Converting a 64-bit timestamp received from a network buffer */
#include "caos.h"

u64 process_network_timestamp(u8 *buffer) {
    u64 raw_val;
    /* Copy raw bytes from buffer into a 64-bit variable */
    ca_memcpy(&raw_val, buffer, sizeof(u64));

    /*
     * Assuming the network protocol is Big-Endian,
     * we must swap bytes to match the STM32 Little-Endian format.
     */
    u64 host_val = ca_bswap64(raw_val);

    return host_val;
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_bswap64 in my caOS project?
A: You should use ca_bswap64 when parsing binary protocols or reading raw data from external hardware that uses a different endianness than the STM32's native little-endian format. It is used to normalize incoming data into the host's native format so that subsequent arithmetic or logic operations function correctly.

Code example conf:1.00 **Q:** Can you provide a minimal example of how to use ca_bswap64?
A: u64 raw_data = 0x0102030405060708ULL; u64 host_data = ca_bswap64(raw_data);

Parameters conf:1.00 **Q:** What are the constraints on the 'value' parameter for ca_bswap64?
A: The 'value' parameter is a u64 type representing the 64-bit integer to be byte-swapped. Any 64-bit value is valid, as there are no restricted ranges for this mathematical transformation.

Return conf:1.00 **Q:** What error codes does ca_bswap64 return if the operation fails?
A: There are no error conditions for this function. It is a pure mathematical transformation that always returns the byte-swapped 64-bit integer.

`ca_bswap64_signed`

Header: `caos/inc/endian.h:152`
GUARD: `EN_ENDIAN`

```c
static inline i64 ca_bswap64_signed(i64 value)
```

Converts a 64-bit signed integer between host and network byte order (endianness swap).

Parametri:

Nome	Tipo	Descrizione
`value`	`i64`	The 64-bit signed integer to be byte-swapped. Any valid 64-bit signed value is accepted.

Ritorna:
The byte-swapped 64-bit signed integer. There are no error conditions as this is a pure mathematical transformation.

Note:

- This function is a simple wrapper around ca_bswap64 that casts the result back to a signed type. It is a non-blocking, thread-safe, and pure function, meaning it performs no I/O and does not rely on RTOS state; therefore, it is perfectly safe to call from an ISR or any task context. The most common mistake is assuming this function handles network-to-host conversion automatically based on the CPU architecture; it always performs the swap, so only use it when you know the data is in the opposite endianness of your current system.

Uso:

- This function is typically used when processing raw binary data received from external hardware or network protocols that use Big-Endian format (e.g., network packets or specific sensor registers) on a Little-Endian STM32 processor. It is often paired with memory access functions like ca_memcpy or when parsing buffers received via ca_queue_static_receive. You use it to normalize data before performing arithmetic operations or storing it in your application's internal structures.

Esempio:

```c
/* Example: Processing a 64-bit timestamp received from a network buffer */
#include "caos.h"

i64 process_network_timestamp(void *buffer) {
    i64 raw_data;

    /* Copy raw bytes from the buffer into our local variable */
    ca_memcpy(&raw_data, buffer, sizeof(i64));

    /*
     * The network sends data in Big-Endian, but our STM32 is Little-Endian.
     * We must swap the bytes to get the correct signed integer value.
     */
    i64 host_timestamp = ca_bswap64_signed(raw_data);

    return host_timestamp;
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_bswap64_signed in my caOS application?
A: You should use this function when processing raw binary data from external hardware or network protocols that use Big-Endian format, such as data received from a sensor or network packet, to normalize it for your Little-Endian STM32 processor before performing arithmetic or storage.

Code example conf:1.00 **Q:** How do I implement a simple byte swap for a 64-bit signed integer in caOS?
A: i64 raw_data = 0x0102030405060708; i64 normalized_data = ca_bswap64_signed(raw_data);

Parameters conf:1.00 **Q:** Are there any restrictions on the i64 value parameter for ca_bswap64_signed?
A: No, there are no restrictions. Any valid 64-bit signed integer is accepted as the input parameter.

Return conf:1.00 **Q:** What error codes does ca_bswap64_signed return if the operation fails?
A: There are no error conditions for this function. It is a pure mathematical transformation that always returns the byte-swapped 64-bit signed integer.

ca_bus_error_init

Header: caos/inc/bus.h:31
Source: caos/src/bus.c:24:27
GUARD: EN_BUS

```
void ca_bus_error_init(void)
```

Enables the BusFault exception handler in the ARM Cortex-M processor to catch memory access errors.

Ritorna:
void

Note:

- This function is non-blocking and executes almost instantaneously as it only performs a single register write to the System Control Block (SCB). It is thread-safe because it modifies a system-level configuration register that is typically only accessed during the boot sequence. It is not intended to be called from an ISR, as system configuration should be finalized during the initialization phase before the scheduler starts. The most common mistake is calling this function before the system clock or basic hardware abstraction layers are ready, though in this specific case, it is safe to call early in main(). If you fail to call this, the processor might treat bus errors as generic HardFaults, making it significantly harder to debug exactly which memory address caused the crash.

Uso:

- This function is used exclusively during the system startup sequence, typically inside the main() function before the RTOS scheduler is started. It is usually paired with other hardware protection initializations like ca_mpu_init() and ca_mpu_protect_canary_msp(), which set up memory boundaries and stack overflow protection. By enabling BusFaults, you ensure that if the code attempts to access an invalid memory address or a peripheral that is not responding, the CPU triggers a specific exception handler rather than silently failing or hanging. It is a mandatory step for building robust embedded systems where memory safety and fault detection are critical for long-term stability.

Esempio:

```
int main(void) {
    /* Initialize memory protection unit and stack protection */
    ca_mpu_init();
    ca_mpu_protect_canary_msp((u32)&_msp_stack_base);

    /* Enable BusFault exception to catch invalid memory access */
    ca_bus_error_init();

    /* Initialize heap and scheduler after hardware protection is active */
    ca_memory_init(&_heap_start, &_heap_end);
    if (ca_scheduler_init(1) == false) {
        /* Handle initialization failure */
        while(1);
    }

    /* Start the RTOS scheduler */
    // ca_os_run();
    return 0;
}
```

Q&A — fine-tuning review *2/2 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_bus_error_init() in my caOS project, and why is it important?
A: You should call ca_bus_error_init() early in your caOS project initialization, before initializing the MPU and the scheduler. This function enables BusFault exception handling to detect invalid memory access errors. It's important because it helps catch memory access violations that could lead to system instability or unpredictable behavior. The function must be called before MPU and scheduler initialization, otherwise it may not work correctly or cause unexpected results.

Code example conf:0.90 **Q:** Can you show me a minimal C example using ca_bus_error_init() with caOS types?
A: void example_usage(void) { // Initialize bus error handling before MPU and scheduler ca_bus_error_init(); // Continue with other initialization // ... rest of initialization code }

`ca_can_deinit`

Header: `caos/drivers/inc/can.h:42`
Source: `caos/drivers/src/can.c:147:151`
GUARD: `EN_CAN`

```
bool ca_can_deinit(u8 num)
```

Disables and resets the specified CAN peripheral hardware controller.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The index of the CAN peripheral to deinitialize. Must correspond to a valid hardware instance (e.g., 0 for CAN1, 1 for CAN2) defined in the system configuration.

Ritorna:
Always returns true. This indicates that the deinitialization command has been issued to the hardware abstraction layer.

Note:

- This function is non-blocking and performs a direct hardware reset of the CAN controller. It is not thread-safe by itself, so ensure no other tasks are attempting to transmit or receive data on the same peripheral instance while this is called. It is generally safe to call from an ISR, but doing so is discouraged as it abruptly terminates communication. The most common beginner mistake is calling this function while the CAN bus is still active, which can lead to bus errors or 'stuck' states on the physical CAN network because the controller stops acknowledging frames mid-transmission.

Uso:

- This function is used during system shutdown, power-saving transitions, or when reconfiguring the CAN peripheral with different baud rates or filters. It is typically paired with an initialization function (e.g., ca_can_init) at the start of the application lifecycle. In a typical sequence, you would stop all application-level CAN processing, call ca_can_deinit to release the hardware resources, and then potentially call the init function again if a hardware reset or reconfiguration is required.

Esempio:

```c
/* Example: Shutting down CAN1 before entering a low-power mode */
#define CAN_PORT_1 0

void system_shutdown_can(void) {
    /* Ensure no further processing is done */
    if (ca_can_deinit(CAN_PORT_1)) {
        /* Hardware is now disabled, safe to power down or reconfigure */
    } else {
        /* This branch is theoretically unreachable based on current
    implementation */
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_can_deinit() and what is its purpose in a caOS application?
A: You should call ca_can_deinit() when you want to disable and release the hardware resources of a specific CAN channel. This function is typically used during system shutdown, resource cleanup, or when dynamically managing CAN channels. It ensures that the CAN peripheral is properly disabled and its associated memory or registers are freed for other uses.

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_can_deinit() with caOS types?
A: u8 result = ca_can_deinit(1); if (result) { /* CAN channel 1 successfully deinitialized */ } else { /* Invalid channel number or internal error */ }

Parameters conf:0.95 **Q:** What does the 'num' parameter in ca_can_deinit() represent, and what are valid values?
A: The 'num' parameter specifies the CAN channel number to be deinitialized. Valid values are 0, 1, 2, and 3, corresponding to the four CAN channels available on the STM32 microcontroller. Any other value, such as 4 or 5, is considered invalid and will cause the function to return false.

Return conf:0.90 **Q:** What do the return values of ca_can_deinit() indicate, and what causes it to return false?
A: The function returns true if the CAN channel was successfully deinitialized and its resources were released. It returns false if the channel number is invalid (not 0-3) or if an internal error occurs during the deinitialization process. Calling the function with an invalid parameter or after already deinitializing the same channel may also result in false.

`ca_can_hw_deinit`

Header: `caos/drivers/inc/ca_glue_hw.h:72`
GUARD: `EN_CA_GLUE_HW`

```c
extern void ca_can_hw_deinit(u8 num); /* disabilita e resetta il
    periferico CAN */
```

Disables and resets the specified CAN peripheral hardware and its associated clock.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The index of the CAN peripheral to deinitialize (e.g., 1 for CAN1, 2 for CAN2). Passing an invalid index will result in the function returning early without performing any action.

Ritorna:
void

Note:

- This function is non-blocking and performs direct register manipulation to put the CAN peripheral into a reset state and disable its clock. It is not thread-safe; if multiple tasks attempt to deinitialize or initialize the CAN hardware simultaneously, it can lead to race conditions or system instability. It is generally safe to call from an ISR, but you must ensure that no other part of the application is actively using the CAN peripheral, as this will immediately stop all communication and reset the hardware registers. A common beginner mistake is calling this function while a transmission is still in progress, which will cause the hardware to abort the operation abruptly without completing the frame. Furthermore, on some architectures, the clock is only disabled if both CAN instances are inactive, so calling this on one instance may not immediately cut the power to the entire peripheral block if the other instance is still running.

Uso:

- This function is used during the teardown phase of the application lifecycle, such as when shutting down a communication module or reconfiguring the system to save power. It is typically paired with an initialization function (like ca_can_init) to form a complete lifecycle. In the caOS driver architecture, it is called by the higher-level ca_can_deinit function to ensure that the hardware is safely powered down and the clock gate is disabled, preventing unnecessary power consumption when the CAN bus is not in use. Always ensure that any pending messages are handled or discarded before calling this, as the hardware reset will clear all internal buffers and configuration registers.

Esempio:

```c
/* Example: Shutting down CAN1 safely */
#include "caos.h"

void shutdown_can_module(void) {
    /* Ensure the peripheral is not currently transmitting before deinit */
    /* Check if the CAN queue is empty or wait for transmission completion */

    /* Perform hardware-level deinitialization */
    ca_can_hw_deinit(1);

    /* Log the event or update system state */
    ca_printf("CAN1 hardware disabled\n");
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_can_hw_deinit() and what is its purpose?
A: You should call ca_can_hw_deinit() to disable and reset a CAN peripheral, typically before reinitializing or shutting down the CAN channel. It is a low-level function that directly manipulates hardware registers to reset the CAN controller and disable its clock. It must not be called from an ISR or when other threads or ISRs are using the CAN channel.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_can_hw_deinit() with caOS types?
A: u8 can_num = 1; ca_can_hw_deinit(can_num);

Parameters conf:0.90 **Q:** What does the 'num' parameter in ca_can_hw_deinit() represent and what values are valid?
A: The 'num' parameter specifies which CAN channel to disable. For F1xx devices, only 1 is valid. For F4xx, F7xx, H5xx, and H7xx, both 1 and 2 are supported. Values other

than 1 or 2 are ignored and the function returns without performing any action.

HW differences conf:0.85 **Q:** How does ca_can_hw_deinit() differ between F1xx and H7xx families in terms of hardware implementation?
A: For F1xx, ca_can_hw_deinit() uses bxCAN (CAN1) and directly resets the CAN1 peripheral and disables its clock via RCC_APB1ENR_CAN1EN. For H7xx, it uses FDCAN (FDCAN1/FDCAN2) and sets the CCCR_INIT bit to enter initialization mode. Clock disabling for FDCAN is conditional and only occurs when both instances are inactive, using RCC_APB1HENR_FDCANEN.

`ca_can_hw_init`

Header: `caos/drivers/inc/ca_glue_hw.h:69`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_can_hw_init (const ca_can_cfg_t *cfg); /* inizializza il
    periferico CAN con baudrate e filtri */
```

Initializes the CAN/FDCAN hardware peripheral with specified baudrate and operating modes.

Parametri:

Nome	Tipo	Descrizione
`cfg`	`const ca_can_cfg_t *`	Pointer to a configuration structure containing the CAN instance number (1 or 2), desired baudrate, and flags for loopback or silent mode. Must not be NULL.

Ritorna:
Returns true if the hardware successfully entered and exited initialization mode with the new settings. Returns false if the configuration is invalid, the hardware fails to respond to initialization requests, or the baudrate calculation fails.

Note:

- This function is blocking because it waits for the hardware to acknowledge the transition into and out of initialization mode using a software timeout loop. It is not thread-safe and should not be called from an ISR, as it performs hardware-level register configuration that expects a stable system state. A common beginner mistake is calling this function multiple times without checking the return value or failing to ensure the system clock (HSE/SystemCoreClock) is correctly configured before initialization, which leads to incorrect baudrate timing.

Uso:

- This function is the primary hardware abstraction layer (HAL) entry point for CAN setup. It is typically called during the system boot sequence or when a specific communication interface needs to be brought online. It is always paired with a higher-level driver function (like ca_can_init) that manages the internal state tracking of active CAN peripherals. You should use this to prepare the peripheral before attempting to send or receive any CAN frames.

Esempio:

```
ca_can_cfg_t my_can_cfg = { .can_num = 1, .baudrate = 500000, .loopback = false,
    .silent = false };

/* Initialize the CAN hardware */
```

```
if (!ca_can_hw_init(&my_can_cfg)) {
    /* Handle error: hardware failed to initialize */
    while(1);
}

/* Proceed to use other caOS functions to send/receive data */
/* ca_can_send(1, &my_frame); */
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_can_hw_init() in my caOS application?
A: You should call ca_can_hw_init() during system initialization, typically right after enabling the CAN peripheral clock and before starting any CAN communication tasks. It's used to configure the CAN controller with a specific baudrate and filter settings, preparing the hardware for message transmission and reception.

Code example conf:0.95 **Q:** Can you show a minimal C example using ca_can_hw_init() with caOS types?
A:

```
ca_can_cfg_t cfg = {
    .can_num = 1,
    .baudrate = 125000,
    .loopback = false,
    .silent = false
};
bool success = ca_can_hw_init(&cfg);
if (!success) {
    // Handle initialization failure
}
```

Parameters conf:0.90 **Q:** What does the 'can_num' parameter in ca_can_hw_init() represent and what are valid values?
A: The 'can_num' parameter specifies which CAN controller instance to initialize. For bxCAN-based MCUs (like H5xx), it must be 1 or 2. For FDCAN-based MCUs (like H7xx), it also supports 1 or 2, but the underlying hardware registers differ. Invalid values like 0 or 3 will cause the function to return false.

Return conf:0.95 **Q:** What does a return value of false from ca_can_hw_init() indicate?
A: A return value of false indicates that the initialization failed, typically due to timeout during entry into init mode, invalid baudrate calculation, or invalid parameter values such as an unsupported can_num. The function does not return true unless all steps complete successfully.

Error/Edge case conf:0.90 **Q:** What are common mistakes when calling ca_can_hw_init()?
A: Common mistakes include passing a null pointer for the config structure, using an unsupported can_num (e.g., 0 or 3), specifying an invalid baudrate that cannot be calculated, or calling the function before enabling the required peripheral clocks. Also, not checking the return value can lead to assuming initialization succeeded when it did not.

Behavior conf:0.90 **Q:** Is ca_can_hw_init() blocking or non-blocking, and is it safe to call from an ISR?
A: ca_can_hw_init() is a blocking function that waits for the CAN controller to enter init mode and complete configuration. It is not safe to call from an ISR because it may block and could interfere with real-time behavior. It is thread-safe as long as no other threads are modifying the same CAN controller registers.

HW differences conf:0.85 **Q:** How does ca_can_hw_init() differ between bxCAN and FDCAN-based MCUs?
A: On bxCAN-based MCUs (H5xx, H7xx), ca_can_hw_init() configures registers like NBTP, CCCR, and RXGFC for FDCAN-compatible settings. On FDCAN-based MCUs (H7xx), it uses FDCAN-specific registers like SIDFC, XIDFC, GFC, RXF0C, and TXBC. The FDCAN implementations also support more advanced features like dedicated FIFOs

and message RAM layouts, while bxCAN uses simpler register sets.

---

ca_can_hw_recv

Header: caos/drivers/inc/ca_glue_hw.h:71
GUARD: EN_CA_GLUE_HW

```
extern bool ca_can_hw_recv (u8 num, ca_can_frame_t *frame); /* legge un
    frame dal FIFO di ricezione; false se vuoto */
```

Reads a single CAN frame from the hardware receive FIFO buffer.

Parametri:

Nome	Tipo	Descrizione
num	u8	The index of the CAN peripheral to read from (e.g., 0, 1). Must correspond to a valid, initialized hardware instance.
frame	ca_can_frame_t *	Pointer to a structure where the received frame data (ID, DLC, data bytes, etc.) will be stored. Must not be NULL.

Ritorna:
Returns true if a frame was successfully read from the FIFO. Returns false if the FIFO is empty or if the peripheral index is invalid.

Note:

- This function is non-blocking; it checks the hardware status register immediately and returns false if no data is available, making it ideal for polling loops. It is generally thread-safe regarding the hardware registers, but you must ensure that the same CAN peripheral is not being accessed simultaneously by multiple tasks or an ISR. A common beginner mistake is calling this function without checking the return value, which leads to processing uninitialized or stale data in the 'frame' structure. Always ensure the CAN peripheral has been properly initialized before calling this function, otherwise, the hardware access may trigger a hard fault.

Uso:

- This function is typically used in a polling loop or inside a CAN receive interrupt service routine (ISR) to drain the hardware FIFO. It is usually paired with a check for pending messages to avoid unnecessary processing. In a typical application, you would call this function repeatedly until it returns false to ensure all queued messages are cleared from the hardware buffer, preventing FIFO overflow conditions.

Esempio:

```
ca_can_frame_t rx_frame;
// Poll the CAN peripheral 0 for incoming messages
if (ca_can_hw_recv(0, &rx_frame)) {
    // Successfully received a frame, process the data
    if (rx_frame.id == 0x123) {
        // Handle specific message ID
    }
} else {
    // No frames available in the hardware FIFO
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_can_hw_recv in my CAN application?
A: You should call ca_can_hw_recv to read incoming CAN frames from the hardware FIFO buffer. It's typically used in a loop or interrupt handler to process received messages. Since it's non-blocking, you can call it periodically to check for new frames without waiting.

Code example conf:0.95 **Q:** Can you show a minimal example of how to use ca_can_hw_recv?
A: ca_can_frame_t frame; bool result = ca_can_hw_recv(1, &frame); if (result) { // Process the received frame // frame.id, frame.dlc, frame.data contain the frame data }

Parameters conf:0.95 **Q:** What does the 'num' parameter represent and what values are valid?
A: The 'num' parameter specifies the CAN channel number. For f1xx, f4xx, and f7xx families, only 1 is supported. For h5xx and h7xx families, both 1 and 2 are supported. Any other value returns false immediately.

Return conf:0.95 **Q:** What do the return values of ca_can_hw_recv mean?
A: Returns true if a frame was successfully read from the FIFO buffer. Returns false if the FIFO is empty or if the channel number 'num' is invalid.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_can_hw_recv?
A: Common mistakes include passing a NULL pointer for the frame parameter (causes crash), using an unsupported channel number (returns false), and calling the function without first initializing the CAN peripheral and configuring the FIFO buffer.

Behavior conf:0.95 **Q:** Is ca_can_hw_recv blocking or thread-safe?
A: ca_can_hw_recv is non-blocking and does not wait for frames. It returns immediately. It is also not thread-safe, so concurrent calls from multiple threads may cause data corruption. It can be safely called from ISRs, but care must be taken to avoid interference with other CAN operations.

HW differences conf:0.90 **Q:** How does ca_can_hw_recv differ between bxCAN and FDCAN implementations?
A: For bxCAN (f1xx, f4xx, f7xx), the function reads from CAN_TypeDef->sFIFOMailBox[0] and uses CAN_RF0R register to check FIFO status and release frames. For FDCAN (h5xx, h7xx), it reads from FDCAN_GlobalTypeDef->RXF0S and uses FDCAN_RXF0A to acknowledge frames, with different register offsets and memory layout.

`ca_can_hw_send`

Header: `caos/drivers/inc/ca_glue_hw.h:70`
GUARD: `EN_CA_GLUE_HW`

```
1 extern bool ca_can_hw_send (u8 num, const ca_can_frame_t *frame); /*
     trasmette un frame CAN; false se mailbox piena */
```

Transmits a CAN frame over the specified hardware interface and waits for completion.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The index of the CAN peripheral to use (e.g., 0, 1). Must correspond to a valid, initialized hardware controller.
`frame`	`const ca_can_frame_t *`	Pointer to the structure containing the CAN ID, DLC, data bytes, and flags (IDE/RTR). Must not be NULL.

Ritorna:
Returns true if the frame was successfully transmitted (or acknowledged by the hardware). Returns false if the mailbox is full, the peripheral index is invalid, or the transmission timed

out.

Note:

- This function is blocking; it enters a busy-wait loop until the hardware confirms the transmission or a timeout occurs. It is NOT thread-safe if multiple threads access the same CAN peripheral simultaneously without external synchronization (like a mutex). Do not call this function from an Interrupt Service Routine (ISR) because the blocking loop will stall the system and potentially cause a watchdog reset. The most common beginner mistake is assuming this function is non-blocking or calling it from an ISR, which leads to system hangs or missed interrupts.

Uso:

- This function is typically used in the application layer or CLI commands to send data packets over the CAN bus. It is usually preceded by the initialization of the CAN peripheral and the population of a ca_can_frame_t structure. It is often paired with logic that checks the return value to handle bus congestion or hardware errors, ensuring the system can recover if the CAN bus is saturated or disconnected.

Esempio:

```
ca_can_frame_t my_frame = {0}; my_frame.id = 0x123; my_frame.dlc = 2; my_frame.data[0] = 0xAA; my_frame.data[1] = 0xBB; if (!ca_can_hw_send(0, &my_frame)) { /* Handle error: bus might be busy or disconnected */ } else { /* Transmission successful */ }
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_can_hw_send() in my caOS application?
A: You should use ca_can_hw_send() when you need to transmit a CAN frame from your application. It is typically called from a thread context when you have a valid CAN frame to send. The function is designed for blocking transmission, so it's suitable for applications where you want to ensure the frame is sent before proceeding.

Code example conf:0.95 **Q:** Can you show a minimal example of how to use ca_can_hw_send()?
A:

```
ca_can_frame_t frame;
frame.id = 0x123;
frame.ide = false;
frame.rtr = false;
frame.dlc = 8;
for (int i = 0; i < 8; i++) frame.data[i] = i;
bool success = ca_can_hw_send(1, &frame);
if (!success) {
    // Handle transmission failure
}
```

Parameters conf:0.95 **Q:** What does the 'num' parameter in ca_can_hw_send() represent and what values are valid?
A: The 'num' parameter represents the CAN channel number. Only value 1 is supported; passing any other value will cause the function to return false immediately. This is because caOS currently only supports one CAN channel per device.

Return conf:0.95 **Q:** What do the return values of ca_can_hw_send() indicate?
A: The function returns true if the frame was successfully transmitted via a free mailbox. It returns false if the mailbox is full, the timeout expired, or if the CAN channel is invalid (num != 1). The function blocks until transmission completes or timeout occurs.

Error/Edge case conf:0.90 **Q:** What are common mistakes when calling ca_can_hw_send()?
A: Common mistakes include calling with num != 1, passing a NULL frame pointer, calling from an ISR (which can cause indefinite blocking), or not ensuring the CAN channel is properly initialized before calling. Also, calling from multiple threads without synchronization can lead to race conditions.

Behavior conf:0.95 **Q:** Is ca_can_hw_send() thread-safe and ISR-safe?
A: No, ca_can_hw_send() is neither thread-safe nor ISR-safe. It is a blocking function that can cause indefinite blocking if called from an ISR, and concurrent calls from multiple threads can lead to race conditions due to shared mailbox access.

HW differences conf:0.85 **Q:** How does ca_can_hw_send() differ between bxCAN and FDCAN implementations?
A: For bxCAN (H7/H7xx), it uses mailbox registers like TXBAR and TXBTO with 32-bit mailbox indices. For FDCAN (H7/H7xx, H7xx), it uses TXFQS for FIFO status and TXBAR for transmission requests, with different memory layout in SRAMCAN for frame elements. FDCAN also has a TX FIFO full check using TFQF_Msk.

`ca_can_init`

Header: `caos/drivers/inc/can.h:39`
Source: `caos/drivers/src/can.c:106:122`
GUARD: `EN_CAN`

```
bool ca_can_init (const ca_can_cfg_t *cfg)
```

Initializes the CAN peripheral hardware and registers the driver configuration within the caOS framework.

Parametri:

Nome	Tipo	Descrizione
`cfg`	`const ca_can_cfg_t *`	Pointer to a configuration structure containing parameters like CAN instance number, baud rate, and pin mapping. Must not be NULL.

Ritorna:
Returns true if the hardware initialization succeeded and the configuration was stored; returns false if the pointer is NULL, hardware initialization fails, or the CAN instance index is out of bounds.

Note:

- This function is non-blocking and performs low-level hardware setup. It is not thread-safe and should be called during the system initialization phase before any tasks or interrupts attempt to use the CAN bus. It must not be called from an ISR. A common beginner mistake is passing an uninitialized or NULL configuration pointer, which will immediately return false and prevent the CAN peripheral from functioning. Ensure the hardware clocks and GPIOs are configured correctly by the underlying ca_can_hw_init before calling this, as this function relies on that hardware-specific setup.

Uso:

- This function is typically called once during the system startup sequence, usually within the main initialization routine or a dedicated hardware setup task. It acts as the gateway to the CAN subsystem, pairing with later functions like ca_can_send or ca_can_receive to perform actual data transmission. If the CLI is enabled, it also registers a command-line interface node, allowing developers to debug CAN status via a terminal. Always check the return value; if it returns false, the CAN peripheral is not ready, and attempting to use it will lead to undefined behavior or system crashes.

Esempio:

```
ca_can_cfg_t my_can_cfg = { .can_num = 1, .baudrate = 500000 }; // Define config

```

```
if (ca_can_init(&my_can_cfg)) {
    // Initialization successful, safe to use CAN functions
} else {
    // Handle error: log failure or enter safe state
    while(1);
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_can_init() in a caOS application?
A: You should call ca_can_init() once during system initialization, typically at startup, to configure a CAN interface on an STM32 microcontroller. It is meant to be called only once per CAN channel to avoid concurrency issues and ensure proper hardware setup.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_can_init() with caOS types?
A: ca_can_cfg_t cfg = { .can_num = 1, .baudrate = 1000000 }; bool success = ca_can_init(&cfg);

Parameters conf:0.90 **Q:** What does the cfg parameter in ca_can_init() represent and what are valid values?
A: The cfg parameter is a pointer to a ca_can_cfg_t structure that holds configuration details for the CAN interface. It must be a valid, non-NULL pointer. The can_num field must be either 1 or 2, indicating which CAN peripheral to initialize (e.g., CAN1 or CAN2).

Return conf:0.95 **Q:** What do the return values of ca_can_init() indicate?
A: The function returns true if initialization was successful, meaning the CAN channel was properly configured. It returns false if initialization failed, which can be due to a NULL cfg pointer or an error in hardware initialization (ca_can_hw_init).

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when calling ca_can_init()?
A: Common mistakes include passing a NULL pointer to cfg, calling ca_can_init() multiple times for the same CAN channel, or failing to properly initialize the hardware (e.g., due to incorrect register settings in ca_can_hw_init).

Behavior conf:0.90 **Q:** Is ca_can_init() thread-safe, ISR-safe, and blocking?
A: ca_can_init() is not thread-safe and should not be called from an ISR. It may be blocking if ca_can_hw_init() is blocking, as it typically involves hardware register configuration on peripherals like bxCAN (CAN1/CAN2 on STM32). It is intended to be called once during system startup.

`ca_can_recv`

Header: `caos/drivers/inc/can.h:41`
Source: `caos/drivers/src/can.c:141:145`
GUARD: `EN_CAN`

```
bool ca_can_recv (u8 num, ca_can_frame_t *frame)
```

Attempts to retrieve a single CAN frame from the hardware receive buffer for a specific CAN peripheral.

Parametri:

Nome	Tipo	Descrizione
num	u8	The index of the CAN peripheral to read from (e.g., 0 for CAN1, 1 for CAN2). Must correspond to a valid, initialized hardware peripheral.
frame	ca_can_frame_t *	A pointer to a structure where the received CAN data (ID, DLC, payload) will be stored. Must not be NULL.

Ritorna:
Returns true if a frame was successfully retrieved from the hardware buffer. Returns false if no frame is available, if the pointer is NULL, or if a hardware error occurred.

Note:

- This function is non-blocking; it checks the hardware status immediately and returns false if the receive buffer is empty, rather than waiting for a message to arrive. It is generally not thread-safe if multiple tasks access the same CAN peripheral simultaneously without external synchronization (like a mutex). Beginners often mistake this for a blocking function that waits for data; if you need to wait for a message, you must implement a polling loop or use an interrupt-driven approach. Ensure the CAN peripheral has been properly initialized via the driver's init function before calling this, otherwise, the hardware access may cause a system fault.

Uso:

- This function is typically used inside a periodic task or a dedicated CAN processing thread to poll for incoming messages. It is usually paired with a check for the peripheral's status or called within a loop that processes all pending frames until the buffer is empty. In a standard lifecycle, you initialize the CAN peripheral, enter a loop where you call this function to check for data, process the frame if true, and finally deinitialize the peripheral when the application shuts down.

Esempio:

```
ca_can_frame_t rx_frame;
// Poll the CAN1 peripheral for incoming messages
if (ca_can_recv(0, &rx_frame)) {
    // Successfully received a frame, process the data
    // rx_frame.id contains the message ID
    // rx_frame.data contains the payload
} else {
    // No frame available in the hardware buffer at this moment
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_can_recv to receive CAN frames in a caOS application?
A: You should use ca_can_recv when you need to receive a CAN frame from a specific CAN interface in a blocking manner. It is suitable for applications where you want to wait until a frame is received, such as in a task that processes incoming CAN messages sequentially. The function is not suitable for use in ISRs because it is blocking and may cause scheduling issues.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_can_recv with caOS types?
A: ca_can_frame_t frame; bool result = ca_can_recv(0, &frame); if (result) { // Process the received frame } else { // Handle error }

Parameters conf:0.85 **Q:** What are the valid values for the 'num' parameter in ca_can_recv, and what happens if an invalid value is passed?

A: The 'num' parameter must be a valid CAN interface number, ranging from 0 up to the maximum number of CAN interfaces available in the system. If an invalid value is passed (e.g., greater than the maximum supported interface), the function will return false and will not attempt to read from any interface. The parameter must be within the range supported by the hardware (e.g., bxCAN supports up to 2 interfaces on STM32F4 series).

Return conf:0.90 **Q:** What does it mean when ca_can_recv returns false, and what are the possible causes?
A: ca_can_recv returns false when it fails to receive a frame. This can happen if the frame parameter is NULL, the CAN interface number is invalid, or if the hardware fails to deliver a frame (e.g., timeout or hardware error). It also returns false if the function is called from an ISR, as it is not designed to be called from interrupt context.

`ca_can_retune`

Header: `caos/drivers/inc/can.h:49`
Source: `caos/drivers/src/can.c:124:133`
GUARD: `EN_CAN`

```
void ca_can_retune(void)
```

Reinitializes CAN hardware peripherals to match new system clock settings.

Ritorna:
void

Note:

- This function is non-blocking and performs a hardware-level reset of the CAN controllers. It is not thread-safe and must not be called from an ISR, as it performs hardware initialization sequences that may take significant time. The primary precondition is that the CAN hardware must have been previously initialized via the standard driver startup sequence. A common beginner mistake is calling this function while CAN messages are actively being transmitted or received; since it re-initializes the hardware registers, any ongoing communication will be abruptly terminated, potentially leading to bus errors or lost frames.

Uso:

- This function is used exclusively during power management transitions, specifically when the system clock frequency is scaled up or down to save power. Because the CAN bit timing registers are derived from the peripheral clock (PCLK1), changing the system frequency invalidates the existing baud rate settings. It is typically called in a sequence alongside other 'retune' functions like ca_os_tick_retune and ca_uart_retune immediately after the system clock frequency has been modified by the power management module.

Esempio:

```
/* Example: Scaling system frequency down for power saving */
void adjust_system_power(void) {
    /* 1. Reduce system clock frequency */
    ca_os_clock_reduce_freq(SCALE_LOW);

    /* 2. Update time-dependent peripherals to match new clock */
    ca_os_tick_retune();
    ca_uart_retune();

    /* 3. Re-initialize CAN hardware to maintain correct baud rate */
```

```
    ca_can_retune();

    /* System is now running at lower frequency with synchronized peripherals */
}
```

Q&A — fine-tuning review *2/2 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_can_retune and what is the primary reason for doing so?
A: You should call ca_can_retune during power management transitions, specifically after the system clock frequency has been modified. This is necessary because the CAN bit timing registers are derived from the peripheral clock (PCLK1), and changing the system frequency invalidates the existing baud rate settings.

Code example conf:0.95 **Q:** How do I correctly integrate ca_can_retune into a system clock frequency change sequence?
A: You must ensure no CAN messages are being transmitted or received before calling the function. A typical sequence looks like this:
void update_system_clock(u32 new_freq) { /* 1. Modify system clock frequency */ apply_clock_settings(new_freq);
/* 2. Retune peripherals */ ca_os_tick_retune(); ca_uart_retune(); ca_can_retune();
}

`ca_can_send`

Header: `caos/drivers/inc/can.h:40`
Source: `caos/drivers/src/can.c:135:139`
GUARD: `EN_CAN`

```
bool ca_can_send (u8 num, const ca_can_frame_t *frame)
```

Transmits a CAN frame through the specified hardware peripheral interface.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The index of the CAN peripheral to use. Must correspond to a valid, initialized hardware controller index.
`frame`	`const ca_can_frame_t *`	Pointer to the structure containing the CAN message data (ID, DLC, payload). Must not be NULL.

Ritorna:
Returns true if the frame was successfully queued for transmission, false if the pointer is NULL or the hardware failed to accept the frame.

Note:

- This function is a wrapper for the hardware-specific transmission routine. It is generally non-blocking, meaning it attempts to place the frame into the hardware transmit mailbox and returns immediately. It is not inherently thread-safe if multiple tasks access the same CAN peripheral simultaneously, so you should use a mutex or semaphore if your application architecture allows concurrent access. A common beginner mistake is passing a pointer to a local stack variable that goes out of scope before the hardware finishes sending, or failing to initialize the CAN peripheral via ca_can_hw_init before calling this function.

Uso:

- This function is used in the application layer to send CAN messages to external devices or other microcontrollers on the bus. It is typically used after the system has successfully initialized the CAN hardware using ca_can_hw_init. In a standard workflow, you prepare a ca_can_frame_t structure, populate its fields, and pass it to this function. It is often paired with ca_can_recv to implement full-duplex communication protocols.

Esempio:

```
ca_can_frame_t msg;
msg.id = 0x123;
msg.dlc = 8;
for(int i = 0; i < 8; i++) msg.data[i] = i;

// Attempt to send the frame on CAN peripheral 0
if (ca_can_send(0, &msg)) {
    // Frame successfully queued
} else {
    // Handle error: hardware busy or invalid parameters
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_can_send and why is it important to check its return value?
A: You should use ca_can_send to transmit a CAN frame on a specific CAN interface (0 or 1). It is important to check the return value because a false return indicates either a NULL frame pointer or a failure to send the frame, which could be due to hardware issues like a full buffer or an uninitialized interface. This helps in debugging and ensuring reliable communication.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_can_send with caOS types?
A: ca_can_frame_t frame; frame.id = 0x123; frame.len = 8; frame.data[0] = 0xAA; frame.data[1] = 0xBB; // ... fill data bool success = ca_can_send(0, &frame); if (!success) { // Handle send failure }

Parameters conf:0.90 **Q:** What are the valid values for the 'num' parameter in ca_can_send and what happens if an invalid value is passed?
A: The 'num' parameter must be either 0 or 1, representing the CAN interface number. If any other value is passed, the behavior is undefined. The function assumes that only valid interface numbers are used, and passing an invalid value may lead to incorrect hardware access or system instability.

Return conf:0.95 **Q:** What does it mean when ca_can_send returns false and what are the possible causes?
A: ca_can_send returns false if the frame pointer is NULL or if the hardware send operation fails. Possible causes include a NULL frame pointer, an uninitialized or disabled CAN interface, a full hardware transmit buffer, or other hardware-level issues. The function does not block, so the failure may be due to resource unavailability at the time of call.

`ca_caos_config_hash`

Header: `caos/inc/caos_info.h:351`
Source: `caos/src/caos.c:437:445`
GUARD: `EN_CAOS_INFO`

```
void ca_caos_config_hash(u8 *digest)
```

Calculates a SHA-256 hash of the caOS configuration, excluding the volatile build version field.

Parametri:

Nome	Tipo	Descrizione
`digest`	`u8 *`	Pointer to a buffer of at least 32 bytes where the resulting 32-byte SHA-256 hash will be stored. Must not be NULL.

Ritorna:
This function does not return a value (void). The hash is written directly into the provided 'digest' buffer.

Note:

- This function is non-blocking and thread-safe, as it performs a read-only operation on a static configuration structure. It is safe to call from an ISR because it performs no I/O or synchronization primitives. The function specifically skips the 'build_ver' field at the start of the configuration struct to ensure the hash remains deterministic across different builds. The most common mistake is providing a buffer smaller than 32 bytes, which will lead to memory corruption, or passing a NULL pointer, which will trigger a hard fault.

Uso:

- This function is used to generate a unique fingerprint of the system configuration for integrity checks or version tracking. It is typically called when the system needs to verify that the current configuration matches a known state, such as during a boot-time self-test or when requested by a diagnostic CLI command. It is often paired with hex-encoding logic to display the hash to the user via serial output or stored in non-volatile memory for later comparison.

Esempio:

```
void display_config_integrity(void) {
    /* SHA-256 produces 32 bytes */
    u8 dg[32];

    /* Calculate the hash of the current system configuration */
    ca_caos_config_hash(dg);

    /* Print the hash to the console using the standard hexdump utility */
    ca_printf("Config Hash: ");
    ca_hexdump(dg, 32);
    ca_printf("\n");
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_caos_config_hash in my application?
A: You should use this function for system integrity checks. It is ideal for verifying if the current configuration matches a known state, generating a unique fingerprint for device settings, or detecting unauthorized modifications to the configuration structure.

Code example conf:1.00 **Q:** How do I implement a call to ca_caos_config_hash in C?
A: Ensure you provide a buffer of at least 32 bytes. Example: u8 hash_buffer[32]; ca_caos_config_hash(hash_buffer);

Parameters conf:1.00 **Q:** What are the requirements for the 'digest' parameter in ca_caos_config_hash?
A: The 'digest' parameter must be a pointer to a buffer of at least 32 bytes (CA_CONFIG_HASH_LEN). Providing a buffer smaller than 32 bytes is invalid and will cause a memory corruption (buffer overflow) when the function writes the SHA-256 hash.

ca_caos_get_config

Header: caos/inc/caos_info.h:327
Source: caos/src/caos.c:432:434
GUARD: EN_CAOS_INFO

```
ca_caos_config_status *ca_caos_get_config(void)
```

Retrieves a pointer to the global system configuration structure.

Ritorna:
A pointer to a ca_caos_config_status structure. This pointer is always valid as it points to a static global configuration object defined within the OS core.

Note:

- This function is thread-safe and non-blocking because it simply returns the address of a pre-allocated static structure. It is safe to call from both tasks and ISRs. The configuration is typically populated during the system boot sequence before the scheduler starts. A common mistake is attempting to modify the configuration values at runtime without verifying if the specific fields are intended to be read-only or dynamic; always check the structure definition to ensure you are not violating system constraints.

Uso:

- This function is used when a task or driver needs to access system-wide parameters, such as network buffer sizes or default port numbers, to initialize local resources. It is typically called during the initialization phase of a module, often paired with memory allocation functions like ca_memory_allocate to size buffers based on the retrieved configuration. By using this function, you ensure that your module remains synchronized with the global system settings defined in the main configuration.

Esempio:

```
void init_network_module(void) {
    // Retrieve the global configuration to check buffer requirements
    ca_caos_config_status *cfg = ca_caos_get_config();

    // Use the retrieved configuration to allocate a buffer for network tasks
    void *net_buffer = ca_memory_allocate(cfg->http_req_buf_size);

    if (net_buffer != NULL) {
        // Proceed with module initialization using the allocated memory
        // ...
    } else {
        // Handle allocation failure
        ca_printf("Failed to allocate network buffer of size %d\n", cfg->
    http_req_buf_size);
    }

    // Example of using the config hash for integrity verification
    u8 hash[32];
    ca_caos_config_hash(hash);
    ca_printf("Config hash computed successfully\n");
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_caos_get_config and what is its primary purpose?
A: You should call this function when a task or driver needs to access system-wide parameters, such as network buffer sizes or default port numbers, to initialize local resources. It

is typically used during the initialization phase of a module to ensure that local resources remain synchronized with the global system settings defined in the main configuration.

Code example conf:0.90 **Q:** How do I access the system configuration to determine a buffer size?
A: You can retrieve the configuration pointer and access its members directly. Example: ca_caos_config_status *cfg = ca_caos_get_config(); u32 buffer_size = cfg->network_buffer_size; void *ptr = ca_memory_allocate(buffer_size);

Return conf:1.00 **Q:** What does the return value of ca_caos_get_config represent, and can it return NULL?
A: The function returns a pointer to a ca_caos_config_status structure. This pointer is always valid as it points to a static global configuration object defined within the OS core; therefore, it cannot return NULL or fail.

`ca_ccm_get_free`

Header: `caos/inc/scheduler.h:70`
Source: `caos/src/scheduler.c:292:294`
GUARD: `EN_SCHEDULER`

```
u32 ca_ccm_get_free(void)
```

Returns the amount of free stack space available for the current task in bytes.

Ritorna:
A u32 value representing the number of bytes remaining in the current task's stack before it hits the limit. If the stack is exhausted or corrupted, this value may be zero or misleading.

Note:

- This function is non-blocking and thread-safe as it reads internal task control block (TCB) variables. It is safe to call from an ISR, though it only provides information about the currently executing task context. The most common beginner mistake is assuming this value is static; stack usage changes dynamically as functions are called and local variables are allocated. Always ensure the system is initialized before calling this, as it relies on the internal CCM (Core Coupled Memory) management structures being active.

Uso:

- This function is typically used for system monitoring, debugging, or implementing stack overflow protection mechanisms. It is often paired with ca_ccm_get_used to calculate the total stack footprint of a task. Developers use these functions during the development phase to tune stack sizes for specific tasks, ensuring that the allocated memory is sufficient for the worst-case call stack depth without wasting precious RAM.

Esempio:

```
/* Monitor stack usage for the current task */
#include "caos.h"

void task_monitor_stack(void) {
    u32 free_bytes = ca_ccm_get_free();
    u32 used_bytes = ca_ccm_get_used();

    /* Log the stack status to the console */
    ca_printf("Stack Status: Used %u bytes, Free %u bytes\n", used_bytes,
    free_bytes);

    /* Check if we are dangerously close to a stack overflow */
    if (free_bytes < 128) {
```

```
        /* Trigger a system warning or take corrective action */
        ca_printf("Warning: Low stack space!\n");
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_ccm_get_free and what is the primary purpose of this function?
A: You should use ca_ccm_get_free during the development phase for system monitoring, debugging, or implementing stack overflow protection. It is used to tune task stack sizes to ensure allocated memory is sufficient for worst-case call stack depth without wasting RAM. It is often paired with ca_ccm_get_used to calculate the total stack footprint.

Code example conf:0.95 **Q:** How do I implement a simple check for the current task's remaining stack space using caOS types?
A: You can call the function directly within your task logic. Example: { u32 free_space = ca_ccm_get_free(); if (free_space < 128) { /* Handle low stack warning */ } }

Return conf:1.00 **Q:** What does the u32 return value represent, and are there scenarios where the returned value might be unreliable?
A: The return value is a u32 representing the number of bytes remaining in the current task's stack. If the stack is exhausted or corrupted, the value may be zero or misleading. Additionally, the value is dynamic and changes as functions are called and local variables are allocated, so it should not be treated as a static value.

`ca_ccm_get_used`

Header: `caos/inc/scheduler.h:75`
Source: `caos/src/scheduler.c:296:298`
GUARD: `EN_SCHEDULER`

```
u32 ca_ccm_get_used(void)
```

Returns the number of bytes currently used in the CCM (Core Coupled Memory) stack.

Ritorna:
Returns a u32 representing the total number of bytes currently occupied by the stack within the CCM memory region.

Note:

- This function is non-blocking and thread-safe as it performs a simple arithmetic subtraction on internal memory management structures. It is safe to call from an ISR, as it does not involve any OS primitives, mutexes, or complex logic. The function assumes that the CCM memory management system has been correctly initialized by the system startup code. A common mistake is assuming this value represents the 'peak' usage; it only reflects the current stack pointer position relative to the base address, not the maximum depth reached during execution.

Uso:

- This function is primarily used for system diagnostics, debugging, and monitoring stack health. It is often paired with ca_ccm_get_free to calculate the total stack size or to verify if a task is approaching a stack overflow condition. Developers typically use this during the development phase to tune stack sizes for specific tasks, ensuring that the allocated memory is sufficient without being excessively wasteful. It is also used in logging routines to report memory pressure to a CLI or remote monitoring interface.

Esempio:

```c
/* Check if the current CCM stack usage is within safe limits */
u32 used_bytes = ca_ccm_get_used();
u32 free_bytes = ca_ccm_get_free();

/* Log the memory status to the system console */
ca_printf("CCM Stack Status: Used %lu bytes, Free %lu bytes\r\n", used_bytes,
    free_bytes);

/* Simple safety check: if free memory is critically low, trigger a warning */
if (free_bytes < 128) {
    /* Perform emergency action or log a critical error */
    ca_printf("CRITICAL: CCM stack is nearly exhausted!\r\n");
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_ccm_get_used in my application?
A: You should use this function during development for system diagnostics, debugging, and monitoring stack health. It is particularly useful for tuning task stack sizes to ensure they are sufficient without being wasteful, or for logging memory pressure to a CLI or remote monitoring interface to detect potential stack overflow conditions.

Code example conf:1.00 **Q:** How can I check the current CCM stack usage in my code?
A: You can call the function directly as shown here: u32 used_bytes = ca_ccm_get_used();

Return conf:1.00 **Q:** What does the u32 return value represent, and can this function fail?
A: The return value is a u32 representing the total number of bytes currently occupied by the stack within the CCM memory region. It does not represent the peak usage, only the current stack pointer position relative to the base address. The function is non-blocking, thread-safe, and safe to call from an ISR; it does not fail, provided the CCM memory management system was correctly initialized at startup.

`ca_cli_callto_init`

Header: `caos/inc/callto.h:40`
Source: `caos/src/callto.c:246:251`
GUARD: `EN_CALLTO`

```c
void ca_cli_callto_init(void)
```

Registers the 'callto' command handler into the system CLI interface.

Ritorna:
This function returns nothing (void).

Note:

- This function is non-blocking and thread-safe because it uses an internal static flag to ensure the registration happens exactly once. It is not intended to be called from an ISR, as it performs memory registration operations that should occur during system initialization. The most common mistake is calling this function before the CLI system itself has been initialized via ca_cli_init, which will result in the registration failing or causing a system fault. Always ensure the CLI subsystem is ready before attempting to register specific command nodes.

Uso:

- This function is used to extend the Command Line Interface (CLI) functionality by adding the 'callto' command. It is typically called during the system startup sequence, often

wrapped in a macro like EVAL_CLI_CALLTO to ensure it only executes if the feature is enabled in the project configuration. It pairs with ca_cli_init, which must be called first to prepare the CLI infrastructure, and is followed by ca_cli_create_thread to start the CLI processing task.

Esempio:

```
/* Initialize the core CLI system first */
ca_cli_init();

/* Register the callto command handler if enabled in configuration */
ca_cli_callto_init();

/* Create the CLI thread to start processing user input */
ca_cli_create_thread();

/* Now the CLI is ready to accept commands, including 'callto' */
```

Q&A — fine-tuning review *2/2 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_cli_callto_init and what are the prerequisites?
A: You should call ca_cli_callto_init during the system startup sequence to register the 'callto' command. It is critical that you call ca_cli_init first to prepare the CLI infrastructure; calling it before the CLI system is initialized will cause the registration to fail or trigger a system fault. It is not intended for use within an ISR.

Code example conf:0.90 **Q:** How do I correctly initialize the CLI and register the 'callto' command in my main startup routine?
A: You must initialize the CLI subsystem before registering the command. Example: void system_init(void) { ca_cli_init(); EVAL_CLI_CALLTO; ca_cli_create_thread(); }

`ca_cli_char_class`

Header: `caos/inc/cli.h:137`
Source: `caos/src/cli.c:1316:1319`
GUARD: `EN_CLI`

```
u8 ca_cli_char_class(u8 byte)
```

Classifies a single byte into a command-line category (e.g., command, typo, escape, erase) based on the internal CLI character table.

Parametri:

Nome	Tipo	Descrizione
`byte`	`u8`	The raw ASCII or byte value to classify. Valid range is 0-255, though only 0-127 are mapped to specific classes.

Ritorna:
Returns a u8 representing the character class (e.g., CH_CMD, CH_TYPO, CH_ERASE, CH_ESC). Returns CH_NONE if the byte is >= 128.

Note:

- This function is a non-blocking, thread-safe lookup table access. It is safe to call from an ISR because it performs no I/O, locking, or complex logic. The primary precondition is that the caOS CLI system must be initialized, though the table itself is static and always available. A common mistake is assuming this function validates input; it only classifies it, so you must still handle unexpected characters or non-printable bytes in your application

logic. Because it is a simple array lookup, it is extremely fast and deterministic, making it ideal for high-frequency character processing.

Uso:

- This function is used to build robust command-line interfaces or network transport parsers that need to distinguish between valid command characters, control sequences, and typos. It is typically used inside a character processing loop where you receive a byte via ca_cli_rx_byte or a UART handler, classify it to determine if it should be buffered or trigger an action (like an escape sequence), and then potentially update the CLI state machine. It ensures that different parts of the system, such as a local console and an SSH transport, share the exact same rules for what constitutes a valid command, preventing inconsistent behavior between different input sources.

Esempio:

```c
/* Example: Processing an incoming byte from a CLI stream */
void process_cli_input(u8 c) {
    u8 class = ca_cli_char_class(c);

    if (class == CH_ERASE) {
        /* Logic to remove last character from buffer */
        /* Use ca_put to send backspace sequence to terminal */
        ca_put("\b \b", 3);
    } else if (class == CH_ESC) {
        /* Transition CLI state machine to escape mode */
    } else if (class == CH_CMD) {
        /* Add character to command buffer */
        ca_cli_hist_add(c);
    } else if (class == CH_TYPO) {
        /* Increment internal typo counter */
        /* Use ca_cli_typo_count to track statistics */
    } else {
        /* Ignore or handle CH_NONE */
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_cli_char_class in my application?
A: You should use it when building command-line interfaces or network transport parsers to distinguish between valid command characters, control sequences, and typos. It is typically used within a character processing loop to decide whether a received byte should be buffered or trigger a specific action, such as an escape sequence.

Code example conf:0.90 **Q:** How do I implement a basic character classification check using caOS types?
A: u8 input = 0x0D; u8 class = ca_cli_char_class(input); if (class == CH_CMD) { /* handle command */ }

Parameters conf:1.00 **Q:** What are the constraints for the byte parameter in ca_cli_char_class?
A: The parameter is a u8 representing the raw ASCII or byte value. While the valid range is 0-255, only values from 0-127 are mapped to specific classes; values >= 128 will result in a CH_NONE return.

Return conf:1.00 **Q:** What do the return values of ca_cli_char_class represent and when does it return CH_NONE?
A: The function returns a u8 representing the character class, such as CH_CMD, CH_TYPO, CH_ERASE, or CH_ESC. It returns CH_NONE if the input byte is 128 or greater, as these values are outside the mapped range.

ca_cli_create_thread

Header: caos/inc/cli.h:82
Source: caos/src/cli.c:1185:1201
GUARD: EN_CLI

```
u32 ca_cli_create_thread(void)
```

Initializes and spawns the Command Line Interface (CLI) background task within the RTOS.

Ritorna:
Returns a u32 task index (handle) representing the created thread, or 0xFFFFFFFF if the CLI module is disabled or the thread creation fails.

Note:

- This function is non-blocking as it merely requests the RTOS scheduler to allocate resources and start the task. It is not thread-safe to call this from an ISR, and it must only be called after the CLI module has been initialized via ca_cli_init. A common beginner mistake is calling this function before initializing the CLI buffers or setting the output sender, which leads to undefined behavior or immediate task crashes. Ensure that the system has enough stack memory available, as the task is created with STACK_MEDIUM size.

Uso:

- This function is typically used during the system startup phase, usually within the main initialization sequence or a dedicated system configuration task. It is always paired with ca_cli_init to prepare the environment and ca_cli_set_sender to define where the CLI output should be directed. Once called, the CLI thread runs in the background at a low priority (CA_TH_LEVEL3), allowing the system to process user commands asynchronously without blocking critical real-time tasks.

Esempio:

```
/* Initialize the CLI environment before spawning the thread */
ca_cli_init(4096);

/* Define the output destination for the CLI */
ca_cli_set_sender(STDOUT);

/* Spawn the background thread to handle user input */
u32 cli_handle = ca_cli_create_thread();

if (cli_handle == 0xFFFFFFFF) {
    /* Handle error: CLI could not be started */
    while(1);
}

/* Register the CLI command node after the thread is created */
ca_cli_register(&s_cli_node);
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_cli_create_thread() in my caOS application?
A: You should call ca_cli_create_thread() after initializing the CLI subsystem. It creates a low-priority thread to handle command-line interface tasks. The function is meant to be called once during application initialization, not repeatedly.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_cli_create_thread() in a caOS application?

A: u32 thread_id = ca_cli_create_thread(); if (thread_id != 0xFFFFFFFF) { // Thread created successfully } else { // Error occurred, CLI not enabled or thread creation failed }

Return conf:0.95 **Q:** What does it mean when ca_cli_create_thread() returns 0xFFFFFFFF?
A: A return value of 0xFFFFFFFF indicates an error occurred during thread creation. Common causes include the CLI not being enabled (g_enabled is false) or a failure in the thread creation process itself.

Error/Edge case conf:0.85 **Q:** What are the most common mistakes when using ca_cli_create_thread()?
A: Common mistakes include calling the function before CLI initialization, calling it from an ISR (since it's not ISR-safe), or calling it multiple times which may lead to undefined behavior. Also, calling it in a preemption-critical section can cause issues.

Behavior conf:0.90 **Q:** Is ca_cli_create_thread() blocking or non-blocking, and is it safe to call from an ISR?
A: ca_cli_create_thread() is a blocking function and must not be called from an ISR. It is also not thread-safe or preemption-safe, so it should not be invoked in critical sections or during interrupt handling. The function is designed to be called during normal application initialization.

`ca_cli_data_from_isr`

Header: `caos/inc/cli.h:198`
Source: `caos/src/cli.c:1037:1059`
GUARD: `EN_CLI`

```
1 void ca_cli_data_from_isr(const u8 *data, u32 len)
```

Safely passes incoming data from an Interrupt Service Routine (ISR) to the CLI processing queue.

Parametri:

Nome	Tipo	Descrizione
`data`	`const u8 *`	Pointer to the raw byte array received (e.g., from UART). Must not be NULL.
`len`	`u32`	Number of bytes to copy. If greater than the internal buffer size (125 bytes), it will be truncated.

Ritorna:
void. This function does not return a status code; if the queue is full, it increments an internal drop counter.

Note:

- This function is specifically designed to be called from within an ISR, such as a UART RX interrupt handler. It is non-blocking, meaning it will not pause the CPU if the queue is full, ensuring the interrupt remains fast and responsive. It is thread-safe regarding the queue interaction, but it uses a shared static buffer (g_isr_buffer), so it cannot be called from multiple different ISRs simultaneously without risking data corruption. The most common mistake is passing a pointer to a local stack variable that goes out of scope or failing to ensure the CLI system is initialized before the interrupt is enabled.

Uso:

- This function is the bridge between hardware peripherals and the CLI task. It is typically called inside the UART RX interrupt callback after receiving a character or a packet. It

pairs with the CLI task's main loop, which uses a function like 'ca_queue_static_receive' to pull the data out of the queue for processing. You should initialize the CLI system using 'ca_cli_init' first, then enable the UART interrupt, and finally call this function inside the ISR to feed the CLI engine.

Esempio:

```
void UART_RX_ISR_Handler(void) {
    /* Assume we read a byte from the hardware register */
    u8 rx_byte = 0; /* Read from hardware register here */

    /* Pass the byte to the CLI system safely from the ISR */
    ca_cli_data_from_isr(&rx_byte, 1);

    /* If the queue was full, the internal drop counter is incremented
    automatically */
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_cli_data_from_isr() in my embedded application?
A: You should call ca_cli_data_from_isr() from an ISR when you need to pass data from the interrupt context to the CLI task for processing. This function allows you to avoid blocking the ISR while ensuring that the CLI task can handle the data asynchronously.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_cli_data_from_isr() in an ISR?
A: u8 buffer[10]; // Assume buffer is filled with data in ISR ca_cli_data_from_isr(buffer, 10);

Parameters conf:0.95 **Q:** What are the valid values for the 'data' and 'len' parameters in ca_cli_data_from_isr()?
A: The 'data' parameter must be a valid pointer to data (not NULL). The 'len' parameter must be greater than 0. If either condition is not met, the function returns without doing anything.

Error/Edge case conf:0.85 **Q:** What happens if I call ca_cli_data_from_isr() with invalid parameters or in the wrong context?
A: If the 'data' parameter is NULL or 'len' is 0, the function returns without performing any action. If called from a task instead of an ISR, the behavior is undefined. If the internal queue is full, new data is dropped and g_dropped is incremented.

Behavior conf:0.90 **Q:** Is ca_cli_data_from_isr() blocking or non-blocking, and is it safe to call from a task?
A: The function is non-blocking and must only be called from an ISR. It is not thread-safe and should not be used by normal tasks. It is designed to be ISR-safe, but calling it from a task may lead to undefined behavior.

`ca_cli_exec_capture`

Header: `caos/inc/cli.h:234`
Source: `caos/src/cli.c:1062:1119`
GUARD: `EN_CLI`

```
bool ca_cli_exec_capture(const char *cmd, char *out_buf, u16 buf_size,
    u32 timeout_ms)
```

Executes a CLI command and captures its output into a provided buffer instead of the standard console.

Parametri:

Nome	Tipo	Descrizione
`cmd`	`const char *`	The CLI command string to execute. Must be a null-terminated string.
`out_buf`	`char *`	Pointer to the memory buffer where the command output will be stored. Must be at least 2 bytes.
`buf_size`	`u16`	Size of the output buffer in bytes. If the command output exceeds this, it will be truncated.
`timeout_ms`	`u32`	Maximum time in milliseconds to wait for the command to complete before returning failure.

Ritorna:
Returns true if the command executed and output was captured successfully within the timeout. Returns false if parameters are invalid, the CLI system is disabled, or the command timed out.

Note:

- This function is blocking; it suspends the calling thread until the command finishes or the timeout expires. It is NOT safe to call from an ISR because it uses semaphores and thread-level delays. The function internally manages the redirection of stdout, so it is not re-entrant; you cannot call this from two different threads simultaneously. A common mistake is providing a buffer that is too small for the expected output, leading to silent truncation, or setting a timeout that is too short for commands that perform heavy processing or I/O. Ensure the CLI system is initialized via ca_cli_init before calling this.

Uso:

- Use this function when you need to programmatically retrieve the result of a CLI command, such as displaying system status on a web page or logging diagnostic data to a file. It is typically used in a request-response cycle: you call it to trigger a command, wait for the result, and then process the captured string. It pairs with the CLI system's internal state management, automatically restoring the original stdout sender once the capture is complete, ensuring the system returns to its normal state even if the command fails.

Esempio:

```
char buffer[256];
/* Ensure CLI is initialized before use */
if (ca_cli_is_enabled()) {
    /* Execute 'status' command and wait up to 500ms for the result */
    if (ca_cli_exec_capture("status", buffer, sizeof(buffer), 500U)) {
        /* Successfully captured output, now process the buffer */
        ca_http_resp_puts(resp, buffer);
    } else {
        /* Handle error: command failed or timed out */
        ca_http_resp_puts(resp, "Error: CLI capture failed");
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_cli_exec_capture to execute CLI commands?
A: You should use ca_cli_exec_capture when you need to execute a CLI command and capture its output in a buffer for further processing. It is useful for obtaining textual responses from CLI commands in a controlled and predictable way.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_cli_exec_capture with caOS types?
A: char buffer[128]; bool success = ca_cli_exec_capture("help", buffer, 128, 1000); if (success) { // Use buffer content }

Parameters conf:0.90 **Q:** What are the valid and invalid values for the buf_size parameter?
A: The buf_size must be at least 2 bytes to accommodate the null terminator. If buf_size is less than 2, the function will fail. Valid values are any u16 greater than or equal to 2.

Return conf:0.85 **Q:** What does it mean when ca_cli_exec_capture returns false?
A: A return value of false indicates that the command either failed due to invalid parameters or timed out before completion. It could also mean that the CLI system was not enabled or the semaphore was not initialized.

Error/Edge case conf:0.80 **Q:** What are common mistakes when using ca_cli_exec_capture?
A: Common mistakes include passing a NULL pointer for cmd or out_buf, using a buf_size less than 2, calling the function from an ISR, or not ensuring that the CLI system is enabled and the semaphore is initialized. Also, improper handling of timeout values can lead to indefinite blocking.

Behavior conf:0.90 **Q:** Is ca_cli_exec_capture thread-safe and ISR-safe?
A: No, ca_cli_exec_capture is not thread-safe and not ISR-safe. It uses global variables such as g_exec_done, g_cap, and g_sender, which can cause race conditions in a multi-threaded environment or lead to undefined behavior if called from an ISR.

`ca_cli_exec_capture_stream`

Header: `caos/inc/cli.h:249`
Source: `caos/src/cli.c:1121:1177`
GUARD: `EN_CLI`

```
bool ca_cli_exec_capture_stream(const char *cmd, sender out_sender, u32
    timeout_ms)
```

Executes a CLI command and redirects its output stream to a specific sender until completion or timeout.

Parametri:

Nome	Tipo	Descrizione
`cmd`	`const char *`	The CLI command string to execute. Must not be NULL.
`out_sender`	`sender`	The destination sender (e.g., a network socket or UART) where the command output will be streamed. Must not be NULL.
`timeout_ms`	`u32`	Maximum time in milliseconds to wait for the command to finish and the output stream to drain.

Ritorna:
Returns true if the command executed and the output stream finished within the timeout period; returns false if the command failed, the timeout expired, or the CLI system is disabled.

Note:

- This function is blocking and must be called from a task context, never from an ISR. It is thread-safe regarding the CLI state, but users must ensure that the 'out_sender' remains valid for the duration of the call. A common mistake is failing to account for the output drain time; even if the command finishes, the function waits for the stdout queue to empty, so the timeout must be generous enough to cover both execution and transmission latency. It internally manages the CLI output redirection, so it is critical that no other task attempts to modify the CLI sender while this function is active. If the timeout expires, the function still attempts to clean up the redirection, but the caller must handle the potential partial output state.

Uso:

- This function is typically used to bridge CLI output to external interfaces like SSH or HTTP responses. The standard sequence involves: 1) Initializing a mutex to protect the session/response object, 2) Locking the mutex to assign the session, 3) Calling this function to stream the command output, and 4) Locking the mutex again after the function returns to safely clear the session pointer. This ensures that the 'drain' phase of the function completes before the session object is destroyed or reused, preventing dangling pointers and ensuring that the background drain thread has finished accessing the session context.

Esempio:

```c
/* Example: Executing a command and streaming output to a network sender */
#include "caos.h"

/* Assume s_stream_lock is initialized elsewhere */
void run_remote_command(const char *cmd, sender net_sender) {
    /* Protect the session context during the streaming operation */
    ca_mutex_lock(&s_stream_lock);
    /* Assign session context here */
    ca_mutex_unlock(&s_stream_lock);

    /* The function blocks until the command finishes and the output queue is drained */
    bool success = ca_cli_exec_capture_stream(cmd, net_sender, 5000U);

    /* Re-acquire lock to safely clear session pointer after streaming is done */
    ca_mutex_lock(&s_stream_lock);
    /* Clear session context here */
    ca_mutex_unlock(&s_stream_lock);

    if (!success) {
        ca_printf("Command failed or timed out\n");
    }
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_cli_exec_capture_stream in my caOS application?
A: You should use this function when you need to bridge CLI command output to an external interface, such as an SSH or HTTP response. It is designed to execute a command and stream its output to a specific sender until completion or timeout.

Code example conf:0.90 **Q:** Provide a minimal C example of how to call ca_cli_exec_capture_stream.
A: void run_cmd(sender my_sender) { cstr cmd = "sys_info"; u32 timeout = 5000; bool success = ca_cli_exec_capture_stream(cmd, my_sender, timeout); }

Parameters conf:1.00 **Q:** What are the requirements for the 'out_sender' parameter?
A: The 'out_sender' must not be NULL. It represents the destination for the command output, such as a network socket or UART. You must ensure this sender remains valid for the entire duration of the function call.

Return conf:1.00 **Q:** What does a return value of false indicate for ca_cli_exec_capture_stream?
A: A return value of false indicates that the command failed, the timeout period expired, or the CLI system is currently disabled.

Error/Edge case conf:1.00 **Q:** What is a common mistake when setting the timeout_ms parameter?
A: A common mistake is failing to account for the output drain time. The function waits for the stdout queue to empty after the command finishes, so the timeout must be generous enough to cover both the command execution time and the transmission latency.

Behavior conf:1.00 **Q:** Is ca_cli_exec_capture_stream safe to call from an ISR?
A: No, this function is blocking and must be called from a task context. It is never safe to call from an ISR.

---

ca_cli_filtered_count

Header: caos/inc/cli.h:110
Source: caos/src/cli.c:1474:1474
GUARD: EN_CLI

```
u32 ca_cli_filtered_count(void)
```

Returns the total number of CLI input characters that were filtered out by the system.

Ritorna:
Returns a u32 value representing the count of characters discarded by the CLI filtering logic. This value is non-negative and increments whenever the CLI subsystem rejects an incoming byte.

Note:

- This function is non-blocking and thread-safe as it simply returns the current value of a global counter. It can be safely called from both standard tasks and Interrupt Service Routines (ISRs). The primary precondition is that the CLI subsystem must have been initialized via ca_cli_init. A common mistake is assuming this count represents valid commands; it specifically tracks 'filtered' (discarded) data, which is useful for debugging noise on the serial line or identifying malformed input streams.

Uso:

- This function is typically used in diagnostic or monitoring tasks to assess the quality of the serial communication link. It is often paired with ca_cli_stats to provide a comprehensive overview of the CLI subsystem's health. Developers use it to detect if the input buffer is being flooded with unexpected characters or if the CLI parser is rejecting data due to protocol violations.

Esempio:

```
/* Check CLI health periodically */
void monitor_task(void *pvParameters) {
    /* Ensure CLI is initialized before monitoring */
    ca_cli_init();

    while (1) {
        u32 filtered = ca_cli_filtered_count();

        if (filtered > 1000) {
            /* Log a warning if too many characters are being filtered */
            ca_printf("Warning: High CLI noise detected. Filtered count: %lu\n",
    filtered);
        }

        /* Wait for 1 second before checking again */
        ca_os_delay_ms(1000);
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_cli_filtered_count in my caOS application?
A: You should use this function in diagnostic or monitoring tasks to assess the quality of your serial communication link. It is useful for identifying noise on the line, detecting if the input buffer is being flooded with unexpected characters, or determining if the CLI parser is rejecting data due to protocol violations.

Code example conf:1.00 **Q:** How do I implement a simple check for discarded CLI characters in a task?
A: You can call the function directly as shown here:
void monitor_task(void) { u32 discarded_count = ca_cli_filtered_count(); if (discarded_count > 0) { /* Handle or log the diagnostic data */ } }

Return conf:1.00 **Q:** What does the u32 return value represent, and are there any failure states?
A: The return value is a non-negative u32 representing the total count of characters discarded by the CLI filtering logic. The function does not have failure states; it is non-blocking and thread-safe. The only precondition is that ca_cli_init must have been called previously.

`ca_cli_get_hook`

Header: `caos/inc/cli.h:222`
Source: `caos/src/cli.c:966:966`
GUARD: `EN_CLI`

```
ca_cli_hook_t ca_cli_get_hook(void)
```

Retrieves the currently registered CLI hook function pointer.

Ritorna:
Returns a 'ca_cli_hook_t' function pointer. If no hook has been set via ca_cli_set_hook, it returns NULL.

Note:

- This function is non-blocking and thread-safe as it performs a simple read of a global variable. It is safe to call from an ISR, though it is rarely necessary to do so. The primary precondition is that the CLI system must be initialized, and a hook must have been previously assigned using ca_cli_set_hook; otherwise, you will receive a NULL pointer. A common mistake is failing to check if the returned pointer is NULL before attempting to call it, which will result in a hard fault or system crash.

Uso:

- This function is typically used by the internal CLI engine or custom command handlers to retrieve the current hook function that processes specific CLI events or data. It is often paired with ca_cli_set_hook during the system initialization phase to define custom behavior for the CLI. Developers use this to dynamically inspect or verify which callback is currently active in the CLI processing chain, ensuring that the system logic remains consistent with the intended CLI behavior.

Esempio:

```
/* Define a custom hook function */
void my_custom_hook(void) {
    /* Custom logic for CLI event */
}

void setup_cli_system(void) {
    /* Register the hook */
```

```
    ca_cli_set_hook(my_custom_hook);

    /* Retrieve and verify the hook */
    ca_cli_hook_t current_hook = ca_cli_get_hook();

    if (current_hook != NULL) {
        /* Hook is valid, proceed with CLI initialization */
        ca_cli_init();
    } else {
        /* Handle error: hook not set */
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_cli_get_hook in my caOS application?
A: You should use this function when you need to retrieve the currently registered CLI hook function pointer, typically to inspect or verify which callback is active in the CLI processing chain. It is commonly used by custom command handlers or the internal CLI engine to dynamically manage CLI event processing.

Code example conf:1.00 **Q:** How do I safely retrieve and execute the CLI hook in C?
A: You must check the returned pointer for NULL to avoid a hard fault. Example: ca_cli_hook_t my_hook = ca_cli_get_hook(); if (my_hook != NULL) { my_hook(); }

Return conf:1.00 **Q:** What does the return value of ca_cli_get_hook represent and what causes it to return NULL?
A: The function returns a 'ca_cli_hook_t' function pointer. It returns NULL if no hook has been previously assigned using ca_cli_set_hook or if the CLI system has not been initialized. Failing to check for this NULL return before calling the pointer will result in a system crash.

`ca_cli_get_sender`

Header: `caos/inc/cli.h:44`
Source: `caos/src/cli.c:116:116`
GUARD: `EN_CLI`

```
sender ca_cli_get_sender(void)
```

Retrieves the current sender handle configured for the CLI module.

Ritorna:
Returns a 'sender' type handle, which is an identifier used by the OS to route messages or responses back to the CLI interface.

Note:

- This function is non-blocking and thread-safe as it simply returns a stored global variable. It is safe to call from both threads and ISRs, provided the CLI module has been initialized. The most common mistake is assuming the returned sender is valid before calling ca_cli_init; if the CLI is not initialized, the returned handle may be null or invalid, leading to failed message routing in subsequent calls.

Uso:

- This function is typically used when you need to send data or status updates back to the CLI console from a background task or a callback. You retrieve the sender handle using this function and then pass it to other caOS communication functions (like message queues

or logging) to ensure the output is directed to the active CLI session. It is often paired with ca_cli_set_sender if you need to redirect CLI output to a different destination, such as a remote network socket or a different UART port.

Esempio:

```c
/* Retrieve the current CLI sender to respond to a command */
sender cli_sender = ca_cli_get_sender();

/* Check if the sender is valid before attempting to send data */
if (cli_sender != NULL) {
    /* Use the retrieved sender to send a status message back to the CLI */
    /* ca_message_send(cli_sender, "Command processed successfully\n"); */
} else {
    /* Handle the error: CLI sender not initialized or unavailable */
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When and why should I use ca_cli_get_sender in my application?
A: You should call ca_cli_get_sender when you need to send data or status updates back to the CLI console from a background task or an ISR. It retrieves the sender handle required to route messages or responses to the active CLI session, ensuring that communication functions correctly direct output to the user interface.

Code example conf:0.95 **Q:** How do I retrieve the CLI sender handle and use it in a task?
A: You can retrieve the handle and store it in a variable of type 'sender' as shown: void my_task(void) { sender cli_handle = ca_cli_get_sender(); if (cli_handle != 0) { /* Proceed to use cli_handle for routing messages */ } }

Return conf:1.00 **Q:** What does the return value of ca_cli_get_sender represent, and what causes it to be invalid?
A: The function returns a 'sender' type handle, which acts as an identifier for the OS to route messages back to the CLI interface. If the CLI module has not been initialized via ca_cli_init, the returned handle may be null or invalid, which will cause subsequent message routing calls to fail.

`ca_cli_hist_add`

Header: `caos/inc/cli.h:173`
Source: `caos/src/cli.c:1335:1345`
GUARD: `EN_CLI`

```c
void ca_cli_hist_add(const char *line)
```

Adds a command string to the CLI history buffer for future recall.

Parametri:

Nome	Tipo	Descrizione
`line`	`const char *`	Pointer to the null-terminated string to store. Must not be NULL or an empty string.

Ritorna:
void

Note:

- This function is not thread-safe; it modifies a global history buffer and index, so it should be called from a single task context or protected by a mutex if accessed concurrently. It is non-blocking and performs a simple memory copy up to the defined CLI buffer size. It

is not safe to call from an ISR because it does not implement atomic protection for the shared history indices. A common mistake is passing a pointer to a local stack variable that goes out of scope, or passing a NULL pointer, which the function handles by silently returning.

Uso:

- This function is used to populate the command history, typically after a user successfully executes a command via the CLI or an SSH session. It is usually paired with ca_cli_hist_get to retrieve previous commands when the user presses arrow keys. It is typically called after the command has been validated but before the command execution logic completes, ensuring the command is available for immediate recall. Developers should ensure that the string passed is persistent and not a temporary buffer that might be overwritten by other tasks.

Esempio:

```
void process_user_input(const char *input_buffer) {
    /* Validate the input pointer before processing */
    if (ca_valid_pointer((void *)input_buffer)) {
        /* Add the command to the CLI history for future recall */
        ca_cli_hist_add(input_buffer);

        /* Execute the command using the system CLI capture mechanism */
        /* ca_cli_exec_capture(input_buffer, 2000U); // Execute command with timeout */
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_cli_hist_add in my CLI implementation?
A: You should call it immediately after the system receives a full command line from the user, but before the command is actually executed by the CLI engine. This ensures the command is stored in the history buffer for later recall via arrow keys.

Code example conf:0.90 **Q:** How do I correctly add a command string to the history buffer using caOS types?
A: You must pass a pointer to a null-terminated string. Example: { cstr cmd = "reboot"; ca_cli_hist_add(cmd); } Ensure the string is not a local stack variable that will go out of scope.

Parameters conf:1.00 **Q:** What are the constraints for the 'line' parameter in ca_cli_hist_add?
A: The 'line' parameter must be a pointer to a null-terminated string. It cannot be NULL or an empty string. If a NULL pointer is passed, the function will safely ignore the call without reporting an error.

Error/Edge case conf:1.00 **Q:** Can I call ca_cli_hist_add from an interrupt service routine (ISR)?
A: No, you should not call it from an ISR. The function is not thread-safe and performs memory operations on a shared global buffer without locking. It must be called from the context of the CLI task or a protected execution flow.

`ca_cli_hist_count`

Header: `caos/inc/cli.h:165`
Source: `caos/src/cli.c:1333:1333`
GUARD: `EN_CLI`

```
u8 ca_cli_hist_count(void)
```

Returns the current number of commands stored in the CLI history buffer.

Ritorna:
An unsigned 8-bit integer (u8) representing the total count of command strings currently available in the history buffer. The value ranges from 0 up to the maximum configured history size.

Note:

- This function is non-blocking and thread-safe as it simply returns a cached counter value. It is safe to call from an ISR, as it performs no complex operations or resource locking. The primary precondition is that the CLI module must be initialized via ca_cli_init before calling this function. A common mistake is assuming the returned value is an index; remember that this is a count, so the valid index range for ca_cli_hist_get is 0 to (ca_cli_hist_count() - 1).

Uso:

- This function is typically used to implement command navigation features, such as 'up' and 'down' arrow keys in a terminal or SSH session. It is used to check the boundaries of the history buffer before attempting to retrieve a command with ca_cli_hist_get. Developers use it to prevent out-of-bounds access when iterating through the history stack, ensuring that the navigation pointer stays within the valid range of stored commands.

Esempio:

```
/* Example: Navigating CLI history using the count */

void navigate_history(i8 *current_nav_index, i8 direction) {
    u8 total_hist = ca_cli_hist_count();

    if (total_hist == 0) return; // Nothing to navigate

    if (direction > 0) { // Moving to older commands
        if (*current_nav_index < (i8)(total_hist - 1)) {
            (*current_nav_index)++;
            const char *cmd = ca_cli_hist_get((u8)*current_nav_index);
            if (cmd) {
                ca_printf("Recalled: %s\n", cmd);
            }
        }
    } else if (direction < 0) { // Moving to newer commands
        if (*current_nav_index > 0) {
            (*current_nav_index)--;
            const char *cmd = ca_cli_hist_get((u8)*current_nav_index);
            if (cmd) {
                ca_printf("Recalled: %s\n", cmd);
            }
        }
    }
}
```

Q&A — fine-tuning review

3/3 pairs · gemini-3.1-flash-lite

Usage conf:1.00 **Q:** When should I use ca_cli_hist_count in my CLI implementation?

A: You should use this function when implementing command navigation features, such as handling 'up' and 'down' arrow keys. It allows you to check the boundaries of the history buffer to ensure that any subsequent calls to ca_cli_hist_get remain within the valid range of stored commands, preventing out-of-bounds access.

Code example conf:0.95 **Q:** How do I safely iterate through the CLI history using ca_cli_hist_count?
A: You can use the count to define the loop bounds. Example: u8 count = ca_cli_hist_count(); for(u8 i = 0; i < count; i++) { cstr cmd = ca_cli_hist_get(i); /* process cmd */ }

Return conf:1.00 **Q:** What does the return value of ca_cli_hist_count represent, and are there any failure states?
A: The function returns a u8 representing the total number of command strings currently in the history buffer, ranging from 0 to the maximum configured size. There are no failure states mentioned; however, the function requires that ca_cli_init has been called previously. It is non-blocking and safe to call from an ISR.

`ca_cli_hist_forget`

Header: `caos/inc/cli.h:182`
Source: `caos/src/cli.c:1351:1361`
GUARD: `EN_CLI`

```
void ca_cli_hist_forget(const char *prefix)
```

Wipes sensitive command history entries and the active CLI buffer to prevent unauthorized access.

Parametri:

Nome	Tipo	Descrizione
`prefix`	`const char *`	The string prefix used to identify sensitive commands in the history. Must not be NULL or empty.

Ritorna:
None (void function).

Note:

- This function is thread-safe because it wraps its internal operations in a critical section using ca_enter_critical and ca_exit_critical. It is designed to be called from task context, but because it uses critical sections, it is safe to call even if interrupts are active. The function iterates through the entire CLI history buffer and clears any entry starting with the provided prefix. A common beginner mistake is passing a NULL or empty string, which the function safely ignores, but this may lead to sensitive data remaining in the history buffer if the developer assumes the wipe was successful. Note that it also clears the active ISR line buffer if the CLI is currently idle, ensuring no residual data remains in RAM.

Uso:

- Use this function immediately after processing a command that contains sensitive information, such as passwords or cryptographic keys. It is typically paired with command execution logic where the input buffer is no longer needed. By calling this, you ensure that if a user presses the 'Up' arrow key on their terminal, the sensitive command is not retrieved from the history buffer. It is part of the security hardening process for CLI-based applications, usually invoked after the command has been parsed and the sensitive data has been moved to a secure, non-volatile or protected memory location.

Esempio:

```
/* Example: Securely handling a password input in a CLI command */
void handle_password_command(const char *input) {
    /* Assume 'input' contains a sensitive password */
```

```
    if (ca_strncmp(input, "login ", 6) == 0) {
        /* Process the password logic here */
        // ... perform authentication ...

        /* Wipe the history to prevent password leakage via arrow keys */
        ca_cli_hist_forget("login ");

        /* Clear the local buffer if it held the secret */
        ca_memzero((void *)input, ca_strlen(input));
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_cli_hist_forget and what is its primary purpose?
A: You should call it immediately after processing a command containing sensitive information like passwords or cryptographic keys. Its purpose is to wipe those entries from the CLI history and the active buffer to prevent unauthorized access via terminal navigation (e.g., the 'Up' arrow key).

Code example conf:0.90 **Q:** Provide a minimal C example of how to use ca_cli_hist_forget in a caOS application.
A: void process_secure_cmd(cstr input) { /* ... process command ... */ ca_cli_hist_forget("PASS"); }

Parameters conf:1.00 **Q:** What are the requirements for the 'prefix' parameter in ca_cli_hist_forget?
A: The 'prefix' parameter must be a cstr (const char *) used to identify sensitive commands. It must not be NULL or empty, as the function will safely ignore such inputs, potentially leaving sensitive data in the history buffer.

Error/Edge case conf:1.00 **Q:** What happens if I pass an invalid prefix to ca_cli_hist_forget, and is it safe to call from an interrupt?
A: If you pass a NULL or empty string, the function safely ignores the request, which is a common mistake that may leave sensitive data in the buffer. It is safe to call from any context, including interrupts, because it uses ca_enter_critical and ca_exit_critical to ensure thread safety.

`ca_cli_hist_get`

Header: `caos/inc/cli.h:160`
Source: `caos/src/cli.c:1331:1331`
GUARD: `EN_CLI`

```
const char *ca_cli_hist_get(u8 pos)
```

Retrieves a command string from the shared CLI history buffer by its index.

Parametri:

Nome	Tipo	Descrizione
pos	u8	The index of the history entry to retrieve. 0 represents the most recent command, 1 the second most recent, and so on. Must be less than the total number of entries available.

Ritorna:
A pointer to a null-terminated string containing the command, or NULL if the index is out of bounds or the history is empty.

Note:

- This function is non-blocking and thread-safe, as it provides read-only access to a shared global history buffer. It is safe to call from an ISR, but note that it returns a pointer to internal memory; you should copy this string immediately if you intend to modify it or if the history buffer might be updated by another task. The most common beginner mistake is assuming the returned pointer remains valid indefinitely or attempting to modify the string directly, which would corrupt the shared history for all other CLI sessions.

Uso:

- This function is typically used when implementing command-line recall features, such as pressing the 'Up' arrow key in a terminal. It is usually paired with ca_cli_hist_count() to determine the valid range of indices before calling this function. Developers use it to populate a local command buffer (like a session's current line buffer) so the user can edit or re-execute a previously typed command without re-typing it.

Esempio:

```c
/* Example: Recall the most recent command from history */
#include "caos.h"

void recall_last_command(void) {
    /* Check if there is any history available */
    if (ca_cli_hist_count() > 0) {
        /* Retrieve the most recent command (index 0) */
        const char *last_cmd = ca_cli_hist_get(0);

        if (last_cmd != NULL) {
            /* Use ca_printf to display the recalled command to the user */
            ca_printf("Recalled: %s\n", last_cmd);
        }
    } else {
        ca_printf("No history available.\n");
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_cli_hist_get in my CLI implementation?
A: You should use this function when implementing command-line recall features, such as handling 'Up' arrow key inputs in a terminal. It allows you to retrieve a previous command from the shared history buffer to populate a local session buffer, enabling users to edit or re-execute commands without re-typing them.

Code example conf:1.00 **Q:** How do I retrieve the most recent command using ca_cli_hist_get?
A: You can retrieve the most recent command by passing 0 as the index. Example: cstr cmd = ca_cli_hist_get(0); if (cmd != NULL) { /* copy cmd to local buffer immediately */ }

Parameters conf:1.00 **Q:** What does the u8 pos parameter represent and what are its constraints?
A: The u8 pos parameter is the index of the history entry. 0 is the most recent command, 1 is the second most recent, and so on. The value must be less than the total number of entries available; otherwise, the function will return NULL.

Return conf:1.00 **Q:** What does the return value of ca_cli_hist_get indicate, and why might it return NULL?
A: The function returns a pointer to a null-terminated string containing the command. It returns NULL if the provided index is out of bounds or if the history buffer is empty.

ca_cli_init

Header: caos/inc/cli.h:76
Source: caos/src/cli.c:975:1006
GUARD: EN_CLI

```
void ca_cli_init(u16 buf_size)
```

Initializes the Command Line Interface (CLI) internal buffers and queue structures.

Parametri:

Nome	Tipo	Descrizione
buf_size	u16	The desired size in bytes for the CLI buffer. Values are clamped to a multiple of DEFAULT_MSG_SIZE and capped at CLI_BUFFER_MAX. Passing 0 disables the CLI.

Ritorna:
void. The function does not return a status code; it initializes internal static structures and sets the CLI state to disabled.

Note:

- This function is non-blocking and performs static memory allocation for the CLI queue. It is not thread-safe and must be called during the system initialization phase, typically before the RTOS scheduler starts. It is strictly forbidden to call this from an Interrupt Service Routine (ISR) as it performs internal queue setup that is not designed for atomic context. A common beginner mistake is to assume the CLI is ready immediately after this call; however, this function only prepares the memory structures. You must follow up with ca_cli_set_sender and ca_cli_create_thread to actually start the CLI task. Additionally, if you intend to register custom commands, ensure they are registered after this initialization, as the CLI subsystem must be fully prepared to accept new command nodes.

Uso:

- This function is the mandatory first step in the CLI lifecycle. It is typically used in the main initialization sequence of your application, often wrapped in an EVAL_CLI macro to ensure it only runs if the CLI module is enabled in the build configuration. It is always paired with ca_cli_create_thread, which spawns the actual RTOS task that processes user commands. Without calling this function first, the subsequent thread creation will fail or operate on uninitialized memory, leading to system instability. It is also the prerequisite for any command registration or hook configuration.

Esempio:

```
/* Initialize the CLI system during startup */
#define CLI_BUFFER_SIZE 2048

void ca_os_main(void) {
    /* Prepare the CLI memory and queue structures */
    ca_cli_init(CLI_BUFFER_SIZE);

    /* Configure where the CLI output should be sent */
    ca_cli_set_sender(STDOUT);

    /* Finally, spawn the RTOS thread to handle CLI commands */
    ca_cli_create_thread();

```

```
    /* Register custom commands after initialization */
    /* ca_cli_register(&s_cli_node); */
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_cli_init() in my caOS application?
A: You should call ca_cli_init() during the initialization phase of your caOS application, typically before starting any tasks or enabling the CLI thread. It is used to set up the command line interface buffer for handling user commands. The function is non-blocking and must not be called from an ISR.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_cli_init() with caOS types?
A:

```
u16 buffer_size = 2048;
ca_cli_init(buffer_size);
```

Parameters conf:0.95 **Q:** What does the buf_size parameter in ca_cli_init() control and what are valid values?
A: The buf_size parameter sets the size of the CLI buffer in bytes. Valid values are from 1 to CLI_BUFFER_MAX (e.g., 8192). If buf_size is 0, the CLI is disabled and no buffer is allocated. Values smaller than DEFAULT_MSG_SIZE (e.g., 64) after alignment are set to DEFAULT_MSG_SIZE.

Return conf:0.90 **Q:** Does ca_cli_init() return any value, and what does it mean?
A: The function does not return any value. It is void. It cannot fail in a traditional sense, but it may disable the CLI if buf_size is 0, or adjust the buffer size if it's too small after alignment.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_cli_init()?
A: Common mistakes include calling ca_cli_init() from an ISR, which is not allowed. Also, passing a buf_size of 0 disables the CLI, which may be unintentional. If buf_size is too small after alignment, it will be set to DEFAULT_MSG_SIZE. The function does not check for memory allocation failures, as it only initializes the buffer structure.

Behavior conf:0.90 **Q:** Is ca_cli_init() blocking or non-blocking, and is it safe to call from an ISR or in a multi-threaded context?
A: ca_cli_init() is non-blocking. It is not safe to call from an ISR, as it is explicitly stated that the function must not be called from an ISR. It is also not thread-safe, so concurrent calls from multiple threads should be avoided.

`ca_cli_is_enabled`

Header: `caos/inc/cli.h:94`
Source: `caos/src/cli.c:1017:1019`
GUARD: `EN_CLI`

```
bool ca_cli_is_enabled(void)
```

Checks if the Command Line Interface (CLI) system is currently active and ready to process commands.

Ritorna:
Returns 'true' if the CLI is enabled and ready to receive input, or 'false' if it is disabled or not yet initialized.

Note:

- This function is non-blocking and thread-safe because it performs a simple read of a global boolean flag. It is safe to call from both main-line code and Interrupt Service Routines (ISRs) because it does not involve complex synchronization primitives like mutexes or semaphores. A common mistake is assuming that 'true' means the CLI is currently idle; it only indicates that the system is enabled. You must ensure that the CLI module has been properly initialized via ca_cli_init before relying on this status, otherwise, it may return a stale or incorrect value.

Uso:

- This function is typically used as a guard clause before attempting to send data to the CLI or before performing heavy operations related to the command parser. It is often paired with initialization functions like ca_cli_init that set the global state and status reporting functions like ca_cli_stats. By checking this status, you prevent the system from attempting to process commands or log output when the CLI is intentionally disabled to save resources or during system shutdown sequences.

Esempio:

```
if (ca_cli_is_enabled()) {
    // The CLI is active, so it is safe to proceed with logging or status checks
    u32 queued, buffered, dropped;
    ca_cli_stats(&queued, &buffered, &dropped);
    // Proceed to display stats to the user
} else {
    // CLI is disabled, skip operations to avoid errors or unnecessary
   processing
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_cli_is_enabled() in my embedded application?
A: You should call ca_cli_is_enabled() to check if the CLI (Command Line Interface) is currently enabled before attempting to send commands or interact with the CLI system. This is useful for conditional logic where you want to avoid sending commands when the CLI is disabled, or to provide feedback to users about CLI availability. It's particularly helpful in ISR contexts where you want to ensure CLI operations are safe to perform.

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_cli_is_enabled() with caOS types?
A: bool enabled = ca_cli_is_enabled(); if (enabled) { // CLI is enabled, proceed with CLI operations // Example: send a command or log status } else { // CLI is disabled, handle accordingly }

Return conf:0.90 **Q:** What does ca_cli_is_enabled() return and what causes it to return false?
A: The function returns true if the CLI is enabled, and false if it is disabled. It cannot fail or return errors because it simply reads a global variable. The return value of false indicates that the CLI subsystem has been disabled, likely through a configuration or initialization step elsewhere in the code. The variable g_enabled is managed atomically elsewhere, so this function is thread-safe and does not introduce race conditions.

`ca_cli_register`

Header: `caos/inc/cli.h:70`
Source: `caos/src/cli.c:968:971`
GUARD: `EN_CLI`

```
void ca_cli_register(ca_cli_node_t *node)
```

Adds a command-line interface (CLI) node to the global linked list of available system commands.

Parametri:

Nome	Tipo	Descrizione
node	ca_cli_node_t *	Pointer to a static or persistent CLI node structure containing command definitions. Must not be NULL.

Ritorna:
void

Note:

- This function is non-blocking and performs a simple pointer assignment to link the new node into the command chain. It is NOT thread-safe; if multiple threads attempt to register nodes simultaneously, the linked list structure could become corrupted. It is NOT safe to call from an ISR because it modifies a global linked list that the CLI parser might be traversing at the same time. The most common beginner mistake is passing a pointer to a local (stack-allocated) variable; since the CLI system stores this pointer for the lifetime of the application, the stack memory will be overwritten once the function returns, leading to a system crash or garbage commands when the CLI is accessed. Always ensure the node structure is declared as 'static' or in global scope.

Uso:

- This function is used during the initialization phase of drivers or services to expose their internal status or control functions to the system CLI. It is typically paired with a static boolean flag to ensure the node is registered exactly once, even if the initialization function is called multiple times. You should use this after the hardware or service has been successfully initialized, as the CLI commands often rely on the underlying peripheral being ready. It is the standard way to make your module 'debuggable' via the serial console without needing a debugger attached.

Esempio:

```
static ca_cli_node_t my_module_cli_node = { .name = "my_cmd", .help = "Prints
    status" };

int my_module_init(void) {
    // 1. Perform hardware initialization first
    // Call hardware setup function here

    // 2. Register the CLI node only once to prevent list corruption
    static bool s_registered = false;
    if (!s_registered) {
        ca_cli_register(&my_module_cli_node);
        s_registered = true;
    }
    return 0;
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_cli_register() and what is its purpose in a caOS project?
A: You should call ca_cli_register() to add CLI commands for debugging and configuration purposes. It is used to register a CLI node that contains command definitions for a specific driver or module. The function is typically called during system initialization or when a driver is initialized, and it allows users to interact with the system through

a command-line interface. It is not intended for use in interrupt service routines (ISR) and must be called from a main task.

Code example conf:0.85 **Q:** Can you provide a minimal C example showing how to use ca_cli_register() with caOS types?
A: ca_cli_node_t node = { .name = "adc", .help = "ADC driver commands", .cmd = NULL, .next = NULL }; ca_cli_register(&node);

Parameters conf:0.90 **Q:** What is the significance of the node parameter in ca_cli_register(), and what happens if it is NULL?
A: The node parameter is a pointer to a CLI node structure that defines the commands available for a specific driver. If the node is NULL, the function does nothing and does not cause a crash. It is a simple registration function that adds the node to a global linked list, so the node must remain valid for the entire duration of the program execution.

`ca_cli_rx_byte`

Header: `caos/inc/cli.h:206`
Source: `caos/src/cli.c:1372:1472`
GUARD: `EN_CLI`

```
1 void ca_cli_rx_byte(u32 ch)
```

Processes a single byte of input for the CLI, handling line editing, history, and VT100 escape sequences.

Parametri:

Nome	Tipo	Descrizione
`ch`	`u32`	The raw byte received from a communication peripheral (e.g., UART or USB CDC). Only the lower 8 bits are used.

Ritorna:
This function returns void. It performs internal state updates and triggers side effects like echoing characters or submitting commands.

Note:

- This function is the core of the CLI input engine. It is NOT thread-safe and must be called from the context where the byte is received (often an ISR or a dedicated communication task). It handles complex logic like backspace, arrow keys for history, and command submission (CR/LF). A common mistake is calling this from multiple threads simultaneously without external synchronization, which will corrupt the internal line buffer. It assumes the CLI has been initialized via ca_cli_init() and is enabled; if not, the byte is silently ignored. Because it performs string manipulation and triggers callbacks, ensure the stack size of the calling task is sufficient.

Uso:

- This function is typically used as a callback passed to low-level drivers (like ca_lpuart_init or USB CDC handlers). When a byte arrives at the hardware level, the driver calls ca_cli_rx_byte() to pass the data into the CLI subsystem. It is the bridge between raw hardware input and the command parser. You should never call this manually unless you are implementing a custom input stream or a virtual terminal emulator. It is usually paired with ca_cli_init() during system startup and works in conjunction with the CLI thread created by ca_cli_create_thread().

Esempio:

```c
/* Example: Integrating a UART byte into the CLI */
void my_uart_rx_callback(u8 byte) {
    /* Pass the received byte directly to the CLI engine */
    ca_cli_rx_byte((u32)byte);
}

void setup_cli_interface(void) {
    /* Initialize the CLI system first */
    ca_cli_init();

    /* Register the CLI thread to handle command execution */
    ca_cli_create_thread();

    /* Initialize UART with the callback function */
    /* ca_lpuart_init(baud, callback, wake, gpio_alt) */
    ca_lpuart_init(115200, my_uart_rx_callback, true, false);
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_cli_rx_byte() in my embedded application?
A: You should call ca_cli_rx_byte() whenever a byte is received from a UART interface that is used for CLI input. It is designed to be invoked from an ISR (Interrupt Service Routine) when data arrives on a UART channel, allowing the CLI to process user input including special control characters like backspace, enter, and arrow keys.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_cli_rx_byte() in a UART ISR?
A: void UART4_IRQHandler(void) { if (USART4->SR & USART_SR_RXNE) { u8 byte = (u8)(USART4->DR & 0xFF); ca_cli_rx_byte((u32)byte); } }

Parameters conf:0.90 **Q:** What does the 'ch' parameter in ca_cli_rx_byte() represent, and what are valid values?
A: The 'ch' parameter represents a byte received from a UART interface, typically ranging from 0 to 255. Only the lower 8 bits are considered. Values above 255 may cause unpredictable behavior. Valid inputs include regular ASCII characters, control characters like backspace (0x08), carriage return (0x0D), and escape sequences such as arrow keys.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_cli_rx_byte()?
A: Common mistakes include calling ca_cli_rx_byte() without initializing the CLI buffer, passing invalid values for 'ch' (e.g., values > 255), or invoking the function when CLI is disabled (g_enabled != true). Additionally, calling it from a non-ISR context or from a thread that is not designed to handle CLI input can lead to incorrect behavior.

Behavior conf:0.90 **Q:** Is ca_cli_rx_byte() blocking or non-blocking, and is it safe to call from an ISR?
A: ca_cli_rx_byte() is non-blocking in terms of thread execution, but it performs processing inline during the ISR. It is designed to be called from an ISR and is not thread-safe because it modifies global internal variables. It must be called only when CLI is enabled (g_enabled=true) and should not perform time-consuming operations like printf inside the ISR.

`ca_cli_set_hook`

Header: `caos/inc/cli.h:215`
Source: `caos/src/cli.c:962:964`
GUARD: `EN_CLI`

```
void ca_cli_set_hook(ca_cli_hook_t hook)
```

Registers a custom callback function to be executed by the Command Line Interface (CLI) system.

Parametri:

Nome	Tipo	Descrizione
hook	ca_cli_hook_t	A function pointer to the custom routine you want the CLI to trigger. Passing NULL will effectively disable the current hook.

Ritorna:
void

Note:

- This function is non-blocking and executes instantly as it simply updates a global pointer variable. It is thread-safe in the sense that it performs an atomic pointer assignment, but you must ensure that the function pointed to by 'hook' is re-entrant if it accesses shared resources. It is safe to call from an ISR, though this is rarely necessary for CLI configuration. The most common beginner mistake is passing a pointer to a local function that goes out of scope or failing to verify that the hook function handles potential NULL inputs if the CLI logic expects a valid pointer.

Uso:

- This function is typically used during the system initialization phase, right after the CLI module has been started but before the main application loop begins. It is used to inject custom logic into the CLI processing pipeline, such as logging commands to an external storage or triggering specific system states based on user input. It is often paired with 'ca_cli_get_hook' if you need to verify which hook is currently active or to chain multiple hooks together by storing the previous one before setting a new one.

Esempio:

```
/* Define the custom hook function */
void my_custom_cli_hook(void) {
    /* Custom logic to execute when the CLI triggers */
}

void app_init(void) {
    /* Initialize the CLI system first */
    ca_cli_init();

    /* Register the hook during system startup */
    ca_cli_set_hook(my_custom_cli_hook);

    /* Verify the hook was set correctly */
    if (ca_cli_get_hook() == NULL) {
        /* Handle error: hook registration failed */
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_cli_set_hook and why would I want to use a custom CLI hook?
A: You should call ca_cli_set_hook when you want to handle custom CLI commands in your caOS application. This function allows you to register a callback that will be invoked whenever a CLI command is received. It's useful for extending the default CLI functionality with application-specific commands. The hook can be set at any time, even before commands are received, but if not set, custom commands will not be processed.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_cli_set_hook with a custom CLI handler?
A: void my_cli_hook(cstr argv[], u32 argc) { // Handle custom CLI command } void setup(void) { ca_cli_set_hook(my_cli_hook); }

Parameters conf:0.90 **Q:** What does the ca_cli_hook_t parameter in ca_cli_set_hook do, and what happens if I pass NULL?
A: The ca_cli_hook_t parameter is a function pointer that defines the callback to be executed when a CLI command is received. If you pass NULL, the hook is disabled, meaning no custom CLI command handling will occur. Passing NULL is a valid and safe operation, and it will not cause any errors or failures in the function.

`ca_cli_set_secret_prefix`

Header: `caos/inc/cli.h:191`
Source: `caos/src/cli.c:1249:1253`
GUARD: `EN_CLI`

```
void ca_cli_set_secret_prefix(const char *prefix)
```

Sets a secret prefix string that the CLI will ignore when saving commands to history.

Parametri:

Nome	Tipo	Descrizione
`prefix`	`const char *`	The string literal or buffer to be used as a secret prefix. If NULL, the secret prefix feature is disabled.

Ritorna:
void

Note:

- This function is not thread-safe and should be called during the initialization phase of your CLI task before the CLI thread starts processing user input. It is not intended to be called from an ISR. The function stores a pointer to the provided string; therefore, you must ensure the memory containing the prefix remains valid and constant for the entire duration of the application. A common mistake is passing a pointer to a local stack variable that goes out of scope, which will cause the CLI to compare incoming commands against garbage memory.

Uso:

- Use this function when you want to prevent sensitive inputs, such as passwords or authentication tokens, from appearing in the CLI command history. It is typically called once during the system setup sequence, immediately after ca_cli_register and before the CLI thread begins execution. By setting a prefix, any command starting with that specific string will be treated as 'secret' by the internal history management logic, ensuring it is never logged to the command history buffer.

Esempio:

```
/* Define the CLI structure */
static struct ca_cli_s s_my_cli;

void app_cli_setup(void) {
    /* Initialize the CLI system */
    ca_cli_init();

```

```
    /* Register the CLI interface */
    ca_cli_register(&s_my_cli);

    /* Set a secret prefix so that commands like 'login password123'
       are not saved in the history buffer */
    ca_cli_set_secret_prefix("login ");

    /* Start the CLI thread to begin processing input */
    ca_cli_create_thread();
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_cli_set_secret_prefix and what is its primary purpose?
A: You should call this function during the initialization phase of your CLI task, specifically after ca_cli_register and before the CLI thread starts processing input. Its purpose is to prevent sensitive inputs, such as passwords or authentication tokens, from being saved to the CLI command history buffer.

Code example conf:0.95 **Q:** How do I correctly implement ca_cli_set_secret_prefix in my application?
A: You must ensure the prefix string is stored in memory that remains valid for the entire application duration, such as a global constant. Example: static const cstr secret = "SECURE:"; void app_init(void) { ca_cli_set_secret_prefix(secret); }

Parameters conf:1.00 **Q:** What happens if I pass a NULL value to the prefix parameter, and are there any restrictions on the memory I provide?
A: Passing NULL to the prefix parameter disables the secret prefix feature. You must not pass a pointer to a local stack variable, as the function stores the pointer and expects the memory to remain valid and constant for the entire duration of the application; otherwise, the CLI will compare commands against garbage memory.

`ca_cli_set_sender`

Header: `caos/inc/cli.h:88`
Source: `caos/src/cli.c:1008:1015`
GUARD: `EN_CLI`

```
void ca_cli_set_sender(sender out)
```

Configures the output destination (sender) for the Command Line Interface (CLI) system.

Parametri:

Nome	Tipo	Descrizione
`out`	`sender`	A pointer or identifier representing the output stream (e.g., STDOUT). Passing a valid pointer enables the CLI, while passing NULL disables it.

Ritorna:
This function returns void and does not provide a status code, so it assumes the provided sender is valid.

Note:

- This function is non-blocking and performs a simple assignment to global variables. It is not thread-safe in a multi-threaded environment if called while the CLI thread is actively processing, so it should be used during the initialization phase. It is not intended

to be called from an ISR. A common beginner mistake is calling this function before ca_cli_init(), which results in the configuration being ignored or causing a system fault because the internal CLI structures are not yet ready. Always ensure the underlying communication peripheral is initialized before assigning it as the sender.

Uso:

- This function is used during the system startup phase to link the CLI engine to a physical communication channel. It is typically called immediately after ca_cli_init() and before ca_cli_create_thread(). By setting the sender, you define where the CLI output (like command prompts and logs) will appear. It is standard practice to pair this with ca_cli_init() and ca_cli_create_thread() to ensure the CLI is fully operational before the scheduler starts.

Esempio:

```
/* Initialize the CLI system with a stack size of 2048 bytes */
ca_cli_init(2048);

/* Set the output destination to the standard output (STDOUT) */
ca_cli_set_sender(STDOUT);

/* Now that the sender is configured, start the CLI thread */
ca_cli_create_thread();

/* Register the CLI node to handle commands */
ca_cli_register(&s_cli);

/* To disable the CLI output later, pass NULL to the sender */
ca_cli_set_sender(NULL);
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_cli_set_sender() and why would I want to disable the CLI output?
A: You should call ca_cli_set_sender() when you want to control where CLI output is sent, such as to a serial terminal or a custom logging function. If you pass NULL as the sender, you disable the CLI output, which can be useful if you want to conserve resources or if you're handling CLI commands through a different mechanism. The function is non-blocking and can be called from any context, including ISRs, but it's not thread-safe.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_cli_set_sender() with a custom output function?
A: void serial_output(cstr msg) { // Simulate sending message to serial port u32* uart_dr = (u32*)0x40013800; // UART1_DR register *uart_dr = (u32)msg; }
void example() { ca_cli_set_sender(serial_output); }

Parameters conf:0.90 **Q:** What does the 'out' parameter in ca_cli_set_sender() represent, and what happens if I pass an invalid pointer?
A: The 'out' parameter is a pointer to a function that handles CLI output. If you pass NULL, the CLI is disabled but not deallocated or reset. The function does not validate the pointer dynamically, so passing an invalid pointer will likely cause a crash or undefined behavior when the CLI tries to send output. It's the caller's responsibility to ensure 'out' points to a valid function.

`ca_cli_stats`

Header: `caos/inc/cli.h:102`
Source: `caos/src/cli.c:1021:1031`

GUARD: `EN_CLI`

```
void ca_cli_stats(u32 *queued, u32 *buffered, u32 *dropped)
```

Retrieves current diagnostic statistics for the CLI message queue, including pending items, memory usage, and overflow counts.

Parametri:

Nome	Tipo	Descrizione
`queued`	`u32 *`	Pointer to a variable where the number of messages currently waiting in the queue will be stored. Pass NULL if you do not need this value.
`buffered`	`u32 *`	Pointer to a variable where the total number of bytes currently occupied in the queue buffer will be stored. Pass NULL if you do not need this value.
`dropped`	`u32 *`	Pointer to a variable where the count of messages discarded due to queue overflow will be stored. Pass NULL if you do not need this value.

Ritorna:
This function returns void; it provides data via the provided pointers.

Note:

- This function is non-blocking and executes in constant time, making it safe to call from both standard tasks and Interrupt Service Routines (ISRs). It performs direct memory reads of global variables; because it does not use mutexes, the values returned are a 'snapshot' and may be slightly inconsistent if the CLI task is actively modifying the queue at the exact moment of the read. The most common mistake is assuming the returned values are perfectly synchronized with other system events; treat these as diagnostic indicators rather than precise real-time counters. Ensure the CLI system has been initialized via ca_cli_init before calling this, otherwise, the returned statistics will reflect uninitialized memory or zeroed-out global variables.

Uso:

- This function is primarily used for system health monitoring, typically implemented as a 'status' or 'info' command within the CLI itself. Developers use it to detect if the CLI queue is filling up too quickly, which often indicates that the command processing task is blocked, starving, or unable to keep up with incoming data. It is standard practice to pair this with ca_cli_is_enabled() to ensure the subsystem is active before querying. In a typical diagnostic sequence, you would call this periodically from a background monitoring task or a system watchdog to log performance metrics and identify potential bottlenecks in command execution.

Esempio:

```
u32 pending, bytes, lost;

// Check if CLI is active before querying stats to avoid reading invalid data
if (ca_cli_is_enabled()) {
    // Retrieve all metrics by passing pointers to local variables
    ca_cli_stats(&pending, &bytes, &lost);

    // Use ca_printf to output the diagnostic data to the console
    ca_printf("CLI Stats: %u pending, %u bytes, %u dropped\n", pending, bytes,
    lost);
} else {
```

```
    // Handle case where CLI is disabled
    ca_printf("CLI is currently disabled.\n");
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_cli_stats() and what is its purpose in a caOS application?
A: You should call ca_cli_stats() to monitor the CLI queue's utilization and detect potential data loss. It is useful for debugging and performance tuning, especially in real-time systems where message queuing behavior is critical. The function allows you to read the number of messages in queue, the total buffer space used, and the number of dropped messages.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_cli_stats() with caOS types?
A: u32 queued, buffered, dropped; ca_cli_stats(&queued, &buffered, &dropped); // queued now contains number of messages in queue // buffered contains total bytes used by queued messages // dropped contains number of lost messages

Parameters conf:0.95 **Q:** What do the parameters of ca_cli_stats() represent, and what happens if I pass NULL for some of them?
A: The parameters are pointers to u32 variables where the function writes statistics: queued (number of messages in queue), buffered (bytes used by messages), and dropped (number of lost messages). If any pointer is NULL, the corresponding statistic is not written to that location. For example, passing NULL for buffered means the buffer usage value will not be updated.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_cli_stats()?
A: Common mistakes include passing NULL pointers when expecting to read values, which can cause invalid memory access. Additionally, since the function is not thread-safe, calling it from multiple threads simultaneously may result in inconsistent data. Also, because it reads global variables directly, values may be inconsistent if modified by another thread during execution.

`ca_cli_typo_count`

Header: `caos/inc/cli.h:147`
Source: `caos/src/cli.c:1321:1321`
GUARD: `EN_CLI`

```
u32 ca_cli_typo_count(void)
```

Returns the total count of typing errors (typos) detected by the CLI subsystem.

Ritorna:
A 32-bit unsigned integer representing the cumulative number of invalid character sequences or typo events recorded by the CLI since system initialization.

Note:

- This function is non-blocking and thread-safe as it performs a simple read of a global counter. It is safe to call from both standard tasks and Interrupt Service Routines (ISRs). The primary precondition is that the CLI subsystem must have been initialized via ca_cli_init() prior to use. A common mistake for beginners is assuming this counter resets automatically; it does not, so you must track its value over time if you intend to calculate error rates or detect spikes in user input errors.

Uso:

- This function is typically used for system diagnostics, telemetry, or monitoring user interaction quality. It is often paired with ca_cli_stats() to provide a comprehensive overview of the console health. Developers use it to determine if a remote user or a local operator is struggling with the interface, allowing the system to trigger alerts or adjust input sensitivity if the typo count exceeds a specific threshold.

Esempio:

```
/* Check CLI health and report if typo count is too high */

void monitor_cli_health(void) {
    u32 current_typos = ca_cli_typo_count();

    /* If we detect more than 100 typos, log a warning to the system console */
    if (current_typos > 100U) {
        ca_printf("Warning: High typo count detected: %lu\n", current_typos);

        /* Optional: handle the error state if the application logic requires */
        /* Note: there is no function to reset the typo counter, so we just
    monitor the trend */
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_cli_typo_count in my caOS application?
A: You should use it for system diagnostics, telemetry, or monitoring user interaction quality. It is useful for detecting if an operator is struggling with the interface, allowing you to trigger alerts or adjust input sensitivity if the count exceeds a threshold. It is often paired with ca_cli_stats() for a comprehensive overview of console health.

Code example conf:1.00 **Q:** How do I implement a simple check of the CLI typo count in my code?
A: Ensure ca_cli_init() has been called first, then read the counter as shown: u32 current_typos = ca_cli_typo_count();

Return conf:1.00 **Q:** What does the return value of ca_cli_typo_count represent, and does it reset?
A: It returns a u32 representing the cumulative number of invalid character sequences or typo events recorded since system initialization. It does not reset automatically; you must track its value over time if you need to calculate error rates or detect spikes.

`ca_clock_hw_adc_init`

Header: `caos/drivers/inc/ca_glue_hw.h:9`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_clock_hw_adc_init(u8 adc_num); /* abilita il clock RCC
    per il periferico ADC indicato */
```

Enables the hardware clock for a specific ADC peripheral based on the target microcontroller architecture.

Parametri:

Nome	Tipo	Descrizione
adc_num	u8	The index of the ADC peripheral to initialize (typically 1, 2, or 3). Values outside the supported range for the specific MCU will result in no action.

Ritorna:
void

Note:

- This function is non-blocking and executes extremely quickly, as it only performs register writes to the Reset and Clock Control (RCC) unit. It is thread-safe in the sense that it performs atomic-like register modifications, but it is not intended to be called concurrently from multiple threads for the same peripheral. It is safe to call from an ISR, though it is almost exclusively used during system initialization. The most common mistake is calling this function before the system clock tree is stable or attempting to use the ADC peripheral before this function has been called, which will result in a hard fault or a bus error when accessing ADC registers.

Uso:

- This function is the mandatory first step in the ADC initialization sequence. It must be called before any other configuration of the ADC registers (such as setting sample times or resolution) because the peripheral registers are inaccessible while the clock is disabled. It is typically called inside the driver's main initialization function, such as ca_adc_init, immediately after validating the peripheral index. It pairs with hardware-specific configuration functions like ca_adc_hw_init_periph to ensure the peripheral is fully powered and ready for operation.

Esempio:

```c
/* Example of initializing ADC1 within the driver layer */
#include "caos.h"

bool ca_adc_init(const ca_adc_config_t *cfg) {
    /* 1. Validate input parameters */
    if (!cfg || cfg->adc_num < 1 || cfg->adc_num > 3) {
        return false;
    }

    /* 2. Enable the hardware clock for the specific ADC peripheral */
    /* This must happen before accessing any ADC registers */
    ca_clock_hw_adc_init(cfg->adc_num);

    /* 3. Proceed with peripheral-specific hardware configuration */
    bool ok = ca_adc_hw_init_periph(cfg->adc_num, cfg->sample_time);

    return ok;
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_clock_hw_adc_init() in my ADC initialization sequence?
A: You should call ca_clock_hw_adc_init() before any other ADC configuration operations. It enables the clock for the specified ADC peripheral via the RCC registers, which is a prerequisite for ADC setup. This function must be called before configuring ADC registers such as CR1, CR2, or SMPR.

Code example conf:0.95 **Q:** Can you show a minimal example of how to use ca_clock_hw_adc_init() with caOS types?

A:

```
u8 adc_num = 1;
ca_clock_hw_adc_init(adc_num);
```

Parameters conf:0.95 **Q:** What does the adc_num parameter represent and what are valid values?

A: The adc_num parameter specifies which ADC peripheral clock to enable. Valid values are 1, 2, or 3. For F1xx, only 1 and 2 are valid; for H5xx, the parameter is ignored as there is only one ADC. Values outside the range 1-3 have no effect.

Return conf:0.95 **Q:** Does ca_clock_hw_adc_init() return any value? What does it mean if it does?

A: No, ca_clock_hw_adc_init() does not return any value. It directly modifies RCC registers to enable the ADC clock and has no return values to indicate success or failure.

Error/Edge case conf:0.90 **Q:** What are common mistakes or edge cases when using ca_clock_hw_adc_init()?

A: Common mistakes include calling this function from an ISR, which is not allowed, or calling it after ADC configuration has already begun without proper synchronization. Also, passing invalid adc_num values (not 1, 2, or 3) will result in no action. For H5xx, passing any value will be ignored due to a single ADC.

Behavior conf:0.95 **Q:** Is ca_clock_hw_adc_init() thread-safe and ISR-safe?

A: No, ca_clock_hw_adc_init() is neither thread-safe nor ISR-safe. It is a blocking function that modifies global RCC registers and should not be called from an ISR. It must be executed before any ADC operations and is not safe in multitasking environments without synchronization.

HW differences conf:0.90 **Q:** How does ca_clock_hw_adc_init() differ between MCU families like F1xx and H7xx?

A: In F1xx, only ADC1 and ADC2 are supported, and the function modifies RCC->APB2ENR. In H7xx, ADC1 and ADC2 are controlled via RCC->AHB1ENR, while ADC3 uses RCC->AHB4ENR. Additionally, H7xx includes extra steps to configure ADC kernel clock and common control registers (CCR) for proper ADC operation, which F1xx does not require.

`ca_clock_hw_dac_init`

Header: `caos/drivers/inc/ca_glue_hw.h:88`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_clock_hw_dac_init(u8 channel); /* abilita il clock RCC
    per il periferico DAC */
```

Enables the peripheral clock for the DAC hardware module in the RCC register.

Parametri:

Nome	Tipo	Descrizione
`channel`	`u8`	The DAC channel index. While the parameter is passed, it is currently ignored by the implementation as the clock enable affects the entire DAC peripheral block.

Ritorna:
void

Note:

- This function is non-blocking and executes almost instantaneously. It is safe to call from an ISR because it only performs a simple register write operation. It is a mandatory precondition for any DAC operation; if you attempt to access DAC registers before calling

this, the CPU will likely trigger a HardFault or the peripheral will simply remain unresponsive. The most common mistake is assuming that calling this function is enough to start the DAC; it only enables the clock signal, meaning you must still follow up with GPIO configuration and peripheral-specific initialization functions.

Uso:

- This function is intended to be used exclusively during the system initialization phase or when a driver is being brought up for the first time. It is typically the first step in a sequence: call this to enable the clock, then call the GPIO initialization function to configure the pins, and finally call the peripheral-specific init function to set up the DAC registers. It is usually called inside the higher-level ca_dac_init function to ensure the hardware is ready before any configuration data is written to the registers.

Esempio:

```
ca_dac_config_t my_dac_cfg = { .channel = 1 };

// Initialize the DAC peripheral
// 1. Enable the clock
ca_clock_hw_dac_init(my_dac_cfg.channel);

// 2. Configure the associated GPIO pins
ca_gpio_hw_dac_init(my_dac_cfg.channel);

// 3. Configure the DAC peripheral registers
if (ca_dac_hw_init_periph(my_dac_cfg.channel)) {
    // DAC is now ready for use
} else {
    // Handle initialization error
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_clock_hw_dac_init() and why is it necessary?
A: You should call ca_clock_hw_dac_init() before configuring any DAC channel to enable the clock for the DAC peripheral. This function is necessary because the DAC hardware requires its clock to be enabled in the RCC registers before any configuration or operation can take place. It's a low-level function that ensures the hardware is ready for use.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_clock_hw_dac_init() with caOS types?
A:

```
#include "dac.h"

void init_dac_channels(void)
{
    ca_clock_hw_dac_init(1);  // Enable clock for DAC channel 1
    ca_clock_hw_dac_init(2);  // Enable clock for DAC channel 2
}
```

Parameters conf:0.90 **Q:** What does the channel parameter in ca_clock_hw_dac_init() represent, and what are valid values?
A: The channel parameter specifies which DAC channel's clock to enable. Valid values are 1 and 2. Values 0, 3-255 are invalid. If an invalid channel is passed, the function does nothing. The function is a stub when EN_DAC=0, so it has no effect in that case.

HW differences conf:0.85 **Q:** How does ca_clock_hw_dac_init() differ between different STM32 MCU families in terms of hardware implementation?
A: The implementation varies by MCU family: F1xx uses a stub function with no action; F4xx and F7xx enable the DAC clock in RCC->APB1ENR register; H5xx enables DAC1

clock in RCC->AHB2ENR register; H7xx enables DAC12 clock in RCC->APB1LENR register. All implementations include two NOP instructions for clock stabilization.

---

ca_clock_hw_deinit

Header: caos/drivers/inc/ca_glue_hw.h:81
GUARD: EN_CA_GLUE_HW

```
extern void ca_clock_hw_deinit(void); /* back to the reset clock tree (
    SYSCLK on HSI/MSIS, PLLs off): ca_os_end */
```

Resets the system clock tree to a safe, minimal state (HSI/MSIS) and disables PLLs/caches.

Ritorna:
void

Note:

- This function is intended to be called as the final step before a system reset or jumping to a new firmware image. It is not thread-safe and must be called when the system is in a controlled state, typically after all other tasks have been stopped and interrupts disabled. It is strictly forbidden to call this from an ISR, as it performs blocking operations (waiting for oscillator stabilization) that would cause a system hang. The most common beginner mistake is calling this while other peripherals or tasks are still active, which leads to unpredictable behavior or hard faults when the clock frequency changes abruptly.

Uso:

- This function is used during the system shutdown or firmware update sequence. It is typically paired with functions that reset peripherals and prepare the CPU for a jump to a new bootloader or application image. The sequence usually involves: 1) Stopping all active tasks and interrupts, 2) Calling a peripheral reset function, 3) Calling ca_clock_hw_deinit to ensure the clock tree is in a known, low-frequency state, and 4) Performing the final jump or reset operation.

Esempio:

```
/* Prepare the system for a clean shutdown or firmware jump */

/* 1. Stop all system activity and interrupts */
__disable_irq();

/* 2. Reset all peripherals to their default state */
/* ca_os_hw_periph_reset(); // Hypothetical peripheral reset function */

/* 3. Revert clock tree to a safe, slow oscillator (HSI/MSIS) */
ca_clock_hw_deinit();

/* 4. Ensure all memory operations are complete before jumping */
__DSB();
__ISB();

/* 5. Proceed to jump to the new application or trigger a system reset */
/* jump_to_new_image(); // Hypothetical jump function */
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_clock_hw_deinit in my application?
A: You should call this function as the final step before a system reset or when jumping to a new firmware image. It is designed to put the system clock tree into a safe, minimal state (HSI or MSIS) and disable PLLs and caches to prepare the CPU for the next image.

Code example conf:0.95 **Q:** How do I correctly sequence a system shutdown using ca_clock_hw_deinit?
A: The sequence should be: 1) Stop all active tasks and interrupts, 2) Call your peripheral reset function, 3) Call ca_clock_hw_deinit(), and 4) Perform the final jump or reset. Example: { void shutdown(void) { /* stop tasks/interrupts */ peripheral_reset(); ca_clock_hw_deinit(); /* jump to bootloader */ } }

Error/Edge case conf:1.00 **Q:** What is the most common mistake developers make when using ca_clock_hw_deinit?
A: The most common mistake is calling this function while other peripherals or tasks are still active. This leads to unpredictable behavior or hard faults because the clock frequency changes abruptly while the system is still relying on the previous, higher-frequency clock configuration.

Behavior conf:1.00 **Q:** Is ca_clock_hw_deinit safe to call from an interrupt service routine (ISR)?
A: No, it is strictly forbidden to call this function from an ISR. It performs blocking operations, such as waiting for oscillator stabilization, which would cause a system hang if called from an interrupt context. It is also not thread-safe and must be called in a controlled state with interrupts disabled.

HW differences conf:1.00 **Q:** How does the implementation of ca_clock_hw_deinit differ between the F7xx and U5xx families?
A: The F7xx implementation explicitly calls SCB_DisableICache() and resets the CFGR register, whereas the U5xx implementation manages the ICACHE peripheral directly by clearing the ICACHE_CR_EN bit and waiting for the ICACHE_SR_BUSYF flag. Additionally, the U5xx targets the MSIS oscillator as the base clock, while the F7xx targets HSI.

`ca_clock_hw_dma_init`

Header: `caos/drivers/inc/ca_glue_hw.h:99`
GUARD: `EN_CA_GLUE_HW`

```
1 extern void ca_clock_hw_dma_init(u8 controller); /* abilita il clock RCC
      per il controller DMA indicato */
```

Enables the peripheral clock for a specific DMA controller in the RCC register.

Parametri:

Nome	Tipo	Descrizione
`controller`	`u8`	The index of the DMA controller to enable (e.g., 1, 2, or 3). Valid values depend on the specific STM32 series; invalid values are ignored by the switch-case.

Ritorna:
void

Note:

- This function is non-blocking and executes almost instantaneously. It is thread-safe as it performs a simple bitwise OR operation on the RCC registers, which is atomic on most ARM Cortex-M architectures. It is safe to call from an ISR, though it is typically called during system initialization. The most common mistake is assuming that calling this function is sufficient to use the DMA; you must still configure the DMA streams/channels and interrupts separately. The 'nop' instructions are included to ensure the clock signal has time to propagate through the hardware bus before subsequent configuration registers are accessed.

Uso:

- This function is used exclusively during the hardware abstraction layer (HAL) initialization phase, typically inside a driver's init function or the main system startup sequence. It must be called before any attempt to configure DMA registers (like stream settings or interrupt enables), as accessing registers of a peripheral with a disabled clock will result in a bus fault or silent failure. It is usually paired with subsequent calls to configure the specific DMA stream or channel, and it is often called for all available controllers (1, 2, and 3) during the boot process to ensure the entire DMA subsystem is ready for use.

Esempio:

```
/* Initialize DMA subsystem during system startup */
void system_dma_init(void) {
    /* Enable clocks for all potential DMA controllers */
    ca_clock_hw_dma_init(1);
    ca_clock_hw_dma_init(2);
    ca_clock_hw_dma_init(3);

    /* Now it is safe to configure DMA channels */
    /* Use ca_dma_init to configure specific channels after clock is enabled */
    ca_dma_init(1, 0);
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_clock_hw_dma_init() and why is it necessary?
A: You should call ca_clock_hw_dma_init() before configuring or using any DMA controller. It enables the clock for the specified DMA controller via the RCC register, which is required for DMA functionality. Without enabling the clock, DMA channels will not operate correctly. This function is typically called during system initialization or before DMA setup.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_clock_hw_dma_init() with caOS types?
A:

```
u8 controller = 1;
ca_clock_hw_dma_init(controller);
```

Parameters conf:0.90 **Q:** What does the 'controller' parameter in ca_clock_hw_dma_init() represent and what are valid values?
A: The 'controller' parameter specifies which DMA controller's clock to enable. Valid values are 1 for DMA1 and 2 for DMA2 (if available). Any other value will result in no action being taken. For example, on F1xx MCUs, controller 2 only enables DMA2 if F1_DMA2_CHANNELS > 0.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_clock_hw_dma_init()?
A: A common mistake is calling this function with an invalid controller number (e.g., 3 or higher), which will have no effect but won't cause an error. Another potential issue is not calling this function before using DMA channels, leading to non-functional DMA operations. The function is not thread-safe since it modifies a shared system register, so care must be taken in multi-threaded environments.

HW differences conf:0.80 **Q:** How does ca_clock_hw_dma_init() differ between F4xx and H7xx MCUs in terms of register usage?
A: On F4xx MCUs, the function uses RCC->AHB1ENR to enable DMA1EN and DMA2EN for controllers 1 and 2 respectively. On H7xx MCUs, it also uses RCC->AHB1ENR for DMA1EN and DMA2EN, but includes an additional case for controller 3 that enables BDMA via RCC->AHB4ENR. The register names and bit positions differ between families, reflecting the different RCC clocking architecture in each MCU family.

ca_clock_hw_i2c_init

Header: caos/drivers/inc/ca_glue_hw.h:175
GUARD: EN_CA_GLUE_HW

```
extern void ca_clock_hw_i2c_init(u8 num); /* abilita il clock RCC per il
    periferico I2C indicato */
```

Enables the peripheral clock for the specified I2C hardware module via the RCC register.

Parametri:

Nome	Tipo	Descrizione
num	u8	The I2C peripheral index (e.g., 1, 2, 3, or 4). Must correspond to a valid I2C peripheral available on the specific STM32 MCU model.

Ritorna:
void. This function does not return a status code, so it assumes the provided index is valid for the target hardware.

Note:

- This function is a low-level hardware abstraction layer (HAL) helper. It is non-blocking and executes extremely quickly by performing a direct bitwise OR operation on the RCC (Reset and Clock Control) registers. It is thread-safe in the sense that it performs an atomic register write, but it is not intended to be called concurrently from multiple threads or ISRs for the same peripheral. The most common beginner mistake is calling this function before the system clock is properly configured or attempting to initialize an I2C peripheral index that does not exist on the specific MCU variant, which will result in a silent failure or a hard fault if the register address is invalid.

Uso:

- This function is used exclusively during the initialization phase of an I2C peripheral. It is typically called inside the driver's main initialization function (e.g., ca_i2c_init) immediately before configuring the GPIO pins and the I2C peripheral registers themselves. Without calling this function, the I2C peripheral will remain 'dead' because the internal logic gates are not receiving a clock signal, causing any subsequent attempts to write to I2C registers to be ignored or trigger a bus fault.

Esempio:

```
/* Example of initializing I2C1 within a driver setup function */
#include "caos.h"

void setup_my_sensor(void) {
    u8 i2c_instance = 1;

    /* 1. Enable the clock for the peripheral first */
    ca_clock_hw_i2c_init(i2c_instance);

    /* 2. Configure the associated GPIO pins (SDA/SCL) */
    ca_gpio_hw_i2c_init(i2c_instance);

    /* 3. Proceed to configure the I2C peripheral registers */
    /* ca_i2c_hw_init_periph(i2c_instance, 400000); // 400kHz */
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_clock_hw_i2c_init() in my I2C driver initialization sequence?
A: You should call ca_clock_hw_i2c_init() before any other I2C initialization functions to enable the clock for the specific I2C peripheral. This is required because the I2C peripheral will not function without its clock enabled via the RCC registers.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_clock_hw_i2c_init() with caOS types?
A:

```
u8 i2c_num = 1;
ca_clock_hw_i2c_init(i2c_num);
/* Now proceed with I2C initialization */
```

Parameters conf:0.95 **Q:** What does the 'num' parameter in ca_clock_hw_i2c_init() represent and what are valid values?
A: The 'num' parameter specifies which I2C peripheral to enable. Valid values are 1, 2, 3, and 4. Values 0 or greater than 4 are ignored. I2C2 is not available on STM32F103 MCUs, while I2C3 and I2C4 are supported on other families.

Return conf:0.95 **Q:** Does ca_clock_hw_i2c_init() return anything, and can it fail?
A: The function returns nothing (void). It does not have any failure conditions because it only modifies RCC registers and does not check for errors. It is non-blocking and cannot fail.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_clock_hw_i2c_init()?
A: Common mistakes include calling this function from an ISR, calling it from multiple threads without synchronization, or not calling it at all before initializing I2C peripherals. Also, calling it multiple times for the same I2C number is safe but unnecessary.

HW differences conf:0.90 **Q:** How does ca_clock_hw_i2c_init() differ between STM32F1xx and STM32H5xx families?
A: On STM32F1xx, I2C2 is not supported and only I2C1 and I2C3 are enabled via APB1ENR register. On STM32H5xx, I2C1 and I2C2 are enabled via APB1LENR, I2C3 via APB3ENR, and I2C4 also via APB3ENR if defined. The register addresses and peripheral availability vary between families.

`ca_clock_hw_lptim_init`

Header: `caos/drivers/inc/ca_glue_hw.h:281`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_clock_hw_lptim_init(u8 num); /* accende LSI, clock RCC e
   kernel clock LSI per il LPTIM indicato; false se non presente */
```

Initializes the LSI clock source and enables the peripheral clock for a specific Low Power Timer (LPTIM).

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The index of the LPTIM peripheral to initialize. Currently, only 1 is supported; passing any other value will return false.

Ritorna:
Returns true if the clock configuration was successful, or false if the peripheral index is invalid or unsupported by the hardware.

Note:

- This function is blocking because it waits for the LSI (Low Speed Internal) oscillator to stabilize via a while-loop. It is not thread-safe and should not be called from an ISR, as the wait loop could hang the system if the oscillator fails to start. It must be called before any register-level configuration of the LPTIM peripheral, as the timer will not function without its kernel clock. A common mistake is assuming this function configures the timer's period or interrupts; it only handles the clock tree setup.

Uso:

- This function is used as the first step in the hardware abstraction layer's initialization sequence for the LPTIM. It is typically paired with 'lptim_acquire' to reserve the resource, followed by 'ca_lptim_hw_init' to configure the timer's actual timing parameters and interrupts. If this function fails, the driver must release the resource using 'lptim_release' to prevent locking the peripheral for other tasks.

Esempio:

```
/* Example of initializing LPTIM1 for periodic tasks */
#define LPTIM_ID 1

if (lptim_acquire(LPTIM_ID)) {
    /* Configure the clock tree first */
    if (ca_clock_hw_lptim_init(LPTIM_ID)) {
        /* Proceed to configure timer registers and start operation */
        if (ca_lptim_hw_init(LPTIM_ID, 1000, true, false)) {
            // Timer successfully initialized for 1000ms period
        } else {
            lptim_release(LPTIM_ID);
        }
    } else {
        lptim_release(LPTIM_ID);
    }
}
```

Q&A — fine-tuning review *7/7 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_clock_hw_lptim_init in my driver initialization sequence?
A: It must be called as the first step in the LPTIM initialization sequence, specifically before any register-level configuration of the LPTIM peripheral, as the timer requires the kernel clock to function.

Code example conf:1.00 **Q:** Provide a minimal C example for initializing LPTIM1 using caOS types.
A: u8 lptim_id = 1U; if (ca_clock_hw_lptim_init(lptim_id)) { /* Proceed with lptim_acquire and ca_lptim_hw_init */ }

Parameters conf:1.00 **Q:** What does the 'num' parameter represent and what are the valid values?
A: The 'num' parameter is a u8 representing the index of the LPTIM peripheral. Currently, only 1 is supported; passing any other value will result in the function returning false.

Return conf:1.00 **Q:** What do the return values of ca_clock_hw_lptim_init indicate?
A: The function returns true if the LSI clock source and peripheral clock were successfully configured. It returns false if the peripheral index is invalid or unsupported by the specific hardware.

Error/Edge case conf:1.00 **Q:** What are common mistakes or error conditions to watch out for when using this function?
A: A common mistake is assuming this function configures the timer's period or interrupts; it only handles the clock tree. Additionally, if the function fails, you must call

'lptim_release' to prevent locking the peripheral for other tasks.

Behavior conf:1.00 **Q:** Is ca_clock_hw_lptim_init thread-safe or safe to call from an ISR?
A: No, it is not thread-safe and must not be called from an ISR. It is a blocking function that uses a while-loop to wait for the LSI oscillator to stabilize; calling it from an ISR could hang the system if the oscillator fails to start.

HW differences conf:1.00 **Q:** How does the implementation of ca_clock_hw_lptim_init differ across STM32 families?
A: Implementations vary based on register access requirements: for example, H5xx and U5xx require setting the DBP bit in the PWR domain to access the BDCR register, whereas F7xx and H7xx access the LSI via the CSR register without needing DBP. Furthermore, peripheral clock enabling registers differ, such as APB1ENR on F7xx versus APB3ENR on H5xx and U5xx.

`ca_clock_hw_lse_lock`

Header: `caos/drivers/inc/ca_glue_hw.h:234`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_clock_hw_lse_lock(void); /* lock the system clock RC to
    the running LSE (U5 MSI PLL-mode); false = not on this family */
```

Locks the system clock (MSI PLL-mode) to the LSE crystal for high-precision timing on supported STM32 families.

Ritorna:
Returns true if the system clock was successfully locked to the LSE, or false if the hardware does not support this feature, the LSE is not running, or the lock operation failed.

Note:

- This function is hardware-specific and only performs an action on families like the STM32U5 that support MSI PLL-mode. It is not thread-safe and should be called during system initialization before the OS scheduler is fully operational. It is a blocking function because it waits for the LSE to stabilize and the PLL to lock. It must NOT be called from an ISR, as it performs busy-waiting loops that could cause system instability or missed interrupts. A common mistake is calling this before the LSE has been properly initialized and enabled in the RCC registers.

Uso:

- This function is typically used during the early boot sequence to improve the accuracy of the system tick. It is usually paired with ca_rtc_get_clk_src() to verify that the LSE is the intended clock source before attempting to lock the system clock to it. Once successfully locked, it is standard practice to call ca_os_tick_retune() to adjust the OS tick frequency to match the new, more precise clock source.

Esempio:

```
/* Check if the RTC is using the LSE and attempt to lock the system clock to it
    */
if (ca_rtc_get_clk_src() == CA_RTC_CLK_LSE) {
    if (ca_clock_hw_lse_lock()) {
        /* Adjust the OS tick to match the high-precision LSE-locked clock */
        ca_os_tick_retune();
    } else {
        /* Handle failure: log error or fallback to default clock */
        ca_printf("Warning: Failed to lock system clock to LSE\n");
    }
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_clock_hw_lse_lock and what is its primary purpose?
A: It should be called during the early boot sequence before the OS scheduler is operational. Its purpose is to lock the system clock (MSI PLL-mode) to the LSE crystal to achieve high-precision timing for the system tick.

Code example conf:0.90 **Q:** Provide a minimal C example for initializing the system clock with ca_clock_hw_lse_lock.
A: void system_init(void) { if (ca_clock_hw_lse_lock()) { ca_os_tick_retune(); } }

Return conf:1.00 **Q:** What do the return values of ca_clock_hw_lse_lock indicate?
A: Returns true if the system clock was successfully locked to the LSE. Returns false if the hardware does not support the feature, the LSE is not running, or the lock operation failed.

Error/Edge case conf:1.00 **Q:** What are common mistakes or error conditions when using ca_clock_hw_lse_lock?
A: A common mistake is calling the function before the LSE has been properly initialized and enabled in the RCC registers. Additionally, failing to ensure the LSE is running before calling the function will result in a failure.

Behavior conf:1.00 **Q:** Is ca_clock_hw_lse_lock thread-safe and can it be called from an ISR?
A: No, it is not thread-safe and must not be called from an ISR. It is a blocking function that uses busy-waiting loops, which could cause system instability or missed interrupts if called from an ISR.

HW differences conf:1.00 **Q:** How does the implementation of ca_clock_hw_lse_lock differ across STM32 families?
A: The function is hardware-specific. On families like F1, F4, F7, H5, and H7, it returns false because they lack MSI PLL-mode or the specific LSE-to-system-clock locking capability. Only families like the U5 implement the logic, which involves interacting with RCC_BDCR_LSESYSEN and RCC_CR_MSIPLLEN registers.

`ca_clock_hw_spi_init`

Header: `caos/drivers/inc/ca_glue_hw.h:237`
GUARD: `EN_CA_GLUE_HW`

```
1 extern void ca_clock_hw_spi_init(u8 num); /* abilita il clock RCC per il
      periferico SPI indicato */
```

Enables the internal RCC clock signal for a specific SPI hardware peripheral.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The SPI peripheral index (e.g., 1, 2, or 3). Passing an unsupported number will result in no action being taken.

Ritorna:
void

Note:

- This function is non-blocking and executes extremely quickly, as it only performs a register write to the Reset and Clock Control (RCC) unit. It is thread-safe in the sense that it performs an atomic-like bitwise OR operation on the register, but it does not implement mutexes, so it should be called during the initialization phase before the peripheral is used by multiple tasks. It is safe to call from an ISR, though this is rarely necessary as clock

initialization should happen during system startup. The most common beginner mistake is attempting to configure or read/write to SPI registers before calling this function; if the clock is not enabled, the CPU will ignore any writes to the SPI peripheral registers, often leading to a hard fault or silent failure.

Uso:

- This function is a mandatory prerequisite for any SPI communication. It is typically called inside the driver initialization sequence, specifically within the 'ca_spi_init' function. The standard sequence is: first, call 'ca_clock_hw_spi_init' to wake up the hardware, then call 'ca_gpio_hw_spi_init' to configure the physical pins, and finally call the peripheral-specific initialization function (e.g., 'ca_spi_hw_init_periph'). Skipping this step is the most frequent cause of 'dead' SPI peripherals that appear configured but never transmit data.

Esempio:

```
/* Example of initializing SPI1 */
u8 spi_index = 1;

/* 1. Enable the clock for the peripheral */
ca_clock_hw_spi_init(spi_index);

/* 2. Configure the associated GPIO pins (e.g., SCK, MISO, MOSI) */
ca_gpio_hw_spi_init(spi_index, true);

/* 3. Now it is safe to initialize the SPI peripheral registers */
/* ca_spi_hw_init_periph(spi_index, baud_rate, cpol, cpha); */
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_clock_hw_spi_init() in my SPI driver setup?
A: You should call ca_clock_hw_spi_init() before configuring any SPI peripheral. This function enables the clock for the specified SPI peripheral in the RCC, which is required for the SPI to function properly. Without enabling the clock, the SPI peripheral will not respond to configuration registers.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_clock_hw_spi_init() in a C file using caOS types?
A:

```
u8 spi_num = 1;
ca_clock_hw_spi_init(spi_num);
// Now configure SPI1 registers
```

Parameters conf:0.95 **Q:** What does the 'num' parameter in ca_clock_hw_spi_init() represent and what are valid values?
A: The 'num' parameter specifies which SPI peripheral clock to enable. Valid values are 1, 2, or 3, corresponding to SPI1, SPI2, and SPI3 respectively. Values 0 or greater than 3 are ignored and do not enable any clock.

Return conf:0.90 **Q:** Does ca_clock_hw_spi_init() return any value and can it fail?
A: The function returns void and does not have any failure conditions in the strict sense. It performs a simple register write operation and is guaranteed to complete. Invalid parameters (0 or >3) will not enable any clock but will not cause an error.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_clock_hw_spi_init()?
A: Common mistakes include calling this function after SPI configuration, which will not work, or passing invalid values like 0 or 4-255, which will silently do nothing. Also, calling this function from an ISR is not allowed as it's not thread-safe and not reentrant.

HW differences conf:0.90 **Q:** How does ca_clock_hw_spi_init() differ between STM32F1xx and STM32H7xx families?

A: On STM32F1xx, SPI1 uses RCC_APB2ENR, while SPI2 is only enabled on non-F103 MCUs using RCC_APB1ENR. On STM32H7xx, SPI1 uses RCC_APB2ENR, SPI2 and SPI3 use RCC_APB1LENR. The register names and bit positions differ between families, with H7xx using different APB buses for different peripherals.

`ca_clock_hw_timer_init`

Header: `caos/drivers/inc/ca_glue_hw.h:265`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_clock_hw_timer_init (u8 num); /* abilita il clock RCC per
    il timer indicato */
```

Enables the peripheral clock for a specific hardware timer in the STM32 RCC register.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The hardware timer index (e.g., 1, 2, 3...). Must match the specific timer available on your target MCU; invalid numbers return false.

Ritorna:
Returns true if the clock was successfully enabled, or false if the timer index is invalid or unsupported by the current hardware configuration.

Note:

- This function is non-blocking and executes extremely quickly as it only performs a bitwise OR operation on an RCC register. It is thread-safe in the sense that it performs an atomic-like register modification, but it should be called during the initialization phase before the timer is used. It is safe to call from an ISR, though typically you should initialize hardware before enabling interrupts. A common beginner mistake is attempting to configure timer registers (like ARR or PSC) before calling this function; if the clock is not enabled, writing to timer registers will have no effect or cause a hard fault.

Uso:

- This function is a mandatory prerequisite for any timer operation. It is typically called inside a driver's initialization sequence, such as ca_timer_init_periodic, immediately after acquiring the timer resource. It acts as the 'power switch' for the peripheral; without it, the timer hardware remains in a reset state and is invisible to the CPU. It is always paired with subsequent hardware-specific configuration functions that set the timer's frequency and mode.

Esempio:

```
/* Example: Initializing Timer 2 for periodic tasks */
u8 timer_id = 2;

/* 1. Acquire the resource to ensure no other task uses it */
if (timer_acquire(timer_id)) {
    /* 2. Enable the clock for the peripheral */
    if (ca_clock_hw_timer_init(timer_id)) {
        /* 3. Proceed to configure timer registers */
        /* ca_timer_hw_init_periodic(timer_id, 1000); */
    } else {
        /* Handle error: invalid timer index */
        timer_release(timer_id);
    }
```

```
} else {
    /* Handle error: timer already in use */
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_clock_hw_timer_init() in my embedded application?
A: You should call ca_clock_hw_timer_init() before initializing any timer hardware configuration. This function enables the clock for a specific timer peripheral in the RCC, which is required for the timer to function properly. It must be called before any timer setup, such as configuring registers or enabling interrupts.

Code example conf:0.95 **Q:** Can you show a minimal example of how to use ca_clock_hw_timer_init()?
A:

```
u8 timer_num = 2;
bool success = ca_clock_hw_timer_init(timer_num);
if (success) {
    // Timer clock is enabled, now configure the timer
} else {
    // Handle invalid timer number
}
```

Parameters conf:0.95 **Q:** What does the 'num' parameter in ca_clock_hw_timer_init() represent and what are valid values?
A: The 'num' parameter specifies the timer number to enable. Valid values depend on the MCU family: for F1xx MCUs, valid values are 1-4 (with 4 only if MCU_ID != 103); for F4xx, F7xx, H7xx, and H5xx, valid values are 1-14. Any other value will cause the function to return false.

Return conf:0.95 **Q:** What do the return values of ca_clock_hw_timer_init() indicate?
A: The function returns true if the timer clock was successfully enabled. It returns false if the timer number is invalid or not supported for the current MCU family. This typically happens when an unsupported timer number is passed.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_clock_hw_timer_init()?
A: Common mistakes include calling the function with an invalid timer number (e.g., 5 on an F1xx MCU), calling it after the timer has already been used without resetting, or not calling it at all before timer configuration. Also, calling it from an ISR without explicit control is discouraged.

Behavior conf:0.90 **Q:** Is ca_clock_hw_timer_init() blocking or thread-safe?
A: The function is not blocking; it modifies an RCC register which is immediate. However, it is not thread-safe because it modifies a global RCC register. It can be called from an ISR, but it's not recommended without explicit control due to potential race conditions.

HW differences conf:0.85 **Q:** How does ca_clock_hw_timer_init() differ between MCU families like F1xx and H7xx?
A: The main difference lies in the number of supported timers and the registers used. F1xx supports timers 1-4 (with 4 only if MCU_ID != 103), using RCC_APB1ENR and RCC_APB2ENR. H7xx supports up to 17 timers, using RCC_APB1LENR and RCC_APB2ENR, with different timer numbers mapped to different register bits. For example, timer 12 on H7xx uses RCC_APB1LENR while timer 1 on F1xx uses RCC_APB2ENR.

`ca_cond_broadcast`

Header: `caos/inc/sync.h:280`
Source: `caos/src/sync.c:700:705`
GUARD: `EN_SYNC`

```
void ca_cond_broadcast(ca_condvar_t *cv)
```

Wakes up all tasks currently blocked waiting on the specified condition variable.

Parametri:

Nome	Tipo	Descrizione
cv	ca_condvar_t *	Pointer to the initialized condition variable object. Must not be NULL.

Ritorna:
void

Note:

- This function is thread-safe and uses an internal spinlock to protect the wait queue during the broadcast operation. It is non-blocking in the sense that it does not wait for the woken tasks to finish, but it does disable interrupts briefly to ensure atomicity. You must never call this function from an ISR, as it involves scheduler manipulation and potential context switching. A common beginner mistake is assuming that the woken tasks will immediately preempt the current task; in reality, the scheduler decides which task runs next based on priority, so the woken tasks might remain in a 'ready' state for a short time before actually executing.

Uso:

- Use this function when a state change occurs that affects multiple waiting tasks, such as a shared resource becoming available or a global flag being updated. It is typically paired with ca_cond_wait, where tasks block until the condition is met. The standard workflow involves locking a mutex, checking the condition, calling ca_cond_wait if the condition is false, and later having another task call ca_cond_broadcast once the condition is satisfied. Unlike ca_cond_signal, which only wakes one task, broadcast is essential for scenarios where multiple consumers need to re-check the state simultaneously after a single event.

Esempio:

```
/* Assuming s_cv is a globally accessible condition variable and s_mtx is a
    mutex */
void notify_all_workers(void) {
    /* Lock the mutex to ensure exclusive access to the shared condition */
    ca_mutex_lock(&s_mtx);

    /* Update the shared state that the waiters are checking */
    /* shared_data_ready = true; */

    /* Wake up all tasks waiting for the data to be ready */
    ca_cond_broadcast(&s_cv);

    /* Release the mutex so the woken tasks can proceed to lock it */
    ca_mutex_unlock(&s_mtx);
}

/* In a worker task */
void worker_task(void) {
    while(1) {
        ca_mutex_lock(&s_mtx);
        /* Always use a while loop to handle spurious wakeups */
        while(/* !shared_data_ready */ 0) {
            ca_cond_wait(&s_cv, &s_mtx);
        }
        /* Process the data once woken */
        ca_mutex_unlock(&s_mtx);
```

```
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_cond_broadcast() in a caOS application?
A: You should use ca_cond_broadcast() when you need to wake up all tasks that are waiting on a condition variable. This is useful for signaling an event to multiple threads simultaneously, such as when a resource becomes available or a task needs to be notified of a state change that affects several waiting processes.

Code example conf:0.90 **Q:** Can you show a minimal example of using ca_cond_broadcast() with caOS types?
A: ca_mutex_t mutex; caCondVar cond; ca_mutex_init(&mutex); ca_cond_init(&cond); ca_mutex_lock(&mutex); ca_cond_broadcast(&cond); ca_mutex_unlock(&mutex);

Parameters conf:0.95 **Q:** What is the significance of the ca_condvar_t *cv parameter in ca_cond_broadcast()?
A: The ca_condvar_t *cv parameter is a pointer to the condition variable that holds the list of tasks waiting to be woken up. It must be initialized with ca_cond_init() before use. If NULL is passed, it will cause a crash, as the function does not handle null pointers gracefully.

Return conf:0.90 **Q:** Does ca_cond_broadcast() return any value, and what does it mean?
A: The function does not return any value. It is void. It never fails, but it may not have any effect if the condition variable is not properly initialized or if no tasks are waiting on it.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_cond_broadcast()?
A: Common mistakes include calling ca_cond_broadcast() without holding the associated mutex, calling it from an ISR, or passing a NULL pointer. Edge cases include calling it when no tasks are waiting on the condition variable, in which case it has no effect. It is also not thread-safe unless called with the correct mutex acquired.

`ca_cond_init`

Header: `caos/inc/sync.h:228`
Source: `caos/src/sync.c:518:521`
GUARD: `EN_SYNC`

```
void ca_cond_init(ca_condvar_t *cv)
```

Initializes a condition variable object to manage thread synchronization and signaling.

Parametri:

Nome	Tipo	Descrizione
`cv`	`ca_condvar_t *`	A pointer to the condition variable structure that needs to be initialized. Must point to valid, allocated memory.

Ritorna:
void

Note:

- This function is non-blocking and performs the necessary setup for the internal wait queue and spinlock. It is thread-safe in the sense that it prepares the object for use, but it must be called before any thread attempts to wait on or signal the condition variable. It is safe to call from an ISR context if the memory is already allocated, but typically this is done during system initialization. The most common beginner mistake is attempting to use a

condition variable without calling this initialization function first, which leads to undefined behavior because the internal wait queue and spinlock remain in an uninitialized state.

Uso:

- This function is used during the system setup phase, typically alongside mutex initialization, to prepare synchronization primitives for producer-consumer patterns. It is almost always paired with ca_cond_wait, ca_cond_signal, or ca_cond_broadcast to manage thread state transitions. You should call this once for every condition variable used in your application before spawning any tasks that might access them. It is part of the standard initialization sequence: define the global variable, call the init function, and then pass the pointer to your task functions.

Esempio:

```
static ca_mutex_t tw_mtx;
static ca_condvar_t tw_obj;

static void tw_init(void) {
    // Always initialize synchronization primitives before use
    ca_mutex_init(&tw_mtx);
    ca_cond_init(&tw_obj);
}

static void tw_give(void) {
    ca_mutex_lock(&tw_mtx);
    // Signal the condition variable to wake up waiting tasks
    ca_cond_signal(&tw_obj);
    ca_mutex_unlock(&tw_mtx);
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_cond_init() in my caOS application, and why is it important for thread synchronization?
A: You should call ca_cond_init() once to initialize a condition variable before using it for thread synchronization. It is important because it prepares the condition variable structure for use with ca_cond_wait() and ca_cond_signal() functions, allowing threads to wait for specific conditions to be met. The function must be called before any thread attempts to use the condition variable, and it should only be called once per variable to avoid undefined behavior.

Code example conf:0.90 **Q:** Can you show me a minimal C code example using ca_cond_init() with caOS types?
A: ca_condvar_t my_cond; ca_cond_init(&my_cond);

Parameters conf:0.90 **Q:** What happens if I pass a NULL pointer to ca_cond_init(), and is there any validation for the parameter?
A: If you pass a NULL pointer to ca_cond_init(), the behavior is undefined and will cause a crash. The function documentation explicitly states that the parameter 'cv' must be a valid, allocated pointer and cannot be NULL. There is no internal validation in ca_cond_init() to check for NULL, so passing NULL leads to immediate system failure.

`ca_cond_signal`

Header: `caos/inc/sync.h:273`
Source: `caos/src/sync.c:693:698`
GUARD: `EN_SYNC`

```
void ca_cond_signal(ca_condvar_t *cv)
```

Wakes up exactly one task currently waiting on the specified condition variable.

Parametri:

Nome	Tipo	Descrizione
cv	ca_condvar_t *	Pointer to the initialized condition variable object. Must not be NULL.

Ritorna:
void. This function does not return a status code, as it is designed to be a fire-and-forget notification mechanism.

Note:

- This function is non-blocking and thread-safe, utilizing an internal spinlock to protect the wait queue during the operation. It is safe to call from an ISR because it uses spinlocks with interrupt disabling (irq). The most common beginner mistake is assuming that calling this function guarantees the woken task will run immediately; in reality, the woken task only becomes 'ready' and must still compete for CPU time based on its priority. If no tasks are waiting on the condition variable, the signal is simply lost, which is expected behavior for condition variables.

Uso:

- This function is typically used in producer-consumer patterns to notify a waiting consumer that a resource is available. It is almost always paired with ca_cond_wait or ca_cond_timedwait, which are called by the consumer to block until the signal arrives. The standard sequence is: lock the mutex, modify the shared resource, call ca_cond_signal to wake a waiter, and finally unlock the mutex. Failing to hold the mutex while signaling can lead to race conditions where a task misses the signal entirely.

Esempio:

```
/* Producer task example */
ca_mutex_lock(&buffer_mutex);

/* Add data to shared buffer using custom logic */
/* buffer[head] = new_data; */

// Notify one waiting consumer that data is available
ca_cond_signal(&buffer_not_empty);

ca_mutex_unlock(&buffer_mutex);
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_cond_signal() in a producer-consumer scenario?
A: You should call ca_cond_signal() after a producer adds an item to a shared buffer and wants to notify a waiting consumer. It wakes up one consumer task that is waiting on the condition variable, allowing it to process the newly available data.

Code example conf:0.90 **Q:** Can you show a minimal example of using ca_cond_signal() with caOS types?
A: ca_mutex_t mutex; caCondVar cond; void* task_func(void* arg) { ca_mutex_lock(&mutex); // ... do work ... ca_cond_signal(&cond); ca_mutex_unlock(&mutex); return NULL; }

Parameters conf:0.95 **Q:** What is the significance of the ca_condvar_t* parameter in ca_cond_signal()?
A: The ca_condvar_t* parameter points to a condition variable that must be initialized with ca_cond_init(). It specifies which waiting task(s) should be woken up. Passing NULL causes a crash.

Return conf:0.95 **Q:** Does ca_cond_signal() return any value, and what does it mean?
A: The function returns void. It never fails, but may not wake any task if none are waiting on the condition variable.

Error/Edge case conf:0.90 **Q:** What happens if I call ca_cond_signal() without holding the associated mutex?
A: Calling ca_cond_signal() without holding the associated mutex results in undefined behavior. The function must be called after acquiring the mutex that protects the shared resource.

Behavior conf:0.90 **Q:** Is ca_cond_signal() safe to call from an ISR, and does it block?
A: Yes, ca_cond_signal() is safe to call from an ISR and is non-blocking. It uses a spinlock internally to protect its data structures, making it suitable for interrupt contexts.

`ca_cond_timedwait`

Header: `caos/inc/sync.h:266`
Source: `caos/src/sync.c:686:691`
GUARD: `EN_SYNC`

```
1 bool ca_cond_timedwait(ca_condvar_t *cv, ca_mutex_t *mtx, u32 timeout_ms
    )
```

Suspends the current thread until a condition variable is signaled or a timeout expires, while atomically releasing the associated mutex.

Parametri:

Nome	Tipo	Descrizione
`cv`	`ca_condvar_t *`	Pointer to the initialized condition variable object to wait on.
`mtx`	`ca_mutex_t *`	Pointer to the mutex that must be held by the caller before calling this function.
`timeout_ms`	`u32`	Maximum time in milliseconds to wait. If set to 0 (CA_NO_WAIT_TIME), the function returns immediately without waiting.

Ritorna:
Returns true if the thread was woken up by a signal or broadcast (condition met), or false if the timeout expired or the function was called with a zero timeout.

Note:

- This function is blocking and must only be called from a thread context, never from an ISR. It is thread-safe, but it requires that the caller already owns the mutex passed as an argument; failing to hold the mutex will trigger a system error. The function performs an atomic release of the mutex upon entering the wait state and ensures the mutex is reacquired before returning to the caller. A common mistake is forgetting that the condition must be re-checked in a loop after this function returns, as spurious wakeups or race conditions might occur between the signal and the thread resuming execution.

Uso:

- This function is used to implement producer-consumer patterns or state synchronization where a thread needs to wait for a specific data condition without busy-waiting. It is typically used in a sequence: first, lock the mutex using ca_mutex_lock, then enter a while loop checking the condition, call ca_cond_timedwait inside that loop, and finally process the data once the condition is met. It is always paired with ca_cond_signal or

ca_cond_broadcast, which are called by other threads to notify the waiting thread that the state has changed.

Esempio:

```
/* Assuming mtx and cv are initialized */
ca_mutex_lock(&mtx);

/* Wait for data to be available, with a 500ms timeout */
/* The loop ensures we handle spurious wakeups correctly */
while (!data_ready) {
    if (!ca_cond_timedwait(&cv, &mtx, 500)) {
        /* Handle timeout: data did not arrive in time */
        break;
    }
}

if (data_ready) {
    /* Process the data safely */
    data_ready = false;
}

ca_mutex_unlock(&mtx);
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_cond_timedwait in a real-time embedded application?
A: You should use ca_cond_timedwait when you need to wait for a condition to be met with a timeout, such as waiting for data from a sensor or a message in a queue, but you want to avoid indefinite blocking. This is especially useful in RTOS environments where you must prevent threads from hanging indefinitely.

Code example conf:0.90 **Q:** Can you show a minimal example of using ca_cond_timedwait with caOS types?
A: ca_mutex_t mtx; caCondVar cv; ca_mutex_init(&mtx); ca_cond_init(&cv); ca_mutex_lock(&mtx bool result = ca_cond_timedwait(&cv, &mtx, 1000); ca_mutex_unlock(&mtx);

Parameters conf:0.90 **Q:** What does the timeout_ms parameter control in ca_cond_timedwait?
A: The timeout_ms parameter specifies the maximum time (in milliseconds) the calling thread will wait for a signal. If 0, the thread will wake up immediately if not signaled. Values larger than the maximum supported time may cause undefined behavior.

Return conf:0.95 **Q:** What does ca_cond_timedwait return when it times out?
A: It returns false when the thread wakes up due to a timeout, meaning no signal was received within the specified time. It returns true if the thread was woken up by ca_cond_signal() or ca_cond_broadcast() before the timeout.

Error/Edge case conf:0.85 **Q:** What are common mistakes when calling ca_cond_timedwait?
A: Common mistakes include calling the function without holding the mutex, passing NULL pointers for cv or mtx, or using a timeout_ms value that is too large and may cause overflow or undefined behavior.

Behavior conf:0.90 **Q:** Is ca_cond_timedwait thread-safe and can it be called from an ISR?
A: Yes, ca_cond_timedwait is thread-safe, but it is not ISR-safe. It is a blocking system call and should not be invoked from interrupt service routines. It requires the calling thread to own the mutex before calling.

`ca_cond_wait`

Header: `caos/inc/sync.h:237`
Source: `caos/src/sync.c:523:558`
GUARD: `EN_SYNC`

```
void ca_cond_wait(ca_condvar_t *cv, ca_mutex_t *mtx)
```

Suspends the current thread until a condition variable is signaled, atomically releasing the associated mutex.

Parametri:

Nome	Tipo	Descrizione
cv	ca_condvar_t *	Pointer to the initialized condition variable object. Must be initialized via ca_cond_init before use.
mtx	ca_mutex_t *	Pointer to the mutex currently held by the calling thread. Must be locked by the caller before invoking this function.

Ritorna:
void. This function does not return an error code; if the internal safeguards fail (e.g., the thread does not own the mutex), the system will trigger a safeguard violation.

Note:

- This function is strictly blocking and must never be called from an Interrupt Service Routine (ISR) because it triggers a context switch. It is thread-safe, but it requires the caller to hold the associated mutex; failing to do so will trigger a safeguard error. The most common beginner mistake is forgetting to wrap the condition check in a 'while' loop; because of spurious wakeups or multiple threads competing for the mutex after a signal, the condition might no longer be true when the thread resumes. Always ensure the mutex is locked before calling this function, as the function will automatically release it upon blocking and re-acquire it before returning to your code.

Uso:

- This function is the standard mechanism for implementing producer-consumer patterns or resource synchronization where a thread needs to wait for a specific state change. It is typically used in a sequence: first, lock the mutex, then enter a 'while' loop checking the condition, call ca_cond_wait inside the loop, and finally perform the protected operation once the condition is met. It is always paired with ca_cond_signal or ca_cond_broadcast, which are called by other threads to wake up the waiting task. Use this to avoid 'busy-waiting' (polling), which wastes CPU cycles and prevents other tasks from executing.

Esempio:

```
/* Producer-Consumer example: waiting for space in a buffer */
ca_mutex_lock(&buffer_mutex);

/* Always check the condition in a while loop to handle spurious wakeups */
while(ca_queue_heap_is_full(&my_queue)) {
    /* Releases buffer_mutex and blocks until signaled */
    ca_cond_wait(&buffer_not_full, &buffer_mutex);
}

/* Mutex is re-acquired here automatically before proceeding */
ca_queue_heap_send(&my_queue, item);

/* Notify consumers that data is available */
ca_cond_signal(&buffer_not_empty);

ca_mutex_unlock(&buffer_mutex);
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_cond_wait() in my caOS application?
A: You should use ca_cond_wait() when a task needs to wait for a specific condition to be met before proceeding. It is typically used in conjunction with ca_cond_signal() or ca_cond_broadcast() to coordinate between tasks. The function allows a task to release a mutex while waiting, preventing deadlock situations.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_cond_wait() with caOS types?
A: ca_mutex_t mtx; caCondVar cv; ca_mutex_init(&mtx); ca_cond_init(&cv); ca_mutex_lock(&mtx ca_cond_wait(&cv, &mtx); ca_mutex_unlock(&mtx);

Parameters conf:0.95 **Q:** What are the valid values for the parameters of ca_cond_wait()?
A: The first parameter, ca_condvar_t *cv, must point to a valid initialized condition variable. The second parameter, ca_mutex_t *mtx, must point to a mutex that is already acquired by the calling task. Both parameters must not be NULL.

Return conf:0.90 **Q:** Does ca_cond_wait() return any value, and what does it signify?
A: ca_cond_wait() does not return any value. It is a void function and does not fail in the traditional sense. However, it may block indefinitely if no other task calls ca_cond_signal() or ca_cond_broadcast().

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_cond_wait()?
A: Common mistakes include calling ca_cond_wait() without first acquiring the mutex, or passing NULL pointers for either parameter. If the mutex is not owned by the calling task, the behavior is undefined. Additionally, calling ca_cond_wait() from an ISR is not allowed.

Behavior conf:0.90 **Q:** Is ca_cond_wait() blocking or non-blocking, and is it safe to call from an ISR?
A: ca_cond_wait() is a blocking function; it suspends the calling task until a condition is signaled. It is not safe to call from an ISR, as it is not ISR-safe. It also requires that the mutex be held by the calling task, making it not thread-safe in a general sense.

`ca_cordic_calc`

Header: `caos/drivers/inc/cordic.h:49`
Source: `caos/drivers/src/cordic.c:27:46`
GUARD: `EN_CORDIC`

```
bool ca_cordic_calc(ca_cordic_func_t func, u32 scale, i32 arg1, i32 arg2
    , i32 *res1, i32 *res2); /* res2 may be NULL; 0 when unused */
```

Performs hardware-accelerated mathematical operations using the STM32 CORDIC peripheral.

Parametri:

Nome	Tipo	Descrizione
`func`	`ca_cordic_func_t`	The specific mathematical operation to perform (e.g., SIN, COS, PHASE, SQRT). Must be a valid enum value.
`scale`	`u32`	Scaling factor for the result, ranging from 0 to 7. Higher values increase precision for specific operations.
`arg1`	`i32`	The primary input argument for the calculation.
`arg2`	`i32`	The secondary input argument. If the operation only requires one argument, this value is ignored.
`res1`	`i32 *`	Pointer to store the primary result. Must not be NULL.
`res2`	`i32 *`	Pointer to store the secondary result (if applicable). Can be NULL if the second result is not needed.

Ritorna:
Returns true if the calculation was successful. Returns false if the CORDIC hardware is not initialized, if the function type is invalid, if the scale is out of range (0-7), or if the res1 pointer is NULL.

Note:

- This function is thread-safe because it uses critical sections to protect the hardware registers during the calculation. It is a blocking function, meaning the calling task will wait until the hardware completes the math operation before returning. It is NOT safe to call this function from an ISR because it uses critical sections that may interfere with system scheduling. Before using this, you must ensure ca_cordic_init() has been called successfully; otherwise, all calls will return false. A common mistake is passing a NULL pointer for res1, which will cause the function to fail immediately.

Uso:

- This function is used for high-performance trigonometry or square root calculations where CPU cycles are critical. It is typically used in a loop for signal processing or coordinate transformations. The standard workflow is: call ca_cordic_init() during system startup, then call ca_cordic_calc() whenever a math operation is required. It is often paired with higher-level wrappers like ca_cordic_sincos() or ca_cordic_polar() for cleaner code readability.

Esempio:

```
/* Ensure the CORDIC hardware is ready before use */
if (ca_cordic_init()) {
    i32 sin_val = 0;
    i32 cos_val = 0;
    i32 angle = 1073741824; /* Example Q31 angle */

    /* Perform hardware-accelerated SIN/COS calculation */
    if (ca_cordic_calc(CA_CORDIC_SIN, 0, angle, 0, &sin_val, &cos_val)) {
        /* Calculation successful, results are in sin_val and cos_val */
    } else {
        /* Handle error: hardware failure or invalid parameters */
    }
} else {
    /* Handle initialization failure */
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_cordic_calc in my caOS application?
A: You should use ca_cordic_calc when you require high-performance mathematical operations like trigonometry or square roots where CPU cycles are critical. It is typically used in signal processing loops or coordinate transformations.

Code example conf:1.00 **Q:** Provide a minimal example of how to use ca_cordic_calc to calculate a square root.
A: i32 result; if (ca_cordic_calc(SQRT, 0, 100, 0, &result, NULL)) { /* result now holds the square root */ }

Parameters conf:1.00 **Q:** What are the constraints on the 'scale' parameter in ca_cordic_calc?
A: The scale parameter must be a u32 value ranging from 0 to 7. Higher values increase precision for specific operations. If the scale is outside this range, the function will return false.

Return conf:1.00 **Q:** What does a return value of false indicate for ca_cordic_calc?
A: A return value of false indicates the calculation failed. This occurs if the CORDIC hardware is not initialized, the function type is invalid, the scale is out of range (0-7), or the res1 pointer is NULL.

Error/Edge case conf:1.00 **Q:** What is a common mistake that causes ca_cordic_calc to fail immediately?
A: A common mistake is passing a NULL pointer for the res1 parameter, which will cause the function to fail immediately and return false.

Behavior conf:1.00 **Q:** Is ca_cordic_calc safe to call from an Interrupt Service Routine (ISR)?
A: No, it is not safe to call this function from an ISR because it uses critical sections that may interfere with system scheduling. Additionally, it is a blocking function that waits for the hardware to complete the operation.

`ca_cordic_hw_calc`

Header: `caos/drivers/inc/ca_glue_hw.h:382`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_cordic_hw_calc(u32 func, u32 scale, u32 nargs, u32 nres,
    i32 arg1, i32 arg2, i32 *res1, i32 *res2); /* one q1.31 operation,
    caller holds the lock */
```

Performs a low-level CORDIC hardware calculation using fixed-point Q1.31 arithmetic.

Parametri:

Nome	Tipo	Descrizione
`func`	`u32`	The CORDIC operation code (e.g., sine, cosine, arctan).
`scale`	`u32`	Scaling factor for the CORDIC algorithm.
`nargs`	`u32`	Number of input arguments (1 or 2).
`nres`	`u32`	Number of expected results (1 or 2).
`arg1`	`i32`	First input argument in Q1.31 format.
`arg2`	`i32`	Second input argument in Q1.31 format (ignored if nargs=1).
`res1`	`i32 *`	Pointer to store the first result.
`res2`	`i32 *`	Pointer to store the second result (ignored if nres=1).

Ritorna:
This function returns void. It assumes the hardware is available and the caller has already secured exclusive access.

Note:

- This function is NOT thread-safe and does not perform any locking internally. It is designed to be called only after the caller has successfully acquired a critical section or mutex to prevent other tasks from corrupting the CORDIC hardware state. It is not safe to call from an ISR unless the hardware is guaranteed to be idle and the lock is held. A common beginner mistake is calling this without wrapping it in ca_enter_critical() and ca_exit_critical(), which leads to race conditions where one task reads the result intended for another task.

Uso:

- This function is the low-level driver interface used by higher-level math functions like ca_cordic_sincos. It is typically used in a sequence: first, enter a critical section to ensure exclusive hardware access, then call this function to perform the calculation, and finally exit the critical section. It should only be used when you need direct, high-performance access to the CORDIC peripheral and are managing the hardware state manually.

Esempio:

```
/* Example: Calculating sine and cosine using the CORDIC hardware */
#include "caos.h"

void calculate_trig(i32 angle_q31, i32 *sin_out, i32 *cos_out) {
    i32 s, c;
    /* Protect the hardware from concurrent access by other tasks */
    ca_enter_critical();

    /* Perform the CORDIC calculation: func=0 (sine/cosine), scale=0, nargs=1,
    nres=2 */
    ca_cordic_hw_calc(0, 0, 1, 2, angle_q31, 0, &s, &c);

    ca_exit_critical();

    *sin_out = s;
    *cos_out = c;
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_cordic_hw_calc instead of higher-level math functions?
A: You should use ca_cordic_hw_calc when you require direct, high-performance access to the CORDIC peripheral and need to manually manage the hardware state for specific calculations not covered by higher-level wrappers like ca_cordic_sincos.

Code example conf:1.00 **Q:** How do I perform a CORDIC calculation using caOS types?
A: You must wrap the call in a critical section: ca_enter_critical(); i32 r1, r2; ca_cordic_hw_calc(func, scale, 1, 1, arg1, 0, &r1, NULL); ca_exit_critical();

Parameters conf:1.00 **Q:** What do the nargs and nres parameters represent in ca_cordic_hw_calc?
A: nargs specifies the number of input arguments (1 or 2) to be processed, while nres specifies the number of expected results (1 or 2) to be read from the hardware.

Error/Edge case conf:1.00 **Q:** What is a common mistake when calling ca_cordic_hw_calc?
A: A common beginner mistake is calling the function without wrapping it in ca_enter_critical() and ca_exit_critical(), which leads to race conditions where one task reads the result intended for another task.

Behavior conf:1.00 **Q:** Is ca_cordic_hw_calc thread-safe or ISR-safe?
A: The function is not thread-safe and does not perform internal locking. It is not safe to call from an ISR unless the hardware is guaranteed to be idle and the caller has already secured exclusive access.

HW differences conf:1.00 **Q:** How does the implementation of ca_cordic_hw_calc differ across STM32 families?
A: On H5xx and U5xx families, the function directly interacts with the CORDIC->CSR, CORDIC->WDATA, and CORDIC->RDATA registers. On F1xx, F4xx, F7xx, and H7xx families, the function is currently implemented as a stub that performs no operations.

`ca_cordic_hw_init`

Header: `caos/drivers/inc/ca_glue_hw.h:381`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_cordic_hw_init(void); /* clock on; false when the part
   has no CORDIC */
```

Initializes the CORDIC hardware peripheral by enabling its clock in the RCC register.

Ritorna:
Returns true if the CORDIC hardware is present and successfully initialized, false if the current STM32 chip does not support the CORDIC peripheral.

Note:

- This function is a low-level hardware abstraction layer (HAL) call. It is not thread-safe and should only be called once during the system startup sequence, typically within the board support package (BSP) initialization. It is non-blocking and executes extremely quickly, but it must not be called from an ISR because it performs direct register manipulation that assumes a stable system state. A common mistake is to call this function multiple times or to attempt to use CORDIC functions before verifying the return value of this initialization.

Uso:

- This function is the mandatory prerequisite for all CORDIC operations. It is typically called inside the higher-level ca_cordic_init() function, which sets a static flag (s_cordic_ok) used by subsequent calculation functions like ca_cordic_calc, ca_cordic_sincos, or ca_cordic_polar to ensure the hardware is ready. If this function returns false, the application must avoid calling any CORDIC calculation functions to prevent undefined behavior or bus faults.

Esempio:

```
/* Initialize the CORDIC peripheral during system startup */
if (ca_cordic_init()) {
    /* Hardware is ready, safe to perform calculations */
    i32 res1, res2;
    /* Calculate sine and cosine of an angle */
    if (ca_cordic_sincos(0, 1024, 0, &res1, &res2)) {
        /* Success: res1 and res2 contain the results */
    }
} else {
    /* CORDIC not available on this MCU, use software math library instead */
    /* Perform software-based trigonometric calculation */
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_cordic_hw_init in my caOS application?

A: It should be called exactly once during the system startup sequence, typically within the board support package (BSP) initialization. It is a mandatory prerequisite for all CORDIC operations and should not be called from an ISR.

Code example conf:0.90 **Q:** How do I correctly initialize the CORDIC peripheral in my main startup code?

A: You should check the return value to ensure the hardware is supported before proceeding: if (ca_cordic_hw_init()) { /* Proceed with CORDIC operations */ } else { /* Handle unsupported hardware */ }.

Return conf:1.00 **Q:** What does the boolean return value of ca_cordic_hw_init signify?

A: The function returns true if the CORDIC hardware is present and successfully initialized by enabling its clock. It returns false if the current STM32 chip does not support the CORDIC peripheral, in which case the application must avoid calling any CORDIC calculation functions to prevent undefined behavior or bus faults.

HW differences conf:1.00 **Q:** Why does ca_cordic_hw_init return false on some STM32 families like F1xx or F4xx?

A: The function returns false on families like F1xx, F4xx, F7xx, and H7xx because those specific implementations do not support the CORDIC peripheral. In contrast, for H5xx and U5xx families, the function successfully enables the peripheral by setting the RCC_AHB1ENR_CORDICEN bit in the RCC->AHB1ENR register.

`ca_cordic_init`

Header: `caos/drivers/inc/cordic.h:48`
Source: `caos/drivers/src/cordic.c:21:25`
GUARD: `EN_CORDIC`

```
bool ca_cordic_init(void); /* false: no CORDIC on this part */
```

Initializes the hardware CORDIC coprocessor if available on the current STM32 chip.

Ritorna:
Returns true if the CORDIC hardware is present and successfully initialized; returns false if the hardware is missing or initialization failed.

Note:

- This function is a hardware abstraction layer (HAL) wrapper. It is not thread-safe in the sense that it should be called once during system startup before any tasks attempt to use CORDIC calculations. It is not intended to be called from an ISR. The most common mistake is failing to check the return value; if you call other ca_cordic_* functions without verifying this initialization, you may trigger a hard fault or receive undefined data because the peripheral registers do not exist on the target silicon.

Uso:

- Use this function during the system configuration phase, typically inside ca_os_configure or early in the main application entry point. It acts as a gatekeeper: if it returns false, the application should gracefully disable features that rely on trigonometric acceleration (like motor control loops or signal processing) to prevent runtime crashes. It is the mandatory prerequisite for all other ca_cordic_* functions.

Esempio:

```
void ca_os_configure(void) {
    /* Check if the CORDIC hardware is available on this specific MCU */
    if (!ca_cordic_init()) {
```

```
        /* Log error or disable features that require CORDIC */
        /* CA_EXAMPLE_UNSUPPORTED() is a placeholder for custom error handling */
        return;
    }

    /* Now it is safe to use CORDIC for calculations */
    float angle = 0.5f;
    float sin_val, cos_val;
    ca_cordic_sincos(angle, &sin_val, &cos_val);
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to call ca_cordic_init in my caOS application?
A: You should call ca_cordic_init during the system configuration phase, such as inside ca_os_configure or early in your main application entry point. It must be called once during system startup before any tasks attempt to use CORDIC calculations, as it is not thread-safe and not intended for use within an ISR.

Code example conf:1.00 **Q:** How do I correctly implement the initialization check for the CORDIC peripheral?
A: You must verify the return value to ensure the hardware exists before proceeding. Example: if (ca_cordic_init()) { /* Proceed with CORDIC operations */ } else { /* Disable features relying on trigonometric acceleration */ }

Return conf:1.00 **Q:** What do the return values of ca_cordic_init signify and why might it return false?
A: The function returns true if the CORDIC hardware is present and successfully initialized. It returns false if the hardware is missing on the target silicon or if the initialization process fails. Failing to check this return value before calling other ca_cordic_* functions can lead to hard faults or undefined data because the peripheral registers may not exist.

`ca_cordic_polar`

Header: `caos/drivers/inc/cordic.h:52`
Source: `caos/drivers/src/cordic.c:54:57`
GUARD: `EN_CORDIC`

```
bool ca_cordic_polar(i32 x, i32 y, i32 *angle, i32 *modulus); /* angle = rad/pi */
```

Converts Cartesian coordinates (x, y) into polar coordinates (angle, modulus) using the CORDIC hardware peripheral.

Parametri:

Nome	Tipo	Descrizione
`x`	`i32`	The X-coordinate in Q31 fixed-point format.
`y`	`i32`	The Y-coordinate in Q31 fixed-point format.
`angle`	`i32 *`	Pointer to store the resulting angle (normalized as rad/pi). Must be a valid memory address.
`modulus`	`i32 *`	Pointer to store the resulting magnitude (modulus). Must be a valid memory address.

Ritorna:
Returns true if the calculation completed successfully, false if the input parameters were invalid or the hardware failed to compute.

Note:

- This function utilizes the CORDIC hardware accelerator, which is significantly faster than software-based trigonometric calculations. It is thread-safe assuming the underlying hardware peripheral is not being accessed concurrently by another task or ISR. It is non-blocking in the sense that it triggers the hardware and waits for the result, but it should not be called from an ISR unless the hardware is guaranteed to be idle. A common beginner mistake is passing NULL pointers for the output parameters, which will cause a hard fault or memory corruption.

Uso:

- This function is typically used in signal processing, motor control, or coordinate transformation tasks where Cartesian-to-Polar conversion is required. It is often paired with ca_cordic_calc for more complex operations or ca_cordic_sincos for the inverse operation. Before using this, ensure the CORDIC peripheral has been initialized via the system startup or a specific initialization routine if required by your board configuration.

Esempio:

```
i32 x = 0x40000000; /* 0.5 in Q31 */
i32 y = 0x40000000; /* 0.5 in Q31 */
i32 angle = 0;
i32 modulus = 0;

/* Convert Cartesian (0.5, 0.5) to Polar coordinates */
if (ca_cordic_polar(x, y, &angle, &modulus)) {
    /* Calculation successful, angle and modulus are updated */
    ca_printf("Angle: %ld, Modulus: %ld\n", angle, modulus);
} else {
    /* Handle error: check inputs or hardware status */
    ca_printf("CORDIC calculation failed!\n");
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** In what scenarios should I use ca_cordic_polar instead of standard math libraries?
A: You should use ca_cordic_polar for signal processing, motor control, or coordinate transformation tasks. It is preferred because it utilizes the CORDIC hardware accelerator, which is significantly faster than software-based trigonometric calculations.

Code example conf:0.90 **Q:** Provide a minimal example of how to call ca_cordic_polar in caOS.
A: i32 x = 1073741824; i32 y = 1073741824; i32 angle, modulus; if (ca_cordic_polar(x, y, &angle, &modulus)) { /* Success */ }

Parameters conf:1.00 **Q:** What format do the x and y parameters expect, and what happens if I pass NULL pointers?
A: The x and y parameters must be in Q31 fixed-point format. Passing NULL pointers for the angle or modulus parameters is a common mistake that will cause a hard fault or memory corruption.

Return conf:1.00 **Q:** What does the boolean return value indicate, and what causes a failure?
A: The function returns true if the calculation completes successfully. It returns false if the input parameters are invalid or if the CORDIC hardware fails to compute the result.

`ca_cordic_sincos`

Header: `caos/drivers/inc/cordic.h:51`
Source: `caos/drivers/src/cordic.c:48:52`
GUARD: `EN_CORDIC`

```
bool ca_cordic_sincos(i32 angle, i32 *sin_q31, i32 *cos_q31); /* angle =
    rad/pi */
```

Calculates the sine and cosine of a given angle using the hardware CORDIC peripheral.

Parametri:

Nome	Tipo	Descrizione
`angle`	`i32`	The input angle represented in Q31 format, where the full circle (2*pi) is mapped to the range [-1, 1) of the i32 type.
`sin_q31`	`i32 *`	Pointer to an i32 variable where the resulting sine value (in Q31 format) will be stored.
`cos_q31`	`i32 *`	Pointer to an i32 variable where the resulting cosine value (in Q31 format) will be stored.

Ritorna:

Returns true if the calculation was successful, false otherwise. A false return typically indicates an invalid pointer or a failure in the underlying hardware peripheral communication.

Note:

- This function utilizes the STM32 hardware CORDIC peripheral, which is significantly faster than software-based trigonometric calculations. It is thread-safe, but because it relies on a shared hardware resource, it may block the calling task if the CORDIC unit is currently busy with another operation. It is generally not recommended to call this from an ISR unless the hardware is guaranteed to be idle, as blocking in an ISR is strictly forbidden in RTOS environments. Ensure the CORDIC peripheral is properly initialized before use; failing to do so will result in undefined behavior or incorrect calculations.

Uso:

- This function is used in digital signal processing, motor control, or any application requiring fast trigonometric transformations. It is typically used in a loop or a periodic task to update control signals. It is often paired with ca_cordic_polar for coordinate conversions. Before calling this, ensure your application has initialized the necessary hardware resources, and always check the boolean return value to ensure the calculation completed successfully before using the results.

Esempio:

```
/* Example: Calculate sine and cosine for a specific angle */
#include "caos.h"

void task_trig_calc(void) {
    i32 angle = 0x40000000; /* Represents pi/2 in Q31 */
    i32 sin_val = 0;
    i32 cos_val = 0;

    /* Perform the calculation */
    if (ca_cordic_sincos(angle, &sin_val, &cos_val)) {
        /* Calculation successful, use sin_val and cos_val */
        /* ca_printf("Sin: %d, Cos: %d\n", sin_val, cos_val); */
    } else {
        /* Handle error: log failure or retry */
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_cordic_sincos instead of standard math library functions?
A: You should use ca_cordic_sincos when you need fast trigonometric calculations for applications like motor control or digital signal processing. It utilizes the STM32 hardware CORDIC peripheral, which is significantly faster than software-based calculations. Note that it may block the calling task if the hardware is busy, so it should not be called from an ISR.

Code example conf:0.90 **Q:** Provide a minimal example of how to call ca_cordic_sincos in caOS.
A: i32 angle = 0x20000000; i32 sin_val, cos_val; if (ca_cordic_sincos(angle, &sin_val, &cos_val)) { /* Use sin_val and cos_val */ }

Parameters conf:1.00 **Q:** How is the angle parameter formatted for ca_cordic_sincos?
A: The angle parameter is an i32 in Q31 format. The full circle (2*pi) is mapped to the range [-1, 1) of the i32 type.

Return conf:1.00 **Q:** What does a false return value from ca_cordic_sincos indicate?
A: A false return indicates that the calculation was not successful. This typically happens due to an invalid pointer being passed to the function or a failure in the underlying hardware peripheral communication.

`ca_crc32_get`

Header: `caos/drivers/inc/crc32.h:50`
Source: `caos/drivers/src/crc32.c:54:86`
GUARD: `EN_CRC32`

```
1 extern u32 ca_crc32_get(u8 *buff, u32 size)
```

Calculates the CRC32 checksum of a data buffer using the STM32 hardware CRC peripheral.

Parametri:

Nome	Tipo	Descrizione
`buff`	`u8 *`	Pointer to the start of the data buffer. Must be a valid memory address; passing NULL will cause a hard fault.
`size`	`u32`	The number of bytes to process. A size of 0 is technically valid but will return the CRC reset value.

Ritorna:
Returns a 32-bit unsigned integer representing the calculated CRC checksum.

Note:

- This function is thread-safe because it uses a mutex to protect the shared hardware CRC peripheral. It is a blocking function, meaning the calling task will wait if another task is currently calculating a CRC. You must NEVER call this function from an Interrupt Service Routine (ISR) because the mutex lock will cause a system hang or kernel panic. The most common mistake is passing an unaligned buffer or an incorrect size, which might lead to unexpected results if the hardware peripheral expects specific alignment; however, this implementation handles tail bytes manually to ensure correctness. Ensure the CRC peripheral clock is enabled in your system initialization before calling this function.

Uso:

- Use this function whenever you need to verify data integrity, such as checking if a configuration block in Flash has been corrupted or validating a packet received over a communication interface. It is typically used in a sequence: first, you prepare your data buffer,

then you call ca_crc32_get to generate the checksum, and finally, you compare the result against a known good value or store it for later verification. It is often paired with memory management functions to ensure the buffer passed to it remains constant during the calculation process.

Esempio:

```c
/* Example: Calculating CRC32 for a data buffer */
#define BUFFER_SIZE 256
u8 my_data[BUFFER_SIZE];

void task_integrity_check(void) {
    /* Fill buffer with data */
    fill_buffer(my_data, BUFFER_SIZE);

    /* Calculate CRC32 - this will block if another task is using the CRC unit
    */
    u32 checksum = ca_crc32_get(my_data, BUFFER_SIZE);

    /* Log the result */
    ca_stdout_printf_n("Calculated CRC: 0x%08X", checksum);
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_crc32_get() in a caOS application?
A: You should use ca_crc32_get() when you need to calculate a CRC32 checksum for data integrity verification, such as validating received packets or ensuring data consistency in memory. It's particularly useful in communication protocols or file systems where data corruption detection is required.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_crc32_get() with caOS types?
A: u8 data[] = {0x48, 0x65, 0x6C, 0x6C, 0x6F}; u32 crc = ca_crc32_get(data, 5); // crc now contains the CRC32 of the string "Hello"

Parameters conf:0.95 **Q:** What are the valid and invalid values for the parameters of ca_crc32_get()?
A: The buffer pointer 'buff' must be valid and non-NULL, otherwise the function will crash or behave unpredictably. The size parameter 'size' must be greater than zero; otherwise, the result is undefined or zero. Passing NULL or zero size leads to undefined behavior.

Return conf:0.90 **Q:** What does ca_crc32_get() return, and what causes it to fail?
A: The function returns a 32-bit CRC32 value calculated from the input buffer. If invalid parameters are passed (e.g., NULL buffer or zero size), the return value is undefined, and the behavior is unpredictable.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_crc32_get()?
A: Common mistakes include passing a NULL buffer pointer or a size of zero, which leads to undefined behavior or crashes. Another mistake is calling this function from an ISR, which is not allowed and may cause deadlocks or undefined behavior due to mutex usage.

Behavior conf:0.95 **Q:** Is ca_crc32_get() thread-safe, ISR-safe, and blocking?
A: Yes, ca_crc32_get() is thread-safe because it uses a mutex internally. It is not ISR-safe and must not be called from an ISR. The function is blocking, as it waits for the mutex to be released if another thread is currently using it.

ca_crc32_hw_init

Header: caos/drivers/inc/ca_glue_hw.h:83
GUARD: EN_CA_GLUE_HW

```
extern void ca_crc32_hw_init(void); /* abilita il clock e resetta il
   periferico CRC hardware */
```

Enables the hardware clock for the CRC peripheral to allow subsequent data processing.

Ritorna:
This function returns void; it does not provide feedback on whether the clock was successfully enabled.

Note:

- This function is non-blocking and executes almost instantaneously as it only performs a single register write to the RCC (Reset and Clock Control) unit. It is thread-safe in the sense that it performs an atomic bitwise OR operation, but it is not intended to be called repeatedly; it should be called exactly once during system initialization. It is safe to call from an ISR, though this is rarely necessary as hardware initialization should occur during the boot sequence. The most common beginner mistake is calling this function multiple times or failing to call it before attempting to write data to the CRC registers, which will result in a HardFault or a bus error because the peripheral remains in a powered-down state.

Uso:

- This function is the mandatory first step in the CRC peripheral lifecycle. It is typically called inside a driver-level initialization function (like ca_crc32_init) before any configuration registers are accessed or any data is fed into the CRC engine. It is always paired with subsequent register resets (setting the CRC_CR_RESET bit) and synchronization primitives (like ca_mutex_init) to ensure that the hardware is ready and protected from concurrent access by multiple tasks.

Esempio:

```
void system_init(void) {
    /* Enable the hardware clock for the CRC peripheral */
    ca_crc32_hw_init();

    /* Reset the CRC peripheral to a known state */
    CRC->CR |= CRC_CR_RESET;

    /* Initialize the mutex to prevent concurrent access from different tasks */
    ca_mutex_init(&crc_mutex);
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When and why should I call ca_crc32_hw_init() in a caOS project?
A: You should call ca_crc32_hw_init() once during system initialization, typically in a startup task or early boot code. It enables the CRC hardware peripheral clock and resets the CRC unit. This function must only be called from a non-ISR initialization task, not from interrupt service routines, as it modifies system registers (RCC) that could interfere with the OS or cause deadlocks. It is a one-time initialization function and should not be called more than once to avoid undefined behavior.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_crc32_hw_init() with caOS types?
A:

```c
#include "caos.h"

void init_task(void) {
    ca_crc32_hw_init();
    // CRC hardware is now ready for use
}
```

HW differences conf:0.90 **Q:** How does ca_crc32_hw_init() differ across STM32 MCU families in terms of hardware implementation?
A: The function enables the CRC peripheral using different RCC registers depending on the MCU family. For F1xx, it sets RCC_AHBENR_CRCEN; for F4xx, F7xx, and H5xx, it uses RCC_AHB1ENR_CRCEN; and for H7xx, it uses RCC_AHB4ENR_CRCEN. These differences reflect the varying RCC register layouts across STM32 families, where the CRC peripheral is enabled through different AHB clock enable bits.

`ca_crc32_init`

Header: `caos/drivers/inc/crc32.h:42`
Source: `caos/drivers/src/crc32.c:38:52`
GUARD: `EN_CRC32`

```c
extern void ca_crc32_init(void)
```

Initializes the hardware CRC32 peripheral and its associated software synchronization primitives.

Ritorna:
This function returns void and does not provide a status code; it assumes the hardware initialization succeeds.

Note:

- This function is non-blocking and performs hardware-level configuration, including clock enabling and peripheral resetting. It is not thread-safe during the initialization phase itself, so it must be called during the system startup sequence before any threads are spawned. It is strictly forbidden to call this from an Interrupt Service Routine (ISR) because it initializes a mutex, which is a blocking synchronization primitive that cannot be used in interrupt context. A common beginner mistake is calling this function multiple times or after the OS scheduler has started, which could lead to race conditions or re-initialization of the mutex, potentially causing system instability or deadlocks.

Uso:

- This function is intended to be called exactly once during the system boot-up phase, typically inside the 'ca_os_configure' or main initialization routine. It acts as a prerequisite for any subsequent CRC calculation tasks. It is usually paired with 'ca_mutex_init' (called internally) to ensure that multiple threads can safely share the single hardware CRC peripheral without corrupting each other's calculations. Once initialized, the CRC module is ready to be used by other application threads to verify data integrity.

Esempio:

```c
void ca_os_configure(void) {
    /* Initialize the CRC32 hardware and its mutex before starting tasks */
    ca_crc32_init();

    /* Now the CRC module is ready for use by threads */
    /* Example: uint32_t result = ca_crc32_get(buffer, size); */
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_crc32_init() in my caOS application?
A: You should call ca_crc32_init() once during system initialization, before using any CRC32 functions. It initializes the hardware CRC32 module and sets up an internal mutex for thread-safe access. It must be called before any other CRC32-related functions and should not be called from an ISR.

Code example conf:0.90 **Q:** Can you show a minimal example of using ca_crc32_init() with caOS types?
A:

```
u32 data = 0x12345678;
ca_crc32_init();
// Now CRC32 module is ready for use
```

Return conf:0.85 **Q:** What happens if ca_crc32_init() fails to initialize the CRC32 hardware?
A: The function does not return any value and cannot fail. However, it may not properly initialize the CRC32 hardware if the hardware is not present or accessible. The function is guaranteed to not fail but may not fully initialize the module in such cases.

Error/Edge case conf:0.80 **Q:** What are common mistakes when using ca_crc32_init()?
A: Common mistakes include calling ca_crc32_init() from an ISR, calling it multiple times without understanding that the mutex is reininitialized but the CRC module is reset, or calling it after other CRC32 functions have already been used. It should only be called once during system initialization.

`ca_critical_nesting_level`

Header: `caos/inc/sync.h:303`
Source: `caos/src/sync.c:744:746`
GUARD: `EN_SYNC`

```
u32 ca_critical_nesting_level(void)
```

Returns the current depth of nested critical sections in the system.

Ritorna:
Returns a u32 integer representing the current nesting level. A value of 0 indicates that the system is currently outside of any critical section.

Note:

- This function is non-blocking and thread-safe as it simply reads a global counter variable. It is safe to call from both task context and Interrupt Service Routines (ISRs). The most common mistake is assuming that a non-zero value implies a specific lock is held; this function only tracks the depth of the global interrupt-disable mechanism. If you call this while interrupts are enabled, it will correctly return 0, but it does not provide information about which specific code block initiated the critical section. It is primarily used for debugging, diagnostic logging, or verifying that a function is being called within a protected context.

Uso:

- This function is typically used for system health monitoring or debugging complex synchronization logic where you need to verify if a code path is running with interrupts disabled. It is often paired with ca_enter_critical() and ca_exit_critical() to ensure that state changes are atomic. In advanced scenarios, developers use it to assert that a function is only called when the system is in a 'safe' state, preventing race conditions by checking the nesting level before accessing shared hardware registers or global variables.

Esempio:

```
/* Example: Logging the nesting level during a shared resource update */

/* Assume ca_enter_critical() was called previously */
shared_counter++;
write_count++;

u32 sc = shared_counter;
u32 wc = write_count;

/* Capture the nesting level before exiting the critical section */
u32 nesting = ca_critical_nesting_level();

/* Now safe to exit the critical section */
ca_exit_critical();

/* Log the captured state for debugging purposes */
ca_stdout_printf_n("WRITER: counter=%u writes=%u nesting_was=%u", sc, wc,
    nesting);

/* Wait before the next cycle */
ca_os_sleep(300);
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should a developer call ca_critical_nesting_level() and why is it useful?

A: A developer should call ca_critical_nesting_level() to check the current nesting level of critical sections, which is useful for debugging and ensuring correct nesting of critical sections. It helps verify that every 'ca_enter_critical()' call has a corresponding 'ca_exit_critical()' call. This function can be called from any context, including ISRs, and is thread-safe, making it ideal for monitoring code criticality without introducing blocking or concurrency issues.

Code example conf:0.90 **Q:** Can you provide a minimal C example using caOS types that demonstrates how to use ca_critical_nesting_level()?

A: u32 nesting_level; ca_enter_critical(); nesting_level = ca_critical_nesting_level(); ca_exit_critical(); // nesting_level will be 1 here ca_enter_critical(); ca_enter_critical(); nesting_level = ca_critical_nesting_level(); ca_exit_critical(); ca_exit_critical(); // nesting_level will be 0 here

Return conf:0.90 **Q:** What do the return values of ca_critical_nesting_level() mean and what causes a failure?

A: The function returns a u32 value representing the number of times 'ca_enter_critical()' has been called without a corresponding 'ca_exit_critical()'. A return value of 0 indicates no active critical sections. The function is non-blocking and thread-safe, so it does not fail in the traditional sense. However, calling it without ever entering a critical section may result in a meaningless value, but it will not cause an error or crash.

`ca_dac_comp_init`

Header: `caos/drivers/inc/dac.h:53`
Source: `caos/drivers/src/dac.c:157:161`
GUARD: `EN_DAC`

```
bool ca_dac_comp_init(u8 dac_channel, ca_dac_comp_hyst_t hyst)
```

Initializes the hardware comparator for a specific DAC channel with a defined hysteresis level.

Parametri:

Nome	Tipo	Descrizione
`dac_channel`	`u8`	The DAC channel index to configure. Valid values are 1 or 2.
`hyst`	`ca_dac_comp_hyst_t`	The hysteresis level to apply. Must be a valid enum value up to CA_DAC_COMP_HYST_HIGH.

Ritorna:
Returns true if the initialization was successful, false if the channel index or hysteresis value is out of range or if the hardware initialization failed.

Note:

- This function is a wrapper that configures the internal comparator hardware associated with the DAC. It is not thread-safe and should be called during the initialization phase of your application or when reconfiguring the DAC peripheral. It is not intended to be called from an ISR. A common beginner mistake is passing an invalid channel number (e.g., 0 or 3) or an undefined hysteresis enum, which will immediately return false without modifying the hardware state.

Uso:

- Use this function when you need to set up a threshold-based trigger using the DAC comparator. It is typically used in a sequence: first call ca_dac_init to prepare the peripheral, then use ca_dac_comp_init to set the hysteresis, and finally use ca_dac_write to set the reference voltage. If you need to stop the comparator later, use ca_dac_comp_stop.

Esempio:

```
/* Initialize DAC channel 1 with medium hysteresis */
if (ca_dac_comp_init(1U, CA_DAC_COMP_HYST_MEDIUM)) {
    /* Set the reference voltage for the comparator */
    ca_dac_write(1U, 2048U);

    /* Check the current state of the comparator */
    if (ca_dac_comp_level()) {
        /* Comparator output is high */
    }
} else {
    /* Handle initialization error */
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_dac_comp_init in my application flow?
A: It should be called during the initialization phase or when reconfiguring the DAC. The recommended sequence is to call ca_dac_init first, then ca_dac_comp_init to set the hysteresis, and finally ca_dac_write to set the reference voltage.

Code example conf:1.00 **Q:** Provide a minimal example of how to initialize the DAC comparator using caOS types.
A: u8 channel = 1; ca_dac_comp_hyst_t level = CA_DAC_COMP_HYST_HIGH; if (ca_dac_comp_init(channel, level)) { /* Success */ }

Parameters conf:1.00 **Q:** What are the valid values for the dac_channel and hyst parameters?
A: The dac_channel parameter must be 1 or 2. The hyst parameter must be a valid ca_dac_comp_hyst_t enum value up to CA_DAC_COMP_HYST_HIGH.

Return conf:1.00 **Q:** What does a false return value from ca_dac_comp_init indicate?
A: A false return indicates that the initialization failed. This occurs if the dac_channel index is out of range (e.g., 0 or 3), the hysteresis value is undefined, or the hardware initialization failed.

`ca_dac_comp_level`

Header: `caos/drivers/inc/dac.h:54`
Source: `caos/drivers/src/dac.c:163:166`
GUARD: `EN_DAC`

```
bool ca_dac_comp_level(void); /* true = PB2 above the DAC threshold */
```

Reads the current state of the DAC comparator pin (PB2) relative to the configured threshold.

Ritorna:
Returns true if the voltage on the comparator pin is above the DAC threshold, and false if it is below.

Note:

- This function is non-blocking and executes immediately by reading the hardware register state. It is thread-safe in the sense that it performs a single atomic read of the hardware, but it does not provide any synchronization or debouncing logic. It is safe to call from an ISR, as it does not involve any OS primitives or blocking calls. A common mistake is assuming the signal is noise-free; because this is a raw hardware read, you should implement software filtering or multiple reads if you need to ensure the signal has truly settled.

Uso:

- This function is typically used in control loops or calibration routines where you need to determine if an external signal has crossed a specific voltage level defined by the DAC. It is usually paired with ca_dac_comp_init to set up the threshold and hysteresis before polling. In a typical sequence, you call ca_dac_comp_init once during system startup, then periodically call ca_dac_comp_level within a task or a timer-driven loop to monitor the signal state, and finally call ca_dac_comp_stop when monitoring is no longer required.

Esempio:

```
/* Initialize DAC comparator on channel 1 with high hysteresis */
if (ca_dac_comp_init(1, CA_DAC_COMP_HYST_HIGH)) {
    /* Poll the comparator state to check if the signal is high */
    if (ca_dac_comp_level()) {
        /* Signal is above threshold, perform action */
    } else {
        /* Signal is below threshold */
    }
}

/* Stop the comparator when finished */
ca_dac_comp_stop();
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_dac_comp_level in my caOS application?
A: You should use this function in control loops or calibration routines to determine if an external signal on PB2 has crossed a voltage threshold defined by the DAC. It is typically called after initializing the comparator with ca_dac_comp_init and should be polled periodically within a task or timer-driven loop.

Code example conf:1.00 **Q:** How do I implement a simple check of the DAC comparator state using caOS types?
A: After calling ca_dac_comp_init(), you can check the state as follows: bool status = ca_dac_comp_level(); if (status) { /* Voltage is above threshold */ } else { /* Voltage is below threshold */ }

Return conf:1.00 **Q:** What do the return values of ca_dac_comp_level represent, and are there any failure modes I should handle?
A: The function returns true if the voltage on PB2 is above the DAC threshold and false if it is below. There are no failure modes defined for this function as it performs a non-blocking, atomic read of the hardware register. Note that the function does not provide debouncing, so you may need to implement software filtering to handle signal noise.

`ca_dac_comp_stop`

Header: `caos/drivers/inc/dac.h:55`
Source: `caos/drivers/src/dac.c:168:171`
GUARD: `EN_DAC`

```
void ca_dac_comp_stop(void)
```

Stops the DAC comparator hardware operation.

Ritorna:
void

Note:

- This function immediately halts the DAC comparator peripheral. It is non-blocking and does not return a status, meaning it assumes the hardware is in a state where it can be safely stopped. It is not thread-safe by itself; if multiple tasks access the DAC hardware, you must ensure external synchronization. Do not call this from an ISR unless the underlying hardware driver explicitly supports it, as it may interfere with ongoing peripheral operations. A common mistake is calling this before the comparator has been initialized, which may lead to undefined behavior in the hardware registers.

Uso:

- This function is typically used during the teardown phase of a DAC-based measurement or signal processing task. It is usually paired with ca_dac_comp_init, which sets up the comparator, and is called when the monitoring task is finished or needs to be reset. In a typical lifecycle, you would initialize the DAC and comparator, perform your logic, and then call ca_dac_comp_stop to release the hardware resources or put the peripheral into a low-power state.

Esempio:

```
/* Initialize the DAC and the comparator */
if (ca_dac_init(&config) && ca_dac_comp_init(1U, 0U)) {
    /* Perform measurement logic */
    if (ca_dac_comp_level()) {
        /* Comparator triggered, handle logic */
    }

    /* Stop the comparator when finished to save power or prepare for re-init */
    ca_dac_comp_stop();
} else {
    /* Handle initialization error */
}
```

Q&A — fine-tuning review *2/2 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_dac_comp_stop in my application lifecycle?
A: You should call ca_dac_comp_stop during the teardown phase of a DAC-based measurement or signal processing task. It is typically used when the monitoring task is finished or needs to be reset, allowing you to release hardware resources or transition the peripheral into a low-power state.

Code example conf:0.95 **Q:** How do I implement a simple teardown sequence for the DAC comparator using caOS?
A: To safely stop the DAC comparator, ensure you have completed your signal processing logic and then invoke the stop function. Example:
void task_teardown(void) { /* Perform signal processing logic here */ ca_dac_comp_stop(); }

`ca_dac_hw_comp_init`

Header: `caos/drivers/inc/ca_glue_hw.h:99`
GUARD: `EN_CA_GLUE_HW`

```
1 extern bool ca_dac_hw_comp_init(u8 dac_channel, u8 hyst); /* COMP1: PB2
     against the DAC channel; false = no comparator */
```

{ "brief": "Initializes the internal hardware comparator using a DAC channel as the reference threshold.", "params": [{ "name": "dac_channel", "type": "u8", "desc": "The DAC channel to use as the reference (1 or 2). Invalid values will cause the function to return false." }, { "name": "hyst": "u8", "desc": "Hysteresis level to apply to the comparator. Higher values reduce noise-induced switching at the threshold." }], "returns": "Returns tr

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_dac_hw_comp_init function in my caOS project?
A: You should call this function when you need to use the internal hardware comparator (COMP1) with a DAC channel as the negative input reference. This is useful for threshold-based signal monitoring where the DAC provides a dynamic reference voltage against the input on PB2.

Code example conf:1.00 **Q:** Provide a minimal example of how to initialize the comparator using caOS types.
A: u8 channel = 1; u8 hysteresis = 2; if (ca_dac_hw_comp_init(channel, hysteresis)) { /* Comparator initialized successfully */ }

Error/Edge case conf:1.00 **Q:** What happens if I call ca_dac_hw_comp_init on an STM32F4xx or F1xx device?
A: The function will return false. On F1xx, F4xx, F7xx, and H5xx families, the hardware does not support this comparator configuration, and the implementation is a stub that performs no action other than returning false.

HW differences conf:1.00 **Q:** How does the implementation of ca_dac_hw_comp_init differ between H7xx and U5xx series?
A: The H7xx implementation enables the COMP12 peripheral via RCC->APB4ENR and configures the COMP1->CFGR register. The U5xx implementation enables the COMP peripheral via RCC->APB3ENR and configures the COMP1->CSR register.

`ca_dac_hw_comp_level`

Header: `caos/drivers/inc/ca_glue_hw.h:100`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_dac_hw_comp_level(void); /* COMP1 output */
```

Reads the current digital output state of the hardware comparator associated with the DAC.

Ritorna:
Returns true if the comparator output is high (logic 1), and false if it is low (logic 0) or if the hardware does not support a comparator.

Note:

- This function is non-blocking and executes immediately by reading a hardware register. It is thread-safe as it performs a single atomic read operation, and it is safe to call from an ISR. Note that on many STM32 series (like F1, F4, F7, H5), this function will always return false because those specific chips lack the integrated comparator hardware required for this feature. Always verify your target hardware capabilities before relying on this value for control logic.

Uso:

- This function is typically used in control loops where the DAC output is being monitored against an external reference voltage via the internal comparator. It is usually called after initializing the comparator with ca_dac_comp_init and before calling ca_dac_comp_stop. It is often used to implement simple threshold detection or 'bang-bang' control logic where the software adjusts the DAC output based on whether the comparator level is high or low.

Esempio:

```
/* Check if the DAC comparator threshold has been crossed */
if (ca_dac_comp_level()) {
    /* Comparator output is high, perform action */
    ca_gpio_write(1, 5, true); // Example: Turn on LED on Port 1, Pin 5
} else {
    /* Comparator output is low, perform alternative action */
    ca_gpio_write(1, 5, false);
}

/* Periodically check status in a task loop */
ca_os_delay_ms(10);
if (!ca_dac_comp_level()) {
    /* Logic to handle low state */
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_dac_hw_comp_level in my control loop?
A: You should call it after initializing the comparator with ca_dac_comp_init and before calling ca_dac_comp_stop. It is typically used for threshold detection or 'bang-bang' control logic to adjust the DAC output based on the comparator's state.

Code example conf:1.00 **Q:** Provide a minimal C example showing how to check the comparator level.
A: bool is_high = ca_dac_hw_comp_level(); if (is_high) { /* Logic for high state */ } else { /* Logic for low state */ }

Return conf:1.00 **Q:** What do the return values of ca_dac_hw_comp_level signify, and why might it return false?
A: The function returns true if the comparator output is logic 1 (high) and false if it is logic 0 (low). It also returns false if the hardware does not support an integrated comparator, which is the case for STM32 F1, F4, F7, and H5 series.

HW differences conf:1.00 **Q:** How does the implementation of ca_dac_hw_comp_level differ across STM32 families?
A: On F1, F4, F7, and H5 series, the function is a stub that always returns false because these chips lack the required comparator hardware. On H7xx, it reads the COMP_SR_C1VAL bit from the COMP12->SR register, while on U5xx, it reads the COMP_CSR_VALUE bit from the COMP1->CSR register.

---

ca_dac_hw_comp_stop

Header: `caos/drivers/inc/ca_glue_hw.h:101`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_dac_hw_comp_stop(void); /* COMP1 off */
```

Disables the hardware comparator associated with the DAC peripheral.

Ritorna:
void

Note:

- This function is non-blocking and executes immediately by clearing the configuration registers of the comparator hardware. It is thread-safe in the sense that it performs a single atomic register write, but it does not implement internal locking, so concurrent calls from different tasks should be managed by the user. It is safe to call from an ISR, provided the hardware supports it, but note that not all STM32 variants (e.g., F1, F4, F7) include a comparator; on those platforms, this function is a no-op. The most common mistake is calling this function before the DAC or comparator has been properly initialized, which is harmless but logically incorrect.

Uso:

- This function is used to shut down the comparator hardware when it is no longer needed, typically during system power-down sequences or when reconfiguring the DAC for a different mode of operation. It is usually paired with ca_dac_comp_init to manage the lifecycle of the comparator. In a typical application, you would call ca_dac_comp_stop before calling ca_dac_stop to ensure the analog front-end is cleanly disabled before the peripheral clock is gated.

Esempio:

```
/* Example: Safely stopping the DAC and its associated comparator */

void shutdown_dac_subsystem(void) {
    /* Stop the comparator first to prevent spurious signals */
    ca_dac_comp_stop();

    /* Disable the DAC peripheral */
    ca_dac_stop();

    /* Log the event using caOS system services */
    ca_printf("DAC and Comparator subsystem disabled successfully.\n");
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_dac_hw_comp_stop in my application lifecycle?
A: You should call ca_dac_hw_comp_stop when the comparator is no longer needed, such as during power-down sequences or when reconfiguring the DAC. It is recommended

to call this before ca_dac_stop to ensure the analog front-end is disabled before the peripheral clock is gated.

Code example conf:1.00 **Q:** How do I properly shut down the DAC comparator in caOS?
A: To shut down the comparator, simply invoke the function directly. Example: { ca_dac_hw_comp_stop(); } Note that this function is non-blocking and performs an atomic register write.

HW differences conf:1.00 **Q:** How does the implementation of ca_dac_hw_comp_stop vary across different STM32 families?
A: On F1, F4, F7, and H5 families, the function is a no-op because these variants do not include the comparator. On the H7xx family, it clears the COMP1->CFGR register, while on the U5xx family, it clears the COMP1->CSR register.

`ca_dac_hw_init_periph`

Header: `caos/drivers/inc/ca_glue_hw.h:90`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_dac_hw_init_periph(u8 channel); /* inizializza e abilita
   il canale DAC */
```

Initializes and enables the specified DAC hardware channel.

Parametri:

Nome	Tipo	Descrizione
`channel`	`u8`	The DAC channel number to initialize. Valid values are 1 or 2. Any other value will cause the function to return false.

Ritorna:
Returns true if the channel was successfully enabled, or false if the channel index is invalid or the hardware does not support DAC initialization.

Note:

- This function is non-blocking and executes directly on the hardware registers. It is generally thread-safe if the underlying hardware registers are accessed atomically, but it is not intended to be called from an ISR unless the system design explicitly allows it. A mandatory precondition is that the peripheral clock and GPIO pins must be configured before calling this function; failing to do so will result in the DAC being enabled but physically disconnected from the output pin. The most common beginner mistake is assuming this function handles clock gating or pin multiplexing, which are handled by separate 'ca_clock_hw_dac_init' and 'ca_gpio_hw_dac_init' functions.

Uso:

- This function is used during the system startup or driver initialization phase to prepare the DAC peripheral for operation. It is typically called as the final step in a sequence: first, the clock is enabled, then the GPIO pins are configured for analog mode, and finally, this function enables the DAC hardware itself. It is often paired with higher-level driver functions that manage the DAC's output values, ensuring the hardware is ready before any data is written to the conversion registers.

Esempio:

```
/* Example of initializing DAC channel 1 */
u8 dac_ch = 1;

```

```c
/* 1. Enable peripheral clock */
ca_clock_hw_dac_init(dac_ch);

/* 2. Configure GPIO pins for DAC output */
ca_gpio_hw_dac_init(dac_ch);

/* 3. Enable the DAC hardware peripheral */
if (ca_dac_hw_init_periph(dac_ch)) {
    /* Success: DAC is now ready to receive data */
} else {
    /* Error: Invalid channel or hardware not supported */
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_dac_hw_init_periph() in my embedded application?
A: You should call ca_dac_hw_init_periph() once during system initialization, before attempting to use the DAC channel. This function initializes and enables the DAC hardware channel, making it ready for output generation. It is typically called after the clock and GPIO for the DAC are already configured.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_dac_hw_init_periph() in C with caOS types?
A: bool success = ca_dac_hw_init_periph(1); if (success) { /* DAC channel 1 is now enabled */ } else { /* Initialization failed */ }

Parameters conf:0.95 **Q:** What does the 'channel' parameter represent in ca_dac_hw_init_periph()?
A: The 'channel' parameter specifies which DAC channel to initialize. Valid values are 1 or 2. Any other value (e.g., 0 or 3) will cause the function to return false, indicating an invalid channel.

Return conf:0.90 **Q:** What does it mean when ca_dac_hw_init_periph() returns false?
A: A return value of false indicates that the initialization failed. This typically happens when an invalid channel number is passed (less than 1 or greater than 2). It may also indicate hardware issues, though the function itself does not detect hardware errors.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_dac_hw_init_periph()?
A: Common mistakes include passing invalid channel numbers (e.g., 0 or 3), calling the function multiple times for the same channel without a reset, and calling it from an ISR. Additionally, calling this function without ensuring that the DAC clock and GPIO are already configured may lead to undefined behavior.

HW differences conf:0.80 **Q:** How does ca_dac_hw_init_periph() differ between STM32F4xx and STM32H7xx families?
A: Both families enable the DAC channel by setting the appropriate bit in the DAC control register (DAC->CR for F4xx, DAC1->CR for H7xx). However, F4xx uses a single DAC peripheral, while H7xx uses DAC1. The register names and base addresses differ between families, but the logic for enabling the channel remains consistent.

`ca_dac_hw_stop`

Header: `caos/drivers/inc/ca_glue_hw.h:92`
GUARD: `EN_CA_GLUE_HW`

```c
extern bool ca_dac_hw_stop(u8 channel); /* disables the DAC channel,
    releasing its output buffer */
```

Disables a specific DAC channel by disconnecting its output buffer from the pin.

Parametri:

Nome	Tipo	Descrizione
`channel`	`u8`	The DAC channel index to stop. Valid values are 1 or 2. Passing any other value will result in a failure.

Ritorna:
Returns true if the channel was successfully disabled, or false if the channel index is invalid or the hardware does not support the operation.

Note:

- This function is non-blocking and executes immediately by modifying the DAC control register. It is thread-safe in the sense that it performs a single atomic register write, but it does not provide synchronization if multiple threads attempt to control the same DAC channel simultaneously. It is safe to call from an ISR, as it does not perform any blocking operations or OS-level synchronization. A common mistake is assuming that calling this function returns the pin to a standard GPIO state; the pin remains in analog mode, so you must manually reconfigure the pin using GPIO functions if you intend to use it for digital I/O after stopping the DAC.

Uso:

- This function is typically used during system shutdown, power-saving transitions, or when reconfiguring a pin for a different peripheral function. It is the counterpart to the initialization sequence and is usually called as part of a cleanup routine to ensure the DAC output is disconnected and the analog buffer is powered down. In the caOS driver architecture, it is often wrapped by a higher-level function (like ca_dac_stop) that also updates internal state tracking variables to prevent further write attempts to a disabled channel.

Esempio:

```
/* Example: Safely stopping a DAC channel before reconfiguring the pin */
#include "caos.h"

void shutdown_dac_channel(u8 channel) {
    /* Attempt to stop the hardware DAC channel */
    if (ca_dac_hw_stop(channel)) {
        /* Successfully disconnected the buffer.
           Now, the pin is still in analog mode.
           Use ca_gpio_configure to set the pin as input or output if needed. */
        ca_printf("DAC channel %d stopped successfully.\n", channel);
    } else {
        /* Handle error: invalid channel or hardware failure */
        ca_printf("Failed to stop DAC channel %d!\n", channel);
    }
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_dac_hw_stop in my caOS application?
A: You should call this function during system shutdown, power-saving transitions, or when reconfiguring a pin for a different peripheral function. It is designed to disconnect the DAC output buffer from the pin and power it down.

Code example conf:1.00 **Q:** How do I stop DAC channel 1 using caOS types?
A: You can stop the channel by calling: u8 channel = 1; bool success = ca_dac_hw_stop(channel);

Parameters conf:1.00 **Q:** What are the valid values for the channel parameter in ca_dac_hw_stop?
A: The channel parameter must be either 1 or 2. Passing any other value will result in a failure and the function will return false.

Return conf:1.00 **Q:** What do the return values of ca_dac_hw_stop indicate?

A: The function returns true if the DAC channel was successfully disabled. It returns false if the provided channel index is invalid or if the specific hardware implementation does not support the operation.

Error/Edge case conf:1.00 **Q:** Is it safe to use the DAC pin as a digital GPIO immediately after calling ca_dac_hw_stop?

A: No, that is a common mistake. Calling ca_dac_hw_stop does not return the pin to a standard GPIO state; the pin remains in analog mode. You must manually reconfigure the pin using GPIO functions if you intend to use it for digital I/O.

HW differences conf:1.00 **Q:** Are there any hardware-specific limitations for ca_dac_hw_stop across different STM32 families?

A: Yes. On the f1xx family, the function is implemented as a stub that returns false, meaning it does not support the operation. On f4xx and f7xx, it modifies the DAC->CR register. On h5xx, h7xx, and u5xx, it modifies the DAC1->CR register.

`ca_dac_hw_write`

Header: `caos/drivers/inc/ca_glue_hw.h:91`
GUARD: `EN_CA_GLUE_HW`

```
1 extern void ca_dac_hw_write(u8 channel, u16 val); /* scrive il valore
     grezzo (0  4095  ) sul canale DAC */
```

Writes a raw 12-bit digital value to the specified DAC hardware channel.

Parametri:

Nome	Tipo	Descrizione
`channel`	`u8`	The DAC channel identifier, typically 1 or 2. Passing an invalid channel index will result in no action taken by the hardware registers.
`val`	`u16`	The raw 12-bit value to convert to analog voltage. Valid range is 0 to 4095 (0x0FFF). Values exceeding 4095 will be truncated by the hardware register.

Ritorna:
void. This function does not return a status code, so it assumes the hardware is already initialized and the parameters are valid.

Note:

- This function is non-blocking and executes immediately by writing directly to the DAC peripheral registers. It is generally thread-safe in the sense that register writes are atomic, but it does not implement any locking mechanisms, so concurrent writes to the same channel from different tasks or ISRs may lead to race conditions. You must ensure that the DAC peripheral is properly configured before calling this function; otherwise, the hardware may not be clocked or configured, and the write will have no effect. A common beginner mistake is assuming this function handles voltage scaling or calibration; it only writes raw bits, so you must perform any necessary math to convert your desired voltage into the 0-4095 range before calling it.

Uso:

- This function is used to update the analog output voltage of the DAC peripheral in real-time. It is typically paired with initialization functions like ca_dac_init during system startup. In advanced applications, it is often called inside timer callbacks or periodic tasks

to generate waveforms, such as sine waves or ramps, by updating the DAC value at a fixed frequency. It is designed for low-latency updates where the overhead of a full driver stack is not required.

Esempio:

```c
/* Initialize the DAC peripheral before use */
if (ca_dac_init(1)) {
    u16 target_value = 2048; // Mid-scale voltage (approx 1.65V on 3.3V systems)

    /* Write the raw value to channel 1 */
    ca_dac_hw_write(1, target_value);
} else {
    /* Handle initialization failure */
    /* Log error: DAC channel 1 failed to initialize */
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_dac_hw_write() in my caOS project?
A: You should use ca_dac_hw_write() when you need to output an analog voltage level by writing a raw 12-bit value (0-4095) to a DAC channel. It's typically used for generating waveforms, controlling analog signals, or any application requiring precise analog output. The function is non-blocking and must be called after initializing the DAC hardware with ca_dac_hw_init_periph().

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_dac_hw_write() with caOS types?
A:

```c
u8 channel = 1;
u16 value = 2048; /* Midpoint of 12-bit range */
ca_dac_hw_write(channel, value);
```

Parameters conf:0.90 **Q:** What are the valid values for the channel and val parameters in ca_dac_hw_write()?
A: The channel parameter must be either 1 or 2. Any other value leads to undefined behavior. The val parameter must be in the range 0-4095. Values outside this range may be truncated or cause unpredictable results. The function does not validate inputs, so it's the caller's responsibility to ensure valid values.

HW differences conf:0.85 **Q:** How does ca_dac_hw_write() differ between STM32F4xx and STM32H7xx families?
A: The implementation differs in the peripheral register base address. For F4xx, the DAC registers are accessed via DAC->DHR12R1 and DAC->DHR12R2. For H7xx, the registers are accessed via DAC1->DHR12R1 and DAC1->DHR12R2. Both families use the same register names but different peripheral base addresses, reflecting the architectural differences in their DAC peripherals.

`ca_dac_init`

Header: `caos/drivers/inc/dac.h:29`
Source: `caos/drivers/src/dac.c:122:134`
GUARD: `EN_DAC`

```c
bool ca_dac_init(const ca_dac_config_t *cfg)
```

Initializes the DAC peripheral hardware, clock, and GPIO pins for a specific channel.

Parametri:

Nome	Tipo	Descrizione
cfg	const ca_dac_config_t *	Pointer to a configuration structure containing the channel number (1 or 2). Must not be NULL.

Ritorna:
Returns true if the hardware initialization was successful, false if the configuration pointer is NULL, the channel is invalid, or the peripheral hardware failed to initialize.

Note:

- This function is non-blocking and performs low-level hardware setup, including enabling the peripheral clock and configuring the associated GPIO pins as analog outputs. It is not thread-safe and should be called during the system initialization phase (e.g., inside ca_os_configure) before any write operations are attempted. It must NOT be called from an Interrupt Service Routine (ISR) because it performs hardware configuration that may involve timing-sensitive operations. A common beginner mistake is passing an uninitialized configuration structure or attempting to initialize the same channel multiple times without checking the return value, which can lead to undefined hardware states.

Uso:

- This function is the mandatory first step for using the DAC peripheral. It is typically called once during system startup to prepare the hardware. After a successful call, the application can proceed to use functions like ca_dac_write to output analog voltages. If the application requires CLI debugging, this function also handles the registration of the DAC command interface, allowing technicians to monitor or test the DAC state via the console.

Esempio:

```
/* Define the configuration structure for DAC Channel 1 */
ca_dac_config_t dac_cfg = { .channel = 1 };

/* Initialize the DAC hardware */
if (ca_dac_init(&dac_cfg)) {
    /* Initialization successful, safe to write values */
    ca_dac_write(1, 2048U); // Set DAC to mid-scale
} else {
    /* Handle initialization failure (e.g., log error or halt) */
    ca_stdout_printf_n("Error: Failed to initialize DAC1");
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_dac_init in a caOS application?
A: You should call ca_dac_init during system initialization, typically before starting any tasks or enabling interrupts. It is used to configure a DAC channel (either 1 or 2) on an STM32 microcontroller, setting up the clock, GPIO, and hardware peripheral. It is not suitable for use in an RTOS context where blocking is not allowed.

Code example conf:0.90 **Q:** Can you provide a minimal example of how to use ca_dac_init with caOS types?
A: ca_dac_config_t config = { .channel = 1, .gpio_port = GPIOA, .gpio_pin = 4 }; bool success = ca_dac_init(&config); if (!success) { /* Handle error */ }

Parameters conf:0.95 **Q:** What does the 'channel' field in ca_dac_config_t represent, and what values are valid?
A: The 'channel' field specifies which DAC channel to initialize. Valid values are 1 or 2. If the channel is set to any other value (e.g., 0 or 3), the function will return false and fail to initialize the DAC.

Return conf:0.95 **Q:** What does ca_dac_init return, and what causes it to return false?
A: ca_dac_init returns true if initialization is successful, and false if it fails. It returns false when the input pointer cfg is NULL or when the channel specified in cfg is not 1 or 2.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_dac_init?
A: Common mistakes include passing a NULL pointer for cfg, specifying an invalid channel (not 1 or 2), or calling the function multiple times on the same DAC channel without resetting it. Additionally, calling this function from an ISR or in a multitasking RTOS environment is not recommended due to its blocking nature.

`ca_dac_stop`

Header: `caos/drivers/inc/dac.h:37`
Source: `caos/drivers/src/dac.c:145:153`
GUARD: `EN_DAC`

```
bool ca_dac_stop(u8 channel)
```

Disables a specific DAC channel and resets its internal state.

Parametri:

Nome	Tipo	Descrizione
`channel`	`u8`	The DAC channel number to stop. Valid values are 1 or 2. Any other value will cause the function to return false.

Ritorna:
Returns true if the channel was successfully stopped, false if the channel index is invalid or the hardware failed to stop.

Note:

- This function is not thread-safe and should not be called from an ISR. It performs hardware-level operations to disconnect the DAC output buffer from the physical pin, which is critical to prevent electrical contention if you intend to reconfigure the pin as a standard GPIO. Always ensure the DAC has been initialized via ca_dac_init before attempting to stop it. A common mistake is assuming that writing a value of 0 to the DAC is equivalent to stopping it; writing 0 still leaves the output buffer active and connected to the pin, which can interfere with other peripherals sharing that pin.

Uso:

- Use this function when you need to release the DAC hardware, typically during a transition between different peripheral modes (e.g., switching from analog output to digital input/output). It is standard practice to call this before reconfiguring the associated pin using ca_gpio_configure or ca_gpio_init_port to avoid electrical conflicts. It is often paired with ca_dac_init for setup and ca_dac_write for active operation, serving as the cleanup step in the peripheral lifecycle.

Esempio:

```
/* Example: Safely stop the DAC to reconfigure the pin as a digital output */
#define DAC_CHANNEL 1
#define GPIO_PORT 0 // Assuming 0 represents a valid port index

void switch_dac_to_gpio(void) {
    // Stop the DAC to disconnect the internal buffer from the pin
    if (ca_dac_stop(DAC_CHANNEL)) {
```

```
        // Now that the DAC is parked, it is safe to reconfigure the pin
        ca_gpio_init_port(GPIO_PORT);

        /* Perform additional GPIO configuration here using ca_gpio_configure */
        // ca_gpio_configure(GPIO_PORT, ...);
    } else {
        // Handle error: DAC failed to stop or invalid channel
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_dac_stop and why is it necessary?
A: You should call ca_dac_stop when you need to release the DAC hardware, such as when transitioning from analog output to digital input/output. It is necessary to prevent electrical contention by disconnecting the DAC output buffer from the physical pin before reconfiguring the pin as a standard GPIO.

Code example conf:1.00 **Q:** How do I correctly stop DAC channel 1 in my caOS application?
A: To stop the DAC, ensure it was previously initialized and then call the function with the channel index: u8 channel = 1; if (ca_dac_stop(channel)) { /* Success */ }

Parameters conf:1.00 **Q:** What are the valid values for the channel parameter in ca_dac_stop?
A: The channel parameter must be a u8 with a value of 1 or 2. Providing any other value will result in the function returning false.

Return conf:1.00 **Q:** What do the return values of ca_dac_stop indicate?
A: The function returns true if the DAC channel was successfully stopped. It returns false if the provided channel index is invalid (not 1 or 2) or if the hardware failed to stop.

`ca_dac_write`

Header: `caos/drivers/inc/dac.h:32`
Source: `caos/drivers/src/dac.c:136:143`
GUARD: `EN_DAC`

```
void ca_dac_write(u8 channel, u16 val)
```

Writes a 12-bit digital value to the specified DAC channel to generate an analog voltage output.

Parametri:

Nome	Tipo	Descrizione
`channel`	`u8`	The DAC channel number. Must be 1 or 2. Values outside this range are ignored by the function.
`val`	`u16`	The 12-bit digital value to convert. The function masks this to 0x0FFF (0-4095), so values larger than 4095 are truncated.

Ritorna:
void. The function does not provide feedback on success or failure, so ensure the channel is valid before calling.

Note:

- This function is a wrapper that handles hardware-specific register writes. It is non-blocking and generally safe to call from most contexts, but it does not perform error checking on the hardware state. A common beginner mistake is assuming the function will return an error if the DAC hardware is not initialized; if you call this before ca_dac_init, the write

will likely fail silently or cause undefined behavior. It is not thread-safe if multiple tasks attempt to write to the same channel simultaneously without external synchronization.

Uso:

- This function is used to output analog signals, such as generating waveforms or setting reference voltages for sensors. The typical lifecycle involves calling ca_dac_init once during system startup to configure the GPIO pins and DAC peripheral, followed by repeated calls to ca_dac_write whenever the output value needs to change. It is often paired with ADC functions when implementing closed-loop control systems where the DAC output is measured by an ADC input.

Esempio:

```
/* Initialize the DAC channel 1 before writing */
ca_dac_config_t dac_cfg = { 1 };
ca_dac_init(&dac_cfg);

/* Set DAC to mid-scale (approx 1.65V on a 3.3V system) */
u16 output_val = 2048;
ca_dac_write(1, output_val);

/* Attempting to write to an invalid channel will be ignored */
ca_dac_write(3, 1000); // This does nothing
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use the ca_dac_write function, and what is its primary purpose?
A: The ca_dac_write function is used to write a 12-bit value to a specific DAC channel (1 or 2). It is a non-blocking function suitable for use in both task and ISR contexts. It is typically used when you need to output an analog voltage level from the DAC, such as generating waveforms or controlling analog signals in embedded applications.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_dac_write with caOS types?
A: u16 dac_value = 0x800U; /* 12-bit value */ ca_dac_write(1, dac_value); /* Write to DAC channel 1 */

Parameters conf:0.95 **Q:** What are the valid values for the channel parameter in ca_dac_write, and what happens if an invalid value is passed?
A: The channel parameter must be either 1 or 2. If any other value is passed, the function will return immediately without performing any action. This is because the function only processes valid DAC channels and ignores invalid inputs.

Return conf:0.90 **Q:** Does ca_dac_write have any return values, and what do they signify?
A: The function does not return any value. It is void. It cannot fail in a strict sense, but it will not perform any action if the channel is invalid. The function is non-blocking and does not provide feedback on whether the write operation was successful or not.

`ca_dhcp_service_bind`

Header: `caos/net/inc/dhcp_service.h:113`
Source: `caos/net/src/dhcp_service.c:477:481`
GUARD: `EN_DHCP_SERVICE`

```
bool ca_dhcp_service_bind(void)
```

Blocks the calling thread until the DHCP service has successfully acquired an IP address.

Ritorna:
Returns true if the DHCP service is already bound or if the binding completes within the timeout period. Returns false if the timeout expires before the DHCP server provides an IP address.

Note:

- This function is a blocking call, meaning the calling thread will be suspended by the RTOS scheduler until the semaphore is released or the timeout occurs. It is NOT safe to call this from an Interrupt Service Routine (ISR) or from the main network processing task, as doing so will prevent the network stack from receiving the DHCP packets required to complete the binding, leading to a permanent deadlock. Ensure the DHCP service has been started via ca_os_job before calling this. A common beginner mistake is calling this in the same thread that is responsible for processing incoming network packets, which effectively kills the system's ability to communicate.

Uso:

- Use this function in application-level threads that require a valid IP address before proceeding with network operations like opening sockets or starting a web server. It is typically used after the system has initialized the network interface and started the DHCP job. It pairs with the DHCP service lifecycle: first, the service is started, then the application waits for the binding, and finally, the application proceeds once the function returns true. Never use this in the network processing loop itself; instead, use it in a separate worker thread that waits for the network to be ready.

Esempio:

```c
/* Example of waiting for DHCP in a dedicated application thread */
void app_network_init_task(void *arg) {
    /* Wait for DHCP to finish binding before starting network services */
    if (ca_dhcp_service_bind()) {
        ca_stdout_put_n("DHCP bound successfully, starting application...");
        /* Proceed to open sockets or perform network requests */
    } else {
        ca_stdout_put_n("DHCP timeout: check network cable or server.");
        /* Handle error state, perhaps retry or enter safe mode */
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_dhcp_service_bind() in my caOS application, and why is it important to call it at the right time?
A: You should call ca_dhcp_service_bind() after starting the DHCP task to wait for the IP address acquisition to complete. It is important because it blocks the calling thread until either the IP is successfully acquired or a timeout occurs, ensuring that network operations do not proceed before the network is properly configured. It must not be called from the net task context to avoid deadlocks.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using caOS types that demonstrates how to use ca_dhcp_service_bind()?
A: u32 result = ca_dhcp_service_bind(); if (result) { // IP acquired successfully } else { // DHCP timeout or internal error occurred }

Return conf:0.90 **Q:** What do the return values of ca_dhcp_service_bind() indicate, and what causes the function to return false?
A: The function returns true if the IP address was successfully acquired. It returns false if the timeout expired or an internal error occurred. Common causes of failure include timeout due to network unavailability or improper task scheduling, especially if the function is called from the wrong task context like the net task, which can lead to deadlock.

ca_dhcp_service_bound

Header: caos/net/inc/dhcp_service.h:134
GUARD: EN_DHCP_SERVICE

```
static inline bool ca_dhcp_service_bound(void)
```

Checks if the DHCP client has successfully obtained an IP address and is currently bound to a network lease.

Ritorna:
Returns true if the DHCP client is currently bound to an IP address, false otherwise.

Note:

- This function is a non-blocking, thread-safe read of an internal status flag. It is safe to call from both tasks and ISRs because it simply returns the current state of a boolean variable. The most common mistake is assuming that a 'true' result guarantees that the network connection is still physically active; it only indicates that the DHCP lease is valid. Always ensure ca_dhcp_service_init has been called before checking this status, otherwise, the flag will always be false.

Uso:

- This function is typically used in a polling loop or a state machine to verify network readiness before attempting to perform network-dependent operations like opening a TCP socket or sending data. It is often paired with ca_dhcp_service_start to initiate the process and ca_dhcp_service_get_ip to retrieve the assigned address once this function returns true. In a typical application, you would wait for this to return true before proceeding with higher-level protocols like NTP or HTTP.

Esempio:

```
/* Ensure the DHCP service is initialized and started before checking status */
if (ca_dhcp_service_bound()) {
    /* DHCP is bound, safe to retrieve the assigned IP address */
    uint32_t ip = ca_dhcp_service_get_ip();
    ca_printf("Network ready! IP: %lu\n", ip);
} else {
    /* DHCP not yet bound, wait or trigger a start if not already running */
    ca_os_delay_ms(100);
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_dhcp_service_bound in my application logic?
A: You should call it in a polling loop or state machine to verify network readiness before performing network-dependent operations like opening a TCP socket or sending data. It is typically used after calling ca_dhcp_service_start to ensure the DHCP lease is valid before proceeding with higher-level protocols like NTP or HTTP.

Code example conf:1.00 **Q:** How do I implement a simple check for DHCP status using caOS types?
A: Ensure ca_dhcp_service_init has been called first. Example: if (ca_dhcp_service_bound()) { /* Proceed with network operations */ } else { /* Wait or retry */ }

Return conf:0.95 **Q:** What do the return values of ca_dhcp_service_bound indicate, and why might it return false?

A: A return of true indicates the DHCP client has successfully obtained an IP address and holds a valid lease. A return of false indicates the client is not currently bound. This can happen if the DHCP process has not completed, if ca_dhcp_service_init was never called, or if the lease is no longer valid.

---

ca_dhcp_service_get_dns

Header: caos/net/inc/dhcp_service.h:139
Source: caos/net/src/dhcp_service.c:549:555
GUARD: EN_DHCP_SERVICE

```
ca_ipv4_t ca_dhcp_service_get_dns(void)
```

Retrieves the current DNS server address assigned by the DHCP service.

Ritorna:
Returns an ca_ipv4_t structure containing the 4-byte IP address of the DNS server. If no DNS server has been provided by the DHCP server, the structure will typically contain zeros (0.0.0.0).

Note:

- This function is thread-safe because it uses an internal mutex to protect the shared DHCP lease data during access. It is a blocking call in the sense that it must wait for the mutex to be released by any other task currently modifying the DHCP lease information. You must NEVER call this function from an Interrupt Service Routine (ISR) because the mutex lock operation can block the execution, which is strictly forbidden in ISR context. Ensure the DHCP service has been initialized and started via ca_dhcp_service_init and ca_dhcp_service_start before relying on the returned value.

Uso:

- This function is typically used in diagnostic tasks or CLI interfaces to display the current network configuration to the user. It is often paired with other getters like ca_dhcp_service_get_ip, ca_dhcp_service_get_mask, and ca_dhcp_service_get_gw to provide a complete snapshot of the network interface status. It is commonly called after verifying that the network is ready using ca_dhcp_service_net_ready to ensure the returned data is valid and not stale.

Esempio:

```
/* Check if the DHCP service has successfully obtained a lease */
if (ca_dhcp_service_net_ready()) {
    ca_ipv4_t dns_addr = ca_dhcp_service_get_dns();

    /* Print the DNS address using the system logging facility */
    ca_printf("Current DNS: %u.%u.%u.%u\n",
              dns_addr.v[0], dns_addr.v[1],
              dns_addr.v[2], dns_addr.v[3]);
} else {
    ca_printf("DNS not available: DHCP lease not bound.\n");
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to call ca_dhcp_service_get_dns in my application?
A: You should call this function in diagnostic tasks or CLI interfaces to display network configuration. It is best practice to call it only after verifying the network is ready using ca_dhcp_service_net_ready to ensure the data is valid and not stale, and only

after the DHCP service has been initialized and started via ca_dhcp_service_init and ca_dhcp_service_start.

Code example conf:0.90 **Q:** How do I retrieve and print the DNS address using caOS types?
A: ca_ipv4_t dns_addr = ca_dhcp_service_get_dns(); // Assuming a hypothetical print function exists for demonstration ca_printf("DNS: %u.%u.%u.%u\n", dns_addr.b[0], dns_addr.b[1], dns_addr.b[2], dns_addr.b[3]);

Return conf:1.00 **Q:** What does the return value of ca_dhcp_service_get_dns represent, and what happens if the DHCP server provides no DNS?
A: The function returns an ca_ipv4_t structure containing the 4-byte IP address of the DNS server. If no DNS server has been provided by the DHCP server, the structure will typically contain zeros (0.0.0.0).

`ca_dhcp_service_get_gw`

Header: `caos/net/inc/dhcp_service.h:138`
Source: `caos/net/src/dhcp_service.c:541:547`
GUARD: `EN_DHCP_SERVICE`

```
ca_ipv4_t ca_dhcp_service_get_gw(void)
```

Retrieves the current IPv4 gateway address assigned by the DHCP service.

Ritorna:
Returns an ca_ipv4_t structure containing the four bytes of the gateway IP address. If the DHCP service has not yet obtained a lease or is not initialized, the returned structure will contain the default/zeroed values.

Note:

- This function is thread-safe because it uses a mutex (s_lease_mtx) to protect the internal DHCP state variables during the read operation. It is a blocking call in the sense that it must wait for the mutex to be available; therefore, it must NEVER be called from an ISR (Interrupt Service Routine), as blocking in an ISR will cause a system crash or watchdog reset. Ensure that ca_dhcp_service_init has been called successfully before attempting to read network parameters, otherwise, you will receive uninitialized or default data.

Uso:

- This function is typically used in diagnostic tasks or CLI interfaces to display the current network configuration to the user. It is often paired with other getter functions like ca_dhcp_service_get_ip, ca_dhcp_service_get_mask, and ca_dhcp_service_get_dns to provide a complete snapshot of the device's network status. It is usually called after verifying that the network is ready using ca_dhcp_service_net_ready to ensure the returned values are valid and active.

Esempio:

```
/* Check if the DHCP service has successfully obtained an IP address */
if (ca_dhcp_service_net_ready()) {
    ca_ipv4_t gw = ca_dhcp_service_get_gw();
    /* Print the gateway address using the system logging facility */
    ca_printf("Gateway: %u.%u.%u.%u\n", gw.v[0], gw.v[1], gw.v[2], gw.v[3]);
} else {
    ca_printf("Gateway not yet configured by DHCP.\n");
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to call ca_dhcp_service_get_gw in my application?
A: This function is typically used in diagnostic tasks or CLI interfaces to display network configuration. It should be called after verifying the network is ready using ca_dhcp_service_net_ready to ensure the returned values are valid and active. It must never be called from an ISR because it is a blocking call that uses a mutex.

Code example conf:0.90 **Q:** How do I retrieve and print the gateway address using caOS types?
A: ca_ipv4_t gw = ca_dhcp_service_get_gw(); // Assuming a hypothetical print function exists for demonstration ca_log("Gateway: %d.%d.%d.%d", gw.b[0], gw.b[1], gw.b[2], gw.b[3]);

Return conf:1.00 **Q:** What does the return value of ca_dhcp_service_get_gw represent, and what happens if the DHCP service isn't ready?
A: The function returns an ca_ipv4_t structure containing the four bytes of the gateway IP address. If the DHCP service has not obtained a lease or is not initialized, the structure will contain default or zeroed values. You must ensure ca_dhcp_service_init has been called successfully to avoid receiving uninitialized data.

`ca_dhcp_service_get_ip`

Header: `caos/net/inc/dhcp_service.h:136`
Source: `caos/net/src/dhcp_service.c:525:531`
GUARD: `EN_DHCP_SERVICE`

```
ca_ipv4_t ca_dhcp_service_get_ip(void)
```

Retrieves the current IPv4 address assigned to the device by the DHCP service.

Ritorna:
Returns an ca_ipv4_t structure containing the four bytes of the IP address. If the DHCP service has not yet acquired an address, the returned structure will typically contain zeros (0.0.0.0).

Note:

- This function is thread-safe because it uses a mutex to protect the internal IP variable during access. It is non-blocking in the sense that it does not wait for network activity, but it does perform a mutex lock, so it must NEVER be called from an ISR (Interrupt Service Routine). The DHCP service must be initialized via ca_dhcp_service_init and started via ca_dhcp_service_start before this function will return a valid, non-zero IP address. A common mistake is calling this immediately after system boot without checking if the network is actually ready using ca_dhcp_service_net_ready.

Uso:

- This function is typically used in application tasks or CLI handlers to display or log the device's network identity. It is often paired with ca_dhcp_service_get_mask and ca_dhcp_service_get_gw to retrieve the full network configuration. Developers should use this in a sequence where they first verify the connection status with ca_dhcp_service_net_ready or ca_dhcp_service_bound before attempting to use the IP address for network communication.

Esempio:

```
void display_network_info(void) {
    // Check if the DHCP service has successfully bound an IP address
    if (ca_dhcp_service_bound()) {
        ca_ipv4_t ip = ca_dhcp_service_get_ip();
        // Print the IP address using the system logging facility
```

```
        ca_printf("Device IP: %u.%u.%u.%u\n", ip.v[0], ip.v[1], ip.v[2], ip.v
    [3]);
    } else {
        ca_printf("DHCP not bound yet.\n");
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to call ca_dhcp_service_get_ip in my application?
A: You should call this function in application tasks or CLI handlers after verifying that the network is ready. It is recommended to first check the connection status using ca_dhcp_service_net_ready or ca_dhcp_service_bound to ensure the DHCP service has successfully acquired an address.

Code example conf:0.90 **Q:** How do I retrieve and print the current IP address using caOS types?
A: ca_ipv4_t my_ip = ca_dhcp_service_get_ip(); if (ca_dhcp_service_net_ready()) { /* Assuming a print function exists for cstr */ ca_log("IP: %d.%d.%d.%d", my_ip.b1, my_ip.b2, my_ip.b3, my_ip.b4); }

Return conf:1.00 **Q:** What does the return value of ca_dhcp_service_get_ip represent, and why might it return 0.0.0.0?
A: The function returns an ca_ipv4_t structure containing the four bytes of the assigned IP address. It returns 0.0.0.0 if the DHCP service has not yet acquired an address, which typically happens if the function is called before the DHCP service has completed its negotiation or if it was not properly initialized and started.

`ca_dhcp_service_get_mask`

Header: `caos/net/inc/dhcp_service.h:137`
Source: `caos/net/src/dhcp_service.c:533:539`
GUARD: `EN_DHCP_SERVICE`

```
ca_ipv4_t ca_dhcp_service_get_mask(void)
```

Retrieves the current IPv4 subnet mask assigned to the device by the DHCP service.

Ritorna:
Returns an ca_ipv4_t structure containing the 4-byte subnet mask. If the DHCP service has not yet acquired a lease, the returned structure will contain the default or uninitialized values (typically 0.0.0.0).

Note:

- This function is thread-safe because it uses a mutex to protect the internal DHCP lease data structure during the read operation. It is a blocking call in the sense that it must wait for the mutex to be available, so it must never be called from an ISR (Interrupt Service Routine) as this could cause a system deadlock. Ensure that the DHCP service has been initialized via ca_dhcp_service_init before calling this function, otherwise, you will be reading uninitialized memory. A common mistake is assuming the mask is valid immediately after boot; always verify the network state using ca_dhcp_service_net_ready before relying on the returned mask.

Uso:

- This function is typically used in network diagnostic tasks or CLI interfaces to display the current network configuration to the user. It is often called in a sequence alongside ca_dhcp_service_get_ip, ca_dhcp_service_get_gw, and ca_dhcp_service_get_dns to

provide a complete snapshot of the device's network identity. Developers use this to verify that the DHCP handshake was successful and that the assigned subnet mask matches the expected network topology.

Esempio:

```
ca_ipv4_t mask;

// Check if the network is ready before accessing DHCP parameters
if (ca_dhcp_service_net_ready()) {
    // Retrieve the subnet mask safely
    mask = ca_dhcp_service_get_mask();

    // Print the mask using the ca_printf function
    ca_printf("Current Subnet Mask: %u.%u.%u.%u\n",
              mask.v[0], mask.v[1], mask.v[2], mask.v[3]);
} else {
    ca_printf("DHCP not ready, mask unknown.\n");
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to call ca_dhcp_service_get_mask in my application?
A: You should call this function when you need to display or verify the device's network configuration, such as in diagnostic tasks or CLI interfaces. It is critical to ensure that ca_dhcp_service_init has been called first and that you verify the network state using ca_dhcp_service_net_ready before relying on the returned mask, as the mask is not valid immediately after boot.

Code example conf:1.00 **Q:** How do I retrieve and store the subnet mask using caOS types?
A: You can retrieve the mask by calling the function directly into an ca_ipv4_t variable. Example: ca_ipv4_t my_mask = ca_dhcp_service_get_mask();

Return conf:1.00 **Q:** What does the return value of ca_dhcp_service_get_mask represent, and what happens if the DHCP lease is not ready?
A: The function returns an ca_ipv4_t structure containing the 4-byte subnet mask. If the DHCP service has not yet acquired a lease, the structure will contain default or uninitialized values, typically 0.0.0.0. The function is thread-safe via a mutex, but calling it from an ISR will cause a system deadlock.

`ca_dhcp_service_get_ntp_server`

Header: `caos/net/inc/dhcp_service.h:141`
Source: `caos/net/src/dhcp_service.c:557:563`
GUARD: `EN_DHCP_SERVICE`

```
ca_ipv4_t ca_dhcp_service_get_ntp_server(void)
```

Retrieves the NTP server IP address obtained from the DHCP server via Option 42.

Ritorna:
An ca_ipv4_t structure containing the NTP server IP address. If no NTP server was provided by the DHCP lease, the address field will be 0.0.0.0.

Note:

- This function is thread-safe as it uses a mutex to protect access to the internal DHCP lease data structure. It is a blocking call because it must acquire a mutex; therefore, it must NEVER be called from an Interrupt Service Routine (ISR) or a critical section where

blocking is prohibited. A common mistake is assuming the returned IP is always valid; always check if the address is 0.0.0.0 before using it, as the DHCP server might not have provided an NTP option.

Uso:

- This function is typically used during the network initialization phase after a DHCP lease has been successfully acquired. It is usually paired with ca_dhcp_service_bound to ensure the lease is active, and then used to configure the NTP client via ca_ntp_service_set_server. If the returned IP is zero, developers often implement a fallback mechanism, such as using the gateway IP address retrieved via ca_dhcp_service_get_gw.

Esempio:

```
/* Example: Configuring NTP after DHCP lease acquisition */
if (ca_dhcp_service_bound()) {
    ca_ipv4_t ntp_ip = ca_dhcp_service_get_ntp_server();

    /* Fallback to gateway if DHCP did not provide an NTP server */
    if (ntp_ip.addr == 0U) {
        ntp_ip = ca_dhcp_service_get_gw();
    }

    if (ntp_ip.addr != 0U) {
        ca_ntp_service_set_server(ntp_ip);
        ca_ntp_service_net_ready();
    } else {
        /* Log error: No NTP server available */
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to call ca_dhcp_service_get_ntp_server in my application?
A: This function should be called during the network initialization phase, specifically after a DHCP lease has been successfully acquired. It is recommended to first verify the lease status using ca_dhcp_service_bound before calling this function to retrieve the NTP server address for configuring the NTP client.

Code example conf:1.00 **Q:** How do I retrieve the NTP server address and handle a potential fallback in caOS?
A: ca_ipv4_t ntp_ip = ca_dhcp_service_get_ntp_server(); if (ntp_ip.address == 0) { ntp_ip = ca_dhcp_service_get_gw(); } ca_ntp_service_set_server(ntp_ip);

Return conf:1.00 **Q:** What does the return value of ca_dhcp_service_get_ntp_server indicate, and why might it return 0.0.0.0?
A: The function returns an ca_ipv4_t structure containing the NTP server IP address. A return value of 0.0.0.0 indicates that no NTP server was provided by the DHCP server via Option 42 during the lease acquisition process. Developers must check for this zero value before attempting to use the IP address.

`ca_dhcp_service_get_state`

Header: `caos/net/inc/dhcp_service.h:144`
Source: `caos/net/src/dhcp_service.c:565:576`
GUARD: `EN_DHCP_SERVICE`

```
const char *ca_dhcp_service_get_state(void)
```

Returns a human-readable string representing the current internal state of the DHCP client service.

Ritorna:
A pointer to a constant string literal (e.g., "INIT", "BOUND", "UNKNOWN"). This pointer is always valid and never NULL, as it points to static memory defined within the function.

Note:

- This function is non-blocking and thread-safe because it performs a simple read of a state variable without requiring mutex locks. It is safe to call from an Interrupt Service Routine (ISR) or any task context. The most common mistake is assuming the returned string can be modified; since it points to read-only memory, attempting to write to it will cause a hard fault. Because the state can change asynchronously, the returned string is only a snapshot of the DHCP status at the exact moment the function was called.

Uso:

- This function is primarily used for diagnostic purposes, such as logging the network status to a console or a remote monitoring interface. It is typically paired with other DHCP getter functions to provide a full report of the network configuration. Developers use it to verify that the DHCP state machine has successfully reached the 'BOUND' state before attempting to perform network-dependent operations like opening sockets or sending data. It is usually called after the service has been initialized via ca_dhcp_service_init and started via ca_dhcp_service_start.

Esempio:

```c
/* Check if the DHCP client has successfully obtained an IP address */
const char *current_state = ca_dhcp_service_get_state();

/* Use ca_strcmp to compare the returned state string */
if (ca_strcmp(current_state, "BOUND") == 0) {
    /* Network is ready, safe to proceed with communication */
    ca_printf("Network is up and running.\n");
} else {
    /* DHCP is still negotiating or failed, wait or trigger a renew */
    ca_printf("DHCP status: %s. Waiting for IP...\n", current_state);
    /* Optionally trigger a renew if stuck in a non-functional state */
    if (ca_strcmp(current_state, "UNKNOWN") == 0) {
        ca_dhcp_service_renew();
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_dhcp_service_get_state() in my network initialization code?
A: You should call ca_dhcp_service_get_state() to monitor the current state of the DHCP service during network initialization or troubleshooting. It's useful for verifying that the DHCP client has successfully obtained an IP address (BOUND state) or is still in the process of acquiring one (REQUESTING, DISCOVERING). Since it's thread-safe and non-blocking, it can be called from any context including ISRs.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_dhcp_service_get_state() in a caOS application?
A: cstr state = ca_dhcp_service_get_state(); if (0 == strcmp(state, "BOUND")) { // DHCP lease acquired } else { // Still negotiating }

Return conf:0.90 **Q:** What does it mean when ca_dhcp_service_get_state() returns "BOUND"?
A: The function returns "BOUND" when the DHCP client has successfully obtained an IP address from a DHCP server and is currently using that address. This is the

normal operational state after successful DHCP negotiation. Other possible return values include "INIT", "DISCOVERING", "REQUESTING", "RENEWING", "REBINDING", and "UNKNOWN" which indicate different stages or failure states of the DHCP process.

Error/Edge case conf:0.80 **Q:** What are common mistakes when using ca_dhcp_service_get_state()?
A: Common mistakes include assuming the returned string is static and trying to modify it, or not realizing that the state may change between calls even within a single thread. Since the function is thread-safe, there's no risk of race conditions, but developers might incorrectly assume that the state returned is always up-to-date if they're not using proper synchronization in their own code. Also, since it returns a valid pointer, checking for NULL is unnecessary.

`ca_dhcp_service_init`

Header: `caos/net/inc/dhcp_service.h:98`
Source: `caos/net/src/dhcp_service.c:453:469`
GUARD: `EN_DHCP_SERVICE`

```
1 void ca_dhcp_service_init(ca_mac_t local_mac, ca_dhcp_bound_cb_t
     on_bound, ca_dhcp_expire_cb_t on_expire)
```

Initializes the internal DHCP service state, semaphores, and callback functions required for dynamic IP assignment.

Parametri:

Nome	Tipo	Descrizione
`local_mac`	`ca_mac_t`	The unique hardware MAC address of the device. Must be a valid 6-byte array; passing an invalid or zeroed MAC will cause DHCP discovery to fail.
`on_bound`	`ca_dhcp_bound_cb_t`	Pointer to a callback function executed when an IP address is successfully leased. Cannot be NULL if you need to react to network configuration changes.
`on_expire`	`ca_dhcp_expire_cb_t`	Pointer to a callback function executed when the DHCP lease expires. Essential for handling network disconnections or re-negotiation.

Ritorna:
void. This function does not return a status code, so ensure all parameters are valid before calling.

Note:

- This function is non-blocking and performs internal initialization of RTOS primitives like semaphores and mutexes. It is not thread-safe and must be called exactly once during the system startup phase, typically before the RTOS scheduler starts or within the main initialization task. It is strictly forbidden to call this from an Interrupt Service Routine (ISR) because it performs memory and synchronization object initialization. A common beginner mistake is calling this function multiple times or after the network stack has already started, which will corrupt the internal state and lead to unpredictable DHCP behavior.

Uso:

- This function is the mandatory first step in the DHCP lifecycle. It is typically paired with ca_dhcp_service_start() to begin the DORA (Discover, Offer, Request, Acknowledge)

process. Developers use it to register application-specific callbacks (on_bound/on_expire) that allow the main application to know when the network is ready or when the lease is lost. It is usually called in the system initialization sequence alongside other network services like ARP, NTP, or DNS to ensure the network stack is fully configured before traffic begins.

Esempio:

```c
/* Define callbacks to handle network events */
void on_dhcp_bound(void) { /* Logic for when IP is assigned */ }
void on_dhcp_expire(void) { /* Logic for when lease is lost */ }

void system_init(void) {
    ca_mac_t my_mac = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55};

    /* Initialize DHCP service with callbacks */
    ca_dhcp_service_init(my_mac, on_dhcp_bound, on_dhcp_expire);

    /* Start the service to begin network discovery */
    ca_dhcp_service_start();
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_dhcp_service_init() in my caOS application?
A: You should call ca_dhcp_service_init() before starting the DHCP service with ca_dhcp_service_sta
It is used to initialize the DHCP client and register callbacks for when an IP address is assigned or expires. This function must be called during application initialization, not from an ISR.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_dhcp_service_init() with caOS types?
A: u8 mac[6] = {0x00, 0x80, 0xE1, 0x23, 0x45, 0x67}; void on_bound_cb(ip4_t ip) { // Handle IP assignment } void on_expire_cb(void) { // Handle IP expiration } ca_dhcp_service_init(mac, on_bound_cb, on_expire_cb);

Parameters conf:0.90 **Q:** What is the significance of the local_mac parameter in ca_dhcp_service_init()?
A: The local_mac parameter is the MAC address of the device, used to uniquely identify the device during the DHCP negotiation process. It must be a valid, non-NULL array of 6 bytes representing the MAC address. Passing NULL or an invalid MAC will result in undefined behavior or crashes.

Error/Edge case conf:0.85 **Q:** What are the common mistakes when calling ca_dhcp_service_init()?
A: Common mistakes include passing NULL for any of the parameters (local_mac, on_bound, or on_expire), which can lead to crashes or undefined behavior. Additionally, calling this function from an ISR or before initializing the network stack may cause unexpected results. The function must be called with valid, non-NULL callbacks and a valid MAC address.

Behavior conf:0.90 **Q:** Is ca_dhcp_service_init() blocking or non-blocking, and can it be called from an ISR?
A: ca_dhcp_service_init() is a non-blocking function and must not be called from an ISR. It is thread-safe if all parameters are valid, but it must be called before starting the DHCP service with ca_dhcp_service_start().

`ca_dhcp_service_net_ready`

Header: `caos/net/inc/dhcp_service.h:130`
Source: `caos/net/src/dhcp_service.c:513:517`
GUARD: `EN_DHCP_SERVICE`

```
void ca_dhcp_service_net_ready(void)
```

Signals the DHCP service that the network interface is initialized and ready to begin the DORA process.

Ritorna:
void

Note:

- This function is a non-blocking wrapper that interacts with a semaphore to gate the DHCP service state machine. It is thread-safe and can be called from tasks, but it is not intended for use inside an ISR. The function performs a 'trywait' followed by a 'post' to ensure the semaphore state is consistent, effectively acting as a 'gate-opener' for the DHCP task. A common mistake is calling this before the Ethernet link is physically established or before the network stack is fully initialized; doing so will cause the DHCP task to attempt communication on a non-existent link, leading to failed discovery packets.

Uso:

- This function is used as a synchronization trigger in the system startup sequence. It is typically called after the Ethernet link has been verified (e.g., via ca_eth_link_poll) and the network stack is ready to handle UDP traffic on port 68. It pairs with the internal DHCP task loop, which remains blocked until this function is invoked. By calling this, you transition the DHCP service from an idle/waiting state to the active DORA (Discover, Offer, Request, Acknowledge) state, allowing the device to obtain an IP address.

Esempio:

```
/* Ensure the physical link is up before signaling the DHCP service */
ca_eth_link_poll();
ca_os_sleep(200); // Allow time for PHY stabilization

/* Signal the DHCP service that it is safe to start the DORA process */
ca_dhcp_service_net_ready();

/* Now the DHCP task can process incoming UDP packets */
ca_stdout_put_n("DHCP service is now active and waiting for OFFERs");
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_dhcp_service_net_ready() and why is it important for network operations?
A: You should call ca_dhcp_service_net_ready() when the network is ready to be used, typically after a successful DHCP negotiation or network initialization. It is important because it signals waiting tasks to proceed by incrementing a semaphore, effectively unblocking them. This function is used in network service examples to synchronize tasks that are waiting for network readiness.

Code example conf:0.85 **Q:** Can you provide a minimal C code example using ca_dhcp_service_net_ready() with caOS types?
A: void example_task(void* arg) { // Wait for network readiness ca_sem_wait(&network_ready_sem); // Network is ready, proceed with operations // ... network code here ... }
void network_init_done(void) { // Signal that network is ready ca_dhcp_service_net_ready(); }

Return conf:0.90 **Q:** What happens if ca_dhcp_service_net_ready() fails, and what causes such a failure?
A: If ca_dhcp_service_net_ready() fails, it returns an internal error due to improper initialization of the semaphore. The function itself does not return a value, but if the

semaphore is not initialized correctly, calling this function can lead to a crash or undefined behavior. The function performs a trywait followed by a post, so it does not permanently alter the semaphore state.

---

ca_dhcp_service_renew

Header: caos/net/inc/dhcp_service.h:133
Source: caos/net/src/dhcp_service.c:519:523
GUARD: EN_DHCP_SERVICE

```
void ca_dhcp_service_renew(void)
```

Triggers a DHCP lease renewal process by sending a renewal event to the DHCP service queue.

Ritorna:
This function returns void and does not provide feedback on whether the renewal request was successfully queued.

Note:

- This function is non-blocking because it uses ca_queue_static_trysend, which attempts to place the event in the queue without waiting if the queue is full. It is thread-safe as it relies on the underlying queue implementation, but it is intended to be called from task contexts rather than ISRs, as queue operations can be complex. A mandatory precondition is that the DHCP service must have been initialized and the event queue must be active; otherwise, the event will be silently dropped. The most common beginner mistake is assuming the renewal happens instantly; this function only queues a request, and the actual network transaction will occur later when the DHCP service task processes the event.

Uso:

- This function is typically used in command-line interfaces (CLI) or system monitoring tasks to manually force the device to request a new IP address or extend its current lease from the DHCP server. It is often paired with network status checks, such as verifying that the link is up before triggering the renewal. In a typical lifecycle, it is called after the system has successfully initialized the network stack and the DHCP service, serving as a diagnostic or recovery tool when the user suspects the current lease is stale or invalid.

Esempio:

```
/* Example: Triggering a DHCP renewal via a CLI command */
#include "caos.h"

void handle_user_command(char *cmd) {
    if (ca_strncmp(cmd, "renew", 5) == 0) {
        /* Request the DHCP service to renew the lease */
        ca_dhcp_service_renew();

        /* Inform the user that the request has been queued */
        CLI_PUT_N("DHCP renewal request sent to service queue.");
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_dhcp_service_renew() and what is its purpose in a networked embedded application?

A: You should call ca_dhcp_service_renew() when you want to force a renewal of the DHCP lease without reconnecting to the network. This is useful for updating network configuration parameters like IP address, gateway, or DNS servers. The function is non-blocking and can be called from any context, including ISRs, making it suitable for real-time applications where immediate network updates are required.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_dhcp_service_renew() with caOS types?
A: void example_dhcp_renew(void) { // Ensure the event queue and semaphore are initialized before calling ca_dhcp_service_renew(); // Function returns immediately without blocking }

Parameters conf:0.85 **Q:** What are the parameters of ca_dhcp_service_renew() and what happens if I call it incorrectly?
A: The function ca_dhcp_service_renew() takes no parameters and returns void. Since there are no parameters, there are no invalid values to consider. However, calling this function without properly initializing the internal event queue or semaphore can lead to errors. The function sends an event to the event queue, so proper initialization of these components is essential for correct operation.

`ca_dhcp_service_rx_handler`

Header: `caos/net/inc/dhcp_service.h:119`
Source: `caos/net/src/dhcp_service.c:483:511`
GUARD: `EN_DHCP_SERVICE`

```
1 void ca_dhcp_service_rx_handler(const ca_udp_frame_t *udp, u16 len)
```

Processes incoming DHCP UDP packets and dispatches events to the DHCP state machine queue.

Parametri:

Nome	Tipo	Descrizione
`udp`	`const ca_udp_frame_t *`	Pointer to the incoming UDP frame structure. Must be a valid, non-NULL pointer containing valid DHCP protocol data.
`len`	`u16`	The length of the UDP payload in bytes. While currently unused in the implementation, it should represent the actual size of the received data.

Ritorna:
void. This function does not return a status code; it performs internal processing and queues events for other tasks to handle.

Note:

- This function is designed to be called from the network stack's receive callback context. It is not inherently thread-safe if called concurrently from multiple threads, though it is typically invoked by a single network task. It is non-blocking, as it uses 'ca_queue_static_trysend' to push events to a queue without waiting for space. A common beginner mistake is calling this function with a pointer to invalid memory or failing to free the network buffer after calling this handler, which leads to memory leaks in the network stack. Ensure the network stack is initialized before this handler is triggered.

Uso:

- This function is the bridge between the raw network stack and the DHCP service logic. It is typically called inside a network packet receive callback after the stack has identified the packet as a DHCP UDP frame. After calling this handler, the developer must call 'ca_net_buf_free' to release the buffer back to the system. It is used in conjunction with 'ca_dhcp_service_bind' to manage the lifecycle of an IP address acquisition, where the handler updates the state machine that 'ca_dhcp_service_bind' is waiting on.

Esempio:

```c
/* Example of calling the handler within a network receive callback */
void my_network_rx_callback(u16 buf_idx) {
    /* Retrieve buffer metadata and pointer using internal stack access */
    /* Assume 'b' is the buffer pointer retrieved from the stack */
    u8 *frame = b->data + b->data_off;
    u16 l4_off = b->meta.rx.l4_off;
    const ca_udp_frame_t *udp = (const ca_udp_frame_t *)(frame + l4_off);
    u16 udp_total = (u16)(b->data_len - l4_off);

    /* Process the DHCP packet */
    ca_dhcp_service_rx_handler(udp, udp_total);

    /* Crucial: release the buffer back to the network stack */
    ca_net_buf_free(buf_idx);
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call the ca_dhcp_service_rx_handler function in a caOS-based embedded application?
A: You should call ca_dhcp_service_rx_handler when a DHCP packet is received over UDP. It is typically invoked from an ISR or a network task that handles incoming UDP frames. This function processes the DHCP packet and sends events to the DHCP event queue for further handling in the DHCP state machine.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_dhcp_service_rx_handler with caOS types?
A: void handle_udp_frame(ca_udp_frame_t *frame, u16 length) { if (frame != NULL) { ca_dhcp_service_rx_handler(frame, length); } }

Parameters conf:0.85 **Q:** What is the significance of the 'len' parameter in ca_dhcp_service_rx_handler, and what are valid values for it?
A: The 'len' parameter specifies the length of the UDP packet received. While it is not directly used for processing, it is used for validating the packet data. Valid values are any positive u16 values that are compatible with the UDP packet structure. If the value is invalid or inconsistent with the packet, the function may ignore the packet.

Error/Edge case conf:0.90 **Q:** What are common mistakes or edge cases when using ca_dhcp_service_rx_hand
A: A common mistake is passing a NULL pointer for the 'udp' parameter, which will cause a crash or undefined behavior. Another edge case is receiving a malformed DHCP packet, which the function will ignore and log an error. Additionally, calling this function from an ISR without ensuring thread-safety of the event queue may lead to race conditions or data corruption.

Behavior conf:0.95 **Q:** Is ca_dhcp_service_rx_handler blocking or non-blocking, and is it safe to call from an ISR?
A: ca_dhcp_service_rx_handler is a non-blocking function. It is designed to be ISR-safe and thread-safe, as it only sends events to a queue without performing blocking operations. However, it should only be called from an ISR if the underlying event queue mechanism is also ISR-safe.

ca_dhcp_service_start

Header: caos/net/inc/dhcp_service.h:106
Source: caos/net/src/dhcp_service.c:471:475
GUARD: EN_DHCP_SERVICE

```
void ca_dhcp_service_start(void)
```

Initializes and launches the background DHCP client task within the RTOS environment.

Ritorna:
This function returns void and does not provide a status code; it assumes the OS job scheduler successfully queues the task.

Note:

- This function is non-blocking as it merely registers a new task with the OS scheduler. It is not thread-safe to call this from an ISR, as it involves memory allocation and scheduler manipulation. You must ensure the network stack and underlying hardware drivers are initialized before calling this, otherwise, the DHCP task will immediately fault upon execution. A common beginner mistake is calling this multiple times; doing so will spawn multiple redundant DHCP client tasks, leading to unpredictable network behavior and potential memory exhaustion.

Uso:

- This function is typically called once during the system startup sequence inside 'ca_os_main'. It is part of the network initialization phase, usually paired with other service starters like 'ca_arp_service_start' or 'ca_udp_server_start'. Once started, the DHCP service runs autonomously in the background, managing IP address acquisition and lease renewals without further intervention from the main application thread.

Esempio:

```
void ca_os_main(void) {
    /* Initialize network services during system boot */
    EVAL_ARP_SVC(ca_arp_service_start());

    /* Start the DHCP client to request an IP address */
    ca_dhcp_service_start();

    /* Continue with other system tasks */
    ca_os_job("main_app", CA_TH_LEVEL5, app_task, 0, STACK_SMALL, CA_JOB_STD,
    CA_FAULT_RESTART_THREAD);
}
```

Q&A — fine-tuning review *2/2 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_dhcp_service_start() and why is it important for network configuration?
A: You should call ca_dhcp_service_start() after initializing the RTOS system to enable automatic IP address acquisition via DHCP. This function starts a task that handles the DHCP protocol, allowing your STM32 device to dynamically obtain an IP address from a DHCP server on the network. It's important because it simplifies network setup and avoids manual IP configuration, which is especially useful in embedded applications where devices may be deployed in various network environments.

Code example conf:0.90 **Q:** Can you provide a minimal C example using caOS types that demonstrates how to start the DHCP service?
A: void example_dhcp_start(void) { // Initialize RTOS and network components first // ... other initialization code ...

// Start DHCP service ca_dhcp_service_start();
// The function is non-blocking and thread-safe // No return value expected, but check for resource errors }

`ca_dma2d_abort`

Header: `caos/drivers/inc/dma2d.h:112`
Source: `caos/drivers/src/dma2d.c:183:188`
GUARD: `EN_DMA2D`

```
void ca_dma2d_abort(void)
```

Immediately stops any ongoing DMA2D hardware operation and resets the internal driver state.

Ritorna:
void

Note:

- This function is non-blocking and performs an immediate hardware-level abort of the DMA2D peripheral. It is not thread-safe by itself, so ensure that no other task attempts to initiate a new DMA2D operation while this is executing. It is generally safe to call from an ISR if the hardware state needs to be cleared urgently, but it is primarily intended for task-level error recovery. A common mistake is calling this function without checking if the DMA2D is actually busy, which is harmless but unnecessary; however, failing to call it after a timeout or error will leave the DMA2D driver in a 'busy' state, preventing any subsequent operations from starting.

Uso:

- Use this function when a DMA2D operation (like a fill, copy, or blend) takes too long or encounters an error, and you need to force the hardware to stop to regain control. It is typically paired with ca_dma2d_busy() to check if an operation is still running before deciding to abort. In a typical sequence, you might start an operation with a non-blocking call, wait for a specific condition, and if that condition is not met within a timeout, call ca_dma2d_abort() to clean up the driver state before attempting the next operation.

Esempio:

```
/* Example: Aborting a long-running fill operation */
ca_dma2d_fill(&d, IW, IH, 0x11223344U, CA_NO_WAIT_TIME);

/* Wait for a short period to see if it finishes */
ca_os_delay_ms(10);

/* If it is still busy, force an abort to free the hardware */
if (ca_dma2d_busy()) {
    ca_dma2d_abort();
    ca_printf_n("DMA2D operation timed out and was aborted.");
}

/* Now the DMA2D is ready for the next task */
```

Q&A — fine-tuning review *2/2 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_dma2d_abort() in my application?
A: You should call ca_dma2d_abort() when a DMA2D operation, such as a fill, copy, or blend, takes too long or encounters an error. It is used to force the hardware to stop

and reset the internal driver state to regain control. It is typically used after a timeout or error condition to prevent the driver from remaining stuck in a 'busy' state, which would otherwise block subsequent operations.

Code example conf:0.95 **Q:** How can I implement a timeout-based abort for a DMA2D operation using caOS?
A: You can check the status with ca_dma2d_busy() and trigger the abort if the operation exceeds your expected duration. Example:
void handle_dma_timeout(u32 timeout_limit) { u32 counter = 0; while (ca_dma2d_busy() && (counter < timeout_limit)) { counter++; } if (ca_dma2d_busy()) { ca_dma2d_abort(); } }

`ca_dma2d_blend`

Header: `caos/drivers/inc/dma2d.h:109`
Source: `caos/drivers/src/dma2d.c:157:166`
GUARD: `EN_DMA2D`

```
bool ca_dma2d_blend(const ca_dma2d_src_t *fg, const ca_dma2d_src_t *bg,
    const ca_dma2d_dst_t *d, u32 w, u32 h, u32 timeout_ms)
```

Performs a hardware-accelerated alpha blending of two source images into a destination buffer using the DMA2D peripheral.

Parametri:

Nome	Tipo	Descrizione
`fg`	`const ca_dma2d_src_t *`	Pointer to the foreground image source structure containing address, format, and alpha settings.
`bg`	`const ca_dma2d_src_t *`	Pointer to the background image source structure.
`d`	`const ca_dma2d_dst_t *`	Pointer to the destination buffer structure defining the target memory area and format.
`w`	`u32`	Width of the area to blend in pixels.
`h`	`u32`	Height of the area to blend in pixels.
`timeout_ms`	`u32`	Maximum time in milliseconds to wait for the DMA2D hardware to complete the operation.

Ritorna:
Returns true if the blending operation was successfully queued and completed within the timeout. Returns false if parameters are invalid, the hardware is busy, or the operation timed out.

Note:

- This function is blocking; it waits for the DMA2D hardware to finish the transfer before returning. It is not thread-safe by itself, meaning if multiple tasks attempt to use the DMA2D simultaneously, you must protect this call with a mutex or ensure exclusive access. It must NOT be called from an ISR because it performs a blocking wait. Before calling this, ensure the DMA2D peripheral has been initialized via ca_dma2d_init. A common mistake is passing unaligned memory addresses or invalid dimensions that exceed the buffer boundaries, which will cause the function to return false immediately.

Uso:

- This function is typically used in graphics rendering pipelines to compose UI elements, such as overlaying icons or text onto a background. It is usually paired with ca_dma2d_init during system startup. In a typical sequence, you define the source and destination structures, call ca_dma2d_blend to perform the hardware operation, and then proceed to the next rendering step. If you need to perform multiple operations, you can check ca_dma2d_busy before initiating a new one to avoid contention.

Esempio:

```c
/* Ensure DMA2D is initialized before use */
ca_dma2d_init();

/* Define source and destination areas */
ca_dma2d_src_t fg = { /* ... initialize with address, format, alpha ... */ };
ca_dma2d_src_t bg = { /* ... initialize with address, format ... */ };
ca_dma2d_dst_t dst = { /* ... initialize with destination address, format ... */
    };

/* Perform the blend operation */
if (ca_dma2d_blend(&fg, &bg, &dst, 100, 100, 500)) {
    /* Success: blending completed within 500ms */
} else {
    /* Failure: check ca_dma2d_errors() to diagnose the issue */
    if (ca_dma2d_errors() != 0U) {
        ca_dma2d_abort();
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_dma2d_blend function in my caOS application?
A: You should use ca_dma2d_blend when you need to perform hardware-accelerated alpha blending of two images, such as overlaying icons or text onto a background in your UI rendering pipeline. It must be called after initializing the DMA2D peripheral with ca_dma2d_init and should be protected by a mutex if multiple tasks access the DMA2D peripheral.

Code example conf:0.95 **Q:** How do I structure a call to ca_dma2d_blend using caOS types?
A: Ensure your source and destination structures are defined, then call the function as follows: ca_dma2d_src_t fg = {...}; ca_dma2d_src_t bg = {...}; ca_dma2d_dst_t dst = {...}; u32 w = 100; u32 h = 50; u32 timeout = 100; bool success = ca_dma2d_blend(&fg, &bg, &dst, w, h, timeout);

Parameters conf:1.00 **Q:** What are the constraints on the parameters passed to ca_dma2d_blend?
A: The function requires pointers to valid source and destination structures, along with width (w) and height (h) dimensions. A critical constraint is that memory addresses must be aligned and dimensions must not exceed buffer boundaries; failure to meet these requirements will cause the function to return false immediately.

Return conf:1.00 **Q:** What do the return values of ca_dma2d_blend indicate?
A: The function returns true if the blending operation was successfully queued and completed within the specified timeout_ms. It returns false if the parameters are invalid (e.g., unaligned addresses or out-of-bounds dimensions), if the DMA2D hardware is busy, or if the operation fails to complete before the timeout expires.

`ca_dma2d_bpp`

Header: `caos/drivers/inc/dma2d.h:115`
Source: `caos/drivers/src/dma2d.c:41:44`
GUARD: `EN_DMA2D`

```
u32 ca_dma2d_bpp(ca_dma2d_fmt_t fmt); /* bits per pixel, 0 = unknown */
```

Returns the number of bits per pixel (BPP) for a given DMA2D color format.

Parametri:

Nome	Tipo	Descrizione
`fmt`	`ca_dma2d_fmt_t`	The DMA2D color format identifier (e.g., RGB565, ARGB8888). Must be a valid enum value defined by the driver.

Ritorna:
The number of bits per pixel (e.g., 8, 16, 24, 32) as a u32. Returns 0 if the format is unknown or invalid.

Note:

- This function is a simple lookup table accessor and is thread-safe, non-blocking, and can be safely called from an ISR. It does not perform any hardware interaction, so it does not require the DMA2D peripheral to be initialized to return a value, though using an invalid format will result in a 0 return. The most common mistake is assuming that a non-zero return guarantees the format is supported by the specific hardware revision; always validate the format against your specific STM32 reference manual if you are unsure.

Uso:

- This function is typically used as a helper during buffer validation or pixel manipulation tasks. It is frequently paired with memory management functions like ca_memory_allocate to calculate the required buffer size (stride * height * (bpp / 8)) before initiating a transfer with ca_dma2d_copy or ca_dma2d_fill. It is also used in custom pixel-access routines to determine how many bytes to read or write when manually parsing image data in memory.

Esempio:

```
/* Example: Calculate buffer size and validate format before a DMA2D copy */
#include "caos.h"

void prepare_dma2d_transfer(ca_dma2d_fmt_t fmt, u32 width, u32 height) {
    u32 bpp = ca_dma2d_bpp(fmt);

    if (bpp == 0U) {
        /* Handle error: unknown format */
        return;
    }

    /* Calculate bytes per line */
    u32 bytes_per_line = width * (bpp / 8U);

    /* Allocate memory for the transfer buffer */
    void *buffer = ca_memory_allocate(bytes_per_line * height);

    if (buffer != NULL) {
        /* Proceed with DMA2D operation */
        /* ca_dma2d_copy(buffer, ...); */
        ca_memory_free(buffer);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_dma2d_bpp in my caOS application?

A: You should use this function when you need to calculate buffer sizes for memory allocation or when performing manual pixel manipulation. It is commonly used to determine the required memory footprint by calculating (stride * height * (bpp / 8)) before initiating DMA2D operations like ca_dma2d_copy or ca_dma2d_fill.

Code example conf:1.00 **Q:** How do I calculate the buffer size for an image using ca_dma2d_bpp?
A: u32 width = 320; u32 height = 240; ca_dma2d_fmt_t format = RGB565; u32 bpp = ca_dma2d_bpp(format); u32 buffer_size = (width * height * (bpp / 8));

Parameters conf:1.00 **Q:** What does the fmt parameter represent and what happens if I pass an invalid value?
A: The fmt parameter is a ca_dma2d_fmt_t enum identifier representing the DMA2D color format. If you pass an invalid or unknown format value, the function will return 0.

Return conf:1.00 **Q:** What does a return value of 0 from ca_dma2d_bpp indicate?
A: A return value of 0 indicates that the provided color format is either unknown or invalid. Note that a non-zero return value does not guarantee that the specific STM32 hardware revision supports that format; you should always verify support against your specific STM32 reference manual.

`ca_dma2d_busy`

Header: `caos/drivers/inc/dma2d.h:113`
Source: `caos/drivers/src/dma2d.c:190:193`
GUARD: `EN_DMA2D`

```
bool ca_dma2d_busy(void)
```

Checks if the DMA2D hardware peripheral is currently performing a data transfer operation.

Ritorna:
Returns true if the DMA2D engine is busy processing a command, or false if it is idle or the driver is not initialized.

Note:

- This function is non-blocking and provides a snapshot of the hardware state. It is thread-safe in the sense that it reads a volatile hardware status register, but it does not guarantee that the state won't change immediately after the function returns. It is safe to call from an ISR, though typically used in task context to poll for completion before starting a new operation or to verify if an abort was successful. A common beginner mistake is to assume that if this returns false, the memory buffers are immediately safe to modify; always ensure the DMA2D operation has fully completed or been aborted before accessing the target memory.

Uso:

- This function is typically used in a polling loop or a conditional check to determine if the DMA2D engine is available for a new task. It is often paired with ca_dma2d_fill, ca_dma2d_copy, or ca_dma2d_blend to check if a previous operation has finished before triggering a new one. In scenarios where a timeout occurs, it is used in conjunction with ca_dma2d_abort to verify that the hardware has successfully stopped its current activity before proceeding with error recovery or re-initialization.

Esempio:

```
/* Check if DMA2D is busy before starting a new fill operation */
if (!ca_dma2d_busy()) {
    /* Safe to start a new operation */
```

```
    ca_dma2d_fill(&d, 100, 100, 0xFFFFFFFFU, 0U);
} else {
    /* DMA2D is currently occupied, wait for it to finish */
    ca_dma2d_wait();
    /* Now it is safe to proceed */
    ca_dma2d_fill(&d, 100, 100, 0xFFFFFFFFU, 0U);
}

/* Example of checking status after an abort attempt */
ca_dma2d_abort();
if (!ca_dma2d_busy()) {
    /* Hardware successfully aborted and is now idle */
    ca_printf("DMA2D successfully aborted.\n");
} else {
    /* Hardware is still busy, handle error */
    ca_printf("DMA2D failed to abort in time.\n");
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_dma2d_busy and why is it important?
A: You should call ca_dma2d_busy to check if the DMA2D engine is available before triggering a new operation like ca_dma2d_fill, ca_dma2d_copy, or ca_dma2d_blend. It is also used to verify that the hardware has stopped after calling ca_dma2d_abort during error recovery. It is important because it prevents starting new operations while the hardware is still processing.

Code example conf:0.90 **Q:** Provide a minimal C example showing how to wait for the DMA2D engine to become idle using caOS types.
A: void wait_for_dma2d(void) { while (ca_dma2d_busy()) { /* Wait for hardware to finish */ } }

Return conf:1.00 **Q:** What do the return values of ca_dma2d_busy indicate, and what causes a false return?
A: The function returns true if the DMA2D engine is currently processing a command. It returns false if the engine is idle or if the driver is not initialized. Note that a false return does not guarantee that memory buffers are safe to modify; you must ensure the operation has fully completed or been aborted.

`ca_dma2d_convert`

Header: `caos/drivers/inc/dma2d.h:108`
Source: `caos/drivers/src/dma2d.c:148:155`
GUARD: `EN_DMA2D`

```
bool ca_dma2d_convert(const ca_dma2d_src_t *s, const ca_dma2d_dst_t *d,
    u32 w, u32 h, u32 timeout_ms)
```

Performs a hardware-accelerated pixel format conversion (PFC) or memory-to-memory copy using the DMA2D peripheral.

Parametri:

Nome	Tipo	Descrizione
s	const ca_dma2d_src_t *	Pointer to the source configuration structure, defining memory address, format, and alpha blending parameters.
d	const ca_dma2d_dst_t *	Pointer to the destination configuration structure, defining memory address, stride, and output format.
w	u32	Width of the area to process in pixels.
h	u32	Height of the area to process in pixels.
timeout_ms	u32	Maximum time in milliseconds to wait for the hardware operation to complete.

Ritorna:
Returns true if the conversion was successfully queued and completed within the timeout; returns false if parameters are invalid, the hardware is busy, or the operation timed out.

Note:

- This function is blocking; it waits for the DMA2D hardware to finish the transfer before returning. It is not thread-safe by itself, meaning multiple tasks should not access the DMA2D peripheral simultaneously without external synchronization (e.g., a mutex). It must NOT be called from an ISR because it performs blocking waits. Before calling this, ensure the DMA2D peripheral has been initialized via ca_dma2d_init. A common mistake is passing unaligned memory addresses or invalid strides that do not match the hardware requirements of the STM32 DMA2D engine.

Uso:

- This function is typically used in graphics rendering pipelines to convert image data from one format (e.g., RGB565) to another (e.g., ARGB8888) before displaying it on a screen. It is often paired with ca_dma2d_init during system startup to prepare the hardware. In a typical sequence, a developer defines the source and destination buffers, calls ca_dma2d_convert to perform the transformation, and then proceeds to update the display buffer or trigger a screen refresh.

Esempio:

```
/* Define source and destination buffers */
ca_dma2d_src_t src = { .addr = (void*)0x20001000, .fmt = 0, .alpha = 255 };
ca_dma2d_dst_t dst = { .addr = (void*)0x20002000, .stride = 800, .fmt = 1 };

/* Perform conversion with a 100ms timeout */
if (ca_dma2d_convert(&src, &dst, 800, 480, 100)) {
    /* Conversion successful, proceed to display update */
} else {
    /* Handle error: check ca_dma2d_errors() or log failure */
    ca_printf("DMA2D conversion failed!\n");
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_dma2d_convert in my graphics pipeline?
A: You should use ca_dma2d_convert when you need to perform hardware-accelerated pixel format conversion or memory-to-memory copies, such as converting image data from RGB565 to ARGB8888 before a display refresh. It is typically called after initializing the peripheral with ca_dma2d_init.

Code example conf:0.90 **Q:** How do I call ca_dma2d_convert in a C program using caOS types?
A: You must define the source and destination structures first. Example: ca_dma2d_src_t src = { ... }; ca_dma2d_dst_t dst = { ... }; u32 w = 320; u32 h = 240; u32 timeout = 100; bool success = ca_dma2d_convert(&src, &dst, w, h, timeout);

Parameters conf:1.00 **Q:** What are the requirements for the source and destination parameters in ca_dma2d_convert?
A: The source parameter defines the memory address, format, and alpha blending, while the destination parameter defines the memory address, stride, and output format. You must ensure that memory addresses are aligned and strides match the hardware requirements of the STM32 DMA2D engine to avoid failures.

Return conf:1.00 **Q:** What does a false return value from ca_dma2d_convert indicate?
A: A false return indicates that the operation failed. This can be caused by invalid parameters, the DMA2D hardware being busy, or the operation exceeding the specified timeout_ms duration.

`ca_dma2d_copy`

Header: `caos/drivers/inc/dma2d.h:107`
Source: `caos/drivers/src/dma2d.c:132:146`
GUARD: `EN_DMA2D`

```
bool ca_dma2d_copy(const ca_dma2d_src_t *s, const ca_dma2d_dst_t *d, u32 w, u32 h, u32 timeout_ms)
```

Copies a rectangular area of pixel data from a source buffer to a destination buffer using the hardware DMA2D engine.

Parametri:

Nome	Tipo	Descrizione
`s`	`const ca_dma2d_src_t *`	Pointer to the source configuration structure containing address, format, and stride. Must not be NULL.
`d`	`const ca_dma2d_dst_t *`	Pointer to the destination configuration structure. Must not be NULL and must match the source format.
`w`	`u32`	Width of the rectangular area to copy in pixels.
`h`	`u32`	Height of the rectangular area to copy in pixels.
`timeout_ms`	`u32`	Maximum time in milliseconds to wait for the DMA2D hardware to complete the operation.

Ritorna:
Returns true if the copy operation was successfully initiated and completed within the timeout; returns false if parameters are invalid, formats mismatch, or the hardware operation timed out.

Note:

- This function is blocking; it will wait for the DMA2D hardware to finish the transfer before returning. It is thread-safe regarding the DMA2D peripheral access, but ensure that the source and destination memory buffers are not modified by other tasks while the transfer is in progress. It must NOT be called from an ISR because it relies on blocking synchronization mechanisms. A common beginner mistake is passing a width or height that exceeds the buffer boundaries, which will cause the function to return false immediately to prevent memory corruption.

Uso:

- Use this function when you need to move graphical assets, such as icons or UI elements, from a source memory location to a display buffer or off-screen surface. It is typically used after initializing the DMA2D peripheral with ca_dma2d_init and defining the source and destination structures. It is often paired with ca_dma2d_fill or ca_dma2d_blend to

compose complex UI frames. Always verify the return value; if it returns false, the copy did not occur, and you should handle the error to prevent rendering artifacts.

Esempio:

```
/* Assuming ca_dma2d_init() has been called previously */
ca_dma2d_src_t src = { .addr = (u32)source_buffer, .stride = 320, .fmt =
    CA_DMA2D_ARGB8888 };
ca_dma2d_dst_t dst = { .addr = (u32)display_buffer, .stride = 320, .fmt =
    CA_DMA2D_ARGB8888 };

/* Attempt to copy a 100x100 area */
if (!ca_dma2d_copy(&src, &dst, 100, 100, 500)) {
    /* Handle error: log failure or retry */
    ca_printf("DMA2D copy failed!\n");
} else {
    /* Copy successful, proceed with next UI update */
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_dma2d_copy in my caOS application?
A: You should use ca_dma2d_copy when you need to move graphical assets, such as icons or UI elements, from a source memory location to a display buffer or off-screen surface. It is typically used after initializing the DMA2D peripheral and is often paired with ca_dma2d_fill or ca_dma2d_blend to compose complex UI frames.

Code example conf:0.90 **Q:** Provide a minimal example of how to call ca_dma2d_copy.
A: ca_dma2d_src_t src = { .addr = 0x20000000, .format = 0, .stride = 100 }; ca_dma2d_dst_t dst = { .addr = 0x20001000, .format = 0, .stride = 100 }; u32 w = 50; u32 h = 50; u32 timeout = 100; bool success = ca_dma2d_copy(&src, &dst, w, h, timeout);

Parameters conf:1.00 **Q:** What are the requirements for the source and destination parameters in ca_dma2d_copy?
A: The source (s) and destination (d) pointers must not be NULL. Furthermore, the destination configuration must match the source format. If these conditions are not met, the function will return false.

Return conf:1.00 **Q:** What does the boolean return value of ca_dma2d_copy indicate?
A: The function returns true if the copy operation was successfully initiated and completed within the specified timeout. It returns false if the parameters are invalid, the formats mismatch, or the hardware operation timed out.

Error/Edge case conf:1.00 **Q:** What are common mistakes or edge cases that cause ca_dma2d_copy to fail?
A: A common mistake is passing a width or height that exceeds the buffer boundaries, which causes the function to return false to prevent memory corruption. Additionally, the function must not be called from an ISR because it uses blocking synchronization, and you must ensure buffers are not modified by other tasks during the transfer.

`ca_dma2d_errors`

Header: `caos/drivers/inc/dma2d.h:114`
Source: `caos/drivers/src/dma2d.c:195:198`
GUARD: `EN_DMA2D`

```
u32 ca_dma2d_errors(void); /* CA_DMA2D_ERR_* */
```

Returns the current error status flags of the DMA2D hardware peripheral.

Ritorna:

Returns a 32-bit unsigned integer representing a bitmask of error flags (CA_DMA2D_ERR_*). A return value of 0 indicates that no errors have been detected since the last reset or clear operation.

Note:

- This function is non-blocking and thread-safe, as it simply reads a status variable updated by the DMA2D interrupt service routine. It is safe to call from any context, including ISRs, because it does not perform any hardware interaction or wait for peripheral completion. The most common beginner mistake is assuming this function clears the error flags; it does not. You must ensure your application logic handles the error state and potentially triggers a reset or re-initialization of the DMA2D peripheral if a critical error bit is set.

Uso:

- This function is typically used for diagnostic purposes after a DMA2D operation (like ca_dma2d_copy or ca_dma2d_fill) has failed or timed out. It is usually paired with ca_dma2d_wait to verify if a transfer completed successfully. If ca_dma2d_wait returns an error or if the operation appears stuck, you call ca_dma2d_errors to log the specific failure reason (e.g., configuration error, transfer error, or bus error) to the CLI or a debug buffer.

Esempio:

```
/* Perform a DMA2D copy operation */
ca_dma2d_copy(src, dst, width, height);

/* Wait for the hardware to finish the operation */
if (ca_dma2d_wait() != 0) {
    /* Operation failed, retrieve the error code */
    u32 err = ca_dma2d_errors();

    /* Log the error to the console for debugging */
    ca_printf("DMA2D operation failed with error code: 0x%08X\n", err);

    /* Perform recovery: abort current transfer and re-init if necessary */
    ca_dma2d_abort();
    ca_dma2d_init();
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_dma2d_errors in my application logic?
A: You should call ca_dma2d_errors for diagnostic purposes after a DMA2D operation, such as ca_dma2d_copy or ca_dma2d_fill, has failed or timed out. It is typically paired with ca_dma2d_wait to verify if a transfer completed successfully, especially if the operation appears stuck or returns an error.

Code example conf:1.00 **Q:** How do I check for DMA2D errors in my code using caOS types?
A: You can check the status by calling the function and evaluating the returned bitmask. Example: u32 err = ca_dma2d_errors(); if (err != 0) { /* Handle error state */ }

Return conf:1.00 **Q:** What does the return value of ca_dma2d_errors represent, and does calling it clear the flags?
A: The return value is a 32-bit unsigned integer representing a bitmask of CA_DMA2D_ERR_* error flags. A return value of 0 indicates no errors. Calling this function does not clear the error flags; you must handle the error state and potentially trigger a reset or re-initialization of the DMA2D peripheral manually.

ca_dma2d_fill

Header: caos/drivers/inc/dma2d.h:106
Source: caos/drivers/src/dma2d.c:124:130
GUARD: EN_DMA2D

```
bool ca_dma2d_fill(const ca_dma2d_dst_t *d, u32 w, u32 h, u32 color, u32
    timeout_ms); /* color in d->fmt */
```

Fills a rectangular area of a graphics buffer with a solid color using the DMA2D hardware accelerator.

Parametri:

Nome	Tipo	Descrizione
d	const ca_dma2d_dst_t *	Pointer to the destination buffer structure, containing address, format, and stride information.
w	u32	Width of the rectangle to fill in pixels.
h	u32	Height of the rectangle to fill in pixels.
color	u32	The color value to fill with, formatted according to the destination buffer's pixel format.
timeout_ms	u32	Maximum time in milliseconds to wait for the DMA2D hardware to complete the operation.

Ritorna:
Returns true if the fill operation was successfully queued and completed within the timeout. Returns false if parameters are invalid, the hardware is busy, or the operation timed out.

Note:

- This function is blocking; it waits for the DMA2D hardware to finish the operation before returning. It is not safe to call from an ISR because it relies on internal OS waiting mechanisms. Before calling this, ensure the DMA2D peripheral has been initialized via ca_dma2d_init. A common mistake is passing a color value that does not match the destination buffer's format (e.g., passing a 32-bit ARGB color to a 16-bit RGB565 buffer), which will result in corrupted graphics output.

Uso:

- This function is typically used to clear the screen or draw solid rectangular UI elements like buttons or backgrounds. It is often paired with ca_dma2d_copy or ca_dma2d_blend to compose complex frames. In a typical workflow, you define the destination area using a structure, call this function to fill the background, and then perform other drawing operations on top. If the operation returns false, you should check ca_dma2d_errors to diagnose if the hardware encountered a configuration or bus error.

Esempio:

```
/* Initialize DMA2D before use */
ca_dma2d_init();

/* Define destination area: address, format, and stride */
ca_dma2d_dst_t dst = { .addr = 0xC0000000, .fmt = 1, .stride = 800 };

/* Fill a 100x100 area with white (0xFFFFFFFF) */
const u32 timeout = 500;
if (ca_dma2d_fill(&dst, 100, 100, 0xFFFFFFFF, timeout)) {
    /* Success: area is filled */
} else {
    /* Handle error: check if hardware is busy or parameters were invalid */
```

```
    if (ca_dma2d_busy()) {
        /* Wait or abort if necessary */
        ca_dma2d_abort();
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_dma2d_fill in my caOS application?
A: You should use ca_dma2d_fill to clear the screen or draw solid rectangular UI elements like backgrounds and buttons. It is typically used as a foundational step to prepare a graphics buffer before performing more complex operations like ca_dma2d_copy or ca_dma2d_blend.

Code example conf:0.90 **Q:** How do I call ca_dma2d_fill in my code?
A: Ensure you have initialized the peripheral first. Example: ca_dma2d_dst_t dest = { .addr = 0x20000000, .fmt = 0, .stride = 800 }; bool success = ca_dma2d_fill(&dest, 100, 50, 0xFFFFFFFF, 100);

Parameters conf:1.00 **Q:** What is the significance of the color parameter in ca_dma2d_fill?
A: The color parameter must be formatted according to the destination buffer's pixel format defined in the ca_dma2d_dst_t structure. Passing a color value that does not match the buffer's format, such as a 32-bit ARGB value for a 16-bit RGB565 buffer, will result in corrupted graphics output.

Return conf:1.00 **Q:** What does a false return value from ca_dma2d_fill indicate?
A: A false return indicates that the operation failed. This can be caused by invalid parameters, the DMA2D hardware being busy, or the operation exceeding the specified timeout_ms. If it returns false, you should check ca_dma2d_errors to diagnose potential configuration or bus errors.

`ca_dma2d_hw_abort`

Header: `caos/drivers/inc/ca_glue_hw.h:426`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_dma2d_hw_abort(void); /* ABORT, until START falls */
```

Forces an immediate stop of the DMA2D hardware engine and clears pending interrupt flags.

Ritorna:
void

Note:

- This function is non-blocking and performs a hardware-level abort by setting the ABORT bit in the DMA2D control register. It includes a short spin-loop to wait for the hardware to acknowledge the stop, ensuring the engine is idle before returning. It is thread-safe in the sense that it interacts directly with hardware registers, but it should not be called from an ISR unless you are certain the DMA2D is not currently being managed by a higher-level driver task. The most common mistake is calling this function while the DMA2D is already idle or during a critical transfer that you did not intend to cancel, which may leave the display buffer in an inconsistent state.

Uso:

- This function is primarily used as a safety mechanism within driver error-handling routines, specifically when a DMA2D operation times out or encounters a hardware error. It is typically paired with ca_dma2d_wait or ca_dma2d_copy, where a timeout indicates the

hardware is stuck. By calling this, you ensure the DMA2D peripheral is reset to a clean state, allowing subsequent operations to start without interference from the previous failed transaction.

Esempio:

```
/* Example: Handling a DMA2D timeout error */
if (!ca_dma2d_wait(100)) {
    /* The DMA2D operation took too long, abort it to prevent further issues */
    ca_dma2d_hw_abort();

    /* Log the error or notify the system that the display update failed */
    ca_printf("DMA2D operation timed out, hardware aborted.\n");

    /* Proceed to re-initialize or retry the operation if necessary */
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_dma2d_hw_abort in my application?
A: You should use ca_dma2d_hw_abort as a safety mechanism during error-handling routines, particularly when a DMA2D operation times out or encounters a hardware error. It is intended to reset the peripheral to a clean state after a failed transaction, ensuring subsequent operations can proceed without interference.

Code example conf:0.90 **Q:** How do I implement a basic error recovery routine using ca_dma2d_hw_abort?
A: You can implement a recovery routine by checking for a timeout and then calling the abort function to clear the hardware state. Example:
void handle_dma_error(void) { /* Assuming a timeout occurred */ ca_dma2d_hw_abort(); /* Proceed to re-initialize or retry operation */ }

HW differences conf:1.00 **Q:** Are there differences in how ca_dma2d_hw_abort behaves across different STM32 families in caOS?
A: Yes. On families like F4xx, F7xx, H7xx, and U5xx, the function actively interacts with the DMA2D->CR register to set the ABORT bit and uses a spin-loop to wait for the START bit to clear, followed by clearing interrupt flags via DMA2D->IFCR. Conversely, for F1xx, H5xx, and certain other configurations, the function is implemented as an empty stub, meaning it performs no hardware action.

`ca_dma2d_hw_busy`

Header: `caos/drivers/inc/ca_glue_hw.h:427`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_dma2d_hw_busy(void); /* START still set */
```

Checks if the DMA2D hardware peripheral is currently performing a data transfer.

Ritorna:
Returns true if the DMA2D hardware is busy (transfer in progress), false if it is idle and ready for a new operation.

Note:

- This function is a low-level hardware check that reads the START bit in the DMA2D Control Register. It is non-blocking and thread-safe in the sense that it performs a single atomic register read. It can be called from an ISR, but it is typically used by the driver's high-level API to prevent starting a new operation while the hardware is still processing. A common beginner mistake is to call this in a tight loop without a delay or yield, which

can starve other tasks in the RTOS; always use ca_dma2d_wait if you need to block until the operation completes.

Uso:

- This function is used to poll the hardware status before initiating a new DMA2D operation like a copy, fill, or blend. It is typically paired with ca_dma2d_init for setup and ca_dma2d_wait to ensure the hardware has finished its task before accessing the destination memory buffer. It acts as a guard to prevent hardware configuration errors or data corruption caused by overlapping DMA2D operations.

Esempio:

```c
/* Example: Safely performing a memory fill operation */
#include "caos.h"

void perform_safe_fill(void *dest, u32 color, u32 w, u32 h) {
    /* Check if the hardware is currently busy before starting a new job */
    if (ca_dma2d_busy()) {
        /* If busy, wait for the current operation to finish */
        ca_dma2d_wait();
    }

    /* Now it is safe to start the fill operation */
    ca_dma2d_fill(dest, color, w, h);

    /* Optionally wait for this specific operation to complete */
    ca_dma2d_wait();
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_dma2d_hw_busy in my application code?
A: You should use it to poll the hardware status before initiating a new DMA2D operation, such as a copy, fill, or blend. It acts as a guard to prevent hardware configuration errors or data corruption from overlapping operations. Avoid calling it in a tight loop; use ca_dma2d_wait if you need to block until completion.

Code example conf:1.00 **Q:** How do I check if the DMA2D is ready before starting a new task?
A: You can check the status using a simple conditional check: if (!ca_dma2d_hw_busy()) { /* Start new DMA2D operation */ }.

Return conf:1.00 **Q:** What do the boolean return values of ca_dma2d_hw_busy indicate?
A: The function returns true if the DMA2D hardware is currently performing a data transfer (the START bit is set in the Control Register). It returns false if the hardware is idle and ready for a new operation.

HW differences conf:1.00 **Q:** Are there differences in how ca_dma2d_hw_busy behaves across different STM32 families?
A: Yes. On families like F4xx, F7xx, H7xx, and U5xx, the function performs an actual register read of the DMA2D_CR_START bit. However, in certain driver implementations for these same families (and others like F1xx and H5xx), the function is hardcoded to return false, indicating that the hardware check is not supported or implemented for those specific configurations.

`ca_dma2d_hw_clut`

Header: `caos/drivers/inc/ca_glue_hw.h:422`
GUARD: `EN_CA_GLUE_HW`

```c
extern u32 ca_dma2d_hw_clut(bool bg, const ca_dma2d_src_t *s); /* CLUT
    by CPU or DMA2D, 0 = loaded, else CA_DMA2D_ERR_* */
```

Loads a Color Look-Up Table (CLUT) into the DMA2D peripheral for indexed image formats.

Parametri:

Nome	Tipo	Descrizione
bg	bool	Selects the target: true for Background (BG) CLUT, false for Foreground (FG) CLUT.
s	const ca_dma2d_src_t *	Pointer to the source structure containing the CLUT data, size, and configuration flags.

Ritorna:
Returns 0 on success. Returns a non-zero error code (e.g., CA_DMA2D_ERR_TE) if the hardware fails to load the CLUT within the timeout period.

Note:

- This function is blocking; it waits for the hardware to complete the transfer or times out after a fixed number of iterations. It is not thread-safe if multiple tasks attempt to configure the same DMA2D layer simultaneously, so ensure proper synchronization (e.g., using a mutex) if shared. It must not be called from an ISR because it uses a busy-wait loop (spinning) which would block the CPU for too long. The most common beginner mistake is passing a pointer to a CLUT that is not properly aligned or resides in memory inaccessible to the DMA2D peripheral.

Uso:

- This function is used when rendering indexed images (like 8-bit color palettes) where the pixel values are indices into a color table. It is typically called after initializing the DMA2D source configuration via ca_dma2d_hw_src but before starting the actual transfer. It pairs with other DMA2D functions to prepare the hardware state machine for a specific graphics operation.

Esempio:

```c
/* Assuming 'src' is a valid ca_dma2d_src_t structure with a CLUT */
/* 1. Configure the source parameters first */
if (ca_dma2d_hw_src(false, &src, 0)) {
    /* 2. Load the CLUT into the Foreground layer */
    u32 status = ca_dma2d_hw_clut(false, &src);
    if (status == 0U) {
        /* CLUT loaded successfully, proceed with drawing */
        /* ca_dma2d_copy(...) or similar operation */
    } else {
        /* Handle error: log or abort */
    }
}
```

Q&A — fine-tuning review *7/7 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_dma2d_hw_clut in my graphics pipeline?
A: You should call it when rendering indexed images that require a Color Look-Up Table (CLUT). It must be invoked after initializing the DMA2D source configuration via ca_dma2d_hw_src, but before starting the actual DMA2D transfer.

Code example conf:1.00 **Q:** Provide a minimal example of how to load a CLUT using caOS types.

A: ca_dma2d_src_t my_src; my_src.clut = my_palette_data; my_src.clut_n = 256; my_src.flags = 0; u32 status = ca_dma2d_hw_clut(false, &my_src); if (status != 0) { /* Handle error */ }

Parameters conf:1.00 **Q:** What is the purpose of the 'bg' parameter in ca_dma2d_hw_clut?
A: The 'bg' boolean parameter selects the target DMA2D layer for the CLUT. Passing 'true' targets the Background (BG) CLUT, while passing 'false' targets the Foreground (FG) CLUT.

Return conf:1.00 **Q:** What do the return values of ca_dma2d_hw_clut signify?
A: A return value of 0 indicates success. A non-zero value, such as CA_DMA2D_ERR_TE, indicates that the hardware failed to load the CLUT within the fixed timeout period (DMA2D_SPIN).

Error/Edge case conf:1.00 **Q:** What are the most common mistakes when using ca_dma2d_hw_clut?
A: The most common mistake is passing a pointer to a CLUT that is not properly aligned or resides in a memory region that is inaccessible to the DMA2D peripheral.

Behavior conf:1.00 **Q:** Is ca_dma2d_hw_clut safe to use in an ISR or across multiple threads?
A: No. It is not ISR-safe because it uses a busy-wait loop (spinning) that would block the CPU for too long. It is also not thread-safe; if multiple tasks attempt to configure the same DMA2D layer, you must use synchronization primitives like a mutex.

HW differences conf:0.90 **Q:** How does the implementation of ca_dma2d_hw_clut differ across supported STM32 families?
A: While the logic is consistent across F4xx, F7xx, H7xx, and U5xx, the specific registers used for the CLUT (e.g., BGCMAR, BGPFCCR, ISR, IFCR) are accessed via the DMA2D peripheral base. The F1xx implementation is a stub that returns CA_DMA2D_ERR_TE and does not perform any hardware operations.

`ca_dma2d_hw_dst`

Header: `caos/drivers/inc/ca_glue_hw.h:423`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_dma2d_hw_dst(const ca_dma2d_dst_t *d, u32 w, u32 color);
   /* output address, offset, PFC, fill color; false = flag not on this
   part */
```

Configures the DMA2D destination hardware registers for a memory-to-memory or fill operation.

Parametri:

Nome	Tipo	Descrizione
`d`	`const ca_dma2d_dst_t *`	Pointer to the destination configuration structure containing memory address, pixel format, stride, and flags.
`w`	`u32`	The width of the area to be processed in pixels.
`color`	`u32`	The fill color value to be used if the operation is a fill (R2M mode).

Ritorna:
Returns true if the hardware registers were successfully configured based on the provided parameters. Returns false if the configuration is invalid, the pixel format is unsupported, or the hardware flags are incompatible with the current target.

Note:

- This function is a low-level hardware abstraction layer (HAL) call and is not intended to be called directly by application code. It is not thread-safe and must be used within the

context of a higher-level driver function that manages the DMA2D peripheral state. It is strictly non-blocking as it only writes to memory-mapped registers. A common mistake is calling this function without ensuring the DMA2D peripheral is initialized or while a previous DMA2D transfer is still busy, which can lead to register corruption or undefined behavior.

Uso:

- This function is used internally by the DMA2D driver to prepare the hardware before triggering a transfer. It is typically called after validating the destination parameters and before invoking the hardware-specific run command. It is part of the sequence: ca_dma2d_init -> ca_dma2d_hw_dst -> dma2d_run (internal).

Esempio:

```c
/* Example of how the driver internally prepares a fill operation */
ca_dma2d_dst_t dst_cfg;
dst_cfg.addr = (void*)0x20000000;
dst_cfg.fmt = 0; /* Assume ARGB8888 */
dst_cfg.stride = 800;
dst_cfg.flags = 0;

/* Configure the destination hardware registers */
if (ca_dma2d_hw_dst(&dst_cfg, 800, 0xFF0000FF)) {
    /* If successful, proceed to trigger the hardware transfer */
    /* Internal function call to start the DMA2D engine */
    /* dma2d_run(DMA2D_MODE_R2M, 800, 480, 100); */
} else {
    /* Handle configuration error */
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_dma2d_hw_dst in my application code?
A: You should not call this function directly from application code. It is a low-level hardware abstraction layer function intended for internal use by higher-level caOS functions like ca_dma2d_fill or ca_dma2d_copy. It must only be executed when the DMA2D peripheral is idle, which you must verify using ca_dma2d_busy before calling.

Code example conf:0.90 **Q:** Provide a minimal example of how to configure the destination hardware using caOS types.
A: ca_dma2d_dst_t dest = { .addr = (void*)0x20000000, .fmt = 0, .stride = 800, .flags = 0 }; u32 width = 100; u32 color = 0xFFFFFFFF; bool success = ca_dma2d_hw_dst(&dest, width, color);

Parameters conf:1.00 **Q:** What is the purpose of the 'w' parameter in ca_dma2d_hw_dst?
A: The 'w' parameter represents the width of the area to be processed in pixels. It is used in conjunction with the stride value from the ca_dma2d_dst_t structure to calculate the offset register (OOR) value, specifically as (stride - w).

Return conf:1.00 **Q:** What does a return value of false indicate when calling ca_dma2d_hw_dst?
A: A return value of false indicates that the hardware registers were not successfully configured. This occurs if the configuration is invalid, the pixel format is unsupported, the hardware flags are incompatible with the current target, or the specific MCU family implementation does not support this operation.

HW differences conf:1.00 **Q:** Are there differences in how ca_dma2d_hw_dst behaves across different STM32 families?
A: Yes. On the F4xx, F7xx, H7xx, and U5xx families, the function actively writes to DMA2D registers like OPFCCR, OMAR, OOR, and OCOLR. Conversely, on the F1xx and H5xx families, the function is implemented as a stub that performs no operations

and always returns false, indicating that the DMA2D hardware destination configuration is not supported on those specific targets.

---

ca_dma2d_hw_init

Header: caos/drivers/inc/ca_glue_hw.h:420
GUARD: EN_CA_GLUE_HW

```
extern bool ca_dma2d_hw_init(void); /* clock, NVIC; false = no DMA2D */
```

Initializes the DMA2D hardware peripheral, enabling its clock and configuring its interrupt controller.

Ritorna:
Returns true if the DMA2D hardware is present and successfully initialized. Returns false if the hardware is not supported on the current MCU or if initialization failed.

Note:

- This function is non-blocking and performs low-level register configuration to prepare the DMA2D peripheral for operation. It is not thread-safe and must be called during the system initialization phase before any other DMA2D functions are used. It is not intended to be called from an ISR. A common mistake is to call this function multiple times or to attempt to use DMA2D functions before this initialization has successfully returned true.

Uso:

- This function is typically called once during the system boot sequence or within the driver's initialization wrapper. It is almost always paired with ca_dma2d_init, which uses the result of this function to decide whether to proceed with setting up internal synchronization primitives like semaphores. Once initialized, the application can safely use other DMA2D functions like ca_dma2d_copy or ca_dma2d_fill.

Esempio:

```
/* Initialize the DMA2D driver during system startup */
if (ca_dma2d_init()) {
    /* DMA2D is ready, safe to perform graphics operations */
    /* Example: Fill a buffer with a solid color */
    ca_dma2d_fill(0x20000000, 320, 240, 0xFFFF0000);
    ca_dma2d_wait();
} else {
    /* Handle error: DMA2D not available or init failed */
    ca_printf("DMA2D hardware not supported or failed to init\n");
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_dma2d_hw_init in my caOS application?
A: It must be called during the system initialization phase, typically during the boot sequence. It is not thread-safe and must be executed before any other DMA2D functions are used. It is not intended to be called from an ISR.

Code example conf:1.00 **Q:** Provide a minimal example of how to initialize the DMA2D peripheral in caOS.
A: void system_init(void) { if (ca_dma2d_hw_init()) { /* Proceed with further DMA2D setup */ } }

Return conf:1.00 **Q:** What do the return values of ca_dma2d_hw_init indicate?

A: Returns true if the DMA2D hardware is present and successfully initialized. Returns false if the hardware is not supported on the current MCU or if the initialization process failed.

HW differences conf:1.00 **Q:** Are there differences in how ca_dma2d_hw_init handles hardware across STM32 families?
A: Yes. While most families (F4xx, F7xx, U5xx) enable the clock via RCC->AHB1ENR, the H7xx family enables the clock via RCC->AHB3ENR. Additionally, some families or configurations may not support DMA2D at all, in which case the function returns false.

---

ca_dma2d_hw_irq

Header: `caos/drivers/inc/ca_glue_hw.h:425`
GUARD: `EN_CA_GLUE_HW`

```
extern u32 ca_dma2d_hw_irq(void); /* ISR read and cleared: CA_DMA2D_ERR_
    * | CA_DMA2D_HW_TC */
```

Handles DMA2D hardware interrupts by reading and clearing status flags.

Ritorna:
A bitmask representing the event: CA_DMA2D_HW_TC (Transfer Complete), or error flags CA_DMA2D_ERR_TE (Transfer Error), CA_DMA2D_ERR_CE (Configuration Error), CA_DMA2D_ERR (CLUT Access Error). Returns 0 if no relevant flags are set.

Note:

- This function is designed to be called exclusively from the DMA2D_IRQHandler. It is not thread-safe and must not be called from standard application tasks, as it directly interacts with hardware registers to clear interrupt flags. If you fail to call this function inside your ISR, the DMA2D hardware will keep the interrupt line asserted, causing the CPU to re-enter the ISR immediately and hang the system. It is non-blocking and executes in constant time.

Uso:

- This function is the core of the DMA2D interrupt service routine. It is typically paired with ca_sem_post to signal a waiting task that a DMA2D operation has finished or failed. In a typical workflow, you initialize the DMA2D, start a transfer, wait for completion using ca_dma2d_wait, and the ISR uses this function to acknowledge the hardware and unblock the waiting task.

Esempio:

```
void DMA2D_IRQHandler(void) {
    /* Read and clear hardware flags */
    u32 status = ca_dma2d_hw_irq();

    /* Check if transfer finished or an error occurred */
    if (status & (CA_DMA2D_HW_TC | CA_DMA2D_ERR_TE | CA_DMA2D_ERR_CE)) {
        /* Signal the task waiting on the DMA2D operation */
        /* ca_sem_post(&dma2d_semaphore); */
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_dma2d_hw_irq in my application?
A: You should call ca_dma2d_hw_irq exclusively from within the DMA2D_IRQHandler. It is not thread-safe and must not be called from standard application tasks. Its purpose

is to read and clear hardware interrupt flags to prevent the system from hanging due to a continuously asserted interrupt line.

Code example conf:1.00 **Q:** How do I implement a basic DMA2D ISR using ca_dma2d_hw_irq?
A: Inside your ISR, call the function to clear flags and handle the returned status. Example: void DMA2D_IRQHandler(void) { u32 status = ca_dma2d_hw_irq(); if (status & CA_DMA2D_HW_TC) { /* Handle completion */ } if (status & (CA_DMA2D_ERR_TE | CA_DMA2D_ERR_CE | CA_DMA2D_ERR_CAE)) { /* Handle error */ } }

Return conf:1.00 **Q:** What do the return values of ca_dma2d_hw_irq indicate?
A: The function returns a bitmask. CA_DMA2D_HW_TC indicates a successful Transfer Complete. Error flags include CA_DMA2D_ERR_TE (Transfer Error), CA_DMA2D_ERR_CE (Configuration Error), and CA_DMA2D_ERR_CAE (CLUT Access Error). If no relevant flags are set, it returns 0.

HW differences conf:1.00 **Q:** Are there differences in how ca_dma2d_hw_irq behaves across different STM32 families?
A: Yes. On F4xx, F7xx, H7xx, and U5xx families, the function actively interacts with the DMA2D->ISR and DMA2D->IFCR registers to manage hardware interrupts. Conversely, for F1xx and H5xx families (as per the provided driver files), the function is implemented as a stub that returns 0, indicating that DMA2D interrupt handling is either not supported or implemented differently in those specific configurations.

`ca_dma2d_hw_src`

Header: `caos/drivers/inc/ca_glue_hw.h:421`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_dma2d_hw_src(bool bg, const ca_dma2d_src_t *s, u32 w); /*
    FG/BG address, offset, PFC, color; false = flag not on this part */
```

Configures the DMA2D hardware source (Foreground or Background) layer registers.

Parametri:

Nome	Tipo	Descrizione
`bg`	`bool`	If true, configures the Background layer (BG). If false, configures the Foreground layer (FG).
`s`	`const ca_dma2d_src_t *`	Pointer to a structure containing source configuration: memory address, pixel format, alpha, color, and stride.
`w`	`u32`	The width of the image line in pixels, used to calculate the correct memory offset (stride - width).

Ritorna:
Returns true if the configuration was successfully applied to the hardware registers. Returns false if the provided flags are invalid for the current hardware or if the DMA2D peripheral is not supported/initialized.

Note:

- This function is a low-level hardware abstraction layer (HAL) call and is not thread-safe; it directly modifies DMA2D registers. It is non-blocking as it only writes to registers, but it must be called only when the DMA2D peripheral is idle (check with ca_dma2d_busy). It is not intended to be called from an ISR unless you are certain the DMA2D is not currently in use by another task. A common beginner mistake is to call this function while a previous DMA2D transfer is still active, which will corrupt the current operation or cause unpredictable graphical artifacts.

Uso:

- This function is typically used within the internal driver sequence for graphics operations like ca_dma2d_copy or ca_dma2d_blend. The standard workflow is: 1) Ensure the DMA2D is idle using ca_dma2d_busy, 2) Call ca_dma2d_hw_src for the source layer, 3) Configure the destination layer, 4) Trigger the transfer, and 5) Wait for completion using ca_dma2d_wait. It is rarely called directly by application code, as higher-level functions handle the register setup automatically.

Esempio:

```c
/* Example: Configuring a source layer for a copy operation */
ca_dma2d_src_t src_cfg = {0};
src_cfg.addr = (u32)source_buffer;
src_cfg.fmt = 0; /* Assume ARGB8888 */
src_cfg.stride = 800;
src_cfg.alpha = 255;

/* Ensure DMA2D is not busy before touching registers */
if (!ca_dma2d_busy()) {
    /* Configure Foreground layer (bg = false) */
    if (ca_dma2d_hw_src(false, &src_cfg, 800)) {
        /* Proceed to configure destination and start transfer */
        /* ca_dma2d_copy(...) would be called here */
    } else {
        /* Handle configuration error */
    }
} else {
    /* Wait for current operation to finish */
    ca_dma2d_wait();
}
```

Q&A — fine-tuning review *7/7 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_dma2d_hw_src in my graphics driver?
A: It is typically used within internal driver sequences like ca_dma2d_copy or ca_dma2d_blend. The standard workflow is to ensure the DMA2D is idle via ca_dma2d_busy, call ca_dma2d_hw_src for the source layer, configure the destination, trigger the transfer, and wait for completion with ca_dma2d_wait.

Code example conf:1.00 **Q:** Provide a minimal C example for configuring the foreground layer using caOS types.
A: ca_dma2d_src_t src = { .addr = 0x20000000, .fmt = 0, .alpha = 255, .am = 0, .color = 0xFFFFFF, .stride = 320, .flags = 0, .clut_n = 0 }; u32 width = 320; bool success = ca_dma2d_hw_src(false, &src, width);

Parameters conf:1.00 **Q:** What is the purpose of the 'w' parameter in ca_dma2d_hw_src?
A: The 'w' parameter represents the width of the image line in pixels. It is used to calculate the memory offset (stride minus width), which ensures the DMA2D hardware correctly handles the memory layout of the source image.

Return conf:1.00 **Q:** What do the return values of ca_dma2d_hw_src indicate?
A: It returns true if the configuration was successfully applied to the hardware registers. It returns false if the provided flags are invalid for the current hardware or if the DMA2D peripheral is not supported or initialized.

Error/Edge case conf:1.00 **Q:** What is a common mistake when calling ca_dma2d_hw_src?
A: A common beginner mistake is calling this function while a previous DMA2D transfer is still active. This will corrupt the current operation or cause unpredictable graphical artifacts.

Behavior conf:1.00 **Q:** Is ca_dma2d_hw_src thread-safe or ISR-safe?
A: It is not thread-safe as it directly modifies DMA2D registers. It is non-blocking, but

it must only be called when the DMA2D peripheral is idle. It is not intended to be called from an ISR unless you are certain the DMA2D is not currently in use by another task.

HW differences conf:0.90 **Q:** Are there differences in how ca_dma2d_hw_src behaves across STM32 families?

A: Yes. On F1xx, H5xx, and certain configurations of F4xx, F7xx, and H7xx, the function is implemented as a stub that returns false and does nothing. On supported families like F4xx, F7xx, H7xx, and U5xx, it directly writes to registers such as DMA2D->FGMAR, DMA2D->FGOR, and DMA2D->FGPFCCR.

`ca_dma2d_hw_start`

Header: `caos/drivers/inc/ca_glue_hw.h:424`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_dma2d_hw_start(u32 mode, u32 w, u32 h); /* NLR, MODE,
    interrupts, START */
```

Triggers the DMA2D hardware peripheral to begin a memory-to-memory transfer or fill operation.

Parametri:

Nome	Tipo	Descrizione
`mode`	`u32`	The DMA2D transfer mode (e.g., memory-to-memory, memory-to-memory with blending, or color fill).
`w`	`u32`	The width of the area to process in pixels.
`h`	`u32`	The height of the area to process in lines.

Ritorna:
void. This function does not return a status; it assumes the hardware configuration (source, destination, and CLUT) has been successfully set up prior to calling.

Note:

- This function is non-blocking; it writes to the DMA2D registers to start the hardware engine and returns immediately. It is not thread-safe by itself, so it must be protected by a mutex or semaphore if multiple tasks access the DMA2D peripheral. It is generally not intended to be called directly from an ISR unless the system design guarantees exclusive access to the hardware. The most common beginner mistake is calling this function without first configuring the source and destination addresses, which will result in undefined memory access or hardware faults.

Uso:

- This function is the final step in the DMA2D operation sequence. It is typically used after calling ca_dma2d_hw_src and ca_dma2d_hw_dst to define the memory buffers. In a typical application, you would initialize the transfer, call this function to start the hardware, and then immediately call ca_dma2d_wait to block the current task until the hardware signals completion via an interrupt.

Esempio:

```
/* Configure the DMA2D transfer parameters */
ca_dma2d_hw_src(false, &source_config, width);
ca_dma2d_hw_dst(&dest_config, width, color);

/* Start the hardware transfer */
ca_dma2d_hw_start(mode, width, height);
```

```

/* Wait for the hardware to finish the operation */
if (ca_dma2d_wait(1000)) {
    /* Transfer completed successfully within 1000ms */
} else {
    /* Handle timeout or error */
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_dma2d_hw_start in my application flow?
A: ca_dma2d_hw_start is the final step in the DMA2D operation sequence. It should be called only after you have successfully configured the source and destination buffers using functions like ca_dma2d_hw_src and ca_dma2d_hw_dst. After calling this function, you should typically call ca_dma2d_wait to block the task until the hardware completes the transfer.

Code example conf:0.90 **Q:** Provide a minimal example of how to initiate a DMA2D transfer using caOS types.
A: u32 mode = 0; u32 width = 320; u32 height = 240; /* Assuming source and destination are already configured */ ca_dma2d_hw_start(mode, width, height); ca_dma2d_wait();

Parameters conf:1.00 **Q:** What do the 'w' and 'h' parameters represent in ca_dma2d_hw_start?
A: The 'w' parameter represents the width of the area to process in pixels, and 'h' represents the height of the area to process in lines. These values are used to configure the Number of Line Register (NLR) of the DMA2D peripheral.

HW differences conf:1.00 **Q:** Are there differences in how ca_dma2d_hw_start is implemented across different STM32 families?
A: Yes. On F4xx, F7xx, H7xx, and U5xx families, the function actively writes to the DMA2D->NLR and DMA2D->CR registers to trigger the hardware. Conversely, on the F1xx and H5xx families, the provided implementation is a stub that performs no operations, indicating that DMA2D hardware support is either not implemented or not available for those specific targets in the current codebase.

`ca_dma2d_init`

Header: `caos/drivers/inc/dma2d.h:105`
Source: `caos/drivers/src/dma2d.c:113:122`
GUARD: `EN_DMA2D`

```
bool ca_dma2d_init(void); /* false = no DMA2D */
```

Initializes the DMA2D hardware peripheral and its associated internal synchronization semaphore.

Ritorna:
Returns true if the DMA2D hardware was successfully initialized or was already ready. Returns false if the hardware initialization failed (e.g., the MCU does not support DMA2D).

Note:

- This function is not thread-safe and must be called during the system initialization phase before any tasks attempt to use DMA2D services. It is not callable from an ISR because it performs hardware configuration and semaphore creation. A common mistake is calling this function from multiple tasks simultaneously; ensure it is invoked only once, typically within the main configuration routine. If it returns false, any subsequent calls to functions like ca_dma2d_copy or ca_dma2d_fill will fail or trigger error flags.

Uso:

- This function acts as the mandatory gateway for all DMA2D operations. It should be called during the system startup sequence, usually inside ca_os_configure, to verify hardware availability before attempting graphical operations. It pairs with other DMA2D functions such as ca_dma2d_copy, ca_dma2d_fill, or ca_dma2d_blend, which rely on the internal semaphore initialized here to manage access to the hardware. If the function returns false, the application should gracefully disable graphical features to prevent undefined behavior.

Esempio:

```
void setup_graphics(void) {
    /* Initialize DMA2D hardware before using any drawing functions */
    if (!ca_dma2d_init()) {
        /* Handle hardware absence: log error or disable UI */
        ca_printf("DMA2D not supported on this hardware\n");
        return;
    }

    /* Now safe to use DMA2D functions */
    /* Example: ca_dma2d_fill(buffer, width, height, color); */
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_dma2d_init in my caOS application?
A: You must call ca_dma2d_init during the system initialization phase, typically inside ca_os_configure, before any tasks attempt to use DMA2D services. It is not thread-safe and must be invoked only once.

Code example conf:0.95 **Q:** How do I correctly initialize the DMA2D peripheral in my startup routine?
A: You should verify the return value of the initialization function to ensure the hardware is available before proceeding. Example:
void ca_os_configure(void) { bool status = ca_dma2d_init(); if (status == false) { /* Handle hardware absence or failure */ } }

Return conf:1.00 **Q:** What do the return values of ca_dma2d_init indicate, and why might it return false?
A: The function returns true if the DMA2D hardware is successfully initialized or already ready. It returns false if initialization fails, which typically occurs if the specific MCU model does not support the DMA2D peripheral. If it returns false, subsequent calls to DMA2D functions will fail.

`ca_dma2d_wait`

Header: `caos/drivers/inc/dma2d.h:111`
Source: `caos/drivers/src/dma2d.c:168:181`
GUARD: `EN_DMA2D`

```
bool ca_dma2d_wait(u32 timeout_ms); /* the operation started with
    CA_NO_WAIT_TIME */
```

Blocks the calling task until the DMA2D hardware operation completes or the specified timeout expires.

Parametri:

Nome	Tipo	Descrizione
`timeout_ms`	`u32`	Time to wait in milliseconds. Use CA_WAIT_FOREVER to block indefinitely, or a specific value for a timeout. Using 0 (CA_NO_WAIT_TIME) is invalid here as it would not wait for completion.

Ritorna:
Returns true if the DMA2D operation finished successfully (Transfer Complete flag set and no errors). Returns false if the operation timed out, encountered a hardware error, or was aborted.

Note:

- This function is blocking; it puts the calling task into a waiting state until the DMA2D interrupt signals completion or the timeout occurs. It is NOT callable from an ISR because it relies on semaphore waiting mechanisms. It is thread-safe, but you must ensure that a DMA2D operation was actually started (e.g., via ca_dma2d_convert or ca_dma2d_fill) before calling this. A common beginner mistake is calling this function without checking if the hardware is actually busy, or failing to handle the false return value, which leaves the system in an inconsistent state if the hardware failed.

Uso:

- This function is used in a typical 'start-then-wait' pattern. First, you initiate a DMA2D operation (like a blend or copy) using a function that returns immediately. Then, you call ca_dma2d_wait to synchronize the task with the hardware. It is essential for ensuring that memory buffers are fully processed before the CPU attempts to read or modify them again. If the function returns false, you should check the hardware status or perform error recovery, such as re-initializing the DMA2D peripheral.

Esempio:

```c
/* Example: Perform a DMA2D blend operation and wait for completion */
ca_dma2d_src_t src = {0}; /* Assume initialized with valid source data */
ca_dma2d_dst_t dst = {0}; /* Assume initialized with valid destination data */

/* Start the blend operation without waiting immediately */
if (ca_dma2d_blend(&src, &dst, 100, 100, CA_NO_WAIT_TIME)) {
    /* Wait for the hardware to finish, with a 500ms timeout */
    if (ca_dma2d_wait(500U)) {
        /* Success: DMA2D finished, safe to use the destination buffer */
    } else {
        /* Failure: Timeout or hardware error occurred */
        ca_dma2d_abort();
    }
} else {
    /* Failed to start the operation */
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_dma2d_wait in my application code?
A: You should call it after initiating a DMA2D operation (such as ca_dma2d_convert or ca_dma2d_fill) that returns immediately. It is used to synchronize the task with the hardware, ensuring memory buffers are fully processed before the CPU accesses them again.

Code example conf:0.90 **Q:** Provide a minimal example of how to use ca_dma2d_wait.
A: void process_frame(void) { ca_dma2d_fill(0x20000000, 0, 0, 100, 100); u32 timeout = 500; if (!ca_dma2d_wait(timeout)) { /* Handle error */ } }

Parameters conf:1.00 **Q:** What are the valid values for the timeout_ms parameter?
A: You can provide a specific u32 value in milliseconds, or use CA_WAIT_FOREVER to block indefinitely. Using 0 (CA_NO_WAIT_TIME) is invalid because it would not wait for the operation to complete.

Return conf:1.00 **Q:** What does a return value of false indicate?
A: A return value of false indicates that the operation timed out, a hardware error occurred, or the operation was aborted. It signifies that the DMA2D operation did not finish successfully.

Error/Edge case conf:1.00 **Q:** Can I call ca_dma2d_wait from an interrupt service routine?
A: No, you cannot call it from an ISR. The function is blocking and relies on semaphore waiting mechanisms to put the calling task into a waiting state, which is not permitted in an ISR context.

`ca_dma_abort`

Header: `caos/drivers/inc/dma.h:142`
Source: `caos/drivers/src/dma.c:179:184`
GUARD: `EN_DMA`

```
void ca_dma_abort(ca_dma_ch_t ch)
```

Immediately stops a DMA channel operation and cleans up associated synchronization primitives.

Parametri:

Nome	Tipo	Descrizione
`ch`	`ca_dma_ch_t`	The handle of the DMA channel to abort. Must be a valid, previously initialized channel handle.

Ritorna:
void

Note:

- This function is non-blocking and performs an immediate hardware-level stop of the DMA stream. It is critical to call this if a timeout occurs during a DMA transfer, as it prevents the DMA controller from continuing to write to memory buffers that may no longer be valid. It is thread-safe in the context of the caOS DMA driver, but it is not intended to be called from an ISR unless the specific hardware implementation supports it. The most common mistake is failing to call this after a failed ca_dma_wait, which leaves the DMA channel in a 'busy' state, preventing future transfers from starting.

Uso:

- Use this function as an emergency cleanup mechanism when a DMA operation exceeds its expected time limit. It is typically paired with ca_dma_wait; if the wait function returns false (indicating a timeout), you must call ca_dma_abort to reset the channel state. After aborting, you should also perform any necessary peripheral-specific cleanup, such as resetting the SPI or UART hardware state, to ensure the system returns to a known idle condition.

Esempio:

```
/* Example: Aborting a stuck DMA transfer */
ca_dma_ch_t my_ch;
/* Assume my_ch is already initialized and configured */

```

```
ca_dma_copy_inc(my_ch, src_ptr, dst_ptr, 1024, 1);

/* Wait for completion with a 50ms timeout */
if (!ca_dma_wait(my_ch, 50U)) {
    /* Timeout occurred: stop the hardware immediately */
    ca_dma_abort(my_ch);

    /* Perform peripheral-specific cleanup here */
    /* e.g., reset the peripheral registers */
}

/* Always release the channel when finished */
ca_dma_release(my_ch);
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_dma_abort and what happens if I skip it?
A: You should call ca_dma_abort as an emergency cleanup mechanism when a DMA operation exceeds its expected time limit, specifically after a ca_dma_wait call returns false. If you fail to call it, the DMA channel remains in a 'busy' state, which prevents any future transfers from starting.

Code example conf:0.95 **Q:** How do I implement a timeout-safe DMA transfer using ca_dma_abort?
A: You should initiate the transfer, wait for completion, and abort on timeout: ca_dma_ch_t my_ch = get_dma_handle(); if (!ca_dma_wait(my_ch, 1000)) { ca_dma_abort(my_ch); /* Perform peripheral-specific cleanup here */ }

Parameters conf:0.90 **Q:** What are the requirements for the ca_dma_ch_t parameter in ca_dma_abort?
A: The parameter 'ch' must be a valid, previously initialized DMA channel handle. Passing an uninitialized or invalid handle is not supported and may lead to undefined behavior in the DMA controller.

`ca_dma_copy_inc`

Header: `caos/drivers/inc/dma.h:120`
Source: `caos/drivers/src/dma.c:171:177`
GUARD: `EN_DMA`

```
bool ca_dma_copy_inc(ca_dma_ch_t ch, const void *src, void *dst, u32
    size, ca_dma_width_t width)
```

Initiates a DMA memory-to-memory copy with incrementing addresses and waits for completion.

Parametri:

Nome	Tipo	Descrizione
`ch`	`ca_dma_ch_t`	DMA channel identifier. Must be a valid channel number (e.g., 0-7 for STM32). Invalid values may cause undefined behavior or access to wrong DMA hardware.
`src`	`const void *`	Pointer to source memory. Must point to valid memory location. NULL causes undefined behavior.
`dst`	`void *`	Pointer to destination memory. Must point to valid memory location. NULL causes undefined behavior.
`size`	`u32`	Number of data items to copy. Must be > 0. Zero or very large values may cause incorrect DMA transfer or overflow.
`width`	`ca_dma_width_t`	Width of each data item (e.g., 8, 16, or 32 bits). Must be one of the supported values. Invalid values may cause incorrect memory access or hardware errors.

Ritorna:
`true`: DMA transfer completed successfully. `false`: DMA transfer failed or was not started due to invalid parameters or hardware error.

Note:

- This function is NOT blocking in the sense that it does not directly wait for the DMA to finish; instead, it uses a semaphore drain mechanism to ensure previous transfers are done before starting a new one.
- It is NOT thread-safe if multiple tasks use the same DMA channel without proper synchronization.
- It is NOT callable from an ISR because it may block or use RTOS primitives like semaphores.
- Precondition: The DMA channel must be initialized and configured properly before calling this function.
- Common mistake: Not calling ca_dma_wait() after this function to check if the transfer completed successfully.

Uso:

- Used in memory-to-memory DMA transfers where both source and destination addresses increment during transfer.
- Typically used in contexts where data needs to be copied from one buffer to another, such as in DSP or communication protocols.
- Paired with ca_dma_wait() to ensure the transfer is complete before proceeding with verification or next steps.

Esempio:

```
/* Example usage in a memory copy operation */
ca_dma_ch_t ch = 0; /* Use DMA channel 0 */
u32 m2m_src[BUF_WORDS]; /* Source buffer */
u32 m2m_dst[BUF_WORDS]; /* Destination buffer */
/* Fill source buffer */
for (u32 i = 0; i < BUF_WORDS; i++) {
m2m_src[i] = i;
}
/* Ensure cache is flushed before DMA starts */
__DSB();
/* Start DMA transfer */
```

```
bool result = ca_dma_copy_inc(ch, m2m_src, m2m_dst, BUF_WORDS, CA_DMA_WIDTH_32);
if (!result) {
/* Handle error */
ca_stdout_put_n("[ex31] M2M: DMA copy failed");
}
/* Wait for transfer to complete */
if (!ca_dma_wait(ch, 100U)) {
ca_stdout_put_n("[ex31] M2M: no DMA TC within 100 ms");
}
/* Verify results */
u32 errs = 0U;
for (u32 i = 0U; i < BUF_WORDS; i++) {
if (m2m_dst[i] != m2m_src[i]) {
errs++;
}
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_dma_copy_inc() instead of a regular memory copy function?

A: You should use ca_dma_copy_inc() when you need to copy large amounts of data efficiently without blocking the CPU. It leverages the DMA controller to handle the data transfer in the background, freeing up the CPU for other tasks. This is especially useful for transferring data to/from peripherals or memory regions where direct CPU access would be too slow or resource-intensive.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_dma_copy_inc() with caOS types?

A: u8 src[100]; u8 dst[100]; bool result = ca_dma_copy_inc(DMA_CHANNEL_1, src, dst, 100, CA_DMA_WIDTH_8); if (result) { ca_dma_wait(DMA_CHANNEL_1); }

Parameters conf:0.90 **Q:** What are the valid values for the width parameter in ca_dma_copy_inc() and what happens if I pass an invalid value?

A: The valid values for the width parameter are CA_DMA_WIDTH_8, CA_DMA_WIDTH_16, and CA_DMA_WIDTH_32. These specify the size of each data element to be copied. If an invalid value is passed, the behavior is undefined and may cause incorrect data transfers or system errors, as the DMA controller will not be configured properly for the specified width.

Return conf:0.95 **Q:** What does it mean if ca_dma_copy_inc() returns false, and what are the common causes of failure?

A: If ca_dma_copy_inc() returns false, it means the function failed to start the DMA transfer. Common causes include passing an invalid DMA channel, providing NULL pointers for src or dst, specifying a size of zero, or attempting to use a DMA channel that is already occupied or not properly configured. The function does not indicate errors during the actual transfer, only at the start of the operation.

`ca_dma_copy_target`

Header: `caos/drivers/inc/dma.h:128`
Source: `caos/drivers/src/dma.c:179:186`
GUARD: `EN_DMA`

```
bool ca_dma_copy_target(ca_dma_ch_t ch, void *mem, volatile void *periph
    , u32 size, ca_dma_dir_t dir, ca_dma_width_t width)
```

Starts a DMA transfer from a peripheral to memory or vice versa, handling semaphore management and hardware setup.

Parametri:

Nome	Tipo	Descrizione
`ch`	`ca_dma_ch_t`	DMA channel identifier. Must be a valid channel number (e.g., 0-7 for STM32). Invalid values may cause undefined behavior or hardware errors.
`mem`	`void *`	Pointer to memory buffer. Must point to valid, accessible memory. NULL causes undefined behavior.
`periph`	`volatile void *`	Pointer to peripheral register address. Must be a valid peripheral address. NULL may cause bus errors or undefined behavior.
`size`	`u32`	Number of bytes to transfer. Must be > 0. Zero may cause hardware errors or no transfer.
`dir`	`ca_dma_dir_t`	Transfer direction: CA_DMA_DIR_P2M (peripheral to memory) or CA_DMA_DIR_M2P (memory to peripheral). Invalid values may cause incorrect transfers.
`width`	`ca_dma_width_t`	Data width: CA_DMA_WIDTH_8, CA_DMA_WIDTH_16, or CA_DMA_WIDTH_32. Invalid values may cause misalignment or incorrect data transfers.

Ritorna:
`true`: DMA transfer started successfully. The function does not wait for completion; it only initiates the transfer. `false`: DMA transfer failed to start. Common causes: invalid channel, invalid memory/peripheral address, invalid size, or hardware error.

Note:

- {'note': 'This function is NOT blocking. It initiates the transfer and returns immediately. The actual transfer happens in the background.'}
- {'note': 'The function uses a semaphore drain (dma_sem_drain) to ensure previous transfers on the same channel are complete before starting a new one.'}
- {'note': 'This function is NOT thread-safe if multiple threads use the same DMA channel without proper synchronization.'}
- {'note': 'This function is NOT callable from an ISR context because it may block or use semaphores that are not ISR-safe.'}
- {'note': 'Precondition: The DMA channel must be properly initialized and enabled before calling this function.'}
- {'note': 'Common mistake: Not checking return value. If the function returns false, the transfer did not start, and further operations may fail.'}

Uso:

- **SPI driver**: Used to initiate DMA RX/TX transfers for SPI communication. Called after preparing the DMA hardware with ca_spi_hw_dma_prepare().
- **UART driver**: Used to initiate DMA RX transfers for UART reception. Called after setting up the DMA channel and enabling the UART DMA receive mode.

Esempio:

```
// This example shows how to start a DMA RX transfer from a UART peripheral to a
    memory buffer. It checks the return value and enables the UART DMA mode
   after starting the transfer.
/* Example: UART RX using DMA */
```

```
USART_TypeDef *inst = g_uart_instances[uart_num];
uint8_t buffer[256];

/* Start DMA transfer from UART peripheral to memory buffer */
if (!ca_dma_copy_target(s_udma[i].rx_ch, buffer,
                        ca_uart_hw_dma_rx_addr(inst),
                        256, CA_DMA_DIR_P2M, CA_DMA_WIDTH_8)) {
    /* Handle error: DMA failed to start */
    return false;
}

/* Enable UART DMA receive mode */
ca_uart_hw_dma_rx_on(inst); /* RXNEIE off, CR3.DMAR on */
return true;
// This example shows how to start a DMA RX transfer from an SPI peripheral to a memory buffer. It checks the return value and cleans up on failure.
/* Example: SPI RX using DMA */
uint8_t rx_buffer[128];

/* Prepare DMA hardware for SPI */
ca_spi_hw_dma_prepare(spi_num, 128, true); /* Enable RX DMA */

/* Start DMA transfer from SPI peripheral to memory */
if (!ca_dma_copy_target(s_sdma[i].rx_ch, rx_buffer,
                        ca_spi_hw_dma_rx_addr(spi_num),
                        128, CA_DMA_DIR_P2M, CA_DMA_WIDTH_8)) {
    /* Handle error: DMA failed to start */
    ca_spi_hw_dma_finish(spi_num, false);
    return 0;
}
return 1;
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_dma_copy_target in a caOS project, and what are the benefits of using DMA over regular memory access?
A: You should use ca_dma_copy_target when you need to transfer data between peripheral registers and RAM without CPU intervention, especially for high-volume or time-critical data transfers. DMA allows the CPU to perform other tasks while the data transfer is handled by the DMA controller, improving system efficiency. It is particularly useful for UART, SPI, or ADC data acquisition where large amounts of data need to be moved quickly.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_dma_copy_target to copy data from an SPI peripheral to RAM?
A: ca_dma_ch_t ch = DMA_CH1; volatile u16 *spi_data = (volatile u16 *)0x40013000; // SPI1_DR u16 buffer[100]; bool result = ca_dma_copy_target(ch, buffer, (volatile void *)spi_data, 200, DMA_FROM_MEM, DMA_WIDTH_16BIT); if (!result) { // Handle error }

Parameters conf:0.90 **Q:** What are the valid values for the ca_dma_dir_t parameter, and what happens if I pass an invalid value?
A: The valid values for ca_dma_dir_t are DMA_TO_MEM and DMA_FROM_MEM. If an invalid value is passed, the copy operation will not function as expected, and the function may return false. The direction determines whether data is copied from the peripheral to memory (DMA_FROM_MEM) or from memory to the peripheral (DMA_TO_MEM).

Return conf:0.95 **Q:** What does it mean when ca_dma_copy_target returns false, and what are the most common causes of this failure?
A: When ca_dma_copy_target returns false, it indicates that the DMA transfer could

not be initiated. Common causes include passing an invalid DMA channel, providing a zero size, using a non-accessible or misaligned buffer, or attempting to use an uninitialized DMA channel. The function does not check for transfer completion, only for successful initiation of the operation.

`ca_dma_free_channel`

Header: `caos/drivers/inc/dma.h:79`
Source: `caos/drivers/src/dma.c:68:89`
GUARD: `EN_DMA`

```
i16 ca_dma_free_channel(ca_dma_ch_t *out)
```

Searches for and reserves the first available DMA channel, returning its handle.

Parametri:

Nome	Tipo	Descrizione
`out`	`ca_dma_ch_t *`	A pointer to a DMA channel structure where the function will store the hardware-specific handle (controller and stream) if a channel is successfully found.

Ritorna:
Returns the index (0 to N) of the reserved channel if successful, or -1 if no free DMA channels are currently available.

Note:

- This function is thread-safe because it uses a spinlock to protect the internal bitmask tracking channel usage. It is non-blocking in the sense that it does not wait for a channel to become free; if none are available, it returns -1 immediately. It is safe to call from an ISR because it uses a spinlock that disables interrupts, preventing deadlocks. A common beginner mistake is failing to check the return value for -1; if you proceed to use an uninitialized or invalid channel handle, the system will likely trigger a hard fault or cause silent data corruption in your peripheral transfers.

Uso:

- This function is the standard entry point for any driver or application task requiring DMA transfers. It is typically called during the initialization phase of a peripheral (like UART, SPI, or memory-to-memory copy) to claim a resource before configuring the transfer parameters. It is almost always paired with a later call to a release function (not shown here) to ensure the channel is returned to the pool once the transfer is complete. In complex drivers, it is often used in conjunction with index-specific allocation functions to handle cases where a peripheral is hard-wired to a specific DMA stream.

Esempio:

```
ca_dma_ch_t my_dma_ch;
i16 channel_idx = ca_dma_free_channel(&my_dma_ch);

if (channel_idx < 0) {
    // Handle error: no DMA channels available
    return;
}

// Successfully reserved, now configure the request
// ca_dma_set_request(my_dma_ch, MY_DMA_REQUEST_ID);
```

```

// Proceed with DMA transfer setup...
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_dma_free_channel, and what is its purpose in a DMA transfer setup?
A: You should call ca_dma_free_channel when you need to allocate a DMA channel for a new transfer. It is used to acquire a free DMA channel that can then be configured and started for data movement. This function ensures that a channel is available before proceeding with DMA operations.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_dma_free_channel in C with caOS types?
A: ca_dma_ch_t ch; i16 channel_idx = ca_dma_free_channel(&ch); if (channel_idx >= 0) { /* Channel successfully allocated */ } else { /* No channels available */ }

Parameters conf:0.90 **Q:** What does the 'out' parameter in ca_dma_free_channel do, and what happens if I pass NULL?
A: The 'out' parameter is a pointer to a structure that will hold information about the allocated DMA channel. If NULL is passed, the function still allocates a channel but does not populate the structure, meaning you lose the reference to the channel details, though the allocation still succeeds.

Return conf:0.95 **Q:** What do the return values of ca_dma_free_channel indicate, and what causes it to return -1?
A: A return value >= 0 indicates the index of a successfully allocated DMA channel. A return value of -1 means no DMA channels are available. The function fails and returns -1 if all channels are already in use and no free channel can be acquired.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_dma_free_channel?
A: Common mistakes include not checking the return value of ca_dma_free_channel, which can lead to logical errors if no channel is actually allocated. Passing NULL is valid but results in losing the channel reference. Also, calling this function without prior DMA initialization may lead to undefined behavior.

Behavior conf:0.90 **Q:** Is ca_dma_free_channel blocking or non-blocking, and can it be safely called from an ISR?
A: ca_dma_free_channel is a blocking function. It waits until a DMA channel becomes available. It is not safe to call from an ISR because it uses a spinlock that could cause deadlock or unpredictable behavior in interrupt contexts.

`ca_dma_free_channel_index`

Header: `caos/drivers/inc/dma.h:84`
Source: `caos/drivers/src/dma.c:91:112`
GUARD: `EN_DMA`

```
i16 ca_dma_free_channel_index(i16 index, ca_dma_ch_t *out)
```

Attempts to reserve a specific DMA channel by its index if it is currently available.

Parametri:

Nome	Tipo	Descrizione
index	i16	The specific hardware DMA channel index to request. Must be non-negative and less than the total available channels and DMA_MAX_CHANNELS.
out	ca_dma_ch_t *	Pointer to a structure where the function will store the internal DMA channel handle if the reservation succeeds.

Ritorna:
Returns the index of the reserved channel on success (>= 0). Returns -1 if the index is out of bounds, if the channel is already in use, or if the operation fails.

Note:

- This function is thread-safe because it uses a spinlock to protect the internal bitmask representing channel availability. It is non-blocking, meaning it will return immediately after checking the status, making it safe to call from most contexts, though it is primarily intended for driver initialization. A common mistake is assuming the channel is yours without checking the return value; if the function returns -1, the 'out' pointer is not updated, and attempting to use an uninitialized handle will lead to undefined behavior or system crashes. Ensure the DMA hardware is initialized before calling this, as the function relies on hardware-specific channel counts.

Uso:

- This function is typically used during peripheral driver initialization (like SPI or UART) when a specific DMA channel is hard-coded or preferred by the hardware configuration. It is usually paired with 'ca_dma_hw_dma_bind' to determine which channel index is required, followed by 'ca_dma_set_request' to configure the channel once successfully reserved. If the specific index is not available, developers often fall back to 'ca_dma_free_channel' to request any available channel instead.

Esempio:

```
ca_dma_ch_t my_dma_ch;
i16 requested_idx = 2; // We want to use DMA channel 2

// Attempt to reserve the specific channel
i16 result = ca_dma_free_channel_index(requested_idx, &my_dma_ch);

if (result >= 0) {
    // Success: channel is reserved and handle is populated
    ca_dma_set_request(my_dma_ch, DMA_REQUEST_UART_RX);
} else {
    // Failure: channel 2 is busy or invalid, handle error appropriately
    handle_dma_error();
}
```

Q&A — fine-tuning review

6/6 pairs · qwen3-coder:latest

Usage conf:0.95 **Q:** When should I call ca_dma_free_channel_index and why?
A: You should call ca_dma_free_channel_index when you want to release a previously allocated DMA channel so that it can be reused by other parts of your application. This is typically done after completing a DMA transfer or when you no longer need the channel for its current purpose.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_dma_free_channel_index with caOS types?

A: i16 channel_index = 2; ca_dma_ch_t channel_info; i16 result = ca_dma_free_channel_index(chan &channel_info); if (result >= 0) { // Channel successfully freed } else { // Error occurred }

Parameters conf:0.95 **Q:** What does the 'index' parameter represent and what are valid values?
A: The 'index' parameter is the index of the DMA channel to be freed. It must be a non-negative integer less than the total number of available DMA channels. Values less than 0 or greater than or equal to the total number of channels are invalid and will cause the function to return -1.

Return conf:0.95 **Q:** What do the return values of ca_dma_free_channel_index mean?
A: A return value >= 0 indicates that the channel was successfully freed and the returned value is the same index that was passed as input. A return value of -1 indicates an error, such as an invalid channel index or a failure to free the channel (e.g., it was already free or out of bounds).

Error/Edge case conf:0.90 **Q:** What are common mistakes or edge cases when using ca_dma_free_channel_inc
A: Common mistakes include passing an invalid index (negative or too large), calling the function on a channel that is already free, or not checking the return value. Not checking the return value can lead to using a channel that was not actually freed, which may cause undefined behavior.

Behavior conf:0.90 **Q:** Is ca_dma_free_channel_index blocking, thread-safe, and ISR-safe?
A: The function is non-blocking and returns immediately. It is thread-safe, using a spinlock to ensure safety in multi-threaded environments. It can be called from an ISR, but it's not recommended in high-latency contexts due to the use of spinlocks.

`ca_dma_free_channel_m2m`

Header: `caos/drivers/inc/dma.h:96`
Source: `caos/drivers/src/dma.c:91:113`
GUARD: `EN_DMA`

```
1 i16 ca_dma_free_channel_m2m(ca_dma_ch_t *out)
```

Allocates a DMA channel specifically capable of memory-to-memory (M2M) transfers.

Parametri:

Nome	Tipo	Descrizione
`out`	`ca_dma_ch_t *`	Pointer to a structure where the allocated channel's controller and stream identifiers will be stored. Must not be NULL.

Ritorna:
Returns the flat index (0 to N) of the allocated channel on success, or -1 if no M2M-capable channel is available.

Note:

- This function is thread-safe as it uses a spinlock to protect the internal DMA channel allocation bitmask. It is NOT callable from an ISR because it uses locking mechanisms that may block or cause deadlocks. The function is non-blocking in terms of waiting for hardware, but it performs a search through available channels. A common mistake is assuming all DMA channels support M2M; on many STM32 devices (like F4/F7), only specific DMA controllers (e.g., DMA2) support M2M, and this function correctly filters for those specific hardware capabilities. Always check if the returned index is >= 0 before using the channel handle.

Uso:

- Use this function when you need to perform high-speed memory copies or peripheral-to-memory transfers that require M2M hardware support. It is typically the first step in a DMA workflow: call this to get a channel, then configure the transfer using functions like ca_dma_set_request or ca_dma_copy_inc, and finally trigger the transfer. Once the operation is complete, you must release the channel using ca_dma_release to make it available for other tasks in the system.

Esempio:

```
ca_dma_ch_t dma_ch;
i16 ch_idx = ca_dma_free_channel_m2m(&dma_ch);

if (ch_idx >= 0) {
    /* Configure the DMA transfer for memory-to-memory */
    /* ca_dma_set_request(ch_idx, ...); */

    /* Perform the copy operation */
    ca_dma_copy_inc(dma_ch, src_buffer, dst_buffer, size);

    /* Wait for the hardware to finish the transfer */
    ca_dma_wait(dma_ch);

    /* Always release the channel back to the OS */
    ca_dma_release(dma_ch);
} else {
    ca_printf_n("Error: No M2M DMA channel available");
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_dma_free_channel_m2m instead of a standard DMA allocation?
A: You should use this function when you specifically require memory-to-memory (M2M) transfer capabilities, such as for high-speed memory copies. It is the first step in a DMA workflow before configuring the transfer with functions like ca_dma_set_request or ca_dma_copy_inc.

Code example conf:1.00 **Q:** How do I correctly call ca_dma_free_channel_m2m in my code?
A: You must provide a pointer to a ca_dma_ch_t structure. Example: ca_dma_ch_t my_ch; i16 idx = ca_dma_free_channel_m2m(&my_ch); if (idx >= 0) { /* proceed with transfer */ }

Parameters conf:1.00 **Q:** What are the requirements for the 'out' parameter in ca_dma_free_channel_m2m?
A: The 'out' parameter must be a pointer to a ca_dma_ch_t structure where the controller and stream identifiers will be stored. It must not be NULL.

Return conf:1.00 **Q:** What do the return values of ca_dma_free_channel_m2m indicate?
A: The function returns the flat index (0 to N) of the allocated channel on success. It returns -1 if no M2M-capable channel is available.

Error/Edge case conf:1.00 **Q:** Why might ca_dma_free_channel_m2m fail to return a valid channel?
A: Failure occurs if no M2M-capable channel is available. A common mistake is assuming all DMA channels support M2M; this function correctly filters for hardware capabilities, as only specific controllers (like DMA2 on STM32 F4/F7) support M2M.

Behavior conf:1.00 **Q:** Is ca_dma_free_channel_m2m safe to call from an interrupt service routine (ISR)?
A: No, it is not callable from an ISR because it uses locking mechanisms that may block or cause deadlocks. However, it is thread-safe for use in tasks as it uses a spinlock to protect the internal allocation bitmask.

ca_dma_get_channel_count

Header: caos/drivers/inc/dma.h:74
Source: caos/drivers/src/dma.c:63:66
GUARD: EN_DMA

```
u8 ca_dma_get_channel_count(void)
```

Returns the total number of DMA channels available on the current STM32 hardware platform.

Ritorna:
Returns an unsigned 8-bit integer (u8) representing the total count of physical DMA channels configured in the hardware abstraction layer. This value is always positive and reflects the hardware capability defined at compile-time or initialization.

Note:

- This function is non-blocking and thread-safe as it performs a simple read operation of a hardware-defined constant or register value. It is safe to call from both main-line code and Interrupt Service Routines (ISRs) because it does not involve complex synchronization primitives like mutexes or semaphores. The most common mistake is assuming this value changes dynamically; it is a static hardware property, so do not expect it to reflect the number of currently 'free' or 'available' channels. Always ensure the DMA subsystem has been initialized before relying on this value, although in most implementations, this function simply returns a constant defined by the hardware configuration.

Uso:

- This function is typically used during the initialization phase of a driver or application to determine the bounds for loops or array allocations related to DMA management. It is frequently paired with functions like ca_dma_free_channel or other channel-management routines to iterate through all available hardware resources. Developers use it to perform boundary checks, ensuring that any index-based access to DMA channels does not exceed the physical limits of the microcontroller. It acts as a safety mechanism to make code portable across different STM32 variants with varying DMA channel counts.

Esempio:

```
/* Example: Iterating through all DMA channels to find a free one */
#include "caos.h"

void find_free_dma_channel(void) {
    u8 total_channels = ca_dma_get_channel_count();

    for (u8 i = 0; i < total_channels; i++) {
        /* Attempt to free/acquire a channel by its index */
        /* ca_dma_free_channel_index returns 0 on success */
        if (ca_dma_free_channel_index(i) == 0) {
            /* Channel i is now acquired for use */
            break;
        }
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_dma_get_channel_count() and why is it important for DMA management?
A: You should call ca_dma_get_channel_count() during system initialization or when dynamically allocating DMA channels to determine how many channels are available

on the hardware. This function is important because it allows safe allocation of DMA resources by ensuring you do not exceed the number of available channels, preventing resource conflicts and system instability. It can be safely called from ISRs and is thread-safe.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_dma_get_channel_count() with caOS types?
A: u8 channel_count = ca_dma_get_channel_count(); if (channel_count > 0) { // Proceed with DMA channel allocation // Example: allocate first channel u8 channel = 0; // Use channel for DMA configuration } else { // Handle hardware initialization failure }

Return conf:0.90 **Q:** What do the return values of ca_dma_get_channel_count() mean and what causes a return value of 0?
A: The function returns a u8 value representing the total number of DMA channels available on the hardware. A return value of 0 indicates that either the hardware has not been initialized correctly or the hardware is not supported by the caOS implementation. This function accesses a hardware register and does not modify system state, so it is safe to call even before full system initialization.

`ca_dma_get_remaining`

Header: `caos/drivers/inc/dma.h:142`
Source: `caos/drivers/src/dma.c:199:204`
GUARD: `EN_DMA`

```
u32 ca_dma_get_remaining(ca_dma_ch_t ch)
```

Returns the number of data units remaining to be transferred by a specific DMA channel.

Parametri:

Nome	Tipo	Descrizione
`ch`	`ca_dma_ch_t`	The handle or identifier of the DMA channel. Must be a valid channel initialized by the system; passing an invalid or uninitialized channel will result in a return value of 0.

Ritorna:
Returns a u32 representing the count of items (bytes, half-words, or words) left in the current DMA transfer. Returns 0 if the channel is invalid (i.e., flat index >= DMA_MAX_CHANNELS), if the transfer is complete, or if the DMA channel is not currently active.

Note:

- This function is non-blocking and thread-safe as it performs a direct hardware register read. It is safe to call from an ISR because it does not attempt to acquire any semaphores or perform context switches. A common mistake is assuming that a return value of 0 means the transfer was successful; it only means no data is left, which could also happen if the transfer was aborted or never started. The function internally converts the channel handle to a flat index and validates it before calling the hardware-specific function, so invalid handles will return 0 without causing a crash.

Uso:

- This function is typically used in communication drivers (like UART or SPI) to poll the progress of a background data transfer. It is often called after a DMA transfer has been initiated to check if the buffer is ready to be processed or to calculate how much data has

been received so far. It is frequently paired with functions that abort or stop the DMA to safely clean up resources once the required data has been processed. It is also used in UART driver functions like ca_uart_dma_remaining() to check how much data is left in the RX buffer.

Esempio:

```c
/* Check how many bytes are left in the UART RX DMA buffer */
u32 remaining = ca_dma_get_remaining(uart_rx_channel);

if (remaining == 0) {
    /* Transfer is complete, safe to process the buffer */
    process_data(buffer);
} else {
    /* Transfer is still in progress, wait or perform other tasks */
    do_other_work();
}

/* Example usage in UART driver */
u32 uart_remaining = ca_uart_dma_remaining(uart_num);
if (uart_remaining == 0) {
    /* DMA transfer complete, process received data */
    handle_uart_data(uart_num);
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_dma_get_remaining in my driver logic?
A: You should use it to poll the progress of background DMA transfers, such as in UART or SPI drivers. It is typically called after initiating a transfer to monitor how much data remains, allowing you to determine when a buffer is ready for processing or to calculate received data volume.

Code example conf:0.90 **Q:** How do I implement a simple check for a DMA channel using caOS types?
A: You can check the remaining count like this: u32 remaining = ca_dma_get_remaining(my_dma_ch) if (remaining == 0) { /* Handle completion */ }

Parameters conf:1.00 **Q:** What happens if I pass an invalid or uninitialized ca_dma_ch_t to the function?
A: The function will return 0. The parameter must be a valid channel identifier that has been properly initialized by the system.

Return conf:1.00 **Q:** Does a return value of 0 guarantee that my DMA transfer finished successfully?
A: No. A return value of 0 only indicates that no data is left to be transferred. This can occur if the transfer completed, if the transfer was aborted, or if the channel was never started. You should not assume success based solely on this value.

`ca_dma_hw_abort`

Header: `caos/drivers/inc/ca_glue_hw.h:115`
GUARD: `EN_CA_GLUE_HW`

```c
extern void ca_dma_hw_abort(u8 flat); /* stops a running transfer, no TC, flags cleared, request kept */
```

Immediately halts a specific DMA stream or channel without triggering completion interrupts.

Parametri:

Nome	Tipo	Descrizione
`flat`	`u8`	The flattened index of the DMA stream or channel. Must be a valid index within the hardware-specific range (e.g., 0 to H7_TOTAL-1). Passing an out-of-bounds value will result in a silent no-op.

Ritorna:
void

Note:

- This function is a low-level hardware abstraction that performs an immediate, non-blocking stop of the DMA controller. It does not wait for the current transfer to finish, nor does it trigger any Transfer Complete (TC) callbacks. It is thread-safe in the sense that it interacts directly with hardware registers, but it should be used with extreme caution as it leaves the DMA peripheral in an indeterminate state. It is not intended to be called from an ISR unless the system design explicitly requires emergency termination of hardware operations. The most common beginner mistake is calling this function without subsequently cleaning up the associated software state, such as semaphores or buffer pointers, which can lead to race conditions or memory corruption.

Uso:

- This function is typically used as the final step in an error recovery sequence or when a task needs to cancel a pending data transfer due to a timeout or system shutdown. It is almost always paired with higher-level functions like ca_dma_abort, which handles the necessary software synchronization (like draining semaphores) before calling this hardware-level abort. Developers should use this when they need to force-stop a peripheral (like an SPI or UART DMA transfer) immediately, ensuring that no further data is written to or read from memory buffers.

Esempio:

```
/* Example: Aborting a DMA transfer due to a timeout */
void handle_dma_timeout(ca_dma_ch_t ch) {
    /* Convert the channel handle to the internal flat index */
    u8 flat = ca_dma_hw_ch_to_flat(ch);

    /* Stop the hardware immediately */
    ca_dma_hw_abort(flat);

    /*
     * After the hardware is stopped, we must ensure the software
     * state is consistent. We drain any pending semaphores that
     * might have been waiting for a completion that will never arrive.
     */
    dma_sem_drain(flat);

    /* Log the event or notify the system that the transfer was cancelled */
    ca_printf("DMA channel %d aborted due to timeout\n", flat);
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_dma_hw_abort instead of a higher-level DMA function?
A: ca_dma_hw_abort is a low-level hardware abstraction used for immediate, non-blocking termination of a DMA transfer, typically during error recovery or emergency

system shutdown. It should be used when you need to force-stop a peripheral immediately to prevent further memory access. It is recommended to pair this with higher-level functions like ca_dma_abort to handle software synchronization, as calling it alone can lead to race conditions or memory corruption.

Code example conf:1.00 **Q:** How do I call ca_dma_hw_abort in my code?
A: You pass the flattened index of the DMA stream or channel to the function. Example: u8 stream_idx = 5; ca_dma_hw_abort(stream_idx);

Parameters conf:1.00 **Q:** What happens if I pass an invalid index to the 'flat' parameter?
A: The 'flat' parameter represents the flattened index of the DMA stream or channel. If you pass an out-of-bounds value (e.g., greater than or equal to the total streams/channels for your specific MCU family), the function will perform a silent no-op.

HW differences conf:1.00 **Q:** How does the implementation of ca_dma_hw_abort differ across STM32 families?
A: The implementation varies based on the hardware architecture: for F1xx, H5xx, and U5xx, it calls chan_stop. For F4xx and F7xx, it calls stream_stop after validating the index against total streams. The H7xx implementation is the most complex, as it checks if the index falls within the DMA1/DMA2 stream range or the BDMA range, calling either stream_stop or bdma_stop accordingly.

`ca_dma_hw_ch_to_flat`

Header: `caos/drivers/inc/ca_glue_hw.h:97`
GUARD: `EN_CA_GLUE_HW`

```
extern u8 ca_dma_hw_ch_to_flat(ca_dma_ch_t ch); /* converte handle (
    controller+canale) in indice flat */
```

Converts a structured DMA channel handle into a unique, zero-based flat index for array indexing.

Parametri:

Nome	Tipo	Descrizione
`ch`	`ca_dma_ch_t`	A structure containing the controller ID and the specific stream or channel number. Must be a valid initialized handle.

Ritorna:
Returns a u8 value representing the unique flat index (0 to N-1). Returns 0xFFU if the provided controller or stream/channel is invalid for the current hardware architecture.

Note:

- This function is non-blocking, thread-safe, and safe to call from an ISR because it performs simple arithmetic without accessing hardware registers or OS primitives. It is a helper function used to map complex hardware-specific channel identifiers into a simple linear array index for internal driver management. The most common mistake is failing to check the return value for 0xFFU before using it as an array index, which can lead to out-of-bounds memory access or system crashes if an invalid channel handle is passed.

Uso:

- This function is typically used by the DMA driver core to translate user-provided channel handles into internal lookup table indices. It is essential when managing global arrays like callback tables or interrupt configuration states. You will see it used in functions like ca_dma_release to ensure that operations are performed on the correct internal data

structures associated with the physical DMA hardware. It acts as the bridge between the user-facing handle and the internal flat-indexed driver state.

Esempio:

```
ca_dma_ch_t my_dma_ch = { .controller = 1, .stream = 2 };
u8 flat_idx = ca_dma_hw_ch_to_flat(my_dma_ch);

if (flat_idx != 0xFFU) {
    /* Safe to use flat_idx to access internal driver arrays */
    /* Call internal function to disable interrupts for this specific flat index */
    /* ca_dma_hw_irq_disable(flat_idx); */
    ca_dma_release(my_dma_ch);
} else {
    /* Handle error: invalid channel provided */
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_dma_hw_ch_to_flat() in a caOS project?
A: You should use ca_dma_hw_ch_to_flat() when you need to convert a DMA channel handle (controller + stream) into a flat index for uniform access to hardware registers or resource management. This is particularly useful when working with DMA controllers that have different numbers of channels per controller, and you want to treat all channels as a single linear array.

Code example conf:0.95 **Q:** Can you show a minimal C example using ca_dma_hw_ch_to_flat() with caOS types?
A:

```
ca_dma_ch_t ch = { .controller = 1, .stream = 3 };
 u8 flat_index = ca_dma_hw_ch_to_flat(ch);
 if (flat_index != 0xFF) {
     // Use flat_index to access hardware registers
 }
```

Parameters conf:0.95 **Q:** What are the valid values for the 'controller' and 'stream' fields in ca_dma_ch_t?
A: The 'controller' field must be 1, 2, or 3. The 'stream' field must be a valid index within the range of channels for that controller. For example, on F1xx, controller 1 can have streams 0 to F1_DMA1_CHANNELS-1, and controller 2 can have streams 0 to F1_DMA2_CHANNELS-1. Invalid values cause the function to return 0xFF.

Return conf:0.95 **Q:** What does a return value of 0xFF from ca_dma_hw_ch_to_flat() indicate?
A: A return value of 0xFF indicates that the input handle (controller or stream) was invalid, meaning either the controller number was not 1, 2, or 3, or the stream index was out of range for the specified controller. This typically results from incorrect parameter values passed to the function.

HW differences conf:0.90 **Q:** How does ca_dma_hw_ch_to_flat() differ between F1xx and H7xx MCUs in terms of channel handling?
A: On F1xx MCUs, DMA has two controllers (1 and 2) with potentially different numbers of channels per controller, and controller 2 is optional. On H7xx MCUs, there are three controllers (1, 2, and 3), with the third controller (BDMA) supporting a different number of channels. The function maps these differently: F1xx uses a flat index within controller limits, while H7xx includes BDMA channels in the flat index calculation.

`ca_dma_hw_copy_inc`

Header: `caos/drivers/inc/ca_glue_hw.h:100`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_dma_hw_copy_inc(u8 flat, const void *src, void *dst, u32
    size, ca_dma_width_t width); /* trasferimento DMA memoria memoria
    con incremento puntatori */
```

Initiates a non-blocking memory-to-memory data transfer using the DMA controller.

Parametri:

Nome	Tipo	Descrizione
`flat`	`u8`	The flattened index of the DMA stream or channel. Must be a valid index mapped to the hardware; invalid indices will cause undefined behavior or system crashes.
`src`	`const void *`	Source memory address. Must be accessible by the DMA controller (e.g., not in TCM or CCM on specific STM32 families).
`dst`	`void *`	Destination memory address. Must be accessible by the DMA controller and writable.
`size`	`u32`	Number of data units to transfer. The unit size is determined by the width parameter.
`width`	`ca_dma_width_t`	Data width (e.g., byte, half-word, word). Must match the alignment of the source and destination buffers.

Ritorna:
Always returns true in the current implementation, as it triggers the hardware peripheral and returns immediately.

Note:

- This function is non-blocking; it configures the DMA registers and returns control to the CPU immediately while the hardware performs the copy in the background. It is not inherently thread-safe if multiple tasks attempt to use the same DMA stream simultaneously, so you must ensure exclusive access via a mutex or semaphore. It is generally safe to call from an ISR, but you must ensure the DMA stream is not already busy. A common beginner mistake is to assume the data is copied by the time the function returns; you must wait for the Transfer Complete (TC) interrupt or flag before accessing the destination buffer.

Uso:

- This function is used for high-speed bulk data movement between memory regions without consuming CPU cycles. It is typically paired with a DMA interrupt handler that clears the completion flag and signals the application that the data is ready. In a typical workflow, you would verify the DMA channel is idle, call this function to start the transfer, and then put the calling task into a blocked state (e.g., waiting on a semaphore) until the DMA interrupt notifies you that the operation is finished.

Esempio:

```
/* Example: Copying a buffer using DMA */
#define DMA_CH_1 0

void transfer_data(const uint8_t *source, uint8_t *destination, uint32_t len) {
    /* Ensure the DMA channel is available before starting */
    if (ca_dma_hw_copy_inc(DMA_CH_1, source, destination, len, CA_DMA_WIDTH_8))
    {
        /* The transfer is now running in the background */
        /* Wait for the DMA interrupt to signal completion */
        // wait_for_dma_completion(DMA_CH_1);
```

```
    } else {
        /* Handle error: DMA channel busy or invalid */
    }
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_dma_hw_copy_inc() instead of a regular memory copy function like memcpy()?
A: You should use ca_dma_hw_copy_inc() when you need to copy data between memory regions efficiently without involving the CPU. This function is ideal for large data transfers where you want to offload the work to the DMA controller, freeing up the CPU for other tasks. It's particularly useful in real-time systems where CPU cycles are at a premium.

Code example conf:0.90 **Q:** Can you provide a minimal example of how to use ca_dma_hw_copy_inc() with caOS types?
A: u8 channel = 0; const u8 src[100] = {0}; u8 dst[100] = {0}; bool success = ca_dma_hw_copy_inc(cha src, dst, 100, 8);

Parameters conf:0.95 **Q:** What does the 'width' parameter in ca_dma_hw_copy_inc() control, and what values are valid?
A: The 'width' parameter specifies the data width of each element being transferred. Valid values are 8, 16, or 32 bits. These values determine how many bits are transferred per cycle. Invalid values may cause alignment errors or corrupted data transfers.

Return conf:0.85 **Q:** What does it mean if ca_dma_hw_copy_inc() returns false?
A: The function returns true if the DMA transfer is successfully initiated. It does not wait for the transfer to complete. If the function returns false, it indicates that the transfer could not be started, likely due to invalid parameters or hardware issues. However, based on the provided documentation, the function always returns true upon successful initiation.

Error/Edge case conf:0.90 **Q:** What are common mistakes or edge cases when using ca_dma_hw_copy_inc()?
A: Common mistakes include passing NULL pointers for src or dst, using invalid channel numbers (outside 0-7), or specifying a size of zero. Additionally, using invalid width values (not 8, 16, or 32) can lead to alignment issues. On some families, passing pointers in CCM/TCM regions may be rejected by safeguards.

Behavior conf:0.90 **Q:** Is ca_dma_hw_copy_inc() blocking or non-blocking, and is it safe to call from an ISR?
A: The function is non-blocking; it initiates the DMA transfer and returns immediately. It is safe to call from an ISR, as it does not perform any blocking operations. However, ensure that the DMA channel is not already in use by another transfer to avoid conflicts.

HW differences conf:0.85 **Q:** How does ca_dma_hw_copy_inc() differ between F1xx and H7xx families in terms of hardware implementation?
A: On F1xx, the function uses DMA_Channel_TypeDef with CCR, CPAR, CMAR, and CNDTR registers. On H7xx, it supports both DMA1/DMA2 streams and BDMA channels. For DMA1/DMA2, it uses DMA_Stream_TypeDef with PAR, M0AR, and NDTR registers, similar to F4xx/F7xx. For BDMA, it uses BDMA_Channel_TypeDef with different register layouts and requires SRAM4/backup region checks.

`ca_dma_hw_copy_target`

Header: `caos/drivers/inc/ca_glue_hw.h:102`
GUARD: `EN_CA_GLUE_HW`

```c
extern bool ca_dma_hw_copy_target(u8 flat, void *mem, volatile void *periph, u32 size, ca_dma_dir_t dir, ca_dma_width_t width); /* trasferimento DMA tra memoria e periferico */
```

Configures and triggers a DMA hardware transfer between a memory buffer and a peripheral register.

Parametri:

Nome	Tipo	Descrizione
`flat`	`u8`	The flattened index of the DMA stream or channel. Must be a valid index returned by the system's DMA mapping logic.
`mem`	`void *`	Pointer to the memory buffer. Must not reside in restricted memory regions like TCM or CCM, depending on the specific STM32 architecture.
`periph`	`volatile void *`	The peripheral register address (e.g., UART Data Register). Must be volatile to prevent compiler optimization.
`size`	`u32`	Number of data units to transfer. The unit size is determined by the width parameter.
`dir`	`ca_dma_dir_t`	Direction of transfer: CA_DMA_DIR_M2P (Memory to Peripheral) or CA_DMA_DIR_P2M (Peripheral to Memory).
`width`	`ca_dma_width_t`	Data width for the transfer (e.g., 8-bit, 16-bit, or 32-bit).

Ritorna:
Always returns true. Note that this function initiates the hardware transfer but does not wait for completion.

Note:

- This function is non-blocking; it configures the DMA registers and returns immediately while the hardware performs the transfer in the background. It is generally thread-safe if the DMA channel is managed by a semaphore, but calling it from an ISR requires extreme caution regarding channel availability. A common beginner mistake is passing a memory address that the DMA controller cannot access (like TCM or CCM on certain STM32 chips), which will trigger a hardware fault or silent failure. Always ensure the DMA channel is not currently busy before calling this, typically by checking a semaphore or status flag.

Uso:

- This function is the low-level engine for DMA operations. It is typically called by higher-level drivers after they have acquired a DMA channel via a semaphore. The standard sequence is: 1) Acquire the DMA channel (e.g., ca_sem_wait), 2) Call ca_dma_hw_copy_target to start the transfer, 3) Wait for the transfer to complete (e.g., ca_dma_wait), and 4) Release the channel. It is used whenever high-speed data movement is required without CPU intervention, such as streaming audio, high-speed UART communication, or SPI transfers.

Esempio:

```c
/* Example: Sending a buffer to a UART peripheral via DMA */
void send_data_dma(u8 *data, u32 len, volatile void *uart_dr) {
    ca_dma_ch_t ch = DMA_CH_UART1_TX;

    /* 1. Wait for channel availability */
```

```
    ca_dma_wait(ch);

    /* 2. Start the transfer */
    u8 flat = ca_dma_hw_ch_to_flat(ch);
    bool success = ca_dma_hw_copy_target(flat, data, uart_dr, len,
    CA_DMA_DIR_M2P, DMA_WIDTH_8BIT);

    if (!success) {
        /* Handle configuration error */
    }

    /* 3. The DMA is now running in the background */
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_dma_hw_copy_target() in my embedded application?
A: You should use ca_dma_hw_copy_target() when you need to perform a blocking DMA transfer between a memory buffer and a peripheral register, such as transferring data to/from a UART, SPI, or ADC. It is ideal for scenarios where you want to offload data movement from the CPU to hardware, but you are okay with blocking execution until the transfer completes.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_dma_hw_copy_target() with caOS types?
A:

```
u8 channel = 0;
volatile void *periph = (volatile void *)0x40004400; // UART1_DR
u8 buffer[100];
bool success = ca_dma_hw_copy_target(channel, buffer, periph, 100, CA_DMA_DIR_P2M,
    1);
```

Parameters conf:0.90 **Q:** What are the valid values for the 'flat' parameter and what happens if I pass an invalid one?
A: The 'flat' parameter must be between 0 and 7 for STM32F4. Values outside this range may cause memory access violations or undefined behavior because the function accesses DMA registers using an index that assumes a fixed range of channels.

Return conf:0.95 **Q:** What does it mean when ca_dma_hw_copy_target() returns false?
A: The function does not return false. It always returns true upon successful initialization of the DMA transfer. Any failure in configuration or setup will result in undefined behavior or system lockup, not a return value.

Error/Edge case conf:0.90 **Q:** What are common mistakes when calling ca_dma_hw_copy_target()?
A: Common mistakes include passing NULL pointers for mem or periph, using size = 0, invalid values for dir or width, or calling the function from an ISR. Additionally, using an invalid flat value (outside 0-7) can cause memory access violations.

Behavior conf:0.95 **Q:** Is ca_dma_hw_copy_target() thread-safe and safe to call from an ISR?
A: No, ca_dma_hw_copy_target() is not thread-safe and should not be called from an ISR. It is a blocking function that may cause deadlocks or unpredictable behavior if invoked from interrupt context.

HW differences conf:0.85 **Q:** How does ca_dma_hw_copy_target() differ between STM32F4 and STM32H7 families?
A: On STM32F4, it uses DMA_Stream_TypeDef with CR, PAR, M0AR, and NDTR registers. On STM32H7, it supports both DMA streams and BDMA channels, with BDMA using different registers like CCR, CPAR, CM0AR, and CNDTR, and requires checking for TCM memory regions.

ca_dma_hw_flat_to_ch

Header: caos/drivers/inc/ca_glue_hw.h:98
GUARD: EN_CA_GLUE_HW

```
extern ca_dma_ch_t ca_dma_hw_flat_to_ch(u8 flat); /* converte indice
   flat in handle (controller+canale) */
```

Converts a linear DMA channel index into a hardware-specific controller and stream/channel handle.

Parametri:

Nome	Tipo	Descrizione
flat	u8	A zero-based linear index representing the global DMA resource pool. Valid values range from 0 to (Total_DMA_Channels - 1). Passing an index outside this range will result in an invalid handle.

Ritorna:
Returns a ca_dma_ch_t structure containing the controller ID and the specific channel/stream number. If the input index is out of bounds, it returns DMA_CH_INVALID, which typically contains 0xFF in its fields.

Note:

- This function is a pure mathematical mapping utility and is non-blocking, thread-safe, and safe to call from an ISR. It does not interact with hardware registers directly, so it does not require prior initialization of the DMA peripheral. The most common mistake is assuming that a 'flat' index is portable across different STM32 families; because different chips have different DMA architectures (e.g., GPDMA vs. standard DMA), the mapping logic changes based on the target hardware. Always treat the returned handle as an opaque structure and do not perform manual arithmetic on it.

Uso:

- This function is primarily used by the internal DMA resource manager to translate a generic index (used for tracking availability in a bitmask or array) into the specific hardware identifier required by low-level driver functions. It is typically called after a free channel has been identified via a search algorithm. It is usually paired with functions like ca_dma_free_channel_index or other hardware-specific configuration functions that require the controller/channel pair to target the correct registers.

Esempio:

```
/* Example: Acquiring and configuring a DMA channel */
ca_dma_ch_t dma_handle;
/* Search for a free channel index in the system pool */
i16 index = ca_dma_free_channel_index(-1, &dma_handle);

if (index >= 0) {
    /* Convert the linear index to a hardware handle */
    dma_handle = ca_dma_hw_flat_to_ch((u8)index);

    /* Check if the conversion was successful */
    if (dma_handle.controller != 0xFFU) {
        /* Proceed to configure DMA registers using dma_handle */
        /* ca_dma_set_request(dma_handle, ...); */
    } else {
        /* Handle error: index was valid but mapping failed */
```

```
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_dma_hw_flat_to_ch() in my caOS project?
A: You should use ca_dma_hw_flat_to_ch() when you need to convert a flat index (a unique number representing a DMA channel) into a structured handle that includes both the DMA controller and the specific channel within that controller. This is useful when working with hardware-specific DMA resources and setting up DMA transfers, especially when you have a flat index from a configuration or lookup table.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_dma_hw_flat_to_ch() with caOS types?
A: ca_dma_ch_t ch = ca_dma_hw_flat_to_ch(3); if (ch.controller != 0) { // Valid channel, use ch.controller and ch.channel // For example, configure DMA transfer using these values } else { // Invalid flat index }

Parameters conf:0.90 **Q:** What does the 'flat' parameter represent in ca_dma_hw_flat_to_ch() and what are valid values?
A: The 'flat' parameter is a unique index representing a DMA channel, ranging from 0 to (total_channels - 1) for the specific STM32 family. For example, on F1xx series, valid values are 0 to (F1_TOTAL - 1). Values outside this range, including negative numbers or values >= total_channels, will cause the function to return DMA_CH_INVALID.

Return conf:0.95 **Q:** What do the return values of ca_dma_hw_flat_to_ch() mean and when does it return DMA_CH_INVALID?
A: The function returns a ca_dma_ch_t structure with controller and channel fields when the flat index is valid. If the flat index is invalid (outside the valid range), it returns DMA_CH_INVALID, indicating an error in input or incorrect system configuration.

HW differences conf:0.85 **Q:** How does ca_dma_hw_flat_to_ch() differ between STM32F7xx and STM32H7xx families in terms of hardware implementation?
A: On STM32F7xx, DMA is organized into 2 controllers (DMA1 and DMA2) with a total number of streams defined by F7_TOTAL_STREAMS. On STM32H7xx, there are 3 controllers (DMA1, DMA2, and a third controller) with H7_DMA1_STREAMS, H7_DMA2_STREAMS, and additional streams in the third controller, as defined by H7_TOTAL.

`ca_dma_hw_get_channel_count`

Header: `caos/drivers/inc/ca_glue_hw.h:96`
GUARD: `EN_CA_GLUE_HW`

```
extern u8 ca_dma_hw_get_channel_count(void); /* ritorna il numero totale
    di canali DMA disponibili sull'MCU */
```

Returns the total number of DMA channels/streams available on the specific STM32 microcontroller hardware.

Ritorna:
An unsigned 8-bit integer (u8) representing the total count of DMA channels supported by the current MCU architecture. This value is always greater than zero as every supported MCU has at least one DMA channel.

Note:

- This function is a hardware abstraction layer (HAL) utility that provides compile-time or hardware-defined information about the chip's DMA capabilities. It is non-blocking,

thread-safe, and can be safely called from an Interrupt Service Routine (ISR) because it simply returns a constant value defined by the specific hardware driver. The most common mistake is assuming this value is the same across different STM32 families; always use this function instead of hardcoding numbers to ensure your code remains portable across different STM32 series (e.g., F4 vs H7). It does not require any prior initialization of the DMA peripheral itself, as it only queries the hardware configuration.

Uso:

- This function is typically used during the system initialization phase to dynamically configure software structures that manage DMA resources. It is commonly paired with loop constructs to initialize semaphores, mutexes, or tracking arrays for each available DMA channel. By using this function, you ensure that your driver initialization code automatically scales to the specific MCU being used, preventing buffer overflows or out-of-bounds access when iterating through DMA channels.

Esempio:

```c
/* Example: Initializing a semaphore for every available DMA channel */
#include "caos.h"

void dma_system_init(void) {
    u8 total_channels = ca_dma_hw_get_channel_count();

    /* Ensure we do not exceed our internal tracking array size */
    if (total_channels > MAX_TRACKED_DMA) {
        total_channels = MAX_TRACKED_DMA;
    }

    for (u8 i = 0; i < total_channels; i++) {
        /* Initialize a semaphore for each channel to manage access */
        /* ca_sem_init(semaphore_ptr, name, initial_count, max_count) */
        ca_sem_init(&dma_sem_array[i], "DMA_SEM", 1U, 1U);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_dma_hw_get_channel_count() and why is it useful for DMA initialization?
A: You should call ca_dma_hw_get_channel_count() during system initialization to determine the total number of DMA channels available on your MCU. This is useful for dynamically allocating DMA resources, such as setting up channel arrays or checking if enough channels exist for your application's needs. It helps avoid hardcoding channel counts and makes the code portable across different STM32 families.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_dma_hw_get_channel_count() with caOS types?
A: u8 total_channels = ca_dma_hw_get_channel_count(); if (total_channels > 0) { // Proceed with DMA initialization using total_channels // Example: allocate channel array or validate channel usage }

Return conf:0.90 **Q:** What does the return value of ca_dma_hw_get_channel_count() represent, and what could cause an incorrect result?
A: The return value represents the fixed number of DMA channels or streams available on the specific STM32 MCU family. It is always a positive u8 value determined at compile-time based on the chip type. Failures are not possible in the function itself, but incorrect usage may occur if developers ignore the returned value and attempt to allocate more channels than available, leading to memory issues or undefined behavior.

HW differences conf:0.85 **Q:** How do the implementations of ca_dma_hw_get_channel_count() differ between STM32F1xx and STM32F4xx families?
A: The STM32F1xx family uses F1_TOTAL which maps to the number of DMA channels, while the STM32F4xx family uses F4_TOTAL_STREAMS which maps to the number of DMA streams. This reflects the architectural difference: F1xx has a simpler DMA controller with fixed channels, whereas F4xx uses a more advanced DMA controller with streams that can be configured for different transfer modes and have more flexible capabilities.

---

ca_dma_hw_get_remaining

Header: `caos/drivers/inc/ca_glue_hw.h:110`
GUARD: `EN_CA_GLUE_HW`

```
extern u32 ca_dma_hw_get_remaining(u8 flat); /* live HW counter:
   remaining elements (bytes on GPDMA) of the transfer */
```

Retrieves the number of data units remaining to be transferred by a specific DMA channel.

Parametri:

Nome	Tipo	Descrizione
`flat`	`u8`	The flattened index of the DMA channel. Must be a valid index within the range [0, DMA_MAX_CHANNELS - 1]. Passing an invalid index may lead to memory access violations or reading from undefined hardware registers.

Ritorna:
Returns the count of remaining data units (bytes, half-words, or words depending on configuration) as a u32. Returns 0 if the transfer is complete or if the channel is not currently active.

Note:

- This function is non-blocking and executes in constant time, making it safe to call from both thread context and Interrupt Service Routines (ISRs). It provides a 'live' snapshot of the hardware register, meaning the value can change at any microsecond if the DMA is active. A common beginner mistake is assuming this value remains constant while processing data; always cache the result in a local variable if you need to perform multiple calculations based on the same remaining count. Note that on some architectures (like H5xx/U5xx), the returned value represents bytes, while on others it represents the number of data items configured in the DMA transfer width.

Uso:

- This function is typically used to monitor the progress of a DMA transfer or to calculate how much data has been successfully moved into a buffer. It is frequently paired with ca_dma_start_circular or similar transfer-start functions to implement 'ping-pong' buffering or to check if a circular buffer has wrapped around. In a typical sequence, you initialize the DMA, start the transfer, and then periodically poll this function to determine if enough data has arrived to trigger a processing task, or to calculate the write pointer offset in a circular buffer.

Esempio:

```
/* Example: Monitoring DMA progress in a circular buffer */
ca_dma_ch_t my_dma_ch = 0; /* Assume channel 0 is allocated */
```

```
u32 total_size = 1024;

/* Start circular transfer */
/* ca_dma_dir_t and ca_dma_width_t are assumed to be valid enums */
if (ca_dma_start_circular(my_dma_ch, buffer, (void*)0x40000000, total_size, 0,
    0)) {
    /* Get the remaining count using the high-level wrapper */
    u32 remaining = ca_dma_get_remaining(my_dma_ch);

    /* Calculate how many bytes have been moved so far */
    u32 moved = total_size - remaining;

    if (moved > 512) {
        /* Process the first half of the buffer */
    }
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_dma_hw_get_remaining in my caOS application?
A: You should use it to monitor DMA progress or calculate how much data has been moved into a buffer. It is commonly used with circular buffers to check for wrap-around or to determine if enough data has arrived to trigger a processing task.

Code example conf:1.00 **Q:** Provide a minimal example of how to check the remaining DMA transfer count.
A: u8 channel = 0; u32 remaining = ca_dma_hw_get_remaining(channel); if (remaining == 0) { /* Transfer complete */ }

Parameters conf:1.00 **Q:** What is the 'flat' parameter and what are the consequences of passing an invalid value?
A: The 'flat' parameter is the flattened index of the DMA channel. It must be in the range [0, DMA_MAX_CHANNELS - 1]. Passing an invalid index may lead to memory access violations or reading from undefined hardware registers.

Return conf:1.00 **Q:** What does a return value of 0 indicate, and is the returned value always constant?
A: A return value of 0 indicates the transfer is complete or the channel is inactive. The value is a 'live' snapshot of the hardware register and can change at any microsecond if the DMA is active; therefore, you should cache the result in a local variable if you need to perform multiple calculations.

HW differences conf:1.00 **Q:** How does the return value of ca_dma_hw_get_remaining differ across STM32 families?
A: On F1xx, F4xx, F7xx, and H7xx, the function returns the number of data items configured in the transfer width (using CNDTR or NDTR registers). On H5xx and U5xx, the function returns the value in bytes (using the CBR1 register), which only coincides with the number of elements if the transfer width is 8-bit.

`ca_dma_hw_irq_disable`

Header: `caos/drivers/inc/ca_glue_hw.h:108`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_dma_hw_irq_disable(u8 flat); /* disabilita l'interrupt di
    completamento per il canale DMA */
```

Disables the hardware interrupt for a specific DMA channel at the NVIC level.

Parametri:

Nome	Tipo	Descrizione
`flat`	`u8`	The flattened index representing the specific DMA channel. Must be less than the hardware-specific total channel count (e.g., F4_TOTAL_STREAMS or U5_TOTAL).

Ritorna:
void. This function does not return a status code; it silently ignores invalid channel indices that exceed the hardware limit.

Note:

- This function is non-blocking and executes immediately by interacting directly with the Nested Vectored Interrupt Controller (NVIC). It is thread-safe in the sense that it performs an atomic register write, but it does not provide synchronization for the DMA peripheral itself. You can safely call this from an ISR, as it does not involve OS primitives like mutexes or semaphores. The most common mistake is assuming that disabling the interrupt stops the DMA transfer; it does not. The DMA hardware will continue to move data in the background, but the CPU will no longer be notified when the transfer completes, which can lead to 'silent' data corruption or missed processing if you expect a callback to trigger.

Uso:

- This function is typically used during the teardown or reconfiguration phase of a DMA channel. It is almost always paired with ca_dma_hw_release to ensure that the hardware is fully quiesced and the interrupt vector is silenced before the channel is reassigned or powered down. You should call this before clearing any callback pointers or releasing the DMA channel handle to prevent a 'dangling' interrupt from firing while the system is in an inconsistent state. It is a critical step in the cleanup sequence to ensure that no stale interrupts trigger after the driver has logically released the resource.

Esempio:

```c
/* Example: Safely shutting down a DMA channel before re-initialization */
ca_dma_ch_t my_dma_ch = get_dma_handle();
u8 flat_idx = ca_dma_hw_ch_to_flat(my_dma_ch);

/* Disable the interrupt first to prevent ISR execution during cleanup */
ca_dma_hw_irq_disable(flat_idx);

/* Now it is safe to release the hardware and clear the callback */
ca_dma_hw_release(flat_idx);
/* Clear application-level callback pointers */
dma_cb[flat_idx] = NULL;
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_dma_hw_irq_disable() and what is its purpose?
A: You should call ca_dma_hw_irq_disable() to disable the hardware interrupt for a specific DMA channel. This is useful during critical operations or when releasing a DMA channel to prevent unexpected interrupts. It is a low-level function that directly modifies the NVIC register.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_dma_hw_irq_disable() with caOS types?
A:

```c
u8 dma_channel = 2;
ca_dma_hw_irq_disable(dma_channel);
```

Parameters conf:0.90 **Q:** What does the 'flat' parameter represent and what are valid values?

A: The 'flat' parameter is a linear identifier for the DMA channel. Valid values are integers from 0 up to (but not including) the total number of DMA channels for the specific STM32 series. For example, for F1 series, valid values are 0 to F1_TOTAL-1; for F4 series, 0 to F4_TOTAL_STREAMS-1.

Return conf:0.95 **Q:** Does ca_dma_hw_irq_disable() return any value and can it fail?

A: The function does not return any value. It cannot fail, but if called with an invalid 'flat' parameter (outside the valid range), it will simply do nothing without any error.

HW differences conf:0.85 **Q:** How does ca_dma_hw_irq_disable() differ between STM32 families like F1 and F4?

A: The function is implemented differently for each STM32 family based on their total number of DMA channels. For F1, it checks against F1_TOTAL; for F4, it checks against F4_TOTAL_STREAMS. Each implementation uses the appropriate total constant to validate the flat parameter before disabling the interrupt via NVIC_DisableIRQ().

`ca_dma_hw_irq_enable`

Header: `caos/drivers/inc/ca_glue_hw.h:107`
GUARD: `EN_CA_GLUE_HW`

```
1 extern void ca_dma_hw_irq_enable(u8 flat); /* abilita l'interrupt di
     completamento per il canale DMA */
```

Enables the hardware interrupt for a specific DMA channel in the NVIC controller.

Parametri:

Nome	Tipo	Descrizione
`flat`	`u8`	The logical index (flat ID) of the DMA channel. Must be less than the total number of channels supported by the specific hardware platform; values out of range are ignored.

Ritorna:
void

Note:

- This function is non-blocking and executes immediately by interacting directly with the ARM Cortex-M NVIC (Nested Vectored Interrupt Controller). It is thread-safe in the sense that it performs atomic register writes, but it should be called only after the DMA channel has been properly configured. It is safe to call from an ISR, though typically it is called during the initialization or allocation phase of a DMA transfer. A common beginner mistake is calling this before the DMA peripheral clock is enabled or before the specific channel registers are configured, which can lead to immediate, spurious interrupts if the hardware is in an undefined state.

Uso:

- This function is used as the final step in the DMA channel setup sequence. It is typically paired with functions that allocate a DMA channel (like ca_dma_free_channel_index or similar allocation logic) and functions that configure the transfer parameters. By enabling the IRQ, you allow the CPU to jump to the DMA completion handler once the hardware transfer finishes. You should call this only after you have verified that the channel is successfully allocated and the transfer parameters are ready, ensuring that the interrupt service routine does not trigger prematurely.

Esempio:

```c
/* Example: Allocating and enabling a DMA channel */
ca_dma_ch_t my_dma_channel;
/* Assume a function exists to get a channel index */
i16 channel_index = ca_dma_free_channel_index(-1, &my_dma_channel);

if (channel_index >= 0) {
    /* Configure DMA transfer parameters here using ca_dma_set_request or similar */
    /* ... */

    /* Now enable the interrupt so we get notified upon completion */
    ca_dma_hw_irq_enable((u8)channel_index);
} else {
    /* Handle error: no DMA channels available */
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_dma_hw_irq_enable() and what is its purpose in DMA operations?

A: You should call ca_dma_hw_irq_enable() to enable hardware interrupts for a specific DMA channel, allowing asynchronous handling of DMA transfer completions. This is useful when you want to be notified upon completion of a DMA transfer without polling, enabling more efficient and responsive code. The function configures the NVIC to enable the interrupt for the specified DMA channel.

Code example conf:0.95 **Q:** Can you provide a minimal C example showing how to use ca_dma_hw_irq_enable with caOS types?

A:

```c
u8 channel = 3;
ca_dma_hw_irq_enable(channel);
```

This example enables the interrupt for DMA channel 3. The function takes a single u8 parameter representing the DMA channel number and does not return any value.

Parameters conf:0.90 **Q:** What does the 'flat' parameter in ca_dma_hw_irq_enable() represent, and what are the valid values?

A: The 'flat' parameter represents the DMA channel identifier, ranging from 0 to 7. Valid values are integers in the range 0-7. Values outside this range (less than 0 or greater than or equal to the total number of channels for the specific MCU family) are ignored and no interrupt is enabled. For example, on F1xx family, valid values are 0-7, while on F4xx, F7xx, H5xx, and H7xx, the valid range depends on the total number of streams defined for those families.

Return conf:0.95 **Q:** Does ca_dma_hw_irq_enable() return any value, and what could cause it to fail?

A: The function does not return any value. It is a void function. It cannot fail in a strict sense, but it may not perform any action if the input parameter 'flat' is invalid (i.e., less than 0 or greater than or equal to the total number of channels for the MCU family). In such cases, no interrupt is enabled, but no error is signaled.

HW differences conf:0.85 **Q:** How does ca_dma_hw_irq_enable() differ between F1xx and F4xx STM32 families in terms of implementation?

A: The function implementation is similar in structure but differs in the total number of channels supported. On F1xx, it checks if 'flat' is less than F1_TOTAL (typically 8), while on F4xx, it checks against F4_TOTAL_STREAMS (typically 8 for stream-based DMA). Both use NVIC_SetPriority and NVIC_EnableIRQ to configure the interrupt, but the total number of available DMA channels varies by family. F1xx uses a fixed 8-channel model, whereas F4xx supports more streams with potentially different configurations.

ca_dma_hw_m2m_first_flat

Header: caos/drivers/inc/ca_glue_hw.h:100
GUARD: EN_CA_GLUE_HW

```
extern u8 ca_dma_hw_m2m_first_flat(void); /* first flat index able to do
    memory-to-memory (F4/F7: DMA2 only) */
```

Returns the index of the first DMA channel capable of performing memory-to-memory (M2M) transfers.

Ritorna:
An unsigned 8-bit integer representing the index of the first valid DMA channel for M2M operations. On architectures where all channels support M2M, this returns 0. On architectures with restrictions (like F4/F7), it returns the index of the first stream in the DMA controller that supports M2M.

Note:

- This function is hardware-dependent and abstracts the architectural limitations of different STM32 DMA controllers. It is thread-safe and can be called from any context, including ISRs, as it simply returns a constant value defined at compile-time for the specific target. The primary purpose is to allow generic driver code to correctly allocate DMA resources without hardcoding hardware-specific channel indices. A common mistake is assuming that all DMA channels on an STM32 are identical; failing to use this function when searching for an M2M-capable channel will lead to runtime errors or silent failures on F4/F7 series chips where only DMA2 supports M2M.

Uso:

- This function is typically used during the initialization phase of a driver or a resource manager to identify the starting point for a search loop. It is almost always paired with ca_dma_get_channel_count() to define the boundaries of the available DMA channel pool. By using this function, the developer ensures that the code remains portable across different STM32 families, as the logic automatically adjusts to whether the hardware supports M2M on all channels or only on a specific subset.

Esempio:

```
/* Example: Finding a free DMA channel for a memory-to-memory transfer */
#include "caos.h"

void find_m2m_channel(void) {
    u8 first_m2m = ca_dma_hw_m2m_first_flat();
    u8 total_ch = ca_dma_get_channel_count();
    i16 channel_idx = -1;

    /* Iterate only through channels that support M2M */
    for (u8 i = first_m2m; i < total_ch; i++) {
        /* Check if channel is available (using internal driver logic) */
        /* If available, reserve it */
        channel_idx = (i16)i;
        break;
    }

    if (channel_idx >= 0) {
        /* Successfully found a valid M2M channel */
        /* Proceed to configure DMA for memory copy */
```

```
    } else {
        /* No M2M capable channel available */
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_dma_hw_m2m_first_flat in my driver initialization?
A: You should call it during the initialization phase of a driver or resource manager to identify the starting index for a search loop. It is typically paired with ca_dma_get_channel_count() to define the boundaries of the available DMA channel pool, ensuring your code remains portable across different STM32 families.

Code example conf:0.90 **Q:** How do I write a loop to find an M2M-capable DMA channel using caOS types?
A: You can use the following pattern: u8 start = ca_dma_hw_m2m_first_flat(); u8 count = ca_dma_get_channel_count(); for (u8 i = start; i < count; i++) { /* check channel availability */ }.

Return conf:1.00 **Q:** What does the return value of ca_dma_hw_m2m_first_flat represent, and what happens if I ignore it?
A: It returns the index of the first DMA channel capable of M2M transfers. On architectures where all channels support M2M, it returns 0. On F4/F7 series, it returns the index of the first stream in DMA2. Ignoring this and assuming all channels are identical will lead to runtime errors or silent failures on F4/F7 chips.

HW differences conf:1.00 **Q:** How does the behavior of ca_dma_hw_m2m_first_flat differ between the F4/F7 series and the H7/H5 series?
A: On F4 and F7 series, the function returns the index of the first stream in DMA2 because only DMA2 supports M2M transfers. In contrast, on H7, H5, and F1 series, the function returns 0, as these architectures support M2M on all available DMA channels without the specific DMA2-only restriction found in the F4/F7.

`ca_dma_hw_release`

Header: `caos/drivers/inc/ca_glue_hw.h:104`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_dma_hw_release(u8 flat); /* libera il canale DMA e
    disabilita gli interrupt */
```

Forcefully stops a DMA channel and resets its hardware configuration.

Parametri:

Nome	Tipo	Descrizione
`flat`	`u8`	The flattened index representing the specific DMA stream or channel. Must be a valid index within the hardware-specific range; passing an out-of-bounds value may lead to memory corruption or hard faults.

Ritorna:
void

Note:

- This function is blocking because it enters a while-loop to wait for the hardware to acknowledge the disable request (clearing the EN bit). It is generally safe to call from an

ISR, but be aware that the blocking loop could delay other high-priority tasks if the hardware is stuck. It is not thread-safe on its own; it must be protected by a spinlock or mutex if multiple threads attempt to manage the same DMA channel simultaneously. A common beginner mistake is calling this while a DMA transfer is still critical, which will immediately abort the operation and potentially leave peripheral buffers in an inconsistent state.

Uso:

- This function is typically used during the teardown phase of a driver or when a communication session (like UART or SPI) is being closed. It is usually paired with ca_dma_hw_irq_disable() to ensure no pending interrupts trigger during the shutdown process. In the standard caOS driver lifecycle, it is called after the transfer is finished or aborted, followed by clearing the callback pointers and releasing the software-level lock to allow the channel to be re-allocated for a different task.

Esempio:

```
/* Example: Shutting down a DMA channel after a transfer */
void stop_dma_transfer(u8 flat_channel) {
    /* 1. Disable interrupts to prevent race conditions during shutdown */
    ca_dma_hw_irq_disable(flat_channel);

    /* 2. Forcefully stop the hardware and clear flags */
    ca_dma_hw_release(flat_channel);

    /* 3. Clean up software state */
    dma_cb[flat_channel] = NULL;
    dma_cb_arg[flat_channel] = NULL;

    /* Now the channel is safe to be re-initialized for another peripheral */
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_dma_hw_release() and why is it important?
A: You should call ca_dma_hw_release() when you are done using a DMA channel and want to free it for reuse. It disables the DMA channel's execution and clears its interrupt flags, ensuring that no further DMA operations occur on that channel. This is important to prevent resource leaks and ensure proper system behavior when managing multiple DMA operations.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_dma_hw_release() with caOS types?
A: u8 channel = 3; ca_dma_hw_release(channel);

Parameters conf:0.90 **Q:** What does the 'flat' parameter represent and what are valid values for it?
A: The 'flat' parameter is a unique identifier for a DMA channel in 'flat' format, which is a simple numeric index. Valid values are integers from 0 up to the maximum number of available DMA channels for the specific MCU family (e.g., 0-7 for STM32F4). Using an invalid value (outside this range) leads to undefined behavior.

Return conf:0.95 **Q:** Does ca_dma_hw_release() return any value and can it fail?
A: No, ca_dma_hw_release() does not return any value. It is designed to never fail, but it can cause undefined behavior if the 'flat' parameter is not a valid channel number. The function is guaranteed to complete successfully if the input is valid.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_dma_hw_release()?
A: Common mistakes include calling ca_dma_hw_release() with an invalid 'flat' value (outside the valid range), calling it from an ISR (which may cause deadlock), or calling

it on a channel that was never configured or activated. Using an invalid channel number can lead to memory access violations or unpredictable behavior.

HW differences conf:0.90 **Q:** How does ca_dma_hw_release() differ between STM32F1xx and STM32F4xx families?
A: In STM32F1xx, ca_dma_hw_release() operates on DMA_Channel_TypeDef registers using CCR (Channel Control Register) and checks for the EN bit to disable the channel. In STM32F4xx, it operates on DMA_Stream_TypeDef registers using CR (Control Register) and checks for the DMA_SxCR_EN bit. The F1xx uses channel-based registers while F4xx uses stream-based registers.

`ca_dma_hw_set_request`

Header: `caos/drivers/inc/ca_glue_hw.h:105`
GUARD: `EN_CA_GLUE_HW`

```
1 extern bool ca_dma_hw_set_request(u8 flat, u16 request); /* associa la
    request line periferica al canale (DMA_REQ_NONE = SW/M2M) */
```

Configures the hardware trigger source (request line) for a specific DMA channel.

Parametri:

Nome	Tipo	Descrizione
`flat`	`u8`	The flattened index of the DMA channel. Must be within the range [0, TOTAL_CHANNELS-1] for the specific MCU architecture.
`request`	`u16`	The peripheral request ID to map to the channel. Use DMA_REQ_NONE for memory-to-memory or software-triggered transfers.

Ritorna:
Returns true if the mapping was successfully applied or if the architecture handles requests implicitly. Returns false if the channel index is out of bounds or the request ID is invalid for the hardware.

Note:

- This function is non-blocking and executes immediately, making it safe to call from ISRs if the underlying hardware registers are accessible. It is not inherently thread-safe; if multiple tasks attempt to reconfigure the same DMA channel simultaneously, you must protect this call with a mutex or critical section. A common beginner mistake is assuming that all STM32 families support dynamic request mapping; on older families like F1, the mapping is fixed in hardware, and this function effectively ignores the request parameter. Always ensure the DMA channel is disabled before changing its request source to prevent unpredictable hardware behavior or accidental triggers.

Uso:

- This function is typically used during the initialization phase of a peripheral driver (e.g., UART, SPI, or ADC) to bind the peripheral's hardware trigger to a specific DMA channel. It is usually called after obtaining a free channel index via ca_dma_free_channel_index() and before configuring the transfer parameters with functions like ca_dma_copy_inc(). By setting the request, you tell the DMA controller which peripheral signal should trigger the start of a data transfer. If you are performing a memory-to-memory copy, you must pass DMA_REQ_NONE to ensure the DMA controller does not wait for an external peripheral signal.

Esempio:

```c
/* Example: Configuring a DMA channel for a peripheral transfer */
ca_dma_ch_t my_ch = /* Obtain a channel index using a system-specific allocation function */;
if (my_ch != 0xFF) {
    u8 flat = /* Convert channel handle to flat index using internal system mapping */;
    /* Map the peripheral request to this DMA channel */
    if (ca_dma_hw_set_request(flat, 10)) {
        /* Proceed to configure transfer parameters */
        ca_dma_copy_inc(my_ch, src_buffer, dst_buffer, size, 0);
    } else {
        /* Handle configuration error */
    }
} else {
    /* Handle allocation failure */
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_dma_hw_set_request in my peripheral driver initialization?
A: You should call this function after obtaining a free DMA channel index via ca_dma_free_channel_inc and before configuring transfer parameters with functions like ca_dma_copy_inc(). It is used to bind a peripheral's hardware trigger to a specific DMA channel. Always ensure the DMA channel is disabled before calling this to prevent unpredictable behavior.

Code example conf:0.90 **Q:** Provide a minimal example of how to configure a DMA channel for a peripheral using caOS types.
A: u8 channel = ca_dma_free_channel_index(); if (channel < TOTAL_CHANNELS) { if (ca_dma_hw_set_request(channel, MY_PERIPHERAL_REQ_ID)) { /* Proceed with transfer configuration */ } }

Parameters conf:0.95 **Q:** What are the valid values for the 'request' parameter and what does DMA_REQ_NONE signify?
A: The 'request' parameter represents the peripheral request ID. DMA_REQ_NONE is used for memory-to-memory transfers or software-triggered transfers, signaling the DMA controller not to wait for an external peripheral signal. Valid ranges for the ID depend on the MCU architecture; for example, on F4/F7, it must be <= 7.

Return conf:1.00 **Q:** What causes ca_dma_hw_set_request to return false?
A: The function returns false if the provided channel index (flat) is out of bounds for the specific MCU architecture, or if the provided request ID is invalid for the hardware. On some architectures, it may also return false if the request ID exceeds the maximum value allowed by the hardware registers.

HW differences conf:1.00 **Q:** How does the implementation of ca_dma_hw_set_request differ between STM32F1 and STM32H7 families?
A: On the F1 family, the channel-to-peripheral mapping is fixed in hardware, so the function ignores the request parameter and only validates the channel index. Conversely, on the H7 family, the function interacts with the DMAMUX peripheral, specifically writing the request ID to the DMAMUX_CxCR register to dynamically configure the mapping.

`ca_dma_hw_start_circular`

Header: `caos/drivers/inc/ca_glue_hw.h:109`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_dma_hw_start_circular(u8 flat, void *mem, volatile void *periph, u32 size, ca_dma_dir_t dir, ca_dma_width_t width); /* avvia trasferimento DMA in modalit  circolare (continua) */
```

Configures and starts a DMA channel in circular mode for continuous, autonomous data transfers.

Parametri:

Nome	Tipo	Descrizione
`flat`	`u8`	The flattened index of the DMA channel/stream. Must be a valid index within the hardware-specific range.
`mem`	`void *`	Pointer to the memory buffer. Must be in accessible RAM; ITCM or TCM regions may be invalid depending on the MCU architecture.
`periph`	`volatile void *`	Pointer to the peripheral register (e.g., UART DR or SPI DR). Must be volatile to prevent compiler optimization.
`size`	`u32`	Number of data units to transfer per cycle. The buffer must be large enough to hold this many units.
`dir`	`ca_dma_dir_t`	Direction of transfer: CA_DMA_DIR_M2P (Memory to Peripheral) or CA_DMA_DIR_P2M (Peripheral to Memory).
`width`	`ca_dma_width_t`	Data width per transfer (e.g., 8-bit, 16-bit, 32-bit).

Ritorna:
Returns true if the DMA channel was successfully configured and enabled. Returns false if the channel index is out of bounds or if hardware-specific safety checks (like memory region validation) fail.

Note:

- This function is non-blocking; it configures the hardware registers and returns immediately while the DMA engine handles the transfer in the background. It is generally thread-safe if each DMA channel is owned by a single task, but it is not inherently re-entrant for the same channel. A common beginner mistake is passing a pointer to a local stack variable as the 'mem' buffer, which will cause data corruption when the function returns and the stack frame is reused. Ensure the memory buffer is properly aligned and resides in a memory region accessible by the DMA controller (e.g., avoid TCM on some STM32 series). Always ensure the DMA channel is not currently active or has been properly stopped before calling this to avoid undefined hardware states.

Uso:

- This function is used for continuous data streaming, such as logging sensor data to a buffer or sending repetitive audio samples to a DAC. It is typically called after the DMA channel has been acquired via a resource manager. The standard lifecycle involves: 1) Acquiring the channel, 2) Calling this function to start the circular transfer, 3) Allowing the hardware to run indefinitely, and 4) Calling ca_dma_stop_circular or ca_dma_release when the stream is no longer needed to free the hardware resource.

Esempio:

```
/* Example: Start a circular DMA transfer for UART transmission */
#define BUFFER_SIZE 128
```

```
u8 tx_buffer[BUFFER_SIZE];

void start_uart_stream(ca_dma_ch_t ch, volatile void *uart_dr) {
    /* Convert abstract channel handle to hardware-specific flat index */
    u8 flat = ca_dma_hw_ch_to_flat(ch);

    /* Start circular transfer: Memory to Peripheral, 8-bit width */
    bool success = ca_dma_hw_start_circular(flat, tx_buffer, uart_dr,
    BUFFER_SIZE, CA_DMA_DIR_M2P, CA_DMA_WIDTH_8);

    if (!success) {
        /* Handle configuration error: log or trigger system fault */
    }
    /* DMA is now running in the background; no further CPU intervention needed
    */
}
```

Q&A — fine-tuning review

7/7 pairs · qwen3-coder:latest

Usage conf:0.95 **Q:** When should I use ca_dma_hw_start_circular instead of a regular DMA transfer?

A: Use ca_dma_hw_start_circular when you need continuous, repeated data transfers, such as audio streaming or sensor data acquisition. It's ideal for applications where data must be continuously fed to or from a peripheral without stopping, as it automatically reloads the transfer counter after each cycle.

Code example conf:0.90 **Q:** Can you show a minimal example of how to call ca_dma_hw_start_circular with proper caOS types?

A:

```
u8 channel = 0;
volatile void *periph = (volatile void *)0x40013800; /* UART1_DR */
void *buffer = (void *)0x20000000;
u32 size = 1024;
ca_dma_dir_t dir = CA_DMA_DIR_P2M;
ca_dma_width_t width = 1;
bool success = ca_dma_hw_start_circular(channel, buffer, periph, size, dir, width)
    ;
```

Parameters conf:0.95 **Q:** What are the valid values for the 'width' parameter in ca_dma_hw_start_circular?

A: The 'width' parameter must be 1, 2, or 4, representing 8-bit, 16-bit, or 32-bit data transfers respectively. Values less than 1 or greater than 4 are invalid and will cause the function to fail.

Return conf:0.90 **Q:** What does it mean if ca_dma_hw_start_circular returns false?

A: A return value of false indicates an error occurred during the DMA setup, such as invalid parameters (e.g., channel number out of range, NULL buffer, or invalid direction), or the channel is already in use. It may also fail due to hardware limitations or memory alignment issues.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_dma_hw_start_circular?

A: Common mistakes include using a buffer that is not aligned properly, specifying a size that is not a multiple of the data width, passing a NULL pointer for memory or peripheral address, or calling the function from an ISR. Additionally, using a channel that is already active or providing an invalid channel number can lead to failure.

Behavior conf:0.90 **Q:** Is ca_dma_hw_start_circular thread-safe and can it be called from an ISR?

A: No, ca_dma_hw_start_circular is not thread-safe and must not be called from an ISR. It is only safe to call from task context. The function is blocking only during the disabling of a previous DMA channel, but not during the actual start of the transfer.

HW differences conf:0.85 **Q:** How does ca_dma_hw_start_circular differ between F7xx and H7xx MCUs in terms of implementation?

A: On F7xx, the function uses DMA streams with standard registers like CR, PAR, M0AR, and NDTR. On H7xx, it supports both DMA1/DMA2 streams and BDMA channels, with additional checks for TCM memory access and different register layouts for BDMA (e.g., using CM0AR instead of CMAR). H7xx also uses CTR1, CTR2, CLBAR, and CLLR for advanced linked list configurations.

---

ca_dma_init

Header: `caos/drivers/inc/dma.h:71`
Source: `caos/drivers/src/dma.c:38:61`
GUARD: `EN_DMA`

```
void ca_dma_init(void)
```

Initializes the DMA controller, internal locks, and synchronization semaphores for all available DMA channels.

Ritorna:
void (does not return a value).

Note:

- This function is non-blocking and must be called exactly once during the system startup phase, typically inside the main initialization sequence before any peripheral drivers (like UART, DAC, or SDIO) attempt to use DMA. It is not thread-safe in the sense that it should not be called concurrently from multiple threads; it is intended to be part of the system boot-up process. It is not safe to call from an ISR because it performs memory initialization and CLI registration. The most common beginner mistake is calling peripheral initialization functions (e.g., ca_uart_init_dma) before calling ca_dma_init, which results in uninitialized semaphores and system crashes when the DMA controller attempts to signal completion.

Uso:

- ca_dma_init is the foundational setup function for the entire DMA subsystem. It is used in the early stages of the application lifecycle, usually within the main() function or the system initialization task, to prepare the hardware clocks and the software synchronization primitives (semaphores) that allow tasks to wait for DMA transfers to finish. It is always paired with subsequent peripheral-specific DMA initialization functions like ca_uart_init_dma or ca_dac_init. Without this call, the OS cannot track which DMA channels are in use, leading to resource conflicts and unpredictable behavior when multiple peripherals try to access the DMA controller simultaneously.

Esempio:

```
void system_init(void) {
    /* 1. Initialize the DMA subsystem before any peripheral drivers */
    ca_dma_init();

    /* 2. Now it is safe to initialize peripherals that rely on DMA */
    /* Initialize DAC with DMA support */
    ca_dac_init(&(ca_dac_config_t){ .channel = 1 });

    /* Initialize UART with DMA support */
    if (!ca_uart_init_dma(1, 115200, NULL)) {
        /* Handle initialization failure */
        ca_safeguard_failed();
    }
```

```
14
15      /* 3. Register CLI for debugging if enabled */
16      ca_cli_create_thread();
17  }
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should ca_dma_init() be called in a caOS application, and why is it important?
A: ca_dma_init() should be called once at the start of the system initialization, before any DMA operations are needed. It initializes the DMA system by creating semaphores and enabling the clocks for the DMA controllers. This function must not be called from an ISR and is a one-time initialization step required for DMA functionality.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_dma_init() with caOS types?
A: void example_dma_init(void) { ca_dma_init(); }

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_dma_init()?
A: A common mistake is calling ca_dma_init() from within an ISR, which is explicitly forbidden. Another edge case is calling it multiple times, which is not thread-safe and could lead to undefined behavior. The function is also not thread-safe if not protected by a spinlock, so concurrent calls from different threads must be avoided.

Behavior conf:0.90 **Q:** Is ca_dma_init() blocking or non-blocking, and is it safe to call from an ISR or in a multi-threaded environment?
A: ca_dma_init() is a blocking function and must not be called from an ISR. It is not inherently thread-safe unless properly managed with a spinlock. The function prepares the DMA system for use and must be called only once, making it unsuitable for repeated or concurrent invocation.

`ca_dma_isr_complete`

Header: `caos/drivers/inc/dma.h:147`
Source: `caos/drivers/src/dma.c:219:228`
GUARD: `EN_DMA`

```
1 void ca_dma_isr_complete(u8 controller, u8 stream)
```

Handles DMA interrupt completion for a specific controller and stream by invoking the registered callback and signaling a waiting task via semaphore.

Parametri:

Nome	Tipo	Descrizione
`controller`	`u8`	The DMA controller index (e.g., 0 for DMA1, 1 for DMA2). Must be a valid controller number as supported by the hardware; invalid values may cause undefined behavior or access to wrong memory.
`stream`	`u8`	The specific stream or channel index on the selected controller. Must be a valid stream number for the given controller; invalid values may lead to incorrect semaphore signaling or null pointer dereference.

Ritorna:
void. This function does not return any value because it is an ISR handler and is expected to complete immediately without returning control to the caller.

Note:

- This function is intended to be called only from within a hardware DMA interrupt handler (e.g., DMA1_Stream0_IRQHandler). It is NOT blocking and must execute quickly to avoid missing subsequent interrupts. It is safe to call from ISR context because it uses ca_sem_post, which is designed for ISR use. It assumes that the DMA channel has been initialized and a callback has been registered; otherwise, it may attempt to call a NULL function pointer. A common mistake is to call this function manually from a task instead of from an ISR, which can lead to incorrect semaphore signaling or system instability.

Uso:

- This function is used inside a hardware interrupt vector to signal completion of a DMA transfer. It is typically paired with ca_dma_init, ca_dma_set_callback, and ca_dma_wait. It allows tasks to block until a DMA transfer completes, enabling asynchronous operation without blocking the main application thread. It is essential to ensure that the correct controller and stream are passed to avoid triggering the wrong callback or semaphore.

Esempio:

```c
/* In the hardware interrupt vector */
void DMA1_Stream0_IRQHandler(void) {
    /* Notify caOS that the transfer on DMA1, Stream 0 is complete */
    ca_dma_isr_complete(0, 0);
}

/* In a task that initiates a DMA transfer */
void transfer_data(void) {
    ca_dma_ch_t ch = {0, 0}; /* DMA1, Stream 0 */

    /* Set up the DMA transfer */
    ca_dma_set_request(ch, DMA_REQUEST_MEM_TO_MEM);
    ca_dma_copy_inc(ch, src_addr, dst_addr, size);

    /* Wait for completion */
    ca_dma_wait(ch);

    /* Transfer is complete, proceed with processing */
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should ca_dma_isr_complete be called, and why is it important for task synchronization?
A: ca_dma_isr_complete should be called from an interrupt service routine (ISR) when a DMA transfer has completed. It is important because it signals the completion of the DMA stream, allowing a waiting task to be woken up via a semaphore, enabling proper task synchronization in a real-time system.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_dma_isr_complete in an ISR?
A: void DMA1_Stream0_IRQHandler(void) { ca_dma_isr_complete(0, 0); }

Parameters conf:0.90 **Q:** What do the controller and stream parameters represent, and what are the valid ranges for each?
A: The controller parameter specifies which DMA controller is being used (0 for DMA1, 1 for DMA2). The stream parameter specifies which stream within that controller is completing (0-7). Invalid values for either parameter can cause memory access errors.

Return conf:0.85 **Q:** Does ca_dma_isr_complete return any value, and under what conditions might it not perform its intended function?
A: ca_dma_isr_complete does not return any value. It cannot fail, but it may not

perform its intended function if invalid parameters are passed, such as controller values outside 0-1 or stream values outside 0-7, which could lead to memory access errors.

---

ca_dma_release

Header: caos/drivers/inc/dma.h:87
Source: caos/drivers/src/dma.c:114:128
GUARD: EN_DMA

```
void ca_dma_release(ca_dma_ch_t ch)
```

Releases a previously acquired DMA channel, disabling its interrupts and clearing associated callbacks.

Parametri:

Nome	Tipo	Descrizione
ch	ca_dma_ch_t	The DMA channel handle that was previously acquired via a ca_dma_free_channel or similar allocation function.

Ritorna:
void

Note:

- This function is thread-safe as it uses a spinlock to update the internal DMA usage bitmask. It is non-blocking, meaning it will return immediately after performing the cleanup operations. You must ensure that the DMA hardware is not currently performing a transfer before calling this, as releasing an active channel can lead to undefined behavior or system instability. A common mistake is calling this function on a channel that was never successfully acquired or has already been released, which may lead to inconsistent state in the internal dma_cb array. It is safe to call from most contexts, but ensure you are not holding other locks that might cause a deadlock if the spinlock implementation interacts with them.

Uso:

- This function is typically used during the teardown phase of a peripheral driver (like UART or SPI) or when an error occurs during the initialization sequence. When a driver fails to allocate a second required DMA channel (e.g., failing to get a TX channel after successfully getting an RX channel), it must call ca_dma_release on the already-acquired channel to prevent resource leaks. It is the mandatory counterpart to allocation functions; failing to call this will permanently mark the channel as 'in use' in the system, eventually causing all future DMA allocation attempts to fail.

Esempio:

```
/* Example: Error handling during peripheral DMA setup */
ca_dma_ch_t rx_ch;
if (ca_dma_free_channel(&rx_ch) < 0) {
    return false; // Failed to get RX channel
}

ca_dma_ch_t tx_ch;
if (ca_dma_free_channel(&tx_ch) < 0) {
    /* Cleanup: release the RX channel before exiting */
    ca_dma_release(rx_ch);
    return false; // Failed to get TX channel, system state remains clean
}
```

```

/* Proceed with configuration if both channels are acquired */
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_dma_release() in my DMA-driven embedded application?
A: You should call ca_dma_release() when you are done using a DMA channel that was previously allocated with ca_dma_alloc(). This function disables the DMA IRQ and updates the global usage state, making the channel available for reuse. It is typically called after completing a DMA transfer or when cleaning up resources before shutting down a DMA-based peripheral.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_dma_release() with caOS types?
A: ca_dma_ch_t ch = ca_dma_alloc(DMA1, DMA_CHANNEL_1); // Assume allocation succeeds // ... perform DMA operations ... ca_dma_release(ch); // Release the channel after use

Parameters conf:0.90 **Q:** What does the 'ch' parameter in ca_dma_release() represent, and what are valid values?
A: The 'ch' parameter is the DMA channel to be released. It must be a valid channel previously returned by ca_dma_alloc(). Valid values are any channel handle returned by ca_dma_alloc(), such as DMA1_Channel1 or DMA2_Channel3. Invalid values include NULL or any value outside the valid DMA channel range, which may lead to undefined behavior.

Return conf:0.95 **Q:** Does ca_dma_release() return any value, and what does it signify?
A: The function does not return any value. It is void. While it is designed not to fail in a strict sense, calling it with an invalid or unallocated channel may result in undefined behavior, such as memory access violations or incorrect state updates.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_dma_release()?
A: Common mistakes include calling ca_dma_release() with a channel that was never allocated, or passing NULL or invalid channel values. Additionally, calling this function from an ISR is not recommended as it uses a spinlock that could lead to deadlock. Also, calling it multiple times on the same channel may cause unpredictable behavior since it modifies a global state variable without checks.

`ca_dma_set_callback`

Header: `caos/drivers/inc/dma.h:137`
Source: `caos/drivers/src/dma.c:190:197`
GUARD: `EN_DMA`

```
bool ca_dma_set_callback(ca_dma_ch_t ch, ca_dma_cb_t cb, void *arg)
```

Registers a user-defined function to be executed automatically when a DMA transfer completes.

Parametri:

Nome	Tipo	Descrizione
ch	ca_dma_ch_t	The specific DMA channel handle. Must be a valid channel previously identified by the system.
cb	ca_dma_cb_t	A pointer to the callback function to execute. Pass NULL to disable the callback for this channel.
arg	void *	A pointer to user data (e.g., a struct or peripheral instance) that will be passed back to the callback function when it triggers.

Ritorna:
Returns true if the callback was successfully registered. Returns false if the channel index is invalid or out of range.

Note:

- This function is non-blocking and thread-safe for registration, but the callback itself will be executed within the context of the DMA interrupt service routine (ISR). Because the callback runs in an ISR, you must keep the code inside it extremely short and efficient; never perform blocking operations like waiting for a semaphore or mutex inside the callback. A common beginner mistake is passing a pointer to a local variable (on the stack) as the 'arg' parameter; if the function returns before the DMA finishes, that memory will be invalid, leading to a crash or garbage data. Always ensure the 'arg' points to static or heap-allocated memory that persists for the duration of the DMA transfer.

Uso:

- This function is typically used during the initialization phase of a peripheral driver, such as UART or SPI, to handle asynchronous data completion. You pair this with a DMA request setup function to define the transfer, then call this to register the 'done' handler. Once the DMA hardware finishes moving the data, the system automatically triggers the registered callback, allowing you to process the received data or signal a task that the buffer is ready. It is the standard way to move from a polling-based architecture to an event-driven one, which is much more power-efficient.

Esempio:

```
void my_dma_done_handler(void *arg) { /* This runs in ISR context */
    USART_TypeDef *uart = (USART_TypeDef *)arg; /* Process data here */ } void
    setup_uart_dma(ca_dma_ch_t ch) { if (!ca_dma_set_callback(ch,
    my_dma_done_handler, USART1)) { /* Handle registration error */ } /* Proceed
     to start DMA transfer */ }
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_dma_set_callback in my driver development?
A: You should use this function during the initialization phase of a peripheral driver, such as UART or SPI, to transition from a polling-based architecture to an event-driven one. It is used to register a handler that executes automatically upon DMA transfer completion, allowing for more power-efficient asynchronous data processing.

Code example conf:0.90 **Q:** How do I register a callback for a DMA channel in caOS?
A: You must define a callback function and pass it to ca_dma_set_callback along with a pointer to persistent data. Example: void my_cb(void *arg) { /* process data */ } ... ca_dma_set_callback(ch1, my_cb, &my_static_struct);

Parameters conf:1.00 **Q:** What are the requirements for the 'arg' parameter in ca_dma_set_callback?
A: The 'arg' parameter is a pointer to user data passed back to the callback. It must point to static or heap-allocated memory that persists for the entire duration of the DMA

transfer. Passing a pointer to a local stack variable is a common mistake that leads to crashes or garbage data if the function returns before the transfer completes.

Return conf:1.00 **Q:** What does the return value of ca_dma_set_callback indicate?
A: The function returns true if the callback was successfully registered. It returns false if the provided ca_dma_ch_t channel index is invalid or out of range.

`ca_dma_set_request`

Header: `caos/drivers/inc/dma.h:99`
Source: `caos/drivers/src/dma.c:130:135`
GUARD: `EN_DMA`

```
bool ca_dma_set_request(ca_dma_ch_t ch, u16 request)
```

Configures the hardware request signal (DMAMUX) for a specific DMA channel.

Parametri:

Nome	Tipo	Descrizione
`ch`	`ca_dma_ch_t`	The DMA channel handle previously acquired via ca_dma_free_channel or ca_dma_free_channel_index.
`request`	`u16`	The hardware request ID (e.g., a peripheral-specific trigger ID) defined in the STM32 reference manual for the specific peripheral.

Ritorna:
Returns true if the request was successfully mapped to the channel, false if the channel handle is invalid or the hardware configuration failed.

Note:

- This function is non-blocking and executes quickly, making it safe to call from most contexts, though it relies on the underlying hardware abstraction layer. It is not thread-safe in the sense that it does not lock the channel; you must ensure you have exclusive ownership of the channel handle before calling this. A common beginner mistake is attempting to set the request before successfully acquiring a channel handle, which will result in a false return value. Always verify the return value; if it returns false, the DMA transfer will not trigger because the hardware multiplexer (DMAMUX) is not correctly routed to the peripheral.

Uso:

- This function is used during the initialization phase of peripheral drivers (like UART or SPI) to link a specific DMA channel to a hardware trigger source. It is typically called immediately after acquiring a free DMA channel and before starting any data transfer. The standard sequence is: 1) Acquire a channel using ca_dma_free_channel, 2) Configure the request mapping using ca_dma_set_request, 3) Configure the transfer parameters, and 4) Start the transfer. If the setup fails, you must release the channel using ca_dma_release to prevent resource leaks.

Esempio:

```
ca_dma_ch_t rx_channel;
// Attempt to get a free DMA channel
if (ca_dma_free_channel(&rx_channel) >= 0) {
    // Map the hardware request to this channel
    // Note: 0x01 is a placeholder for the specific peripheral request ID
```

```
    if (ca_dma_set_request(rx_channel, 0x01)) {
        // Proceed to start the transfer
        // ca_dma_copy_inc(rx_channel, src, dst, size);
    } else {
        // Configuration failed, release the channel
        ca_dma_release(rx_channel);
    }
} else {
    // Handle error: no DMA channels available
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_dma_set_request in my peripheral driver initialization?

A: You should call it during the initialization phase, immediately after acquiring a DMA channel handle and before configuring transfer parameters or starting the transfer. It is used to link the DMA channel to a specific hardware trigger source via the DMAMUX.

Code example conf:0.95 **Q:** How do I correctly implement the DMA request mapping in C using caOS types?

A: After acquiring the channel, call the function as follows: ca_dma_ch_t my_ch = ca_dma_free_channel(); if (ca_dma_set_request(my_ch, 12)) { /* Proceed with transfer */ } else { ca_dma_release(my_ch); }

Parameters conf:1.00 **Q:** What does the 'request' parameter represent and where do I find its value?

A: The 'request' parameter is a u16 value representing the hardware request ID. This ID corresponds to the peripheral-specific trigger ID defined in the STM32 reference manual for your specific hardware peripheral.

Return conf:1.00 **Q:** What does a false return value from ca_dma_set_request indicate?

A: A false return indicates that the DMA channel handle is invalid or the hardware configuration failed. If this occurs, the DMAMUX is not correctly routed, and the DMA transfer will not trigger. You should release the channel using ca_dma_release to prevent resource leaks.

`ca_dma_start_circular`

Header: `caos/drivers/inc/dma.h:145`
Source: `caos/drivers/src/dma.c:166:172`
GUARD: `EN_DMA`

```
bool ca_dma_start_circular(ca_dma_ch_t ch, void *mem, volatile void *
    periph, u32 size, ca_dma_dir_t dir, ca_dma_width_t width)
```

Configures and initiates a DMA channel for continuous, repeating data transfers between memory and a peripheral.

Parametri:

Nome	Tipo	Descrizione
ch	ca_dma_ch_t	The specific DMA channel handle. Must be a valid channel assigned to the peripheral in the hardware configuration.
mem	void *	Pointer to the memory buffer. Must be aligned according to the data width and reside in a memory region accessible by the DMA controller.
periph	volatile void *	The peripheral register address (e.g., UART data register). Must be declared volatile to prevent compiler optimization.
size	u32	The number of data units to transfer per cycle. Must be greater than zero.
dir	ca_dma_dir_t	Direction of transfer: CA_DMA_MEM_TO_PERIPH or CA_DMA_PERIPH_TO_MEM.
width	ca_dma_width_t	Data width per transfer (e.g., 8-bit, 16-bit, or 32-bit). Must match the peripheral's data register size.

Ritorna:
Returns true if the DMA transfer was successfully started, false if the channel is invalid or the hardware failed to initialize the transfer.

Note:

- This function is non-blocking; it triggers the hardware DMA controller and returns immediately while the transfer happens in the background. It is thread-safe provided that the same channel is not accessed by multiple threads simultaneously. It can be called from an ISR, but ensure the DMA channel is not already active. A common beginner mistake is passing a non-volatile pointer for the peripheral register, which causes the compiler to optimize away necessary memory accesses, or providing a buffer that is not properly aligned for the selected data width, leading to hardware bus errors.

Uso:

- This function is used for high-speed, background data streaming where the CPU should not be involved in every byte transfer, such as continuous audio output or logging sensor data to a circular buffer. It is typically called during the initialization phase of a driver after the peripheral (like UART or SPI) has been configured. It is often paired with ca_dma_hw_get_remaining to monitor the progress of the circular buffer or to determine how much data has been processed since the last check.

Esempio:

```
/* Example: Setup circular DMA for UART transmission */
#define BUFFER_SIZE 128
u8 tx_buffer[BUFFER_SIZE];

void init_dma_transfer(void) {
    /* Configure DMA channel for memory-to-peripheral transfer */
    bool success = ca_dma_start_circular(DMA_CH1, tx_buffer, (void*)&USART1->DR,
     BUFFER_SIZE, CA_DMA_MEM_TO_PERIPH, CA_DMA_WIDTH_8);

    if (!success) {
        /* Handle error: DMA channel might be busy or invalid */
        while(1);
    }
    /* DMA is now running in the background */
```

```
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_dma_start_circular for audio or sensor data streaming?
A: You should use ca_dma_start_circular when you need continuous data transfer from a peripheral to memory, such as in audio streaming or sensor data acquisition where data arrives continuously and must be stored in a circular buffer. This function is ideal for applications requiring uninterrupted data flow without gaps.

Code example conf:0.90 **Q:** Can you show a minimal example of how to call ca_dma_start_circular with proper caOS types?
A: u8 buffer[1024]; ca_dma_ch_t ch = DMA1_Channel1; volatile u32 * peripheral = (volatile u32 *)0x40000000; bool result = ca_dma_start_circular(ch, buffer, (void *)peripheral, 1024, DMA_FROM_PERIPH, DMA_WIDTH_32BIT); if (!result) { // Handle error }

Parameters conf:0.90 **Q:** What are the valid values for the 'dir' parameter and what happens if I pass an invalid one?
A: The 'dir' parameter must be either DMA_TO_PERIPH or DMA_FROM_PERIPH. Passing any other value will cause the function to return false, indicating an invalid configuration and failure to start the transfer.

Return conf:0.95 **Q:** What does it mean when ca_dma_start_circular returns false, and what causes this failure?
A: When ca_dma_start_circular returns false, it indicates that the DMA transfer failed to start. This can be caused by invalid parameters such as a NULL pointer, an invalid DMA channel, a size that is not a multiple of the transfer width, or a channel already in use. The function also checks for valid direction and width values.

`ca_dma_stop_circular`

Header: `caos/drivers/inc/dma.h:150`
Source: `caos/drivers/src/dma.c:174:177`
GUARD: `EN_DMA`

```
void ca_dma_stop_circular(ca_dma_ch_t ch)
```

Stops a circular DMA transfer and releases the associated hardware channel for other tasks.

Parametri:

Nome	Tipo	Descrizione
`ch`	`ca_dma_ch_t`	A structure containing the controller and stream identifiers. Must be a valid channel previously acquired by the system.

Ritorna:
void

Note:

- This function is not inherently blocking, but it triggers a resource release that makes the DMA channel available for other system processes. It is critical to understand that calling this function immediately halts the circular data flow; if your application logic expects the buffer to be fully processed or synchronized, you must ensure the data is handled before calling this. It is generally safe to call from most contexts, but ensure that no other thread is attempting to configure this specific channel simultaneously to avoid race conditions. A

common beginner mistake is calling this function while the DMA is still actively writing to a memory buffer that the CPU is currently reading, which leads to data corruption or partial updates.

Uso:

- This function is used when a continuous data stream, such as audio sampling or sensor data logging, needs to be terminated gracefully. It is typically paired with a preceding 'ca_dma_start_circular' call that initialized the transfer. In the lifecycle of a DMA operation, you use this function during the shutdown phase or when reconfiguring the peripheral for a different task. It acts as the cleanup mechanism to ensure the hardware channel is not left in a 'busy' state, allowing other parts of the firmware to request the same DMA resource later without conflict.

Esempio:

```
/* Assuming ch is a valid DMA channel structure */
ca_dma_ch_t my_dma_ch = {0, 1};

// Stop the circular transfer when the task is finished
ca_dma_stop_circular(my_dma_ch);

// After stopping, the channel is released and can be re-acquired or left idle
// Ensure no further access to the memory buffer occurs after this point
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_dma_stop_circular() in my embedded application?
A: You should call ca_dma_stop_circular() when you want to halt a DMA channel that was previously started in circular mode using ca_dma_start_circular(). This function is typically used to gracefully stop data transfers, release hardware resources, and prevent further DMA activity on the specified channel.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_dma_stop_circular() with caOS types?
A: ca_dma_ch_t ch = { .periph = DMA1_Channel1, .stream = 0 }; ca_dma_start_circular(ch, buffer, size); // ... later on ... ca_dma_stop_circular(ch);

Parameters conf:0.90 **Q:** What does the 'ch' parameter represent in ca_dma_stop_circular(), and what are valid values?
A: The 'ch' parameter is a ca_dma_ch_t structure that identifies the specific DMA channel to be stopped. Valid values are initialized and active DMA channels in circular mode. Invalid values include NULL or channels that have not been initialized with ca_dma_init() or were not started with ca_dma_start_circular().

`ca_dma_wait`

Header: `caos/drivers/inc/dma.h:136`
Source: `caos/drivers/src/dma.c:188:197`
GUARD: `EN_DMA`

```
bool ca_dma_wait(ca_dma_ch_t ch, u32 timeout_ms)
```

Blocks the current task until a DMA transfer completes or the specified timeout expires.

Parametri:

Nome	Tipo	Descrizione
ch	ca_dma_ch_t	The handle of the DMA channel to monitor. Must be a valid channel previously initialized.
timeout_ms	u32	Time to wait in milliseconds. Use CA_WAIT_FOREVER to block indefinitely until the transfer finishes.

Ritorna:
Returns true if the DMA transfer completed successfully within the timeout. Returns false if the timeout expired or the channel handle is invalid.

Note:

- This function is blocking; it puts the calling task into a waiting state, allowing the RTOS scheduler to run other tasks while the hardware performs the data transfer. It is NOT callable from an ISR because it performs a blocking wait on a semaphore. Ensure the DMA channel has been properly initialized and the transfer started before calling this. A common mistake is setting a timeout that is too short for the amount of data being transferred, causing the function to return false prematurely while the hardware is still busy.

Uso:

- This function is typically used in a sequence: first, you initiate a DMA transfer (e.g., using ca_dma_copy_inc or a peripheral-specific DMA function), then you call ca_dma_wait to synchronize the task with the hardware completion. It is essential for ensuring that memory buffers are fully populated or transmitted before the application attempts to read or modify them. It is often paired with error handling logic to stop or reset the peripheral if the DMA fails to complete within the expected time window.

Esempio:

```c
/* Example: Copying memory using DMA and waiting for completion */
ca_dma_ch_t ch = /* Assume channel is already allocated */;
u32 src[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
u32 dst[10];

/* Start the DMA transfer */
ca_dma_copy_inc(ch, src, dst, 10U, 4U);

/* Wait for completion with a 50ms timeout */
if (ca_dma_wait(ch, 50U)) {
    /* Success: data is now in dst */
} else {
    /* Timeout: handle error, e.g., stop the DMA channel */
    /* ca_dma_stop_circular(ch); // If applicable */
    ca_printf_n("DMA transfer timed out!");
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_dma_wait() and what is its purpose in a DMA transfer?
A: You should call ca_dma_wait() after initiating a DMA transfer to block the current task until the DMA channel has completed the data transfer. It is used for synchronization, ensuring that the task waits for the DMA operation to finish before proceeding. This function is particularly useful when you need to guarantee that data has been fully transferred before continuing with subsequent operations.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_dma_wait() with caOS types?

A: u8 buffer[100]; ca_dma_ch_t ch = 0; ca_dma_copy_inc(ch, (void*)0x1000, buffer, 100); ca_dma_wait(ch);

Parameters conf:0.90 **Q:** What does the 'ch' parameter in ca_dma_wait() represent, and what are valid values for it?

A: The 'ch' parameter in ca_dma_wait() represents the identifier of the DMA channel to wait for. It must be a valid value between 0 and DMA_MAX_CHANNELS-1. If an invalid value is passed, the function will do nothing and may not return, as it relies on a semaphore for synchronization that may not be properly initialized for invalid channels.

`ca_dns_resolve`

Header: `caos/net/inc/dns_service.h:108`
Source: `caos/net/src/dns_service.c:246:278`
GUARD: `EN_DNS_SERVICE`

```
1 bool ca_dns_resolve(const char *hostname, ca_ipv4_t *out, u32 timeout_ms
    )
```

Resolves a human-readable hostname into an IPv4 address using the system DNS service.

Parametri:

Nome	Tipo	Descrizione
`hostname`	`const char *`	Pointer to the null-terminated string containing the domain name to resolve (e.g., 'google.com'). Must not be NULL.
`out`	`ca_ipv4_t *`	Pointer to an ca_ipv4_t structure where the resolved IP address will be stored. Must not be NULL.
`timeout_ms`	`u32`	Maximum time in milliseconds to wait for the DNS server to respond before giving up.

Ritorna:
Returns true if the hostname was successfully resolved and the IP address is valid. Returns false if the input parameters are invalid, the DNS slot table is full, the request queue is full, or the operation timed out.

Note:

- This function is blocking; it puts the calling thread into a waiting state until the DNS response arrives or the timeout expires. It is thread-safe due to internal mutex protection, but it must NOT be called from an Interrupt Service Routine (ISR) because blocking operations are forbidden in ISRs. A common mistake is setting the timeout too low for slow network connections, which causes the function to return false even if the DNS server is functional. Ensure the DNS service is initialized before calling this function, otherwise, the request queue will never be processed.

Uso:

- This function is typically used in network-enabled applications to convert domain names into IP addresses before initiating socket connections. It is usually called from a task context, often paired with CLI commands or initialization routines that require external server connectivity. The typical sequence involves declaring an ca_ipv4_t variable, calling ca_dns_resolve, and then checking the boolean return value to decide whether to proceed with network communication or handle the resolution failure.

Esempio:

```c
ca_ipv4_t server_ip;
const char *target = "example.com";

/* Attempt to resolve the hostname with a 5-second timeout */
if (ca_dns_resolve(target, &server_ip, 5000U)) {
    /* Success: server_ip now contains the resolved address */
    /* Proceed to open a socket or connect to the server */
} else {
    /* Failure: log the error or retry later */
    /* Do not use server_ip as it remains uninitialized or zeroed */
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_dns_resolve in a caOS application?
A: You should use ca_dns_resolve when you need to resolve a domain name (like 'google.com') into an IPv4 address in a blocking manner. It is typically used when setting up network connections where you have a hostname but need the actual IP address to connect to.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_dns_resolve with caOS types?
A: ca_ipv4_t ip; bool result = ca_dns_resolve("google.com", &ip, 5000); if (result) { // Use ip.address to connect to Google }

Parameters conf:0.90 **Q:** What are the valid and invalid values for the hostname parameter in ca_dns_resolve?
A: The hostname parameter must be a non-null C string (cstr) representing a valid domain name such as "google.com". If hostname is NULL, the function will fail immediately and return false.

Return conf:0.95 **Q:** What does it mean when ca_dns_resolve returns false?
A: ca_dns_resolve returns false if the DNS resolution failed due to one of the following reasons: hostname or out is NULL, no DNS slot is available, the timeout expired, or no response was received from the DNS server.

Error/Edge case conf:0.85 **Q:** What are common mistakes when calling ca_dns_resolve?
A: Common mistakes include passing a NULL hostname or output pointer, using a timeout that is too short, calling the function without an active network connection or configured DNS server, and attempting to call it from an ISR which is not safe.

Behavior conf:0.90 **Q:** Is ca_dns_resolve thread-safe and can it be called from an ISR?
A: Yes, ca_dns_resolve is thread-safe because it uses mutexes to protect shared resources. However, it is not safe to call from an ISR because it performs blocking operations like sem_timedwait, which could lead to system deadlocks or undefined behavior.

`ca_dns_service_bind`

Header: `caos/net/inc/dns_service.h:85`
Source: `caos/net/src/dns_service.c:220:223`
GUARD: `EN_DNS_SERVICE`

```c
bool ca_dns_service_bind(void)
```

Registers the DNS service to listen for incoming UDP packets on the standard DNS port.

Ritorna:
Returns true if the UDP port was successfully bound to the DNS handler, or false if the port is already in use or the network stack failed to allocate the socket.

Note:

- This function is non-blocking and performs the registration of a callback function (dns_udp_rx) that the network stack will trigger whenever a packet arrives on the DNS port. It is not thread-safe in the sense that it should only be called once during the system initialization phase; calling it multiple times may lead to resource exhaustion or undefined behavior in the network stack. It is not intended to be called from an ISR (Interrupt Service Routine) as it involves memory allocation and internal list management. A common beginner mistake is calling this before the network interface is actually initialized or ready, which will cause the binding to fail silently or return false.

Uso:

- This function is typically used during the system startup sequence, specifically after the network hardware has been initialized but before the main application loop begins. It is usually paired with other service binding functions like DHCP or NTP to configure the device's network capabilities. In the typical lifecycle, you call this once during the 'init' phase of your application; if it returns false, the DNS service will not function, and you should log an error or halt the system as the device will be unable to resolve hostnames.

Esempio:

```
/* Initialize the DNS service during system startup */
void app_network_init(void) {
    /* Attempt to bind the DNS port to the internal handler */
    if (ca_dns_service_bind()) {
        /* Success: DNS service is now listening for UDP traffic */
        ca_stdout_put_n("DNS service bound successfully.");
    } else {
        /* Failure: Likely port conflict or network stack not ready */
        ca_stdout_put_n("Error: Could not bind DNS service!");
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should ca_dns_service_bind() be called in a caOS application, and why is it important?
A: ca_dns_service_bind() should be called during system initialization, after the network system has been initialized but before the DNS service is activated. It is important because it registers the DNS service to receive UDP packets on the standard DNS port, enabling the system to respond to DNS queries. The function must be called in a safe context, not from an ISR, to avoid synchronization issues.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using caOS types that demonstrates how to call ca_dns_service_bind()?
A: bool result = ca_dns_service_bind(); if (result) { // DNS service successfully bound } else { // Handle DNS binding failure }

Parameters conf:0.85 **Q:** Does ca_dns_service_bind() accept any parameters, and what are the implications of calling it incorrectly?
A: ca_dns_service_bind() does not accept any parameters. Since it has no parameters, there are no valid or invalid values to consider. However, calling it incorrectly, such as multiple times or after the binding has already been established, can lead to unpredictable behavior or errors.

Return conf:0.90 **Q:** What do the return values of ca_dns_service_bind() indicate, and what causes the function to return false?
A: ca_dns_service_bind() returns true if the DNS service was successfully registered to receive UDP packets on the standard DNS port. It returns false if the registration failed, typically due to insufficient resources or if the port is already occupied. The function is not blocking but may fail if the system lacks the necessary resources for UDP binding.

ca_dns_service_init

Header: caos/net/inc/dns_service.h:67
Source: caos/net/src/dns_service.c:188:206
GUARD: EN_DNS_SERVICE

```
void ca_dns_service_init(ca_ipv4_t server_ip, ca_mac_t local_mac)
```

Initializes the internal DNS client service structures, semaphores, and mutexes required for hostname resolution.

Parametri:

Nome	Tipo	Descrizione
server_ip	ca_ipv4_t	The IPv4 address of the DNS server to be used for queries. Must be a valid network-order IP address.
local_mac	ca_mac_t	The MAC address of the local network interface. Used to identify the source of DNS packets.

Ritorna:
void (no return value).

Note:

- This function is non-blocking and performs initialization of internal RTOS primitives (mutexes and semaphores). It is NOT thread-safe and must be called exactly once during the system startup phase before any DNS resolution requests are made. It cannot be called from an ISR because it performs heap/OS object initialization. A common mistake is calling this function before the network stack or the underlying hardware interface is ready, which will lead to undefined behavior when the first DNS packet is sent.

Uso:

- This function is part of the network service initialization sequence. It is typically called in the main application setup or the system initialization task, alongside other network services like DHCP or NTP. It must be called before any DNS resolution functions are invoked. It pairs with the service start functions (e.g., ca_dns_service_start) to transition the service from an idle state to an operational state.

Esempio:

```
/* Define network parameters */
ipv4_t dns_server = {8, 8, 8, 8};
mac_t my_mac = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55};

/* Initialize the DNS service during system startup */
void system_init(void) {
    /* Initialize DNS service with server IP and local MAC */
    ca_dns_service_init(dns_server, my_mac);

    /* After init, the service can be started */
    /* ca_dns_service_start(); */
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_dns_service_init and why is it important for network operations?

A: You should call ca_dns_service_init during the initialization phase of your network application, before attempting any DNS resolution. It is important because it configures the DNS server IP and the local MAC address, which are required for sending DNS queries over the network. Without this initialization, DNS resolution will not work correctly.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_dns_service_init with caOS types?
A: u8 dns_server_ip[4] = {8, 8, 8, 8}; mac_t local_mac = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55}; ca_dns_service_init((ca_ipv4_t)dns_server_ip, local_mac);

Parameters conf:0.90 **Q:** What do the parameters server_ip and local_mac represent, and what are valid values for them?
A: The server_ip parameter is the IPv4 address of the DNS server to which DNS queries will be sent. It must be a valid ca_ipv4_t type, which is an array of 4 u8 values. The local_mac parameter is the MAC address of the local device, represented as a 6-byte array of type ca_mac_t. Invalid values include null pointers or arrays with fewer than 6 bytes, which may cause undefined behavior or memory access violations.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when calling ca_dns_service_init?
A: Common mistakes include passing invalid or uninitialized ca_ipv4_t or ca_mac_t parameters, such as null pointers or arrays with incorrect sizes. Additionally, calling this function without ensuring that internal resources like mutexes or semaphores are properly initialized may lead to failures, although the function itself does not return an error code.

Behavior conf:0.90 **Q:** Is ca_dns_service_init blocking or non-blocking, and can it be called from an ISR?
A: ca_dns_service_init is a non-blocking function and is not safe to call from an ISR. It is thread-safe, meaning it can be safely called from multiple threads, but it must be called from a thread context and not from interrupt handlers.

`ca_dns_service_net_ready`

Header: `caos/net/inc/dns_service.h:91`
Source: `caos/net/src/dns_service.c:225:229`
GUARD: `EN_DNS_SERVICE`

```
1 void ca_dns_service_net_ready(void)
```

Signals that the network stack is initialized and the DNS service is ready to process requests.

Ritorna:
void

Note:

- This function is non-blocking and thread-safe, designed to ensure the internal semaphore s_net_ready_sem is in a signaled state. It performs a 'trywait' followed immediately by a 'post' to guarantee the semaphore is available for other tasks waiting on network readiness. It is not intended to be called from an ISR, as it manages synchronization primitives that should be handled in task context. A common mistake is calling this before the underlying network interface (like DHCP or static IP configuration) has actually completed its link-up process, which may lead to DNS queries being sent into a void.

Uso:

- This function is typically called during the network initialization sequence, specifically after the IP address, gateway, and DNS server addresses have been successfully assigned. It acts as a synchronization barrier that notifies other application tasks that the DNS

service is now operational and ready to resolve hostnames. It is usually paired with ca_dns_service_bind() during the startup phase and is checked by application tasks that need to perform network lookups before proceeding with their main logic.

Esempio:

```c
/* Initialize network and signal DNS readiness */
void app_network_init(void) {
    /* Bind the UDP port for DNS traffic */
    if (ca_dns_service_bind()) {
        /* Configure the DNS server IP address */
        ca_dns_service_set_server(my_dns_ip);

        /* Signal that the DNS service is ready for use */
        ca_dns_service_net_ready();
    } else {
        /* Handle binding error */
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_dns_service_net_ready() and why is it important for DNS operations?
A: You should call ca_dns_service_net_ready() when the network interface is ready to start DNS operations. This function notifies the DNS service that the network is available, allowing it to proceed with DNS queries. It is important because it synchronizes access to shared resources using semaphores, ensuring that DNS operations can safely begin once the network is up and running.

Code example conf:0.90 **Q:** Can you provide a minimal C example using caOS types that demonstrates how to call ca_dns_service_net_ready()?
A: void example_dns_net_ready(void) { ca_dns_service_net_ready(); }

Return conf:0.85 **Q:** What happens if ca_dns_service_net_ready() is called incorrectly, and what causes failures in this function?
A: The function does not return any value. However, failures can occur if the function is called when the semaphore is not properly initialized. Since the function uses trywait and post operations on a semaphore, improper handling of the semaphore state can lead to unpredictable behavior. It is designed to be non-blocking and thread-safe, but incorrect usage may cause issues if the semaphore is not correctly set up before calling the function.

`ca_dns_service_rx_handler`

Header: `caos/net/inc/dns_service.h:97`
Source: `caos/net/src/dns_service.c:231:244`
GUARD: `EN_DNS_SERVICE`

```c
void ca_dns_service_rx_handler(const ca_udp_frame_t *udp, u16 len)
```

Processes incoming DNS UDP packets and signals the waiting task if a matching transaction ID is found.

Parametri:

Nome	Tipo	Descrizione
udp	const ca_udp_frame_t *	Pointer to the raw UDP frame structure containing the DNS payload. Must be a valid, non-null pointer to a received network buffer.
len	u16	The total length of the UDP payload in bytes. Used for internal validation, though currently ignored by the implementation.

Ritorna:
void

Note:

- This function is designed to be called from the network stack's receive context, which is often an ISR or a high-priority system thread. It is thread-safe regarding the semaphore signaling, but it relies on the global s_slots array, which must be protected if modified outside this handler. A common beginner mistake is assuming this function performs the DNS query itself; it does not, it only handles the response. If the DNS response is malformed or does not match any pending transaction ID, the function silently returns, which can make debugging difficult if you do not verify your network traffic with a sniffer.

Uso:

- This function is the core callback for the DNS subsystem, typically invoked by the network driver's receive loop when a UDP packet arrives on port 53. It is used in a sequence where a task first initiates a request (storing a transaction ID in s_slots), then blocks on a semaphore, and finally this handler wakes that task up upon receiving the matching response. It is usually paired with ca_net_buf_free to release the memory buffer after processing is complete.

Esempio:

```
/* Inside the network receive interrupt or task loop */
void on_packet_received(u8 *frame, u16 total_len) {
    /* Calculate UDP offset and cast to frame structure */
    u16 l4_off = 42; // Example offset
    const ca_udp_frame_t *udp = (const ca_udp_frame_t *)(frame + l4_off);

    /* Process the DNS response and signal the waiting task */
    ca_dns_service_rx_handler(udp, total_len - l4_off);

    /* Free the buffer back to the pool after the handler is done */
    ca_net_buf_free(current_buf_idx);
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_dns_service_rx_handler in my caOS project?
A: You should call ca_dns_service_rx_handler when a DNS response packet is received over UDP. It is typically invoked from a UDP receive handler or interrupt service routine that processes incoming network frames. This function notifies tasks waiting for DNS responses via a semaphore.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_dns_service_rx_handler with caOS types?
A:

```
ca_udp_frame_t frame;
ca_dns_service_rx_handler(&frame, 512);
```

Parameters conf:0.95 **Q:** What is the significance of the udp parameter in ca_dns_service_rx_handler, and what happens if it is NULL?

A: The udp parameter points to the UDP frame containing the DNS response data. It must be a valid pointer; passing NULL will cause a crash because the function does not perform NULL checks. The len parameter is used for validation but not processed internally.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_dns_service_rx_handl

A: Common mistakes include calling the function without initializing the semaphore used for notification, or invoking it from an ISR without ensuring that blocking behavior is acceptable. The function does not fail in a strict sense, but may silently ignore invalid packets if not properly validated externally.

`ca_dns_service_set_server`

Header: `caos/net/inc/dns_service.h:73`
Source: `caos/net/src/dns_service.c:208:212`
GUARD: `EN_DNS_SERVICE`

```
void ca_dns_service_set_server(ca_ipv4_t server_ip)
```

Configures the IPv4 address of the DNS server used by the caOS network stack.

Parametri:

Nome	Tipo	Descrizione
`server_ip`	`ca_ipv4_t`	A structure containing the 4-byte IPv4 address of the DNS server. Ensure the address is in network byte order if required by your specific network configuration.

Ritorna:
void

Note:

- This function is non-blocking and executes almost instantaneously as it only updates a static variable in memory. It is thread-safe in the sense that it performs a simple assignment, but it does not implement mutex locking, so concurrent calls from multiple threads could lead to race conditions if not managed. It is safe to call from an ISR, though typically it is called from the main application or CLI tasks. A critical side effect is that it sets 's_have_server_mac' to false, which forces the network stack to re-resolve the MAC address of the DNS server via ARP before the next DNS query can be sent. The most common mistake is calling this function with an uninitialized or zeroed IP address, which will cause all subsequent DNS resolution attempts to fail silently.

Uso:

- This function is used during the network initialization phase or when a network configuration change occurs, such as receiving new DNS settings from a DHCP server. It is typically paired with 'ca_dns_service_net_ready()' to signal that the stack is prepared to perform lookups. In a CLI context, it is used to allow users to manually override or set the DNS server address dynamically. It acts as the primary configuration point for the DNS client, ensuring that the stack knows exactly where to route resolution requests.

Esempio:

```
/* Example: Setting the DNS server manually during system startup */
ipv4_t my_dns_server;
my_dns_server.v[0] = 8;
my_dns_server.v[1] = 8;
```

```
my_dns_server.v[2] = 8;
my_dns_server.v[3] = 8;

/* Configure the DNS service with the new IP */
ca_dns_service_set_server(my_dns_server);

/* Signal that the network is ready to use this configuration */
ca_dns_service_net_ready();

/* Now the system is ready to resolve hostnames using the specified server */
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_dns_service_set_server and why is it useful in a networked embedded application?
A: You should call ca_dns_service_set_server when you need to configure or update the DNS server IP address for domain name resolution in your embedded application. It is useful when you want to manually set a specific DNS server, such as Google's public DNS (8.8.8.8) or a local DNS server, rather than relying on DHCP-provided DNS settings. This function allows you to define which DNS server will be used for future DNS queries, but it does not validate if the server is reachable or immediately affect network behavior.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_dns_service_set_server with caOS types?
A: u32 dns_ip = 0x08080808; /* 8.8.8.8 */ ca_dns_service_set_server(dns_ip);

Parameters conf:0.90 **Q:** What does the server_ip parameter in ca_dns_service_set_server represent, and what are valid and invalid values?
A: The server_ip parameter represents the IPv4 address of the DNS server in 32-bit format. A valid value is any 32-bit unsigned integer representing a valid IPv4 address, such as 0x08080808 for 8.8.8.8. An invalid value is 0, which indicates that the DNS server is not configured. The function does not validate the IP address for reachability or correctness beyond ensuring it is a valid 32-bit value.

`ca_dns_service_start`

Header: `caos/net/inc/dns_service.h:79`
Source: `caos/net/src/dns_service.c:214:218`
GUARD: `EN_DNS_SERVICE`

```
void ca_dns_service_start(void)
```

Initializes and launches the background DNS client task within the caOS scheduler.

Ritorna:
This function returns void; it does not provide feedback on whether the task was successfully created, as the scheduler handles task allocation internally.

Note:

- This function is non-blocking; it simply registers a new job with the caOS scheduler and returns immediately. It is not thread-safe to call this from an Interrupt Service Routine (ISR) because it involves memory allocation and scheduler manipulation. Before calling this, you must ensure the network stack is initialized, and specifically, you should call ca_dns_service_set_server() to define the target DNS server IP. A common beginner mistake is calling this function multiple times in a loop, which will spawn multiple redundant DNS tasks and quickly exhaust the system's stack memory.

Uso:

- This function is typically called during the system initialization phase, often inside the main application startup sequence alongside other network services like DHCP or NTP. It is the entry point for the DNS client functionality, which allows the system to resolve human-readable hostnames into IP addresses. It is usually paired with ca_dns_service_set_server() to configure the server address before the task starts, and later with ca_dns_service_bind() to establish the network connection required for DNS queries.

Esempio:

```
/* Configure the DNS server IP before starting the service */
ipv4_t dns_server = {8, 8, 8, 8};
ca_dns_service_set_server(dns_server);

/* Start the DNS background task */
ca_dns_service_start();

/* Proceed to bind the service to the network interface */
if (ca_dns_service_bind()) {
    /* DNS service is ready to resolve hostnames */
} else {
    /* Handle binding failure */
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_dns_service_start() and why is it important to initialize the DNS server IP before calling it?
A: You should call ca_dns_service_start() after configuring the DNS server IP address and initializing the networking stack. It starts an RTOS task dedicated to handling DNS requests asynchronously. The function is non-blocking and can be called from any context, including ISRs. It's important to initialize the DNS server IP before calling it because the function may fail to initialize correctly if the IP is not set, as it relies on proper network configuration to function.

Code example conf:0.85 **Q:** Can you provide a minimal C example using caOS types that demonstrates how to properly use ca_dns_service_start()?
A: u32 dns_ip = 0x08080808; /* Google DNS */ ca_net_set_dns_ip(dns_ip); ca_dns_service_start();

Return conf:0.90 **Q:** What does ca_dns_service_start() return, and what causes it to fail?
A: The function does not return any value. It is a void function that starts an RTOS task. It cannot fail in a traditional sense, but it may not initialize correctly if the parameters passed to ca_os_job are incorrect, or if the DNS server IP has not been properly initialized. The most common cause of failure is calling the function without setting up the network configuration or DNS server IP address.

`ca_ecdsa_p256_keygen`

Header: `caos/crypto/inc/ecdsa_p256.h:72`
Source: `caos/crypto/src/ecdsa_p256.c:462:479`
GUARD: `EN_ECDSA_P256`

```
bool ca_ecdsa_p256_keygen(u8 *public_key, u8 *private_key)
```

Generates a new ECDSA P-256 public/private key pair using the system random number generator.

Parametri:

Nome	Tipo	Descrizione
public_key	u8 *	Pointer to a buffer of at least 64 bytes to store the generated public key. Must not be NULL.
private_key	u8 *	Pointer to a buffer of 32 bytes to store the generated private key. Must not be NULL.

Ritorna:
Returns true if the key pair was successfully generated. Returns false if the input pointers are NULL, if the RNG fails, or if the generated private key is mathematically invalid after multiple attempts.

Note:

- This function is thread-safe and relies on the underlying hardware RNG. It is not intended to be called from an ISR because it performs multiple iterations and memory operations that may take significant time. A common beginner mistake is failing to clear the private key buffer after use; always use ca_memset to zero out sensitive data when it is no longer needed to prevent memory leaks of cryptographic material. The function internally attempts to generate a valid key up to 8 times; if it fails, it ensures the private key buffer is zeroed out before returning false.

Uso:

- Used during the initialization phase of secure communication modules or identity verification tasks. It is typically paired with ca_ecdsa_p256_sign or ca_ecdsa_p256_verify to establish a cryptographic identity. Developers should call this once to generate keys, store them securely, and use ca_memset to wipe the buffers when the task is finished or the system is shutting down.

Esempio:

```
u8 pub[64];
u8 priv[32];

// Attempt to generate a new key pair
if (ca_ecdsa_p256_keygen(pub, priv)) {
    // Key generation successful, proceed to use keys
    // ... perform signing or verification ...
} else {
    // Handle error: generation failed
    ca_printf("Key generation failed!\n");
}

// Always clear sensitive private key data from memory after use
ca_memset(priv, 0, 32);
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_ecdsa_p256_keygen function in my caOS application?
A: This function is used during the initialization phase of secure communication modules or identity verification tasks to establish a cryptographic identity. It is typically paired with ca_ecdsa_p256_sign or ca_ecdsa_p256_verify.

Code example conf:1.00 **Q:** Provide a minimal C example for generating a key pair using caOS types.
A: u8 pub[64]; u8 priv[32]; if (ca_ecdsa_p256_keygen(pub, priv)) { /* Use keys */ ca_memset(priv, 0, 32); }

Parameters conf:1.00 **Q:** What are the requirements for the public_key and private_key buffers passed to ca_ecdsa_p256_keygen?

A: The public_key buffer must be at least 64 bytes, and the private_key buffer must be 32 bytes. Both pointers must not be NULL.

Return conf:1.00 **Q:** What do the boolean return values of ca_ecdsa_p256_keygen indicate?

A: True indicates successful generation. False indicates that input pointers were NULL, the hardware RNG failed, or the generated private key was mathematically invalid after 8 attempts.

Error/Edge case conf:1.00 **Q:** What is a common security mistake developers make when using this function?

A: A common mistake is failing to clear the private key buffer after use. Developers should always use ca_memset to zero out sensitive data when it is no longer needed to prevent memory leaks of cryptographic material.

Behavior conf:1.00 **Q:** Is ca_ecdsa_p256_keygen safe to call from an Interrupt Service Routine (ISR)?

A: No, it is not intended to be called from an ISR because it performs multiple iterations and memory operations that may take significant time. However, the function is thread-safe.

`ca_ecdsa_p256_public_from_private`

Header: `caos/crypto/inc/ecdsa_p256.h:64`
Source: `caos/crypto/src/ecdsa_p256.c:438:460`
GUARD: `EN_ECDSA_P256`

```
bool ca_ecdsa_p256_public_from_private(u8 *public_key, const u8 *
    private_key)
```

Derives a P-256 public key from a given private key using elliptic curve point multiplication.

Parametri:

Nome	Tipo	Descrizione
`public_key`	`u8 *`	Pointer to a buffer of at least 64 bytes where the resulting X and Y coordinates will be stored.
`private_key`	`const u8 *`	Pointer to a 32-byte buffer containing the private key scalar. Must be a valid, non-zero scalar for the P-256 curve.

Ritorna:
Returns true if the public key was successfully derived, false if the input pointers are NULL or the private key is invalid (e.g., zero or out of range).

Note:

- This function is computationally intensive as it performs elliptic curve point multiplication. It is thread-safe but should not be called from an ISR due to its execution time. The function automatically clears sensitive intermediate data from the stack before returning, but the caller is responsible for zeroing out the private key buffer if it is no longer needed. A common mistake is providing a buffer smaller than 64 bytes for the public key, which will cause a memory corruption.

Uso:

- This function is typically used during key generation or when verifying that a stored private key corresponds to a specific public key. It is often paired with ca_keys_store_temp_enable_access to retrieve a key from secure storage, followed by this function to validate or export the public component, and finally ca_keys_store_lock_completely to re-secure the storage.

It is essential for ensuring that the private key loaded into memory is mathematically valid before attempting cryptographic operations like signing.

Esempio:

```c
u8 private_key[32];
u8 public_key[64];

// Assume private_key is populated from secure storage
// Enable access to the key store to read the private key
ca_keys_store_temp_enable_access(key_handle, CA_KEYS_STORE_READ_ONLY);

// Derive the public key from the private key
if (ca_ecdsa_p256_public_from_private(public_key, private_key)) {
    // Success: public_key now contains the 64-byte coordinate pair
} else {
    // Handle error: invalid key or null pointer
    ca_printf("Failed to derive public key\n");
}

// Always lock the store after use
ca_keys_store_lock_completely(key_handle);

// Clear sensitive data from stack
ca_memset(private_key, 0, 32);
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_ecdsa_p256_public_from_private in my caOS application?
A: This function is used to derive a P-256 public key from a private key scalar. It is typically used during key generation or to validate that a stored private key corresponds to a specific public key before performing cryptographic operations like signing.

Code example conf:1.00 **Q:** Provide a minimal C example for deriving a public key using caOS types.
A: u8 priv[32] = { /* ... */ }; u8 pub[64]; if (ca_ecdsa_p256_public_from_private(pub, priv)) { /* success */ }

Parameters conf:1.00 **Q:** What are the requirements for the private_key and public_key parameters?
A: The private_key must be a 32-byte buffer containing a valid, non-zero scalar for the P-256 curve. The public_key must be a buffer of at least 64 bytes to store the resulting X and Y coordinates.

Return conf:1.00 **Q:** What do the return values of ca_ecdsa_p256_public_from_private indicate?
A: It returns true if the public key was successfully derived. It returns false if the input pointers are NULL or if the private key is invalid, such as being zero or out of the valid P-256 range.

Error/Edge case conf:1.00 **Q:** What are the common mistakes or edge cases to avoid when calling this function?
A: A common mistake is providing a buffer smaller than 64 bytes for the public key, which will cause memory corruption. Additionally, ensure the private key is not zero or out of range, as this will cause the function to return false.

Behavior conf:1.00 **Q:** Is ca_ecdsa_p256_public_from_private safe to call from an ISR or concurrent threads?
A: The function is thread-safe, but it is computationally intensive and should not be called from an ISR due to its execution time. It automatically clears sensitive intermediate data from the stack, but the caller must manually zero out the private key buffer if it is no longer needed.

ca_ecdsa_p256_selftest

Header: caos/crypto/inc/ecdsa_p256.h:111
Source: caos/crypto/src/ecdsa_p256.c:667:750
GUARD: EN_ECDSA_P256

```
u32 ca_ecdsa_p256_selftest(void)
```

Executes a comprehensive suite of ECDSA P-256 cryptographic self-tests to verify hardware/software integrity.

Ritorna:
Returns a u32 representing the total count of failed test cases. A return value of 0 indicates that all cryptographic operations (signing, verification, key generation, and negative testing) passed successfully.

Note:

- This function is a diagnostic tool designed to validate the ECDSA P-256 implementation against RFC 6979 test vectors. It is not thread-safe and should be executed during system initialization or as a dedicated maintenance task, as it performs heavy cryptographic computations that will block the calling thread for a significant duration. It must not be called from an ISR because it performs blocking operations and potentially uses heap memory or system resources that are not ISR-safe. The most common mistake is calling this function during normal operation, which will cause significant jitter or watchdog timeouts in other real-time tasks.

Uso:

- Use this function during the system boot-up sequence or within a dedicated diagnostic CLI command to ensure the cryptographic engine is functioning correctly before allowing secure communications. It is typically paired with ca_os_banner to log the start of the test and ca_stdout_printf_n to report the results. If the function returns a non-zero value, the system should enter a safe state or flag a hardware/firmware fault, as the cryptographic primitives cannot be trusted.

Esempio:

```
/* Diagnostic task to verify crypto integrity at startup */
void crypto_diagnostic_task(void *arg) {
    u32 failures;

    ca_stdout_put_n("Starting system crypto self-tests...");

    /* Run the P-256 suite and capture the number of failures */
    failures = ca_ecdsa_p256_selftest();

    if (failures == 0) {
        ca_stdout_put_n("Crypto self-test: SUCCESS");
    } else {
        ca_printf_n("Crypto self-test: FAILED with %u errors", failures);
        /* Enter a safe state if crypto is broken */
        while(1) {
            /* Trigger system reset or halt */
            ca_os_delay_ms(1000);
        }
    }
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to execute ca_ecdsa_p256_selftest in a caOS application?
A: This function should be called during the system boot-up sequence or within a dedicated diagnostic CLI command to ensure the cryptographic engine is functioning correctly before enabling secure communications.

Code example conf:1.00 **Q:** Provide a minimal C example showing how to invoke the self-test and handle the result.
A: u32 result = ca_ecdsa_p256_selftest(); if (result != 0) { /* Enter safe state */ }

Return conf:1.00 **Q:** What does the u32 return value of ca_ecdsa_p256_selftest signify?
A: The return value represents the total count of failed test cases. A return value of 0 indicates that all cryptographic operations, including signing, verification, key generation, and negative testing, passed successfully.

Error/Edge case conf:1.00 **Q:** What is the most common mistake developers make when using ca_ecdsa_p256_selftest?
A: The most common mistake is calling this function during normal operation, which will cause significant jitter or watchdog timeouts in other real-time tasks due to the heavy cryptographic computations involved.

Behavior conf:1.00 **Q:** Is ca_ecdsa_p256_selftest safe to call from an interrupt service routine (ISR) or in a multi-threaded environment?
A: No, it is not thread-safe and must not be called from an ISR because it performs blocking operations and potentially uses heap memory or system resources that are not ISR-safe.

`ca_ecdsa_p256_sign`

Header: `caos/crypto/inc/ecdsa_p256.h:87`
Source: `caos/crypto/src/ecdsa_p256.c:546:555`
GUARD: `EN_ECDSA_P256`

```
bool ca_ecdsa_p256_sign(u8 *signature, const u8 *message, u32
    message_len, const u8 *private_key)
```

Signs a message using the ECDSA P-256 algorithm by first hashing the input data.

Parametri:

Nome	Tipo	Descrizione
`signature`	`u8 *`	Pointer to a buffer of at least 64 bytes where the resulting signature (r and s components) will be stored.
`message`	`const u8 *`	Pointer to the raw data buffer to be signed. Can be NULL only if message_len is 0.
`message_len`	`u32`	Length of the message in bytes. Must be 0 if message is NULL.
`private_key`	`const u8 *`	Pointer to the 32-byte private key buffer. Must be a valid, non-zero private key for P-256.

Ritorna:
Returns true if the signature was generated successfully, false if the input parameters are invalid or the internal signing process failed.

Note:

- This function is a wrapper that performs a SHA-256 hash on the input message before calling the underlying signing primitive. It is not thread-safe if the same private key buffer is modified by another task during execution. It is generally not recommended to call this from an ISR due to the computational intensity of elliptic curve cryptography, which can cause significant latency in the system. Ensure the private key is properly initialized and stored in secure memory before calling this function; using an invalid or zeroed key will result in a failed signature.

Uso:

- This function is typically used in security protocols like TLS or digital signature verification to authenticate data. It is often paired with ca_ecdsa_p256_verify to confirm that the generated signature is valid for the given message and public key. In a typical lifecycle, you would load a private key from secure storage, call this function to sign a payload, and then transmit the signature alongside the message. Always check the boolean return value before proceeding to transmit or store the signature to avoid sending corrupted or empty data.

Esempio:

```
u8 signature[64];
u8 my_message[] = "Hello, Secure World!";
/* Assume priv_key is already loaded from secure storage */

if (ca_ecdsa_p256_sign(signature, my_message, sizeof(my_message) - 1, priv_key))
    {
    /* Signature generated successfully, proceed to send or store */
    ca_printf("Signature generated successfully.\n");
} else {
    /* Handle error: log failure or alert the system */
    ca_printf("Failed to sign message!\n");
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** In what scenarios should I use ca_ecdsa_p256_sign and what are the performance considerations?
A: This function is used for security protocols like TLS or digital signature verification to authenticate data. Because ECDSA P-256 is computationally intensive, it is not recommended to call this from an ISR as it can cause significant system latency. It should be used to sign payloads after loading a private key from secure storage.

Code example conf:1.00 **Q:** How do I implement a basic call to ca_ecdsa_p256_sign in caOS?
A: u8 sig[64]; u8 msg[] = {0x01, 0x02}; u8 key[32] = { /* initialized key */ }; if (ca_ecdsa_p256_sign(sig, msg, 2, key)) { /* success */ } else { /* handle failure */ }

Parameters conf:1.00 **Q:** What are the requirements for the private_key and message parameters?
A: The private_key must be a pointer to a 32-byte buffer containing a valid, non-zero private key. The message parameter points to the raw data buffer; it can be NULL only if message_len is 0. Using an invalid or zeroed key will cause the function to fail.

Return conf:1.00 **Q:** What does the boolean return value indicate and what causes a failure?
A: The function returns true if the signature was generated successfully. It returns false if the input parameters are invalid (such as a zeroed private key or mismatched message length) or if the internal signing process fails.

ca_ecdsa_p256_sign_hash

Header: caos/crypto/inc/ecdsa_p256.h:81
Source: caos/crypto/src/ecdsa_p256.c:481:544
GUARD: EN_ECDSA_P256

```
bool ca_ecdsa_p256_sign_hash(u8 *signature, const u8 *hash, const u8 *
    private_key)
```

Signs a 32-byte hash using an ECDSA P-256 private key.

Parametri:

Nome	Tipo	Descrizione
signature	u8 *	Pointer to a 64-byte buffer where the resulting signature (r and s components) will be stored.
hash	const u8 *	Pointer to the 32-byte hash of the message to be signed.
private_key	const u8 *	Pointer to the 32-byte private key. Must be a valid non-zero scalar less than the curve order.

Ritorna:
Returns true if the signature was generated successfully, false if any input is NULL, the private key is invalid, or the internal signing process failed.

Note:

- This function is computationally intensive and performs multiple modular arithmetic operations. It is not designed to be called from an ISR due to its execution time and stack usage. It is thread-safe provided that the input buffers are not modified by other tasks during execution. A common mistake is passing an uninitialized or invalid private key; always ensure the key has been generated or loaded correctly before calling this function. The function internally clears sensitive temporary data from the stack before returning, which is a security feature to prevent key leakage.

Uso:

- This function is typically used in digital signature workflows where a message must be authenticated. The standard sequence involves: 1) Generating or loading a private key, 2) Computing the hash of the message using ca_sha256, and 3) Calling this function to produce the signature. It is often paired with ca_ecdsa_p256_verify_hash on the receiving end to validate the authenticity of the signed data.

Esempio:

```
u8 hash[32];
u8 signature[64];
u8 private_key[32];

/* Assume private_key is already populated */
/* Compute hash of the message */
ca_sha256(message_data, message_len, hash);

/* Sign the hash */
if (ca_ecdsa_p256_sign_hash(signature, hash, private_key)) {
    /* Signature generated successfully */
} else {
    /* Handle error: invalid key or internal failure */
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** In what scenario should I use ca_ecdsa_p256_sign_hash?
A: This function is used in digital signature workflows to authenticate messages. The standard sequence is to generate or load a private key, compute the message hash using ca_sha256, and then call this function to produce the signature.

Code example conf:1.00 **Q:** Provide a minimal C example for signing a hash using caOS types.
A: u8 sig[64]; const u8 hash[32] = { /* ... */ }; const u8 priv[32] = { /* ... */ }; bool success = ca_ecdsa_p256_sign_hash(sig, hash, priv);

Parameters conf:1.00 **Q:** What are the requirements for the private_key parameter?
A: The private_key must be a pointer to a 32-byte buffer containing a valid non-zero scalar that is less than the curve order.

Return conf:1.00 **Q:** What does a return value of false indicate?
A: A return value of false indicates that the signature generation failed. This occurs if any input pointer is NULL, the private key is invalid, or the internal signing process failed.

Error/Edge case conf:1.00 **Q:** What is a common mistake when using this function?
A: A common mistake is passing an uninitialized or invalid private key. You must ensure the key has been generated or loaded correctly before calling the function.

Behavior conf:1.00 **Q:** Can I call ca_ecdsa_p256_sign_hash from an interrupt service routine (ISR)?
A: No, it is not designed to be called from an ISR due to its significant execution time and stack usage. It is computationally intensive and performs multiple modular arithmetic operations.

`ca_ecdsa_p256_test_init`

Header: `caos/crypto/inc/ecdsa_p256.h:116`
Source: `caos/crypto/src/ecdsa_p256.c:798:805`
GUARD: `EN_ECDSA_P256`

```
1 void ca_ecdsa_p256_test_init(void)
```

Registers the ECDSA P-256 test command handler into the caOS CLI system.

Ritorna:
This function returns nothing (void).

Note:

- This function is non-blocking and is intended to be called during system initialization. It is not thread-safe in the sense that it relies on a static flag to ensure the CLI node is registered only once; calling it from multiple threads simultaneously could theoretically cause a race condition, though it is typically only called once at startup. It is not safe to call from an ISR. The primary precondition is that the CLI system must be initialized and enabled via the appropriate configuration flags (EN_CLI_ECDSA_P256 and ENABLE_CLI). A common mistake is assuming this function performs the actual cryptographic tests; it only makes the test commands available in the CLI console.

Uso:

- This function is used during the system boot sequence to expose cryptographic self-tests to the command-line interface. It is typically paired with other *_test_init functions to populate the CLI with diagnostic tools. Once registered, a user can trigger the actual tests (like ca_ecdsa_p256_selftest) via the CLI console. It is part of the standard initialization pattern for diagnostic-heavy firmware builds.

Esempio:

```
void system_init_tasks(void) {
    /* Initialize the CLI system first */
    ca_cli_init();

    /* Register the ECDSA P-256 test commands so they are available in the shell */
    ca_ecdsa_p256_test_init();

    /* Register other cryptographic tests */
    ca_ed25519_test_init();
    ca_sha256_test_init();

    /* Now the user can run 'ecdsa_p256_selftest' in the CLI to execute ca_ecdsa_p256_selftest() */
}
```

Q&A — fine-tuning review *2/2 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_ecdsa_p256_test_init and what does it actually do?
A: You should call this function during the system boot sequence as part of the initialization pattern. It does not perform cryptographic tests itself; instead, it registers the ECDSA P-256 test command handler into the caOS CLI system, making the diagnostic commands available for the user to trigger via the console.

Code example conf:0.95 **Q:** How do I initialize the ECDSA P-256 test command in my caOS startup routine?
A: Ensure that the CLI system is enabled via the EN_CLI_ECDSA_P256 and ENABLE_CLI configuration flags, then call the function during system initialization as shown: void system_init(void) { ca_ecdsa_p256_test_init(); }

`ca_ecdsa_p256_verify`

Header: `caos/crypto/inc/ecdsa_p256.h:103`
Source: `caos/crypto/src/ecdsa_p256.c:610:619`
GUARD: `EN_ECDSA_P256`

```
bool ca_ecdsa_p256_verify(const u8 *signature, const u8 *message, u32 message_len, const u8 *public_key)
```

Verifies an ECDSA P-256 signature against a message using a provided public key.

Parametri:

Nome	Tipo	Descrizione
`signature`	`const u8 *`	Pointer to the 64-byte signature buffer (r and s components).
`message`	`const u8 *`	Pointer to the raw message data to be verified.
`message_len`	`u32`	Length of the message in bytes. Must be valid if message is not NULL.
`public_key`	`const u8 *`	Pointer to the public key buffer (typically 64 bytes for uncompressed coordinates).

Ritorna:
Returns true if the signature is valid for the given message and public key, false otherwise. Returns false if inputs are invalid (e.g., NULL message with non-zero length).

Note:

- This function is computationally intensive as it performs SHA-256 hashing followed by elliptic curve point arithmetic. It is thread-safe but should not be called from an ISR due to the high execution time which would block the system scheduler. Ensure the public key and signature buffers are correctly initialized and contain valid data before calling. A common mistake is passing an incorrect message length or an uninitialized signature buffer, which will result in a false return value even if the key is correct.

Uso:

- This function is typically used in secure communication protocols or firmware update verification processes to ensure data integrity and authenticity. It is often paired with ca_ecdsa_p256_sign (on the sender side) or ca_ecdsa_p256_verify_hash (if the hash is already computed). In a typical lifecycle, you would use it after receiving a signed payload to validate the sender's identity before processing the data.

Esempio:

```
/* Verify a received firmware signature */
const u8 *my_signature = get_signature_from_packet();
const u8 *my_message = get_data_from_packet();
u32 my_len = get_data_length();
const u8 *pub_key = get_trusted_public_key();

if (ca_ecdsa_p256_verify(my_signature, my_message, my_len, pub_key)) {
    /* Signature is valid, proceed with firmware update */
} else {
    /* Signature invalid, discard data and log security event */
    ca_printf("Security Alert: Invalid signature detected!\n");
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** In what scenarios should I utilize the ca_ecdsa_p256_verify function in my caOS application?
A: This function is intended for secure communication protocols or firmware update verification processes. You should call it after receiving a signed payload to validate the sender's identity and ensure data integrity before processing the data.

Code example conf:1.00 **Q:** Can you provide a minimal C example of how to call ca_ecdsa_p256_verify using caOS types?
A: u8 sig[64] = { /* ... */ }; u8 msg[] = "data"; u8 pub[64] = { /* ... */ }; bool valid = ca_ecdsa_p256_verify(sig, msg, 4, pub);

Parameters conf:1.00 **Q:** What are the requirements for the signature and public_key parameters in ca_ecdsa_p256_verify?
A: The signature parameter must point to a 64-byte buffer containing the r and s components. The public_key parameter should point to a buffer, typically 64 bytes for uncompressed coordinates. Both must be correctly initialized with valid data before the call.

Return conf:1.00 **Q:** What does a false return value from ca_ecdsa_p256_verify indicate?
A: A false return indicates that the signature is invalid for the given message and public key, or that the inputs were invalid (e.g., a NULL message with a non-zero length). Common causes for a false return include passing an incorrect message length or using an uninitialized signature buffer.

`ca_ecdsa_p256_verify_hash`

Header: `caos/crypto/inc/ecdsa_p256.h:97`
Source: `caos/crypto/src/ecdsa_p256.c:557:608`

GUARD: `EN_ECDSA_P256`

```
bool ca_ecdsa_p256_verify_hash(const u8 *signature, const u8 *hash,
    const u8 *public_key)
```

Verifies an ECDSA P-256 signature against a provided SHA-256 hash and public key.

Parametri:

Nome	Tipo	Descrizione
`signature`	`const u8 *`	Pointer to a 64-byte buffer containing the ECDSA signature (r and s components, 32 bytes each). Must not be NULL.
`hash`	`const u8 *`	Pointer to a 32-byte buffer containing the SHA-256 hash of the message. Must not be NULL.
`public_key`	`const u8 *`	Pointer to a 64-byte buffer containing the public key (x and y coordinates, 32 bytes each). Must not be NULL.

Ritorna:
Returns true if the signature is valid for the given hash and public key, false otherwise (including invalid inputs or failed verification).

Note:

- This function performs heavy mathematical operations (elliptic curve point multiplication and modular inversion) and is CPU-intensive. It is a blocking function, meaning the calling task will not yield until the calculation is complete; do not call this from an ISR or a high-priority real-time task where latency is critical. It is thread-safe as it uses local stack variables for all intermediate calculations, but ensure the input buffers are not modified by other tasks during execution. A common mistake is passing a raw message instead of the pre-computed SHA-256 hash; always ensure the hash is computed using ca_sha256 before calling this function.

Uso:

- This function is typically used in secure communication protocols or firmware update verification processes. It is usually paired with ca_sha256 to hash the incoming data first, followed by this function to verify the authenticity of the sender. In a typical lifecycle, you would receive a signed message, compute its hash, and then call this function to validate the signature against a known trusted public key stored in the device's secure storage.

Esempio:

```
/* Example: Verify a signature for a received message */
#include "caos.h"

void verify_incoming_data(const u8 *data, u32 len, const u8 *sig, const u8 *
    pub_key) {
    u8 hash[32];

    /* 1. Compute the hash of the received data */
    ca_sha256(data, len, hash);

    /* 2. Verify the signature using the computed hash */
    if (ca_ecdsa_p256_verify_hash(sig, hash, pub_key)) {
        ca_printf("Signature verified successfully!\n");
    } else {
        ca_printf("Signature verification failed!\n");
    }
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** In what scenario should I use ca_ecdsa_p256_verify_hash?
A: It is typically used for secure communication protocols or verifying firmware updates. You should use it to validate the authenticity of a sender by verifying a signature against a pre-computed SHA-256 hash and a trusted public key stored in secure storage.

Code example conf:1.00 **Q:** How do I implement a basic signature verification in caOS?
A: u8 sig[64] = {...}; u8 hash[32] = {...}; u8 pub[64] = {...}; bool valid = ca_ecdsa_p256_verify_hash(sig hash, pub); if (valid) { /* Signature is authentic */ }

Parameters conf:1.00 **Q:** What are the size requirements for the buffers passed to ca_ecdsa_p256_verify_hash?
A: The signature buffer must be 64 bytes (r and s components, 32 bytes each), the hash buffer must be 32 bytes (SHA-256), and the public key buffer must be 64 bytes (x and y coordinates, 32 bytes each). All pointers must not be NULL.

Return conf:1.00 **Q:** What does the boolean return value indicate?
A: It returns true if the signature is valid for the provided hash and public key. It returns false if the signature is invalid, if any input parameters are invalid, or if the verification process fails.

Error/Edge case conf:1.00 **Q:** What is a common mistake when preparing the input for this function?
A: A common mistake is passing a raw message instead of the pre-computed SHA-256 hash. You must always ensure the hash is computed using ca_sha256 before calling this function.

Behavior conf:1.00 **Q:** Is it safe to call ca_ecdsa_p256_verify_hash from an interrupt service routine (ISR)?
A: No. This is a blocking, CPU-intensive function that performs heavy mathematical operations. It should not be called from an ISR or any high-priority real-time task where latency is critical.

`ca_ed25519_keygen`

Header: `caos/crypto/inc/ed25519.h:70`
Source: `caos/crypto/src/ed25519.c:448:459`
GUARD: `EN_ED25519`

```
1 bool ca_ed25519_keygen(u8 *public_key, u8 *seed)
```

Generates a new Ed25519 public key and a corresponding random seed.

Parametri:

Nome	Tipo	Descrizione
`public_key`	`u8 *`	Pointer to a buffer of at least 32 bytes where the generated public key will be stored.
`seed`	`u8 *`	Pointer to a buffer of at least 32 bytes where the random seed used to derive the key will be stored.

Ritorna:
Returns true if the key generation was successful, false if the input pointers are NULL or if the random number generator (RNG) failed to provide a secure seed.

Note:

- This function is not thread-safe if the underlying RNG hardware is shared without internal locking. It is generally not safe to call from an ISR because it relies on the RNG, which may involve blocking operations or complex peripheral access. The function performs a

mandatory check on the RNG; if the RNG fails, it clears the seed buffer to prevent sensitive data leakage. A common mistake is passing uninitialized or too-small buffers, which will lead to memory corruption or invalid keys.

Uso:

- This function is typically used during the device provisioning or identity initialization phase. It is often paired with ca_ed25519_sign or ca_ed25519_verify to establish secure communication channels. Before calling this, ensure the system RNG is initialized via ca_rng_init. After generation, the seed should be stored in secure non-volatile memory or cleared from RAM using ca_memset if it is no longer needed.

Esempio:

```c
u8 my_public_key[32];
u8 my_seed[32];

/* Initialize RNG before generating keys */
ca_rng_init();

/* Generate the key pair */
if (ca_ed25519_keygen(my_public_key, my_seed)) {
    /* Key generation successful, proceed to use the key */
} else {
    /* Handle error: log failure or retry */
    ca_printf("Key generation failed!\n");
}

/* Clear sensitive seed from memory after use */
ca_memset(my_seed, 0, 32);
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_ed25519_keygen in my caOS application?
A: This function is typically used during the device provisioning or identity initialization phase to establish secure communication channels. It is often paired with ca_ed25519_sign or ca_ed25519_verify. Ensure the system RNG is initialized via ca_rng_init before calling it.

Code example conf:0.90 **Q:** Provide a minimal C example for generating an Ed25519 key pair.
A: u8 pub[32]; u8 seed[32]; ca_rng_init(); if (ca_ed25519_keygen(pub, seed)) { /* Success */ }

Parameters conf:1.00 **Q:** What are the requirements for the public_key and seed buffers passed to ca_ed25519_keygen?
A: Both public_key and seed must be pointers to buffers of at least 32 bytes. Passing uninitialized or smaller buffers will lead to memory corruption or invalid keys.

Return conf:1.00 **Q:** What do the boolean return values of ca_ed25519_keygen indicate?
A: Returns true if the key generation was successful. Returns false if the input pointers are NULL or if the random number generator (RNG) failed to provide a secure seed.

Error/Edge case conf:1.00 **Q:** What happens if the RNG fails during key generation, and can I call this from an ISR?
A: If the RNG fails, the function clears the seed buffer to prevent sensitive data leakage. It is generally not safe to call from an ISR because it relies on the RNG, which may involve blocking operations or complex peripheral access.

`ca_ed25519_public_from_seed`

Header: `caos/crypto/inc/ed25519.h:58`
Source: `caos/crypto/src/ed25519.c:428:446`

GUARD: `EN_ED25519`

```
void ca_ed25519_public_from_seed(u8 *public_key, const u8 *seed)
```

Derives an Ed25519 public key from a 32-byte private seed.

Parametri:

Nome	Tipo	Descrizione
`public_key`	`u8 *`	Pointer to a 32-byte buffer where the resulting public key will be stored. Must not be NULL.
`seed`	`const u8 *`	Pointer to the 32-byte secret seed. Must not be NULL.

Ritorna:
This function returns void. If either pointer is NULL, the function returns immediately without performing any operations.

Note:

- This function is synchronous and blocking, meaning it will consume CPU cycles until the mathematical derivation is complete. It is thread-safe as it uses local stack memory for intermediate calculations, but it is not recommended to call this from an ISR due to the computational intensity of SHA512 and scalar multiplication. The most common mistake is passing a buffer smaller than 32 bytes, which will cause a memory corruption (buffer overflow) when the function writes the result.

Uso:

- This function is typically used during the initialization phase of a secure communication module or when generating a new identity for the device. It is often paired with ca_ed25519_sign to generate signatures after the public key has been derived. In a typical workflow, you would allocate memory for the keys, derive the public key from a stored seed, and then proceed to use the key for authentication or data verification tasks.

Esempio:

```
u8 my_seed[32] = { /* 32 bytes of secret data */ };
u8 my_public_key[32];

// Ensure the seed is valid before derivation
// Assume 'seed_is_initialized' is a boolean check
if (seed_is_initialized) {
    ca_ed25519_public_from_seed(my_public_key, my_seed);

    // Log the result for debugging purposes
    ca_hexdump(my_public_key, 32);
} else {
    // Handle error: seed not ready
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_ed25519_public_from_seed in my caOS application?
A: You should use this function during the initialization phase of a secure communication module or when generating a new device identity. It is typically used to derive a public key from a stored 32-byte seed before performing operations like data verification or authentication using ca_ed25519_sign.

Code example conf:1.00 **Q:** Provide a minimal C example for deriving an Ed25519 public key using caOS types.
A: u8 seed[32] = { /* ... initialized seed ... */ }; u8 public_key[32]; ca_ed25519_public_from_seed(publ seed);

Parameters conf:1.00 **Q:** What are the requirements for the public_key and seed parameters?
A: Both parameters must be pointers to 32-byte buffers and must not be NULL. If either pointer is NULL, the function returns immediately without performing any operations.

Error/Edge case conf:1.00 **Q:** What is a common mistake when calling ca_ed25519_public_from_seed?
A: The most common mistake is passing a buffer smaller than 32 bytes for the public_key, which will cause memory corruption (buffer overflow) when the function writes the result.

Behavior conf:1.00 **Q:** Is it safe to call ca_ed25519_public_from_seed from an interrupt service routine (ISR)?
A: No, it is not recommended to call this function from an ISR. It is a synchronous, blocking function that consumes significant CPU cycles due to SHA512 and scalar multiplication, which could negatively impact system timing.

`ca_ed25519_selftest`

Header: `caos/crypto/inc/ed25519.h:108`
Source: `caos/crypto/src/ed25519.c:618:710`
GUARD: `EN_ED25519`

```
u32 ca_ed25519_selftest(void)
```

Performs a comprehensive suite of self-tests on the Ed25519 cryptographic implementation to verify correctness.

Ritorna:
Returns a u32 value representing the total number of failed test cases. A return value of 0 indicates that all cryptographic operations (key generation, signing, and verification) passed successfully.

Note:

- This function is a diagnostic tool designed to validate the integrity of the Ed25519 library. It is not thread-safe and should be executed during the system initialization phase or within a dedicated diagnostic task. It is a blocking function that performs heavy computational work, so it must never be called from an Interrupt Service Routine (ISR). Ensure that the system clock and memory management are fully initialized before calling this function, as it relies on internal buffers and standard caOS output routines.

Uso:

- This function is typically used during the boot-up sequence or in a factory test mode to ensure the cryptographic hardware or software implementation is functioning correctly. It is often paired with ca_os_banner to identify the system state and ca_stdout_printf_n to log the results. Developers should call this before enabling any network or security services that depend on Ed25519 signatures to prevent the system from operating with faulty cryptographic primitives.

Esempio:

```
/* Example: Running Ed25519 self-test during system startup */
#include "caos.h"

void system_diagnostic_task(void) {
    u32 error_count;

    ca_stdout_printf_n("Starting system diagnostics...");

    /* Execute the Ed25519 self-test suite */
    error_count = ca_ed25519_selftest();
```

```
11
12      if (error_count == 0) {
13          ca_stdout_printf_n("Crypto self-test: SUCCESS");
14      } else {
15          ca_stdout_printf_n("Crypto self-test: FAILED with %u errors",
    error_count);
16          /* Halt or enter safe mode if crypto is unreliable */
17          while(1) {
18              ca_os_yield();
19          }
20      }
21  }
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to execute ca_ed25519_selftest in a caOS project?
A: It should be executed during the system initialization phase or within a dedicated diagnostic task, specifically before enabling any network or security services that rely on Ed25519 signatures to ensure the cryptographic primitives are functioning correctly.

Code example conf:1.00 **Q:** How do I implement a basic check for the Ed25519 library during boot-up?
A: You can call the function and check the return value: u32 failures = ca_ed25519_selftest(); if (failures != 0) { /* Handle error */ }

Return conf:1.00 **Q:** What does the u32 return value from ca_ed25519_selftest indicate?
A: The return value represents the total number of failed test cases. A return value of 0 indicates that all cryptographic operations, including key generation, signing, and verification, passed successfully.

Error/Edge case conf:1.00 **Q:** What are the common mistakes or edge cases to avoid when calling ca_ed25519_selftest?
A: A common mistake is calling the function before the system clock and memory management are fully initialized, as it relies on internal buffers and standard caOS output routines. Additionally, it must not be called from an Interrupt Service Routine (ISR).

Behavior conf:1.00 **Q:** Is ca_ed25519_selftest thread-safe or non-blocking?
A: The function is not thread-safe and is a blocking function that performs heavy computational work. It must never be called from an ISR.

`ca_ed25519_sign`

Header: `caos/crypto/inc/ed25519.h:84`
Source: `caos/crypto/src/ed25519.c:461:506`
GUARD: `EN_ED25519`

```
1  void ca_ed25519_sign(u8 *signature, const u8 *message, u32 message_len,
       const u8 *seed, const u8 *public_key)
```

Generates an Ed25519 digital signature for a given message using a private seed.

Parametri:

Nome	Tipo	Descrizione
`signature`	`u8 *`	Buffer of 64 bytes to store the resulting signature. Must be pre-allocated.
`message`	`const u8 *`	Pointer to the data to be signed.
`message_len`	`u32`	Length of the message in bytes.
`seed`	`const u8 *`	32-byte private seed used to derive the signing key.
`public_key`	`const u8 *`	32-byte public key corresponding to the seed.

Ritorna:
void. The function does not return a status code; it performs internal validation and returns early if pointers are NULL.

Note:

- This function is computationally intensive as it performs multiple SHA-512 hashing operations and elliptic curve scalar multiplications. It is not thread-safe if the same buffers are accessed concurrently by other tasks. It is not safe to call from an ISR due to the high execution time and stack usage. Ensure that the 'signature', 'seed', and 'public_key' buffers are valid and properly aligned before calling. A common mistake is passing a NULL pointer or an insufficient buffer size, which will cause the function to return silently without signing.

Uso:

- This function is typically used in secure communication protocols or firmware update verification processes. It is usually preceded by ca_ed25519_public_from_seed to generate the public key from the seed, and followed by ca_ed25519_verify on the receiver side to authenticate the message. It is often used in conjunction with ca_memory_allocate to ensure buffers are safely placed in memory before processing.

Esempio:

```c
u8 my_signature[64];
u8 my_seed[32] = { /* ... initialized seed ... */ };
u8 my_pubkey[32];
const u8 *msg = (const u8 *)"Hello World";
u32 msg_len = 11;

/* Derive public key first */
ca_ed25519_public_from_seed(my_pubkey, my_seed);

/* Sign the message */
ca_ed25519_sign(my_signature, msg, msg_len, my_seed, my_pubkey);

/* Verify the signature to ensure integrity */
if (ca_ed25519_verify(my_signature, msg, msg_len, my_pubkey)) {
    /* Signature is valid */
} else {
    /* Signature is invalid */
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** In what scenarios should I use the ca_ed25519_sign function?
A: This function is used for generating Ed25519 digital signatures, typically for secure communication protocols or verifying firmware updates. It is usually preceded by ca_ed25519_public_from_seed to derive the public key and followed by ca_ed25519_verify on the receiver side.

Code example conf:1.00 **Q:** Provide a minimal C example of how to call ca_ed25519_sign using caOS types.

A: u8 sig[64]; u8 msg[] = "data"; u8 seed[32]; u8 pub[32]; ca_ed25519_sign(sig, msg, 4, seed, pub);

Parameters conf:1.00 **Q:** What are the requirements for the signature, seed, and public_key buffers?

A: The signature buffer must be pre-allocated with 64 bytes. The seed and public_key buffers must each be 32 bytes. All buffers must be valid and properly aligned before calling the function.

Error/Edge case conf:1.00 **Q:** What happens if I pass a NULL pointer or an insufficient buffer size to ca_ed25519_sign?

A: The function performs internal validation and will return silently without performing the signing operation if a NULL pointer is passed or if buffer sizes are insufficient.

Behavior conf:1.00 **Q:** Is it safe to call ca_ed25519_sign from an interrupt service routine (ISR) or concurrently from multiple tasks?

A: No, it is not safe to call from an ISR due to high execution time and stack usage. It is also not thread-safe if the same buffers are accessed concurrently by other tasks.

`ca_ed25519_test_init`

Header: `caos/crypto/inc/ed25519.h:113`
Source: `caos/crypto/src/ed25519.c:757:764`
GUARD: `EN_ED25519`

```
void ca_ed25519_test_init(void)
```

Registers the ED25519 cryptographic test commands into the system CLI.

Ritorna:
void

Note:

- This function is a wrapper that registers a command-line interface (CLI) node for ED25519 testing. It is not thread-safe by itself, but it uses a static flag to ensure the registration happens only once, preventing duplicate entries in the CLI command list. It is intended to be called during the system initialization phase before the scheduler starts. Do not call this from an Interrupt Service Routine (ISR) as it performs memory-related operations via the CLI registration process.

Uso:

- Use this function during the early system boot sequence, typically inside your main initialization routine before calling ca_os_run. It is usually paired with other test initialization functions like ca_sha512_test_init or ca_x25519_test_init to populate the CLI with diagnostic tools. Once registered, you can interact with the ED25519 module via the serial console to perform manual key generation or verification tests.

Esempio:

```
#include "caos.h"

void system_init_tasks(void) {
    /* Initialize the CLI system first */
    ca_cli_init();

    /* Register ED25519 test commands so they are available in the shell */
    ca_ed25519_test_init();

```

```
    /* Run the built-in self-test to ensure crypto hardware/software is healthy
    */
    if (ca_ed25519_selftest() != 0) {
        /* Handle critical failure if crypto self-test fails */
        while(1) { /* System halt */ }
    }

    /* Start the OS scheduler */
    ca_os_run(true);
}
```

Q&A — fine-tuning review *2/2 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_ed25519_test_init and why is it important to call it at that time?
A: You should call ca_ed25519_test_init during the early system boot sequence, specifically before calling ca_os_run. This is necessary because the function performs memory-related operations to register CLI nodes, which must be completed before the scheduler starts. It should not be called from an ISR.

Code example conf:0.95 **Q:** How do I initialize the ED25519 test commands in my main routine?
A: You should call the function during your system initialization phase as shown:
int main(void) { ca_ed25519_test_init(); ca_sha512_test_init(); ca_os_run(); return 0; }

`ca_ed25519_verify`

Header: `caos/crypto/inc/ed25519.h:100`
Source: `caos/crypto/src/ed25519.c:508:547`
GUARD: `EN_ED25519`

```
bool ca_ed25519_verify(const u8 *signature, const u8 *message, u32
    message_len, const u8 *public_key)
```

Verifies an Ed25519 digital signature against a provided message and public key.

Parametri:

Nome	Tipo	Descrizione
`signature`	`const u8 *`	Pointer to the 64-byte signature buffer. Must not be NULL.
`message`	`const u8 *`	Pointer to the data that was signed. Can be NULL if message_len is 0.
`message_len`	`u32`	Length of the message in bytes.
`public_key`	`const u8 *`	Pointer to the 32-byte public key buffer. Must not be NULL.

Ritorna:
Returns true if the signature is valid for the given message and public key, false otherwise. Returns false if any input pointer is invalid or if the signature format is non-canonical.

Note:

- This function is computationally intensive as it performs elliptic curve point multiplication. It is thread-safe as it uses local stack variables for all intermediate calculations, but it is not recommended to call this from an ISR due to the high execution time which could cause jitter in real-time tasks. Ensure the public key and signature buffers are correctly aligned and initialized before calling. A common mistake is passing an incorrect message

length or a corrupted public key, which will always result in a false return value even if the signature format is technically correct.

Uso:

- This function is typically used in secure communication protocols or firmware update verification processes. It is usually paired with ca_ed25519_public_from_seed to derive the key or used after receiving a signed packet over a network interface. The typical sequence is: receive the signed data, verify the signature using this function, and only if it returns true, proceed to process the message content.

Esempio:

```c
/* Example: Verifying a signed firmware update packet */
#define SIGNATURE_SIZE 64
#define PUBKEY_SIZE 32

void process_signed_data(u8 *data, u32 len, u8 *sig, u8 *pubkey) {
    /* Verify the signature before trusting the data */
    if (ca_ed25519_verify(sig, data, len, pubkey)) {
        /* Signature is valid, proceed with processing */
        ca_printf("Signature verified successfully.\n");
    } else {
        /* Signature invalid, discard data to prevent security breach */
        ca_printf("Security Alert: Invalid signature detected!\n");
    }
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** In what scenarios should I use the ca_ed25519_verify function?
A: This function is intended for secure communication protocols or firmware update verification. It is typically used after receiving a signed packet to validate the data before processing it, often following the use of ca_ed25519_public_from_seed to derive the public key.

Code example conf:1.00 **Q:** Provide a minimal C example for verifying an Ed25519 signature in caOS.
A: u8 sig[64] = { /* ... */ }; u8 msg[] = { /* ... */ }; u8 pub[32] = { /* ... */ }; bool valid = ca_ed25519_verify(sig, msg, (u32)sizeof(msg), pub); if (valid) { /* Proceed with processing */ }

Parameters conf:1.00 **Q:** What are the requirements for the signature and public_key parameters?
A: The signature parameter must point to a 64-byte buffer, and the public_key parameter must point to a 32-byte buffer. Both pointers must not be NULL, and the buffers must be correctly aligned and initialized before the call.

Return conf:1.00 **Q:** What do the return values of ca_ed25519_verify indicate?
A: The function returns true if the signature is valid for the provided message and public key. It returns false if the signature is invalid, if any input pointer is NULL, if the signature format is non-canonical, or if the message length is incorrect.

Error/Edge case conf:1.00 **Q:** Why might ca_ed25519_verify return false even if the signature format is correct?
A: Common mistakes include passing an incorrect message length or using a corrupted public key. These conditions will cause the function to return false even if the signature format is technically correct.

Behavior conf:1.00 **Q:** Is it safe to call ca_ed25519_verify from an Interrupt Service Routine (ISR)?
A: No, it is not recommended to call this from an ISR. The function is computationally intensive due to elliptic curve point multiplication, which could cause significant jitter in

real-time tasks. While it is thread-safe because it uses local stack variables, its execution time is too high for ISR contexts.

---

ca_enter_critical

Header: caos/inc/sync.h:291
Source: caos/src/sync.c:714:728
GUARD: EN_SYNC

```
void ca_enter_critical(void)
```

Disables all maskable interrupts to create a protected, atomic execution block.

Ritorna:
This function returns void and does not provide a status code.

Note:

- This function is non-blocking and executes immediately by modifying the CPU's PRIMASK register to disable interrupts. It is thread-safe in the sense that it prevents context switching, but it is NOT intended to be called from an ISR, as ISRs are already running with higher priority or disabled interrupts. The implementation supports nesting, meaning you can call it multiple times; however, you must ensure that every call to ca_enter_critical is paired with exactly one call to ca_exit_critical. A common beginner mistake is forgetting to call the exit function, which will leave the system with interrupts permanently disabled, effectively freezing the RTOS scheduler and all peripheral communication.

Uso:

- Use this function when you need to perform a 'read-modify-write' operation on a shared variable that is accessed by both the main thread and an interrupt service routine. By entering a critical section, you guarantee that the CPU will not jump to an ISR in the middle of your calculation, preventing data corruption or race conditions. It is typically used in a sequence: call ca_enter_critical, perform the sensitive operation, and immediately call ca_exit_critical to restore the previous interrupt state. It is often paired with global counters or shared buffers that require atomicity, and you can verify the nesting depth using ca_critical_nesting_level if debugging complex logic.

Esempio:

```
/* Example: Protecting a shared counter from race conditions */
void update_shared_data(void) {
    /* Enter critical section to ensure atomicity */
    ca_enter_critical();

    /* Perform sensitive operations while interrupts are disabled */
    /* Assume shared_counter and write_count are global variables */
    shared_counter++;
    write_count++;

    /* Check nesting level if needed for debugging */
    u32 current_nesting = ca_critical_nesting_level();
    (void)current_nesting;

    /* Exit critical section to restore previous interrupt state */
    ca_exit_critical();
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_enter_critical() in my caOS application?
A: You should call ca_enter_critical() when you need to protect a critical section of code from being interrupted by other ISRs. This ensures atomicity of operations that modify shared data or require exclusive access to hardware resources. It is typically used in scenarios where race conditions could occur if multiple threads or ISRs access the same resource concurrently.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_enter_critical() and ca_exit_critical() with caOS types?
A: u32 shared_counter = 0; void isr_handler(void) { ca_enter_critical(); shared_counter++; ca_exit_critical(); }

Return conf:0.90 **Q:** What does ca_enter_critical() return, and can it fail?
A: ca_enter_critical() does not return any value. It is a void function and does not fail under normal circumstances. However, improper usage such as calling it without a matching ca_exit_critical() can lead to deadlock or loss of interrupts, which are not considered failures of the function itself but rather misuse of the API.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_enter_critical()?
A: Common mistakes include calling ca_enter_critical() from within an ISR, failing to pair it with ca_exit_critical(), or nesting calls without proper handling. Since the function is not recursive-safe, improper nesting can cause deadlock. Additionally, if interrupts are disabled for too long, it may affect system responsiveness or cause missed deadlines in real-time applications.

`ca_eth_dbg_dump`

Header: `caos/drivers/inc/eth.h:121`
Source: `caos/drivers/src/eth.c:251:254`
GUARD: `EN_ETH`

```
1 void ca_eth_dbg_dump(void)
```

Prints the internal state and hardware registers of the Ethernet controller to the system console for debugging.

Ritorna:
This function returns void and does not provide a status code, meaning it assumes the hardware is accessible when called.

Note:

- This function is a diagnostic tool that triggers a low-level hardware dump of the Ethernet peripheral. It is non-blocking in terms of RTOS task scheduling, but it may take significant time to execute because it performs multiple serial output operations to the console. It is generally NOT safe to call from an Interrupt Service Routine (ISR) because it likely uses blocking I/O or shared resources that could cause a deadlock or system instability. The most common beginner mistake is calling this function inside a high-frequency loop, which will flood the console and severely degrade system performance or cause the watchdog timer to reset the MCU.

Uso:

- This function is primarily used during the development and testing phase to troubleshoot network connectivity or buffer management issues. It is typically paired with CLI (Command Line Interface) handlers to allow developers to inspect the Ethernet state on demand, or placed inside error-checking routines to capture the state of the hardware when a memory leak or network failure is detected. It is often used in conjunction with buffer

monitoring functions to verify that the driver is correctly releasing network packets back to the pool.

Esempio:

```c
/* Example: Triggering a dump via a CLI command or error check */
void handle_eth_debug_command(void) {
    /* Check if the Ethernet hardware is active before dumping */
    /* We use ca_eth_link_poll to verify if the interface is responsive */
    if (ca_eth_link_poll()) {
        /* Print status to console using ca_stdout_printf_n */
        ca_stdout_printf_n("Dumping Ethernet hardware state...\n");
        ca_eth_dbg_dump();
    } else {
        ca_stdout_printf_n("Error: Ethernet link is down or driver not ready.\n");
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_eth_dbg_dump() during development and what is its purpose?
A: You should call ca_eth_dbg_dump() during the development phase to print debug information about the Ethernet interface's state. It is useful for troubleshooting and understanding the current status of the Ethernet hardware. The function is intended for debugging only and must not be used in production code. It should not be called from an ISR because it might cause synchronization or context issues.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_eth_dbg_dump() with caOS types?
A: void example_eth_debug(void) { ca_eth_dbg_dump(); }

Parameters conf:0.85 **Q:** Does ca_eth_dbg_dump() have any parameters, and what happens if I pass invalid values?
A: The function ca_eth_dbg_dump() does not take any parameters. Therefore, there are no parameter values to validate or invalidate. Calling it with any arguments will result in a compilation error since the function signature expects no inputs.

`ca_eth_deinit`

Header: `caos/drivers/inc/eth.h:109`
Source: `caos/drivers/src/eth.c:226:229`
GUARD: `EN_ETH`

```c
void ca_eth_deinit(void)
```

Shuts down the Ethernet hardware peripheral and releases associated resources.

Ritorna:
This function returns void and does not provide a status code, meaning it assumes the hardware shutdown will succeed.

Note:

- This function is non-blocking and is intended to put the Ethernet controller into a low-power or disabled state. It is not thread-safe by itself, so you must ensure that no other tasks are attempting to send or receive packets while this is being called, or you risk a hard fault or undefined behavior. It is generally not safe to call this from an Interrupt Service Routine (ISR) unless the underlying hardware driver explicitly supports it, as it

may involve complex register sequences or delays. The most common beginner mistake is calling this function while the network stack is still active, which leads to dangling pointers or attempts to access hardware registers that have been disabled.

Uso:

- This function is typically used during the system shutdown sequence or when reconfiguring the network interface. It is the counterpart to an initialization function (e.g., ca_eth_init) and is usually called within a 'stop' or 'deinit' callback of a network interface structure. You should pair this with a preceding call to stop any active network tasks or DMA transfers to ensure the hardware is in a quiescent state before the power-down sequence begins.

Esempio:

```
/* Example of stopping the network interface safely */
void shutdown_network_system(void) {
    /* 1. Ensure no further network processing occurs */
    // Stop any application-level network tasks here

    /* 2. Now safely deinitialize the hardware */
    ca_eth_deinit();

    /* 3. The hardware is now off; safe to re-init later with ca_eth_init() */
}
```

Q&A — fine-tuning review *2/2 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_eth_deinit() in my caOS application, and what are the risks of calling it at the wrong time?
A: You should call ca_eth_deinit() when you need to shut down or restart the Ethernet module, specifically before powering off or resetting the network hardware. It must only be called when no other Ethernet operations are in progress, such as during packet transmission or reception. Calling it during active network operations can lead to data loss or system crashes because it disables the hardware controller. This function is blocking and must not be invoked from an ISR, and it is not thread-safe.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to properly use ca_eth_deinit() with caOS types?
A: void example_eth_cleanup(void) { // Ensure no Ethernet activity is ongoing ca_eth_deinit(); // Disables Ethernet hardware resources // At this point, Ethernet is no longer functional }

`ca_eth_hw_dbg_dump`

Header: `caos/drivers/inc/ca_glue_hw.h:122`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_eth_hw_dbg_dump (void); /* stampa su debug i registri DMA
    /MAC */
```

Prints the internal state of the Ethernet DMA and MAC registers to the debug console.

Ritorna:
void

Note:

- This function is a diagnostic tool that provides a snapshot of the Ethernet hardware registers, such as DMA status and descriptor indices. It is non-blocking and executes quickly, but it relies on the underlying ca_stdout_printf_n function, which may be slow depending on the output interface (e.g., UART or ITM). It is generally safe to call from an ISR, but be aware that printing large amounts of data from an interrupt context can significantly increase interrupt latency and jitter. The most common mistake is calling this function when the Ethernet peripheral has not been initialized, which may result in reading invalid or zeroed register values that do not reflect the actual hardware state.

Uso:

- This function is used exclusively for troubleshooting network connectivity issues, such as dropped packets, DMA errors, or unexpected Ethernet interrupts. It is typically called manually during a debugging session or triggered automatically when a critical error is detected in the Ethernet driver state machine. It is often paired with other diagnostic functions like ca_eth_pool_held_count to verify if the system is running out of memory buffers or if the DMA engine has stalled due to a descriptor ownership error.

Esempio:

```c
/* Example: Debugging a suspected Ethernet stall */
void check_network_health(void) {
    /* Check if we are holding too many buffers, indicating a potential leak */
    if (ca_eth_pool_held_count() > 10) {
        /* Print hardware registers to identify if DMA is stuck */
        ca_eth_hw_dbg_dump();
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_eth_hw_dbg_dump() and what is its purpose in debugging?
A: You should call ca_eth_hw_dbg_dump() only during debugging sessions to inspect the state of Ethernet DMA and MAC hardware registers. It is useful for diagnosing network issues or verifying hardware configuration. The function is intended for development and debugging only, not for production code.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_eth_hw_dbg_dump(with caOS types?
A:

```c
void example_usage(void)
{
    ca_eth_hw_dbg_dump();
}
```

Parameters conf:0.95 **Q:** Does ca_eth_hw_dbg_dump() accept any parameters, and what are their meanings?
A: No, ca_eth_hw_dbg_dump() does not accept any parameters. It is a void function that reads and prints hardware register values directly from the Ethernet peripheral.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_eth_hw_dbg_dump()?
A: Common mistakes include calling this function from an ISR, which may cause synchronization issues. It is also not thread-safe and requires a properly configured debug output system. Additionally, it is a no-op in some implementations (e.g., f4xx), so it may not produce any output if not correctly implemented for the target hardware.

HW differences conf:0.90 **Q:** How does ca_eth_hw_dbg_dump() differ between F7xx, H5xx, and H7xx MCU families?
A: The function differs in the registers it reads and prints. For F7xx, it prints DMASR, DMABMR, rx_idx, and des0. For H5xx, it prints DMACSR, DMACIER, rx_idx, and

des3. For H7xx, it prints multiple registers including DMACSR, DMACIER, DMACRCR, DMACRDTPR, along with NVIC status and debug counters, providing more detailed hardware state information.

---

ca_eth_hw_deinit

Header: caos/drivers/inc/ca_glue_hw.h:118
GUARD: EN_CA_GLUE_HW

```
extern void ca_eth_hw_deinit (void); /* ferma DMA e MAC Ethernet */
```

Safely shuts down the Ethernet hardware peripheral, stops DMA transfers, and releases associated memory buffers.

Ritorna:
void

Note:

- This function is non-blocking and performs direct register manipulation to disable the Ethernet MAC, DMA engines, and interrupts. It is not thread-safe; you must ensure no other tasks are attempting to send or receive data while this is executing, or the system may enter an undefined state. It is generally not safe to call from an ISR because it performs cleanup operations that might conflict with ongoing network stack processing. A common beginner mistake is calling this function while the network stack is still active, which leads to memory leaks if the internal buffer pools are not correctly returned to the system, or hard faults if the DMA attempts to access memory that has been de-initialized.

Uso:

- This function is used during system shutdown, power-saving transitions, or when reconfiguring the network interface. It is typically the final step in a teardown sequence, following the stopping of the network stack and any application-level protocols. It is designed to be paired with 'ca_eth_init' to reset the hardware to a known clean state before re-initialization. In the caOS architecture, it acts as the hardware-specific cleanup routine called by the higher-level 'ca_eth_deinit' wrapper.

Esempio:

```
/* Example of safely shutting down the Ethernet interface */
void shutdown_network_interface(void) {
    /* 1. Stop the high-level network stack first */
    // ca_net_link_down();

    /* 2. Disable the hardware peripheral and release buffers */
    ca_eth_hw_deinit();

    /* 3. Now it is safe to re-initialize or power down */
    // ca_eth_init(...);
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_eth_hw_deinit() in my caOS application?
A: You should call ca_eth_hw_deinit() when you want to completely shut down the Ethernet hardware, typically during system shutdown or when no more network operations are expected. It is important to ensure no active network tasks are running before calling this function.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_eth_hw_deinit() with caOS types?
A:

```c
void example_eth_deinit(void) {
    ca_eth_hw_deinit();
}
```

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_eth_hw_deinit()?
A: Common mistakes include calling ca_eth_hw_deinit() from within an ISR, which is not thread-safe or ISR-safe as per documentation. Also, calling it while network operations are still active may lead to undefined behavior. Additionally, since it's a blocking function without timeout, it should be used carefully in time-sensitive contexts.

Behavior conf:0.95 **Q:** Is ca_eth_hw_deinit() blocking or non-blocking, and is it safe to call from an ISR?
A: ca_eth_hw_deinit() is a blocking function with no timeout. It is not safe to call from an ISR because it is not designed for interrupt contexts and is not thread-safe. It must be called when no network operations are in progress.

HW differences conf:0.90 **Q:** How does ca_eth_hw_deinit() differ between F7xx, H5xx, and H7xx microcontrollers?
A: On F7xx, ca_eth_hw_deinit() disables the ETH interrupt, clears DMA interrupt enable register, stops the DMA transmitter and receiver, disables MAC transmitter and receiver, and turns off the Ethernet clocks. On H5xx, it performs similar steps but uses different register names like DMACIER, DMACTCR, DMACRCR, and MACCR. On H7xx, it also includes freeing pre-allocated network buffers from the rx and tx pools, which is not present in other families.

`ca_eth_hw_dma_sync`

Header: `caos/drivers/inc/ca_glue_hw.h:126`
GUARD: `EN_CA_GLUE_HW`

```c
extern void ca_eth_hw_dma_sync (void); /* stops the DMA, rebuilds the
   descriptor rings, restarts: back to the post-init state */
```

Synchronizes the Ethernet DMA engine by stopping hardware, clearing buffers, and resetting descriptor rings.

Ritorna:
void

Note:

- This function is a heavy-duty maintenance operation. It is not thread-safe and must not be called from an ISR, as it performs memory management (freeing buffers) and hardware register manipulation that requires a stable system state. It is a blocking operation that halts all network traffic while it rebuilds the DMA descriptors. A common beginner mistake is calling this function during high-traffic periods or from within a network callback, which can lead to race conditions or system instability. Always ensure the network stack is in a state where it can tolerate a temporary loss of connectivity before invoking this.

Uso:

- This function is typically used as a recovery mechanism when the Ethernet hardware enters an inconsistent state, such as after a buffer overflow or a communication timeout. It is often paired with ca_eth_init during system startup or as a 'reset' command in a CLI interface to clear out stale descriptors. By returning parked buffers to the pool, it prevents memory

leaks that would otherwise occur if the DMA engine were simply restarted without cleaning up the pending RX/TX slots. It is the standard way to force the Ethernet peripheral back to a clean, post-initialization state without performing a full system reboot.

Esempio:

```c
/* Example: Resetting the Ethernet DMA engine via a CLI command */
#include "caos.h"

void handle_eth_reset_command(void) {
    /* Stop network processing to ensure no race conditions during sync */
    /* ca_net_link_down(); */

    /* Perform the hardware synchronization */
    ca_eth_hw_dma_sync();

    /* Log the action to the user */
    ca_printf("Ethernet DMA rings have been reset and hardware restarted.\n");

    /* Resume network processing */
    /* ca_net_link_up(); */
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I invoke ca_eth_hw_dma_sync in my caOS application?
A: You should use this function as a recovery mechanism when the Ethernet hardware enters an inconsistent state, such as after a communication timeout or a buffer overflow. It is also useful as a reset command in a CLI to clear stale descriptors without performing a full system reboot.

Code example conf:1.00 **Q:** How do I correctly call ca_eth_hw_dma_sync in my code?
A: Ensure the network stack can tolerate a temporary loss of connectivity, then call the function directly: ca_eth_hw_dma_sync();. Do not call it from an ISR or during high-traffic periods.

Error/Edge case conf:1.00 **Q:** What are the common mistakes when using ca_eth_hw_dma_sync?
A: A common mistake is calling this function from within a network callback or during high-traffic periods, which can lead to race conditions or system instability. Additionally, calling it from an ISR is prohibited as it performs memory management and hardware register manipulation.

Behavior conf:1.00 **Q:** Is ca_eth_hw_dma_sync thread-safe and non-blocking?
A: No, it is not thread-safe and it is a blocking operation. It halts all network traffic while it rebuilds the DMA descriptors and performs memory management, so it requires a stable system state before invocation.

HW differences conf:1.00 **Q:** How does the implementation of ca_eth_hw_dma_sync differ between STM32 families like F4xx and H5xx?
A: On F4xx and F7xx, the function resets the DMAOMR and MACCR registers and calls desc_init. On H5xx and H7xx, the function additionally iterates through RX and TX pools to explicitly free parked buffers using ca_net_buf_free to prevent memory leaks, and uses DMACTCR/DMACRCR registers instead of DMAOMR.

`ca_eth_hw_init`

Header: `caos/drivers/inc/ca_glue_hw.h:115`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_eth_hw_init (const ca_eth_cfg_t *cfg); /* inizializza MAC
    , DMA e PHY Ethernet */
```

Initializes the Ethernet MAC, DMA, and PHY hardware peripherals on STM32 microcontrollers.

Parametri:

Nome	Tipo	Descrizione
cfg	const ca_eth_cfg_t *	Pointer to a configuration structure containing MAC address, PHY address, RMII/MII mode selection, and callback function pointers. Must not be NULL.

Ritorna:
Returns true if the hardware initialization, DMA reset, and register configuration succeeded. Returns false if the configuration pointer is NULL, the DMA fails to reset, or the hardware is unsupported on the current MCU target.

Note:

- This function is not thread-safe and should be called during the system startup phase before any network tasks are spawned. It is a blocking function because it performs hardware-level resets and waits for PHY auto-negotiation to complete, which can take up to 500ms. It must NOT be called from an Interrupt Service Routine (ISR) due to its long execution time and reliance on blocking loops. A common beginner mistake is failing to ensure the PHY hardware reset pin (controlled via GPIO) is toggled before calling this function, or providing an incorrect PHY address, which will cause the initialization to hang or fail silently.

Uso:

- This function is the primary entry point for the Ethernet driver stack. It is typically called once during system initialization after the GPIOs for the Ethernet interface have been configured. It is also used in recovery scenarios, such as in ca_eth_restart, to re-initialize the hardware if a link failure or fatal DMA error is detected. It pairs with desc_init() internally to set up memory buffers and configures the NVIC to handle Ethernet interrupts if callbacks are provided.

Esempio:

```
ca_eth_cfg_t eth_config = { .phy_addr = 0, .rmii = true, .mac = {{0x00, 0x80, 0
    xE1, 0x00, 0x00, 0x00}}, .rx_cb = my_rx_handler, .tx_done_cb = NULL, .
    error_cb = my_err_handler }; if (!ca_eth_hw_init(&eth_config)) { /* Handle
    initialization failure, perhaps log error or retry */ }
```

Q&A — fine-tuning review

7/7 pairs · qwen3-coder:latest

Usage conf:0.95 **Q:** When should I call ca_eth_hw_init() in my application?
A: You should call ca_eth_hw_init() during system initialization, typically after all hardware peripherals are configured and before starting network operations. It initializes the Ethernet MAC and DMA controllers for packet transmission and reception.

Code example conf:0.85 **Q:** Can you show a minimal example of how to use ca_eth_hw_init() with caOS types?
A:

```
ca_eth_cfg_t cfg;
cfg.rmii = true;
cfg.mac_addr[0] = 0x00;
cfg.mac_addr[1] = 0x11;
cfg.mac_addr[2] = 0x22;
cfg.mac_addr[3] = 0x33;
cfg.mac_addr[4] = 0x44;
```

```
cfg.mac_addr[5] = 0x55;
cfg.rx_cb = NULL;
cfg.tx_cb = NULL;
cfg.err_cb = NULL;
bool success = ca_eth_hw_init(&cfg);
```

Parameters conf:0.90 **Q:** What does the 'cfg' parameter in ca_eth_hw_init() represent and what are valid values?
A: The 'cfg' parameter is a pointer to a ca_eth_cfg_t structure containing configuration options. It must not be NULL. Valid fields include 'rmii' (true for RMII mode), 'mac_addr' (6-byte MAC address), and callback pointers for RX/TX/err handling. Invalid values include NULL pointer or invalid MAC address bytes.

Return conf:0.95 **Q:** What does ca_eth_hw_init() return and what causes it to fail?
A: It returns true on success, false on failure. Failures occur if 'cfg' is NULL, or if hardware initialization fails such as a timeout during DMA reset or PHY auto-negotiation.

Error/Edge case conf:0.90 **Q:** What are common mistakes when calling ca_eth_hw_init()?
A: Common mistakes include passing a NULL pointer for 'cfg', not initializing the MAC address, forgetting to enable clocks for required peripherals like RCC, or not handling PHY reset before calling this function. Also, calling it before PHY is ready can cause timeouts.

Behavior conf:0.95 **Q:** Is ca_eth_hw_init() blocking or non-blocking, and is it safe to call from an ISR?
A: It is a blocking function that waits for hardware initialization and PHY auto-negotiation. It is not ISR-safe and not thread-safe. It requires preconditions like enabled clocks and valid configuration before being called.

HW differences conf:0.90 **Q:** How does ca_eth_hw_init() differ between H7 and H7+ families in terms of hardware initialization?
A: On H7+, ca_eth_hw_init() includes dynamic speed/duplex configuration based on PHY auto-negotiation (reading BMSR and ANLPAR), whereas H7 family does not perform this dynamic configuration. Additionally, H7+ uses MPU region setup for DMA memory to avoid cache coherency issues, which is absent in H7.

`ca_eth_hw_isr`

Header: `caos/drivers/inc/ca_glue_hw.h:124`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_eth_hw_isr (void); /* gestore IRQ Ethernet: notifica
   ricezione e azzeramento flag */
```

Hardware Interrupt Service Routine (ISR) for the Ethernet peripheral to handle packet reception, transmission completion, and error events.

Ritorna:
void. This function does not return a value as it is triggered by hardware interrupt signals.

Note:

- This function is an Interrupt Service Routine (ISR) and must be mapped directly to the Ethernet global interrupt vector in the vector table. It is non-blocking by design, as it must execute as quickly as possible to avoid missing subsequent network packets or causing system latency. It is not thread-safe in the traditional sense because it preempts the main application code; therefore, any shared data accessed within the callbacks (s_rx_cb, s_tx_done_cb, s_error_cb) must be protected using atomic operations or critical sections. A common beginner mistake is performing heavy processing or blocking operations

(like waiting for a mutex or a long delay) inside these callbacks, which will cause the Ethernet peripheral to hang or drop packets due to the ISR taking too long to complete.

Uso:

- This function is called automatically by the STM32 hardware whenever an Ethernet event occurs, such as a frame arriving or a transmission buffer becoming available. It acts as the bridge between the low-level hardware registers and the higher-level application logic. Typically, you initialize the Ethernet peripheral using ca_eth_init, which internally sets up the necessary callback pointers. Once the hardware is configured and interrupts are enabled in the NVIC, the ISR will trigger these callbacks whenever the corresponding hardware flags are set, allowing the application to react to network traffic asynchronously.

Esempio:

```c
/* Example of how the ISR is linked to the hardware vector table */
void ETH_IRQHandler(void) {
    /* This is the standard STM32 vector name that calls our driver ISR */
    ca_eth_hw_isr();
}

/* Application setup code */
void network_init(void) {
    /* Initialize the Ethernet hardware */
    ca_eth_init();

    /* The driver internally manages the callback pointers (s_rx_cb, etc.) */
    /* which are triggered by ca_eth_hw_isr when packets arrive or are sent. */

    /* Enable the Ethernet interrupt in the NVIC */
    /* NVIC_EnableIRQ(ETH_IRQn); */
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should ca_eth_hw_isr be called, and why is it important for Ethernet handling?
A: ca_eth_hw_isr is an interrupt service routine (ISR) that must be called by the hardware interrupt controller when an Ethernet interrupt occurs. It is important because it handles the notification of received frames, released transmission slots, and network errors, ensuring proper communication with the Ethernet peripheral.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_eth_hw_isr with caOS types?
A:

```c
void my_rx_callback(void) {
    /* Handle received frame */
}

void my_tx_callback(void) {
    /* Handle transmitted frame */
}

void my_error_callback(u32 flags) {
    /* Handle Ethernet errors */
}

void setup_eth_callbacks(void) {
    s_rx_cb = my_rx_callback;
    s_tx_done_cb = my_tx_callback;
    s_error_cb = my_error_callback;
}
```

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_eth_hw_isr, and what errors can occur?

A: A common mistake is not clearing interrupt flags properly, which can cause infinite loops or repeated interrupts. Additionally, calling ca_eth_hw_isr manually from user code instead of letting the interrupt controller invoke it will lead to incorrect behavior. The function assumes that the callback functions (s_rx_cb, s_tx_done_cb, s_error_cb) are registered before use.

Behavior conf:0.90 **Q:** Is ca_eth_hw_isr blocking or non-blocking, and is it safe to call from a thread context?

A: ca_eth_hw_isr is a blocking function because it performs immediate operations such as clearing interrupt flags and invoking callbacks. It is ISR-safe and can be called from interrupt context, but it is only thread-safe if the registered callbacks are also thread-safe.

HW differences conf:0.85 **Q:** How does ca_eth_hw_isr differ between STM32F4, F7, H5, and H7 families in terms of registers used?

A: The register names differ between families: F4/F7 use ETH->DMASR, H5 uses ETH->DMACSR, and H7 also uses ETH->DMACSR. Additionally, the bit definitions vary slightly; for example, F4/F7 use ETH_DMASR_RS for receive, while H5/H7 use ETH_DMACSR_RI for the same purpose.

`ca_eth_hw_link_poll`

Header: `caos/drivers/inc/ca_glue_hw.h:125`
GUARD: `EN_CA_GLUE_HW`

```
1 extern bool ca_eth_hw_link_poll (void); /* reads the link status from
     the PHY; true = link up */
```

Reads the physical Ethernet link status directly from the PHY register.

Ritorna:
Returns true if the Ethernet link is detected (Link Up), and false if the cable is disconnected, the link is down, or the hardware driver does not support link polling.

Note:

- This function is non-blocking but performs an I/O operation over the MDIO/MDC bus, which can take several microseconds to complete. It is generally thread-safe if the underlying PHY read implementation uses proper mutexes or atomic access to the MDIO peripheral, but you should verify this for your specific hardware target. It is typically safe to call from an ISR if the MDIO bus is not shared with other tasks, but be aware that blocking the CPU for the duration of the PHY read might impact real-time performance. A common beginner mistake is calling this function too frequently in a tight loop, which can saturate the MDIO bus and delay other critical Ethernet management tasks. Note that on some hardware targets (e.g., f1xx), this function is a stub that always returns false, so always check your specific hardware implementation.

Uso:

- This function is used by the Ethernet driver's background task or a periodic timer to monitor the physical connection status. It is typically paired with ca_eth_link_poll, which acts as a wrapper to detect state changes and trigger user-defined callbacks. You should call this only after the Ethernet hardware has been successfully initialized via ca_eth_init. If the function returns false, the application should stop attempting to transmit data via ca_eth_send to avoid unnecessary timeouts or hardware errors.

Esempio:

```c
/* Periodic task to monitor Ethernet link status */
void eth_monitor_task(void) {
    /* Check if the link status has changed and trigger callbacks if needed */
    bool current_link = ca_eth_link_poll();

    if (current_link) {
        /* Link is up, safe to process network traffic */
        // Proceed with network stack operations
    } else {
        /* Link is down, stop network operations to prevent errors */
        // Halt network transmission
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should a developer call ca_eth_hw_link_poll() and why is it important to check the link status?
A: A developer should call ca_eth_hw_link_poll() to determine whether the Ethernet link is up or down. It is important because the link status affects the operation of the Ethernet driver and network communication. Ignoring the return value can lead to incorrect assumptions about network connectivity. The function is non-blocking and must be used in contexts where safe access to the PHY is guaranteed.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using caOS types that demonstrates how to use ca_eth_hw_link_poll()?
A: bool link_up = ca_eth_hw_link_poll(); if (link_up) { /* Ethernet link is active */ } else { /* Ethernet link is down */ }

Return conf:0.90 **Q:** What do the return values of ca_eth_hw_link_poll() indicate, and what causes the function to return false?
A: The function returns true if the Ethernet link is up, and false if the link is down or in case of hardware error or unimplemented functionality. A return value of false can be caused by hardware issues, incorrect PHY access, or lack of implementation in certain MCU families such as F1xx and F4xx.

HW differences conf:0.95 **Q:** How does ca_eth_hw_link_poll() differ in implementation across different STM32 families like F7xx, H5xx, and H7xx?
A: In F7xx, H5xx, and H7xx families, ca_eth_hw_link_poll() reads the PHY BMSR register to check bit 2 (PHY_BMSR_LINK) to determine link status. In contrast, F1xx and F4xx families implement this function as a stub that always returns false, indicating no functional link polling support in those families.

`ca_eth_hw_phy_read`

Header: `caos/drivers/inc/ca_glue_hw.h:119`
GUARD: `EN_CA_GLUE_HW`

```c
extern u16 ca_eth_hw_phy_read (u8 phy, u8 reg); /* legge un registro
    MDIO del PHY */
```

Reads a 16-bit value from a specific register of an external Ethernet PHY chip via the MDIO interface.

Parametri:

Nome	Tipo	Descrizione
phy	u8	The PHY address on the MDIO bus (typically 0-31). Ensure this matches your hardware schematic.
reg	u8	The specific register address within the PHY to read (typically 0-31).

Ritorna:
Returns the 16-bit data read from the PHY register. Returns 0 if the hardware does not support the operation or if the read fails due to a timeout.

Note:

- This function is a blocking call; it enters a software loop waiting for the hardware MDIO busy bit to clear. It is not thread-safe, meaning if two tasks call this simultaneously, the MDIO bus state will be corrupted. It is generally safe to call from an ISR if the hardware is not being accessed elsewhere, but this is discouraged due to the blocking nature which can cause jitter. A common beginner mistake is calling this function too frequently, which can saturate the MDIO bus and delay critical network stack operations.

Uso:

- This function is used during the Ethernet initialization sequence to probe the PHY, check link status, or configure auto-negotiation settings. It is typically paired with ca_eth_hw_phy_write to perform read-modify-write operations on PHY registers. You should call this after the Ethernet peripheral clock is enabled and the MDIO clock divider is configured, otherwise, the hardware will not respond and the function will return a timeout value.

Esempio:

```
/* Example: Reading the PHY Status Register (usually reg 1) to check link status */
#define PHY_ADDR 0x01
#define PHY_STATUS_REG 0x01

u16 status = ca_eth_hw_phy_read(PHY_ADDR, PHY_STATUS_REG);

/* Check if the link is up (bit 2 is often the link status bit) */
if (status & 0x0004) {
    /* Link is established */
} else {
    /* Link is down, handle accordingly */
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_eth_hw_phy_read() in my caOS project?
A: You should use ca_eth_hw_phy_read() when you need to read configuration or status registers from an Ethernet PHY chip. This is useful for diagnosing network issues, checking link status, or configuring PHY-specific features like auto-negotiation or power management.

Code example conf:0.90 **Q:** Can you show a minimal example of how to read a PHY register using ca_eth_hw_phy_read()?
A: u16 phy_status = ca_eth_hw_phy_read(0, 1); // Read status register of PHY at address 0 if (phy_status != 0) { // Process PHY status }

Parameters conf:0.95 **Q:** What are the valid ranges for the phy and reg parameters in ca_eth_hw_phy_read()?
A: The phy parameter must be between 0 and 31 (inclusive), and the reg parameter must also be between 0 and 31 (inclusive). Values outside these ranges may cause incorrect reads or timeouts.

Return conf:0.90 **Q:** What does ca_eth_hw_phy_read() return when it fails to read a register?
A: The function returns 0 when it fails to read a register. This can happen if the Ethernet interface is not initialized, or if the MDIO bus is busy or malfunctioning.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_eth_hw_phy_read()?
A: Common mistakes include calling the function without initializing the Ethernet hardware first, using invalid PHY or register addresses, and not checking the return value. Not checking the return value can lead to incorrect assumptions about the PHY's state.

HW differences conf:0.80 **Q:** How does ca_eth_hw_phy_read() differ between F7xx and H5xx/H7xx families?
A: Both F7xx and H5xx/H7xx families use similar MDIO register access patterns, but they differ in peripheral registers. F7xx uses ETH->MACMIIAR and ETH->MACMIIDR, while H5xx/H7xx use ETH->MACMDIOAR and ETH->MACMDIODR. The bit positions for PHY address and register index are also slightly different between families.

`ca_eth_hw_phy_write`

Header: `caos/drivers/inc/ca_glue_hw.h:120`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_eth_hw_phy_write (u8 phy, u8 reg, u16 val); /* scrive un registro MDIO del PHY */
```

Writes a 16-bit value to a specific register of an external Ethernet PHY chip via the MDIO interface.

Parametri:

Nome	Tipo	Descrizione
`phy`	`u8`	The PHY address on the MDIO bus, typically ranging from 0 to 31.
`reg`	`u8`	The specific register address within the PHY to be written, usually 0 to 31.
`val`	`u16`	The 16-bit data value to be stored in the target PHY register.

Ritorna:
void. This function does not return a status, meaning it assumes the write operation will succeed or simply times out if the hardware is unresponsive.

Note:

- This function is blocking; it enters a busy-wait loop (polling the 'Busy' bit) until the hardware completes the MDIO transaction or a timeout counter expires. It is not thread-safe, meaning if two tasks call this simultaneously, the MDIO bus state will be corrupted; you must protect access with a mutex if multiple tasks interact with the PHY. It is generally safe to call from an ISR, but because it contains a busy-wait loop, it will stall the CPU and potentially cause jitter in time-critical interrupts. A common beginner mistake is calling this function repeatedly in a high-frequency loop without checking the link status, which can flood the MDIO bus and cause communication delays.

Uso:

- This function is used to configure PHY settings such as auto-negotiation, speed, or duplex modes. It is typically called during the Ethernet driver initialization sequence or when the link state changes. It is often paired with 'ca_eth_hw_phy_read' to perform a read-modify-write operation, where you read a register, change specific bits, and write the value

back. It is part of the low-level hardware abstraction layer (HAL) and is usually wrapped by higher-level functions like 'ca_eth_phy_write' which might include additional logic or logging.

Esempio:

```c
/* Example: Configure PHY to force 100Mbps Full Duplex */
#define PHY_ADDR 0x01
#define PHY_BCR_REG 0x00
#define PHY_BCR_FORCE_100_FD 0x2100

void configure_ethernet_phy(void) {
    /* Write the configuration value to the Basic Control Register */
    ca_eth_hw_phy_write(PHY_ADDR, PHY_BCR_REG, PHY_BCR_FORCE_100_FD);

    /* Note: No return value to check, so we assume success if the hardware is
    functional */
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_eth_hw_phy_write and what is its purpose in Ethernet configuration?
A: You should call ca_eth_hw_phy_write to configure the PHY (Physical Layer) device connected to the Ethernet MAC controller. It writes a value to a specific register of the PHY using the MDIO (Management Data Input/Output) interface. This function is essential for setting up the PHY's operational parameters such as speed, duplex mode, or other specific features before network communication can begin.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_eth_hw_phy_write with caOS types?
A:

```c
u16 phy_addr = 0;
u16 reg_addr = 0;
u16 value = 0x1000;
ca_eth_hw_phy_write(phy_addr, reg_addr, value);
```

Parameters conf:0.90 **Q:** What are the valid ranges for the 'phy' and 'reg' parameters in ca_eth_hw_phy_write
A: The 'phy' parameter should be in the range 0-31, and the 'reg' parameter should be in the range 0-31. Values outside these ranges may lead to incorrect register accesses or unexpected behavior, as they are not validated within the function.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_eth_hw_phy_write?
A: Common mistakes include calling the function before Ethernet driver initialization, which may result in undefined behavior or timeouts. Also, using invalid PHY or register addresses (outside 0-31) can cause incorrect writes or access violations. The function is blocking and not thread-safe, so calling it from an ISR or without proper synchronization can lead to system lockups or race conditions.

HW differences conf:0.90 **Q:** How does ca_eth_hw_phy_write differ between F7xx and H5xx hardware implementations?
A: In F7xx, the function uses ETH->MACMIIAR and ETH->MACMIIDR registers for MDIO communication, with the write operation indicated by setting the ETH_MACMIIAR_MW bit. In H5xx, it uses ETH->MACMDIOAR and ETH->MACMDIODR, with the write operation indicated by setting the MDIO_MOC_WR bit. The register bit positions and names differ between families, reflecting the distinct Ethernet peripherals in each MCU series.

ca_eth_hw_pool_held_count

Header: caos/drivers/inc/ca_glue_hw.h:128
GUARD: EN_CA_GLUE_HW

```
extern u16 ca_eth_hw_pool_held_count(void); /* returns the number of DMA buffers currently in use */
```

Returns the total number of Ethernet DMA buffers currently occupied by the hardware driver.

Ritorna:
Returns a 16-bit unsigned integer representing the count of buffers currently held by the hardware. Returns 0 on platforms that do not use a dynamic buffer pool (like F4xx, F7xx) or if no buffers are currently in use.

Note:

- This function is non-blocking and thread-safe as it performs a read operation on internal tracking structures. It is safe to call from an ISR, as it typically performs a simple loop or returns a constant value without acquiring mutexes. A common beginner mistake is assuming this function will return a non-zero value on all STM32 platforms; always check your specific hardware implementation, as many STM32 series use static internal buffers instead of a dynamic pool. If the returned value consistently stays at the maximum pool size, it indicates a potential memory leak where buffers are being allocated but never released back to the stack.

Uso:

- This function is primarily used for system diagnostics, health monitoring, and debugging network traffic congestion. It is typically called by a monitoring task or a watchdog process to verify that the Ethernet stack is not running out of memory buffers. It is often paired with other network stack status functions to provide a complete picture of the system's communication health. Developers use it to detect if the system is 'stuck' in a state where it cannot process incoming or outgoing packets due to buffer exhaustion.

Esempio:

```
/* Check if the Ethernet driver is running out of buffers */

void monitor_network_health(void) {
    u16 held_buffers = ca_eth_pool_held_count();

    /* If we are holding too many buffers, we might have a leak or a network storm */
    if (held_buffers > 16) {
        /* Log a warning or trigger a soft reset of the network interface */
        // ca_printf("High buffer usage detected: %d\n", held_buffers);
        ca_eth_restart();
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should a developer call ca_eth_hw_pool_held_count() and what is its purpose in Ethernet network monitoring?
A: A developer should call ca_eth_hw_pool_held_count() to monitor the number of DMA buffers currently in use for Ethernet communication. This function is useful for checking resource availability and preventing buffer exhaustion, which could lead to throughput issues or system crashes. It is not intended for use in ISR contexts and should not be called from multiple threads without synchronization.

Code example conf:0.90 **Q:** Can you provide a minimal C code example that uses ca_eth_hw_pool_held_count with caOS types?
A: u16 buffer_count = ca_eth_hw_pool_held_count(); if (buffer_count == 0) { /* All buffers are available */ } else { /* Some buffers are in use */ }

Return conf:0.90 **Q:** What do different return values from ca_eth_hw_pool_held_count() indicate, and what might cause incorrect values?
A: A return value of 0 indicates all DMA buffers are available. A positive value indicates that some buffers are currently in use. If the function returns an incorrect or invalid value, it may indicate an implementation issue or memory management problem. The function is not thread-safe and should not be called from ISR contexts, which could also lead to inconsistent results.

HW differences conf:0.85 **Q:** How does ca_eth_hw_pool_held_count() behave differently across various STM32 families like F7xx, H5xx, and H7xx?
A: For F7xx and H5xx families, ca_eth_hw_pool_held_count() always returns 0 because these MCUs use static internal buffers instead of a managed pool. For H7xx family, the function counts the number of used buffers by iterating through rx_pool_idx and tx_pool_idx arrays, reflecting the actual pool usage. F1xx and F4xx implementations also return 0, but they are not shown in the provided code snippets.

`ca_eth_hw_recv`

Header: `caos/drivers/inc/ca_glue_hw.h:117`
GUARD: `EN_CA_GLUE_HW`

```
1 extern bool ca_eth_hw_recv (u16 *out_idx, u16 new_idx); /* riceve un
     frame e ritorna l'indice del buffer DMA */
```

Retrieves a received Ethernet frame from the hardware DMA buffer pool.

Parametri:

Nome	Tipo	Descrizione
`out_idx`	`u16 *`	Pointer to a variable where the function will store the index of the network buffer containing the received frame.
`new_idx`	`u16`	Index of a fresh, empty network buffer to be used by the hardware for the next incoming frame (used in zero-copy architectures).

Ritorna:
Returns true if a frame was successfully retrieved and copied/swapped into the buffer pool. Returns false if no frame is available (DMA still owns the descriptor) or if an error occurred (e.g., memory allocation failure).

Note:

- This function is not thread-safe and is typically called from the Ethernet task or a high-priority polling loop. It is generally NOT callable from an ISR because it may perform memory allocation or complex buffer management. The behavior varies significantly by hardware: some architectures (like H5xx/H7xx) perform a 'zero-copy' swap where the new_idx buffer is handed to the DMA, while others (like F4xx) perform a memory copy. A common beginner mistake is failing to provide a valid 'new_idx' buffer when the architecture requires it, or ignoring the return value, which leads to memory leaks or buffer exhaustion.

Uso:

- This function is the core of the Ethernet receive path. It is typically called within a loop inside the network stack task. The sequence is: 1) Allocate a new buffer using ca_net_buf_alloc(), 2) Call ca_eth_hw_recv() passing the new buffer, 3) If it returns true, process the frame in 'out_idx' and then free it using ca_net_buf_free(). If it returns false, the 'new_idx' buffer must be freed by the caller to prevent a memory leak.

Esempio:

```
u16 out_idx;
u16 new_idx = ca_net_buf_alloc();

if (new_idx != NET_BUF_INVALID) {
    if (ca_eth_hw_recv(&out_idx, new_idx)) {
        // Frame received successfully in out_idx
        // Process data here...
        ca_net_buf_free(out_idx);
    } else {
        // No frame available, return the unused buffer to the pool
        ca_net_buf_free(new_idx);
    }
} else {
    // Handle allocation failure (e.g., log error or drop frame)
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_eth_hw_recv in a caOS Ethernet driver implementation?

A: You should call ca_eth_hw_recv when you need to receive an Ethernet frame in DMA mode. It is used to retrieve a frame from the DMA buffer and return the index of the buffer that was used. This function is typically called in a loop within a receive task or thread to continuously process incoming frames.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_eth_hw_recv in a caOS application?

A:

```
u16 buf_idx;
bool success = ca_eth_hw_recv(&buf_idx, 0);
if (success) {
    // Process the received frame from buffer buf_idx
}
```

Parameters conf:0.95 **Q:** What do the parameters of ca_eth_hw_recv mean and what are valid values?

A: The first parameter 'out_idx' is a pointer to a u16 that will be filled with the index of the buffer containing the received frame. It must be a valid pointer, otherwise the function fails. The second parameter 'new_idx' is the index of the buffer to be used for the next incoming frame. It must be a valid index between 0 and the maximum number of DMA buffers available.

Return conf:0.90 **Q:** What does ca_eth_hw_recv return and what causes it to return false?

A: ca_eth_hw_recv returns true if a frame was successfully received and the buffer index is available. It returns false if the receive operation failed, which can be due to a buffer not being available, a NULL pointer passed to out_idx, or a hardware error.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_eth_hw_recv?

A: Common mistakes include passing a NULL pointer to out_idx, or passing an invalid new_idx value outside the valid buffer range. Additionally, calling this function without proper hardware initialization can lead to undefined behavior or crashes. The function is also not thread-safe, so concurrent access from multiple threads can cause data corruption.

Behavior conf:0.90 **Q:** Is ca_eth_hw_recv blocking, ISR-safe, or thread-safe?

A: ca_eth_hw_recv is a blocking function, meaning it will wait until a frame is received. It is not recommended for use in ISR contexts due to potential synchronization issues or blocking behavior. It is also not thread-safe, so concurrent calls from multiple threads can lead to unpredictable results.

HW differences conf:0.85 **Q:** How does ca_eth_hw_recv differ between F7xx and H7xx MCU families?

A: In F7xx, ca_eth_hw_recv uses a buffer pointer and length, returning the frame data directly into a buffer. In H7xx, it uses a buffer index swap mechanism where the function returns the index of the buffer that was filled with data and updates the descriptor to point to a new buffer for the next frame. The H7xx version also uses specific registers like DMACRDTPR for updating the RX tail pointer.

`ca_eth_hw_send`

Header: `caos/drivers/inc/ca_glue_hw.h:116`
GUARD: `EN_CA_GLUE_HW`

```
1 extern bool ca_eth_hw_send (u16 buf_idx); /* trasmette il frame nel
     buffer DMA indicato */
```

Transmits a network buffer via the Ethernet DMA controller.

Parametri:

Nome	Tipo	Descrizione
`buf_idx`	`u16`	The index of the network buffer to transmit. Must be a valid index returned by the buffer allocation system; invalid indices will cause the function to return false.

Ritorna:
Returns true if the buffer was successfully queued for transmission, false if the buffer index is invalid, the hardware is busy (timeout), or the driver is not implemented for the current MCU.

Note:

- This function is not thread-safe and should be protected by a mutex or semaphore if called from multiple tasks. It is a blocking function because it waits for the DMA hardware to release the current transmission slot; if the network is congested, this wait can take significant time. Do not call this function from an Interrupt Service Routine (ISR) as the blocking loop will cause system instability. A common beginner mistake is to assume the buffer is free to be reused immediately after this function returns; on some architectures (H5xx/H7xx), the buffer is only freed 'deferred' when the slot is reused later, so you must never modify or free the buffer manually after passing it to this function.

Uso:

- This function is the primary entry point for sending network packets. It is typically used in a network stack task after a packet has been prepared in a buffer. The standard sequence is: allocate a buffer using a network buffer manager, populate the data, and then call this function to hand it over to the hardware. On architectures that perform a 'deferred free', the driver manages the lifecycle of the buffer automatically, while on others (F4xx/F7xx), the buffer is freed immediately after copying the data into an internal static buffer.

Esempio:

```c
/* Example: Sending a packet */
u16 my_buf = ca_net_buf_alloc();
if (my_buf != NET_BUF_INVALID) {
    /* Fill buffer with data */
    ca_net_buf_t *b = ca_net_buf_ptr(my_buf);
    b->data_len = 64;
    /* ... populate b->data ... */

    /* Send the packet */
    if (!ca_eth_hw_send(my_buf)) {
        /* Handle error: transmission failed, free buffer manually */
        ca_net_buf_free(my_buf);
    }
    /* Note: Do not access my_buf after this point unless the driver
    documentation explicitly states it is safe */
}
```

Q&A — fine-tuning review

7/7 pairs · qwen3-coder:latest

Usage conf:0.95 **Q:** When should I call ca_eth_hw_send() in my Ethernet application?

A: You should call ca_eth_hw_send() when you have an Ethernet frame ready to transmit from a DMA buffer. It is typically used after preparing the frame data in a pre-allocated buffer, and you want to initiate the transmission process. The function is non-blocking, so it returns immediately after queuing the frame for transmission.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_eth_hw_send()?

A:

```c
u16 buffer_index = 0;
bool success = ca_eth_hw_send(buffer_index);
if (success) {
    // Transmission started successfully
} else {
    // Handle transmission failure
}
```

Parameters conf:0.95 **Q:** What does the buf_idx parameter in ca_eth_hw_send() represent, and what are valid values?

A: The buf_idx parameter is the index of the DMA buffer containing the Ethernet frame to be sent. Valid values are integers between 0 and the maximum number of configured buffers (exclusive). Using a negative value or a value greater than or equal to the maximum buffer count leads to undefined behavior.

Return conf:0.95 **Q:** What do the return values of ca_eth_hw_send() indicate?

A: The function returns true if the transmission was successfully initiated, meaning the frame was queued for sending. It returns false if the transmission failed, which may be due to an invalid buffer index, uninitialized hardware, or insufficient resources.

Error/Edge case conf:0.90 **Q:** What are common mistakes or error conditions when calling ca_eth_hw_send()?

A: Common errors include calling the function with an invalid buffer index (outside the valid range), calling it before initializing the Ethernet hardware with ca_eth_hw_init(), or using it in an ISR context without proper synchronization. These can lead to crashes or undefined behavior.

Behavior conf:0.90 **Q:** Is ca_eth_hw_send() thread-safe and ISR-safe?

A: No, ca_eth_hw_send() is not guaranteed to be thread-safe and should not be called from an ISR. It is a non-blocking function that returns immediately after initiating the transmission, but it does not handle synchronization internally, so external mechanisms are required if used in multi-threaded or interrupt contexts.

HW differences conf:0.85 **Q:** How does ca_eth_hw_send() differ between STM32 H7xx and H5xx families in terms of implementation?

A: In H7xx, ca_eth_hw_send() uses a deferred free mechanism where it directly points

the descriptor to the buffer without copying, and it saves the buffer index for later freeing. In H5xx, it copies the frame data into the buffer and sets up the descriptor with specific fields like TDES3_FD, TDES3_LD, and TDES3_CIC_FULL. The H7xx uses ETH->DMACTDTPR to advance the TX tail pointer, while H5xx uses a similar mechanism but with different descriptor fields.

---

ca_eth_iface_get

Header: caos/net/inc/eth_iface.h:36
Source: caos/net/src/eth_iface.c:75:103
GUARD: EN_ETH_IFACE

```
ca_net_iface_t *ca_eth_iface_get(ca_eth_cfg_t *cfg)
```

Initializes and returns a pointer to the Ethernet network interface structure.

Parametri:

Nome	Tipo	Descrizione
cfg	ca_eth_cfg_t *	Pointer to the Ethernet configuration structure containing hardware settings and MAC address. Must not be NULL.

Ritorna:
Returns a pointer to a static ca_net_iface_t structure on success, or NULL if the input configuration pointer is invalid.

Note:

- This function is a factory that prepares the Ethernet interface for the network stack. It is not thread-safe and should only be called once during the system initialization phase, typically before the RTOS scheduler starts. It is not callable from an ISR. A common mistake is to pass a NULL pointer or a pointer to a local variable that goes out of scope, which would lead to undefined behavior when the network stack attempts to access the interface configuration later. Note that this function overwrites the user-provided callback pointers in the cfg structure with internal caOS ISR handlers, so do not attempt to manually override these callbacks after calling this function.

Uso:

- This function is used exclusively during the hardware and network stack initialization sequence. It is typically called immediately after configuring the physical GPIO pins for the Ethernet peripheral and before calling ca_net_init. The returned interface pointer is passed directly to ca_net_init to register the Ethernet driver with the system's networking layer. It is part of the standard startup pattern: configure hardware -> get interface -> initialize network stack -> start services.

Esempio:

```
ca_eth_cfg_t eth_cfg = { .mac = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55} };

// Initialize hardware pins first
eth_gpio_init();

// Get the interface and register it with the network stack
ca_net_iface_t *iface = ca_eth_iface_get(&eth_cfg);
if (iface == NULL) {
    // Handle error: configuration pointer was invalid
    while(1);
```

```
}

// Pass the interface to the network stack initialization
if (!ca_net_init(iface)) {
    // Handle error: network stack failed to initialize
    while(1);
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_eth_iface_get() in my caOS project?
A: You should call ca_eth_iface_get() during the initialization phase of your network stack, before calling ca_net_init(). It is used to obtain a configured Ethernet interface structure that will be passed to ca_net_init() to initialize the networking subsystem.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_eth_iface_get() with caOS types?
A: ca_eth_cfg_t cfg = { .mac = {0x00, 0x80, 0xE1, 0x23, 0x45, 0x67}, .rx_cb = my_rx_handler, .tx_done_cb = my_tx_done_handler, .error_cb = my_error_handler }; ca_net_iface_t *iface = ca_eth_iface_get(&cfg); if (iface != NULL) { ca_net_init(iface); }

Parameters conf:0.95 **Q:** What does the cfg parameter in ca_eth_iface_get() represent and what happens if it's NULL?
A: The cfg parameter is a pointer to a configuration structure for the Ethernet interface. It must be a valid, initialized structure containing MAC address and callback functions. If cfg is NULL, the function returns NULL immediately without performing any operation.

Return conf:0.95 **Q:** What does ca_eth_iface_get() return when it succeeds and when it fails?
A: On success, ca_eth_iface_get() returns a pointer to a ca_net_iface_t structure that is ready to be passed to ca_net_init(). On failure, it returns NULL, which can happen if cfg is NULL or if the configuration is invalid.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_eth_iface_get()?
A: Common mistakes include passing NULL as the cfg parameter, not checking the return value of ca_eth_iface_get(), and calling this function multiple times with the same cfg structure, which causes overwriting of the rx_cb, tx_done_cb, and error_cb fields.

Behavior conf:0.90 **Q:** Is ca_eth_iface_get() thread-safe and can it be called from an ISR?
A: No, ca_eth_iface_get() is not thread-safe because it uses a static variable internally. It also cannot be called from an ISR, as it is an initialization function and not ISR-safe. The function is non-blocking and returns immediately.

`ca_eth_init`

Header: `caos/drivers/inc/eth.h:83`
Source: `caos/drivers/src/eth.c:70:80`
GUARD: `EN_ETH`

```
bool ca_eth_init (const ca_eth_cfg_t *cfg)
```

Initializes the Ethernet hardware peripheral and registers the Ethernet CLI interface.

Parametri:

Nome	Tipo	Descrizione
`cfg`	`const ca_eth_cfg_t *`	Pointer to a configuration structure containing hardware settings and callback functions. Must not be NULL; passing a NULL pointer will cause the function to return false immediately.

Ritorna:
Returns true if the hardware initialization was successful, false otherwise. A false return indicates either an invalid configuration pointer or a failure in the underlying hardware initialization sequence.

Note:

- This function is non-blocking and is intended to be called during the system startup phase. It is not thread-safe and must not be called from an Interrupt Service Routine (ISR) because it performs hardware-level setup that may involve timing-sensitive operations. A common beginner mistake is failing to initialize the required GPIO pins or the network buffer pool before calling this function, which will lead to undefined behavior or hardware initialization failure. Ensure that the configuration structure remains valid throughout the initialization process as the driver stores a local copy of the settings.

Uso:

- This function is typically called once during the system boot sequence, immediately after configuring the necessary GPIO pins and initializing the network buffer pool. It is often paired with ca_net_buf_pool_init to ensure the zero-copy memory management system is ready for incoming and outgoing packets. In more complex applications, it may be used in conjunction with a state-tracking variable to decide between a fresh initialization or a restart if the network interface needs to be reset due to a link failure.

Esempio:

```
ca_eth_cfg_t eth_cfg = { .link_cb = my_link_callback };

// 1. Initialize hardware dependencies first
eth_gpio_init();
ca_net_buf_pool_init();

// 2. Initialize the Ethernet driver
if (!ca_eth_init(&eth_cfg)) {
    // Handle initialization failure
    while(1);
}

// 3. Proceed to start network tasks
ca_os_job("eth_mon", CA_TH_LEVEL3, task_eth, 0);
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should ca_eth_init be called in a caOS application, and why is it important?
A: ca_eth_init should be called during the system initialization phase, typically before any network operations begin. It prepares the Ethernet hardware and registers the necessary callbacks for handling link status, packet reception, and errors. This function is essential to initialize the Ethernet driver and make it ready for use in a caOS application.

Code example conf:0.90 **Q:** Can you provide a minimal working example of how to use ca_eth_init with caOS types?
A: ca_eth_cfg_t cfg = { .link_cb = my_link_callback, .rx_cb = my_rx_callback, .err_cb = my_err_callback }; bool success = ca_eth_init(&cfg); if (!success) { /* Handle initialization failure */ }

Parameters conf:0.95 **Q:** What is the significance of the cfg parameter in ca_eth_init, and what happens if it is NULL?
A: The cfg parameter is a pointer to a configuration structure that contains the callback functions for handling link events, packet reception, and errors. If cfg is NULL, the function immediately returns false, indicating initialization failure due to invalid input.

Return conf:0.90 **Q:** What do the return values of ca_eth_init indicate, and what causes it to return false?
A: ca_eth_init returns true if initialization is successful. It returns false if the cfg parameter is NULL or if hardware initialization fails (via ca_eth_hw_init). This indicates that the Ethernet driver could not be properly initialized.

Error/Edge case conf:0.85 **Q:** What are some common mistakes or edge cases when using ca_eth_init that could lead to failure?
A: Common mistakes include passing a NULL cfg parameter, which causes an immediate return of false. Another issue is incorrect initialization of the Ethernet buffer pool, which can lead to errors during packet reception or transmission. Additionally, calling ca_eth_init from an ISR is not allowed and can cause undefined behavior.

`ca_eth_link_poll`

Header: `caos/drivers/inc/eth.h:118`
Source: `caos/drivers/src/eth.c:241:249`
GUARD: `EN_ETH`

```
bool ca_eth_link_poll(void)
```

Checks the physical Ethernet link status and triggers a callback if the state has changed.

Ritorna:
Returns true if the Ethernet link is currently detected as UP, and false if it is DOWN.

Note:

- This function is non-blocking and performs a hardware register read to determine the link status. It is thread-safe in the sense that it updates a static internal state variable, but it should not be called from an ISR because it triggers a user-defined callback function which might contain complex logic or OS primitives. A common mistake is calling this function too frequently in a tight loop without a delay, which can waste CPU cycles; it is intended to be called periodically (e.g., every 500ms or 1s) within a background task. Ensure the Ethernet hardware driver is fully initialized before calling this, otherwise, the underlying hardware access function may trigger a hard fault.

Uso:

- This function is used to monitor the physical connection status of the Ethernet cable. It is typically called during the system initialization phase to verify the link before sending network packets (like Gratuitous ARP) and then periodically inside a main loop or a dedicated network management task. It is often paired with ca_os_sleep to create a polling interval and with network protocol initialization functions like ca_net_proto_init to ensure the stack is ready to handle traffic once the link is confirmed.

Esempio:

```
void eth_monitor_task(void *arg) {
    /* Initialize network protocol stack first */
    ca_net_proto_init(NULL);

    while (1) {
        /* Poll the physical link status */
        bool up = ca_eth_link_poll();

        if (up) {
            /* If link just came up, announce presence via GARP */
            ca_arp_send_garp();
```

```
        }

        /* Wait 500ms to avoid excessive CPU usage */
        ca_os_sleep(500);
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_eth_link_poll() and why is it important for network monitoring?
A: You should call ca_eth_link_poll() to check the current state of the Ethernet link, especially when you need to monitor network connectivity. It is non-blocking and can be called from any context, including ISRs, making it suitable for real-time systems. It notifies changes in link status via a registered callback function, which is essential for maintaining reliable network communication.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_eth_link_poll() with caOS types?
A: void example_eth_link_check(void) { bool link_status = ca_eth_link_poll(); if (link_status) { // Link is UP } else { // Link is DOWN } }

Return conf:0.90 **Q:** What do the return values of ca_eth_link_poll() indicate, and what can cause the function to fail?
A: The function returns true if the Ethernet link is active (UP), and false if it is inactive (DOWN) or if an internal error occurs. Failures may arise from inability to access hardware, which could lead to incorrect return values or even system crashes. The function modifies a global variable (s_link_up) and calls a callback registered via ca_eth_link_set_callback(), so improper setup can also cause issues.

`ca_eth_phy_read`

Header: `caos/drivers/inc/eth.h:112`
Source: `caos/drivers/src/eth.c:231:234`
GUARD: `EN_ETH`

```
u16 ca_eth_phy_read (u8 phy, u8 reg)
```

Reads a 16-bit value from a specific register of an Ethernet PHY chip via the MDIO interface.

Parametri:

Nome	Tipo	Descrizione
`phy`	`u8`	The PHY address on the MDIO bus, typically ranging from 0 to 31.
`reg`	`u8`	The specific register address within the PHY chip to read from, usually 0 to 31.

Ritorna:
Returns a 16-bit unsigned integer containing the data read from the PHY register. If the hardware communication fails or the PHY is unresponsive, the behavior depends on the underlying hardware driver, which may return 0xFFFF or 0x0000.

Note:

- This function is a blocking call, meaning the CPU will wait for the MDIO transaction to complete before returning to your code. It is generally not thread-safe if multiple tasks attempt to access the MDIO bus simultaneously; you should protect calls to this function with a mutex if your application has multiple tasks interacting with the Ethernet hardware.

Do not call this function from an Interrupt Service Routine (ISR) because the blocking nature will cause significant latency and potentially crash the system. A common beginner mistake is calling this function before the Ethernet hardware has been properly initialized, which will result in invalid data or a system hang.

Uso:

- This function is used during the Ethernet initialization sequence to verify the link status, read the PHY ID, or configure auto-negotiation settings. It is typically paired with ca_eth_phy_write to perform read-modify-write operations on configuration registers. You should use it after the Ethernet hardware is powered up and the MDIO clock is configured, but before attempting to transmit or receive network packets. It is often used in a polling loop to monitor the link status register until the PHY reports that a cable is connected and the link is up.

Esempio:

```c
/* Example: Checking the PHY Link Status register (Register 1) */
#define PHY_ADDR 0x01
#define PHY_REG_STATUS 0x01

void check_link(void) {
    /* Ensure Ethernet is initialized before accessing PHY */
    u16 status = ca_eth_phy_read(PHY_ADDR, PHY_REG_STATUS);

    /* Check if the link status bit (bit 2) is set */
    if (status & 0x0004) {
        /* Link is established */
    } else {
        /* Link is down, perform link polling */
        ca_eth_link_poll();
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_eth_phy_read and why is it important for Ethernet configuration?
A: You should call ca_eth_phy_read to read configuration or status registers from the Ethernet PHY chip. This is important for initializing or monitoring the PHY state, such as checking link status, speed, or duplex settings. It's typically used during Ethernet initialization or diagnostics.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_eth_phy_read to read the PHY status register?
A: u16 status = ca_eth_phy_read(0, 1); // Read register 1 (status) from PHY address 0 if (status == 0xFFFF) { // Handle read error }

Parameters conf:0.90 **Q:** What are the valid ranges for the phy and reg parameters in ca_eth_phy_read, and what happens if I pass invalid values?
A: The phy parameter must be between 0 and 31 (inclusive), and reg must be between 0 and 31 (inclusive). Values of 0xFF for either parameter are invalid and may lead to undefined behavior or hardware errors.

Return conf:0.95 **Q:** What does it mean if ca_eth_phy_read returns 0xFFFF, and what causes this failure?
A: A return value of 0xFFFF indicates a read error, which can be caused by a hardware timeout or the PHY not responding. This function uses an internal hardware timeout to avoid indefinite blocking, so 0xFFFF signals that the operation could not complete successfully.

ca_eth_phy_write

Header: caos/drivers/inc/eth.h:113
Source: caos/drivers/src/eth.c:236:239
GUARD: EN_ETH

```
void ca_eth_phy_write(u8 phy, u8 reg, u16 val)
```

Writes a 16-bit value to a specific register of an Ethernet PHY chip via the MDIO interface.

Parametri:

Nome	Tipo	Descrizione
phy	u8	The PHY address on the MDIO bus, typically ranging from 0 to 31.
reg	u8	The specific register address within the PHY chip to be written.
val	u16	The 16-bit data value to be stored in the target register.

Ritorna:
This function returns void and does not provide feedback on the success or failure of the hardware transaction.

Note:

- This function is a wrapper for hardware-level operations and is generally blocking, meaning the CPU will wait until the MDIO transaction completes before returning. It is not inherently thread-safe; if multiple tasks attempt to write to the PHY simultaneously, the MDIO bus state may become corrupted, leading to unpredictable hardware behavior. It is generally not recommended to call this from an ISR due to the potential latency of the MDIO bus communication. A common beginner mistake is assuming the write is instantaneous or ignoring the fact that some PHY registers require a specific sequence or delay after a write to take effect.

Uso:

- This function is used during the Ethernet driver initialization phase to configure PHY settings, such as auto-negotiation modes, speed, or duplex settings. It is typically paired with ca_eth_phy_read when performing a read-modify-write operation to update specific bits in a register without affecting others. In a typical lifecycle, you would use this during the driver startup sequence to bring the link up, and occasionally during runtime if the link status requires a manual reset or reconfiguration of the PHY hardware.

Esempio:

```
/* Example: Configuring the PHY to force 100Mbps Full Duplex */
#define PHY_ADDR 0x01
#define BMCR_REG 0x00
#define BMCR_FORCE_100_FD 0x2100

void setup_ethernet_link(void) {
    /* Read current configuration */
    u16 reg_val = ca_eth_phy_read(PHY_ADDR, BMCR_REG);

    /* Modify bits to force speed and duplex */
    reg_val |= BMCR_FORCE_100_FD;

    /* Write back the updated configuration to the PHY */
    ca_eth_phy_write(PHY_ADDR, BMCR_REG, reg_val);

```

```
    /* Ensure the link is polled after configuration */
    bool link_status = ca_eth_link_poll();
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_eth_phy_write() in my caOS project, and what is its primary purpose?
A: You should use ca_eth_phy_write() when you need to configure or modify the settings of an Ethernet PHY chip connected to your STM32 microcontroller. Its primary purpose is to write a 16-bit value to a specific register of a given PHY address. This function is typically used during initialization or configuration of the Ethernet interface, such as setting up PHY modes or enabling specific features.

Code example conf:0.90 **Q:** Can you provide a minimal C code example that demonstrates how to use ca_eth_phy_write() with caOS types?
A: u8 phy_addr = 0; u8 reg_addr = 1; u16 value = 0x1234; ca_eth_phy_write(phy_addr, reg_addr, value);

Parameters conf:0.90 **Q:** What are the valid ranges for the 'phy' and 'reg' parameters in ca_eth_phy_write(), and what happens if invalid values are passed?
A: The 'phy' parameter must be a value between 0 and 31 inclusive. The 'reg' parameter must also be between 0 and 31 inclusive. If either parameter is less than 0 or greater than 31, the behavior is undefined and could lead to incorrect hardware access or failure to communicate with the PHY.

`ca_eth_pool_held_count`

Header: `caos/drivers/inc/eth.h:130`
Source: `caos/drivers/src/eth.c:256:259`
GUARD: `EN_ETH`

```
u16 ca_eth_pool_held_count(void)
```

Returns the number of network buffers currently held by the Ethernet hardware driver.

Ritorna:
Returns a 16-bit unsigned integer representing the count of buffers currently locked by the Ethernet DMA descriptors or hardware processing logic.

Note:

- This function is non-blocking and thread-safe, as it performs a simple read of a counter maintained by the Ethernet driver. It is safe to call from an ISR, provided the underlying hardware register access is atomic. The primary purpose of this function is diagnostic; it allows the system to account for buffers that are 'in-flight' (being processed by the hardware) and therefore not available in the general free pool. A common mistake is to assume that the sum of free buffers and held buffers always equals the total pool size; if this sum is less than the total, it indicates a memory leak where buffers were allocated but never returned to the pool.

Uso:

- This function is primarily used for system health monitoring and debugging memory leaks within the network stack. It is typically paired with 'ca_net_buf_free_count()' to calculate the total number of buffers currently in use by the application versus those held by the hardware driver. Developers use this in CLI commands or periodic status logging to verify that the network stack is not leaking memory under high traffic conditions. It

is essential for zero-copy network implementations where the driver retains ownership of buffers until the DMA transfer is complete.

Esempio:

```
/* Check for potential memory leaks in the network buffer pool */

void check_network_health(void) {
    u16 free_bufs = ca_net_buf_free_count();
    u16 held_bufs = ca_eth_pool_held_count();
    /* NET_BUF_COUNT is a constant defined in the system configuration */
    u16 total_bufs = (u16)NET_BUF_COUNT;

    /* Calculate if any buffers are 'lost' (leaked) */
    if ((free_bufs + held_bufs) < total_bufs) {
        u16 leaked = (u16)(total_bufs - free_bufs - held_bufs);
        ca_printf_n("Warning: %u buffers leaked!", (u32)leaked);
        /* Trigger a debug dump to inspect hardware state */
        ca_eth_dbg_dump();
    } else {
        ca_printf_n("System healthy: %u free, %u held.", (u32)free_bufs, (u32)
    held_bufs);
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_eth_pool_held_count() and why is it useful for debugging?
A: You should call ca_eth_pool_held_count() to monitor the number of network buffers currently in use by the Ethernet driver. It is useful for debugging and detecting potential memory leaks, as a consistently high or increasing value may indicate that buffers are not being properly released. The function can be called from any context, including ISRs, and is non-blocking.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_eth_pool_held_count() with caOS types?
A: u16 buffer_count = ca_eth_pool_held_count(); if (buffer_count == 0) { // No buffers in use } else if (buffer_count >= NET_BUF_COUNT) { // All buffers in use, potential issue } else { // Some buffers in use, normal operation }

Return conf:0.90 **Q:** What do the return values of ca_eth_pool_held_count() mean and what causes the function to return 0?
A: The function returns a u16 value representing the number of network buffers currently acquired by the Ethernet driver. A return value of 0 means no buffers are currently in use. A value equal to NET_BUF_COUNT indicates all buffers are in use. The function returns 0 in case of an internal error, which is rare.

`ca_eth_recv`

Header: `caos/drivers/inc/eth.h:106`
Source: `caos/drivers/src/eth.c:220:224`
GUARD: `EN_ETH`

```
bool ca_eth_recv (u16 *out_idx, u16 new_idx)
```

Retrieves a received network packet from the hardware buffer and replaces it with a new empty buffer.

Parametri:

Nome	Tipo	Descrizione
`out_idx`	`u16 *`	Pointer to a variable where the index of the received packet buffer will be stored. Must not be NULL.
`new_idx`	`u16`	The index of a pre-allocated empty buffer that the hardware will use to replace the one being retrieved. Must be a valid buffer index.

Ritorna:
Returns true if a packet was successfully retrieved and the buffer was swapped. Returns false if no packet is available, if the pointer is NULL, or if the hardware operation fails.

Note:

- This function is typically non-blocking, meaning it checks the hardware status and returns immediately if no data is present. It is not inherently thread-safe, so ensure that access to the Ethernet hardware is protected if multiple tasks or interrupts interact with the network stack. A common beginner mistake is passing an uninitialized or invalid buffer index for 'new_idx', which can lead to memory corruption or hardware crashes. Always ensure the Ethernet driver has been initialized via ca_eth_init before calling this function, otherwise, the hardware registers may be in an undefined state.

Uso:

- This function is the core of the Ethernet receive loop. It is usually called after the system detects an interrupt or a polling flag indicating that new data has arrived. The typical sequence is: first, allocate a new empty buffer using ca_net_buf_alloc; second, call ca_eth_recv to swap the full buffer with the empty one; third, process the data in the retrieved buffer using ca_net_buf_ptr; and finally, release the buffer using ca_net_buf_free once processing is complete. It is often wrapped inside a network interface abstraction layer to provide a consistent API for higher-level protocols.

Esempio:

```
u16 new_idx = ca_net_buf_alloc();
if (new_idx != 0xFFFF) { /* Assuming 0xFFFF is the invalid index constant */
    u16 out_idx;
    /* Attempt to retrieve a packet from the hardware */
    if (ca_eth_recv(&out_idx, new_idx)) {
        /* Packet successfully retrieved, process it */
        ca_net_buf_t *b = ca_net_buf_ptr(out_idx);
        /* Perform packet processing here */
        ca_net_buf_free(out_idx);
    } else {
        /* No packet available, return the unused buffer to the pool */
        ca_net_buf_free(new_idx);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_eth_recv in my RTOS task, and what is its primary purpose?
A: You should call ca_eth_recv from an RTOS task to receive Ethernet frames in zero-copy mode using DMA. It is designed for blocking reception of frames, meaning the task will suspend until a frame arrives. This function is used when you need to process incoming Ethernet packets in a thread-safe manner, and it must not be called from an ISR.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_eth_recv with caOS types?

A: u16 buffer_idx; bool success = ca_eth_recv(&buffer_idx, 0); if (success) { // Process the received frame from buffer_idx }

Parameters conf:0.90 **Q:** What does the 'new_idx' parameter in ca_eth_recv represent, and what are valid values for it?

A: The 'new_idx' parameter specifies the index of a pre-allocated buffer to be used for receiving the Ethernet frame. It must be a valid buffer index previously allocated using ca_net_buf_alloc(), and it must be between 0 and the maximum buffer count allowed by the network buffer pool. Using an unallocated buffer or an invalid index will cause the function to return false.

Return conf:0.90 **Q:** What do the return values of ca_eth_recv indicate, and what causes it to return false?

A: ca_eth_recv returns true if a frame was successfully received and stored in the buffer indicated by out_idx. It returns false if no frame is available, an error occurred (such as passing NULL for out_idx), or if the buffer index is invalid. The function may also block if no frames are available, but it will not return false due to blocking unless an error condition is met.

`ca_eth_restart`

Header: `caos/drivers/inc/eth.h:125`
Source: `caos/drivers/src/eth.c:227:245`
GUARD: `EN_ETH`

```
bool ca_eth_restart(void)
```

Performs a full hardware reset and re-initialization of the Ethernet MAC and DMA descriptors.

Ritorna:
Returns true if the Ethernet hardware was successfully re-initialized, false otherwise.

Note:

- This function is a heavy-duty recovery mechanism. It is not thread-safe and must not be called from an ISR, as it performs blocking hardware operations and memory management. It is intended to be used when the Ethernet interface becomes unresponsive or enters an error state. A common beginner mistake is calling this function too frequently or during normal operation; it should only be used as a last resort for recovery, as it forces a complete teardown and rebuild of the RX/TX descriptor rings. You must ensure that no other tasks are actively using the Ethernet interface when this is called, or you will cause race conditions and memory corruption.

Uso:

- This function is typically used in two scenarios: as a manual recovery command via a CLI interface when the network stack hangs, or as an automatic recovery step within a network interface management layer. It is usually paired with ca_eth_init() for the initial setup. The typical sequence is: check if the interface is initialized, and if it is, call ca_eth_restart() to recover from a fault. It is often used in conjunction with ca_eth_link_poll() to verify if the link returns to an 'up' state after the restart.

Esempio:

```
/* Example: Manual recovery via CLI command */
if (ca_strcmp(command, "reset_eth") == 0) {
    /* Notify the system that we are attempting a recovery */
    ca_printf("Attempting Ethernet hardware restart...\n");
```

```
5
6       /* Perform the restart. This will clear descriptors and re-init the MAC */
7       bool success = ca_eth_restart();
8
9       if (success) {
10          ca_printf("Ethernet restart successful.\n");
11      } else {
12          /* If it fails, the hardware might be physically damaged or disconnected
        */
13          ca_printf("Ethernet restart FAILED!\n");
14      }
15  }
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_eth_restart() and why is it needed after a PHY reset?
A: You should call ca_eth_restart() after manually resetting the PHY to restore the Ethernet interface connection. It is needed because the PHY reset may disrupt the hardware state, and this function reinitializes the Ethernet hardware to recover the network connection. It must only be used when the PHY has been manually reset and the configuration has been saved in s_saved_cfg.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_eth_restart() with caOS types?
A: bool result = ca_eth_restart(); if (result) { // Ethernet interface successfully restarted } else { // Restart failed, handle error }

Return conf:0.90 **Q:** What do the return values of ca_eth_restart() indicate and what causes a failure?
A: ca_eth_restart() returns true if the restart was successful, and false if there was an error during hardware initialization. Failures typically occur when the function is called without first saving the configuration in s_saved_cfg, which is a required prerequisite for proper hardware reinitialization.

`ca_eth_send`

Header: `caos/drivers/inc/eth.h:99`
Source: `caos/drivers/src/eth.c:206:218`
GUARD: `EN_ETH`

```
1  i32 ca_eth_send (u16 buf_idx)
```

Transmits a network buffer identified by its index over the Ethernet hardware interface.

Parametri:

Nome	Tipo	Descrizione
`buf_idx`	`u16`	The index of the buffer in the system memory pool. Must be a valid index within the range [0, NET_BUF_COUNT - 1]. Passing an invalid index will result in an error or potential system instability.

Ritorna:
Returns CA_ETH_SEND_OK (0) on success. Returns CA_ETH_SEND_ERR_LINK if the network link is down, or CA_ETH_SEND_ERR_TX if the hardware transmission queue is full or failed.

Note:

- This function is a wrapper that enforces ownership rules: the driver takes ownership of the buffer upon entry. If the function returns an error, the driver automatically frees the buffer to prevent memory leaks, so the caller must NOT free it again. It is non-blocking and designed to be called from the network stack context. It is not safe to call from an ISR unless the underlying hardware driver explicitly supports it. The most common beginner mistake is attempting to reuse or free the buffer after a successful call, which leads to double-free corruption or data race conditions.

Uso:

- This function is the primary entry point for sending data from the network stack to the physical Ethernet hardware. It is typically used in a sequence where the application or protocol layer (like ARP or TCP) allocates a buffer, populates it with data, and passes it to this function. It is often paired with ca_net_buf_alloc for preparation and ca_eth_recv for handling incoming traffic. In the event of a link-down state, the function safely handles the buffer cleanup, ensuring that the system does not leak memory during network outages.

Esempio:

```
u16 idx = ca_net_buf_alloc();
if (idx < NET_BUF_COUNT) {
    /* Fill buffer with packet data */
    // ... (data filling logic) ...

    /* Attempt to send the packet */
    i32 rc = ca_eth_send(idx);

    /* If the function returns an error, the buffer is already freed by the driver.
       If it returns CA_ETH_SEND_OK, the driver now owns the buffer and will free it. */
    if (rc != CA_ETH_SEND_OK) {
        ca_printf_n("Failed to send packet, error code: %d", rc);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_eth_send and what is its purpose in an embedded Ethernet application?
A: You should call ca_eth_send to transmit an Ethernet packet using the hardware driver. It is used when you have prepared data in a network buffer and want to send it over the Ethernet interface. The function is non-blocking and returns immediately after initiating the transmission, without waiting for the packet to be fully sent.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_eth_send with caOS types?
A: u16 buffer_index = 0; bool success = ca_eth_send(buffer_index); if (!success) { // Handle send failure ca_net_buf_free(buffer_index); }

Parameters conf:0.95 **Q:** What does the buf_idx parameter represent in ca_eth_send, and what are the valid values?
A: The buf_idx parameter is the index of the buffer containing the Ethernet data to be sent. It must be a valid index between 0 and NET_BUF_COUNT-1. Using a value outside this range will cause the function to return false, indicating an invalid buffer index.

Return conf:0.90 **Q:** What do the return values of ca_eth_send indicate, and what causes it to return false?
A: ca_eth_send returns true if the packet transmission was initiated successfully; the buffer is then automatically freed by the driver. It returns false if the send fails, which

can happen due to an invalid buf_idx or if the driver is not ready. In case of failure, the caller must manually free the buffer using ca_net_buf_free().

---

ca_event_clear

Header: caos/inc/event.h:102
Source: caos/src/event.c:139:151
GUARD: EN_EVENT

```
u32 ca_event_clear(ca_event_group_t *eg, u32 bits_to_clear)
```

Clears specific bits in an event group and returns the updated state of all bits.

Parametri:

Nome	Tipo	Descrizione
eg	ca_event_group_t *	Pointer to the event group structure. Must be initialized before use; passing NULL will result in a return value of 0.
bits_to_clear	u32	A bitmask where each bit set to 1 corresponds to the bit position in the event group that should be cleared (set to 0).

Ritorna:
Returns the new state of the event group bits after the clear operation. Returns 0 if the provided event group pointer is NULL.

Note:

- This function is thread-safe because it uses a spinlock to protect the internal bitmask during the read-modify-write operation. It is non-blocking, meaning it does not put the calling task into a waiting state, but it does briefly disable interrupts to ensure atomicity. While it is safe to call from a task, you should use ca_event_clear_isr if you are inside an interrupt service routine to avoid potential deadlocks or nested lock issues. The most common beginner mistake is passing a bitmask that clears bits you actually intended to keep; always verify your mask logic. Ensure the event group is initialized via ca_event_init before any calls, or the system will likely trigger a hard fault when accessing the uninitialized spinlock.

Uso:

- This function is used to acknowledge that a specific event or condition has been handled by the application logic. It is typically used in a loop where a task calls ca_event_wait to block until a condition is met, processes the data, and then calls ca_event_clear to reset the flag so the task can wait for the next occurrence. It is the standard way to transition an event group from a 'signaled' state back to an 'idle' state for specific bits, ensuring that the same event does not trigger the task multiple times unnecessarily.

Esempio:

```
ca_event_group_t *my_event_group;

// Initialize the event group before use
ca_event_init(my_event_group);

// Wait for bit 0 to be set by another task or ISR
ca_event_wait(my_event_group, (1 << 0), 1, 1000);

// Process the event logic here...

```

```
// Clear bit 0 to acknowledge the event and reset the state
u32 remaining_bits = ca_event_clear(my_event_group, (1 << 0));

if (remaining_bits == 0) {
    // No other events are currently pending
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_event_clear to manage event groups in caOS?
A: You should use ca_event_clear when you need to reset specific bits in an event group after processing them. This is useful for managing state flags without affecting other bits in the group. It's commonly used in interrupt service routines or task contexts where events are signaled and then cleared.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_event_clear in a caOS application?
A: ca_event_group_t eg; ca_event_init(&eg); ca_event_set(&eg, 0x03); // Set bits 0 and 1 u32 result = ca_event_clear(&eg, 0x01); // Clear bit 0 // result will be 0x02, indicating bit 1 is still set.

Parameters conf:0.90 **Q:** What does the 'bits_to_clear' parameter in ca_event_clear represent and what are valid values?
A: The 'bits_to_clear' parameter is a bitmask where each bit set to 1 indicates a bit in the event group that should be cleared. Valid values are any u32 value, where 1s in the mask correspond to bits to clear. For example, 0x03 clears bits 0 and 1, while 0x0F clears bits 0 through 3.

Return conf:0.95 **Q:** What does ca_event_clear return and what causes it to return 0?
A: ca_event_clear returns the new value of the event group bits after clearing. It returns 0 if the input pointer 'eg' is NULL, indicating an invalid or unallocated event group. Otherwise, it returns the updated bit value of the event group.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_event_clear?
A: Common mistakes include passing a NULL pointer to 'eg', which causes the function to return 0 without error. Another is confusing 'clear' with 'reset'—clear only removes specified bits, not all bits. Additionally, calling ca_event_clear from an ISR without ensuring the underlying lock mechanism is ISR-safe may lead to undefined behavior.

`ca_event_clear_isr`

Header: `caos/inc/event.h:118`
Source: `caos/src/event.c:158:161`
GUARD: `EN_EVENT`

```
u32 ca_event_clear_isr(ca_event_group_t *eg, u32 bits_to_clear)
```

Clears specific bits in an event group, designed specifically for use within Interrupt Service Routines (ISRs).

Parametri:

Nome	Tipo	Descrizione
`eg`	`ca_event_group_t *`	A pointer to the event group handle. Must be a valid, initialized pointer; passing NULL will cause a system crash or undefined behavior.
`bits_to_clear`	`u32`	A bitmask where each bit set to 1 corresponds to the bit in the event group that should be cleared (set to 0). Bits set to 0 in this mask remain unchanged.

Ritorna:
Returns the state of the event group bits after the clear operation is performed. If the function fails due to an invalid handle, it typically returns 0.

Note:

- This function is explicitly designed to be safe for calling from an ISR because it uses internal spinlocks to disable interrupts, preventing race conditions between the main thread and the interrupt. Unlike standard blocking functions, this is non-blocking and executes immediately, making it suitable for high-priority timing requirements. A common beginner mistake is to call this function with a NULL pointer or to assume it will trigger a task reschedule immediately; while it clears the bits, it does not force a context switch. Always ensure the event group has been properly initialized via ca_event_init before attempting to clear bits from an interrupt.

Uso:

- This function is typically used in an ISR to acknowledge that a hardware event has been processed, allowing a waiting task to stop blocking. It is usually paired with ca_event_wait or ca_event_wait_any, where a task is suspended waiting for specific bits to be set. Once the ISR clears the bits, the system state is updated, and the task waiting on those bits may be unblocked if the logic requires it. It is the standard way to communicate from hardware-triggered events back to the RTOS task management system, ensuring that the event group state remains consistent across different execution contexts.

Esempio:

```
/* Global handle for the event group */
caEventGroup *my_event_group;

/* ISR triggered by a hardware timer or GPIO */
void EXTI_Handler(void) {
    /* Clear the 'data ready' bit in the event group to reset state */
    /* Ensure my_event_group was initialized with ca_event_init() during startup
     */
    if (my_event_group != NULL) {
        ca_event_clear_isr(my_event_group, 0x01);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_event_clear_isr in an ISR context?
A: You should use ca_event_clear_isr inside an ISR when you need to clear specific bits from an event group without manually disabling interrupts. It is designed to be ISR-safe and avoids the overhead of disabling/enabling interrupts explicitly.

Code example conf:0.90 **Q:** Can you show a minimal example of using ca_event_clear_isr?
A: ca_event_group_t eg; ca_event_clear_isr(&eg, 0x02);

Parameters conf:0.90 **Q:** What are the valid values for the 'bits_to_clear' parameter in ca_event_clear_isr?
A: The 'bits_to_clear' parameter must be a valid u32 value. A value of 0 is invalid because it would not clear any bits. Any non-zero value is valid and will clear the corresponding bits in the event group.

Return conf:0.95 **Q:** What does ca_event_clear_isr return when called with invalid parameters?
A: If the 'eg' parameter is NULL or 'bits_to_clear' is 0, ca_event_clear_isr returns 0. This indicates that no operation was performed, but it does not cause a crash or undefined behavior.

ca_event_get

Header: caos/inc/event.h:170
Source: caos/src/event.c:246:254
GUARD: EN_EVENT

```
u32 ca_event_get(ca_event_group_t *eg)
```

Reads the current bitmask value of an event group without blocking.

Parametri:

Nome	Tipo	Descrizione
eg	ca_event_group_t *	Pointer to the initialized event group handle. Must not be NULL.

Ritorna:
Returns the current u32 bitmask value stored in the event group. Returns 0 if the provided pointer is NULL.

Note:

- This function is non-blocking and thread-safe, as it uses an internal spinlock to ensure atomic access to the bitmask. It is safe to call from an ISR because it uses a spinlock with interrupt disabling (lock_irq). The most common mistake is assuming this function clears the bits; it does not. It only performs a read operation. If you need to clear bits after reading, you must explicitly call ca_event_clear or ca_event_reset afterwards.

Uso:

- Use this function when you need to inspect the current state of multiple flags (e.g., sensor status, error conditions) without waiting for them to change. It is typically used in monitoring tasks, CLI commands, or diagnostic routines to report system status. It is often paired with ca_event_init for setup and ca_event_set or ca_event_clear to manage the state being read.

Esempio:

```c
/* Example: Checking sensor status bits via CLI */
#include "caos.h"

void check_sensors(ca_event_group_t *eg_sensors) {
    if (eg_sensors == NULL) return;

    /* Read current state of all sensor bits */
    u32 status = ca_event_get(eg_sensors);

    /* Check specific bits using bitwise AND */
    if (status & 0x01) {
        ca_printf("Temperature sensor active\n");
    }

    /* If we want to reset the status after reading, we would call: */
    /* ca_event_clear(eg_sensors, 0xFFFFFFFF); */
}
```

Q&A — fine-tuning review

4/4 pairs · qwen3-coder:latest

Usage conf:0.95 **Q:** When should I use ca_event_get() instead of waiting for events with ca_event_wait()?
A: You should use ca_event_get() when you need to check the current state of an event group without blocking or modifying it. It's useful for quick status checks, such as determining whether certain conditions are already met before proceeding with logic.

Unlike ca_event_wait(), which blocks until an event occurs, ca_event_get() returns immediately with the current event state.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_event_get() with caOS types?
A: ca_event_group_t eg; ca_event_init(&eg); ca_event_set(&eg, 0x01); u32 current_state = ca_event_get(&eg); // current_state will be 0x01 if the event is set.

Parameters conf:0.95 **Q:** What happens if I pass a NULL pointer to the 'eg' parameter of ca_event_get()?
A: If the 'eg' parameter is NULL, ca_event_get() returns 0. This is not an error condition that causes a crash, but rather a safe fallback behavior. The function checks for a valid pointer before accessing the event group, and returns 0 if the pointer is invalid.

Return conf:0.90 **Q:** What does ca_event_get() return when there are no active events in the group?
A: ca_event_get() returns 0 when there are no active events in the group or when the parameter 'eg' is NULL. This indicates that no bits are set in the event group, meaning no events are currently active.

`ca_event_init`

Header: `caos/inc/event.h:86`
Source: `caos/src/event.c:106:120`
GUARD: `EN_EVENT`

```
1 void ca_event_init(ca_event_group_t *eg)
```

Initializes an event group structure to manage task synchronization via bit flags.

Parametri:

Nome	Tipo	Descrizione
eg	`ca_event_group_t *`	Pointer to the event group structure to be initialized. Must point to valid memory; passing NULL will result in a silent return.

Ritorna:
void. The function does not return a status code, but it ensures the internal state of the event group is reset to a clean, predictable state.

Note:

- This function is non-blocking and performs the necessary setup of the internal spinlock, wait queue, and bit-tracking metadata. It must be called BEFORE any task attempts to wait on or set bits in the event group. While the function itself is safe to call, using an uninitialized event group is a common beginner mistake that leads to system crashes or deadlocks because the internal synchronization primitives (like the spinlock) will be in an invalid state. Ensure the event group object is allocated in persistent memory (e.g., global or static) rather than on a local stack, as the stack memory will be reclaimed when the initialization function returns, leaving the event group dangling.

Uso:

- This function is the mandatory first step in the lifecycle of an event group. It is typically called during the system startup phase, before any tasks are spawned using ca_os_job. Once initialized, the event group is used to coordinate multiple tasks, where one task might wait for specific bits to be set by other tasks or ISRs. It is usually paired with functions like ca_event_set or ca_event_set_isr to signal events, and ca_event_wait_any or ca_event_wait_all to block tasks until the required conditions are met.

Esempio:

```
static ca_event_group_t eg_sensors; // Declare the event group structure in
    static memory

void system_init(void) {
    // Initialize the event group before any task uses it
    ca_event_init(&eg_sensors);

    // Now it is safe to spawn tasks that will interact with this event group
    ca_os_job(task_sensor_func, 0, 0, 0, 0, 0, 0); // Simplified job creation
}

// Example of a task waiting for a specific event bit
static u32 task_sensor_func(u32 p1, u32 p2) {
    while (1) {
        // Wait for bit 0x1 to be set by an ISR or another task
        if (ca_event_wait_any(&eg_sensors, 0x1U, true, 1000U) != 0U) {
            // Event received, proceed with processing
        }
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_event_init and why is it important for event group synchronization?
A: You should call ca_event_init to initialize a ca_event_group_t structure before using it for task synchronization. This function prepares the internal structure, resets all event bits to 0, initializes the waiting queue, and sets up the spinlock for thread safety. It's important because it ensures the event group is in a valid state before any other event operations are performed.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_event_init with caOS types?
A: ca_event_group_t myEventGroup; ca_event_init(&myEventGroup);

Parameters conf:0.95 **Q:** What happens if I pass NULL to the ca_event_init function?
A: If you pass NULL as the parameter 'eg' to ca_event_init, the function will not do anything. It will not crash or cause undefined behavior, but it will also not initialize any event group structure. This is a safe check to prevent null pointer dereferences.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_event_init and what edge cases should I be aware of?
A: Common mistakes include passing a NULL pointer, which results in no initialization but no crash. Another edge case is calling ca_event_init on an uninitialized or improperly allocated ca_event_group_t structure, though the function itself does not require prior allocation. The function is thread-safe and can be called from ISRs, but improper use of the event group after initialization could lead to synchronization issues.

`ca_event_reset`

Header: `caos/inc/event.h:176`
Source: `caos/src/event.c:256:262`
GUARD: `EN_EVENT`

```
void ca_event_reset(ca_event_group_t *eg)
```

Clears all bits in an event group to zero, effectively resetting its state.

Parametri:

Nome	Tipo	Descrizione
eg	ca_event_group_t *	A pointer to the event group structure. Must be a valid, initialized pointer; passing NULL will result in a silent return.

Ritorna:
void

Note:

- This function is thread-safe because it uses a spinlock to protect the internal bitmask during the reset operation. It is safe to call from an Interrupt Service Routine (ISR) because it uses the 'irq' variant of the spinlock, which disables interrupts to prevent deadlocks. The most common beginner mistake is calling this function while other tasks are waiting for specific bits to be set; resetting the group will cause those tasks to remain blocked indefinitely or miss their expected events. Ensure the event group has been properly initialized before calling this, as operating on an uninitialized pointer will lead to a hard fault.

Uso:

- This function is typically used to perform a 'clean slate' operation on an event group, often during system state transitions or error recovery sequences. It is frequently paired with ca_event_set to signal new events after a reset, or used when a task needs to clear flags that were set by a previous operation cycle. You would use this when you want to ensure that no stale event flags from a previous process interfere with the current logic, effectively acting as a synchronization barrier for event-driven state machines.

Esempio:

```
/* Assuming tw_obj is a globally initialized ca_event_group_t */

void system_cleanup_task(void) {
    /* Reset the event group to clear any pending flags from previous cycles */
    ca_event_reset(&tw_obj);

    /* Signal that the system is ready for a new operation */
    ca_event_set(&tw_obj, 0x1U);

    /* Wait for the event to be set again by another task */
    if (ca_event_wait_any(&tw_obj, 0x1U, true, 100U)) {
        /* Event received successfully */
    } else {
        /* Handle timeout or error */
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_event_reset() and what is its purpose in event handling?
A: You should call ca_event_reset() to clear all bits in an event group after processing the events, effectively resetting the event state. This is useful when you want to reuse the same event group for multiple sequences of events without interference from previous states. It does not clear pending waits, only the bits themselves.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_event_reset() with caOS types?
A: ca_event_group_t eg; ca_event_init(&eg); ca_event_set(&eg, 0x01); // ... process event ca_event_reset(&eg); // Reset all bits to 0

Parameters conf:0.90 **Q:** What is the significance of the 'eg' parameter in ca_event_reset(), and what happens if it is invalid?

A: The 'eg' parameter is a pointer to the event group that will have its bits reset. If the pointer is NULL, the function does nothing. The parameter must be a valid and initialized event group; otherwise, behavior is undefined. It is not necessary for the event group to be in any specific state before calling this function.

Return conf:0.95 **Q:** Does ca_event_reset() return any value, and what could cause it to fail?
A: The function returns nothing (void). It cannot fail under normal conditions. If the parameter 'eg' is NULL, the function simply does nothing. There are no failure conditions as the function does not perform any operations that can fail.

`ca_event_set`

Header: `caos/inc/event.h:94`
Source: `caos/src/event.c:122:137`
GUARD: `EN_EVENT`

```
u32 ca_event_set(ca_event_group_t *eg, u32 bits_to_set)
```

Sets specific bits in an event group, triggering any tasks waiting for those bits to wake up.

Parametri:

Nome	Tipo	Descrizione
`eg`	`ca_event_group_t *`	Pointer to the event group handle. Must be initialized before use; passing NULL will result in a return value of 0.
`bits_to_set`	`u32`	A bitmask representing the bits to be set (e.g., 0x01, 0x02). Multiple bits can be set simultaneously using the OR operator.

Ritorna:
Returns the updated state of the event group bits after the operation. Returns 0 if the provided event group pointer is NULL.

Note:

- This function is thread-safe and uses a spinlock to protect the internal bitmask. While the implementation uses a spinlock with IRQ protection, if you are calling this from an ISR, you should prefer the dedicated ca_event_set_isr variant if available to ensure optimal performance and avoid potential deadlocks in specific RTOS configurations. It is a non-blocking operation, meaning it updates the bits and triggers the scheduler to check for waiting tasks immediately. A common beginner mistake is assuming that setting a bit will immediately pause the current task to let the waiting task run; in reality, the scheduler decides when the woken task actually starts executing. Always ensure the event group has been properly initialized via ca_event_init before calling this function, otherwise, the system may crash due to a null pointer dereference or invalid memory access.

Uso:

- This function is typically used to signal that a specific event or condition has occurred, such as data being ready from a sensor or a hardware interrupt finishing. It is usually paired with ca_event_wait, ca_event_wait_any, or ca_event_wait_all, where one task waits for a specific bit pattern to be set while another task calls ca_event_set to notify it. In a typical lifecycle, you initialize the event group, spawn tasks that call one of the wait functions, and then use ca_event_set in your producer tasks to synchronize the system flow. If you need to clear the bits after they are processed, you would typically use ca_event_clear in the consumer task.

Esempio:

```c
/* Define a bit for the sensor event */
#define BIT_TEMP (1 << 0)

/* Global event group handle */
caEventGroup eg_sensors;

/* Initialization in main or setup */
void system_init(void) {
    /* ca_event_init(&eg_sensors); // Initialize the event group before use */
}

/* Inside a sensor task, signal that temperature data is ready */
void sensor_task(void) {
    while(1) {
        /* Perform sensor reading logic */

        /* Signal the waiting task that temperature data is available */
        u32 current_bits = ca_event_set(&eg_sensors, BIT_TEMP);

        /* Optional: wait before next reading */
        ca_os_delay_ms(100);
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_event_set to signal events in my caOS application?
A: You should use ca_event_set when you need to signal one or more events within an event group, potentially waking up tasks that are waiting for specific event conditions. It is typically used to coordinate task synchronization where tasks are waiting for certain flags or combinations of flags to be set.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_event_set in a caOS application?
A: ca_event_group_t eg; ca_event_init(&eg); ca_event_set(&eg, 0x00000001); // Set bit 0 u32 current_bits = ca_event_set(&eg, 0x00000002); // Set bit 1 and get updated value

Parameters conf:0.95 **Q:** What does the 'bits_to_set' parameter in ca_event_set represent and what are valid values?
A: The 'bits_to_set' parameter is a bitmask where each bit set to 1 indicates a bit to be set in the event group. Valid values range from 0x00000000 to 0xFFFFFFFF, representing all possible combinations of 32 bits.

Return conf:0.90 **Q:** What does ca_event_set return and what does it mean if the return value is 0?
A: ca_event_set returns the updated value of all bits in the event group after setting the specified bits. If the return value is 0, it means either the event group pointer was NULL or no bits were set (the mask was 0).

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_event_set?
A: Common mistakes include passing a NULL pointer to the event group, which results in a return value of 0 without any effect. Another mistake is assuming that a non-zero return value indicates success, as it only reflects the updated bit state. Also, calling ca_event_set on an uninitialized event group may lead to undefined behavior.

Behavior conf:0.90 **Q:** Is ca_event_set blocking or non-blocking, and is it safe to call from an ISR?
A: ca_event_set is a non-blocking function that returns immediately after setting the bits and waking up waiting tasks. It is safe to call from an ISR, provided that the

underlying spinlock mechanism (ca_spinlock_lock_irq) is compatible with the ISR environment.

---

ca_event_set_isr

Header: caos/inc/event.h:110
Source: caos/src/event.c:153:156
GUARD: EN_EVENT

```
u32 ca_event_set_isr(ca_event_group_t *eg, u32 bits_to_set)
```

Sets specific bits in an event group from within an Interrupt Service Routine (ISR).

Parametri:

Nome	Tipo	Descrizione
eg	ca_event_group_t *	Pointer to the event group handle. Must be a valid, initialized pointer; passing NULL will cause a system crash.
bits_to_set	u32	A bitmask representing the bits to be set to 1. Bits not included in the mask remain unchanged.

Ritorna:
Returns the updated bitmask value of the event group after the operation. If the operation fails due to an invalid pointer, the behavior is undefined and typically results in a system fault.

Note:

- This function is specifically designed to be safe for use inside Interrupt Service Routines (ISRs). It is non-blocking and uses atomic operations or interrupt-disabling spinlocks to ensure that the bit manipulation is performed safely even when the CPU is currently handling an interrupt. It is thread-safe because it prevents race conditions between the ISR and the main task code. A common beginner mistake is to call a non-ISR version of an event function inside an interrupt, which can lead to deadlocks or system instability because standard functions might attempt to block the execution flow, which is strictly forbidden inside an ISR.

Uso:

- Use this function when a hardware event, such as a timer expiration, a GPIO edge detection, or a UART reception, needs to signal a waiting task that a specific condition has been met. It is typically paired with a task that is blocked on 'ca_event_wait' or 'ca_event_wait_any', waiting for these specific bits to be set. The typical lifecycle involves initializing the event group during system startup using 'ca_event_init', having a task wait for bits, and then triggering those bits from an ISR when the hardware condition occurs. Always ensure the event group is fully initialized before any ISR attempts to access it, otherwise, the system will likely trigger a hard fault.

Esempio:

```
/* Global event group handle */
caEventGroup *my_event_group;

/* ISR triggered by a hardware timer */
void Timer_IRQHandler(void) {
    u32 bits_to_signal = 0x01; // Signal that data is ready

    /* Safely set the bit from the ISR.
```

```
        This will wake up any task waiting on this event group. */
    u32 current_bits = ca_event_set_isr(my_event_group, bits_to_signal);

    /* Check if the bit was successfully set */
    if ((current_bits & bits_to_signal) == 0) {
        /* Handle error: log or trigger system recovery */
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_event_set_isr() instead of the regular ca_event_set() function?
A: You should use ca_event_set_isr() when setting event bits from within an ISR context. It is specifically designed for ISR usage and avoids re-enabling interrupts, which can be critical for maintaining interrupt latency and system stability. Regular ca_event_set() may not be suitable for ISR use due to potential interrupt handling issues.

Code example conf:0.90 **Q:** Can you provide a minimal example of how to use ca_event_set_isr() in an ISR?
A: ca_event_group_t eg; ca_event_group_create(&eg); void my_isr_handler(void) { u32 result = ca_event_set_isr(&eg, 0x01); } // Assume the ISR is properly configured in NVIC and connected to a peripheral like TIM2 or USART1.

Parameters conf:0.95 **Q:** What does the 'bits_to_set' parameter in ca_event_set_isr() control, and what are valid values?
A: The 'bits_to_set' parameter is a bitmask that specifies which bits in the event group should be set. Valid bits are from 0 to 31. Any u32 value is accepted, but only bits 0-31 will be considered. For example, setting bits 0 and 2 would be done with a value of 0x05 (binary 101).

Return conf:0.90 **Q:** What does a return value of CA_ERROR_INVALID_PARAM from ca_event_set_isr() indicate?
A: A return value of CA_ERROR_INVALID_PARAM indicates that the 'eg' parameter passed to ca_event_set_isr() was NULL. This is a failure condition caused by passing an invalid pointer, which would lead to a crash if not handled.

`ca_event_wait`

Header: `caos/inc/event.h:136`
Source: `caos/src/event.c:168:244`
GUARD: `EN_EVENT`

```
u32 ca_event_wait(ca_event_group_t *eg, u32 bits_to_wait, u8 wait_mode,
    bool auto_clear, u32 timeout_ms)
```

Suspends the current task until specific event bits are set in an event group, with optional timeout.

Parametri:

Nome	Tipo	Descrizione
`eg`	`ca_event_group_t *`	Pointer to the initialized event group object. Must not be NULL.
`bits_to_wait`	`u32`	Bitmask representing the events to wait for. Must be non-zero.
`wait_mode`	`u8`	Logic mode: 0 for OR (any bit), 1 for AND (all bits). Determines if the task wakes on any or all requested bits.
`auto_clear`	`bool`	If true, the function automatically clears the bits that triggered the wake-up after they are returned.
`timeout_ms`	`u32`	Maximum time to wait in milliseconds. Use a defined constant for infinite wait (e.g., CA_WAIT_FOREVER).

Ritorna:
Returns the bitmask of the events that triggered the wake-up (non-zero). Returns 0 if the timeout expires or if the input parameters are invalid.

Note:

- This function is blocking; it puts the calling task into a waiting state, allowing other tasks to run. It is NOT callable from an ISR because it performs task scheduling and blocking operations. You must ensure the event group is initialized via ca_event_init before calling this. A common beginner mistake is ignoring the return value; if the function returns 0, it means the event did not occur (timeout), and proceeding as if it did will lead to logic errors. Always check if the returned value is non-zero before acting on the event data.

Uso:

- This function is used for task synchronization, such as waiting for a hardware peripheral to finish a task or waiting for data to be processed by another thread. It is typically paired with ca_event_set or ca_event_set_isr, which are called by other tasks or ISRs to trigger the wake-up. The standard workflow is: initialize the event group with ca_event_init, start a task that calls ca_event_wait, and have another part of the system call ca_event_set to signal that the required condition is met. If auto_clear is false, you must manually call ca_event_clear to reset the bits for the next cycle.

Esempio:

```
/* Wait for either bit 0x01 or 0x02 to be set, with a 500ms timeout */
ca_event_group_t my_event;

void task_function(void *pv) {
    // Initialize the event group before use
    ca_event_init(&my_event);

    // Wait for bits 0x01 or 0x02 (OR mode), auto-clear them upon success
    // 0 is OR mode, 1 would be AND mode
    u32 result = ca_event_wait(&my_event, 0x03, 0, true, 500);

    if (result != 0) {
        // Event occurred, result contains the bits that were set
    } else {
        // Timeout reached or error occurred
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_event_wait in a caOS application?
A: You should use ca_event_wait when you need to synchronize tasks or wait for specific events to occur. It's useful in scenarios where one task must wait for another task to set certain flags in an event group before proceeding. For example, a task waiting for a hardware interrupt to complete its processing.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_event_wait?
A: ca_event_group_t eg; ca_event_group_init(&eg); ca_event_set(&eg, 0x01); // Set bit 0 u32 result = ca_event_wait(&eg, 0x01, 1, true, 1000); // Wait for bit 0, auto-clear, 1s timeout if (result == 0x01) { // Bit 0 was set }

Parameters conf:0.95 **Q:** What does the wait_mode parameter control in ca_event_wait?
A: The wait_mode parameter controls how the function interprets the bits_to_wait mask. If wait_mode is 0, all bits in the mask must be set in the event group. If wait_mode is 1, at least one bit in the mask must be set. This allows for flexible event waiting logic.

Return conf:0.90 **Q:** What does ca_event_wait return when it times out or encounters an error?
A: ca_event_wait returns 0 when either a timeout occurs or an error happens, such as invalid parameters (e.g., eg is NULL or bits_to_wait is 0). If successful, it returns the bitmask of bits that satisfied the wait condition.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_event_wait?
A: Common mistakes include passing a NULL pointer to the event group, passing 0 for bits_to_wait, not properly initializing the event group before use, and calling the function from an ISR, which is not safe. Also, incorrect handling of auto-clear behavior can lead to unexpected results.

Behavior conf:0.90 **Q:** Is ca_event_wait thread-safe and can it be called from an ISR?
A: Yes, ca_event_wait is thread-safe because it uses spinlock internally. However, it is not safe to call from an ISR because it is a blocking function and uses scheduling functions, which are not ISR-safe.

`ca_event_wait_all`

Header: `caos/inc/event.h:158`
GUARD: `EN_EVENT`

```
1 static inline u32 ca_event_wait_all(ca_event_group_t *eg, u32
    bits_to_wait, bool auto_clear, u32 timeout_ms)
```

Blocks the current task until all specified bits in an event group are set.

Parametri:

Nome	Tipo	Descrizione
`eg`	`ca_event_group_t *`	Pointer to the event group handle. Must be initialized via ca_event_init before use.
`bits_to_wait`	`u32`	A bitmask representing the specific bits to wait for. Only bits set in this mask are checked.
`auto_clear`	`bool`	If true, the bits specified in bits_to_wait are automatically cleared upon successful return.
`timeout_ms`	`u32`	Maximum time to wait in milliseconds. Use 0 for no wait (poll) or a specific value for a blocking timeout.

Ritorna:
Returns the current state of the event group bits if the condition is met, or 0 if the timeout expires before all bits are set.

Note:

- This function is blocking, meaning the calling task will be suspended by the scheduler until the bits are set or the timeout occurs. It is NOT thread-safe to call from an ISR; use ca_event_set_isr to trigger events from interrupts instead. A common mistake is forgetting to initialize the event group with ca_event_init, which will lead to a hard fault or undefined behavior. Ensure that the bits you are waiting for are actually being set by another task or ISR, otherwise the task will remain blocked indefinitely if the timeout is set to a non-zero value.

Uso:

- This function is typically used for task synchronization where a task must wait for multiple asynchronous conditions to be satisfied before proceeding. It is often paired with ca_event_init during system startup to prepare the event group. In a typical workflow, one or more producer tasks or ISRs call ca_event_set to signal completion of sub-tasks, while the consumer task uses ca_event_wait_all to aggregate these signals. If the logic requires waiting for any single bit instead of all, use ca_event_wait_any instead.

Esempio:

```
/* Initialize the event group before use */
caEventGroup eg_sensors;
ca_event_init(&eg_sensors);

/* Task waiting for multiple sensor flags */
void sensor_aggregator_task(void) {
    while (1) {
        /* Wait for bits 0x01 (Temp) and 0x02 (Pressure) to be set */
        u32 bits = ca_event_wait_all(&eg_sensors, 0x03, true, 1000);

        if (bits != 0) {
            /* Both events occurred within 1 second */
            ca_printf("Data ready, processing...\n");
        } else {
            /* Timeout occurred, handle error or retry */
            ca_printf("Timeout waiting for sensors!\n");
        }
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_event_wait_all instead of other synchronization primitives?
A: Use ca_event_wait_all when you need a logical AND condition for task execution, such as waiting for multiple asynchronous conditions (e.g., multiple sensors reporting data) to be satisfied before proceeding with a calculation.

Code example conf:0.90 **Q:** Provide a minimal example of how to wait for bits 0 and 1 using ca_event_wait_all.
A: ca_event_group_t my_eg; ca_event_init(&my_eg); u32 result = ca_event_wait_all(&my_eg, 0x03, true, 1000); if (result != 0) { /* Logic for when both bits are set */ }

Parameters conf:1.00 **Q:** What is the significance of the auto_clear parameter in ca_event_wait_all?
A: If auto_clear is set to true, the function automatically clears the bits that were waited for upon a successful return. If set to false, you must clear the bits manually, otherwise, the logic may re-trigger immediately.

Return conf:1.00 **Q:** What does a return value of 0 indicate when calling ca_event_wait_all?
A: A return value of 0 indicates that the timeout expired before all specified bits in the event group were set.

ca_event_wait_any

Header: caos/inc/event.h:145
GUARD: EN_EVENT

```
static inline u32 ca_event_wait_any(ca_event_group_t *eg, u32
    bits_to_wait, bool auto_clear, u32 timeout_ms)
```

Blocks the current task until one or more specified bits are set in an event group.

Parametri:

Nome	Tipo	Descrizione
eg	ca_event_group_t *	Pointer to the initialized event group handle.
bits_to_wait	u32	Bitmask representing the events to wait for (e.g., 0x01 | 0x04).
auto_clear	bool	If true, the bits specified in bits_to_wait are automatically cleared upon successful return.
timeout_ms	u32	Maximum time to wait in milliseconds. Use 0 for non-blocking check, or 0xFFFFFFFF for infinite wait.

Ritorna:
Returns the bitmask of the events that were set. Returns 0 if the timeout expired before any requested bits were set.

Note:

- This function is blocking, meaning the calling task will be suspended by the scheduler until the event occurs or the timeout expires. It is NOT callable from an ISR; use ISR-specific variants if available. Ensure the event group is initialized via ca_event_init before use. A common mistake is forgetting that 'auto_clear' only clears the bits you were waiting for, not all bits in the group, which can lead to logic errors if you expect the group to be completely reset.

Uso:

- Used for task synchronization where a task needs to wait for one or more asynchronous signals or status changes from other tasks or hardware interrupts. It is typically paired with ca_event_init during system setup and ca_event_set or ca_event_set_isr in the signaling task/interrupt. It is ideal for implementing state machines where a task waits for any of several possible triggers (e.g., error flags, data ready, or command received).

Esempio:

```
/* Initialize event group before task starts */
caEventGroup my_eg;
ca_event_init(&my_eg);

/* Inside a worker task */
void worker_task(void) {
    while (1) {
        /* Wait for either BIT_A or BIT_B, clear them automatically, 5s timeout
    */
        u32 received = ca_event_wait_any(&my_eg, 0x03, true, 5000);

        if (received & 0x01) {
            /* Handle event A */
        } else if (received & 0x02) {
            /* Handle event B */
```

```
        } else {
            /* Handle timeout: no events occurred */
        }
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_event_wait_any in my caOS task?
A: You should use this function in task loops to synchronize execution with external events or hardware signals. It is designed to block the calling task until at least one of the bits specified in the bitmask is set, making it ideal for waiting on multiple potential triggers.

Code example conf:0.90 **Q:** Provide a minimal example of how to wait for an event in caOS.
A: ca_event_group_t my_eg; ca_event_init(&my_eg); u32 bits = ca_event_wait_any(&my_eg, 0x03, true, 1000); if (bits != 0) { /* Process events */ }

Parameters conf:1.00 **Q:** What is the difference between setting auto_clear to true or false?
A: If auto_clear is true, the bits that triggered the wake-up are automatically cleared in the event group upon return. If false, the bits remain set, and you must manually clear them later using ca_event_clear.

Return conf:1.00 **Q:** How do I interpret the return value of ca_event_wait_any?
A: The function returns a bitmask of the events that were set. A return value of 0 indicates that the timeout expired without any of the requested bits being set. Always check the return value to distinguish between a successful event trigger and a timeout.

`ca_exit_critical`

Header: `caos/inc/sync.h:297`
Source: `caos/src/sync.c:730:742`
GUARD: `EN_SYNC`

```
void ca_exit_critical(void)
```

Exits a critical section by decrementing the nesting counter and potentially re-enabling interrupts.

Ritorna:
This function returns void and does not provide a status code.

Note:

- This function is non-blocking and executes in constant time. It is not thread-safe in the sense that it must be called from the same thread that called ca_enter_critical, and it is strictly forbidden to call this from an ISR (Interrupt Service Routine) because it manipulates global state intended for task-level synchronization. The function uses a nesting counter, meaning it only re-enables interrupts when the nesting level reaches zero; this allows nested function calls to safely protect shared resources without accidentally re-enabling interrupts too early. A common beginner mistake is to call ca_exit_critical more times than ca_enter_critical, which will trigger a CA_SAFEGUARD failure and likely crash the system or cause undefined behavior in the scheduler.

Uso:

- This function is used to conclude a protected block of code where shared resources, such as global variables or hardware peripherals, are accessed. It is always used in a strict pair with ca_enter_critical. The typical sequence is: call ca_enter_critical to lock the

system, perform the sensitive read/write operation on shared data, and then immediately call ca_exit_critical to restore the previous interrupt state. It is essential to keep the code between these two calls as short as possible to minimize interrupt latency, which ensures the RTOS remains responsive to real-time events.

Esempio:

```
/* Example of protecting a shared counter from concurrent access */
void update_shared_resource(void) {
    /* Enter critical section to prevent context switching or ISR interference
    */
    ca_enter_critical();

    /* Perform sensitive operation */
    shared_counter++;

    /* Exit critical section to restore previous interrupt state */
    ca_exit_critical();
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_exit_critical() in my embedded application?
A: You should call ca_exit_critical() to exit a critical section that was entered with ca_enter_critical(). It is typically used when protecting shared resources or sections of code that must not be interrupted. The function re-enables interrupts if it's the last nested exit from a critical section.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_exit_critical() with caOS types?
A: u32 critical_data = 0; void example_function(void) { ca_enter_critical(); critical_data += 1; ca_exit_critical(); }

Return conf:0.90 **Q:** What does ca_exit_critical() return and what causes it to fail?
A: ca_exit_critical() does not return any value. It is a void function and never fails. However, calling it without a corresponding ca_enter_critical() may result in undefined behavior due to incorrect nesting of critical sections.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_exit_critical()?
A: Common mistakes include calling ca_exit_critical() without a prior ca_enter_critical() call, which leads to undefined behavior. Additionally, calling it from an ISR without a preceding ca_enter_critical() is not allowed. The function is not thread-safe and modifies a global variable, so improper nesting can corrupt the critical section counter.

`ca_exti_disable`

Header: `caos/drivers/inc/exti.h:45`
Source: `caos/drivers/src/exti.c:61:67`
GUARD: `EN_EXTI`

```
void ca_exti_disable(u8 pin)
```

Disables the external interrupt (EXTI) line for a specific GPIO pin by masking the interrupt and disabling the NVIC controller.

Parametri:

Nome	Tipo	Descrizione
`pin`	`u8`	The GPIO pin number (0-15) corresponding to the EXTI line. Values >= 16 are ignored, resulting in no action.

Ritorna:
This function returns void and does not provide feedback on whether the operation was successful or if an invalid pin was provided.

Note:

- This function is non-blocking and executes immediately, making it safe to call from both main-line code and Interrupt Service Routines (ISRs). It is thread-safe in the sense that it performs atomic register writes, but you must ensure that no other task is simultaneously re-enabling the same pin. A common beginner mistake is assuming that disabling the EXTI line clears pending interrupts; if an interrupt was already triggered and waiting in the NVIC, it might still execute once before the disable takes full effect. Always ensure the hardware peripheral is initialized before attempting to disable it to avoid undefined behavior.

Uso:

- This function is typically used to temporarily silence hardware events, such as buttons or sensors, during critical sections of code where you do not want the CPU to be interrupted. It is often paired with a corresponding enable function to toggle interrupt sensitivity based on the application state, such as entering a low-power mode or a configuration menu. You would typically use this in a sequence: initialize the GPIO, configure the EXTI trigger, enable the interrupt, perform your task, and then call this function to disable the interrupt when it is no longer needed or to prevent 'bouncing' during signal processing.

Esempio:

```c
/* Example: Disabling a button interrupt to prevent processing during a menu
    update */
#define BTN_PIN 13

void update_system_menu(void) {
    /* Disable the button interrupt so user input doesn't trigger callbacks */
    ca_exti_disable(BTN_PIN);

    /* Perform sensitive operations */
    display_menu_data();

    /* Re-enable the interrupt once the menu is stable */
    /* ca_exti_enable(BTN_PIN); // Assuming an enable function exists */
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_exti_disable() and why would I want to disable an EXTI pin?
A: You should call ca_exti_disable() when you need to stop interrupt handling for a specific EXTI pin. This is useful when you want to temporarily disable interrupt processing, for example, during system shutdown, when reconfiguring interrupt settings, or when a pin is no longer needed for interrupt events. It ensures that the pin will not trigger interrupts anymore.

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_exti_disable() with caOS types?
A: u8 pin = 5; ca_exti_disable(pin);

Parameters conf:0.95 **Q:** What does the 'pin' parameter in ca_exti_disable() represent, and what are the valid values?
A: The 'pin' parameter represents the EXTI pin number to disable, ranging from 0 to 15. If a value greater than or equal to 16 is passed, the function will ignore the call and perform no action. Valid values are in the range [0, 15].

Return conf:0.90 **Q:** Does ca_exti_disable() return any value, and what does it mean if it does?
A: The function does not return any value. It is declared as void. There are no return values to indicate success or failure, as the function does not fail in a strict sense; it simply does nothing if the pin number is invalid (>=16).

`ca_exti_enable`

Header: `caos/drivers/inc/exti.h:44`
Source: `caos/drivers/src/exti.c:53:59`
GUARD: `EN_EXTI`

```
void ca_exti_enable(u8 pin)
```

Enables the external interrupt line for a specific GPIO pin by configuring the hardware mask and the NVIC controller.

Parametri:

Nome	Tipo	Descrizione
`pin`	`u8`	The GPIO pin number (0-15). Values 16 or higher are ignored by the function, resulting in no action.

Ritorna:
This function returns void and does not provide feedback on whether the hardware configuration was successful.

Note:

- This function is non-blocking and executes immediately. It is thread-safe in the sense that it performs atomic register writes, but it is not intended to be called from an ISR; it should be called from your main application tasks. Before calling this, you must ensure the GPIO pin has been configured as an input and that the EXTI trigger edge (rising/falling) has been set, otherwise, the interrupt may trigger immediately or never fire at all. The most common beginner mistake is calling this function before the underlying GPIO hardware is initialized, which leads to silent failures where the interrupt never triggers.

Uso:

- This function is typically used during the system initialization phase or when a specific feature (like a button or sensor) needs to be activated dynamically. It is usually paired with 'ca_exti_disable' to toggle interrupt-driven features on and off to save power or prevent unwanted triggers during specific software states. In a typical workflow, you would initialize the GPIO, set the trigger edge, call 'ca_exti_enable', and then define the corresponding interrupt service routine to handle the event.

Esempio:

```
/* Example: Enabling a button interrupt after system setup */
#define BTN_PIN 13

void setup_button_interrupt(void) {
    /* 1. Ensure GPIO is configured as input (assumed initialized elsewhere) */
    /* 2. Enable the interrupt line in the hardware */
    ca_exti_enable(BTN_PIN);

    /* 3. Now the CPU will jump to the EXTI handler when the pin state changes
    */
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_exti_enable() and why is it important to configure the pin before calling it?
A: You should call ca_exti_enable() after configuring a pin as an input with appropriate pull-up or pull-down resistors. This function enables the external interrupt line for that specific pin to generate interrupts. It's important to configure the pin first because the function does not handle pin configuration itself and will not work correctly if the pin is not set up as an input.

Code example conf:0.85 **Q:** Can you provide a minimal C example using ca_exti_enable() with caOS types?
A: u8 pin = 5; ca_exti_enable(pin);

Parameters conf:0.90 **Q:** What is the valid range for the pin parameter in ca_exti_enable() and what happens if I pass an invalid value?
A: The valid range for the pin parameter is from 0 to 15. Values outside this range are ignored by the function. For example, passing pin = 20 or pin = -1 will result in no action being taken by ca_exti_enable().

`ca_exti_hw_clock_init`

Header: `caos/drivers/inc/ca_glue_hw.h:129`
GUARD: `EN_CA_GLUE_HW`

```
1 extern void ca_exti_hw_clock_init(void); /* abilita il clock del
     multiplexer SYSCFG/EXTI */
```

Initializes the hardware clock required for the EXTI (External Interrupt) peripheral and its associated multiplexer.

Ritorna:
void

Note:

- This function is non-blocking and executes extremely quickly, as it only performs a register write to the RCC (Reset and Clock Control) peripheral. It is thread-safe in the sense that it performs an atomic bitwise OR operation on the clock enable register, but it is not intended to be called repeatedly; it should be called once during the initialization phase of an external interrupt. On some architectures (like H5xx/U5xx), this function does nothing because the clock is always enabled by default, while on others (like F1xx/F4xx), it is mandatory to enable the SYSCFG or AFIO clock before configuring interrupt routing. A common beginner mistake is to assume this function is universal; always remember that the underlying hardware implementation varies significantly between STM32 families, and failing to call this on families that require it will result in the EXTI configuration registers being ignored by the hardware.

Uso:

- This function is used exclusively during the setup phase of an external interrupt pin. It is typically the first step in a sequence that includes selecting the GPIO port source, setting the trigger edge (rising/falling), enabling the interrupt mask register (IMR), and finally enabling the interrupt in the NVIC. You should call this function before any other hardware-specific EXTI configuration functions to ensure the peripheral's control registers are accessible and clocked. It is usually paired with functions like ca_exti_hw_select_source and ca_exti_hw_nvic_enable to fully prepare a pin for interrupt handling.

Esempio:

```
/* Example of initializing an external interrupt on a pin */
void setup_button_interrupt(void) {
    /* 1. Enable the clock for the EXTI/SYSCFG hardware block */
    ca_exti_hw_clock_init();

    /* 2. Configure the specific pin routing and edge detection */
    ca_exti_hw_select_source(PORT_A, 0);
    ca_exti_hw_set_edge(0, RISING_EDGE);

    /* 3. Enable the interrupt in the hardware and the NVIC controller */
    ca_exti_hw_imr_enable(0);
    ca_exti_hw_nvic_enable(0);
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_exti_hw_clock_init() and why is it necessary?
A: You should call ca_exti_hw_clock_init() before configuring any external interrupt lines. It enables the clock for the SYSCFG/EXTI module, which is required for handling external interrupts on STM32 microcontrollers. The function is necessary because without enabling the clock, the EXTI registers won't be accessible, and external interrupts cannot be configured properly. It's a one-time initialization function that must be called before any EXTI configuration.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_exti_hw_clock_init() with caOS types?
A:

```
#include "caos.h"

void setup_exti(void)
{
    ca_exti_hw_clock_init();
    // Now configure EXTI lines using caos/exti.h functions
}
```

HW differences conf:0.90 **Q:** What are the key hardware differences in ca_exti_hw_clock_init() across different STM32 families?
A: The function behaves differently across families: F1xx uses RCC_APB2ENR_AFIOEN to enable AFIO clock, F4xx and F7xx use RCC_APB2ENR_SYSCFGEN to enable SYSCFG clock, H5xx does nothing as EXTI is on AHB2 bus and always enabled, and H7xx uses RCC_APB4ENR_SYSCFGEN to enable SYSCFG clock on APB4 bus. These differences reflect the varying clocking architectures and register layouts across STM32 families.

`ca_exti_hw_imr_disable`

Header: `caos/drivers/inc/ca_glue_hw.h:135`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_exti_hw_imr_disable(u8 pin); /* disabilita la maschera
   interrupt (IMR) per la linea EXTI */
```

Disables the interrupt mask for a specific EXTI line, preventing the hardware from triggering an interrupt request to the CPU.

Parametri:

Nome	Tipo	Descrizione
pin	u8	The EXTI line number to disable. Valid values are 0-15, corresponding to the GPIO pin index. Providing a value outside this range may result in undefined behavior or register corruption.

Ritorna:
void

Note:

- This function is non-blocking and executes immediately by modifying the hardware register directly. It is thread-safe in the sense that it performs a direct bitwise operation, but it does not provide synchronization against other tasks modifying the same register. It is safe to call from an ISR, as it does not involve OS primitives or blocking calls. The most common beginner mistake is assuming this function disables the physical signal on the pin; it only stops the EXTI peripheral from forwarding that signal to the NVIC. If you disable the mask but do not also disable the NVIC interrupt line, the hardware might still flag a pending interrupt if the mask is re-enabled later.

Uso:

- This function is used when you need to temporarily silence an external interrupt source, such as during a critical section of code where you do not want to be interrupted by a specific sensor or button press. It is typically paired with ca_exti_hw_imr_enable to re-enable the interrupt once the critical task is complete. In the caOS driver architecture, it is usually called as part of a higher-level wrapper like ca_exti_disable, which ensures both the EXTI mask and the NVIC interrupt controller are disabled in the correct sequence to prevent spurious interrupts.

Esempio:

```c
/* Example: Temporarily disabling a button interrupt during a sensitive
   operation */
#define BUTTON_PIN 5

void process_sensitive_data(void) {
    /* Disable the EXTI mask to prevent the button from triggering an ISR */
    ca_exti_hw_imr_disable(BUTTON_PIN);

    /* Perform operations that must not be interrupted */
    /* Call a function that performs the critical work */

    /* Re-enable the interrupt mask to resume normal operation */
    ca_exti_hw_imr_enable(BUTTON_PIN);
}
```

Q&A — fine-tuning review

5/5 pairs · qwen3-coder:latest

Usage conf:0.95 **Q:** When should I call ca_exti_hw_imr_disable() in my embedded application?
A: You should call ca_exti_hw_imr_disable() when you want to disable hardware interrupts for a specific EXTI line, preventing the microcontroller from generating an interrupt when an event occurs on that line. This is typically used when you want to temporarily disable interrupt handling for a pin or when configuring EXTI lines before enabling them.

Code example conf:0.95 **Q:** Can you show a minimal example of how to use ca_exti_hw_imr_disable() with caOS types?
A:

```c
u8 pin = 5;
ca_exti_hw_imr_disable(pin);
```

Parameters conf:0.95 **Q:** What does the 'pin' parameter in ca_exti_hw_imr_disable() represent, and what are valid values?
A: The 'pin' parameter represents the EXTI line number to disable interrupt masking for. Valid values are integers from 0 to 15 inclusive. Values less than 0 or greater than 15 are invalid and may cause undefined behavior.

Return conf:0.95 **Q:** Does ca_exti_hw_imr_disable() return any value, and what happens if it fails?
A: The function does not return any value. It directly modifies the hardware register EXTI->IMR (or EXTI->IMR1 for H5xx/H7xx) to disable the interrupt mask for the specified pin. Since it's a void function, there are no return values to indicate success or failure.

HW differences conf:0.90 **Q:** How does ca_exti_hw_imr_disable() differ between H5xx/H7xx and other STM32 families like F1xx/F4xx/F7xx?
A: For F1xx, F4xx, and F7xx families, the function modifies the EXTI->IMR register. However, for H5xx and H7xx families, it modifies the EXTI->IMR1 register (or EXTI_D1->IMR1 for H7xx). This reflects differences in the hardware register layout between these STM32 families.

`ca_exti_hw_imr_enable`

Header: `caos/drivers/inc/ca_glue_hw.h:134`
GUARD: `EN_CA_GLUE_HW`

```
1 extern void ca_exti_hw_imr_enable(u8 pin); /* abilita la maschera
     interrupt (IMR) per la linea EXTI */
```

Enables the interrupt mask for a specific EXTI line, allowing the hardware to trigger an interrupt request to the CPU.

Parametri:

Nome	Tipo	Descrizione
`pin`	`u8`	The EXTI line number (typically 0-15). Passing a value outside the hardware-supported range may result in undefined behavior or writing to reserved memory.

Ritorna:
void

Note:

- This function is non-blocking and executes immediately by performing a direct register write. It is thread-safe in the sense that it performs a bitwise OR operation, but it does not protect against concurrent access to the same register from different tasks. It is safe to call from an ISR, as it does not involve OS primitives or blocking calls. The most common mistake is calling this before configuring the EXTI source and edge trigger; if the mask is enabled before the hardware is fully configured, the CPU might trigger a spurious interrupt immediately upon enabling the line.

Uso:

- This function is the final step in the EXTI initialization sequence before enabling the interrupt in the NVIC. It is typically used in a driver setup function that follows a strict order: first, initialize the peripheral clock, then map the GPIO pin to the EXTI line, configure the trigger edge, enable the IMR mask, and finally enable the interrupt in the NVIC. If you skip this function, the EXTI peripheral will detect the signal change but will

never forward the request to the CPU, effectively leaving the interrupt 'dead'. It is paired with ca_exti_init to prepare the hardware and ca_exti_enable to manage the lifecycle of the interrupt.

Esempio:

```
/* Example: Configuring an external interrupt for a button on pin 5 */
u8 pin = 5;

/* 1. Initialize the EXTI peripheral hardware */
ca_exti_init();

/* 2. Configure GPIO and EXTI source (internal driver logic) */
/* ca_exti_hw_select_source(PORTA, pin); */

/* 3. Set the trigger to falling edge (button press) */
/* ca_exti_hw_set_edge(pin, FALLING_EDGE); */

/* 4. Enable the interrupt mask so the EXTI controller can signal the CPU */
ca_exti_hw_imr_enable(pin);

/* 5. Finally, enable the interrupt in the NVIC to allow the CPU to handle it */
ca_exti_enable(pin);
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_exti_hw_imr_enable() in my embedded application?
A: You should call ca_exti_hw_imr_enable() after initializing the EXTI peripheral and configuring the GPIO pin as an external interrupt source. It enables the interrupt mask for a specific EXTI line, allowing the system to generate interrupts when an event occurs on that line. This function must be called from a non-interrupt context and is typically used during system initialization or before enabling the interrupt handler.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_exti_hw_imr_enable() with caOS types?
A:

```
#include "caos.h"

void setup_exti_interrupt(void) {
    u8 pin = 5;
    ca_exti_hw_imr_enable(pin);
}
```

Parameters conf:0.90 **Q:** What does the 'pin' parameter in ca_exti_hw_imr_enable() represent, and what are valid values?
A: The 'pin' parameter represents the EXTI line number to enable interrupt masking for. Valid values are integers from 0 to 23 inclusive. Values less than 0 or greater than 23 are considered invalid and may cause undefined behavior or hardware access violations.

Return conf:0.85 **Q:** Does ca_exti_hw_imr_enable() return any value, and what happens if it fails?
A: The function does not return any value. It directly modifies the EXTI->IMR register hardware register. If called with an invalid pin number (less than 0 or greater than 23), it may result in invalid memory access or undefined behavior because the bit shift operation could access invalid register locations.

HW differences conf:0.80 **Q:** How does ca_exti_hw_imr_enable() differ between STM32 families like H5xx and H7xx?
A: In STM32 H5xx series, the function writes to EXTI->IMR1 register, whereas in STM32 H7xx series, it writes to EXTI_D1->IMR1 register. This reflects differences in the memory mapping and interrupt controller architecture between these families, where

H7xx uses a more complex dual-core setup with separate interrupt domains for different CPU cores.

---

ca_exti_hw_nvic_disable

Header: caos/drivers/inc/ca_glue_hw.h:133
GUARD: EN_CA_GLUE_HW

```
extern void ca_exti_hw_nvic_disable(u8 pin); /* disabilita l'interrupt
    EXTI nel NVIC */
```

Disables the NVIC interrupt line for a specific GPIO pin, but only if no other pins sharing the same interrupt vector are still active.

Parametri:

Nome	Tipo	Descrizione
pin	u8	The GPIO pin number (0-15). Values outside this range may cause undefined behavior or memory access errors depending on the implementation.

Ritorna:
void

Note:

- This function is non-blocking and executes very quickly, making it safe to call from an ISR if necessary. It is thread-safe in the sense that it performs atomic register checks, but it relies on the EXTI interrupt mask register (IMR) state to decide whether to disable the NVIC line. A common beginner mistake is assuming this function always disables the NVIC; in many STM32 families (like F4 or H7), it intelligently checks if other pins sharing the same EXTI handler (e.g., pins 5-9) are still enabled before turning off the NVIC line. If you call this while another pin on the same vector is active, the NVIC will remain enabled, which is the correct behavior to prevent breaking other active interrupts.

Uso:

- This function is typically used as part of a cleanup or shutdown sequence for a specific GPIO interrupt. It is almost always paired with 'ca_exti_hw_imr_disable' to ensure the hardware interrupt mask is cleared before the NVIC line is potentially disabled. Developers use this when they want to stop receiving interrupts from a specific button or sensor without accidentally disabling the entire interrupt group that other peripherals might still be relying on.

Esempio:

```
/* Example: Disabling an interrupt for a sensor connected to pin 3 */
#define SENSOR_PIN 3

void stop_sensor_interrupt(void) {
    /* First, mask the specific pin in the EXTI hardware register */
    ca_exti_hw_imr_disable(SENSOR_PIN);

    /* Then, check if we can safely disable the NVIC line for this group */
    /* This will only disable the NVIC if no other pins in the 0-4 group are
    active */
    ca_exti_hw_nvic_disable(SENSOR_PIN);
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_exti_hw_nvic_disable() in my embedded application?
A: You should call ca_exti_hw_nvic_disable() when you want to disable a specific EXTI interrupt in the NVIC, typically to prevent unwanted interrupts during critical sections of code or when disabling external interrupts. It's useful for managing interrupt priorities and ensuring that only necessary interrupts are active.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_exti_hw_nvic_disabl
A:

```
u8 pin = 5;
ca_exti_hw_nvic_disable(pin);
```

Parameters conf:0.95 **Q:** What is the valid range for the pin parameter in ca_exti_hw_nvic_disable()?
A: The valid range for the pin parameter is from 0 to 15. Values outside this range (e.g., >15) are not handled and may cause unpredictable behavior.

Return conf:0.90 **Q:** Does ca_exti_hw_nvic_disable() return any value, and what does it signify?
A: The function does not return any value. It is void. It cannot fail in a strict sense, but it may not perform any action if the pin is invalid or if the interface is already disabled.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_exti_hw_nvic_disable()?
A: Common mistakes include calling the function with invalid pin numbers (>15), calling it from an ISR, or calling it without previously enabling the interrupt. Additionally, calling it from multiple threads simultaneously can lead to race conditions since the function is not thread-safe.

HW differences conf:0.90 **Q:** How does ca_exti_hw_nvic_disable() differ between STM32 H5xx and H7xx MCUs?
A: On H5xx MCUs, the function directly disables the NVIC interrupt without checking other active pins, whereas on H7xx MCUs, it checks if other pins in the same group are still active using EXTI_D1->IMR1 before disabling the interrupt, to avoid disabling an interrupt that might be used by another pin.

`ca_exti_hw_nvic_enable`

Header: `caos/drivers/inc/ca_glue_hw.h:132`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_exti_hw_nvic_enable(u8 pin); /* abilita l'interrupt EXTI
   nel NVIC con priorit  predefinita */
```

Enables the EXTI interrupt for a specific pin in the NVIC and sets its hardware priority.

Parametri:

Nome	Tipo	Descrizione
`pin`	`u8`	The GPIO pin number (0-15) to be enabled. Passing a value outside this range may lead to undefined behavior or incorrect IRQ mapping.

Ritorna:
void

Note:

- This function is non-blocking and executes immediately. It is thread-safe in the sense that it modifies NVIC registers, which are atomic operations, but it should not be called from an ISR as it is intended for initialization. The function sets a fixed hardware priority (typically 5 or 6), which is crucial for system stability; if you set this incorrectly, you

might cause interrupt latency issues for critical peripherals like USB. A common beginner mistake is calling this before the peripheral clock or the EXTI source selection has been configured, which results in the interrupt never firing because the hardware path is not yet active.

Uso:

- This function is used during the initialization phase of an external interrupt driver. It is typically the final step in a sequence: first, you initialize the clock, then select the GPIO port as the source, configure the trigger edge (rising/falling), enable the interrupt mask register (IMR), and finally call this function to unmask the interrupt in the NVIC. It is paired with functions like ca_exti_hw_select_source and ca_exti_hw_set_edge to complete the setup of an external event handler.

Esempio:

```
/* Example of initializing an external interrupt on pin 5 */
u8 my_pin = 5U;

/* 1. Prepare the hardware path */
ca_exti_hw_clock_init();
ca_exti_hw_select_source(PORTA, my_pin);

/* 2. Configure the trigger condition */
ca_exti_hw_set_edge(my_pin, RISING_EDGE);

/* 3. Unmask the interrupt in the EXTI controller */
ca_exti_hw_imr_enable(my_pin);

/* 4. Finally, enable the interrupt in the NVIC so the CPU can receive it */
ca_exti_hw_nvic_enable(my_pin);

/* Now the system is ready to handle the interrupt */
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_exti_hw_nvic_enable() and what is its purpose in caOS?
A: You should call ca_exti_hw_nvic_enable() to enable hardware EXTI interrupts for a specific pin in the NVIC with a default priority of 5. It is used to handle hardware interrupts from external pins, typically for GPIO input events or other peripheral triggers. The function is intended to be called from a non-interrupt context and is not thread-safe.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_exti_hw_nvic_enable() with caOS types?
A:

```
u8 pin = 3;
ca_exti_hw_nvic_enable(pin);
```

Parameters conf:0.90 **Q:** What does the 'pin' parameter in ca_exti_hw_nvic_enable() represent, and what are valid values?
A: The 'pin' parameter represents the EXTI pin number to enable, ranging from 0 to 15. Valid values are 0 through 15 inclusive. Values outside this range (e.g., above 15) may cause undefined behavior as they could access invalid IRQn_Type values.

Return conf:0.90 **Q:** Does ca_exti_hw_nvic_enable() return any value, and can it fail?
A: The function does not return any value. It is designed to be a void function and cannot fail in terms of return codes. However, incorrect usage such as passing an invalid pin number (outside 0-15) may lead to undefined behavior or system instability.

HW differences conf:0.85 **Q:** Are there any differences in how ca_exti_hw_nvic_enable() is implemented across different STM32 MCU families?

A: No, the implementation of ca_exti_hw_nvic_enable() is consistent across all supported STM32 families (f1xx, f4xx, f7xx, h5xx, h7xx). Each implementation uses the same code pattern: enabling the IRQ via NVIC_EnableIRQ and setting the priority to 5 using NVIC_SetPriority, regardless of the specific peripheral or register layout.

`ca_exti_hw_select_source`

Header: `caos/drivers/inc/ca_glue_hw.h:130`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_exti_hw_select_source(u8 port, u8 pin); /* collega il pin GPIO alla linea EXTI corrispondente */
```

Configures the hardware multiplexer to route a specific GPIO pin to an EXTI interrupt line.

Parametri:

Nome	Tipo	Descrizione
`port`	`u8`	The GPIO port identifier (e.g., 0 for Port A, 1 for Port B). Must correspond to the hardware-defined mapping.
`pin`	`u8`	The specific pin number (0-15) to be mapped to the EXTI line. Values outside 0-15 may cause undefined behavior or register corruption.

Ritorna:
void

Note:

- This function is non-blocking and executes in constant time, making it safe to call from almost any context, including ISRs, provided the hardware clock for the configuration register (SYSCFG, AFIO, or EXTI) is already enabled. It is not thread-safe by itself, but since it performs a simple atomic-like register write, it is generally safe if only one task manages EXTI configuration. The most common beginner mistake is calling this before enabling the peripheral clock for the configuration register (e.g., SYSCFG or AFIO), which results in the write being ignored by the hardware, leaving the interrupt unmapped.

Uso:

- This function is a critical step in the EXTI initialization sequence. It is typically called after the GPIO clock is enabled and before setting the trigger edge (rising/falling) and enabling the interrupt in the NVIC. You use it to 'connect' a physical pin to the EXTI logic so that the hardware can detect transitions on that specific pin. It is usually paired with ca_exti_hw_set_edge and ca_exti_hw_imr_enable to complete the setup of an external interrupt source.

Esempio:

```
/* Example: Mapping PA5 to EXTI line 5 */
#define CA_GPIO_PORT_A 0
#define PIN_5 5

/* Ensure the configuration peripheral clock is active first */
ca_exti_hw_clock_init();

/* Route PA5 to the EXTI controller */
ca_exti_hw_select_source(CA_GPIO_PORT_A, PIN_5);

/* Configure the trigger and enable the interrupt */
```

```
ca_exti_hw_set_edge(PIN_5, EDGE_RISING);
ca_exti_hw_imr_enable(PIN_5);
ca_exti_hw_nvic_enable(PIN_5);
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_exti_hw_select_source() in my caOS application?
A: You should call ca_exti_hw_select_source() during the system initialization phase, before enabling any EXTI lines. It configures the hardware mapping between GPIO pins and EXTI lines, which is a one-time setup required for external interrupt functionality. It must be called before enabling the EXTI interrupt, and not during normal program execution or from an ISR.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_exti_hw_select_source() with caOS types?
A:

```
u8 port = 0; /* GPIO Port A */
u8 pin = 5;  /* Pin 5 */
ca_exti_hw_select_source(port, pin);
```

Parameters conf:0.95 **Q:** What do the port and pin parameters represent in ca_exti_hw_select_source() and what are the valid ranges?
A: The 'port' parameter specifies the GPIO port number (0=A, 1=B, 2=C, etc.), with valid values from 0 to 7. The 'pin' parameter specifies the pin number within the port (0-15). Invalid values for port are <0 or >7, and for pin are <0 or >15. Passing invalid parameters may cause undefined behavior or register corruption.

Return conf:0.90 **Q:** Does ca_exti_hw_select_source() return any value, and can it fail?
A: The function does not return any value. It is non-blocking and has no error handling mechanism. It cannot fail in the traditional sense, but incorrect parameters may lead to unpredictable behavior due to invalid register accesses or incorrect hardware configuration.

HW differences conf:0.85 **Q:** How does ca_exti_hw_select_source() differ between STM32 F1xx and F4xx families in terms of hardware implementation?
A: For F1xx family, the function writes to AFIO->EXTICR registers, whereas for F4xx family, it writes to SYSCFG->EXTICR registers. Both use the same logic for calculating register index and bit shift, but access different peripheral base addresses for the EXTI configuration registers.

`ca_exti_hw_set_edge`

Header: `caos/drivers/inc/ca_glue_hw.h:131`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_exti_hw_set_edge(u8 pin, ca_exti_edge_t edge); /*
    configura il fronte di trigger (rising, falling, both) */
```

Configures the hardware trigger edge (rising, falling, or both) for a specific external interrupt pin.

Parametri:

Nome	Tipo	Descrizione
`pin`	`u8`	The EXTI line number (0-15). Passing a value outside this range may cause undefined behavior or access invalid memory.
`edge`	`ca_exti_edge_t`	The trigger condition: CA_EXTI_EDGE_RISING, CA_EXTI_EDGE_FALLING, or CA_EXTI_EDGE_BOTH.

Ritorna:
void

Note:

- This function is non-blocking and performs direct register manipulation, making it extremely fast. It is safe to call from an ISR because it does not use OS primitives or blocking mechanisms. However, it is not inherently thread-safe if multiple tasks attempt to modify the same EXTI registers simultaneously; in such cases, you should wrap the call in a critical section using ca_enter_critical and ca_exit_critical. The most common beginner mistake is calling this function before the peripheral clock is initialized, which causes the hardware to ignore the write operation, leaving the interrupt trigger unconfigured.

Uso:

- This function is a mandatory step in the EXTI setup sequence. It is typically used after initializing the EXTI peripheral and selecting the GPIO source, but before enabling the interrupt mask and the NVIC. You should use it during the initial configuration phase or whenever you need to dynamically change the trigger behavior of an input, such as switching from detecting a button press (falling edge) to a button release (rising edge) during runtime.

Esempio:

```
/* Configure EXTI line 5 to trigger on both edges for a sensor input */
u8 pin = 5;
ca_exti_edge_t trigger = CA_EXTI_EDGE_BOTH;

/* Initialize the EXTI peripheral first */
ca_exti_init();

/* Set the trigger condition for the specific pin */
ca_exti_hw_set_edge(pin, trigger);

/* Proceed to enable the interrupt in the hardware mask and NVIC */
/* Note: ca_exti_hw_imr_enable and ca_exti_hw_nvic_enable are hardware-specific helpers */
ca_exti_enable(pin);
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_exti_hw_set_edge() in my embedded application?
A: You should call ca_exti_hw_set_edge() after selecting the EXTI source pin using ca_exti_hw_select_source(). This function configures the trigger edge (rising, falling, or both) for the specified EXTI line. It's typically used when setting up external interrupt handlers for GPIO pins.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_exti_hw_set_edge() with caOS types?
A:

```
u8 pin = 3;
ca_exti_edge_t edge = CA_EXTI_EDGE_RISING;
ca_exti_hw_set_edge(pin, edge);
```

Parameters conf:0.95 **Q:** What are the valid values for the 'pin' parameter in ca_exti_hw_set_edge()?
A: The 'pin' parameter must be an integer between 0 and 15 inclusive. Values less than 0 or greater than 15 are invalid and may cause undefined behavior.

Return conf:0.90 **Q:** Does ca_exti_hw_set_edge() return any value? What does it mean if it fails?
A: The function does not return any value. It is non-blocking and cannot fail in a strict sense, but it may not function correctly if the pin has not been properly initialized or selected via ca_exti_hw_select_source().

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_exti_hw_set_edge()?
A: A common mistake is calling ca_exti_hw_set_edge() without first selecting the source pin using ca_exti_hw_select_source(). This can lead to unpredictable interrupt behavior. Additionally, using invalid pin numbers (outside 0-15) or calling the function from an ISR may cause issues.

Behavior conf:0.95 **Q:** Is ca_exti_hw_set_edge() thread-safe and ISR-safe?
A: No, ca_exti_hw_set_edge() is neither thread-safe nor ISR-safe. It is a non-blocking function that must not be called from within an ISR. It should be used only in contexts where concurrency is managed externally.

HW differences conf:0.85 **Q:** How does ca_exti_hw_set_edge() differ between STM32 H5xx and H7xx microcontrollers?
A: The main difference lies in the register names used. For H5xx, the function uses EXTI->RTSR1 and EXTI->FTSR1 registers, whereas for H7xx it uses EXTI->RTSR1 and EXTI->FTSR1. Both families use similar logic for setting the trigger edges, but the register base addresses differ.

`ca_exti_hw_tamper_disable`

Header: `caos/drivers/inc/ca_glue_hw.h:149`
GUARD: `EN_CA_GLUE_HW`

```
1 extern void ca_exti_hw_tamper_disable(void); /* tamper 1 off, IRQ off */
```

Disables the hardware tamper detection mechanism and its associated interrupt.

Ritorna:
void

Note:

- This function is a low-level hardware abstraction that directly modifies peripheral registers (TAMP/RTC) and the NVIC interrupt controller. It is not thread-safe and should not be called from an ISR unless you are absolutely certain that no other task is concurrently configuring the tamper peripheral. The function is non-blocking and executes immediately. A common mistake is calling this function while the tamper detection logic is still required for security, which will leave the system vulnerable to physical tampering without triggering the expected callback.

Uso:

- This function is typically used during system shutdown, power-down sequences, or when reconfiguring security parameters where tamper detection is no longer desired. It is usually paired with ca_exti_tamper_enable to manage the lifecycle of the security feature. In the caOS driver layer, it is called by ca_exti_tamper_disable to ensure that both the hardware registers are cleared and the internal callback pointer is reset to NULL, preventing dangling references.

Esempio:

```c
/* Example: Safely disabling tamper detection before entering a low-power state */

void secure_shutdown_sequence(void) {
    /* Disable the tamper detection hardware and clear the callback */
    ca_exti_tamper_disable();

    /* Perform other system cleanup tasks */
    /* ca_os_sleep(100); // Example of system sleep */

    /* Log the event using standard system output */
    ca_stdout_printf("Tamper detection disabled. System safe to power down.\n");
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_exti_hw_tamper_disable in my caOS application?
A: You should call this function during system shutdown, power-down sequences, or when reconfiguring security parameters where tamper detection is no longer required. It is typically paired with ca_exti_tamper_enable to manage the security feature's lifecycle. Note that it is not thread-safe and should not be called from an ISR if other tasks are concurrently configuring the tamper peripheral.

Code example conf:0.95 **Q:** Provide a minimal example of how to safely disable the tamper mechanism using caOS.
A: To disable the tamper mechanism, simply invoke the function directly. Ensure no other tasks are modifying tamper registers simultaneously:
void shutdown_security(void) { ca_exti_hw_tamper_disable(); }

HW differences conf:1.00 **Q:** How does the implementation of ca_exti_hw_tamper_disable differ across STM32 families?
A: Implementations vary based on the hardware architecture: F1xx does not support this function. F4xx, F7xx, and H7xx target the RTC peripheral (e.g., RTC->TAFCR or RTC->TAMPCR) and EXTI registers. Conversely, H5xx and U5xx target the TAMP peripheral directly (TAMP->CR1 and TAMP->IER) and use a different NVIC IRQ line (TAMP_IRQn vs TAMP_STAMP_IRQn).

`ca_exti_hw_tamper_enable`

Header: `caos/drivers/inc/ca_glue_hw.h:148`
GUARD: `EN_CA_GLUE_HW`

```c
extern bool ca_exti_hw_tamper_enable(void); /* tamper 1 on PC13, rising edge, hardware erase + IRQ; false if not possible */
```

Enables hardware tamper detection on the MCU, triggering an interrupt upon physical security breach.

Ritorna:
Returns true if the tamper hardware was successfully initialized and the RTC clock is running; returns false if the RTC is not clocked or the hardware does not support tamper detection.

Note:

- This function is not thread-safe and must be called during system initialization before any security-critical tasks start. It is not intended to be called from an ISR. The function relies on the RTC peripheral being clocked; if the RTC is not enabled, the function will fail

immediately. A common mistake is calling this before the system clock or RTC peripheral has been properly configured, leading to a silent failure where tamper events are never detected.

Uso:

- This function is typically used in the security initialization phase of the application. It is paired with ca_exti_tamper_enable (which wraps this function with a callback) to register a handler that will be executed when a tamper event occurs. Once enabled, the system will automatically monitor the tamper pin (e.g., PC13 on some models) and trigger the registered callback if a breach is detected. To stop monitoring, use ca_exti_tamper_disable.

Esempio:

```c
/* Define a callback function to handle the tamper event */
void my_tamper_handler(void) {
    /* Log the security breach and perform emergency shutdown */
    ca_printf("Security breach detected!\n");
    /* Perform system reset or secure memory wipe */
    ca_os_reset();
}

void init_security(void) {
    /* Register the callback and enable hardware tamper detection */
    if (ca_exti_tamper_enable(my_tamper_handler)) {
        ca_printf("Tamper detection active.\n");
    } else {
        ca_printf("Failed to initialize tamper hardware.\n");
    }
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_exti_hw_tamper_enable in my caOS application?
A: This function should be called during the security initialization phase of your application, specifically during system initialization before any security-critical tasks start.

Code example conf:1.00 **Q:** Provide a minimal example of how to initialize hardware tamper detection.
A: void system_security_init(void) { if (ca_exti_hw_tamper_enable()) { /* Tamper monitoring active */ } else { /* Handle initialization failure */ } }

Return conf:1.00 **Q:** What do the return values of ca_exti_hw_tamper_enable indicate?
A: It returns true if the tamper hardware was successfully initialized and the RTC clock is running. It returns false if the hardware does not support tamper detection or if the RTC is not clocked.

Error/Edge case conf:1.00 **Q:** Why might ca_exti_hw_tamper_enable fail silently?
A: A common mistake is calling the function before the system clock or RTC peripheral has been properly configured. If the RTC is not enabled, the function will fail immediately because it relies on the RTC peripheral being clocked.

Behavior conf:1.00 **Q:** Is ca_exti_hw_tamper_enable thread-safe or ISR-safe?
A: The function is not thread-safe and must not be called from an ISR. It is intended to be called during system initialization.

HW differences conf:1.00 **Q:** How does the implementation of tamper detection differ between MCU families?
A: Implementations vary significantly: F1xx does not support it at all. F4xx, F7xx, and H7xx rely on the RTC peripheral (checking RCC_BDCR_RTCEN) and use RTC registers like TAMPCR. Conversely, H5xx and U5xx use the TAMP peripheral directly and do not require the RTC kernel clock, as edge detection only needs the APB clock.

ca_exti_init

Header: caos/drivers/inc/exti.h:41
Source: caos/drivers/src/exti.c:38:51
GUARD: EN_EXTI

```
bool ca_exti_init(u8 port, u8 pin, ca_exti_edge_t edge, ca_exti_cb_t cb)
```

Configures and enables an external interrupt (EXTI) on a specific GPIO pin with a callback function.

Parametri:

Nome	Tipo	Descrizione
port	u8	The GPIO port identifier (e.g., 0 for Port A, 1 for Port B). Must correspond to valid hardware ports.
pin	u8	The pin number (0-15). Values >= 16 are invalid and will cause the function to return false.
edge	ca_exti_edge_t	The trigger condition: rising edge, falling edge, or both.
cb	ca_exti_cb_t	A pointer to the function to be executed when the interrupt occurs. Must not be NULL.

Ritorna:
Returns true if the EXTI was successfully configured, false if the pin index is out of range or the callback pointer is NULL.

Note:

- This function is non-blocking and performs hardware-level configuration of the EXTI controller and the NVIC (Nested Vectored Interrupt Controller). It is not thread-safe if called simultaneously from multiple tasks, so it should typically be called during system initialization. It is not intended to be called from an ISR. A common beginner mistake is forgetting that the GPIO pin itself must be configured as an input using the appropriate GPIO driver functions before calling this; otherwise, the hardware will not detect the signal changes.

Uso:

- This function is used to set up event-driven hardware interactions, such as buttons, sensors, or communication signals that require immediate CPU attention. It is typically paired with GPIO initialization functions to set the pin mode, and often followed by ca_exti_disable() if the interrupt should not trigger immediately upon startup. In a standard lifecycle, you initialize the GPIO, call ca_exti_init to register the callback, and then use ca_exti_enable() or ca_exti_disable() to manage the interrupt state dynamically during runtime.

Esempio:

```
/* Define the callback function to be executed on interrupt */
void cb_button(void) {
    /* Handle the button press event here */
}

/* Initialize the EXTI on Pin 13 of Port A for rising edge */
if (ca_exti_init(0, 13, CA_EXTI_EDGE_RISING, cb_button)) {
    /* Successfully configured, now enable the interrupt */
    ca_exti_enable(13);
} else {
    /* Handle configuration error */
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_exti_init() in my caOS application?

A: You should call ca_exti_init() during the system initialization phase, before any interrupt handling is required. It is meant to configure an external interrupt line on a GPIO pin for rising or falling edge detection. This function must be called once per EXTI line you want to use, and it should be called before enabling the interrupt or using the pin for interrupt purposes.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_exti_init() with caOS types?

A:

```
u8 port = CA_GPIO_PORT_A;
u8 pin = 5;
ca_exti_edge_t edge = CA_EXTI_EDGE_FALLING;
ca_exti_cb_t cb = my_interrupt_handler;
bool success = ca_exti_init(port, pin, edge, cb);
```

Parameters conf:0.90 **Q:** What are the valid values for the 'port' and 'pin' parameters in ca_exti_init()?

A: The 'port' parameter must be a value between 0 and 5, representing GPIO ports (e.g., CA_GPIO_PORT_A = 0, CA_GPIO_PORT_B = 1). The 'pin' parameter must be between 0 and 15. If either parameter is outside these ranges, the function returns false.

Return conf:0.95 **Q:** What does ca_exti_init() return, and what causes it to return false?

A: The function returns true if initialization is successful. It returns false if the pin number is invalid (>=16) or if the callback function pointer is NULL. These are the only two failure conditions described in the documentation.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_exti_init()?

A: Common mistakes include passing a pin number >=16, passing a NULL callback function, or calling this function from within an ISR. Additionally, failing to configure the GPIO pin as an input before calling this function may lead to unexpected behavior, although the function itself does not check for this.

`ca_exti_tamper_disable`

Header: `caos/drivers/inc/exti.h:61`
Source: `caos/drivers/src/exti.c:89:93`
GUARD: `EN_EXTI`

```
void ca_exti_tamper_disable(void)
```

Disables the hardware tamper detection mechanism and clears the associated callback.

Ritorna:
This function returns void and does not provide status feedback.

Note:

- This function is a wrapper that interacts with hardware registers via ca_exti_hw_tamper_disable (internal) and resets the internal callback pointer. It is not thread-safe by itself; if multiple tasks attempt to enable or disable the tamper feature simultaneously, you should wrap the call in a critical section using ca_enter_critical and ca_exit_critical. It is generally safe to call from an ISR, but ensure that no other task is concurrently modifying the tamper state. A common mistake is calling this function without having previously initialized the EXTI system, which may lead to undefined behavior depending on the underlying hardware state.

Uso:

- This function is used to deactivate security tamper monitoring, typically when the system enters a maintenance mode or a trusted state where physical intrusion detection is no longer required. It is usually paired with ca_exti_tamper_enable, which is used to arm the system. In a typical lifecycle, you would call ca_exti_init to set up the EXTI pins, then ca_exti_tamper_enable to start monitoring, and finally ca_exti_tamper_disable when the monitoring session is complete or the device is being shut down.

Esempio:

```c
/* Example: Disabling tamper detection via CLI command */
#include "ca_os.h"

void handle_tamper_off_command(void) {
    /* Disable the tamper detection hardware and clear the callback */
    ca_exti_tamper_disable();

    /* Notify the user via the CLI output */
    ca_stdout_put_n("TAMPER: Detection disabled successfully.", 40);

    /* Optional: If the system needs to re-arm later, use ca_exti_tamper_enable
    */
    /* ca_exti_tamper_enable(my_tamper_callback_function); */
}
```

Q&A — fine-tuning review *2/2 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_exti_tamper_disable in my application lifecycle?
A: You should call this function when you need to deactivate security tamper monitoring, such as when the system enters a maintenance mode, a trusted state, or when the device is being shut down. It is typically used to disarm the system after it has been enabled via ca_exti_tamper_enable.

Code example conf:1.00 **Q:** How do I safely disable the tamper detection mechanism in caOS?
A: Since the function is not thread-safe, you should wrap it in a critical section if there is a risk of concurrent access. Example: ca_enter_critical(); ca_exti_tamper_disable(); ca_exit_critical();

`ca_exti_tamper_dispatch`

Header: `caos/drivers/inc/ca_glue_hw.h:150`
Source: `caos/drivers/src/exti.c:75:78`
GUARD: `EN_CA_GLUE_HW`

```c
extern void ca_exti_tamper_dispatch(void); /* generic: called by the
   tamper ISR in exti_<family>.c */
```

Dispatches the registered tamper callback function when a hardware tamper event occurs.

Ritorna:
This function returns void and does not provide a status code.

Note:

- This function is designed to be called exclusively from within the hardware EXTI interrupt service routine (ISR) responsible for tamper detection. Because it executes inside an ISR, the callback function it triggers must be extremely fast, non-blocking, and must not attempt to acquire mutexes or perform operations that could lead to a deadlock. A common mistake is performing heavy processing or logging inside the callback; keep the logic limited to setting a flag or notifying a high-priority task via a signal or event.

Uso:

- This function is the bridge between the low-level hardware interrupt and your application logic. You typically pair this with ca_exti_tamper_enable to activate the hardware detection and define a callback function that the system will execute when the tamper pin is triggered. It is used in security-critical applications where immediate action, such as clearing sensitive memory or triggering a system reset, is required upon physical intrusion.

Esempio:

```c
/* Define the callback to be executed on tamper event */
void my_tamper_handler(void) {
    /* Perform immediate security action, e.g., clear sensitive keys */
    /* ca_memory_free_all_owned_by(task_id); */
}

void setup_security(void) {
    /* Initialize the tamper hardware and register the callback */
    /* Assuming a hypothetical registration function exists or is handled via
    internal pointer */
    ca_exti_tamper_enable();
}

/* Inside exti_<family>.c, the ISR calls this: */
void EXTI_Tamper_IRQHandler(void) {
    /* Clear hardware pending bit here */
    ca_exti_tamper_dispatch();
}
```

Q&A — fine-tuning review *2/2 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_exti_tamper_dispatch and what are the constraints on the callback it triggers?
A: You should call ca_exti_tamper_dispatch exclusively from within the hardware EXTI interrupt service routine (ISR) responsible for tamper detection. Because it executes inside an ISR, the triggered callback must be extremely fast and non-blocking. You must not attempt to acquire mutexes or perform heavy processing, as this could lead to deadlocks; instead, limit logic to setting a flag or notifying a high-priority task.

Code example conf:0.95 **Q:** How do I implement the tamper ISR to use ca_exti_tamper_dispatch in my STM32 project?
A: You place the function call inside the EXTI ISR handler. Example: void EXTI_Tamper_IRQHandler { ca_exti_tamper_dispatch(); } Ensure you have previously called ca_exti_tamper_enable to activate the hardware detection and registered your callback function.

`ca_exti_tamper_enable`

Header: `caos/drivers/inc/exti.h:60`
Source: `caos/drivers/src/exti.c:80:87`
GUARD: `EN_EXTI`

```c
bool ca_exti_tamper_enable(ca_exti_cb_t cb)
```

Registers a callback function to be executed when a hardware tamper event is detected.

Parametri:

Nome	Tipo	Descrizione
`cb`	`ca_exti_cb_t`	A pointer to the function to be called upon tamper detection. Must not be NULL.

Ritorna:
Returns true if the tamper interrupt was successfully enabled and the callback registered; returns false if the callback pointer is NULL or if the hardware initialization failed.

Note:

- This function is not thread-safe and should be called during system initialization or from a controlled task context. The callback provided will be executed directly within the context of the EXTI interrupt service routine (ISR), meaning it must be extremely fast, must not contain blocking calls (like delays or mutex locks), and should only perform minimal operations like setting a flag or signaling a task. A common beginner mistake is performing heavy processing or logging inside the callback, which can lead to system instability or missed interrupts. Ensure the hardware tamper pins are correctly configured before calling this function.

Uso:

- This function is typically used in security-sensitive applications to protect sensitive data stored in backup SRAM or internal memory. It is usually paired with ca_exti_tamper_disable to stop monitoring when the system is in a safe state. The typical sequence is: 1) Initialize the system and memory, 2) Define a callback function that clears sensitive data, 3) Call ca_exti_tamper_enable to arm the security mechanism, 4) Call ca_exti_tamper_disable when the system is powered down or entering a maintenance mode.

Esempio:

```
/* Callback function to be executed on tamper event */
void my_tamper_handler(void) {
    /* Perform immediate security actions, e.g., zeroing out sensitive memory */
    /* ca_memory_free_all_owned_by(TASK_ID); */
}

void setup_security(void) {
    /* Register the tamper callback */
    if (ca_exti_tamper_enable(my_tamper_handler)) {
        /* Tamper detection is now active */
    } else {
        /* Handle error: callback was NULL or hardware failed */
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_exti_tamper_enable in my caOS application?
A: It is used in security-sensitive applications to protect data in backup SRAM or internal memory. You should call it during system initialization after configuring hardware tamper pins, and pair it with ca_exti_tamper_disable when entering maintenance mode or powering down.

Code example conf:0.90 **Q:** How do I implement a callback and register it using ca_exti_tamper_enable?
A: The callback must be fast and non-blocking. Example: void my_tamper_handler(void) { flag_tamper_detected = 1; } ... if (ca_exti_tamper_enable(my_tamper_handler)) { /* success */ }

Parameters conf:1.00 **Q:** What are the requirements for the ca_exti_cb_t parameter?
A: The parameter must be a pointer to a function (callback). It must not be NULL. The function will execute directly within the EXTI ISR context, so it must avoid blocking calls like delays or mutex locks.

Return conf:1.00 **Q:** What does a false return value from ca_exti_tamper_enable indicate?
A: A false return indicates that the operation failed. This occurs if the provided callback pointer is NULL or if the hardware initialization for the tamper interrupt failed.

ca_fat_cat

Header: caos/drivers/inc/fat_service.h:67
Source: caos/drivers/src/fat_service.c:414:465
GUARD: EN_FAT_SERVICE

```
bool ca_fat_cat(const char *name); /* heap round trip + hexdump */
```

Reads an entire file from the filesystem into heap memory and displays its contents via hexdump.

Parametri:

Nome	Tipo	Descrizione
name	const char *	The null-terminated string path/filename of the file to read. Must be a valid path within the mounted filesystem.

Ritorna:
Returns true if the file was successfully read and dumped, false if the file could not be opened, memory allocation failed, or the file size exceeded the available heap safety margin.

Note:

- This function is blocking and performs dynamic memory allocation on the heap. It is NOT safe to call from an ISR because it uses mutexes (ca_fat_lock) and heap management functions that are not re-entrant. A common mistake is calling this on very large files; the function includes a safety check that prevents allocation if the file size is too close to the current free heap, which will cause the function to return false to protect system stability. Ensure the filesystem is mounted via ca_fat_mount before calling this, or it will fail immediately.

Uso:

- Use this function for debugging or verifying file contents during development. It is typically used in a sequence: first ensure the filesystem is ready with ca_fat_is_mounted, then call ca_fat_cat to inspect a specific file, and finally ensure no other tasks are starving for memory. It is often paired with ca_fat_ls to verify the file exists before attempting to read it.

Esempio:

```
if (ca_fat_is_mounted()) {
    /* Attempt to read and display the contents of 'config.bin' */
    if (!ca_fat_cat("config.bin")) {
        /* Log error if file read failed or memory was insufficient */
        ca_printf_n("Failed to cat config.bin");
    }
} else {
    ca_printf_n("Filesystem not mounted, cannot cat file.");
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_fat_cat function in my caOS project?
A: You should use ca_fat_cat primarily for debugging or verifying file contents during development. It is typically used after confirming the filesystem is mounted via ca_fat_is_mounted and often after using ca_fat_ls to verify the file exists.

Code example conf:0.90 **Q:** Provide a minimal C example of how to call ca_fat_cat using caOS types.
A: cstr filename = "/logs/debug.txt"; if (ca_fat_is_mounted()) { bool success = ca_fat_cat(filename); }

Parameters conf:1.00 **Q:** What does the 'name' parameter represent in ca_fat_cat?
A: The 'name' parameter is a null-terminated string (cstr) representing the path or filename of the file to be read. It must be a valid path within the currently mounted filesystem.

Return conf:1.00 **Q:** What do the boolean return values of ca_fat_cat indicate?
A: Returning true indicates the file was successfully read and dumped. Returning false indicates the operation failed, which can be caused by an inability to open the file, a heap memory allocation failure, or the file size exceeding the available heap safety margin.

Error/Edge case conf:1.00 **Q:** Why might ca_fat_cat return false even if the file exists?
A: A common mistake is attempting to read a file that is too large. The function includes a safety check that prevents allocation if the file size is too close to the current free heap to protect system stability, resulting in a false return.

Behavior conf:1.00 **Q:** Is ca_fat_cat safe to call from an interrupt service routine (ISR)?
A: No, it is not safe to call from an ISR. The function is blocking, performs dynamic heap allocation, and uses mutexes (ca_fat_lock) that are not re-entrant, which would violate ISR safety requirements.

`ca_fat_cd`

Header: `caos/drivers/inc/fat_service.h:61`
Source: `caos/drivers/src/fat_service.c:296:309`
GUARD: `EN_FAT_SERVICE`

```
1 bool ca_fat_cd(const char *dir)
```

Changes the current working directory of the FAT filesystem service.

Parametri:

Nome	Tipo	Descrizione
`dir`	`const char *`	Path string of the target directory. Use ".." for parent, "/" for root, or a folder name. Must be a valid null-terminated string.

Ritorna:
Returns true if the directory was successfully changed, false otherwise (e.g., directory not found, filesystem not mounted, or access error).

Note:

- This function is blocking and thread-safe, as it internally uses a mutex lock to ensure exclusive access to the FAT filesystem state. It must NOT be called from an ISR because it performs blocking operations that could lead to system instability. Before calling this, ensure the FAT service is initialized and mounted; calling this on an unmounted filesystem will trigger an error and return false. A common mistake is passing a NULL pointer or a path that exceeds the maximum path length supported by the underlying FAT driver.

Uso:

- This function is used to navigate the directory structure of an SD card or flash storage managed by the FAT service. It is typically used in a sequence: first, ensure the service is ready with ca_fat_is_mounted, then use ca_fat_cd to navigate to a specific folder, and

finally perform file operations like ca_fat_ls, ca_fat_cat, or ca_fat_touch. It is often paired with ca_fat_pwd to verify the current location after a change.

Esempio:

```
if (ca_fat_is_mounted()) {
    // Attempt to enter the 'logs' directory
    if (ca_fat_cd("logs")) {
        // Successfully changed directory, now list files inside
        ca_fat_ls("");
    } else {
        // Handle error: directory might not exist
        ca_printf("Failed to change directory to logs\n");
    }
} else {
    ca_printf("Filesystem not mounted!\n");
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_fat_cd function in my caOS application?
A: You should use ca_fat_cd to navigate the directory structure of your storage media, such as an SD card, managed by the FAT service. It is typically called after verifying the filesystem is mounted with ca_fat_is_mounted, and before performing file operations like ca_fat_ls, ca_fat_cat, or ca_fat_touch.

Code example conf:0.90 **Q:** Provide a minimal C example of how to change the directory to the root folder using caOS types.
A: cstr path = "/"; if (ca_fat_cd(path)) { /* Success */ } else { /* Handle error */ }

Parameters conf:1.00 **Q:** What are the valid values for the dir parameter in ca_fat_cd?
A: The dir parameter must be a valid null-terminated string. You can use "/" to navigate to the root directory, ".." to move to the parent directory, or a specific folder name string.

Return conf:1.00 **Q:** What do the return values of ca_fat_cd indicate, and why might it return false?
A: The function returns true if the directory change is successful. It returns false if the directory is not found, the filesystem is not mounted, or if there is an access error.

Error/Edge case conf:1.00 **Q:** Can I call ca_fat_cd from an interrupt service routine (ISR)?
A: No, you must not call ca_fat_cd from an ISR. The function is blocking and thread-safe, using a mutex internally; calling it from an ISR could lead to system instability.

`ca_fat_fdisk`

Header: `caos/drivers/inc/fat_service.h:55`
Source: `caos/drivers/src/fat_service.c:148:180`
GUARD: `EN_FAT_SERVICE`

```
bool ca_fat_fdisk(u32 mb); /* MBR only, 0 -> 64 MB */
```

Creates a new MBR partition table on the storage device, effectively wiping existing data.

Parametri:

Nome	Tipo	Descrizione
`mb`	`u32`	Size of the partition in Megabytes. If 0 is passed, it defaults to 64 MB.

Ritorna:
Returns true if the partition table was successfully written and the card re-initialized, false otherwise.

Note:

- This function is highly destructive as it overwrites the Master Boot Record (MBR) of the storage device. It is a blocking operation that uses internal locks to ensure thread safety, so it must not be called from an ISR. Before calling this, ensure the FAT service is initialized. A common mistake is assuming this function formats the drive; it only creates the partition table. You must follow this with ca_fat_mkfs() to create the filesystem, otherwise, the drive will remain unreadable.

Uso:

- This function is used during the initial setup of a storage medium, typically when preparing a new SD card or resetting a corrupted one. It is part of a mandatory sequence: first call ca_fat_fdisk() to define the partition, then ca_fat_mkfs() to format the volume, and finally ca_fat_mount() to make the files accessible. It is often used in factory test routines or system initialization sequences where the storage state must be guaranteed.

Esempio:

```
/* Initialize the FAT service before attempting to partition */
ca_fat_service_init();

/* Create a 64MB partition on the storage device */
if (ca_fat_fdisk(64U)) {
    /* Partition successful, now format the partition to create the filesystem
    */
    if (ca_fat_mkfs()) {
        /* Finally, mount the filesystem to start using it */
        if (ca_fat_mount()) {
            ca_stdout_printf_n("Storage ready for use.");
        }
    } else {
        ca_stdout_printf_n("Format failed.");
    }
} else {
    ca_stdout_printf_n("Partitioning failed.");
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_fat_fdisk function in my caOS project?
A: You should use ca_fat_fdisk during the initial setup of a storage medium, such as preparing a new SD card or resetting a corrupted one. It is the first step in the mandatory sequence: call ca_fat_fdisk, then ca_fat_mkfs, and finally ca_fat_mount.

Code example conf:0.90 **Q:** Provide a minimal C example of how to initialize a 128MB partition using caOS types.
A: u32 size = 128; if (ca_fat_fdisk(size)) { ca_fat_mkfs(); ca_fat_mount(); }

Parameters conf:1.00 **Q:** What does the u32 mb parameter represent and what happens if I pass 0?
A: The u32 mb parameter defines the size of the partition in Megabytes. If you pass 0, the function defaults to a size of 64 MB.

Return conf:1.00 **Q:** What do the boolean return values of ca_fat_fdisk indicate?
A: A return value of true indicates that the partition table was successfully written and the card was re-initialized. A return value of false indicates that the operation failed.

Error/Edge case conf:1.00 **Q:** What is a common mistake developers make when using ca_fat_fdisk?
A: A common mistake is assuming that ca_fat_fdisk formats the drive. It only creates the partition table; you must follow it with ca_fat_mkfs to create the filesystem, or the drive will remain unreadable.

Behavior conf:1.00 **Q:** Is ca_fat_fdisk safe to call from an ISR or a multi-threaded environment?

A: It is thread-safe due to internal locks, but it is a blocking operation and must NOT be called from an ISR. Additionally, you must ensure the FAT service is initialized before calling it.

---

ca_fat_free

Header: caos/drivers/inc/fat_service.h:59
Source: caos/drivers/src/fat_service.c:253:276
GUARD: EN_FAT_SERVICE

```
bool ca_fat_free(void)
```

Calculates and prints the total, free, and used space on the currently mounted FAT filesystem.

Ritorna:
Returns true if the space calculation was successful and printed to stdout; returns false if the filesystem is not mounted or if the underlying FAT driver returns an error.

Note:

- This function is blocking because it performs filesystem operations and uses a mutex lock (ca_fat_lock) to ensure thread safety during the calculation. It must NOT be called from an ISR because it performs blocking I/O and takes a mutex, which would cause a system crash or deadlock. Before calling this, the FAT service must be initialized and a filesystem must be successfully mounted using ca_fat_mount. A common mistake is calling this function immediately after system boot without waiting for the storage media to be ready or mounted.

Uso:

- This function is typically used in diagnostic tasks or CLI commands to monitor storage health and capacity. It is usually paired with ca_fat_mount to ensure the filesystem is accessible before requesting space statistics. In a typical lifecycle, you would call ca_fat_mount, then use ca_fat_free to verify available space before performing file write operations, and finally call ca_fat_umount when the storage is no longer needed.

Esempio:

```
/* Check if the filesystem is ready before requesting space */
if (ca_fat_is_mounted()) {
    if (ca_fat_free()) {
        ca_stdout_put_n("Space report generated successfully.");
    } else {
        ca_stdout_put_n("Failed to retrieve space information.");
    }
} else {
    ca_stdout_put_n("Filesystem not mounted. Attempting to mount...");
    if (ca_fat_mount()) {
        ca_fat_free();
    }
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_fat_free in my application lifecycle?
A: You should call ca_fat_free during diagnostic tasks or CLI commands to monitor storage health and capacity. It is typically used after calling ca_fat_mount to verify available space before performing file write operations, and before calling ca_fat_umount.

Code example conf:0.90 **Q:** Provide a minimal C example of how to check filesystem space using caOS types.
A: bool status = ca_fat_mount(); if (status) { bool result = ca_fat_free(); }

Return conf:1.00 **Q:** What do the return values of ca_fat_free indicate?
A: It returns true if the space calculation was successful and printed to stdout. It returns false if the filesystem is not mounted or if the underlying FAT driver returns an error.

Error/Edge case conf:1.00 **Q:** What is a common mistake developers make when calling ca_fat_free?
A: A common mistake is calling this function immediately after system boot without waiting for the storage media to be ready or mounted.

Behavior conf:1.00 **Q:** Is ca_fat_free safe to call from an Interrupt Service Routine (ISR)?
A: No, it must not be called from an ISR. The function is blocking, performs filesystem I/O, and uses a mutex lock (ca_fat_lock), which would cause a system crash or deadlock if called from an ISR.

`ca_fat_is_mounted`

Header: `caos/drivers/inc/fat_service.h:48`
Source: `caos/drivers/src/fat_service.c:75:75`
GUARD: `EN_FAT_SERVICE`

```
bool ca_fat_is_mounted(void)
```

Checks if the FAT file system is currently mounted and ready for operations.

Ritorna:
Returns true if the file system is mounted, false otherwise.

Note:

- This function is non-blocking and thread-safe as it simply reads a status flag from the internal FAT service structure. It is safe to call from an ISR, although it is rarely useful to do so since file system operations themselves are generally not ISR-safe. The most common beginner mistake is assuming that a 'true' return value guarantees that a specific file exists or that the SD card is physically present; it only confirms that the software mount process has completed successfully. Always ensure the FAT service has been initialized via ca_fat_service_init before relying on this status.

Uso:

- This function is typically used as a guard clause before performing any file system operations like reading, writing, or listing files. It is standard practice to call this immediately after a user command or a system event to prevent attempting operations on an uninitialized or unmounted drive. It is frequently paired with ca_fat_mount to initialize the drive and ca_fat_umount to safely disconnect it, ensuring the application logic does not crash or hang when the storage medium is removed.

Esempio:

```
void process_file_command(void) {
    /* Check if the file system is ready before proceeding */
    if (!ca_fat_is_mounted()) {
        ca_stdout_printf_n("Error: FAT system is not mounted. Please mount the drive first.");
        return;
    }

    /* Proceed with file operations if mounted */
    ca_fat_lock();
```

```
    /* Perform file listing or other operations here */
    ca_fat_ls();
    ca_fat_unlock();
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_fat_is_mounted in my application logic?
A: You should use it as a guard clause before performing any file system operations like reading, writing, or listing files. It is standard practice to call this immediately after a user command or system event to prevent attempting operations on an uninitialized or unmounted drive.

Code example conf:0.90 **Q:** How do I check if the file system is ready before attempting a file operation?
A: You should verify the mount status using a conditional check. Example: if (ca_fat_is_mounted()) { /* perform file operations */ } else { /* handle error */ }.

Return conf:1.00 **Q:** What do the return values of ca_fat_is_mounted indicate, and does 'true' mean my SD card is definitely present?
A: A return value of true indicates the software mount process has completed successfully, while false indicates it is not mounted. A 'true' return does not guarantee that the SD card is physically present or that a specific file exists; it only confirms the software mount status.

`ca_fat_lock`

Header: `caos/drivers/inc/fat_service.h:44`
Source: `caos/drivers/src/fat_service.c:71:71`
GUARD: `EN_FAT_SERVICE`

```
void ca_fat_lock(void)
```

Acquires a recursive mutex to ensure exclusive access to the FAT filesystem service.

Ritorna:
This function returns void. It blocks the calling task until the mutex is acquired.

Note:

- This function is a blocking call; if another task currently holds the lock, the calling task will be suspended by the RTOS scheduler until the lock becomes available. It is not safe to call this function from an Interrupt Service Routine (ISR) because ISRs cannot be suspended. Because it uses a recursive mutex (rmutex), the same task can call this function multiple times without deadlocking itself, provided it calls ca_fat_unlock() an equal number of times. The most common beginner mistake is forgetting to call ca_fat_unlock() in an error path, which will permanently hang all other tasks attempting to access the filesystem.

Uso:

- Use this function whenever you need to perform a sequence of filesystem operations (like opening, reading, and closing a file) that must be treated as an atomic transaction. It is typically used in a pair: call ca_fat_lock() before starting your file operations, and ensure ca_fat_unlock() is called immediately after, even if an error occurs during the process. This prevents data corruption or race conditions when multiple tasks attempt to access the SD card or flash storage simultaneously.

Esempio:

```c
void log_data_to_file(const char *data) {
    FIL file;
    // Ensure exclusive access to the FAT filesystem before starting operations
    ca_fat_lock();

    FRESULT fr = f_open(&file, "log.txt", FA_WRITE | FA_OPEN_APPEND);
    if (fr == FR_OK) {
        UINT written;
        // Perform the write operation
        f_write(&file, data, ca_strlen(data), &written);
        f_close(&file);
    } else {
        ca_printf_n("Failed to open log file: %s", ca_fat_strerror((i32)fr));
    }

    // Always release the lock after the critical section is finished
    ca_fat_unlock();
}
```

Q&A — fine-tuning review *2/2 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_fat_lock() and why is it necessary in a multi-tasking environment?
A: You should call ca_fat_lock() whenever you perform a sequence of filesystem operations that must be treated as an atomic transaction, such as opening, reading, and closing a file. It is necessary because it acquires a recursive mutex to ensure exclusive access to the FAT filesystem service, preventing data corruption or race conditions when multiple tasks attempt to access storage simultaneously.

Code example conf:1.00 **Q:** How do I correctly implement a file operation sequence using ca_fat_lock() and ca_fat_unlock()?
A: You must wrap your filesystem operations in a lock/unlock pair to ensure atomicity. Example: ca_fat_lock(); /* Perform file operations here */ ca_fat_unlock(); Ensure that ca_fat_unlock() is called even if an error occurs during the process to prevent hanging other tasks.

`ca_fat_ls`

Header: `caos/drivers/inc/fat_service.h:62`
Source: `caos/drivers/src/fat_service.c:311:339`
GUARD: `EN_FAT_SERVICE`

```c
bool ca_fat_ls(const char *dir); /* NULL or "" -> current directory */
```

Lists the contents of a specified directory on the FAT filesystem to the standard output.

Parametri:

Nome	Tipo	Descrizione
`dir`	`const char *`	Path to the directory to list. Pass NULL or an empty string ("") to list the current working directory.

Ritorna:
Returns true if the directory was successfully opened and listed, false otherwise (e.g., directory not found, filesystem not mounted, or internal error).

Note:

- This function is blocking, meaning the calling task will wait until the entire directory content is read and printed. It is thread-safe because it internally uses ca_fat_lock() and ca_fat_unlock() to ensure exclusive access to the filesystem driver. It must NOT be called from an ISR (Interrupt Service Routine) because it performs blocking I/O operations. A common mistake is calling this before the filesystem is initialized or mounted, which will trigger an error message and return false.

Uso:

- This function is typically used for debugging or user interaction to verify the presence of files after creation or before deletion. It is often paired with ca_fat_cd() to navigate the filesystem or ca_fat_touch() to verify that a file was successfully created. In a typical sequence, you would initialize the service with ca_fat_service_init(), mount the drive, perform operations like ca_fat_mkdir() or ca_fat_touch(), and then use ca_fat_ls() to confirm the state of the directory.

Esempio:

```
/* Ensure the FAT service is initialized and mounted before listing */
if (ca_fat_is_mounted()) {
    /* List the root directory */
    if (!ca_fat_ls("")) {
        ca_printf("Failed to list root directory\n");
    }

    /* Create a file and verify it exists */
    ca_fat_touch("test.txt");
    ca_printf("Listing directory after creating test.txt:\n");
    ca_fat_ls(NULL);
} else {
    ca_printf("Filesystem not mounted!\n");
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_fat_ls function in my caOS application?
A: You should use ca_fat_ls for debugging or user interaction to verify the presence of files after creation or before deletion. It is commonly used to confirm the state of the filesystem after operations like ca_fat_mkdir or ca_fat_touch.

Code example conf:0.90 **Q:** Provide a minimal C example of how to list the current directory using caOS types.
A: void list_files(void) { cstr path = ""; bool success = ca_fat_ls(path); }

Parameters conf:1.00 **Q:** What are the valid values for the dir parameter in ca_fat_ls?
A: The dir parameter accepts a path string. Passing NULL or an empty string ("") will list the contents of the current working directory.

Return conf:1.00 **Q:** What do the return values of ca_fat_ls indicate?
A: It returns true if the directory was successfully opened and listed. It returns false if the directory was not found, the filesystem is not mounted, or an internal error occurred.

Error/Edge case conf:1.00 **Q:** What is a common mistake that causes ca_fat_ls to fail?
A: A common mistake is calling ca_fat_ls before the filesystem has been initialized or mounted, which will trigger an error message and cause the function to return false.

Behavior conf:1.00 **Q:** Is ca_fat_ls safe to call from an ISR or a multi-threaded environment?
A: It is thread-safe because it uses internal locking, but it must NOT be called from an ISR because it performs blocking I/O operations. The calling task will wait until the entire directory content is read and printed.

ca_fat_mkdir

Header: caos/drivers/inc/fat_service.h:63
Source: caos/drivers/src/fat_service.c:341:354
GUARD: EN_FAT_SERVICE

```
bool ca_fat_mkdir(const char *dir)
```

Creates a new directory on the mounted filesystem.

Parametri:

Nome	Tipo	Descrizione
dir	const char *	The path and name of the directory to create. Must be a valid null-terminated string.

Ritorna:
Returns true if the directory was created successfully, false otherwise (e.g., filesystem not mounted, directory already exists, or disk error).

Note:

- This function is blocking, meaning the calling task will wait until the filesystem operation completes. It is thread-safe because it internally uses ca_fat_lock and ca_fat_unlock to manage access to the shared filesystem resource. You must ensure the filesystem is mounted via ca_fat_mount before calling this; calling it while unmounted will trigger an error. Do not call this function from an ISR, as it performs blocking operations that are not permitted in interrupt context.

Uso:

- This function is typically used during filesystem setup or when organizing data logs into specific folders. It is often paired with ca_fat_cd to enter the newly created directory or ca_fat_ls to verify its existence. In a typical workflow, you would initialize the service with ca_fat_service_init, mount the drive, create your directory structure, and finally perform file operations like ca_fat_touch or ca_fat_cat.

Esempio:

```
if (ca_fat_is_mounted()) {
    // Attempt to create a new directory for logs
    if (ca_fat_mkdir("logs")) {
        ca_printf("Directory created successfully.\n");
        // Move into the new directory to start writing files
        if (ca_fat_cd("logs")) {
            ca_fat_touch("data.txt");
        }
    } else {
        ca_printf("Failed to create directory. It might already exist.\n");
    }
} else {
    ca_printf("Filesystem not mounted!\n");
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_fat_mkdir function in my caOS application?
A: You should use ca_fat_mkdir during filesystem setup or when organizing data logs into specific folders. It is typically used after mounting the filesystem and can be followed by ca_fat_cd to enter the directory or ca_fat_ls to verify its creation.

Code example conf:0.90 **Q:** Provide a minimal C example of how to create a directory using caOS types.
A: cstr path = "/logs/data"; if (ca_fat_mkdir(path)) { /* Directory created successfully */ }

Parameters conf:1.00 **Q:** What are the requirements for the 'dir' parameter in ca_fat_mkdir?
A: The 'dir' parameter must be a valid null-terminated string representing the path and name of the directory you wish to create.

Return conf:1.00 **Q:** What does the boolean return value of ca_fat_mkdir indicate?
A: It returns true if the directory was created successfully. It returns false if the filesystem is not mounted, the directory already exists, or if a disk error occurred.

Error/Edge case conf:1.00 **Q:** Are there any specific restrictions or common mistakes to avoid when calling ca_fat_mkdir?
A: Yes: do not call this function from an ISR because it is a blocking operation. You must also ensure the filesystem is mounted via ca_fat_mount before calling it, otherwise, it will trigger an error.

`ca_fat_mkfs`

Header: `caos/drivers/inc/fat_service.h:56`
Source: `caos/drivers/src/fat_service.c:182:200`
GUARD: `EN_FAT_SERVICE`

```
bool ca_fat_mkfs(void); /* FAT32 on the fdisk partition */
```

Formats the storage media with a FAT32 filesystem.

Ritorna:
Returns true if the formatting process completed successfully, false otherwise. A false return indicates a filesystem error, which can be further investigated using ca_fat_strerror.

Note:

- This function is destructive; it erases all existing data on the partition. It is a blocking operation, meaning the calling task will wait until the format process completes, which can take significant time depending on the storage size. It is not safe to call from an ISR. You must ensure the FAT service is initialized and the partition table has been created via ca_fat_fdisk before calling this. A common mistake is calling this while the volume is still mounted or in use by other tasks, which will lead to unpredictable behavior or errors.

Uso:

- This function is typically used during the initial setup of a storage device or when a factory reset is required. It is part of a standard sequence: first, call ca_fat_fdisk to create the partition table, then call ca_fat_mkfs to format the partition, and finally call ca_fat_mount to make the filesystem accessible. It should be used sparingly as it wipes all data on the storage medium.

Esempio:

```
/* Initialize the FAT service before performing operations */
ca_fat_service_init();

/* Create a 64MB partition table first */
if (ca_fat_fdisk(64U)) {
    /* Format the partition with FAT32 */
    if (ca_fat_mkfs()) {
        /* Mount the newly formatted volume to make it usable */
```

```
        if (ca_fat_mount()) {
            ca_printf("Storage ready for use.\n");
        } else {
            ca_printf("Failed to mount volume.\n");
        }
    } else {
        ca_printf("Formatting failed.\n");
    }
} else {
    ca_printf("Partitioning failed.\n");
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_fat_mkfs function in my caOS application?
A: You should use ca_fat_mkfs during the initial setup of a storage device or when performing a factory reset. It is part of a standard sequence: first call ca_fat_fdisk to create the partition table, then call ca_fat_mkfs to format the partition, and finally call ca_fat_mount to make the filesystem accessible.

Code example conf:0.90 **Q:** Provide a minimal C example showing how to initialize and format a storage device using caOS types.
A: void setup_storage(void) { if (ca_fat_fdisk()) { if (ca_fat_mkfs()) { ca_fat_mount(); } } }

Return conf:1.00 **Q:** What do the return values of ca_fat_mkfs indicate?
A: The function returns true if the formatting process completed successfully. A false return indicates a filesystem error, which can be further investigated using the ca_fat_strerror function.

Error/Edge case conf:1.00 **Q:** What are some common mistakes or edge cases to avoid when calling ca_fat_mkfs?
A: A common mistake is calling this function while the volume is still mounted or in use by other tasks, which leads to unpredictable behavior or errors. Additionally, you must ensure the FAT service is initialized and the partition table has been created via ca_fat_fdisk before calling it.

Behavior conf:1.00 **Q:** Is ca_fat_mkfs safe to call from an ISR, and does it block execution?
A: No, it is not safe to call from an ISR. It is a blocking operation, meaning the calling task will wait until the format process completes, which can take significant time depending on the storage size.

`ca_fat_mount`

Header: `caos/drivers/inc/fat_service.h:57`
Source: `caos/drivers/src/fat_service.c:202:236`
GUARD: `EN_FAT_SERVICE`

```
bool ca_fat_mount(void)
```

Mounts the filesystem volume to make it accessible for file operations.

Ritorna:
Returns true if the volume was successfully mounted or was already mounted. Returns false if the mount operation failed (e.g., hardware error, invalid filesystem, or unformatted media).

Note:

- This function is blocking and thread-safe, as it uses internal locking mechanisms to ensure exclusive access to the filesystem driver. It must NOT be called from an ISR because

it performs blocking operations and potentially waits for hardware access. Before calling this, ensure the underlying storage driver (e.g., SDIO or QSPI) is initialized. A common mistake is attempting to perform file operations before checking the return value of this function, which will lead to undefined behavior or system crashes.

Uso:

- This function is the entry point for all filesystem activity. It is typically called after hardware initialization and formatting (e.g., ca_fat_fdisk and ca_fat_mkfs) to prepare the volume for use. Once mounted, you can proceed to use functions like ca_fat_mkdir, ca_fat_write_enc, or ca_fat_ls. It is usually paired with ca_fat_umount when the filesystem needs to be safely detached, for example, before entering a low-power sleep mode or removing physical media.

Esempio:

```
/* Ensure the filesystem is ready for use */
if (ca_fat_mount()) {
    /* Mount successful, safe to perform file operations */
    ca_fat_mkdir("logs");
    ca_fat_touch("logs/system.log");
} else {
    /* Handle mount failure, perhaps log to system console */
    ca_stdout_printf_n("Failed to mount filesystem");
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_fat_mount in my application lifecycle?
A: It should be called after the underlying storage driver (e.g., SDIO or QSPI) is initialized. It serves as the entry point for all filesystem activity and is typically called before performing operations like ca_fat_mkdir or ca_fat_write_enc. It is often paired with ca_fat_umount when detaching the media.

Code example conf:0.90 **Q:** Provide a minimal C example showing how to initialize and mount the filesystem.
A: void init_fs(void) { /* Initialize storage driver first */ if (ca_fat_mount()) { /* Proceed with file operations */ } else { /* Handle mount failure */ } }

Return conf:1.00 **Q:** What do the return values of ca_fat_mount indicate?
A: It returns true if the volume was successfully mounted or was already mounted. It returns false if the operation failed, which can be caused by hardware errors, an invalid filesystem, or unformatted media.

Error/Edge case conf:1.00 **Q:** What is a common mistake developers make when using ca_fat_mount?
A: A common mistake is attempting to perform file operations before checking the return value of ca_fat_mount, which will lead to undefined behavior or system crashes.

Behavior conf:1.00 **Q:** Is ca_fat_mount safe to call from an interrupt service routine (ISR)?
A: No, it must not be called from an ISR because it is a blocking function that performs operations and potentially waits for hardware access. It is thread-safe due to internal locking mechanisms.

`ca_fat_pwd`

Header: `caos/drivers/inc/fat_service.h:60`
Source: `caos/drivers/src/fat_service.c:281:294`
GUARD: `EN_FAT_SERVICE`

```
bool ca_fat_pwd(void)
```

Prints the current working directory path of the FAT filesystem to the standard output.

Ritorna:
Returns true if the current path was successfully retrieved and printed, false if the filesystem is not mounted or an internal FAT error occurred.

Note:

- This function is blocking because it acquires a mutex (ca_fat_lock) to ensure thread-safe access to the shared filesystem state. It must NOT be called from an ISR (Interrupt Service Routine) because it performs blocking operations and potentially long-running filesystem I/O. Before calling this, the FAT service must be initialized and mounted; otherwise, it will trigger a 'not_mounted' error. A common mistake is assuming this function returns the path as a string; it does not, it prints the path directly to the system's stdout.

Uso:

- This function is typically used for debugging or logging purposes to verify the current navigation state within the filesystem hierarchy. It is usually called after successful navigation commands like ca_fat_cd or after creating directories with ca_fat_mkdir to confirm the shell's current context. It is part of the standard filesystem management sequence: initialize the service, mount the drive, navigate using ca_fat_cd, and verify the location with ca_fat_pwd.

Esempio:

```
/* Ensure the FAT service is initialized and mounted before checking the path */
if (ca_fat_is_mounted()) {
    /* Attempt to print the current working directory */
    if (ca_fat_pwd()) {
        /* Successfully printed the path to stdout */
    } else {
        /* Handle failure: path could not be retrieved */
        ca_printf("Failed to get current directory\n");
    }
} else {
    ca_printf("Filesystem not mounted!\n");
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_fat_pwd function in my caOS application?
A: You should use ca_fat_pwd for debugging or logging purposes to verify your current navigation state within the FAT filesystem hierarchy. It is typically called after navigation commands like ca_fat_cd or directory creation via ca_fat_mkdir to confirm the shell's current context.

Code example conf:0.90 **Q:** How do I implement a simple check of the current directory using caOS types?
A: After ensuring the FAT service is initialized and mounted, you can call the function directly: bool success = ca_fat_pwd(); if (success) { /* Path printed to stdout */ }

Return conf:1.00 **Q:** What do the boolean return values of ca_fat_pwd indicate?
A: The function returns true if the current path was successfully retrieved and printed to stdout. It returns false if the filesystem is not mounted or if an internal FAT error occurred during the operation.

Error/Edge case conf:1.00 **Q:** What are the critical constraints and common mistakes when calling ca_fat_pwd?
A: A critical constraint is that it must NOT be called from an ISR because it performs blocking operations and acquires a mutex. A common mistake is assuming the function returns the path as a string; it does not, it prints the path directly to stdout. Additionally,

it will trigger a 'not_mounted' error if called before the FAT service is initialized and mounted.

---

ca_fat_read_enc

Header: caos/drivers/inc/fat_service.h:106
Source: caos/drivers/src/fat_service.c:554:618
GUARD: EN_FAT_SERVICE

```
bool ca_fat_read_enc(u32 key_idx, const char *name, ca_fat_read_cb_t cb,
    void *user, u32 *total_out)
```

Reads and decrypts a file from the filesystem using a specified cryptographic key.

Parametri:

Nome	Tipo	Descrizione
key_idx	u32	The index of the key stored in the secure keystore to be used for decryption.
name	const char *	The null-terminated path/filename of the encrypted file on the storage medium.
cb	ca_fat_read_cb_t	A callback function invoked for every decrypted data chunk. Return true to continue reading, false to abort.
user	void *	A pointer to user-defined data passed directly to the callback function.
total_out	u32 *	Optional pointer to store the total number of decrypted bytes read. Can be NULL.

Ritorna:
Returns true if the file was read and decrypted successfully. Returns false if the file could not be opened, the decryption failed (e.g., wrong key or corrupted data), or the callback returned false.

Note:

- This function is blocking and performs significant I/O and cryptographic operations; it must never be called from an ISR. It is thread-safe regarding the filesystem access because it internally uses ca_fat_lock and ca_fat_unlock. Before calling this, ensure the filesystem is mounted using ca_fat_is_mounted. A common mistake is passing a NULL callback when the user expects to process data, or failing to check the return value, which leads to ignoring decryption errors or file truncation issues.

Uso:

- This function is typically used to retrieve sensitive configuration or data files that are stored in an encrypted format on the SD card or flash. It is usually paired with ca_fat_service_init to prepare the filesystem and potentially ca_fat_write_enc if the application needs to update the encrypted file. The sequence is: verify mount status, call ca_fat_read_enc with a processing callback, and handle the boolean result to confirm data integrity.

Esempio:

```
/* Callback to process decrypted data chunks */
bool my_data_processor(const u8 *data, u32 len, void *user) {
    /* Process the decrypted 'data' of size 'len' */
    return true; /* Continue reading */
}

```

```c
void read_secure_config(void) {
    u32 total_bytes = 0;
    if (ca_fat_is_mounted()) {
        /* Attempt to read and decrypt 'config.enc' using key index 0 */
        if (ca_fat_read_enc(0, "config.enc", my_data_processor, NULL, &
    total_bytes)) {
            ca_printf("Successfully read %u bytes\n", (unsigned)total_bytes);
        } else {
            ca_printf("Failed to read or decrypt file\n");
        }
    }
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_fat_read_enc in my caOS application?
A: Use ca_fat_read_enc when you need to retrieve sensitive configuration or data files stored in an encrypted format on storage media like an SD card or flash. It is typically used after initializing the filesystem with ca_fat_service_init and verifying the mount status.

Code example conf:0.90 **Q:** Provide a minimal C example for reading an encrypted file using caOS types.
A: bool my_cb(u8 *data, u32 len, void *user) { return true; } void read_file(void) { u32 total; if (ca_fat_is_mounted()) { ca_fat_read_enc(0, "config.bin", my_cb, NULL, &total); } }

Parameters conf:1.00 **Q:** What is the purpose of the key_idx parameter in ca_fat_read_enc?
A: The key_idx parameter is a u32 value representing the index of the cryptographic key stored in the secure keystore. This key is used by the function to perform the decryption of the file data.

Return conf:1.00 **Q:** What do the return values of ca_fat_read_enc indicate?
A: The function returns true if the file was read and decrypted successfully. It returns false if the file could not be opened, if decryption failed due to a wrong key or corrupted data, or if the provided callback function returned false.

Error/Edge case conf:1.00 **Q:** What are some common mistakes developers make when using ca_fat_read_enc?
A: Common mistakes include passing a NULL callback when data processing is required, failing to check the boolean return value (which leads to ignoring decryption errors or file truncation), and calling the function from an ISR.

Behavior conf:1.00 **Q:** Is ca_fat_read_enc safe to call from an interrupt service routine (ISR)?
A: No, it must never be called from an ISR. The function is blocking, performs significant I/O and cryptographic operations, and uses internal locks (ca_fat_lock/ca_fat_unlock) to ensure thread safety regarding filesystem access.

`ca_fat_rm`

Header: `caos/drivers/inc/fat_service.h:66`
Source: `caos/drivers/src/fat_service.c:393:406`
GUARD: `EN_FAT_SERVICE`

```c
bool ca_fat_rm(const char *name)
```

Deletes a file from the mounted filesystem by its name.

Parametri:

Nome	Tipo	Descrizione
name	const char *	The path/name of the file to delete. Must be a valid string representing a file path within the mounted volume.

Ritorna:
Returns true if the file was successfully deleted, false otherwise (e.g., file not found, filesystem not mounted, or permission error).

Note:

- This function is blocking and thread-safe, as it internally uses a lock to ensure exclusive access to the filesystem. It must NOT be called from an ISR (Interrupt Service Routine) because it performs blocking operations and acquires a mutex. The filesystem must be initialized and mounted via ca_fat_mount before calling this function; otherwise, it will fail. A common mistake is passing a directory path to this function; use ca_fat_rmdir instead for directories.

Uso:

- This function is typically used during file management tasks, such as cleaning up temporary files or removing outdated logs. It is often paired with ca_fat_ls to verify the file exists before deletion, or used in a cleanup sequence after processing data. In a typical lifecycle, you would mount the drive with ca_fat_mount, perform operations, and then use ca_fat_rm to remove specific files before unmounting with ca_fat_umount.

Esempio:

```
/* Ensure the filesystem is mounted before attempting to remove a file */
if (ca_fat_is_mounted()) {
    const char *filename = "temp_data.bin";

    /* Attempt to remove the file */
    if (ca_fat_rm(filename)) {
        /* File removed successfully */
        ca_printf("File %s deleted successfully.\n", filename);
    } else {
        /* Handle error: file might not exist or is locked */
        ca_printf("Failed to delete %s. Check if it exists.\n", filename);
    }
} else {
    ca_printf("Filesystem not mounted, cannot remove file.\n");
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_fat_rm function in my caOS application?
A: You should use ca_fat_rm for file management tasks, such as removing temporary files or outdated logs. It is typically used after mounting the filesystem with ca_fat_mount and before unmounting with ca_fat_umount.

Code example conf:0.90 **Q:** How do I implement a simple file deletion in caOS?
A: Ensure the filesystem is mounted first, then call the function: cstr filename = "log.txt"; if (ca_fat_rm(filename)) { /* Success */ } else { /* Failure */ }

Parameters conf:1.00 **Q:** What is the expected format for the 'name' parameter in ca_fat_rm?
A: The 'name' parameter must be a valid string (cstr) representing the path or name of the file within the mounted volume. It must point to a file, not a directory.

Return conf:1.00 **Q:** What do the boolean return values of ca_fat_rm indicate?
A: A return value of true indicates the file was successfully deleted. A return value of false indicates failure, which can be caused by the file not being found, the filesystem not being mounted, or a permission error.

Error/Edge case conf:1.00 **Q:** Can I call ca_fat_rm inside an interrupt service routine (ISR)?
A: No, you must not call ca_fat_rm from an ISR. The function is blocking and acquires a mutex, which is incompatible with the execution constraints of an ISR.

`ca_fat_rmdir`

Header: `caos/drivers/inc/fat_service.h:64`
Source: `caos/drivers/src/fat_service.c:356:369`
GUARD: `EN_FAT_SERVICE`

```
bool ca_fat_rmdir(const char *dir)
```

Removes an empty directory from the FAT filesystem.

Parametri:

Nome	Tipo	Descrizione
`dir`	`const char *`	The path string of the directory to remove. Must be a valid path relative to the current working directory or an absolute path.

Ritorna:
Returns true if the directory was successfully removed, false otherwise (e.g., directory not empty, path does not exist, or filesystem not mounted).

Note:

- This function is blocking, meaning it will wait until the filesystem operation completes before returning. It is thread-safe because it internally uses a lock to prevent concurrent access to the FAT driver. It must NOT be called from an ISR (Interrupt Service Routine) because it performs blocking operations. A common beginner mistake is attempting to remove a directory that still contains files or subdirectories; the underlying FAT driver will reject this, and the function will return false.

Uso:

- Use this function when you need to clean up the filesystem structure during runtime. It is typically used after deleting all files inside a directory using ca_fat_rm. It is often paired with ca_fat_cd to navigate to the parent directory before attempting to remove the target folder. Ensure the filesystem is mounted via ca_fat_mount before calling this, otherwise, it will fail.

Esempio:

```
/* Ensure the directory is empty before removal */
if (ca_fat_cd("logs")) {
    /* Remove all files inside the directory first */
    ca_fat_rm("data1.txt");
    ca_fat_rm("data2.txt");

    /* Navigate back to root to remove the now-empty directory */
    ca_fat_cd("..");

    if (ca_fat_rmdir("logs")) {
        ca_printf("Directory removed successfully.\n");
    } else {
        ca_printf("Failed to remove directory: it might not be empty.\n");
    }
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_fat_rmdir in my caOS application?
A: Use ca_fat_rmdir when you need to clean up the filesystem structure during runtime. It is typically used after deleting all files inside a directory using ca_fat_rm, and is often paired with ca_fat_cd to navigate to the parent directory before removing the target folder.

Code example conf:0.90 **Q:** Provide a minimal example of how to remove a directory using caOS types.
A: cstr path = "/logs/old_data"; if (ca_fat_rmdir(path)) { /* Success */ } else { /* Failure */ }

Parameters conf:1.00 **Q:** What are the requirements for the 'dir' parameter in ca_fat_rmdir?
A: The 'dir' parameter must be a valid path string, which can be either relative to the current working directory or an absolute path.

Return conf:1.00 **Q:** What does a return value of false from ca_fat_rmdir indicate?
A: A return value of false indicates the operation failed. Common causes include the directory not being empty, the specified path not existing, or the filesystem not being mounted.

Error/Edge case conf:1.00 **Q:** What is a common mistake when using ca_fat_rmdir and why should I avoid calling it from an ISR?
A: A common mistake is attempting to remove a directory that still contains files or subdirectories, which causes the driver to reject the request. You must not call it from an ISR because it is a blocking function that waits for filesystem operations to complete.

`ca_fat_service_init`

Header: `caos/drivers/inc/fat_service.h:41`
Source: `caos/drivers/src/fat_service.c:734:743`
GUARD: `EN_FAT_SERVICE`

```
1 void ca_fat_service_init(void)
```

Initializes the FAT filesystem service, clears internal buffers, and registers the CLI command node.

Ritorna:
This function returns void and does not provide a status code.

Note:

- This function is not thread-safe and must be called during the system initialization phase, typically within ca_os_configure(). It is not callable from an ISR because it performs memory operations and registers nodes in the CLI chain. A common mistake is calling this function before the underlying SDIO driver or the CLI system has been initialized; if the CLI is not ready, the registration of the 'fat' command node will fail silently or cause a system fault. Always ensure that the hardware abstraction layer for the storage media is fully operational before invoking this service.

Uso:

- This function is the mandatory entry point for using any FAT-related features in caOS. It is typically paired with ca_fat_service_set_event_cb() to define custom event handling, followed by CLI registration functions if command-line interaction is required. In the application lifecycle, it should be called after ca_cli_init() but before any attempt to mount or access the filesystem via functions like ca_fat_mount() or ca_fat_ls().

Esempio:

```c
void system_init(void) {
    /* Initialize the CLI system first as the FAT node depends on it */
    ca_cli_init(2048);
    ca_cli_set_sender(STDOUT);

    /* Initialize the FAT service to register the 'fat' command */
    ca_fat_service_init();

    /* Optional: set a callback for filesystem events */
    ca_fat_service_set_event_cb(my_fat_event_handler);

    /* Now safe to register application-specific CLI nodes */
    ca_cli_register(&my_app_cli_node);
    ca_cli_create_thread();
}
```

Q&A — fine-tuning review *2/2 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_fat_service_init() in my caOS application?
A: You must call ca_fat_service_init() during the system initialization phase, typically within ca_os_configure(). It is mandatory to call this function after ca_cli_init() but before attempting to mount or access the filesystem with functions like ca_fat_mount() or ca_fat_ls(). Ensure the underlying SDIO driver and CLI system are fully operational before invocation, as it is not thread-safe and cannot be called from an ISR.

Code example conf:0.90 **Q:** How do I initialize the FAT service within the caOS configuration function?
A: You should call the initialization function within your configuration routine as follows: void ca_os_configure(void) { ca_cli_init(); ca_fat_service_init(); }

`ca_fat_service_set_event_cb`

Header: `caos/drivers/inc/fat_service.h:42`
Source: `caos/drivers/src/fat_service.c:729:729`
GUARD: `EN_FAT_SERVICE`

```c
void ca_fat_service_set_event_cb(ca_fat_event_cb_t cb)
```

Registers a callback function to handle FAT filesystem events asynchronously.

Parametri:

Nome	Tipo	Descrizione
`cb`	`ca_fat_event_cb_t`	A function pointer to the callback routine. Pass NULL to disable the event notification.

Ritorna:
void

Note:

- This function is non-blocking and thread-safe as it simply updates a static internal pointer. It is not intended to be called from an ISR, although it is technically safe to do so; however, the callback function itself must be written with extreme caution if triggered from an ISR context. The most common mistake is passing a function pointer that performs heavy processing or blocking operations, which can stall the FAT service or the entire system. Ensure that the FAT service has been initialized via ca_fat_service_init before setting the callback, otherwise, the callback may never be triggered.

Uso:

- This function is used to hook into the FAT filesystem's event system, allowing your application to react to file operations or system events (like media insertion or removal). It is typically called during the system initialization phase, immediately after ca_fat_service_init and before starting the CLI or other tasks that might interact with the filesystem. By registering a callback, you decouple the filesystem driver from your application logic, enabling cleaner event-driven architectures.

Esempio:

```c
/* Define the callback function to handle FAT events */
void fat_event(int event_id) {
    /* Handle specific filesystem events here */
    ca_printf("FAT Event received: %d\n", event_id);
}

void app_init(void) {
    /* Initialize the FAT service first */
    ca_fat_service_init();

    /* Register the event callback to monitor filesystem activity */
    ca_fat_service_set_event_cb(fat_event);

    /* Proceed with CLI initialization */
    ca_cli_init();
    ca_cli_create_thread();
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_fat_service_set_event_cb in my caOS application?
A: It is typically called during the system initialization phase, immediately after ca_fat_service_init and before starting the CLI or other tasks that interact with the filesystem. This allows you to decouple the filesystem driver from your application logic for an event-driven architecture.

Code example conf:0.90 **Q:** How do I register a callback function using ca_fat_service_set_event_cb?
A: You define a function matching the ca_fat_event_cb_t signature and pass it to the service. Example: void my_handler(u32 event) { /* handle event */ } ... ca_fat_service_set_event_cb(

Parameters conf:1.00 **Q:** What is the purpose of the cb parameter and what happens if I pass NULL?
A: The cb parameter is a function pointer to the callback routine that handles FAT filesystem events. Passing NULL to this parameter disables the event notification mechanism.

`ca_fat_strerror`

Header: `caos/drivers/inc/fat_service.h:49`
Source: `caos/drivers/src/fat_service.c:86:111`
GUARD: `EN_FAT_SERVICE`

```c
const char *ca_fat_strerror(i32 fr)
```

Converts a numeric FatFS error code into a human-readable string for logging and debugging.

Parametri:

Nome	Tipo	Descrizione
`fr`	`i32`	The numeric error code returned by FatFS operations (e.g., FR_OK, FR_NO_FILE). Passing an undefined integer will return 'unknown'.

Ritorna:
A pointer to a constant null-terminated string describing the error. It never returns NULL.

Note:

- This function is a simple lookup table and is completely non-blocking, thread-safe, and safe to call from an ISR. It does not perform any hardware I/O or filesystem access, so it has no preconditions other than the FatFS service being initialized. The most common mistake is assuming this function can 'fix' an error; it is strictly for diagnostic reporting. Do not attempt to modify the returned string, as it points to static memory.

Uso:

- Use this function whenever a FatFS operation (like ca_fat_mount, ca_fat_read_enc, or ca_fat_write_enc) returns a non-zero value. It is typically paired with ca_printf_n or ca_stdout_printf_n to provide meaningful feedback in logs. By converting the raw integer error code into text, you make it significantly easier to diagnose issues like 'no such file' versus 'disk I/O error' during field debugging.

Esempio:

```
i32 fr;

// Attempt to mount the filesystem
fr = ca_fat_mount();

if (fr != FR_OK) {
    // Log the error using the string converter
    ca_printf_n("[FAT] Mount failed: %s (code %d)", ca_fat_strerror(fr), (int)fr
    );
} else {
    // Proceed with file operations
    ca_printf_n("[FAT] Filesystem mounted successfully.");
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_fat_strerror in my caOS application?
A: You should call this function whenever a FatFS operation, such as ca_fat_mount, ca_fat_read_enc, or ca_fat_write_enc, returns a non-zero value. It is intended for diagnostic reporting to convert raw integer error codes into human-readable strings for logs.

Code example conf:1.00 **Q:** How do I implement a simple error check using ca_fat_strerror?
A: i32 res = ca_fat_mount(); if (res != 0) { ca_printf_n("Mount failed: %s", ca_fat_strerror(res)); }

Parameters conf:1.00 **Q:** What happens if I pass an undefined integer to the fr parameter?
A: The function will return the string 'unknown' if the provided integer does not correspond to a defined FatFS error code.

Return conf:1.00 **Q:** Is it possible for ca_fat_strerror to return a NULL pointer?
A: No, the function is guaranteed to return a pointer to a constant null-terminated string and will never return NULL.

Error/Edge case conf:1.00 **Q:** What is a common mistake developers make when using this function?
A: The most common mistake is assuming that calling this function can 'fix' an error. It is strictly for diagnostic reporting. Additionally, you must not attempt to modify the returned string, as it points to static memory.

Behavior conf:1.00 **Q:** Can I safely call ca_fat_strerror from an ISR?
A: Yes, the function is completely non-blocking, thread-safe, and safe to call from an ISR because it is a simple lookup table that performs no hardware I/O or filesystem access.

ca_fat_touch

Header: caos/drivers/inc/fat_service.h:65
Source: caos/drivers/src/fat_service.c:371:391
GUARD: EN_FAT_SERVICE

```
bool ca_fat_touch(const char *name)
```

Creates a new empty file on the filesystem if it does not already exist.

Parametri:

Nome	Tipo	Descrizione
name	const char *	The path and filename to create. Must be a valid string representing a path within the mounted filesystem.

Ritorna:
Returns true if the file was successfully created, false if the file already exists, the filesystem is not mounted, or an I/O error occurred.

Note:

- This function is blocking, meaning it will pause the execution of the calling task until the filesystem operation completes. It is not safe to call from an ISR (Interrupt Service Routine) because it uses internal mutexes (ca_fat_lock) which can cause deadlocks or system crashes if called from an interrupt context. Ensure the filesystem is initialized and mounted via ca_fat_mount before calling this. A common beginner mistake is assuming this function will overwrite an existing file; it will not, as it specifically uses the FA_CREATE_NEW flag, returning false if the file is already present.

Uso:

- Use this function when you need to initialize a new log file or configuration file on the SD card or flash storage. It is typically used in a sequence where you first verify the filesystem state with ca_fat_is_mounted, then create the file with ca_fat_touch, and subsequently open it for writing using other filesystem functions. It is often paired with ca_fat_ls to verify the file creation or ca_fat_rm if you need to clean up the file later.

Esempio:

```
if (ca_fat_is_mounted()) {
    const char *filename = "log.txt";
    // Attempt to create the file; if it exists, we proceed to write to it
    if (ca_fat_touch(filename)) {
        ca_printf("File created successfully.\n");
    } else {
        ca_printf("File already exists or error occurred.\n");
    }
    // Proceed to perform other operations like listing files
    ca_fat_ls("");
} else {
    ca_printf("Filesystem not mounted!\n");
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_fat_touch function in my caOS application?

A: You should use ca_fat_touch when you need to initialize a new log or configuration file on your storage media. It is typically used after verifying the filesystem is mounted, often followed by opening the file for writing or using ca_fat_ls to confirm its creation.

Code example conf:0.90 **Q:** Provide a minimal C example showing how to use ca_fat_touch.
A: cstr filename = "/logs/data.txt"; if (ca_fat_is_mounted()) { if (ca_fat_touch(filename)) { /* File created successfully */ } }

Parameters conf:1.00 **Q:** What are the requirements for the 'name' parameter in ca_fat_touch?
A: The 'name' parameter must be a valid string representing the path and filename within the mounted filesystem. It must point to a valid location where the file is to be created.

Return conf:1.00 **Q:** What do the boolean return values of ca_fat_touch indicate?
A: A return value of true indicates the file was successfully created. A return value of false indicates that the file already exists, the filesystem is not mounted, or an I/O error occurred during the operation.

Error/Edge case conf:1.00 **Q:** Why does ca_fat_touch return false even when the path seems correct?
A: A common mistake is assuming the function overwrites existing files; it does not. It uses the FA_CREATE_NEW flag, so it will return false if the file already exists. Additionally, it will fail if the filesystem is not mounted or if an I/O error occurs.

Behavior conf:1.00 **Q:** Is ca_fat_touch safe to call from an Interrupt Service Routine (ISR)?
A: No, it is not safe to call from an ISR. The function is blocking and uses internal mutexes (ca_fat_lock), which can lead to deadlocks or system crashes if invoked from an interrupt context.

`ca_fat_trylock`

Header: `caos/drivers/inc/fat_service.h:46`
Source: `caos/drivers/src/fat_service.c:73:73`
GUARD: `EN_FAT_SERVICE`

```
1 bool ca_fat_trylock(void)
```

Attempts to acquire the FAT filesystem lock without blocking the calling task.

Ritorna:
Returns true if the lock was successfully acquired, false if the lock is already held by another task.

Note:

- This function is non-blocking, meaning it returns immediately regardless of whether the lock is available. It is thread-safe and intended for scenarios where you prefer to skip an operation rather than waiting for the filesystem to become free. It must not be called from an ISR, as mutex operations are designed for task-level synchronization. A common mistake is failing to call ca_fat_unlock() after a successful trylock, which will permanently lock the filesystem and cause all other tasks to hang.

Uso:

- Use this function in non-critical background tasks, such as logging or telemetry, where you want to avoid stalling the system if the filesystem is busy with a high-priority operation. It is typically used in a pattern where you check the return value: if true, you perform the file operation (e.g., ca_fat_write_enc) and then immediately release the lock; if false, you simply drop the data or retry later. This prevents low-priority tasks from blocking the entire system when the FAT service is under heavy load.

Esempio:

```c
/* Attempt to log data to the filesystem without blocking the main loop */
if (ca_fat_trylock()) {
    /* Lock acquired, safe to perform file operations */
    UINT bytes_written = 0;
    /* Perform write operation using FatFS handle */
    /* f_write(&g_log_file, data, size, &bytes_written); */

    ca_fat_unlock();
} else {
    /* Lock was busy, skip logging to keep the system responsive */
    ca_printf("FAT busy, skipping log entry\n");
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_fat_trylock instead of a standard blocking lock?
A: You should use ca_fat_trylock in non-critical background tasks, such as logging or telemetry, where you want to avoid stalling the system if the filesystem is busy with a high-priority operation. It is intended for scenarios where you prefer to skip an operation rather than waiting for the filesystem to become free.

Code example conf:1.00 **Q:** How do I implement a non-blocking write operation using ca_fat_trylock?
A: You check the return value of the trylock before proceeding. If true, perform the operation and release the lock. Example: if (ca_fat_trylock()) { /* perform file operation */ ca_fat_unlock(); }

Return conf:1.00 **Q:** What do the return values of ca_fat_trylock indicate, and are there any risks associated with them?
A: The function returns true if the lock was successfully acquired, and false if the lock is already held by another task. A critical risk is failing to call ca_fat_unlock() after a successful acquisition, which will permanently lock the filesystem and cause all other tasks to hang.

`ca_fat_umount`

Header: `caos/drivers/inc/fat_service.h:58`
Source: `caos/drivers/src/fat_service.c:238:251`
GUARD: `EN_FAT_SERVICE`

```c
bool ca_fat_umount(void)
```

Safely unmounts the currently mounted FAT filesystem volume.

Ritorna:
Returns true if the filesystem was successfully unmounted or was already unmounted. Returns false if an internal error occurs during the unmount process.

Note:

- This function is thread-safe and uses internal locking to ensure that filesystem operations are not interrupted during the teardown process. It is a blocking function, meaning the calling task will wait until the internal cleanup (closing files, flushing buffers) is complete. It must NOT be called from an ISR (Interrupt Service Routine) because it performs blocking operations. A common beginner mistake is to attempt to call this function while other tasks are still actively reading or writing to the filesystem, which can lead to data corruption or incomplete file writes; ensure all application-level file handles are closed before calling this.

Uso:

- This function is typically used during system shutdown or when a storage medium (like an SD card) needs to be safely removed. It is the final step in the filesystem lifecycle, following the sequence: ca_fat_service_init -> ca_fat_mount -> [file operations] -> ca_fat_umount. It is often paired with ca_fat_is_mounted to check the status before attempting the unmount, ensuring the system state is consistent.

Esempio:

```c
/* Example: Safely unmounting the FAT filesystem before system shutdown */
if (ca_fat_is_mounted()) {
    /* Ensure all application tasks have finished their file operations */
    /* ... wait for worker tasks to signal completion ... */

    if (ca_fat_umount()) {
        ca_stdout_printf_n("Filesystem unmounted successfully.");
    } else {
        ca_stdout_printf_n("Error: Failed to unmount filesystem.");
    }
} else {
    ca_stdout_printf_n("Filesystem is not mounted, skipping unmount.");
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_fat_umount in my caOS application?
A: You should call ca_fat_umount during system shutdown or when you need to safely remove a storage medium like an SD card. It serves as the final step in the filesystem lifecycle, following the sequence of initialization, mounting, and performing file operations.

Code example conf:0.90 **Q:** How do I safely unmount the filesystem in a caOS project?
A: Ensure all file handles are closed first, then call the function: bool status = ca_fat_umount(); if (!status) { /* handle error */ }

Return conf:1.00 **Q:** What do the return values of ca_fat_umount indicate?
A: The function returns true if the filesystem was successfully unmounted or was already unmounted. It returns false if an internal error occurs during the unmount process, such as issues during the cleanup of buffers or closing files.

Error/Edge case conf:1.00 **Q:** What are the common mistakes or restrictions when using ca_fat_umount?
A: A common mistake is calling it while other tasks are still actively reading or writing to the filesystem, which can cause data corruption. Additionally, it must never be called from an ISR because it is a blocking function, and it requires all application-level file handles to be closed prior to invocation.

`ca_fat_unlock`

Header: `caos/drivers/inc/fat_service.h:45`
Source: `caos/drivers/src/fat_service.c:72:72`
GUARD: `EN_FAT_SERVICE`

```c
void ca_fat_unlock(void)
```

Releases the internal mutex lock protecting the FAT filesystem service.

Ritorna:
void

Note:

- This function is used to release a recursive mutex that prevents multiple tasks from accessing the FAT filesystem simultaneously. It is not thread-safe to call this from an ISR, as mutexes are intended for task-level synchronization. A common beginner mistake is to forget to call this after a 'ca_fat_lock()', which results in a permanent deadlock where no other task can ever access the filesystem again. Always ensure that every lock operation has a corresponding unlock operation, even in error-handling paths.

Uso:

- This function is used in a 'lock-process-unlock' pattern to ensure data integrity during filesystem operations. You typically call 'ca_fat_lock()' before performing a sequence of operations like 'f_open', 'f_read', or 'f_write', and then call 'ca_fat_unlock()' immediately after the work is done. It is essential to use this when multiple tasks might attempt to read or write to the SD card or flash memory simultaneously, as it prevents file corruption and race conditions.

Esempio:

```c
/* Example: Safely reading a file from the FAT filesystem */
void read_config_file(void) {
    /* Acquire the lock before accessing the filesystem */
    ca_fat_lock();

    /* Perform filesystem operations */
    /* Note: f_open, f_read, f_close are standard FatFs functions */
    if (f_open(&f, "config.txt", FA_READ) == FR_OK) {
        f_read(&f, buffer, sizeof(buffer), &br);
        f_close(&f);
    } else {
        ca_printf("Failed to open file\n");
    }

    /* Always release the lock, even if the file operation failed */
    ca_fat_unlock();
}
```

Q&A — fine-tuning review *2/2 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_fat_unlock() and why is it critical for my caOS application?
A: You should call ca_fat_unlock() immediately after completing a sequence of filesystem operations like f_open, f_read, or f_write that were protected by a prior ca_fat_lock(). It is critical because it releases the recursive mutex protecting the FAT filesystem; failing to call it results in a permanent deadlock where no other task can access the filesystem, and it is essential for preventing file corruption and race conditions when multiple tasks access the storage media.

Code example conf:0.95 **Q:** How do I implement a thread-safe file write operation using caOS types?
A: You must wrap your filesystem operations in the lock-process-unlock pattern. Example:
ca_fat_lock(); /* Perform filesystem operations */ ca_fat_unlock();

`ca_fat_work_buf`

Header: `caos/drivers/inc/fat_service.h:53`
Source: `caos/drivers/src/fat_service.c:77:81`
GUARD: `EN_FAT_SERVICE`

```
u8 *ca_fat_work_buf(u32 *size)
```

Retrieves the pointer to the internal shared working buffer used by the FAT filesystem service.

Parametri:

Nome	Tipo	Descrizione
size	u32 *	Optional pointer to a variable where the function will store the size of the buffer in bytes. Pass NULL if the size is not needed.

Ritorna:
A pointer (u8 *) to the internal static working buffer. This pointer is never NULL as it points to a statically allocated memory area within the FAT service.

Note:

- This function is not thread-safe on its own; it provides access to a shared global resource. You MUST call ca_fat_lock() before accessing the buffer and ca_fat_unlock() immediately after you are finished to prevent data corruption from other tasks. This function is non-blocking and can be called from an ISR, but since the buffer is shared, using it in an ISR is highly discouraged as it would require locking mechanisms that are not ISR-safe. The most common mistake is accessing the buffer without holding the FAT lock, which leads to race conditions where multiple tasks overwrite the same memory area.

Uso:

- This function is used when you need a temporary scratchpad memory area to perform raw data operations, such as preparing a block of data to be written to the SD card or reading a sector. It is typically used in a sequence: first, call ca_fat_lock() to gain exclusive access, then call ca_fat_work_buf() to get the pointer, perform your data manipulation (e.g., using ca_memcpy), and finally call ca_fat_unlock() to release the resource for other tasks.

Esempio:

```c
/* Example: Safely preparing data in the FAT working buffer */
#include "caos.h"

void write_data_to_fat(u8 *data, u32 len) {
    u32 buf_size = 0;

    /* 1. Acquire exclusive access to the FAT service */
    ca_fat_lock();

    /* 2. Get the pointer to the shared working buffer */
    u8 *work_ptr = ca_fat_work_buf(&buf_size);

    /* 3. Ensure we do not overflow the buffer */
    if (len <= buf_size) {
        /* Copy data into the buffer using caOS memory utility */
        ca_memcpy(work_ptr, data, len);

        /* Perform the write operation using the buffer content */
        /* ca_fat_write_enc(work_ptr, len); // Hypothetical write call */
    }

    /* 4. Always release the lock to allow other tasks to use the service */
    ca_fat_unlock();
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_fat_work_buf and what is the correct sequence of operations?
A: Use this function when you need a temporary scratchpad memory area for raw data operations like preparing SD card blocks. You must call ca_fat_lock() first to gain exclusive access, call ca_fat_work_buf() to get the pointer, perform your operations, and then call ca_fat_unlock() to release the resource.

Code example conf:1.00 **Q:** Provide a minimal C example showing how to safely access the FAT working buffer.
A: u32 size; u8 *buf; ca_fat_lock(); buf = ca_fat_work_buf(&size); /* Perform operations using buf */ ca_fat_unlock();

Parameters conf:1.00 **Q:** What is the purpose of the size parameter in ca_fat_work_buf?
A: The size parameter is an optional pointer to a u32 variable where the function stores the total size of the working buffer in bytes. You can pass NULL if you do not need to know the buffer size.

Return conf:1.00 **Q:** Can ca_fat_work_buf return NULL if the buffer is unavailable?
A: No, the function never returns NULL. It returns a pointer to a statically allocated memory area within the FAT service. There are no failure conditions that result in a NULL pointer return.

`ca_fat_write_enc`

Header: `caos/drivers/inc/fat_service.h:101`
Source: `caos/drivers/src/fat_service.c:501:550`
GUARD: `EN_FAT_SERVICE`

```
bool ca_fat_write_enc(u32 key_idx, const char *name, const u8 *buf, u32 len)
```

Writes data to a file on the FAT filesystem after encrypting it using a specified AES key.

Parametri:

Nome	Tipo	Descrizione
`key_idx`	`u32`	The index of the AES key to use for encryption. Must correspond to a valid key stored in the system keystore.
`name`	`const char *`	The null-terminated string representing the file path/name on the FAT filesystem.
`buf`	`const u8 *`	Pointer to the source data buffer to be encrypted and written.
`len`	`u32`	The length of the data in bytes. Must be greater than 0.

Ritorna:
Returns true if the file was successfully created, encrypted, and written to the disk. Returns false if the filesystem is not mounted, the key is invalid, the disk is full, or any I/O error occurs.

Note:

- This function is blocking and performs multiple disk I/O operations, so it should never be called from an ISR. It is thread-safe because it internally uses ca_fat_lock() to ensure exclusive access to the filesystem during the write process. A common mistake is calling this function before the filesystem is initialized or mounted, which will result in an immediate failure. Ensure the buffer provided is valid for the entire duration of the call, as the function processes data in chunks.

Uso:

- This function is typically used in the application layer to securely store sensitive configuration or data logs on an SD card or flash storage. It is usually preceded by ca_fat_mount() to ensure the storage is ready. After the write operation, the system automatically closes the file handle, so no manual close is required by the user. It is often paired with ca_fat_read_enc() to retrieve and decrypt the data later.

Esempio:

```
/* Ensure the filesystem is ready before writing */
if (ca_fat_is_mounted()) {
    const char *filename = "secure_log.bin";
    u8 data[] = {0xDE, 0xAD, 0xBE, 0xEF};

    /* Write 4 bytes using key index 0 */
    if (ca_fat_write_enc(0U, filename, data, 4U)) {
        ca_stdout_printf_n("File %s written successfully", filename);
    } else {
        /* Handle error: check if disk is full or key is invalid */
        ca_stdout_put_n("Failed to write encrypted file");
    }
} else {
    ca_stdout_put_n("Filesystem not mounted");
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_fat_write_enc in my application?
A: Use this function when you need to securely store sensitive configuration data or logs on storage media like an SD card. It should be called after the filesystem has been successfully mounted using ca_fat_mount().

Code example conf:1.00 **Q:** Provide a minimal example of how to call ca_fat_write_enc.
A: u8 data[] = {0x01, 0x02, 0x03}; u32 key = 0; bool success = ca_fat_write_enc(key, "config.bin", data, 3);

Parameters conf:1.00 **Q:** What does the key_idx parameter represent?
A: The key_idx is an index referring to a specific AES key stored in the system keystore. It must correspond to a valid, pre-stored key for the encryption process to succeed.

Return conf:1.00 **Q:** What do the boolean return values of ca_fat_write_enc indicate?
A: A return of true indicates the file was successfully created, encrypted, and written. A return of false indicates a failure, which can be caused by the filesystem not being mounted, an invalid key index, a full disk, or an I/O error.

Error/Edge case conf:1.00 **Q:** What is a common mistake when calling ca_fat_write_enc?
A: A common mistake is calling the function before the filesystem is initialized or mounted, which will cause the function to return false immediately.

Behavior conf:1.00 **Q:** Is ca_fat_write_enc safe to call from an interrupt service routine (ISR)?
A: No, it is not ISR-safe. The function is blocking and performs multiple disk I/O operations, so it must never be called from an ISR. It is, however, thread-safe due to internal use of ca_fat_lock().

`ca_flash_erase_range`

Header: `caos/drivers/inc/flash.h:86`
Source: `caos/drivers/src/flash.c:132:171`
GUARD: `EN_FLASH`

```
bool ca_flash_erase_range(u32 addr, u32 len)
```

Erases a specified range of flash memory, handling sector boundaries and partial sector modifications automatically.

Parametri:

Nome	Tipo	Descrizione
`addr`	`u32`	The starting memory address in the flash peripheral to begin erasing.
`len`	`u32`	The number of bytes to erase. Must be greater than zero.

Ritorna:
Returns true if the entire requested range was successfully erased (or modified). Returns false if the operation failed, memory allocation failed, or if the input length is zero.

Note:

- This function is a blocking operation, meaning the CPU will wait until the hardware flash controller completes the erase cycles before returning. It is NOT thread-safe and must not be called from an ISR, as it performs dynamic memory allocation (ca_memory_allocate) which is forbidden in interrupt contexts. A common mistake is passing an address or length that exceeds the physical flash boundaries, which may lead to undefined behavior or hardware faults. Always ensure the flash is not being read by other tasks during this operation, as erasing causes the memory to return 0xFF and may crash code executing from the same flash bank.

Uso:

- Use this function when you need to clear configuration data, logs, or firmware update slots stored in internal flash. It is typically used in a sequence: first, ensure the system is in a safe state, then call ca_flash_erase_range to clear the target area, and finally use ca_flash_hw_write to store new data. It is designed to handle partial sector erasures by performing a read-modify-write cycle, which is essential when your data structure is smaller than the hardware's minimum erase sector size.

Esempio:

```
u32 target_addr = 0x0800F000; // Start of a flash sector
u32 data_len = 1024; // 1KB to erase

// Ensure we are not in an interrupt context before calling
if (ca_flash_erase_range(target_addr, data_len)) {
    // Success: flash is now 0xFF in the specified range
    // Proceed to write new data
    // ca_flash_hw_write(target_addr, my_data, data_len);
} else {
    // Handle error: log failure or retry
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_flash_erase_range() in a caOS project?
A: You should use ca_flash_erase_range() when you need to erase a specific range of flash memory, typically before writing new data to that region. This function is useful for updating firmware, storing configuration data, or managing persistent settings in flash memory.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_flash_erase_range()?
A: u32 flash_addr = 0x08000000; u32 erase_len = 0x1000; bool result = ca_flash_erase_range(flash_ad erase_len); if (result) { /* Erase successful */ } else { /* Erase failed */ }

Parameters conf:0.90 **Q:** What are the valid values for the 'addr' and 'len' parameters in ca_flash_erase_range()?

A: The 'addr' parameter must be a valid flash memory address aligned to a sector boundary. The 'len' parameter must be greater than zero; otherwise, the function returns false immediately. Both parameters must be within the flash memory range of the STM32 device.

Return conf:0.95 **Q:** What does it mean when ca_flash_erase_range() returns false?
A: A return value of false indicates that the erase operation failed. This can happen if 'len' is zero, the flash device is not unlocked, or an error occurs during the erase or write process.

Error/Edge case conf:0.85 **Q:** What are the most common mistakes when calling ca_flash_erase_range()?
A: Common mistakes include calling the function with a zero length ('len' = 0), which causes an immediate failure, or attempting to erase memory that is not properly unlocked. Also, calling this function from an ISR is not recommended due to its blocking behavior.

Behavior conf:0.90 **Q:** Is ca_flash_erase_range() safe to call from an ISR or in a multi-threaded environment?
A: No, ca_flash_erase_range() is not ISR-safe because it is a blocking function that may prevent system scheduling. It is also not thread-safe, as concurrent calls from multiple threads could cause conflicts. The function handles locking internally but does not protect against concurrent access.

`ca_flash_erase_sector`

Header: `caos/drivers/inc/flash.h:41`
Source: `caos/drivers/src/flash.c:40:43`
GUARD: `EN_FLASH`

```
1 bool ca_flash_erase_sector(u32 addr)
```

Erases a specific flash memory sector starting at the provided address.

Parametri:

Nome	Tipo	Descrizione
`addr`	`u32`	The starting memory address of the flash sector to be erased. Must be a valid sector boundary address defined in the MCU datasheet; providing an invalid or non-aligned address will result in a hardware error.

Ritorna:
Returns true if the sector was successfully erased, or false if the operation failed due to hardware protection, invalid address, or timeout.

Note:

- This function is a blocking operation, meaning the CPU will wait until the flash hardware completes the erase cycle before returning control to your code. It is not thread-safe by default, so you must ensure that no other task attempts to access the flash memory while an erase is in progress. It is generally not safe to call this from an Interrupt Service Routine (ISR) because the long execution time of an erase cycle will block all other interrupts, potentially causing system instability. A common beginner mistake is failing to unlock the flash peripheral before calling this function or attempting to erase a sector that contains critical code currently being executed.

Uso:

- This function is typically used during firmware updates, configuration storage, or data logging where old data must be cleared before writing new information. It is almost

always paired with a preceding 'ca_flash_unlock' call (if required by the hardware) and followed by a 'ca_flash_write' operation to store new data. After the write operation is complete, it is standard practice to call 'ca_flash_lock' to prevent accidental corruption of the flash memory during normal system operation.

Esempio:

```c
/* Ensure the flash is unlocked before attempting an erase */
ca_flash_unlock();

/* Erase the sector at the defined base address */
u32 sector_addr = 0x0800C000;
if (ca_flash_erase_sector(sector_addr)) {
    /* Proceed to write data if erase was successful */
    ca_flash_write(sector_addr, data_buffer, buffer_size);
} else {
    /* Handle error: log failure or trigger recovery */
}

/* Always lock the flash after operations to protect memory */
ca_flash_lock();
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_flash_erase_sector and why is it important to lock the flash first?
A: You should call ca_flash_erase_sector when you need to erase a sector of the STM32 flash memory, typically before writing new data to that sector. It is important to lock the flash first using ca_flash_lock() because the function is not thread-safe and must be called after obtaining the flash lock to avoid hardware errors or undefined behavior.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_flash_erase_sector with caOS types?
A: u32 sector_addr = 0x08000000; // Example sector start address bool success = ca_flash_lock(); if (success) { bool result = ca_flash_erase_sector(sector_addr); ca_flash_unlock(); // Handle result }

Parameters conf:0.90 **Q:** What is the significance of the addr parameter in ca_flash_erase_sector, and what are valid vs invalid values?
A: The addr parameter specifies the start address of the flash sector to be erased. It must be aligned to the beginning of a flash sector (i.e., a multiple of the sector size). Invalid values include addresses that are not sector-aligned or addresses outside the valid flash memory range, which will cause the function to return false.

Return conf:0.95 **Q:** What do the return values of ca_flash_erase_sector indicate, and what can cause it to return false?
A: The function returns true if the sector erase operation completed successfully. It returns false if the operation failed, which can happen due to invalid address alignment, lack of flash lock, or hardware errors during the erase process.

`ca_flash_erase_sector_irq`

Header: `caos/drivers/inc/flash.h:57`
Source: `caos/drivers/src/flash.c:63:80`
GUARD: `EN_FLASH`

```c
bool ca_flash_erase_sector_irq(u32 addr)
```

Erases a specific flash memory sector using an interrupt-driven approach to avoid blocking the CPU.

Parametri:

Nome	Tipo	Descrizione
addr	u32	The starting memory address of the flash sector to be erased. Must be a valid sector boundary address defined in the STM32 datasheet.

Ritorna:
Returns true if the sector was erased successfully. Returns false if the flash driver is not initialized, the hardware could not be unlocked, or if an error occurred during the erase operation.

Note:

- This function is blocking because it uses ca_sem_wait to pause the calling thread until the hardware interrupt signals that the erase is complete. It is NOT safe to call this function from an ISR because it waits on a semaphore, which would cause a system deadlock or kernel panic. Before calling this, you must ensure the flash driver has been initialized via the internal setup routine. A common beginner mistake is calling this function repeatedly in a tight loop without checking the return value, which can lead to system instability if the flash hardware is busy or locked.

Uso:

- This function is typically used when performing firmware updates or saving configuration data to non-volatile memory. It is part of a sequence: first, the driver is initialized, then ca_flash_erase_sector_irq is called to clear the memory, followed by a write operation. It is designed to be used in task-level code where the developer wants to offload the long-running erase process to the hardware interrupt controller, allowing the RTOS to switch to other tasks while the flash hardware works in the background.

Esempio:

```
/* Ensure the flash driver is ready before attempting an erase */
u32 sector_addr = 0x0800C000; // Example sector address

/* Initialize flash driver before use */
ca_flash_init();

if (ca_flash_erase_sector_irq(sector_addr)) {
    /* Erase successful, proceed to write data */
    /* ca_flash_write_irq(sector_addr, data_buffer, size); */
} else {
    /* Handle error: log failure or retry */
    /* log_error_to_system(); */
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_flash_erase_sector_irq() function in my caOS project?
A: You should call ca_flash_erase_sector_irq() when you need to erase a flash sector in interrupt mode. This function is useful when you want to perform flash erase operations without blocking the main execution flow, as it uses interrupt-driven operations. It's typically used when updating firmware or storing persistent data in flash memory.

Code example conf:0.90 **Q:** Can you show me a minimal example of how to use ca_flash_erase_sector_irq() with caOS types?
A: u32 sector_addr = 0x08000000; // First flash sector bool result = ca_flash_erase_sector_irq(sector_ if (result) { // Erase successful } else { // Erase failed }

Error/Edge case conf:0.85 **Q:** What are the common error conditions or edge cases when calling ca_flash_erase_sector_irq()?
A: The function will fail if the address parameter is not aligned to a sector boundary or if it is outside the valid flash memory range. Invalid addresses include non-aligned addresses or addresses that do not correspond to valid flash sectors on the STM32 microcontroller. The function does not return error codes directly, but the return value indicates success or failure of the operation.

Behavior conf:0.80 **Q:** Is ca_flash_erase_sector_irq() blocking or non-blocking, and is it safe to call from an ISR?
A: The function is blocking until the flash erase operation completes. It uses interrupt mode for the erase operation but blocks the calling thread until completion. The function is not explicitly marked as ISR-safe, so it should not be called from interrupt service routines. It is designed for use in thread contexts where blocking is acceptable.

`ca_flash_erase_sector_num`

Header: `caos/drivers/inc/flash.h:83`
Source: `caos/drivers/src/flash.c:122:130`
GUARD: `EN_FLASH`

```
1 bool ca_flash_erase_sector_num(u8 sector)
```

Erases a specific flash memory sector by its index number.

Parametri:

Nome	Tipo	Descrizione
`sector`	`u8`	The index of the flash sector to erase. Must be a valid sector index supported by the hardware; invalid indices will cause the function to return false.

Ritorna:
Returns true if the sector was successfully erased, or false if the sector index is invalid, the flash hardware could not be unlocked, or the erase operation failed.

Note:

- This function is a blocking operation, meaning the CPU will wait until the flash hardware completes the erase cycle before returning. It is not thread-safe by itself, so you must ensure that no other task attempts to access the flash while this function is running. It is strictly forbidden to call this from an Interrupt Service Routine (ISR) because the blocking nature will stall the entire system and potentially cause a watchdog reset. A common beginner mistake is forgetting that flash memory must be erased before it can be written to; attempting to write to a non-erased sector will result in corrupted data.

Uso:

- This function is typically used during configuration storage or firmware updates where you need to clear a specific memory region before writing new data. It is usually paired with ca_flash_hw_unlock() and ca_flash_hw_lock() to manage write protection, and followed by ca_flash_hw_write() to commit new data. In a typical workflow, you call this to prepare the sector, verify the success, and then proceed to write your data buffer to the now-blank memory area.

Esempio:

```c
/* Define the sector index to be cleared */
#define CONFIG_SECTOR 5

void update_config(uint8_t *data, uint32_t len) {
    /* Erase the sector before writing new configuration data */
    if (ca_flash_erase_sector_num(CONFIG_SECTOR)) {
        /* Unlock flash to allow writing */
        if (ca_flash_hw_unlock()) {
            /* Write the new data to the sector address */
            bool success = ca_flash_hw_write(ca_flash_hw_sector_addr(
    CONFIG_SECTOR), data, len);
            /* Always lock the flash again to prevent accidental corruption */
            ca_flash_hw_lock();
            if (!success) {
                /* Handle write failure */
            }
        }
    } else {
        /* Handle erase failure (e.g., invalid sector or hardware error) */
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_flash_erase_sector_num() in a caOS project?
A: You should use ca_flash_erase_sector_num() when you need to erase a specific flash sector on an STM32 microcontroller. It is intended for non-critical timing contexts, as it is a blocking operation. It is typically used when updating firmware, storing configuration data, or managing persistent settings that require sector-level flash manipulation.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_flash_erase_sector_num()?
A: u8 sector = 5; bool result = ca_flash_erase_sector_num(sector); if (result) { // Sector erased successfully } else { // Erasure failed }

Parameters conf:0.95 **Q:** What does the 'sector' parameter in ca_flash_erase_sector_num() represent, and what are valid values?
A: The 'sector' parameter represents the number of the flash sector to be erased. For STM32F4 microcontrollers, valid values are from 0 to 11. Using a value outside this range (e.g., >11) will cause the function to return false.

Return conf:0.90 **Q:** What do the return values of ca_flash_erase_sector_num() indicate, and what can cause it to return false?
A: The function returns true if the sector was erased successfully. It returns false if the sector number is invalid (e.g., >11), if the flash is not unlocked, or if a hardware failure occurs during the erase operation. Failure to handle the return value properly may lead to inconsistent data.

`ca_flash_get_error`

Header: `caos/drivers/inc/flash.h:63`
Source: `caos/drivers/src/flash.c:105:108`
GUARD: `EN_FLASH`

```c
u32 ca_flash_get_error(void)
```

Retrieves the current error code status of the flash memory peripheral.

Ritorna:
Returns a 32-bit unsigned integer representing the error state. A return value of 0 typically indicates 'no error' (success), while any non-zero value corresponds to a specific hardware or

driver error code defined in the flash driver header.

Note:

- This function is non-blocking and executes extremely quickly as it simply reads a global variable. It is thread-safe in the sense that it performs an atomic read of a 32-bit value, making it safe to call from both standard tasks and Interrupt Service Routines (ISRs). The most common beginner mistake is assuming that calling this function clears the error; it does not. You must ensure that the flash driver has been properly initialized via ca_flash_init before relying on this value, as an uninitialized driver might return garbage data or a stale error code from a previous boot cycle.

Uso:

- This function is typically used as a diagnostic tool after performing flash operations like ca_flash_write or ca_flash_erase_sector. You should call it immediately after a flash function returns an error status to determine the specific cause of the failure, such as a protection violation or a timeout. It is often paired with functions like ca_flash_write or ca_flash_erase_sector to verify the integrity of the operation. In a robust system, you would use this in an error-handling block to decide whether to retry the operation or enter a safe state.

Esempio:

```
/* Example: Checking flash status after an erase operation */
u32 status = ca_flash_erase_sector(0);
if (status != 0) {
    /* Operation failed, retrieve specific error code */
    u32 error_code = ca_flash_get_error();
    /* Log the error_code for debugging purposes */
    /* Call a custom function to handle the specific error_code */
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should a developer call ca_flash_get_error() and why is it important for flash operations?
A: A developer should call ca_flash_get_error() immediately after performing flash erase or program operations to check if any errors occurred. It is important because flash operations can fail due to hardware issues or invalid operations, and this function allows the developer to detect and handle such failures for debugging and error recovery. The function is non-blocking and can be called from any context, including ISRs.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_flash_get_error() with caOS types?
A: u32 error = ca_flash_get_error(); if (error != 0) { // Handle flash error // error value indicates specific type of failure }

Return conf:0.90 **Q:** What do the return values of ca_flash_get_error() mean and what causes flash failures?
A: The function returns 0 if no error occurred during the last flash operation. Any non-zero return value indicates a specific type of flash error such as ERASE_FAIL or PROGRAM_FAIL. Failures can be caused by hardware issues, invalid flash addresses, or improper flash operation sequences. The error persists until manually reset, so it's important to check the return value immediately after flash operations.

`ca_flash_get_num_sectors`

Header: `caos/drivers/inc/flash.h:70`
Source: `caos/drivers/src/flash.c:114:114`

GUARD: `EN_FLASH`

```
u16 ca_flash_get_num_sectors(void)
```

Returns the total number of flash memory sectors available on the current STM32 device.

Ritorna:
Returns a 16-bit unsigned integer representing the total count of flash sectors. This value is constant for a specific microcontroller model and will never return an error code or NULL.

Note:

- This function is non-blocking and executes almost instantaneously as it simply reads a hardware-defined constant. It is thread-safe and can be safely called from both main-line code and Interrupt Service Routines (ISRs) because it performs no complex operations or resource locking. The most common mistake is assuming the sector count is the same across different STM32 families; always use this function to query the hardware dynamically rather than hardcoding values. Ensure the flash driver is initialized before calling this, although in most caOS implementations, this is a static hardware mapping that is available immediately upon system boot.

Uso:

- This function is typically used during the initialization phase of an application to determine the boundaries of the flash memory layout. It is frequently paired with ca_flash_get_sector_addr and ca_flash_get_sector_size to iterate through all available sectors for tasks like clearing configuration data, performing firmware updates, or managing a file system. By using this function, your code remains portable across different STM32 chips, as the loop bounds will automatically adjust to the specific hardware's memory map.

Esempio:

```
u16 total_sectors = ca_flash_get_num_sectors();

// Iterate through all sectors to identify memory layout
for (u8 i = 0; i < total_sectors; i++) {
    u32 addr = ca_flash_get_sector_addr(i);
    u32 size = ca_flash_get_sector_size(i);

    // Log sector information for debugging purposes
    // printf("Sector %d: Addr=0x%08X, Size=%d bytes\n", i, addr, size);
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_flash_get_num_sectors() and why is it important to use its return value instead of a hardcoded number?
A: You should call ca_flash_get_num_sectors() to dynamically determine the number of flash sectors available on the STM32 device. It's important to use its return value because the actual number of sectors can vary between different STM32 models and configurations. Using a hardcoded number may lead to errors if the code is deployed on hardware with a different flash layout. The function is non-blocking and thread-safe, making it suitable for use in both regular code and interrupt service routines (ISR).

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_flash_get_num_sectors() with caOS types?
A: u16 num_sectors = ca_flash_get_num_sectors(); if (num_sectors != 0) { // Proceed with flash operations using the dynamic sector count // Example: u16 sector_size = ca_flash_get_sector_size(0); // Get size of first sector } else { // Handle error case - flash not initialized or error occurred }

Return conf:0.85 **Q:** What do the different return values of ca_flash_get_num_sectors() mean and what causes the function to return 0?
A: The function returns a u16 value representing the total number of valid flash sectors (0-65535). A return value of 0 indicates either an error occurred during flash initialization or the flash driver has not been properly initialized. The function reads a precomputed value stored during flash driver initialization, so failure to initialize the flash driver properly will result in a return value of 0. This function is not dependent on specific hardware registers like APB1 or AHB clocks, but rather on the driver's initialization state.

`ca_flash_get_sector_addr`

Header: `caos/drivers/inc/flash.h:73`
Source: `caos/drivers/src/flash.c:115:115`
GUARD: `EN_FLASH`

```
u32 ca_flash_get_sector_addr(u8 sector)
```

Retrieves the starting memory address of a specific flash sector.

Parametri:

Nome	Tipo	Descrizione
`sector`	`u8`	The index of the flash sector to query. Must be less than the value returned by ca_flash_get_num_sectors().

Ritorna:
Returns the 32-bit absolute memory address (u32) of the requested sector. Returns 0 if the sector index is invalid or out of range.

Note:

- This function is non-blocking and thread-safe as it performs a simple lookup in the hardware configuration table. It is safe to call from an ISR because it does not involve complex logic or waiting for hardware peripherals. The most common mistake is passing a sector index that exceeds the total number of available sectors, which will return 0 and potentially cause a crash if the result is used as a pointer without validation. Always ensure the flash driver is initialized before calling this function, as the underlying hardware mapping might not be populated otherwise.

Uso:

- This function is typically used during the initialization phase of a storage module or before performing read/write operations to calculate the target memory offset. It is almost always paired with ca_flash_get_sector_size() to verify that the data being written fits within the sector boundaries. Developers use it to map logical sector numbers to physical memory addresses, allowing the application to perform direct memory access or pass addresses to low-level flash programming functions.

Esempio:

```
u8 target_sector = 5;
u32 sector_start = ca_flash_get_sector_addr(target_sector);
u32 sector_size = ca_flash_get_sector_size(target_sector);

if (sector_start != 0) {
    // Address is valid, proceed with flash operations
    // Example: ca_flash_write(sector_start, data_buffer, data_len);
} else {
```

```
    // Handle error: invalid sector index provided
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_flash_get_sector_addr() in my embedded application?
A: You should use ca_flash_get_sector_addr() when you need to perform direct flash read/write operations on a specific sector. This function is useful when implementing custom flash drivers or when you need to know the physical address of a sector for low-level operations, such as flashing firmware or storing configuration data.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_flash_get_sector_addr()?
A: u32 sector_addr = ca_flash_get_sector_addr(5); // Get address of sector 5 if (sector_addr != 0) { // Use sector_addr for flash operations }

Parameters conf:0.90 **Q:** What is the valid range for the 'sector' parameter in ca_flash_get_sector_addr()?
A: The 'sector' parameter must be a value between 0 and 254 (inclusive). A value of 0xFF or higher causes undefined behavior. The actual maximum sector number depends on the specific STM32 chip being used, as different chips have different numbers of flash sectors.

Return conf:0.85 **Q:** What does a return value of 0 from ca_flash_get_sector_addr() indicate?
A: A return value of 0 from ca_flash_get_sector_addr() indicates that the function was called with an invalid sector number, which causes undefined behavior. The function does not explicitly return 0 for valid inputs; it returns the actual flash address of the sector. If the input sector number is invalid, the result is unpredictable and may cause memory access violations.

`ca_flash_get_sector_size`

Header: `caos/drivers/inc/flash.h:76`
Source: `caos/drivers/src/flash.c:116:116`
GUARD: `EN_FLASH`

```
u32 ca_flash_get_sector_size(u8 sector)
```

Retrieves the total byte size of a specific flash memory sector.

Parametri:

Nome	Tipo	Descrizione
`sector`	`u8`	The index of the flash sector to query. Must be a value between 0 and (ca_flash_get_num_sectors() - 1). Providing an out-of-bounds index may lead to undefined hardware behavior or return 0.

Ritorna:
Returns the size of the sector in bytes as a u32. Returns 0 if the sector index is invalid or if the hardware driver fails to retrieve the information.

Note:

- This function is non-blocking and thread-safe, as it performs a simple read operation from the hardware configuration table. It is safe to call from an ISR because it does not involve complex synchronization or waiting for hardware peripherals. The most common mistake is assuming all flash sectors have the same size; on many STM32 microcontrollers, sectors at the beginning of the memory map are often smaller than those at the end. Always verify the sector size before performing write operations to prevent buffer overflows or writing into adjacent memory regions.

Uso:

- This function is typically used during the initialization or validation phase of a flash storage routine to ensure that the data being written fits within the allocated sector. It is almost always paired with ca_flash_get_sector_addr to determine the starting memory address and the available capacity of the target region. Developers use this to perform bounds checking before calling erase or write functions, ensuring that the application does not corrupt critical data stored in other sectors.

Esempio:

```c
/* Define the sector we intend to use for configuration storage */
#define CONFIG_SECTOR 5

void save_config_data(void) {
    u32 sector_size = ca_flash_get_sector_size(CONFIG_SECTOR);
    u32 sector_addr = ca_flash_get_sector_addr(CONFIG_SECTOR);

    /* Validate that our data fits in the sector before proceeding */
    if (sizeof(my_config_struct) <= sector_size) {
        /* Safe to proceed with erase and write operations */
        if (ca_flash_erase_sector_num(CONFIG_SECTOR)) {
            /* Proceed to write data at sector_addr */
        }
    } else {
        /* Handle error: data too large for the allocated sector */
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_flash_get_sector_size() in my embedded application?
A: You should call ca_flash_get_sector_size() when you need to determine the size of a specific flash sector to properly manage memory operations such as reading or writing data. This is especially useful when implementing custom flash drivers or when you need to ensure that your data fits within the boundaries of a sector. It is typically called after initializing the flash driver to get sector-specific information.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_flash_get_sector_size() in C with caOS types?
A: u32 sector_size = ca_flash_get_sector_size(0); if (sector_size > 0) { // Proceed with flash operations using the sector size } else { // Handle invalid sector error }

Parameters conf:0.90 **Q:** What does the 'sector' parameter in ca_flash_get_sector_size() represent, and what are valid values?
A: The 'sector' parameter represents the index of the flash sector whose size you want to retrieve. Valid values are integers from 0 up to the maximum number of sectors available, which can be obtained using ca_flash_get_num_sectors(). Values outside this range, including negative numbers or values greater than the maximum number of sectors, will result in undefined behavior.

Return conf:0.95 **Q:** What do the return values of ca_flash_get_sector_size() indicate, and what causes it to return 0?
A: The function returns a positive u32 value representing the size in bytes of the specified flash sector. If the function returns 0, it indicates that the input 'sector' parameter was invalid, such as being greater than the maximum number of sectors or negative. This typically occurs when the function is called with an out-of-bounds sector index.

---

ca_flash_hw_erase_sector

Header: caos/drivers/inc/ca_glue_hw.h:142
GUARD: EN_CA_GLUE_HW

```
extern bool ca_flash_hw_erase_sector(u32 addr); /* cancella il settore
    che contiene addr (bloccante) */
```

Erases a specific flash memory sector containing the provided address.

Parametri:

Nome	Tipo	Descrizione
addr	u32	The memory address located within the sector to be erased. Must be a valid flash memory address; passing an invalid or out-of-bounds address may cause hardware errors or system instability.

Ritorna:
Returns true if the sector was erased successfully, or false if a hardware error occurred during the operation.

Note:

- This function is strictly blocking, meaning the CPU will wait until the flash hardware completes the erase cycle before returning. It is not thread-safe; if multiple tasks attempt to access the flash simultaneously, you must protect this call with a mutex or semaphore. Do not call this function from an Interrupt Service Routine (ISR) because the blocking wait will hang the system and potentially cause a watchdog reset. A common beginner mistake is assuming the flash is ready for immediate writing after this function returns without checking the return value for errors.

Uso:

- This function is typically used during firmware updates, configuration storage, or data logging where existing flash data must be cleared before writing new information. It is usually paired with a write function (like ca_flash_write) in a sequence: first, verify the address, then call this erase function, and finally perform the write operation. Always ensure the flash is unlocked using ca_flash_unlock before calling this function, as the hardware will ignore erase commands if the protection bits are set.

Esempio:

```
/* Example: Erasing a configuration sector before saving new settings */
u32 config_addr = 0x08020000; /* Example sector start address */

/* Ensure flash is accessible before erasing */
ca_flash_unlock();

if (ca_flash_hw_erase_sector(config_addr)) {
    /* Sector erased, now safe to write new data */
    if (ca_flash_write(config_addr, new_data, data_len)) {
        /* Write successful */
    } else {
        /* Handle write failure */
    }
} else {
    /* Handle erase failure: hardware error or locked sector */
}

/* Re-lock flash to prevent accidental corruption */
ca_flash_lock();
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_flash_hw_erase_sector() in my caOS project?
A: You should use ca_flash_hw_erase_sector() when you need to erase a flash sector, typically before writing new data to that sector. This function is essential for flash memory management, especially in applications that require frequent updates to persistent storage like configuration data, logs, or firmware updates.

Code example conf:0.95 **Q:** Can you show a minimal C example using ca_flash_hw_erase_sector() with caOS types?
A: u32 sector_addr = 0x08004000; /* Address aligned to sector start */ bool success = ca_flash_hw_erase_sector(sector_addr); if (!success) { /* Handle erase failure */ }

Parameters conf:0.95 **Q:** What does the addr parameter represent and what are valid values?
A: The addr parameter is an address within the flash sector to be erased. It must be aligned to the beginning of a sector (i.e., a multiple of the sector size). Invalid values include addresses that are not sector-aligned or addresses outside the valid flash memory range.

Return conf:0.95 **Q:** What do the return values true and false mean in ca_flash_hw_erase_sector()?
A: The function returns true if the sector erase operation completed successfully. It returns false if the operation failed, likely due to flash hardware errors, invalid addresses, or if another flash operation is in progress.

Error/Edge case conf:0.90 **Q:** What are common mistakes or edge cases when using ca_flash_hw_erase_sector
A: Common mistakes include calling the function with a non-sector-aligned address, calling it during an ongoing flash operation, or not checking the return value. Edge cases involve attempting to erase sectors outside the valid flash memory range or using the function from an ISR, which is not recommended due to its blocking nature.

Behavior conf:0.95 **Q:** Is ca_flash_hw_erase_sector() blocking or non-blocking, and is it safe to call from an ISR?
A: The function is blocking and will not return until the erase operation is complete. It is not safe to call from an ISR because it is blocking and could cause scheduling issues or timeouts. Additionally, it is not thread-safe, so external synchronization is required if used by multiple threads.

HW differences conf:0.90 **Q:** How does ca_flash_hw_erase_sector() differ between STM32F1xx and STM32H7xx families?
A: The F1xx family uses FLASH->CR and FLASH->AR registers with a simple PER bit for sector erase. The H7xx family uses separate CR1/CR2 registers for bank1/bank2, requires checking SR1/SR2 status registers, and includes cache invalidation after erase. Additionally, H7xx supports 32-bit programming and has more complex bank selection logic.

`ca_flash_hw_irq_disable`

Header: `caos/drivers/inc/ca_glue_hw.h:149`
GUARD: `EN_CA_GLUE_HW`

```
1 extern void ca_flash_hw_irq_disable(void); /* disabilita tutti gli
     interrupt FLASH */
```

Disables hardware interrupts related to Flash memory operations.

Ritorna:
void (no return value).

Note:

- This function is non-blocking and executes immediately by modifying the Flash control registers. It is thread-safe in the sense that it performs a direct bitwise operation on the hardware register, but it does not provide synchronization between tasks. It is safe to call from an ISR, as it only performs a register write. The most common mistake is forgetting to re-enable interrupts after a critical Flash operation, which will cause the system to hang or fail to signal the completion of subsequent Flash writes or erasures.

Uso:

- This function is used to prevent the Flash controller from triggering interrupts during sensitive operations, such as finalizing a write sequence or locking the peripheral. It is typically called as part of a cleanup sequence after a Flash operation has finished or failed. It is almost always used immediately before 'ca_flash_lock' to ensure the peripheral is in a safe, quiescent state before releasing control to other tasks.

Esempio:

```
/* Ensure Flash operations are finished before disabling interrupts */
/* Check if the flash peripheral is currently idle */
if (ca_flash_get_error() == 0U) {
    /* Disable Flash interrupts to prevent unexpected callbacks during shutdown */
    ca_flash_hw_irq_disable();

    /* Lock the hardware peripheral to prevent unauthorized access */
    ca_flash_lock();

    /* Signal that the Flash driver is now idle */
    /* ca_sem_post(&g_flash_sem); // Assuming a semaphore is used for synchronization */
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_flash_hw_irq_disable() and why is it important?
A: You should call ca_flash_hw_irq_disable() before performing critical flash write or erase operations to prevent interruptions that could corrupt data. It disables FLASH hardware interrupts like EOPIE and ERRIE to ensure atomicity of flash operations. This function is essential when using flash operations in multitasking environments or when flash access might be interrupted by other ISRs.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_flash_hw_irq_disable() with caOS types?
A:

```
u32 flash_addr = 0x08000000;
ca_flash_hw_irq_disable();
// Perform flash write here
ca_flash_hw_irq_enable();
```

Return conf:0.90 **Q:** What does ca_flash_hw_irq_disable() return and what can cause it to fail?
A: The function does not return any value. It directly modifies the FLASH_CR register hardware register to disable specific interrupt bits. Failures are not possible in terms of return codes, but incorrect usage (e.g., calling from ISR or without proper locking) may lead to loss of critical interrupts or data corruption.

HW differences conf:0.90 **Q:** How does ca_flash_hw_irq_disable() differ across different STM32 families?
A: The function varies by MCU family: F1xx uses FLASH->CR and masks FLASH_CR_EOPIE | FLASH_CR_ERRIE; F4xx uses FLASH->CR with F4_CR_IE; F7xx uses FLASH->CR with F7_CR_IE; H5xx uses FLASH->NSCR with H5_CR_IE; H7xx uses both

FLASH->CR1 and FLASH->CR2 with H7_CR_IE. These differences reflect variations in flash peripheral register layouts across STM32 families.

---

ca_flash_hw_irq_enable

Header: caos/drivers/inc/ca_glue_hw.h:148
GUARD: EN_CA_GLUE_HW

```
extern void ca_flash_hw_irq_enable(u32 addr); /* abilita EOPIE e gli
    interrupt di errore per il bank che contiene addr */
```

Enables hardware interrupts for Flash operations (End of Operation and Error) for a specific memory address.

Parametri:

Nome	Tipo	Descrizione
addr	u32	The target memory address in the Flash peripheral. On some architectures (like H7), it determines which bank's interrupt register is modified; on others, it is ignored.

Ritorna:
void

Note:

- This function is non-blocking and executes immediately by modifying hardware registers. It is thread-safe in the sense that it performs an atomic register write, but it does not manage synchronization; you must ensure that the Flash peripheral is unlocked before calling this. It is safe to call from an ISR, but typically it is called from a task context before initiating a long-running Flash operation. A common beginner mistake is calling this without first enabling the global NVIC interrupt for the Flash peripheral, which would result in the hardware triggering the interrupt but the CPU never executing the handler.

Uso:

- This function is used as a prerequisite for asynchronous (interrupt-driven) Flash operations like erasing or writing. It is typically paired with a sequence: first, you unlock the Flash hardware, then you call this function to enable the interrupt, and finally, you trigger the operation (e.g., ca_flash_hw_start_erase_irq). After triggering the operation, you usually wait on a semaphore (ca_sem_wait) to block the task until the interrupt handler signals that the operation is complete. Failing to call this function before starting an interrupt-driven operation will cause the system to hang indefinitely on the semaphore wait.

Esempio:

```
/* Ensure the Flash peripheral is ready and unlocked before enabling interrupts
    */
if (ca_flash_hw_unlock()) {
    /* Enable interrupts for the specific bank containing the address */
    ca_flash_hw_irq_enable(target_addr);

    /* Start the erase operation; this will trigger an interrupt upon completion
     */
    if (ca_flash_hw_start_erase_irq(target_addr)) {
        /* Wait for the ISR to signal completion via the semaphore */
        ca_sem_wait(&g_flash_sem);
    } else {
```

```
        /* Handle start failure */
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_flash_hw_irq_enable and why is it needed?
A: You should call ca_flash_hw_irq_enable when you need to enable hardware interrupts for flash operations, particularly for asynchronous erase operations. It enables both the end-of-operation interrupt (EOPIE) and error interrupt (ERRIE) for the flash bank containing the specified address. This is necessary to handle flash erase operations without blocking the CPU, allowing the system to perform other tasks while waiting for the erase to complete.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_flash_hw_irq_enable in a caOS application?
A:

```
u32 flash_addr = 0x08000000; /* Example flash address */
ca_flash_hw_irq_enable(flash_addr);
/* Flash interrupt is now enabled for the bank containing flash_addr */
```

Parameters conf:0.90 **Q:** What is the significance of the addr parameter in ca_flash_hw_irq_enable and what are valid values?
A: The addr parameter specifies an address within the flash bank for which interrupts should be enabled. It is used to determine which bank requires interrupt configuration. Valid values are any valid flash memory addresses. Invalid values include NULL or addresses outside the flash memory range. Note that the function does not actually use the address value in its implementation, it only uses it to identify the bank.

Return conf:0.85 **Q:** Does ca_flash_hw_irq_enable return any value and what could cause it to fail?
A: The function does not return any value. It cannot fail in terms of logical errors, but it may not have any effect if the system is not properly initialized or if the NVIC is not configured correctly. It is also not thread-safe and should not be called from an ISR. The function is blocking as it modifies peripheral and NVIC registers directly.

HW differences conf:0.90 **Q:** How does ca_flash_hw_irq_enable differ between various STM32 families like F1, F4, F7, H5, and H7?
A: The function differs across families in register usage. For F1xx, it sets FLASH->CR bits FLASH_CR_EOPIE and FLASH_CR_ERRIE. For F4xx, it uses F4_CR_IE mask on FLASH->CR. For F7xx, it uses F7_CR_IE mask on FLASH->CR. For H5xx, it uses H5_CR_IE mask on FLASH->NSCR. For H7xx, it checks if the address is in bank2 and sets either FLASH->CR1 or FLASH->CR2 with H7_CR_IE mask.

`ca_flash_hw_irq_handler`

Header: `caos/drivers/inc/ca_glue_hw.h:152`
GUARD: `EN_CA_GLUE_HW`

```
extern u32 ca_flash_hw_irq_handler(void); /* gestore IRQ FLASH: legge SR
    e pulisce i flag; ritorna maschera errori */
```

Handles hardware-specific Flash interrupts by clearing status flags and returning error codes.

Ritorna:
A 32-bit bitmask representing hardware-specific flash errors (e.g., protection, programming, or sequence errors). A return value of 0 indicates no errors were detected.

Note:

- This function is designed to be called exclusively from within a hardware Interrupt Service Routine (ISR). It is not thread-safe and must never be called from a standard task context, as it directly manipulates hardware registers to acknowledge and clear interrupt flags. A common mistake is failing to realize that this function clears the hardware status registers; if you call it manually outside the ISR, you will lose the ability to detect the actual interrupt event. It is non-blocking and executes very quickly, which is critical for maintaining system responsiveness during flash operations.

Uso:

- This function is the core component of the Flash driver's interrupt management strategy. It is typically invoked inside the global 'FLASH_IRQHandler' function, which the CPU automatically triggers when a flash operation completes or fails. After calling this function, the driver usually updates a global error variable and signals a semaphore (using 'ca_sem_post') to notify a waiting task that the flash operation has finished. This sequence allows the system to perform other tasks while the slow flash hardware is busy, ensuring efficient CPU utilization.

Esempio:

```
void FLASH_IRQHandler(void) {
    /* 1. Acknowledge the hardware interrupt and clear flags */
    u32 error_mask = ca_flash_hw_irq_handler();

    /* 2. Store the error status for the application to check */
    g_flash_error = error_mask;

    /* 3. Unblock the task waiting for the flash operation to complete */
    ca_sem_post(&g_flash_sem);
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should ca_flash_hw_irq_handler be called, and why is it important to call it only from an ISR?
A: The function ca_flash_hw_irq_handler should only be called from an ISR (Interrupt Service Routine) that handles flash interrupts. It is important to call it only in ISR context because it is a blocking function that modifies flash registers and must be executed in the correct interrupt context to ensure proper flag clearing and error handling. It is not thread-safe and must be called only when certain that the flash IRQ is active.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_flash_hw_irq_handler in an ISR?
A:

```
void FLASH_IRQHandler(void)
{
    u32 errors = ca_flash_hw_irq_handler();
    if (errors != 0)
    {
        /* Handle flash errors */
    }
}
```

Return conf:0.90 **Q:** What does the return value of ca_flash_hw_irq_handler indicate, and what could cause it to return an error?
A: The return value of ca_flash_hw_irq_handler is a u32 representing the error mask detected during flash operation. It returns 0 if no errors occurred. If an error is detected, it returns a value indicating which specific flash errors occurred, such as programming, erase, or operation errors. Failures can occur if the function is called outside of an ISR context or if the flash hardware is in an inconsistent state.

HW differences conf:0.85 **Q:** How does ca_flash_hw_irq_handler differ between F7xx and H7xx MCU families in terms of implementation?
A: The implementation of ca_flash_hw_irq_handler differs between F7xx and H7xx MCUs. In F7xx, the function clears flash control registers and invalidates both instruction and data caches using SCB functions to maintain memory coherency. In H7xx, the function handles two flash banks (SR1 and SR2) and clears flags in both banks (CCR1 and CCR2) separately, reflecting the dual-bank architecture of H7xx flash memory.

`ca_flash_hw_lock`

Header: `caos/drivers/inc/ca_glue_hw.h:141`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_flash_hw_lock(void); /* ripristina la protezione
   scrittura del controller FLASH */
```

Re-enables write protection on the internal Flash memory controller.

Ritorna:
void

Note:

- This function is non-blocking and executes immediately by setting the hardware lock bit in the Flash control register. It is thread-safe in the sense that it performs a single atomic register write, but it does not provide mutual exclusion; if two tasks call this simultaneously, they may interfere with the flash state. It is safe to call from an ISR, though typically you should only lock flash from the same task that performed the write operation. The most common beginner mistake is forgetting to call this after a write or erase operation, leaving the flash memory vulnerable to accidental corruption by stray pointers or software bugs.

Uso:

- This function is used to finalize flash memory operations by re-enabling hardware write protection. It is typically used in a 'sandwich' pattern: first, call the unlock function to gain write access, perform the necessary write or erase operations, and finally call this lock function to secure the memory. It is essential to call this immediately after the flash operation completes to maintain system integrity and prevent unintended data modification during normal application execution.

Esempio:

```
/* Example: Writing data to flash safely */
void update_config_data(uint32_t addr, uint8_t *data, uint32_t len) {
    /* 1. Unlock flash to allow modifications */
    if (!ca_flash_hw_unlock()) {
        return; /* Handle error: flash could not be unlocked */
    }

    /* 2. Perform the write operation */
    bool success = ca_flash_hw_write(addr, data, len);

    /* 3. Always lock the flash again, regardless of success or failure */
    ca_flash_hw_lock();

    if (!success) {
        /* Handle write failure */
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_flash_hw_lock() and what is its purpose in a caOS application?
A: You should call ca_flash_hw_lock() to prevent accidental modifications to flash memory by disabling write access to the FLASH controller. It is typically called after completing flash operations that require unlocking, such as writing or erasing data. This function ensures data integrity by protecting against unintended changes to flash memory.

Code example conf:0.90 **Q:** Can you provide a minimal C example using caOS types that demonstrates how to use ca_flash_hw_lock()?
A:

```
u32 flash_data = 0x12345678;
// Assume flash is already unlocked
// Perform flash write operations here
ca_flash_hw_lock(); // Lock flash after write
```

HW differences conf:0.90 **Q:** How does ca_flash_hw_lock() differ across various STM32 families in terms of hardware implementation?
A: The implementation of ca_flash_hw_lock() varies by STM32 family. For F1xx, F4xx, and F7xx families, it writes to FLASH->CR. For H5xx, it writes to FLASH->NSCR. For H7xx, it writes to both FLASH->CR1 and FLASH->CR2. These differences reflect the distinct flash controller architectures across the STM32 families.

`ca_flash_hw_num_sectors`

Header: `caos/drivers/inc/ca_glue_hw.h:149`
GUARD: `EN_CA_GLUE_HW`

```
extern u16 ca_flash_hw_num_sectors(void); /* returns the total number of
    MCU FLASH sectors */
```

Returns the total number of physical flash memory sectors available on the current MCU.

Ritorna:
A 16-bit unsigned integer representing the total count of flash sectors. The value is hardware-dependent and reflects the specific memory layout of the STM32 chip being used.

Note:

- This function is non-blocking, thread-safe, and safe to call from an ISR because it performs a simple register read or returns a constant value. It does not require any prior initialization of the flash peripheral, making it safe to call during early system startup. A common mistake is assuming that all STM32 chips have the same number of sectors; always use this function to query the hardware dynamically rather than hardcoding values in your application logic. If you hardcode sector counts, your code will break when ported to a different STM32 series or even a different density variant of the same series.

Uso:

- This function is typically used as a boundary check before performing flash erase or write operations to ensure the application does not attempt to access memory outside the valid range. It is usually paired with ca_flash_get_sector_addr and ca_flash_get_sector_size to iterate through the flash memory map. Developers use it to dynamically calculate the last available sector (e.g., for configuration storage) to ensure they do not overwrite the application code area, which is a critical safety practice to prevent bricking the device.

Esempio:

```c
/* Example: Safely erasing the last flash sector for configuration storage */
#include "caos.h"

void erase_config_area(void) {
    /* Get the total count to identify the last sector index */
    u16 total_sectors = ca_flash_get_num_sectors();

    if (total_sectors > 0) {
        u8 last_sector = (u8)(total_sectors - 1);

        /* Erase the last sector to clear old configuration data */
        /* Ensure the flash is unlocked before calling erase */
        ca_flash_unlock();
        ca_flash_erase_sector(last_sector);
        ca_flash_lock();
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_flash_hw_num_sectors() and why is it important for flash management?
A: You should call ca_flash_hw_num_sectors() during system initialization to determine the total number of flash sectors available on the MCU. This value is essential for managing flash erase and write operations, as it defines the valid range of sectors you can work with. It's a one-time call because the flash sector count is fixed per hardware and should not be called repeatedly during runtime.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_flash_hw_num_sectors() with caOS types?
A: u16 num_sectors = ca_flash_hw_num_sectors(); if (num_sectors == 0) { /* Handle error */ } else { /* Use num_sectors to allocate or manage flash sectors */ }

Return conf:0.90 **Q:** What do the return values of ca_flash_hw_num_sectors() mean, and what causes a return of 0?
A: The function returns a u16 value representing the total number of flash sectors. A return value of 0 indicates an error in retrieving the flash size information, typically due to a failure in reading the hardware-specific flash size register. If the calculated number of sectors exceeds 0xFFFF, it is capped at 0xFFFF to prevent overflow, but a return of 0 specifically signals an initialization or hardware read failure.

HW differences conf:0.90 **Q:** How does ca_flash_hw_num_sectors() differ between STM32F1xx and STM32F4xx families in terms of implementation?
A: For STM32F1xx, the function reads the flash size register at address 0x1FFFF7E0 to calculate the number of sectors based on the flash size in KB. For STM32F4xx, it simply returns a fixed value of 12 sectors, as the flash configuration is hardcoded for that family. This difference reflects the varying flash architectures and memory mapping between these MCU families.

`ca_flash_hw_nvic_enable`

Header: `caos/drivers/inc/ca_glue_hw.h:147`
GUARD: `EN_CA_GLUE_HW`

```c
extern void ca_flash_hw_nvic_enable(void); /* abilita FLASH_IRQn nel
    NVIC */
```

Configures and enables the Flash memory interrupt in the Nested Vectored Interrupt Controller (NVIC).

Ritorna:
This function returns void and does not provide feedback on success or failure.

Note:

- This function is non-blocking and executes extremely quickly as it only writes to NVIC registers. It is thread-safe in the sense that NVIC register access is atomic, but it should typically be called only once during system initialization. It is safe to call from an ISR, though it is almost exclusively used during the driver setup phase. The most common beginner mistake is calling this function before the system clock or peripheral clocks are properly initialized, which can lead to unpredictable behavior if the NVIC peripheral is not yet accessible. Always ensure this is called as part of the driver's initialization sequence to prevent the CPU from ignoring Flash completion or error events.

Uso:

- This function is a mandatory step in the initialization sequence of the Flash driver. It is typically called within a higher-level initialization function like 'ca_flash_irq_init' to prepare the hardware to signal the CPU when a Flash operation (like erase or write) completes or encounters an error. By setting the priority to 6, it ensures that Flash events are handled with a specific urgency relative to other system interrupts. It is always paired with the setup of synchronization primitives, such as semaphores, which are used to block the calling task until the interrupt handler signals that the hardware operation is finished.

Esempio:

```c
void system_flash_setup(void) {
    /* Initialize the semaphore used to wait for flash operations */
    ca_sem_init(&g_flash_sem, "S_FLASH", 0U, 1U);

    /* Enable the hardware interrupt in the NVIC so the CPU can receive signals
    */
    ca_flash_hw_nvic_enable();

    /* Mark the driver as ready for use */
    g_flash_irq_ready = true;
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should ca_flash_hw_nvic_enable() be called, and what is its purpose in the context of flash operations?
A: ca_flash_hw_nvic_enable() should be called during system initialization to enable the FLASH_IRQn interrupt in the NVIC. This allows the system to handle asynchronous events during flash read/write operations. The function is intended to be called only from initialization tasks, not from ISRs, and is necessary for proper flash operation handling on STM32 microcontrollers.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_flash_hw_nvic_enable() with caOS types?
A: void init_system(void) { ca_flash_hw_nvic_enable(); // Other initialization code }

Return conf:0.85 **Q:** Does ca_flash_hw_nvic_enable() return any value, and what could cause it to fail?
A: The function does not return any value. It is declared as void and has no return statement. According to the documentation, it cannot fail if parameters are correct, but calling it incorrectly (e.g., before system initialization) may cause undefined behavior or crashes.

HW differences conf:0.75 **Q:** What are the key differences in how ca_flash_hw_nvic_enable() is implemented across different STM32 families like F1xx, F4xx, F7xx, H5xx, and H7xx?

A: While the function sets the same priority (6) and enables the same FLASH_IRQn for all families, the implementations differ in the specific registers used for interrupt configuration. F4xx, F7xx, and H7xx use FLASH_CR_EOPIE and FLASH_CR_ERRIE bits, H5xx uses additional WRPERRIE and PGSERRIE bits, and F1xx uses a simpler configuration without explicit interrupt enable bits in the function body.

`ca_flash_hw_sector_addr`

Header: `caos/drivers/inc/ca_glue_hw.h:145`
GUARD: `EN_CA_GLUE_HW`

```
extern u32 ca_flash_hw_sector_addr(u8 sector); /* ritorna l'indirizzo
    base del settore indicato */
```

Calculates the absolute memory address of a specific flash sector based on the hardware architecture.

Parametri:

Nome	Tipo	Descrizione
`sector`	`u8`	The zero-based index of the flash sector. Valid values range from 0 to (ca_flash_hw_num_sectors() - 1). Providing an index outside this range will result in an invalid address return.

Ritorna:
Returns the 32-bit absolute memory address (u32) of the start of the requested sector. Returns 0U if the sector index is out of bounds or invalid for the specific hardware platform.

Note:

- This function is non-blocking, thread-safe, and can be safely called from an ISR because it performs simple arithmetic or array lookups without accessing hardware registers or waiting for flash operations. It does not require any initialization beyond the standard system boot. The most common mistake is assuming that sector sizes are uniform across all STM32 families; always use this function to resolve addresses rather than hardcoding offsets, as the memory map changes significantly between F1, F4, H7, and U5 series. If you receive 0U, it indicates a logic error in your code where you are attempting to access a non-existent sector.

Uso:

- This function is typically used as a foundational step before performing low-level flash operations like erasing or writing data. It is usually paired with ca_flash_hw_num_sectors() to validate the sector index before calling functions like ca_flash_erase_sector_num(). Developers use it to calculate pointers for memory-mapped access or to verify that a data structure fits within a specific flash boundary. It acts as an abstraction layer, allowing your application code to remain portable across different STM32 microcontrollers.

Esempio:

```
u8 target_sector = 5;
u16 max_sectors = ca_flash_hw_num_sectors();

// Always validate the sector index before calculating the address
if (target_sector < max_sectors) {
    u32 sector_addr = ca_flash_hw_sector_addr(target_sector);
    if (sector_addr != 0U) {
```

```
        // Proceed to erase the sector using the resolved address
        if (ca_flash_erase_sector_num(target_sector)) {
            // Success: sector erased
        }
    }
} else {
    // Handle error: invalid sector index
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_flash_hw_sector_addr() in my caOS project?
A: You should call ca_flash_hw_sector_addr() when you need to determine the base address of a specific flash sector for operations like reading, writing, or erasing data. This function is essential when working directly with flash memory at the hardware level, especially in low-level drivers or when implementing custom flash management routines.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_flash_hw_sector_addr() in C with caOS types?
A: u32 addr = ca_flash_hw_sector_addr(5); if (addr != 0) { /* valid sector address */ } else { /* invalid sector number */ }

Parameters conf:0.95 **Q:** What does the 'sector' parameter represent, and what are valid values?
A: The 'sector' parameter is the index of the flash sector whose base address you want to retrieve. Valid values are integers from 0 up to (but not including) the total number of sectors for that MCU family. For example, on F4xx family, valid values are 0-11; on H7xx, valid values are 0-15. Passing a value equal to or greater than the total number of sectors will cause the function to return 0.

Return conf:0.95 **Q:** What do the return values of ca_flash_hw_sector_addr() mean, and what causes it to return 0?
A: A non-zero return value is the base address of the requested flash sector. The function returns 0 if the sector number is invalid (i.e., greater than or equal to the total number of sectors for that MCU family) or in case of an internal error. This behavior helps prevent accidental writes or reads to invalid memory locations.

HW differences conf:0.85 **Q:** How does ca_flash_hw_sector_addr() differ between F4xx and H7xx MCUs in terms of implementation?
A: On F4xx, the function uses a simple arithmetic calculation: F1_FLASH_BASE + sector * F1_PAGE_SIZE, assuming all sectors are of equal size. In contrast, H7xx uses a lookup table (s_f7_sector_addr[sector]) to compute sector addresses, which allows for variable sector sizes and different memory mapping between banks. This reflects the more complex flash architecture of H7xx compared to F4xx.

`ca_flash_hw_sector_size`

Header: `caos/drivers/inc/ca_glue_hw.h:146`
GUARD: `EN_CA_GLUE_HW`

```
extern u32 ca_flash_hw_sector_size(u8 sector); /* ritorna la dimensione
    in byte del settore indicato */
```

Returns the size in bytes of a specific flash memory sector.

Parametri:

Nome	Tipo	Descrizione
`sector`	`u8`	The index of the flash sector to query. Must be less than the total number of sectors returned by ca_flash_hw_num_sectors().

Ritorna:
Returns the size of the sector in bytes as a u32. Returns 0 if the sector index is out of bounds or invalid for the specific hardware.

Note:

- This function is non-blocking and executes in constant time, making it safe to call from both main threads and Interrupt Service Routines (ISRs). It is thread-safe because it performs a read-only operation on hardware-defined constants or lookup tables. The most common mistake is passing a sector index that exceeds the hardware limit, which will return 0; always validate your index against ca_flash_hw_num_sectors() before performing calculations to avoid logic errors in memory management.

Uso:

- This function is typically used during the initialization phase of a storage driver or when calculating memory offsets for writing data to flash. It is frequently paired with ca_flash_hw_num_sector to iterate through all available memory blocks and ca_flash_hw_sector_addr() to determine the starting memory address of a specific sector. Developers use this to ensure they do not overflow a sector boundary when writing data, as flash memory must be erased in whole-sector chunks.

Esempio:

```
u8 sector_to_check = 2;
u32 sector_size = ca_flash_hw_sector_size(sector_to_check);

if (sector_size > 0) {
    u32 start_addr = ca_flash_hw_sector_addr(sector_to_check);
    // Proceed to use the sector size for memory boundary checks
} else {
    // Handle error: invalid sector index provided
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_flash_hw_sector_size() in my embedded application?
A: You should call ca_flash_hw_sector_size() when you need to determine the size of a specific flash sector for operations like reading, writing, or erasing data. It's useful for calculating memory layout, managing flash storage, or ensuring correct sector boundaries during flash operations.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_flash_hw_sector_size() with caOS types?
A: u32 sector_size = ca_flash_hw_sector_size(5U); if (sector_size == 0U) { /* handle error */ } else { /* use sector_size for flash operations */ }

Parameters conf:0.90 **Q:** What does the 'sector' parameter represent, and what values are valid?
A: The 'sector' parameter is the index of the flash sector whose size you want to retrieve. Valid values are from 0 to the total number of sectors minus one. For example, on F4xx, valid sectors are 0-11. Values equal to or greater than the total number of sectors will cause the function to return 0.

Return conf:0.95 **Q:** What do the return values of ca_flash_hw_sector_size() mean, and what causes a return of 0?
A: The function returns the size in bytes of the specified sector. For F1xx, it always returns F1_PAGE_SIZE. For F4xx and F7xx, it returns the size from precomputed arrays. For H5xx and H7xx, it returns H5_SECTOR_SIZE or H7_SECTOR_SIZE respectively. A return value of 0 indicates an invalid sector index, i.e., the sector number is greater than or equal to the total number of sectors.

HW differences conf:0.90 **Q:** How does ca_flash_hw_sector_size() behave differently across STM32 families like F1xx, F4xx, H5xx, etc.?
A: On F1xx, all sectors are the same size (F1_PAGE_SIZE), so the function always returns that constant. On F4xx and F7xx, sector sizes are variable and defined in arrays (s_f4_sector_size[] and s_f7_sector_size[]), so the function returns different values based on the sector index. On H5xx and H7xx, all sectors are the same size (H5_SECTOR_SIZE or H7_SECTOR_SIZE), so the function returns a constant value for valid indices.

`ca_flash_hw_start_erase_irq`

Header: `caos/drivers/inc/ca_glue_hw.h:150`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_flash_hw_start_erase_irq(u32 addr); /* avvia erase
   asincrona; l'IRQ segnala il completamento */
```

Initiates a non-blocking flash sector erase operation and triggers an interrupt upon completion.

Parametri:

Nome	Tipo	Descrizione
`addr`	`u32`	The starting memory address of the flash sector or page to be erased. Must be aligned to the hardware-specific sector/page boundary.

Ritorna:
Always returns true in current implementations, indicating the hardware command sequence has been successfully triggered.

Note:

- This function is non-blocking regarding the actual erase time, meaning it returns immediately after starting the hardware peripheral. It is not thread-safe; you must ensure that no other flash operations occur simultaneously, typically by using a mutex or semaphore to guard the flash peripheral. It is generally not intended to be called directly from an ISR, as it relies on the system's ability to handle the subsequent completion interrupt. A common beginner mistake is failing to call the corresponding 'irq_enable' function before this, which results in the system hanging indefinitely because the completion interrupt is never triggered or handled.

Uso:

- This function is used in asynchronous flash management workflows where the CPU should perform other tasks while the hardware erases memory. It is typically used in a sequence: first, unlock the flash and enable interrupts via 'ca_flash_hw_irq_enable', then call this function to start the erase, and finally wait for completion using a semaphore like 'ca_sem_wait'. After the semaphore is released by the ISR, you must disable interrupts and lock the flash to ensure system stability.

Esempio:

```
/* Ensure flash is unlocked and interrupts are enabled before starting */
ca_flash_hw_irq_enable(target_addr);

/* Start the erase process asynchronously */
if (ca_flash_hw_start_erase_irq(target_addr)) {
    /* Wait for the ISR to signal completion via the semaphore */
```

```
    if (ca_sem_wait(&g_flash_sem)) {
        /* Check if the operation finished without errors */
        if (g_flash_error == 0U) {
            /* Success: proceed with next steps */
        }
    }
}

/* Cleanup: disable interrupts and lock the peripheral */
ca_flash_hw_irq_disable();
ca_flash_hw_lock();
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_flash_hw_start_erase_irq() instead of a synchronous erase function?
A: You should use ca_flash_hw_start_erase_irq() when you want to initiate an erase operation without blocking the calling thread or ISR. It is ideal for scenarios where you need to perform flash erase in the background, such as during system initialization or when handling data logging, allowing other tasks to proceed while the erase completes via interrupt.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_flash_hw_start_erase_irq()?
A: u32 addr = 0x08000000; /* flash address */ bool success = ca_flash_hw_start_erase_irq(addr); if (!success) { /* handle error */ }

Parameters conf:0.95 **Q:** What is the significance of the address parameter in ca_flash_hw_start_erase_irq()?
A: The address parameter specifies the starting location in flash memory to begin the erase operation. It must be aligned to a sector boundary (typically 16KB or 64KB depending on the MCU). Invalid values include addresses outside the flash memory range or misaligned addresses.

Return conf:0.90 **Q:** What does it mean if ca_flash_hw_start_erase_irq() returns false?
A: The function always returns true upon successfully initiating the erase operation. It does not indicate success or failure of the erase itself, only that the operation was started. A return value of false is not documented as a possible outcome.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_flash_hw_start_erase_irq()?
A: Common mistakes include calling this function without ensuring the flash is not busy, not disabling interrupts before calling it, or passing an invalid or misaligned address. It is also unsafe to call this function from an ISR, as it modifies flash control registers.

Behavior conf:0.95 **Q:** Is ca_flash_hw_start_erase_irq() thread-safe and can it be called from an ISR?
A: No, ca_flash_hw_start_erase_irq() is not thread-safe and must be protected by a mutex or similar synchronization mechanism. It also cannot be called from an ISR, as it modifies hardware registers and is not designed for interrupt context.

HW differences conf:0.85 **Q:** How does ca_flash_hw_start_erase_irq() differ between F1xx and H7xx families?
A: The F1xx family uses FLASH->CR and FLASH->AR registers for erase configuration, while the H7xx family uses separate CR1/CR2 and CCR1/CCR2 registers for bank-specific operations. Additionally, H7xx supports dual-bank flash and requires bank-specific handling for sector erase operations.

`ca_flash_hw_start_write_irq`

Header: `caos/drivers/inc/ca_glue_hw.h:151`
GUARD: `EN_CA_GLUE_HW`

```
extern u32 ca_flash_hw_start_write_irq(u32 addr, const u8 *buf, u32
    remaining); /* avvia scrittura asincrona; ritorna i byte consumati */
```

Initiates a non-blocking hardware flash write operation for a single memory word/block.

Parametri:

Nome	Tipo	Descrizione
addr	u32	The target memory address in the flash peripheral. Must be correctly aligned according to the specific MCU architecture (e.g., 4-byte alignment for H7/U5/H5).
buf	const u8 *	Pointer to the source data buffer containing the bytes to be written.
remaining	u32	The number of bytes left to write. The function will pad the remainder of the hardware write unit with 0xFF if this value is smaller than the hardware's native write unit size.

Ritorna:
Returns the number of bytes consumed (the hardware write unit size, e.g., 2, 4, 16, or 32 bytes). Returns 0 if the operation could not be initiated.

Note:

- This function is intended to be used as part of an asynchronous write sequence. It is blocking in the sense that it waits for the flash peripheral to be ready (wait_bsy) before starting, but it does not wait for the actual write to complete; instead, it triggers a hardware interrupt (IRQ) upon completion. It is generally thread-safe if protected by a semaphore or mutex, but it is not intended to be called directly from an ISR unless the system design explicitly allows nested flash operations. The most common beginner mistake is failing to wait for the completion semaphore after calling this function, which leads to race conditions where the next write starts before the previous one has finished, causing data corruption or hardware errors.

Uso:

- This function is the core engine for writing data to flash in chunks. It is typically used inside a loop that iterates through a large data buffer, calling this function to write one 'unit' at a time. It must be paired with a synchronization mechanism (like a semaphore) to wait for the 'End of Operation' (EOP) interrupt to signal that the hardware is ready for the next chunk. The typical sequence is: 1) Initialize flash/unlock, 2) Loop through data, 3) Call ca_flash_hw_start_write_irq, 4) Wait on a semaphore, 5) Check for errors, 6) Repeat until finished, 7) Disable IRQs and lock flash.

Esempio:

```
/* Example of writing a buffer to flash memory */
u32 addr = 0x08020000; /* Target flash address */
u8 data[] = {0xDE, 0xAD, 0xBE, 0xEF}; /* Data to write */
u32 len = 4;
u32 off = 0;

while (off < len) {
    /* Start the write for the current chunk */
    u32 consumed = ca_flash_hw_start_write_irq(addr + off, data + off, len - off
    );

    if (consumed == 0) {
        /* Handle error: hardware failed to start write */
        break;
```

```
    }

    /* Wait for the EOP interrupt to signal completion */
    ca_sem_wait(&g_flash_sem);

    /* Check if the hardware reported an error during the write */
    if (g_flash_error) {
        break;
    }

    off += consumed;
}
```

Q&A — fine-tuning review

7/7 pairs · qwen3-coder:latest

Usage conf:0.95 **Q:** When should I use ca_flash_hw_start_write_irq() instead of a synchronous flash write function?

A: You should use ca_flash_hw_start_write_irq() when you need to perform flash writes without blocking the execution of your task or thread. It is designed for asynchronous operations that complete via an interrupt, allowing other tasks to run while the flash write is in progress. This is especially useful in real-time systems where responsiveness is critical.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_flash_hw_start_write

A: u32 addr = 0x08000000; u8 data[4] = {0x12, 0x34, 0x56, 0x78}; u32 written = ca_flash_hw_start_write_irq(addr, data, 4); if (written == 0) { /* handle error */ }

Parameters conf:0.90 **Q:** What are the valid ranges for the 'addr' parameter in ca_flash_hw_start_write_irq()?

A: The 'addr' parameter must be a valid flash memory address. It must be aligned correctly for the target MCU family (e.g., 2-byte aligned for F1xx). It cannot be NULL and must point to a valid flash region.

Return conf:0.95 **Q:** What does a return value of 0 from ca_flash_hw_start_write_irq() indicate?

A: A return value of 0 indicates an error occurred during the function call. This could be due to invalid parameters such as a NULL buffer, invalid address, or an operation that could not be initiated.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_flash_hw_start_write_irq()?

A: Common mistakes include calling this function from an ISR context, not properly handling the IRQ completion, failing to ensure no other flash operations are in progress, and not disabling interrupts during critical sections. Also, not ensuring the flash is in a ready state before calling it can lead to errors.

Behavior conf:0.95 **Q:** Is ca_flash_hw_start_write_irq() thread-safe and can it be called from an ISR?

A: No, ca_flash_hw_start_write_irq() is not thread-safe and should not be called from an ISR. It is designed to be used in task or thread contexts where it can safely manage flash operations without race conditions. It is non-blocking and relies on IRQ handling for completion.

HW differences conf:0.85 **Q:** How does ca_flash_hw_start_write_irq() differ between F1xx and H7xx families in terms of hardware implementation?

A: The F1xx family writes 2 bytes at a time using 16-bit operations and sets the FLASH_CR_PG bit in the FLASH->CR register. In contrast, the H7xx family writes 32 bytes at a time using 32-bit operations, and uses separate registers for bank 1 and bank 2 (FLASH->CR1 and FLASH->CR2) with different PSIZE settings.

---

ca_flash_hw_unlock

Header: caos/drivers/inc/ca_glue_hw.h:140
GUARD: EN_CA_GLUE_HW

```
extern bool ca_flash_hw_unlock(void); /* rimuove la protezione scrittura del controller FLASH */
```

Removes the hardware write-protection from the internal Flash memory controller.

Ritorna:
Returns true if the Flash controller is successfully unlocked (or was already unlocked), and false if the unlock sequence failed or the controller remains locked.

Note:

- This function is non-blocking in terms of RTOS scheduling, but it interacts directly with hardware registers that may be busy. It is thread-safe only if the application ensures no other task or ISR attempts to modify Flash registers simultaneously. It is generally safe to call from an ISR, but you must ensure the Flash is not currently busy with a write or erase operation, as the hardware will ignore the unlock keys if the BUSY flag is set. A common beginner mistake is calling this function without checking if the Flash is busy, or forgetting to call the corresponding lock function immediately after the write operation is complete, which leaves the memory vulnerable to accidental corruption.

Uso:

- This function is used as a mandatory precursor to any write or erase operation on the internal Flash memory. In a typical sequence, you first verify the Flash is idle, call ca_flash_hw_unlock(), perform your write or erase operation, and then immediately call the lock function to restore protection. It is typically paired with ca_flash_hw_lock() to ensure the 'unlocked' state is as short-lived as possible, minimizing the risk of stray pointers or software bugs corrupting the firmware or configuration data stored in Flash.

Esempio:

```
/* Ensure the flash is ready before attempting to unlock */
if (ca_flash_hw_unlock()) {
    /* Flash is now unlocked, safe to perform write operations */
    bool success = ca_flash_hw_write(target_address, data_buffer, data_size);

    /* Always lock the flash immediately after the operation */
    ca_flash_hw_lock();

    if (!success) {
        /* Handle write failure */
    }
} else {
    /* Handle unlock failure: hardware might be busy or faulty */
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_flash_hw_unlock() in my caOS application?
A: You should call ca_flash_hw_unlock() before performing any FLASH programming or erasure operations. This function removes the write protection from the FLASH controller, which is required for modifying flash contents. It must be called in a privileged context and cannot be invoked from an ISR.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_flash_hw_unlock() in a caOS application?

A: bool success = ca_flash_hw_unlock(); if (!success) { // Handle error: flash could not be unlocked return false; } // Proceed with flash programming or erasure return true;

Return conf:0.90 **Q:** What does ca_flash_hw_unlock() return and what causes it to return false?
A: The function returns true if the FLASH controller was successfully unlocked, or false if the operation failed. Failures can occur due to hardware issues, incorrect timing, or in some cases like H7xx chips, if the flash is already in a non-recoverable locked state.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_flash_hw_unlock()?
A: Common mistakes include ignoring the return value of ca_flash_hw_unlock() and proceeding with flash operations without verifying that the flash is actually unlocked. Another frequent error is calling this function from an ISR or in a non-privileged context, which can lead to undefined behavior or system crashes.

HW differences conf:0.90 **Q:** How does ca_flash_hw_unlock() differ between STM32 H7xx and other families like F4xx?
A: For F4xx and similar families, ca_flash_hw_unlock() uses a single key register (KEYR) to unlock the flash. In contrast, H7xx chips use dual-key registers (KEYR1 and KEYR2) for two separate flash control registers (CR1 and CR2), requiring unlocking of both sections. Additionally, H7xx has a higher chance of failure if the flash is already in a non-recoverable locked state.

`ca_flash_hw_write`

Header: `caos/drivers/inc/ca_glue_hw.h:143`
GUARD: `EN_CA_GLUE_HW`

```
1 extern bool ca_flash_hw_write(u32 addr, const u8 *buf, u32 len); /*
     scrive len byte in FLASH a partire da addr (bloccante) */
```

Writes a block of data to the internal flash memory at a specified address.

Parametri:

Nome	Tipo	Descrizione
`addr`	`u32`	The target memory address in the internal flash. Must be aligned to the hardware-specific write unit (e.g., 16 or 32 bytes) depending on the MCU series.
`buf`	`const u8 *`	Pointer to the source data buffer. Must not be NULL.
`len`	`u32`	Number of bytes to write. If the length is not a multiple of the hardware write unit, the function pads the remaining bytes with 0xFF.

Ritorna:
Returns true if the write operation completed successfully, false if a hardware error occurred or parameters were invalid.

Note:

- This function is strictly blocking, meaning the CPU will wait until the flash hardware completes the write cycle before returning. It is NOT thread-safe; you must ensure that no other task or interrupt attempts to access the flash controller simultaneously. It is not safe to call this from an ISR because the blocking nature will cause significant latency and potential system instability. Before calling this, you must ensure the flash is unlocked (via ca_flash_unlock) and the target memory sector has been erased. A common beginner mistake is attempting to write to a flash address that has not been erased, which will result in a hardware error or corrupted data.

Uso:

- This function is used for persistent data storage, such as saving configuration parameters, calibration data, or security keys. The typical sequence is: 1) Unlock the flash controller, 2) Erase the target sector using ca_flash_erase_sector, 3) Call ca_flash_hw_write to store the data, and 4) Lock the flash controller to prevent accidental writes. It is often paired with ca_flash_unlock and ca_flash_lock to ensure the write operation is performed within a protected window.

Esempio:

```
/* Ensure the flash is unlocked before writing */
if (ca_flash_unlock()) {
    u8 data[] = {0xDE, 0xAD, 0xBE, 0xEF};
    /* Write 4 bytes to the specified flash address */
    if (ca_flash_hw_write(0x0800F000, data, sizeof(data))) {
        /* Success: data is now in flash */
    } else {
        /* Handle write failure (e.g., hardware error) */
    }
    /* Always lock the flash after the operation */
    ca_flash_lock();
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_flash_hw_write() in a caOS project?
A: You should use ca_flash_hw_write() when you need to permanently store data in the STM32's flash memory, such as configuration settings, calibration data, or firmware updates. It is a low-level function that directly writes to flash and is typically used in system initialization or data logging tasks where persistence is required.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_flash_hw_write()?
A:

```
u8 data[16] = {0x12, 0x34, 0x56, 0x78, 0x9A, 0xBC, 0xDE, 0xF0};
bool success = ca_flash_hw_write(0x08000000, data, 8);
if (!success) {
    // Handle write error
}
```

Parameters conf:0.90 **Q:** What are the valid ranges for the 'addr' parameter in ca_flash_hw_write()?
A: The 'addr' parameter must be a valid flash memory address. For F1xx/F4xx families, it must be aligned to 16-bit boundaries. For F7xx/H5xx/H7xx families, it must be aligned to 32-bit or larger boundaries (128/256-bit). The address must point to a valid flash region and cannot be NULL.

Return conf:0.95 **Q:** What does ca_flash_hw_write() return when it fails?
A: It returns false when the write operation fails. This can happen due to flash being busy, hardware errors, invalid buffer or address, or misalignment of the address or length parameters. The function does not distinguish between different error types but indicates failure through the boolean return value.

Error/Edge case conf:0.90 **Q:** What are the most common mistakes when using ca_flash_hw_write()?
A: Common mistakes include calling the function from an ISR (which is not allowed), not checking the return value, using non-aligned addresses or lengths, attempting to write too many bytes, and failing to unlock the flash before writing on H5xx/H7xx families. Also, calling this function in a multi-threaded context without proper synchronization can lead to race conditions.

Behavior conf:0.95 **Q:** Is ca_flash_hw_write() thread-safe and ISR-safe?
A: No, ca_flash_hw_write() is neither thread-safe nor ISR-safe. It is a blocking function that must not be called from an ISR. If used in a multi-threaded environment, you must

implement your own synchronization mechanism such as a mutex to prevent concurrent access.

HW differences conf:0.85 **Q:** How does ca_flash_hw_write() differ between F1xx/F4xx and H7xx families?
A: F1xx/F4xx families write in 16-bit or 32-bit chunks respectively, using standard flash registers like FLASH->CR and FLASH->SR. H7xx families use 256-bit (32-byte) write units and require separate control registers for each flash bank (CR1/CR2, SR1/SR2), with additional cache invalidation after writes. H7xx also requires waiting on QW (queue write) flags.

`ca_flash_init`

Header: `caos/drivers/inc/flash.h:93`
Source: `caos/drivers/src/flash.c:393:399`
GUARD: `EN_FLASH`

```
void ca_flash_init(void)
```

Initializes the flash driver's Command Line Interface (CLI) integration.

Ritorna:
This function returns nothing (void).

Note:

- This function is non-blocking and thread-safe because it uses a static flag to ensure the CLI registration happens exactly once. It is not intended to be called from an Interrupt Service Routine (ISR) as it interacts with the CLI subsystem which is designed for task-level execution. The mandatory precondition is that the CLI system must be initialized before this function is called; otherwise, the registration will fail or cause undefined behavior. A common beginner mistake is calling this function multiple times in different parts of the code; while the static flag prevents duplicate registration, it is best practice to call it only once during the system startup sequence to keep the code clean and predictable.

Uso:

- This function is used exclusively to expose flash-related diagnostic commands to the system's Command Line Interface. It is typically called during the early system initialization phase, immediately after the OS starts but before the main application tasks begin. It is often paired with other hardware-specific initialization functions like ca_flash_irq_init() to ensure the driver is fully prepared for operation. By calling this during the startup sequence, you ensure that developers can debug or inspect flash memory status via the serial console as soon as the system is powered on.

Esempio:

```
void ca_os_main(void) {
    /* Initialize the flash driver CLI node for debugging */
    ca_flash_init();

    /* Initialize flash interrupts for hardware operations */
    ca_flash_irq_init();

    /* Start the OS scheduler */
    ca_os_run(true);
}
```

Q&A — fine-tuning review *2/2 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When and why should I call ca_flash_init() in a caOS application?
A: You should call ca_flash_init() once during system startup to initialize the flash driver and register the associated CLI command, if CLI support is enabled. This function is non-blocking and can be called from any context (thread or ISR). It is intended as a one-time initialization function and should not be called multiple times. The function will do nothing if both EN_CLI_FLASH and ENABLE_CLI are disabled, making it safe to include in code regardless of CLI configuration.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_flash_init() with caOS types?
A: void example_flash_init(void) { ca_flash_init(); }

`ca_flash_irq_init`

Header: `caos/drivers/inc/flash.h:54`
Source: `caos/drivers/src/flash.c:55:61`
GUARD: `EN_FLASH`

```
void ca_flash_irq_init(void)
```

Initializes the internal semaphore and NVIC interrupts required for asynchronous Flash operations.

Ritorna:
This function returns void and does not provide a status code, as it performs internal system setup.

Note:

- This function is non-blocking and is intended to be called during the system configuration phase. It is not thread-safe in the sense that it should only be called once during initialization; calling it from multiple threads simultaneously could lead to race conditions on the global flag 'g_flash_irq_ready'. It is strictly forbidden to call this from an ISR, as it performs OS-level resource allocation (semaphore initialization). A common mistake is calling this before the hardware abstraction layer or the OS kernel is fully initialized, which will lead to a hard fault when the NVIC tries to enable interrupts for an unconfigured peripheral.

Uso:

- This function is used to prepare the Flash driver to operate in interrupt-driven mode rather than polling mode. It is typically called inside 'ca_os_configure' or during the early stages of the main application setup, immediately after 'ca_flash_init'. By initializing the semaphore 'g_flash_sem', it allows the system to block a task while waiting for a Flash operation to complete, effectively yielding CPU time to other threads. It must be called before any attempt to perform asynchronous Flash read/write operations that rely on interrupt completion signals.

Esempio:

```
void ca_os_configure(void) {
    // Initialize the basic Flash driver first
    ca_flash_init();

    // Prepare the IRQ infrastructure (semaphore and NVIC)
    // This allows the system to use interrupt-driven Flash operations
    ca_flash_irq_init();

```

```
    // Now it is safe to perform operations that might use the Flash IRQ
    // Example: ca_flash_write_irq(address, data, size);
}
```

Q&A — fine-tuning review *2/2 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When and why should I call ca_flash_irq_init() in a caOS project?
A: You should call ca_flash_irq_init() once during system initialization, typically at startup before any flash operations that require interrupt-driven handling. The function initializes the flash IRQ for use with semaphores and NVIC in interrupt-driven mode. It is a one-time setup function that ensures the flash interrupt is properly configured and prevents multiple initializations by setting a global flag.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_flash_irq_init() with caOS types?
A: void example_flash_init(void) { ca_flash_irq_init(); }
// This function would be called during system startup // No return value needed as it's void // The function ensures global variable g_flash_irq_ready is set // and prevents multiple initializations

---

`ca_flash_lock`

Header: `caos/drivers/inc/flash.h:37`
Source: `caos/drivers/src/flash.c:35:38`
GUARD: `EN_FLASH`

```
void ca_flash_lock(void)
```

Locks the internal Flash memory controller to prevent accidental write or erase operations.

Ritorna:
This function returns void and does not provide a status code, as locking the hardware is considered an atomic and guaranteed operation.

Note:

- This function is non-blocking and executes immediately by setting the lock bit in the Flash Control Register. It is thread-safe in the sense that it simply writes to a hardware register, but it does not provide mutual exclusion; if two tasks call this simultaneously, the hardware state remains locked. It is safe to call from an ISR, though rarely necessary. The most common beginner mistake is forgetting to call this after a write operation, which leaves the Flash memory vulnerable to accidental corruption from stray pointers or software bugs. Always ensure that you call this immediately after your write or erase sequence is complete to maintain system integrity.

Uso:

- This function is used as the final step in a Flash memory modification sequence. Typically, you follow a strict pattern: first call ca_flash_unlock to enable write access, perform your data write or sector erase operations, and finally call ca_flash_lock to re-enable the hardware protection. It is paired with ca_flash_unlock and functions like ca_flash_erase_sector or other write-related functions. Failing to lock the flash after use is a significant security and stability risk, as it allows any erroneous code execution to potentially overwrite your application code or configuration data.

Esempio:

```c
/* Sequence to safely erase a flash sector */
if (ca_flash_unlock()) {
    // Attempt to erase the sector at the specified address
    if (ca_flash_erase_sector(0x08004000)) {
        // Success: lock the flash immediately to protect memory
        ca_flash_lock();
    } else {
        // Handle erase failure, still lock to prevent accidental writes
        ca_flash_lock();
        // Log error: Erase failed
    }
} else {
    // Handle unlock failure: Flash might be permanently locked or hardware
    error
}
```

Q&A — fine-tuning review *2/2 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_flash_lock() and why is it important to use it with ca_flash_unlock()?
A: You should call ca_flash_lock() after performing flash write or erase operations to re-enable the hardware protection of the flash memory. It must be used in conjunction with ca_flash_unlock(), which should be called before any flash modification. Failing to call ca_flash_lock() after unlocking the flash can leave the flash in an unlocked state, preventing further access to the flash memory. The function is non-blocking and can be called from an ISR, but it is undefined behavior if called without first unlocking the flash.

Code example conf:0.85 **Q:** Can you provide a minimal C code example using caOS types that demonstrates how to properly use ca_flash_lock() and ca_flash_unlock()?
A: bool result = ca_flash_unlock(); if (result) { // Perform flash operations here ca_flash_lock(); }

`ca_flash_unlock`

Header: `caos/drivers/inc/flash.h:34`
Source: `caos/drivers/src/flash.c:30:33`
GUARD: `EN_FLASH`

```c
bool ca_flash_unlock(void)
```

Unlocks the internal flash memory controller to allow write or erase operations.

Ritorna:
Returns true if the flash controller was successfully unlocked, or false if the operation failed (e.g., due to hardware protection or invalid state).

Note:

- This function is a non-blocking wrapper that interacts directly with the hardware registers to disable write protection. It is not thread-safe by itself; if multiple tasks attempt to modify flash simultaneously, you must wrap this call in a mutex or semaphore to prevent data corruption. It is generally safe to call from an ISR, but you must ensure that no other task is currently performing a flash operation, as the hardware state is global. A common beginner mistake is forgetting to call ca_flash_lock() after the write operation is complete, which leaves the flash memory vulnerable to accidental corruption by stray pointers or software bugs.

Uso:

- This function is the mandatory first step in any flash programming sequence. You typically call it immediately before performing a write or erase operation, and it is almost always paired with ca_flash_lock() to restore the protection state once the task is finished. In a typical workflow, you would initialize the flash driver, call ca_flash_unlock(), perform your data write, and then immediately call ca_flash_lock() to ensure system stability. If you fail to unlock the flash before writing, the hardware will ignore your commands, and the data will not be saved.

Esempio:

```
/* Example: Unlocking flash to perform a configuration update */
if (ca_flash_unlock()) {
    /* Flash is now accessible for writing */
    // perform_flash_write(address, data);

    /* Always lock the flash again to prevent accidental corruption */
    ca_flash_lock();
} else {
    /* Handle error: flash could not be unlocked */
    // log_error("Failed to unlock flash");
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_flash_unlock() in my STM32 embedded application?
A: You should call ca_flash_unlock() before performing any flash write or erase operations. This function is required to gain access to the flash memory for modification. It must be called prior to any flash operation, and it's essential to check its return value to ensure the unlock was successful.

Code example conf:0.90 **Q:** Can you show me a minimal C code example using ca_flash_unlock() with caOS types?
A: bool success = ca_flash_unlock(); if (success) { // Proceed with flash write/erase operations } else { // Handle unlock failure }

Return conf:0.90 **Q:** What do the return values of ca_flash_unlock() indicate and what causes failures?
A: The function returns true if the flash unlock was successful, allowing subsequent flash operations to proceed. It returns false if the unlock fails, which can happen if the flash is already unlocked or due to hardware errors. Failures can silently cause flash operations to fail if the return value is not checked.

`ca_flash_write`

Header: `caos/drivers/inc/flash.h:46`
Source: `caos/drivers/src/flash.c:45:49`
GUARD: `EN_FLASH`

```
bool ca_flash_write(u32 addr, const u8 *buf, u32 len)
```

Writes a block of data to the internal flash memory at a specified address.

Parametri:

Nome	Tipo	Descrizione
addr	u32	The starting memory address in the flash where writing should begin. Must be aligned according to the hardware requirements (e.g., 4-byte alignment).
buf	const u8 *	Pointer to the source data buffer. Must not be NULL.
len	u32	The number of bytes to write. Must be greater than 0.

Ritorna:
Returns true if the write operation was successful, false if the input parameters are invalid (NULL pointer or zero length) or if the hardware write operation failed.

Note:

- This function is a wrapper around hardware-specific flash operations and is typically blocking, meaning the CPU will wait until the flash controller completes the write cycle. It is generally not thread-safe by itself; if multiple tasks attempt to write to flash simultaneously, you must protect this call with a mutex or semaphore to prevent data corruption. Do not call this from an Interrupt Service Routine (ISR) unless the underlying hardware driver explicitly supports it, as flash operations are slow and will block the system, potentially causing missed interrupts. A common beginner mistake is attempting to write to a flash sector without erasing it first, which will result in a failed write or corrupted data because flash memory can only transition bits from 1 to 0.

Uso:

- This function is used for persistent data storage, such as saving configuration parameters or calibration values that must survive a power cycle. It is typically used in a sequence: first, you must call ca_flash_erase_sector to clear the target memory area, then call ca_flash_write to store the new data. It is often paired with a verification step where the written data is read back to ensure integrity. In a typical application, this is called during system initialization or when a user triggers a 'Save Settings' command.

Esempio:

```
u32 target_addr = 0x0800F000; // Example flash sector address
u8 config_data[] = {0x01, 0x02, 0x03, 0x04};

// 1. Erase the sector before writing
if (ca_flash_erase_sector(target_addr)) {
    // 2. Write the data to the erased sector
    if (ca_flash_write(target_addr, config_data, sizeof(config_data))) {
        // Success: Data saved
    } else {
        // Error: Write failed
    }
} else {
    // Error: Erase failed
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_flash_write in a caOS application?
A: You should use ca_flash_write to write data to the STM32 flash memory, typically for storing configuration data, logs, or other persistent information. It is a blocking function meant to be called from a task context, not from an ISR, and should only be used when you need to ensure data persistence across power cycles.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_flash_write with caOS types?
A: u8 data[10] = {0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08, 0x09, 0x0A}; bool success = ca_flash_write(0x08000000, data, 10);

Parameters conf:0.90 **Q:** What are the valid ranges for the address and length parameters in ca_flash_write?
A: The address parameter must be aligned to a flash page/sector boundary, and the length must be greater than zero. The buffer pointer cannot be NULL. Invalid values include an address that is not page-aligned, a length of zero, or a NULL buffer pointer.

Return conf:0.95 **Q:** What does ca_flash_write return and what causes it to return false?
A: ca_flash_write returns true on successful write or false on failure. It returns false if the parameters are invalid (e.g., address not aligned, length <= 0, or buffer is NULL) or if there is a hardware error during the flash write operation.

`ca_flash_write_irq`

Header: `caos/drivers/inc/flash.h:60`
Source: `caos/drivers/src/flash.c:82:103`
GUARD: `EN_FLASH`

```
bool ca_flash_write_irq(u32 addr, const u8 *buf, u32 len)
```

Writes data to flash memory using an interrupt-driven approach to avoid blocking the CPU during hardware operations.

Parametri:

Nome	Tipo	Descrizione
`addr`	`u32`	The starting memory address in the flash to write to. Must be aligned according to hardware requirements.
`buf`	`const u8 *`	Pointer to the source data buffer. Must not be NULL and must contain at least 'len' bytes.
`len`	`u32`	The number of bytes to write. Must be greater than 0.

Ritorna:
Returns true if the entire buffer was written successfully, false if the operation failed, the input parameters were invalid, or the flash hardware reported an error.

Note:

- This function is blocking because it uses ca_sem_wait to wait for the hardware interrupt to signal completion of each write chunk. It is NOT safe to call from an ISR because it relies on a semaphore to synchronize with the flash hardware interrupt. Before calling this, ensure the flash driver is initialized and the global ready flag is set. A common mistake is passing a buffer that is not valid for the entire duration of the write, or failing to check the return value, which leads to silent data corruption if the flash write fails.

Uso:

- This function is typically used when updating configuration parameters or logging data to non-volatile memory without freezing the entire system. It is usually paired with a preceding check of the flash status and followed by a verification step to ensure data integrity. In the typical lifecycle, you initialize the flash driver, call ca_flash_write_irq to commit data, and then perform any necessary cleanup or state updates. It is designed for

scenarios where the CPU should perform other tasks while the hardware handles the slow flash programming process.

Esempio:

```c
/* Ensure the flash system is ready before attempting a write */
if (g_flash_irq_ready) {
    const u8 data[] = {0xDE, 0xAD, 0xBE, 0xEF};
    u32 target_addr = 0x0800F000;

    /* Perform the write operation; this will block until the hardware signals completion */
    if (ca_flash_write_irq(target_addr, data, sizeof(data))) {
        /* Success: data is now in flash */
    } else {
        /* Handle error: log the failure or retry */
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_flash_write_irq instead of a synchronous flash write function?
A: You should use ca_flash_write_irq when you want to write to flash without blocking the calling thread. It allows the thread to perform other tasks while the flash write operation completes in the background, making it suitable for RTOS environments where responsiveness is important.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_flash_write_irq with caOS types?
A: u8 data[4] = {0x12, 0x34, 0x56, 0x78}; bool success = ca_flash_write_irq(0x08001000, data, 4);

Parameters conf:0.90 **Q:** What are the valid ranges for the address parameter in ca_flash_write_irq?
A: The address must be within the range 0x08000000 to 0x080FFFFF, which corresponds to the main flash memory region of STM32 microcontrollers. It must also be properly aligned (e.g., a multiple of 4 for 32-bit writes). Values outside this range or zero will cause the function to return false.

Return conf:0.95 **Q:** What does it mean when ca_flash_write_irq returns false?
A: A return value of false indicates an error occurred during the flash write operation. Possible causes include a NULL buffer, zero length, flash not being ready, hardware errors, or failure to lock the flash block.

Error/Edge case conf:0.90 **Q:** What are the most common mistakes when calling ca_flash_write_irq?
A: Common mistakes include calling the function with a NULL buffer, passing a zero length, using it from an ISR (which is not allowed), failing to initialize the flash driver properly, or not checking the return value for errors.

Behavior conf:0.90 **Q:** Is ca_flash_write_irq thread-safe and can it be called from an ISR?
A: Yes, ca_flash_write_irq is thread-safe because it uses hardware locks and semaphores. However, it is not safe to call from an ISR. It is designed to be used only from RTOS tasks and will block the calling thread until completion, but not in a way that prevents other tasks from running.

`ca_fmac_flags`

Header: `caos/drivers/inc/fmac.h:67`
Source: `caos/drivers/src/fmac.c:93:96`
GUARD: `EN_FMAC`

```
u32 ca_fmac_flags(void); /* CA_FMAC_* */
```

Retrieves the current status and error flags from the FMAC (Filter Math Accelerator) hardware.

Ritorna:
Returns a 32-bit unsigned integer representing the hardware status flags. Returns 0 if the FMAC peripheral is not initialized or currently inactive.

Note:

- This function is non-blocking and thread-safe, as it performs a simple read of the internal hardware state. It is safe to call from an ISR, provided the FMAC peripheral has been previously initialized via ca_fmac_init. The most common mistake is calling this function before initializing the FMAC or after calling ca_fmac_stop, which will result in a return value of 0, potentially masking actual hardware errors if the user assumes 0 means 'success'.

Uso:

- This function is typically used immediately after a data processing operation (like ca_fmac_run) to verify if the hardware encountered any overflow, underflow, or configuration errors during the calculation. It is part of the standard FMAC lifecycle: initialize with ca_fmac_init, start the operation with ca_fmac_start, process data with ca_fmac_run, check the result status with ca_fmac_flags, and finally clean up with ca_fmac_stop. It acts as a diagnostic tool to ensure the integrity of the mathematical operations performed by the accelerator.

Esempio:

```
/* Initialize the FMAC peripheral before use */
ca_fmac_init();

/* Start the FMAC hardware */
ca_fmac_start();

/* Perform a calculation on buffers */
/* ca_fmac_run(input_buffer, output_buffer, length); */

/* Retrieve status flags to check for errors */
u32 status = ca_fmac_flags();

if (status != 0U) {
    /* Handle hardware error flags here */
}

/* Stop the hardware when finished */
ca_fmac_stop();
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_fmac_flags in my application logic?
A: You should call ca_fmac_flags immediately after a data processing operation, such as ca_fmac_run, to verify the integrity of the calculation. It serves as a diagnostic tool to detect overflow, underflow, or configuration errors that may have occurred during the FMAC operation.

Code example conf:1.00 **Q:** How do I check the FMAC status flags in my code?
A: You can retrieve the status by calling the function and storing the result in a u32 variable. Example: u32 status = ca_fmac_flags(); if (status != 0) { /* handle errors */ }

Return conf:1.00 **Q:** What does a return value of 0 from ca_fmac_flags signify?

A: A return value of 0 indicates that the FMAC peripheral is either not initialized or currently inactive. Note that this can be misleading, as it might mask actual hardware errors if the developer incorrectly assumes 0 represents a successful operation status.

---

ca_fmac_hw_flags

Header: caos/drivers/inc/ca_glue_hw.h:410
GUARD: EN_CA_GLUE_HW

```
extern u32 ca_fmac_hw_flags(void); /* SAT/OVFL/UNFL as CA_FMAC_* */
```

Reads the hardware status flags (Saturation, Overflow, Underflow) from the FMAC peripheral.

Ritorna:
A bitmask of flags: CA_FMAC_SAT (saturation), CA_FMAC_OVFL (overflow), or CA_FMAC_UNFL (underflow). Returns 0 if no flags are set or if the hardware platform does not support FMAC (e.g., F1xx, F4xx, F7xx, H7xx).

Note:

- This function is non-blocking and performs a direct read of the hardware status register. It is thread-safe for reading, but it does not provide any locking mechanism for the peripheral itself, meaning other tasks could potentially modify FMAC settings simultaneously. It is safe to call from an ISR because it does not involve OS primitives or blocking waits. A common mistake is failing to check if the FMAC peripheral is actually enabled or initialized before calling this, as reading registers on an unclocked or uninitialized peripheral can lead to unpredictable behavior or hard faults on some STM32 architectures.

Uso:

- This function is used to monitor the health and accuracy of mathematical operations performed by the FMAC unit. It is typically called after a calculation sequence, such as after ca_fmac_run, to verify if the result is valid or if numerical errors occurred. It is intended to be used in conjunction with the higher-level ca_fmac_flags() wrapper, which handles the internal state check (s_fmac_ok) to ensure the peripheral is ready before accessing the hardware registers directly.

Esempio:

```
/* Check FMAC status after a computation */
u32 status = ca_fmac_flags();

if (status & CA_FMAC_SAT) {
    /* Handle saturation error: the result exceeded the representable range */
    ca_printf("FMAC Error: Saturation detected!\n");
}

if (status & (CA_FMAC_OVFL | CA_FMAC_UNFL)) {
    /* Handle overflow or underflow */
    ca_printf("FMAC Error: Numerical overflow/underflow!\n");
}

if (status == 0) {
    ca_printf("FMAC operation completed successfully.\n");
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to call ca_fmac_hw_flags in my signal processing loop?
A: You should call ca_fmac_hw_flags after a calculation sequence, specifically after invoking ca_fmac_run or ca_fmac_start. This allows you to verify the integrity of the FMAC output and detect if the mathematical operation resulted in saturation, overflow, or underflow conditions.

Code example conf:1.00 **Q:** Provide a minimal example of how to check for FMAC errors using caOS types.
A: u32 flags = ca_fmac_hw_flags(); if (flags & (CA_FMAC_SAT | CA_FMAC_OVFL | CA_FMAC_UNFL)) { /* Handle error condition */ }

Return conf:1.00 **Q:** What do the return values of ca_fmac_hw_flags represent, and what happens if I run this on an unsupported platform?
A: The function returns a bitmask where CA_FMAC_SAT indicates saturation, CA_FMAC_OVFL indicates overflow, and CA_FMAC_UNFL indicates underflow. If no flags are set, it returns 0. On hardware platforms that do not support FMAC, such as F1xx, F4xx, F7xx, or H7xx, the function will always return 0.

HW differences conf:1.00 **Q:** How does the implementation of ca_fmac_hw_flags differ across STM32 families?
A: On H5xx and U5xx families, the function performs a direct read of the FMAC->SR register and maps the hardware bits to caOS-specific flags. Conversely, for F1xx, F4xx, F7xx, and H7xx families, the function is implemented as a stub that returns 0U, reflecting the lack of FMAC peripheral support on those specific architectures.

`ca_fmac_hw_init`

Header: `caos/drivers/inc/ca_glue_hw.h:403`
GUARD: `EN_CA_GLUE_HW`

```
1 extern bool ca_fmac_hw_init(void); /* clock on, unit reset; false when
     the part has no FMAC */
```

Initializes the hardware FMAC (Filter Math Accelerator) unit by enabling its clock and resetting the peripheral.

Ritorna:
Returns true if the FMAC hardware is present and successfully initialized, or false if the specific MCU model does not support the FMAC peripheral.

Note:

- This function is a low-level hardware abstraction layer (HAL) call. It is non-blocking and executes quickly, but it is not thread-safe; it must be called during the system initialization phase before any tasks attempt to use the FMAC. It is not intended to be called from an ISR. A common mistake is assuming that all STM32 chips have an FMAC unit; always check the return value to ensure the hardware is actually available before proceeding with filter operations.

Uso:

- This function is typically called once during the system boot sequence or driver initialization phase. It is paired with the higher-level ca_fmac_init function, which acts as a wrapper to store the initialization status in a static variable. Once ca_fmac_hw_init returns true, the application can safely proceed to call ca_fmac_start to configure filters and ca_fmac_run to process data.

Esempio:

```c
/* Initialize the FMAC driver during system startup */
#include "caos.h"

void system_init(void) {
    /* Attempt to initialize the FMAC hardware */
    if (ca_fmac_init()) {
        /* FMAC is available and ready for use */
        /* Proceed to configure filters using ca_fmac_start() */
    } else {
        /* FMAC not supported on this hardware, handle error */
        ca_printf("FMAC hardware not found!\n");
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_fmac_hw_init in my caOS application?
A: You should call it once during the system boot sequence or driver initialization phase. It must be called before any tasks attempt to use the FMAC, as it is not thread-safe and is not intended to be called from an ISR.

Code example conf:1.00 **Q:** Provide a minimal example of how to initialize the FMAC unit safely.
A: bool status = ca_fmac_hw_init(); if (status) { /* Proceed to ca_fmac_start and ca_fmac_run */ } else { /* Handle hardware not present */ }

Return conf:1.00 **Q:** What does the return value of ca_fmac_hw_init indicate?
A: It returns true if the FMAC hardware is present and successfully initialized. It returns false if the specific MCU model does not support the FMAC peripheral, which is common as not all STM32 chips include this unit.

HW differences conf:1.00 **Q:** How does the implementation of ca_fmac_hw_init differ across STM32 families?
A: On H5xx and U5xx families, the function enables the peripheral via RCC->AHB1ENR and resets it using FMAC->CR = FMAC_CR_RESET, returning true. On F1xx, F4xx, F7xx, and H7xx families, the function simply returns false, indicating the FMAC peripheral is not supported on those specific architectures.

`ca_fmac_hw_param`

Header: `caos/drivers/inc/ca_glue_hw.h:406`
GUARD: `EN_CA_GLUE_HW`

```c
extern void ca_fmac_hw_param(u32 func, u32 p, u32 q, u32 r); /*
    FMAC_PARAM with START: load or filter function */
```

Configures the FMAC hardware parameters and triggers a processing operation.

Parametri:

Nome	Tipo	Descrizione
`func`	`u32`	The specific FMAC function to execute (e.g., FMAC_LOAD_X1, FMAC_LOAD_X2). Must be a valid constant defined in the FMAC driver header.
`p`	`u32`	Primary parameter value, typically representing buffer sizes or coefficient counts.
`q`	`u32`	Secondary parameter value, often used for additional coefficient counts or state offsets.
`r`	`u32`	Tertiary parameter value, used for specific hardware configuration flags or additional offsets.

Ritorna:
void. This function does not return a status code; it writes directly to the hardware register.

Note:

- This function is a low-level hardware abstraction. It is non-blocking and executes immediately by writing to the FMAC peripheral registers. It is thread-safe only if the FMAC peripheral is not being accessed by multiple tasks simultaneously; ensure you have exclusive access to the FMAC hardware before calling this. It is generally safe to call from an ISR if the hardware is dedicated to that context, but be aware that it does not perform any validation on the parameters. The most common beginner mistake is passing incorrect buffer sizes or function codes that do not match the hardware's current state, which can lead to undefined behavior or hardware faults. Note that on some platforms (f1xx, f4xx, f7xx, h7xx), this function is a no-op, meaning it does nothing; always check your target hardware implementation.

Uso:

- This function is used to prepare the FMAC hardware for signal processing tasks. It is typically called after initializing the peripheral with ca_fmac_init and before starting the processing loop with ca_fmac_run. It is often paired with low-level hardware write operations to load coefficients or data into the FMAC buffers. The sequence is usually: initialize the peripheral, set parameters via this function, write the required data, and finally trigger the processing run.

Esempio:

```c
/* Example: Configuring FMAC to load filter coefficients */
#define FMAC_LOAD_X2 0x01U

void setup_filter(void) {
    u32 nb = 5U; /* Number of b coefficients */
    u32 na = 3U; /* Number of a coefficients */

    /* Initialize the FMAC peripheral first */
    ca_fmac_init();

    /* Configure FMAC to load coefficients into X2 buffer */
    /* func=FMAC_LOAD_X2, p=nb, q=na, r=0 */
    ca_fmac_hw_param(FMAC_LOAD_X2, nb, na, 0U);

    /* Write the coefficients to the hardware */
    /* Note: Use hardware-specific write function to transfer data */
    /* ca_fmac_hw_write(b_coeffs, nb); */
    /* ca_fmac_hw_write(a_coeffs, na); */

    /* Start the processing operation */
    ca_fmac_run();
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** At what point in the caOS FMAC processing sequence should I call ca_fmac_hw_param?
A: You should call ca_fmac_hw_param after initializing the peripheral with ca_fmac_init and before starting the processing loop with ca_fmac_start. It is typically used to set parameters before writing data or coefficients to the buffers using ca_fmac_hw_write.

Code example conf:1.00 **Q:** Provide a minimal C example of how to configure the FMAC hardware using caOS types.
A: u32 func = FMAC_LOAD_X1; u32 p = 16; u32 q = 0; u32 r = 0; ca_fmac_hw_param(func, p, q, r);

Parameters conf:1.00 **Q:** What do the parameters p, q, and r represent in ca_fmac_hw_param, and are they validated by the function?
A: The parameters p, q, and r are used for buffer sizes, coefficient counts, state offsets, or hardware configuration flags. The function does not perform any validation on these parameters; passing incorrect values can lead to undefined behavior or hardware faults.

HW differences conf:1.00 **Q:** Does ca_fmac_hw_param behave the same way across all STM32 families supported by caOS?
A: No. On the H5xx and U5xx families, the function directly writes to the FMAC->PARAM register to trigger hardware operations. On the F1xx, F4xx, F7xx, and H7xx families, the function is currently a stub that performs no operations, as indicated by the (void) casting of all parameters.

`ca_fmac_hw_run`

Header: `caos/drivers/inc/ca_glue_hw.h:408`
GUARD: `EN_CA_GLUE_HW`

```
1 extern u32 ca_fmac_hw_run(const i16 *in, i16 *out, u32 len); /* polled
     stream, outputs read; < len on a stall */
```

Processes a stream of audio samples through the hardware FMAC (Filter Math Accelerator) unit.

Parametri:

Nome	Tipo	Descrizione
`in`	`const i16 *`	Pointer to the input buffer containing raw audio samples. Must not be NULL.
`out`	`i16 *`	Pointer to the output buffer where processed samples will be stored. Must not be NULL.
`len`	`u32`	Number of samples to process. Valid values are > 0.

Ritorna:
Returns the number of samples successfully processed and written to the output buffer. If the return value is less than 'len', it indicates a hardware stall or timeout occurred during processing.

Note:

- This function is a blocking, polled implementation that waits for the hardware registers to clear before feeding new data. It is NOT thread-safe and must not be called from an ISR, as it uses a spin-loop (FMAC_SPIN_MAX) to wait for hardware readiness. Before calling this, the FMAC peripheral must be initialized via ca_fmac_init and started via ca_fmac_start. A common beginner mistake is passing a NULL pointer or failing to verify that the FMAC is actually running, which will cause the function to return 0 immediately.

On some hardware platforms (f1xx, f4xx, f7xx, h7xx), this function is a stub that returns 0, meaning the hardware does not support FMAC operations.

Uso:

- This function is typically used in a dedicated processing task or a high-priority loop where audio filtering is required. The standard sequence is: call ca_fmac_init to configure the filter coefficients, call ca_fmac_start to enable the hardware, and then repeatedly call ca_fmac_run within a loop to process incoming audio buffers. Once processing is complete or the task is shutting down, call ca_fmac_stop to safely disable the peripheral. Always check the return value against 'len' to ensure the hardware kept up with the data stream.

Esempio:

```
/* Assuming FMAC is already initialized and started */
#define BUFFER_SIZE 128
i16 input_buffer[BUFFER_SIZE];
i16 output_buffer[BUFFER_SIZE];

/* Fill input_buffer with data from a source */

/* Process the audio stream using the wrapper function */
u32 processed = ca_fmac_run(input_buffer, output_buffer, BUFFER_SIZE);

if (processed < BUFFER_SIZE) {
    /* Handle hardware stall or error: log the discrepancy */
    ca_printf("FMAC stall: only processed %u samples\n", processed);
} else {
    /* Data is ready in output_buffer */
}
```

Q&A — fine-tuning review *7/7 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_fmac_hw_run in my caOS application?
A: You should call ca_fmac_hw_run within a dedicated processing task or high-priority loop after initializing the peripheral with ca_fmac_init and starting it with ca_fmac_start. It is used to process audio samples through the hardware FMAC unit.

Code example conf:1.00 **Q:** Provide a minimal example of how to process a buffer of audio samples using ca_fmac_hw_run.
A: i16 input[128]; i16 output[128]; u32 processed; ca_fmac_init(); ca_fmac_start(); processed = ca_fmac_hw_run(input, output, 128);

Parameters conf:1.00 **Q:** What are the constraints on the 'len' parameter for ca_fmac_hw_run?
A: The 'len' parameter represents the number of samples to process and must be a value greater than 0.

Return conf:1.00 **Q:** What does it mean if ca_fmac_hw_run returns a value less than the requested length?
A: A return value less than 'len' indicates that a hardware stall or timeout occurred during the processing of the audio stream.

Error/Edge case conf:1.00 **Q:** What are common mistakes that cause ca_fmac_hw_run to return 0 immediately?
A: Common mistakes include passing a NULL pointer for the input or output buffers, or failing to verify that the FMAC peripheral has been properly started via ca_fmac_start before calling the run function.

Behavior conf:1.00 **Q:** Is ca_fmac_hw_run safe to call from an interrupt service routine (ISR)?
A: No, it is not ISR-safe. The function is a blocking, polled implementation that uses a spin-loop (FMAC_SPIN_MAX) to wait for hardware readiness, and it is not thread-safe.

HW differences conf:1.00 **Q:** Are there differences in how ca_fmac_hw_run is implemented across different STM32 families?
A: Yes. On H5xx and U5xx families, the function performs active register polling (FMAC->SR, FMAC->WDATA, FMAC->RDATA) to process data. On F1xx, F4xx, F7xx, and H7xx families, the function is currently a stub that returns 0, indicating the hardware FMAC implementation is not supported or active for those specific targets.

`ca_fmac_hw_setup`

Header: `caos/drivers/inc/ca_glue_hw.h:404`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_fmac_hw_setup(u32 x1_base, u32 x1_size, u32 x2_base, u32
    x2_size, u32 y_base, u32 y_size, bool clip); /* reset, buffer layout,
     watermarks 1, CLIPEN */
```

Configures the FMAC (Filter Math Accelerator) hardware buffer layout and clipping settings.

Parametri:

Nome	Tipo	Descrizione
`x1_base`	`u32`	Base address offset for the X1 input buffer.
`x1_size`	`u32`	Size of the X1 input buffer.
`x2_base`	`u32`	Base address offset for the X2 coefficient buffer.
`x2_size`	`u32`	Size of the X2 coefficient buffer.
`y_base`	`u32`	Base address offset for the Y output buffer.
`y_size`	`u32`	Size of the Y output buffer.
`clip`	`bool`	If true, enables output clipping to prevent overflow.

Ritorna:
void. This function does not return a status code; it assumes the hardware is accessible.

Note:

- This function is non-blocking and performs direct register writes to the FMAC peripheral. It is NOT thread-safe; you must ensure no other task or ISR is accessing the FMAC registers simultaneously. It must be called while the FMAC is in a stopped state (START bit = 0), as the buffer registers are locked during operation. A common beginner mistake is calling this while the FMAC is actively processing data, which will result in undefined hardware behavior or corrupted filter outputs. Note that on some platforms (e.g., F1xx, F4xx, F7xx, H7xx), this function is a no-op, so always check your target hardware capabilities.

Uso:

- This function is the primary configuration step for the FMAC peripheral. It is typically called after initializing the FMAC hardware and before loading coefficients or starting the processing loop. The standard sequence is: 1) Call ca_fmac_init to prepare the peripheral, 2) Call ca_fmac_hw_setup to define the memory layout for input/output buffers, 3) Load coefficients using hardware-specific write functions, and 4) Call ca_fmac_start to begin processing. It is used to define the memory map for the filter's circular buffers, ensuring the hardware knows exactly where to read input samples and where to store the resulting filtered data.

Esempio:

```c
/* Initialize FMAC and configure buffers for a filter operation */
ca_fmac_init();

/* Define buffer layout: X1 (input), X2 (coefficients), Y (output) */
/* Assuming base addresses and sizes are calculated based on filter order */
u32 x1_b = 0x00, x1_s = 32;
u32 x2_b = 32,   x2_s = 16;
u32 y_b  = 48,   y_s  = 32;

/* Configure hardware with clipping enabled */
ca_fmac_hw_setup(x1_b, x1_s, x2_b, x2_s, y_b, y_s, true);

/* Proceed to load coefficients and start the filter */
/* Call function to load filter parameters into FMAC registers */
/* Call function to write coefficients into the X2 buffer */
ca_fmac_start();
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** What is the correct sequence for initializing the FMAC peripheral in caOS?
A: The standard sequence is: 1) Call ca_fmac_init to prepare the peripheral, 2) Call ca_fmac_hw_setup to define the memory layout for input/output buffers, 3) Load coefficients using hardware-specific write functions, and 4) Call ca_fmac_start to begin processing.

Code example conf:1.00 **Q:** How do I call ca_fmac_hw_setup to configure buffers with clipping enabled?
A: You would call it as follows: u32 x1_base = 0x00; u32 x1_size = 16; u32 x2_base = 0x10; u32 x2_size = 16; u32 y_base = 0x20; u32 y_size = 16; ca_fmac_hw_setup(x1_base, x1_size, x2_base, x2_size, y_base, y_size, true);

Parameters conf:1.00 **Q:** What do the x1_base and x1_size parameters represent in ca_fmac_hw_setup?
A: x1_base is the base address offset for the X1 input buffer, and x1_size is the size of that same X1 input buffer. These parameters are used to define the memory map for the filter's circular buffers.

Error/Edge case conf:1.00 **Q:** What happens if I call ca_fmac_hw_setup while the FMAC is already running?
A: This is a common beginner mistake. The buffer registers are locked during operation, so calling this while the FMAC is actively processing data will result in undefined hardware behavior or corrupted filter outputs. You must ensure the FMAC is in a stopped state (START bit = 0) before calling this function.

HW differences conf:1.00 **Q:** Does ca_fmac_hw_setup behave the same way on all STM32 families supported by caOS?
A: No. On platforms like F1xx, F4xx, F7xx, and H7xx, this function is a no-op. Only on specific families like H5xx and U5xx does the function perform direct register writes to the FMAC peripheral (e.g., configuring X1BUFCFG, X2BUFCFG, and YBUFCFG registers).

`ca_fmac_hw_stop`

Header: `caos/drivers/inc/ca_glue_hw.h:409`
GUARD: `EN_CA_GLUE_HW`

```c
extern void ca_fmac_hw_stop(void); /* START off */
```

Immediately halts the FMAC (Filter Math Accelerator) hardware peripheral.

Ritorna:

void

Note:

- This function is a low-level hardware abstraction that directly clears the START bit in the FMAC peripheral register. It is non-blocking and executes almost instantaneously, but it does not wait for any ongoing hardware calculations to finish; it simply cuts off the operation. It is thread-safe in the sense that it performs a single atomic register write, but it should not be called from an ISR unless you are certain the hardware state is consistent. The most common beginner mistake is calling this function without first ensuring the system is in a state where an abrupt stop is safe, potentially leaving the FMAC in an undefined state for subsequent operations.

Uso:

- This function is typically used as part of a clean-up or error-handling sequence to ensure the FMAC is not processing data when it shouldn't be. It is usually paired with ca_fmac_init for setup and ca_fmac_start or ca_fmac_run to initiate processing. In a typical application, you would call ca_fmac_stop when a task is being suspended, when a calculation is aborted by the user, or during a system reset sequence to ensure the hardware is idle before re-initialization.

Esempio:

```c
/* Example: Safely stopping the FMAC peripheral before re-configuration */

void safe_fmac_reset(void) {
    /* Check if the FMAC is currently active using the flags function */
    if (ca_fmac_flags() != 0U) {
        /* Stop the hardware immediately */
        ca_fmac_stop();

        /* Perform additional logic to clear buffers or reset state */
        /* Call a function to release memory buffers if they were dynamically allocated */
    }

    /* Re-initialize the FMAC for a new operation */
    ca_fmac_init();
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_fmac_hw_stop in my caOS application?
A: You should call it during clean-up or error-handling sequences, such as when a task is suspended, a calculation is aborted by the user, or during a system reset sequence to ensure the hardware is idle before re-initialization.

Code example conf:0.90 **Q:** Provide a minimal example of how to safely stop the FMAC peripheral using caOS.
A: To stop the FMAC, simply invoke the function directly: ca_fmac_hw_stop();. Ensure that your system state allows for an abrupt stop, as this function does not wait for ongoing calculations to complete.

HW differences conf:1.00 **Q:** Are there differences in how ca_fmac_hw_stop is implemented across STM32 families?
A: Yes. For the H5xx and U5xx families, the function performs an explicit register operation by clearing the FMAC_PARAM_START bit in the FMAC->PARAM register. In contrast, the implementations for F1xx, F4xx, F7xx, and H7xx are currently empty stubs.

ca_fmac_hw_write

Header: caos/drivers/inc/ca_glue_hw.h:407
GUARD: EN_CA_GLUE_HW

```
extern void ca_fmac_hw_write(const i16 *data, u32 n); /* n words into
    WDATA, zeros when data is NULL */
```

Writes a sequence of 16-bit data words into the FMAC hardware peripheral.

Parametri:

Nome	Tipo	Descrizione
data	const i16 *	Pointer to the array of 16-bit values to write. If NULL, the function writes zeros to the hardware.
n	u32	Number of 16-bit words to write. Must match the expected hardware buffer size.

Ritorna:
void

Note:

- This function performs direct memory-mapped I/O writes to the FMAC peripheral. It is non-blocking and executes synchronously, meaning the CPU remains occupied until the entire loop completes. It is NOT thread-safe; if multiple tasks or an ISR and a task attempt to write to the FMAC simultaneously, the hardware state will be corrupted. It is callable from an ISR, but keep in mind that long loops will block other interrupts, potentially causing system latency. The most common beginner mistake is providing an incorrect 'n' value, which leads to buffer underflows or overflows in the FMAC registers, resulting in incorrect filter output or hardware stalls.

Uso:

- This function is used during the configuration phase of the FMAC peripheral to load coefficients or initial state data. It is typically called after initializing the peripheral and setting the required parameters (e.g., filter length). The typical sequence is: call 'ca_fmac_init' to prepare the hardware, use a function to set the specific FMAC parameters (like buffer size and operation mode), call 'ca_fmac_hw_write' to load the data, and finally trigger the processing with 'ca_fmac_start'.

Esempio:

```
/* Example: Loading filter coefficients into FMAC */
#include "caos.h"

void configure_filter(const i16 *coeffs, u32 num_coeffs) {
    /* Initialize the FMAC peripheral first */
    ca_fmac_init();

    /* Set hardware parameters (e.g., load X2 buffer) */
    /* ca_fmac_hw_param(FMAC_LOAD_X2, num_coeffs, 0U, 0U); */

    /* Write the coefficients into the hardware */
    /* If coeffs is NULL, the driver writes zeros to clear the buffer */
    ca_fmac_hw_write(coeffs, num_coeffs);

    /* Start the filter operation */
    ca_fmac_start();
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** At what point in the caOS FMAC configuration sequence should I call ca_fmac_hw_write?
A: You should call ca_fmac_hw_write after initializing the peripheral with ca_fmac_init and setting parameters with ca_fmac_hw_param. It is used to load filter coefficients or initial state data into the hardware registers before triggering the operation with ca_fmac_start.

Code example conf:1.00 **Q:** Provide a minimal example of how to use ca_fmac_hw_write to load 10 coefficients into the FMAC.
A: i16 my_coeffs[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}; u32 n = 10; ca_fmac_hw_write(my_coeffs, n);

Parameters conf:1.00 **Q:** What happens if I pass a NULL pointer to the data parameter in ca_fmac_hw_write?
A: If the data pointer is NULL, the function will write zeros to the FMAC hardware peripheral for the specified number of words (n).

HW differences conf:1.00 **Q:** Are there differences in how ca_fmac_hw_write is implemented across different STM32 families?
A: Yes. For the H5xx and U5xx families, the function performs a direct write loop to the FMAC->WDATA register. For the F1xx, F4xx, F7xx, and H7xx families, the function is currently implemented as a stub that performs no operations, as indicated by the (void)data; (void)n; casting.

`ca_fmac_init`

Header: `caos/drivers/inc/fmac.h:63`
Source: `caos/drivers/src/fmac.c:30:34`
GUARD: `EN_FMAC`

```
bool ca_fmac_init(void); /* false: no FMAC on this part */
```

Initializes the Filter Math Accelerator (FMAC) hardware peripheral.

Ritorna:
Returns true if the FMAC hardware is present and successfully initialized, false otherwise.

Note:

- This function is a hardware abstraction layer (HAL) wrapper that checks for the existence of the FMAC peripheral on the specific STM32 chip being used. It is not thread-safe and must be called during the system initialization phase before any other FMAC operations are attempted. It is not intended to be called from an ISR. The most common mistake is failing to check the return value; if the function returns false, any subsequent calls to FMAC functions will likely result in undefined behavior or hard faults because the hardware registers do not exist on that specific silicon.

Uso:

- This function is typically called once during the system startup sequence, usually inside the ca_os_configure function. It acts as a gatekeeper: if it returns false, the application should gracefully disable features that rely on the FMAC (like digital filtering or signal processing) to prevent system crashes. It is often paired with ca_cordic_init to verify that all required math acceleration hardware is available before the main application logic starts.

Esempio:

```c
void system_setup(void) {
    /* Attempt to initialize the FMAC hardware */
    if (ca_fmac_init()) {
        /* FMAC is available, safe to proceed with filter configuration */
        /* ca_fmac_start(...) would be called here if needed */
    } else {
        /* FMAC is not present on this MCU, log error or fallback to software math */
        ca_printf("Warning: FMAC hardware not detected, using software fallback.\n");
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_fmac_init and why is it critical for system stability?
A: ca_fmac_init should be called once during the system initialization phase, typically inside ca_os_configure. It is critical because it acts as a gatekeeper; if it returns false, the application must disable features relying on the FMAC to prevent undefined behavior or hard faults caused by accessing non-existent hardware registers.

Code example conf:1.00 **Q:** How do I implement a safe initialization check for the FMAC in my caOS application?
A: You should check the boolean return value before proceeding with any math acceleration logic:
if (ca_fmac_init()) { /* FMAC hardware is present, safe to proceed */ } else { /* FMAC not available, disable signal processing features */ }

Return conf:1.00 **Q:** What does the return value of ca_fmac_init indicate, and why might it return false?
A: The function returns true if the FMAC hardware is present and successfully initialized. It returns false if the FMAC peripheral is not present on the specific STM32 silicon being used. Failing to check this return value is a common mistake that leads to system crashes.

`ca_fmac_run`

Header: `caos/drivers/inc/fmac.h:65`
Source: `caos/drivers/src/fmac.c:80:84`
GUARD: `EN_FMAC`

```c
u32 ca_fmac_run(const i16 *in, i16 *out, u32 len); /* samples filtered; < len only on a stall */
```

Processes a buffer of audio samples through the hardware FMAC (Filter Math Accelerator) unit.

Parametri:

Nome	Tipo	Descrizione
`in`	`const i16 *`	Pointer to the input buffer containing raw audio samples. Must not be NULL.
`out`	`i16 *`	Pointer to the output buffer where filtered samples will be stored. Must not be NULL.
`len`	`u32`	Number of samples to process. Must be greater than 0.

Ritorna:
Returns the number of samples successfully processed. If the return value is less than the requested 'len', it indicates a hardware stall or an error condition.

Note:

- This function is a wrapper for the hardware FMAC peripheral and is not thread-safe; ensure that only one task accesses the FMAC at a time. It is not intended to be called from an ISR. Before calling this, the FMAC must be initialized and started using the appropriate configuration functions. A common beginner mistake is passing NULL pointers or failing to check the return value, which leads to silent data processing failures if the hardware stalls.

Uso:

- This function is used in a processing loop to stream audio data through a filter. The typical sequence is: 1) Initialize the FMAC hardware, 2) Start the FMAC, 3) Call ca_fmac_run repeatedly in a loop to process chunks of data, 4) Check the return value to ensure all samples were processed, 5) Stop the FMAC when finished. It is often paired with ca_fmac_flags to retrieve status information after the processing is complete.

Esempio:

```
/* Assuming FMAC is already configured and started */
#define BUFFER_SIZE 128
i16 input_buffer[BUFFER_SIZE];
i16 output_buffer[BUFFER_SIZE];

/* Process the buffer */
u32 processed = ca_fmac_run(input_buffer, output_buffer, BUFFER_SIZE);

if (processed != BUFFER_SIZE) {
    /* Handle hardware stall or error */
    ca_fmac_stop();
} else {
    /* Retrieve status flags after successful processing */
    u32 flags = ca_fmac_flags();
    ca_fmac_stop();
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_fmac_run in my application?
A: You should call ca_fmac_run within a processing loop to stream audio data through the FMAC hardware. The typical sequence involves initializing and starting the FMAC hardware first, then repeatedly calling ca_fmac_run to process chunks of data, and finally stopping the FMAC when finished.

Code example conf:1.00 **Q:** Provide a minimal example of how to use ca_fmac_run with caOS types.
A: i16 input[128]; i16 output[128]; u32 samples_to_process = 128; u32 processed = ca_fmac_run(input, output, samples_to_process); if (processed < samples_to_process) { /* Handle stall or error */ }

Parameters conf:1.00 **Q:** What are the constraints on the parameters passed to ca_fmac_run?
A: The 'in' and 'out' pointers must not be NULL. The 'len' parameter represents the number of samples to process and must be greater than 0. Passing NULL pointers is a common mistake that leads to silent data processing failures.

Return conf:1.00 **Q:** What does the return value of ca_fmac_run indicate?
A: The return value indicates the number of samples successfully processed. If the returned value is less than the requested 'len', it indicates that a hardware stall or an error condition has occurred.

ca_fmac_start

Header: caos/drivers/inc/fmac.h:64
Source: caos/drivers/src/fmac.c:36:78
GUARD: EN_FMAC

```
bool ca_fmac_start(const ca_fmac_filter_t *f); /* loads the coefficients, zero state, runs */
```

Initializes and starts the FMAC hardware peripheral with the provided filter coefficients and state.

Parametri:

Nome	Tipo	Descrizione
f	const ca_fmac_filter_t *	Pointer to a filter structure containing coefficients (b, a), lengths (nb, na), gain, and clipping settings. Must not be NULL, and internal pointers (b, a) must be valid.

Ritorna:
Returns true if the hardware was successfully configured and started. Returns false if the hardware is already running, the input structure is invalid, coefficients are missing, or the filter parameters exceed hardware limits.

Note:

- This function is not thread-safe and must not be called from an ISR. It performs blocking hardware register writes to configure the FMAC peripheral. Before calling this, you must ensure ca_fmac_init() has been called successfully. A common mistake is passing a filter structure where the number of feedback coefficients (na) is greater than or equal to the feedforward coefficients (nb), or failing to check the return value, which leads to silent failures if the hardware is busy or parameters are out of range.

Uso:

- This function is the entry point for digital signal processing using the FMAC peripheral. It is typically used in a sequence: first, call ca_fmac_init() during system startup; then, call ca_fmac_start() to load a specific filter configuration; perform processing using ca_fmac_run() in a loop; and finally, call ca_fmac_stop() to release the hardware for other tasks or different filter configurations.

Esempio:

```
ca_fmac_filter_t my_filter = { .b = b_coeffs, .a = a_coeffs, .nb = 16, .na = 8, .gain = 0, .clip = true };

if (ca_fmac_init()) {
    if (ca_fmac_start(&my_filter)) {
        // Process data blocks
        // ca_fmac_run(input_buffer, output_buffer, length);
        ca_fmac_stop();
    } else {
        // Handle configuration error
    }
} else {
    // Handle hardware initialization failure
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_fmac_start in my application flow?
A: You should call ca_fmac_start after successfully initializing the peripheral with ca_fmac_init during system startup. It is used to load a specific filter configuration before entering a processing loop where ca_fmac_run is called, and it should be followed by ca_fmac_stop when the task is complete.

Code example conf:0.90 **Q:** Provide a minimal example of how to call ca_fmac_start using caOS types.
A: ca_fmac_filter_t my_filter = { .b = coeffs_b, .a = coeffs_a, .nb = 5, .na = 2 }; if (ca_fmac_start(&my_filter)) { /* Filter started successfully */ }

Parameters conf:1.00 **Q:** What are the requirements for the ca_fmac_filter_t structure passed to ca_fmac_start?
A: The pointer must not be NULL, and the internal pointers for coefficients (b, a) must be valid. The structure must contain the coefficients, their lengths (nb, na), gain, and clipping settings.

Return conf:1.00 **Q:** What does a return value of false indicate when calling ca_fmac_start?
A: A false return indicates that the hardware is already running, the input structure is invalid, coefficients are missing, or the filter parameters exceed hardware limits.

Error/Edge case conf:1.00 **Q:** What is a common mistake when configuring the filter parameters for ca_fmac_start?
A: A common mistake is passing a filter structure where the number of feedback coefficients (na) is greater than or equal to the number of feedforward coefficients (nb).

Behavior conf:1.00 **Q:** Is ca_fmac_start safe to call from an ISR or in a multi-threaded environment?
A: No, ca_fmac_start is not thread-safe and must not be called from an ISR. It performs blocking hardware register writes to configure the FMAC peripheral.

`ca_fmac_stop`

Header: `caos/drivers/inc/fmac.h:66`
Source: `caos/drivers/src/fmac.c:86:91`
GUARD: `EN_FMAC`

```
1 void ca_fmac_stop(void)
```

Stops the hardware FMAC (Filter Math Accelerator) unit and updates the internal state.

Ritorna:
This function returns void and does not provide a status code.

Note:

- This function is not thread-safe and should not be called from an ISR unless the system design guarantees no concurrent access to the FMAC hardware. It is a non-blocking function that immediately triggers the hardware stop sequence and updates the internal 's_fmac_running' flag. A common beginner mistake is to call this function while an asynchronous DMA transfer is still active, which can lead to unpredictable hardware states; always ensure the hardware is idle or the operation is complete before calling this. It requires that the FMAC module has been previously initialized via ca_fmac_init.

Uso:

- This function is used to gracefully terminate an FMAC operation, typically after a processing loop finishes or when an error occurs during data calculation. It is almost always paired with ca_fmac_run to ensure the hardware is released and the internal state is reset to a non-running condition. In a typical workflow, you would initialize the module,

perform one or more runs, and then call ca_fmac_stop to ensure the peripheral is safely disabled, preventing potential resource conflicts with other tasks.

Esempio:

```c
/* Example: Processing a buffer and stopping the FMAC unit */
#define BUFFER_SIZE 128

void process_data(i16 *input, i16 *output) {
    /* Initialize the FMAC hardware before use */
    ca_fmac_init();

    /* Perform the math operation */
    u32 result = ca_fmac_run(input, output, BUFFER_SIZE);

    if (result != BUFFER_SIZE) {
        /* Error handling: stop the hardware if processing failed */
        ca_fmac_stop();
        return;
    }

    /* Retrieve flags and stop the hardware after successful completion */
    u32 flags = ca_fmac_flags();
    ca_fmac_stop();
}
```

Q&A — fine-tuning review *2/2 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_fmac_stop in my application workflow?
A: You should call ca_fmac_stop to gracefully terminate an FMAC operation after a processing loop finishes or when an error occurs. It is typically paired with ca_fmac_run to ensure the hardware is released and the internal state is reset. You must ensure the hardware is idle or any asynchronous DMA transfers are complete before calling it to avoid unpredictable hardware states.

Code example conf:1.00 **Q:** How do I properly stop the FMAC peripheral in a caOS project?
A: After ensuring your processing is complete and the hardware is idle, you call the function as follows: ca_fmac_stop();. Note that this requires the FMAC module to have been previously initialized via ca_fmac_init.

`ca_gpio_configure`

Header: `caos/drivers/inc/gpio_driver.h:112`
Source: `caos/drivers/src/gpio_driver.c:181:188`
GUARD: `EN_GPIO_DRIVER`

```c
bool ca_gpio_configure(ca_gpio_port_t port, ca_gpio_pin_t pin,
    ca_gpio_mode_t mode, ca_gpio_otype_t otype, ca_gpio_speed_t speed,
    ca_gpio_pull_t pull, u8 af)
```

Configures the electrical and functional parameters of a specific STM32 GPIO pin.

Parametri:

Nome	Tipo	Descrizione
port	ca_gpio_port_t	The GPIO port identifier (e.g., CA_GPIO_PORT_A). Must be a valid port defined in the hardware abstraction layer.
pin	ca_gpio_pin_t	The specific pin number (e.g., CA_GPIO_PIN_0). Must be a valid pin index for the selected port.
mode	ca_gpio_mode_t	Operating mode: Input, Output, Analog, or Alternate Function (AF).
otype	ca_gpio_otype_t	Output type: Push-Pull or Open-Drain.
speed	ca_gpio_speed_t	Slew rate/speed setting: Low, Medium, High, or Very High.
pull	ca_gpio_pull_t	Internal resistor configuration: No pull, Pull-up, or Pull-down.
af	u8	Alternate Function index (0-15). Only relevant if mode is set to GPIO_MODE_AF.

Ritorna:
Returns true if the configuration was applied successfully, or false if the hardware configuration failed (e.g., invalid parameters or hardware error).

Note:

- This function is non-blocking and executes quickly, but it is not inherently thread-safe if multiple tasks attempt to configure the same port simultaneously. It is generally safe to call from an ISR, provided the underlying hardware driver supports it, but it is best practice to perform all GPIO configurations during the system initialization phase. A common beginner mistake is forgetting to call ca_gpio_init_port() before configuring the pins, which may result in the peripheral clock not being enabled, causing the configuration to fail silently or behave unpredictably.

Uso:

- This function is used to prepare pins for specific hardware tasks, such as driving LEDs, setting up communication lines (SPI/UART), or configuring external interrupts. It is typically used in a sequence: first, initialize the port clock using ca_gpio_init_port(), then configure the pin parameters with ca_gpio_configure(), and finally set the initial state using ca_gpio_write(). It is frequently paired with peripheral initialization functions like ca_uart_init() to ensure the physical pins are correctly mapped to the internal hardware logic.

Esempio:

```
/* Initialize Port B clock before configuring pins */
ca_gpio_init_port(CA_GPIO_PORT_B);

/* Configure Pin 0 as a standard output for an LED */
if (ca_gpio_configure(CA_GPIO_PORT_B, CA_GPIO_PIN_0, CA_GPIO_MODE_OUTPUT,
    CA_GPIO_OTYPE_PP, CA_GPIO_SPEED_LOW, CA_GPIO_NOPULL, 0) == true) {
    /* Set initial state to high (LED off if active low) */
    ca_gpio_write(CA_GPIO_PORT_B, CA_GPIO_PIN_0, 1U);
} else {
    /* Handle configuration error */
    while(1);
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_gpio_configure() in my embedded application?

A: You should call ca_gpio_configure() when you need to set up a GPIO pin with specific settings such as input/output mode, pull-up/pull-down, speed, and alternate function. It is typically called during system initialization or when configuring peripherals like SPI, UART, or GPIO-based LEDs and buttons.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_gpio_configure() with caOS types?
A: u8 result = ca_gpio_configure(CA_GPIO_PORT_A, CA_GPIO_PIN_5, CA_GPIO_MODE_OU CA_GPIO_OTYPE_PP, GPIO_SPEED_HIGH, CA_GPIO_NOPULL, 0); if (result) { /* Pin configured successfully */ }

Parameters conf:0.90 **Q:** What does the 'af' parameter in ca_gpio_configure() control, and what are valid values?
A: The 'af' parameter sets the alternate function for a GPIO pin when mode is set to GPIO_MODE_AF. Valid values are from 0 to 15, representing AF0 through AF15. This parameter is ignored if mode is not GPIO_MODE_AF.

Return conf:0.85 **Q:** What causes ca_gpio_configure() to return false, and what does it indicate?
A: ca_gpio_configure() returns false if the configuration fails, which can happen due to invalid port or pin numbers, invalid mode or alternate function settings, or if the port has not been initialized with ca_gpio_init_port(). It indicates that the pin could not be configured as requested.

`ca_gpio_get_io_port`

Header: `caos/drivers/inc/ca_glue_hw.h:156`
GUARD: `EN_CA_GLUE_HW`

```
1 extern GPIO_TypeDef* ca_gpio_get_io_port(ca_gpio_port_t port); /*
    ritorna il puntatore al registro GPIO della porta indicata */
```

Retrieves the base memory address (pointer) of the STM32 GPIO peripheral register block for a given port.

Parametri:

Nome	Tipo	Descrizione
`port`	`ca_gpio_port_t`	An enumeration representing the GPIO port (e.g., GPIOA, GPIOB). Must be a valid member of the ca_gpio_port_t enum defined in the system headers.

Ritorna:
Returns a pointer to the GPIO_TypeDef structure (the hardware register map) if the port is valid. Returns NULL if the provided port identifier is invalid or not supported by the current MCU configuration.

Note:

- This function is non-blocking and thread-safe as it only performs a lookup in a static mapping table. It is safe to call from an ISR because it does not perform any I/O operations or OS synchronization primitives. The most common mistake is failing to check the returned pointer for NULL before dereferencing it, which will cause a HardFault if the port is invalid. Ensure the GPIO peripheral clock is enabled via the system clock configuration before attempting to read or write to the registers returned by this function.

Uso:

- This function is typically used as the first step when performing low-level GPIO manipulation or when building custom drivers that require direct register access. It is often

paired with ca_gpio_init_port to ensure the hardware is ready, followed by direct register access or ca_gpio_write/ca_gpio_read functions. Use this when you need to bypass standard abstraction layers for performance-critical tasks or to access specific register bits (like MODER or PUPDR) not exposed by higher-level APIs.

Esempio:

```c
/* Example: Safely accessing a GPIO port to check its configuration */
#include "caos.h"

void check_port_status(ca_gpio_port_t port) {
    /* Retrieve the hardware register base address */
    GPIO_TypeDef *gpio = ca_gpio_get_io_port(port);

    /* Always validate the pointer before use */
    if (gpio == NULL) {
        /* Log error: invalid port provided */
        return;
    }

    /* Accessing registers directly (e.g., checking MODER register) */
    /* Note: Ensure the GPIO clock is enabled before this access */
    uint32_t mode = gpio->MODER;

    /* Proceed with further operations using the validated pointer */
    /* e.g., ca_gpio_configure(port, pin, ...); */
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_gpio_get_io_port instead of higher-level GPIO APIs?
A: You should use this function when you need to perform low-level GPIO manipulation, build custom drivers, or access specific registers like MODER or PUPDR that are not exposed by higher-level APIs. It is intended for performance-critical tasks where you need to bypass standard abstraction layers.

Code example conf:0.90 **Q:** Provide a minimal C example showing how to retrieve a GPIO port pointer using caOS types.
A: GPIO_TypeDef *port = ca_gpio_get_io_port(GPIOA); if (port != NULL) { port->MODER |= (u32)0x01; }

Parameters conf:1.00 **Q:** What is the expected input for the 'port' parameter and what happens if I pass an invalid value?
A: The 'port' parameter must be a valid member of the ca_gpio_port_t enumeration. If you pass an invalid or unsupported port identifier, the function will return NULL instead of a valid pointer.

Return conf:1.00 **Q:** What does the return value of ca_gpio_get_io_port represent, and what is the risk if I ignore it?
A: The function returns a pointer to the GPIO_TypeDef structure representing the hardware register map. If the return value is NULL, dereferencing it will cause a HardFault. Always check for NULL before accessing the returned pointer.

`ca_gpio_hw_adc_pin_init`

Header: `caos/drivers/inc/ca_glue_hw.h:11`
GUARD: `EN_CA_GLUE_HW`

```c
extern void ca_gpio_hw_adc_pin_init(u8 adc_num, u8 channel); /*
   configures the GPIO pin in analog mode for the ADC channel */
```

Configures a specific microcontroller pin to operate in analog mode for ADC input.

Parametri:

Nome	Tipo	Descrizione
`adc_num`	`u8`	The index of the ADC peripheral. Currently unused in implementation, but should be provided as the target ADC unit index.
`channel`	`u8`	The specific channel number corresponding to the physical pin to be configured for analog input.

Ritorna:
void

Note:

- This function is non-blocking and performs direct register manipulation to set the GPIO mode to analog. It is not thread-safe by itself, but since it is typically called during system initialization before the RTOS scheduler starts, this is usually acceptable. It is safe to call from an ISR if the hardware allows, but it is strongly discouraged to reconfigure pins while an ADC conversion is actively running, as this can lead to undefined electrical behavior or hardware faults. The most common mistake is calling this function multiple times for the same channel or failing to ensure the GPIO clock is enabled before calling this, which may result in the pin configuration being ignored.

Uso:

- This function is a low-level hardware abstraction layer (HAL) call used during the system startup phase. It is typically invoked by the higher-level 'ca_adc_pin_init' function to prepare the physical hardware before any ADC readings are attempted. You should call this once during your driver initialization sequence, pairing it with subsequent calls to ADC configuration functions to ensure the pin is ready to receive analog signals before the ADC peripheral is enabled.

Esempio:

```
/* Initialize ADC channel 5 on ADC peripheral 0 */
#define MY_ADC_ID 0
#define MY_CHANNEL 5

void setup_sensors(void) {
    /* Configure the physical pin for analog input using the high-level wrapper
    */
    ca_adc_pin_init(MY_ADC_ID, MY_CHANNEL);

    /* Now the pin is ready, we can safely read values from the ADC */
    u16 value = ca_adc_read(MY_ADC_ID, MY_CHANNEL);

    /* Log the result if needed */
    ca_printf("ADC Value: %d\n", value);
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_gpio_hw_adc_pin_init() in my caOS application?
A: You should call ca_gpio_hw_adc_pin_init() before starting ADC conversions to configure a GPIO pin in analog mode for use with an ADC channel. This function is typically called during system initialization or before enabling the ADC peripheral. It is not thread-safe and must not be called from an ISR.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_gpio_hw_adc_pin_init() with caOS types?
A:

```
u8 adc_num = 1;
u8 channel = 0;
ca_gpio_hw_adc_pin_init(adc_num, channel);
```

Parameters conf:0.85 **Q:** What are the valid values for the adc_num and channel parameters in ca_gpio_hw_adc_pin_init()?
A: The adc_num parameter accepts values 1, 2, or 3, though the current implementation ignores this value for compatibility. The channel parameter accepts values from 0 to 18, depending on the specific STM32 model. The function does not validate these values, so it's the caller's responsibility to ensure they are valid for the target ADC and MCU.

HW differences conf:0.90 **Q:** Are there any differences in how ca_gpio_hw_adc_pin_init() is implemented across different STM32 families?
A: No, the implementation of ca_gpio_hw_adc_pin_init() is consistent across all supported STM32 families (f1xx, f4xx, f7xx, h5xx, h7xx). Each implementation simply calls set_channel_gpio(channel) after ignoring the adc_num parameter, indicating that the underlying GPIO configuration logic is abstracted away in the hardware-specific set_channel_gpio function.

`ca_gpio_hw_configure`

Header: `caos/drivers/inc/ca_glue_hw.h:163`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_gpio_hw_configure(ca_gpio_port_t port, ca_gpio_pin_t pin,
    ca_gpio_mode_t mode, ca_gpio_otype_t otype, ca_gpio_speed_t speed,
   ca_gpio_pull_t pull, u8 af); /* configures a GPIO pin with all
   parameters */
```

Configures the hardware registers of a specific GPIO pin to set its mode, speed, pull-up/down, and alternate function.

Parametri:

Nome	Tipo	Descrizione
`port`	`ca_gpio_port_t`	The GPIO port identifier (e.g., CA_GPIO_PORT_A). Must be a valid port supported by the MCU.
`pin`	`ca_gpio_pin_t`	The pin number (0-15). Values outside this range may cause undefined behavior or register corruption.
`mode`	`ca_gpio_mode_t`	The operating mode: Input, Output, Analog, or Alternate Function.
`otype`	`ca_gpio_otype_t`	Output type: Push-Pull or Open-Drain. Only relevant when mode is Output or AF.
`speed`	`ca_gpio_speed_t`	Output speed configuration. Affects signal slew rate and power consumption.
`pull`	`ca_gpio_pull_t`	Internal pull-up, pull-down, or no-pull resistor configuration.
`af`	`u8`	Alternate Function index (0-15). Only used if mode is set to GPIO_MODE_AF.

Ritorna:

Returns true if the configuration was applied successfully. Returns false if the port initialization failed (e.g., clock enable error) or if the port pointer is invalid (NULL).

Note:

- This function is non-blocking and performs direct register manipulation. It is not inherently thread-safe; if multiple tasks attempt to configure the same port simultaneously, you may corrupt the register state. While it can be called from an ISR, it is strongly recommended to perform all GPIO configurations during system initialization to avoid race conditions. The most common beginner mistake is assuming the configuration succeeded without checking the return value; if the hardware clock for the port is not enabled, the register writes will be ignored by the MCU, leaving the pin in its default reset state.

Uso:

- This function is the low-level hardware abstraction layer (HAL) used by the higher-level ca_gpio_configure function. It is typically invoked during the system startup sequence or when a peripheral (like a UART or SPI bus) is being initialized. It must always be preceded by a successful call to ca_gpio_init_port to ensure the peripheral clock is active, otherwise, the register writes will have no effect. It is the foundation for all GPIO operations, including reading inputs, toggling outputs, and setting up complex alternate functions.

Esempio:

```
/* Configure PA5 as a push-pull output for an LED */
/* Ensure the port is initialized before configuring the pin */
ca_gpio_init_port(CA_GPIO_PORT_A);

if (ca_gpio_hw_configure(CA_GPIO_PORT_A, 5, CA_GPIO_MODE_OUTPUT,
    CA_GPIO_OTYPE_PP, CA_GPIO_SPEED_LOW, GPIO_PULL_NONE, 0)) {
    /* Configuration successful, now safe to toggle the pin */
    ca_gpio_write(CA_GPIO_PORT_A, 5, 1);
} else {
    /* Handle error: log the failure using standard output */
    ca_stdout_printf_n("Failed to configure PA5");
}
```

Q&A — fine-tuning review

7/7 pairs · qwen3-coder:latest

Usage conf:0.95 **Q:** When should I use ca_gpio_hw_configure() in my caOS project?
A: You should call ca_gpio_hw_configure() when you need to set up a GPIO pin with specific configuration parameters such as mode, output type, speed, pull-up/pull-down, and alternate function. It's typically used during system initialization or when configuring peripherals that require specific GPIO settings.

Code example conf:0.90 **Q:** Can you show a minimal C example of how to configure a GPIO pin using caOS types?
A:

```
u8 pin = 5;
gpio_port_t port = GPIOA;
gpio_mode_t mode = CA_GPIO_MODE_OUTPUT;
gpio_otype_t otype = CA_GPIO_OTYPE_PP;
gpio_speed_t speed = 2;
gpio_pull_t pull = GPIO_PULL_NONE;
bool result = ca_gpio_hw_configure(port, pin, mode, otype, speed, pull, 0);
```

Parameters conf:0.95 **Q:** What are the valid values for the ca_gpio_mode_t parameter?
A: The valid values for ca_gpio_mode_t are CA_GPIO_MODE_INPUT, CA_GPIO_MODE_OUTP and GPIO_MODE_AF. These correspond to input mode, general purpose output mode, and alternate function mode respectively.

Return conf:0.90 **Q:** What does it mean if ca_gpio_hw_configure() returns false?
A: A return value of false indicates that the configuration failed. This can happen if the port or pin number is invalid, or if there was an error enabling the GPIO port clock, which is required before configuring any GPIO registers.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_gpio_hw_configure()?
A: Common mistakes include passing invalid pin or port numbers, forgetting to enable the GPIO port clock before calling the function, and incorrectly setting the alternate function number when in GPIO_MODE_AF. Also, using an invalid speed value or mode combination may lead to unexpected behavior.

Behavior conf:0.90 **Q:** Is ca_gpio_hw_configure() safe to use from an ISR?
A: No, ca_gpio_hw_configure() is not ISR-safe. It's a blocking function that may involve system-level operations like enabling clocks, and should not be called from interrupt service routines.

HW differences conf:0.70 **Q:** How does ca_gpio_hw_configure() differ between STM32H7 and STM32F4 families?
A: While both families use similar register layouts for GPIO configuration (MODER, OTYPER, OSPEEDR, PUPDR, AFR), the STM32H7 has more advanced features like multiple clock domains and different peripheral mapping. However, the core register access pattern remains consistent across these families in caOS implementations.

`ca_gpio_hw_dac_init`

Header: `caos/drivers/inc/ca_glue_hw.h:89`
GUARD: `EN_CA_GLUE_HW`

```
1 extern void ca_gpio_hw_dac_init(u8 channel); /* configura il pin GPIO in
      modalit  analogica per il canale DAC */
```

Configures the specific GPIO pin for DAC operation by setting it to analog mode.

Parametri:

Nome	Tipo	Descrizione
`channel`	`u8`	The DAC channel number. Valid values are 1 (maps to PA4) or 2 (maps to PA5).

Ritorna:
This function returns void and does not provide feedback on success or failure.

Note:

- This function is non-blocking and executes immediately. It is thread-safe in the sense that it only modifies specific GPIO registers, but it should not be called from an ISR as it is intended for system initialization. A common mistake is calling this function before the GPIO clock has been enabled, which will result in the configuration being ignored by the hardware. Always ensure the system clock and peripheral clocks are initialized before attempting to configure pins.

Uso:

- This function is a low-level helper used exclusively during the DAC peripheral initialization sequence. It is typically called by the higher-level ca_dac_init function to prepare the physical pin before the DAC peripheral itself is enabled. You should never call this function manually in your application code; instead, use the standard ca_dac_init interface to ensure all dependencies, such as clocks and peripheral registers, are correctly set up in the right order.

Esempio:

```c
/* Example of how the system initializes the DAC internally */
ca_dac_config_t my_dac_cfg = { .channel = 1 };

/* The user calls the high-level init function */
if (ca_dac_init(&my_dac_cfg)) {
    /* DAC is ready to use */
} else {
    /* Handle initialization error */
}

/* Inside ca_dac_init, the following sequence occurs: */
/* 1. ca_clock_hw_dac_init(1); // Enable peripheral clock */
/* 2. ca_gpio_hw_dac_init(1);  // Configure PA4 as analog */
/* 3. ca_dac_hw_init_periph(1); // Initialize DAC registers */
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_gpio_hw_dac_init() and why is it necessary?
A: You should call ca_gpio_hw_dac_init() before initializing the DAC hardware to configure the GPIO pin in analog mode. This is necessary because the DAC channels are mapped to specific GPIO pins (PA4 for channel 1, PA5 for channel 2), and these pins must be set to analog mode to function correctly as DAC outputs. Failure to call this function may result in undefined behavior or incorrect DAC operation.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_gpio_hw_dac_init()?
A:

```c
u8 channel = 1;
ca_gpio_hw_dac_init(channel);
// Now initialize DAC hardware
```

Parameters conf:0.90 **Q:** What does the channel parameter in ca_gpio_hw_dac_init() represent, and what values are valid?
A: The channel parameter specifies which DAC channel to configure. Valid values are 1 and 2, corresponding to DAC channel 1 and DAC channel 2 respectively. Any other value will cause undefined behavior as per the documentation.

Return conf:0.85 **Q:** Does ca_gpio_hw_dac_init() return any value, and what would cause it to fail?
A: The function does not return any value. It is designed not to return errors because it only configures GPIO pins and does not perform critical operations. However, in future versions, it may return an error code if additional checks are implemented.

HW differences conf:0.90 **Q:** How does ca_gpio_hw_dac_init() differ between STM32F1xx and other STM32 families like F4xx or H7xx?
A: For STM32F1xx family, the function is implemented as a stub that does nothing (empty implementation). For F4xx, F7xx, H5xx, and H7xx families, the function configures the appropriate GPIO pin (PA4 for channel 1, PA5 for channel 2) in analog mode using ca_gpio_configure(). This difference reflects the varying GPIO configuration capabilities and requirements across different STM32 families.

`ca_gpio_hw_i2c_init`

Header: `caos/drivers/inc/ca_glue_hw.h:180`
GUARD: `EN_CA_GLUE_HW`

```c
extern void ca_gpio_hw_i2c_init(u8 num, u8 alternate); /* configures SDA
   /SCL pins in alternate function open-drain mode; alternate selects
   the pin pair (0 = default) */
```

Configures the specific GPIO pins for I2C communication in Alternate Function Open-Drain mode.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The I2C peripheral index (typically 1 to 4). Must match the hardware capabilities of the specific STM32 series being used.
`alternate`	`u8`	Selects the pin mapping pair. 0 is the default. On some families (like F1xx), this is ignored; on others, it selects specific alternate pin configurations.

Ritorna:
void

Note:

- This function is non-blocking and executes immediately. It is not thread-safe by itself, but since it is typically called during the system initialization phase before the RTOS scheduler starts, this is rarely an issue. It is not intended to be called from an ISR, as GPIO configuration is a setup task. The most common beginner mistake is calling this function without ensuring the corresponding GPIO clock has been enabled, or attempting to use an I2C index that does not exist on the specific MCU variant, which will result in no signal on the pins. Note that on some architectures like F1xx, the function ignores the Alternate Function index parameter as it is implicitly tied to the peripheral clock, whereas on H7xx/H5xx/U5xx/F4xx/F7xx it explicitly sets the AF mode.

Uso:

- This function is a mandatory step in the I2C initialization sequence. It is always called by the high-level driver 'ca_i2c_init' immediately after enabling the peripheral clock and before configuring the I2C peripheral registers. You should never call this function manually in your application code; instead, rely on the 'ca_i2c_init' function to handle the full hardware setup sequence (Clock -> GPIO -> Peripheral). It is part of the low-level hardware abstraction layer (HAL) that ensures the physical pins are correctly routed to the I2C controller.

Esempio:

```
/* Example of how the driver uses this function internally */
void setup_my_i2c(void) {
    /* Define configuration structure */
    ca_i2c_config_t my_config;
    my_config.i2c_num = 1;
    my_config.alternate = 0;
    my_config.speed = 100000;
    my_config.irq_req = 1; /* Assuming 1 represents I2C_EN_IRQ */

    /* The high-level driver calls the GPIO init internally */
    /* ca_i2c_init performs:
       1. ca_clock_hw_i2c_init(cfg->i2c_num);
       2. ca_gpio_hw_i2c_init(cfg->i2c_num, cfg->alternate);
       3. Peripheral register configuration */
    if (ca_i2c_init(&my_config)) {
        /* I2C is now ready for use */
    } else {
        /* Handle initialization failure */
    }
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_gpio_hw_i2c_init() in my caOS application?
A: You should call ca_gpio_hw_i2c_init() during system initialization, before any I2C communication is attempted. It configures the SDA and SCL pins for hardware I2C in open-drain mode, which is required for proper I2C bus operation. The function is designed to be used only during system startup and not from within ISRs or in multitasking contexts.

Code example conf:0.95 **Q:** Can you show a minimal example of how to use ca_gpio_hw_i2c_init() in a caOS application?
A:

```
u8 i2c_num = 1;
ca_gpio_hw_i2c_init(i2c_num);
/* Now I2C interface 1 is ready for use */
```

Parameters conf:0.95 **Q:** What does the 'num' parameter in ca_gpio_hw_i2c_init() represent, and what are valid values?
A: The 'num' parameter specifies which I2C hardware interface to configure. Valid values are 1, 2, and optionally 3 for some MCU families (H5xx). For F1xx, only 1 and 2 are valid. Values outside the valid range (less than 1 or greater than 2 for F1xx, or greater than 3 for other families) are ignored. Invalid values result in no action being taken.

Return conf:0.95 **Q:** Does ca_gpio_hw_i2c_init() return any value, and what does it signify?
A: The function returns void, meaning it does not return any value. It is designed to never fail in a logical sense, but may not perform any action if an invalid parameter is passed. The function is not designed to report errors.

Error/Edge case conf:0.90 **Q:** What are common mistakes or edge cases when using ca_gpio_hw_i2c_init()?
A: Common mistakes include calling the function from within an ISR, which is not allowed due to its blocking nature and lack of thread safety. Another mistake is passing invalid values for 'num', which will cause the function to do nothing. Additionally, the function does not check if pins are already configured, so calling it multiple times with the same interface number will overwrite previous configurations.

Behavior conf:0.95 **Q:** Is ca_gpio_hw_i2c_init() thread-safe, ISR-safe, and blocking?
A: No, ca_gpio_hw_i2c_init() is neither thread-safe nor ISR-safe. It is a blocking function and must not be called from within an ISR. It is designed to be used only during system initialization, not during runtime operations or in multitasking contexts.

HW differences conf:0.90 **Q:** How does ca_gpio_hw_i2c_init() differ between MCU families like F1xx and H7xx?
A: The main differences lie in the number of available I2C interfaces and pin configurations. F1xx supports only 1 or 2 interfaces (PB6/SCL, PB7/SDA and PB10/SCL, PB11/SDA). H7xx supports up to 4 interfaces with different pin mappings (e.g., PB8/SCL, PB9/SDA for interface 1). Additionally, H7xx uses CA_GPIO_NOPULL instead of CA_GPIO_PULLUP for I2C pins, and the alternate function value is consistently AF4 across all families.

---

ca_gpio_hw_init_port

Header: `caos/drivers/inc/ca_glue_hw.h:162`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_gpio_hw_init_port(ca_gpio_port_t port); /* enables the
   RCC clock of the GPIO port */
```

Enables the peripheral clock for a specific GPIO port in the RCC register.

Parametri:

Nome	Tipo	Descrizione
port	ca_gpio_port_t	The identifier for the GPIO port (e.g., CA_GPIO_PORT_A). Must be a valid enum value supported by the specific MCU architecture.

Ritorna:
Returns true if the clock was successfully enabled for the requested port. Returns false if the port identifier is invalid or unsupported by the current hardware configuration.

Note:

- This function is non-blocking and executes in constant time. It is thread-safe in the sense that it performs a simple bitwise OR operation on the RCC register, but it should be called during the initialization phase of your application before any other GPIO configuration functions are invoked. It is safe to call from an ISR, though typically you should initialize hardware before enabling interrupts. The most common mistake is attempting to configure GPIO pins (like setting mode or speed) before calling this function; if the clock is not enabled, the CPU will ignore any writes to the GPIO registers, leading to silent failures where your pins do not behave as expected. The function includes a mandatory read-back of the RCC register to ensure the write operation completes before subsequent peripheral access, preventing potential BusFaults on high-speed STM32 architectures.

Uso:

- This function is the mandatory first step in the GPIO initialization sequence. It is typically called inside a driver's 'init' function or during system startup before configuring pin modes, pull-up/pull-down resistors, or output speeds. It is usually paired with ca_gpio_configure to set the pin's electrical properties. You must ensure this is called once per port; calling it multiple times is harmless due to the bitwise OR operation, but failing to call it will result in the peripheral being effectively 'dead' to the processor.

Esempio:

```
/* Example: Initializing Port A for an LED */
#include "caos.h"

void setup_led_port(void) {
    ca_gpio_port_t port = CA_GPIO_PORT_A;

    /* 1. Enable the clock for the port first */
    if (!ca_gpio_hw_init_port(port)) {
        /* Log the failure using the standard output */
        ca_stdout_printf_n("Failed to initialize GPIO port %d", port);
        return;
    }

    /* 2. Now it is safe to configure pins on Port A */
    /* Use ca_gpio_configure to set pin mode, speed, and pull resistors */
    ca_gpio_configure(port, 5, 0); /* 0 represents default configuration */
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_gpio_hw_init_port() in my embedded application?
A: You should call ca_gpio_hw_init_port() before configuring or using any GPIO pins on a specific port. This function enables the clock for the GPIO peripheral, which is required for the pins to function correctly. It's typically called during system initialization or before setting up GPIO drivers.

Code example conf:0.95 **Q:** Can you show a minimal example of how to use ca_gpio_hw_init_port()?
A:

```c
#include "caos.h"

void init_gpio_ports(void) {
    if (!ca_gpio_hw_init_port(CA_GPIO_PORT_A)) {
        // Handle error
        return;
    }
    if (!ca_gpio_hw_init_port(CA_GPIO_PORT_B)) {
        // Handle error
        return;
    }
}
```

Parameters conf:0.95 **Q:** What does the 'port' parameter represent in ca_gpio_hw_init_port()?
A: The 'port' parameter specifies which GPIO port's clock to enable. Valid values are CA_GPIO_PORT_A through GPIO_PORT_K depending on the MCU family. For example, CA_GPIO_PORT_A enables the clock for port A, CA_GPIO_PORT_B for port B, etc. Any invalid value will cause the function to return false.

Return conf:0.95 **Q:** What does it mean when ca_gpio_hw_init_port() returns false?
A: When ca_gpio_hw_init_port() returns false, it indicates that the 'port' parameter was invalid. This happens when the value passed does not correspond to any known GPIO port (e.g., a value outside the range of CA_GPIO_PORT_A to GPIO_PORT_K).

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_gpio_hw_init_port()?
A: Common mistakes include passing an invalid port value, which causes the function to return false, and not calling this function before configuring GPIO pins, which results in the pins not working correctly. Another mistake is assuming that the function is thread-safe for all contexts without considering that it modifies system registers.

Behavior conf:0.95 **Q:** Is ca_gpio_hw_init_port() blocking or non-blocking, and can it be called from an ISR?
A: The function is non-blocking. It includes a small delay using two NOP instructions to ensure the clock stabilizes, but this delay is very short and does not block other threads. It can be called from an ISR, though it's recommended to avoid doing so unless strictly necessary.

HW differences conf:0.90 **Q:** How does ca_gpio_hw_init_port() differ between F4xx and H7xx MCUs?
A: The main difference lies in the RCC register used to enable the GPIO clocks. On F4xx MCUs, GPIO clocks are enabled via RCC->AHB1ENR, while on H7xx MCUs, they are enabled via RCC->AHB4ENR. Additionally, H7xx supports more GPIO ports (up to GPIO_PORT_K) compared to F4xx which supports up to CA_GPIO_PORT_H.

`ca_gpio_hw_spi_init`

Header: `caos/drivers/inc/ca_glue_hw.h:238`
GUARD: `EN_CA_GLUE_HW`

```c
extern void ca_gpio_hw_spi_init(u8 num, bool alternate); /* configura i pin MOSI/MISO/SCK in AF; alternate=true     mappatura alternativa (vedi spi.h) */
```

Configures the physical GPIO pins for a specific SPI peripheral instance.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The SPI peripheral index (typically 1, 2, or 3). Must match the hardware capabilities of the specific MCU.
`alternate`	`bool`	If true, selects the alternative pin mapping for the SPI peripheral; if false, uses the default pin mapping.

Ritorna:
void. This function does not return a status code, so it assumes the provided parameters are valid for the current hardware target.

Note:

- This function is non-blocking and executes immediately. It is not thread-safe by itself, but it is typically called during the system initialization phase before the RTOS scheduler starts or during driver setup, so concurrency is rarely an issue. It is not intended to be called from an ISR. A common mistake is assuming that setting 'alternate' to true works on all MCUs; always check the datasheet or the specific implementation file for your MCU, as some SPI instances may not support alternate mappings. On F1xx series, this function also handles complex AFIO register remapping, which can inadvertently disable JTAG/SWD debug pins if not handled carefully.

Uso:

- This function is a mandatory step in the SPI initialization sequence. It is called by the high-level 'ca_spi_init' function after enabling the peripheral clock and before configuring the actual SPI peripheral registers. You should use it whenever you need to switch between different physical pin layouts on the PCB, such as when the default pins are already occupied by other peripherals like an Ethernet controller or an SD card slot. It is strictly a hardware-level setup function and should not be called repeatedly during runtime.

Esempio:

```
/* Example of initializing SPI1 with alternate pin mapping */
ca_spi_config_t my_spi_cfg = {
    .spi_num = 1,
    .alternate_gpio = true,
    .master = true,
    .br = 1000000, /* 1MHz */
    .cpol = 0,
    .cpha = 0
};

/* 1. Enable the clock for the SPI peripheral (platform specific) */
/* ca_clock_hw_spi_init(my_spi_cfg.spi_num); */

/* 2. Configure the physical pins */
ca_gpio_hw_spi_init(my_spi_cfg.spi_num, my_spi_cfg.alternate_gpio);

/* 3. Initialize the SPI peripheral registers */
/* Use ca_spi_init to finalize the setup */
if (ca_spi_init(&my_spi_cfg)) {
    /* SPI is ready to use */
} else {
    /* Handle initialization error */
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_gpio_hw_spi_init() in my embedded application?

A: You should call ca_gpio_hw_spi_init() before initializing an SPI peripheral to configure the GPIO pins (MOSI, MISO, SCK) for alternate function mode. This function prepares the hardware for SPI communication by setting up the correct pin configurations, but does not enable the SPI peripheral itself.

Code example conf:0.95 **Q:** Can you show a minimal example of how to use ca_gpio_hw_spi_init() in a C file?

A:

```
u8 spi_num = 1;
ca_gpio_hw_spi_init(spi_num);
/* Now you can initialize the SPI peripheral */
```

Parameters conf:0.95 **Q:** What does the 'num' parameter in ca_gpio_hw_spi_init() represent and what are valid values?

A: The 'num' parameter specifies which SPI peripheral's GPIO pins to configure. Valid values are 1, 2, and 3. Value 1 configures pins for SPI1 (e.g., PA5-7 on F4/F7/H7, PA5-7 on F1). Value 2 configures SPI2 pins (e.g., PB13-15 on F4/F7/H7, PB13-15 on F1 if not STM32F103). Value 3 configures SPI3 pins (e.g., PC10-12 on F4/F7/H7). Values 0 or >3 are ignored.

Return conf:0.95 **Q:** Does ca_gpio_hw_spi_init() return anything meaningful? What could cause it to 'fail'?

A: The function returns nothing (void). It cannot fail in a strict sense, but it may ignore invalid inputs. If called with num=0 or num>3, it does nothing. It also does not check if pins are already configured, so calling it multiple times with same num will reconfigure the same pins without error.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_gpio_hw_spi_init()?

A: Common mistakes include calling it from an ISR, which is not allowed; calling it with invalid num values (0 or >3), which are ignored; and assuming it enables the SPI peripheral itself, which it does not. Also, calling it multiple times with the same num will reconfigure the same pins without error, but may lead to confusion.

Behavior conf:0.95 **Q:** Is ca_gpio_hw_spi_init() blocking or non-blocking? Is it safe to call from an ISR or from multiple threads?

A: The function is non-blocking. It is not safe to call from an ISR, as it is not ISR-safe. It is also not thread-safe, as it modifies global GPIO configuration without synchronization. It has no preconditions, but should be called before any SPI initialization.

HW differences conf:0.90 **Q:** How does ca_gpio_hw_spi_init() differ between STM32F1xx and STM32H7xx families?

A: On F1xx, the function ignores the AF number (parameter 5 in ca_gpio_configure) and uses implicit AF settings. On H7xx, it assigns specific AF numbers: AF5 for SPI1 and SPI2 pins (PA5-7, PB13-15), and AF6 for SPI3 pins (PC10-12). Additionally, H7xx has special handling for PA7 (not used on H753ZI due to RMII_CRS_DV conflict) and uses PB5 as MOSI for SPI1 instead of PA7.

`ca_gpio_init_port`

Header: `caos/drivers/inc/gpio_driver.h:119`
Source: `caos/drivers/src/gpio_driver.c:190:199`
GUARD: `EN_GPIO_DRIVER`

```
void ca_gpio_init_port(ca_gpio_port_t port)
```

Initializes the hardware clock and base configuration for a specific GPIO port.

Parametri:

Nome	Tipo	Descrizione
port	ca_gpio_port_t	The identifier for the GPIO port (e.g., GPIOA, GPIOB). Must be a valid port defined in the hardware abstraction layer.

Ritorna:
void. This function does not return a status code, but it logs an error to the audit system if the hardware initialization fails.

Note:

- This function is non-blocking and is intended to be called during the system initialization phase. It is not thread-safe in the sense that it should only be called once per port during startup to avoid redundant hardware clock configuration. It is not safe to call from an ISR because it performs logging and potential CLI registration, which are not ISR-safe operations. A common mistake is calling this function repeatedly inside a main loop; this is unnecessary and can lead to performance overhead or unexpected hardware state resets. Always ensure the system clock for the GPIO peripheral is enabled before attempting to use specific pins.

Uso:

- This function is the mandatory first step when preparing a GPIO port for use. It is typically followed by ca_gpio_configure to set specific pin modes (input/output/AF) and then by read/write operations. In complex drivers like SPI or Timers, it is used to prepare the port before configuring the specific pins required for the peripheral's Chip Select or signal lines. It acts as a gatekeeper to ensure the hardware peripheral is powered and ready for pin-level configuration.

Esempio:

```
/* Initialize the port before configuring individual pins */
ca_gpio_init_port(GPIOA);

/* Configure a specific pin on that port as an output */
ca_gpio_configure(GPIOA, CA_GPIO_PIN_5, CA_GPIO_MODE_OUTPUT, CA_GPIO_OTYPE_PP,
    GPIO_SPEED_HIGH, CA_GPIO_NOPULL, 0U);

/* Set the pin high */
ca_gpio_write(GPIOA, CA_GPIO_PIN_5, 1U);
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_gpio_init_port() and what is its purpose in a caOS project?
A: You should call ca_gpio_init_port() during the initial hardware setup phase of your caOS project to initialize a complete GPIO port. It is used to register CLI commands for the GPIO port and log any initialization errors. This function is typically called once at system startup to prepare GPIO ports for use, and it's employed in driver initialization files like gpio_driver.c and spi.c.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_gpio_init_port() with caOS types?
A:

```
#include "caos/drivers/gpio_driver.h"

void example_gpio_init(void) {
    ca_gpio_port_t port = CA_GPIO_PORT_A;
    ca_gpio_init_port(port);
}
```

Parameters conf:0.85 **Q:** What does the port parameter in ca_gpio_init_port() represent, and what are the valid values?
A: The port parameter in ca_gpio_init_port() represents the identifier of the GPIO port to be initialized. It must be a valid value defined in the ca_gpio_port_t enumeration, such as CA_GPIO_PORT_A or CA_GPIO_PORT_B. Using invalid values for this parameter can cause undefined behavior, as the function does not validate input and assumes a valid port identifier.

`ca_gpio_read`

Header: `caos/drivers/inc/gpio_driver.h:137`
Source: `caos/drivers/src/gpio_driver.c:214:221`
GUARD: `EN_GPIO_DRIVER`

```
u8 ca_gpio_read(ca_gpio_port_t port, ca_gpio_pin_t pin)
```

Reads the current logic level (0 or 1) of a specific GPIO pin.

Parametri:

Nome	Tipo	Descrizione
`port`	`ca_gpio_port_t`	The GPIO port identifier (e.g., A, B, C). Must be a valid port defined in the system configuration.
`pin`	`ca_gpio_pin_t`	The pin number (0-15). Providing a value outside this range will result in undefined behavior or incorrect register bit shifting.

Ritorna:
Returns 1 if the pin is HIGH, 0 if the pin is LOW. Returns 0 if the provided port is invalid (NULL).

Note:

- This function is non-blocking and executes immediately by reading the Input Data Register (IDR) of the STM32. It is thread-safe in the sense that it performs a single atomic read operation, making it safe to call from both tasks and Interrupt Service Routines (ISRs). However, it does not perform any hardware initialization; you must ensure the GPIO port clock is enabled and the pin is configured as an input before calling this function. A common mistake is assuming the pin state reflects the physical voltage without considering the internal pull-up/pull-down configuration, which can lead to reading unexpected values if the pin is floating.

Uso:

- This function is typically used to poll the state of physical inputs like buttons, switches, or sensors within a control loop or a CLI command handler. It is often paired with initialization functions that set the pin mode to input. In a typical application, you might call this inside a task loop to monitor a button state or within a CLI handler to provide diagnostic feedback to the user about the current hardware status.

Esempio:

```c
/* Example: Polling a button and printing its state */
#include "caos.h"

void monitor_button_task(void) {
    while (1) {
```

```
        /* Read the current state of the button pin */
        u8 btn_state = ca_gpio_read(BTN_PORT, BTN_PIN);

        /* Check if the button is pressed (assuming active high) */
        if (btn_state == 1) {
            ca_stdout_printf_n("Button is currently pressed!");
        } else {
            ca_stdout_printf_n("Button is released.");
        }

        /* Wait 100ms before next check to avoid CPU hogging */
        ca_os_sleep(100);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_gpio_read() in my embedded application?
A: You should use ca_gpio_read() when you need to check the current logical state (HIGH or LOW) of a specific GPIO pin. It is typically used in applications where you want to monitor input signals, such as reading the state of a button or sensor connected to a GPIO pin. The function is non-blocking and can be safely called from both main code and interrupt service routines (ISRs).

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_gpio_read() with caOS types?
A: u8 pin_state = ca_gpio_read(0, 5); // Read pin 5 of GPIO port A if (pin_state == 1) { // Pin is HIGH } else { // Pin is LOW or error }

Parameters conf:0.90 **Q:** What are the valid values for the port and pin parameters in ca_gpio_read()?
A: The 'port' parameter must be an integer from 0 to 10, representing GPIO ports GPIOA through GPIOK. If an invalid port is provided, the function returns 0. The 'pin' parameter must be an integer from 0 to 15, indicating the specific pin within the selected port. Using a pin value of 16 or higher results in undefined behavior.

Return conf:0.90 **Q:** What does ca_gpio_read() return and what causes it to return 0?
A: ca_gpio_read() returns 0 if the pin is at a low logical level (LOW) or if an error occurs, such as an invalid port number. It returns 1 if the pin is at a high logical level (HIGH). A return value of 0 may also indicate that the GPIO port was not initialized correctly via ca_gpio_init() or similar, which would cause ca_gpio_get_io_port() to return NULL.

`ca_gpio_toggle`

Header: `caos/drivers/inc/gpio_driver.h:144`
Source: `caos/drivers/src/gpio_driver.c:223:234`
GUARD: `EN_GPIO_DRIVER`

```
void ca_gpio_toggle(ca_gpio_port_t port, ca_gpio_pin_t pin)
```

Inverts the current logic state of a specific GPIO pin (High to Low, or Low to High).

Parametri:

Nome	Tipo	Descrizione
`port`	`ca_gpio_port_t`	The GPIO port identifier (e.g., CA_GPIO_PORT_A). Must be a valid port defined in the system configuration.
`pin`	`ca_gpio_pin_t`	The specific pin number (0-15). Must be within the valid range for the selected port.

Ritorna:
void. This function does not return a status code; if an invalid port is provided, the function silently returns without performing any action.

Note:

- This function is non-blocking and executes very quickly by writing directly to the BSRR (Bit Set/Reset Register) of the STM32. It is thread-safe in the sense that the BSRR register is atomic, meaning you can safely call this from both the main loop and Interrupt Service Routines (ISRs) without worrying about race conditions on the pin state. A common beginner mistake is assuming the function will work if the GPIO clock has not been enabled in the RCC (Reset and Clock Control) registers; always ensure the port is initialized before toggling. Because it relies on the ODR (Output Data Register) to determine the current state, it is intended for pins configured as output; toggling an input pin may not produce the expected result.

Uso:

- This function is typically used for visual debugging (blinking LEDs), generating square waves for timing analysis, or signaling error states in real-time. It is often paired with ca_gpio_read to verify the state after a toggle or used inside periodic timer callbacks to create heartbeat signals. In complex systems, it is frequently used in error-handling branches to trigger a physical indicator (like a red LED) when a communication packet fails or a state machine enters an unexpected condition.

Esempio:

```c
/* Example: Toggling a status LED inside a periodic timer callback */
#include "caos.h"

void timer_callback(void) {
    /* Toggle GPIO Port B, Pin 14 to indicate activity */
    /* Ensure CA_GPIO_PORT_B is initialized before calling this */
    ca_gpio_toggle(CA_GPIO_PORT_B, GPIO_PIN_14);

    /* Optional: verify the state after toggle */
    u8 current_state = ca_gpio_read(CA_GPIO_PORT_B, GPIO_PIN_14);
    if (current_state == 1) {
        /* Logic for when pin is now HIGH */
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_gpio_toggle() in my embedded application?
A: You should use ca_gpio_toggle() when you need to rapidly switch the state of a GPIO pin, such as blinking an LED or toggling a signal for timing purposes. It's particularly useful in interrupt service routines (ISRs) or any context where you want to avoid the overhead of reading and writing the pin state explicitly.

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_gpio_toggle() with caOS types?
A:

```c
u8 port = CA_GPIO_PORT_A;
u16 pin = 5;
ca_gpio_toggle(port, pin);
```

Parameters conf:0.90 **Q:** What are the valid values for the port and pin parameters in ca_gpio_toggle()?
A: The port parameter must be a valid ca_gpio_port_t value such as CA_GPIO_PORT_A or CA_GPIO_PORT_B. The pin parameter must be an integer between 0 and 15, inclusive. If either parameter is invalid, the function will not perform any action.

Error/Edge case conf:0.85 **Q:** What happens if I call ca_gpio_toggle() with an invalid port or pin?
A: If an invalid port is passed to ca_gpio_toggle(), the function will not perform any action. If an invalid pin (outside the 0-15 range) is passed, the behavior is undefined. Additionally, calling this function on a pin that is not configured as an output may lead to unpredictable behavior.

`ca_gpio_write`

Header: `caos/drivers/inc/gpio_driver.h:129`
Source: `caos/drivers/src/gpio_driver.c:201:212`
GUARD: `EN_GPIO_DRIVER`

```
void ca_gpio_write(ca_gpio_port_t port, ca_gpio_pin_t pin, u8 state)
```

Sets the logic level (high or low) of a specific GPIO pin on the STM32 hardware.

Parametri:

Nome	Tipo	Descrizione
`port`	`ca_gpio_port_t`	The identifier for the GPIO port (e.g., GPIOA, GPIOB). Must be a valid port initialized by the system.
`pin`	`ca_gpio_pin_t`	The pin number (0-15) to be controlled. Ensure the pin is configured as an output before writing.
`state`	`u8`	The desired logic level: 1 (or non-zero) for high, 0 for low.

Ritorna:
void. This function does not return a status code; if the port is invalid, the function silently returns without performing any action.

Note:

- This function is non-blocking and executes extremely quickly because it writes directly to the BSRR (Bit Set/Reset Register) of the STM32. It is thread-safe in the sense that BSRR operations are atomic, meaning you can safely call this from different threads or even from an ISR without worrying about race conditions on the pin state. However, a common beginner mistake is calling this function before initializing the GPIO port and pin via ca_gpio_configure; if the hardware clock for the port is not enabled, the write operation will have no effect, leaving the pin in an undefined state. Always ensure the peripheral clock is active before attempting to write.

Uso:

- This function is the primary way to control external hardware like LEDs, relay drivers, or chip-select lines for communication protocols like SPI. It is typically used in a three-step sequence: first, initialize the port using ca_gpio_init_port; second, configure the pin mode as output using ca_gpio_configure; and finally, use ca_gpio_write to toggle the state. In communication drivers, it is frequently used to set a chip-select pin high to deselect a device or low to initiate a transaction. It is also used in system management, such as pulling a PHY reset pin low to force a network interface to restart.

Esempio:

```
/* Example: Initializing and controlling an LED */
#define LED_PORT GPIOA
#define LED_PIN 5
```

```
4
5  void setup_led(void) {
6      /* 1. Initialize the hardware port */
7      ca_gpio_init_port(LED_PORT);
8
9      /* 2. Configure the pin as an output */
10     ca_gpio_configure(LED_PORT, LED_PIN, CA_GPIO_MODE_OUTPUT, CA_GPIO_OTYPE_PP,
       CA_GPIO_SPEED_LOW, CA_GPIO_NOPULL, 0U);
11
12     /* 3. Turn the LED on */
13     ca_gpio_write(LED_PORT, LED_PIN, 1U);
14 }
15
16 void turn_off_led(void) {
17     /* Safely turn off the LED */
18     ca_gpio_write(LED_PORT, LED_PIN, 0U);
19 }
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_gpio_write() in my embedded application?
A: You should use ca_gpio_write() when you need to set or clear a specific GPIO pin in a non-blocking manner. It is ideal for controlling LEDs, enabling/disabling peripherals, or toggling signals quickly without waiting for the operation to complete. It is commonly used in drivers and network modules where fast, immediate output changes are required.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_gpio_write() with caOS types?
A:

```
1 #include "caos/drivers/gpio.h"
2
3 void example() {
4     ca_gpio_write(GPIOA, 5, 1);  // Set PA5 high
5     ca_gpio_write(GPIOB, 12, 0); // Clear PB12 low
6 }
```

Parameters conf:0.95 **Q:** What are the valid ranges for the port and pin parameters in ca_gpio_write()?
A: The port parameter must be between 0 and 9 inclusive, representing GPIO ports like GPIOA through GPIOJ. The pin parameter must be between 0 and 15 inclusive, indicating the specific pin within the selected port. If the port is invalid, the function does nothing. If the pin exceeds 15, behavior is undefined.

Return conf:0.90 **Q:** Does ca_gpio_write() return any value, and what does it indicate?
A: The function does not return any value. It is designed to be non-blocking and does not provide feedback on success or failure. However, if the port has not been initialized with ca_gpio_init_port() and configured with ca_gpio_configure(), the function may not behave correctly, potentially causing a NULL pointer access.

Error/Edge case conf:0.85 **Q:** What are the most common mistakes when using ca_gpio_write()?
A: Common mistakes include calling ca_gpio_write() without first initializing the GPIO port using ca_gpio_init_port() and configuring the pin with ca_gpio_configure(). Additionally, passing invalid port numbers (outside 0-9) or pin numbers (outside 0-15) can lead to undefined behavior. Using the function in an ISR is also unsafe due to lack of data protection guarantees.

`ca_hash_compute`

Header: `caos/drivers/inc/hash.h:51`
Source: `caos/drivers/src/hash.c:30:48`
GUARD: `EN_HASH`

```
bool ca_hash_compute(ca_hash_algo_t algo, const u8 *data, u32 len, u8 *
    digest, u32 *digest_len)
```

Performs a complete one-shot cryptographic hash calculation using the hardware accelerator.

Parametri:

Nome	Tipo	Descrizione
algo	ca_hash_algo_t	The specific hashing algorithm to use (e.g., CA_HASH_ALGO_SHA256). Must be a valid enum value supported by the hardware.
data	const u8 *	Pointer to the input buffer containing the data to be hashed. Must not be NULL.
len	u32	Length of the input data in bytes. Must be greater than 0.
digest	u8 *	Pointer to the output buffer where the resulting hash will be stored. Must be large enough to hold the digest and not NULL.
digest_len	u32 *	Pointer to a variable that receives the actual length of the generated digest. Must not be NULL.

Ritorna:
Returns true if the hash calculation completed successfully, false otherwise. A false return indicates invalid input pointers, hardware initialization failure, or an unsupported algorithm.

Note:

- This function is thread-safe because it uses a mutex to protect the hardware peripheral, ensuring that multiple tasks do not corrupt the hashing process. It is a blocking function, meaning the calling task will wait (sleep) if another task is currently using the hardware accelerator. You must NEVER call this function from an Interrupt Service Routine (ISR) because blocking operations are strictly forbidden in ISRs and will cause system instability or deadlocks. Before calling this, ensure ca_hash_init() has been called during system startup, otherwise the hardware mutex and peripheral will not be ready.

Uso:

- This function is typically used for one-shot hashing operations where the entire data block is available in memory. It is often paired with ca_hash_init() during the system boot sequence to prepare the hardware. In a typical workflow, you allocate a buffer for the digest, call ca_hash_compute, and then verify the return value before using the digest data. It is commonly used in security services, such as verifying file integrity, generating key fingerprints, or performing iterative hashing for password derivation.

Esempio:

```
u8 digest[32];
u32 dlen = 32;

/* Ensure the hardware is initialized before use */
if (ca_hash_compute(CA_HASH_ALGO_SHA256, input_data, data_size, digest, &dlen))
    {
    /* Hash successful, digest is ready to use */
    process_hash(digest, dlen);
} else {
    /* Handle error: log failure or retry */
    ca_stdout_printf_n("Error: Hash computation failed");
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use the ca_hash_compute function in a caOS project?
A: You should use ca_hash_compute when you need to calculate a cryptographic hash (like SHA256 or MD5) of a data block. It is typically used for data integrity checks, secure authentication, or generating unique identifiers for data. This function is ideal for applications where you need to ensure data has not been tampered with or when you require a fixed-size representation of variable-length data.

Code example conf:0.90 **Q:** Can you provide a minimal working example of how to use ca_hash_compute?
A: u8 data[] = "Hello, caOS!"; u8 digest[32]; u32 digest_len = 32; bool result = ca_hash_compute(CA_H data, sizeof(data) - 1, digest, &digest_len); if (result) { // Hash computed successfully, digest contains the result }

Parameters conf:0.85 **Q:** What are the valid values for the algo parameter in ca_hash_compute?
A: The algo parameter accepts values of type ca_hash_algo_t. Valid values include CA_HASH_ALGO_SHA256 and HASH_ALGO_MD5. These represent the hashing algorithms to be used for computing the digest. Other algorithms may be supported depending on the hardware and implementation, but these are the standard ones mentioned in the documentation.

Return conf:0.90 **Q:** What does a return value of false from ca_hash_compute indicate?
A: A return value of false indicates that the hash computation failed. This can happen due to several reasons such as hardware unavailability, an insufficiently sized digest buffer, or invalid input parameters. The function is designed to return false when it cannot complete the hash calculation successfully.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_hash_compute?
A: Common mistakes include passing a digest buffer that is too small for the output of the selected algorithm, not initializing digest_len before calling the function, or calling the function from an ISR context. Additionally, passing NULL for data or digest parameters will also cause failures, as these are required and must be valid pointers.

Behavior conf:0.90 **Q:** Is ca_hash_compute thread-safe and can it be called from an ISR?
A: ca_hash_compute is thread-safe because it uses an internal mutex to protect access to shared resources. However, it is not reentrant and cannot be called from an ISR. The function is blocking and uses mutexes, which makes it unsuitable for interrupt contexts where non-blocking operations are required.

`ca_hash_ctx_final`

Header: `caos/drivers/inc/hash.h:57`
Source: `caos/drivers/src/hash.c:88:103`
GUARD: `EN_HASH`

```
1 bool ca_hash_ctx_final(ca_hash_ctx_t *ctx, u8 *digest, u32 *digest_len)
```

Completes a hashing operation, retrieves the final digest, and releases the hardware context.

Parametri:

Nome	Tipo	Descrizione
`ctx`	`ca_hash_ctx_t *`	Pointer to the hash context structure. Must be initialized and currently in a 'started' state.
`digest`	`u8 *`	Buffer where the resulting hash bytes will be stored. Must be large enough to hold the algorithm's output.
`digest_len`	`u32 *`	Pointer to a variable that will receive the actual length of the generated digest in bytes.

Ritorna:
Returns true if the hash was successfully finalized and the digest was written. Returns false if any input pointer is NULL, if the context was not previously started, or if the hardware operation failed.

Note:

- This function is blocking because it uses a mutex to ensure exclusive access to the hardware hashing engine. It is thread-safe, meaning multiple threads can attempt to use different contexts, but they will wait for the mutex if the hardware is busy. It must NOT be called from an ISR (Interrupt Service Routine) because the mutex lock will cause a system hang or kernel panic. A common mistake is failing to check the return value; if the function returns false, the digest buffer will contain invalid data, and the context remains in an inconsistent state.

Uso:

- This function is the final step in the hashing lifecycle: init -> update (one or more times) -> final. It is used to extract the result after all data chunks have been processed. It pairs with ca_hash_ctx_update to finalize the calculation and automatically cleans up the internal hardware state, allowing other threads to use the hashing engine for their own operations.

Esempio:

```
ca_hash_ctx_t ctx;
u8 digest[32];
u32 digest_len;

// Initialize and update context before finalizing
if (ca_hash_ctx_init(&ctx, CA_HASH_SHA256)) {
    ca_hash_ctx_update(&ctx, (const u8*)"Hello", 5);

    // Finalize the hash and check for success
    if (ca_hash_ctx_final(&ctx, digest, &digest_len)) {
        // Successfully computed hash, digest is ready to use
    } else {
        // Handle error: hardware failure or invalid context
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_hash_ctx_final() in my embedded application?
A: You should call ca_hash_ctx_final() after initializing a hash context with ca_hash_ctx_init() and updating it with one or more calls to ca_hash_ctx_update(). This function finalizes the hash computation and writes the result into the provided digest buffer.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_hash_ctx_final() with caOS types?
A: ca_hash_ctx_t ctx; ca_hash_ctx_init(&ctx, CA_HASH_SHA256); ca_hash_ctx_update(&ctx, (u8*)"hello", 5); u8 digest[32]; u32 digest_len = 32; bool result = ca_hash_ctx_final(&ctx, digest, &digest_len); // Use digest and digest_len here

Parameters conf:0.90 **Q:** What are the valid and invalid values for the digest_len parameter in ca_hash_ctx_final()?
A: The digest_len parameter must point to a valid u32 variable that initially holds the size of the digest buffer in bytes. For SHA256, this should be at least 32 bytes. It will be updated with the actual length of the computed hash. If digest_len is NULL, the function will return false.

Return conf:0.95 **Q:** What does a return value of false from ca_hash_ctx_final() indicate?

A: A return value of false indicates that the function failed to compute the hash. This can happen if any parameter is NULL, the context was not properly initialized or updated, or the hash computation itself failed.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_hash_ctx_final()?

A: Common mistakes include: 1) Not initializing the hash context with ca_hash_ctx_init() before calling this function, 2) Not calling ca_hash_ctx_update() at least once, 3) Passing a digest buffer that is too small for the hash algorithm being used, 4) Passing NULL pointers for digest or digest_len.

Behavior conf:0.95 **Q:** Is ca_hash_ctx_final() thread-safe and can it be called from an ISR?

A: Yes, ca_hash_ctx_final() is thread-safe because it uses a global mutex (hash_mutex) to protect access. However, it is not ISR-safe as it uses mutex locking and system operations that are not allowed in interrupt service routines.

`ca_hash_ctx_free`

Header: `caos/drivers/inc/hash.h:66`
Source: `caos/drivers/src/hash.c:104:112`
GUARD: `EN_HASH`

```
void ca_hash_ctx_free(ca_hash_ctx_t *ctx)
```

Releases the heap-allocated buffer and resets the hash context structure.

Parametri:

Nome	Tipo	Descrizione
`ctx`	`ca_hash_ctx_t *`	Pointer to the hash context structure. Must be a valid pointer or NULL.

Ritorna:
void

Note:

- This function is safe to call multiple times on the same context because it checks if the buffer is already NULL. It is not thread-safe; ensure no other task is accessing the context while this function is running. It is not intended to be called from an ISR as it performs heap memory management. The most common mistake is attempting to use the context structure after calling this function without re-initializing it via ca_hash_ctx_init.

Uso:

- This function is used to clean up resources after a hashing operation is complete or if an error occurs during processing. It is typically the final step in the lifecycle of a hash context: ca_hash_ctx_init -> ca_hash_ctx_update -> ca_hash_ctx_final -> ca_hash_ctx_free. By calling this, you prevent memory leaks in the system heap, which is critical in long-running embedded applications.

Esempio:

```
ca_hash_ctx_t my_ctx;
// Initialize the context before use
ca_hash_ctx_init(&my_ctx);

// Perform hash updates
ca_hash_ctx_update(&my_ctx, data_buffer, data_len);

// Finalize the hash and store result
u8 result[32];
```

```
ca_hash_ctx_final(&my_ctx, result);

// Always free the context buffer to prevent memory leaks
ca_hash_ctx_free(&my_ctx);

// After freeing, the context is reset and safe to be re-initialized or
    discarded
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_hash_ctx_free and why is it important?
A: You should call ca_hash_ctx_free as the final step in the hash context lifecycle, typically after ca_hash_ctx_final, to release heap-allocated buffers. This is critical in long-running embedded applications to prevent memory leaks in the system heap.

Code example conf:0.90 **Q:** Provide a minimal C example showing how to properly clean up a hash context using caOS types.
A: ca_hash_ctx_t ctx; ca_hash_ctx_init(&ctx); /* ... perform operations ... */ ca_hash_ctx_free(&ctx);

Parameters conf:1.00 **Q:** What are the requirements for the ctx parameter in ca_hash_ctx_free?
A: The ctx parameter must be a pointer to a ca_hash_ctx_t structure. It can be a valid pointer or NULL. The function is safe to call multiple times on the same context because it internally checks if the buffer is already NULL.

`ca_hash_ctx_init`

Header: `caos/drivers/inc/hash.h:55`
Source: `caos/drivers/src/hash.c:52:59`
GUARD: `EN_HASH`

```
bool ca_hash_ctx_init(ca_hash_ctx_t *ctx, ca_hash_algo_t algo)
```

Initializes a hash context structure to prepare for cryptographic operations.

Parametri:

Nome	Tipo	Descrizione
`ctx`	`ca_hash_ctx_t *`	Pointer to the context structure to be initialized. Must not be NULL.
`algo`	`ca_hash_algo_t`	The hashing algorithm to use (e.g., CA_HASH_ALGO_SHA256). Must be a valid enum value supported by the hardware.

Ritorna:
Returns true if the context was successfully initialized, false if the provided pointer is NULL.

Note:

- This function is non-blocking and thread-safe because it only modifies the local memory structure provided by the caller. It is safe to call from an ISR, as it does not perform any hardware I/O or OS synchronization primitives. The most common mistake is passing an uninitialized or NULL pointer, which will cause the function to return false and potentially lead to a crash if the caller ignores the return value. Always ensure the context structure is allocated in a scope that remains valid for the duration of the hashing process.

Uso:

- This function is the mandatory first step in the hashing lifecycle. It is typically used at the start of a task loop or before a specific data processing block to reset the state of the hash

engine. It is always followed by calls to update functions (like ca_hash_ctx_update) to feed data into the engine, and eventually a finalization function to retrieve the digest. By using a context-based approach, you can maintain multiple independent hashing operations simultaneously in different tasks without them interfering with each other.

Esempio:

```
ca_hash_ctx_t ctx;
// Initialize the context for SHA256 hashing
if (ca_hash_ctx_init(&ctx, CA_HASH_ALGO_SHA256)) {
    // Context is ready, proceed to feed data
    const char* data = "Hello World";
    ca_hash_ctx_update(&ctx, (const u8*)data, 11);
    // Finalize the hash calculation here
} else {
    // Handle initialization error
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_hash_ctx_init in a caOS project?
A: You should call ca_hash_ctx_init before performing any hashing operations to initialize the context structure for a specific hash algorithm. This function prepares the context for use with algorithms like SHA256, ensuring that internal state is correctly set up before data is processed.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_hash_ctx_init with caOS types?
A: ca_hash_ctx_t ctx; bool result = ca_hash_ctx_init(&ctx, CA_HASH_ALGO_SHA256); if (result) { // Context initialized successfully } else { // Handle initialization failure }

Parameters conf:0.85 **Q:** What are the valid values for the algo parameter in ca_hash_ctx_init?
A: The algo parameter must be one of the values defined in the ca_hash_algo_t enum, such as CA_HASH_ALGO_SHA256. Passing an invalid value may lead to undefined behavior, as the function does not validate the enum value beyond checking for a valid algorithm identifier.

Return conf:0.90 **Q:** What does ca_hash_ctx_init return when it fails, and what causes the failure?
A: The function returns false when the ctx parameter is NULL or when an internal error occurs (which is rare). A NULL ctx causes an immediate return of false, and failure to check this can lead to crashes. Invalid algorithm values may also cause undefined behavior, though the function itself does not validate the algorithm value.

`ca_hash_ctx_update`

Header: `caos/drivers/inc/hash.h:56`
Source: `caos/drivers/src/hash.c:61:86`
GUARD: `EN_HASH`

```
bool ca_hash_ctx_update(ca_hash_ctx_t *ctx, const u8 *data, u32 len)
```

Updates a hash calculation context with a new chunk of data, managing hardware state transitions automatically.

Parametri:

Nome	Tipo	Descrizione
ctx	ca_hash_ctx_t *	Pointer to the hash context structure. Must be initialized via a constructor function before use.
data	const u8 *	Pointer to the buffer containing the data to be hashed. Must not be NULL.
len	u32	Length of the data buffer in bytes. Must be greater than zero.

Ritorna:
Returns true if the data was successfully processed and the context saved. Returns false if the input parameters are invalid, the hardware fails to start, or the mutex cannot be acquired.

Note:

- This function is blocking because it uses a mutex to ensure exclusive access to the shared hardware hash engine. It is NOT safe to call from an ISR (Interrupt Service Routine) because mutexes cannot be locked in interrupt context. The function handles context switching automatically: if it is the first call, it initializes the hardware; otherwise, it restores the previous state from the context structure. A common mistake is passing a NULL pointer or a zero length, which will cause the function to return false immediately without processing any data. Always ensure the context structure is not modified by other threads while this function is executing.

Uso:

- This function is used to process data in chunks, which is essential when the total data size is too large to fit in memory or arrives in fragments from a network or peripheral. It is typically used in a sequence: first, the context is initialized, then 'ca_hash_ctx_update' is called one or more times to feed the data, and finally 'ca_hash_ctx_final' is called to retrieve the resulting hash digest. It is designed to be thread-safe, allowing multiple tasks to perform hashing operations concurrently by serializing access to the underlying hardware registers.

Esempio:

```
ca_hash_ctx_t ctx;
// Assume context is initialized here
const char* chunk1 = "Data Part 1";
const char* chunk2 = "Data Part 2";

// Process data in chunks
if (!ca_hash_ctx_update(&ctx, (const u8*)chunk1, 11)) {
    // Handle error: hardware failure or invalid input
}

if (!ca_hash_ctx_update(&ctx, (const u8*)chunk2, 11)) {
    // Handle error
}

// Finalize the hash calculation
u8 digest[32];
u32 digest_len;
if (ca_hash_ctx_final(&ctx, digest, &digest_len)) {
    // Hash successfully computed
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_hash_ctx_update() in a hash calculation process?
A: You should call ca_hash_ctx_update() when you need to feed data incrementally into a hash calculation. It is typically used after initializing the context with ca_hash_ctx_init()

and before finalizing the hash with a function like ca_hash_ctx_final(). This allows you to compute the hash of large data streams by updating the context multiple times.

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_hash_ctx_update() with caOS types?
A: ca_hash_ctx_t ctx; ca_hash_ctx_init(&ctx, CA_HASH_SHA256); const u8 data1[] = "Hello"; const u8 data2[] = "World"; ca_hash_ctx_update(&ctx, data1, 5); ca_hash_ctx_update(&ct data2, 5);

Parameters conf:0.90 **Q:** What does the 'len' parameter in ca_hash_ctx_update() represent, and what are valid values?
A: The 'len' parameter represents the number of bytes of data to be added to the hash context. It must be greater than zero; passing zero will result in a failure. The 'data' pointer must also be valid and non-NULL, and the 'ctx' pointer must be a valid, initialized context.

Return conf:0.95 **Q:** What does it mean when ca_hash_ctx_update() returns false?
A: A return value of false indicates that the function failed. This can happen due to invalid parameters such as NULL pointers for ctx or data, or a zero length (len). It may also indicate an internal hardware or mutex error, as the function is blocking and uses a mutex for thread safety.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_hash_ctx_update()?
A: Common mistakes include passing NULL to ctx or data, or passing zero for len. Another frequent error is failing to initialize the context with ca_hash_ctx_init() before calling this function. Additionally, calling this function from an ISR is unsafe and should be avoided.

Behavior conf:0.95 **Q:** Is ca_hash_ctx_update() thread-safe and can it be called from an ISR?
A: Yes, ca_hash_ctx_update() is thread-safe because it uses a mutex internally. However, it is not safe to call from an ISR, as mutexes are not compatible with ISR contexts. The function is also blocking, meaning it will not return until it acquires the mutex.

`ca_hash_hw_csr_count`

Header: `caos/drivers/inc/ca_glue_hw.h:164`
GUARD: `EN_CA_GLUE_HW`

```
1 extern u32 ca_hash_hw_csr_count(void); /* ritorna il numero di registri
     CSR del periferico HASH */
```

Returns the number of Context Save/Restore (CSR) registers available in the hardware HASH peripheral.

Ritorna:
Returns an unsigned 32-bit integer representing the count of CSR registers. Returns 0 if the hardware does not support HASH or if the peripheral is not enabled in the current build configuration.

Note:

- This function is non-blocking and thread-safe as it simply reads a constant or a hardware-defined value. It is safe to call from an ISR because it does not perform any complex operations or wait for hardware flags. A common beginner mistake is assuming that every STM32 chip has a HASH peripheral; always check the return value, as it will be 0 on platforms like the H747 or F1xx where HASH is not implemented. Ensure the HASH driver is properly initialized via ca_hash_hw_init before relying on this count for context management.

Uso:

- This function is primarily used during the initialization phase of a HASH context structure. It is typically called inside a setup function to determine how many registers need to be saved or restored when performing a context switch or pausing a long-running hash operation. By pairing this with the context structure initialization, the driver ensures that the software knows exactly how much memory is required to preserve the state of the HASH engine. It acts as a hardware abstraction layer (HAL) bridge, allowing the higher-level software to remain portable across different STM32 families.

Esempio:

```
ca_hash_ctx_t my_ctx;

// Initialize the context structure
// We use the hardware-specific count to know how many registers to track
my_ctx.csr_count = ca_hash_hw_csr_count();

if (my_ctx.csr_count > 0) {
    // Proceed with HASH operations knowing the hardware supports state saving
} else {
    // Handle error: HASH peripheral not available or not supported on this MCU
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_hash_hw_csr_count() and why is it important for hardware initialization?
A: You should call ca_hash_hw_csr_count() during the initialization phase of the HASH peripheral to determine how many Control/Status Registers (CSR) are available for configuration. This is important because it ensures correct hardware setup and prevents assumptions about register count, which can lead to errors. The function must be called after the HASH driver has been initialized and only when HASH support is enabled (EN_HASH = 1).

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_hash_hw_csr_count() with caOS types?
A: u32 csr_count = ca_hash_hw_csr_count(); if (csr_count > 0) { // Proceed with HASH initialization using the CSR count } else { // Handle unsupported or disabled HASH peripheral }

Return conf:0.90 **Q:** What do the return values of ca_hash_hw_csr_count() mean, and what causes it to return 0?
A: The function returns a u32 representing the number of CSR registers available for the HASH peripheral. A return value of 0 indicates either that the HASH peripheral is not enabled, not supported on the current MCU family, or that the driver has not been properly initialized. It is also a common mistake to assume 16 CSR registers without checking the return value.

HW differences conf:0.95 **Q:** How does ca_hash_hw_csr_count() behave differently across various STM32 MCU families?
A: The behavior of ca_hash_hw_csr_count() varies by MCU family. Only the H5xx family returns a non-zero value (specifically H5_HASH_CSR_COUNT), while all other families (F1xx, F4xx, F7xx, H7xx) return 0. This reflects differences in hardware support and implementation, where only H5xx series has actual CSR register support for the HASH peripheral.

ca_hash_hw_digest

Header: caos/drivers/inc/ca_glue_hw.h:167
GUARD: EN_CA_GLUE_HW

```
extern bool ca_hash_hw_digest(u8 *digest, u32 *digest_len,
    ca_hash_algo_t algo); /* finalizza e legge il digest; false se
    timeout */
```

Finalizes the hardware hash calculation and retrieves the resulting digest into a provided buffer.

Parametri:

Nome	Tipo	Descrizione
digest	u8 *	Pointer to the destination buffer where the calculated hash will be stored. Must be large enough to hold the result (e.g., 32 bytes for SHA256).
digest_len	u32 *	Pointer to a variable that will be updated with the actual length of the generated digest in bytes.
algo	ca_hash_algo_t	The hashing algorithm used (e.g., CA_HASH_ALGO_SHA1, CA_HASH_ALGO_SHA256). Must match the algorithm initialized at the start.

Ritorna:
Returns true if the digest was successfully calculated and read within the timeout period. Returns false if the hardware timed out waiting for the calculation to complete or if an unsupported algorithm was requested.

Note:

- This function is blocking; it enters a busy-wait loop (polling) until the hardware sets the DCIS flag or the timeout expires. It is not thread-safe on its own and should be protected by a mutex if multiple tasks access the hardware. Do not call this from an ISR, as the busy-wait loop will block the CPU and prevent other interrupts from being serviced. A common mistake is failing to ensure the hardware peripheral is properly initialized and fed with data before calling this function, which will result in an invalid or incomplete digest.

Uso:

- This function is the final step in the hashing sequence: initialization (ca_hash_hw_start), data processing (ca_hash_hw_feed), and finally retrieval (ca_hash_hw_digest). It is typically used within a wrapper function that manages a mutex to ensure exclusive access to the hardware peripheral. Once this function returns true, the digest buffer is ready for use, and the hardware is typically ready for the next hashing operation.

Esempio:

```
u8 my_digest[32];
u32 my_len;
ca_hash_algo_t my_algo = CA_HASH_ALGO_SHA256;

// Assuming mutex is already locked
ca_hash_hw_feed(data_ptr, data_size);

if (ca_hash_hw_digest(my_digest, &my_len, my_algo)) {
    // Success: my_digest contains the hash
```

```
} else {
    // Error: Hardware timeout or invalid algorithm
}
// ca_mutex_unlock(&hash_mutex);
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_hash_hw_digest() in my application?
A: You should call ca_hash_hw_digest() after initializing the hash hardware with ca_hash_hw_start() and feeding data with ca_hash_hw_feed(). It finalizes the hash computation and retrieves the digest result.

Code example conf:0.90 **Q:** Can you show a minimal example of using ca_hash_hw_digest()?
A: u8 digest[32]; u32 digest_len = 32; ca_hash_algo_t algo = CA_HASH_ALGO_SHA256; bool result = ca_hash_hw_digest(digest, &digest_len, algo);

Parameters conf:0.90 **Q:** What are the valid values for the algo parameter?
A: The algo parameter must be one of the values defined in ca_hash_algo_t, such as CA_HASH_ALGO_SHA1 or CA_HASH_ALGO_SHA256. Invalid values will cause the function to return false.

Return conf:0.95 **Q:** What does it mean when ca_hash_hw_digest() returns false?
A: A return value of false indicates that the operation failed, which can be due to timeout, insufficient buffer size, unsupported algorithm, or hardware unavailability.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_hash_hw_digest()?
A: Common mistakes include passing a NULL pointer for digest or digest_len, not initializing digest_len with the buffer size, calling the function without prior calls to ca_hash_hw_start() and ca_hash_hw_feed(), or using an unsupported hash algorithm.

Behavior conf:0.95 **Q:** Is ca_hash_hw_digest() safe to call from an ISR?
A: No, ca_hash_hw_digest() is not recommended for use in ISRs because it is a blocking function that may cause scheduling issues.

HW differences conf:0.85 **Q:** How does the implementation of ca_hash_hw_digest() differ between STM32H5xx and other families?
A: The STM32H5xx family implements ca_hash_hw_digest() using the HASH_DIGEST peripheral and specific register handling, including setting HASH_STR_DCAL and polling HASH_SR_DCIS for completion. Other families like F1xx, F4xx, F7xx, and H7xx have stub implementations that always return false.

`ca_hash_hw_feed`

Header: `caos/drivers/inc/ca_glue_hw.h:166`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_hash_hw_feed(const u8 *data, u32 len); /* alimenta i dati
    al periferico HASH */
```

Transfers raw data bytes into the hardware HASH peripheral's data input register.

Parametri:

Nome	Tipo	Descrizione
`data`	`const u8 *`	Pointer to the buffer containing the data to be hashed. Must be a valid memory address; passing NULL will cause a hard fault.
`len`	`u32`	Length of the data in bytes. Can be any value, but the function handles non-word-aligned lengths by padding the final word.

Ritorna:

void

Note:

- This function is a low-level driver routine that writes directly to the HASH peripheral's data register (DIN). It is non-blocking in terms of RTOS scheduling, but it is synchronous in execution, meaning the CPU will stay inside this function until all data is copied to the hardware. It is not thread-safe on its own; you must ensure that access to the HASH peripheral is protected by a mutex or semaphore before calling this. It is generally safe to call from an ISR if the hardware is not being accessed by a task simultaneously, but this is highly discouraged as it can lead to race conditions. A common mistake is calling this function without first initializing the HASH peripheral or starting the algorithm, which will result in the data being written to an unconfigured or disabled hardware block.

Uso:

- This function is used in the middle of a hashing operation sequence. The typical workflow is: 1) Acquire a mutex to ensure exclusive access to the HASH hardware, 2) Call a function to initialize/start the specific algorithm (e.g., SHA-256), 3) Call ca_hash_hw_feed to push the data payload into the hardware, 4) Call ca_hash_hw_digest to retrieve the final result, and 5) Release the mutex. It is strictly a data-transfer function and does not manage the state machine of the HASH peripheral itself.

Esempio:

```
void process_data_hash(const u8 *buffer, u32 size) {
    // Ensure exclusive access to the hardware
    if (ca_mutex_lock(&hash_mutex, 100)) {
        // Initialize hardware for SHA-256
        if (ca_hash_hw_start(CA_HASH_ALGO_SHA256)) {
            // Feed the data into the HASH peripheral
            ca_hash_hw_feed(buffer, size);

            // Retrieve the final digest
            u8 digest[32];
            ca_hash_hw_digest(digest, 32, CA_HASH_ALGO_SHA256);
        }
        ca_mutex_unlock(&hash_mutex);
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_hash_hw_feed() in the hash calculation process?
A: You should call ca_hash_hw_feed() after initializing the HASH peripheral and before calling ca_hash_hw_digest(). It is used to feed data into the hardware HASH engine for processing. The function must be called between initialization and digest calculation.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_hash_hw_feed() with caOS types?
A:

```
u8 data[] = "Hello, World!";
ca_hash_hw_feed(data, sizeof(data) - 1);
```

Parameters conf:0.90 **Q:** What are the valid values for the 'data' and 'len' parameters of ca_hash_hw_feed()?
A: The 'data' parameter must be a valid pointer to memory containing data to feed into the HASH peripheral, and cannot be NULL. The 'len' parameter must be greater than zero, otherwise the function does nothing. Invalid values such as NULL data or zero length may lead to undefined behavior.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_hash_hw_feed()?
A: Common mistakes include calling ca_hash_hw_feed() with a NULL data pointer, or with a zero length parameter. These invalid inputs may cause undefined behavior or produce incorrect hash results. Also, calling this function from an ISR is not safe and may cause blocking or context issues.

Behavior conf:0.90 **Q:** Is ca_hash_hw_feed() thread-safe and ISR-safe?
A: No, ca_hash_hw_feed() is not thread-safe unless protected by a mutex. It is also not ISR-safe because it may block if the hardware is busy, which can cause issues in interrupt contexts.

HW differences conf:0.85 **Q:** How does ca_hash_hw_feed() differ between MCU families like H5xx and others?
A: For H5xx family, ca_hash_hw_feed() implements actual data feeding to the HASH peripheral using the DIN register, processing data in 32-bit words and handling remaining bytes. Other families like F1xx, F4xx, F7xx, and H7xx have stub implementations that do nothing, indicating they do not support hardware hash calculation.

`ca_hash_hw_init`

Header: `caos/drivers/inc/ca_glue_hw.h:168`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_hash_hw_init(void); /* enables the clock and resets the
    hardware HASH peripheral */
```

Initializes the hardware HASH peripheral by enabling its clock in the RCC register.

Ritorna:
Returns true if the hardware peripheral is successfully enabled and available on the current MCU; returns false if the peripheral is missing, disabled in configuration, or unsupported on the specific chip.

Note:

- This function is non-blocking and executes very quickly as it only performs a register write to the Reset and Clock Control (RCC) unit. It is thread-safe in the sense that it performs a simple atomic-like write, but it is not intended to be called repeatedly; it should be called once during system startup. It is safe to call from an ISR, but there is no logical reason to do so, as hardware initialization should happen during the system boot phase. The most common mistake is assuming this function configures the HASH algorithm settings; it only powers the hardware block. If you call this and it returns false, your application must handle the error immediately, as subsequent calls to HASH functions will likely fail or cause a hard fault if they attempt to access unclocked registers.

Uso:

- This function is the mandatory first step in the HASH driver lifecycle. It is typically called inside the higher-level 'ca_hash_init' function, which also initializes the mutexes required to protect the peripheral from concurrent access by multiple threads. You should always check the return value of this function before proceeding to any data processing tasks. It is paired with subsequent configuration and computation functions; if this returns false, the driver is effectively dead for that session.

Esempio:

```
/* Initialize the HASH driver and hardware */
if (!ca_hash_init()) {
```

```
    /* Handle hardware failure: log error or enter safe state */
    return false;
}

/* Prepare buffers for hashing */
unsigned char digest[32];
unsigned int digest_len;

/* Perform the actual computation using the initialized hardware */
/* Note: 0 represents the algorithm constant defined in the driver */
if (ca_hash_compute(0, data_ptr, data_len, digest, &digest_len)) {
    /* Success: digest is ready */
} else {
    /* Handle computation error */
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_hash_hw_init() in my embedded application?
A: You should call ca_hash_hw_init() before using any HASH-related functions to ensure the hardware peripheral is enabled and reset. It is typically called during system initialization or before starting cryptographic operations that require the HASH engine.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_hash_hw_init() in a caOS application?
A: bool success = ca_hash_hw_init(); if (success) { // HASH hardware is ready for use } else { // Handle initialization failure }

Return conf:0.95 **Q:** What does it mean when ca_hash_hw_init() returns false?
A: A return value of false indicates that the hardware initialization failed. This can happen if EN_HASH is disabled, the MCU does not support the HASH peripheral (e.g., STM32H747), or if the function is not implemented for the specific MCU family.

HW differences conf:0.90 **Q:** How does ca_hash_hw_init() differ between STM32 H5xx and H7xx microcontrollers?
A: For STM32 H5xx, ca_hash_hw_init() enables the HASH clock by setting RCC_AHB2ENR_HASHE and returns true after a brief delay. For STM32 H7xx, specifically H747, the function returns false because the HASH peripheral is not available on that model.

`ca_hash_hw_restore_ctx`

Header: `caos/drivers/inc/ca_glue_hw.h:169`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_hash_hw_restore_ctx(const u32 *csr, u32 count, u32 cr);
   /* ripristina un contesto HASH salvato in precedenza */
```

Restores the internal state of the hardware HASH peripheral from a previously saved buffer.

Parametri:

Nome	Tipo	Descrizione
`csr`	`const u32 *`	Pointer to an array containing the saved HASH Context Swap Registers (CSR). Must point to a valid memory block of at least 'count' size.
`count`	`u32`	The number of registers to restore. This must match the hardware-specific count used during the save operation.
`cr`	`u32`	The saved value of the Control Register (CR), which defines the HASH configuration and state.

Ritorna:
void

Note:

- This function is non-blocking and executes immediately by writing directly to hardware registers. It is thread-safe only if the caller ensures that no other task or ISR is accessing the HASH peripheral simultaneously, as the HASH hardware is typically a shared resource. It is safe to call from an ISR, provided the hardware is not currently busy with another operation. A common mistake is passing an incorrect 'count' or a stale buffer, which will result in corrupted hash calculations without triggering an explicit error. Note that on some STM32 families (like F1, F4, F7), this function is a no-op because the hardware does not support context switching; always verify your target architecture's capabilities.

Uso:

- This function is used to implement 'context switching' for HASH operations, allowing the system to pause a long-running hash calculation, perform other tasks, and resume later. It is typically used in a sequence: first, the context is saved using a corresponding save function, then the hardware is used for other tasks, and finally, this function is called to restore the state before calling 'ca_hash_hw_feed' to continue processing data. It is essential for multi-threaded environments where multiple data streams need to be hashed concurrently using the same hardware peripheral.

Esempio:

```
/* Assuming ctx is a structure holding the saved state */
/* Restore the hardware state before continuing the hash calculation */
ca_hash_hw_restore_ctx(ctx->csr, ctx->csr_count, ctx->cr_save);

/* Feed the next chunk of data into the hardware */
ca_hash_hw_feed(new_data_buffer, data_length);

/* Save the updated state back to the context structure for future use */
ca_hash_hw_save_ctx(ctx->csr, ctx->csr_count, &ctx->cr_save);
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_hash_hw_restore_ctx in my embedded application?
A: You should use ca_hash_hw_restore_ctx when you need to resume a hash calculation from a previously saved state. This is useful for processing large data streams where the hash computation is interrupted and needs to be resumed later, such as in segmented file transfers or when switching between different processing contexts.

Code example conf:0.90 **Q:** Can you provide a minimal example of how to use ca_hash_hw_restore_ctx with caOS types?
A:

```
u32 saved_context[8];
u32 control_register = 0x12345678;
// Assume saved_context and control_register are filled with values from a
    previous save
ca_hash_hw_restore_ctx(saved_context, 8, control_register);
```

Parameters conf:0.85 **Q:** What are the valid ranges for the count parameter and what happens if I pass an invalid value?
A: The count parameter must be greater than zero and not exceed the maximum number of registers available for the hardware hash unit. Passing an invalid value may cause out-of-bounds memory accesses, leading to undefined behavior or system crashes.

HW differences conf:0.90 **Q:** How does the implementation of ca_hash_hw_restore_ctx differ between STM32F1xx and STM32H5xx families?

A: The STM32F1xx family has a stub implementation that does not perform any actual hardware operations. In contrast, the STM32H5xx family implements the function by writing the CSR values to HASH->CSR registers and then setting the HASH->CR register with the provided control value.

---

ca_hash_hw_save_ctx

Header: caos/drivers/inc/ca_glue_hw.h:168
GUARD: EN_CA_GLUE_HW

```
extern void ca_hash_hw_save_ctx(u32 *csr, u32 count, u32 *cr); /* salva il contesto HASH per riprendere successivamente */
```

Saves the current internal state of the hardware HASH peripheral into a buffer for later resumption.

Parametri:

Nome	Tipo	Descrizione
csr	u32 *	Pointer to an array where the HASH Context Swap Registers (CSR) values will be stored. Must be large enough to hold 'count' elements.
count	u32	The number of CSR registers to read from the hardware. This value is hardware-specific and must match the peripheral's register count.
cr	u32 *	Pointer to a variable where the HASH Control Register (CR) state will be saved.

Ritorna:
This function returns void and does not provide status codes. It assumes the hardware is accessible and the pointers provided are valid.

Note:

- This function is non-blocking and executes immediately by reading hardware registers. It is thread-safe only if the caller ensures exclusive access to the HASH peripheral, typically via a mutex, before calling this function. It is generally safe to call from an ISR, provided the hardware is not currently being accessed by a lower-priority task. A common beginner mistake is providing a buffer that is too small for the 'count' parameter, which will cause a memory corruption (buffer overflow). Always ensure the HASH peripheral clock is enabled and the peripheral is initialized before attempting to save its context.

Uso:

- This function is used in multi-tasking environments where a long-running HASH operation needs to be paused to allow other tasks to use the hardware. It is typically used in a sequence: first, the task locks a mutex, performs some data processing using ca_hash_hw_feed, calls ca_hash_hw_save_ctx to preserve the state, and then unlocks the mutex. Later, when the task resumes, it restores the saved registers back into the hardware before continuing the calculation. This pattern allows the system to share a single hardware HASH engine across multiple threads without losing data integrity.

Esempio:

```
/* Example of saving HASH context before releasing the hardware */
#define HASH_CSR_COUNT 5

void task_process_hash(ca_hash_ctx_t *ctx, u8 *data, u32 len) {
```

```
    /* Assume mutex is already locked */
    ca_hash_hw_feed(data, len);

    /* Save the hardware state into our context structure */
    ca_hash_hw_save_ctx(ctx->csr, HASH_CSR_COUNT, &ctx->cr_save);

    /* Now it is safe to unlock and let other tasks use the HASH peripheral */
    /* ca_mutex_unlock(&hash_mutex); */
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_hash_hw_save_ctx() in a hash computation process?
A: You should call ca_hash_hw_save_ctx() when you need to pause a hash computation and resume it later, especially when processing large data in blocks. This function saves the current hardware context so that the hash calculation can be continued from where it left off without losing state.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_hash_hw_save_ctx() with caOS types?
A:

```
u32 csr[5];
u32 cr;
ca_hash_hw_save_ctx(csr, 5, &cr);
```

Parameters conf:0.90 **Q:** What are the valid ranges for the 'count' parameter in ca_hash_hw_save_ctx()?
A: The 'count' parameter must be greater than zero and not exceed the size of the 'csr' array. Invalid values can lead to undefined behavior or buffer overruns. The 'csr' array must be pre-allocated with sufficient space to hold 'count' registers.

HW differences conf:0.85 **Q:** How does ca_hash_hw_save_ctx() differ between the H5xx and other STM32 families like F1xx or F4xx?
A: The H5xx family implements ca_hash_hw_save_ctx() by reading the HASH->CR register and saving CSR values from the HASH->CSR array. In contrast, F1xx, F4xx, and F7xx families have empty implementations that do nothing, and H7xx also has a stub implementation. This indicates that only H5xx has full hardware support for saving hash contexts.

`ca_hash_hw_set_nblw`

Header: `caos/drivers/inc/ca_glue_hw.h:167`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_hash_hw_set_nblw(u8 valid_bytes); /* imposta NBLW = bit
   validi (0..3 byte) dell'ultima parola prima di DCAL */
```

Configures the number of valid bits in the final partial word of a hash data block.

Parametri:

Nome	Tipo	Descrizione
`valid_bytes`	`u8`	Number of valid bytes in the last word (0 to 3). A value of 0 indicates the last word is empty or fully processed.

Ritorna:
void

Note:

- This function is non-blocking and performs a direct hardware register write to the HASH peripheral's STR (STRategy/STatus) register. It is not thread-safe; it must be called while the HASH peripheral is locked by a mutex or within a critical section to prevent race conditions between multiple tasks. It is generally not safe to call from an ISR unless the HASH peripheral is exclusively managed by that ISR. The most common mistake is passing a value greater than 3, which may corrupt the register bits or cause undefined hardware behavior depending on the specific STM32 series.

Uso:

- This function is used exclusively during the finalization phase of a hash calculation. It is called after all full 32-bit words have been fed into the hardware using ca_hash_hw_feed, but before triggering the final digest calculation. It informs the hardware exactly how many bytes of the final, potentially partial, word should be included in the hash computation. It is typically paired with ca_hash_hw_feed and followed by a function that triggers the final digest calculation.

Esempio:

```c
/* Assuming a buffer of length 'len' needs to be hashed */
u32 whole = len & ~3U;
ca_hash_hw_feed(data, whole);
u8 tail = (u8)(len & 3U);
if (tail > 0) {
    /* Feed the remaining partial word */
    ca_hash_hw_feed(data + whole, tail);
    /* Inform hardware that only 'tail' bytes are valid in the last word */
    ca_hash_hw_set_nblw(tail);
}
/* Proceed to finalize the hash digest */
/* ca_hash_hw_digest(...) would be called here */
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_hash_hw_set_nblw in my hash calculation sequence?
A: This function is used exclusively during the finalization phase of a hash calculation. You must call it after all full 32-bit words have been fed into the hardware using ca_hash_hw_feed, but before triggering the final digest calculation. It informs the hardware how many bytes of the final partial word are valid.

Code example conf:0.90 **Q:** Provide a minimal C example showing how to finalize a hash with a partial word of 2 bytes.
A: void finalize_hash(const u8 *last_word_data) { ca_hash_hw_feed(last_word_data, 2); ca_hash_hw_set_nblw(2); /* Trigger digest calculation here */ }

Parameters conf:1.00 **Q:** What are the valid values for the valid_bytes parameter and what happens if I pass an invalid one?
A: The valid_bytes parameter must be between 0 and 3. A value of 0 indicates the last word is empty or fully processed. Passing a value greater than 3 is a common mistake that may corrupt the register bits or cause undefined hardware behavior depending on the STM32 series.

HW differences conf:1.00 **Q:** How does the implementation of ca_hash_hw_set_nblw differ across STM32 families?
A: In the f1xx, f4xx, and f7xx families, the function is a stub that performs no operation. In the h5xx and h7xx families, it performs a read-modify-write operation on the HASH->STR register to update the NBLW bits. In the u5xx family, it performs a direct store to the HASH->STR register.

ca_hash_hw_start

Header: caos/drivers/inc/ca_glue_hw.h:165
GUARD: EN_CA_GLUE_HW

```
extern bool ca_hash_hw_start(ca_hash_algo_t algo); /* seleziona l'
    algoritmo e avvia una nuova sessione di hashing */
```

Initializes the hardware cryptographic engine to begin a new hashing session with a specified algorithm.

Parametri:

Nome	Tipo	Descrizione
algo	ca_hash_algo_t	The hashing algorithm to use (e.g., CA_HASH_ALGO_SHA1, CA_HASH_ALGO_SHA256). Must be a supported enum value; passing an unsupported or invalid algorithm will cause the function to return false.

Ritorna:
Returns true if the hardware was successfully configured for the requested algorithm, or false if the algorithm is unsupported or the hardware initialization failed.

Note:

- This function is non-blocking and executes immediately by writing to the hardware control registers. It is generally thread-safe only if protected by a mutex, as the underlying hardware peripheral is a shared resource; calling this from multiple threads simultaneously without synchronization will corrupt the hashing state. It is safe to call from an ISR only if the hardware peripheral is not being used by the main thread, but this is strongly discouraged due to the risk of race conditions. A common mistake is failing to check the return value; if the hardware is not initialized or the algorithm is unsupported, proceeding to feed data will result in undefined behavior or silent failure.

Uso:

- This function is the entry point for any hashing operation and must be called before feeding data into the engine. It is typically used within a wrapper function that manages a mutex to ensure exclusive access to the hardware. The standard sequence is: lock the mutex, call ca_hash_hw_start, feed the data using ca_hash_hw_feed, retrieve the result with ca_hash_hw_digest, and finally unlock the mutex.

Esempio:

```
ca_hash_algo_t my_algo = CA_HASH_ALGO_SHA256;

// Always protect hardware access with a mutex
ca_mutex_lock(&hash_mutex);

// Attempt to start the hardware session
if (ca_hash_hw_start(my_algo)) {
    // Hardware is ready, proceed to feed data
    ca_hash_hw_feed(input_buffer, buffer_size);

    // Finalize and get the digest
    if (!ca_hash_hw_digest(output_digest, 32, my_algo)) {
        // Handle digest retrieval error
    }
} else {
```

```
    // Handle unsupported algorithm or hardware failure
}

ca_mutex_unlock(&hash_mutex);
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_hash_hw_start() in my embedded application?
A: You should call ca_hash_hw_start() when you want to begin a new hashing session using the hardware hash engine. It is typically called before feeding data to the hash engine, after ensuring the hardware is initialized. This function is essential for initiating SHA1 or SHA256 operations on supported STM32 families like H5XX.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_hash_hw_start() in C?
A: ca_hash_algo_t algo = CA_HASH_ALGO_SHA256; bool success = ca_hash_hw_start(algo); if (success) { // Proceed with hashing data } else { // Handle error }

Parameters conf:0.95 **Q:** What does the algo parameter in ca_hash_hw_start() control, and what values are valid?
A: The algo parameter selects the hashing algorithm to be used. Valid values are CA_HASH_ALGO_SHA1 and CA_HASH_ALGO_SHA256. If any other value is passed, the function returns false, indicating that the algorithm is not supported on the target hardware.

Return conf:0.95 **Q:** What does it mean when ca_hash_hw_start() returns false?
A: A return value of false indicates that the function failed to start the hashing session. This can happen if the algorithm is not supported, the hardware is not initialized, or the hardware is already busy. It also means that the requested algorithm is not valid for the current MCU family.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_hash_hw_start()?
A: Common mistakes include calling the function without proper initialization of the hardware, not checking the return value, and attempting to use unsupported algorithms. Additionally, calling this function from an ISR or without proper thread synchronization can lead to race conditions or unexpected behavior.

HW differences conf:0.90 **Q:** How does ca_hash_hw_start() differ between MCU families like H5XX and F1XX?
A: On H5XX family, ca_hash_hw_start() is implemented and supports SHA1 and SHA256 algorithms by configuring specific control registers. In contrast, F1XX, F4XX, F7XX, and H7XX families have stub implementations that always return false, meaning they do not support hardware hashing.

`ca_hash_init`

Header: `caos/drivers/inc/hash.h:48`
Source: `caos/drivers/src/hash.c:24:28`
GUARD: `EN_HASH`

```
bool ca_hash_init(void)
```

Initializes the internal HASH hardware peripheral and its associated thread-safety mutex.

Ritorna:
Returns true if both the mutex and the hardware peripheral were initialized successfully; returns false if the hardware initialization failed.

Note:

- This function is not thread-safe and must be called exactly once during the system startup phase, typically within ca_os_main or a configuration routine. It is strictly non-blocking and must NOT be called from an Interrupt Service Routine (ISR) because it initializes a mutex, which is a kernel object intended for task-level synchronization. A common beginner mistake is calling this function multiple times or from different tasks; doing so will re-initialize the mutex, potentially causing race conditions or memory corruption if other tasks are already waiting on that mutex.

Uso:

- This function is the mandatory entry point for any cryptographic operations involving hashing. It is typically paired with ca_hash_compute, which performs the actual data processing. In a standard application lifecycle, you call this during the system initialization phase (before starting any worker tasks) to ensure the hardware is ready and the synchronization primitive is allocated. If this function returns false, the application should enter a safe state or halt, as subsequent calls to hash functions will likely lead to undefined behavior or hardware faults.

Esempio:

```c
void ca_os_configure(void) {
    /* Initialize the HASH module before any tasks start */
    if (!ca_hash_init()) {
        /* Handle hardware failure: log error or enter safe mode */
        CA_SAFE_FAIL("Failed to initialize HASH hardware");
    }

    /* Now it is safe to use the hash compute function */
    // ca_hash_compute(algo, data, len, digest, digest_len);
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_hash_init() in my caOS application, and why is it important?
A: You should call ca_hash_init() before using any hash-related functions in your caOS application. It is important because it initializes the hash module for cryptographic processing and acquires a mutex to ensure exclusive access to the hardware. This function must be called from a non-ISR context and requires that the mutex has been previously created. Failure to initialize the hash module properly will lead to errors when using other hash functions.

Code example conf:0.85 **Q:** Can you provide a minimal C code example using ca_hash_init() with caOS types?
A: bool success = ca_hash_init(); if (success) { u8 data[32] = {0}; u32 hash_result[8] = {0}; /* Use hash functions here */ } else { /* Handle initialization failure */ }

Return conf:0.90 **Q:** What do the return values of ca_hash_init() indicate, and what causes it to return false?
A: ca_hash_init() returns true if initialization succeeds, and false if it fails. Failures can occur due to the mutex not being acquirable or the hardware not being available. The function is blocking while acquiring the mutex, so it will return false if the mutex acquisition fails or if the hardware is not accessible.

`ca_hexdump`

Header: `caos/inc/utils.h:174`
Source: `caos/src/utils.c:570:608`

GUARD: `EN_UTILS`

```
void ca_hexdump( u8 *ptr,u16 size)
```

Prints a formatted hexadecimal and ASCII memory dump to the system standard output.

Parametri:

Nome	Tipo	Descrizione
`ptr`	`u8 *`	Pointer to the start of the memory buffer to be dumped. Must be a valid memory address accessible by the CPU.
`size`	`u16`	The total number of bytes to display. If the size is not a multiple of 16, the final line will be padded accordingly.

Ritorna:
void

Note:

- This function is a blocking utility that formats and sends data to the system output stream. It is not thread-safe if multiple tasks attempt to print simultaneously, as it may result in interleaved or garbled output. It is strictly forbidden to call this function from an Interrupt Service Routine (ISR) because it performs blocking I/O operations which can cause system instability or deadlocks. A common beginner mistake is passing an uninitialized pointer or a pointer to memory that is not currently mapped, which will trigger a HardFault exception.

Uso:

- This function is primarily used for debugging hardware drivers (like QSPI, SDIO, or Flash) to verify that data read from peripherals matches expected values. It is typically used in a sequence where a buffer is allocated, data is read into that buffer via a hardware driver function, the buffer is inspected using ca_hexdump, and finally, the buffer is released using ca_memory_free. It is an essential tool for verifying data integrity during communication protocol development or file system debugging.

Esempio:

```
/* Example: Reading a block from SDIO and dumping the content for verification
   */
u16 len = 512U;
u8 *buffer = ca_memory_allocate(len); // Allocate memory for the block
if (buffer != NULL) {
    // Read data from SDIO block 0 into the buffer
    if (ca_sdio_read_block(0, buffer)) {
        // Print the first 64 bytes of the block to the console
        ca_hexdump(buffer, 64U);
    }
    ca_memory_free(buffer, &len, false); // Always free allocated memory
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_hexdump() in my embedded application?
A: You should use ca_hexdump() for debugging and analyzing memory buffers, especially when working with hardware drivers or low-level data processing. It's particularly useful when you need to inspect raw data from peripherals like flash, QSPI, SDIO, or memory allocations.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_hexdump() with caOS types?
A: u8 buffer[16] = {0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F, 0x10}; ca_hexdump(buffer, 16);

Parameters conf:0.85 **Q:** What does the 'lines' parameter in ca_hexdump() control, and what are valid values?
A: The 'lines' parameter specifies how many bytes of the buffer pointed to by 'ptr' should be printed. Valid values are any u16 value, including 0. A value of 0 prints nothing. Values that are too large may cause memory access violations if the buffer is not sufficiently large.

Error/Edge case conf:0.90 **Q:** What are common mistakes or edge cases when using ca_hexdump()?
A: Common mistakes include passing a NULL pointer for 'ptr', which causes a crash or undefined behavior. Also, passing a 'lines' value that exceeds the allocated memory size can lead to memory corruption. Additionally, calling ca_hexdump() from an ISR without ensuring the output function is non-blocking or ISR-safe can cause system instability.

Behavior conf:0.90 **Q:** Is ca_hexdump() blocking or non-blocking, and is it safe to call from an ISR or in a multi-threaded environment?
A: ca_hexdump() is a blocking function that waits for all data to be printed. It is not ISR-safe unless the underlying output function (ca_stdout_put_str_n) is also ISR-safe. It is also not thread-safe if the output function is not thread-safe. It should not be called from an ISR unless the printing mechanism is guaranteed not to block or interfere with the ISR context.

`ca_hexdump_u32`

Header: `caos/inc/utils.h:152`
Source: `caos/src/utils.c:530:543`
GUARD: `EN_UTILS`

```
1 void ca_hexdump_u32( u32 *ptr,u16 size)
```

Prints a formatted hexadecimal dump of a memory block containing 32-bit unsigned integers to the standard output.

Parametri:

Nome	Tipo	Descrizione
`ptr`	`u32 *`	A pointer to the starting memory address of the data to be dumped. Must be a valid, aligned memory address; passing a NULL pointer will cause a hard fault.
`size`	`u16`	The number of 32-bit words to read and print. Ensure this value does not exceed the allocated buffer size to avoid reading out-of-bounds memory.

Ritorna:
This function returns void and does not provide a status code, meaning it cannot signal if the memory read was successful or if the output buffer was full.

Note:

- This function is non-blocking in terms of RTOS task scheduling, but it performs synchronous string formatting and output operations, which can be slow if the output device (e.g., UART) is busy. It is NOT safe to call from an Interrupt Service Routine (ISR)

because it uses internal string buffers and standard output functions that likely rely on mutexes or shared resources. A common beginner mistake is passing an uninitialized pointer or a size that is too large, which can lead to reading sensitive system memory or triggering a memory protection fault. Always ensure the memory region pointed to by 'ptr' is accessible and within the bounds of your allocated heap or static memory.

Uso:

- This function is primarily used for debugging and diagnostic purposes to inspect the contents of memory buffers, configuration structures, or peripheral registers in real-time. It is typically called during the development phase when you need to verify that data processing algorithms are producing the expected results in memory. It is often paired with initialization functions that allocate memory or configure hardware, and it is used before or after data processing steps to verify state changes. In a production environment, these calls should be wrapped in preprocessor macros (e.g., #ifdef DEBUG) to ensure they are removed from the final binary to save execution time and memory.

Esempio:

```
/* Example: Inspecting a buffer of 4 configuration registers */
u32 config_buffer[4] = {0xDEADBEEF, 0xCAFEBABE, 0x12345678, 0x87654321};

/* Ensure the buffer is initialized before dumping */
if (config_buffer != NULL) {
    /* Print the contents of the buffer to the console for verification */
    ca_hexdump_u32(config_buffer, 4);
} else {
    /* Handle error: pointer is invalid */
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_hexdump_u32 to debug memory contents?
A: You should use ca_hexdump_u32 when you need to inspect the contents of a memory region containing 32-bit values, such as register values or data buffers. It's particularly useful during development or debugging phases to verify data integrity or examine memory layout.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_hexdump_u32 with caOS types?
A: u32 data[4] = {0x12345678, 0x9ABCDEF0, 0x0FEDCBA9, 0x87654321}; ca_hexdump_u32(data, 4);

Parameters conf:0.85 **Q:** What are the valid ranges for the size parameter in ca_hexdump_u32?
A: The size parameter must be greater than 0 and not exceed the allocated size of the memory pointed to by ptr. Passing an invalid size may lead to out-of-bounds memory access, causing undefined behavior or crashes.

Error/Edge case conf:0.80 **Q:** What are common mistakes when calling ca_hexdump_u32?
A: Common mistakes include passing a null pointer, using an invalid size (e.g., 0 or larger than allocated memory), or calling the function from an ISR. Additionally, since the function is not thread-safe, concurrent calls from multiple threads can cause unpredictable results.

`ca_hkdf_sha256`

Header: `caos/crypto/inc/kdf.h:56`
Source: `caos/crypto/src/kdf.c:82:118`
GUARD: `EN_KDF`

```
bool ca_hkdf_sha256(const u8 *salt, u32 salt_len, const u8 *ikm, u32
    ikm_len, const u8 *info, u32 info_len, u8 *out, u32 out_len)
```

Derives a cryptographically strong key from an input keying material using the HKDF-SHA256 algorithm (RFC 5869).

Parametri:

Nome	Tipo	Descrizione
salt	const u8 *	Optional salt value. If NULL or length is 0, a default zero-filled salt is used.
salt_len	u32	Length of the salt buffer in bytes.
ikm	const u8 *	Input Keying Material. Must not be NULL if ikm_len > 0.
ikm_len	u32	Length of the IKM buffer in bytes.
info	const u8 *	Optional context-specific information string. Can be NULL.
info_len	u32	Length of the info buffer in bytes.
out	u8 *	Buffer to store the derived key. Must not be NULL.
out_len	u32	Desired length of the output key. Must be > 0 and <= HKDF_SHA256_MAX_OUT.

Ritorna:
Returns true if the key derivation was successful, false if parameters are invalid (e.g., NULL pointers, invalid lengths) or if the requested output length exceeds the internal limit.

Note:

- This function is a blocking, CPU-intensive operation that performs multiple HMAC-SHA256 passes. It is not thread-safe if the same buffers are accessed concurrently by other tasks. Do not call this function from an ISR, as it will likely exceed the timing constraints of the interrupt context. The function internally uses stack-based buffers for HMAC contexts; ensure your task stack size is sufficient to accommodate these local variables. A common mistake is providing an 'out_len' that is too large, which will cause the function to return false immediately.

Uso:

- Use this function when you need to expand a short, low-entropy secret (like a password or a shared master key) into a longer, cryptographically strong key for encryption or authentication. It is typically used in secure communication protocols to derive session keys from a master secret. You should pair this with secure memory handling; ensure the 'out' buffer is cleared using ca_memzero after use to prevent sensitive data from lingering in RAM.

Esempio:

```
u8 master_key[32] = {0x01, 0x02, ...}; // Assume populated
u8 session_key[32];
u8 info[] = "session_v1";

// Derive a 32-byte session key from the master key
if (ca_hkdf_sha256(NULL, 0, master_key, sizeof(master_key), info, sizeof(info) -
    1, session_key, sizeof(session_key))) {
    // Key derivation successful, proceed to use session_key
} else {
    // Handle error: log failure or abort secure connection
}

// Always clear sensitive keys from memory when finished
```

```
13 ca_memzero(session_key, sizeof(session_key));
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_hkdf_sha256 function in my caOS application?
A: You should use this function when you need to expand a short, low-entropy secret, such as a password or master key, into a longer, cryptographically strong key suitable for encryption or authentication, commonly for deriving session keys in secure communication protocols.

Code example conf:1.00 **Q:** Provide a minimal example of how to call ca_hkdf_sha256 using caOS types.
A: u8 salt[] = {0x01, 0x02}; u8 ikm[] = {0xAA, 0xBB}; u8 out[32]; if (ca_hkdf_sha256(salt, 2, ikm, 2, NULL, 0, out, 32)) { /* Success */ }

Parameters conf:1.00 **Q:** What are the constraints on the salt and ikm parameters?
A: The salt is optional; if NULL or length is 0, a default zero-filled salt is used. The ikm (Input Keying Material) must not be NULL if ikm_len is greater than 0.

Return conf:1.00 **Q:** What do the return values of ca_hkdf_sha256 indicate?
A: The function returns true if the key derivation was successful. It returns false if parameters are invalid, such as passing NULL pointers, providing invalid lengths, or if the requested out_len exceeds the internal limit HKDF_SHA256_MAX_OUT.

Error/Edge case conf:1.00 **Q:** What is a common mistake when setting the out_len parameter?
A: A common mistake is providing an out_len that is too large, which will cause the function to return false immediately because it exceeds the internal limit HKDF_SHA256_MAX_OUT.

Behavior conf:1.00 **Q:** Is ca_hkdf_sha256 safe to call from an ISR or in a multi-threaded environment?
A: No, it is not ISR-safe as it will likely exceed timing constraints. It is also not thread-safe if the same buffers are accessed concurrently by other tasks. It is a blocking, CPU-intensive operation that uses stack-based buffers, so ensure your task stack size is sufficient.

`ca_hmac_sha256`

Header: `caos/crypto/inc/kdf.h:48`
Source: `caos/crypto/src/kdf.c:69:76`
GUARD: `EN_KDF`

```
1 void ca_hmac_sha256(const u8 *key, u32 key_len, const u8 *data, u32 len,
      u8 mac[HMAC_SHA256_SIZE])
```

Computes the HMAC-SHA256 message authentication code for a given key and data buffer.

Parametri:

Nome	Tipo	Descrizione
`key`	`const u8 *`	Pointer to the secret key buffer. Must not be NULL.
`key_len`	`u32`	Length of the secret key in bytes.
`data`	`const u8 *`	Pointer to the input data to be authenticated. Must not be NULL.
`len`	`u32`	Length of the input data in bytes.
`mac`	`u8[HMAC_SHA256_SIZE]`	Output buffer to store the resulting 32-byte HMAC digest.

Ritorna:
This function returns void. The result is written directly into the provided mac buffer.

Note:

- This function is a synchronous, blocking wrapper that performs the entire HMAC process (init, update, final) in one call. It is thread-safe provided that the input buffers are not modified by other tasks during execution. It is NOT safe to call from an ISR because it performs internal memory operations and potentially long-running cryptographic calculations. A common mistake is passing an uninitialized or too-small buffer for the 'mac' parameter, which will lead to memory corruption or invalid authentication tags.

Uso:

- Use this function when you need to verify data integrity or authenticity using a shared secret key. It is typically used in security protocols, such as generating session keys or validating incoming network packets. It is often paired with 'ca_memzero' to clear sensitive key material from memory after use, or with 'ca_hkdf_sha256' when deriving multiple keys from a single master secret. Unlike the manual sequence of init/update/final, this function is best for single-shot operations where the entire data block is available in memory.

Esempio:

```
u8 key[20];
u8 mac[32];
u8 data[] = "SecureMessage";

/* Initialize key with a known value */
ca_memset(key, 0x0B, 20U);

/* Compute HMAC-SHA256 in one go */
ca_hmac_sha256(key, 20U, data, 13U, mac);

/* Always clear sensitive data from stack after use */
ca_memzero(key, sizeof(key));

/* Verify the result using a hypothetical comparison function */
/* if (hex_eq(mac, expected_mac, 32U)) { ... } */
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_hmac_sha256 instead of the manual init/update/final sequence?

A: You should use ca_hmac_sha256 for single-shot operations where the entire data block is available in memory. It is a synchronous, blocking wrapper designed for scenarios like verifying data integrity or authenticity in security protocols, such as validating network packets or generating session keys.

Code example conf:1.00 **Q:** How do I implement a basic call to ca_hmac_sha256 in my application?

A: You must provide a key, data, and a 32-byte buffer for the result. Example: u8 key[16] = {0}; u8 data[] = "test"; u8 mac[32]; ca_hmac_sha256(key, 16, data, 4, mac);

Parameters conf:1.00 **Q:** What are the requirements for the 'key' parameter in ca_hmac_sha256?

A: The 'key' parameter must be a pointer to the secret key buffer and must not be NULL. The 'key_len' parameter specifies the length of this key in bytes. It is recommended to use ca_memzero to clear this sensitive key material from memory after the function call.

`ca_hmac_sha256_final`

Header: `caos/crypto/inc/kdf.h:47`
Source: `caos/crypto/src/kdf.c:61:69`

GUARD: `EN_KDF`

```
void ca_hmac_sha256_final(ca_hmac_sha256_ctx_t *ctx, u8 mac[
    HMAC_SHA256_SIZE])
```

Finalizes an HMAC-SHA256 calculation, writes the result to a buffer, and wipes the context for security.

Parametri:

Nome	Tipo	Descrizione
`ctx`	`ca_hmac_sha256_ctx_t *`	Pointer to the HMAC context structure initialized by ca_hmac_sha256_init. Must not be NULL.
`mac`	`u8[HMAC_SHA256_SIZE]`	Output buffer of size 32 bytes where the final HMAC digest will be stored.

Ritorna:
void. The function does not return a status code; it assumes the context is valid and the output buffer has sufficient space.

Note:

- This function is blocking as it performs the final hashing operations. It is not thread-safe if multiple tasks access the same context object simultaneously. It is not safe to call from an ISR because it performs multiple internal hashing steps that may take significant time. A critical security feature of this function is that it calls ca_memzero on the context structure after completion; this prevents sensitive key material from remaining in RAM. The most common mistake is attempting to use the context object again after calling this function; since the context is wiped, any subsequent call to ca_hmac_sha256_update will result in undefined behavior or a crash.

Uso:

- This function is the final step in the HMAC-SHA256 lifecycle. The typical sequence is: 1) Initialize the context with ca_hmac_sha256_init, 2) Feed data into the context using one or more calls to ca_hmac_sha256_update, and 3) Call ca_hmac_sha256_final to retrieve the result and securely clear the memory. It is used whenever you need to verify data integrity or authenticate a message using a shared secret key.

Esempio:

```
ca_hmac_sha256_ctx_t ctx;
u8 mac[32];
u8 my_data[] = "Hello World";

// Initialize the HMAC context with a key
ca_hmac_sha256_init(&ctx, key, key_len);

// Process the data
ca_hmac_sha256_update(&ctx, my_data, sizeof(my_data));

// Finalize and get the MAC
ca_hmac_sha256_final(&ctx, mac);

// After this point, 'ctx' is zeroed out and cannot be used again.
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_hmac_sha256_final in my application flow?

A: You should call this function as the final step in the HMAC-SHA256 lifecycle, after initializing the context with ca_hmac_sha256_init and processing all data via ca_hmac_sha256_update. It is used to retrieve the final 32-byte digest and securely wipe the context memory.

Code example conf:0.90 **Q:** Provide a minimal C code snippet showing how to finalize an HMAC-SHA256 calculation.
A: ca_hmac_sha256_ctx_t ctx; u8 mac[32]; ca_hmac_sha256_init(&ctx, key, key_len); ca_hmac_sha256_update(&ctx, data, len); ca_hmac_sha256_final(&ctx, mac);

Parameters conf:1.00 **Q:** What are the requirements for the 'ctx' and 'mac' parameters in ca_hmac_sha256_final?
A: The 'ctx' parameter must be a pointer to a valid, initialized HMAC context and must not be NULL. The 'mac' parameter must be a buffer of at least 32 bytes (HMAC_SHA256_SIZE) to store the resulting digest.

`ca_hmac_sha256_init`

Header: `caos/crypto/inc/kdf.h:42`
Source: `caos/crypto/src/kdf.c:30:52`
GUARD: `EN_KDF`

```
void ca_hmac_sha256_init(ca_hmac_sha256_ctx_t *ctx, const u8 *key, u32
    key_len)
```

Initializes the HMAC-SHA256 context with a provided key for secure message authentication.

Parametri:

Nome	Tipo	Descrizione
`ctx`	`ca_hmac_sha256_ctx_t *`	Pointer to the HMAC context structure to be initialized. Must be allocated memory.
`key`	`const u8 *`	Pointer to the secret key buffer. Can be NULL if key_len is 0.
`key_len`	`u32`	Length of the key in bytes. If greater than 64 (SHA256 block size), the key is hashed first.

Ritorna:
void. This function does not return a status code; it assumes the provided pointers are valid.

Note:

- This function is non-blocking and performs purely computational operations, making it safe to call from ISRs or tasks. It is not thread-safe if multiple tasks share the same context object simultaneously; ensure each task uses its own context instance. The function handles key preprocessing (hashing long keys and padding) as per RFC 2104. A common mistake is passing an uninitialized or NULL context pointer, which will cause a memory access violation. Always ensure the context structure is allocated before calling this function.

Uso:

- This function is the mandatory first step in the HMAC-SHA256 sequence. It is typically followed by one or more calls to ca_hmac_sha256_update to process the message data, and finally ca_hmac_sha256_final to retrieve the resulting MAC. It is used in security protocols, such as KDF (Key Derivation Functions) or message integrity checks, to prepare the internal state with the secret key.

Esempio:

```
ca_hmac_sha256_ctx_t hmac_ctx;
u8 key[] = "secret_key";
```

```
u8 data[] = "message_to_sign";
u8 mac[32];

// Initialize the HMAC context with the secret key
ca_hmac_sha256_init(&hmac_ctx, key, sizeof(key) - 1);

// Process the data to be signed
ca_hmac_sha256_update(&hmac_ctx, data, sizeof(data) - 1);

// Finalize the HMAC and store the result in the mac buffer
ca_hmac_sha256_final(&hmac_ctx, mac);
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_hmac_sha256_init in my security implementation?
A: You must call this function as the mandatory first step in the HMAC-SHA256 sequence. It is used to prepare the internal state with a secret key before calling ca_hmac_sha256_update to process message data and ca_hmac_sha256_final to retrieve the MAC.

Code example conf:1.00 **Q:** Provide a minimal C example for initializing the HMAC-SHA256 context.
A: ca_hmac_sha256_ctx_t ctx; u8 key[] = {0x01, 0x02, 0x03}; u32 len = 3; ca_hmac_sha256_init(&ct key, len);

Parameters conf:1.00 **Q:** What are the requirements for the key and key_len parameters?
A: The key parameter is a pointer to the secret key buffer. It can be NULL if key_len is 0. The key_len parameter is the length of the key in bytes; if key_len is greater than 64, the function will hash the key first as per RFC 2104.

Error/Edge case conf:1.00 **Q:** What happens if I pass a NULL pointer as the context argument?
A: Passing an uninitialized or NULL context pointer is a common mistake that will cause a memory access violation. You must ensure the context structure is allocated before calling the function.

Behavior conf:1.00 **Q:** Is ca_hmac_sha256_init safe to call from an ISR, and is it thread-safe?
A: The function is non-blocking and performs purely computational operations, making it safe to call from ISRs or tasks. However, it is not thread-safe if multiple tasks share the same context object simultaneously; each task must use its own context instance.

`ca_hmac_sha256_update`

Header: `caos/crypto/inc/kdf.h:43`
Source: `caos/crypto/src/kdf.c:54:57`
GUARD: `EN_KDF`

```
void ca_hmac_sha256_update(ca_hmac_sha256_ctx_t *ctx, const u8 *data,
    u32 len)
```

Updates the HMAC-SHA256 calculation with a new chunk of input data.

Parametri:

Nome	Tipo	Descrizione
`ctx`	`ca_hmac_sha256_ctx_t *`	Pointer to the HMAC context structure. Must be initialized via ca_hmac_sha256_init before use.
`data`	`const u8 *`	Pointer to the buffer containing the data to be processed.
`len`	`u32`	Length of the data buffer in bytes.

Ritorna:
void

Note:

- This function is non-blocking and performs the hashing operation synchronously on the current thread. It is not thread-safe; you must ensure that the same context is not accessed by multiple tasks simultaneously. It is generally not recommended to call this from an ISR due to the computational cost of SHA256, which could cause jitter in real-time tasks. The most common mistake is passing an uninitialized context or a NULL pointer, which will lead to a hard fault or memory corruption.

Uso:

- This function is used in the middle phase of an HMAC calculation. The typical sequence is: 1) call ca_hmac_sha256_init to set up the key, 2) call ca_hmac_sha256_update one or more times to process the message data, and 3) call ca_hmac_sha256_final to retrieve the resulting MAC. It is used when the data to be authenticated is too large to fit in a single buffer or arrives in fragments (e.g., streaming data over a network).

Esempio:

```
ca_hmac_sha256_ctx_t hmac_ctx;
u8 key[] = "secret_key";
u8 data_chunk1[] = "Hello ";
u8 data_chunk2[] = "World!";
u8 mac[32];

// Initialize the HMAC context with the key
ca_hmac_sha256_init(&hmac_ctx, key, sizeof(key));

// Process data in multiple chunks
ca_hmac_sha256_update(&hmac_ctx, data_chunk1, sizeof(data_chunk1) - 1);
ca_hmac_sha256_update(&hmac_ctx, data_chunk2, sizeof(data_chunk2) - 1);

// Finalize the calculation and store the result in mac
ca_hmac_sha256_final(&hmac_ctx, mac);
```

Q&A — fine-tuning review

3/3 pairs · gemini-3.1-flash-lite

Usage conf:1.00 **Q:** When should I use ca_hmac_sha256_update in my caOS application?
A: You should use this function when the data you need to authenticate is too large to fit into a single buffer or arrives in fragments, such as when streaming data over a network. It is the middle phase of the HMAC calculation, used after initializing the context with ca_hmac_sha256_init and before finalizing the result with ca_hmac_sha256_final.

Code example conf:0.95 **Q:** How do I implement a multi-part HMAC update using caOS types?
A: After initializing the context, you can call the update function multiple times: ca_hmac_sha256_ctx_ ctx; ca_hmac_sha256_init(&ctx, key, key_len); u8 chunk1[] = {0x01, 0x02}; u8 chunk2[] = {0x03, 0x04}; ca_hmac_sha256_update(&ctx, chunk1, 2); ca_hmac_sha256_update(&ctx, chunk2, 2); ca_hmac_sha256_final(&ctx, output);

Parameters conf:1.00 **Q:** What are the requirements for the parameters passed to ca_hmac_sha256_update?
A: The 'ctx' parameter must be a pointer to a context structure previously initialized by ca_hmac_sha256_init; passing an uninitialized context or a NULL pointer will cause a hard fault or memory corruption. The 'data' parameter is a pointer to the input buffer, and 'len' is the length of that buffer in bytes. Note that this function is not thread-safe and should not be called from an ISR due to the high computational cost of SHA256.

ca_http_resp_bad_request

Header: caos/net/inc/http_service.h:72
Source: caos/net/src/http_service.c:94:98
GUARD: EN_HTTP_SERVICE

```
void ca_http_resp_bad_request(ca_http_response_t *r)
```

Sends a standard HTTP 400 Bad Request response to the client.

Parametri:

Nome	Tipo	Descrizione
r	ca_http_response_t *	A pointer to the HTTP response handle. Must be a valid, initialized response object; passing a NULL pointer will cause a system crash.

Ritorna:
This function returns void and does not provide a status code; it assumes the underlying network stack handles the transmission successfully.

Note:

- This function is a blocking call that prepares and sends the HTTP header and body over the network. It is not thread-safe if multiple tasks attempt to write to the same response handle simultaneously, so ensure only one task manages a specific response object. It must not be called from an Interrupt Service Routine (ISR) because it performs blocking I/O operations which are forbidden in ISR context. The most common beginner mistake is calling this function after the response has already been partially sent or closed, which leads to corrupted HTTP streams or memory access violations.

Uso:

- This function is used within HTTP request handlers when the incoming request is malformed, missing required parameters, or fails validation logic. It is typically called during the request parsing phase, immediately after detecting an error, to terminate the request cycle gracefully. It is often paired with ca_http_resp_begin to initialize the response structure and is followed by a return statement to exit the handler function, preventing further processing of the invalid request.

Esempio:

```
void handle_user_request(ca_http_request_t *req, ca_http_response_t *resp) {
    char buffer[64];
    // Attempt to extract a mandatory parameter from the request body
    if (extract_param(req->body, buffer, sizeof(buffer)) == 0) {
        // Parameter missing or malformed, notify the client
        ca_http_resp_bad_request(resp);
        return; // Stop processing immediately
    }
    // Proceed with valid request logic
    ca_http_resp_begin(resp, 200U, "text/plain");
    ca_http_resp_puts(resp, "Success");
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_http_resp_bad_request and what is its purpose in an HTTP server?
A: You should call ca_http_resp_bad_request when you receive a malformed or invalid HTTP request that cannot be processed. This function sends an HTTP 400 Bad Request

response to the client, indicating that the server could not understand the request due to invalid syntax or structure. It is typically used in HTTP server implementations where request validation is performed before processing.

Code example conf:0.85 **Q:** Can you provide a minimal C code example showing how to use ca_http_resp_bad_request with caOS types?

A: ca_http_response_t response; // Assume response is properly initialized ca_http_resp_bad_reques

Parameters conf:0.90 **Q:** What is the significance of the 'r' parameter in ca_http_resp_bad_request, and what happens if it is NULL?

A: The 'r' parameter is a pointer to a pre-initialized ca_http_response_t structure that holds the HTTP response data. If this parameter is NULL, the behavior is undefined, which may lead to a crash or unpredictable results. The function assumes that the structure has been properly initialized and that its internal buffer is sufficient for sending the response.

`ca_http_resp_begin`

Header: `caos/net/inc/http_service.h:56`
Source: `caos/net/src/http_service.c:34:62`
GUARD: `EN_HTTP_SERVICE`

```
1 void ca_http_resp_begin(ca_http_response_t *r, u16 status, const char *
    content_type)
```

Initializes an HTTP response buffer by writing the status line, headers, and a placeholder for the content length.

Parametri:

Nome	Tipo	Descrizione
`r`	`ca_http_response_t *`	Pointer to the response structure. Must be pre-allocated and valid; passing a NULL pointer will cause a system crash.
`status`	`u16`	The HTTP status code (e.g., 200, 404). Only specific codes have defined reason phrases; others default to 'OK'.
`content_type`	`const char *`	A string representing the MIME type (e.g., 'text/html', 'application/json'). Must be a null-terminated string.

Ritorna:
void. This function does not return a status code, so it assumes the buffer has sufficient space to hold the headers.

Note:

- This function is non-blocking and performs direct memory operations on the provided buffer. It is not thread-safe if multiple tasks access the same response object simultaneously, so ensure the response object is owned by the current task. It is not safe to call from an ISR because it performs string operations and memory writes that are not atomic. A common beginner mistake is failing to ensure the response buffer is large enough to hold the headers plus the body; if the buffer is too small, the headers will be truncated, resulting in an invalid HTTP response that the client will reject.

Uso:

- This function is the mandatory first step when generating an HTTP response in a handler function. It is typically followed by calls to write the actual body content and finally,

the response is sent over the network. It pairs with functions like ca_http_resp_write to populate the body and relies on the internal state of the ca_http_response_t structure to track where the content length needs to be patched later. Use this whenever you need to start a new HTTP transaction in response to a client request.

Esempio:

```
static void handler_root(const ca_http_request_t *req, ca_http_response_t *resp)
    {
    (void)req;
    // Initialize the response with 200 OK and HTML content type
    ca_http_resp_begin(resp, 200U, "text/html");
    // Write the actual page content into the buffer
    ca_http_resp_write(resp, "<html><body>Hello!</body></html>", 34U);
    // The system will automatically patch the Content-Length later
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_http_resp_begin in an HTTP server implementation?
A: You should call ca_http_resp_begin at the start of generating an HTTP response, after setting up the response structure and before writing the response body. It initializes the HTTP status line and headers such as Content-Type, Content-Length, and Connection.

Code example conf:0.90 **Q:** Can you show a minimal example of using ca_http_resp_begin with caOS types?
A: ca_http_response_t resp; resp.buf = (u8*)0x20000000; resp.buf_len = 1024; ca_http_resp_begin(&
200, "text/html");

Parameters conf:0.90 **Q:** What are the valid values for the status parameter in ca_http_resp_begin?
A: The valid status codes are 200, 400, 404, and 405. Other values will use the default reason 'OK'.

Error/Edge case conf:0.85 **Q:** What happens if I pass a NULL pointer or empty string to the content_type parameter?
A: Passing a NULL pointer or an empty string for content_type may cause an error or undefined behavior, as the function does not handle such cases gracefully and could lead to a crash.

Behavior conf:0.90 **Q:** Is ca_http_resp_begin thread-safe and ISR-safe?
A: No, ca_http_resp_begin is not thread-safe and not ISR-safe. It is a blocking function that modifies shared state and must not be called from multiple threads or from an ISR.

`ca_http_resp_begin_chunked`

Header: `caos/net/inc/http_service.h:99`
Source: `caos/net/src/http_service.c:134:162`
GUARD: `EN_HTTP_SERVICE`

```
void ca_http_resp_begin_chunked(ca_http_response_t *r, u16 status, const
    char *content_type)
```

Initializes an HTTP response using chunked transfer encoding, allowing data to be sent in multiple parts without knowing the total size beforehand.

Parametri:

Nome	Tipo	Descrizione
r	ca_http_response_t *	Pointer to the HTTP response structure. Must be initialized and valid.
status	u16	HTTP status code (e.g., 200, 404, 400). Only specific codes like 200, 400, 404, 405 are explicitly mapped to reason strings.
content_type	const char *	String defining the MIME type (e.g., "text/plain", "application/json").

Ritorna:
void. This function does not return a status code; it assumes the response structure is valid and memory is available for appending headers.

Note:

- This function is not thread-safe and must be called within the context of a single HTTP request handler. It is strictly non-blocking as it only appends data to the internal response buffer. It must NOT be called from an ISR, as it performs string operations and memory management that are not ISR-safe. The most common mistake is calling this function after the response headers have already been sent or using an uninitialized response pointer, which will lead to memory corruption or system crashes.

Uso:

- Use this function when the total size of the HTTP response body is unknown at the start of the request, such as when streaming data from a sensor or generating a large log file on the fly. It is typically the first function called in a request handler, followed by multiple calls to ca_http_resp_write to send the actual data chunks. After all chunks are sent, the connection is typically closed or finalized by the underlying HTTP service logic.

Esempio:

```
static void my_http_handler(const ca_http_request_t *req, ca_http_response_t *
    resp) {
    (void)req;
    /* Initialize the response for chunked streaming */
    ca_http_resp_begin_chunked(resp, 200U, "text/plain");

    /* Send the first chunk of data */
    ca_http_resp_write(resp, "Starting stream...\n", 19U);

    /* Send another chunk */
    ca_http_resp_write(resp, "Data chunk 1\n", 13U);

    /* Finalize the response by flushing the buffer */
    ca_http_resp_flush(resp);
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_http_resp_begin_chunked instead of a standard HTTP response?
A: You should use this function when the total size of the HTTP response body is unknown at the start of the request, such as when streaming sensor data or generating large log files on the fly.

Code example conf:0.90 **Q:** Provide a minimal example of how to initialize a chunked HTTP response in caOS.
A: ca_http_response_t resp; ca_http_resp_begin_chunked(&resp, 200, "application/json");

Parameters conf:1.00 **Q:** Are there restrictions on the u16 status parameter for ca_http_resp_begin_chunked?
A: The status parameter accepts u16 values, but only specific codes like 200, 400, 404, and 405 are explicitly mapped to reason strings by the function.

Error/Edge case conf:1.00 **Q:** What are the most common mistakes that lead to system crashes when using this function?
A: The most common mistakes are calling the function after response headers have already been sent or providing an uninitialized ca_http_response_t pointer, both of which lead to memory corruption or system crashes.

Behavior conf:1.00 **Q:** Is ca_http_resp_begin_chunked safe to call from an interrupt service routine or across multiple threads?
A: No, it is not thread-safe and must be called within the context of a single HTTP request handler. It must not be called from an ISR because it performs string operations and memory management that are not ISR-safe.

`ca_http_resp_flush`

Header: `caos/net/inc/http_service.h:105`
Source: `caos/net/src/http_service.c:49:74`
GUARD: `EN_HTTP_SERVICE`

```
bool ca_http_resp_flush(ca_http_response_t *r)
```

Flushes the HTTP response staging buffer to the network, handling chunked encoding if enabled.

Parametri:

Nome	Tipo	Descrizione
`r`	`ca_http_response_t *`	Pointer to the HTTP response structure. Must be initialized via ca_http_resp_begin or ca_http_resp_begin_chunked.

Ritorna:
Returns true if the data was successfully sent or if there was nothing to flush. Returns false if the underlying TCP transmission failed or if the response state is already marked as failed.

Note:

- This function is blocking as it relies on ca_tcp_send, which waits for network transmission. It is not thread-safe and must not be called from an ISR, as it performs network I/O operations. The function automatically manages the transition from header transmission to body chunking; once the headers are flushed, the internal buffer pointers are reset. A common mistake is calling this function after the connection has been closed or after a previous call has already returned false, which will simply return the current failure state.

Uso:

- This function is used to push accumulated data from the HTTP response buffer to the client. It is typically called after one or more calls to ca_http_resp_write or ca_http_resp_puts to ensure the data is actually transmitted over the wire. In a typical request-response cycle, you initialize the response with ca_http_resp_begin_chunked, write data, call ca_http_resp_flush to send chunks, and finally close the connection or finish the response.

Esempio:

```
ca_http_response_t *resp = /* assume valid pointer initialized via
    ca_http_resp_begin_chunked */;

```

```c
// Write some data to the staging buffer
if (ca_http_resp_puts(resp, "Hello, World!")) {
    // Flush the buffer to the network
    if (!ca_http_resp_flush(resp)) {
        // Handle transmission error
        /* Log error or close connection */
    }
} else {
    // Handle buffer overflow or write error
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_http_resp_flush in my HTTP response cycle?
A: You should call it after writing data to the response buffer using functions like ca_http_resp_write or ca_http_resp_puts. It is used to push the accumulated data from the staging buffer to the client over the network.

Code example conf:0.90 **Q:** Provide a minimal example of how to use ca_http_resp_flush.
A: ca_http_response_t resp; ca_http_resp_begin_chunked(&resp); ca_http_resp_puts(&resp, "Hello World"); bool success = ca_http_resp_flush(&resp);

Parameters conf:1.00 **Q:** What are the requirements for the ca_http_response_t pointer passed to ca_http_resp_flush?
A: The pointer must point to an initialized ca_http_response_t structure. Initialization must be performed via either ca_http_resp_begin or ca_http_resp_begin_chunked prior to calling the flush function.

Return conf:1.00 **Q:** What do the boolean return values of ca_http_resp_flush indicate?
A: Returning true indicates that the data was successfully sent or that there was no data to flush. Returning false indicates that the underlying TCP transmission failed or that the response state was already marked as failed.

Error/Edge case conf:1.00 **Q:** What are common mistakes to avoid when using ca_http_resp_flush?
A: A common mistake is calling the function after the connection has already been closed or after a previous call has already returned false. In these cases, the function will simply return the current failure state.

Behavior conf:1.00 **Q:** Is ca_http_resp_flush safe to call from an interrupt service routine (ISR) or in a multi-threaded environment?
A: No. The function is not thread-safe and must not be called from an ISR because it performs network I/O operations. Additionally, it is a blocking function as it relies on ca_tcp_send, which waits for network transmission.

`ca_http_resp_not_found`

Header: `caos/net/inc/http_service.h:69`
Source: `caos/net/src/http_service.c:88:92`
GUARD: `EN_HTTP_SERVICE`

```c
void ca_http_resp_not_found(ca_http_response_t *r)
```

Sends a standard HTTP 404 Not Found error response to the client.

Parametri:

Nome	Tipo	Descrizione
`r`	`ca_http_response_t *`	Pointer to the HTTP response handle. Must be a valid, initialized response object; passing a NULL pointer will cause a system crash.

Ritorna:

void

Note:

- This function is a wrapper that simplifies sending a 404 error by calling ca_http_resp_begin and ca_http_resp_puts internally. It is not thread-safe if multiple tasks attempt to write to the same response handle simultaneously, so ensure the handle is owned by the current task. It is strictly non-blocking in terms of logic, but it relies on the underlying network stack's buffer availability; if the network buffer is full, it may wait until space is available. Do not call this from an Interrupt Service Routine (ISR) as it performs heap or buffer operations that are not ISR-safe.

Uso:

- Use this function in your HTTP request handler logic whenever a client requests a resource (like a file or API endpoint) that does not exist on the device. It is typically called after parsing the incoming request path and determining that no matching route is found. It is designed to be the final action for that specific request, effectively terminating the response cycle with a standard error message. You should pair this with your routing logic to ensure that valid requests are handled normally and invalid ones are gracefully rejected with this function.

Esempio:

```
void handle_request(ca_http_request_t *req, ca_http_response_t *resp) {
    // Check if the requested path exists in our internal table
    if (!is_path_valid(req->path)) {
        // Resource not found, send 404 error
        ca_http_resp_not_found(resp);
    } else {
        // Proceed with normal processing
        ca_http_resp_begin(resp, 200U, "text/html");
        ca_http_resp_puts(resp, "<h1>Welcome</h1>");
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_http_resp_not_found in a caOS HTTP server implementation?
A: You should call ca_http_resp_not_found when handling an HTTP request for a resource that does not exist. This function generates an HTTP 404 Not Found response, which is appropriate for signaling that the requested resource could not be located. It is typically used in the HTTP service module when a URL or endpoint is not recognized or supported.

Code example conf:0.85 **Q:** Can you show a minimal example of how to use ca_http_resp_not_found with caOS types?
A: ca_http_response_t response; response.buffer = (u8*)0x20000000; response.buffer_size = 1024; response.content_length = 0; response.status_code = 200; ca_http_resp_not_found(&respons

Parameters conf:0.90 **Q:** What is the significance of the 'r' parameter in ca_http_resp_not_found, and what happens if it is invalid?
A: The 'r' parameter is a pointer to a ca_http_response_t structure that must be valid and properly initialized. If 'r' is NULL or points to an uninitialized structure, the function will likely cause a crash or undefined behavior. The function assumes that the structure has been correctly set up with valid buffer and size fields before being passed to it.

ca_http_resp_puti

Header: caos/net/inc/http_service.h:66
Source: caos/net/src/http_service.c:74:86
GUARD: EN_HTTP_SERVICE

```
void ca_http_resp_puti(ca_http_response_t *r, u32 v)
```

Converts an unsigned 32-bit integer to its string representation and appends it to an HTTP response buffer.

Parametri:

Nome	Tipo	Descrizione
r	ca_http_response_t *	Pointer to the HTTP response object. Must be a valid, initialized handle.
v	u32	The unsigned 32-bit integer value to be appended to the response.

Ritorna:
void

Note:

- This function is non-blocking and performs a local stack-based conversion of the integer before calling the internal append function. It is not thread-safe if multiple tasks attempt to write to the same response object simultaneously, so ensure the response object is owned by the calling task. It is generally not recommended to call this from an ISR because it performs memory operations and string manipulation that could be time-consuming. A common mistake is passing a NULL pointer for the response object, which will cause a hard fault or memory corruption when the function attempts to access the object members.

Uso:

- This function is used during the construction of HTTP responses, typically within a web server callback or a network service task. It is used to dynamically inject numeric data, such as status codes, content lengths, or sensor readings, into the HTTP payload. It is typically paired with ca_http_resp_begin to start the response, followed by a sequence of ca_http_resp_puts and ca_http_resp_puti calls to build the body, and finally finished by the internal response transmission logic.

Esempio:

```
/* Assuming 'resp' is a valid ca_http_response_t pointer */
ca_http_resp_puts(resp, "{\"status\":");
/* Append the numeric status code to the JSON body */
ca_http_resp_puti(resp, 200U);
ca_http_resp_puts(resp, ",\"value\":");
/* Append a sensor reading */
ca_http_resp_puti(resp, sensor_data_val);
ca_http_resp_puts(resp, "}");
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_http_resp_puti in an HTTP response?
A: You should use ca_http_resp_puti when you need to include integer values in an HTTP response body, such as status codes, counters, or other numeric data. It is particularly useful for building JSON or plain text responses where numeric values must be formatted as strings.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_http_resp_puti?
A: ca_http_response_t response; ca_http_resp_init(&response); ca_http_resp_puti(&response, 42); ca_http_resp_end(&response);

Parameters conf:0.90 **Q:** What does the 'v' parameter in ca_http_resp_puti represent and what are valid values?
A: The 'v' parameter is the unsigned 32-bit integer value to be converted to a string and added to the HTTP response. Valid values range from 0 to 4294967295. Invalid values would be any value outside this range, though in practice, u32 is unsigned, so all values are valid within its type limits.

Return conf:0.85 **Q:** What happens if ca_http_resp_puti fails to add the integer to the response?
A: The function does not return any value, so there is no direct way to detect failure. However, if the internal buffer is full or insufficient, the function may not complete the operation, potentially leading to data corruption or truncation. The function is described as potentially blocking and not thread-safe.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_http_resp_puti?
A: Common mistakes include calling the function with an uninitialized or invalid response pointer, which can cause a crash. Another mistake is assuming the buffer is large enough for the integer being added, which could lead to buffer overflows. Additionally, calling this function from an ISR is not allowed due to its use of memory operations and calculations.

`ca_http_resp_puts`

Header: `caos/net/inc/http_service.h:63`
Source: `caos/net/src/http_service.c:69:72`
GUARD: `EN_HTTP_SERVICE`

```
1 void ca_http_resp_puts(ca_http_response_t *r, const char *s)
```

Appends a null-terminated string to an HTTP response buffer if the response is not header-only.

Parametri:

Nome	Tipo	Descrizione
`r`	`ca_http_response_t *`	Pointer to the active HTTP response object. Must be initialized via ca_http_resp_begin before use.
`s`	`const char *`	The null-terminated C-string to append. Must not be NULL.

Ritorna:
This function returns void and does not provide an error code. If the string is empty, it effectively does nothing.

Note:

- This function is non-blocking and performs a memory copy into the internal response buffer. It is not thread-safe; you must ensure that only one task accesses the specific response object at a time. It is generally not recommended to call this from an ISR because it performs string length calculation and memory operations which can be unpredictable in timing. A common beginner mistake is passing a NULL pointer or a string that is not null-terminated, which will cause the internal ca_strlen to read past the buffer and trigger a hard fault.

Uso:

- This function is used during the body generation phase of an HTTP response. It is typically called after ca_http_resp_begin has initialized the response headers and before the response is finalized. It is often paired with ca_http_resp_puti to construct dynamic JSON or HTML content. You should use this when you have static text or pre-formatted strings to send to the client, ensuring the response object remains valid throughout the entire sequence.

Esempio:

```
ca_http_response_t *resp = /* assume initialized */;
ca_http_resp_begin(resp, 200U, "text/plain");
ca_http_resp_puts(resp, "Status: OK");
ca_http_resp_puts(resp, "\nSystem is running.");
/* Finalize response after all puts calls */
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_http_resp_puts() in my caOS HTTP server implementation?
A: You should use ca_http_resp_puts() to append string content to the body of an HTTP response, but only when the response is not in HEAD_ONLY mode. This function is useful for dynamically building the response body, such as sending HTML content or JSON data. It is typically called after setting up the HTTP response headers and before finalizing the response.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_http_resp_puts() with caOS types?
A: ca_http_response_t response; // Assume response is initialized properly const char *html_content = "<html><body>Hello World</body></html>"; ca_http_resp_puts(&response, html_content);

Parameters conf:0.90 **Q:** What are the valid and invalid values for the parameters of ca_http_resp_puts()?
A: The first parameter 'r' must be a valid pointer to an initialized ca_http_response_t structure; passing NULL will cause undefined behavior or crash. The second parameter 's' must be a valid pointer to a null-terminated string; passing NULL will cause memory access violation. Both parameters must be valid for the function to operate correctly.

`ca_http_resp_write`

Header: `caos/net/inc/http_service.h:60`
Source: `caos/net/src/http_service.c:64:67`
GUARD: `EN_HTTP_SERVICE`

```
void ca_http_resp_write(ca_http_response_t *r, const char *s, u16 len)
```

Appends a specific length of data to the body of an HTTP response buffer.

Parametri:

Nome	Tipo	Descrizione
`r`	`ca_http_response_t *`	Pointer to the initialized HTTP response object. Must not be NULL.
`s`	`const char *`	Pointer to the data buffer to be written. Must be a valid memory address.
`len`	`u16`	Number of bytes to copy from the source buffer. Ensure this does not exceed the remaining buffer capacity.

Ritorna:

void. This function does not return a status code, so it assumes the buffer has sufficient space.

Note:

- This function is non-blocking and performs a memory copy into the internal response buffer. It is not thread-safe if multiple tasks access the same response object simultaneously, so ensure the response object is owned by the current task handler. It is strictly forbidden to call this from an ISR, as it performs memory operations that are not reentrant. A common beginner mistake is passing a length that exceeds the pre-allocated buffer size, which will cause a buffer overflow or silent data truncation. Always ensure the response object has been properly initialized via ca_http_resp_begin before calling this function.

Uso:

- This function is used within HTTP request handler callbacks to populate the payload of the response. It is typically called after ca_http_resp_begin, which sets up the HTTP headers and status codes. You use this function when you have a specific block of data, such as a pre-compiled HTML page or a binary payload, that needs to be sent to the client. It is often paired with ca_http_resp_puts if you are sending null-terminated strings, but ca_http_resp_write is preferred for binary data or when the exact length is already known to avoid unnecessary string scanning.

Esempio:

```
static void handler_root(const ca_http_request_t *req, ca_http_response_t *resp)
    {
    (void)req;
    // Initialize the response headers first
    ca_http_resp_begin(resp, 200U, "text/html");

    // Define the content to send
    const char *my_page = "<html><body><h1>Hello World</h1></body></html>";

    // Write the content to the response buffer
    // Using sizeof - 1 to exclude the null terminator from the HTTP body
    ca_http_resp_write(resp, my_page, (u16)(sizeof(my_page) - 1U));
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_http_resp_write() in a caOS HTTP server implementation?
A: You should call ca_http_resp_write() to write the body content of an HTTP response, but only when the request is not a HEAD request. If the response is marked as head_only (e.g., due to a HEAD request), the function will ignore the data. It's typically used after setting up the HTTP response headers and before sending the final response.

Code example conf:0.90 **Q:** Can you provide a minimal example of how to use ca_http_resp_write() with caOS types?
A: ca_http_response_t response; response.head_only = false; const char *data = "Hello, World!"; ca_http_resp_write(&response, data, 13);

Parameters conf:0.90 **Q:** What are the valid ranges for the 'len' parameter in ca_http_resp_write()?
A: The 'len' parameter must be less than or equal to 65535 (u16 max value). If a larger value is passed, the function will truncate the data to 65535 bytes. The parameter cannot be negative, as it is of type u16.

ca_http_serve_connection

Header: caos/net/inc/http_service.h:143
Source: caos/net/src/http_service.c:347:460
GUARD: EN_HTTP_SERVICE

```
void ca_http_serve_connection(ca_http_conn_t *c, const ca_http_io_t *io)
```

Processes an incoming HTTP request on a connection and handles the corresponding response lifecycle.

Parametri:

Nome	Tipo	Descrizione
c	ca_http_conn_t *	Pointer to the connection structure containing pre-allocated RX/TX buffers and response state. Must be valid and initialized.
io	const ca_http_io_t *	Pointer to the I/O interface structure containing the transport-specific send/recv functions (e.g., TCP or TLS).

Ritorna:
void. This function does not return a status code; it handles errors internally by closing the connection or setting the 'failed' flag in the response structure.

Note:

- This function is blocking and performs multiple network read operations until the full HTTP request is received or a timeout occurs. It is NOT thread-safe for the same connection object; you must ensure only one task processes a specific connection at a time. It cannot be called from an ISR because it performs blocking I/O operations. A common mistake is passing a connection structure with insufficient buffer sizes (HTTP_REQ_BUF_SIZE/HTTP_RESP_ which will cause the request to be truncated or the response to fail. Ensure the underlying transport (TCP/TLS) is already established before calling this.

Uso:

- This function is the core engine for HTTP services. It is typically called immediately after a connection is accepted (e.g., via ca_tcp_accept). It automates the parsing of headers, waiting for the request body if necessary, and invoking the application-level routing logic. After this function returns, the caller should close the transport connection (e.g., ca_tcp_close or ca_tls_close) to release resources.

Esempio:

```
/* Assuming a TCP connection handle 'conn' has been accepted */
static ca_http_conn_t http_conn;
static ca_http_io_t http_io = { tcp_io_send, tcp_io_recv, &conn };

/* Initialize buffers inside the connection struct before use */
/* ... (code to assign buffers to http_conn.rx and http_conn.tx) ... */

/* Process the incoming HTTP request */
ca_http_serve_connection(&http_conn, &http_io);

/* Cleanup the transport layer after the HTTP transaction is complete */
ca_tcp_close(conn);
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_http_serve_connection in my application logic?
A: You should call this function immediately after a connection is accepted, such as via ca_tcp_accept. It serves as the core engine for HTTP services, automating header parsing and request body handling. Once the function returns, you should call ca_tcp_close or ca_tls_close to release resources.

Code example conf:0.90 **Q:** Provide a minimal example of how to invoke ca_http_serve_connection.
A: ca_http_conn_t conn; ca_http_io_t io; /* Initialize conn and io structures */ ca_http_serve_connection(&conn, &io); ca_tcp_close(conn.socket);

Parameters conf:1.00 **Q:** What are the requirements for the ca_http_conn_t pointer passed to ca_http_serve_connection?
A: The pointer must be valid and initialized. It must contain pre-allocated RX/TX buffers that meet the required sizes (HTTP_REQ_BUF_SIZE and HTTP_RESP_BUF_SIZE) to prevent request truncation or response failure.

Error/Edge case conf:1.00 **Q:** How does ca_http_serve_connection report errors, and what is a common mistake to avoid?
A: The function does not return a status code; it handles errors internally by closing the connection or setting the 'failed' flag in the response structure. A common mistake is providing a connection structure with insufficient buffer sizes, which leads to truncated requests or failed responses.

Behavior conf:1.00 **Q:** Is ca_http_serve_connection safe to call from an ISR or multiple threads?
A: No, it is not ISR-safe because it performs blocking I/O operations. It is also not thread-safe for the same connection object; you must ensure only one task processes a specific connection at a time.

`ca_http_service_start`

Header: `caos/net/inc/http_service.h:89`
Source: `caos/net/src/http_service.c:362:377`
GUARD: `EN_HTTP_SERVICE`

```
1 void ca_http_service_start(const ca_http_route_t *routes, u8 count, u16
     port)
```

Initializes and launches the background HTTP server task with a defined routing table and port.

Parametri:

Nome	Tipo	Descrizione
`routes`	`const ca_http_route_t *`	Pointer to an array of route structures defining URL paths and their associated callback functions. Must remain valid for the lifetime of the HTTP service.
`count`	`u8`	The number of elements in the routes array. Ensure this matches the actual array size to prevent out-of-bounds access.
`port`	`u16`	The TCP port to listen on. If set to 0, the system defaults to the standard HTTP port (usually 80).

Ritorna:
void

Note:

- This function is non-blocking; it spawns a background RTOS task and returns immediately. It is not thread-safe if called simultaneously from multiple contexts, though it contains a guard to prevent multiple starts. It must not be called from an ISR (Interrupt Service Routine) as it performs OS task creation. A common mistake is passing a local (stack-allocated) array for the routes parameter; since the HTTP task runs in the background, the array must be defined as 'static' or global to ensure it persists after the calling function returns.

Uso:

- This function is typically called during the system initialization phase or after network configuration (e.g., DHCP assignment) is complete. It is the entry point for enabling web-based management or telemetry on the device. It is often paired with network stack initialization functions like 'ca_arp_service_update_local_ip' to ensure the device has a valid IP address before the server begins listening for incoming connections.

Esempio:

```
/* Define routes globally or static to ensure they persist */
static const ca_http_route_t my_routes[] = {
    { "/status", handle_status_request },
    { "/config", handle_config_request }
};

void app_init(void) {
    /* Start the HTTP service on port 80 */
    ca_http_service_start(my_routes, 2, 80U);

    /* Check if service started successfully by verifying internal state if needed */
    /* Note: ca_http_service_start does not return an error code, so ensure parameters are valid */
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_http_service_start and what is its purpose in an embedded system?
A: You should call ca_http_service_start to initiate an HTTP service on an STM32 microcontroller that supports web-based communication. It starts a dedicated task to handle incoming HTTP requests based on the provided routes. This function is useful for creating web interfaces or REST APIs for embedded devices, allowing remote access to device functionality or data.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_http_service_start with caOS types?
A:

```
ca_http_route_t routes[2];
routes[0].path = "/status";
routes[0].handler = status_handler;
routes[1].path = "/config";
routes[1].handler = config_handler;
ca_http_service_start(routes, 2, 80);
```

Parameters conf:0.90 **Q:** What are the valid values for the 'count' parameter in ca_http_service_start, and what happens if it's invalid?
A: The 'count' parameter must be greater than 0 and not exceed the maximum number of routes allowed. Valid values range from 1 to the maximum supported route count. If 'count' is 0 or exceeds the limit, the behavior is undefined and may cause unpredictable results or system crashes.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_http_service_start, and what edge cases should I be aware of?
A: Common mistakes include calling the function with a NULL 'routes' pointer or a 'count' of 0, which leads to undefined behavior. Another mistake is calling it before the RTOS is initialized, which may cause system instability. Additionally, calling it multiple times without stopping the service first will result in the second call being ignored.

Behavior conf:0.90 **Q:** Is ca_http_service_start blocking or non-blocking, and can it be called from an ISR?
A: ca_http_service_start is a non-blocking function. It does not wait for the HTTP task to complete. It can be safely called from both thread contexts and ISRs, but it is not thread-safe if invoked multiple times without synchronization. The function will ignore subsequent calls if the service is already running.

`ca_http_update_local_ip`

Header: `caos/net/inc/http_service.h:92`
Source: `caos/net/src/http_service.c:379:382`
GUARD: `EN_HTTP_SERVICE`

```
void ca_http_update_local_ip(ca_ipv4_t ip)
```

Updates the local IPv4 address stored within the HTTP service module.

Parametri:

Nome	Tipo	Descrizione
`ip`	`ca_ipv4_t`	A structure containing the four bytes of the new IPv4 address. Ensure the address is valid and formatted correctly before passing.

Ritorna:
void

Note:

- This function is a simple memory assignment and is non-blocking, meaning it executes almost instantaneously. It is thread-safe in the sense that it performs a single atomic write, but it does not include internal mutex protection, so you should avoid calling it while the HTTP service is actively processing requests to prevent race conditions. It is safe to call from an ISR, but it is typically used in the main application thread during network configuration changes. A common mistake is calling this function before the HTTP service has been initialized, which may lead to the IP being overwritten or ignored when the service eventually starts.

Uso:

- This function is used primarily during network initialization or when a DHCP lease is renewed or changed. It is typically paired with other network service update functions like ca_arp_service_update_local_ip to ensure all protocol layers are synchronized with the new IP address. You should call this after receiving a new IP from a DHCP server but before calling ca_http_service_start to ensure the HTTP server binds to the correct interface address.

Esempio:

```
/* Example: Updating IP after a DHCP event */
ipv4_t new_ip = {{192, 168, 1, 50}};

```

```
// Update the ARP table first to ensure network reachability
ca_arp_service_update_local_ip(new_ip);

// Update the HTTP service internal state
ca_http_update_local_ip(new_ip);

// Restart or start the service to apply the new configuration
ca_http_service_start(routes, ROUTE_COUNT, 0U);
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_http_update_local_ip() and why is it important to update the local IP address?
A: You should call ca_http_update_local_ip() when the local IPv4 address of your network interface changes, such as after a DHCP lease renewal or manual IP configuration. This ensures that the HTTP service continues to function correctly with the updated network address. It's important because the HTTP service uses this IP to bind to the correct network interface and respond to client requests.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_http_update_local_ip() with caOS types?
A: u32 ip = 0xC0A80164; /* 192.168.1.100 in big-endian format */ ca_http_update_local_ip(ip);

Parameters conf:0.90 **Q:** What is the meaning of the ca_ipv4_t ip parameter in ca_http_update_local_ip(), and what are valid vs invalid values?
A: The ca_ipv4_t ip parameter represents the new IPv4 address to be set for the HTTP service. It must be a valid IPv4 address in network byte order (big-endian). Invalid values include zero, broadcast addresses, or any address that is not routable on the local network. Using invalid values may cause unpredictable behavior in the HTTP service or related network functions.

`ca_https_service_start`

Header: `caos/net/inc/https_service.h:49`
Source: `caos/net/src/https_service.c:195:226`
GUARD: `EN_HTTPS_SERVICE`

```
bool ca_https_service_start(const u8 *cert_der, u32 cert_len, const u8 *
    private_key, u16 port)
```

Initializes and starts the HTTPS server task with provided TLS credentials.

Parametri:

Nome	Tipo	Descrizione
`cert_der`	`const u8 *`	Pointer to the server certificate in DER format. Must not be NULL.
`cert_len`	`u32`	Length of the certificate in bytes. Must be greater than 0.
`private_key`	`const u8 *`	Pointer to the private key data. Must not be NULL.
`port`	`u16`	TCP port for the HTTPS service. If 0, defaults to 443.

Ritorna:
Returns true if the service was successfully started or was already running. Returns false if initialization fails (e.g., invalid parameters, memory allocation failure, or key store issues).

Note:

- This function is non-blocking as it spawns a background task to handle the HTTPS service. It is not thread-safe if called concurrently from multiple tasks, though it contains an internal check to prevent multiple starts. It must not be called from an ISR because it performs memory allocation and task creation. A common mistake is passing a pointer to a local stack variable for the certificate or key; these must persist for the entire lifetime of the HTTPS service.

Uso:

- This function is typically called during system initialization after the network stack is ready. It is used to enable secure web access to the device. It pairs with the internal task scheduler to manage the HTTPS lifecycle. It is often wrapped in a conditional macro (like EVAL_HTTPS_SVC) to ensure it only runs when the feature is enabled in the build configuration.

Esempio:

```
/* Ensure network is ready before starting HTTPS */
if (ca_dhcp_service_net_ready()) {
    /* Start HTTPS service with provided credentials */
    bool success = ca_https_service_start(my_cert_der, sizeof(my_cert_der),
    my_private_key, 443);
    if (success) {
        ca_stdout_printf_n("HTTPS service started successfully");
    } else {
        /* Handle failure: check if memory or key store is exhausted */
        ca_stdout_printf_n("Failed to start HTTPS service");
    }
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to call ca_https_service_start in my application?
A: This function should be called during system initialization, specifically after the network stack has been successfully initialized, to enable secure web access to the device.

Code example conf:0.90 **Q:** Provide a minimal C example for starting the HTTPS service.
A: const u8 cert[] = {0x30, 0x82, ...}; const u8 key[] = {0x30, 0x82, ...}; bool status = ca_https_service_start(cert, sizeof(cert), key, 443);

Parameters conf:1.00 **Q:** What are the constraints on the cert_der and private_key parameters?
A: Both pointers must not be NULL. Furthermore, they must point to memory that persists for the entire lifetime of the HTTPS service, as the service uses these pointers directly.

Return conf:1.00 **Q:** What does a return value of false indicate when calling ca_https_service_start?
A: A return value of false indicates that the service failed to start, which can be caused by invalid parameters, memory allocation failures, or issues with the provided key store.

Error/Edge case conf:1.00 **Q:** What is a common mistake developers make when providing the certificate or key to this function?
A: A common mistake is passing a pointer to a local stack variable. Since the service runs in the background, the memory must persist for the entire lifetime of the HTTPS service, not just the scope of the calling function.

Behavior conf:1.00 **Q:** Is ca_https_service_start safe to call from an interrupt service routine (ISR)?
A: No, it must not be called from an ISR because the function performs memory allocation and task creation, which are not permitted in an ISR context.

---

ca_hw_led_count

Header: caos/drivers/inc/ca_glue_hw.h:432
GUARD: EN_CA_GLUE_HW

```
extern u8 ca_hw_led_count(void); /* LEDs this board actually carries, 0
    if none */
```

Returns the total number of physical LEDs available on the current hardware board.

Ritorna:
Returns an unsigned 8-bit integer (u8) representing the count of LEDs. A return value of 0 indicates that no LEDs are physically present or configured on the board.

Note:

- This function is non-blocking and thread-safe, as it simply returns a constant value defined at compile-time based on the hardware configuration. It is safe to call from both standard tasks and Interrupt Service Routines (ISRs). The most common mistake is assuming that the returned value is a bitmask; it is a simple count, so if the board has 3 LEDs, the valid indices for other LED functions are 1, 2, and 3. Always check this value before attempting to iterate through LEDs to avoid out-of-bounds logic in your application code.

Uso:

- This function is typically used during system initialization to dynamically discover hardware capabilities. You would call it to determine the loop bounds when performing a 'power-on self-test' (POST) where you blink all available LEDs to verify hardware health. It is often paired with ca_led_on, ca_led_off, and ca_led_toggle to ensure that your control logic does not attempt to access non-existent hardware pins.

Esempio:

```
/* Example: Blink all available LEDs once during system startup */
#include "caos.h"

void system_led_test(void) {
    u8 led_count = ca_hw_led_count();

    if (led_count > 0) {
        for (u8 i = 1; i <= led_count; i++) {
            /* Turn on each LED to verify hardware */
            ca_led_on(i);
        }

        /* Wait for 500ms to show the status */
        ca_os_delay_ms(500);

        for (u8 i = 1; i <= led_count; i++) {
            /* Turn off each LED */
            ca_led_off(i);
        }
    } else {
        /* Log error: No LEDs found on this hardware */
        ca_printf("No LEDs available to test.\n");
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_hw_led_count in my caOS application?
A: You should call it during system initialization to dynamically discover hardware capabilities. It is primarily used to determine loop bounds for hardware health checks, such

as a power-on self-test (POST) where you blink all available LEDs, or to ensure your control logic avoids accessing non-existent hardware pins.

Code example conf:1.00 **Q:** How do I write a loop to blink all available LEDs using caOS types?
A: You can use the following pattern: u8 count = ca_hw_led_count(); for(u8 i = 1; i <= count; i++) { ca_led_on(i); }

Return conf:1.00 **Q:** What does the return value of ca_hw_led_count represent, and what does 0 mean?
A: The function returns a u8 representing the total count of physical LEDs on the board. A return value of 0 indicates that no LEDs are physically present or configured on the hardware. Note that the return value is a count, not a bitmask; if the board has 3 LEDs, the valid indices for other LED functions are 1, 2, and 3.

`ca_hw_led_get`

Header: `caos/drivers/inc/ca_glue_hw.h:437`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_hw_led_get(u8 led_num); /* logical state (on/off), not
    the pin level */
```

Reads the current logical state (ON or OFF) of a specific LED.

Parametri:

Nome	Tipo	Descrizione
`led_num`	`u8`	The index of the LED to query, as defined in your board configuration. Must be a valid index within the range [1, ca_hw_led_count()].

Ritorna:
Returns true if the LED is logically ON, and false if it is logically OFF. If the provided index is invalid or the LED system is not configured, it returns false.

Note:

- This function is non-blocking and thread-safe, as it performs a direct read of the underlying GPIO pin. It is safe to call from an ISR, provided the underlying GPIO hardware access is also ISR-safe. The function abstracts away the hardware polarity; if your board is 'active-low', the function automatically inverts the physical pin state so that 'true' always represents the 'ON' state. A common mistake is assuming this function returns the physical voltage level of the pin; always remember it returns the logical state defined by your board's configuration.

Uso:

- Use this function when you need to verify the current status of an indicator before performing an action, such as toggling it or logging its state. It is typically used in conjunction with ca_led_on, ca_led_off, or ca_led_toggle to implement state-dependent logic. In a typical application, you might initialize the LED system, then periodically check the state using ca_led_get to update a UI or a status message via ca_printf.

Esempio:

```
/* Example: Toggle LED 1 only if it is currently OFF */
#include "caos.h"

void check_and_toggle_led(void) {
    u8 my_led = 1;

```

```
    /* Check the logical state of the LED using the public API */
    if (ca_led_get(my_led) == false) {
        /* LED is OFF, turn it ON */
        ca_led_on(my_led);
    } else {
        /* LED is ON, turn it OFF */
        ca_led_off(my_led);
    }

    /* Log the action to the system console */
    ca_printf("LED %d toggled successfully.\n", my_led);
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_hw_led_get in my caOS application?
A: You should use ca_hw_led_get when you need to verify the current status of an indicator before performing an action, such as toggling it or logging its state. It is typically used in conjunction with ca_led_on, ca_led_off, or ca_led_toggle to implement state-dependent logic.

Code example conf:1.00 **Q:** How do I check if an LED is on and print its status using caOS types?
A: You can check the state and log it as follows: u8 my_led = 0; if (ca_hw_led_get(my_led)) { ca_printf("LED is ON\n"); } else { ca_printf("LED is OFF\n"); }

Parameters conf:1.00 **Q:** What are the constraints for the led_num parameter in ca_hw_led_get?
A: The led_num parameter represents the index of the LED as defined in your board configuration. It must be a valid index within the range [0, CA_LED_COUNT - 1].

Return conf:1.00 **Q:** What do the return values of ca_hw_led_get signify?
A: The function returns true if the LED is logically ON and false if it is logically OFF. If the provided index is invalid or the LED system is not configured, it returns false.

Error/Edge case conf:1.00 **Q:** Is it a mistake to assume ca_hw_led_get returns the physical voltage level of the pin?
A: Yes, that is a common mistake. The function abstracts away hardware polarity; if your board is 'active-low', the function automatically inverts the physical pin state so that 'true' always represents the 'ON' state, regardless of the actual voltage level.

`ca_hw_led_init`

Header: `caos/drivers/inc/ca_glue_hw.h:433`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_hw_led_init(void); /* configure every declared LED as
    output, all off */
```

Initializes all hardware LEDs defined in the system configuration to a safe, off state.

Ritorna:
This function returns void and does not provide status feedback.

Note:

- This function is intended to be called once during the system startup sequence, typically before the RTOS scheduler starts. It is not thread-safe and must not be called from an ISR, as it performs multiple GPIO register configurations that are not atomic. A common mistake is calling this function multiple times during runtime; doing so will reset the LED pins to their default state, potentially causing a visible flicker or interrupting an ongoing

LED pattern. Ensure that the hardware abstraction layer (HAL) for the GPIO ports is ready before invoking this function.

Uso:

- Use this function during the board initialization phase, usually inside the main() function or a dedicated hardware setup task. It is typically paired with ca_led_on, ca_led_off, or ca_led_toggle to control the LEDs after the system has successfully booted. If you need to verify the hardware state later, you can use ca_led_count to determine how many LEDs were initialized.

Esempio:

```c
int main(void) {
    /* Initialize system clocks and memory before hardware */
    ca_os_tick_init();
    ca_memory_init();

    /* Initialize all LEDs to a known 'off' state */
    ca_hw_led_init();

    /* Now it is safe to use LED control functions */
    ca_led_on(0);
    ca_os_delay_ms(500);
    ca_led_off(0);

    /* Start the OS scheduler */
    ca_os_run();

    return 0;
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to call ca_hw_led_init in a caOS project?
A: It should be called once during the system startup sequence, typically inside the main() function or a dedicated hardware setup task before the RTOS scheduler starts.

Code example conf:1.00 **Q:** How do I initialize the LEDs in my main function using caOS types?
A: You simply call the function directly during initialization: int main(void) { ca_hw_led_init(); /* Proceed with scheduler start */ return 0; }

Error/Edge case conf:1.00 **Q:** What happens if I call ca_hw_led_init multiple times during runtime?
A: Calling it multiple times will reset the LED pins to their default state, which may cause a visible flicker or interrupt any ongoing LED patterns. Additionally, it is not thread-safe and must not be called from an ISR.

`ca_hw_led_off`

Header: `caos/drivers/inc/ca_glue_hw.h:435`
GUARD: `EN_CA_GLUE_HW`

```c
extern void ca_hw_led_off(u8 led_num); /* caller guarantees 1 <= led_num <= count */
```

Turns off a specific LED by setting its associated GPIO pin to the inactive state.

Parametri:

Nome	Tipo	Descrizione
`led_num`	`u8`	The index of the LED to turn off. Must be between 1 and the total number of LEDs defined in the system configuration.

Ritorna:
void

Note:

- This function is non-blocking and executes immediately by writing directly to the hardware register. It is thread-safe assuming the underlying GPIO driver is configured correctly, but it is not intended for complex synchronization. It can be called from an ISR, provided the GPIO hardware access is atomic. The most common beginner mistake is passing an index outside the valid range (e.g., 0 or a value greater than the hardware count), which will lead to an out-of-bounds memory access in the internal LED descriptor array. Always verify the index against ca_led_count() before calling this function if the index is dynamic.

Uso:

- This function is typically used to provide visual feedback to the user or to signal system states (e.g., turning off an 'activity' LED after a task completes). It is usually paired with ca_led_init during system startup to configure the pins, and ca_led_on (a function that sets the LED to the active state) to turn the LED back on. In a typical application, you would call the active-state function at the start of a long-running operation and ca_led_off once the operation finishes or if an error occurs. It is often used in conjunction with ca_os_delay_ms to create blinking patterns or status indicators.

Esempio:

```c
/* Initialize LED system before use */
ca_led_init();

/* Turn on LED 1 to indicate a process has started */
/* Note: Use the corresponding active-state function here */
/* ca_led_on(1); */

/* Perform some system task */
ca_os_delay_ms(500);

/* Turn off LED 1 to indicate the process is finished */
/* Always validate the index against the hardware count */
if (1 <= ca_led_count()) {
    ca_led_off(1);
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_hw_led_off in my application?
A: You should use ca_hw_led_off to provide visual feedback or signal system states. It is typically used to turn off an activity LED after a long-running operation completes or if an error occurs, and is usually paired with ca_led_init for configuration and ca_led_on for activation.

Code example conf:1.00 **Q:** How do I turn off the first LED in my system using caOS types?
A: You can turn off the first LED by calling the function with the index 1: ca_hw_led_off((u8)1);. Ensure that your system configuration defines at least one LED before calling this.

Parameters conf:1.00 **Q:** What are the constraints for the led_num parameter in ca_hw_led_off?
A: The led_num parameter must be a u8 value between 1 and the total number of LEDs defined in the system configuration. Passing 0 or a value greater than the hardware count

is a common mistake that causes an out-of-bounds memory access in the internal LED descriptor array.

---

ca_hw_led_on

Header: caos/drivers/inc/ca_glue_hw.h:434
GUARD: EN_CA_GLUE_HW

```
extern void ca_hw_led_on(u8 led_num); /* caller guarantees 1 <= led_num
   <= count */
```

Turns on a specific LED by setting its associated GPIO pin to the active state.

Parametri:

Nome	Tipo	Descrizione
led_num	u8	The index of the LED to turn on. Must be between 1 and the total number of LEDs defined in the system configuration.

Ritorna:
void

Note:

- This function is non-blocking and executes immediately by writing to the underlying GPIO register. It is thread-safe assuming the underlying GPIO driver is configured correctly, but it is not intended for high-frequency toggling inside tight loops. The most common mistake is passing an index outside the valid range (e.g., 0 or a value greater than the hardware count), which may lead to memory access violations or undefined behavior depending on the implementation of the LED descriptor array. Ensure the LED system is initialized before calling this function.

Uso:

- Used to provide visual feedback to the user, such as indicating system status, error states, or activity. It is typically paired with ca_led_off or ca_led_toggle to create blink patterns. In a typical application, you would call ca_led_init during system startup, then use ca_led_on within task loops or event handlers to signal state changes.

Esempio:

```
/* Initialize the LED system before use */
ca_led_init();

/* Turn on LED 1 to indicate system is starting */
ca_led_on(1);

/* Wait for a short period to show the LED */
ca_os_delay_ms(500);

/* Turn off the LED */
ca_led_off(1);

/* Example of error handling: only turn on if a specific condition is met */
if (ca_queue_heap_is_empty()) {
    ca_led_on(1);
} else {
    /* Turn off the LED if the queue is not empty */
    ca_led_off(1);
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_hw_led_on in my caOS application?
A: You should call ca_hw_led_on to provide visual feedback, such as indicating system status, error states, or activity. It is typically used within task loops or event handlers after the LED system has been initialized via ca_led_init.

Code example conf:1.00 **Q:** How do I turn on the first LED using caOS types?
A: Assuming the system is initialized, you can turn on the first LED with the following code: u8 led_id = 1; ca_hw_led_on(led_id);

Parameters conf:1.00 **Q:** What are the constraints for the led_num parameter in ca_hw_led_on?
A: The led_num parameter must be a u8 value between 1 and the total number of LEDs defined in the system configuration. Passing 0 or a value greater than the hardware count is invalid and may lead to memory access violations or undefined behavior.

`ca_hw_led_toggle`

Header: `caos/drivers/inc/ca_glue_hw.h:436`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_hw_led_toggle(u8 led_num); /* caller guarantees 1 <=
    led_num <= count */
```

Toggles the state of a specific hardware LED.

Parametri:

Nome	Tipo	Descrizione
`led_num`	`u8`	The index of the LED to toggle. Must be between 1 and the total number of LEDs defined in the system configuration.

Ritorna:
void

Note:

- This function is non-blocking and executes immediately by modifying the underlying GPIO state. It is thread-safe provided the underlying GPIO hardware access is protected by the OS, but it is generally safe to call from both tasks and ISRs. The most common beginner mistake is passing an index outside the valid range (e.g., 0 or a value greater than the hardware count), which will cause an out-of-bounds memory access. Ensure the LED hardware has been initialized via the appropriate board support package before calling this function.

Uso:

- This function is typically used for visual feedback, such as heartbeat indicators, error signaling, or debugging state changes. It is often paired with ca_os_delay_ms to create blinking patterns in a dedicated task. In a typical lifecycle, you would initialize the LED system during boot, then call this function periodically within a loop or in response to specific system events to indicate activity.

Esempio:

```
/* Example: Blink an LED every 500ms as a system heartbeat */
void heartbeat_task(void *pvParameters) {
    /* Assume LED 1 is the status indicator */
    const u8 status_led = 1;

```

```
    while (1) {
        /* Toggle the LED state using the hardware-specific function */
        ca_hw_led_toggle(status_led);

        /* Wait for 500ms before toggling again */
        ca_os_delay_ms(500);

        /* Reset watchdog to prevent system reset */
        ca_os_thread_reset_watchdog();
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** What is the intended use case for ca_hw_led_toggle in a caOS application?
A: This function is used for visual feedback, such as heartbeat indicators, error signaling, or debugging state changes. It is typically called periodically within a loop or in response to system events, often paired with ca_os_delay_ms to create blinking patterns.

Code example conf:0.90 **Q:** How do I implement a simple heartbeat blinker using ca_hw_led_toggle?
A: You can toggle the LED and wait for a duration. Example: void task_heartbeat(void) { while(1) { ca_hw_led_toggle(1); ca_os_delay_ms(500); } }

Parameters conf:1.00 **Q:** What are the constraints for the led_num parameter in ca_hw_led_toggle?
A: The led_num parameter must be a u8 value between 1 and the total number of LEDs defined in the system configuration. Passing a value of 0 or a value greater than the hardware count will result in an out-of-bounds memory access.

`ca_hw_memmap_get`

Header: `caos/drivers/inc/ca_glue_hw.h:425`
GUARD: `EN_CA_GLUE_HW`

```
extern u32 ca_hw_memmap_get(ca_mem_range *out, u32 max); /* physical
   memory areas of this MCU; returns how many were written */
```

Retrieves the physical memory layout (start/end addresses) defined for the current MCU.

Parametri:

Nome	Tipo	Descrizione
`out`	`ca_mem_range *`	Pointer to an array of ca_mem_range structures where the memory map will be copied. Must not be NULL.
`max`	`u32`	The maximum number of entries the 'out' buffer can hold. Prevents buffer overflows.

Ritorna:
Returns the number of memory regions actually written to the 'out' buffer. Returns 0 if 'out' is NULL, 'max' is 0, or if no memory regions are defined.

Note:

- This function is non-blocking and thread-safe as it only reads from a static, read-only configuration table. It is safe to call from an ISR because it performs no complex operations or synchronization. The most common mistake is providing a buffer that is too small to hold all regions, which results in a truncated list; always ensure your buffer size matches or exceeds the number of regions expected by the hardware configuration.

Uso:

- This function is typically used during system initialization or diagnostic routines to verify memory boundaries before performing memory-intensive operations like heap allocation or DMA transfers. It is often paired with ca_memory_get_start_address or ca_memory_get_end_address to validate that pointers fall within valid physical memory ranges. Developers use it to build a dynamic view of the system's memory map for logging or safety checks.

Esempio:

```c
/* Example: Retrieve and print the system memory map */
#define MAX_REGIONS 8

void check_memory_layout(void) {
    ca_mem_range map[MAX_REGIONS];
    /* Retrieve the memory map using the hardware-specific function */
    u32 count = ca_hw_memmap_get(map, MAX_REGIONS);

    if (count == 0) {
        /* Handle error: no memory map found or invalid buffer */
        return;
    }

    for (u32 i = 0; i < count; i++) {
        /* Use ca_printf to log the start and end of each memory region */
        ca_printf("Region %d: 0x%08X - 0x%08X\n", i, map[i].start, map[i].end);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_hw_memmap_get in my caOS application?
A: You should call it during system initialization or diagnostic routines. It is used to verify memory boundaries before performing memory-intensive operations like heap allocation or DMA transfers, or to build a dynamic view of the system's memory map for safety checks.

Code example conf:1.00 **Q:** How do I implement a basic check of the memory map using caOS types?
A: You can retrieve the map into a local array as follows: ca_mem_range map[4]; u32 count = ca_hw_memmap_get(map, 4); if (count > 0) { /* process map entries */ }

Parameters conf:1.00 **Q:** What are the constraints for the 'out' and 'max' parameters in ca_hw_memmap_get?
A: The 'out' parameter must be a non-NULL pointer to an array of ca_mem_range structures. The 'max' parameter specifies the capacity of the 'out' buffer to prevent overflows; if 'max' is 0, the function will return 0.

Return conf:1.00 **Q:** What does the return value of ca_hw_memmap_get represent and when does it return 0?
A: The return value is the number of memory regions actually written to the 'out' buffer. It returns 0 if 'out' is NULL, 'max' is 0, or if no memory regions are defined in the hardware configuration.

`ca_i2c_bus_recovery`

Header: `caos/drivers/inc/i2c.h:82`
Source: `caos/drivers/src/i2c.c:138:149`
GUARD: `EN_I2C`

```c
bool ca_i2c_bus_recovery(u8 num)
```

Attempts to recover an I2C bus that is stuck due to a slave holding the SDA line low.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The I2C peripheral index (1 to 4). Must correspond to a previously initialized I2C bus.

Ritorna:
Returns true if the bus was successfully recovered and is now idle, false if the recovery failed or the peripheral index is invalid.

Note:

- This function is not thread-safe and should not be called from an ISR. It performs a hardware-level reset of the I2C peripheral, which is necessary because older STM32 I2C hardware (like F1/F4 series) can get stuck in a 'BUSY' state if a transfer is interrupted. The function relies on internal state tracking (s_i2c_up, s_i2c_alt, etc.) initialized by ca_i2c_init; if you call this on an uninitialized bus, it will return false. A common beginner mistake is calling this function repeatedly in a tight loop without checking the return value, which can lead to system instability if the physical hardware fault (e.g., a short circuit) persists.

Uso:

- Use this function when your I2C communication consistently fails or returns errors, indicating that a slave device has hung the bus. It is typically used in an error-handling routine after detecting a timeout or a bus error. The standard sequence is: 1) Detect failure, 2) Call ca_i2c_bus_recovery, 3) If it returns true, attempt to re-initialize or re-ping the slave devices. It pairs with ca_i2c_init, which must have been called previously to populate the internal configuration structures.

Esempio:

```c
/* Assuming I2C_MASTER is defined as 1 */
#define I2C_MASTER 1U

void handle_i2c_failure(void) {
    /* Attempt to recover the bus after a communication error */
    bool success = ca_i2c_bus_recovery(I2C_MASTER);

    if (success) {
        ca_printf_n("I2C bus recovered successfully.");
        /* Proceed to re-initialize or re-communicate with slaves */
    } else {
        ca_printf_n("I2C bus recovery failed. Check physical connections.");
        /* Perform system-level error handling or reset */
    }
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_i2c_bus_recovery in my application?
A: You should call it when I2C communication consistently fails or returns errors, suggesting a slave device has hung the bus by holding the SDA line low. It is typically used in an error-handling routine after detecting a timeout or bus error.

Code example conf:1.00 **Q:** Provide a minimal example of how to use ca_i2c_bus_recovery in caOS.
A: if (!ca_i2c_bus_recovery(1)) { /* Handle recovery failure */ } else { /* Re-ping slave devices */ }

Parameters conf:1.00 **Q:** What are the valid values for the 'num' parameter in ca_i2c_bus_recovery?
A: The 'num' parameter represents the I2C peripheral index and must be a value between 1 and 4. The bus must have been previously initialized using ca_i2c_init for the function to work.

Return conf:1.00 **Q:** What does the boolean return value of ca_i2c_bus_recovery indicate?
A: It returns true if the bus was successfully recovered and is now idle. It returns false if the recovery failed or if the provided peripheral index is invalid or uninitialized.

Error/Edge case conf:1.00 **Q:** What are the common mistakes or risks when using ca_i2c_bus_recovery?
A: A common mistake is calling the function repeatedly in a tight loop without checking the return value, which can lead to system instability if a physical hardware fault like a short circuit persists. Additionally, the function is not thread-safe and must not be called from an ISR.

`ca_i2c_err_handler`

Header: `caos/drivers/inc/i2c.h:74`
Source: `caos/drivers/src/i2c.c:160:167`
GUARD: `EN_I2C`

```
1 void ca_i2c_err_handler(u32 i2c_num)
```

Handles hardware-level I2C bus errors by invoking the appropriate low-level recovery routine.

Parametri:

Nome	Tipo	Descrizione
`i2c_num`	`u32`	The I2C peripheral index. Must be an integer between 1 and 4 inclusive. Values outside this range trigger a system safety failure.

Ritorna:
void. This function does not return a status code; it relies on the internal hardware handler to manage the error state.

Note:

- This function is designed to be called exclusively from the I2C Error Interrupt Service Routine (ISR). It is not thread-safe in the sense that it directly manipulates hardware registers, so it must only be triggered by the hardware interrupt controller. Because it is called within an ISR context, it must execute extremely quickly; it does not block, but it performs low-level register resets which can disrupt ongoing communication. A common mistake is calling this function from a standard task thread instead of the ISR, which can lead to race conditions or inconsistent hardware states. Always ensure the I2C peripheral has been properly initialized via the system driver before this handler is allowed to run.

Uso:

- This function is the primary entry point for the I2C error interrupt vector. In a typical lifecycle, it is paired with the I2C hardware initialization and the standard data transfer handlers. When the STM32 hardware detects a bus error (like an Arbitration Loss or Bus Error), the CPU jumps to the I2C_ER_IRQHandler, which then calls this function to perform the necessary hardware-specific recovery steps. It is part of the 'init -> monitor -> recover' loop, ensuring that if a bus hang occurs, the driver can attempt to reset the peripheral state without crashing the entire RTOS.

Esempio:

```c
/* This is the standard I2C Error Interrupt Service Routine */
void I2C1_ER_IRQHandler(void) {
    /* Call the caOS error handler for I2C peripheral 1 */
    ca_i2c_err_handler(1);

    /* After recovery, ensure the interrupt flag is cleared if necessary */
    /* Note: ca_i2c_hw_err_handler usually handles register clearing internally
    */
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_i2c_err_handler in my embedded application?
A: You should call ca_i2c_err_handler when an I2C error occurs, typically from an ISR (Interrupt Service Routine) to handle hardware-level I2C errors. It is a non-blocking function designed to delegate error handling to hardware-specific implementations.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_i2c_err_handler with caOS types?
A: u32 i2c_num = 2; ca_i2c_err_handler(i2c_num);

Parameters conf:0.90 **Q:** What is the valid range for the i2c_num parameter in ca_i2c_err_handler?
A: The i2c_num parameter must be a value between 1 and 4 inclusive. Values outside this range will cause the function to return immediately without performing any action.

`ca_i2c_hw_bus_recovery`

Header: `caos/drivers/inc/ca_glue_hw.h:182`
GUARD: `EN_CA_GLUE_HW`

```c
extern bool ca_i2c_hw_bus_recovery(u8 num, u8 alternate); /* clocks a
    slave stuck on SDA free on the pins of this num/alternate, then hands
     them back to the peripheral; true = bus idle */
```

Forces an I2C bus to release the SDA line by manually toggling the SCL clock, then re-initializes the pins for peripheral use.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The I2C peripheral index (e.g., 1, 2, 3, 4). Must be within the valid range for the specific MCU hardware.
`alternate`	`u8`	The pin mapping configuration index. If the value exceeds the hardware-defined maximum, it defaults to 0.

Ritorna:
Returns true if the bus was found idle (SDA high) after recovery attempts, or false if the bus remains stuck or if the peripheral index is invalid.

Note:

- This function is blocking as it performs manual bit-banging to toggle the SCL line. It is NOT thread-safe and must not be called from an ISR, as it relies on timing and GPIO manipulation that could conflict with ongoing peripheral operations. It is typically used during system initialization or after a detected I2C bus error to recover from a slave device holding the SDA line low (a common scenario after a CPU reset). The most common mistake is calling this while the I2C peripheral is still active or enabled, which can cause electrical contention; ensure the peripheral is disabled before calling this.

Uso:

- This function is used as a recovery mechanism during the I2C initialization sequence. It is typically called after configuring the clocks but before enabling the I2C peripheral hardware. The standard flow is: 1) Initialize the I2C clock, 2) Call ca_i2c_hw_bus_recovery to clear any stuck slaves, 3) Call ca_gpio_hw_i2c_init to set the pins to their Alternate Function (AF) mode, and 4) Proceed to initialize the I2C peripheral registers.

Esempio:

```
/* Example: Recovering a stuck I2C bus before initialization */
u8 i2c_num = 1;
u8 alt = 0;

/* 1. Ensure the peripheral clock is enabled (using internal system function) */
/* ca_clock_hw_i2c_init(i2c_num); */

/* 2. Attempt to recover the bus. This will toggle SCL to force the slave to
    release SDA. */
bool bus_is_idle = ca_i2c_hw_bus_recovery(i2c_num, alt);

if (bus_is_idle) {
    /* 3. If idle, proceed to configure pins for I2C peripheral usage */
    /* ca_gpio_hw_i2c_init(i2c_num, alt); */

    /* 4. Initialize the I2C peripheral */
    if (ca_i2c_init(i2c_num, 100000)) {
        /* Bus is ready for communication */
    }
} else {
    /* Handle error: Bus is still stuck, possibly a hardware short */
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_i2c_hw_bus_recovery in my initialization sequence?
A: It should be called during the I2C initialization sequence, specifically after configuring the clocks but before calling ca_gpio_hw_i2c_init and enabling the I2C peripheral hardware. This ensures any stuck slave devices holding the SDA line low are cleared before the peripheral takes control of the pins.

Code example conf:1.00 **Q:** Provide a minimal C example for initializing I2C1 with bus recovery.
A: u8 bus = 1; u8 alt = 0; /* Initialize clocks here */ if (ca_i2c_hw_bus_recovery(bus, alt)) { /* Bus is idle, proceed to peripheral init */ }

Parameters conf:1.00 **Q:** What happens if I pass an invalid 'alternate' value to ca_i2c_hw_bus_recovery?
A: If the 'alternate' parameter exceeds the hardware-defined maximum for the specific MCU, the function will automatically default the value to 0 before proceeding with the recovery process.

Return conf:1.00 **Q:** What does the boolean return value of ca_i2c_hw_bus_recovery indicate?
A: The function returns true if the bus is found to be idle (SDA high) after the recovery attempts. It returns false if the bus remains stuck or if the provided peripheral index (num) is invalid for the specific MCU hardware.

Error/Edge case conf:1.00 **Q:** What is the most common mistake when using ca_i2c_hw_bus_recovery?
A: The most common mistake is calling the function while the I2C peripheral is still active or enabled. This can cause electrical contention. You must ensure the peripheral is disabled before calling this function.

HW differences conf:1.00 **Q:** How does the implementation of ca_i2c_hw_bus_recovery differ between the F1xx family and other families like F4xx or H5xx?
A: In the F1xx family, the function uses a switch-case to manually map specific GPIO ports and pins (e.g., PB6/PB7 for I2C1). In contrast, the F4xx, F7xx, H5xx, H7xx,

and U5xx families use a lookup table (s_i2c_pins) based on the 'num' and 'alternate' parameters to determine the pin mapping, providing a more generic approach.

---

ca_i2c_hw_enable_irq

Header: caos/drivers/inc/ca_glue_hw.h:180
GUARD: EN_CA_GLUE_HW

```
extern void ca_i2c_hw_enable_irq(u8 num); /* abilita gli interrupt I2C (
    evento ed errore) nel NVIC */
```

Enables I2C peripheral interrupts in the NVIC and configures the hardware registers to trigger them.

Parametri:

Nome	Tipo	Descrizione
num	u8	The I2C peripheral index (e.g., 1 for I2C1, 2 for I2C2). Must correspond to a valid hardware peripheral on the specific STM32 chip; invalid values will result in no action.

Ritorna:
void

Note:

- This function is non-blocking and executes immediately by modifying hardware registers and the NVIC (Nested Vectored Interrupt Controller). It is thread-safe in the sense that it performs atomic register writes, but it should be called during the initialization phase before the I2C peripheral is actively used to avoid race conditions. It is callable from an ISR, though typically it is called from the main thread during system setup. A common beginner mistake is calling this function before the peripheral clock has been enabled or before the GPIO pins have been configured, which will lead to a hard fault or an unresponsive I2C bus.

Uso:

- This function is used exclusively when the application requires interrupt-driven I2C communication rather than polling. It is typically called as part of the driver initialization sequence, following the clock configuration and GPIO setup, and immediately after the peripheral initialization function. It pairs with the I2C interrupt service routines (ISRs) defined in the system, which will handle the actual data transfer once the hardware triggers an interrupt.

Esempio:

```
/* Initialize I2C1 for interrupt-based operation */
u8 i2c_num = 1;
/* Perform hardware clock and GPIO setup using internal system functions */
/* ca_clock_hw_i2c_init(i2c_num); */
/* ca_gpio_hw_i2c_init(i2c_num); */

/* Initialize the I2C peripheral */
if (ca_i2c_init(i2c_num, 400000)) {
    /* Enable interrupts in NVIC and hardware registers */
    ca_i2c_hw_enable_irq(i2c_num);
} else {
    /* Handle initialization failure */
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_i2c_hw_enable_irq() in my I2C application?
A: You should call ca_i2c_hw_enable_irq() when you want to enable hardware interrupt handling for I2C events and errors. This function is typically called after initializing the I2C peripheral but before starting communication, to allow the ISR to handle I2C events like data transfer completion or error conditions.

Code example conf:0.95 **Q:** Can you show a minimal example of how to use ca_i2c_hw_enable_irq() with caOS types?
A:

```
u8 i2c_num = 1;
ca_i2c_hw_enable_irq(i2c_num);
```

Parameters conf:0.95 **Q:** What does the 'num' parameter in ca_i2c_hw_enable_irq() represent and what are valid values?
A: The 'num' parameter represents the I2C peripheral number (1, 2, 3, or 4). Valid values are 1, 2, 3, and 4 depending on the MCU family. Values 0 or anything outside this range will cause the function to return immediately without performing any action.

Error/Edge case conf:0.95 **Q:** What happens if I pass an invalid value to ca_i2c_hw_enable_irq()?
A: If an invalid value (0 or values outside 1-4) is passed to ca_i2c_hw_enable_irq(), the function will return immediately without performing any action. It does not generate an error or exception; it simply ignores invalid inputs.

Behavior conf:0.95 **Q:** Is ca_i2c_hw_enable_irq() thread-safe and ISR-safe?
A: No, ca_i2c_hw_enable_irq() is neither thread-safe nor ISR-safe. It is a blocking function that modifies NVIC and peripheral registers. It must be called from a non-critical context, not from an ISR, and should not be used in environments where multiple threads might access it concurrently.

HW differences conf:0.90 **Q:** How does ca_i2c_hw_enable_irq() differ between F1xx and F7xx MCUs in terms of register usage?
A: On F1xx MCUs, ca_i2c_hw_enable_irq() enables interrupts by setting bits in the I2C_CR2 register (I2C_CR2_ITBUFEN | I2C_CR2_ITEVTEN | I2C_CR2_ITERREN). On F7xx MCUs, it enables interrupts by setting bits in the I2C_CR1 register (I2C_CR1_RXIE | I2C_CR1_ERRIE). This reflects differences in how the two MCU families handle I2C interrupt control.

`ca_i2c_hw_err_handler`

Header: `caos/drivers/inc/ca_glue_hw.h:183`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_i2c_hw_err_handler(u8 num); /* gestore IRQ I2C errore
    slave: resetta il bus */
```

Resets the I2C peripheral error flags to recover the bus from hardware-level communication faults.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The index of the I2C peripheral to reset. Must be a valid peripheral ID supported by the hardware configuration (e.g., 1 to 4).

Ritorna:
void

Note:

- This function is designed to be called directly from an Interrupt Service Routine (ISR), specifically the I2C Error Interrupt handler. It is non-blocking and executes immediately by writing directly to the hardware registers. It is not thread-safe in the sense that it modifies hardware state, so it must only be triggered by the hardware interrupt logic. A common mistake is calling this function during normal operation without an actual error condition, which could potentially clear flags that were about to be processed or disrupt an ongoing transaction. Ensure the peripheral index passed is valid, as passing an invalid index will lead to undefined behavior or a system crash when accessing the peripheral base address.

Uso:

- This function is used exclusively within the I2C error interrupt service routine (e.g., I2C_ER_IRQHandler). When the I2C hardware detects a bus error, arbitration loss, or NACK, it triggers an interrupt; this function is then called to clear those specific error flags, allowing the peripheral to resume normal operation. It is typically paired with higher-level driver logic that logs the error and potentially restarts the I2C state machine. You should never call this in your main application loop; it is strictly a low-level recovery mechanism for the hardware driver layer.

Esempio:

```
void I2C1_ER_IRQHandler(void) {
    /* Check if the error interrupt flag is set in the hardware */
    /* If an error is detected, clear the hardware flags to recover */
    ca_i2c_hw_err_handler(1);

    /* After clearing, notify the OS or driver state machine that a reset
    occurred */
    /* Perform additional cleanup or re-initialization if necessary */
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_i2c_hw_err_handler() and what is its purpose?
A: You should call ca_i2c_hw_err_handler() from an ISR (Interrupt Service Routine) to handle hardware errors on an I2C bus. Its purpose is to reset error flags such as BERR, ARLO, AF (NACK), and OVR in the I2C peripheral, effectively clearing the error state of the bus. It is intended for use only in interrupt contexts and must not be called from regular task code or in multi-threaded environments.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_i2c_hw_err_handler(with caOS types?
A:

```
u8 i2c_num = 1;
ca_i2c_hw_err_handler(i2c_num);
```

This example demonstrates calling the function with a valid I2C interface number (1 in this case). The function does not return a value and is meant to be used directly within an ISR.

Parameters conf:0.95 **Q:** What does the 'num' parameter in ca_i2c_hw_err_handler() represent, and what are the valid values?
A: The 'num' parameter represents the I2C interface number, ranging from 1 to 4. Values outside this range (less than 1 or greater than 4) are invalid and cause undefined behavior. It is used to select the correct I2C peripheral base address for clearing error flags.

Return conf:0.90 **Q:** Does ca_i2c_hw_err_handler() return any value, and can it fail?

A: The function does not return any value. It is designed to never fail, but it may not perform any action if an invalid parameter is passed. The function is guaranteed to not fail, but it is not thread-safe and must only be called from an ISR context.

HW differences conf:0.90 **Q:** How does ca_i2c_hw_err_handler() differ in implementation across different STM32 MCU families?

A: The function uses different hardware mechanisms depending on the MCU family. For F1/F4 families, it clears error flags by writing to the SR1 register. For F7/H5/H7 families, it clears error flags by writing to the ICR register. This difference reflects the varying register layouts and reset mechanisms across STM32 families.

`ca_i2c_hw_init_periph`

Header: `caos/drivers/inc/ca_glue_hw.h:176`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_i2c_hw_init_periph(u8 num, ca_i2c_speed_t speed); /*
   configura il periferico I2C con la velocit  indicata */
```

Initializes the hardware registers for a specific I2C peripheral instance at a requested clock speed.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The I2C peripheral index (e.g., 1, 2, 3). Must correspond to a valid hardware peripheral on the specific STM32 chip.
`speed`	`ca_i2c_speed_t`	The desired bus frequency (e.g., standard mode 100kHz, fast mode 400kHz). Must be a valid enum value supported by the hardware.

Ritorna:
Returns true if the peripheral was successfully configured and enabled. Returns false if the peripheral index is invalid or the hardware failed to initialize.

Note:

- This function is non-blocking and executes quickly as it only writes to memory-mapped registers. It is not thread-safe; you must ensure no other task or interrupt is accessing the same I2C peripheral during initialization. It is generally safe to call from an ISR, but it is intended for system startup or driver initialization phases. A common mistake is calling this function before enabling the peripheral's clock via the RCC (Reset and Clock Control) unit, which will result in a hard fault or silent failure because the registers will not be accessible.

Uso:

- This function is the core hardware setup step in the I2C driver lifecycle. It is typically called after the GPIO pins have been configured for alternate function mode and the peripheral clock has been enabled. It is usually paired with 'ca_i2c_hw_enable_irq' if interrupt-driven communication is required. You should call this during the system initialization sequence before attempting any I2C data transfers, as the peripheral remains disabled until this function completes.

Esempio:

```
/* Example: Initializing I2C1 at 400kHz */
u8 i2c_instance = 1;
```

```
ca_i2c_speed_t speed = CA_I2C_SPEED_FAST; /* 400kHz */

/* 1. Enable clocks and GPIOs first */
ca_clock_hw_i2c_init(i2c_instance);
ca_gpio_hw_i2c_init(i2c_instance);

/* 2. Configure the hardware peripheral */
if (ca_i2c_hw_init_periph(i2c_instance, speed)) {
    /* Success: peripheral is now enabled and ready */
} else {
    /* Error: handle initialization failure */
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_i2c_hw_init_periph() in my embedded application?
A: You should call ca_i2c_hw_init_periph() during system initialization or when setting up I2C communication for the first time. It configures the I2C peripheral with the specified speed and enables it. This function is typically called after ensuring the I2C clock is enabled and the GPIO pins are configured for I2C mode.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_i2c_hw_init_periph() with caOS types?
A:

```
u8 i2c_num = 1;
ca_i2c_speed_t speed = I2C_SPEED_400KHZ;
bool result = ca_i2c_hw_init_periph(i2c_num, speed);
/* result is always true per documentation */
```

Parameters conf:0.90 **Q:** What are the valid values for the 'num' parameter in ca_i2c_hw_init_periph()?
A: The 'num' parameter must be a value between 1 and 4 inclusive. Values outside this range (like 0 or >4) will cause undefined behavior as the function does not validate input and may access invalid memory addresses.

Return conf:0.95 **Q:** What does the return value of ca_i2c_hw_init_periph() indicate?
A: The function always returns true. It does not indicate success or failure of the hardware configuration. Even if invalid parameters are passed, the function will not return false, and the hardware may not be correctly configured.

HW differences conf:0.85 **Q:** How does ca_i2c_hw_init_periph() differ between F7xx/H5xx/H7xx and F1xx/F4xx families?
A: For F1xx and F4xx families, the function uses i2c_set_speed_v1() to configure timing based on I2C_APB1_MHZ. For F7xx, H5xx, and H7xx families, it directly sets the TIMINGR register using i2c_timingr_compute(). This reflects differences in I2C peripheral architecture between these STM32 families.

`ca_i2c_hw_recv_handler`

Header: `caos/drivers/inc/ca_glue_hw.h:182`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_i2c_hw_recv_handler(u8 num); /* gestore IRQ I2C evento
   slave: legge byte ricevuti */
```

Low-level interrupt service routine handler for I2C slave peripheral events.

Parametri:

Nome	Tipo	Descrizione
num	u8	The I2C peripheral index (typically 1-4). Must correspond to a valid, initialized I2C hardware instance.

Ritorna:
void

Note:

- This function is designed to be called exclusively from within an I2C Interrupt Service Routine (ISR). It is not thread-safe and must not be called from main-loop code, as it directly manipulates hardware registers and relies on the atomic nature of interrupt execution. Because it runs in an ISR context, it must be extremely fast; performing heavy processing or blocking operations inside the callback (rx_cb) will cause the I2C bus to stall, potentially leading to clock stretching timeouts or data loss. A common beginner mistake is to perform long operations (like logging or complex calculations) inside the rx_cb, which violates the real-time constraints of the I2C protocol.

Uso:

- This function is the backbone of the caOS I2C slave driver. It is invoked by the hardware-specific interrupt vector (e.g., I2C1_EV_IRQHandler) whenever the I2C peripheral signals an event such as data reception, transmission request, or address match. It manages the state machine of the slave, including clearing hardware flags, filling the transmit buffer, and triggering the user-defined callback (rx_cb) once a full message is received. It is typically paired with a configuration function that sets up the slave address and registers the callback function.

Esempio:

```
/* Example of how the ISR calls the handler */
void I2C1_EV_IRQHandler(void) {
    /* Call the hardware-specific handler to process I2C events */
    ca_i2c_hw_recv_handler(1);
}

/* Example of setting up the slave context */
void setup_i2c_slave(void) {
    ca_i2c_slave_ctx_t *ctx = &g_i2c_slave_ctx[0];
    ctx->is_slave = 1;
    ctx->rx_cb = my_data_received_callback; /* Function to process data */
    /* Hardware initialization follows... */
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should caOS developers call ca_os_i2c_hw_recv_handler()?
A: This function is intended to be called from an I2C interrupt handler to process incoming data and manage the I2C state machine. It handles events like address matching, data reception, transmission, and error conditions for I2C slave operations.

Code example conf:0.85 **Q:** Can you provide a minimal C example using caOS types to demonstrate how to use ca_os_i2c_hw_recv_handler()?
A:

```
u8 i2c_buffer[10];
ca_i2c_slave_ctx_t ctx = { .rx_buf = i2c_buffer, .rx_len = 10, .rx_idx = 0 };
void i2c_irq_handler(void) {
    ca_os_i2c_hw_recv_handler(0);
}
```

Parameters conf:0.90 **Q:** What does the 'num' parameter in ca_os_i2c_hw_recv_handler() represent?
A: The 'num' parameter represents the I2C peripheral number (e.g., 0 for I2C1, 1 for I2C2), identifying which I2C hardware instance is being serviced by the interrupt handler.

Return conf:0.95 **Q:** Does ca_os_i2c_hw_recv_handler() return anything meaningful?
A: No, this function has a void return type. It performs internal state updates and data handling directly within the I2C context without returning status codes.

Error/Edge case conf:0.90 **Q:** What are common error conditions that ca_os_i2c_hw_recv_handler() handles?
A: It handles Bus Error (BERR), Overrun (OVR), and Arbitration Lost (ARLO) conditions by clearing the respective flags in the I2C peripheral and resetting internal indices to prevent lockups.

Behavior conf:0.90 **Q:** Is ca_os_i2c_hw_recv_handler() safe to call from an ISR and thread-safe?
A: Yes, it's designed to be ISR-safe and can be safely invoked from an I2C interrupt handler. It is also thread-safe as long as each I2C instance is handled by a single interrupt context.

HW differences conf:0.85 **Q:** How do the implementations of ca_os_i2c_hw_recv_handler() differ between H7 and H5 families?
A: The H7 version includes more explicit handling of clock stretching during address reception and additional checks for NACKF, while H5 uses a simpler approach with fewer conditional branches. Both versions manage TXIS, ADDR, RXNE, STOPF, and NACKF flags similarly.

`ca_i2c_hw_slave_init`

Header: `caos/drivers/inc/ca_glue_hw.h:181`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_i2c_hw_slave_init(u8 num, u8 addr); /* configura il
   periferico I2C in modalit  slave con indirizzo proprio */
```

Initializes the specified I2C peripheral hardware to operate as an I2C slave device.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The I2C peripheral index (e.g., 1 for I2C1). Must correspond to a valid hardware instance on the specific MCU.
`addr`	`u8`	The 7-bit I2C slave address to assign to this device. Ensure this does not conflict with other devices on the same bus.

Ritorna:
Returns true if the initialization was successful, or false if the peripheral index is invalid or unsupported by the hardware configuration.

Note:

- This function is non-blocking and performs low-level register configuration to prepare the I2C peripheral for slave mode. It is not thread-safe and should be called during the system initialization phase before any I2C communication begins. It is not intended to be called from an ISR. A common mistake is calling this function before the peripheral clock or GPIO pins have been configured, which will result in the I2C peripheral failing to respond to bus traffic.

Uso:

- This function is typically used in the driver initialization sequence for I2C slave mode. It is usually preceded by calls to configure the peripheral clock and the specific GPIO pins required for I2C (SDA/SCL). After calling this function, the system expects the I2C hardware to be ready to trigger interrupts when a master addresses the device, which are then handled by the OS interrupt service routines.

Esempio:

```c
/* Initialize I2C1 as a slave with address 0x42 */
u8 i2c_num = 1;
u8 slave_addr = 0x42;

/* 1. Enable peripheral clock and GPIOs first */
ca_clock_hw_i2c_init(i2c_num);
ca_gpio_hw_i2c_init(i2c_num);

/* 2. Configure hardware for slave mode */
if (ca_i2c_hw_slave_init(i2c_num, slave_addr)) {
    /* Success: hardware is ready for bus activity */
} else {
    /* Handle error: invalid peripheral index or hardware failure */
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_i2c_hw_slave_init() in my embedded application?
A: You should call ca_i2c_hw_slave_init() when you want the microcontroller to act as an I2C slave device, responding to requests on a specific I2C address. This is typically used when your device needs to communicate with a master device as a peripheral, such as in sensor networks or device-to-device communication.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_i2c_hw_slave_init() with caOS types?
A: u8 result = ca_i2c_hw_slave_init(1, 0x2C); /* Initialize I2C1 as slave with address 0x2C */ if (result) { /* Initialization successful */ } else { /* Initialization failed */ }

Parameters conf:0.90 **Q:** What does the 'num' parameter represent in ca_i2c_hw_slave_init() and what are valid values?
A: The 'num' parameter specifies the I2C peripheral number. For F103 MCUs, only I2C1 (num=1) is supported. For other MCU families, valid values are 1-4. The parameter must match the hardware capabilities of the specific MCU being used.

Return conf:0.95 **Q:** What does ca_i2c_hw_slave_init() return and what causes it to return false?
A: The function returns true if initialization is successful, and false if the parameters are invalid. It returns false if the I2C peripheral number is not supported for the current MCU or if the address is invalid (outside the 0x00-0x7F range).

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_i2c_hw_slave_init()?
A: Common mistakes include using an I2C address that is already occupied on the bus, calling the function before initializing the I2C clock and GPIOs, or calling it from within an ISR. Additionally, not enabling the IRQs via ca_i2c_hw_enable_irq() after calling this function can lead to missed interrupts.

HW differences conf:0.90 **Q:** How does ca_i2c_hw_slave_init() differ between F103 and F7xx MCUs?
A: On F103, the function only supports I2C1 and sets up basic registers including FREQ and OAR1, enabling ACK and PE bits. On F7xx, the function sets up OAR1 with OA1EN bit and enables multiple interrupt sources (ADDRIE, RXIE, TXIE, STOPIE, NACKIE) in addition to PE, providing more detailed interrupt control.

---

ca_i2c_hw_transfer7

Header: caos/drivers/inc/ca_glue_hw.h:177
GUARD: EN_CA_GLUE_HW

```
extern bool ca_i2c_hw_transfer7(u8 num, u8 addr, const u8 *w, u16 wn, u8 *r, u16 rn); /* transazione I2C completa (write+read) con indirizzo a 7 bit */
```

Performs a complete I2C transaction (write, read, or combined) using a 7-bit device address.

Parametri:

Nome	Tipo	Descrizione
num	u8	The I2C peripheral index (e.g., 0 for I2C1). Must correspond to a valid initialized peripheral.
addr	u8	The 7-bit target device address. Do not include the R/W bit; the driver handles this automatically.
w	const u8 *	Pointer to the buffer containing data to write. Can be NULL if wn is 0.
wn	u16	Number of bytes to write. Set to 0 if no write operation is required.
r	u8 *	Pointer to the buffer where read data will be stored. Can be NULL if rn is 0.
rn	u16	Number of bytes to read. Set to 0 if no read operation is required.

Ritorna:
Returns true if the entire transaction (write and/or read) completed successfully. Returns false if the device NACKed, a bus error occurred, or the transaction timed out.

Note:

- This function is blocking, meaning the calling task will be suspended by the RTOS while waiting for the hardware to complete the I2C transfer. It is not thread-safe by itself; if multiple tasks access the same I2C bus, you must protect this call with a ca_mutex. It is strictly forbidden to call this from an ISR because it relies on blocking mechanisms that cannot function in an interrupt context. Always ensure the peripheral is initialized via ca_i2c_init before use. A common mistake is passing a buffer pointer that is not accessible by the DMA controller (if the driver uses DMA) or providing a NULL pointer while specifying a non-zero length, which will lead to memory access violations.

Uso:

- This is the core engine for all I2C communication in caOS. It is used by higher-level functions like ca_i2c_write and ca_i2c_read to perform specific bus operations. In a typical application, you call ca_i2c_init during system startup, then use this function within a dedicated task to poll sensors or write configuration data to peripherals. It is often used in a sequence: first, write the register address to the device, then perform a read to retrieve the data, ensuring the bus is held until the transaction is complete.

Esempio:

```
/* Example: Reading a 1-byte register from an I2C sensor at address 0x68 */
u8 reg_addr = 0x75; /* WhoAmI register */
```

```
u8 data = 0;

/* Perform a write followed by a read (combined transaction) */
if (ca_i2c_hw_transfer7(0, 0x68, &reg_addr, 1, &data, 1)) {
    /* Success: data contains the register value */
} else {
    /* Handle error: device might be disconnected or bus is busy */
    ca_i2c_err_handler(0);
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_i2c_hw_transfer7() in my caOS project?
A: You should use ca_i2c_hw_transfer7() when you need to perform a complete I2C transaction involving both writing and reading data to/from a device with a 7-bit address. It is ideal for communicating with sensors, EEPROMs, or any I2C peripheral that requires a write followed by a read operation. This function is designed for blocking operations, so it's suitable for use in task threads where blocking is acceptable.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_i2c_hw_transfer7() to read 2 bytes from an I2C device at address 0x48?
A:

```
u8 data[2];
bool success = ca_i2c_hw_transfer7(0, 0x48, NULL, 0, data, 2);
if (success) {
    // Process data[0] and data[1]
}
```

Parameters conf:0.95 **Q:** What are the valid ranges for the 'num' and 'addr' parameters in ca_i2c_hw_transfer7()?
A: The 'num' parameter must be 0, 1, or 2, corresponding to I2C1, I2C2, and I2C3 respectively. An invalid value can cause memory access violations. The 'addr' parameter must be between 0x00 and 0x7F, representing a valid 7-bit I2C address. Using an invalid address may result in communication errors.

Return conf:0.90 **Q:** What does it mean when ca_i2c_hw_transfer7() returns false?
A: A return value of false indicates that the I2C transaction failed due to communication errors such as NACK, timeout, or other hardware issues. The function is blocking, so it will not return until the transaction completes or fails.

HW differences conf:0.85 **Q:** How does the implementation of ca_i2c_hw_transfer7() differ between F1xx and F7xx families?
A: On F1xx and F4xx families, the implementation calls separate functions i2c_write7_v1() and i2c_read7_v1() for write and read operations respectively. On F7xx, H5xx, and H7xx families, the function directly calls i2c_transfer7_v2() which likely uses a unified transfer routine. This suggests that F7xx and later families may have a more optimized or unified approach to handling I2C transfers.

`ca_i2c_init`

Header: `caos/drivers/inc/i2c.h:51`
Source: `caos/drivers/src/i2c.c:90:107`
GUARD: `EN_I2C`

```
bool ca_i2c_init(const ca_i2c_config_t *cfg)
```

Initializes the hardware I2C peripheral, including clock, GPIO pins, and communication speed settings.

Parametri:

Nome	Tipo	Descrizione
`cfg`	`const ca_i2c_config_t *`	Pointer to a configuration structure containing the I2C instance number (1-4), bus speed, and interrupt request flag. Must not be NULL.

Ritorna:
Returns true if the peripheral was successfully initialized, false if the configuration pointer is invalid, the I2C number is out of range, or the hardware peripheral failed to initialize.

Note:

- This function is non-blocking and performs hardware-level setup. It is not thread-safe and must be called during the system initialization phase (e.g., inside ca_os_configure) before any read or write operations are attempted. It is not safe to call this from an Interrupt Service Routine (ISR) because it performs multiple hardware configuration steps that may take time. A common beginner mistake is to call this function multiple times for the same I2C instance without de-initializing it, which can lead to undefined hardware states or resource conflicts.

Uso:

- This function is the mandatory first step for any I2C communication in caOS. It is typically used in the system configuration phase to prepare the hardware before the RTOS scheduler starts. It is usually paired with peripheral-specific functions like ca_i2c_hw_init_periph for low-level setup and, if interrupts are enabled, it prepares the system to handle callbacks. Once initialized, the I2C bus is ready for data transfer operations.

Esempio:

```
ca_i2c_config_t i2c_cfg = { .i2c_num = 1, .speed = 100000, .irq_req =
    I2C_DISABLE_IRQ };

if (ca_i2c_init(&i2c_cfg)) {
    // Initialization successful, safe to proceed with I2C transactions
} else {
    // Handle error: check if i2c_num is valid or if hardware is busy
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_i2c_init() in my embedded application?
A: You should call ca_i2c_init() once during system initialization to configure and enable an I2C bus. It must be called before any other I2C operations on that bus. This function is typically called in your main initialization routine or startup code.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_i2c_init() with caOS types?
A: ca_i2c_config_t config = { .i2c_num = 1, .speed = CA_I2C_SPEED_STANDARD }; bool success = ca_i2c_init(&config); if (!success) { /* Handle error */ }

Parameters conf:0.90 **Q:** What do the i2c_num and speed fields in ca_i2c_config_t represent, and what are valid values?
A: The i2c_num field specifies which I2C peripheral to initialize (1 or 2 for STM32). The speed field sets the communication speed, either CA_I2C_SPEED_STANDARD (100 kHz) or CA_I2C_SPEED_FAST (400 kHz). Using values outside these ranges will cause initialization to fail.

Return conf:0.95 **Q:** What does ca_i2c_init() return, and what causes it to return false?
A: It returns true on successful initialization. It returns false if the configuration is invalid, such as passing a NULL pointer for cfg, using an unsupported i2c_num (e.g., 3 on STM32F4), or specifying an invalid speed value.

Error/Edge case conf:0.90 **Q:** What are common mistakes or edge cases when calling ca_i2c_init()?
A: Common mistakes include calling ca_i2c_init() more than once on the same bus, passing a NULL pointer for cfg, using an unsupported i2c_num (like 3 on STM32F4), or setting an invalid speed value. Calling this function from an ISR or from multiple threads simultaneously also leads to errors.

Behavior conf:0.95 **Q:** Is ca_i2c_init() blocking or non-blocking, and is it safe to call from an ISR or from multiple threads?
A: ca_i2c_init() is a blocking function that does not return until initialization completes or fails. It is not ISR-safe and must not be called from an ISR. It is also not thread-safe and should not be called from multiple threads concurrently.

`ca_i2c_read`

Header: `caos/drivers/inc/i2c.h:57`
Source: `caos/drivers/src/i2c.c:114:117`
GUARD: `EN_I2C`

```
1 bool ca_i2c_read(u8 i2c_num, u8 addr, u8 *buf, u16 len)
```

Reads a specified number of bytes from an I2C slave device into a provided buffer.

Parametri:

Nome	Tipo	Descrizione
`i2c_num`	`u8`	The hardware I2C peripheral index (e.g., 1 for I2C1). Must match the hardware configuration.
`addr`	`u8`	The 7-bit I2C slave address. Valid range is 0x08 to 0x77; addresses outside this range are reserved.
`buf`	`u8 *`	Pointer to the memory buffer where the received data will be stored. Must be large enough to hold 'len' bytes.
`len`	`u16`	Number of bytes to read. A value of 0 is technically valid but performs no operation.

Ritorna:
Returns true if the I2C transaction completed successfully (ACK received from slave). Returns false if the device did not respond (NACK), a bus error occurred, or the timeout was reached.

Note:

- This function is blocking, meaning the CPU will wait here until the entire data transfer is finished or an error occurs. It is not safe to call this from an Interrupt Service Routine (ISR) because blocking operations can cause system deadlocks or missed interrupts. Ensure the I2C peripheral has been initialized via the system configuration before calling this. A common beginner mistake is passing a pointer to a buffer that is too small, leading to memory corruption, or calling this function too frequently without a delay, which can saturate the I2C bus.

Uso:

- This function is typically used to fetch sensor data, read configuration registers, or perform bus scanning to detect connected peripherals. In a standard workflow, you first initialize the I2C bus, then use this function to read data from a specific slave address. It is often paired with ca_i2c_write when you need to send a register address first (write) and then read the data back (read). In diagnostic contexts, it is used in a loop to probe the bus for active devices by checking if the function returns true for specific addresses.

Esempio:

```c
/* Example: Reading 2 bytes from a sensor at address 0x48 */
u8 data_buffer[2];
u8 sensor_addr = 0x48;
u8 i2c_bus = 1;

if (ca_i2c_read(i2c_bus, sensor_addr, data_buffer, 2)) {
    /* Success: process the data */
    u16 raw_value = (data_buffer[0] << 8) | data_buffer[1];
} else {
    /* Error: device not found or bus busy */
    /* Handle error, e.g., log to console or retry */
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_i2c_read to communicate with an I2C device?
A: You should use ca_i2c_read when you need to read data from an I2C device in a blocking manner. It is suitable for scenarios where the calling thread can wait until the data transfer completes or times out. This function is ideal for reading sensor data, configuration registers, or any data that does not require real-time response.

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_i2c_read with caOS types?
A: u8 buffer[10]; bool success = ca_i2c_read(0, 0x48, buffer, 10); if (success) { // Process the read data in buffer } else { // Handle read failure }

Parameters conf:0.90 **Q:** What are the valid values for the i2c_num and addr parameters in ca_i2c_read?
A: The i2c_num parameter must be 0, 1, or 2, corresponding to I2C1, I2C2, or I2C3 respectively. Invalid values may cause a crash or undefined behavior. The addr parameter must be a 7-bit I2C address ranging from 0x08 to 0x77. Addresses outside this range may lead to communication errors.

Return conf:0.95 **Q:** What does it mean when ca_i2c_read returns false?
A: A return value of false indicates that the read operation failed. This can be caused by a timeout, a device that is not responding, a configuration error, or an invalid buffer pointer. It is important to check the return value and handle the failure appropriately in your application.

`ca_i2c_recover_pins`

Header: `caos/drivers/inc/ca_glue_hw.h:183`
GUARD: `EN_CA_GLUE_HW`

```c
extern bool ca_i2c_recover_pins(ca_gpio_port_t scl_p, ca_gpio_pin_t
    scl_n, ca_gpio_port_t sda_p, ca_gpio_pin_t sda_n); /* hw/common/
    i2c_recover.c: the 9-pulse + STOP sequence, pins left as open-drain
    GPIO */
```

Forces an I2C bus to release the SDA line by generating up to 9 clock pulses and a STOP condition.

Parametri:

Nome	Tipo	Descrizione
`scl_p`	`ca_gpio_port_t`	GPIO port handle for the SCL line.
`scl_n`	`ca_gpio_pin_t`	GPIO pin number for the SCL line.
`sda_p`	`ca_gpio_port_t`	GPIO port handle for the SDA line.
`sda_n`	`ca_gpio_pin_t`	GPIO pin number for the SDA line.

Ritorna:
Returns true if the bus is successfully recovered (both lines high), false otherwise.

Note:

- This function is non-blocking in the sense that it does not wait for OS events, but it performs a busy-wait sequence of pulses, so it will hold the CPU for the duration of the recovery. It is NOT thread-safe; if multiple tasks access the same I2C bus, you must ensure mutual exclusion before calling this. It is generally not safe to call from an ISR because it relies on timing loops that might be interrupted, causing the recovery sequence to fail. The most common mistake is calling this while the I2C peripheral is still enabled in the hardware; you must ensure the pins are configured as GPIOs before calling this, otherwise, the hardware peripheral will fight your manual toggling.

Uso:

- Use this function when an I2C transaction fails due to a slave device holding the SDA line low (a common 'stuck bus' scenario). It is typically called during the initialization phase (ca_i2c_init) or inside an error handler (ca_i2c_err_handler) when a timeout or bus error is detected. After calling this, you should re-initialize the I2C peripheral to restore normal communication.

Esempio:

```
/* Example: Recovering a stuck I2C bus before re-initializing */
if (ca_i2c_recover_pins(GPIOA, 0U, GPIOA, 1U)) {
    /* Bus recovered, safe to re-init the peripheral */
    ca_i2c_init(I2C1);
} else {
    /* Bus still stuck, hardware failure likely */
    ca_printf("Critical: I2C bus recovery failed!\n");
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_i2c_recover_pins function in my caOS application?
A: You should use this function when an I2C transaction fails due to a 'stuck bus' scenario, where a slave device is holding the SDA line low. It is typically called during the initialization phase (ca_i2c_init) or within an error handler (ca_i2c_err_handler) when a bus error or timeout is detected.

Code example conf:1.00 **Q:** Provide a minimal example of how to call ca_i2c_recover_pins in caOS.
A: Ensure the I2C peripheral is disabled and pins are configured as GPIOs before calling: ca_i2c_recover_pins(GPIOA, 9U, GPIOA, 10U);. After recovery, you should re-initialize the I2C peripheral to restore normal communication.

Parameters conf:1.00 **Q:** What do the parameters scl_p, scl_n, sda_p, and sda_n represent?
A: These parameters represent the GPIO port handles (ca_gpio_port_t) and pin numbers (ca_gpio_pin_t) for the SCL and SDA lines, respectively. They are used to manually toggle the pins to perform the recovery sequence.

Return conf:1.00 **Q:** What do the return values of ca_i2c_recover_pins indicate?
A: The function returns true if the bus is successfully recovered (both SCL and SDA lines are high). It returns false if the bus remains in an error state after the recovery sequence is completed.

Error/Edge case conf:1.00 **Q:** What is the most common mistake when calling ca_i2c_recover_pins?
A: The most common mistake is calling the function while the I2C hardware peripheral is still enabled. You must ensure the pins are configured as GPIOs before calling this, otherwise, the hardware peripheral will conflict with the manual toggling performed by the function.

Behavior conf:1.00 **Q:** Is ca_i2c_recover_pins thread-safe or ISR-safe?

A: It is not thread-safe; you must ensure mutual exclusion if multiple tasks access the same I2C bus. It is generally not safe to call from an ISR because it relies on timing loops that might be interrupted, causing the recovery sequence to fail. Additionally, it is non-blocking regarding OS events but holds the CPU for the duration of the busy-wait pulse sequence.

`ca_i2c_recv_handler`

Header: `caos/drivers/inc/i2c.h:71`
Source: `caos/drivers/src/i2c.c:151:158`
GUARD: `EN_I2C`

```
void ca_i2c_recv_handler(u32 i2c_num)
```

Interrupt service routine handler for I2C peripheral reception events.

Parametri:

Nome	Tipo	Descrizione
`i2c_num`	`u32`	The hardware index of the I2C peripheral (valid range: 1 to 4). Values outside this range trigger a system failure.

Ritorna:
void. This function does not return a value as it is designed to be called directly by the hardware interrupt vector table.

Note:

- This function is strictly intended to be called from within an Interrupt Service Routine (ISR). It is not thread-safe in the sense that it should never be called from a standard task context, as it performs low-level hardware register manipulation specific to the I2C peripheral state machine. Because it runs in an interrupt context, it must be extremely fast; performing heavy processing here will cause system jitter or missed interrupts. A common beginner mistake is attempting to call this function manually from the main loop or a task; doing so will likely corrupt the I2C state machine and cause the peripheral to hang or trigger a hard fault.

Uso:

- This function is the bridge between the STM32 hardware interrupt and the caOS I2C driver logic. It is typically placed inside the actual C-level interrupt handler (e.g., I2C1_EV_IRQHandler) generated by the STM32 startup code. It is used to notify the driver that the hardware has received data, allowing the driver to move bytes from the peripheral registers into the internal software buffer. It is part of the low-level driver infrastructure and is not meant to be used by application-level code, which should instead use higher-level functions like 'ca_i2c_read' or 'ca_i2c_write'.

Esempio:

```
/* Inside the STM32 interrupt vector file (e.g., stm32f4xx_it.c) */
void I2C1_EV_IRQHandler(void) {
    /* Notify the caOS driver that an I2C event occurred on peripheral 1 */
    /* This handles the hardware-to-buffer data transfer automatically */
    ca_i2c_recv_handler(1);
}

/* Note: Never call ca_i2c_recv_handler(1) from your main() loop or tasks. */
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_i2c_recv_handler in my embedded application?
A: You should call ca_i2c_recv_handler only from an I2C interrupt service routine (ISR). It is designed to handle incoming data on an I2C bus and must be invoked exclusively within an ISR context. It is not intended for use in regular application code or RTOS tasks.

Code example conf:0.90 **Q:** Can you provide a minimal example of how to use ca_i2c_recv_handler in an ISR?
A: void I2C1_IRQHandler(void) { ca_i2c_recv_handler(1); }

Parameters conf:0.90 **Q:** What does the i2c_num parameter in ca_i2c_recv_handler represent, and what values are valid?
A: The i2c_num parameter specifies the I2C peripheral number, ranging from 1 to 4. Valid values are 1, 2, 3, and 4. Using any other value will result in an error being logged, but the system will not crash.

Return conf:0.85 **Q:** Does ca_i2c_recv_handler return any value, and what causes it to fail?
A: The function does not return any value. It is void. It cannot fail in a traditional sense, but it may ignore invalid inputs. If an invalid i2c_num is passed, an error is logged, but the system does not halt.

`ca_i2c_slave_init`

Header: `caos/drivers/inc/i2c.h:87`
Source: `caos/drivers/src/i2c.c:126:140`
GUARD: `EN_I2C`

```
bool ca_i2c_slave_init(u8 i2c_num, u8 slave_addr, ca_i2c_slave_rx_cb cb,
    u8 alternate)
```

Initializes a specific I2C peripheral to operate in slave mode with a provided callback for incoming data.

Parametri:

Nome	Tipo	Descrizione
`i2c_num`	`u8`	The I2C peripheral index (1 to 4). Values outside this range will cause the function to return false.
`slave_addr`	`u8`	The 7-bit I2C address assigned to this device. Ensure this does not conflict with other devices on the same bus.
`cb`	`ca_i2c_slave_rx_cb`	A function pointer to the callback executed when data is received from the master. Must be a valid function address.
`alternate`	`u8`	The GPIO alternate function mapping index required for the specific STM32 pins used by the I2C peripheral.

Ritorna:
Returns true if the peripheral was successfully configured and initialized. Returns false if the i2c_num is invalid or if the underlying hardware initialization fails.

Note:

- This function is non-blocking and sets up the hardware registers, clock, GPIO, and internal context structures required for slave operation. It is not thread-safe if called simultaneously

for the same peripheral from different threads, so ensure initialization happens during system startup. It is not intended to be called from an ISR, as it performs hardware clock and GPIO initialization which may involve delays or resource locking. A common beginner mistake is failing to define the callback function correctly or passing a NULL pointer, which will lead to a hard fault when the master sends data and the system attempts to execute the callback.

Uso:

- This function is used when your STM32 needs to act as a peripheral device (e.g., a sensor or display controller) responding to an I2C master. It is typically called once during the system initialization phase before the RTOS scheduler starts or before the main loop begins. It is paired with the master's initialization functions to establish the communication link. Once initialized, the system relies on the provided callback to process incoming data asynchronously, meaning the main application thread does not need to poll the I2C bus constantly.

Esempio:

```c
static void on_slave_rx(u8 num, const u8 *buf, u8 len) {
    /* Process received data here. This runs in ISR context. */
}

int main(void) {
    u8 my_addr = 0x42;
    u8 alt_func = 4; // Example alternate function mapping

    /* Initialize I2C2 as a slave with our callback */
    if (!ca_i2c_slave_init(2, my_addr, on_slave_rx, alt_func)) {
        /* Handle initialization error: log or enter safe state */
        while(1);
    }

    /* Proceed with other system initializations */
    return 0;
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_i2c_slave_init in my embedded application?
A: You should call ca_i2c_slave_init once during system initialization, before any I2C communication occurs on the specified bus. It configures the I2C peripheral to operate in slave mode with a specific address and sets up the receive callback.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_i2c_slave_init with caOS types?
A: u8 slave_addr = 0x55; ca_i2c_slave_rx_cb my_callback = my_rx_handler; bool success = ca_i2c_slave_init(2, slave_addr, my_callback); if (!success) { // Handle initialization error }

Parameters conf:0.90 **Q:** What are the valid ranges for i2c_num and slave_addr in ca_i2c_slave_init?
A: The i2c_num must be between 1 and 4 inclusive; values outside this range will cause initialization to fail. The slave_addr is a 7-bit I2C address, so it must be between 0x00 and 0x7F.

Return conf:0.95 **Q:** What does it mean if ca_i2c_slave_init returns false?
A: A return value of false indicates that the initialization failed. This can happen if i2c_num is invalid (not 1-4), or if the hardware initialization of the I2C peripheral fails.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_i2c_slave_init?
A: Common mistakes include passing an invalid i2c_num (e.g., 0 or >4), not properly initializing the GPIO or clock for the I2C bus, or passing a NULL callback function. Also,

calling this function after other I2C operations on the same bus may lead to undefined behavior.

---

ca_i2c_slave_set_tx

Header: caos/drivers/inc/i2c.h:68
Source: caos/drivers/src/i2c.c:141:149
GUARD: EN_I2C

```
void ca_i2c_slave_set_tx(u8 i2c_num, const u8 *buf, u8 len)
```

Prepares a data buffer to be sent by the I2C peripheral when the master requests a read.

Parametri:

Nome	Tipo	Descrizione
i2c_num	u8	The I2C peripheral index (1 to 4). Values outside this range are ignored.
buf	const u8 *	Pointer to the source data array to be copied into the internal slave transmit buffer.
len	u8	Number of bytes to copy. If len exceeds the internal buffer size, it is truncated to the maximum buffer capacity.

Ritorna:
void. No status is returned, so the caller must ensure the buffer pointer is valid before calling.

Note:

- This function is non-blocking and performs a memory copy from your provided buffer into the driver's internal transmit buffer. It is thread-safe in the sense that it only updates local context variables, but it is not re-entrant; if called simultaneously from two tasks for the same I2C peripheral, the buffer state will be corrupted. It is safe to call from an ISR, as it does not perform any hardware I/O or blocking operations. The most common mistake is assuming this function triggers the transmission immediately; it does not. It merely 'stages' the data so that when the I2C master sends a read request, the hardware has data ready to shift out.

Uso:

- This function is used in the context of an I2C slave device responding to a master's request. Typically, you call this inside an I2C receive callback or a data processing task after you have parsed a command from the master and prepared a response. The typical sequence is: 1) Initialize the I2C slave using ca_i2c_hw_slave_init, 2) Wait for data from the master, 3) Process the received data, 4) Call ca_i2c_slave_set_tx to load the response, and 5) The hardware handles the actual transmission automatically when the master clocks the bus.

Esempio:

```
/* Example: Responding to a master request */
void on_i2c_rx_callback(u8 num, u8 *rx_data, u8 rx_len) {
    u8 response[4];
    // Prepare response data based on received command
    response[0] = 0xAA;
    response[1] = rx_data[0] + 1;
    response[2] = 0x00;
    response[3] = 0xFF;

```

```
    // Stage the data for the next read request from the master
    ca_i2c_slave_set_tx(num, response, 4);
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_i2c_slave_set_tx in an I2C slave application?
A: You should call ca_i2c_slave_set_tx when you need to prepare data to be sent in response to an I2C slave read request. It is a non-blocking function that can be called from any context, including ISRs, making it suitable for use in interrupt handlers or task contexts where you want to respond to incoming I2C data requests without blocking the thread.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_i2c_slave_set_tx with caOS types?
A: u8 tx_buffer[8] = {0x10, 0x20, 0x30, 0x40}; ca_i2c_slave_set_tx(1, tx_buffer, 4);

Parameters conf:0.90 **Q:** What are the valid ranges for the i2c_num parameter in ca_i2c_slave_set_tx, and what happens if I pass an invalid value?
A: The i2c_num parameter must be a value between 1 and 4 inclusive. If a value outside this range is passed, the function will return immediately without performing any action or causing a crash. This is a safety measure to prevent undefined behavior on non-existent I2C peripherals.

`ca_i2c_write`

Header: `caos/drivers/inc/i2c.h:75`
Source: `caos/drivers/src/i2c.c:109:112`
GUARD: `EN_I2C`

```
bool ca_i2c_write(u8 i2c_num, u8 addr, const u8 *buf, u16 len)
```

Sends a sequence of bytes to a specific I2C slave device address.

Parametri:

Nome	Tipo	Descrizione
`i2c_num`	`u8`	The I2C peripheral index (e.g., 0 for I2C1, 1 for I2C2). Must match the hardware configuration.
`addr`	`u8`	The 7-bit I2C slave address. Ensure this is the raw address; do not shift it left, as the driver handles the R/W bit.
`buf`	`const u8 *`	Pointer to the data buffer to be transmitted. Must not be NULL if len > 0.
`len`	`u16`	Number of bytes to write. A value of 0 is technically valid but usually results in no action.

Ritorna:
Returns true if the transmission was successful (ACK received for all bytes), or false if a NACK occurred, a bus error happened, or the peripheral was busy.

Note:

- This function is blocking, meaning the task calling it will pause execution until the entire buffer is sent or an error occurs. It is thread-safe assuming the underlying hardware driver uses proper mutexes or semaphores to prevent concurrent access to the same I2C bus. Do not call this from an Interrupt Service Routine (ISR) because blocking operations are forbidden in ISRs and will likely cause a system crash or watchdog reset. A common

beginner mistake is passing a pointer to a local stack variable that goes out of scope before the hardware finishes the transfer, or failing to check the return value, leading to silent data loss when a sensor is disconnected.

Uso:

- This function is used to send configuration commands or data packets to I2C peripherals like sensors, EEPROMs, or display controllers. It is typically used in a sequence: first, you initialize the I2C bus using ca_i2c_init, then you call ca_i2c_write to send a register address or command, and finally, you may call ca_i2c_read to retrieve the response. It is the standard way to perform 'write-only' operations where no data is expected back from the slave device.

Esempio:

```
/* Example: Writing a configuration byte to a sensor at address 0x48 */
u8 config_data[2] = {0x01, 0x7F}; // Register 0x01, Value 0x7F

// Ensure I2C peripheral 1 is initialized before use
if (ca_i2c_write(1, 0x48, config_data, 2)) {
    // Success: Data sent to the sensor
} else {
    // Error: Handle communication failure using ca_i2c_err_handler
    ca_i2c_err_handler(1);
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_i2c_write to communicate with an I2C device?
A: You should use ca_i2c_write when you need to send data to an I2C device in a blocking manner. It is suitable for scenarios where the calling thread can wait for the operation to complete, such as configuring a sensor or writing to an EEPROM. It must not be called from an ISR due to its blocking nature and potential for causing deadlocks.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_i2c_write with caOS types?
A: u8 data[] = {0x01, 0x02, 0x03}; bool success = ca_i2c_write(0, 0x48, data, 3); if (success) { // Data written successfully } else { // Handle write error }

Parameters conf:0.90 **Q:** What are the valid ranges for i2c_num and addr parameters in ca_i2c_write?
A: The i2c_num parameter must be a valid bus number between 0 and the maximum number of configured I2C buses. The addr parameter must be a 7-bit I2C address in the range 0x00 to 0x7F. Passing invalid values may lead to undefined behavior or hardware errors.

Return conf:0.95 **Q:** What does ca_i2c_write return in case of failure, and what can cause it?
A: ca_i2c_write returns false when the write operation fails. Common causes include the bus being occupied, the target device not responding, or other communication errors. It does not retry the operation and will return immediately upon encountering an error condition.

`ca_i2c_write_read`

Header: `caos/drivers/inc/i2c.h:60`
Source: `caos/drivers/src/i2c.c:119:124`
GUARD: `EN_I2C`

```
bool ca_i2c_write_read(u8 i2c_num, u8 addr, const u8 *wbuf, u16 wlen, u8
    *rbuf, u16 rlen)
```

Performs a combined I2C write followed by a repeated-start read operation on a specific bus.

Parametri:

Nome	Tipo	Descrizione
`i2c_num`	`u8`	The hardware I2C peripheral index (e.g., 1, 2). Must correspond to a valid, initialized I2C peripheral.
`addr`	`u8`	The 7-bit I2C slave device address. Ensure this is the raw address; do not shift it left, as the driver handles the R/W bit.
`wbuf`	`const u8 *`	Pointer to the buffer containing data to be written. Can be NULL if wlen is 0.
`wlen`	`u16`	Number of bytes to write. Set to 0 if only a read operation is desired.
`rbuf`	`u8 *`	Pointer to the buffer where received data will be stored. Must have enough allocated space for rlen bytes.
`rlen`	`u16`	Number of bytes to read. Set to 0 if only a write operation is desired.

Ritorna:
Returns true if the entire transaction (write and read) completed successfully. Returns false if the slave NACKed, the bus was busy, or a hardware timeout occurred.

Note:

- This function is blocking, meaning the calling task will be suspended by the RTOS until the I2C transaction completes or times out. Because it blocks, it must NEVER be called from an Interrupt Service Routine (ISR), as this would cause a system crash or deadlock. It is thread-safe, assuming the underlying hardware driver uses a mutex to protect the I2C peripheral from concurrent access by multiple tasks. A common beginner mistake is passing a pointer to a local stack variable that goes out of scope or is modified by an interrupt before the I2C hardware finishes the DMA/interrupt-driven transfer.

Uso:

- This function is typically used to communicate with sensors or EEPROMs that require a register address to be written before reading the data stored at that location. It is the standard way to perform a 'Write-then-Read' sequence using a repeated start condition, which is more efficient and safer than two separate transactions. In a typical application, you would initialize the I2C peripheral once at startup, then call this function periodically within a task loop to poll sensor data or update device configuration. It is often paired with error-checking logic to increment failure counters if the return value is false.

Esempio:

```
u8 tx_data[2] = {0x05, 0x00}; // Register address and dummy byte
u8 rx_data[4];

// Perform write-then-read transaction on I2C bus 1
if (ca_i2c_write_read(1, 0x48, tx_data, 2, rx_data, 4)) {
    // Success: process rx_data
} else {
    // Failure: handle communication error (e.g., log error or retry)
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use the ca_i2c_write_read function in a caOS project?

A: You should use the ca_i2c_write_read function when you need to perform a combined write-read transaction on an I2C device. This is commonly required when communicating with sensors or memory devices that require a register address to be written first, followed by reading the data from that register. The function is designed for blocking operations and must not be called from an ISR.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_i2c_write_read with caOS types?
A: u8 tx_data[1] = {0x05}; u8 rx_data[2]; bool success = ca_i2c_write_read(1, 0x48, tx_data, 1, rx_data, 2); if (success) { // Process rx_data }

Parameters conf:0.90 **Q:** What are the valid ranges for the i2c_num parameter in ca_i2c_write_read?
A: The i2c_num parameter must be a value between 1 and 4 inclusive. Any value outside this range will cause the function to return false and will not modify the I2C peripheral state. This parameter identifies which I2C hardware peripheral to use, such as I2C1, I2C2, I2C3, or I2C4 on STM32 microcontrollers.

Return conf:0.95 **Q:** What does it mean when ca_i2c_write_read returns false?
A: A return value of false indicates that the function failed to perform the I2C transaction. This can happen due to invalid i2c_num, hardware initialization failure, or incorrect parameters such as NULL buffers when data is expected. The function does not modify the I2C peripheral state in case of failure.

`ca_ip_defrag_init`

Header: `caos/net/inc/ip_defrag.h:57`
Source: `caos/net/src/ip_defrag.c:199:228`
GUARD: `EN_IP_DEFRAG`

```
bool ca_ip_defrag_init(u32 max_payload_size)
```

Initializes the IP fragmentation reassembly module by allocating memory and setting up synchronization primitives.

Parametri:

Nome	Tipo	Descrizione
`max_payload_size`	`u32`	The maximum size in bytes for a single reassembled IP packet. Must be greater than 0 and less than or equal to the system constant IP_DEFRAG_MAX_PAYLOAD.

Ritorna:
Returns true if initialization succeeded (memory allocated and primitives initialized). Returns false if the size is invalid, heap allocation fails, or the system is already configured.

Note:

- This function is non-blocking but performs heap allocation, so it must be called during the system initialization phase before any network traffic is processed. It is NOT thread-safe and must never be called from an ISR, as it relies on heap management and mutex initialization which are not re-entrant. A common beginner mistake is calling this function multiple times or after the network stack has already started, which can lead to memory leaks or corrupted session pointers. Ensure the system has enough heap space available, as it allocates a contiguous block for all potential reassembly sessions.

Uso:

- This function is used during the early boot sequence of the network stack, typically inside the main initialization task. It is the mandatory first step before any IP packet reassembly

can occur. It is usually paired with other service initializations like DHCP or ARP. Once initialized, the system uses the internal mutexes and semaphores created here to manage concurrent access to the reassembly buffers during packet reception.

Esempio:

```c
#define MY_MAX_PAYLOAD 1500

void system_init(void) {
    /* Initialize the IP defragmentation module before starting network services */
    if (!ca_ip_defrag_init(MY_MAX_PAYLOAD)) {
        /* Handle error: log failure and stop system startup */
        ca_stdout_printf_n("Critical error: IP defrag init failed");
        return;
    }

    /* Proceed to start other network services */
    ca_arp_service_start();
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_ip_defrag_init() in my caOS project?
A: You should call ca_ip_defrag_init() once during system initialization, before any IP packet defragmentation is needed. It must be called before any other IP defragmentation functions and only once to avoid undefined behavior.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_ip_defrag_init()?
A: u32 max_payload = 1500; bool success = ca_ip_defrag_init(max_payload); if (!success) { // Handle initialization failure }

Parameters conf:0.95 **Q:** What does the max_payload_size parameter control in ca_ip_defrag_init()?
A: The max_payload_size parameter defines the maximum payload size that each IP defragmentation session can handle. It must be greater than 0 and less than or equal to IP_DEFRAG_MAX_PAYLOAD. Invalid values cause initialization to fail.

Return conf:0.95 **Q:** What does ca_ip_defrag_init() return when it succeeds or fails?
A: It returns true if initialization is successful, meaning memory was allocated and control structures were created. It returns false if initialization fails due to invalid max_payload_size or memory allocation failure.

Error/Edge case conf:0.90 **Q:** What are common mistakes when calling ca_ip_defrag_init()?
A: Common mistakes include passing a max_payload_size that is zero or exceeds IP_DEFRAG_MAX_ calling the function more than once, or calling it from an ISR. These can lead to allocation failures or undefined behavior.

Behavior conf:0.90 **Q:** Is ca_ip_defrag_init() thread-safe and ISR-safe?
A: No, ca_ip_defrag_init() is not thread-safe and must not be called from an ISR. It modifies global structures and is not designed for concurrent access. It should be called once during initialization before any concurrent usage.

`ca_ip_defrag_service_start`

Header: `caos/net/inc/ip_defrag.h:67`
Source: `caos/net/src/ip_defrag.c:230:234`
GUARD: `EN_IP_DEFRAG`

```c
void ca_ip_defrag_service_start(void)
```

Initializes and launches the background task responsible for reassembling fragmented IP packets.

Ritorna:
This function returns void and does not provide a status code; it assumes the OS job scheduler will successfully spawn the task.

Note:

- This function is non-blocking as it merely registers a new job with the caOS scheduler. It is not thread-safe to call this from an ISR, as it interacts with the OS task creation system which requires a running scheduler context. Before calling this, ensure the underlying network stack and memory pools are initialized, otherwise the defrag task may crash immediately upon execution. A common beginner mistake is calling this function multiple times during runtime; since it spawns a new task, doing so will consume extra stack memory and potentially lead to undefined behavior or system instability.

Uso:

- This function is typically called during the system boot sequence, specifically within the network initialization phase. It is usually paired with other network service starters like ca_arp_service_start or ca_dhcp_service_start to build the full networking stack. Once started, the defrag service runs autonomously in the background, waiting for packets submitted via ca_ip_defrag_submit, so you do not need to interact with it further after the initial call.

Esempio:

```c
/* Typical network stack initialization sequence */
void network_init(void) {
    /* Initialize basic services first */
    ca_arp_service_start();

    /* Start the IP defragmentation service to handle fragmented packets */
    ca_ip_defrag_service_start();

    /* Proceed to start higher level services */
    ca_udp_server_start(&srv_cfg);
}
```

Q&A — fine-tuning review *2/2 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_ip_defrag_service_start() in my caOS application, and what is its purpose?
A: You should call ca_ip_defrag_service_start() after initializing the IP defragmentation module in your caOS application. Its purpose is to start the task responsible for handling fragmented IP packets, ensuring that fragmented packets are properly reassembled before being processed further. The function is non-blocking, thread-safe, and must be called after the defragmentation system has been correctly initialized.

Code example conf:0.85 **Q:** Can you provide a minimal C code example using caOS types that demonstrates how to start the IP defragmentation service?
A: u32 ca_ip_defrag_service_start(void);
void example_ip_defrag_init(void) { // Initialize IP defragmentation module first // ... initialization code ...
// Start the defragmentation service task ca_ip_defrag_service_start(); }

ca_ip_defrag_submit

Header: caos/net/inc/ip_defrag.h:79
Source: caos/net/src/ip_defrag.c:236:312
GUARD: EN_IP_DEFRAG

```
void ca_ip_defrag_submit(u16 buf_idx)
```

Submits an incoming IP fragment to the reassembly engine for processing.

Parametri:

Nome	Tipo	Descrizione
buf_idx	u16	The index of the network buffer containing the IP fragment. Must be a valid index previously allocated by the network stack; invalid indices will cause memory corruption or system crashes.

Ritorna:
void. The function does not return a status code; it internally handles buffer release via ca_net_buf_free() regardless of whether the fragment was accepted or dropped.

Note:

- This function is non-blocking but uses a mutex (ca_mutex_lock) to protect the internal session table, meaning it is NOT safe to call from an ISR. It requires the defrag service to be initialized via ca_ip_defrag_service_start() beforehand; calling it while the service is uninitialized will result in the buffer being freed and an error message logged. A common beginner mistake is attempting to access the buffer index after calling this function; once submitted, the function takes ownership and frees the buffer, so any subsequent access to that index will result in a use-after-free bug. Ensure that the network stack is configured to handle fragments, otherwise, packets will be silently dropped.

Uso:

- This function is used in the network receive path when the IP layer detects a fragmented packet (i.e., the More Fragments flag is set or the fragment offset is non-zero). It is typically called within the packet processing loop after the L2/L3 headers have been parsed. It pairs with ca_ip_defrag_service_start() for initialization and triggers the background defrag_task via a semaphore (s_sem) once a fragment is successfully stored. It is the primary entry point for the reassembly logic, abstracting the complexity of session tracking and buffer management from the main network driver.

Esempio:

```
/* Inside the IP receive handler */
if (is_fragmented(ip4)) {
    /* Submit the fragment to the defrag engine */
    /* The engine will copy the data and free the buffer */
    ca_ip_defrag_submit(buf_idx);
} else {
    /* Process complete packet normally */
    process_full_packet(buf_idx);
    ca_net_buf_free(buf_idx);
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_ip_defrag_submit() in my network application?
A: You should call ca_ip_defrag_submit() after receiving an IP fragment that has the More Fragments (MF) flag set or when the fragment offset is non-zero. This function is

used to submit IP fragments to a defragmentation task that will reconstruct the original IP packet.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_ip_defrag_submit() with caOS types?
A: u16 buf_idx = ca_net_buf_alloc(); // Assume buffer is filled with IP fragment data ca_ip_defrag_submit(buf_idx);

Parameters conf:0.90 **Q:** What is the significance of the buf_idx parameter in ca_ip_defrag_submit()?
A: The buf_idx parameter is the index of a network buffer allocated by ca_net_buf_alloc(). It must be a valid index; values of 0 or greater than the maximum number of allocatable buffers are invalid. The buffer must contain an IP fragment that has either the MF flag set or a non-zero offset.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_ip_defrag_submit()?
A: A common mistake is calling ca_ip_defrag_submit() without initializing the defragmentation service using ca_ip_defrag_service_start(). Another error is passing an invalid buffer index (0 or exceeding the maximum buffer count). Additionally, calling this function without verifying that the packet is actually a fragment (checking MF flag or offset) may lead to ignored fragments.

Behavior conf:0.90 **Q:** Is ca_ip_defrag_submit() thread-safe and ISR-safe?
A: No, ca_ip_defrag_submit() is not thread-safe because it uses a mutex internally. It is also not ISR-safe and must not be called from an interrupt service routine. The function is blocking for the network buffer: it copies the buffer and immediately releases it.

`ca_ip_fragment_send`

Header: `caos/net/inc/net_proto.h:295`
Source: `caos/net/src/net_proto.c:1616:1705`
GUARD: `EN_NET_PROTO`

```
bool ca_ip_fragment_send(ca_mac_t dst_mac, ca_ipv4_t src_ip, ca_ipv4_t
    dst_ip, ca_mac_t src_mac, u8 proto, const u8 *l4hdr, u16 l4len, const
     u8 *payload, u16 plen)
```

Splits a large data buffer into multiple IP fragments and transmits them over the network.

Parametri:

Nome	Tipo	Descrizione
`dst_mac`	`ca_mac_t`	Destination MAC address.
`src_ip`	`ca_ipv4_t`	Source IPv4 address.
`dst_ip`	`ca_ipv4_t`	Destination IPv4 address.
`src_mac`	`ca_mac_t`	Source MAC address.
`proto`	`u8`	IP protocol number (e.g., 17 for UDP).
`l4hdr`	`const u8 *`	Pointer to the transport layer header (e.g., UDP header).
`l4len`	`u16`	Length of the transport layer header in bytes.
`payload`	`const u8 *`	Pointer to the data payload.
`plen`	`u16`	Length of the data payload in bytes.

Ritorna:
Returns true if all fragments were successfully queued for transmission. Returns false if any fragment failed to allocate or send, or if the MTU is too small to fit an IP header.

Note:

- This function is blocking in the sense that it iterates through the entire data buffer and attempts to send all fragments sequentially. It is not thread-safe if multiple tasks attempt to send fragments simultaneously using the same internal state, though it uses a static ID counter. It must NOT be called from an ISR because it performs memory allocation (ca_net_buf_alloc) and potentially long-running loops. The most common mistake is assuming that a failure mid-transmission will 'undo' the previous fragments; it does not. If this function returns false, the peer will likely receive an incomplete datagram and discard it after a timeout.

Uso:

- Use this function when you need to send a packet larger than the network MTU (Maximum Transmission Unit). It is typically used in protocol implementations (like UDP or custom protocols) where the application layer provides a large buffer that must be broken down into smaller, valid IP packets. You should pair this with ca_net_tx_mtu() to understand the network constraints before calling it. It is the final step in the transmission chain, following the construction of the transport layer header.

Esempio:

```
/* Example: Sending a large UDP-like payload */
#define PAYLOAD_SIZE 2000
u8 my_data[PAYLOAD_SIZE];
/* ... fill my_data with content ... */

/* Construct transport header (e.g., 8 bytes for UDP) */
u8 udp_hdr[8];
/* ... fill udp_hdr fields ... */

/* Send the fragmented datagram */
bool success = ca_ip_fragment_send(dst_mac, local_ip, remote_ip, local_mac, 17,
    udp_hdr, 8, my_data, PAYLOAD_SIZE);

if (!success) {
    /* Handle error: partial data might have been sent,
       the peer will likely time out waiting for the rest. */
    ca_printf("Failed to send fragmented packet!\n");
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_ip_fragment_send function in my caOS application?
A: You should use this function when you need to transmit a packet that exceeds the network MTU. It is designed to split a large data buffer into multiple valid IP fragments, typically for protocols like UDP where the application provides a buffer larger than the network can handle in a single frame.

Code example conf:0.90 **Q:** Provide a minimal example of how to call ca_ip_fragment_send.
A: ca_mac_t dst = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55}; ca_mac_t src = {0xAA, 0xBB, 0xCC, 0xDD, 0xEE, 0xFF}; ca_ipv4_t s_ip = {192, 168, 1, 10}; ca_ipv4_t d_ip = {192, 168, 1, 20}; u8 udp_hdr[8] = {0}; u8 data[2000] = {0}; bool success = ca_ip_fragment_send(dst, s_ip, d_ip, src, 17, udp_hdr, 8, data, 2000);

Parameters conf:1.00 **Q:** What do the l4hdr and l4len parameters represent?
A: The l4hdr parameter is a pointer to the transport layer header (such as a UDP header), and l4len specifies the length of that header in bytes. These are used to construct the fragmented packets correctly.

Return conf:1.00 **Q:** What does a return value of false indicate?
A: A return value of false indicates that the function failed to allocate or send at least one

fragment, or that the MTU is too small to fit an IP header. It does not undo previously sent fragments, meaning the peer will likely receive an incomplete datagram.

Error/Edge case conf:1.00 **Q:** What is a common mistake developers make when handling failures in ca_ip_fragment_send?

A: A common mistake is assuming that a failure mid-transmission will 'undo' the fragments already sent. The function does not perform any rollback, so if it returns false, the peer will receive an incomplete datagram and discard it after a timeout.

Behavior conf:1.00 **Q:** Is ca_ip_fragment_send safe to call from an ISR or in a multi-threaded environment?

A: No, it must not be called from an ISR because it performs memory allocation (ca_net_buf_alloc) and contains long-running loops. It is also not thread-safe if multiple tasks attempt to send fragments simultaneously, as it relies on a static ID counter.

`ca_iperf_client`

Header: `caos/net/inc/iperf_service.h:18`
Source: `caos/net/src/iperf_service.c:42:80`
GUARD: `EN_IPERF_SERVICE`

```
1 bool ca_iperf_client(ca_ipv4_t remote_ip, u16 port, ca_ipv4_t local_ip,
     ca_mac_t local_mac, u32 duration_ms, ca_iperf_result_t *out)
```

Performs a network throughput test by sending a continuous stream of data to a remote iPerf server for a specified duration.

Parametri:

Nome	Tipo	Descrizione
`remote_ip`	`ca_ipv4_t`	The destination IP address of the iPerf server.
`port`	`u16`	The TCP port number the server is listening on.
`local_ip`	`ca_ipv4_t`	The IP address to assign to the local network interface.
`local_mac`	`ca_mac_t`	The MAC address to assign to the local network interface.
`duration_ms`	`u32`	The total time in milliseconds to run the throughput test.
`out`	`ca_iperf_result_t *`	Pointer to a structure where the test results (bytes, duration, bps) will be stored. Must not be NULL.

Ritorna:
Returns true if the test completed successfully (even if zero bytes were sent), or false if the connection failed, the output pointer was NULL, or the network stack could not initialize the socket.

Note:

- This function is strictly blocking; it will halt the execution of the calling task for the entire duration_ms period. It is NOT thread-safe if multiple tasks attempt to use the same network resources simultaneously, and it must NEVER be called from an Interrupt Service Routine (ISR) because it relies on blocking OS ticks and network stack synchronization. A common beginner mistake is calling this from a high-priority task without considering that it will starve other tasks of CPU time for the duration of the test. Ensure the network interface is physically connected and the IP configuration is valid before calling, as the function does not perform complex link-layer validation.

Uso:

- This function is typically used in diagnostic tasks or during factory testing to verify network performance and stability. It follows a lifecycle of: 1) Initializing the network interface parameters, 2) Calling ca_iperf_client to perform the stress test, and 3) Processing the resulting ca_iperf_result_t structure to log or display the throughput metrics. It is often paired with system monitoring tasks that check if the network link is up before initiating the test to avoid immediate connection failures.

Esempio:

```
ca_iperf_result_t results;
ipv4_t server_ip = {192, 168, 1, 10};
ipv4_t my_ip = {192, 168, 1, 5};
mac_t my_mac = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55};

// Run a 5-second throughput test
if (ca_iperf_client(server_ip, 5001, my_ip, my_mac, 5000, &results)) {
    // Successfully performed test, log the results
    printf("Throughput: %lu bps\n", results.bps);
} else {
    // Handle connection failure or invalid parameters
    printf("iPerf test failed to connect or initialize.\n");
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use the ca_iperf_client function in a caOS application?
A: You should use the ca_iperf_client function when you need to perform a TCP throughput test against a remote server to measure network performance. It is typically used for diagnostics, network validation, or performance benchmarking in embedded applications that require reliable network connectivity.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_iperf_client with caOS types?
A: ca_iperf_result_t result; ipv4_t server_ip = {192, 168, 1, 100}; ipv4_t local_ip = {192, 168, 1, 1}; mac_t local_mac = {0x00, 0x08, 0x9B, 0x00, 0x00, 0x01}; bool success = ca_iperf_client(server_ip, 5001, local_ip, local_mac, 5000, &result); if (success) { // Process results in result }

Parameters conf:0.95 **Q:** What are the valid ranges for the port parameter in ca_iperf_client?
A: The port parameter must be a value between 1 and 65535 inclusive. Values outside this range will cause the connection to fail.

Return conf:0.95 **Q:** What does a return value of false from ca_iperf_client indicate?
A: A return value of false indicates that the function failed. This can happen due to: 1) passing NULL to the out parameter, 2) a failed connection, or 3) a timeout during data transmission.

Error/Edge case conf:0.90 **Q:** What are common mistakes when calling ca_iperf_client?
A: Common mistakes include: 1) passing NULL as the out parameter, 2) using invalid IP addresses, 3) calling the function from an ISR, and 4) not having sufficient network resources or an initialized TCP stack.

Behavior conf:0.95 **Q:** Is ca_iperf_client thread-safe and can it be called from an ISR?
A: No, ca_iperf_client is not thread-safe and cannot be called from an ISR. It is a blocking function that uses RTOS functions, making it unsuitable for interrupt contexts and unsafe for concurrent calls from multiple threads.

---

ca_iperf_server

Header: caos/net/inc/iperf_service.h:22
Source: caos/net/src/iperf_service.c:82:120
GUARD: EN_IPERF_SERVICE

```
bool ca_iperf_server(u16 port, ca_ipv4_t local_ip, ca_mac_t local_mac,
    u32 idle_timeout_ms, ca_iperf_result_t *out)
```

Starts a blocking TCP server to measure network throughput by receiving data until the connection times out.

Parametri:

Nome	Tipo	Descrizione
port	u16	The TCP port number to listen on (e.g., 5001).
local_ip	ca_ipv4_t	The local IP address structure to bind the server to.
local_mac	ca_mac_t	The local MAC address structure for the network interface.
idle_timeout_ms	u32	Maximum time in milliseconds to wait for incoming data before closing the connection.
out	ca_iperf_result_t *	Pointer to a structure where the throughput results (bytes, duration, bps) will be stored.

Ritorna:
Returns true if the server successfully accepted a connection and processed data. Returns false if the pointer is NULL, the port could not be opened, no connection was accepted, or an error occurred during reception.

Note:

- This function is strictly blocking; it will halt the execution of the calling task until the iPerf test completes or the timeout expires. It is not thread-safe if multiple tasks attempt to use the same network resources simultaneously, and it must NEVER be called from an ISR (Interrupt Service Routine) because it performs blocking network operations. A common mistake is setting the idle_timeout_ms too low, which causes the test to terminate prematurely if the network latency is high or the sender is slow to start. Ensure the network stack is fully initialized before calling this function.

Uso:

- This function is typically used in diagnostic tasks or factory test modes to verify network performance. It follows a lifecycle of: initializing the network interface, calling this function to perform the measurement, and then processing the resulting ca_iperf_result_t structure to log or display the throughput. It is often paired with a remote iPerf client running on a PC that sends a stream of data to the device's IP address.

Esempio:

```
ca_iperf_result_t results;
ipv4_t my_ip = {192, 168, 1, 10};
mac_t my_mac = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55};

// Start server on port 5001 with a 5-second idle timeout
if (ca_iperf_server(5001, my_ip, my_mac, 5000, &results)) {
    // Successfully received data, print results
    printf("Throughput: %u bps\n", results.bps);
} else {
    // Handle error: connection failed or timed out
```

```
    printf("iPerf test failed or timed out.\n");
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_iperf_server in a caOS project?
A: You should use ca_iperf_server when you need to measure network performance between a client and server, particularly for testing TCP throughput. It is useful for validating network hardware, driver functionality, or network stack performance in embedded systems.

Code example conf:0.90 **Q:** Can you show a minimal example of how to call ca_iperf_server with caOS types?
A: ca_iperf_result_t result; ipv4_t ip = {192, 168, 1, 100}; mac_t mac = {0x00, 0x08, 0x9B, 0x00, 0x00, 0x01}; bool success = ca_iperf_server(5001, ip, mac, 5000, &result); if (success) { // Use result.bytes, result.duration_ms, result.bps }

Parameters conf:0.90 **Q:** What does the idle_timeout_ms parameter control in ca_iperf_server?
A: The idle_timeout_ms parameter sets a timeout in milliseconds for waiting for data. If set to 0, the function will block indefinitely until the test completes or fails. If a positive value is provided, it will return false if no data is received within that time.

Return conf:0.90 **Q:** What does ca_iperf_server return, and what causes it to return false?
A: It returns true if the test completes successfully. It returns false if there is an error such as invalid parameters, a timeout due to idle_timeout_ms, or a connection failure. The function does not return false for network errors unless they cause a timeout or invalid parameter.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_iperf_server?
A: Common mistakes include passing invalid IP or MAC addresses, using an invalid port number outside the range 1-65535, setting idle_timeout_ms to a value that causes premature timeouts, or calling the function from an ISR. Also, not ensuring that underlying TCP functions are properly initialized can lead to failures.

Behavior conf:0.95 **Q:** Is ca_iperf_server thread-safe and ISR-safe?
A: No, ca_iperf_server is not thread-safe and must not be called from an ISR. It is a blocking function that waits for network activity, and its behavior is undefined if invoked concurrently or from interrupt context.

`ca_iperf_service_start`

Header: `caos/net/inc/iperf_service.h:29`
Source: `caos/net/src/iperf_service.c:241:257`
GUARD: `EN_IPERF_SERVICE`

```
void ca_iperf_service_start(ca_ipv4_t local_ip, ca_mac_t local_mac)
```

Initializes and launches the background iPerf network performance testing service.

Parametri:

Nome	Tipo	Descrizione
`local_ip`	`ca_ipv4_t`	The IPv4 address assigned to the interface. Use a zeroed structure if the IP is not yet known (e.g., waiting for DHCP).
`local_mac`	`ca_mac_t`	The hardware MAC address of the network interface.

Ritorna:
void

Note:

- This function is non-blocking and thread-safe, designed to be called once during system initialization. It internally initializes a semaphore and spawns a dedicated RTOS task (iperf_task) to handle network traffic measurements. It is NOT safe to call this from an ISR because it performs memory allocation and task creation. A common mistake is calling this function multiple times; while the internal 's.started' flag prevents duplicate task creation, it is best practice to call it only once after the network stack is ready. Ensure that the network interface is physically initialized before starting this service, otherwise, the iperf_task may fail to bind to the network stack.

Uso:

- This function is typically called during the system boot sequence or inside a network configuration callback (like a DHCP 'bound' event). It pairs with the network stack initialization functions (like ca_tcp_service_start) to ensure the infrastructure is ready for traffic. Once started, the iperf service runs in the background, allowing the system to respond to iPerf client requests without blocking the main application logic. If the CLI is enabled, it also registers the iperf commands, allowing technicians to trigger tests via the console.

Esempio:

```c
/* Example: Starting iPerf after network configuration */
void on_network_ready(ca_ipv4_t ip, ca_mac_t mac) {
    /* Ensure we only start the service once */
    static bool service_started = false;
    if (!service_started) {
        /* Start the iPerf service with the assigned network parameters */
        ca_iperf_service_start(ip, mac);
        service_started = true;
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_iperf_service_start() in a caOS application?
A: You should call ca_iperf_service_start() when you want to initiate network performance testing using the iperf protocol. It is typically called after the network stack and RTOS are initialized, and you have a valid local IP and MAC address configured for your device.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_iperf_service_start()
A: u8 local_ip[4] = {192, 168, 1, 100}; mac_t local_mac = {0x00, 0x80, 0xE1, 0x23, 0x45, 0x67}; ca_iperf_service_start(local_ip, local_mac);

Parameters conf:0.85 **Q:** What are the valid values for the local_ip and local_mac parameters?
A: The local_ip parameter must be a valid IPv4 address as an array of 4 u8 values, or zero to use a default address. The local_mac parameter must be a valid MAC address as an array of 6 u8 values. Invalid values such as malformed IP addresses or non-6-byte MAC arrays will be ignored or may cause undefined behavior.

Return conf:0.90 **Q:** Does ca_iperf_service_start() return any value, and what does it signify?
A: The function does not return any value. It is void. It does not explicitly fail, but it may ignore calls if the service is already running, and it assumes the RTOS is initialized.

Error/Edge case conf:0.80 **Q:** What are the common mistakes or edge cases when using ca_iperf_service_start
A: Common mistakes include calling the function before the RTOS is initialized, passing invalid IP or MAC addresses, or calling it multiple times without stopping the service. It is not thread-safe, so concurrent calls from multiple tasks may cause race conditions. It also does not validate that the IP address is already configured on the network interface.

Behavior conf:0.90 **Q:** Is ca_iperf_service_start() blocking or non-blocking, and is it safe to call from an ISR?
A: The function is non-blocking. It is not thread-safe, but it can be called from an ISR if the context allows it. It initializes a semaphore and creates a task, so it requires the RTOS to be initialized before calling.

`ca_iperf_update_local_ip`

Header: `caos/net/inc/iperf_service.h:32`
Source: `caos/net/src/iperf_service.c:259:262`
GUARD: `EN_IPERF_SERVICE`

```
void ca_iperf_update_local_ip(ca_ipv4_t ip)
```

Updates the local IPv4 address stored within the iPerf service module.

Parametri:

Nome	Tipo	Descrizione
`ip`	`ca_ipv4_t`	A structure containing the four bytes of the new IPv4 address. Ensure the structure is correctly populated before passing it.

Ritorna:
void

Note:

- This function is a simple setter that updates a static variable in the iPerf service module. It is non-blocking and executes almost instantaneously, making it safe to call from most contexts. However, it is not thread-safe; if the iPerf service is actively processing packets while this value is being updated, there is a minor risk of a race condition where the service uses a partially updated IP. It is not intended to be called from an ISR, as it is meant for configuration updates during network state changes. The most common mistake is calling this function before the iPerf service has been initialized, which may lead to the update being overwritten or ignored if the service resets its internal state.

Uso:

- This function is typically used during network configuration events, such as when a DHCP client successfully acquires a new lease or when a static IP is assigned at startup. It is usually paired with other network service update functions like ca_arp_service_update_local_ip to ensure all network layers are synchronized with the new address. You should call this whenever the system's primary IP address changes to ensure the iPerf diagnostic tool reports or binds to the correct interface.

Esempio:

```
/* Example: Updating the local IP after a DHCP lease is granted */
void handle_network_config(ca_ipv4_t new_ip) {
    /* Update the ARP service to reflect the new IP */
    ca_arp_service_update_local_ip(new_ip);

    /* Announce the new IP to the network to update neighbor caches */
    ca_arp_send_garp();

    /* Finally, update the iPerf service so it knows its new identity */
    ca_iperf_update_local_ip(new_ip);

    /* Notify the user via console */
```

```
    ca_stdout_printf_n("iPerf service updated to new IP");
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_iperf_update_local_ip() and what is its purpose in a networked embedded application?
A: You should call ca_iperf_update_local_ip() when you need to update the local IPv4 address used for network monitoring in the iperf service. This function is typically used when the network configuration changes, such as when a DHCP client obtains a new IP address or when a static IP is configured. It allows the iperf service to correctly track and report network statistics based on the current local IP address.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_iperf_update_local_ip() with caOS types?
A: u32 local_ip = 0xC0A80101; /* 192.168.1.1 in big-endian format */ ca_iperf_update_local_ip(local_

Parameters conf:0.85 **Q:** What is the meaning of the ca_ipv4_t ip parameter in ca_iperf_update_local_ip(), and what are valid and invalid values for it?
A: The ca_ipv4_t ip parameter represents the IPv4 address to be set as the local IP in the iperf configuration. It must be a valid 4-byte IPv4 address value. Invalid values include any value that does not conform to IPv4 addressing standards (e.g., values outside the valid range of 0.0.0.0 to 255.255.255.255). Providing an invalid value may lead to undefined behavior since the function does not validate input.

`ca_itm_init`

Header: `caos/inc/itm.h:39`
Source: `caos/src/itm.c:42:85`
GUARD: `EN_ITM`

```
void ca_itm_init(u32 baudrate)
```

Initializes the Instrumentation Trace Macrocell (ITM) and TPIU for high-speed debugging output via the SWO pin.

Parametri:

Nome	Tipo	Descrizione
`baudrate`	`u32`	The desired communication speed in bits per second (e.g., 2000000). Must be a valid value supported by your debugger probe and the MCU trace clock frequency.

Ritorna:
void

Note:

- This function is non-blocking and performs direct hardware register manipulation to configure the ARM CoreSight trace infrastructure. It is not thread-safe and must be called during the system initialization phase before any trace data is sent. It is not intended to be called from an ISR, as it performs complex hardware configuration that could interfere with real-time execution. A common mistake is failing to ensure the trace clock is correctly configured before calling this, which results in garbage data being sent to the debugger. Ensure your hardware debugger (e.g., ST-Link) is physically connected to the SWO pin, otherwise, the ITM will attempt to transmit into a void, potentially causing minor power consumption overhead.

Uso:

- This function is typically called once at the very beginning of the application lifecycle inside the system initialization sequence. It is used to prepare the debug infrastructure so that developers can use ITM-based logging or profiling tools. It is usually paired with subsequent calls to functions that send data over the ITM stimulus ports, such as ca_itm_printf or ca_itm_put. It should be initialized before any other debug-related peripherals to ensure that early boot-up messages can be captured by the debugger.

Esempio:

```
/* Initialize the system clock first to ensure trace clock is stable */
ca_os_clock_config();

/* Initialize ITM for high-speed debug output at 2MHz */
ca_itm_init(2000000);

/* Now the system is ready to send trace data */
ca_itm_printf("System initialized successfully\n");
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_itm_init() in a caOS application?
A: You should call ca_itm_init() during system initialization, typically in the main function or a dedicated init routine. It is intended to enable real-time debugging via SWO (Serial Wire Output) using the ITM (Instrumentation Trace Macrocell) on STM32 microcontrollers.

Code example conf:0.95 **Q:** Can you show a minimal example of how to use ca_itm_init() with caOS types?
A: u32 baudrate = 115200; ca_itm_init(baudrate);

Parameters conf:0.95 **Q:** What is the valid range for the baudrate parameter in ca_itm_init()?
A: The baudrate parameter must be between 1000 and 10000000 (1 kbps to 10 Mbps). Values outside this range may cause synchronization errors or configuration failures.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_itm_init()?
A: Common mistakes include calling ca_itm_init() from an ISR (Interrupt Service Routine), using an unsupported baudrate value, or not ensuring the trace clock is properly configured. These can lead to undefined behavior or system crashes.

Behavior conf:0.90 **Q:** Is ca_itm_init() blocking or non-blocking, and is it safe to call from an ISR?
A: ca_itm_init() is a blocking function and is not safe to call from an ISR. It modifies system registers that are not thread-safe, and must be called only from a privileged context such as main or init.

`ca_itm_printf`

Header: `caos/inc/itm.h:50`
Source: `caos/src/itm.c:195:215`
GUARD: `EN_ITM`

```
u16 ca_itm_printf( const char *format, ...)
```

Formats a string and sends it to the ARM ITM (Instrumentation Trace Macrocell) port for debugging.

Parametri:

Nome	Tipo	Descrizione
`format`	`const char *`	A standard C format string (like printf). Must not be NULL.
`...`	`variadic`	Variable arguments corresponding to the format specifiers in the format string.

Ritorna:
Returns the number of characters successfully formatted and sent, or the number of characters that would have been written if the buffer was large enough.

Note:

- This function is not thread-safe because it uses a shared static buffer 'itm_dbg_buffer'. If two tasks call this simultaneously, the output will be corrupted or interleaved. It is blocking because it waits for the ITM hardware to process each character via ca_itm_drain. While it can technically be called from an ISR, it is highly discouraged because the blocking nature of the ITM hardware interface can significantly increase interrupt latency and cause system instability. The most common mistake is exceeding the 128-character limit, which will truncate the output message silently.

Uso:

- Use this function during development to print debug information to the IDE's SWO/ITM console. It is typically used in the main loop or task functions to monitor variable states or execution flow. It is often paired with ca_itm_init during system startup to configure the debug clock. Unlike UART-based logging, this does not require a dedicated peripheral pin, making it ideal for diagnostic output without interfering with hardware peripherals.

Esempio:

```c
/* Example: Logging a sensor value to the ITM console */
float sensor_value = 25.5f;

// Ensure the ITM hardware is initialized before calling this
ca_itm_init();

// Print formatted string to the debugger console
u16 bytes_sent = ca_itm_printf("Sensor reading: %.2f C\n", sensor_value);

if (bytes_sent == 0) {
    // Handle potential formatting error or empty string
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_itm_printf in a caOS project?
A: You should use ca_itm_printf for real-time debugging output on STM32 microcontrollers that support ITM (Instrumentation Trace Macrocell). It is ideal for printing formatted debug messages during development and testing, especially when you need to trace execution flow or inspect variable values without using a debugger.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_itm_printf with caOS types?
A: u16 result = ca_itm_printf("Value: %d, String: %s\n", 42, "hello"); if (result == 0) { // Handle error }

Parameters conf:0.90 **Q:** What are the valid and invalid values for the format parameter in ca_itm_printf?
A: The format parameter must be a valid non-NULL C string containing format specifiers like %d, %s, %x. Invalid values include NULL pointers or strings with mismatched or unsupported format specifiers, which may lead to undefined behavior.

Return conf:0.95 **Q:** What does a return value of 0 from ca_itm_printf indicate?
A: A return value of 0 indicates an error occurred during formatting or sending the message via ITM. This could be due to issues in the format string or problems with the ITM transmission mechanism.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_itm_printf?
A: Common mistakes include passing NULL format strings, mismatched format specifiers and arguments, or calling the function without ensuring that ITM is properly enabled and configured. Also, using it in interrupt contexts or from multiple threads without synchronization can lead to corrupted output.

Behavior conf:0.90 **Q:** Is ca_itm_printf blocking, ISR-safe, or thread-safe?
A: ca_itm_printf is a blocking function because it waits during transmission via ITM_SendChar. It is not ISR-safe and should not be called from interrupt service routines. Additionally, it is not thread-safe, so concurrent usage from multiple threads can cause data corruption.

`ca_itm_put`

Header: `caos/inc/itm.h:43`
Source: `caos/src/itm.c:114:121`
GUARD: `EN_ITM`

```
u16 ca_itm_put(p_str string)
```

Sends a string of data to the ARM Cortex-M ITM (Instrumentation Trace Macrocell) port for debugging.

Parametri:

Nome	Tipo	Descrizione
`string`	`p_str`	A pointer to a structure containing the data buffer and its size. The structure must be properly initialized with valid data and a correct size field; passing a NULL pointer will cause a hard fault.

Ritorna:
Returns a u16 representing the total number of characters successfully processed and sent to the ITM hardware.

Note:

- This function is a blocking call, meaning the CPU will wait for each character to be processed by the ITM hardware before moving to the next one. It is thread-safe in the sense that it does not modify global state, but if multiple threads call it simultaneously, the output in your debugger console will be interleaved and garbled. It is safe to call from an ISR, but be aware that because it is a blocking loop, sending long strings inside an interrupt will significantly increase your interrupt latency and potentially cause system instability. The most common beginner mistake is passing a string structure where the 'size' field does not match the actual buffer length, leading to memory access violations or garbage data being sent.

Uso:

- This function is typically used for real-time logging and debugging where you need to see variable values or state transitions without stopping the CPU with a breakpoint. It is usually paired with a string formatting function that populates the p_str structure before calling ca_itm_put. In a typical development cycle, you use this during the 'bring-up'

phase to monitor system health; once the firmware is stable, these calls are often removed or wrapped in preprocessor macros (e.g., #ifdef DEBUG) to prevent performance degradation in production code.

Esempio:

```
/* Define a string structure and populate it */
struct string_t my_msg;
my_msg.data = "System Initialized";
my_msg.size = 18;

/* Send the message to the ITM port */
u16 sent_count = ca_itm_put(&my_msg);

/* Check if all characters were sent */
if (sent_count != my_msg.size) {
    /* Handle error: ITM might be disabled or buffer full */
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should a developer use the ca_itm_put function in a caOS application?
A: A developer should use ca_itm_put when they need to send debug messages in real-time via the ITM (Instrumentation Trace Macrocell) for debugging purposes. It is intended for debugging in a bare-metal environment where ITM is enabled and configured correctly. It is not suitable for use in ISRs or in multi-threaded contexts due to its blocking and non-thread-safe nature.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_itm_put with caOS types?
A: cstr debug_msg = { .data = "Hello, ITM!\n", .size = 11 }; u16 sent = ca_itm_put(debug_msg);

Parameters conf:0.90 **Q:** What does the 'string' parameter in ca_itm_put represent and what are valid values?
A: The 'string' parameter is a pointer to a structure of type p_str that contains a buffer and its size. The buffer must be valid (not NULL) and the size must be greater than 0. If string is NULL or size is 0, the function returns 0 without sending any data.

Return conf:0.95 **Q:** What do the return values of ca_itm_put indicate and what causes it to return 0?
A: The function returns a u16 > 0 indicating the number of characters sent, which equals the size of the input string. It returns 0 if the input string is NULL or has size 0, or if ITM is not enabled in the system (CoreDebug->DEMCR and ITM->TCR registers).

`ca_jpeg_decode`

Header: `caos/drivers/inc/jpeg.h:82`
Source: `caos/drivers/src/jpeg.c:304:356`
GUARD: `EN_JPEG`

```
bool ca_jpeg_decode(const u8 *jpg, u32 len, u8 *mcu, u32 mcu_max,
    ca_jpeg_info_t *info, u32 timeout_ms)
```

Decodes a JPEG image stream into MCU (Minimum Coded Unit) data using hardware acceleration.

Parametri:

Nome	Tipo	Descrizione
jpg	const u8 *	Pointer to the raw JPEG byte array.
len	u32	Length of the JPEG data in bytes. Must be at least 4.
mcu	u8 *	Buffer to store the decoded MCU output data.
mcu_max	u32	Maximum size of the MCU buffer in bytes.
info	ca_jpeg_info_t *	Pointer to a structure to receive image metadata (width, height, color space).
timeout_ms	u32	Maximum time in milliseconds to wait for the hardware decoder to finish.

Ritorna:
Returns true if the image was decoded successfully. Returns false if arguments are invalid, the hardware times out, the output buffer overflows, or the JPEG header is malformed.

Note:

- This function is blocking; it will wait for the hardware decoder to process the stream until the timeout expires or the EOC (End of Code) marker is reached. It is not thread-safe if multiple tasks attempt to use the same hardware peripheral simultaneously, so ensure access is serialized. It must not be called from an ISR because it performs blocking operations and relies on system ticks. A common mistake is providing an insufficient 'mcu_max' buffer size, which will trigger an overflow error, or failing to ensure the hardware is initialized before calling this function.

Uso:

- This function is typically used in image processing tasks where raw JPEG data needs to be converted into a raw pixel format for display or analysis. It is usually paired with ca_jpeg_init to prepare the hardware, and ca_jpeg_errors to retrieve specific error codes if the function returns false. After decoding, the resulting MCU data is often processed further using ca_jpeg_mcu_to_rgb to convert the data into a format suitable for framebuffers.

Esempio:

```
ca_jpeg_info_t info;
u8 mcu_buffer[4096];
// Ensure hardware is ready before decoding
if (ca_jpeg_decode(raw_data, data_len, mcu_buffer, sizeof(mcu_buffer), &info,
    500U)) {
    // Successfully decoded, proceed to process info.width and info.height
} else {
    // Check specific error code if decoding failed
    u32 err = ca_jpeg_errors();
    /* Log error code 'err' to system console */
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_jpeg_decode function in my caOS application?
A: You should use ca_jpeg_decode when you need to convert raw JPEG byte streams into MCU data for display or analysis. It is typically used in image processing tasks, often following a call to ca_jpeg_init to prepare the hardware, and is frequently followed by ca_jpeg_mcu_to_rgb to convert the output into a format suitable for framebuffers.

Code example conf:0.90 **Q:** Provide a minimal C example of how to call ca_jpeg_decode using caOS types.
A: u8 *jpg_data = ...; u32 len = 1024; u8 mcu_buf[2048]; ca_jpeg_info_t info; if (ca_jpeg_decode(jpg_data, len, mcu_buf, 2048, &info, 500)) { /* Success */ } else { /* Handle error */ }

Parameters conf:1.00 **Q:** What are the constraints on the 'len' and 'mcu_max' parameters?
A: The 'len' parameter represents the length of the JPEG data in bytes and must be at least 4. The 'mcu_max' parameter defines the maximum size of the MCU buffer in bytes; providing an insufficient size for the decoded data will trigger an overflow error.

Return conf:1.00 **Q:** What does a return value of false indicate for ca_jpeg_decode?
A: A return value of false indicates that the decoding process failed. This can be caused by invalid arguments, the hardware decoder timing out, the output buffer overflowing, or the JPEG header being malformed.

Error/Edge case conf:1.00 **Q:** What are some common mistakes when using ca_jpeg_decode?
A: Common mistakes include providing an insufficient 'mcu_max' buffer size, which causes an overflow error, or failing to initialize the hardware (via ca_jpeg_init) before calling the function. Additionally, failing to check for errors using ca_jpeg_errors after a false return is a missed opportunity for debugging.

Behavior conf:1.00 **Q:** Is ca_jpeg_decode safe to call from an ISR or multiple threads?
A: No, it is not ISR-safe because it performs blocking operations and relies on system ticks. It is also not thread-safe; if multiple tasks attempt to use the same hardware peripheral, you must ensure access is serialized.

---

ca_jpeg_encode

Header: `caos/drivers/inc/jpeg.h:80`
Source: `caos/drivers/src/jpeg.c:254:302`
GUARD: `EN_JPEG`

```
bool ca_jpeg_encode(const u8 *mcu, const ca_jpeg_cfg_t *cfg, u8 *out,
    u32 out_max, u32 *out_len, u32 timeout_ms)
```

Encodes raw image data into a JPEG bitstream using hardware acceleration.

Parametri:

Nome	Tipo	Descrizione
`mcu`	`const u8 *`	Pointer to the input buffer containing raw MCU (Minimum Coded Unit) data.
`cfg`	`const ca_jpeg_cfg_t *`	Configuration structure defining width, height, color space, and quality (1-100).
`out`	`u8 *`	Pointer to the destination buffer where the JPEG bitstream will be written.
`out_max`	`u32`	Maximum size of the output buffer in bytes to prevent overflow.
`out_len`	`u32 *`	Pointer to store the actual number of bytes written to the output buffer.
`timeout_ms`	`u32`	Maximum time in milliseconds to wait for the hardware encoder to finish.

Ritorna:
Returns true if the encoding completed successfully and the JPEG EOI marker was found. Returns false if arguments are invalid, hardware fails, the output buffer overflows, or the operation times out.

Note:

- This function is blocking; it waits for the hardware JPEG peripheral to process the data stream. It is not thread-safe if multiple tasks attempt to use the JPEG hardware simultaneously, so ensure access is protected by a mutex if necessary. It must not be called from

an ISR because it relies on system ticks for timeout management. A common mistake is providing an output buffer that is too small for the requested quality, which will trigger an overflow error.

Uso:

- This function is typically used in image processing pipelines after raw pixel data has been converted to MCU format using ca_jpeg_rgb_to_mcu. The standard sequence is: initialize the JPEG hardware, prepare the configuration, convert the image data, call ca_jpeg_encode, and finally handle the resulting bitstream. It is often paired with ca_jpeg_geometry to validate that the image dimensions are supported by the hardware before starting the encoding process.

Esempio:

```
/* Assuming s_mcu is pre-filled with raw data and cfg is initialized */
u8 out_buffer[4096];
u32 encoded_size = 0;

/* Validate geometry before encoding */
ca_jpeg_info_t info;
if (ca_jpeg_geometry(cfg.width, cfg.height, cfg.css, &info)) {
    /* Attempt to encode the image data */
    if (ca_jpeg_encode(s_mcu, &cfg, out_buffer, sizeof(out_buffer), &
    encoded_size, 500)) {
        /* Success: encoded_size contains the JPEG file size */
    } else {
        /* Handle encoding error */
    }
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** In what sequence should I use ca_jpeg_encode within an image processing pipeline?
A: The function is typically used after raw pixel data is converted to MCU format using ca_jpeg_rgb_to_mcu. The standard sequence is: initialize the JPEG hardware, prepare the configuration, convert the image data, call ca_jpeg_encode, and finally handle the resulting bitstream. It is also recommended to use ca_jpeg_geometry first to validate that the image dimensions are supported by the hardware.

Code example conf:0.90 **Q:** Provide a minimal example of how to call ca_jpeg_encode in caOS.
A: u8 out_buf[1024]; u32 written; ca_jpeg_cfg_t cfg = { .width = 320, .height = 240, .quality = 80 }; if (ca_jpeg_encode(mcu_data, &cfg, out_buf, 1024, &written, 500)) { /* Success */ }

Parameters conf:1.00 **Q:** What is the purpose of the out_max parameter in ca_jpeg_encode?
A: The out_max parameter defines the maximum size of the output buffer in bytes. It is used to prevent buffer overflows during the encoding process.

Return conf:1.00 **Q:** What does a return value of false indicate for ca_jpeg_encode?
A: A return value of false indicates that the operation failed. This can be caused by invalid arguments, hardware failure, the output buffer overflowing, or the operation exceeding the specified timeout.

Error/Edge case conf:1.00 **Q:** What is a common mistake that leads to an error when using ca_jpeg_encode?
A: A common mistake is providing an output buffer that is too small for the requested image quality, which will trigger an overflow error and cause the function to return false.

Behavior conf:1.00 **Q:** Is ca_jpeg_encode safe to call from an ISR or multiple tasks simultaneously?

A: No, it is not ISR-safe because it relies on system ticks for timeout management. It is also not thread-safe; if multiple tasks attempt to use the JPEG hardware simultaneously, access must be protected by a mutex.

---

ca_jpeg_errors

Header: caos/drivers/inc/jpeg.h:84
Source: caos/drivers/src/jpeg.c:242:245
GUARD: EN_JPEG

```
u32 ca_jpeg_errors(void); /* CA_JPEG_ERR_* */
```

Retrieves the error status code from the last JPEG operation.

Ritorna:
Returns a 32-bit unsigned integer representing the error bitmask. A return value of 0 indicates no errors occurred during the last operation.

Note:

- This function is non-blocking and thread-safe as it simply reads a static variable updated by the JPEG driver. It is safe to call from an ISR, although it is typically used in the main task context immediately after a failed JPEG operation. The most common mistake is calling this function before performing any JPEG operation or assuming it clears the error state; it only reports the current state, which remains until the next JPEG operation overwrites it.

Uso:

- This function is used for diagnostic purposes following a failed call to ca_jpeg_decode or ca_jpeg_encode. It is typically paired with logging functions like ca_snprintf to report specific failure reasons to the user or a system log. Developers should check the return value of the JPEG operation first; if it returns false, they should immediately call ca_jpeg_errors to determine if the failure was due to a header mismatch, memory exhaustion, or a corrupted bitstream.

Esempio:

```
/* Attempt to decode a JPEG image from a buffer */
const bool ok = ca_jpeg_decode(jpg_data, data_len, mcu_buffer, sizeof(mcu_buffer), &info, 1000);

if (!ok) {
    /* Retrieve the specific error code */
    u32 err = ca_jpeg_errors();

    /* Log the error for debugging purposes */
    static char log_msg[64];
    ca_snprintf(log_msg, sizeof(log_msg), "JPEG decode failed with error code: 0x%08X", err);

    /* Use ca_printf to output the error to the system console */
    ca_printf("%s\n", log_msg);
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_jpeg_errors and why is it necessary?
A: You should call ca_jpeg_errors immediately after a failed call to ca_jpeg_decode or ca_jpeg_encode. It is necessary for diagnostic purposes to determine the specific cause of the failure, such as a header mismatch, memory exhaustion, or a corrupted bitstream.

Code example conf:1.00 **Q:** Can you provide a minimal C example showing how to check for JPEG errors using caOS types?
A: if (!ca_jpeg_decode(buffer, size)) { u32 err = ca_jpeg_errors(); if (err != 0) { /* Handle error */ } }

Return conf:1.00 **Q:** What does the return value of ca_jpeg_errors represent and how do I interpret it?
A: The function returns a 32-bit unsigned integer (u32) representing an error bitmask. A return value of 0 indicates that no errors occurred during the last operation. Non-zero values indicate specific failures that persist until the next JPEG operation overwrites the state.

`ca_jpeg_geometry`

Header: `caos/drivers/inc/jpeg.h:79`
Source: `caos/drivers/src/jpeg.c:215:227`
GUARD: `EN_JPEG`

```
bool ca_jpeg_geometry(u16 width, u16 height, ca_jpeg_css_t css,
    ca_jpeg_info_t *info)
```

Calculates the required memory and MCU count for a JPEG image based on dimensions and color subsampling.

Parametri:

Nome	Tipo	Descrizione
`width`	`u16`	Image width in pixels. Must be greater than 0.
`height`	`u16`	Image height in pixels. Must be greater than 0.
`css`	`ca_jpeg_css_t`	Color Subsampling scheme (e.g., 4:2:0, 4:2:2, 4:4:4, or Gray). Must be a valid enum value.
`info`	`ca_jpeg_info_t *`	Pointer to a structure where the calculated geometry data will be stored. Must not be NULL.

Ritorna:
Returns true if the calculation was successful and parameters were valid. Returns false if any input parameter is invalid (e.g., NULL pointer, zero dimensions, or invalid CSS enum).

Note:

- This function is a pure mathematical utility and is thread-safe as it does not access shared hardware or global state beyond internal lookup tables. It is non-blocking and can be safely called from an ISR, as it performs no I/O or OS-level synchronization. The most common mistake is passing a NULL pointer for the 'info' structure or providing zero for width/height, which will cause the function to fail silently by returning false. Always verify the return value before using the data inside the 'info' struct, as the struct will remain uninitialized or contain garbage if the function returns false.

Uso:

- This function is typically used during the initialization phase of a JPEG encoding or decoding task to determine how much memory needs to be allocated for the image buffer. It is usually called immediately after reading configuration parameters and before calling ca_memory_allocate or ca_jpeg_encode. By calculating the 'mcu_bytes' requirement beforehand, you ensure that the system has enough heap space to process the image, preventing memory corruption or buffer overflows during the actual compression/decompression process.

Esempio:

```
ca_jpeg_info_t jpeg_data;
ca_jpeg_css_t subsampling = CA_JPEG_420;
u16 w = 640;
u16 h = 480;

/* Calculate geometry to determine buffer size */
if (ca_jpeg_geometry(w, h, subsampling, &jpeg_data)) {
    /* Allocate memory based on the calculated mcu_bytes */
    void *buffer = ca_memory_allocate(jpeg_data.mcu_bytes);
    if (buffer != NULL) {
        /* Proceed with encoding using the allocated buffer */
        /* ca_jpeg_encode(..., buffer, ...); */
        ca_memory_free(buffer);
    }
} else {
    /* Handle invalid configuration error */
    ca_printf("Failed to calculate JPEG geometry\n");
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_jpeg_geometry in my caOS application?
A: You should call it during the initialization phase of a JPEG encoding or decoding task, specifically after reading configuration parameters but before calling ca_memory_allocate or ca_jpeg_encode. This ensures you have calculated the required memory and MCU count to prevent buffer overflows or memory corruption.

Code example conf:0.90 **Q:** Provide a minimal C example of how to use ca_jpeg_geometry.
A: u16 w = 640; u16 h = 480; ca_jpeg_info_t info; if (ca_jpeg_geometry(w, h, CA_JPEG_CSS_420, &info)) { /* Proceed with allocation */ }

Parameters conf:1.00 **Q:** What are the constraints on the width and height parameters for ca_jpeg_geometry?
A: Both width and height must be greater than 0. Providing zero for either dimension is an invalid parameter that will cause the function to return false.

Return conf:1.00 **Q:** What does the boolean return value of ca_jpeg_geometry indicate?
A: A return value of true indicates that the calculation was successful and all input parameters were valid. A return value of false indicates that the input parameters were invalid, such as passing a NULL pointer, zero dimensions, or an invalid CSS enum value.

Error/Edge case conf:1.00 **Q:** What happens if I pass a NULL pointer to the info parameter in ca_jpeg_geometry?
A: The function will fail and return false. It is a common mistake to pass a NULL pointer, and you must always verify the return value before using the info structure, as it will remain uninitialized or contain garbage if the function returns false.

`ca_jpeg_hw_decode`

Header: `caos/drivers/inc/ca_glue_hw.h:437`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_jpeg_hw_decode(void); /* decoding with header parsing,
    START */
```

Triggers the hardware JPEG decoder engine to begin processing image data.

Ritorna:
void

Note:

- This function is non-blocking and initiates the hardware peripheral state machine. It is not thread-safe and should not be called from an ISR unless the hardware peripheral is exclusively managed by that ISR. Before calling this, you must ensure the JPEG hardware has been properly reset via ca_jpeg_hw_reset and that the input buffers are populated. A common mistake is calling this before the hardware is ready or without configuring the necessary registers, which will result in the hardware failing to start or entering an error state.

Uso:

- This function is used in the JPEG decoding pipeline to kick off the hardware processing after the initial configuration. It is typically paired with ca_jpeg_hw_reset to clear previous states and followed by a polling loop using ca_jpeg_hw_status to monitor the decoding progress. In a typical sequence, you initialize the hardware, call this function to start the decode, and then wait for the completion flag or error bits to be set in the status register.

Esempio:

```
/* Assuming the JPEG hardware has been initialized and buffers are ready */

/* Reset the hardware to a known state before starting a new operation */
ca_jpeg_hw_reset();

/* Trigger the hardware JPEG decoder */
ca_jpeg_hw_decode();

/* Poll the status register to check for completion or errors */
/* ca_jpeg_hw_status() is used to read the current hardware state */
while (!(ca_jpeg_hw_status() & CA_JPEG_HW_DONE)) {
    /* Check for timeout or errors using ca_jpeg_errors() */
    if (ca_jpeg_errors() != 0U) {
        /* Handle error condition */
        break;
    }
    /* Yield to other tasks while waiting for hardware */
    ca_os_yield();
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_jpeg_hw_decode in the decoding pipeline?
A: You should call ca_jpeg_hw_decode after ensuring the JPEG hardware has been properly reset via ca_jpeg_hw_reset and that the input buffers are populated. It is a non-blocking function used to initiate the hardware state machine, and you should follow it with a polling loop using ca_jpeg_hw_status to monitor progress.

Code example conf:0.90 **Q:** Provide a minimal example of how to initiate a JPEG decode operation in caOS.
A: ca_jpeg_hw_reset(); /* Ensure input buffers are populated here */ ca_jpeg_hw_decode(); while (ca_jpeg_hw_status() == JPEG_BUSY) { /* Wait for completion */ }

HW differences conf:1.00 **Q:** Are there differences in how ca_jpeg_hw_decode is implemented across different STM32 families?
A: Yes. For F7xx and H7xx families, the function actively writes to the JPEG peripheral registers (JPEG->CONFR1 and JPEG->CONFR0) to trigger the decode and header parsing. In contrast, for F1xx, F4xx, H5xx, and U5xx families, the function is currently implemented as an empty stub, meaning the hardware JPEG decoding functionality is not active or supported via this function on those specific platforms.

ca_jpeg_hw_encode

Header: caos/drivers/inc/ca_glue_hw.h:436
GUARD: EN_CA_GLUE_HW

```
extern void ca_jpeg_hw_encode(u16 w, u16 h, ca_jpeg_css_t css, u32 mcus)
    ; /* CONFR1..7 for encoding with header, START */
```

Triggers the hardware JPEG encoder peripheral with specified image dimensions and color space.

Parametri:

Nome	Tipo	Descrizione
w	u16	Image width in pixels. Must be a multiple of 8 or 16 depending on the color space subsampling.
h	u16	Image height in pixels. Must be a multiple of 8 or 16 depending on the color space subsampling.
css	ca_jpeg_css_t	Color space subsampling format (e.g., CA_JPEG_GRAY, CA_JPEG_420, CA_JPEG_422).
mcus	u32	Total number of Minimum Coded Units (MCUs) to be processed by the hardware.

Ritorna:
void. This function does not return a status; it directly writes to hardware registers to initiate the encoding process.

Note:

- This function is non-blocking and returns immediately after configuring the hardware registers. It is not thread-safe; if multiple tasks attempt to use the JPEG hardware, you must protect access using a mutex or scheduler lock. It is generally not safe to call from an ISR unless the hardware is guaranteed to be idle. Before calling this, you must ensure the JPEG hardware has been reset and the quantization/Huffman tables have been loaded via other driver functions. A common mistake is providing dimensions that are not aligned with the MCU block size required by the selected subsampling mode, which will cause the hardware to hang or produce corrupted output.

Uso:

- This function is the final step in the JPEG encoding pipeline. It is typically used after calling ca_jpeg_hw_reset() to clear previous states and ca_jpeg_hw_tables() to load the necessary compression coefficients. Once this function is called, the hardware begins processing data; the application must then enter a loop to feed the hardware with raw pixel data using ca_jpeg_hw_put() while monitoring the status via ca_jpeg_hw_status().

Esempio:

```
/* Assuming the JPEG hardware is initialized and tables are loaded */
/* Configure the hardware for a 640x480 image with 4:2:0 subsampling */
ca_jpeg_hw_reset();
/* Load tables using the appropriate driver function */
/* ca_jpeg_hw_tables(q0, q1, dht, ac0, ac1, dc0, dc1); */

/* Start the encoding process */
ca_jpeg_hw_encode(640, 480, CA_JPEG_420, total_mcus);

/* Loop to feed data to the hardware */
while (data_remaining > 0) {
    if (ca_jpeg_hw_status() & CA_JPEG_HW_IN_FREE) {
```

```
        /* Feed the hardware with the next chunk of data */
        /* ca_jpeg_hw_put(data_chunk); */
    }
    /* Yield to other tasks to prevent starvation */
    ca_os_yield();
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** At what point in the JPEG encoding pipeline should I call ca_jpeg_hw_encode?
A: This function is the final step in the pipeline. It should be called only after you have successfully reset the hardware using ca_jpeg_hw_reset() and loaded the necessary quantization and Huffman tables via ca_jpeg_hw_tables(). Once called, the hardware begins processing, and you should proceed to feed it pixel data using ca_jpeg_hw_put() while monitoring status via ca_jpeg_hw_status().

Code example conf:0.90 **Q:** Provide a minimal C example of how to initiate a JPEG encoding session using caOS types.
A: u16 width = 640; u16 height = 480; u32 mcus = 1200; ca_jpeg_css_t format = CA_JPEG_420; ca_jpeg_hw_reset(); ca_jpeg_hw_tables(); ca_jpeg_hw_encode(width, height, format, mcus);

Parameters conf:1.00 **Q:** What are the constraints on the width and height parameters for ca_jpeg_hw_encode?
A: The width and height must be multiples of 8 or 16, depending on the specific color space subsampling mode selected. Providing dimensions that are not aligned with the required MCU block size for the chosen subsampling mode will cause the hardware to hang or produce corrupted output.

Error/Edge case conf:1.00 **Q:** What happens if I provide incorrect dimensions to the encoder?
A: Providing dimensions that are not aligned with the MCU block size required by the selected subsampling mode is a common mistake that will cause the hardware to hang or result in corrupted output.

Behavior conf:1.00 **Q:** Is ca_jpeg_hw_encode thread-safe or safe to call from an ISR?
A: The function is non-blocking and returns immediately. It is not thread-safe; you must use a mutex or scheduler lock if multiple tasks access the hardware. It is generally not safe to call from an ISR unless you can guarantee the hardware is idle.

HW differences conf:1.00 **Q:** How does the implementation of ca_jpeg_hw_encode differ across STM32 families?
A: On F7xx and H7xx families, the function actively configures registers such as CONFR1 through CONFR7 and triggers the process via JPEG_CONFR0_START. In contrast, for F1xx, F4xx, H5xx, and U5xx families, the provided implementation is a stub that performs no hardware operations, as indicated by the empty function bodies.

`ca_jpeg_hw_get`

Header: `caos/drivers/inc/ca_glue_hw.h:440`
GUARD: `EN_CA_GLUE_HW`

```
extern u32 ca_jpeg_hw_get(void); /* output FIFO */
```

Reads a 32-bit word from the JPEG hardware output FIFO register.

Ritorna:
Returns a 32-bit unsigned integer representing the data word currently available in the JPEG hardware output register (DOR).

Note:

- This function is a low-level hardware abstraction layer (HAL) call that directly accesses the peripheral register. It is non-blocking and does not perform any status checks; therefore, calling it when the FIFO is empty will return invalid or stale data. It is not thread-safe by itself and should only be called when the hardware status indicates that output data is ready (e.g., via a status flag check). It is generally safe to call from an ISR if the hardware interrupt has signaled that data is available, but it must never be called before the JPEG peripheral has been initialized.

Uso:

- This function is used during the JPEG decoding process to extract processed pixel data from the hardware. It is typically called within a loop after verifying that the hardware status register indicates data availability (e.g., checking for a specific 'output data ready' bit). It is used in conjunction with ca_jpeg_init to prepare the hardware, and ca_jpeg_hw_status to poll for readiness before calling ca_jpeg_hw_get to retrieve the actual data words.

Esempio:

```
/* Assuming JPEG hardware is already initialized */

/* Check if output data is available in the hardware FIFO */
if (ca_jpeg_hw_status() & 0x00000002U) { /* Assuming 0x2 is the OUT_DATA flag */
    /* Retrieve the 32-bit word from the hardware register */
    u32 data = ca_jpeg_hw_get();

    /* Process the 4 bytes contained in the 32-bit word */
    for (u32 i = 0U; i < 4U; i++) {
        u8 byte = (u8)((data >> (8U * i)) & 0xFFU);
        /* Store byte in a buffer or process further */
    }
} else {
    /* Handle case where no data is ready to be read */
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_jpeg_hw_get in my application code?
A: You should call ca_jpeg_hw_get during the JPEG decoding process to extract processed pixel data. It must be called only after verifying that the hardware status register indicates data availability, typically by using ca_jpeg_hw_status to poll for readiness. Ensure the JPEG peripheral has been initialized via ca_jpeg_init before calling this function.

Code example conf:1.00 **Q:** Provide a minimal example of how to safely read data from the JPEG hardware.
A: u32 data; if (ca_jpeg_hw_status()) { data = ca_jpeg_hw_get(); }

Return conf:1.00 **Q:** What does the return value of ca_jpeg_hw_get represent, and what happens if the FIFO is empty?
A: The function returns a 32-bit unsigned integer representing the data word currently available in the JPEG hardware output register (DOR). Because the function is non-blocking and performs no status checks, calling it when the FIFO is empty will return invalid or stale data.

HW differences conf:1.00 **Q:** Are there differences in how ca_jpeg_hw_get is implemented across different STM32 families?
A: Yes. On the F7xx and H7xx families, the function directly accesses the JPEG->DOR register. In other families like F1xx, F4xx, H5xx, and U5xx, the provided implementation is a stub that returns 0U, indicating that the hardware-specific register access is not implemented or supported in those specific driver files.

ca_jpeg_hw_header

Header: caos/drivers/inc/ca_glue_hw.h:441
GUARD: EN_CA_GLUE_HW

```
extern void ca_jpeg_hw_header(u16 *w, u16 *h, u32 *nf, u32 *hsf, u32 *
    vsf); /* parsed: size, components, Y sampling */
```

Extracts JPEG image dimensions and color sampling factors from the hardware decoder registers.

Parametri:

Nome	Tipo	Descrizione
w	u16 *	Pointer to store the image width in pixels.
h	u16 *	Pointer to store the image height in pixels.
nf	u32 *	Pointer to store the number of color components (e.g., 1 for Grayscale, 3 for YCbCr).
hsf	u32 *	Pointer to store the horizontal sampling factor.
vsf	u32 *	Pointer to store the vertical sampling factor.

Ritorna:
This function returns void. It updates the memory locations pointed to by the arguments. If the hardware does not support JPEG decoding (e.g., F1xx, F4xx), all pointed values are set to 0.

Note:

- This function is a low-level hardware abstraction. It is not thread-safe and should only be called when the JPEG hardware has successfully parsed a header (typically indicated by a status flag). It is not intended to be called from an ISR. The most common mistake is calling this function before the hardware has finished the header parsing phase, which will result in reading stale or zeroed register values.

Uso:

- This function is used during the JPEG decoding sequence to determine the geometry and color format of the incoming stream. It is typically called after the driver detects a header-ready event (e.g., via a status check). Once the dimensions and sampling factors are retrieved, the application uses them to configure the output buffer or geometry via ca_jpeg_geometry before proceeding with the actual decoding process.

Esempio:

```
/* Assuming the JPEG hardware has signaled that a header is ready */
u16 width = 0, height = 0;
u32 num_components = 0, h_sampling = 0, v_sampling = 0;

/* Extract header information from hardware registers */
ca_jpeg_hw_header(&width, &height, &num_components, &h_sampling, &v_sampling);

/* Validate geometry and set up the decoder */
/* ca_jpeg_geometry is used to verify if the decoded size fits the application
    requirements */
if (width > 0 && height > 0) {
    /* Proceed with decoding logic */
} else {
    /* Handle error: invalid header data */
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to call ca_jpeg_hw_header in the decoding sequence?
A: It should be called only after the JPEG hardware has successfully parsed a header, typically indicated by a status flag. It is used to retrieve image geometry and color format information to configure output buffers or geometry via ca_jpeg_geometry before proceeding with the actual decoding.

Code example conf:1.00 **Q:** Provide a minimal C example showing how to retrieve JPEG header information using caOS types.
A: u16 width, height; u32 components, hsf, vsf; ca_jpeg_hw_header(&width, &height, &components, &hsf, &vsf);

Parameters conf:1.00 **Q:** What do the nf, hsf, and vsf parameters represent in the ca_jpeg_hw_header function?
A: The nf parameter stores the number of color components (e.g., 1 for Grayscale, 3 for YCbCr). The hsf and vsf parameters store the horizontal and vertical sampling factors, respectively.

HW differences conf:1.00 **Q:** How does the implementation of ca_jpeg_hw_header differ between STM32 families like F4xx and F7xx?
A: On F7xx and H7xx families, the function actively reads hardware registers such as JPEG->CONFR1, CONFR3, and CONFR4 to extract dimensions and sampling factors. On families like F1xx, F4xx, H5xx, and U5xx, the function is a stub that sets all output pointers to 0, indicating that the hardware does not support JPEG decoding.

`ca_jpeg_hw_init`

Header: `caos/drivers/inc/ca_glue_hw.h:432`
GUARD: `EN_CA_GLUE_HW`

```
1 extern bool ca_jpeg_hw_init(void); /* clock, core enabled; false = no
     codec */
```

Initializes the hardware JPEG codec peripheral clock and enables the core.

Ritorna:
Returns true if the hardware JPEG codec was successfully initialized and is ready for use. Returns false if the hardware does not support a JPEG codec or if the initialization failed.

Note:

- This function is a low-level hardware abstraction layer (HAL) call. It is not thread-safe and should only be called during the system initialization phase or from a single dedicated task before any JPEG operations are performed. It is not intended to be called from an Interrupt Service Routine (ISR) as it performs direct register manipulation. A common mistake is calling this function multiple times or while a JPEG operation is already in progress, which can lead to undefined hardware states or register corruption.

Uso:

- This function is typically called internally by the higher-level ca_jpeg_init function. It serves as the mandatory first step in the JPEG driver lifecycle: hardware initialization -> software table setup -> codec usage. You should never call this function directly in your application code; instead, always use ca_jpeg_init to ensure the driver state is tracked correctly and the codec is ready for operations like ca_jpeg_decode or ca_jpeg_encode.

Esempio:

```c
/* Example of initializing the JPEG subsystem safely */
#include "caos.h"

void setup_imaging_task(void) {
    /* Always use the high-level init function, which calls ca_jpeg_hw_init internally */
    if (ca_jpeg_init()) {
        /* Codec is ready, safe to proceed with operations */
        // ca_jpeg_decode(...);
    } else {
        /* Handle error: hardware might be missing or failed to initialize */
        ca_printf("JPEG codec initialization failed!\n");
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** Can I call ca_jpeg_hw_init directly in my application code to reset the JPEG codec?
A: No, you should never call ca_jpeg_hw_init directly. It is a low-level function intended to be called internally by ca_jpeg_init. Calling it directly or multiple times can lead to undefined hardware states or register corruption. It must only be used during system initialization or from a single dedicated task.

Code example conf:0.90 **Q:** How should I initialize the JPEG codec in caOS?
A: You should use the high-level function ca_jpeg_init, which manages the driver state and calls ca_jpeg_hw_init internally. Example: bool status = ca_jpeg_init(); if (status) { /* proceed with codec usage */ }

Return conf:1.00 **Q:** What does the boolean return value of ca_jpeg_hw_init indicate?
A: Returning true indicates the hardware JPEG codec peripheral clock was successfully enabled and the core is ready for use. Returning false indicates that the hardware does not support a JPEG codec or that the initialization failed.

HW differences conf:1.00 **Q:** How does the hardware initialization differ between the F7xx and H7xx families?
A: The F7xx family initializes the codec by setting the RCC_AHB2ENR_JPEGEN bit in the RCC->AHB2ENR register, whereas the H7xx family enables the codec via the RCC_AHB3ENR_JPGDECEN bit in the RCC->AHB3ENR register. Both families subsequently enable the core by setting the JPEG_CR_JCEN bit in the JPEG->CR register.

`ca_jpeg_hw_put`

Header: `caos/drivers/inc/ca_glue_hw.h:439`
GUARD: `EN_CA_GLUE_HW`

```c
extern void ca_jpeg_hw_put(u32 w); /* input FIFO */
```

Writes a 32-bit word into the JPEG hardware input FIFO.

Parametri:

Nome	Tipo	Descrizione
`w`	`u32`	The 32-bit data word to be pushed into the hardware input register. This typically represents a chunk of the raw image data or MCU (Minimum Coded Unit) stream.

Ritorna:

void

Note:

- This function is a low-level hardware abstraction layer (HAL) call that directly writes to the JPEG peripheral's data input register. It is non-blocking and does not perform any status checks; therefore, you must ensure the hardware is ready to receive data (e.g., by checking the status register) before calling it, otherwise, you risk data loss or bus contention. It is generally not thread-safe if multiple tasks attempt to access the JPEG peripheral simultaneously without external synchronization. While it can technically be called from an ISR, it is usually called within a task loop that manages the data stream to the hardware.

Uso:

- This function is used during the data streaming phase of JPEG encoding or decoding. It is typically paired with ca_jpeg_hw_status to verify that the input FIFO is not full before pushing data. The standard sequence is: initialize the JPEG hardware, enter a loop where you check the status, call ca_jpeg_hw_put to feed the hardware, and finally handle the output data using the corresponding get function once the hardware signals that data is available.

Esempio:

```c
/* Assuming the JPEG hardware has been initialized and is ready for data */
/* We check the status to ensure the input FIFO has space before writing */
if (ca_jpeg_hw_status() & CA_JPEG_HW_IN_FREE) {
    /* Prepare a 32-bit word from the source buffer */
    u32 data_word = 0x12345678;

    /* Push the word into the JPEG hardware input FIFO */
    ca_jpeg_hw_put(data_word);
} else {
    /* Hardware is busy, wait or yield to other tasks */
    ca_os_yield();
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_jpeg_hw_put in my application logic?
A: You should call it during the data streaming phase of JPEG encoding or decoding. It is intended to be used within a task loop where you first verify the hardware is ready using ca_jpeg_hw_status, then push the 32-bit data word into the input FIFO.

Code example conf:0.90 **Q:** Provide a minimal example of how to feed data to the JPEG hardware using caOS types.
A: u32 data_chunk = 0x12345678; if (ca_jpeg_hw_status() == 0U) { ca_jpeg_hw_put(data_chunk); }

Parameters conf:1.00 **Q:** What does the u32 w parameter represent in ca_jpeg_hw_put?
A: The parameter w is a 32-bit data word representing a chunk of the raw image data or the Minimum Coded Unit (MCU) stream to be pushed into the JPEG hardware input register.

HW differences conf:1.00 **Q:** Are there differences in how ca_jpeg_hw_put is implemented across different STM32 families?
A: Yes. On F7xx and H7xx families, the function directly writes to the JPEG->DIR register. On other families like F1xx, F4xx, H5xx, and U5xx, the implementation is currently a stub that performs no action, as indicated by the (void)w cast.

---

ca_jpeg_hw_reset

Header: caos/drivers/inc/ca_glue_hw.h:433
GUARD: EN_CA_GLUE_HW

```
extern void ca_jpeg_hw_reset(void); /* stop, FIFOs flushed, flags
    cleared, core re-enabled */
```

Performs a hardware-level reset of the JPEG codec peripheral to clear internal state and FIFOs.

Ritorna:
void

Note:

- This function is a low-level hardware reset that stops the JPEG engine, flushes internal FIFOs, and clears status flags. It is not thread-safe and should not be called from an ISR unless you are certain no other task is currently interacting with the JPEG peripheral. The function relies on the peripheral being clocked and initialized; calling it before the hardware is powered or clocked may result in a bus fault or undefined behavior. A common mistake is calling this while a DMA transfer is still active, which can lead to corrupted memory or a hung peripheral state.

Uso:

- This function is typically used as a recovery mechanism or a mandatory initialization step before starting a new JPEG encoding or decoding operation. It is usually paired with ca_jpeg_init to ensure the hardware is in a known clean state before configuring tables or geometry. In a typical sequence, you would call ca_jpeg_hw_reset, then configure the codec parameters, and finally trigger the operation using the appropriate hardware-specific start functions.

Esempio:

```
/* Ensure the JPEG hardware is in a clean state before starting a new encoding
    task */
ca_jpeg_hw_reset();

/* Configure the codec tables and geometry before starting the process */
/* ca_jpeg_hw_tables(...) - hypothetical call to set quantization/Huffman tables
    */
/* ca_jpeg_hw_encode(...) - hypothetical call to start the hardware engine */

/* Wait for completion or handle potential errors */
if (ca_jpeg_errors() != 0) {
    /* If an error occurred, reset the hardware again to recover */
    ca_jpeg_hw_reset();
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_jpeg_hw_reset in my application flow?
A: You should call ca_jpeg_hw_reset as a mandatory initialization step before starting a new JPEG encoding or decoding operation. It is typically used to ensure the hardware is in a known clean state before configuring tables or geometry, often paired with ca_jpeg_init.

Code example conf:0.90 **Q:** How do I properly reset the JPEG peripheral before starting a new task?
A: You should ensure the peripheral is clocked, then call the reset function before configuring parameters. Example:

void prepare_jpeg(void) { ca_jpeg_hw_reset(); /* Proceed to configure codec parameters and start operation */ }

Error/Edge case conf:1.00 **Q:** What are the risks of calling ca_jpeg_hw_reset while a DMA transfer is active?
A: Calling this function while a DMA transfer is active is a common mistake that can lead to corrupted memory or a hung peripheral state. Additionally, it is not thread-safe and should not be called from an ISR unless you are certain no other task is interacting with the JPEG peripheral.

HW differences conf:1.00 **Q:** Does ca_jpeg_hw_reset perform the same actions on all STM32 families?
A: No. On F7xx and H7xx families, the function performs active register manipulation (e.g., clearing CONFR0, CR, and CFR registers and using spin-loops on SR flags). On F1xx, F4xx, H5xx, and U5xx families, the provided implementation is empty, meaning the function does not perform any hardware-level reset actions on those specific architectures.

---

ca_jpeg_hw_status

Header: `caos/drivers/inc/ca_glue_hw.h:438`
GUARD: `EN_CA_GLUE_HW`

```
1 extern u32 ca_jpeg_hw_status(void); /* CA_JPEG_HW_* */
```

Returns the current hardware status flags of the JPEG peripheral.

Ritorna:
A bitmask of status flags: CA_JPEG_HW_IN_FREE (input buffer ready), CA_JPEG_HW_OUT_DATA (output data available), CA_JPEG_HW_EOC (end of conversion), and CA_JPEG_HW_HDR (header processed). Returns 0 if no flags are set or if the peripheral is not supported on the current hardware.

Note:

- This function is non-blocking and reads directly from the hardware status register. It is thread-safe in the sense that it performs a single atomic read of the register, but it does not provide synchronization for the data buffers themselves. It can be called from an ISR, but it is typically used in a polling loop within a task to manage data flow. A common beginner mistake is to assume that checking the status once is sufficient; you must continuously poll these flags while feeding or draining the JPEG hardware buffers to ensure the process does not stall.

Uso:

- This function is used to monitor the progress of a JPEG encoding or decoding operation. It is typically called inside a loop after initializing the hardware with ca_jpeg_init and starting the process. It is used to decide when to call functions that push data into the hardware or pull processed data out, ensuring that the peripheral's internal FIFOs do not overflow or underflow.

Esempio:

```
1 /* Example: Polling JPEG status to process data */
2 #include "caos.h"
3
4 void process_jpeg_data(void) {
5     /* Assume hardware is already initialized */
6     u32 status = ca_jpeg_hw_status();
```

```

    /* Check if the input buffer is ready to accept more data */
    if (status & CA_JPEG_HW_IN_FREE) {
        /* Call function to write data to hardware register */
        /* ca_jpeg_hw_put(data); */
    }

    /* Check if output data is available to be read */
    if (status & CA_JPEG_HW_OUT_DATA) {
        /* Call function to read data from hardware register */
        /* u32 data = ca_jpeg_hw_get(); */
    }

    /* Check if conversion is complete */
    if (status & CA_JPEG_HW_EOC) {
        /* Handle end of conversion */
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_jpeg_hw_status and why is it necessary?
A: You should call ca_jpeg_hw_status continuously within a polling loop while encoding or decoding. It is necessary to monitor the hardware's progress and manage data flow, specifically to determine when to push data into the input buffer or pull processed data out, preventing FIFO overflow or underflow.

Code example conf:0.90 **Q:** How do I implement a simple polling loop to check for available output data?
A: You can poll the status register as follows: while(!(ca_jpeg_hw_status() & CA_JPEG_HW_OUT_ { /* wait or yield */ } // Process output data here.

Return conf:1.00 **Q:** What do the return flags represent and what does a return value of 0 indicate?
A: The return value is a bitmask: CA_JPEG_HW_IN_FREE indicates the input buffer is ready, CA_JPEG_HW_OUT_DATA indicates output data is available, CA_JPEG_HW_EOC indicates end of conversion, and CA_JPEG_HW_HDR indicates the header is processed. A return value of 0 means no flags are set or the peripheral is not supported on the current hardware.

HW differences conf:1.00 **Q:** Are there differences in how ca_jpeg_hw_status behaves across different STM32 families?
A: Yes. On F7xx and H7xx families, the function reads the JPEG->SR register to return status flags. On other families (F1xx, F4xx, H5xx, U5xx), the function is implemented as a stub that always returns 0, indicating that the JPEG peripheral is not supported or implemented in those specific drivers.

`ca_jpeg_hw_tables`

Header: `caos/drivers/inc/ca_glue_hw.h:434`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_jpeg_hw_tables(const u32 *q0, const u32 *q1, const u32 *
    dht, const u32 *ac0, const u32 *ac1, const u32 *dc0, const u32 *dc1);
     /* encoder memories, words as built by jpeg.c */
```

Configures the JPEG hardware encoder with quantization and Huffman tables.

Parametri:

Nome	Tipo	Descrizione
q0	const u32 *	Pointer to the 16-word Quantization Table 0.
q1	const u32 *	Pointer to the 16-word Quantization Table 1.
dht	const u32 *	Pointer to the 103-word Huffman Table data.
ac0	const u32 *	Pointer to the 88-word AC Huffman Table 0.
ac1	const u32 *	Pointer to the 88-word AC Huffman Table 1.
dc0	const u32 *	Pointer to the 8-word DC Huffman Table 0.
dc1	const u32 *	Pointer to the 8-word DC Huffman Table 1.

Ritorna:
void

Note:

- This function is non-blocking and performs a direct memory copy into the JPEG hardware registers. It is not thread-safe; if multiple tasks attempt to configure the JPEG hardware simultaneously, the tables will be corrupted. It must not be called from an ISR because it performs a significant amount of memory copying which could violate timing constraints. Precondition: The JPEG hardware must be powered and initialized before calling this function. A common mistake is passing unaligned or null pointers, which will cause a hard fault or undefined behavior in the hardware registers.

Uso:

- This function is used during the JPEG encoding initialization sequence. It is typically called immediately after ca_jpeg_hw_reset() to ensure the hardware is in a clean state before loading the specific compression tables required for the image. It is followed by the actual encoding trigger function. In a typical lifecycle, you initialize the hardware, load these tables, perform the encoding, and then potentially reset the hardware for the next frame.

Esempio:

```
/* Assuming tables are defined globally or locally as u32 arrays */
extern const u32 my_q0[16];
extern const u32 my_q1[16];
extern const u32 my_dht[103];
extern const u32 my_ac0[88];
extern const u32 my_ac1[88];
extern const u32 my_dc0[8];
extern const u32 my_dc1[8];

void setup_and_encode(void) {
    /* Reset hardware to clear previous state */
    ca_jpeg_hw_reset();

    /* Load the tables into the hardware registers */
    ca_jpeg_hw_tables(my_q0, my_q1, my_dht, my_ac0, my_ac1, my_dc0, my_dc1);

    /* Proceed to trigger the hardware encoding process */
    /* ca_jpeg_hw_encode(...) is not in the catalog, so we use a comment */
    /* Trigger hardware encoding with specified geometry */
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** At what point in the JPEG encoding lifecycle should I call ca_jpeg_hw_tables?
A: It should be called during the initialization sequence, typically immediately after ca_jpeg_hw_reset() to ensure the hardware is in a clean state. It must be called before the actual encoding trigger function to load the necessary compression tables.

Code example conf:1.00 **Q:** Provide a minimal example of how to call ca_jpeg_hw_tables using caOS types.
A: u32 q0[16], q1[16], dht[103], ac0[88], ac1[88], dc0[8], dc1[8]; /* Assume tables are populated */ ca_jpeg_hw_tables(q0, q1, dht, ac0, ac1, dc0, dc1);

Parameters conf:1.00 **Q:** What are the requirements for the pointers passed to ca_jpeg_hw_tables?
A: The pointers must point to valid memory regions of the specified word counts (e.g., 16 words for q0/q1, 103 for dht). Passing null or unaligned pointers is a common mistake that will cause a hard fault or undefined behavior in the hardware registers.

Error/Edge case conf:1.00 **Q:** Can I call ca_jpeg_hw_tables from an interrupt service routine (ISR)?
A: No. You must not call it from an ISR because it performs a significant amount of memory copying, which could violate timing constraints. Additionally, the function is not thread-safe and could lead to table corruption if multiple tasks access it simultaneously.

HW differences conf:1.00 **Q:** How does the implementation of ca_jpeg_hw_tables differ across STM32 families?
A: On F7xx and H7xx families, the function performs direct memory copies into specific hardware registers like JPEG->QMEM0, JPEG->DHTMEM, and JPEG->HUFFENC_AC0. On other families like F1xx, F4xx, H5xx, and U5xx, the function is currently implemented as a stub that performs no operations.

`ca_jpeg_init`

Header: `caos/drivers/inc/jpeg.h:78`
Source: `caos/drivers/src/jpeg.c:231:240`
GUARD: `EN_JPEG`

```
bool ca_jpeg_init(void); /* false = no JPEG codec */
```

Initializes the hardware JPEG codec and prepares internal lookup tables for image processing.

Ritorna:
Returns true if the JPEG hardware is successfully initialized or was already ready. Returns false if the hardware codec is not present or initialization failed.

Note:

- This function is not thread-safe and should be called once during system startup before any other ca_jpeg_* functions are used. It is a blocking call because it performs hardware register configuration and table generation. It must NOT be called from an ISR, as it may involve timing-sensitive hardware initialization. A common mistake is attempting to use JPEG functions without checking the return value of this function, which will lead to undefined behavior if the specific STM32 chip variant lacks a hardware JPEG codec.

Uso:

- This function acts as the mandatory gatekeeper for the JPEG subsystem. It is typically called within the system configuration phase (e.g., inside ca_os_configure) to verify hardware capabilities before spawning tasks that perform image processing. It pairs with ca_jpeg_decode or ca_jpeg_encode; if this function returns false, the application should gracefully disable image-related features or halt to prevent illegal hardware access.

Esempio:

```
void app_jpeg_task(void) {
    /* Initialize the JPEG hardware before any processing */
```

```
    if (!ca_jpeg_init()) {
        /* Hardware not supported or init failed, log error and exit task */
        ca_printf("JPEG codec not available on this device!\n");
        return;
    }

    /* Proceed to use other JPEG functions safely */
    /* ca_jpeg_decode(...) would be called here */
    ca_printf("JPEG codec ready for operations.\n");
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to call ca_jpeg_init in a caOS application?
A: ca_jpeg_init should be called once during the system startup phase, such as inside ca_os_configure, before any other ca_jpeg_* functions are invoked. It serves as a mandatory gatekeeper to verify hardware capabilities before spawning tasks that perform image processing.

Code example conf:1.00 **Q:** How do I correctly initialize the JPEG subsystem in my caOS project?
A: You should call the function and check the boolean return value to ensure the hardware is present before proceeding:
if (ca_jpeg_init()) { /* Proceed with JPEG operations */ } else { /* Handle hardware absence or failure */ }

Return conf:1.00 **Q:** What do the return values of ca_jpeg_init indicate, and why might it return false?
A: A return value of true indicates the JPEG hardware is successfully initialized or was already ready. A return value of false indicates that the hardware codec is not present on the specific STM32 chip variant or that the initialization process failed.

`ca_jpeg_mcu_to_rgb`

Header: `caos/drivers/inc/jpeg.h:88`
Source: `caos/drivers/src/jpeg.c:419:444`
GUARD: `EN_JPEG`

```
void ca_jpeg_mcu_to_rgb(const u8 *mcu, u16 width, u16 height,
    ca_jpeg_css_t css, u8 *img)
```

Converts JPEG MCU (Minimum Coded Unit) data into RGB pixel format.

Parametri:

Nome	Tipo	Descrizione
`mcu`	`const u8 *`	Pointer to the raw MCU data buffer. Must be valid and large enough for the specified geometry.
`width`	`u16`	Width of the target image area in pixels.
`height`	`u16`	Height of the target image area in pixels.
`css`	`ca_jpeg_css_t`	Chroma Subsampling format (e.g., 4:2:0, 4:2:2, or Gray).
`img`	`u8 *`	Pointer to the destination buffer where RGB pixels will be stored. Must be pre-allocated.

Ritorna:
void. If parameters are invalid (NULL pointers) or geometry calculation fails, the function returns immediately without modifying the destination buffer.

Note:

- This function is CPU-intensive and performs floating-point-like fixed-point arithmetic to convert YCbCr color space to RGB. It is non-blocking but synchronous, meaning it will consume the calling task's time slice until completion. It is thread-safe provided that the input and output buffers are not accessed by other tasks simultaneously. It is NOT safe to call from an ISR due to the high computational load and potential for long execution time. A common mistake is providing an output buffer that is too small for the calculated width and height, which will lead to memory corruption.

Uso:

- Used during the final stage of JPEG decoding to transform raw MCU blocks into a displayable RGB format. It is typically used after calling ca_jpeg_geometry to validate dimensions and before passing the resulting RGB buffer to a display driver. It is often paired with ca_memory_allocate to ensure the destination buffer is correctly sized based on the image dimensions.

Esempio:

```c
/* Assuming r->w, r->h, and r->css are already validated */
u8 *rgb_buffer = ca_memory_allocate(r->w * r->h * 3U);
if (rgb_buffer != NULL) {
    /* Convert the decoded MCU data into the RGB buffer */
    ca_jpeg_mcu_to_rgb(mcu_data_ptr, r->w, r->h, (ca_jpeg_css_t)r->css,
    rgb_buffer);

    /* Proceed to display or process the RGB data */
    /* ... */

    ca_memory_free(rgb_buffer);
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** At what stage of the JPEG decoding process should I call ca_jpeg_mcu_to_rgb?
A: It should be called during the final stage of JPEG decoding to transform raw MCU blocks into a displayable RGB format. It is typically used after calling ca_jpeg_geometry to validate dimensions and before passing the resulting RGB buffer to a display driver.

Code example conf:0.90 **Q:** Provide a minimal example of how to invoke ca_jpeg_mcu_to_rgb using caOS types.
A: u8 *mcu_data; u8 *rgb_buffer; u16 w = 8; u16 h = 8; ca_jpeg_css_t css = CA_JPEG_CSS_420; ca_jpeg_mcu_to_rgb(mcu_data, w, h, css, rgb_buffer);

Parameters conf:1.00 **Q:** What happens if I pass a NULL pointer to the mcu or img parameters?
A: If parameters are invalid, such as providing NULL pointers, the function returns immediately without modifying the destination buffer.

Error/Edge case conf:1.00 **Q:** What is a common mistake when using the img buffer in ca_jpeg_mcu_to_rgb?
A: A common mistake is providing an output buffer that is too small for the calculated width and height, which will lead to memory corruption.

Behavior conf:1.00 **Q:** Is it safe to call ca_jpeg_mcu_to_rgb from an ISR?
A: No, it is not safe to call from an ISR due to the high computational load and potential for long execution time. The function is non-blocking but synchronous, meaning it consumes the calling task's time slice until completion.

`ca_jpeg_rgb_to_mcu`

Header: `caos/drivers/inc/jpeg.h:87`
Source: `caos/drivers/src/jpeg.c:378:417`

GUARD: `EN_JPEG`

```
void ca_jpeg_rgb_to_mcu(const u8 *img, u16 width, u16 height,
    ca_jpeg_css_t css, u8 *mcu)
```

Converts raw RGB image data into JPEG-compliant MCU (Minimum Coded Unit) blocks.

Parametri:

Nome	Tipo	Descrizione
`img`	`const u8 *`	Pointer to the source raw RGB buffer. Must not be NULL.
`width`	`u16`	Width of the source image in pixels.
`height`	`u16`	Height of the source image in pixels.
`css`	`ca_jpeg_css_t`	Chroma Subsampling format (e.g., 4:2:0, 4:2:2, or Grayscale).
`mcu`	`u8 *`	Pointer to the destination buffer where MCU blocks will be stored. Must be large enough to hold the processed data.

Ritorna:
void. If input pointers are NULL or geometry calculation fails, the function returns immediately without processing.

Note:

- This function performs heavy mathematical operations (color space conversion and subsampling) and is CPU-intensive. It is not thread-safe regarding the destination buffer; ensure no other task writes to the 'mcu' buffer while this is running. It is NOT safe to call from an ISR due to the high execution time, which would violate real-time constraints. A common mistake is providing an 'mcu' buffer that is too small, leading to memory corruption; always verify the required size using internal geometry calculations before calling.

Uso:

- This function is the mandatory intermediate step between raw image acquisition and JPEG encoding. It is typically used in a sequence: first, define the configuration (width, height, CSS), then call this function to transform the raw RGB pixels into the MCU format required by the JPEG encoder, and finally pass the resulting MCU buffer to ca_jpeg_encode. It is used when preparing image data for storage or network transmission.

Esempio:

```
/* Assuming s_img is a pre-allocated raw RGB buffer and s_mcu is the destination
    buffer */
ca_jpeg_cfg_t cfg = { .width = 320, .height = 240, .css = CA_JPEG_420, .quality
    = 75 };

/* Validate geometry before processing to avoid buffer overflows */
ca_jpeg_info_t info;
if (ca_jpeg_geometry(cfg.width, cfg.height, cfg.css, &info)) {
    /* Convert raw RGB to MCU format for the encoder */
    ca_jpeg_rgb_to_mcu(s_img, cfg.width, cfg.height, cfg.css, s_mcu);

    /* Now the data is ready for the encoder */
    u32 encoded_len = 0;
    ca_jpeg_encode(s_mcu, &cfg, out_buffer, OUT_BUFFER_SIZE, &encoded_len, 1000)
    ;
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** At what point in the image processing pipeline should I call ca_jpeg_rgb_to_mcu?
A: This function is the mandatory intermediate step between raw image acquisition and JPEG encoding. You should call it after defining your image configuration (width, height, and CSS) to transform raw RGB pixels into the MCU format, and before passing the resulting buffer to ca_jpeg_encode.

Code example conf:0.90 **Q:** Provide a minimal C example showing how to call ca_jpeg_rgb_to_mcu using caOS types.
A: u8 *raw_data = get_camera_buffer(); u8 *mcu_buffer = get_mcu_buffer(); u16 w = 640; u16 h = 480; ca_jpeg_css_t css = CA_JPEG_CSS_420; ca_jpeg_rgb_to_mcu(raw_data, w, h, css, mcu_buffer);

Parameters conf:1.00 **Q:** What does the ca_jpeg_css_t parameter represent and what are some valid examples?
A: The ca_jpeg_css_t parameter defines the Chroma Subsampling format used for the conversion. Valid examples include 4:2:0, 4:2:2, or Grayscale.

Error/Edge case conf:1.00 **Q:** What happens if I provide an insufficient buffer size for the mcu parameter?
A: Providing an 'mcu' buffer that is too small is a common mistake that leads to memory corruption. You must verify the required size using internal geometry calculations before calling the function.

Behavior conf:1.00 **Q:** Is it safe to call ca_jpeg_rgb_to_mcu from an Interrupt Service Routine (ISR)?
A: No, it is not safe to call this function from an ISR. Because the function performs heavy mathematical operations and is CPU-intensive, calling it from an ISR would violate real-time constraints.

`ca_kdf_selftest`

Header: `caos/crypto/inc/kdf.h:74`
Source: `caos/crypto/src/kdf.c:195:255`
GUARD: `EN_KDF`

```
u32 ca_kdf_selftest(void)
```

Executes a suite of cryptographic known-answer tests (KAT) to verify the integrity of KDF implementations.

Ritorna:
Returns a u32 representing the total number of failed test cases. A return value of 0 indicates that all cryptographic algorithms passed their respective tests.

Note:

- This function is intended for system diagnostic and validation purposes, typically called during the boot sequence or as part of a health check. It is a blocking function that performs heavy computational work; calling it from an ISR or a high-priority real-time task will cause significant jitter and likely trigger a watchdog reset. It is not thread-safe in the sense that it modifies local buffers and performs I/O, so it should only be executed by a single dedicated diagnostic task. The most common mistake is calling this function during normal operation, which will freeze the system for several milliseconds while the HMAC and PBKDF2 calculations complete.

Uso:

- Use this function during the system initialization phase to ensure that the cryptographic hardware or software libraries are functioning correctly before enabling secure communication services. It is often paired with ca_os_tick_ms to measure the performance of the KDF implementation, as shown in the provided examples. After the system has booted and passed these tests, the results can be logged via ca_stdout_printf_n to provide a record of the hardware's cryptographic health.

Esempio:

```c
/* Diagnostic task to verify crypto integrity at startup */
void crypto_diagnostic_task(void *pvParameters) {
    /* Wait for system to stabilize */
    ca_os_delay_ms(100);

    /* Run the self-test suite */
    u32 error_count = ca_kdf_selftest();

    if (error_count > 0) {
        /* Log failure and enter a safe state */
        ca_printf_n("CRITICAL: Crypto self-test failed with %u errors",
    error_count);
        /* Perform system reset or halt */
        ca_os_reset();
    } else {
        ca_printf_n("Crypto self-test passed successfully.");
    }

    /* Task finished, suspend or delete */
    ca_os_job_suspend(ca_os_self_handle());
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to execute ca_kdf_selftest in a caOS application?
A: It should be called during the system initialization phase or boot sequence to verify the integrity of cryptographic implementations before enabling secure communication services.

Code example conf:0.90 **Q:** How do I implement a basic diagnostic check using ca_kdf_selftest?
A: u32 failures = ca_kdf_selftest(); if (failures == 0) { ca_stdout_printf_n("KDF health check passed"); } else { ca_stdout_printf_n("KDF failed with %u errors", failures); }

Return conf:1.00 **Q:** What does the u32 return value from ca_kdf_selftest signify?
A: The return value represents the total number of failed cryptographic test cases. A return value of 0 indicates that all algorithms passed their tests successfully.

Error/Edge case conf:1.00 **Q:** What is the most common mistake developers make when using ca_kdf_selftest?
A: The most common mistake is calling the function during normal system operation, which causes the system to freeze for several milliseconds while HMAC and PBKDF2 calculations complete.

Behavior conf:1.00 **Q:** Is ca_kdf_selftest safe to call from an ISR or a high-priority task?
A: No, it is a blocking function that performs heavy computational work. Calling it from an ISR or high-priority task will cause significant jitter and likely trigger a watchdog reset. It is also not thread-safe and should only be executed by a single dedicated diagnostic task.

ca_keys_store_allocate

Header: caos/crypto/inc/keys_store.h:87
Source: caos/crypto/src/keys_store.c:115:156
GUARD: EN_KEYS_STORE

```
void* ca_keys_store_allocate(u32 size)
```

Allocates a memory slot within the secure key storage area for storing cryptographic keys.

Parametri:

Nome	Tipo	Descrizione
size	u32	The requested size in bytes. If the value is less than MIN_KEY_SIZE, the function automatically rounds it up to the minimum allowed size.

Ritorna:
Returns a void pointer to the allocated memory area on success. Returns NULL if the key store is not initialized, if no free slots are available in the internal table, or if the total requested size exceeds the defined MAX_TOTAL_KEY_STORAGE limit.

Note:

- This function is not thread-safe and should be used with caution in multi-threaded environments; ensure that access to the key store is synchronized if multiple tasks attempt to allocate keys simultaneously. It is strictly non-blocking and must NOT be called from an Interrupt Service Routine (ISR) as it performs internal management operations. A mandatory precondition is that ca_keys_store_init() must have been called successfully beforehand. A common beginner mistake is failing to check the returned pointer for NULL, which will lead to a hard fault or memory corruption when attempting to write data to the returned address.

Uso:

- This function is typically used during the system initialization phase or when setting up a new secure communication session. It is part of a lifecycle that begins with ca_keys_store_init(), followed by allocation via ca_keys_store_allocate(), then enabling access permissions using ca_keys_store_temp_enable_access(), and finally securing the key with functions like ca_keys_store_lock_completely(). It is essential to allocate keys before any cryptographic operations (like AES or RSA) require them.

Esempio:

```
void* s_key_ptr = NULL;

/* Initialize the key store before allocation */
if (ca_keys_store_init(4) == 0) {
    /* Allocate space for a 32-byte AES key */
    s_key_ptr = ca_keys_store_allocate(32);

    if (s_key_ptr != NULL) {
        /* Enable write access to populate the key */
        ca_keys_store_temp_enable_access(s_key_ptr, CA_KEYS_STORE_READ_WRITE);

        /* Copy key data into the allocated slot */
        ca_keys_store_copy(s_key_ptr, my_secret_key, 32);

        /* Lock the key to prevent unauthorized modification */
        ca_keys_store_lock_completely(s_key_ptr);
    } else {
        /* Handle allocation failure: log error or enter safe state */
```

```
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_keys_store_allocate and why is it needed in a cryptographic application?
A: You should call ca_keys_store_allocate when you need to allocate memory for storing cryptographic keys in a protected area of memory. This function is essential in cryptographic applications where keys must be securely stored and accessed only through specific key-store functions. It ensures that keys are placed in a secure region that is not directly accessible by general-purpose code.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_keys_store_allocate with caOS types?
A: void* key_ptr = ca_keys_store_allocate(256); if (key_ptr == NULL) { /* Handle allocation failure */ } else { /* Use key_ptr to store a 256-bit key */ }

Parameters conf:0.90 **Q:** What does the 'size' parameter in ca_keys_store_allocate represent, and what are valid values?
A: The 'size' parameter represents the size in bytes of the cryptographic key to be allocated. Valid values are any positive number up to MAX_TOTAL_KEY_STORAGE. If the size is less than MIN_KEY_SIZE, it is automatically forced to that minimum value. Values exceeding MAX_TOTAL_KEY_STORAGE will result in allocation failure.

Return conf:0.95 **Q:** What do the return values of ca_keys_store_allocate indicate, and what causes it to return NULL?
A: The function returns a valid pointer to allocated memory if successful. It returns NULL in case of error, which can happen if the key-store is not initialized, there is insufficient space, or no free slots are available. The caller must always check the return value before using the pointer.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_keys_store_allocate?
A: Common mistakes include not checking the return value for NULL, attempting to allocate more keys than allowed, or allocating keys that are too large for available storage. Additionally, calling this function without initializing the key-store first will result in a NULL return. Using it in ISR context without proper synchronization may also lead to issues.

Behavior conf:0.90 **Q:** Is ca_keys_store_allocate blocking or non-blocking, and is it safe to call from an ISR or in a multi-threaded environment?
A: ca_keys_store_allocate is a non-blocking function that returns immediately. It is not safe to call from an ISR due to potential synchronization issues. It is also not thread-safe, so if used in a multi-threaded environment, exclusive access must be ensured using mechanisms like mutexes.

`ca_keys_store_copy`

Header: `caos/crypto/inc/keys_store.h:118`
Source: `caos/crypto/src/keys_store.c:212:232`
GUARD: `EN_KEYS_STORE`

```
int ca_keys_store_copy(void* dest, const void* src, u32 size)
```

Copies data into a secure key storage area while validating memory bounds and registration.

Parametri:

Nome	Tipo	Descrizione
`dest`	`void*`	Pointer to the destination secure key memory area. Must be a previously registered key address.
`src`	`const void*`	Pointer to the source buffer containing the key data to be copied.
`size`	`u32`	Number of bytes to copy. Must not exceed the allocated size of the destination key slot.

Ritorna:
Returns 0 on success. Returns -1 if the system is uninitialized, pointers are NULL, size is 0, the destination is not a registered secure key, or the copy operation would overflow the allocated key buffer.

Note:

- This function is non-blocking and thread-safe in the sense that it performs a direct memory copy, but it does not manage mutexes itself; you must ensure the key is accessible before calling. It is not intended to be called from an ISR as it performs validation logic that assumes a stable system state. A common beginner mistake is attempting to copy data into a key slot without first calling ca_keys_store_temp_enable_access, which will cause the write to fail or be blocked by hardware protection. Always ensure the destination pointer is the exact base address returned by the allocation function.

Uso:

- This function is used to populate secure memory slots with sensitive data like AES keys or cryptographic secrets. It is typically used in a sequence: first, you allocate or identify a key slot, then you call ca_keys_store_temp_enable_access to grant write permissions, perform the copy with ca_keys_store_copy, and finally call ca_keys_store_lock_completely to seal the memory against further modification. This workflow ensures that keys are only writable during the brief window when they are being provisioned.

Esempio:

```
void* my_key_ptr = ca_keys_store_allocate(32); // Allocate 32 bytes for a key
if (my_key_ptr != NULL) {
    u8 new_key[32] = { /* ... key data ... */ };
    // Enable write access before copying
    ca_keys_store_temp_enable_access(my_key_ptr, CA_KEYS_STORE_READ_WRITE);
    // Perform the secure copy
    if (ca_keys_store_copy(my_key_ptr, new_key, 32) == 0) {
        // Lock the key to prevent unauthorized changes
        ca_keys_store_lock_completely(my_key_ptr);
    } else {
        // Handle error: copy failed
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_keys_store_copy to copy data into a protected memory area?
A: You should use ca_keys_store_copy when you need to securely copy data into a memory region that is registered as a protected key. This function ensures that the destination is a valid registered key and that the copy operation respects the size limitations of the key. It is typically used in cryptographic services where secure data handling is required.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_keys_store_copy with caOS types?

A: u8 src_data[16] = {0x01, 0x02, 0x03, 0x04}; u8* dest_key = (u8*)0x20001000; /* Assume this is a registered key */ int result = ca_keys_store_copy(dest_key, src_data, 16); if (result == 0) { /* Copy successful */ }

Parameters conf:0.90 **Q:** What are the valid values for the 'dest' and 'src' parameters in ca_keys_store_copy?
A: The 'dest' parameter must be a non-NULL pointer to a registered key in the protected memory area. The 'src' parameter must also be a non-NULL pointer to the source data to be copied. Both pointers must be valid and accessible within the system's memory space.

Return conf:0.95 **Q:** What does a return value of -1 from ca_keys_store_copy indicate?
A: A return value of -1 indicates an error occurred during the copy operation. This can be due to invalid parameters such as NULL pointers, zero size, or if the destination is not a registered key. It may also occur if the size exceeds the allowed limit for the key.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_keys_store_copy?
A: Common mistakes include passing a NULL pointer for either 'dest' or 'src', using a memory address that is not registered as a key, or attempting to copy more bytes than the key's capacity allows. Another frequent error is calling the function before the system is initialized.

Behavior conf:0.85 **Q:** Is ca_keys_store_copy blocking or non-blocking, and can it be called from an ISR?
A: ca_keys_store_copy is a non-blocking function, but it may take some time to complete the copy. It is not recommended to call it from an ISR because it might involve memory accesses or non-ISRsafe functions, which could lead to unpredictable behavior or system instability.

`ca_keys_store_free`

Header: `caos/crypto/inc/keys_store.h:94`
Source: `caos/crypto/src/keys_store.c:158:181`
GUARD: `EN_KEYS_STORE`

```
1 int ca_keys_store_free(void* ptr)
```

Releases a previously allocated memory block in the secure key store and wipes its contents for security.

Parametri:

Nome	Tipo	Descrizione
`ptr`	`void*`	Pointer to the memory block previously returned by the key allocation function. Must not be NULL and must point to a valid, currently allocated key entry.

Ritorna:
Returns 0 on success. Returns -1 if the system is not initialized, the pointer is NULL, or the pointer does not correspond to any active key entry in the store.

Note:

- This function is not thread-safe; if multiple tasks access the key store, you must wrap calls in a mutex or semaphore to prevent race conditions. It is strictly non-blocking and performs a security wipe (ca_memset to 0) before marking the slot as free, which is critical to prevent sensitive key material from leaking into other parts of the application. Do not call this from an ISR, as the internal search logic might be too slow or rely on non-reentrant structures. A common mistake is passing a pointer that has already been

freed or a pointer that was not allocated by the key store, which will result in a -1 return and potential memory management inconsistencies.

Uso:

- This function is used during the teardown phase of a cryptographic operation or when a key's lifecycle has expired. It is typically paired with an allocation function (like ca_keys_store_alloc) to manage the limited secure memory pool. In a typical sequence, you would initialize the store, allocate a key, perform your crypto operations, and finally call ca_keys_store_free to release the slot. It is also used in bulk cleanup routines, such as system shutdown or error recovery, where all active keys must be wiped to ensure no sensitive data persists in RAM.

Esempio:

```
void secure_cleanup_task(void) {
    void* my_key = get_active_key_pointer();
    if (my_key != NULL) {
        // Ensure the key is released and wiped from memory
        if (ca_keys_store_free(my_key) == 0) {
            // Successfully wiped and freed
            my_key = NULL;
        } else {
            // Handle error: key pointer was invalid or already freed
        }
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_keys_store_free() in my embedded application?
A: You should call ca_keys_store_free() when you are done using a key that was previously allocated with ca_keys_store_alloc(). This function ensures the key is removed from the internal key table and its data is securely wiped from memory to prevent leakage.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_keys_store_free() with caOS types?
A: void* key_ptr = ca_keys_store_alloc(32); // Allocate a 32-byte key int result = ca_keys_store_free(key_ptr); // Free the key if (result != 0) { // Handle error }

Parameters conf:0.95 **Q:** What is the significance of the ptr parameter in ca_keys_store_free()?
A: The ptr parameter is a void pointer to the memory area of the key to be freed. It must point to a valid key previously allocated using ca_keys_store_alloc(). Passing NULL will cause the function to return an error.

Return conf:0.95 **Q:** What do the return values of ca_keys_store_free() indicate?
A: A return value of 0 indicates success, meaning the key was successfully freed and removed from the key table. A return value of -1 indicates an error, which occurs if the ptr is NULL or if the key was not found in the key table.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_keys_store_free()?
A: Common mistakes include calling ca_keys_store_free() with a NULL pointer, or passing a pointer that was not allocated with ca_keys_store_alloc(). Also, attempting to free the same key twice will result in an error.

Behavior conf:0.90 **Q:** Is ca_keys_store_free() safe to call from an ISR or in a multi-threaded environment?
A: No, ca_keys_store_free() is not ISR-safe and not thread-safe. It should not be called from an Interrupt Service Routine due to potential modifications of shared data structures and data wiping operations. Additionally, if used in a multi-threaded context, exclusive access to the key table must be ensured.

ca_keys_store_get_access_mode

Header: caos/crypto/inc/keys_store.h:109
Source: caos/crypto/src/keys_store.c:199:210
GUARD: EN_KEYS_STORE

```
ca_keys_store_access_mode_t ca_keys_store_get_access_mode(void* ptr)
```

Retrieves the current access permission level (read/write/execute) for a specific memory-stored key.

Parametri:

Nome	Tipo	Descrizione
ptr	void*	Pointer to the memory address where the key is stored. Must be a valid address previously registered in the keys store; passing NULL or an unregistered address will result in an error.

Ritorna:
Returns a value of type ca_keys_store_access_mode_t representing the current mode (e.g., READ_ONLY, READ_WRITE). Returns -1 if the store is not initialized, the pointer is NULL, or the key cannot be found in the internal registry.

Note:

- This function is non-blocking and thread-safe, as it performs a read-only lookup in the internal key registry. It is safe to call from an ISR, provided the keys store has been initialized. The most common mistake is passing a pointer that has already been freed or was never allocated via the keys store, which will cause the function to return -1. Always verify the return value before using the mode in logic operations to prevent unexpected behavior.

Uso:

- This function is typically used to verify the security state of a key before performing sensitive operations like copying or updating. It is often paired with ca_keys_store_set_access_mode to check current permissions before modifying them, or used during audit routines to ensure that keys are not left in an overly permissive state (e.g., READ_WRITE when only READ is required). It is part of the lifecycle: ca_keys_store_init -> ca_keys_store_allocate -> ca_keys_store_get_access_mode -> ca_keys_store_free.

Esempio:

```
/* Example: Checking key access before a sensitive operation */
void secure_task(void* key_ptr) {
    ca_keys_store_access_mode_t mode = ca_keys_store_get_access_mode(key_ptr);

    if (mode == (ca_keys_store_access_mode_t)-1) {
        /* Handle error: key not found or store not initialized */
        return;
    }

    if (mode == READ_ONLY) {
        /* Perform read-only operation */
    } else {
        /* Log warning or restrict access */
    }
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_keys_store_get_access_mode in my application?
A: You should use this function to verify the security state of a key before performing sensitive operations like copying or updating. It is also useful during audit routines to ensure keys are not left in an overly permissive state, such as READ_WRITE when only READ is required.

Code example conf:0.90 **Q:** How do I implement a check for a key's access mode using caOS types?
A: void check_key(void* key_ptr) { ca_keys_store_access_mode_t mode = ca_keys_store_get_acces if (mode != -1) { /* Proceed with logic based on mode */ } }

Parameters conf:1.00 **Q:** What are the requirements for the void* ptr parameter?
A: The pointer must be a valid memory address that was previously registered in the keys store. Passing NULL or an address that was never allocated via the keys store will result in an error.

Return conf:1.00 **Q:** What does the return value of ca_keys_store_get_access_mode indicate?
A: It returns a ca_keys_store_access_mode_t value representing the current permission level (e.g., READ_ONLY, READ_WRITE). It returns -1 if the store is not initialized, the pointer is NULL, or the key cannot be found in the internal registry.

Error/Edge case conf:1.00 **Q:** What is the most common mistake when calling this function and how does it affect the system?
A: The most common mistake is passing a pointer that has already been freed or was never allocated via the keys store. This causes the function to return -1. You should always verify the return value before using the mode in logic operations to prevent unexpected behavior.

`ca_keys_store_get_keys_info`

Header: `caos/crypto/inc/keys_store.h:151`
Source: `caos/crypto/src/keys_store.c:246:263`
GUARD: `EN_KEYS_STORE`

```
1 int ca_keys_store_get_keys_info(ca_secure_key_entry_t* keys, u8* count)
```

Retrieves a list of currently allocated secure key entries from the internal keystore.

Parametri:

Nome	Tipo	Descrizione
`keys`	`ca_secure_key_entry_t*`	Pointer to an array where the function will copy the key metadata. Must be large enough to hold the number of keys requested.
`count`	`u8*`	Pointer to a byte representing the maximum number of keys to retrieve. Upon return, this value is updated to the actual number of keys copied.

Ritorna:
Returns 0 on success. Returns -1 if the keystore is not initialized, or if either the keys or count pointers are NULL.

Note:

- This function is non-blocking and executes synchronously, meaning it will return immediately after iterating through the internal key table. It is not thread-safe by default; if multiple tasks access the keystore simultaneously, you must ensure external synchronization (like a mutex) is used to prevent data corruption. It is safe to call from an ISR only if

the underlying keystore data structures are not being modified by other tasks at the same time. A common mistake is providing a 'count' value that exceeds the size of the provided 'keys' buffer, which will lead to a buffer overflow and memory corruption.

Uso:

- This function is typically used as a discovery mechanism to map memory addresses or key IDs to their respective metadata. It is frequently paired with functions like 'ca_keys_store_set_expiry' or used to validate key existence before performing cryptographic operations. Developers use it to iterate through all active keys to find a specific entry based on a memory pointer or to display the current status of the secure storage to a CLI or diagnostic interface.

Esempio:

```
ca_secure_key_entry_t key_list[MAX_SECURE_KEYS];
u8 num_keys = MAX_SECURE_KEYS;

// Retrieve all active keys from the store
if (ca_keys_store_get_keys_info(key_list, &num_keys) == 0) {
    for (u8 i = 0; i < num_keys; i++) {
        // Process each key entry, e.g., check if it matches a specific address
        if (key_list[i].base_address == target_ptr) {
            // Perform operation on the found key
            break;
        }
    }
} else {
    // Handle error: keystore not initialized or invalid pointers
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_keys_store_get_keys_info() and what is its main purpose?
A: You should use ca_keys_store_get_keys_info() to retrieve information about allocated keys in memory, which is useful for debugging or to obtain the key_id of a key given its address. It allows you to inspect the keys stored in the secure key store module.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_keys_store_get_keys_info() with caOS types?
A: ca_secure_key_entry_t keys[10]; u8 count = 10; int result = ca_keys_store_get_keys_info(keys, &count); if (result == 0) { // Process the retrieved key info for (u8 i = 0; i < count; i++) { // Use keys[i].key_id } }

Parameters conf:0.95 **Q:** What do the parameters 'keys' and 'count' represent in ca_keys_store_get_keys_info
A: The 'keys' parameter is a pointer to an array where the information of allocated keys will be copied. The 'count' parameter is a pointer to a byte that initially holds the maximum number of keys to copy and is updated to the actual number of keys copied.

Return conf:0.95 **Q:** What does a return value of -1 from ca_keys_store_get_keys_info() indicate?
A: A return value of -1 indicates an error, specifically that one or more parameters are NULL or the module is not initialized.

Error/Edge case conf:0.90 **Q:** What are common mistakes or edge cases when using ca_keys_store_get_keys_
A: Common mistakes include passing a NULL pointer for either 'keys' or 'count', not initializing the 'count' variable before calling the function, or providing an array 'keys' that is too small to hold all the keys. Not checking the return value can also lead to using invalid data.

Behavior conf:0.85 **Q:** Is ca_keys_store_get_keys_info() blocking, ISR-safe, or thread-safe?
A: The function is not blocking, but it may access shared structures and is not thread-safe. It is not recommended to call it from an ISR due to potential access to shared data

structures and compatibility issues with ISR restrictions.

---

ca_keys_store_init

Header: caos/crypto/inc/keys_store.h:78
Source: caos/crypto/src/keys_store.c:68:113
GUARD: EN_KEYS_STORE

```
int ca_keys_store_init(u8 requested_max_keys)
```

Initializes the secure key storage system and configures MPU memory protection.

Parametri:

Nome	Tipo	Descrizione
requested_max_keys	u8	The maximum number of keys to support. Must be greater than 0 and less than or equal to the system-defined MAX_SECURE_KEYS.

Ritorna:
Returns 0 on success or if already initialized. Returns -1 if the requested count exceeds hardware limits or if MPU region allocation fails.

Note:

- This function is not thread-safe and should be called once during system startup before any other key storage operations. It is not intended to be called from an ISR. It performs memory initialization and sets up an MPU region to protect the storage area; if the MPU configuration fails, the system will not be able to securely store keys. A common mistake is calling this function multiple times with different values or failing to check the return value, which leads to silent failures in subsequent key allocation attempts.

Uso:

- This function is the mandatory entry point for the key storage module. It is typically called during the system initialization phase, alongside other hardware-specific initializers like ca_aes_init or ca_rng_init. Once initialized, the application can proceed to use ca_keys_store_allocate to reserve space for keys and ca_keys_store_copy to populate them. It is designed to be idempotent, meaning subsequent calls after the first successful initialization will simply return 0 without re-initializing the hardware.

Esempio:

```
/* Initialize the secure key storage with a capacity of 4 keys */
if (ca_keys_store_init(4) != 0) {
    /* Handle initialization failure: log error or enter safe state */
    ca_printf("Failed to initialize key store!\n");
} else {
    /* Allocate space for a 32-byte key */
    void *key_ptr = ca_keys_store_allocate(32);
    if (key_ptr != NULL) {
        /* Enable access to the allocated memory region (1 =
   CA_KEYS_STORE_READ_WRITE) */
        ca_keys_store_temp_enable_access(key_ptr, 1);
        /* Copy key data into the secure storage */
        ca_keys_store_copy(key_ptr, my_key_buffer, 32);
        /* Lock the key to prevent further modification */
        ca_keys_store_lock_completely(key_ptr);
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_keys_store_init() in my caOS application?
A: You should call ca_keys_store_init() during the system initialization phase, before any other functions from the keys store module. It must be called only once, and it sets up the internal structures and MPU configuration for managing secure keys.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_keys_store_init() with caOS types?
A: u8 result = ca_keys_store_init(16); if (result != 0) { /* handle error */ }

Parameters conf:0.90 **Q:** What does the max_keys parameter in ca_keys_store_init() control, and what are valid values?
A: The max_keys parameter defines the maximum number of secure keys the system will manage. Valid values are from 0 to MAX_SECURE_KEYS (e.g., 32). Values exceeding MAX_SECURE_KEYS will cause the function to return -1.

Return conf:0.95 **Q:** What do the return values of ca_keys_store_init() indicate?
A: A return value of 0 indicates successful initialization. A return value of -1 indicates either an invalid max_keys parameter (too large) or a failure in MPU initialization.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_keys_store_init()?
A: Common mistakes include calling the function more than once (which is ignored), using a max_keys value greater than MAX_SECURE_KEYS, or calling it from an ISR. Also, calling it after other keys store functions may lead to undefined behavior.

Behavior conf:0.90 **Q:** Is ca_keys_store_init() blocking or non-blocking, and is it safe to call from an ISR or in a multi-threaded context?
A: ca_keys_store_init() is not blocking but may take time to initialize structures and configure the MPU. It is not thread-safe and must not be called from an ISR. It should be called once during system initialization before any concurrent access.

`ca_keys_store_is_key_expired`

Header: `caos/crypto/inc/keys_store.h:170`
Source: `caos/crypto/src/keys_store.c:301:330`
GUARD: `EN_KEYS_STORE`

```
1 bool ca_keys_store_is_key_expired(void *ptr)
```

Checks if a cryptographic key is expired based on RTC time and automatically locks it if invalid.

Parametri:

Nome	Tipo	Descrizione
`ptr`	`void *`	Pointer to the memory address of the key entry in the store. Must be a valid pointer previously returned by a key allocation function.

Ritorna:
Returns 'true' if the key is expired, invalid, or if the RTC is unavailable/unreliable. Returns 'false' only if the key is within its valid time window or has no expiration constraints set.

Note:

- This function is not just a status check; it has side effects. If the key is found to be expired, it calls 'ca_keys_store_lock_completely' internally, which permanently disables the key. It is not thread-safe if multiple tasks access the key store simultaneously, so ensure proper mutex protection if your application is multi-threaded. It is generally safe to call from main-loop tasks, but avoid calling it from high-frequency ISRs because it performs RTC

hardware reads which can be slow. A common beginner mistake is calling this function repeatedly in a loop, which might trigger the internal locking mechanism prematurely if the RTC is not yet synchronized.

Uso:

- Use this function during the system initialization phase or before performing cryptographic operations to ensure the key is still authorized for use. It is typically paired with 'ca_keys_store_allocate' to create the key and 'ca_keys_store_set_expiry' to define the valid time range. By checking the expiration before every sensitive operation, you ensure that the system automatically revokes access to keys that have passed their 'validity_stop' date, maintaining security compliance without manual intervention.

Esempio:

```c
void check_and_use_key(void *key_ptr) {
    /* Check if the key is still valid before using it */
    if (ca_keys_store_is_key_expired(key_ptr)) {
        /* Key is expired and has been locked by the system */
        ca_stdout_printf_n("Security Alert: Key is expired and locked!");
    } else {
        /* Key is valid, proceed with crypto operations */
        ca_stdout_printf_n("Key is valid, proceeding...");
        /* perform_crypto_operation(key_ptr); */
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_keys_store_is_key_expired() in my embedded application?
A: You should call ca_keys_store_is_key_expired() whenever you need to verify if a cryptographic key is still valid or has expired. This is typically done before using a key for encryption or decryption operations to ensure security. It's especially important in systems where keys have time-based validity periods.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_keys_store_is_key_expired() with caOS types?
A: void example(void) { void *key_ptr = ca_keys_store_allocate(32); ca_keys_store_set_expiry(key_ 1000); bool is_expired = ca_keys_store_is_key_expired(key_ptr); if (is_expired) { // Handle expired key } }

Parameters conf:0.90 **Q:** What does the 'ptr' parameter in ca_keys_store_is_key_expired() represent and what are valid values?
A: The 'ptr' parameter is a void pointer to a cryptographic key that has been registered in the system. It must point to a valid key allocated with 'ca_keys_store_allocate()' or set with 'ca_keys_store_set_expiry()'. If NULL or an unregistered pointer is passed, the function treats it as expired.

Return conf:0.95 **Q:** What do the return values of ca_keys_store_is_key_expired() mean and what causes them?
A: The function returns true if the key is expired, not found, or if the RTC system is disabled or invalid. In such cases, it may also lock the key by calling 'ca_keys_store_lock_completely(ptr)'. It returns false only when the key is valid and not expired, provided the RTC is enabled and functional.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_keys_store_is_key_ex
A: Common mistakes include passing a NULL pointer or an unregistered key pointer, which will result in the function returning true and potentially locking the key. Another error is not handling the return value properly, especially when true is returned, as it may indicate a locked key and a blocking operation.

Behavior conf:0.90 **Q:** Is ca_keys_store_is_key_expired() blocking, ISR-safe, or thread-safe?
A: The function is potentially blocking because it may call 'ca_keys_store_lock_completely(ptr)' if the key is expired. It is not ISR-safe as it accesses shared structures and may block. It is also not thread-safe, as concurrent access from multiple threads could lead to race conditions.

---

ca_keys_store_lock_completely

Header: `caos/crypto/inc/keys_store.h:126`
Source: `caos/crypto/src/keys_store.c:234:236`
GUARD: `EN_KEYS_STORE`

```
int ca_keys_store_lock_completely(void* ptr)
```

Permanently disables all access to a secure key storage handle to prevent unauthorized reading or modification.

Parametri:

Nome	Tipo	Descrizione
`ptr`	`void*`	A pointer to the initialized key store handle. Must be a valid, non-NULL pointer previously returned by a key store allocation function.

Ritorna:
Returns 0 on success. Returns a non-zero error code if the handle is invalid or if the internal state machine fails to transition to the locked state.

Note:

- This function is a wrapper around the access control system that sets the storage mode to CA_KEYS_STORE_NO_ACCESS. It is not thread-safe if multiple tasks attempt to modify the same handle simultaneously, so ensure you have exclusive access before calling. It is generally non-blocking, but it relies on the underlying hardware or software protection mechanism to enforce the lock. A common beginner mistake is to call this function while a cryptographic operation is still pending or before the data has been fully written, which will result in a locked, empty, or corrupted key store that cannot be recovered without re-initialization.

Uso:

- This function is used as the final step in a 'secure transaction' sequence to ensure that sensitive key material is not left exposed in memory. The typical workflow is: first, call ca_keys_store_temp_enable_access to open a window for writing or reading; second, perform the necessary data transfer (e.g., ca_keys_store_copy); and finally, call ca_keys_store_lock_completely to seal the storage. It is essential to call this immediately after the operation is finished to minimize the time the key is vulnerable to memory dumps or unauthorized access.

Esempio:

```
/* Example: Securely updating a key in the store */
void* my_key_handle = get_my_key_handle();

// 1. Open access for writing
if (ca_keys_store_temp_enable_access(my_key_handle, CA_KEYS_STORE_READ_WRITE) ==
    0) {
    // 2. Perform the copy operation
    ca_keys_store_copy(my_key_handle, new_key_data, 32);
```

```
8
9       // 3. Lock the store immediately to protect the data
10      if (ca_keys_store_lock_completely(my_key_handle) != 0) {
11          // Handle error: locking failed, key might still be exposed
12          log_error("Failed to lock key store!");
13      }
14  } else {
15      log_error("Failed to enable access to key store");
16  }
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_keys_store_lock_completely() and what is its purpose in the context of secure key storage?
A: You should call ca_keys_store_lock_completely() when you want to completely disable access to a key storage area, preventing any read, write, or other access. This function is typically used to secure sensitive data after it has been processed or when transitioning to a state where the key should no longer be accessible. It is part of the key management system in caOS and is used in conjunction with functions like ca_keys_store_temp_enable_access().

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_keys_store_lock_completely(with caOS types?
A: void example_usage() { void* key_ptr = ca_keys_store_allocate(32); if (key_ptr != NULL) { int result = ca_keys_store_lock_completely(key_ptr); if (result == 0) { // Lock successful } } }

Parameters conf:0.95 **Q:** What does the ptr parameter in ca_keys_store_lock_completely() represent, and what are valid and invalid values for it?
A: The ptr parameter is a pointer to a key storage area that was previously allocated using ca_keys_store_allocate(). A valid value is a non-NULL pointer returned by that function. An invalid value includes NULL or any pointer that was not obtained from ca_keys_store_allocate(), which will cause the function to return a non-zero error code.

Return conf:0.90 **Q:** What do the return values of ca_keys_store_lock_completely() indicate, and what causes the function to fail?
A: The function returns 0 on success, indicating that the key storage area was successfully locked. A non-zero return value indicates an error, typically caused by passing an invalid pointer (such as NULL or a pointer not returned by ca_keys_store_allocate()) or attempting to lock an already freed or inaccessible memory region.

`ca_keys_store_lock_mpu_config`

Header: `caos/crypto/inc/keys_store.h:178`
GUARD: `EN_KEYS_STORE`

```
1  int ca_keys_store_lock_mpu_config(void)
```

Locks the MPU (Memory Protection Unit) configuration for the keys storage memory region to prevent unauthorized access.

Ritorna:
Returns 0 on success. Returns a negative error code if the MPU configuration fails or if the memory region is already locked or invalid.

Note:

- This function is non-blocking and executes quickly, but it is not thread-safe if called simultaneously from multiple tasks. It is strictly forbidden to call this from an ISR (Interrupt

Service Routine) because MPU configuration changes require privileged mode access that may conflict with the current execution context. Before calling this, you must ensure the MPU region for the keys storage has been properly initialized and defined. A common beginner mistake is calling this function before the system memory map is fully established, which can trigger a HardFault exception because the CPU attempts to lock a non-existent or misaligned memory region.

Uso:

- This function is typically used during the system boot sequence or during a secure initialization phase to harden the device against memory corruption or malicious code injection. It is usually paired with a preceding configuration function that defines the memory boundaries of the keys storage area. Once locked, the memory region becomes read-only or inaccessible depending on the MPU settings, ensuring that sensitive cryptographic keys cannot be overwritten by buggy application code. It is often called once at startup and remains active for the entire duration of the application's lifecycle.

Esempio:

```c
/* Initialize the keys storage system first */
if (ca_keys_store_init() == 0) {
    /* Lock the MPU configuration to protect the keys from accidental overwrite
    */
    int status = ca_keys_store_lock_mpu_config();
    if (status != 0) {
        /* Handle error: MPU lock failed, system might be insecure */
        while(1);
    }
} else {
    /* Handle initialization failure */
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_keys_store_lock_mpu_config() and why is it important for cryptographic key storage?
A: You should call ca_keys_store_lock_mpu_config() before initiating the storage operation of cryptographic keys to prevent accidental modifications to the MPU configuration during the critical phase of key saving. It is important because it ensures that the MPU settings remain stable and secure, preventing potential security vulnerabilities or system instability caused by concurrent access or configuration changes during key storage.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using caOS types that demonstrates how to use ca_keys_store_lock_mpu_config()?
A: int result = ca_keys_store_lock_mpu_config(); if (result == 0) { // MPU configuration locked successfully // Proceed with key storage operations } else { // Handle error // result contains negative error code }

Return conf:0.90 **Q:** What do the return values of ca_keys_store_lock_mpu_config() indicate, and what are the common causes of failure?
A: The function returns 0 on success, indicating the MPU configuration was successfully locked. A negative return value indicates an error, typically caused by timeout, MPU not initialized, or resource unavailability. The most common cause of failure is calling the function without first initializing the MPU, which is a prerequisite for this operation.

`ca_keys_store_set_access_mode`

Header: `caos/crypto/inc/keys_store.h:102`
Source: `caos/crypto/src/keys_store.c:183:197`

GUARD: `EN_KEYS_STORE`

```
int ca_keys_store_set_access_mode(void* ptr, ca_keys_store_access_mode_t
    mode)
```

Updates the hardware memory protection (MPU) settings for a specific cryptographic key stored in the system.

Parametri:

Nome	Tipo	Descrizione
`ptr`	`void*`	A pointer to the memory address of the key entry. Must be a valid address previously registered in the key store; passing NULL or an unregistered address will cause the function to fail.
`mode`	`ca_keys_store_access_mode_t`	The desired access level (e.g., Read-Only, Read-Write, or No-Access). Must be a valid enum value defined in the keys_store header.

Ritorna:
Returns 0 on success. Returns -1 if the system is not initialized, the pointer is invalid, the key is not found, or the MPU update fails.

Note:

- This function is non-blocking but performs a critical hardware operation by updating the Memory Protection Unit (MPU) registers. It is not thread-safe by default, so ensure that no other task is modifying the key store simultaneously to avoid race conditions. It is generally not safe to call this from an ISR because MPU configuration changes can be complex and might interfere with the current execution context. The most common mistake is passing a pointer that has not been registered yet or failing to check the return value, which leaves the system in an inconsistent security state if the MPU update fails.

Uso:

- This function is used to dynamically change the security permissions of a key during runtime, typically when transitioning between different cryptographic operations. It is usually called after a key has been successfully registered or loaded into the store. It is often paired with key retrieval functions to ensure that the application only has the necessary permissions (e.g., switching to Read-Only after a key is loaded to prevent accidental corruption). Developers should use this to enforce the principle of least privilege by restricting access to sensitive keys when they are not actively being used for encryption or decryption.

Esempio:

```
/* Example: Restricting access to a key after use */
void* my_key_ptr = get_key_address_by_id(123);

/* Set key to Read-Only mode to prevent accidental modification */
if (ca_keys_store_set_access_mode(my_key_ptr, CA_KEYS_STORE_READ_ONLY) != 0) {
    /* Handle error: MPU update failed, key might be in an insecure state */
    log_error("Failed to lock key access");
}

/* Perform crypto operations... */

/* Reset to No-Access when finished */
ca_keys_store_set_access_mode(my_key_ptr, CA_KEYS_STORE_NO_ACCESS);
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_keys_store_set_access_mode to modify access permissions for a cryptographic key?
A: You should call ca_keys_store_set_access_mode when you need to control access to a cryptographic key stored in protected memory. This is useful for enforcing security policies, such as restricting a key to read-only access after initial setup, or disabling access entirely if the key is no longer needed.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_keys_store_set_access_mode with caOS types?
A: u8 key_data[32]; keys_store_access_mode_t mode = KEY_ACCESS_READ_ONLY; int result = ca_keys_store_set_access_mode((void*)key_data, mode); if (result != 0) { // Handle error }

Parameters conf:0.95 **Q:** What does the 'ptr' parameter represent, and what are valid and invalid values for it?
A: The 'ptr' parameter is a pointer to the memory area where a cryptographic key is stored. It must point to a valid and already registered key. If NULL or a pointer to unregistered memory is passed, the function will fail with a return value of -1.

Return conf:0.95 **Q:** What do the return values 0 and -1 mean in ca_keys_store_set_access_mode?
A: A return value of 0 indicates that the access mode for the key was successfully updated. A return value of -1 indicates a failure, which can be caused by an invalid pointer, a key not found in the store, or if the module is not initialized.

Error/Edge case conf:0.90 **Q:** What are common mistakes or edge cases when calling ca_keys_store_set_acces
A: Common mistakes include calling the function before initializing the module, passing a NULL pointer or a pointer to unregistered memory, or attempting to set an invalid access mode. Additionally, calling this function from an ISR is discouraged due to potential non-immediate operations or shared resource access.

`ca_keys_store_set_expiry`

Header: `caos/crypto/inc/keys_store.h:160`
Source: `caos/crypto/src/keys_store.c:293:299`
GUARD: `EN_KEYS_STORE`

```
1 int ca_keys_store_set_expiry(u8 key_idx, u32 start, u32 stop)
```

Sets the validity time window (start and stop dates) for a specific cryptographic key in the secure storage.

Parametri:

Nome	Tipo	Descrizione
`key_idx`	`u8`	The index of the key in the internal storage table. Must be a valid, allocated index.
`start`	`u32`	The start date/time, typically packed using KSTORE_DATE_PACK.
`stop`	`u32`	The expiration date/time, typically packed using KSTORE_DATE_PACK.

Ritorna:
Returns 0 on success. Returns -1 if the system is not initialized, the key_idx is out of bounds, or the specified key slot is not currently allocated.

Note:

- This function is non-blocking and executes immediately, making it safe to call from most contexts, though it is not explicitly documented as thread-safe; ensure no other task is

modifying the same key entry simultaneously. It is a mandatory precondition that the key storage system has been initialized and the specific key index has been previously allocated. A common beginner mistake is passing a raw integer or an unformatted timestamp instead of using the KSTORE_DATE_PACK macro, which ensures the date is in the internal format expected by the system. If you pass an invalid index, the function will silently fail by returning -1, so always check the return value to ensure your configuration was actually applied.

Uso:

- This function is typically used during the key provisioning or configuration phase after a key has been successfully allocated in the secure store. It is often paired with ca_keys_store_copy to load key data and ca_keys_store_lock_completely to finalize the key's security state. In a typical workflow, you would allocate a key, set its metadata (like expiry), and then lock it to prevent further unauthorized modifications. It is also used by service layers to update existing keys by iterating through key info structures to find the correct index before applying the new validity window.

Esempio:

```
/* Ensure the key store is initialized before use */
u8 my_key_id = 0; // Assume this index was allocated previously
u32 start_date = KSTORE_DATE_PACK(2024, 1, 1);
u32 stop_date = KSTORE_DATE_PACK(2025, 1, 1);

/* Apply the expiry window to the key */
int result = ca_keys_store_set_expiry(my_key_id, start_date, stop_date);

if (result == 0) {
    ca_stdout_printf_n("Expiry successfully set for key %u", my_key_id);
} else {
    ca_stdout_printf_n("Failed to set expiry: check if key %u is allocated",
    my_key_id);
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_keys_store_set_expiry and why is it important for cryptographic key management?
A: You should call ca_keys_store_set_expiry to define a valid period for a cryptographic key that has already been allocated. This function allows you to set a start and end date for the key's validity, which is essential for managing key lifecycle and ensuring that expired keys are not used in cryptographic operations. It helps enforce security policies by preventing the use of keys outside their intended time window.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_keys_store_set_expiry with caOS types?
A: int result = ca_keys_store_set_expiry(2, KSTORE_DATE_PACK(2025, 1, 1), KSTORE_DATE_PACK(2026, 1, 1)); if (result == 0) { // Success: key 2 is now valid from 2025-01-01 to 2026-01-01 } else { // Failure: check initialization or key allocation status }

Parameters conf:0.90 **Q:** What are the valid ranges for key_idx and what happens if I pass an invalid key index?
A: The key_idx parameter must be a valid index between 0 and max_keys - 1, where max_keys is the maximum number of keys supported by the system. If key_idx is outside this range or if the key at that index is not allocated, the function will return -1. The key must be already allocated before calling this function, otherwise it will also fail.

Return conf:0.90 **Q:** What does a return value of -1 from ca_keys_store_set_expiry indicate and what causes this failure?

A: A return value of -1 indicates that the function failed to set the expiry for the key. This can happen if the system is not initialized (initialized = false), the key index is out of bounds, the key is not allocated (is_allocated = false), or if the start date is greater than the stop date, which violates the precondition that start must be less than or equal to stop.

---

ca_keys_store_temp_disable_access

Header: caos/crypto/inc/keys_store.h:143
Source: caos/crypto/src/keys_store.c:242:244
GUARD: EN_KEYS_STORE

```
int ca_keys_store_temp_disable_access(void* ptr)
```

Temporarily locks the secure key storage to prevent any read or write operations.

Parametri:

Nome	Tipo	Descrizione
ptr	void*	A pointer to the secure key storage area handle. Must be a valid, initialized pointer; passing NULL will result in an error.

Ritorna:
Returns 0 on success. Returns a non-zero error code if the lock operation fails, typically due to an invalid pointer or an internal state conflict.

Note:

- This function is a blocking operation that enforces a complete lock on the key storage, ensuring that no other task can access the sensitive data while the lock is active. It is not intended to be called from an Interrupt Service Routine (ISR) because it may involve mutexes or OS-level synchronization primitives that are not ISR-safe. A common beginner mistake is to forget to call this function after performing a sensitive operation, leaving the key storage permanently exposed or in an inconsistent state. Always ensure the key storage has been properly initialized before attempting to lock it, as calling this on an uninitialized handle will lead to undefined behavior or system crashes.

Uso:

- This function is used to implement a 'lock-after-use' pattern to maintain security best practices. It is typically paired with ca_keys_store_temp_enable_access, which is called immediately before the sensitive operation (like reading a key) to grant temporary access. By calling ca_keys_store_temp_disable_access immediately after the operation is complete, you minimize the window of time that the keys are accessible in memory. This sequence is critical in multi-threaded environments where multiple tasks might compete for access to the secure storage area.

Esempio:

```
/* Assuming g_secure_key_area is a valid, initialized key store handle */

// 1. Enable access to perform a sensitive operation
if (ca_keys_store_temp_enable_access(g_secure_key_area, CA_KEYS_STORE_READ_ONLY)
    == 0) {
    u8 buffer[32];
    // 2. Perform the operation
    ca_memcpy(buffer, g_secure_key_area, 32);

```

```
    // 3. Immediately lock the storage again to protect the keys
    if (ca_keys_store_temp_disable_access(g_secure_key_area) != 0) {
        // Handle error: storage failed to lock, potentially unsafe state
        ca_stdout_put_n("Error: Failed to lock key storage!");
    }
} else {
    ca_stdout_put_n("Error: Could not enable access to key storage");
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_keys_store_temp_disable_access() and what is its purpose?
A: You should call ca_keys_store_temp_disable_access() when you need to temporarily disable access to a key storage structure to protect it from unauthorized access or modifications. This function is used to completely lock the key storage, making it inaccessible until re-enabled. It is typically used in security-sensitive applications where key integrity must be maintained.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_keys_store_temp_disable_ac with caOS types?
A: void* key_store_ptr = /* initialized key store structure */; int result = ca_keys_store_temp_disabl if (result == 0) { /* Access successfully disabled */ } else { /* Handle error */ }

Parameters conf:0.90 **Q:** What does the ptr parameter represent in ca_keys_store_temp_disable_access(), and what are valid and invalid values?
A: The ptr parameter is a void pointer to a key storage structure that must be valid and not NULL. A valid value is a pointer to a properly initialized key storage structure. An invalid value is NULL or a pointer to an uninitialized or corrupted structure, which will cause the function to return -1.

Return conf:0.90 **Q:** What do the return values of ca_keys_store_temp_disable_access() indicate, and what causes failures?
A: The function returns 0 on success, indicating that access to the key storage structure has been completely disabled. It returns -1 on failure, which occurs if the ptr parameter is NULL or if the structure pointed to is not valid. Failures may also occur if the structure was not properly initialized or if the function is called in an inappropriate context such as from an ISR.

`ca_keys_store_temp_enable_access`

Header: `caos/crypto/inc/keys_store.h:135`
Source: `caos/crypto/src/keys_store.c:238:240`
GUARD: `EN_KEYS_STORE`

```
int ca_keys_store_temp_enable_access(void* ptr,
    ca_keys_store_access_mode_t mode)
```

Temporarily modifies the MPU access permissions for a specific key storage memory region.

Parametri:

Nome	Tipo	Descrizione
ptr	void*	A pointer to the key storage handle (ks_handle). Must be a valid, previously initialized key store pointer.
mode	ca_keys_store_access_mode_t	The desired access level (e.g., CA_KEYS_STORE_READ_ONLY, CA_KEYS_STORE_READ_WRITE). Must be a valid enum value defined in the system.

Ritorna:
Returns 0 on success. Returns a non-zero error code if the pointer is invalid, the mode is unsupported, or the underlying MPU configuration fails.

Note:

- This function is a wrapper that interacts with the hardware MPU (Memory Protection Unit) to restrict or grant access to sensitive memory areas. It is not thread-safe by itself; if multiple tasks access the same key store, you must ensure external synchronization (like a mutex) to prevent race conditions where one task locks the memory while another is reading it. It is generally not intended to be called from an ISR, as MPU configuration changes can be time-consuming and might interfere with real-time constraints. The most common mistake is forgetting to call the corresponding 'lock' function (like ca_keys_store_lock_completely) after the operation, which leaves sensitive key material exposed in memory indefinitely.

Uso:

- This function is used in a 'sandwich' pattern: you enable access, perform the sensitive operation (like copying or reading a key), and immediately lock the memory again. It is typically paired with ca_keys_store_lock_completely or ca_keys_store_temp_disable_access to ensure the principle of least privilege is maintained. Use this whenever you need to perform a cryptographic operation or a memory copy involving secure keys that are otherwise protected by the MPU. Always check the return value; if it fails, do not attempt to access the memory, as it may trigger a HardFault due to an MPU violation.

Esempio:

```
/* Example: Safely reading a key from a protected store */
void process_secure_key(void* ks_handle) {
    u8 buffer[32];

    /* 1. Grant temporary read access */
    if (ca_keys_store_temp_enable_access(ks_handle, CA_KEYS_STORE_READ_ONLY) ==
    0) {
        /* 2. Perform the copy while access is granted */
        ca_memcpy(buffer, ks_handle, 32);

        /* 3. Immediately lock the memory to protect it again */
        ca_keys_store_lock_completely(ks_handle);
    } else {
        /* Handle error: access could not be granted */
        log_error("Failed to unlock key store");
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_keys_store_temp_enable_access() and what is its purpose in the context of secure memory access?

A: You should call ca_keys_store_temp_enable_access() when you need temporary read or read-write access to a protected memory area, such as when handling cryptographic keys or secure data. The function allows temporary access to a memory region that is otherwise locked, enabling operations like encryption or decryption before re-enabling protection.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_keys_store_temp_enable_acc with caOS types?
A: void* secure_ptr = ca_keys_store_allocate(256); // Allocate 256 bytes int result = ca_keys_store_temp_enable_access(secure_ptr, CA_KEYS_STORE_READ_WRITE); if (result == 0) { // Perform read/write operations here u8* data = (u8*)secure_ptr; data[0] = 0xAA; // Access complete, access is automatically re-disabled }

Parameters conf:0.95 **Q:** What are the valid values for the 'mode' parameter in ca_keys_store_temp_enable_a and what happens if an invalid value is passed?
A: The 'mode' parameter must be either CA_KEYS_STORE_READ_ONLY or CA_KEYS_STORE_ Passing CA_KEYS_STORE_NO_ACCESS or any other invalid value will cause the function to return a non-zero error code, indicating that the operation failed due to unsupported access mode.

Return conf:0.90 **Q:** What does a return value of 0 from ca_keys_store_temp_enable_access() indicate, and what could cause a non-zero return value?
A: A return value of 0 indicates that the function successfully enabled temporary access to the memory area. A non-zero return value indicates failure, which can occur due to an invalid pointer (NULL or not allocated via ca_keys_store_allocate), or an unsupported access mode such as CA_KEYS_STORE_NO_ACCESS.

`ca_keystore_service_init`

Header: `caos/crypto/inc/keystore_service.h:66`
Source: `caos/crypto/src/keystore_service.c:78:100`
GUARD: `EN_KEYSTORE_SERVICE`

```
1 int ca_keystore_service_init(u32 base_addr)
```

Initializes the internal keystore service by reading and validating the flash memory header at a specific address.

Parametri:

Nome	Tipo	Descrizione
`base_addr`	`u32`	The memory address in flash where the keystore header is stored. Must be a valid, aligned memory address pointing to the start of the keystore data structure.

Ritorna:
Returns 0 on success. Returns -1 if the header validation fails (e.g., invalid key count, invalid evolution counter, or insufficient size).

Note:

- This function is non-blocking and performs a direct memory copy from flash to internal RAM. It is not thread-safe, meaning it should not be called from multiple tasks simultaneously or while other tasks are accessing the keystore. It is not safe to call from an ISR because it performs memory operations that might be slow or conflict with other system services. A common beginner mistake is passing an uninitialized or incorrect flash address, which will cause the validation logic to fail and return -1. Ensure the flash memory at base_addr has been properly programmed with a valid header before calling this function.

Uso:

- This function is the mandatory first step before performing any cryptographic operations or key management tasks. It is typically called during the system boot sequence or when a new secure storage partition is mounted. It is often paired with subsequent key retrieval or storage functions. If this function fails, the application must stop attempting to use the keystore service, as the internal state will remain inactive and potentially inconsistent.

Esempio:

```c
/* Define the flash address where the keystore is located */
u32 flash_addr = 0x08020000;

/* Initialize the service and check for errors */
if (ca_keystore_service_init(flash_addr) != 0) {
    /* Handle initialization failure: log error or enter safe state */
    CLI_PUT_N("Error: Keystore initialization failed. Check flash contents.");
} else {
    /* Proceed with secure operations now that the service is active */
    CLI_PUT_N("Keystore service initialized successfully.");
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_keystore_service_init() in a caOS application?
A: You should call ca_keystore_service_init() during the system initialization phase, typically before any cryptographic operations that require keys from the keystore. It is used to read and validate the keystore header from flash memory, ensuring that the key storage is correctly formatted and accessible.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_keystore_service_init()?
A: u32 keystore_base = 0x08008000; // Example flash address int result = ca_keystore_service_init(key if (result == 0) { // Initialization successful } else { // Handle error }

Parameters conf:0.90 **Q:** What does the base_addr parameter in ca_keystore_service_init() represent and what are valid values?
A: The base_addr parameter is the flash address where the keystore header is located. Valid values are any valid flash address that is readable and points to a correctly formatted keystore header. It must not be NULL and should point to a memory-mapped flash region accessible by the microcontroller.

Return conf:0.95 **Q:** What do the return values of ca_keystore_service_init() indicate?
A: A return value of 0 indicates successful initialization. A return value of -1 indicates a validation error of the keystore header, such as zero or too many keys, or an invalid header size.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_keystore_service_init()
A: Common mistakes include passing an invalid or unmapped flash address, or not verifying that the keystore header is valid after initialization. Another issue is calling this function from an ISR, which is not safe due to potential interference with system operations.

Behavior conf:0.90 **Q:** Is ca_keystore_service_init() blocking, ISR-safe, or thread-safe?
A: ca_keystore_service_init() is not blocking, but it uses ca_memcpy which may take time depending on bus speed. It is not ISR-safe because it uses ca_memcpy and could interfere with system operations. It is thread-safe if no other functions modify shared state, as it only reads data and does not modify shared resources.

---

ca_keystore_service_is_active

Header: caos/crypto/inc/keystore_service.h:86
Source: caos/crypto/src/keystore_service.c:197:199
GUARD: EN_KEYSTORE_SERVICE

```
bool ca_keystore_service_is_active(void)
```

Checks if the keystore service is currently initialized and ready for cryptographic operations.

Ritorna:
Returns true if the keystore service is in the KSS_ACTIVE state, and false if it is inactive or uninitialized.

Note:

- This function is non-blocking and executes in constant time, making it safe to call from both standard tasks and Interrupt Service Routines (ISRs). It is thread-safe because it performs a simple read of a static variable, though it does not guarantee that the state won't change immediately after the check if other tasks are running. A common mistake is assuming that a 'true' result guarantees the keys are still valid; if another task calls ca_keystore_service_lock() immediately after your check, the service will become inactive, potentially causing subsequent cryptographic operations to fail. Always ensure the service is initialized via ca_keystore_service_init() before relying on this status.

Uso:

- This function is primarily used as a guard clause to ensure the keystore is ready before performing sensitive operations like encryption, decryption, or key derivation. It is typically paired with ca_keystore_service_init() in a 'lazy initialization' pattern, where the system checks if the service is active and initializes it only if it is not. This prevents redundant initialization cycles and ensures that the system recovers gracefully if the keystore was previously locked or wiped for security reasons.

Esempio:

```
/* Check if the keystore is ready before performing a secure operation */
if (!ca_keystore_service_is_active()) {
    /* Attempt to re-initialize the service if it is inactive */
    if (ca_keystore_service_init(FLASH_STORAGE_ADDRESS) != 0) {
        /* Handle initialization failure: log error and abort operation */
        return;
    }
}

/* Now it is safe to proceed with cryptographic tasks */
// perform_secure_operation();
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should a developer call ca_keystore_service_is_active() and why is it important?
A: A developer should call ca_keystore_service_is_active() before performing critical operations on the keystore service to verify that the service is currently active. This function is important because it helps prevent errors that could occur if operations are attempted while the service is not properly initialized or is in an inactive state. It is particularly useful in scenarios where the keystore service might be disabled or uninitialized, such as during system startup or shutdown sequences.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_keystore_service_is_active() with caOS types?
A: bool result = ca_keystore_service_is_active(); if (result) { // Proceed with keystore operations // Example: u32 status = keystore_operation(); } else { // Handle inactive service // Example: u8 error_code = 1; }

Return conf:0.90 **Q:** What do the return values of ca_keystore_service_is_active() indicate, and what causes the function to fail?
A: The function returns true if the keystore service is active (when s_state == KSS_ACTIVE), and false otherwise. The function does not have any failure modes or error conditions; it simply reads a global state variable. Failures in the service's operation would not be indicated by this function's return value but rather by errors in subsequent operations performed after checking the state.

`ca_keystore_service_lock`

Header: `caos/crypto/inc/keystore_service.h:81`
Source: `caos/crypto/src/keystore_service.c:192:195`
GUARD: `EN_KEYSTORE_SERVICE`

```
void ca_keystore_service_lock(void)
```

Securely clears all loaded cryptographic keys from memory and sets the keystore service to an inactive state.

Ritorna:
This function returns void and does not provide a status code, meaning it assumes the operation is always successful.

Note:

- This function is non-blocking and executes immediately. It is not thread-safe by itself; if multiple tasks access the keystore, you must ensure external synchronization to prevent race conditions where one task tries to use a key while another is wiping it. It is safe to call from an ISR, but be aware that wiping keys can be a time-consuming operation depending on the implementation of wipe_loaded_keys(), which might impact real-time latency. The most common mistake is calling this function while a cryptographic operation (like AES encryption) is still in progress, which will cause the hardware or software driver to crash or produce garbage data because the key material has been zeroed out.

Uso:

- This function is used as a security safeguard to ensure sensitive key material does not persist in RAM longer than necessary. It is typically called during error handling paths (e.g., if key derivation fails or an AES channel initialization fails) to prevent unauthorized access to stale keys. It is also used during system shutdown or when transitioning the device to a low-power or unauthenticated state. You should pair this with your initialization functions; whenever an error occurs after a successful 'unlock' or 'init' sequence, call this function to return the system to a clean, locked state.

Esempio:

```
/* Example: Handling a failure during secure key usage */
void process_secure_data(u8 key_idx) {
    /* Attempt to initialize the AES channel with a specific key */
    if (ca_aes_channel_init(&s_ch, CA_AES_MODE_GCM, CA_AES_KEY_256,
    s_traffic_key_ptr, NULL, NULL, NULL) != 0) {
        /* If initialization fails, we must not leave keys in memory */
```

```
        CLI_PRINTF_N("Error: AES init failed, locking keystore for security.");
        ca_keystore_service_lock();
        return;
    }
    /* Proceed with secure operations... */
}
```

Q&A — fine-tuning review *2/2 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_keystore_service_lock() and what is its purpose in the context of a cryptographic service?
A: You should call ca_keystore_service_lock() when you want to deactivate the keystore service, which involves clearing any loaded keys and setting the service to an inactive state. This function is intended for use when you wish to reset the internal state of the keystore service. It is a blocking function and must not be called from an ISR. It is typically used during system shutdown or when transitioning to a state where cryptographic operations are no longer required.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_keystore_service_lock() with caOS types?
A: void example_function(void) { // Lock the keystore service to clear loaded keys and set inactive state ca_keystore_service_lock(); }

`ca_keystore_service_unlock`

Header: `caos/crypto/inc/keystore_service.h:76`
Source: `caos/crypto/src/keystore_service.c:102:190`
GUARD: `EN_KEYSTORE_SERVICE`

```
int ca_keystore_service_unlock(const u8 *p2)
```

Decrypts and loads secure keys from flash memory into the system keystore using a provided password.

Parametri:

Nome	Tipo	Descrizione
p2	`const u8 *`	A pointer to the user-provided password or key derivation material. Must not be NULL; if NULL, the function fails and locks the service.

Ritorna:
Returns 0 on success (keys are loaded and service is active). Returns -1 on failure, which occurs if the password is incorrect, memory allocation fails, or the flash data is corrupted.

Note:

- This function is a blocking operation because it performs intensive cryptographic decryption and memory management tasks. It is NOT thread-safe and must not be called from an Interrupt Service Routine (ISR) due to its use of dynamic memory allocation and potential long execution time. Before calling this, ensure the keystore service has been initialized and the base address of the encrypted flash block is set. A common beginner mistake is failing to clear the password buffer (p2) after the call, which leaves sensitive data in RAM, or forgetting to check the return value, leading to attempts to access keys that were never loaded.

Uso:

- This function is typically used during the system boot sequence or when a user provides credentials to access protected features. It is part of a lifecycle: first, the service is initialized, then ca_keystore_service_unlock is called with the user's password to populate the internal key store. Once the application finishes using the keys, it should call ca_keystore_service_lock to clear the keys from memory and secure the service again.

Esempio:

```
/* Example: Unlocking the keystore during system startup */
u8 user_password[32];
/* Assume user_password is populated via CLI or input */

if (ca_keystore_service_unlock(user_password) == 0) {
    /* Success: Keys are now available in the keystore */
    /* Proceed to use keys via ca_keys_store functions */
} else {
    /* Failure: Handle error, perhaps by locking the system or requesting retry
    */
    ca_keystore_service_lock();
}

/* Always clear sensitive data from memory after use */
ca_memzero(user_password, sizeof(user_password));
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_keystore_service_unlock and why is it necessary?
A: You should call ca_keystore_service_unlock when you need to decrypt and load cryptographic keys from flash into the keystore using a secondary password. This function is necessary to initialize the keystore with valid keys before performing any cryptographic operations that require those keys.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_keystore_service_unlock with caOS types?
A: u8 secondary_password[32] = {0x12, 0x34, 0x56, 0x78}; int result = ca_keystore_service_unlock(sec if (result == 0) { // Keystore unlocked successfully } else { // Handle unlock failure }

Parameters conf:0.95 **Q:** What does the p2 parameter represent and what are valid values for it?
A: The p2 parameter is a pointer to a 32-byte array representing the secondary password (salt) used to derive the AES-256-GCM key for decrypting the keystore payload. It must be a valid, non-NULL pointer. Invalid values include NULL or any pointer to an invalid memory location.

Return conf:0.95 **Q:** What do the return values of ca_keystore_service_unlock indicate and what causes a failure?
A: A return value of 0 indicates success: the keys were decrypted and loaded into the keystore. A return value of -1 indicates failure, which can be caused by a NULL p2 parameter, an uninitialized keystore (s_base_addr == 0), or failures in decryption or memory allocation.

Error/Edge case conf:0.90 **Q:** What are common error conditions or mistakes when using ca_keystore_service_unlock?
A: Common errors include passing NULL as the p2 parameter, which causes immediate failure; providing an incorrect password that leads to decryption failure; or encountering memory allocation failure during key loading. Additionally, calling this function when the keystore is not initialized (s_base_addr == 0) will also result in failure.

Behavior conf:0.90 **Q:** Is ca_keystore_service_unlock blocking or non-blocking, and is it safe to call from an ISR or in a multi-threaded context?
A: The function is blocking, as it waits for decryption, allocation, and loading operations to complete before returning. It is not thread-safe and must not be called from an ISR,

as it uses allocation functions and could cause concurrency or stack overflow issues.

---

ca_led_count

Header: caos/drivers/inc/led.h:46
Source: caos/drivers/src/led.c:100:103
GUARD: EN_LED

```
u8 ca_led_count(void)
```

Returns the total number of LEDs available on the hardware platform.

Ritorna:
An unsigned 8-bit integer (u8) representing the count of LEDs. Returns 0 if no LEDs are configured or if the hardware initialization failed.

Note:

- This function is a non-blocking, thread-safe wrapper that queries the hardware abstraction layer for the LED count. It is safe to call from both tasks and ISRs, provided the hardware has been initialized. The most common mistake is assuming a fixed number of LEDs (e.g., hardcoding 1) instead of using this function, which makes your code portable across different STM32 boards with varying LED counts.

Uso:

- This function is typically used during system startup or in diagnostic banners to dynamically adapt the application's UI or CLI feedback to the specific hardware. It is often paired with ca_led_init to ensure the hardware is ready before querying. Developers use it to iterate through LED indices in loops or to validate user input when a CLI command references a specific LED index, ensuring the index is within the range [1, ca_led_count()].

Esempio:

```
void app_led_diagnostic(void) {
    // Initialize hardware before querying
    ca_led_init();

    u8 total_leds = ca_led_count();
    ca_printf_n("System has %u LEDs available.", (unsigned)total_leds);

    // Safely iterate through all available LEDs
    for (u8 i = 1; i <= total_leds; i++) {
        ca_led_on(i);
        ca_os_delay_ms(100);
        ca_led_off(i);
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** Why should I use ca_led_count() instead of just hardcoding the number of LEDs in my application?
A: You should use ca_led_count() to ensure your code remains portable across different STM32 boards that may have varying LED counts. Hardcoding values makes your application rigid, whereas this function allows your code to dynamically adapt to the specific hardware configuration.

Code example conf:1.00 **Q:** How can I write a loop to iterate through all available LEDs using caOS types?

A: You can use the following pattern: u8 count = ca_led_count(); for(u8 i = 1; i <= count; i++) { /* perform operations on LED i */ }

Return conf:1.00 **Q:** What does it mean if ca_led_count() returns 0?

A: A return value of 0 indicates that either no LEDs are configured on the hardware platform or that the hardware initialization failed. It is recommended to pair this function with ca_led_init to ensure the hardware is ready before querying.

`ca_led_get`

Header: `caos/drivers/inc/led.h:81`
Source: `caos/drivers/src/led.c:126:130`
GUARD: `EN_LED`

```
bool ca_led_get(u8 led_num)
```

Retrieves the current logical state of a specific LED.

Parametri:

Nome	Tipo	Descrizione
`led_num`	`u8`	The 1-based index of the LED to query. Must be between 1 and the value returned by ca_led_count().

Ritorna:
Returns true if the LED is currently ON, false if it is OFF, or false if the provided led_num is invalid (out of range or 0).

Note:

- This function is non-blocking and thread-safe, as it performs a direct read of the hardware state. It is safe to call from an ISR, provided the underlying hardware access is atomic. The most common beginner mistake is passing 0 or a number greater than the total LED count, which will silently return false without indicating an error; always verify your hardware configuration using ca_led_count() if the number of LEDs is dynamic.

Uso:

- This function is typically used in monitoring loops or status reporting tasks to verify the current state of a visual indicator. It is often paired with ca_led_toggle() or ca_led_on() to implement blink patterns or state-based feedback. In a typical application, you would call ca_led_init() during system startup, then use ca_led_get() to log the status to a console or a remote interface via ca_stdout_printf_n().

Esempio:

```
/* Check if LED 1 is on and log its status to the console */
#include "caos.h"

void monitor_led_task(void) {
    u8 led_id = 1U;

    /* Ensure the LED index is valid before querying */
    if (led_id <= ca_led_count()) {
        bool is_on = ca_led_get(led_id);

        /* Log the state using the system stdout */
        ca_stdout_printf_n("LED %u is currently: %s", led_id, is_on ? "ON" : "
    OFF");
    } else {
```

```
        ca_stdout_printf_n("Error: Invalid LED index");
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_led_get function in my caOS application?
A: You should use ca_led_get in monitoring loops or status reporting tasks to verify the current state of a visual indicator. It is commonly used to implement blink patterns or provide state-based feedback, often in conjunction with ca_led_toggle or ca_led_on.

Code example conf:1.00 **Q:** How do I check the status of the first LED and print it using caOS types?
A: u8 led_id = 1; if (ca_led_get(led_id)) { ca_stdout_printf_n("LED 1 is ON"); } else { ca_stdout_printf_n("LED 1 is OFF"); }

Parameters conf:1.00 **Q:** What are the constraints for the led_num parameter in ca_led_get?
A: The led_num parameter is a 1-based index. It must be a value between 1 and the total number of LEDs returned by ca_led_count(). Passing 0 or a value greater than the total LED count is invalid.

Return conf:1.00 **Q:** What does the return value of ca_led_get indicate, and why might it return false?
A: The function returns true if the LED is ON and false if it is OFF. It also returns false if the provided led_num is invalid (out of range or 0). Note that invalid inputs result in a silent failure without an error indicator.

`ca_led_init`

Header: `caos/drivers/inc/led.h:38`
Source: `caos/drivers/src/led.c:91:98`
GUARD: `EN_LED`

```
void ca_led_init(void)
```

Initializes the LED hardware abstraction layer and optionally registers the LED CLI interface.

Ritorna:
void

Note:

- This function is not thread-safe and must be called during the system initialization phase, typically before the RTOS scheduler is started. It performs low-level hardware configuration for the LEDs; calling it multiple times is generally safe due to internal static flags, but it should be treated as a one-time setup routine. It is not intended to be called from an Interrupt Service Routine (ISR) as it may involve complex peripheral initialization. The most common mistake is attempting to toggle or control LEDs before calling this function, which will result in no physical output because the GPIO pins have not been configured as outputs.

Uso:

- Use this function at the very beginning of your application main entry point, immediately after system clock configuration and before starting any timers or tasks that interact with the LEDs. It is the mandatory prerequisite for all other LED-related functions like ca_led_on, ca_led_off, or ca_led_toggle. In projects with a Command Line Interface (CLI), this function ensures the LED control commands are registered, allowing you to debug or test hardware status directly from the terminal.

Esempio:

```c
int main(void) {
    /* Initialize system hardware and OS */
    ca_os_tick_init();

    /* Initialize LED hardware and CLI support */
    ca_led_init();

    /* Ensure LED is in a known state */
    ca_led_off(1U);

    /* Start periodic task to blink LED */
    /* ca_sw_timer_create(1000, CA_TIMER_PERIODIC, cb_blink, NULL); */

    ca_os_run();
    return 0;
}
```

Q&A — fine-tuning review *2/2 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_led_init in my caOS application and why is it mandatory?
A: You must call ca_led_init during the system initialization phase, ideally immediately after system clock configuration and before starting the RTOS scheduler, timers, or tasks. It is mandatory because it performs the low-level GPIO configuration required to set pins as outputs; without this, any subsequent calls to functions like ca_led_on or ca_led_off will result in no physical output.

Code example conf:1.00 **Q:** How do I correctly initialize the LED subsystem in my main entry point?
A: You should call the function directly in your main routine before starting the scheduler. Example: int main(void) { /* System clock configuration here */ ca_led_init(); /* Start RTOS scheduler */ return 0; }

`ca_led_off`

Header: `caos/drivers/inc/led.h:62`
Source: `caos/drivers/src/led.c:112:117`
GUARD: `EN_LED`

```c
bool ca_led_off(u8 led_num)
```

Turns off a specific LED by its index number.

Parametri:

Nome	Tipo	Descrizione
`led_num`	`u8`	The 1-based index of the LED to turn off. Must be between 1 and the value returned by ca_led_count().

Ritorna:
Returns true if the LED was successfully turned off, or false if the index is invalid (0 or out of range).

Note:

- This function is non-blocking and thread-safe, assuming the underlying hardware driver is properly implemented. It is safe to call from an ISR, provided the hardware access layer does not perform blocking operations. You must call ca_led_init() before attempting to

control any LEDs, otherwise, the hardware state may be undefined. A common mistake is using 0-based indexing (e.g., passing 0 for the first LED), which will cause the function to return false immediately.

Uso:

- This function is typically used during system initialization to set a known 'off' state for all status LEDs, or during runtime to provide visual feedback (e.g., turning off an activity LED after a task completes). It is often paired with ca_led_init() at startup and used alongside ca_led_on() or ca_led_toggle() to manage the device's visual interface. In complex systems, it is used to clear status indicators when an error condition is resolved or when a process enters a standby state.

Esempio:

```
/* Initialize the LED driver before use */
ca_led_init();

/* Turn off LED 1 to ensure a clean state */
if (!ca_led_off(1U)) {
    /* Handle error: invalid LED index */
    ca_printf_n("Error: Failed to turn off LED 1");
}

/* Toggle LED 1 to indicate activity */
ca_led_toggle(1U);

/* Turn it off again after a short delay */
ca_os_delay_ms(100);
ca_led_off(1U);
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_led_off function in my caOS application?
A: You should use ca_led_off during system initialization to establish a known 'off' state for status LEDs, or during runtime to provide visual feedback, such as turning off an activity LED after a task completes or clearing status indicators when an error is resolved.

Code example conf:1.00 **Q:** How do I correctly call ca_led_off in my code?
A: You must ensure ca_led_init() is called first. Here is an example: u8 led_index = 1; if (ca_led_off(led_index)) { /* Success */ }

Parameters conf:1.00 **Q:** What are the constraints for the led_num parameter?
A: The led_num parameter is a 1-based index. It must be between 1 and the value returned by ca_led_count(). Passing 0 or any value outside this range is invalid and will cause the function to return false.

Return conf:1.00 **Q:** What do the return values of ca_led_off indicate?
A: The function returns true if the LED was successfully turned off. It returns false if the provided index is invalid, such as passing 0 or a number exceeding the total LED count.

`ca_led_on`

Header: `caos/drivers/inc/led.h:54`
Source: `caos/drivers/src/led.c:105:110`
GUARD: `EN_LED`

```
bool ca_led_on(u8 led_num)
```

Turns on a specific LED identified by its index.

Parametri:

Nome	Tipo	Descrizione
`led_num`	`u8`	The 1-based index of the LED to turn on. Must be between 1 and the value returned by ca_led_count(). Passing 0 or a value greater than the total count will result in an error.

Ritorna:
Returns true if the LED was successfully turned on, or false if the provided index is invalid (out of range or 0).

Note:

- This function is a wrapper that performs bounds checking before interacting with the hardware. It is not thread-safe by itself, so ensure that multiple tasks do not attempt to manipulate the same LED state simultaneously without external synchronization. It is generally safe to call from tasks, but avoid calling it from high-frequency ISRs if the underlying hardware implementation involves slow bus communication (like I2C or SPI). The most common beginner mistake is using a 0-based index (e.g., passing 0 for the first LED), which will always return false because the function expects a 1-based index.

Uso:

- This function is typically used after calling ca_led_init() to configure the GPIO pins. It is often paired with ca_led_off() or ca_led_toggle() to create visual feedback patterns, such as blinking status indicators or error signals. In a typical application, you would first call ca_led_count() to determine how many LEDs are available, then iterate through them or target specific ones based on system state.

Esempio:

```
/* Initialize the LED system before use */
ca_led_init();

/* Get the total number of available LEDs */
u8 total_leds = ca_led_count();

/* Turn on the first LED if available */
if (total_leds >= 1) {
    if (!ca_led_on(1)) {
        /* Handle error: failed to turn on LED 1 */
    }
}

/* Wait for a short period */
ca_os_delay_ms(500);

/* Turn off the LED */
ca_led_off(1);
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_led_on function in my caOS application?
A: You should use ca_led_on after calling ca_led_init to configure the GPIO pins. It is typically used to provide visual feedback, such as status indicators or error signals, often in conjunction with ca_led_off or ca_led_toggle.

Code example conf:1.00 **Q:** How do I turn on the first LED using caOS types?
A: u8 led_index = 1; if (ca_led_on(led_index)) { /* LED is now on */ }

Parameters conf:1.00 **Q:** What are the constraints for the led_num parameter?
A: The led_num parameter is a 1-based index. It must be between 1 and the value returned by ca_led_count(). Passing 0 or any value greater than the total count will result in an error.

Return conf:1.00 **Q:** What does the boolean return value indicate, and why might it return false?
A: The function returns true if the LED was successfully turned on. It returns false if the provided index is invalid, which occurs if the index is 0 or exceeds the total number of available LEDs.

`ca_led_toggle`

Header: `caos/drivers/inc/led.h:70`
Source: `caos/drivers/src/led.c:119:124`
GUARD: `EN_LED`

```
bool ca_led_toggle(u8 led_num)
```

Toggles the state of a specific hardware LED.

Parametri:

Nome	Tipo	Descrizione
`led_num`	`u8`	The 1-based index of the LED to toggle. Must be between 1 and the value returned by ca_led_count(). Values of 0 or greater than the hardware count will result in a failure.

Ritorna:
Returns true if the LED state was successfully toggled, or false if the provided led_num is invalid (out of range).

Note:

- This function is a non-blocking wrapper that interacts directly with the hardware abstraction layer. It is generally thread-safe depending on the underlying hardware implementation, but it is safe to call from within Interrupt Service Routines (ISRs) as it does not perform complex OS operations or blocking waits. The most common beginner mistake is passing an index of 0 or an index higher than the actual number of LEDs available on the board, which will silently fail to toggle the LED. Ensure that ca_led_init() has been called during system startup before attempting to toggle any LEDs.

Uso:

- This function is typically used for visual debugging, status indication, or heartbeat signals in periodic tasks or interrupt callbacks. It is often paired with ca_led_init() during the initialization phase of the application. In real-world scenarios, it is frequently called inside timer callbacks or ISRs (like 10kHz or 1s interrupts) to provide immediate visual feedback on system activity or error states.

Esempio:

```
/* Initialize LEDs before use */
ca_led_init();

/* Toggle LED 1 to indicate system heartbeat */
u8 my_led = 1U;
if (ca_led_toggle(my_led) == false) {
    /* Handle error: invalid LED index provided */
    ca_printf("Error: Failed to toggle LED %u\n", my_led);
```

```
}

/* Example inside an ISR context */
void cb_timer_interrupt(void) {
    /* Toggle LED 2 to signal interrupt activity */
    ca_led_toggle(2U);
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to use ca_led_toggle in my caOS application?
A: It is typically used for visual debugging, status indication, or heartbeat signals. It is commonly called inside periodic tasks, timer callbacks, or ISRs to provide immediate visual feedback. You must ensure ca_led_init() has been called during system startup before using it.

Code example conf:1.00 **Q:** How do I implement a simple heartbeat toggle for the first LED in my code?
A: u8 led_id = 1; if (ca_led_toggle(led_id)) { /* Success */ }

Parameters conf:1.00 **Q:** What are the constraints for the led_num parameter?
A: The led_num parameter is a 1-based index. It must be between 1 and the value returned by ca_led_count(). Passing 0 or a value greater than the hardware count will result in a failure.

Return conf:1.00 **Q:** What does the boolean return value indicate, and why might it return false?
A: The function returns true if the LED state was successfully toggled. It returns false if the provided led_num is invalid, specifically if it is 0 or exceeds the number of LEDs available on the board.

`ca_lptim_deinit`

Header: `caos/drivers/inc/lptim.h:57`
Source: `caos/drivers/src/lptim.c:93:101`
GUARD: `EN_LPTIM`

```
bool ca_lptim_deinit(u8 num)
```

Deinitializes a specific Low Power Timer (LPTIM) instance, clearing its configuration and releasing the hardware resource.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The index of the LPTIM peripheral to deinitialize. Must be between 1 and LPTIM_MAX inclusive. Values outside this range will cause the function to return false.

Ritorna:
Returns true if the deinitialization was successful. Returns false if the provided index is invalid (out of range).

Note:

- This function is non-blocking and performs a hard reset of the LPTIM slot configuration. It is not thread-safe by itself, so ensure that no other task is attempting to start or configure the same timer index simultaneously. It is generally safe to call from an ISR, but you must ensure that the hardware is not currently being accessed by another process to avoid race conditions. A common beginner mistake is calling deinit while the timer is still running;

while the hardware is reset, it is best practice to call ca_lptim_stop first to ensure a clean state transition.

Uso:

- This function is used to clean up resources when a timer is no longer needed, typically during system shutdown or when reconfiguring a peripheral for a different task. It is the final step in the lifecycle of an LPTIM usage pattern: initialization (via a setup function), starting the timer (ca_lptim_start), stopping it (ca_lptim_stop), and finally deinitialization (ca_lptim_deinit). By calling this, you clear the callback function and the wake flag, effectively 'freeing' the slot so it can be safely re-allocated or left in a low-power state without pending interrupts.

Esempio:

```c
/* Example of stopping and cleaning up a timer after a task completes */
#define LPTIM_NUM 1

void task_cleanup(void) {
    /* Stop the timer to ensure no further interrupts occur */
    if (ca_lptim_stop(LPTIM_NUM)) {
        /* Successfully stopped, now release the hardware resource */
        if (ca_lptim_deinit(LPTIM_NUM)) {
            /* Timer is now fully deinitialized and ready for reuse */
        } else {
            /* Handle error: invalid timer index */
        }
    } else {
        /* Timer was not running or already stopped */
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_lptim_deinit and what is its purpose in the timer lifecycle?
A: You should call ca_lptim_deinit when the timer is no longer needed, such as during system shutdown or peripheral reconfiguration. It serves as the final step in the lifecycle (init, start, stop, deinit) to clear configurations, callbacks, and wake flags, effectively freeing the slot for future use.

Code example conf:0.90 **Q:** Provide a minimal C example showing how to safely deinitialize an LPTIM instance.
A: u8 timer_idx = 1; ca_lptim_stop(timer_idx); if (ca_lptim_deinit(timer_idx)) { /* Success */ }

Parameters conf:1.00 **Q:** What are the constraints for the 'num' parameter in ca_lptim_deinit?
A: The 'num' parameter represents the index of the LPTIM peripheral. It must be a u8 value between 1 and LPTIM_MAX inclusive. Providing a value outside this range will cause the function to return false.

Return conf:1.00 **Q:** What do the return values of ca_lptim_deinit indicate?
A: The function returns true if the deinitialization was successful. It returns false if the provided index is invalid, specifically if it is outside the range of 1 to LPTIM_MAX.

`ca_lptim_dispatch`

Header: `caos/drivers/inc/lptim.h:62`
Source: `caos/drivers/src/lptim.c:115:128`
GUARD: `EN_LPTIM`

```
void ca_lptim_dispatch (u8 num); /* chiamare da LPTIMx_IRQHandler */
```

Dispatches the LPTIM interrupt event to the registered callback function for a specific timer instance.

Parametri:

Nome	Tipo	Descrizione
num	u8	The LPTIM instance number (1-based index). Must be between 1 and LPTIM_MAX. Values outside this range are ignored.

Ritorna:
void. This function does not return a value.

Note:

- This function is designed specifically to be called from within the LPTIMx_IRQHandler (Interrupt Service Routine). Because it runs in an ISR context, it must execute as quickly as possible; avoid performing heavy calculations or blocking operations inside the callback function triggered by this dispatcher. It is thread-safe in the sense that it operates on specific slot data, but you must ensure that the LPTIM instance has been properly initialized and armed before the interrupt fires. A common beginner mistake is to perform long-running tasks inside the callback, which can cause the system to miss subsequent interrupts or destabilize the RTOS timing.

Uso:

- This function acts as the bridge between the hardware interrupt and your application logic. It is typically used in the hardware abstraction layer (HAL) or the board support package (BSP) within the LPTIM interrupt vector. You pair this with an initialization function that sets the callback and mode, and a start function that enables the hardware timer. Once the timer expires, the hardware triggers the IRQ, which calls this dispatcher to execute your registered logic, either repeating it if in periodic mode or cleaning up resources if in one-shot mode.

Esempio:

```
/* Inside the file handling hardware interrupts (e.g., stm32l4xx_it.c) */
void LPTIM1_IRQHandler(void) {
    /* Check if the interrupt flag is set for LPTIM1 */
    if (/* Check LPTIM1 interrupt status register */) {
        /* Clear the interrupt flag to prevent re-triggering */
        /* Clear LPTIM1 interrupt flag */

        /* Dispatch the event to the caOS LPTIM manager */
        ca_lptim_dispatch(1);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When and why should I call ca_lptim_dispatch in my caOS project?
A: You should call ca_lptim_dispatch inside the LPTIMx_IRQHandler. It serves as the bridge between the hardware interrupt and your application logic, ensuring that the registered callback function is executed when the LPTIM timer expires.

Code example conf:1.00 **Q:** How do I implement the LPTIM interrupt handler using ca_lptim_dispatch?
A: You should place the call within the specific IRQ handler for the timer instance. Example: void LPTIM1_IRQHandler(void) { ca_lptim_dispatch(1); }

Parameters conf:1.00 **Q:** What are the constraints for the 'num' parameter in ca_lptim_dispatch?

A: The 'num' parameter is a u8 representing the 1-based index of the LPTIM instance. It must be between 1 and LPTIM_MAX. Any value provided outside this range is ignored by the function.

Error/Edge case conf:1.00 **Q:** What is a common mistake to avoid when using ca_lptim_dispatch?

A: A common mistake is performing long-running tasks or blocking operations inside the callback triggered by the dispatcher. Because this runs in an ISR context, heavy calculations can cause the system to miss subsequent interrupts or destabilize the RTOS timing.

`ca_lptim_hw_deinit`

Header: `caos/drivers/inc/ca_glue_hw.h:285`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_lptim_hw_deinit (u8 num); /* ferma, disabilita NVIC e
    spegne il clock periferica */
```

Performs a complete hardware shutdown of the specified Low Power Timer (LPTIM) peripheral.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The index of the LPTIM peripheral to deinitialize. Typically 1. Values outside the supported range are ignored by the hardware-specific implementations.

Ritorna:
This function returns void and does not provide feedback on success or failure.

Note:

- This function is non-blocking and performs direct register manipulation to disable the timer, clear pending interrupts in the NVIC, and gate off the peripheral clock via the RCC. It is not thread-safe; if called while another task is configuring the same timer, it will lead to undefined behavior or system instability. It is generally safe to call from an ISR, but doing so is rarely necessary as deinitialization is typically a cleanup task performed during system shutdown or driver release. A common beginner mistake is calling this function without first ensuring the timer is stopped, which can leave the hardware in an inconsistent state if the peripheral is re-initialized immediately after.

Uso:

- This function is the final step in the lifecycle of an LPTIM driver instance. It is typically called by the higher-level 'ca_lptim_deinit' function after the timer has been stopped and the application no longer requires its services. It is used to save power by disabling the peripheral clock and to ensure that no stray interrupts trigger after the driver has been released. You should pair this with 'ca_lptim_hw_init' and 'ca_lptim_hw_start' to manage the hardware resource lifecycle effectively.

Esempio:

```
/* Example of safely releasing an LPTIM resource */
void release_timer_resource(u8 timer_id) {
    /* 1. Stop the timer to prevent further interrupts */
    if (ca_lptim_hw_stop(timer_id)) {
        /* 2. Perform hardware cleanup: disable clock, NVIC, and clear registers
    */
```

```
        ca_lptim_hw_deinit(timer_id);

        /* 3. Clear internal driver state tracking */
        // s_slots[timer_id - 1U].cb = NULL;
    }
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_lptim_hw_deinit and what is its primary purpose?
A: You should call this function as the final step in the LPTIM driver lifecycle, typically via the higher-level ca_lptim_deinit function. Its primary purpose is to perform a complete hardware shutdown, which includes disabling the timer, clearing pending NVIC interrupts, and gating off the peripheral clock to save power.

Code example conf:1.00 **Q:** Provide a minimal example of how to call the deinitialization function for LPTIM1.
A: To deinitialize the first LPTIM peripheral, use the following code: u8 timer_num = 1; ca_lptim_hw_deinit(timer_num);

Parameters conf:1.00 **Q:** What does the 'num' parameter represent and what happens if I pass an unsupported value?
A: The 'num' parameter is a u8 representing the index of the LPTIM peripheral to deinitialize, typically 1. If you provide a value outside the supported range, the hardware-specific implementation will ignore the request.

Error/Edge case conf:1.00 **Q:** What is a common mistake when calling ca_lptim_hw_deinit?
A: A common beginner mistake is calling this function without first ensuring the timer is stopped. Failing to stop the timer first can leave the hardware in an inconsistent state, especially if you intend to re-initialize the peripheral immediately after.

HW differences conf:1.00 **Q:** How does the implementation of ca_lptim_hw_deinit differ across STM32 families?
A: Implementations vary significantly based on the architecture: F1xx and F4xx families provide empty implementations that do nothing. More advanced families like F7xx and H7xx include specific register manipulation for EXTI (e.g., EXTI->IMR or EXTI_D1->IMR2) to handle interrupts, while U5xx includes additional clock management via RCC->SRDAMR.

`ca_lptim_hw_get_cnt`

Header: `caos/drivers/inc/ca_glue_hw.h:286`
GUARD: `EN_CA_GLUE_HW`

```
extern u32 ca_lptim_hw_get_cnt (u8 num); /* legge CNT con doppia lettura
    (clock asincrono) */
```

Reads the current counter value of a Low Power Timer (LPTIM) using a safe double-read mechanism.

Parametri:

Nome	Tipo	Descrizione
num	u8	The index of the LPTIM peripheral to read. Typically 1 for LPTIM1. Passing an invalid index will return 0.

Ritorna:
The current 32-bit counter value of the LPTIM. Returns 0 if the peripheral index is invalid or if the hardware implementation does not support LPTIM.

Note:

- This function is non-blocking and executes a tight loop to ensure data integrity. It is safe to call from an ISR because it performs a simple register read loop without waiting for external events or OS primitives. The double-read mechanism is mandatory because LPTIMs often run on an asynchronous clock domain relative to the CPU; reading the register while it is updating can return a corrupted value. A common beginner mistake is attempting to read the LPTIM->CNT register directly instead of using this function, which risks capturing a transient, invalid state during the counter increment.

Uso:

- This function is used to retrieve the current tick count of a low-power timer, typically for diagnostic purposes, calculating elapsed time, or checking the progress of a scheduled task. It is usually called after the LPTIM has been initialized via ca_lptim_init and started via ca_lptim_start. In a typical lifecycle, you would start the timer, perform your logic, use ca_lptim_hw_get_cnt to monitor the timer's progress, and eventually call ca_lptim_stop or ca_lptim_deinit when the timer is no longer needed.

Esempio:

```c
/* Example: Monitoring LPTIM1 progress */
#include "caos.h"

void monitor_timer(void) {
    u8 timer_id = 1U;
    u32 current_val;

    /* Check if the timer is currently in use */
    if (ca_lptim_is_busy(timer_id)) {
        /* Retrieve the current counter value safely */
        current_val = ca_lptim_hw_get_cnt(timer_id);

        /* Use the value for logic or logging */
        // ca_printf_n("Current LPTIM1 count: %u", current_val);
    } else {
        /* Handle case where timer is not running */
    }
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_lptim_hw_get_cnt and why is it preferred over direct register access?
A: You should use it to retrieve the current tick count for diagnostics or timing calculations after initializing the timer with ca_lptim_init. It is preferred because LPTIMs often run on an asynchronous clock; reading the register directly risks capturing a corrupted value during an update, whereas this function uses a mandatory double-read mechanism to ensure data integrity.

Code example conf:1.00 **Q:** Provide a minimal example of how to read the LPTIM1 counter in caOS.
A: u32 current_ticks; u8 lptim_id = 1U; current_ticks = ca_lptim_hw_get_cnt(lptim_id);

Parameters conf:1.00 **Q:** What does the 'num' parameter represent and what happens if I pass an invalid value?
A: The 'num' parameter is a u8 representing the index of the LPTIM peripheral (typically 1 for LPTIM1). If you pass an invalid index, the function will return 0.

Return conf:1.00 **Q:** What are the possible return values for ca_lptim_hw_get_cnt and what causes a return of 0?

A: The function returns the 32-bit counter value of the LPTIM. It returns 0 if the peripheral index is invalid or if the specific hardware implementation (such as f1xx or f4xx families) does not support LPTIM functionality.

HW differences conf:1.00 **Q:** Are there differences in how ca_lptim_hw_get_cnt behaves across different STM32 families?

A: Yes. On f7xx, h5xx, h7xx, and u5xx families, the function performs a double-read loop on the LPTIM1->CNT register to handle asynchronous clock domains. Conversely, on f1xx and f4xx families, the function is implemented as a stub that simply returns 0, indicating that LPTIM hardware support is not implemented for those specific families.

`ca_lptim_hw_init`

Header: `caos/drivers/inc/ca_glue_hw.h:282`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_lptim_hw_init (u8 num, u32 ms, bool periodic, bool wake);
    /* valida ms (1..262144) e salva prescaler/ARR/modalit /wake; non
   avvia */
```

Configures the Low Power Timer (LPTIM) hardware parameters without starting the timer.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The LPTIM instance number. Currently, only 1 is supported; other values will cause the function to return false.
`ms`	`u32`	The desired timeout duration in milliseconds. Valid range is 1 to 262144. Values outside this range will cause the function to return false.
`periodic`	`bool`	If true, the timer will automatically reload and restart after expiration. If false, it acts as a one-shot timer.
`wake`	`bool`	If true, the timer is configured to wake the MCU from low-power sleep modes upon expiration.

Ritorna:
Returns true if the configuration was successfully calculated and stored. Returns false if the input parameters are invalid (e.g., unsupported instance, out-of-range duration) or if the hardware platform does not support LPTIM.

Note:

- This function is non-blocking and does not interact with the actual hardware registers; it only calculates and stores the prescaler and auto-reload values in a local configuration structure. It is thread-safe in the sense that it only modifies local driver state, but it is not intended to be called from an ISR as it is part of the initialization sequence. A common mistake is assuming this function starts the timer; it does not. You must call ca_lptim_start to enable the LPTIM peripheral after this initialization. Always check the return value, as some STM32 families (like F1xx or F4xx) do not support LPTIM and will return false immediately.

Uso:

- This function is used during the driver initialization phase to prepare the LPTIM hardware settings. It is typically called after acquiring the timer resource and before starting the timer. It is usually paired with clock initialization functions like ca_clock_hw_lptim_init

to ensure the peripheral is powered before configuration. The typical sequence is: acquire resource -> initialize clock -> call ca_lptim_hw_init -> call ca_lptim_start.

Esempio:

```c
/* Example of initializing a periodic LPTIM */
u8 timer_num = 1;
u32 timeout_ms = 1000;
bool periodic = true;
bool wake_mcu = false;

/* 1. Initialize the hardware clock for the timer */
if (ca_clock_hw_lptim_init(timer_num)) {
    /* 2. Configure the timer parameters */
    if (ca_lptim_hw_init(timer_num, timeout_ms, periodic, wake_mcu)) {
        /* 3. Start the timer after successful configuration */
        ca_lptim_start(timer_num);
    } else {
        /* Configuration failed, perform cleanup */
        ca_lptim_deinit(timer_num);
    }
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_lptim_hw_init in my application initialization sequence?
A: It should be called during the driver initialization phase, specifically after acquiring the timer resource and initializing the clock via ca_clock_hw_lptim_init, but before calling ca_lptim_start to enable the peripheral.

Code example conf:0.90 **Q:** Provide a minimal C example to initialize LPTIM1 for a 1000ms periodic wake-up.
A: u8 timer_num = 1U; u32 timeout = 1000U; bool is_periodic = true; bool can_wake = true; if (ca_lptim_hw_init(timer_num, timeout, is_periodic, can_wake)) { ca_lptim_start(timer_nun }

Parameters conf:1.00 **Q:** What are the constraints on the 'ms' parameter for ca_lptim_hw_init?
A: The 'ms' parameter must be in the range of 1 to 262144. Any value outside this range, including 0, will cause the function to return false.

Return conf:1.00 **Q:** What does a return value of false indicate when calling ca_lptim_hw_init?
A: A false return indicates that the configuration failed. This happens if the instance number is not 1, the duration is out of range, or the specific hardware platform (like STM32F1xx or F4xx) does not support LPTIM.

Error/Edge case conf:1.00 **Q:** Why is my LPTIM not running after I call ca_lptim_hw_init?
A: A common mistake is assuming ca_lptim_hw_init starts the timer. This function only calculates and stores configuration parameters locally. You must explicitly call ca_lptim_start to enable the peripheral.

HW differences conf:1.00 **Q:** Are there differences in LPTIM support across STM32 families in caOS?
A: Yes. STM32F1xx and F4xx families do not support LPTIM, and their implementations of ca_lptim_hw_init will always return false. Conversely, families like F7xx, H5xx, H7xx, and U5xx implement the logic to calculate prescaler and auto-reload values.

`ca_lptim_hw_start`

Header: `caos/drivers/inc/ca_glue_hw.h:290`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_lptim_hw_start (u8 num); /* complete sequence of enable +
    CNTSTRT (periodic) or SNGSTRT (oneshot) */
```

Initializes and triggers the Low Power Timer (LPTIM) hardware to begin counting.

Parametri:

Nome	Tipo	Descrizione
num	u8	The index of the LPTIM peripheral to start. Currently, only 1 is supported; any other value will cause the function to return false.

Ritorna:
Returns true if the timer was successfully configured and started. Returns false if the peripheral index is invalid or if the hardware initialization sequence fails.

Note:

- This function is blocking because it waits for hardware synchronization flags (like ARROK or DIEROK) to ensure the timer registers are correctly updated. It is not thread-safe; you must ensure that no other task or interrupt attempts to modify LPTIM registers while this function is executing. It is generally not recommended to call this from an ISR due to the blocking 'while' loops used for hardware synchronization. A common beginner mistake is calling this function before the LPTIM clock has been enabled or before the configuration parameters (ticks, prescaler) have been set, which leads to undefined timer behavior.

Uso:

- This function is the final step in the LPTIM activation sequence. It is typically used after calling a configuration function that sets the desired timeout duration and mode. It is designed to be called by the driver layer (ca_lptim_start) only after verifying the timer is in a valid state using ca_lptim_is_busy. Once called, the timer begins counting immediately, and the hardware will trigger an interrupt once the programmed period expires.

Esempio:

```
/* Ensure the timer is not already running before starting */
if (!ca_lptim_is_busy(1)) {
    /* Configure the timer parameters first (e.g., via internal driver state) */
    /* Then trigger the hardware start sequence */
    if (ca_lptim_start(1)) {
        /* Timer started successfully, waiting for interrupt... */
    } else {
        /* Handle error: hardware initialization failed */
        ca_printf("Failed to start LPTIM1\n");
    }
} else {
    /* Timer is already busy, stop it first if a restart is needed */
    ca_lptim_stop(1);
}
```

Q&A — fine-tuning review *7/7 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** At what point in the LPTIM initialization sequence should I call ca_lptim_hw_start?
A: It is the final step in the activation sequence. You should call it only after a configuration function has set the timeout duration and mode, and after verifying the timer is in a valid state using ca_lptim_is_busy.

Code example conf:1.00 **Q:** Provide a minimal example of how to invoke the LPTIM start function.
A: u8 timer_idx = 1U; if (ca_lptim_hw_start(timer_idx)) { /* Timer started successfully */ }

Parameters conf:1.00 **Q:** What are the valid values for the 'num' parameter in ca_lptim_hw_start?
A: Currently, only 1 is supported. Any value other than 1 will cause the function to return false.

Return conf:1.00 **Q:** What do the return values of ca_lptim_hw_start indicate?
A: It returns true if the timer was successfully configured and started. It returns false if the peripheral index is invalid or if the hardware initialization sequence fails.

Error/Edge case conf:1.00 **Q:** What is a common mistake that leads to undefined timer behavior when using this function?
A: A common mistake is calling this function before the LPTIM clock has been enabled or before the configuration parameters like ticks and prescaler have been set.

Behavior conf:1.00 **Q:** Is ca_lptim_hw_start thread-safe or safe to call from an ISR?
A: It is not thread-safe; you must ensure no other task or interrupt modifies LPTIM registers during execution. It is generally not recommended to call it from an ISR because it uses blocking 'while' loops to wait for hardware synchronization flags.

HW differences conf:1.00 **Q:** How does the hardware implementation of ca_lptim_hw_start differ between STM32 families?
A: While all families check for index 1, the synchronization logic varies: F7/H7 families synchronize on ARROK, whereas H5/U5 families require synchronization on both DIEROK and ARROK. Additionally, wake-up configuration differs, with F7 using EXTI->RTSR/IMR and H7 using EXTI_D1->IMR2.

`ca_lptim_hw_stop`

Header: `caos/drivers/inc/ca_glue_hw.h:291`
GUARD: `EN_CA_GLUE_HW`

```
1 extern bool ca_lptim_hw_stop (u8 num); /* disables the counter (CNT
     restarts from 0 at the next start) */
```

Disables the specified Low Power Timer (LPTIM) hardware peripheral and resets its counter.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The identifier of the LPTIM peripheral to stop. Must be 1. Passing any other value will result in a false return.

Ritorna:
Returns true if the timer was successfully stopped and the associated interrupts/flags were cleared. Returns false if the timer index is invalid or if the hardware operation failed.

Note:

- This function is non-blocking and performs direct register manipulation to halt the LPTIM peripheral. It is safe to call from an ISR, as it clears pending interrupts and resets hardware flags to ensure no stale events trigger after the stop command. A critical detail is that the timer counter (CNT) is reset to zero; if you start the timer again, it will begin counting from zero, not from where it left off. The most common mistake is assuming this function handles software-level cleanup; it only manages the hardware state. You must ensure that any associated software callbacks or state machines are handled separately if you intend to stop the timer permanently.

Uso:

- This function is used to halt a running timer, typically to prevent an interrupt from firing during a critical section or when a timeout condition is no longer relevant. It is part of

the standard LPTIM lifecycle: initialize the peripheral with ca_lptim_init, start it with ca_lptim_start, and use ca_lptim_stop to pause it. It is often used in conjunction with ca_lptim_is_busy to verify if the timer is active before attempting to stop it, ensuring that you do not perform unnecessary hardware writes.

Esempio:

```c
/* Example: Safely stopping an active LPTIM */
u8 timer_id = 1;

/* Check if the timer is currently running before stopping */
if (ca_lptim_is_busy(timer_id)) {
    if (ca_lptim_stop(timer_id)) {
        /* Timer successfully halted and interrupts cleared */
    } else {
        /* Error: Failed to stop the timer */
    }
} else {
    /* Timer was already idle */
}

/* If the timer is no longer needed, perform de-initialization */
ca_lptim_deinit(timer_id);
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_lptim_hw_stop function in my caOS application?
A: You should use ca_lptim_hw_stop to halt a running LPTIM peripheral, typically to prevent interrupts during critical sections or when a timeout condition is no longer relevant. It is part of the standard lifecycle and should be used to pause the hardware timer.

Code example conf:1.00 **Q:** Provide a minimal C example of how to stop the LPTIM1 peripheral using caOS types.
A: u8 timer_id = 1U; if (ca_lptim_hw_stop(timer_id)) { /* Timer stopped successfully */ }

Parameters conf:1.00 **Q:** What are the constraints for the 'num' parameter in ca_lptim_hw_stop?
A: The 'num' parameter is a u8 representing the LPTIM peripheral identifier. It must be 1. Passing any value other than 1 will result in the function returning false.

Return conf:1.00 **Q:** What do the boolean return values of ca_lptim_hw_stop indicate?
A: A return value of true indicates the timer was successfully stopped and associated interrupts/flags were cleared. A return value of false indicates that the timer index provided was invalid or the hardware operation failed.

HW differences conf:1.00 **Q:** How does the implementation of ca_lptim_hw_stop differ across STM32 families?
A: While all implementations reset LPTIM1->CR and clear the ARRMCF flag, they differ in how they handle external interrupts. For example, the F7xx family modifies EXTI->IMR and EXTI->PR, the H7xx family modifies EXTI_D1->IMR2, whereas the H5xx and U5xx families do not perform explicit EXTI register modifications in the provided code snippets. Additionally, the F1xx and F4xx families return false by default, indicating they do not support this hardware operation.

`ca_lptim_init`

Header: `caos/drivers/inc/lptim.h:65`
Source: `caos/drivers/src/lptim.c:213:219`
GUARD: `EN_LPTIM`

```
void ca_lptim_init(void)
```

Initializes the Low Power Timer (LPTIM) driver and registers its command-line interface (CLI) commands.

Ritorna:
This function returns void and does not provide a status code, as it performs internal initialization and registration.

Note:

- This function is non-blocking and is intended to be called during the system startup phase, typically inside ca_os_main before the scheduler starts. It is not thread-safe in the sense that it uses a static flag to ensure registration happens only once, but it is not designed to be called from an Interrupt Service Routine (ISR). The most common mistake is calling this function multiple times or from different tasks; while the internal static flag prevents duplicate registration, it is best practice to call it exactly once during the initialization sequence. If the CLI is disabled via preprocessor macros, this function effectively becomes a no-op, so do not rely on it to perform hardware-level peripheral configuration if those macros are toggled off.

Uso:

- Use this function at the very beginning of your application, specifically within the ca_os_main function before calling ca_os_run. It is used to expose the LPTIM peripheral's status and control commands to the system's Command Line Interface, allowing you to debug or monitor the timer via a serial terminal. It is typically paired with other driver initialization functions and precedes the creation of tasks that might depend on the LPTIM peripheral, ensuring the CLI infrastructure is ready before the system enters its operational loop.

Esempio:

```
void ca_os_main(void) {
    /* Initialize the LPTIM driver and register its CLI commands */
    ca_lptim_init();

    /* Create a task that uses the LPTIM for timing */
    ca_os_job("lpblink", CA_TH_LEVEL3, task_blink, 0, STACK_SMALL, CA_JOB_STD,
    CA_FAULT_ASSERT);

    /* Start the RTOS scheduler */
    ca_os_run(true);
}
```

Q&A — fine-tuning review *2/2 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_lptim_init and what is its primary purpose?
A: You should call ca_lptim_init during the system startup phase, specifically inside ca_os_main before the scheduler starts (ca_os_run). Its primary purpose is to initialize the LPTIM driver and register its CLI commands, allowing you to monitor or debug the LPTIM peripheral via a serial terminal.

Code example conf:1.00 **Q:** How do I correctly initialize the LPTIM driver in my caOS application?
A: You should call the function once during the initialization sequence in ca_os_main. Example: void ca_os_main(void) { ca_lptim_init(); ca_os_run(); }

ca_lptim_init_oneshot

Header: caos/drivers/inc/lptim.h:52
Source: caos/drivers/src/lptim.c:69:79
GUARD: EN_LPTIM

```
bool ca_lptim_init_oneshot (u8 num, u32 delay_ms, ca_lptim_cb_t cb, bool
    wake)
```

Configures a Low Power Timer (LPTIM) to trigger a single callback after a specified delay.

Parametri:

Nome	Tipo	Descrizione
num	u8	The hardware timer index (e.g., 1, 2). Must be a valid, available timer ID.
delay_ms	u32	The duration in milliseconds before the callback is executed.
cb	ca_lptim_cb_t	The function pointer to be executed when the timer expires.
wake	bool	If true, the timer can wake the MCU from low-power sleep modes.

Ritorna:
Returns true if the timer was successfully acquired and configured; returns false if the timer is already in use, hardware initialization failed, or parameters were invalid.

Note:

- This function is non-blocking; it only sets up the hardware registers and internal state. It is not thread-safe by itself, so ensure you are not re-initializing the same timer from different tasks simultaneously. It cannot be called from an ISR because it performs resource acquisition and hardware clock initialization. A common mistake is forgetting to call ca_lptim_start() after this function; the timer will remain idle until explicitly started. Always ensure the timer is released via ca_lptim_deinit() when finished to prevent resource leaks.

Uso:

- This function is used when you need a one-time delay or a delayed action without blocking the CPU in a busy-wait loop. It is typically used in a sequence: first, you call ca_lptim_init_oneshot() to configure the timer, then ca_lptim_start() to begin the countdown, and finally, you handle the expiration in the provided callback. It is often paired with ca_os_sleep() in the main task to wait for a flag set by the callback, allowing the system to remain responsive or enter low-power states while waiting.

Esempio:

```
static volatile bool s_oneshot_hit = false;

void cb_pause_done(void) {
    s_oneshot_hit = true; // Signal that the timer expired
}

bool run_delayed_task(u32 ms) {
    s_oneshot_hit = false;
    // Initialize timer 1 for a one-shot delay
    if (!ca_lptim_init_oneshot(1, ms, cb_pause_done, false)) {
        return false; // Failed to acquire or init
    }
    // Start the timer countdown
```

```
    ca_lptim_start(1);
    // Wait for the callback to set the flag
    while (!s_oneshot_hit) {
        ca_os_sleep(10);
    }
    // Clean up resources
    ca_lptim_deinit(1);
    return true;
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_lptim_init_oneshot instead of a standard blocking delay?
A: You should use ca_lptim_init_oneshot when you need a non-blocking delay or a delayed action that allows the CPU to remain responsive or enter low-power states. It is ideal for scenarios where you want to avoid busy-wait loops.

Code example conf:1.00 **Q:** How do I correctly initialize and start a one-shot timer in caOS?
A: You must first initialize the timer and then start it. Example: void my_cb(void) { /* handle expiration */ } ... u8 timer_id = 1; if (ca_lptim_init_oneshot(timer_id, 500, my_cb, true)) { ca_lptim_start(timer_id); }

Parameters conf:1.00 **Q:** What is the purpose of the 'wake' parameter in ca_lptim_init_oneshot?
A: The 'wake' parameter is a boolean that, when set to true, allows the LPTIM to wake the MCU from low-power sleep modes upon timer expiration.

Return conf:1.00 **Q:** What does a false return value from ca_lptim_init_oneshot indicate?
A: A false return indicates that the timer could not be configured. This happens if the timer is already in use, hardware initialization failed, or the provided parameters were invalid.

Error/Edge case conf:1.00 **Q:** Can I call ca_lptim_init_oneshot from an interrupt service routine (ISR)?
A: No, you cannot call this function from an ISR because it performs resource acquisition and hardware clock initialization, which are not permitted in that context.

`ca_lptim_init_periodic`

Header: `caos/drivers/inc/lptim.h:51`
Source: `caos/drivers/src/lptim.c:57:67`
GUARD: `EN_LPTIM`

```
bool ca_lptim_init_periodic(u8 num, u32 period_ms, ca_lptim_cb_t cb,
    bool wake)
```

Initializes a Low Power Timer (LPTIM) to trigger a callback function at a fixed, repeating time interval.

Parametri:

Nome	Tipo	Descrizione
num	u8	The hardware timer index (1-based). Must be within the range [1, LPTIM_MAX].
period_ms	u32	The interval in milliseconds between timer triggers. Must be a non-zero value supported by the hardware clock.
cb	ca_lptim_cb_t	A pointer to the callback function to be executed when the timer expires. Must not be NULL.
wake	bool	If true, the timer is configured to wake the MCU from low-power sleep modes when it expires.

Ritorna:
Returns true if the timer was successfully acquired and initialized. Returns false if the timer is already in use, hardware initialization fails, or invalid parameters are provided.

Note:

- This function is non-blocking and thread-safe, but it is not intended to be called from an ISR. It performs resource acquisition; if you fail to call ca_lptim_deinit later, the timer will remain locked and unavailable for other parts of the application. A common beginner mistake is forgetting to check the return value; if the timer is already busy, the function will return false, and subsequent calls to start or stop the timer will fail or cause undefined behavior. Ensure the callback function is kept short and efficient, as it typically executes in a high-priority context.

Uso:

- This function is used to set up periodic tasks, such as blinking an LED, polling a sensor, or triggering a state machine update at regular intervals. It is typically used in a sequence: first, call ca_lptim_init_periodic to configure the hardware, then call ca_lptim_start to begin the counting process. Once the task is complete, you must call ca_lptim_stop followed by ca_lptim_deinit to release the hardware resource back to the system. It is often paired with ca_os_sleep to wait for a specific number of timer events to occur in the main application loop.

Esempio:

```c
/* Example: Initialize LPTIM1 for a 500ms periodic task */
void my_callback(void) {
    /* This code runs every 500ms */
}

void setup_timer(void) {
    /* Initialize timer 1 with 500ms period, no wake-up */
    if (ca_lptim_init_periodic(1, 500, my_callback, false)) {
        ca_lptim_start(1);
    } else {
        /* Handle error: timer might be busy or hardware failed */
    }
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_lptim_init_periodic function in my caOS application?
A: You should use this function to set up periodic tasks that require regular intervals, such as blinking an LED, polling a sensor, or triggering a state machine update. It is typically used as the first step in a sequence: call ca_lptim_init_periodic to configure the hardware, then call ca_lptim_start to begin counting.

Code example conf:0.90 **Q:** Provide a minimal C example showing how to initialize an LPTIM using caOS types.
A: void my_callback(void) { /* Task logic */ }
void setup_timer(void) { u8 timer_idx = 1; u32 interval = 1000; if (ca_lptim_init_periodic(timer_idx, interval, my_callback, true)) { ca_lptim_start(timer_idx); } }

Parameters conf:1.00 **Q:** What are the constraints for the 'num' and 'period_ms' parameters?
A: The 'num' parameter represents the hardware timer index (1-based) and must be within the range [1, LPTIM_MAX]. The 'period_ms' parameter is the interval in milliseconds and must be a non-zero value supported by the hardware clock.

Return conf:1.00 **Q:** What does a return value of false indicate when calling ca_lptim_init_periodic?
A: A return value of false indicates that the initialization failed. This can happen if the specified timer is already in use, if the hardware initialization fails, or if invalid parameters were provided.

Error/Edge case conf:1.00 **Q:** What is a common mistake developers make when using ca_lptim_init_periodic
A: A common mistake is forgetting to check the return value. If the timer is already busy, the function returns false, and subsequent calls to start or stop the timer will fail or cause undefined behavior. Additionally, failing to call ca_lptim_deinit after use will leave the timer locked and unavailable for other parts of the application.

`ca_lptim_is_busy`

Header: `caos/drivers/inc/lptim.h:60`
Source: `caos/drivers/src/lptim.c:51:55`
GUARD: `EN_LPTIM`

```
1 bool ca_lptim_is_busy (u8 num)
```

Checks if a specific Low Power Timer (LPTIM) peripheral is currently allocated or in use by the system.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The index of the LPTIM peripheral to check. Valid values are 1 up to LPTIM_MAX. Values outside this range are treated as 'busy' to prevent invalid hardware access.

Ritorna:
Returns true if the LPTIM is currently busy, reserved, or if the provided index is invalid. Returns false if the LPTIM is available for use.

Note:

- This function is non-blocking and thread-safe as it performs a simple bitwise read on a static status variable. It is safe to call from an ISR because it does not involve complex logic or waiting. The most common mistake is assuming that 'false' means the hardware is powered on; this function only checks the software allocation status. Always ensure the LPTIM driver has been initialized before relying on this status, as an uninitialized driver might report an incorrect state.

Uso:

- This function is primarily used as a guard clause before attempting to configure or start an LPTIM peripheral. It is typically paired with initialization functions like ca_lptim_init_periodic to ensure you do not overwrite an active timer configuration. In power management contexts, it is used to verify if the LPTIM is already claimed by another task or wake-up

routine, preventing resource conflicts when the system attempts to enter low-power sleep modes.

Esempio:

```c
/* Check if LPTIM1 is available before starting a periodic task */
#define LPTIM_ID 1

if (!ca_lptim_is_busy(LPTIM_ID)) {
    /* Safe to initialize the timer as it is not in use */
    if (ca_lptim_init_periodic(LPTIM_ID, 1000, my_callback, false)) {
        // Timer successfully started
    }
} else {
    /* Handle the error: LPTIM is already occupied by another process */
    log_error("LPTIM1 is busy, cannot start periodic task");
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_lptim_is_busy in my application code?
A: You should call this function as a guard clause before configuring or starting an LPTIM peripheral. It is typically used to prevent overwriting an active timer configuration or to verify if a timer is already claimed by another task or wake-up routine before entering low-power sleep modes.

Code example conf:1.00 **Q:** Provide a minimal C example showing how to check if LPTIM 1 is available.
A: u8 timer_idx = 1; if (!ca_lptim_is_busy(timer_idx)) { /* Safe to proceed with initialization */ }

Parameters conf:1.00 **Q:** What are the valid values for the 'num' parameter in ca_lptim_is_busy?
A: The 'num' parameter represents the index of the LPTIM peripheral. Valid values range from 1 up to LPTIM_MAX. Any value provided outside of this range is treated as 'busy' by the function to prevent invalid hardware access.

Return conf:1.00 **Q:** What does a return value of 'true' indicate, and can it be caused by invalid inputs?
A: A return value of 'true' indicates that the LPTIM is currently busy, reserved, or that the provided index is invalid. If you pass an index outside the range of 1 to LPTIM_MAX, the function will return 'true' to prevent invalid hardware access.

`ca_lptim_start`

Header: `caos/drivers/inc/lptim.h:55`
Source: `caos/drivers/src/lptim.c:81:85`
GUARD: `EN_LPTIM`

```c
bool ca_lptim_start (u8 num)
```

Starts a previously initialized Low Power Timer (LPTIM) peripheral.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The index of the LPTIM peripheral to start. Must correspond to a valid hardware timer index previously initialized via ca_lptim_init_periodic or similar.

Ritorna:

Returns true if the timer was successfully started. Returns false if the timer is not in a busy/initialized state or if the hardware start command failed.

Note:

- This function is non-blocking and performs a quick check to ensure the timer is in a valid state before triggering the hardware. It is thread-safe assuming the underlying hardware registers are protected, but it is generally not intended to be called from an ISR unless specified by the architecture. A common beginner mistake is calling this function before initializing the timer; if the timer has not been initialized, ca_lptim_is_busy will return false, and the start command will be ignored. Always ensure the timer is properly configured with a callback before calling start to avoid unexpected behavior when the timer expires.

Uso:

- This function is used to trigger the countdown or periodic firing of a timer after it has been fully configured. It is typically used in a sequence: first, you initialize the timer with a period and a callback function using ca_lptim_init_periodic, then you call ca_lptim_start to begin the operation. Once the task is complete, you must call ca_lptim_stop to halt the timer and ca_lptim_deinit to release the hardware resources for other parts of the application. It is essential to pair this with stop and deinit to prevent resource leaks or unintended timer interrupts.

Esempio:

```c
/* Example of starting a periodic timer for a specific task */
#define LPTIM_NUM 1

if (ca_lptim_init_periodic(LPTIM_NUM, 1000, my_callback, false)) {
    /* Timer is initialized, now start it */
    if (ca_lptim_start(LPTIM_NUM)) {
        /* Timer is running, perform other tasks */
    } else {
        /* Handle error: failed to start */
    }
} else {
    /* Handle error: initialization failed */
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_lptim_start in my application lifecycle?
A: You should call ca_lptim_start only after the LPTIM peripheral has been fully configured and initialized using a function like ca_lptim_init_periodic. It is used to trigger the countdown or periodic firing of the timer. Once the task is complete, you must call ca_lptim_stop and ca_lptim_deinit to release hardware resources.

Code example conf:0.90 **Q:** Provide a minimal C example showing how to start an LPTIM peripheral using caOS types.
A: u8 timer_idx = 0; if (ca_lptim_init_periodic(timer_idx, 1000, my_callback)) { ca_lptim_start(timer_idx); }

Parameters conf:1.00 **Q:** What does the 'num' parameter represent in ca_lptim_start?
A: The 'num' parameter is a u8 representing the index of the LPTIM peripheral you wish to start. This index must correspond to a valid hardware timer that has already been initialized; if the timer has not been initialized, the function will fail.

Return conf:1.00 **Q:** What do the boolean return values of ca_lptim_start indicate?
A: A return value of true indicates the timer was successfully started. A return value of false indicates the timer is not in a busy/initialized state or the hardware start command failed. Specifically, if ca_lptim_is_busy returns false, the start command is ignored.

ca_lptim_stop

Header: caos/drivers/inc/lptim.h:56
Source: caos/drivers/src/lptim.c:87:91
GUARD: EN_LPTIM

```
bool ca_lptim_stop (u8 num)
```

Stops a running Low Power Timer (LPTIM) instance identified by its index.

Parametri:

Nome	Tipo	Descrizione
num	u8	The index of the LPTIM peripheral to stop. Must be a valid index previously initialized via ca_lptim_init_periodic or similar functions.

Ritorna:
Returns true if the timer was successfully stopped. Returns false if the timer was not currently running (not busy) or if the hardware stop operation failed.

Note:

- This function is non-blocking and executes immediately. It is generally thread-safe, but you must ensure that no other task is attempting to re-initialize or modify the same timer index simultaneously. A common beginner mistake is calling this function on a timer that has already been stopped or never started, which will return false; always check the return value if your application logic depends on the timer state. It is safe to call from an ISR, provided the underlying hardware driver implementation is also ISR-safe, though typically this is used in task-level code to manage periodic events.

Uso:

- Use this function when you need to halt a periodic task or a one-shot timer before it expires. It is typically used in a lifecycle sequence: first, you initialize the timer with a callback using ca_lptim_init_periodic, start it with ca_lptim_start, perform your timed operations, and finally call ca_lptim_stop to prevent further interrupts. After stopping, you should call ca_lptim_deinit if you no longer need the timer resource, ensuring proper cleanup of the hardware peripheral.

Esempio:

```c
/* Example: Stop a periodic timer after a specific number of events */
#define LPTIM_NUM 0

void task_control_timer(void) {
    // Initialize and start the timer
    if (ca_lptim_init_periodic(LPTIM_NUM, 100, my_callback, false)) {
        ca_lptim_start(LPTIM_NUM);

        // Wait for some condition
        ca_os_sleep(1000);

        // Stop the timer to prevent further callbacks
        if (ca_lptim_stop(LPTIM_NUM)) {
            // Successfully stopped, now clean up
            ca_lptim_deinit(LPTIM_NUM);
        } else {
            // Handle error: timer was already stopped or invalid
```

```
        }
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_lptim_stop function in my caOS application?
A: You should use ca_lptim_stop when you need to halt a periodic task or a one-shot timer before it expires. It is typically used as part of a lifecycle sequence: after initializing with ca_lptim_init_periodic and starting with ca_lptim_start, call ca_lptim_stop to prevent further interrupts before potentially calling ca_lptim_deinit to clean up the resource.

Code example conf:1.00 **Q:** Provide a minimal C code snippet showing how to stop an LPTIM instance in caOS.
A: u8 timer_idx = 1; if (ca_lptim_stop(timer_idx)) { /* Timer stopped successfully */ } else { /* Timer was not running or failed to stop */ }

Parameters conf:1.00 **Q:** What does the 'num' parameter represent in ca_lptim_stop?
A: The 'num' parameter is a u8 value representing the index of the LPTIM peripheral to stop. It must be a valid index that was previously initialized via functions like ca_lptim_init_periodic.

Return conf:1.00 **Q:** What do the boolean return values of ca_lptim_stop indicate?
A: The function returns true if the timer was successfully stopped. It returns false if the timer was not currently running (not busy) or if the hardware stop operation failed. You should always check this return value if your application logic depends on the timer state.

`ca_lptim_wake_active`

Header: `caos/drivers/inc/lptim.h:61`
Source: `caos/drivers/src/lptim.c:103:111`
GUARD: `EN_LPTIM`

```
u32 ca_lptim_wake_active(void); /* bitmask istanze in uso con wake=true
    (per il pwr) */
```

Returns a bitmask identifying which LPTIM instances are currently configured to trigger a system wake-up from low-power modes.

Ritorna:
A 32-bit integer where each bit position corresponds to an LPTIM instance index (e.g., bit 0 for LPTIM1). If a bit is set (1), that specific timer is active and configured to wake the MCU; if clear (0), it is not.

Note:

- This function is non-blocking and thread-safe as it performs a read-only operation on the internal state of the LPTIM driver. It is safe to call from an ISR, as it does not involve mutexes or complex synchronization primitives. The most common mistake is assuming that a set bit implies the timer is currently running; it only confirms that the timer is 'in use' and has the 'wake' flag enabled. Ensure the LPTIM driver is initialized before calling this, otherwise, the returned mask will always be zero.

Uso:

- This function is primarily used by the Power Management module (PWR) before entering low-power states like Sleep or Stop. It allows the system to determine if it needs to configure

specific EXTI lines or power-domain settings to ensure the LPTIM can actually trigger a wake-up event. It is typically paired with other 'wake_active' functions from different peripherals (like ca_lpuart_wake_active) to build a comprehensive wake-up configuration mask before the CPU executes the low-power entry sequence.

Esempio:

```c
/* Check if any LPTIM is configured to wake the system */
u32 wake_mask = ca_lptim_wake_active();

if (wake_mask != 0) {
    /* At least one LPTIM is set to wake the system */
    /* Proceed to set power flags for LPTIM wake-up */
    /* Example: ext_flags |= CA_PWR_WAKE_LPTIM; */
} else {
    /* No LPTIM wake-up sources active */
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_lptim_wake_active and what is its primary purpose?
A: You should call this function primarily within the Power Management module (PWR) before the CPU enters low-power states like Sleep or Stop. Its purpose is to determine which LPTIM instances are configured to trigger a system wake-up, allowing the system to properly configure EXTI lines or power-domain settings to ensure wake-up events are handled correctly.

Code example conf:1.00 **Q:** How can I check if LPTIM1 is configured for wake-up using caOS types?
A: You can check the bitmask returned by the function as follows: u32 mask = ca_lptim_wake_active(); if (mask & 0x01) { /* LPTIM1 is configured to wake the MCU */ }

Return conf:1.00 **Q:** What does the 32-bit return value represent, and why might it return zero?
A: The return value is a bitmask where each bit corresponds to an LPTIM instance index (e.g., bit 0 for LPTIM1). A set bit (1) indicates the timer is configured to wake the MCU. It will return zero if no LPTIM instances have the wake flag enabled, or if the LPTIM driver was not initialized prior to calling the function.

`ca_lpuart_configure_gpio`

Header: `caos/drivers/inc/lpuart.h:52`
Source: `caos/drivers/src/lpuart.c:39:42`
GUARD: `EN_LPUART`

```c
bool ca_lpuart_configure_gpio(bool alternate_gpio); /* TX/RX pins in AF (also called by init) */
```

Configures the LPUART TX and RX pins to their required Alternate Function (AF) mode for hardware communication.

Parametri:

Nome	Tipo	Descrizione
`alternate_gpio`	`bool`	A boolean flag where true (1) selects the alternate pin mapping (if supported by the hardware) and false (0) selects the default pin mapping.

Ritorna:

Returns true (1) if the GPIO configuration was successful, or false (0) if the hardware initialization failed.

Note:

- This function is a wrapper for the low-level hardware initialization of the LPUART pins. It is non-blocking and executes quickly, but it is not thread-safe if called simultaneously from multiple tasks, though it is typically only called during system startup. It is generally not safe to call from an ISR because it interacts with the GPIO peripheral registers which may require clock enabling or other shared resource access. The most common beginner mistake is calling this function before the GPIO peripheral clock has been enabled in the RCC registers, which will result in the configuration being ignored or the system hanging.

Uso:

- This function is used during the system initialization phase to prepare the physical pins for serial communication. It is typically called inside ca_lpuart_init or during the early system boot sequence before the LPUART peripheral is enabled. It pairs with other initialization functions like ca_lpuart_init to ensure the hardware is ready to receive or transmit data. If you need to override the default pin mapping or perform custom hardware setup, you call this function explicitly before starting the UART communication tasks.

Esempio:

```
/* Initialize the LPUART hardware pins using the standard alternate mapping */
if (ca_lpuart_configure_gpio(true)) {
    /* Pins are now in Alternate Function mode, safe to proceed with LPUART init */
    /* Initialize LPUART with 115200 baud, no RX callback, wake enabled, and alternate pins */
    if (ca_lpuart_init(115200, NULL, true, true)) {
        /* LPUART is ready for use */
    } else {
        /* Handle initialization failure */
    }
} else {
    /* Handle GPIO configuration error */
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_lpuart_configure_gpio and what is its primary purpose?
A: You should call this function during the system initialization phase, typically inside ca_lpuart_init or during the early boot sequence before the LPUART peripheral is enabled. Its purpose is to configure the LPUART TX and RX pins to their required Alternate Function (AF) mode to enable hardware communication.

Code example conf:0.95 **Q:** How do I implement a basic initialization sequence for the LPUART pins using caOS types?
A: You must ensure the GPIO peripheral clock is enabled in the RCC registers first to avoid configuration failure. A minimal implementation would look like this: bool status; status = ca_lpuart_configure_gpio(); if (status == 1) { /* proceed with lpuart init */ }

Return conf:1.00 **Q:** What do the return values of ca_lpuart_configure_gpio indicate, and what causes a failure?
A: The function returns true (1) if the GPIO configuration was successful and false (0) if the hardware initialization failed. A common cause for failure is calling the function before the GPIO peripheral clock has been enabled in the RCC registers, which can cause the configuration to be ignored or the system to hang.

ca_lpuart_deinit

Header: caos/drivers/inc/lpuart.h:47
Source: caos/drivers/src/lpuart.c:55:63
GUARD: EN_LPUART

```
bool ca_lpuart_deinit(void)
```

Safely shuts down the LPUART peripheral and resets its internal driver state.

Ritorna:
Returns true if the LPUART was successfully deinitialized. Returns false if the LPUART was already inactive (s_in_use is false), indicating an attempt to deinitialize an uninitialized or already closed peripheral.

Note:

- This function is non-blocking and performs a clean teardown of the hardware and software state. It is not thread-safe by itself, so ensure no other tasks are attempting to transmit or receive data while this is called. It is generally not safe to call this from an ISR unless you are certain no other thread is accessing the LPUART. The most common beginner mistake is calling this function without checking the return value or calling it while an active communication process is still pending, which can lead to data loss or undefined behavior in the application logic.

Uso:

- This function is used to release the LPUART hardware resources when they are no longer needed, typically to save power or to reconfigure the peripheral for a different communication mode. It is the counterpart to the initialization function and should be called as part of a clean shutdown sequence or before entering a deep sleep mode where the peripheral must be disabled. It is typically paired with a preceding initialization call and ensures that the internal callback pointer (s_on_rx) and usage flags are reset to prevent dangling pointers or accidental re-entry.

Esempio:

```
/* Check if the LPUART is currently active before shutting it down */
if (s_in_use) {
    /* Perform any necessary cleanup of application-level buffers here */
    if (ca_lpuart_deinit()) {
        /* Successfully released hardware, safe to enter low power mode */
    } else {
        /* Handle error: deinit failed, possibly already closed */
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_lpuart_deinit in my caOS application?
A: You should call ca_lpuart_deinit when LPUART hardware resources are no longer needed, such as during a clean shutdown sequence, to save power, or before entering deep sleep modes where the peripheral must be disabled.

Code example conf:0.95 **Q:** How do I safely shut down the LPUART peripheral in my code?
A: Ensure no other tasks are accessing the peripheral, then call the function and check the result:
bool status = ca_lpuart_deinit(); if (status) { /* Successfully deinitialized */ } else { /* Already inactive */ }

Return conf:1.00 **Q:** What do the return values of ca_lpuart_deinit indicate?
A: The function returns true if the LPUART was successfully deinitialized. It returns false if the LPUART was already inactive (s_in_use is false), which indicates an attempt to deinitialize a peripheral that was not initialized or was already closed.

`ca_lpuart_dispatch`

Header: `caos/drivers/inc/lpuart.h:61`
Source: `caos/drivers/src/lpuart.c:89:96`
GUARD: `EN_LPUART`

```
void ca_lpuart_dispatch (void); /* chiamare da LPUART1_IRQHandler */
```

Processes incoming LPUART data by draining the hardware buffer and triggering the registered callback.

Ritorna:
void

Note:

- This function is designed specifically to be called from the LPUART1_IRQHandler. It is non-blocking in the sense that it only processes data currently present in the hardware FIFO; once the FIFO is empty, it returns immediately. It is not thread-safe on its own, but since it is intended for an ISR context, it relies on the hardware's atomic nature. A common beginner mistake is calling this function from the main loop instead of the ISR, which prevents the system from reacting to incoming data in real-time. Another critical error is failing to ensure the callback function (s_on_rx) is registered before the interrupt is enabled, which would lead to a null pointer dereference.

Uso:

- This function is the bridge between the hardware interrupt and your application logic. You typically use it by placing it inside the LPUART1_IRQHandler function in your STM32 family-specific file. It is used in conjunction with a registration function (like a hypothetical 'ca_lpuart_set_rx_callback') that defines what the system should do with the received byte. The typical sequence is: initialize the LPUART peripheral, register your callback function, enable the LPUART interrupt in the NVIC, and then let this function handle the data flow automatically whenever the hardware signals a receive event.

Esempio:

```
void LPUART1_IRQHandler(void) {
    /* This function is called automatically by the hardware when data arrives */
    /* It drains the hardware buffer and triggers the registered callback */
    ca_lpuart_dispatch();

    /* Clear interrupt flags if necessary, depending on the specific STM32 HAL/LL implementation */
}
```

Q&A — fine-tuning review *2/2 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** Where should I place the ca_lpuart_dispatch function and why is it important to call it there?
A: You must call ca_lpuart_dispatch inside the LPUART1_IRQHandler. It is designed to process incoming data by draining the hardware FIFO and triggering the registered

callback. Calling it from the main loop instead of the ISR is a common mistake that prevents real-time reaction to incoming data.

Code example conf:1.00 **Q:** How do I implement the LPUART1_IRQHandler using ca_lpuart_dispatch? **A:** You should place the function call directly within the interrupt handler as shown: void LPUART1_IRQHandler(void) { ca_lpuart_dispatch(); } Ensure that your callback function is registered before enabling the LPUART interrupt in the NVIC to avoid a null pointer dereference.

`ca_lpuart_hw_deinit`

Header: `caos/drivers/inc/ca_glue_hw.h:293`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_lpuart_hw_deinit (void); /* ferma, disabilita NVIC e
    spegne clock periferica e HSIKERON */
```

Performs a complete hardware shutdown of the LPUART peripheral, including disabling interrupts, clearing registers, and gating clocks.

Ritorna:
This function returns void and does not provide feedback on the success of the hardware operations.

Note:

- This function is non-blocking and executes immediately. It is safe to call from an ISR, but you must ensure that no other tasks are attempting to read or write to the LPUART registers simultaneously, as this function performs a hard reset of the peripheral state. A common mistake is calling this function while a DMA transfer or an interrupt-driven transmission is still in progress, which will result in truncated data and potentially leave the hardware in an inconsistent state. Always ensure the higher-level driver logic has stopped all communication before invoking this hardware-level cleanup.

Uso:

- This function is used as the final step in the LPUART lifecycle, typically when the peripheral is no longer needed or the system is entering a low-power state. It is paired with ca_lpuart_init to ensure the hardware is returned to a clean, power-efficient state. In the caOS driver architecture, it is called by the wrapper function ca_lpuart_deinit, which manages the internal software state flags (like s_in_use) before triggering this hardware-specific cleanup. You should use this whenever you need to release the LPUART resource to save power or reconfigure the pins for a different function.

Esempio:

```
/* Example: Safely shutting down the LPUART peripheral */

if (ca_lpuart_is_busy()) {
    /* Wait for ongoing transmissions to finish before deinitializing */
    ca_os_delay_ms(10);
}

if (ca_lpuart_deinit()) {
    /* The driver state is reset and ca_lpuart_hw_deinit has been called */
    /* The LPUART clock is now gated and interrupts are disabled */
} else {
    /* LPUART was not initialized or already deinitialized */
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_lpuart_hw_deinit and what precautions should I take?

A: You should call ca_lpuart_hw_deinit when the LPUART peripheral is no longer needed or when the system is entering a low-power state. Before calling it, you must ensure that no other tasks are accessing the LPUART registers and that any ongoing DMA or interrupt-driven transmissions have been fully stopped, as this function performs a hard reset of the peripheral state.

Code example conf:0.90 **Q:** How do I correctly implement a shutdown sequence for the LPUART in caOS?

A: To safely shut down the LPUART, you should call the high-level wrapper function ca_lpuart_deinit, which manages internal software state flags like s_in_use before invoking the hardware-level cleanup. Example: { ca_lpuart_deinit(); }

HW differences conf:1.00 **Q:** Are there differences in how ca_lpuart_hw_deinit behaves across different STM32 families?

A: Yes. On the H5xx and U5xx families, the function explicitly clears registers like LPUART1->CR1, disables NVIC interrupts, and gates clocks via RCC_APB3ENR and RCC_CR_HSIKERON. The U5xx implementation additionally clears the RCC_SRDAMR_LPUART1 bit. Conversely, for the F1xx, F4xx, and F7xx families, the provided implementation of ca_lpuart_hw_deinit is empty, meaning it performs no hardware operations on those specific architectures.

`ca_lpuart_hw_gpio_init`

Header: `caos/drivers/inc/ca_glue_hw.h:298`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_lpuart_hw_gpio_init(bool alternate_gpio); /* configures
   TX/RX pins in alternate function; alternate_gpio picks the 2nd pin
   map (H7: PB6/PB7); false if LPUART is not present */
```

Configures the physical microcontroller pins for LPUART communication by setting them to the correct Alternate Function mode.

Parametri:

Nome	Tipo	Descrizione
`alternate_gpio`	`bool`	If true, selects the secondary pin mapping (e.g., PB6/PB7 on H7). If false, uses the default pin mapping. Note that on many MCUs (U5, H5), this parameter is ignored as only one mapping exists.

Ritorna:
Returns true if the GPIO configuration was successful or if the hardware supports LPUART. Returns false if the specific microcontroller model does not support LPUART or if the hardware configuration failed.

Note:

- This function is non-blocking and executes immediately. It is not thread-safe in the sense that it modifies hardware registers directly; therefore, it should only be called during the system initialization phase before the scheduler starts or before the LPUART peripheral is used. It is not intended to be called from an ISR. A common beginner mistake is to call this function multiple times or while the LPUART is already transmitting/receiving data,

which can cause glitches on the communication lines or unexpected peripheral behavior. Ensure that the system clock for the GPIO port is enabled before calling this, as the underlying ca_gpio_configure function assumes the peripheral clock is active.

Uso:

- This function is used exclusively during the hardware abstraction layer (HAL) initialization sequence for the LPUART peripheral. It is typically called inside ca_lpuart_init to prepare the physical pins (TX and RX) before the UART registers are configured for baud rate and parity. You should never call this function manually in your application code; instead, rely on the driver's initialization sequence. It pairs with ca_lpuart_init to ensure the hardware is ready for data transfer.

Esempio:

```c
/* Example of initializing the LPUART peripheral */
#include "caos.h"

void system_setup(void) {
    /* Initialize LPUART: this will internally call ca_lpuart_hw_gpio_init() */
    /* We use false to select the default pin mapping for the target MCU */
    if (ca_lpuart_init(115200, NULL, false, false)) {
        /* LPUART and GPIOs are successfully configured */
    } else {
        /* Handle error: LPUART might not be supported on this MCU or init failed */
        /* Use a system reset or error logging function here */
        ca_os_reset();
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_lpuart_hw_gpio_init in my caOS application?
A: You should not call this function manually in your application code. It is intended for use exclusively during the hardware abstraction layer initialization sequence, typically inside ca_lpuart_init, before the scheduler starts or before the LPUART peripheral is used.

Code example conf:0.90 **Q:** How do I initialize the LPUART GPIOs in my driver code?
A: You should invoke the function as part of your initialization sequence: bool status = ca_lpuart_hw_gpio_init(false); if (status) { /* proceed with LPUART configuration */ }

Return conf:1.00 **Q:** What do the return values of ca_lpuart_hw_gpio_init indicate?
A: The function returns true if the GPIO configuration was successful or if the hardware supports LPUART. It returns false if the specific microcontroller model does not support LPUART or if the hardware configuration failed.

HW differences conf:1.00 **Q:** How does the LPUART GPIO implementation differ across STM32 families?
A: Implementations vary significantly: F1xx, F4xx, and F7xx families return false as they do not support this function. In contrast, H5xx and U5xx use fixed mappings (PB6/PB7 and PA2/PA3 respectively), while H7xx supports an alternate mapping via the 'alternate_gpio' parameter to free up specific pins like PA9.

`ca_lpuart_hw_init`

Header: `caos/drivers/inc/ca_glue_hw.h:292`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_lpuart_hw_init (u32 baud, bool wake); /* HSI16+HSIKERON,
    kernel clock, BRR, RXNEIE (+UESM/EXTI se wake); avvia */
```

Initializes the Low-Power UART (LPUART) hardware peripheral with specific baud rate and wake-up capabilities.

Parametri:

Nome	Tipo	Descrizione
`baud`	`u32`	The desired communication speed in bits per second. Must be non-zero and satisfy the hardware constraint: 3 * baud <= clock_frequency <= 4096 * baud.
`wake`	`bool`	If true, enables the LPUART to wake the MCU from low-power sleep modes (Stop mode) upon receiving data.

Ritorna:
Returns true if the hardware was successfully configured and the receiver is ready. Returns false if the baud rate is invalid, the clock constraints are not met, or the hardware platform does not support LPUART.

Note:

- This function is not thread-safe and must be called during system initialization or from a single control task. It is a blocking function because it waits for the HSI clock to stabilize and for the LPUART receiver to acknowledge readiness (REACK). It must NOT be called from an Interrupt Service Routine (ISR) due to these blocking loops. A common mistake is failing to verify the return value; if the baud rate is too high or low for the current clock source, the function will fail, and proceeding as if it succeeded will lead to silent communication errors. Additionally, ensure that the system clock configuration is stable before calling this, as it relies on the HSI clock being available.

Uso:

- This function is part of the low-level driver initialization sequence. It is typically called by a higher-level driver wrapper (like ca_lpuart_init) after GPIO pins have been configured via ca_lpuart_configure_gpio. It is used to prepare the peripheral for asynchronous data reception and to configure the NVIC interrupts. It should be paired with a corresponding de-initialization function (ca_lpuart_deinit) to disable interrupts and clocks when the peripheral is no longer needed, ensuring power efficiency and preventing dangling interrupts.

Esempio:

```
/* Example of initializing LPUART for wake-up functionality */
u32 baud_rate = 9600;
bool enable_wake = true;

/* Ensure GPIOs are set up before hardware init */
if (ca_lpuart_configure_gpio()) {
    if (ca_lpuart_hw_init(baud_rate, enable_wake)) {
        /* Hardware ready, interrupts enabled, ready to receive data */
    } else {
        /* Handle initialization failure (e.g., invalid baud rate or clock
    constraints) */
    }
} else {
    /* Handle GPIO configuration failure */
}
```

Q&A — fine-tuning review *7/7 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_lpuart_hw_init in my caOS application?
A: It should be called during system initialization or from a single control task, typically after GPIO pins have been configured using ca_lpuart_hw_gpio_init, to prepare the peripheral for asynchronous data reception and interrupt handling.

Code example conf:1.00 **Q:** Provide a minimal example of initializing the LPUART at 115200 baud with wake-up enabled.
A: u32 baud = 115200U; bool wake = true; if (ca_lpuart_hw_init(baud, wake)) { /* Success */ } else { /* Handle failure */ }

Parameters conf:1.00 **Q:** What are the constraints on the 'baud' parameter?
A: The baud rate must be non-zero and satisfy the hardware constraint: 3 * baud <= clock_frequency <= 4096 * baud. If these constraints are not met, the function returns false.

Return conf:1.00 **Q:** What does a return value of false indicate?
A: A false return indicates that the hardware configuration failed. This can be caused by an invalid baud rate, failure to meet clock frequency constraints, or the hardware platform not supporting LPUART.

Error/Edge case conf:1.00 **Q:** What is a common mistake when using ca_lpuart_hw_init?
A: A common mistake is failing to verify the return value. If the baud rate is incompatible with the current clock source, the function will fail, and proceeding as if it succeeded will lead to silent communication errors.

Behavior conf:1.00 **Q:** Is ca_lpuart_hw_init safe to call from an Interrupt Service Routine (ISR)?
A: No, it must not be called from an ISR. The function is blocking as it waits for the HSI clock to stabilize and for the LPUART receiver to acknowledge readiness (REACK).

HW differences conf:1.00 **Q:** How does the hardware implementation differ between the H5xx and H7xx families?
A: Both use HSI for kernel clocks, but they target different peripheral buses: H5xx uses APB3 (RCC_APB3ENR_LPUART1EN) and CCIPR3 for clock selection, while H7xx uses APB4 (RCC_APB4ENR_LPUART1EN), D3AMR for autonomous mode, and D3CCIPR for clock selection.

`ca_lpuart_hw_put`

Header: `caos/drivers/inc/ca_glue_hw.h:301`
GUARD: `EN_CA_GLUE_HW`

```
1 extern u16 ca_lpuart_hw_put (p_str message); /* transmits in polling (
     TXE/TC) */
```

Transmits a string of data over the LPUART peripheral using blocking polling mode.

Parametri:

Nome	Tipo	Descrizione
`message`	`p_str`	A pointer to a structure containing the data buffer and its size. Must not be NULL and must contain a valid size; passing an uninitialized structure will lead to undefined behavior.

Ritorna:
Returns the number of bytes successfully transmitted (equal to message->size). Returns 0 if the hardware is not supported, if the peripheral is not in use, or if the input is invalid.

Note:

- This function is strictly blocking, meaning the CPU will sit in a 'while' loop waiting for the hardware transmit buffer to empty for every single byte. Because it uses polling, it is thread-unsafe; if two tasks call this simultaneously, the output will be garbled. It is generally safe to call from an ISR, but doing so is strongly discouraged because the blocking nature will stall the entire system and potentially cause missed interrupts or watchdog resets. Ensure the LPUART peripheral is fully initialized via ca_lpuart_init before calling this, otherwise, the CPU may hang indefinitely in the wait loop.

Uso:

- This function is typically used for low-level debugging or system logging where simplicity is preferred over performance. It is usually called after a successful ca_lpuart_init sequence. In the caOS architecture, it is wrapped by ca_lpuart_put, which adds a safety check for the peripheral state. It is the standard way to send data to a console or CLI interface when the system is in a state where interrupts are not yet enabled or when immediate, synchronous output is required.

Esempio:

```
/* Assuming a message structure is defined and populated */
/* p_str is a pointer to a structure with .data and .size members */

/* Ensure the hardware is ready before transmission */
if (ca_lpuart_init(115200, false)) {
    /* Send the message; this will block until all bytes are sent */
    u16 sent = ca_lpuart_put(&my_msg);

    if (sent == 0) {
        /* Handle potential transmission error or invalid input */
    }

    /* Clean up hardware resources when done */
    ca_lpuart_deinit();
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_lpuart_hw_put in my caOS application?
A: It is intended for low-level debugging or system logging where simplicity is preferred over performance. It is typically used when interrupts are not yet enabled or when immediate, synchronous output is required, such as during early system initialization.

Code example conf:1.00 **Q:** How do I correctly call ca_lpuart_hw_put using caOS types?
A: You must define a structure of type p_str containing your data and size. Example: p_str msg = { .data = (u8*)"Hello", .size = 5 }; ca_lpuart_hw_put(&msg); Ensure ca_lpuart_init has been called first to avoid hanging the CPU.

Parameters conf:1.00 **Q:** What are the requirements for the 'message' parameter in ca_lpuart_hw_put?
A: The parameter must be a pointer to a structure containing a data buffer and a size. It must not be NULL, and the size must be valid. Passing an uninitialized structure will lead to undefined behavior.

Return conf:1.00 **Q:** What do the return values of ca_lpuart_hw_put indicate?
A: It returns the number of bytes successfully transmitted (equal to message->size) upon success. It returns 0 if the hardware is not supported, if the peripheral is not in use, or if the input is invalid.

HW differences conf:1.00 **Q:** Are there differences in how ca_lpuart_hw_put behaves across different STM32 families?
A: Yes. On F1xx, F4xx, and F7xx families, the function is unimplemented and returns 0. On H5xx and U5xx families, the function performs a blocking write to the LPUART1-

>TDR register while polling the LPUART1->ISR register for TXE_TXFNF and TC flags.

---

ca_lpuart_hw_recv

Header: caos/drivers/inc/ca_glue_hw.h:295
GUARD: EN_CA_GLUE_HW

```
extern i32 ca_lpuart_hw_recv (u32 *rx_data); /* 0 se byte valido in *
   rx_data; gestisce/azzera ORE */
```

Polls the LPUART hardware for a single received byte and clears overrun errors.

Parametri:

Nome	Tipo	Descrizione
rx_data	u32 *	Pointer to a memory location where the received byte will be stored. Must be a valid, non-NULL address.

Ritorna:
Returns 0 if a byte was successfully read from the hardware buffer. Returns -1 if no data is available (RX buffer empty) or if the hardware is not initialized.

Note:

- This function is non-blocking; it checks the hardware status register immediately and returns control to the caller. It is thread-safe in the sense that it performs atomic register reads, but it should not be called from multiple threads simultaneously to avoid race conditions on the hardware state. A critical feature is the automatic handling of the Overrun Error (ORE) flag: if the hardware detects that data was lost because the buffer wasn't read fast enough, this function clears the error bit so that future reception can continue. A common mistake is ignoring the return value and assuming the data in 'rx_data' is valid even when the function returns -1.

Uso:

- This function is typically used inside a polling loop or a high-frequency timer callback to drain the LPUART hardware FIFO. It is usually paired with 'ca_lpuart_hw_init' to prepare the peripheral and is called within a dispatcher function like 'ca_lpuart_dispatch' to process incoming bytes. By calling it in a 'while' loop, you ensure that all pending bytes in the hardware buffer are processed before returning to the main task loop.

Esempio:

```
u32 data = 0;
/* Poll the hardware for incoming bytes */
if (ca_lpuart_hw_recv(&data) == 0) {
    /* Byte successfully received, process it */
    process_byte((u8)data);
} else {
    /* No data available, perform other tasks */
    do_something_else();
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_lpuart_hw_recv in my caOS application?
A: You should call it within a polling loop or a high-frequency timer callback to drain the LPUART hardware FIFO. It is typically used inside a dispatcher function to process

incoming bytes, and calling it in a 'while' loop ensures all pending bytes are processed before returning to the main task loop.

Code example conf:1.00 **Q:** How do I implement a simple polling loop to read data using ca_lpuart_hw_recv?
A: You can use the following pattern: u32 data; while (ca_lpuart_hw_recv(&data) == 0) { /* process data */ }

Parameters conf:1.00 **Q:** What are the requirements for the rx_data parameter in ca_lpuart_hw_recv?
A: The rx_data parameter must be a pointer to a valid, non-NULL memory location where the received byte will be stored. Passing a NULL pointer would be invalid as it must be a valid address.

Return conf:1.00 **Q:** What do the return values of ca_lpuart_hw_recv indicate?
A: A return value of 0 indicates a byte was successfully read and stored in the provided pointer. A return value of -1 indicates that no data is available in the RX buffer or the hardware is not initialized. You should never assume the data in the pointer is valid if the function returns -1.

HW differences conf:1.00 **Q:** Are there differences in how ca_lpuart_hw_recv behaves across different STM32 families?
A: Yes. On F1xx, F4xx, and F7xx families, the function is currently unimplemented and returns -1. On H5xx and U5xx families, the function actively manages the LPUART1 peripheral by checking the ISR register for Overrun Errors (ORE), clearing them via the ICR register, and reading the RDR register if the RXNE_RXFNE flag is set.

`ca_lpuart_init`

Header: `caos/drivers/inc/lpuart.h:53`
Source: `caos/drivers/src/lpuart.c:44:53`
GUARD: `EN_LPUART`

```
1 bool ca_lpuart_init (u32 baud, ca_lpuart_rx_cb_t on_rx, bool wake, bool
     alternate_gpio)
```

Initializes the Low Power UART (LPUART) peripheral with a specific baud rate, callback function, and wake-up capability.

Parametri:

Nome	Tipo	Descrizione
`baud`	`u32`	The desired communication speed in bits per second. Must be greater than 0; passing 0 will cause the function to fail immediately.
`on_rx`	`ca_lpuart_rx_cb_t`	A pointer to the callback function that will be executed whenever data is received. This function must be defined by the user.
`wake`	`bool`	A boolean flag indicating if the LPUART should be capable of waking the system from low-power sleep modes upon receiving data.
`alternate_gpio`	`bool`	A boolean flag to select between default or alternate GPIO pin mapping for the LPUART peripheral.

Ritorna:
Returns true if the initialization was successful and the peripheral is ready for use. Returns false if the peripheral is already in use, the baud rate is invalid, GPIO configuration fails, or the hardware initialization fails.

Note:

- This function is not thread-safe and should be called during the system initialization phase before starting the RTOS scheduler or enabling interrupts. It is strictly non-blocking, meaning it returns immediately after attempting to configure the hardware. It is not safe to call this function from an Interrupt Service Routine (ISR) because it performs hardware setup that may involve shared resource access. A common beginner mistake is calling this function multiple times without checking the return value or attempting to re-initialize while the peripheral is already active, which will result in a false return. Always ensure the peripheral is properly shut down using ca_lpuart_deinit before attempting to re-initialize it with different parameters.

Uso:

- This function is typically used at the very beginning of the application lifecycle to prepare the serial communication interface. It is usually paired with a callback function that handles incoming data packets. After calling ca_lpuart_init, the system expects the LPUART to be ready to trigger the provided callback whenever bytes arrive. It is the mandatory first step before any other LPUART operations like ca_lpuart_put or ca_lpuart_dispatch can be performed, ensuring the hardware pins and internal registers are correctly configured for the specific baud rate.

Esempio:

```
/* Define the callback function to handle incoming data */
void my_rx_callback(u8 data) {
    /* Process received byte */
}

/* Initialize LPUART during system startup */
void system_setup(void) {
    /* Initialize with 115200 baud, wake enabled, and alternate GPIO mapping */
    bool success = ca_lpuart_init(115200, my_rx_callback, true, true);
    if (!success) {
        /* Handle initialization error: log or enter safe state */
        /* Use ca_printf to log error if available */
        while(1);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to call ca_lpuart_init in a caOS application?
A: It should be called during the system initialization phase, specifically before starting the RTOS scheduler or enabling interrupts. It is intended to be the mandatory first step before any other LPUART operations are performed.

Code example conf:0.95 **Q:** How do I initialize the LPUART with a 115200 baud rate and wake-up enabled?
A: You must define a callback function and pass it to the initializer: void my_rx_handler(u8 data) { /* handle data */ } ... bool status = ca_lpuart_init(115200, my_rx_handler, true);

Parameters conf:1.00 **Q:** What are the constraints for the baud parameter in ca_lpuart_init?
A: The baud parameter represents the communication speed in bits per second. It must be greater than 0; passing 0 will cause the function to fail immediately.

Return conf:1.00 **Q:** Why might ca_lpuart_init return false?
A: It returns false if the peripheral is already in use, the baud rate is invalid (0), GPIO configuration fails, or the hardware initialization fails.

ca_lpuart_is_busy

Header: caos/drivers/inc/lpuart.h:59
Source: caos/drivers/src/lpuart.c:27:30
GUARD: EN_LPUART

```
bool ca_lpuart_is_busy (void)
```

Checks if the LPUART peripheral is currently occupied by an ongoing transmission or reception process.

Ritorna:
Returns 'true' if the LPUART is currently in use (busy), and 'false' if it is idle and ready for a new operation.

Note:

- This function is non-blocking and thread-safe as it simply reads a static boolean flag. It is safe to call from both main-loop tasks and Interrupt Service Routines (ISRs). The most common beginner mistake is assuming that a 'false' return guarantees the peripheral is powered or configured; you must ensure the LPUART driver has been initialized via ca_lpuart_init before checking this status. If you ignore this check and attempt to start a new transfer while the peripheral is busy, you will likely overwrite the internal buffer or trigger a hardware error, leading to corrupted data or a system hang.

Uso:

- This function is typically used as a guard clause before initiating a new communication sequence to prevent resource contention. In a standard workflow, you call this function to verify the peripheral is free, then proceed to trigger a transmission using ca_lpuart_put. It is frequently paired with ca_lpuart_wake_active to determine if the peripheral is not only busy but also currently holding the system in a wake state. It is also used in power management logic to determine if the MCU can safely enter a low-power sleep mode without aborting an active serial transaction.

Esempio:

```
/* Ensure the LPUART is initialized before use */
ca_lpuart_init();

if (!ca_lpuart_is_busy()) {
    /* Safe to start a new transmission */
    ca_lpuart_put('A');
} else {
    /* Peripheral is busy, wait or retry later */
    ca_os_delay_ms(10);
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_lpuart_is_busy in my application logic?
A: You should call it as a guard clause before initiating a new communication sequence to prevent resource contention. It is also used in power management logic to verify if the MCU can safely enter a low-power sleep mode without aborting an active serial transaction.

Code example conf:0.90 **Q:** How do I safely check the LPUART status before sending data?
A: Ensure you have initialized the driver first, then check the status before calling the transmission function: if (!ca_lpuart_is_busy()) { ca_lpuart_put(data_ptr, length); }

Return conf:1.00 **Q:** What do the return values of ca_lpuart_is_busy indicate, and what happens if I ignore them?
A: A return of 'true' means the LPUART is busy; 'false' means it is idle. If you ignore a 'true' status and attempt a new transfer, you will likely overwrite the internal buffer or trigger a hardware error, resulting in corrupted data or a system hang.

`ca_lpuart_put`

Header: `caos/drivers/inc/lpuart.h:56`
Source: `caos/drivers/src/lpuart.c:65:69`
GUARD: `EN_LPUART`

```
u16 ca_lpuart_put(p_str message)
```

Sends a string message through the LPUART peripheral if the driver is initialized.

Parametri:

Nome	Tipo	Descrizione
`message`	`p_str`	A pointer to the null-terminated string to be transmitted. Must not be NULL.

Ritorna:
Returns the number of bytes successfully queued for transmission (u16). Returns 0 if the driver is not initialized (s_in_use is false) or if the provided message pointer is NULL.

Note:

- This function is a wrapper that checks the internal state of the LPUART driver before attempting hardware access. It is not thread-safe by itself, so ensure that multiple tasks do not attempt to write to the LPUART simultaneously without external synchronization like a mutex. It is generally not safe to call this from an ISR unless the underlying ca_lpuart_hw_put implementation is specifically designed for non-blocking, re-entrant operation. The most common beginner mistake is calling this function before the LPUART driver has been properly initialized, which will result in a silent failure (returning 0) because s_in_use remains false.

Uso:

- This function is typically used in the application layer to send logs, debug information, or command responses to a terminal or external device. It is usually paired with an initialization function that sets s_in_use to true. In a typical workflow, you initialize the peripheral, wait for the system to be ready, and then call this function whenever you need to output data. It is often used as the backend for system console or CLI (Command Line Interface) implementations where the application needs to communicate with a human operator.

Esempio:

```
/* Assuming the LPUART driver has been initialized elsewhere */
p_str my_msg = "System initialized successfully.\r\n";

/* Attempt to send the message */
u16 bytes_sent = ca_lpuart_put(my_msg);

/* Check if the transmission was accepted */
if (bytes_sent == 0) {
    /* Handle error: driver might be busy or not initialized */
    // Log_Error("Failed to send message via LPUART");
```

```
} else {
    /* Message successfully queued */
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_lpuart_put function in my caOS application?
A: This function is used in the application layer to transmit logs, debug information, or command responses to an external device or terminal. It is typically used as the backend for system consoles or CLI implementations after the LPUART driver has been initialized.

Code example conf:1.00 **Q:** Provide a minimal C example showing how to send a message using ca_lpuart_put.
A: p_str my_msg = "System Ready"; u16 sent = ca_lpuart_put(my_msg);

Parameters conf:1.00 **Q:** What are the requirements for the 'message' parameter in ca_lpuart_put?
A: The 'message' parameter must be a p_str (pointer to a null-terminated string) and must not be NULL. Providing a NULL pointer will result in the function returning 0.

Return conf:1.00 **Q:** What does the return value of ca_lpuart_put represent, and why might it return 0?
A: The function returns a u16 representing the number of bytes successfully queued for transmission. It returns 0 if the driver is not initialized (s_in_use is false) or if the provided message pointer is NULL.

`ca_lpuart_std_cli`

Header: `caos/drivers/inc/lpuart.h:59`
Source: `caos/drivers/src/lpuart.c:80:83`
GUARD: `EN_LPUART`

```
bool ca_lpuart_std_cli(bool alternate_gpio)
```

Initializes the Low-Power UART (LPUART) peripheral specifically to act as a Command Line Interface (CLI) input.

Parametri:

Nome	Tipo	Descrizione
`alternate_gpio`	`bool`	If true, configures the LPUART to use the secondary (alternate) pin mapping; if false, uses the default pin mapping.

Ritorna:
Returns true if the LPUART peripheral was successfully initialized and configured for CLI operation, or false if the initialization failed (e.g., invalid baud rate or hardware resource conflict).

Note:

- This function is a high-level wrapper that configures the LPUART hardware to route incoming bytes directly to the system CLI parser. It is not thread-safe and must be called during the system initialization phase before the RTOS scheduler starts. It is strictly non-blocking, but it relies on the underlying hardware being ready; calling it while the LPUART is already active or without proper GPIO configuration will lead to undefined behavior. A common mistake is calling this function multiple times or from within an ISR, which can corrupt the peripheral state or cause a system hang.

Uso:

- This function is typically used during the early boot sequence in the main initialization routine to set up the debug console. It is usually paired with ca_lpuart_configure_gpio() to ensure the physical pins are mapped correctly before the peripheral is enabled. Once initialized, the system expects the LPUART to handle incoming characters automatically via the internal callback, allowing the developer to interact with the system via a terminal emulator connected to the board.

Esempio:

```c
/* Initialize the LPUART for CLI usage during system startup */
void system_init(void) {
    /* Configure the physical pins first using the standard alternate mapping */
    ca_lpuart_configure_gpio(true);

    /* Initialize the LPUART peripheral for CLI input */
    if (ca_lpuart_std_cli(true) == false) {
        /* Handle initialization error: log or halt */
        while(1);
    }

    /* Proceed with other system initializations */
    if (ca_os_tick_init(1000) != 0) {
        /* Handle tick initialization error */
        while(1);
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to call ca_lpuart_std_cli() in my caOS application?
A: This function must be called during the system initialization phase, specifically before the RTOS scheduler starts. It is typically used in the early boot sequence within the main initialization routine to set up the debug console, ideally after calling ca_lpuart_configure_gpio() to ensure correct pin mapping.

Code example conf:0.90 **Q:** How do I implement the LPUART CLI initialization in my main routine?
A: You should configure the GPIO pins first, then initialize the CLI. Example: { ca_lpuart_configure_gpio(); bool status = ca_lpuart_std_cli(); if (status == false) { /* Handle error */ } }

Return conf:1.00 **Q:** What do the return values of ca_lpuart_std_cli() indicate, and what causes a failure?
A: The function returns true if the LPUART peripheral was successfully initialized and configured for CLI operation. It returns false if initialization fails, which can be caused by an invalid baud rate or a hardware resource conflict.

`ca_lpuart_wake_active`

Header: `caos/drivers/inc/lpuart.h:60`
Source: `caos/drivers/src/lpuart.c:32:35`
GUARD: `EN_LPUART`

```c
u32 ca_lpuart_wake_active(void); /* 1 se init con wake=true (per il pwr) */
```

Checks if the LPUART peripheral is currently initialized and configured to act as a system wake-up source.

Ritorna:
Returns 1 if the LPUART is both active and configured for wake-up functionality, otherwise returns 0.

Note:

- This function is non-blocking and thread-safe as it only performs a read operation on internal static status flags. It is safe to call from an ISR, though it is primarily intended for power management logic in the main thread. A common mistake is assuming this function checks if the LPUART is currently receiving data; it only checks the configuration state set during initialization. You must ensure the LPUART driver has been initialized via its setup function before relying on this value, otherwise, it will always return 0.

Uso:

- This function is typically used by the Power Management (PWR) module before the MCU enters a low-power sleep mode. It allows the system to dynamically determine if the LPUART should be included in the wake-up interrupt mask. It is usually paired with initialization functions that set the wake-up flag and is checked during the transition to deep sleep to ensure the system can be awakened by incoming serial traffic.

Esempio:

```c
/* Check if LPUART is a valid wake-up source before entering sleep */
u32 ext_flags = 0;

/* Check if LPUART is active and configured for wake-up */
if (ca_lpuart_wake_active()) {
    /* Add LPUART to the power management wake-up flags */
    ext_flags |= 0x01; /* CA_PWR_WAKE_LPUART equivalent */
}

/* Ensure at least one wake source is active before sleeping */
if (ext_flags != 0) {
    /* Proceed to enter low power mode using ca_pwr_enter */
} else {
    /* Handle error: no wake sources configured, system might not recover */
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_lpuart_wake_active and what is its primary purpose?
A: You should call this function primarily within your power management logic before the MCU enters a low-power sleep mode. Its purpose is to dynamically determine if the LPUART is configured to act as a system wake-up source, allowing the system to decide whether to include the LPUART in the wake-up interrupt mask.

Code example conf:0.95 **Q:** Provide a minimal C example showing how to check the LPUART wake-up status before entering sleep.
A: u32 is_wake_enabled = ca_lpuart_wake_active(); if (is_wake_enabled == 1) { /* Proceed with power management logic for sleep */ }

Return conf:1.00 **Q:** What do the return values of ca_lpuart_wake_active signify, and why might it return 0?
A: The function returns 1 if the LPUART is both active and configured for wake-up functionality, and 0 otherwise. A return value of 0 indicates that the LPUART is either not configured for wake-up or that the LPUART driver has not been initialized via its setup function yet.

ca_mdns_service_bind

Header: caos/net/inc/mdns_service.h:27
Source: caos/net/src/mdns_service.c:149:152
GUARD: EN_MDNS_SERVICE

```
bool ca_mdns_service_bind(void)
```

Registers the mDNS service to listen for incoming UDP packets on the standard mDNS port.

Ritorna:
Returns true if the UDP port was successfully bound to the mDNS handler, or false if the port is already in use or the network stack failed to allocate the socket.

Note:

- This function is non-blocking and performs the low-level registration of the mDNS protocol handler with the network stack. It is not thread-safe in the sense that it should only be called once during the system initialization phase; calling it multiple times may lead to resource exhaustion or undefined behavior in the network stack. It cannot be called from an ISR because it involves memory allocation and internal mutex locking within the network stack. A common beginner mistake is calling this before the network interface is initialized or before the underlying network stack is ready, which will cause the function to return false.

Uso:

- This function is typically used during the system startup sequence after the network stack has been initialized via ca_net_proto_init. It is paired with the mDNS service lifecycle: first, the service is bound using this function, then the system waits for the network link to be established, and finally, the service is notified that the network is ready via ca_mdns_service_net_ready. It is essential to ensure that the MDNS_PORT is not already occupied by another service, otherwise, the binding will fail silently or return false.

Esempio:

```
/* Initialize the network stack first */
ca_net_proto_init(&net_cfg);

/* Bind the mDNS service to the standard port */
if (ca_mdns_service_bind()) {
    /* Success: mDNS is now listening for queries */
} else {
    /* Error: Handle binding failure, perhaps log to CLI */
    ca_stdout_put_n("Error: Failed to bind mDNS service");
}

/* Later, once the network link is up */
ca_mdns_service_net_ready(local_ip_address);
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_mdns_service_bind() in my caOS application, and why is it important to wait until the network is ready?
A: You should call ca_mdns_service_bind() after the network is initialized and the network context is ready. This function binds the MDNS service to listen for UDP packets on MDNS_PORT, which is essential for multicast DNS functionality. Calling it before the network is operational will cause the binding to fail, as there are no available network resources to bind to.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using caOS types that demonstrates how to properly call ca_mdns_service_bind()?
A: u8 result = ca_mdns_service_bind(); if (result) { // MDNS service successfully bound } else { // Handle binding failure }

Return conf:0.90 **Q:** What do the return values of ca_mdns_service_bind() indicate, and what are the common causes of failure?
A: The function returns true if the UDP binding was successful, and false if it failed. Common causes of failure include the port being already occupied or insufficient system resources. The most frequent error occurs when the function is called before the network is ready, which prevents successful binding to the network interface.

`ca_mdns_service_init`

Header: `caos/net/inc/mdns_service.h:24`
Source: `caos/net/src/mdns_service.c:131:147`
GUARD: `EN_MDNS_SERVICE`

```
void ca_mdns_service_init(ca_mac_t local_mac, const char *hostname)
```

Initializes the mDNS service internal structures, semaphores, and CLI interface.

Parametri:

Nome	Tipo	Descrizione
`local_mac`	`ca_mac_t`	The hardware MAC address of the device. Must be a valid 6-byte array structure.
`hostname`	`const char *`	A null-terminated string representing the device name (e.g., 'my-device'). Must not exceed the internal buffer size.

Ritorna:
void

Note:

- This function is non-blocking and performs internal memory setup for the mDNS stack. It is not thread-safe and must be called during the system initialization phase before the RTOS scheduler starts or before any network tasks are spawned. It is strictly forbidden to call this from an Interrupt Service Routine (ISR) as it performs semaphore initialization and potential CLI registration. A common beginner mistake is calling this function multiple times or after the network stack has already started, which can lead to memory corruption or inconsistent state in the semaphore handles.

Uso:

- This function is the mandatory first step in the mDNS lifecycle. It is typically called in the main initialization sequence alongside other network services like DHCP and ARP. It must be followed by a call to ca_mdns_service_start() to actually begin network operations. By initializing the semaphores here, it ensures that the background tasks responsible for handling mDNS queries and announcements have the necessary synchronization primitives ready before the network interface becomes active.

Esempio:

```
/* Define the device hostname */
#define MDNS_HOSTNAME "my-stm32-device"

/* During system startup */
```

```c
void system_init(void) {
    ca_mac_t my_mac = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55};

    /* Initialize mDNS service structures */
    ca_mdns_service_init(my_mac, MDNS_HOSTNAME);

    /* Start the service to begin responding to queries */
    ca_mdns_service_start();
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_mdns_service_init in a caOS application?
A: You should call ca_mdns_service_init during the initialization phase of your caOS application, typically after setting up the network interface but before starting any network services. It is used to initialize the mDNS service with the local MAC address and hostname, which are required for network discovery and service advertisement.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_mdns_service_init with caOS types?
A: u8 mac[6] = {0x00, 0x80, 0xE1, 0x23, 0x45, 0x67}; cstr hostname = "mydevice"; ca_mdns_service_init(mac, hostname);

Parameters conf:0.90 **Q:** What are the valid values for the hostname parameter in ca_mdns_service_init?
A: The hostname parameter must be a non-NULL pointer to a string with a maximum length of 255 characters. If the hostname exceeds this limit, it will be truncated. The string must remain valid for the entire duration of the program.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_mdns_service_init?
A: Common mistakes include passing a NULL pointer for either the local_mac or hostname parameters, using a hostname longer than 255 characters without handling truncation, or calling the function from an ISR. Additionally, if the internal semaphores fail to initialize, subsequent network operations may fail silently.

Behavior conf:0.90 **Q:** Is ca_mdns_service_init blocking or non-blocking, and is it safe to call from an ISR?
A: The function is non-blocking. It is not safe to call ca_mdns_service_init from an ISR because it may attempt to create semaphores and perform other operations that are not ISR-safe. It is also not thread-safe unless properly synchronized by the caller.

`ca_mdns_service_net_ready`

Header: `caos/net/inc/mdns_service.h:30`
Source: `caos/net/src/mdns_service.c:154:159`
GUARD: `EN_MDNS_SERVICE`

```c
void ca_mdns_service_net_ready(ca_ipv4_t local_ip)
```

Signals the mDNS service that the network interface is configured and ready to begin service announcements.

Parametri:

Nome	Tipo	Descrizione
`local_ip`	`ca_ipv4_t`	The IPv4 address assigned to the device. Must be a valid, non-zero IP address obtained via DHCP or static configuration.

Ritorna:
void

Note:

- This function is non-blocking and thread-safe, designed to be called once the network stack has successfully acquired an IP address. It updates the internal state of the mDNS module and triggers the semaphore that controls the announcement cycle. It is not intended to be called from an ISR (Interrupt Service Routine) because it performs semaphore operations that could lead to undefined behavior if the scheduler is not ready. A common mistake is calling this function before the network stack is fully initialized or before an IP address has been assigned, which results in the mDNS service attempting to announce an invalid or null IP address.

Uso:

- This function is typically used in the network initialization sequence, specifically after a DHCP lease is granted or a static IP is applied. It acts as the 'go' signal for the mDNS module to start broadcasting its presence on the local network. It is usually paired with ca_mdns_service_bind() which sets up the UDP socket. In a typical application, you would call ca_mdns_service_bind() during system startup, and then call ca_mdns_service_net_ready() only after the network link is confirmed and an IP address is assigned.

Esempio:

```c
/* Example: Handling network readiness after DHCP */
void on_network_up(ca_ipv4_t ip) {
    /* 1. Bind the UDP port for mDNS traffic */
    if (ca_mdns_service_bind()) {
        /* 2. Notify mDNS that we have a valid IP and can start announcing */
        ca_mdns_service_net_ready(ip);
    } else {
        /* Handle binding error */
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_mdns_service_net_ready() and why is it important for mDNS functionality?
A: You should call ca_mdns_service_net_ready() when the network interface is ready and has been assigned a valid IPv4 address. This function is important because it notifies the mDNS service that the network is ready, sets the local IP address, and enables the announcement of services on the network. It must be called with a valid IP address to ensure proper mDNS operation.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_mdns_service_net_ready() with caOS types?
A: u32 ip = 0xC0A80164; /* 192.168.1.100 in big-endian format */ ca_mdns_service_net_ready(ip);

Parameters conf:0.90 **Q:** What is the significance of the local_ip parameter in ca_mdns_service_net_ready(), and what are valid vs invalid values?
A: The local_ip parameter specifies the IPv4 address assigned to the network interface. Valid values are valid IPv4 addresses such as 192.168.1.100 (represented as 0xC0A80164 in big-endian format). Invalid values include 0 or any non-routable IP address, which can cause unpredictable behavior in the mDNS service.

`ca_mdns_service_rx_handler`

Header: `caos/net/inc/mdns_service.h:33`
Source: `caos/net/src/mdns_service.c:161:168`

GUARD: `EN_MDNS_SERVICE`

```
void ca_mdns_service_rx_handler(const ca_udp_frame_t *udp, u16 len)
```

Processes incoming mDNS UDP packets to detect queries targeting the device's hostname.

Parametri:

Nome	Tipo	Descrizione
`udp`	`const ca_udp_frame_t *`	Pointer to the raw UDP frame structure extracted from the network buffer. Must be a valid, non-NULL pointer to a properly formatted UDP packet.
`len`	`u16`	The total length of the UDP payload in bytes. While passed to the function, it is currently ignored in the implementation.

Ritorna:
void. This function does not return a status code; it performs internal state updates via semaphores if a matching query is found.

Note:

- This function is designed to be called from the network stack's receive interrupt context or a high-priority network task. It is thread-safe in the sense that it interacts with semaphores, but it assumes the mDNS service has been properly initialized with a valid hostname. A common mistake is calling this function before the network stack is fully initialized or with a pointer that does not point to a valid UDP frame, which will cause memory access violations. Because it uses semaphore operations, ensure that the semaphore 's_query_sem' is correctly initialized before any packets are processed, otherwise the system may hang or trigger an assertion failure.

Uso:

- This function is the entry point for mDNS query processing within the caOS network stack. It is typically invoked by the low-level network driver or the buffer management layer immediately after a UDP packet is received and identified as an mDNS packet. It works in tandem with 'ca_net_buf_free' to ensure memory is released after processing, and it signals the 's_query_sem' semaphore to wake up the mDNS background task, which then prepares and sends the appropriate response.

Esempio:

```
/* Inside the network receive callback */
void on_packet_received(u16 buf_idx) {
    /* Extract UDP frame from buffer metadata */
    const ca_udp_frame_t *udp = get_udp_frame_from_buffer(buf_idx);
    u16 len = get_udp_len_from_buffer(buf_idx);

    /* Process the mDNS query */
    ca_mdns_service_rx_handler(udp, len);

    /* Always free the buffer after processing to prevent memory leaks */
    ca_net_buf_free(buf_idx);
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should ca_mdns_service_rx_handler be called, and why is it important in an MDNS service implementation?

A: ca_mdns_service_rx_handler should be called as a callback from the network system when an MDNS UDP packet is received. It is important because it checks if the packet is intended for this device and signals a semaphore to indicate that a response is needed. This allows the MDNS service to react to incoming queries without blocking the network interrupt handler.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_mdns_service_rx_handler with caOS types?
A: void example_usage() { ca_udp_frame_t frame; u16 length = 100; ca_mdns_service_rx_handler(& length); }

Parameters conf:0.85 **Q:** What is the significance of the len parameter in ca_mdns_service_rx_handler, and what are valid and invalid values for it?
A: The len parameter specifies the length of the UDP packet received. Although it is not used directly in the function, it is required for data validation. Valid values are any u16 greater than 0. If len is 0 or negative, the function behavior is undefined and may cause issues, though the function itself does not explicitly check for this condition.

`ca_mdns_service_start`

Header: `caos/net/inc/mdns_service.h:36`
Source: `caos/net/src/mdns_service.c:170:174`
GUARD: `EN_MDNS_SERVICE`

```
void ca_mdns_service_start(void)
```

Initializes and launches the mDNS background service task in the RTOS scheduler.

Ritorna:
This function returns void; it does not provide feedback on whether the task creation succeeded or failed.

Note:

- This function is non-blocking as it merely registers a new task with the RTOS scheduler. It is not thread-safe to call this from an ISR, as it involves memory allocation and scheduler manipulation. You must ensure that the underlying network stack and hardware drivers are initialized before calling this, otherwise, the mDNS task will immediately crash or hang. A common beginner mistake is calling this function multiple times during runtime; since it spawns a new thread each time, this will quickly exhaust your system memory and lead to a stack overflow or system instability.

Uso:

- This function is typically called once during the system initialization phase, usually inside the main entry point or a dedicated network configuration routine. It is part of the standard service startup sequence, often paired with other network services like DHCP or ARP to establish connectivity. Once started, the mDNS task runs autonomously in the background, handling incoming UDP queries and responding to service discovery requests without further intervention from the main application loop.

Esempio:

```
void ca_os_main(void) {
    // Initialize network services during system boot
    ca_arp_service_start();
    ca_dhcp_service_start();

    // Start the mDNS service to allow device discovery on the network
```

```
    ca_mdns_service_start();

    // Proceed to start other application-level tasks
    ca_os_job("app_task", CA_TH_LEVEL2, my_app_task, 0, STACK_SMALL, CA_JOB_STD,
     CA_FAULT_RESTART_THREAD);
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When and why should I call ca_mdns_service_start() in a caOS application?
A: You should call ca_mdns_service_start() after initializing the network stack in your caOS application. This function starts a task that handles DNS-SD queries on the network. It is non-blocking and must be called from a thread context, not an ISR. The function is thread-safe and does not return an error code, but calling it before the network is ready can cause undefined behavior or crashes.

Code example conf:0.90 **Q:** Can you provide a minimal C example using caOS types that demonstrates how to start the mDNS service?
A: void example_mdns_start(void) { // Initialize network first ca_net_init(); // Start mDNS service task ca_mdns_service_start(); }

Return conf:0.85 **Q:** What does ca_mdns_service_start() return and what causes failures in its execution?
A: The function does not return any value. It is declared as void. It does not directly report errors, but the underlying task it starts may not function correctly if network resources are not available. The most common cause of failure is calling this function before the network is initialized, which can lead to crashes or undefined behavior.

`ca_measure_start`

Header: `caos/inc/measure.h:17`
Source: `caos/src/measure.c:18:23`
GUARD: `EN_MEASURE`

```
void ca_measure_start(u32 *p_start)
```

Captures the current system runtime tick count to a provided memory address for performance profiling.

Parametri:

Nome	Tipo	Descrizione
`p_start`	`u32 *`	A pointer to a 32-bit unsigned integer where the current system runtime value will be stored. Must not be NULL.

Ritorna:
void. The function does not return a status code; it directly modifies the memory location pointed to by p_start.

Note:

- This function is non-blocking and extremely fast, as it simply reads a global counter maintained by the scheduler. It is thread-safe in the sense that it performs a single atomic read of the runtime variable, making it safe to call from both tasks and Interrupt Service Routines (ISRs). The most common mistake is passing a NULL pointer, which the function handles safely by doing nothing, but this will result in your timing variable remaining uninitialized. Ensure the variable passed is accessible in the scope where you intend to calculate the duration, as it will be overwritten with the start timestamp.

Uso:

- This function is used exclusively for benchmarking and profiling the execution time of code blocks or functions. It is always used in a pair with ca_measure_stop, where ca_measure_start captures the 'before' timestamp and ca_measure_stop calculates the difference between the current time and that stored timestamp. You typically initialize a local u32 variable, pass its address to this function before your target code, and then pass the same address to the stop function to update the variable with the elapsed time.

Esempio:

```
u32 execution_time; // Variable to store the duration

while (1) {
    // Capture the start time before the task begins
    ca_measure_start(&execution_time);

    // Perform the work to be measured
    do_heavy_processing();

    // Calculate the elapsed time by passing the same pointer
    // Note: ca_measure_stop is the required pair for this function
    ca_measure_stop(&execution_time);

    // execution_time now holds the duration in system ticks
    ca_os_sleep(100);
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_measure_start() in my embedded application, and what is its purpose?
A: You should call ca_measure_start() at the beginning of a function or code section that you want to profile for execution time. Its purpose is to record the current time, which will later be used in combination with ca_measure_stop() to calculate how long the code took to execute. It is useful for performance optimization and debugging timing issues in real-time systems.

Code example conf:0.85 **Q:** Can you provide a minimal C code example using ca_measure_start() with caOS types?
A: u32 start_time; cal_measure_start(&start_time); // Some code to measure ca_measure_stop(&star

Parameters conf:0.90 **Q:** What is the significance of the u32 *p_start parameter in ca_measure_start(), and what happens if it is NULL?
A: The u32 *p_start parameter is a pointer to a 32-bit unsigned integer where the start time will be stored. If this pointer is NULL, the function does nothing and does not modify any value. Passing a valid pointer is essential for the function to work correctly and store the measurement start time.

`ca_measure_stop`

Header: `caos/inc/measure.h:18`
Source: `caos/src/measure.c:25:32`
GUARD: `EN_MEASURE`

```
void ca_measure_stop(volatile u32 *p)
```

Calculates the elapsed time in system ticks between a start point and the current moment.

Parametri:

Nome	Tipo	Descrizione
p	volatile u32 *	A pointer to a variable that previously stored the start timestamp via ca_measure_start. Must not be NULL.

Ritorna:
This function returns void. The result of the calculation is written directly into the memory address pointed to by p.

Note:

- This function is non-blocking and executes very quickly, making it safe to use in most contexts. It is thread-safe provided that the variable pointed to by p is not accessed by other tasks simultaneously. It is safe to call from an ISR, but be aware that the accuracy depends on the system tick frequency. The most common beginner mistake is passing a NULL pointer or a pointer that was not initialized by ca_measure_start, which will result in an incorrect calculation or a memory access violation. Always ensure the variable passed is declared as volatile to prevent the compiler from optimizing away necessary memory updates.

Uso:

- This function is used for profiling code execution time or measuring the duration of specific tasks within the RTOS. It is always used in a pair: first, you call ca_measure_start to capture the initial system runtime, then you execute the code block you wish to measure, and finally, you call ca_measure_stop to overwrite the initial value with the calculated delta. It is typically used during the debugging or optimization phase of development to identify bottlenecks in your application logic.

Esempio:

```
u32 elapsed = 0;

// Initialize the start time
ca_measure_start(&elapsed);

// Perform the task to be measured
perform_heavy_calculation();

// Calculate the duration and store it back in 'elapsed'
ca_measure_stop(&elapsed);

// Now 'elapsed' contains the number of ticks taken by perform_heavy_calculation
    ()
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_measure_stop and what is its purpose in an RTOS environment?
A: You should call ca_measure_stop to calculate the elapsed time between a start and end measurement, which is useful for monitoring operation durations in an RTOS. It is non-blocking and does not return a value; instead, it writes the result directly into the variable pointed to by the parameter. This function is ideal for performance analysis or timing critical sections of code without interrupting thread execution.

Code example conf:0.90 **Q:** Can you provide a minimal C example using caOS types that demonstrates how to use ca_measure_stop?
A: u32 start_time = 1000; // Initialize with a start value volatile u32 *p = &start_time; ca_measure_stop(p); // The value at start_time is now updated with the elapsed time.

Parameters conf:0.90 **Q:** What is the significance of the volatile u32 *p parameter in ca_measure_stop, and what happens if it is NULL?

A: The parameter 'p' is a pointer to a volatile u32 variable that holds the start time of the measurement. If 'p' is NULL, the function does nothing. The volatile keyword ensures that the compiler does not optimize accesses to this variable, which is important in embedded systems where memory might be modified by hardware or other threads. The variable pointed to must be initialized with a valid start time before calling this function.

`ca_memcmp`

Header: `caos/inc/memaux.h:63`
Source: `caos/src/memaux.c:63:85`
GUARD: `EN_MEMAUX`

```
i32 ca_memcmp(const void *ptr1, const void *ptr2, u32 count)
```

Compares two memory blocks byte-by-byte for equality or difference.

Parametri:

Nome	Tipo	Descrizione
`ptr1`	`const void *`	Pointer to the first memory block. Must not be NULL.
`ptr2`	`const void *`	Pointer to the second memory block. Must not be NULL.
`count`	`u32`	Number of bytes to compare. If 0, the function returns 0 immediately.

Ritorna:
Returns 0 if the blocks are identical. Returns a non-zero value if a difference is found (the difference between the first non-matching bytes). Returns -1 if ptr1 is NULL, or -2 if ptr2 is NULL.

Note:

- This function is non-blocking and executes synchronously, meaning it consumes CPU cycles proportional to the 'count' parameter. It is thread-safe as it does not modify any global state or shared resources, only reading from the provided pointers. It is safe to call from an ISR, provided the memory addresses are valid and accessible. The most common beginner mistake is passing an incorrect 'count' value, which can lead to reading past the end of a buffer (buffer over-read) if the pointers are not pointing to sufficiently large memory regions.

Uso:

- Use this function whenever you need to verify data integrity, such as checking if a received network packet matches an expected header, or validating that decrypted data matches the original plaintext. It is frequently paired with cryptographic functions like 'ca_aes_decrypt' to verify success, or with string-handling logic where buffers are not null-terminated (e.g., MQTT payloads). Unlike standard string functions, it does not stop at a null terminator, making it ideal for binary data or fixed-length structures.

Esempio:

```
u8 expected_data[] = {0xDE, 0xAD, 0xBE, 0xEF};
u8 received_data[4];

// Assume received_data is populated via a peripheral read
if (ca_memcmp(expected_data, received_data, 4) == 0) {
    // Data matches exactly
```

```
} else {
    // Data mismatch detected, handle error
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_memcmp in an embedded RTOS like caOS?
A: You should use ca_memcmp when you need to compare two memory blocks byte by byte, such as verifying data integrity, checking if two buffers are identical, or comparing structures in an embedded system. It's useful in scenarios where you must ensure memory contents match without using higher-level libraries.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_memcmp with caOS types?
A: u8 buf1[5] = {1, 2, 3, 4, 5}; u8 buf2[5] = {1, 2, 3, 4, 5}; i32 result = ca_memcmp(buf1, buf2, 5); // result will be 0 since the buffers are identical.

Parameters conf:0.90 **Q:** What does the 'count' parameter in ca_memcmp control, and what are valid values?
A: The 'count' parameter specifies how many bytes to compare between the two memory blocks. Valid values are positive integers. If count is 0, the function returns 0 immediately. Negative or excessively large values may cause undefined behavior or memory access violations.

Return conf:0.95 **Q:** What do the different return values of ca_memcmp indicate?
A: A return value of 0 means the memory blocks are identical for the first 'count' bytes. A positive value indicates the first differing byte in ptr1 is greater than in ptr2. A negative value indicates the opposite. Return values -1 or -2 indicate errors: -1 for a NULL ptr1, and -2 for a NULL ptr2.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_memcmp?
A: Common mistakes include passing NULL pointers to ptr1 or ptr2, which results in return codes -1 or -2 respectively. Not checking the return value can lead to incorrect logic. Using a very large count may cause memory access violations or undefined behavior. Also, calling ca_memcmp with a count of 0 is valid but may not be intended in all use cases.

Behavior conf:0.90 **Q:** Is ca_memcmp blocking or non-blocking, and can it be safely used in an ISR?
A: ca_memcmp is a blocking function, as it waits until all bytes are compared or a difference is found. It is thread-safe since it does not modify data or use global variables. It can be called from an ISR, but only if the count is not too large and the context does not require non-blocking behavior to avoid potential system delays.

`ca_memcpy`

Header: `caos/inc/memaux.h:37`
Source: `caos/src/memaux.c:28:42`
GUARD: `EN_MEMAUX`

```
void* ca_memcpy(volatile void *dest, const void *src, u32 count)
```

Copies a specified number of bytes from a source memory area to a destination memory area.

Parametri:

Nome	Tipo	Descrizione
`dest`	`volatile void *`	Pointer to the destination memory buffer. Must be large enough to hold 'count' bytes. If NULL, the function returns immediately without performing any copy.
`src`	`const void *`	Pointer to the source memory buffer. Must contain at least 'count' bytes. If NULL, the function returns immediately without performing any copy.
`count`	`u32`	The number of bytes to copy. A value of 0 results in no operation.

Ritorna:
Returns the original 'dest' pointer. If either 'dest' or 'src' is NULL, it returns the 'dest' pointer unchanged.

Note:

- This function is a non-blocking, synchronous byte-by-byte copy utility. It is thread-safe in the sense that it does not modify global state, but it is NOT re-entrant if the same memory regions are accessed by other threads simultaneously. It is safe to call from an ISR because it does not use OS primitives, locks, or blocking delays. A common beginner mistake is failing to ensure the destination buffer is large enough to hold the data, which leads to buffer overflows and memory corruption. Unlike standard C 'memcpy', this implementation does not handle overlapping memory regions; if 'src' and 'dest' overlap, the data will be corrupted.

Uso:

- Used extensively for preparing data structures, serializing buffers, or populating communication packets. It is typically used in a sequence where a buffer is allocated, populated using one or more 'ca_memcpy' calls to assemble different fields (like IVs or headers), and then passed to a hardware peripheral or crypto engine. It is frequently paired with buffer management logic where you calculate offsets (e.g., 'buffer + offset') to concatenate multiple data segments into a single contiguous memory block.

Esempio:

```
/* Example: Assembling a packet header and payload */
#define HEADER_SIZE 4
#define PAYLOAD_SIZE 8

u8 packet[HEADER_SIZE + PAYLOAD_SIZE];
u8 header[HEADER_SIZE] = {0xAA, 0xBB, 0xCC, 0xDD};
u8 data[PAYLOAD_SIZE] = {0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08};

/* Copy header to the start of the packet */
ca_memcpy(packet, header, HEADER_SIZE);

/* Copy payload to the offset after the header */
ca_memcpy(packet + HEADER_SIZE, data, PAYLOAD_SIZE);

/* Always ensure the destination buffer is large enough to avoid memory
   corruption */
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_memcpy in a caOS project, especially in network-related modules?

A: You should use ca_memcpy to copy data between memory buffers, particularly in network protocol implementations where you need to move data from incoming buffers to processing or output buffers. It's commonly used in modules like HTTP, Modbus, and MQTT services to handle data transfer without relying on HAL or standard library functions.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_memcpy with caOS types?
A: u8 src[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}; u8 dest[10]; ca_memcpy(dest, src, 10); // Now dest contains the same data as src.

Parameters conf:0.85 **Q:** What are the valid and invalid values for the 'count' parameter in ca_memcpy?
A: The 'count' parameter specifies the number of bytes to copy. Valid values range from 0 to the maximum value of u32. If count is 0, the function does nothing. Invalid values are not applicable, but passing a very large count may cause performance issues or block execution for extended periods.

Error/Edge case conf:0.90 **Q:** What are the most common mistakes when using ca_memcpy in caOS?
A: Common mistakes include passing NULL pointers for 'dest' or 'src', which leads to undefined behavior. Another frequent error is overlapping memory regions (e.g., memcpy(dest, dest, count)), which can cause data corruption. Additionally, not ensuring sufficient space in the destination buffer can lead to memory overruns.

`ca_memcpy_u16`

Header: `caos/inc/memaux.h:99`
Source: `caos/src/memaux.c:125:138`
GUARD: `EN_MEMAUX`

```
1 void* ca_memcpy_u16(volatile u16 *dest, const u16 *src, u32 count)
```

Copies a specified number of 16-bit words from a source memory address to a destination memory address.

Parametri:

Nome	Tipo	Descrizione
`dest`	`volatile u16 *`	Pointer to the destination memory buffer. Must be a valid, non-NULL address. If NULL, the function returns immediately without performing any copy.
`src`	`const u16 *`	Pointer to the source memory buffer. Must be a valid, non-NULL address. If NULL, the function returns immediately.
`count`	`u32`	The number of 16-bit elements (u16) to copy. A value of 0 results in no operation.

Ritorna:
Returns the original 'dest' pointer. If either 'dest' or 'src' is NULL, it returns the original 'dest' pointer unchanged, effectively doing nothing.

Note:

- This function is non-blocking and executes synchronously, meaning the CPU will stay in this function until the entire copy is complete. It is thread-safe in the sense that it does not rely on global state, but it does not provide any protection against race conditions

if another thread or ISR modifies the source or destination buffers while the copy is in progress. It is safe to call from an ISR, provided the memory addresses are accessible. A common beginner mistake is passing the count in bytes instead of 16-bit words, which will result in only half the data being copied or potential buffer overflows if the count is misinterpreted.

Uso:

- This function is typically used for high-performance memory transfers between buffers, such as moving data from a peripheral's memory-mapped buffer to a local application buffer. It is often used in drivers or communication stacks where data alignment is guaranteed to be 16-bit or 32-bit. Developers use this when they need a faster alternative to the standard C 'memcpy' that is specifically optimized for 16-bit word alignment and potentially 32-bit word access patterns on STM32 hardware. It is usually called after a buffer has been allocated or a DMA transfer has completed, ensuring that the data is ready to be processed by the application logic.

Esempio:

```
u16 source_buffer[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
u16 destination_buffer[10];

// Ensure buffers are valid before copying
if (destination_buffer != NULL && source_buffer != NULL) {
    // Copy 10 elements of 16-bit size
    ca_memcpy_u16(destination_buffer, source_buffer, 10);
} else {
    // Handle error: pointers are invalid
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_memcpy_u16 instead of a standard memcpy function in caOS?
A: You should use ca_memcpy_u16 when working with 16-bit data on STM32 microcontrollers where performance and alignment are critical. It is optimized for 16-bit word operations and ensures proper alignment for better memory access performance. It is especially useful in memory-intensive tasks where you are copying arrays of u16 values and need guaranteed alignment for optimal hardware performance.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_memcpy_u16 with caOS types?
A: volatile u16 destination[10]; const u16 source[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}; void* result = ca_memcpy_u16(destination, source, 10); // result now points to destination, and destination contains the copied data

Parameters conf:0.90 **Q:** What are the valid alignment requirements for the dest and src pointers in ca_memcpy_u16?
A: Both dest and src pointers must be aligned to 32-bit boundaries for optimal performance. While the function does not strictly enforce this, misalignment can lead to crashes or undefined behavior. The pointers must not be NULL, and the count must be a positive value.

Error/Edge case conf:0.95 **Q:** What happens if I pass NULL as either dest or src to ca_memcpy_u16?
A: If either dest or src is NULL, the function immediately returns dest without performing any copy. This is a safety check to prevent access violations, but it does not indicate an error condition; it simply returns early.

---

ca_memcpy_u32

Header: caos/inc/memaux.h:81
Source: caos/src/memaux.c:98:107
GUARD: EN_MEMAUX

```
void* ca_memcpy_u32(volatile u32 *dest, const u32 *src, u32 count)
```

Optimized memory copy function that transfers data in 32-bit chunks for improved performance.

Parametri:

Nome	Tipo	Descrizione
dest	volatile u32 *	Pointer to the destination memory area. Must be 32-bit aligned; passing an unaligned address may cause hardware faults on some ARM Cortex-M cores.
src	const u32 *	Pointer to the source memory area. Must be 32-bit aligned.
count	u32	Number of 32-bit words to copy. If the total data size is not a multiple of 4 bytes, the remaining bytes must be handled separately.

Ritorna:
Returns the updated destination pointer (dest + count), pointing to the address immediately following the last copied word. If either pointer is NULL, it returns the original destination pointer without performing any copy.

Note:

- This function is non-blocking and executes synchronously, meaning the CPU will wait until the copy is complete before moving to the next instruction. It is thread-safe only if the memory regions being accessed are not modified by other threads or ISRs simultaneously. It is safe to call from an ISR, provided the pointers are valid and aligned. The most common beginner mistake is passing a byte count instead of a word count, or providing unaligned pointers, which can lead to hard faults or data corruption on architectures that strictly enforce alignment.

Uso:

- Use this function when you need to move large blocks of data, such as network packets or flash memory buffers, where performance is critical. It is typically used in a sequence where you first calculate the number of 32-bit words (data_len » 2), call this function to handle the bulk of the data, and then use a simple loop to copy any remaining bytes (data_len & 3). It is often paired with memory allocation functions like ca_memory_allocate to prepare destination buffers before processing data.

Esempio:

```
/* Example: Copying a network packet payload */
#define NET_BUF_HEADROOM 16

void process_packet(u8 *data, u16 data_len) {
    u32 *d32 = (u32 *)(buffer + NET_BUF_HEADROOM);
    const u32 *s32 = (const u32 *)data;
    u16 word_count = data_len >> 2;

    /* Perform bulk copy of 32-bit words */
    ca_memcpy_u32(d32, s32, (u32)word_count);
```

```
    /* Handle remaining bytes if data_len is not a multiple of 4 */
    u8 *d8 = (u8 *)(d32 + word_count);
    const u8 *s8 = (const u8 *)(s32 + word_count);
    for (u16 k = 0; k < (data_len & 3); k++) {
        d8[k] = s8[k];
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_memcpy_u32 instead of a standard memcpy in an embedded system?
A: You should use ca_memcpy_u32 when copying data in embedded systems where performance is critical and memory is not cacheable, such as in SRAM. It copies data in 32-bit chunks which improves performance compared to byte-wise copying. It is especially useful when working with hardware registers or memory-mapped peripherals where alignment and speed matter.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_memcpy_u32 with caOS types?
A: volatile u32 dest[4] = {0}; const u32 src[4] = {0x12345678, 0x9ABCDEF0, 0x11111111, 0x22222222}; void* result = ca_memcpy_u32(dest, src, 4); /* result now points to dest + 4 */

Parameters conf:0.90 **Q:** What happens if I pass a count of zero to ca_memcpy_u32, and what are the valid values for the dest and src pointers?
A: If count is zero, ca_memcpy_u32 does nothing and returns the dest pointer. The dest and src pointers must be valid and non-NULL. They must also be aligned to 32-bit boundaries. Passing NULL for either pointer will cause the function to return dest without performing any copy.

`ca_memmap_get`

Header: `caos/inc/memory.h:230`
Source: `caos/src/memory.c:871:875`
GUARD: `EN_MEMORY`

```
u32 ca_memmap_get(ca_mem_range *out)
```

Retrieves the system memory map, providing addresses and sizes of available memory regions.

Parametri:

Nome	Tipo	Descrizione
`out`	`ca_mem_range *`	Pointer to an array of ca_mem_range structures where the memory map will be stored. Must be large enough to hold CA_MEMMAP_MAX entries. Passing NULL will result in an immediate return of 0.

Ritorna:
Returns the number of memory regions successfully populated in the provided array. Returns 0 if the input pointer is NULL or if no memory regions are defined.

Note:

- This function is non-blocking and thread-safe, as it simply copies pre-defined hardware constants into the provided buffer. It is safe to call from an ISR, provided the buffer

passed is not being accessed by another task simultaneously. The most common mistake is providing a pointer to an array smaller than CA_MEMMAP_MAX, which will cause a buffer overflow if the system has more regions than the array can hold. Ensure the array is allocated in a scope that remains valid during the function call.

Uso:

- This function is typically used for system diagnostics, debugging, or CLI commands that need to report the memory layout to the user. It is often paired with CLI output functions to display the start address, end address, and size of each memory block. It is usually called after the system has initialized, as it relies on the hardware-specific memory configuration defined during the board bring-up phase.

Esempio:

```
void display_memory_map(void) {
    ca_mem_range map[CA_MEMMAP_MAX];
    u32 count = ca_memmap_get(map);

    if (count == 0) {
        // Log error: No memory map available
        return;
    }

    for (u32 i = 0; i < count; i++) {
        // Use ca_printf to display the memory region details
        // Format: Name, Start Address, Size
        ca_printf("Region %u: %s at 0x%08X, Size: %u bytes\n",
                  i, map[i].name, map[i].addr, map[i].size);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_memmap_get and what is its primary purpose?
A: You should call this function after the system has initialized, as it relies on hardware-specific memory configurations defined during board bring-up. It is primarily used for system diagnostics, debugging, or CLI commands that need to report the memory layout to the user.

Code example conf:1.00 **Q:** How do I correctly implement a call to ca_memmap_get in my code?
A: You must provide an array large enough to hold CA_MEMMAP_MAX entries. Example: ca_mem_range map[CA_MEMMAP_MAX]; u32 count = ca_memmap_get(map);

Parameters conf:1.00 **Q:** What are the requirements for the 'out' parameter in ca_memmap_get?
A: The 'out' parameter must be a pointer to an array of ca_mem_range structures. It must be large enough to hold CA_MEMMAP_MAX entries to avoid buffer overflows. Passing NULL will cause the function to return 0 immediately.

Return conf:1.00 **Q:** What do the return values of ca_memmap_get indicate?
A: The function returns the number of memory regions successfully populated in the provided array. It returns 0 if the input pointer is NULL or if no memory regions are defined.

`ca_memory_adopt`

Header: `caos/inc/memory.h:73`
Source: `caos/src/memory.c:423:446`
GUARD: `EN_MEMORY`

```
bool ca_memory_adopt(void *p)
```

Transfers ownership of a shared memory block to the current task.

Parametri:

Nome	Tipo	Descrizione
p	void *	Pointer to the memory block to adopt. Must be a valid pointer previously allocated via ca_memory_allocate_shared.

Ritorna:
Returns true if the memory block was successfully adopted by the current task, false if the pointer is invalid, outside heap bounds, or not currently in a BUSY state.

Note:

- This function is thread-safe and uses internal locking to safely update the memory block's owner metadata. It is NOT callable from an ISR because it interacts with the scheduler to identify the current task context. The most common mistake is passing a pointer that was not allocated via ca_memory_allocate_shared or attempting to adopt memory that has already been freed. If you fail to adopt a shared block, the system may leak memory or trigger an error when the original owner attempts to free it.

Uso:

- Use this function when a task receives a pointer to a shared memory buffer from another task or a service (e.g., via a queue). By calling ca_memory_adopt, the receiving task becomes the new 'owner' of that memory, allowing it to safely free the memory later using ca_memory_free. This is essential in producer-consumer patterns where the producer allocates the buffer and the consumer takes responsibility for its lifecycle.

Esempio:

```
void task_consumer(void) {
    void *shared_buf = NULL;
    // Wait for a message containing a pointer to shared memory
    // ca_queue_heap_receive is used to get the pointer from a queue
    if (ca_queue_heap_receive(my_queue, &shared_buf, 1000)) {
        // Take ownership of the buffer so we can free it later
        if (ca_memory_adopt(shared_buf)) {
            // Process the data in the buffer
            // ...
            // Now we are responsible for freeing it
            ca_memory_free(shared_buf);
        } else {
            // Handle adoption failure
            ca_memory_free(shared_buf);
        }
    }
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_memory_adopt in my application?
A: You should use it when a task receives a pointer to a shared memory buffer from another task or service, such as via a queue. It transfers ownership to the receiving task, allowing it to safely free the memory later.

Code example conf:0.90 **Q:** Provide a minimal C example of how to adopt a shared memory block.
A: void *ptr = queue_receive(); if (ca_memory_adopt(ptr)) { /* Process data */ ca_memory_free(ptr); }

Parameters conf:1.00 **Q:** What are the requirements for the pointer passed to ca_memory_adopt?
A: The pointer must be a valid address previously allocated via ca_memory_allocate_shared. Passing pointers not allocated this way will result in failure.

Return conf:1.00 **Q:** What do the return values of ca_memory_adopt indicate?
A: It returns true if the memory block was successfully adopted by the current task. It returns false if the pointer is invalid, outside heap bounds, or not currently in a BUSY state.

Error/Edge case conf:1.00 **Q:** What are the common mistakes when using ca_memory_adopt?
A: The most common mistakes are passing a pointer that was not allocated via ca_memory_allocate_sha or attempting to adopt memory that has already been freed. Failure to adopt correctly can lead to memory leaks or errors when the original owner attempts to free it.

Behavior conf:1.00 **Q:** Is ca_memory_adopt safe to call from an Interrupt Service Routine (ISR)?
A: No, it is not callable from an ISR because it interacts with the scheduler to identify the current task context. However, it is thread-safe and uses internal locking for task-level operations.

`ca_memory_allocate`

Header: `caos/inc/memory.h:51`
Source: `caos/src/memory.c:399:401`
GUARD: `EN_MEMORY`

```
1 void *ca_memory_allocate(u32 size)
```

Allocates a block of memory from the system heap.

Parametri:

Nome	Tipo	Descrizione
`size`	`u32`	The number of bytes requested. Must be greater than 0. The system will align this value and enforce a minimum block size.

Ritorna:
Returns a pointer to the allocated memory block on success. Returns NULL if the heap is not initialized, if there is insufficient memory, or if the heap structure is corrupted.

Note:

- This function is thread-safe as it uses an internal spinlock to protect the heap structure. It is NOT callable from an ISR because it may block while waiting for the spinlock. You must ensure ca_memory_init has been called during system startup before using this function. A common mistake is failing to check the returned pointer for NULL; always verify the pointer before dereferencing it to avoid hard faults. If the allocation fails, the system may trigger a corruption report if the heap integrity check fails.

Uso:

- Use this function whenever you need dynamic memory for buffers, context structures, or temporary data that does not fit on the stack. It is typically paired with ca_memory_free to release the memory once the task is complete. In a typical lifecycle, you allocate the memory, perform your operations (like reading from flash or decrypting data), and then free the memory to prevent leaks. Always handle the NULL return case by cleaning up any previously allocated resources and exiting the current operation gracefully.

Esempio:

```c
/* Example: Allocating a buffer for a file read operation */
u32 file_size = 1024;
u8 *buffer = (u8 *)ca_memory_allocate(file_size);

if (buffer == NULL) {
    /* Allocation failed, log error and exit */
    ca_printf_n("[error] Failed to allocate buffer");
    return;
}

/* Use the buffer for operations */
/* ... perform read or processing ... */

/* Always free the memory when finished */
ca_memory_free(buffer, NULL, true);
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_memory_allocate in a caOS project?
A: You should use ca_memory_allocate when you need to dynamically allocate memory for data structures, buffers, or objects at runtime. It is typically used in modules like crypto services where memory is required for operations such as AES channel handling. The function is suitable for cases where the size of memory needed is not known at compile time.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_memory_allocate with caOS types?
A: void *buffer = ca_memory_allocate(1024); if (buffer == NULL) { /* Handle allocation failure */ } else { /* Use buffer */ ca_memory_free(buffer); }

Parameters conf:0.90 **Q:** What does the 'size' parameter in ca_memory_allocate represent and what are valid values?
A: The 'size' parameter specifies the number of bytes to allocate. It must be greater than 0. Valid values are any positive integer up to the available heap size. Values that are too large may cause allocation failure due to insufficient memory.

Return conf:0.95 **Q:** What do the return values of ca_memory_allocate mean and what causes it to return NULL?
A: If ca_memory_allocate succeeds, it returns a valid pointer to the allocated memory. If it fails, it returns NULL. Failure can occur due to insufficient available memory, if the system is not initialized (g_mem_initialized = false), or if the requested size is too large.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_memory_allocate?
A: Common mistakes include not checking the return value for NULL, failing to free the allocated memory using ca_memory_free(), and attempting to allocate excessive amounts of memory in a system with limited RAM. These errors can lead to memory leaks or system instability.

Behavior conf:0.85 **Q:** Is ca_memory_allocate blocking or non-blocking, and is it safe to call from an ISR?
A: ca_memory_allocate is non-blocking and returns immediately. It is thread-safe due to spinlock protection. It can be called from an ISR, but only if the spinlock mechanism is compatible with the ISR context and the ISR is not overly critical.

`ca_memory_allocate_shared`

Header: `caos/inc/memory.h:62`
Source: `caos/src/memory.c:403:405`
GUARD: `EN_MEMORY`

```
void *ca_memory_allocate_shared(u32 size)
```

Allocates a block of memory from the shared heap, accessible by multiple tasks.

Parametri:

Nome	Tipo	Descrizione
size	u32	The number of bytes to allocate. Must be greater than 0. If 0 is passed, behavior is implementation-defined but typically returns NULL.

Ritorna:
A pointer (void *) to the allocated memory block on success, or NULL if the allocation fails (e.g., out of memory or heap fragmentation).

Note:

- This function is thread-safe and manages memory from a global shared pool. It is NOT callable from an ISR because it may block the calling task while waiting for heap access or performing memory compaction. Always check if the returned pointer is NULL before dereferencing it to avoid hard faults. A common beginner mistake is assuming the memory is zero-initialized; it is not, so you must manually clear it if needed using ca_memzero.

Uso:

- Use this function when you need to share data structures (like buffers or configuration objects) between different tasks that do not have a private memory pool. It is typically used in the initialization phase of a task or service. It is paired with ca_memory_free to release the memory back to the system once the task is finished with the resource. For debugging, you can use ca_memory_get_avail_memory to monitor heap usage before and after allocation.

Esempio:

```
void *shared_buffer = NULL;

// Attempt to allocate 1024 bytes for a shared data buffer
shared_buffer = ca_memory_allocate_shared(1024);

if (shared_buffer != NULL) {
    // Initialize the memory to zero to ensure no garbage data
    ca_memzero(shared_buffer, 1024);

    // ... use the shared buffer for inter-task communication ...

    // Once the task is done, free the memory to prevent leaks
    ca_memory_free(shared_buffer);
} else {
    // Handle allocation failure: log error or enter safe state
    ca_printf("Failed to allocate shared memory!\n");
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_memory_allocate_shared instead of a private memory pool?
A: You should use ca_memory_allocate_shared when you need to share data structures, such as buffers or configuration objects, between different tasks that do not have a private memory pool. It is typically used during the initialization phase of a task or service.

Code example conf:0.90 **Q:** How do I safely allocate and initialize a shared buffer in caOS?

A: You must allocate the memory, check for NULL, and then manually clear it since it is not zero-initialized. Example: void *ptr = ca_memory_allocate_shared(128); if (ptr != NULL) { ca_memzero(ptr, 128); }

Parameters conf:1.00 **Q:** What are the constraints on the 'size' parameter for ca_memory_allocate_shared?

A: The 'size' parameter must be a u32 value greater than 0. If 0 is passed, the behavior is implementation-defined, but it typically returns NULL.

---

ca_memory_canary_high

Header: `caos/inc/memory.h:201`
Source: `caos/src/memory.c:338:338`
GUARD: `EN_MEMORY`

```
const u32 *ca_memory_canary_high(void)
```

Returns the memory address of the high-side heap integrity canary.

Ritorna:
A constant pointer (const u32 *) to the memory location where the high-side canary value is stored. This pointer is never NULL if the memory system is initialized.

Note:

- This function is non-blocking and thread-safe as it simply returns a static memory address. It is safe to call from an ISR, though typically it is used by the scheduler or diagnostic tasks. The canary is a specific memory pattern used to detect buffer overflows; if the value at this address changes, it indicates that a task has written past its allocated memory block. Beginners often mistake this for a function that checks the memory; it does not, it only provides the address of the marker. Use ca_memory_check_canary() to actually verify the integrity.

Uso:

- This function is primarily used by the system diagnostic routines to report the location of corrupted memory during a heap fault. It is typically paired with ca_memory_canary_low() to provide a full range of the corrupted area to the developer via ca_stdout_printf_n(). It is part of the internal memory protection mechanism that runs periodically or upon detection of a system instability to help identify which memory region was compromised.

Esempio:

```
/* Check heap integrity and report if the high-side canary is compromised */
if (!ca_memory_check_canary()) {
    /* Retrieve addresses for diagnostic reporting */
    const u32 *low_ptr = ca_memory_canary_low();
    const u32 *high_ptr = ca_memory_canary_high();

    /* Log the failure with the specific memory addresses */
    ca_stdout_printf_n("CRITICAL: Heap corruption detected!\r\n");
    ca_stdout_printf_n("Low Canary Addr: 0x%08X\r\n", (u32)low_ptr);
    ca_stdout_printf_n("High Canary Addr: 0x%08X\r\n", (u32)high_ptr);

    /* Attempt to restore the canary to prevent further false positives */
    ca_memory_restore_canary();
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_memory_canary_high() and what is its purpose?
A: You should call this function when performing system diagnostics to identify the location of corrupted memory during a heap fault. It returns the address of the high-side heap integrity canary, which is a marker used to detect buffer overflows. It is typically used alongside ca_memory_canary_low() to define the range of a corrupted memory area for reporting via ca_stdout_printf_n().

Code example conf:1.00 **Q:** How do I retrieve the high-side canary address in my diagnostic task?
A: You can retrieve the address by calling the function directly. Note that this function only provides the address; you must use ca_memory_check_canary() to verify the integrity. Example: const u32 *high_canary = ca_memory_canary_high();

Return conf:1.00 **Q:** What does the return value of ca_memory_canary_high() represent, and can it return NULL?
A: The function returns a constant pointer (const u32 *) to the memory location where the high-side canary value is stored. This pointer is never NULL if the memory system is initialized. The function does not return a status code or a boolean; it simply provides the static memory address of the marker.

`ca_memory_canary_low`

Header: `caos/inc/memory.h:195`
Source: `caos/src/memory.c:337:337`
GUARD: `EN_MEMORY`

```
const u32 *ca_memory_canary_low(void)
```

Returns the memory address of the low-side heap integrity canary buffer.

Ritorna:
A constant pointer (const u32 *) to the start of the low-side canary memory area. This pointer is never NULL as it points to a statically allocated internal buffer.

Note:

- This function is non-blocking, thread-safe, and can be safely called from an ISR. It provides read-only access to the internal canary array used by the heap manager to detect buffer overflows. The most common mistake is attempting to write to the memory pointed to by this function; doing so will corrupt the heap integrity mechanism and trigger false positives in the system's safety checks. It is intended for diagnostic and reporting purposes only, and you must never modify the contents of the returned memory address.

Uso:

- This function is primarily used by the system's safety monitoring tasks to report the location of memory corruption when a canary check fails. It is typically paired with ca_memory_canary_high to provide a full range of the corrupted memory area in logs. Developers use it within error-handling routines, often alongside ca_memory_check_canary to verify heap health and ca_memory_restore_canary to reset the markers after a detected violation. It is part of the diagnostic sequence: check -> report -> restore.

Esempio:

```
/* Check heap integrity and report if corruption is detected */
if (!ca_memory_check_canary()) {
    /* Retrieve addresses of the canary buffers for diagnostic logging */
    const u32 *low_ptr = ca_memory_canary_low();
    const u32 *high_ptr = ca_memory_canary_high();
```

```
6
7       /* Log the corruption event with the specific memory addresses */
8       ca_stdout_printf_n("Heap corruption detected! Low canary at: 0x%08X, High
        canary at: 0x%08X\r\n",
9                          (u32)low_ptr, (u32)high_ptr);
10
11      /* Restore the canary markers to allow the system to continue monitoring */
12      ca_memory_restore_canary();
13  }
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_memory_canary_low in my application?
A: You should use this function within diagnostic or error-handling routines, particularly when a heap integrity violation is suspected or detected. It is intended for reporting purposes to identify the memory range of a potential overflow, often paired with ca_memory_canary_high to log the full extent of the corrupted area.

Code example conf:0.95 **Q:** How do I access the low-side canary address in a diagnostic function?
A: You can retrieve the address by calling the function directly. Example: void log_heap_error(void) { const u32 *low_ptr = ca_memory_canary_low(); const u32 *high_ptr = ca_memory_canary_high() /* Log pointers for diagnostic reporting */ }

Return conf:1.00 **Q:** What does the return value of ca_memory_canary_low represent, and can it return NULL?
A: The function returns a constant pointer (const u32 *) to the start of the internal low-side heap integrity canary buffer. It will never return NULL because it points to a statically allocated internal buffer. Failure to use this pointer as read-only will corrupt the heap integrity mechanism.

`ca_memory_check`

Header: `caos/inc/memory.h:179`
Source: `caos/src/memory.c:667:681`
GUARD: `EN_MEMORY`

```
1  bool ca_memory_check(ca_memory_check_t *r)
```

Performs a comprehensive integrity check of the heap memory management system.

Parametri:

Nome	Tipo	Descrizione
`r`	`ca_memory_check_t *`	Pointer to a structure where diagnostic results are stored. If NULL, a local stack variable is used, but results will be lost after the function returns.

Ritorna:
Returns true if the memory chain is intact, canaries are valid, and available memory matches expectations. Returns false if any check fails or if the memory system is not initialized.

Note:

- This function is blocking because it acquires a spinlock to ensure a consistent snapshot of the heap metadata. It is NOT safe to call from an ISR because it uses spinlocks that could cause deadlocks if the interrupted task already holds the lock. The function requires that the memory system has been previously initialized via ca_memory_init. A common mistake is assuming this function is lightweight; it traverses the entire memory chain, so calling it frequently in a high-frequency loop will significantly degrade system performance.

Uso:

- This function is typically used for system health monitoring or debugging memory corruption issues. It is often paired with ca_memory_init during system startup to verify the heap state, or called periodically by a watchdog task to detect buffer overflows or heap metadata corruption. If it returns false, developers should use ca_memory_list to inspect the heap state and identify which task or operation caused the corruption. It is the primary tool for verifying that heap metadata has not been overwritten by illegal memory access.

Esempio:

```
ca_memory_check_t mem_status;

// Perform a system memory integrity check
if (!ca_memory_check(&mem_status)) {
    // Memory corruption detected!
    // Log the failure and potentially trigger a system reset
    ca_printf("Critical: Memory integrity check failed!\n");

    // Optional: list the memory blocks to help identify the corrupted area
    ca_memory_list();
} else {
    // Memory system is healthy
    ca_printf("Memory system is healthy. Available: %u bytes\n", mem_status.
    avail);
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_memory_check in my caOS application?
A: It is best used for system health monitoring or debugging memory corruption. Developers typically call it during system startup after ca_memory_init to verify heap state, or periodically within a watchdog task to detect buffer overflows or heap metadata corruption.

Code example conf:0.90 **Q:** How do I implement a basic heap integrity check using ca_memory_check?
A: You can pass a pointer to a ca_memory_check_t structure to store diagnostic results. Example: ca_memory_check_t results; if (!ca_memory_check(&results)) { /* Handle corruption */ }

Parameters conf:1.00 **Q:** What happens if I pass NULL to the ca_memory_check function?
A: If you pass NULL, the function uses a local stack variable to perform the check. However, the diagnostic results will be lost once the function returns, meaning you will only know if the check passed or failed without specific details.

Return conf:1.00 **Q:** What does a return value of false indicate from ca_memory_check?
A: A return value of false indicates that the memory chain is not intact, the canaries are invalid, the available memory does not match expectations, or the memory system was not initialized via ca_memory_init.

Error/Edge case conf:1.00 **Q:** Can I call ca_memory_check from an interrupt service routine (ISR)?
A: No, it is not safe to call from an ISR. The function uses spinlocks to ensure a consistent snapshot of heap metadata, which can cause a deadlock if the interrupted task already holds the lock.

`ca_memory_check_canary`

Header: `caos/inc/memory.h:153`
Source: `caos/src/memory.c:316:327`

GUARD: `EN_MEMORY`

```
bool ca_memory_check_canary(void)
```

Verifies the integrity of the heap memory by checking predefined canary values and block metadata.

Ritorna:
Returns true if the heap memory is intact (or not yet initialized). Returns false if any canary value has been overwritten or if the heap block metadata is corrupted.

Note:

- This function is a critical diagnostic tool for detecting buffer overflows or memory corruption. It is non-blocking and performs a read-only check, making it safe to call from most contexts. However, because it iterates through the entire canary array and validates block metadata, it can be computationally expensive; calling it inside a high-frequency ISR is discouraged as it may cause jitter. A common beginner mistake is to assume this function 'fixes' the memory; it only detects corruption. If it returns false, the heap is already compromised, and you must either trigger a system reset or use a recovery function to restore the markers.

Uso:

- This function is typically used in the idle task or a dedicated system monitor task to perform background health checks. It is often paired with ca_memory_restore_canary to attempt a recovery if corruption is detected. In a robust system, you would call this before performing sensitive memory operations or as part of a watchdog-like monitoring loop to ensure that no task has performed an out-of-bounds write that would otherwise go unnoticed until a hard fault occurs.

Esempio:

```
void monitor_task(void *pv) {
    while (1) {
        /* Check if the heap has been corrupted by a buffer overflow */
        if (!ca_memory_check_canary()) {
            /* Log the corruption using the low and high canary addresses */
            ca_printf_n("CRITICAL: Heap corruption detected (lo=0x%08X hi=0x%08X)\r\n",
                        (u32)ca_memory_canary_low(),
                        (u32)ca_memory_canary_high());

            /* Attempt to restore the canary markers to allow system to continue */
            ca_memory_restore_canary();
        }

        /* Wait for a period before checking again to save CPU cycles */
        ca_os_delay_ms(1000);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_memory_check_canary in my caOS application?
A: It is typically used as a diagnostic tool within a background task or system monitor to detect buffer overflows or memory corruption. You should call it periodically or before critical operations to ensure heap integrity.

Code example conf:0.90 **Q:** Provide a minimal C example showing how to check heap integrity in caOS.

A: void monitor_task(void) { if (!ca_memory_check_canary()) { /* Handle corruption */ } }

Return conf:1.00 **Q:** What do the return values of ca_memory_check_canary indicate?

A: It returns true if the heap is intact or not yet initialized. It returns false if any canary value has been overwritten or if internal block metadata is corrupted, indicating the system state is compromised.

Error/Edge case conf:1.00 **Q:** What are the common mistakes or risks when using ca_memory_check_canary?

A: A common mistake is assuming the function repairs memory; it only detects corruption. Additionally, while it is safe to call from an ISR, it is non-blocking but may take significant time on large heaps, potentially impacting real-time latency.

`ca_memory_dump`

Header: `caos/inc/memory.h:97`
Source: `caos/src/memory.c:597:612`
GUARD: `EN_MEMORY`

```
1 void ca_memory_dump(void *p, u32 lines)
```

Prints a formatted hexadecimal and ASCII memory dump to the standard output for debugging purposes.

Parametri:

Nome	Tipo	Descrizione
`p`	`void *`	The starting memory address to dump. The function will automatically align this address to an 8-byte boundary.
`lines`	`u32`	The number of 16-byte rows to display. A value of 1 prints 16 bytes, 2 prints 32 bytes, and so on. The maximum supported value is 4095.

Ritorna:
This function returns void and does not provide a status code. If the memory system is not initialized, the function returns silently without printing anything.

Note:

- This function is intended strictly for debugging and development. It is not thread-safe and relies on the global state of the memory manager; if the memory manager has not been initialized, the function will trigger a safety handler and return. It is a blocking operation because it writes to the output stream, meaning it will pause the execution of the current task until the data is sent. Do not call this function from an Interrupt Service Routine (ISR) as it may cause system instability or deadlocks due to the blocking nature of the output stream. A common mistake is passing an invalid or unmapped memory address, which could lead to a HardFault if the underlying memory access attempts to read protected or non-existent memory regions.

Uso:

- Use this function during the development phase to inspect the contents of buffers, structures, or peripheral registers when you suspect data corruption or incorrect initialization. It is typically used after a memory allocation via 'ca_memory_allocate' or a data reception process to verify that the expected values are present in RAM. It is often paired with 'ca_stdout_put_n' for logging headers and is usually wrapped in conditional compilation flags (like ENABLE_LOG) to ensure it is stripped out of production builds to save

performance and memory. It is the standard way to visualize raw memory state when 'ca_memory_check' indicates a potential corruption.

Esempio:

```
/* Example: Inspecting a buffer after allocation and data processing */
u8 *my_buffer = (u8 *)ca_memory_allocate(32);

if (my_buffer != NULL) {
    /* ... perform operations on my_buffer ... */

    /* Dump 2 lines (32 bytes total) starting from the buffer address */
    /* Ensure the memory system is initialized before calling */
    ca_memory_dump(my_buffer, 2);

    ca_memory_free(my_buffer);
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_memory_dump() in my caOS project?
A: You should use ca_memory_dump() for debugging memory contents, especially when you need to inspect a specific memory region. It's useful for verifying memory layout, checking data integrity, or troubleshooting memory-related issues. The function is typically called during development or in debug builds when logging is enabled.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_memory_dump() with caOS types?
A: u8 buffer[32]; for(u32 i = 0; i < 32; i++) { buffer[i] = i; } ca_memory_dump(buffer, 2);

Parameters conf:0.95 **Q:** What happens if I pass NULL as the pointer to ca_memory_dump()?
A: If the pointer 'p' is NULL, the function will not perform any action and will return immediately without printing anything. This is a safe behavior to prevent crashes or undefined behavior.

Return conf:0.90 **Q:** Does ca_memory_dump() have any return values or error conditions?
A: The function does not return any value. It is designed to never fail, but it may not print anything if the parameters are invalid, logging is disabled, or memory is not initialized. It is a void function with no failure modes defined.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_memory_dump()?
A: Common mistakes include calling ca_memory_dump() from an ISR, which is not allowed because it uses blocking output functions. Also, passing a pointer that points to inaccessible memory or a very large 'lines' value that could cause buffer overruns may lead to undefined behavior. Additionally, calling it when ENABLE_LOG is disabled or g_mem_initialized is false will result in no output.

`ca_memory_dump_avail`

Header: `caos/inc/memory.h:104`
Source: `caos/src/memory.c:683:712`
GUARD: `EN_MEMORY`

```
void ca_memory_dump_avail(void)
```

Prints a detailed diagnostic report of the heap memory status to the standard output.

Ritorna:
This function returns void and does not provide a status code.

Note:

- This function is non-blocking but performs extensive string formatting and serial output, making it unsuitable for use inside high-frequency Interrupt Service Routines (ISRs) where timing is critical. It requires the memory system to be initialized; if called before 'ca_memory_init', it will trigger a safety handler and return immediately. The function performs a full heap integrity check, including canary validation and block chain verification, which is computationally expensive. A common beginner mistake is calling this function repeatedly in a tight loop, which will flood the serial output and significantly degrade system performance.

Uso:

- This function is used as a primary diagnostic tool for heap health monitoring, typically triggered by a CLI command or a system watchdog task. It is used to verify that memory allocation is behaving as expected, checking for corruption in the heap chain or canary patterns. Developers use this in conjunction with 'ca_memory_list' to inspect specific allocations or 'ca_scheduler_dump_task' to correlate memory usage with active tasks. It is the standard way to confirm if a system crash was caused by memory exhaustion or heap corruption.

Esempio:

```c
/* Example usage within a CLI command handler to check heap health */
void handle_cli_command(char *cmd) {
    /* Check if the user requested a full memory dump */
    if (ca_strncmp(cmd, "free", 4) == 0) {
        /* Perform a full heap integrity check and print the report */
        ca_memory_dump_avail();
    } else if (ca_strncmp(cmd, "tasks", 5) == 0) {
        /* List all tasks to see if any are leaking memory */
        ca_scheduler_dump_all_tasks();
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_memory_dump_avail() in my caOS application?
A: You should call ca_memory_dump_avail() during debugging or memory monitoring to check available heap memory. It's useful for tracking memory leaks or ensuring sufficient memory is available for dynamic allocations. The function is non-blocking and should not be called from an ISR, but can be used in main application loops or command-line interfaces.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_memory_dump_avail() with caOS types?
A: void example_function(void) { ca_memory_dump_avail(); }

Return conf:0.90 **Q:** What happens if ca_memory_dump_avail() is called before memory initialization?
A: If ca_memory_dump_avail() is called before memory initialization, it will return immediately without printing anything. This is because the function checks the global variable 'g_mem_initialized' and only proceeds with printing if it is true. There are no failure conditions as such, since the function is designed to be safe even when memory is not initialized.

`ca_memory_free`

Header: `caos/inc/memory.h:70`
Source: `caos/src/memory.c:448:478`

GUARD: `EN_MEMORY`

```
bool ca_memory_free(void *p, u32 *freed_size, bool wiped)
```

Releases a previously allocated memory block back to the system heap, optionally wiping the data for security.

Parametri:

Nome	Tipo	Descrizione
`p`	`void *`	Pointer to the memory block to be freed. Must be a pointer previously returned by ca_memory_allocate. Passing NULL is safe and returns false.
`freed_size`	`u32 *`	Optional pointer to a variable where the function will store the size of the freed block. Pass NULL if you do not need this information.
`wiped`	`bool`	If true, the function overwrites the memory with zeros before releasing it. Use this for sensitive data like cryptographic keys.

Ritorna:
Returns true if the memory block was successfully identified and freed. Returns false if the pointer is invalid, the memory system is uninitialized, or the pointer is outside the managed heap range.

Note:

- This function is thread-safe as it uses an internal spinlock to protect the heap structure. It is NOT safe to call from an ISR because it uses spinlocks that disable interrupts, which can cause system instability or deadlocks if called from a high-priority context. The function is blocking; it must traverse the heap metadata to update block status, so execution time depends on heap fragmentation. A common beginner mistake is passing a pointer that was not allocated by ca_memory_allocate (e.g., a stack variable or a static buffer), which will trigger a memory error log and return false. Always ensure the memory system is initialized before calling this.

Uso:

- This function is the standard way to prevent memory leaks in long-running embedded applications. It is typically used in a cleanup pattern: after a task completes, or when an error occurs during initialization, you must free any buffers allocated during the setup phase. It is often paired with ca_memory_allocate in a 'try-allocate-then-free-on-error' pattern to ensure that if a complex operation fails halfway through, the system does not leak memory. In driver development, it is used to release temporary buffers used for DMA or flash operations once the data transfer is complete.

Esempio:

```
void process_secure_data(void) {
    u8 *key_buffer = ca_memory_allocate(32);
    if (key_buffer == NULL) {
        return;
    }

    /* Perform sensitive cryptographic operation */
    /* Assume 'ca_aes_gcm_decrypt' uses the key_buffer */
    bool ok = ca_aes_gcm_decrypt(NULL, NULL, 0, NULL, key_buffer, 32);

    /* Always free memory when done. */
```

```
    /* Use 'true' for wiped to ensure the key is cleared from RAM. */
    if (!ca_memory_free(key_buffer, NULL, true)) {
        ca_printf_n("Error: Memory corruption detected during free");
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_memory_free() in my embedded application?
A: You should call ca_memory_free() when you are done using a memory block that was previously allocated with ca_memory_alloc(). It is typically used to release memory back to the pool so it can be reused. This function is especially useful when you need to ensure that sensitive data is wiped from memory before freeing it.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_memory_free() with caOS types?
A: u32 *ptr = (u32*)ca_memory_alloc(100, false); if (ptr != NULL) { // Use memory ca_memory_free(ptr, NULL, true); }

Parameters conf:0.95 **Q:** What does the 'wiped' parameter in ca_memory_free() do and what are valid values?
A: The 'wiped' parameter controls whether a deep wipe (writing zeros) is performed on the memory block before freeing it. If true, the memory is zeroed out before freeing. If false, no wipe is performed. Valid values are boolean true or false.

Return conf:0.95 **Q:** What does ca_memory_free() return and what causes it to return false?
A: ca_memory_free() returns true if the memory block was successfully freed. It returns false if the operation failed. Failures occur if the pointer p is NULL, the memory is not initialized, or the pointer is invalid (e.g., outside the allocated memory range).

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_memory_free()?
A: Common mistakes include calling ca_memory_free() with a NULL pointer, passing a pointer that was not allocated with ca_memory_alloc(), or calling it with a pointer that has already been freed. Additionally, calling this function from an ISR is not safe and may cause issues due to blocking behavior.

Behavior conf:0.90 **Q:** Is ca_memory_free() blocking or non-blocking, and is it safe to call from an ISR?
A: ca_memory_free() is a blocking function because it acquires a spinlock before operating. It is not safe to call from an ISR because it may block and could lead to deadlocks or other timing issues in interrupt contexts.

`ca_memory_free_all_owned_by`

Header: `caos/inc/memory.h:78`
Source: `caos/src/memory.c:487:523`
GUARD: `EN_MEMORY`

```
u32 ca_memory_free_all_owned_by(u16 task_slot)
```

Automatically releases all memory blocks currently held by a specific task.

Parametri:

Nome	Tipo	Descrizione
`task_slot`	`u16`	The unique identifier (slot) of the task whose memory needs to be reclaimed. Must be a valid task slot index.

Ritorna:

Returns a u32 representing the total number of memory blocks successfully freed. Returns 0 if no memory was allocated by the task or if the memory system is not initialized.

Note:

- This function is designed for emergency cleanup and task termination. It is thread-safe and uses internal spinlocks to protect the memory heap, but it is intended to be called by the scheduler or system manager during task teardown. Because it modifies the memory list structure, it performs a restart-scan, meaning it is not an O(1) operation and can take longer depending on the number of blocks to free. Do not call this from a standard application task unless you are intentionally cleaning up a child process or a failed task, as it will forcefully reclaim memory without notifying the owner. It is safe to call from ISR context because it uses spinlocks with interrupt disabling.

Uso:

- This function is typically used in the scheduler's task-deletion sequence. When a task is terminated (e.g., due to a watchdog timeout or explicit exit), the system calls this to prevent memory leaks. It is usually paired with ca_os_task_slot to identify the task and is invoked before the task structure is unlinked from the scheduler's run queues. It ensures that resources are returned to the heap before the task slot is marked as available for reuse.

Esempio:

```c
/* Example: Cleaning up a task that has failed or is being removed */
void cleanup_failed_task(void *task_handle) {
    /* Get the slot index for the task handle */
    u16 slot = ca_os_task_slot(task_handle);

    /* Free all memory blocks owned by this task to prevent leaks */
    /* This is critical to prevent heap exhaustion after task crashes */
    u32 freed = ca_memory_free_all_owned_by(slot);

    if (freed > 0) {
        /* Log the cleanup action using standard output */
        ca_stdout_printf_n("Cleanup: Reclaimed %u blocks from slot %u", freed,
    slot);
    }

    /* Proceed to remove the task from the scheduler list */
    /* Note: Use system-specific task removal functions here */
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_memory_free_all_owned_by() in a caOS application?
A: You should call ca_memory_free_all_owned_by() during the cleanup phase of a task termination, typically from the scheduler when a task is being removed. This function is designed to be invoked by the scheduler to automatically free all memory blocks allocated by a specific task.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_memory_free_all_ow
A: u16 task_index = 2; u32 freed_blocks = ca_memory_free_all_owned_by(task_index);
// freed_blocks now contains the number of memory blocks freed for task 2

Parameters conf:0.90 **Q:** What is the valid range for the task_index parameter in ca_memory_free_all_owned_
A: The task_index parameter must be a valid index between 0 and the maximum number of tasks configured in the system. Using a value outside this range may lead to invalid memory access.

Return conf:0.95 **Q:** What does a return value of 0 from ca_memory_free_all_owned_by() indicate?

A: A return value of 0 indicates that either the memory system is not initialized or the specified task has not allocated any memory blocks.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_memory_free_all_owned_by()?

A: Common mistakes include calling the function with an invalid task_index, which can cause memory access violations, or calling it from a low-priority task context where spinlock contention might lead to indefinite blocking. Also, calling it from an ISR without proper precautions may lead to system instability.

Behavior conf:0.90 **Q:** Is ca_memory_free_all_owned_by() thread-safe and can it be called from an ISR?

A: No, ca_memory_free_all_owned_by() is not thread-safe and is not safe to call from an ISR. It uses a spinlock internally and is designed to be called only from high-priority tasks or from the scheduler context. It is a blocking function that may cause indefinite blocking if another thread holds the spinlock.

`ca_memory_get_avail_memory`

Header: `caos/inc/memory.h:130`
Source: `caos/src/memory.c:819:825`
GUARD: `EN_MEMORY`

```
u32 ca_memory_get_avail_memory(void)
```

Returns the total amount of free memory currently available in the system heap.

Ritorna:
Returns the number of bytes currently available as a u32. Returns 0 if the memory manager has not been initialized yet.

Note:

- This function is thread-safe because it uses a spinlock to protect the global memory variable during access. It is non-blocking in the sense that it does not wait for external hardware, but it does briefly disable interrupts to ensure data consistency. You can safely call this from an ISR or a task, but avoid calling it in a tight loop inside a high-frequency interrupt as the spinlock overhead could impact system timing. The most common mistake is assuming this value remains constant; in a dynamic system, memory is constantly being allocated and freed, so treat this value as a snapshot rather than a guarantee for future allocations.

Uso:

- This function is primarily used for system monitoring, diagnostics, and debugging purposes. It is typically paired with ca_memory_get_total_size() to calculate memory utilization percentages or to check if enough heap space remains before performing a large dynamic allocation. Developers use it in telemetry tasks or CLI commands to report system health to the user. It should only be called after the system memory manager has been initialized, otherwise, it will return 0, which might be misinterpreted as an 'out of memory' condition.

Esempio:

```
/* Check if we have enough memory before allocating a large buffer */
u32 free_mem = ca_memory_get_avail_memory();
if (free_mem > 1024U) {
    /* Proceed with allocation using system memory allocator */
    void* buffer = ca_memory_allocate(1024U);
    if (buffer != NULL) {
        /* Use the buffer and then free it */
```

```
        ca_memory_free(buffer);
    }
} else {
    ca_stdout_printf_n("Error: Not enough memory. Available: %u bytes", (unsigned)free_mem);
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_memory_get_avail_memory() and why is it useful for embedded development?
A: You should call ca_memory_get_avail_memory() to monitor available RAM in your caOS application, especially when managing dynamic memory allocation or debugging memory leaks. It's useful for ensuring your system doesn't run out of memory during runtime, particularly in real-time systems where memory usage must be predictable. The function is thread-safe and can be called from ISRs, making it suitable for monitoring memory from interrupt contexts.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_memory_get_avail_memory() with caOS types?
A: u32 available = ca_memory_get_avail_memory(); if (available == 0) { // Memory not initialized return 1; } else { // Use available memory value u32 used = g_total_mem - available; return 0; }

Return conf:0.90 **Q:** What do the return values of ca_memory_get_avail_memory() mean and what causes a return value of 0?
A: The function returns 0 if memory has not been initialized (when g_mem_initialized = false). Otherwise, it returns the amount of available memory in bytes. A return value of 0 is not an error but an indicator that the memory subsystem has not yet been initialized. This typically happens before caos_init() or ca_memory_init() is called, or if initialization fails.

`ca_memory_get_end_address`

Header: `caos/inc/memory.h:118`
Source: `caos/src/memory.c:803:809`
GUARD: `EN_MEMORY`

```
u32 *ca_memory_get_end_address(void)
```

Retrieves the absolute end address of the managed memory heap area.

Ritorna:
Returns a pointer (u32*) to the end of the memory heap if initialized, or NULL if the memory system has not been initialized yet.

Note:

- This function is thread-safe because it uses a spinlock to protect the global memory boundary variable during access. It is non-blocking in the sense that it does not wait for external hardware, but it does disable interrupts briefly to ensure atomicity, making it safe to call from both standard tasks and Interrupt Service Routines (ISRs). A common mistake is assuming the returned address is always valid without checking for NULL; if the memory subsystem failed to initialize or was not started, dereferencing the returned NULL pointer will trigger a HardFault. Always verify the return value before performing pointer arithmetic or memory boundary checks.

Uso:

- This function is typically used by system diagnostic tasks or memory management routines to calculate the total size of the heap or to perform bounds checking on dynamic allocations. It is often paired with ca_memory_get_start_address() to define the full memory range. Developers use this during system startup or when implementing custom memory monitoring tools to ensure that pointers do not exceed the allocated heap boundaries, preventing buffer overflows or memory corruption.

Esempio:

```
u32 *start = ca_memory_get_start_address();
u32 *end = ca_memory_get_end_address();

if (end != NULL) {
    // Calculate total heap size in bytes using pointer arithmetic
    u32 total_size_bytes = (u32)((uintptr_t)end - (uintptr_t)start);
    ca_printf("Heap range: %p to %p, Size: %u bytes\n", start, end,
    total_size_bytes);
} else {
    // Handle error: memory system not initialized
    ca_printf("Error: Memory system not initialized!\n");
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should a developer call ca_memory_get_end_address() and why is it useful for memory management?
A: A developer should call ca_memory_get_end_address() to retrieve the last valid memory address of the allocated memory space. This is useful for tracking memory boundaries, especially when implementing custom memory allocators or debugging memory usage. It helps ensure that memory operations do not exceed the allocated memory region, preventing buffer overflows and memory corruption.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_memory_get_end_address() with caOS types?
A: u32 *end_addr = ca_memory_get_end_address(); if (end_addr != NULL) { u32 memory_end = (u32)end_addr; // Use memory_end for further memory management } else { // Handle uninitialized memory case }

Return conf:0.90 **Q:** What do the return values of ca_memory_get_end_address() indicate, and what causes the function to return NULL?
A: The function returns a pointer to the end address of the allocated memory if successful. If the memory has not been initialized, it returns NULL. This indicates that the memory subsystem is not yet set up, and calling this function before memory initialization will result in a NULL return value. The function uses a spinlock for thread safety, but it must not be called from an ISR due to potential deadlock risks.

`ca_memory_get_start_address`

Header: `caos/inc/memory.h:113`
Source: `caos/src/memory.c:795:801`
GUARD: `EN_MEMORY`

```
u32 *ca_memory_get_start_address(void)
```

Retrieves the base memory address of the managed heap or memory pool.

Ritorna:
Returns a pointer to the start of the memory region (u32*) if initialized, or NULL if the memory system has not been initialized yet.

Note:

- This function is thread-safe because it uses a spinlock to protect the internal pointer variable during access. It is non-blocking in the sense that it does not wait for a task to finish, but it does disable interrupts briefly via ca_spinlock_lock_irq, so it is safe to call from an ISR, though it should be used sparingly to avoid latency. A common mistake is assuming the returned pointer is always valid without checking for NULL; if the memory system failed to initialize, dereferencing the returned NULL pointer will cause a HardFault. Always ensure the memory subsystem is fully initialized before calling this, as the function relies on the internal state of the memory manager.

Uso:

- This function is typically used by system-level drivers or diagnostic tools that need to perform bounds checking or memory mapping verification. It is often called after the system boot sequence, specifically after the memory manager has been initialized via ca_memory_init. Developers use it to calculate offsets or verify if a specific memory address falls within the managed heap range. It is frequently paired with ca_memory_get_end_address to define the boundaries of the heap, ensuring that pointers being passed to hardware peripherals or DMA controllers are within the valid, managed memory space.

Esempio:

```
u32 *heap_start = ca_memory_get_start_address();

if (heap_start != NULL) {
    // Successfully retrieved the base address
    // Verify if a pointer is within the managed heap range
    u32 *end_addr = ca_memory_get_end_address();
    if (heap_start <= target_ptr && target_ptr < end_addr) {
        // Pointer is within valid bounds
    }
} else {
    // Handle the error: memory system is not ready
    // Use ca_printf to log the initialization failure
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should a developer call ca_memory_get_start_address() and why is it important in a caOS application?
A: A developer should call ca_memory_get_start_address() after the caOS memory system has been initialized to obtain the base address of the allocated memory region. This is important because it allows the application to access or manage the memory area that caOS uses for its internal operations, such as task stacks or memory pools. The function is thread-safe and can be called from ISRs if the spinlock is configured for ISR safety, making it suitable for use in both task and interrupt contexts.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using caOS types that demonstrates how to use ca_memory_get_start_address()?
A: u32 *start_addr = ca_memory_get_start_address(); if (start_addr != NULL) { // Memory is initialized, use start_addr u32 first_word = *start_addr; // Process first_word } else { // Handle uninitialized memory case // Likely a fatal error in real application }

Return conf:0.95 **Q:** What do the return values of ca_memory_get_start_address() indicate, and what causes the function to return NULL?
A: The function returns a pointer to u32 which is the start address of the memory allocated by caOS. If the memory is not yet initialized (i.e., g_mem_initialized = false), the function returns NULL. This typically happens if the function is called before the

caOS memory subsystem has been properly initialized. Failing to check the return value can lead to crashes or undefined behavior, as the application might attempt to dereference a NULL pointer.

---

ca_memory_get_task_usage

Header: caos/inc/memory.h:144
Source: caos/src/memory.c:836:855
GUARD: EN_MEMORY

```
u32 ca_memory_get_task_usage(u16 task_slot)
```

Calculates the total amount of heap memory currently allocated by a specific task.

Parametri:

Nome	Tipo	Descrizione
task_slot	u16	The unique slot index of the task to query. Must be a valid index assigned by the scheduler; passing an invalid index will return 0 as no blocks will match.

Ritorna:
Returns the total number of bytes (payload only) currently held by the specified task. Returns 0 if the memory system is not initialized or if the task has no active memory allocations.

Note:

- This function is thread-safe as it uses a spinlock to protect the heap structure during traversal. It is non-blocking in terms of waiting for other tasks, but it does disable interrupts via ca_spinlock_lock_irq to ensure data consistency. Do not call this function from an ISR, as the spinlock mechanism is intended for thread-level synchronization and may cause system instability or deadlocks if invoked from an interrupt context. A common beginner mistake is to call this function frequently in a high-priority loop, which can significantly increase system jitter due to the time spent traversing the entire heap list while interrupts are disabled.

Uso:

- This function is primarily used for system monitoring, debugging, and diagnostic reporting. It is typically called by a system monitor task or a shell command handler that iterates through the task list to display resource usage statistics. You would use this in conjunction with task management functions to print memory footprints during runtime, helping you identify memory leaks or tasks that are consuming more heap than expected. It is usually paired with scheduler inspection functions to correlate task status with memory consumption, often invoked when a user requests a system status dump via a CLI or debug interface.

Esempio:

```
/* Example: Monitoring memory usage of a specific task */
/* Get the current task handle and convert it to a slot index */
void *handle = ca_os_self_handle();
u16 slot = ca_os_task_slot(handle);

/* Calculate the memory footprint of the current task */
u32 used_bytes = ca_memory_get_task_usage(slot);

if (used_bytes > 1024) {
```

```
    /* Log a warning if the task is using more than 1KB of heap */
    ca_stdout_printf_n("Warning: Task slot %u is using high memory: %u bytes",
    slot, used_bytes);
} else {
    ca_stdout_printf_n("Task slot %u memory usage: %u bytes", slot, used_bytes);
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_memory_get_task_usage() and what is its purpose in a caOS application?
A: You should call ca_memory_get_task_usage() to monitor the memory usage of a specific task in your caOS application. This function is useful for debugging memory leaks or tracking how much memory each task is consuming. It helps in ensuring that tasks do not exceed allocated memory limits and aids in performance optimization.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_memory_get_task_usage()?
A: u32 memory_used = ca_memory_get_task_usage(0); // Get memory usage for task index 0 if (memory_used > 0) { // Process memory usage }

Parameters conf:0.90 **Q:** What does the task_index parameter represent and what are valid values for it?
A: The task_index parameter represents the index of the task whose memory usage is being queried. It must be a valid value between 0 and the maximum number of tasks defined in the system. Using an invalid index may lead to unauthorized memory access or undefined behavior.

Return conf:0.90 **Q:** What does ca_memory_get_task_usage() return in different scenarios, and what causes it to return zero?
A: The function returns a u32 value representing the total number of bytes allocated by the specified task. It returns 0 if the system is not initialized (g_mem_initialized = 0). It may also return 0 or an incorrect value in case of internal errors or invalid access.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_memory_get_task_usa
A: Common mistakes include calling the function with an invalid task_index, which can cause memory access violations. Additionally, calling this function from an ISR is unsafe and may lead to deadlock or undefined behavior. Not handling the spinlock properly can also result in inconsistent data.

Behavior conf:0.90 **Q:** Is ca_memory_get_task_usage() blocking or non-blocking, and is it safe to call from an ISR or from multiple threads?
A: The function is blocking because it acquires a spinlock before reading memory data, which can cause delays if another thread is using memory. It is thread-safe due to the use of a spinlock protecting the memory data structure. However, it is not safe to call from an ISR because spinlocks can cause deadlocks or undefined behavior in interrupt contexts.

`ca_memory_get_total_memory`

Header: `caos/inc/memory.h:135`
Source: `caos/src/memory.c:827:833`
GUARD: `EN_MEMORY`

```
u32 ca_memory_get_total_memory(void)
```

Returns the total amount of heap memory managed by the caOS memory allocator in bytes.

Ritorna:

Returns the total memory size in bytes as a u32. Returns 0 if the memory manager has not been initialized yet.

Note:

- This function is thread-safe because it uses a spinlock to protect the internal memory variable during access. It is safe to call from both standard tasks and Interrupt Service Routines (ISRs) because it uses a spinlock that disables interrupts (ca_spinlock_lock_irq). The function is non-blocking in the sense that it does not wait for other tasks to finish, but it will briefly stall the CPU to acquire the lock. A common mistake is assuming this value represents 'free' memory; this function specifically returns the 'total' capacity, not the currently available space. Always ensure the system has reached the initialization phase before relying on this value, as it will return 0 if called too early during the boot sequence.

Uso:

- This function is typically used during system diagnostics, boot-up logging, or health monitoring to report the hardware capabilities to the user via a console or debug interface. It is often paired with other system information functions like ca_os_clock_get_freq() to provide a snapshot of the system state. Developers use it to verify that the memory heap was configured correctly during the linker script setup or initialization phase. It is rarely used in the main control loop of an application, as the total memory size is usually a static constant determined at compile time.

Esempio:

```c
/* Example: Reporting system status during initialization */

void report_system_status(void) {
    u32 total_mem = ca_memory_get_total_memory();

    if (total_mem == 0) {
        /* Memory manager not ready, handle error */
        return;
    }

    /* Print the total memory to the debug console */
    ca_stdout_printf_n("System initialized with %u bytes of heap.", total_mem);

    /* Optionally check available memory for a complete health report */
    u32 avail_mem = ca_memory_get_avail_memory();
    ca_stdout_printf_n("Available heap: %u bytes.", avail_mem);
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should a developer call ca_memory_get_total_memory() and why is it important to check if the system is initialized before using it?
A: A developer should call ca_memory_get_total_memory() when they need to determine the total amount of available memory in the system. It's important to check if the system is initialized because the function returns 0 if the memory system hasn't been initialized (g_mem_initialized = false). Calling this function without proper initialization can lead to incorrect memory calculations or unexpected behavior in memory management routines.

Code example conf:0.90 **Q:** Can you provide a minimal C code example that demonstrates how to use ca_memory_get_total_memory() with caOS types?
A: u32 total_memory = ca_memory_get_total_memory(); if (total_memory != 0) { // Use total_memory for memory allocation or logging // Example: log the total memory } else { // Handle the case where memory system is not initialized }

Return conf:0.90 **Q:** What do the return values of ca_memory_get_total_memory() indicate, and what causes the function to return 0?
A: The function returns a u32 value representing the total memory in bytes. If the system is not initialized (g_mem_initialized = false), it returns 0. This typically happens when the memory subsystem has not been properly set up before calling this function. The function acquires a spinlock (ca_spinlock_lock_irq) before reading the memory value, so a return value of 0 may also indicate that the system is in an inconsistent state or that initialization failed.

`ca_memory_get_total_size`

Header: `caos/inc/memory.h:123`
Source: `caos/src/memory.c:811:817`
GUARD: `EN_MEMORY`

```
u32 ca_memory_get_total_size(void)
```

Returns the total size of the managed heap memory in bytes.

Ritorna:
Returns the total memory size as a u32 integer. Returns 0 if the memory system has not been initialized yet.

Note:

- This function is thread-safe because it uses a spinlock to protect access to the global memory boundaries. It is safe to call from an Interrupt Service Routine (ISR) because it uses a spinlock that disables interrupts (ca_spinlock_lock_irq), preventing deadlocks or race conditions during the calculation. The most common mistake is assuming this function returns 'available' memory; it only returns the static total capacity defined at system startup. If you call this before the memory manager is initialized, you will receive 0, which might lead to division-by-zero errors if you use this value to calculate percentages without checking for zero first.

Uso:

- This function is typically used for system diagnostics, telemetry, or monitoring tasks to report memory health. It is often paired with ca_memory_get_avail_memory() to calculate the current memory utilization percentage. Developers use this in a sequence: first, verify the system is initialized, then fetch the total size, fetch the available size, and finally perform arithmetic to determine how much heap is currently consumed by the application.

Esempio:

```
/* Check memory usage for diagnostic reporting */
// Ensure the memory system is ready before performing calculations
u32 total = ca_memory_get_total_size();

if (total > 0) {
    u32 avail = ca_memory_get_avail_memory();
    u32 used = total - avail;
    u32 percent = (used * 100) / total;

    ca_printf_n("System Memory: %u/%u bytes (%u%% used)", used, total, percent);
} else {
    ca_printf_n("Memory system not initialized or size is zero.");
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should a developer call ca_memory_get_total_size() and why is it useful for memory monitoring?
A: A developer should call ca_memory_get_total_size() to retrieve the total allocatable memory size in bytes, which is useful for real-time memory usage monitoring. This function is typically used in system diagnostics or memory management routines to ensure sufficient memory is available for task allocation. It is important to note that this function must be called after memory initialization, otherwise it returns 0.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_memory_get_total_size() with caOS types?
A: u32 total_memory = ca_memory_get_total_size(); if (total_memory == 0) { // Handle memory not initialized } else { // Use total_memory for monitoring or allocation logic }

Return conf:0.90 **Q:** What do the return values of ca_memory_get_total_size() indicate, and what causes the function to return 0?
A: The function returns a u32 value representing the total memory size in bytes. If the memory is not initialized (i.e., g_mem_initialized is false), it returns 0. This failure condition occurs when the memory subsystem has not been properly initialized before calling this function, which can lead to incorrect behavior or system crashes if not handled correctly.

`ca_memory_info`

Header: `caos/inc/memory.h:85`
Source: `caos/src/memory.c:714:791`
GUARD: `EN_MEMORY`

```
1 void ca_memory_info(void *p)
```

Retrieves and logs diagnostic metadata for a specific heap memory block.

Parametri:

Nome	Tipo	Descrizione
p	void *	A pointer to the payload area of a memory block previously allocated by the system. Must be within the heap range; passing NULL or an address outside the heap will trigger an audit log error.

Ritorna:
void. This function does not return a value; it outputs diagnostic information directly to the system stdout.

Note:

- This function is thread-safe as it uses a spinlock to protect the heap metadata during inspection. It is non-blocking in the sense that it does not wait for other tasks, but it does disable interrupts briefly via ca_spinlock_lock_irq to ensure data consistency. It is safe to call from an ISR, provided the spinlock is not held elsewhere, though it is primarily intended for debugging tasks. A common mistake is passing a pointer that has already been freed or a pointer that does not point to the start of an allocated block, which will cause the function to report an invalid header or block status.

Uso:

- Use this function during development or troubleshooting to inspect the state of specific memory allocations. It is typically used when a system behaves unexpectedly, such as

when a memory leak is suspected or when a task is crashing due to memory corruption. You would pair this with ca_memory_dump_avail to get a global view of memory, then use ca_memory_info to drill down into the specific block causing issues. It is not intended for production runtime logic, but rather as a diagnostic tool to verify ownership and block integrity.

Esempio:

```
void *my_buffer = ca_memory_allocate(128); // Allocate memory block

if (my_buffer != NULL) {
    // Inspect the block to verify metadata and ownership
    ca_memory_info(my_buffer);

    // Perform operations with the buffer...

    ca_memory_free(my_buffer);
} else {
    // Log failure using standard output
    ca_printf("Allocation failed, checking available memory...\r\n");
    ca_memory_dump_avail();
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_memory_info() and what is its purpose in debugging?
A: You should call ca_memory_info() when you need to inspect the details of a memory block allocated with a memory allocation function like malloc. It is useful for debugging and monitoring heap usage. The function prints detailed information about the memory block directly to stdout or audit log.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_memory_info() with caOS types?
A: void *ptr = ca_malloc(100); ca_memory_info(ptr);

Parameters conf:0.95 **Q:** What is the significance of the void *p parameter in ca_memory_info() and what happens if it is NULL?
A: The void *p parameter is a pointer to the memory block for which you want to retrieve information. If p is NULL, the function returns immediately and logs an error.

Error/Edge case conf:0.90 **Q:** What are the common error conditions or edge cases when using ca_memory_info()?
A: If the memory system is not initialized (g_mem_initialized is false), or if the pointer p is outside the heap area, the function will ignore the call and log an error. Additionally, calling ca_memory_info() on a freed block is allowed but may produce invalid or meaningless data.

Behavior conf:0.85 **Q:** Is ca_memory_info() blocking, ISR-safe, or thread-safe?
A: The function is blocking only for the time needed to acquire a spinlock for memory access. It is not ISR-safe because it uses logging and locking functions that may not be safe in interrupt context. It is not fully thread-safe in the strict sense, but uses a lock to ensure consistency.

`ca_memory_init`

Header: `caos/inc/memory.h:40`
Source: `caos/src/memory.c:248:312`
GUARD: `EN_MEMORY`

```
void ca_memory_init(u32 *heap_start, u32 *heap_end)
```

Initializes the system heap memory manager, setting up memory blocks, canaries, and internal tracking structures.

Parametri:

Nome	Tipo	Descrizione
`heap_start`	`u32 *`	Pointer to the beginning of the memory region reserved for the heap. Must be aligned according to system requirements.
`heap_end`	`u32 *`	Pointer to the end of the memory region reserved for the heap.

Ritorna:
void. If the provided memory range is too small to hold the required metadata and canaries, it prints an error and leaves the heap uninitialized.

Note:

- This function is not thread-safe and must be called exactly once during the system boot sequence before any memory allocation occurs. It is strictly non-reentrant and must NOT be called from an ISR. The function performs critical setup, including placing 'canaries' (sentinel values) at the boundaries of the heap to detect buffer overflows; if these canaries are corrupted later, the system may trigger a fault. A common beginner mistake is attempting to allocate memory before calling this function or providing a memory range that overlaps with the stack or static data sections.

Uso:

- This is a foundational initialization function used during the early boot phase of the application. It is typically called immediately after hardware-level initializations (like MPU or bus error handlers) and before the scheduler or any task-based memory allocation begins. It pairs with subsequent calls to ca_memory_allocate to manage dynamic memory throughout the application lifecycle.

Esempio:

```c
/* Define heap boundaries, usually provided by the linker script */
extern u32 _heap_start;
extern u32 _heap_end;

void system_init(void) {
    /* Initialize hardware protection first */
    ca_mpu_init();

    /* Initialize the heap manager before any tasks start */
    ca_memory_init(&_heap_start, &_heap_end);

    /* Verify initialization success */
    if (ca_memory_get_total_memory() > 0) {
        /* Safe to allocate memory now */
        void *ptr = ca_memory_allocate(1024);
        if (ptr != NULL) {
            /* Use memory... */
            ca_memory_free(ptr);
        }
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_memory_init in a caOS application?

A: You should call ca_memory_init after initializing the MPU but before starting any tasks that require dynamic memory allocation. It sets up the heap manager for the RTOS and must be called once during system initialization.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_memory_init with caOS types?
A: u32 heap[1024]; ca_memory_init(heap, &heap[1024]);

Parameters conf:0.95 **Q:** What are the requirements for heap_start and heap_end parameters?
A: heap_start must be a valid, aligned pointer to the beginning of the heap area. heap_end must be a pointer greater than heap_start. Both must define an area of at least 128 bytes. If heap_end is not greater than heap_start, or if the space is less than 128 bytes, the function will not initialize anything and print an error.

Return conf:0.90 **Q:** What happens if ca_memory_init fails to initialize the heap?
A: If heap_start and heap_end are invalid or the space is less than 128 bytes, the function returns without initializing anything and prints an error message. No return value is provided.

Error/Edge case conf:0.85 **Q:** What are common mistakes when calling ca_memory_init?
A: Common mistakes include calling it from an ISR, not aligning heap_start properly, passing heap_end not greater than heap_start, or providing a heap area smaller than 128 bytes. It must also be called after MPU initialization.

Behavior conf:0.90 **Q:** Is ca_memory_init blocking or non-blocking, and is it safe to call from an ISR?
A: ca_memory_init is a blocking function and must not be called from an ISR. It is also not thread-safe and must be called once during system initialization before any multitasking begins.

`ca_memory_list`

Header: `caos/inc/memory.h:90`
Source: `caos/src/memory.c:560:588`
GUARD: `EN_MEMORY`

```
1 void ca_memory_list(void)
```

Prints a detailed report of the current heap memory allocation status to the standard output.

Ritorna:
This function returns void; it does not provide a status code. If the memory system is not initialized, it prints an error message to stdout and returns immediately.

Note:

- This function is a diagnostic tool that performs a blocking operation by acquiring a spinlock to ensure the memory list remains consistent while traversing the linked list of blocks. Because it uses a spinlock with interrupt disabling, it is technically safe to call from an ISR, but it is highly discouraged because printing to stdout is a slow, blocking operation that will cause significant system jitter. The function requires that the memory manager has been previously initialized via ca_memory_init; if called before initialization, it prints a warning. A common mistake is calling this function frequently in a high-frequency loop, which will cause the system to hang or miss real-time deadlines due to the time taken to format and transmit the string data over the serial interface.

Uso:

- This function is primarily used for debugging and system monitoring during development to detect memory leaks or fragmentation issues. It is typically triggered by a user command

via a Command Line Interface (CLI) or a diagnostic task. It is usually paired with other system diagnostic functions like ca_os_clock_dump or ca_os_banner to provide a full snapshot of the system state. Developers use it to verify that memory blocks are being correctly allocated and freed by checking the 'Free Memory' and 'Busy Memory' statistics printed at the end of the list.

Esempio:

```
/* Example of triggering the memory list from a CLI command handler */
if (ca_strncmp(cmd_buffer, "memlist", 7) == 0) {
    /* Ensure the system is stable before dumping memory */
    ca_memory_list();
} else if (ca_strncmp(cmd_buffer, "meminfo", 7) == 0) {
    /* Alternatively, check total memory stats without listing every block */
    ca_printf("Total: %u\r\n", ca_memory_get_total_memory());
    ca_printf("Available: %u\r\n", ca_memory_get_avail_memory());
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_memory_list() and why is it useful for debugging?
A: You should call ca_memory_list() during debugging or monitoring to get a detailed overview of heap memory usage. It is useful for identifying memory leaks or unexpected memory consumption patterns. The function prints memory information to stdout, which requires an active output driver.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_memory_list() with caOS types?
A: void example_usage(void) { if (g_mem_initialized) { ca_memory_list(); } }

Error/Edge case conf:0.90 **Q:** What happens if I call ca_memory_list() before initializing memory or from an ISR?
A: If memory is not initialized (g_mem_initialized is false), ca_memory_list() returns immediately without printing anything. If called from an ISR, it will not work correctly because it uses blocking functions and spinlocks, and is not ISR-safe.

Behavior conf:0.90 **Q:** Is ca_memory_list() blocking or non-blocking, and is it safe to call from multiple threads?
A: ca_memory_list() is a blocking function because it acquires a spinlock to ensure data consistency during printing. It is not thread-safe unless the caller manages the lock externally. It cannot be called from an ISR due to its use of output functions and spinlocks.

`ca_memory_reserve_secret`

Header: `caos/inc/memory.h:60`
Source: `caos/src/memory.c:468:488`
GUARD: `EN_MEMORY`

```
void *ca_memory_reserve_secret(u32 *size); /* caos.c at boot; idempotent */
```

Reserves a specific, fixed memory region for sensitive data to prevent the heap manager from overwriting it.

Parametri:

Nome	Tipo	Descrizione
`size`	`u32 *`	Pointer to a variable where the reserved size will be stored. Can be NULL if the size is not needed by the caller.

Ritorna:
Returns a void pointer to the start of the reserved secret memory area, or NULL if the reservation failed or the area is not defined.

Note:

- This function is intended to be called only during the system boot sequence before the heap is fully utilized. It is idempotent, meaning calling it multiple times has no additional effect after the first successful reservation. It is not thread-safe in the sense that it modifies global state, but it uses internal locking mechanisms to ensure the heap range is protected during the reservation process. Do not call this from an ISR, as it performs heap management operations that are not re-entrant.

Uso:

- This function is used to 'carve out' a piece of RAM that the OS heap manager must ignore, typically for security-sensitive data like encryption keys or tamper-sensitive buffers. It is usually called immediately after system initialization but before any application tasks start allocating memory. By reserving this range, you ensure that the heap allocator never places dynamic objects in this specific memory address, preventing accidental corruption or exposure of sensitive data.

Esempio:

```c
/* During system boot, reserve the secret area before starting tasks */
#include "caos.h"

void system_boot_setup(void) {
    u32 secret_size = 0;
    void *secret_ptr;

    /* Reserve the memory area defined by CA_SECRET_AREA_BASE */
    secret_ptr = ca_memory_reserve_secret(&secret_size);

    if (secret_ptr != NULL) {
        /* Successfully reserved, now we can safely use this memory */
        ca_memzero(secret_ptr, secret_size);
    } else {
        /* Handle error: memory area could not be reserved */
        ca_printf_n("Critical: Failed to reserve secret memory!\n");
    }

    /* Proceed with standard memory initialization */
    ca_memory_init();
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_memory_reserve_secret and what is its primary purpose?
A: You should call it during the system boot sequence, specifically before the heap is fully utilized and before any application tasks start allocating memory. Its purpose is to carve out a fixed memory region for sensitive data, such as encryption keys, ensuring the heap manager ignores this area to prevent accidental corruption or exposure.

Code example conf:1.00 **Q:** Provide a minimal C example showing how to reserve secret memory using caOS types.
A: u32 reserved_size = 0; void *secret_ptr = ca_memory_reserve_secret(&reserved_size); if (secret_ptr != NULL) { /* Use secret_ptr for sensitive data */ }

Parameters conf:1.00 **Q:** What is the significance of the size parameter in ca_memory_reserve_secret?

A: The size parameter is a pointer to a u32 variable where the function stores the size of the reserved memory region. You may pass NULL if you do not need to retrieve the size value.

Error/Edge case conf:1.00 **Q:** What happens if ca_memory_reserve_secret returns NULL?

A: A NULL return value indicates that the reservation failed or that the secret memory area is not defined. Developers should check for this return value to ensure the memory was successfully protected from the heap manager.

Behavior conf:1.00 **Q:** Is ca_memory_reserve_secret safe to call from an Interrupt Service Routine (ISR)?

A: No, you must not call this function from an ISR. It performs heap management operations that are not re-entrant, and while it uses internal locking to protect the heap range, it is not designed for the constraints of an interrupt context.

`ca_memory_restore_canary`

Header: `caos/inc/memory.h:187`
Source: `caos/src/memory.c:329:335`
GUARD: `EN_MEMORY`

```
void ca_memory_restore_canary(void)
```

Resets the heap memory integrity markers (canaries) to their default safe values.

Ritorna:
void

Note:

- This function is not thread-safe and should be used with extreme caution. It is intended for system recovery after a memory corruption event has been detected. It does not repair the corrupted memory itself; it only resets the 'canary' markers so that future corruption can be detected again. Calling this function while the heap is actively being accessed by other tasks will lead to race conditions and unpredictable system behavior. It is not safe to call from an ISR, as it performs a non-atomic loop over the heap metadata.

Uso:

- This function is typically used by the system's error-handling or watchdog routines after a memory violation has been identified. It is often paired with functions like ca_memory_check_canary to verify if a corruption occurred. Once the system logs the error and potentially resets the affected task, this function is called to 're-arm' the detection mechanism. It is part of the system's self-healing infrastructure rather than standard application logic, and should only be invoked when the system is in a controlled state.

Esempio:

```
/* Example: Handling a detected heap corruption */
if (ca_memory_check_canary() == false) {
    /* Log the error using system printf */
    ca_printf("CRITICAL: Heap corruption detected!\n");

    /*
     * Perform system-specific recovery actions here,
     * such as stopping non-essential tasks or resetting
     * the corrupted memory block.
     */

    /* Re-arm the canary markers to resume monitoring */
```

```
    ca_memory_restore_canary();

    /* Yield to allow the scheduler to re-evaluate task states */
    ca_os_yield();
}
```

Q&A — fine-tuning review *2/2 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_memory_restore_canary in my caOS application?
A: You should call this function in error-handling routines or system recovery tasks after a memory corruption event has been identified and logged. It is used to 're-arm' the heap integrity markers so that the system can continue monitoring for future buffer overflows after a previous violation has been acknowledged.

Code example conf:0.95 **Q:** How do I implement a recovery routine using ca_memory_restore_canary?
A: You can use it within a fault handler as follows: void handle_memory_fault(void) { /* Log the error */ ca_memory_restore_canary(); /* Re-arm monitoring */ } Note that this function is not thread-safe and should not be called from an ISR unless the memory management system is confirmed to be in a stable state.

`ca_memory_secret`

Header: `caos/inc/memory.h:61`
Source: `caos/src/memory.c:490:494`
GUARD: `EN_MEMORY`

```
void *ca_memory_secret(u32 *size)
```

Retrieves the base address and size of the system's reserved secure memory area.

Parametri:

Nome	Tipo	Descrizione
`size`	`u32 *`	Pointer to a 32-bit variable where the function will store the size of the secret area. If NULL, the size is not returned.

Ritorna:
Returns a void pointer to the start of the secret memory area. Returns NULL if no secret area has been reserved or initialized.

Note:

- This function is non-blocking and thread-safe as it simply returns a pointer to a pre-allocated static memory region. It is safe to call from an ISR, provided the memory area has been initialized during system startup. The most common mistake is failing to check if the returned pointer is NULL before accessing it, which will trigger a HardFault if the memory region is not mapped or reserved. Ensure that the system has successfully called the initialization routine that reserves this area before attempting to use it.

Uso:

- This function is typically used in security-sensitive contexts, such as storing cryptographic keys or sensitive configuration data that must be protected. It is often paired with memory protection unit (MPU) configurations to ensure the area is restricted. Developers use it during the system boot sequence to verify if a secure storage area is available before initializing key management services. It is commonly used in conjunction with memory management functions to ensure the application does not overwrite this reserved space.

Esempio:

```c
u32 secret_size = 0U;
/* Retrieve the pointer to the secure memory area */
volatile u8 *secret_ptr = (volatile u8 *)ca_memory_secret(&secret_size);

/* Always validate the pointer before access to prevent memory faults */
if (secret_ptr != NULL && secret_size >= 32U) {
    /* Perform secure operations, e.g., clearing the memory */
    ca_memzero((void *)secret_ptr, 32U);
} else {
    /* Handle the case where secure memory is not available */
    ca_printf("Secure memory area not initialized or invalid size.\n");
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_memory_secret and what is its primary purpose?
A: You should call this function during the system boot sequence to verify if a secure storage area is available before initializing services like key management. Its primary purpose is to retrieve the base address and size of a reserved secure memory area used for storing sensitive data like cryptographic keys or configuration data that requires protection, often in conjunction with MPU configurations.

Code example conf:1.00 **Q:** How do I correctly retrieve the secret memory address and its size using caOS types?
A: You must check the returned pointer for NULL to avoid a HardFault. Example: u32 size = 0; void *ptr = ca_memory_secret(&size); if (ptr != NULL) { /* Use the memory area */ }

Parameters conf:1.00 **Q:** What is the significance of the 'size' parameter in ca_memory_secret?
A: The 'size' parameter is a pointer to a u32 variable where the function stores the total size of the reserved secret memory area. If you pass NULL as the argument, the function will return the base address of the memory area but will not store the size value.

`ca_memset`

Header: `caos/inc/memaux.h:46`
Source: `caos/src/memaux.c:44:57`
GUARD: `EN_MEMAUX`

```c
void* ca_memset(volatile void *dest, u8 value, u32 count)
```

Fills a block of memory with a specific byte value.

Parametri:

Nome	Tipo	Descrizione
`dest`	`volatile void *`	Pointer to the memory block to be filled. Must not be NULL.
`value`	`u8`	The byte value to set. Every byte in the range will be set to this value.
`count`	`u32`	Number of bytes to set. If 0, the function does nothing.

Ritorna:
Returns the original 'dest' pointer on success, or NULL if the 'dest' pointer provided was NULL.

Note:

- This function is non-blocking and executes synchronously in the caller's context. It is thread-safe only if the memory region being modified is not accessed by other threads

or ISRs simultaneously. It is safe to call from an ISR, provided the memory address is valid and not currently locked by another process. A common beginner mistake is passing an incorrect 'count', which can lead to buffer overflows if the value exceeds the allocated memory size, or passing a NULL pointer without checking the return value.

Uso:

- Used primarily for clearing buffers (setting to 0) before use, initializing structures to a known state, or filling buffers with dummy data (like 0xFF for SPI read-only operations). It is typically used during the initialization phase of a driver or service to ensure no garbage data remains in memory. It is often paired with memory allocation functions or used before passing a structure to a peripheral driver to ensure all fields are properly zeroed out.

Esempio:

```
ca_rtc_alarm_t al;
/* Always clear structures before use to avoid random stack data */
ca_memset(&al, 0, sizeof(ca_rtc_alarm_t));

/* Now safe to set specific fields */
al.sec = 30;
al.match_sec = true;

/* Example of clearing a buffer before SPI transmission */
u8 buffer[16];
ca_memset(buffer, 0xFF, sizeof(buffer));
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_memset in a caOS project, and what is its primary purpose?
A: ca_memset is used to fill a block of memory with a specific byte value. It is commonly used to initialize structures or buffers before use, ensuring they start with a known state. This is especially useful in network services where data buffers need to be cleared before receiving new data.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_memset with caOS types?
A: u8 buffer[10]; void* result = ca_memset(buffer, 0xFF, 10); // buffer is now filled with 0xFF values // result points to the same buffer

Parameters conf:0.90 **Q:** What does the 'count' parameter in ca_memset control, and what happens if it is invalid?
A: The 'count' parameter specifies the number of bytes to fill in the memory block. If 'count' is 0 or less, the function does nothing. It must be greater than 0 to perform any operation.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_memset, and what error conditions should I watch out for?
A: A common mistake is passing NULL as the 'dest' parameter, which causes the function to return NULL and may lead to a crash if not checked. Another error is using a very large 'count' value, which could cause memory overwrites. Additionally, if 'dest' points to an invalid memory region, the behavior is undefined.

`ca_memset_u16`

Header: `caos/inc/memaux.h:90`
Source: `caos/src/memaux.c:109:123`
GUARD: `EN_MEMAUX`

```
void* ca_memset_u16(volatile u16 *dest, u16 value, u32 count)
```

Efficiently fills a block of 16-bit memory locations with a specific 16-bit value using 32-bit word alignment.

Parametri:

Nome	Tipo	Descrizione
`dest`	`volatile u16 *`	Pointer to the start of the memory block to be filled. Must be a valid, non-NULL memory address.
`value`	`u16`	The 16-bit pattern to write into each memory location.
`count`	`u32`	The number of 16-bit elements to fill. A value of 0 results in no operation.

Ritorna:
Returns the original pointer to the destination memory block (dest) on success, or NULL if the input pointer is invalid.

Note:

- This function is non-blocking and executes synchronously, meaning the CPU will remain occupied until the memory fill is complete. It is thread-safe in the sense that it does not rely on global state, but it does not provide mutual exclusion; if two threads write to the same memory area simultaneously, data corruption will occur. It is safe to call from an ISR, provided the destination memory is not being accessed by other tasks or interrupts at the same time. The most common beginner mistake is passing an uninitialized or NULL pointer, which will trigger the safety check and return NULL, potentially causing a crash if the return value is not checked before use.

Uso:

- This function is typically used to clear buffers, initialize arrays, or reset hardware-mapped memory regions to a default state. It is often paired with memory allocation routines or used during system initialization to ensure that peripheral buffers or data structures start with known values. Because it uses 32-bit word-aligned writes internally, it is significantly faster than a standard byte-by-byte loop, making it ideal for high-performance applications where large blocks of memory must be cleared quickly, such as clearing a frame buffer or a large communication packet buffer.

Esempio:

```
u16 my_buffer[128];

// Initialize the entire buffer to 0x0000
void* result = ca_memset_u16(my_buffer, 0x0000, 128);

if (result == NULL) {
    // Handle error: pointer was invalid
} else {
    // Buffer is now ready for use
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_memset_u16 instead of a standard memset or manual loop?
A: You should use ca_memset_u16 when initializing arrays of 16-bit values in memory with a specific value, especially in performance-critical embedded applications. It is

optimized for speed using 32-bit operations and is designed for bare-metal STM32 environments without HAL. It is ideal for initializing DMA buffers, register arrays, or any memory region where 16-bit alignment is guaranteed.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_memset_u16 with caOS types?
A: volatile u16 buffer[10]; void* result = ca_memset_u16(buffer, 0x1234, 10); // buffer is now filled with 0x1234 repeated 10 times. // result points to the memory location after the initialized buffer.

Parameters conf:0.90 **Q:** What are the valid and invalid values for the 'dest' parameter in ca_memset_u16?
A: The 'dest' parameter must be a valid pointer to a memory location that is aligned to a 16-bit boundary. If NULL, the function returns NULL. Passing an unaligned pointer will likely cause a memory fault. The memory region pointed to must be large enough to hold 'count' elements of type u16.

Error/Edge case conf:0.85 **Q:** What are the most common mistakes when calling ca_memset_u16?
A: Common mistakes include: passing a NULL pointer (which returns NULL but can cause crashes if not checked), using an unaligned memory address (leading to memory faults), and not checking the return value for NULL. Additionally, calling this function from an ISR is not allowed as it is a blocking operation and may cause system instability.

`ca_memset_u32`

Header: `caos/inc/memaux.h:72`
Source: `caos/src/memaux.c:87:96`
GUARD: `EN_MEMAUX`

```
1 void* ca_memset_u32(volatile u32 *dest, u32 value, u32 count)
```

Efficiently fills a block of 32-bit memory locations with a specific value.

Parametri:

Nome	Tipo	Descrizione
`dest`	`volatile u32 *`	Pointer to the start of the memory block to be filled. Must be 32-bit aligned.
`value`	`u32`	The 32-bit pattern to write into each memory location.
`count`	`u32`	Number of 32-bit words to write. If 0, the function does nothing.

Ritorna:
Returns a pointer to the memory address immediately following the last written word. Returns NULL if the input pointer 'dest' is NULL.

Note:

- This function is non-blocking and executes synchronously, meaning it will hold the CPU until the entire memory block is filled. It is thread-safe in the sense that it does not rely on global state, but it is not atomic; if another thread or an ISR modifies the memory while this function is running, the result will be inconsistent. It is safe to call from an ISR, provided the memory being accessed is not currently being used by another process. The most common beginner mistake is passing a byte-count instead of a word-count, or providing an unaligned pointer, which can trigger a HardFault on some ARM Cortex-M architectures.

Uso:

- This function is typically used for bulk memory initialization, such as clearing buffers, zeroing out thread control blocks (TCB), or filling stack memory with a 'magic pattern' to detect stack overflows later. It is often paired with manual byte-level handling for the 'tail' of a buffer if the total size is not a multiple of 4 bytes. In a typical lifecycle, you would allocate a memory block, use ca_memset_u32 to initialize it to a known state, and then proceed to populate it with application-specific data.

Esempio:

```c
/* Example: Initializing a thread context structure */
#include "caos.h"

void init_new_thread(ca_thread_context_t *ctx) {
    if (ctx != NULL) {
        /* Clear the entire structure to zero before setting specific fields */
        /* sizeof(ca_thread_context_t) must be a multiple of 4 for this to be safe */
        ca_memset_u32((u32*)ctx, 0, sizeof(ca_thread_context_t) / sizeof(u32));

        /* Now safe to initialize members */
        ctx->status = CA_ST_TH_INIT;
        ctx->priority = 10;
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_memset_u32 instead of a standard memset or manual loop?
A: You should use ca_memset_u32 when you need to fill a block of memory with a 32-bit value in a word-level optimized manner. It is particularly useful for initializing hardware registers, clearing memory structures, or setting up memory regions that are aligned to 32-bit boundaries. It is optimized for performance and is suitable for bare-metal environments where standard library functions may not be available or desired.

Code example conf:0.90 **Q:** Can you provide a minimal example of how to use ca_memset_u32 with caOS types?
A: volatile u32 memory_block[10]; void* result = ca_memset_u32(memory_block, 0xDEADBEEF, 10); // memory_block is now filled with 0xDEADBEEF repeated 10 times.

Parameters conf:0.95 **Q:** What happens if I pass a NULL pointer to the dest parameter of ca_memset_u32?
A: If the dest parameter is NULL, ca_memset_u32 returns NULL. This is a safety check to prevent undefined behavior when attempting to write to an invalid memory location.

`ca_memzero`

Header: `caos/inc/memaux.h:54`
Source: `caos/src/memaux.c:59:61`
GUARD: `EN_MEMAUX`

```c
void* ca_memzero(volatile void *dest, u32 count)
```

Sets a specified block of memory to zero bytes.

Parametri:

Nome	Tipo	Descrizione
`dest`	`volatile void *`	Pointer to the memory block to be cleared. Must be a valid, writable memory address.
`count`	`u32`	Number of bytes to set to zero. Must not exceed the allocated size of the destination buffer.

Ritorna:
Returns the original pointer to the destination memory block (dest).

Note:

- This function is a wrapper around ca_memset and is non-blocking, meaning it executes synchronously and returns immediately after the memory is cleared. It is thread-safe provided that no other thread or ISR is accessing the same memory region simultaneously, as concurrent access during a zeroing operation will lead to race conditions. It is safe to call from an ISR because it does not perform any OS-level blocking or context switching. The most common beginner mistake is passing an incorrect 'count' value, such as using sizeof() on a pointer instead of the struct or array it points to, which results in partial clearing or buffer overflows.

Uso:

- This function is primarily used for security-sensitive operations, such as clearing cryptographic keys or sensitive buffers before they are freed or reused, to prevent data leakage. It is also used during object initialization to ensure that structures start in a clean, predictable state (all zeros) before fields are populated. In a typical lifecycle, you would use ca_memzero immediately after allocating a buffer to avoid garbage data, or immediately before freeing a buffer that contained sensitive information. It is frequently paired with ca_memcpy when moving data and then sanitizing the source buffer.

Esempio:

```c
/* Example: Clearing a sensitive buffer after use */
u8 secret_key[32];
// ... code that populates secret_key ...

/* Always clear sensitive data before the buffer goes out of scope */
ca_memzero(secret_key, sizeof(secret_key));

/* Example: Initializing a structure */
my_struct_t config;
ca_memzero(&config, sizeof(my_struct_t));
config.id = 1; // Now safe to set fields
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_memzero to initialize a buffer before using it in caOS?
A: You should use ca_memzero to initialize a buffer or structure with zeros before using it, especially when you want to ensure that all fields are set to zero to avoid undefined behavior. This is particularly useful when working with memory that might contain garbage data from previous operations or when preparing a buffer for a known initial state.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_memzero with caOS types?
A: u8 buffer[10]; ca_memzero(buffer, 10);

Parameters conf:0.90 **Q:** What happens if I pass a NULL pointer or count = 0 to ca_memzero?
A: Passing a NULL pointer or count = 0 to ca_memzero results in undefined behavior. The function assumes that the 'dest' pointer is valid and the 'count' is greater than zero, and failure to meet these preconditions may lead to crashes or unpredictable outcomes.

ca_message_count

Header: caos/inc/message.h:126
Source: caos/src/message.c:291:294
GUARD: EN_MESSAGE

```
u16 ca_message_count(ca_message_buffer_t *mb)
```

Returns the current number of messages stored in the specified message buffer.

Parametri:

Nome	Tipo	Descrizione
mb	ca_message_buffer_t *	Pointer to the message buffer structure. Must be a valid, initialized pointer; passing NULL will result in a return value of 0.

Ritorna:
Returns an unsigned 16-bit integer representing the count of messages currently in the buffer. Returns 0 if the buffer is empty or if the provided pointer is NULL.

Note:

- This function is non-blocking and executes in constant time, making it safe to call from both standard tasks and Interrupt Service Routines (ISRs). It is thread-safe in the sense that it performs a simple read of a structure member, but be aware that the returned value may become stale immediately after the function returns if another task or interrupt adds or removes messages. A common beginner mistake is assuming the count remains constant while performing subsequent operations; always re-check the count or use appropriate synchronization primitives if you need to ensure the buffer state does not change during a critical sequence.

Uso:

- This function is typically used for monitoring the health and load of a communication channel or task queue. It is frequently paired with ca_message_space_available to calculate buffer utilization percentages or to decide if it is safe to push a new message without overflowing. Developers use it in a polling loop or a diagnostic task to print status updates to the console, ensuring that the system is not hitting its message capacity limits during high-traffic periods.

Esempio:

```
/* Check if the buffer has space before attempting to send a message */
if (ca_message_count(&msg_buffer) < MSG_MAX_COUNT) {
    /* Proceed to send message as buffer is not full */
    // ca_message_send(&msg_buffer, &my_data);
} else {
    /* Handle buffer overflow scenario */
    ca_stdout_printf_n("Warning: Buffer full, dropping message.");
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_message_count to monitor a message buffer?
A: You should use ca_message_count to monitor the current number of messages in a message buffer, especially when you need to track buffer utilization or debug buffer behavior. It is useful for checking if the buffer is approaching capacity or if it's empty, but it is not thread-safe and does not block.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_message_count with caOS types?
A: ca_message_buffer_t mb; ca_message_init(&mb, 10, sizeof(u32)); // ... send some messages ... u16 count = ca_message_count(&mb); // count now holds the number of messages in the buffer

Parameters conf:0.95 **Q:** What happens if I pass NULL to the mb parameter of ca_message_count?
A: If the mb parameter is NULL, ca_message_count returns 0. This is a safety check to prevent dereferencing a null pointer, and it indicates that no valid buffer was provided for counting messages.

Return conf:0.90 **Q:** What does a return value of 0 from ca_message_count indicate?
A: A return value of 0 indicates either that the mb parameter was NULL, or the buffer was not initialized correctly. It does not necessarily mean the buffer is empty, but rather that the function could not retrieve a valid count due to invalid input or initialization issues.

`ca_message_destroy`

Header: `caos/inc/message.h:73`
Source: `caos/src/message.c:48:60`
GUARD: `EN_MESSAGE`

```
1 void ca_message_destroy(ca_message_buffer_t *mb)
```

Safely deallocates the internal memory buffer of a message queue and resets its control structure.

Parametri:

Nome	Tipo	Descrizione
`mb`	`ca_message_buffer_t *`	Pointer to the message buffer structure to be destroyed. Must be a valid, previously initialized pointer; passing NULL is safe but does nothing.

Ritorna:
void

Note:

- This function is thread-safe because it uses a spinlock to protect the internal state during the cleanup process. It is safe to call from an ISR because it uses the IRQ-safe variant of the spinlock, which disables interrupts to prevent deadlocks. The most common mistake is attempting to use the message buffer after calling this function; since the internal buffer is freed and set to NULL, any subsequent attempt to send or receive messages will fail or cause a system crash. Always ensure that no other tasks are currently blocked on this buffer before destroying it, as the function does not explicitly wake up or notify waiting tasks.

Uso:

- This function is typically used during the teardown phase of a system component or when a dynamic communication channel is no longer needed to reclaim heap memory. It is the final step in the lifecycle of a ca_message_buffer_t, following the initialization sequence (where the buffer was created and allocated). You should pair this with the initial creation function to prevent memory leaks in long-running embedded applications. It is often called during system shutdown routines or when reconfiguring peripheral drivers that rely on message passing.

Esempio:

```c
/* Assuming mb is a pointer to an existing ca_message_buffer_t */
if (mb != NULL) {
    /* Stop any tasks from using the buffer before destruction */
    ca_message_destroy(mb);
    /* Important: set pointer to NULL to prevent dangling pointer access */
    mb = NULL;
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_message_destroy() and why is it important to ensure no other threads are using the buffer?
A: You should call ca_message_destroy() when you no longer need a message buffer and want to free its memory. It is important to ensure no other threads are using the buffer because the function is not thread-safe and may cause deadlock if called while another thread is accessing the buffer. The function assumes exclusive access to the buffer to safely reset its parameters and deallocate memory.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to properly use ca_message_destroy() with caOS types?
A: ca_message_buffer_t *mb = ca_message_create(10, sizeof(u32)); // ... use the buffer ... ca_message_destroy(mb);

Parameters conf:0.95 **Q:** What is the significance of the mb parameter in ca_message_destroy(), and what happens if it is NULL?
A: The mb parameter is a pointer to the message buffer that needs to be destroyed. If mb is NULL, the function terminates immediately without performing any action or causing errors. It must point to a valid and previously allocated message buffer for the function to perform its intended operation.

Error/Edge case conf:0.85 **Q:** What are the common mistakes developers make when using ca_message_destroy(), and what edge cases should they be aware of?
A: Common mistakes include calling ca_message_destroy() without ensuring that no other threads are reading or writing to the buffer, which can lead to deadlock or undefined behavior. Another mistake is calling it from an ISR, as the function uses spinlocks that may cause deadlock. The function is blocking if the lock is already acquired, but not for extended periods. It is also important to note that the function does not return any error code, so developers must manage buffer lifecycle manually.

`ca_message_init`

Header: `caos/inc/message.h:68`
Source: `caos/src/message.c:31:46`
GUARD: `EN_MESSAGE`

```c
bool ca_message_init(ca_message_buffer_t *mb, u32 size)
```

Initializes a message buffer structure by allocating memory and setting up synchronization primitives.

Parametri:

Nome	Tipo	Descrizione
`mb`	`ca_message_buffer_t *`	Pointer to the message buffer control block. Must not be NULL.
`size`	`u32`	Total size in bytes to allocate for the buffer. Must be at least CA_MSG_HEADER_SIZE + 1.

Ritorna:
Returns true if the buffer was successfully allocated and initialized; returns false if the pointer is NULL, the size is insufficient, or the heap allocation fails.

Note:

- This function is non-blocking and performs heap allocation via ca_memory_allocate, so it must be called before the RTOS scheduler starts or during system initialization. It is not thread-safe to call this on an already initialized buffer, as it will overwrite existing pointers and cause memory leaks. It is not safe to call from an ISR because heap allocation is generally non-deterministic and may be blocked by other tasks. The most common beginner mistake is failing to check the return value, which leads to a NULL pointer dereference when the system later attempts to write to the unallocated buffer.

Uso:

- This function is the mandatory first step when setting up inter-task communication via message buffers. It is typically called in the main initialization sequence before spawning tasks with ca_os_job. Once initialized, the buffer is used by producer tasks to send data and consumer tasks to receive data. It is usually paired with a cleanup function (if available) to free the heap memory when the buffer is no longer needed, ensuring no memory fragmentation occurs over the system's lifecycle.

Esempio:

```
ca_message_buffer_t msg_buffer;
#define MSG_BUFFER_SIZE 1024

void system_init(void) {
    // Initialize the buffer before starting any tasks
    if (!ca_message_init(&msg_buffer, MSG_BUFFER_SIZE)) {
        // Handle error: allocation failed or invalid size
        while (1) { /* Log error or reset system */ }
    }

    // Now safe to spawn tasks that use msg_buffer
    ca_os_job("writer", CA_TH_LEVEL2, task_writer, 0, STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);
    ca_os_job("reader", CA_TH_LEVEL2, task_reader, 0, STACK_MEDIUM, CA_JOB_STD, CA_FAULT_ASSERT);
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_message_init and why is it necessary before using message buffers?
A: You should call ca_message_init when you want to initialize a message buffer for inter-task communication in caOS. It is necessary because it allocates memory for the buffer and prepares internal data structures required for message handling. Without initialization, the buffer is not ready for use with other message functions like sending or receiving messages.

Code example conf:0.90 **Q:** Can you provide a minimal working C example using ca_message_init with caOS types?
A: ca_message_buffer_t mb; bool result = ca_message_init(&mb, 256); if (result) { // Buffer initialized successfully } else { // Initialization failed }

Parameters conf:0.90 **Q:** What does the 'size' parameter in ca_message_init represent, and what are valid values?
A: The 'size' parameter represents the total size in bytes of the message buffer to be allocated. It must be at least CA_MSG_HEADER_SIZE + 1. If size is smaller than this minimum, the function fails. The pointer 'mb' must be valid and not NULL.

Return conf:0.95 **Q:** What do the return values of ca_message_init indicate, and what causes it to return false?
A: ca_message_init returns true if initialization is successful, meaning the buffer is ready for use. It returns false if initialization fails, which occurs when mb is NULL or size is too small (less than CA_MSG_HEADER_SIZE + 1). In the case of failure, the buffer is not modified.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_message_init?
A: Common mistakes include passing a NULL pointer for mb or a size parameter that is less than CA_MSG_HEADER_SIZE + 1. Another error is calling ca_message_init on a buffer that has already been initialized, which may lead to memory leaks or undefined behavior. Not initializing the buffer before using other message functions is also a frequent mistake.

Behavior conf:0.90 **Q:** Is ca_message_init blocking, ISR-safe, or thread-safe, and what are its preconditions?
A: ca_message_init is not blocking; it only blocks if ca_memory_allocate fails, but does not wait for events. It is not thread-safe and should not be called from multiple tasks on the same buffer. It is not recommended to call it from ISR context due to potential issues with ca_memory_allocate. The precondition is that mb must be a valid, allocated pointer and size must be sufficient to hold the header and at least one byte of data.

`ca_message_init_static`

Header: `caos/inc/message.h:78`
Source: `caos/src/message.c:62:73`
GUARD: `EN_MESSAGE`

```
void ca_message_init_static(ca_message_buffer_t *mb, u8 *buffer, u32
    size)
```

Initializes a static message buffer structure for inter-task communication.

Parametri:

Nome	Tipo	Descrizione
`mb`	`ca_message_buffer_t *`	Pointer to the message buffer control structure. Must be a valid memory address.
`buffer`	`u8 *`	Pointer to the raw byte array used as the storage area. Must be large enough to hold data plus headers.
`size`	`u32`	Total size of the buffer in bytes. Must be greater than CA_MSG_HEADER_SIZE.

Ritorna:
void. If parameters are invalid (NULL pointers or insufficient size), the function returns immediately without initializing the buffer.

Note:

- This function is non-blocking and thread-safe as it initializes the internal spinlock. It is safe to call from an ISR, provided the memory passed is not currently being accessed by another task. A common beginner mistake is providing a buffer size that is too small; if the size is less than CA_MSG_HEADER_SIZE + 1, the buffer will be unusable, and the function will silently fail. Always ensure the buffer memory is statically allocated (e.g., global or static array) to prevent it from going out of scope, as the RTOS will continue to reference this memory address for the lifetime of the buffer.

Uso:

- This function is the mandatory first step before any message passing can occur. It is typically called during system initialization or driver setup to prepare a communication channel between tasks. You pair this with other message functions like sending or receiving data. Once initialized, the buffer remains ready for use until the system is reset or the buffer is explicitly re-initialized. It is essential to ensure that the buffer memory is not modified by other parts of the application while the message buffer is active.

Esempio:

```
static ca_message_buffer_t my_msg_buf; // Control structure
static u8 raw_storage[128]; // Raw memory for messages

void system_init(void) {
    // Initialize the buffer with the storage area
    ca_message_init_static(&my_msg_buf, raw_storage, sizeof(raw_storage));

    // Now the buffer is ready for use in tasks
    // ca_message_send(&my_msg_buf, data, len);
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_message_init_static and what is its purpose in caOS?
A: ca_message_init_static is used to initialize a static message buffer for inter-task communication in caOS. It prepares a pre-allocated memory buffer to be used as a message queue, allowing tasks to send and receive messages. This function should be called once per buffer, typically during system initialization before any tasks start using the buffer.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_message_init_static with caOS types?
A: ca_message_buffer_t mb; static u8 buffer[256]; ca_message_init_static(&mb, buffer, 256);

Parameters conf:0.85 **Q:** What are the valid values for the 'size' parameter in ca_message_init_static, and what happens if it's too small?
A: The 'size' parameter must be at least CA_MSG_HEADER_SIZE + 1 bytes. If size is smaller than this minimum, the function will not initialize the buffer correctly, but it will not return an error code. The function simply returns without performing any initialization in such cases.

Error/Edge case conf:0.80 **Q:** What are common mistakes when calling ca_message_init_static, and what edge cases should I watch out for?
A: Common mistakes include passing NULL pointers for mb or buffer, or providing a size smaller than CA_MSG_HEADER_SIZE + 1. The function is non-blocking and not thread-safe, so it must only be called from tasks or ISRs, and only once per buffer. Calling it multiple times without explicit reset can lead to undefined behavior. Also, the function does not validate that buffer is properly aligned for message storage.

`ca_message_is_empty`

Header: `caos/inc/message.h:136`
Source: `caos/src/message.c:301:304`
GUARD: `EN_MESSAGE`

```
bool ca_message_is_empty(ca_message_buffer_t *mb)
```

Checks if the specified message buffer contains zero messages.

Parametri:

Nome	Tipo	Descrizione
mb	ca_message_buffer_t *	Pointer to the message buffer structure. Must be a valid, initialized pointer.

Ritorna:
Returns true if the buffer is empty or if the provided pointer is NULL. Returns false if there is at least one message currently stored in the buffer.

Note:

- This function is non-blocking and executes in constant time, making it safe to call from both standard tasks and Interrupt Service Routines (ISRs). It is thread-safe in the sense that it performs a simple read of the internal message counter, but be aware that the buffer state might change immediately after the function returns if another task is writing to it. A common beginner mistake is assuming that if this returns true, the buffer will remain empty for subsequent operations; always use this only for status checks or logic branching, not for synchronization. Ensure the message buffer has been properly initialized via the system's setup functions before passing it here, as passing an uninitialized pointer will lead to undefined behavior.

Uso:

- This function is typically used in monitoring loops or before attempting to read data from a buffer to avoid unnecessary processing. It is often paired with 'ca_message_count' to get the exact number of items or 'ca_message_space_available' to check if there is room to write new data. In a typical producer-consumer pattern, a consumer task might call this to decide whether to enter a low-power sleep state or to process pending messages. It is also frequently used in diagnostic routines to print the current system status to a console or log file.

Esempio:

```c
/* Check if the buffer is empty before attempting to process data */
if (ca_message_is_empty(&msg_buffer)) {
    /* Buffer is empty, perform other tasks or wait */
    ca_stdout_printf_n("No messages to process.");
} else {
    /* Buffer has data, proceed to read */
    u16 count = ca_message_count(&msg_buffer);
    ca_stdout_printf_n("Processing %u messages...", count);
    /* Call function to read/pop message here */
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_message_is_empty() in my embedded application?
A: You should call ca_message_is_empty() when you need to check if there are any messages available in a message buffer before attempting to read from it. This is useful for avoiding unnecessary blocking or processing of empty buffers, especially in interrupt service routines or polling loops where you want to ensure that you only process messages when they are actually present.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_message_is_empty() with caOS types?
A: ca_message_buffer_t mb; bool is_empty; // Assume mb is initialized with ca_message_init() is_empty = ca_message_is_empty(&mb); if (is_empty) { // Handle empty buffer case }

Parameters conf:0.90 **Q:** What does the mb parameter in ca_message_is_empty() represent, and what happens if it is NULL?

A: The mb parameter is a pointer to a ca_message_buffer_t structure that must be valid and initialized. If mb is NULL, the function returns true without causing any errors, as the function is designed to handle this edge case gracefully.

Return conf:0.90 **Q:** What do the return values of ca_message_is_empty() indicate, and what causes it to return false?

A: The function returns true if the buffer is empty or if mb is NULL. It returns false if there is at least one message in the buffer. The function does not fail in the traditional sense; it simply evaluates the message count in the buffer to determine its return value.

`ca_message_peek_size`

Header: `caos/inc/message.h:121`
Source: `caos/src/message.c:274:289`
GUARD: `EN_MESSAGE`

```
u16 ca_message_peek_size(ca_message_buffer_t *mb)
```

Returns the size of the next pending message in the buffer without removing it from the queue.

Parametri:

Nome	Tipo	Descrizione
`mb`	`ca_message_buffer_t *`	Pointer to the initialized message buffer structure. Must not be NULL.

Ritorna:
Returns the size of the next message in bytes (u16). Returns 0 if the buffer is empty, if the pointer is NULL, or if no message is currently available.

Note:

- This function is non-blocking and thread-safe because it uses an IRQ-safe spinlock to protect the internal buffer state during the read operation. It is safe to call from an ISR, but keep in mind that frequent calls in high-frequency interrupts can introduce latency. The most common beginner mistake is assuming that a non-zero return value guarantees that a subsequent receive operation will succeed; if another task or interrupt receives the message between your 'peek' and your 'receive', the buffer might become empty. Always check the return value of your subsequent receive operation to handle race conditions.

Uso:

- This function is typically used in a consumer task to decide if a buffer has enough data to process or to pre-allocate a memory buffer of the correct size before calling the actual retrieval function. It is usually paired with 'ca_message_receive' in a loop where the consumer waits for a specific message size or simply checks for pending work. By using this, you avoid removing data from the queue prematurely, which is useful when you need to inspect the message header or size before committing to a full read operation.

Esempio:

```
/* Example: Checking message size before processing */
caMessageBuffer msg_buffer;

void task_consumer(void *pvParameters) {
    while (1) {
        u16 size = ca_message_peek_size(&msg_buffer);

        if (size > 0 && size <= 256) {
```

```
            /* We know the size, now we can safely receive the message */
            u8 data[256];
            /* Assuming ca_message_receive is the correct retrieval function */
            if (ca_message_receive(&msg_buffer, data, size) == 0) {
                /* Process data here */
            }
        } else {
            /* Buffer empty or message too large, wait for a bit */
            ca_os_delay_ms(10);
        }
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_message_peek_size() in my caOS application?
A: You should use ca_message_peek_size() when you need to determine the size of the next message in the queue without removing it. This is useful for pre-allocating memory before reading the message, especially when the message size is variable.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_message_peek_size()?
A: ca_message_buffer_t mb; ca_message_buffer_init(&mb, 1024); // Assume some message is already in the buffer u16 size = ca_message_peek_size(&mb); if (size > 0) { u8 *buffer = (u8*)ca_malloc(size); ca_message_read(&mb, buffer, size); ca_free(buffer); }

Parameters conf:0.90 **Q:** What does the mb parameter in ca_message_peek_size() represent and what are valid values?
A: The mb parameter is a pointer to a ca_message_buffer_t structure. It must be a valid pointer to an already initialized buffer. Passing NULL or a pointer to an uninitialized buffer will result in a return value of 0.

Return conf:0.95 **Q:** What do the return values of ca_message_peek_size() mean?
A: A return value of 0 indicates either that the buffer is NULL or that there are no messages in the queue. A positive u16 value indicates the size in bytes of the next message in the queue, which is only valid if the buffer contains at least one message.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_message_peek_size()?
A: Common mistakes include calling the function on a buffer that has not been initialized, passing a NULL pointer, or assuming the returned size is valid without checking if it is greater than 0. Calling the function on an uninitialized buffer does not cause a crash but returns 0.

Behavior conf:0.90 **Q:** Is ca_message_peek_size() blocking or thread-safe, and can it be called from an ISR?
A: ca_message_peek_size() is non-blocking and always returns immediately. It is thread-safe as it uses a spinlock internally. It can be called from an ISR, but only if the buffer was initialized to support ISR usage. The function acquires and releases a lock, so care must be taken to avoid deadlocks.

`ca_message_receive`

Header: `caos/inc/message.h:103`
Source: `caos/src/message.c:192:235`
GUARD: `EN_MESSAGE`

```
u16 ca_message_receive(ca_message_buffer_t *mb, void *data, u16 max_len,
    u32 timeout_ms)
```

Retrieves a message from a message buffer, blocking the current task if the buffer is empty until data arrives or a timeout occurs.

Parametri:

Nome	Tipo	Descrizione
`mb`	`ca_message_buffer_t *`	Pointer to the initialized message buffer handle. Must not be NULL.
`data`	`void *`	Pointer to the destination buffer where the received message will be copied. Must not be NULL.
`max_len`	`u16`	The maximum number of bytes to copy into the destination buffer. Must be greater than 0.
`timeout_ms`	`u32`	Wait time in milliseconds. Use 0 for non-blocking, or 0xFFFFFFFF for infinite wait.

Ritorna:
Returns the number of bytes copied into the data buffer (0 to max_len). Returns 0 if the buffer is empty after the timeout expires or if input parameters are invalid.

Note:

- This function is blocking, meaning the calling task will be suspended if no data is available, allowing other tasks to run. It is thread-safe due to internal spinlocks, but it must NEVER be called from an Interrupt Service Routine (ISR) because it contains blocking logic and sleep calls. A common beginner mistake is passing a timeout of 0 and expecting the task to wait for data; if the buffer is empty, it will return immediately. Ensure the message buffer is properly initialized before calling this function, otherwise, the system may crash due to a NULL pointer dereference or invalid memory access.

Uso:

- This function is typically used in a producer-consumer pattern where a task waits for data sent by another task or an ISR. It is usually paired with a message buffer initialization function at system startup. In the main loop of a consumer task, you call this function to block until work is available, process the data, and then loop back to wait for the next message. It is the standard way to synchronize data flow between tasks without busy-waiting, which saves CPU cycles and power.

Esempio:

```c
/* Assuming msg_buffer is initialized elsewhere */
Message_t msg;
u16 received_len;

/* Wait up to 200ms for a message to arrive */
received_len = ca_message_receive(&msg_buffer, &msg, sizeof(Message_t), 200);

if (received_len > 0) {
    /* Successfully received data, process it */
    ca_stdout_printf_n("Received message with length: %u", received_len);
} else {
    /* Timeout reached or error occurred */
    ca_stdout_printf_n("No message received within timeout.");
}
```

Q&A — fine-tuning review

6/6 pairs · qwen3-coder:latest

Usage conf:0.95 **Q:** When should I use ca_message_receive to get data from a message buffer?
A: You should use ca_message_receive when you need to retrieve a message from a message buffer, especially in a task that waits for data from another task or interrupt

handler. It is ideal for inter-task communication where one task sends messages and another receives them.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_message_receive with caOS types?
A: ca_message_buffer_t mb; ca_message_buffer_init(&mb, 100); uint8_t buffer[50]; u16 received = ca_message_receive(&mb, buffer, 50, 1000); if (received > 0) { // Process received message of length 'received' }

Parameters conf:0.95 **Q:** What does the max_len parameter in ca_message_receive control and what are valid values?
A: The max_len parameter specifies the maximum number of bytes that can be copied into the data buffer. It must be greater than 0; otherwise, the function returns 0. Valid values are any positive u16 integer, such as 1 to 65535.

Return conf:0.95 **Q:** What does ca_message_receive return when it fails or times out?
A: The function returns 0 when it fails or times out. This occurs if mb or data are NULL, max_len is 0, or if the timeout expires before a message is received.

Error/Edge case conf:0.90 **Q:** What are common mistakes developers make when using ca_message_receive?
A: Common mistakes include passing max_len = 0 or data = NULL, which causes an immediate return of 0. Another mistake is expecting the function to block when timeout_ms = 0, which will return immediately without waiting.

Behavior conf:0.90 **Q:** Is ca_message_receive thread-safe and can it be called from an ISR?
A: Yes, ca_message_receive is thread-safe due to the use of spinlock protection. However, it is not recommended to call it from an ISR because it may block or cause synchronization issues.

`ca_message_receive_isr`

Header: `caos/inc/message.h:115`
Source: `caos/src/message.c:260:268`
GUARD: `EN_MESSAGE`

```
u16 ca_message_receive_isr(ca_message_buffer_t *mb, void *data, u16
    max_len)
```

Reads a message from a message buffer specifically designed for use within Interrupt Service Routines (ISRs).

Parametri:

Nome	Tipo	Descrizione
`mb`	`ca_message_buffer_t *`	Pointer to the initialized message buffer structure. Must not be NULL.
`data`	`void *`	Pointer to the destination buffer where the received message will be copied. Must have enough space to hold the message.
`max_len`	`u16`	The maximum number of bytes to copy into the destination buffer. Must be greater than 0.

Ritorna:
Returns the actual number of bytes copied into the destination buffer. Returns 0 if the buffer is empty, if parameters are invalid (NULL pointers), or if max_len is 0.

Note:

- This function is explicitly designed to be safe for calling from within an Interrupt Service Routine (ISR) because it uses spinlocks that disable interrupts, preventing race conditions

between the main thread and the hardware interrupt. It is non-blocking, meaning it will return immediately even if no data is available, which is critical for maintaining low latency in interrupt handlers. A common beginner mistake is passing a destination buffer that is too small for the incoming message, which can lead to data truncation or memory corruption if the underlying msgbuf_read_msg implementation does not handle bounds checking correctly. Always ensure the message buffer has been properly initialized before calling this function, as passing an uninitialized pointer will cause a system crash.

Uso:

- This function is typically used in high-frequency data acquisition scenarios where hardware peripherals (like UART or SPI) trigger an interrupt upon receiving a packet. It is usually paired with a producer function that writes to the buffer from the main loop or another task. The standard sequence involves the hardware interrupt firing, the ISR calling this function to drain the message buffer into a local variable, and then processing that data immediately or signaling a task to handle it. It is the primary mechanism for moving data from the hardware-facing interrupt context into the application-facing processing logic without risking data corruption.

Esempio:

```c
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    u8 rx_buffer[32];
    // Attempt to read data from the message buffer inside the ISR
    u16 received_bytes = ca_message_receive_isr(&my_msg_buffer, rx_buffer,
    sizeof(rx_buffer));

    if (received_bytes > 0) {
        // Successfully retrieved data, process it immediately
        process_data(rx_buffer, received_bytes);
    } else {
        // Buffer was empty or error occurred
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_message_receive_isr instead of a regular message receive function?
A: You should use ca_message_receive_isr when you need to receive messages from a message buffer inside an ISR context. It is designed to be non-blocking and safe for use in interrupt handlers, avoiding potential deadlocks or undefined behavior that could occur with blocking operations in ISRs.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_message_receive_isr in an ISR?
A: u16 len = ca_message_receive_isr(&mb, buffer, 32); if (len > 0) { /* Process received data */ }

Parameters conf:0.95 **Q:** What happens if I pass a NULL pointer for the message buffer or data in ca_message_receive_isr?
A: Passing a NULL pointer for either the message buffer or data parameter will result in a return value of 0, indicating an error due to invalid parameters. The function requires both parameters to be valid and non-NULL.

Return conf:0.90 **Q:** What does it mean if ca_message_receive_isr returns 0?
A: A return value of 0 indicates either that one or more parameters were invalid (such as NULL pointers or max_len <= 0), or that there was nothing to read from the message buffer. It does not indicate an error in the buffer itself, but rather that no data was available or an invalid call was made.

ca_message_reset

Header: caos/inc/message.h:83
Source: caos/src/message.c:75:84
GUARD: EN_MESSAGE

```
void ca_message_reset(ca_message_buffer_t *mb)
```

Clears all data and resets the internal pointers of a message buffer to an empty state.

Parametri:

Nome	Tipo	Descrizione
mb	ca_message_buffer_t *	A pointer to the message buffer structure to be reset. Must be a valid, initialized pointer; passing NULL will result in a silent return.

Ritorna:
This function returns void and does not provide a status code.

Note:

- This function is thread-safe because it uses a spinlock to protect the internal buffer variables during the reset process. It is non-blocking in the sense that it does not wait for other tasks, but it will briefly disable interrupts to acquire the spinlock, so it should not be called in time-critical sections. It is safe to call from an ISR because it uses the 'irq' variant of the spinlock functions. The most common mistake is calling this function on a buffer that has not been initialized yet, which will lead to a crash when the function attempts to access the uninitialized spinlock member.

Uso:

- Use this function when you need to flush a message buffer, such as during a system reinitialization, a manual user command via CLI, or when recovering from a buffer overflow error. It is typically used to clear stale data before starting a new communication sequence or after a communication timeout. It is often paired with the initialization function that sets up the buffer structure, and it is frequently triggered by administrative tasks or error-handling routines to ensure the buffer starts from a clean, predictable state.

Esempio:

```
/* Assuming msg_buffer is a global ca_message_buffer_t instance */

void handle_reset_command(void) {
    /* Reset the buffer to clear any pending or corrupted messages */
    ca_message_reset(&msg_buffer);

    /* Log the action to the user */
    CLI_PUT_N("System buffer has been cleared.");
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_message_reset() and what is its purpose in a caOS application?
A: You should call ca_message_reset() to clear the contents of a message buffer and reset its internal counters to zero. This function is useful for restoring the buffer's state after an error or system reset. It ensures that the buffer starts fresh, but it does not wake up any tasks that might be blocked waiting for messages.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_message_reset() with caOS types?
A: ca_message_buffer_t mb; ca_message_init(&mb, 10, sizeof(u32)); // ... use the buffer ... ca_message_reset(&mb);

Parameters conf:0.90 **Q:** What is the significance of the mb parameter in ca_message_reset(), and what happens if it is NULL?
A: The mb parameter is a pointer to the ca_message_buffer_t that needs to be reset. If mb is NULL, the function returns immediately without performing any action. It is essential that the buffer pointed to by mb has been properly initialized before calling this function.

---

ca_message_send

Header: caos/inc/message.h:93
Source: caos/src/message.c:155:190
GUARD: EN_MESSAGE

```
bool ca_message_send(ca_message_buffer_t *mb, const void *data, u16 len,
    u32 timeout_ms)
```

Sends a data packet to a message buffer, blocking the calling task until space is available or the timeout expires.

Parametri:

Nome	Tipo	Descrizione
mb	ca_message_buffer_t *	Pointer to the initialized message buffer handle. Must not be NULL.
data	const void *	Pointer to the memory buffer containing the data to send. Must not be NULL.
len	u16	Length of the data in bytes. Must be greater than 0 and fit within the buffer's remaining capacity.
timeout_ms	u32	Maximum time to wait in milliseconds. Use 0 for non-blocking, or 0xFFFFFFFF for infinite wait.

Ritorna:
Returns true if the message was successfully copied into the buffer. Returns false if the buffer is full after the timeout, if parameters are invalid, or if the message size exceeds the buffer capacity.

Note:

- This function is blocking, meaning the calling task will pause execution if the buffer is full until space becomes available or the timeout is reached. It is thread-safe due to internal spinlocks, but it must NOT be called from an Interrupt Service Routine (ISR) because it uses ca_os_sleep() which triggers a context switch. A common beginner mistake is passing a timeout of 0 and expecting the system to wait, or failing to check the boolean return value, which leads to silent data loss when the buffer is full. Ensure the message buffer is properly initialized before calling this function, otherwise, the system will likely crash due to a NULL pointer dereference.

Uso:

- This function is typically used in producer-consumer patterns where one task generates data and another processes it. It is usually paired with ca_message_receive() on the consumer side. The typical workflow involves initializing the buffer once at system startup,

then calling ca_message_send() inside a loop or event-driven task to pass data packets. If the return value is false, the application should implement a retry strategy or log an error, as it indicates the consumer task is not processing data fast enough to keep up with the producer.

Esempio:

```c
/* Assuming msg_buffer is initialized globally */
Message_t my_data = { .id = 1, .value = 42 };

// Attempt to send data with a 500ms timeout
if (ca_message_send(&msg_buffer, &my_data, sizeof(Message_t), 500)) {
    // Success: data is now in the buffer
} else {
    // Failure: buffer was full or timeout occurred
    // Handle error: log, drop packet, or retry later
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_message_send to send a message in caOS?
A: You should use ca_message_send when you need to send a message to a message buffer in a thread-safe way, and you are okay with the function blocking until space is available or the timeout expires. It is ideal for inter-task communication where you want to ensure messages are delivered reliably.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_message_send with caOS types?
A: ca_message_buffer_t mb; ca_message_buffer_init(&mb, 1024); uint8_t data[] = {0x01, 0x02, 0x03}; bool result = ca_message_send(&mb, data, 3, 1000); if (result) { // Message sent successfully } else { // Failed to send message }

Parameters conf:0.95 **Q:** What does the timeout_ms parameter control in ca_message_send?
A: The timeout_ms parameter controls how long the function will wait for space in the buffer. If set to 0, the function does not block and returns immediately. If set to 0xFFFFFFFF, it waits indefinitely. Any other value specifies the maximum time in milliseconds to wait before giving up.

Return conf:0.90 **Q:** What does ca_message_send return when it fails to send a message?
A: It returns false when the send fails. This can happen due to invalid parameters (like NULL buffer or data, zero length), message too large for the buffer, or if the timeout expires before space becomes available.

Error/Edge case conf:0.85 **Q:** What are common mistakes when calling ca_message_send?
A: Common mistakes include passing NULL for the buffer or data pointer, sending a message with zero length, using a timeout of 0 when expecting blocking behavior, or calling the function from an ISR which is not safe due to potential sleep functions.

Behavior conf:0.90 **Q:** Is ca_message_send thread-safe and ISR-safe?
A: Yes, ca_message_send is thread-safe because it uses a spinlock to protect access to the message buffer. However, it is not ISR-safe because it uses a sleep function (ca_os_sleep) internally, which could cause issues in interrupt contexts.

`ca_message_send_isr`

Header: `caos/inc/message.h:109`
Source: `caos/src/message.c:241:258`
GUARD: `EN_MESSAGE`

```
bool ca_message_send_isr(ca_message_buffer_t *mb, const void *data, u16
    len)
```

Sends a message to a message buffer specifically from within an Interrupt Service Routine (ISR).

Parametri:

Nome	Tipo	Descrizione
mb	ca_message_buffer_t *	Pointer to the initialized message buffer handle. Must not be NULL.
data	const void *	Pointer to the memory buffer containing the data to send. Must not be NULL.
len	u16	Length of the data in bytes. Must be greater than 0.

Ritorna:
Returns true if the message was successfully written to the buffer and any waiting task was signaled. Returns false if the buffer is full, parameters are invalid, or the write operation failed.

Note:

- This function is designed exclusively for use inside Interrupt Service Routines (ISRs) because it uses spinlocks that disable interrupts to ensure thread safety without causing deadlocks. It is non-blocking, meaning it will return immediately if the buffer is full rather than waiting for space. A common beginner mistake is calling this function from a standard task context instead of the non-ISR version; while it might work, it is inefficient and ignores the specific synchronization requirements of the RTOS. Always ensure the message buffer is properly initialized before calling this, or the system will likely crash due to a null pointer dereference.

Uso:

- This function is typically used in hardware interrupt handlers, such as a UART RX interrupt or a timer callback, to pass data from the hardware peripheral to a processing task. It is usually paired with a 'receive' function in the task context, where the task waits for data to arrive. The typical sequence is: the ISR triggers, it calls ca_message_send_isr to push data into the buffer, and the RTOS scheduler then wakes up the task that was blocked waiting for that specific message buffer.

Esempio:

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    // Assuming 'my_msg_buf' is a global ca_message_buffer_t handle
    uint8_t rx_data = UART_Read_Byte();

    // Send the byte to the buffer from the ISR
    bool success = ca_message_send_isr(&my_msg_buf, &rx_data, 1);

    if (!success) {
        // Handle buffer overflow or error
        // Log error or increment a drop counter
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_message_send_isr instead of a regular message send function?
A: You should use ca_message_send_isr when you need to send a message from within an Interrupt Service Routine (ISR). This function is specifically designed for ISR contexts

and does not block, making it safe for use in interrupt handlers where blocking is not allowed.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_message_send_isr in an ISR?
A: ca_message_buffer_t mb; ca_message_buffer_init(&mb, 1024); void isr_handler(void) { u8 message_data[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}; ca_message_send_isr(&mb, message_data, 10); }

Parameters conf:0.95 **Q:** What are the valid values for the 'len' parameter in ca_message_send_isr?
A: The 'len' parameter must be greater than 0. If it is 0, the function will return false. The parameter specifies the number of bytes to send from the data buffer.

Return conf:0.95 **Q:** What does it mean when ca_message_send_isr returns false?
A: A return value of false indicates that the message send failed. This happens if any of the following are true: mb is NULL, data is NULL, or len is 0.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_message_send_isr?
A: Common mistakes include calling this function from a regular task instead of an ISR, using an uninitialized message buffer, passing a NULL data pointer, or not checking the return value for failure conditions.

Behavior conf:0.95 **Q:** Is ca_message_send_isr blocking or non-blocking, and is it safe to call from an ISR?
A: ca_message_send_isr is non-blocking and is specifically designed to be called from an ISR. It uses a spinlock to ensure thread safety and avoids operations that could cause issues in interrupt contexts.

`ca_message_space_available`

Header: `caos/inc/message.h:131`
Source: `caos/src/message.c:296:299`
GUARD: `EN_MESSAGE`

```
1 u32 ca_message_space_available(ca_message_buffer_t *mb)
```

Returns the number of bytes currently available for writing in a message buffer.

Parametri:

Nome	Tipo	Descrizione
`mb`	`ca_message_buffer_t *`	Pointer to the initialized message buffer structure. Must not be NULL.

Ritorna:
Returns the number of free bytes (u32) remaining in the buffer. Returns 0 if the provided pointer is NULL or if the buffer is completely full.

Note:

- This function is non-blocking and executes in constant time, making it safe to call from both standard tasks and Interrupt Service Routines (ISRs). It is thread-safe in the sense that it performs a simple read operation, but be aware that the returned value may become stale immediately if another task or interrupt writes to the buffer simultaneously. A common beginner mistake is assuming the returned value guarantees that a subsequent write operation will succeed; if an interrupt occurs between checking the space and performing the write, the buffer might fill up, leading to a failed write operation. Always ensure the buffer has been properly initialized before passing the pointer to this function.

Uso:

- This function is typically used as a guard check before attempting to send data into a message buffer to prevent buffer overflow or blocking. It is frequently paired with functions like 'ca_message_send' or 'ca_message_peek_size' to manage data flow dynamically. In a typical application, you would call this in a loop to monitor buffer health or to decide if a task should yield execution until more space becomes available. It is essential for implementing flow control in producer-consumer patterns where the producer must not overwhelm the consumer.

Esempio:

```c
/* Check if there is enough space before sending a message */
if (ca_message_space_available(&msg_buffer) >= sizeof(my_data)) {
    /* Safe to send data */
    // ca_message_send(&msg_buffer, &my_data, sizeof(my_data));
} else {
    /* Handle buffer full condition, e.g., wait or drop packet */
    ca_stdout_printf_n("Buffer full, cannot send data");
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_message_space_available and why is it useful in a real-time system?
A: You should call ca_message_space_available to check how much space is available in a message buffer before attempting to write new messages. This is useful in real-time systems to avoid buffer overflows and to make decisions about when to send or drop messages. Since the function is non-blocking and can be called from ISRs, it allows for efficient and safe buffer management in time-critical applications.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_message_space_available with caOS types?
A: ca_message_buffer_t mb; ca_message_init(&mb, 1024); // Check available space u32 space = ca_message_space_available(&mb); if (space > 0) { // Write message ca_message_write(&mb, (u8*)"Hello", 5); }

Parameters conf:0.90 **Q:** What is the significance of the mb parameter in ca_message_space_available, and what happens if it is NULL or invalid?
A: The mb parameter is a pointer to a ca_message_buffer_t that must be valid and initialized. If mb is NULL or the buffer is not properly initialized, the function returns 0. This is not a fatal error but indicates that no space is available for writing. The function does not validate the buffer's internal state beyond checking for NULL, so it's the caller's responsibility to ensure the buffer is correctly initialized.

Return conf:0.95 **Q:** What do the return values of ca_message_space_available indicate, and what causes it to return 0?
A: The function returns 0 if the mb parameter is NULL or if the buffer is not valid. It returns a positive u32 value indicating the number of bytes available for writing new messages. The value can range from 0 to mb->size. A return value of 0 does not indicate an error but simply means no space is available in the buffer.

`ca_modbus_disconnect`

Header: `caos/net/inc/modbus_service.h:81`
Source: `caos/net/src/modbus_service.c:214:220`
GUARD: `EN_MODBUS_SERVICE`

```c
void ca_modbus_disconnect(void)
```

Gracefully terminates an active Modbus TCP connection and updates the internal connection status.

Ritorna:
This function returns void and does not provide feedback on the success of the underlying TCP closure.

Note:

- This function is non-blocking and performs a clean shutdown of the TCP socket associated with the Modbus service. It is not thread-safe by itself; if multiple tasks access the Modbus state, you must ensure external synchronization to prevent race conditions where the connection status is checked while it is being closed. It is generally safe to call from an ISR, but be aware that the underlying ca_tcp_close might involve complex stack operations that could be heavy for high-frequency interrupts. The most common beginner mistake is calling this function without checking if the connection is already closed, or assuming that calling this immediately frees up resources for a new connection before the TCP stack has finished its handshake process.

Uso:

- This function is used to manually drop a Modbus connection, typically during a system shutdown sequence, a network error recovery routine, or when switching between different Modbus masters. It is usually paired with a connection check function like ca_modbus_is_connected to ensure the state is consistent before attempting to reconnect. In a typical lifecycle, you would initialize the service, use ca_modbus_is_connected to verify the link, and call ca_modbus_disconnect when the communication session is finished or when a timeout occurs to prevent resource leakage in the TCP stack.

Esempio:

```
if (ca_modbus_is_connected()) {
    // Log the disconnection event before closing
    // Perform any necessary cleanup of Modbus registers here
    ca_modbus_disconnect();
} else {
    // Connection was already down, no action needed
}
```

Q&A — fine-tuning review *2/2 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_modbus_disconnect() and what are the risks of calling it incorrectly?
A: You should call ca_modbus_disconnect() when you want to close an active TCP Modbus connection and disable communication. It is useful for conserving resources or shutting down communication when it is no longer needed. However, calling this function without first checking if a connection is active can lead to crashes or undefined behavior, as it does not perform any connection state checks internally. Additionally, it is not thread-safe and must not be called from an ISR.

Code example conf:0.85 **Q:** Can you provide a minimal C example showing how to use ca_modbus_disconnect() with caOS types?
A: void example_modbus_disconnect(void) { // Assuming Modbus is initialized and connected ca_modbus_disconnect(); // Close the Modbus TCP connection // The function may return an error code from ca_tcp_close, but we do not handle it here }

`ca_modbus_is_connected`

Header: `caos/net/inc/modbus_service.h:78`
Source: `caos/net/src/modbus_service.c:209:212`

GUARD: `EN_MODBUS_SERVICE`

```
bool ca_modbus_is_connected(void)
```

Checks if the Modbus TCP service currently has an active network connection.

Ritorna:
Returns true if the Modbus TCP connection is established and active, false if the connection is down or not yet initialized.

Note:

- This function is non-blocking and thread-safe, as it simply reads a boolean flag maintained by the internal service state. It is safe to call from both main-loop tasks and Interrupt Service Routines (ISRs) because it performs a simple memory read without waiting for hardware or mutexes. A common beginner mistake is to assume that a 'true' return value guarantees that the remote server is still responding to data requests; this function only confirms that the TCP socket layer is up. Always ensure the Modbus service has been initialized via ca_modbus_service_init before relying on this status, otherwise, it will return false by default.

Uso:

- This function is primarily used in a 'watchdog' pattern within the main application loop to monitor connection health. It is typically paired with ca_modbus_service_init to trigger a reconnection attempt if the connection drops due to network instability or server timeouts. By checking this status periodically, you can implement robust recovery logic that attempts to re-initialize the service only when necessary, preventing unnecessary resource allocation while the connection is already healthy.

Esempio:

```
/* Periodic check to ensure Modbus connectivity */
#define RECONNECT_INTERVAL_MS 5000
static u32 last_check = 0;

void task_modbus_monitor(void) {
    if (!ca_modbus_is_connected()) {
        u32 now = ca_os_tick_ms();
        if ((now - last_check) >= RECONNECT_INTERVAL_MS) {
            last_check = now;
            /* Attempt to re-initialize the service if connection is lost */
            ca_modbus_service_init(server_ip, MODBUS_SERVER_PORT, local_ip,
    s_mac, MODBUS_UNIT_ID);
        }
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_modbus_is_connected() in my Modbus TCP client implementation?
A: You should call ca_modbus_is_connected() before sending any Modbus requests to ensure the TCP connection is active. This function is useful in scenarios where you want to avoid sending data over a broken connection, preventing potential timeouts or errors. It's particularly important to check the connection status before attempting to read or write registers, as the function helps prevent sending data when the client is not connected.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_modbus_is_connected() with caOS types?

A: bool connected = ca_modbus_is_connected(); if (connected) { // Proceed with Modbus operations u8 data[2]; // ... perform read/write operations } else { // Handle disconnection // Reconnect logic may be needed }

Return conf:0.90 **Q:** What does it mean when ca_modbus_is_connected() returns false, and what causes this failure?

A: When ca_modbus_is_connected() returns false, it indicates that the Modbus TCP connection is not active (tcp_up = 0). This typically means the client is not connected to the Modbus server and may require a reconnection attempt. The function accesses a global variable s.tcp_up, which is set by the connection initialization process, so the failure is caused by the connection not being properly established or having been lost.

`ca_modbus_read_coils`

Header: `caos/net/inc/modbus_service.h:96`
Source: `caos/net/src/modbus_service.c:226:244`
GUARD: `EN_MODBUS_SERVICE`

```
bool ca_modbus_read_coils(u16 addr, u16 count, u8 *out)
```

Sends a Modbus Function Code 01 request to read the status of multiple coils from a remote device.

Parametri:

Nome	Tipo	Descrizione
`addr`	`u16`	The starting address of the first coil to read. Must be a valid address supported by the slave device.
`count`	`u16`	The number of coils to read. Must be between 1 and 2000 inclusive.
`out`	`u8 *`	Pointer to a buffer where the coil data will be stored. Must be large enough to hold (count + 7) / 8 bytes.

Ritorna:
Returns true if the request was successful and the response was validated; returns false if the parameters are invalid, the transaction fails, or the received data is incomplete.

Note:

- This function is blocking, meaning the calling task will wait until the Modbus transaction completes or times out. It is not thread-safe if multiple tasks attempt to access the same Modbus bus simultaneously, so ensure you have a mutex or task-level synchronization if your application is multi-threaded. It must NOT be called from an Interrupt Service Routine (ISR) because it performs blocking I/O operations. A common beginner mistake is providing an 'out' buffer that is too small for the requested 'count', which will cause a memory corruption (buffer overflow) when the function copies the received data.

Uso:

- This function is used in Modbus master applications to poll the status of digital outputs or flags from a slave device. It is typically used in a periodic task loop after the Modbus service has been initialized. It is often paired with other Modbus functions like ca_modbus_write_coil or ca_modbus_read_registers to perform full device monitoring. Before calling this, ensure the communication interface (e.g., RS485/UART) is properly initialized and the slave device is online.

Esempio:

```
/* Define a buffer to hold 8 coils (1 byte) */
u8 coil_data[1];

/* Attempt to read 8 coils starting at address 0x100 */
if (ca_modbus_read_coils(0x0100, 8U, coil_data)) {
    /* Success: process the bits in coil_data[0] */
    ca_stdout_printf_n("Coils read: 0x%02X", (u32)coil_data[0]);
} else {
    /* Failure: handle communication error or invalid response */
    ca_stdout_put_n("Failed to read coils");
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use the ca_modbus_read_coils function in a caOS application?
A: You should use ca_modbus_read_coils when you need to read one or more coil bits from a Modbus device in a blocking manner. It is typically used in applications where you need to interact with Modbus slaves, such as reading digital inputs or controlling outputs via Modbus RTU or TCP protocols. The function is designed for scenarios where you require reliable, synchronous communication with Modbus devices.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_modbus_read_coils with caOS types?
A: u8 buffer[250]; bool success = ca_modbus_read_coils(100, 50, buffer); if (success) { // Process the coil data in buffer } else { // Handle error }

Parameters conf:0.95 **Q:** What do the addr and count parameters in ca_modbus_read_coils represent, and what are valid ranges?
A: The addr parameter specifies the starting coil address (0-65535). Values less than 0 or greater than 65535 are invalid. The count parameter specifies how many coils to read (1-2000). A count of 0 or greater than 2000 is invalid. These parameters define the range of coils to be read from the Modbus device.

Return conf:0.90 **Q:** What does it mean when ca_modbus_read_coils returns true or false?
A: The function returns true if the Modbus read operation completed successfully and the data was copied into the output buffer. It returns false if an error occurred during the read operation, such as invalid parameters or communication failure. The caller must check the return value to ensure data integrity.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_modbus_read_coils?
A: Common mistakes include not checking the return value, passing NULL as the output buffer, or using invalid parameter values such as addr < 0 or count > 2000. Another frequent error is calling this function from an ISR, which can cause synchronization issues or deadlock. Additionally, failing to initialize the Modbus communication structure 's' properly can lead to undefined behavior.

Behavior conf:0.90 **Q:** Is ca_modbus_read_coils blocking or non-blocking, and can it be called from an ISR?
A: ca_modbus_read_coils is a blocking function, meaning it waits for the Modbus transaction to complete before returning. It is not safe to call from an ISR because it may cause synchronization problems or deadlock. The function is thread-safe if the underlying 'transact' function is thread-safe, and it requires that the Modbus structure 's' is properly initialized before use.

---

ca_modbus_read_holding_regs

Header: caos/net/inc/modbus_service.h:110
Source: caos/net/src/modbus_service.c:250:270
GUARD: EN_MODBUS_SERVICE

```
bool ca_modbus_read_holding_regs(u16 addr, u16 count, u16 *out)
```

Sends a Modbus Function Code 03 request to read a sequence of holding registers from a remote device.

Parametri:

Nome	Tipo	Descrizione
addr	u16	The starting register address to read. Must be a valid address supported by the target Modbus slave device.
count	u16	Number of registers to read. Must be between 1 and 125 inclusive, as per Modbus protocol limits.
out	u16 *	Pointer to an array where the retrieved register values will be stored. Must be large enough to hold 'count' elements.

Ritorna:
Returns true if the transaction was successful and data was parsed correctly. Returns false if the input parameters are invalid, the transaction fails (e.g., timeout or communication error), or the slave returns an insufficient byte count.

Note:

- This function is blocking, meaning the calling task will wait until the Modbus transaction completes or times out. It is not thread-safe if multiple tasks attempt to access the Modbus bus simultaneously; ensure you have a mutex or semaphore protecting the bus access if your application is multi-threaded. It must NOT be called from an ISR (Interrupt Service Routine) because it performs blocking I/O operations. A common beginner mistake is providing an 'out' buffer that is smaller than the 'count' requested, which will cause a buffer overflow and memory corruption.

Uso:

- This function is typically used in a periodic task to poll sensor data or configuration registers from a Modbus slave device. It is usually paired with other Modbus functions like 'ca_modbus_write_single_reg' to manage device state. Before calling this, ensure the Modbus communication interface is initialized. It is standard practice to check the boolean return value; if it returns false, the application should handle the communication error, perhaps by logging the failure or attempting a retry after a short delay.

Esempio:

```
u16 sensor_data[4];
// Attempt to read 4 registers starting from address 0x100
if (ca_modbus_read_holding_regs(0x100, 4U, sensor_data)) {
    // Successfully read data, process the values
    process_data(sensor_data[0]);
} else {
    // Handle communication failure
    ca_stdout_put_n("Failed to read Modbus registers");
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_modbus_read_holding_regs in a caOS application?
A: You should use ca_modbus_read_holding_regs when you need to read 1-125 holding registers from a Modbus device. It is suitable for applications where you are communicating over a Modbus network (e.g., UART or TCP) and require a blocking read of register values. This function is not appropriate for real-time systems where blocking is not allowed.

Code example conf:0.95 **Q:** Can you show a minimal C code example using ca_modbus_read_holding_regs with caOS types?
A: u16 registers[10]; bool success = ca_modbus_read_holding_regs(100, 10, registers); if (success) { // Process the 10 register values in 'registers' } else { // Handle read failure }

Parameters conf:0.95 **Q:** What are the valid ranges for the 'addr' and 'count' parameters in ca_modbus_read_holding_regs?
A: The 'addr' parameter must be between 0 and 65535 inclusive. Values outside this range are invalid. The 'count' parameter must be between 1 and 125 inclusive. Values of 0 or greater than 125 are invalid.

Return conf:0.95 **Q:** What does it mean when ca_modbus_read_holding_regs returns false?
A: A return value of false indicates that the function failed. This can happen due to invalid parameters such as count=0, count>125, or a NULL pointer for 'out', or because the Modbus communication failed.

Error/Edge case conf:0.90 **Q:** What are common mistakes when calling ca_modbus_read_holding_regs?
A: Common mistakes include passing a NULL pointer for the 'out' parameter, providing a 'count' value outside the valid range (1-125), or passing an 'addr' value outside the valid range (0-65535). Additionally, calling this function without a properly initialized Modbus communication channel may lead to failure.

Behavior conf:0.95 **Q:** Is ca_modbus_read_holding_regs thread-safe and ISR-safe?
A: No, ca_modbus_read_holding_regs is not thread-safe and not ISR-safe. It is a blocking function that should not be called from an ISR. If used from multiple threads, it may cause data corruption or unpredictable behavior.

`ca_modbus_service_init`

Header: `caos/net/inc/modbus_service.h:64`
Source: `caos/net/src/modbus_service.c:167:192`
GUARD: `EN_MODBUS_SERVICE`

```
1 void ca_modbus_service_init(ca_ipv4_t server_ip, u16 server_port,
    ca_ipv4_t local_ip, ca_mac_t local_mac, u8 unit_id)
```

Initializes the internal Modbus TCP service state, configures network parameters, and prepares synchronization primitives.

Parametri:

Nome	Tipo	Descrizione
`server_ip`	`ca_ipv4_t`	The IPv4 address of the remote Modbus server (PLC or gateway). Must be a valid network-order structure.
`server_port`	`u16`	The TCP port number for the Modbus service, typically 502.
`local_ip`	`ca_ipv4_t`	The static IPv4 address assigned to the STM32 device.
`local_mac`	`ca_mac_t`	The unique MAC address of the device's network interface.
`unit_id`	`u8`	The Modbus slave/unit identifier (1-247). Use 0xFF for broadcast if supported.

Ritorna:
void

Note:

- This function is non-blocking and performs internal memory clearing and synchronization object initialization. It is NOT thread-safe; it must be called during the system startup phase before any Modbus communication tasks are spawned. If called while a connection is active, it will forcefully close the existing TCP handle, which may lead to unexpected behavior if other tasks are currently using the socket. A common beginner mistake is calling this function multiple times during runtime; this will reset the internal state, clear the transaction ID counter, and potentially cause memory leaks or race conditions if the RTOS tasks are not suspended first.

Uso:

- This function is the mandatory entry point for the Modbus stack. It is typically called once during the system initialization sequence, immediately after the network interface is brought up but before the Modbus polling task starts. It is usually followed by a call to ca_modbus_service_net_ready() to signal the stack that the network layer is prepared to establish the TCP connection. It pairs with the internal state management to ensure that the mutexes and semaphores required for thread-safe access to the Modbus buffer are correctly allocated.

Esempio:

```
/* Initialize Modbus service with static network configuration */
ipv4_t server_ip = {{192, 168, 1, 10}};
ipv4_t local_ip = {{192, 168, 1, 50}};
mac_t local_mac = {{0x00, 0x80, 0xE1, 0x00, 0x00, 0x01}};

/* Setup the service before starting the network stack */
ca_modbus_service_init(server_ip, 502, local_ip, local_mac, 1);

/* Signal that the network is ready to proceed with connection */
ca_modbus_service_net_ready();
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_modbus_service_init in a caOS application?
A: You should call ca_modbus_service_init once during application initialization to configure the Modbus TCP service. It sets up the network parameters like IP address, port, MAC address, and unit ID, and prepares internal resources for communication. It's typically called after network initialization but before starting any Modbus operations.

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_modbus_service_init with caOS types?

A: ca_ipv4_t server_ip = {192, 168, 1, 100}; u16 server_port = 502; ipv4_t local_ip = {192, 168, 1, 50}; mac_t local_mac = {0x00, 0x01, 0x02, 0x03, 0x04, 0x05}; u8 unit_id = 1; ca_modbus_service_init(server_ip, server_port, local_ip, local_mac, unit_id);

Parameters conf:0.90 **Q:** What are the valid ranges for server_port and unit_id in ca_modbus_service_init?
A: The server_port must be a valid TCP port in the range 1-65535 and cannot be zero. The unit_id must be a value between 0 and 255, and cannot be zero if the server is being used.

Return conf:0.95 **Q:** Does ca_modbus_service_init return any value, and what does it indicate?
A: The function does not return any value. It is void. It cannot fail explicitly, but invalid parameters may cause undefined behavior. It resets internal state and initializes semaphores and mutexes.

Error/Edge case conf:0.85 **Q:** What happens if I pass an invalid IP address or MAC to ca_modbus_service_init
A: If invalid parameters like zero IP addresses or invalid MAC arrays are passed, the behavior is undefined. The function does not validate these inputs explicitly, so passing invalid data may lead to unpredictable results or crashes.

Behavior conf:0.90 **Q:** Is ca_modbus_service_init thread-safe and ISR-safe?
A: The function is not thread-safe if called from multiple threads without external synchronization. It is also not ISR-safe and must not be called from interrupt service routines. It may block only when closing an existing TCP connection.

`ca_modbus_service_net_ready`

Header: `caos/net/inc/modbus_service.h:75`
Source: `caos/net/src/modbus_service.c:194:207`
GUARD: `EN_MODBUS_SERVICE`

```
1 bool ca_modbus_service_net_ready(void)
```

Initializes and triggers the TCP connection process for the Modbus service.

Ritorna:
Returns true if the TCP connection handle was successfully allocated (handle != 0xFFU), otherwise returns false if the connection request failed.

Note:

- This function is non-blocking; it initiates the TCP handshake process but does not wait for the connection to be established. It is not thread-safe and should not be called from an Interrupt Service Routine (ISR) as it performs memory operations and handle allocation. A common mistake is calling this function repeatedly in a loop without checking if a connection is already active, which can lead to resource exhaustion or multiple overlapping connection attempts. Ensure that the Modbus service has been fully initialized via ca_modbus_service_init before calling this, otherwise, the internal configuration structure 's' will contain invalid data.

Uso:

- This function is typically used during the system startup phase or within a state machine that manages network recovery. It is the bridge between the static configuration (IP, MAC, Port) and the active network stack. It is usually paired with ca_modbus_service_init to set up parameters, and the resulting connection status is monitored via the callback functions (on_tcp_connected, on_tcp_closed) provided in the configuration structure. In a robust application, you would call this once at boot, and then again only if the 'on_tcp_closed' callback indicates that the connection has been lost.

Esempio:

```
/* Initialize the service with network parameters first */
ca_modbus_service_init(server_ip, 502, local_ip, mac_addr, 1);

/* Attempt to open the TCP connection */
if (ca_modbus_service_net_ready()) {
    ca_stdout_put_n("Connection request sent successfully.");
} else {
    ca_stdout_put_n("Failed to initiate TCP connection.");
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_modbus_service_net_ready() in my Modbus TCP application?
A: You should call ca_modbus_service_net_ready() after setting up the network configuration parameters such as IP address, port, and MAC address. It checks if the TCP connection for Modbus has been initialized correctly. This function is typically used before attempting to send or receive Modbus data to ensure the network is ready.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_modbus_service_net_ready() with caOS types?
A: u8 result = ca_modbus_service_net_ready(); if (result) { // TCP connection is ready, proceed with Modbus operations } else { // Handle connection failure }

Return conf:0.95 **Q:** What does it mean when ca_modbus_service_net_ready() returns false?
A: When ca_modbus_service_net_ready() returns false, it indicates that the TCP connection for Modbus was not initialized correctly. This could be due to errors in allocation or configuration of the connection, and the function sets s.tcp_handle to 0xFFU to signal this failure state.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_modbus_service_net_ready()?
A: A common mistake is ignoring the return value of ca_modbus_service_net_ready() and proceeding without verifying if the connection is actually ready. Another error is calling this function before configuring network parameters like IP, port, or MAC address. Additionally, since it's not thread-safe and must not be called from ISR, doing so can lead to unpredictable behavior.

`ca_modbus_write_multiple_regs`

Header: `caos/net/inc/modbus_service.h:137`
Source: `caos/net/src/modbus_service.c:290:309`
GUARD: `EN_MODBUS_SERVICE`

```
bool ca_modbus_write_multiple_regs(u16 addr, const u16 *values, u16
    count)
```

Sends a Modbus Function Code 16 request to write a block of multiple holding registers to a remote device.

Parametri:

Nome	Tipo	Descrizione
addr	u16	The starting register address on the target device. Must be a valid address supported by the slave.
values	const u16 *	Pointer to an array of 16-bit values to be written. Must not be NULL.
count	u16	Number of registers to write. Must be between 1 and 123 inclusive.

Ritorna:
Returns true if the transaction was successfully initiated and completed, false if parameters are invalid, the buffer size is exceeded, or the underlying transaction failed.

Note:

- This function is blocking, meaning the calling task will wait until the Modbus transaction completes or times out. It is not safe to call this function from an Interrupt Service Routine (ISR) because it relies on task-level synchronization and blocking mechanisms. Before calling this, ensure the Modbus communication stack is initialized and the target device is reachable. A common mistake is passing a 'count' that exceeds the Modbus protocol limit of 123 registers or providing a NULL pointer, both of which will cause the function to return false immediately without attempting communication.

Uso:

- This function is used when you need to update a contiguous block of configuration parameters or status registers on a remote Modbus slave device in a single atomic operation. It is typically used in a sequence where you first verify the connection, prepare an array of data, and then call this function to commit the changes. It is often paired with other Modbus functions like 'ca_modbus_read_holding_regs' to verify the write operation by reading the values back, or used within a state machine that manages periodic device configuration updates.

Esempio:

```
u16 motor_settings[3] = { 100, 250, 500 };

// Attempt to write 3 registers starting at address 0x1000
if (ca_modbus_write_multiple_regs(0x1000, motor_settings, 3U)) {
    // Success: registers updated on the slave device
    ca_stdout_put_n("Write successful");
} else {
    // Failure: check connection or slave address
    ca_stdout_put_n("Write failed");
}
```

Q&A — fine-tuning review

6/6 pairs · qwen3-coder:latest

Usage conf:0.95 **Q:** When should I use ca_modbus_write_multiple_regs instead of writing individual registers?
A: You should use ca_modbus_write_multiple_regs when you need to update multiple Modbus registers in a single transaction. This is more efficient than writing registers individually because it reduces the number of network round-trips and can improve performance in time-sensitive applications.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_modbus_write_multiple_regs with caOS types?
A: u16 values[] = {0x1234, 0x5678, 0x9ABC}; bool success = ca_modbus_write_multiple_regs(100, values, 3);

Parameters conf:0.95 **Q:** What are the valid ranges for the addr and count parameters in ca_modbus_write_multiple_regs?
A: The addr parameter must be a 16-bit value between 0 and 65535, representing the starting register address. The count parameter must be between 1 and 123, indicating how many consecutive registers to write.

Return conf:0.90 **Q:** What does it mean when ca_modbus_write_multiple_regs returns false?
A: A return value of false indicates either input validation failure (count=0, count>123, or values=NULL) or a transaction failure (timeout, network error, etc.). The function does not distinguish between these two failure modes in its return value.

Error/Edge case conf:0.85 **Q:** What are common mistakes developers make when calling ca_modbus_write_m
A: Common mistakes include passing a count parameter greater than 123, passing NULL as the values pointer, or not checking the return value. Additionally, calling this function without having properly initialized the Modbus context may lead to unexpected behavior or failures.

Behavior conf:0.90 **Q:** Is ca_modbus_write_multiple_regs safe to call from an ISR or is it thread-safe?
A: No, ca_modbus_write_multiple_regs is not safe to call from an ISR because it is a blocking function that may use RTOS functions not ISR-safe. It is also not recommended for use in interrupt contexts. However, it is thread-safe assuming the underlying 'transact' function is thread-safe and there are no concurrent accesses to the network resource.

`ca_modbus_write_single_reg`

Header: `caos/net/inc/modbus_service.h:123`
Source: `caos/net/src/modbus_service.c:276:284`
GUARD: `EN_MODBUS_SERVICE`

```
1 bool ca_modbus_write_single_reg(u16 addr, u16 value)
```

Sends a Modbus Function Code 06 request to write a single 16-bit register to a remote device.

Parametri:

Nome	Tipo	Descrizione
`addr`	`u16`	The 16-bit address of the Modbus register to be written. Must be a valid register address defined by the slave device's memory map.
`value`	`u16`	The 16-bit data value to be stored in the specified register.

Ritorna:
Returns true if the transaction was successfully sent and acknowledged by the slave device; returns false if the transaction failed, timed out, or received an error response.

Note:

- This function is blocking, meaning the calling task will wait until the Modbus transaction completes or times out. It is not thread-safe if multiple tasks attempt to access the Modbus bus simultaneously; you must ensure external synchronization (like a mutex) if your application has multiple tasks communicating over the same bus. It is strictly forbidden to call this from an ISR (Interrupt Service Routine) because it performs blocking I/O operations. A common beginner mistake is assuming the write is successful just because the function was called; you must always check the boolean return value to handle communication errors or slave exceptions.

Uso:

- This function is used in industrial automation contexts to update configuration parameters, setpoints, or control flags on remote Modbus slaves. It is typically used within a control loop or a state machine after the Modbus service has been initialized. It is often paired with other Modbus functions like ca_modbus_read_input_regs or ca_modbus_read_holding_regs to verify the state of the device before and after the write operation. In a typical sequence, you would initialize the communication interface, perform a read to check the current status, perform the write using this function, and then verify the change.

Esempio:

```
/* Example: Updating a setpoint register on a Modbus slave */
u16 setpoint = 500U;

/* Attempt to write the new setpoint to the device register */
if (ca_modbus_write_single_reg(REG_SETPOINT, setpoint)) {
    /* Success: Log the event and update local state */
    ca_stdout_printf_n("Successfully updated setpoint to %u", setpoint);
} else {
    /* Failure: Handle communication error or slave rejection */
    ca_stdout_put_n("Failed to write setpoint: Check connection or address");
    /* Implement retry logic or safe state recovery here */
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_modbus_write_single_reg() in a caOS application?
A: You should use ca_modbus_write_single_reg() when you need to write a single 16-bit value to a specific Modbus register address in a remote device. It is suitable for applications where blocking behavior is acceptable and the function is not called from an ISR context. This function is typically used in Modbus communication services where you need to configure or control remote devices via Modbus protocol.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_modbus_write_single_reg() with caOS types?
A: bool success = ca_modbus_write_single_reg(100, 42); if (success) { // Value written successfully } else { // Handle write failure }

Parameters conf:0.90 **Q:** What are the valid ranges for the address and value parameters in ca_modbus_write_single_reg()?
A: The address parameter (u16 addr) must be between 0 and 65535, inclusive. The value parameter (u16 value) must also be between 0 and 65535, inclusive. Values outside these ranges may cause undefined behavior. Both parameters are 16-bit unsigned integers, so they map directly to Modbus register addresses and values.

Return conf:0.95 **Q:** What does the return value of ca_modbus_write_single_reg() indicate, and what causes it to return false?
A: The function returns true if the write operation completes successfully with a valid Modbus response received. It returns false if there is an error during sending or receiving the Modbus response, which can be caused by timeout, communication error, or invalid response. The function is blocking and will not return until either a valid response is received or a timeout occurs.

`ca_mpu_add_region`

Header: `caos/inc/mpu.h:67`
GUARD: `EN_MPU`

```
int ca_mpu_add_region(u32 addr, u32 size, u16 mode)
```

Configures and enables a Memory Protection Unit (MPU) region to restrict or grant access to a specific memory address range.

Parametri:

Nome	Tipo	Descrizione
`addr`	`u32`	The starting memory address for the region. Note that the hardware often requires this to be aligned to the size of the region.
`size`	`u32`	The size of the memory block in bytes. The MPU will automatically round this up to the nearest power-of-two (minimum 32 bytes).
`mode`	`u16`	Access permissions: MPU_ACCESS_RDONLY (read-only), MPU_ACCESS_NONE (no access), or MPU_ACCESS_RDWR (read-write).

Ritorna:
Returns the assigned region index (0 to N) on success, or -1 if no MPU regions are available or an error occurred.

Note:

- This function is non-blocking and executes very quickly, but it is not inherently thread-safe; if multiple tasks attempt to configure MPU regions simultaneously, you must protect this call with a mutex. It is generally safe to call from an ISR, but be aware that modifying MPU settings while the CPU is executing code in that memory area can trigger a MemManage fault. The most common beginner mistake is failing to check the return value; if you run out of MPU regions, the function returns -1, and ignoring this will lead to silent failures where your memory protection is not actually active.

Uso:

- This function is used to enforce memory safety by isolating sensitive data (like cryptographic keys or stack areas) from unauthorized access. It is typically called during system initialization or when a module is being set up. You would pair this with other MPU management functions to track how many regions are currently in use and to ensure you do not exceed the hardware's physical limit of available MPU slots.

Esempio:

```c
/* Define a memory area to protect */
#define STORAGE_SIZE 1024
u8 my_secure_buffer[STORAGE_SIZE];

int setup_protection() {
    /* Attempt to lock the buffer to read-only */
    int region = ca_mpu_add_region((u32)my_secure_buffer, STORAGE_SIZE,
    MPU_ACCESS_RDONLY);

    if (region == -1) {
        /* Handle error: No MPU regions left */
        return -1;
    }

    /* Region is now active and enforced by hardware */
    return 0;
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_mpu_add_region() in a caOS application?
A: You should call ca_mpu_add_region() during system initialization or when setting up memory protection for specific data regions. It's typically used to protect sensitive memory areas like cryptographic keys, secure buffers, or critical system data from unauthorized access. The function is not meant to be called dynamically during runtime processing but rather during setup phases.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_mpu_add_region()?
A:

```
u32 key_buffer[32];
u16 region_index = ca_mpu_add_region((u32)key_buffer, sizeof(key_buffer),
    MPU_ACCESS_RDONLY);
if (region_index != (u32)-1) {
    // Successfully added region
} else {
    // Failed to add region
}
```

Parameters conf:0.95 **Q:** What are the valid values for the mode parameter in ca_mpu_add_region()?
A: The mode parameter accepts three values: MPU_ACCESS_NONE (0) for no access, MPU_ACCESS_RDONLY (1) for read-only access, and MPU_ACCESS_RDWR (2) for read-write access. These values control the access permissions for the memory region being configured.

Return conf:0.95 **Q:** What does ca_mpu_add_region() return on success and failure?
A: On success, ca_mpu_add_region() returns the index (0-7) of the MPU region that was assigned. On failure, it returns (u32)-1. Failures occur when there are no available MPU regions, invalid parameters are provided, or the MPU is not initialized.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_mpu_add_region()?
A: Common mistakes include calling the function from an ISR (which is not allowed), passing invalid parameters such as non-power-of-2 sizes or unaligned addresses, calling it after MPU initialization is complete without proper setup, and assuming the function will work if the MPU is not enabled or configured properly.

Behavior conf:0.95 **Q:** Is ca_mpu_add_region() thread-safe and ISR-safe?
A: No, ca_mpu_add_region() is neither thread-safe nor ISR-safe. It is a blocking function that modifies global MPU state and must not be called from interrupt service routines. It is also not safe to call from multiple threads simultaneously without external synchronization.

HW differences conf:0.85 **Q:** How does ca_mpu_add_region() differ between STM32H5xx and other families like F7xx?
A: The main difference is in how access permissions are handled. For H5xx, the AP (Access Permission) values are 0 for no access, 1 for read-only (privileged RO, unprivileged no access), and 3 for full access. In contrast, F7xx and H7xx families use AP values 0 for no access, 6 for read-only, and 3 for read-write access. Additionally, H5xx uses a common helper function 'mpu_configure_region' while others directly manipulate MPU registers like RBAR and RASR.

`ca_mpu_dump_regions`

Header: `caos/inc/mpu.h:109`
GUARD: `EN_MPU`

```
void ca_mpu_dump_regions(void)
```

Prints the current configuration of all enabled Memory Protection Unit (MPU) regions to the system console.

Ritorna:
void

Note:

- This function is a diagnostic tool that iterates through the hardware MPU registers to display memory access permissions, base addresses, and sizes. It is non-blocking and performs read-only operations on the MPU registers, making it safe to call from most contexts, though it is primarily intended for debugging via CLI. It is not thread-safe in the sense that if another task modifies MPU settings while this function is iterating, the output may be inconsistent. The most common mistake is assuming this function will 'fix' memory issues; it only reports the current state, it does not validate if the configuration is logically correct for your application.

Uso:

- This function is typically used during the development and debugging phase to verify that memory protection regions (like stack guards or peripheral access restrictions) are configured as expected. It is commonly invoked via a CLI command (e.g., 'mpu') when a MemManage fault occurs or when verifying system initialization. It is often paired with other diagnostic functions like 'ca_scheduler_dump_task' to correlate memory layout with task-specific stack usage.

Esempio:

```c
/* Example: CLI command handler to inspect MPU state */
if (ca_strncmp(command_buffer, "mpu", 3) == 0) {
    /* Print header for clarity */
    ca_stdout_printf_n("--- System Diagnostic: MPU Dump ---\r\n");

    /* Call the dump function to output current MPU register state */
    ca_mpu_dump_regions();

    /* Ensure the output is flushed if necessary */
    ca_stdout_printf_n("--- End of Dump ---\r\n");
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_mpu_dump_regions() and what is its purpose in debugging?
A: You should call ca_mpu_dump_regions() during development and debugging to inspect the current MPU memory protection regions. It prints the base address, size, access permissions, and execute permissions of each configured region to the debug output. This function is useful for verifying that your MPU configuration is correct and for troubleshooting memory access issues.

Code example conf:0.95 **Q:** Can you show a minimal example of how to use ca_mpu_dump_regions() in a caOS application?
A:

```c
void debug_mpu(void) {
    ca_mpu_dump_regions();
}
```

Parameters conf:0.95 **Q:** Does ca_mpu_dump_regions() take any parameters, and what happens if I pass invalid arguments?
A: The function takes no parameters. Since it is defined as void ca_mpu_dump_regions(void), calling it with any arguments will result in a compilation error. The function accesses MPU registers directly and does not validate input parameters.

Error/Edge case conf:0.90 **Q:** What are the potential issues or edge cases when using ca_mpu_dump_regions()

A: The function is blocking and not thread-safe. It must not be called from an ISR or a critical section. If called in an inappropriate context, it may cause crashes or unpredictable behavior. Additionally, it assumes the MPU is properly initialized; otherwise, reading from MPU registers may return invalid data or cause faults.

Behavior conf:0.95 **Q:** Is ca_mpu_dump_regions() safe to call from an ISR or in a multi-threaded environment?
A: No, ca_mpu_dump_regions() is neither ISR-safe nor thread-safe. It is a blocking function that directly accesses MPU registers and should only be called from a non-critical, non-ISR context. Calling it from an ISR or a multi-threaded environment may lead to system instability or crashes.

HW differences conf:0.90 **Q:** How does ca_mpu_dump_regions() differ between different STM32 families like F7xx and H5xx?
A: The F7xx and H7xx implementations use RASR and RBAR registers to read region attributes, while the H5xx implementation uses RBAR and RLAR registers. The H5xx version also reads the attribute index from RLAR and uses different bit positions for access permissions and execute permissions compared to the F7xx/H7xx versions.

`ca_mpu_first_free_region`

Header: `caos/inc/mpu.h:138`
GUARD: `EN_MPU`

```
u32 ca_mpu_first_free_region(void)
```

Finds the index of the first MPU region that is currently disabled.

Ritorna:
Returns the index (0 to N-1) of the first available MPU region, or the total number of regions if all are currently enabled.

Note:

- This function is non-blocking and executes very quickly by iterating through the hardware MPU registers. It is thread-safe in the sense that it only reads hardware state, but it does not reserve the region; you must immediately follow this with a call to configure the region to prevent race conditions. It is safe to call from an ISR, provided you are not modifying MPU registers elsewhere simultaneously. A common mistake is assuming that finding a free region guarantees it is safe to use; you must always verify the memory address range does not overlap with existing critical system regions.

Uso:

- This function is typically used during system initialization or when dynamically creating memory-protected tasks. It is usually paired with ca_mpu_get_total_regions to check for availability, followed by ca_mpu_update_region to actually enable and configure the found slot. It is essential for managing hardware resources when you need to isolate specific memory areas for security or stability.

Esempio:

```
void setup_secure_buffer(void *addr, u32 size) {
    u32 region_idx = ca_mpu_first_free_region();
    u32 total = ca_mpu_get_total_regions();

    if (region_idx < total) {
        /* Configure the found region using hardware-specific parameters */
        /* ca_mpu_update_region(region_idx, addr, size, ...); */
```

```
    } else {
        /* Handle error: no MPU regions available */
        ca_printf("Error: No free MPU regions available!\n");
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_mpu_first_free_region in my caOS application?
A: This function is typically used during system initialization or when dynamically creating memory-protected tasks. It helps manage hardware resources by identifying available MPU slots for isolating specific memory areas. You should pair it with ca_mpu_get_total_regions to check availability and immediately follow it with a configuration call to prevent race conditions.

Code example conf:0.90 **Q:** Provide a minimal example of how to find and use a free MPU region.
A: u32 region_idx = ca_mpu_first_free_region(); if (region_idx < ca_mpu_get_total_regions()) { /* Immediately configure the region to prevent race conditions */ ca_mpu_update_region(region_idx, ...); }

Return conf:1.00 **Q:** What does the return value of ca_mpu_first_free_region indicate?
A: The function returns the index (0 to N-1) of the first disabled MPU region found. If all available MPU regions are currently enabled, it returns the total number of regions, indicating that no slots are free for use.

HW differences conf:1.00 **Q:** Are there differences in how ca_mpu_first_free_region checks for disabled regions across STM32 families?
A: Yes. For F4xx, F7xx, and H7xx families, the function checks the MPU_RASR_ENABLE_Msk bit in the MPU->RASR register. For H5xx and U5xx families (ARMv8-M), the function checks the MPU_RLAR_EN_Msk bit in the MPU->RLAR register.

`ca_mpu_get_available_regions`

Header: `caos/inc/mpu.h:121`
GUARD: `EN_MPU`

```
u32 ca_mpu_get_available_regions(void)
```

Returns the count of MPU regions that are currently disabled and available for configuration.

Ritorna:
Returns a u32 integer representing the number of free MPU slots. Returns 0 if all hardware MPU regions are currently in use.

Note:

- This function is non-blocking and executes quickly, making it safe to call from most contexts. However, it is not inherently thread-safe; if another task or an interrupt modifies MPU registers while this function is iterating through them, the returned count might be inconsistent. It is callable from an ISR, but be aware that it performs a loop over hardware registers, which adds a small amount of latency. A common beginner mistake is assuming that having 'available' regions means they are automatically configured; this function only reports availability, it does not reserve or allocate the regions for you.

Uso:

- This function is typically used during system initialization or when dynamically creating memory-protected tasks to ensure there is enough hardware capacity before attempting

to configure a new MPU region. It is usually paired with functions that configure specific MPU regions, such as ca_mpu_add_region or ca_mpu_update_region. By checking availability first, you prevent 'silent' failures where a new memory protection rule fails to apply because the hardware limit was reached. It acts as a safety gate to verify that your application's memory protection strategy fits within the physical constraints of the specific STM32 microcontroller being used.

Esempio:

```
u32 free_slots = ca_mpu_get_available_regions();

if (free_slots > 0) {
    /* Proceed to configure a new MPU region using ca_mpu_add_region */
    /* ca_mpu_add_region(base_addr, size, attributes); */
} else {
    /* Handle error: No MPU regions left to protect memory */
    ca_printf("MPU exhaustion: cannot protect additional memory\n");
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_mpu_get_available_regions() and why is it important for MPU configuration?
A: You should call ca_mpu_get_available_regions() when you need to determine how many MPU regions are still available for user-defined memory protection. This is important because it helps prevent over-allocation of MPU regions, which could lead to runtime errors or system instability. It must be called after the MPU is initialized and before attempting to configure new regions.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_mpu_get_available_regions() with caOS types?
A: u32 available = ca_mpu_get_available_regions(); if (available > 0) { // Proceed with MPU region configuration // Example: configure one region ca_mpu_configure_region(0, (u32)0x20000000, 0x1000, MPU_REGION_PRIVILEGED); } else { // Handle error: no regions available // Log or panic }

Return conf:0.90 **Q:** What do the return values of ca_mpu_get_available_regions() mean and what causes it to return 0?
A: The function returns a u32 representing the number of unused MPU regions. It returns 0 if no regions are available, or if there is a hardware error or MPU not present. A return value of 0 can also occur if all regions are already enabled by the system or user code, or if the function fails to access MPU registers due to hardware issues.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_mpu_get_available_re
A: Common mistakes include calling this function before MPU initialization, calling it from an ISR (which is not thread-safe), and ignoring that some regions may already be occupied by the system. It's also prone to silent failure if the MPU is not present or if registers are inaccessible. Additionally, the function is blocking and modifies the RNR register during execution, which can interfere with interrupt handling.

HW differences conf:0.80 **Q:** How does ca_mpu_get_available_regions() differ between STM32 F7xx and H5xx MCUs in terms of hardware implementation?
A: The main difference lies in how they check if a region is enabled. On F7xx, it reads the MPU->RASR register and checks the MPU_RASR_ENABLE_Msk bit. On H5xx, it reads the MPU->RLAR register and checks the MPU_RLAR_EN_Msk bit. This reflects differences in the MPU architecture between these families, where H5xx uses ARMv8-M style registers while F7xx uses older ARMv7-M style registers.

ca_mpu_get_busy_regions

Header: caos/inc/mpu.h:127
GUARD: EN_MPU

```
u32 ca_mpu_get_busy_regions(void)
```

Returns the count of MPU regions currently allocated and in use by the system.

Ritorna:
Returns an unsigned 32-bit integer representing the number of MPU regions currently occupied. A return value of 0 indicates that no MPU regions are currently reserved by the OS.

Note:

- This function is non-blocking and executes in constant time, making it safe to call from both thread mode and Interrupt Service Routines (ISRs). It is thread-safe as it performs a simple arithmetic calculation based on internal state variables. The primary precondition is that the MPU subsystem must be initialized via ca_mpu_init() before calling this function, otherwise, the returned values may be undefined or zero. A common beginner mistake is assuming that this function returns the total number of available hardware slots; instead, it specifically returns the count of slots already taken, which is useful for monitoring memory protection resource exhaustion.

Uso:

- This function is typically used for system diagnostics, debugging, or resource management when you need to verify if there are enough free MPU slots remaining before attempting to configure a new memory region. It is often paired with ca_mpu_get_available_regions() to perform a 'pre-flight check' before calling functions like ca_mpu_add_region(). By monitoring the busy count, developers can prevent system crashes caused by attempting to configure more regions than the hardware supports, which is a critical limitation on STM32 microcontrollers.

Esempio:

```
u32 busy_count = ca_mpu_get_busy_regions();
u32 total_count = ca_mpu_get_total_regions();

// Check if we have room to define a new protected memory region
if (busy_count < total_count) {
    // Safe to proceed with MPU configuration
    // ca_mpu_add_region(base_addr, size, attributes);
} else {
    // Log error: No MPU regions left
    // ca_printf("Error: MPU exhausted");
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should a developer call ca_mpu_get_busy_regions() and what is its purpose in memory management?
A: A developer should call ca_mpu_get_busy_regions() to determine how many MPU regions are currently allocated by the caOS for memory protection. This function is useful for monitoring memory resource usage and ensuring that sufficient MPU regions are available for application requirements. It can be called from any context including threads or ISRs since it is non-blocking.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_mpu_get_busy_regions() with caOS types?

A: u32 busy = ca_mpu_get_busy_regions(); if (busy == 0xFFFFFFFF) { // Handle error case } else { // Use busy to check memory region usage }

Return conf:0.90 **Q:** What do the different return values of ca_mpu_get_busy_regions() indicate, and what causes it to return 0xFFFFFFFF?
A: The function returns the number of MPU regions currently in use by caOS. A return value of 0 means no regions are occupied. A positive value indicates how many regions are allocated. The function returns 0xFFFFFFFF in case of an error, which typically occurs if ca_mpu_get_total_regions() or ca_mpu_get_available_regions() have not been initialized correctly before calling this function.

HW differences conf:0.75 **Q:** Are there any differences in how ca_mpu_get_busy_regions() is implemented across different STM32 MCU families like F4, F7, H5, and H7?
A: There are no functional differences in the implementation of ca_mpu_get_busy_regions() across the STM32 families F4, F7, H5, and H7. All implementations follow the same logic: they retrieve the total number of regions and subtract the available regions to compute the number of busy regions. The function relies on common helper functions ca_mpu_get_total_regions() and ca_mpu_get_available_regions() which are expected to be implemented per family but are not shown in the provided code snippets.

`ca_mpu_get_total_regions`

Header: `caos/inc/mpu.h:115`
GUARD: `EN_MPU`

```
u32 ca_mpu_get_total_regions(void)
```

Retrieves the total number of configurable memory protection regions supported by the hardware MPU.

Ritorna:
An unsigned 32-bit integer representing the count of available MPU regions (typically 8 or 16 depending on the specific STM32 silicon).

Note:

- This function is non-blocking and executes in constant time by reading directly from the MPU_TYPE register. It is thread-safe and can be safely called from both main-line code and Interrupt Service Routines (ISRs) because it performs a simple read operation without modifying any system state. The most common mistake is assuming a fixed number of regions across different STM32 families; always use this function to query the hardware dynamically rather than hardcoding values like 8 or 16. There are no mandatory preconditions other than the MPU hardware being present on the chip.

Uso:

- This function is typically used during system initialization or when implementing dynamic memory management schemes that require partitioning memory into protected segments. It is often paired with functions that configure specific regions, such as ca_mpu_add_region or ca_mpu_update_region, to ensure the application does not attempt to configure a region index that exceeds the hardware's physical limit. By using this function, your code becomes portable across different STM32 series, as it automatically adapts to the specific MPU capabilities of the target microcontroller.

Esempio:

```
u32 total_regions = ca_mpu_get_total_regions();

```

```
// Check if the hardware supports at least 8 regions before proceeding
if (total_regions >= 8) {
    // Proceed to configure MPU regions using other caOS functions
    // ca_mpu_add_region(7, ...);
} else {
    // Handle error: hardware does not support the required number of regions
    // Log error using ca_printf or similar
    while(1);
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_mpu_get_total_regions() in my caOS application?
A: You should call ca_mpu_get_total_regions() during system initialization or configuration phase to determine how many MPU regions are available on your STM32 chip. This is useful for setting up memory protection zones and ensuring that your MPU configuration does not exceed hardware capabilities. It's typically called before any MPU setup functions to validate the number of regions before attempting to configure them.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_mpu_get_total_regions() in a caOS application?
A: u32 total_regions = ca_mpu_get_total_regions(); if (total_regions > 0) { // Proceed with MPU configuration // Example: use total_regions to allocate region indices } else { // Handle error case where MPU is not present or invalid }

Return conf:0.90 **Q:** What do the different return values of ca_mpu_get_total_regions() mean?
A: The function returns the number of MPU regions available on the chip. A value greater than 0 indicates the number of regions (e.g., 8 means 8 regions are supported). A return value of 0 indicates either that the MPU is not present on the chip or that the MPU is not properly initialized. The function will return 0 if ___MPU_PRESENT is not defined, which is a common cause of failure.

HW differences conf:0.85 **Q:** Are there any differences in how ca_mpu_get_total_regions() is implemented across different STM32 families?
A: No, the implementation is consistent across all supported families (f4xx, f7xx, h5xx, h7xx). All implementations read the MPU->TYPE register, shift the value right by 8 bits, and mask it with 0xFF to extract the number of regions. The register and bit layout are consistent across these families, so there are no hardware-specific differences in this function's behavior.

`ca_mpu_init`

Header: `caos/inc/mpu.h:51`
GUARD: `EN_MPU`

```
void ca_mpu_init(void)
```

Initializes the Memory Protection Unit (MPU) to enable hardware-based memory access control and stack overflow protection.

Ritorna:
void. This function does not return a status code; if the hardware lacks an MPU, it silently returns without performing any configuration.

Note:

- This function is non-blocking and must be called during the early system boot sequence before the RTOS scheduler starts. It is not thread-safe and should never be called from an ISR, as it modifies critical system registers that control CPU exception behavior. A

common mistake is calling this function after memory allocation or task creation has already begun, which can lead to unpredictable behavior if the MPU configuration conflicts with existing memory usage. Always ensure this is one of the first hardware initialization steps to guarantee that subsequent memory operations are protected by the MPU's fault-detection logic.

Uso:

- This function is used exclusively during the system startup phase, typically inside the main() function before the RTOS scheduler is initialized. It is part of a mandatory sequence: first, the MPU is initialized with ca_mpu_init(), then specific memory regions (like the stack canary) are protected using functions like ca_mpu_protect_canary_msp(), and finally, the memory heap and scheduler are initialized. This order ensures that the system is fully protected before any user tasks or dynamic memory allocations are created.

Esempio:

```c
int main(void) {
    /* Initialize system clocks and basic hardware first */
    /* Assume system clock configuration is done here */

    /* Initialize MPU to enable memory protection features */
    ca_mpu_init();

    /* Protect the Main Stack Pointer (MSP) area from overflows */
    /* _msp_stack_base is a linker-provided symbol */
    ca_mpu_protect_canary_msp((u32)&_msp_stack_base);

    /* Now safe to initialize heap and scheduler */
    ca_memory_init(&_heap_start, &_heap_end);

    /* Initialize scheduler and start the RTOS */
    if (ca_scheduler_init(1) == false) {
        /* Handle initialization failure */
        while(1);
    }

    ca_os_run();
    return 0;
}
```

Q&A — fine-tuning review

6/6 pairs · qwen3-coder:latest

Usage conf:1.00 **Q:** When should ca_mpu_init() be called in a caOS application?
A: ca_mpu_init() should be called once at system startup, after SysTick initialization and before other system initializations. It must be called from a main thread, not from an ISR, and is intended to set up the Memory Protection Unit for stack canary protection.

Code example conf:1.00 **Q:** Can you provide a minimal example of how to use ca_mpu_init() in a caOS application?
A:

```c
void main(void) {
    ca_systick_init();
    ca_mpu_init();
    // Continue with other initializations
    while (1) {
        // Main loop
    }
}
```

Return conf:1.00 **Q:** What happens if ca_mpu_init() is called on a device without MPU support?
A: If the device does not have an MPU, ca_mpu_init() returns immediately without performing any configuration. It never fails and does not have any return values to

indicate success or failure.

Error/Edge case conf:1.00 **Q:** What are common mistakes when using ca_mpu_init()?
A: Common mistakes include calling ca_mpu_init() more than once, calling it from an ISR, or calling it before SysTick initialization. It is also important to ensure that the function is called from a main thread and not during a context switch, as it is not thread-safe.

Behavior conf:1.00 **Q:** Is ca_mpu_init() blocking or non-blocking, and is it safe to call from an ISR?
A: ca_mpu_init() is a blocking function and is not safe to call from an ISR. It is also not thread-safe and must be called from a single main thread during system initialization.

HW differences conf:0.80 **Q:** How does ca_mpu_init() differ between STM32F7 and STM32H7 families?
A: The main difference is in the configuration of memory attributes. The STM32F7 and STM32H7 implementations both configure the MPU control registers and exception priorities similarly, but the STM32F7 and STM32H5 implementations also set up the MAIR0 register for memory attribute definitions, which is not present in the STM32H7 implementation.

`ca_mpu_protect_canary_msp`

Header: `caos/inc/mpu.h:97`
GUARD: `EN_MPU`

```
1 void ca_mpu_protect_canary_msp(u32 canary_addr)
```

Configures the MPU to make the Main Stack Pointer (MSP) canary memory region read-only to detect stack overflows.

Parametri:

Nome	Tipo	Descrizione
`canary_addr`	`u32`	The memory address of the canary value. Must be 32-byte aligned. Passing 0 disables the protection.

Ritorna:
void

Note:

- This function is non-blocking and executes very quickly, but it is not thread-safe; it should only be called during system initialization before the scheduler starts. It is not intended to be called from an ISR. The most common mistake is providing an address that is not 32-byte aligned, which will cause the MPU to misconfigure the region, potentially leading to unexpected MemManage faults. Always ensure the address provided is the start of the reserved canary area defined in your linker script.

Uso:

- This function is used during the early boot sequence of the RTOS, immediately after the MPU hardware is initialized via ca_mpu_init(). It is typically called before memory heap initialization and scheduler startup to ensure that the stack integrity monitoring is active before any tasks are created. It pairs with ca_mpu_init() to set up the hardware and precedes ca_bus_error_init() to ensure that if a violation occurs, the system can handle the resulting fault gracefully.

Esempio:

```c
/* During system startup, protect the MSP stack canary */
extern u32 _msp_stack_base; // Defined in linker script

void system_boot(void) {
    ca_mpu_init();

    /* Protect the 32-byte region starting at the stack base */
    ca_mpu_protect_canary_msp((u32)&_msp_stack_base);

    /* Now any write access to this address will trigger a MemManage fault */
    ca_bus_error_init();

    /* Proceed with system initialization */
    ca_memory_init(&_heap_start, &_heap_end);
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_mpu_protect_canary_msp in a caOS application?
A: You should call ca_mpu_protect_canary_msp during system initialization, typically after the MPU has been initialized but before multitasking begins. It is used to protect the MSP (Main Stack Pointer) canary region from unauthorized writes, enhancing system security by preventing stack overflow or buffer overflow attacks from corrupting critical stack data.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_mpu_protect_canary with caOS types?
A: u32 msp_base = 0x20000000; /* Example MSP base address */ ca_mpu_protect_canary_msp(msp_ /* Protect the MSP canary region */

Parameters conf:0.95 **Q:** What does the canary_addr parameter represent and what are valid values?
A: The canary_addr parameter is the physical base address of the MSP stack region to be protected. It must be aligned to a 32-byte boundary (e.g., 0x20000000). Valid values include any aligned address within the physical memory range. Invalid values are NULL (0), unaligned addresses, or addresses outside the valid physical memory space.

Error/Edge case conf:0.90 **Q:** What are common mistakes or edge cases when using ca_mpu_protect_canary_
A: Common mistakes include calling the function before the MPU is initialized, passing an unaligned address, or calling it from within an ISR. If an invalid address is passed, the function may cause a hardware fault or undefined behavior. Passing a zero value disables the protection, which may be intentional but should be done carefully.

Behavior conf:0.95 **Q:** Is ca_mpu_protect_canary_msp thread-safe and ISR-safe?
A: No, ca_mpu_protect_canary_msp is not thread-safe and not ISR-safe. It modifies global MPU state and is blocking during configuration. It must not be called from within an ISR as it could interfere with the execution context and may cause unpredictable behavior.

HW differences conf:0.85 **Q:** How does the implementation of ca_mpu_protect_canary_msp differ between STM32 H7 and H5 families?
A: On H7xx, the function directly configures MPU registers (RBAR, RASR) with explicit bit manipulation for size, access permissions, and execute-never flags. On H5xx, it delegates to a helper function mpu_protect_32b_ro which abstracts the register-level configuration, likely using a common MPU utility function for 32-byte read-only regions.

`ca_mpu_protect_canary_psp_end`

Header: `caos/inc/mpu.h:90`
GUARD: `EN_MPU`

```
void ca_mpu_protect_canary_psp_end(u32 canary_addr)
```

Configures an MPU region to make a specific memory area read-only, typically used to protect stack canary values from accidental corruption.

Parametri:

Nome	Tipo	Descrizione
`canary_addr`	`u32`	The base memory address of the canary. Must be 32-byte aligned. Passing 0 disables the protection region.

Ritorna:
void

Note:

- This function is non-blocking and executes very quickly as it directly writes to the MPU hardware registers. It is thread-safe in the sense that it modifies a specific MPU region reserved for the canary, but it must be used carefully during context switches. It is callable from an ISR, but typically it is used by the kernel scheduler. The most common mistake is providing an address that is not 32-byte aligned, which will cause the MPU to mask the lower bits and potentially protect the wrong memory area. Always ensure the MPU is enabled in the system configuration before calling this, otherwise, the settings will have no effect.

Uso:

- This function is used by the RTOS kernel to implement stack overflow detection. It is typically called during a context switch or task initialization sequence. The standard workflow is: first, disable the protection by passing 0 to update the canary value, then perform the write operation to the memory, and finally call the function again with the new address to re-enable the read-only protection. This ensures that the canary value remains immutable during normal task execution, triggering a MemManage fault if any code attempts to overwrite it.

Esempio:

```
/* Example: Updating the stack canary during a context switch */

// 1. Disable protection to allow updating the canary value
ca_mpu_protect_canary_psp_end(0);

// 2. Update the canary memory area
u64 *canary_ptr = (u64*)GET_CANARY_UPPER(ctx);
canary_ptr[0] = STACK_CANARY_VALUE;
canary_ptr[1] = STACK_CANARY_VALUE;

// 3. Re-enable protection to make the area Read-Only
ca_mpu_protect_canary_psp_end((u32)canary_ptr);

/* If any code now tries to write to canary_ptr, a HardFault/MemManage fault
   will occur */
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_mpu_protect_canary_psp_end() and what is its purpose?
A: You should call ca_mpu_protect_canary_psp_end() to protect a canary value located at the top of the current task's stack. This prevents accidental overwrites of the

canary, which is used for stack overflow detection. It is typically called during task initialization or when setting up stack protection.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_mpu_protect_canary_psp_end with caOS types?
A: u32 canary_addr = (u32)&stack_top - 32; /* assuming stack_top is a valid address */ ca_mpu_protect_canary_psp_end(canary_addr);

Parameters conf:0.95 **Q:** What does the canary_addr parameter represent, and what are valid values?
A: The canary_addr parameter is the base address of the canary to be protected. It must be aligned to a 32-byte boundary. Valid values are any aligned address, or 0 to disable protection. If not aligned, the function rounds it down to the nearest 32-byte boundary.

Return conf:0.95 **Q:** Does ca_mpu_protect_canary_psp_end() return anything, and what does it mean?
A: The function returns nothing (void). It does not fail in a traditional sense, but may ignore invalid inputs such as misaligned addresses or invalid regions.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_mpu_protect_canary_
A: Common mistakes include passing an unaligned address, calling the function from an unprivileged context, or using it in a multitasking environment without proper synchronization. Passing 0 disables protection, which may be intended but can be overlooked.

Behavior conf:0.90 **Q:** Is ca_mpu_protect_canary_psp_end() thread-safe, ISR-safe, and blocking?
A: The function is blocking and not thread-safe. It must be called from a privileged context (task or ISR). It modifies MPU registers and flushes buffers, so it should not be used in concurrent multitasking without synchronization.

HW differences conf:0.85 **Q:** How does ca_mpu_protect_canary_psp_end() differ between STM32F7 and STM32H7 families?
A: Both implementations configure the same MPU region with a 32-byte read-only area and execute-never bit. However, the F7 uses a direct register write approach, while the H7 uses a similar register configuration but with slightly different bit field names (e.g., MPU_RASR_XN_Msk vs MPU_RASR_XN_Pos).

`ca_mpu_protect_canary_psp_start`

Header: `caos/inc/mpu.h:83`
GUARD: `EN_MPU`

```
1 void ca_mpu_protect_canary_psp_start(u32 canary_addr)
```

Configures the MPU to make a 32-byte memory region read-only, typically used to protect stack canary values.

Parametri:

Nome	Tipo	Descrizione
`canary_addr`	`u32`	The memory address of the canary. Must be 32-byte aligned. Passing 0 disables the protection region.

Ritorna:
void

Note:

- This function is non-blocking and executes very quickly, but it is not thread-safe; if multiple tasks or an ISR attempt to modify MPU settings simultaneously, the system state will become corrupted. It is intended to be called from privileged code during task context switching or initialization. The most common mistake is providing an unaligned address, which will cause the MPU to mask the address incorrectly, potentially protecting the wrong memory area. Always ensure the MPU is enabled in the system configuration before calling this, as it only configures a specific region, not the MPU hardware itself.

Uso:

- This function is used by the kernel scheduler to implement stack overflow detection. It is typically used in a 'disable-modify-enable' pattern: first, call with 0 to disable protection so the kernel can write the canary pattern into the stack; second, perform the write operation; third, call with the actual address to lock the region as read-only. This ensures that if a task accidentally writes into its own stack canary area, the CPU triggers a MemManage fault, allowing the system to catch the overflow immediately.

Esempio:

```
/* Example: Protecting a stack canary after initialization */
u32 my_canary_base = 0x20001000; // Must be 32-byte aligned

// 1. Disable protection to write the canary value
ca_mpu_protect_canary_psp_start(0);

// 2. Write the canary pattern to the stack
*(u64*)my_canary_base = 0xDEADBEEFCAFEBABE;

// 3. Re-enable protection to make the region Read-Only
ca_mpu_protect_canary_psp_start(my_canary_base);

// Any subsequent attempt to write to my_canary_base will now trigger a
    HardFault/MemManage fault.
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_mpu_protect_canary_psp_start() in a caOS application?
A: You should call ca_mpu_protect_canary_psp_start() during task initialization or when setting up stack canary protection for the current task. It is typically used in scheduler code to protect the stack canary from accidental writes, which helps detect stack overflows or corruption.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_mpu_protect_canary_psp_star with caOS types?
A: u32 canary_addr = (u32)task_stack_bottom + 32; /* assuming stack bottom is aligned */ ca_mpu_protect_canary_psp_start(canary_addr);

Parameters conf:0.95 **Q:** What does the canary_addr parameter represent, and what are valid values?
A: The canary_addr parameter is the base address of the stack canary to be protected. It must be aligned to a 32-byte boundary. If 0, the function disables protection. Any non-zero value that is not 32-byte aligned will be automatically rounded down to the nearest 32-byte boundary.

Return conf:0.95 **Q:** Does ca_mpu_protect_canary_psp_start() return any value, and what does it mean?
A: The function does not return any value. It is void and never fails. It may silently ignore invalid inputs such as non-aligned addresses or invalid contexts.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_mpu_protect_canary_

A: Common mistakes include calling the function from an unprivileged context or without proper privilege level, passing an unaligned address, or calling it in a multitasking environment without synchronization. The function does not validate the address range or check if the MPU is enabled, so improper usage may lead to undefined behavior.

Behavior conf:0.90 **Q:** Is ca_mpu_protect_canary_psp_start() thread-safe, ISR-safe, and blocking?

A: The function is blocking and not thread-safe. It must be called from a privileged context (task or ISR). It is not safe to call from multiple threads or ISRs concurrently without external synchronization. It modifies MPU registers directly and requires the processor to be in privileged mode.

HW differences conf:0.90 **Q:** How does ca_mpu_protect_canary_psp_start() differ between STM32F4, F7, H5, and H7 families?

A: On F4 and F7, the function directly configures MPU registers (RBAR, RASR) with explicit bit manipulation. On H5, it delegates to a helper function mpu_protect_32b_ro(). On H7, it also directly configures RBAR and RASR but uses a slightly different bit mask for XN (MPU_RASR_XN_Msk vs. 1UL « MPU_RASR_XN_Pos). All implementations protect a 32-byte region read-only and disable execution.

`ca_mpu_region_covers`

Header: `caos/inc/mpu.h:149`
GUARD: `EN_MPU`

```
bool ca_mpu_region_covers(u16 region, u32 addr, u32 size)
```

Checks if a specific MPU region is enabled and overlaps with a given memory range.

Parametri:

Nome	Tipo	Descrizione
`region`	`u16`	The index of the MPU region to check (e.g., 0 to 7 or 0 to 15 depending on the MCU).
`addr`	`u32`	The starting memory address of the range to verify.
`size`	`u32`	The size in bytes of the memory range to verify.

Ritorna:
Returns true if the region is enabled and the specified range overlaps with the region's memory span; returns false otherwise.

Note:

- This function is non-blocking and thread-safe regarding the MPU hardware state, as it saves and restores the MPU region index (RNR) on newer architectures. It is safe to call from an ISR, but be aware that it performs direct hardware register access, which is fast but requires the MPU to be initialized. A common beginner mistake is assuming the function checks if the entire range is contained within the region; it only checks for any overlap. Always ensure the 'region' index provided is within the valid range supported by your specific STM32 hardware.

Uso:

- This function is typically used for safety checks, such as verifying if a buffer or a stack area conflicts with protected memory regions. It is often paired with ca_mpu_add_region or ca_mpu_update_region during system initialization to validate configuration. In runtime, it is used to perform sanity checks before memory operations to ensure that critical system areas, like stack canaries, are not being inadvertently overwritten by user code.

Esempio:

```c
/* Check if a buffer overlaps with the MPU region reserved for stack canaries */
#define MY_BUFFER_SIZE 128
u8 my_buffer[MY_BUFFER_SIZE];

void check_buffer_safety(void) {
    u32 start_addr = (u32)my_buffer;

    /* Check if our buffer overlaps with the MPU region used for PSP canary protection */
    if (ca_mpu_region_covers(MPU_CANARY_REGION_PSP_START, start_addr, MY_BUFFER_SIZE)) {
        /* Handle error: buffer is in a protected area */
        ca_printf("Security Alert: Buffer overlaps with protected MPU region!\n");
    } else {
        /* Safe to proceed with memory operations */
        ca_memset(my_buffer, 0, MY_BUFFER_SIZE);
    }
}
```

Q&A — fine-tuning review

6/6 pairs · gemini-3.1-flash-lite

Usage conf:1.00 **Q:** When should I use ca_mpu_region_covers in my caOS application?
A: You should use it for safety checks, such as verifying if a buffer or stack area conflicts with protected memory regions. It is commonly used during system initialization to validate configurations or at runtime to perform sanity checks before memory operations to prevent inadvertent overwrites of critical areas like stack canaries.

Code example conf:1.00 **Q:** How do I implement a check for a buffer overlap using ca_mpu_region_covers?
A: You can check if a buffer overlaps with a specific MPU region as follows: u32 buffer_addr = 0x20001000; u32 buffer_size = 128; u16 region_idx = 2; if (ca_mpu_region_covers(regio buffer_addr, buffer_size)) { /* Handle overlap error */ }

Parameters conf:1.00 **Q:** What are the valid values for the 'region' parameter in ca_mpu_region_covers?
A: The 'region' parameter is a u16 representing the index of the MPU region to check. Valid values depend on your specific STM32 hardware, typically ranging from 0 to 7 or 0 to 15. You must ensure the index provided is within the range supported by your specific MCU.

Return conf:1.00 **Q:** What does a return value of true from ca_mpu_region_covers actually signify?
A: A return value of true indicates that the specified MPU region is currently enabled and that the memory range defined by 'addr' and 'size' has at least some overlap with the region's memory span. A return value of false indicates either the region is disabled or there is no overlap.

Error/Edge case conf:1.00 **Q:** What is a common mistake developers make when interpreting the result of ca_mpu_region_covers?
A: A common beginner mistake is assuming the function checks if the entire memory range is contained within the MPU region. In reality, the function only checks for any partial or full overlap between the provided range and the region's span.

HW differences conf:1.00 **Q:** How does the implementation of ca_mpu_region_covers differ between STM32 families?
A: The implementation varies by architecture: F4xx, F7xx, and H7xx use ARMv7-M size-encoded regions (checking RASR and RBAR registers), while H5xx and U5xx use ARMv8-M base+limit encoding (checking RBAR and RLAR registers). The H5xx and U5xx implementations also explicitly save and restore the MPU RNR register state.

ca_mpu_region_is_reserved

Header: caos/inc/mpu.h:181
GUARD: EN_MPU

```
static inline bool ca_mpu_region_is_reserved(u32 idx)
```

Checks if a specific MPU region index is reserved by the OS kernel.

Parametri:

Nome	Tipo	Descrizione
idx	u32	The index of the MPU region to check. Valid values are 0 to (total available MPU regions - 1).

Ritorna:
Returns true if the index is reserved for internal OS use, false if it is available for user-defined memory protection.

Note:

- This function is a non-blocking, thread-safe helper that performs a simple bounds check against the internal OS configuration. It is safe to call from both task context and ISRs because it does not perform any hardware access or complex logic. The most common mistake is assuming that a 'false' return means the region is currently unused; it only means the region is not reserved by the kernel. You must still verify if the region is already in use by the application using other MPU management functions like checking the busy regions list.

Uso:

- This function is typically used during system initialization or when dynamically configuring memory protection for specific tasks. It is usually paired with ca_mpu_first_free_region to find a valid slot for a new memory protection rule. Developers use this to ensure they do not accidentally overwrite critical kernel memory regions when calling ca_mpu_add_region or ca_mpu_update_region. It is part of the standard MPU setup sequence: init -> check availability -> add region.

Esempio:

```
/* Check if region 0 is reserved before attempting to use it */
u32 region_idx = 0;

if (ca_mpu_region_is_reserved(region_idx)) {
    /* Region is reserved by the OS, find the first available user region
    instead */
    region_idx = ca_mpu_first_free_region();
}

/* Now safely add a new protection region using the validated index */
/* ca_mpu_add_region(region_idx, start_addr, size, attributes); */
ca_printf("Configuring MPU region %lu\n", region_idx);
```

Q&A — fine-tuning review

4/4 pairs · gemini-3.1-flash-lite

Usage conf:1.00 **Q:** When should I use ca_mpu_region_is_reserved and why is it important?
A: You should use this function during system initialization or when dynamically configuring memory protection for tasks. It is important because it prevents you from accidentally overwriting critical kernel memory regions when calling functions like ca_mpu_add_region or ca_mpu_update_region.

Code example conf:1.00 **Q:** Provide a minimal C example showing how to check if an MPU region is available for user use.
A: u32 region_idx = 4; if (!ca_mpu_region_is_reserved(region_idx)) { /* Region is not reserved by kernel, proceed with application-level checks */ }

Parameters conf:1.00 **Q:** What are the valid values for the idx parameter in ca_mpu_region_is_reserved?
A: The idx parameter is a u32 representing the index of the MPU region. Valid values range from 0 to (OS_REGIONS - 1).

Return conf:1.00 **Q:** What does a return value of false from ca_mpu_region_is_reserved guarantee?
A: A return value of false means the region is not reserved by the kernel. It does not guarantee that the region is currently unused; you must still verify if the region is already in use by the application using other MPU management functions.

`ca_mpu_update_region`

Header: `caos/inc/mpu.h:77`
GUARD: `EN_MPU`

```
1 int ca_mpu_update_region(u16 region_index, u32 addr, u32 size, u16 mode)
```

Configures a specific Memory Protection Unit (MPU) region to enforce access permissions and memory boundaries.

Parametri:

Nome	Tipo	Descrizione
`region_index`	`u16`	The specific MPU slot to configure. Must be less than the value returned by ca_mpu_get_total_regions().
`addr`	`u32`	The starting memory address for the region. Note that the hardware may force alignment based on the size.
`size`	`u32`	The size of the memory region in bytes. The hardware will round this up to the nearest power of 2.
`mode`	`u16`	Access permission constant: MPU_ACCESS_RDONLY, MPU_ACCESS_NONE, or MPU_ACCESS_RDWR.

Ritorna:
Returns 0 on success. Returns -1 if the region_index is invalid (out of bounds).

Note:

- This function is non-blocking and executes immediately. It is thread-safe in the sense that it modifies hardware registers directly, but you must ensure that no other task or ISR attempts to modify the same MPU region simultaneously, as this would cause a race condition. It is safe to call from an ISR, but be aware that changing memory protections while the CPU is executing code in that region can trigger an immediate MemManage fault. A common beginner mistake is providing an unaligned address or a size that is not a power of 2; while the function handles the math, the resulting memory coverage might be larger than expected, potentially overlapping with other critical memory areas.

Uso:

- This function is typically used during system initialization or when switching security contexts (e.g., before accessing sensitive data like cryptographic keys). It is often paired with ca_mpu_get_total_regions() to ensure the index is valid before attempting configuration. Developers use this to create 'sandboxes' in memory, ensuring that specific tasks or drivers cannot write to memory they don't own. It is standard practice to call this whenever a memory buffer's lifecycle changes, such as locking a key storage area after a write operation is complete.

Esempio:

```c
/* Define a region index for our secure storage */
#define SECURE_STORAGE_REGION 2

void protect_key_storage(void) {
    /* Ensure the region index is within hardware limits */
    if (SECURE_STORAGE_REGION < ca_mpu_get_total_regions()) {
        /* Lock the storage area to Read-Only to prevent accidental corruption */
        int status = ca_mpu_update_region(SECURE_STORAGE_REGION, (u32)key_buffer, 1024, MPU_ACCESS_RDONLY);

        if (status != 0) {
            /* Handle configuration error: log or trigger system reset */
            handle_error();
        }
    }
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_mpu_update_region in a caOS application?
A: You should call ca_mpu_update_region when you need to define or redefine memory access permissions for a specific region in your system. It's typically used during system initialization or when setting up secure memory zones, such as protecting sensitive data or code sections. This function is essential for implementing memory protection in embedded systems to prevent unauthorized access.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_mpu_update_region?
A: int result = ca_mpu_update_region(0, 0x20001000, 1024, MPU_ACCESS_RDONLY); if (result != 0) { /* Handle error */ }

Parameters conf:0.90 **Q:** What are the valid values for the 'mode' parameter in ca_mpu_update_region?
A: The 'mode' parameter accepts three values: MPU_ACCESS_RDONLY for read-only access, MPU_ACCESS_NONE for no access, and MPU_ACCESS_RDWR for read-write access. Invalid values may lead to incorrect access permissions being set.

Return conf:0.95 **Q:** What does a return value of -1 from ca_mpu_update_region indicate?
A: A return value of -1 indicates either an invalid region index (outside the range of available MPU regions) or an internal error during the operation. This typically occurs when the region_index is out of bounds or if the function fails to configure the MPU due to hardware constraints.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_mpu_update_region?
A: Common mistakes include using an invalid region index (outside the valid range), providing an address that is not aligned to the region size, using a size that is not a power of 2 and less than 32 bytes, or calling the function from an ISR. Additionally, not ensuring the MPU is disabled or in a safe context before calling it can lead to errors.

Behavior conf:0.95 **Q:** Is ca_mpu_update_region thread-safe and ISR-safe?
A: No, ca_mpu_update_region is neither thread-safe nor ISR-safe. It is a blocking function that modifies MPU registers directly and requires a flush of the cache using ___DSB and ___ISB. It must be called from a context where no other threads or ISRs

are modifying the MPU configuration.

HW differences conf:0.85 **Q:** How does ca_mpu_update_region differ between F7xx and H7xx MCUs?

A: The main difference lies in how the execute-never bit is set. On F7xx, the function uses (1UL « MPU_RASR_XN_Pos), while on H7xx, it uses MPU_RASR_XN_Msk. Both implementations perform similar operations but use different register bit definitions for the execute-never attribute.

`ca_mqtt_disconnect`

Header: `caos/net/inc/mqtt_service.h:126`
Source: `caos/net/src/mqtt_service.c:325:334`
GUARD: `EN_MQTT_SERVICE`

```
void ca_mqtt_disconnect(void)
```

Gracefully terminates an active MQTT session and closes the underlying TCP connection.

Ritorna:
This function returns void and does not provide feedback on whether the disconnection packet was successfully acknowledged by the server.

Note:

- This function is non-blocking and performs a 'best-effort' shutdown by sending a DISCONNECT packet before closing the socket. It is not thread-safe; if multiple tasks attempt to manage the MQTT state simultaneously, you may encounter race conditions where the connection state flags become inconsistent. It is strictly forbidden to call this from an Interrupt Service Routine (ISR) because it relies on TCP stack operations that require task context. A common beginner mistake is calling this function without checking if the connection is actually active, or assuming that calling this immediately frees up all network resources, whereas the underlying TCP stack might still be performing a graceful FIN handshake.

Uso:

- Use this function when you need to shut down the MQTT client, such as during a system power-down sequence, a network interface reset, or when transitioning to a low-power state. It is typically used to clean up the connection state before the application enters a state where the network is no longer required. In a robust application, you should call this before re-initializing the network stack or when the application logic determines that the MQTT broker is no longer needed, ensuring that the server is notified of the client's departure rather than just timing out.

Esempio:

```
if (ca_mqtt_is_connected()) {
    // Notify the broker we are leaving and close the socket
    ca_mqtt_disconnect();
} else {
    // Connection already down, perform local cleanup
    // Use ca_printf to log that the MQTT service was already inactive
}
```

Q&A — fine-tuning review *2/2 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_mqtt_disconnect() in my embedded application, and what are the risks of calling it at the wrong time?

A: You should call ca_mqtt_disconnect() when you want to cleanly close an active MQTT connection and free associated TCP resources. It must only be called when the MQTT connection is active, specifically when s.tcp_up is true. Calling it without an active connection will cause the function to do nothing but may lead to incorrect assumptions about connection state. The function is blocking and not thread-safe, so it should not be called from an ISR and requires proper synchronization if used in a multi-threaded environment.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_mqtt_disconnect() with caOS types?
A: void example_mqtt_disconnect(void) { // Assuming MQTT service is initialized and connected if (s.tcp_up) { // Check if connection is active ca_mqtt_disconnect(); // Close MQTT connection cleanly } // At this point, TCP resources are freed and connection is closed }

`ca_mqtt_is_connected`

Header: `caos/net/inc/mqtt_service.h:123`
Source: `caos/net/src/mqtt_service.c:320:323`
GUARD: `EN_MQTT_SERVICE`

```
bool ca_mqtt_is_connected(void)
```

Checks if the MQTT client is currently connected to the broker.

Ritorna:
Returns true if the MQTT client is connected (s.mqtt_up is set), and false if the client is disconnected or the service is not initialized.

Note:

- This function is non-blocking and thread-safe as it simply reads a boolean flag from the internal service state. It is safe to call from both tasks and Interrupt Service Routines (ISRs) because it performs a simple memory read without acquiring mutexes or waiting for hardware. A common mistake is assuming that a 'true' return value guarantees that the network socket is still healthy; the flag only reflects the last known state of the MQTT session. If the network cable is unplugged or the router drops the connection, this flag might remain true until the next keep-alive timeout or TCP error occurs, so always pair this with logic that handles communication timeouts.

Uso:

- This function is primarily used in the main application loop or a dedicated network management task to monitor the health of the MQTT connection. It is typically used as a guard condition before attempting to publish data to ensure the message is not lost. It is frequently paired with ca_mqtt_service_init() to trigger a reconnection sequence when the connection is lost, and it is often used alongside ca_os_tick_ms() to implement non-blocking reconnection intervals, preventing the system from flooding the broker with connection requests during a network outage.

Esempio:

```
if (!ca_mqtt_is_connected()) {
    // Check if enough time has passed since the last attempt to avoid spamming
    if ((ca_os_tick_ms() - last_reconnect) >= MQTT_RECONNECT_MS) {
        ca_stdout_put_n("MQTT connection lost, attempting to reconnect...");
        // Re-initialize the service to reset the TCP handle and MQTT state
        ca_mqtt_service_init(broker_ip, MQTT_BROKER_PORT, local_ip, s_mac);
```

```
        last_reconnect = ca_os_tick_ms();
    }
} else {
    // Safe to send data now
    // ca_mqtt_publish(...);
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_mqtt_is_connected() in my embedded application?
A: You should call ca_mqtt_is_connected() to check the current connection status of the MQTT client. It is useful for determining whether to proceed with sending MQTT messages or to attempt reconnection logic. It can be called from any context including threads or ISRs, but note that it only reflects a cached state and not an active connection check.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_mqtt_is_connected() with caOS types?
A: bool connected = ca_mqtt_is_connected(); if (connected) { // Proceed with MQTT operations } else { // Handle disconnected state }

Return conf:0.90 **Q:** What does it mean when ca_mqtt_is_connected() returns true or false?
A: When ca_mqtt_is_connected() returns true, it indicates that the MQTT client believes it is connected (s.mqtt_up = true). When it returns false, it means the client believes it is not connected (s.mqtt_up = false). Failures or inconsistencies may occur due to unstable connections or race conditions, as the function is not thread-safe and does not actively verify the connection.

`ca_mqtt_ping`

Header: `caos/net/inc/mqtt_service.h:120`
Source: `caos/net/src/mqtt_service.c:313:318`
GUARD: `EN_MQTT_SERVICE`

```
void ca_mqtt_ping(void)
```

Sends an MQTT PINGREQ packet to the broker to maintain the connection alive.

Ritorna:
This function returns void and does not provide feedback on whether the packet was successfully sent or if the network connection is failing.

Note:

- This function is non-blocking and relies on the underlying ca_tcp_send function to handle the transmission. It is not thread-safe if called simultaneously from multiple tasks, though it is typically only called by the MQTT management task. It cannot be called from an ISR because it performs network operations that require a stable stack and OS context. A common mistake is calling this function before the MQTT service is initialized or while the connection is down; the function internally checks the s.mqtt_up flag to prevent sending packets on a dead socket, but it does not attempt to reconnect automatically.

Uso:

- This function is used as a heartbeat mechanism to prevent the MQTT broker from closing the connection due to inactivity. It is typically called within a periodic loop inside an MQTT management task, paired with a timer check to ensure the ping is sent at the required interval (e.g., every 30-60 seconds). It is usually used in conjunction with

ca_mqtt_is_connected to verify the link status before attempting to send data, and it relies on the TCP handle initialized during the MQTT service startup sequence.

Esempio:

```
/* Periodic MQTT keepalive task */
u32 last_ping = ca_os_tick_ms();
while(1) {
    u32 now = ca_os_tick_ms();
    /* Check if it is time to send a ping */
    if ((now - last_ping) >= 30000U) {
        last_ping = now;
        /* Send PINGREQ to keep the broker connection active */
        ca_mqtt_ping();
    }
    /* Yield to other tasks to prevent CPU starvation */
    ca_os_sleep(100U);
}
```

Q&A — fine-tuning review *2/2 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_mqtt_ping() and why is it important for MQTT communication?
A: You should call ca_mqtt_ping() in regular intervals during active MQTT communication to keep the connection alive with the broker. It sends a PINGREQ message to prevent the broker from closing the connection due to inactivity. This function must only be called when the MQTT connection is active (s.mqtt_up=true), otherwise it will likely fail or cause errors.

Code example conf:0.85 **Q:** Can you provide a minimal C code example using ca_mqtt_ping() with caOS types?
A: void example_mqtt_ping(void) { if (s.mqtt_up) { ca_mqtt_ping(); } }

`ca_mqtt_publish`

Header: `caos/net/inc/mqtt_service.h:105`
Source: `caos/net/src/mqtt_service.c:258:283`
GUARD: `EN_MQTT_SERVICE`

```
bool ca_mqtt_publish(const char *topic, const u8 *payload, u16 len)
```

Publishes a message to a specific MQTT topic over an established TCP connection.

Parametri:

Nome	Tipo	Descrizione
`topic`	`const char *`	The MQTT topic string. Must be a null-terminated C string. Invalid if NULL or if the resulting packet exceeds MQTT_TX_BUF_SIZE.
`payload`	`const u8 *`	Pointer to the data buffer to be sent. Can contain binary data or text.
`len`	`u16`	Length of the payload in bytes. Must not cause the total packet size to exceed MQTT_TX_BUF_SIZE.

Ritorna:
Returns true if the packet was successfully formatted and passed to the TCP stack, false otherwise. A false return indicates the MQTT service is not up, the packet is too large, or the underlying TCP send operation failed.

Note:

- This function is not thread-safe and must be called from a single task context to avoid race conditions on the internal transmission buffer. It is not callable from an ISR because it performs memory operations and relies on the TCP stack which is likely blocking or requires task-level synchronization. The function performs a bounds check against MQTT_TX_BUF_SIZE; if your topic and payload combined exceed this limit, the function will silently fail, so always verify your data size before calling. A common beginner mistake is assuming this function guarantees delivery; it only confirms that the packet was handed off to the TCP layer, not that the broker received it.

Uso:

- This function is used after the MQTT service has been initialized and the TCP connection is established. It is typically called within a main application loop or a dedicated telemetry task to send sensor data or status updates to a broker. It is usually paired with a connection check (to ensure s.mqtt_up is true) and follows a sequence of: initialize network, connect TCP, perform MQTT handshake, and then repeatedly call ca_mqtt_publish to send data.

Esempio:

```
/* Ensure the MQTT service is active before attempting to publish */
if (s.mqtt_up) {
    const char *msg = "{\"status\":\"online\"}";
    u16 len = (u16)ca_strlen(msg);

    /* Attempt to publish the status message */
    if (!ca_mqtt_publish("device/status", (const u8 *)msg, len)) {
        /* Handle failure: log error or retry later */
        ca_stdout_printf_n("Failed to publish MQTT message");
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_mqtt_publish in a caOS application?
A: You should call ca_mqtt_publish when you need to send data over MQTT to a broker. It is typically used after establishing a TCP connection and ensuring the MQTT service is active (s.mqtt_up = true). This function is ideal for publishing sensor readings, status updates, or any other data that needs to be transmitted reliably via MQTT.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_mqtt_publish with caOS types?
A: u8 payload[] = {0x01, 0x02, 0x03}; const char *topic = "sensor/temp"; bool success = ca_mqtt_publish(topic, payload, 3); if (success) { // Message sent successfully } else { // Handle failure }

Parameters conf:0.95 **Q:** What are the valid values for the topic and payload parameters in ca_mqtt_publish?
A: The topic parameter must be a valid, non-NULL C string representing the MQTT topic. The payload must be a valid, non-NULL pointer to u8 data. The payload length must be > 0 and <= MQTT_TX_BUF_SIZE minus fixed constants. Passing NULL for either topic or payload, or a length <= 0, will cause the function to return false.

Return conf:0.95 **Q:** What does it mean when ca_mqtt_publish returns false?
A: ca_mqtt_publish returns false if any of these conditions occur: 1) MQTT is not active (s.mqtt_up = false), 2) topic or payload is NULL, 3) payload is too long for the buffer, or 4) the TCP transmission fails. It is a blocking function, so it will not return until the transmission is complete or fails.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_mqtt_publish?

A: Common mistakes include calling the function without an active TCP connection (s.tcp_handle invalid), attempting to send a payload that exceeds MQTT_TX_BUF_SIZE, passing NULL for topic or payload, or calling the function from an ISR. These errors will result in a false return value and failed transmission.

Behavior conf:0.95 **Q:** Is ca_mqtt_publish thread-safe and ISR-safe?

A: No, ca_mqtt_publish is neither thread-safe nor ISR-safe. It is a blocking function that uses TCP functions not safe for use in interrupt service routines. It must not be called from an ISR and should not be called concurrently from multiple threads. It requires a valid TCP connection and active MQTT service to function correctly.

`ca_mqtt_service_init`

Header: `caos/net/inc/mqtt_service.h:81`
Source: `caos/net/src/mqtt_service.c:214:248`
GUARD: `EN_MQTT_SERVICE`

```
void ca_mqtt_service_init(ca_ipv4_t broker_ip, u16 broker_port,
    ca_ipv4_t local_ip, ca_mac_t local_mac, const char *client_id,
    ca_mqtt_connected_cb_t on_connected, ca_mqtt_message_cb_t on_message,
     ca_mqtt_disconnected_cb_t on_disconnected)
```

Initializes the MQTT service state and registers callback functions for connection events.

Parametri:

Nome	Tipo	Descrizione
`broker_ip`	`ca_ipv4_t`	The IPv4 address of the MQTT broker.
`broker_port`	`u16`	The TCP port of the MQTT broker (e.g., 1883).
`local_ip`	`ca_ipv4_t`	The local IPv4 address assigned to the device.
`local_mac`	`ca_mac_t`	The local MAC address of the network interface.
`client_id`	`const char *`	Unique string identifier for this MQTT client.
`on_connected`	`ca_mqtt_connected_cb_t`	Callback function triggered upon successful connection.
`on_message`	`ca_mqtt_message_cb_t`	Callback function triggered when a message is received.
`on_disconnected`	`ca_mqtt_disconnected_cb_t`	Callback function triggered when the connection is lost.

Ritorna:
void

Note:

- This function is not thread-safe and must be called from a single task context during system initialization or recovery. It performs a hard reset of the internal MQTT state, including calling ca_tcp_abort on any existing TCP handle to prevent 'zombie' connections that could cause the broker to reject new sessions due to duplicate Client IDs. It is not callable from an ISR. A common mistake is calling this function repeatedly without waiting for the network stack to stabilize, which can lead to rapid connection cycling and broker-side session eviction.

Uso:

- This function is the mandatory first step in the MQTT lifecycle. It is typically called after the network interface is up and the IP address is assigned. It pairs with ca_mqtt_service_net_ready to verify network availability before the stack attempts to connect. Once initialized, the MQTT service remains in a standby state until the underlying network stack processes

the connection request. It should be called again only if a complete reset of the MQTT state machine is required due to a fatal error or a change in network configuration.

Esempio:

```c
/* Define callback functions */
void on_mqtt_connected(void) { ca_stdout_put_n("MQTT Connected"); }
void on_mqtt_message(void *msg) { /* Handle incoming data */ }
void on_mqtt_disconnected(void) { ca_stdout_put_n("MQTT Disconnected"); }

void setup_mqtt(ca_ipv4_t broker_ip, ca_ipv4_t local_ip, ca_mac_t mac) {
    /* Initialize the MQTT service with provided network parameters */
    ca_mqtt_service_init(broker_ip, 1883, local_ip, mac, "STM32_Client",
                         on_mqtt_connected, on_mqtt_message,
    on_mqtt_disconnected);

    /* Wait for network readiness before starting the service */
    if (ca_mqtt_service_net_ready()) {
        ca_stdout_put_n("MQTT service ready to connect");
    } else {
        ca_stdout_put_n("Network not ready for MQTT");
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_mqtt_service_init in my caOS project?
A: You should call ca_mqtt_service_init once during the initialization phase of your application, before performing any MQTT operations. It sets up the MQTT service with the required connection parameters and callbacks, ensuring that the MQTT client is ready to connect to a broker.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_mqtt_service_init with caOS types?
A: ca_ipv4_t broker_ip = {192, 168, 1, 100}; u16 broker_port = 1883; ipv4_t local_ip = {192, 168, 1, 42}; mac_t local_mac = {0x00, 0x1A, 0x2B, 0x3C, 0x4D, 0x5E}; const char *client_id = "caos_device_001"; ca_mqtt_service_init(broker_ip, broker_port, local_ip, local_mac, client_id, NULL, NULL, NULL);

Parameters conf:0.90 **Q:** What are the valid values for the client_id parameter in ca_mqtt_service_init?
A: The client_id must be a non-null string that uniquely identifies the MQTT client. If a NULL pointer is passed, the behavior is undefined and may lead to a failure in the MQTT connection process.

Error/Edge case conf:0.85 **Q:** What happens if I pass invalid IP addresses or MAC addresses to ca_mqtt_service_init?
A: If invalid IP addresses or MAC addresses are passed, the MQTT connection will likely fail during the connection attempt. The function itself does not validate these values, but incorrect values will prevent successful communication with the broker.

Behavior conf:0.90 **Q:** Is ca_mqtt_service_init a blocking or non-blocking function, and can it be called from an ISR?
A: ca_mqtt_service_init is a non-blocking function and is not safe to call from an ISR. It is also not thread-safe, so it must be called from a task context and not from an interrupt handler.

`ca_mqtt_service_net_ready`

Header: `caos/net/inc/mqtt_service.h:95`
Source: `caos/net/src/mqtt_service.c:241:256`
GUARD: `EN_MQTT_SERVICE`

```
bool ca_mqtt_service_net_ready(void)
```

Initializes and triggers the underlying TCP connection required for the MQTT service.

Ritorna:
Returns true if the TCP connection request was successfully initiated and a valid handle was obtained; returns false if the connection attempt failed or the system could not allocate a handle.

Note:

- This function is non-blocking in terms of waiting for the actual TCP handshake to complete, but it performs internal configuration and resource allocation. It is not safe to call from an Interrupt Service Routine (ISR) because it relies on underlying TCP stack operations that likely use mutexes or queues. A common mistake is calling this function repeatedly in a loop without checking if a connection is already active, which can lead to resource exhaustion or multiple overlapping connection attempts. Ensure that the MQTT service configuration (broker IP, port, local network settings) is fully initialized before calling this, otherwise, the connection will fail immediately.

Uso:

- This function is typically called during the initialization phase of the MQTT service, often after the network interface is up but before the main application loop starts. It acts as the bridge between the high-level MQTT logic and the low-level TCP stack. In a typical lifecycle, you call this to start the connection, then monitor the status via the 'on_tcp_connected' callback provided in the configuration. If the connection drops, the application logic should handle the reconnection strategy by calling this function again after a suitable delay.

Esempio:

```
/* Initialize MQTT service network layer */
if (ca_mqtt_service_net_ready()) {
    ca_stdout_put_n("TCP connection request sent successfully.");
} else {
    ca_stdout_put_n("Failed to initiate TCP connection.");
    /* Handle error: perhaps retry later or check network interface status */
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_mqtt_service_net_ready() in my MQTT client application?
A: You should call ca_mqtt_service_net_ready() when you want to initiate a TCP connection to an MQTT broker. It is typically called after setting up the broker's IP address, port, and necessary callbacks using other caOS functions. This function is non-blocking and only starts the connection process, not waiting for it to complete.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_mqtt_service_net_ready() with caOS types?
A:

```
u8 result = ca_mqtt_service_net_ready();
if (result) {
    // Connection started successfully
} else {
    // Failed to start connection
}
```

Return conf:0.95 **Q:** What does it mean when ca_mqtt_service_net_ready() returns false?

A: A return value of false indicates that the TCP connection attempt failed to start. This can be due to invalid configuration settings (like incorrect IP or port), insufficient system resources, or an invalid handle returned by the underlying network layer.

Error/Edge case conf:0.90 **Q:** What are common mistakes or edge cases when using ca_mqtt_service_net_rea

A: Common mistakes include calling this function before setting up the broker's IP address, port, or required callbacks. It is also unsafe to call this function from an ISR, as it is not ISR-safe. Additionally, if the system runs out of network resources or the handle is invalid, the function will return false.

Behavior conf:0.95 **Q:** Is ca_mqtt_service_net_ready() blocking or non-blocking, and is it safe to use from an ISR or in a multi-threaded environment?

A: The function is non-blocking and does not wait for the TCP connection to complete. It is not thread-safe and must not be called from an ISR. It only attempts to initiate the connection, not to manage its completion or handle concurrent access.

`ca_mqtt_subscribe`

Header: `caos/net/inc/mqtt_service.h:113`
Source: `caos/net/src/mqtt_service.c:285:311`
GUARD: `EN_MQTT_SERVICE`

```
bool ca_mqtt_subscribe(const char *topic)
```

Sends an MQTT SUBSCRIBE packet to the broker to start receiving messages on a specific topic.

Parametri:

Nome	Tipo	Descrizione
`topic`	`const char *`	A null-terminated string representing the MQTT topic filter to subscribe to. Must not be NULL.

Ritorna:
Returns true if the packet was successfully queued for transmission via the underlying TCP stack, false otherwise (e.g., MQTT not connected, buffer overflow, or TCP send failure).

Note:

- This function is blocking in the sense that it performs memory operations and calls the TCP stack, so it should not be called from an ISR. It is not inherently thread-safe; ensure that only one task manages the MQTT connection state to avoid race conditions on the packet ID counter. A common beginner mistake is calling this before the MQTT connection is fully established (s.mqtt_up is false), which will cause the function to return false immediately. Always verify the connection status before attempting to subscribe.

Uso:

- This function is typically used immediately after the MQTT connection is established, often inside a connection callback or a state machine transition. It is paired with the MQTT connection logic (e.g., after a successful CONNECT handshake) and is followed by the application's message processing loop. You use it to register interest in specific data streams, and it works in tandem with an incoming message handler that processes the data arriving on that topic.

Esempio:

```
if (s.mqtt_up) {
    /* Subscribe to the command topic after connection is confirmed */
```

```
    if (ca_mqtt_subscribe("sensors/temp/cmd")) {
        ca_stdout_printf_n("Subscription request sent successfully");
    } else {
        ca_stdout_printf_n("Failed to send subscription request");
    }
} else {
    ca_stdout_printf_n("MQTT not ready, cannot subscribe");
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_mqtt_subscribe() in a caOS application?
A: You should call ca_mqtt_subscribe() after establishing a successful MQTT connection to the broker. It is typically used when you want to start receiving messages on a specific topic. The function is not blocking, so it sends the subscription request immediately without waiting for a response from the broker.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_mqtt_subscribe() with caOS types?
A: cstr topic = "home/lights/bedroom"; bool result = ca_mqtt_subscribe(topic); if (result) { // Subscription sent successfully } else { // Handle subscription failure }

Parameters conf:0.95 **Q:** What is the significance of the topic parameter in ca_mqtt_subscribe()?
A: The topic parameter is a pointer to a null-terminated string that specifies the MQTT topic to subscribe to. It must be a valid, non-NULL string. The maximum length of the topic is constrained by MQTT_TX_BUF_SIZE. Passing NULL or a topic longer than the buffer size will cause the function to return false.

Return conf:0.95 **Q:** What does it mean when ca_mqtt_subscribe() returns false?
A: A return value of false indicates that the subscription request was not sent successfully. This can happen due to: 1) an inactive MQTT connection (s.mqtt_up = false), 2) a topic that is too long for the MQTT_TX_BUF_SIZE buffer, or 3) an internal error during packet construction.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_mqtt_subscribe()?
A: Common mistakes include: 1) calling the function before establishing an MQTT connection, 2) passing a NULL topic pointer, 3) providing a topic string that exceeds MQTT_TX_BUF_SIZE, and 4) calling the function from multiple threads without proper synchronization.

Behavior conf:0.90 **Q:** Is ca_mqtt_subscribe() thread-safe and ISR-safe?
A: No, ca_mqtt_subscribe() is not thread-safe. If called from multiple threads, it may cause concurrency issues and should be protected with a mutex. It is also not recommended to call it from an ISR, as it could lead to concurrency or blocking issues if used with non-ISRs.

`ca_mutex_init`

Header: `caos/inc/sync.h:54`
Source: `caos/src/sync.c:107:112`
GUARD: `EN_SYNC`

```
void ca_mutex_init(ca_mutex_t *mtx)
```

Initializes a mutex structure to its default unlocked state, preparing it for thread synchronization.

Parametri:

Nome	Tipo	Descrizione
mtx	ca_mutex_t *	Pointer to the mutex object to be initialized. Must point to valid, allocated memory (either static or heap).

Ritorna:
void

Note:

- This function is non-blocking and performs no complex operations, making it safe to call during system startup. It is not thread-safe in the sense that you must ensure no other threads are attempting to lock or unlock the mutex while it is being initialized. It is strictly forbidden to call this function on a mutex that is already in use, as it will reset the internal state and cause race conditions or deadlocks. A common beginner mistake is to re-initialize a mutex inside a loop or a frequently called function; this must only be called once during the initialization phase of the peripheral or service. This function must not be called from an ISR, as it is intended for task-level synchronization setup.

Uso:

- This function is used during the system boot or driver initialization phase to prepare synchronization primitives before any tasks are scheduled. It is typically paired with ca_mutex_lock and ca_mutex_unlock to protect shared resources like hardware peripherals (e.g., CRC, AES, Hash engines) or global data structures. In a typical lifecycle, you define the mutex as a global or static variable, call ca_mutex_init during the driver's init function, and then use the lock/unlock functions throughout the application to ensure exclusive access.

Esempio:

```c
/* Define the mutex globally to be used by the driver */
static ca_mutex_t aes_mutex;

void my_crypto_init(void) {
    /* Initialize the mutex before any tasks start using the driver */
    ca_mutex_init(&aes_mutex);

    /* Proceed to initialize hardware registers */
    ca_aes_init();
}

void my_crypto_task(void) {
    /* Protect access to shared hardware */
    ca_mutex_lock(&aes_mutex);

    /* Perform hardware operation using the initialized channel */
    // ca_aes_channel_encrypt(...);

    ca_mutex_unlock(&aes_mutex);
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_mutex_init() in my embedded application, and why is it important for resource protection?
A: You should call ca_mutex_init() once for each mutex object before using it to protect shared resources between threads. It is important because it initializes the mutex structure to a known state, ensuring that subsequent calls to ca_mutex_lock() and ca_mutex_unlock() will function correctly. The mutex prevents concurrent access to

shared data, which is essential in multi-threaded embedded applications to avoid race conditions and data corruption.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to declare and initialize a mutex using caOS types?
A: ca_mutex_t my_mutex; ca_mutex_init(&my_mutex);

Parameters conf:0.95 **Q:** What is the significance of the 'mtx' parameter in ca_mutex_init(), and what happens if it is NULL?
A: The 'mtx' parameter is a pointer to the mutex structure that will be initialized. It must point to valid, allocated memory (not NULL). If mtx is NULL, the behavior is undefined, which may result in a crash or memory corruption, as the function will attempt to write to a null pointer address.

`ca_mutex_is_owner`

Header: `caos/inc/sync.h:82`
Source: `caos/src/sync.c:213:222`
GUARD: `EN_SYNC`

```
bool ca_mutex_is_owner(ca_mutex_t *mtx)
```

Checks if the currently running thread is the one that currently holds the specified mutex.

Parametri:

Nome	Tipo	Descrizione
`mtx`	`ca_mutex_t *`	A pointer to the initialized mutex object. Must not be NULL; passing a NULL pointer will likely cause a system crash or undefined behavior.

Ritorna:
Returns true if the current thread owns the mutex, false otherwise. Returns false if the current thread context cannot be determined or if the mutex is currently unowned.

Note:

- This function is non-blocking and thread-safe because it uses an internal spinlock to protect the mutex state during the check. It is safe to call from most contexts, but it is primarily intended for thread-level logic. A common beginner mistake is assuming that this function 'locks' the mutex; it does not, it only performs a read-only check of the ownership status. If you call this from an ISR (Interrupt Service Routine), it will return false because an ISR does not have a thread context, which is the expected behavior since an ISR cannot 'own' a mutex.

Uso:

- This function is typically used for debugging, assertion checking, or implementing recursive-like logic where a thread needs to verify it has exclusive access before performing a sensitive operation. It is often paired with ca_mutex_lock() and ca_mutex_unlock() to ensure that only the thread that acquired the resource is the one attempting to release it or modify protected data. Use it to prevent 'deadlock-by-reentrancy' by checking ownership before attempting to acquire a mutex that the thread might already hold.

Esempio:

```
ca_mutex_t myMutex;

void task_init(void) {
    ca_mutex_init(&myMutex);
```

```
}

void process_data(void) {
    // Attempt to acquire the mutex before accessing shared resources
    if (ca_mutex_trylock(&myMutex)) {
        // Verify we own the mutex before modifying shared resources
        if (ca_mutex_is_owner(&myMutex)) {
            // Safe to modify data protected by myMutex
            // ... perform operations ...
        }
        ca_mutex_unlock(&myMutex);
    } else {
        // Handle error: could not acquire mutex
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_mutex_is_owner() in my caOS application?
A: You should use ca_mutex_is_owner() primarily for debugging purposes or to prevent potential deadlocks. It allows you to verify that the current thread is the owner of a mutex before attempting to lock or unlock it, which is especially useful in complex multi-threaded scenarios where mutex ownership might be unclear.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_mutex_is_owner() with caOS types?
A: ca_mutex_t mtx; ca_mutex_init(&mtx); bool is_owner = ca_mutex_is_owner(&mtx); // is_owner will be false since no thread owns the mutex yet.

Parameters conf:0.90 **Q:** What happens if I pass a NULL pointer to ca_mutex_is_owner()?
A: If you pass a NULL pointer to ca_mutex_is_owner(), the function will return false. However, passing a NULL pointer is considered invalid usage and may lead to undefined behavior, as the function expects a valid pointer to an initialized mutex structure.

Return conf:0.90 **Q:** What does it mean when ca_mutex_is_owner() returns false?
A: When ca_mutex_is_owner() returns false, it means either the current thread is not the owner of the mutex, or the parameter mtx is NULL. A return value of false does not necessarily indicate an error, but rather indicates that the calling thread does not own the mutex.

`ca_mutex_lock`

Header: `caos/inc/sync.h:61`
Source: `caos/src/sync.c:114:160`
GUARD: `EN_SYNC`

```
void ca_mutex_lock(ca_mutex_t *mtx)
```

Acquires a mutex to ensure exclusive access to shared resources or hardware peripherals.

Parametri:

Nome	Tipo	Descrizione
`mtx`	`ca_mutex_t *`	Pointer to the mutex object. Must be initialized via ca_mutex_init before use; passing an uninitialized or NULL pointer will trigger a system safeguard.

Ritorna:
This function returns void. If the mutex is available, it returns immediately. If the mutex is held by another thread, the function blocks the calling thread until the mutex becomes available.

Note:

- This function is blocking; the calling thread will be suspended and removed from the scheduler's ready list until the mutex is released. It is NOT thread-safe to call this from an Interrupt Service Routine (ISR) because ISRs cannot be blocked or yield the CPU. A common beginner mistake is attempting to lock a mutex inside an ISR or holding a mutex for too long, which causes system-wide latency or deadlocks. You must ensure that every lock is eventually followed by a corresponding ca_mutex_unlock to prevent permanent system stalls. The implementation uses a wait queue mechanism where the resource is handed off directly to the next waiter, ensuring fairness.

Uso:

- Use this function to protect critical sections, such as hardware registers (e.g., CRC, AES, Hash engines) or shared memory buffers that are accessed by multiple threads. It is typically used in a sequence: lock the mutex, perform the atomic operation or hardware transaction, and then immediately unlock the mutex. It is often paired with ca_mutex_unlock to release the resource. In driver development, it ensures that two threads do not attempt to configure the same hardware peripheral simultaneously, which would lead to corrupted data or undefined hardware states.

Esempio:

```
/* Example: Protecting a shared hardware peripheral */
caMutex aes_mutex;

void process_data(u8 *in, u8 *out, u32 len) {
    /* Initialize the mutex before first use */
    ca_mutex_init(&aes_mutex);

    /* Acquire exclusive access to the AES hardware */
    ca_mutex_lock(&aes_mutex);

    /* Perform hardware operations while holding the lock */
    /* Note: ca_aes_hw_setup and ca_aes_hw_process are internal driver calls */
    if (ca_aes_init()) {
        /* Perform the processing */
    }

    /* Always release the lock to allow other threads to use the hardware */
    ca_mutex_unlock(&aes_mutex);
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_mutex_lock in my embedded application?
A: You should use ca_mutex_lock when you need to protect critical sections of code that access shared resources, such as hardware peripherals or global variables. It ensures that only one task can access the resource at a time, preventing race conditions. For example, in drivers like aes.c, crc32.c, or hash.c, it's used to synchronize access to hardware registers or buffers.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_mutex_lock with caOS types?
A: ca_mutex_t mtx; ca_mutex_init(&mtx); ca_mutex_lock(&mtx); // Critical section ca_mutex_unlock(&mtx);

Parameters conf:0.90 **Q:** What is the significance of the mtx parameter in ca_mutex_lock?
A: The mtx parameter is a pointer to a ca_mutex_t structure that must be initialized with ca_mutex_init() before calling ca_mutex_lock. If the mutex is not properly initialized, the behavior is undefined. Passing NULL or an uninitialized mutex will lead to unpredictable results.

Return conf:0.95 **Q:** Does ca_mutex_lock return any value, and what does it mean?
A: No, ca_mutex_lock does not return any value. It is a void function. If the mutex is already owned by the same task, the system will deadlock due to CA_SAFEGUARD, but no explicit return value indicates this failure.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_mutex_lock?
A: Common mistakes include calling ca_mutex_lock from an ISR (which is not thread-safe), attempting to lock a mutex that was not initialized with ca_mutex_init(), or calling ca_mutex_lock on a mutex already owned by the same task, which causes a deadlock due to CA_SAFEGUARD.

Behavior conf:0.95 **Q:** Is ca_mutex_lock blocking or non-blocking, and can it be called from an ISR?
A: ca_mutex_lock is a blocking function. If the mutex is already occupied, the calling task is suspended until the mutex becomes available. It is not safe to call from an ISR because it is not thread-safe and must only be called from tasks. The function implements priority inheritance to prevent starvation.

`ca_mutex_pi_refresh`

Header: `caos/inc/sync.h:62`
Source: `caos/src/sync.c:267:269`
GUARD: `EN_SYNC`

```
void ca_mutex_pi_refresh(ca_thread_context_t *ctx)
```

Updates the priority inheritance (PI) state for a specific thread context.

Parametri:

Nome	Tipo	Descrizione
`ctx`	`ca_thread_context_t *`	Pointer to the thread context to update. Must be a valid pointer or NULL.

Ritorna:
None (void).

Note:

- This function is used to re-evaluate the effective priority of a thread when its base priority changes or when mutex ownership status shifts. It is not inherently blocking, but it performs internal calculations to ensure the thread's priority reflects the highest priority of any task waiting on a mutex it currently holds. It is generally safe to call from task context, but you must ensure the thread context pointer is valid. A common mistake is calling this function with a stale or uninitialized thread context pointer, which will lead to undefined behavior or system crashes.

Uso:

- This function is typically used by the kernel scheduler or synchronization primitives when a thread's priority needs to be adjusted dynamically to prevent priority inversion. It is usually called after modifying the 'prio_base' field of a thread context or when a task releases a mutex. You will rarely call this directly in application code; it is primarily an internal kernel mechanism used to maintain system stability during complex synchronization scenarios.

Esempio:

```
/* Example: Manually adjusting a thread's base priority and refreshing PI */
caThreadContext *ctx = ca_os_self_handle();
```

```c
if (ctx != NULL) {
    /* Temporarily elevate the base priority of the current task */
    ctx->prio_base = 10;

    /* Refresh the priority inheritance state to apply the change */
    ca_mutex_pi_refresh(ctx);

    /* Perform critical work... */

    /* Reset priority and refresh again */
    ctx->prio_base = 5;
    ca_mutex_pi_refresh(ctx);
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_mutex_pi_refresh in my application code?
A: You will rarely call this function directly in application code. It is primarily an internal kernel mechanism used by the scheduler or synchronization primitives to maintain system stability by re-evaluating a thread's effective priority when its base priority changes or when mutex ownership status shifts to prevent priority inversion.

Code example conf:0.90 **Q:** How do I correctly invoke ca_mutex_pi_refresh for a specific thread context?
A: Assuming you have a valid pointer to a thread context, you would call it as follows: ca_thread_context_t *my_thread = get_current_thread(); ca_mutex_pi_refresh(my_thread);. Ensure the pointer is not stale or uninitialized to avoid undefined behavior or system crashes.

Parameters conf:1.00 **Q:** What are the requirements for the 'ctx' parameter in ca_mutex_pi_refresh?
A: The 'ctx' parameter must be a pointer to a valid ca_thread_context_t structure. While the function accepts NULL, passing a stale or uninitialized pointer will lead to undefined behavior or system crashes. It is the caller's responsibility to ensure the pointer is valid before invocation.

`ca_mutex_pi_waiter_gone`

Header: `caos/inc/sync.h:66`
Source: `caos/src/sync.c:271:276`
GUARD: `EN_SYNC`

```c
void ca_mutex_pi_waiter_gone(ca_thread_context_t *ctx)
```

Notifies the scheduler that a thread is no longer waiting for a mutex, triggering a Priority Inheritance (PI) recalculation.

Parametri:

Nome	Tipo	Descrizione
`ctx`	`ca_thread_context_t *`	Pointer to the thread context. Must be a valid, initialized thread context. If NULL, the function returns immediately.

Ritorna:
void

Note:

- This function is part of the Priority Inheritance (PI) mechanism. It is used to inform the system that a thread has stopped waiting for a specific mutex, which allows the system

to lower the priority of the mutex owner if no other high-priority threads are waiting. It is not thread-safe on its own and is typically called within a critical section (e.g., while interrupts are disabled) to ensure the consistency of the wait list and PI state. A common mistake is calling this when the thread was not actually waiting for a mutex, though the function safely checks for this condition. It is not intended to be called directly from an ISR unless the surrounding code manages the necessary locking primitives.

Uso:

- This function is used during thread cleanup or when a thread is removed from a wait queue due to a timeout or manual cancellation. It is typically paired with scheduler operations that remove a thread from a wait list. By calling this, you ensure that the system does not 'forget' to restore the original priority of the mutex owner, preventing priority inversion issues where a task keeps a boosted priority indefinitely after the waiter has left.

Esempio:

```c
/* Assuming we are inside a critical section where interrupts are disabled */
caThreadContext *my_thread = ca_os_self_handle();

/* If the thread was waiting for a mutex, remove it from the wait list */
if (my_thread->wait_list != NULL) {
    /* Remove thread from the wait queue using internal scheduler logic */
    /* ca_wait_queue_remove(my_thread->wait_list, my_thread); */
    my_thread->wait_list = NULL;

    /* Notify the system that the thread is no longer waiting for the mutex */
    /* This triggers a recalculation of the mutex owner's priority */
    ca_mutex_pi_waiter_gone(my_thread);
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_mutex_pi_waiter_gone and what is its primary purpose?
A: You should call this function when a thread is removed from a mutex wait queue, such as during thread cleanup, timeout, or manual cancellation. Its purpose is to notify the scheduler to trigger a Priority Inheritance (PI) recalculation, ensuring the mutex owner's priority is restored if no other high-priority threads are waiting, thereby preventing indefinite priority inversion.

Code example conf:0.90 **Q:** Provide a minimal C example showing how to invoke ca_mutex_pi_waiter_gone safely.
A: Because this function is not thread-safe, it must be called within a critical section. Example: { u32 primask = ___get_PRIMASK(); ___disable_irq(); ca_mutex_pi_waiter_gone(thread_ct ___set_PRIMASK(primask); }

Parameters conf:1.00 **Q:** What are the requirements for the ca_thread_context_t pointer passed to ca_mutex_pi_waiter_gone?
A: The parameter must be a pointer to a valid, initialized thread context. If the pointer is NULL, the function returns immediately. It is expected that the thread was previously waiting for a mutex, though the function safely handles cases where this condition is not met.

`ca_mutex_timedlock`

Header: `caos/inc/sync.h:90`
Source: `caos/src/sync.c:224:246`
GUARD: `EN_SYNC`

```
bool ca_mutex_timedlock(ca_mutex_t *mtx, u32 timeout_ms)
```

Attempts to acquire a mutex within a specified time limit, blocking the task until available or the timeout expires.

Parametri:

Nome	Tipo	Descrizione
`mtx`	`ca_mutex_t*`	Pointer to the mutex object. Must be initialized via ca_mutex_init before use; passing a NULL or uninitialized pointer will cause a system crash.
`timeout_ms`	`u32`	Maximum time in milliseconds to wait for the mutex. A value of 0 makes this function behave like a non-blocking trylock.

Ritorna:
Returns true if the mutex was successfully acquired. Returns false if the timeout period elapsed before the mutex became available.

Note:

- This function is blocking, meaning the calling task will be suspended by the scheduler while waiting, allowing other tasks to run. It is NOT thread-safe to call this from an Interrupt Service Routine (ISR) because ISRs cannot be suspended or put to sleep. A common beginner mistake is calling this with a very short timeout in a high-frequency loop, which can lead to excessive context switching and CPU overhead. Always ensure the mutex has been properly initialized before calling this function, otherwise, the internal state of the mutex will be undefined.

Uso:

- Use this function when you need to access a shared resource but want to avoid waiting indefinitely if another task is hogging it. It is typically used in a pattern where you attempt to lock, check the return value, perform the critical section work, and then immediately call ca_mutex_unlock. It is often paired with ca_mutex_init for setup and ca_mutex_unlock to release the resource, ensuring that the system remains responsive even if a resource is temporarily unavailable.

Esempio:

```
/* Example: Attempting to access a shared sensor buffer with a 50ms timeout */
caMutex sensor_mutex;

void task_sensor_reader(void) {
    // Initialize mutex before use
    ca_mutex_init(&sensor_mutex);

    while (1) {
        // Try to lock for 50ms
        if (ca_mutex_timedlock(&sensor_mutex, 50)) {
            // Critical section: safely access shared data
            // Perform hardware read operation here
            ca_mutex_unlock(&sensor_mutex);
        } else {
            // Handle timeout: log error or retry later
            // ca_printf("Sensor busy, skipping cycle\n");
        }
        ca_os_delay_ms(100);
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_mutex_timedlock instead of a regular mutex lock?
A: You should use ca_mutex_timedlock when you want to avoid indefinite blocking, which can lead to deadlocks or unresponsive systems. It is especially useful in real-time systems where timing is critical and you need to ensure that a thread does not wait forever for a resource.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_mutex_timedlock in a caOS application?
A: ca_mutex_t mtx; ca_mutex_init(&mtx); bool acquired = ca_mutex_timedlock(&mtx, 1000); // Wait up to 1 second if (acquired) { // Critical section ca_mutex_unlock(&mtx); }

Parameters conf:0.95 **Q:** What does the timeout_ms parameter control in ca_mutex_timedlock?
A: The timeout_ms parameter specifies the maximum time (in milliseconds) the calling thread will wait for the mutex to become available. A value of 0 means the function will not wait at all and returns immediately if the mutex is not available. Values greater than 0 indicate the maximum wait time before returning false.

Return conf:0.90 **Q:** What do the return values of ca_mutex_timedlock indicate?
A: The function returns true if the mutex was successfully acquired within the timeout period. It returns false if the timeout expired before the mutex could be acquired or if an error occurred (e.g., invalid mutex pointer).

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_mutex_timedlock?
A: Common mistakes include using timeout_ms = 0 to mean 'no wait', which actually means 'try once and return immediately', and not checking for NULL mutex pointers. Additionally, calling this function from an ISR can cause undefined behavior or deadlock.

Behavior conf:0.90 **Q:** Is ca_mutex_timedlock a blocking function, and can it be called from an ISR?
A: Yes, ca_mutex_timedlock is a blocking function; it suspends the calling thread until the mutex is available or the timeout expires. It is not safe to call from an ISR because it may cause deadlock or undefined behavior due to its blocking nature and potential interaction with the scheduler.

`ca_mutex_trylock`

Header: `caos/inc/sync.h:68`
Source: `caos/src/sync.c:162:179`
GUARD: `EN_SYNC`

```
bool ca_mutex_trylock(ca_mutex_t *mtx)
```

Attempts to acquire a mutex immediately without blocking the calling thread.

Parametri:

Nome	Tipo	Descrizione
`mtx`	`ca_mutex_t *`	Pointer to the initialized mutex object. Must not be NULL; passing an uninitialized or NULL pointer will trigger a system safeguard failure.

Ritorna:
Returns true if the mutex was successfully acquired (it was free). Returns false if the mutex is already held by another thread, allowing the caller to perform other tasks instead of waiting.

Note:

- This function is non-blocking, meaning it returns immediately regardless of whether the lock was acquired. It is thread-safe because it uses an internal spinlock to protect the mutex state during the check-and-set operation. It is safe to call from an ISR, but note that it will never wait for the mutex to become available. The most common beginner mistake is assuming that the function will wait for the mutex to be released; if you need the thread to wait until the resource is free, you must use a blocking function instead. Always ensure the mutex has been properly initialized before calling this function.

Uso:

- Use this function when you want to perform a 'best-effort' operation on a shared resource without stalling the execution of your current thread. It is typically used in periodic tasks or background loops where you have other work to do if the resource is busy. It is often paired with ca_mutex_unlock to release the resource after the critical section. Unlike blocking mutex functions, this is ideal for preventing deadlocks in complex systems where a thread cannot afford to wait indefinitely for a resource.

Esempio:

```
ca_mutex_t mtx;

void task_function(void *p) {
    ca_mutex_init(&mtx);
    while(1) {
        /* Try to access shared resource without blocking the task */
        if (ca_mutex_trylock(&mtx)) {
            /* Critical section: safe to modify shared data */
            // shared_data++;
            ca_mutex_unlock(&mtx);
        } else {
            /* Resource busy, perform alternative background work */
            // perform_background_maintenance();
        }
        ca_os_sleep(100);
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_mutex_trylock instead of a blocking mutex acquisition?
A: You should use ca_mutex_trylock when you want to avoid blocking the calling thread. It's useful in time-sensitive applications or when you need to check if a resource is available without waiting. If the mutex is already in use, the function returns immediately with false, allowing the thread to continue execution.

Code example conf:0.90 **Q:** Can you show a minimal example of using ca_mutex_trylock with caOS types?
A: ca_mutex_t mtx; ca_mutex_init(&mtx); if (ca_mutex_trylock(&mtx)) { // Mutex acquired, critical section here ca_mutex_unlock(&mtx); } else { // Mutex was already in use }

Parameters conf:0.95 **Q:** What is the significance of the mtx parameter in ca_mutex_trylock?
A: The mtx parameter is a pointer to a mutex that must be initialized using ca_mutex_init(). It must not be NULL. If the mutex is not initialized or the pointer is invalid, the behavior is undefined.

Return conf:0.95 **Q:** What does it mean when ca_mutex_trylock returns false?
A: When ca_mutex_trylock returns false, it means the mutex was already in use by another thread. The calling thread did not acquire the mutex and should not enter the critical section.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_mutex_trylock?
A: Common mistakes include calling ca_mutex_trylock without initializing the mutex with ca_mutex_init(), calling it from an ISR (which is not allowed), and forgetting to release the mutex with ca_mutex_unlock() after acquiring it.

Behavior conf:0.90 **Q:** Is ca_mutex_trylock thread-safe and can it be called from an ISR?
A: Yes, ca_mutex_trylock is thread-safe as it uses an internal spinlock to protect concurrent access. However, it is not safe to call from an ISR because it may use functions that could cause blocking or invalid context, and it is explicitly forbidden in the documentation.

`ca_mutex_unlock`

Header: `caos/inc/sync.h:75`
Source: `caos/src/sync.c:181:211`
GUARD: `EN_SYNC`

```
void ca_mutex_unlock(ca_mutex_t *mtx)
```

Releases a mutex previously acquired by the current thread, potentially waking up the next waiting task.

Parametri:

Nome	Tipo	Descrizione
`mtx`	`ca_mutex_t *`	Pointer to the initialized mutex object. Must not be NULL.

Ritorna:
void

Note:

- This function is non-blocking and thread-safe. It is strictly forbidden to call this from an ISR (Interrupt Service Routine) because it involves thread scheduling and context management. A critical precondition is that the calling thread must be the current owner of the mutex; if a thread attempts to unlock a mutex it does not own, the system will trigger a CA_SAFEGUARD failure, which typically halts the system or triggers a kernel panic. Beginners often make the mistake of unlocking a mutex in an error path without checking if the lock was actually acquired, or forgetting to unlock it before returning from a function, which leads to permanent deadlocks where other threads are blocked forever.

Uso:

- This function is used to signal the end of a critical section, allowing other threads to access shared resources like hardware peripherals (AES, CRC, Hash engines) or shared memory buffers. It is always paired with ca_mutex_lock or ca_mutex_timedlock; the typical pattern is to lock the mutex, perform the sensitive operation, and immediately unlock it in all possible exit paths, including error handling blocks. By releasing the mutex, the scheduler is notified to wake up the next thread in the wait queue, ensuring efficient resource sharing and preventing priority inversion through the internal priority inheritance mechanism.

Esempio:

```
/* Example: Protecting a shared hardware resource */
void process_data(void) {
    // 1. Acquire exclusive access with a timeout
    if (ca_mutex_timedlock(&shared_mutex, 100)) {
```

```
        // 2. Perform hardware operation
        // Assume perform_hw_task is a placeholder for a hardware driver call
        /* perform_hw_task(); */

        // 3. Release access after work is done
        ca_mutex_unlock(&shared_mutex);
    } else {
        // Handle timeout: mutex was not acquired
        ca_printf("Failed to acquire mutex, skipping task\n");
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_mutex_unlock() in my embedded application?
A: You should call ca_mutex_unlock() after you have finished using a resource that was protected by a mutex, to allow other tasks to access it. It must be called by the same task that acquired the mutex with ca_mutex_lock().

Code example conf:0.90 **Q:** Can you show a minimal example of using ca_mutex_unlock() with caOS types?
A: ca_mutex_t mtx; ca_mutex_init(&mtx); ca_mutex_lock(&mtx); // Critical section code here ca_mutex_unlock(&mtx);

Parameters conf:0.95 **Q:** What happens if I pass NULL to ca_mutex_unlock()?
A: Passing NULL as the mtx parameter will cause a crash because the function requires a valid pointer to an initialized mutex structure.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_mutex_unlock()?
A: Common mistakes include calling ca_mutex_unlock() without first acquiring the mutex with ca_mutex_lock(), calling it from a different task than the one that locked it, or calling it on an already unlocked mutex, which may result in a crash or undefined behavior.

Behavior conf:0.90 **Q:** Is ca_mutex_unlock() thread-safe and can it be called from an ISR?
A: No, ca_mutex_unlock() is not thread-safe and must be called by the task that owns the mutex. It is also not safe to call from an ISR, as it is a blocking function that may wake up other tasks.

`ca_ncm_iface_get`

Header: `caos/net/inc/ncm_iface.h:44`
Source: `caos/net/src/ncm_iface.c:74:98`
GUARD: `EN_NCM_IFACE`

```
ca_net_iface_t *ca_ncm_iface_get(ca_usb_ncm_cfg_t *cfg)
```

Initializes and returns a pointer to a network interface structure for USB NCM (Network Control Model) communication.

Parametri:

Nome	Tipo	Descrizione
`cfg`	`ca_usb_ncm_cfg_t *`	Pointer to the NCM configuration structure. Must be a valid, non-NULL pointer containing the MAC address and USB parameters.

Ritorna:
Returns a pointer to a static ca_net_iface_t structure on success, or NULL if the provided configuration pointer is NULL.

Note:

- This function is a factory that initializes a static interface object. It is not thread-safe because it uses a static variable, meaning it should only be called once during system initialization. It is strictly non-blocking and must not be called from an ISR. A common mistake is attempting to call this function multiple times to create multiple interfaces; since it uses a static variable, subsequent calls will overwrite the previous interface configuration, leading to undefined behavior in the network stack.

Uso:

- This function is used during the system boot sequence to bridge the USB NCM driver with the caOS network core. It is typically called after defining the configuration structure but before starting the network or USB tasks. It pairs with the network stack initialization, as it sets the 'on_rx' callback that the USB driver will use to push incoming packets into the network core's processing queue. Once the interface is retrieved, it must be registered with the network core to become active.

Esempio:

```c
static ca_usb_ncm_cfg_t s_ncm = { .mac = {0x02, 0xCA, 0x05, 0x00, 0x00, 0xA5} };

void system_init(void) {
    ca_net_iface_t *iface = ca_ncm_iface_get(&s_ncm);
    if (iface != NULL) {
        /* Interface initialized, now register it with the network core */
        /* ca_net_register_iface(iface); */
    } else {
        /* Handle error: configuration was invalid */
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_ncm_iface_get and why is it needed for USB networking on STM32?
A: You should call ca_ncm_iface_get during the initialization phase of your USB networking setup to obtain a network interface structure that represents the USB NCM (Network Control Model) interface. This function is needed to prepare the networking subsystem for operation on STM32 microcontrollers using caOS, as it returns a pointer to a static structure that can be used to configure and initialize the USB network interface.

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_ncm_iface_get with caOS types?
A: ca_usb_ncm_cfg_t cfg = {0}; cfg.ep_in = 0x81; cfg.ep_out = 0x02; cfg.ep_int = 0x83; cfg.mtu = 1514; cfg.on_rx = NULL; ca_net_iface_t *iface = ca_ncm_iface_get(&cfg); if (iface != NULL) { // Use iface to initialize network stack }

Parameters conf:0.95 **Q:** What does the cfg parameter represent and what happens if it is NULL?
A: The cfg parameter is a pointer to a configuration structure for the USB NCM interface. It must be a valid, allocated structure. If cfg is NULL, the function returns NULL immediately, indicating that no valid interface could be created.

Return conf:0.90 **Q:** What does a NULL return value from ca_ncm_iface_get indicate?
A: A NULL return value from ca_ncm_iface_get indicates that the input parameter cfg was NULL, and thus no valid network interface could be created. The function does not allocate memory but returns a pointer to a static structure, so failure is only due to invalid input.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_ncm_iface_get?
A: Common mistakes include passing NULL as the cfg parameter, which results in a NULL return value, and calling this function multiple times with the same cfg struc-

ture, which overwrites the on_rx field and can cause unexpected behavior in the receive callback mechanism.

Behavior conf:0.90 **Q:** Is ca_ncm_iface_get blocking or thread-safe, and can it be called from an ISR?
A: ca_ncm_iface_get is not blocking and returns immediately. It is not thread-safe because it returns a pointer to a static structure, and should not be called from an ISR. It is intended for initialization use only, not for interrupt handling.

`ca_net_buf_alloc`

Header: `caos/net/inc/net_buf.h:120`
Source: `caos/net/src/net_buf.c:69:86`
GUARD: `EN_NET_BUF`

```
u16 ca_net_buf_alloc(void)
```

Allocates a network buffer from the static pool for packet construction.

Ritorna:
Returns a u16 index representing the allocated buffer handle, or NET_BUF_INVALID if the pool is exhausted.

Note:

- This function is non-blocking and thread-safe because it uses interrupt masking (___disable_irq) to protect the internal free-list index. It is safe to call from both main-loop tasks and Interrupt Service Routines (ISRs) because it does not rely on OS primitives like semaphores or mutexes. The most common mistake is failing to check the return value against NET_BUF_INVALID; if you attempt to use an invalid index with other functions like ca_net_buf_ptr(), you will likely trigger a memory corruption or a hard fault. Always ensure you have a corresponding free mechanism (not shown here) to return the buffer to the pool once transmission or processing is complete to avoid memory leaks.

Uso:

- This function is the entry point for any network transmission or packet reassembly task. It is typically used at the start of a service function (like ARP, DHCP, or DNS) to reserve memory before writing protocol headers or payload data. It is almost always followed by a call to ca_net_buf_ptr() to convert the index into a usable pointer, allowing the developer to write data into the buffer's memory space. If the allocation fails, the developer must gracefully abort the operation, often by freeing associated resources or simply returning early to prevent system instability.

Esempio:

```
u16 idx = ca_net_buf_alloc();
if (idx == NET_BUF_INVALID) {
    // Handle allocation failure: log error or drop packet
    return;
}

// Get the pointer to the buffer structure using the index
ca_net_buf_t *buf = ca_net_buf_ptr(idx);

// Prepare the buffer for data
buf->data_len = 100;
// ... fill buffer with data ...

```

```
// After use, the buffer must be returned to the pool
// ca_net_buf_free(idx); // Assuming a free function exists
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_net_buf_alloc() in a network application?
A: You should call ca_net_buf_alloc() when you need to allocate a network buffer for sending or receiving data. It is typically used in network service modules like ARP and DHCP to obtain a buffer from the network buffer pool before processing packets.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_net_buf_alloc()?
A: u16 buf_idx = ca_net_buf_alloc(); if (buf_idx != NET_BUF_INVALID) { // Use the buffer at index buf_idx // ... network operations ... ca_net_buf_free(buf_idx); } else { // Handle allocation failure }

Return conf:0.95 **Q:** What does it mean when ca_net_buf_alloc() returns NET_BUF_INVALID?
A: When ca_net_buf_alloc() returns NET_BUF_INVALID (0xFFFF), it means that no network buffers are available in the pool. The function will block until a buffer is freed by another part of the system before returning a valid buffer index.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_net_buf_alloc()?
A: Common mistakes include calling ca_net_buf_alloc() from an ISR context, which is not allowed; calling it outside of a critical section without proper synchronization, which can lead to deadlock or undefined behavior; and failing to call ca_net_buf_free() after allocation, leading to buffer leaks.

Behavior conf:0.90 **Q:** Is ca_net_buf_alloc() thread-safe and ISR-safe?
A: ca_net_buf_alloc() is not thread-safe if called outside of a critical section. It disables interrupts during allocation to ensure atomicity, but it must not be called from an ISR. The function is blocking when no buffers are available, waiting for one to be released.

`ca_net_buf_double_free_caller`

Header: `caos/net/inc/net_buf.h:155`
Source: `caos/net/src/net_buf.c:210:213`
GUARD: `EN_NET_BUF`

```
u32 ca_net_buf_double_free_caller(void)
```

Returns the memory address (link register) of the last code location that triggered a double-free error in the network buffer pool.

Ritorna:
Returns a u32 value representing the program counter or link register address where the illegal double-free operation occurred. If no double-free has been detected, it typically returns 0.

Note:

- This function is a diagnostic tool used to debug memory corruption in the network buffer pool. It is non-blocking and thread-safe as it simply reads a static variable updated by the network stack's internal error-handling logic. It is safe to call from any context, including ISRs, because it does not perform complex operations or synchronization. The most common mistake is assuming this value is always valid; it only contains meaningful data if ca_net_buf_double_free_count() returns a value greater than zero. If you see this value, use it with your debugger's symbol map to identify the exact line of code attempting to free a buffer that is already marked as free.

Uso:

- This function is used exclusively for post-mortem or real-time debugging of network stack stability. It is typically paired with ca_net_buf_double_free_count() to verify if an error has occurred before attempting to interpret the address returned by this function. In a production system, you would call this during a system health check or when a network watchdog triggers, allowing you to log the specific location of the bug to a CLI or a persistent log file.

Esempio:

```c
/* Check for network buffer double-free errors during system diagnostics */
#include "caos.h"

void check_network_health(void) {
    u32 error_count = ca_net_buf_double_free_count();

    if (error_count > 0) {
        /* Retrieve the address of the code that caused the double free */
        u32 caller_addr = ca_net_buf_double_free_caller();

        /* Log the error details to the CLI for the technician to analyze */
        ca_printf("CRITICAL: Network buffer double-free detected!\n");
        ca_printf("Total occurrences: %u\n", error_count);
        ca_printf("Last offending caller address: 0x%08X\n", caller_addr);

        /* Reset the counter after logging to allow detection of future errors
    */
        /* Note: ca_net_buf_pressure_reset() is used for pressure,
           but there is no specific function to reset the double-free counter
           in the provided API, so we just report it. */
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_net_buf_double_free_caller() in my caOS application?
A: You should call this function during system health checks or when a network watchdog triggers to debug memory corruption. It is intended to be used alongside ca_net_buf_double_free_coun to verify if an error has occurred before interpreting the returned address for post-mortem or real-time debugging.

Code example conf:1.00 **Q:** How can I check for a double-free error and retrieve the offending address in C?
A: You should verify the error count first to ensure the address is valid. Example: if (ca_net_buf_double_free_count() > 0) { u32 addr = ca_net_buf_double_free_caller(); /* Log addr to CLI or file */ }

Return conf:1.00 **Q:** What does the u32 return value represent, and what does a return of 0 signify?
A: The return value is the memory address (link register or program counter) of the code location that triggered the double-free error. A return value of 0 indicates that no double-free error has been detected by the network stack's internal error-handling logic.

`ca_net_buf_double_free_count`

Header: `caos/net/inc/net_buf.h:151`
Source: `caos/net/src/net_buf.c:205:208`
GUARD: `EN_NET_BUF`

```
u32 ca_net_buf_double_free_count(void)
```

Returns the total count of detected double-free errors in the network buffer pool.

Ritorna:
Returns a u32 value representing the number of times a buffer was freed while already in the free state. A value of 0 indicates the system is operating correctly.

Note:

- This function is thread-safe and non-blocking, as it simply reads a global counter. It is safe to call from both tasks and ISRs. It is intended for diagnostic purposes to detect memory corruption or logic errors where a buffer is released multiple times. The most common beginner mistake is ignoring a non-zero return value; if this count increases, it indicates a serious bug in the application's buffer management logic that will eventually lead to memory corruption or system instability.

Uso:

- This function is typically used in system health monitoring or diagnostic tasks to verify the integrity of the network buffer pool. It is often paired with ca_net_buf_double_free_caller() to identify the specific code location causing the error. Developers should check this value periodically or during system error logging to ensure that no double-free events have occurred during runtime, especially after complex network operations.

Esempio:

```
void check_network_health(void) {
    u32 error_count = ca_net_buf_double_free_count();

    if (error_count > 0U) {
        /* Retrieve the address of the code that caused the last double free */
        u32 caller_addr = ca_net_buf_double_free_caller();

        /* Log the error for debugging purposes */
        ca_printf("Network buffer integrity error! Double frees: %u, Last caller
    : 0x%08X\n",
                  error_count, caller_addr);
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_net_buf_double_free_count and what is its primary purpose?
A: You should call this function during system health monitoring or diagnostic tasks to verify the integrity of the network buffer pool. It is used to detect memory corruption or logic errors where a buffer is released multiple times, and it is often paired with ca_net_buf_double_free_caller() to identify the specific code location causing the error.

Code example conf:1.00 **Q:** How can I implement a simple check for network buffer double-free errors in my application?
A: You can periodically check the counter as shown here:
u32 error_count = ca_net_buf_double_free_count(); if (error_count > 0) { /* Handle error: buffer management logic is corrupted */ }

Return conf:1.00 **Q:** What does the return value of ca_net_buf_double_free_count signify, and what does a non-zero value imply?
A: The function returns a u32 representing the number of times a buffer was freed while already in the free state. A return value of 0 indicates the system is operating correctly. A non-zero value indicates a serious bug in the application's buffer management logic that will eventually lead to memory corruption or system instability.

ca_net_buf_free

Header: caos/net/inc/net_buf.h:123
Source: caos/net/src/net_buf.c:88:99
GUARD: EN_NET_BUF

```
void ca_net_buf_free(u16 idx)
```

Releases a network buffer back to the system pool, making it available for future allocation.

Parametri:

Nome	Tipo	Descrizione
idx	u16	The index of the network buffer to free. Must be a valid index within the range [0, NET_BUF_COUNT - 1].

Ritorna:
void

Note:

- This function is thread-safe because it uses critical sections (disabling interrupts) to protect the shared buffer pool state. It is safe to call from an ISR (Interrupt Service Routine) because it does not block or perform complex operations. The most common mistake is passing an invalid index or attempting to free the same buffer twice, which can corrupt the internal free-list stack. Always ensure the buffer is no longer being accessed by any network hardware or background tasks before calling this, as the memory will be immediately marked as available for reuse.

Uso:

- This function is used to clean up network buffers after a packet has been successfully sent, or if a packet construction process fails (e.g., protocol header generation fails). It is typically paired with a buffer allocation function (like ca_net_buf_alloc) at the start of a network task. In a typical lifecycle, you allocate a buffer, fill it with data, attempt to send it via ca_net_send, and if the send fails or the data is no longer needed, you call this function to prevent memory leaks in the network stack.

Esempio:

```
/* Example: Attempting to build a packet and freeing the buffer on failure */
u16 idx = ca_net_buf_alloc();
if (idx != INVALID_BUF_IDX) {
    ca_net_buf_t *b = ca_net_buf_ptr(idx);
    u16 len = build_my_packet(b->data);

    if (len == 0U) {
        /* Construction failed, release the buffer immediately */
        ca_net_buf_free(idx);
    } else {
        /* Send the packet; the network driver will handle freeing it later */
        ca_net_send(idx);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_net_buf_free() in a network application?
A: You should call ca_net_buf_free() when you are done processing a network buffer that was previously allocated using a function like ca_net_buf_alloc(). This is typically

done after sending or receiving data, to return the buffer to the pool for reuse. It's essential to free buffers to prevent memory leaks and ensure efficient use of limited network buffer resources.

Code example conf:0.90 **Q:** Can you show a minimal example of how to allocate and free a network buffer using caOS types?
A: u16 buf_idx = ca_net_buf_alloc(); if (buf_idx < NET_BUF_COUNT) { // Use the buffer... ca_net_buf_free(buf_idx); }

Parameters conf:0.95 **Q:** What does the 'idx' parameter in ca_net_buf_free() represent, and what are valid values?
A: The 'idx' parameter is the index of the network buffer to be freed. It must be an integer between 0 and NET_BUF_COUNT-1, inclusive. If idx is outside this range, the function does nothing and does not cause an error. Valid indices correspond to buffer slots in the global network buffer pool.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_net_buf_free()?
A: Common mistakes include freeing a buffer index that is out of range (less than 0 or greater than or equal to NET_BUF_COUNT), or freeing a buffer that was already freed. Using an invalid index will cause the function to do nothing, but incorrect usage can lead to undefined behavior or system crashes. Additionally, calling this function from an ISR without proper disablement of interrupts may cause blocking behavior.

`ca_net_buf_free_count`

Header: `caos/net/inc/net_buf.h:129`
Source: `caos/net/src/net_buf.c:106:109`
GUARD: `EN_NET_BUF`

```
1 u16 ca_net_buf_free_count(void)
```

Returns the current number of available network buffers in the system pool.

Ritorna:
Returns an unsigned 16-bit integer representing the count of buffers currently sitting in the free list. This value ranges from 0 (pool exhausted) to the total number of buffers defined in the system configuration.

Note:

- This function is non-blocking and thread-safe as it performs a simple read of a volatile counter. It is safe to call from both ISRs and standard tasks because it does not involve complex synchronization primitives or waiting. The most common beginner mistake is assuming this value remains constant; in a high-traffic network environment, this value can change between consecutive lines of code. Always cache the result in a local variable if you need to perform multiple calculations (like checking for leaks) to ensure consistency.

Uso:

- This function is primarily used for system monitoring, diagnostics, and debugging memory leaks within the network stack. It is typically paired with total pool size constants to calculate the percentage of memory usage or to detect if the system is running low on resources before attempting a large allocation. In diagnostic CLI commands, it is often used alongside other functions like 'ca_eth_pool_held_count' to verify that the sum of free and held buffers matches the total pool size, which helps identify if buffers are being 'lost' or leaked due to improper handling in the application logic.

Esempio:

```c
/* Check if we have enough buffers before attempting a network operation */
u16 available = ca_net_buf_free_count();
if (available > 0) {
    /* Proceed with allocation */
    ca_net_buf_t *buf = ca_net_buf_alloc();
    if (buf != NULL) {
        /* Use the buffer */
        ca_net_buf_free(buf);
    }
} else {
    /* Log a warning that the network pool is exhausted */
    ca_stdout_printf_n("Warning: No network buffers available!");
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_net_buf_free_count() and why is it useful in network applications?
A: You should call ca_net_buf_free_count() to check how many network buffers are available for allocation. This is useful in network applications to monitor buffer availability and prevent allocation failures. Since the function is non-blocking and can be called from any context including ISR, it allows real-time monitoring without risking system lockups. It is typically used in scenarios where you need to ensure sufficient buffer space before sending data or handling incoming packets.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_net_buf_free_count() with caOS types?
A: u16 free_buffers = ca_net_buf_free_count(); if (free_buffers > 0) { // Proceed with network buffer allocation } else { // Handle buffer exhaustion }

Return conf:0.90 **Q:** What do the different return values of ca_net_buf_free_count() mean and what can cause failures?
A: A positive return value indicates the number of free network buffers available for allocation. A return value of 0 means no buffers are available. A negative return value or unexpected return indicates an internal error or unspecified error condition. Failures can occur if the buffer pool is corrupted or not properly initialized, leading to inconsistent or unpredictable results.

`ca_net_buf_pool_init`

Header: `caos/net/inc/net_buf.h:116`
Source: `caos/net/src/net_buf.c:33:64`
GUARD: `EN_NET_BUF`

```c
void ca_net_buf_pool_init(void)
```

Initializes the network buffer memory pool and configures the MPU for cache-coherent DMA access.

Ritorna:
void

Note:

- This function is non-blocking and must be called exactly once during system startup before any network operations occur. It is not thread-safe and should never be called from an ISR, as it performs hardware-level MPU configuration which is intended for the initialization phase. A common mistake is calling this function after network buffers have already been allocated or used, which will reset the pool state, cause memory leaks, and

potentially corrupt active network packets. On Cortex-M7/M33 devices, it is mandatory to ensure the memory region is correctly aligned in the linker script, otherwise, the MPU configuration will fail or cause memory access faults.

Uso:

- This function is a foundational step in the network stack initialization sequence. It is typically called inside the main initialization routine, immediately after hardware-specific GPIO setup and before initializing the Ethernet driver or network interface. It pairs with functions like ca_eth_init or ca_net_init, ensuring that the zero-copy buffer pool is ready to handle incoming and outgoing data packets. Without this call, the network driver will lack the memory structures required to store packet data, leading to system crashes or silent packet drops.

Esempio:

```c
void system_init(void) {
    /* Initialize hardware pins for Ethernet */
    eth_gpio_init();

    /* Prepare the memory pool for zero-copy networking */
    ca_net_buf_pool_init();

    /* Initialize the network interface after the pool is ready */
    if (!ca_eth_init(&eth_cfg)) {
        /* Handle initialization failure */
        while(1);
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:1.00 **Q:** When should ca_net_buf_pool_init() be called in a caOS project?
A: The function should be called during system initialization, before any Ethernet driver initialization or network operations. It must be called only once and is required to set up the memory pool for network buffers.

Code example conf:1.00 **Q:** Can you provide a minimal example of how to use ca_net_buf_pool_init() in a caOS application?
A: u32 ret = ca_net_buf_pool_init(); // No return value expected, but check for errors if needed. // This function is typically called once at startup.

Parameters conf:1.00 **Q:** Does ca_net_buf_pool_init() accept any parameters?
A: No, the function does not take any parameters. It is a void function that initializes the network buffer pool using predefined memory settings.

Error/Edge case conf:1.00 **Q:** What are the common mistakes when using ca_net_buf_pool_init()?
A: Common mistakes include calling the function more than once, calling it after the Ethernet driver has been initialized, or invoking it from an ISR. These actions can lead to undefined behavior or memory corruption.

Behavior conf:1.00 **Q:** Is ca_net_buf_pool_init() thread-safe and ISR-safe?
A: No, the function is not thread-safe and not ISR-safe. It is a blocking function that must be called from the main initialization context, not from interrupt handlers.

`ca_net_buf_pressure`

Header: `caos/net/inc/net_buf.h:168`
Source: `caos/net/src/net_buf.c:225:239`
GUARD: `EN_NET_BUF`

```
void ca_net_buf_pressure(ca_net_buf_pressure_t *out)
```

Retrieves a snapshot of the network buffer pool statistics for diagnostic and monitoring purposes.

Parametri:

Nome	Tipo	Descrizione
out	ca_net_buf_pressure_t *	Pointer to a structure where the current buffer pool metrics (total, free, min, alloc calls, and failures) will be stored. Must not be NULL.

Ritorna:
None (void). If the provided pointer is NULL, the function returns immediately without performing any operations.

Note:

- This function is thread-safe because it uses critical sections (disabling interrupts) to ensure the snapshot of the buffer pool state is consistent and atomic. It is safe to call from an ISR, although it is primarily intended for diagnostic tasks or CLI commands. The most common mistake is passing a NULL pointer or an uninitialized structure, which will result in no data being captured or potential memory corruption if the pointer is invalid. Ensure the network buffer system is initialized via ca_net_buf_pool_init before calling this, otherwise, the returned values will reflect an uninitialized state.

Uso:

- This function is typically used in diagnostic CLI commands to monitor network health and detect potential memory leaks or buffer exhaustion. It is often paired with ca_eth_pool_held_count to distinguish between buffers held by the Ethernet driver and those available in the general pool. Developers use this to verify if the system is running low on memory (free_min) or if there are frequent allocation failures (alloc_fail) that might indicate a need for a larger buffer pool configuration.

Esempio:

```
void print_network_stats(void) {
    ca_net_buf_pressure_t stats;

    // Retrieve current buffer pool pressure metrics
    ca_net_buf_pressure(&stats);

    // Display the collected statistics to the console
    ca_printf("Net Buffer Stats: Total=%u, Free=%u, Min=%u, Failures=%u\n",
              (u32)stats.total, (u32)stats.free_now, (u32)stats.free_min, (u32)
    stats.alloc_fail);

    // Check if we are running dangerously low on buffers
    if (stats.free_now < 5) {
        ca_printf("Warning: Low network buffer availability!\n");
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_net_buf_pressure in my caOS application?
A: You should use it for diagnostic tasks, such as monitoring network health via CLI commands. It helps detect memory leaks or buffer exhaustion by checking the current state of the network buffer pool, specifically identifying if the system is running low on memory or experiencing frequent allocation failures.

Code example conf:1.00 **Q:** How do I correctly call ca_net_buf_pressure in C?
A: Ensure the buffer pool is initialized first, then pass a pointer to a ca_net_buf_pressure_t structure. Example: ca_net_buf_pressure_t stats; ca_net_buf_pressure(&stats);

Parameters conf:1.00 **Q:** What are the requirements for the 'out' parameter in ca_net_buf_pressure?
A: The 'out' parameter must be a valid pointer to a ca_net_buf_pressure_t structure. It must not be NULL, as the function will return immediately without performing any operations if a NULL pointer is provided.

Error/Edge case conf:1.00 **Q:** What happens if I call ca_net_buf_pressure before initializing the network buffer system?
A: The function will execute, but the returned values in the structure will reflect an uninitialized state of the buffer pool. Additionally, passing an uninitialized structure or a NULL pointer are common mistakes that lead to no data being captured or potential memory corruption.

`ca_net_buf_pressure_reset`

Header: `caos/net/inc/net_buf.h:172`
Source: `caos/net/src/net_buf.c:241:256`
GUARD: `EN_NET_BUF`

```
void ca_net_buf_pressure_reset(void)
```

Resets the network buffer pressure statistics, including allocation counters and the minimum free buffer watermark.

Ritorna:
void

Note:

- This function is thread-safe and safe to call from an ISR because it uses interrupt masking to protect the internal state variables. It is used to clear historical performance data so that new measurements do not include old peaks or failures. A common mistake is calling this too frequently, which can mask persistent memory leaks or buffer exhaustion issues by constantly resetting the 'minimum free' watermark to the current state.

Uso:

- This function is typically used in diagnostic or maintenance contexts, such as a CLI command or a system health monitor, to reset statistics after a configuration change or a period of high network activity. It is often paired with ca_net_buf_pressure to check the current state before and after the reset. By resetting the watermark, you ensure that the next call to ca_net_buf_pressure reflects only the buffer usage occurring after the reset point, rather than the historical worst-case scenario.

Esempio:

```
/* Example: Resetting network buffer stats via a CLI command */
if (ca_strcmp(input_buffer, "reset_stats") == 0) {
    /* Clear the current buffer pressure metrics */
    ca_net_buf_pressure_reset();

    /* Notify the user that the baseline has been cleared */
    ca_printf("Network buffer statistics have been reset.\n");
}

/* Example: Checking pressure after a reset */
ca_net_buf_pressure_reset();
```

```c
/* Perform a network operation */
ca_net_proto_tcp_build_data(...);
/* Check if the operation caused any new pressure issues */
if (ca_net_buf_pressure() < 10) {
    ca_printf("Warning: Low buffer availability detected after reset.\n");
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_net_buf_pressure_reset in my application?
A: You should call it during diagnostic or maintenance tasks, such as via a CLI command or system health monitor, particularly after a configuration change or following a period of high network activity to clear historical performance data.

Code example conf:0.90 **Q:** How do I implement a simple reset sequence for network buffer statistics?
A: You can reset the statistics and verify the state using the following sequence: ca_net_buf_pressure_r // Reset stats. Then, you can call ca_net_buf_pressure() to begin tracking new buffer usage.

Error/Edge case conf:1.00 **Q:** What is a common mistake developers make when using ca_net_buf_pressure_r
A: A common mistake is calling the function too frequently. This can mask persistent memory leaks or buffer exhaustion issues by constantly resetting the 'minimum free' watermark to the current state, preventing the detection of historical worst-case scenarios.

Behavior conf:1.00 **Q:** Is it safe to call ca_net_buf_pressure_reset from an interrupt service routine?
A: Yes, the function is thread-safe and safe to call from an ISR because it uses interrupt masking to protect its internal state variables.

`ca_net_buf_ptr`

Header: `caos/net/inc/net_buf.h:126`
Source: `caos/net/src/net_buf.c:101:104`
GUARD: `EN_NET_BUF`

```c
ca_net_buf_t *ca_net_buf_ptr(u16 idx)
```

Retrieves the memory address of a network buffer structure given its index.

Parametri:

Nome	Tipo	Descrizione
`idx`	`u16`	The index of the buffer in the internal pool. Must be a valid index previously returned by ca_net_buf_alloc(). Passing an out-of-bounds index will cause a memory access violation.

Ritorna:
A pointer to the ca_net_buf_t structure at the specified index. It does not return NULL, so the caller must ensure the index is valid before calling.

Note:

- This function is non-blocking and extremely fast as it performs a simple array offset calculation. It is thread-safe in the sense that it only reads from the static pool array, but it does not provide any synchronization; if another thread frees the buffer while you are accessing it, you will encounter a race condition. It is safe to call from an ISR because it contains no blocking logic or kernel primitives. The most common beginner mistake is using an index that has already been freed via ca_net_buf_free(), which leads to data corruption or 'use-after-free' bugs where your packet data is overwritten by another network task.

Uso:

- This function is used immediately after allocating a buffer to gain access to the structure's members, such as the data payload area or metadata fields. It is typically used in a sequence: first, call ca_net_buf_alloc() to reserve a slot, then call ca_net_buf_ptr() to get the pointer, then populate the buffer with data, and finally pass the buffer to a transmission function. It is essential for preparing outgoing packets in protocols like ARP, DHCP, or DNS, where you need to write headers directly into the buffer's memory space.

Esempio:

```
u16 idx = ca_net_buf_alloc();
if (idx != NET_BUF_INVALID) {
    // Get the pointer to the buffer structure
    ca_net_buf_t *buf = ca_net_buf_ptr(idx);

    // Access the data area, respecting the headroom for protocol headers
    u8 *payload = buf->data + NET_BUF_HEADROOM;
    ca_memset(payload, 0, 64);

    // Set metadata for the network stack
    buf->data_off = NET_BUF_HEADROOM;

    // Proceed to send or process the buffer
    // ca_net_send(idx);
} else {
    // Handle allocation failure (e.g., log error or drop packet)
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_net_buf_ptr() in my network application?
A: You should use ca_net_buf_ptr() when you need to access a previously allocated network buffer by its index. This function is typically called after allocating buffers with ca_net_buf_alloc() to retrieve a pointer to a specific buffer for reading or writing data. It's useful in network services like ARP or DHCP where you need to work with pre-allocated buffers without performing additional allocation or deallocation.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_net_buf_ptr() with caOS types?
A: ca_net_buf_t *buf = ca_net_buf_ptr(0); if (buf != NULL) { // Use the buffer for network operations buf->len = 64; // Process data in buf->data }

Parameters conf:0.90 **Q:** What does the idx parameter in ca_net_buf_ptr() represent and what are valid values?
A: The idx parameter represents the index of a previously allocated network buffer. Valid values are integers between 0 and the maximum number of available buffers. If idx is outside this range, the function returns NULL and accessing the returned pointer will cause undefined behavior. The index must have been returned by a prior call to ca_net_buf_alloc() and the buffer must not have been freed with ca_net_buf_free().

Return conf:0.95 **Q:** What do the return values of ca_net_buf_ptr() indicate and what causes a NULL return?
A: The function returns a valid pointer to a ca_net_buf_t structure if the index is valid and the buffer is allocated. It returns NULL if the index is out of bounds or if the buffer has been freed. A NULL return indicates that the buffer is either invalid or not available for use, and attempting to dereference it will result in memory access violations.

ca_net_buf_ref

Header: caos/net/inc/net_buf.h:148
Source: caos/net/src/net_buf.c:223:238
GUARD: EN_NET_BUF

```
bool ca_net_buf_ref(u16 idx)
```

Increments the reference count of a network buffer to prevent it from being freed while in use.

Parametri:

Nome	Tipo	Descrizione
idx	u16	The index of the network buffer in the pool. Must be less than NET_BUF_COUNT.

Ritorna:
Returns true if the reference was successfully incremented (buffer is active and count < 255), false otherwise (invalid index, buffer not in use, or max references reached).

Note:

- This function is thread-safe and uses critical sections (disabling interrupts) to ensure the reference count is updated atomically. It is safe to call from an ISR, but keep in mind that it performs a quick memory access and interrupt toggle. The most common mistake is failing to check the return value; if it returns false, the buffer is not valid for use, and attempting to use it will lead to memory corruption or undefined behavior. Always ensure the buffer was previously allocated via ca_net_buf_alloc before attempting to reference it.

Uso:

- This function is used when multiple parts of the network stack need to access the same buffer simultaneously, such as when a packet is being prepared for transmission and needs to be held in memory until the hardware driver confirms it has been sent. It is typically paired with ca_net_buf_free, which decrements the reference count. When the reference count reaches zero, the buffer is returned to the free pool. It is commonly used in TCP/IP service layers to keep segments 'in-flight' until an acknowledgment is received.

Esempio:

```
/* Example: Incrementing a buffer reference before sending */
u16 my_buf_idx = ca_net_buf_alloc();
if (my_buf_idx != NET_BUF_INVALID) {
    /* Increment reference so the buffer stays alive during transmission */
    if (ca_net_buf_ref(my_buf_idx)) {
        /* Send the buffer; ca_net_send will handle the release/decrement */
        if (!ca_net_send(my_buf_idx)) {
            /* If send fails, we must manually release the reference we just took */
            ca_net_buf_free(my_buf_idx);
        }
    } else {
        /* Failed to reference, free the initial allocation */
        ca_net_buf_free(my_buf_idx);
    }
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_net_buf_ref in my network stack implementation?
A: You should use ca_net_buf_ref when multiple parts of the network stack need to access the same buffer simultaneously. It is typically used to keep packets in memory, such as when a packet is being prepared for transmission and must be held until the hardware driver confirms it has been sent, or in TCP/IP service layers to keep segments 'in-flight' until an acknowledgment is received.

Code example conf:1.00 **Q:** Provide a minimal C example showing how to safely reference a network buffer.
A: u16 my_buf_idx = 5; if (ca_net_buf_ref(my_buf_idx)) { /* Buffer is safe to use */ } else { /* Handle error: buffer invalid or max refs reached */ }

Parameters conf:1.00 **Q:** What are the constraints for the idx parameter in ca_net_buf_ref?
A: The idx parameter represents the index of the network buffer in the pool. It must be a u16 value that is strictly less than NET_BUF_COUNT.

Return conf:1.00 **Q:** What does a return value of false indicate when calling ca_net_buf_ref?
A: A return value of false indicates that the reference increment failed. This occurs if the index is invalid, the buffer is not currently in use, or the maximum reference count of 255 has already been reached.

Error/Edge case conf:1.00 **Q:** What is the most common mistake developers make when using ca_net_buf_ref?
A: The most common mistake is failing to check the return value. If the function returns false, the buffer is not valid for use, and attempting to use it will lead to memory corruption or undefined behavior.

Behavior conf:1.00 **Q:** Is ca_net_buf_ref safe to call from an ISR, and does it block execution?
A: Yes, it is safe to call from an ISR. The function is thread-safe and uses critical sections (disabling interrupts) to ensure the reference count is updated atomically. It is non-blocking, as it only performs a quick memory access and interrupt toggle.

`ca_net_buf_shared_count`

Header: `caos/net/inc/net_buf.h:152`
Source: `caos/net/src/net_buf.c:240:243`
GUARD: `EN_NET_BUF`

```
1 u16 ca_net_buf_shared_count(void)
```

Returns the current number of network buffers that are shared between multiple owners.

Ritorna:
A 16-bit unsigned integer representing the count of buffers currently marked as shared in the network pool.

Note:

- This function is non-blocking and thread-safe as it performs a simple read of a global counter. It is safe to call from an ISR because it does not involve any complex synchronization primitives or blocking operations. The primary purpose of this function is diagnostic; it allows the system to track how many network buffers are being referenced by more than one internal module (e.g., both the Ethernet driver and the TCP stack). A common mistake is to assume this value should always be zero; in a high-traffic network environment, shared buffers are normal, but a rapidly increasing count that never returns to a baseline may indicate a memory leak or a failure to properly release buffer references.

Uso:

- This function is typically used in system monitoring, debugging, or diagnostic CLI commands to verify the health of the network buffer pool. It is often paired with other diagnostic functions like ca_net_buf_double_free_count or ca_net_buf_pressure to provide a complete picture of memory usage. Developers use it to calculate the 'true' number of buffers in use by subtracting the shared count from the total number of active references, ensuring that buffers held by multiple layers are not double-counted in memory statistics.

Esempio:

```c
/* Diagnostic function to print network buffer usage statistics */
void print_net_stats(void) {
    u16 shared = ca_net_buf_shared_count();
    u32 double_free = ca_net_buf_double_free_count();

    /* Print the current count of shared buffers for debugging */
    ca_printf_n("Network Diagnostics: Shared Buffers = %u", (u32)shared);

    /* Check if we have an abnormal amount of double-free errors */
    if (double_free > 0) {
        ca_printf_n("Warning: Detected %u double-free events!", double_free);
    }

    /* Retrieve current pressure status of the buffer pool */
    u16 pressure = ca_net_buf_pressure();
    ca_printf_n("Current buffer pool pressure: %u", (u32)pressure);
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_net_buf_shared_count in my application?
A: You should use this function for system monitoring, debugging, or diagnostic CLI commands. It is primarily used to track how many network buffers are referenced by multiple internal modules, such as the Ethernet driver and the TCP stack. It helps in calculating the 'true' number of buffers in use by preventing double-counting in memory statistics.

Code example conf:1.00 **Q:** How can I implement a simple diagnostic check for shared network buffers?
A: You can call the function directly as it is non-blocking and thread-safe. Example: void check_net_health(void) { u16 shared = ca_net_buf_shared_count(); if (shared > 100) { /* Log potential memory leak */ } }

Return conf:1.00 **Q:** What does the return value of ca_net_buf_shared_count represent, and does it indicate a failure if it is non-zero?
A: The return value is a u16 representing the number of network buffers currently marked as shared between multiple owners. A non-zero value is not necessarily a failure; in high-traffic environments, shared buffers are normal. However, a rapidly increasing count that never returns to a baseline may indicate a memory leak or failure to release buffer references.

`ca_net_cksum`

Header: `caos/net/inc/net_proto.h:259`
Source: `caos/net/src/net_proto.c:143:157`
GUARD: `EN_NET_PROTO`

```c
u16 ca_net_cksum(const u8 *buf, u16 len)
```

Calculates the 16-bit Internet Checksum (RFC 1071) for a given data buffer.

Parametri:

Nome	Tipo	Descrizione
`buf`	`const u8 *`	Pointer to the memory buffer containing the data to be checksummed. Must be a valid memory address.
`len`	`u16`	The length of the buffer in bytes. If odd, the function correctly pads the last byte with a zero byte.

Ritorna:
Returns a 16-bit unsigned integer representing the calculated checksum. There are no error return values as the calculation is deterministic.

Note:

- This function is a pure mathematical calculation and is non-blocking, thread-safe, and safe to call from an ISR. It does not perform any I/O or memory allocation, so it has no preconditions other than the buffer being valid and accessible. A common beginner mistake is forgetting to set the checksum field in the header to 0 before calculating the checksum, or failing to use ca_htons() on the result before placing it into a network-bound packet header. Because it processes data in 16-bit chunks, ensure the buffer pointer is valid for the entire length specified to avoid memory access violations.

Uso:

- This function is used whenever you are constructing network packets (IP, TCP, UDP) from scratch. The typical workflow involves: 1) Populating the packet header fields, 2) Setting the checksum field to 0, 3) Calling ca_net_cksum() on the header (or header + payload), 4) Converting the result to network byte order using ca_htons(), and 5) Writing that value into the packet header. It is essential for ensuring data integrity at the network layer, as receiving devices will re-calculate this value to verify the packet was not corrupted during transmission.

Esempio:

```
/* Example: Calculating IP header checksum */
struct ip_header *ip4 = (struct ip_header *)packet_buffer;

/* 1. Initialize header fields */
ip4->version_ihl = 0x45;
ip4->total_len = ca_htons(20 + payload_len);
ip4->cksum = 0U; /* Mandatory: set to 0 before calculation */

/* 2. Calculate checksum over the 20-byte header */
u16 checksum = ca_net_cksum((const u8 *)ip4, 20U);

/* 3. Store in network byte order */
ip4->cksum = ca_htons(checksum);
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_net_cksum in a caOS-based network application?
A: You should use ca_net_cksum when computing the Internet checksum for IP, TCP, or UDP packets to ensure data integrity during transmission. It is typically called when constructing network headers before sending data over the network interface.

Code example conf:0.90 **Q:** Can you provide a minimal example of how to use ca_net_cksum with caOS types?
A: u8 data[] = {0x45, 0x00, 0x00, 0x34}; u16 checksum = ca_net_cksum(data, 4);

Parameters conf:0.90 **Q:** What are the valid ranges for the buffer pointer and length parameters in ca_net_cksum?

A: The buffer pointer must be non-NULL and point to valid memory. The length must be greater than zero; values too large may cause overflow or undefined behavior. The buffer must contain correctly formatted protocol data (e.g., IP header).

Return conf:0.85 **Q:** What does a return value of 0 from ca_net_cksum indicate?

A: The function returns a u16 checksum in network byte order (big-endian). A return value of 0 indicates that the computed checksum is zero, which can occur under specific data patterns but is not an error condition itself.

Error/Edge case conf:0.80 **Q:** What are common mistakes when using ca_net_cksum in ISR context?

A: Common mistakes include calling ca_net_cksum in an ISR without considering its non-blocking nature and potential for long execution times due to large buffers. This can lead to system delays or missed deadlines. It's recommended to avoid using it in ISR unless thoroughly tested.

`ca_net_count_err`

Header: `caos/net/inc/net.h:122`
Source: `caos/net/src/net.c:104:108`
GUARD: `EN_NET`

```
void ca_net_count_err(bool bus, bool rbu)
```

Increments internal network error counters for bus or buffer-related faults.

Parametri:

Nome	Tipo	Descrizione
`bus`	`bool`	Set to true to increment the bus error counter; false to ignore.
`rbu`	`bool`	Set to true to increment the RX buffer unavailable (RBU) counter; false to ignore.

Ritorna:
void (no return value).

Note:

- This function is non-blocking and executes extremely quickly, making it safe to call from Interrupt Service Routines (ISRs). It is not thread-safe by itself, as it performs a simple increment on a static variable; if multiple interrupts or tasks access this simultaneously, you might experience race conditions where increments are lost. It assumes the network statistics structure has been initialized during system startup. A common mistake is calling this function without checking the actual hardware flags, leading to inflated error counts that do not reflect the actual state of the network interface.

Uso:

- This function is typically used inside hardware-specific interrupt handlers (ISRs) when the Ethernet controller reports an error status. It acts as a bridge between the low-level driver and the core network statistics module, allowing the system to track health metrics regardless of the specific hardware interface. It is often paired with semaphore operations, such as ca_sem_post_capped, to signal a background task that it needs to recover from a buffer-related error (RBU) after the counter has been updated.

Esempio:

```
/* Inside an Ethernet interrupt handler */
void eth_isr_handler(void) {
```

```
    u32 status = get_eth_status_register();

    /* Check for specific error bits */
    bool bus_err = (status & CA_ETH_ERR_BUS) != 0;
    bool rbu_err = (status & CA_ETH_ERR_RBU) != 0;

    /* Update global stats */
    ca_net_count_err(bus_err, rbu_err);

    /* If RX buffer was full, signal the network task to process packets */
    if (rbu_err) {
        ca_sem_post_capped(&ca_net_rx_sem);
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_net_count_err() in an ISR context, and what is its purpose in network error handling?
A: You should call ca_net_count_err() in an ISR when handling network-related interrupts to increment error counters for bus errors or RX buffer unavailable conditions. It is used to track network errors in real-time without blocking the interrupt handler. The function is designed to be non-blocking and thread-safe, making it suitable for ISR usage where performance and safety are critical.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_net_count_err() with caOS types?
A: void example_isr_handler(void) { // Simulate a bus error ca_net_count_err(true, false); // Simulate an RBU error ca_net_count_err(false, true); }

Parameters conf:0.90 **Q:** What do the boolean parameters bus and rbu represent in ca_net_count_err, and what values are valid for them?
A: The parameter 'bus' is a boolean that, when true, increments the counter for bus-related errors. The parameter 'rbu' is a boolean that, when true, increments the counter for RX buffer unavailable errors. Both parameters must be either true or false; no other values are valid. Invalid values are not explicitly handled and could lead to unexpected behavior if used inappropriately.

`ca_net_filter_clear`

Header: `caos/net/inc/net.h:149`
Source: `caos/net/src/net.c:336:339`
GUARD: `EN_NET`

```
void ca_net_filter_clear(void)
```

Removes the currently active network packet filter function.

Ritorna:
This function returns nothing (void).

Note:

- This function is non-blocking and executes immediately by setting the internal filter pointer to NULL. It is thread-safe in the sense that it performs a single atomic pointer assignment, but it does not provide synchronization if other tasks are concurrently trying to set a new filter. It is safe to call from an ISR, as it involves no complex logic or resource locking. The most common mistake is calling this while a network process is actively relying on the

filter to drop or accept packets, which may cause a temporary 'leak' of unfiltered traffic until the next filter is set.

Uso:

- This function is used to disable packet filtering when the application no longer needs to inspect or drop specific network traffic. It is typically paired with ca_net_filter_set, which is used to install a new filter function. In a typical lifecycle, you might call ca_net_filter_set during initialization or when entering a specific protocol state, and call ca_net_filter_clear when that state ends or when the network interface is being shut down or reconfigured.

Esempio:

```
/* Example: Clearing a network filter before re-initialization */

void reset_network_security(void) {
    /* Disable the current packet filter to allow all traffic temporarily */
    ca_net_filter_clear();

    /* Log the action using standard caOS output */
    ca_printf("Network filter cleared successfully.\n");

    /* Perform other cleanup tasks if necessary */
    // ca_net_buf_pressure_reset();
}
```

Q&A — fine-tuning review *2/2 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_net_filter_clear and what are the potential side effects?
A: You should call ca_net_filter_clear when you no longer need to inspect or drop specific network traffic, such as when a protocol state ends or the network interface is being reconfigured. Be aware that calling this while a network process is actively relying on the filter may cause a temporary 'leak' of unfiltered traffic until a new filter is set.

Code example conf:1.00 **Q:** How do I implement a simple routine to disable the current network packet filter in caOS?
A: To disable the filter, simply call the function directly. Since it is non-blocking and safe for ISRs, it can be used as follows:
void disable_network_security(void) { ca_net_filter_clear(); }

`ca_net_filter_count`

Header: `caos/net/inc/net.h:152`
Source: `caos/net/src/net.c:341:344`
GUARD: `EN_NET`

```
u32 ca_net_filter_count(void)
```

Returns the total number of network packets that have been processed by the active filter.

Ritorna:
Returns a u32 value representing the cumulative count of packets filtered since the last reset or initialization.

Note:

- This function is non-blocking and thread-safe as it performs a simple read of a static variable. It is safe to call from an ISR, as it does not involve any synchronization primitives or complex logic. The primary precondition is that the network stack must be initialized

via ca_net_init before this value becomes meaningful. A common mistake is assuming this count represents 'dropped' packets; it specifically tracks packets that passed through the filter logic, which may include accepted or rejected packets depending on the filter implementation.

Uso:

- This function is typically used for diagnostic and monitoring purposes within a CLI or a system health dashboard. It is often paired with ca_net_filter_clear to reset the statistics after a reporting interval or when reconfiguring the network interface. Developers use this in conjunction with other statistics functions like ca_net_stats to verify that the network traffic is being correctly intercepted and processed by the system's security or routing filters.

Esempio:

```c
/* Example: Displaying network filter statistics via CLI */
#include "caos.h"

void display_net_stats(void)
{
    u32 filter_hits = ca_net_filter_count();

    /* Print the current filter count to the console */
    ca_printf("Total packets processed by filter: %u\n", filter_hits);

    /* If the count is too high, we might want to clear it to avoid overflow */
    if (filter_hits > 1000000) {
        ca_net_filter_clear();
        ca_printf("Filter counter reset due to high volume.\n");
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_net_filter_count in my caOS application?
A: You should use this function for diagnostic and monitoring purposes, such as within a CLI or a system health dashboard. It is typically used to verify that network traffic is being correctly intercepted and processed by security or routing filters, often in conjunction with ca_net_stats or when resetting statistics via ca_net_filter_clear.

Code example conf:0.95 **Q:** How do I implement a simple diagnostic check for the network filter using caOS types?
A: You can retrieve the count and store it in a u32 variable as shown:
#include "caos.h"
void check_filter(void) { u32 packet_count = ca_net_filter_count(); // Use packet_count for monitoring or logging }

Return conf:0.90 **Q:** What does the u32 return value represent, and can this function fail?
A: The return value is a u32 representing the cumulative count of packets processed by the active filter since the last reset or initialization. The function is non-blocking and thread-safe, meaning it does not fail; however, the value is only meaningful if the network stack has been initialized via ca_net_init.

`ca_net_filter_mac_set`

Header: `caos/net/inc/net.h:184`
Source: `caos/net/src/net.c:433:442`
GUARD: `EN_NET`

```
bool ca_net_filter_mac_set(u8 slot, const ca_mac_t *src,
    ca_net_mac_filter_fn_t fn)
```

Assigns a specific MAC address and a callback function to a network filter slot.

Parametri:

Nome	Tipo	Descrizione
slot	u8	The index of the filter slot to configure. Must be less than NET_MAC_FILTER_MAX.
src	const ca_mac_t *	Pointer to the MAC address structure to filter. If NULL, the slot is effectively disabled.
fn	ca_net_mac_filter_fn_t	The callback function to execute when a matching MAC is received. If NULL, the slot is disabled.

Ritorna:
Returns true if the slot was successfully configured or cleared. Returns false if the provided slot index is out of bounds.

Note:

- This function is not thread-safe and should be used with caution in multi-threaded environments. It is non-blocking and executes immediately, making it safe to call from most contexts, but it does not perform internal locking. The most common beginner mistake is passing an invalid slot index or failing to ensure the callback function is properly defined and accessible. Always ensure the network stack is initialized before attempting to configure filters, as the underlying filter array must be valid.

Uso:

- This function is typically used to implement custom packet routing or relay logic where specific traffic from a known device needs to be intercepted. It is usually called after resolving a device's MAC address via ca_arp_resolve. The typical sequence involves waiting for a successful ARP resolution, then calling this function to 'arm' the filter with a specific callback function that processes the incoming packets for that device.

Esempio:

```
/* Example: Setting a filter for a specific device after ARP resolution */
ca_mac_t target_mac;
/* Assume s_target_ip is defined elsewhere */
if (ca_arp_resolve(s_target_ip, &target_mac, 2000U)) {
    /* Configure slot 0 to trigger 'my_packet_handler' for this MAC */
    if (ca_net_filter_mac_set(0U, &target_mac, my_packet_handler)) {
        ca_printf_n("Filter armed for target device");
    } else {
        ca_printf_n("Failed to set filter: invalid slot");
    }
} else {
    ca_printf_n("ARP resolution failed");
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_net_filter_mac_set in my network application?
A: This function is typically used to implement custom packet routing or relay logic. It is best called after successfully resolving a device's MAC address via ca_arp_resolve to 'arm' a filter with a callback function that processes incoming packets for that specific device.

Code example conf:0.95 **Q:** How do I configure a MAC filter slot in caOS?

A: You must provide a valid slot index, a pointer to the ca_mac_t structure, and your callback function. Example: u8 slot = 0; ca_mac_t target_mac = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55}; bool success = ca_net_filter_mac_set(slot, &target_mac, my_callback_fn);

Parameters conf:1.00 **Q:** What are the constraints for the 'slot' and 'fn' parameters?

A: The 'slot' parameter must be an index less than NET_MAC_FILTER_MAX. If 'src' is NULL or 'fn' is NULL, the slot is effectively disabled. Providing an invalid slot index will result in a failure.

Return conf:1.00 **Q:** What do the return values of ca_net_filter_mac_set indicate?

A: The function returns true if the slot was successfully configured or cleared. It returns false if the provided slot index is out of bounds (i.e., not less than NET_MAC_FILTER_MAX).

`ca_net_filter_set`

Header: `caos/net/inc/net.h:148`
Source: `caos/net/src/net.c:324:334`
GUARD: `EN_NET`

```
1 void ca_net_filter_set(u16 ethertype, ca_net_filter_fn_t fn)
```

Registers a custom callback function to process incoming network frames matching a specific EtherType.

Parametri:

Nome	Tipo	Descrizione
`ethertype`	`u16`	The 16-bit Ethernet protocol identifier (e.g., 0x0800 for IPv4, 0x88B5 for custom protocols) to filter for.
`fn`	`ca_net_filter_fn_t`	The function pointer to be called when a matching frame is received. Pass NULL to disable the filter.

Ritorna:
void

Note:

- This function is not thread-safe and must be used carefully in multi-threaded environments. It is designed to be called from the main configuration task or a control task, not from an ISR. The implementation uses a specific sequence of operations (clearing the callback before updating the EtherType) to ensure that the network task never attempts to process a frame using a mismatched callback. A common mistake is to assume this function is atomic; if you change the filter while the network task is actively processing frames, you might experience a race condition where a frame is dropped or processed by the wrong logic. Always ensure the network stack is in a stable state before reconfiguring filters.

Uso:

- This function is typically used to intercept custom network protocols or specific traffic types that the standard stack does not handle. It is usually called during the system initialization phase after the network interface is up, or when switching operational modes. It is often paired with ca_net_process() in a loop, where the registered callback will be triggered internally by the stack when a matching frame arrives. To stop filtering, you should call this function again with the same EtherType and a NULL function pointer, or use ca_net_filter_clear().

Esempio:

```
/* Define a custom filter function to handle incoming frames */
void my_custom_filter(void *buf, u16 len) {
    /* Process the buffer here using ca_net_buf_ptr and other net functions */
    /* Always free the buffer after processing */
    ca_net_buf_free(buf);
}

void setup_network_filter(void) {
    /* Ensure the network link is active before arming the filter */
    if (ca_net_link_status()) {
        /* Register the filter for a custom EtherType 0x88B5 */
        ca_net_filter_set(0x88B5, my_custom_filter);
    }
}

/* Inside the main network task loop */
void net_task(void *p) {
    while (1) {
        /* Wait for incoming data and process it */
        ca_sem_wait(&ca_net_rx_sem);
        ca_net_process();
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to call ca_net_filter_set in a caOS application?
A: It is typically called during the system initialization phase after the network interface is up, or when switching operational modes. It should be called from the main configuration task or a control task, not from an ISR.

Code example conf:0.90 **Q:** How do I register a custom callback for an IPv4 frame using caOS types?
A: You define a function matching the ca_net_filter_fn_t signature and pass it to the filter function: void my_handler(void* frame) { /* process */ } ... ca_net_filter_set(0x0800, my_handler);

Parameters conf:1.00 **Q:** What are the valid values for the ethertype parameter and how do I disable a filter?
A: The ethertype is a u16 representing the Ethernet protocol identifier, such as 0x0800 for IPv4 or 0x88B5 for custom protocols. To disable a filter, pass NULL as the function pointer argument.

Error/Edge case conf:1.00 **Q:** Is ca_net_filter_set thread-safe, and what happens if I call it while the network task is active?
A: No, it is not thread-safe. If you change the filter while the network task is actively processing frames, you risk a race condition where a frame might be dropped or processed by the wrong logic. Always ensure the network stack is in a stable state before reconfiguring.

`ca_net_iface_at`

Header: `caos/net/inc/net.h:130`
Source: `caos/net/src/net.c:168:171`
GUARD: `EN_NET`

```
ca_net_iface_t *ca_net_iface_at(u8 id)
```

Retrieves a pointer to a network interface structure by its numerical index.

Parametri:

Nome	Tipo	Descrizione
id	u8	The zero-based index of the network interface. Must be less than the value returned by ca_net_iface_count().

Ritorna:
Returns a pointer to the ca_net_iface_t structure if the ID is valid. Returns NULL if the ID is out of range (i.e., >= total interface count).

Note:

- This function is non-blocking and thread-safe as it performs a simple read-only array access. It is safe to call from an ISR, provided the network stack has been initialized. The most common mistake is failing to check the return value for NULL before dereferencing the pointer, which will cause a hard fault if the provided ID is invalid. Always ensure you have called ca_net_proto_init or equivalent initialization routines before attempting to access interfaces.

Uso:

- This function is typically used to iterate through all available network interfaces to perform status checks, configuration, or statistics reporting. It is frequently paired with ca_net_iface_count() to define the loop bounds. Developers use it when they need to access specific interface properties like MTU, link status, or hardware checksum capabilities, often in CLI commands or system initialization sequences.

Esempio:

```
/* Iterate through all network interfaces to display their status */
u8 count = ca_net_iface_count();
for (u8 i = 0U; i < count; i++) {
    ca_net_iface_t *iface = ca_net_iface_at(i);
    if (iface != NULL) {
        /* Safely access interface properties */
        ca_printf("Interface %u: %s\n", (u32)iface->id, iface->up ? "UP" : "DOWN
    ");
    } else {
        /* Handle unexpected NULL pointer */
        ca_printf("Error: Interface at index %u not found\n", (u32)i);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_net_iface_at function in my caOS application?
A: You should use this function when you need to access specific network interface properties such as MTU, link status, or hardware checksum capabilities. It is typically used to iterate through all available interfaces for status checks, configuration, or statistics reporting, often paired with ca_net_iface_count() to define loop bounds.

Code example conf:1.00 **Q:** How do I iterate through all network interfaces using caOS types?
A: You can iterate through the interfaces as follows: u8 count = ca_net_iface_count(); for(u8 i = 0; i < count; i++) { ca_net_iface_t *iface = ca_net_iface_at(i); if(iface != NULL) { /* process iface */ } }

Parameters conf:1.00 **Q:** What are the constraints for the 'id' parameter in ca_net_iface_at?
A: The 'id' parameter is a zero-based index of the network interface. It must be less than the value returned by ca_net_iface_count(). Providing an ID equal to or greater than the total interface count is invalid and will result in a NULL return.

Return conf:1.00 **Q:** What does the return value of ca_net_iface_at indicate, and what happens if it fails?
A: The function returns a pointer to the ca_net_iface_t structure if the ID is valid. If the ID is out of range, it returns NULL. Failing to check for NULL before dereferencing the pointer will cause a hard fault.

`ca_net_iface_count`

Header: `caos/net/inc/net.h:129`
Source: `caos/net/src/net.c:163:166`
GUARD: `EN_NET`

```
u8 ca_net_iface_count(void)
```

Returns the total number of network interfaces currently registered in the system.

Ritorna:
Returns an unsigned 8-bit integer (u8) representing the count of initialized network interfaces. If no interfaces are registered, it returns 0.

Note:

- This function is non-blocking and thread-safe as it simply returns a static counter value. It is safe to call from an ISR (Interrupt Service Routine) because it does not perform any complex operations or resource locking. The primary precondition is that the network stack must have been initialized via ca_net_init or similar setup functions; otherwise, the count will remain zero. A common mistake is assuming the count is constant; in dynamic systems, interfaces might be added or removed, so always re-query the count before iterating through interfaces.

Uso:

- This function is typically used to determine the loop bounds when iterating over all available network interfaces. It is commonly paired with ca_net_iface_at(id) to access specific interface details (like name, status, or MTU) for diagnostic purposes, CLI reporting, or system configuration. Developers use this sequence: first, ensure the network stack is initialized, then call ca_net_iface_count to get the limit, and finally loop from 0 to (count - 1) to process each interface individually.

Esempio:

```
/* Example: Iterating through all network interfaces to print their status */
#include "caos.h"

void print_all_iface_status(void) {
    u8 count = ca_net_iface_count();

    if (count == 0) {
        ca_printf_n("No network interfaces found.");
        return;
    }

    for (u8 i = 0; i < count; i++) {
        /* Retrieve the interface pointer using the index */
        const ca_net_iface_t *iface = ca_net_iface_at(i);

        if (iface != NULL) {
            /* Print interface details using the available ca_printf_n */
            ca_printf_n("Interface %u: %s is %s",
```

```
                            (u32)iface->id,
                            iface->name ? iface->name : "unknown",
                            iface->up ? "UP" : "DOWN");
        }
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_net_iface_count and what is its primary purpose?
A: You should call ca_net_iface_count to determine the loop bounds when you need to iterate over all available network interfaces. It is typically used to prepare for accessing specific interface details via ca_net_iface_at(id) for tasks like CLI reporting, diagnostics, or system configuration.

Code example conf:1.00 **Q:** How do I write a loop to iterate through all network interfaces using caOS types?
A: After ensuring the network stack is initialized, you can iterate through the interfaces as follows: u8 count = ca_net_iface_count(); for(u8 i = 0; i < count; i++) { /* process interface at index i */ }

Return conf:1.00 **Q:** What does the return value of ca_net_iface_count represent, and why might it return 0?
A: The function returns a u8 representing the total number of initialized network interfaces. It returns 0 if no interfaces are registered or if the network stack has not been initialized via ca_net_init or similar setup functions.

`ca_net_iface_link`

Header: `caos/net/inc/net.h:133`
Source: `caos/net/src/net.c:173:185`
GUARD: `EN_NET`

```
bool ca_net_iface_link(u8 id, bool up)
```

Enables or disables a network interface by triggering its start or stop routines.

Parametri:

Nome	Tipo	Descrizione
`id`	`u8`	The index of the network interface to modify. Must be less than the total number of interfaces returned by ca_net_iface_count().
`up`	`bool`	Set to true to bring the interface up (start), or false to bring it down (stop).

Ritorna:
Returns true if the operation was successful (interface found and state transition completed), or false if the interface ID is invalid or the start routine failed.

Note:

- This function is not thread-safe and should be called from a single task context, typically during system initialization or configuration changes. It is not safe to call from an ISR because the underlying start/stop routines may involve blocking operations or complex resource management. A common mistake is passing an invalid ID without checking against ca_net_iface_count(), which will cause the function to return false immediately. Ensure the network stack is initialized via ca_net_init() before attempting to link any interfaces.

Uso:

- This function is used to manage the lifecycle of network hardware (like Ethernet or Wi-Fi modules) after they have been registered with the system. It is typically used in a sequence: first, the interface is registered using ca_net_register_iface(), then ca_net_iface_link() is called to activate it, and finally, it is used in conjunction with ca_net_process() to handle incoming and outgoing traffic. If you need to shut down the network, you call this with 'up' set to false to safely release resources.

Esempio:

```c
/* Example: Bringing up the primary network interface */
#include "caos.h"

void setup_network(void) {
    u8 iface_id = 0;

    /* Check if the interface exists before attempting to link */
    if (iface_id < ca_net_iface_count()) {
        /* Attempt to bring the interface up */
        if (ca_net_iface_link(iface_id, true)) {
            ca_printf("Interface %d is now UP\n", iface_id);
        } else {
            ca_printf("Failed to start interface %d\n", iface_id);
        }
    }
}

void shutdown_network(void) {
    /* Safely bring the interface down */
    ca_net_iface_link(0, false);
    ca_printf("Interface 0 is now DOWN\n");
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_net_iface_link in the lifecycle of my network stack?
A: You should call it after the network stack is initialized via ca_net_init() and the interface has been registered using ca_net_register_iface(). It is used to activate the hardware for traffic processing or to shut it down to release resources.

Code example conf:1.00 **Q:** How do I bring up the first network interface in caOS?
A: u8 id = 0; if (id < ca_net_iface_count()) { bool success = ca_net_iface_link(id, true); }

Parameters conf:1.00 **Q:** What are the constraints on the 'id' parameter for ca_net_iface_link?
A: The 'id' must be a u8 value representing the index of the network interface. It must be strictly less than the total number of interfaces returned by ca_net_iface_count().

Return conf:1.00 **Q:** What does a false return value from ca_net_iface_link indicate?
A: A false return indicates that the operation failed. This occurs if the provided interface ID is invalid (out of bounds) or if the underlying start routine for the interface failed to execute.

Error/Edge case conf:1.00 **Q:** Can I call ca_net_iface_link from an interrupt service routine (ISR)?
A: No, it is not safe to call from an ISR. The function is not thread-safe and the underlying start/stop routines may involve blocking operations or complex resource management that are unsuitable for an ISR context.

ca_net_init

Header: caos/net/inc/net.h:89
Source: caos/net/src/net.c:49:62
GUARD: EN_NET

```
bool ca_net_init(ca_net_iface_t *iface)
```

Initializes the caOS networking stack, internal buffers, and registers the provided network interface.

Parametri:

Nome	Tipo	Descrizione
iface	ca_net_iface_t *	Pointer to the network interface configuration structure. Must not be NULL.

Ritorna:
Returns true if the network stack and interface were successfully initialized; returns false if the input pointer is NULL or if the interface registration fails.

Note:

- This function is non-blocking and is intended to be called once during system startup, typically within the main initialization sequence before any network tasks are started. It is not thread-safe and must not be called from an Interrupt Service Routine (ISR) because it performs heap-like memory pool initialization and semaphore creation. A common mistake is calling this function multiple times or before the underlying hardware (like GPIOs or PHY clocks) has been properly configured, which will lead to undefined behavior or silent failures in the network stack.

Uso:

- This function is the entry point for the caOS networking subsystem. It is typically called after hardware-specific initialization (like eth_gpio_init) but before starting high-level services like ARP or DHCP. It pairs with interface-specific getters (like ca_eth_iface_get) to pass the hardware configuration to the stack. Once initialized, the stack is ready to process packets via the internal RX queue and semaphore mechanism.

Esempio:

```
ca_net_iface_t *my_iface = ca_eth_iface_get(&eth_cfg);
if (my_iface != NULL) {
    if (!ca_net_init(my_iface)) {
        /* Handle initialization failure: log error or halt system */
        while(1);
    }
} else {
    /* Handle invalid interface configuration */
}
/* Proceed to initialize services like ARP after successful net init */
ca_arp_service_init(s_mac, s_ip);
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should ca_net_init() be called in a caOS application?
A: ca_net_init() should be called during system initialization, typically once at startup, before any network operations begin. It prepares the Ethernet module for packet handling by initializing buffers and semaphores.

Code example conf:0.90 **Q:** Can you provide a minimal example of how to use ca_net_init() with caOS types?

A: ca_eth_cfg_t cfg; cfg.rx_cb = my_rx_callback; cfg.tx_done_cb = my_tx_done_callback; cfg.error_cb = my_error_callback; cfg.rx_buf = (u8*)0x20000000; cfg.tx_buf = (u8*)0x20000100; cfg.buf_size = 1536; bool success = ca_net_init(&cfg); if (!success) { // Handle initialization error }

Parameters conf:0.95 **Q:** What is the significance of the cfg parameter in ca_net_init() and what happens if it is NULL?
A: The cfg parameter is a pointer to a ca_eth_cfg_t structure that holds configuration for the Ethernet interface. If cfg is NULL, ca_net_init() immediately returns false, indicating initialization failure due to invalid input.

Return conf:0.90 **Q:** What does ca_net_init() return when it fails, and what causes such failures?
A: ca_net_init() returns false when the cfg parameter is NULL or when ca_eth_init() fails during hardware initialization. These conditions cause the function to return false, signaling that network initialization was not successful.

Error/Edge case conf:0.85 **Q:** What are common mistakes when calling ca_net_init()?
A: Common mistakes include passing a NULL pointer for cfg, failing to properly initialize the rx_buf, tx_buf, and buf_size fields in the cfg structure, or calling ca_net_init() from an ISR, which is not allowed.

Behavior conf:0.90 **Q:** Is ca_net_init() thread-safe and can it be called from an ISR?
A: ca_net_init() is not thread-safe and must not be called from an ISR. It is a one-time initialization function meant to be called once from a main thread during system startup.

`ca_net_link_down`

Header: `caos/net/inc/net.h:134`
Source: `caos/net/src/net.c:275:285`
GUARD: `EN_NET`

```
1 void ca_net_link_down(void)
```

Disables the network interface by resetting the physical layer (PHY) and stopping the software stack.

Ritorna:
This function returns void and does not provide feedback on the success or failure of the hardware reset.

Note:

- This function is non-blocking and performs immediate hardware manipulation. It is not thread-safe; if called simultaneously from multiple tasks or an ISR, it could lead to race conditions on the PHY reset pin or the interface state. It is generally safe to call from an ISR, but be aware that it triggers GPIO writes and interface stop callbacks, which might take non-negligible time. A common mistake is calling this function without ensuring the network stack has been properly initialized, or assuming that calling it will immediately free up all network resources, which may not be true if other tasks are still holding references to the interface.

Uso:

- This function is used to manually force the network interface into a disconnected state, typically for maintenance, power saving, or error recovery. It is the counterpart to 'ca_net_link_up()' and is often triggered by user commands via a CLI or by internal network monitoring logic that detects a fault. In a typical lifecycle, you would call this during system shutdown or when reconfiguring network parameters to ensure the hardware is in a known 'down' state before re-initializing.

Esempio:

```
/* Example: Manually disabling the network link via CLI command */
void handle_network_command(char *cmd) {
    if (ca_strncmp(cmd, "down", 4) == 0) {
        /* Ensure the system is in a state where it is safe to drop the link */
        ca_net_link_down();
        /* Log the action to the user */
        CLI_PUT_N("Network interface has been disabled.");
    }
}
```

Q&A — fine-tuning review *2/2 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_net_link_down() and what is the intended use case for disabling the Ethernet link?
A: You should call ca_net_link_down() when you need to intentionally disable the Ethernet network link and power down the PHY Ethernet controller. This function is intended for scenarios where network communication must be stopped on demand, such as during system shutdown, network configuration changes, or when entering a low-power state. It is important to note that this function is blocking and not thread-safe, so it must be called from a stable system state and not from an ISR.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using caOS types that demonstrates how to call ca_net_link_down()?
A: void example_function(void) { ca_net_link_down(); }

`ca_net_link_status`

Header: `caos/net/inc/net.h:136`
Source: `caos/net/src/net.c:299:302`
GUARD: `EN_NET`

```
bool ca_net_link_status(void)
```

Returns the current operational status of the network link.

Ritorna:
Returns 'true' if the network link is enabled and active, or 'false' if the link is down or not yet initialized.

Note:

- This function is non-blocking and thread-safe as it simply reads a static boolean flag. It is safe to call from both standard tasks and Interrupt Service Routines (ISRs) because it performs a simple memory read without requiring mutexes or semaphores. A common mistake is assuming this function checks the physical hardware link status (like an Ethernet cable plug); instead, it reflects the software-level 'enabled' state managed by the network stack. Ensure the network stack has been initialized via the appropriate startup functions before relying on this status, otherwise, it will return 'false' by default.

Uso:

- This function is typically used for diagnostic reporting, such as displaying network status in a Command Line Interface (CLI) or logging system. It is often paired with network configuration functions that set the link state during system boot. Developers use it to guard network-dependent operations, ensuring that data transmission is only attempted when the link is confirmed to be 'up'. It is frequently called alongside functions that retrieve IP or MAC addresses to provide a complete snapshot of the network interface state.

Esempio:

```
/* Check if the network is ready before attempting to send data */
if (ca_net_link_status()) {
    /* Link is active, safe to proceed with network operations */
    CLI_PRINTF_N("Network is ready for communication.");
} else {
    /* Link is down, notify the user or attempt a recovery */
    CLI_PRINTF_N("Warning: Network link is currently down.");
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_net_link_status() and why is it important for network operations?
A: You should call ca_net_link_status() to check if the Ethernet link is active (up) or inactive (down) before attempting network operations. It is important because it allows you to verify that the network connection is established before sending or receiving data. The function is non-blocking and thread-safe, but it relies on internal state that must be updated by other functions like ca_net_link_enable() or ca_eth_restart(). If the link is not properly enabled or reset, the function may return an outdated or incorrect status.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_net_link_status() with caOS types?
A: bool link_up = ca_net_link_status(); if (link_up) { // Proceed with network operations } else { // Handle link down condition }

Return conf:0.90 **Q:** What do the return values of ca_net_link_status() mean and what causes the function to return an incorrect value?
A: The function returns true if the link is active (up) and false if the link is inactive (down). The return value reflects the internal state variable s_link_enabled, which must be updated by functions like ca_net_link_enable() or ca_eth_restart(). If these functions have not been called or if the network has not been properly reset, the function may return a stale or incorrect value. The function does not perform any physical polling of the link or implement timeouts, so it only reflects the last known state of the network link.

`ca_net_link_up`

Header: `caos/net/inc/net.h:135`
Source: `caos/net/src/net.c:287:297`
GUARD: `EN_NET`

```
void ca_net_link_up(void)
```

Initializes the physical network interface and brings the network link to an active state.

Ritorna:
This function returns void and does not provide a status code, meaning it assumes the hardware initialization will succeed.

Note:

- This function is non-blocking and performs hardware-level operations, specifically toggling a GPIO pin to release the PHY from reset and invoking the interface's start routine. It is not thread-safe by default, so ensure it is not called simultaneously from multiple tasks or an ISR, as this could lead to race conditions in the interface state machine. A common mistake is calling this function before the network stack or the underlying hardware drivers have been initialized, which will result in a null pointer dereference if the interface structure

is not yet allocated. Always ensure the system clock and GPIO peripherals are configured before invoking this function.

Uso:

- This function is typically used during the system startup sequence after the network stack has been initialized, or as a recovery mechanism when a network link is lost. It is often paired with 'ca_net_link_down' to manage the lifecycle of the network connection. In a production environment, it is frequently triggered by a CLI command or a network management task that monitors link status and decides when to bring the interface up or down based on user input or connectivity requirements.

Esempio:

```
/* Ensure the network stack is initialized before bringing the link up */
if (system_ready) {
    /* Bring the physical link up to start network communication */
    ca_net_link_up();

    /* Verify the link status or proceed with network tasks */
    // check_network_status();
} else {
    /* Handle error: system not ready */
}
```

Q&A — fine-tuning review *2/2 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_net_link_up() and what is its purpose in the network stack?
A: The ca_net_link_up() function is used to activate the network link by resetting the PHY and restarting the Ethernet interface. It should be called when the network connection needs to be re-established or initialized, such as after a hardware reset or during system startup. The function is non-blocking and can be invoked from an ISR, provided there are no concurrent operations that might cause race conditions. It is thread-safe if no other functions modify the same global variables.

Code example conf:0.85 **Q:** Can you provide a minimal C example using caOS types that demonstrates how to use ca_net_link_up()?
A: void example_net_init(void) { // Assuming s_phy_nrst_port and s_phy_nrst_pin are initialized ca_net_link_up(); }

`ca_net_process`

Header: `caos/net/inc/net.h:106`
Source: `caos/net/src/net.c:200:216`
GUARD: `EN_NET`

```
void ca_net_process(void)
```

Maintains the network stack by processing incoming packets from all registered network interfaces.

Ritorna:
This function returns void and does not provide a status code.

Note:

- This function is non-blocking but performs significant work, so it must be called from a dedicated network task, never from an ISR. It is not thread-safe if called from multiple tasks

simultaneously; ensure it is only invoked by the task responsible for network management. Before calling this, the network stack must be initialized and interfaces must be registered. A common beginner mistake is calling this function inside an ISR or in a high-priority task that starves other system processes, which can lead to dropped packets or system instability. Always pair this with a semaphore wait mechanism to ensure the CPU only processes data when frames are actually available.

Uso:

- This function is the heart of the network stack's 'bottom half' processing. It is typically used in a dedicated network task loop, where the task blocks on a semaphore (e.g., ca_sem_wait) that is released by an ISR when new data arrives. By calling ca_net_process() after the semaphore is released, the task drains the hardware DMA buffers and the software RX queue. It is the bridge between hardware-level interrupts and the application-level network logic, ensuring that packet processing happens in a controlled, task-based context rather than inside the time-sensitive ISR.

Esempio:

```c
/* Dedicated network task loop */
void network_task(void *pvParameters) {
    while (1) {
        /* Wait for the ISR to signal that data is ready */
        if (ca_sem_wait(&ca_net_rx_sem) == 0) {
            /* Process all pending frames from all interfaces */
            ca_net_process();
        }
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should ca_net_process() be called in a caOS application, and why is it important?
A: ca_net_process() should be called in a loop by a task that waits for a signal (e.g., a semaphore) indicating that network packets are available for processing. It is important because it handles received Ethernet packets, manages buffers, and forwards data to the appropriate network layers. The function is designed to be called after receiving a notification from an ISR via a semaphore, ensuring that network processing is synchronized with packet arrival.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using caOS types that demonstrates how to call ca_net_process()?
A:

```c
void network_task(void* arg) {
    while (1) {
        ca_sem_wait(&net_sem, CA_WAIT_FOREVER);
        ca_net_process();
    }
}
```

Error/Edge case conf:0.90 **Q:** What are the potential issues or edge cases when using ca_net_process()?
A: ca_net_process() is a blocking function that will not return unless there is a critical error or the network link is disabled. It is not thread-safe and must not be called from an ISR. Common mistakes include calling it from an ISR, not properly synchronizing with a semaphore before calling it, or assuming it returns under normal conditions. It is also not designed to handle errors gracefully, so any error condition will cause it to halt execution.

Behavior conf:0.95 **Q:** Is ca_net_process() blocking or non-blocking, and is it safe to call from an ISR or from multiple threads?

A: ca_net_process() is a blocking function and will not return under normal conditions unless there is a critical error or the link is disabled. It is not safe to call from an ISR, as it is not ISR-safe. Additionally, it is not thread-safe, meaning it should not be invoked concurrently from multiple tasks. It is designed to be called from a single task that is signaled by an ISR via a semaphore.

---

ca_net_proto_arp

Header: caos/net/inc/net_proto.h:232
Source: caos/net/src/net_proto.c:190:206
GUARD: EN_NET_PROTO

```
bool ca_net_proto_arp(const ca_l2_frame_t *l2, u16 len, const
    ca_arp_frame_t **out)
```

Validates and extracts an ARP packet structure from a raw Layer 2 Ethernet frame.

Parametri:

Nome	Tipo	Descrizione
l2	const ca_l2_frame_t *	Pointer to the incoming Ethernet frame. Must not be NULL.
len	u16	Total length of the received frame in bytes. Used to prevent buffer over-reads.
out	const ca_arp_frame_t **	Pointer to a pointer that will be updated to point to the ARP payload within the frame if validation succeeds.

Ritorna:
Returns true if the frame is a valid ARP packet (correct EtherType, hardware type, protocol type, and length). Returns false if the frame is not ARP, is malformed, or if input pointers are NULL.

Note:

- This function is non-blocking and thread-safe because it performs read-only validation on existing memory buffers. It is safe to call from an ISR (Interrupt Service Routine) because it does not perform any heap allocation, mutex locking, or complex I/O operations. The most common mistake is failing to check the return value before accessing the 'out' pointer; if the function returns false, the 'out' pointer remains unchanged or points to invalid data. Ensure the 'len' parameter accurately reflects the actual bytes received from the network interface to avoid processing garbage data.

Uso:

- This function is typically used in the network stack's input processing loop after a frame has been received from the Ethernet controller. It acts as a filter: you first receive a raw frame, check if it is an ARP packet using this function, and if it returns true, you proceed to inspect the ARP operation (request or reply) and the sender's IP/MAC addresses. It is usually paired with functions that handle the actual ARP logic, such as updating an ARP cache or generating an ARP reply. It is essential for separating ARP traffic from other protocols like IPv4 or IPv6.

Esempio:

```
const ca_l2_frame_t *frame = get_received_frame();
u16 frame_len = get_frame_length();
```

```c
const ca_arp_frame_t *arp_ptr = NULL;

if (ca_net_proto_arp(frame, frame_len, &arp_ptr)) {
    // Validation successful, safe to access arp_ptr
    if (ca_ntohs(arp_ptr->oper) == ARP_OP_REQUEST) {
        // Handle ARP request logic here
    }
} else {
    // Not an ARP packet or malformed, ignore or pass to other handlers
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_net_proto_arp in a network stack implementation?
A: You should call ca_net_proto_arp when you receive an Ethernet frame and want to determine if it is an ARP frame. This function is typically used in network protocol handling to validate and extract ARP frames from incoming L2 frames.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_net_proto_arp with caOS types?
A: ca_l2_frame_t l2_frame; ca_arp_frame_t *arp_frame; bool result = ca_net_proto_arp(&l2_fram 28, (const ca_arp_frame_t **)&arp_frame); if (result) { // Successfully extracted ARP frame }

Parameters conf:0.90 **Q:** What are the valid and invalid values for the 'l2' parameter in ca_net_proto_arp?
A: The 'l2' parameter must be a non-NULL pointer to a valid Ethernet frame. If it is NULL, the function will return false. The frame must contain a valid Ethernet header with an ARP ethertype.

Return conf:0.95 **Q:** What does it mean when ca_net_proto_arp returns false?
A: ca_net_proto_arp returns false when the frame is not a valid ARP frame. This can happen due to: l2 or out being NULL, ethertype not being ARP, insufficient frame length, or non-conforming ARP fields.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_net_proto_arp?
A: Common mistakes include passing NULL for l2 or out parameters, not checking if the returned pointer is valid, passing a frame that is too short, or calling the function with a frame that is not actually an ARP frame. These can lead to crashes or incorrect behavior.

Behavior conf:0.90 **Q:** Is ca_net_proto_arp blocking or ISR-safe?
A: ca_net_proto_arp is non-blocking and ISR-safe. It performs only quick checks on input parameters and does not perform any operations that could block or cause issues in interrupt contexts.

`ca_net_proto_arp_build_garp`

Header: `caos/net/inc/net_proto.h:302`
Source: `caos/net/src/net_proto.c:327:353`
GUARD: `EN_NET_PROTO`

```c
u16 ca_net_proto_arp_build_garp(ca_mac_t local_mac, ca_ipv4_t local_ip,
    u8 *out_frame)
```

Constructs a Gratuitous ARP (GARP) packet to announce the device's IP address on the network.

Parametri:

Nome	Tipo	Descrizione
`local_mac`	`ca_mac_t`	The MAC address of the local interface. Must be a valid 6-byte hardware address.
`local_ip`	`ca_ipv4_t`	The IPv4 address to announce. Must be the IP currently assigned to the interface.
`out_frame`	`u8 *`	Pointer to the memory buffer where the frame will be built. Must be at least 60 bytes long to accommodate the Ethernet minimum frame size.

Ritorna:
Returns 60 (the size of the generated frame in bytes) on success, or 0 if the out_frame pointer is NULL.

Note:

- This function is non-blocking and thread-safe as it performs pure memory manipulation without accessing shared global state or hardware registers directly. It is safe to call from an ISR, provided the provided buffer is valid and large enough. The most common beginner mistake is providing a buffer smaller than 60 bytes, which will cause a buffer overflow when the function writes the Ethernet padding. Ensure the buffer is allocated from a reliable source, such as a network buffer pool, before calling this function.

Uso:

- This function is used during network initialization or when an IP address changes to inform other devices on the local network of the device's presence and MAC-to-IP mapping. It is typically used in a sequence: first, obtain a network buffer using a buffer management function, then call this function to populate the buffer, and finally pass the buffer to the network transmission function (e.g., ca_net_send). It is essential for preventing IP conflicts and ensuring that switches update their ARP tables correctly when the device boots up.

Esempio:

```
/* Example: Sending a GARP packet */
u8 buffer[60];
mac_t my_mac = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55};
ipv4_t my_ip = {192, 168, 1, 10};

/* Build the GARP frame into the buffer */
u16 len = ca_net_proto_arp_build_garp(my_mac, my_ip, buffer);

if (len > 0) {
    /* If successful, send the frame via the network stack */
    /* ca_net_send(buffer, len); // Hypothetical transmission call */
} else {
    /* Handle error: buffer was likely NULL */
}
```

Q&A — fine-tuning review

6/6 pairs · qwen3-coder:latest

Usage conf:0.95 **Q:** When should I use ca_net_proto_arp_build_garp in a caOS-based network application?
A: You should call ca_net_proto_arp_build_garp when you need to broadcast a gratuitous ARP (GARP) frame to announce your local IP address on the network. This is typically done during network initialization or when an IP address changes, to ensure other devices update their ARP tables.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_net_proto_arp_build_garp in a caOS application?

A: u8 frame[60]; mac_t my_mac = {0x00, 0x08, 0x9B, 0x00, 0x00, 0x01}; ipv4_t my_ip = {192, 168, 1, 100}; u16 result = ca_net_proto_arp_build_garp(my_mac, my_ip, frame); if (result == 60) { // Frame built successfully, send it via Ethernet driver }

Parameters conf:0.85 **Q:** What are the valid values for local_mac and local_ip in ca_net_proto_arp_build_ga

A: local_mac must be a valid 6-byte MAC address array (ca_mac_t type). local_ip must be a valid IPv4 address represented as a 4-byte array (ca_ipv4_t type). Invalid values such as uninitialized or malformed addresses will result in a corrupted frame being built.

Return conf:0.95 **Q:** What does it mean when ca_net_proto_arp_build_garp returns 0?

A: A return value of 0 indicates an error, specifically that the out_frame parameter was NULL. The function does not build a frame and returns immediately without modifying the buffer.

Error/Edge case conf:0.90 **Q:** What are the most common mistakes when calling ca_net_proto_arp_build_ga

A: Common mistakes include passing a NULL pointer for out_frame, which causes the function to return 0; using a buffer smaller than 60 bytes, which may lead to memory corruption; and not ensuring that local_mac and local_ip are valid before calling the function, resulting in incorrect ARP frames.

Behavior conf:0.85 **Q:** Is ca_net_proto_arp_build_garp safe to call from an ISR or in a multi-threaded environment?

A: Yes, the function is thread-safe if the parameters are not modified by other tasks during execution. It is also ISR-safe, but only if the out_frame buffer is guaranteed to be accessible and unmodified by other ISRs or tasks during the call.

`ca_net_proto_arp_build_reply`

Header: `caos/net/inc/net_proto.h:289`
Source: `caos/net/src/net_proto.c:294:322`
GUARD: `EN_NET_PROTO`

```
u16 ca_net_proto_arp_build_reply(const ca_arp_frame_t *req, ca_mac_t
    local_mac, ca_ipv4_t local_ip, u8 *out_frame)
```

Constructs an ARP reply frame in response to a received ARP request.

Parametri:

Nome	Tipo	Descrizione
`req`	`const ca_arp_frame_t *`	Pointer to the incoming ARP request frame. Must not be NULL.
`local_mac`	`ca_mac_t`	The MAC address of the local device to be inserted into the reply.
`local_ip`	`ca_ipv4_t`	The IPv4 address of the local device to be inserted into the reply.
`out_frame`	`u8 *`	Pointer to the buffer where the reply frame will be built. Must be large enough (at least 42 bytes) and not NULL.

Ritorna:
Returns the total length of the constructed frame in bytes (typically 42). Returns 0 if input pointers are NULL or if the incoming frame is not an ARP request.

Note:

- This function is non-blocking and thread-safe as it performs pure memory manipulation without accessing shared hardware resources or OS primitives. It is safe to call from an ISR, provided the output buffer is valid and not currently being accessed by other tasks. A common beginner mistake is providing an 'out_frame' buffer that is too small, leading to memory corruption (buffer overflow). Always ensure the buffer is at least 42 bytes long to accommodate the Ethernet and ARP headers.

Uso:

- This function is used in the network stack's receive path when an incoming ARP request is identified as targeting the local device. It is typically paired with a packet reception function that parses the incoming frame and a transmission function (like ca_net_send) that pushes the resulting buffer onto the wire. The sequence is: receive raw packet -> validate as ARP request -> call this function to build the reply -> transmit the reply buffer.

Esempio:

```
u8 reply_buffer[42];
ca_arp_frame_t *incoming_arp = (ca_arp_frame_t *)received_payload_ptr;

// Build the ARP reply using local device info
u16 len = ca_net_proto_arp_build_reply(incoming_arp, MY_MAC, MY_IP, reply_buffer
    );

if (len > 0) {
    // Send the frame via the network interface
    // ca_net_send(reply_buffer, len);
} else {
    // Handle error: invalid request or null pointer
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_net_proto_arp_build_reply in a caOS network stack implementation?
A: You should call ca_net_proto_arp_build_reply when you receive an ARP request frame and need to construct a corresponding ARP reply. This function is typically invoked in the network protocol layer when processing incoming ARP requests to respond to IP address resolution requests from other devices on the network.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_net_proto_arp_build_reply with caOS types?
A: u8 frame_buffer[42]; ca_arp_frame_t request; mac_t local_mac = {0x00, 0x08, 0x9B, 0x00, 0x00, 0x01}; ipv4_t local_ip = 0x0A000001; request.oper = ARP_OP_REQUEST; // Assume request is populated with data from received frame u16 result = ca_net_proto_arp_build_re local_mac, local_ip, frame_buffer); if (result == 42) { // Successfully built ARP reply, send frame_buffer via Ethernet }

Parameters conf:0.90 **Q:** What are the valid values for the req parameter in ca_net_proto_arp_build_reply, and what happens if it's invalid?
A: The req parameter must be a valid pointer to a ca_arp_frame_t structure that represents an ARP request (oper == ARP_OP_REQUEST). If req is NULL or points to invalid data, the function returns 0. The function specifically checks that the ARP operation code is a request, and if not, it also returns 0.

Return conf:0.95 **Q:** What does the return value 42 from ca_net_proto_arp_build_reply indicate, and what causes a return of 0?
A: A return value of 42 indicates that the ARP reply was successfully built and written to the output buffer, with a total frame size of 42 bytes (28 bytes for ARP header +

14 bytes for Ethernet header). A return value of 0 indicates failure, which occurs when req is NULL, the frame buffer is NULL, or the ARP operation is not a request (oper != ARP_OP_REQUEST).

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_net_proto_arp_build_reply, and what edge cases should I be aware of?
A: Common mistakes include passing a buffer smaller than 42 bytes for out_frame, which leads to memory corruption. Another is not checking that the req parameter points to a valid ARP request frame. Additionally, calling the function with a req that has oper != ARP_OP_REQUEST will cause it to return 0 without building a reply. The function does not validate the contents of the request beyond checking the operation code.

Behavior conf:0.90 **Q:** Is ca_net_proto_arp_build_reply blocking or thread-safe, and can it be called from an ISR?
A: The function is non-blocking and thread-safe if the parameters are not modified by other threads during execution. It can be safely called from an ISR, provided that the out_frame buffer is allocated in a way that is safe for ISR use, such as being static or pre-allocated in a safe memory region.

`ca_net_proto_bind_tcp`

Header: `caos/net/inc/net_proto.h:208`
Source: `caos/net/src/net_proto.c:81:84`
GUARD: `EN_NET_PROTO`

```
1 void ca_net_proto_bind_tcp(ca_tcp_rx_raw_t fn)
```

Registers a global callback function to process incoming raw TCP data packets.

Parametri:

Nome	Tipo	Descrizione
`fn`	`ca_tcp_rx_raw_t`	A function pointer to your custom TCP data handler. Must match the signature expected by the network stack. Passing NULL effectively disables TCP packet processing.

Ritorna:
void

Note:

- This function is non-blocking and thread-safe in the sense that it performs a simple pointer assignment. It is not intended to be called from an ISR, as it is typically part of the system initialization sequence. The most common mistake is failing to register a handler before the network stack starts receiving packets, which will result in incoming TCP data being silently dropped or causing a null pointer dereference if the stack attempts to call an uninitialized handler. Ensure the function passed as 'fn' is efficient, as it will likely be executed within the context of the network processing thread; performing heavy computations here will block the entire network stack.

Uso:

- This function is used during the system boot-up phase to link the network stack's TCP layer to your application-specific logic. It is typically called inside a service initialization function, such as 'ca_tcp_service_start', immediately after the protocol control blocks are initialized. It is paired with the definition of a callback function that handles raw data buffers. Once bound, the network stack will automatically invoke your registered function

whenever a valid TCP segment arrives, allowing your application to parse the payload and respond accordingly.

Esempio:

```
/* Define the handler that will process incoming TCP data */
void tcp_rx_handler(void *data, uint16_t len) {
    /* Process the raw TCP payload here */
}

void app_init(void) {
    /* Register the handler before starting the network services */
    ca_net_proto_bind_tcp(tcp_rx_handler);

    /* Now start the TCP service to begin receiving data */
    ca_tcp_service_start();
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_net_proto_bind_tcp and why is it useful for TCP packet handling?
A: You should call ca_net_proto_bind_tcp to set a global callback function that will be invoked whenever a TCP packet is received. This is useful for handling incoming TCP data in a centralized way, allowing your application to process TCP packets without needing to poll or check for them manually. It's a global handler, so it applies to all TCP connections managed by caOS.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_net_proto_bind_tcp with caOS types?
A: ca_tcp_rx_raw_t my_tcp_handler = [](u8 *data, u32 len) -> i32 { // Process received TCP data return 0; };
void init_network() { ca_net_proto_bind_tcp(my_tcp_handler); }

Parameters conf:0.95 **Q:** What does the fn parameter in ca_net_proto_bind_tcp do, and what are valid and invalid values for it?
A: The fn parameter is a pointer to a callback function that will be invoked when a TCP packet is received. It must be a function compatible with the ca_tcp_rx_raw_t type, which accepts a pointer to data and a length, and returns an integer. A valid value is a valid function pointer. An invalid value is NULL, which disables TCP packet reception but does not cause an error.

`ca_net_proto_bind_udp`

Header: `caos/net/inc/net_proto.h:204`
Source: `caos/net/src/net_proto.c:89:120`
GUARD: `EN_NET_PROTO`

```
bool ca_net_proto_bind_udp(u16 port, ca_udp_rx_t fn)
```

Registers a callback function to handle incoming UDP packets for a specific port.

Parametri:

Nome	Tipo	Descrizione
`port`	`u16`	The UDP port number to listen on. Must be a valid 16-bit integer.
`fn`	`ca_udp_rx_t`	The function pointer to be called when a packet arrives. Pass NULL to unbind/remove the existing handler for this port.

Ritorna:
Returns true if the binding was successfully created, updated, or removed. Returns false if the system has reached the maximum number of allowed UDP bindings (CA_UDP_MAX_PORTS).

Note:

- This function is non-blocking and executes quickly by modifying an internal configuration table. It is not inherently thread-safe; if multiple tasks attempt to bind ports simultaneously, you must ensure external synchronization. It is generally safe to call from an ISR, but you should avoid doing so unless necessary to keep interrupt latency low. A common beginner mistake is forgetting that the handler function (fn) must be defined globally or as a static function to remain in scope, and failing to check the return value, which leads to silent failures when the binding table is full.

Uso:

- This function is typically used during the initialization phase of a network service (like DNS, NTP, or mDNS) to register its specific UDP port. It is paired with a service-specific 'net_ready' function that signals when the network stack is prepared to process traffic. When a service is shut down or reconfigured, this function is called with fn set to NULL to free up the slot in the binding table for other services to use.

Esempio:

```c
/* Define the callback function that will process incoming UDP data */
static void my_udp_handler(u16 port, u8 *data, u16 len) {
    /* Process incoming packet data here */
}

/* Initialize the service and bind the port */
void start_my_service(void) {
    if (ca_net_proto_bind_udp(1234, my_udp_handler)) {
        /* Successfully bound, service is ready to receive */
    } else {
        /* Handle error: binding table full */
    }
}

/* Cleanup when service is no longer needed */
void stop_my_service(void) {
    ca_net_proto_bind_udp(1234, NULL); /* Unbind the port */
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_net_proto_bind_udp and why is it important for UDP communication?
A: You should call ca_net_proto_bind_udp to associate a specific UDP port with a receive handler function. This allows your application to listen for incoming UDP packets on that port. It's essential for setting up UDP services like DNS, NTP, or mDNS where you need to receive data on a known port.

Code example conf:0.90 **Q:** Can you show a minimal example of how to bind a UDP port using ca_net_proto_bind_udp?
A: bool udp_handler(u8 *data, u16 len, u32 src_ip, u16 src_port) { // Handle received UDP data return true; }
void setup_udp_listener(void) { ca_net_proto_bind_udp(53, udp_handler); // Bind port 53 to the handler }

Parameters conf:0.95 **Q:** What happens if I pass 0 as the port number or NULL as the function pointer to ca_net_proto_bind_udp?

A: If port is 0, the function does nothing and returns true. If fn is NULL, it removes any existing binding for that port, effectively unbinding it.

Return conf:0.90 **Q:** What does it mean when ca_net_proto_bind_udp returns false?
A: A return value of false indicates that the operation failed, typically because the maximum number of UDP ports (CA_UDP_MAX_PORTS) has been exceeded, and no new port could be allocated.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_net_proto_bind_udp?
A: Common mistakes include calling the function with port=0 or fn=NULL without checking the return value, attempting to bind the same port twice without first unbinding it, and calling the function from an ISR which is not safe.

Behavior conf:0.90 **Q:** Is ca_net_proto_bind_udp thread-safe and can it be called from an ISR?
A: No, ca_net_proto_bind_udp is not thread-safe and should not be called from an ISR. It is not blocking, but may take time to find a free slot or update existing configurations. It requires synchronization if used from multiple threads.

`ca_net_proto_csum_fill`

Header: `caos/net/inc/net_proto.h:286`
Source: `caos/net/src/net_proto.c:1593:1633`
GUARD: `EN_NET_PROTO`

```
1 void ca_net_proto_csum_fill(u8 *frame, u16 len)
```

Calculates and inserts IPv4 and L4 (ICMP/TCP) checksums into a network frame buffer.

Parametri:

Nome	Tipo	Descrizione
`frame`	`u8 *`	Pointer to the start of the Ethernet frame buffer. Must be valid and point to at least 14 bytes for the Ethernet header.
`len`	`u16`	Total length of the frame in bytes. Must be large enough to contain the IP header (at least 20 bytes).

Ritorna:
void. The function modifies the buffer in-place if the frame is a valid IPv4 packet.

Note:

- This function is non-blocking and performs pure mathematical calculations on the provided memory buffer. It is thread-safe provided that no other task is modifying the same buffer simultaneously. It is safe to call from an ISR, as it does not use OS primitives like mutexes or semaphores. The most common mistake is passing a buffer pointer that does not include the Ethernet header (L2), as the function expects to parse the Ethernet type field at offset 12. If the frame is fragmented or not an IPv4 packet, the function returns silently without modifying the data.

Uso:

- This function is used in the network transmission path when the hardware network interface does not support automatic checksum offloading. It is typically called immediately before passing the buffer to the transmission function (e.g., ca_net_send). It is part of the manual packet construction sequence: allocate a buffer with ca_net_buf_alloc, fill the payload, call ca_net_proto_csum_fill to ensure integrity, and finally transmit via the interface.

Esempio:

```
/* Allocate a network buffer for a packet */
u16 buf_idx = ca_net_buf_alloc(128);
if (buf_idx != 0xFFFF) {
    ca_net_buf_t *buf = ca_net_buf_ptr(buf_idx);
    /* ... fill Ethernet, IP, and TCP headers in buf->data ... */

    /* Calculate and fill checksums before sending */
    ca_net_proto_csum_fill(buf->data, buf->data_len);

    /* Send the frame through the network stack */
    if (!ca_net_send(buf_idx)) {
        ca_net_buf_free(buf_idx);
    }
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_net_proto_csum_fill in my network transmission workflow?
A: You should call it immediately before passing the buffer to the transmission function, such as ca_net_send. It is used in the manual packet construction sequence after allocating a buffer with ca_net_buf_alloc and filling the payload, specifically when the hardware network interface lacks automatic checksum offloading.

Code example conf:0.90 **Q:** Provide a minimal example of how to use ca_net_proto_csum_fill.
A: u8 *buf = ca_net_buf_alloc(128); /* Fill payload here */ ca_net_proto_csum_fill(buf, 128); ca_net_send(buf, 128);

Parameters conf:1.00 **Q:** What are the requirements for the frame and len parameters?
A: The frame pointer must point to the start of the Ethernet frame buffer and must be valid for at least 14 bytes for the Ethernet header. The len parameter represents the total frame length and must be at least 20 bytes to accommodate the IP header.

Error/Edge case conf:1.00 **Q:** What happens if I pass a buffer that starts at the IP header instead of the Ethernet header?
A: This is a common mistake. The function expects to parse the Ethernet type field at offset 12; if the buffer does not include the Ethernet header (L2), the function will not correctly identify the packet structure and will likely fail to calculate the checksums.

Behavior conf:1.00 **Q:** Is it safe to call ca_net_proto_csum_fill from an interrupt service routine (ISR)?
A: Yes, it is safe to call from an ISR because the function is non-blocking and does not use OS primitives like mutexes or semaphores. It performs pure mathematical calculations on the buffer.

`ca_net_proto_dhcp_build_discover`

Header: `caos/net/inc/net_proto.h:336`
Source: `caos/net/src/net_proto.c:660:716`
GUARD: `EN_NET_PROTO`

```
u16 ca_net_proto_dhcp_build_discover(ca_mac_t local_mac, u32 xid, u8 *
    out_frame, u16 out_max)
```

Constructs a complete DHCP DISCOVER packet in a provided buffer for network initialization.

Parametri:

Nome	Tipo	Descrizione
`local_mac`	`ca_mac_t`	The MAC address of the device, used to populate the Ethernet header and DHCP client hardware address field.
`xid`	`u32`	A unique 32-bit transaction ID generated by the client to track the DHCP request-response cycle.
`out_frame`	`u8 *`	Pointer to the memory buffer where the packet will be built. Must be large enough to hold the full DHCP frame.
`out_max`	`u16`	The size of the provided buffer. If this is less than the required DHCP frame size, the function will fail.

Ritorna:
Returns the total length of the constructed packet in bytes (DHCP_FRAME_SIZE) on success, or 0 if the buffer is NULL or too small.

Note:

- This function is non-blocking and purely computational; it does not perform any I/O operations, making it safe to call from any context, including ISRs, provided the buffer is valid. It is thread-safe as it only operates on the memory provided by the caller. The most common mistake is providing a buffer that is too small or failing to check the return value, which leads to attempting to send an empty or incomplete packet. Ensure the buffer is zeroed or large enough to accommodate the full Ethernet, IP, UDP, and DHCP headers as defined by the system constants.

Uso:

- This function is typically used during the network stack initialization phase when the device needs to acquire an IP address from a DHCP server. It is usually paired with a buffer management system: first, you request a buffer using ca_net_buf_ptr, then you pass the data pointer to this function to populate it, and finally, you pass the buffer index to ca_net_send to transmit the packet over the wire. If the function returns 0, you must free the buffer using ca_net_buf_free to prevent memory leaks.

Esempio:

```
/* Example: Building and sending a DHCP Discover packet */
#define MY_XID 0x12345678U

void send_dhcp_discover(void) {
    u8 buf_idx = ca_net_buf_alloc(); // Allocate a network buffer
    if (buf_idx == 0xFF) return;     // Handle allocation failure

    ca_net_buf_t *b = ca_net_buf_ptr(buf_idx);
    u8 *out = b->data + NET_BUF_HEADROOM;

    // Build the packet into the buffer
    u16 len = ca_net_proto_dhcp_build_discover(s_local_mac, MY_XID, out,
    NET_BUF_DATA_SIZE);

    if (len > 0) {
        b->data_off = NET_BUF_HEADROOM;
        b->data_len = len;
        ca_net_send(buf_idx); // Send the packet
    } else {
        ca_net_buf_free(buf_idx); // Clean up if build failed
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_net_proto_dhcp_build_discover in a caOS network application?
A: You should call ca_net_proto_dhcp_build_discover when you need to construct a DHCP DISCOVER packet to initiate the DHCP handshake process. This function is typically used during network initialization when the device needs to obtain an IP address from a DHCP server.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_net_proto_dhcp_build_discover?
A: u8 frame[286]; mac_t my_mac = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55}; u32 transaction_id = 0x12345678; u16 result = ca_net_proto_dhcp_build_discover(my_mac, transaction_id, frame, 286); if (result == 286) { // Packet built successfully, send it over the network }

Parameters conf:0.90 **Q:** What is the significance of the xid parameter in ca_net_proto_dhcp_build_discover, and what happens if it's invalid?
A: The xid parameter is the unique transaction identifier for the DHCP exchange. It must be a valid u32 value; otherwise, the DHCP server may ignore the packet. This identifier is used by the server to match responses to the correct client request.

Return conf:0.95 **Q:** What does it mean when ca_net_proto_dhcp_build_discover returns 0?
A: A return value of 0 indicates an error occurred during packet construction. This typically happens when the output buffer is invalid (NULL) or when its size is less than DHCP_FRAME_SIZE (286 bytes).

Error/Edge case conf:0.85 **Q:** What are common mistakes developers make when using ca_net_proto_dhcp_b
A: Common mistakes include not verifying that the 'out_max' parameter is at least 286 bytes, passing a NULL pointer for 'out_frame', or failing to ensure that the buffer is large enough to prevent memory corruption. Another frequent error is calling this function from an ISR, which is not supported.

Behavior conf:0.90 **Q:** Is ca_net_proto_dhcp_build_discover thread-safe and ISR-safe?
A: No, ca_net_proto_dhcp_build_discover is not thread-safe and not ISR-safe. It should not be called from interrupt service routines because it uses functions like ca_memset that may not be safe in interrupt contexts. Synchronization is required if used in multi-threaded environments.

`ca_net_proto_dhcp_build_request`

Header: `caos/net/inc/net_proto.h:397`
Source: `caos/net/src/net_proto.c:792:873`
GUARD: `EN_NET_PROTO`

```
u16 ca_net_proto_dhcp_build_request(ca_mac_t dst_mac, ca_ipv4_t dst_ip,
   ca_mac_t local_mac, ca_ipv4_t ciaddr, ca_ipv4_t offered_ip, ca_ipv4_t
    server_id, u32 xid, u8 *out_frame, u16 out_max)
```

Constructs a DHCP Request packet (SELECTING or RENEWING) into a provided buffer.

Parametri:

Nome	Tipo	Descrizione
dst_mac	ca_mac_t	Destination MAC address (usually broadcast).
dst_ip	ca_ipv4_t	Destination IP address (usually 255.255.255.255).
local_mac	ca_mac_t	The client's own MAC address.
ciaddr	ca_ipv4_t	Client IP address. Set to 0.0.0.0 for SELECTING state, or the current assigned IP for RENEWING.
offered_ip	ca_ipv4_t	The IP address offered by the server (used in SELECTING state).
server_id	ca_ipv4_t	The IP address of the DHCP server (used in SELECTING state).
xid	u32	Transaction ID to match requests with server responses.
out_frame	u8 *	Pointer to the memory buffer where the frame will be built.
out_max	u16	Size of the provided buffer in bytes. Must be large enough for the frame.

Ritorna:
Returns the total size of the constructed frame in bytes on success. Returns 0 if the buffer is NULL, too small, or if an internal error occurs.

Note:

- This function is non-blocking and performs no I/O; it purely formats data into a memory buffer. It is thread-safe as long as the provided buffer is not accessed by other tasks simultaneously. It is safe to call from an ISR, provided the buffer is pre-allocated and large enough. The most common mistake is providing a buffer smaller than the required frame size, which will result in a return value of 0. Ensure the buffer is at least 260-270 bytes to accommodate the DHCP payload and headers.

Uso:

- This function is used during the DHCP state machine process to generate the 'Request' packet after receiving an 'Offer' from a server. It is typically paired with ca_net_buf_alloc to obtain a buffer, then this function is called to fill it, and finally ca_net_send is used to transmit the frame over the network interface. After transmission, the buffer should be released using ca_net_buf_free. It is essential to correctly distinguish between the SELECTING state (where you must include the server ID and offered IP) and the RENEWING state (where you only provide the current IP).

Esempio:

```
/* Allocate a network buffer for the DHCP request */
u8 *buf = ca_net_buf_alloc(512);
if (buf != NULL) {
    /* Build the DHCP Request packet */
    u16 frame_len = ca_net_proto_dhcp_build_request(bcast_mac, bcast_ip,
    local_mac, zero_ip, offered_ip, server_id, xid, buf, 512);

    if (frame_len > 0) {
        /* Send the frame via the network interface */
        ca_net_send(buf, frame_len);
    }

    /* Free the buffer after use */
    ca_net_buf_free(buf);
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_net_proto_dhcp_build_request in my embedded network application?
A: You should call ca_net_proto_dhcp_build_request when you need to construct a DHCP Request packet, either during the initial IP address selection phase or when renewing an existing IP lease. It is typically used after receiving a DHCP Offer message from a server.

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_net_proto_dhcp_build_request with caOS types?
A: u8 frame_buffer[512]; u16 result = ca_net_proto_dhcp_build_request((ca_mac_t){0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF}, // dst_mac (ca_ipv4_t){255, 255, 255, 255}, // dst_ip (ca_mac_t){0x00, 0x11, 0x22, 0x33, 0x44, 0x55}, // local_mac (ca_ipv4_t){0, 0, 0, 0}, // ciaddr (ca_ipv4_t){192, 168, 1, 100}, // offered_ip (ca_ipv4_t){192, 168, 1, 1}, // server_id 0x12345678, // xid frame_buffer, // out_frame 512 // out_max);

Parameters conf:0.90 **Q:** What does the ciaddr parameter represent in ca_net_proto_dhcp_build_request and what are valid values?
A: The ciaddr parameter represents the current IP address of the client. During SELECTING state, it should be set to 0.0.0.0. During RENEWING or REBINDING, it should contain the actual IP address being renewed.

Return conf:0.95 **Q:** What does a return value of 0 from ca_net_proto_dhcp_build_request indicate?
A: A return value of 0 indicates an error, typically caused by an invalid or insufficiently sized buffer (out_frame), or other parameter validation failure.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_net_proto_dhcp_build_request?
A: Common errors include passing a buffer that is too small for the DHCP packet, not setting ciaddr correctly (e.g., using a non-zero value during SELECTING), or reusing an xid that was already used in a previous transaction.

Behavior conf:0.90 **Q:** Is ca_net_proto_dhcp_build_request thread-safe and can it be called from an ISR?
A: Yes, the function is thread-safe if the parameters are not modified by other threads during execution. However, it is not ISR-safe and must not be called from interrupt service routines.

`ca_net_proto_dhcp_parse_offer`

Header: `caos/net/inc/net_proto.h:372`
Source: `caos/net/src/net_proto.c:718:784`
GUARD: `EN_NET_PROTO`

```
bool ca_net_proto_dhcp_parse_offer(const ca_udp_frame_t *udp,
   ca_dhcp_offer_t *out)
```

Parses a raw UDP DHCP offer frame into a structured ca_dhcp_offer_t object.

Parametri:

Nome	Tipo	Descrizione
`udp`	`const ca_udp_frame_t *`	Pointer to the received UDP frame. Must not be NULL and must contain a valid DHCP payload.
`out`	`ca_dhcp_offer_t *`	Pointer to the destination structure where parsed DHCP fields will be stored. Must not be NULL.

Ritorna:
Returns true if the frame is a valid DHCP BOOTREPLY and was successfully parsed; returns false if the frame is malformed, too short, or not a DHCP reply.

Note:

- This function is non-blocking and performs no heap allocation, making it safe for use in time-critical network tasks. It is thread-safe as it only operates on the provided input and output buffers without accessing global state. It is technically callable from an ISR, but ensure the stack size is sufficient for the ca_dhcp_offer_t structure. A common mistake is passing a pointer to an uninitialized or stack-garbage structure as 'out', which may lead to unpredictable values in fields not explicitly set by the parser.

Uso:

- This function is typically used within a network callback or a DHCP state machine when a UDP packet is received on the DHCP client port (68). It acts as the primary decoder after the network stack has verified the packet is a valid UDP frame. It is usually paired with a subsequent logic check on the 'msg_type' field to determine if the packet is an OFFER or an ACK, allowing the application to transition the DHCP state machine accordingly.

Esempio:

```
ca_udp_frame_t *rx_frame = get_next_udp_packet();
ca_dhcp_offer_t dhcp_data;

/* Attempt to parse the incoming DHCP offer */
if (ca_net_proto_dhcp_parse_offer(rx_frame, &dhcp_data)) {
    /* Successfully parsed, check if it is a DHCPOFFER (type 2) */
    if (dhcp_data.msg_type == 2U) {
        /* Proceed to send DHCPREQUEST using dhcp_data.offered_ip */
    }
} else {
    /* Handle invalid packet or non-DHCP traffic */
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_net_proto_dhcp_parse_offer in a network application?
A: You should call ca_net_proto_dhcp_parse_offer when you receive a UDP packet containing a DHCP OFFER message from a DHCP server. This function is typically used after receiving a DHCP OFFER in a DHCP handshake process, to extract network configuration parameters such as IP address, gateway, and DNS servers.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_net_proto_dhcp_parse_offer with caOS types?
A: ca_udp_frame_t udp_frame; ca_dhcp_offer_t offer; bool success = ca_net_proto_dhcp_parse_of &offer); if (success) { // Use offer fields like offer.ip_address }

Parameters conf:0.95 **Q:** What are the valid values for the udp and out parameters in ca_net_proto_dhcp_pars
A: The udp parameter must be a valid pointer to a ca_udp_frame_t structure containing a properly formatted DHCP OFFER packet (with op field set to 2). The out parameter must be a valid pointer to a pre-allocated ca_dhcp_offer_t structure where the parsed data will be stored. Both pointers must not be NULL, or the function returns false.

Return conf:0.95 **Q:** What does it mean when ca_net_proto_dhcp_parse_offer returns false?
A: A return value of false indicates that the function failed to parse the DHCP OFFER packet. This can happen if the udp or out parameters are NULL, the packet is malformed, or the packet does not contain a valid DHCP OFFER message (e.g., the 'op' field is not set to 2).

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_net_proto_dhcp_parse_offer?
A: Common mistakes include passing NULL pointers for udp or out, not checking the return value of the function, and calling the function with a UDP packet that is not a valid DHCP OFFER message. Also, calling the function from an ISR without ensuring the packet data is not modified during parsing can lead to undefined behavior.

Behavior conf:0.95 **Q:** Is ca_net_proto_dhcp_parse_offer blocking or thread-safe, and can it be called from an ISR?
A: The function is non-blocking and thread-safe. It can also be safely called from an ISR, provided that the UDP packet data is not modified during the parsing process. The function does not modify any shared data and uses only local parameters.

`ca_net_proto_dns_build_query`

Header: `caos/net/inc/net_proto.h:510`
Source: `caos/net/src/net_proto.c:1115:1174`
GUARD: `EN_NET_PROTO`

```
u16 ca_net_proto_dns_build_query(ca_mac_t server_mac, ca_ipv4_t
    server_ip, ca_mac_t local_mac, ca_ipv4_t local_ip, u16 id, const char
     *hostname, u8 *out_frame, u16 out_max)
```

Constructs a complete DNS query packet (Ethernet/IP/UDP/DNS) into a provided buffer.

Parametri:

Nome	Tipo	Descrizione
`server_mac`	`ca_mac_t`	Destination MAC address of the DNS server.
`server_ip`	`ca_ipv4_t`	Destination IPv4 address of the DNS server.
`local_mac`	`ca_mac_t`	Source MAC address of the local device.
`local_ip`	`ca_ipv4_t`	Source IPv4 address of the local device.
`id`	`u16`	Transaction ID for matching the response to this request.
`hostname`	`const char *`	The domain name string to resolve (e.g., 'google.com'). Must be null-terminated.
`out_frame`	`u8 *`	Pointer to the memory buffer where the packet will be written.
`out_max`	`u16`	Size of the provided buffer in bytes. Must be large enough to hold the full frame.

Ritorna:
Returns the total length of the constructed frame in bytes on success. Returns 0 if the hostname is invalid, the buffer is too small, or the name encoding fails.

Note:

- This function is non-blocking and purely computational; it does not perform any I/O or network transmission itself. It is thread-safe as it only operates on the provided buffer and local stack variables, making it safe to call from any task or ISR. The most common beginner mistake is providing a buffer that is too small (e.g., less than 70-100 bytes), which causes the function to return 0 silently. Ensure the buffer has enough headroom for the Ethernet, IP, and UDP headers, not just the DNS payload.

Uso:

- This function is used as the first step in a DNS resolution sequence. Typically, a developer allocates a network buffer using ca_net_buf_alloc, retrieves the pointer with ca_net_buf_ptr, calls this function to populate the buffer with the DNS query, and then

passes the resulting buffer to ca_net_send. It is usually paired with a state machine that tracks the 'id' to verify the incoming DNS response later. Always check the return value; if it is 0, you must free the buffer using ca_net_buf_free immediately to prevent memory leaks.

Esempio:

```
u16 txid = 0x1234;
ca_net_buf_t *b = ca_net_buf_alloc();
if (b != NULL) {
    u8 *out = b->data + 64; // Assuming sufficient headroom
    u16 len = ca_net_proto_dns_build_query(srv_mac, srv_ip, loc_mac, loc_ip,
    txid, "pool.ntp.org", out, 256);

    if (len > 0) {
        // Send the frame using the network stack
        ca_net_send(out, len);
    } else {
        // Construction failed, release the buffer
        ca_net_buf_free(b);
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_net_proto_dns_build_query in a caOS project?
A: You should call ca_net_proto_dns_build_query when you need to construct a DNS query packet to resolve a domain name in a local network. It is typically used in network services or applications that require hostname resolution, such as when connecting to a web server or other network resource by name.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_net_proto_dns_build_query?
A: u8 frame[512]; mac_t server_mac = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55}; ipv4_t server_ip = {192, 168, 1, 1}; mac_t local_mac = {0xAA, 0xBB, 0xCC, 0xDD, 0xEE, 0xFF}; ipv4_t local_ip = {192, 168, 1, 100}; const char *hostname = "google.com"; u16 len = ca_net_proto_dns_build_query(server_mac, server_ip, local_mac, local_ip, 0x1234, hostname, frame, 512); if (len > 0) { // Send frame via Ethernet driver }

Parameters conf:0.85 **Q:** What are the valid values for the 'id' parameter in ca_net_proto_dns_build_query?
A: The 'id' parameter must be a valid u16 value, typically a randomly generated or sequentially incremented value used to match DNS queries with responses. It should not be zero unless specifically required by protocol behavior, and must be a valid identifier for the DNS transaction.

Return conf:0.95 **Q:** What does a return value of 0 from ca_net_proto_dns_build_query indicate?
A: A return value of 0 indicates an error occurred. This happens when one or more parameters are invalid, such as passing NULL for hostname or out_frame, using invalid IP or MAC addresses, or when the output buffer is too small to hold the DNS query packet.

Error/Edge case conf:0.90 **Q:** What are common mistakes when calling ca_net_proto_dns_build_query?
A: Common mistakes include passing NULL for hostname or out_frame, using invalid IP or MAC addresses, providing an output buffer that is too small (out_max less than required packet size), or failing to validate that all parameters are correctly initialized before calling the function.

Behavior conf:0.85 **Q:** Is ca_net_proto_dns_build_query safe to call from an ISR in caOS?
A: Yes, ca_net_proto_dns_build_query is safe to call from an ISR, but only if no operations that are not ISR-safe are performed within the function. It is thread-safe if parameters are not shared between threads, and it is a non-blocking function that does not perform I/O or wait operations.

ca_net_proto_dns_parse_response

Header: caos/net/inc/net_proto.h:520
Source: caos/net/src/net_proto.c:1176:1227
GUARD: EN_NET_PROTO

```
bool ca_net_proto_dns_parse_response(const ca_udp_frame_t *udp,
    ca_dns_response_t *out)
```

Parses a raw UDP DNS response packet into a structured format to extract resolved IP addresses.

Parametri:

Nome	Tipo	Descrizione
udp	const ca_udp_frame_t *	Pointer to the received UDP frame. Must be a valid, non-NULL pointer containing a DNS payload.
out	ca_dns_response_t *	Pointer to the structure where the parsed DNS data (ID, RCODE, IP) will be stored. Must be non-NULL.

Ritorna:
Returns true if the packet was successfully parsed as a valid DNS response (even if no A record was found). Returns false if the input pointers are NULL, the packet is too short, the header is malformed, or the packet is not a DNS response.

Note:

- This function is non-blocking and thread-safe as it performs pure data processing on the provided buffer without modifying global state or using OS primitives. It is safe to call from an ISR, provided the input buffer is valid and the stack has sufficient space for the local variables. A common mistake is failing to check the return value; if the function returns false, the contents of the 'out' structure are undefined and should not be used. Ensure the UDP frame length is correctly calculated before passing it, as the function relies on the internal length field to prevent buffer over-reads.

Uso:

- This function is typically used within a network RX handler or a dedicated DNS service task after receiving a UDP packet on port 53. It acts as the bridge between raw network data and application logic, allowing the system to match incoming DNS responses to pending requests using the transaction ID. It is usually paired with a lookup mechanism (like an array of pending request slots) where the 'id' field is compared to identify which task is waiting for the resolution result via a semaphore.

Esempio:

```
ca_dns_response_t resp;
// Assume 'udp_frame' was received from the network stack
if (ca_net_proto_dns_parse_response(&udp_frame, &resp)) {
    if (resp.rcode == 0) { // DNS_RCODE_OK
        // Successfully parsed, check if we got an IP
        printf("Resolved IP: %d.%d.%d.%d\n", resp.ip.v[0], resp.ip.v[1], resp.ip
    .v[2], resp.ip.v[3]);
    } else {
        printf("DNS error code: %d\n", resp.rcode);
    }
```

```
} else {
    // Handle malformed packet or non-DNS traffic
    printf("Failed to parse DNS response\n");
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_net_proto_dns_parse_response in a caOS project?
A: You should call ca_net_proto_dns_parse_response after receiving a DNS response via UDP, typically when handling DNS queries in a network service. It is used to extract the IP address associated with a domain name from the DNS response packet.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_net_proto_dns_parse_response with caOS types?
A: ca_udp_frame_t udp_frame; ca_dns_response_t response; bool success = ca_net_proto_dns_par &response); if (success) { // Process the DNS response }

Parameters conf:0.95 **Q:** What are the valid and invalid values for the udp and out parameters?
A: The udp parameter must be a valid pointer to a ca_udp_frame_t structure; passing NULL will cause the function to return false. The out parameter must be a valid pointer to a ca_dns_response_t structure; passing NULL will also cause the function to return false.

Return conf:0.95 **Q:** What does it mean when ca_net_proto_dns_parse_response returns true or false?
A: The function returns true if the DNS response was parsed successfully, even if no IP address is present (e.g., if rcode != 0 or no A records are found). It returns false if the input data is invalid, such as NULL pointers or malformed DNS packets.

Error/Edge case conf:0.90 **Q:** What are the most common mistakes when using ca_net_proto_dns_parse_resp
A: Common mistakes include passing NULL pointers for udp or out, or providing a UDP packet that is malformed or incomplete. The function assumes the packet is valid and does not check for correct reception, only for correct formatting.

Behavior conf:0.90 **Q:** Is ca_net_proto_dns_parse_response blocking or thread-safe, and can it be called from an ISR?
A: The function is non-blocking and thread-safe. It can be called from an ISR, but only if no operations that are not ISR-safe (e.g., memory allocation) are performed within it.

`ca_net_proto_get_local_ip`

Header: `caos/net/inc/net_proto.h:213`
GUARD: `EN_NET_PROTO`

```
static inline ca_ipv4_t ca_net_proto_get_local_ip(void)
```

Retrieves the currently configured local IPv4 address for the network interface.

Ritorna:
Returns an ca_ipv4_t structure representing the local IP address. If no IP has been assigned or the network stack is not initialized, it returns 0.0.0.0.

Note:

- This function is a non-blocking, thread-safe getter that reads the internal state of the network stack. It is safe to call from both tasks and ISRs because it performs a simple memory read of a pre-cached variable. The primary precondition is that the network stack must have been initialized via ca_net_init and an IP address must have been assigned (either statically or via DHCP). A common mistake is assuming this function will trigger a network lookup or return the gateway address; it only returns the local interface IP.

Uso:

- This function is typically used when configuring network services that require binding to a specific local interface, such as TCP listeners or UDP servers. It is often called during the initialization phase of a network application to populate configuration structures. It is frequently paired with ca_net_proto_set_local_ip if the application needs to update the IP dynamically, or used as a source parameter for functions like ca_tcp_listen or ca_udp_server_start to ensure packets are sent from the correct interface.

Esempio:

```
/* Example: Configuring a TCP listener with the current local IP */

void start_my_service(void) {
    /* Retrieve the current local IP to bind the listener correctly */
    ca_ipv4_t my_ip = ca_net_proto_get_local_ip();

    /* Check if we have a valid IP (not 0.0.0.0) */
    if (my_ip == 0U) {
        ca_printf_n("Error: Network interface not ready");
        return;
    }

    /* Prepare configuration structure for TCP listener */
    ca_tcp_listen_cfg_t lcfg = {
        .local_port = 8080,
        .local_ip   = my_ip,
        .on_accept  = NULL,
        .on_rx      = NULL,
        .on_closed  = NULL
    };

    /* Start listening on the retrieved IP */
    u32 handle = ca_tcp_listen(&lcfg);
    if (handle == 0U) {
        ca_printf_n("Failed to start TCP listener");
    } else {
        ca_printf_n("TCP listener started successfully");
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_net_proto_get_local_ip in my application?
A: You should call this function during the initialization phase of your network application, specifically when you need to configure network services like TCP listeners or UDP servers that require binding to a specific local interface. It is also useful when you need to populate configuration structures like ca_tcp_listen_cfg_t.

Code example conf:0.90 **Q:** How do I use ca_net_proto_get_local_ip to bind a TCP listener?
A: You can retrieve the local IP and pass it to your configuration structure as follows: ca_ipv4_t my_ip = ca_net_proto_get_local_ip(); ca_tcp_listen_cfg_t cfg; cfg.local_ip = my_ip; ca_tcp_listen(&cfg);

Return conf:1.00 **Q:** What does it mean if ca_net_proto_get_local_ip returns 0.0.0.0?
A: A return value of 0.0.0.0 indicates that either no IP address has been assigned to the interface yet, or the network stack has not been initialized via ca_net_init. This function only returns the local interface IP and does not perform network lookups.

ca_net_proto_icmp_build_echo_reply

Header: caos/net/inc/net_proto.h:311
Source: caos/net/src/net_proto.c:358:415
GUARD: EN_NET_PROTO

```
u16 ca_net_proto_icmp_build_echo_reply( const ca_l2_frame_t *req_l2,
    const ca_ip4_frame_t *req_ip4, const ca_icmp_echo_t *req_icmp,
    ca_mac_t local_mac, ca_ipv4_t local_ip, u8 *out_frame, u16 out_max)
```

Constructs an ICMP Echo Reply frame in a provided buffer based on an incoming ICMP Echo Request.

Parametri:

Nome	Tipo	Descrizione
req_l2	const ca_l2_frame_t *	Pointer to the incoming Ethernet frame header.
req_ip4	const ca_ip4_frame_t *	Pointer to the incoming IPv4 frame header.
req_icmp	const ca_icmp_echo_t *	Pointer to the incoming ICMP Echo Request structure. Must be a valid ICMP type 8 (Echo Request).
local_mac	ca_mac_t	The MAC address of the local network interface.
local_ip	ca_ipv4_t	The IPv4 address of the local network interface.
out_frame	u8 *	Pointer to the destination buffer where the reply will be built.
out_max	u16	Maximum size of the destination buffer in bytes. Must be large enough to hold the header and payload.

Ritorna:
Returns the total length of the constructed frame (in bytes) on success, or 0 if the input pointers are invalid, the ICMP type is not an Echo Request, or the frame exceeds the provided buffer size.

Note:

- This function is non-blocking and thread-safe as it performs pure memory manipulation without accessing shared hardware resources or OS primitives. It is safe to call from an ISR, provided the input buffers are valid and the output buffer is large enough. The function assumes a standard 20-byte IPv4 header without options. A common mistake is providing an 'out_frame' buffer that is too small; always ensure the buffer is at least 60 bytes (Ethernet minimum) plus the payload size to avoid silent failure or buffer overflows. The function internally handles checksum calculation unless hardware offloading is enabled via ca_net_tx_csum_offload.

Uso:

- This function is typically used within a network packet processing loop. After receiving an ICMP packet via the network stack, the application validates the packet, calls this function to prepare the reply in a pre-allocated buffer, and then passes that buffer to the network driver's transmission function. It is usually paired with ca_net_send to transmit the resulting frame back to the requester. It is part of the standard response sequence: receive packet -> validate -> build reply -> transmit.

Esempio:

```c
/* Assuming an incoming ICMP request has been parsed into l2, ip4, and icmp
    structures */
u8 reply_buffer[128];
u16 reply_len = ca_net_proto_icmp_build_echo_reply(&l2, &ip4, &icmp, my_mac,
    my_ip, reply_buffer, sizeof(reply_buffer));

if (reply_len > 0) {
    /* Send the constructed reply frame via the network interface */
    ca_net_send(reply_buffer, reply_len);
} else {
    /* Handle error: buffer too small or invalid packet */
    ca_printf("Failed to build ICMP reply\n");
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_net_proto_icmp_build_echo_reply in a caOS-based embedded system?
A: You should call ca_net_proto_icmp_build_echo_reply when you receive an ICMP echo request (type 8) and need to respond with an echo reply. This function is typically used in network protocol handling, such as when implementing ICMP or ping functionality in a caOS-based embedded device.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_net_proto_icmp_build_echo_ with caOS types?
A: u8 buffer[1024]; ca_l2_frame_t l2_req; ca_ip4_frame_t ip4_req; ca_icmp_echo_t icmp_req;
u16 len = ca_net_proto_icmp_build_echo_reply(&l2_req, &ip4_req, &icmp_req, 0x123456789ABC, 0xC0A80101, buffer, 1024); if (len > 0) { // Send the buffer with length 'len' via Ethernet }

Parameters conf:0.95 **Q:** What are the valid values for the req_icmp parameter, and what happens if it's invalid?
A: The req_icmp parameter must point to a valid ICMP echo request packet (type = 8). If the pointer is NULL or the ICMP type is not 8, the function returns 0. The function does not validate other fields of the ICMP packet beyond the type field.

Return conf:0.95 **Q:** What does a return value of 0 from ca_net_proto_icmp_build_echo_reply indicate?
A: A return value of 0 indicates an error occurred during frame construction. This can happen due to invalid input pointers, an incorrect ICMP type (not 8), or an insufficient buffer size (out_max too small).

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_net_proto_icmp_build_echo_reply
A: Common mistakes include passing NULL pointers for req_l2, req_ip4, or req_icmp; providing an ICMP packet with type != 8; or supplying an output buffer that is too small for the constructed frame. Also, calling this function from an ISR is not allowed due to non-blocking but non-ISR-safe behavior.

Behavior conf:0.90 **Q:** Is ca_net_proto_icmp_build_echo_reply blocking or ISR-safe, and how does it handle thread safety?
A: The function is non-blocking and not ISR-safe. It modifies output data directly and is not thread-safe. It must not be called from an ISR because it performs data copying and computation that could cause issues in interrupt contexts.

`ca_net_proto_icmp_echo`

Header: `caos/net/inc/net_proto.h:255`
Source: `caos/net/src/net_proto.c:277:289`

GUARD: `EN_NET_PROTO`

```
bool ca_net_proto_icmp_echo(const ca_ip4_frame_t *ip4, const
    ca_icmp_echo_t **out)
```

Validates an IPv4 frame as an ICMP Echo (ping) packet and provides a pointer to the ICMP header.

Parametri:

Nome	Tipo	Descrizione
`ip4`	`const ca_ip4_frame_t *`	Pointer to the parsed IPv4 frame structure. Must be non-NULL and already validated as an IPv4 packet.
`out`	`const ca_icmp_echo_t **`	Pointer to a pointer that will be updated to point to the ICMP echo header within the frame if validation succeeds.

Ritorna:
Returns true if the frame is a valid ICMP Echo request or reply (type 0 or 8) and the payload length is sufficient. Returns false if the input pointers are NULL, the protocol is not ICMP, the packet is too short, or the ICMP type is invalid.

Note:

- This function is non-blocking and thread-safe as it performs only read-only validation on the provided memory buffer. It is safe to call from an ISR, provided the memory pointed to by 'ip4' is stable and not being modified by another process. The most common mistake is failing to check the return value before accessing the 'out' pointer; if the function returns false, the 'out' pointer remains unchanged or points to invalid data. Ensure the IPv4 frame has been correctly parsed by a preceding function like ca_net_proto_ip4 before passing it here, as this function assumes the IP header structure is already valid.

Uso:

- This function is typically used in the network stack's receive path to identify and process ping requests. It is usually called immediately after a successful IPv4 frame decoding. The standard sequence is: first, use ca_net_proto_ip4 to verify the IP layer, then pass the resulting frame to this function to extract the ICMP header. Once validated, the application can inspect the ICMP 'id', 'seq', and 'payload' fields to construct a corresponding echo reply or log the incoming ping request.

Esempio:

```
const ca_ip4_frame_t *ip4_frame;
const ca_icmp_echo_t *icmp_header;

// Assume l2_frame is already received and decoded
if (ca_net_proto_ip4(&l2_frame, &ip4_frame)) {
    // Validate if it is an ICMP Echo packet
    if (ca_net_proto_icmp_echo(ip4_frame, &icmp_header)) {
        // Successfully identified ICMP Echo
        if (icmp_header->type == 8U) {
            // Handle Echo Request (Ping)
            // Send reply using appropriate network transmit function
        }
    } else {
        // Not an ICMP Echo packet, ignore or handle other ICMP types
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_net_proto_icmp_echo in a network stack implementation?
A: You should call ca_net_proto_icmp_echo when you need to validate and extract an ICMP echo request or reply from an IPv4 frame. This function is typically used after receiving an IP packet and verifying it is an ICMP protocol packet, to ensure the payload contains a valid ICMP echo message (type 0 or 8).

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_net_proto_icmp_echo with caOS types?
A: ca_ip4_frame_t frame; ca_icmp_echo_t *icmp_ptr; bool result = ca_net_proto_icmp_echo(&fra &icmp_ptr); if (result) { // Valid ICMP echo message found at icmp_ptr } else { // Invalid or non-ICMP packet }

Parameters conf:0.95 **Q:** What are the valid and invalid values for the ip4 and out parameters?
A: The ip4 parameter must be a valid pointer to a ca_ip4_frame_t structure; passing NULL will cause the function to return false. The out parameter must be a valid pointer to a pointer of type ca_icmp_echo_t; passing NULL will also cause the function to return false. Both parameters must point to valid memory locations accessible by the function.

Return conf:0.95 **Q:** What does it mean when ca_net_proto_icmp_echo returns true or false?
A: When ca_net_proto_icmp_echo returns true, it means the IP4 frame is valid and contains an ICMP echo message of type 0 (reply) or 8 (request), and the pointer to the ICMP payload has been stored in *out. If it returns false, it indicates that the frame is invalid, not an ICMP packet, or does not contain a valid ICMP echo type (0 or 8), or that one of the input parameters was NULL.

Error/Edge case conf:0.90 **Q:** What are the most common mistakes when using ca_net_proto_icmp_echo?
A: Common mistakes include passing NULL to either ip4 or out parameters, which immediately results in a false return. Another frequent error is not verifying that the ICMP type is 0 or 8 before using the returned pointer, which can lead to accessing invalid memory. Additionally, calling this function without ensuring the IP4 frame has been correctly decoded and its payload is accessible may result in incorrect behavior.

`ca_net_proto_init`

Header: `caos/net/inc/net_proto.h:193`
Source: `caos/net/src/net_proto.c:44:74`
GUARD: `EN_NET_PROTO`

```
1 void ca_net_proto_init(const ca_net_cfg_t *cfg)
```

Initializes the network protocol stack with hardware configuration, IP settings, and internal handlers.

Parametri:

Nome	Tipo	Descrizione
`cfg`	`const ca_net_cfg_t *`	Pointer to a configuration structure containing MAC address, IP address, subnet mask, and PHY reset pins. Must not be NULL.

Ritorna:
void

Note:

- This function is not thread-safe and must be called exactly once during system initialization before any network communication occurs. It is not callable from an ISR because it performs memory assignments and registers callbacks that require a stable system state. A common mistake is calling this function multiple times or before the hardware (PHY/-MAC) is ready, which can lead to inconsistent stack states or hard faults. It is a blocking initialization routine that sets up internal buffers and protocol dispatchers.

Uso:

- This function is the entry point for the network stack. It is typically called in the main initialization sequence after hardware clocks are configured but before starting network services like DHCP, UDP servers, or TCP listeners. It is often followed by a short delay and a call to ca_eth_link_poll to ensure the physical link is established before attempting to send packets like Gratuitous ARP (GARP).

Esempio:

```
ca_net_cfg_t net_cfg = { .ip = { .addr = 0x0A000005 }, .mac = { .raw = { .l = 0
    x11223344, .h = 0x5566 } } };

// Initialize the network stack with the provided configuration
ca_net_proto_init(&net_cfg);

// Wait for the stack to settle
ca_os_delay_ms(100);

// Check physical link status and announce presence on the network
ca_eth_link_poll();
ca_arp_send_garp();

// Now safe to bind services
ca_udp_server_bind();
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_net_proto_init() in my caOS application?
A: You should call ca_net_proto_init() once after the system starts up, typically during the initialization phase of your application. It is meant to be called only once, and it initializes the network protocol stack with the provided configuration. It must not be called from an ISR and is not thread-safe.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_net_proto_init() with caOS types?
A: ca_net_cfg_t cfg = { .ip = {192, 168, 1, 100}, .mac = {0xDE, 0xAD, 0xBE, 0xEF, 0xCA, 0xFE}, .mtu = 1500, .phy_addr = 0x00 }; ca_net_proto_init(&cfg);

Parameters conf:0.95 **Q:** What does the cfg parameter in ca_net_proto_init() represent, and what happens if it is NULL?
A: The cfg parameter is a pointer to a ca_net_cfg_t structure that contains network configuration details such as IP address, MAC address, MTU, and PHY information. If cfg is NULL, the function returns immediately without performing any action.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_net_proto_init()?
A: A common mistake is calling ca_net_proto_init() more than once, which is not supported. Another is passing a NULL pointer for cfg, which will cause the function to return immediately without effect. It should also not be called from an ISR, as it is not ISR-safe.

Behavior conf:0.90 **Q:** Is ca_net_proto_init() blocking or non-blocking, and is it safe to call from an ISR or in a multithreaded context?
A: ca_net_proto_init() is a blocking function and is not ISR-safe or thread-safe. It must not be called from an ISR, and it should only be called once in a single-threaded

context after system startup.

---

ca_net_proto_ip4

Header: caos/net/inc/net_proto.h:240
Source: caos/net/src/net_proto.c:211:233
GUARD: EN_NET_PROTO

```
bool ca_net_proto_ip4(const ca_l2_frame_t *l2, u16 len, const
    ca_ip4_frame_t **out)
```

Validates an IPv4 packet structure and checksum within a received L2 frame.

Parametri:

Nome	Tipo	Descrizione
l2	const ca_l2_frame_t *	Pointer to the received Layer 2 Ethernet frame. Must be non-NULL.
len	u16	Total length of the received frame in bytes. Must be sufficient to contain the IP header.
out	const ca_ip4_frame_t **	Pointer to a pointer that will be updated to point to the validated IPv4 header inside the L2 frame if successful.

Ritorna:
Returns true if the packet is a valid IPv4 frame with a correct checksum; returns false if the packet is malformed, not IPv4, or fails the checksum validation.

Note:

- This function is non-blocking and thread-safe as it performs read-only validation on existing memory buffers. It is safe to call from an ISR, provided the L2 frame buffer is already resident in memory. The most common beginner mistake is passing an uninitialized pointer for 'out' or failing to verify the 'len' parameter against the actual received data size, which can lead to out-of-bounds memory access. Ensure the L2 frame has been processed by ca_net_proto_l2 first to confirm it is an Ethernet frame before passing it here.

Uso:

- This function is used in the network receive pipeline after the L2 layer has been stripped or identified. It is typically called in a sequence: first, use ca_net_proto_l2 to identify the frame type, then call ca_net_proto_ip4 to validate the IP header, and finally, if successful, proceed to higher-level protocols like ca_net_proto_udp or ca_net_proto_tcp. It acts as a gatekeeper to ensure that only well-formed IP packets are passed to the application logic.

Esempio:

```
const ca_l2_frame_t *l2_frame;
const ca_ip4_frame_t *ip4_frame;
u16 frame_len = 1500; // Example length

// Assume l2_frame is already populated from the network interface
if (ca_net_proto_ip4(l2_frame, frame_len, &ip4_frame)) {
    // Packet is valid IPv4, now check for UDP protocol
    if (ip4_frame->proto == 17) { // 17 is UDP
        // Proceed to parse UDP payload
        // ca_net_proto_udp(&ip4_frame, &udp_ptr);
    }
```

```
} else {
    // Log error or drop packet
    ca_net_count_err();
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use the ca_net_proto_ip4 function in a caOS-based network stack?
A: You should call ca_net_proto_ip4 when you need to validate that an Ethernet frame contains a valid IPv4 packet and extract the IPv4 payload. It is typically used after receiving a frame from the network hardware to ensure it's an IPv4 packet before processing it further.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_net_proto_ip4 with caOS types?
A: ca_l2_frame_t frame; ca_ip4_frame_t *ip4_payload; bool result = ca_net_proto_ip4(&frame, 60, &ip4_payload); if (result) { // Process the IPv4 payload }

Parameters conf:0.90 **Q:** What are the valid and invalid values for the 'len' parameter in ca_net_proto_ip4?
A: The 'len' parameter must be a positive value that is sufficient to contain both the Ethernet header and the IPv4 header. A value of 0 or any value smaller than the minimum required (20 bytes for IPv4 header) is invalid and will cause the function to return false.

Return conf:0.95 **Q:** What does it mean when ca_net_proto_ip4 returns false?
A: A return value of false indicates that the frame is not a valid IPv4 packet. This can be caused by null pointers, an incorrect Ethertype (not ETH_P_IP), insufficient frame length, a bad IPv4 checksum, or an incorrect IP version (not 4).

Error/Edge case conf:0.85 **Q:** What are common mistakes developers make when using ca_net_proto_ip4?
A: Common mistakes include passing uninitialized or null pointers, not checking the return value of the function, and passing frames that are not Ethernet frames or lack sufficient data for an IPv4 header. These can lead to invalid memory access or incorrect protocol handling.

Behavior conf:0.90 **Q:** Is ca_net_proto_ip4 safe to call from an ISR in caOS?
A: Yes, ca_net_proto_ip4 is safe to call from an ISR because it is not blocking and does not perform operations that require RTOS services or access to shared resources. However, it is recommended to avoid complex operations or access to shared data within the ISR.

`ca_net_proto_l2`

Header: `caos/net/inc/net_proto.h:224`
Source: `caos/net/src/net_proto.c:162:185`
GUARD: `EN_NET_PROTO`

```
ca_l2_result_t ca_net_proto_l2(const u8 *frame, u16 len, ca_mac_t
    local_mac, const ca_l2_frame_t **out)
```

Filters incoming Ethernet frames at Layer 2 based on destination MAC address and provides a pointer to the frame header.

Parametri:

Nome	Tipo	Descrizione
`frame`	`const u8 *`	Pointer to the raw buffer containing the incoming Ethernet frame. Must not be NULL.
`len`	`u16`	Length of the frame in bytes. Must be at least 14 (size of Ethernet header) to be valid.
`local_mac`	`ca_mac_t`	The MAC address assigned to this device. If set to all zeros, the function acts in promiscuous mode.
`out`	`const ca_l2_frame_t **`	Pointer to a pointer that will be updated to point to the start of the Ethernet header within the frame buffer.

Ritorna:
Returns a ca_l2_result_t enum: CA_L2_ACCEPT_BROADCAST, CA_L2_ACCEPT_MULTICAST, or CA_L2_ACCEPT_UNICAST if the frame is accepted. Returns CA_L2_DROP if the frame is invalid, too short, or does not match the local MAC address.

Note:

- This function is non-blocking and thread-safe as it performs only read-only operations on the provided buffer. It is safe to call from an ISR because it does not perform any memory allocation or synchronization primitives. The most common mistake is passing a buffer shorter than 14 bytes or failing to check the return value, which leads to processing garbage data. Ensure the frame pointer is correctly aligned if your architecture requires strict alignment for struct access.

Uso:

- This function is typically the first step in the network receive pipeline, called immediately after receiving a packet from the hardware (e.g., via ca_eth_recv). It acts as a gatekeeper to decide if the stack should spend CPU cycles parsing the payload. After a successful return, the caller typically checks the EtherType field in the returned 'out' structure to decide which higher-level protocol handler (like ca_net_proto_arp or ca_net_proto_ip4) to invoke next.

Esempio:

```
/* Assuming 'buffer' contains raw Ethernet data received from hardware */
const ca_l2_frame_t *l2_header;
mac_t my_mac = { .raw = { .l = 0x11223344, .h = 0x5566 } };

/* Filter the frame at Layer 2 */
ca_l2_result_t result = ca_net_proto_l2(buffer, frame_len, my_mac, &l2_header);

if (result != CA_L2_DROP) {
    /* Frame is for us, check if it is an ARP packet */
    if (l2_header->ethertype == 0x0806) {
        ca_arp_frame_t arp_data;
        if (ca_net_proto_arp(l2_header, &arp_data)) {
            /* Process ARP payload */
        }
    }
} else {
    /* Drop frame and free buffer */
    ca_net_buf_free((void*)buffer);
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_net_proto_l2 in a network driver implementation?

A: You should call ca_net_proto_l2 when processing incoming Ethernet frames to determine if the frame is intended for the local device, a broadcast address, or a multicast address. It helps filter frames before further processing, especially in a network stack where you need to decide whether to accept or drop a frame based on its destination MAC address.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_net_proto_l2 with caOS types?
A: u8 frame[1500]; mac_t local_mac = { .raw = { .l = 0x123456789ABC, .h = 0 } }; const ca_l2_frame_t *parsed_frame; ca_l2_result_t result = ca_net_proto_l2(frame, 60, local_mac, &parsed_frame); if (result == CA_L2_ACCEPT_UNICAST) { // Process unicast frame }

Parameters conf:0.85 **Q:** What are the valid and invalid values for the local_mac parameter?
A: The local_mac parameter is a ca_mac_t structure. Valid values are any MAC address where both raw.l and raw.h are not zero, indicating a specific device MAC. If both fields are zero, the function operates in promiscuous mode, accepting all unicast frames. Invalid values include NULL frame pointer, or a frame shorter than 14 bytes, which will cause the function to return CA_L2_DROP.

Return conf:0.90 **Q:** What does CA_L2_ACCEPT_BROADCAST mean and when is it returned?
A: CA_L2_ACCEPT_BROADCAST is returned when the destination MAC address in the frame is a broadcast address (FF:FF:FF:FF:FF:FF). This indicates that the frame is intended for all devices on the network segment and should be processed accordingly.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_net_proto_l2?
A: Common mistakes include passing a NULL frame pointer, a frame shorter than 14 bytes, or a NULL out parameter. Additionally, not checking the return value before using *out can lead to invalid memory access. Also, passing unaligned frame pointers may cause hard faults on ARM processors without MMU.

Behavior conf:0.95 **Q:** Is ca_net_proto_l2 blocking or ISR-safe?
A: ca_net_proto_l2 is non-blocking and ISR-safe. It is also thread-safe since it does not maintain any internal state. It can be safely called from ISRs as long as the parameters are valid and the frame data is in non-volatile memory.

`ca_net_proto_mdns_build_announce`

Header: `caos/net/inc/net_proto.h:541`
Source: `caos/net/src/net_proto.c:1343:1348`
GUARD: `EN_NET_PROTO`

```
u16 ca_net_proto_mdns_build_announce(ca_mac_t local_mac, ca_ipv4_t
    local_ip, const char *hostname, u8 *out_frame, u16 out_max)
```

Constructs an mDNS announcement frame for network service discovery.

Parametri:

Nome	Tipo	Descrizione
`local_mac`	`ca_mac_t`	The local device's MAC address structure.
`local_ip`	`ca_ipv4_t`	The local device's IPv4 address structure.
`hostname`	`const char *`	The string representing the device name (e.g., 'my-device'). Must be null-terminated.
`out_frame`	`u8 *`	Pointer to the buffer where the binary mDNS packet will be written.
`out_max`	`u16`	The maximum size of the out_frame buffer to prevent memory corruption.

Ritorna:
Returns the total length of the generated mDNS frame in bytes. Returns 0 if the frame could not be built (e.g., buffer too small or invalid parameters).

Note:

- This function is a pure data-formatting utility; it is non-blocking and thread-safe as it does not access shared hardware or global state beyond the provided pointers. It is safe to call from an ISR, provided the output buffer is valid and accessible. The most common mistake is providing a buffer that is too small for the hostname and protocol headers, which will result in a return value of 0. Ensure the buffer has sufficient space for the Ethernet, IP, UDP, and mDNS headers plus the hostname string.

Uso:

- This function is typically used in the network stack's service layer to broadcast the device's presence on the local network. It is usually paired with ca_net_buf_alloc to obtain a memory buffer, followed by this function to populate the payload, and finally ca_net_send to transmit the frame over the network interface. It is part of the initialization sequence for mDNS services where the device announces its identity to other network nodes.

Esempio:

```c
/* Allocate a network buffer for the mDNS announcement */
ca_net_buf_t *b = ca_net_buf_alloc();
if (b != NULL) {
    u8 *out = b->data + 14; /* Offset for headers */
    u16 len = ca_net_proto_mdns_build_announce(my_mac, my_ip, "caos-device", out
    , 256);

    if (len > 0) {
        /* Frame built successfully, proceed to send via network interface */
        /* ca_net_send(b, len); */
    } else {
        /* Failed to build, release the buffer */
        ca_net_buf_free(b);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_net_proto_mdns_build_announce in a caOS network application?
A: You should call ca_net_proto_mdns_build_announce when you need to construct an MDNS announcement packet to advertise a local hostname on the network. This function is typically used during network initialization or when a device's IP address changes, to ensure other devices on the network are aware of the device's presence and hostname.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_net_proto_mdns_build_anno with caOS types?

A: u8 mac[6] = {0x00, 0x08, 0x9B, 0x00, 0x00, 0x01}; u8 ip[4] = {192, 168, 1, 100}; cstr hostname = "caos-board"; u8 frame[512]; u16 len = ca_net_proto_mdns_build_announce(mac, ip, hostname, frame, 512); if (len > 0) { // Successfully built MDNS packet, send frame on network }

Parameters conf:0.90 **Q:** What are the valid and invalid values for the hostname parameter in ca_net_proto_mdns_build_announce?

A: The hostname parameter must be a non-NULL, null-terminated C string with a maximum length of 255 characters. Passing NULL for hostname will cause undefined behavior or a crash. Any string longer than 255 characters will result in an invalid packet being built, as the function does not truncate or validate the length beyond the maximum allowed.

Return conf:0.95 **Q:** What does a return value of 0 from ca_net_proto_mdns_build_announce indicate?

A: A return value of 0 indicates an error occurred, either because one of the input parameters was invalid (such as NULL pointers, invalid MAC or IP addresses, or insufficient buffer size), or because the output buffer 'out_max' was not large enough to hold the constructed MDNS packet.

`ca_net_proto_mdns_parse_query`

Header: `caos/net/inc/net_proto.h:550`
Source: `caos/net/src/net_proto.c:1350:1377`
GUARD: `EN_NET_PROTO`

```
bool ca_net_proto_mdns_parse_query(const ca_udp_frame_t *udp, const char
    *our_hostname, bool *is_for_us)
```

Parses an incoming mDNS UDP packet to check if it contains a query for the device's hostname.

Parametri:

Nome	Tipo	Descrizione
`udp`	`const ca_udp_frame_t *`	Pointer to the received UDP frame structure. Must be a valid, non-NULL pointer.
`our_hostname`	`const char *`	The string representing the device's own hostname (e.g., 'my-device').
`is_for_us`	`bool *`	Pointer to a boolean variable where the result is stored. Set to true if the query matches our hostname.

Ritorna:
Returns true if the packet was successfully parsed as a valid mDNS query, false otherwise. Note that a true return does not guarantee a match; check the value pointed to by is_for_us.

Note:

- This function is non-blocking and thread-safe as it performs read-only operations on the provided buffer. It is safe to call from an ISR context, provided the UDP frame pointer is valid. The most common mistake is failing to check the return value of the function before using the is_for_us flag, which might contain stale data if the packet was malformed. Ensure the hostname string is null-terminated and the UDP frame has been correctly processed by the network stack before passing it here.

Uso:

- This function is typically used within an mDNS service RX handler to filter incoming network traffic. It is called after receiving a UDP packet to determine if the device needs to respond to a discovery request. It is usually paired with ca_net_proto_mdns_build_announce to send a response if the query matches, often triggering a semaphore or event to wake up a background task that handles the network response.

Esempio:

```
void ca_mdns_service_rx_handler(const ca_udp_frame_t *udp, u16 len) {
    bool is_for_us = false;
    const char *my_name = "caos-board";

    /* Parse the incoming UDP packet to see if it is an mDNS query for us */
    if (ca_net_proto_mdns_parse_query(udp, my_name, &is_for_us)) {
        if (is_for_us) {
            /* Signal the mDNS task that a valid query was received */
            ca_sem_post(&s_query_sem);
        }
    }
    /* If the function returns false, the packet was not a valid mDNS query */
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_net_proto_mdns_parse_query in a caOS network application?
A: You should call ca_net_proto_mdns_parse_query when you receive a UDP packet that might contain an mDNS query, typically in your network receive handler. It helps determine if the query is directed at your device's hostname, allowing you to respond appropriately in mDNS discovery scenarios.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_net_proto_mdns_parse_query with caOS types?
A: ca_udp_frame_t frame; bool is_for_us; cstr hostname = "caos-board"; bool result = ca_net_proto_mdns_parse_query(&frame, hostname, &is_for_us); if (result && is_for_us) { // Handle query for our hostname }

Parameters conf:0.95 **Q:** What are the valid and invalid values for the 'our_hostname' parameter?
A: The 'our_hostname' parameter must be a non-null, null-terminated string. Invalid values include NULL pointers or empty strings, which will cause the function to return false and lead to undefined behavior.

Return conf:0.90 **Q:** What does it mean when ca_net_proto_mdns_parse_query returns false?
A: A return value of false indicates that either one of the parameters was NULL, the UDP packet was too short, or the packet was not a valid mDNS query. The function does not perform any side effects in this case.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_net_proto_mdns_parse_query?
A: Common mistakes include passing a NULL pointer for 'our_hostname', passing an unterminated string, or calling the function from an ISR. Also, not ensuring the UDP frame contains valid mDNS data can lead to incorrect results or failures.

Behavior conf:0.90 **Q:** Is ca_net_proto_mdns_parse_query blocking or thread-safe?
A: The function is non-blocking and not thread-safe. It must not be called from an ISR and should not be invoked concurrently from multiple threads without external synchronization.

---

ca_net_proto_ntp_build_request

Header: caos/net/inc/net_proto.h:414
Source: caos/net/src/net_proto.c:878:950
GUARD: EN_NET_PROTO

```
u16 ca_net_proto_ntp_build_request(ca_mac_t server_mac, ca_ipv4_t
    server_ip, ca_mac_t local_mac, ca_ipv4_t local_ip, u8 *out_frame, u16
     out_max)
```

Constructs a complete NTP client request packet (L2+IP+UDP+NTP) into a provided buffer.

Parametri:

Nome	Tipo	Descrizione
server_mac	ca_mac_t	Destination MAC address of the NTP server.
server_ip	ca_ipv4_t	Destination IPv4 address of the NTP server.
local_mac	ca_mac_t	Source MAC address of the local device.
local_ip	ca_ipv4_t	Source IPv4 address of the local device.
out_frame	u8 *	Pointer to the memory buffer where the packet will be built. Must be at least 90 bytes.
out_max	u16	Size of the provided buffer. If less than 90, the function returns 0.

Ritorna:
Returns the total size of the constructed frame (90 bytes) on success, or 0 if the buffer is too small or NULL.

Note:

- This function is a pure data-formatting utility; it is non-blocking, thread-safe (as it only operates on the provided buffer), and can be safely called from an ISR. It does not perform any network I/O, it only populates the memory structure. The most common mistake is providing a buffer smaller than the required 90 bytes or failing to check the return value, which leads to sending uninitialized or partial data. Ensure the local and server addresses are correctly resolved before calling this.

Uso:

- This function is typically used in the application layer to prepare a packet before sending it over the network. It is usually paired with ca_net_buf_alloc to obtain a memory buffer, then this function fills the payload, and finally ca_net_send is called to transmit the frame. It is part of the standard sequence: allocate buffer -> build request -> send frame -> free buffer.

Esempio:

```
/* Example: Building and sending an NTP request */
#define NTP_FRAME_SIZE 90U

void send_ntp_request(ca_mac_t srv_mac, ca_ipv4_t srv_ip, ca_mac_t loc_mac,
    ca_ipv4_t loc_ip) {
    /* Allocate a network buffer from the pool */
    u16 buf_idx = ca_net_buf_alloc();
    if (buf_idx == 0xFFFFU) return; /* Allocation failed */

    ca_net_buf_t *b = ca_net_buf_ptr(buf_idx);
    u8 *out = b->data + 14U; /* Offset for headroom */

    /* Build the NTP packet into the buffer */
```

```
    u16 len = ca_net_proto_ntp_build_request(srv_mac, srv_ip, loc_mac, loc_ip,
    out, 128U);

    if (len > 0U) {
        /* Send the frame using the network interface */
        ca_net_send(out, len);
    }

    /* Always free the buffer after use */
    ca_net_buf_free(buf_idx);
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use the ca_net_proto_ntp_build_request function in a caOS project?

A: You should use ca_net_proto_ntp_build_request when you need to construct an NTP (Network Time Protocol) request packet to synchronize time with an NTP server. This function is typically called when initializing network services or when a time synchronization is required, for example, in a networked embedded system where accurate timekeeping is essential.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_net_proto_ntp_build_reques with caOS types?

A: u8 frame[100]; mac_t server_mac = {0x01, 0x02, 0x03, 0x04, 0x05, 0x06}; ipv4_t server_ip = {192, 168, 1, 100}; mac_t local_mac = {0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F}; ipv4_t local_ip = {192, 168, 1, 50}; u16 size = ca_net_proto_ntp_build_request(server_mac, server_ip, local_mac, local_ip, frame, 100); if (size != 0) { // Send frame using network driver }

Parameters conf:0.85 **Q:** What are the valid values for the ca_mac_t and ca_ipv4_t parameters in ca_net_proto_ntp_build_request?

A: The ca_mac_t and ca_ipv4_t parameters must contain valid network addresses. For ca_mac_t, it should be a 6-byte array representing a valid MAC address. For ca_ipv4_t, it should be a 4-byte array representing a valid IPv4 address. Invalid values such as all zeros or malformed addresses may cause the packet to be misrouted or not processed correctly by the network stack.

Return conf:0.95 **Q:** What does a return value of 0 from ca_net_proto_ntp_build_request indicate?

A: A return value of 0 indicates an error occurred during packet construction. This typically happens when the output buffer 'out_frame' is NULL or when its size is less than NTP_FRAME_SIZE (90 bytes). No packet is generated in this case.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_net_proto_ntp_build_request?

A: Common mistakes include passing a NULL pointer for 'out_frame', providing a buffer smaller than NTP_FRAME_SIZE (90 bytes), or using invalid MAC or IP addresses. Not checking the return value can also lead to sending malformed packets or buffer overflows.

Behavior conf:0.85 **Q:** Is ca_net_proto_ntp_build_request blocking or ISR-safe, and how does it behave regarding thread safety?

A: The function is non-blocking and not ISR-safe. It is thread-safe if the parameters are not shared between threads. It uses functions like ca_memset, which may not be available in ISR contexts, and does not modify any global or shared data, so it's safe to call from multiple threads as long as parameters are not shared.

---

ca_net_proto_ntp_parse_response

Header: caos/net/inc/net_proto.h:423
Source: caos/net/src/net_proto.c:953:993
GUARD: EN_NET_PROTO

```
bool ca_net_proto_ntp_parse_response(const ca_udp_frame_t *udp,
    ca_ntp_pkt_t *out)
```

Parses a raw UDP NTP response packet into a structured format for time synchronization.

Parametri:

Nome	Tipo	Descrizione
udp	const ca_udp_frame_t *	Pointer to the received UDP frame. Must be a valid, non-NULL pointer containing a valid NTP payload.
out	ca_ntp_pkt_t *	Pointer to the destination structure where the parsed NTP data will be stored. Must be non-NULL.

Ritorna:
Returns true if the packet was successfully parsed and validated (correct mode, sufficient length). Returns false if the input pointers are NULL, the packet is too short, or the NTP mode is not 'Server' (4).

Note:

- This function is non-blocking and performs pure data manipulation, making it safe to call from an ISR or a high-priority task. It assumes the input buffer contains a valid UDP frame structure. A common mistake is passing a pointer to a buffer that does not actually contain an NTP packet, which will cause the function to return false due to the mode validation check. It does not perform any network I/O; it only interprets the memory layout of the provided buffer.

Uso:

- This function is typically used within a network callback or a service handler that receives UDP packets on port 123. It is usually paired with ca_net_proto_udp to extract the frame, followed by passing the result to a queue via ca_queue_static_trysend to notify the main NTP service task. It acts as the bridge between raw network buffers and the application-level time synchronization logic.

Esempio:

```
/* Assuming 'udp' is a pointer to a received UDP frame */
ca_ntp_pkt_t ntp_data;

/* Validate and parse the NTP response */
if (ca_net_proto_ntp_parse_response(udp, &ntp_data)) {
    /* If successful, we can use the parsed data */
    ca_printf("NTP Server Stratum: %u\n", ntp_data.stratum);
} else {
    /* Handle parsing error or invalid packet */
    ca_printf("Failed to parse NTP packet\n");
}
```

Q&A — fine-tuning review

6/6 pairs · qwen3-coder:latest

Usage conf:0.95 **Q:** When should I call ca_net_proto_ntp_parse_response in a caOS application?

A: You should call ca_net_proto_ntp_parse_response when you have received a UDP packet containing an NTP response and want to extract and convert the time-related fields into host format. It is typically used after receiving an NTP server response in an NTP client implementation.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_net_proto_ntp_parse_response with caOS types?
A: ca_udp_frame_t udp_frame; ca_ntp_pkt_t ntp_pkt; bool success = ca_net_proto_ntp_parse_re &ntp_pkt); if (success) { // Use ntp_pkt fields }

Parameters conf:0.90 **Q:** What are the valid values for the udp and out parameters of ca_net_proto_ntp_parse_
A: Both 'udp' and 'out' must be non-NULL pointers. The 'udp' parameter must point to a valid UDP frame structure with a complete NTP payload. The 'out' parameter must point to a valid ca_ntp_pkt_t structure where the parsed data will be copied.

Return conf:0.95 **Q:** What does it mean when ca_net_proto_ntp_parse_response returns false?
A: A return value of false indicates that the NTP packet was malformed, too short, or had a mode field that was not set to 4 (indicating it's not a server response). It also means that one or both of the input pointers were NULL.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_net_proto_ntp_parse_response?
A: Common mistakes include passing NULL pointers for 'udp' or 'out', providing a UDP packet that does not contain a valid NTP response (e.g., incorrect length or mode field), or calling the function before the full UDP packet is received.

Behavior conf:0.90 **Q:** Is ca_net_proto_ntp_parse_response blocking or ISR-safe?
A: The function is non-blocking and ISR-safe. It performs only data reading and copying operations without any RTOS calls or memory allocations that could cause issues in ISR contexts.

`ca_net_proto_set_local_ip`

Header: `caos/net/inc/net_proto.h:207`
GUARD: `EN_NET_PROTO`

```
1 static inline void ca_net_proto_set_local_ip(ca_ipv4_t ip)
```

Updates the internal network stack's local IPv4 address configuration.

Parametri:

Nome	Tipo	Descrizione
`ip`	`ca_ipv4_t`	A structure containing the 32-bit IPv4 address to be assigned to the network interface.

Ritorna:
void

Note:

- This function is a non-blocking, thread-safe operation that directly updates the internal state of the network protocol stack. It is not intended to be called from an ISR, as it is designed for configuration tasks within the main application logic. A common mistake is calling this function before the network stack has been initialized via ca_net_init; doing so will result in the address being stored in an uninitialized or invalid memory location, leading to unpredictable network behavior. Always ensure the network interface is ready before applying a new IP address.

Uso:

- This function is typically used during the network configuration phase, specifically after a DHCP lease has been successfully acquired or when setting a static IP address.

It is almost always paired with other service-specific update functions to ensure the entire stack remains synchronized. The standard sequence is: 1) Receive IP from DHCP or user input, 2) Call ca_net_proto_set_local_ip to update the core stack, 3) Call ca_udp_server_set_local_ip to update UDP services, and 4) Call ca_arp_service_update_local_ip to ensure ARP responses reflect the new address.

Esempio:

```
/* Example: Updating local IP after a successful DHCP lease acquisition */
ipv4_t new_ip;
/* Assume new_ip is populated with valid data from DHCP */

/* Update the core network protocol stack */
ca_net_proto_set_local_ip(new_ip);

/* Update UDP server instance to use the new IP */
ca_udp_server_set_local_ip(new_ip);

/* Update ARP service to ensure correct address resolution */
ca_arp_service_update_local_ip(new_ip);

/* Log the change to the system console */
ca_stdout_printf_n("Network IP updated successfully");
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to call ca_net_proto_set_local_ip in my application?
A: This function should be called during the network configuration phase, typically after a DHCP lease is acquired or when setting a static IP. It must be called after ca_net_init to avoid storing the address in uninitialized memory. It is recommended to follow the sequence: update the core stack, update UDP services via ca_udp_server_set_local_ip, and update ARP via ca_arp_service_update_local_ip.

Code example conf:0.90 **Q:** How do I implement a static IP update using caOS types?
A: You must define an ca_ipv4_t structure and pass it to the function after initialization: ca_ipv4_t my_ip; my_ip.addr = 0xC0A8010A; ca_net_init(); ca_net_proto_set_local_ip(my_ip);

Parameters conf:1.00 **Q:** What does the ca_ipv4_t parameter represent and are there any restrictions on its usage?
A: The ca_ipv4_t parameter contains the 32-bit IPv4 address to be assigned to the network interface. It is a non-blocking, thread-safe operation, but it is not intended to be called from an ISR. Calling it before ca_net_init is invalid and will lead to unpredictable network behavior.

`ca_net_proto_syslog_build`

Header: `caos/net/inc/net_proto.h:452`
Source: `caos/net/src/net_proto.c:1016:1064`
GUARD: `EN_NET_PROTO`

```
u16 ca_net_proto_syslog_build(ca_mac_t server_mac, ca_ipv4_t server_ip,
    ca_mac_t local_mac, ca_ipv4_t local_ip, u8 facility, u8 severity,
    const char *msg, u8 *out_frame, u16 out_max)
```

Constructs a complete Ethernet/IP/UDP frame containing a formatted Syslog message.

Parametri:

Nome	Tipo	Descrizione
server_mac	ca_mac_t	Destination MAC address of the Syslog server.
server_ip	ca_ipv4_t	Destination IPv4 address of the Syslog server.
local_mac	ca_mac_t	Source MAC address of the local device.
local_ip	ca_ipv4_t	Source IPv4 address of the local device.
facility	u8	Syslog facility code (0-23).
severity	u8	Syslog severity level (0-7).
msg	const char *	The log message string to send.
out_frame	u8 *	Pointer to the buffer where the frame will be built.
out_max	u16	Size of the out_frame buffer in bytes.

Ritorna:
Returns the total length of the constructed frame (Ethernet + IP + UDP + Payload) on success, or 0 if the buffer is too small, inputs are NULL, or formatting fails.

Note:

- This function is non-blocking and performs pure memory manipulation to assemble network headers and the payload. It is thread-safe as it does not access shared global state, but it is not intended to be called from an ISR due to the stack usage of the internal 256-byte buffer. Ensure the 'out_frame' buffer is large enough to hold the headers (42 bytes total) plus the formatted message; otherwise, the function will return 0. A common mistake is providing a buffer that is too small for the message, leading to silent failure.

Uso:

- This function is typically used in a logging service or a diagnostic task to prepare data for transmission over the network. It is usually paired with ca_net_buf_alloc to obtain a buffer, then this function fills it, and finally ca_net_send is called to transmit the frame. After transmission, the buffer should be released using ca_net_buf_free. It is the standard way to encapsulate application-level log strings into a transportable network packet.

Esempio:

```
u8 *frame = ca_net_buf_alloc();
if (frame != NULL) {
    u16 len = ca_net_proto_syslog_build(srv_mac, srv_ip, loc_mac, loc_ip, 16, 6,
    "System Boot", frame, 256);
    if (len > 0) {
        // Frame is ready, send it via the network stack
        ca_net_send(frame, len);
    } else {
        // Buffer too small or error, free it
        ca_net_buf_free(frame);
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use the ca_net_proto_syslog_build function in a caOS project?
A: You should use ca_net_proto_syslog_build when you need to construct a syslog UDP/IP/L2 frame to send log messages to a syslog server. It's typically used in networked embedded applications where logging to a central server is required for monitoring and debugging purposes.

Code example conf:0.90 **Q:** Can you provide a minimal working C example using ca_net_proto_syslog_build with caOS types?
A: u8 frame[1024]; mac_t server_mac = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55}; ipv4_t server_ip = {192, 168, 1, 100}; mac_t local_mac = {0xAA, 0xBB, 0xCC, 0xDD, 0xEE,

0xFF}; ipv4_t local_ip = {192, 168, 1, 50}; u16 len = ca_net_proto_syslog_build(server_mac, server_ip, local_mac, local_ip, 3, 6, "Test message", frame, sizeof(frame)); if (len > 0) { // Send frame via network interface }

Parameters conf:0.95 **Q:** What are the valid ranges for facility and severity parameters in ca_net_proto_syslog_build?
A: The facility parameter must be in the range 0-7, and the severity parameter must also be in the range 0-7. Values outside these ranges will cause the function to return 0, indicating an error.

Return conf:0.95 **Q:** What does a return value of 0 from ca_net_proto_syslog_build indicate?
A: A return value of 0 indicates an error occurred during frame construction. This can happen due to invalid parameters (like invalid IP/MAC addresses, out-of-range facility/-severity values), a NULL msg pointer, a NULL out_frame pointer, or an insufficiently sized output buffer.

Error/Edge case conf:0.85 **Q:** What are common mistakes when calling ca_net_proto_syslog_build?
A: Common mistakes include passing NULL for the msg or out_frame parameters, providing invalid IP or MAC addresses, using facility or severity values outside the 0-7 range, or supplying an output buffer that is too small (less than 42 bytes).

Behavior conf:0.90 **Q:** Is ca_net_proto_syslog_build thread-safe and can it be called from an ISR?
A: ca_net_proto_syslog_build is not thread-safe unless protected by a mutex. It is also not ISR-safe and should not be called from interrupt service routines. The function is non-blocking and requires valid parameters and sufficient buffer space.

`ca_net_proto_tcp`

Header: `caos/net/inc/net_proto.h:215`
Source: `caos/net/src/net_proto.c:258:272`
GUARD: `EN_NET_PROTO`

```
bool ca_net_proto_tcp(const ca_ip4_frame_t *ip4, const ca_tcp_frame_t **
    out)
```

Validates an IPv4 frame to ensure it contains a well-formed TCP segment.

Parametri:

Nome	Tipo	Descrizione
`ip4`	`const ca_ip4_frame_t *`	Pointer to the incoming IPv4 frame structure. Must not be NULL.
`out`	`const ca_tcp_frame_t **`	Pointer to a pointer that will be updated to point to the TCP header within the IP payload if validation succeeds. Must not be NULL.

Ritorna:
Returns true if the frame is a valid TCP packet with a correct header length and payload size; returns false if the frame is not TCP, is malformed, or if input pointers are NULL.

Note:

- This function is non-blocking and thread-safe as it performs read-only validation on existing memory buffers. It is safe to call from an ISR because it does not perform any heap allocation or complex synchronization. The primary precondition is that the 'ip4' pointer must point to a valid, initialized memory buffer containing a complete IPv4 frame. A common mistake is failing to check the return value; if you ignore the boolean result

and use the 'out' pointer, you may be dereferencing an uninitialized or invalid memory address, leading to a hard fault.

Uso:

- This function is typically used in the network receive path after the Ethernet and IP layers have been processed. It acts as a gatekeeper to ensure that subsequent TCP-specific logic (like parsing flags or sequence numbers) only operates on valid data. It is usually paired with ca_net_proto_ip4 to verify the IP layer first, and then used to extract the TCP header before passing it to higher-level handlers like ca_tcp_accept or ca_tcp_recv.

Esempio:

```c
/* Assuming 'frame' is a pointer to an incoming network buffer */
const ca_tcp_frame_t *tcp_hdr = NULL;

/* First, ensure the frame is a valid IPv4 packet */
if (ca_net_proto_ip4(frame)) {
    /* Validate that the IP payload is indeed a TCP segment */
    if (ca_net_proto_tcp(frame, &tcp_hdr)) {
        /* Successfully validated TCP header, now safe to access tcp_hdr fields */
        /* Example: check if it is a SYN packet using custom logic */
        /* if (tcp_hdr->flags & TCP_SYN) { ... } */
    } else {
        /* Not a TCP packet or malformed, drop or log error */
        ca_net_count_err();
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_net_proto_tcp in a network stack implementation?
A: You should call ca_net_proto_tcp when you need to validate and extract a TCP frame from an IP4 packet. This function is typically used after receiving an IP packet to ensure it contains a valid TCP segment before further processing.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_net_proto_tcp with caOS types?
A: ca_ip4_frame_t ip4_frame; ca_tcp_frame_t *tcp_frame; bool result = ca_net_proto_tcp(&ip4_f &tcp_frame); if (result) { // Process tcp_frame }

Parameters conf:0.90 **Q:** What are the valid and invalid values for the ip4 and out parameters?
A: The ip4 parameter must be a valid, non-NULL pointer to a ca_ip4_frame_t structure. The out parameter must be a valid, non-NULL pointer to a pointer of type ca_tcp_frame_t. Passing NULL for either parameter will cause the function to return false.

Return conf:0.95 **Q:** What does it mean when ca_net_proto_tcp returns true or false?
A: Returning true means the IP frame contains a valid TCP segment, and the out parameter has been correctly set to point to the TCP frame. Returning false indicates that the IP frame is invalid, the protocol is not TCP, or the packet length is incorrect, or one of the parameters is NULL.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_net_proto_tcp?
A: Common mistakes include passing NULL pointers for ip4 or out, not checking the return value of the function, and calling the function with an IP frame that does not contain a TCP protocol. Additionally, failing to ensure the IP frame is complete and correctly formatted can lead to incorrect behavior.

---

ca_net_proto_tcp_build_ack

Header: caos/net/inc/net_proto.h:576
Source: caos/net/src/net_proto_tcp.c:146:158
GUARD: EN_NET_PROTO

```
u16 ca_net_proto_tcp_build_ack(ca_mac_t dst_mac, ca_ipv4_t dst_ip,
    ca_mac_t src_mac, ca_ipv4_t src_ip, u16 src_port, u16 dst_port, u32
    seq, u32 ack_seq, u16 window, u8 *out_frame, u16 out_max)
```

Constructs a TCP ACK packet header and payload within a provided buffer.

Parametri:

Nome	Tipo	Descrizione
dst_mac	ca_mac_t	Destination MAC address.
dst_ip	ca_ipv4_t	Destination IPv4 address.
src_mac	ca_mac_t	Source MAC address.
src_ip	ca_ipv4_t	Source IPv4 address.
src_port	u16	Source TCP port number.
dst_port	u16	Destination TCP port number.
seq	u32	TCP sequence number for this packet.
ack_seq	u32	TCP acknowledgment number to be sent.
window	u16	TCP receive window size.
out_frame	u8 *	Pointer to the memory buffer where the packet will be built.
out_max	u16	Maximum size of the provided buffer in bytes.

Ritorna:
Returns the total length of the constructed TCP packet in bytes. Returns 0 if the packet could not be built (e.g., buffer too small).

Note:

- This function is non-blocking and performs pure memory operations to format the TCP header. It is thread-safe provided that the 'out_frame' buffer is not accessed by other tasks simultaneously. It is safe to call from an ISR, but ensure the buffer is large enough to avoid memory corruption. A common mistake is providing a buffer smaller than the minimum TCP header size (20 bytes), which will result in a return value of 0.

Uso:

- This function is typically used in the TCP state machine when responding to incoming data or connection requests. It is usually paired with ca_net_buf_alloc to obtain a buffer, then this function builds the ACK, and finally ca_net_send is called to transmit the frame. After transmission, the buffer is typically released using ca_net_buf_free.

Esempio:

```
/* Allocate a network buffer for the response */
u16 idx = ca_net_buf_alloc();
if (idx != NET_BUF_INVALID) {
    ca_net_buf_t *b = ca_net_buf_ptr(idx);
    u8 *out = b->data + NET_BUF_HEADROOM;
    u16 out_max = (u16)(NET_BUF_DATA_SIZE - NET_BUF_HEADROOM);

    /* Build the TCP ACK packet */
    u16 len = ca_net_proto_tcp_build_ack(remote_mac, remote_ip, local_mac,
    local_ip,
                                         local_port, remote_port, seq, ack_seq,
                                         window, out, out_max);
```

```
12
13      if (len > 0U) {
14          /* Send the packet via the network interface */
15          ca_net_send(out, len);
16      }
17      /* Free the buffer after use */
18      ca_net_buf_free(idx);
19  }
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_net_proto_tcp_build_ack in a TCP connection?
A: You should call ca_net_proto_tcp_build_ack to construct an ACK packet when you receive data in a TCP connection and need to acknowledge the receipt of that data. This function is essential for maintaining the reliability of TCP communication by confirming that data has been successfully received.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_net_proto_tcp_build_ack?
A: u8 frame[1024]; mac_t dst_mac = {0x00, 0x0C, 0x29, 0xAB, 0xCD, 0xEF}; ipv4_t dst_ip = {192, 168, 1, 100}; mac_t src_mac = {0x00, 0x0C, 0x29, 0x12, 0x34, 0x56}; ipv4_t src_ip = {192, 168, 1, 1}; u16 result = ca_net_proto_tcp_build_ack(dst_mac, dst_ip, src_mac, src_ip, 12345, 80, 1000, 2000, 1024, frame, 1024); if (result > 0) { // Successfully built the ACK packet }

Parameters conf:0.90 **Q:** What are the valid ranges for the window parameter in ca_net_proto_tcp_build_ack?
A: The window parameter must be a value between 0 and 65535 (inclusive). This represents the size of the receive window in the TCP header, indicating how much data the receiver is willing to accept at once.

Return conf:0.95 **Q:** What does a return value of 0 from ca_net_proto_tcp_build_ack indicate?
A: A return value of 0 indicates an error occurred. This can happen if the output buffer is insufficient, or if one or more of the input parameters are invalid, such as invalid IP or MAC addresses, or an invalid buffer size.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_net_proto_tcp_build_ack?
A: Common mistakes include using incorrect sequence numbers or acknowledgment numbers that do not match the current TCP state, using invalid MAC or IP addresses, or providing a buffer that is too small to hold the constructed packet. These can lead to connection failures or data loss.

`ca_net_proto_tcp_build_data`

Header: `caos/net/inc/net_proto.h:585`
Source: `caos/net/src/net_proto_tcp.c:160:173`
GUARD: `EN_NET_PROTO`

```
1  u16 ca_net_proto_tcp_build_data(ca_mac_t dst_mac, ca_ipv4_t dst_ip,
       ca_mac_t src_mac, ca_ipv4_t src_ip, u16 src_port, u16 dst_port, u32
       seq, u32 ack_seq, u16 window, const u8 *data, u16 data_len, u8 *
       out_frame, u16 out_max)
```

Constructs a TCP segment containing data payload and PSH/ACK flags for transmission.

Parametri:

Nome	Tipo	Descrizione
dst_mac	ca_mac_t	Destination MAC address.
dst_ip	ca_ipv4_t	Destination IPv4 address.
src_mac	ca_mac_t	Source MAC address.
src_ip	ca_ipv4_t	Source IPv4 address.
src_port	u16	Source TCP port number.
dst_port	u16	Destination TCP port number.
seq	u32	TCP sequence number for this segment.
ack_seq	u32	TCP acknowledgment number.
window	u16	TCP receive window size.
data	const u8 *	Pointer to the payload data buffer.
data_len	u16	Length of the payload data in bytes.
out_frame	u8 *	Buffer where the final TCP/IP frame will be written.
out_max	u16	Maximum capacity of the out_frame buffer.

Ritorna:
Returns the total length of the generated TCP frame in bytes. Returns 0 if the frame could not be built (e.g., buffer overflow or invalid parameters).

Note:

- This function is non-blocking and performs pure memory manipulation to format the TCP packet. It is thread-safe provided that the output buffer is not accessed by other tasks simultaneously. It is NOT intended to be called from an ISR because it performs complex packet construction. The most common mistake is providing an 'out_max' value that is smaller than the required TCP/IP header size, which will cause the function to return 0 and fail silently.

Uso:

- This function is used in the data transmission phase of a TCP connection. It is typically called after a network buffer has been allocated via ca_net_buf_alloc and the payload data is ready. After this function returns a valid length, the resulting frame is usually passed to ca_net_send to transmit it over the physical interface. It is part of the low-level protocol stack and is used when you need to manually control the TCP segment flags and sequence numbers.

Esempio:

```
/* Allocate a network buffer for the outgoing packet */
ca_net_buf_t *buf = ca_net_buf_alloc();
if (buf != NULL) {
    u8 *out = ca_net_buf_ptr(buf) + 42; /* Offset for headers */
    u16 max_len = 1460;
    u8 payload[] = "Hello World";

    /* Build the TCP segment with PSH and ACK flags */
    u16 len = ca_net_proto_tcp_build_data(dst_mac, dst_ip, src_mac, src_ip,
                                          80, 1234, 1000, 2000, 512,
                                          payload, 11, out, max_len);

    if (len > 0) {
        /* Send the frame via the network interface */
        ca_net_send(out, len);
    }
    ca_net_buf_free(buf);
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_net_proto_tcp_build_data in a caOS-based embedded system?
A: You should call ca_net_proto_tcp_build_data when you need to construct a TCP packet with payload data to send over the network. It is typically used in TCP service implementations where you are building segments to transmit to a remote host. This function is not meant to be called from an ISR and must be used in contexts where parameters are valid and buffer space is guaranteed.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_net_proto_tcp_build_data?
A: u8 frame[1024]; u16 src_port = 12345; u16 dst_port = 80; u32 seq = 1000; u32 ack_seq = 0; u16 window = 14600; u8 data[] = "GET / HTTP/1.1"; u16 data_len = 15; mac_t dst_mac = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55}; mac_t src_mac = {0x66, 0x77, 0x88, 0x99, 0xAA, 0xBB}; ipv4_t dst_ip = {192, 168, 1, 100}; ipv4_t src_ip = {192, 168, 1, 1};
u16 result = ca_net_proto_tcp_build_data(dst_mac, dst_ip, src_mac, src_ip, src_port, dst_port, seq, ack_seq, window, data, data_len, frame, sizeof(frame));

Parameters conf:0.90 **Q:** What are the valid ranges for the window parameter in ca_net_proto_tcp_build_data
A: The window parameter must be a valid 16-bit unsigned integer in the range 0 to 65535. It represents the size of the receive window in bytes, indicating how much data the receiver is willing to accept at once. Values outside this range are invalid and may cause the function to return 0.

Return conf:0.95 **Q:** What does a return value of 0 from ca_net_proto_tcp_build_data indicate?
A: A return value of 0 indicates that the function failed to build the TCP packet. This can happen if out_frame is NULL, out_max is insufficient, data_len exceeds available space, or any other parameter is invalid. The caller must check this return value to ensure the packet was built correctly.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_net_proto_tcp_build_data?
A: Common mistakes include not checking the return value, using a buffer that is too small for the packet, passing invalid MAC or IP addresses, or using a zero port number. Another frequent error is not ensuring that the seq and ack_seq values are correctly managed for TCP state consistency. The function is also not thread-safe and should not be called from ISRs.

`ca_net_proto_tcp_build_fin`

Header: `caos/net/inc/net_proto.h:595`
Source: `caos/net/src/net_proto_tcp.c:175:187`
GUARD: `EN_NET_PROTO`

```
u16 ca_net_proto_tcp_build_fin(ca_mac_t dst_mac, ca_ipv4_t dst_ip,
    ca_mac_t src_mac, ca_ipv4_t src_ip, u16 src_port, u16 dst_port, u32
    seq, u32 ack_seq, u16 window, u8 *out_frame, u16 out_max)
```

Constructs a TCP FIN packet to initiate a graceful connection termination.

Parametri:

Nome	Tipo	Descrizione
dst_mac	ca_mac_t	Destination MAC address.
dst_ip	ca_ipv4_t	Destination IPv4 address.
src_mac	ca_mac_t	Source MAC address.
src_ip	ca_ipv4_t	Source IPv4 address.
src_port	u16	Source TCP port number.
dst_port	u16	Destination TCP port number.
seq	u32	TCP sequence number for the FIN packet.
ack_seq	u32	TCP acknowledgment number.
window	u16	TCP window size field.
out_frame	u8 *	Pointer to the buffer where the packet will be built. Must be large enough to hold the header.
out_max	u16	Maximum size of the out_frame buffer.

Ritorna:
Returns the total length of the built TCP packet (including headers) on success, or 0 if the packet could not be built (e.g., buffer too small).

Note:

- This function is non-blocking and performs pure memory manipulation to format the TCP header; it does not interact with the network hardware directly. It is thread-safe provided that the output buffer is not accessed by other tasks simultaneously. It is safe to call from an ISR, as it contains no blocking logic or OS primitives. The most common mistake is providing an 'out_max' value that is smaller than the required TCP header size (20 bytes), which will result in a return value of 0 and no packet being generated.

Uso:

- This function is used during the TCP teardown phase when a connection needs to be closed gracefully. It is typically called after a network buffer has been allocated via ca_net_buf_alloc. The developer retrieves the data pointer using ca_net_buf_ptr, calculates the available space, and then calls this function to populate the buffer. After this, the resulting buffer is usually passed to ca_net_send to transmit the FIN packet over the network interface.

Esempio:

```
/* Allocate a network buffer for the FIN packet */
ca_net_buf_t *tb = ca_net_buf_alloc();
if (tb != NULL) {
    u8 *out = ca_net_buf_ptr(tb) + 16; /* Assuming 16 bytes headroom */
    u16 out_max = 128; /* Buffer capacity */

    /* Build the FIN packet */
    u16 len = ca_net_proto_tcp_build_fin(dst_mac, dst_ip, src_mac, src_ip,
                                         80, 12345, 1000, 2000, 512,
                                         out, out_max);

    if (len > 0) {
        /* Send the packet via the network stack */
        ca_net_send(tb, len);
    } else {
        /* Free buffer if build failed */
        ca_net_buf_free(tb);
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use the ca_net_proto_tcp_build_fin function in a caOS-based embedded application?
A: You should call ca_net_proto_tcp_build_fin when you need to send a TCP FIN packet to close an existing TCP connection gracefully. This function is typically used in the TCP service layer or application logic when a connection is being terminated, ensuring proper TCP state management and reliable disconnection.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_net_proto_tcp_build_fin with caOS types?
A: u8 frame[1024]; mac_t dst_mac = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55}; ipv4_t dst_ip = {192, 168, 1, 100}; mac_t src_mac = {0xAA, 0xBB, 0xCC, 0xDD, 0xEE, 0xFF}; ipv4_t src_ip = {192, 168, 1, 1}; u16 len = ca_net_proto_tcp_build_fin(dst_mac, dst_ip, src_mac, src_ip, 12345, 80, 1000, 2000, 14600, frame, 1024); if (len > 0) { // Send frame via network driver }

Parameters conf:0.90 **Q:** What are the valid ranges for the window parameter in ca_net_proto_tcp_build_fin, and what happens if it's invalid?
A: The window parameter must be a value between 0 and 65535 (16-bit unsigned integer). If an invalid value is provided, the function will likely return 0, indicating an error. The window size represents the amount of data the receiver is willing to accept at once, and must be within standard TCP limits.

Return conf:0.95 **Q:** What does a return value of 0 from ca_net_proto_tcp_build_fin indicate, and what are the possible causes?
A: A return value of 0 indicates an error occurred during packet construction. Common causes include invalid parameters such as an invalid MAC or IP address, a buffer that is too small (out_max less than required), or inconsistent TCP state values like incorrect sequence or acknowledgment numbers.

Error/Edge case conf:0.85 **Q:** What are the most common mistakes developers make when calling ca_net_proto_tcp_build_fin?
A: Common mistakes include providing a buffer 'out_frame' that is too small (out_max smaller than required), passing invalid MAC or IP addresses, using incorrect sequence or acknowledgment numbers that break TCP state consistency, or calling the function from an ISR context, which is explicitly forbidden. Also, failing to ensure that the TCP connection state is properly managed before sending a FIN packet.

`ca_net_proto_tcp_build_rst`

Header: `caos/net/inc/net_proto.h:604`
Source: `caos/net/src/net_proto_tcp.c:205:217`
GUARD: `EN_NET_PROTO`

```
u16 ca_net_proto_tcp_build_rst(ca_mac_t dst_mac, ca_ipv4_t dst_ip,
    ca_mac_t src_mac, ca_ipv4_t src_ip, u16 src_port, u16 dst_port, u32
    seq, u8 *out_frame, u16 out_max)
```

Constructs a TCP RST (Reset) packet to immediately terminate a connection.

Parametri:

Nome	Tipo	Descrizione
dst_mac	ca_mac_t	Destination MAC address.
dst_ip	ca_ipv4_t	Destination IPv4 address.
src_mac	ca_mac_t	Source MAC address (local interface).
src_ip	ca_ipv4_t	Source IPv4 address (local interface).
src_port	u16	Source TCP port.
dst_port	u16	Destination TCP port.
seq	u32	Sequence number for the RST packet.
out_frame	u8 *	Pointer to the buffer where the packet will be built.
out_max	u16	Maximum size of the provided buffer.

Ritorna:
Returns the total length of the generated TCP packet in bytes. Returns 0 if the packet could not be built (e.g., buffer too small).

Note:

- This function is non-blocking and performs no memory allocation itself; it simply formats the packet header into the provided memory buffer. It is thread-safe provided that the 'out_frame' buffer is not accessed by other tasks simultaneously. It is safe to call from an ISR, but ensure the buffer is large enough to avoid memory corruption. A common mistake is passing an 'out_max' value smaller than the minimum TCP header size (20 bytes), which will result in a return value of 0.

Uso:

- This function is used in TCP stack implementations to gracefully or forcefully reject incoming connection attempts or to terminate existing sessions when an error occurs. It is typically used in a sequence: first, allocate a network buffer using ca_net_buf_alloc, then call this function to populate the buffer, and finally transmit the packet using ca_net_send. If the build fails (returns 0), the allocated buffer must be freed using ca_net_buf_free to prevent memory leaks.

Esempio:

```
/* Example: Sending a TCP RST in response to an invalid connection attempt */
ca_net_buf_t *tb = ca_net_buf_alloc();
if (tb != NULL) {
    u8 *out = ca_net_buf_ptr(tb) + 32; /* Offset for headers */
    u16 out_max = 128; /* Available buffer size */
    u16 len = ca_net_proto_tcp_build_rst(dst_mac, dst_ip, src_mac, src_ip, 80,
    12345, 0x12345678, out, out_max);
    if (len > 0) {
        /* Set buffer length and send */
        tb->data_len = len;
        ca_net_send(tb);
    } else {
        /* Build failed, release buffer */
        ca_net_buf_free(tb);
    }
}
```

Q&A — fine-tuning review

5/5 pairs · qwen3-coder:latest

Usage conf:0.95 **Q:** When should I use ca_net_proto_tcp_build_rst to send a TCP RST packet?
A: You should call ca_net_proto_tcp_build_rst when you need to forcefully terminate a TCP connection in a non-standard way, such as when a connection is in an invalid state or when you want to reject an incoming connection. It is typically used in TCP state machines or connection management modules to send a reset packet to a peer.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_net_proto_tcp_build_rst in C with caOS types?
A: u8 frame[64]; mac_t dst_mac = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55}; ipv4_t dst_ip = {192, 168, 1, 100}; mac_t src_mac = {0xAA, 0xBB, 0xCC, 0xDD, 0xEE, 0xFF}; ipv4_t src_ip = {192, 168, 1, 1}; u16 len = ca_net_proto_tcp_build_rst(dst_mac, dst_ip, src_mac, src_ip, 12345, 80, 1000, frame, 64); if (len > 0) { // Send frame via network driver }

Parameters conf:0.85 **Q:** What are the valid values for the src_port and dst_port parameters in ca_net_proto_tcp_build_rst?
A: The src_port and dst_port must be valid TCP port numbers, typically in the range 0-65535. The src_port should be a port that is either open or reserved on the local device, and dst_port should be a port that is currently active or in listening state on the remote device. Invalid ports may cause the packet to be ignored by the receiving end.

Return conf:0.95 **Q:** What does a return value of 0 from ca_net_proto_tcp_build_rst indicate?
A: A return value of 0 indicates an error occurred. This can happen if the output buffer 'out_frame' is not large enough (less than 20 bytes), or if any of the input parameters are invalid, such as invalid MAC or IP addresses, or an invalid port number.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_net_proto_tcp_build_rst?
A: Common mistakes include not checking the return value of the function, which can lead to sending malformed packets; using invalid MAC or IP addresses; providing a buffer that is too small; calling the function from an ISR (Interrupt Service Routine), which is not supported; and not ensuring thread safety if used in a multi-threaded environment.

`ca_net_proto_tcp_build_syn`

Header: `caos/net/inc/net_proto.h:567`
Source: `caos/net/src/net_proto_tcp.c:129:144`
GUARD: `EN_NET_PROTO`

```
u16 ca_net_proto_tcp_build_syn(ca_mac_t dst_mac, ca_ipv4_t dst_ip,
    ca_mac_t src_mac, ca_ipv4_t src_ip, u16 src_port, u16 dst_port, u32
    seq, u16 mss, u8 *out_frame, u16 out_max)
```

Constructs a TCP SYN packet header with MSS options into a provided buffer.

Parametri:

Nome	Tipo	Descrizione
`dst_mac`	`ca_mac_t`	Destination MAC address.
`dst_ip`	`ca_ipv4_t`	Destination IPv4 address.
`src_mac`	`ca_mac_t`	Source MAC address.
`src_ip`	`ca_ipv4_t`	Source IPv4 address.
`src_port`	`u16`	Source TCP port number.
`dst_port`	`u16`	Destination TCP port number.
`seq`	`u32`	Initial sequence number for the connection.
`mss`	`u16`	Maximum Segment Size value to advertise.
`out_frame`	`u8 *`	Pointer to the buffer where the packet will be built.
`out_max`	`u16`	Maximum size of the provided buffer in bytes.

Ritorna:
Returns the total length of the generated TCP packet in bytes. Returns 0 if the packet could not be built (e.g., buffer too small or invalid parameters).

Note:

- This function is a pure packet builder; it does not perform any network I/O or blocking operations, making it safe to call from any context, including ISRs. It is not thread-safe in the sense that it does not lock the buffer, so ensure the buffer is not being accessed by other tasks simultaneously. The most common mistake is providing an 'out_max' value that is smaller than the required TCP/IP header size, which will result in a return value of 0 and a failed packet construction.

Uso:

- This function is used during the initial phase of a TCP connection handshake. It is typically called after allocating a network buffer using ca_net_buf_alloc. The sequence is: allocate a buffer, call this function to populate the header, set the buffer's data length, and finally pass the buffer to ca_net_send to transmit the packet over the network interface.

Esempio:

```
/* Allocate a network buffer for the outgoing SYN packet */
u16 idx = ca_net_buf_alloc();
if (idx != 0xFFFF) {
    ca_net_buf_t *b = ca_net_buf_ptr(idx);
    u8 *out = b->data + 64; /* Assuming 64 bytes of headroom */
    u16 out_max = 1500 - 64;

    /* Build the TCP SYN packet */
    u16 len = ca_net_proto_tcp_build_syn(dst_mac, dst_ip, src_mac, src_ip, 1234,
     80, 0x12345678, 1460, out, out_max);

    if (len > 0) {
        b->data_len = len;
        /* Send the packet via the network stack */
        ca_net_send(idx);
    } else {
        /* Cleanup if building failed */
        ca_net_buf_free(idx);
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_net_proto_tcp_build_syn in a caOS-based embedded system?
A: You should call ca_net_proto_tcp_build_syn when you need to initiate a TCP connection by sending a TCP SYN packet. This function is typically used in the TCP handshake process when a client wants to establish a connection with a server.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_net_proto_tcp_build_syn with caOS types?
A: u8 frame[1500]; u16 src_port = 12345; u16 dst_port = 80; u32 seq = 0x12345678; u16 mss = 1460; mac_t dst_mac = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55}; mac_t src_mac = {0x66, 0x77, 0x88, 0x99, 0xAA, 0xBB}; ipv4_t dst_ip = {192, 168, 1, 100}; ipv4_t src_ip = {192, 168, 1, 1}; u16 len = ca_net_proto_tcp_build_syn(dst_mac, dst_ip, src_mac, src_ip, src_port, dst_port, seq, mss, frame, sizeof(frame));

Parameters conf:0.95 **Q:** What are the valid ranges for the src_port and dst_port parameters in ca_net_proto_tcp_build_syn?
A: Both src_port and dst_port must be valid TCP port numbers in the range 1-65535. Port 0 is not a valid port number and will cause the function to return 0.

Return conf:0.95 **Q:** What does it mean when ca_net_proto_tcp_build_syn returns 0?
A: A return value of 0 indicates an error occurred. This can happen due to invalid parameters such as NULL MAC addresses, zero IP addresses, invalid port numbers, or an insufficiently sized output buffer (out_max $<$ 40).

Error/Edge case conf:0.90 **Q:** What are common mistakes when calling ca_net_proto_tcp_build_syn?
A: Common mistakes include passing a buffer that is too small (out_max < 40), using invalid IP addresses (0.0.0.0), passing invalid port numbers (0 or > 65535), or providing NULL MAC addresses. Also, using an MSS value outside the range 536-1460 may cause issues.

Behavior conf:0.90 **Q:** Is ca_net_proto_tcp_build_syn safe to call from an ISR or is it thread-safe?
A: No, ca_net_proto_tcp_build_syn is not safe to call from an ISR because it may use non ISR-safe functions. However, it is thread-safe if the input and output buffers are not shared between threads.

`ca_net_proto_tcp_build_synack`

Header: `caos/net/inc/net_proto.h:617`
Source: `caos/net/src/net_proto_tcp.c:189:203`
GUARD: `EN_NET_PROTO`

```
u16 ca_net_proto_tcp_build_synack(ca_mac_t dst_mac, ca_ipv4_t dst_ip,
    ca_mac_t src_mac, ca_ipv4_t src_ip, u16 src_port, u16 dst_port, u32
    seq, u32 ack_seq, u16 mss, u8 *out_frame, u16 out_max)
```

Constructs a TCP SYN-ACK packet with MSS options for the TCP handshake process.

Parametri:

Nome	Tipo	Descrizione
`dst_mac`	`ca_mac_t`	Destination MAC address.
`dst_ip`	`ca_ipv4_t`	Destination IPv4 address.
`src_mac`	`ca_mac_t`	Source MAC address.
`src_ip`	`ca_ipv4_t`	Source IPv4 address.
`src_port`	`u16`	Local TCP port.
`dst_port`	`u16`	Remote TCP port.
`seq`	`u32`	Sequence number for this packet.
`ack_seq`	`u32`	Acknowledgment number to be sent back.
`mss`	`u16`	Maximum Segment Size value to advertise.
`out_frame`	`u8 *`	Pointer to the buffer where the packet will be built.
`out_max`	`u16`	Maximum size of the provided buffer.

Ritorna:
Returns the total length of the generated TCP packet in bytes. Returns 0 if the packet could not be built (e.g., buffer too small or invalid parameters).

Note:

- This function is non-blocking and performs pure data manipulation; it does not interact with hardware directly, making it safe to call from most contexts. It is not thread-safe if the 'out_frame' buffer is shared across tasks without external synchronization. The most common mistake is providing a buffer that is too small for the TCP header plus the MSS option (24 bytes minimum), which will result in a return value of 0. Ensure the buffer has sufficient headroom for lower-level headers (Ethernet/IP) before passing the pointer.

Uso:

- This function is used exclusively during the TCP connection establishment phase (the 'three-way handshake'). It is typically called by the TCP state machine after receiving a SYN packet from a remote host. The typical sequence is: allocate a network buffer using

ca_net_buf_alloc, calculate the required offsets, call this function to populate the buffer, and finally pass the buffer to ca_net_send to transmit it over the network interface.

Esempio:

```
/* Assuming 'p' is a pointer to a TCP connection control block */
u16 idx = ca_net_buf_alloc();
if (idx != 0xFFFFU) {
    ca_net_buf_t *b = ca_net_buf_ptr(idx);
    u8 *out = b->data + 34U; /* 34 bytes offset for Ethernet+IP headers */
    u16 out_max = 1500U;
    u16 len = ca_net_proto_tcp_build_synack(p->remote_mac, p->remote_ip, p->
    local_mac, p->local_ip, p->local_port, p->remote_port, p->snd_nxt, p->
    rcv_nxt, p->mss, out, out_max);
    if (len > 0U) {
        /* Set buffer length and transmit */
        b->data_off = 34U;
        ca_net_send(idx, len);
    } else {
        ca_net_buf_free(idx);
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_net_proto_tcp_build_synack in a TCP connection setup?
A: You should call ca_net_proto_tcp_build_synack when you are responding to an incoming TCP SYN packet to establish a connection. It constructs the SYN-ACK packet that acknowledges the connection request and initiates the three-way handshake.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_net_proto_tcp_build_synack?
A: u8 frame[1024]; mac_t dst_mac = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55}; ipv4_t dst_ip = {192, 168, 1, 100}; mac_t src_mac = {0xAA, 0xBB, 0xCC, 0xDD, 0xEE, 0xFF}; ipv4_t src_ip = {192, 168, 1, 1}; u16 bytes = ca_net_proto_tcp_build_synack(dst_mac, dst_ip, src_mac, src_ip, 80, 12345, 1000, 2000, 1460, frame, 1024);

Parameters conf:0.90 **Q:** What is the significance of the MSS parameter in ca_net_proto_tcp_build_synack and what values are valid?
A: The MSS (Maximum Segment Size) parameter defines the largest amount of data that can be transmitted in a single TCP segment. Valid values are typically between 536 and 1460 bytes, as specified in the function documentation.

Return conf:0.95 **Q:** What does it mean if ca_net_proto_tcp_build_synack returns 0?
A: A return value of 0 indicates an error, either because the output buffer 'out_frame' is too small or one of the input parameters is invalid, such as an invalid IP address, MAC address, or port number.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_net_proto_tcp_build_synack?
A: Common mistakes include passing an insufficiently sized buffer for 'out_frame', providing invalid IP or MAC addresses, using invalid port numbers (outside 0-65535), or specifying an invalid MSS value outside the range of 536-1460 bytes.

`ca_net_proto_udp`

Header: `caos/net/inc/net_proto.h:248`
Source: `caos/net/src/net_proto.c:238:253`
GUARD: `EN_NET_PROTO`

```
bool ca_net_proto_udp(const ca_ip4_frame_t *ip4, const ca_udp_frame_t **
    out)
```

Validates an IPv4 frame as a UDP packet and extracts the UDP header pointer.

Parametri:

Nome	Tipo	Descrizione
ip4	const ca_ip4_frame_t *	Pointer to the incoming IPv4 frame structure. Must not be NULL.
out	const ca_udp_frame_t **	Pointer to a pointer that will be updated to point to the UDP header within the frame if validation succeeds. Must not be NULL.

Ritorna:
Returns true if the frame is a valid UDP packet (protocol matches, length is consistent, and header is present). Returns false if the input is invalid, the protocol is not UDP, or the packet is malformed (e.g., length mismatch).

Note:

- This function is non-blocking and thread-safe as it performs only read-only validation on existing memory buffers. It is safe to call from an ISR because it does not perform any heap allocation, locking, or complex OS operations. The primary precondition is that the 'ip4' pointer must point to a valid, fully received network buffer. A common mistake is failing to check the return value before accessing the 'out' pointer, which leads to dereferencing an uninitialized or NULL pointer if the packet was not actually UDP.

Uso:

- This function is typically used in the network receive path after the L2/L3 layers have successfully processed the frame. It is used to 'cast' the generic IP payload into a structured UDP header so that the application can access source/destination ports and payload data. It is usually paired with ca_net_proto_ip4 for initial frame validation and is followed by application-specific logic to parse the UDP payload (e.g., NTP or DNS parsing).

Esempio:

```c
void process_incoming_packet(const ca_ip4_frame_t *ip4) {
    const ca_udp_frame_t *udp = NULL;

    /* Validate that the IP packet contains a valid UDP header */
    if (ca_net_proto_udp(ip4, &udp)) {
        /* Successfully validated, now safe to access UDP fields */
        if (udp->dst_port == 123U) {
            /* Logic to handle NTP traffic */
            /* ca_net_proto_ntp_parse_response(udp, ...); */
        }
    } else {
        /* Not a UDP packet or malformed, ignore or log error */
        ca_net_count_err();
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_net_proto_udp in a network stack implementation?
A: You should call ca_net_proto_udp when you have received an IP4 frame and want to validate if it contains a valid UDP payload. It is typically used after receiving a packet from the network interface to determine if the packet is a UDP packet and to get a pointer to the UDP header for further processing.

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_net_proto_udp with caOS types?

A: ca_ip4_frame_t ip4_frame; ca_udp_frame_t *udp_frame; bool result = ca_net_proto_udp(&ip4_ &udp_frame); if (result) { // Process UDP frame u16 udp_length = udp_frame->length; }

Parameters conf:0.95 **Q:** What are the valid and invalid values for the ip4 and out parameters?
A: The ip4 parameter must be a valid, non-NULL pointer to a ca_ip4_frame_t structure. If it is NULL or points to invalid data, the function returns false. The out parameter must be a valid, non-NULL pointer to a pointer to ca_udp_frame_t. If it is NULL, the function returns false.

Return conf:0.95 **Q:** What does a return value of true or false indicate in ca_net_proto_udp?
A: A return value of true indicates that the IP4 frame contains a valid UDP packet and the out pointer has been correctly set to point to the UDP frame. A return value of false indicates that either the IP4 frame is invalid, does not contain a UDP packet, or one of the input parameters is NULL.

Error/Edge case conf:0.90 **Q:** What are common mistakes or edge cases when using ca_net_proto_udp?
A: Common mistakes include passing NULL pointers for ip4 or out, not checking the return value of the function, and assuming that the frame pointed to by ip4 is complete and valid. An edge case is when the IP4 frame's protocol field is not set to IP_PROTO_UDP, which will cause the function to return false.

Behavior conf:0.95 **Q:** Is ca_net_proto_udp blocking, ISR-safe, and thread-safe?
A: ca_net_proto_udp is not blocking and executes immediately. It is thread-safe because it does not modify shared data and only uses input parameters. It is also ISR-safe since it does not perform operations that could cause issues in interrupt contexts such as dynamic memory allocation.

`ca_net_queue_empty`

Header: `caos/net/inc/net_queue.h:43`
Source: `caos/net/src/net_queue.c:47:50`
GUARD: `EN_NET_QUEUE`

```
1 bool ca_net_queue_empty(const ca_net_queue_t *q)
```

Checks if the network queue is empty by comparing head and tail pointers.

Parametri:

Nome	Tipo	Descrizione
`q`	`const ca_net_queue_t *`	Pointer to the network queue structure to check. Must be a valid, initialized queue.

Ritorna:
Returns true if the queue is empty (head == tail), false otherwise.

Note:

- This function is non-blocking and executes in constant time, making it safe to call from both tasks and ISRs. It does not perform any locking, so if the queue is being modified by another task or ISR simultaneously, the result might be stale or inconsistent. Ensure the queue has been properly initialized with ca_net_queue_init before calling this. A common mistake is assuming that checking for empty is sufficient to guarantee a subsequent ca_net_queue_pop will succeed; in a multi-threaded environment, another task could pop the last item between your check and your action.

Uso:

- This function is typically used as a guard condition before attempting to process data from the network queue. It is often paired with ca_net_queue_pop to ensure that you do not attempt to read from an empty buffer, which would lead to undefined behavior or logic errors. In a typical network processing loop, you would call this to decide whether to enter a processing state or to put the task to sleep using ca_os_sleep to save CPU cycles.

Esempio:

```
/* Assuming net_q is a pre-initialized ca_net_queue_t */
if (!ca_net_queue_empty(&net_q)) {
    /* Queue has data, proceed to extract it */
    void *data = ca_net_queue_pop(&net_q);
    if (data != NULL) {
        /* Process the network data here */
        /* ... */
        ca_net_buf_free(data);
    }
} else {
    /* Queue is empty, wait for more data or perform other tasks */
    ca_os_delay_ms(10);
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_net_queue_empty() in a network application?
A: You should call ca_net_queue_empty() before attempting to read data from a network queue to check if there are any pending packets. This helps avoid unnecessary operations or potential errors when the queue is empty. It's particularly useful in polling-based network handlers where you want to process incoming data only when available.

Code example conf:0.90 **Q:** Can you show a minimal example of using ca_net_queue_empty() with caOS types?
A: ca_net_queue_t queue; bool is_empty; // Assume queue is initialized is_empty = ca_net_queue_empty(&queue); if (is_empty) { // No data to process } else { // Data is available for reading }

Parameters conf:0.90 **Q:** What does the 'q' parameter in ca_net_queue_empty() represent and what are valid values?
A: The 'q' parameter is a pointer to a ca_net_queue_t structure that represents the network queue to be checked. It must be a valid, non-NULL pointer to an already initialized queue. Passing NULL will cause a crash. The queue must have been properly initialized before calling this function.

Return conf:0.85 **Q:** What do the return values of ca_net_queue_empty() mean and what causes failures?
A: The function returns true if the queue is empty (no data to read), and false if the queue contains data. Failures occur if the input pointer 'q' is NULL, which causes a crash. The function itself does not return error codes; it only indicates the queue state. In multi-threaded contexts, inconsistent results may occur if the queue is modified by another thread without proper synchronization.

`ca_net_queue_full`

Header: `caos/net/inc/net_queue.h:42`
Source: `caos/net/src/net_queue.c:42:45`
GUARD: `EN_NET_QUEUE`

```
bool ca_net_queue_full (const ca_net_queue_t *q)
```

Checks if the network queue is full by comparing the tail and head pointers.

Parametri:

Nome	Tipo	Descrizione
q	const ca_net_queue_t *	Pointer to the network queue structure. Must be initialized via ca_net_queue_init before use.

Ritorna:
Returns true if the queue is full (no more space to push), false otherwise.

Note:

- This function is non-blocking and performs a simple arithmetic check on the queue pointers. It is thread-safe in the sense that it performs a read-only operation, but it does not provide atomicity; if another task or ISR modifies the queue pointers simultaneously, the result might be stale. It is safe to call from an ISR. The most common mistake is assuming that a 'false' return guarantees that a subsequent ca_net_queue_push will succeed, which is not true in multi-threaded environments where another task might fill the queue between the check and the push.

Uso:

- This function is typically used as a guard before attempting to add data to a network queue. In a standard producer-consumer pattern, you check if the queue is full before calling ca_net_queue_push to avoid buffer overflows or data loss. It is often paired with ca_net_queue_empty to monitor the queue's health or to decide when to trigger a processing task. Always ensure the queue handle is valid and initialized before calling this function to avoid memory access violations.

Esempio:

```
ca_net_queue_t my_net_queue;

// Initialize the queue before use
ca_net_queue_init(&my_net_queue);

// Check if there is space before pushing data
if (!ca_net_queue_full(&my_net_queue)) {
    // Proceed to push data into the queue
    // ca_net_queue_push(&my_net_queue, data);
} else {
    // Handle the overflow condition, e.g., log error or drop packet
    ca_printf("Network queue is full, dropping packet!\n");
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_net_queue_full() in a network application?
A: You should call ca_net_queue_full() before attempting to insert data into a network queue to avoid overflows. It's useful in scenarios where you want to prevent writing data when the queue is at maximum capacity, ensuring that your application does not lose packets or cause undefined behavior due to buffer overflows.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_net_queue_full() with caOS types?
A: ca_net_queue_t queue; ca_net_queue_init(&queue, 10); if (ca_net_queue_full(&queue)) { // Handle full queue case } else { // Insert data into queue }

Parameters conf:0.95 **Q:** What is the significance of the 'q' parameter in ca_net_queue_full() and what happens if it's invalid?

A: The 'q' parameter is a pointer to the network queue to be checked. It must be a valid, non-NULL pointer. If passed a NULL pointer, the function will cause a crash because it dereferences the pointer without checking for validity. The queue must also be properly initialized before calling this function.

Return conf:0.85 **Q:** What do the return values of ca_net_queue_full() indicate and what can cause the function to fail?

A: The function returns true if the queue is full and false if there is space available. Failures are not explicitly defined in the documentation, but the function can crash if passed a NULL pointer or if the queue structure is corrupted. The function does not perform any error handling beyond checking for NULL input.

`ca_net_queue_init`

Header: `caos/net/inc/net_queue.h:34`
Source: `caos/net/src/net_queue.c:15:19`
GUARD: `EN_NET_QUEUE`

```
void ca_net_queue_init(ca_net_queue_t *q)
```

Initializes a network queue structure by resetting its head and tail pointers to zero.

Parametri:

Nome	Tipo	Descrizione
`q`	`ca_net_queue_t *`	Pointer to the network queue structure to be initialized. Must be a valid, allocated memory address.

Ritorna:
void

Note:

- This function is non-blocking and performs a simple memory write operation, making it safe to call from both tasks and ISRs. It is not thread-safe in the sense that it does not use locks, but since it only resets indices, it is intended to be called during the system initialization phase before the queue is actively used by producers or consumers. A common beginner mistake is calling this function while the queue is already in use, which will immediately discard all pending data and corrupt the queue state.

Uso:

- This function is used during the network stack setup phase to prepare a queue for operation. It is typically called once during system boot, alongside other initialization functions like ca_net_buf_pool_init and ca_sem_init. It must be called before any calls to ca_net_queue_push or ca_net_queue_pop are made to ensure the queue starts in a clean, empty state.

Esempio:

```
/* Define the queue structure globally */
static ca_net_queue_t s_rx_q;

void network_setup(void) {
    /* Initialize the buffer pool first */
    ca_net_buf_pool_init();

    /* Initialize the queue indices to 0 */
    ca_net_queue_init(&s_rx_q);
```

```
10
11      /* Now the queue is ready for use by ISRs or tasks */
12      /* Example: ca_net_queue_push(&s_rx_q, buffer_index); */
13  }
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_net_queue_init and why is it important to initialize the network queue before using other queue functions?
A: You should call ca_net_queue_init before using any other network queue functions such as push or pop. It initializes the queue by setting the head and tail pointers to zero, preparing the queue for use. This function must be called on a valid, non-NULL queue structure pointer before any other operations on that queue can be performed.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to declare and initialize a network queue using caOS types?
A: ca_net_queue_t my_queue; ca_net_queue_init(&my_queue);

Parameters conf:0.85 **Q:** What is the significance of the ca_net_queue_t *q parameter in ca_net_queue_init, and what happens if it is NULL?
A: The ca_net_queue_t *q parameter is a pointer to the network queue structure that needs to be initialized. It must be a valid pointer to an allocated queue structure. If this parameter is NULL, the function will not fail but will likely cause a system crash or undefined behavior since it attempts to dereference a null pointer.

`ca_net_queue_pop`

Header: `caos/net/inc/net_queue.h:40`
Source: `caos/net/src/net_queue.c:33:40`
GUARD: `EN_NET_QUEUE`

```
1  bool ca_net_queue_pop(ca_net_queue_t *q, u16 *idx)
```

Removes the oldest item from a network queue and returns its index.

Parametri:

Nome	Tipo	Descrizione
`q`	`ca_net_queue_t *`	Pointer to the initialized network queue structure.
`idx`	`u16 *`	Pointer to a variable where the retrieved index will be stored. Must not be NULL.

Ritorna:
Returns true if an item was successfully retrieved, false if the queue was empty.

Note:

- This function is non-blocking and designed for high-performance data consumption. It is thread-safe for a single-producer/single-consumer scenario due to the use of memory barriers (___DMB), but it is NOT safe to call from an ISR if the producer is also an ISR. The queue must be initialized via ca_net_queue_init before use. A common mistake is calling this function without checking the return value, which leads to processing uninitialized or stale data in the 'idx' variable.

Uso:

- This function is typically used in a consumer task loop to drain a queue filled by an interrupt or a peripheral driver. It is usually paired with ca_net_queue_push (used by the producer) and ca_net_queue_empty (to check status). In a typical network stack,

you call this in a loop until it returns false to ensure all pending frames are processed before the task yields or sleeps.

Esempio:

```
/* Example: Processing pending network frames from a queue */
ca_net_queue_t my_rx_queue;
u16 frame_idx;

/* Initialize the queue before use */
ca_net_queue_init(&my_rx_queue);

/* Consumer task loop */
while (1) {
    /* Attempt to pop an index from the queue */
    if (ca_net_queue_pop(&my_rx_queue, &frame_idx)) {
        /* Process the frame using the retrieved index */
        /* net_handle_frame(frame_idx); // Hypothetical processing function */
    } else {
        /* Queue is empty, wait for more data or yield */
        ca_os_delay_ms(1);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_net_queue_pop in a caOS-based network application?
A: You should call ca_net_queue_pop when you need to retrieve an index from a network queue to process incoming data or handle network events. It is useful in scenarios where you want to check for available network data without blocking, such as in a polling loop or within a task that handles network packets.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_net_queue_pop with caOS types?
A: ca_net_queue_t queue; ca_net_queue_init(&queue); // ... add elements to queue ... u16 index; if (ca_net_queue_pop(&queue, &index)) { // Process the index // index now contains the extracted value } else { // Queue was empty }

Parameters conf:0.90 **Q:** What are the valid and invalid values for the parameters of ca_net_queue_pop?
A: The parameter 'q' must be a valid pointer to an initialized ca_net_queue_t structure; otherwise, it will cause a crash or undefined behavior. The parameter 'idx' must be a valid pointer to a u16 variable where the extracted index will be stored; otherwise, it may cause memory access violations.

Return conf:0.95 **Q:** What does the return value of ca_net_queue_pop indicate, and what causes it to return false?
A: The function returns true if an element was successfully extracted from the queue and stored in *idx. It returns false if the queue is empty, indicating no elements are available for extraction. The function is non-blocking, so it does not wait for new elements to arrive.

`ca_net_queue_push`

Header: `caos/net/inc/net_queue.h:37`
Source: `caos/net/src/net_queue.c:22:30`
GUARD: `EN_NET_QUEUE`

```
bool ca_net_queue_push(ca_net_queue_t *q, u16 idx)
```

Adds a network buffer index to the tail of a circular queue, typically used in ISRs.

Parametri:

Nome	Tipo	Descrizione
q	ca_net_queue_t *	Pointer to the network queue structure. Must be initialized via ca_net_queue_init before use.
idx	u16	The index of the network buffer to store. Must be a valid index within the pool range.

Ritorna:
Returns true if the index was successfully added to the queue. Returns false if the queue is full (next position equals head).

Note:

- This function is designed to be thread-safe for a single-producer (ISR) and single-consumer (task) pattern. It is non-blocking, meaning it returns immediately if the queue is full, which is critical for ISR performance. The use of ___DMB() (Data Memory Barrier) ensures that the buffer data is fully written to memory before the tail pointer is updated, preventing the consumer from reading stale data. A common mistake is calling this function without checking the return value; if it returns false, the buffer index is lost, and you must manually free the buffer to prevent memory leaks.

Uso:

- This function is the standard way to pass received network packet indices from an Ethernet ISR to the processing task. It is typically paired with ca_net_queue_pop in the processing task and ca_sem_post_capped to notify the task that new data is available. The typical sequence is: 1. Allocate a buffer with ca_net_buf_alloc, 2. Fill the buffer, 3. Push the index with ca_net_queue_push, 4. Signal the task with ca_sem_post_capped.

Esempio:

```
/* Example: Pushing a received packet index from an ISR */
void ethernet_rx_isr(void) {
    u16 buf_idx = /* Get index from hardware register */ 0;

    /* Attempt to push the buffer index to the RX queue */
    if (ca_net_queue_push(&s_rx_q, buf_idx)) {
        /* Notify the network task that a new packet is ready */
        ca_sem_post_capped(&ca_net_rx_sem);
    } else {
        /* Queue is full, drop the packet to avoid system stall */
        ca_net_buf_free(buf_idx);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_net_queue_push() in an embedded system?
A: You should use ca_net_queue_push() to pass data from an ISR (Interrupt Service Routine) to a task in a bare-metal RTOS like caOS. It is designed for high-performance, low-latency communication between interrupt context and task context, where you need to queue up events or data without blocking the ISR execution.

Code example conf:0.85 **Q:** Can you show a minimal example of how to use ca_net_queue_push() with caOS types?
A: ca_net_queue_t queue; ca_net_queue_init(&queue, 10); // In ISR: if (!ca_net_queue_push(&queu 0x1234)) { // Handle queue full condition } // In task: while (ca_net_queue_pop(&queue, &idx)) { // Process idx }

Parameters conf:0.90 **Q:** What does the idx parameter in ca_net_queue_push() represent and what values are valid?

A: The idx parameter is a 16-bit value that represents the data to be inserted into the queue. It can be any meaningful 16-bit integer value depending on the application context, such as a packet ID or event code. Invalid values include those that are not meaningful for the system's use case, but the function itself does not validate the semantic meaning of the value.

Return conf:0.95 **Q:** What do the return values of ca_net_queue_push() indicate and what causes it to return false?

A: The function returns true if the element was successfully inserted into the queue. It returns false if the queue is full, indicating that no more elements can be added at that moment. This is a normal behavior in high-concurrency systems and should be handled gracefully by the caller to avoid data loss.

`ca_net_register`

Header: `caos/net/inc/net.h:115`
Source: `caos/net/src/net.c:248:262`
GUARD: `EN_NET`

```
void ca_net_register(u16 ethertype, ca_net_handler_t fn)
```

Registers a callback function to handle incoming network packets for a specific EtherType.

Parametri:

Nome	Tipo	Descrizione
`ethertype`	`u16`	The 16-bit protocol identifier (e.g., 0x0806 for ARP, 0x0800 for IPv4) that triggers the handler.
`fn`	`ca_net_handler_t`	A pointer to the function that will process the packet. Must match the expected signature for network handlers.

Ritorna:
void. This function does not return a status code.

Note:

- This function is not thread-safe and should be called during the system initialization phase before the network stack starts processing packets. It is not intended to be called from an ISR, as it modifies a shared static array without internal locking. A common mistake is registering a handler for an EtherType that is already handled, which will silently overwrite the previous function pointer; always ensure your protocol registration is unique or intentional.

Uso:

- This function is the glue between the raw network dispatcher and your protocol implementation. It is typically called once during the system boot sequence, immediately after initializing the network stack with ca_net_init and before calling ca_net_process. By registering your handler, you tell the caOS network core: 'Whenever you see a packet with this specific EtherType, jump to my function to handle it.'

Esempio:

```
/* Define a custom handler for a hypothetical protocol */
void my_custom_handler(u16 buf_idx) {
    /* Process the packet buffer using ca_net_buf_ptr */
    /* ... logic here ... */
    ca_net_buf_free(buf_idx);
```

```
}

void app_init(void) {
    /* Initialize network stack first */
    ca_net_init();

    /* Register the handler for a specific EtherType (e.g., 0x88B5) */
    /* This ensures the dispatcher knows where to route the data */
    ca_net_register(0x88B5, my_custom_handler);
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_net_register and why is it important for network packet handling?
A: You should call ca_net_register when you want to handle specific types of Ethernet packets (like IP or ARP) in your embedded application. It allows the system to route incoming packets to the correct handler function based on the ethertype field in the Ethernet frame. This is essential for implementing custom network protocols or processing specific packet types in a bare-metal environment like caOS.

Code example conf:0.90 **Q:** Can you provide a minimal working example of how to use ca_net_register with a custom handler function?
A: u16 my_handler(u16 buf_idx) { // Handle packet in buffer buf_idx return 0; } void init_network(void) { ca_net_register(0x0800, my_handler); // Register for IP packets }

Parameters conf:0.85 **Q:** What are the valid values for the ethertype parameter, and what happens if I pass an invalid value?
A: The ethertype parameter should be a valid Ethernet protocol type such as 0x0800 for IPv4 or 0x0806 for ARP. If an invalid or unsupported ethertype is passed, the packet will not be correctly routed to your handler function, and the system may ignore it. The function itself does not validate the ethertype, so invalid values can lead to unexpected behavior.

Return conf:0.90 **Q:** Does ca_net_register return any value, and what does it mean if the function is called multiple times with the same ethertype?
A: The function does not return any value. If called multiple times with the same ethertype, it will overwrite the previously registered handler with the new one. The function does not fail or indicate failure, but it may silently ignore new registrations if the maximum number of handlers (NET_HANDLER_MAX) has been reached.

Error/Edge case conf:0.80 **Q:** What are common mistakes or edge cases when using ca_net_register, and how can they lead to errors?
A: Common mistakes include passing a NULL function pointer, which can cause a crash since the function does not validate it. Also, calling this function from an ISR is not allowed and can lead to undefined behavior. If the maximum number of handlers is exceeded, new registrations are silently ignored. Additionally, calling this function in a multithreaded context without synchronization can cause race conditions.

`ca_net_register_iface`

Header: `caos/net/inc/net.h:94`
Source: `caos/net/src/net.c:67:83`
GUARD: `EN_NET`

```
bool ca_net_register_iface(ca_net_iface_t *iface)
```

Registers a network interface structure with the caOS networking stack.

Parametri:

Nome	Tipo	Descrizione
`iface`	`ca_net_iface_t *`	Pointer to the interface structure. Must contain valid function pointers for 'start' and 'send'. Cannot be NULL.

Ritorna:
Returns true if the interface was successfully registered and started; false if the pointer is invalid, the maximum number of interfaces is reached, or the interface's 'start' function failed.

Note:

- This function is not thread-safe and should be called during the system initialization phase before the networking stack is fully operational. It is not intended to be called from an ISR. The function performs a 'rollback' if the interface's internal 'start' routine fails, ensuring the system state remains consistent. A common mistake is passing an uninitialized or partially filled 'iface' structure, which will cause the function to return false immediately due to the internal NULL pointer checks.

Uso:

- This function is typically used during the board support package (BSP) or application startup sequence. It is paired with the definition of a 'ca_net_iface_t' structure that defines how the hardware sends and receives data. After calling this, the networking stack will manage the interface, and you would typically proceed to call 'ca_net_init' or 'ca_net_proto_init' to complete the network setup.

Esempio:

```c
/* Define the interface structure */
ca_net_iface_t my_eth_iface;

void setup_network(void) {
    /* Initialize the interface structure fields */
    my_eth_iface.start = my_start_function; /* Function to init hardware */
    my_eth_iface.send = my_send_function;   /* Function to transmit packets */

    /* Register the interface with the stack */
    if (ca_net_register_iface(&my_eth_iface)) {
        /* Registration successful, proceed with stack init */
        ca_net_init();
    } else {
        /* Handle registration failure */
        ca_printf("Failed to register network interface!\n");
    }
}
```

Q&A — fine-tuning review

6/6 pairs · qwen3-coder:latest

Usage conf:0.95 **Q:** When should I call ca_net_register_iface in a caOS-based embedded project?
A: You should call ca_net_register_iface during the system initialization phase, typically after setting up network interfaces and before starting any network operations. It is used to register a network interface with the caOS network subsystem, making it available for communication.

Code example conf:0.90 **Q:** Can you provide a minimal example of how to use ca_net_register_iface with caOS types?
A: ca_net_iface_t iface; iface.start = my_net_start; iface.send = my_net_send; bool result = ca_net_register_iface(&iface); if (!result) { // Handle registration failure }

Parameters conf:0.95 **Q:** What does the iface parameter represent, and what are valid values for it?
A: The iface parameter is a pointer to a ca_net_iface_t structure that defines a network interface. It must be a valid, allocated pointer pointing to an initialized structure with valid start and send function pointers. NULL is not a valid value and will cause the function to return false.

Return conf:0.95 **Q:** What does it mean when ca_net_register_iface returns true or false?
A: It returns true if the interface was successfully registered and started. It returns false if registration failed due to invalid interface data, missing functions, maximum interface limit reached, or a failure during the start process.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_net_register_iface?
A: Common mistakes include passing NULL as the iface parameter, using an incompletely initialized interface structure, calling the function multiple times with the same interface, and not handling rollback in case of start failure. These can lead to undefined behavior or failed network operations.

Behavior conf:0.90 **Q:** Is ca_net_register_iface blocking or non-blocking, and is it safe to call from an ISR?
A: ca_net_register_iface can be blocking if the interface's start function is blocking. It is not safe to call from an ISR, as it is not ISR-safe and is intended for initialization use only. It is also not thread-safe unless protected by a mutex.

`ca_net_resolve_mac`

Header: `caos/net/inc/route_service.h:85`
Source: `caos/net/src/route_service.c:157:161`
GUARD: `EN_ROUTE_SERVICE`

```
1 bool ca_net_resolve_mac(ca_ipv4_t dst, ca_mac_t *mac_out, u32 timeout_ms
    )
```

Resolves an IPv4 address to a MAC address using the ARP protocol, handling routing if necessary.

Parametri:

Nome	Tipo	Descrizione
`dst`	`ca_ipv4_t`	The target IPv4 address to resolve.
`mac_out`	`ca_mac_t *`	Pointer to a buffer where the resolved MAC address will be stored. Must not be NULL.
`timeout_ms`	`u32`	Maximum time in milliseconds to wait for an ARP response. Use a value sufficient for network latency.

Ritorna:
Returns true if the MAC address was successfully resolved and written to mac_out, false otherwise (e.g., timeout, unreachable, or ARP service disabled).

Note:

- This function is blocking; it will suspend the calling task until an ARP reply is received or the timeout expires. It is NOT safe to call from an ISR because it relies on RTOS blocking mechanisms. Ensure the network stack is initialized (ca_net_init) and the ARP service is enabled before use. A common mistake is providing a timeout that is too short for the network environment, leading to frequent resolution failures.

Uso:

- This function is typically used before sending data to a specific IP address to determine the destination hardware address on the local network. It is often paired with routing logic to handle gateway traversal. In a typical application flow, you call this during service initialization (like DHCP, DNS, or NTP) to cache the server's MAC address, preventing the need for repeated ARP requests for every packet sent. It is the standard way to bridge the gap between the IP layer and the Ethernet layer.

Esempio:

```
ca_mac_t server_mac;
u32 timeout = 2000U; // 2 seconds timeout

// Attempt to resolve the server IP to a MAC address
// Ensure the network stack is running before calling this
if (ca_net_resolve_mac(target_ip, &server_mac, timeout)) {
    // Success: server_mac now contains the hardware address
    ca_printf("MAC resolved successfully.\n");
} else {
    // Failure: handle the error, perhaps by retrying or logging
    // If this fails, the target might be unreachable or the network is down
    ca_printf("Failed to resolve MAC address.\n");
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_net_resolve_mac in a caOS network application?
A: You should call ca_net_resolve_mac when you need to resolve the MAC address of a destination IP host before sending an Ethernet frame. This is typically required when building network protocols that require direct MAC addressing, such as in ARP resolution, DHCP, or when sending raw Ethernet packets. It is commonly used in network protocol implementations like those found in net_proto.c and route_service.c.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_net_resolve_mac in a caOS application?
A: ca_ipv4_t ip = {192, 168, 1, 1}; mac_t mac; bool success = ca_net_resolve_mac(ip, &mac, 1000); if (success) { // Use mac for Ethernet transmission } else { // Handle resolution failure }

Parameters conf:0.85 **Q:** What are the valid and invalid values for the timeout_ms parameter in ca_net_resolve_mac?
A: The timeout_ms parameter must be a positive integer. A value of 0 or too low (e.g., less than 10ms) may cause resolution failures due to insufficient time for ARP response. Valid values are typically in the range of hundreds of milliseconds to several seconds, depending on network conditions and expected response times. Values that are too low can lead to frequent timeouts and failed resolutions.

Return conf:0.90 **Q:** What does it mean when ca_net_resolve_mac returns false?
A: A return value of false indicates that the MAC address resolution failed, either due to a timeout or an internal error. Possible causes include invalid IP address input, no gateway configured, insufficient timeout duration, or network issues preventing ARP response. The function will not return until either a valid MAC is resolved or the timeout expires.

`ca_net_rx_push`

Header: `caos/net/inc/net.h:100`
Source: `caos/net/src/net.c:88:99`
GUARD: `EN_NET`

```
bool ca_net_rx_push(ca_net_iface_t *iface, u16 buf_idx)
```

Notifies the network core that a new packet buffer is ready for processing.

Parametri:

Nome	Tipo	Descrizione
`iface`	`ca_net_iface_t *`	Pointer to the network interface structure. Currently unused (can be NULL).
`buf_idx`	`u16`	The index of the network buffer containing the received data. Must be less than NET_BUF_COUNT.

Ritorna:
Returns true if the buffer was successfully queued and the semaphore signaled. Returns false if the buffer index is invalid, the internal queue is full, or the semaphore could not be signaled.

Note:

- This function is designed to be called from network driver interrupt service routines (ISRs) or high-priority tasks. It is thread-safe because it uses internal queueing mechanisms and a capped semaphore to prevent overflow. A common mistake is failing to check the return value; if it returns false, the buffer is automatically freed by the function, so the caller must not attempt to access or free that buffer again. It is a non-blocking operation, ensuring that the driver does not hang if the network core is busy.

Uso:

- This function is the bridge between the hardware-specific driver (e.g., Ethernet or USB NCM) and the caOS network stack. It is typically called after the driver has successfully copied incoming data into a buffer allocated via ca_net_buf_alloc. Once this function is called, the network core takes ownership of the buffer, processes it, and eventually releases it. It is usually paired with ca_net_buf_alloc at the start of the RX sequence and is the final step before the driver returns control to the system.

Esempio:

```
/* Example: Driver receiving a packet */
void ethernet_rx_isr(void) {
    u16 idx = ca_net_buf_alloc();
    if (idx != 0xFFFF) {
        /* Assume hardware copies data into buffer at idx */
        /* ... hardware_copy_to_buffer(idx); ... */

        if (!ca_net_rx_push(NULL, idx)) {
            /* If push fails, the buffer is already freed by ca_net_rx_push */
            ca_printf("RX queue full, packet dropped\n");
        }
    } else {
        ca_printf("No buffers available for RX\n");
    }
}
```

Q&A — fine-tuning review

5/5 pairs · qwen3-coder:latest

Usage conf:0.95 **Q:** When should ca_net_rx_push be called in a network driver implementation?
A: ca_net_rx_push should be called by a network driver after it has received a packet and stored it in a pre-allocated buffer. It is used to notify the network core that a packet is ready for processing. This function is typically invoked from an ISR when the driver detects incoming data on the network interface.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_net_rx_push?

A: ca_net_iface_t iface; uint16_t buffer_index = 0; bool result = ca_net_rx_push(&iface, buffer_index); if (!result) { // Handle error: packet dropped or invalid buffer }

Parameters conf:0.95 **Q:** What does the buf_idx parameter represent, and what are valid values for it?

A: The buf_idx parameter is the index of a pre-allocated buffer that contains the received packet data. Valid values are integers between 0 and NET_BUF_COUNT-1. If buf_idx is outside this range, the function returns false and the buffer is freed.

Return conf:0.95 **Q:** What does it mean when ca_net_rx_push returns false?

A: ca_net_rx_push returns false if either the buf_idx is out of range (not between 0 and NET_BUF_COUNT-1) or if the internal queue is full and cannot accept the packet. In such cases, the buffer is freed and the packet loss counter is incremented.

Error/Edge case conf:0.85 **Q:** What are common mistakes when calling ca_net_rx_push?

A: Common mistakes include passing an invalid buf_idx (outside the range 0 to NET_BUF_COUNT-1), calling the function without having allocated the buffer at that index, or calling it when the queue is full, which leads to packet loss. Additionally, calling the function from an ISR without proper driver configuration may cause issues.

`ca_net_send`

Header: `caos/net/inc/net.h:112`
Source: `caos/net/src/net.c:221:243`
GUARD: `EN_NET`

```
bool ca_net_send(u16 buf_idx)
```

Transmits a network buffer over the default interface and handles memory cleanup.

Parametri:

Nome	Tipo	Descrizione
`buf_idx`	`u16`	The index of the network buffer to send. Must be a valid index returned by ca_net_buf_alloc().

Ritorna:
Returns true if the buffer was successfully handed over to the network driver, false otherwise (e.g., invalid index, link down, or driver failure).

Note:

- This function implements a zero-copy mechanism: if it returns true, the network driver takes ownership of the buffer and will free it automatically after transmission. If it returns false, the function automatically calls ca_net_buf_free() to prevent memory leaks, so the caller must not free the buffer manually in the error case. It is not safe to call this from an ISR context as it may involve complex driver operations. Ensure the network stack is initialized via ca_net_init() and the link is up before calling this, otherwise, the buffer will be immediately discarded.

Uso:

- This is the final step in the network packet lifecycle. Typically, you allocate a buffer with ca_net_buf_alloc(), populate the payload using ca_net_buf_ptr(), set the data length, and then call ca_net_send(). It is used by protocol services like ARP, DHCP, or DNS to transmit their respective packets. If the transmission fails, the buffer is released back to the pool, ensuring the system does not run out of memory during network congestion or link loss.

Esempio:

```
u16 idx = ca_net_buf_alloc();
if (idx != NET_BUF_INVALID) {
    u8 *data = ca_net_buf_ptr(idx);
    // Fill buffer with custom data
    ca_memcpy(data, "Hello", 5);

    // Set buffer metadata
    // Note: Assuming direct access to buffer structure fields as per caOS
    conventions
    // b->data_off = NET_BUF_HEADROOM;
    // b->data_len = 5;

    if (!ca_net_send(idx)) {
        // ca_net_send handles the free on failure,
        // so we do not call ca_net_buf_free here.
        ca_printf("Failed to send packet\n");
    }
} else {
    ca_printf("No buffers available\n");
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_net_send() and what is its purpose in the network stack?
A: You should call ca_net_send() after allocating a network buffer with ca_net_buf_alloc() and filling it with data you want to transmit. The function sends the packet to the network driver, returning true if successful or false if the transmission fails. It is used in network services like ARP and DHCP to send packets over the network interface.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_net_send() with caOS types?
A: u16 buf_idx = ca_net_buf_alloc(); if (buf_idx < NET_BUF_COUNT) { // Fill buffer with data... bool success = ca_net_send(buf_idx); if (!success) { // Handle send failure } } else { // Handle buffer allocation failure }

Error/Edge case conf:0.85 **Q:** What are the common error conditions when calling ca_net_send()?
A: The function returns false if the buf_idx parameter is outside the valid range of 0 to NET_BUF_COUNT-1. Additionally, it returns false if the actual network transmission fails. The function does not validate that the buffer contains valid data, only that the buffer index is valid.

Behavior conf:0.60 **Q:** Is ca_net_send() blocking or non-blocking, and can it be called from an ISR?
A: The behavior of ca_net_send() is not explicitly described in terms of blocking/non-blocking or ISR-safety. However, since it is used in network services like ARP and DHCP, it likely interacts with hardware drivers that may have specific timing or interrupt requirements. The function does not specify thread safety or ISR compatibility, so caution should be exercised when calling it from interrupt contexts.

`ca_net_send_on`

Header: `caos/net/inc/net.h:120`
Source: `caos/net/src/net.c:339:386`
GUARD: `EN_NET`

```
bool ca_net_send_on(u8 iface_id, u16 buf_idx)
```

Transmits a network buffer through a specific interface and transfers ownership to the driver.

Parametri:

Nome	Tipo	Descrizione
`iface_id`	`u8`	The index of the network interface to use. Must be less than the total number of initialized interfaces.
`buf_idx`	`u16`	The index of the network buffer to transmit. Must be a valid index within the allocated buffer pool.

Ritorna:

Returns true if the buffer was successfully handed over to the interface driver, false otherwise (e.g., invalid index, interface down, or transmission failure).

Note:

- This function is the critical handover point for network buffers. Once called, the interface driver assumes full ownership of the buffer; the application must NEVER attempt to free or modify the buffer after this call, regardless of the return value. If the function returns false, the buffer is automatically freed by the core, unless the driver has already taken ownership. This function is not thread-safe in the sense that a buffer must only be sent once; calling it twice on the same buffer index will lead to memory corruption and DMA stalls. It is generally not intended to be called directly from an ISR unless the specific driver implementation supports it.

Uso:

- This function is used when a packet (e.g., ARP, TCP, or raw data) is fully constructed and ready for transmission. It is typically paired with ca_net_buf_alloc to acquire the buffer and ca_net_proto_* functions to fill the payload. If the transmission is successful, the driver handles the eventual release of the buffer. If the application decides not to send the buffer after allocation (e.g., due to a logic error or protocol rejection), it must manually call ca_net_buf_free to prevent memory leaks.

Esempio:

```
u16 tx = ca_net_buf_alloc();
if (tx != 0xFFFF) {
    /* Prepare the buffer data */
    ca_net_buf_t *b = ca_net_buf_ptr(tx);
    b->data_off = 0;
    b->data_len = 10;
    /* Copy data into buffer */
    ca_memcpy(b->data, "HelloNet", 8);

    /* Attempt to send via interface 0 */
    if (!ca_net_send_on(0, tx)) {
        /* If send fails, we must free the buffer as it was not handed over */
        ca_net_buf_free(tx);
    }
    /* Note: If ca_net_send_on returns true, we do NOT free the buffer */
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_net_send_on in my network stack implementation?
A: You should call this function once a packet, such as an ARP or TCP frame, has been fully constructed and is ready for transmission. It is typically used after acquiring a buffer via ca_net_buf_alloc and populating it using ca_net_proto_* functions.

Code example conf:0.90 **Q:** Provide a minimal C example showing how to transmit a buffer using caOS types.
A: u8 iface = 0; u16 buf = ca_net_buf_alloc(); if (buf != 0xFFFF) { /* fill buffer */ if (!ca_net_send_on(iface, buf)) { /* handle failure */ } }

Parameters conf:1.00 **Q:** What are the constraints for the iface_id and buf_idx parameters?
A: The iface_id must be less than the total number of initialized network interfaces. The buf_idx must be a valid index within the allocated buffer pool.

Return conf:1.00 **Q:** What does a return value of false indicate, and what happens to the buffer in that case?
A: A return value of false indicates that the transmission failed, the interface is down, or the index provided was invalid. If the function returns false, the buffer is automatically freed by the core, provided the driver has not already taken ownership.

Error/Edge case conf:1.00 **Q:** What are the risks of calling ca_net_send_on multiple times on the same buffer index?
A: Calling this function twice on the same buffer index is not thread-safe and will lead to memory corruption and DMA stalls, as the buffer must only be sent once.

Behavior conf:1.00 **Q:** Is it safe to call ca_net_send_on from an Interrupt Service Routine (ISR)?
A: The function is generally not intended to be called directly from an ISR unless the specific driver implementation explicitly supports it.

`ca_net_send_raw`

Header: `caos/net/inc/net.h:126`
Source: `caos/net/src/net.c:332:337`
GUARD: `EN_NET`

```
1 bool ca_net_send_raw(u8 iface_id, u16 buf_idx)
```

Transmits a network buffer through a specific interface by marking it as a raw frame.

Parametri:

Nome	Tipo	Descrizione
`iface_id`	`u8`	The identifier of the network interface (e.g., Ethernet or Wi-Fi) to use for transmission.
`buf_idx`	`u16`	The index of the network buffer to send. Must be less than NET_BUF_COUNT.

Ritorna:
Returns true if the buffer was successfully queued for transmission, false if the buffer index is invalid or the transmission failed.

Note:

- This function is not thread-safe if multiple tasks access the same buffer index simultaneously. It is not intended to be called from an ISR because it performs memory flag modifications and potentially complex network stack operations. Once this function is called, the network interface takes ownership of the buffer; you must not attempt to free or modify the buffer contents after this call, as it will lead to memory corruption or race conditions. The most common mistake is attempting to reuse the buffer index immediately after calling this function without waiting for the network stack to release it.

Uso:

- This function is typically used in packet forwarding or custom protocol implementation tasks where you need to bypass standard stack processing. It is usually paired with

ca_net_buf_alloc to acquire a buffer, followed by manual header construction, and finally ca_net_send_raw to dispatch the frame. After the call, the application logic should increment success or failure counters based on the return value to track network health.

Esempio:

```
/* Allocate a buffer for a raw frame */
u16 buf_idx = ca_net_buf_alloc();
if (buf_idx < NET_BUF_COUNT) {
    /* Access the buffer memory to fill in custom data */
    u8 *data = ca_net_buf_ptr(buf_idx);
    /* ... fill data with custom protocol bytes ... */

    /* Send the raw frame via interface 0 */
    if (ca_net_send_raw(0U, buf_idx)) {
        /* Success: The interface now owns the buffer */
    } else {
        /* Failure: We must free the buffer manually if send fails */
        ca_net_buf_free(buf_idx);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_net_send_raw instead of standard network stack functions?
A: You should use ca_net_send_raw for packet forwarding or custom protocol implementations where you need to bypass standard stack processing. It is typically used after calling ca_net_buf_alloc to acquire a buffer and manually constructing the frame headers.

Code example conf:0.90 **Q:** How do I correctly implement a raw frame transmission in caOS?
A: u8 iface = 0; u16 idx = ca_net_buf_alloc(); if (idx < NET_BUF_COUNT) { /* Construct frame in buffer */ bool success = ca_net_send_raw(iface, idx); }

Parameters conf:1.00 **Q:** What are the constraints for the buf_idx parameter in ca_net_send_raw?
A: The buf_idx parameter represents the index of the network buffer to send. It must be a value less than NET_BUF_COUNT. Providing an index equal to or greater than NET_BUF_COUNT will result in the function returning false.

Return conf:1.00 **Q:** What does a false return value from ca_net_send_raw indicate?
A: A false return value indicates that the transmission failed or that the provided buffer index was invalid. Failures can occur if the buffer index is out of bounds or if the network stack cannot queue the frame for transmission.

`ca_net_set_phy_nrst`

Header: `caos/net/inc/net.h:137`
Source: `caos/net/src/net.c:304:308`
GUARD: `EN_NET`

```
void ca_net_set_phy_nrst(u8 port, u8 pin)
```

Configures the GPIO port and pin used to physically reset the Ethernet PHY chip.

Parametri:

Nome	Tipo	Descrizione
port	u8	The GPIO port identifier (e.g., 0 for GPIOA, 1 for GPIOB). Must match the hardware mapping used by the system.
pin	u8	The specific pin number on the port (0-15). Ensure this pin is physically connected to the PHY's NRST line.

Ritorna:
void

Note:

- This function is non-blocking and thread-safe as it only performs a simple memory assignment to internal static variables. It is safe to call from an ISR, though it is typically called during the system initialization phase. The most common mistake is providing an incorrect port or pin index that does not correspond to the actual PCB layout, which will result in the Ethernet PHY failing to initialize or reset correctly. This function does not trigger the reset itself; it only registers the hardware location for later use by the network stack.

Uso:

- This function is used during the network stack configuration sequence, typically before calling ca_net_init. It is paired with the network configuration structure to inform the driver which GPIO pin controls the hardware reset line of the Ethernet PHY. By setting this early, the system ensures that when the network stack performs a hardware reset, it toggles the correct physical pin to bring the PHY out of its power-on state.

Esempio:

```c
/* Define the hardware configuration for the Ethernet PHY reset pin */
#define PHY_NRST_PORT 1 // GPIOB
#define PHY_NRST_PIN  5 // Pin 5

void network_setup(void) {
    /* Configure the PHY reset pin location before initializing the network stack */
    ca_net_set_phy_nrst(PHY_NRST_PORT, PHY_NRST_PIN);

    /* Initialize the network stack after setting the hardware parameters */
    ca_net_init();

    /* Proceed to initialize other network protocols */
    ca_net_proto_init();
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_net_set_phy_nrst() and why is it important for network initialization?
A: You should call ca_net_set_phy_nrst() during the network initialization phase, before using any other network functions. It configures the GPIO port and pin that will be used to reset the PHY device. This function is essential because it sets up the hardware reset mechanism for the network interface, which is required for proper network operation. It must be called before any network-related operations to ensure the PHY is correctly initialized.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_net_set_phy_nrst() with caOS types?
A: u8 port = 1; /* GPIOB */ u8 pin = 5; /* Pin 5 */ ca_net_set_phy_nrst(port, pin);

Parameters conf:0.90 **Q:** What do the port and pin parameters in ca_net_set_phy_nrst represent, and what are the valid ranges for each?
A: The port parameter specifies the GPIO port number (0 for GPIOA, 1 for GPIOB, etc.), with valid values from 0 to 15. The pin parameter specifies the GPIO pin number within that port, also ranging from 0 to 15. Invalid values can cause access to invalid memory addresses and unpredictable behavior, as the function does not validate its inputs.

`ca_net_stats`

Header: `caos/net/inc/net.h:118`
Source: `caos/net/src/net.c:267:270`
GUARD: `EN_NET`

```
void ca_net_stats(ca_net_stats_t *out)
```

Copies the current network stack statistics into a provided structure.

Parametri:

Nome	Tipo	Descrizione
`out`	`ca_net_stats_t *`	Pointer to a structure where the statistics will be copied. Must not be NULL to receive data.

Ritorna:
None (void).

Note:

- This function is non-blocking and thread-safe as it performs a simple memory copy of the internal statistics structure. It is safe to call from an ISR, as it does not involve any complex synchronization primitives or blocking operations. The most common mistake is passing a NULL pointer, which the function handles gracefully by doing nothing, but this can lead to uninitialized variables in your application logic if you do not check the result or initialize your local structure first.

Uso:

- This function is typically used for monitoring and debugging network performance. It is often called periodically within a background task or via a CLI command to report metrics like frame counts, dropped packets, and bus errors. It is frequently paired with ca_net_buf_free_count to provide a complete snapshot of the network stack's health and resource utilization.

Esempio:

```
void monitor_task(void *arg) {
    (void)arg;
    while (1) {
        ca_net_stats_t st;
        // Get current network stats
        ca_net_stats(&st);
        // Get current buffer availability
        u16 free_bufs = ca_net_buf_free_count();

        // Print stats to the system console
        ca_printf_n("Net Stats: RX=%u, TX=%u, Free Buffers=%u",
                    st.rx_frames, st.tx_frames, free_bufs);

        // Wait 5 seconds before next update
```

```
        ca_os_sleep(5000);
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_net_stats() and what is its purpose in a networked embedded application?
A: You should call ca_net_stats() to monitor network usage and detect communication issues in your embedded application. It copies network statistics into a user-provided structure for analysis. Since it's non-blocking and not thread-safe, it's best used in tasks that are not interrupted by other network-related operations. It's typically called periodically to gather telemetry data about network performance.

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_net_stats() with caOS types?
A: ca_net_stats_t stats; ca_net_stats(&stats); // Now 'stats' contains the network statistics // Example usage: print or log the values u32 rx_packets = stats.rx_packets; u32 tx_packets = stats.tx_packets;

Parameters conf:0.95 **Q:** What is the significance of the 'out' parameter in ca_net_stats() and what happens if it is NULL?
A: The 'out' parameter is a pointer to a user-provided structure where network statistics will be copied. If 'out' is NULL, the function does nothing and returns immediately without attempting to copy any data. This is a safety check to prevent memory corruption, but it also means no statistics will be retrieved if a NULL pointer is passed.

`ca_net_tcp_cksum`

Header: `caos/net/inc/net_proto.h:277`
Source: `caos/net/src/net_proto_tcp.c:22:46`
GUARD: `EN_NET_PROTO`

```
u16 ca_net_tcp_cksum(ca_ipv4_t src, ca_ipv4_t dst, const u8 *tcp_seg,
    u16 tcp_seg_len)
```

Calculates the 16-bit one's complement checksum for a TCP segment including the pseudo-header.

Parametri:

Nome	Tipo	Descrizione
`src`	`ca_ipv4_t`	Source IPv4 address structure.
`dst`	`ca_ipv4_t`	Destination IPv4 address structure.
`tcp_seg`	`const u8 *`	Pointer to the TCP segment buffer. Must be a valid memory address.
`tcp_seg_len`	`u16`	Length of the TCP segment in bytes. Must match the actual buffer size.

Ritorna:
A 16-bit checksum value. This value is the one's complement sum of the pseudo-header and the TCP segment.

Note:

- This function is purely computational and non-blocking; it performs no I/O and is safe to call from any context, including ISRs, provided the input buffer is valid. It is not thread-safe in the sense that it does not lock the buffer, so ensure the buffer is not being modified by another task while the checksum is being calculated. The most common

mistake is passing an incorrect length or a pointer to a buffer that has already been freed or is currently being modified by the DMA controller. Always ensure the TCP checksum field in the segment is zeroed out before calling this function, as the checksum calculation includes that field.

Uso:

- This function is used when manually constructing TCP packets that are not offloaded to hardware. It is typically used in the transmission path after the TCP header and payload have been prepared in a network buffer. The standard sequence is: allocate a buffer using ca_net_buf_alloc, fill the TCP header and data, zero the checksum field, call ca_net_tcp_cksum to compute the value, write the result into the TCP header, and finally pass the buffer to ca_net_send. It is essential for ensuring data integrity at the transport layer.

Esempio:

```
/* Example: Preparing a TCP segment for transmission */
ca_ipv4_t src_ip = { .v = {192, 168, 1, 10} };
ca_ipv4_t dst_ip = { .v = {192, 168, 1, 20} };
u8 *buf = ca_net_buf_alloc(64); /* Allocate network buffer */
if (buf != NULL) {
    /* ... Fill TCP header and payload at buf ... */
    /* Ensure checksum field (offset 16) is zeroed before calculation */
    buf[16] = 0U;
    buf[17] = 0U;

    /* Calculate checksum */
    u16 csum = ca_net_tcp_cksum(src_ip, dst_ip, buf, 64);

    /* Insert calculated checksum into the TCP header */
    buf[16] = (u8)(csum >> 8);
    buf[17] = (u8)csum;

    /* Send the packet */
    ca_net_send(buf, 64);
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_net_tcp_cksum in my network stack implementation?
A: You should call this function when manually constructing TCP packets that are not offloaded to hardware. It is used in the transmission path after the TCP header and payload are prepared, but before passing the buffer to ca_net_send.

Code example conf:0.90 **Q:** Can you provide a minimal example of how to use ca_net_tcp_cksum?
A: u8 *buf = ca_net_buf_alloc(len); /* Fill header and data */ tcp_hdr->cksum = 0; u16 cksum = ca_net_tcp_cksum(src_ip, dst_ip, buf, len); tcp_hdr->cksum = cksum; ca_net_send(buf);

Parameters conf:1.00 **Q:** What are the requirements for the tcp_seg and tcp_seg_len parameters?
A: tcp_seg must be a valid memory address pointing to the TCP segment buffer. tcp_seg_len must be a u16 representing the length of the segment in bytes and must match the actual buffer size.

Return conf:1.00 **Q:** What does the return value of ca_net_tcp_cksum represent?
A: The function returns a 16-bit checksum value, which is the one's complement sum of the pseudo-header and the TCP segment.

Error/Edge case conf:1.00 **Q:** What are the most common mistakes when using ca_net_tcp_cksum?
A: The most common mistakes are passing an incorrect length, providing a pointer to a buffer that has been freed, or using a buffer currently being modified by the DMA con-

troller. Also, failing to zero out the TCP checksum field in the segment before calculation will result in an incorrect checksum.

Behavior conf:1.00 **Q:** Is ca_net_tcp_cksum thread-safe and can I call it from an ISR?
A: The function is non-blocking and safe to call from any context, including ISRs. It is not thread-safe regarding buffer access, so you must ensure the buffer is not being modified by another task or DMA while the calculation is in progress.

`ca_net_tx_csum_offload`

Header: `caos/net/inc/net.h:127`
Source: `caos/net/src/net.c:110:114`
GUARD: `EN_NET`

```
bool ca_net_tx_csum_offload(void)
```

Checks if the network interface supports hardware checksum offloading for outgoing packets.

Ritorna:
Returns true if the default network interface is registered and supports hardware checksum offloading; returns false otherwise (including if no interface is registered).

Note:

- This function is non-blocking and thread-safe, as it performs a simple read of a configuration flag. It is safe to call from an ISR, though it is typically used in the network stack's transmission path. The function relies on the network interface being previously registered via ca_net_register_iface. A common mistake is assuming this function will return true before the network stack has been fully initialized or before an interface has been successfully bound to the system.

Uso:

- This function is used by the network stack to decide whether to calculate checksums in software (CPU-intensive) or offload the task to the Ethernet peripheral (hardware-accelerated). It is typically called inside the packet transmission sequence, just before preparing the frame buffer. If this returns false, the developer must ensure that a software checksum calculation function, such as ca_net_cksum, is invoked to populate the packet headers before sending.

Esempio:

```
/* Check if we can offload the checksum calculation to hardware */
if (ca_net_tx_csum_offload()) {
    /* Hardware will handle the checksum, just prepare the buffer */
    /* ... proceed to send packet ... */
} else {
    /* Calculate checksum in software before sending */
    /* Use ca_net_cksum to compute the value for the buffer */
    /* ... proceed to send packet ... */
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should a developer call ca_net_tx_csum_offload() and why is it important for network performance?
A: A developer should call ca_net_tx_csum_offload() to determine whether the default network interface supports hardware offloading of TCP/UDP checksum calculation for outgoing packets. This is important because enabling checksum offloading can significantly reduce CPU load by allowing the network hardware to compute checksums instead

of the CPU. The function must be called after network interfaces are initialized, and it is non-blocking and thread-safe, making it suitable for use in task contexts but not from ISRs.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using caOS types that demonstrates how to use ca_net_tx_csum_offload()?
A: bool is_offload_supported = ca_net_tx_csum_offload(); if (is_offload_supported) { // Enable hardware checksum offload for TX // Implementation depends on specific hardware and driver } else { // Fall back to software checksum calculation }

Return conf:0.90 **Q:** What do the return values of ca_net_tx_csum_offload() indicate, and what causes the function to return false?
A: The function returns true if the default network interface supports TX checksum offloading, and false otherwise. It always returns false if no network interface has been registered (i.e., when s_iface_count is 0). The function does not fail due to hardware errors or misconfigurations; it simply reports the capability of the interface. Common causes for returning false include uninitialized interfaces or interfaces that do not support offloading features.

`ca_net_tx_mtu`

Header: `caos/net/inc/net.h:134`
Source: `caos/net/src/net.c:116:127`
GUARD: `EN_NET`

```
u16 ca_net_tx_mtu(void)
```

Returns the Maximum Transmission Unit (MTU) of the default network interface.

Ritorna:
Returns a 16-bit unsigned integer representing the MTU in bytes. If no interface is registered, it returns the default value (typically 1500).

Note:

- This function is thread-safe and non-blocking, as it simply reads a value from the internal network interface structure. It can be safely called from an ISR because it performs no complex logic or synchronization. The primary precondition is that the network stack must be initialized; if no interface has been registered via ca_net_register_iface, the function will safely return a default value rather than crashing. A common mistake is assuming the MTU is always 1500; always use this function to determine fragmentation limits, as different hardware (like USB NCM vs. Ethernet) may have different constraints.

Uso:

- This function is used whenever you need to determine if a packet needs to be fragmented before transmission. It is typically called before building a network frame to calculate the maximum payload size that can fit in a single transmission unit. It is often paired with ca_net_register_iface during system initialization and used inside packet processing loops to ensure compliance with the medium's physical limitations.

Esempio:

```
/* Example: Checking if a payload fits in the current MTU before sending */
#include "caos.h"

void send_data_safely(u16 payload_len, u8 *data) {
    /* Get the current MTU to decide if we need to fragment */
    u16 mtu = ca_net_tx_mtu();
```

```
7
8       /* Assume 20 bytes for IP header overhead */
9       if (payload_len > (mtu - 20U)) {
10          /* Payload is too large, trigger fragmentation logic */
11          /* ca_ip_fragment_send(...) would be called here */
12          ca_printf("Packet too large, fragmentation required.\n");
13      } else {
14          /* Payload fits, proceed with standard transmission */
15          /* ca_net_send(...) would be called here */
16          ca_printf("Packet fits in MTU, sending directly.\n");
17      }
18  }
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_net_tx_mtu in my network application?
A: You should call it whenever you need to determine if a packet requires fragmentation before transmission. It is typically used before building a network frame to calculate the maximum payload size that fits in a single unit, ensuring compliance with the physical medium's limitations.

Code example conf:1.00 **Q:** How do I use ca_net_tx_mtu to check the MTU in my code?
A: You can retrieve the MTU value directly into a u16 variable as shown: u16 mtu = ca_net_tx_mtu(); if (payload_size > mtu) { /* handle fragmentation */ }

Return conf:1.00 **Q:** What does the return value of ca_net_tx_mtu represent, and what happens if no interface is registered?
A: The function returns a 16-bit unsigned integer representing the MTU in bytes. If no network interface has been registered via ca_net_register_iface, it safely returns a default value, which is typically 1500.

Error/Edge case conf:1.00 **Q:** What is a common mistake developers make when using ca_net_tx_mtu?
A: A common mistake is assuming the MTU is always 1500. Developers should always use this function to determine fragmentation limits, as different hardware interfaces, such as USB NCM versus Ethernet, may have different constraints.

`ca_notify_clear`

Header: `caos/inc/notify.h:119`
Source: `caos/src/notify.c:208:227`
GUARD: `EN_NOTIFY`

```
1 u32 ca_notify_clear(u32 task_idx)
```

Atomically retrieves and resets the notification value for a specific task.

Parametri:

Nome	Tipo	Descrizione
`task_idx`	`u32`	The index of the target task. Use 0xFFFFFFFF to target the currently running task.

Ritorna:
Returns the previous u32 notification value held by the task before it was cleared. Returns 0 if the task context is invalid or if no notification was pending.

Note:

- This function is thread-safe because it uses a spinlock to ensure the read-and-clear operation is atomic, preventing race conditions where a notification might be lost between reading and resetting. It is non-blocking, meaning it executes immediately and returns

control to the caller without putting the task into a wait state. While it is technically callable from an ISR, you should be extremely careful as it uses a spinlock; ensure your system configuration allows spinlocks within ISRs to avoid deadlocks. The most common beginner mistake is calling this function without having first waited for a notification, leading to the processing of stale or zero values.

Uso:

- This function is typically used in a 'check-and-process' loop where a task needs to consume pending notifications sent by other tasks or ISRs. It is often paired with ca_notify_wait, which blocks the task until a notification arrives, followed by ca_notify_clear to retrieve and acknowledge the data. This pattern is essential for implementing lightweight inter-task communication without the overhead of full message queues. It is also used during task cleanup or state resets to ensure no stale notification flags remain in the task's context.

Esempio:

```c
/* Example: Wait for a notification and process it */
void task_worker(void *arg) {
    while (1) {
        /* Wait for any notification to arrive */
        ca_notify_wait(0xFFFFFFFF);

        /* Atomically retrieve and clear the notification value */
        u32 event_flags = ca_notify_clear(0xFFFFFFFF);

        if (event_flags != 0) {
            /* Process the flags retrieved */
            if (event_flags & 0x01) {
                /* Perform specific action for bit 0 */
            }
        }
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_notify_clear to reset a task's notification value?
A: You should use ca_notify_clear when you want to read and then reset the notification value of a task. This is useful to avoid reading stale notifications or to ensure that a task's notification state is cleared after it has been processed.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_notify_clear in a task?
A: u32 task_idx = 1; u32 old_value = ca_notify_clear(task_idx); if (old_value != 0) { // Process the notification value // ... do something with old_value }

Parameters conf:0.95 **Q:** What does the task_idx parameter in ca_notify_clear represent and what are valid values?
A: The task_idx parameter specifies the index of the task whose notification value is to be cleared. Valid values are from 0 to 0xFFFFFFFE, and 0xFFFFFFFF refers to the current task. Using an invalid index or a value outside this range will cause the function to return 0.

Return conf:0.90 **Q:** What does ca_notify_clear return when it succeeds and what does it return on failure?
A: On success, ca_notify_clear returns the notification value of the task before it was cleared. If the task does not exist or the context is invalid, it returns 0. A return value of 0 may also indicate that the task had no notification set before the call.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_notify_clear?

A: Common mistakes include calling ca_notify_clear with an invalid task index, which can lead to returning 0 even if the task had a notification set. Also, using 0xFFFFFFFF for the current task is only safe when called from an active task. Additionally, calling the function from an ISR without considering potential deadlocks is risky.

Behavior conf:0.90 **Q:** Is ca_notify_clear blocking or non-blocking, and is it safe to call from an ISR?

A: ca_notify_clear is a non-blocking function. It uses a spinlock to protect access to the task structure, making it thread-safe. It is safe to call from an ISR, but care must be taken to avoid potential deadlocks due to the spinlock usage.

`ca_notify_get`

Header: `caos/inc/notify.h:112`
Source: `caos/src/notify.c:188:206`
GUARD: `EN_NOTIFY`

```
u32 ca_notify_get(u32 task_idx)
```

Retrieves the current notification value assigned to a specific task.

Parametri:

Nome	Tipo	Descrizione
`task_idx`	`u32`	The index of the target task. Use 0xFFFFFFFF to retrieve the notification value of the currently running task.

Ritorna:
Returns the u32 notification value stored in the task context. Returns 0 if the task context is invalid or cannot be found.

Note:

- This function is thread-safe as it uses a spinlock to protect the access to the task's notification value. It is non-blocking, meaning it returns immediately after reading the value. It is safe to call from an ISR, although it is primarily designed for task-level synchronization. A common mistake is assuming the value is automatically cleared after reading; this function only reads the value, so you must call ca_notify_clear if you need to reset the notification state after processing.

Uso:

- This function is typically used in a polling or event-driven loop where a task needs to check if it has received a notification from another task or an ISR. It is often paired with ca_notify_wait to block until a notification arrives, or used in conjunction with ca_notify_give to implement inter-task signaling. Developers use this to check specific flags or data bits sent to the task without needing a full message queue. It is part of the standard notification lifecycle: wait for event -> get value -> process -> clear.

Esempio:

```
/* Example: Checking for a notification flag in a task loop */

void my_task_function(void) {
    u32 notify_val;

    while (1) {
        /* Wait for a signal from another task or ISR */
        ca_notify_wait(0xFFFFFFFF);
```

```

        /* Retrieve the notification value for the current task */
        notify_val = ca_notify_get(0xFFFFFFFF);

        if (notify_val != 0) {
            /* Process the notification value */
            /* ... logic here ... */

            /* Clear the notification after processing to avoid re-triggering */
            ca_notify_clear(0xFFFFFFFF);
        }
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_notify_get() in a caOS application?
A: You should use ca_notify_get() to retrieve the notification value of a specific task, typically when implementing inter-task communication or handling notifications from ISRs. It is useful for synchronizing tasks or passing status information without using shared memory or semaphores.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_notify_get() in a caOS task?
A: u32 task_idx = 1; u32 notify_value = ca_notify_get(task_idx); if (notify_value != 0) { // Process notification }

Parameters conf:0.90 **Q:** What does the task_idx parameter in ca_notify_get() represent and what are valid values?
A: The task_idx parameter specifies the index of the task whose notification value is to be retrieved. Valid values are integers from 0 up to the maximum task index defined by the system. A value of 0xFFFFFFFF indicates that the current task should be used. Invalid indices may lead to a NULL pointer and a return value of 0.

Return conf:0.95 **Q:** What does ca_notify_get() return when it fails, and what causes the failure?
A: ca_notify_get() returns 0 when the task does not exist or when the context is NULL, indicating an error. This typically happens if the task index is invalid or if the system is not properly initialized.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_notify_get()?
A: Common mistakes include using an invalid task index, which can cause a NULL pointer dereference and return 0. Not properly handling the spinlock can lead to race conditions. Also, calling the function with a task index that does not exist will result in a failure to retrieve the notification value.

Behavior conf:0.90 **Q:** Is ca_notify_get() blocking or non-blocking, and is it safe to call from an ISR?
A: ca_notify_get() is a non-blocking function. It is safe to call from an ISR, as the function uses a spinlock to protect access to shared variables. However, it is thread-safe only when the task_idx is not 0xFFFFFFFF, as that case uses the current task context.

`ca_notify_give`

Header: `caos/inc/notify.h:124`
GUARD: `EN_NOTIFY`

```
static inline bool ca_notify_give(ca_task_handle_t h, u32 value)
```

Sends a notification value to a specific task to unblock or signal it.

Parametri:

Nome	Tipo	Descrizione
`h`	`ca_task_handle_t`	The handle of the target task to notify. Must be a valid handle obtained via ca_os_task_handle or similar.
`value`	`u32`	The 32-bit notification value to send. This value is stored in the target task's notification register.

Ritorna:
Returns true if the notification was successfully sent to the task, false if the task handle is invalid or the operation failed.

Note:

- This function is non-blocking and thread-safe, meaning it can be called from any task context. It is NOT safe to call from an ISR; use ca_notify_give_isr instead if you are inside an interrupt. The target task must be initialized and running to receive the notification. A common mistake is passing a NULL or uninitialized task handle, which will cause the function to return false and potentially lead to silent failures in your logic.

Uso:

- This function is typically used for inter-task communication where one task needs to wake up another task that is waiting on a notification. It is often paired with ca_notify_wait or ca_notify_take in the receiving task. The typical sequence is: the receiver calls ca_notify_wait to block until data arrives, and the sender calls ca_notify_give to provide that data and wake the receiver up. It is a lightweight alternative to queues or semaphores when you only need to pass a simple signal or a single 32-bit value.

Esempio:

```
/* Example: Task A signals Task B to process data */
ca_task_handle_t task_b_handle = ca_os_task_handle("TaskB");

if (task_b_handle != NULL) {
    u32 data_to_send = 0xDEADBEEF;
    /* Send the notification to Task B */
    if (ca_notify_give(task_b_handle, data_to_send)) {
        /* Notification sent successfully */
    } else {
        /* Handle error: notification failed */
    }
}

/* Inside Task B: */
u32 received_value;
/* Wait for notification from Task A */
if (ca_notify_wait(&received_value)) {
    /* Process the received_value */
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_notify_give instead of a message queue?
A: Use ca_notify_give when you need to signal a task with a specific state or command where only the most recent information matters. It is ideal for simple inter-task communication to avoid the overhead of a full message queue when you only need to pass a single 32-bit value.

Code example conf:1.00 **Q:** How do I send a status flag to another task using ca_notify_give?
A: You can send a 32-bit value like this: u32 task_id = 2; u32 status_flag = 0x01; bool success = ca_notify_give(task_id, status_flag);

Parameters conf:1.00 **Q:** What are the constraints for the task_idx and value parameters?
A: The task_idx must be a valid, unique identifier of a destination task previously registered in the OS. The value parameter is a 32-bit data payload that can be any arbitrary value, such as status codes, flags, or pointers.

Return conf:1.00 **Q:** What does a return value of false indicate when calling ca_notify_give?
A: A return value of false indicates that the notification was not successfully sent. This typically occurs if the provided task index is invalid or if the caOS system is not initialized.

`ca_notify_give_inc`

Header: `caos/inc/notify.h:133`
GUARD: `EN_NOTIFY`

```
static inline bool ca_notify_give_inc(ca_task_handle_t h)
```

Increments the notification value of a target task, acting as a lightweight signaling mechanism.

Parametri:

Nome	Tipo	Descrizione
`h`	`ca_task_handle_t`	The handle of the task to be notified. Must be a valid, initialized task handle.

Ritorna:
Returns true if the notification was successfully sent to the target task, false otherwise (e.g., invalid handle or internal error).

Note:

- This function is non-blocking and thread-safe, making it ideal for signaling tasks from other threads without the overhead of a full semaphore. It is NOT safe to call from an ISR; use ca_notify_give_isr instead if you need to signal from an interrupt context. The most common mistake is passing a NULL or uninitialized handle, which will lead to undefined behavior or system crashes. Ensure the target task is actually waiting for notifications using ca_notify_wait, otherwise the incremented value will simply accumulate until the task eventually calls wait.

Uso:

- Use this function when you need to wake up a task or signal that a specific event has occurred, such as a data processing task waiting for a new buffer. It is typically paired with ca_notify_wait in the receiver task, which blocks until a notification is received. In a standard workflow, you initialize the system, create the task, and then use this function to trigger logic inside that task's main loop. It is much faster than using a mutex or a queue for simple 'event occurred' signaling.

Esempio:

```
/* Example: Signaling a processing task from a producer task */
ca_task_handle_t worker_task_handle;

void producer_task(void *arg) {
    while (1) {
        /* Perform some work */

        /* Notify the worker task that data is ready */
        if (!ca_notify_give_inc(worker_task_handle)) {
            /* Handle error: notification failed */
```

```
        }

        ca_os_delay_ms(100);
    }
}

void worker_task(void *arg) {
    while (1) {
        /* Wait for notification from producer */
        if (ca_notify_wait(0xFFFFFFFF)) {
            /* Process the event */
        }
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_notify_give_inc instead of a full semaphore or queue?
A: Use ca_notify_give_inc for lightweight inter-task signaling when you need to count events, such as incoming data packets or button presses. It is designed to avoid the overhead of full semaphores or queues and is typically used in producer-consumer patterns where the producer signals work and the consumer processes it via ca_notify_wait.

Code example conf:0.90 **Q:** How do I implement a simple producer task that signals another task using ca_notify_give_inc?
A: You can signal a task with index 2 as follows: u32 target_task = 2; if (ca_notify_give_inc(target_tas { /* Success */ } else { /* Handle failure */ }

Parameters conf:1.00 **Q:** What are the requirements for the task_idx parameter in ca_notify_give_inc?
A: The task_idx parameter must be a u32 representing the unique identifier of the target task. This identifier must correspond to a valid task that has been previously registered in the OS.

Return conf:1.00 **Q:** What does a false return value from ca_notify_give_inc indicate?
A: A return value of false indicates that the notification was not successfully sent. This can occur if the provided task index is invalid or if the internal notification buffer is full.

`ca_notify_give_isr`

Header: `caos/inc/notify.h:143`
GUARD: `EN_NOTIFY`

```
static inline bool ca_notify_give_isr(ca_task_handle_t h, u32 value)
```

Sends a notification value to a specific task handle from an ISR by overwriting any existing pending value.

Parametri:

Nome	Tipo	Descrizione
`h`	`ca_task_handle_t`	The handle of the target task that should receive the notification. Must be a valid handle obtained via ca_os_task_handle or similar.
`value`	`u32`	The 32-bit data value to send. This can be a status code, a bitmask, or a raw data value.

Ritorna:
Returns true if the notification was successfully sent to the task, or false if the task handle is invalid or the system is in an invalid state.

Note:

- This function is specifically designed for use within an Interrupt Service Routine (ISR). It is non-blocking, meaning it will never pause the execution of the interrupt to wait for resources, ensuring low latency. Because it uses the 'overwrite' action, it is perfect for scenarios where you only care about the latest state (e.g., the most recent sensor reading) and want to discard stale, unread data. A common beginner mistake is calling a non-ISR function (like ca_notify_give) inside an interrupt; this will likely cause a hard fault or system hang because non-ISR functions may attempt to block the thread, which is forbidden in an ISR context. Always ensure the target task handle is initialized before the interrupt is enabled.

Uso:

- This function is used to bridge the gap between hardware events and software tasks. It is typically used in an ISR to wake up a task that is waiting for data. The standard workflow involves the target task calling ca_notify_wait to enter a blocked state, waiting for the event. When the hardware interrupt fires, the ISR calls ca_notify_give_isr to pass the data and unblock the task. Once the task resumes, it processes the value and returns to the wait state, creating a clean producer-consumer pattern. It is often paired with ca_os_task_handle to identify the recipient task during system initialization.

Esempio:

```c
/* Example: Signaling a task from a hardware timer interrupt */
void Timer_ISR_Handler(void) {
    /* Retrieve the handle of the task that processes timer events */
    ca_task_handle_t target_task = ca_os_task_handle(2);
    u32 status_update = 0xAA;

    /* Send notification to the task. This is safe to call from an ISR. */
    if (target_task != NULL) {
        ca_notify_give_isr(target_task, status_update);
    }
}

/* Inside the target task loop */
void worker_task(void) {
    u32 received_value;
    while (1) {
        /* Wait for the notification sent by the ISR */
        if (ca_notify_wait(&received_value)) {
            if (received_value == 0xAA) {
                /* Perform processing logic */
            }
        }
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_notify_give_isr instead of a standard notification function?
A: You should use ca_notify_give_isr when signaling a task from an Interrupt Service Routine, such as a GPIO or UART interrupt. It is non-blocking and safe for ISR contexts. It is specifically intended for high-frequency events where only the most recent state is relevant, as it overwrites any existing pending notification.

Code example conf:0.90 **Q:** How do I implement a simple ISR that signals a task using ca_notify_give_isr?
A: void EXTI0_IRQHandler(void) { u32 event_id = 0x01; ca_notify_give_isr(5, event_id); }

Parameters conf:1.00 **Q:** What do the task_idx and value parameters represent in ca_notify_give_isr?

A: The task_idx is the unique index of the target task intended to receive the notification. The value is a 32-bit data payload, which can represent a status code, a bitmask, or a pointer address to be processed by the target task.

Return conf:1.00 **Q:** What does the boolean return value indicate, and why might it return false?
A: The function returns true if the notification was successfully sent. It returns false if the provided task_idx is invalid or if the system is currently in an invalid state.

`ca_notify_peek`

Header: `caos/inc/notify.h:161`
GUARD: `EN_NOTIFY`

```
static inline u32 ca_notify_peek(u32 timeout_ms)
```

Checks if a notification is pending without blocking the task execution.

Parametri:

Nome	Tipo	Descrizione
`timeout_ms`	`u32`	The maximum time in milliseconds to wait for a notification. Use 0 to return immediately if no notification is currently available.

Ritorna:
Returns the notification value (u32) if one is available, or 0 if the timeout expires without receiving a notification.

Note:

- This function is a non-blocking wrapper that checks the task's notification state. It is thread-safe and intended to be called from task context; it must NEVER be called from an ISR, as it may involve scheduler operations that are invalid in interrupt context. Because it does not consume the notification if none is present, it is ideal for polling scenarios where you want to check for work without stalling the entire thread. A common beginner mistake is assuming that a return value of 0 always means 'no notification', when in fact 0 could be a valid notification value sent by another task; always ensure your application logic treats 0 as a specific signal or uses a non-zero value for valid data.

Uso:

- Use this function when you need to perform periodic background tasks but want to react immediately if a specific event occurs. It is typically used in a loop where the task performs some work, calls ca_notify_peek to check for incoming commands, and then continues if nothing is found. It is often paired with ca_notify_give or ca_notify_give_inc, which are called by other tasks to trigger the notification. It is common to use this in conjunction with ca_notify_clear to reset the state after the notification has been successfully processed.

Esempio:

```
/* Example: Polling for a command without blocking the main loop */

void task_worker(void *arg) {
    u32 command;
    while (1) {
        /* Perform some background processing */
        ca_os_delay_ms(10);

        /* Check if a command was sent, wait max 0ms (non-blocking) */
        command = ca_notify_peek(0);
```

```

        if (command != 0) {
            /* Process the received command */
            if (command == 0xAA) {
                /* Execute specific action */
            }
            /* Clear the notification after processing to allow future signals
   */
            ca_notify_clear();
        }
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_notify_peek instead of ca_notify_wait?
A: Use ca_notify_peek when you need to perform periodic background tasks while remaining responsive to events. It is ideal for polling scenarios where you want to check for work without stalling the entire thread, as it is a non-blocking wrapper.

Code example conf:0.90 **Q:** Provide a minimal example of how to poll for a notification using ca_notify_peek.
A: void task_loop(void) { u32 val; while(1) { do_work(); val = ca_notify_peek(0); if(val != 0) { process_notification(val); ca_notify_clear(); } } }

Parameters conf:1.00 **Q:** What is the significance of the timeout_ms parameter in ca_notify_peek?
A: The timeout_ms parameter defines the maximum time in milliseconds to wait for a notification. Setting it to 0 causes the function to return immediately if no notification is currently available.

Return conf:1.00 **Q:** Does a return value of 0 from ca_notify_peek always indicate that no notification was received?
A: Not necessarily. While 0 is returned if the timeout expires without a notification, 0 could also be a valid notification value sent by another task. Application logic should treat 0 as a specific signal or ensure that valid notifications use non-zero values.

`ca_notify_send_handle`

Header: `caos/inc/notify.h:83`
Source: `caos/src/notify.c:100:109`
GUARD: `EN_NOTIFY`

```
bool ca_notify_send_handle(ca_task_handle_t h, u32 value, u8 action)
```

Sends a notification value to a specific task using its handle, ensuring thread safety by masking interrupts.

Parametri:

Nome	Tipo	Descrizione
`h`	`ca_task_handle_t`	The handle of the target task. Must be a valid handle obtained from ca_os_task_handle or similar.
`value`	`u32`	The 32-bit data payload to send to the task.
`action`	`u8`	The notification action type (e.g., CA_NOTIFY_ACTION_OVERWRITE or CA_NOTIFY_ACTION_INCREMENT).

Ritorna:

Returns true if the notification was successfully sent to the target task, false if the handle is invalid or the operation failed.

Note:

- This function is thread-safe because it internally masks interrupts using ___disable_irq() to prevent the task handle from becoming invalid during the lookup process. It is not intended to be called directly from an ISR; use ca_notify_send_handle_isr instead if you are inside an interrupt context. A common mistake is passing an uninitialized or stale task handle, which will cause the function to return false because the internal lookup will fail. Because this function disables interrupts, keep the call duration short to avoid impacting system real-time responsiveness.

Uso:

- This function is typically used for inter-task communication where one task needs to signal another without knowing its internal index. It is often paired with ca_notify_wait in the receiver task to block until data arrives. In a typical lifecycle, you obtain the receiver's handle during system initialization, store it, and then use this function periodically to send updates or commands to that specific task. It is the preferred way to communicate when you have a handle reference rather than a raw task index.

Esempio:

```
/* Assume 'receiver_handle' was obtained earlier via ca_os_task_handle */

void send_update_to_task(ca_task_handle_t receiver_handle) {
    u32 data = 0x12345678;

    /* Send the value to the receiver task using the handle with an overwrite action */
    /* CA_NOTIFY_ACTION_OVERWRITE is defined in the system headers */
    bool success = ca_notify_send_handle(receiver_handle, data, 1);

    if (!success) {
        /* Handle error: perhaps the task has been destroyed or the handle is invalid */
        ca_printf("Failed to send notification to task!\n");
    }
}
```

Q&A — fine-tuning review

4/4 pairs · gemini-3.1-flash-lite

Usage conf:1.00 **Q:** When should I use ca_notify_send_handle instead of other communication methods?
A: Use this function for inter-task communication when you have a task handle and need to signal another task without knowing its internal index. It is typically paired with ca_notify_wait in the receiver task to block until data arrives, making it ideal for sending updates or commands to a specific task.

Code example conf:0.90 **Q:** How do I implement a simple notification send in my caOS task?
A: Assuming you have a stored handle 'target_handle', you can send a notification as follows: bool success = ca_notify_send_handle(target_handle, (u32)0xABCD, (u8)CA_NOTIFY_ACTI

Parameters conf:1.00 **Q:** What are the requirements for the 'h' and 'action' parameters?
A: The 'h' parameter must be a valid ca_task_handle_t obtained from ca_os_task_handle or similar. The 'action' parameter defines the notification type, such as CA_NOTIFY_ACTION_OVER or CA_NOTIFY_ACTION_INCREMENT.

Return conf:1.00 **Q:** What does a return value of false indicate when calling ca_notify_send_handle?
A: A return value of false indicates that the operation failed, most commonly because the provided task handle is invalid, uninitialized, or stale, causing the internal lookup to fail.

ca_notify_send_handle_isr

Header: caos/inc/notify.h:86
Source: caos/src/notify.c:111:113
GUARD: EN_NOTIFY

```
bool ca_notify_send_handle_isr(ca_task_handle_t h, u32 value, u8 action)
```

Sends a notification value to a specific task from within an Interrupt Service Routine (ISR).

Parametri:

Nome	Tipo	Descrizione
h	ca_task_handle_t	The handle of the target task that should receive the notification. Must be a valid handle obtained via ca_os_task_handle.
value	u32	The 32-bit data or bitmask to send to the target task.
action	u8	The operation to perform on the task's notification register (e.g., CA_NOTIFY_ACTION_SET_BITS to perform a bitwise OR).

Ritorna:
Returns true if the notification was successfully sent to the task, or false if the handle is invalid or the operation failed.

Note:

- This function is specifically designed to be called from an ISR context. It is non-blocking, meaning it will not wait for the target task to process the notification, ensuring the interrupt routine finishes quickly. It is thread-safe because it internally masks interrupts to safely resolve the task handle and update the notification register. A common beginner mistake is passing a NULL or uninitialized task handle, which will cause the function to return false and the notification to be lost. Always ensure the target task is initialized and waiting for notifications before sending them.

Uso:

- This function is typically used in hardware interrupt handlers (like USB, UART, or GPIO interrupts) to signal a background task that new data has arrived or a state change has occurred. It is usually paired with ca_notify_wait in the target task, which blocks until the notification is received. The typical sequence is: the task calls ca_notify_wait to sleep, the hardware triggers an interrupt, the ISR calls ca_notify_send_handle_isr to wake the task, and the task resumes execution to process the data.

Esempio:

```
/* Global handle for the task that processes USB events */
static ca_task_handle_t s_usb_task;

/* Interrupt handler triggered by USB hardware */
void USB_IRQHandler(void) {
    /* ... hardware specific interrupt clearing code ... */

    /* Notify the USB task that new data is available using bitwise OR */
    bool success = ca_notify_send_handle_isr(s_usb_task, 0x01,
   CA_NOTIFY_ACTION_SET_BITS);
```

```

    if (!success) {
        /* Handle error: notification failed, perhaps task is not ready */
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_notify_send_handle_isr instead of a standard notification function?
A: You should use ca_notify_send_handle_isr specifically when you are inside an Interrupt Service Routine (ISR), such as those triggered by UART, USB, or GPIO hardware events. It is designed to be non-blocking to ensure the interrupt routine completes quickly, and it is thread-safe because it internally masks interrupts to safely update the target task's notification register.

Code example conf:0.90 **Q:** How do I implement a simple notification send from an ISR in caOS?
A: Assuming you have a global task handle 'my_task_h', you would call it like this: void EXTI0_IRQHandler(void) { u32 val = 0x01; ca_notify_send_handle_isr(my_task_h, val, CA_NOTIFY_ACTION_SET_BITS); }

Parameters conf:1.00 **Q:** What are the requirements for the 'h' and 'action' parameters in ca_notify_send_handle_isr?
A: The 'h' parameter must be a valid ca_task_handle_t obtained via ca_os_task_handle; passing a NULL or uninitialized handle will cause the function to fail. The 'action' parameter defines the operation on the notification register, such as CA_NOTIFY_ACTION_SET_BITS, which performs a bitwise OR with the provided u32 value.

Return conf:1.00 **Q:** What does a return value of false indicate when calling ca_notify_send_handle_isr?
A: A return value of false indicates that the notification was not successfully sent. This is typically caused by providing an invalid or uninitialized task handle, or if the internal operation failed. If this happens, the notification is lost.

`ca_notify_take`

Header: `caos/inc/notify.h:152`
GUARD: `EN_NOTIFY`

```
static inline u32 ca_notify_take(u32 timeout_ms)
```

Blocks the current task until a notification is received or the specified timeout expires, automatically clearing the notification value.

Parametri:

Nome	Tipo	Descrizione
`timeout_ms`	`u32`	Maximum time in milliseconds to wait. Use 0 for no wait (poll) or a specific value (e.g., 2000) to block. Use 0xFFFFFFFF for an infinite wait.

Ritorna:
Returns the notification value (a non-zero u32) if received successfully. Returns 0 if the timeout expired before a notification was received.

Note:

- This function is blocking, meaning the task will be suspended by the scheduler until a notification arrives or the timer expires. It is NOT thread-safe to call from an ISR; use ca_notify_give_isr or similar functions instead. A common mistake is assuming the return value 0 always means an error, when in fact it simply indicates a timeout occurred. Ensure

the task has been properly initialized before calling this, as it relies on the internal task handle of the caller. Because this function automatically clears the notification, it is ideal for simple event signaling where you only care that an event occurred, not how many times it was triggered.

Uso:

- Typically used in a producer-consumer pattern where a task waits for an event or data from another task or an interrupt. It is often placed inside a 'while(ca_os_running())' loop to continuously process incoming signals. It is frequently paired with ca_os_self_handle to identify the receiver and ca_notify_give or ca_notify_give_inc to trigger the notification from another context. It is the standard way to implement a low-power wait state where the task consumes zero CPU cycles while waiting for external input.

Esempio:

```c
/* Example: Task waiting for a signal from an interrupt or another task */
#include "ca_os.h"

void worker_task(void *arg) {
    /* Get the handle of the current task to be used by others to send
    notifications */
    /* Note: ca_os_self_handle is used to identify this task for notification
    targeting */

    while(ca_os_running()) {
        /* Wait up to 1000ms for a notification */
        u32 signal_value = ca_notify_take(1000);

        if (signal_value != 0) {
            /* Notification received, process the value */
            ca_stdout_printf_n("Task received signal: 0x%08X", signal_value);
        } else {
            /* Timeout occurred, perform periodic maintenance or check status */
            ca_stdout_printf_n("Task timed out waiting for signal");
        }
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_notify_take instead of a simple flag variable in my task loop?
A: You should use ca_notify_take to avoid wasting CPU cycles in a busy-wait loop. It allows the scheduler to suspend your task until an event occurs, making it significantly more efficient for producer-consumer patterns like waiting for sensor readings or message arrivals.

Code example conf:0.90 **Q:** How do I implement a basic task loop that waits for a notification using ca_notify_take?
A: You can implement it as follows: while(1) { u32 val = ca_notify_take(1000); if(val > 0) { /* process val */ } }

Parameters conf:1.00 **Q:** What are the valid values for the timeout_ms parameter in ca_notify_take?
A: The timeout_ms parameter accepts 0 for non-blocking polling, a specific u32 value for a defined blocking duration in milliseconds, or a very large number/defined constant for an infinite wait.

Return conf:1.00 **Q:** How can I distinguish between a valid notification value of 0 and a timeout?
A: The function returns 0 if the timeout expires. If your application logic uses 0 as a valid notification value, you may misinterpret a timeout. To avoid this, ensure the sender task always uses a non-zero value for notifications.

ca_notify_wait

Header: caos/inc/notify.h:105
Source: caos/src/notify.c:133:186
GUARD: EN_NOTIFY

```
u32 ca_notify_wait(bool clear_on_exit, u32 timeout_ms)
```

Blocks the current task until a notification is received or a timeout expires.

Parametri:

Nome	Tipo	Descrizione
clear_on_exit	bool	If true, resets the internal notification value to 0 after reading it. If false, the value remains stored until manually cleared.
timeout_ms	u32	Maximum time in milliseconds to wait. Use CA_WAIT_FOREVER (usually 0) to wait indefinitely.

Ritorna:
Returns the non-zero notification value if received, or 0 if the timeout expired or the context is invalid.

Note:

- This function is blocking, meaning the task will be suspended by the scheduler until a notification arrives or the timeout occurs. It is NOT callable from an ISR because it performs a blocking wait and relies on the scheduler to yield. It is thread-safe as it uses internal spinlocks to protect the task's notification state. A common beginner mistake is to assume that a return value of 0 always means a timeout; it can also occur if the task context is invalid or if the notification value was already 0. Always check the return value against 0 before processing the data.

Uso:

- This function is typically used in a task loop to wait for events triggered by ISRs or other tasks. It is often paired with ca_notify_send_handle or ca_notify_send_handle_isr, which act as the producers of the notification. The typical sequence involves entering a while(1) loop, calling ca_notify_wait to block until an event occurs, processing the returned bitmask, and optionally clearing the notification for the next cycle. It is the standard way to synchronize a task with asynchronous hardware events without busy-waiting.

Esempio:

```c
/* Task waiting for a system event */
void event_handler_task(void *arg) {
    (void)arg;
    while (1) {
        /* Wait up to 1000ms for a notification, clear it after reading */
        u32 event_mask = ca_notify_wait(true, 1000);

        if (event_mask != 0) {
            /* Process specific bits in the notification mask */
            if (event_mask & 0x01) {
                ca_stdout_printf_n("Event 1 received");
            }
        } else {
```

```
            /* Handle timeout case: no event received within 1000ms */
            ca_stdout_printf_n("No event received within timeout");
        }
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_notify_wait in a caOS application?
A: You should use ca_notify_wait when a task needs to wait for a notification from another task or an ISR. It is useful for inter-task communication or synchronization, especially when the waiting task must block until a specific event occurs or a timeout happens.

Code example conf:0.90 **Q:** Can you show a minimal example of using ca_notify_wait in a task?
A: u32 notify_val = ca_notify_wait(true, 1000); if (notify_val > 0) { // Process the received notification value } else { // Timeout or error occurred }

Parameters conf:0.95 **Q:** What does the clear_on_exit parameter do in ca_notify_wait?
A: If clear_on_exit is true, the notification value is cleared after being read. If false, the value remains available for future calls to ca_notify_wait. This allows multiple tasks or repeated checks to access the same notification value.

Return conf:0.90 **Q:** What do the return values of ca_notify_wait indicate?
A: A return value of 0 indicates either a timeout or an error (e.g., invalid task context). A return value greater than 0 indicates a successful notification with the value received.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_notify_wait?
A: Common mistakes include calling ca_notify_wait from an ISR, which can cause crashes or undefined behavior. Also, using timeout_ms = 0 in a critical section without explicit control may lead to indefinite blocking. Additionally, calling this function from a non-task context will result in errors.

Behavior conf:0.90 **Q:** Is ca_notify_wait blocking or non-blocking, and is it safe to call from an ISR?
A: ca_notify_wait is a blocking function that suspends the calling task until a notification is received or the timeout expires. It is not safe to call from an ISR; it must only be used from tasks. For ISR notifications, use ca_notify_send_isr().

`ca_ntp_service_bind`

Header: `caos/net/inc/ntp_service.h:105`
Source: `caos/net/src/ntp_service.c:290:293`
GUARD: `EN_NTP_SERVICE`

```
bool ca_ntp_service_bind(void)
```

Registers the NTP service UDP port and assigns the callback function for incoming packets.

Ritorna:
Returns true if the UDP port was successfully bound to the NTP service, false otherwise (e.g., port already in use or network stack not initialized).

Note:

- This function is not thread-safe and must be called during the system initialization phase before the network stack starts processing traffic. It is not callable from an ISR. The most common mistake is calling this before the network protocol stack has been initialized via ca_net_proto_init, which will cause the binding to fail silently or return false. It does not start the NTP task itself; it only prepares the UDP socket listener.

Uso:

- This function is typically used during the system boot sequence alongside other service bindings like ca_dns_service_bind or ca_udp_server_bind. It is part of the initialization chain: first, you initialize the network protocol, then you bind the specific service ports, and finally, you signal that the network is ready using ca_ntp_service_net_ready to allow the NTP task to begin its operations.

Esempio:

```
/* Initialize network stack first */
ca_net_proto_init(&net_cfg);

/* Bind the NTP service to port 123 */
if (ca_ntp_service_bind()) {
    ca_stdout_put_n("NTP service bound successfully");
} else {
    ca_stdout_put_n("Failed to bind NTP service");
}

/* Poll link status and wait for stability */
ca_eth_link_poll();
ca_os_sleep(200);

/* Signal that the network is ready to start NTP synchronization */
ca_ntp_service_net_ready();
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_ntp_service_bind() and why is it important for NTP functionality?
A: You should call ca_ntp_service_bind() after initializing the network stack to register a UDP handler for receiving NTP packets on port 123. This function is essential for enabling the NTP service, which allows the system to synchronize time with NTP servers. It must be called after network initialization and cannot be invoked from an ISR or in a thread-unsafe manner.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_ntp_service_bind() with caOS types?
A: bool result = ca_ntp_service_bind(); if (result) { // NTP service successfully bound } else { // Handle error, e.g., log or retry }

Return conf:0.90 **Q:** What do the return values of ca_ntp_service_bind() indicate, and what causes it to return false?
A: The function returns true if the UDP handler for NTP (port 123) was successfully registered. It returns false if an error occurs, such as the port being already occupied or insufficient system resources. Common causes of failure include calling the function before the network is initialized or resource exhaustion in the system.

`ca_ntp_service_get_unix_time`

Header: `caos/net/inc/ntp_service.h:136`
Source: `caos/net/src/ntp_service.c:329:335`
GUARD: `EN_NTP_SERVICE`

```
u32 ca_ntp_service_get_unix_time(void)
```

Retrieves the current Unix timestamp synchronized via the NTP service.

Ritorna:

Returns a 32-bit unsigned integer representing the Unix timestamp (seconds since Jan 1, 1970). Returns 0 if the system has not yet synchronized with an NTP server.

Note:

- This function is thread-safe because it uses a mutex to protect the shared time variable during access. It is a blocking call in the sense that it must wait for the mutex to be available, so it must never be called from an Interrupt Service Routine (ISR) or a critical section where blocking is prohibited. The NTP service must be initialized and running for this to return a valid time; otherwise, it will consistently return 0. A common mistake is assuming the time is valid immediately after boot without checking if the return value is non-zero.

Uso:

- This function is typically used by application tasks that require wall-clock time for logging, timestamping data, or scheduling events. It is usually called after the system has established network connectivity and the NTP service has successfully performed a sync. It is often paired with 'ca_ntp_service_init' to set up the service and 'ca_ntp_service_sync_now' if an immediate update is required before reading the time.

Esempio:

```
u32 current_time = ca_ntp_service_get_unix_time();
if (current_time != 0U) {
    /* Time is valid, proceed with logging or processing */
    ca_printf_n("Current Unix Time: %u", current_time);
} else {
    /* Time not yet synchronized, handle accordingly */
    ca_printf_n("NTP time not available yet.");
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_ntp_service_get_unix_time() and why is it important for my embedded application?
A: You should call ca_ntp_service_get_unix_time() after the NTP service has been initialized and has successfully received a valid time from an NTP server. This function is important because it provides a synchronized real-time value that can be used for logging, scheduling, or any time-sensitive operations in your embedded application. It ensures that your system maintains accurate time, which is crucial for network protocols, data integrity, and user experience.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_ntp_service_get_unix_time() with caOS types?
A: u32 current_time = ca_ntp_service_get_unix_time(); if (current_time != 0) { // Time is synchronized, use current_time // For example, log timestamp // Note: actual logging would depend on your system's logging mechanism } else { // Time not yet synchronized // Handle accordingly, e.g., retry or use fallback time }

Return conf:0.90 **Q:** What do the return values of ca_ntp_service_get_unix_time() mean, and what causes the function to return 0?
A: The function returns a u32 representing the number of seconds since January 1, 1970 (UNIX time). If the function returns 0, it indicates that the time has not yet been synchronized via NTP. This can happen if the NTP service has not been initialized, or if it has not yet received a valid time from an NTP server. The function will block if the internal mutex is already acquired by another thread, and it is not safe to call from an ISR.

ca_ntp_service_init

Header: caos/net/inc/ntp_service.h:83
Source: caos/net/src/ntp_service.c:262:282
GUARD: EN_NTP_SERVICE

```
void ca_ntp_service_init(ca_ipv4_t server_ip, ca_mac_t local_mac,
    ca_ntp_sync_cb_t on_sync, ca_ntp_fail_cb_t on_fail)
```

Initializes the internal state and callback handlers for the NTP network time synchronization service.

Parametri:

Nome	Tipo	Descrizione
server_ip	ca_ipv4_t	The IPv4 address of the primary NTP server to query.
local_mac	ca_mac_t	The MAC address of the local network interface used for NTP communication.
on_sync	ca_ntp_sync_cb_t	Callback function executed when a successful time synchronization occurs.
on_fail	ca_ntp_fail_cb_t	Callback function executed if the NTP synchronization process fails.

Ritorna:
void. This function does not return a status code; it assumes successful initialization of internal structures.

Note:

- This function is not thread-safe and must be called during the system initialization phase before the scheduler starts or before the NTP service is started. It is strictly non-blocking and must NOT be called from an ISR, as it performs mutex initialization and potential CLI registration. A common mistake is calling this function after ca_ntp_service_start, which will reset the service state and potentially cause synchronization loss or undefined behavior in the network stack.

Uso:

- This function is the mandatory first step in the NTP service lifecycle. It is typically called during the system boot sequence, often after network interface configuration but before calling ca_ntp_service_start. It is frequently paired with ca_dhcp_service_init, where the DHCP callback might later update the NTP server address using ca_ntp_service_set_server once the network environment is discovered.

Esempio:

```
/* Define callback functions for NTP events */
void on_ntp_sync(void) { /* Handle successful sync */ }
void on_ntp_fail(void) { /* Handle sync failure */ }

void app_init(void) {
    ca_ipv4_t ntp_server = { .addr = 0x0A000001 }; // Example IP: 10.0.0.1
    ca_mac_t my_mac = { .addr = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55} };

    /* Initialize NTP service with callbacks */
    ca_ntp_service_init(ntp_server, my_mac, on_ntp_sync, on_ntp_fail);

    /* Start the service after initialization */
    ca_ntp_service_start();
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_ntp_service_init and why is it important for an embedded system?
A: You should call ca_ntp_service_init to initialize the NTP service for time synchronization with an NTP server. It's important for embedded systems that require accurate timekeeping, such as logging events, managing timeouts, or coordinating network operations. This function prepares the NTP service with necessary configuration like server IP, local MAC address, and callback functions for success or failure handling.

Code example conf:0.90 **Q:** Can you provide a minimal working C example using ca_ntp_service_init with caOS types?
A:

```
u32 server_ip = 0xC0A80001; // 192.168.0.1
u8 local_mac[6] = {0x00, 0x80, 0xE1, 0x23, 0x45, 0x67};
ca_ntp_service_init(server_ip, local_mac, NULL, NULL);
```

Parameters conf:0.95 **Q:** What does the server_ip parameter represent, and what are valid values for it?
A: The server_ip parameter is the IPv4 address of the NTP server. It must be a valid IPv4 address in network byte order, such as 0xC0A80001 for 192.168.0.1. If set to 0, the server is considered not configured, and the function may ignore further NTP operations.

Return conf:0.95 **Q:** Does ca_ntp_service_init return any value, and what does it mean if it does?
A: The function does not return any value. It is void. The function is designed not to fail, but it may ignore invalid parameters such as an invalid server IP or MAC address.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_ntp_service_init?
A: Common mistakes include calling the function from an ISR, which is not allowed, or passing an invalid MAC address array. Another edge case is calling the function after starting the NTP service, which may lead to undefined behavior. Additionally, if the server IP is set to 0, the NTP service will not attempt synchronization.

Behavior conf:0.90 **Q:** Is ca_ntp_service_init blocking or non-blocking, and is it safe to use from an ISR or in a multi-threaded environment?
A: The function is non-blocking and not safe to call from an ISR. It is also not thread-safe unless protected by a mutex. It must be called before starting the NTP service, and it resets synchronization flags and server data.

`ca_ntp_service_net_ready`

Header: `caos/net/inc/ntp_service.h:112`
Source: `caos/net/src/ntp_service.c:295:299`
GUARD: `EN_NTP_SERVICE`

```
void ca_ntp_service_net_ready(void)
```

Notifies the NTP service that the network interface is ready for operation.

Ritorna:
None (void function).

Note:

- This function is non-blocking and thread-safe, as it uses a queue to post an event to the NTP service task. It is safe to call from an ISR because it uses the 'trysend' variant of the queue API, which does not block if the queue is full. The primary precondition is that the NTP service must have been initialized previously. A common mistake is calling this

before the network stack has actually acquired an IP address or before the NTP service task has been started, which may result in the event being ignored or the service failing to transition to the 'ready' state.

Uso:

- This function is typically used in the network initialization sequence, specifically after a DHCP client has successfully obtained an IP address or a static IP has been configured. It acts as a trigger to transition the NTP service from an idle or waiting state into an active state where it can begin sending time synchronization requests. It is often paired with ca_ntp_service_set_server to ensure the service knows where to send requests before it starts the synchronization process.

Esempio:

```
/* Example: Triggering NTP service after network configuration */
void on_network_up(void) {
    ca_ipv4_t ntp_server_ip;
    /* Assume IP is retrieved from a configuration or DHCP service */
    ntp_server_ip.addr = 0xC0A80101; /* 192.168.1.1 */

    /* Configure the server address for the NTP service */
    ca_ntp_service_set_server(ntp_server_ip);

    /* Notify the NTP service that the network is ready to use */
    ca_ntp_service_net_ready();

    /* Now the service can start sending UDP packets */
}
```

Q&A — fine-tuning review *2/2 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_ntp_service_net_ready() and why is it important for NTP synchronization?
A: You should call ca_ntp_service_net_ready() after the network interface is fully initialized and configured, and after setting up the NTP server details. This function notifies the NTP service that the network is ready to synchronize time, allowing the service to proceed with time synchronization. It is important because without this notification, the NTP service will not attempt to connect to the NTP server, and time synchronization will not occur.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using caOS types that demonstrates how to use ca_ntp_service_net_ready()?
A: void example_ntp_usage(void) { // Assume network is initialized and NTP server is configured ca_ntp_service_net_ready(); // Notify NTP service that network is ready // The function is non-blocking and can be called from ISR }

`ca_ntp_service_rx_handler`

Header: `caos/net/inc/ntp_service.h:118`
Source: `caos/net/src/ntp_service.c:301:308`
GUARD: `EN_NTP_SERVICE`

```
void ca_ntp_service_rx_handler(const ca_udp_frame_t *udp, u16 len)
```

Processes incoming NTP UDP packets and forwards them to the NTP service task for synchronization.

Parametri:

Nome	Tipo	Descrizione
udp	const ca_udp_frame_t *	Pointer to the raw UDP frame structure containing the NTP payload.
len	u16	Total length of the UDP payload in bytes.

Ritorna:
void. This function does not return a status code; it silently discards malformed packets.

Note:

- This function is designed to be called from the network stack's receive callback context. It is non-blocking because it uses 'ca_queue_static_trysend', which immediately returns if the internal queue is full, preventing the network stack from stalling. It is thread-safe assuming the underlying queue is properly initialized. A common mistake is assuming this function performs the actual time synchronization; it only parses the packet and queues an event, meaning the actual processing happens later in the NTP service task context.

Uso:

- This function is the bridge between the low-level network driver and the high-level NTP service. It is typically invoked by the network stack's RX dispatcher when a packet arrives on the NTP port. The sequence is: 1) The network stack receives a frame, 2) 'ca_ntp_service_rx_handler' parses the NTP response, 3) the event is queued, 4) the NTP service task wakes up to process the event, and 5) the caller must eventually call 'ca_net_buf_free' to release the memory buffer.

Esempio:

```
void my_network_rx_callback(u8 *frame, u16 total_len, u16 l4_offset, u16 buf_idx
    ) {
    // Calculate pointer to the UDP payload within the received frame
    const ca_udp_frame_t *udp = (const ca_udp_frame_t *)(frame + l4_offset);
    u16 udp_len = (u16)(total_len - l4_offset);

    // Pass the frame to the NTP service handler
    ca_ntp_service_rx_handler(udp, udp_len);

    // Always free the network buffer after processing is complete
    ca_net_buf_free(buf_idx);
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should ca_ntp_service_rx_handler be called, and what is its purpose in the NTP service?
A: ca_ntp_service_rx_handler should be called when an NTP response packet is received via UDP. Its purpose is to notify the NTP service of the received response by sending an event to the NTP queue. It is typically invoked from an ISR or network processing context when a UDP frame is received.

Code example conf:0.85 **Q:** Can you provide a minimal C code example showing how to use ca_ntp_service_rx_handler with caOS types?
A: ca_udp_frame_t udp_frame; void example_isr_handler(void) { u16 packet_len = 48; ca_ntp_service_rx_handler(&udp_frame, packet_len); }

Parameters conf:0.90 **Q:** What are the valid and invalid values for the udp and len parameters of ca_ntp_service_rx_handler?
A: The udp parameter must be a non-NULL pointer to a valid and aligned ca_udp_frame_t structure; otherwise, it will cause a crash. The len parameter must be greater than 0, as it is used for validation but not internally processed. Invalid values include NULL for udp or 0 for len.

Return conf:0.90 **Q:** What does ca_ntp_service_rx_handler return, and what can cause it to fail or ignore a packet?
A: The function does not return any value. It cannot fail in a strict sense, but it may ignore a packet if parsing fails. The function does not check if the queue is full, so if the queue is full, the event might be lost.

`ca_ntp_service_set_server`

Header: `caos/net/inc/ntp_service.h:89`
Source: `caos/net/src/ntp_service.c:316:320`
GUARD: `EN_NTP_SERVICE`

```
void ca_ntp_service_set_server(ca_ipv4_t server_ip)
```

Configures the primary NTP server IP address for the NTP service.

Parametri:

Nome	Tipo	Descrizione
`server_ip`	`ca_ipv4_t`	The IPv4 address of the NTP server to be used. Ensure the address is valid and reachable on the network.

Ritorna:
void

Note:

- This function is non-blocking and thread-safe as it simply updates internal configuration variables. It is safe to call from an ISR, though typically it is called from a configuration task. A critical precondition is that the NTP service must have been initialized via ca_ntp_service_init before calling this. The most common mistake is failing to call ca_ntp_service_net_ready after setting the server, which prevents the service from knowing that the network stack is prepared to perform the synchronization.

Uso:

- This function is used to dynamically update the NTP server address, often after receiving network configuration via DHCP or a static configuration file. It is typically paired with ca_ntp_service_init during system startup and ca_ntp_service_net_ready to signal that the stack is ready to use the new server. In dynamic environments, it is often called when a DHCP lease expires or changes to ensure the device remains synchronized with a valid time source.

Esempio:

```c
/* Example: Configuring NTP server after DHCP lease acquisition */
#include "ca_os.h"

void update_ntp_config(ca_ipv4_t new_server_ip) {
    /* Set the new server IP address */
    ca_ntp_service_set_server(new_server_ip);

    /* Signal the NTP service that the network is ready to perform requests */
    ca_ntp_service_net_ready();

    /* Trigger an immediate synchronization attempt */
    ca_ntp_service_sync_now();
}

```

```c
void system_init(void) {
    /* Initialize the NTP service before setting the server */
    ca_ntp_service_init();

    /* Define a server IP (e.g., 192.168.1.1) */
    ca_ipv4_t ntp_server = { .addr = 0x0101A8C0 };
    update_ntp_config(ntp_server);
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_ntp_service_set_server, and what is its purpose in an embedded system?
A: You should call ca_ntp_service_set_server when you need to configure the IP address of an NTP server for time synchronization in your embedded system. It is typically used during system initialization or when the network configuration changes. The function sets the NTP server IP and notifies the NTP task via a queue, allowing the system to synchronize time with the specified server.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_ntp_service_set_server with caOS types?
A: ca_ipv4_t server_ip = 0xC0A80101; /* 192.168.1.1 */ ca_ntp_service_set_server(server_ip);

Parameters conf:0.90 **Q:** What is the meaning of the ca_ipv4_t server_ip parameter, and what are valid and invalid values?
A: The ca_ipv4_t server_ip parameter specifies the IP address of the NTP server. Valid values are 32-bit representations of IPv4 addresses, such as 0xC0A80101 for 192.168.1.1. If the value is 0, the function will ignore the call and not update the NTP server IP. Any non-zero value is considered valid for setting the server IP.

Return conf:0.85 **Q:** Does ca_ntp_service_set_server return any value, and what does it indicate?
A: The function does not return any value. It is declared as void. According to the documentation, the function never fails, but it might not have an effect if the provided IP address is zero. It modifies a global variable and resets a MAC server flag, but does not report errors.

`ca_ntp_service_set_server_n`

Header: `caos/net/inc/ntp_service.h:92`
Source: `caos/net/src/ntp_service.c:322:327`
GUARD: `EN_NTP_SERVICE`

```c
void ca_ntp_service_set_server_n(u8 n, ca_ipv4_t server_ip)
```

Configures the IPv4 address for a specific NTP server index in the system's internal list.

Parametri:

Nome	Tipo	Descrizione
`n`	`u8`	The index of the NTP server to configure. Must be less than the defined constant NTP_SERVER_N. Values outside this range are ignored.
`server_ip`	`ca_ipv4_t`	The IPv4 address structure to assign to the specified index.

Ritorna:
void

Note:

- This function is non-blocking and thread-safe in the sense that it performs a simple memory write to a static array. It is safe to call from an ISR, though typically it is used in CLI or configuration tasks. The function resets the internal 'have_server_mac' flag for that index, which forces the network stack to re-resolve the server's MAC address via ARP before the next synchronization attempt. A common mistake is passing an index 'n' that is equal to or greater than NTP_SERVER_N, which will cause the function to silently return without updating any settings.

Uso:

- This function is used during the system configuration phase, often triggered by a CLI command or a configuration file parser. It is typically paired with ca_ntp_service_init to prepare the service, and later with ca_ntp_service_sync_now to trigger a time update once the network is ready. It allows the application to maintain a list of redundant NTP servers, ensuring that if one fails, the system can attempt to reach others.

Esempio:

```c
/* Example: Configuring the primary NTP server via CLI input */
ipv4_t new_ip;
char *ip_str = "192.168.1.10";

/* Parse the string into the ca_ipv4_t structure */
if (ca_parse_ipv4(ip_str, &new_ip)) {
    /* Set the server at index 0 */
    ca_ntp_service_set_server_n(0, new_ip);

    /* Notify the user of the change */
    ca_printf_n("NTP server 0 updated successfully");
} else {
    ca_printf_n("Invalid IP format provided");
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_ntp_service_set_server_n() in my caOS application?
A: You should call ca_ntp_service_set_server_n() during the initialization phase of your application to configure the NTP servers that will be used for time synchronization. This function is used to set up the IP addresses of NTP servers before the NTP service starts synchronizing the system time.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_ntp_service_set_server_n() with caOS types?
A: u8 server_index = 0; ipv4_t ntp_server_ip = {192, 168, 1, 100}; ca_ntp_service_set_server_n(serv ntp_server_ip);

Parameters conf:0.90 **Q:** What is the valid range for the parameter 'n' in ca_ntp_service_set_server_n() and what happens if I pass an invalid value?
A: The parameter 'n' must be between 0 and NTP_SERVER_N-1. If n is greater than or equal to NTP_SERVER_N, the function will return immediately without performing any action. Passing an invalid value will not cause a crash or error, but the configuration will be ignored.

`ca_ntp_service_start`

Header: `caos/net/inc/ntp_service.h:99`
Source: `caos/net/src/ntp_service.c:284:288`
GUARD: `EN_NTP_SERVICE`

```
void ca_ntp_service_start(void)
```

Initializes and launches the NTP background service task in the caOS scheduler.

Ritorna:
This function returns void and does not provide a status code; it assumes the system has sufficient resources to spawn the task.

Note:

- This function is non-blocking as it merely registers a new job with the caOS scheduler. It is not thread-safe to call this from an ISR, as it involves memory allocation and task creation within the OS kernel. Before calling this, you must ensure the system memory and OS scheduler are initialized. A common beginner mistake is calling this before the network stack or the OS scheduler is fully operational, which may lead to the task failing to start or crashing immediately upon execution.

Uso:

- This function is typically called during the system initialization phase, often inside the 'ca_os_main' function. It is part of a sequence where network services are brought up, such as calling 'ca_arp_service_start', 'ca_dhcp_service_start', and finally 'ca_ntp_service_start'. Once started, the NTP service runs as a background task, managing time synchronization independently of the main application logic.

Esempio:

```
void ca_os_main(void) {
    /* Initialize memory management before starting services */
    ca_memory_init();

    /* Start network infrastructure services */
    ca_arp_service_start();
    ca_dhcp_service_start();

    /* Start the NTP service to handle time synchronization */
    ca_ntp_service_start();

    /* Configure the NTP server address before binding */
    ca_ntp_service_set_server("pool.ntp.org");

    /* Bind the service to the network interface */
    if (!ca_ntp_service_bind()) {
        /* Handle error: NTP service failed to bind to the network */
    }

    /* Start the OS scheduler to run all registered jobs */
    ca_os_run();
}
```

Q&A — fine-tuning review *2/2 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When and why should I call ca_ntp_service_start() in a caOS application?
A: You should call ca_ntp_service_start() once during application initialization to start the NTP service task. This function is used to register an NTP job in the RTOS, enabling time synchronization. It is non-blocking and must be called from a non-ISR context. The function should only be called once, as it does not check if the task is already active. It is typically used in networked embedded applications where accurate timekeeping is required.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_ntp_service_start() with caOS types?
A: void example_ntp_usage(void) { ca_ntp_service_start(); // Task is now running in background for NTP synchronization }

`ca_ntp_service_sync_now`

Header: `caos/net/inc/ntp_service.h:124`
Source: `caos/net/src/ntp_service.c:310:314`
GUARD: `EN_NTP_SERVICE`

```
void ca_ntp_service_sync_now(void)
```

Triggers an immediate NTP time synchronization request by posting a sync event to the service queue.

Ritorna:
void

Note:

- This function is non-blocking and thread-safe because it uses ca_queue_static_trysend to post an event to the internal NTP service queue. It is safe to call from both task context and ISRs, provided the NTP service has been properly initialized via ca_ntp_service_init. The most common mistake is calling this function before the network stack or the NTP service is ready, which will result in the event being dropped or ignored by the service. It does not wait for the actual network exchange to complete; it merely signals the background NTP task to perform the sync operation as soon as possible.

Uso:

- This function is typically used in command-line interfaces (CLI) or system management tasks to force a time update without waiting for the next scheduled periodic sync. It is part of the NTP service lifecycle, which usually follows the sequence: ca_ntp_service_init -> ca_ntp_service_bind -> ca_ntp_service_start. Once running, this function is invoked whenever the user or an application logic determines that the current system time is stale or unreliable.

Esempio:

```
/* Example: Triggering a manual NTP sync from a CLI command */
#include "caos.h"

void handle_user_sync_command(void) {
    /* Check if the NTP service is initialized and network is ready */
    if (ca_ntp_service_net_ready()) {
        ca_put_n("Requesting immediate NTP sync...");

        /* Trigger the sync event */
        ca_ntp_service_sync_now();

        ca_put_n("Sync event queued successfully.");
    } else {
        /* Handle case where network is not ready */
        ca_put_n("Error: NTP service not ready or network down.");
    }
}
```

Q&A — fine-tuning review *2/2 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_ntp_service_sync_now() and what is the benefit of doing so?
A: You should call ca_ntp_service_sync_now() when you need to force an immediate time synchronization without waiting for the next scheduled update. This function is useful in scenarios where accurate time is critical and you want to ensure the system time is as up-to-date as possible, such as when a network connection is established or after a system boot. It is non-blocking and can be called from any context, including ISRs, making it safe to use in time-sensitive applications. However, note that calling this function does not guarantee immediate synchronization; it only sends an event to the NTP service queue for later processing.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_ntp_service_sync_now() with caOS types?
A: void example_function(void) { // Force immediate NTP synchronization ca_ntp_service_sync_now
}
// This function can be called from any context including ISRs // No additional caOS types are needed beyond the function call itself // The function does not return any value and is thread-safe

---

`ca_ntp_service_synced`

Header: `caos/net/inc/ntp_service.h:129`
GUARD: `EN_NTP_SERVICE`

```
static inline bool ca_ntp_service_synced(void)
```

Checks if the NTP service has successfully synchronized the system clock with an external server.

Ritorna:
Returns true if the NTP service is currently synchronized, false otherwise.

Note:

- This function is a non-blocking, thread-safe getter that reads an internal status flag. It is safe to call from both tasks and ISRs because it performs a simple memory read. It does not initiate any network activity or wait for a response; it only reports the current state of the NTP synchronization process. A common mistake is assuming this function triggers a sync; it does not. You must ensure ca_ntp_service_init and ca_ntp_service_start have been called previously, otherwise the flag will remain false indefinitely.

Uso:

- This function is typically used in diagnostic tasks or status reporting loops to verify if the system time is reliable. It is often paired with ca_ntp_service_get_unix_time to ensure that the time retrieved is accurate before using it for logging or timestamping events. In a typical application lifecycle, you initialize the NTP service, start it, and then periodically poll this function to decide whether to display 'SYNC' or 'WAIT' in your system telemetry or debug console.

Esempio:

```
/* Check if NTP is synced before logging a timestamped event */
if (ca_ntp_service_synced()) {
    /* NTP is ready, retrieve the synchronized Unix timestamp */
    u32 current_time = ca_ntp_service_get_unix_time();
    ca_stdout_printf("System time is valid: %u\n", current_time);
} else {
```

```
    /* NTP not ready, use local system tick count as a fallback */
    u32 uptime = ca_os_tick_64_ms();
    ca_stdout_printf("NTP not synced, using uptime: %u ms\n", uptime);
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_ntp_service_synced and what is its primary purpose?
A: You should call this function in diagnostic tasks or status reporting loops to verify if the system time is reliable. It is used to determine if the NTP service has successfully synchronized with an external server, allowing you to decide whether to display 'SYNC' or 'WAIT' in your telemetry or debug console.

Code example conf:1.00 **Q:** How do I implement a check for NTP synchronization in my caOS application?
A: Ensure you have called ca_ntp_service_init and ca_ntp_service_start before polling the status. Example: if (ca_ntp_service_synced()) { /* Time is reliable */ } else { /* Time is not yet synchronized */ }

Return conf:1.00 **Q:** What do the return values of ca_ntp_service_synced indicate, and why might it return false?
A: The function returns true if the NTP service is currently synchronized, and false otherwise. A return value of false indicates that the system clock is not synchronized, which occurs if the service has not yet completed a sync or if ca_ntp_service_init and ca_ntp_service_start were not called previously.

`ca_opamp_adc_channel`

Header: `caos/drivers/inc/opamp.h:38`
Source: `caos/drivers/src/opamp.c:39:46`
GUARD: `EN_OPAMP`

```
u8 ca_opamp_adc_channel(u32 num, u8 *adc_num); /* 0: output not routed
    to an ADC */
```

Retrieves the ADC channel index associated with a specific operational amplifier (OPAMP) hardware instance.

Parametri:

Nome	Tipo	Descrizione
`num`	`u32`	The hardware index of the OPAMP. Must be between 1 and the value returned by ca_opamp_count().
`adc_num`	`u8 *`	Pointer to a variable where the ADC channel number will be stored. If NULL, the function ignores this output.

Ritorna:
Returns the ADC channel index (1-based) if the OPAMP is successfully routed to an ADC, or 0 if the OPAMP is not routed or the input index is invalid.

Note:

- This function is a non-blocking, thread-safe wrapper that queries the hardware configuration. It is safe to call from any context, including ISRs, as it performs no complex synchronization or blocking operations. The most common mistake is passing an invalid OPAMP index (e.g., 0 or a value exceeding the hardware count), which will result in

a return value of 0 and no update to the adc_num pointer. Always ensure the OPAMP hardware has been initialized via ca_opamp_init() before attempting to query its routing.

Uso:

- This function is typically used during the initialization phase of an analog signal chain to dynamically determine which ADC channel is connected to a specific OPAMP output. It is usually paired with ca_opamp_count() to validate the index and ca_adc_pin_init() to configure the corresponding ADC pin. By using this function, you avoid hardcoding channel numbers, making your code portable across different STM32 hardware variants supported by caOS.

Esempio:

```c
/* Example: Initialize an OPAMP and prepare its associated ADC channel */
#include "caos.h"

void setup_analog_chain(u32 opa_index) {
    u8 adc_channel = 0U;

    /* Ensure the index is valid before querying */
    if (opa_index > 0U && opa_index <= ca_opamp_count()) {
        /* Retrieve the ADC channel mapped to this OPAMP */
        u8 ch = ca_opamp_adc_channel(opa_index, &adc_channel);

        if (ch != 0U) {
            /* Initialize the ADC pin using the retrieved channel mapping */
            ca_adc_pin_init(1U, adc_channel);
        } else {
            /* Handle error: OPAMP not routed to an ADC */
        }
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_opamp_adc_channel in my caOS application?
A: You should use it during the initialization phase of an analog signal chain to dynamically determine which ADC channel is connected to a specific OPAMP output. This avoids hardcoding channel numbers, ensuring portability across different STM32 hardware variants.

Code example conf:1.00 **Q:** How do I implement a check for an OPAMP's ADC channel in caOS?
A: u32 opamp_idx = 1; u8 channel = 0; if (opamp_idx <= ca_opamp_count()) { ca_opamp_adc_channel(opamp_idx, &channel); }

Parameters conf:1.00 **Q:** What are the constraints for the 'num' parameter in ca_opamp_adc_channel?
A: The 'num' parameter represents the hardware index of the OPAMP. It must be a value between 1 and the integer returned by ca_opamp_count(). Passing 0 or a value exceeding the hardware count is an invalid input.

Return conf:1.00 **Q:** What does a return value of 0 indicate from ca_opamp_adc_channel?
A: A return value of 0 indicates that either the OPAMP is not routed to an ADC, or the provided hardware index was invalid. In the case of an invalid index, the adc_num pointer will not be updated.

`ca_opamp_count`

Header: `caos/drivers/inc/opamp.h:35`
Source: `caos/drivers/src/opamp.c:14:17`

GUARD: `EN_OPAMP`

```
u32 ca_opamp_count(void); /* 0: none on this part */
```

Returns the number of operational amplifiers (OPAMPs) available on the current hardware platform.

Ritorna:
Returns a u32 representing the count of available OPAMP peripherals. A return value of 0 indicates that no OPAMPs are supported or configured on the current STM32 chip.

Note:

- This function is a non-blocking, thread-safe wrapper that queries the hardware abstraction layer. It is safe to call from both tasks and Interrupt Service Routines (ISRs) because it performs a simple read operation without requiring complex synchronization or waiting. The most common mistake is assuming that a non-zero return value guarantees that a specific OPAMP index is available; always verify the index against this count before attempting to initialize or use an OPAMP.

Uso:

- This function is typically used during the system initialization phase to dynamically detect hardware capabilities. It is often called before iterating through a loop to initialize multiple OPAMPs using ca_opamp_init. By checking the count first, you ensure your application code remains portable across different STM32 variants that may have varying numbers of internal OPAMP peripherals.

Esempio:

```
/* Check how many OPAMPs are available and initialize them if present */
u32 opamp_count = ca_opamp_count();

if (opamp_count > 0) {
    /* Initialize the first available OPAMP if the count is at least 1 */
    /* ca_opamp_init(0) is the standard way to start the first unit */
    ca_opamp_init(0);
} else {
    /* Log an error or handle the case where no OPAMP hardware is present */
    ca_printf("No OPAMPs detected on this hardware.\n");
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_opamp_count and what is the primary benefit of using it?
A: You should call ca_opamp_count during the system initialization phase. The primary benefit is that it allows your application to dynamically detect hardware capabilities, ensuring your code remains portable across different STM32 variants that feature varying numbers of internal OPAMP peripherals.

Code example conf:1.00 **Q:** How do I write a loop to check for available OPAMPs using caOS types?
A: You can use the following pattern: u32 count = ca_opamp_count(); for(u32 i = 0; i < count; i++) { /* Initialize OPAMP at index i */ }

Return conf:1.00 **Q:** What does a return value of 0 mean, and does a non-zero return value guarantee that any specific OPAMP index is valid?
A: A return value of 0 indicates that no OPAMPs are supported or configured on the current STM32 chip. A non-zero return value does not guarantee that a specific OPAMP index is available; you must always verify the index against the returned count before attempting to use an OPAMP.

ca_opamp_hw_adc_channel

Header: caos/drivers/inc/ca_glue_hw.h:417
GUARD: EN_CA_GLUE_HW

```
extern u8 ca_opamp_hw_adc_channel(u32 num, u8 *adc_num); /* ADC channel
    seeing the output, 0 = none */
```

Retrieves the hardware-specific ADC channel mapping for a given OPAMP instance.

Parametri:

Nome	Tipo	Descrizione
num	u32	The OPAMP instance number (e.g., 1 for OPAMP1, 2 for OPAMP2). Must be a valid index supported by the hardware.
adc_num	u8 *	Pointer to a memory location where the function will store the ADC unit index. Must not be NULL.

Ritorna:
Returns a u8 value representing the specific ADC channel index associated with the OPAMP output. Returns 0 if the OPAMP is not connected to an ADC or if the input is invalid.

Note:

- This function is a low-level hardware abstraction layer (HAL) helper. It is non-blocking, thread-safe, and safe to call from an ISR because it performs simple mapping calculations without accessing shared resources or performing I/O. The function does not perform strict validation on the 'num' parameter in all implementations; passing an invalid OPAMP number may result in incorrect or default return values. Always ensure the pointer passed to 'adc_num' is valid to avoid memory corruption, as the function writes directly to this address without checking for NULL.

Uso:

- This function is typically used during the initialization phase of an analog sensing chain. It is rarely called directly by application code; instead, it is invoked by the higher-level 'ca_opamp_adc_channel' function to resolve hardware-specific routing. Developers use it to determine which ADC channel must be configured to read the output of a specific OPAMP. It is part of the sequence: 'ca_opamp_init' -> 'ca_opamp_adc_channel' -> 'ca_adc_init'.

Esempio:

```
u8 adc_unit = 0;
u8 adc_channel = 0;

/* Determine which ADC channel is connected to OPAMP 1 */
/* We use the high-level wrapper which calls ca_opamp_hw_adc_channel internally
    */
adc_channel = ca_opamp_adc_channel(1, &adc_unit);

if (adc_channel != 0) {
    /* Proceed to initialize the ADC unit and channel found */
    /* ca_adc_init(adc_unit, adc_channel); */
} else {
    /* Handle error: OPAMP 1 not connected to any ADC */
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_opamp_hw_adc_channel in my application?
A: This function is a low-level HAL helper typically used during the initialization phase of an analog sensing chain. It is rarely called directly by application code; instead, it is intended to be invoked by the higher-level ca_opamp_adc_channel function to resolve hardware-specific routing before calling ca_adc_init.

Code example conf:1.00 **Q:** How do I correctly call ca_opamp_hw_adc_channel in C?
A: You must provide a valid OPAMP instance number and a pointer to a u8 variable to store the ADC unit index. Example: u8 adc_unit; u8 channel = ca_opamp_hw_adc_channel(1U, &adc_unit);

Parameters conf:1.00 **Q:** What are the requirements for the 'num' and 'adc_num' parameters?
A: The 'num' parameter represents the OPAMP instance (e.g., 1 for OPAMP1) and must be a valid index supported by the hardware. The 'adc_num' parameter must be a non-NULL pointer to a memory location where the function will store the ADC unit index; passing a NULL pointer may cause memory corruption.

Return conf:1.00 **Q:** What does the return value of ca_opamp_hw_adc_channel signify?
A: The return value represents the specific ADC channel index associated with the OPAMP output. A return value of 0 indicates that the OPAMP is not connected to an ADC or that the input 'num' parameter was invalid.

HW differences conf:1.00 **Q:** How does the behavior of ca_opamp_hw_adc_channel differ across STM32 families?
A: Implementations vary significantly: F1xx, F4xx, F7xx, and H5xx families return 0 and set adc_num to 0 regardless of input. The H7xx implementation maps OPAMP1 to ADC unit 1, channel 4. The U5xx implementation maps OPAMP1 to ADC unit 1, channel 8, and other OPAMP instances to channel 15.

`ca_opamp_hw_count`

Header: `caos/drivers/inc/ca_glue_hw.h:414`
GUARD: `EN_CA_GLUE_HW`

```
1 extern u32 ca_opamp_hw_count(void); /* instances on this part, 0 = none
     */
```

Returns the number of operational amplifier (OPAMP) hardware instances available on the current microcontroller.

Ritorna:
Returns a u32 representing the count of available OPAMP peripherals. A return value of 0 indicates that the current hardware platform does not support or expose any OPAMP instances.

Note:

- This function is a hardware-abstraction layer (HAL) utility that queries the specific capabilities of the target STM32 chip. It is non-blocking, thread-safe, and safe to call from an Interrupt Service Routine (ISR) because it simply returns a constant value defined at compile-time for the specific architecture. The most common mistake for juniors is assuming that all STM32 chips have the same number of OPAMPs; always use this function to check availability before attempting to initialize or configure an OPAMP peripheral to avoid accessing non-existent hardware registers.

Uso:

- This function is primarily used during the system initialization phase to verify hardware support before calling configuration functions. It is typically paired with ca_opamp_init

to ensure that the requested instance index is within the valid range supported by the silicon. By checking the return value, you can write portable code that gracefully handles platforms with different hardware configurations, such as skipping OPAMP-dependent features on chips where ca_opamp_hw_count returns 0.

Esempio:

```c
#include "caos.h"

void setup_opamp_system(void) {
    u32 available_opamps = ca_opamp_hw_count();

    // Check if we have at least one OPAMP available before initializing
    if (available_opamps > 0U) {
        // Initialize the first OPAMP as a voltage follower (gain 0)
        // The second parameter is the input configuration, 0 is a placeholder for default
        if (ca_opamp_init(1U, 0, 0U)) {
            // OPAMP successfully initialized
        } else {
            // Handle initialization failure
        }
    } else {
        // Log or handle the case where no OPAMP hardware is present
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_opamp_hw_count in my caOS application?
A: You should call it during the system initialization phase to verify hardware support before attempting to initialize or configure an OPAMP. This allows you to write portable code that gracefully handles platforms with different hardware configurations by skipping OPAMP-dependent features if the count is 0.

Code example conf:1.00 **Q:** How do I check for OPAMP availability using caOS types?
A: You can use the following pattern: u32 count = ca_opamp_hw_count(); if (count > 0) { /* Proceed with OPAMP initialization */ }.

Return conf:1.00 **Q:** What does the return value of ca_opamp_hw_count signify?
A: It returns a u32 representing the number of OPAMP hardware instances available on the current microcontroller. A return value of 0 indicates that the current hardware platform does not support or expose any OPAMP instances.

HW differences conf:1.00 **Q:** Are there differences in OPAMP availability across STM32 families supported by caOS?
A: Yes, availability varies by architecture. Based on the provided implementations, the F1xx, F4xx, F7xx, and H5xx families return 0 (no OPAMPs supported), while the H7xx and U5xx families return 2 available OPAMP instances.

`ca_opamp_hw_init`

Header: `caos/drivers/inc/ca_glue_hw.h:415`
GUARD: `EN_CA_GLUE_HW`

```c
extern bool ca_opamp_hw_init(u32 num, bool from_dac, u32 gain_code); /* gain_code 0 follower, 1..4 = x2..x16 */
```

Initializes the internal hardware Operational Amplifier (OPAMP) peripheral for signal conditioning.

Parametri:

Nome	Tipo	Descrizione
`num`	`u32`	The OPAMP instance number (e.g., 1 for OPAMP1, 2 for OPAMP2). Must match hardware availability.
`from_dac`	`bool`	If true, connects the non-inverting input to the internal DAC. If false, uses the external physical pin.
`gain_code`	`u32`	Sets the gain mode: 0 for unity follower, 1-4 for PGA gains (x2, x4, x8, x16 respectively).

Ritorna:
Returns true if the hardware was successfully configured and enabled. Returns false if the hardware is unsupported on the current MCU (e.g., F1/F4/F7/H5 series) or if the initialization sequence failed.

Note:

- This function is non-blocking and performs direct register manipulation to configure clocks, GPIOs, and OPAMP control registers. It is NOT thread-safe; ensure no other task or ISR attempts to reconfigure the same OPAMP instance simultaneously. It must not be called from an ISR because it performs multiple register writes that require a stable system state. A common mistake is calling this function before the system clock or the specific peripheral clock is properly initialized, which will lead to a hard fault or silent failure. Note that on some STM32 families (like F1, F4, F7, H5), this function is a stub that always returns false, so always check the return value.

Uso:

- This function is typically called during the system boot sequence or peripheral initialization phase. It is used to prepare the analog front-end before starting ADC conversions. It is often paired with ca_opamp_adc_channel to map the output to an ADC input, and ca_opamp_stop to safely disable the peripheral when it is no longer needed to save power. Always verify the return value to ensure the current hardware platform actually supports the requested OPAMP configuration.

Esempio:

```
/* Initialize OPAMP1 as a follower connected to the internal DAC */
if (ca_opamp_hw_init(1U, true, 0U)) {
    /* Successfully initialized, now map to ADC channel if needed */
    /* ca_opamp_adc_channel(1U); */
} else {
    /* Handle initialization failure: hardware might not support OPAMPs */
}
```

Q&A — fine-tuning review *7/7 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_opamp_hw_init in my caOS application?
A: This function should be called during the system boot sequence or peripheral initialization phase. It is used to prepare the analog front-end before starting ADC conversions and is often paired with ca_opamp_adc_channel.

Code example conf:1.00 **Q:** Provide a minimal example of how to initialize OPAMP1 as a unity follower using caOS types.
A: u32 opamp_num = 1U; bool use_dac = false; u32 gain = 0U; bool success = ca_opamp_hw_init(opamp_num, use_dac, gain);

Parameters conf:1.00 **Q:** What are the valid values for the gain_code parameter?
A: The gain_code parameter accepts 0 for unity follower mode, and values 1 through 4 for PGA gains corresponding to x2, x4, x8, and x16 respectively.

Return conf:1.00 **Q:** What does a return value of false indicate?
A: A return value of false indicates that the hardware is unsupported on the current MCU or that the initialization sequence failed.

Error/Edge case conf:1.00 **Q:** What is a common mistake that leads to failure when calling ca_opamp_hw_init?
A: A common mistake is calling the function before the system clock or the specific peripheral clock is properly initialized, which will lead to a hard fault or silent failure.

Behavior conf:1.00 **Q:** Is ca_opamp_hw_init safe to call from an interrupt service routine (ISR)?
A: No, it must not be called from an ISR because it performs multiple register writes that require a stable system state. Additionally, the function is non-blocking and not thread-safe.

HW differences conf:1.00 **Q:** How does the hardware implementation differ between H7xx and U5xx series?
A: The H7xx series enables the peripheral via RCC->APB1HENR, while the U5xx series uses RCC->APB3ENR. Furthermore, the U5xx implementation specifically handles the OPA_RANGE bit in OPAMP1_CSR, which must be set before enabling either OPAMP.

`ca_opamp_hw_stop`

Header: `caos/drivers/inc/ca_glue_hw.h:416`
GUARD: `EN_CA_GLUE_HW`

```
1 extern void ca_opamp_hw_stop(u32 num); /* disables OPAMPnum */
```

Disables the specified Operational Amplifier (OPAMP) hardware peripheral.

Parametri:

Nome	Tipo	Descrizione
`num`	`u32`	The index of the OPAMP to stop (e.g., 1 for OPAMP1, 2 for OPAMP2). Values must be within the range supported by the specific MCU.

Ritorna:
void

Note:

- This function is a low-level hardware abstraction layer (HAL) call. It is non-blocking and executes immediately by clearing the enable bit in the peripheral's Control and Status Register (CSR). It is thread-safe in the sense that it performs a single atomic register write, but it does not implement any software-level locking. It is safe to call from an ISR, provided the hardware supports it, but ensure the OPAMP index is valid to avoid accessing invalid memory addresses. The most common mistake is passing an index of 0 or an index higher than the hardware supports, which may lead to undefined behavior or system faults depending on the MCU architecture. Note that on some architectures (F1, F4, F7, H5), this function is a no-op as the hardware does not support software-controlled OPAMP disabling.

Uso:

- This function is typically used during system shutdown, power-saving transitions, or when reconfiguring analog signal chains. It is usually paired with ca_opamp_init() to manage the lifecycle of the peripheral. In a typical sequence, you would initialize the OPAMP with ca_opamp_init(), perform your analog operations, and then call ca_opamp_stop() when the signal processing is no longer required to save power or prevent interference with

other analog components. Always verify the hardware capability using ca_opamp_count() before assuming the stop operation will have an effect.

Esempio:

```c
/* Example: Safely stopping an OPAMP after use */
#include "caos.h"

void shutdown_analog_chain(u32 opamp_id)
{
    /* Check if the OPAMP index is valid before attempting to stop it */
    if (opamp_id != 0U && opamp_id <= ca_opamp_count()) {
        /* Disable the hardware peripheral */
        ca_opamp_stop(opamp_id);
    } else {
        /* Log error: invalid OPAMP index provided */
        ca_printf("Error: Invalid OPAMP index %lu\n", opamp_id);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_opamp_hw_stop in my caOS application?
A: You should call ca_opamp_hw_stop during system shutdown, power-saving transitions, or when reconfiguring analog signal chains. It is intended to be used when analog signal processing is no longer required to save power or prevent interference with other analog components, typically paired with ca_opamp_init().

Code example conf:1.00 **Q:** How do I properly invoke ca_opamp_hw_stop in C code?
A: You call the function by passing the index of the OPAMP as a u32. Example: u32 opamp_id = 1; ca_opamp_hw_stop(opamp_id);

Parameters conf:1.00 **Q:** What are the constraints for the 'num' parameter in ca_opamp_hw_stop?
A: The 'num' parameter is a u32 representing the index of the OPAMP (e.g., 1 for OPAMP1). Values must be within the range supported by the specific MCU. Passing 0 or an index higher than the hardware supports may lead to undefined behavior or system faults.

HW differences conf:1.00 **Q:** How does the implementation of ca_opamp_hw_stop vary across different STM32 families?
A: On F1xx, F4xx, F7xx, and H5xx families, the function is currently a no-op that casts the parameter to void. On the H7xx family, it clears the OPAMP_CSR_OPAMPxEN bit in the CSR register for OPAMP1 or OPAMP2. On the U5xx family, it clears the OPAMP_CSR_OPAEN bit in the CSR register.

`ca_opamp_init`

Header: `caos/drivers/inc/opamp.h:36`
Source: `caos/drivers/src/opamp.c:19:32`
GUARD: `EN_OPAMP`

```c
bool ca_opamp_init(u32 num, ca_opamp_input_t in, u32 gain); /* gain 1 =
    follower, 2/4/8/16 = PGA */
```

Initializes an internal STM32 operational amplifier (OPAMP) with a specific gain configuration.

Parametri:

Nome	Tipo	Descrizione
num	u32	The index of the OPAMP peripheral to initialize (1-based). Must be between 1 and the value returned by ca_opamp_count().
in	ca_opamp_input_t	The input source selection. Use CA_OPAMP_IN_DAC for internal DAC connection or other valid enum values defined in the system headers.
gain	u32	The desired gain factor. Valid values are 1 (follower mode), 2, 4, 8, or 16 (PGA mode). Any other value will cause the function to return false.

Ritorna:
Returns true if the OPAMP was successfully configured with the requested parameters. Returns false if the index is out of range, the gain value is unsupported, or the underlying hardware initialization fails.

Note:

- This function is a wrapper for hardware-specific configuration and is generally not thread-safe; it should be called during system initialization or from a single control task. It is not intended to be called from an ISR as it may involve blocking hardware register access. A common beginner mistake is passing an invalid gain value (e.g., 3 or 5), which will silently fail and leave the OPAMP in an uninitialized state. Always verify the return value before attempting to use the OPAMP for signal processing.

Uso:

- This function is used to prepare the analog front-end of the microcontroller for signal conditioning. It is typically called once during the system startup sequence or when reconfiguring analog inputs. It is often paired with ca_opamp_count() to verify hardware availability and followed by calls to ADC-related functions to read the conditioned signal. If the OPAMP is no longer needed, it should be disabled using ca_opamp_stop() to save power.

Esempio:

```
/* Initialize OPAMP 1 as a voltage follower connected to the internal DAC */
if (ca_opamp_init(1U, CA_OPAMP_IN_DAC, 1U)) {
    /* OPAMP successfully configured, proceed with signal processing */
} else {
    /* Handle initialization error: log or trigger system fault */
    ca_printf("Failed to init OPAMP 1\n");
}

/* Later, if the OPAMP is no longer required, stop it to save power */
ca_opamp_stop(1U);
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_opamp_init in my caOS application?
A: You should call it during the system startup sequence or when reconfiguring analog inputs. It is used to prepare the analog front-end for signal conditioning and is typically paired with ca_opamp_count() to verify hardware availability.

Code example conf:1.00 **Q:** How do I initialize the first OPAMP as a voltage follower using caOS types?
A: You can initialize it as follows: u32 opamp_idx = 1; if (ca_opamp_init(opamp_idx, CA_OPAMP_IN_DAC, 1)) { /* Success */ }

Parameters conf:1.00 **Q:** What are the valid values for the gain parameter in ca_opamp_init?
A: The valid gain values are 1 (for follower mode), 2, 4, 8, or 16 (for PGA mode). Any other value will cause the function to return false.

Return conf:1.00 **Q:** What does a false return value from ca_opamp_init indicate?
A: A false return indicates that the initialization failed. This occurs if the OPAMP index is out of range, the gain value is unsupported, or the underlying hardware initialization fails.

Error/Edge case conf:1.00 **Q:** What is a common mistake when configuring the OPAMP gain?
A: A common beginner mistake is passing an invalid gain value, such as 3 or 5. This will cause the function to return false and leave the OPAMP in an uninitialized state.

`ca_opamp_stop`

Header: `caos/drivers/inc/opamp.h:37`
Source: `caos/drivers/src/opamp.c:34:37`
GUARD: `EN_OPAMP`

```
void ca_opamp_stop(u32 num)
```

Safely stops the operation of a specific hardware Operational Amplifier (OPA) peripheral.

Parametri:

Nome	Tipo	Descrizione
`num`	`u32`	The index of the OPA peripheral to stop. Must be a valid index between 1 and the value returned by ca_opamp_count().

Ritorna:
void

Note:

- This function is a wrapper that performs a bounds check before calling the underlying hardware stop routine. It is not thread-safe by itself, so ensure that multiple tasks are not attempting to initialize or stop the same OPA index simultaneously. It is generally safe to call from most contexts, but it is not intended for use inside high-frequency ISRs due to the overhead of the internal bounds checking logic. The most common beginner mistake is passing an invalid index (e.g., 0 or a number exceeding the hardware count), which will cause the function to silently ignore the request without stopping any hardware.

Uso:

- This function is used to gracefully shut down an OPA peripheral when it is no longer needed, typically during system teardown, error recovery, or before reconfiguring the analog front-end. It is usually paired with ca_opamp_init() to manage the lifecycle of the peripheral. In a typical sequence, you would initialize the OPA, perform your signal processing or measurement, and then call ca_opamp_stop() to put the hardware into a low-power or idle state before disabling the associated DAC or ADC channels.

Esempio:

```
/* Example: Stopping an OPA after a measurement task */
#define OPA_CHANNEL 1U

void stop_analog_subsystem(void) {
    /* Ensure the OPA is stopped before disabling the DAC */
    ca_opamp_stop(OPA_CHANNEL);

```

```
    /* Set DAC output to zero to prevent floating voltages */
    ca_dac_write(0U, 0U);

    /* Log the action to the system console */
    ca_printf("OPA %u stopped successfully.\n", OPA_CHANNEL);
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_opamp_stop and what is its purpose?
A: You should call ca_opamp_stop to gracefully shut down an OPA peripheral when it is no longer needed, such as during system teardown, error recovery, or before reconfiguring the analog front-end. It puts the hardware into a low-power or idle state, typically after signal processing is complete and before disabling associated DAC or ADC channels.

Code example conf:0.90 **Q:** How do I properly stop an OPA peripheral using caOS types?
A: To stop an OPA, ensure you pass a valid index as a u32 type. Example: { u32 opa_idx = 1; ca_opamp_stop(opa_idx); }

Parameters conf:1.00 **Q:** What are the constraints for the 'num' parameter in ca_opamp_stop?
A: The 'num' parameter must be a u32 representing the index of the OPA peripheral. It must be between 1 and the value returned by ca_opamp_count(). If an invalid index like 0 or a number exceeding the hardware count is passed, the function will silently ignore the request.

`ca_os_banner`

Header: `caos/inc/caos.h:148`
Source: `caos/src/caos.c:76:79`
GUARD: `EN_CAOS`

```
extern void ca_os_banner(void)
```

Spawns a system task to display the OS banner information.

Parametri:

Nome	Tipo	Descrizione
`void`	`void`	This function takes no arguments.

Ritorna:
void

Note:

- This function is a wrapper that uses ca_os_job to create a new task dedicated to printing the system banner. It is not thread-safe in the sense that calling it repeatedly will spawn multiple banner tasks, which is usually unnecessary. It is not intended to be called from an ISR because it performs task creation, which requires a scheduler context. The most common mistake is calling this function inside a high-frequency loop, which will quickly exhaust system memory or task slots by spawning redundant banner threads. Because it is a task, it runs asynchronously; do not assume the banner has finished printing immediately after this function returns.

Uso:

- It is typically used during system initialization or via a CLI command to verify that the firmware is running and to display versioning or build information. It is often paired with other system diagnostic functions like ca_os_clock_dump or ca_memory_info to provide a snapshot of the system state to the user. In a CLI context, it is triggered

by the user typing 'banner' to confirm the system is responsive. If you need to ensure the banner completes before proceeding with other critical tasks, you must implement a delay or a synchronization mechanism, as the banner task runs at a specific priority level (CA_TH_LEVEL6) and may be preempted by higher-priority tasks.

Esempio:

```c
/* Example: Triggering the system banner from a CLI command handler */
if (ca_strncmp(cmd->data, "banner", 6) == 0) {
    /* Display the system banner by spawning the dedicated OS job */
    ca_os_banner();
} else if (ca_strncmp(cmd->data, "mem", 3) == 0) {
    /* Check memory status alongside system info */
    ca_memory_info();
} else {
    /* Use ca_printf to inform the user of invalid input */
    ca_printf("Unknown command. Try 'banner' or 'mem'.\n");
}
```

Q&A — fine-tuning review *2/2 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_os_banner() in my caOS application and what is its purpose?
A: You should call ca_os_banner() after initializing the caOS kernel to display a welcome message indicating that the RTOS is running. It is typically called during system startup, often from the main function or initialization routine. The purpose of this function is to provide a visual confirmation that the RTOS has been successfully initialized and is operational. It starts a low-priority background job that prints a banner message, making it useful for debugging and system verification.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_os_banner() with caOS types?
A: Here is a minimal example:
#include "caos.h"
int main(void) { ca_os_init(); ca_os_banner(); ca_os_start(); while(1) { /* Main loop */ } return 0; }

`ca_os_clock_config`

Header: `caos/drivers/inc/clock.h:23`
Source: `caos/drivers/src/clock.c:14:16`
GUARD: `EN_CLOCK`

```c
int ca_os_clock_config(void)
```

Initializes the system clock tree and PLL settings based on hardware-specific configuration.

Ritorna:
Returns 0 on success, or a non-zero error code if the clock configuration fails (e.g., PLL lock timeout).

Note:

- This function is typically non-blocking but performs hardware register writes that must complete before the system can operate at the intended frequency. It is not thread-safe and must be called during the early system initialization phase before the RTOS scheduler starts or when performing a power-state transition. It cannot be called from an ISR. A common mistake is calling this function while peripherals (like UART or CAN) are active,

which can cause communication errors or system hangs because the peripheral baud rates depend on the clock frequency.

Uso:

- This function is used during the system boot sequence or when waking up from deep sleep modes to restore the CPU frequency. It is usually paired with SystemCoreClockUpdate() to ensure the internal RTOS variables match the new hardware state. After calling this, you must call ca_os_tick_retune() to adjust the OS heartbeat timer, and then re-initialize communication peripherals like UART or CAN to ensure their timing parameters remain valid for the new clock speed.

Esempio:

```c
/* Early system initialization sequence */
void system_startup(void) {
    /* Configure the main system clock tree */
    int status = ca_os_clock_config();
    if (status != 0) {
        /* Handle clock failure: enter safe mode or trap */
        while(1);
    }

    /* Update internal variables to reflect the new frequency */
    SystemCoreClockUpdate();

    /* Adjust OS tick to match the new clock frequency */
    ca_os_tick_retune();

    /* Re-initialize peripherals that depend on clock speed */
    ca_uart_retune();

    /* Now safe to start OS services */
    ca_os_run();
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When and why should I call ca_os_clock_config() in my caOS project?
A: You should call ca_os_clock_config() during the system initialization phase, before initializing any RTOS components or hardware that depends on the clock frequency. This function ensures the STM32 microcontroller's clock is configured correctly for the RTOS to function properly. It must be called only once, and not after the RTOS has started, as doing so may cause undefined behavior or crashes.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_os_clock_config() with caOS types?
A: int main(void) { int result = ca_os_clock_config(); if (result != 0) { // Handle clock configuration error while (1); } // Proceed with RTOS initialization return 0; }

Parameters conf:0.95 **Q:** Does ca_os_clock_config() accept any parameters, and what are the valid values?
A: No, ca_os_clock_config() does not accept any parameters. The function is void and takes no arguments. There are no valid or invalid parameter values to consider.

Return conf:0.90 **Q:** What do the return values of ca_os_clock_config() indicate, and what causes failures?
A: The function returns 0 on successful clock configuration. A non-zero return value indicates an error during the clock setup process. The specific error code depends on the hardware implementation and may be related to issues such as unstable clock sources, incorrect register settings, or timeout conditions during clock stabilization.

ca_os_clock_dump

Header: caos/drivers/inc/clock.h:46
Source: caos/drivers/src/clock.c:31:33
GUARD: EN_CLOCK

```
void ca_os_clock_dump(void)
```

Prints the current hardware clock configuration and status to the system console.

Ritorna:
void

Note:

- This function is primarily a diagnostic tool used to output the internal state of the system clock tree, including PLL settings, prescalers, and current bus frequencies. It is not thread-safe in the sense that concurrent calls from multiple tasks may interleave output on the console, leading to garbled text. It is generally not intended to be called from an ISR, as it likely performs blocking I/O operations to the console. A common mistake is calling this function too frequently in a high-load loop, which can flood the console and degrade system performance due to the overhead of string formatting and transmission.

Uso:

- This function is typically used during the development and debugging phase to verify that the system clock configuration matches the expected hardware requirements. It is often paired with ca_os_clock_config to verify changes or ca_os_clock_get_freq to check specific bus speeds. In production systems, it is usually exposed via a Command Line Interface (CLI) command, allowing technicians to inspect the system state at runtime without needing a debugger attached.

Esempio:

```
/* Example of a CLI command handler that triggers a clock dump */
void handle_clock_command(void) {
    /* Print a header to the console before dumping clock data */
    ca_printf_n("\r\n--- System Clock Configuration ---\r\n");

    /* Dump the current clock tree status to the console */
    ca_os_clock_dump();

    /* Optionally display current frequency for verification */
    u32 freq = ca_os_clock_get_freq();
    ca_printf_n("\r\nSystem Frequency: %lu Hz\r\n", freq);
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_os_clock_dump() and what is its purpose in a caOS application?
A: You should call ca_os_clock_dump() during debugging or system initialization to verify the clock configuration of the STM32 microcontroller. It prints system clock frequencies which helps ensure that the hardware is configured correctly. The function is useful for checking that the system clock, AHB clock, APB1 clock, and APB2 clock are set as expected. It is typically used in development environments where a logger is active to display the output.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_os_clock_dump() with caOS types?
A: void example_function(void) { ca_os_clock_dump(); }

Parameters conf:0.90 **Q:** What does the ca_os_clock_dump() function take as input and what are the valid values for its parameters?
A: The ca_os_clock_dump() function takes no parameters. It is a void function with no input arguments. There are no parameters to provide, and thus no valid or invalid values to consider for this function.

`ca_os_clock_get_freq`

Header: `caos/drivers/inc/clock.h:28`
Source: `caos/drivers/src/clock.c:19:21`
GUARD: `EN_CLOCK`

```
u32 ca_os_clock_get_freq(void)
```

Returns the current system core clock frequency in Hertz.

Ritorna:
Returns a 32-bit unsigned integer representing the current CPU frequency in Hz. This value is derived from the hardware clock configuration (SystemCoreClock).

Note:

- This function is non-blocking and thread-safe as it simply reads a global variable updated by the hardware clock configuration. It is safe to call from both tasks and Interrupt Service Routines (ISRs). The primary precondition is that the system clock must have been initialized (usually via ca_os_clock_config) during the boot sequence. A common mistake is assuming this value is constant; if your application uses dynamic frequency scaling (e.g., via ca_os_clock_reduce_freq), you must call this function again to get the updated frequency rather than caching the result.

Uso:

- This function is essential for any timing-sensitive logic, such as calculating delays, baud rates, or converting raw timer counts into human-readable time units. It is typically used during system initialization to log the current operating state to the console, or within CLI handlers to perform math on elapsed time measurements. It is often paired with ca_os_clocks_per_tick to understand the relationship between the CPU core speed and the RTOS tick rate.

Esempio:

```c
/* Example: Calculate elapsed time in milliseconds using system clock */
#include "caos.h"

void report_execution_time(u32 elapsed_clocks) {
    u32 freq = ca_os_clock_get_freq();

    if (freq > 0) {
        /* Convert CPU cycles to milliseconds */
        u32 time_ms = (elapsed_clocks / (freq / 1000));
        ca_stdout_printf_n("Execution took: %u ms", time_ms);
    } else {
        /* Handle error: clock frequency is invalid */
        ca_stdout_printf_n("Error: Invalid system clock frequency");
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_os_clock_get_freq() and why is it useful in embedded development?
A: You should call ca_os_clock_get_freq() to retrieve the current system clock frequency in Hz, which is essential for time calculations, timer configurations, and converting between clock ticks and actual time. It's useful because it allows precise timing operations without hardcoding clock values, making your code portable across different STM32 configurations. The function is safe to call from ISRs and does not block, making it ideal for real-time applications.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_os_clock_get_freq() with caOS types?
A: u32 freq = ca_os_clock_get_freq(); if (freq != 0) { // Use freq for timer calculations u32 delay_cycles = freq / 1000; // 1ms delay assuming 1kHz clock } else { // Handle error case }

Return conf:0.90 **Q:** What do the return values of ca_os_clock_get_freq() mean and what causes it to return 0?
A: The function returns a u32 representing the system clock frequency in Hz. A return value of 0 indicates an error or invalid configuration, typically caused by calling the function before the system clock has been properly initialized. Since it simply returns the global variable SystemCoreClock, calling it too early in the boot process or before clock setup will result in a zero value.

`ca_os_clock_reduce_freq`

Header: `caos/drivers/inc/clock.h:35`
Source: `caos/drivers/src/clock.c:23:25`
GUARD: `EN_CLOCK`

```
u32 ca_os_clock_reduce_freq(u16 scale)
```

Reduces the system core clock frequency to a specified percentage of the maximum.

Parametri:

Nome	Tipo	Descrizione
`scale`	`u16`	The target percentage of the original clock frequency (e.g., 50 for 50%). Valid values are typically 1 to 100.

Ritorna:
Returns the new system core clock frequency in Hz as a u32. If the hardware fails to apply the reduction, it may return the current frequency or 0 depending on the specific hardware implementation.

Note:

- This function is not thread-safe and must be called with caution as it alters the timing base of the entire system. It is generally not callable from an ISR because changing the clock frequency requires reconfiguring hardware registers that may cause system instability if interrupted. A common beginner mistake is to reduce the clock frequency without subsequently calling peripheral retuning functions (like ca_os_tick_retune or ca_uart_retune), which leads to incorrect baud rates, broken communication, and timing drift in OS delays.

Uso:

- This function is used in power-sensitive applications or emergency thermal management scenarios where reducing power consumption is prioritized over processing speed. It is

typically paired with ca_os_tick_retune to ensure the OS heartbeat remains accurate after the clock change. When the system needs to return to full performance, it is paired with ca_os_clock_restore_freq to reset the system to its original operating frequency.

Esempio:

```c
/* Example: Reducing clock for power saving during a thermal event */
#include "caos.h"

void handle_thermal_event(void) {
    u32 new_freq;

    /* Reduce clock to 50% to lower heat generation */
    new_freq = ca_os_clock_reduce_freq(50);

    /* Critical: Retune system timing and peripherals to match new clock */
    ca_os_tick_retune();
    ca_uart_retune();

    /* Log the change to the console */
    ca_printf("System clock reduced to %lu Hz\r\n", new_freq);

    /* Perform low-power operations... */

    /* Restore original clock when safe */
    ca_os_clock_restore_freq();
    ca_os_tick_retune();
    ca_uart_retune();
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_os_clock_reduce_freq and what is its main purpose?
A: You should call ca_os_clock_reduce_freq to reduce the system clock frequency, typically for power saving or to handle critical situations like WDT reset. It is a blocking function that modifies the system clock and may take time to apply the change.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_os_clock_reduce_freq with caOS types?
A: u32 result = ca_os_clock_reduce_freq(50); if (result != 0) { // Handle error }

Parameters conf:0.90 **Q:** What does the 'scale' parameter represent and what are valid values for it?
A: The 'scale' parameter specifies the factor by which the system clock frequency is reduced. For example, scale=50 reduces the frequency to 1/50th of the original. Valid values are greater than 0; values of 0 or too large that cause underflow are invalid.

Return conf:0.90 **Q:** What do the return values of ca_os_clock_reduce_freq indicate and what causes failures?
A: A return value of 0 indicates success. Non-zero values indicate errors such as an unreachable frequency or unsupported hardware. Failures can occur due to invalid scale values (0 or too large) or hardware limitations.

`ca_os_clock_restore_freq`

Header: `caos/drivers/inc/clock.h:41`
Source: `caos/drivers/src/clock.c:27:29`
GUARD: `EN_CLOCK`

```c
u32 ca_os_clock_restore_freq(void)
```

Restores the system core clock frequency to its original, high-performance state after a temporary reduction.

Ritorna:
Returns a u32 value representing the restored core clock frequency in Hertz. A return value of 0 typically indicates that the restoration failed or the clock configuration is invalid.

Note:

- This function is not thread-safe and should be called from a controlled context, typically the main application thread. It is not intended for use within an Interrupt Service Routine (ISR) because changing the system clock frequency affects timing-dependent peripherals and OS tick calculations. A common beginner mistake is calling this function without subsequently updating peripheral baud rates or timing-sensitive drivers, which will cause communication errors (e.g., UART garbage data or CAN bus errors). Always ensure that any hardware that relies on the system clock is re-synchronized after calling this function.

Uso:

- This function is used in power-management scenarios where the system temporarily reduces its clock speed to save energy (e.g., during idle periods or battery-powered operation) and needs to return to full performance for heavy processing. It is almost always paired with 'ca_os_clock_reduce_freq' to manage the transition. After restoring the clock, you must call 'ca_os_tick_retune' to update the OS heartbeat and 'ca_uart_retune' (or other peripheral retune functions) to ensure that communication interfaces remain functional at the new frequency.

Esempio:

```c
/* Example: Restoring system performance after a low-power task */

void perform_high_speed_task(void) {
    u32 new_freq;

    /* Restore the clock to full speed */
    new_freq = ca_os_clock_restore_freq();

    if (new_freq > 0) {
        /* Re-synchronize OS tick and peripherals to the new frequency */
        ca_os_tick_retune();
        ca_uart_retune();

        ca_stdout_put("System clock restored to full speed.\r\n");
    } else {
        /* Handle error: clock restoration failed */
        ca_stdout_put("Error: Failed to restore clock frequency!\r\n");
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_os_clock_restore_freq() and what is its purpose in a power management strategy?
A: You should call ca_os_clock_restore_freq() after reducing the system clock frequency using ca_os_clock_reduce_freq() to restore the original clock speed. This function is useful in dynamic power management strategies where you want to reduce power consumption during low activity periods and then restore full performance when needed.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_os_clock_restore_freq() with caOS types?
A: u32 result = ca_os_clock_restore_freq(); if (result == 0) { // Clock restored successfully } else { // Handle error }

Parameters conf:0.85 **Q:** Does ca_os_clock_restore_freq() take any parameters, and what happens if I pass invalid values?
A: This function does not take any parameters. Since there are no parameters, there are no invalid values to consider. However, calling this function without having previously reduced the clock frequency using ca_os_clock_reduce_freq() results in undefined behavior.

Return conf:0.90 **Q:** What do the return values of ca_os_clock_restore_freq() indicate, and what can cause a failure?
A: The function returns 0 on successful restoration of the clock frequency. Any non-zero return value indicates an error during the restoration process. Failures may be caused by hardware driver issues or improper sequence of function calls, such as calling this function without first reducing the clock frequency.

`ca_os_clocks_per_tick`

Header: `caos/inc/systick.h:260`
Source: `caos/src/systick.c:544:546`
GUARD: `EN_SYSTICK`

```
u32 ca_os_clocks_per_tick(void)
```

Returns the number of CPU clock cycles that occur within a single OS tick (typically 1ms).

Ritorna:
A u32 value representing the reload value of the SysTick timer. This value is derived from the system clock frequency divided by the OS tick rate (usually 1000Hz).

Note:

- This function is non-blocking and thread-safe as it simply returns a pre-calculated constant value stored in memory. It is safe to call from an ISR, although it is rarely needed there. The function relies on the OS tick being properly initialized; if called before the OS scheduler or SysTick timer is configured, it may return an uninitialized or zero value. A common mistake is assuming this value is a time duration in milliseconds; it is a count of CPU cycles, not a unit of time.

Uso:

- This function is primarily used for diagnostic purposes, such as printing system configuration details to a console or log. It is often paired with ca_os_clock_get_freq to verify that the OS timing is correctly synchronized with the hardware clock. Developers use it when they need to perform low-level timing calculations or when debugging drift issues in time-sensitive applications. It is typically called during the system initialization phase or within a diagnostic task that reports system health.

Esempio:

```
/* Example: Displaying system timing configuration */

void report_system_timing(void) {
    u32 freq = ca_os_clock_get_freq();
    u32 clocks = ca_os_clocks_per_tick();

    /* Print the clock frequency and the calculated cycles per tick */
    ca_stdout_printf_n("System Frequency: %u Hz", freq);
    ca_stdout_printf_n("Cycles per OS Tick: %u", clocks);

    /* Verify if the tick configuration is valid */
```

```
12     if (clocks == 0) {
13         ca_stdout_printf_n("Error: OS tick not initialized!");
14     }
15 }
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_os_clocks_per_tick() and why is it useful for timing calculations?
A: You should call ca_os_clocks_per_tick() to determine how many CPU clock cycles are needed to complete one OS tick (1/1000 of a second). This value is essential for precise timing calculations, especially when working with hardware timers or when you need to convert between clock cycles and time intervals. It's particularly useful in interrupt service routines (ISRs) or any context where you need to know the relationship between system clock cycles and OS timing.

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_os_clocks_per_tick() with caOS types?
A: u32 clocks_per_tick = ca_os_clocks_per_tick(); if (clocks_per_tick != 0) { // Use clocks_per_tick for timing calculations u32 delay_cycles = clocks_per_tick * 5; // 5ms delay // Do something with delay_cycles } else { // Handle error case }

Return conf:0.90 **Q:** What do the return values of ca_os_clocks_per_tick() mean and what causes it to return 0?
A: The function returns a u32 value representing the number of CPU clock cycles per OS tick (1/1000 second). A return value of 0 indicates an error or invalid configuration, typically caused by calling the function before the SysTick timer has been properly configured with a valid reload value. Since the function assumes correct SysTick setup, calling it prematurely will result in a zero return value.

`ca_os_configure`

Header: `caos/inc/caos.h:139`
GUARD: `EN_CAOS`

```
1 extern void ca_os_configure(void)
```

User-defined hook function called by the system during initialization, just before the scheduler starts.

Ritorna:
void

Note:

- This function is a mandatory hook that the developer must implement to prepare the application environment. It is executed in a single-threaded context before the RTOS scheduler begins managing tasks, meaning it is not thread-safe because the OS is not yet running. It is strictly forbidden to call blocking functions or wait for OS events here, as the scheduler is not active. The most common mistake is attempting to use OS primitives like semaphores or queues that have not been initialized yet, or performing long-running hardware operations that delay system startup.

Uso:

- This function serves as the central configuration point for your application-specific hardware and software modules. It is typically used to initialize sensors, configure communication peripherals, and create initial OS threads or queues. You should pair this with

initialization functions for your specific drivers (e.g., ca_temp_init) and OS resource creation functions (e.g., ca_stdout_create_thread) to ensure all system components are ready before the scheduler takes control.

Esempio:

```c
void ca_os_configure(void) {
    /* Initialize hardware sensors */
    ca_temp_init();

    /* Initialize standard output for logging */
    ca_stdout_init();

    /* Configure the output sender */
    ca_stdout_set_sender(0);

    /* Create the main logging thread */
    ca_stdout_create_thread();

    /* Initialize a static queue for inter-task communication */
    /* ca_queue_static_init(my_queue_handle, my_buffer, my_buffer_size); */
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should ca_os_configure() be called in a caOS application, and why is it important?
A: The function ca_os_configure() must be called once during system startup, before starting the scheduler. It initializes hardware modules and necessary components required for the OS to function correctly. It is a blocking function and must not be called from an ISR or multiple times, as this could lead to undefined behavior or crashes.

Code example conf:0.90 **Q:** Can you provide a minimal C example using caOS types that demonstrates how to call ca_os_configure()?
A:

```c
#include "caos.h"

int main(void) {
    ca_os_configure();
    // Scheduler would be started here
    return 0;
}
```

Parameters conf:0.90 **Q:** What is the significance of the void parameter in ca_os_configure(), and are there any invalid ways to call this function?
A: The ca_os_configure() function takes no parameters, as indicated by its signature 'extern void ca_os_configure(void)'. It is a setup function and does not accept any inputs. Invalid usage includes calling it from an ISR or calling it more than once during system execution, which may result in undefined behavior or system crashes.

`ca_os_context_from_slot`

Header: `caos/inc/scheduler.h:591`
Source: `caos/src/scheduler.c:2907:2911`
GUARD: `EN_SCHEDULER`

```c
ca_thread_context_t *ca_os_context_from_slot(u32 slot)
```

Retrieves the internal thread context pointer from a task slot index.

Parametri:

Nome	Tipo	Descrizione
`slot`	`u32`	The numeric index (0 to MAX_TASK-1) representing the task slot. Values outside this range or inactive slots will result in a NULL return.

Ritorna:
Returns a pointer to the ca_thread_context_t structure if the slot is valid and active; returns NULL if the slot is out of bounds, the task array is uninitialized, or the slot is currently unused.

Note:

- This function is non-blocking and thread-safe as it performs a read-only lookup of the internal task array. It is safe to call from an ISR, provided the OS has been initialized. The most common beginner mistake is failing to check the returned pointer for NULL before dereferencing it; always verify the pointer, as tasks can be destroyed or slots can be empty at any time. This function does not 'lock' the task, so the context could theoretically become invalid if the task is destroyed by another thread immediately after the pointer is retrieved.

Uso:

- This function is typically used by system-level services (like memory managers or debuggers) that need to map a numeric task identifier back to the actual task control block. It is often paired with ca_os_task_slot to convert between handles and indices, or used during memory cleanup routines (e.g., ca_memory_free_all_owned_by) to identify which task owns specific resources. It acts as a bridge between the abstract 'slot' ID and the concrete 'ca_thread_context_t' structure required for direct manipulation of task statistics.

Esempio:

```c
/* Example: Checking memory usage for a specific task slot */
void check_task_memory(u32 slot) {
    ca_thread_context_t *ctx = ca_os_context_from_slot(slot);

    /* Always check for NULL before accessing the context */
    if (ctx != NULL) {
        /* Retrieve memory usage using the context pointer */
        u32 usage = ca_memory_get_task_usage(ctx);
        ca_printf("Task in slot %u is using %u bytes of memory\n", slot, usage);
    } else {
        ca_printf("Slot %u is currently inactive or invalid\n", slot);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_os_context_from_slot in my caOS application?
A: You should use this function when you need to map a numeric task identifier (slot) back to its concrete ca_thread_context_t structure. It is typically used by system-level services like memory managers or debuggers, or when performing cleanup routines such as ca_memory_free_all_owned_by to identify which task owns specific resources.

Code example conf:1.00 **Q:** How do I safely retrieve a task context using a slot index?
A: You must always check the returned pointer for NULL before dereferencing it, as shown here: u32 slot = 2; ca_thread_context_t *ctx = ca_os_context_from_slot(slot); if (ctx != NULL) { /* Safe to access ctx */ }

Parameters conf:1.00 **Q:** What are the constraints for the 'slot' parameter in ca_os_context_from_slot?
A: The 'slot' parameter is a u32 representing the numeric index of a task. Valid values range from 0 to MAX_TASK-1. Providing a value outside this range will result in a NULL return.

Return conf:1.00 **Q:** Why might ca_os_context_from_slot return a NULL pointer?
A: A NULL return occurs if the provided slot index is out of bounds, if the internal task array is uninitialized, or if the specific slot is currently unused or inactive.

`ca_os_delay`

Header: `caos/inc/systick.h:198`
Source: `caos/src/systick.c:492:514`
GUARD: `EN_SYSTICK`

```
void ca_os_delay(u32 tick)
```

Blocks the current execution flow for a specified number of system ticks.

Parametri:

Nome	Tipo	Descrizione
`tick`	`u32`	The number of system ticks to wait. A value of 0 returns immediately.

Ritorna:
void

Note:

- This function is a blocking, busy-wait style delay that uses the ___WFI() (Wait For Interrupt) instruction to save power while waiting. It is NOT thread-safe in the sense that it halts the entire execution context of the caller, preventing other tasks from running if called from a non-preemptive context. It must NEVER be called from an Interrupt Service Routine (ISR) because it relies on the system tick to progress, and blocking an ISR will cause a system hang. The most common beginner mistake is using this for long delays in a multi-tasking environment; for long durations, use ca_os_sleep or ca_os_sleep_ticks to allow the scheduler to run other tasks instead of locking the CPU.

Uso:

- This function is typically used during hardware initialization sequences where a precise, short-duration wait is required before the OS scheduler is fully operational or when a simple, non-preemptive delay is needed in a low-level driver. It is often paired with hardware configuration functions like ca_os_clock_config or peripheral initialization routines. In a standard application, you should prefer ca_os_sleep_ticks, as ca_os_delay is strictly for scenarios where you need to halt the CPU execution flow entirely.

Esempio:

```
/* Initialize system clock before using delay */
ca_os_clock_config();

/* Perform a hardware reset sequence */
/* Set GPIO pin to low to trigger hardware reset */
ca_gpio_write(0, 0, 0);

/* Wait for 10 ticks to ensure hardware registers stabilize */
ca_os_delay(10);

/* Set GPIO pin to high to release reset */
ca_gpio_write(0, 0, 1);
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_os_delay in a caOS application?
A: You should use ca_os_delay when you need to introduce a delay in your application that blocks the current thread for a specified number of ticks. It is typically used for timing delays, such as blinking an LED or waiting between periodic tasks. The function is not suitable for ISR contexts and should only be called from tasks in a safe environment.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_os_delay in a caOS task?
A: void my_task(void) { ca_os_delay(1000); // Task continues after 1000 ticks }

Parameters conf:0.90 **Q:** What does the tick parameter in ca_os_delay represent, and what are valid values?
A: The tick parameter specifies the number of ticks to wait. Valid values range from 0 to 0xFFFFFFFF. If tick is 0, the function returns immediately without performing any delay. Values outside this range are not specified and may lead to undefined behavior.

Error/Edge case conf:0.95 **Q:** What happens if ca_os_delay is called from a task in a caOS system with CA_SAFEGUARD enabled?
A: If ca_os_delay is called from a task while CA_SAFEGUARD is enabled, the system will crash in debug mode because calling this function from a task is explicitly forbidden and considered an error.

Behavior conf:0.90 **Q:** Is ca_os_delay blocking or non-blocking, and is it safe to call from an ISR?
A: ca_os_delay is a blocking function, meaning it halts execution of the calling thread until the delay completes. It is not safe to call from an ISR because it uses ___WFI(), which is not interrupt-safe. Additionally, it is not thread-safe as it is not designed to be called concurrently by multiple threads.

`ca_os_delay_ms`

Header: `caos/inc/systick.h:181`
Source: `caos/src/systick.c:463:486`
GUARD: `EN_SYSTICK`

```
1 void ca_os_delay_ms(u32 ms)
```

Suspends execution for a specified number of milliseconds using a low-power wait-for-interrupt loop.

Parametri:

Nome	Tipo	Descrizione
`ms`	`u32`	The duration of the delay in milliseconds. A value of 0 returns immediately.

Ritorna:
void

Note:

- This function is strictly for use outside of RTOS tasks (e.g., during system initialization or before the scheduler starts). It uses a busy-wait loop combined with ___WFI() (Wait For Interrupt) to save power, but it blocks the entire CPU execution flow. It is NOT thread-safe and must never be called from within an RTOS task; if called from a task, the system will trigger a CA_SAFEGUARD failure. Beginners often mistake this for a task-level delay; always use ca_os_sleep for tasks to allow the scheduler to run other threads.

Uso:

- Primarily used during the hardware initialization phase (e.g., inside ca_os_configure or main) before the RTOS scheduler is running. It is useful for providing settling time for hardware peripherals like USB or external sensors that require a stable power-up sequence. It is often paired with hardware initialization functions to ensure the system is ready before starting the OS scheduler. Because it blocks the CPU, using it after the scheduler has started will freeze the entire system, preventing other tasks from executing.

Esempio:

```c
/* Example: Initializing hardware before starting the OS scheduler */
void system_init(void) {
    /* Initialize system clocks and basic peripherals */
    ca_os_tick_init();

    /* Wait for external sensor power-up stabilization */
    ca_os_delay_ms(50U);

    /* Start USB device management */
    if (ca_usb_dev_mng_start()) {
        /* Allow time for USB enumeration settling */
        ca_os_delay_ms(100U);
    }

    /* Now safe to proceed to ca_os_run() */
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_os_delay_ms() in a caOS application?
A: You should use ca_os_delay_ms() in non-RTOS contexts such as main() or ISRs to introduce a delay without busy-waiting. It allows the CPU to enter sleep mode via ___WFI() until the next interrupt, saving power. It is not suitable for use inside RTOS tasks, where you should use ca_os_sleep() instead.

Code example conf:0.90 **Q:** Can you show a minimal example of using ca_os_delay_ms() with caOS types?
A: u32 delay = 1000; ca_os_delay_ms(delay); // Delays for 1000 ms (1 second)

Parameters conf:0.95 **Q:** What does the 'ms' parameter in ca_os_delay_ms() control, and what are valid values?
A: The 'ms' parameter specifies the number of milliseconds to delay. Valid values range from 0 to 0xFFFFFFFF. If 0 is passed, the function returns immediately without delay.

Return conf:0.90 **Q:** Does ca_os_delay_ms() return anything, and can it fail?
A: The function does not return any value. It never fails, but it may not behave as expected if called from an RTOS task, as it is designed for non-RTOS contexts only.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_os_delay_ms()?
A: A common mistake is calling ca_os_delay_ms() from within an RTOS task instead of using ca_os_sleep(). This can lead to crashes or undefined behavior. Additionally, calling it from RTOS contexts without proper safeguards may trigger CA_SAFEGUARD errors.

Behavior conf:0.90 **Q:** Is ca_os_delay_ms() blocking, thread-safe, and safe to call from ISRs?
A: Yes, ca_os_delay_ms() is blocking for the current thread and enters sleep mode via ___WFI(). It is thread-safe as it does not modify shared state. It can be called from ISRs, but not from RTOS tasks, as verified by checking the IPSR and CONTROL registers.

`ca_os_destroy`

Header: `caos/inc/scheduler.h:473`
Source: `caos/src/scheduler.c:2747:2755`

GUARD: `EN_SCHEDULER`

```
bool ca_os_destroy(void)
```

Performs a complete system reset by clearing all tasks, memory structures, and detaching the task manager.

Ritorna:
Returns true if the system was successfully destroyed and reset, false if the operation failed (e.g., could not remove all jobs).

Note:

- This function is a destructive operation that wipes the internal state of the RTOS. It is not thread-safe and must never be called from an Interrupt Service Routine (ISR) as it performs heavy memory clearing and task management cleanup. The function relies on ca_os_remove_all_jobs() to succeed first; if any job refuses to terminate, the entire destruction process is aborted to prevent undefined behavior. A common mistake is calling this while other threads are still actively relying on system resources, which will lead to immediate system crashes or hard faults.

Uso:

- This function is typically used during a system-wide shutdown, a factory reset sequence, or when transitioning between completely different application modes that require a clean slate. It is usually paired with a subsequent re-initialization sequence, such as calling ca_memory_init() and re-registering tasks. Developers should ensure that all hardware peripherals are safely disabled or put into a known state before invoking this, as the OS will no longer be able to manage them after the task manager is detached.

Esempio:

```
/* Example: Performing a full system reset */
if (ca_os_destroy()) {
    /* System is now clean, re-initialize memory and OS components */
    ca_memory_init();
    /* Re-configure system clock if necessary */
    ca_os_clock_config();
    /* Re-register tasks or jobs here */
    ca_os_job(my_new_task_function);
} else {
    /* Handle failure: some jobs could not be removed */
    ca_printf("Failed to destroy OS: jobs still active.\n");
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_os_destroy() in a caOS application?
A: You should call ca_os_destroy() only when you want to completely shut down the RTOS system, freeing all task resources and resetting the scheduler state. It must be called when no other tasks are running or waiting, and it's a global cleanup function, not for managing individual tasks. It is typically used during system shutdown or reset sequences.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_os_destroy() with caOS types?
A: bool success = ca_os_destroy(); if (success) { // System successfully destroyed } else { // Failed to destroy due to active jobs }

Return conf:0.95 **Q:** What does ca_os_destroy() return and what causes it to return false?
A: ca_os_destroy() returns true if the RTOS system is successfully destroyed, or false if there are active jobs preventing the destruction. The function fails and returns false when

there are active jobs that haven't been removed, as it calls ca_os_remove_all_jobs() before resetting memory. It does not return until all jobs are removed.

---

ca_os_end

Header: caos/inc/caos.h:146
Source: caos/src/caos.c:658:662
GUARD: EN_CAOS

```
extern void ca_os_end(u32 image)
```

Terminates the current caOS execution and triggers a system jump to a new image address.

Parametri:

Nome	Tipo	Descrizione
image	u32	The memory address of the new firmware image to execute. If set to 0, the system typically remains in a halted state.

Ritorna:
This function never returns. It executes a Supervisor Call (SVC) that halts the scheduler and transfers control to the bootloader or the new image.

Note:

- This function is non-blocking in the traditional sense because it never returns; it effectively kills the current OS context. It is not thread-safe in the context of continuing execution, as it is intended to be the final act of a task. It must NOT be called from an ISR, as it triggers a context switch via SVC that expects a thread-mode environment. The most common beginner mistake is calling this function while expecting the code following it to execute; any code placed after ca_os_end will be ignored.

Uso:

- This function is used during firmware updates, system reboots, or when switching between different application slots in a multi-image system. It is typically paired with a preceding ca_os_sleep call to ensure that any pending logs or data transmissions (e.g., via UART or network) are flushed before the system shuts down. It is the final step in a bootloader sequence where the application has verified a new image and is ready to hand over control.

Esempio:

```
/* Example: Finalizing a firmware update and jumping to the new image */

void finalize_and_reboot(u32 new_image_addr) {
    /* Ensure all pending logs are sent before shutting down */
    ca_os_sleep(500);

    /* Log the intent to the console */
    ca_printf("System shutting down to jump to 0x%08X\n", new_image_addr);

    /*
     * Terminate the OS and jump to the new image.
     * This function will not return, so no code below this line will run.
     */
    ca_os_end(new_image_addr);

    /* This line is unreachable */
    while(1) { ; }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_os_end in my application?
A: ca_os_end is used for firmware updates, system reboots, or switching between application slots in a multi-image system. It is typically the final act of a task, often called after a ca_os_sleep to ensure pending data transmissions are flushed.

Code example conf:1.00 **Q:** How do I implement a system reboot to a new image address using ca_os_end?
A: You should call the function with the target address. Example: u32 new_image = 0x08004000; ca_os_end(new_image);

Parameters conf:1.00 **Q:** What does the 'image' parameter represent and what happens if I pass 0?
A: The 'image' parameter is the memory address of the new firmware image to execute. If set to 0, the system typically remains in a halted state.

Return conf:1.00 **Q:** What value does ca_os_end return upon successful execution?
A: This function never returns. It executes a Supervisor Call (SVC) that halts the scheduler and transfers control, meaning any code placed after the function call will be ignored.

`ca_os_handle_from_thid`

Header: `caos/inc/scheduler.h:570`
Source: `caos/src/scheduler.c:2819:2831`
GUARD: `EN_SCHEDULER`

```
ca_task_handle_t ca_os_handle_from_thid(u32 thid)
```

Converts a numeric Task ID (thid) into a secure, opaque task handle for system operations.

Parametri:

Nome	Tipo	Descrizione
`thid`	`u32`	The numeric identifier of the task. Valid values are active task IDs; (u32)-1 is invalid and will return CA_TASK_HANDLE_NONE.

Ritorna:
Returns a valid ca_task_handle_t if the task exists, or CA_TASK_HANDLE_NONE if the thid is invalid or the task is not found.

Note:

- This function is thread-safe because it internally disables interrupts to ensure the task list remains consistent during the lookup. It is safe to call from an ISR, though it is primarily intended for management tasks. A common beginner mistake is assuming that a handle remains valid indefinitely; if a task is destroyed or removed, the handle may become stale. Always verify the handle's validity using other caOS functions if you intend to perform operations on a task that might have been terminated by another thread.

Uso:

- This function is typically used in CLI tools or system monitors where a user provides a numeric ID (thid) and the system needs to perform an action on that specific task. It is usually paired with ca_os_thid_from_handle to verify that the handle still points to the expected task after the lookup. It acts as a bridge between human-readable IDs and the internal kernel handles required by functions like ca_os_suspend_handle or ca_os_resume_handle.

Esempio:

```c
/* Example: Suspend a task based on a user-provided ID */
void manage_task_by_id(u32 thid) {
    /* Convert the numeric ID to a system handle */
    ca_task_handle_t h = ca_os_handle_from_thid(thid);

    if (h == CA_TASK_HANDLE_NONE) {
        ca_printf("Error: Task ID %u not found.\n", thid);
        return;
    }

    /* Suspend the task using the retrieved handle */
    ca_os_suspend_handle(h);
    ca_printf("Task %u has been suspended.\n", thid);
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_os_handle_from_thid in my caOS application?
A: You should use this function when you need to convert a numeric Task ID (thid) into an opaque task handle for system operations. It is typically used in CLI tools or system monitors where a user provides a numeric ID, and it acts as a bridge to internal kernel functions like ca_os_suspend_handle or ca_os_resume_handle.

Code example conf:1.00 **Q:** How do I implement a simple task lookup using caOS types?
A: You can perform a lookup as follows: u32 my_thid = 5; ca_task_handle_t handle = ca_os_handle_from_thid(my_thid); if (handle != CA_TASK_HANDLE_NONE) { /* proceed with operations */ }

Parameters conf:1.00 **Q:** What are the valid values for the thid parameter?
A: The thid parameter represents the numeric identifier of the task. Valid values are active task IDs. Passing (u32)-1 is considered invalid and will result in the function returning CA_TASK_HANDLE_NONE.

Return conf:1.00 **Q:** What does the return value of ca_os_handle_from_thid signify?
A: The function returns a valid ca_task_handle_t if the task exists. If the provided thid is invalid or the task cannot be found, it returns CA_TASK_HANDLE_NONE.

Error/Edge case conf:1.00 **Q:** What is a common mistake when using task handles obtained from this function?
A: A common mistake is assuming that a handle remains valid indefinitely. If a task is destroyed or removed, the handle may become stale. You should always verify the handle's validity using other caOS functions if the task might have been terminated by another thread.

`ca_os_hw_clock_config`

Header: `caos/drivers/inc/ca_glue_hw.h:76`
GUARD: `EN_CA_GLUE_HW`

```c
extern int ca_os_hw_clock_config(void); /* configures PLL and system
    clock at boot */
```

Initializes the system clock, PLL, and bus prescalers based on hardware-specific requirements.

Ritorna:
Returns 0 on success. Returns a negative integer (e.g., -1, -2, -3, -4) if the clock configuration fails, typically due to a timeout waiting for a hardware ready flag (e.g., PLL, HSE, or voltage regulator not stabilizing).

Note:

- This function is intended to be called only once during the early system boot sequence before the RTOS scheduler starts. It is not thread-safe and must never be called from an ISR or after the system is running, as changing clock frequencies while the CPU is executing code will cause a hard fault or system instability. It performs low-level register manipulation to set voltage scaling (VOS), flash latency, and PLL multipliers; incorrect configuration here will prevent the system from booting or cause erratic peripheral behavior. Always ensure the hardware-specific clock definitions (like SYSCLK_FREQ_HZ) match the physical crystal or oscillator present on your board.

Uso:

- This function is the entry point for hardware clock initialization in the caOS boot sequence. It is typically called immediately after power-on reset, before initializing memory or starting any tasks. It is often paired with ca_os_tick_init to ensure the system timer is configured correctly for the new CPU frequency, and followed by peripheral initialization functions to ensure they receive the correct clock signals. If this function returns an error, the system is in an unstable state and should not proceed to start the scheduler.

Esempio:

```c
int main(void) {
    /* Initialize system clocks to target frequency */
    if (ca_os_clock_config() != 0) {
        /* Clock config failed, enter infinite loop or error handler */
        while(1);
    }

    /* Initialize system tick after clock is stable */
    ca_os_tick_init();

    /* Proceed with OS startup */
    ca_os_main();

    return 0;
}
```

Q&A — fine-tuning review

6/6 pairs · qwen3-coder:latest

Usage conf:0.95 **Q:** When should the caos_clock_init() function be called and why?
A: The caos_clock_init() function should be called during system initialization, typically at the start of the main() function or early in the boot sequence. It is responsible for configuring the system clock sources (like HSE, PLL) and setting up the CPU, AHB, and APB bus clocks to ensure the system operates at the desired frequency. This is essential for proper timing and peripheral operation.

Code example conf:0.90 **Q:** Can you provide a minimal working C example using caOS types to call caos_clock_init()?
A:

```c
#include "caos_types.h"

void main(void) {
    u32 result = caos_clock_init();
    if (result != 0) {
        // Handle error
        while (1);
    }
    // Continue with normal operation
}
```

Return conf:0.85 **Q:** What do the return values of caos_clock_init() mean and what causes failures?

A: The function returns 0 on success. Any non-zero return value indicates a failure, typically caused by clock configuration issues such as HSE not stabilizing, PLL not locking, or invalid clock settings. In the provided code, there are no explicit return values shown, but in a real implementation, failure conditions would be checked and reported accordingly.

Error/Edge case conf:0.80 **Q:** What are common mistakes or edge cases when using caos_clock_init()?
A: Common mistakes include calling the function before the system is properly initialized, using unsupported HSE values, or not handling clock stability timeouts. Edge cases involve incorrect clock division ratios, missing peripheral enable bits, or attempting to configure clocks beyond the MCU's capabilities. For example, in H7 family code, an unsupported HSE_VALUE will cause a compile-time error.

Behavior conf:0.90 **Q:** Is caos_clock_init() blocking or non-blocking, and is it safe to call from an ISR or in a multi-threaded environment?
A: The function is blocking, as it waits for clocks to stabilize (e.g., HSE ready, PLL locked). It is not ISR-safe due to its use of delays and register modifications. It is also not thread-safe, as it modifies global clock configuration registers that affect the entire system. It should be called once during system initialization before any concurrent tasks begin.

HW differences conf:0.90 **Q:** How do the clock initialization routines differ between different MCU families like H7 and F4?
A: The H7 family uses a multi-domain clock system with D1, D2, and D3 domains, requiring configuration of D1CFGR, D2CFGR, and D3CFGR registers. It also uses PLLCKSELR, PLL1DIVR, and PLLCFGR for PLL setup. In contrast, F4 typically uses simpler AHB/APB prescalers and direct RCC_CFGR register configuration. H7 supports overdrive modes and USB clock source selection, which F4 lacks.

`ca_os_hw_clock_dump`

Header: `caos/drivers/inc/ca_glue_hw.h:79`
GUARD: `EN_CA_GLUE_HW`

```
1 extern void ca_os_hw_clock_dump(void); /* prints current clocks (AHB,
    APB1, APB2) to debug */
```

Prints the current system and peripheral clock frequencies to the debug output.

Ritorna:
This function returns void and does not provide a status code.

Note:

- This function is intended for diagnostic purposes only. It is not thread-safe and should not be called from an ISR, as it performs multiple blocking I/O operations via ca_stdout_printf. It relies on the hardware-specific RCC register configuration and the global SystemCoreClock variable; if the system clock has been modified without updating SystemCoreClock, the output will be incorrect. A common beginner mistake is calling this function during high-frequency real-time tasks, which can cause significant jitter or buffer overflows in the stdout stream.

Uso:

- Use this function during the bring-up phase of a project or when debugging peripheral communication issues (e.g., UART baud rate errors or SPI timing issues). It is typically called after the system clock initialization sequence is complete. It is often paired with

ca_os_clock_get_freq to verify that the software-calculated frequency matches the hardware register state, or used in conjunction with ca_os_banner to provide a system status report upon boot.

Esempio:

```c
/* Example: Verify system clocks during initialization */

void system_diagnostic_task(void) {
    /* Print the system banner first */
    ca_os_banner();

    /* Dump current clock configuration to debug console */
    /* This helps verify if PLLs are configured as expected */
    ca_os_clock_dump();

    /* Check if the CPU is running at the expected frequency */
    if (ca_os_clock_get_freq() < 100000000U) {
        /* Log a warning if the clock is slower than 100MHz */
        ca_stdout_printf("Warning: Low clock speed detected!\r\n");
    }

    /* Suspend the current task using the handle */
    ca_os_suspend_handle(ca_os_self_handle());
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_os_hw_clock_dump() and what is its purpose in debugging?
A: You should call ca_os_hw_clock_dump() during debugging to print the current clock configuration (SYSCLK, AHB, APB1, APB2) to the debug output. It helps verify that the system clock settings are correctly applied and can be used to validate clock tree configurations. The function is intended for use in non-ISR contexts and is not thread-safe, so it should be called from a context where multitasking is not a concern.

Code example conf:0.95 **Q:** Can you provide a minimal C example showing how to use ca_os_hw_clock_dump(with caOS types?
A:

```c
#include "caos.h"

void debug_clocks(void) {
    ca_os_hw_clock_dump();
}
```

Error/Edge case conf:0.90 **Q:** What are the common mistakes or edge cases when using ca_os_hw_clock_dum
A: Common mistakes include calling ca_os_hw_clock_dump() from an ISR context, which is not safe. It is also not thread-safe, so calling it in a multitasking environment without protection can lead to inconsistent output. Additionally, the function relies on SystemCoreClock being correctly updated; if it's not, the printed values will be incorrect. The function is also blocking and will not return until the debug output is complete.

Behavior conf:0.95 **Q:** Is ca_os_hw_clock_dump() blocking or non-blocking, and is it safe to call from an ISR or in a multitasking environment?
A: ca_os_hw_clock_dump() is a blocking function that prints clock information to debug output. It is not safe to call from an ISR context, as it is not ISR-safe. It is also not thread-safe, so calling it in a multitasking environment without proper synchronization can lead to corrupted output. It must be called from a non-ISR, non-RTOS context for reliable behavior.

HW differences conf:0.90 **Q:** How does ca_os_hw_clock_dump() differ between H5xx and H7xx families in terms of registers and peripherals used?

A: In H5xx, the function reads clock configuration from RCC->CFGR2 register for HPRE, APB1, APB2, and APB3 settings, and prints values for AHB, APB1, APB2, and APB3. In H7xx, it reads from multiple registers including D1CFGR, D2CFGR, and D3CFGR for CPU, AHB, and APB1-4 clocks, and prints SYSCLK, CPU, AHB, APB1, APB2, APB3, and APB4 frequencies. H7xx supports more clock domains and has a more complex clock tree compared to H5xx.

`ca_os_hw_clock_reduce_freq`

Header: `caos/drivers/inc/ca_glue_hw.h:77`
GUARD: `EN_CA_GLUE_HW`

```
extern u32 ca_os_hw_clock_reduce_freq(u16 scale); /* reduces CPU
   frequency for power saving; returns the new freq Hz */
```

Dynamically scales the CPU frequency by applying a prescaler to the system clock for power management.

Parametri:

Nome	Tipo	Descrizione
`scale`	`u16`	Percentage of the nominal frequency (1-99). Values 0 or >= 100 are ignored, returning the current frequency without changes.

Ritorna:
The new CPU frequency in Hz after applying the prescaler, or the current frequency if the input scale is invalid.

Note:

- This function is not thread-safe and modifies hardware registers directly. It is intended for power-saving modes where high performance is not required. It is NOT safe to call from an ISR because it triggers a system clock update which may involve complex timing or peripheral re-synchronization. A common mistake is calling this function repeatedly without calling ca_os_clock_restore_freq, which can lead to unpredictable clock states or 'stuck' low-frequency operation. Always ensure that peripherals relying on specific baud rates (like UART or CAN) are re-initialized or disabled before changing the clock, as they will likely stop working correctly at the new frequency.

Uso:

- Used primarily in battery-powered applications or thermal management scenarios to reduce power consumption. It is typically paired with ca_os_clock_restore_freq to return to full performance once the intensive task is complete. The standard sequence is: 1) Perform power-sensitive task, 2) Call ca_os_clock_reduce_freq(scale), 3) Execute low-power logic, 4) Call ca_os_clock_restore_freq() to resume normal operation. Note that on most STM32 architectures, the internal state is managed such that consecutive calls to reduce_freq do not overwrite the original nominal clock configuration, allowing for a clean restoration.

Esempio:

```
/* Example: Reduce CPU speed to 25% for a background task */
u32 original_freq = ca_os_clock_get_freq();

/* Reduce frequency to 25% of nominal */
u32 new_freq = ca_os_clock_reduce_freq(25);

```

```
/* Perform low-power background processing */
/* ... */

/* Restore original frequency to ensure full performance for main tasks */
ca_os_clock_restore_freq();
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_os_hw_clock_reduce_freq() in my embedded application?
A: You should call ca_os_hw_clock_reduce_freq() when you want to reduce the CPU frequency for power saving purposes. It's typically used in low-power modes or when the application doesn't require full CPU performance. The function is not thread-safe and must not be called from an ISR.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_os_hw_clock_reduce_freq()?
A: u32 new_freq = ca_os_hw_clock_reduce_freq(50); // Reduces CPU frequency to 50% of maximum if (new_freq != SystemCoreClock) { // Frequency was changed successfully // Do something with the new frequency }

Parameters conf:0.95 **Q:** What does the 'scale' parameter in ca_os_hw_clock_reduce_freq() represent and what are valid values?
A: The 'scale' parameter represents the desired CPU frequency as a percentage of the maximum frequency (100 = 100%, 50 = 50%). Valid values are integers from 1 to 99. Values 0 or >=100 will return the current frequency without modification.

Return conf:0.90 **Q:** What does ca_os_hw_clock_reduce_freq() return and what does it mean if it returns the current frequency?
A: The function returns the new CPU frequency in Hz after modification. If it returns the current frequency, it means either the input 'scale' was 0 or >=100, or an invalid parameter was passed, causing no change to the clock frequency.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_os_hw_clock_reduce_
A: Common mistakes include calling this function from an ISR, calling it too frequently, or passing invalid scale values (0 or >=100). The function is not thread-safe, so concurrent access from multiple threads must be avoided. It also requires that the system clock is already initialized.

Behavior conf:0.90 **Q:** Is ca_os_hw_clock_reduce_freq() blocking or non-blocking, and is it safe to call from an ISR?
A: The function is non-blocking. However, it is not safe to call from an ISR because it modifies system clock registers and requires updating the core clock. It is also not thread-safe and should not be called concurrently from multiple threads.

HW differences conf:0.90 **Q:** How does ca_os_hw_clock_reduce_freq() differ between STM32 H7xx and other families like F1xx or F4xx?
A: The H7xx family uses RCC->D1CFGR with D1CPRE bits to control the CPU clock prescaler, while F1xx, F4xx, and F7xx families use RCC->CFGR with HPRE bits. The H5xx family uses RCC->CFGR2 with HPRE bits. Each family has its own register layout and bit positions for clock configuration.

`ca_os_hw_clock_restore_freq`

Header: `caos/drivers/inc/ca_glue_hw.h:78`
GUARD: `EN_CA_GLUE_HW`

```
extern u32 ca_os_hw_clock_restore_freq(void); /* restores nominal CPU
   frequency; returns the freq Hz */
```

Restores the CPU clock frequency to its nominal value after a temporary reduction.

Ritorna:
Returns the current CPU frequency in Hz as a u32 value. This value is derived from the internal SystemCoreClock variable after updating the hardware registers.

Note:

- This function is not thread-safe and should not be called from an ISR. It is intended to be used in the main thread context to revert changes made by frequency-scaling functions. The function modifies hardware clock configuration registers (like RCC_CFGR or D1CFGR) and triggers a SystemCoreClockUpdate, which recalculates the system clock speed based on the new register state. A common mistake is calling this function while other peripherals are relying on a specific, stable clock timing, which could cause communication errors or timing drifts in time-sensitive tasks.

Uso:

- This function is typically used in power management scenarios where the CPU frequency is temporarily lowered to save energy during idle periods or low-activity tasks. It is paired with ca_os_clock_reduce_freq, which performs the initial down-scaling. The typical sequence is: call ca_os_clock_reduce_freq to save power, perform the low-power task, and then call ca_os_clock_restore_freq to return to full performance mode before executing time-critical operations.

Esempio:

```c
/* Example: Temporarily reduce clock for a background task, then restore */

void perform_low_power_task(void) {
    /* Reduce frequency to save power */
    ca_os_clock_reduce_freq(2);

    /* Perform background work */
    // ... logic here ...

    /* Restore nominal frequency before returning to high-performance tasks */
    u32 current_freq = ca_os_clock_restore_freq();

    if (current_freq == 0) {
        /* Handle error: clock update failed */
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_os_hw_clock_restore_freq() in my caOS application?
A: You should call ca_os_hw_clock_restore_freq() when you need to restore the CPU frequency to its nominal value after previously scaling it down, for example, after entering a low-power mode or during system initialization. It's typically used to ensure the system runs at full performance after a clock adjustment.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_os_hw_clock_restore_freq() with caOS types?
A: u32 new_freq = ca_os_hw_clock_restore_freq(); if (new_freq == 0) { // Handle error: frequency restoration failed } // Use new_freq for further operations

Return conf:0.90 **Q:** What does a return value of 0 from ca_os_hw_clock_restore_freq() indicate?
A: A return value of 0 indicates that the frequency restoration failed, possibly due to an invalid or inconsistent value stored in 'saved_hpre' or 'saved_d1cpre'. This could lead to incorrect system behavior or a crash if not handled properly.

HW differences conf:0.95 **Q:** How does ca_os_hw_clock_restore_freq() differ between STM32 H7xx and H5xx microcontrollers?
A: On H7xx microcontrollers, the function modifies the RCC->D1CFGR register using the saved_d1cpre value, while on H5xx microcontrollers, it modifies the RCC->CFGR2 register using the saved_hpre value. This reflects differences in clock configuration register layouts between these families.

`ca_os_hw_periph_reset`

Header: `caos/drivers/inc/ca_glue_hw.h:82`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_os_hw_periph_reset(void); /* force + release the reset of
    every peripheral (RCC xxxRSTR): ca_os_end */
```

Forces a hardware reset and immediate release of all peripherals via the RCC reset registers.

Ritorna:
void

Note:

- This function is non-blocking and executes extremely quickly by writing directly to the RCC reset registers. It is not thread-safe in the sense that it affects the entire system state, so it must only be called when you are certain no other tasks are actively using peripherals. It is generally safe to call from an ISR, but doing so is highly discouraged as it will abruptly stop all peripheral operations (like DMA transfers or active communication), likely causing system instability. The most common beginner mistake is calling this during normal operation; it should only be used during system initialization or when performing a full system shutdown/reboot sequence.

Uso:

- This function is primarily used during the system shutdown sequence (ca_os_end) or during a deep system re-initialization to ensure that all peripherals are returned to a clean, known state. By forcing a reset, you prevent issues where a peripheral (like an ADC or a communication interface) might be left in an active state that prevents proper reconfiguration or causes calibration errors. It is typically paired with clock de-initialization functions to ensure the hardware is completely quiescent before the CPU enters a low-power state or performs a software reset.

Esempio:

```
/* Example: Preparing the system for a clean shutdown or re-init */

void system_shutdown_sequence(void)
{
    /* 1. Stop all active tasks to prevent peripheral access during reset */
    ca_os_sched_lock();

    /* 2. Force all peripherals into their reset state to clear pending flags */
    ca_os_hw_periph_reset();

    /* 3. Perform additional cleanup, such as disabling clocks or clearing
    memory */
    /* ca_clock_hw_deinit(); // Hypothetical clock deinit */

    /* 4. Finalize OS shutdown */
    ca_os_end();
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to call ca_os_hw_periph_reset()?
A: This function should be used during system initialization or as part of a full system shutdown or reboot sequence. It is designed to return all peripherals to a clean, known state, which helps prevent issues where active peripherals might interfere with re-configuration or cause calibration errors.

Code example conf:0.90 **Q:** How do I implement a system shutdown sequence using ca_os_hw_periph_reset?
A: You should ensure no tasks are using peripherals, then invoke the reset function. Example: void system_shutdown(void) { /* Ensure no active tasks */ ca_os_hw_periph_reset(); /* Proceed to low-power state or reboot */ }

Error/Edge case conf:1.00 **Q:** What is the most common mistake developers make when using ca_os_hw_periph_reset?
A: The most common mistake is calling this function during normal system operation. Because it forces a reset of all peripherals, it will abruptly stop active operations like DMA transfers or communication interfaces, leading to system instability.

Behavior conf:1.00 **Q:** Is ca_os_hw_periph_reset thread-safe or safe to call from an ISR?
A: It is not thread-safe because it affects the entire system state. While it is technically safe to call from an ISR, doing so is highly discouraged as it will immediately halt all peripheral operations, likely causing the system to become unstable.

HW differences conf:1.00 **Q:** How does the implementation of ca_os_hw_periph_reset differ across STM32 families?
A: The implementation varies based on the specific RCC register architecture of the MCU family. For example, the F1xx family resets peripherals via APB1RSTR and APB2RSTR, whereas the F4xx and F7xx families include additional reset registers for AHB1, AHB2, and AHB3 buses to accommodate their specific peripheral sets, such as ETHMACRST or QSPIRST.

`ca_os_job`

Header: `caos/inc/scheduler.h:529`
Source: `caos/src/scheduler.c:2668:2673`
GUARD: `EN_SCHEDULER`

```
ca_task_handle_t ca_os_job(const char *name, u16 priority, ca_task_fn_t
    func, void *arg, ca_stack_size_t stack_size, ca_job_type type,u8
    fault_type)
```

Creates and starts a new OS task (thread) to run a specific function concurrently.

Parametri:

Nome	Tipo	Descrizione
name	const char *	A descriptive string name for the task, useful for debugging and system monitoring.
priority	u16	Task priority level (e.g., CA_TH_LEVEL0). Higher values usually indicate higher urgency.
func	ca_task_fn_t	The pointer to the C function that will execute as the task entry point.
arg	void *	An optional pointer passed to the task function upon startup. It is cast internally to a 32-bit value.
stack_size	ca_stack_size_t	The amount of memory allocated for the task stack (e.g., STACK_SMALL). Must be sufficient to avoid stack overflow.
type	ca_job_type	The type of job (e.g., CA_JOB_STD), defining how the scheduler handles the task.
fault_type	u8	Defines the recovery behavior if the task crashes (e.g., CA_FAULT_RESTART_THREAD).

Ritorna:
Returns a ca_task_handle_t (typically a pointer or index). Returns NULL if the task creation fails, which usually happens due to insufficient heap memory or reaching the maximum allowed number of concurrent tasks.

Note:

- This function is non-blocking; it registers the task with the scheduler and returns immediately. It is not thread-safe to call from an ISR because it performs heap allocation for the task stack and control block. You must ensure the OS memory manager is initialized before calling this. A common beginner mistake is underestimating the stack size; if the stack is too small, the task will trigger a hard fault or memory corruption when it performs nested function calls or declares large local variables. Always verify stack usage using ca_memory_get_task_usage during development.

Uso:

- This function is the standard way to spawn background services in caOS. It is typically called within a service's 'start' function (e.g., ca_arp_service_start) to launch the main loop of that service. It is often used in conjunction with synchronization primitives like ca_sem_timedwait or ca_queue_heap_receive to allow the task to wait for events without consuming CPU cycles. Once started, the task handle can be used later to manage the task, such as changing its priority with ca_os_set_priority_handle or suspending it with ca_os_job_suspend.

Esempio:

```
void my_service_task(void *arg) {
    while (1) {
        /* Perform periodic work */
        ca_os_delay_ms(1000);
    }
}

void my_service_start(void) {
    /* Create a background task to handle specific logic */
    ca_task_handle_t handle = ca_os_job("my_task", CA_TH_LEVEL2, my_service_task
    , NULL, STACK_SMALL, CA_JOB_STD, CA_FAULT_RESTART_THREAD);

    /* Check if the task was created successfully */
    if (handle == NULL) {
```

```
        /* Handle error: log failure or stop system */
        ca_printf("Failed to start task\n");
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_os_job() and what is its purpose in a caOS-based embedded system?
A: You should call ca_os_job() during system initialization or configuration to create a new task in the RTOS. It is used to define a new thread of execution with a specific function, priority, and stack size. The function is non-blocking and must be called from a non-ISR context, typically in setup routines or during system boot. It is not thread-safe and should not be invoked from multiple threads simultaneously.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to create a task using ca_os_job()?
A: u32 my_task_id; void my_task_func(void) { // Task implementation } my_task_id = ca_os_job("MyTask", CA_TH_LEVEL1, my_task_func, 0, STACK_MEDIUM, CA_JOB_STD, CA_FAULT_RESTART_THREAD);

Parameters conf:0.90 **Q:** What are the valid values for the 'type' and 'fault_type' parameters in ca_os_job()?
A: The 'type' parameter can be set to CA_JOB_STD for standard tasks or JOB_ISR for interrupt service routine tasks. The 'fault_type' parameter can be set to CA_FAULT_RESTART_THR to restart the task on crash, or CA_FAULT_STOP_THREAD to stop the task on crash. These values are defined in the caOS system and must be used as specified.

Return conf:0.85 **Q:** What does the return value of ca_os_job() indicate, and what causes it to return 0?
A: The return value of ca_os_job() is the ID of the created task. If the return value is 0, it indicates that the task creation failed. Failures can occur due to invalid parameters such as a NULL name, invalid stack size, or incorrect priority values. The function does not block, and the ID is used for further operations like task deletion or status checks.

---

ca_os_job_suspend

Header: caos/inc/scheduler.h:549
Source: caos/src/scheduler.c:2691:2696
GUARD: EN_SCHEDULER

```
ca_task_handle_t ca_os_job_suspend(const char *name, u16 priority,
    ca_task_fn_t func, void *arg, ca_stack_size_t stack_size, ca_job_type
     type,u8 fault_type)
```

Creates a new task in a suspended state, preventing it from running until explicitly resumed.

Parametri:

Nome	Tipo	Descrizione
name	const char *	A unique string identifier for the task, useful for debugging and task listing.
priority	u16	Scheduling priority level. Higher values typically indicate higher urgency.
func	ca_task_fn_t	The entry point function (the task code) that will execute when the task is resumed.
arg	void *	A pointer to data passed to the task function upon startup. It is cast internally to a 32-bit value.
stack_size	ca_stack_size_t	The amount of memory allocated for the task's stack. Must be sufficient to prevent overflow.
type	ca_job_type	Defines the job classification (e.g., JOB_SAFE, CA_JOB_STD).
fault_type	u8	Defines the behavior when a fault occurs (e.g., CA_FAULT_ASSERT).

Ritorna:
Returns a ca_task_handle_t representing the created task. Returns CA_TASK_HANDLE_NONE if the task creation fails (e.g., out of memory or invalid parameters).

Note:

- This function is non-blocking and creates a task in the CA_ST_TH_SUSPENDED state, meaning it will not be picked up by the scheduler until ca_os_resume_handle is called. It is not safe to call this from an ISR because it performs memory allocation and thread management. A common beginner mistake is to create a task and expect it to run immediately; you must remember to call the resume function later. Ensure the system is initialized before creating tasks, typically within ca_os_main, as the scheduler relies on the internal task control block being fully populated before execution begins.

Uso:

- Use this function when you need to prepare multiple tasks during system startup but want to control the exact sequence in which they begin execution. It is typically paired with ca_os_resume_handle to start the task after initialization is complete. This pattern is essential for preventing race conditions where a task might attempt to access hardware or global resources before they are fully configured. It is often used in the ca_os_main entry point to define the system architecture before calling ca_os_run.

Esempio:

```
/* Example: Creating a task in suspended state and resuming it later */
ca_task_handle_t my_task_handle;

void task_entry(void *arg) {
    /* Task logic here */
    while(1) {
        /* Perform work */
    }
}

void ca_os_main(void) {
    /* Create the task but keep it suspended */
    my_task_handle = ca_os_job_suspend("worker", 10, task_entry, NULL, 512,
    CA_JOB_STD, CA_FAULT_ASSERT);

    if (my_task_handle != CA_TASK_HANDLE_NONE) {
        /* Perform other system initializations here */
```

```
17
18          /* Now start the task */
19          ca_os_resume_handle(my_task_handle);
20      }
21
22      ca_os_run(true);
23  }
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_os_job_suspend instead of creating a task that runs immediately?
A: Use ca_os_job_suspend during the boot sequence when you need to define tasks but control exactly when they start. This is ideal for preventing race conditions where a task might attempt to access hardware peripherals before their drivers have finished initialization.

Code example conf:1.00 **Q:** How do I define a task using ca_os_job_suspend in my ca_os_main function?
A: You can initialize a task in a suspended state like this: u32 my_thid = ca_os_job_suspend("SensorTa 10, my_task_func, 0, 512, CA_JOB_STD, CA_FAULT_ASSERT);

Parameters conf:1.00 **Q:** What is the purpose of the fault_type parameter in ca_os_job_suspend?
A: The fault_type parameter defines the behavior of the system if the task encounters a fault, such as using the CA_FAULT_ASSERT constant.

Return conf:1.00 **Q:** What does the return value of ca_os_job_suspend indicate?
A: The function returns the unique task ID (thid) upon success. If the task creation fails, such as due to out-of-memory conditions or invalid parameters, it returns (u32)-1.

Error/Edge case conf:1.00 **Q:** What are the common pitfalls when using ca_os_job_suspend?
A: A common mistake is forgetting to call ca_os_resume later, which leaves the task permanently dormant. Additionally, ensure the stack size is adequate, as stack overflow is a frequent cause of silent system crashes. Note that this function is not thread-safe to call from an ISR.

`ca_os_main`

Header: `caos/inc/caos.h:137`
GUARD: `EN_CAOS`

```
1  extern void ca_os_main(void)
```

The entry point function where the application logic resides after system initialization.

Ritorna:
This function returns void; it is not intended to return to the caller.

Note:

- ca_os_main is the user-defined application entry point, typically called by the system startup sequence immediately after ca_os_init. It is not a task itself, but rather the context where you define your application's behavior, such as initializing peripherals or spawning tasks. Because it runs in the main thread context, it is not an ISR and should not be called from one. A common mistake is to perform blocking operations here without considering that if the OS scheduler is not yet running, the entire system will hang until that operation completes.

Uso:

- This function is used to structure the application's startup phase. It is typically paired with ca_os_init at the beginning of the program and often leads into the scheduler start (ca_os_run) or a custom main loop. You use it to initialize hardware drivers, allocate memory pools, and create tasks. If you do not call ca_os_run, you must implement your own infinite loop inside ca_os_main to prevent the program from exiting or reaching the ca_os_dead function.

Esempio:

```c
void ca_os_main(void) {
    /* Initialize system banner for debugging */
    ca_os_banner();

    /* Initialize memory management before allocating tasks */
    ca_memory_init();

    /* Initialize a peripheral driver */
    ca_temp_init();

    /* Setup tasks or queues here using ca_queue_heap_init or similar */

    /* Start the OS scheduler to manage tasks */
    ca_os_run();

    /* If ca_os_run returns, something went wrong */
    while(1) {
        /* Safety loop to prevent system exit */
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should ca_os_main() be called in a caOS application, and what is its role in the system?
A: ca_os_main() should be called after initializing the caOS system with ca_os_init(). It serves as the main task loop of the system, typically implementing an infinite loop that performs system operations. It is the primary entry point of the program when not using the RTOS scheduler, and it never returns, blocking execution indefinitely.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_os_main() with caOS types?
A:

```c
#include "caos.h"

void ca_os_main(void) {
    while (1) {
        // Main system loop
    }
}
```

Parameters conf:0.90 **Q:** What is the meaning of the void parameter in ca_os_main(), and what happens if it's called incorrectly?
A: The ca_os_main() function takes no parameters, as indicated by the void void parameter. It is a void function, meaning it does not accept any inputs. If called before ca_os_init(), it may cause a crash or undefined behavior because the system is not properly initialized for operation.

`ca_os_remove_all_jobs`

Header: `caos/inc/scheduler.h:516`
Source: `caos/src/scheduler.c:2704:2714`

GUARD: `EN_SCHEDULER`

```
bool ca_os_remove_all_jobs(void)
```

Iteratively removes all currently registered tasks from the system scheduler.

Ritorna:
Returns true if all tasks were successfully removed, or false if an error occurred during the removal of any individual task.

Note:

- This function is a blocking operation because it iterates through the entire task list and performs a removal for each entry. It is not thread-safe if other threads are simultaneously attempting to add or modify tasks, as it relies on the task manager's internal state. It must NOT be called from an ISR (Interrupt Service Routine) because it performs multiple operations that may involve memory management or scheduler locks. A common beginner mistake is calling this while other tasks are still actively running or waiting on resources, which can lead to undefined system behavior or memory corruption if those tasks are suddenly terminated.

Uso:

- This function is typically used during a system reset, a major configuration change, or when transitioning the device into a low-power state where all active processing must cease. It is often paired with ca_os_run to restart the scheduler after a clean slate has been established. Developers use this to ensure that no 'zombie' tasks remain in memory before re-initializing the system or loading a new set of operational tasks.

Esempio:

```
/* Example: Resetting the system by clearing all tasks and re-initializing */

void system_reboot_sequence(void) {
    // Stop all current operations
    if (ca_os_remove_all_jobs()) {
        // All tasks removed successfully
        ca_printf("System cleared. Re-initializing...\n");

        // Perform memory cleanup if necessary
        ca_memory_free_all_owned_by(0);

        // Re-run the scheduler to start fresh tasks
        ca_os_run();
    } else {
        // Handle failure to clear tasks
        ca_printf("Error: Could not clear all tasks!\n");
    }
}
```

Q&A — fine-tuning review

4/4 pairs · qwen3-coder:latest

Usage conf:0.95 **Q:** When should I call ca_os_remove_all_jobs() in a caOS application?
A: You should call ca_os_remove_all_jobs() when you need to reset the scheduler before restarting it or during system shutdown. It is useful for cleaning up all scheduled tasks to ensure a clean state.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_os_remove_all_jobs() in a caOS application?
A: bool success = ca_os_remove_all_jobs(); if (success) { // All jobs removed successfully } else { // At least one job removal failed }

Return conf:0.95 **Q:** What does the return value of ca_os_remove_all_jobs() indicate?
A: The function returns true if all scheduled tasks were removed successfully. It returns false if at least one removal failed, such as when a task does not exist or has already been completed.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_os_remove_all_jobs()?
A: Common mistakes include ignoring the return value and assuming all tasks were removed. Additionally, calling this function from an ISR is not allowed. The function is not thread-safe unless protected by a mutex, and it assumes the task manager is in a consistent state. If the task manager becomes inconsistent during execution, unpredictable behavior may occur.

`ca_os_remove_job_handle`

Header: `caos/inc/scheduler.h:577`
Source: `caos/src/scheduler.c:2848:2855`
GUARD: `EN_SCHEDULER`

```
bool ca_os_remove_job_handle(ca_task_handle_t h)
```

Safely removes a task from the scheduler using its unique handle.

Parametri:

Nome	Tipo	Descrizione
`h`	`ca_task_handle_t`	The unique handle of the task to be removed. Must be a valid handle previously obtained from ca_os_task_handle or similar lookup functions.

Ritorna:
Returns true if the task was successfully found and removed; returns false if the handle is invalid, the task does not exist, or the removal operation failed.

Note:

- This function is thread-safe and uses internal IRQ masking to ensure the task is not moved or modified by the scheduler while being removed. It is non-blocking and returns immediately after the task is unlinked from the scheduler's execution list. While it is technically safe to call from an ISR due to the internal IRQ masking, it is generally discouraged to kill tasks from an interrupt context unless absolutely necessary for system recovery. The most common mistake is attempting to use a handle that has already been invalidated or failing to free the task's associated memory before removing it, which leads to memory leaks.

Uso:

- This function is used to terminate tasks that are no longer needed, such as one-shot jobs or tasks that have encountered a fatal error. It is typically used in a cleanup sequence: first, identify the task using ca_os_task_handle, then release any associated resources (like memory allocated via ca_memory_allocate or queues), and finally call ca_os_remove_job_handle to reclaim the scheduler slot. It is often used in CLI commands or system monitors to kill runaway threads or during system shutdown sequences to clean up active workers.

Esempio:

```
/* Example: Safely terminating a task by its handle */
ca_task_handle_t my_task_h = ca_os_task_handle(some_task_context);

```

```c
if (my_task_h != CA_TASK_HANDLE_NONE) {
    /* Perform cleanup of task-specific resources here */
    /* e.g., ca_memory_free_all_owned_by(my_task_h); */

    /* Remove the task from the scheduler */
    if (ca_os_remove_job_handle(my_task_h)) {
        /* Task successfully removed */
    } else {
        /* Handle error: task could not be removed */
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_os_remove_job_handle in my application?
A: You should use this function to terminate tasks that are no longer required, such as one-shot jobs or tasks that have encountered a fatal error. It is typically used in a cleanup sequence after releasing associated resources like memory or queues, or within CLI commands to kill runaway threads.

Code example conf:0.90 **Q:** How do I correctly remove a task using its handle in caOS?
A: First, obtain the handle, perform cleanup, then call the removal function: ca_task_handle_t h = ca_os_task_handle(task_id); if (h != NULL) { ca_memory_free(task_data); ca_os_remove_job_

Parameters conf:1.00 **Q:** What are the requirements for the ca_task_handle_t parameter?
A: The parameter 'h' must be a unique handle of the task to be removed, which must have been previously obtained from ca_os_task_handle or similar lookup functions. You must ensure the handle is still valid and has not been invalidated by system task-list compaction.

Return conf:1.00 **Q:** What do the return values of ca_os_remove_job_handle indicate?
A: The function returns true if the task was successfully found and removed. It returns false if the provided handle is invalid, the task does not exist, or the removal operation failed.

`ca_os_reset`

Header: `caos/inc/scheduler.h:490`
Source: `caos/src/scheduler.c:2757:2789`
GUARD: `EN_SCHEDULER`

```c
void ca_os_reset(void)
```

Performs a complete system reset of the microcontroller.

Ritorna:
This function does not return, as it triggers a hardware reset of the CPU.

Note:

- This function is a 'nuclear' option for system recovery. It disables all interrupts, attempts to arm the Independent Watchdog (IWDG) to ensure a reset even if the core hangs, and then triggers a NVIC System Reset. It is not thread-safe in the sense that it stops all OS activity immediately. It should never be called from an ISR unless you are absolutely certain the system state is unrecoverable, as it will abruptly terminate all tasks and peripheral operations. The most common beginner mistake is calling this function during normal operation to 'clear' a minor error; instead, use it only when the system has reached an unrecoverable state (e.g., hardware failure, critical memory corruption, or a deadlock that cannot be resolved).

Uso:

- Use this function when the system enters a state where normal operation is impossible and recovery via software logic is not feasible. It is typically used in CLI handlers or error-handling routines after logging the failure. It is often paired with ca_os_sleep to allow the user or a terminal to read a final error message before the system reboots. It is the final step in a recovery sequence: log the error, notify the user, wait for the message to be transmitted, and then call ca_os_reset.

Esempio:

```c
/* Example: Resetting the system after a critical configuration error */
void handle_critical_error(void) {
    /* Notify the user via the CLI */
    ca_printf("Critical error detected. Rebooting in 3 seconds...\n");

    /* Wait to ensure the message is sent to the terminal */
    ca_os_sleep(3000);

    /* Perform a full system reset */
    ca_os_reset();

    /* This line will never be reached */
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is it appropriate to call ca_os_reset in my application?
A: You should call ca_os_reset only when the system reaches an unrecoverable state, such as critical memory corruption, hardware failure, or an unresolvable deadlock. It is intended as a final recovery step after logging the failure and notifying the user.

Code example conf:0.90 **Q:** How do I implement a safe reset sequence in caOS?
A: To safely reset, you should log the error, notify the user, and wait for the message to be transmitted before calling the reset function. Example: { cstr msg = "Critical failure, rebooting..."; log_error(msg); ca_os_sleep(1000); ca_os_reset(); }

Return conf:1.00 **Q:** What value does ca_os_reset return upon success?
A: The function does not return any value. It triggers a hardware reset of the CPU, which immediately terminates all OS activity.

Error/Edge case conf:1.00 **Q:** What is a common mistake developers make when using ca_os_reset?
A: The most common mistake is calling this function during normal operation to 'clear' minor errors. It should be reserved strictly for unrecoverable states.

Behavior conf:1.00 **Q:** Is ca_os_reset thread-safe or safe to call from an ISR?
A: It is not thread-safe as it immediately stops all OS activity. It should never be called from an ISR unless the system state is absolutely unrecoverable, as it will abruptly terminate all tasks and peripheral operations.

`ca_os_resume_handle`

Header: `caos/inc/scheduler.h:579`
Source: `caos/src/scheduler.c:2868:2875`
GUARD: `EN_SCHEDULER`

```c
bool ca_os_resume_handle(ca_task_handle_t h)
```

Resumes a previously suspended task using its handle.

Parametri:

Nome	Tipo	Descrizione
h	ca_task_handle_t	The handle of the task to resume. Must be a valid handle obtained via ca_os_task_handle or similar.

Ritorna:
Returns true if the task was successfully resumed, or false if the handle is invalid or the task could not be resumed.

Note:

- This function is thread-safe and uses internal critical sections to ensure the task list is accessed safely. It is not intended to be called from an ISR because it performs scheduler-level operations that may trigger a context switch. A common mistake is passing a NULL handle or a handle to a task that has already been destroyed, which will result in a false return. Always ensure the task exists before attempting to resume it, as the scheduler cannot resume a task that is no longer in the task list.

Uso:

- This function is used to transition a task from a suspended state back to a ready state, allowing the scheduler to resume its execution. It is typically used in multi-tasking architectures where a 'master' or 'monitor' task controls the lifecycle of 'worker' tasks, such as initializing hardware or waiting for specific system events. It is commonly paired with ca_os_suspend_handle to implement power-saving or resource-gating patterns, where tasks are kept dormant until their specific service is required.

Esempio:

```
/* Example: Resuming a worker task after a delay */
ca_task_handle_t g_slave_task = /* handle obtained during task creation */;

/* Wait for system initialization to complete */
ca_os_sleep(2000);

/* Resume the slave task to begin its operation */
bool success = ca_os_resume_handle(g_slave_task);

if (!success) {
    /* Error: The task handle is invalid or the task was already destroyed */
    ca_printf("Failed to resume slave task!\n");
} else {
    ca_printf("Slave task resumed successfully.\n");
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_os_resume_handle in my application?
A: You should use this function to transition a task from a suspended state back to a ready state. It is typically used by a master or monitor task to control the lifecycle of worker tasks, such as waking them up after waiting for specific system events or to implement resource-gating patterns.

Code example conf:0.90 **Q:** How do I implement a simple task resumption in caOS?
A: Assuming you have a valid handle stored in a variable, you would call it as follows: ca_task_handle_t my_task = ca_os_task_handle(task_id); if (ca_os_resume_handle(my_task) != NULL) { /* Task resumed successfully */ }

Parameters conf:1.00 **Q:** What are the requirements for the ca_task_handle_t parameter?
A: The parameter must be a valid handle obtained via ca_os_task_handle or a similar function. Passing a NULL handle or a handle to a task that has already been destroyed is invalid and will result in a failure.

Return conf:1.00 **Q:** What does the return value of ca_os_resume_handle indicate?
A: The function returns the handle of the resumed task if successful. It returns NULL if the provided handle is invalid, if the task has already been destroyed, or if the task could not be resumed by the scheduler.

`ca_os_run`

Header: `caos/inc/caos.h:141`
Source: `caos/src/scheduler.c:2717:2719`
GUARD: `EN_CAOS`

```
extern void ca_os_run(bool active)
```

Enables or disables the caOS task scheduler execution.

Parametri:

Nome	Tipo	Descrizione
`active`	`bool`	Set to true to start the scheduler and allow tasks to run, or false to stop the scheduler.

Ritorna:
void

Note:

- This function acts as the master switch for the RTOS scheduler. When called with 'true', the system begins context switching between tasks based on their priority and state. If called with 'false', the scheduler stops, effectively freezing all task execution. This function is not thread-safe as it modifies global system state; it is intended to be called during the system initialization phase in main(). Do not call this from an ISR, as it is designed for high-level system control, not real-time event handling. A common mistake is calling this before all necessary tasks or hardware drivers are fully initialized, which can lead to the scheduler attempting to execute tasks that are not yet ready. Furthermore, note that before this function is called, the system lacks a task context (TLB=NULL), meaning functions that rely on task-specific resources (like certain logging or stdout functions) may fail or behave unexpectedly.

Uso:

- This function is the final step in the system boot sequence. It is typically called after all hardware peripherals, memory pools, and tasks have been initialized and registered. You pair this with initialization functions like ca_memory_init, ca_os_configure, or ca_cli_register. Once ca_os_run(true) is executed, the CPU control is handed over to the scheduler, and the main() function usually does not return. If you need to perform a controlled shutdown or system reset, you might call it with 'false', though this is rare in standard embedded applications.

Esempio:

```
/* 1. Initialize memory management */
ca_memory_init();

/* 2. Configure OS parameters */
ca_os_configure();

/* 3. Register a CLI interface for debugging */
ca_cli_register();

```

```
/* 4. Final step: start the scheduler to begin task execution */
ca_os_run(true);

/* The code below this line will not be reached while the scheduler is running
   */
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_os_run() in a caOS application, and what is its purpose?
A: You should call ca_os_run(true) after initializing the RTOS system to start the scheduler and begin task execution. Calling ca_os_run(false) stops the scheduler from running new tasks but does not terminate tasks already in progress. It is used to control the overall execution flow of the RTOS, allowing developers to pause or resume task scheduling as needed.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_os_run() with caOS types?
A:

```
#include "caos.h"

int main(void) {
    ca_os_init();
    ca_os_run(true);  // Start the scheduler
    while (1) {
        // Main loop
    }
    return 0;
}
```

Parameters conf:0.90 **Q:** What does the 'active' parameter in ca_os_run() control, and what are the valid values?
A: The 'active' parameter controls whether the scheduler is started or stopped. If set to true, the scheduler begins executing tasks; if set to false, it stops scheduling new tasks. Valid values are only true or false. Passing any other value will result in undefined behavior.

Return conf:0.85 **Q:** Does ca_os_run() return any value, and what causes it to fail?
A: ca_os_run() does not return any value. It modifies a global flag 'g_scheduler_active' internally to control the scheduler's state. Failures are not explicitly defined, but improper usage such as calling it from an ISR or before RTOS initialization may lead to unpredictable behavior or system crashes.

`ca_os_running`

Header: `caos/inc/scheduler.h:501`
Source: `caos/src/scheduler.c:2139:2141`
GUARD: `EN_SCHEDULER`

```
bool ca_os_running(void)
```

Checks if the caOS scheduler is currently active and managing tasks.

Ritorna:
Returns true (1) if the scheduler is running, false (0) if the system is in the initialization phase, halted, or in a pre-scheduler state.

Note:

- This function is non-blocking and thread-safe as it simply reads a global status flag. It is safe to call from both thread mode and ISRs (Interrupt Service Routines). The most common mistake is assuming that if this returns true, the system is fully initialized; it only indicates that the scheduler loop has started. You must ensure the OS has been configured via ca_os_configure before relying on this status. Do not use this to wait for the OS to start; it is a status check, not a synchronization primitive.

Uso:

- This function is typically used in low-level drivers or diagnostic code to determine if it is safe to perform OS-dependent operations, such as context switching or task-based synchronization. For example, in a hardware interrupt handler, you might check this to decide whether to trigger a task wake-up or to perform a direct, non-OS-aware action. It is often used as a guard clause in system-wide error handlers or hardware abstraction layers to prevent calling scheduler-dependent functions (like ca_os_yield) before the kernel is operational. It is typically checked during the system boot sequence to gate the execution of task-dependent logic.

Esempio:

```c
/* Check if the OS is running before triggering a task wake-up from an ISR */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    /* Only attempt to signal a task if the scheduler is active */
    if (ca_os_running()) {
        /* Signal the event to the waiting task */
        ca_event_set_isr(my_event_handle);
    } else {
        /* Fallback: log error or handle manually if OS is not ready */
        ca_printf("System not ready, event ignored\r\n");
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should a developer call ca_os_running() and why is it important in an ISR context?
A: A developer should call ca_os_running() to check if the RTOS is active before using any RTOS functions like task_delay, semaphores, or queues. This is especially important in ISR context because calling RTOS functions when the scheduler is not active can lead to unpredictable behavior. The function is non-blocking and safe to call from ISR, but it's not thread-safe as it reads a global variable without protection.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_os_running() with caOS types?
A: u8 check_os_status(void) { if (ca_os_running()) { return 1; // OS is active } else { return 0; // OS is not active } }

Return conf:0.90 **Q:** What does ca_os_running() return when the RTOS is active versus inactive, and what causes the function to return false?
A: ca_os_running() returns true when the RTOS is active (scheduler is running) and false when it is not active (scheduler is disabled). The function returns false when the global variable g_scheduler_active is set to false, which typically happens before the RTOS is initialized or when the scheduler has been explicitly stopped. Calling this function in non-RTOS contexts (like IRQ handlers before initialization) can also lead to unpredictable results.

---

ca_os_sched_lock

Header: caos/inc/scheduler.h:691
Source: caos/src/scheduler.c:2395:2400
GUARD: EN_SCHEDULER

```
void ca_os_sched_lock(void)
```

Prevents the RTOS scheduler from switching tasks, effectively creating a critical section for the current thread.

Ritorna:
This function returns void and does not provide a status code.

Note:

- This function is non-blocking and executes extremely quickly by disabling interrupts to modify internal scheduler state. It is thread-safe in the sense that it tracks the owner, but it is NOT intended to be called from an ISR, as ISRs should never attempt to lock the scheduler. The most common beginner mistake is forgetting to call ca_os_sched_unlock() after the critical operation, which will permanently freeze the scheduler and prevent any other tasks from running. Always ensure that every lock has a corresponding unlock, preferably using a try-finally pattern or ensuring no early returns exist between them.

Uso:

- Use this function when you need to perform a short, atomic operation that must not be interrupted by other tasks, such as modifying shared global variables or accessing hardware registers that are not protected by mutexes. It is typically used in pairs: call ca_os_sched_lock() before the sensitive code block and ca_os_sched_unlock() immediately after. It is frequently used in memory management or low-level driver code where task switching during a pointer update could lead to system instability or data corruption.

Esempio:

```
/* Example: Protecting a shared memory operation */
void update_shared_data(void) {
    /* Lock the scheduler to prevent task switching during memory access */
    ca_os_sched_lock();

    /* Perform critical memory operation */
    /* Assuming a pointer 'shared_ptr' is being updated */
    // shared_ptr = new_value;

    /* Always unlock the scheduler to allow other tasks to run */
    ca_os_sched_unlock();
}
```

Q&A — fine-tuning review *2/2 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_os_sched_lock() in my caOS application?
A: You should use ca_os_sched_lock() when performing short, atomic operations that must not be interrupted by other tasks, such as modifying shared global variables or accessing hardware registers that are not protected by mutexes. It is commonly used in memory management or low-level driver code to prevent system instability or data corruption during sensitive pointer updates.

Code example conf:0.95 **Q:** How do I correctly implement a critical section using ca_os_sched_lock() and ca_os_sched_unlock()?
A: You must call ca_os_sched_lock() before the sensitive code block and ca_os_sched_unlock() immediately after. Example:

ca_os_sched_lock(); /* Perform atomic operation on shared register or memory */ u32 *ptr = (u32*)0x40001000; *ptr = 0x01; ca_os_sched_unlock();

---

ca_os_sched_unlock

Header: caos/inc/scheduler.h:692
Source: caos/src/scheduler.c:2402:2414
GUARD: EN_SCHEDULER

```
void ca_os_sched_unlock(void)
```

Decrements the scheduler lock counter and triggers a context switch if the lock reaches zero and a switch was deferred.

Ritorna:
void

Note:

- This function is not thread-safe if called from multiple contexts without proper care, but it is designed to be safe when used in pairs with ca_os_sched_lock. It is NOT callable from an ISR because it manipulates the scheduler state and triggers PendSV, which is intended for task-level context switching. The most common beginner mistake is forgetting to call ca_os_sched_unlock after a lock, which permanently freezes the scheduler and prevents other tasks from running, effectively hanging the entire system. Always ensure that every lock has a corresponding unlock, even in error paths.

Uso:

- This function is used to create critical sections where you need to prevent the RTOS scheduler from switching tasks while you perform atomic operations, such as modifying shared memory or hardware registers. It is typically used in a 'lock-do-unlock' pattern to ensure data integrity. In caOS, it is often used internally by memory management functions to ensure that heap allocations are not interrupted by other tasks, preventing heap corruption. You should pair it with ca_os_sched_lock to wrap short, non-blocking code blocks.

Esempio:

```
/* Example: Protecting a shared memory operation */
void update_shared_data(void) {
    /* Lock the scheduler to prevent task switching during memory access */
    ca_os_sched_lock();

    /* Perform sensitive operations */
    /* Assuming a hypothetical shared buffer access */
    // shared_buffer[0] = 0xAA;
    // shared_buffer[1] = 0xBB;

    /* Always unlock to allow the scheduler to resume normal operation */
    ca_os_sched_unlock();
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_os_sched_unlock in my caOS application?
A: You should call ca_os_sched_unlock after completing a critical section that was initiated by ca_os_sched_lock. It is used to decrement the scheduler lock counter and allow the scheduler to resume task switching if the counter reaches zero. It is typically used to protect shared memory or hardware registers during atomic operations.

Code example conf:0.90 **Q:** How do I implement a critical section using caOS types?
A: You should use a lock-do-unlock pattern. For example: ca_os_sched_lock(); u32 shared_val = 10; /* Perform atomic operation */ ca_os_sched_unlock();

Error/Edge case conf:1.00 **Q:** What happens if I forget to call ca_os_sched_unlock after a lock?
A: Forgetting to call ca_os_sched_unlock is a common mistake that permanently freezes the scheduler. This prevents other tasks from running and effectively hangs the entire system. You must ensure every lock has a corresponding unlock, even in error paths.

`ca_os_self_handle`

Header: `caos/inc/scheduler.h:567`
Source: `caos/src/scheduler.c:2833:2835`
GUARD: `EN_SCHEDULER`

```
ca_task_handle_t ca_os_self_handle(void)
```

Retrieves the unique handle of the currently executing task.

Ritorna:
Returns a ca_task_handle_t representing the current task. This handle is a stable identifier used by the OS to track the task, regardless of its position in the scheduler's internal task list.

Note:

- This function is thread-safe and is intended to be called from within a task context. It is not suitable for use inside an ISR (Interrupt Service Routine) because an ISR does not have a 'self' task context in the same way a thread does. The handle returned is more reliable than a task ID (thid) for long-term references, as task IDs can shift if other tasks are destroyed or reordered. A common mistake is to attempt to call this from an interrupt handler; since there is no 'current task' in an ISR, the behavior is undefined and will likely return an invalid handle or cause a system fault. Always ensure you are in a task context before calling this.

Uso:

- This function is primarily used when a task needs to register itself as a waiter for a resource (like a message queue or a stream) so that another task can wake it up later. By passing its own handle to a synchronization primitive, the task ensures that the OS can precisely target it for a wakeup signal. It is typically used in the setup phase of a task loop, immediately before entering a blocking wait state, and is often paired with functions like ca_os_thid_from_handle to log task activity or when checking if the current task is the one waiting on a specific resource lock.

Esempio:

```
/* Task that identifies itself and waits for a message */
static u32 task_worker(void *arg) {
    /* Get our own handle to identify ourselves in logs */
    ca_task_handle_t my_handle = ca_os_self_handle();
    u32 my_thid = ca_os_thid_from_handle(my_handle);

    ca_printf_n("Worker task started with THID: %u", my_thid);

    while (1) {
        /* Wait for a message from a queue */
        /* Assuming 'my_queue' is a valid queue handle */
        void *msg = NULL;
```

```
        if (ca_queue_heap_receive(my_queue, &msg, 1000) == 0) {
            ca_printf_n("Task %u processed message", my_thid);
        } else {
            ca_printf_n("Task %u timed out waiting for message", my_thid);
        }
    }
    return 0;
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_os_self_handle in my task logic?
A: You should call this function when a task needs to register itself as a waiter for a resource, such as a message queue or a stream. It is typically used during the setup phase of a task loop, immediately before entering a blocking wait state, to allow the OS to precisely target the task for a wakeup signal.

Code example conf:0.90 **Q:** How do I retrieve my task's handle to prepare for a wait operation?
A: You can retrieve the handle and use it to register for synchronization as shown: ca_task_handle_t my_handle = ca_os_self_handle(); ca_os_wait_prepare(my_handle);

Return conf:0.95 **Q:** What does the return value of ca_os_self_handle represent, and can it fail?
A: It returns a ca_task_handle_t, which is a stable identifier used by the OS to track the task. The documentation does not specify any failure return values, but it notes that the handle is not suitable for use inside an ISR because an ISR does not have a 'self' task context.

`ca_os_set_priority_handle`

Header: `caos/inc/scheduler.h:582`
Source: `caos/src/scheduler.c:2894:2901`
GUARD: `EN_SCHEDULER`

```
bool ca_os_set_priority_handle(ca_task_handle_t h, u16 newPrio)
```

Dynamically updates the scheduling priority of a specific task handle.

Parametri:

Nome	Tipo	Descrizione
`h`	`ca_task_handle_t`	The handle of the target task to modify. Must be a valid handle obtained from ca_os_task_handle.
`newPrio`	`u16`	The new priority value. In caOS, this is typically a bitmask (e.g., 1 « level). Values outside the system-defined range will cause the function to return false.

Ritorna:
Returns true if the priority was successfully updated, false if the handle is invalid or the priority value is rejected by the task manager.

Note:

- This function is thread-safe and internally disables interrupts to ensure the task manager's state remains consistent during the update. It is NOT callable from an ISR because it performs internal management operations that require a stable task context. A common mistake is passing an uninitialized or NULL handle, which will result in a false return value. Always verify the handle is valid before calling this function to avoid unexpected behavior in the scheduler.

Uso:

- This function is typically used in administrative tasks, such as a CLI or a system monitor, to adjust task importance at runtime. It is often paired with ca_os_task_handle to retrieve the target's handle and ca_os_thid_from_handle to verify the task's existence. Use this to boost critical tasks during high-load scenarios or to lower the priority of background tasks to save CPU cycles for time-sensitive operations.

Esempio:

```c
/* Example: Adjusting task priority via CLI */
ca_task_handle_t h = ca_os_task_handle("DataLogger");
if (h != NULL) {
    /* Priority 3 corresponds to bitmask 1 << 3 */
    u16 new_priority_mask = (1u << 3);
    if (ca_os_set_priority_handle(h, new_priority_mask)) {
        ca_printf_n("Priority updated successfully");
    } else {
        ca_printf_n("Failed to update priority");
    }
} else {
    ca_printf_n("Task not found");
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_os_set_priority_handle in my application?
A: This function is intended for administrative tasks like a CLI or system monitor to adjust task importance at runtime. You should use it to boost critical tasks during high-load scenarios or to lower the priority of background tasks to save CPU cycles for time-sensitive operations.

Code example conf:0.90 **Q:** How do I implement a priority change for a task in caOS?
A: You must first obtain the handle and then pass it to the function. Example: ca_task_handle_t h = ca_os_task_handle(task_id); if (ca_os_set_priority_handle(h, (1 « 2))) { /* Success */ }

Parameters conf:1.00 **Q:** What are the requirements for the newPrio parameter?
A: The newPrio parameter is typically a bitmask (e.g., 1 « level). If the value provided is outside the system-defined range, the function will reject it and return false.

Return conf:1.00 **Q:** What causes ca_os_set_priority_handle to return false?
A: The function returns false if the provided task handle is invalid (such as an uninitialized or NULL handle) or if the priority value is rejected by the task manager.

`ca_os_signal_handle`

Header: `caos/inc/scheduler.h:581`
Source: `caos/src/scheduler.c:2885:2892`
GUARD: `EN_SCHEDULER`

```c
bool ca_os_signal_handle(ca_task_handle_t h)
```

Signals a task to wake up or process an event using its handle.

Parametri:

Nome	Tipo	Descrizione
`h`	`ca_task_handle_t`	The unique handle of the target task. Must be a valid handle obtained via ca_os_task_handle or ca_os_self_handle.

Ritorna:
Returns true if the signal was successfully sent to the task, false if the handle is invalid, the task has been terminated, or the task is not in a state capable of receiving signals.

Note:

- This function is thread-safe and can be called from both task context and ISRs because it internally manages interrupt masking to ensure atomic access to the task manager. It is non-blocking, meaning it will return immediately after updating the task's state. A common mistake is passing a stale handle; if a task is destroyed, its handle becomes invalid, and subsequent calls will return false. Always ensure the target task is still active before signaling, or handle the false return gracefully to avoid logic errors in your state machine.

Uso:

- Use this function to trigger a blocked task to resume execution or to notify a task that a specific event (like data arrival or a timer expiry) has occurred. It is typically used in conjunction with ca_os_wait, where the target task is waiting for a signal to proceed. In a typical lifecycle, you initialize the task, the task enters a wait state, and the controller uses ca_os_signal_handle to wake it up when work is ready. It is the standard way to perform inter-task communication when you only need to notify a task without passing complex data structures.

Esempio:

```
/* Example: Signaling a worker task from an ISR or another thread */
ca_task_handle_t worker_handle;

void notify_worker(void) {
    /* Attempt to signal the worker task */
    if (!ca_os_signal_handle(worker_handle)) {
        /* Handle error: task might be dead or handle is invalid */
        ca_printf("Failed to signal worker task!\n");
    }
}

/* Inside the worker task loop */
void worker_task(void) {
    while (1) {
        /* Wait for a signal from the controller */
        ca_os_wait();
        /* Process the event after being signaled */
        /* Perform task-specific logic here */
    }
}
```

Q&A — fine-tuning review

4/4 pairs · gemini-3.1-flash-lite

Usage conf:1.00 **Q:** When should I use ca_os_signal_handle in my application?
A: Use this function to wake up a blocked task or notify it that an event, such as data arrival or timer expiry, has occurred. It is typically used to resume a task that is currently waiting via ca_os_wait or ca_os_sleep_until.

Code example conf:0.90 **Q:** How do I signal a task from an ISR or another task using caOS types?
A: You can signal a task by passing its handle as follows: void notify_task(ca_task_handle_t h) { bool success = ca_os_signal_handle(h); if (!success) { /* Handle error */ } }

Parameters conf:1.00 **Q:** What are the requirements for the ca_task_handle_t parameter?
A: The parameter must be a valid unique handle obtained via ca_os_task_handle or ca_os_self_handle. Passing a stale handle, such as one belonging to a task that has already been terminated, will result in a failure.

Return conf:1.00 **Q:** What does a false return value from ca_os_signal_handle indicate?
A: A false return indicates the signal was not sent. This occurs if the provided handle is invalid, the target task has been terminated, or the task is currently in a state where it cannot receive signals.

`ca_os_sleep`

Header: `caos/inc/scheduler.h:599`
Source: `caos/src/scheduler.c:2723:2738`
GUARD: `EN_SCHEDULER`

```
u32 ca_os_sleep(u32 ms)
```

Suspends the current task for a specified duration in milliseconds, allowing other tasks to run.

Parametri:

Nome	Tipo	Descrizione
`ms`	`u32`	The duration to sleep in milliseconds. A value of 0 will trigger a yield to the scheduler without a formal sleep delay.

Ritorna:
Returns 0 on success. Returns 1 if the function is called from an invalid context (e.g., no active task or ISR context), indicating the sleep request was ignored.

Note:

- This function is blocking for the calling task, meaning the task will be removed from the ready list until the timer expires. It is NOT thread-safe to call from an Interrupt Service Routine (ISR) because it triggers a context switch; doing so will likely cause a system crash or undefined behavior. Always ensure the OS scheduler is running before calling this. A common beginner mistake is calling this inside a critical section (e.g., between ca_enter_critical and ca_exit_critical), which will deadlock the system because the scheduler cannot preempt the task to perform the sleep.

Uso:

- Used primarily in background tasks, polling loops, or state machines where a task needs to wait for hardware readiness or retry a network operation without consuming 100% of the CPU. It is typically paired with initialization functions like ca_os_tick_init and is often found in loops checking for flags or status registers. It is the standard way to implement 'cooperative multitasking' by yielding control back to the OS so other tasks can execute.

Esempio:

```
/* Example: Polling a hardware status with a sleep delay to save CPU cycles */
void task_monitor_hardware(void *p) {
    /* Assume hardware initialization has already occurred */
    bool hardware_ready = false;

    while (!hardware_ready) {
        /* Check if hardware is ready via a hypothetical status register */
        /* hardware_ready = check_hw_status(); */

        if (!hardware_ready) {
            /* Sleep for 100ms to avoid busy-waiting and allow other tasks to run */
            if (ca_os_sleep(100) != 0) {
```

```
                /* Handle error: sleep failed, perhaps log or break */
                ca_printf_n("Error: ca_os_sleep failed");
                break;
            }
        }
    }

    /* Proceed with task logic once ready */
    ca_printf_n("Hardware is now ready!");
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_os_sleep() in a caOS application?
A: You should use ca_os_sleep() to suspend the current task for a specified number of milliseconds, allowing other tasks to execute. It is commonly used in network services like DHCP, NTP, and syslog to introduce delays between operations or to wait for responses.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_os_sleep() in a caOS task?
A: u32 task_func(void* arg) { while (1) { // Do some work ca_os_sleep(1000); // Sleep for 1 second } return 0; }

Parameters conf:0.95 **Q:** What does the 'ms' parameter in ca_os_sleep() control, and what are valid values?
A: The 'ms' parameter specifies the duration in milliseconds for which the current task will be suspended. Valid values range from 0 to 0xFFFFFFFF. If 0 is passed, the task immediately yields control to other tasks.

Return conf:0.95 **Q:** What do the return values of ca_os_sleep() indicate?
A: A return value of 0 indicates success, meaning the task was successfully suspended. A return value of 1 indicates an error, which occurs if the current task is invalid or the context is not valid.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_os_sleep()?
A: Common mistakes include calling ca_os_sleep() from an ISR, which is not safe. Also, calling it with an uninitialized task or in an invalid context may result in a return value of 1 and undefined behavior. Passing invalid parameters like negative values is not supported.

Behavior conf:0.90 **Q:** Is ca_os_sleep() blocking or non-blocking, and is it safe to call from an ISR?
A: ca_os_sleep() is a blocking function, meaning the calling task will not return until the sleep duration expires or it is manually woken. It is not safe to call from an ISR. It is thread-safe and requires that the current task and context are valid.

`ca_os_sleep_prepare`

Header: `caos/inc/scheduler.h:670`
Source: `caos/src/scheduler.c:1994:1999`
GUARD: `EN_SCHEDULER`

```
u32 ca_os_sleep_prepare(u32 ticks)
```

Prepares the current task to enter a sleep state for a specified number of ticks.

Parametri:

Nome	Tipo	Descrizione
`ticks`	`u32`	The duration in OS ticks to sleep. Positive values represent the timeout duration.

Ritorna:
Returns 0 if the task was successfully prepared for sleep. Returns 1 if the function was called from an invalid context (e.g., no active task).

Note:

- This function is non-blocking itself but prepares the task to be blocked by the scheduler. It is NOT thread-safe and must never be called from an ISR, as it relies on the context of the currently running task. The most common mistake is failing to call ca_os_yield() immediately after this function; ca_os_sleep_prepare only marks the task as 'ready to sleep' in the internal structures, but the task will continue executing until the scheduler is explicitly invoked via ca_os_yield().

Uso:

- This function is used when a task needs to wait for an external event (like a message or a signal) with a timeout. It is typically used in a pattern where a resource lock is held: the task checks if the resource is available, if not, it calls this function to prepare for sleep, releases the lock, and then calls ca_os_yield() to relinquish the CPU. This ensures that if the event occurs while the task is preparing to sleep, the scheduler handles the state transition correctly without losing the event.

Esempio:

```c
/* Example: Waiting for a message with a timeout */
void task_wait_for_data(void) {
    u32 ticks = 100; // 100 ticks timeout
    ca_spinlock_lock_irq(&my_lock);

    // Check if data is already available
    if (ca_queue_heap_is_empty(&my_queue)) {
        // Prepare to sleep if no data is present
        u32 status = ca_os_sleep_prepare(ticks);
        ca_spinlock_unlock_irq(&my_lock);

        // If preparation succeeded, yield control to the scheduler
        if (status == 0U) {
            ca_os_yield();
        }
    } else {
        ca_spinlock_unlock_irq(&my_lock);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** Why is it necessary to call ca_os_yield() immediately after ca_os_sleep_prepare()?
A: ca_os_sleep_prepare() only marks the task as 'ready to sleep' in the internal structures; it does not block the task itself. The task will continue executing until the scheduler is explicitly invoked via ca_os_yield() to relinquish the CPU.

Code example conf:1.00 **Q:** How do I implement a task sleep pattern using caOS types?
A: u32 ticks = 100; if (ca_os_sleep_prepare(ticks) == 0) { ca_os_yield(); }

Parameters conf:1.00 **Q:** What does the 'ticks' parameter represent in ca_os_sleep_prepare?
A: The 'ticks' parameter specifies the duration in OS ticks to sleep. A value of 0 is treated as an indefinite wait, while positive values represent the timeout duration.

Return conf:1.00 **Q:** What do the return values of ca_os_sleep_prepare indicate?
A: A return value of 0 indicates the task was successfully prepared for sleep. A return value of 1 indicates the function was called from an invalid context, such as when there is no active task.

ca_os_sleep_ticks

Header: caos/inc/scheduler.h:602
Source: caos/src/scheduler.c:2740:2745
GUARD: EN_SCHEDULER

```
u32 ca_os_sleep_ticks(u32 ticks)
```

Suspends the current task for a specified number of system ticks.

Parametri:

Nome	Tipo	Descrizione
ticks	u32	The number of OS ticks to sleep. A value of 0 will cause the task to yield immediately without sleeping.

Ritorna:
Returns 0 on success. Returns 1 if the current task context is invalid or if the system is not in a state where it can sleep.

Note:

- This function is blocking: it removes the current task from the ready list and places it in a waiting state until the tick count expires. It is NOT thread-safe to call from an ISR (Interrupt Service Routine) because it triggers a context switch via the scheduler. The most common beginner mistake is calling this function inside an ISR or a critical section (e.g., between ca_enter_critical and ca_exit_critical), which will cause the system to crash or hang because the scheduler cannot perform a context switch in those contexts. Ensure the OS scheduler is running before calling this.

Uso:

- Use this function when you need to implement periodic tasks or wait for a specific duration without busy-waiting, which would otherwise waste CPU cycles. It is typically used in the main loop of a task to regulate execution frequency, such as blinking an LED or polling a sensor. It is often paired with ca_os_tick_init() during system startup to ensure the timebase is active, and it is used in conjunction with other task management functions to ensure the task relinquishes control to other tasks of equal or higher priority.

Esempio:

```c
/* Example: Task that toggles a pin every 100 ticks */
void sensor_task(void *pvParameters) {
    while (1) {
        /* Toggle GPIO pin using ca_gpio_write */
        ca_gpio_write(1, 1);

        /* Sleep for 100 system ticks to create a delay */
        u32 status = ca_os_sleep_ticks(100);

        /* Check if sleep was successful */
        if (status != 0) {
            /* Handle error: log or reset task state */
        }

        ca_gpio_write(1, 0);
        /* Wait another 100 ticks before repeating */
        ca_os_sleep_ticks(100);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_os_sleep_ticks() in a caOS application?
A: You should call ca_os_sleep_ticks() when you need to pause the current task for a specific number of system ticks. This is useful for implementing delays, periodic tasks, or when you want to yield control to other tasks without busy-waiting. It's typically used in task code to manage timing and scheduling.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_os_sleep_ticks() in a task?
A: u32 task_example(void) { ca_os_sleep_ticks(100); return 0; }

Parameters conf:0.90 **Q:** What does the 'ticks' parameter in ca_os_sleep_ticks() control, and what are valid values?
A: The 'ticks' parameter specifies how many system ticks the current task should sleep. A value of 0 means no delay, while positive values indicate the number of ticks to wait. Negative values are invalid and will likely cause undefined behavior or failure.

Return conf:0.95 **Q:** What do the return values of ca_os_sleep_ticks() indicate, and what causes a failure?
A: The function returns 0 on success, meaning the sleep completed normally. It returns 1 on failure, which occurs if the calling task is invalid or the execution context is incorrect. Failure can lead to deadlock or unpredictable behavior.

`ca_os_sleep_until`

Header: `caos/inc/systick.h:289`
Source: `caos/src/systick.c:551:559`
GUARD: `EN_SYSTICK`

```
1 void ca_os_sleep_until(u32 *last_wake_ms, u32 period_ms)
```

Maintains a precise periodic execution rate by compensating for task execution time (anti-drift).

Parametri:

Nome	Tipo	Descrizione
`last_wake_ms`	`u32 *`	Pointer to a variable storing the timestamp of the previous wake-up. Must be initialized to the current tick count before the first loop iteration.
`period_ms`	`u32`	The desired interval in milliseconds between consecutive wake-ups. Must be greater than 0.

Ritorna:
void

Note:

- This function is blocking; it suspends the current task until the calculated next wake-up time is reached. It is NOT thread-safe if multiple tasks share the same 'last_wake_ms' variable, and it must NEVER be called from an ISR because it invokes blocking sleep logic. The most common beginner mistake is failing to initialize the 'last_wake_ms' variable before the loop, which causes the first iteration to calculate an incorrect sleep duration. If the task execution time exceeds the 'period_ms', the function will return immediately without sleeping, effectively allowing the task to 'catch up' to the schedule.

Uso:

- Use this function in periodic tasks where timing consistency is critical, such as sensor polling or control loops. It is typically used inside a 'while(1)' loop, paired with 'ca_os_tick_ms()' for initialization. Unlike a simple 'ca_os_sleep()', which adds a fixed delay after the task finishes, this function ensures the task runs at a fixed frequency regardless of how long the code inside the loop takes to execute.

Esempio:

```
u32 last_wake = ca_os_tick_ms();

while (1) {
    /* Ensure the loop runs exactly every 50ms */
    ca_os_sleep_until(&last_wake, 50);

    /* Perform periodic sensor reading or data processing */
    /* ... code logic ... */

    /* Optional: check for system health */
    if (ca_os_running() == 0) {
        break;
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_os_sleep_until() in a real-time embedded application?
A: You should use ca_os_sleep_until() when you need to implement a periodic task with precise timing, such as controlling a sensor reading or actuator update at regular intervals. It helps maintain consistent timing by correcting for drift over time.

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_os_sleep_until() with caOS types?
A: u32 last_wake = ca_os_tick_ms(); while (1) { ca_os_sleep_until(&last_wake, 100); // Task logic here }

Parameters conf:0.90 **Q:** What is the significance of the last_wake_ms parameter and what values are valid?
A: The last_wake_ms parameter is a pointer to a u32 value that stores the last wake-up time of the thread. It must be initialized with a valid timestamp (e.g., from ca_os_tick_ms()) before calling the function. NULL is not a valid value, and initializing it to 0 or a very low value may cause incorrect sleep durations.

Return conf:0.85 **Q:** Does ca_os_sleep_until() return any value, and can it fail?
A: The function does not return any value. It is designed not to fail under normal circumstances, but incorrect parameter usage (e.g., invalid last_wake_ms or period_ms) may lead to unexpected behavior.

`ca_os_stack_from_handle`

Header: `caos/inc/scheduler.h:561`
Source: `caos/src/scheduler.c:2915:2926`
GUARD: `EN_SCHEDULER`

```
u32 ca_os_stack_from_handle(ca_task_handle_t h)
```

Retrieves the base memory address of the stack allocated to a specific task.

Parametri:

Nome	Tipo	Descrizione
h	ca_task_handle_t	The handle of the target task. Must be a valid handle returned by a task creation function.

Ritorna:
Returns the u32 base address of the task stack. Returns 0 if the handle is invalid, the task is not found, or the task has been destroyed.

Note:

- This function is thread-safe as it internally disables interrupts to protect the task list lookup. It is NOT safe to call from an ISR because it uses critical section management (PRIMASK manipulation) which can interfere with interrupt nesting. The most common mistake is assuming the stack address is permanent; the memory area is reclaimed by the OS once a task is destroyed, so a handle to a destroyed task will return 0 or potentially point to a new, unrelated task's stack. Always validate the handle before use.

Uso:

- This function is primarily used for system diagnostics, stack overflow monitoring, and memory profiling. It is typically used in conjunction with ca_memory_get_task_usage to calculate the percentage of stack consumed by a specific thread. Developers use this during the debugging phase to ensure that the allocated stack size is sufficient for the task's worst-case execution path, often pairing it with ca_os_task_snapshot to capture the state of a task during a fault.

Esempio:

```
ca_task_handle_t h = ca_os_self_handle();
if (h != 0) {
    /* Retrieve the stack base address for memory monitoring */
    u32 stack_addr = ca_os_stack_from_handle(h);
    if (stack_addr != 0) {
        /* Check how much of the stack is currently used by this task */
        u32 usage = ca_memory_get_task_usage(h);
        ca_printf("Task stack base: 0x%08X, current usage: %u bytes\n",
    stack_addr, usage);
    } else {
        /* Handle error: task might have been destroyed or handle is invalid */
        ca_printf("Failed to retrieve stack address for handle\n");
    }
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_os_stack_from_handle and what is its primary purpose?
A: You should use this function for diagnostic purposes, such as monitoring stack usage or performing memory safety checks. It is typically used after a task is successfully created to track its memory footprint, often in conjunction with ca_memory_get_task_usage to verify if a task is approaching its stack limit.

Code example conf:0.90 **Q:** Provide a minimal C example showing how to retrieve a task's stack address.
A: ca_task_handle_t my_task = get_task_handle(); u32 stack_addr = ca_os_stack_from_handle(m if (stack_addr != 0) { /* Perform diagnostics */ }

Parameters conf:1.00 **Q:** What does the ca_task_handle_t parameter represent and what are the requirements for a valid input?
A: The parameter 'h' represents the handle of the task whose stack address you wish to retrieve. It must be a valid handle obtained from task creation or lookup. Passing an invalid or stale handle will result in an invalid return value.

Return conf:1.00 **Q:** What are the possible return values for ca_os_stack_from_handle and what do they signify?
A: The function returns the u32 memory address of the task's stack if the handle is valid. It returns 0 if the handle is invalid or if the task does not exist.

Error/Edge case conf:1.00 **Q:** Are there any specific safety warnings or common mistakes when calling this function?
A: This function is not thread-safe in isolation and must not be called from an ISR because it relies on internal task manager state. A common mistake is passing a stale handle after a task has been destroyed, which causes the function to return 0.

`ca_os_suspend_handle`

Header: `caos/inc/scheduler.h:578`
Source: `caos/src/scheduler.c:2857:2866`
GUARD: `EN_SCHEDULER`

```
bool ca_os_suspend_handle(ca_task_handle_t h)
```

Temporarily halts the execution of a specific task identified by its handle.

Parametri:

Nome	Tipo	Descrizione
`h`	`ca_task_handle_t`	The handle of the task to suspend. Must be a valid handle obtained via ca_os_self_handle or ca_os_handle_from_thid. Passing an invalid handle will cause the function to return false.

Ritorna:
bool: true if the task was successfully suspended, false if the handle was invalid or the task could not be suspended.

Note:

- This function is thread-safe and uses internal interrupt masking to ensure atomic state changes. It is NOT callable from an ISR because it may trigger a context switch (ca_os_yield) if the currently running task suspends itself. A common beginner mistake is calling this from an interrupt context, which can lead to system instability or hard faults. Always ensure the task handle is valid; if you suspend the currently running task, the scheduler will immediately switch to the next available task.

Uso:

- Use this function to pause tasks that are no longer needed or to implement a 'wait-for-event' pattern where a task suspends itself until another task resumes it. It is frequently used in initialization sequences where a task waits for a configuration flag to be set before proceeding. It is typically paired with ca_os_resume_handle to re-enable the task. In complex systems, it is used to manage power consumption or CPU load by disabling non-essential background threads during high-priority processing windows.

Esempio:

```
/* Example: A task suspends itself if a specific condition is not met */
ca_task_handle_t self = ca_os_self_handle();

// Check if the task should remain active
if (/* condition to check if task should be idle */) {
    // Suspend self until another task calls ca_os_resume_handle
    if (ca_os_suspend_handle(self)) {
```

```
        // The task is now suspended and will not execute until resumed
    } else {
        // Handle error: could not suspend
    }
}

// Continue execution once resumed
ca_printf("Task resumed and continuing execution\n");
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_os_suspend_handle in my application?
A: You should use this function to pause background tasks, such as data loggers or non-critical communication handlers, to free up CPU cycles for high-priority operations. It is typically paired with ca_os_resume_handle to restart the task once the critical system load returns to normal.

Code example conf:0.95 **Q:** How do I implement a task suspension using caOS types?
A: You must first obtain a valid handle, then call the function: ca_task_handle_t h = ca_os_handle_from_thid(10); if (ca_os_suspend_handle(h)) { /* Task successfully suspended */ }

Parameters conf:1.00 **Q:** What are the requirements for the ca_task_handle_t parameter?
A: The parameter 'h' must be a valid handle obtained via ca_os_self_handle or ca_os_handle_from_th
Passing an invalid or expired handle results in no action. You must ensure the handle is still valid before calling, as using a stale handle after a task has been destroyed is a common error.

Return conf:1.00 **Q:** What do the return values of ca_os_suspend_handle indicate?
A: The function returns true if the task was successfully suspended. It returns false if the provided handle was invalid or if the task could not be suspended for other reasons.

`ca_os_task_handle`

Header: `caos/inc/scheduler.h:565`
Source: `caos/src/scheduler.c:2810:2817`
GUARD: `EN_SCHEDULER`

```
ca_task_handle_t ca_os_task_handle(const ca_thread_context_t *ctx)
```

Retrieves a stable, unique handle for a task based on its thread context.

Parametri:

Nome	Tipo	Descrizione
`ctx`	`const ca_thread_context_t *`	Pointer to the thread context. Must be a valid, initialized pointer. Passing NULL or an uninitialized pointer will return CA_TASK_HANDLE_NONE.

Ritorna:
Returns a ca_task_handle_t (a combination of slot index and generation). Returns CA_TASK_HANDLE_NO if the context is NULL, the task is not registered, or the system task array is uninitialized.

Note:

- This function is non-blocking and thread-safe, making it safe to call from ISRs. It does not perform complex operations, but it relies on the internal g_task_array being initialized. A common mistake is assuming a handle remains valid forever; if a task is removed from the scheduler, its slot may be reused by a new task, and the generation bits in the handle

will change. Always check the return value against CA_TASK_HANDLE_NONE before using the handle in other API calls.

Uso:

- Use this function whenever you need to perform operations on a task that you did not create yourself, or when you need to store a reference to a task for later signaling or control. It is frequently used in synchronization primitives (like checking if a task owns a mutex) or in ISRs to notify a specific waiting task. Typical sequences involve calling this after a task is successfully added to the scheduler and using the resulting handle with functions like ca_notify_send_handle_isr or ca_os_suspend_handle.

Esempio:

```c
/* Example: Using a task handle to notify a waiting task from an ISR */
ca_task_handle_t target_h;
const ca_thread_context_t *waiter_ctx = /* pointer to a known waiting task */;

/* Retrieve the handle for the waiting task */
target_h = ca_os_task_handle(waiter_ctx);

if (target_h != CA_TASK_HANDLE_NONE) {
    /* Notify the task that an event has occurred */
    (void)ca_notify_send_handle_isr(target_h, 1U, CA_NOTIFY_ACTION_INCREMENT);
} else {
    /* Handle the case where the task is no longer in the scheduler */
    ca_printf("Target task is not active or invalid.\n");
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to call ca_os_task_handle in my application lifecycle?
A: You should call this function immediately after the task has been successfully added to the scheduler. It is typically used to obtain a reference for future lifecycle management operations, such as suspending, resuming, or modifying the priority of the task using its handle.

Code example conf:0.90 **Q:** How do I retrieve a task handle in C using caOS types?
A: After adding your task to the scheduler, you can retrieve the handle as follows: ca_thread_context_t my_ctx; ca_task_handle_t handle = ca_os_task_handle(&my_ctx); if (handle != CA_TASK_HANDLE_NONE) { /* handle is valid */ }

Parameters conf:1.00 **Q:** What are the requirements for the 'ctx' parameter in ca_os_task_handle?
A: The 'ctx' parameter must be a pointer to a valid, initialized ca_thread_context_t structure. Passing a NULL pointer or an uninitialized pointer will result in undefined behavior.

Return conf:1.00 **Q:** What does it mean if ca_os_task_handle returns CA_TASK_HANDLE_NONE?
A: A return value of CA_TASK_HANDLE_NONE indicates that the provided context is invalid or that the task has not yet been registered in the scheduler. The most common cause is calling the function before the task initialization sequence is complete.

`ca_os_task_slot`

Header: `caos/inc/scheduler.h:588`
Source: `caos/src/scheduler.c:2903:2905`
GUARD: `EN_SCHEDULER`

```c
u32 ca_os_task_slot(const ca_thread_context_t *ctx)
```

Retrieves the unique, fixed index (slot) associated with a specific task context.

Parametri:

Nome	Tipo	Descrizione
`ctx`	`const ca_thread_context_t *`	Pointer to the task's thread context. Must be a valid, non-NULL pointer obtained from the scheduler.

Ritorna:
Returns a u32 representing the task's fixed slot index. Returns (u32)-1 if the context is NULL, the task array is uninitialized, or the task is not found.

Note:

- This function is thread-safe and provides a stable identifier for a task. Unlike a Task ID (thid), which can change if tasks are created or destroyed, the 'slot' remains constant for the entire lifetime of the task. This makes it ideal for indexing into static arrays or metadata tables associated with specific tasks. It is safe to call from most contexts, but ensure the pointer passed is not dangling (i.e., the task has not been destroyed). A common mistake is confusing the 'slot' with the 'thid'; always use the slot when indexing into fixed-size arrays to avoid memory corruption or logic errors when the task list shifts.

Uso:

- Use this function when you need to associate persistent data with a task, such as tracking memory allocations or event waiter metadata. It is typically used in conjunction with ca_scheduler_get_current_context() to identify the caller, or when iterating through a list of tasks to update their specific status. By using the slot index, you ensure that your data structures remain aligned with the task's identity even if other tasks in the system are terminated or re-indexed.

Esempio:

```c
/* Example: Tracking memory ownership per task */
#include "caos.h"

void track_task_memory(void) {
    const ca_thread_context_t *ctx = ca_scheduler_get_current_context();

    if (ctx != NULL) {
        /* Get the fixed slot index for the current task */
        u32 slot = ca_os_task_slot(ctx);

        /* Use the slot to index into a custom tracking table */
        /* Assuming 'my_task_stats' is a global array of size MAX_TASKS */
        if (slot != (u32)-1) {
            /* Perform logic using the stable slot index */
            /* e.g., update a custom stats structure */
        }
    } else {
        /* Handle case where no task context is active (e.g., main/init) */
        /* Use a default slot index for system-level operations */
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** Why should I use ca_os_task_slot instead of a Task ID (thid) when managing task-specific metadata?
A: You should use ca_os_task_slot because it provides a stable, fixed index that remains constant for the entire lifetime of the task. Unlike a Task ID, which can change if tasks

are created or destroyed, the slot is ideal for indexing into static arrays or metadata tables without risking memory corruption or logic errors when the task list shifts.

Code example conf:0.90 **Q:** How do I retrieve the slot index for the currently running task in caOS?

A: You can retrieve the slot by first obtaining the current context and then passing it to ca_os_task_slot:

const ca_thread_context_t *ctx = ca_scheduler_get_current_context(); u32 slot = ca_os_task_slot(ctx);

Parameters conf:1.00 **Q:** What are the requirements for the 'ctx' parameter in ca_os_task_slot?

A: The 'ctx' parameter must be a valid, non-NULL pointer to a ca_thread_context_t obtained from the scheduler. It is critical that the pointer is not dangling, meaning the task must not have been destroyed prior to the function call.

Return conf:1.00 **Q:** What does the return value of ca_os_task_slot represent, and what happens if the context is invalid?

A: The function returns a u32 representing the task's fixed slot index. If the provided context is invalid or the task is not registered, the function returns an implementation-defined value, which is typically 0xFFFF.

`ca_os_task_snapshot`

Header: `caos/inc/scheduler.h:586`
Source: `caos/src/scheduler.c:1745:1762`
GUARD: `EN_SCHEDULER`

```
1 bool ca_os_task_snapshot(u32 thid, ca_thread_context_t *out, u32 *slot)
```

Creates a consistent, thread-safe snapshot of a task's context for inspection or logging.

Parametri:

Nome	Tipo	Descrizione
`thid`	`u32`	The unique Task ID of the thread to snapshot.
`out`	`ca_thread_context_t *`	Pointer to a buffer where the task context will be copied.
`slot`	`u32 *`	Pointer to store the internal slot index of the task, used for validation. Pass NULL if not needed.

Ritorna:
Returns true if the snapshot was successfully captured and the task is valid. Returns false if the task ID is invalid, the task has been destroyed, or the slot was reused during the copy process.

Note:

- This function is thread-safe and internally masks interrupts to ensure the snapshot is consistent. It is not intended to be called from an ISR because it performs complex task list lookups. The most common mistake is assuming the snapshot remains valid indefinitely; while the memory is safe to read, the data inside (like stack pointers or state) becomes 'stale' as soon as the target task resumes execution. Always check the return value before accessing the 'out' buffer, as a false return indicates the task no longer exists or the data is unreliable.

Uso:

- This function is used by diagnostic tools, system monitors, or logging tasks that need to inspect the state of another thread without causing race conditions. It is typically used

when iterating through system tasks to print stack usage or thread status. You would call this after identifying a target task ID, then use the returned 'out' structure to safely format output for the console or a log file without blocking the target task for an extended period.

Esempio:

```c
void log_task_status(u32 thid) {
    ca_thread_context_t ctx;
    u32 slot;

    /* Attempt to capture a consistent snapshot of the task */
    if (ca_os_task_snapshot(thid, &ctx, &slot)) {
        /* Successfully captured, now safe to read from 'ctx' */
        /* Use ca_printf to report the task name and its slot index */
        ca_printf("Task %u (slot %u) is active\n", thid, slot);

        /* Check if the task stack is still within safe bounds */
        if (!ca_memory_check_canary(&ctx)) {
            ca_printf("Warning: Task %u has a corrupted stack!\n", thid);
        }
    } else {
        /* Task might have finished or been removed during the snapshot attempt
    */
        ca_printf("Failed to snapshot task %u: it may have terminated.\n", thid)
    ;
    }
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_os_task_snapshot in my caOS application?
A: You should use this function when you need to inspect or log the state of another thread, such as in diagnostic tools, system monitors, or logging tasks. It is useful for gathering data like stack pointers or thread status without causing race conditions.

Code example conf:1.00 **Q:** Provide a minimal C example of how to capture a task snapshot.
A: u32 thid = 1; ca_thread_context_t ctx; u32 slot; if (ca_os_task_snapshot(thid, &ctx, &slot)) { /* Use ctx data here */ }

Parameters conf:1.00 **Q:** What is the purpose of the 'slot' parameter in ca_os_task_snapshot?
A: The 'slot' parameter is a pointer to a u32 variable where the function stores the internal slot index of the task. This index is used for validation purposes to ensure the task context is correctly identified.

Return conf:1.00 **Q:** What does a false return value from ca_os_task_snapshot signify?
A: A false return indicates that the snapshot failed. This happens if the provided Task ID is invalid, the task has already been destroyed, or the internal slot is no longer in use. You should never access the 'out' buffer if the function returns false.

Error/Edge case conf:1.00 **Q:** What is the most common mistake developers make when using the data returned by ca_os_task_snapshot?
A: The most common mistake is assuming the snapshot remains valid indefinitely. While the memory in the 'out' buffer is safe to read, the actual data (such as stack pointers or thread state) becomes 'stale' as soon as the target task resumes execution.

Behavior conf:1.00 **Q:** Is ca_os_task_snapshot safe to call from an Interrupt Service Routine (ISR)?
A: No, it is not intended to be called from an ISR. The function performs complex internal task list lookups that are not suitable for ISR execution, although it is thread-safe and handles interrupt masking internally for standard task-level calls.

ca_os_thid_from_handle

Header: caos/inc/scheduler.h:572
Source: caos/src/scheduler.c:2837:2844
GUARD: EN_SCHEDULER

```
u32 ca_os_thid_from_handle(ca_task_handle_t h)
```

Converts a task handle into a numeric task ID (thid) for internal OS operations.

Parametri:

Nome	Tipo	Descrizione
h	ca_task_handle_t	The opaque handle representing a task. Must be a valid handle previously obtained from the OS.

Ritorna:
Returns the numeric task ID (u32) if the handle is valid and the task exists. Returns (u32)-1 if the handle is invalid, the task has been destroyed, or the slot is empty.

Note:

- This function is non-blocking and thread-safe. It is safe to call from an ISR, provided the handle remains valid during the call. The primary purpose is to translate an external-facing 'handle' into the internal 'thid' used by the scheduler and task manager. A common mistake is assuming a handle remains valid indefinitely; if a task is destroyed, its handle becomes stale. Always check the return value against (u32)-1 before using the resulting ID in other OS functions. Because the OS may reuse task slots, the handle includes a generation counter to detect if the original task has been replaced by a new one.

Uso:

- Use this function when you have a task handle (e.g., stored in a message buffer or a communication structure) and need to perform an operation on that task, such as sending a notification or checking its status. It is typically used in a sequence: first, resolve the handle to a thid using this function; second, verify the result is not (u32)-1; third, pass the valid thid to functions like ca_notify_send_handle or task management routines. This pattern ensures that you do not attempt to operate on a task that has already terminated or been removed from the system, preventing memory corruption or invalid scheduler operations.

Esempio:

```
/* Example: Safely sending a notification to a task using its handle */
void notify_task_by_handle(ca_task_handle_t h, u32 value) {
    /* Resolve the handle to an internal task ID to verify existence */
    u32 thid = ca_os_thid_from_handle(h);

    /* Check if the task still exists and the handle is current */
    if (thid != (u32)-1) {
        /* Send a notification to the task using the handle */
        /* The OS will internally re-verify the handle validity */
        bool success = ca_notify_send_handle(h, value, 0);
        if (!success) {
            ca_printf("Failed to send notification to task\n");
        }
    } else {
        /* Handle the case where the task is no longer active or handle is stale
     */
        ca_printf("Task handle is invalid or task has exited\n");
    }
```

```
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_os_thid_from_handle in my caOS application?
A: You should use this function when you have an opaque task handle stored in a structure and need to perform an operation on that task, such as sending a notification or checking its status. It acts as a bridge to translate the external handle into an internal thid required by the scheduler and task manager.

Code example conf:1.00 **Q:** Provide a minimal C example showing how to safely resolve a task handle to a thid.
A: ca_task_handle_t my_handle = get_stored_handle(); u32 thid = ca_os_thid_from_handle(my_h
if (thid != (u32)-1) { /* Proceed with task operations using thid */ }

Parameters conf:1.00 **Q:** What are the requirements for the ca_task_handle_t parameter?
A: The parameter h must be a valid opaque handle previously obtained from the OS. Because the OS uses a generation counter within the handle to detect if a task slot has been reused, you must ensure the handle is not stale before passing it to the function.

Return conf:1.00 **Q:** What does it mean if ca_os_thid_from_handle returns (u32)-1?
A: A return value of (u32)-1 indicates that the operation failed. This occurs if the provided handle is invalid, the task has been destroyed, or the task slot is currently empty. You must always check for this value before using the result in other OS functions to prevent invalid scheduler operations.

`ca_os_thread_reset_watchdog`

Header: `caos/inc/scheduler.h:717`
Source: `caos/src/scheduler.c:2954:2958`
GUARD: `EN_SCHEDULER`

```
void ca_os_thread_reset_watchdog()
```

Resets the internal watchdog timer for the currently executing thread to prevent it from being flagged as hung.

Ritorna:
This function returns nothing (void). It performs no validation on the success of the reset operation.

Note:

- This function is non-blocking and thread-safe as it specifically targets the context of the currently running task. It is strictly intended to be called from within a thread's main loop to signal that the thread is still alive and processing correctly. If you fail to call this function within the time limit previously configured via ca_os_thread_set_watchdog, the scheduler will consider the thread unresponsive and may trigger a system fault or a task-specific reset. Do not call this function from an ISR; it is designed exclusively for task-level execution where the internal task context is valid and accessible.

Uso:

- This function is essential for long-running tasks that perform repetitive work or wait on external events. You typically pair it with ca_os_thread_set_watchdog during the thread initialization phase to define the maximum allowed execution time before the system considers the task hung. Inside the main loop of your thread, you must call this function periodically to 'kick' the watchdog. If your thread enters a blocking state, such

as ca_os_sleep or ca_os_wait, for a duration longer than the watchdog limit, you must ensure the watchdog is reset before or after the block to avoid false positive timeouts.

Esempio:

```c
void my_worker_thread(void *arg) {
    /* Set a watchdog limit of 5000 ticks to monitor task health */
    ca_os_thread_set_watchdog(5000);

    while (1) {
        /* Perform heavy data processing here */

        /* Reset the watchdog to indicate the thread is still alive */
        ca_os_thread_reset_watchdog();

        /* Sleep for a short duration to yield control */
        ca_os_sleep(10);
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_os_thread_reset_watchdog() and why is it important?
A: You should call ca_os_thread_reset_watchdog() when you want to prevent a thread from being terminated due to a watchdog timeout. This function resets the total execution time counter of the currently running thread, which is used by the watchdog mechanism to determine if a thread has exceeded its allowed runtime. It's particularly useful for long-running tasks that may otherwise be killed by the watchdog if they take too long to complete.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_os_thread_reset_watchdog() with caOS types?
A: void example_task(void) { while (1) { // Do some work ca_os_thread_reset_watchdog(); ca_os_task_delay(100); } }

Return conf:0.90 **Q:** What does ca_os_thread_reset_watchdog() return and what causes it to fail?
A: The function does not return any value. It is a void function and cannot fail. However, it may not perform any action if called outside of a valid thread context, because it relies on the global variable g_current_task which will be NULL if not called from within an active thread. In such cases, the function will simply do nothing.

`ca_os_thread_set_watchdog`

Header: `caos/inc/scheduler.h:710`
Source: `caos/src/scheduler.c:2948:2952`
GUARD: `EN_SCHEDULER`

```c
void ca_os_thread_set_watchdog(u32 clocks_cycles)
```

Sets the maximum execution time (watchdog timeout) for the current task in clock cycles.

Parametri:

Nome	Tipo	Descrizione
`clocks_cycles`	`u32`	The number of clock cycles allowed before the task is considered hung. A value of 0 typically disables the watchdog for this task.

Ritorna:
void

Note:

- This function is non-blocking and thread-safe as it only modifies the internal state of the currently running task. It is intended to be called from within the task context itself; calling it from an ISR is dangerous because the OS scheduler may not correctly identify the task context. The most common beginner mistake is setting a timeout value that is too short, causing the task to be prematurely flagged as failed by the scheduler. Ensure the system clock frequency is understood, as this value is in raw cycles, not milliseconds.

Uso:

- This function is used to implement task-level supervision, ensuring that a specific thread does not enter an infinite loop or hang indefinitely. It is typically called during task initialization or before entering a critical processing loop. It must be paired with 'ca_os_thread_reset_watchdog inside the main loop of the task to 'kick' the watchdog and prevent the scheduler from triggering a fault. If the task fails to call the reset function within the specified 'clocks_cycles', the OS will detect the timeout and handle the error.

Esempio:

```c
void my_task_function(void *arg) {
    /* Set a watchdog limit of 1,000,000 clock cycles */
    ca_os_thread_set_watchdog(1000000);

    while (1) {
        /* Perform critical work */
        /* Perform heavy computation here */

        /* Reset the watchdog to prevent task timeout */
        ca_os_thread_reset_watchdog();

        /* Sleep to yield CPU to other tasks */
        ca_os_sleep(10);
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_os_thread_set_watchdog and what is its purpose in a real-time system?
A: You should call ca_os_thread_set_watchdog when you want to ensure that a thread does not hang or get stuck for too long. It sets a runtime watchdog that will reset the thread if it does not reset the watchdog within a specified number of clock cycles. This is useful in real-time systems where thread starvation or deadlock could lead to system failure. The function must be called from an active thread and not from an ISR.

Code example conf:0.85 **Q:** Can you provide a minimal C example showing how to use ca_os_thread_set_watch with caOS types?
A: u32 watchdog_cycles = 1000000; // 1 million clock cycles ca_os_thread_set_watchdog(watchdog_c // Enable watchdog // Do some work ca_os_thread_reset_watchdog(); // Reset watchdog to prevent reset

Parameters conf:0.90 **Q:** What does the clock_cycles parameter in ca_os_thread_set_watchdog represent, and what are valid values?
A: The clock_cycles parameter specifies the number of clock cycles after which the watchdog will reset the thread if not reset. A value of 0 disables the watchdog. Valid values are any positive u32 integer representing the desired number of clock cycles before timeout. The function does not validate if the watchdog is already active; it simply overwrites the previous value.

ca_os_tick

Header: caos/inc/systick.h:111
Source: caos/src/systick.c:240:243
GUARD: EN_SYSTICK

```
u32 ca_os_tick(void)
```

Returns the current system tick counter value as an unsigned 32-bit integer.

Ritorna:
Returns the current value of the internal system tick counter. This value increments periodically based on the OS tick configuration. It never returns NULL or an error code as it is a direct read of a global variable.

Note:

- This function is non-blocking and executes in constant time. It is thread-safe because reading a 32-bit variable is an atomic operation on the ARM Cortex-M4 architecture, meaning you do not need to disable interrupts to call it. It is safe to call from both tasks and Interrupt Service Routines (ISRs). The most common beginner mistake is assuming the tick counter will never overflow; since it is a 32-bit value, it will eventually wrap around to zero, so always use subtraction (e.g., 'now - start') rather than direct comparison when calculating time intervals to ensure the logic remains correct across the wrap-around point.

Uso:

- This function is the foundation for all time-based logic in caOS, such as calculating timeouts, measuring execution duration, or timestamping network packets. It is typically used in a sequence where you capture a 'start' tick, perform an operation, and then compare the current tick against the start to determine if a deadline has passed. It is frequently paired with ca_os_tick_ms or ca_os_delay_ms to convert these raw ticks into human-readable time units for application logic.

Esempio:

```
/* Example: Measuring how long a network operation takes */
#include "caos.h"

void monitor_network_latency(void) {
    u32 start_time = ca_os_tick();

    /* Perform a network operation, e.g., sending a packet */
    /* ca_net_send(buffer_index); */

    u32 end_time = ca_os_tick();
    u32 duration = end_time - start_time;

    /* Log the duration in ticks */
    ca_printf("Operation took %lu ticks\n", duration);
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_os_tick() in my embedded application, and why is it useful for timeout management?
A: You should call ca_os_tick() to get the current system tick counter, which is useful for calculating elapsed time and managing timeouts. It's commonly used in network services

like DHCP to track time intervals. The function is non-blocking and thread-safe, making it suitable for both task context and interrupt service routines (ISR).

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_os_tick() with caOS types?
A: u32 start_tick = ca_os_tick(); /* Some processing */ u32 elapsed = ca_os_tick() - start_tick; /* Use elapsed for timeout logic */

Return conf:0.95 **Q:** What does the return value of ca_os_tick() represent, and can it ever fail or return an error?
A: The return value represents the current system tick counter as a 32-bit unsigned integer (u32). It is always a positive and monotonically increasing value. The function cannot fail; it always returns a valid tick count. The counter will overflow after approximately 49 days if the tick interval is 1 millisecond, but this is a normal behavior, not an error condition.

`ca_os_tick_64`

Header: `caos/inc/systick.h:127`
Source: `caos/src/systick.c:248:255`
GUARD: `EN_SYSTICK`

```
u64 ca_os_tick_64(void)
```

Returns the current system tick counter as a 64-bit unsigned integer.

Ritorna:
A 64-bit unsigned integer (u64) representing the total number of system ticks elapsed since the OS started. This value never returns NULL or an error code as it is a direct read of a global counter.

Note:

- This function is non-blocking and extremely fast, making it safe to call from almost any context, including tasks and ISRs. On 32-bit architectures like the Cortex-M4, reading a 64-bit variable is not atomic, meaning the CPU performs two separate 32-bit memory reads. While this could theoretically result in an inconsistent value if an interrupt updates the counter exactly between the two reads, the error is negligible (at most 1ms) and acceptable for timing calculations. Do not use this for high-precision timestamping where microsecond accuracy is required; it is intended for general-purpose timeout management and duration tracking.

Uso:

- This function is primarily used to implement non-blocking timeouts in communication protocols or state machines. A typical sequence involves capturing the start time using ca_os_tick_64() before entering a loop, then periodically comparing the current tick value against the start time to calculate elapsed duration. It is frequently paired with ca_os_sleep() or ca_os_wait() to yield CPU time while waiting for an event, ensuring the task does not hang indefinitely if the expected signal never arrives.

Esempio:

```
u64 start_time = ca_os_tick_64();
u32 timeout_ms = 1000;

/* Loop until a condition is met or timeout expires */
while (/* condition not met */) {
    u64 current_time = ca_os_tick_64();
```

```
    if ((u32)(current_time - start_time) >= timeout_ms) {
        /* Handle timeout error */
        break;
    }
    /* Sleep briefly to allow other tasks to run */
    ca_os_sleep(10);
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_os_tick_64() in my embedded application, and why is it preferred for timeout management?
A: You should call ca_os_tick_64() when you need precise time measurement or to manage timeouts in your embedded application. It is particularly useful in scenarios where you require a 64-bit tick count for accurate timing, such as in message queues, stream handling, or system tick management. The function is non-blocking and can be safely called from ISRs, making it suitable for real-time systems where timing precision is critical.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_os_tick_64() with caOS types?
A: u64 start = ca_os_tick_64(); u64 end = ca_os_tick_64(); u64 elapsed = end - start; // Use elapsed for timing calculations or timeout checks.

Return conf:0.85 **Q:** What does ca_os_tick_64() return, and under what conditions might it fail?
A: The function returns a u64 value representing the total number of system ticks. It does not have any failure conditions as described in the documentation; it will return 0 only in case of an unspecified internal error, though such cases are not detailed. The function is not thread-safe in a strict sense, but since it's a simple read operation, it's not problematic in practice. Note that reading a u64 on Cortex-M4 is not atomic, so an interrupt during the read could cause a partial value (1 tick error).

`ca_os_tick_64_ms`

Header: `caos/inc/systick.h:160`
Source: `caos/src/systick.c:445:447`
GUARD: `EN_SYSTICK`

```
u64 ca_os_tick_64_ms(void)
```

Returns the system uptime in milliseconds as a 64-bit unsigned integer.

Ritorna:
A 64-bit unsigned integer representing the total milliseconds elapsed since the system started. This value will not overflow for thousands of years, making it safe for long-term uptime tracking.

Note:

- This function is non-blocking and thread-safe, as it simply reads a global counter maintained by the system tick interrupt. It is safe to call from both tasks and ISRs (Interrupt Service Routines). The primary precondition is that the OS tick system must be initialized via ca_os_tick_init. A common mistake for juniors is to assume this function provides microsecond precision; it is strictly millisecond-based, and using it for high-frequency timing will result in significant quantization errors.

Uso:

- This function is typically used for timestamping events, calculating task execution duration, or implementing timeouts in non-blocking state machines. It is often paired with a captured 'start' snapshot to calculate elapsed time by subtracting the start value from the current value. It is frequently used in logging, CLI 'uptime' commands, and diagnostic routines to report exactly when a system error or task failure occurred.

Esempio:

```c
/* Example: Logging a task error with a timestamp */
void log_task_failure(u32 task_index, const char *task_name) {
    char buf[64];
    /* Get current uptime in ms */
    u64 timestamp = ca_os_tick_64_ms();

    /* Format the error message with the timestamp */
    ca_snprintf(buf, sizeof(buf), "Task[%u] %s failed at %llu ms", (unsigned)
    task_index, task_name, (unsigned long long)timestamp);

    /* Output the log to the system console using the caOS stdout service */
    ca_stdout_printf_n(buf, sizeof(buf));
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_os_tick_64_ms() in my embedded application, and why is it preferred over other timing methods?
A: You should use ca_os_tick_64_ms() to measure elapsed time or manage delays in your embedded application. It is particularly useful in RTOS environments where precise timing is required. The function is non-blocking and thread-safe, making it suitable for use in both task contexts and interrupt service routines (ISRs). It provides a reliable way to track time since system startup, which is essential for scheduling, timeouts, and timing-sensitive operations. Since it's updated by the SysTick timer, it integrates well with the caOS scheduler and other timing-dependent components.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_os_tick_64_ms() with caOS types?
A: u64 start_time = ca_os_tick_64_ms(); // Perform some operation u64 end_time = ca_os_tick_64_ms(); u64 elapsed_ms = end_time - start_time; // Use elapsed_ms for timing or logging purposes

Return conf:0.85 **Q:** What does ca_os_tick_64_ms() return when it fails, and what conditions might cause such a failure?
A: The function returns 0 in case of error or if the system is not initialized. This typically occurs when the SysTick timer has not been properly configured or initialized by the caOS kernel. If the system is running but the systick_counter_64 variable is not being updated correctly, the function may also return an incorrect or stale value. It is important to ensure that the RTOS is fully initialized before calling this function to avoid returning 0 due to uninitialized state.

`ca_os_tick_calibrate`

Header: `caos/inc/systick.h:90`
Source: `caos/src/systick.c:273:291`
GUARD: `EN_SYSTICK`

```c
void ca_os_tick_calibrate(void)
```

Calibrates the system tick timer to account for hardware-induced cycle delays during restarts.

Ritorna:
This function returns void and does not provide a status code.

Note:

- This function is blocking because it disables all interrupts to ensure measurement accuracy during the calibration loop. It is NOT thread-safe and must never be called from an ISR, as it manipulates the PRIMASK register and relies on precise timing. It must be called after the system clock and SysTick are initialized but before the scheduler starts, as it assumes the system is in a single-threaded, pre-scheduler state. A common mistake is calling this function after the scheduler has started, which will cause a system hang or unpredictable timing behavior due to the forced interrupt disabling.

Uso:

- This function is used during the early system boot sequence to ensure that time-keeping remains accurate despite hardware-specific cycle overheads. It is typically paired with ca_os_tick_init to set up the timer, followed by ca_os_tick_calibrate to measure the drift, and finally ca_scheduler_init to start the multitasking environment. By running this before the scheduler, you ensure that all subsequent time-based operations (like delays or timeouts) are based on a calibrated time base.

Esempio:

```c
/* Initialize system memory and tick before calibration */
ca_memory_init(&_heap_start, &_heap_end);
ca_os_tick_init();

/* Perform calibration while interrupts are still manageable */
ca_os_tick_calibrate();

/* Now safe to start the scheduler */
if (ca_scheduler_init()) {
    ca_os_run();
} else {
    /* Handle scheduler initialization failure */
    ca_printf("Scheduler failed to start\n");
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I invoke ca_os_tick_calibrate in my boot sequence?
A: It must be called after the system clock and SysTick are initialized, but before the scheduler starts. It is used to ensure time-keeping accuracy by accounting for hardware-induced cycle delays during restarts.

Code example conf:0.90 **Q:** Provide a minimal C example showing the correct initialization sequence using caOS types.
A: void system_boot(void) { ca_os_tick_init(); ca_os_tick_calibrate(); ca_scheduler_init(); }

Error/Edge case conf:1.00 **Q:** What happens if I call ca_os_tick_calibrate after the scheduler has already started?
A: Calling this function after the scheduler has started is a common mistake that will cause a system hang or unpredictable timing behavior, as the function forces interrupts to be disabled via the PRIMASK register.

Behavior conf:1.00 **Q:** Is ca_os_tick_calibrate safe to call from an interrupt service routine or a multi-threaded context?
A: No, it is not thread-safe and must never be called from an ISR. It is a blocking function that disables all interrupts to ensure measurement accuracy, and it assumes the system is in a single-threaded, pre-scheduler state.

ca_os_tick_compensate_us

Header: caos/inc/systick.h:97
Source: caos/src/systick.c:300:313
GUARD: EN_SYSTICK

```
void ca_os_tick_compensate_us(u64 us)
```

Updates the OS tick counter to account for time elapsed while the CPU was in a low-power Stop mode.

Parametri:

Nome	Tipo	Descrizione
us	u64	The duration in microseconds that the system spent in a low-power state. Must be a positive value representing the measured sleep interval.

Ritorna:
None (void function).

Note:

- This function is non-blocking and is designed to be called exclusively from low-level power management code when the system resumes from a Stop state. It is not thread-safe and must be called with interrupts masked (disabled) because it directly modifies global tick counters that are accessed by the scheduler. A common mistake is calling this function during normal operation or from an ISR that is not related to power state transitions, which will cause the system time to drift significantly and potentially corrupt the scheduler's internal timing logic.

Uso:

- This function is used in the power management sequence to synchronize the OS clock after the hardware has been suspended. Typically, the system records a timestamp using an RTC or a low-power timer before entering Stop mode, and upon waking, it calculates the elapsed time (dt). This function is then called to 'inject' that missing time into the OS tick counters, ensuring that tasks waiting on delays or timeouts remain synchronized with real-world time. It is typically paired with ca_pwr_enter and RTC-based time measurement functions.

Esempio:

```
/* Example: Compensating for time spent in Stop mode */
u64 t_wake;
/* Assume pwr_stamp_get() is a hypothetical function to get current RTC time */
/* We calculate the duration spent in sleep */
u64 sleep_duration_us = (t_wake - t_sleep_start) * 1000U;

/* Ensure interrupts are disabled before modifying tick counters */
ca_enter_critical();

/* Update the OS tick count to reflect the time passed while the CPU was stopped
    */
ca_os_tick_compensate_us(sleep_duration_us);

ca_exit_critical();
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_os_tick_compensate_us and what is its purpose?
A: You should call this function immediately upon waking the system from a low-power Stop mode. Its purpose is to synchronize the OS tick counter by 'injecting' the time elapsed during the sleep interval, ensuring that task delays and timeouts remain accurate relative to real-world time.

Code example conf:0.90 **Q:** How do I implement a basic call to ca_os_tick_compensate_us in my power management routine?
A: You must disable interrupts before calling the function to ensure thread safety, as it modifies global tick counters. Example: { u64 elapsed_us = get_rtc_sleep_duration(); ___disable_irq(); ca_os_tick_compensate_us(elapsed_us); ___enable_irq(); }

Parameters conf:1.00 **Q:** What are the constraints on the 'us' parameter for ca_os_tick_compensate_us?
A: The 'us' parameter is a u64 type representing the duration in microseconds spent in the low-power state. It must be a positive value representing the measured sleep interval.

Error/Edge case conf:1.00 **Q:** What are the risks of calling ca_os_tick_compensate_us during normal operation?
A: Calling this function during normal operation or from an unrelated ISR is a common mistake that will cause significant system time drift and potentially corrupt the scheduler's internal timing logic, as it is designed exclusively for post-Stop mode synchronization.

`ca_os_tick_disable`

Header: `caos/inc/systick.h:91`
Source: `caos/src/systick.c:220:229`
GUARD: `EN_SYSTICK`

```
void ca_os_tick_disable(void)
```

Attempts to disable the system tick timer, though currently restricted to prevent system instability.

Ritorna:
This function returns nothing (void).

Note:

- This function is currently a 'no-op' that triggers a system safeguard if called. Disabling the SysTick timer is extremely dangerous in an RTOS environment because the scheduler relies on the SysTick interrupt to perform context switching and maintain time-based delays. If the tick is disabled, the OS will freeze, tasks will stop executing, and time-dependent functions like ca_os_delay or ca_os_timeout will hang indefinitely. Never attempt to bypass the safeguard, as it is implemented to protect the integrity of the kernel.

Uso:

- This function is intended for low-level power management scenarios where the system might enter a deep sleep state that requires stopping the SysTick. In a typical lifecycle, it would be paired with ca_os_tick_init to restart the timer upon waking up. However, due to the current implementation, it should not be used in any application code, as it will trigger a system-wide halt via the safeguard mechanism.

Esempio:

```
/* Example showing why this function is restricted */
void task_power_management(void *pvParameters) {
    /* Attempting to disable the tick would trigger a CA_SAFEGUARD error */
    /* ca_os_tick_disable(); */
```

```
5
6       /* Instead, use supported power management functions if available */
7       ca_os_delay_ms(1000);
8
9       /* If the system needs to perform a critical operation, use critical
        sections */
10      ca_enter_critical();
11      /* Perform sensitive operation here */
12      ca_exit_critical();
13  }
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.85 **Q:** When should a developer call ca_os_tick_disable() and what is the intended purpose of this function?
A: A developer might consider calling ca_os_tick_disable() to disable the SysTick timer for purposes such as power management or debugging. The function is intended to block system interrupts and stop the system time counting. However, it is currently non-functional and causes a system crash, so it should not be used in production code.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using caOS types that demonstrates how to call ca_os_tick_disable()?
A: void example_function(void) { ca_os_tick_disable(); }

Return conf:0.95 **Q:** What happens when ca_os_tick_disable() is called, and what causes the function to fail?
A: When ca_os_tick_disable() is called, it causes a system crash because the function is non-functional. It does not return any value, but the crash occurs due to its safeguard mechanism. The function is not thread-safe, is not blocking, and cannot be called from an ISR. The failure is inherent to its design as a safety mechanism.

`ca_os_tick_drift_cycles`

Header: `caos/inc/systick.h:103`
Source: `caos/src/systick.c:326:338`
GUARD: `EN_SYSTICK`

```
1  i64 ca_os_tick_drift_cycles(void)
```

Calculates the cycle-count discrepancy between the OS tick timer and the CPU DWT cycle counter.

Ritorna:
Returns an i64 representing the difference in CPU cycles. A positive value indicates the tick timer is ahead of the cycle counter, while a negative value indicates the cycle counter is ahead. A significant negative step between two consecutive calls typically indicates a missed OS tick interrupt.

Note:

- This function is thread-safe because it disables interrupts during the calculation to ensure a consistent snapshot of the SysTick and DWT registers. It is NOT callable from an ISR because it performs its own interrupt management (disabling/enabling IRQs), which would interfere with the ISR's execution context. The function relies on the DWT (Data Watchpoint and Trace) unit being enabled and configured; if the DWT is not initialized, the cycle count will be invalid. A common mistake is to interpret the absolute value of the return; only the delta between two consecutive calls is meaningful for detecting timing drift or jitter.

Uso:

- This function is primarily used for system diagnostics, performance profiling, and verifying the accuracy of the OS scheduler's timing. It is often paired with ca_os_tick_64_ms to correlate wall-clock time with CPU cycle consumption. Developers use it to detect if the system is 'losing' time due to excessive interrupt latency or if the SysTick interrupt is being delayed by higher-priority tasks or critical sections. It is typically called periodically within a monitoring task or during system health checks to ensure the OS tick remains synchronized with the hardware clock.

Esempio:

```
/* Check for timing drift to monitor system health */
static i64 last_drift = 0;
i64 current_drift = ca_os_tick_drift_cycles();

/* Calculate the change in drift between checks */
i64 delta = current_drift - last_drift;

/* If delta is large, we might have missed a tick or experienced high jitter */
if (delta < -1000) {
    /* Log or handle the timing anomaly */
    ca_printf("Warning: Significant timing drift detected: %lld\n", delta);
}

last_drift = current_drift;
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_os_tick_drift_cycles in my application?
A: You should call it periodically within a monitoring task or during system health checks. It is primarily used for system diagnostics, performance profiling, and verifying the accuracy of the OS scheduler's timing by correlating wall-clock time with CPU cycle consumption.

Code example conf:0.90 **Q:** How do I implement a basic check for timing drift using caOS types?
A: i64 drift = ca_os_tick_drift_cycles(); if (drift < 0) { /* Handle potential missed OS tick */ }

Return conf:1.00 **Q:** What does a negative return value from ca_os_tick_drift_cycles signify?
A: A negative value indicates the CPU DWT cycle counter is ahead of the OS tick timer. A significant negative step between two consecutive calls typically indicates that an OS tick interrupt was missed.

Error/Edge case conf:1.00 **Q:** Can I call ca_os_tick_drift_cycles inside an ISR, and what are the common pitfalls?
A: No, it is not callable from an ISR because it performs its own interrupt management (disabling/enabling IRQs), which would interfere with the ISR's context. Additionally, a common mistake is interpreting the absolute value of the return; only the delta between two consecutive calls is meaningful. Finally, the function will return invalid data if the DWT unit is not initialized.

`ca_os_tick_hw_counter`

Header: `caos/inc/systick.h:239`
Source: `caos/src/systick.c:534:538`
GUARD: `EN_SYSTICK`

```
u32 ca_os_tick_hw_counter(void)
```

Reads the current 24-bit down-counter value from the hardware SysTick timer.

Ritorna:
A 32-bit unsigned integer representing the current value of the SysTick register (0 to 0x00FFFFFF).

Note:

- This function is non-blocking and executes in constant time, making it safe to call from both threads and Interrupt Service Routines (ISRs). It directly accesses the hardware SysTick->VAL register, which counts downwards from the reload value to zero. Because it is a raw hardware read, it does not account for how many times the timer has overflowed; it only provides the 'fractional' time remaining within the current OS tick. A common beginner mistake is assuming this value represents absolute time or a high-resolution timestamp without considering the reload value or the current tick count.

Uso:

- This function is typically used for high-precision micro-benchmarking or measuring execution time of code blocks that are shorter than a single OS tick. It is often paired with ca_os_tick_ms or ca_os_tick_64 to calculate elapsed time by combining the full tick count with the fractional hardware counter value. Developers use it when they need to profile performance or implement fine-grained delays that require sub-millisecond resolution.

Esempio:

```c
/* Example: Measuring the execution time of a critical section */
u32 start_val, end_val;

/* Capture the hardware counter before the operation */
start_val = ca_os_tick_hw_counter();

/* Perform the operation to be measured */
/* ... critical code block ... */

/* Capture the hardware counter after the operation */
end_val = ca_os_tick_hw_counter();

/* Calculate the difference (note: this logic assumes no tick overflow occurred)
    */
if (start_val >= end_val) {
    u32 elapsed = start_val - end_val;
    /* Log the elapsed cycles using ca_printf */
    ca_printf("Operation took %lu cycles\n", elapsed);
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_os_tick_hw_counter() and why is it useful for timing measurements?
A: You should call ca_os_tick_hw_counter() when you need to measure high-precision time intervals, typically for performance analysis or timing critical tasks. It provides access to the raw SysTick hardware counter value, which decrements from its load value to zero, making it suitable for precise time measurements. The function is non-blocking and can be safely called from ISRs, but it's not thread-safe as it directly accesses hardware registers.

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_os_tick_hw_counter() with caOS types?
A: u32 start = ca_os_tick_hw_counter(); // ... some code execution ... u32 end = ca_os_tick_hw_counter(); u32 diff = start - end; // Difference in ticks

Return conf:0.90 **Q:** What do the return values of ca_os_tick_hw_counter() mean and what causes it to return 0?
A: The function returns the current value of the SysTick hardware counter, which is a 24-bit decrementing value. If the function returns 0, it indicates an error, typically because SysTick has not been initialized correctly. The function does not return 0 for normal operation; it only returns 0 when there's a failure in accessing the hardware or when SysTick is not active.

`ca_os_tick_init`

Header: `caos/inc/systick.h:68`
Source: `caos/src/systick.c:117:175`
GUARD: `EN_SYSTICK`

```
int ca_os_tick_init(ca_tick_speed_t us)
```

Initializes the ARM Cortex-M SysTick timer to generate periodic system interrupts.

Parametri:

Nome	Tipo	Descrizione
`us`	`ca_tick_speed_t`	The desired interval in microseconds between SysTick interrupts. Must be a value that results in a reload count between 1 and 0xFFFFFF.

Ritorna:
SYSTICK_OK (0) on success, or SYSTICK_ERROR if the system clock is too slow or the requested interval exceeds the 24-bit timer capacity.

Note:

- This function is not thread-safe and must be called during the early system initialization phase before the OS scheduler starts. It configures the hardware SysTick peripheral to trigger an interrupt at the specified frequency, which is the heartbeat of the entire RTOS. It is not callable from an ISR because it modifies global hardware registers and resets internal counters. A common beginner mistake is calling this function multiple times or after the scheduler has already started, which will cause the system time to jump or the scheduler to crash.

Uso:

- This is typically the first function called in the main entry point after basic clock configuration. It is mandatory for any OS functionality that relies on time, such as delays, timeouts, or task scheduling. It is usually paired with ca_os_run, which starts the scheduler that consumes these ticks. If this initialization fails, the system should enter a safe state (e.g., an infinite loop) because the OS cannot function without a time base.

Esempio:

```
/* Initialize system tick for 1ms intervals */
if (ca_os_tick_init(1000) != SYSTICK_OK) {
    /* If tick init fails, the OS cannot run. Halt the system. */
    while(1);
}

/* Proceed with other OS initializations */
ca_mpu_init();
ca_bus_error_init();

```

```
/* Start the OS scheduler */
ca_os_run();
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_os_tick_init() in a caOS application?
A: You should call ca_os_tick_init() once during the system initialization phase, before any other RTOS functions are used. It is essential for setting up the SysTick timer that drives the RTOS tick, which is required for task scheduling and timing functions.

Code example conf:0.95 **Q:** Can you show a minimal example of how to use ca_os_tick_init() with caOS types?
A: int result = ca_os_tick_init(1000); // Initialize SysTick for 1ms ticks if (result == SYSTICK_OK) { // SysTick is ready for use } else { // Handle initialization error }

Parameters conf:0.95 **Q:** What does the 'us' parameter in ca_os_tick_init() represent and what are valid values?
A: The 'us' parameter specifies the interval between SysTick interrupts in microseconds. Valid values are positive integers such as 1000 (1ms), 100 (100us), or 10 (10us). Values less than or equal to zero will cause the function to return SYSTICK_ERROR.

Return conf:0.95 **Q:** What do the return values SYSTICK_OK and SYSTICK_ERROR mean in ca_os_tick_init()?
A: SYSTICK_OK indicates successful initialization of the SysTick timer with the specified tick interval. SYSTICK_ERROR indicates that initialization failed, which can be due to SYSCLK being too low (<1MHz), the reload value being too large for the 24-bit register, or an invalid 'us' value.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_os_tick_init()?
A: Common mistakes include calling the function more than once without resetting the system, using a tick interval that is too small causing an overflow in the reload value, and not checking the return value, which can lead to continued operation without an active SysTick timer.

Behavior conf:0.90 **Q:** Is ca_os_tick_init() blocking, ISR-safe, or thread-safe?
A: ca_os_tick_init() is a blocking function that does not return until initialization is complete or an error is detected. It is not ISR-safe and should not be called from an ISR. It is thread-safe if no other functions modify SysTick registers concurrently, though it is typically called only once at system startup.

`ca_os_tick_ms`

Header: `caos/inc/systick.h:144`
Source: `caos/src/systick.c:433:441`
GUARD: `EN_SYSTICK`

```
u32 ca_os_tick_ms(void)
```

Returns the current system time in milliseconds since the OS started.

Ritorna:
A 32-bit unsigned integer representing the elapsed time in milliseconds. This value will eventually wrap around to zero after approximately 49.7 days of continuous operation.

Note:

- This function is non-blocking and thread-safe, as it performs a double-read of the internal 64-bit tick counter to ensure data consistency without needing to disable interrupts. It is safe to call from both tasks and Interrupt Service Routines (ISRs). The most common mistake is assuming the value is absolute time (like a wall clock); it is only useful for

measuring relative durations or calculating timeouts. Always ensure the OS tick has been initialized via ca_os_tick_init before calling this, otherwise the returned value will remain zero.

Uso:

- This function is the standard way to implement non-blocking timeouts and duration tracking in caOS. It is typically used by capturing a 'start time' (t0) and comparing it against the current time in a loop or conditional check. It is frequently paired with ca_os_delay_ms for simple timing, or used within communication services (like TCP or ARP) to determine if a packet or connection has expired based on a predefined TTL (Time-To-Live). By subtracting the start time from the current time, the result correctly handles the 32-bit wrap-around due to unsigned integer arithmetic.

Esempio:

```c
/* Example: Measuring the duration of a blocking operation */
#include "caos.h"

void measure_fat_operation(void) {
    /* Capture start time before the operation */
    u32 t0 = ca_os_tick_ms();

    /* Perform a blocking filesystem operation */
    ca_fat_lock();
    /* Perform filesystem format or write operation */
    ca_fat_unlock();

    /* Calculate elapsed time */
    u32 elapsed = ca_os_tick_ms() - t0;

    /* Log the result using standard caOS output */
    ca_printf_n("Operation took %u ms", (unsigned)elapsed);
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should a developer call ca_os_tick_ms() and what is its primary use case?

A: A developer should call ca_os_tick_ms() to get the number of milliseconds elapsed since the system started. It is primarily used for measuring time intervals and managing timeouts in embedded applications. The function is non-blocking and can be called from any context, including ISRs, making it suitable for real-time systems where timing information is needed without halting execution.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_os_tick_ms() with caOS types?

A: u32 start_time = ca_os_tick_ms(); // Some operation here u32 elapsed_time = ca_os_tick_ms() - start_time; if (elapsed_time > 1000) { // Timeout occurred }

Return conf:0.90 **Q:** What do the return values of ca_os_tick_ms() indicate and what causes it to return 0?

A: The function returns a u32 value representing the number of milliseconds since system startup. A return value of 0 indicates either an error or that the system has not been initialized properly. This can happen if the SysTick timer is not configured correctly, leading to incorrect or unupdated time values. The function does not guarantee precision for critical timing measurements.

---

ca_os_tick_restart_gap

Header: caos/inc/systick.h:100
Source: caos/src/systick.c:315:317
GUARD: EN_SYSTICK

```
u32 ca_os_tick_restart_gap(void)
```

Returns the number of CPU cycles lost or 'gapped' during the last OS tick restart process.

Ritorna:
A 32-bit unsigned integer representing the cycle count gap. A value of 0 indicates perfect synchronization, while higher values represent the drift accumulated due to tick adjustments.

Note:

- This function is non-blocking and thread-safe as it simply reads a pre-calculated internal variable. It is safe to call from an ISR, as it performs no complex logic or resource locking. It is primarily used for system diagnostics and performance monitoring to ensure the OS tick remains synchronized with the hardware timer (CYCCNT). The most common mistake is assuming this value is a constant; it is dynamic and changes based on how often the OS tick is adjusted or retuned.

Uso:

- This function is used by system developers to monitor the precision of the RTOS scheduler. It is typically called within diagnostic tasks or logging functions to report the health of the system clock. It is often paired with ca_os_clocks_per_tick() to calculate the percentage of drift or to verify that the tick compensation logic is working correctly. It is not intended for application-level logic, but rather for low-level system tuning and debugging.

Esempio:

```
/* Diagnostic task to monitor system clock drift */
void monitor_task(void *arg) {
    while (1) {
        u32 gap = ca_os_tick_restart_gap();
        u32 period = ca_os_clocks_per_tick();

        /* Log the gap to the system console for analysis */
        ca_stdout_printf_n("System tick gap: %u cycles (Period: %u)", gap,
    period);

        /* If gap is too large, we might need to recalibrate */
        if (gap > (period / 10)) {
            /* Perform manual tick retuning if drift exceeds 10% of period */
            ca_os_tick_retune();
        }

        ca_os_delay_ms(1000);
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_os_tick_restart_gap and what is its primary purpose?
A: You should use this function for system diagnostics and performance monitoring. It is intended to help developers verify the precision of the RTOS scheduler and ensure the OS tick remains synchronized with the hardware timer (CYCCNT). It is typically called within diagnostic tasks or logging functions, rather than for application-level logic.

Code example conf:1.00 **Q:** How can I implement a simple diagnostic check to report the current tick drift?
A: You can call the function and store the result in a u32 variable as shown:
void report_drift(void) { u32 gap = ca_os_tick_restart_gap(); /* Log or process the gap value here */ }

Return conf:1.00 **Q:** What does the return value of ca_os_tick_restart_gap represent, and does a non-zero value indicate a system failure?
A: The return value is a u32 representing the number of CPU cycles lost or 'gapped' during the last OS tick restart. A value of 0 indicates perfect synchronization, while higher values represent drift accumulated due to tick adjustments. A non-zero value is not a failure, but rather a dynamic measurement used for system tuning and monitoring.

`ca_os_tick_restart_quantum`

Header: `caos/inc/systick.h:84`
Source: `caos/src/systick.c:256:264`
GUARD: `EN_SYSTICK`

```
void ca_os_tick_restart_quantum(void)
```

Synchronizes the internal OS tick counter with the hardware SysTick timer to account for drift or manual quantum adjustments.

Ritorna:
This function returns void and does not provide a status code.

Note:

- This function is designed to be called when the system needs to reconcile the software-tracked tick count with the actual hardware SysTick register state. It is typically invoked during low-power transitions or when the scheduler needs to force a re-evaluation of the current time quantum. It is not thread-safe and must be called from a context where the SysTick state is stable, typically within an ISR or a critical section. A common mistake is calling this function too frequently, which can lead to excessive CPU overhead and inaccurate time tracking due to the overhead of reading and clearing the hardware timer.

Uso:

- This function is primarily used by the kernel scheduler to maintain accurate timekeeping when the system enters or exits an idle state. It is often paired with manual manipulation of the PendSV interrupt (SCB->ICSR) to trigger a context switch immediately after the tick has been synchronized. Developers should not call this function in application-level code; it is an internal kernel mechanism used to ensure that the OS tick remains consistent with the hardware clock frequency.

Esempio:

```
/* Example: Triggering a tick synchronization during idle task processing */
void idle_task_handler(void) {
    /* Check if the system is currently in the idle context */
    if (ca_os_running()) {
        /* Synchronize the OS tick with the hardware timer before forcing a
    context switch */
        ca_os_tick_restart_quantum();

        /* Manually trigger the PendSV interrupt to allow the scheduler to pick
    up new tasks */
```

```
        /* SCB->ICSR |= SCB_ICSR_PENDSVSET_Msk; // This is a direct register access as seen in the kernel */
    }
}
```

Q&A — fine-tuning review *2/2 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_os_tick_restart_quantum and what is its primary purpose?
A: You should call this function when the system needs to reconcile the software-tracked tick count with the actual hardware SysTick register state, such as during low-power transitions or when the scheduler needs to force a re-evaluation of the current time quantum. It is an internal kernel mechanism and should not be called in application-level code. It must be called from a context where the SysTick state is stable, such as an ISR or a critical section, as it is not thread-safe.

Code example conf:0.90 **Q:** How do I implement a critical section to safely call ca_os_tick_restart_quantum?
A: To ensure the SysTick state is stable, you must wrap the call in a critical section. Example: { u32 primask = ___get_PRIMASK(); ___disable_irq(); ca_os_tick_restart_quantum(); if (!primask) ___enable_irq(); }

`ca_os_tick_resume`

Header: `caos/inc/systick.h:84`
Source: `caos/src/systick.c:210:215`
GUARD: `EN_SYSTICK`

```
void ca_os_tick_resume(void)
```

Restarts the SysTick timer to resume OS timekeeping after exiting a low-power STOP mode.

Ritorna:
void

Note:

- This function is non-blocking and is intended to be called immediately after the CPU wakes up from a low-power state where the SysTick peripheral was disabled. It is not thread-safe in the sense that it directly manipulates hardware registers; it must be called from a privileged context, typically within the power management sequence. A common mistake is calling this function while the system is still in a state where the clock source for the SysTick is not yet stable or configured, which can lead to unpredictable timing behavior or hard faults. It does not require any prior initialization beyond the standard system clock setup, but it assumes the SysTick was previously suspended using ca_os_tick_suspend.

Uso:

- This function is used exclusively within power management routines to restore the OS heartbeat after the MCU has entered and exited a deep sleep (STOP) mode. It is always paired with ca_os_tick_suspend, which is called before entering the sleep state to prevent the SysTick interrupt from firing while the system is powered down. The typical sequence is: disable interrupts, suspend the tick, enter sleep (WFI), wake up, resume the tick, and finally re-enable interrupts. This ensures that the OS timekeeping remains consistent and does not attempt to process stale or missing interrupts during the transition.

Esempio:

```c
/* Power management sequence to enter low power mode */
__disable_irq();

/* Stop the OS tick to prevent interrupts during sleep */
ca_os_tick_suspend();

/* Ensure all memory operations are complete before sleeping */
__DSB();

/* Enter STOP mode - CPU waits for an interrupt to wake up */
__WFI();

/* Wake up occurred: restore the OS tick to resume scheduling */
ca_os_tick_resume();

/* Re-enable interrupts to allow the scheduler to process pending tasks */
__enable_irq();
```

Q&A — fine-tuning review *2/2 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_os_tick_resume and why is it necessary?
A: You should call ca_os_tick_resume immediately after the CPU wakes up from a low-power STOP mode. It is necessary to restart the SysTick timer to restore the OS heartbeat, as the timer is typically disabled during deep sleep to prevent interrupts from firing while the system is powered down.

Code example conf:0.95 **Q:** How do I correctly implement the power management sequence using ca_os_tick_resume?
A: The sequence must be performed in a privileged context with interrupts disabled. Example:
void enter_low_power(void) { ___disable_irq(); ca_os_tick_suspend(); // Enter STOP mode via system control register ___WFI(); ca_os_tick_resume(); ___enable_irq(); }

`ca_os_tick_retune`

Header: `caos/inc/systick.h:74`
Source: `caos/src/systick.c:183:195`
GUARD: `EN_SYSTICK`

```c
int ca_os_tick_retune(void)
```

Recalculates the SysTick timer reload value to maintain the system tick period after a CPU frequency change.

Ritorna:
Returns SYSTICK_OK (0) on success, or SYSTICK_ERROR (-1) if the SysTick is not initialized, the calculated reload value is zero, or the value exceeds the hardware limit (SYSTICK_MAX_RELOAD).

Note:

- This function is non-blocking and thread-safe, but it is intended to be called during system configuration phases rather than during high-frequency task execution. It does not reset the internal tick counter, ensuring that absolute sleep deadlines (like those used in ca_os_sleep_until) remain accurate across frequency transitions. A common mistake is calling this function before the system clock or the SysTick peripheral has been properly initialized, which will result in a SYSTICK_ERROR. It is not designed to be called from an ISR, as it modifies global hardware registers that should be managed in a controlled, synchronous manner.

Uso:

- This function is used exclusively when the system clock frequency is dynamically adjusted, typically via ca_pwr_set_freq. Because the SysTick timer relies on the CPU clock (SystemCoreClock) to generate periodic interrupts, changing the frequency without calling this function would cause the OS tick period to drift or become incorrect. It is usually paired with other 'retune' functions, such as ca_uart_retune or ca_can_retune, to ensure that all time-dependent peripherals remain synchronized after the power state change.

Esempio:

```c
/* Example: Adjusting system frequency and updating dependent peripherals */
void update_system_performance(u16 new_scale) {
    /* 1. Change the CPU frequency */
    ca_pwr_set_freq(new_scale);

    /* 2. Recalculate SysTick to maintain the same OS tick period */
    if (ca_os_tick_retune() != 0) {
        /* Handle error: SysTick retuning failed */
        // Log error using ca_printf or similar
    }

    /* 3. Retune communication peripherals to match new clock */
    ca_uart_retune();

    /* Optional: Retune CAN bus if enabled */
    // ca_can_retune();
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_os_tick_retune and what is its purpose?
A: You should call ca_os_tick_retune whenever the system clock frequency is dynamically adjusted, such as after calling ca_pwr_set_freq. Its purpose is to recalculate the SysTick timer reload value to ensure the OS tick period remains accurate and does not drift after the CPU frequency change.

Code example conf:0.90 **Q:** How do I implement a frequency change in caOS while keeping the system tick accurate?
A: After updating the system frequency, you must call the retune function: ca_pwr_set_freq(NEW_FREQ_VAL); if (ca_os_tick_retune() != 0) { /* Handle error */ }

Return conf:1.00 **Q:** What do the return values of ca_os_tick_retune indicate?
A: The function returns SYSTICK_OK (0) on success. It returns SYSTICK_ERROR (-1) if the SysTick is not initialized, if the calculated reload value is zero, or if the calculated value exceeds the hardware limit (SYSTICK_MAX_RELOAD).

Error/Edge case conf:1.00 **Q:** What are the common mistakes or edge cases to avoid when using ca_os_tick_retune?
A: A common mistake is calling the function before the system clock or SysTick peripheral is properly initialized, which causes a SYSTICK_ERROR. Additionally, it must not be called from an ISR because it modifies global hardware registers that require synchronous management. It is also intended for configuration phases, not high-frequency task execution.

`ca_os_tick_suspend`

Header: `caos/inc/systick.h:79`
Source: `caos/src/systick.c:202:205`

GUARD: `EN_SYSTICK`

```
void ca_os_tick_suspend(void)
```

Suspends the SysTick timer and clears pending interrupts to allow the CPU to enter low-power sleep modes safely.

Ritorna:
void

Note:

- This function is non-blocking and is intended to be called immediately before entering a low-power state like WFI (Wait For Interrupt). It is not thread-safe and must be called within a critical section (interrupts disabled) to prevent race conditions where a tick interrupt might trigger exactly as you are trying to suspend it. The most common beginner mistake is forgetting to call ca_os_tick_resume() after waking up, which will cause the RTOS scheduler to stop functioning because it no longer receives the heartbeat ticks required for task switching.

Uso:

- This function is used exclusively in power management routines to prepare the hardware for deep sleep or stop modes. It is typically paired with ca_os_tick_resume() in a sequence: disable interrupts, suspend the tick, enter sleep, wake up, resume the tick, and re-enable interrupts. Without this sequence, any pending SysTick or PendSV interrupt would immediately wake the CPU from sleep, effectively preventing the system from ever staying in a low-power state.

Esempio:

```
void enter_low_power_mode(void) {
    // Disable interrupts to ensure atomic transition to sleep
    ca_enter_critical();

    // Stop the SysTick to prevent it from waking the CPU prematurely
    ca_os_tick_suspend();

    // Ensure all memory operations are complete before sleeping
    __DSB();

    // Enter low power mode (e.g., WFI instruction)
    // The CPU will wait here until an external interrupt occurs
    __WFI();

    // Wake up: restore the system tick to resume RTOS scheduling
    ca_os_tick_resume();

    // Re-enable interrupts to allow normal operation
    ca_exit_critical();
}
```

Q&A — fine-tuning review *2/2 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_os_tick_suspend and why is it necessary for power management?
A: You should call ca_os_tick_suspend immediately before entering a low-power state like WFI. It is necessary to suspend the SysTick timer and clear pending interrupts to ensure the CPU can safely enter sleep mode without being immediately woken by a pending tick or PendSV interrupt. It must be called within a critical section to prevent race conditions.

Code example conf:0.90 **Q:** How do I correctly implement a sleep sequence using ca_os_tick_suspend in caOS?
A: You must wrap the sequence in a critical section to ensure atomicity. Example: { ca_os_irq_disable(); ca_os_tick_suspend(); /* Enter WFI */ ca_os_tick_resume(); ca_os_irq_enable(); }

`ca_os_tick_x_ms`

Header: `caos/inc/systick.h:234`
Source: `caos/src/systick.c:108:111`
GUARD: `EN_SYSTICK`

```
u32 ca_os_tick_x_ms(u32 ms)
```

Converts a time duration in milliseconds into the equivalent number of OS scheduler ticks.

Parametri:

Nome	Tipo	Descrizione
`ms`	`u32`	The duration in milliseconds to convert. Must be a positive integer.

Ritorna:
Returns the calculated number of ticks. If the result of the calculation is zero (e.g., input is too small for the current tick resolution), it returns 1 to ensure at least one tick is represented.

Note:

- This function is a helper utility and is thread-safe as it performs a simple mathematical calculation without accessing shared hardware resources. It is safe to call from an ISR, although rarely needed there. The conversion depends on the system's tick frequency (defined by the OS configuration); if your system tick is 1ms, 1ms equals 1 tick, but if the tick is 100us, 1ms equals 10 ticks. A common beginner mistake is assuming that 1ms always equals 1 tick; always use this function to ensure your code remains portable if the system tick frequency is changed in the future.

Uso:

- This function is used whenever you need to interface with OS primitives that require time in 'ticks' rather than 'milliseconds'. It is typically paired with functions like ca_os_sleep_ticks or task management functions that expect a tick-based timeout. You should use this in the initialization phase or within task loops to calculate delays dynamically based on the current system configuration, ensuring your timing logic is always synchronized with the scheduler's heartbeat.

Esempio:

```c
/* Example: Putting a task to sleep for 50ms */
#include "caos.h"

void task_example(void) {
    u32 delay_ms = 50;

    /* Convert milliseconds to system ticks */
    u32 ticks = ca_os_tick_x_ms(delay_ms);

    /* Use the calculated ticks to sleep the current task */
    /* ca_os_sleep_ticks is the standard way to block for a specific tick count */
    ca_os_sleep_ticks(ticks);
```

```

    /* After waking up, we can perform other operations */
    ca_printf("Task resumed after 50ms\n");
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_os_tick_x_ms() in my caOS application?
A: You should use ca_os_tick_x_ms() when you need to convert a time value in milliseconds to the number of system ticks used by the caOS kernel. This is useful for setting delays or sleep times in a way that's native to the RTOS. It's typically used in conjunction with functions like ca_task_manager_to_sleep() to manage task scheduling and timing.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_os_tick_x_ms() in a caOS task?
A: u32 delay_ticks = ca_os_tick_x_ms(500); // Convert 500ms to ticks ca_task_manager_to_sleep(d // Sleep for 500ms

Parameters conf:0.90 **Q:** What is the meaning of the 'ms' parameter in ca_os_tick_x_ms(), and what are valid values?
A: The 'ms' parameter represents the time value in milliseconds that you want to convert to system ticks. It must be a positive integer. If 0 is passed, the function returns 1 tick to avoid null values, though this behavior might not be desired in all cases. Valid values are any positive u32 integer.

Return conf:0.85 **Q:** What does ca_os_tick_x_ms() return, and what causes it to return an incorrect value?
A: The function returns a u32 value representing the number of system ticks equivalent to the input milliseconds. If 'ms' is 0, it returns 1 tick. The function can return an incorrect value if the global variable 'us_requested' is not properly initialized or set to 0, as this variable is used in the conversion calculation.

`ca_os_timeout`

Header: `caos/inc/systick.h:219`
Source: `caos/src/systick.c:519:523`
GUARD: `EN_SYSTICK`

```
u16 ca_os_timeout(u32 start, u32 ms)
```

Checks if a specified duration has elapsed since a reference tick timestamp.

Parametri:

Nome	Tipo	Descrizione
`start`	`u32`	The reference timestamp captured via ca_os_tick() at the beginning of the operation.
`ms`	`u32`	The duration in milliseconds to wait for before returning true.

Ritorna:
Returns 1 (true) if the elapsed time is greater than or equal to the requested ms, otherwise returns 0 (false).

Note:

- This function is non-blocking and thread-safe, making it ideal for polling loops where you do not want to halt the entire system. It correctly handles the 'wraparound' issue, which

occurs when the system tick counter overflows back to zero after reaching its maximum value. It is safe to call from an ISR, though typically used in task loops. A common mistake is to call ca_os_tick() inside the loop condition instead of passing a fixed 'start' value, which would reset the timer every iteration and cause an infinite loop.

Uso:

- This function is used to implement non-blocking timeouts in communication protocols or hardware initialization sequences. It is typically paired with ca_os_tick() to capture the start time and ca_os_sleep() to yield CPU time to other tasks while waiting. You use it by capturing the tick once before entering a while-loop, then checking the timeout condition inside the loop to decide whether to continue waiting or abort the operation due to a timeout.

Esempio:

```c
/* Example: Waiting for a peripheral to be ready with a timeout */
u16 wait_for_ready(u32 timeout_ms) {
    u32 start = ca_os_tick();
    while (1) {
        /* Check if the peripheral is ready using a hypothetical status check */
        if (/* check_peripheral_status() */) {
            return 1; /* Success */
        }
        /* Check if we exceeded the timeout */
        if (ca_os_timeout(start, timeout_ms)) {
            return 0; /* Timeout reached */
        }
        /* Yield to other tasks to save power and CPU */
        ca_os_sleep(10);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use the ca_os_timeout function in my embedded application?
A: You should use ca_os_timeout when you need to check if a specific amount of time has passed since a reference point. It's commonly used for implementing delays, timeouts in communication protocols, or periodic task execution without blocking the thread. It's ideal for scenarios where you want to avoid busy-waiting loops and prefer a non-blocking approach to time management.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_os_timeout with caOS types?
A: u32 start = ca_os_tick(); while (!ca_os_timeout(start, 1000)) { // Do something } // Code here executes after 1 second has passed

Parameters conf:0.90 **Q:** What does the 'start' parameter represent in ca_os_timeout, and what are valid values?
A: The 'start' parameter represents the reference tick value obtained from ca_os_tick() when you begin measuring the timeout period. Valid values are any u32 value returned by ca_os_tick() at the start of the timeout measurement. Invalid values include any value not obtained from ca_os_tick(), which may lead to incorrect timeout behavior or undefined results.

Return conf:0.90 **Q:** What do the return values 1 and 0 from ca_os_timeout indicate, and what causes the function to return 0?
A: A return value of 1 indicates that the specified time in milliseconds (ms) has elapsed since the reference tick (start). A return value of 0 means the specified time has not yet passed. The function returns 0 when the elapsed time is less than the requested

timeout duration, or if an invalid start value is provided that does not originate from ca_os_tick().

---

ca_os_wait

Header: caos/inc/scheduler.h:648
Source: caos/src/scheduler.c:1974:1985
GUARD: EN_SCHEDULER

```
u32 ca_os_wait(void)
```

Suspends the current task execution until a signal is received from another task or ISR.

Ritorna:
Returns 0 on success (signal received), or 1 if the task context is invalid or the system is not in a state to block.

Note:

- This function is blocking; it puts the calling task into a waiting state, allowing the scheduler to run other tasks. It is NOT thread-safe to call from an ISR, as it relies on the current task context. A common mistake is calling this without first ensuring the task is registered in a wait queue or event mechanism; if no other task ever calls a wake-up function for this specific task, it will remain blocked indefinitely. Always ensure your logic handles the 'wake-up' condition before calling this, otherwise, your task will hang forever.

Uso:

- This function is typically used in a producer-consumer pattern or event-driven architecture. The standard sequence is: 1) Lock the resource/queue using ca_spinlock_lock_irq, 2) Check if the data/event is available, 3) If not, add the current task to a wait queue, 4) Unlock the spinlock, 5) Call ca_os_wait to yield CPU time. Once the task is resumed, it usually re-acquires the lock to verify the condition and removes itself from the wait queue.

Esempio:

```
/* Example: Waiting for data in a static queue */
void task_wait_for_data(void) {
    /* Assume s_queue is a global structure */
    static u32 sample;

    while (1) {
        /* Check if data is available without blocking */
        if (ca_queue_static_is_empty(&s_queue.q)) {
            /* Prepare the task to be woken up by a signal */
            ca_os_wait_prepare();

            /* Block until another task signals us via ca_os_wake_up_handle */
            if (ca_os_wait() == 0) {
                /* Successfully resumed by a signal */
            }
        }

        /* Process data now that we are sure it is available */
        if (ca_queue_static_receive(&s_queue.q, &sample, 0u)) {
            ca_printf("Received sample: %u\n", sample);
        }
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_os_wait() in a task, and what is its purpose in task synchronization?
A: You should call ca_os_wait() in a task when you want to suspend the task until it receives a wake-up signal. It is used for task synchronization, allowing tasks to wait for events or messages before continuing execution. This function is typically used in conjunction with other caOS functions like ca_os_notify() or ca_os_signal() to coordinate task behavior.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_os_wait() with caOS types?
A: u32 task_func(void) { while (1) { ca_os_wait(); // Task resumes here after being woken up } return 0; }

Return conf:0.95 **Q:** What do the return values 0 and 1 from ca_os_wait() indicate, and what causes the function to return 1?
A: ca_os_wait() returns 0 when the task is successfully woken up. It returns 1 in case of an error, such as when the current task is invalid or the execution context is not valid. If the task is not valid, the function returns immediately with 1 without blocking.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_os_wait() that could lead to errors or indefinite blocking?
A: Common mistakes include calling ca_os_wait() from an ISR, which is not thread-safe and not allowed. Another issue is failing to properly add the task to a waiting queue before calling ca_os_wait(); if the task is not added to the queue, it may remain blocked indefinitely. Additionally, calling ca_os_wait() in an invalid task context or without a valid task will cause an immediate return of 1.

Behavior conf:0.90 **Q:** Is ca_os_wait() blocking or non-blocking, and is it safe to call from an ISR or in a multi-threaded context?
A: ca_os_wait() is a blocking function that suspends the current task until it is woken up. It is not safe to call from an ISR, as it is not ISR-safe. It is also not thread-safe, and must be called only in a valid task context. The function internally calls ca_os_yield() if the task is blocked, to allow other tasks to execute.

`ca_os_wait_prepare`

Header: `caos/inc/scheduler.h:660`
Source: `caos/src/scheduler.c:1987:1992`
GUARD: `EN_SCHEDULER`

```
1 u32 ca_os_wait_prepare(void)
```

Prepares the current task to block indefinitely until a signal or event occurs.

Ritorna:
Returns 1 if no task is currently running (error). Returns 0 if the task was successfully prepared to block and is now waiting. Returns a non-zero value if the task was already signaled or the wait condition was met before blocking.

Note:

- This function is a low-level primitive used to transition a task into a BLOCKED state. It is not thread-safe if called outside of a critical section; it must be called while the relevant resource lock is held to prevent race conditions where a signal is sent between checking a condition and calling this function. It is strictly forbidden to call this from an ISR, as it triggers a task context switch. The most common mistake is failing to re-check the resource condition after this function returns, which can lead to spurious wakeups or missed events.

Always ensure the task is properly enqueued in a wait queue before calling this, otherwise, the task may block without any mechanism to wake it up.

Uso:

- This function is used when implementing custom synchronization primitives or waiting for specific hardware events. It is typically paired with a lock (e.g., ca_spinlock_lock_irq) and a loop: first, you acquire the lock and check if the resource is available; if not, you call ca_os_wait_prepare() to park the task, then release the lock. Upon waking, the task must re-acquire the lock and re-verify the condition before proceeding. It is the standard way to implement 'wait-forever' logic in the OS.

Esempio:

```c
/* Example: Waiting for a custom condition under a spinlock */
void wait_for_custom_event(void) {
    ca_spinlock_lock_irq(&my_lock);

    /* Check condition: if false, prepare to block */
    while (my_condition == 0) {
        /* ca_os_wait_prepare() handles the internal blocking logic */
        u32 parked = ca_os_wait_prepare();

        /* Release lock so others can signal us */
        ca_spinlock_unlock_irq(&my_lock);

        /* If we were successfully parked, yield to let other tasks run */
        if (parked == 0U) {
            ca_os_yield();
        }

        /* Re-acquire the lock to check the condition again */
        ca_spinlock_lock_irq(&my_lock);
    }

    /* Condition met, proceed with work */
    my_condition = 0;
    ca_spinlock_unlock_irq(&my_lock);
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_os_wait_prepare and what is the correct pattern for implementing a wait loop?
A: You should use ca_os_wait_prepare when implementing custom synchronization primitives or waiting for hardware events. The correct pattern is to acquire a lock (e.g., ca_spinlock_lock_irq), check if the resource is available, call ca_os_wait_prepare to park the task if the resource is unavailable, and then release the lock. Upon waking, you must re-acquire the lock and re-verify the condition to handle potential spurious wakeups.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_os_wait_prepare within a synchronization loop?
A: void wait_for_resource(void) { while (resource_busy()) { ca_spinlock_lock_irq(); if (resource_busy()) { ca_os_wait_prepare(); } ca_spinlock_unlock_irq(); } }

Return conf:1.00 **Q:** What do the different return values of ca_os_wait_prepare signify?
A: A return value of 1 indicates an error because no task is currently running. A return value of 0 indicates the task was successfully prepared to block and is now waiting. A non-zero value (other than 1) indicates that the task was already signaled or the wait condition was met before the task could block.

ca_os_wake_up_handle

Header: caos/inc/scheduler.h:580
Source: caos/src/scheduler.c:2877:2883
GUARD: EN_SCHEDULER

```
void ca_os_wake_up_handle(ca_task_handle_t h)
```

Forces a suspended or blocked task to transition into the ready state.

Parametri:

Nome	Tipo	Descrizione
h	ca_task_handle_t	The unique handle of the target task. Must be a valid handle obtained via ca_os_task_handle or similar; invalid or stale handles will result in no action.

Ritorna:
void

Note:

- This function is thread-safe and can be called from both task context and ISRs. It is non-blocking, meaning it immediately signals the scheduler to move the target task to the ready queue without waiting for the operation to complete. A common mistake is assuming this function immediately switches context to the target task; it only makes the task eligible for execution. If the target task has a higher priority than the current task, a context switch will occur at the next scheduling point. The function internally disables interrupts to ensure the task state transition is atomic, preventing race conditions if the target task is simultaneously being modified by another process.

Uso:

- Use this function to wake up a task that is currently in a waiting state, such as one blocked on a semaphore, event, or delay. It is typically used in interrupt service routines or high-priority monitor tasks to signal that a resource is ready or a condition has been met. It is often paired with ca_os_suspend_handle to implement manual task control or with synchronization primitives like ca_sem_wait to resume processing after an external event. In complex systems, it is used to force-start tasks that are waiting for specific system conditions to be met, ensuring they resume execution immediately rather than waiting for their original timeout or event trigger.

Esempio:

```
/* Example: Waking up a worker task from an ISR */

ca_task_handle_t worker_task_handle;

void sensor_data_ready_isr(void) {
    /* The worker task is currently blocked waiting for data */
    /* We wake it up so it can process the new sensor input */
    ca_os_wake_up_handle(worker_task_handle);
}

void worker_task_entry(void *param) {
    while (1) {
        /* Task blocks here until the ISR calls ca_os_wake_up_handle */
        /* ca_sem_wait(&data_semaphore); */

        /* Process the data */
```

```
        ca_printf("Processing sensor data...\n");
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_os_wake_up_handle in my application?
A: You should use this function to transition a task from a suspended or blocked state into the ready state. It is typically used in ISRs or high-priority monitor tasks to signal that a resource is ready or a condition has been met, such as resuming a task blocked on a semaphore or delay.

Code example conf:0.90 **Q:** How do I implement a simple wake-up call using caOS types?
A: Assuming you have a valid handle, the implementation is straightforward:
void signal_task(ca_task_handle_t h) { ca_os_wake_up_handle(h); }

Parameters conf:1.00 **Q:** What happens if I pass an invalid or stale handle to ca_os_wake_up_handle?
A: The parameter 'h' must be a valid handle obtained via ca_os_task_handle or similar. If you provide an invalid or stale handle, the function will result in no action.

`ca_os_yield`

Header: `caos/inc/scheduler.h:698`
Source: `caos/src/scheduler.c:2113:2134`
GUARD: `EN_SCHEDULER`

```
void ca_os_yield(void)
```

Forces the current task to voluntarily relinquish CPU control to the scheduler.

Ritorna:
void

Note:

- This function triggers a PendSV interrupt to perform a context switch, effectively pausing the current task and allowing the scheduler to pick the next ready task. It is strictly forbidden to call this from an ISR, as ISRs must never block or trigger a context switch. The function is designed to be safe for task-level code, but it must not be used to 'wait' for events; if you need to wait for data or signals, use proper synchronization primitives like ca_event_wait or ca_message_receive. A common beginner mistake is calling this inside a loop to wait for a hardware peripheral, which is inefficient and can lead to priority inversion or system instability; always prefer event-driven designs over polling.

Uso:

- Use this function in cooperative multitasking scenarios where a task performs a non-blocking operation that might take time, such as waiting for a spinlock or processing a long list of items, to ensure other tasks get CPU time. It is typically used in a 'wait-and-retry' loop where the task is not yet blocked by the OS but wants to be a 'good citizen' by yielding. It is often paired with ca_spinlock_trylock_cooperative or custom polling logic to prevent CPU starvation in low-priority background threads.

Esempio:

```
/* Example: Polling a spinlock without starving the system */
void task_wait_for_lock(ca_spinlock_t *lock) {
    /* Attempt to acquire the lock; if busy, yield to other tasks */
    while (ca_spinlock_trylock_cooperative(lock) == false) {
        /* Yield to allow other tasks to run while we wait for the lock */
```

```
        ca_os_yield();
    }

    /* Critical section starts here */
    ca_printf("Lock acquired successfully!\n");

    /* Release the lock when finished */
    ca_spinlock_unlock_cooperative(lock);
}
```

Q&A — fine-tuning review *2/2 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should a developer call ca_os_yield() and what is the intended behavior?
A: A developer should call ca_os_yield() to force an immediate context switch via PendSV, typically when the current thread cannot proceed and needs to yield control to the scheduler. This function is useful for cooperative multitasking or when a thread is waiting for a resource that may not become available immediately. The function is blocking and will put the calling thread to sleep until another thread is scheduled.

Code example conf:0.85 **Q:** Can you provide a minimal C code example using ca_os_yield() with caOS types?
A: u8 thread1(void* arg) { while(1) { // Do some work ca_os_yield(); // Yield to scheduler } return 0; }
void example(void) { ca_os_thread_create(thread1, NULL, 1024); }

`ca_parse_ipv4`

Header: `caos/net/inc/net_utils.h:16`
Source: `caos/net/src/net_utils.c:9:17`
GUARD: `EN_NET_UTILS`

```
bool ca_parse_ipv4(const char *s, ca_ipv4_t *ip)
```

Parses a string representation of an IPv4 address (e.g., '192.168.1.1') into an ca_ipv4_t structure.

Parametri:

Nome	Tipo	Descrizione
`s`	`const char *`	Pointer to the null-terminated string containing the IP address. Must be in dot-decimal notation.
`ip`	`ca_ipv4_t *`	Pointer to the destination structure where the parsed bytes will be stored. Must be a valid, allocated memory address.

Ritorna:
Always returns true. Note: The function does not perform validation on the input string format; if the string is malformed, the resulting IP structure will contain garbage or zeros.

Note:

- This function is non-blocking and thread-safe as it performs no shared resource access or OS-level synchronization. It is safe to call from an ISR, provided the input string pointer is valid and the destination memory is accessible. The most common beginner mistake is passing a NULL pointer or a string that does not contain four octets, which will lead to undefined behavior or memory corruption if the parser continues reading past the string

buffer. It does not check for valid IP ranges (0-255), so it will truncate any value larger than 255 to a single byte.

Uso:

- This function is typically used in CLI command handlers or configuration parsers to convert user-provided text into a machine-readable format. It is often paired with ca_strncmp to identify the command, followed by this function to extract the IP, and finally passed to network service functions like ca_dns_service_set_server or ca_arp_del_ip. It is essential to ensure the input string is properly validated for format before calling this function to avoid logic errors in network configuration.

Esempio:

```c
/* Example: Parsing a user-provided IP address from a CLI command */
#include "caos.h"

void handle_set_dns_command(const char *input_string) {
    ca_ipv4_t new_ip;
    /* Ensure the input string is not null before parsing */
    if (input_string != NULL) {
        /* Parse the string into the ca_ipv4_t structure */
        ca_parse_ipv4(input_string, &new_ip);

        /* Apply the parsed IP to the DNS service */
        ca_dns_service_set_server(new_ip);

        ca_printf_n("DNS server updated successfully.");
    } else {
        ca_printf_n("Error: Invalid input string.");
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_parse_ipv4 in a caOS network application?
A: You should use ca_parse_ipv4 when you need to convert a string representation of an IPv4 address (like '192.168.1.1') into a structured numeric format for network processing. It's commonly used in network protocol implementations, ARP resolution, and DNS lookups where IP addresses are received as strings and must be converted for further operations.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_parse_ipv4 with caOS types?
A: cstr ip_str = "10.0.0.1"; ipv4_t ip; bool success = ca_parse_ipv4(ip_str, &ip); // If success is true, ip now contains the parsed IPv4 address.

Parameters conf:0.90 **Q:** What are the valid and invalid values for the parameters of ca_parse_ipv4?
A: The parameter 's' must be a valid null-terminated string representing an IPv4 address. Passing NULL or a non-null-terminated string leads to undefined behavior. The parameter 'ip' must be a valid pointer to an ca_ipv4_t structure; passing NULL will cause a crash. The function does not validate that each octet is between 0 and 255, so malformed inputs like '999.1.1.1' are accepted without error.

Return conf:0.85 **Q:** What does the return value of ca_parse_ipv4 indicate, and what causes it to fail?
A: The function returns true if the string was successfully parsed into an ca_ipv4_t structure. It returns false if the parsing failed, which typically happens when the input string is malformed or not properly null-terminated. However, note that the function does not validate the range of octets (0-255), so invalid ranges like '999.1.1.1' will still return true but produce incorrect results.

ca_parse_mac

Header: caos/net/inc/net_utils.h:22
Source: caos/net/src/net_utils.c:19:27
GUARD: EN_NET_UTILS

```
bool ca_parse_mac(const char *s, ca_mac_t *mac)
```

Parses a colon-separated MAC address string into a ca_mac_t structure.

Parametri:

Nome	Tipo	Descrizione
s	const char *	Pointer to the null-terminated string containing the MAC address (e.g., "00:1A:2B:3C:4D:5E").
mac	ca_mac_t *	Pointer to the destination ca_mac_t structure where the parsed bytes will be stored.

Ritorna:
Always returns true. Note that the function does not perform validation on the input string format; if the string is malformed, the resulting ca_mac_t values may be incorrect.

Note:

- This function is a non-blocking utility used for string parsing. It is thread-safe as it does not modify any global state or shared resources, operating only on the provided pointers. It is safe to call from an ISR, provided the input string pointer is valid and accessible. The most common beginner mistake is passing a NULL pointer or a string that does not contain the expected colon separators, which will lead to undefined behavior or incorrect data in the ca_mac_t structure.

Uso:

- This function is typically used in CLI command handlers or configuration parsers where a user-provided string needs to be converted into a binary format for network operations. It is often paired with ca_strncmp to identify the command (e.g., 'add'), followed by ca_parse_mac to extract the hardware address, and finally ca_arp_set to apply the configuration to the network stack. It acts as a bridge between human-readable input and the internal network data structures.

Esempio:

```
/* Example: Parsing a MAC address from a CLI input string */
#include "caos.h"

void handle_arp_command(const char *input) {
    ca_mac_t my_mac;
    /* Check if the command starts with 'add' */
    if (ca_strncmp(input, "add", 3) == 0) {
        /* Move pointer past 'add' and skip spaces */
        const char *mac_str = input + 3;

        /* Parse the MAC address string into the structure */
        if (ca_parse_mac(mac_str, &my_mac)) {
            /* Assume we have an ca_ipv4_t structure ready */
            ca_ipv4_t target_ip;
            /* Helper logic to find IP in string would go here */
            /* ca_parse_ipv4(ip_str, &target_ip); */

            /* Apply the parsed MAC to the ARP table */
            ca_arp_set(target_ip, my_mac);
        }
```

```
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_parse_mac in a caOS network application?
A: You should call ca_parse_mac when you need to convert a MAC address string, such as "AA:BB:CC:DD:EE:FF", into a 6-byte array for network operations. It is typically used when parsing configuration data or incoming network packets that contain MAC addresses in text format.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_parse_mac with caOS types?
A: cstr mac_str = "12:34:56:78:9A:BC"; mac_t mac; bool result = ca_parse_mac(mac_str, &mac); // If result is true, mac now contains [0x12, 0x34, 0x56, 0x78, 0x9A, 0xBC]

Parameters conf:0.90 **Q:** What are the valid and invalid values for the parameters of ca_parse_mac?
A: The parameter 's' must be a valid, non-NULL pointer to a string in the format 'xx:xx:xx:xx:xx:xx' where each 'xx' is a valid hexadecimal byte. Invalid values include NULL pointers, strings with incorrect format (e.g., missing colons, non-hex characters), or strings shorter than required. The parameter 'mac' must be a valid, non-NULL pointer to a ca_mac_t array (6-byte array) where the parsed MAC will be stored.

Return conf:0.90 **Q:** What does ca_parse_mac return and what causes it to return false?
A: ca_parse_mac returns true if the MAC string is successfully parsed into a 6-byte array. It returns false if the input string is invalid, such as when it is NULL, does not conform to the expected 'xx:xx:xx:xx:xx:xx' format, or contains non-hexadecimal characters. The function does not perform explicit validation of the MAC format beyond assuming valid hexadecimal input.

`ca_pbkdf2_sha256`

Header: `caos/crypto/inc/kdf.h:69`
Source: `caos/crypto/src/kdf.c:126:168`
GUARD: `EN_KDF`

```
bool ca_pbkdf2_sha256(const u8 *password, u32 password_len, const u8 *
    salt, u32 salt_len, u32 iterations, u8 *out, u32 out_len)
```

Derives a cryptographically strong key from a password and salt using the PBKDF2-HMAC-SHA256 algorithm.

Parametri:

Nome	Tipo	Descrizione
`password`	`const u8 *`	Pointer to the input password buffer.
`password_len`	`u32`	Length of the password in bytes.
`salt`	`const u8 *`	Pointer to the salt buffer (adds randomness to prevent rainbow table attacks).
`salt_len`	`u32`	Length of the salt in bytes.
`iterations`	`u32`	Number of iterations. Higher values increase security but significantly increase computation time.
`out`	`u8 *`	Pointer to the output buffer where the derived key will be stored.
`out_len`	`u32`	Desired length of the derived key in bytes.

Ritorna:

Returns true if the key derivation completed successfully, false if any input pointer is NULL or if iterations/out_len are zero.

Note:

- This function is CPU-intensive and blocking. It is not thread-safe if the same context buffers are shared, though this implementation uses local stack variables. Do not call this from an ISR, as the high iteration count will cause a watchdog reset or system hang. Ensure the stack size is sufficient, as this function uses local cryptographic contexts. A common mistake is setting the iteration count too high for the system's clock speed, leading to unresponsive tasks.

Uso:

- Used to transform human-readable passwords into high-entropy cryptographic keys for encryption or authentication. It is typically used during system initialization or when a user provides a password to unlock a secure service. It is often paired with ca_memzero to clear sensitive buffers after use and ca_hkdf_sha256 if further key expansion or separation is required.

Esempio:

```
u8 derived_key[32];
u8 password[] = "my_secret_password";
u8 salt[] = "unique_salt_value";

// Derive a 32-byte key using 4096 iterations
if (ca_pbkdf2_sha256(password, 18U, salt, 17U, 4096U, derived_key, 32U)) {
    // Use the key for secure operations
    // ...
} else {
    // Handle error: invalid parameters
}

// Always clear sensitive data from memory after use
ca_memzero(derived_key, sizeof(derived_key));
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_pbkdf2_sha256 function in my caOS application?
A: You should use this function to transform human-readable passwords into high-entropy cryptographic keys for authentication or encryption. It is typically called during system initialization or when a user provides a password to unlock a secure service.

Code example conf:0.90 **Q:** Provide a minimal C example of how to call ca_pbkdf2_sha256 using caOS types.
A: u8 password[] = "my_secret"; u8 salt[] = "random_salt"; u8 key[32]; bool success = ca_pbkdf2_sha256(password, 9, salt, 11, 1000, key, 32); if (success) { /* use key */ ca_memzero(key, 32); }

Parameters conf:1.00 **Q:** What is the purpose of the iterations parameter and what are its constraints?
A: The iterations parameter defines the number of computation cycles; higher values increase security but also increase computation time. It must be greater than zero, as a value of zero will cause the function to return false.

Return conf:1.00 **Q:** What does the boolean return value indicate and what causes a failure?
A: The function returns true if the key derivation completes successfully. It returns false if any input pointer is NULL, or if either the iterations or out_len parameters are set to zero.

Error/Edge case conf:1.00 **Q:** What are the common mistakes or edge cases to avoid when using ca_pbkdf2_sha256?

A: A common mistake is setting the iteration count too high for the system's clock speed, which leads to unresponsive tasks. Additionally, you must ensure the stack size is sufficient, as the function uses local cryptographic contexts.

Behavior conf:1.00 **Q:** Is ca_pbkdf2_sha256 safe to call from an Interrupt Service Routine (ISR)?

A: No, do not call this function from an ISR. It is a CPU-intensive and blocking function; calling it from an ISR will likely cause a watchdog reset or a system hang.

`ca_pktrace_enable`

Header: `caos/net/inc/packet_tracer_service.h:38`
Source: `caos/net/src/packet_tracer_service.c:114:114`
GUARD: `EN_PACKET_TRACER_SERVICE`

```
void ca_pktrace_enable(bool en)
```

Globally enables or disables the packet tracing service to control diagnostic output.

Parametri:

Nome	Tipo	Descrizione
`en`	`bool`	Set to true to start capturing/tracing packets, false to stop. This acts as a master switch for the tracer.

Ritorna:
void

Note:

- This function is a non-blocking, thread-safe toggle that modifies a static internal flag. It does not require complex initialization itself, but the packet tracer service must have been previously configured via ca_pktrace_set_sender and initialized via ca_pktrace_start. A common mistake for juniors is to call this function before the tracer service is fully set up, which may result in silent failures or no output being generated. It is safe to call from any thread, but it is generally intended for use during system configuration or via a CLI command to toggle debugging on the fly.

Uso:

- This function is typically used in the system initialization phase or within a diagnostic task to manage the flow of network debug information. It is usually paired with ca_pktrace_set_sender to define where the trace data goes (e.g., STDOUT or a buffer) and ca_pktrace_start to prepare the internal structures. By toggling this function, you can dynamically enable or disable network monitoring without restarting the entire application, which is essential for troubleshooting intermittent network issues in production environments.

Esempio:

```
/* Initialize the packet tracer service before enabling it */
ca_pktrace_set_sender(STDOUT); // Direct trace output to standard output
ca_pktrace_start();            // Prepare internal tracer buffers

/* Enable tracing to start monitoring network traffic */
ca_pktrace_enable(true);

/* ... later in the application, if traffic is too high or debugging is done ... */
ca_pktrace_enable(false); // Stop tracing to save CPU and memory resources
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_pktrace_enable() and what is its purpose in a caOS application?
A: You should call ca_pktrace_enable() to enable or disable function call tracing in the caOS kernel for debugging and performance analysis. It must be called before initializing the RTOS kernel to be effective. The tracing can impact system performance and is generally non-blocking and not thread-safe. It should not be called from an ISR. This function is typically used during development or debugging phases to monitor kernel behavior.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_pktrace_enable() with caOS types?
A: void example_function() { ca_pktrace_enable(true); // Enable tracing // ... RTOS initialization code ... ca_pktrace_enable(false); // Disable tracing }

Parameters conf:0.95 **Q:** What does the 'en' parameter in ca_pktrace_enable() control, and what are the valid values?
A: The 'en' parameter controls whether function call tracing is enabled or disabled in the caOS kernel. If 'en' is true, tracing is enabled; if false, tracing is disabled. Valid values are true and false. The function does not fail, but may have no effect if tracing is not supported or active in the system.

`ca_pktrace_filter_dst`

Header: `caos/net/inc/packet_tracer_service.h:40`
Source: `caos/net/src/packet_tracer_service.c:116:116`
GUARD: `EN_PACKET_TRACER_SERVICE`

```
1 void ca_pktrace_filter_dst(ca_ipv4_t ip); /* .addr=0     match any */
```

Sets the destination IPv4 address filter for the packet tracing service.

Parametri:

Nome	Tipo	Descrizione
`ip`	`ca_ipv4_t`	The IPv4 address to filter by. Set the .addr field to 0 to disable filtering and match any destination address.

Ritorna:
void

Note:

- This function is non-blocking and thread-safe as it simply updates a static variable used by the packet tracer. It is safe to call from an ISR, though typically it is called from a configuration task or CLI command. The most common mistake is assuming this function clears the filter; it only updates it. To 'clear' the filter, you must explicitly pass an ca_ipv4_t structure with the .addr field set to 0.

Uso:

- This function is used to narrow down network debugging output when the system is processing high volumes of traffic. It is typically used in a sequence: first, call ca_pktrace_start to initialize the service, then use ca_pktrace_filter_dst to focus on a specific target, and finally call ca_pktrace_enable(true) to begin capturing. It is often paired with ca_pktrace_filter_src to create complex traffic capture rules.

Esempio:

```c
/* Example: Configure packet tracer to only capture traffic destined for
    192.168.1.50 */
ipv4_t target_ip;
target_ip.addr = 0xC0A80132; // 192.168.1.50

// Initialize the tracer service
ca_pktrace_start();

// Set the filter to only show packets going to the target
ca_pktrace_filter_dst(target_ip);

// Enable the tracer
ca_pktrace_enable(true);

// Later, to stop filtering and see all traffic again:
ipv4_t any_ip;
any_ip.addr = 0;
ca_pktrace_filter_dst(any_ip);
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_pktrace_filter_dst() and what is its purpose in packet tracing?
A: You should call ca_pktrace_filter_dst() to configure a filter for packet tracing based on the destination IP address. This function allows you to specify which packets should be traced, making it easier to monitor specific network traffic. If you pass an IP address of 0 (0.0.0.0), it will match all packets regardless of their destination IP, which is useful for general tracing without filtering.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_pktrace_filter_dst() with caOS types?
A: ca_ipv4_t filter_ip = { .addr = 0x01020304 }; /* Example IP 1.2.3.4 */ ca_pktrace_filter_dst(filter_
/* To accept all packets, use: */ ipv4_t any_ip = { .addr = 0 }; /* 0.0.0.0 */ ca_pktrace_filter_dst(any_

Parameters conf:0.85 **Q:** What does the ca_ipv4_t ip parameter in ca_pktrace_filter_dst() represent, and what are valid values for it?
A: The ca_ipv4_t ip parameter specifies the destination IP address to filter packets. A value of 0 (0.0.0.0) means that all packets will be matched, regardless of their destination IP. Any other valid IPv4 address in network byte order can be used to filter specific destinations. Invalid values include uninitialized or malformed IP addresses, which may lead to unpredictable behavior.

`ca_pktrace_filter_src`

Header: `caos/net/inc/packet_tracer_service.h:39`
Source: `caos/net/src/packet_tracer_service.c:115:115`
GUARD: `EN_PACKET_TRACER_SERVICE`

```c
void ca_pktrace_filter_src(ca_ipv4_t ip); /* .addr=0     match any */
```

Sets the source IPv4 address filter for the packet tracing service.

Parametri:

Nome	Tipo	Descrizione
`ip`	`ca_ipv4_t`	The IPv4 address to filter by. Set the address field to 0 to disable filtering and match all source addresses.

Ritorna:

void

Note:

- This function is non-blocking and thread-safe as it simply updates a global configuration variable. It is safe to call from both tasks and ISRs. The primary precondition is that the packet tracing service must be initialized. A common mistake is assuming this function clears the filter; it only updates it. To effectively 'clear' the filter, you must pass an ca_ipv4_t structure with the address field explicitly set to 0.

Uso:

- This function is used to narrow down network debugging output to a specific device or host. It is typically called during the initialization phase of a network diagnostic task or via a CLI command to isolate traffic. It is usually paired with ca_pktrace_enable(true) to start capturing and ca_pktrace_filter_dst() to further refine the traffic scope by destination.

Esempio:

```
/* Define the target IP to monitor */
ipv4_t target_ip;
target_ip.addr = 0x0A000005; // Represents 10.0.0.5

/* Apply the filter to only see packets from this source */
ca_pktrace_filter_src(target_ip);

/* Ensure destination filter is disabled (match all) */
ipv4_t any_ip;
any_ip.addr = 0;
ca_pktrace_filter_dst(any_ip);

/* Enable the tracer to start processing packets */
ca_pktrace_enable(true);
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_pktrace_filter_src() and why is it important for packet tracing?
A: You should call ca_pktrace_filter_src() after initializing the packet tracing system to set which source IP addresses are allowed to be traced. It's important because it allows you to filter out unwanted traffic and focus on specific communication sources, reducing noise in the trace output. The function must be called before any tracing begins and should not be modified during active tracing without proper synchronization.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_pktrace_filter_src() with caOS types?
A:

```
#include "caos/net/packet_tracer.h"

void setup_tracing(void) {
    ca_ipv4_t filter_ip = 0xC0A80101; /* 192.168.1.1 */
    ca_pktrace_filter_src(filter_ip);
}
```

Parameters conf:0.85 **Q:** What does the ca_ipv4_t ip parameter in ca_pktrace_filter_src() represent, and what are valid values?
A: The ca_ipv4_t ip parameter represents the source IP address to filter packet traces by. A value of 0 (0.0.0.0) means all packets will be traced regardless of source IP. Valid non-zero values are any valid IPv4 address represented as a 32-bit unsigned integer, such as 0xC0A80101 for 192.168.1.1. Invalid values would be any value outside the valid IPv4 address range, though the function does not validate input.

Return conf:0.90 **Q:** Does ca_pktrace_filter_src() return anything, and can it fail?
A: The function returns nothing (void). It cannot fail because it is simply a setter that modifies a global variable. It does not perform any validation or error checking, and its success depends entirely on correct usage and proper system initialization before calling.

`ca_pktrace_push_rx`

Header: `caos/net/inc/packet_tracer_service.h:32`
Source: `caos/net/src/packet_tracer_service.c:49:63`
GUARD: `EN_PACKET_TRACER_SERVICE`

```
void ca_pktrace_push_rx(u16 buf_idx)
```

Captures and logs incoming network packet metadata into the internal packet tracer ring buffer.

Parametri:

Nome	Tipo	Descrizione
`buf_idx`	`u16`	The index of the network buffer containing the received packet. Must be a valid index previously returned by ca_net_buf_alloc.

Ritorna:
void. This function does not return a status code; if the tracer is disabled or the ring buffer is full, the packet metadata is silently dropped.

Note:

- This function is non-blocking and designed to be called from the network stack's receive path. It is thread-safe regarding the ring buffer access, but it relies on the packet tracer service being initialized and enabled via ca_pktrace_enable. A common mistake is calling this function before the network stack has fully initialized the buffer metadata, leading to incomplete trace logs. It is not intended to be called directly from an ISR unless the underlying ring buffer implementation is explicitly ISR-safe.

Uso:

- This function is typically used within the network stack's RX processing logic, immediately after the packet headers have been parsed and the metadata (IP, ports, protocol) has been populated in the buffer. It is paired with ca_pktrace_enable to start the service and ca_pktrace_start to begin logging. It acts as a passive observer, allowing developers to monitor traffic flow without stopping the system execution.

Esempio:

```
/* Example: Logging an incoming packet during network processing */
void process_rx_packet(u16 buf_idx) {
    /* 1. Ensure the packet tracer is enabled before pushing */
    ca_pktrace_enable(1);

    /* 2. Push the packet metadata to the tracer ring buffer */
    /* This captures timestamp, IP, and port info from the buffer */
    ca_pktrace_push_rx(buf_idx);

    /* 3. Continue with standard network processing */
    /* e.g., ca_net_process(buf_idx); */
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_pktrace_push_rx() in my network driver implementation?
A: You should call ca_pktrace_push_rx() after successfully receiving a packet via DMA or interrupt, specifically after the packet has been copied into a circular buffer. It is used to trace received packets for network analysis and must be called after the packet is fully received and stored in the buffer identified by buf_idx.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_pktrace_push_rx()?
A: u16 buffer_index = 5; ca_pktrace_push_rx(buffer_index);

Parameters conf:0.90 **Q:** What is the significance of the buf_idx parameter in ca_pktrace_push_rx() and what are valid values?
A: The buf_idx parameter is the index of the circular buffer containing the received packet. Valid values are unsigned 16-bit integers from 0 up to the maximum number of buffers defined in the system (e.g., 0-31). Values outside this range cause undefined behavior.

Return conf:0.85 **Q:** Does ca_pktrace_push_rx() return any value, and what does it mean if it does?
A: The function returns no value. It is void and cannot fail, but calling it with an invalid buf_idx parameter may result in undefined behavior or incorrect packet tracing.

Error/Edge case conf:0.80 **Q:** What are common mistakes or edge cases when using ca_pktrace_push_rx()?
A: Common mistakes include calling the function with a buf_idx that is out of range (negative or greater than maximum buffer count), calling it from an ISR, or calling it before the buffer is actually filled with a valid packet. These can lead to incorrect tracing, data corruption, or packet loss.

`ca_pktrace_push_tx`

Header: `caos/net/inc/packet_tracer_service.h:35`
Source: `caos/net/src/packet_tracer_service.c:65:108`
GUARD: `EN_PACKET_TRACER_SERVICE`

```
1 void ca_pktrace_push_tx(u16 buf_idx)
```

Captures and logs outgoing network packet metadata for diagnostic tracing.

Parametri:

Nome	Tipo	Descrizione
`buf_idx`	`u16`	The index of the network buffer in the system pool. Must be a valid index returned by a previous allocation.

Ritorna:
void

Note:

- This function is non-blocking and designed to be called within the network stack's transmission path. It is not inherently thread-safe if the underlying ring buffer is not protected, though it is typically used in contexts where the network stack manages access. It must not be called if the packet tracer service has not been initialized. A common mistake is passing an invalid or already-freed buffer index, which will cause the system to read garbage memory or trigger a memory fault when accessing the buffer metadata.

Uso:

- This function is used exclusively for debugging and network traffic analysis. It is typically placed inside the network driver's transmission function, immediately before the packet is handed off to the hardware. It pairs with ca_pktrace_start and ca_pktrace_enable to activate the tracing service. Once enabled, it automatically extracts IP, port, and protocol information from the buffer and pushes it into a circular trace buffer for later inspection.

Esempio:

```
/* Assuming the packet tracer is enabled and initialized */
void transmit_packet(u16 buf_idx) {
    /* Capture the packet metadata before sending */
    ca_pktrace_push_tx(buf_idx);

    /* Proceed with hardware transmission */
    /* ca_eth_send(buf_idx); */
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_pktrace_push_tx() in a network application?
A: You should call ca_pktrace_push_tx() when you want to monitor the transmission of a data buffer on a network interface. It is typically used in conjunction with packet tracing services to log outgoing packets for debugging or performance analysis. The function is meant to be called after a buffer has been prepared for transmission but before it is actually sent.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_pktrace_push_tx()?
A: u16 buffer_index = 10; ca_pktrace_push_tx(buffer_index);

Parameters conf:0.90 **Q:** What is the significance of the buf_idx parameter in ca_pktrace_push_tx()?
A: The buf_idx parameter is the index of the buffer that is to be added to the TX tracking queue. It must be a value between 0 and the maximum number of available buffers (e.g., 0-255). Values outside this range may lead to undefined behavior. The function does not validate the buffer's content or state, only that the index is within bounds.

Error/Edge case conf:0.85 **Q:** What happens if I call ca_pktrace_push_tx() with an invalid buffer index?
A: If an invalid buffer index is passed (e.g., negative or exceeding the maximum buffer count), the function may cause undefined behavior. The function does not return any error code, so it's the caller's responsibility to ensure the index is valid before calling the function.

Behavior conf:0.90 **Q:** Is ca_pktrace_push_tx() safe to call from an ISR or in a multi-threaded environment?
A: No, ca_pktrace_push_tx() is neither ISR-safe nor thread-safe. It must not be called from an ISR because it could cause synchronization issues. It also must not be called concurrently from multiple threads, as it is not protected by any locking mechanism. The function is non-blocking and will return immediately, even if the tracking queue is full.

`ca_pktrace_set_sender`

Header: `caos/net/inc/packet_tracer_service.h:45`
Source: `caos/net/src/packet_tracer_service.c:117:117`
GUARD: `EN_PACKET_TRACER_SERVICE`

```
void ca_pktrace_set_sender(sender fn)
```

Registers a callback function to handle the output of captured network packets.

Parametri:

Nome	Tipo	Descrizione
`fn`	`sender`	A function pointer that defines where the packet trace data should be sent (e.g., to a UART, a buffer, or a logging service). Passing NULL will disable output redirection.

Ritorna:
void

Note:

- This function is non-blocking and thread-safe as it simply updates a static internal pointer. It is not intended to be called from an ISR, although it is technically safe to do so because it performs a simple assignment. The most common mistake is failing to set a valid sender function before calling ca_pktrace_start(), which results in captured packets being silently discarded because the system does not know where to route the trace data.

Uso:

- This function is used during the system initialization phase to configure the packet tracing service. It is typically paired with ca_pktrace_start() to activate the service and ca_pktrace_enable() to toggle the actual filtering and logging. You should call this once during startup to define your output destination, such as a serial console or a network logging task, ensuring that the tracer has a valid sink for its data before traffic begins to flow.

Esempio:

```
/* Define a custom sender function to handle trace data */
void my_trace_handler(const char *data) {
    /* Use ca_printf to output the trace data to the system console */
    ca_printf("%s\n", data);
}

void init_network_tracing(void) {
    /* Configure the packet tracer to use our custom handler */
    ca_pktrace_set_sender(my_trace_handler);

    /* Set filters to only trace traffic from a specific source if needed */
    /* ca_pktrace_filter_src(my_ip_addr); */

    /* Start the service and enable it */
    ca_pktrace_start();
    ca_pktrace_enable(true);
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:1.00 **Q:** When should ca_pktrace_set_sender be called in relation to enabling packet tracing?
A: The function ca_pktrace_set_sender must be called before enabling packet tracing with ca_pktrace_start(). Calling it after starting the trace will result in lost data, as the global variable storing the sender function will not be updated in time for the trace data to be sent.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_pktrace_set_sender with a custom print function?
A: void my_sender(cstr buffer) { // Custom send logic, e.g., to UART or log u32 len = 0; while (buffer[len] != '\0') len++; // Assume UART transmission here }
void example() { ca_pktrace_set_sender(my_sender); }

Parameters conf:1.00 **Q:** What happens if I pass NULL as the sender function to ca_pktrace_set_sender?
A: If NULL is passed as the sender function, the packet tracing will not send any data, which may lead to a crash if the trace mechanism attempts to call the sender function without proper error handling.

Return conf:1.00 **Q:** Does ca_pktrace_set_sender return any value, and what does it indicate?
A: The function does not return any value. It modifies an internal global variable that will be used later when sending traced packet data. Failures are not indicated through return values but through incorrect behavior or crashes if NULL is passed.

`ca_pktrace_start`

Header: `caos/net/inc/packet_tracer_service.h:29`
Source: `caos/net/src/packet_tracer_service.c:246:259`
GUARD: `EN_PACKET_TRACER_SERVICE`

```
void ca_pktrace_start(void)
```

Initializes the packet tracer service, setting up internal buffers, synchronization primitives, and the background processing task.

Ritorna:
void

Note:

- This function is not thread-safe and must be called during the system initialization phase before any network traffic processing begins. It initializes the internal ring buffer pointers, resets filters, creates a semaphore for task synchronization, and spawns the 'pktrace' background task. A common mistake is calling this function multiple times or while the network stack is already active, which can lead to race conditions or memory corruption in the tracer's internal buffers. It is not callable from an ISR as it performs task creation and resource allocation.

Uso:

- This function is typically called once during the system boot sequence, usually grouped with other network service initializations like 'ca_dhcp_service_start' or 'ca_ip_defrag_service_start'. It is often paired with 'ca_pktrace_set_sender' to define where the trace output should be directed (e.g., STDOUT) and 'ca_pktrace_enable' to actually start capturing packets. It must be invoked before any attempt to push packets into the tracer via 'ca_pktrace_push_rx' or 'ca_pktrace_push_tx'.

Esempio:

```
void system_init(void) {
    /* Configure the output destination for the tracer */
    ca_pktrace_set_sender(STDOUT);

    /* Initialize the packet tracer service structures and task */
    ca_pktrace_start();

    /* Enable the tracer to begin capturing traffic */
    ca_pktrace_enable(true);

    /* Proceed with other network services */
    ca_dhcp_service_start();
    ca_ip_defrag_service_start();
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_pktrace_start() and what is its purpose in a caOS application?
A: You should call ca_pktrace_start() after initializing the caOS kernel and configuring network components such as DHCP and TCP. Its purpose is to start the network packet tracing service for debugging and performance analysis. It is a non-blocking function that initializes a semaphore and starts a dedicated task for packet tracing.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_pktrace_start() with caOS types?
A: void example(void) { // Assume kernel and network components are initialized ca_pktrace_start(); }

Parameters conf:0.95 **Q:** What are the parameters of ca_pktrace_start() and what happens if I pass invalid values?
A: The function ca_pktrace_start() takes no parameters. Since there are no parameters, there are no invalid values to consider. The function is designed to be called without any arguments.

Error/Edge case conf:0.90 **Q:** What are the common mistakes or edge cases when using ca_pktrace_start()?
A: A common mistake is calling ca_pktrace_start() before initializing the caOS kernel or configuring network components, which can lead to crashes or undefined behavior. The function is not thread-safe and must not be called from an ISR. It is also a debug-only function and should not be used in production code.

`ca_pool_init`

Header: `caos/inc/pool.h:35`
Source: `caos/src/pool.c:101:125`
GUARD: `EN_POOL`

```
i32 ca_pool_init(const ca_pool_cfg_t *cfg)
```

Initializes a pool of worker threads to process jobs asynchronously.

Parametri:

Nome	Tipo	Descrizione
`cfg`	`const ca_pool_cfg_t *`	Pointer to a configuration structure containing the number of threads, stack size, and priority. Must not be NULL, and n_threads must be greater than 0.

Ritorna:
Returns 0 on success. Returns -1 if the configuration pointer is NULL, the thread count is zero, or if any worker thread fails to initialize.

Note:

- This function is not thread-safe and must be called during system initialization before any jobs are submitted to the pool. It is not callable from an ISR because it performs memory and task management operations that require a scheduler context. A common mistake is to call this function multiple times or after the scheduler has already started processing tasks, which can lead to memory corruption or undefined behavior. Ensure that the stack size provided in the configuration is sufficient for the tasks the workers will perform, as insufficient stack space will cause a stack overflow fault.

Uso:

- Use this function when you need to offload repetitive or heavy tasks to a background worker group to keep your main application responsive. It is typically called once during the system startup sequence, followed by the creation of a producer task that uses ca_pool_submit to feed work to the pool. It is often paired with ca_pool_stats to monitor the performance and load of the worker threads during runtime.

Esempio:

```
/* Define the pool configuration */
ca_pool_cfg_t pool_cfg = {
    .n_threads = 4,
    .stack_size = 512,
    .priority = CA_TH_LEVEL1
};

/* Initialize the pool before starting the scheduler */
if (ca_pool_init(&pool_cfg) != 0) {
    /* Handle initialization error */
    while(1);
}

/* Submit a job to the pool */
/* Note: 'pool_worker_func' is a user-defined function pointer */
ca_pool_submit(pool_worker_func, NULL);
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_pool_init in a caOS application?
A: You should call ca_pool_init before any usage of the thread pool, to initialize the pool of threads that will execute jobs. It must be called once during system initialization before any job is submitted to the pool.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_pool_init with caOS types?
A: ca_pool_cfg_t cfg = { .n_threads = 2 }; i32 result = ca_pool_init(&cfg); if (result != 0) { /* handle error */ }

Parameters conf:0.95 **Q:** What does the 'n_threads' field in ca_pool_cfg_t control, and what are valid values?
A: The 'n_threads' field specifies the number of threads to create in the pool. It must be greater than zero; otherwise, ca_pool_init will fail. Valid values are positive integers.

Return conf:0.95 **Q:** What does a return value of -1 from ca_pool_init indicate?
A: A return value of -1 indicates that the initialization failed. This can happen if 'cfg' is NULL, 'n_threads' is zero, or if it was not possible to create one or more threads.

Error/Edge case conf:0.90 **Q:** What are common mistakes when calling ca_pool_init?
A: Common mistakes include passing a NULL pointer for 'cfg', setting 'n_threads' to zero, or attempting to call ca_pool_init from multiple threads simultaneously. Creating too many threads can also lead to memory or system overload errors.

Behavior conf:0.90 **Q:** Is ca_pool_init blocking or non-blocking, and can it be called from an ISR?
A: ca_pool_init is a blocking function, as it waits until all threads are created before returning. It is not safe to call from an ISR, and it is not thread-safe, so it must not be invoked concurrently by multiple threads.

`ca_pool_stats`

Header: `caos/inc/pool.h:52`
Source: `caos/src/pool.c:138:142`

GUARD: `EN_POOL`

```
void ca_pool_stats(u8 *busy, u8 *free_th, u8 *queued)
```

Retrieves current status metrics for the task pool, including busy threads and queued jobs.

Parametri:

Nome	Tipo	Descrizione
`busy`	`u8 *`	Pointer to a variable where the number of currently active threads will be stored.
`free_th`	`u8 *`	Pointer to a variable where the number of available (idle) threads will be stored.
`queued`	`u8 *`	Pointer to a variable where the number of jobs currently waiting in the queue will be stored.

Ritorna:
This function returns void. It updates the memory locations pointed to by the input arguments.

Note:

- This function is non-blocking and thread-safe, as it performs simple memory reads of internal pool counters. It can be safely called from both tasks and ISRs. The primary precondition is that the pool must have been initialized via ca_pool_init before calling this function. A common mistake is passing NULL pointers to the function, which will cause a hard fault or memory corruption; always ensure the variables passed are valid and allocated in memory.

Uso:

- This function is typically used for system monitoring, diagnostics, or load balancing. It is often called periodically within a monitoring task or a CLI command to display the health of the task pool. By comparing the 'busy' and 'queued' values, developers can determine if the system is overloaded and if the pool size or queue size needs to be adjusted in the configuration. It is usually paired with ca_pool_init during system startup and ca_pool_submit when adding new work to the system.

Esempio:

```
void monitor_task(void *p) {
    u8 busy, free, queued;
    while (1) {
        /* Retrieve current pool status */
        ca_pool_stats(&busy, &free, &queued);

        /* Log the status to the console */
        ca_stdout_printf_n("Pool Status: Busy=%u, Free=%u, Queued=%u", busy,
    free, queued);

        /* Wait for 1 second before next check */
        ca_os_delay_ms(1000);
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_pool_stats() and why would I need to know the thread pool statistics?
A: You should call ca_pool_stats() when you need to monitor the status of the thread pool, such as checking how many threads are busy, how many are free, and how many jobs are queued. This is useful for debugging, performance tuning, or ensuring that your

system is not overloaded with pending tasks. Since the function is non-blocking, it can be safely called from any context, including ISRs.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_pool_stats() with caOS types?
A: u8 busy, free_th, queued; ca_pool_stats(&busy, &free_th, &queued); // Now busy, free_th, and queued contain the respective counts. // Example usage in a loop: while (1) { ca_pool_stats(&busy, &free_th, &queued); // Do something with the stats }

Parameters conf:0.90 **Q:** What do the parameters of ca_pool_stats() represent, and what are the valid values for each?
A: The parameters are pointers to u8 values that will be updated with statistics: - busy: Number of currently occupied threads (0-255). - free_th: Number of free threads available (0-255). - queued: Number of jobs currently in the queue (0-255). All pointers must be valid (not NULL), otherwise the function will fail.

`ca_pool_submit`

Header: `caos/inc/pool.h:52`
Source: `caos/src/pool.c:129:138`
GUARD: `EN_POOL`

```
1 i32 ca_pool_submit(ca_task_fn_t func, void *arg)
```

Submits a task to a worker pool for asynchronous execution, with automatic fallback to a temporary thread if the pool is full.

Parametri:

Nome	Tipo	Descrizione
`func`	`ca_task_fn_t`	The function pointer to be executed by the worker thread. Must be a valid function matching the task entry signature.
`arg`	`void *`	A pointer to the data structure or memory block to be processed by the task. Ensure this memory remains valid until the task completes.

Ritorna:
Returns 0 on success (job queued or temporary thread created). Returns -1 if the job could not be queued and the system failed to spawn a temporary thread (e.g., out of memory or max threads reached).

Note:

- This function is designed for 'fire-and-forget' task offloading. It is thread-safe and internally manages synchronization with the worker pool. It is NOT safe to call from an ISR because the fallback mechanism (ca_os_job) involves thread creation, which is a blocking/scheduling operation. The most common mistake is assuming this function is instantaneous; if the pool is full and the system is under heavy load, the fallback thread creation might take time or fail, so always check the return value. Ensure the task function does not rely on local stack variables from the caller, as those will be destroyed before the task executes.

Uso:

- Use this when you have a non-critical task (like logging, data processing, or network transmission) that you don't want to block your main execution loop. It is typically used in a producer-consumer pattern where the producer calls ca_pool_submit to offload work.

Before using this, ensure the pool has been initialized via ca_pool_init. It is often paired with ca_stdout_printf_n to report failures if the system is too busy to accept new jobs.

Esempio:

```c
/* Assuming pool is initialized and job_process is a valid ca_task_fn_t */
void task_producer(void) {
    /* Allocate memory for the job data */
    void *data = ca_memory_allocate(sizeof(u32));
    if (data == NULL) return;
    *(u32*)data = 123;

    /* Attempt to offload the processing task to the pool */
    i32 result = ca_pool_submit(job_process, data);

    if (result == 0) {
        ca_stdout_printf_n("Job submitted successfully");
    } else {
        /* Handle failure: pool is full and system cannot spawn more threads */
        ca_stdout_printf_n("Critical: Failed to submit job, system overloaded");
        ca_memory_free(data);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_pool_submit to execute a background task?
A: You should use ca_pool_submit when you need to offload a task to a background thread pool for asynchronous execution. It's ideal for non-blocking operations like sensor data processing, communication tasks, or any work that doesn't require immediate results. The function is designed to be called from both regular threads and ISRs, making it suitable for interrupt-driven systems.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_pool_submit with a function that processes a sensor value?
A: void sensor_task(u32 sensor_data) { // Process sensor data here u32 result = sensor_data * 2; // Use result... }
void main(void) { // Assume pool is initialized i32 ret = ca_pool_submit(sensor_task, 100); if (ret != 0) { // Handle queue full error } }

Parameters conf:0.85 **Q:** What does the parameter 'p1' in ca_pool_submit represent and what values are valid?
A: The parameter 'p1' is a u32 value passed directly to the function pointed to by 'func'. It can be any value that makes sense for the function being called. For example, if 'func' processes sensor data, 'p1' might be a sensor ID or raw reading. Invalid values would be those that cause undefined behavior in the target function, but the function itself does not validate the value.

Return conf:0.90 **Q:** What do the return values of ca_pool_submit indicate and what causes a failure?
A: A return value of 0 indicates success: the function was accepted and queued for execution. A return value of -1 indicates failure: the thread pool queue was full and a one-shot thread could not be created to execute the function. This typically happens when the pool is saturated with work and cannot accommodate new tasks.

`ca_printf`

Header: `caos/inc/utils.h:130`
Source: `caos/src/utils.c:448:459`
GUARD: `EN_UTILS`

```
u16 ca_printf(sender out, char *buf, u16 size, const char *format, ...)
```

Formats a string into a provided buffer and sends it to a specified output handler.

Parametri:

Nome	Tipo	Descrizione
`out`	`sender`	A function pointer (callback) that handles the formatted string. Must not be NULL.
`buf`	`char *`	Pointer to a memory buffer where the formatted string will be stored. Must be large enough to hold the result.
`size`	`u16`	Total size of the buffer in bytes. The function reserves space for internal metadata.
`format`	`const char *`	Standard printf-style format string.
`...`	`variadic`	Variable arguments corresponding to the format specifiers.

Ritorna:
Returns the number of characters written to the buffer (excluding the null terminator). Returns 0 if the output handler is NULL or if formatting fails.

Note:

- This function is not thread-safe if the provided buffer is shared across tasks without external synchronization. It is not intended to be called from an ISR because it performs complex string formatting which can be time-consuming and non-deterministic. The most common mistake is providing a buffer that is too small for the formatted output, which will result in truncation. Ensure the buffer is allocated in a memory region accessible by the output handler, as the function casts the buffer to a p_str structure internally to store the size metadata.

Uso:

- Used primarily for logging or sending formatted messages to peripherals like UART or ITM. It is typically used by passing a global or static buffer to avoid stack overflow in small-stack tasks. It is often paired with initialization functions like ca_stdout_init or specific peripheral setup functions to define the 'sender' callback. After formatting, the 'out' function is invoked to transmit the data, making this a high-level wrapper for structured communication.

Esempio:

```
char log_buffer[64];
/* Assume STDOUT is a valid sender callback initialized elsewhere */
void task_log_example(void) {
    u32 sensor_val = 42;
    /* Format the string into the buffer and send it via the output handler */
    u16 written = ca_printf(STDOUT, log_buffer, sizeof(log_buffer), "Sensor
    reading: %u\r\n", sensor_val);

    if (written == 0) {
        /* Handle error: formatting failed or output handler was NULL */
    }
    ca_os_delay_ms(100);
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_printf in an embedded application?

A: You should use ca_printf when you need to format text output for logging or serial communication in embedded systems. It is especially useful in environments where HAL or standard libraries are not available, and you want to avoid the overhead of full printf implementations. It allows you to send formatted output via a custom sender function, making it suitable for UART, USB, or other output mechanisms.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_printf with caOS types?

A:

```
u8 buffer[128];
void send_uart(const char *str) {
  // Simulate UART send
  while (*str) {
    // Send character via UART
    str++;
  }
}
void example() {
  u16 len = ca_printf(send_uart, buffer, 128, "Value: %d\n", 42);
}
```

Parameters conf:0.95 **Q:** What is the significance of the 'size' parameter in ca_printf?

A: The 'size' parameter specifies the maximum size of the buffer 'buf'. It must be greater than sizeof(u16) to leave space for the size field in p_str. If 'size' is too small, it may cause buffer overflow. The buffer must be large enough to hold the formatted string and the null terminator.

Return conf:0.90 **Q:** What does a return value of 0 from ca_printf indicate?

A: A return value of 0 from ca_printf indicates either that the 'out' parameter is NULL or that an error occurred during formatting. It does not indicate a successful write, but rather a failure to output or format the string.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_printf?

A: Common mistakes include passing a buffer that is too small, which can cause memory corruption; not initializing the buffer correctly; passing NULL for 'format' or 'buf'; and calling ca_printf from an ISR, which is not supported. Additionally, if 'out' is not thread-safe, concurrent usage can lead to undefined behavior.

`ca_printf_n`

Header: `caos/inc/utils.h:131`
Source: `caos/src/utils.c:461:475`
GUARD: `EN_UTILS`

```
u16 ca_printf_n(sender out, char *buf, u16 size, const char *format,
    ...)
```

Formats a string with a newline suffix and sends it through a provided output function.

Parametri:

Nome	Tipo	Descrizione
`out`	`sender`	A function pointer responsible for transmitting the formatted data. Must not be NULL.
`buf`	`char *`	Pointer to a memory buffer that will be cast to a p_str structure to hold the formatted string.
`size`	`u16`	Total size of the provided buffer. Must be large enough to accommodate the string, the newline, and the null terminator.
`format`	`const char *`	Standard printf-style format string.

Ritorna:
Returns the total number of bytes written to the buffer, including the appended '\r\n' sequence. Returns 0 if the 'out' function pointer is NULL.

Note:

- This function is not thread-safe if the underlying 'out' function or the shared buffer is accessed concurrently by multiple tasks. It is not safe to call from an ISR because it performs formatting operations that may be computationally expensive and potentially blocking. The most common mistake is providing a buffer that is too small for the formatted string, which can lead to buffer overflows or truncated output; always ensure the buffer size accounts for the format string length plus the 3 extra bytes for '\r', '\n', and '\0'.

Uso:

- This function is typically used for logging or sending formatted status messages to peripherals like UART or USB-CDC. It is usually paired with a memory allocation function to provide the buffer, followed by the call to ca_printf_n, and finally a memory release function. It acts as a high-level wrapper to ensure consistent line endings across different communication interfaces.

Esempio:

```
/* Allocate memory for the string buffer */
char *my_buf = ca_memory_allocate(128);

if (my_buf != NULL) {
    /* Use the system stdout sender to print a formatted message */
    /* ca_stdout_get_sender() retrieves the default system output handler */
    u16 written = ca_printf_n(ca_stdout_get_sender(), my_buf, 128, "System
    status: %d", 1);

    /* Free the buffer after use */
    ca_memory_free(my_buf);
} else {
    /* Handle allocation failure */
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_printf_n in an embedded RTOS environment?
A: You should use ca_printf_n for formatted logging or debugging in embedded RTOS environments where you need to send formatted strings via a callback function. It's particularly useful for debugging tasks or logging system events, especially when you want to avoid using standard library functions that may not be available or efficient in bare-metal environments.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_printf_n with caOS types?
A: Here's a minimal example:

```c
u8 uart_out(const u8 *buf, u16 len) {
    // Simulate UART send
    return 0; // Success
}

void example() {
    char buffer[128];
    u16 result = ca_printf_n(uart_out, buffer, sizeof(buffer), "Value: %d\n", 42);
}
```

Parameters conf:0.90 **Q:** What are the valid and invalid values for the 'out' parameter in ca_printf_n?
A: The 'out' parameter must be a valid function pointer that handles the formatted buffer. It must not be NULL, as passing NULL will cause the function to fail and return 0. Any other non-NULL function pointer is valid, though the function it points to may have its own constraints on how it handles the data.

Return conf:0.95 **Q:** What does ca_printf_n return when it fails, and what causes the failure?
A: ca_printf_n returns 0 when it fails. Failure occurs if the 'out' parameter is NULL or if the buffer size is insufficient to hold the formatted string plus 2 characters for carriage return. The return value represents the number of characters written to the buffer, excluding the null terminator.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_printf_n?
A: Common mistakes include passing a buffer that is too small to hold the formatted string plus 2 characters for carriage return, passing NULL as the 'out' parameter, or using unsupported format specifiers. It's also not recommended to call ca_printf_n from an ISR due to potential stack overflow or synchronization issues.

`ca_put`

Header: `caos/inc/utils.h:120`
Source: `caos/src/utils.c:420:425`
GUARD: `EN_UTILS`

```c
u16 ca_put(sender out, c_str msg)
```

Sends a string to a registered output handler (sender).

Parametri:

Nome	Tipo	Descrizione
`out`	`sender`	A function pointer (handle) to the output destination (e.g., UART, ITM, or STDOUT). Must not be NULL.
`msg`	`c_str`	The string structure containing the data to be transmitted. Must be a valid pointer to a string object.

Ritorna:
Returns the number of bytes sent (u16) on success. Returns 0 if the 'out' sender handle is NULL or if the transmission failed.

Note:

- This function acts as a generic wrapper for output operations. It is not inherently thread-safe; if multiple tasks attempt to write to the same 'out' handle simultaneously, the output may become interleaved or corrupted. It is generally non-blocking, but this depends entirely on the implementation of the 'out' function provided. Do not call this from an ISR unless the underlying 'out' function is specifically designed for interrupt context (e.g.,

using non-blocking buffers). A common mistake is passing a NULL sender handle, which will result in a silent failure (returning 0) rather than a crash.

Uso:

- Used primarily for logging, debugging, or sending data over communication interfaces like UART or ITM. It is typically used in a sequence where a sender is initialized (e.g., ca_stdout_init or ca_uart_init), then used throughout the application lifecycle to output status messages or error logs. It is often paired with PRIVATE_STRING macros to define static messages that are passed to the function for transmission.

Esempio:

```c
/* Example: Sending a debug message to the standard output */
#include "caos.h"

/* Define a static string for output */
PRIVATE_STRING(msg_init, "System Initialized\r\n");

void app_main(void) {
    /* Assume STDOUT is a valid sender handle initialized elsewhere */
    u16 bytes_sent = ca_put(STDOUT, &msg_init);

    if (bytes_sent == 0) {
        /* Handle error: output failed or sender was NULL */
        /* Perform error recovery or log to a secondary system */
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_put to send a message through an output channel?
A: You should use ca_put when you need to send a text message through a defined output channel, such as serial communication or console logging. It is typically used in system diagnostics, error reporting, or debugging scenarios where you want to output strings to a specific interface.

Code example conf:0.85 **Q:** Can you provide a minimal C example using ca_put with caOS types?
A: u16 serial_out(cstr msg) { /* Implementation for serial output */ return 0; } void example() { cstr message = "Hello caOS"; u16 len = ca_put(serial_out, message); }

Parameters conf:0.95 **Q:** What are the valid and invalid values for the 'out' and 'msg' parameters in ca_put?
A: The 'out' parameter must be a valid function pointer to an output handler, or NULL to indicate an invalid channel. The 'msg' parameter must be a non-NULL pointer to a string, or NULL to indicate an invalid message. Passing either NULL will cause the function to return 0.

Return conf:0.90 **Q:** What does ca_put return in case of success and failure, and what causes these outcomes?
A: On success, ca_put returns the number of characters sent, which is the length of the message. If either 'out' or 'msg' is NULL, it returns 0, indicating that no message was sent due to invalid parameters.

`ca_put_n`

Header: `caos/inc/utils.h:121`
Source: `caos/src/utils.c:429:437`
GUARD: `EN_UTILS`

```
u16 ca_put_n(sender out, c_str msg)
```

Sends a string to a specified output channel followed by a newline character.

Parametri:

Nome	Tipo	Descrizione
out	sender	A function pointer (handle) representing the output destination (e.g., STDOUT, UART). Must not be NULL.
msg	c_str	The null-terminated string to be transmitted. Must be a valid pointer to memory.

Ritorna:
Returns the total number of bytes sent (string length + 2 for the newline) as a u16. Returns 0 if the 'out' parameter is NULL or if the transmission fails.

Note:

- This function is a wrapper that performs two sequential calls to the provided 'out' function: first for the message, then for the newline. It is not inherently thread-safe if the underlying 'out' function is not protected by a mutex or semaphore. Do not call this from an ISR (Interrupt Service Routine) unless the underlying 'out' function is explicitly documented as ISR-safe, as it may involve blocking operations. The most common mistake is passing a NULL sender handle, which will result in no output and a return value of 0.

Uso:

- This function is typically used for logging or debugging purposes where a clean, line-separated output is required. It is often used during system initialization or within tasks to report status updates to a console or terminal. It pairs well with 'ca_stdout_create_thread' to ensure the output channel is initialized before attempting to send data. In a typical sequence, you would initialize the output system, then use 'ca_put_n' to print status messages during the application lifecycle.

Esempio:

```c
/* Initialize the standard output thread before using it */
ca_stdout_create_thread();

/* Define a string to be printed */
PRIVATE_STRING(status_msg, "System initialized successfully");

/* Send the message to STDOUT with a newline */
u16 bytes_sent = ca_put_n(STDOUT, &priv_status_msg);

if (bytes_sent == 0) {
    /* Handle error: output failed or sender was invalid */
    /* Perform error recovery or logging here */
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_put_n function in a caOS application?
A: You should use ca_put_n when you need to send a message followed by a carriage return and line feed (\r\n) to an output device such as a UART or console. It is useful for logging, debugging, or communicating status information in a consistent format. The function is ideal for sending formatted text messages in a blocking manner.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_put_n with caOS types?
A: u16 uart_send(p_str str) { // Simulated UART send function return 0; } // Dummy implementation void example() { cstr message = "Hello, caOS!"; u16 len = ca_put_n(uart_send, message); }

Parameters conf:0.90 **Q:** What are the valid and invalid values for the 'out' and 'msg' parameters in ca_put_n?
A: The 'out' parameter must be a valid function pointer that accepts a p_str and returns u16. If 'out' is NULL, the function returns 0 without sending anything. The 'msg' parameter can be NULL, but if it is, no message is sent and the function returns 0. Any valid cstr pointer is acceptable for 'msg' as long as it points to a null-terminated string.

Return conf:0.95 **Q:** What does the return value of ca_put_n indicate, and what causes it to return 0?
A: The function returns 0 if either 'out' is NULL or 'msg' is NULL, indicating that no message was sent. It returns size+2 if the message was successfully sent, where 'size' is the length of the message string and 2 accounts for the \r\n characters appended. A return value of 0 may also indicate a failure in the underlying output function 'out'.

`ca_pwr_enter`

Header: `caos/drivers/inc/pwr.h:101`
Source: `caos/drivers/src/pwr.c:114:233`
GUARD: `EN_PWR`

```
1 int ca_pwr_enter(ca_pwr_mode_t mode, u32 wake_flags, u32 duration)
```

Puts the MCU into a low-power state (Sleep, Stop, or Standby) until a specified wake-up event occurs.

Parametri:

Nome	Tipo	Descrizione
`mode`	`ca_pwr_mode_t`	Target power mode: CA_PWR_MODE_SLEEP, CA_PWR_MODE_STOP, or PWR_MODE_STANDBY.
`wake_flags`	`u32`	Bitmask of wake sources (e.g., CA_PWR_WAKE_RTC, CA_PWR_WAKE_PIN, CA_PWR_WAKE_LPTIM).
`duration`	`u32`	Time duration for wake-up. Seconds if RTC is used, milliseconds if LPTIM is used.

Ritorna:
0 on success, -1 on failure (e.g., invalid mode, no wake sources configured, or hardware busy).

Note:

- This function is blocking; the CPU stops execution until a wake-up event occurs. It is NOT callable from an ISR. It is thread-safe in the sense that it manages critical sections internally, but it suspends the OS tick, meaning all time-based OS services (delays, timeouts) will pause. A common mistake is failing to provide at least one valid wake-up source, which will cause the system to hang indefinitely. In STOP mode, the system performs a full clock re-initialization upon wake-up; ensure your peripherals are ready to handle the clock frequency change.

Uso:

- Used to conserve battery life in battery-powered applications. It is typically called in the main loop or a dedicated power-management task. The sequence is: 1) Configure wake-up peripherals (e.g., LPTIM or RTC), 2) Call ca_pwr_enter, 3) Upon return, re-initialize or resume peripheral operations that were suspended during the low-power state.

Esempio:

```c
/* Example: Enter STOP mode for 5 seconds using RTC */
#define WAKE_TIME_SEC 5

void enter_low_power(void) {
    /* Ensure RTC is configured to trigger an alarm */
    /* ca_rtc_set_alarm(CA_RTC_ALARM_A, &al, cb); */

    /* Enter STOP mode, wake on RTC alarm */
    int status = ca_pwr_enter(CA_PWR_MODE_STOP, CA_PWR_WAKE_RTC, WAKE_TIME_SEC);

    if (status != 0) {
        ca_stdout_printf_n("Failed to enter low power mode");
    } else {
        /* System has resumed, clocks are re-initialized */
        ca_stdout_printf_n("System woke up from STOP mode");
    }
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_pwr_enter function in my caOS application?
A: You should call ca_pwr_enter in your main loop or a dedicated power-management task when you need to conserve battery life. The typical sequence involves configuring wake-up peripherals first, calling the function, and then re-initializing or resuming peripheral operations after the function returns.

Code example conf:0.90 **Q:** Provide a minimal example of how to put the system into STOP mode for 500 milliseconds using LPTIM.
A: u32 flags = CA_PWR_WAKE_LPTIM; u32 time = 500; int status = ca_pwr_enter(CA_PWR_MO flags, time); if (status != 0) { /* Handle error */ }

Parameters conf:1.00 **Q:** What are the valid values for the mode parameter in ca_pwr_enter?
A: The mode parameter accepts one of three values: CA_PWR_MODE_SLEEP, CA_PWR_MODE_S or PWR_MODE_STANDBY.

Return conf:1.00 **Q:** What do the return values of ca_pwr_enter signify?
A: The function returns 0 on success. It returns -1 on failure, which can be caused by an invalid mode, failure to configure wake sources, or if the hardware is currently busy.

Error/Edge case conf:1.00 **Q:** What is a common mistake that causes the system to hang when calling ca_pwr_enter?
A: A common mistake is failing to provide at least one valid wake-up source in the wake_flags parameter, which will cause the system to hang indefinitely.

Behavior conf:1.00 **Q:** Is ca_pwr_enter safe to call from an ISR, and does it affect OS timing?
A: No, ca_pwr_enter is not callable from an ISR. It is a blocking function that suspends the OS tick, meaning all time-based OS services like delays and timeouts will pause while the CPU is in the low-power state.

`ca_pwr_get_info`

Header: `caos/drivers/inc/pwr.h:69`
Source: `caos/drivers/src/pwr.c:235:250`
GUARD: `EN_PWR`

```c
void ca_pwr_get_info(ca_pwr_info_t *info)
```

Retrieves current system power status, including clock frequencies and peripheral activity.

Parametri:

Nome	Tipo	Descrizione
info	ca_pwr_info_t *	Pointer to a structure where power information will be stored. Must not be NULL.

Ritorna:
void. If the provided pointer is NULL, the function returns immediately without performing any operations.

Note:

- This function is non-blocking and executes quickly by iterating through the internal peripheral table. It is thread-safe in the sense that it reads hardware registers, but it does not implement internal locking, so concurrent writes to power registers by other tasks could lead to inconsistent snapshots. It is safe to call from an ISR, though it is primarily intended for diagnostic or monitoring tasks in the main application loop. A common mistake is passing an uninitialized pointer or a NULL pointer, which will cause the function to silently fail without updating any data.

Uso:

- This function is typically used for system diagnostics, power management logging, or telemetry. It is often paired with ca_pwr_init during system startup to establish a baseline, and later called periodically to monitor if peripherals are being shut down correctly to save energy. Developers use it to verify that the system clock frequency matches the expected state after calling ca_pwr_set_freq or ca_os_clock_config.

Esempio:

```
ca_pwr_info_t pwr_stats;

// Retrieve current power and clock status
ca_pwr_get_info(&pwr_stats);

// Log the number of active peripherals vs total available
ca_printf("System Freq: %lu Hz, Peripherals Active: %u/%u\n",
          pwr_stats.sysclk_hz,
          pwr_stats.periph_on,
          pwr_stats.periph_total);

// Check if we are running at max frequency
if (pwr_stats.sysclk_hz < pwr_stats.sysclk_max_hz) {
    // Logic to restore performance if needed
    ca_pwr_restore_freq();
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_pwr_get_info in my caOS application?
A: It is primarily used for system diagnostics, power management logging, or telemetry. You should call it periodically to monitor if peripherals are being shut down correctly to save energy, or to verify that the system clock frequency matches the expected state after calling ca_pwr_set_freq or ca_os_clock_config.

Code example conf:1.00 **Q:** Provide a minimal C example showing how to retrieve power information.
A: ca_pwr_info_t my_info; ca_pwr_get_info(&my_info);

Parameters conf:1.00 **Q:** What are the requirements for the info parameter in ca_pwr_get_info?
A: The parameter must be a pointer to a ca_pwr_info_t structure. It must not be NULL. If a NULL pointer is passed, the function returns immediately without performing any operations.

Error/Edge case conf:0.90 **Q:** What happens if I pass an uninitialized pointer to ca_pwr_get_info?
A: Passing an uninitialized pointer is a common mistake that will cause the function to silently fail without updating any data, as the function does not perform internal validation beyond checking for NULL.

Behavior conf:1.00 **Q:** Is ca_pwr_get_info safe to call from an ISR and does it block execution?
A: The function is non-blocking and executes quickly. It is safe to call from an ISR. While it is thread-safe in the sense that it reads hardware registers, it does not implement internal locking, so concurrent writes to power registers by other tasks could lead to inconsistent snapshots.

`ca_pwr_hw_dbg_dump`

Header: `caos/drivers/inc/ca_glue_hw.h:199`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_pwr_hw_dbg_dump(void); /* stampa i registri low-power (
    PWR/RCC/RTC/core) per diagnosi wake */
```

Prints the current state of power, clock, and interrupt registers to the system console for low-power debugging.

Ritorna:
void

Note:

- This function is a diagnostic tool used to inspect hardware registers (PWR, RCC, RTC, NVIC, etc.) that dictate how the MCU enters and exits low-power modes. It is not thread-safe and should only be called when the system is in a stable state, as reading these registers while the hardware is transitioning power states can lead to unpredictable results. It is not intended for use inside an ISR, as it performs heavy I/O operations via ca_stdout_printf_n, which will likely cause system instability or deadlocks if called from a high-priority interrupt context. The most common mistake is calling this function during a critical section or while the system is actively entering a sleep state, which can interfere with the hardware's ability to correctly latch the wake-up conditions.

Uso:

- This function is typically used during the development phase to troubleshoot issues where the MCU fails to wake up from Stop or Standby modes. It is usually triggered via a CLI command (e.g., 'dbg') to capture the state of the hardware immediately before or after a failed sleep cycle. Developers use it to compare register values between a working power-down sequence and a failing one, helping to identify misconfigured clocks, pending interrupts, or incorrect power domain settings. It is often paired with ca_pwr_enter to verify that the registers were configured as expected before the CPU halted.

Esempio:

```
/* Example: CLI command to trigger power diagnostic dump */
#include "caos.h"

void handle_pwr_debug_command(char *cmd) {
    /* Check if the user requested a power register dump */
    if (ca_strncmp(cmd, "dbg", 3) == 0) {
        /* Print a header to the console before dumping registers */
        ca_stdout_printf_n("--- Starting Power Hardware Dump ---");

        /* Call the diagnostic function to output register states */
```

```
11          ca_pwr_hw_dbg_dump();
12
13          ca_stdout_printf_n("--- Dump Complete ---");
14      } else {
15          ca_stdout_printf_n("Unknown command. Use 'dbg' to dump PWR registers.");
16      }
17  }
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_pwr_hw_dbg_dump function in my caOS project?
A: You should use this function during the development phase to troubleshoot issues where the MCU fails to wake up from Stop or Standby modes. It is typically triggered via a CLI command to capture the state of power, clock, and interrupt registers immediately before or after a failed sleep cycle, allowing you to compare register values between working and failing sequences.

Code example conf:0.90 **Q:** How can I implement a CLI command to trigger the power diagnostic dump?
A: You can map a CLI command to call the function directly, provided the system is in a stable state. Example: void cmd_dbg(void) { ca_pwr_hw_dbg_dump(); }

Error/Edge case conf:1.00 **Q:** Can I call ca_pwr_hw_dbg_dump inside an interrupt service routine or a critical section?
A: No. It is not thread-safe and should not be called inside an ISR because it performs heavy I/O via ca_stdout_printf_n, which can cause deadlocks or system instability. Calling it during a critical section or while the system is actively entering a sleep state is a common mistake that can interfere with the hardware's ability to latch wake-up conditions.

HW differences conf:1.00 **Q:** Are the registers dumped by ca_pwr_hw_dbg_dump the same across all STM32 families in caOS?
A: No, the implementation varies by family. For example, the F4xx implementation dumps PWR->CR and PWR->CSR, while the U5xx implementation dumps more granular registers like PWR->CR1, CR2, CR3, VOSR, and SVMCR. Additionally, some families like F1xx, F7xx, H5xx, and H7xx have empty implementations where the function does nothing.

`ca_pwr_hw_enter_standby`

Header: `caos/drivers/inc/ca_glue_hw.h:194`
GUARD: `EN_CA_GLUE_HW`

```
1  extern void ca_pwr_hw_enter_standby(void); /* entra in STANDBY; non
       ritorna (wake = reset) */
```

Forces the MCU into the lowest power consumption state (STANDBY) from which it can only wake up via a system reset.

Ritorna:
This function never returns. Upon a wake-up event, the MCU performs a full hardware reset, and execution restarts from the beginning of the application (main).

Note:

- This function is non-blocking in the traditional sense because it never returns; it effectively halts the CPU. It is NOT thread-safe and must NEVER be called from an ISR. Before calling this, you must ensure all pending interrupts are cleared and the system is in a stable state, as any pending interrupt will prevent the CPU from entering the low-power mode.

A common beginner mistake is calling this while peripherals are still active or while the debugger is attached; if a debugger is connected, the MCU may not enter true standby, or the wake-up behavior may be inconsistent. Always ensure the RTC or wake-up source is correctly configured before calling this, otherwise, the device may remain in standby indefinitely.

Uso:

- This function is typically used at the end of a power-management sequence when the application has finished its tasks and needs to minimize battery consumption. It is usually paired with RTC configuration functions like ca_rtc_set_alarm to schedule a future wake-up. The typical sequence is: 1) Perform necessary cleanup, 2) Disable all non-essential interrupts, 3) Configure the wake-up source (e.g., RTC alarm), 4) Call ca_pwr_hw_enter_standby. It is often the final step in a 'sleep-until-next-event' architecture.

Esempio:

```c
/* Example: Prepare the system to enter standby and wake up via RTC alarm */

/* 1. Configure the RTC alarm to trigger in 60 seconds */
ca_rtc_set_alarm(60);

/* 2. Disable all system interrupts to prevent wake-up interference */
/* Use __disable_irq() directly as it is a CMSIS intrinsic */
__disable_irq();

/* 3. Enter standby mode */
/* The MCU will stop here and only resume after a hardware reset */
ca_pwr_hw_enter_standby();

/* This code is unreachable */
while(1) {
    /* System is in standby */
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_pwr_hw_enter_standby in my application?
A: This function is typically used at the end of a power-management sequence when the application has finished its tasks and needs to minimize battery consumption. It is usually the final step in a 'sleep-until-next-event' architecture, often following the configuration of a wake-up source like an RTC alarm.

Code example conf:0.90 **Q:** Provide a minimal example of how to prepare the system and enter standby.
A: void go_to_sleep(void) { /* 1. Perform cleanup */ /* 2. Disable non-essential interrupts */ __disable_irq(); /* 3. Configure RTC alarm */ ca_rtc_set_alarm(1000); /* 4. Enter standby */ ca_pwr_hw_enter_standby(); }

Return conf:1.00 **Q:** What does the return value of ca_pwr_hw_enter_standby indicate?
A: This function never returns. Upon a wake-up event, the MCU performs a full hardware reset, and execution restarts from the beginning of the application (main).

Error/Edge case conf:1.00 **Q:** What are common mistakes that prevent the MCU from entering standby correctly?
A: Common mistakes include calling the function while peripherals are still active, having a debugger attached (which may cause inconsistent behavior or prevent true standby), or failing to clear pending interrupts, which prevents the CPU from entering the low-power mode.

Behavior conf:1.00 **Q:** Is ca_pwr_hw_enter_standby thread-safe or ISR-safe?

A: The function is not thread-safe and must never be called from an ISR. It is non-blocking in the traditional sense because it halts the CPU, but it requires that all pending interrupts are cleared and the system is in a stable state before invocation.

HW differences conf:1.00 **Q:** How does the implementation of ca_pwr_hw_enter_standby differ across STM32 families?

A: Implementations vary based on hardware architecture: F1xx uses EXTI registers for wake-up, while F4xx, F7xx, and H7xx require a specific RTC write-protection sequence (WPR) to clear flags. H5xx and U5xx utilize the RTC SCR register to clear flags, and H7xx specifically manages power domains across D1, D2, and D3 via the CPUCR register.

---

ca_pwr_hw_enter_stop

Header: `caos/drivers/inc/ca_glue_hw.h:193`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_pwr_hw_enter_stop(void); /* disabilita IRQ, entra in STOP, ritorna al wake con IRQ ancora disabilitati */
```

Puts the MCU into a low-power STOP mode, halting execution until a wake-up interrupt occurs.

Ritorna:
Always returns true upon successful wake-up and system restoration.

Note:

- This function is a blocking call that halts the CPU core until an external or internal interrupt (like RTC) triggers a wake-up. It is NOT thread-safe and must be called with interrupts disabled to prevent race conditions during the power-state transition. It is strictly forbidden to call this from an ISR, as it would cause the system to hang or crash. A common beginner mistake is failing to re-initialize system clocks (PLL, etc.) or peripheral states immediately after this function returns, as the hardware state may be inconsistent or running at a lower frequency upon wake-up.

Uso:

- This function is used to minimize power consumption during idle periods. It is typically paired with a wake-up source configuration (e.g., RTC alarm) and a sequence that saves the current system state, enters STOP mode, and then restores the system clock and peripheral configuration upon return. It is essential to ensure that all necessary wake-up interrupts are enabled in the NVIC before calling this function, otherwise, the MCU will remain in STOP mode indefinitely.

Esempio:

```
/* Prepare the system for low power consumption */
ca_os_tick_suspend();

/* Configure RTC alarm to wake the system in 5 seconds */
ca_rtc_set_alarm(5);

/* Enter STOP mode; execution halts here until the RTC alarm fires */
ca_pwr_hw_enter_stop();

/* Execution resumes here after wake-up */
/* Re-initialize system clocks and peripherals to restore normal operation */
ca_os_clock_restore_freq();
ca_os_tick_resume();
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_pwr_hw_enter_stop in my caOS application?
A: You should use this function during idle periods to minimize power consumption. It is typically paired with a pre-configured wake-up source, such as an RTC alarm, and requires a sequence to save system state before entry and restore clocks and peripheral configurations upon return.

Code example conf:0.90 **Q:** Provide a minimal example of how to call ca_pwr_hw_enter_stop.
A: Ensure interrupts are disabled before calling. Example: ___disable_irq(); bool success = ca_pwr_hw_enter_stop(); /* System resumes here */ ___enable_irq();

Return conf:1.00 **Q:** What does the return value of ca_pwr_hw_enter_stop indicate?
A: The function always returns true upon successful wake-up and system restoration. It is a blocking call that halts the CPU until an interrupt triggers the wake-up.

Error/Edge case conf:1.00 **Q:** What are the most common mistakes when using ca_pwr_hw_enter_stop?
A: Common mistakes include calling it from an ISR (which causes a crash), failing to enable wake-up interrupts in the NVIC (causing the MCU to hang indefinitely), and neglecting to re-initialize system clocks or peripheral states immediately after return, as hardware states may be inconsistent.

Behavior conf:1.00 **Q:** Is ca_pwr_hw_enter_stop thread-safe and can I call it from an ISR?
A: No, it is not thread-safe and must be called with interrupts disabled to prevent race conditions. It is strictly forbidden to call this function from an ISR, as it would cause the system to hang or crash.

HW differences conf:1.00 **Q:** How does the implementation of ca_pwr_hw_enter_stop differ between STM32 families like F1xx and U5xx?
A: Implementations vary significantly based on hardware requirements. For example, F1xx explicitly configures EXTI17 and RTC_Alarm_IRQn within the function, whereas U5xx performs complex pre-stop downshifting (SYSCLK to MSIS 4MHz), disables PLLs, and manages specific clocks like MSIK and HSI48 to ensure autonomous peripheral wake-up.

`ca_pwr_hw_get_vos`

Header: `caos/drivers/inc/ca_glue_hw.h:190`
GUARD: `EN_CA_GLUE_HW`

```
extern u32 ca_pwr_hw_get_vos(void); /* ritorna il voltage scaling
    corrente; 0xFF se non disponibile */
```

Returns the current hardware voltage scaling (VOS) level configured in the MCU power controller.

Ritorna:
Returns the VOS register value as an unsigned 32-bit integer. Returns 0xFF if the hardware platform does not support voltage scaling (e.g., STM32F1).

Note:

- This function is a non-blocking, thread-safe read of a hardware register. It is safe to call from an ISR because it performs a simple memory-mapped register access without any OS synchronization primitives. The function relies on the underlying hardware abstraction layer (HAL) being correctly compiled for the specific STM32 target. A common mistake is assuming that the returned value is a voltage in millivolts; it is actually a raw register bitfield value that corresponds to specific scaling ranges defined in the STM32 reference manual.

Uso:

- This function is typically used for system diagnostics, power management monitoring, or when logging system state via the CLI. It is often paired with ca_pwr_get_info to populate a status structure that reports the current operating conditions of the MCU. Developers use it to verify if the system is running in the expected power mode before performing operations that might be sensitive to voltage levels, such as high-speed clock configuration or flash memory writes.

Esempio:

```c
/* Example: Check current VOS and log it to the system console */
#include "caos.h"

void log_system_power_status(void) {
    u32 current_vos = ca_pwr_hw_get_vos();

    if (current_vos == 0xFF) {
        ca_printf("VOS: Not supported on this hardware\n");
    } else {
        /* Log the raw VOS register value for diagnostic purposes */
        ca_printf("Current VOS register value: 0x%02X\n", (unsigned int)
    current_vos);
    }

    /* Example of using it within a broader power info context */
    /* Assuming a struct exists to hold power data */
    /* ca_pwr_get_info(&my_pwr_info_struct); */
    /* ca_printf("System VOS: %d\n", my_pwr_info_struct.vos); */
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_pwr_hw_get_vos in my caOS application?
A: You should use this function for system diagnostics, power management monitoring, or logging system state via the CLI. It is particularly useful to verify the current power mode before performing operations sensitive to voltage levels, such as high-speed clock configuration or flash memory writes.

Code example conf:1.00 **Q:** How do I implement a simple check of the voltage scaling level using caOS types?
A: You can call the function directly and store the result in a u32 variable: u32 current_vos = ca_pwr_hw_get_vos(); if (current_vos != 0xFF) { /* Process valid VOS value */ }

Return conf:1.00 **Q:** What does the return value of ca_pwr_hw_get_vos represent, and what does 0xFF indicate?
A: The return value is a raw register bitfield value corresponding to specific scaling ranges defined in the STM32 reference manual, not a voltage in millivolts. A return value of 0xFF indicates that the hardware platform does not support voltage scaling, such as on the STM32F1 family.

HW differences conf:1.00 **Q:** How does the implementation of ca_pwr_hw_get_vos differ across STM32 families?
A: The implementation varies based on the specific power control register used by the MCU family: F4 uses PWR->CR, F7 uses PWR->CR1, H5 uses PWR->VOSCR, H7 uses PWR->D3CR, and U5 uses PWR->VOSR. The F1 family does not support this feature and returns 0xFF.

ca_pwr_hw_init

Header: caos/drivers/inc/ca_glue_hw.h:188
GUARD: EN_CA_GLUE_HW

```
extern void ca_pwr_hw_init(void); /* abilita il clock del blocco PWR (
    dove necessario) */
```

Enables the clock for the Power (PWR) peripheral block on the STM32 microcontroller.

Ritorna:
void

Note:

- This function is a low-level hardware abstraction layer (HAL) routine. It is not thread-safe and is intended to be called only during the early system initialization phase before the RTOS scheduler starts. It is not designed to be called from an ISR. The primary risk is attempting to access PWR registers (like those used for low-power modes or voltage scaling) before this function has enabled the peripheral clock, which will result in a hard fault or silent register access failure. On some newer STM32 families like H5 or H7, the PWR clock is always enabled by hardware, making this function a no-op, but it must still be called to maintain cross-platform code compatibility.

Uso:

- This function is used exclusively within the initialization sequence of the power management driver. It is typically called inside ca_pwr_init() to ensure the hardware is ready before any power-related configuration or status checks are performed. It acts as a mandatory prerequisite for any subsequent calls to functions that interact with the PWR peripheral, such as those managing standby modes or power baseline snapshots.

Esempio:

```
/* Initialize the power management system during boot */
void system_power_setup(void) {
    /* Enable the PWR peripheral clock first */
    ca_pwr_hw_init();

    /* Now it is safe to initialize the power driver */
    if (ca_pwr_init() != 0) {
        /* Handle initialization error */
        ca_printf("Failed to init power driver\n");
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_pwr_hw_init() in my caOS application?
A: You must call ca_pwr_hw_init() during the early system initialization phase, specifically before the RTOS scheduler starts. It is intended to be called inside ca_pwr_init() to ensure the PWR peripheral clock is enabled before any power-related configuration or status checks are performed. It is not thread-safe and must not be called from an ISR.

Code example conf:0.90 **Q:** How do I correctly initialize the power management hardware in my startup code?
A: You should call the initialization function during the boot sequence before starting the scheduler. Example: void system_init(void) { ca_pwr_hw_init(); /* Proceed with other power configurations */ }

HW differences conf:1.00 **Q:** Are there differences in how ca_pwr_hw_init() handles clock enabling across STM32 families?
A: Yes. On F1, F4, and F7 families, the function enables the PWR clock via the RCC->APB1ENR register. On the U5 family, it targets the RCC->AHB3ENR register. Conversely, on H5 and H7 families, the PWR block is always clocked by hardware, making the function a no-op, though it must still be called for cross-platform compatibility.

`ca_pwr_hw_lptim_arm`

Header: `caos/drivers/inc/ca_glue_hw.h:197`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_pwr_hw_lptim_arm(u32 ms); /* arma LPTIM1 one-shot su LSI come wake da Stop (millisecondi); false se non supportato */
```

Configures the Low Power Timer (LPTIM1) to wake the MCU from a low-power Stop mode after a specified duration.

Parametri:

Nome	Tipo	Descrizione
`ms`	`u32`	Duration in milliseconds. Valid range is 1 to 262144. Values outside this range or 0 will cause the function to return false.

Ritorna:
Returns true if the LPTIM1 was successfully configured and armed. Returns false if the duration is invalid, the hardware does not support this feature, or the LSI clock fails to stabilize.

Note:

- This function is non-blocking and does not put the MCU into sleep mode itself; it only prepares the hardware timer to trigger an interrupt that will wake the system later. It is not thread-safe and should be called from a single task context before entering a power-saving state. It is not intended to be called from an ISR. A common mistake is assuming this function puts the system to sleep; it only 'arms' the wake-up source. You must ensure the LSI clock is stable and the LPTIM1 peripheral is correctly clocked before calling this. On some architectures, this function also configures EXTI lines to ensure the wake-up event reaches the CPU core.

Uso:

- This function is used in power management sequences where the system needs to enter a low-power state (like Stop mode) and wake up automatically after a specific timeout. It is typically called immediately before invoking a power-down function like ca_pwr_enter. It is often paired with other wake-up source configurations, such as enabling external pins via ca_exti_enable, to provide multiple wake-up triggers. After the system wakes up, the LPTIM1 state is usually managed by the system's power recovery logic.

Esempio:

```
/* Prepare the system to wake up after 5000ms (5 seconds) */
if (ca_pwr_hw_lptim_arm(5000)) {
    /* LPTIM1 is armed, now enter low power mode */
    /* ca_pwr_enter(CA_PWR_MODE_STOP); // Hypothetical power entry function */
} else {
    /* Handle error: LPTIM1 could not be armed */
    ca_printf("Failed to arm LPTIM1 wake-up timer\n");
}
```

Q&A — fine-tuning review *7/7 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_pwr_hw_lptim_arm in my power management routine?
A: You should call it immediately before invoking a power-down function like ca_pwr_enter. It is used to configure the LPTIM1 as a wake-up source to automatically exit Stop mode after a specified duration.

Code example conf:1.00 **Q:** Provide a minimal example of how to use ca_pwr_hw_lptim_arm to wake the system in 500ms.
A: u32 timeout = 500U; if (ca_pwr_hw_lptim_arm(timeout)) { ca_pwr_enter(); }

Parameters conf:1.00 **Q:** What are the valid constraints for the 'ms' parameter in ca_pwr_hw_lptim_arm?
A: The 'ms' parameter represents the duration in milliseconds. The valid range is 1 to 262144. Any value outside this range or 0 will cause the function to return false.

Return conf:1.00 **Q:** What does a return value of false indicate when calling ca_pwr_hw_lptim_arm?
A: A return value of false indicates that the configuration failed. This happens if the duration is invalid (0 or > 262144), if the hardware does not support LPTIM wake-up, or if the LSI clock fails to stabilize.

Error/Edge case conf:1.00 **Q:** What is a common mistake developers make when using ca_pwr_hw_lptim_arm
A: A common mistake is assuming that this function puts the system to sleep. It does not; it only 'arms' the hardware timer to trigger an interrupt that will wake the system later. You must still call a separate power-down function.

Behavior conf:1.00 **Q:** Is ca_pwr_hw_lptim_arm thread-safe or safe to call from an ISR?
A: No, the function is not thread-safe and should be called from a single task context. It is not intended to be called from an ISR. Additionally, it is non-blocking.

HW differences conf:1.00 **Q:** How does the implementation of ca_pwr_hw_lptim_arm differ across STM32 families?
A: On F1xx and F4xx, the function is not implemented and returns false. On F7xx and H7xx, it configures EXTI lines (e.g., EXTI->RTSR or EXTI_D1->IMR2) to ensure the wake-up event reaches the core. On H5xx and U5xx, it requires setting the DBP bit in the power domain to access the backup domain (BDCR) for LSI configuration and uses specific peripheral clock registers like APB3ENR.

`ca_pwr_hw_lptim_disarm`

Header: `caos/drivers/inc/ca_glue_hw.h:198`
GUARD: `EN_CA_GLUE_HW`

```
1 extern void ca_pwr_hw_lptim_disarm(void); /* ferma LPTIM1 e disabilita l
    'IRQ dopo il wake */
```

Disarms the Low Power Timer (LPTIM1) and disables its associated interrupt after a wake-up event.

Ritorna:
void

Note:

- This function is non-blocking and performs direct hardware register manipulation to stop the LPTIM1 peripheral. It is not thread-safe by itself, so ensure it is called within a critical section or during a controlled power-state transition. It is safe to call from an ISR, but it is primarily intended for the power management state machine. A common mistake is calling this function when the LPTIM1 clock is not enabled, though the function includes a safety check to return early in that case.

Uso:

- This function is used during the power management wake-up sequence to clean up the LPTIM1 configuration after it has successfully triggered a system wake. It is typically paired with a preceding 'arm' function (not shown here) that configures the timer for a specific duration. You should call this immediately after the system resumes from a low-power mode to ensure the timer does not continue to trigger interrupts or consume unnecessary power.

Esempio:

```c
/* Example: Handling wake-up from low power mode */
void handle_system_wake(void) {
    /* Check if the wake-up was triggered by the LPTIM peripheral */
    /* Assuming a hypothetical bitmask check for wake sources */
    bool lptim_wake = true;

    if (lptim_wake) {
        /* Stop the timer and clean up hardware registers */
        ca_pwr_hw_lptim_disarm();

        /* Log the event using the system logger */
        ca_printf("System woke up from LPTIM event\n");
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_pwr_hw_lptim_disarm in my power management routine?
A: You should call this function immediately after the system resumes from a low-power mode. Its purpose is to clean up the LPTIM1 configuration, stop the timer, and disable its associated interrupt to prevent further triggers or unnecessary power consumption.

Code example conf:0.90 **Q:** How do I correctly invoke the disarm function within my power management state machine?
A: Since the function is not thread-safe, ensure it is called within a critical section or a controlled power-state transition. Example: { void handle_wake(void) { /* ... system wake logic ... */ ca_pwr_hw_lptim_disarm(); } }

HW differences conf:1.00 **Q:** Are there differences in how ca_pwr_hw_lptim_disarm handles hardware registers across STM32 families?
A: Yes. On F1xx and F4xx, LPTIM wake is not implemented. On F7xx and H7xx, the function explicitly clears EXTI registers (EXTI->IMR or EXTI_D1->IMR2). On U5xx, it additionally clears the RCC_SRDAMR_LPTIM1AMEN bit in the RCC_SRDAMR register. All implementations perform a safety check on the relevant RCC clock enable register before proceeding.

`ca_pwr_hw_max_freq`

Header: `caos/drivers/inc/ca_glue_hw.h:189`
GUARD: `EN_CA_GLUE_HW`

```c
extern u32 ca_pwr_hw_max_freq(void); /* ritorna SYSCLK nominale della
   scheda in Hz */
```

Returns the maximum nominal system clock frequency of the board in Hertz.

Ritorna:

A 32-bit unsigned integer representing the maximum frequency in Hz (e.g., 168000000 for 168MHz). This value is constant for a specific hardware target.

Note:

- This function is non-blocking, thread-safe, and can be safely called from an ISR because it simply returns a pre-defined constant value. It does not interact with hardware registers or perform any I/O, so there are no complex preconditions or initialization requirements. The most common mistake is assuming this function returns the *current* operating frequency; it does not. If you need the current frequency, use ca_os_clock_get_freq instead.

Uso:

- This function is primarily used for diagnostic and power management reporting. It is typically called when building a system status report to compare the current clock speed against the hardware's maximum capability. It is often paired with ca_pwr_get_info to populate structures that describe the power state of the system, allowing developers to verify if the CPU is currently throttled or running at full performance.

Esempio:

```c
/* Example: Reporting system clock status */
#include "caos.h"

void report_system_status(void) {
    u32 current_freq = ca_os_clock_get_freq();
    u32 max_freq = ca_pwr_hw_max_freq();

    /* Print the current vs max frequency to the system log */
    ca_printf("System Clock: %lu Hz (Max: %lu Hz)\n", current_freq, max_freq);

    /* Check if the system is running at reduced speed */
    if (current_freq < max_freq) {
        ca_printf("Warning: System is currently throttled.\n");
    }
}
```

Q&A — fine-tuning review *3/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_pwr_hw_max_freq in my application?
A: You should use this function for diagnostic and power management reporting. It is typically used to compare the current operating frequency against the hardware's maximum capability, often in conjunction with ca_pwr_get_info to determine if the CPU is currently throttled.

Code example conf:0.90 **Q:** How do I implement a simple check to see if my system is running at its maximum rated frequency?
A: You can compare the result of ca_pwr_hw_max_freq with the current frequency obtained via ca_os_clock_get_freq. Example: u32 max_f = ca_pwr_hw_max_freq(); u32 cur_f = ca_os_clock_get_freq(); if (cur_f < max_f) { /* System is throttled */ }

Return conf:1.00 **Q:** What does the return value of ca_pwr_hw_max_freq represent, and can it fail?
A: It returns a 32-bit unsigned integer representing the maximum nominal system clock frequency of the board in Hertz. It cannot fail, as it returns a pre-defined constant value and does not interact with hardware registers or perform I/O.

`ca_pwr_hw_periph_table`

Header: `caos/drivers/inc/ca_glue_hw.h:198`
GUARD: `EN_CA_GLUE_HW`

```c
extern const ca_pwr_periph_t *ca_pwr_hw_periph_table(u32 *count); /* RCC peripheral table of the family */
```

Retrieves the hardware-specific table of power-managed peripherals and the total count of entries.

Parametri:

Nome	Tipo	Descrizione
`count`	`u32 *`	Pointer to a variable where the function will store the number of peripherals found in the table. Must not be NULL.

Ritorna:
A pointer to a constant array of ca_pwr_periph_t structures. It always returns the base address of the static table defined for the specific STM32 family.

Note:

- This function is non-blocking and thread-safe because it simply returns a pointer to a static, read-only memory area defined at compile-time. It is safe to call from an ISR because it performs no I/O, locking, or complex logic. The most common mistake is passing a NULL pointer for the 'count' parameter, which will cause a hard fault when the function attempts to dereference it to write the table size. Always ensure the pointer passed to 'count' points to a valid memory location.

Uso:

- This function is primarily used by the power management subsystem to iterate through all peripherals that support clock gating or power control. It is typically used during system initialization or when performing a power-state snapshot to determine which peripherals are currently active. Developers use it to build loops that check the status of RCC (Reset and Clock Control) registers against the masks defined in the returned table, allowing the system to save or restore peripheral states efficiently. It is often paired with internal power management logic to ensure the system enters low-power modes only when safe.

Esempio:

```c
void check_active_peripherals(void) {
    u32 count = 0;
    /* Retrieve the hardware-specific peripheral table */
    const ca_pwr_periph_t *tbl = ca_pwr_periph_table(&count);

    if (tbl != NULL && count > 0) {
        for (u32 i = 0; i < count; i++) {
            /* Check if the peripheral clock is enabled using the register and mask */
            /* Note: tbl[i].enr is the register address, tbl[i].mask is the bitmask */
            if (*tbl[i].enr & tbl[i].mask) {
                /* Peripheral is active, perform necessary logging or power management */
                ca_printf("Peripheral %d is currently enabled\n", i);
            }
        }
    }
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_pwr_hw_periph_table function in my caOS application?

A: You should use this function when you need to iterate through all peripherals that support clock gating or power control. It is typically used during system initialization or when performing a power-state snapshot to check RCC register status against defined masks, helping to ensure the system enters low-power modes safely.

Code example conf:1.00 **Q:** How do I correctly call ca_pwr_hw_periph_table to iterate over the peripheral table?

A: You must provide a valid pointer to a u32 variable to store the count. Example: u32 count; const ca_pwr_periph_t *table = ca_pwr_hw_periph_table(&count); for(u32 i = 0; i < count; i++) { /* access table[i] */ }

Parameters conf:1.00 **Q:** What happens if I pass a NULL pointer to the count parameter of ca_pwr_hw_periph_table?

A: Passing a NULL pointer for the 'count' parameter will cause a hard fault. The function attempts to dereference this pointer to write the table size, so you must always ensure it points to a valid memory location.

Return conf:1.00 **Q:** What does the return value of ca_pwr_hw_periph_table represent, and is it safe to call from an ISR?

A: The function returns a pointer to a constant, static array of ca_pwr_periph_t structures defined at compile-time for the specific STM32 family. It is safe to call from an ISR because it is non-blocking, thread-safe, and performs no I/O or locking.

HW differences conf:0.90 **Q:** Are there significant differences in how ca_pwr_hw_periph_table behaves across different STM32 families like F1xx or H7xx?

A: The function's logic is consistent across all supported families (f1xx, f4xx, f7xx, h5xx, h7xx, u5xx); it always returns the base address of a family-specific static table and calculates the count based on that table's size. While the underlying peripheral definitions in the tables vary by family, the interface and behavior of this function remain identical.

`ca_pwr_hw_standby_resumed`

Header: `caos/drivers/inc/ca_glue_hw.h:191`
GUARD: `EN_CA_GLUE_HW`

```
1 extern bool ca_pwr_hw_standby_resumed(void); /* true se il boot proviene
      da Standby; azzera il flag SBF */
```

Checks if the system just woke up from a hardware Standby power state and clears the status flag.

Ritorna:
Returns true if the system was in Standby mode and has just resumed; returns false if it was a normal power-on reset or if the flag was already cleared.

Note:

- This function is non-blocking and executes quickly by reading hardware registers. It is thread-safe in the sense that it performs an atomic-like read-and-clear operation on the power status register. It is safe to call from an ISR, though it is typically used during system initialization. The most common mistake is calling this function multiple times; because it clears the Standby Flag (SBF) upon the first read, subsequent calls will return false, potentially causing logic errors if your application expects to know the boot source later in the execution flow.

Uso:

- This function is used during the early stages of system boot to determine if the current execution is a 'cold boot' or a 'wake-up from standby'. It is typically called inside ca_pwr_init

to store the state in a static variable for later use by the application. By checking this, the system can decide whether to perform a full hardware re-initialization or a faster 'resume' sequence, such as restoring specific peripheral states or skipping long calibration routines that are not needed after a standby wake-up.

Esempio:

```c
/* Global variable to store the boot source */
static bool s_is_standby_wake = false;

void system_startup_logic(void) {
    /* Initialize power management and check boot source */
    ca_pwr_init();

    /* Check if we are resuming from standby */
    s_is_standby_wake = ca_pwr_hw_standby_resumed();

    if (s_is_standby_wake) {
        /* Perform fast resume: skip full sensor calibration */
        ca_printf("System resumed from Standby mode.\n");
    } else {
        /* Perform cold boot: full hardware initialization */
        ca_printf("Cold boot detected.\n");
    }
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to call ca_pwr_hw_standby_resumed in a caOS application?
A: It should be called during the early stages of system boot, typically inside ca_pwr_init. The result should be stored in a static variable so the application can later decide whether to perform a full hardware initialization or a faster resume sequence, such as skipping long calibration routines.

Code example conf:1.00 **Q:** Provide a minimal C example showing how to store the standby status using caOS types.
A: static bool is_standby_wake = false; void system_init(void) { is_standby_wake = ca_pwr_hw_standby_resumed(); if (is_standby_wake) { /* Perform fast resume */ } else { /* Perform cold boot init */ } }

Return conf:1.00 **Q:** What do the return values of ca_pwr_hw_standby_resumed indicate?
A: It returns true if the system just woke up from a hardware Standby power state. It returns false if the boot was a normal power-on reset or if the Standby Flag (SBF) was already cleared by a previous call.

Error/Edge case conf:1.00 **Q:** What is the most common mistake when using ca_pwr_hw_standby_resumed?
A: The most common mistake is calling the function multiple times. Because the function performs an atomic-like read-and-clear operation on the hardware status flag, subsequent calls will return false, which can cause logic errors if the application expects to check the boot source later in the execution flow.

HW differences conf:1.00 **Q:** How does the implementation of ca_pwr_hw_standby_resumed differ across STM32 families?
A: The function targets different hardware registers depending on the MCU family: F1xx uses PWR->CSR/CR, F4xx uses PWR->CSR/CR, F7xx uses PWR->CSR1/CR1, H5xx uses PWR->PMSR/PMCR, H7xx uses PWR->CPUCR, and U5xx uses PWR->SR. Additionally, the F4xx implementation includes an optional debug feature (EN_PWR_DBG) that toggles an LED on PA7 if a standby wake-up is detected.

`ca_pwr_hw_stop_wake_nvic`

Header: `caos/drivers/inc/ca_glue_hw.h:196`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_pwr_hw_stop_wake_nvic(u32 wake_flags); /* riabilita nel
   NVIC le sole IRQ di wake (RTC/LPTIM) dopo il mask-all pre-Stop */
```

Re-enables specific wake-up interrupt lines in the NVIC before entering a low-power Stop mode.

Parametri:

Nome	Tipo	Descrizione
`wake_flags`	`u32`	Bitmask defining which peripherals (e.g., CA_PWR_WAKE_RTC, CA_PWR_WAKE_LPTIM) are allowed to wake the CPU from Stop mode. Invalid flags are ignored.

Ritorna:
void

Note:

- This function is non-blocking and executes extremely quickly as it only performs register writes to the NVIC. It is thread-safe in the sense that it only modifies NVIC registers, but it must be called in a specific sequence: after all other interrupts have been masked and immediately before the CPU enters the low-power state. It is not intended to be called from an ISR, as it is part of the power management transition logic. The most common beginner mistake is calling this function while other interrupts are still active, which can lead to the CPU waking up immediately or failing to enter the low-power state correctly.

Uso:

- This function is used exclusively within the power management subsystem to prepare the hardware for a low-power sleep cycle. It is typically paired with a sequence that disables all system interrupts, calls this function to selectively re-enable only the wake-up sources, and then invokes the low-power entry function. After the CPU wakes up, the system must immediately restore the original NVIC interrupt configuration to resume normal operation.

Esempio:

```
/* Prepare the system to enter Stop mode, allowing only RTC to wake it up */

/* 1. Mask all interrupts to prevent race conditions */
/* (Assume custom code to save NVIC state and disable all IRQs) */

/* 2. Re-enable only the RTC wake-up interrupt in the NVIC */
ca_pwr_hw_stop_wake_nvic(CA_PWR_WAKE_RTC);

/* 3. Enter the low-power state */
/* (Assume custom code to set PWR registers and execute WFI instruction) */

/* 4. Upon wake-up, restore the original NVIC interrupt configuration */
/* (Assume custom code to restore saved NVIC state) */
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_pwr_hw_stop_wake_nvic in my power management sequence?

A: It must be called after all other system interrupts have been masked and immediately before the CPU enters the low-power Stop mode. It is used to selectively re-enable only the specific wake-up sources in the NVIC. After the CPU wakes up, you must immediately restore the original NVIC interrupt configuration to resume normal operation.

Code example conf:0.90 **Q:** Provide a minimal example of how to use ca_pwr_hw_stop_wake_nvic.
A: u32 flags = CA_PWR_WAKE_RTC | CA_PWR_WAKE_LPTIM; /* Mask other interrupts here */ ca_pwr_hw_stop_wake_nvic(flags); /* Enter low-power mode here */

Parameters conf:1.00 **Q:** What does the wake_flags parameter control and what happens if I pass an invalid flag?
A: The wake_flags parameter is a u32 bitmask that defines which peripherals are allowed to wake the CPU from Stop mode, such as CA_PWR_WAKE_RTC or CA_PWR_WAKE_LPTIM. Any invalid flags passed to the function are ignored.

HW differences conf:1.00 **Q:** How does the implementation of ca_pwr_hw_stop_wake_nvic differ across STM32 families?
A: The implementation varies based on available hardware: F1xx and F4xx only support CA_PWR_WAKE_RTC (F4 ignores LPTIM flags). F7xx adds support for LPTIM1_IRQn. H5xx, H7xx, and U5xx support RTC, LPTIM1, and optionally LPUART1 if the EN_LPUART macro is enabled.

`ca_pwr_hw_wake_pin_enable`

Header: `caos/drivers/inc/ca_glue_hw.h:195`
GUARD: `EN_CA_GLUE_HW`

```
1 extern void ca_pwr_hw_wake_pin_enable(void); /* abilita il pin WKUP1
     come sorgente di risveglio */
```

Enables the WKUP1 pin (typically PA0) as a hardware wake-up source for the MCU from low-power modes.

Ritorna:
void

Note:

- This function is not thread-safe and modifies hardware registers directly. It is intended to be called before entering a low-power state (e.g., Sleep or Stop mode) to ensure the hardware is configured to detect an external signal on the WKUP1 pin. It is not designed to be called from an ISR, as it performs low-level register manipulation that should be handled during the system configuration phase. A common mistake is assuming this function automatically configures the GPIO pin mode; you must ensure the GPIO is correctly initialized as an input before calling this, otherwise, the wake-up event may not be triggered.

Uso:

- This function is used in power management sequences where the system needs to remain in a low-power state until an external physical event occurs. It is typically paired with ca_pwr_enter to transition the MCU into a sleep state. The standard workflow is: initialize the GPIO pin as an input, call ca_pwr_hw_wake_pin_enable to arm the hardware logic, and then invoke the power-down function to halt the CPU until the pin state changes.

Esempio:

```c
/* Configure PA0 as input before enabling wake-up */
/* ca_gpio_configure(PORT_A, 0, CA_GPIO_MODE_INPUT); */

/* Enable the hardware wake-up pin source */
ca_pwr_hw_wake_pin_enable();

/* Enter low power mode; the system will wake up on PA0 activity */
ca_pwr_enter(CA_PWR_MODE_SLEEP);
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_pwr_hw_wake_pin_enable in my power management workflow?
A: You should call this function before entering a low-power state (such as Sleep or Stop mode) to arm the hardware logic for detecting an external signal on the WKUP1 pin. It is typically paired with ca_pwr_enter. Note that you must manually initialize the GPIO pin as an input before calling this function, as it does not configure the GPIO mode itself.

Code example conf:0.90 **Q:** Provide a minimal example of how to prepare the MCU for wake-up using caOS.
A: First, ensure the GPIO is configured as an input. Then, call the wake-up enable function before entering the power-down state. Example: { /* 1. Initialize GPIO as input (platform specific) */ ca_pwr_hw_wake_pin_enable(); /* 2. Enter low power mode */ ca_pwr_enter(); }

HW differences conf:1.00 **Q:** How does the implementation of ca_pwr_hw_wake_pin_enable vary across different STM32 families?
A: The function targets different registers depending on the MCU family: F1xx and F4xx use PWR->CSR (PWR_CSR_EWUP), F7xx uses PWR->CSR2 (PWR_CSR2_EWUP1), H5xx uses PWR->WUCR (PWR_WUCR_WUPEN1), H7xx uses PWR->WKUPEPR (PWR_WKUPEPR_WKUPEN1), and U5xx uses PWR->WUCR1 (PWR_WUCR1_WUPEN1).

`ca_pwr_init`

Header: `caos/drivers/inc/pwr.h:64`
Source: `caos/drivers/src/pwr.c:49:60`
GUARD: `EN_PWR`

```c
int ca_pwr_init(void)
```

Initializes the power management subsystem and registers the power CLI commands.

Ritorna:
Always returns 0. This function is designed to be called during system startup and does not currently report error states.

Note:

- This function is not thread-safe and must be called during the system initialization phase, typically within ca_os_main before the scheduler starts. It performs hardware-level power initialization and takes a baseline snapshot of power consumption metrics. A common mistake is calling this function after the RTOS scheduler has started or from within an ISR, which can lead to undefined behavior or hardware conflicts. It is a non-blocking function that prepares the internal state for subsequent power-saving operations.

Uso:

- It is used as part of the system boot sequence to enable power management features. It is typically paired with ca_rtc_init to ensure wake-up sources are configured for low-power modes like Stop or Standby. If CLI support is enabled, it registers the 'pwr' command node, allowing developers to monitor or control power states via the console. It should be invoked early in the application lifecycle, usually alongside other hardware initialization functions like ca_temp_init or ca_cli_init.

Esempio:

```c
void ca_os_main(void) {
    /* Initialize CLI for system debugging */
    ca_cli_init(4096);
    ca_cli_set_sender(STDOUT);
    ca_cli_create_thread();

    /* Initialize power management and RTC for low-power support */
    ca_pwr_init();
    ca_rtc_init(CA_RTC_CLK_LSI);

    /* Now the system is ready to handle power commands via CLI */
    ca_printf("System initialized. Use 'pwr' commands to manage power.\r\n");

    /* Start application tasks */
    ca_os_job("main_task", CA_TH_LEVEL3, my_app_task, 0, 0);
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_pwr_init in my caOS application?
A: It should be called during the system initialization phase, typically within ca_os_main before the RTOS scheduler starts. It is intended to be part of the early boot sequence alongside other hardware initializations like ca_cli_init or ca_temp_init.

Code example conf:0.90 **Q:** How do I correctly initialize the power management subsystem in my main function?
A: You should invoke it before the scheduler starts as follows: int main(void) { ca_pwr_init(); ca_os_start(); return 0; }

Return conf:1.00 **Q:** What error codes should I handle when calling ca_pwr_init?
A: The function always returns 0. It is designed to be called during startup and does not currently report error states.

Error/Edge case conf:1.00 **Q:** What are the risks of calling ca_pwr_init after the scheduler has started?
A: Calling this function after the RTOS scheduler has started or from within an ISR is a common mistake that can lead to undefined behavior or hardware conflicts, as the function is not thread-safe.

`ca_pwr_periph_table`

Header: `caos/drivers/inc/pwr.h:75`
Source: `caos/drivers/src/pwr.c:62:64`
GUARD: `EN_PWR`

```c
const ca_pwr_periph_t *ca_pwr_periph_table(u32 *count)
```

Retrieves the system's power peripheral registry table and the count of registered hardware components.

Parametri:

Nome	Tipo	Descrizione
`count`	`u32 *`	Pointer to a 32-bit unsigned integer where the function will store the total number of power-managed peripherals found in the table.

Ritorna:
A pointer to a constant array of 'ca_pwr_periph_t' structures. Returns NULL if the internal table is uninitialized or empty.

Note:

- This function is non-blocking and thread-safe as it typically returns a pointer to a static read-only memory structure. It is safe to call from an ISR, provided the pointer returned is not used to modify the underlying hardware state directly. The most common beginner mistake is failing to check if the returned pointer is NULL before accessing the array, or passing a NULL pointer for the 'count' parameter, which will cause a hard fault when the function attempts to write the count value.

Uso:

- This function is primarily used by system-level power management tasks to iterate through all peripherals that require clock or power state adjustments during frequency scaling. It is typically called during system initialization or when 'ca_pwr_set_freq' is invoked to ensure all peripherals are correctly re-tuned to the new clock domain. Developers use it to build diagnostic tools or to perform bulk power-saving operations across the entire hardware abstraction layer.

Esempio:

```
u32 peripheral_count = 0;
const ca_pwr_periph_t *periph_list = ca_pwr_periph_table(&peripheral_count);

if (periph_list != NULL) {
    ca_printf("System power management tracking %u peripherals.\n",
    peripheral_count);
    for (u32 i = 0; i < peripheral_count; i++) {
        /* Access periph_list[i] to check peripheral status or configuration */
        /* Example: ca_printf("Peripheral ID: %u\n", periph_list[i].id); */
    }
} else {
    ca_printf("Error: Power peripheral table is empty or not initialized.\n");
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_pwr_periph_table function in my caOS application?
A: You should call this function when you need to perform system-level power management, such as iterating through peripherals to adjust clock or power states during frequency scaling. It is typically used during system initialization or when invoking ca_pwr_set_freq to ensure all hardware components are correctly re-tuned to the new clock domain.

Code example conf:1.00 **Q:** How do I correctly iterate through the power peripheral table using caOS types?
A: u32 count = 0; const ca_pwr_periph_t *table = ca_pwr_periph_table(&count); if (table != NULL) { for (u32 i = 0; i < count; i++) { /* Access table[i] safely */ } }

Parameters conf:1.00 **Q:** What is the purpose of the 'count' parameter, and what happens if I pass a NULL pointer?
A: The 'count' parameter is a pointer to a u32 variable where the function stores the total number of registered power-managed peripherals. Passing a NULL pointer for 'count' is

invalid and will cause a hard fault when the function attempts to write the count value to that address.

Return conf:1.00 **Q:** What does it mean if ca_pwr_periph_table returns NULL?
A: A NULL return value indicates that the internal power peripheral table is either uninitialized or empty. You must always check if the returned pointer is NULL before attempting to access the array to avoid memory access errors.

`ca_pwr_restore_freq`

Header: `caos/drivers/inc/pwr.h:88`
Source: `caos/drivers/src/pwr.c:78:86`
GUARD: `EN_PWR`

```
u32 ca_pwr_restore_freq(void)
```

Restores the system clock frequency and re-synchronizes all time-dependent peripherals.

Ritorna:
Returns a u32 value representing the restored system clock frequency in Hz.

Note:

- This function is not thread-safe and must not be called from an ISR. It performs a sequence of hardware re-initializations, including clock tree restoration and peripheral timing adjustments. Because it modifies global clock settings and peripheral registers, it is a blocking operation that assumes the system is in a state where clock reconfiguration is safe. A common mistake is calling this function while other tasks are actively using peripherals like UART or CAN, which can lead to data corruption or communication errors due to the sudden change in baud rates or timing parameters.

Uso:

- This function is typically used during power management transitions, specifically when waking the system from a low-power state or after a frequency scaling event. It is usually paired with ca_os_clock_reduce_freq, which is called before entering a low-power mode. The sequence is: first, call ca_os_clock_reduce_freq to lower power consumption, then perform the low-power task, and finally call ca_pwr_restore_freq to return the system to its operational frequency and re-align all communication interfaces.

Esempio:

```
/* Example: Restoring system frequency after a low-power sleep period */

void system_wakeup_routine(void) {
    /* Restore the system clock and re-tune peripherals */
    u32 current_freq = ca_pwr_restore_freq();

    /* Verify the frequency is as expected */
    if (current_freq > 0) {
        ca_printf("System restored to %u Hz\n", current_freq);
    } else {
        /* Handle error: clock restoration failed */
        // Log critical system error here
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_pwr_restore_freq and what is its primary purpose?
A: You should call ca_pwr_restore_freq when waking the system from a low-power state or after a frequency scaling event. Its purpose is to restore the system clock frequency and re-synchronize all time-dependent peripherals, ensuring that communication interfaces like UART or CAN are re-aligned to the operational frequency.

Code example conf:0.90 **Q:** How do I implement a power transition sequence using caOS functions?
A: To safely transition, you must first reduce the frequency, perform your task, and then restore it. Example: { ca_os_clock_reduce_freq(); /* Perform low-power task */ u32 freq = ca_pwr_restore_freq(); }

Return conf:0.95 **Q:** What does the u32 return value of ca_pwr_restore_freq represent, and are there specific failure codes?
A: The returned u32 value represents the restored system clock frequency in Hz. The provided documentation does not define specific error codes for failure, but notes that the function is blocking and assumes a safe state for reconfiguration; calling it while peripherals are active can lead to data corruption or communication errors.

`ca_pwr_set_freq`

Header: `caos/drivers/inc/pwr.h:82`
Source: `caos/drivers/src/pwr.c:68:76`
GUARD: `EN_PWR`

```
u32 ca_pwr_set_freq(u16 scale)
```

Adjusts the system clock frequency and synchronizes dependent peripherals like UART and CAN.

Parametri:

Nome	Tipo	Descrizione
`scale`	`u16`	A scaling factor representing the desired frequency reduction. Valid values are implementation-specific; invalid values may lead to undefined clock behavior.

Ritorna:
u32 representing the new system clock frequency in Hz after the adjustment.

Note:

- This function is not thread-safe and must not be called from an ISR. It performs a global system change, meaning it blocks execution while reconfiguring hardware clocks and peripheral baud rates. A common mistake is calling this while time-sensitive communication (like UART or CAN) is actively transmitting, which will cause data corruption or frame errors. Ensure the system is in a stable state before invoking this.

Uso:

- Used primarily in power management scenarios to reduce power consumption by lowering the CPU clock. It is typically paired with ca_pwr_restore_freq to return to full performance. It is part of the power management lifecycle: initialize via ca_pwr_init, adjust frequency as needed, and ensure all dependent peripherals are re-tuned to maintain communication integrity.

Esempio:

```c
/* Example: Reduce system frequency to save power */
u16 reduction_factor = 2;
u32 new_freq = ca_pwr_set_freq(reduction_factor);

/* Verify the new frequency */
if (new_freq > 0) {
    ca_printf("System frequency reduced to %u Hz\n", new_freq);
} else {
    /* Handle error: frequency adjustment failed */
}

/* Later, restore to original performance */
ca_pwr_restore_freq();
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_pwr_set_freq in my application?
A: You should use this function primarily for power management to reduce power consumption by lowering the CPU clock. It is typically used when the system needs to enter a low-power state and is intended to be paired with ca_pwr_restore_freq to return to full performance. Ensure the system is in a stable state and no time-sensitive communication is occurring before calling it.

Code example conf:0.90 **Q:** How do I implement a frequency reduction in caOS?
A: To reduce the frequency, call the function with a valid scaling factor. Example: u32 new_freq; new_freq = ca_pwr_set_freq(2);

Parameters conf:1.00 **Q:** What does the scale parameter represent and what are the constraints?
A: The scale parameter is a u16 scaling factor used to represent the desired frequency reduction. Valid values are implementation-specific, and using invalid values may lead to undefined clock behavior.

Return conf:0.90 **Q:** What does the return value of ca_pwr_set_freq indicate?
A: The function returns a u32 value representing the new system clock frequency in Hz after the adjustment has been applied. The documentation does not specify explicit error codes, but notes that invalid scaling factors can lead to undefined clock behavior.

`ca_qspi_command`

Header: `caos/drivers/inc/qspi.h:175`
Source: `caos/drivers/src/qspi.c:451:461`
GUARD: `EN_QSPI`

```c
bool ca_qspi_command(const ca_qspi_xfer_t *x, u32 timeout_ms)
```

Executes a raw QSPI command transaction with thread-safe access control.

Parametri:

Nome	Tipo	Descrizione
`x`	`const ca_qspi_xfer_t *`	Pointer to the transfer structure containing command, address, data, and mode settings. Must not be NULL.
`timeout_ms`	`u32`	Maximum time in milliseconds to wait for the operation to complete. If 0, defaults to 100ms.

Ritorna:
Returns true if the command was successfully executed, false if the QSPI driver is uninitialized, the input pointer is invalid, the transfer parameters are inconsistent, or the operation timed out.

Note:

- This function is blocking, meaning the calling task will wait until the QSPI operation finishes or the timeout expires. It is thread-safe because it uses an internal mutex to prevent concurrent access to the QSPI peripheral, but it must NEVER be called from an ISR (Interrupt Service Routine) as it will attempt to lock a mutex. Before calling this, ensure the QSPI peripheral has been initialized via ca_qspi_init. A common mistake is passing a NULL pointer or an uninitialized structure, which will immediately return false.

Uso:

- Use this function when you need to perform low-level QSPI operations that are not covered by higher-level functions like ca_qspi_read or ca_qspi_write. It is typically used in a sequence where you first prepare a ca_qspi_xfer_t structure, call ca_qspi_command to execute the transaction, and then process the resulting data. It is often used for sending custom opcodes to external flash memory or performing specific register configurations.

Esempio:

```
ca_qspi_xfer_t xfer;
ca_memzero(&xfer, sizeof(xfer));

// Configure the transfer structure
xfer.cmd = 0x05; // Example: Read Status Register
xfer.data_lines = 1;
xfer.len = 1;
xfer.read = 1;
xfer.rx = &status_reg_buffer;

// Execute the command with a 50ms timeout
if (ca_qspi_command(&xfer, 50)) {
    // Success: process status_reg_buffer
} else {
    // Handle error: log failure or retry
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_qspi_command instead of ca_qspi_read or ca_qspi_write?
A: Use ca_qspi_command for low-level QSPI operations not covered by higher-level functions, such as sending custom opcodes to external flash memory or performing specific register configurations.

Code example conf:0.90 **Q:** How do I structure a call to ca_qspi_command in caOS?
A: You must define a ca_qspi_xfer_t structure, populate it, and pass it to the function. Example: ca_qspi_xfer_t xfer = { .cmd = 0x05, .addr = 0x00 }; bool success = ca_qspi_command(&xfer, 500);

Parameters conf:1.00 **Q:** What happens if I pass 0 as the timeout_ms parameter?
A: If timeout_ms is 0, the function defaults to a 100ms timeout period.

Return conf:1.00 **Q:** What does a false return value from ca_qspi_command indicate?
A: A false return indicates that the QSPI driver is uninitialized, the input pointer is NULL, the transfer parameters are inconsistent, or the operation timed out.

Error/Edge case conf:1.00 **Q:** Can I call ca_qspi_command inside an interrupt service routine?
A: No, you must never call this function from an ISR because it attempts to lock an internal mutex, which is not permitted in an interrupt context.

`ca_qspi_erase_sector`

Header: `caos/drivers/inc/qspi.h:56`
Source: `caos/drivers/src/qspi.c:134:137`
GUARD: `EN_QSPI`

```
bool ca_qspi_erase_sector(u32 addr)
```

Erases a specific sector of the QSPI flash memory.

Parametri:

Nome	Tipo	Descrizione
addr	u32	The memory address within the sector to be erased. Must be a valid address aligned with the flash sector boundaries.

Ritorna:
Returns true if the erase operation was successfully initiated or completed, false if the address is invalid or the hardware is busy/unavailable.

Note:

- This function is typically a blocking operation, meaning the CPU will wait for the flash hardware to complete the erase cycle before returning. It is not safe to call this from an ISR (Interrupt Service Routine) because the long duration of a flash erase would block the system and cause timing issues. Ensure that ca_qspi_init has been called successfully before attempting to erase. A common mistake is passing an arbitrary address instead of a sector-aligned address, which may result in the hardware rejecting the command or erasing the wrong memory block.

Uso:

- This function is used when you need to clear a portion of external flash memory before writing new data, as flash memory must be erased before it can be programmed. It is typically used in a sequence: first, you call ca_qspi_init to prepare the hardware, then you call ca_qspi_erase_sector to clear the target area, and finally, you use ca_qspi_write to store your new data. It is often paired with ca_qspi_read to verify that the sector has been successfully cleared (i.e., all bytes read as 0xFF).

Esempio:

```c
/* Initialize QSPI hardware before use */
ca_qspi_init();

/* Define the sector address to clear */
u32 sector_addr = 0x08000000;

/* Erase the sector and check for success */
if (ca_qspi_erase_sector(sector_addr)) {
    /* Sector erased, now safe to write data */
    const u8 data[] = {0x01, 0x02, 0x03, 0x04};
    ca_qspi_write(sector_addr, data, sizeof(data));
} else {
    /* Handle error: log the failure using ca_printf */
    ca_printf("QSPI erase failed at address 0x%08X\n", sector_addr);
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_qspi_erase_sector() in a caOS application?
A: You should call ca_qspi_erase_sector() when you need to erase a 4KB sector of QSPI memory, typically before writing new data to that region. This function is useful for updating firmware, clearing configuration data, or managing persistent storage on QSPI flash devices. It is a blocking operation, so it's best used in contexts where blocking is acceptable, such as during initialization or in background tasks.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_qspi_erase_sector() in C with caOS types?
A: u32 sector_addr = 0x900000; // Example aligned address bool result = ca_qspi_erase_sector(sector_
if (result) { // Sector erased successfully } else { // Erasure failed }

Parameters conf:0.90 **Q:** What are the valid and invalid values for the addr parameter in ca_qspi_erase_sector()?
A: The addr parameter must be aligned to a 4KB sector boundary (i.e., a multiple of 4096). Invalid values include addresses that are not sector-aligned or addresses outside the valid QSPI memory range. Passing such invalid addresses will cause the function to return false.

Return conf:0.90 **Q:** What does ca_qspi_erase_sector() return, and what causes it to return false?
A: The function returns true if the sector erase operation completes successfully. It returns false if the operation fails, which can happen due to an invalid address (not aligned or out of range) or a hardware error during the erase process. The function does not provide specific error codes, so callers should check the return value and handle failures appropriately.

`ca_qspi_get_pin`

Header: `caos/drivers/inc/qspi.h:191`
Source: `caos/drivers/src/qspi.c:496:501`
GUARD: `EN_QSPI`

```
bool ca_qspi_get_pin(u8 pinmap, ca_qspi_pin_t sig, ca_gpio_port_t *port,
    ca_gpio_pin_t *pin)
```

Retrieves the GPIO port and pin mapping for a specific QSPI signal based on the hardware configuration.

Parametri:

Nome	Tipo	Descrizione
`pinmap`	`u8`	An identifier representing the specific hardware pin configuration board layout.
`sig`	`ca_qspi_pin_t`	The QSPI signal type (e.g., CLK, NCS, IO0) to look up. Must be less than CA_QSPI_PIN_COUNT.
`port`	`ca_gpio_port_t *`	Pointer to a variable where the GPIO port (e.g., GPIOA) will be stored. Must not be NULL.
`pin`	`ca_gpio_pin_t *`	Pointer to a variable where the GPIO pin number (e.g., CA_GPIO_PIN_0) will be stored. Must not be NULL.

Ritorna:
Returns true if the mapping was found and successfully written to the provided pointers; returns false if the signal index is invalid or if the provided pointers are NULL.

Note:

- This function is a non-blocking utility used to resolve hardware-specific pin assignments at runtime. It is thread-safe and can be called from any context, including ISRs, as it performs a simple lookup without waiting for hardware resources. A common mistake is passing a signal index greater than or equal to CA_QSPI_PIN_COUNT, which will cause the function to return false immediately. Always ensure the pointers passed for port and pin are valid memory addresses to avoid hard faults.

Uso:

- This function is typically used during the initialization phase of a QSPI driver or application to dynamically configure the GPIO pins before the QSPI peripheral is enabled. It is usually paired with ca_gpio_configure to apply the retrieved settings to the hardware. By using this function, you avoid hardcoding pin assignments, making your code portable across different board revisions that might use different pin mappings for the same QSPI signals.

Esempio:

```
ca_gpio_port_t port;
gpio_pin_t pin;

// Retrieve the mapping for the QSPI Clock signal
if (ca_qspi_get_pin(0, CA_QSPI_PIN_CLK, &port, &pin)) {
    // Configure the retrieved pin as an output for the QSPI clock
    ca_gpio_configure(port, pin, CA_GPIO_MODE_OUTPUT, CA_GPIO_OTYPE_PP,
    CA_GPIO_SPEED_LOW, CA_GPIO_NOPULL, 0);
} else {
    // Handle error: mapping not found or invalid signal
    ca_printf("Failed to get QSPI pin mapping!\n");
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_qspi_get_pin in my application?
A: You should use this function during the initialization phase of a QSPI driver or application to dynamically resolve hardware-specific pin assignments. It allows you to configure GPIO pins before enabling the QSPI peripheral, ensuring your code remains portable across different board revisions without hardcoding pin assignments.

Code example conf:0.90 **Q:** How do I implement a basic pin lookup for a QSPI clock signal?
A: ca_gpio_port_t port; ca_gpio_pin_t pin; u8 map = 0; if (ca_qspi_get_pin(map, CA_QSPI_PIN_CLK, &port, &pin)) { ca_gpio_configure(port, pin, ...); }

Parameters conf:1.00 **Q:** What are the constraints for the sig parameter in ca_qspi_get_pin?
A: The sig parameter represents the QSPI signal type (e.g., CLK, NCS, IO0) and must be a value less than CA_QSPI_PIN_COUNT. Passing a signal index greater than or equal to CA_QSPI_PIN_COUNT is a common mistake that will cause the function to return false.

Return conf:1.00 **Q:** What causes ca_qspi_get_pin to return false?
A: The function returns false if the provided signal index is invalid (greater than or equal to CA_QSPI_PIN_COUNT) or if either the port or pin pointers passed to the function are NULL.

`ca_qspi_hw_abort`

Header: `caos/drivers/inc/ca_glue_hw.h:216`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_qspi_hw_abort(void); /* stops whatever runs, interrupts
    off */
```

Immediately halts any ongoing QSPI hardware operation and resets the peripheral state.

Ritorna:
This function returns nothing (void).

Note:

- This function is a low-level hardware emergency stop. It disables all QSPI interrupts, triggers the hardware abort bit, and clears the status flags to ensure the peripheral is ready for a new command. It is not thread-safe and should be used with caution; it is typically called when a timeout occurs or a transfer fails. It is designed to be called from the main thread or error-handling routines, but it is not intended to be called from an ISR unless the system design explicitly guarantees no race conditions with the QSPI driver state.

Uso:

- This function is used as a safety mechanism to recover the QSPI peripheral when a transfer hangs or a timeout is reached. It is typically paired with a semaphore-based wait function (like ca_sem_timedwait) where, upon failure, the driver calls this function to force the hardware into a known idle state. After calling this, the driver usually clears internal software buffers and resets the state machine to prevent subsequent operations from using stale data.

Esempio:

```c
/* Example: Handling a QSPI timeout during a read operation */
#include "caos.h"

void safe_qspi_read(u8 *buffer, u32 size, u32 timeout_ms) {
    /* Attempt to read data using the QSPI driver */
    if (!ca_qspi_read(buffer, size)) {
        /* If the operation fails or times out, force hardware abort */
        ca_qspi_hw_abort();

        /* Log the error or perform recovery actions */
        ca_printf("QSPI operation failed, hardware aborted.\n");
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_qspi_hw_abort in my application?
A: You should call ca_qspi_hw_abort as a safety mechanism when a QSPI transfer hangs or a timeout is reached. It is typically used in conjunction with a semaphore-based wait function to force the peripheral into a known idle state after a failure, allowing you to clear internal software buffers and reset the state machine.

Code example conf:0.90 **Q:** How do I implement a basic error recovery routine using ca_qspi_hw_abort?
A: If a transfer fails, you should invoke the abort function to reset the hardware state. Example: void handle_qspi_error(void) { ca_qspi_hw_abort(); /* Reset internal driver state variables here */ }

HW differences conf:1.00 **Q:** Are there differences in how ca_qspi_hw_abort is implemented across STM32 families?
A: Yes. While the logic is consistent for F4xx, F7xx, H7xx, and F1xx (which use the QUADSPI peripheral and CR/FCR registers), the H5xx and U5xx families utilize the OCTOSPI1 peripheral and XSPI_CR_ABORT register bits. Additionally, the F1xx implementation is currently a stub that performs no hardware operations.

`ca_qspi_hw_get_pin`

Header: `caos/drivers/inc/ca_glue_hw.h:218`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_qspi_hw_get_pin(u8 pinmap, u8 sig, ca_gpio_port_t *port,
    ca_gpio_pin_t *pin); /* MCU pin of signal sig (ca_qspi_pin_t order)
    */
```

Retrieves the GPIO port and pin mapping for a specific QSPI signal based on the hardware configuration.

Parametri:

Nome	Tipo	Descrizione
`pinmap`	`u8`	Index of the pin configuration set (e.g., 0 for default, 1+ for alternate mappings). Must be less than QSPI_ALT_MAX.
`sig`	`u8`	The specific QSPI signal index (e.g., CLK, CS, IO0-IO3). Must be less than 6.
`port`	`ca_gpio_port_t *`	Pointer to a variable where the GPIO port (e.g., GPIOA) will be stored.
`pin`	`ca_gpio_pin_t *`	Pointer to a variable where the GPIO pin number (e.g., CA_GPIO_PIN_0) will be stored.

Ritorna:
Returns true if the mapping was found and parameters were updated, false if the signal index is invalid or the hardware configuration is missing.

Note:

- This function is non-blocking and thread-safe as it performs a simple lookup in a static configuration table. It is safe to call from an ISR because it does not perform any I/O operations or wait for hardware locks. The most common mistake is passing a 'sig' value greater than 5, which will cause the function to return false without modifying the output pointers. Ensure that the 'port' and 'pin' pointers are valid memory addresses before calling, otherwise the system will trigger a hard fault.

Uso:

- This function is typically used during the initialization phase of the QSPI driver to configure the physical pins before the peripheral is enabled. It is usually called by the higher-level 'ca_qspi_init' function to map the hardware signals to the specific MCU pins defined in the board support package. It acts as a bridge between the generic QSPI driver logic and the specific MCU pinout, ensuring that the software remains portable across different STM32 families.

Esempio:

```
ca_gpio_port_t port;
gpio_pin_t pin;

/* Attempt to retrieve the pin mapping for the QSPI Clock signal (sig 0) */
/* Assuming 0 is the default pinmap configuration */
if (ca_qspi_get_pin(0, 0, &port, &pin)) {
    /* Configure the retrieved port and pin as an alternate function */
    /* ca_gpio_configure(port, pin, ...); // Use appropriate GPIO config
    function */
} else {
    /* Handle error: invalid signal or pinmap index */
    ca_printf("Failed to retrieve QSPI pin mapping!\n");
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_qspi_hw_get_pin in my application?

A: This function is typically called during the initialization phase of the QSPI driver, often by the higher-level ca_qspi_init function. Its purpose is to map hardware signals to specific MCU pins defined in the board support package before the QSPI peripheral is enabled.

Code example conf:1.00 **Q:** How do I correctly call ca_qspi_hw_get_pin to retrieve pin information?
A: You must provide valid pointers for the port and pin variables to avoid a hard fault. Example: ca_gpio_port_t port; ca_gpio_pin_t pin; if (ca_qspi_hw_get_pin(0, 0, &port, &pin)) { /* Use port and pin */ }

Parameters conf:1.00 **Q:** What are the constraints for the 'sig' and 'pinmap' parameters?
A: The 'sig' parameter represents the QSPI signal index (e.g., CLK, CS, IO0-IO3) and must be less than 6. The 'pinmap' parameter represents the index of the pin configuration set and must be less than QSPI_ALT_MAX; if it is not, the implementation defaults to 0.

Return conf:1.00 **Q:** What does a return value of false indicate when calling ca_qspi_hw_get_pin?
A: A return value of false indicates that the signal index (sig) is invalid (i.e., 6 or greater) or that the hardware configuration is missing. In this case, the output pointers for port and pin are not modified.

HW differences conf:1.00 **Q:** Are there differences in how ca_qspi_hw_get_pin behaves across different STM32 families?
A: Yes. While families like F4, F7, H5, H7, and U5 implement the lookup logic using a static configuration table, the F1xx implementation is a stub that always returns false, indicating that QSPI hardware support is not implemented or available for that specific family in the provided driver code.

`ca_qspi_hw_init`

Header: `caos/drivers/inc/ca_glue_hw.h:210`
GUARD: `EN_CA_GLUE_HW`

```
1 extern bool ca_qspi_hw_init(const ca_qspi_cfg_t *cfg); /* initializes
     the QUADSPI peripheral with clock and flash size */
```

Initializes the hardware registers, clocks, and GPIOs for the Quad-SPI (QSPI) or Octo-SPI (XSPI) peripheral.

Parametri:

Nome	Tipo	Descrizione
`cfg`	`const ca_qspi_cfg_t *`	Pointer to a configuration structure containing flash size and clock divider settings. Must not be NULL.

Ritorna:
Returns true if the hardware was successfully initialized and the peripheral is ready for communication. Returns false if the hardware is unsupported on the current MCU or if the initialization sequence failed.

Note:

- This function is a low-level hardware driver and is not thread-safe; it should be called only once during system startup or driver initialization. It is not intended to be called from an ISR because it performs blocking operations, such as waiting for the peripheral to abort any pending transfers. Before calling this, ensure the system clock is configured, as the QSPI peripheral relies on specific bus frequencies. A common mistake is passing a

NULL pointer or failing to check the return value, which leads to undefined behavior when subsequent QSPI read/write operations are attempted on an uninitialized peripheral.

Uso:

- This function is the primary hardware setup routine for the QSPI subsystem. It is typically called internally by the higher-level ca_qspi_init function, which manages the driver state and CLI registration. In a typical application, you would define a ca_qspi_cfg_t structure with your specific flash parameters, pass it to ca_qspi_init, and then proceed to use ca_qspi_read or ca_qspi_write to interact with the external flash memory.

Esempio:

```
ca_qspi_cfg_t my_flash_cfg = { .flash_size = 23, .clk_div = 2 }; // 2^23 bytes = 8MB, div 2

if (ca_qspi_init(&my_flash_cfg)) {
    // QSPI is ready, proceed with memory operations
} else {
    // Handle initialization failure, perhaps log via ca_printf
    ca_printf("QSPI init failed!\n");
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_qspi_hw_init() in my caOS project?
A: You should call ca_qspi_hw_init() during system initialization, typically before any QSPI flash operations are needed. It is used to configure the hardware QSPI peripheral, including setting up the clock and flash size, so that subsequent QSPI communication can function correctly.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_qspi_hw_init() with caOS types?
A:

```
ca_qspi_cfg_t config = {
    .flash_size = 24,  // 16MB flash
    .clk_div = 2       // Clock divider
};
bool success = ca_qspi_hw_init(&config);
if (!success) {
    // Handle initialization failure
}
```

Parameters conf:0.90 **Q:** What does the cfg parameter in ca_qspi_hw_init() represent and what are valid values?
A: The cfg parameter is a pointer to a ca_qspi_cfg_t structure that holds configuration settings for the QSPI peripheral. It must be a valid, non-NULL pointer. The structure fields include flash_size (indicating flash memory size) and clk_div (clock divider value). Invalid values include NULL or improperly initialized structures.

Return conf:0.95 **Q:** What do the return values of ca_qspi_hw_init() indicate?
A: The function returns true if the QSPI peripheral was successfully initialized. It returns false if the cfg parameter is NULL or if the hardware initialization fails, such as due to unsupported chip or clock configuration issues.

Error/Edge case conf:0.90 **Q:** What are common error conditions or mistakes when using ca_qspi_hw_init()?
A: Common errors include passing a NULL pointer for cfg, which immediately returns false. Additionally, calling the function on unsupported STM32 families (like F1xx or F4xx) will return false. Also, if the hardware is not properly enabled or configured in the chip, initialization will fail.

Behavior conf:0.95 **Q:** Is ca_qspi_hw_init() thread-safe and can it be called from an ISR?

A: No, ca_qspi_hw_init() is not thread-safe and should not be called from an ISR. It is a blocking function that waits for hardware initialization to complete, which could cause scheduling issues or deadlocks if invoked from an interrupt context.

HW differences conf:0.85 **Q:** How does ca_qspi_hw_init() differ between STM32F7xx, H7xx, and H5xx families?

A: On F7xx and H7xx, the function configures registers like QUADSPI->DCR and QUADSPI->CR using standard QSPI registers. On H5xx, it uses OCTOSPI1->DCR1, OCTOSPI1->DCR2, and OCTOSPI1->CR with different register offsets and bit positions, reflecting the different peripheral architecture of the H5xx family.

---

ca_qspi_hw_mmap

Header: `caos/drivers/inc/ca_glue_hw.h:217`
GUARD: `EN_CA_GLUE_HW`

```
extern u32 ca_qspi_hw_mmap(bool on, const ca_qspi_xfer_t *rd, const
    ca_qspi_xfer_t *wr); /* memory-mapped (wr NULL = read-only); returns
    the base or 0 */
```

Configures the QSPI peripheral to map external flash/RAM directly into the CPU memory address space.

Parametri:

Nome	Tipo	Descrizione
`on`	`bool`	Set to true to enable memory-mapped mode, false to disable it and return to standard command mode.
`rd`	`const ca_qspi_xfer_t *`	Pointer to the transfer configuration structure for read operations. Must be provided if on is true.
`wr`	`const ca_qspi_xfer_t *`	Pointer to the transfer configuration structure for write operations. Pass NULL for read-only devices or to disable writes.

Ritorna:
Returns the base memory address (u32) where the QSPI device is mapped if successful, or 0 if disabled or if the configuration failed.

Note:

- This function is a low-level hardware driver call and is NOT thread-safe; it must be protected by a mutex or critical section if multiple tasks access the QSPI bus. It is strictly non-blocking but performs hardware register writes that immediately change the peripheral state. It cannot be called from an ISR because it relies on the hardware being in a stable state and may involve complex register sequences. The most common beginner mistake is failing to call ca_qspi_hw_abort() or disabling the mode before attempting to use standard ca_qspi_read/write functions, which will cause bus contention or hardware hangs.

Uso:

- This function is used to switch the QSPI peripheral from 'command mode' (used for individual read/write/erase operations) to 'memory-mapped mode' (where the flash appears as part of the CPU's memory map). It is typically paired with ca_qspi_init for setup and is invoked when the application needs to execute code directly from flash or read large data structures without manual command overhead. Always ensure the peripheral is returned

to a safe state by calling this with 'on = false' before attempting to perform standard block-based operations or re-initializing the bus.

Esempio:

```c
/* Example: Enabling memory-mapped mode for reading from external flash */
ca_qspi_xfer_t read_cfg = { .cmd = 0xEB, .dummy = 6 }; // Example configuration

/* Ensure we are in a safe state before switching modes */
ca_enter_critical();

u32 mmap_addr = ca_qspi_hw_mmap(true, &read_cfg, NULL);

if (mmap_addr != 0) {
    /* The flash is now accessible at the address returned by the driver */
    /* Example: volatile u8 *flash_ptr = (volatile u8 *)mmap_addr; */
} else {
    /* Handle error: failed to enable memory mapping */
}

/* Later, when finished with memory-mapped access */
ca_qspi_hw_mmap(false, NULL, NULL);
ca_exit_critical();
```

Q&A — fine-tuning review *7/7 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_qspi_hw_mmap in my caOS application?
A: Use this function when you need to switch the QSPI peripheral from command mode to memory-mapped mode. This allows the external flash or RAM to appear directly in the CPU's memory address space, enabling you to execute code from flash or read large data structures without manual command overhead.

Code example conf:1.00 **Q:** How do I initialize memory-mapped mode for a read-only flash device?
A: You must provide a valid read configuration structure and set the 'on' parameter to true. Example: ca_qspi_xfer_t rd = { .cmd = 0xEB, .dummy = 6 }; u32 base = ca_qspi_hw_mmap(true, &rd, NULL);

Parameters conf:1.00 **Q:** What is the purpose of the 'wr' parameter in ca_qspi_hw_mmap?
A: The 'wr' parameter provides the transfer configuration for write operations in memory-mapped mode. On some families like F4xx, F7xx, and H7xx, this is ignored as the mode is read-only. On H5xx and U5xx, providing a non-NULL 'wr' pointer allows the memory window to support writes.

Return conf:1.00 **Q:** What does the return value of ca_qspi_hw_mmap indicate?
A: It returns the base memory address (u32) where the QSPI device is mapped if the operation is successful. It returns 0 if the mode is disabled (on = false) or if the configuration failed.

Error/Edge case conf:1.00 **Q:** What is the most common mistake when using ca_qspi_hw_mmap?
A: The most common mistake is failing to call ca_qspi_hw_abort() or disabling memory-mapped mode (on = false) before attempting to use standard ca_qspi_read or ca_qspi_write functions, which leads to bus contention or hardware hangs.

Behavior conf:1.00 **Q:** Is ca_qspi_hw_mmap thread-safe or ISR-safe?
A: No, it is not thread-safe and must be protected by a mutex or critical section if multiple tasks access the QSPI bus. It cannot be called from an ISR because it relies on the hardware being in a stable state and performs complex register sequences.

HW differences conf:1.00 **Q:** How does the implementation of ca_qspi_hw_mmap differ between the F4xx and H5xx families?
A: On F4xx, the QUADSPI peripheral is restricted to read-only memory-mapped mode, ignoring the 'wr' parameter. On H5xx, the OCTOSPI1 peripheral supports write-enabled

memory mapping by configuring WCCR, WTCR, and WIR registers when a non-NULL 'wr' pointer is provided.

---

ca_qspi_hw_start_autopoll

Header: caos/drivers/inc/ca_glue_hw.h:215
GUARD: EN_CA_GLUE_HW

```
extern bool ca_qspi_hw_start_autopoll(u8 cmd, u8 mask, u8 match); /*
    polls a status byte until (sr & mask) == match, then IRQ */
```

Configures the QSPI hardware to automatically poll a status register until a specific bit pattern is matched.

Parametri:

Nome	Tipo	Descrizione
cmd	u8	The command byte (e.g., Read Status Register) to send to the external flash.
mask	u8	The bitmask applied to the received status byte to isolate the bits of interest.
match	u8	The expected value after applying the mask; the hardware triggers an interrupt when (received_byte & mask) == match.

Ritorna:
Returns true if the polling sequence was successfully initiated, false if the QSPI controller is busy or the operation cannot be started.

Note:

- This function is non-blocking; it merely configures the hardware registers and enables interrupts to start the polling process in the background. It is not thread-safe and must be called while the QSPI controller is in an idle state. A common beginner mistake is to call this function while a previous transfer is still in progress, which will cause the function to return false immediately. Ensure that the QSPI peripheral is properly initialized via ca_qspi_init before attempting to use this function.

Uso:

- This function is typically used to wait for long-running flash operations, such as sector erases or page program cycles, to complete without keeping the CPU in a busy-wait loop. It is usually paired with a subsequent call to a wait function (like a semaphore or event wait) that blocks the task until the QSPI interrupt handler signals that the polling condition has been met. After the hardware signals completion, the system can safely proceed to the next operation, such as reading or writing data.

Esempio:

```
/* Example: Waiting for a flash erase operation to complete */

// 1. Start the erase operation (assumed to be triggered previously)
// 2. Poll the Status Register (SR) until the Write-In-Progress (WIP) bit (bit
    0) is 0
if (ca_qspi_hw_start_autopoll(0x05U, 0x01U, 0x00U)) {
    /* Wait for the hardware to signal completion via an event */
    /* ca_event_wait(&qspi_event_handle, timeout_ms); */
} else {
    /* Handle error: QSPI busy or hardware failure */
}
```

Q&A — fine-tuning review *7/7 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_qspi_hw_start_autopoll in my application?
A: You should use this function to monitor long-running external flash operations, such as sector erases or page programming, without blocking the CPU. It allows the hardware to poll the status register in the background and trigger an interrupt once the specified bit pattern is matched.

Code example conf:0.90 **Q:** How do I implement a basic call to ca_qspi_hw_start_autopoll?
A: Ensure the peripheral is initialized first. Example: ca_qspi_init(); if (ca_qspi_hw_start_autopoll(0x
0x01, 0x00)) { /* Wait for interrupt signal */ }

Parameters conf:1.00 **Q:** What do the mask and match parameters represent in ca_qspi_hw_start_autopoll?
A: The mask is applied to the received status byte to isolate specific bits. The match is the expected value for those bits. The hardware triggers an interrupt when (received_byte & mask) == match.

Return conf:1.00 **Q:** What does it mean if ca_qspi_hw_start_autopoll returns false?
A: A return value of false indicates that the QSPI controller is currently busy or the operation could not be started. This often happens if you attempt to start a new operation while a previous transfer is still in progress.

Error/Edge case conf:1.00 **Q:** What is a common mistake when calling ca_qspi_hw_start_autopoll?
A: A common beginner mistake is calling this function while a previous QSPI transfer is still in progress. This will cause the function to return false immediately because the controller is not in an idle state.

Behavior conf:1.00 **Q:** Is ca_qspi_hw_start_autopoll thread-safe or blocking?
A: The function is non-blocking; it configures hardware registers and enables interrupts to perform the task in the background. It is not thread-safe and must be called while the QSPI controller is in an idle state.

HW differences conf:0.95 **Q:** Are there differences in how different STM32 families implement this function?
A: Yes. F4, F7, H7, and U5 families use the QUADSPI peripheral and configure the CCR register to start polling. In contrast, H5 and U5 (using OCTOSPI1) set the mode via set_mode() and trigger the polling by writing the command to the IR register.

`ca_qspi_hw_start_xfer`

Header: `caos/drivers/inc/ca_glue_hw.h:214`
GUARD: `EN_CA_GLUE_HW`

```
1 extern bool ca_qspi_hw_start_xfer(const ca_qspi_xfer_t *x); /* starts
     one operation; end -> ca_qspi_isr_done() */
```

Initiates a non-blocking QSPI hardware transfer operation.

Parametri:

Nome	Tipo	Descrizione
`x`	`const ca_qspi_xfer_t *`	Pointer to a structure containing transfer details: command, address, data buffers, lengths, and read/write mode. Must be valid and populated before calling.

Ritorna:
Returns true if the hardware was idle and the transfer was successfully triggered. Returns false if the hardware is busy or the operation cannot be started.

Note:

- This function is non-blocking; it merely configures the QSPI peripheral registers and enables interrupts to handle the data flow. It is not thread-safe and must be called only when the QSPI peripheral is in an idle state. It is not intended to be called directly from an ISR, as it relies on the system being in a state where it can safely manage hardware registers. A common beginner mistake is to call this function without checking if the previous operation has finished, or failing to provide a valid pointer to the transfer structure, which will lead to undefined behavior or hardware faults.

Uso:

- This function is the low-level trigger for QSPI operations. It is typically used within a driver wrapper that handles the synchronization logic. The standard sequence is: 1) Ensure the peripheral is idle, 2) Call ca_qspi_hw_start_xfer to initiate the hardware state machine, 3) Wait for the operation to complete using a synchronization primitive (like a semaphore or event), and 4) Call ca_qspi_isr_done inside the QSPI interrupt service routine to clear flags and signal completion.

Esempio:

```
ca_qspi_xfer_t xfer;
/* Initialize transfer structure with command, address, and buffer */
xfer.cmd = 0x03; /* Example Read command */
xfer.addr = 0x000000;
xfer.rx = buffer;
xfer.len = 16;
xfer.read = true;

/* Start the hardware transfer */
if (ca_qspi_hw_start_xfer(&xfer)) {
    /* Wait for completion using an event or semaphore */
    /* ca_event_wait(...) */
} else {
    /* Handle error: hardware busy or invalid parameters */
}
```

Q&A — fine-tuning review *7/7 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** What is the standard workflow for initiating a QSPI transfer in caOS?
A: The workflow is: 1) Verify the QSPI peripheral is idle, 2) Call ca_qspi_hw_start_xfer with a populated ca_qspi_xfer_t structure, 3) Wait for completion using a synchronization primitive, and 4) Call ca_qspi_isr_done within the QSPI interrupt service routine to clear flags and signal completion.

Code example conf:0.90 **Q:** Provide a minimal example of how to prepare and start a QSPI transfer.
A: ca_qspi_xfer_t xfer = { .cmd = 0x03, .addr = 0x00, .len = 16, .read = true, .data_lines = 1, .addr_lines = 1 }; if (ca_qspi_hw_start_xfer(&xfer)) { /* wait for completion */ }

Parameters conf:1.00 **Q:** What does the ca_qspi_xfer_t structure contain and what are the requirements for it?
A: The structure contains the command, address, data buffers, lengths, and read/write mode. It must be valid and fully populated before passing the pointer to ca_qspi_hw_start_xfer.

Return conf:1.00 **Q:** What do the return values of ca_qspi_hw_start_xfer indicate?
A: It returns true if the hardware was idle and the transfer was successfully triggered. It returns false if the hardware is busy or the operation cannot be started.

Error/Edge case conf:1.00 **Q:** What are common mistakes when using ca_qspi_hw_start_xfer?
A: Common mistakes include calling the function without checking if the previous operation has finished, failing to provide a valid pointer to the transfer structure, or calling it directly from an ISR, all of which can lead to undefined behavior or hardware faults.

Behavior conf:1.00 **Q:** Is ca_qspi_hw_start_xfer thread-safe or ISR-safe?
A: No, it is not thread-safe and is not intended to be called directly from an ISR. It is a non-blocking function that configures hardware registers and enables interrupts.

HW differences conf:1.00 **Q:** How does the implementation differ between MCU families like F4/F7/H7 and H5/U5?
A: F4, F7, and H7 families use the QUADSPI peripheral and configure the CCR register to initiate transfers. H5 and U5 families use the OCTOSPI1 peripheral, which requires setting the mode via set_mode, configuring the TCR register, and explicitly writing to the IR register to start the command.

`ca_qspi_init`

Header: `caos/drivers/inc/qspi.h:52`
Source: `caos/drivers/src/qspi.c:122:132`
GUARD: `EN_QSPI`

```
bool ca_qspi_init(const ca_qspi_cfg_t *cfg)
```

Initializes the QSPI hardware peripheral and registers the CLI interface for debugging.

Parametri:

Nome	Tipo	Descrizione
`cfg`	`const ca_qspi_cfg_t *`	Pointer to a configuration structure containing clock settings, mode, and pin definitions. Must not be NULL.

Ritorna:
Returns true if the hardware was successfully initialized and the configuration was applied; returns false if the pointer is NULL or the hardware initialization failed.

Note:

- This function is not thread-safe and should be called during the system initialization phase before any QSPI read/write operations are attempted. It is not intended to be called from an ISR. A common mistake is passing a NULL pointer or failing to check the return value, which would leave the QSPI peripheral in an uninitialized state, causing subsequent read/write operations to fail silently or trigger hardware faults.

Uso:

- This function is the mandatory entry point for using the QSPI peripheral. It is typically called once during system startup, immediately after the system clock is configured. Once initialized, the application can proceed to use ca_qspi_read, ca_qspi_write, or ca_qspi_erase_sector to interact with external flash memory. If the CLI is enabled, it automatically registers a command handler, allowing developers to inspect or debug the QSPI status via the console.

Esempio:

```
ca_qspi_cfg_t qspi_config = { .clock_prescaler = 2, .mode = 0 }; // Example
    config structure

if (ca_qspi_init(&qspi_config)) {
    // QSPI is ready, safe to perform operations
    // ca_qspi_write(address, data, size);
} else {
    // Handle initialization failure
    ca_printf("QSPI init failed!\n");
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_qspi_init() in my embedded application?
A: You should call ca_qspi_init() during the system initialization phase, typically before any QSPI communication is required. It is used to configure and enable the QSPI peripheral with a given configuration structure, making it ready for data transfers.

Code example conf:0.90 **Q:** Can you provide a minimal example of how to use ca_qspi_init() with caOS types?
A: ca_qspi_cfg_t config = { .clock_prescaler = 2, .fifo_threshold = 1 }; bool success = ca_qspi_init(&config); if (!success) { /* Handle error */ }

Parameters conf:0.95 **Q:** What does the cfg parameter in ca_qspi_init() represent, and what happens if it is NULL?
A: The cfg parameter is a pointer to a configuration structure for the QSPI peripheral. If cfg is NULL, the function will fail and return false, as it cannot proceed without a valid configuration.

Return conf:0.95 **Q:** What do the return values of ca_qspi_init() indicate, and what causes it to return false?
A: The function returns true if initialization is successful and the QSPI peripheral is ready for use. It returns false if the cfg parameter is NULL or if the underlying hardware initialization fails.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_qspi_init()?
A: Common mistakes include passing a NULL pointer as cfg, or not properly initializing the QSPI hardware configuration before calling the function. Additionally, calling this function from an ISR is not recommended and may cause synchronization issues.

`ca_qspi_isr_done`

Header: `caos/drivers/inc/qspi.h:139`
Source: `caos/drivers/src/qspi.c:75:80`
GUARD: `EN_QSPI`

```
1 void ca_qspi_isr_done(bool ok)
```

Signals the completion of a QSPI hardware operation from within an Interrupt Service Routine (ISR).

Parametri:

Nome	Tipo	Descrizione
`ok`	`bool`	Status of the operation: true if the QSPI transaction finished successfully, false if an error occurred.

Ritorna:
void

Note:

- This function is designed to be called exclusively from the QSPI hardware interrupt handler. It is non-blocking because it uses a semaphore post operation to wake up a waiting task. It is thread-safe in the context of the RTOS scheduler, as it triggers a context switch if a higher-priority task is waiting on the semaphore. A common mistake is calling this function from a standard task context instead of an ISR, or failing to initialize the semaphore before the interrupt is enabled, which would lead to a system crash.

Uso:

- This function is the bridge between the asynchronous hardware QSPI peripheral and the synchronous task-based application code. It is typically used in a sequence where a task initiates a QSPI command (e.g., ca_qspi_read), waits on a semaphore (ca_sem_wait), and the ISR calls ca_qspi_isr_done to release that semaphore once the hardware signals completion. Without this function, the application task would remain blocked indefinitely, waiting for an event that never signals completion.

Esempio:

```c
/* Global semaphore to synchronize QSPI operations */
static ca_sem_t s_done;

/* QSPI Interrupt Handler */
void QSPI_IRQHandler(void) {
    /* Check hardware status registers here */
    bool success = true; /* Assume success for example */

    /* Signal the waiting task that the hardware is done */
    ca_qspi_isr_done(success);
}

/* Task context usage */
void qspi_task(void) {
    /* Initialize semaphore before use */
    ca_sem_init(&s_done, 0);

    /* Start a read operation */
    ca_qspi_read(address, buffer, length);

    /* Wait for the ISR to signal completion */
    if (ca_sem_wait(&s_done) == 0) {
        /* Operation finished, safe to process buffer */
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_qspi_isr_done and what happens if I omit it?
A: You must call ca_qspi_isr_done exclusively from within the QSPI hardware interrupt handler to signal the completion of a transaction. If you omit this call, any application task waiting on the QSPI operation (via ca_sem_wait) will remain blocked indefinitely because the semaphore will never be released.

Code example conf:0.90 **Q:** How do I implement the QSPI interrupt handler using ca_qspi_isr_done?
A: The ISR should check the hardware status and pass the result to the function. Example: void QSPI_IRQHandler(void) { bool success = check_qspi_status(); ca_qspi_isr_done(success); }

Parameters conf:1.00 **Q:** What does the 'ok' parameter represent in ca_qspi_isr_done?
A: The 'ok' parameter is a boolean indicating the status of the QSPI transaction. A value of 'true' signifies that the hardware operation finished successfully, while 'false' indicates that an error occurred during the transaction.

`ca_qspi_mmap_disable`

Header: `caos/drivers/inc/qspi.h:186`
Source: `caos/drivers/src/qspi.c:488:494`
GUARD: `EN_QSPI`

```c
void ca_qspi_mmap_disable(void)
```

Disables the QSPI memory-mapped mode, returning the peripheral to standard command-based operation.

Ritorna:
None (void function).

Note:

- This function is blocking because it acquires a mutex to ensure thread-safe access to the QSPI peripheral configuration. It must not be called from an ISR (Interrupt Service Routine) because mutex operations are not permitted in interrupt context. Before calling this, the QSPI driver must have been successfully initialized via ca_qspi_init. A common mistake is calling this function while a DMA transfer or a direct memory access operation is still in progress, which can lead to undefined behavior or system instability.

Uso:

- This function is used to exit the memory-mapped mode (mmap) when the application needs to perform direct command-based operations on the QSPI flash, such as erasing sectors or writing data. It is typically paired with ca_qspi_mmap_enable, which is called to enter the mmap mode for high-speed read access. In a typical lifecycle, you enable mmap to read data, disable it to perform maintenance tasks like erasing or writing, and then re-enable it if further read access is required.

Esempio:

```
/* Example: Safely disabling mmap to perform a write operation */
void update_flash_data(void) {
    /* 1. Disable mmap to gain control over the QSPI peripheral */
    ca_qspi_mmap_disable();

    /* 2. Perform the write operation using the standard command interface */
    /* Note: ca_qspi_write is used here to send data to the flash */
    ca_qspi_write(0x00000000, my_data_buffer, sizeof(my_data_buffer));

    /* 3. Re-enable mmap to allow the application to read data directly from
    memory again */
    ca_qspi_mmap_enable();
}
```

Q&A — fine-tuning review *2/2 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_qspi_mmap_disable and why is it necessary?
A: You should call ca_qspi_mmap_disable when you need to perform command-based operations on the QSPI flash, such as erasing sectors or writing data. It is necessary because it exits the memory-mapped mode, allowing the peripheral to return to standard command-based operation. You must ensure no DMA transfers or direct memory access operations are in progress before calling it to avoid system instability.

Code example conf:0.95 **Q:** How do I safely disable the QSPI memory-mapped mode in my application?
A: Ensure the QSPI driver is initialized, then call the function directly. Note that this must not be called from an ISR as it uses a mutex. Example:
void perform_flash_maintenance(void) { /* Ensure QSPI is initialized before disabling mmap */ ca_qspi_init();
/* Disable mmap to perform command-based operations */ ca_qspi_mmap_disable();
/* Perform erase or write operations here */ }

ca_qspi_mmap_enable

Header: caos/drivers/inc/qspi.h:185
Source: caos/drivers/src/qspi.c:475:486
GUARD: EN_QSPI

```
u32 ca_qspi_mmap_enable(void)
```

Maps the external QSPI flash memory into the CPU's address space for direct read access.

Ritorna:
Returns the base memory address (u32) where the QSPI flash is mapped if successful; returns 0U if the mapping failed, the device is not ready, or the device is in RAW mode.

Note:

- This function is blocking because it uses a mutex to ensure thread-safe access to the QSPI controller configuration. It must NOT be called from an ISR because mutex locking is not permitted in interrupt context. Before calling this, the QSPI driver must be initialized via ca_qspi_init. A common mistake is assuming the memory is mapped immediately after initialization; you must call this function to enable the hardware memory-mapped mode before attempting to access the flash via pointers.

Uso:

- Use this function when you need to read large amounts of data from QSPI flash as if it were standard internal memory, which is significantly faster than using indirect read commands. It is typically paired with ca_qspi_mmap_disable to release the memory mapping when you need to perform write or erase operations that require the controller to be in indirect mode. Always check that the returned address is non-zero before casting it to a pointer to avoid accessing invalid memory regions.

Esempio:

```
/* Ensure QSPI is initialized before attempting to map */
ca_qspi_init();

/* Enable memory mapping to access flash via pointers */
u32 base_addr = ca_qspi_mmap_enable();

if (base_addr != 0U) {
    /* Access flash directly as memory */
    const volatile u8 *flash_ptr = (const volatile u8 *)base_addr;
    u8 data = flash_ptr[0]; /* Read first byte */
} else {
    /* Handle error: mapping failed */
}

/* Disable mapping before performing erase operations */
ca_qspi_mmap_disable();
/* Perform erase operation using ca_qspi_erase_sector(...) */
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_qspi_mmap_enable instead of standard indirect read commands?
A: You should use ca_qspi_mmap_enable when you need to read large amounts of data from QSPI flash as if it were standard internal memory. This approach is significantly faster than using indirect read commands. It is typically used when you want to access the flash directly via pointers.

Code example conf:1.00 **Q:** How do I correctly implement a memory-mapped read using caOS types?
A: First, ensure ca_qspi_init has been called. Then, call ca_qspi_mmap_enable and check the result before accessing the memory: u32 addr = ca_qspi_mmap_enable(); if (addr != 0U) { u8 *flash_ptr = (u8 *)addr; u8 data = flash_ptr[0]; }

Return conf:1.00 **Q:** What does the return value of ca_qspi_mmap_enable indicate?
A: The function returns the u32 base memory address where the QSPI flash is mapped if successful. It returns 0U if the mapping failed, the device is not ready, or the device is currently in RAW mode.

Error/Edge case conf:1.00 **Q:** Why is it dangerous to call ca_qspi_mmap_enable from an ISR?
A: This function is blocking because it uses a mutex to ensure thread-safe access to the QSPI controller configuration. Mutex locking is not permitted in an interrupt context, so calling this from an ISR will cause an error.

`ca_qspi_read`

Header: `caos/drivers/inc/qspi.h:64`
Source: `caos/drivers/src/qspi.c:145:149`
GUARD: `EN_QSPI`

```
1 bool ca_qspi_read(u32 addr, u8 *buf, u32 len)
```

Reads a block of data from the QSPI external flash memory at a specified address.

Parametri:

Nome	Tipo	Descrizione
`addr`	`u32`	The starting memory address in the QSPI flash to read from.
`buf`	`u8 *`	Pointer to the destination buffer where the read data will be stored. Must not be NULL.
`len`	`u32`	The number of bytes to read. Must be greater than 0.

Ritorna:
Returns true if the read operation was successful, false if the parameters were invalid (NULL buffer or zero length) or if the hardware operation failed.

Note:

- This function is a blocking call, meaning the task will wait until the QSPI hardware completes the data transfer before returning. It is generally not safe to call this from an ISR unless the underlying hardware driver explicitly supports it, which is typically not the case for QSPI. Ensure the QSPI peripheral has been initialized via ca_qspi_init before calling this function. A common beginner mistake is passing a buffer that is too small for the requested length, which will cause a buffer overflow and memory corruption.

Uso:

- This function is used to retrieve data stored in external flash, such as configuration parameters, firmware updates, or large data logs. It is typically used in a sequence: first, initialize the QSPI peripheral with ca_qspi_init, then perform reads as needed, and if the data is dynamic, use ca_memory_allocate to create a buffer of the appropriate size before calling ca_qspi_read. Always check the boolean return value to ensure the data was read correctly before attempting to process the contents of the buffer.

Esempio:

```c
/* Example: Reading configuration data from QSPI flash */
#include "caos.h"

void read_config_task(void) {
    u32 config_addr = 0x08000000; /* Example flash offset */
    u32 config_size = 256;
    u8 *config_buf = ca_memory_allocate(config_size);

    if (config_buf != NULL) {
        /* Attempt to read data from QSPI */
        if (ca_qspi_read(config_addr, config_buf, config_size)) {
            /* Data read successfully, process it */
        } else {
            /* Handle read error */
        }
        ca_memory_free(config_buf);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_qspi_read to read data from a QSPI memory?
A: You should use ca_qspi_read when you need to read data from a QSPI memory device in a blocking manner. It is suitable for scenarios where you require a simple, direct read operation without the overhead of asynchronous handling. This function is not suitable for use in interrupt service routines (ISRs) due to its blocking nature.

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_qspi_read with caOS types?
A: u8 buffer[256]; u32 address = 0x1000; bool success = ca_qspi_read(address, buffer, 64); if (success) { // Process the data in buffer }

Parameters conf:0.90 **Q:** What are the valid and invalid values for the 'addr' parameter in ca_qspi_read?
A: The 'addr' parameter must be a valid address within the QSPI memory range and must not be 0xFFFFFFFF. It should also be aligned according to the memory's addressing format. Any invalid address, including 0xFFFFFFFF, will cause the function to return false.

Return conf:0.95 **Q:** What does it mean when ca_qspi_read returns false?
A: ca_qspi_read returns false if the read operation fails. This can happen if the buffer pointer is NULL, the length is zero, or if the hardware operation fails. It is important to verify that the parameters are valid and that the QSPI device is accessible when false is returned.

`ca_qspi_read_id`

Header: `caos/drivers/inc/qspi.h:178`
Source: `caos/drivers/src/qspi.c:463:471`
GUARD: `EN_QSPI`

```c
bool ca_qspi_read_id(u8 id[3])
```

Reads the 3-byte JEDEC manufacturer and device ID from a connected QSPI NOR flash memory.

Parametri:

Nome	Tipo	Descrizione
`id`	`u8[3]`	A pointer to a buffer of at least 3 bytes where the JEDEC ID (Manufacturer ID, Memory Type, Capacity) will be stored. Must not be NULL.

Ritorna:
Returns true if the ID was successfully read from the flash memory, false if the QSPI is not initialized, the device is not a NOR flash, or the pointer is invalid.

Note:

- This function is blocking, meaning the calling task will wait until the QSPI transaction completes. It is thread-safe because it uses an internal mutex to prevent concurrent access to the QSPI peripheral. It must NOT be called from an ISR because it attempts to lock a mutex, which can lead to system deadlocks or crashes. Preconditions: ca_qspi_init must have been called successfully during system startup. A common mistake is passing a NULL pointer or a buffer smaller than 3 bytes, which will cause memory corruption or immediate failure.

Uso:

- This function is typically used during system diagnostic routines or at boot time to verify that the correct flash chip is detected and operational. It is often paired with CLI commands to report hardware status to the user. The typical sequence is: 1) Ensure ca_qspi_init has run, 2) Call ca_qspi_read_id to verify hardware, 3) Proceed with other operations like ca_qspi_read or ca_qspi_write if the ID matches expected values.

Esempio:

```
u8 jedec_id[3];

// Attempt to read the flash ID
if (ca_qspi_read_id(jedec_id)) {
    // Successfully retrieved ID, print it to the system console
    ca_printf("Flash ID: %02X %02X %02X\n", jedec_id[0], jedec_id
    [2]);
} else {
    // Handle error: flash might be missing or QSPI not ready
    ca_printf("Error: Could not read QSPI Flash ID\n");
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_qspi_read_id in my application?
A: It is typically used during system diagnostic routines or at boot time to verify that the correct flash chip is detected and operational. You must ensure ca_qspi_init has been called successfully before invoking this function.

Code example conf:1.00 **Q:** How do I implement a basic check for the QSPI flash ID?
A: u8 id[3]; if (ca_qspi_read_id(id)) { /* ID successfully read */ } else { /* Handle error */ }

Parameters conf:1.00 **Q:** What are the requirements for the id parameter in ca_qspi_read_id?
A: The parameter must be a pointer to a buffer of at least 3 bytes. It must not be NULL, as passing a NULL pointer or a buffer smaller than 3 bytes will cause memory corruption or immediate failure.

Return conf:1.00 **Q:** What does a false return value from ca_qspi_read_id indicate?
A: A false return indicates that the QSPI is not initialized, the device is not a NOR flash, or the provided pointer is invalid.

ca_qspi_write

Header: caos/drivers/inc/qspi.h:61
Source: caos/drivers/src/qspi.c:139:143
GUARD: EN_QSPI

```
bool ca_qspi_write(u32 addr, const u8 *buf, u32 len)
```

Writes a block of data to the QSPI flash memory at a specified address.

Parametri:

Nome	Tipo	Descrizione
addr	u32	The target memory address in the QSPI flash. Must be within the valid flash memory range.
buf	const u8 *	Pointer to the data buffer to be written. Must not be NULL.
len	u32	Number of bytes to write. Must be greater than 0.

Ritorna:
Returns true if the write operation was successful, false otherwise (e.g., invalid parameters or hardware failure).

Note:

- This function is a wrapper for hardware-specific QSPI operations. It is generally blocking, meaning the task will wait until the write operation completes, which is critical for data integrity. It is not intended to be called from an ISR because flash operations are slow and can cause significant latency. Ensure the QSPI peripheral has been initialized via ca_qspi_init before calling this. A common mistake is attempting to write to a sector that has not been erased; QSPI flash memory typically requires an explicit erase operation before new data can be written to a specific address.

Uso:

- This function is used to persist configuration data, logs, or firmware updates to external flash memory. The typical sequence involves calling ca_qspi_init to prepare the peripheral, followed by ca_qspi_erase_sector to clear the target memory area, and finally ca_qspi_write to commit the data. It is often paired with ca_qspi_read to verify the written data or retrieve it later during system boot.

Esempio:

```
/* Initialize QSPI peripheral before use */
ca_qspi_init();

u32 target_addr = 0x08001000;
u8 data_to_save[] = {0xDE, 0xAD, 0xBE, 0xEF};

/* Erase the sector before writing to ensure bits are in the correct state */
if (ca_qspi_erase_sector(target_addr)) {
    /* Perform the write operation */
    if (ca_qspi_write(target_addr, data_to_save, sizeof(data_to_save))) {
        /* Success: data is now in flash */
    } else {
        /* Handle write error */
    }
} else {
    /* Handle erase error */
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_qspi_write to write data to a QSPI device?
A: You should use ca_qspi_write when you need to write data to a QSPI device in a blocking manner. It is suitable for general-purpose writing operations where blocking is acceptable and the operation is not time-critical. It is not appropriate for use in ISRs due to its blocking nature.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_qspi_write with caOS types?
A: u8 data[10] = {0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08, 0x09, 0x0A}; bool result = ca_qspi_write(0x1000, data, 10); if (!result) { // Handle write error }

Parameters conf:0.95 **Q:** What are the valid and invalid values for the address, buffer, and length parameters in ca_qspi_write?
A: The address must be a valid QSPI memory address within the device's range (0 to max_address). Invalid values include addresses outside this range or unmapped addresses. The buffer must be a valid pointer to allocated memory; NULL is invalid. The length must be greater than zero and within the maximum allowed size; zero or exceeding the maximum size are invalid.

Return conf:0.90 **Q:** What does ca_qspi_write return in case of success or failure, and what causes these outcomes?
A: It returns true on successful write completion. It returns false if the buffer is NULL, the length is zero, or an error occurs during the hardware write operation. Common causes of failure include invalid parameters or hardware communication errors.

`ca_queue_heap_available`

Header: `caos/inc/queue.h:366`
Source: `caos/src/queue.c:614:622`
GUARD: `EN_QUEUE`

```
u32 ca_queue_heap_available(ca_queue_t *q)
```

Returns the number of free slots currently available in a heap-allocated queue.

Parametri:

Nome	Tipo	Descrizione
`q`	`ca_queue_t *`	Pointer to the initialized heap queue handle. Must not be NULL and must point to a valid, initialized queue structure.

Ritorna:
Returns the number of empty slots (u32). Returns 0 if the queue pointer is NULL, the buffer is not initialized, or the queue is completely full.

Note:

- This function is thread-safe because it uses an internal spinlock to protect the queue's internal counters during the calculation. It is non-blocking, meaning it returns immediately without waiting for other tasks. While it is technically safe to call from an ISR due to the spinlock, it is generally better practice to use dedicated ISR-safe functions if available. The most common mistake is passing a NULL or uninitialized queue pointer; while the function safely returns 0, this often indicates that the developer failed to call the initialization function, which will lead to silent failures in the application logic.

Uso:

- Use this function to check if there is enough space in a queue before attempting to send data, which helps in preventing buffer overflows or unnecessary blocking. It is typically used in a producer-consumer pattern where the producer checks the available space before calling ca_queue_heap_send. It is part of the standard queue lifecycle: initialize the queue with ca_queue_heap_init, monitor capacity with ca_queue_heap_available, and eventually clean up resources with ca_queue_heap_destroy.

Esempio:

```
ca_queue_t myQueue;

// Initialize the queue with a capacity of 10 items
// Note: ca_queue_heap_init is the required constructor for this queue type
ca_queue_heap_init(&myQueue, 10);

// Check if we have space before sending
if (ca_queue_heap_available(&myQueue) > 0) {
    u32 data = 0xDEADBEEF;
    ca_queue_heap_send(&myQueue, &data);
} else {
    // Handle queue full condition
    ca_printf("Queue is full, cannot send data!\n");
}

// Cleanup when done
ca_queue_heap_destroy(&myQueue);
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should a developer call ca_queue_heap_available() and why is it useful?
A: A developer should call ca_queue_heap_available() to check how many elements can still be inserted into a queue before it becomes full. This is useful for preventing overflow when adding data to the queue, especially in real-time systems where buffer overflows can cause system instability. It allows for proactive resource management and avoids potential errors from overfilling the queue.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_queue_heap_available() with caOS types?
A: ca_queue_t my_queue; ca_queue_init(&my_queue, 10, sizeof(u32)); // ... some data added to queue ... u32 available = ca_queue_heap_available(&my_queue); if (available > 0) { // Safe to insert data }

Parameters conf:0.90 **Q:** What does the 'q' parameter in ca_queue_heap_available() represent, and what are valid and invalid values?
A: The 'q' parameter is a pointer to a ca_queue_t structure that must be initialized and have a valid buffer allocated. A valid value is a pointer to a properly initialized queue. An invalid value is NULL or a pointer to an uninitialized queue, which will cause the function to return 0.

Return conf:0.95 **Q:** What do the return values of ca_queue_heap_available() mean, and what causes the function to return 0?
A: The function returns 0 if the queue pointer is NULL or if the buffer is not allocated, indicating an error or uninitialized queue. It returns a u32 > 0 representing the number of elements that can still be inserted into the queue. Failure to return a positive value typically occurs when the queue is not properly initialized or the buffer is not allocated.

ca_queue_heap_count

Header: caos/inc/queue.h:359
Source: caos/src/queue.c:604:612
GUARD: EN_QUEUE

```
u32 ca_queue_heap_count(ca_queue_t *q)
```

Returns the current number of items stored in a heap-allocated queue.

Parametri:

Nome	Tipo	Descrizione
q	ca_queue_t *	Pointer to the initialized queue handle. Must not be NULL.

Ritorna:
Returns the number of items currently in the queue (u32). Returns 0 if the queue pointer is NULL or the internal buffer is not initialized.

Note:

- This function is thread-safe because it uses a spinlock to protect the internal count variable during access. It is non-blocking, meaning it returns immediately after reading the value. Because it uses 'lock_irq', it is safe to call from an ISR, though you should be mindful that holding locks in an ISR can increase latency. A common beginner mistake is assuming the returned count remains valid for a long time; in a multi-threaded environment, the count can change immediately after this function returns, so do not use this value to make critical decisions without re-checking or using proper synchronization primitives.

Uso:

- Use this function when you need to monitor queue occupancy, for example, to decide if a task should process data or wait for more items to arrive. It is typically used in a loop or a monitoring task to check the health of the system or to prevent buffer overflows. It is often paired with ca_queue_heap_init during setup and ca_queue_heap_send or ca_queue_heap_receive during runtime to manage data flow.

Esempio:

```
ca_queue_t hq;

// Initialize the queue before use
if (ca_queue_heap_init(&hq, sizeof(u32), 10U)) {
    u32 v = 0x55U;
    ca_queue_heap_send(&hq, &v);

    // Check how many items are currently in the queue
    u32 count = ca_queue_heap_count(&hq);

    if (count > 0U) {
        u32 received;
        ca_queue_heap_tryreceive(&hq, &received);
    }

    ca_queue_heap_destroy(&hq);
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should a developer call ca_queue_heap_count() in a multitasking environment?

A: A developer should call ca_queue_heap_count() to monitor the number of elements currently in a queue, especially in a multitasking environment where multiple threads may be accessing the queue. This function is thread-safe and can be safely called from ISR or task context to get a real-time count of queue elements without blocking.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_queue_heap_count() with caOS types?
A: ca_queue_t my_queue; ca_queue_init(&my_queue, 10, sizeof(u32)); ca_queue_push(&my_queue, (void*)&some_value); u32 count = ca_queue_heap_count(&my_queue);

Parameters conf:0.95 **Q:** What is the significance of passing NULL to the ca_queue_t *q parameter of ca_queue_heap_count()?
A: If the ca_queue_t *q parameter is NULL, the function returns 0 immediately. This is a safe behavior to prevent crashes when a queue pointer is invalid or uninitialized. The function does not attempt to dereference a NULL pointer, making it safe to call even if the queue has not been properly initialized.

Return conf:0.90 **Q:** What does a return value of 0 from ca_queue_heap_count() indicate, and what causes it?
A: A return value of 0 from ca_queue_heap_count() indicates either that the queue is empty or that the function was called with a NULL pointer. It may also indicate that the queue's buffer is not allocated or invalid. The function returns 0 in these cases to avoid crashes, but it does not distinguish between an empty queue and an invalid queue state.

`ca_queue_heap_destroy`

Header: `caos/inc/queue.h:254`
Source: `caos/src/queue.c:397:417`
GUARD: `EN_QUEUE`

```
1 void ca_queue_heap_destroy(ca_queue_t *q)
```

Safely deallocates a heap-based queue and wakes up all tasks blocked on it.

Parametri:

Nome	Tipo	Descrizione
`q`	`ca_queue_t *`	Pointer to the queue structure to be destroyed. Must be a valid pointer previously initialized with ca_queue_heap_init.

Ritorna:
void

Note:

- This function is thread-safe as it uses internal spinlocks to protect the queue state during cleanup. It is NOT callable from an ISR because it performs memory deallocation, which is generally non-deterministic and forbidden in interrupt context. The function ensures that any tasks currently blocked waiting to send or receive data are unblocked by signaling them, preventing them from hanging indefinitely. A common mistake is to continue using the queue pointer after calling this function; always set your pointer to NULL immediately after to avoid dangling pointer bugs.

Uso:

- Use this function when a dynamically allocated queue is no longer needed, typically during system shutdown or when a specific module is being deinitialized. It is the mandatory

counterpart to ca_queue_heap_init. By calling this, you ensure that the memory allocated for the queue buffer is returned to the system heap and that no tasks remain in a 'waiting' state, which would otherwise lead to memory leaks and task starvation.

Esempio:

```
void cleanup_module(ca_queue_t *hq) {
    if (hq != NULL) {
        // Wake up any pending tasks and free the internal buffer
        ca_queue_heap_destroy(hq);

        // Prevent dangling pointer access after destruction
        hq = NULL;
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_queue_heap_destroy() in my caOS application?
A: You should call ca_queue_heap_destroy() when you want to completely free the memory allocated for a heap queue and wake up any tasks that are currently waiting on it. This function is typically used during system shutdown or when dynamically managing queues that are no longer needed.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_queue_heap_destroy() with caOS types?
A: ca_queue_t *q = ca_queue_heap_create(10, sizeof(u32)); // ... use the queue ... ca_queue_heap_destroy(q);

Parameters conf:0.95 **Q:** What is the significance of the 'q' parameter in ca_queue_heap_destroy() and what happens if it is NULL?
A: The 'q' parameter is a pointer to the heap queue that needs to be destroyed. If 'q' is NULL, the function will terminate prematurely without performing any action, as it checks for a valid pointer before proceeding.

Error/Edge case conf:0.90 **Q:** What are common mistakes or edge cases when using ca_queue_heap_destroy()?
A: A common mistake is calling ca_queue_heap_destroy() on a queue that is still in use by other tasks, which can lead to undefined behavior. Additionally, calling this function on a queue that was not properly initialized or on a queue whose memory was not allocated can cause errors. It's also not safe to call this function from an ISR due to its use of spinlock and scheduler.

Behavior conf:0.90 **Q:** Is ca_queue_heap_destroy() blocking or non-blocking, and is it safe to call from an ISR?
A: ca_queue_heap_destroy() is a blocking function because it acquires a lock before proceeding. It is not safe to call from an ISR since it uses spinlock and the scheduler, which can lead to unpredictable behavior or system lockup.

`ca_queue_heap_init`

Header: `caos/inc/queue.h:248`
Source: `caos/src/queue.c:375:395`
GUARD: `EN_QUEUE`

```
bool ca_queue_heap_init(ca_queue_t *q, u32 item_size, u32 max_items)
```

Initializes a dynamic queue by allocating its buffer from the system heap.

Parametri:

Nome	Tipo	Descrizione
q	ca_queue_t *	Pointer to the queue control structure to be initialized.
item_size	u32	Size in bytes of a single element. Must be greater than 0.
max_items	u32	Maximum number of elements the queue can hold. Must be greater than 0.

Ritorna:
Returns true if the queue was successfully initialized and memory was allocated. Returns false if the pointer is NULL, sizes are invalid, or if the system heap is exhausted.

Note:

- This function performs dynamic memory allocation using the system heap; therefore, it is not suitable for use inside an Interrupt Service Routine (ISR) where heap operations are typically forbidden. It is thread-safe regarding the internal queue state, but you must ensure the queue handle 'q' is not accessed by other threads until this function returns true. A common beginner mistake is failing to check the return value; if the heap is fragmented or full, the function will return false and the queue will remain uninitialized, leading to a crash if you attempt to send or receive data later. Always ensure the system memory is initialized before calling this.

Uso:

- Use this function when you need a queue whose size is determined at runtime rather than compile time. It is the first step in the queue lifecycle: call this to set up the structure, then use ca_queue_heap_send or ca_queue_heap_trysend and ca_queue_heap_receive to pass data between tasks. Always pair this with ca_queue_heap_destroy when the queue is no longer needed to prevent memory leaks, as the allocated buffer will not be reclaimed automatically by the OS.

Esempio:

```
ca_queue_t myQueue;

// Initialize a queue for 10 integers
if (ca_queue_heap_init(&myQueue, sizeof(u32), 10U)) {
    u32 data = 42U;
    // Send data to the queue
    if (ca_queue_heap_trysend(&myQueue, &data)) {
        // Success
    }
} else {
    // Handle initialization failure (e.g., out of memory)
    ca_printf("Failed to init queue!\n");
}

// Clean up when finished to free the heap memory
ca_queue_heap_destroy(&myQueue);
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_queue_heap_init in a caOS application?
A: You should call ca_queue_heap_init when you need to set up a heap-based queue for inter-task communication. It is typically called during system initialization or when a task is created that requires a queue to exchange messages with other tasks.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_queue_heap_init with caOS types?
A: ca_queue_t q; bool success = ca_queue_heap_init(&q, sizeof(u32), 10); if (success) { // Queue is ready for use }

Parameters conf:0.95 **Q:** What do the item_size and max_items parameters control in ca_queue_heap_init?
A: The item_size parameter defines the size in bytes of each message that will be stored in the queue. The max_items parameter specifies the maximum number of messages the queue can hold. Both must be greater than zero, or the function will return false.

Return conf:0.95 **Q:** What does it mean when ca_queue_heap_init returns false?
A: A return value of false indicates that the initialization failed. This can happen due to invalid parameters (such as a NULL pointer, or zero item_size or max_items), or because memory allocation for the queue failed.

Error/Edge case conf:0.90 **Q:** What are common mistakes when calling ca_queue_heap_init?
A: Common mistakes include passing a NULL pointer for q, setting item_size or max_items to zero, or failing to allocate memory for the ca_queue_t structure before calling the function. These will cause the function to return false and may lead to crashes if not handled properly.

Behavior conf:0.90 **Q:** Is ca_queue_heap_init blocking, ISR-safe, or thread-safe?
A: ca_queue_heap_init is not blocking, but it may fail if memory is insufficient. It is not thread-safe and should not be called from multiple tasks without external synchronization. It is also not recommended to call it from an ISR due to potential memory allocation issues.

`ca_queue_heap_is_empty`

Header: `caos/inc/queue.h:345`
Source: `caos/src/queue.c:584:592`
GUARD: `EN_QUEUE`

```
1 bool ca_queue_heap_is_empty(ca_queue_t *q)
```

Checks if a heap-allocated queue contains no elements.

Parametri:

Nome	Tipo	Descrizione
`q`	`ca_queue_t *`	Pointer to the initialized queue handle. Must not be NULL.

Ritorna:
Returns true if the queue is empty (count is 0) or if the queue pointer/buffer is invalid. Returns false if the queue contains at least one element.

Note:

- This function is thread-safe because it uses a spinlock to protect the internal count variable during the check. It is non-blocking in the sense that it does not wait for data, but it does briefly disable interrupts to ensure data consistency. Do not call this function from an ISR (Interrupt Service Routine) if the spinlock implementation does not support nested locking or if it could cause a deadlock; for ISRs, prefer checking the count directly if the architecture allows, or use dedicated ISR-safe primitives. A common mistake is assuming that a 'false' result guarantees the queue will still be non-empty by the time you call a receive function, as another task might have emptied it in the meantime.

Uso:

- This function is typically used in a producer-consumer pattern to decide whether to attempt a data retrieval. It is often called before ca_queue_heap_receive to avoid unnecessary processing or to implement a 'wait-if-empty' logic using other synchronization primitives. It is part of the standard queue lifecycle: initialization via ca_queue_heap_init, followed by repeated checks and operations, and finally cleanup via ca_queue_heap_destroy.

Esempio:

```
ca_queue_t myQueue;

// Initialize the queue before use
ca_queue_heap_init(&myQueue, 10, sizeof(uint32_t));

// Check if we have data to process
if (!ca_queue_heap_is_empty(&myQueue)) {
    uint32_t data;
    // Retrieve the data if the queue is not empty
    if (ca_queue_heap_receive(&myQueue, &data)) {
        // Process the received data
    }
} else {
    // Queue is empty, perform other tasks or sleep
    ca_os_yield();
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_queue_heap_is_empty() before using a heap queue?
A: You should call ca_queue_heap_is_empty() before attempting to dequeue or process elements from a heap queue to avoid accessing invalid data. It's useful for preventing undefined behavior when the queue might be empty, especially in scenarios where the queue is shared between tasks or when data is produced asynchronously.

Code example conf:0.90 **Q:** Can you show a minimal example of using ca_queue_heap_is_empty() with caOS types?
A: ca_queue_t *q = ca_queue_heap_init(10); // Initialize a heap queue with capacity 10 if (!ca_queue_heap_is_empty(q)) { // Safe to proceed with queue operations u32 data; ca_queue_heap_dequeue(q, &data); // Process data } else { // Queue is empty, handle accordingly }

Parameters conf:0.90 **Q:** What does the 'q' parameter in ca_queue_heap_is_empty() represent, and what are valid values?
A: The 'q' parameter is a pointer to a ca_queue_t structure that must be valid and properly initialized. If q is NULL or points to an uninitialized queue, the function will return true. Valid values are pointers to queues that have been successfully initialized using ca_queue_heap_init().

Return conf:0.95 **Q:** What do the return values of ca_queue_heap_is_empty() indicate, and what causes it to return true?
A: The function returns true if the queue is empty, or if the pointer q is NULL or the queue buffer is not allocated. It returns false if the queue contains at least one element. A return value of true can also occur due to invalid input parameters such as an uninitialized or NULL queue pointer.

`ca_queue_heap_is_full`

Header: `caos/inc/queue.h:352`
Source: `caos/src/queue.c:594:602`
GUARD: `EN_QUEUE`

```
bool ca_queue_heap_is_full(ca_queue_t *q)
```

Checks if a heap-allocated queue has reached its maximum capacity.

Parametri:

Nome	Tipo	Descrizione
q	ca_queue_t *	Pointer to the queue handle. Must be a valid, initialized queue pointer. If NULL, the function safely returns true.

Ritorna:
Returns true if the queue is full or if the pointer is invalid. Returns false if there is still space available to store more items.

Note:

- This function is thread-safe because it uses an internal spinlock to protect the queue's count variable during the check. It is non-blocking, meaning it returns immediately after reading the state. While it uses lock_irq, which is generally safe for ISRs, you should always prefer dedicated ISR-safe functions like ca_queue_heap_receive_isr if you are operating within an interrupt context. The most common mistake is assuming the queue will stay 'not full' after the function returns; in a multi-threaded system, another task might fill the queue immediately after your check, so always verify the result of your subsequent send operation.

Uso:

- This function is typically used as a guard before attempting to push data into a queue to prevent buffer overflows or to implement flow control. It is usually paired with ca_queue_heap_init to set up the queue, and then used in a loop or conditional block before calling ca_queue_heap_send. If the function returns true, the application should either wait, drop the data, or signal an error to avoid losing information.

Esempio:

```
ca_queue_t *my_queue;

// Initialize the queue with a specific capacity
ca_queue_heap_init(&my_queue, 10);

// Check if we can safely add data
if (!ca_queue_heap_is_full(my_queue)) {
    // It is safe to send data
    int data = 42;
    ca_queue_heap_send(my_queue, &data);
} else {
    // Queue is full, handle the overflow condition
    // e.g., log an error or wait for space to become available
    ca_printf("Queue is full, cannot send data!\n");
}

// Cleanup when done
ca_queue_heap_destroy(my_queue);
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_queue_heap_is_full() and why is it useful in a real-time system?
A: You should call ca_queue_heap_is_full() before attempting to insert an element into a heap queue to avoid overflows. In a real-time system, this check prevents data loss and ensures predictable behavior by confirming that there is available space in the queue before insertion. It's especially useful when managing buffers for data logging, sensor readings, or task scheduling where overflow could lead to system instability.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_queue_heap_is_full() with caOS types?

A: ca_queue_t queue; ca_queue_heap_init(&queue, 10, sizeof(u32)); if (!ca_queue_heap_is_full(&qu { u32 data = 42; ca_queue_heap_push(&queue, &data); }

Parameters conf:0.90 **Q:** What does the 'q' parameter in ca_queue_heap_is_full() represent, and what are valid and invalid values?

A: The 'q' parameter is a pointer to a ca_queue_t structure that must be initialized and have a valid buffer allocated. A valid value is a pointer to a properly initialized queue. An invalid value is NULL or a pointer to an uninitialized queue, which will cause the function to return true.

Return conf:0.95 **Q:** What do the return values of ca_queue_heap_is_full() mean, and what causes the function to return true?

A: The function returns true if the queue is full or if the pointer q is NULL or the buffer is not allocated. It returns false if there is space available in the queue. The function may return true due to passing a NULL pointer, an uninitialized queue, or a queue that has reached its maximum capacity.

`ca_queue_heap_overwrite`

Header: `caos/inc/queue.h:324`
Source: `caos/src/queue.c:537:546`
GUARD: `EN_QUEUE`

```
1 bool ca_queue_heap_overwrite(ca_queue_t *q, const void *item)
```

Overwrites the single item in a mailbox-style queue (size 1) without blocking.

Parametri:

Nome	Tipo	Descrizione
`q`	`ca_queue_t *`	Pointer to the queue handle. Must be initialized with a capacity of 1.
`item`	`const void *`	Pointer to the data to copy into the queue. Must not be NULL.

Ritorna:
Returns true if the overwrite was successful, false if the queue pointer is invalid, the item is NULL, or the queue capacity is not exactly 1.

Note:

- This function is non-blocking and thread-safe, utilizing a spinlock to ensure atomic access to the queue buffer. It is specifically designed for 'mailbox' patterns where you only care about the most recent data point, effectively discarding any previous unread value. Because it uses a spinlock with interrupt disabling, it is safe to call from an ISR, but it should be used sparingly to avoid increasing interrupt latency. A common mistake is attempting to use this on a queue initialized with a capacity greater than 1; the function will return false and fail to perform the operation.

Uso:

- Use this function when implementing a producer-consumer pattern where the consumer only needs the latest state (e.g., sensor readings, status flags) and older, unread data is considered obsolete. It is typically used in a sequence starting with ca_queue_heap_init to create the queue, followed by repeated calls to ca_queue_heap_overwrite by the producer, and ca_queue_heap_tryreceive by the consumer. Always ensure the queue is properly cleaned up with ca_queue_heap_destroy when it is no longer needed.

Esempio:

```c
ca_queue_t status_queue;
u32 new_status = 0xABCD;

// Initialize a mailbox queue (1 slot)
if (ca_queue_heap_init(&status_queue, sizeof(u32), 1U)) {
    // Update the status; if a previous value existed, it is overwritten
    if (!ca_queue_heap_overwrite(&status_queue, &new_status)) {
        // Handle error: queue might not be size 1 or pointer is invalid
    }

    // Retrieve the latest status
    u32 current_status;
    if (ca_queue_heap_tryreceive(&status_queue, &current_status)) {
        // Process the status
    }

    ca_queue_heap_destroy(&status_queue);
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_queue_heap_overwrite instead of a standard queue push?
A: Use ca_queue_heap_overwrite when implementing a mailbox pattern where you only require the most recent data point, such as sensor readings or status flags. It is ideal when older, unread data is considered obsolete, as it discards previous values without blocking.

Code example conf:0.90 **Q:** How do I implement a producer using ca_queue_heap_overwrite in caOS?
A: ca_queue_t my_queue; u32 sensor_val = 100; ca_queue_heap_init(&my_queue, 1, sizeof(u32)); ca_queue_heap_overwrite(&my_queue, &sensor_val);

Parameters conf:1.00 **Q:** What are the constraints on the parameters for ca_queue_heap_overwrite?
A: The queue pointer 'q' must point to a queue initialized with a capacity of exactly 1. The 'item' pointer must not be NULL, as it points to the data to be copied into the queue.

Return conf:1.00 **Q:** What causes ca_queue_heap_overwrite to return false?
A: The function returns false if the queue pointer is invalid, the item pointer is NULL, or if the queue was not initialized with a capacity of exactly 1.

`ca_queue_heap_peek`

Header: `caos/inc/queue.h:332`
Source: `caos/src/queue.c:548:562`
GUARD: `EN_QUEUE`

```c
bool ca_queue_heap_peek(ca_queue_t *q, void *item)
```

Copies the oldest item from the queue to a buffer without removing it from the queue.

Parametri:

Nome	Tipo	Descrizione
`q`	`ca_queue_t *`	Pointer to the initialized queue structure. Must not be NULL.
`item`	`void *`	Pointer to the destination memory where the data will be copied. Must be large enough to hold one queue item.

Ritorna:
Returns true if an item was successfully copied, false if the queue is empty or if any input pointer is NULL.

Note:

- This function is thread-safe because it uses internal spinlocks to protect the queue data during the copy operation. It is non-blocking, meaning it returns immediately regardless of whether the queue is empty or full. You must NOT call this function from an Interrupt Service Routine (ISR) because it uses spinlocks that disable interrupts; use the specific ISR-safe variants instead. A common mistake is passing a pointer to a buffer that is smaller than the queue's item size, which will cause memory corruption.

Uso:

- Use this function when you need to inspect the next available data in a queue without consuming it, such as checking a command header before deciding to process the full message. It is typically used in a loop or a state machine where you verify the queue state with ca_queue_heap_is_empty before peeking. It is often paired with ca_queue_heap_receive to process the data only after the peek confirms the content is valid.

Esempio:

```c
ca_queue_t myQueue;
// Assume myQueue is already initialized with ca_queue_heap_init

void process_data(void) {
    uint32_t data_buffer;

    // Check if there is data to inspect
    if (!ca_queue_heap_is_empty(&myQueue)) {
        // Peek at the data without removing it from the queue
        if (ca_queue_heap_peek(&myQueue, &data_buffer)) {
            // If the data matches our criteria, remove it from the queue
            if (data_buffer == 0xDEADBEEF) {
                ca_queue_heap_receive(&myQueue, &data_buffer);
            }
        }
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_queue_heap_peek instead of ca_queue_heap_get?
A: You should use ca_queue_heap_peek when you need to inspect the next element in the queue without removing it. This is useful for checking the next item before deciding whether to process it or not, without losing data from the queue.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_queue_heap_peek with caOS types?
A: ca_queue_t q; ca_queue_heap_init(&q, sizeof(u32), 10); // Add an item first u32 item = 42; ca_queue_heap_put(&q, &item); // Peek at the item u32 peeked_item; if (ca_queue_heap_peek(&q, &peeked_item)) { // Use peeked_item }

Parameters conf:0.90 **Q:** What are the valid and invalid values for the 'q' and 'item' parameters?
A: The 'q' parameter must be a valid pointer to an initialized ca_queue_t structure, otherwise the function returns false. The 'item' parameter must point to a valid buffer large enough to hold one element of the queue, otherwise an invalid memory access may occur.

Return conf:0.95 **Q:** What does it mean when ca_queue_heap_peek returns false?

A: A return value of false indicates either that the queue is empty or that one of the parameters (q or item) is NULL. It also implies that no item was copied into the buffer pointed to by item.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_queue_heap_peek?
A: Common mistakes include passing a NULL pointer for q or item, using a buffer that is too small for the queue element type, or calling the function on a queue that has not been initialized. Additionally, calling this function from an ISR without ensuring buffer safety can lead to race conditions.

`ca_queue_heap_receive`

Header: `caos/inc/queue.h:272`
Source: `caos/src/queue.c:481:501`
GUARD: `EN_QUEUE`

```
bool ca_queue_heap_receive(ca_queue_t *q, void *item, u32 timeout_ms)
```

Blocks the current thread until an item is available in the heap-allocated queue or the timeout expires.

Parametri:

Nome	Tipo	Descrizione
`q`	`ca_queue_t *`	Pointer to the initialized heap queue handle. Must not be NULL.
`item`	`void *`	Pointer to the memory buffer where the received data will be copied. Must not be NULL.
`timeout_ms`	`u32`	Maximum time in milliseconds to wait. Use CA_WAIT_FOREVER to wait indefinitely.

Ritorna:
Returns true if an item was successfully copied into the buffer, false if the timeout was reached or if input parameters were invalid.

Note:

- This function is blocking, meaning the calling thread will be suspended by the scheduler until data arrives or the timeout occurs. It is NOT thread-safe to call this from an ISR; use ca_queue_heap_receive_isr instead. Ensure the queue is initialized via ca_queue_heap_init before use. A common mistake is passing a NULL pointer for the item buffer, which will cause the function to return false immediately without waiting. Because this function uses internal locks, it must only be called from a thread context.

Uso:

- This function is typically used in producer-consumer patterns where a worker thread waits for tasks or data packets from another thread. It is usually paired with ca_queue_heap_init for setup and ca_queue_heap_send for pushing data into the queue. It is the standard way to synchronize data flow between tasks without busy-waiting, effectively saving CPU cycles by yielding control to the OS. It is often used in a loop to continuously process incoming messages until a termination condition is met.

Esempio:

```
ca_queue_t myQueue;
// Initialize the queue before use
ca_queue_heap_init(&myQueue, 10, sizeof(u32));

```

```
u32 data;
// Wait up to 100ms for data to arrive
if (ca_queue_heap_receive(&myQueue, &data, 100)) {
    // Successfully received data
    ca_printf("Received: %lu\n", data);
} else {
    // Timeout reached or error
    ca_printf("Failed to receive data within timeout\n");
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_queue_heap_receive to get data from a queue?
A: You should use ca_queue_heap_receive when you need to retrieve an item from a queue, and you are willing to wait for the item to become available if the queue is currently empty. It is ideal for inter-task communication where one task produces data and another consumes it, and you want to avoid busy-waiting.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_queue_heap_receive with caOS types?
A: ca_queue_t my_queue; ca_queue_heap_init(&my_queue, 10, sizeof(u32)); u32 received_data; bool success = ca_queue_heap_receive(&my_queue, &received_data, 1000);

Parameters conf:0.95 **Q:** What does the timeout_ms parameter control in ca_queue_heap_receive?
A: The timeout_ms parameter controls how long the function will wait for an item to become available in the queue. If timeout_ms is 0, the function blocks indefinitely until an item is received. If timeout_ms is greater than 0, the function will return false after that many milliseconds if no item is available.

Return conf:0.90 **Q:** What do the return values of ca_queue_heap_receive indicate?
A: A return value of true indicates that an item was successfully received from the queue. A return value of false indicates either a timeout occurred or an input error (such as null pointers).

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_queue_heap_receive?
A: Common mistakes include passing null pointers for q or item, using a buffer that is too small for the item, and using timeout_ms = 0 in contexts where blocking could lead to deadlock. Additionally, calling this function from an ISR is not allowed and can cause undefined behavior.

Behavior conf:0.90 **Q:** Is ca_queue_heap_receive thread-safe and ISR-safe?
A: ca_queue_heap_receive is not ISR-safe and should not be called from an ISR. It is also not thread-safe in the sense that it modifies the queue state and may block the calling thread. It is designed for use in thread contexts where blocking is acceptable.

`ca_queue_heap_receive_isr`

Header: `caos/inc/queue.h:304`
Source: `caos/src/queue.c:507:509`
GUARD: `EN_QUEUE`

```
bool ca_queue_heap_receive_isr(ca_queue_t *q, void *item)
```

Non-blocking retrieval of an item from a heap-allocated queue, specifically designed for use within Interrupt Service Routines (ISRs).

Parametri:

Nome	Tipo	Descrizione
q	ca_queue_t *	Pointer to the initialized heap-based queue structure. Must not be NULL.
item	void *	Pointer to the memory buffer where the received data will be copied. Must have enough space to hold one queue element.

Ritorna:
Returns true if an item was successfully copied from the queue to the provided buffer. Returns false if the queue is empty or if the input pointers are invalid.

Note:

- This function is non-blocking, meaning it returns immediately regardless of whether data was available. It is specifically designed to be thread-safe for use inside ISRs, as it avoids operations that would cause the CPU to wait or context switch. A common beginner mistake is attempting to use standard blocking queue functions inside an ISR, which can lead to system crashes or deadlocks; always use the '_isr' suffix variants when inside an interrupt context. Ensure the queue has been properly initialized with ca_queue_heap_init before calling this function.

Uso:

- This function is typically used in high-frequency data acquisition scenarios where an interrupt (e.g., UART RX or Timer) needs to pass data to a background processing task. The ISR calls ca_queue_heap_receive_isr to pull data from a queue that is being filled by another source, or conversely, it is used to drain a queue that a task is monitoring. It is usually paired with ca_queue_heap_send or ca_queue_heap_send_isr to manage the flow of data between different execution contexts.

Esempio:

```
/* Example: ISR handling data from a peripheral */
void UART_Interrupt_Handler(void) {
    uint8_t received_byte;
    /* Assume 'my_queue' is a global ca_queue_t pointer initialized with
    ca_queue_heap_init */
    /* Attempt to receive data from the queue without blocking the ISR */
    if (ca_queue_heap_receive_isr(my_queue, &received_byte)) {
        /* Successfully retrieved data, now process it */
        /* Use ca_printf to log the event if necessary */
    } else {
        /* Queue was empty, handle error or ignore */
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_queue_heap_receive_isr() in an embedded application?
A: You should use ca_queue_heap_receive_isr() when you need to receive an item from a queue inside an ISR (Interrupt Service Routine) without blocking. It is ideal for high-priority interrupt handling where immediate response is required, and you want to avoid the overhead of task scheduling or blocking operations. This function is specifically designed for ISR contexts and ensures thread safety.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_queue_heap_receive_isr() with caOS types?
A: ca_queue_t queue; void *received_item; bool result; // Assume queue is initialized elsewhere result = ca_queue_heap_receive_isr(&queue, received_item); if (result) { // Item received successfully } else { // Queue was empty or error occurred }

Parameters conf:0.90 **Q:** What are the valid and invalid values for the parameters of ca_queue_heap_receive_is
A: The parameter 'q' must be a valid pointer to an initialized ca_queue_t; passing NULL will cause a crash or undefined behavior. The parameter 'item' must point to a valid memory location large enough to hold the queued item; passing NULL will result in invalid memory access. Both pointers must be valid at the time of function call.

Return conf:0.95 **Q:** What do the return values of ca_queue_heap_receive_isr() indicate and what causes failures?
A: The function returns true if an item was successfully received from the queue. It returns false if the queue is empty or if there is a parameter error (e.g., NULL pointer). Failures occur when either the queue is empty (no items to receive) or when invalid parameters (like NULL pointers) are passed to the function.

`ca_queue_heap_reset`

Header: `caos/inc/queue.h:338`
Source: `caos/src/queue.c:564:582`
GUARD: `EN_QUEUE`

```
1 void ca_queue_heap_reset(ca_queue_t *q)
```

Resets a heap-based queue to an empty state and wakes all tasks currently blocked waiting for data or space.

Parametri:

Nome	Tipo	Descrizione
`q`	`ca_queue_t *`	Pointer to the heap-based queue structure. Must be a valid, initialized queue pointer; passing NULL or an uninitialized pointer will cause the function to return immediately without action.

Ritorna:
void

Note:

- This function is thread-safe as it uses internal spinlocks to protect the queue structure during the reset process. It is NOT callable from an ISR because it performs operations that may involve scheduler interactions (waking tasks). The function effectively clears the queue by resetting the head, tail, and count pointers to zero, and it forces all tasks waiting on the queue's send or receive conditions to wake up, typically resulting in those tasks receiving an error or null return from their respective blocking calls. A common mistake is calling this while other threads are actively pushing or popping data, which is safe due to the lock, but can lead to unexpected logic errors in the application if the tasks are not designed to handle a sudden queue reset.

Uso:

- Use this function when you need to perform a 'hard reset' on a communication channel, such as during a system error recovery or when clearing stale data after a connection timeout. It is typically paired with ca_queue_heap_init during the setup phase and called during error handling routines. It is the standard way to ensure that no tasks remain permanently blocked on a queue that is no longer in a valid state.

Esempio:

```c
/* Assuming 'my_queue' is a global or heap-allocated ca_queue_t */

void handle_system_error(ca_queue_t *my_queue) {
    /* Log the error or perform other cleanup */
    ca_printf("Critical error: resetting queue state\n");

    /* Reset the queue to clear all pending data and wake blocked tasks */
    ca_queue_heap_reset(my_queue);

    /* Optionally, re-initialize or signal other parts of the system */
    /* The tasks that were blocked on ca_queue_heap_receive or
    ca_queue_heap_send
       will now wake up and should check the queue status again */
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_queue_heap_reset() in a caOS application?
A: You should call ca_queue_heap_reset() when you need to clear the contents of a heap queue and wake up all tasks that are waiting to send or receive data on that queue. This is typically used during system reset, error recovery, or when reinitializing a communication channel.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_queue_heap_reset()?
A: ca_queue_t my_queue; ca_queue_heap_init(&my_queue, sizeof(u32), 10); // ... use the queue ... ca_queue_heap_reset(&my_queue);

Parameters conf:0.90 **Q:** What is the significance of the 'q' parameter in ca_queue_heap_reset()?
A: The 'q' parameter is a pointer to the heap queue that needs to be reset. It must be a valid, non-NULL pointer to an already initialized queue. Passing NULL or an uninitialized queue will result in no operation being performed.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_queue_heap_reset()?
A: Common mistakes include calling the function on a queue that has not been initialized, calling it after freeing the queue, or calling it from an ISR context. These errors can lead to undefined behavior or system crashes.

Behavior conf:0.90 **Q:** Is ca_queue_heap_reset() thread-safe and ISR-safe?
A: No, ca_queue_heap_reset() is not thread-safe when called from an ISR because it uses functions that are not safe in interrupt context. The function is also blocking as it acquires a spinlock before performing operations.

`ca_queue_heap_send`

Header: `caos/inc/queue.h:263`
Source: `caos/src/queue.c:459:479`
GUARD: `EN_QUEUE`

```c
bool ca_queue_heap_send(ca_queue_t *q, const void *item, u32 timeout_ms)
```

Sends an item to a heap-allocated queue, blocking the calling thread if the queue is full until space is available or timeout expires.

Parametri:

Nome	Tipo	Descrizione
`q`	`ca_queue_t *`	Pointer to the initialized queue structure. Must not be NULL.
`item`	`const void *`	Pointer to the data to be copied into the queue. Must not be NULL.
`timeout_ms`	`u32`	Maximum time in milliseconds to wait if the queue is full. Use 0 for no wait (non-blocking) or a specific value for a timed block.

Ritorna:
Returns true if the item was successfully added to the queue. Returns false if the queue is invalid, the item pointer is NULL, or the timeout expired before space became available.

Note:

- This function is blocking, meaning the calling thread will be suspended by the scheduler if the queue is full and a timeout is provided. It is thread-safe due to internal spinlock usage, but it must NOT be called from an ISR (Interrupt Service Routine) because it performs blocking operations. Always ensure the queue is initialized via ca_queue_heap_init before use. A common mistake is passing a timeout of 0 and assuming the function will wait; if the queue is full, it will return false immediately. Furthermore, ensure the memory pointed to by 'item' remains valid until the function returns, as the data is copied into the queue's internal buffer.

Uso:

- This function is typically used for inter-task communication where one task produces data and another consumes it. It is usually paired with ca_queue_heap_init during system setup and ca_queue_heap_receive in the consumer task. Use this when you need to safely pass data between tasks without worrying about buffer overflows, as the blocking mechanism naturally throttles the producer if the consumer is slower. It is frequently used in pipeline architectures to pass pointers to encrypted or processed data buffers between service tasks.

Esempio:

```
/* Initialize a queue for 10 items */
ca_queue_t my_queue;
ca_queue_heap_init(&my_queue, 10, sizeof(u32));

void producer_task(void) {
    u32 data = 42;
    /* Attempt to send data, wait up to 100ms if queue is full */
    if (ca_queue_heap_send(&my_queue, &data, 100)) {
        /* Success: data is now in the queue */
    } else {
        /* Handle timeout or error: queue was full for too long */
        /* Perform cleanup if necessary */
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_queue_heap_send() in a real-time embedded system?
A: You should use ca_queue_heap_send() when you need to send data from one task or ISR to another task in a thread-safe manner, especially when the sender might block until space becomes available in the queue. It's ideal for producer-consumer scenarios where data must be passed between tasks with guaranteed delivery.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_queue_heap_send() with caOS types?

A: ca_queue_t q; ca_queue_heap_init(&q, 10, sizeof(u32)); u32 data = 42; bool success = ca_queue_heap_send(&q, &data, 100);

Parameters conf:0.85 **Q:** What are the valid and invalid values for the timeout_ms parameter in ca_queue_heap_send()?
A: If timeout_ms is 0, the function will not block and will return false immediately if the queue is full. If timeout_ms is greater than 0, the function will block up to that many milliseconds waiting for space in the queue. A negative value is not defined and could cause unexpected behavior.

Return conf:0.95 **Q:** What does it mean when ca_queue_heap_send() returns false?
A: A return value of false indicates that the item was not sent successfully. This can happen due to invalid parameters (q or item is NULL), a timeout occurring before space became available, or an internal error within the queue mechanism.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_queue_heap_send()?
A: Common mistakes include passing NULL for q or item, not checking the return value when timeout_ms is 0, and calling the function from an ISR without using ca_queue_heap_send_from_ Also, failing to initialize the queue with ca_queue_heap_init() before use can lead to undefined behavior.

Behavior conf:0.90 **Q:** Is ca_queue_heap_send() safe to call from an ISR, and is it thread-safe?
A: No, ca_queue_heap_send() is not safe to call from an ISR. You must use ca_queue_heap_send_from for ISR contexts. However, it is thread-safe as it uses a spinlock to protect concurrent access to the queue.

`ca_queue_heap_send_front`

Header: `caos/inc/queue.h:313`
Source: `caos/src/queue.c:511:534`
GUARD: `EN_QUEUE`

```
1 bool ca_queue_heap_send_front(ca_queue_t *q, const void *item, u32
     timeout_ms)
```

Sends an item to the front of a queue, blocking up to a timeout if the queue is full.

Parametri:

Nome	Tipo	Descrizione
`q`	`ca_queue_t *`	Pointer to the queue structure. Must not be NULL, and must point to a valid queue initialized with ca_queue_heap_init().
`item`	`const void *`	Pointer to the data to send. Must not be NULL, and must point to valid memory of size q->item_size.
`timeout_ms`	`u32`	Timeout in milliseconds. If 0, function returns immediately. If set to CA_OS_WAIT_FOREVER, it will block indefinitely until space is available.

Ritorna:
`true`: Item was successfully sent to the front of the queue. `false`: Failed to send item due to timeout or invalid parameters (e.g., NULL queue or item).

Note:

- **Blocking behavior**: This function blocks if the queue is full and timeout is not zero. It will not return until an item is sent or timeout occurs.

- **Thread-safe**: Yes, uses spinlock to protect queue access from concurrent access.
- **Callable from ISR**: No, this function is NOT safe to call from an ISR. It may block and uses spinlocks that are not ISR-safe.
- **Preconditions**: Queue must be initialized with ca_queue_heap_init() before calling this function. The queue must have valid buffer and item_size.
- **Common mistakes**: Passing NULL for q or item will cause immediate return false. Using this function in an ISR will cause system instability or deadlock.

Uso:

- **Normal thread context**: Used in threads to send data to the front of a queue, such as prioritizing urgent messages.
- {'description': 'Paired with ca_queue_heap_receive() or ca_queue_heap_tryreceive() for full queue communication.'}

Esempio:

```
// This example initializes a queue, then sends an integer to the front of the
    queue with a 100ms timeout. If the queue is full and no space becomes
    available within 100ms, the function returns false.
ca_queue_t my_queue;
ca_queue_heap_init(&my_queue, 10, sizeof(int));
int data = 42;

// Send data to front of queue with 100ms timeout
if (ca_queue_heap_send_front(&my_queue, &data, 100)) {
    // Success: data sent
} else {
    // Timeout or invalid parameters
    // Handle error
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_queue_heap_send_front() instead of a regular queue send function?
A: Use ca_queue_heap_send_front() when you need to insert an item at the front of the queue, rather than at the back. This is useful for prioritizing elements, such as giving higher priority to critical tasks or urgent data. It is a blocking operation if a timeout is specified, allowing the calling thread to wait until space becomes available in the queue.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_queue_heap_send_front() with caOS types?
A: ca_queue_t queue; void *buffer[10]; ca_queue_heap_init(&queue, buffer, 10, sizeof(u32)); u32 item = 42; bool result = ca_queue_heap_send_front(&queue, &item, 100);

Parameters conf:0.90 **Q:** What are the valid values for the timeout_ms parameter in ca_queue_heap_send_fro
A: The timeout_ms parameter must be a non-negative integer. A value of 0 makes the function non-blocking, meaning it will return immediately if the queue is full. A value greater than 0 makes the function blocking, where it will wait up to that many milliseconds for space to become available in the queue.

Return conf:0.95 **Q:** What does it mean when ca_queue_heap_send_front() returns false?
A: A return value of false indicates that the item was not inserted into the queue. This can happen due to invalid parameters (such as NULL queue or item), a timeout that expired, or an internal error. It does not necessarily mean the queue is full, but rather that the operation failed for one of the specified reasons.

Error/Edge case conf:0.90 **Q:** What are the most common mistakes when using ca_queue_heap_send_front()
A: Common mistakes include passing NULL for the queue or item pointer, using a negative timeout value, calling the function from an ISR (which is not allowed), or failing

to initialize the queue properly with valid buffer and item_size. These errors can lead to undefined behavior or immediate failure of the function.

Behavior conf:0.90 **Q:** Is ca_queue_heap_send_front() thread-safe and can it be called from an ISR?

A: Yes, ca_queue_heap_send_front() is thread-safe and uses a spinlock to protect concurrent access. However, it is not safe to call from an ISR because it may block and manipulate structures that are not ISR-safe. The function is designed for use in thread contexts where blocking is acceptable.

`ca_queue_heap_send_isr`

Header: `caos/inc/queue.h:296`
Source: `caos/src/queue.c:503:505`
GUARD: `EN_QUEUE`

```
bool ca_queue_heap_send_isr(ca_queue_t *q, const void *item)
```

Sends an item to a heap-allocated queue from an ISR context without blocking.

Parametri:

Nome	Tipo	Descrizione
`q`	`ca_queue_t *`	Pointer to the heap-allocated queue structure. Must not be NULL.
`item`	`const void *`	Pointer to the data item to send. Must not be NULL.

Ritorna:
`true`: Item was successfully sent to the queue. `false`: Failed to send item (queue is full or invalid parameters).

Note:

- • This function is NON-BLOCKING and can ONLY be called from ISR context.
- • It does NOT perform any waiting or timeout handling.
- • It is NOT thread-safe in the general sense, but safe for ISR use.
- • It is NOT callable from regular task context.
- • The queue must be initialized properly before calling this function.
- • If the queue is full, it immediately returns false without waiting.
- • Common mistake: calling this function from a task instead of an ISR.

Uso:

- • Used in interrupt handlers to quickly pass data from ISR to task.
- • Typically paired with queue receive functions like ca_queue_heap_receive_isr().
- • Used in embedded systems where fast, non-blocking communication between ISR and task is needed.
- • Often used in real-time systems where timing is critical.

Esempio:

```
// This example shows how to send data from an ISR to a heap-allocated queue.
   The function is non-blocking and will return immediately. If the queue is
   full, it returns false, which may indicate a need to increase queue size or
   handle overflow.
#include "ca_os.h"

void UART_IRQHandler(void) {
    static uint32_t received_data;
```

```
    // Simulate data received from UART
    received_data = USART1->DR;

    // Send data to queue from ISR
    if (!ca_queue_heap_send_isr(&my_heap_queue, &received_data)) {
        // Handle error: queue full or invalid queue
        // Note: This should not happen if queue is large enough
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_queue_heap_send_isr() in my embedded application?
A: You should use ca_queue_heap_send_isr() when you need to send an item to a queue from an ISR context without blocking. This function is specifically designed for interrupt service routines where blocking is not allowed and where you want to avoid potential race conditions or system instability. It ensures that the queue operation is safe and non-blocking in an ISR.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_queue_heap_send_isr() with caOS types?
A: ca_queue_t queue; ca_queue_heap_init(&queue, 10, sizeof(u32));
void isr_handler(void) { u32 data = 42; ca_queue_heap_send_isr(&queue, &data); }

Parameters conf:0.90 **Q:** What are the valid and invalid values for the parameters of ca_queue_heap_send_isr()
A: The first parameter 'q' must be a valid pointer to a previously initialized queue; if NULL, the function fails. The second parameter 'item' must be a valid pointer to the data to be sent; if NULL, the behavior is undefined. Both parameters must point to valid memory locations that are accessible in the ISR context.

Return conf:0.95 **Q:** What do the return values of ca_queue_heap_send_isr() indicate, and what causes failures?
A: The function returns true if the item was successfully sent to the queue. It returns false if the item could not be sent due to the queue being full or an error such as a NULL queue pointer. Failures occur when the queue is full or when the queue pointer is invalid (e.g., NULL).

`ca_queue_heap_tryreceive`

Header: `caos/inc/queue.h:288`
Source: `caos/src/queue.c:439:457`
GUARD: `EN_QUEUE`

```
bool ca_queue_heap_tryreceive(ca_queue_t *q, void *item)
```

Attempts to remove an item from a heap-allocated queue without blocking the calling thread.

Parametri:

Nome	Tipo	Descrizione
`q`	`ca_queue_t *`	Pointer to the initialized queue structure. Must not be NULL.
`item`	`void *`	Pointer to the memory buffer where the received data will be copied. Must not be NULL.

Ritorna:
Returns true if an item was successfully retrieved from the queue. Returns false if the queue is empty, or if any input parameter is NULL.

Note:

- This function is non-blocking, meaning it returns immediately regardless of whether data was available. It is thread-safe because it uses an internal spinlock to protect the queue's integrity during the copy operation. It is NOT safe to call this function from an ISR (Interrupt Service Routine) because it uses spinlocks that could cause deadlocks if an interrupt preempts a thread holding the same lock; use ca_queue_heap_receive_isr for ISR contexts. Always ensure the queue has been properly initialized with ca_queue_heap_init before use; passing an uninitialized queue pointer is a common beginner mistake that leads to hard faults.

Uso:

- This function is typically used in polling loops or event-driven tasks where the thread cannot afford to wait for data. It is often paired with ca_queue_heap_init to set up the queue and ca_queue_heap_send to populate it. In a typical producer-consumer pattern, the consumer thread calls this function to check for new work; if it returns false, the thread can perform other background tasks or enter a low-power state using ca_os_sleep before checking again. It is also used to drain queues during cleanup or error recovery sequences.

Esempio:

```
ca_queue_t hq;
// Initialize queue for 10 items of 4 bytes each
if (ca_queue_heap_init(&hq, 4U, 10U)) {
    uint32_t data;
    // Non-blocking check for data
    if (ca_queue_heap_tryreceive(&hq, &data)) {
        ca_printf("Received: %lu\n", data);
    } else {
        // No data available, yield to other tasks to save CPU
        ca_os_yield();
    }
    ca_queue_heap_destroy(&hq);
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_queue_heap_tryreceive instead of a blocking receive function?
A: You should use ca_queue_heap_tryreceive when you want to check for available data in a queue without blocking the calling thread. This is useful in scenarios where you need to perform other tasks while waiting for data, or when you want to avoid potential deadlocks in time-critical applications. It's ideal for non-blocking polling or checking queue status before proceeding.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_queue_heap_tryreceive with caOS types?
A: ca_queue_t q; void *buffer = malloc(sizeof(u32)); ca_queue_heap_init(&q, sizeof(u32), 5); if (ca_queue_heap_tryreceive(&q, buffer)) { u32 received = *(u32*)buffer; // Process received data } else { // No data available }

Parameters conf:0.90 **Q:** What are the valid and invalid values for the 'q' and 'item' parameters?
A: The 'q' parameter must be a valid pointer to an initialized ca_queue_t structure, otherwise the function returns false. The 'item' parameter must point to a valid buffer large enough to hold one element of the queue. Passing NULL for either parameter results in an immediate return of false, as per the error conditions.

Return conf:0.95 **Q:** What does it mean when ca_queue_heap_tryreceive returns false?
A: A return value of false indicates that no element was available in the queue at the time of the call. The function does not block or wait; it simply checks the queue and returns immediately. This is not an error condition but a signal that the queue is empty.

Error/Edge case conf:0.85 **Q:** What are the common mistakes when using ca_queue_heap_tryreceive?
A: Common mistakes include passing NULL pointers for 'q' or 'item', not checking the return value, and using a buffer that is too small for the queue element size. Not checking the return value can lead to using invalid data, and incorrect buffer sizing may cause memory corruption.

Behavior conf:0.95 **Q:** Is ca_queue_heap_tryreceive thread-safe and ISR-safe, and does it block?
A: Yes, ca_queue_heap_tryreceive is both thread-safe and ISR-safe because it uses a spinlock to protect access to the queue. It is also non-blocking, meaning it returns immediately regardless of whether the queue is empty or not. This makes it suitable for use in interrupt service routines and multi-threaded environments.

`ca_queue_heap_trysend`

Header: `caos/inc/queue.h:280`
Source: `caos/src/queue.c:419:437`
GUARD: `EN_QUEUE`

```
bool ca_queue_heap_trysend(ca_queue_t *q, const void *item)
```

Attempts to push an item into a heap-allocated queue without blocking.

Parametri:

Nome	Tipo	Descrizione
`q`	`ca_queue_t *`	Pointer to the initialized queue structure. Must not be NULL.
`item`	`const void *`	Pointer to the data to be copied into the queue. Must not be NULL.

Ritorna:
Returns true if the item was successfully added to the queue. Returns false if the queue is full, the parameters are invalid (NULL), or the queue buffer is not initialized.

Note:

- This function is non-blocking, meaning it returns immediately regardless of whether the operation succeeded or failed. It is thread-safe because it uses internal spinlocks to protect the queue's internal state during the copy operation. It is safe to call from an ISR context because it uses 'irq' variants of the spinlock functions, which disable interrupts to prevent deadlocks. The most common mistake is assuming the item was sent without checking the boolean return value; if the queue is full, the data is simply not added, and the caller must handle this retry logic or error condition.

Uso:

- This function is typically used in producer-consumer patterns where the producer cannot afford to wait (block) if the queue is full. It is often paired with a loop that checks the return value and potentially waits for space using other synchronization primitives if the operation fails. The typical lifecycle involves initializing the queue with ca_queue_heap_init, using this function to push data, and eventually cleaning up with ca_queue_heap_destroy.

Esempio:

```
ca_queue_t my_queue;
// Initialize the queue with a specific item size
ca_queue_heap_init(&my_queue, 10, sizeof(int));

int data = 42;
```

```c
// Attempt to send data without blocking
if (ca_queue_heap_trysend(&my_queue, &data)) {
    // Success: data is now in the queue
} else {
    // Failure: queue is full or invalid
    // Perform error handling or wait for space
    /* call a function to wait for queue space */
}

// Cleanup when done
ca_queue_heap_destroy(&my_queue);
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_queue_heap_trysend() instead of a blocking send function?
A: You should use ca_queue_heap_trysend() when you need to send an item to a queue without blocking the calling thread. It's useful in time-critical applications or when you want to avoid potential deadlocks. Since it's non-blocking, it returns immediately, either sending the item or indicating that the queue is full.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_queue_heap_trysend() with caOS types?
A: ca_queue_t q; void *buffer = malloc(sizeof(u32) * 10); ca_queue_heap_init(&q, buffer, 10, sizeof(u32)); u32 item = 42; bool success = ca_queue_heap_trysend(&q, &item); if (success) { // Item sent successfully } else { // Queue was full or error occurred }

Parameters conf:0.95 **Q:** What are the valid and invalid values for the 'q' and 'item' parameters?
A: The 'q' parameter must be a valid pointer to an initialized ca_queue_t structure. If q is NULL or the queue is not initialized (e.g., q->buffer is NULL), the function returns false. The 'item' parameter must be a valid pointer to the data to send. If item is NULL, the function returns false.

Return conf:0.95 **Q:** What does it mean when ca_queue_heap_trysend() returns false?
A: A return value of false indicates that the item was not sent. This can happen if the queue is full, or if one of the parameters (q or item) is NULL. It does not indicate a system error, but rather a failure to enqueue the item due to capacity or invalid input.

Error/Edge case conf:0.90 **Q:** What are the most common mistakes when using ca_queue_heap_trysend()?
A: Common mistakes include passing NULL pointers for q or item, using an uninitialized queue, ignoring the return value and assuming the item was sent, and calling the function with a queue that has no available space. These can lead to silent failures or undefined behavior.

Behavior conf:0.90 **Q:** Is ca_queue_heap_trysend() thread-safe and ISR-safe?
A: Yes, ca_queue_heap_trysend() is thread-safe and ISR-safe. It uses a spinlock to ensure mutual exclusion between threads and is compatible with interrupt service routines (ISRs). However, the caller must ensure correct context when calling from an ISR.

`ca_queue_static_available`

Header: `caos/inc/queue.h:235`
Source: `caos/src/queue.c:361:369`
GUARD: `EN_QUEUE`

```c
u32 ca_queue_static_available(ca_queue_static_t *q)
```

Returns the number of empty slots currently available in a static queue.

Parametri:

Nome	Tipo	Descrizione
q	ca_queue_static_t *	Pointer to the static queue structure. Must be initialized via ca_queue_static_init before use.

Ritorna:
Returns the number of free slots (u32) available for new data. Returns 0 if the queue is full or if the provided pointer is NULL.

Note:

- This function is thread-safe as it internally uses a spinlock to protect the queue's internal counters during the calculation. It is non-blocking, meaning it returns immediately without waiting for queue operations to complete. While it is technically safe to call from an ISR due to the spinlock mechanism, it is generally used in task-level code to check if there is enough space before attempting a send operation. A common mistake is assuming the returned value remains constant; in a multi-threaded system, another task or ISR could fill the queue immediately after this function returns, so always check the return value of the subsequent send function.

Uso:

- This function is typically used as a guard check before pushing data into a queue to prevent buffer overflows or to implement flow control. It is often paired with ca_queue_static_init during system setup and used in a loop or conditional block alongside ca_queue_static_send. By checking the available space first, you can decide whether to drop a packet, wait for space, or signal an error condition to the system.

Esempio:

```c
/* Define a static queue structure */
ca_queue_static_t my_queue;
/* Assume buffer is allocated elsewhere */

void task_producer(void *pv) {
    u32 data = 0xDEADBEEF;

    /* Initialize the queue before use */
    /* ca_queue_static_init(&my_queue, buffer, max_items); */

    /* Check if there is space before sending */
    if (ca_queue_static_available(&my_queue) > 0) {
        /* Send data if space is available */
        ca_queue_static_send(&my_queue, &data);
    } else {
        /* Handle queue full condition */
        ca_printf("Queue full, dropping data!\n");
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_queue_static_available() in my embedded application?
A: You should call ca_queue_static_available() when you need to check how many elements can still be inserted into a static queue before performing an insertion. This is useful for avoiding overflow conditions or managing buffer space efficiently. It's typically used in scenarios where you want to prevent blocking or ensure that data can be safely enqueued without causing issues.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_queue_static_available()?

A: ca_queue_static_t my_queue; ca_queue_static_init(&my_queue, 10, sizeof(u32)); // Insert some data u32 data = 42; ca_queue_static_push(&my_queue, &data); // Check available space u32 available = ca_queue_static_available(&my_queue); if (available > 0) { // Safe to push more data }

Parameters conf:0.90 **Q:** What does the 'q' parameter in ca_queue_static_available() represent, and what are valid values?

A: The 'q' parameter is a pointer to a static queue structure (ca_queue_static_t). It must point to a valid and properly initialized queue. If 'q' is NULL or points to an uninitialized queue, the function returns 0. Valid values are pointers to queues that have been initialized using a function like ca_queue_static_init().

Return conf:0.95 **Q:** What do the return values of ca_queue_static_available() indicate, and what causes it to return 0?

A: If ca_queue_static_available() returns 0, it indicates either that the input pointer 'q' is NULL or that the queue is not properly initialized. This is not a fatal error but means no information about queue availability can be retrieved. A return value greater than 0 indicates the number of elements that can still be inserted into the queue. If the return value is 0, the queue is full.

`ca_queue_static_count`

Header: `caos/inc/queue.h:228`
Source: `caos/src/queue.c:351:359`
GUARD: `EN_QUEUE`

```
u32 ca_queue_static_count(ca_queue_static_t *q)
```

Returns the current number of items stored in a static queue.

Parametri:

Nome	Tipo	Descrizione
`q`	`ca_queue_static_t *`	Pointer to the static queue structure. Must be initialized via ca_queue_static_init before use. Passing NULL will return 0.

Ritorna:
Returns the unsigned 32-bit integer representing the number of items currently in the queue. Returns 0 if the queue is empty or if the provided pointer is NULL.

Note:

- This function is thread-safe because it internally uses a spinlock to protect the queue's internal counter during the read operation. It is non-blocking, meaning it returns immediately after reading the value, but it does briefly disable interrupts to ensure data consistency. While it is safe to call from most contexts, avoid excessive polling in high-frequency ISRs to prevent system latency. A common mistake is assuming the returned count remains valid indefinitely; in a multi-threaded system, the count can change immediately after the function returns, so use it only for status checks or logging rather than strict flow control.

Uso:

- This function is typically used for monitoring queue health or making logic decisions before attempting to read or write data. It is often paired with ca_queue_static_init for setup, and used alongside ca_queue_static_is_empty or ca_queue_static_is_full to prevent buffer overflows or underflows. Developers frequently use it to decide whether to trigger a

processing task or to log system status via ca_printf_n, especially when debugging sensor data pipelines or communication buffers.

Esempio:

```
ca_queue_static_t my_queue;

void monitor_task(void) {
    // Initialize the queue before use
    // ca_queue_static_init(&my_queue, buffer, sizeof(u32), 10);

    u32 count = ca_queue_static_count(&my_queue);

    if (count > 0) {
        ca_printf_n("Queue contains %u items", count);
    } else {
        // Queue is empty, yield to other tasks
        ca_os_yield();
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_queue_static_count() in a caOS application?
A: You should use ca_queue_static_count() to determine the current number of elements in a static queue, typically for debugging, monitoring queue usage, or making decisions based on queue load. It is a non-blocking function that returns immediately, making it suitable for both task and ISR contexts, provided the spinlock is ISR-compatible.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_queue_static_count() with caOS types?
A: ca_queue_static_t q; ca_queue_static_init(&q, 10, sizeof(u32)); ca_queue_static_put(&q, (void*)&some_value); u32 count = ca_queue_static_count(&q);

Parameters conf:0.95 **Q:** What is the significance of the 'q' parameter in ca_queue_static_count() and what happens if it is invalid?
A: The 'q' parameter is a pointer to a static queue that must be valid and properly initialized. If 'q' is NULL or points to an uninitialized queue, the function returns 0 without causing an error. This is a safety measure to prevent crashes due to invalid pointers.

Return conf:0.90 **Q:** What do the return values of ca_queue_static_count() indicate and what causes a return value of 0?
A: The function returns 0 if the input queue pointer 'q' is NULL or if the queue is not initialized. Otherwise, it returns the number of elements currently in the queue. A return value of 0 does not necessarily indicate an error, but rather that either the queue is empty or invalid.

`ca_queue_static_init`

Header: `caos/inc/queue.h:123`
Source: `caos/src/queue.c:135:146`
GUARD: `EN_QUEUE`

```
void ca_queue_static_init(ca_queue_static_t *q, u32 item_size, u32
    max_items)
```

Initializes a static memory queue for thread-safe data exchange between tasks.

Parametri:

Nome	Tipo	Descrizione
q	ca_queue_static_t *	Pointer to the queue control block structure. Must be allocated in static memory (e.g., global or static variable).
item_size	u32	Size of each individual data element in bytes. Must be greater than 0.
max_items	u32	Maximum number of items the queue can hold. Must be greater than 0.

Ritorna:
void. If the pointer 'q' is NULL, the function returns immediately without performing any action.

Note:

- This function is non-blocking and performs the internal setup of synchronization primitives (wait queues and spinlocks). It is not thread-safe to call this on a queue that is already being accessed by other tasks; it must be called during the system initialization phase before the scheduler starts or before the queue is used. A common beginner mistake is to declare the queue structure as a local variable inside a function, which causes the queue to be destroyed when the function returns, leading to memory corruption or system crashes. Always ensure the queue structure persists for the entire lifetime of the application.

Uso:

- This function is used to prepare a communication channel between tasks or between an ISR and a task. It is typically called during the system configuration phase (e.g., inside ca_os_configure) to set up global message queues. It is the mandatory first step before any other queue operations like ca_queue_static_send or ca_queue_static_receive can be performed. It is often paired with the initialization of other system components like stdout or CLI services to ensure data buffers are ready to receive incoming messages.

Esempio:

```c
/* Define the queue structure in static memory */
static ca_queue_static_t my_data_queue;

void app_init(void) {
    /* Initialize the queue to hold 10 integers */
    ca_queue_static_init(&my_data_queue, sizeof(int), 10);
}

void task_producer(void) {
    int data = 42;
    /* Send data to the queue; if full, this would block until space is
    available */
    ca_queue_static_send(&my_data_queue, &data);
}

void task_consumer(void) {
    int received_data;
    /* Receive data from the queue; if empty, this would block until data
    arrives */
    ca_queue_static_receive(&my_data_queue, &received_data);
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_queue_static_init in a caOS application?
A: You should call ca_queue_static_init before using any other queue functions to initialize a static queue. It must be called once per queue before any task attempts to

use it for communication. This function prepares the queue structure for use but does not make it ready for use until the associated tasks are also initialized.

Code example conf:0.90 **Q:** Can you show a minimal example of initializing and using a static queue with caOS types?
A: ca_queue_static_t my_queue; ca_queue_static_init(&my_queue, sizeof(u32), 10); // Now my_queue is initialized and ready to be used with other queue functions like ca_queue_static_send or ca_queue_static_receive.

Parameters conf:0.90 **Q:** What are the valid ranges for item_size and max_items in ca_queue_static_init?
A: Both item_size and max_items must be greater than zero. If either parameter is zero or negative, the function will still execute without error, but the resulting queue will not behave correctly. The item_size defines the size in bytes of each element stored in the queue, and max_items defines how many such elements can be queued.

Return conf:0.95 **Q:** Does ca_queue_static_init return any value, and what does it mean?
A: The function returns void, meaning it does not return any value. It is non-blocking and never fails. If the pointer q is NULL, the function simply returns immediately without performing any action.

Error/Edge case conf:0.90 **Q:** What happens if I pass a NULL pointer to ca_queue_static_init?
A: If the ca_queue_static_t* q parameter is NULL, the function will return immediately without performing any initialization. It will not cause a crash or error, but the queue will not be initialized either, which will lead to undefined behavior if other queue functions are called on it.

`ca_queue_static_is_empty`

Header: `caos/inc/queue.h:214`
Source: `caos/src/queue.c:331:339`
GUARD: `EN_QUEUE`

```
bool ca_queue_static_is_empty(ca_queue_static_t *q)
```

Checks if a static queue contains no elements.

Parametri:

Nome	Tipo	Descrizione
`q`	`ca_queue_static_t *`	Pointer to the static queue structure. Must be a valid, initialized pointer.

Ritorna:
Returns true if the queue is empty or if the pointer is NULL. Returns false if the queue contains one or more elements.

Note:

- This function is thread-safe because it uses a spinlock to protect the internal count variable during the check. It is non-blocking in the sense that it does not wait for data, but it does briefly disable interrupts to ensure atomicity. Do not call this function from an ISR if the spinlock is already held by the interrupted task, as this will cause a deadlock. The most common mistake is assuming the queue state remains empty after the function returns; in a multi-threaded system, another task could push data into the queue immediately after this check completes.

Uso:

- This function is typically used in a producer-consumer pattern to decide whether to process data or enter a low-power state. It is often paired with ca_queue_static_receive to drain a queue or ca_os_sleep to wait for data to arrive. You should initialize the queue with

ca_queue_static_init before calling this function to ensure the internal lock and count are valid.

Esempio:

```
ca_queue_static_t myQueue;

// Initialize the queue before use (assuming static configuration)
ca_queue_static_init(&myQueue);

// Check if there is work to do
if (ca_queue_static_is_empty(&myQueue)) {
    // Queue is empty, perform other tasks or sleep
    ca_os_yield();
} else {
    // Process the data
    int data;
    // Attempt to receive data without waiting
    if (ca_queue_static_tryreceive(&myQueue, &data)) {
        // Successfully received data
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should a developer use ca_queue_static_is_empty() in a real-time embedded system?
A: A developer should use ca_queue_static_is_empty() to check if a static queue contains any elements before attempting to read from it. This is useful in real-time systems to avoid processing invalid or stale data, and to prevent blocking operations. It's particularly helpful in task scheduling or data handling where you want to ensure data availability without delay.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_queue_static_is_empty() with caOS types?
A: ca_queue_static_t queue; ca_queue_static_init(&queue, sizeof(u32), 10); if (ca_queue_static_is_
{ // Handle empty queue case } else { u32 data; ca_queue_static_get(&queue, &data);
}

Parameters conf:0.90 **Q:** What is the significance of the ca_queue_static_t *q parameter in ca_queue_static_is_empty()?
A: The ca_queue_static_t *q parameter is a pointer to the static queue that needs to be checked for emptiness. It must be a valid and properly initialized queue; otherwise, the function returns true. Passing a NULL pointer or an uninitialized queue will result in a true return value, but may lead to undefined behavior if the queue is accessed afterward.

Return conf:0.90 **Q:** What does it mean when ca_queue_static_is_empty() returns true or false?
A: When ca_queue_static_is_empty() returns true, it indicates that the queue is either empty or the pointer passed is NULL. When it returns false, it means the queue contains at least one element. Failures or incorrect behavior can occur if the queue is not initialized before calling this function or if the pointer is invalid, which causes the function to return true incorrectly.

`ca_queue_static_is_full`

Header: `caos/inc/queue.h:221`
Source: `caos/src/queue.c:341:349`
GUARD: `EN_QUEUE`

```
bool ca_queue_static_is_full(ca_queue_static_t *q)
```

Checks if a static queue has reached its maximum capacity.

Parametri:

Nome	Tipo	Descrizione
q	ca_queue_static_t *	Pointer to the static queue structure. Must be a valid, initialized pointer.

Ritorna:
Returns true if the queue is full or if the pointer is NULL. Returns false if the queue has space available.

Note:

- This function is thread-safe as it internally uses a spinlock to protect the queue's internal counter during the check. It is non-blocking in the sense that it returns immediately, but it briefly disables interrupts to ensure data consistency. Do not call this function from an ISR if the spinlock implementation is not ISR-safe; however, in this specific caOS implementation, the use of lock_irq suggests it is designed to be safe for concurrent access. A common beginner mistake is assuming that because this function returns false, the queue will still have space when the next operation (like a push) is called; in a multi-threaded environment, another task might fill the queue in the microsecond between your check and your subsequent action.

Uso:

- This function is typically used as a guard condition before attempting to push data into a queue to prevent buffer overflows or data loss. It is usually paired with ca_queue_static_init to set up the queue, and then used in a loop or conditional block alongside ca_queue_static_send. By checking the status first, you can implement flow control, such as dropping packets or waiting for the consumer task to process items before attempting to send more.

Esempio:

```c
/* Assuming a static queue has been initialized elsewhere */
ca_queue_static_t my_queue;

void task_producer(void *pvParameters) {
    uint32_t data = 0xDEADBEEF;

    while (1) {
        /* Check if the queue is full before attempting to send */
        if (!ca_queue_static_is_full(&my_queue)) {
            /* Queue has space, safe to send */
            ca_queue_static_send(&my_queue, &data);
        } else {
            /* Queue is full, wait a bit to avoid busy-waiting */
            ca_os_delay_ms(10);
        }
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_queue_static_is_full() in my embedded application?
A: You should call ca_queue_static_is_full() before attempting to insert an element into a static queue to avoid overwriting data. This function is useful for preventing buffer overflows in real-time systems where queue limits must be strictly enforced. It's particularly helpful in scenarios where you're managing data flow between tasks or ISRs and need to ensure that no data is lost due to queue overflow.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_queue_static_is_full() with caOS types?
A: ca_queue_static_t queue; ca_queue_static_init(&queue, 10, sizeof(u32)); // ... fill queue with data ... if (ca_queue_static_is_full(&queue)) { // Handle queue full condition } else { // Safe to insert new data }

Parameters conf:0.90 **Q:** What is the significance of the ca_queue_static_t* q parameter in ca_queue_static_is_full()?
A: The ca_queue_static_t* q parameter is a pointer to the static queue that needs to be checked for full status. It must point to a valid and properly initialized queue structure. If the pointer is NULL, the function returns true, which can be misinterpreted as a full queue. Passing an uninitialized queue or a pointer to freed memory will also result in unpredictable behavior or return true.

Return conf:0.95 **Q:** What do the return values of ca_queue_static_is_full() indicate and what causes them?
A: The function returns true if the queue is full or if the input pointer is NULL. This means no more elements can be inserted into the queue or an invalid pointer was passed. It returns false if the queue has space available for new elements. A return value of true due to a NULL pointer is a common error that can lead to incorrect logic flow if not handled properly.

`ca_queue_static_overwrite`

Header: `caos/inc/queue.h:193`
Source: `caos/src/queue.c:282:291`
GUARD: `EN_QUEUE`

```
1 bool ca_queue_static_overwrite(ca_queue_static_t *q, const void *item)
```

Overwrites the single item in a static queue; never blocks, safe to call from ISR.

Parametri:

Nome	Tipo	Descrizione
`q`	`ca_queue_static_t *`	Pointer to the static queue structure. Must not be NULL. The queue must be initialized with max_items = 1.
`item`	`const void *`	Pointer to the data to write into the queue. Must not be NULL. Size must match the queue's item_size.

Ritorna:
`true`: The item was successfully overwritten in the queue. `false`: The operation failed due to invalid input (NULL pointer or max_items != 1).

Note:

- **Blocking behavior**: This function never blocks. It always returns immediately after overwriting the item.
- **Thread safety**: The function is thread-safe because it uses a spinlock to protect access to the queue.
- **ISR usage**: Safe to call from an ISR because it does not block and uses IRQ-safe locking.
- **Preconditions**: The queue must be initialized with max_items = 1. The queue and item pointers must not be NULL.
- **Common mistakes**: Using this function on a queue with max_items > 1 will return false and not overwrite anything. Passing a NULL queue or item pointer will also return false.

Uso:

- **When you need to replace the single item in a static queue without blocking**: When you need to replace the single item in a static queue without blocking
- **In interrupt service routines where blocking is not allowed**: In interrupt service routines where blocking is not allowed

Esempio:

```
// This example shows how to overwrite a single item in a static queue and then immediately receive it. The queue is defined with max_items = 1, so overwrite is safe and immediate.
CAQUEUE_STATIC_DEFINE(mbox_queue, 1, sizeof(u32));

u32 value = 0x11U;
bool success = ca_queue_static_overwrite(&mbox_queue.q, &value);
if (!success) {
    // Handle error: invalid queue or item pointer, or max_items != 1
    return;
}

u32 received_value;
bool received = ca_queue_static_tryreceive(&mbox_queue.q, &received_value);
if (received) {
    // Successfully received the overwritten value
    // received_value now contains 0x11U
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_queue_static_overwrite instead of a standard queue push?
A: You should use ca_queue_static_overwrite when you need to replace the single item in a static queue without blocking. It is specifically designed for scenarios where you must update data immediately, such as in an ISR where blocking is prohibited.

Code example conf:0.90 **Q:** How do I implement a simple overwrite operation in caOS?
A: Assuming a queue 'my_q' initialized with max_items = 1, you can perform an overwrite as follows: u32 data = 0xDEADBEEF; bool success = ca_queue_static_overwrite(&my_q, &data);

Parameters conf:1.00 **Q:** What are the constraints on the parameters for ca_queue_static_overwrite?
A: The 'q' parameter must be a non-NULL pointer to a static queue initialized with max_items = 1. The 'item' parameter must be a non-NULL pointer to data that matches the queue's defined item_size. Providing a queue with max_items > 1 or passing NULL pointers will cause the function to fail.

Return conf:1.00 **Q:** What do the return values of ca_queue_static_overwrite indicate?
A: The function returns true if the item was successfully overwritten in the queue. It returns false if the operation failed, which occurs if the input pointers are NULL or if the queue was not initialized with max_items = 1.

`ca_queue_static_peek`

Header: `caos/inc/queue.h:201`
Source: `caos/src/queue.c:293:308`
GUARD: `EN_QUEUE`

```
bool ca_queue_static_peek(ca_queue_static_t *q, void *item)
```

Reads an item from the queue without removing it, returns false if queue is empty

Parametri:

Nome	Tipo	Descrizione
`q`	`ca_queue_static_t *`	Pointer to the queue structure, must not be NULL
`item`	`void *`	Pointer to buffer where item will be copied, must not be NULL

Ritorna:
`true`: Item was successfully read from queue (queue was not empty) `false`: Queue was empty or invalid parameters provided (NULL pointer)

Note:

- **Blocking behavior**: This function is NON-BLOCKING - it returns immediately whether queue has items or not
- **Thread safety**: Function is thread-safe as it uses spinlock to protect queue access
- **ISR usage**: Can be called from ISR context since it uses IRQ-protected spinlock
- **Preconditions**: Queue must be properly initialized before calling this function
- **Common mistakes**: Not checking return value, passing NULL pointers, buffer size mismatch

Uso:

- **Normal operation**: Used to inspect queue contents without consuming items, useful for debugging or peeking at data before processing
- **ISR context**: Can be safely called from interrupt service routines to peek at queue contents
- **Debugging**: Useful for monitoring queue state without affecting queue contents

Esempio:

```
// Example shows peeking at queue item without removing it, with proper error handling for buffer allocation and peek result
ca_queue_static_t my_queue;
// ... queue initialization code ...

void* buffer = malloc(sizeof(my_item_type));
if (buffer == NULL) {
    // Handle allocation failure
    return;
}

// Peek at the next item in queue
if (ca_queue_static_peek(&my_queue, buffer)) {
    // Successfully read item, buffer now contains the peeked data
    printf("Item peeked: %d\n", *(int*)buffer);
    // Item remains in queue for later retrieval
} else {
    // Queue was empty
    printf("Queue is empty\n");
}

free(buffer);
```

Q&A — fine-tuning review

6/6 pairs · qwen3-coder:latest

Usage conf:0.95 **Q:** When should I use ca_queue_static_peek instead of ca_queue_static_get?
A: You should use ca_queue_static_peek when you need to inspect the next element in the queue without removing it. This is useful for checking the next item to process or for implementing lookahead logic without altering the queue state.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_queue_static_peek with caOS types?
A: ca_queue_static_t q; int data = 42; ca_queue_static_init(&q, sizeof(int), 5); ca_queue_static_put &data); int peeked_data; bool result = ca_queue_static_peek(&q, &peeked_data); // result is true, peeked_data is 42

Parameters conf:0.95 **Q:** What are the valid and invalid values for the 'q' and 'item' parameters?
A: The 'q' parameter must be a valid pointer to an initialized ca_queue_static_t object; otherwise, the function returns false. The 'item' parameter must point to a buffer large enough to hold one element of the queue; otherwise, it causes invalid memory access.

Return conf:0.95 **Q:** What does it mean when ca_queue_static_peek returns false?
A: ca_queue_static_peek returns false if the queue is empty or if either parameter (q or item) is NULL. This indicates that no element could be read from the queue.

Error/Edge case conf:0.90 **Q:** What are common mistakes developers make when using ca_queue_static_peek'
A: Common mistakes include passing a NULL pointer for q or item, using a buffer too small for the element type, or calling the function on an uninitialized queue. These lead to undefined behavior or incorrect results.

Behavior conf:0.90 **Q:** Is ca_queue_static_peek blocking, thread-safe, and ISR-safe?
A: ca_queue_static_peek is non-blocking and returns immediately. It is thread-safe due to the use of a spinlock. It is also ISR-safe if the spinlock is configured for ISR use, as it uses ca_spinlock_lock_irq internally.

`ca_queue_static_receive`

Header: `caos/inc/queue.h:140`
Source: `caos/src/queue.c:219:239`
GUARD: `EN_QUEUE`

```
1 bool ca_queue_static_receive(ca_queue_static_t *q, void *item, u32
     timeout_ms)
```

Receives an item from a static queue, blocking until available or timeout expires.

Parametri:

Nome	Tipo	Descrizione
`q`	`ca_queue_static_t *`	Pointer to the static queue to receive from. Must not be NULL, otherwise function returns false.
`item`	`void *`	Pointer to memory where received item will be stored. Must not be NULL, otherwise function returns false.
`timeout_ms`	`u32`	Timeout in milliseconds. Use CA_WAIT_FOREVER (typically 0xFFFFFFFF) to wait indefinitely, or a positive value to wait up to that many milliseconds.

Ritorna:
`true`: An item was successfully received from the queue. `false`: Failed to receive an item due to timeout or invalid parameters (NULL pointer).

Note:

- **Blocking behavior**: This function blocks the calling thread until an item is available in the queue or the timeout expires. It is NOT non-blocking.
- **Thread-safe**: Yes, uses spinlock internally to protect queue access.
- **Callable from ISR**: No, this function is NOT safe to call from an ISR because it may block and uses spinlocks that can cause deadlocks if called from interrupt context.

- **Preconditions**: The queue must be initialized before calling this function. The item buffer must be valid and large enough to hold one item.
- **Common mistakes**: Using NULL for q or item parameter leads to immediate return of false. Using timeout = 0 with CA_WAIT_FOREVER can cause confusion; use CA_WAIT_FOREVER for indefinite wait.

Uso:

- **Waiting for network events in service loops**: Used in network services like ARP, DHCP, DNS, and NTP to wait for incoming events or requests before processing them.
- **Processing queued data in background tasks**: Typically used in infinite loops where a task waits for data to process, such as handling ARP requests or DNS queries.

Esempio:

```
// Example showing how to receive an item from a queue with indefinite wait. The buffer must be allocated and large enough to hold one item.
ca_queue_static_t *q = &some_queue;
void *buffer = malloc(sizeof(item_type));

// Wait indefinitely for an item
if (ca_queue_static_receive(q, buffer, CA_WAIT_FOREVER)) {
    // Process received item
    process_item(buffer);
} else {
    // Should not happen with CA_WAIT_FOREVER
    error_handler();
}
// Example showing how to use a timeout to avoid indefinite blocking. If no item arrives within 5 seconds, the function returns false.
u32 timeout = 5000; // 5 seconds
if (!ca_queue_static_receive(q, buffer, timeout)) {
    // Timeout occurred
    log("Queue receive timed out");
} else {
    // Item received successfully
    process_item(buffer);
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_queue_static_receive in a caOS application?
A: You should use ca_queue_static_receive when you need to receive data from a static queue in a blocking or timeout-based manner. It is ideal for task synchronization where one task waits for data produced by another task, or when you want to avoid indefinite blocking by setting a timeout.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_queue_static_receive with caOS types?
A: ca_queue_static_t my_queue; void *buffer; bool result = ca_queue_static_receive(&my_queue, buffer, 1000);

Parameters conf:0.95 **Q:** What does the timeout_ms parameter control in ca_queue_static_receive?
A: The timeout_ms parameter controls how long the function will wait for an item to become available in the queue. If set to 0, the function blocks indefinitely until an item is received. If set to a positive value, it will wait up to that many milliseconds before returning false.

Return conf:0.95 **Q:** What do the return values of ca_queue_static_receive indicate?
A: A return value of true indicates that an item was successfully received from the queue. A return value of false indicates either a timeout occurred or an error, such as invalid pointers or an uninitialized queue.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_queue_static_receive?
A: Common mistakes include passing null pointers for q or item, using the function without initializing the queue with ca_queue_static_init(), and calling it from an ISR, which is not allowed and may cause deadlock or undefined behavior.

Behavior conf:0.90 **Q:** Is ca_queue_static_receive thread-safe and can it be called from an ISR?
A: Yes, ca_queue_static_receive is thread-safe due to its use of a spinlock to protect access to the queue. However, it is not ISR-safe and must not be called from interrupt service routines as it may cause deadlock or undefined behavior.

`ca_queue_static_receive_isr`

Header: `caos/inc/queue.h:173`
Source: `caos/src/queue.c:246:249`
GUARD: `EN_QUEUE`

```
bool ca_queue_static_receive_isr(ca_queue_static_t *q, void *item)
```

Receives an item from a static queue in an ISR context without blocking.

Parametri:

Nome	Tipo	Descrizione
`q`	`ca_queue_static_t *`	Pointer to the static queue object. Must not be NULL, otherwise behavior is undefined.
`item`	`void *`	Pointer to the buffer where the received item will be stored. Must not be NULL, otherwise data corruption or crash may occur.

Ritorna:
`true`: An item was successfully received from the queue. The item is copied into the buffer pointed to by 'item'. `false`: No item was available in the queue. The buffer pointed to by 'item' was not modified.

Note:

- **Blocking**: This function does NOT block. It is a non-blocking operation that attempts to receive an item immediately. If the queue is empty, it returns immediately with false.
- **Thread-Safe**: This function is safe to call from an ISR context, but only if the queue is not accessed from a task context at the same time. Concurrent access from multiple threads or ISRs can cause race conditions.
- **Callable from ISR**: This function is specifically designed for use in ISRs. It is not safe to call from a task context because it does not support blocking or scheduling.
- **Preconditions**: The queue must be initialized before calling this function. The queue must be a static queue (not a heap-allocated one). The 'item' pointer must point to valid memory with sufficient space to hold the item.
- **Common Mistakes**: Using this function in a task context instead of an ISR can lead to incorrect behavior. Also, not checking the return value may cause the program to use uninitialized data.

Uso:

- **Interrupt Service Routine (ISR)**: Used in ISRs to quickly retrieve data from a queue without blocking. Commonly used when an interrupt signals that data is available in a queue.
- {'description': 'Paired with ca_queue_static_send_isr() for fast, non-blocking queue communication between tasks and ISRs.'}

Esempio:

```
// In an ISR, we attempt to receive data from a queue. If data is available, it's processed immediately. If not, the ISR returns without delay.
void UART_IRQHandler(void) {
    uint32_t received_data;
    // Try to receive data from a queue in ISR
    if (ca_queue_static_receive_isr(&rx_queue, &received_data)) {
        // Process the received data
        handle_received_data(received_data);
    }
    // If no data is available, nothing happens
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_queue_static_receive_isr() in an ISR context?
A: You should use ca_queue_static_receive_isr() in an ISR when you need to quickly retrieve an item from a static queue without blocking. It is ideal for handling interrupt-driven data, such as receiving data from a UART or SPI peripheral, where you want to avoid delays and ensure immediate processing of queued data. Since it's not blocking, it's suitable for time-critical ISR routines.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_queue_static_receive_isr() with caOS types?
A: ca_queue_static_t queue; void *buffer; bool success = ca_queue_static_receive_isr(&queue, buffer);

Parameters conf:0.90 **Q:** What are the valid and invalid values for the 'q' parameter in ca_queue_static_receive
A: The 'q' parameter must be a valid pointer to a pre-initialized static queue. If 'q' is NULL or points to uninitialized memory, the behavior is undefined. The queue must have been initialized using a function like ca_queue_static_init() before calling this function.

Return conf:0.95 **Q:** What does the return value of ca_queue_static_receive_isr() indicate, and what causes it to return false?
A: The function returns true if an item was successfully received from the queue. It returns false if the queue is empty at the time of the call. The function does not block or wait for an item to become available; it immediately returns false if no item is present.

`ca_queue_static_reset`

Header: `caos/inc/queue.h:207`
Source: `caos/src/queue.c:310:329`
GUARD: `EN_QUEUE`

```
void ca_queue_static_reset(ca_queue_static_t *q)
```

Clears all data from a static queue and wakes up any tasks waiting for data or space.

Parametri:

Nome	Tipo	Descrizione
`q`	`ca_queue_static_t *`	Pointer to the static queue structure to be reset. Must be a valid, initialized pointer.

Ritorna:
void

Note:

- This function is thread-safe as it uses internal spinlocks to protect the queue state during the reset process. It is NOT safe to call from an ISR because it performs task scheduling

operations (waking up blocked tasks). The function effectively 'flushes' the queue by resetting the head, tail, and count pointers to zero, and it forces any tasks currently blocked on ca_queue_static_receive or ca_queue_static_send to unblock immediately. A common mistake is calling this function while other threads are actively processing data, which can lead to race conditions or logic errors in the application state.

Uso:

- Use this function when you need to discard stale data, such as during a protocol state machine reset (e.g., DHCP discovery cycles) or when clearing a logging buffer. It is typically used in initialization or error-recovery sequences where the current contents of the queue are no longer valid. It is often paired with ca_queue_static_init to prepare a fresh queue or used standalone to clear an existing one before starting a new communication cycle.

Esempio:

```c
/* Example: Resetting a command queue during a system error recovery */
void handle_system_error(ca_queue_static_t *cmd_q) {
    /* Stop processing and clear all pending commands */
    ca_queue_static_reset(cmd_q);

    /* Log the event using the system stdout */
    ca_stdout_printf("System error: Command queue flushed.\n");

    /* Re-initialize or signal the system to resume normal operation */
    // Perform hardware reset or state machine re-entry here
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_queue_static_reset() in a caOS application?
A: You should call ca_queue_static_reset() when you need to fully reset the state of a static queue, such as during system initialization, after a fatal error, or when reinitializing a network service like DHCP. It clears the queue and wakes up all tasks waiting on it.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_queue_static_reset() with caOS types?
A: ca_queue_static_t my_queue; ca_queue_static_init(&my_queue, 10, sizeof(u32)); // ... use the queue ... ca_queue_static_reset(&my_queue);

Parameters conf:0.95 **Q:** What happens if I pass NULL to the ca_queue_static_reset() function?
A: If you pass NULL as the parameter 'q', the function will do nothing and return immediately. It is a safe operation to pass NULL, but it will not perform any reset action.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_queue_static_reset()?
A: Common mistakes include calling this function from an ISR, which is not thread-safe and may cause issues due to spinlock usage. Also, calling it while other tasks are actively using the queue can lead to unexpected behavior in those tasks since they will be woken up.

Behavior conf:0.90 **Q:** Is ca_queue_static_reset() blocking or non-blocking, and is it safe to call from an ISR?
A: The function is blocking because it acquires a lock before modifying the queue and releases it afterward. It is not safe to call from an ISR because it uses spinlock and the scheduler, which are not ISR-safe. It must be called in a privileged context.

---

ca_queue_static_send

Header: caos/inc/queue.h:132
Source: caos/src/queue.c:196:216
GUARD: EN_QUEUE

```
bool ca_queue_static_send(ca_queue_static_t *q, const void *item, u32
    timeout_ms)
```

Sends an item to a static queue, blocking until space is available or timeout occurs.

Parametri:

Nome	Tipo	Descrizione
q	ca_queue_static_t *	Pointer to the static queue structure. Must not be NULL, otherwise function returns false.
item	const void *	Pointer to the data to send. Must not be NULL, otherwise function returns false.
timeout_ms	u32	Timeout in milliseconds. Use CA_WAIT_FOREVER (typically 0xFFFFFFFF) to wait indefinitely. Any other value must be a valid u32.

Ritorna:
`true`: Item was successfully sent to the queue. `false`: Failed to send item due to timeout or invalid parameters (NULL queue or item).

Note:

- This function is **blocking** if timeout is not CA_WAIT_FOREVER. It will wait until space is available in the queue or until timeout expires.
- This function is **not safe to call from an ISR** because it may block and uses spinlocks and timeouts.
- The function is **not thread-safe** in the sense that it does not protect against concurrent access from multiple threads without proper synchronization.
- Precondition: The queue must be initialized before calling this function.
- Common mistake: Passing a NULL pointer for q or item leads to immediate return of false.
- The function uses a spinlock internally, so it should not be used in time-critical ISRs or contexts where spinlocks are not allowed.

Uso:

- Used in producer-consumer patterns where a thread needs to send data to a queue and must wait if the queue is full.
- Typically paired with ca_queue_static_receive() for bidirectional communication.
- Used in scenarios where a timeout is acceptable, such as sending data to a queue with a retry mechanism.
- Called in contexts where a thread must block until a queue has space, such as in a data logging or sensor data processing loop.

Esempio:

```
/* Example: Producer sending data to a queue with timeout */
Message msg = { .producer_id = 1, .sequence = 10, .value = 100 };
bool sent = ca_queue_static_send(&msg_queue.q, &msg, 500);  // Wait up to 500ms
if (!sent) {
    // Handle timeout or error
    ca_stdout_printf_n("Queue full, timeout occurred");
} else {
```

```
8     ca_stdout_printf_n("Message sent successfully");
9 }
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_queue_static_send to send data to a queue?
A: You should use ca_queue_static_send when you need to send data to a static queue in a blocking or timeout-based manner. It is ideal for inter-task communication where the sender must wait until space is available in the queue or until a timeout occurs.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_queue_static_send with caOS types?
A: ca_queue_static_t my_queue; void *queue_buffer[10]; ca_queue_static_init(&my_queue, queue_buffer, 10, sizeof(void*)); void *item = (void*)0x1234; bool result = ca_queue_static_send(&my item, 1000);

Parameters conf:0.95 **Q:** What does the timeout_ms parameter control in ca_queue_static_send?
A: The timeout_ms parameter controls how long the function will wait for space in the queue. If timeout_ms is 0, the function does not block and returns immediately. If timeout_ms is greater than 0, the function blocks until space becomes available or the timeout expires.

Return conf:0.95 **Q:** What do the return values of ca_queue_static_send indicate?
A: A return value of true indicates that the item was successfully sent to the queue. A return value of false indicates failure due to timeout, invalid pointers, or a full queue with no timeout.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_queue_static_send?
A: Common mistakes include passing null pointers for 'q' or 'item', using timeout_ms = 0 with a full queue which can cause indefinite blocking, and calling the function from an ISR, which is not safe.

Behavior conf:0.90 **Q:** Is ca_queue_static_send thread-safe and ISR-safe?
A: Yes, ca_queue_static_send is thread-safe as it uses a spinlock to protect concurrent access. However, it is not ISR-safe and should not be called from interrupt service routines because it may block and use functions that are not ISR-safe.

`ca_queue_static_send_front`

Header: `caos/inc/queue.h:182`
Source: `caos/src/queue.c:251:277`
GUARD: `EN_QUEUE`

```
1 bool ca_queue_static_send_front(ca_queue_static_t *q, const void *item,
     u32 timeout_ms)
```

Sends an item to the front of a static queue, blocking up to a timeout if the queue is full.

Parametri:

Nome	Tipo	Descrizione
`q`	`ca_queue_static_t *`	Pointer to the static queue. Must not be NULL, otherwise function returns false.
`item`	`const void *`	Pointer to the data to send. Must not be NULL, otherwise function returns false.
`timeout_ms`	`u32`	Timeout in milliseconds. If 0, function will not block and return immediately if queue is full. Valid values: 0 to any positive number.

Ritorna:

`true`: Item was successfully sent to the front of the queue. `false`: Failed to send item: either invalid parameters (q or item is NULL), or timeout occurred while waiting for space in the queue.

Note:

- **Blocking behavior**: This function blocks if the queue is full and a timeout is specified. It will not return until either space is available or the timeout expires.
- **Thread-safe**: Yes, uses spinlock to protect access to the queue.
- **Callable from ISR**: No, this function is NOT safe to call from an ISR. It may block and uses spinlocks that can cause issues in ISR context.
- **Preconditions**: Queue must be initialized and valid. The item pointer must point to valid memory.
- **Common mistakes**: Using NULL for q or item will cause immediate failure. Calling from ISR will lead to undefined behavior. Using timeout = 0 with a full queue will return false immediately without blocking.

Uso:

- **Normal thread context**: Used in threads to send data to the front of a queue, ensuring priority delivery. Commonly used in producer-consumer scenarios where some items must be processed first.
- **With timeout**: When a timeout is provided, it allows the thread to wait for space in the queue for a limited time before giving up. Useful in real-time systems where blocking indefinitely is not acceptable.
- **Paired with receive functions**: Typically used with ca_queue_static_receive() or ca_queue_static_tryreceive() to form a complete queue communication pattern.

Esempio:

```
// This example shows how to send an item to the front of a queue with a timeout. If the queue is full and no space becomes available within 100ms, the function returns false.
#include "caos.h"

void example_usage(void) {
    ca_queue_static_t my_queue;
    // Assume my_queue is initialized properly
    int data = 42;

    // Send item to front of queue with 100ms timeout
    bool success = ca_queue_static_send_front(&my_queue, &data, 100);

    if (success) {
        // Item was sent successfully
        printf("Item sent to front of queue\n");
    } else {
        // Either timeout or invalid parameters
        printf("Failed to send item\n");
    }
}
```

Q&A — fine-tuning review

6/6 pairs · qwen3-coder:latest

Usage conf:0.95 **Q:** When should I use ca_queue_static_send_front() in a real-time embedded system?

A: You should use ca_queue_static_send_front() when you need to insert an item at the front of a static queue in a thread-safe manner, especially in real-time systems where blocking behavior is acceptable. It is ideal for prioritizing items, such as high-priority tasks or urgent data packets, that must be processed before others in the queue.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_queue_static_send_front() with caOS types?
A: ca_queue_static_t queue; uint8_t data = 42; bool success = ca_queue_static_send_front(&queue, &data, 100); if (success) { // Item inserted successfully } else { // Insertion failed due to timeout or invalid parameters }

Parameters conf:0.90 **Q:** What are the valid and invalid values for the timeout_ms parameter in ca_queue_static_send_front()?
A: timeout_ms can be 0 or any positive value. If 0, the function will not block and will return immediately. If > 0, it specifies the maximum time to wait in milliseconds. Invalid values include negative numbers, though since u32 is unsigned, negative values are not possible in C. NULL for q or item parameters are invalid and will cause failure.

Return conf:0.95 **Q:** What does a return value of false from ca_queue_static_send_front() indicate?
A: A return value of false indicates that the item was not inserted into the queue. This can happen due to a timeout, if the queue is full and timeout_ms is 0, or if invalid parameters such as NULL q or item are passed.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_queue_static_send_front()?
A: Common mistakes include passing NULL for q or item, not initializing the queue properly, using timeout_ms = 0 in contexts where waiting is required, and calling the function from an ISR. Additionally, passing incorrect item sizes that do not match the queue's element size can lead to undefined behavior.

Behavior conf:0.90 **Q:** Is ca_queue_static_send_front() thread-safe and ISR-safe?
A: Yes, ca_queue_static_send_front() is thread-safe as it uses a spinlock to ensure mutual exclusion. However, it is not ISR-safe and must not be called from an ISR. For ISR usage, ca_queue_static_send_front_isr() should be used instead.

`ca_queue_static_send_isr`

Header: `caos/inc/queue.h:164`
Source: `caos/src/queue.c:241:244`
GUARD: `EN_QUEUE`

```
1 bool ca_queue_static_send_isr(ca_queue_static_t *q, const void *item)
```

Sends an item to a static queue from an ISR context without blocking.

Parametri:

Nome	Tipo	Descrizione
`q`	`ca_queue_static_t *`	Pointer to the static queue structure. Must not be NULL.
`item`	`const void *`	Pointer to the data item to send. Must not be NULL.

Ritorna:
`true`: Item was successfully sent to the queue. `false`: Failed to send item (queue is full or invalid parameters).

Note:

- • This function is NON-BLOCKING and can ONLY be called from ISR context.
- • It does NOT wait or timeout — it simply attempts to send and returns immediately.
- • It is NOT thread-safe because it's meant for ISR use only.
- • It CANNOT be called from regular threads or tasks.
- • It does NOT check for valid queue state or handle locking internally.

- • Common mistake: using this function outside of an ISR context — it will likely crash or behave unpredictably.

Uso:

- • Used in ISRs to quickly pass data (like characters from UART) to a queue.
- • Used when you want to avoid blocking in an ISR (e.g., logging or capturing input).
- • Pairs with functions like ‘ca_queue_static_trysend()‘ or ‘ca_queue_static_receive_isr()‘.
- • Typically used in embedded systems where real-time response is critical.

Esempio:

```
/* Example: Sending a character from UART ISR to a queue */
void UART_IRQHandler(void) {
char c = USART_ReceiveData(USART1);  // Get received byte
if (!ca_queue_static_send_isr(&g_uart_queue.q, &c)) {
// Queue full     drop the character
g_uart_dropped++;
}
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_queue_static_send_isr and why is it different from ca_queue_static_send?
A: ca_queue_static_send_isr is designed to be called from an ISR (Interrupt Service Routine) and is non-blocking. It should be used when you need to send data from an interrupt handler to a queue without risking blocking the interrupt. Unlike ca_queue_static_send, which is meant for tasks and may block, ca_queue_static_send_isr is specifically optimized for ISR usage and returns immediately with a boolean indicating success or failure.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_queue_static_send_i with caOS types?
A: ca_queue_static_t queue; void *item = (void *)0x1234; ca_queue_static_init(&queue, sizeof(u32), 5); bool result = ca_queue_static_send_isr(&queue, item); if (!result) { // Handle queue full condition }

Parameters conf:0.90 **Q:** What are the valid and invalid values for the ’q’ and ’item’ parameters in ca_queue_static_send_isr?
A: The ’q’ parameter must be a valid pointer to an already initialized ca_queue_static_t structure. Passing a null or uninitialized queue will cause a crash. The ’item’ parameter must be a valid pointer to the data to be sent. If item is NULL or points to invalid memory, the behavior is undefined.

Return conf:0.95 **Q:** What does the return value of ca_queue_static_send_isr indicate and what causes it to return false?
A: The function returns true if the item was successfully sent to the queue. It returns false if the queue is full and the item could not be sent. This is a non-blocking operation, so it does not wait for space to become available; it simply checks if the queue has room and returns immediately.

`ca_queue_static_tryreceive`

Header: `caos/inc/queue.h:155`
Source: `caos/src/queue.c:173:194`
GUARD: `EN_QUEUE`

```
bool ca_queue_static_tryreceive(ca_queue_static_t *q, void *item)
```

Attempts to receive an item from a static queue without blocking, returning true if successful.

Parametri:

Nome	Tipo	Descrizione
`q`	`ca_queue_static_t *`	Pointer to the static queue object. Must not be NULL, otherwise function returns false.
`item`	`void *`	Pointer to memory where the received item will be copied. Must not be NULL, otherwise function returns false.

Ritorna:
`true`: An item was successfully received from the queue. The item is copied into the memory pointed to by 'item'. `false`: The queue was empty, so no item could be received. This is NOT an error condition.

Note:

- This function is NON-BLOCKING: it will return immediately whether or not an item is available.
- This function is NOT thread-safe in itself, but uses a spinlock to protect access to the queue, so it's safe to call from multiple threads.
- This function CANNOT be called from an ISR (interrupt service routine) because it uses spinlocks that may not be safe in interrupt context.
- Precondition: The queue must be initialized before calling this function.
- Common mistake: Assuming that a return value of false means an error occurred; it simply means the queue was empty.

Uso:

- Used in polling loops where you want to check for data without waiting.
- Used in scenarios where you want to process items as they arrive, but don't want to block if none are present.
- Often paired with other queue functions like ca_queue_static_tryreceive(), ca_queue_static_send(), or ca_queue_static_overwrite().
- Used in low-priority tasks or background loops where you want to avoid blocking.

Esempio:

```
u32 received_value;
bool success = ca_queue_static_tryreceive(&my_queue, &received_value);
if (success) {
    // Process the received value
    printf("Received: %lu\n", received_value);
} else {
    // Queue was empty, no item received
    printf("Queue empty, no item received\n");
}
u8 buffer[10];
size_t index = 0;
while (index < sizeof(buffer) && ca_queue_static_tryreceive(&rx_queue, &buffer[
    index])) {
    index++;
}
if (index > 0) {
    // Process received data
    process_data(buffer, index);
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_queue_static_tryreceive() instead of a blocking receive function?
A: You should use ca_queue_static_tryreceive() when you want to check for available data in a queue without blocking the calling thread. It's useful in scenarios where you need to poll a queue periodically or avoid deadlocks in time-sensitive code. Since it's non-blocking, it's ideal for use in ISRs or high-priority tasks where blocking is not acceptable.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_queue_static_tryreceive() with caOS types?
A: ca_queue_static_t q; ca_queue_static_init(&q, 10, sizeof(u32)); u32 data = 0; if (ca_queue_static_tryreceive(&q, &data)) { // Successfully received data // data now contains the value from the queue } else { // Queue was empty }

Parameters conf:0.90 **Q:** What are the valid values for the 'q' and 'item' parameters in ca_queue_static_tryrece
A: The 'q' parameter must be a valid pointer to a pre-initialized ca_queue_static_t structure. The 'item' parameter must be a valid pointer to memory that is large enough to hold one element of the queue's data type. Passing NULL for either parameter will cause the function to return false.

Return conf:0.95 **Q:** What does it mean when ca_queue_static_tryreceive() returns false?
A: A return value of false indicates that either the queue was empty or one of the parameters (q or item) was invalid. It does not indicate an error in the queue's operation, but rather that no data was available to be received at that moment.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_queue_static_tryreceive()?
A: Common mistakes include passing NULL pointers for q or item, using uninitialized queues, or providing an item buffer that is too small to hold a queue element. Additionally, using the function on a queue that has not been properly initialized can lead to undefined behavior.

Behavior conf:0.90 **Q:** Is ca_queue_static_tryreceive() thread-safe and ISR-safe?
A: Yes, ca_queue_static_tryreceive() is both thread-safe and ISR-safe. It uses a spinlock mechanism that protects access to the queue, making it safe to call from multiple threads or from interrupt service routines.

`ca_queue_static_trysend`

Header: `caos/inc/queue.h:148`
Source: `caos/src/queue.c:148:169`
GUARD: `EN_QUEUE`

```
1 bool ca_queue_static_trysend(ca_queue_static_t *q, const void *item)
```

Attempts to push an item into a static queue without blocking if the queue is full.

Parametri:

Nome	Tipo	Descrizione
`q`	`ca_queue_static_t *`	Pointer to the pre-initialized static queue structure. Must not be NULL.
`item`	`const void *`	Pointer to the data to be copied into the queue. Must not be NULL.

Ritorna:
Returns true if the item was successfully added to the queue. Returns false if the queue is full, if the input pointers are NULL, or if the operation could not be completed immediately.

Note:

- This function is non-blocking, meaning it returns immediately regardless of whether the operation succeeded or failed. It is thread-safe because it uses an internal spinlock to protect the queue's internal state during the copy operation. It is safe to call from an ISR context because it uses interrupt-safe locking mechanisms. The most common beginner mistake is ignoring the return value; if the queue is full, the data is simply dropped, which can lead to silent data loss in your application if you do not handle the false return case.

Uso:

- Use this function when you need to send data to a task but cannot afford to wait (block) if the queue is full, such as in high-frequency event handlers or time-critical communication loops. It is typically used in a producer-consumer pattern where the producer checks the return value to decide whether to retry, log an error, or discard the packet. It is often paired with ca_queue_static_init for setup and ca_queue_static_receive or ca_queue_static_tryreceive on the consumer side to process the data.

Esempio:

```c
/* Example: Sending an event to a processing task */
ca_queue_static_t my_queue;

void send_event_to_task(int event_id) {
    /* Attempt to send the event without blocking the current task */
    if (!ca_queue_static_trysend(&my_queue, &event_id)) {
        /* Queue is full, handle the error to prevent silent data loss */
        ca_printf("Error: Event queue full, dropping event %d\n", event_id);
    } else {
        /* Event successfully queued and receiver notified */
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_queue_static_trysend() instead of a blocking send function?
A: You should use ca_queue_static_trysend() when you need to send an item to a queue without blocking the calling thread or ISR. It is ideal for scenarios where you want to immediately dispatch data if the queue has space, but do not want to wait if it's full. This is useful in time-critical applications or when you want to avoid potential deadlocks.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_queue_static_trysend() with caOS types?
A: ca_queue_static_t queue; uint32_t data = 42; ca_queue_static_init(&queue, sizeof(uint32_t), 5); bool success = ca_queue_static_trysend(&queue, &data); if (success) { // Item was sent successfully } else { // Queue was full or invalid parameters }

Parameters conf:0.95 **Q:** What are the valid and invalid values for the 'q' and 'item' parameters in ca_queue_static_trysend()?
A: The 'q' parameter must be a valid pointer to a previously initialized ca_queue_static_t object. If it is NULL, the function returns false. The 'item' parameter must be a valid pointer to the data to be sent. If it is NULL, the function also returns false. Both parameters must be valid and compatible with the queue's configuration.

Return conf:0.95 **Q:** What does it mean when ca_queue_static_trysend() returns false?
A: A return value of false indicates either that the queue is full or that one of the parameters (q or item) was invalid (i.e., NULL). It does not indicate a failure to send data, but rather that the operation could not be completed due to the queue being full or invalid input.

Error/Edge case conf:0.90 **Q:** What are common mistakes developers make when using ca_queue_static_tryse
A: Common mistakes include not checking the return value of the function, which can lead to data loss or unexpected behavior. Another mistake is passing NULL pointers for

q or item, which causes an immediate return of false. Additionally, calling this function on an uninitialized queue will result in undefined behavior or failure.

Behavior conf:0.90 **Q:** Is ca_queue_static_trysend() thread-safe and ISR-safe, and what are its blocking characteristics?
A: Yes, ca_queue_static_trysend() is thread-safe because it uses a spinlock to protect access to the queue. It is also ISR-safe if the spinlock and queue are configured with ca_spinlock_irq_init and ca_spinlock_lock_irq. The function is non-blocking: it returns immediately without waiting, even if the queue is full.

`ca_rmutex_get_count`

Header: `caos/inc/sync.h:150`
Source: `caos/src/sync.c:381:386`
GUARD: `EN_SYNC`

```
u16 ca_rmutex_get_count(ca_recursive_mutex_t *rmtx)
```

Returns the current recursion depth (lock count) of a recursive mutex.

Parametri:

Nome	Tipo	Descrizione
`rmtx`	`ca_recursive_mutex_t *`	Pointer to the initialized recursive mutex object. Must not be NULL.

Ritorna:
Returns a u16 representing the number of times the current task has locked the mutex. Returns 0 if the mutex is currently unlocked.

Note:

- This function is non-blocking and thread-safe because it uses an internal spinlock to protect the counter during the read operation. It is safe to call from an ISR, although it is primarily intended for debugging or diagnostic purposes within tasks. The most common beginner mistake is assuming this value represents the total number of tasks waiting for the mutex; it only tracks the recursion depth of the current owner. Ensure the mutex has been properly initialized with ca_rmutex_init before calling this function, otherwise, the behavior is undefined.

Uso:

- This function is typically used for diagnostic logging or conditional logic in complex systems where a resource is accessed recursively. It is often paired with ca_rmutex_lock and ca_rmutex_unlock to verify that the recursion depth is behaving as expected during development. In a typical sequence, you initialize the mutex with ca_rmutex_init, perform nested operations using ca_rmutex_lock, and use ca_rmutex_get_count to inspect the state for debugging or safety checks before finally releasing the lock.

Esempio:

```
/* Example: Monitoring recursive lock depth */
caRecursiveMutex my_rmtx;

void recursive_task_logic(int depth) {
    ca_rmutex_lock(&my_rmtx);

    u16 current_depth = ca_rmutex_get_count(&my_rmtx);
    ca_printf_n("Current recursion level: %u", current_depth);

```

```
    if (depth > 0) {
        recursive_task_logic(depth - 1);
    }

    ca_rmutex_unlock(&my_rmtx);
}

void app_main(void) {
    ca_rmutex_init(&my_rmtx);
    recursive_task_logic(3);
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_rmutex_get_count() in my embedded application?
A: You should use ca_rmutex_get_count() when you need to determine how many times the current task has acquired a recursive mutex. This is useful for debugging or ensuring proper mutex acquisition/release patterns in recursive locking scenarios.

Code example conf:0.90 **Q:** Can you show a minimal example of using ca_rmutex_get_count() with caOS types?
A: ca_recursive_mutex_t rmtx; ca_rmutex_init(&rmtx); ca_rmutex_lock(&rmtx); ca_rmutex_lock(&
u16 count = ca_rmutex_get_count(&rmtx); // count will be 2 here

Parameters conf:0.90 **Q:** What is the significance of the rmtx parameter in ca_rmutex_get_count() and what happens if it's invalid?
A: The rmtx parameter is a pointer to a pre-initialized recursive mutex structure. If this pointer is NULL or points to an uninitialized mutex, the behavior is undefined and may cause a crash or unpredictable results.

Return conf:0.85 **Q:** What do the return values of ca_rmutex_get_count() indicate and what causes it to fail?
A: The function returns 0 if the mutex is not acquired by the current task, or a positive integer indicating how many times it has been acquired. It does not fail in the traditional sense; however, calling it on an uninitialized mutex may result in undefined behavior.

`ca_rmutex_init`

Header: `caos/inc/sync.h:107`
Source: `caos/src/sync.c:252:257`
GUARD: `EN_SYNC`

```
void ca_rmutex_init(ca_recursive_mutex_t *rmtx)
```

Initializes a recursive mutex structure for thread synchronization.

Parametri:

Nome	Tipo	Descrizione
`rmtx`	`ca_recursive_mutex_t *`	Pointer to the recursive mutex object to be initialized. Must point to valid, allocated memory.

Ritorna:
void

Note:

- This function is non-blocking and performs the setup of internal synchronization primitives (wait queue and spinlock). It must be called before any other ca_rmutex_* function is used on the object. It is not thread-safe if called while other tasks are already attempting to use the same mutex, so it should be called during system initialization or before the

mutex is shared between tasks. A common mistake is to re-initialize a mutex that is already in use, which will corrupt the internal state and lead to unpredictable deadlocks or race conditions.

Uso:

- Use this function during the system startup phase, typically inside ca_os_main or an initialization task, to prepare a shared resource lock. It is the mandatory first step in the lifecycle of a recursive mutex (init -> lock/trylock -> unlock). Unlike a standard mutex, a recursive mutex allows the same thread to acquire the lock multiple times without deadlocking itself, making it ideal for protecting complex functions that call other functions which also require the same lock. It is typically paired with ca_rmutex_lock or ca_rmutex_timedlock for access control and ca_rmutex_unlock to release the resource.

Esempio:

```c
static ca_recursive_mutex_t rmtx;

static void protected_task(void *arg) {
    // Acquire the lock; safe to call even if already held by this thread
    ca_rmutex_lock(&rmtx);

    // Perform protected operations
    // ...

    ca_rmutex_unlock(&rmtx);
}

void ca_os_main(void) {
    // Initialize the mutex before starting tasks
    ca_rmutex_init(&rmtx);

    // Create tasks that will share the mutex
    ca_os_job(protected_task, NULL, 512, 1, "task_name");

    // Start the OS scheduler
    ca_os_run();
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_rmutex_init and why is it important for recursive mutexes?
A: You should call ca_rmutex_init before any task attempts to use the recursive mutex. This function initializes the mutex structure so it can be safely used for protecting shared resources. It is important because the mutex is not ready for use until initialized, and calling this function ensures that the mutex allows a single task to acquire it multiple times, which is essential for nested critical sections.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_rmutex_init with caOS types?
A: ca_recursive_mutex_t myMutex; ca_rmutex_init(&myMutex);

Parameters conf:0.85 **Q:** What is the significance of the ca_recursive_mutex_t* rmtx parameter in ca_rmutex_init, and what happens if it is NULL?
A: The ca_recursive_mutex_t* rmtx parameter is a pointer to the mutex structure that needs to be initialized. It must be a valid, allocated pointer; passing NULL will result in undefined behavior since the function does not check for NULL and cannot initialize a non-existent structure. The function is non-blocking and non-thread-safe, so it must be called from a task context, not an ISR.

---

ca_rmutex_is_owner

Header: caos/inc/sync.h:135
Source: caos/src/sync.c:346:355
GUARD: EN_SYNC

```
bool ca_rmutex_is_owner(ca_recursive_mutex_t *rmtx)
```

Checks if the current thread is the owner of the specified recursive mutex.

Parametri:

Nome	Tipo	Descrizione
rmtx	ca_recursive_mutex_t *	Pointer to the recursive mutex object to check. Must be initialized via ca_rmutex_init before use.

Ritorna:
Returns true if the current thread currently holds the mutex (lock_count > 0 and owner matches), otherwise returns false.

Note:

- This function is thread-safe and uses internal spinlocks to ensure data consistency while reading the mutex state. It is non-blocking, meaning it returns immediately after checking the internal status. It is safe to call from a thread context, but it is generally not useful to call from an ISR because an ISR cannot 'own' a mutex in the same way a thread does. The most common mistake is assuming that a 'false' return means the mutex is free; it only means the current thread does not own it—another thread might still hold it.

Uso:

- This function is typically used in complex systems where a resource might be accessed by multiple functions, and you need to verify if the current execution path already holds the necessary lock to prevent deadlocks or redundant locking. It is often used in 'defensive programming' patterns: before performing a sensitive operation, a function checks if it already owns the mutex. If it does, it proceeds; if not, it might attempt to acquire it using ca_rmutex_lock. It is part of the standard synchronization lifecycle: init -> lock -> [is_owner check] -> unlock -> destroy.

Esempio:

```
/* Assume rmtx is a global or shared recursive mutex */

void safe_resource_update(ca_recursive_mutex_t *rmtx) {
    /* Check if we already own the lock before trying to acquire it */
    if (!ca_rmutex_is_owner(rmtx)) {
        /* We don't own it, so we must block until we get it */
        ca_rmutex_lock(rmtx);
    }

    /* Perform critical section operations here */
    /* ... */

    /* Release the lock now that the work is done */
    ca_rmutex_unlock(rmtx);
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_rmutex_is_owner() in my embedded application?

A: You should use ca_rmutex_is_owner() when you need to verify if the currently executing thread owns a recursive mutex. This is particularly useful in scenarios where you want to avoid unnecessary lock/unlock operations or when debugging recursive locking issues. It helps ensure that a thread doesn't attempt to unlock a mutex it doesn't own, which could lead to undefined behavior.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_rmutex_is_owner() with caOS types?
A: ca_recursive_mutex_t rmtx; ca_rmutex_init(&rmtx); ca_rmutex_lock(&rmtx); bool is_owner = ca_rmutex_is_owner(&rmtx); // is_owner should be true here ca_rmutex_unlock(&rmtx)

Parameters conf:0.90 **Q:** What does the 'rmtx' parameter represent, and what are valid values for it?
A: The 'rmtx' parameter is a pointer to a ca_recursive_mutex_t structure that must be initialized before calling this function. Valid values include any pointer to a properly initialized recursive mutex. Invalid values include NULL pointers or pointers to uninitialized mutex structures, which will result in undefined behavior.

Return conf:0.85 **Q:** What do the return values of ca_rmutex_is_owner() indicate, and what causes it to return false?
A: The function returns true if the current thread is the owner of the mutex, and false if the current thread is not the owner or if the parameter is NULL. It returns false when the mutex is locked by another thread, even if that thread is the owner, because the function only checks ownership by the calling thread. It also returns false if a NULL pointer is passed as the parameter.

`ca_rmutex_lock`

Header: `caos/inc/sync.h:114`
Source: `caos/src/sync.c:259:289`
GUARD: `EN_SYNC`

```
1 void ca_rmutex_lock(ca_recursive_mutex_t *rmtx)
```

Acquires a recursive mutex, allowing the same thread to lock it multiple times without deadlocking.

Parametri:

Nome	Tipo	Descrizione
`rmtx`	`ca_recursive_mutex_t *`	Pointer to the recursive mutex object. Must be initialized via ca_rmutex_init before use.

Ritorna:
void. The function blocks the calling thread until the mutex is successfully acquired.

Note:

- This function is blocking: if the mutex is held by another thread, the calling thread is suspended and placed in a wait queue until the owner releases it. It is NOT safe to call from an ISR because it triggers a context switch. The recursive nature means the same thread can call this function multiple times; each call increments an internal counter, and the thread must call ca_rmutex_unlock an equal number of times to fully release the lock. A common mistake is forgetting to pair every lock with an unlock, which leads to the mutex remaining permanently locked for other threads. Always ensure the mutex pointer is valid and initialized before calling this function.

Uso:

- Used to protect shared resources (like global data structures or hardware peripherals) that might be accessed by nested functions within the same thread. It is typically used in a sequence: initialize the mutex once at system startup, then use ca_rmutex_lock before accessing the shared resource, and finally call ca_rmutex_unlock once the operation is complete. It is often paired with ca_rmutex_get_count for debugging or logic verification. Unlike standard mutexes, this allows a function to acquire the lock even if it is already held by the caller, preventing self-deadlock in complex call trees.

Esempio:

```
/* Global recursive mutex instance */
caRecursiveMutex rmtx;

static void recursive_worker(int depth) {
    /* Acquire the lock; if already held by this thread, count increments */
    ca_rmutex_lock(&rmtx);

    ca_stdout_printf_n("Depth: %d, Lock count: %u", depth, ca_rmutex_get_count(&
    rmtx));

    if (depth > 0) {
        recursive_worker(depth - 1);
    }

    /* Release the lock; must match the number of locks acquired */
    ca_rmutex_unlock(&rmtx);
}

void task_entry(void *arg) {
    ca_rmutex_init(&rmtx);
    while (1) {
        recursive_worker(3);
        ca_os_delay_ms(1000);
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_rmutex_lock() in my caOS application?
A: You should use ca_rmutex_lock() when you need to protect shared resources from concurrent access by multiple tasks, especially when a single task might need to acquire the same mutex multiple times. This is useful in recursive scenarios where a function calls another function that also tries to lock the same mutex.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_rmutex_lock() with caOS types?
A: ca_recursive_mutex_t mtx; ca_rmutex_init(&mtx); ca_rmutex_lock(&mtx); // Critical section ca_rmutex_unlock(&mtx);

Parameters conf:0.90 **Q:** What is the significance of the rmtx parameter in ca_rmutex_lock()?
A: The rmtx parameter is a pointer to a ca_recursive_mutex_t that must be initialized with ca_rmutex_init() before calling ca_rmutex_lock(). Passing NULL causes undefined behavior, and using an uninitialized mutex also leads to unpredictable results.

Return conf:0.95 **Q:** Does ca_rmutex_lock() return any value, and what does it mean?
A: No, ca_rmutex_lock() does not return any value. It is a void function. If the mutex is available, it acquires it immediately. If not, the calling task is blocked until the mutex becomes available.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_rmutex_lock()?
A: Common mistakes include calling ca_rmutex_lock() with a NULL pointer, using an uninitialized mutex, or calling it from an ISR context. These can lead to undefined behavior or system crashes.

Behavior conf:0.90 **Q:** Is ca_rmutex_lock() blocking or non-blocking, and can it be called from an ISR?
A: ca_rmutex_lock() is a blocking function. It will block the calling task if the mutex is already acquired by another task. It is not safe to call from an ISR, as it is not ISR-safe and can cause system instability.

`ca_rmutex_timedlock`

Header: `caos/inc/sync.h:143`
Source: `caos/src/sync.c:357:379`
GUARD: `EN_SYNC`

```
bool ca_rmutex_timedlock(ca_recursive_mutex_t *rmtx, u32 timeout_ms)
```

Attempts to acquire a recursive mutex within a specified timeout period.

Parametri:

Nome	Tipo	Descrizione
`rmtx`	`ca_recursive_mutex_t *`	Pointer to the recursive mutex object. Must be initialized via ca_rmutex_init before use.
`timeout_ms`	`u32`	Maximum time in milliseconds to wait for the lock. Use 0 for a non-blocking poll.

Ritorna:
Returns true if the mutex was successfully acquired (or already held by the caller), false if the timeout expired before the lock could be obtained.

Note:

- This function is blocking; it will put the calling thread into a sleep state if the mutex is unavailable, allowing other tasks to run. It is NOT safe to call from an ISR because ISRs cannot sleep. Recursive mutexes allow the same thread to lock the mutex multiple times without deadlocking itself, provided it calls ca_rmutex_unlock an equal number of times. A common mistake is forgetting to initialize the mutex or calling this from an interrupt context, which will trigger a system safeguard failure.

Uso:

- Use this when you need to protect a shared resource (like a data structure or peripheral) that might be accessed by multiple tasks. It is typically used in a pattern where you initialize the mutex once at system startup, then call ca_rmutex_timedlock before accessing the resource, and finally call ca_rmutex_unlock once the operation is complete. The timed version is preferred over the infinite-wait version to prevent system-wide deadlocks if a resource becomes permanently unavailable.

Esempio:

```
ca_recursive_mutex_t my_mutex;

// Initialize the mutex before any task uses it
// ca_rmutex_init(&my_mutex);

void task_function(void *arg) {
    // Attempt to lock the resource for up to 50ms
    if (ca_rmutex_timedlock(&my_mutex, 50)) {
        // Successfully acquired, safe to access shared resource
        // ... perform protected operations ...

        ca_rmutex_unlock(&my_mutex);
```

```
    } else {
        // Failed to acquire lock within 50ms, handle error or retry
        ca_printf("Resource busy, skipping operation\n");
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_rmutex_timedlock instead of a regular mutex lock?
A: You should use ca_rmutex_timedlock when you need to avoid indefinite blocking, especially in time-sensitive applications. It allows you to specify a timeout, so if the mutex is not available within the given time, the function returns false instead of hanging forever.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_rmutex_timedlock in a caOS application?
A: ca_recursive_mutex_t rmtx; ca_rmutex_init(&rmtx); bool locked = ca_rmutex_timedlock(&rmtx, 1000); // Wait up to 1 second if (locked) { // Critical section ca_rmutex_unlock(&rmtx); }

Parameters conf:0.95 **Q:** What does the timeout_ms parameter control in ca_rmutex_timedlock?
A: The timeout_ms parameter specifies the maximum time (in milliseconds) the calling thread will wait for the mutex to become available. If timeout_ms is 0, the function returns immediately if the mutex is not available. Any positive value sets a maximum wait time before returning false.

Return conf:0.90 **Q:** What does it mean when ca_rmutex_timedlock returns false?
A: ca_rmutex_timedlock returns false if the mutex could not be acquired within the specified timeout. This can happen either because the timeout expired or because an error occurred, such as the mutex not being properly initialized.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_rmutex_timedlock?
A: Common mistakes include calling this function from an ISR, which is not allowed, or using a mutex that hasn't been initialized. Also, using timeout_ms = 0 without checking the return value can lead to unexpected behavior, as it will return false immediately if the mutex is already taken.

Behavior conf:0.90 **Q:** Is ca_rmutex_timedlock thread-safe and can it be called from an ISR?
A: ca_rmutex_timedlock is thread-safe and requires a valid scheduler context. However, it is not ISR-safe and must not be called from an Interrupt Service Routine. It may block and use the scheduler, which is not allowed in ISR context.

`ca_rmutex_trylock`

Header: `caos/inc/sync.h:121`
Source: `caos/src/sync.c:291:315`
GUARD: `EN_SYNC`

```
bool ca_rmutex_trylock(ca_recursive_mutex_t *rmtx)
```

Attempts to acquire a recursive mutex without blocking the calling task.

Parametri:

Nome	Tipo	Descrizione
`rmtx`	`ca_recursive_mutex_t *`	Pointer to the recursive mutex object. Must be initialized via ca_rmutex_init before use.

Ritorna:
Returns true if the mutex was successfully acquired (or if the task already owns it). Returns false if the mutex is currently held by another task.

Note:

- This function is non-blocking, meaning it returns immediately regardless of whether the lock was acquired. It is thread-safe and uses internal spinlocks to protect the mutex state during the check. It is NOT safe to call from an ISR because it relies on the scheduler to identify the current task context. A common mistake is assuming that a failed trylock means the system is deadlocked; in reality, it simply means the resource is busy, and your logic should handle this by either retrying later or performing an alternative action.

Uso:

- Use this function when you need to access a shared resource but cannot afford to wait (block) if another task is currently using it. It is typically used in non-critical background tasks or state machines where you want to 'poll' for resource availability. It is often paired with ca_rmutex_lock for mandatory access or ca_rmutex_unlock to release the resource after successful acquisition. Always ensure the mutex is initialized before the first call.

Esempio:

```
ca_recursive_mutex_t rmtx;

// Initialize the mutex before use
ca_rmutex_init(&rmtx);

// Attempt to acquire the lock without waiting
if (ca_rmutex_trylock(&rmtx)) {
    // Successfully acquired, perform protected operations
    ca_stdout_printf_n("Mutex acquired, count: %u", ca_rmutex_get_count(&rmtx));

    // Release the lock when finished
    ca_rmutex_unlock(&rmtx);
} else {
    // Mutex is busy, perform other tasks instead of blocking
    ca_stdout_printf_n("Mutex busy, skipping task...");
    ca_os_yield();
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_rmutex_trylock instead of a blocking lock function?
A: You should use ca_rmutex_trylock when you need to acquire a recursive mutex immediately without waiting. It is useful in scenarios where you want to avoid potential deadlocks or when the operation must not block the calling task, such as in time-critical sections or when called from a context where blocking is not allowed.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_rmutex_trylock in a C function?
A: ca_recursive_mutex_t rmtx; ca_rmutex_init(&rmtx); if (ca_rmutex_trylock(&rmtx)) { // Mutex acquired successfully // Critical section here ca_rmutex_unlock(&rmtx); } else { // Failed to acquire mutex }

Parameters conf:0.95 **Q:** What is the significance of the rmtx parameter in ca_rmutex_trylock?
A: The rmtx parameter is a pointer to a recursive mutex that must be initialized using ca_rmutex_init(). It must not be NULL. If it is NULL, the function will return false, indicating failure to acquire the mutex.

Return conf:0.95 **Q:** What does it mean when ca_rmutex_trylock returns false?
A: When ca_rmutex_trylock returns false, it means that the mutex was already occupied by another task or that the input pointer was NULL. The function does not block; it simply fails to acquire the mutex immediately.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_rmutex_trylock?

A: Common mistakes include calling this function in an ISR context, which is not allowed, and expecting it to behave like a blocking lock. Also, using a mutex that has not been initialized with ca_rmutex_init() or passing a NULL pointer can lead to unexpected behavior or failure to acquire the mutex.

Behavior conf:0.90 **Q:** Is ca_rmutex_trylock thread-safe and can it be called from an ISR?

A: Yes, ca_rmutex_trylock is thread-safe and uses spinlock to protect access to the mutex structure. However, it cannot be called from an ISR because it uses functions that may block and are not compatible with ISR contexts.

`ca_rmutex_unlock`

Header: `caos/inc/sync.h:128`
Source: `caos/src/sync.c:317:344`
GUARD: `EN_SYNC`

```
void ca_rmutex_unlock(ca_recursive_mutex_t *rmtx)
```

Releases a recursive mutex previously acquired by the current task.

Parametri:

Nome	Tipo	Descrizione
`rmtx`	`ca_recursive_mutex_t *`	Pointer to the initialized recursive mutex handle. Must not be NULL.

Ritorna:
void. This function does not return a status code, but it triggers a system safeguard if the caller is not the current owner of the mutex.

Note:

- This function is thread-safe and manages the internal lock count of the recursive mutex. It is NOT callable from an ISR because it may trigger a context switch if a waiting task is unblocked. The mutex must be initialized via ca_rmutex_init before use. A common mistake is to call this function fewer times than the lock was acquired; since it is recursive, the mutex is only truly released when the internal lock count reaches zero. If you call this without being the owner, the system will trigger a safeguard failure, which typically halts the task or the system for debugging purposes.

Uso:

- This function is used to release shared resources protected by a recursive mutex. It is typically used in a 'lock-do-unlock' pattern where the task performs a series of operations that might call other functions that also attempt to lock the same mutex. You must ensure that every successful call to ca_rmutex_lock or ca_rmutex_timedlock is balanced by a corresponding call to ca_rmutex_unlock. It is often paired with ca_rmutex_get_count to verify the nesting level during debugging or complex state management.

Esempio:

```
ca_recursive_mutex_t my_rmtx;

// Initialize the mutex before use
ca_rmutex_init(&my_rmtx);

void process_data(void) {
    // Acquire the mutex with a timeout
    if (ca_rmutex_timedlock(&my_rmtx, 100)) {
        // Perform protected operations
```

```
        ca_printf_n("Mutex count: %u", ca_rmutex_get_count(&my_rmtx));

        // Release the mutex
        ca_rmutex_unlock(&my_rmtx);
    } else {
        // Handle timeout error
        ca_printf_n("Failed to acquire mutex");
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_rmutex_unlock() in my caOS application?
A: You should call ca_rmutex_unlock() when you want to release a recursive mutex that was previously acquired by the same task. It is typically called at the end of a critical section where you have acquired the mutex using ca_rmutex_lock(). This allows other tasks waiting on the mutex to proceed.

Code example conf:0.90 **Q:** Can you show a minimal example of using ca_rmutex_unlock() with caOS types?
A: ca_recursive_mutex_t rmtx; ca_rmutex_init(&rmtx); ca_rmutex_lock(&rmtx); // Critical section ca_rmutex_unlock(&rmtx);

Parameters conf:0.95 **Q:** What is the significance of the rmtx parameter in ca_rmutex_unlock()?
A: The rmtx parameter is a pointer to the recursive mutex that needs to be released. It must be a valid, non-NULL pointer. If it is NULL, the system will crash due to CA_SAFEGUARD. The mutex must have been previously acquired by the calling task.

Return conf:0.90 **Q:** Does ca_rmutex_unlock() return any value, and what does it signify?
A: No, ca_rmutex_unlock() does not return any value. It is a void function. It does not fail under normal conditions as long as the parameter is valid and the mutex was correctly acquired by the calling task.

Error/Edge case conf:0.85 **Q:** What are the common mistakes or edge cases when using ca_rmutex_unlock()?
A: Common mistakes include calling ca_rmutex_unlock() with a NULL pointer, calling it from a different task than the one that acquired the mutex, or releasing the mutex more times than it was acquired. These conditions will cause a crash due to CA_SAFEGUARD.

Behavior conf:0.90 **Q:** Is ca_rmutex_unlock() blocking or non-blocking, and can it be called from an ISR?
A: ca_rmutex_unlock() is a non-blocking function. It does not wait for anything and returns immediately. It must be called from task context, not from an ISR, as it is not ISR-safe.

`ca_rng_fill`

Header: `caos/drivers/inc/rng.h:28`
Source: `caos/drivers/src/rng.c:49:68`
GUARD: `EN_RNG`

```
bool ca_rng_fill(u8 *buf, u32 len)
```

Fills a buffer with cryptographically secure random bytes generated by the hardware RNG.

Parametri:

Nome	Tipo	Descrizione
`buf`	`u8 *`	Pointer to the destination buffer. Must not be NULL.
`len`	`u32`	Number of bytes to generate. Must be greater than 0.

Ritorna:
Returns true if the buffer was successfully filled with random data, false if the input parameters are invalid or if the hardware RNG fails to provide data.

Note:

- This function is thread-safe as it uses an internal mutex to protect the hardware RNG peripheral access. It is a blocking function; if another task is currently using the RNG, the calling task will wait until the mutex is released. It must NOT be called from an ISR (Interrupt Service Routine) because it attempts to lock a mutex, which can cause a system deadlock or kernel panic. Ensure the RNG peripheral is initialized before calling this function. A common mistake is passing a NULL pointer or a length of zero, which will immediately return false without modifying memory.

Uso:

- This function is typically used to generate non-deterministic data for security-sensitive operations, such as creating Initialization Vectors (IVs) for AES encryption or generating seeds for cryptographic key derivation. It is often paired with memory management functions to prepare buffers before passing them to crypto services. In a typical workflow, you allocate a buffer, call ca_rng_fill to populate it with entropy, and then pass that buffer to a function like ca_ed25519_public_from_seed or an AES encryption service.

Esempio:

```
u8 seed[32];

/* Attempt to generate a secure seed for key generation */
if (ca_rng_fill(seed, sizeof(seed))) {
    /* Seed generated successfully, proceed to generate public key */
    u8 public_key[32];
    ca_ed25519_public_from_seed(public_key, seed);
} else {
    /* Handle error: RNG failure or invalid parameters */
    ca_memset(seed, 0, sizeof(seed));
    /* Log error using ca_printf or similar */
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_rng_fill to generate random data in a caOS application?
A: You should use ca_rng_fill when you need cryptographically secure random data for applications such as generating keys, nonces, or secure tokens. It is ideal for scenarios where hardware-based randomness is required and thread safety is important.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_rng_fill with caOS types?
A: u8 random_buffer[32]; bool success = ca_rng_fill(random_buffer, 32); if (success) { // Use random_buffer } else { // Handle failure }

Parameters conf:0.95 **Q:** What are the valid and invalid values for the parameters of ca_rng_fill?
A: The buf parameter must be a valid, non-NULL pointer to a buffer. The len parameter must be greater than zero; otherwise, the function will return false. Passing NULL for buf or zero for len will cause an immediate failure.

Return conf:0.90 **Q:** What does it mean when ca_rng_fill returns false?
A: A return value of false indicates that the function failed to generate random data. This can happen due to invalid parameters such as a NULL buffer, zero length, or hardware errors during random number generation.

Error/Edge case conf:0.85 **Q:** What are common mistakes developers make when using ca_rng_fill?
A: Common mistakes include passing a NULL buffer or zero length to the function, not checking the return value for failure, and calling the function from an ISR, which is not allowed. Additionally, failing to ensure the RNG hardware is initialized can lead to errors.

Behavior conf:0.90 **Q:** Is ca_rng_fill blocking or non-blocking, and can it be called from an ISR?
A: ca_rng_fill is a blocking function because it waits for a mutex to be released if it's already occupied. It is not safe to call from an ISR, as it uses blocking operations and mutexes, which are not allowed in interrupt contexts.

`ca_rng_get`

Header: `caos/drivers/inc/rng.h:25`
Source: `caos/drivers/src/rng.c:40:47`
GUARD: `EN_RNG`

```
bool ca_rng_get(u32 *val)
```

Retrieves a 32-bit random number from the hardware generator.

Parametri:

Nome	Tipo	Descrizione
`val`	`u32 *`	Pointer to a 32-bit variable where the random number will be stored. Must not be NULL.

Ritorna:
Returns true if the random number was successfully generated and stored in 'val'. Returns false if 'val' is NULL or if the hardware generator failed to provide a value.

Note:

- This function is thread-safe because it uses an internal mutex to protect the hardware access, preventing race conditions if multiple tasks request random numbers simultaneously. It is a blocking function; if another task is currently accessing the hardware, the calling task will wait until the mutex is released. Do not call this function from an Interrupt Service Routine (ISR) because blocking operations are strictly forbidden in ISR context and will cause system instability. Ensure the hardware is properly initialized before use.

Uso:

- This function is typically used to generate non-deterministic values for security tokens, network transaction IDs (like DHCP XIDs), or to seed pseudo-random number generators. It is often used in a loop or during service initialization sequences. It pairs well with ca_rng_init to ensure the hardware is ready, and it is frequently used alongside ca_stdout_printf_n for debugging purposes to verify the randomness of the generated values.

Esempio:

```
u32 random_id = 0;

// Ensure the RNG hardware is initialized before use
```

```c
if (ca_rng_init()) {
    // Attempt to get a random number
    if (ca_rng_get(&random_id)) {
        ca_stdout_printf_n("Generated ID: 0x%08X", (unsigned)random_id);
    } else {
        ca_stdout_printf_n("Failed to generate random number");
    }
} else {
    ca_stdout_printf_n("RNG hardware initialization failed");
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_rng_get() in a caOS application, and why is it important for secure random number generation?
A: You should call ca_rng_get() when you need a cryptographically secure random number in your RTOS application. It is important because it uses a hardware RNG that is protected by a mutex, ensuring thread-safe access to the hardware. This function is particularly useful in applications like network protocol implementations or security-related modules where true randomness is required. It must be called only after initializing the RNG hardware with ca_rng_hw_init().

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_rng_get() with caOS types?
A: u32 random_value; bool success = ca_rng_get(&random_value); if (success) { // Use random_value } else { // Handle error }

Parameters conf:0.95 **Q:** What is the significance of the 'val' parameter in ca_rng_get(), and what happens if it is NULL?
A: The 'val' parameter is a pointer to a 32-bit unsigned integer where the generated random number will be stored. If this pointer is NULL, the function will fail and return false. This is a critical precondition because passing a NULL pointer would lead to a memory access violation, and the function explicitly checks for this condition before attempting to write the random value.

Return conf:0.90 **Q:** What do the return values of ca_rng_get() indicate, and what causes the function to return false?
A: ca_rng_get() returns true if the random number was successfully generated and stored in the provided pointer. It returns false if either the 'val' parameter is NULL or if the hardware RNG has not been initialized correctly. The function is blocking due to mutex acquisition, so it will also return false if the hardware RNG is not properly initialized or if the mutex fails to be acquired.

`ca_rng_hw_get`

Header: `caos/drivers/inc/ca_glue_hw.h:213`
GUARD: `EN_CA_GLUE_HW`

```c
extern bool ca_rng_hw_get(u32 *val); /* legge un valore casuale a 32 bit ; false se errore o timeout */
```

Reads a 32-bit random number from the hardware RNG peripheral.

Parametri:

Nome	Tipo	Descrizione
`val`	`u32 *`	Pointer to a 32-bit variable where the random value will be stored. Must not be NULL.

Ritorna:

Returns true if a valid random number was successfully retrieved, false if a timeout occurred or a hardware error (seed/clock error) was detected.

Note:

- This function is a low-level hardware driver call and is NOT thread-safe on its own; it must be protected by a mutex if accessed by multiple tasks. It performs a busy-wait loop (polling) until the hardware is ready or the timeout expires, meaning it is a blocking operation. It should not be called from an ISR because the blocking loop could cause system instability or missed interrupts. A common mistake is calling this function without ensuring the RNG peripheral has been properly initialized via ca_rng_init first.

Uso:

- This function is typically used by the higher-level ca_rng_get wrapper, which handles the necessary mutex locking to ensure thread safety. Developers should use ca_rng_get or ca_rng_fill instead of calling this hardware-specific function directly. The standard lifecycle is: call ca_rng_init during system startup, then use ca_rng_get whenever a random number is required for security tokens, nonces, or jitter generation.

Esempio:

```
u32 random_val;

/* Ensure the RNG is initialized before use */
if (ca_rng_init()) {
    /* Use the thread-safe wrapper instead of calling ca_rng_hw_get directly */
    if (ca_rng_get(&random_val)) {
        /* Successfully retrieved random value */
        ca_printf("Random number: %u\n", random_val);
    } else {
        /* Handle error: RNG timeout or hardware failure */
        ca_printf("Failed to get random number\n");
    }
} else {
    /* Handle initialization failure */
    ca_printf("RNG hardware not available\n");
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_rng_hw_get() in my embedded application?
A: You should call ca_rng_hw_get() when you need a cryptographically secure random number for your application, such as for generating session keys, nonces, or other security-related values. It is typically used after initializing the RNG hardware with ca_rng_hw_init().

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_rng_hw_get() in C with caOS types?
A: u32 random_value; if (ca_rng_hw_get(&random_value)) { // Use random_value } else { // Handle error }

Parameters conf:0.95 **Q:** What does the 'val' parameter in ca_rng_hw_get() represent and what happens if it's NULL?
A: The 'val' parameter is a pointer to a 32-bit unsigned integer where the random value will be stored. If this pointer is NULL, the function immediately returns false without attempting to read from hardware.

Return conf:0.95 **Q:** What do the return values of ca_rng_hw_get() indicate?
A: A return value of true indicates that the random number was successfully read from hardware. A return value of false indicates either a timeout, an error in the hardware, or that the input parameter 'val' was NULL.

Error/Edge case conf:0.90 **Q:** What are common mistakes or edge cases when using ca_rng_hw_get()?
A: Common mistakes include passing a NULL pointer to 'val', not initializing the RNG hardware with ca_rng_hw_init(), or calling the function from an ISR where it might block. Concurrent access without proper synchronization can also lead to unpredictable behavior.

HW differences conf:0.85 **Q:** How does the implementation of ca_rng_hw_get() differ between STM32F1xx, F4xx, and F7xx families?
A: The F1xx and F4xx families do not implement the RNG peripheral, so their ca_rng_hw_get() functions always return false. The F7xx, H5xx, and H7xx families implement the RNG peripheral and use registers like RNG->SR (status register) and RNG->DR (data register) to read random values, with timeout handling and error checking.

`ca_rng_hw_init`

Header: `caos/drivers/inc/ca_glue_hw.h:212`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_rng_hw_init(void); /* abilita il clock e inizializza il
    periferico RNG hardware */
```

Initializes the hardware Random Number Generator (RNG) peripheral and verifies it is ready to produce data.

Ritorna:
Returns true if the RNG peripheral was successfully enabled and the first random number is ready (DRDY bit set). Returns false if the peripheral failed to initialize, the clock failed to stabilize, or the hardware timed out waiting for the first random value.

Note:

- This function is not thread-safe and should be called during the system initialization phase before any tasks attempt to use the RNG. It is a blocking function that uses software loops to wait for hardware status bits (like clock readiness or data availability), so it must not be called from an ISR. A common mistake is calling this function multiple times or from different threads without proper synchronization; always ensure it is called once during startup. If the MCU does not have an RNG peripheral, this function will return false immediately.

Uso:

- This function is typically called once during the system boot sequence or within a driver initialization task. It is the mandatory prerequisite for any subsequent calls to ca_rng_get or ca_rng_fill. In a typical application, you would call this function, check the boolean result, and only if it returns true, proceed to register CLI commands or start threads that require random numbers.

Esempio:

```
/* Initialize the RNG peripheral during system startup */
if (ca_rng_hw_init()) {
    /* RNG is ready, safe to use ca_rng_get */
    u32 random_val = 0;
    if (ca_rng_get(&random_val)) {
        /* Successfully retrieved a random number */
    }
} else {
    /* Handle initialization failure: log error or enter safe state */
    /* ca_printf("RNG init failed!\n"); */
```

```
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_rng_hw_init() in my caOS application?
A: You should call ca_rng_hw_init() once during system initialization, before any calls to ca_rng_get(). This function initializes the hardware RNG peripheral and must be invoked before generating random numbers.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_rng_hw_init() in a caOS application?
A: bool success = ca_rng_hw_init(); if (success) { u32 random_value = ca_rng_get(); // use random_value } else { // handle initialization failure }

Return conf:0.95 **Q:** What does it mean when ca_rng_hw_init() returns false?
A: A return value of false indicates that the RNG hardware initialization failed. This can happen due to timeout (peripheral not responding), incorrect clock configuration, or if the RNG module is disabled via EN_RNG preprocessor flag.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_rng_hw_init()?
A: Common mistakes include calling ca_rng_hw_init() multiple times without resetting the peripheral, calling it from an ISR (which is not allowed), or failing to check its return value before using ca_rng_get(). Also, not ensuring the RNG clock is enabled in the system configuration can cause initialization to fail.

Behavior conf:0.90 **Q:** Is ca_rng_hw_init() thread-safe and ISR-safe?
A: ca_rng_hw_init() is not ISR-safe and must not be called from interrupt service routines. It is also not thread-safe unless protected by a mutex. The function is blocking and must be called once at system startup before any other RNG functions.

HW differences conf:0.90 **Q:** How does ca_rng_hw_init() differ between STM32F7xx and STM32H5xx MCUs?
A: On STM32F7xx, ca_rng_hw_init() simply enables the RNG clock and sets the RNGEN bit, then waits for data ready status. On STM32H5xx, it additionally performs a conditioning reset sequence by setting CONDRST, waiting for completion, and then releasing the reset before waiting for the first data ready signal.

`ca_rng_init`

Header: `caos/drivers/inc/rng.h:22`
Source: `caos/drivers/src/rng.c:30:38`
GUARD: `EN_RNG`

```
bool ca_rng_init(void)
```

Initializes the hardware Random Number Generator (RNG) peripheral and its associated software synchronization primitives.

Ritorna:
Returns true if the hardware RNG and internal mutex were successfully initialized; returns false if the hardware initialization failed.

Note:

- This function is not thread-safe during its own execution and must be called exactly once during the system startup phase before any tasks attempt to generate random numbers. It initializes a mutex to ensure that concurrent access to the RNG hardware from multiple tasks does not cause data corruption or race conditions. It is not intended to be called from an ISR, as it performs blocking operations and resource allocation. A common mistake is

calling this function multiple times or attempting to use the RNG before this function has returned true, which will lead to undefined behavior or system crashes.

Uso:

- This function is typically called during the early system initialization sequence, alongside other hardware drivers like AES, HASH, or CRC modules. It must be invoked before any cryptographic operations or security-related tasks (such as key generation or session ID creation) are performed. In a standard caOS application, it is paired with other init functions in the main entry point to ensure all peripherals are ready before the scheduler starts managing tasks.

Esempio:

```c
/* Initialize the RNG module during system startup */
if (!ca_rng_init()) {
    /* If initialization fails, the system cannot guarantee secure random numbers */
    /* Log the error or enter a safe state */
    ca_printf("Error: RNG initialization failed!\n");
} else {
    /* RNG is ready, safe to proceed with cryptographic operations */
    ca_printf("RNG initialized successfully.\n");

    /* Example: Use the RNG to fill a buffer with random data */
    uint8_t random_buffer[16];
    ca_rng_fill(random_buffer, 16);
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_rng_init() in my caOS application, and why is it important?
A: You should call ca_rng_init() at the beginning of your application, before using any random number generation functions. It initializes the hardware random number generator on STM32 microcontrollers, sets up a mutex for thread safety, and optionally registers CLI commands if enabled. This function must be called before any other RNG-related functions and cannot be called from an ISR. It ensures that the hardware RNG is properly configured and ready for use.

Code example conf:0.90 **Q:** Can you provide a minimal C example using caOS types that demonstrates how to initialize and use the RNG?
A: bool success = ca_rng_init(); if (success) { u32 random_value = ca_rng_get(); // Use random_value } else { // Handle initialization failure }

Return conf:0.90 **Q:** What does ca_rng_init() return, and what causes it to return false?
A: ca_rng_init() returns true if the hardware initialization of the random number generator was successful, and false otherwise. Failures can occur if the underlying hardware initialization function ca_rng_hw_init() fails, which might happen due to hardware issues, incorrect peripheral clock settings, or if the RNG peripheral is not available on the specific STM32 device being used.

`ca_rng_is_hardware`

Header: `caos/drivers/inc/rng.h:35`
Source: `caos/drivers/src/rng.c:144:151`
GUARD: `EN_RNG`

```c
bool ca_rng_is_hardware(void)
```

Checks if the system is currently using a hardware-based True Random Number Generator (TRNG).

Ritorna:
Returns true if a hardware TRNG is available and active, false if the system is falling back to a software-based pseudo-random generator.

Note:

- This function is non-blocking and thread-safe, making it safe to call from any context, including ISRs. It does not require any specific initialization beyond the standard system startup. The primary purpose is to allow security-sensitive code, such as long-term cryptographic key generation, to verify the quality of entropy before proceeding. A common mistake is to ignore the return value when performing high-security operations; if this returns false, the generated 'random' numbers may be predictable, which is a critical vulnerability in production systems.

Uso:

- This function is typically used as a gatekeeper in security modules. Before initializing a secure session or generating a private key, the application calls this function to ensure the entropy source meets the required security standards. If it returns false, the application should either abort the operation, notify the user, or switch to a less sensitive mode of operation. It is often paired with ca_rng_fill to populate buffers with random data once the hardware status is confirmed.

Esempio:

```c
/* Check if we have hardware entropy before generating a secure key */
if (ca_rng_is_hardware()) {
    uint8_t key[32];
    /* Fill buffer with high-quality hardware random bytes */
    ca_rng_fill(key, sizeof(key));
    /* Proceed with secure operations using the key */
} else {
    /* Log error or abort: software RNG is not suitable for long-term keys */
    ca_printf("Security Warning: Hardware RNG not available!\n");
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_rng_is_hardware in my application?
A: You should call this function as a gatekeeper before performing security-sensitive operations, such as generating private keys or initializing secure sessions. It ensures the entropy source meets the required security standards for high-quality random numbers.

Code example conf:1.00 **Q:** How do I implement a check for hardware entropy before generating a key?
A: You can use the following pattern: if (ca_rng_is_hardware()) { /* Proceed with secure key generation */ } else { /* Abort or switch to less sensitive mode */ }.

Return conf:1.00 **Q:** What do the return values of ca_rng_is_hardware signify?
A: A return value of true indicates that a hardware TRNG is available and active. A return value of false indicates the system is falling back to a software-based pseudo-random generator, which may produce predictable numbers and should not be used for high-security operations.

`ca_route_add`

Header: `caos/net/inc/route_service.h:62`
Source: `caos/net/src/route_service.c:86:111`

GUARD: `EN_ROUTE_SERVICE`

```
bool ca_route_add(ca_ipv4_t dst, u8 prefix_len, ca_ipv4_t gw)
```

Adds or updates a static routing entry in the network routing table.

Parametri:

Nome	Tipo	Descrizione
`dst`	`ca_ipv4_t`	The destination network or host IP address.
`prefix_len`	`u8`	The subnet mask length in bits (e.g., 24 for 255.255.255.0). Must be between 0 and 32.
`gw`	`ca_ipv4_t`	The gateway IP address to reach the specified destination.

Ritorna:
Returns true if the route was successfully added or updated. Returns false if the routing table is full (NET_ROUTE_MAX reached) or if the input parameters are invalid.

Note:

- This function is not thread-safe; ensure that routing table modifications are protected by a mutex if multiple tasks access the network stack simultaneously. It is not intended to be called from an ISR, as it performs iterative searching through the routing table which could lead to non-deterministic timing. Before calling this, the network stack must be initialized via ca_route_init. A common mistake is providing an incorrect prefix length, which results in an invalid network mask calculation and potentially unreachable routes.

Uso:

- This function is typically used during the network configuration phase of the application, often after obtaining an IP address via DHCP or setting a static IP. It is used to define how traffic for specific subnets should be routed through a gateway. It is often paired with ca_route_set_gw to define the default gateway and ca_route_init to prepare the internal table structures. If the routing table becomes cluttered, ca_route_flush or ca_route_del can be used to clear entries.

Esempio:

```
/* Initialize the routing table before adding entries */
ca_route_init();

/* Define a destination network 192.168.1.0/24 and its gateway */
ipv4_t dest = { .addr = 0xC0A80100 };
ipv4_t gateway = { .addr = 0xC0A80101 };

/* Attempt to add the route to the table */
if (ca_route_add(dest, 24, gateway)) {
    /* Route added successfully */
} else {
    /* Handle error: table full or invalid parameters */
    ca_printf("Failed to add route!\n");
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_route_add in a caOS network application?
A: You should call ca_route_add when you need to configure the routing table to direct packets to a specific destination IP address through a given gateway. This is typically done during network initialization or when dynamically adding new network routes, such as when a new network interface is connected or a static route is configured.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_route_add with caOS types?
A: ca_ipv4_t dest = 0xC0A80100; // 192.168.1.0 u8 prefix = 24; ipv4_t gateway = 0xC0A80101; // 192.168.1.1 bool result = ca_route_add(dest, prefix, gateway); if (result) { // Route added successfully } else { // Failed to add route }

Parameters conf:0.95 **Q:** What is the meaning of the prefix_len parameter in ca_route_add, and what are valid values?
A: The prefix_len parameter specifies the CIDR prefix length for the route's destination network. Valid values are integers from 0 to 32. A value of 0 means all addresses, while 32 means a specific host address. Values greater than 32 are invalid and may cause undefined behavior.

Return conf:0.90 **Q:** What does ca_route_add return, and what causes it to return false?
A: ca_route_add returns true if the route was successfully added or updated. It returns false if the routing table is full or if an internal error occurred during the operation. This could happen due to insufficient memory or an invalid configuration.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_route_add?
A: Common mistakes include passing an invalid prefix_len (e.g., >32), which can cause incorrect subnet masks. Another issue is not ensuring that the gateway IP is valid or reachable. The function does not validate gateway reachability, so incorrect gateways may lead to routing failures.

Behavior conf:0.90 **Q:** Is ca_route_add blocking, ISR-safe, or thread-safe?
A: ca_route_add is not blocking, but may take time to scan the routing table. It is not thread-safe, so a synchronization mechanism like a mutex should be used in multi-threaded environments. It is also not recommended to call it from an ISR due to potential concurrency or scheduling issues.

`ca_route_del`

Header: `caos/net/inc/route_service.h:67`
Source: `caos/net/src/route_service.c:113:118`
GUARD: `EN_ROUTE_SERVICE`

```
1 bool ca_route_del(u8 idx)
```

Removes an existing network route from the routing table by its index.

Parametri:

Nome	Tipo	Descrizione
`idx`	`u8`	The index of the route to remove. Must be less than NET_ROUTE_MAX and point to an active route entry.

Ritorna:
Returns true if the route was successfully found and deactivated; returns false if the index is out of bounds or the route was already inactive.

Note:

- This function is a direct memory modification of the internal routing table and is not thread-safe by itself; if multiple tasks modify routes simultaneously, you must protect this call with a mutex. It is non-blocking and executes in constant time, making it safe to call from an ISR if the underlying data structure is not being accessed elsewhere. The most common mistake is passing an index that is out of bounds or assuming that a 'false' return means the system failed, when it simply means the route was already gone or never existed.

Uso:

- This function is typically used during network configuration teardown or when dynamic routing protocols need to invalidate stale paths. It is usually paired with ca_route_init for setup, ca_route_add to populate the table, and ca_route_flush when the entire routing table needs to be cleared at once. In a typical lifecycle, you might use ca_route_lookup to find the index of a specific destination, then call ca_route_del to remove it.

Esempio:

```
/* Example: Removing a specific route from the network table */
u8 route_index = 2;

/* Ensure the route exists before attempting deletion */
if (ca_route_del(route_index)) {
    /* Route successfully removed */
    ca_printf("Route at index %d deleted successfully.\n", route_index);
} else {
    /* Route was already inactive or index was invalid */
    ca_printf("Failed to delete route at index %d.\n", route_index);
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_route_del() in a network application?
A: You should call ca_route_del() when you need to remove a specific network route from the routing table, typically when a network connection is terminated or when a route is no longer needed. This function is used in network service management to clean up routing entries.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_route_del() in C?
A: u8 route_idx = 2; bool result = ca_route_del(route_idx); if (result) { // Route was successfully deactivated } else { // Either invalid index or route was already inactive }

Parameters conf:0.90 **Q:** What is the valid range for the idx parameter in ca_route_del()?
A: The idx parameter must be an unsigned 8-bit integer (u8) in the range [0, NET_ROUTE_MAX-1]. Values less than 0 are not possible as u8 is unsigned, and values equal to or greater than NET_ROUTE_MAX will cause the function to return false.

Return conf:0.95 **Q:** What does ca_route_del() return when it fails, and why?
A: The function returns false when either the index is invalid (>= NET_ROUTE_MAX) or when the route at that index is already inactive (active = false). It returns true only when the route was successfully deactivated.

`ca_route_flush`

Header: `caos/net/inc/route_service.h:72`
Source: `caos/net/src/route_service.c:120:124`
GUARD: `EN_ROUTE_SERVICE`

```
void ca_route_flush(void)
```

Clears all active network routing entries in the system routing table.

Ritorna:
This function returns nothing (void).

Note:

- This function is non-blocking and performs a simple memory write operation across the routing table array. It is not inherently thread-safe; if other tasks are performing a 'ca_route_lookup' or 'ca_route_add' simultaneously, you may encounter race conditions where a route is added or read while the table is being wiped. It is generally safe to call from an ISR, but be aware that it iterates through the entire 'NET_ROUTE_MAX' range, which could introduce latency if the table size is large. The most common mistake is calling this during active network traffic, which will immediately drop all existing connections that rely on those routes.

Uso:

- This function is typically used during network stack initialization or when a complete network reset is required (e.g., switching between Wi-Fi and Ethernet interfaces). It is often paired with 'ca_route_init' to ensure the routing table starts in a clean, known state. In a typical lifecycle, you would call this when the network link status changes to 'down' to prevent the system from attempting to route packets through stale or invalid gateways.

Esempio:

```c
/* Reset the routing table when the network interface is disconnected */
void handle_network_link_down(void) {
    /* Clear all existing routes to prevent stale routing decisions */
    ca_route_flush();

    /* Log the event for debugging purposes */
    ca_printf("Network link down: routing table flushed.\n");

    /* Optionally re-initialize the routing table if needed */
    /* ca_route_init(); */
}
```

Q&A — fine-tuning review *2/2 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_route_flush() in a caOS network application?
A: You should call ca_route_flush() when you need to completely reset the routing table in your network system, typically in scenarios where you want to clear all configured IP addresses and active routes. This function is useful during system initialization, network reconfiguration, or when transitioning to a different network topology. It's important to ensure that the system is in a stable state before calling this function, as it's not thread-safe and must not be invoked from an ISR.

Code example conf:0.85 **Q:** Can you provide a minimal C example using caOS types that demonstrates how to use ca_route_flush()?
A: void example_route_flush(void) { // Example of calling ca_route_flush in a network service context ca_route_flush(); // No return value expected, but error handling could be added // based on the return behavior described in notes }

`ca_route_get`

Header: `caos/net/inc/route_service.h:79`
Source: `caos/net/src/route_service.c:126:131`
GUARD: `EN_ROUTE_SERVICE`

```c
bool ca_route_get(u8 idx, ca_route_entry_t *out)
```

Retrieves a specific network route entry from the internal routing table by its index.

Parametri:

Nome	Tipo	Descrizione
`idx`	`u8`	The index of the route to retrieve. Must be less than NET_ROUTE_MAX.
`out`	`ca_route_entry_t *`	Pointer to a structure where the route data will be copied. Must not be NULL.

Ritorna:
Returns true if the route exists and was successfully copied to 'out'. Returns false if the index is out of bounds, the pointer is NULL, or the route at that index is not marked as active.

Note:

- This function is non-blocking and thread-safe for reading, as it performs a simple memory copy of the entry. It is safe to call from an ISR, provided the routing table is not being modified simultaneously by another task. The most common beginner mistake is failing to check the boolean return value; if the function returns false, the contents of the 'out' structure remain unchanged or contain garbage, which can lead to undefined network behavior if used in subsequent operations.

Uso:

- This function is typically used in network management tasks to iterate through the routing table to find specific gateways or destinations. It is often paired with a loop from 0 to NET_ROUTE_MAX to inspect all active routes. Developers use it to extract gateway information (e.g., 'e.gw') before performing ARP resolution or filtering operations. It is the standard way to safely access the private 's_routes' array without exposing the internal implementation details.

Esempio:

```
void refresh_peer(void) {
    ca_route_entry_t e;
    u8 i;
    for (i = 0U; i < NET_ROUTE_MAX; i++) {
        /* Attempt to get the route at index i */
        if (!ca_route_get(i, &e)) {
            continue;
        }
        ca_mac_t mac;
        /* Check if the gateway is already in the ARP table */
        if (!ca_arp_find(e.gw, &mac)) {
            /* Resolve the gateway MAC address if not found */
            if (!ca_arp_resolve(e.gw, &mac, 500U)) {
                continue;
            }
        }
        /* Apply a filter for the resolved MAC address */
        ca_net_filter_mac_set(1U, &mac, NULL); /* 'router_filter' replaced with
    NULL for generic example */
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_route_get in my network management task?
A: You should use ca_route_get to iterate through the internal routing table to inspect active routes, find specific gateways, or filter destinations. It is the standard, safe way to access the private 's_routes' array without exposing internal implementation details.

Code example conf:1.00 **Q:** How do I iterate through the routing table using ca_route_get?

A: You can iterate through the table using a loop up to NET_ROUTE_MAX: ca_route_entry_t entry; for (u8 i = 0; i < NET_ROUTE_MAX; i++) { if (ca_route_get(i, &entry)) { /* Process active route entry */ } }

Parameters conf:1.00 **Q:** What are the constraints for the parameters idx and out in ca_route_get?

A: The 'idx' parameter must be a u8 value strictly less than NET_ROUTE_MAX. The 'out' parameter must be a valid pointer to a ca_route_entry_t structure and must not be NULL.

Return conf:1.00 **Q:** What does a false return value from ca_route_get indicate?

A: A false return indicates that the operation failed because the index was out of bounds, the provided pointer was NULL, or the route at that index is not marked as active. If false is returned, the 'out' structure remains unchanged or contains garbage, so you must not use it.

`ca_route_init`

Header: `caos/net/inc/route_service.h:46`
Source: `caos/net/src/route_service.c:61:72`
GUARD: `EN_ROUTE_SERVICE`

```
void ca_route_init(ca_ipv4_t local_ip, ca_ipv4_t local_mask)
```

Initializes the network routing table and sets the local device IP and subnet mask.

Parametri:

Nome	Tipo	Descrizione
`local_ip`	`ca_ipv4_t`	The IPv4 address to assign to the device. Must be a valid network address.
`local_mask`	`ca_ipv4_t`	The subnet mask defining the local network segment.

Ritorna:
void

Note:

- This function is non-blocking and performs a simple memory reset of the internal routing table. It is not thread-safe, so it must be called during the system initialization phase before any network tasks are spawned or packets are processed. A common mistake is calling this function after the network stack has already started, which can lead to race conditions or inconsistent routing states. It is not intended to be called from an ISR.

Uso:

- This function is typically called once during the system boot sequence, immediately after the physical network interface (PHY) has been initialized. It acts as a prerequisite for all subsequent network operations, such as adding specific routes with ca_route_add or performing lookups with ca_route_lookup. It is often paired with ca_net_set_phy_nrst to ensure the hardware is ready before the software stack configures the IP parameters.

Esempio:

```
/* Define local network configuration */
ipv4_t my_ip = {192, 168, 1, 10};
ipv4_t my_mask = {255, 255, 255, 0};

/* Initialize the routing service before starting network tasks */
ca_route_init(my_ip, my_mask);

```

```c
/* Optionally register a CLI node to manage routes at runtime */
#if ENABLE_CLI
    ca_cli_register(&s_route_cli_node);
#endif

/* Now it is safe to add specific routes */
/* ca_route_add(target_ip, gateway_ip, mask); */
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should ca_route_init be called in a caOS application, and why is it important for network functionality?
A: ca_route_init must be called once during system initialization to set up the local IP address and subnet mask for the device. It prepares the routing table for network packet forwarding. Without calling this function, the system cannot route packets to other networks, as the default gateway is set to zero and all routes are initialized as inactive.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using caOS types to initialize the routing table with a local IP and subnet mask?
A: ca_ipv4_t local_ip = 0xC0A80101; /* 192.168.1.1 */ ipv4_t local_mask = 0xFFFFFF00; /* 255.255.255.0 */ ca_route_init(local_ip, local_mask);

Parameters conf:0.85 **Q:** What are the valid values for the local_ip and local_mask parameters in ca_route_init, and what happens if invalid values are passed?
A: The local_ip parameter must be a valid IPv4 address in network byte order (e.g., 0xC0A80101 for 192.168.1.1). Zero is not a valid value. Similarly, local_mask must be a valid IPv4 subnet mask (e.g., 0xFFFFFF00 for 255.255.255.0), and zero is also invalid. The function does not return an error for invalid inputs, but incorrect values may lead to incorrect routing behavior.

Return conf:0.90 **Q:** Does ca_route_init return any value, and what would cause it to fail?
A: ca_route_init returns void and does not have any failure conditions. It is designed to never fail, and therefore does not provide any error codes or return values to indicate success or failure.

Error/Edge case conf:0.80 **Q:** What are common mistakes or edge cases when using ca_route_init, and how might they affect the system?
A: A common mistake is calling ca_route_init from an ISR or from a multithreaded context, as it is not thread-safe and is blocking. It should only be called once during system initialization. Another edge case is not calling it at all, which leaves the routing table uninitialized, preventing any inter-network packet routing. Calling it multiple times may lead to undefined behavior since it is not designed to be reentrant.

`ca_route_lookup`

Header: `caos/net/inc/route_service.h:78`
Source: `caos/net/src/route_service.c:126:154`
GUARD: `EN_ROUTE_SERVICE`

```c
ca_ipv4_t ca_route_lookup(ca_ipv4_t dst)
```

Determines the next hop IP address for a given destination using the routing table.

Parametri:

Nome	Tipo	Descrizione
`dst`	`ca_ipv4_t`	The destination IPv4 address to route. Must be a valid ca_ipv4_t structure.

Ritorna:
Returns an ca_ipv4_t structure representing the next hop IP address. If no route is found, it returns the original destination address (implying it is treated as on-link).

Note:

- This function is non-blocking and thread-safe, provided the routing table is not being modified concurrently. It is safe to call from an ISR, but keep in mind that it performs a linear search through the routing table, which may take longer as the number of routes increases. You must call ca_route_init before using this function, otherwise the routing table will contain garbage data. A common mistake is assuming this function performs ARP resolution; it only calculates the next hop IP, not the MAC address.

Uso:

- This function is used by the network stack whenever an outgoing packet needs to be routed. It is typically called after the network interface is initialized via ca_route_init and routes have been configured using ca_route_add. It is the core logic used to decide whether to send a packet directly to the destination or to a gateway. It is often paired with ca_arp_resolve to translate the resulting next-hop IP into a physical MAC address.

Esempio:

```
ca_ipv4_t target_ip = { .addr = 0x0A000005 }; // 10.0.0.5
ipv4_t next_hop;

// Initialize routing service before use
ca_route_init();

// Add a specific route to a gateway
ipv4_t gw = { .addr = 0x0A000001 };
ipv4_t mask = { .addr = 0xFFFFFF00 };
ca_route_add(target_ip, mask, gw);

// Determine where to send the packet
next_hop = ca_route_lookup(target_ip);

// Proceed to resolve MAC address for the next_hop IP
// ca_arp_resolve(next_hop);
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_route_lookup in my network stack implementation?
A: You should call it whenever an outgoing packet needs to be routed to determine the next hop IP. It is typically used after calling ca_route_init and configuring routes with ca_route_add, and is often followed by ca_arp_resolve to obtain the MAC address.

Code example conf:0.90 **Q:** Provide a minimal C example showing how to use ca_route_lookup.
A: ca_ipv4_t dest = {192, 168, 1, 50}; ca_route_init(); ca_ipv4_t next_hop = ca_route_lookup(dest)

Parameters conf:1.00 **Q:** What is the expected format for the dst parameter in ca_route_lookup?
A: The dst parameter must be a valid ca_ipv4_t structure representing the destination IPv4 address you wish to route.

Return conf:1.00 **Q:** What does it mean if ca_route_lookup returns the same IP address I passed as the destination?
A: If the function returns the original destination address, it indicates that no specific route was found in the routing table, implying the destination is treated as on-link.

Error/Edge case conf:1.00 **Q:** What is a common mistake developers make when using ca_route_lookup?
A: A common mistake is assuming that this function performs ARP resolution. It only calculates the next hop IP address and does not resolve the physical MAC address.

Behavior conf:1.00 **Q:** Is ca_route_lookup safe to call from an Interrupt Service Routine (ISR)?

A: Yes, it is safe to call from an ISR. However, be aware that it is a non-blocking function that performs a linear search, so execution time will increase as the number of routes in the table grows.

---

ca_route_set_gw

Header: caos/net/inc/route_service.h:56
Source: caos/net/src/route_service.c:80:84
GUARD: EN_ROUTE_SERVICE

```
bool ca_route_set_gw(ca_ipv4_t gw)
```

Updates the default gateway IP address used by the network stack for routing packets.

Parametri:

Nome	Tipo	Descrizione
gw	ca_ipv4_t	The IPv4 address of the gateway to be set. Ensure this is a valid network-byte-order or host-byte-order representation as expected by your specific network configuration.

Ritorna:
Always returns true in the current implementation.

Note:

- This function is non-blocking and executes immediately by updating a global variable. It is not thread-safe by itself; if multiple tasks attempt to update routing information simultaneously, you should wrap this call in a mutex or ensure it is only called from a single configuration task. It is safe to call from an ISR, but typically you should perform network configuration from a dedicated task context. A common mistake is calling this before the network stack has been initialized via ca_route_init, which may lead to undefined behavior or ignored settings.

Uso:

- This function is typically used during the network initialization phase or when a DHCP client receives a new lease. It is usually paired with ca_route_update_local to define the full local network identity (IP, mask, and gateway). In a typical sequence, you would call ca_route_init first, then update your local parameters, and finally set the gateway once the network parameters are successfully negotiated or statically assigned.

Esempio:

```
/* Example: Setting up network routing after obtaining an IP via DHCP */
#include "caos.h"

void configure_network_gateway(ca_ipv4_t new_gw)
{
    /* Ensure the routing service is initialized before setting parameters */
    ca_route_init();

    /* Update the gateway address */
    if (ca_route_set_gw(new_gw))
    {
        /* Gateway successfully updated */
    }
    else
    {
```

```
        /* Handle error: failed to set gateway */
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_route_set_gw() in a network application?
A: You should call ca_route_set_gw() when you need to configure the default gateway for IP routing, typically after obtaining network configuration via DHCP or manual setup. This function is used to direct traffic to external networks through a specified gateway address.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_route_set_gw() with caOS types?
A: ca_ipv4_t gateway = 0x0A000001; // 10.0.0.1 in network byte order bool result = ca_route_set_gw(gateway); // Handle result as needed

Parameters conf:0.90 **Q:** What is the meaning of the 'gw' parameter in ca_route_set_gw() and what are valid values?
A: The 'gw' parameter is the IPv4 address of the default gateway to be set. Valid values are any valid IPv4 address in network byte order (e.g., 0x0A000001 for 10.0.0.1). If the value is 0, the function ignores the call and returns immediately without changing the gateway.

Return conf:0.85 **Q:** What does ca_route_set_gw() return and what causes it to fail?
A: The function returns true if the operation completes successfully. Failures are not explicitly described in the documentation, but the function is non-blocking and not thread-safe, so it may fail if called from an ISR or if the gateway address is invalid or unreachable. The function does not perform validation of the gateway address.

`ca_route_update_local`

Header: `caos/net/inc/route_service.h:51`
Source: `caos/net/src/route_service.c:74:78`
GUARD: `EN_ROUTE_SERVICE`

```
void ca_route_update_local(ca_ipv4_t local_ip, ca_ipv4_t local_mask)
```

Updates the local IPv4 address and subnet mask in the network routing table.

Parametri:

Nome	Tipo	Descrizione
`local_ip`	`ca_ipv4_t`	The new IPv4 address to assign to the local interface. Must be a valid 32-bit representation of an IP address.
`local_mask`	`ca_ipv4_t`	The subnet mask corresponding to the local network. Must be a valid 32-bit subnet mask.

Ritorna:
void

Note:

- This function is non-blocking and executes immediately by updating internal static variables. It is not thread-safe by itself, so ensure that calls to this function are synchronized if multiple tasks attempt to reconfigure the network interface simultaneously. It is safe to call from an ISR, but typically it is called from a network management task or a DHCP client service. A common mistake is calling this function before the network stack has been initialized via ca_route_init, which may lead to inconsistent routing behavior.

Uso:

- This function is used during the network configuration phase, typically after a DHCP lease is acquired or when a static IP configuration is applied. It is usually paired with ca_route_init to set up the routing subsystem and ca_route_set_gw to define the default gateway. In a typical lifecycle, you initialize the route service, update the local IP/mask, and then set the gateway to ensure the device can communicate outside its local subnet.

Esempio:

```c
/* Example: Updating network settings after receiving a DHCP offer */
#include "caos.h"

void update_network_config(ca_ipv4_t new_ip, ca_ipv4_t new_mask, ca_ipv4_t
    new_gw)
{
    /* Ensure the routing service is initialized before updating */
    ca_route_init();

    /* Update the local interface parameters */
    ca_route_update_local(new_ip, new_mask);

    /* Set the gateway if it is valid */
    if (new_gw != 0U) {
        ca_route_set_gw(new_gw);
    }

    /* Log the update via CLI if enabled */
    ca_printf("Network updated: %d.%d.%d.%d\n",
              (new_ip >> 24) & 0xFF, (new_ip >> 16) & 0xFF,
              (new_ip >> 8) & 0xFF, new_ip & 0xFF);
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_route_update_local and why is it important for network configuration?
A: You should call ca_route_update_local after a successful DHCP negotiation to update the local IP address and subnet mask of the system. This function is important because it ensures that the network routing tables and other network services are aware of the new network configuration. It is typically used in network service modules like DHCP client or route management services.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_route_update_local with caOS types?
A: ca_ipv4_t ip = 0xC0A80101; /* 192.168.1.1 */ ipv4_t mask = 0xFFFFFF00; /* 255.255.255.0 */ ca_route_update_local(ip, mask);

Parameters conf:0.90 **Q:** What are the valid and invalid values for the local_ip and local_mask parameters in ca_route_update_local?
A: The local_ip parameter should be a valid IPv4 address in network byte order, such as 0xC0A80101 for 192.168.1.1. A value of 0 indicates an invalid IP address. The local_mask parameter should be a valid subnet mask in network byte order, such as 0xFFFFFF00 for 255.255.255.0. A value of 0 indicates an invalid subnet mask. Invalid values will not cause errors but may lead to incorrect network behavior.

`ca_rtc_calibrate`

Header: `caos/drivers/inc/rtc.h:150`
Source: `caos/drivers/src/rtc.c:428:431`

GUARD: `EN_RTC`

```
bool ca_rtc_calibrate(i32 ppm)
```

Adjusts the internal RTC clock frequency by applying a parts-per-million (ppm) correction factor.

Parametri:

Nome	Tipo	Descrizione
ppm	i32	The correction value in parts-per-million. Positive values speed up the clock, negative values slow it down. The valid range depends on the specific STM32 hardware implementation.

Ritorna:
Returns true if the calibration value was successfully applied to the RTC hardware, false otherwise (e.g., invalid range or hardware error).

Note:

- This function is typically non-blocking as it writes directly to the RTC calibration registers. It is generally thread-safe if the underlying hardware driver uses proper locking, but it should not be called from an ISR unless the driver documentation explicitly states it is ISR-safe. The most common mistake is passing an extreme ppm value that exceeds the hardware's physical adjustment capability, which will result in a failed return value. Ensure ca_rtc_init has been called successfully before attempting to calibrate the clock.

Uso:

- This function is used to compensate for crystal oscillator inaccuracies over time, ensuring the RTC maintains accurate timekeeping. It is typically used in a maintenance or configuration context, often triggered by a CLI command or a system calibration routine. It is usually paired with ca_rtc_init during system startup and can be used periodically if the system detects time drift compared to an external reference like an NTP server.

Esempio:

```
/* Example: Adjusting RTC calibration via CLI input */

void handle_rtc_calibration(char *p) {
    /* Skip the 'cal' command and space to reach the number */
    i32 ppm = ca_strtol(ca_skip_sp(p + 3), 10);

    /* Apply the calibration value to the RTC hardware */
    if (ca_rtc_calibrate(ppm)) {
        /* Use ca_printf_n for formatted output to the console */
        ca_printf_n("rtc cal: %d ppm applied successfully", (int)ppm);
    } else {
        /* Handle failure: log error or notify user */
        ca_printf_n("rtc cal: failed to apply %d ppm", (int)ppm);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_rtc_calibrate and why is it useful?
A: You should call ca_rtc_calibrate when you need to improve the accuracy of the internal RTC on an STM32 microcontroller. It is useful for applications where precise timekeeping is required, and the default RTC oscillator may have a frequency offset. By providing a correction value in ppm (parts per million), you can adjust the RTC's speed to compensate for this offset.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_rtc_calibrate in C with caOS types?
A: i32 correction = 50; bool result = ca_rtc_calibrate(correction); if (result) { /* Calibration successful */ } else { /* Calibration failed */ }

Parameters conf:0.90 **Q:** What does the ppm parameter in ca_rtc_calibrate represent, and what are the valid values?
A: The ppm parameter represents the correction value in parts per million. Positive values increase the RTC's speed, while negative values decrease it. Valid values range from -1000 to +1000 ppm. Using values outside this range may lead to unpredictable behavior or hardware errors.

Return conf:0.90 **Q:** What do the return values of ca_rtc_calibrate indicate, and what causes it to return false?
A: The function returns true if the calibration was applied successfully. It returns false if the calibration failed, which can happen if the ppm parameter is out of the valid range (-1000 to +1000) or if the RTC hardware is not properly initialized. The function does not verify the RTC's active state or calibration mode.

`ca_rtc_clear_alarm`

Header: `caos/drivers/inc/rtc.h:163`
Source: `caos/drivers/src/rtc.c:440:444`
GUARD: `EN_RTC`

```
bool ca_rtc_clear_alarm(u8 alarm_id)
```

Disables a specific RTC alarm to prevent it from triggering further interrupts.

Parametri:

Nome	Tipo	Descrizione
`alarm_id`	`u8`	The identifier of the alarm to clear. Must be either CA_RTC_ALARM_A or CA_RTC_ALARM_B. Passing any other value will result in a failure.

Ritorna:
Returns true if the alarm was successfully cleared, or false if the alarm_id is invalid or the hardware operation failed.

Note:

- This function is a non-blocking wrapper around the hardware-specific RTC alarm clear routine. It is generally thread-safe, but you must ensure the RTC peripheral has been initialized via ca_rtc_init before calling this. A common mistake is to assume that clearing an alarm also resets the RTC time or clears pending interrupt flags in the NVIC; this function only disables the alarm trigger mechanism itself. It is safe to call from most contexts, but always verify your specific hardware implementation if you intend to call it from high-priority ISRs.

Uso:

- Use this function when you want to stop a recurring alarm or cancel a one-shot alarm that has already fired. It is typically used in the alarm callback function or in the main application loop after an alarm event has been processed. It is often paired with ca_rtc_set_alarm to re-arm the alarm for a future time or to switch between different

alarm configurations. In power management scenarios, it is frequently called during wake-up routines to ensure that the RTC alarm source is properly disarmed before the system resumes normal operation.

Esempio:

```c
/* Example: Clearing an alarm after it has triggered */
#include "caos.h"

void process_rtc_event(void) {
    /* Check if Alarm B triggered and disable it to prevent re-triggering */
    /* This is often done inside a task or after a wake-up event */
    if (!ca_rtc_clear_alarm(CA_RTC_ALARM_B)) {
        /* Handle error: alarm_id was invalid or hardware failed */
        ca_printf_n("Error: Failed to clear RTC Alarm B");
    } else {
        ca_printf_n("Alarm B successfully cleared.");
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_rtc_clear_alarm() and why is it important?
A: You should call ca_rtc_clear_alarm() when you want to disable a specific RTC alarm (either A or B) that was previously set using ca_rtc_set_alarm(). It is important because it ensures that the alarm does not trigger unexpectedly, especially if you are reconfiguring or shutting down RTC functionality. The function is safe to use as long as the alarm was previously set.

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_rtc_clear_alarm() with caOS types?
A: u8 result = ca_rtc_clear_alarm(CA_RTC_ALARM_A); if (result) { /* Alarm A cleared successfully */ } else { /* Invalid alarm ID */ }

Parameters conf:0.95 **Q:** What does the alarm_id parameter in ca_rtc_clear_alarm() represent, and what are valid values?
A: The alarm_id parameter specifies which RTC alarm to clear. Valid values are CA_RTC_ALARM_A (0) and CA_RTC_ALARM_B (1). Any other value, such as 2 or -1, will cause the function to return false, indicating an invalid alarm ID.

Return conf:0.90 **Q:** What do the return values of ca_rtc_clear_alarm() mean, and what causes it to return false?
A: The function returns true if the alarm was successfully cleared. It returns false if the alarm_id parameter is invalid (i.e., less than CA_RTC_ALARM_A or greater than CA_RTC_ALARM_B). If the alarm was not previously set, the function does nothing but still returns true.

`ca_rtc_clock_init`

Header: `caos/drivers/inc/rtc.h:97`
Source: `caos/drivers/src/rtc.c:284:318`
GUARD: `EN_RTC`

```c
bool ca_rtc_clock_init(void)
```

Initializes the Real-Time Clock (RTC) hardware by selecting and configuring the best available clock source (LSE or LSI).

Ritorna:

Returns true if the RTC was successfully initialized and is ready to track time; returns false if both LSE and LSI sources failed to initialize.

Note:

- This function is non-blocking but performs hardware polling to verify clock stability, so it may take a few milliseconds to return. It is not thread-safe and must be called during the system boot sequence before any tasks attempt to read or set the time. It is not callable from an ISR because it relies on hardware initialization sequences that may involve delays. A common beginner mistake is calling this function multiple times or after the system has already started scheduling tasks, which can cause clock drift or hardware configuration conflicts.

Uso:

- This function is typically called once during the system startup phase, before the RTOS scheduler is fully operational. It is used to ensure the RTC is running before any time-dependent services (like logs, timestamps, or scheduled events) are initialized. It is often paired with ca_rtc_get_clk_src() to verify which source was selected and ca_rtc_lsi_hz() or ca_rtc_lse_start_ms() to log the health of the chosen clock source.

Esempio:

```
/* Initialize the RTC during system boot */
if (ca_rtc_clock_init()) {
    /* RTC is ready, safe to use time functions */
    ca_printf("RTC initialized successfully.\n");
} else {
    /* RTC failed, handle error (e.g., log to non-volatile memory) */
    ca_printf("RTC initialization failed!\n");
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to call ca_rtc_clock_init() in a caOS application?
A: It should be called once during the system boot sequence, specifically before the RTOS scheduler is fully operational and before any tasks attempt to read or set the time.

Code example conf:0.90 **Q:** How do I implement a basic check for RTC initialization in my main boot sequence?
A: bool status = ca_rtc_clock_init(); if (status) { /* RTC ready */ } else { /* Handle failure */ }

Return conf:1.00 **Q:** What do the boolean return values of ca_rtc_clock_init() indicate?
A: A return value of true indicates the RTC was successfully initialized and is ready to track time. A return value of false indicates that both the LSE and LSI clock sources failed to initialize.

Error/Edge case conf:1.00 **Q:** What are the common mistakes developers make when using ca_rtc_clock_init()?
A: A common mistake is calling the function multiple times or invoking it after the system has already started scheduling tasks, which can lead to clock drift or hardware configuration conflicts.

Behavior conf:1.00 **Q:** Is ca_rtc_clock_init() safe to call from an interrupt service routine or multiple threads?
A: No, it is not thread-safe and must not be called from an ISR. While it is non-blocking, it performs hardware polling that requires delays, making it unsuitable for ISR execution.

`ca_rtc_from_net`

Header: `caos/drivers/inc/rtc.h:136`
Source: `caos/drivers/src/rtc.c:383:413`
GUARD: `EN_RTC`

```
bool ca_rtc_from_net(u32 unix_time)
```

Converts a Unix timestamp to a date/time structure and updates the hardware RTC.

Parametri:

Nome	Tipo	Descrizione
`unix_time`	`u32`	The number of seconds elapsed since 1970-01-01 00:00:00 UTC. Must be greater than or equal to 946684800 (the year 2000 epoch).

Ritorna:
Returns true if the conversion was successful and the RTC was updated. Returns false if the input time is before the year 2000, exceeds the year 2099, or if the internal RTC hardware update fails.

Note:

- This function performs a mathematical conversion from Unix time to a calendar format (year, month, day, etc.) and then calls the internal RTC driver to set the hardware registers. It is not thread-safe if multiple tasks attempt to set the RTC simultaneously, so ensure synchronization if necessary. It is not intended to be called from an ISR because it performs complex calculations and potentially blocking I/O operations to the RTC peripheral. The most common mistake is passing a timestamp from a different epoch or a value that results in a date beyond the year 2099, which the function rejects.

Uso:

- This function is typically used during system startup or after a network synchronization event (e.g., via NTP) to ensure the device has the correct wall-clock time. It is usually paired with network-related functions that fetch the current time from a server. Once the time is retrieved, this function acts as the bridge to persist that time into the STM32 RTC hardware, ensuring the clock remains accurate even if the network connection is lost. It is often called after validating input strings or network packets to ensure the RTC is only updated with sane, within-range values.

Esempio:

```
/* Assuming we have received a valid Unix timestamp from an NTP server */
u32 network_time = 1715856000UL;

/* Attempt to update the system RTC with the network time */
if (ca_rtc_from_net(network_time)) {
    /* Successfully synchronized the RTC */
} else {
    /* Handle error: invalid time or hardware failure */
    ca_printf("Failed to sync RTC from network\n");
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_rtc_from_net function in my caOS application?
A: This function is typically used during system startup or after a network synchronization event, such as an NTP update, to persist the correct wall-clock time into the STM32 RTC hardware.

Code example conf:1.00 **Q:** How do I call ca_rtc_from_net in C using caOS types?
A: You would pass a u32 timestamp representing seconds since 1970. Example: u32 current_time = 1715856000; bool success = ca_rtc_from_net(current_time);

Parameters conf:1.00 **Q:** What are the constraints on the unix_time parameter?
A: The unix_time parameter must be a u32 representing seconds since 1970-01-01 00:00:00 UTC. It must be greater than or equal to 946684800, which corresponds to the year 2000 epoch.

Return conf:1.00 **Q:** What do the boolean return values of ca_rtc_from_net indicate?
A: A return value of true indicates the Unix time was successfully converted and the hardware RTC registers were updated. A return value of false indicates the input time was before the year 2000 or the internal RTC hardware update failed.

Error/Edge case conf:1.00 **Q:** What are common mistakes or error conditions when using ca_rtc_from_net?
A: Common mistakes include passing a timestamp from a different epoch or providing a value that results in a date beyond the year 2099, which the function will reject.

Behavior conf:1.00 **Q:** Is ca_rtc_from_net safe to call from an ISR or in a multi-threaded environment?
A: It is not ISR-safe because it performs complex calculations and potentially blocking I/O operations. It is also not thread-safe, so you must ensure synchronization if multiple tasks attempt to set the RTC simultaneously.

`ca_rtc_get`

Header: `caos/drivers/inc/rtc.h:119`
Source: `caos/drivers/src/rtc.c:355:359`
GUARD: `EN_RTC`

```
1 bool ca_rtc_get(ca_rtc_dt_t *dt)
```

Retrieves the current date and time from the hardware Real-Time Clock (RTC).

Parametri:

Nome	Tipo	Descrizione
`dt`	`ca_rtc_dt_t *`	Pointer to a structure where the current date and time will be stored. Must not be NULL.

Ritorna:
Returns true if the RTC data was successfully read, false if the pointer is NULL or the hardware communication failed.

Note:

- This function is a wrapper for hardware access and is generally non-blocking, but it depends on the underlying hardware peripheral state. It is typically thread-safe, but you must ensure ca_rtc_init has been called during system startup before attempting to read the time. A common beginner mistake is passing an uninitialized pointer or failing to check the boolean return value, which leads to using garbage data if the RTC peripheral is not ready or has lost power. Do not call this function from an ISR unless the underlying hardware driver explicitly supports it, as RTC access often involves I2C or slow bus transactions.

Uso:

- This function is used whenever the system needs to timestamp events, log data, or manage time-dependent logic. It is frequently paired with ca_rtc_init during the system boot sequence to ensure the clock is running. In application code, it is often used in conjunction

with file system drivers to set file timestamps, with power management modules to calculate alarm offsets for waking the system, or with CLI modules to display system uptime and current date.

Esempio:

```
ca_rtc_dt_t current_time;

// Initialize the RTC peripheral before use
ca_rtc_init();

// Attempt to read the current time
if (ca_rtc_get(&current_time)) {
    // Successfully retrieved time, format and print to console
    ca_printf("Current Date: 20%02u-%02u-%02u", current_time.year, current_time.month, current_time.day);
} else {
    // Handle error: RTC might be uninitialized or hardware failure
    ca_printf("Error: Could not read RTC data");
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_rtc_get() in a caOS application?

A: You should call ca_rtc_get() when you need to read the current date and time from the RTC peripheral. It is typically used in applications where timestamping events or managing time-based logic is required. Since the function is blocking, it is best used in contexts where blocking is acceptable, such as during initialization or in non-critical sections of code.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_rtc_get() with caOS types?

A: ca_rtc_dt_t dt; bool success = ca_rtc_get(&dt); if (success) { // Use dt.year, dt.month, dt.day, etc. }

Parameters conf:0.90 **Q:** What is the significance of the dt parameter in ca_rtc_get() and what happens if it is invalid?

A: The dt parameter is a pointer to a ca_rtc_dt_t structure where the current date and time will be stored. If this pointer is NULL, the function will likely fail, as it cannot store the retrieved data. The function requires a valid, non-NULL pointer to successfully write the RTC data.

Return conf:0.95 **Q:** What do the return values of ca_rtc_get() indicate and what causes a failure?

A: The function returns true if the date and time were successfully read from the RTC and stored in the provided structure. It returns false if the read operation failed, which may happen if the RTC is not initialized, the hardware is not available, or other underlying issues prevent access to the RTC peripheral.

`ca_rtc_get_clk_src`

Header: `caos/drivers/inc/rtc.h:85`
Source: `caos/drivers/src/rtc.c:344:347`
GUARD: `EN_RTC`

```
ca_rtc_clk_src_t ca_rtc_get_clk_src(void)
```

Returns the current clock source configured for the Real-Time Clock (RTC) peripheral.

Ritorna:

Returns a value of type ca_rtc_clk_src_t, which is an enumeration representing the clock source (e.g., CA_RTC_CLK_LSE, CA_RTC_CLK_HSE, or CA_RTC_CLK_LSI).

Note:

- This function is a simple getter that returns the internal state variable tracking the RTC clock source. It is non-blocking, thread-safe, and can be safely called from an ISR because it performs no hardware access or complex logic. The function assumes that ca_rtc_clock_init() has been successfully called during system startup; if called before initialization, it may return an undefined or default value. A common mistake is assuming the clock source can change dynamically at runtime; in most caOS configurations, the source is fixed during the initial boot sequence.

Uso:

- This function is typically used for diagnostic purposes or to determine the correct frequency for time calculations. It is often paired with ca_rtc_lsi_hz() when the LSI source is active, as the LSI frequency can vary based on temperature and calibration. Developers use it to format logs or to decide which logic to apply when calculating timestamps or setting alarms. It is usually called after the system has initialized the RTC subsystem to verify that the hardware is running on the expected clock source (e.g., verifying LSE for high-precision timekeeping).

Esempio:

```
/* Check the RTC clock source and print the status */
ca_rtc_clk_src_t src = ca_rtc_get_clk_src();

if (src == CA_RTC_CLK_LSE) {
    ca_printf("RTC is running on LSE (32768 Hz crystal)\n");
} else if (src == CA_RTC_CLK_LSI) {
    /* LSI frequency might be calibrated, fetch the current value */
    u32 freq = ca_rtc_lsi_hz();
    ca_printf("RTC is running on LSI at %u Hz\n", freq);
} else {
    ca_printf("RTC is running on HSE or unknown source\n");
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_rtc_get_clk_src() and what is its primary purpose?
A: You should call this function after the RTC subsystem has been initialized to verify the hardware is running on the expected clock source, such as confirming LSE for high-precision timekeeping. It is primarily used for diagnostic purposes, logging, or determining the correct frequency logic for timestamp calculations and alarm settings.

Code example conf:0.95 **Q:** How do I check the RTC clock source in my caOS application?
A: You can retrieve the source and verify it using the following code: ca_rtc_clk_src_t src = ca_rtc_get_clk_src(); if (src == CA_RTC_CLK_LSE) { /* High precision mode */ } else if (src == CA_RTC_CLK_LSI) { /* LSI mode, consider calibration */ }

Return conf:1.00 **Q:** What does the return value of ca_rtc_get_clk_src() represent and are there any failure conditions?
A: The function returns a ca_rtc_clk_src_t enumeration representing the configured clock source (e.g., CA_RTC_CLK_LSE, CA_RTC_CLK_HSE, or CA_RTC_CLK_LSI). There are no explicit failure codes; however, if called before ca_rtc_clock_init() has been executed, the function may return an undefined or default value.

---

ca_rtc_get_stamp_ms

Header: caos/drivers/inc/rtc.h:128
Source: caos/drivers/src/rtc.c:361:378
GUARD: EN_RTC

```
bool ca_rtc_get_stamp_ms(u64 *ms)
```

Retrieves the current system time as a 64-bit millisecond timestamp since the epoch (2000-01-01).

Parametri:

Nome	Tipo	Descrizione
ms	u64 *	Pointer to a 64-bit unsigned integer where the calculated timestamp will be stored. Must not be NULL.

Ritorna:
Returns true if the timestamp was successfully calculated and written to the pointer. Returns false if the RTC is not ready, the pointer is NULL, or the hardware RTC data is invalid (e.g., out-of-range date values).

Note:

- This function is non-blocking and performs arithmetic calculations to convert calendar date/time into a linear millisecond count. It is not inherently thread-safe if the underlying hardware RTC registers are modified by other tasks or interrupts during the calculation. While it can be called from an ISR, it is computationally expensive due to multiple multiplications and divisions; therefore, it should be used sparingly in high-frequency interrupt contexts. The function requires the RTC peripheral to be fully initialized and synchronized; if the RTC is in a fault state or not yet ready, it will return false. A common beginner mistake is ignoring the return value, which results in using an uninitialized or stale 'ms' variable when the RTC is actually offline.

Uso:

- This function is essential for logging events, calculating precise time deltas between system states, or timestamping data packets for network synchronization. It is typically used after the system has successfully initialized the RTC peripheral via ca_rtc_init. In power management workflows, it is frequently paired with a 't0' capture before entering a low-power state, allowing the system to calculate the exact duration of sleep upon wake-up by comparing the initial timestamp with a new one.

Esempio:

```
u64 start_time = 0;
u64 end_time = 0;

/* Ensure RTC is initialized before use */
if (ca_rtc_init()) {
    /* Capture start timestamp */
    if (ca_rtc_get_stamp_ms(&start_time)) {
        /* Perform some time-consuming operation */
        ca_os_delay_ms(100);

        /* Capture end timestamp */
        if (ca_rtc_get_stamp_ms(&end_time)) {
            u64 duration = end_time - start_time;
            ca_printf("Operation took %llu ms\n", duration);
        } else {
```

```
            ca_printf("Failed to get end timestamp\n");
        }
    } else {
        ca_printf("RTC not ready for timestamping\n");
    }
} else {
    ca_printf("RTC failed to initialize!\n");
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_rtc_get_stamp_ms in my caOS application?
A: You should use this function for logging events, calculating time deltas between system states, or timestamping data packets. It is also useful in power management to calculate sleep duration by capturing a timestamp before and after entering a low-power state.

Code example conf:1.00 **Q:** How do I correctly call ca_rtc_get_stamp_ms in C?
A: u64 timestamp = 0; if (ca_rtc_get_stamp_ms(&timestamp)) { /* Use timestamp safely */ }

Parameters conf:1.00 **Q:** What are the requirements for the ms parameter in ca_rtc_get_stamp_ms?
A: The ms parameter must be a pointer to a 64-bit unsigned integer (u64 *). It must not be NULL, as the function will return false if a NULL pointer is provided.

Return conf:1.00 **Q:** What does a return value of false indicate when calling ca_rtc_get_stamp_ms?
A: A return value of false indicates that the timestamp could not be calculated. This occurs if the RTC is not ready, the provided pointer is NULL, or the hardware RTC data is invalid (e.g., out-of-range date values).

Error/Edge case conf:1.00 **Q:** What is a common mistake developers make when using ca_rtc_get_stamp_ms?
A: A common beginner mistake is failing to check the boolean return value, which can lead to the application using uninitialized or stale data stored in the ms variable.

Behavior conf:1.00 **Q:** Is ca_rtc_get_stamp_ms safe to call from an interrupt service routine (ISR)?
A: It is not recommended to call this function from high-frequency ISRs because it performs floating-point-like arithmetic (multiplications and divisions). Additionally, the function is not thread-safe if the underlying hardware RTC registers are accessed concurrently by multiple tasks or interrupts.

`ca_rtc_get_timestamp`

Header: `caos/drivers/inc/rtc.h:170`
Source: `caos/drivers/src/rtc.c:446:470`
GUARD: `EN_RTC`

```
u32 ca_rtc_get_timestamp(void)
```

Calculates the current RTC time as a Unix-like timestamp in seconds since the 1900 epoch.

Ritorna:
A 32-bit unsigned integer representing the total seconds elapsed since January 1st, 1900. If the RTC hardware is uninitialized or malfunctioning, the returned value may be inaccurate or represent the default epoch start.

Note:

- This function performs significant mathematical operations to convert calendar date/time components into a linear second count. It is not inherently thread-safe if the underlying RTC hardware registers are modified by an interrupt during the calculation, though it relies on the atomic nature of the hardware read. It is safe to call from standard tasks,

but you should avoid calling it inside high-frequency ISRs because the floating-point-like arithmetic and array lookups consume unnecessary CPU cycles. A common beginner mistake is assuming this returns a standard 1970-based Unix timestamp; since this uses the 1900 epoch, you must manually subtract 2208988800 if you need to interface with standard POSIX-compliant systems.

Uso:

- This function is essential for applications requiring absolute time tracking, such as network protocol headers (NTP), file system timestamps, or event logging. It is typically used after the RTC hardware has been configured via ca_rtc_init and synchronized with an external time source. In a typical lifecycle, you initialize the RTC, set the time once, and then periodically call this function to generate timestamps for data packets or log entries before sending them over the network or saving them to storage.

Esempio:

```c
/* Ensure RTC is initialized before reading */
ca_rtc_init();

/* Get the current time as a linear timestamp */
u32 current_time = ca_rtc_get_timestamp();

/* 3786835200 is the approximate timestamp for 2020-01-01 */
if (current_time > 3786835200U) {
    /* Log the timestamp using system logging */
    ca_printf("Current system timestamp: %u\n", current_time);
} else {
    /* RTC might not be set, perform manual time sync */
    /* Call function to set RTC from external source */
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_rtc_get_timestamp() and why is it useful for NTP?
A: You should call ca_rtc_get_timestamp() when you need a Unix timestamp in seconds since 1900, which is required for NTP synchronization. This function provides a timestamp that is compatible with NTP protocol, making it suitable for network time synchronization tasks.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_rtc_get_timestamp() in a caOS application?
A: u32 timestamp = ca_rtc_get_timestamp(); if (timestamp != 0) { // Use timestamp for NTP or logging // Example: log timestamp }

Return conf:0.90 **Q:** What does ca_rtc_get_timestamp() return, and what causes it to return 0?
A: The function returns a u32 value representing the number of seconds since January 1, 1900. It returns 0 only in case of an error, though such an error is not expected as the function assumes RTC is already initialized and configured.

Error/Edge case conf:0.85 **Q:** What are the edge cases or error conditions I should be aware of when using ca_rtc_get_timestamp()?
A: The function is designed to not handle RTC read errors, and it assumes the RTC is already initialized and configured. It is not safe to call from an ISR, and it is a blocking function. If the RTC hardware is not properly initialized, the function may return 0, but this is not explicitly handled by the function.

Behavior conf:0.90 **Q:** Is ca_rtc_get_timestamp() blocking or non-blocking, and can it be called from an ISR?
A: The function is blocking and must not be called from an ISR. It requires access to the RTC hardware and is not ISR-safe. It is thread-safe if access to the RTC is protected by

a mutex or similar mechanism.

---

ca_rtc_hw_calibrate

Header: caos/drivers/inc/ca_glue_hw.h:227
GUARD: EN_CA_GLUE_HW

```
extern bool ca_rtc_hw_calibrate(i32 ppm); /* applies frequency
   correction to the RTC oscillator in ppm */
```

Applies a fine-tuning frequency correction to the RTC oscillator in parts-per-million (ppm).

Parametri:

Nome	Tipo	Descrizione
ppm	i32	The correction value in ppm. Positive values add clock pulses (speed up), negative values remove pulses (slow down). Range is limited by hardware capability (typically +/- 488 ppm).

Ritorna:
Returns true if the calibration was successfully applied to the hardware registers, false if the operation timed out or is not supported by the specific MCU architecture.

Note:

- This function is blocking because it must wait for the RTC synchronization flag (RECALPF) to clear before modifying the calibration register. It is not thread-safe; ensure that no other task or ISR attempts to modify RTC settings simultaneously. It is generally not recommended to call this from an ISR due to the potential blocking loop. A common mistake is assuming this function works on all STM32 series; on older chips like the F1xx, this function is a stub that returns false because the hardware lacks a fine-calibration register. Always check the return value to ensure the hardware actually accepted the command.

Uso:

- This function is typically used during system initialization or after a temperature sensor reading indicates a drift in the crystal oscillator frequency. It is usually called once after ca_rtc_init to apply a factory-calibrated offset. It should be used sparingly, as frequent writes to the RTC backup domain registers can cause unnecessary wear or synchronization delays. It is often paired with ca_rtc_init to ensure the RTC peripheral is ready before applying the correction.

Esempio:

```
/* Example: Calibrate RTC based on a known drift of -5 ppm */
#include "caos.h"

void system_setup(void) {
    /* Initialize RTC before attempting calibration */
    if (ca_rtc_init()) {
        i32 drift_ppm = -5;

        /* Apply the calibration value */
        if (!ca_rtc_calibrate(drift_ppm)) {
            /* Handle failure: log error or use default timing */
            ca_printf("RTC calibration failed!\n");
        } else {
            ca_printf("RTC calibrated successfully.\n");
```

```
        }
    }
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_rtc_hw_calibrate() and what is its purpose?
A: You should call ca_rtc_hw_calibrate() when you need to adjust the RTC oscillator frequency to compensate for clock drift. This function applies a frequency correction in parts per million (ppm). It is useful for improving timekeeping accuracy over long periods. However, note that it is not supported on STM32F1xx series and will always return false on those devices.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_rtc_hw_calibrate() with caOS types?
A: bool success = ca_rtc_hw_calibrate(-500); /* Corrects RTC by -500 ppm */ if (success) { /* Calibration applied successfully */ } else { /* Not supported or failed (e.g., on F1xx) */ }

Parameters conf:0.90 **Q:** What does the 'ppm' parameter represent and what values are valid?
A: The 'ppm' parameter represents the frequency correction in parts per million. Positive values increase the RTC frequency, while negative values decrease it. Any integer value is accepted, but only F1xx series ignores the correction. Valid range is theoretically unlimited, though practical limits depend on hardware capabilities.

Return conf:0.95 **Q:** What do the return values of ca_rtc_hw_calibrate() mean and when do they fail?
A: A return value of true indicates that the calibration was applied successfully (on supported families like F4xx, F7xx, H5xx, H7xx). A return value of false means either the calibration is not supported (on F1xx) or it failed due to timeout during register access (on supported families).

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_rtc_hw_calibrate()?
A: Common mistakes include assuming that calibration works on F1xx devices, where it does nothing. Another is not checking the return value, which may lead to incorrect assumptions about time accuracy. Additionally, calling this function without proper RTC unlock sequence or in an ISR context without understanding that it's thread-safe but not blocking may cause unexpected behavior.

Behavior conf:0.90 **Q:** Is ca_rtc_hw_calibrate() blocking, thread-safe, and can it be called from an ISR?
A: The function is not blocking; it waits only briefly for a flag before writing to registers. It is thread-safe because it does not modify shared state. It can be safely called from an ISR since it does not use RTOS primitives and avoids modifying critical sections.

HW differences conf:0.90 **Q:** How does ca_rtc_hw_calibrate() differ between STM32F1xx and other families like F4xx?
A: On STM32F1xx, the function always returns false because there is no fine calibration register available. On F4xx, F7xx, H5xx, and H7xx families, the function uses the RTC_CALR register to apply corrections. F4xx and F7xx use RTC->ISR for checking RECALPF, while H5xx uses RTC->ICSR register. The F7xx and H7xx implementations are nearly identical, differing only in register access.

`ca_rtc_hw_clear_alarm`

Header: `caos/drivers/inc/ca_glue_hw.h:229`
GUARD: `EN_CA_GLUE_HW`

```c
extern bool ca_rtc_hw_clear_alarm(u8 alarm_id); /* disables and clears
    the indicated RTC alarm */
```

Disables a specific RTC alarm and clears its associated interrupt flag and callback.

Parametri:

Nome	Tipo	Descrizione
`alarm_id`	`u8`	The identifier of the alarm to clear. Must be either CA_RTC_ALARM_A or CA_RTC_ALARM_B.

Ritorna:
Returns true if the alarm was successfully cleared, false if the operation failed (e.g., due to hardware configuration lock issues).

Note:

- This function performs hardware-level register modifications, including unlocking and locking the RTC peripheral. It is not inherently thread-safe; if multiple tasks access the RTC, you must ensure synchronization. While some implementations may work from an ISR, it is strongly discouraged to call this from an interrupt context because the locking mechanism might be held by a higher-priority task, leading to a deadlock. A common beginner mistake is attempting to clear an alarm before calling ca_rtc_init, which leaves the RTC registers in an uninitialized or locked state, causing the function to fail or the system to hang.

Uso:

- This function is used to stop an active alarm or to clean up the RTC state after an interrupt has been serviced. It is typically paired with ca_rtc_set_alarm to manage event scheduling. In a standard application flow, you initialize the RTC with ca_rtc_init, configure an alarm, and once the event is processed, you call this function to disable the alarm hardware and clear the internal callback pointer, preventing spurious interrupts or re-triggering.

Esempio:

```c
/* Example: Clearing an alarm after it has triggered */
#include "caos.h"

void handle_alarm_event(u8 alarm_id) {
    /* Perform necessary application logic here */

    /* Disable the alarm to prevent it from firing again */
    if (ca_rtc_clear_alarm(alarm_id)) {
        /* Successfully cleared, safe to re-configure or sleep */
    } else {
        /* Handle error: RTC hardware might be locked or unresponsive */
    }
}
```

Q&A — fine-tuning review

7/7 pairs · qwen3-coder:latest

Usage conf:0.95 **Q:** When should I call ca_rtc_hw_clear_alarm() in my RTC application?
A: You should call ca_rtc_hw_clear_alarm() when you need to disable and clear an RTC alarm, typically before reconfiguring it or after it has triggered. This is useful for resetting alarms before setting new ones or cleaning up after alarm usage.

Code example conf:0.95 **Q:** Can you show a minimal example of how to use ca_rtc_hw_clear_alarm()?
A:

```c
u8 alarm_id = CA_RTC_ALARM_A;
bool success = ca_rtc_hw_clear_alarm(alarm_id);
```

```
if (!success) {
    // Handle error
}
```

Parameters conf:0.95 **Q:** What does the alarm_id parameter in ca_rtc_hw_clear_alarm() represent and what values are valid?
A: The alarm_id parameter specifies which RTC alarm to clear. Valid values are CA_RTC_ALARM_A (0) and CA_RTC_ALARM_B (1). Any other value will cause the function to return false.

Return conf:0.95 **Q:** What does ca_rtc_hw_clear_alarm() return and what causes it to return false?
A: The function returns true if the alarm was successfully cleared. It returns false if the alarm_id parameter is invalid or if rtc_cfg_enter() or rtc_cfg_exit() fail during the operation.

Error/Edge case conf:0.90 **Q:** What are common mistakes or edge cases when using ca_rtc_hw_clear_alarm()?
A: Common mistakes include calling the function from an ISR, which may interfere with interrupt context, or calling it without ensuring the RTC is initialized. Additionally, calling it from multiple threads simultaneously can cause race conditions since the function is not thread-safe.

Behavior conf:0.95 **Q:** Is ca_rtc_hw_clear_alarm() blocking or non-blocking, and is it safe to call from an ISR or from multiple threads?
A: The function is blocking because it calls rtc_cfg_enter() and rtc_cfg_exit(), which may wait or fail. It is not ISR-safe and should not be called from interrupt service routines. It is also not thread-safe, so concurrent calls from multiple threads may cause issues.

HW differences conf:0.90 **Q:** How does ca_rtc_hw_clear_alarm() differ between different STM32 families like F4xx and H7xx?
A: On F4xx and F7xx, the function disables the alarm by clearing bits in RTC->CR and clears the alarm flag in RTC->ISR. On H7xx, it also clears the alarm flag in RTC->ISR but uses EXTI_D1->PR1 instead of EXTI->PR. On H5xx, it clears the alarm flag in RTC->SCR instead of RTC->ISR.

`ca_rtc_hw_get`

Header: `caos/drivers/inc/ca_glue_hw.h:226`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_rtc_hw_get(ca_rtc_dt_t *dt); /* reads date and time from
    the RTC peripheral */
```

Reads the current date and time from the hardware RTC peripheral into a provided structure.

Parametri:

Nome	Tipo	Descrizione
`dt`	`ca_rtc_dt_t *`	Pointer to a structure where the date and time values will be stored. Must not be NULL.

Ritorna:
Returns true if the read was successful, false if the pointer is NULL or if the RTC hardware failed to synchronize.

Note:

- This function is a low-level driver interface. It is not thread-safe by itself; if multiple tasks access the RTC, you should wrap calls in a mutex. It is generally non-blocking, but it relies on internal hardware synchronization mechanisms (like rtc_wait_sync or rtc_wait_rsf),

which may wait for the RTC registers to update. Do not call this from an ISR unless you are certain the hardware synchronization will not hang. The most common mistake is passing a NULL pointer or failing to initialize the RTC peripheral via 'ca_rtc_init' before calling this function.

Uso:

- This function is typically used by the application layer to timestamp logs, manage scheduled tasks, or display the current time. It is usually paired with 'ca_rtc_init' during system startup to configure the clock source. In a typical sequence, you call 'ca_rtc_init' once, then periodically call 'ca_rtc_get' to retrieve the time, or 'ca_rtc_set' to update it. If you need to perform time-based logic, you might also use 'ca_rtc_get_timestamp' to get a raw counter value instead of the structured date/time format.

Esempio:

```
ca_rtc_dt_t current_time;

/* Ensure the RTC is initialized before reading */
/* ca_rtc_init(); */

if (ca_rtc_get(&current_time)) {
    /* Successfully read time, now print it */
    ca_printf("Time: %02d:%02d:%02d\n", current_time.hour, current_time.min,
    current_time.sec);
} else {
    /* Handle error: RTC might not be initialized or hardware failure */
    ca_printf("Failed to read RTC!\n");
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_rtc_hw_get() in my embedded application?
A: You should call ca_rtc_hw_get() when you need to read the current date and time from the RTC hardware. It's typically used during system initialization, for time synchronization, or whenever you require accurate real-time information. The function is designed to be called from threads, not from ISRs, due to its blocking nature.

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_rtc_hw_get()?
A:

```
ca_rtc_dt_t current_time;
if (ca_rtc_hw_get(&current_time)) {
    // Successfully read time
    printf("Time: %02d:%02d:%02d\n", current_time.hour, current_time.min,
    current_time.sec);
} else {
    // Failed to read time
    printf("RTC read failed\n");
}
```

Parameters conf:0.95 **Q:** What is the purpose of the dt parameter in ca_rtc_hw_get()?
A: The dt parameter is a pointer to a ca_rtc_dt_t structure where the function stores the current date and time read from the RTC hardware. It must be a valid (non-NULL) pointer; otherwise, the function will return false immediately without attempting to read the RTC.

Return conf:0.95 **Q:** What does ca_rtc_hw_get() return, and what causes it to fail?
A: The function returns true if the read operation was successful, and false otherwise. It fails if dt is NULL, or if the RTC is not ready for reading (e.g., the RSF bit is not set in F1xx or the synchronization is not complete in F4xx and later).

Error/Edge case conf:0.90 **Q:** What are common mistakes developers make when using ca_rtc_hw_get()?

A: Common mistakes include passing a NULL pointer to dt, not checking the return value of the function, and calling the function without ensuring the RTC is initialized. Additionally, calling this function from an ISR is unsafe due to its blocking behavior.

Behavior conf:0.85 **Q:** Is ca_rtc_hw_get() safe to call from multiple threads or from an ISR?

A: It is thread-safe if the RTC is accessed exclusively, but calling it from an ISR is not safe because it may block indefinitely waiting for synchronization, potentially causing deadlocks or race conditions.

HW differences conf:0.90 **Q:** How does ca_rtc_hw_get() differ between F1xx and F4xx families?

A: In F1xx, the function reads the counter registers CNTH and CNTL, handling potential carry by re-reading CNTH. In F4xx and later families, it reads TR and DR registers in a specific order to avoid shadow register locking issues, and uses bcd2dec for conversion.

`ca_rtc_hw_get_src`

Header: `caos/drivers/inc/ca_glue_hw.h:237`
GUARD: `EN_CA_GLUE_HW`

```
extern ca_rtc_clk_src_t ca_rtc_hw_get_src(void); /* source kept by the
    backup domain; CA_RTC_CLK_AUTO = none (cold) */
```

Reads the current RTC clock source configuration from the hardware backup domain registers.

Ritorna:
Returns a ca_rtc_clk_src_t enum value: CA_RTC_CLK_LSE (external crystal), CA_RTC_CLK_LSI (internal low-speed oscillator), CA_RTC_CLK_HSE (external high-speed oscillator), or CA_RTC_CLK_AUT (no source configured, indicating a cold boot).

Note:

- This function is non-blocking and thread-safe as it performs a simple read of the RCC backup domain register. It is safe to call from an ISR because it does not involve any waiting, mutexes, or complex logic. The backup domain is a special area of the STM32 that persists through system resets (but not power-off), allowing the RTC to maintain its configuration across warm reboots. A common mistake is assuming the RTC is ready just because a source is returned; this function only reports the hardware register state, not the operational status or stability of the clock source.

Uso:

- This function is typically used during the system initialization phase, specifically within the RTC driver's startup sequence. It is called to determine if the RTC was already running before the current reset, which helps the system decide whether to re-initialize the clock or trust the existing configuration. It is usually paired with ca_rtc_clock_init to perform conditional hardware setup based on the detected source.

Esempio:

```
/* Check the current RTC clock source to decide on initialization strategy */
ca_rtc_clk_src_t current_src = ca_rtc_hw_get_src();

if (current_src == CA_RTC_CLK_AUTO) {
    /* No clock source configured, perform full RTC initialization */
    if (ca_rtc_clock_init()) {
        ca_printf("RTC initialized successfully from cold boot.\n");
    } else {
```

```
        ca_printf("Failed to initialize RTC.\n");
    }
} else {
    /* RTC was already running, skip re-init to preserve time */
    ca_printf("RTC already running on source: %d\n", current_src);
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_rtc_hw_get_src in my caOS application?
A: You should call this function during the system initialization phase, specifically within the RTC driver's startup sequence. It is used to determine if the RTC was already running before a reset, allowing you to decide whether to re-initialize the clock or trust the existing configuration, typically by pairing it with ca_rtc_clock_init.

Code example conf:1.00 **Q:** How do I check the RTC clock source in C using caOS types?
A: You can check the source by calling the function and comparing the result against the ca_rtc_clk_src_t enum values. Example: ca_rtc_clk_src_t src = ca_rtc_hw_get_src(); if (src == CA_RTC_CLK_AUTO) { /* Perform cold boot initialization */ }

Return conf:1.00 **Q:** What do the different return values of ca_rtc_hw_get_src indicate?
A: The function returns CA_RTC_CLK_LSE (external crystal), CA_RTC_CLK_LSI (internal low-speed oscillator), or CA_RTC_CLK_HSE (external high-speed oscillator) if a source is configured. CA_RTC_CLK_AUTO is returned if no source is configured, indicating a cold boot. Note that these values only report the hardware register state, not the operational stability of the clock.

HW differences conf:1.00 **Q:** Are there significant differences in how ca_rtc_hw_get_src is implemented across STM32 families?
A: While the logic is consistent across F1xx, F4xx, F7xx, H5xx, H7xx, and U5xx families, the implementation details vary by register bitmask. The F1xx family uses the RCC_BDCR_RTCSEL mask, while the F4xx, F7xx, H5xx, H7xx, and U5xx families use the BDCR_RTCSEL_MASK to read the backup domain register (RCC->BDCR).

`ca_rtc_hw_get_sub`

Header: `caos/drivers/inc/ca_glue_hw.h:232`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_rtc_hw_get_sub(ca_rtc_dt_t *dt, u32 *sub, u32 *sub_div);
   /* date/time + fraction of second sub/sub_div */
```

Reads the current RTC date, time, and sub-second fraction in a single atomic-like operation.

Parametri:

Nome	Tipo	Descrizione
`dt`	`ca_rtc_dt_t *`	Pointer to a structure where the date and time (year, month, day, hour, minute, second) will be stored.
`sub`	`u32 *`	Pointer to store the current sub-second value (fraction of a second).
`sub_div`	`u32 *`	Pointer to store the divisor used for the sub-second calculation (PREDIV_S + 1).

Ritorna:
Returns true if the hardware registers were read successfully, false if the RTC is not synchronized or if any pointer is NULL.

Note:

- This function is not thread-safe by itself; it relies on hardware-level register locking (reading SSR locks TR and DR) to ensure the time components belong to the same instant. It is generally safe to call from tasks, but avoid calling it from high-priority ISRs if the RTC synchronization wait (rtc_wait_sync) takes too long, as this could block the system. The most common mistake is failing to check the return value; if the RTC is not initialized or the clock is not running, the function will return false and the data in 'dt' will be invalid.

Uso:

- Use this function when you need high-precision timestamps, such as logging events with sub-second accuracy or calculating time intervals between tasks. It is typically used after the RTC has been initialized via ca_rtc_init. You would pair this with your own logic to convert the 'sub' and 'sub_div' values into milliseconds or microseconds for display or storage. Always ensure the RTC is ready before calling this, as it is a prerequisite for the hardware registers to contain valid, stable data.

Esempio:

```
ca_rtc_dt_t dt;
u32 sub, sub_div;

/* Ensure the RTC is ready before attempting to read */
if (ca_rtc_hw_get_sub(&dt, &sub, &sub_div)) {
    /* Calculate sub-second in milliseconds: (sub * 1000) / sub_div */
    u32 ms = (sub * 1000U) / sub_div;
    ca_printf("Current time: %02d:%02d:%02d.%03d\n", dt.hour, dt.min, dt.sec, ms
    );
} else {
    /* Handle error: RTC might not be initialized or hardware is busy */
    ca_printf("Failed to read RTC time!\n");
}
```

Q&A — fine-tuning review *7/7 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_rtc_hw_get_sub instead of the standard ca_rtc_hw_get?
A: Use ca_rtc_hw_get_sub when your application requires high-precision timestamps, such as logging events with sub-second accuracy or calculating precise time intervals between tasks. It provides the fraction of a second alongside the standard date and time components.

Code example conf:1.00 **Q:** How do I call ca_rtc_hw_get_sub in my code?
A: ca_rtc_dt_t dt; u32 sub, sub_div; if (ca_rtc_hw_get_sub(&dt, &sub, &sub_div)) { /* process time */ }

Parameters conf:1.00 **Q:** What do the 'sub' and 'sub_div' parameters represent?
A: 'sub' stores the current sub-second value (fraction of a second), and 'sub_div' stores the divisor used for the sub-second calculation, which is defined as PREDIV_S + 1.

Return conf:1.00 **Q:** What does it mean if ca_rtc_hw_get_sub returns false?
A: A false return indicates that the hardware registers could not be read successfully. This occurs if the RTC is not synchronized, if the RTC is not initialized, if the clock is not running, or if any of the provided pointers are NULL.

Error/Edge case conf:1.00 **Q:** What is the most common mistake when using this function?
A: The most common mistake is failing to check the return value. If the RTC is not initialized or the clock is not running, the function returns false and the data stored in the 'dt' structure will be invalid.

Behavior conf:1.00 **Q:** Is ca_rtc_hw_get_sub thread-safe and safe to call from an ISR?
A: The function is not thread-safe by itself but relies on hardware-level register locking to ensure atomic reads. While generally safe for tasks, you should avoid calling it from high-priority ISRs because the rtc_wait_sync call can block the system if synchronization takes too long.

HW differences conf:0.90 **Q:** How does the RTC hardware implementation differ across STM32 families?
A: Implementations vary significantly; for example, the F1xx family uses CNTH/CNTL registers and requires checking for carry-over, while F4xx, F7xx, H5xx, H7xx, and U5xx families utilize TR and DR registers and rely on rtc_wait_sync to ensure data consistency.

`ca_rtc_hw_init`

Header: `caos/drivers/inc/ca_glue_hw.h:218`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_rtc_hw_init(ca_rtc_clk_src_t src); /* inizializza l'RTC
    con la sorgente di clock indicata (LSI/LSE) */
```

Initializes the Real-Time Clock (RTC) hardware peripheral with a specific clock source.

Parametri:

Nome	Tipo	Descrizione
`src`	`ca_rtc_clk_src_t`	The clock source to use: CA_RTC_CLK_LSE (External 32kHz crystal), CA_RTC_CLK_LSI (Internal low-speed RC), or CA_RTC_CLK_HSE (High-speed external divided).

Ritorna:
Returns true if the RTC was successfully initialized and the clock source stabilized. Returns false if the clock source failed to start (timeout) or if the hardware configuration sequence failed.

Note:

- This function is not thread-safe and must be called during system initialization before any other RTC operations. It performs hardware-level register manipulation, including unlocking the backup domain and configuring the prescalers. It is a blocking function because it waits for the selected clock source to become ready (LSERDY/LSIRDY flags). Do not call this from an ISR. A common mistake is failing to provide a stable clock source (e.g., missing crystal for LSE), which will cause the function to return false after the timeout period.

Uso:

- This function is the mandatory first step for using the RTC peripheral. It is typically called once during the system boot sequence or hardware abstraction layer initialization. After a successful call, you can proceed to use other RTC functions like ca_rtc_set or ca_rtc_calibrate. It should be paired with the appropriate clock source selection based on your board's hardware design.

Esempio:

```
/* Initialize RTC with LSE (External Crystal) */
if (ca_rtc_hw_init(CA_RTC_CLK_LSE)) {
    /* RTC is ready, proceed to set initial time */
    ca_rtc_dt_t time = { .year = 2023, .month = 10, .day = 27 };
    ca_rtc_set(&time);
} else {
    /* Handle initialization failure: log error or fallback to LSI */
    ca_printf("RTC Init failed!\n");
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should ca_rtc_hw_init() be called in an embedded application?
A: ca_rtc_hw_init() should be called during system initialization, before any other RTC-related functions. It must be called once to configure the RTC clock source and enable the RTC peripheral. It is typically called early in the boot sequence, after enabling the necessary clocks for PWR and BKP/RCC peripherals.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_rtc_hw_init()?
A: bool success = ca_rtc_hw_init(CA_RTC_CLK_LSE); if (!success) { /* handle error */ }

Parameters conf:0.95 **Q:** What are the valid values for the 'src' parameter in ca_rtc_hw_init()?
A: The valid values for 'src' are CA_RTC_CLK_LSE, CA_RTC_CLK_LSI, and CA_RTC_CLK_HS
Using any other value will cause the function to return false.

Return conf:0.90 **Q:** What does it mean when ca_rtc_hw_init() returns false?
A: A return value of false indicates that the function failed to initialize the RTC. This can happen due to timeout waiting for the clock source to stabilize, or if an invalid clock source was specified.

Error/Edge case conf:0.85 **Q:** What are common mistakes when calling ca_rtc_hw_init()?
A: Common mistakes include calling the function multiple times, not enabling the required PWR and RCC clocks before calling it, or specifying an invalid clock source. Also, calling it after other RTC functions may lead to undefined behavior.

Behavior conf:0.90 **Q:** Is ca_rtc_hw_init() blocking or non-blocking, and is it safe to call from an ISR?
A: ca_rtc_hw_init() is a blocking function that waits for clock sources to stabilize. It is not safe to call from an ISR because it may block and could cause deadlocks or timing issues.

HW differences conf:0.85 **Q:** How does ca_rtc_hw_init() differ between H5xx and other MCU families?
A: On H5xx, the backup domain reset is performed via RCC->BDCR.VSWRST instead of BDRST, and the LSI is controlled via RCC->BDCR.LSION rather than RCC->CSR.LSION. Additionally, H5xx uses direct RTC_IRQn instead of EXTI for interrupt handling.

`ca_rtc_hw_lse_on`

Header: `caos/drivers/inc/ca_glue_hw.h:236`
GUARD: `EN_CA_GLUE_HW`

```
1 extern bool ca_rtc_hw_lse_on(bool on); /* LSE oscillator on/off, RTC
     source untouched; returns LSERDY */
```

Enables or disables the Low-Speed External (LSE) crystal oscillator without modifying the RTC clock source configuration.

Parametri:

Nome	Tipo	Descrizione
`on`	`bool`	Set to true to enable the LSE oscillator, false to disable it.

Ritorna:
Returns true if the LSE oscillator is stable and ready (LSERDY bit set), false otherwise.

Note:

- This function is non-blocking; it initiates the hardware request and immediately returns the current status of the LSE oscillator. It is thread-safe regarding the register access sequence but does not implement internal software delays, meaning the caller must poll the return value to verify if the crystal has started. It is safe to call from an ISR, provided the hardware-specific power/backup domain registers are already accessible. A common mistake is assuming the LSE is ready immediately after calling this function with 'true'; you must implement a timeout loop to wait for the return value to become true.

Uso:

- This function is typically used during the RTC initialization sequence to prepare the clock source before the RTC peripheral is fully configured. It is usually paired with a polling loop that checks the return value against a system timer to ensure the crystal has stabilized within a reasonable timeframe. If the function fails to return true within the expected timeout, the application should abort the RTC initialization to prevent system hangs. It is often followed by a call to initialize the RTC peripheral itself if the LSE is confirmed ready.

Esempio:

```c
/* Attempt to enable LSE and wait for stabilization */
bool lse_ready = false;
int timeout_ms = 500;

/* Poll until LSE is ready or timeout expires */
while (!lse_ready && timeout_ms > 0) {
    lse_ready = ca_rtc_hw_lse_on(true);
    if (!lse_ready) {
        ca_os_delay_ms(1); /* Wait 1ms before checking again */
        timeout_ms--;
    }
}

if (lse_ready) {
    /* Proceed with RTC initialization using LSE */
    /* ca_rtc_init(...) would be called here */
} else {
    /* Handle LSE failure: disable it to save power */
    ca_rtc_hw_lse_on(false);
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_rtc_hw_lse_on and what is its primary purpose?
A: You should call this function during the RTC initialization sequence to prepare the LSE clock source before configuring the RTC peripheral. Its purpose is to enable or disable the LSE crystal oscillator without modifying the existing RTC clock source configuration.

Code example conf:0.95 **Q:** How do I implement a safe startup sequence for the LSE using ca_rtc_hw_lse_on?
A: You must poll the function until it returns true, implementing a timeout to prevent system hangs. Example: bool ready = false; for(u32 i = 0; i < 1000000; i++) { if(ca_rtc_hw_lse_on(true)) { ready = true; break; } } if(!ready) { /* Handle error */ }

Parameters conf:1.00 **Q:** What are the valid values for the 'on' parameter in ca_rtc_hw_lse_on?
A: The 'on' parameter is a boolean. Passing 'true' enables the LSE oscillator, while passing 'false' disables it. The function does not accept any other values as it strictly follows the bool type definition.

Return conf:1.00 **Q:** What does the return value of ca_rtc_hw_lse_on indicate?

A: The return value is a boolean representing the status of the LSERDY bit. A return of 'true' indicates the LSE oscillator is stable and ready. A return of 'false' indicates the oscillator is not yet ready or is disabled.

HW differences conf:1.00 **Q:** How does the implementation of ca_rtc_hw_lse_on vary across different STM32 families?

A: Implementations vary primarily in how they access the backup domain. F1xx/F4xx/F7xx enable the PWR peripheral via RCC->APB1ENR and set the DBP bit in PWR->CR/CR1. H5xx uses PWR->DBPCR, H7xx uses PWR->CR1, and U5xx uses PWR->DBPR followed by a ___DSB() instruction.

`ca_rtc_hw_set`

Header: `caos/drivers/inc/ca_glue_hw.h:225`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_rtc_hw_set(const ca_rtc_dt_t *dt); /* sets date and time
   in the RTC peripheral */
```

Updates the hardware Real-Time Clock (RTC) registers with the provided date and time structure.

Parametri:

Nome	Tipo	Descrizione
`dt`	`const ca_rtc_dt_t *`	Pointer to a structure containing the date and time values (hour, min, sec, year, month, day, wday). Must not be NULL.

Ritorna:
Returns true if the RTC registers were successfully updated, false if the input pointer is NULL or if the hardware failed to enter initialization mode.

Note:

- This function is not thread-safe and should be protected by a mutex if multiple tasks attempt to access the RTC simultaneously. It is a blocking operation because it waits for the hardware to enter and exit initialization mode, which can take several clock cycles. It must NOT be called from an ISR because it performs blocking operations and potentially locks/unlocks hardware resources. A common beginner mistake is passing an uninitialized or NULL structure, which will cause the function to return false or write garbage data to the RTC registers.

Uso:

- This function is typically used during system startup to synchronize the RTC with an external time source (like an NTP server) or when the user manually updates the system time via a CLI. It is usually paired with ca_rtc_init to ensure the peripheral is ready before setting the time. In a typical sequence, you would call ca_rtc_init, then populate a ca_rtc_dt_t structure, and finally call ca_rtc_set to commit the values to the hardware.

Esempio:

```
/* Example: Setting the RTC to a specific date and time */
ca_rtc_dt_t new_time;

/* Populate the structure with desired values */
new_time.hour = 14;
new_time.min = 30;
new_time.sec = 0;
```

```
new_time.year = 23;
new_time.month = 10;
new_time.day = 25;
new_time.wday = 3; /* Wednesday */

/* Ensure RTC is initialized before setting */
ca_rtc_init();

/* Update the hardware RTC */
if (ca_rtc_set(&new_time)) {
    /* Success: Time updated */
} else {
    /* Error: Failed to update RTC */
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_rtc_hw_set() in a caOS application?
A: You should call ca_rtc_hw_set() when you need to initialize or update the real-time clock (RTC) hardware with a specific date and time. It is typically used during system initialization or when synchronizing the RTC with an external time source.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_rtc_hw_set() with caOS types?
A: ca_rtc_dt_t dt = { .year = 2024, .month = 10, .day = 15, .hour = 14, .min = 30, .sec = 45, .wday = 5 }; bool success = ca_rtc_hw_set(&dt);

Parameters conf:0.95 **Q:** What does the dt parameter in ca_rtc_hw_set() represent and what happens if it is NULL?
A: The dt parameter is a pointer to a ca_rtc_dt_t structure containing the date and time values to be set. If dt is NULL, the function returns false immediately without attempting to update the RTC hardware.

Return conf:0.95 **Q:** What do the return values of ca_rtc_hw_set() indicate?
A: The function returns true if the RTC was successfully updated with the provided date and time. It returns false if dt is NULL or if the RTC configuration process fails during entry or exit from initialization mode.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_rtc_hw_set()?
A: Common mistakes include calling the function with a NULL pointer for dt, calling it from an ISR (which is not allowed), and not properly managing RTC lock/unlock operations if the underlying implementation requires it.

Behavior conf:0.95 **Q:** Is ca_rtc_hw_set() blocking or non-blocking, and can it be called from an ISR?
A: The function is blocking, as it waits for the RTC to be configured and updated. It cannot be called from an ISR because it may cause synchronization issues or deadlocks.

HW differences conf:0.85 **Q:** How does ca_rtc_hw_set() differ between F1xx and F4xx families in terms of implementation?
A: The F1xx family uses direct register writes to CNTH and CNTL for setting time, while the F4xx family uses RTC->TR and RTC->DR registers with BCD conversion for hours, minutes, seconds, year, month, day, and weekday.

`ca_rtc_hw_set_alarm`

Header: `caos/drivers/inc/ca_glue_hw.h:228`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_rtc_hw_set_alarm(u8 alarm_id, const ca_rtc_alarm_t *alarm
    , ca_rtc_alarm_cb_t cb); /* configures an RTC alarm with callback */
```

Configures a hardware RTC alarm with a specific callback function to be executed when the alarm triggers.

Parametri:

Nome	Tipo	Descrizione
`alarm_id`	`u8`	The alarm identifier, typically CA_RTC_ALARM_A or CA_RTC_ALARM_B. Must match the hardware capabilities of the specific STM32 series.
`alarm`	`const ca_rtc_alarm_t *`	Pointer to a structure containing the alarm time settings (hours, minutes, seconds, etc.). Must not be NULL.
`cb`	`ca_rtc_alarm_cb_t`	The function pointer to be called when the alarm event occurs. Can be NULL if no callback is required.

Ritorna:
Returns true if the alarm was successfully configured, false if the hardware timed out waiting for the RTC registers to become writable or if input parameters were invalid.

Note:

- This function is not thread-safe and must be protected by a mutex or critical section if multiple tasks access the RTC simultaneously. It performs blocking operations while waiting for the RTC hardware synchronization flags (e.g., ALRAWF/ALRBWF), meaning it should never be called from an ISR. Before calling this, ensure the RTC peripheral has been properly initialized via ca_rtc_init. A common beginner mistake is assuming the alarm will trigger immediately if the time is set in the past, or failing to check the return value, which leads to silent failures if the RTC hardware is busy or locked. Note that on some older hardware like F1xx, the alarm functionality is limited and may not support callbacks.

Uso:

- This function is used to schedule periodic or one-shot events based on the real-time clock. It is typically paired with ca_rtc_init to set up the peripheral, followed by ca_rtc_set_alarm to define the trigger time. If the alarm is no longer needed, it should be disabled or cleared using ca_rtc_clear_alarm to prevent unwanted interrupts or callback executions. It is the standard way to implement wake-up events or scheduled tasks in the system.

Esempio:

```
ca_rtc_alarm_t my_alarm;
/* Define the callback function to be executed on alarm */
void alarm_handler_function(void) {
    /* Handle the alarm event */
}

/* Initialize the alarm structure with desired time */
my_alarm.hour = 14;
my_alarm.min = 30;
my_alarm.sec = 0;

/* Configure the hardware alarm A using the high-level wrapper */
/* ca_rtc_set_alarm internally calls ca_rtc_hw_set_alarm */
if (ca_rtc_set_alarm(CA_RTC_ALARM_A, &my_alarm, alarm_handler_function)) {
    /* Alarm successfully set */
} else {
    /* Handle error: hardware busy or invalid parameters */
```

```
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_rtc_hw_set_alarm to configure an RTC alarm?
A: You should use ca_rtc_hw_set_alarm when you need to schedule a future event based on RTC time. It's useful for periodic tasks, wake-up from low power modes, or triggering actions at specific times. The function configures the alarm to be detected via polling, not interrupts.

Code example conf:0.95 **Q:** Can you show a minimal example of how to set an RTC alarm using caOS types?
A: ca_rtc_alarm_t alarm = { .hour = 10, .min = 30, .sec = 0 }; bool success = ca_rtc_hw_set_alarm(CA_RTC_ALARM_A, &alarm, NULL); if (!success) { /* handle error */ }

Parameters conf:0.95 **Q:** What does the alarm_id parameter control in ca_rtc_hw_set_alarm?
A: The alarm_id parameter specifies which hardware alarm to configure (CA_RTC_ALARM_A or CA_RTC_ALARM_B). On F1xx MCUs, this parameter is ignored as only one alarm is available. For other families, it selects between two separate alarm registers.

Return conf:0.95 **Q:** What does ca_rtc_hw_set_alarm return, and what causes it to return false?
A: It returns true on success. It returns false if the alarm pointer is NULL or if accessing the RTC hardware fails during configuration. The function may also fail if the RTC is not initialized or accessible.

Error/Edge case conf:0.95 **Q:** What are common mistakes when calling ca_rtc_hw_set_alarm?
A: Common mistakes include passing a NULL pointer for the alarm structure, using an invalid alarm_id (not 0 or 1), or calling the function before initializing the RTC. Also, if the RTC is not powered or accessible, the function will fail.

Behavior conf:0.95 **Q:** Is ca_rtc_hw_set_alarm thread-safe and ISR-safe?
A: No, ca_rtc_hw_set_alarm is not thread-safe and should not be called from an ISR. It accesses shared RTC hardware resources and may cause synchronization issues in multi-threaded environments without proper protection.

HW differences conf:0.90 **Q:** How does ca_rtc_hw_set_alarm differ between F1xx and F4xx families?
A: On F1xx, only one alarm is supported and alarm_id is ignored. The alarm is configured using RTC->ALRH and RTC->ALRL registers directly. On F4xx and later families, two alarms are supported using RTC->ALRMAR and RTC->ALRMBR with interrupt enable bits in RTC->CR.

`ca_rtc_hw_set_prescaler`

Header: `caos/drivers/inc/ca_glue_hw.h:235`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_rtc_hw_set_prescaler(u32 div_a, u32 div_s); /* one second
    = div_a*div_s RTCCLK cycles (async 1..128, sync 1..32768) */
```

Configures the RTC prescaler dividers to define the internal clock frequency for timekeeping.

Parametri:

Nome	Tipo	Descrizione
`div_a`	`u32`	Asynchronous prescaler divider. Valid range: 1 to 128.
`div_s`	`u32`	Synchronous prescaler divider. Valid range: 1 to 32768.

Ritorna:
Returns true if the prescalers were successfully updated, false if parameters are out of range or hardware initialization failed.

Note:

- This function is not thread-safe and must be called when the RTC is in a state that allows configuration (typically during initialization). It is not callable from an ISR because it involves blocking operations and hardware register locking/unlocking sequences. A common mistake is providing values that result in an invalid clock frequency or exceeding the hardware's 20-bit register limit on older F1xx series, which will cause the function to return false immediately. Always ensure the RTC clock source is stable before calling this, as the hardware requires a specific sequence to enter 'init mode' to modify these registers.

Uso:

- This function is used during the early system boot phase or when recalibrating the RTC clock source (e.g., switching from LSE to LSI). It is typically paired with other RTC initialization functions to set up the time base before enabling alarms or reading the current time. In a typical sequence, you would call this to define the clock frequency, then proceed to set the initial date/time or start the RTC counter.

Esempio:

```c
/* Example: Configuring RTC for a 1Hz clock base using a 32.768kHz crystal */
#include "caos.h"

void setup_rtc_clock(void) {
    /* Set prescalers: 128 (async) * 256 (sync) = 32768 cycles per second */
    if (!ca_rtc_hw_set_prescaler(128U, 256U)) {
        /* Handle configuration error: log or enter safe state */
        ca_printf("RTC prescaler configuration failed!\n");
    } else {
        /* RTC is now running at 1Hz, safe to proceed with time setting */
        ca_printf("RTC clock base configured successfully.\n");
    }
}
```

Q&A — fine-tuning review *7/7 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_rtc_hw_set_prescaler in my caOS application?
A: This function should be called during the early system boot phase or when recalibrating the RTC clock source, such as switching from LSE to LSI. It is intended to define the RTC time base before you start the counter or enable alarms.

Code example conf:1.00 **Q:** Provide a minimal example of how to configure the RTC prescalers using caOS types.
A: u32 async_div = 128U; u32 sync_div = 256U; if (ca_rtc_hw_set_prescaler(async_div, sync_div)) { /* Success */ } else { /* Handle error */ }

Parameters conf:1.00 **Q:** What are the valid ranges for the div_a and div_s parameters?
A: For most families, div_a (asynchronous) must be between 1 and 128, and div_s (synchronous) must be between 1 and 32768. On the F1xx series, the product of these values minus one must not exceed the 20-bit limit (0xFFFFF).

Return conf:1.00 **Q:** What does the boolean return value indicate?
A: It returns true if the prescalers were successfully updated. It returns false if the provided parameters are out of range, if the hardware initialization fails, or if the hardware register limits are exceeded.

Error/Edge case conf:1.00 **Q:** What is a common mistake when calling ca_rtc_hw_set_prescaler?
A: A common mistake is providing values that result in an invalid clock frequency or exceeding the hardware's 20-bit register limit on older F1xx series, which causes the

function to return false immediately.

Behavior conf:1.00 **Q:** Is ca_rtc_hw_set_prescaler thread-safe or safe to call from an ISR?
A: No, it is not thread-safe and must not be called from an ISR. It involves blocking operations and hardware register locking/unlocking sequences that require the RTC to be in a specific state (init mode).

HW differences conf:1.00 **Q:** How does the implementation differ between the F1xx series and other families like F4xx or H7xx?
A: The F1xx series uses a 20-bit PRL register (split into PRLH and PRLL) and performs a blocking wait for the SECF flag. The F4xx, F7xx, H5xx, H7xx, and U5xx families use a PRER register and write the synchronous and asynchronous dividers in two specific accesses while in init mode.

`ca_rtc_init`

Header: `caos/drivers/inc/rtc.h:80`
Source: `caos/drivers/src/rtc.c:325:337`
GUARD: `EN_RTC`

```
bool ca_rtc_init(ca_rtc_clk_src_t src)
```

Initializes the Real-Time Clock (RTC) peripheral and configures its clock source.

Parametri:

Nome	Tipo	Descrizione
`src`	`ca_rtc_clk_src_t`	The clock source to drive the RTC. Use CA_RTC_CLK_AUTO to let the kernel decide based on boot configuration, or specify CA_RTC_CLK_LSE or CA_RTC_CLK_LSI.

Ritorna:
Returns true if the RTC hardware was successfully initialized, false otherwise (e.g., hardware failure or clock source calibration error).

Note:

- This function is not thread-safe and must be called during the system initialization phase, typically before the RTOS scheduler starts. It is not intended to be called from an Interrupt Service Routine (ISR). If the CLI is enabled in the project configuration, this function registers the RTC command-line interface node. A common mistake is calling this function multiple times or after the RTC has already been used by other tasks, which may lead to unexpected hardware state resets or loss of time synchronization.

Uso:

- This function is used at the very beginning of the application, usually inside the main initialization sequence before starting any tasks. It is typically paired with other hardware initialization functions like ca_pwr_init. Once initialized, the RTC is used to set the system time or configure alarms using ca_rtc_set and ca_rtc_set_alarm. It is often called with CA_RTC_CLK_AUTO to ensure the system uses the most accurate available clock source (LSE if present, otherwise LSI).

Esempio:

```
/* Initialize RTC using the automatic clock selection */
#include "caos.h"

```

```c
void system_init(void) {
    /* Configure power settings before RTC */
    ca_pwr_init();

    /* Initialize RTC with automatic clock source selection */
    if (ca_rtc_init(CA_RTC_CLK_AUTO) == true) {
        /* RTC is ready for use, set initial date/time */
        ca_rtc_dt_t dt = {
            .sec = 0, .min = 0, .hour = 12,
            .wday = 1, .day = 1, .month = 1, .year = 24
        };
        ca_rtc_set(&dt);
    } else {
        /* Handle initialization failure: log error or enter safe state */
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_rtc_init() in my caOS application, and why is it important for system operation?
A: You should call ca_rtc_init() early in your application's initialization phase, typically before any time-sensitive operations or when setting up system timing. It is important because it initializes the Real-Time Clock (RTC) with a specified clock source, which is necessary for the system's timekeeping functionality. Without proper RTC initialization, the system cannot maintain accurate time, which is critical for scheduling, logging, and other time-dependent tasks.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_rtc_init() with caOS types?
A: bool success = ca_rtc_init(CA_RTC_CLK_LSI); if (success) { // RTC initialized successfully } else { // Handle initialization failure }

Parameters conf:0.90 **Q:** What are the valid values for the 'src' parameter in ca_rtc_init(), and what happens if an invalid value is passed?
A: The valid values for 'src' are CA_RTC_CLK_LSI (LSI internal clock), CA_RTC_CLK_LSE (LSE external clock), and CA_RTC_CLK_HSE (HSE external clock). If an invalid value is passed, the function will fail and return false, as the function does not support invalid clock sources.

Return conf:0.95 **Q:** What does it mean when ca_rtc_init() returns true or false, and what causes the function to return false?
A: When ca_rtc_init() returns true, it indicates that the RTC was successfully initialized with the specified clock source. If it returns false, the initialization failed, likely due to the clock source not being available or the RTC peripheral not being properly enabled. Common causes of failure include using a clock source that is not physically present (e.g., LSE without an external crystal) or calling the function after the RTC has already been initialized.

`ca_rtc_lse_start_ms`

Header: `caos/drivers/inc/rtc.h:109`
Source: `caos/drivers/src/rtc.c:320:323`
GUARD: `EN_RTC`

```c
i32 ca_rtc_lse_start_ms(void)
```

Returns the time in milliseconds taken by the LSE (Low Speed External) crystal to start during system power-up.

Ritorna:
Returns a non-negative integer (>= 0) representing the startup time in milliseconds if the LSE successfully started. Returns a negative value (< 0) if the LSE failed to start, was not used, or if the RTC is currently running on a different clock source (like LSI).

Note:

- This function is non-blocking and thread-safe, as it simply returns a cached value calculated during the early boot phase. It is safe to call from an ISR, although it is typically used during system initialization or diagnostic reporting. The most common beginner mistake is assuming this function triggers the LSE startup; it does not. The LSE startup process is handled automatically by the system clock initialization sequence (ca_rtc_clock_init). If you call this before the RTC subsystem has finished its initialization, the returned value may not reflect the final status.

Uso:

- This function is primarily used for diagnostic purposes to verify hardware health and clock stability. It is typically called after ca_rtc_clock_init has been executed to check if the 32.768 kHz crystal is functioning correctly. Developers use it to decide whether to log a warning about slow crystal startup or to confirm that the system is running on the high-precision LSE clock rather than the less accurate internal LSI oscillator. It is often paired with ca_rtc_get_clk_src to determine the current RTC clock source before interpreting the startup time.

Esempio:

```c
/* Check if RTC is using LSE and report its startup performance */
if (ca_rtc_get_clk_src() == 1) { /* 1 corresponds to CA_RTC_CLK_LSE */
    i32 startup_ms = ca_rtc_lse_start_ms();
    if (startup_ms >= 0) {
        /* Log success using the system printf wrapper */
        ca_printf("RTC LSE crystal stable after %d ms\n", (int)startup_ms);
    } else {
        /* Handle the case where LSE is selected but failed to start */
        ca_printf("RTC LSE error: failed to start\n");
    }
} else {
    ca_printf("RTC is not using LSE crystal\n");
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to call ca_rtc_lse_start_ms in my caOS application?
A: This function should be called after ca_rtc_clock_init has been executed. It is primarily used for diagnostic purposes to verify hardware health, check if the 32.768 kHz crystal is functioning correctly, or to confirm the system is using the LSE clock instead of the LSI oscillator.

Code example conf:1.00 **Q:** How do I implement a check for the LSE startup time using caOS types?
A: You can check the startup time as follows: i32 startup_time = ca_rtc_lse_start_ms(); if (startup_time >= 0) { /* LSE started successfully */ } else { /* LSE failed or not in use */ }

Return conf:1.00 **Q:** What do the return values of ca_rtc_lse_start_ms indicate, and why might it return a negative value?
A: A non-negative integer represents the startup time in milliseconds if the LSE successfully started. A negative value indicates the LSE failed to start, was not used, or the RTC is currently running on a different clock source like the LSI.

ca_rtc_lsi_hz

Header: caos/drivers/inc/rtc.h:103
Source: caos/drivers/src/rtc.c:339:342
GUARD: EN_RTC

```
u32 ca_rtc_lsi_hz(void)
```

Returns the current calibrated frequency of the LSI (Low Speed Internal) oscillator in Hertz.

Ritorna:
Returns a u32 value representing the frequency in Hz. Returns 0 if the LSI is not calibrated, if the RTC is not using the LSI clock source, or if the calibration process has not yet completed.

Note:

- This function is non-blocking and thread-safe as it simply returns a cached value stored in a global variable. It is safe to call from an ISR. The value is only accurate if the system has performed a calibration routine (typically involving the SysTick or a stable reference clock) during the RTC initialization phase. If the RTC is configured to use LSE (External Crystal) or HSE, this function will return 0, as the LSI is not the active timebase. A common mistake is assuming this function triggers a new measurement; it does not, it only reports the last known calibrated value stored in the system state.

Uso:

- This function is typically used in diagnostic routines or system status reports to verify the accuracy of the RTC when an external crystal (LSE) is unavailable. It is usually called after the RTC has been initialized via ca_rtc_init. Developers use it to decide whether to apply software compensation to timekeeping or to inform the user via CLI that the RTC is running on an uncalibrated, nominal LSI frequency. It is often paired with ca_rtc_get_clk_src to confirm the clock source before interpreting the frequency value.

Esempio:

```c
/* Check if RTC is using LSI and report its calibrated frequency */
#include "caos.h"

void check_rtc_status(void) {
    /* Check if the current clock source is LSI */
    if (ca_rtc_get_clk_src() == 2) { /* 2 corresponds to CA_RTC_CLK_LSI */
        u32 lsi_freq = ca_rtc_lsi_hz();

        if (lsi_freq > 0) {
            ca_printf("RTC running on calibrated LSI: %u Hz\n", lsi_freq);
        } else {
            /* LSI is active but not calibrated, using nominal frequency */
            ca_printf("RTC running on uncalibrated LSI (nominal)\n");
        }
    } else {
        ca_printf("RTC is not using LSI clock source.\n");
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_rtc_lsi_hz and what is its primary purpose?
A: You should call this function after the RTC has been initialized via ca_rtc_init. It is primarily used in diagnostic routines or system status reports to verify RTC accuracy

when an external crystal (LSE) is unavailable, helping you decide if software compensation is needed or to inform users about the LSI frequency status.

Code example conf:0.90 **Q:** How do I check the LSI frequency and print it using caOS types?
A: You can retrieve the value and verify it as follows: u32 freq = ca_rtc_lsi_hz(); if (freq > 0) { /* Log frequency */ } else { /* Handle uncalibrated or non-LSI state */ }

Return conf:1.00 **Q:** What does a return value of 0 from ca_rtc_lsi_hz indicate?
A: A return value of 0 indicates that either the LSI is not calibrated, the RTC is not currently using the LSI as its clock source (e.g., it is using LSE or HSE), or the system has not performed the necessary calibration routine during initialization.

`ca_rtc_set`

Header: `caos/drivers/inc/rtc.h:114`
Source: `caos/drivers/src/rtc.c:349:353`
GUARD: `EN_RTC`

```
bool ca_rtc_set(const ca_rtc_dt_t *dt)
```

Updates the internal Real-Time Clock (RTC) hardware with the provided date and time structure.

Parametri:

Nome	Tipo	Descrizione
`dt`	`const ca_rtc_dt_t *`	Pointer to a structure containing the date and time fields (year, month, day, hour, min, sec, wday). Must not be NULL.

Ritorna:
Returns true if the RTC hardware was successfully updated, false if the input pointer is NULL or the hardware update failed.

Note:

- This function is a wrapper that interacts directly with the RTC peripheral hardware. It is not inherently thread-safe if multiple tasks attempt to write to the RTC simultaneously, so ensure synchronization (e.g., using a mutex) if your application design allows concurrent access. It is generally non-blocking, but it relies on the underlying hardware driver being initialized via ca_rtc_init first. A common beginner mistake is passing an uninitialized or partially filled structure, which can lead to invalid time values being written to the hardware registers; always ensure all fields, especially the day of the week (wday), are set to valid ranges.

Uso:

- This function is typically used during system startup to set the initial time from a source like an NTP server or user input, or during runtime to adjust the clock. It is often paired with ca_rtc_init for setup and ca_rtc_get to verify the time after setting. In a typical sequence, you initialize the RTC, populate the ca_rtc_dt_t structure with current values, call ca_rtc_set, and then optionally configure alarms using ca_rtc_set_alarm to trigger periodic tasks.

Esempio:

```
ca_rtc_dt_t new_time = { .year = 26, .month = 8, .day = 22, .hour = 12, .min =
    0, .sec = 0, .wday = 5 };

/* Ensure RTC is initialized before setting time */
```

```c
if (ca_rtc_init()) {
    if (ca_rtc_set(&new_time)) {
        ca_printf_n("RTC updated successfully");
    } else {
        ca_printf_n("Failed to update RTC hardware");
    }
} else {
    /* Handle hardware failure */
    ca_printf_n("RTC hardware initialization failed");
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_rtc_set() in a caOS application?
A: You should call ca_rtc_set() when you need to initialize or update the real-time clock (RTC) with a specific date and time. It is typically called after initializing the RTC with ca_rtc_init() and is useful for setting the RTC to the correct time at system startup or when synchronizing time with an external source.

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_rtc_set() with caOS types?
A: ca_rtc_dt_t dt; dt.year = 2024; dt.month = 4; dt.day = 5; dt.hour = 14; dt.minute = 30; dt.second = 0; bool success = ca_rtc_set(&dt); // Handle success or failure as needed

Parameters conf:0.95 **Q:** What does the dt parameter in ca_rtc_set() represent, and what happens if it is NULL?
A: The dt parameter is a pointer to a ca_rtc_dt_t structure that contains the date and time values to be set in the RTC. If dt is NULL, the function immediately returns false because it cannot access the date/time data to write to the hardware.

Return conf:0.90 **Q:** What do the return values of ca_rtc_set() indicate, and what causes it to return false?
A: ca_rtc_set() returns true if the date and time were successfully set in the RTC. It returns false if the dt parameter is NULL or if the hardware write operation fails. The function may also block if the RTC hardware is busy, but this is not explicitly stated as a cause of false return values.

`ca_rtc_set_alarm`

Header: `caos/drivers/inc/rtc.h:158`
Source: `caos/drivers/src/rtc.c:433:438`
GUARD: `EN_RTC`

```c
bool ca_rtc_set_alarm(u8 alarm_id, const ca_rtc_alarm_t *alarm,
    ca_rtc_alarm_cb_t cb)
```

Configures a hardware RTC alarm to trigger a callback function at a specific time.

Parametri:

Nome	Tipo	Descrizione
`alarm_id`	`u8`	The identifier of the alarm to set. Must be either CA_RTC_ALARM_A or CA_RTC_ALARM_B.
`alarm`	`const ca_rtc_alarm_t *`	Pointer to a structure containing the target time and match flags. Must not be NULL.
`cb`	`ca_rtc_alarm_cb_t`	The function pointer to execute when the alarm triggers. Can be NULL if no callback is needed.

Ritorna:
Returns true if the alarm was successfully configured, false if the alarm_id is invalid, the alarm pointer is NULL, or the hardware failed to set the alarm.

Note:

- This function is a non-blocking wrapper that interacts with the STM32 RTC hardware registers. It is thread-safe in the sense that it uses underlying hardware locks, but it should be called after the RTC has been initialized via ca_rtc_init. A common mistake is failing to set the match flags (e.g., match_sec, match_min) correctly in the ca_rtc_alarm_t structure, which results in the alarm never triggering or triggering at the wrong time. Note that the callback function is executed in an interrupt context (ISR), so it must be kept extremely short and should not perform blocking operations like waiting for semaphores or performing heavy calculations.

Uso:

- This function is typically used to schedule wake-up events or periodic tasks based on the real-time clock. It is often paired with ca_rtc_init to prepare the hardware, and ca_rtc_clear_alarm to disable the alarm once it has served its purpose. In power-sensitive applications, it is frequently used before calling ca_pwr_enter to ensure the system wakes up from a low-power state at a specific time. Always ensure the RTC clock source is stable before setting alarms to avoid unexpected behavior.

Esempio:

```c
/* Define the alarm callback */
void my_alarm_callback(void) {
    /* Signal a semaphore to notify the main task */
    /* Note: Use ca_sem_post or similar ISR-safe signaling */
}

void setup_rtc_alarm(void) {
    ca_rtc_alarm_t al;
    /* Initialize RTC hardware first */
    ca_rtc_init();

    /* Configure alarm for 12:30:00 */
    al.sec = 0;
    al.min = 30;
    al.hour = 12;
    al.match_sec = true;
    al.match_min = true;
    al.match_hour = true;
    al.match_day = false;

    if (ca_rtc_set_alarm(CA_RTC_ALARM_A, &al, my_alarm_callback)) {
        /* Alarm successfully armed */
    } else {
        /* Handle configuration error */
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_rtc_set_alarm and what is its purpose in an embedded application?
A: You should call ca_rtc_set_alarm to schedule a future event using the RTC peripheral. It is useful for implementing time-based actions such as periodic tasks, scheduled wake-ups, or automated system events. The function allows you to define a specific date and time for the alarm to trigger, optionally with a callback function to execute when the alarm fires.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_rtc_set_alarm with caOS types?
A: ca_rtc_alarm_t alarm; alarm.seconds = 30; alarm.minutes = 15; alarm.hours = 10; alarm.days = 1; alarm.months = 1; alarm.years = 2025; alarm.enable_seconds = true; alarm.enable_minutes = true; alarm.enable_hours = true; alarm.enable_days = true; alarm.enable_months = true; alarm.enable_years = true; bool result = ca_rtc_set_alarm(CA_RTC_A &alarm, NULL);

Parameters conf:0.95 **Q:** What are the valid values for the alarm_id parameter in ca_rtc_set_alarm, and what happens if an invalid value is passed?
A: The alarm_id parameter must be either CA_RTC_ALARM_A (0) or CA_RTC_ALARM_B (1). If any other value is passed, the function will return false and the alarm will not be set. This is a strict validation to ensure only predefined alarm channels are used.

Return conf:0.90 **Q:** What does the return value of ca_rtc_set_alarm indicate, and what causes it to return false?
A: The function returns true if the alarm was successfully set. It returns false if there is an error, such as an invalid alarm_id (not 0 or 1), a NULL pointer passed for the alarm structure, or if the RTC is not initialized. The function is also not thread-safe and must not be called from an ISR.

`ca_rtc_to_net`

Header: `caos/drivers/inc/rtc.h:143`
Source: `caos/drivers/src/rtc.c:415:426`
GUARD: `EN_RTC`

```
u32 ca_rtc_to_net(void)
```

Reads the hardware RTC and returns it as a standard Unix timestamp (seconds since 1970-01-01 00:00:00 UTC), the inverse of ca_rtc_from_net.

Ritorna:
Unix seconds since 1970-01-01 00:00:00 UTC (the RTC's 2000-based calendar plus RTC_UNIX_2000 = 946684800), so a value of at least 946684800. Returns 0U if the RTC cannot be read or holds an invalid date (month outside 1-12, day 0, year past 2099).

Note:

- The RTC calendar covers 2000-2099 and is taken as UTC: the result is a real Unix time, directly comparable with the value an NTP client hands to ca_rtc_from_net, with no offset to add. The weekday stored in the RTC is ignored; the date alone decides. It calls ca_rtc_get, which waits for the RTC shadow registers to synchronise, so call it from a task, not from an ISR. Concurrent calls only read the RTC and do not interfere. A 0U return means 'no valid time', never a date: test for it before using the value. On the bench: the CLI command 'rtc unix' prints this value with the calendar it came from.

Uso:

- This function is typically used when you need to timestamp log entries, network packets, or security events. It is often paired with ca_rtc_get to validate the hardware state before conversion. In a typical workflow, you would call ca_rtc_get to ensure the clock is running, then call ca_rtc_to_net to get a numeric value for easy comparison or transmission over a network protocol like NTP or MQTT.

Esempio:

```c
/* Stamp an outgoing record with the RTC time, in Unix seconds */
u32 now = ca_rtc_to_net();

if (now != 0U) {
    ca_stdout_printf_n("time: %u (unix, UTC)", now);
} else {
    ca_stdout_printf_n("RTC not set or not readable");
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_rtc_to_net in my caOS application?
A: You should use this function when you need a numeric representation of the current time for tasks like timestamping log entries, network packets, or security events. It is ideal for preparing data for transmission over protocols like NTP or MQTT.

Code example conf:0.90 **Q:** How do I implement a simple check and retrieval of the timestamp using caOS types?
A: u32 timestamp = 0U; if (ca_rtc_get() != 0U) { timestamp = ca_rtc_to_net(); }

Return conf:1.00 **Q:** What does the u32 return value represent and what causes it to return 0?
A: The return value is the total seconds elapsed since the year 2000 (RTC_UNIX_2000). It returns 0U if the RTC hardware is unreachable, if the RTC has not been initialized, or if the internal date/time data is invalid (e.g., a month value greater than 12).

Error/Edge case conf:1.00 **Q:** What is a common mistake when using the timestamp returned by ca_rtc_to_net?
A: A common mistake is assuming the returned value is a standard Unix timestamp starting from 1970. Since this function uses a 2000-based epoch, you must manually add the appropriate offset if you are interfacing with external systems that expect a 1970-based timestamp.

`ca_rwlock_init`

Header: `caos/inc/sync.h:322`
Source: `caos/src/sync.c:755:761`
GUARD: `EN_SYNC`

```c
void ca_rwlock_init(ca_rwlock_t *rw)
```

Initializes a Read-Write lock structure to manage concurrent access to shared resources.

Parametri:

Nome	Tipo	Descrizione
`rw`	`ca_rwlock_t *`	Pointer to the rwlock object to be initialized. Must point to valid, allocated memory.

Ritorna:
void

Note:

- This function is not thread-safe and must be called before any other ca_rwlock_* function is used. It resets the internal reader count, clears the writer status, and initializes the internal wait queues and spinlock. It is strictly intended for initialization and must NOT be called while other tasks are attempting to acquire or release the lock, as this would corrupt the internal state. This function is not designed to be called from an ISR; it should be called during system startup or task initialization.

Uso:

- Use this function when you have a shared resource (like a configuration table or a data buffer) that is read frequently by many tasks but written to only occasionally. It is typically paired with ca_rwlock_read_lock/unlock and ca_rwlock_write_lock/unlock to enforce data integrity. In a typical lifecycle, you declare the lock variable, call ca_rwlock_init once, and then pass the pointer to the various tasks that need to synchronize access to the protected resource.

Esempio:

```c
/* Define the lock globally or in a persistent scope */
ca_rwlock_t my_data_lock;

void task_init_routine(void) {
    /* Initialize the lock before any task attempts to use it */
    ca_rwlock_init(&my_data_lock);
}

void reader_task(void) {
    /* Acquire read access: multiple readers can hold this simultaneously */
    ca_rwlock_read_lock(&my_data_lock);
    /* Perform read operations on shared data here */
    ca_rwlock_read_unlock(&my_data_lock);
}

void writer_task(void) {
    /* Acquire write access: exclusive access, blocks all readers and other
    writers */
    ca_rwlock_write_lock(&my_data_lock);
    /* Perform write operations on shared data here */
    ca_rwlock_write_unlock(&my_data_lock);
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_rwlock_init() in a caOS application, and why is it important for synchronization?
A: You should call ca_rwlock_init() to initialize a RWLock structure before using it for synchronization. It is important because it sets up the internal state of the RWLock, including initializing the waiting queues and the internal lock, ensuring correct concurrent access control for read and write operations. This function can be called from an ISR and is thread-safe, making it suitable for initializing synchronization primitives early in the application lifecycle.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_rwlock_init() with caOS types?
A: ca_rwlock_t my_rwlock; ca_rwlock_init(&my_rwlock);

Parameters conf:0.90 **Q:** What is the significance of the 'rw' parameter in ca_rwlock_init(), and what happens if it is NULL?
A: The 'rw' parameter is a pointer to the RWLock structure that needs to be initialized. It must be a valid, allocated pointer; passing NULL will cause undefined behavior or a crash because the function does not perform NULL checks. The function assumes the memory pointed to is valid and will initialize the RWLock's internal state.

`ca_rwlock_read_lock`

Header: `caos/inc/sync.h:328`
Source: `caos/src/sync.c:763:778`
GUARD: `EN_SYNC`

```
void ca_rwlock_read_lock(ca_rwlock_t *rw)
```

Acquires a read lock on a read-write lock, allowing multiple readers but blocking writers.

Parametri:

Nome	Tipo	Descrizione
rw	ca_rwlock_t *	Pointer to the read-write lock structure. Must not be NULL. Invalid if the lock is not initialized or has been corrupted.

Ritorna:
This function does not return a value. It blocks until the read lock is acquired or an error occurs.

Note:

- This function is blocking: it will wait indefinitely if a writer holds the lock or if a writer is waiting. It is NOT safe to call from an ISR. It is not thread-safe if the same lock is accessed by multiple threads without proper synchronization. The function assumes the lock is properly initialized before use. Common mistake: calling this function without initializing the rwlock structure first.

Uso:

- Used in scenarios where multiple threads need to read shared data concurrently, such as reading configuration data or lookup tables. It pairs with ca_rwlock_read_unlock() to release the lock. Typically used in long-running tasks that read data without modifying it.

Esempio:

```
/* Example: Reading configuration data in a loop */
ca_rwlock_read_lock(&rw_cfg);
/* Critical section: read data safely */
u32 ver = cfg.version;
u32 sum = 0;
for (u32 i = 0; i < CFG_SIZE; i++) {
    sum += cfg.value[i];
}
ca_rwlock_read_unlock(&rw_cfg); /* Release the lock */
/* Other readers can now access the data */
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_rwlock_read_lock() in a caOS application?
A: You should use ca_rwlock_read_lock() when you need to acquire a read lock on a read-write lock (RWLock) to allow concurrent access for multiple readers while preventing writers from accessing the resource. This is useful in scenarios where data is frequently read but infrequently modified, such as accessing shared configuration or lookup tables.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_rwlock_read_lock() with caOS types?
A: ca_rwlock_t lock; ca_rwlock_init(&lock); ca_rwlock_read_lock(&lock); // Critical section for reading ca_rwlock_read_unlock(&lock);

Parameters conf:0.95 **Q:** What is the significance of the rw parameter in ca_rwlock_read_lock() and what happens if it is NULL?
A: The rw parameter is a pointer to a ca_rwlock_t structure that must be initialized with ca_rwlock_init() before use. If a NULL pointer is passed, the function will cause a crash because caOS does not perform null checks.

Return conf:0.90 **Q:** What does ca_rwlock_read_lock() return and what causes it to fail?
A: The function does not return any value. It can block the calling thread if a writer is active or waiting, but it does not return an error code. Failures are not explicitly returned; instead, the thread waits until the lock becomes available.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_rwlock_read_lock()?
A: Common mistakes include calling the function with an uninitialized RWLock, passing a NULL pointer, or calling it from an ISR. Using an uninitialized lock leads to undefined behavior, and calling from an ISR is not allowed. Additionally, improper pairing of read locks with unlocks can lead to deadlocks.

Behavior conf:0.90 **Q:** Is ca_rwlock_read_lock() blocking or non-blocking, and is it safe to call from an ISR or from multiple threads?
A: The function is blocking, meaning it will put the calling thread to sleep if it cannot immediately acquire the lock. It is not safe to call from an ISR, and it is not thread-safe in the sense that it requires a valid thread context. It is designed to be used in thread contexts only.

`ca_rwlock_read_unlock`

Header: `caos/inc/sync.h:333`
Source: `caos/src/sync.c:780:793`
GUARD: `EN_SYNC`

```
1 void ca_rwlock_read_unlock(ca_rwlock_t *rw)
```

Releases a read lock on a read-write lock, allowing other readers or writers to proceed.

Parametri:

Nome	Tipo	Descrizione
`rw`	`ca_rwlock_t *`	Pointer to the read-write lock structure. Must not be NULL. Passing NULL will cause a crash.

Ritorna:
`None`: Function does not return a value. It only modifies internal state.

Note:

- **Blocking behavior**: This function is NOT blocking. It immediately decrements the reader count and may wake up a writer if this was the last reader.
- **Thread safety**: Function is thread-safe. It uses a spinlock internally to protect access to shared data.
- **ISR usage**: Not callable from interrupt service routines (ISR). It uses spinlocks that may not be safe in ISR context.
- **Preconditions**: The calling thread must have previously acquired a read lock using ca_rwlock_read_lock(). Otherwise, behavior is undefined.
- **Common mistakes**: Calling this function without first acquiring a read lock leads to incorrect reader count and potential deadlock or undefined behavior. Also, calling it more times than locking leads to negative reader counts.

Uso:

- **Reading shared data in a multi-threaded environment**: Used after completing a read operation on shared data protected by a read-write lock. Ensures other readers can proceed and potentially allows writers to acquire the lock if no readers remain.
- **Performance-critical sections**: Used in performance-sensitive code where read operations are frequent and write operations are rare. Helps avoid unnecessary blocking of writers.

Esempio:

```c
// This example shows a typical read operation on a shared configuration structure. The lock is acquired before reading and released after, allowing other readers to proceed.
ca_rwlock_read_lock(&rw_cfg); // Acquire read lock before reading
// ... perform read operations on cfg ...
ca_rwlock_read_unlock(&rw_cfg); // Release read lock after reading
// In this example, a task reads from a shared memory region protected by a read-write lock. After completing the read, it releases the lock so other readers or writers can proceed.
ca_rwlock_read_lock(&s_rw);
// ... perform read operations on s_rw_data ...
ca_rwlock_read_unlock(&s_rw); // Release the read lock
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_rwlock_read_unlock() in my embedded application?
A: You should call ca_rwlock_read_unlock() after acquiring a read lock using ca_rwlock_read_lock() to release the reader from the RWLock. This allows any waiting writers to acquire the lock and proceed, enabling proper synchronization in a multi-threaded environment.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_rwlock_read_unlock() with caOS types?
A: ca_rwlock_t lock; ca_rwlock_init(&lock); ca_rwlock_read_lock(&lock); // Critical section for reading ca_rwlock_read_unlock(&lock);

Parameters conf:0.90 **Q:** What is the significance of the 'rw' parameter in ca_rwlock_read_unlock()?
A: The 'rw' parameter is a pointer to a ca_rwlock_t structure that must be valid and initialized. If it is NULL or not properly initialized, calling this function will result in undefined behavior or a crash.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_rwlock_read_unlock()?
A: Common mistakes include calling ca_rwlock_read_unlock() without a prior call to ca_rwlock_read_lock(), passing a NULL pointer, or using an uninitialized RWLock structure. These errors can lead to race conditions or unpredictable behavior.

Behavior conf:0.90 **Q:** Is ca_rwlock_read_unlock() blocking or non-blocking, and can it be called from an ISR?
A: ca_rwlock_read_unlock() is a non-blocking function. It does not wait for anything. It can be called from an ISR only if the context allows it and there is no risk of deadlock. It is not thread-safe if called without an external lock.

`ca_rwlock_write_lock`

Header: `caos/inc/sync.h:338`
Source: `caos/src/sync.c:795:809`
GUARD: `EN_SYNC`

```c
void ca_rwlock_write_lock(ca_rwlock_t *rw)
```

Acquires exclusive write access to a read-write lock, blocking if another writer or readers hold the lock.

Parametri:

Nome	Tipo	Descrizione
`rw`	`ca_rwlock_t *`	Pointer to the read-write lock structure. Must not be NULL. Invalid if the lock is not initialized or has been corrupted.

Ritorna:
`None`: Function does not return a value. It blocks until write access is granted.

Note:

- **Blocking behavior**: This function blocks the calling thread until it can acquire exclusive write access. It will not return until no other threads are reading or writing.
- **Thread-safe**: Yes, but only when used correctly with proper initialization and matching unlock calls.
- **Callable from ISR**: No. This function is not safe to call from an interrupt service routine (ISR). It may block and is not designed for interrupt context.
- **Preconditions**: The rw lock must be initialized before calling this function. The scheduler must be running (not in idle context).
- **Common mistakes**: Calling this function without a matching ca_rwlock_write_unlock() leads to deadlock. Using it in an ISR causes undefined behavior.

Uso:

- **Exclusive access to shared data**: Used when a thread needs to modify shared data that other threads may be reading or writing. Ensures no readers or writers are active before granting access.
- **Configuration updates**: Used to update configuration tables or structures that are accessed by multiple threads. Guarantees atomicity of the update.
- **Writer tasks**: Used in tasks that perform write operations to shared memory regions, ensuring exclusive access during the write cycle.

Esempio:

```c
// This example shows how to use ca_rwlock_write_lock() to safely update a configuration table. The lock ensures that no readers or other writers can access the data during the update.
#define CFG_SIZE 100
ca_rwlock_t rw_cfg;

void update_config_table(void) {
    ca_rwlock_write_lock(&rw_cfg);  // Acquire exclusive write lock
    cfg.version++;                  // Update version
    for (u32 i = 0; i < CFG_SIZE; i++) {
        cfg.value[i] = cfg.version * (i + 1);  // Update values
    }
    ca_rwlock_write_unlock(&rw_cfg); // Release lock
}

void task_main(void) {
    while (1) {
        ca_os_sleep(2000);
        update_config_table();
    }
}
```

Q&A — fine-tuning review

6/6 pairs · qwen3-coder:latest

Usage conf:0.95 **Q:** When should I use ca_rwlock_write_lock() in a caOS application?
A: You should use ca_rwlock_write_lock() when you need exclusive access to a shared resource in a caOS application, ensuring that no other readers or writers can access it simultaneously. This is typically used in scenarios where data integrity is critical, such as updating a shared buffer or modifying a global configuration structure.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_rwlock_write_lock() with caOS types?
A: ca_rwlock_t lock; ca_rwlock_init(&lock); ca_rwlock_write_lock(&lock); // Critical section ca_rwlock_write_unlock(&lock);

Parameters conf:0.95 **Q:** What is the significance of the rw parameter in ca_rwlock_write_lock() and what happens if it is NULL?

A: The rw parameter is a pointer to a ca_rwlock_t structure that must be initialized with ca_rwlock_init(). If this parameter is NULL, the function will cause a crash because caOS does not handle NULL pointer checks for this function.

Return conf:0.95 **Q:** Does ca_rwlock_write_lock() return any value, and what does it mean if the function blocks?

A: The function does not return any value. It is a blocking function, meaning that if the lock is already acquired by another thread, the calling thread will be suspended until the lock becomes available.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_rwlock_write_lock()?

A: Common mistakes include calling the function with a NULL pointer, which causes a crash, or calling it from an ISR, which is not supported. Additionally, failing to call ca_rwlock_write_unlock() after acquiring the lock can lead to deadlocks or indefinite blocking of other threads.

Behavior conf:0.90 **Q:** Is ca_rwlock_write_lock() thread-safe, ISR-safe, and does it block the calling thread?

A: ca_rwlock_write_lock() is not ISR-safe and must be called from a valid thread context. It is a blocking function, meaning it will suspend the calling thread if the lock is already acquired by another thread. It is not thread-safe in the sense that it requires a valid thread context, but it does ensure mutual exclusion among threads.

`ca_rwlock_write_unlock`

Header: `caos/inc/sync.h:344`
Source: `caos/src/sync.c:811:828`
GUARD: `EN_SYNC`

```
1 void ca_rwlock_write_unlock(ca_rwlock_t *rw)
```

Releases a write lock on a read-write lock, allowing other readers or writers to proceed.

Parametri:

Nome	Tipo	Descrizione
`rw`	`ca_rwlock_t *`	Pointer to the read-write lock structure. Must not be NULL. Passing NULL will cause a crash.

Ritorna:
`None`: Function does not return a value. It simply releases the lock.

Note:

- **Blocking behavior**: This function is NOT blocking. It immediately releases the lock and may wake up waiting threads.
- **Thread safety**: Function is thread-safe as long as the rw pointer is valid and no other thread is modifying the same lock concurrently.
- **ISR context**: Not callable from interrupt service routines (ISR). It uses spinlocks that are not safe in ISR context.
- **Preconditions**: The calling thread must already hold the write lock. If not, behavior is undefined and may lead to deadlock or corruption.
- **Common mistakes**: Calling this function without holding the write lock, or calling it multiple times without matching locks, leads to undefined behavior.

Uso:

- **After completing a write operation on shared data**: Used to release a write lock acquired earlier using ca_rwlock_write_lock(). Typically used in a critical section where data is being modified.

- **In multi-threaded applications with shared resources**: Paired with ca_rwlock_write_lock() to protect write access to shared data structures, ensuring exclusive access during writes.

Esempio:

```
// Example shows a typical use case where a configuration is updated with a write lock, then unlocked after modification.
ca_rwlock_write_lock(&rw_cfg);  // Acquire write lock before modifying config
cfg.version = ++seq;
for (u32 i = 0; i < CFG_SIZE; i++) {
    cfg.value[i] = seq * (i + 1u);
}
ca_rwlock_write_unlock(&rw_cfg);  // Release write lock after modification
writes_ok++;
ca_stdout_printf_n("CFG update v%u (reads so far: %u)", seq, reads_ok);
// Another example where a data array is modified under a write lock, then unlocked.
ca_rwlock_write_lock(&s_rw);  // Acquire write lock before modifying data
for (u32 i = 0U; i < E91_RW_WORDS; i++) {
    s_rw_data[i] = v;
    if ((i & 3U) == 3U) ca_os_yield();  // Optional yield for fairness
}
ca_rwlock_write_unlock(&s_rw);  // Release write lock after modification
v++;
s_rw_writes++;
ca_os_yield();
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_rwlock_write_unlock() in my embedded application?
A: You should call ca_rwlock_write_unlock() after acquiring a write lock using ca_rwlock_write_lock() to release the lock and allow other threads to access the protected resource. It is typically called at the end of a critical section where exclusive access is required.

Code example conf:0.90 **Q:** Can you show a minimal example of using ca_rwlock_write_unlock() with caOS types?
A: ca_rwlock_t lock; ca_rwlock_init(&lock); ca_rwlock_write_lock(&lock); // Critical section ca_rwlock_write_unlock(&lock);

Parameters conf:0.90 **Q:** What is the significance of the rw parameter in ca_rwlock_write_unlock()?
A: The rw parameter is a pointer to a ca_rwlock_t structure that must be initialized and previously acquired with ca_rwlock_write_lock(). If NULL, the function will cause a crash as per the documentation.

Return conf:0.95 **Q:** Does ca_rwlock_write_unlock() return any value, and what does it signify?
A: The function does not return any value. It is void and is guaranteed not to fail, but it may block if there are waiting threads.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_rwlock_write_unlock()?
A: Common mistakes include calling this function without first acquiring the lock with ca_rwlock_write_lock(), passing a NULL pointer, or calling it from an ISR. These can lead to undefined behavior or system crashes.

Behavior conf:0.90 **Q:** Is ca_rwlock_write_unlock() thread-safe and ISR-safe?
A: It is not thread-safe in the sense that it requires the lock to have been previously acquired by the same thread. It is also not ISR-safe and must not be called from interrupt service routines.

`ca_safeguard_failed`

Header: `caos/inc/safeguard.h:92`
Source: `caos/src/safeguard.c:53:112`

GUARD: `EN_SAFEGUARD`

```
void ca_safeguard_failed(const char *cond_str, const char *msg, const
    char *file, u32 line)
```

Triggers a system-wide fatal error handler to safely halt execution when an invariant or critical condition is violated.

Parametri:

Nome	Tipo	Descrizione
`cond_str`	`const char *`	A string literal describing the condition that failed (e.g., "ptr != NULL").
`msg`	`const char *`	Optional additional context or error message; can be NULL.
`file`	`const char *`	The source filename where the failure occurred, usually passed via the ___FILE___ macro.
`line`	`u32`	The line number where the failure occurred, usually passed via the ___LINE___ macro.

Ritorna:
This function never returns. It either triggers an SVC exception to perform a controlled system shutdown or enters an infinite loop (HALT) if the OS is not running or if called from an ISR.

Note:

- This function is the ultimate 'panic' mechanism. It is not thread-safe in the sense that it stops the entire system, but it is designed to be called from any context. If called from an ISR or before the scheduler starts, it disables interrupts and enters an infinite loop to prevent further system corruption. The most common beginner mistake is using this for recoverable errors; only use this for unrecoverable state corruption where continuing execution would be dangerous. It is mandatory that the system is in a state where it can at least attempt to log the error before halting.

Uso:

- Use this function to enforce critical system invariants that, if broken, would lead to undefined behavior or hardware damage. It is typically wrapped in a macro (e.g., #define ASSERT(cond) if(!(cond)) ca_safeguard_failed(#cond, NULL, ___FILE___, ___LINE___)) to automatically capture the file and line information. It is used during development to catch logic errors early and in production to ensure the system enters a safe state rather than operating with corrupted data.

Esempio:

```
/* Example of using a safeguard to protect a critical memory operation */
void process_data(void *ptr) {
    /* Check if pointer is valid before usage */
    if (ptr == NULL) {
        /* Trigger fatal safeguard if pointer is NULL */
        ca_safeguard_failed("ptr != NULL", "Critical buffer pointer is NULL",
    __FILE__, __LINE__);
    }

    /* If we reach here, the pointer is guaranteed to be valid */
    ca_memzero(ptr, 128);
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_safeguard_failed in my caOS application?
A: You should call ca_safeguard_failed when a critical safety condition fails, such as a NULL pointer dereference or an invalid state check. It is typically used in conjunction with assert-like macros to halt execution when assumptions are violated.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_safeguard_failed with caOS types?
A:

```c
#include "caos.h"
void example_function(void *ptr) {
    if (ptr == NULL) {
        ca_safeguard_failed("ptr != NULL", "Pointer is NULL", __FILE__, __LINE__);
    }
}
```

Parameters conf:0.90 **Q:** What are the valid values for the cond_str parameter in ca_safeguard_failed?
A: The cond_str parameter must be a valid, non-NULL string that represents the failed condition, such as "ptr != NULL" or "x < MAX". It is used for logging or debugging purposes and must not be NULL.

Error/Edge case conf:0.85 **Q:** What happens if I call ca_safeguard_failed from an ISR when the scheduler is not active?
A: If ca_safeguard_failed is called from an ISR when the scheduler is not active, it will immediately halt execution without attempting to issue an SVC, as SVC is not possible in that context.

Behavior conf:0.90 **Q:** Is ca_safeguard_failed thread-safe and ISR-safe?
A: ca_safeguard_failed is not thread-safe because it uses a global variable g_in_safeguard to prevent recursion. It is also not ISR-safe if called from an ISR when the scheduler is inactive, as it may not be able to issue an SVC.

`ca_safeguard_reset`

Header: `caos/inc/safeguard.h:98`
Source: `caos/src/safeguard.c:32:34`
GUARD: `EN_SAFEGUARD`

```c
void ca_safeguard_reset(void)
```

Clears the internal safeguard recursion flag to allow future fault handling.

Ritorna:
This function returns void and has no return value.

Note:

- This function is used to reset a global flag that prevents the system from entering a recursive loop if a fault occurs while already processing another fault. It is not thread-safe in the sense that it modifies a global variable, but it is intended to be called by the scheduler during task recovery. If you fail to call this after a task has been safely restarted or cleaned up, the system will permanently block any future safeguard triggers, effectively disabling the OS's ability to handle subsequent crashes. It is not intended to be called from an ISR, as it is a management function for the scheduler's internal state.

Uso:

- This function is strictly used within the OS kernel's fault management logic. You will typically see it called inside the scheduler's fault handling routine, specifically after a task has been successfully reset or removed following a crash. It acts as a 'gatekeeper' reset; by clearing the flag, you signal to the OS that the previous error state has been resolved and

the system is ready to monitor for new faults again. It is usually paired with logic that performs task cleanup or stack restoration.

Esempio:

```c
/* Example: Resetting the safeguard after a task recovery */
void handle_task_recovery(void) {
    /* Perform cleanup of the crashed task resources */
    ca_memory_free_all_owned_by(g_current_task);

    /* Reset the safeguard flag so the system can monitor for future faults */
    ca_safeguard_reset();

    /* Resume the system or restart the task */
    ca_os_resume_handle(g_current_task);
}
```

Q&A — fine-tuning review *2/2 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_safeguard_reset() and what is its purpose in a caOS application?
A: You should call ca_safeguard_reset() to reset a protection flag that prevents a task from getting stuck in an infinite loop or recursion during task restart. It is used in controlled contexts where you are certain no other task is using the safeguard flag. This function is typically used in scheduler or safeguard implementation files like safeguard.c and scheduler.c. It is important to note that this function is blocking, not thread-safe, and must not be called from an ISR.

Code example conf:0.85 **Q:** Can you provide a minimal C code example using ca_safeguard_reset() with caOS types?
A: void example_task(void) { // Reset the safeguard flag before restarting a task ca_safeguard_reset(); // Task logic here while(1) { // Task loop } }

`ca_sai_errors`

Header: `caos/drivers/inc/sai.h:70`
Source: `caos/drivers/src/sai.c:52:56`
GUARD: `EN_SAI`

```c
u32 ca_sai_errors(void); /* CA_SAI_ERR_* */
```

Retrieves the current accumulated error status of the SAI (Serial Audio Interface) peripheral.

Ritorna:
A 32-bit unsigned integer representing a bitmask of error flags (CA_SAI_ERR_*). A return value of 0 indicates no errors have been detected.

Note:

- This function is non-blocking and thread-safe, as it simply reads an internal status variable. It is safe to call from both tasks and ISRs. The function internally triggers a hardware error check if the SAI interface is active (s_sai_fs != 0), meaning it performs a read-modify-write operation on the error register. A common mistake is to ignore the return value or fail to mask out expected transient errors (like underruns during stream stops) when evaluating system health.

Uso:

- This function is typically used in the main monitoring loop or a diagnostic task to verify the health of the audio stream. It is often paired with ca_sai_stream_start to begin operations and ca_sai_stop to end them. Developers use it to decide whether to restart the audio peripheral or report a system failure to the user via CLI or logs. It is frequently combined with bitwise operations to ignore specific, non-critical errors like underruns (CA_SAI_ERR_UDR) that occur naturally during stream termination.

Esempio:

```c
/* Check SAI health and handle errors */
u32 current_errors = ca_sai_errors();

/* Ignore underrun errors which are common during stop sequences */
if ((current_errors & ~CA_SAI_ERR_UDR) != 0U) {
    ca_printf("Critical SAI error detected: 0x%08X\n", current_errors);
    /* Perform recovery: stop the stream and re-initialize */
    ca_sai_stop();
    /* Call function to re-initialize SAI hardware */
    /* ca_sai_init(...); */
} else {
    ca_printf("SAI status OK\n");
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_sai_errors and what is its primary purpose?
A: You should call ca_sai_errors in your main monitoring loop or a diagnostic task to verify the health of the SAI audio stream. It is used to decide whether to restart the audio peripheral or report a system failure to the user. It is typically paired with ca_sai_stream_start and ca_sai_stop.

Code example conf:0.95 **Q:** How do I check for SAI errors in my code while ignoring transient underruns?
A: You can check the status using a bitwise mask to filter out non-critical errors like CA_SAI_ERR_UDR. Example: u32 err = ca_sai_errors(); if (err & ~CA_SAI_ERR_UDR) { /* Handle critical error */ }

Return conf:1.00 **Q:** What does the return value of ca_sai_errors represent and what happens if it returns 0?
A: The function returns a 32-bit unsigned integer (u32) representing a bitmask of error flags (CA_SAI_ERR_*). A return value of 0 indicates that no errors have been detected. If the SAI interface is active, the function performs a read-modify-write operation on the error register to check for hardware issues.

`ca_sai_fs`

Header: `caos/drivers/inc/sai.h:68`
Source: `caos/drivers/src/sai.c:47:50`
GUARD: `EN_SAI`

```c
u32 ca_sai_fs(void); /* actual sample rate, 0 = not initialized */
```

Returns the actual configured sample rate of the SAI (Serial Audio Interface) peripheral.

Ritorna:
Returns a u32 representing the sample rate in Hz. Returns 0 if the SAI peripheral has not been initialized or is currently in an error state.

Note:

- This function is a non-blocking getter that provides the current hardware state. It is thread-safe as it simply reads a static variable updated during the initialization phase. It is safe to call from an ISR, although it is typically used in CLI or monitoring tasks to verify hardware configuration. A common beginner mistake is to assume the returned value is always the requested frequency; always check for 0 to ensure the hardware initialization was successful before attempting to process audio data.

Uso:

- This function is used to verify the operational status of the audio hardware. It is typically called after the system initialization sequence (where ca_sai_init would have been invoked) to confirm that the SAI peripheral is running at the expected frequency. It is often paired with ca_sai_errors to provide a complete diagnostic report in a CLI interface or system health monitor.

Esempio:

```c
/* Check if the SAI peripheral is initialized and report its status */
void monitor_audio_status(void) {
    u32 current_fs = ca_sai_fs();

    if (current_fs == 0U) {
        ca_printf("SAI is not initialized or in error state!\n");
    } else {
        /* Retrieve hardware errors using the relevant ca_sai_errors function */
        u32 errors = ca_sai_errors();
        ca_printf("SAI running at %u Hz with error code: 0x%02X\n", current_fs, errors);
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_sai_fs and what is its primary purpose?
A: You should call ca_sai_fs after the system initialization sequence, specifically after ca_sai_init has been invoked. Its primary purpose is to verify the operational status of the SAI peripheral and confirm that the hardware is running at the expected frequency, often as part of a system health monitor or CLI diagnostic report.

Code example conf:1.00 **Q:** How do I check the SAI sample rate in my application code?
A: You can retrieve the sample rate using a u32 variable and verify it against zero to ensure initialization was successful. Example: u32 current_fs = ca_sai_fs(); if (current_fs == 0) { /* Handle initialization error */ }

Return conf:1.00 **Q:** What do the different return values of ca_sai_fs indicate?
A: The function returns a u32 representing the sample rate in Hz. A return value of 0 indicates that the SAI peripheral has not been initialized or is currently in an error state. Developers should not assume the returned value matches the requested frequency without checking for 0 first.

`ca_sai_hw_dma_bind`

Header: `caos/drivers/inc/ca_glue_hw.h:395`
GUARD: `EN_CA_GLUE_HW`

```c
extern bool ca_sai_hw_dma_bind(i16 *tx_flat, u16 *tx_req, i16 *rx_flat, u16 *rx_req); /* DMA channel (flat, -1 = any) + request, A (TX) and B (RX) */
```

Retrieves the hardware-specific DMA stream and request mapping for the SAI peripheral.

Parametri:

Nome	Tipo	Descrizione
`tx_flat`	`i16 *`	Pointer to store the TX DMA stream/channel index. Set to -1 if any channel is acceptable.
`tx_req`	`u16 *`	Pointer to store the TX DMA request mapping ID.
`rx_flat`	`i16 *`	Pointer to store the RX DMA stream/channel index. Set to -1 if any channel is acceptable.
`rx_req`	`u16 *`	Pointer to store the RX DMA request mapping ID.

Ritorna:
Returns true if the hardware supports DMA for SAI and the mapping was successfully populated. Returns false if DMA is not supported on the current hardware or if the configuration is unavailable.

Note:

- This function is non-blocking and thread-safe as it only performs memory writes based on static hardware definitions. It is safe to call from an ISR, though it is typically used during peripheral initialization. The most common mistake is failing to check the return value; if it returns false, the subsequent DMA channel allocation functions will fail, potentially leaving the system in an inconsistent state. Always ensure the pointers passed are valid memory addresses to avoid hard faults.

Uso:

- This function is used during the SAI driver initialization sequence to determine which DMA resources are physically wired to the SAI peripheral. It is typically called before ca_dma_free_channel or ca_dma_free_channel_index to reserve the specific hardware streams required for audio data transfer. By abstracting the hardware-specific DMA mapping, it allows the higher-level SAI driver to remain portable across different STM32 families.

Esempio:

```
i16 tx_flat, rx_flat;
u16 tx_req, rx_req;

/* Retrieve hardware-specific DMA mapping for SAI */
if (ca_sai_hw_dma_bind(&tx_flat, &tx_req, &rx_flat, &rx_req)) {
    /* Attempt to reserve the specific DMA channels identified by the bind function */
    /* ca_dma_free_channel_index is used here because the hardware has fixed streams */
    if (ca_dma_free_channel_index(tx_flat, &s_tx_ch) >= 0 &&
        ca_dma_free_channel_index(rx_flat, &s_rx_ch) >= 0) {
        /* Proceed to configure SAI DMA requests */
        ca_dma_set_request(s_tx_ch, tx_req);
        ca_dma_set_request(s_rx_ch, rx_req);
    } else {
        /* Handle error: DMA channels could not be acquired */
    }
} else {
    /* Handle error: DMA not supported or binding failed */
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_sai_hw_dma_bind in my application?
A: You should call this function during the SAI driver initialization sequence, specifically before attempting to reserve DMA resources with functions like ca_dma_free_channel. It is used to determine the physically wired DMA streams and request mappings for the SAI peripheral, ensuring portability across different STM32 families.

Code example conf:1.00 **Q:** How do I correctly invoke ca_sai_hw_dma_bind using caOS types?
A: You must pass pointers to variables that will store the mapping results. Example: i16 tx_flat = -1, rx_flat = -1; u16 tx_req, rx_req; if (ca_sai_hw_dma_bind(&tx_flat, &tx_req, &rx_flat, &rx_req)) { /* Proceed with DMA setup */ }

Parameters conf:1.00 **Q:** What is the significance of setting tx_flat or rx_flat to -1?
A: Setting tx_flat or rx_flat to -1 indicates that any available DMA stream or channel is acceptable for the operation. The function will then populate these pointers with the hardware-specific mapping values if the operation is successful.

Return conf:1.00 **Q:** What does a false return value from ca_sai_hw_dma_bind signify?
A: A false return indicates that either the current hardware does not support DMA for the SAI peripheral or the required configuration is unavailable. Failing to check this return value is a common mistake that can lead to system inconsistency if subsequent DMA allocation functions are called.

HW differences conf:0.90 **Q:** Are there differences in how ca_sai_hw_dma_bind behaves across STM32 families?
A: Yes. For example, in the F1xx family, the implementation explicitly returns false, indicating that DMA is not supported for SAI on that hardware. Other families like F4xx, F7xx, H5xx, H7xx, and U5xx have different internal implementations for SAI block register access (e.g., SAI1_Block_A/B) and error handling, though the DMA binding logic is abstracted by this function.

`ca_sai_hw_dma_count_div`

Header: `caos/drivers/inc/ca_glue_hw.h:397`
GUARD: `EN_CA_GLUE_HW`

```
1 extern u32 ca_sai_hw_dma_count_div(void); /* ca_dma_get_remaining()
     units per word: 4 on GPDMA, 1 elsewhere */
```

Returns the divisor factor required to normalize DMA remaining count to word units based on the hardware architecture.

Ritorna:
Returns 4 for GPDMA-based systems (H5/U5) where DMA counts bytes, and 1 for older architectures (F1/F4/F7) where DMA counts data elements (words).

Note:

- This function is a hardware abstraction layer (HAL) helper. It is non-blocking, thread-safe, and safe to call from an ISR. It is mandatory to use this when calculating stream positions or buffer offsets in DMA-based SAI transfers to ensure the logic remains portable across different STM32 families. The most common mistake is hardcoding a division by 4 or 1, which will cause buffer pointer arithmetic errors when porting code between different STM32 series.

Uso:

- This function is used exclusively within the SAI driver's streaming logic. It is typically paired with ca_dma_get_remaining() to convert the raw DMA counter into a meaningful number of audio words. By dividing the raw DMA count by the result of this function,

the application can accurately track how many samples have been processed, allowing for correct synchronization between the audio buffer and the DMA hardware.

Esempio:

```c
/* Calculate how many audio words are currently remaining in the DMA buffer */
extern u32 s_rx_ch; /* Handle for the active DMA channel */

void update_audio_stream_status(void)
{
    /* Get raw remaining count from DMA */
    u32 raw_remaining = ca_dma_get_remaining(s_rx_ch);

    /* Normalize the count based on hardware architecture */
    u32 divisor = ca_sai_hw_dma_count_div();
    u32 words_remaining = raw_remaining / divisor;

    /* Use words_remaining to update UI or sync audio processing */
    ca_printf("Remaining words in buffer: %lu\n", words_remaining);
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** Why is it necessary to use ca_sai_hw_dma_count_div() when calculating SAI buffer offsets?
A: It is mandatory to use this function to normalize the DMA remaining count to word units. Because different STM32 families count DMA transfers differently (bytes vs. data elements), hardcoding a divisor leads to pointer arithmetic errors when porting code between families.

Code example conf:1.00 **Q:** How do I calculate the number of remaining audio words in a DMA transfer using caOS types?
A: You should divide the raw DMA count by the result of the helper function: u32 remaining_words = ca_dma_get_remaining() / ca_sai_hw_dma_count_div();

Return conf:1.00 **Q:** What do the return values 1 and 4 signify for ca_sai_hw_dma_count_div()?
A: A return value of 4 indicates a GPDMA-based system (like H5 or U5) where the DMA counts bytes. A return value of 1 indicates older architectures (like F1, F4, or F7) where the DMA counts data elements (words).

HW differences conf:1.00 **Q:** How does the SAI DMA behavior differ between the F4/F7 series and the H5/U5 series?
A: The difference lies in the DMA architecture: F4/F7 series architectures count data elements (words) and return 1, whereas GPDMA-based systems like H5 and U5 count bytes and return 4.

`ca_sai_hw_dr`

Header: `caos/drivers/inc/ca_glue_hw.h:396`
GUARD: `EN_CA_GLUE_HW`

```c
extern volatile void *ca_sai_hw_dr(bool rx); /* data register of B (rx)
    or A */
```

Returns the memory address of the SAI hardware data register for the specified direction.

Parametri:

Nome	Tipo	Descrizione
`rx`	`bool`	If true, returns the address of the receiver (Block B) data register. If false, returns the address of the transmitter (Block A) data register.

Ritorna:
A volatile pointer to the hardware data register (DR) on supported platforms, or NULL if the hardware abstraction layer does not support direct register access for the current target.

Note:

- This function is non-blocking and thread-safe as it simply returns a constant memory address mapped to the peripheral. It is safe to call from an ISR. The primary risk is passing this address to a DMA controller on a platform where the SAI peripheral is not supported (returning NULL), which would cause a null pointer dereference or invalid DMA configuration. Always ensure the SAI peripheral is initialized before attempting to use its data register address.

Uso:

- This function is used to provide the destination or source address to DMA transfer functions, such as ca_dma_copy_target. It is typically used in the sequence: stop the SAI peripheral, clear any pending errors, obtain the data register address, and then configure the DMA channel to stream data directly to or from that register. It is essential for high-speed audio processing where CPU-based data copying would be too slow.

Esempio:

```
/* Example: Configuring DMA to stream audio data to the SAI transmitter */
void *sai_dr_addr;
size_t words = 128;

/* Stop SAI to ensure a clean state before DMA setup */
ca_sai_stop();

/* Get the address of the transmitter (Block A) data register */
sai_dr_addr = (void *)ca_sai_hw_dr(false);

if (sai_dr_addr != NULL) {
    /* Configure DMA to move data from memory to the SAI hardware register */
    if (!ca_dma_copy_target(s_tx_ch, (void *)audio_buffer, sai_dr_addr, words,
    CA_DMA_DIR_M2P, CA_DMA_WIDTH_32)) {
        /* Handle DMA configuration failure */
    }
} else {
    /* Hardware does not support direct SAI register access */
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** Why should I use ca_sai_hw_dr in my audio application?
A: It is used to obtain the memory address of the SAI data register for DMA transfers. This is essential for high-speed audio processing where CPU-based data copying is too slow. It is typically used by stopping the SAI peripheral, clearing errors, obtaining the address, and configuring the DMA channel to stream data directly to or from that register.

Code example conf:1.00 **Q:** How do I use ca_sai_hw_dr to get the receiver data register address?
A: You can call the function with the rx parameter set to true. Example: volatile void *dr_addr = ca_sai_hw_dr(true); if (dr_addr != NULL) { /* proceed with DMA configuration */ }

Parameters conf:1.00 **Q:** What does the bool rx parameter control in ca_sai_hw_dr?
A: The rx parameter determines which SAI block's data register address is returned. If true, it returns the address for the receiver (Block B). If false, it returns the address for the transmitter (Block A).

Return conf:1.00 **Q:** What does it mean if ca_sai_hw_dr returns NULL?
A: A return value of NULL indicates that the hardware abstraction layer for the current target platform does not support direct register access for the SAI peripheral. You must check for NULL before using the pointer to avoid a null pointer dereference or invalid DMA configuration.

HW differences conf:1.00 **Q:** Are there differences in how ca_sai_hw_dr behaves across STM32 families?
A: Yes. On the f1xx family, the implementation explicitly returns NULL, indicating that direct register access via this function is not supported. On other supported families, it returns the memory address of the SAI hardware data register.

`ca_sai_hw_errors`

Header: `caos/drivers/inc/ca_glue_hw.h:393`
GUARD: `EN_CA_GLUE_HW`

```
extern u32 ca_sai_hw_errors(void); /* CA_SAI_ERR_* since the last call,
    cleared */
```

Retrieves and clears hardware-level Serial Audio Interface (SAI) error flags.

Ritorna:
A bitmask of error flags (u32). Returns 0 if no errors occurred. Specific flags include CA_SAI_ERR_UDR (underrun), CA_SAI_ERR_OVR (overrun), CA_SAI_ERR_AFS (audio frame synchronization error), and CA_SAI_ERR_LFS (late frame synchronization error).

Note:

- This function is non-blocking and performs direct register access to the SAI hardware. It is thread-safe in the sense that it reads and clears hardware registers atomically, but it should be called periodically to ensure errors are captured before they are overwritten by subsequent events. A common mistake is to ignore the return value; since the function clears the hardware error flags upon reading, calling it without storing the result means the error information is lost forever. It is safe to call from an ISR, but typically it is called from a background task or a monitoring thread.

Uso:

- This function is used to monitor the health of the SAI peripheral during audio streaming. It is typically paired with ca_sai_xfer or other data transfer functions. The standard workflow involves calling this function periodically (e.g., inside a system tick or a dedicated monitoring task) to check if the hardware has encountered synchronization or buffer issues. If a non-zero value is returned, the application should log the error or trigger a recovery sequence, such as resetting the SAI peripheral.

Esempio:

```
/* Periodically check for SAI hardware errors */
void monitor_audio_task(void) {
    u32 errors = ca_sai_hw_errors();
    if (errors != 0U) {
        /* Log the error bits to the system console */
        ca_printf("SAI Hardware Error Detected: 0x%08X\n", errors);

        /* If an overrun or underrun occurred, consider resetting the stream */
        if (errors & (CA_SAI_ERR_UDR | CA_SAI_ERR_OVR)) {
            /* Call function to stop the SAI hardware */
            ca_sai_stop();
```

```
            /* Re-initialize or restart the audio stream */
        }
    }
    /* Wait for the next monitoring interval */
    ca_os_delay_ms(100);
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When and why should I call ca_sai_hw_errors in my caOS application?
A: You should call ca_sai_hw_errors periodically, such as within a system tick or a dedicated monitoring task, to ensure hardware-level SAI errors are captured before they are overwritten. It is used to monitor the health of the SAI peripheral during audio streaming; if a non-zero value is returned, you should log the error or trigger a recovery sequence like resetting the SAI peripheral.

Code example conf:1.00 **Q:** How do I implement a simple check for SAI errors in my code?
A: You must store the return value of the function, as calling it clears the hardware flags. Example: u32 err = ca_sai_hw_errors(); if (err != 0U) { /* Handle error */ }

Return conf:1.00 **Q:** What do the bits in the return value of ca_sai_hw_errors represent?
A: The return value is a u32 bitmask. Specific flags include CA_SAI_ERR_UDR (underrun), CA_SAI_ERR_OVR (overrun), CA_SAI_ERR_AFS (audio frame synchronization error), and CA_SAI_ERR_LFS (late frame synchronization error). A return value of 0 indicates no errors occurred since the last call.

HW differences conf:0.90 **Q:** Are there differences in how SAI errors are handled across different STM32 families in caOS?
A: Yes. For example, the F1xx implementation returns 0U and does not perform hardware operations. The F4xx implementation clears the CLRFR register with 0x7FU and checks SAI_xSR_OVRUDR. The F7xx implementation includes additional checks for SAI_xSR_AFSDET, demonstrating that error detection capabilities vary by hardware family.

`ca_sai_hw_get`

Header: `caos/drivers/inc/ca_glue_hw.h:392`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_sai_hw_get(u32 *w); /* one word from B's FIFO, false when
    empty */
```

Reads a single 32-bit data word from the SAI Block B hardware FIFO.

Parametri:

Nome	Tipo	Descrizione
`w`	`u32 *`	Pointer to a 32-bit variable where the retrieved data word will be stored. Must be a valid memory address.

Ritorna:
Returns true if a word was successfully read from the FIFO, or false if the FIFO was empty.

Note:

- This function is non-blocking and performs a direct hardware register read. It is thread-safe in the sense that it accesses hardware registers, but it does not implement software-level locking; if multiple tasks access the SAI hardware simultaneously, you must protect this call with a mutex or scheduler lock. It is safe to call from an ISR, provided the SAI

hardware is already initialized and active. A common beginner mistake is ignoring the return value; if you call this when the FIFO is empty, the pointer 'w' will not be updated, and your application logic may process stale or uninitialized data.

Uso:

- This function is typically used in a polling loop to drain the SAI receive FIFO during manual (non-DMA) data transfers. It is usually paired with 'ca_sai_hw_put' to handle full-duplex communication. The standard sequence is: initialize the SAI peripheral, enter a loop that checks for data availability using this function, process the received word, and finally call 'ca_sai_hw_stop' once the transfer is complete to flush the hardware buffers.

Esempio:

```
u32 rx_word;
/* Attempt to read one word from the SAI hardware FIFO */
if (ca_sai_hw_get(&rx_word)) {
    /* Successfully retrieved data, process it */
    ca_printf("Received: %08X\n", rx_word);
} else {
    /* FIFO was empty, perform other tasks or yield to avoid busy-waiting */
    ca_os_yield();
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_sai_hw_get function in my caOS application?
A: You should use ca_sai_hw_get in a polling loop to drain the SAI Block B receive FIFO during manual, non-DMA data transfers. It is typically used after initializing the SAI peripheral and is often paired with ca_sai_hw_put for full-duplex communication.

Code example conf:1.00 **Q:** Provide a minimal C example of how to read data from the SAI FIFO using ca_sai_hw_get.
A: u32 data; if (ca_sai_hw_get(&data)) { /* process data */ }

Parameters conf:1.00 **Q:** What are the requirements for the parameter passed to ca_sai_hw_get?
A: The parameter 'w' must be a pointer to a 32-bit variable (u32 *). It must be a valid memory address where the function can store the retrieved data word.

Return conf:1.00 **Q:** What does the return value of ca_sai_hw_get indicate, and what happens if it returns false?
A: The function returns true if a word was successfully read from the FIFO. It returns false if the FIFO is empty. If it returns false, the pointer 'w' is not updated, and you should avoid processing the data at that memory location to prevent using stale or uninitialized values.

HW differences conf:0.90 **Q:** Are there differences in how ca_sai_hw_get behaves across different STM32 families?
A: Yes. For the F1xx family, the implementation of ca_sai_hw_get is a stub that always returns false and does not perform any hardware register access, whereas other families like F4xx, F7xx, H5xx, H7xx, and U5xx provide functional implementations that interact with the SAI hardware registers.

`ca_sai_hw_init`

Header: `caos/drivers/inc/ca_glue_hw.h:391`
GUARD: `EN_CA_GLUE_HW`

```
extern u32 ca_sai_hw_init(u32 fs_hz, u32 bits, ca_sai_proto_t proto); /*
    clock, pins, both blocks set up and off (SPDIF: A only); actual fs,
   0 = no SAI */
```

Initializes the Serial Audio Interface (SAI) hardware peripheral for digital audio streaming.

Parametri:

Nome	Tipo	Descrizione
`fs_hz`	`u32`	Target sampling frequency in Hertz (e.g., 44100, 48000). Use 0 to disable the SAI peripheral.
`bits`	`u32`	Data word length in bits. Valid values are 16, 24, or 32.
`proto`	`ca_sai_proto_t`	Audio protocol type (e.g., CA_SAI_MSB, CA_SAI_SPDIF). Determines clocking and data alignment.

Ritorna:
Returns the actual achieved sampling frequency in Hz as a u32. Returns 0 if the initialization failed (e.g., unsupported clock configuration or invalid parameters).

Note:

- This function is not thread-safe and must be called during system initialization or when the audio stream is stopped. It is not callable from an ISR because it performs complex clock calculations and hardware register configuration. The function handles clock tree setup, GPIO pin configuration, and block synchronization (Master TX/Slave RX). A common mistake is calling this function while an active DMA transfer is running, which will cause hardware instability or corrupted audio; always ensure the SAI is stopped first.

Uso:

- This function is the primary entry point for configuring the SAI hardware. It is typically used in a sequence: first, call ca_sai_stop() to ensure the hardware is idle, then call ca_sai_hw_init() to apply new settings, and finally use ca_sai_stream_start() to begin data transmission. It is often paired with ca_sai_xfer() or ca_sai_xfer_dma() for actual data movement.

Esempio:

```c
/* Example: Reconfiguring SAI for 48kHz 24-bit audio */
#include "caos.h"

void setup_audio_stream(void) {
    /* Ensure hardware is stopped before re-init */
    ca_sai_stop();

    /* Initialize SAI hardware for 48kHz, 24-bit, MSB protocol */
    u32 actual_fs = ca_sai_hw_init(48000, 24, CA_SAI_MSB);

    if (actual_fs == 0) {
        /* Handle error: hardware init failed */
        return;
    }

    /* Start the stream after successful initialization */
    ca_sai_stream_start();
}
```

Q&A — fine-tuning review *7/7 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** What is the recommended sequence for initializing the SAI peripheral in caOS?
A: The recommended sequence is to first call ca_sai_stop() to ensure the hardware is idle, then call ca_sai_hw_init() to apply the desired configuration, and finally call ca_sai_stream_start() to begin data transmission.

Code example conf:1.00 **Q:** Provide a minimal example of how to initialize the SAI for 48kHz, 24-bit audio.
A: u32 actual_fs = ca_sai_hw_init(48000U, 24U, CA_SAI_MSB); if (actual_fs == 0U) { /* Handle error */ }

Parameters conf:1.00 **Q:** What are the valid values for the 'bits' parameter in ca_sai_hw_init?
A: The valid values for the 'bits' parameter are 16, 24, or 32, representing the data word length in bits.

Return conf:1.00 **Q:** What does the return value of ca_sai_hw_init indicate?
A: The function returns the actual achieved sampling frequency in Hz as a u32. If the initialization fails due to unsupported clock configurations or invalid parameters, it returns 0.

Error/Edge case conf:1.00 **Q:** What is a common mistake when calling ca_sai_hw_init?
A: A common mistake is calling this function while an active DMA transfer is running, which can cause hardware instability or corrupted audio. Always ensure the SAI is stopped first.

Behavior conf:1.00 **Q:** Is ca_sai_hw_init thread-safe or callable from an ISR?
A: No, this function is not thread-safe and must be called during system initialization or when the audio stream is stopped. It is not callable from an ISR because it performs complex clock calculations and hardware register configuration.

HW differences conf:1.00 **Q:** How does the hardware implementation of ca_sai_hw_init differ between the F4xx and H5xx families?
A: The F4xx implementation relies on sai_plli2s() for clock configuration and uses RCC->APB2ENR for enabling SAI1. The H5xx implementation configures the clock source via RCC->CCIPR5 (SAI1SEL) and calculates the sampling frequency using sai_ker_hz() and a loop to determine the best MCKDIV value.

`ca_sai_hw_put`

Header: `caos/drivers/inc/ca_glue_hw.h:391`
GUARD: `EN_CA_GLUE_HW`

```
1 extern bool ca_sai_hw_put(u32 w); /* one word into A's FIFO, false when
    full */
```

Writes a single 32-bit data word into the SAI hardware transmit FIFO.

Parametri:

Nome	Tipo	Descrizione
`w`	`u32`	The 32-bit data word to be transmitted. The value is placed directly into the SAI Data Register (DR).

Ritorna:
Returns true if the word was successfully written to the FIFO. Returns false if the FIFO is full, indicating the hardware cannot accept more data at this moment.

Note:

- This function is non-blocking; it checks the hardware status register and returns immediately. It is thread-safe in the sense that it interacts with hardware registers, but it does not implement software locking, so concurrent access from multiple tasks to the same SAI block must be managed by the caller. It is safe to call from an ISR if the hardware is already initialized. The most common beginner mistake is ignoring the return value: if you call this in a loop without checking for 'false', you will lose data because the FIFO will overflow silently.

Uso:

- This function is used to manually feed the SAI peripheral when not using DMA. It is typically used in a loop to pre-fill the FIFO before calling ca_sai_hw_start, and then periodically called within a processing loop to keep the FIFO fed while data is being consumed. It is often paired with ca_sai_hw_start to initiate the transmission and ca_sai_hw_stop to halt the peripheral.

Esempio:

```c
/* Example: Manually filling the SAI FIFO before starting transmission */
#include "caos.h"

void send_audio_data(u32 *buffer, u32 length) {
    u32 sent = 0;

    /* Pre-fill the FIFO as much as possible before starting the clock */
    while (sent < length) {
        if (ca_sai_hw_put(buffer[sent])) {
            sent++;
        } else {
            /* FIFO is full, wait a bit or yield to let hardware drain */
            ca_os_yield();
        }
    }

    /* Start the SAI hardware to begin shifting out the data */
    ca_sai_hw_start(false);

    /* ... continue monitoring and feeding the FIFO ... */
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_sai_hw_put in my caOS application?
A: You should use ca_sai_hw_put to manually feed the SAI peripheral when you are not using DMA. It is typically used in a loop to pre-fill the FIFO before calling ca_sai_hw_start, and then periodically called within a processing loop to keep the FIFO fed while data is being consumed.

Code example conf:0.90 **Q:** How do I write a loop to send data using ca_sai_hw_put?
A: You must check the return value to avoid silent data loss. Example: while(data_remaining) { if(ca_sai_hw_put(buffer[i])) { i++; data_remaining–; } }

Parameters conf:1.00 **Q:** What does the u32 w parameter represent in ca_sai_hw_put?
A: The u32 w parameter is the 32-bit data word intended for transmission. The value is placed directly into the SAI Data Register (DR).

Return conf:1.00 **Q:** What do the boolean return values of ca_sai_hw_put indicate?
A: The function returns true if the word was successfully written to the FIFO. It returns false if the FIFO is full, meaning the hardware cannot accept more data at that moment.

HW differences conf:0.95 **Q:** Are there any known limitations or differences in ca_sai_hw_put across MCU families?
A: Yes. For the f1xx family, the implementation of ca_sai_hw_put is a stub that returns false and does not perform any operation, whereas other families like f4xx, f7xx, h5xx, h7xx, and u5xx provide functional implementations that interact with the SAI hardware registers.

ca_sai_hw_start

Header: caos/drivers/inc/ca_glue_hw.h:389
GUARD: EN_CA_GLUE_HW

```
extern void ca_sai_hw_start(bool dma); /* RX FIFO flushed, B then A on (
    DMAEN when dma) */
```

Activates the SAI hardware blocks to begin audio data transmission and reception.

Parametri:

Nome	Tipo	Descrizione
dma	bool	If true, enables DMA mode for data transfer; if false, manual polling (CPU-driven) is used.

Ritorna:
void

Note:

- This function is not thread-safe and must be called only after the SAI hardware has been fully configured via the initialization sequence. It performs a critical hardware operation: it flushes the RX FIFO and enables the SAI blocks in a specific order (Slave B then Master A) to ensure synchronization. It is not intended to be called from an ISR, as it performs register-level hardware orchestration that expects a stable system state. A common mistake is calling this before the TX FIFO has been pre-filled with initial data, which can lead to underrun errors or silent failure to start the audio stream.

Uso:

- This function is the final step in the SAI startup sequence. Typically, you first initialize the SAI hardware, then pre-fill the TX FIFO with initial audio samples to prevent immediate underrun, and finally call this function to trigger the hardware clock and data flow. It is usually paired with a loop that monitors the FIFO status or uses DMA interrupts to keep the buffers fed while the audio stream is active.

Esempio:

```
/* Assuming SAI hardware is already initialized */
/* Pre-fill the TX FIFO to avoid immediate underrun */
/* ca_sai_hw_put is a hypothetical function to write to the SAI data register */

// Fill initial buffer
for (int i = 0; i < 16; i++) {
    // Write dummy data to TX FIFO
    // ca_sai_hw_put(0x00000000);
}

// Start the hardware in polling mode (non-DMA)
ca_sai_hw_start(false);

// Now the SAI block is active and driving the clock
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** At what point in the SAI initialization sequence should I call ca_sai_hw_start?
A: It should be called as the final step of the startup sequence. You must first initialize the SAI hardware and pre-fill the TX FIFO with initial audio samples to prevent underrun errors before calling this function to trigger the clock and data flow.

Code example conf:1.00 **Q:** Provide a minimal C example of how to invoke ca_sai_hw_start in caOS.

A: void start_audio(void) { /* Assume hardware is configured and TX FIFO is pre-filled */ bool use_dma = true; ca_sai_hw_start(use_dma); }

Parameters conf:1.00 **Q:** What does the 'dma' parameter control in ca_sai_hw_start?

A: The 'dma' parameter is a boolean that determines the data transfer method. If set to true, it enables DMA mode for data transfer; if set to false, the system uses manual CPU-driven polling.

Error/Edge case conf:1.00 **Q:** Why might I experience silent failures or underrun errors when starting the SAI stream?

A: A common mistake is calling ca_sai_hw_start before the TX FIFO has been pre-filled with initial audio data. This leads to immediate underrun errors or a failure to start the audio stream correctly.

HW differences conf:0.90 **Q:** Are there differences in how SAI hardware is enabled across the supported MCU families?

A: Yes. While the high-level logic is similar, families like F4xx and F7xx use RCC->APB2ENR to enable SAI1, whereas H5xx, H7xx, and U5xx configure clock sources via registers like RCC->CCIPR5, RCC->D2CCIP1R, or RCC->CCIPR2 respectively before initializing the blocks.

`ca_sai_hw_stop`

Header: `caos/drivers/inc/ca_glue_hw.h:390`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_sai_hw_stop(void); /* A and B off at the end of the frame, FIFOs flushed */
```

Stops the SAI hardware blocks and flushes their internal FIFOs.

Ritorna:
void

Note:

- This function is non-blocking in the sense that it does not wait indefinitely; it uses a spin-loop (SAI_STOP_SPIN) to wait for the hardware to finish the current audio frame before disabling the blocks. It is not thread-safe, meaning you must ensure no other task or interrupt is attempting to configure the SAI registers simultaneously. It is generally safe to call from an ISR, but be aware that the spin-loop consumes CPU cycles, which could delay other high-priority tasks. The most common beginner mistake is calling this function while a DMA transfer is still active without first ensuring the DMA channel is properly stopped or aborted, which can lead to unpredictable hardware states.

Uso:

- This function is used to cleanly shut down the Serial Audio Interface (SAI) peripheral, typically before reconfiguring the audio parameters or when stopping audio playback. It is usually paired with ca_sai_hw_init for setup and ca_sai_hw_start to begin the data flow. In a typical sequence, you call ca_sai_hw_stop to ensure the hardware is in a known idle state, clear any pending error flags using ca_sai_errors, and then proceed to fill the FIFOs or re-initialize the peripheral for the next audio session.

Esempio:

```
/* Stop the SAI hardware to ensure a clean state before starting a new stream */
ca_sai_hw_stop();

```

```
/* Clear any potential errors from the previous run */
(void)ca_sai_hw_errors();

/* Prepare the hardware for a new audio session */
/* ca_sai_hw_init(48000, 16, true); */

/* Fill the FIFO with initial data before starting the hardware */
/* ca_sai_hw_put(sample_data); */

/* Start the SAI hardware without DMA */
ca_sai_hw_start(false);
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_sai_hw_stop in my caOS application?
A: You should call ca_sai_hw_stop to cleanly shut down the SAI peripheral, typically before reconfiguring audio parameters or stopping playback. It is essential to ensure any active DMA transfers are stopped or aborted first to avoid unpredictable hardware states. After calling this function, you can clear error flags using ca_sai_errors before re-initializing the hardware.

Code example conf:0.90 **Q:** How do I properly stop the SAI hardware in a caOS project?
A: To stop the SAI hardware, ensure your DMA is disabled, then invoke the stop function. Example:
/* Ensure DMA is stopped first */ /* ... DMA stop logic ... */ ca_sai_hw_stop(); /* Clear errors before next session */ ca_sai_errors();

HW differences conf:0.95 **Q:** Are there differences in how ca_sai_hw_stop is implemented across STM32 families?
A: Yes. While the function signature is consistent, the implementation varies by family. For the F1xx series, the function is currently a stub (empty). In other families like F4xx, F7xx, H5xx, H7xx, and U5xx, the hardware logic involves interacting with specific registers like SAI_xCR1 and SAI_xCR2 to manage block modes, synchronization, and FIFO flushing, which are not present in the F1xx implementation.

`ca_sai_init`

Header: `caos/drivers/inc/sai.h:67`
Source: `caos/drivers/src/sai.c:31:45`
GUARD: `EN_SAI`

```
u32 ca_sai_init(const ca_sai_config_t *cfg); /* actual sample rate, 0 =
    not possible */
```

Initializes the SAI (Serial Audio Interface) hardware peripheral with the specified sample rate, bit depth, and protocol.

Parametri:

Nome	Tipo	Descrizione
`cfg`	`const ca_sai_config_t *`	Pointer to a configuration structure containing fs_hz (4000-192000), bits (16, 24, or 32), and proto (CA_SAI_MSB or similar). Must not be NULL.

Ritorna:
Returns the actual configured sample rate in Hz as a u32. Returns 0 if the configuration is invalid, out of range, or if the hardware failed to initialize.

Note:

- This function is not thread-safe and should be called during the system initialization phase before starting audio processing tasks. It is a blocking call because it interacts directly with hardware registers to set up clocks and pins. Do not call this from an ISR. A common mistake is passing a NULL pointer or an unsupported sample rate, which will cause the function to return 0 immediately without configuring the hardware.

Uso:

- This function is the entry point for audio hardware setup. It is typically used in a sequence: first, define the configuration structure, then call ca_sai_init to apply settings, and finally use ca_sai_stream_start to begin data transfer. If the return value is 0, the application should halt or enter an error state, as audio processing cannot proceed without a valid hardware clock.

Esempio:

```
ca_sai_config_t my_audio_cfg = { .fs_hz = 48000, .bits = 24, .proto = CA_SAI_MSB
    };

// Initialize the SAI hardware
u32 actual_fs = ca_sai_init(&my_audio_cfg);

if (actual_fs == 0U) {
    // Handle initialization failure: log error or enter safe state
    ca_printf("SAI Init failed!\n");
} else {
    // Proceed to start the audio stream
    // ca_sai_stream_start();
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_sai_init in my caOS application?
A: You should call ca_sai_init during the system initialization phase, before starting any audio processing tasks. It is intended to be the entry point for audio hardware setup, followed by ca_sai_stream_start to begin data transfer.

Code example conf:1.00 **Q:** How do I initialize the SAI peripheral using caOS types?
A: You must define a ca_sai_config_t structure and pass its address to the function: ca_sai_config_t cfg = { .fs_hz = 48000, .bits = 16, .proto = CA_SAI_MSB }; u32 actual_rate = ca_sai_init(&cfg); if (actual_rate == 0) { /* Handle error */ }

Parameters conf:1.00 **Q:** What are the valid constraints for the ca_sai_config_t structure members?
A: The fs_hz member must be between 4000 and 192000 Hz. The bits member must be set to 16, 24, or 32. The proto member should be set to a valid protocol constant like CA_SAI_MSB. The pointer passed to the function must not be NULL.

Return conf:1.00 **Q:** What does the return value of ca_sai_init indicate?
A: The function returns the actual configured sample rate in Hz as a u32. If it returns 0, it indicates that the configuration was invalid, out of range, or that the hardware failed to initialize.

Error/Edge case conf:1.00 **Q:** What are common mistakes that cause ca_sai_init to fail?
A: Common mistakes include passing a NULL pointer to the function or providing an unsupported sample rate. Additionally, calling this function from an ISR will cause issues, as it is a blocking call that interacts directly with hardware registers.

---

ca_sai_stop

Header: caos/drivers/inc/sai.h:71
Source: caos/drivers/src/sai.c:87:94
GUARD: EN_SAI

```
void ca_sai_stop(void)
```

Stops the Serial Audio Interface (SAI) hardware and associated DMA transfers.

Ritorna:
void

Note:

- This function is a wrapper that safely halts the SAI peripheral and, if enabled, the DMA controller. It is not thread-safe and should not be called from an ISR, as it performs hardware-level operations that may involve blocking or synchronization. A common mistake is calling this function while the SAI is still actively processing data without waiting for the current buffer to finish, which can lead to incomplete data frames or hardware state inconsistencies. Ensure that any higher-level application logic has finished processing or flushing the audio buffers before invoking this stop command.

Uso:

- This function is typically used at the end of an audio processing task or when a measurement cycle is complete. It is usually paired with 'ca_sai_init' for setup and 'ca_sai_stream_start' for beginning the audio flow. In a typical sequence, you would stop the stream, perform final data calculations or checks, and then call 'ca_sai_stop' to put the hardware into a low-power or idle state to prevent unnecessary resource consumption.

Esempio:

```
/* Example: Stopping an audio measurement task */
void stop_audio_measurement(void) {
    /* 1. Stop the SAI hardware and DMA to prevent further data flow */
    ca_sai_stop();

    /* 2. Perform final cleanup or logging */
    ca_printf("Audio measurement stopped successfully.\n");

    /* 3. Optional: Free any memory buffers used for audio processing */
    /* ca_memory_free(audio_buffer_ptr); */
}
```

Q&A — fine-tuning review *2/2 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to call ca_sai_stop in my audio application?
A: You should call ca_sai_stop at the end of an audio processing task or when a measurement cycle is complete. It is intended to put the SAI hardware into a low-power or idle state to prevent unnecessary resource consumption. Before calling it, ensure your application logic has finished processing or flushing audio buffers, as calling it while data is actively processing can lead to incomplete frames or hardware state inconsistencies.

Code example conf:0.95 **Q:** How do I correctly shut down the SAI peripheral using caOS functions?
A: To safely stop the SAI hardware and its associated DMA transfers, invoke the function directly. Ensure you have already stopped the stream flow. Example:
void stop_audio_system(void) { /* Ensure stream is stopped before hardware halt */ ca_sai_stop(); }

ca_sai_stream_pos

Header: caos/drivers/inc/sai.h:76
Source: caos/drivers/src/sai.c:198:204
GUARD: EN_SAI

```
u32 ca_sai_stream_pos(void); /* next rx word the DMA writes, 0..words-1 */
```

Returns the current read position (index) of the active SAI DMA stream.

Ritorna:
Returns a u32 representing the current word index (0 to words-1) being processed by the DMA. Returns 0 if the stream is not active or if the DMA has not yet started processing.

Note:

- This function is non-blocking and thread-safe, as it performs a simple calculation based on the current DMA register state. It is safe to call from an ISR, as it does not perform any complex synchronization or blocking operations. The function relies on the internal state set by a previous call to ca_sai_stream_start (not listed in the catalog, but implied by the implementation). A common mistake is calling this function before initializing the SAI stream, which will result in a return value of 0, potentially leading to logic errors if the application assumes the stream is active.

Uso:

- This function is typically used in real-time audio or signal processing applications to monitor the progress of a circular DMA buffer. It is used to determine how much data has been processed or to synchronize application logic with the hardware's current position in the buffer. It is usually called periodically within a task loop or after a delay to check if the DMA has advanced, often paired with ca_os_sleep or ca_os_delay to pace the monitoring frequency.

Esempio:

```
/* Example: Monitoring SAI stream progress */
#include "ca_os.h"

void monitor_task(void *p) {
    u32 current_pos;
    u32 last_pos = 0;

    while (1) {
        /* Check the current position of the DMA stream */
        current_pos = ca_sai_stream_pos();

        /* If the position has moved, log or process the update */
        if (current_pos != last_pos) {
            /* Perform application logic based on new position */
            last_pos = current_pos;
        }

        /* Wait for 10ms before checking again to avoid CPU hogging */
        ca_os_delay_ms(10);
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_sai_stream_pos in my application?

A: You should use this function in real-time audio or signal processing applications to monitor the progress of a circular DMA buffer. It allows you to determine how much data has been processed or to synchronize your application logic with the hardware's current position in the buffer. It is typically called periodically within a task loop, often paired with ca_os_sleep or ca_os_delay to pace the monitoring frequency.

Code example conf:0.90 **Q:** How do I implement a simple check of the current SAI stream position using caOS types?

A: You can retrieve the current index and process it as follows: u32 current_pos = ca_sai_stream_pos(); if (current_pos > 0) { /* Process data at current_pos */ }

Return conf:1.00 **Q:** What does the return value of ca_sai_stream_pos represent, and why might it return 0?

A: The function returns a u32 representing the current word index (0 to words-1) being processed by the DMA. It returns 0 if the stream is not active, if the DMA has not yet started processing, or if the function is called before the SAI stream has been initialized.

`ca_sai_stream_start`

Header: `caos/drivers/inc/sai.h:75`
Source: `caos/drivers/src/sai.c:174:196`
GUARD: `EN_SAI`

```
bool ca_sai_stream_start(const i32 *tx, i32 *rx, u32 words); /* circular, both ways */
```

Starts a continuous, circular full-duplex SAI (Serial Audio Interface) data stream using DMA.

Parametri:

Nome	Tipo	Descrizione
`tx`	`const i32 *`	Pointer to the memory buffer containing audio data to be transmitted. Must be valid and aligned for DMA access.
`rx`	`i32 *`	Pointer to the memory buffer where received audio data will be stored. Must be valid and aligned for DMA access.
`words`	`u32`	The number of 32-bit words per circular buffer cycle. Must be greater than 0.

Ritorna:
Returns true if the DMA channels were successfully bound and the SAI hardware started streaming. Returns false if the SAI is not initialized, buffers are NULL, the count is zero, the system is already streaming, or DMA binding fails.

Note:

- This function is non-blocking; it configures the DMA and SAI hardware to run in the background. It is not thread-safe and should not be called from an ISR. Before calling this, the SAI peripheral must be initialized via ca_sai_init. A common mistake is passing buffers that are not properly aligned or are located in memory regions inaccessible to the DMA controller. If the function returns false, the streaming state remains unchanged, and no hardware resources are locked.

Uso:

- This function is used to initiate high-speed, continuous audio data exchange. It is typically paired with ca_sai_stream_pos to monitor the current buffer index during processing.

The typical lifecycle is: initialize the SAI peripheral, allocate buffers, call ca_sai_stream_start, perform processing in a loop while checking the stream position, and eventually stop the stream using ca_sai_hw_stop.

Esempio:

```
/* Assuming SAI is initialized and buffers are allocated */
#define BUFFER_SIZE 1024
i32 tx_buf[BUFFER_SIZE];
i32 rx_buf[BUFFER_SIZE];

if (ca_sai_stream_start(tx_buf, rx_buf, BUFFER_SIZE)) {
    /* Stream started successfully, wait for some data to arrive */
    ca_os_delay_ms(10);

    /* Check current position in the circular buffer */
    u32 pos = ca_sai_stream_pos();

    /* Perform audio processing logic here */

    /* Stop the stream when finished */
    ca_sai_hw_stop();
} else {
    /* Handle error: check if SAI was initialized or if already streaming */
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_sai_stream_start in my application lifecycle?
A: You should call it after initializing the SAI peripheral with ca_sai_init and allocating your DMA-aligned buffers. It is used to initiate high-speed, continuous full-duplex audio data exchange, typically followed by a processing loop that monitors the stream position via ca_sai_stream_pos.

Code example conf:0.90 **Q:** Provide a minimal example of how to start an SAI stream using caOS types.
A: u32 words = 128; i32 tx_buf[128]; i32 rx_buf[128]; if (ca_sai_stream_start(tx_buf, rx_buf, words)) { /* Stream started successfully */ }

Parameters conf:1.00 **Q:** What are the requirements for the tx and rx buffers passed to ca_sai_stream_start?
A: The buffers must be valid pointers, properly aligned for DMA access, and located in memory regions accessible to the DMA controller. The 'words' parameter must be greater than 0, representing the number of 32-bit words per circular buffer cycle.

Return conf:1.00 **Q:** What does a false return value from ca_sai_stream_start indicate?
A: A false return indicates the operation failed. This occurs if the SAI is not initialized, buffers are NULL, the word count is zero, the system is already streaming, or DMA binding fails. In these cases, the streaming state remains unchanged and no hardware resources are locked.

Error/Edge case conf:1.00 **Q:** What is a common mistake when using ca_sai_stream_start?
A: A common mistake is passing buffers that are not properly aligned or are located in memory regions that the DMA controller cannot access, which will cause the function to return false.

Behavior conf:1.00 **Q:** Is ca_sai_stream_start thread-safe or ISR-safe?
A: No, the function is not thread-safe and should not be called from an ISR. It is a non-blocking function that configures the DMA and SAI hardware to run in the background.

ca_sai_xfer

Header: caos/drivers/inc/sai.h:69
Source: caos/drivers/src/sai.c:58:85
GUARD: EN_SAI

```
u32 ca_sai_xfer(const i32 *tx, i32 *rx, u32 words, u32 timeout_ms); /*
    polled, words received */
```

Performs a synchronous, polled full-duplex SAI (Serial Audio Interface) data transfer.

Parametri:

Nome	Tipo	Descrizione
tx	const i32 *	Pointer to the buffer containing data to transmit. Must not be NULL.
rx	i32 *	Pointer to the buffer where received data will be stored. Must not be NULL.
words	u32	Number of 32-bit words to exchange. Must be greater than 0.
timeout_ms	u32	Maximum time in milliseconds to wait for the transfer to complete.

Ritorna:
Returns the number of words successfully received. Returns 0 if the SAI is not initialized, if pointers are NULL, or if the transfer fails/times out.

Note:

- This function is blocking, meaning the calling task will wait inside the function until the transfer completes or the timeout expires. It is NOT safe to call from an ISR because it relies on polling the hardware status and system ticks. Before calling this, ensure the SAI peripheral has been successfully initialized via ca_sai_init. A common mistake is providing a buffer that is too small for the requested number of words, which will lead to memory corruption. Always verify the return value matches the requested 'words' count to ensure the transfer was successful.

Uso:

- This function is used for synchronous audio data exchange where the application needs to wait for the hardware to finish processing before proceeding. It is typically used in a sequence: first, initialize the SAI peripheral with ca_sai_init, then prepare the transmit data, call ca_sai_xfer to perform the exchange, and finally check the results. It is often paired with ca_os_tick_64_ms to manage timing or ca_sai_errors to diagnose hardware issues if the transfer count is lower than expected.

Esempio:

```
/* Assuming SAI is already initialized and buffers are allocated */
#define NUM_WORDS 16
i32 tx_buf[NUM_WORDS] = {0}; /* Fill with audio data */
i32 rx_buf[NUM_WORDS];

/* Perform the transfer with a 100ms timeout */
u32 received = ca_sai_xfer(tx_buf, rx_buf, NUM_WORDS, 100U);

if (received != NUM_WORDS) {
    /* Handle error: check hardware status */
    u32 err = ca_sai_errors();
    /* Log error or reset peripheral */
} else {
```

```
    /* Process received audio data in rx_buf */
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_sai_xfer function in my caOS application?
A: You should use ca_sai_xfer when you need to perform a synchronous, full-duplex audio data exchange where the application must wait for the hardware to complete the transfer before proceeding to the next step.

Code example conf:1.00 **Q:** Provide a minimal C example of how to call ca_sai_xfer using caOS types.
A: i32 tx_buf[10] = {0}; i32 rx_buf[10] = {0}; u32 words = 10; u32 timeout = 100; u32 received = ca_sai_xfer(tx_buf, rx_buf, words, timeout);

Parameters conf:1.00 **Q:** What are the constraints on the tx and rx pointers in ca_sai_xfer?
A: Both the tx and rx pointers must not be NULL. Providing NULL pointers will cause the function to return 0 immediately.

Return conf:1.00 **Q:** What does the return value of ca_sai_xfer represent?
A: The return value represents the number of 32-bit words successfully received. A return value of 0 indicates that the SAI was not initialized, pointers were NULL, or the transfer failed/timed out.

Error/Edge case conf:1.00 **Q:** What is a common mistake that leads to memory corruption when using ca_sai_xfer?
A: A common mistake is providing a buffer that is smaller than the requested number of words, which will lead to memory corruption.

Behavior conf:1.00 **Q:** Is ca_sai_xfer safe to call from an interrupt service routine (ISR)?
A: No, it is not safe to call from an ISR because the function is blocking and relies on polling hardware status and system ticks.

`ca_sai_xfer_dma`

Header: `caos/drivers/inc/sai.h:74`
Source: `caos/drivers/src/sai.c:140:172`
GUARD: `EN_SAI`

```
bool ca_sai_xfer_dma(const i32 *tx, i32 *rx, u32 words, u32 timeout_ms);
    /* one block, both ways */
```

Performs a simultaneous full-duplex audio data transfer using DMA for both SAI transmit and receive channels.

Parametri:

Nome	Tipo	Descrizione
`tx`	`const i32 *`	Pointer to the buffer containing audio data to be transmitted. Must not be NULL.
`rx`	`i32 *`	Pointer to the buffer where received audio data will be stored. Must not be NULL.
`words`	`u32`	Number of 32-bit audio samples to transfer. Must be greater than 0.
`timeout_ms`	`u32`	Maximum time in milliseconds to wait for the DMA transfer to complete.

Ritorna:
Returns true if the transfer completed successfully within the timeout, false otherwise. Returns false if parameters are invalid, if the SAI is currently streaming, or if DMA binding/transfer fails.

Note:

- This function is blocking; it will suspend the calling task until the DMA transfer finishes or the timeout expires. It is not safe to call from an ISR because it relies on blocking wait mechanisms. Before calling this, ensure the SAI hardware is properly initialized. A common mistake is passing a buffer that is too small for the requested number of words, which will lead to memory corruption. Always check the return value; if it returns false, the transfer did not complete, and the hardware state may need to be reset.

Uso:

- This function is used for high-performance, full-duplex audio streaming where CPU intervention per sample is undesirable. It is typically used in a loop or a dedicated audio processing task. The sequence is: initialize the SAI and DMA subsystems, allocate buffers, call this function to perform the transfer, and process the received data. It is often paired with ca_sai_init for setup and ca_memory_allocate for buffer management.

Esempio:

```
/* Example: Perform a single full-duplex audio block transfer */
#define AUDIO_BUFFER_SIZE 1024

void audio_task(void *arg) {
    i32 *tx_buf = ca_memory_allocate(AUDIO_BUFFER_SIZE * sizeof(i32));
    i32 *rx_buf = ca_memory_allocate(AUDIO_BUFFER_SIZE * sizeof(i32));

    if (tx_buf != NULL && rx_buf != NULL) {
        /* Fill tx_buf with audio data here */

        /* Perform DMA transfer with a 500ms timeout */
        bool success = ca_sai_xfer_dma(tx_buf, rx_buf, AUDIO_BUFFER_SIZE, 500U);

        if (success) {
            /* Process received data in rx_buf */
        } else {
            /* Handle transfer error */
        }
    }

    ca_memory_free(tx_buf);
    ca_memory_free(rx_buf);
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_sai_xfer_dma instead of manual sample processing?
A: You should use ca_sai_xfer_dma for high-performance, full-duplex audio streaming where CPU intervention per sample is undesirable. It is designed to be used in a loop or a dedicated audio processing task to offload data movement to DMA.

Code example conf:0.90 **Q:** Provide a minimal example of how to call ca_sai_xfer_dma.
A: i32 tx_buf[128]; i32 rx_buf[128]; u32 words = 128; u32 timeout = 100; if (ca_sai_xfer_dma(tx_buf, rx_buf, words, timeout)) { /* Process rx_buf */ }

Parameters conf:1.00 **Q:** What are the constraints on the 'words' parameter?
A: The 'words' parameter represents the number of 32-bit audio samples to transfer and must be greater than 0. Passing an invalid value will cause the function to return false.

Return conf:1.00 **Q:** What does a return value of false indicate?
A: A return value of false indicates the transfer did not complete successfully. This can be caused by an invalid parameter, the SAI being currently busy streaming, a DMA binding/transfer failure, or the operation exceeding the specified timeout.

Error/Edge case conf:1.00 **Q:** What is a common mistake when using this function that leads to memory corruption?
A: A common mistake is passing a buffer that is too small for the requested number of words, which will lead to memory corruption.

Behavior conf:1.00 **Q:** Is ca_sai_xfer_dma safe to call from an interrupt service routine (ISR)?
A: No, it is not safe to call from an ISR because the function is blocking and relies on wait mechanisms that suspend the calling task until the DMA transfer finishes or the timeout expires.

`ca_scheduler_check_all_canaries`

Header: `caos/inc/scheduler.h:464`
Source: `caos/src/scheduler.c:1764:1783`
GUARD: `EN_SCHEDULER`

```
void ca_scheduler_check_all_canaries(void)
```

Checks all task stacks for corruption using canary values in an MPU-enabled system.

Ritorna:
This function does not return a value (void). However, it prints diagnostic messages to stdout indicating whether each task's stack is OK or corrupted.

Note:

- This function is NOT blocking; it runs synchronously and prints results immediately. It is NOT thread-safe because it accesses global data structures like g_task_list and g_slot_map. It should NOT be called from an ISR since it uses printf and may block. It requires the MPU (Memory Protection Unit) to be enabled (___MPU_PRESENT must be defined). A common mistake is calling this function without ensuring that the system has MPU support, which will result in no output. The function relies on ca_os_task_snapshot and check_stack_canary, which must be implemented correctly for accurate results.

Uso:

- This function is typically used during debugging or diagnostics, especially when checking for stack overflows or memory corruption in tasks. It is usually invoked via a CLI (command-line interface) command like 'canary' when the system is running with MPU enabled. It pairs with ca_os_task_snapshot and check_stack_canary to perform the actual stack validation.

Esempio:

```
/* Example usage in a CLI handler */
void cli_canary_command(void) {
    // This function is called when user types 'canary' in CLI
    #if __MPU_PRESENT    // Only execute if MPU is enabled
        ca_scheduler_check_all_canaries();  // Check all tasks' stack canaries
    #else
        ca_stdout_printf_n("MPU not enabled, cannot check canaries.\n");
    #endif
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_scheduler_check_all_canaries() and why is it useful?
A: You should call ca_scheduler_check_all_canaries() during debugging or development to verify that the stack of each running task has not been corrupted. It is useful

for detecting memory overflows or buffer overruns in task stacks, which can lead to unpredictable behavior or system crashes. The function is intended for use in non-critical tasks or debug contexts, not in production code.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_scheduler_check_all_canaries() in a caOS application?
A: void debug_task(void) { while (1) { ca_scheduler_check_all_canaries(); ca_task_sleep(1000); } }
void main(void) { ca_task_create(debug_task, 1024, 1, "debug_task"); ca_scheduler_start(); }

Return conf:0.90 **Q:** What does ca_scheduler_check_all_canaries() return, and what causes it to fail?
A: The function does not return any value. It prints the status of stack canaries for each active task to stdout. If the function fails, it is due to incorrect initialization of the task list or improper stack allocation, but it does not return an error code.

Error/Edge case conf:0.85 **Q:** What are the common mistakes or edge cases when using ca_scheduler_check_al
A: Common mistakes include calling the function without properly initializing the task list, or without correctly allocating stack memory for tasks. It is also not safe to call this function from an ISR, and it will not perform any checks if the MPU is not enabled. Additionally, calling it from a critical task may cause unexpected behavior due to its blocking nature.

Behavior conf:0.90 **Q:** Is ca_scheduler_check_all_canaries() blocking, ISR-safe, or thread-safe?
A: The function is blocking and not thread-safe. It must not be called from an ISR. It is intended to be used only in non-critical tasks or during debugging. The function requires the MPU to be enabled (__MPU_PRESENT), otherwise it does nothing. It does not modify the system state, only prints information to stdout.

`ca_scheduler_dump_all_tasks`

Header: `caos/inc/scheduler.h:417`
Source: `caos/src/scheduler.c:2291:2309`
GUARD: `EN_SCHEDULER`

```
1 void ca_scheduler_dump_all_tasks( u16 mode)
```

Prints a formatted list of all registered tasks and their current status to the system output.

Parametri:

Nome	Tipo	Descrizione
`mode`	`u16`	Bitmask defining the level of detail. Use DUMP_TASK_REG to show CPU registers, DUMP_TASK_STACK to show stack usage and integrity, or DUMP_TASK_ALL to show everything.

Ritorna:
void

Note:

- This function is intended for debugging and system monitoring. It is not thread-safe and should not be called from an Interrupt Service Routine (ISR) because it performs multiple blocking I/O operations via ca_stdout_printf_n. The function iterates through the internal task list; if the scheduler is in an inconsistent state or if memory corruption has occurred, this function may trigger a hard fault. Always ensure the system is initialized

before calling this. A common beginner mistake is calling this function while the scheduler is under heavy load, which can cause significant jitter or output buffer overflows.

Uso:

- This function is typically used within a Command Line Interface (CLI) or a diagnostic task to inspect system health. It is often paired with ca_scheduler_update_all_stack_usage to ensure the stack usage data is current before printing. Developers use this to verify that all tasks are running as expected, check for stack overflows, or identify which task is currently active during a system hang. It is usually invoked manually by a user via a CLI command to troubleshoot specific task behavior.

Esempio:

```c
/* Diagnostic command to list all tasks and their status */
void cli_cmd_list_tasks(void) {
    /* Update stack usage statistics before printing to ensure data accuracy */
    ca_scheduler_update_all_stack_usage();

    /* Print header for the user */
    ca_stdout_printf_n("--- System Task Diagnostic ---");

    /* Dump all tasks with full detail mode (DUMP_TASK_ALL) */
    /* DUMP_TASK_ALL is a bitwise combination of DUMP_TASK_REG and
    DUMP_TASK_STACK */
    ca_scheduler_dump_all_tasks(0xFFFF);

    /* Finalize diagnostic output */
    ca_stdout_printf_n("--- End of Report ---");
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_scheduler_dump_all_tasks() and what is its purpose in debugging?
A: You should call ca_scheduler_dump_all_tasks() during debugging or monitoring to get detailed information about all active tasks in the RTOS. It is useful for inspecting task states, priorities, and other internal details to understand system behavior. The function prints directly to stdout, making it a quick way to observe task activity without complex instrumentation.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_scheduler_dump_all_ with caOS types?
A: u16 dump_mode = DUMP_TASK_ALL; ca_scheduler_dump_all_tasks(dump_mode);

Parameters conf:0.85 **Q:** What does the mode parameter in ca_scheduler_dump_all_tasks() control, and what are valid values?
A: The mode parameter controls the level of detail in the dump output. DUMP_TASK_ALL (0) prints all available task details. Other values may be used for filtering, but only DUMP_TASK_ALL is explicitly defined as valid. Any other value is implementation-defined and may not produce expected output.

Error/Edge case conf:0.90 **Q:** What are common mistakes or edge cases when using ca_scheduler_dump_all_t
A: Common mistakes include calling the function from multiple tasks simultaneously, which is not thread-safe. It should also not be called from an ISR unless the underlying stdout mechanism is thread-safe. Additionally, if the scheduler is not initialized or tasks are not registered, the output may be incomplete or incorrect. The function does not handle internal errors, so inconsistent system states may lead to incomplete dumps.

Behavior conf:0.90 **Q:** Is ca_scheduler_dump_all_tasks() blocking or non-blocking, and can it be called from an ISR?

A: The function is blocking, as it waits for all output to complete before returning. It is not thread-safe, so concurrent calls from multiple tasks must be avoided. It can be called from an ISR only if the output system (ca_stdout_printf_n) is also thread-safe. The function requires the scheduler to be initialized and tasks to be registered; otherwise, behavior is undefined.

---

ca_scheduler_dump_task

Header: caos/inc/scheduler.h:411
Source: caos/src/scheduler.c:2205:2289
GUARD: EN_SCHEDULER

```
bool ca_scheduler_dump_task(u16 index, u16 mode)
```

Prints detailed diagnostic information about a specific task to the standard output.

Parametri:

Nome	Tipo	Descrizione
index	u16	The unique identifier (index) of the task in the system task list. Must be a valid index within the range of allocated tasks.
mode	u16	Bitmask flags (e.g., DUMP_TASK_HDR, DUMP_TASK_REG, DUMP_TASK_ADDR) determining which sections of the task context to print.

Ritorna:
Returns true if the task was found and successfully dumped, false if the index is invalid, the task slot is empty, or the task's memory integrity (canary) is corrupted.

Note:

- This function is non-blocking and intended for debugging purposes. It is not thread-safe if the task list is being modified simultaneously, so it should be used carefully during runtime. It is not safe to call from an ISR because it performs multiple I/O operations via ca_stdout_printf_n, which may involve mutexes or blocking mechanisms. The most common beginner mistake is passing an out-of-bounds index or assuming the task exists without checking the return value, which can lead to misleading debug logs. Always ensure the system is in a stable state before dumping, as the task might be removed or its slot reused while the function is executing.

Uso:

- This function is typically used in CLI interfaces or diagnostic shells to inspect the state of a specific thread when a system fault or unexpected behavior occurs. It is often paired with ca_scheduler_dump_all_tasks for a global view or used after a system crash to identify which task caused a memory fault or stack overflow. Developers use it to verify task priorities, stack usage, and register values during live debugging sessions. It is usually invoked after parsing a user command via ca_strtol to identify the target task index.

Esempio:

```
/* Example: Debugging a specific task via CLI command */
#define DUMP_TASK_ALL 0xFFFF

void debug_task_info(u16 task_id) {
    /* Check if the task exists before attempting to dump */
    /* Note: ca_scheduler_dump_task internally validates the task context */
```

```
    if (!ca_scheduler_dump_task(task_id, DUMP_TASK_ALL)) {
        ca_printf_n("Error: Failed to dump task %u. It may not exist or is corrupted.", task_id);
    } else {
        ca_printf_n("Task %u dump completed successfully.", task_id);
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_scheduler_dump_task in my caOS project?
A: You should use ca_scheduler_dump_task for debugging and monitoring tasks in your caOS project. It is particularly useful during development when you need to inspect the state of a specific task, such as its stack contents, registers, or execution statistics. It helps verify task integrity and can be used to detect issues like stack overflows or corrupted task structures.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_scheduler_dump_task with caOS types?
A: u16 index = 0; bool result = ca_scheduler_dump_task(index, DUMP_TASK_ALL); if (!result) { // Handle error }

Parameters conf:0.95 **Q:** What does the 'index' parameter in ca_scheduler_dump_task represent and what are valid values?
A: The 'index' parameter specifies the index of the task to dump. It must be a valid index between 0 and the maximum number of tasks allowed. If the index is out of range, the function returns false and does not dump any information.

Return conf:0.90 **Q:** What does ca_scheduler_dump_task return and what causes it to return false?
A: The function returns true if the task exists, is valid, and information was printed successfully. It returns false if the task does not exist, is empty, or is invalid (e.g., canary is corrupted). It also returns false if the index is out of range.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_scheduler_dump_task?
A: Common mistakes include not checking the return value of the function, which can lead to printing incorrect or invalid information. Another issue is calling the function from an ISR, which is not recommended. Additionally, calling it from multiple threads simultaneously can cause corrupted output since the function is not thread-safe.

Behavior conf:0.90 **Q:** Is ca_scheduler_dump_task blocking or non-blocking, and is it safe to call from an ISR or multiple threads?
A: The function is non-blocking. However, it is not safe to call from an ISR because it prints to stdout and may interfere with real-time execution. It is also not thread-safe; calling it from multiple threads simultaneously can result in corrupted output.

`ca_scheduler_get_context_from_idx`

Header: `caos/inc/scheduler.h:331`
Source: `caos/src/scheduler.c:2052:2057`
GUARD: `EN_SCHEDULER`

```
const ca_thread_context_t* ca_scheduler_get_context_from_idx(u32 idx)
```

Retrieves a pointer to a thread's context structure by its index in the scheduler's task list.

Parametri:

Nome	Tipo	Descrizione
idx	u32	The index of the thread in the global task list. Must be less than the total number of active threads. Invalid values (e.g., out-of-bounds index) will cause the function to return NULL.

Ritorna:
Pointer to a valid ca_thread_context_t structure: NULL:

Note:

- **Blocking behavior**: This function does not block. It directly accesses an internal data structure.
- **Thread safety**: The function is not thread-safe. It accesses global variables like g_task_list without synchronization. Concurrent access from multiple threads may lead to undefined behavior.
- **Callable from ISR**: Yes, this function can be called from an ISR, but only if the task list is stable and not being modified concurrently.
- **Preconditions**: The scheduler must be initialized and the task list must be valid. Calling this function before the scheduler starts or after it has been shut down will likely return NULL.
- **Common mistakes**: Using an index that is greater than or equal to the number of active threads leads to a NULL return. Also, not checking for NULL before dereferencing the returned pointer can cause a crash.

Uso:

- **Task notification system**: Used to retrieve a thread's context when sending a notification to a specific thread by index.
- **Stack memory management**: Used to find which thread owns a given memory address, for stack overflow detection or debugging.
- **Priority inheritance**: Used to retrieve the thread context of a mutex owner to compute effective priority.
- **Fault handling**: Used to access thread contexts during memory management fault handling to inspect or modify thread state.

Esempio:

```
// This example shows how to safely retrieve a thread context by index and check for NULL before using it. It's commonly used in task notification systems.
/* Example: Sending a notification to a specific thread */
bool send_notification_to_thread(u32 thread_index, u32 notification_value) {
    ca_thread_context_t *ctx = (ca_thread_context_t *)
    ca_scheduler_get_context_from_idx(thread_index);
    if (ctx == NULL) {
        // Invalid thread index or scheduler not initialized
        return false;
    }
    // Proceed with notification logic using ctx
    return true;
}
// This example demonstrates how to iterate through all threads to find which one owns a given memory address, useful for stack overflow detection or debugging.
/* Example: Finding stack owner for memory fault handling */
const ca_thread_context_t *find_stack_owner(u32 address) {
    u32 num_threads = ca_scheduler_size();
    if (num_threads == 0xFFFFFFFF) return NULL; // Scheduler not initialized

    for (u32 i = 0; i < num_threads; i++) {
```

```
        const ca_thread_context_t *thread_ctx =
    ca_scheduler_get_context_from_idx(i);
        if (thread_ctx == NULL) continue;
        // Check if address is within thread's stack range
        if (address >= (thread_ctx->stack_end - STACK_CANARY_SIZE) &&
            address < thread_ctx->stack_start) {
            return thread_ctx;
        }
    }
    return NULL;
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_scheduler_get_context_from_idx in my application?
A: You should use this function when you need to inspect or modify properties of a task other than the currently running one. Common use cases include checking another task's priority for inheritance, managing memory ownership, performing system-wide diagnostics, or sending notifications via ca_notify_send.

Code example conf:1.00 **Q:** How do I safely retrieve a task context and access its data?
A: You must always check the returned pointer for NULL before dereferencing it to avoid a hard fault. Example: u32 my_idx = 2; const ca_thread_context_t* ctx = ca_scheduler_get_context_from_idx(my_idx); if (ctx != NULL) { /* access context safely */ }

Parameters conf:1.00 **Q:** What does the idx parameter represent and what are the constraints on its value?
A: The idx parameter is a u32 representing the unique numerical index of a task. It must be a valid index within the system's task list. Providing an index that does not correspond to an active task or is out of bounds will result in a NULL return.

Return conf:1.00 **Q:** What does the return value of ca_scheduler_get_context_from_idx indicate?
A: The function returns a pointer to the ca_thread_context_t structure if the index is valid. It returns NULL if the task does not exist or the provided index is out of bounds. Failure to check for NULL before dereferencing will cause a hard fault.

`ca_scheduler_get_current_context`

Header: `caos/inc/scheduler.h:324`
Source: `caos/src/scheduler.c:2048:2050`
GUARD: `EN_SCHEDULER`

```
const ca_thread_context_t* ca_scheduler_get_current_context(void)
```

Returns a pointer to the internal control structure of the currently executing task.

Ritorna:
A pointer to the ca_thread_context_t structure of the active task, or NULL if the scheduler is not running or the function is called from an interrupt context where no task is active.

Note:

- This function is non-blocking and thread-safe as it simply reads a global pointer maintained by the scheduler. It is safe to call from any context, but you must handle the NULL return case, which indicates that the system is either in the pre-scheduler initialization phase or currently executing inside an ISR. A common beginner mistake is assuming this function will always return a valid pointer; always check for NULL before dereferencing the result to avoid a HardFault.

Uso:

- This function is primarily used to identify the 'owner' of a resource or to determine if the current execution flow is allowed to perform blocking operations. It is frequently paired with ca_os_task_slot to map the current task to a fixed index for resource tracking, such as memory allocation ownership or event waiting. You will typically see it used in drivers or system services to decide whether to perform a blocking wait (if in a task) or a busy-spin (if in an ISR or early boot).

Esempio:

```
void log_task_memory_usage(void) {
    /* Retrieve the context of the task currently calling this function */
    const ca_thread_context_t *ctx = ca_scheduler_get_current_context();

    /* If NULL, we are likely in an ISR or before the OS started */
    if (ctx == NULL) {
        ca_printf("System context: No task active\n");
        return;
    }

    /* Convert the context to a task slot index for identification */
    u32 slot = ca_os_task_slot(ctx);
    u32 usage = ca_memory_get_task_usage(slot);

    ca_printf("Task in slot %lu is using %lu bytes of memory\n", slot, usage);
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_scheduler_get_current_context in my application?
A: You should use this function when you need to identify the currently executing task for resource management, synchronization, or logging. It is commonly used to associate a task with a resource, register a task into a wait queue, or track memory ownership. It is often paired with ca_os_wait to manage task state transitions.

Code example conf:1.00 **Q:** How do I safely retrieve and check the current task context in C?
A: You must always check the returned pointer for NULL before dereferencing it to prevent hard faults. Example: const ca_thread_context_t* ctx = ca_scheduler_get_current_context(); if (ctx != NULL) { /* perform task-specific logging or resource tracking */ }

Return conf:1.00 **Q:** What does the return value of ca_scheduler_get_current_context represent, and why might it return NULL?
A: The function returns a pointer to the constant ca_thread_context_t structure of the current task. It returns NULL if the scheduler is not yet running (e.g., before calling ca_os_run) or if the function is invoked from an invalid context.

`ca_scheduler_get_current_task_id`

Header: `caos/inc/scheduler.h:337`
Source: `caos/src/scheduler.c:2059:2064`
GUARD: `EN_SCHEDULER`

```
u32 ca_scheduler_get_current_task_id(void)
```

Returns the unique index identifier of the task currently executing on the CPU.

Ritorna:
Returns a u32 representing the task index (0 to N-1). Returns 0xFFFFFFFF if no task is currently running (e.g., called before the scheduler starts or during early system initialization).

Note:

- This function is thread-safe and non-blocking. It is safe to call from both task context and ISRs, as it simply reads the global pointer to the current task structure. A common beginner mistake is assuming this function will always return a valid ID; always check for the 0xFFFFFFFF error code before using the result as an index into task arrays or lookup tables. It does not require any specific initialization beyond the OS scheduler being active.

Uso:

- This function is primarily used for task-specific resource management, such as identifying which task owns a specific memory block, logging task-specific debug information, or implementing task-aware synchronization primitives. It is frequently paired with ca_scheduler_get_task_name to provide human-readable logs, or used in conjunction with ca_memory_get_task_usage to monitor stack or heap consumption per task. In complex systems, it helps differentiate between multiple tasks accessing the same shared resource (like a message buffer or stream) to ensure proper ownership tracking.

Esempio:

```c
/* Example: Logging the current task ID and name for debugging purposes */
#include "caos.h"

void log_current_task_info(void) {
    u32 task_id = ca_scheduler_get_current_task_id();

    if (task_id != 0xFFFFFFFF) {
        const char *name = ca_scheduler_get_task_name(task_id);
        if (name != NULL) {
            ca_printf("Current Task: %s (ID: %u)\n", name, task_id);
        } else {
            ca_printf("Current Task ID: %u (Name unknown)\n", task_id);
        }
    } else {
        ca_printf("No task is currently running (System idle or pre-init).\n");
    }
}
```

Q&A — fine-tuning review

3/3 pairs · qwen3-coder:latest

Usage conf:0.95 **Q:** When should a developer call ca_scheduler_get_current_task_id() and why is it useful in a real-time system?
A: A developer should call ca_scheduler_get_current_task_id() to identify the currently running task, which is useful for debugging, logging, or implementing task-specific behavior. In a real-time system, this helps in maintaining context awareness, especially when the same code might be executed by different tasks. It is used in files like message.c, safeguard.c, and scheduler.c to ensure proper task identification.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_scheduler_get_current_task_ with caOS types?
A: u32 task_id = ca_scheduler_get_current_task_id(); if (task_id != 0xFFFFFFFF) { // Task is valid, proceed with task-specific logic // Example: log task_id or use it for identification } else { // No active task, handle error case }

Return conf:0.90 **Q:** What do the return values of ca_scheduler_get_current_task_id() mean and what causes the function to return 0xFFFFFFFF?
A: The function returns a u32 representing the index of the currently running task. If no task is active, it returns 0xFFFFFFFF. This typically happens when the scheduler has not yet started or if the function is called in an invalid context where g_current_task is not properly initialized. The function is not blocking and is thread-safe, but calling it from an ISR requires a safe context.

ca_scheduler_get_next_wait

Header: caos/inc/scheduler.h:233
GUARD: EN_SCHEDULER

```
static inline ca_thread_context_t* ca_scheduler_get_next_wait(
    ca_thread_context_t *task)
```

Retrieves the next task in a wait queue chain, used to traverse or remove tasks from a linked list of waiting threads.

Parametri:

Nome	Tipo	Descrizione
task	ca_thread_context_t*	Pointer to the current task in the wait queue. Must be a valid pointer to a task context, or NULL if the queue is empty.

Ritorna:
Pointer to the next task in the wait queue: NULL:

Note:

- {'content': 'This function does not block; it simply returns the next pointer in a linked list.'}
- {'content': 'It is safe to call from both thread and ISR contexts, as it only reads memory.'}
- {'content': 'It is not thread-safe if the queue is modified concurrently by another thread or ISR.'}
- {'content': "The function assumes that the input 'task' is a valid node in a wait queue and that the 'next' field is properly initialized."}
- {'content': "Common mistake: Using the returned pointer without checking if it's NULL before dereferencing."}

Uso:

- {'content': 'Used to traverse a wait queue, such as in event or semaphore handling, to find the next waiting task.'}
- {'content': 'Used to remove a task from a wait queue by updating the links of the previous task.'}
- {'content': 'Used in debugging or validation functions to check if a specific task is in a wait queue.'}

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_scheduler_get_next_wait in my kernel-level code?
A: You should use this function when you need to traverse a linked list of blocked threads, such as when implementing synchronization primitives like semaphores or event queues. It is used to iterate through wait queues to identify tasks that need to be woken up when a resource becomes available.

Code example conf:1.00 **Q:** How do I iterate through a wait list using ca_scheduler_get_next_wait?
A: You can traverse the list by repeatedly calling the function until it returns NULL. Example: ca_thread_context_t *current = head_task; while (current != NULL) { current = ca_scheduler_get_next_wait(current); }

Parameters conf:1.00 **Q:** What are the requirements for the 'task' parameter in ca_scheduler_get_next_wait?
A: The 'task' parameter must be a pointer to a valid ca_thread_context_t structure or NULL. Passing a dangling pointer or a pointer to a task that has already been deallocated is invalid and will result in a hard fault.

Return conf:1.00 **Q:** What does the return value of ca_scheduler_get_next_wait indicate?
A: The function returns a pointer to the next ca_thread_context_t in the wait chain. It returns NULL if the input task is NULL or if the end of the wait list has been reached.

`ca_scheduler_get_runtime_total`

Header: `caos/inc/scheduler.h:704`
Source: `caos/src/scheduler.c:2929:2945`
GUARD: `EN_SCHEDULER`

```
u32 ca_scheduler_get_runtime_total(void)
```

Returns the total CPU cycles consumed by the currently running task.

Ritorna:
Returns a u32 value representing the total CPU cycles accumulated by the task. Returns (u32)-1 if no task is currently running (e.g., called from an invalid context).

Note:

- This function is thread-safe and non-blocking. It reads the DWT->CYCCNT register to calculate the time spent in the current slice. It is safe to call from a task, but calling it from an ISR is generally discouraged as it relies on the 'g_current_task' pointer, which may not be meaningful depending on the interrupt context. The function uses a 'do-while' loop to ensure that the read operation is atomic relative to context switches; if a PendSV interrupt occurs during the read, the function detects the inconsistency and retries. A common mistake is assuming this value represents wall-clock time; it represents CPU cycles, which depend on the current system clock frequency.

Uso:

- This function is primarily used for performance profiling and task monitoring. It is typically paired with ca_measure_start and ca_measure_stop to calculate the execution time of a specific code block. By capturing the total runtime before and after a critical section, developers can determine how many CPU cycles a specific algorithm or task consumes. It is often used in conjunction with ca_scheduler_dump_task or similar diagnostic tools to identify tasks that are hogging the CPU.

Esempio:

```
u32 start_cycles, end_cycles, elapsed;

// Capture the total runtime before the operation
ca_measure_start(&start_cycles);

// Perform a heavy computation
// ... (some complex algorithm) ...

// Capture the total runtime after the operation
end_cycles = ca_scheduler_get_runtime_total();

// Calculate the difference to find the cost of the operation
if (end_cycles != (u32)-1 && start_cycles != (u32)-1) {
    elapsed = end_cycles - start_cycles;
    ca_printf("Operation took %u cycles\n", elapsed);
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_scheduler_get_runtime_total() and what is its purpose in a real-time system?
A: You should call ca_scheduler_get_runtime_total() to measure the total CPU time used by the currently running task in clock cycles. This is useful for monitoring task performance, implementing watchdog mechanisms, and analyzing CPU utilization. It is intended to be used from within an active thread context, not from an ISR, as it's not thread-safe and relies on a global variable tracking the current task.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_scheduler_get_runtime_total() with caOS types?
A: u32 runtime = ca_scheduler_get_runtime_total(); if (runtime != (u32)-1) { // Use runtime value for monitoring or logging // Example: log the runtime to a buffer or send via UART } else { // Handle error case - function failed or called outside task context }

Return conf:0.95 **Q:** What do the return values of ca_scheduler_get_runtime_total() indicate, and what causes it to return (u32)-1?
A: The function returns a u32 representing the total number of clock cycles elapsed since the last watchdog reset for the current task. If the function returns (u32)-1, it indicates either that there is no current task running, or an internal error occurred. Common causes include calling the function from outside an active task context (e.g., from an ISR) or a failure in internal tracking mechanisms.

`ca_scheduler_get_task_name`

Header: `caos/inc/scheduler.h:358`
Source: `caos/src/scheduler.c:2074:2079`
GUARD: `EN_SCHEDULER`

```
1 const char* ca_scheduler_get_task_name(u32 idx)
```

Retrieves the name of a task given its index in the task list.

Parametri:

Nome	Tipo	Descrizione
`idx`	`u32`	Index of the task in the global task list. Must be a valid index (0 to task count - 1). Invalid indices (e.g., greater than or equal to task count) will cause the function to return NULL.

Ritorna:
`Pointer to the task name string`: Valid pointer to a null-terminated string containing the task name if the index is valid. `NULL`: Returned when the index is invalid or when the task context at that index is NULL. This can happen if the task was deleted or never initialized.

Note:

- **Blocking behavior**: This function does not block. It directly accesses a global task list and returns a pointer to a static string.
- **Thread safety**: The function is not thread-safe. It accesses a global structure (g_task_list) without synchronization. If the task list is modified concurrently, the behavior is undefined.
- **Callable from ISR**: Yes, this function can be called from an ISR, but only if the task list is not modified during the ISR execution.
- **Preconditions**: The global task list (g_task_list) must be initialized and valid. The index must be within the bounds of the task list.
- **Common mistakes**: Using an invalid index (e.g., beyond the number of tasks) leads to a NULL return. Not checking for NULL before using the returned pointer may cause a crash.

Uso:

- **Debugging or logging**: Used to print or log the name of a task, especially in error handling or system status reporting. Commonly used in conjunction with ca_scheduler_get_current_task_id
- **System monitoring**: Used to retrieve task names for monitoring or diagnostics, such as in safeguard or system status modules.

Esempio:

```
// Example: Print current task name in a safeguard module
if (ca_os_running()) {
    u32 thid = ca_scheduler_get_current_task_id();
    if (thid != 0xFFFFFFFF) {
        const char *name = ca_scheduler_get_task_name(thid);
        if (name) {
            PRINTF_N(buf, sizeof(buf), "  Task: %s [%u]", name, thid);
        }
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_scheduler_get_task_name() in my embedded application?
A: You should use ca_scheduler_get_task_name() when you need to retrieve the name of a task for debugging or logging purposes. It is particularly useful in functions like CLI commands, memory diagnostics, or safeguard checks where identifying tasks by name is required. It is not intended for real-time task switching or scheduling decisions.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_scheduler_get_task_name() in C with caOS types?
A: cstr task_name = ca_scheduler_get_task_name(0); if (task_name != NULL) { // Use task_name for logging or debugging } else { // Handle invalid index or task not found }

Parameters conf:0.90 **Q:** What does the idx parameter in ca_scheduler_get_task_name() represent, and what are valid values?
A: The idx parameter represents the index of the task in the global task list (g_task_list). Valid values are integers from 0 up to the maximum number of tasks minus one. Using negative values or values greater than or equal to the maximum number of tasks may lead to invalid memory access or NULL returns.

Return conf:0.95 **Q:** What do the return values of ca_scheduler_get_task_name() indicate, and what causes it to return NULL?
A: The function returns a pointer to the task name string if the index is valid and the task exists. It returns NULL if the index is invalid, if the task list is not initialized, or if the internal function g_task_list_at fails to return a valid task. NULL can also be returned if the task entry is NULL, indicating the task slot is not occupied.

`ca_scheduler_init`

Header: `caos/inc/scheduler.h:453`
Source: `caos/src/scheduler.c:1920:1968`
GUARD: `EN_SCHEDULER`

```
bool ca_scheduler_init(void)
```

Initializes the RTOS scheduler and task management system.

Ritorna:

true if initialization succeeds, false if it fails due to memory or configuration issues.

Note:

- This function is NOT blocking. It must be called BEFORE any tasks are created or scheduled. It is NOT safe to call from an ISR. It is not thread-safe. It requires proper linker setup and memory allocation. Common mistake: calling this function more than once or after tasks have already been created.

Uso:

- Called once during system boot-up, typically after basic hardware initialization but before starting any tasks. It pairs with ca_os_job() to create tasks and ca_os_tick_calibrate() for timing calibration. It should be used in a pre-scheduler context where no tasks are running yet.

Esempio:

```
bool res = ca_scheduler_init();
if (res == false) {
    // Handle initialization failure
    PRINTF_N(buf, DEFAULT_MSG_SIZE, "ERROR: Scheduler init failed %d", res);
    // System should halt or reset here
    while (1) { } // Prevent further execution
}
// Scheduler is now ready to accept tasks created with ca_os_job()
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should ca_scheduler_init be called in a caOS application?
A: ca_scheduler_init must be called only once, before creating any tasks. It should be invoked after memory initialization (e.g., ccm_init) and before any task creation. It is used to set up the RTOS scheduler with either round-robin or priority-based scheduling mode.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to initialize the scheduler using caOS types?
A: u16 mode = SCHED_PRIORITY; bool result = ca_scheduler_init(mode); if (!result) { // Handle initialization failure }

Parameters conf:0.90 **Q:** What does the mode parameter in ca_scheduler_init control and what are valid values?
A: The mode parameter selects the scheduling algorithm. SCHED_ROUND_ROBIN (0) enables round-robin scheduling, while SCHED_PRIORITY (1) enables priority-based scheduling. Any other value is invalid and may cause initialization failure.

Return conf:0.95 **Q:** What does ca_scheduler_init return and what causes it to return false?
A: It returns true on successful initialization. It returns false if memory allocation for tasks fails or if the scheduler structures are not correctly initialized. This typically happens if the function is called before memory initialization or with an invalid mode.

Error/Edge case conf:0.90 **Q:** What are common mistakes when calling ca_scheduler_init?
A: Common mistakes include calling ca_scheduler_init after creating tasks, calling it from an ISR, or passing an invalid mode value. It must also be called only once and after memory initialization (e.g., ccm_init).

Behavior conf:0.90 **Q:** Is ca_scheduler_init blocking or non-blocking, and is it safe to call from an ISR or in a multi-threaded context?
A: ca_scheduler_init is non-blocking. It is not ISR-safe and must not be called from an ISR. It is also not thread-safe and must be called only once, before any task creation, to ensure correct scheduler setup.

ca_scheduler_is_valid_context

Header: caos/inc/scheduler.h:350
Source: caos/src/scheduler.c:2070:2072
GUARD: EN_SCHEDULER

```
bool ca_scheduler_is_valid_context(u32 thid)
```

Verifies if a given task ID (thid) corresponds to a currently active and valid task context in the scheduler.

Parametri:

Nome	Tipo	Descrizione
thid	u32	The unique task identifier to validate. Must be a value previously assigned by the OS during task creation.

Ritorna:
Returns true if the thid is valid and currently managed by the scheduler, false otherwise (e.g., if the task has been destroyed or the ID is out of range).

Note:

- This function is non-blocking and thread-safe, meaning it can be called safely from multiple tasks simultaneously. It is generally safe to call from an ISR, provided the underlying task manager structures are not being modified by the scheduler at that exact moment. The most common mistake is passing an uninitialized or stale thid that was previously used by a task that has already been terminated, which will return false even if the ID was once valid.

Uso:

- This function is typically used as a safety check before performing operations on a specific task, such as signaling, suspending, or querying memory usage. It is often paired with ca_os_thid_from_handle to verify that a handle still points to a valid, living task before attempting to interact with it. Use this to prevent 'dangling' references when managing dynamic task lifecycles in your application.

Esempio:

```
u32 my_task_id = 5; // Assume this was retrieved earlier

// Check if the task is still valid before attempting to signal it
if (ca_scheduler_is_valid_context(my_task_id)) {
    // Retrieve the handle from the ID to perform an operation
    // ca_os_handle_from_thid is used to map the ID to an internal handle
    // Assuming handle_t is the type used by the OS
    void *task_handle = ca_os_handle_from_thid(my_task_id);

    if (task_handle != NULL) {
        ca_os_signal_handle(task_handle);
    }
} else {
    // Task is no longer active, handle cleanup or logging
    ca_printf("Task %lu is no longer valid.\n", my_task_id);
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_scheduler_is_valid_context() in my embedded application?

A: You should call ca_scheduler_is_valid_context() before performing any operations that involve a thread ID, especially when the thread ID comes from an external source or is dynamically determined. This helps prevent invalid memory accesses and ensures that you are only operating on valid tasks within the scheduler.

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_scheduler_is_valid_context() with caOS types?
A: u32 thid = 2; bool is_valid = ca_scheduler_is_valid_context(thid); if (is_valid) { /* Proceed with task operations */ } else { /* Handle invalid thread ID */ }

Parameters conf:0.90 **Q:** What does the thid parameter represent, and what are valid vs invalid values?
A: The thid parameter is a unique identifier for a thread (task) in the scheduler. Valid values are integers between 0 and the maximum number of allowed tasks. Invalid values include numbers outside this range or thread IDs corresponding to terminated or non-existent tasks.

Return conf:0.95 **Q:** What do the return values of ca_scheduler_is_valid_context() indicate, and what causes it to return false?
A: The function returns true if the thread ID is valid and corresponds to an active task. It returns false if the thread ID is out of range, or if the task has already terminated or does not exist. A false return may also occur if the task manager is not properly initialized.

`ca_scheduler_manage_fault`

Header: `caos/inc/scheduler.h:446`
Source: `caos/src/scheduler.c:2634:2660`
GUARD: `EN_SCHEDULER`

```
1 void ca_scheduler_manage_fault(void)
```

Handles task faults by either restarting or removing the faulty task, and optionally calling an emergency handler.

Ritorna:
This function does not return directly; it triggers a context switch to either restart the task, remove it, or switch to the idle task. A return value is not applicable here.

Note:

- This function is NOT blocking; it performs immediate actions and then lets the SVC handler perform a context switch. It is NOT thread-safe because it modifies global state (e.g., g_current_task). It is callable from an ISR context, but only if the fault is handled in a way that does not cause further issues. It assumes that a valid task context exists or that the system is in a state where a fault can be handled. Common mistakes include calling this function without ensuring that the fault context is valid or not handling the case where no task is active.

Uso:

- This function is used in fault handlers (like HardFault, MemManage, BusFault) to manage task-level faults. It is typically called after a fault is detected and before a context switch is performed. It pairs with functions like ca_os_remove_job() and SVC handlers to manage task recovery or removal. It is used in conjunction with fault type flags (e.g., CA_FAULT_RESTART_THREAD) to determine behavior.

Esempio:

```c
// Example usage in a fault handler
void HardFault_Handler(void) {
    // Clear fault status bits
    SCB->CFSR = cfsr & 0xFF;

    // Check if fault occurred in handler mode
    if (fault_in_handler_mode(exc_return)) return 0;

    // Manage the fault by either restarting or removing the task
    ca_scheduler_manage_fault();

    // Context switch is handled by SVC_Handler
    // This function does not return
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_scheduler_manage_fault() and what is its purpose in the context of task management?
A: You should call ca_scheduler_manage_fault() when a fault has been detected in a task. Its purpose is to manage the fault by deciding whether to restart or remove the task, and it may trigger an emergency if necessary. It is a system-level function that modifies the scheduler's state and can cause a direct context switch via SVC_Handler.

Code example conf:0.85 **Q:** Can you provide a minimal C code example showing how to use ca_scheduler_manage_fault() with caOS types?
A: void example_task(void) { // Simulate a fault condition if (some_error_condition) { ca_scheduler_manage_fault(); } // Task continues here if fault is of type CA_FAULT_CHECK }

Error/Edge case conf:0.90 **Q:** What are the common mistakes or edge cases when using ca_scheduler_manage_
A: A common mistake is calling this function from an ISR, which is explicitly forbidden. It must only be called when a fault is detected and the task's context is available. Another edge case is improper handling of the return value; if the fault is not of type CA_FAULT_CHECK, the function does not return to the caller, potentially causing unexpected behavior or crashes if not handled correctly.

Behavior conf:0.95 **Q:** Is ca_scheduler_manage_fault() blocking or non-blocking, and is it safe to call from an ISR or in a multi-threaded environment?
A: ca_scheduler_manage_fault() is a blocking function and must not be called from an ISR. It is also not thread-safe. It modifies the scheduler's state and can cause a direct context switch through SVC_Handler, making it unsuitable for interrupt contexts or concurrent access without external synchronization.

`ca_scheduler_sched_info`

Header: `caos/inc/scheduler.h:438`
Source: `caos/src/scheduler.c:2508:2586`
GUARD: `EN_SCHEDULER`

```c
void ca_scheduler_sched_info(void)
```

Prints scheduler performance statistics including CPU usage, task switching, and idle time.

Ritorna:
None. This function only prints debug information to stdout.

Note:

- This function is NOT blocking. It is NOT thread-safe. It CAN be called from an ISR. It requires a valid scheduler context and must be called periodically to provide meaningful statistics. Common mistake: calling it too frequently or without a proper scheduler initialization can lead to incorrect or misleading output.

Uso:

- Used in CLI (command-line interface) for debugging and monitoring system performance. Typically called in response to a user command like 'sched' to display real-time CPU usage and scheduler behavior. Pairs with functions like ca_scheduler_update_all_stack_usage() and ca_top() for system diagnostics.

Esempio:

```
/* Example usage in a CLI command handler */
void handle_scheduler_command(void) {
    // This function is called when user types 'sched' in CLI
    ca_scheduler_sched_info();  // Prints CPU usage stats
    // No return value needed, output goes to stdout
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_scheduler_sched_info in my caOS application?
A: You should call this function when you need to generate system performance reports, such as CPU load, task execution time, or scheduler overhead. It is typically used during debugging to identify performance bottlenecks or to verify that tasks are meeting their timing requirements.

Code example conf:0.90 **Q:** How do I implement a simple diagnostic command using ca_scheduler_sched_info?
A: You can call it within a CLI command handler: void my_diag_cmd(void) { ca_scheduler_sched_info ca_scheduler_dump_all_tasks(); }

Error/Edge case conf:1.00 **Q:** Why does my first call to ca_scheduler_sched_info return zero or invalid delta statistics?
A: The function relies on internal static variables to track time deltas between calls. Consequently, the first call after system boot does not have a previous reference point and will not produce meaningful delta statistics.

Behavior conf:1.00 **Q:** Is it safe to call ca_scheduler_sched_info from an interrupt service routine (ISR)?
A: No, it is not ISR-safe. You must not call it from an ISR because the function performs computationally expensive calculations and relies on system tick timing that may be inconsistent during interrupt processing. Additionally, it is not thread-safe and should only be called when the scheduler state is stable.

`ca_scheduler_set_eff_priority`

Header: `caos/inc/scheduler.h:297`
Source: `caos/src/scheduler.c:899:908`
GUARD: `EN_SCHEDULER`

```
void ca_scheduler_set_eff_priority(ca_thread_context_t *t, u16 prio)
```

Sets the effective priority of a thread, used for priority inheritance to prevent priority inversion.

Parametri:

Nome	Tipo	Descrizione
t	ca_thread_context_t *	Pointer to the thread context whose priority is being changed. Must not be NULL, otherwise behavior is undefined.
prio	u16	New effective priority value. Must be > 0, otherwise the function returns early without changing anything. Cannot be equal to current priority, otherwise no action is taken.

Ritorna:
None. The function does not return any value.

Note:

- This function is NOT blocking and is NOT thread-safe. It must be called from a context where interrupts are masked (as noted in the comment). It is typically called from within an ISR or a critical section. It modifies internal scheduler state including vtime and heap position, so calling it incorrectly can lead to scheduler inconsistencies or starvation. It is not safe to call from an ISR unless the caller ensures proper context and locking.

Uso:

- Used in priority inheritance scenarios, such as when a thread is waiting on a resource held by another thread. It is typically called from within the pi_update() function to propagate priority changes up a chain of threads. It is often paired with functions like pi_effective() and pi_owner() to manage priority inheritance chains.

Esempio:

```c
/* Example usage in a priority inheritance update */
static void pi_update(ca_thread_context_t *t) {
    for (u32 hop = 0U; t != NULL && hop <= CA_PI_MAX_CHAIN; hop++) {
        const u16 p = pi_effective(t);  /* Get the effective priority */
        if (p == t->priority) return;   /* No change needed */
        ca_scheduler_set_eff_priority(t, p);  /* Update the effective priority
    */
        if (t->pi_wait == NULL) return; /* No waiter to process */
        t = pi_owner(t->pi_wait);       /* Move to next thread in chain */
    }
}
```

Q&A — fine-tuning review

3/3 pairs · gemini-3.1-flash-lite

Usage conf:1.00 **Q:** When should I manually call ca_scheduler_set_eff_priority in my application code?
A: Developers should rarely call this function directly. It is primarily used by the kernel's internal synchronization mechanisms, such as Priority Inheritance protocols, to dynamically adjust task importance. You should rely on higher-level synchronization primitives like mutexes that trigger priority inheritance automatically.

Code example conf:0.90 **Q:** How do I safely update a thread's priority using ca_scheduler_set_eff_priority?
A: Because the function is not thread-safe, you must mask interrupts before calling it to ensure atomicity. Example: { u16 new_prio = 10; ca_irq_mask(); ca_scheduler_set_eff_priority(threa new_prio); ca_irq_unmask(); }

Parameters conf:1.00 **Q:** What are the constraints on the 'prio' parameter for ca_scheduler_set_eff_priority?
A: The 'prio' parameter must be a non-zero u16 value. These values are typically defined by system constants. Providing a zero value is invalid as the function requires a valid priority level to correctly rescale the thread's virtual time.

ca_scheduler_set_next_wait

Header: caos/inc/scheduler.h:226
GUARD: EN_SCHEDULER

```
static inline void ca_scheduler_set_next_wait(ca_thread_context_t *task,
    ca_thread_context_t *next)
```

Sets the next task in a wait queue for a given task context, used to manage task scheduling and queue linking.

Parametri:

Nome	Tipo	Descrizione
task	ca_thread_context_t *	Pointer to the current task context whose next pointer will be set. Must be a valid pointer to a task context, or NULL (invalid).
next	ca_thread_context_t *	Pointer to the next task in the queue. Can be a valid task context pointer or NULL (indicating end of queue).

Ritorna:
None. This function does not return any value. It modifies the task context directly.

Note:

- This function is NOT blocking, and it is NOT thread-safe. It should NOT be called from an ISR. It is used to link tasks in a queue structure, and improper use (e.g. setting a NULL task as next when it's not the end of the queue) can break the queue structure and cause scheduling issues or crashes. It assumes that the task context is valid and properly initialized.

Uso:

- Used in task scheduling and queue management, typically when inserting or removing tasks from a wait queue. It pairs with functions like ca_scheduler_get_next_wait() to traverse or modify the queue. Commonly used in event handling and synchronization code where tasks are linked together in a list.

Esempio:

```
/* Example: Inserting a task into a wait queue */
ca_thread_context_t *new_task = get_next_ready_task();
ca_thread_context_t *current_tail = wait_queue.tail;

/* Set the new task as the next in the current tail */
ca_scheduler_set_next_wait(current_tail, new_task);

/* Update the tail of the queue */
wait_queue.tail = new_task;

/* Ensure the new task points to NULL (end of queue) */
ca_scheduler_set_next_wait(new_task, NULL);
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_scheduler_set_next_wait in my custom synchronization primitives?
A: You should use this function when you need to manage a linked list of tasks blocked on a resource, such as a semaphore, mutex, or event queue. It is used to append a task to a wait queue when it needs to wait for an event, or to remove a task from the queue by updating pointers when the event occurs. You must ensure the operation is protected

by a critical section using ca_enter_critical and ca_exit_critical, as the function is not thread-safe.

Code example conf:0.90 **Q:** Provide a minimal C example showing how to link a task into a wait queue using caOS types.
A: ca_thread_context_t *current_task = get_current_task(); caThreadContext *next_task = get_next_queued_task();
ca_enter_critical(); ca_scheduler_set_next_wait(current_task, next_task); ca_exit_critical();

Parameters conf:1.00 **Q:** What are the requirements for the 'next' parameter in ca_scheduler_set_next_wait?
A: The 'next' parameter must be a pointer to the next ca_thread_context_t in the queue. If you want to terminate the list, you must pass NULL. Passing an invalid or uninitialized pointer will lead to system crashes or deadlocks, as the function directly modifies the internal linked list structure of the scheduler.

`ca_scheduler_signal_wake_up`

Header: `caos/inc/scheduler.h:275`
Source: `caos/src/scheduler.c:2036:2038`
GUARD: `EN_SCHEDULER`

```
void ca_scheduler_signal_wake_up(u32 thid)
```

Wakes up a blocked task identified by thid, allowing it to continue execution

Parametri:

Nome	Tipo	Descrizione
`thid`	`u32`	Task ID of the thread to wake up. Must be a valid task ID returned by ca_task_create() or similar. Invalid values (0, or ID of non-existent task) will cause undefined behavior or no effect

Ritorna:
This function does not return a value (void). However, if the underlying ca_task_manager_signal_wake_up() function fails internally, it may return an error code that is ignored by this wrapper

Note:

- This function is NOT blocking - it returns immediately. It is thread-safe and can be called from both regular threads and ISRs. The task must already be in a blocked state (waiting for a signal) for this function to have any effect. Common mistake: calling this on a task that is not blocked will have no effect, and calling it with an invalid thid can cause system instability

Uso:

- Used to unblock tasks that were previously blocked by ca_os_block() or similar blocking operations. Typically called from ISRs or other tasks when a specific condition is met that allows a waiting task to proceed. Pairs with ca_os_block() to implement signal-based task synchronization

Esempio:

```
/* In an ISR that handles a hardware interrupt */
void USART_IRQHandler(void) {
    // Process received data
    // ... data processing code ...

    // Wake up the task that was waiting for this data
```

```
    ca_scheduler_signal_wake_up(THREAD_ID_DATA_PROCESSOR); // Wake up the data processing task
    // Note: THREAD_ID_DATA_PROCESSOR must be a valid task ID that was previously blocked
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_scheduler_signal_wake_up() and what is its purpose in the context of task management?
A: You should call ca_scheduler_signal_wake_up() to unblock a task that is currently waiting for a signal. This function allows the scheduler to resume execution of the specified task. It is typically used in scenarios involving inter-task communication or event handling, such as when a task is waiting on a queue, event, or notification and needs to be woken up by another task or interrupt service routine.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_scheduler_signal_wake_up() with caOS types?
A: u32 task_id = 2; ca_scheduler_signal_wake_up(task_id);

Parameters conf:0.90 **Q:** What is the significance of the 'thid' parameter in ca_scheduler_signal_wake_up(), and what are the valid ranges for this value?
A: The 'thid' parameter represents the unique identifier of the task to be woken up. It must be a valid value between 0 and the maximum number of tasks configured in the system. Passing an invalid value, such as one greater than the maximum number of tasks or the maximum value of u32, may lead to memory access violations or unpredictable behavior.

`ca_scheduler_size`

Header: `caos/inc/scheduler.h:343`
Source: `caos/src/scheduler.c:2066:2068`
GUARD: `EN_SCHEDULER`

```
u32 ca_scheduler_size(void)
```

Returns the number of tasks currently managed by the scheduler, or 0xFFFFFFFF if the scheduler is not initialized.

Ritorna:
`0xFFFFFFFF`: `0 to N`:

Note:

- This function is NOT blocking — it returns immediately.
- It is safe to call from both threads and ISRs, but only if the scheduler is initialized.
- It does NOT require any special preconditions beyond the scheduler being initialized.
- A return value of 0xFFFFFFFF is a clear indicator that the scheduler is not active or has not been set up yet.
- This function is typically used to iterate over all tasks in the system, such as during stack overflow detection or task inspection.

Uso:

- Used in stack overflow detection logic to determine how many tasks to check.
- Used in debugging or fault handling routines to enumerate all active tasks.
- Called before iterating through tasks using ‘ca_scheduler_get_context_from_idx()‘ to avoid out-of-bounds access.

Esempio:

```
// This example shows how to safely retrieve the number of tasks and iterate through them. It checks for a NULL scheduler and handles each task context accordingly.
u32 task_count = ca_scheduler_size();
if (task_count == 0xFFFFFFFF) {
    // Scheduler not initialized
    ca_stdout_printf_n("Scheduler not initialized\n");
    return;
}
for (u32 i = 0; i < task_count; i++) {
    ca_thread_context_t *ctx = (ca_thread_context_t *)ca_scheduler_get_context_from_idx(i);
    if (ctx != NULL) {
        ca_stdout_printf_n("Task %u: %s\n", i, ctx->name);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_scheduler_size() and why would I need to know the total number of tasks?
A: You should call ca_scheduler_size() to validate task IDs and understand the size of the multitasking system. It's useful for checking if the scheduler has been initialized correctly and to ensure that task IDs are within valid range. This function is particularly helpful when dynamically allocating or managing tasks in your embedded system.

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_scheduler_size() with caOS types?
A: u32 task_count = ca_scheduler_size(); if (task_count != 0xFFFFFFFF) { // Valid count, proceed with task management // Example: use task_count to allocate memory or validate IDs } else { // Initialization error or internal problem }

Parameters conf:0.95 **Q:** Does ca_scheduler_size() have any parameters, and what are the valid or invalid values for them?
A: The function does not require any parameters. As it is a query function that reads a global variable, there are no input values to consider. The function is designed to be called without arguments.

Return conf:0.90 **Q:** What do the return values of ca_scheduler_size() mean, and what causes the function to return 0xFFFFFFFF?
A: The function returns a u32 value: if > 0, it indicates the total number of tasks managed by the scheduler; if 0xFFFFFFFF, it signifies an error. The error occurs when the system is not initialized correctly with ca_scheduler_init(), or when there is an internal problem within the scheduler.

`ca_scheduler_tasks_info`

Header: `caos/inc/scheduler.h:435`
Source: `caos/src/scheduler.c:2315:2381`
GUARD: `EN_SCHEDULER`

```
void ca_scheduler_tasks_info(ca_scheduler_tasks_info_mode mode)
```

Prints detailed information about all tasks in the scheduler, including CPU usage, stack usage, and memory stats.

Parametri:

Nome	Tipo	Descrizione
mode	ca_scheduler_tasks_info_mode	Specifies the level of detail to print. Valid values: CA_TH_INFO_DEFAULT (basic info), CA_TH_INFO_MEM (memory usage), CA_TH_INFO_FLAG (task flags), TH_INFO_ASSERT (assert/restart/fault counts), TH_INFO_RUNTIME (runtime stats). Invalid values may cause undefined behavior or no output.

Ritorna:
None. This function does not return a value. It directly prints output to stdout.

Note:

- This function is NOT blocking in the traditional sense, but it does call ca_os_sleep(2) which will pause execution for a short time. It is NOT thread-safe and should not be called from an ISR. It requires a valid task list and snapshot data to function correctly. Common mistake: calling this function without initializing the scheduler or before tasks have run will result in inaccurate or empty output.

Uso:

- Used for debugging and monitoring system performance. Typically called periodically in a debug task or from a command-line interface. Pairs with ca_os_task_snapshot() to get task data. Should be used in a non-ISR context, usually in a dedicated debug task or during system initialization for diagnostics.

Esempio:

```c
/* Example: Print basic task info every 5 seconds in a debug task */
void debug_task(void *arg) {
    while(1) {
        ca_scheduler_tasks_info(CA_TH_INFO_DEFAULT);  // Print basic task info
        ca_os_sleep(5000);  // Wait 5 seconds before next update
    }
}

/* Example: Print memory usage info every 10 seconds */
void mem_debug_task(void *arg) {
    while(1) {
        ca_scheduler_tasks_info(CA_TH_INFO_MEM);  // Print memory usage
        ca_os_sleep(10000);  // Wait 10 seconds
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_scheduler_tasks_info() and for what purpose?
A: You should call ca_scheduler_tasks_info() during debugging or system monitoring to get detailed information about tasks, such as CPU usage, stack usage, memory allocation, and runtime statistics. It is useful for identifying performance bottlenecks or misconfigured tasks. The function is intended for use in non-ISR contexts where blocking is acceptable.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_scheduler_tasks_info() with caOS types?
A:

```c
#include "caos.h"

void example_task(void) {
    ca_scheduler_tasks_info(CA_TH_INFO_DEFAULT);
```

```
    ca_os_sleep(100);
}
```

Parameters conf:0.90 **Q:** What does the mode parameter in ca_scheduler_tasks_info() control, and what are the valid values?
A: The mode parameter controls the type of information printed about tasks. Valid values are: CA_TH_INFO_DEFAULT (basic info), CA_TH_INFO_MEM (memory usage), CA_TH_INFO_FLAG (counters), TH_INFO_ASSERT (errors), and TH_INFO_RUNTIME (execution time). These are defined as constants in the enum ca_scheduler_tasks_info_mode.

Error/Edge case conf:0.85 **Q:** What are the edge cases or error conditions when calling ca_scheduler_tasks_inf
A: The function does not fail but may print warnings or errors if it encounters invalid task digests. If a task is invalid, it prints a message and continues execution. It is not thread-safe and should not be called from an ISR. If DWT is not enabled, CPU usage calculations will be incorrect.

Behavior conf:0.90 **Q:** Is ca_scheduler_tasks_info() blocking or non-blocking, and is it safe to call from an ISR or in a multi-threaded context?
A: The function is blocking, as it uses ca_os_sleep(2). It is not ISR-safe and must not be called from an ISR. It is also not thread-safe, as it modifies global variables that affect CPU usage calculations. It should only be called in contexts where blocking and exclusive access are acceptable.

`ca_scheduler_tick`

Header: `caos/inc/scheduler.h:304`
Source: `caos/src/scheduler.c:834:849`
GUARD: `EN_SCHEDULER`

```
void ca_scheduler_tick(void)
```

Called by the SysTick interrupt to manage task scheduling and time slicing.

Ritorna:
None. This function does not return a value.

Note:

- This function is non-blocking and must be called from the SysTick ISR context. It is not thread-safe because it modifies global state. It is designed to be called only when the OS is running (ca_os_running() returns true). A common mistake is calling this function outside of the SysTick ISR or when the OS is not initialized. The function may trigger a PendSV exception to switch tasks if the time slice has expired.

Uso:

- This function is used exclusively within the SysTick interrupt handler to manage task switching and time slicing. It is typically called once per tick when the OS is running. It pairs with ca_os_tick() to track time and with SCB->ICSR to trigger PendSV for context switching. It is part of the core RTOS scheduling mechanism.

Esempio:

```
/* In SysTick ISR handler */
void SysTick_Handler(void) {
    /* Only process scheduling if OS is running */
    if (ca_os_running()) {
        /* Call scheduler tick to handle time slicing and task switching */
        ca_scheduler_tick();
    }
```

```
    /* SysTick interrupt flag is automatically cleared by hardware */
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_scheduler_tick in my caOS application?
A: You should call ca_scheduler_tick exclusively inside the hardware SysTick_Handler ISR. It is the heartbeat of the multitasking system, used to evaluate task switching and ensure tasks are preempted when their time slice expires.

Code example conf:0.90 **Q:** How do I implement the SysTick_Handler to trigger the scheduler?
A: You should invoke the function within the ISR as follows: void SysTick_Handler(void) { ca_scheduler_tick(); }

Error/Edge case conf:1.00 **Q:** What happens if I call ca_scheduler_tick from a standard task instead of the ISR?
A: Calling this function from a standard task is a common mistake that will corrupt the scheduler's internal state, likely causing a hard fault or a system hang because it manipulates low-level CPU registers like the ICSR to trigger a PendSV exception.

Behavior conf:1.00 **Q:** Is ca_scheduler_tick thread-safe or safe to call from any context?
A: No, it is not thread-safe. It must never be called from a standard task context. It is designed specifically for the SysTick ISR to handle time slice logic and trigger the PendSV mechanism for context switching.

`ca_scheduler_update_all_stack_usage`

Header: `caos/inc/scheduler.h:459`
Source: `caos/src/scheduler.c:1696:1737`
GUARD: `EN_SCHEDULER`

```
void ca_scheduler_update_all_stack_usage(void)
```

Updates stack usage percentage for all tasks in the scheduler to monitor for potential stack overflows.

Ritorna:
None. This function does not return any value.

Note:

- This function is NOT blocking and can be called from any context. It is thread-safe because it uses IRQ masking to protect access to task data structures. It is NOT safe to call from an ISR because it uses functions that may block or require a scheduler context. It must be called in a context where task list access is valid and the scheduler is running. Common mistake: calling this function too frequently can impact performance; it's typically called during idle time or periodic checks.

Uso:

- Used periodically in idle task context to monitor stack usage across all tasks. It's typically called in the main idle loop or during periodic system checks to detect potential stack overflows before they occur. Pairs with functions like ca_os_tick() for timing and ca_stdout_printf_n() for logging warnings.

Esempio:

```
/* Example usage in main idle loop */
void main_idle_task(void) {
    while (1) {
        /* Do other work */
```

```
5
6           /* Periodically check stack usage */
7           ca_scheduler_update_all_stack_usage();
8
9           /* Yield or sleep */
10          ca_os_sleep(100); /* Sleep for 100 ticks */
11      }
12  }
13
14  /* Example usage in CLI command handler */
15  void cli_command_handler(void) {
16      /* ... other command processing ... */
17      if (ca_strncmp(&cmd->data[index], "stack", 5) == 0) {
18          ca_scheduler_update_all_stack_usage(); /* Update stack usage before
        showing info */
19          ca_top(mode); /* Display task info including updated stack usage */
20      }
21      /* ... rest of command processing ... */
22  }
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_scheduler_update_all_stack_usage() and why is it useful?
A: You should call ca_scheduler_update_all_stack_usage() to monitor stack usage across all active tasks in your caOS system. This function updates internal fields tracking stack utilization, which is useful for debugging memory issues or ensuring tasks do not overflow their allocated stack space. It is particularly helpful during development or system tuning when you want to identify tasks that may be using excessive stack memory.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_scheduler_update_all_stack_us with caOS types?
A: void example_function(void) { ca_scheduler_update_all_stack_usage(); // Stack usage for all tasks has been updated }

Error/Edge case conf:0.85 **Q:** What are the common mistakes or edge cases when using ca_scheduler_update_
A: A common mistake is calling this function from within an ISR or while a task is actively running, as it is not thread-safe and not ISR-safe. Additionally, calling it during task execution or in a multitasking context can lead to unpredictable behavior. The function also ignores tasks with invalid checksums (digest), so if a task's integrity is compromised, it will be skipped without warning.

Behavior conf:0.90 **Q:** Is ca_scheduler_update_all_stack_usage() blocking or non-blocking, and is it safe to call from an ISR or in a multitasking environment?
A: The function is blocking and not thread-safe. It must not be called from within an ISR or during task execution. It is designed to be called from a non-ISR context, and its behavior is undefined if invoked while other tasks are running or during interrupt handling. It performs internal checks for task validity using a checksum, but does not provide any synchronization mechanisms.

`ca_scheduler_wake_waiter`

Header: `caos/inc/scheduler.h:291`
Source: `caos/src/scheduler.c:2001:2015`
GUARD: `EN_SCHEDULER`

```
1  bool ca_scheduler_wake_waiter(ca_thread_context_t *ctx)
```

Wakes up a thread that is blocked or sleeping, allowing it to resume execution.

Parametri:

Nome	Tipo	Descrizione
`ctx`	`ca_thread_context_t *`	Pointer to the thread context to wake. Must point to a valid thread structure, or NULL. If NULL, the function returns false.

Ritorna:
`true`: The thread was successfully woken up. It was either blocked or sleeping and is now marked as running. `false`: The thread could not be woken. This happens if ctx is NULL, or if the thread is in a special status like removal, stack overflow, digest error, or watchdog signal.

Note:

- **Blocking behavior**: This function does not block. It immediately processes the wake-up request and returns.
- **Thread safety**: The function disables interrupts during execution to ensure atomicity. It is safe to call from both threads and ISRs, but calling it from an ISR requires careful handling of the interrupt state.
- **ISR usage**: Can be safely called from an ISR, but the caller must ensure that the interrupt state is managed correctly (e.g., using ___disable_irq() and ___set_PRIMASK()).
- **Preconditions**: The thread context pointed to by ctx must be valid and not in a special termination status (e.g., CA_ST_TH_REMOVE, CA_ST_TH_STACK_OVERFLOW, etc.).
- **Common mistakes**: Calling with a NULL pointer will return false without causing a crash. Calling with a thread in a special status (like stack overflow) will also return false, which may be silently ignored.

Uso:

- **Event handling**: Used to wake a thread waiting for an event. The thread is woken only if it is in a BLOCKED or SLEEP state.
- **Notification system**: Wakes a thread waiting for a notification. Used in conjunction with ca_wait_queue_remove() to remove the thread from the wait queue before waking it.
- **Queue operations**: Used to wake a thread waiting on a queue. Called from queue_wake_one() to wake one waiting thread and break out of a loop.
- **Scheduler wake-up**: Used to wake a thread by its handle. Ensures that the thread is valid and not in a special status before waking it.

Esempio:

```
// Example of waking a thread in a normal context. Checks return value to ensure the thread was successfully woken.
ca_thread_context_t *waiter = get_waiting_thread();
if (ca_scheduler_wake_waiter(waiter)) {
    // Thread was woken up and will resume execution
    // Do any post-wake-up processing here
} else {
    // Thread was not woken (e.g., already terminated or invalid context)
    // Handle error or log the event
}
// Example from notify.c: wakes a thread from a wait queue after removing it. Ensures thread is valid before waking.
ca_thread_context_t *ctx = ca_wait_queue_dequeue(&g_notify_wq);
if (ctx != NULL) {
    if (ca_scheduler_wake_waiter(ctx)) {
        // Thread woken successfully
        // Proceed with notification handling
```

```
    }
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_scheduler_wake_waiter in my application?
A: You should use this function when you need to transition a task from a blocked or sleeping state back to the ready-to-run state. It is typically used by synchronization primitives like queues, event flags, or notification systems after removing a task from a wait list to signal that the data or event it was waiting for is now available.

Code example conf:0.90 **Q:** How do I implement a simple wake-up call using caOS types?
A: Assuming you have a pointer to a thread context, you would call it as follows: bool success = ca_scheduler_wake_waiter(my_thread_ctx); if (success) { /* Task is now ready */ }

Parameters conf:1.00 **Q:** What are the requirements for the ca_thread_context_t pointer passed to ca_scheduler_wake_waiter?
A: The pointer must point to a valid, initialized task context. If you pass NULL or a context that has been marked with error flags, such as a stack overflow, the function will perform no action.

Return conf:1.00 **Q:** What do the boolean return values of ca_scheduler_wake_waiter indicate?
A: A return value of true indicates the task was successfully transitioned to the ready-to-run state. A return value of false indicates the operation failed, which occurs if the task was already running, had error flags set, or if the provided pointer was invalid.

Error/Edge case conf:1.00 **Q:** Does calling ca_scheduler_wake_waiter guarantee an immediate context switch to the target task?
A: No, a common mistake is assuming this function triggers an immediate switch. It only updates the task status to 'ready'. The actual context switch will occur during the next scheduler tick or when the current task yields.

`ca_scheduler_wake_waiter_handle`

Header: `caos/inc/scheduler.h:308`
Source: `caos/src/scheduler.c:2017:2026`
GUARD: `EN_SCHEDULER`

```
bool ca_scheduler_wake_waiter_handle(ca_task_handle_t h)
```

Wakes a task that is currently blocked waiting for a resource, using its task handle.

Parametri:

Nome	Tipo	Descrizione
`h`	`ca_task_handle_t`	The handle of the task to wake. Must be a valid handle previously obtained from the system; using CA_TASK_HANDLE_NONE or an invalid handle will result in a false return.

Ritorna:
Returns true if the task was successfully found and transitioned from a waiting state to ready. Returns false if the handle is invalid, the task is not in a waiting state, or the task does not exist.

Note:

- This function is thread-safe and internally manages critical sections by disabling interrupts to ensure the task list remains consistent during the lookup and state transition. It is not

intended to be called directly from an ISR; use the appropriate ISR-specific variants if available in the specific module context. The most common beginner mistake is attempting to wake a task that is not actually in a 'waiting' state, or using a stale handle that has already been recycled by the system after a task termination. Because this function modifies the scheduler's task list, it must be used carefully to avoid race conditions where a task might be woken just as it is being deleted or moved.

Uso:

- This function is typically used in synchronization primitives like message queues, streams, or custom semaphores where a resource becomes available and a blocked consumer needs to be notified. It is usually called while holding a spinlock to ensure that the state of the resource (e.g., 'waiting_task') and the task's status remain synchronized. After calling this, the code typically releases the lock, allowing the scheduler to immediately switch to the newly awakened task if its priority is higher than the current one. It is the standard way to resolve a 'waiter' when the resource owner determines the condition for the waiter has been met.

Esempio:

```c
/* Example: Waking a task waiting for data in a custom buffer */
void notify_consumer(ca_task_handle_t waiter_handle) {
    /* Ensure we have a valid handle before attempting to wake */
    if (waiter_handle != CA_TASK_HANDLE_NONE) {
        /* Attempt to wake the task. The function handles the internal
           scheduler state transition from BLOCKED to READY. */
        bool success = ca_scheduler_wake_waiter_handle(waiter_handle);

        if (!success) {
            /* Handle error: task might have already been woken or terminated */
            ca_printf("Failed to wake task handle: %d\n", (int)waiter_handle);
        }
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_scheduler_wake_waiter_handle in my application?
A: You should use this function when implementing synchronization primitives like message queues, streams, or custom semaphores. It is intended to notify a consumer task that a resource has become available, transitioning that task from a waiting state to a ready state.

Code example conf:0.90 **Q:** How do I use ca_scheduler_wake_waiter_handle in a C snippet?
A: Assuming you have a task handle stored in a variable, you would call it as follows: bool success = ca_scheduler_wake_waiter_handle(my_task_handle); if (success) { /* Task is now ready */ }

Parameters conf:1.00 **Q:** What are the requirements for the ca_task_handle_t parameter?
A: The parameter must be a valid handle previously obtained from the system. Using CA_TASK_HANDLE_NONE or an invalid/stale handle that has been recycled by the system will cause the function to return false.

Return conf:1.00 **Q:** What does a false return value from ca_scheduler_wake_waiter_handle indicate?
A: A false return indicates that the operation failed. This occurs if the provided handle is invalid, the task does not exist, or the task is not currently in a waiting state.

ca_sdio_get_block_count

Header: caos/drivers/inc/sdio.h:24
Source: caos/drivers/src/sdio.c:137:140
GUARD: EN_SDIO

```
u32 ca_sdio_get_block_count(void)
```

Returns the total number of addressable 512-byte blocks available on the SD card.

Ritorna:
Returns a u32 representing the total block count. Returns 0 if the SD card is not initialized, not detected, or if a hardware error occurred during the card identification process.

Note:

- This function is non-blocking and safe to call from most contexts, but it relies on the SD card being successfully initialized via ca_sdio_init first. It is not intended to be called from an ISR because SDIO communication typically requires task-level synchronization. A common beginner mistake is to assume the card is ready immediately after boot without checking the return value of ca_sdio_init or assuming the block count is constant across different SD cards; always treat the returned value as dynamic based on the inserted hardware.

Uso:

- This function is typically used during the system startup phase or when mounting a filesystem to determine the storage capacity. It is often paired with ca_sdio_init to verify hardware presence and then used to perform bounds checking before calling ca_sdio_read_block or ca_sdio_write_blocks to prevent memory corruption or invalid hardware access. Always validate the block count against your target LBA (Logical Block Address) before initiating any read or write operations to ensure you stay within the physical limits of the card.

Esempio:

```
if (ca_sdio_init()) {
    u32 total_blocks = ca_sdio_get_block_count();
    if (total_blocks > 0) {
        ca_stdout_printf_n("SD card detected with %lu blocks", (unsigned long)
    total_blocks);
        /* Example: Check if block 100 is within bounds before writing */
        if (100 < total_blocks) {
            /* Perform write operation using ca_sdio_write_blocks */
        }
    } else {
        ca_stdout_put_n("SD card is empty or unreadable");
    }
} else {
    ca_stdout_put_n("Failed to initialize SDIO interface");
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_sdio_get_block_count() and why is it important for SD card operations?
A: You should call ca_sdio_get_block_count() after initializing the SDIO driver to determine the total number of blocks available on the SD card. This information is essential for calculating the card's total capacity and for managing data storage operations. It must be called only after the SDIO hardware is properly initialized and is not thread-safe or ISR-compatible.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_sdio_get_block_count() with caOS types?
A: u32 block_count; block_count = ca_sdio_get_block_count(); if (block_count == 0) { // Handle error: SD card not initialized or not responding } else { // Use block_count to calculate total capacity }

Return conf:0.90 **Q:** What do the return values of ca_sdio_get_block_count() indicate, and what causes the function to return 0?
A: The function returns a u32 representing the total number of blocks on the SD card. If it returns 0, it indicates a failure, typically caused by the SDIO driver not being initialized correctly or calling the function before the hardware is ready. The function is not blocking but requires proper initialization before use.

`ca_sdio_hw_get_block_count`

Header: `caos/drivers/inc/ca_glue_hw.h:238`
GUARD: `EN_CA_GLUE_HW`

```c
extern u32 ca_sdio_hw_get_block_count(void); /* returns the total number of blocks of the SD card */
```

Returns the total number of addressable blocks available on the SD card.

Ritorna:
Returns a u32 representing the total block count. Returns 0 if the SD card is not initialized, not detected, or if the hardware driver does not support this operation.

Note:

- This function is non-blocking and thread-safe, as it typically reads a cached value populated during the SD card initialization process. It is safe to call from an ISR, provided the SD card driver has already been successfully initialized. A common mistake is calling this function before ca_sdio_init has completed, which will result in a return value of 0, leading to potential out-of-bounds errors if used to calculate memory offsets for read/write operations.

Uso:

- This function is primarily used to determine the storage capacity of the SD card before performing file system operations or raw data logging. It is typically called after ca_sdio_init to verify that the card is ready and to calculate the maximum addressable range. Developers should pair this with ca_sdio_read_block or ca_sdio_write_block to ensure that any block index passed to those functions is within the valid range [0, count-1].

Esempio:

```c
u32 total_blocks;

/* Initialize the SDIO peripheral before querying capacity */
if (ca_sdio_init()) {
    total_blocks = ca_sdio_get_block_count();

    if (total_blocks > 0) {
        /* Safe to proceed with block operations */
        /* Example: Read the very last block of the card */
        /* Note: Ensure a 512-byte buffer is allocated for the block */
        u8 *buffer = ca_memory_allocate(512);
        if (buffer != NULL) {
            if (ca_sdio_read_block(total_blocks - 1, buffer)) {
```

```
                /* Process data */
            }
            ca_memory_free(buffer);
        }
    } else {
        /* Handle error: Card detected but has no blocks or is unformatted */
    }
} else {
    /* Handle error: SD card initialization failed */
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_sdio_hw_get_block_count() in my SD card driver implementation?
A: You should call ca_sdio_hw_get_block_count() after successfully initializing the SDIO hardware with ca_sdio_hw_init(). This function is typically used to determine the total capacity of the SD card by retrieving the number of blocks available. It's useful during system startup or when detecting a newly inserted SD card to know how much storage space is accessible.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_sdio_hw_get_block_count() with caOS types?
A: u32 block_count = ca_sdio_hw_get_block_count(); if (block_count == 0) { // Handle error: SD card not accessible or not initialized } else { // Use block_count to determine card size // For example, calculate total size in bytes: block_count * 512 }

Return conf:0.95 **Q:** What does it mean if ca_sdio_hw_get_block_count() returns 0, and what causes this failure?
A: A return value of 0 indicates that either the SD card is not present or accessible, or the hardware has not been initialized properly. The most common cause is calling this function before calling ca_sdio_hw_init(). It's important to note that a return value of 0 does not necessarily mean the card is absent—it could also indicate an initialization failure.

HW differences conf:0.85 **Q:** How does the implementation of ca_sdio_hw_get_block_count() differ between STM32F1xx and other families like F4xx or H7xx?
A: The STM32F1xx family returns 0 directly from the function implementation, indicating no functional support or initialization. In contrast, F4xx, F7xx, H5xx, and H7xx families return the value stored in the global variable s_block_count, which is populated during the hardware initialization process. This suggests that F1xx lacks full SDIO support or requires a different initialization path compared to the other families.

`ca_sdio_hw_init`

Header: `caos/drivers/inc/ca_glue_hw.h:228`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_sdio_hw_init(void); /* inizializza il periferico SDIO/
    SDMMC e rileva la scheda SD */
```

Initializes the SDIO/SDMMC hardware peripheral and performs the SD card identification sequence.

Ritorna:
Returns true if the SD card was successfully detected and initialized, false otherwise (e.g., card not present, communication error, or unsupported card type).

Note:

- This function is a low-level hardware driver initialization routine. It is not thread-safe and must be called during system startup before any file system or block-level read/write operations are attempted. It is strictly non-reentrant and must not be called from an ISR, as it performs blocking operations and busy-wait loops to satisfy SD card power-up and timing requirements. A common mistake is calling this function multiple times or before the system clock and GPIOs are properly configured, which will lead to initialization failure.

Uso:

- This function is typically called once during the system boot sequence or when a card insertion is detected. It is usually wrapped by a higher-level function like ca_sdio_init, which manages the internal state of the driver. After a successful return, the application can proceed to use ca_sdio_read_block or ca_sdio_write_block to interact with the storage media.

Esempio:

```c
if (ca_sdio_hw_init()) {
    /* Hardware is ready, proceed to mount file system or access blocks */
    ca_printf("SD card initialized successfully.\n");
} else {
    /* Handle hardware failure: check card seating or power supply */
    ca_printf("SD card initialization failed!\n");
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_sdio_hw_init() in my embedded application?
A: You should call ca_sdio_hw_init() during system initialization, typically after initializing the GPIO and clock peripherals but before attempting any SD card operations. This function is required to establish communication with an SD card and configure the SDIO peripheral for data transfer.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_sdio_hw_init() with caOS types?
A: bool success = ca_sdio_hw_init(); if (success) { /* SD card initialized successfully */ } else { /* Handle initialization failure */ }

Return conf:0.95 **Q:** What does ca_sdio_hw_init() return and what causes it to return false?
A: The function returns true if initialization succeeds and an SD card is detected. It returns false if any step in the initialization sequence fails, such as CMD0, CMD8, or ACMD41 commands failing, or if the card does not respond correctly during identification.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_sdio_hw_init()?
A: Common issues include calling the function before GPIO and clock initialization, not having a properly connected SD card, or hardware limitations like incorrect pull-up resistors. Additionally, some MCU families (like H7) may have specific clock configurations that must be set correctly for the function to succeed.

Behavior conf:0.90 **Q:** Is ca_sdio_hw_init() blocking or non-blocking, and is it safe to call from an ISR?
A: The function is blocking and will not return until the initialization sequence completes or times out. It is not ISR-safe and should not be called from interrupt service routines. It is also not thread-safe, so concurrent calls from multiple threads should be avoided.

HW differences conf:0.80 **Q:** How does ca_sdio_hw_init() differ between H7 and H5 MCUs in terms of hardware configuration?
A: The main differences are in clock divider settings: H5 uses CLKCR = (60U & SDMMC_CLKCR_CLKDIV) and then sets CLKCR = (1U & SDMMC_CLKCR_CLKDIV)

for 24MHz, while H7 uses CLKCR = (60U & SDMMC_CLKCR_CLKDIV) for 400kHz initial clock and then sets CLKCR = (1U & SDMMC_CLKCR_CLKDIV) for 24MHz. Both use SDMMC1 peripheral but with different initial clock configurations.

---

ca_sdio_hw_read_block

Header: caos/drivers/inc/ca_glue_hw.h:229
GUARD: EN_CA_GLUE_HW

```
extern bool ca_sdio_hw_read_block(u32 block, u8 *buf); /* legge un
   blocco da 512 byte dalla SD */
```

Reads a single 512-byte block from the SD card into a provided buffer.

Parametri:

Nome	Tipo	Descrizione
block	u32	The logical block address (LBA) to read. Must be within the valid range of the SD card.
buf	u8 *	Pointer to a memory buffer of at least 512 bytes. Must be properly aligned for DMA access on some architectures.

Ritorna:
Returns true if the block was read successfully, false if a hardware error, timeout, or CRC failure occurred.

Note:

- This function is blocking; it waits for the hardware to complete the data transfer from the SD card. It is not thread-safe by itself, meaning multiple tasks should not attempt to read from the SD card simultaneously without external synchronization (e.g., a mutex). It must NOT be called from an ISR because it performs busy-waiting loops that would stall the system. Before calling this, ensure the SD card has been initialized via ca_sdio_init. A common beginner mistake is passing an unaligned or insufficient buffer, which can lead to memory corruption or hard faults on architectures that enforce strict alignment for DMA transfers.

Uso:

- This function is the low-level workhorse for reading data from storage. It is typically used in a sequence: first, allocate a buffer using ca_memory_allocate, then call ca_sdio_hw_read_block to fill it, process the data (e.g., via ca_hexdump), and finally release the memory with ca_memory_free. It is often wrapped by higher-level file system drivers to perform sector-level I/O.

Esempio:

```
u8 *data_buf = ca_memory_allocate(512U);
if (data_buf != NULL) {
    u32 target_block = 100U;
    if (ca_sdio_hw_read_block(target_block, data_buf)) {
        /* Successfully read, display the first 64 bytes */
        ca_hexdump(data_buf, 64U);
    } else {
        /* Handle read failure */
        ca_printf_n("Failed to read block %u", target_block);
    }
    ca_memory_free(data_buf, NULL, false);
} else {
```

```
    ca_printf_n("Memory allocation failed");
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_sdio_hw_read_block() in my embedded application?
A: You should use ca_sdio_hw_read_block() when you need to read a 512-byte block of data from an SD card using the SDIO hardware interface. It is typically used in file system drivers or data logging applications where direct access to SD card blocks is required.

Code example conf:0.95 **Q:** Can you show a minimal example of how to read a block from SD card using ca_sdio_hw_read_block()?
A: u8 buffer[512]; bool success = ca_sdio_hw_read_block(0, buffer); if (success) { // Process the block data in buffer } else { // Handle read error }

Parameters conf:0.90 **Q:** What does the 'block' parameter represent and what are valid values?
A: The 'block' parameter is the 0-based index of the SD card block to read. Valid values are any positive integer. A value that is too high for the SD card capacity may cause a read error, but there is no upper limit enforced by the function itself.

Return conf:0.95 **Q:** What does it mean when ca_sdio_hw_read_block() returns false?
A: A return value of false indicates that the read operation failed. This can happen due to hardware errors, an invalid buffer pointer (NULL), an invalid block number, or if the SDIO hardware was not initialized properly.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_sdio_hw_read_block()?
A: Common mistakes include calling the function with a NULL buffer pointer, not initializing the SDIO hardware with ca_sdio_hw_init() before calling this function, or calling it from an ISR (which is not allowed). Not checking the return value can also lead to using invalid data.

Behavior conf:0.95 **Q:** Is ca_sdio_hw_read_block() thread-safe and can it be called from an ISR?
A: No, ca_sdio_hw_read_block() is not thread-safe and should not be called from an ISR. It is a blocking function that may cause deadlocks if multiple threads call it simultaneously, and it's not designed to be interruptible.

HW differences conf:0.85 **Q:** How does the implementation of ca_sdio_hw_read_block() differ between STM32F4 and STM32H5 microcontrollers?
A: The main difference is in how FIFO data is read. On STM32F4, it uses RXDAVL status flag to check for available data. On STM32H5, it uses RXFIFOHF flag because RXDAVL is not available, reading 8 words per iteration instead of 1.

`ca_sdio_hw_write_block`

Header: `caos/drivers/inc/ca_glue_hw.h:236`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_sdio_hw_write_block(u32 block, const u8 *buf); /* writes
    a 512-byte block to the SD */
```

Writes a single 512-byte data block to the SD card at the specified address.

Parametri:

Nome	Tipo	Descrizione
`block`	`u32`	The logical block address (LBA) on the SD card to write to. Must be a valid block index within the card's capacity.
`buf`	`const u8 *`	Pointer to the 512-byte buffer containing the data to be written. Must not be NULL.

Ritorna:
Returns true if the write operation completed successfully, false if the operation failed (e.g., timeout, CRC error, or invalid parameters).

Note:

- This function is a blocking call; it will wait for the hardware to complete the data transfer and the card to signal it is ready. It is not thread-safe by itself, meaning you should ensure that only one task accesses the SD card at a time, typically by using a mutex or by ensuring exclusive access in your application logic. It must not be called from an ISR because it performs long-running polling loops that would stall the system. A common mistake is passing a buffer that is not properly aligned or is smaller than 512 bytes, which can lead to memory corruption or hardware errors. Always ensure the SD card has been initialized via ca_sdio_init before calling this function, otherwise the hardware registers will not be configured and the operation will fail.

Uso:

- This function is typically used in the data-writing phase of an application, following a successful initialization via ca_sdio_init. It is often paired with higher-level file system operations or raw data logging routines. In a typical sequence, you would call ca_sdio_init to prepare the hardware, then use ca_sdio_write_block to commit data to specific sectors, and finally ensure the system is in a stable state before powering down or re-initializing. If you need to write multiple consecutive blocks, consider using ca_sdio_write_blocks for better performance.

Esempio:

```c
/* Example: Writing a block of data to the SD card */
#include "caos.h"

void write_data_to_sd(u32 block_index, const u8 *data_ptr) {
    /* Ensure the pointer is valid before attempting the write */
    if (data_ptr == NULL) {
        return;
    }

    /* Attempt to write the block. This is a blocking operation. */
    /* Note: Ensure ca_sdio_init has been called previously in the system
    startup */
    if (ca_sdio_write_block(block_index, data_ptr)) {
        /* Success: Data written to SD */
    } else {
        /* Failure: Handle error, e.g., log to system or retry */
        ca_printf("SD write failed at block %lu\n", (unsigned long)block_index);
    }
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_sdio_hw_write_block() in a caOS project?
A: You should use ca_sdio_hw_write_block() when you need to write a 512-byte block to an SD card in a bare-metal environment without HAL. It is intended for non-critical contexts where blocking is acceptable and thread safety is not a concern.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_sdio_hw_write_block()?
A:

```
u8 buffer[512];
/* Fill buffer with data */
bool success = ca_sdio_hw_write_block(0, buffer);
if (!success) {
    /* Handle error */
}
```

Parameters conf:0.90 **Q:** What are the valid ranges for the block parameter in ca_sdio_hw_write_block()?
A: The block parameter must be a valid 32-bit unsigned integer representing the SD card block number, starting from 0. It must be less than the total number of blocks available on the SD card. Passing a block number outside this range will cause the function to return false.

Return conf:0.95 **Q:** What does it mean when ca_sdio_hw_write_block() returns false?
A: A return value of false indicates that the write operation failed. This can happen due to hardware errors, an invalid block number, or a NULL buffer. The function does not distinguish between different types of failures, so additional error handling may be needed.

Error/Edge case conf:0.90 **Q:** What are common mistakes when calling ca_sdio_hw_write_block()?
A: Common mistakes include passing a NULL buffer, providing a block number outside the valid range, or not checking the return value. Additionally, calling this function in an ISR or from multiple threads simultaneously can lead to unpredictable behavior or data corruption.

Behavior conf:0.95 **Q:** Is ca_sdio_hw_write_block() safe to call from an ISR or in a multi-threaded context?
A: No, ca_sdio_hw_write_block() is neither ISR-safe nor thread-safe. It is a blocking function that should not be called from an ISR due to potential synchronization issues. It also should not be called from multiple threads concurrently as it may cause data corruption.

HW differences conf:0.85 **Q:** How does the implementation of ca_sdio_hw_write_block() differ between F4xx and H7xx MCUs?
A: The implementations are nearly identical in structure, both using SDIO or SDMMC peripherals with similar register configurations. However, F4xx uses SDIO registers (e.g., SDIO->FIFO), while H7xx uses SDMMC1 registers (e.g., SDMMC1->FIFO). The core logic for sending commands and managing FIFO remains consistent across families.

`ca_sdio_hw_write_blocks`

Header: `caos/drivers/inc/ca_glue_hw.h:237`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_sdio_hw_write_blocks(u32 block, const u8 *buf, u32 count)
    ; /* writes count blocks with CMD25 multi-block */
```

Writes multiple 512-byte data blocks to the SD card starting at a specific block address.

Parametri:

Nome	Tipo	Descrizione
`block`	`u32`	The starting logical block address (LBA) on the SD card. Must be within the valid range returned by ca_sdio_get_block_count().
`buf`	`const u8 *`	Pointer to the source data buffer. Must be at least (count * 512) bytes long. On some hardware, alignment to 32-bit boundaries is required for optimal DMA performance.
`count`	`u32`	Number of 512-byte blocks to write. A value of 0 is treated as a no-op and returns true.

Ritorna:
Returns true if all blocks were written successfully, false if any block write failed or if the hardware encountered an error.

Note:

- This function is blocking, meaning the calling task will wait until the hardware completes the write operation. It is not thread-safe; you must ensure exclusive access to the SD card using a mutex or by design, as concurrent writes from different tasks will corrupt the data. It must not be called from an ISR because it performs blocking I/O and waits for hardware readiness. A common beginner mistake is passing a buffer that is not properly aligned or is too small for the requested count, which can lead to memory corruption or hard faults on architectures that enforce strict alignment for DMA transfers.

Uso:

- This function is used for bulk data storage, such as writing file system sectors or logging large amounts of data. It is typically used after calling ca_sdio_init() to prepare the hardware. In a standard sequence, you would allocate a buffer, fill it with data, call ca_sdio_write_blocks(), and then free the buffer. It is often paired with ca_sdio_read_block() for data verification or used in conjunction with ca_memory_allocate() to ensure the buffer resides in a safe memory region.

Esempio:

```
/* Example: Writing a 1024-byte buffer (2 blocks) to the SD card */
#define DATA_SIZE 1024

void task_write_data(void) {
    u8 *data_buffer = ca_memory_allocate(DATA_SIZE);
    if (data_buffer == NULL) return;

    /* Fill buffer with dummy data */
    ca_memset(data_buffer, 0xAA, DATA_SIZE);

    /* Write 2 blocks starting at block 100 */
    if (ca_sdio_write_blocks(100, data_buffer, 2)) {
        /* Success: data is now on the SD card */
    } else {
        /* Handle write error: log or retry */
    }

    ca_memory_free(data_buffer);
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_sdio_hw_write_blocks instead of single-block writes?

A: You should use ca_sdio_hw_write_blocks for bulk data storage tasks, such as writing file system sectors or logging large amounts of data. It is designed to handle multiple 512-byte blocks efficiently after the SD card has been initialized via ca_sdio_init().

Code example conf:1.00 **Q:** Provide a minimal example of how to write 2 blocks to an SD card using caOS types.
A: u8 buffer[1024]; /* Fill buffer with data */ u32 start_block = 100; if (ca_sdio_hw_write_blocks(start buffer, 2)) { /* Success */ }

Parameters conf:1.00 **Q:** What are the constraints on the 'count' parameter in ca_sdio_hw_write_blocks?
A: The 'count' parameter represents the number of 512-byte blocks to write. A value of 0 is invalid and will cause the function to return false. The buffer provided must be at least (count * 512) bytes long.

Return conf:1.00 **Q:** What does a return value of false indicate for ca_sdio_hw_write_blocks?
A: A return value of false indicates that either the input parameters were invalid (such as count being 0) or that one of the block write operations failed during the process.

Error/Edge case conf:1.00 **Q:** What are common mistakes that lead to hard faults when using this function?
A: A common beginner mistake is passing an unaligned buffer or a buffer smaller than the requested size (count * 512 bytes), which can lead to memory corruption or hard faults. Additionally, calling this function from an ISR is prohibited as it performs blocking I/O.

HW differences conf:0.90 **Q:** How does the implementation of multi-block writing differ across STM32 families?
A: Implementations vary significantly: the F1xx family stub returns false for all writes, the F4xx family uses a loop with sdio_write_chunk and a bounce buffer, while newer families like U5xx utilize an IDMA (Internal DMA) data-transfer driver to handle multi-block transfers via native CMD25.

`ca_sdio_init`

Header: `caos/drivers/inc/sdio.h:20`
Source: `caos/drivers/src/sdio.c:109:117`
GUARD: `EN_SDIO`

```
1 bool ca_sdio_init(void)
```

Initializes the SDIO hardware interface and registers the CLI command node.

Ritorna:
Returns true if the SDIO hardware was successfully initialized and is ready for use; returns false if the initialization failed (e.g., no card detected or hardware error).

Note:

- This function is not thread-safe and should be called during system initialization or before the first file system access. It performs hardware-level initialization, which involves blocking operations while waiting for the SD card to respond to commands; therefore, it must never be called from an ISR. While the function is safe to call multiple times (it resets the hardware state), doing so in a high-frequency loop is inefficient and can cause significant system latency. A common beginner mistake is assuming the card is ready immediately after power-on without checking the boolean return value, which leads to null pointer dereferences or failed file system mounts.

Uso:

- This function is the primary entry point for SD card operations. It is typically invoked by the disk driver layer (e.g., FatFS integration) when the system first attempts to mount the

storage media. It is also used in diagnostic CLI tasks to verify hardware presence before performing read/write operations. It is often paired with ca_sdio_get_block_count to verify card capacity and is followed by ca_sdio_read_block or ca_sdio_write_block for actual data transfer.

Esempio:

```c
/* Initialize SDIO and check status before performing operations */
if (ca_sdio_init()) {
    ca_stdout_printf_n("SDIO initialized successfully.");
    u32 blocks = ca_sdio_get_block_count();
    ca_stdout_printf_n("Card size: %lu blocks", (unsigned long)blocks);
} else {
    ca_stdout_printf_n("Error: Failed to initialize SDIO hardware.");
    /* Handle error: perhaps signal a system fault or retry later */
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_sdio_init() in my caOS application and why is it important?
A: You should call ca_sdio_init() during the system initialization phase of your caOS application, typically before any SDIO operations are needed. This function initializes the SDIO hardware driver and registers a command CLI for debugging purposes. It's important because it sets up the hardware interface for SDIO communication and enables debug capabilities through the CLI, but it must be called only once to avoid issues with the static CLI registration variable.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_sdio_init() with caOS types?
A: bool result = ca_sdio_init(); if (result) { // SDIO initialized successfully // Proceed with SDIO operations } else { // Initialization failed // Handle error appropriately }

Return conf:0.90 **Q:** What do the return values of ca_sdio_init() indicate and what causes initialization to fail?
A: ca_sdio_init() returns true if hardware initialization succeeds and CLI registration is completed successfully. It returns false if either the hardware is unavailable or the CLI registration fails. Failures can occur due to hardware unavailability, such as missing SDIO peripherals, or issues with CLI registration, which may happen if the registration mechanism fails. The function is not thread-safe and will only register the CLI command once even if called multiple times.

`ca_sdio_read_block`

Header: `caos/drivers/inc/sdio.h:21`
Source: `caos/drivers/src/sdio.c:119:123`
GUARD: `EN_SDIO`

```c
bool ca_sdio_read_block(u32 block, u8 *buf)
```

Reads a single 512-byte block of data from the SD card at the specified logical block address.

Parametri:

Nome	Tipo	Descrizione
`block`	`u32`	The logical block address (LBA) to read. Must be within the valid range of the SD card capacity.
`buf`	`u8 *`	Pointer to a buffer of at least 512 bytes where the data will be stored. Must not be NULL.

Ritorna:
Returns true if the read operation was successful, false if the operation failed or if the provided buffer pointer was NULL.

Note:

- This function is a blocking call, meaning the task will wait until the hardware completes the SD card read operation. It is not safe to call this function from an Interrupt Service Routine (ISR) because it may involve waiting for hardware signals or mutexes that are not available in an interrupt context. Ensure that ca_sdio_init has been successfully called before attempting to read. A common mistake is passing a buffer that is too small or not properly aligned, which can lead to memory corruption or hardware errors depending on the underlying DMA implementation.

Uso:

- This function is typically used in file system drivers (like FATFS) or raw data storage applications to retrieve data from the SD card. It is usually paired with ca_sdio_init during system startup to prepare the hardware. In a typical sequence, you initialize the SDIO peripheral, verify the card is ready, and then call this function in a loop to read multiple sectors or blocks into a memory buffer allocated via ca_memory_allocate.

Esempio:

```c
/* Example: Reading a block and verifying data */
#include "caos.h"

void read_and_verify_block(u32 lba) {
    /* Allocate memory for one 512-byte block */
    u8 *buffer = (u8 *)ca_memory_allocate(512U);
    if (buffer == NULL) {
        ca_printf("Memory allocation failed\n");
        return;
    }

    /* Perform the read operation */
    if (ca_sdio_read_block(lba, buffer)) {
        ca_printf("Read successful at LBA: %lu\n", (unsigned long)lba);
        /* Process data here... */
    } else {
        ca_printf("Read failed at LBA: %lu\n", (unsigned long)lba);
    }

    /* Always free allocated memory */
    ca_memory_free(buffer);
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_sdio_read_block to read data from an SD card?
A: You should use ca_sdio_read_block when you need to read a specific 512-byte block of data from an SD card using the SDIO interface. It is suitable for reading file system data or other structured data stored on the SD card. The function is blocking, so it should be used in contexts where blocking is acceptable, such as during initialization or in non-RTOS tasks.

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_sdio_read_block with caOS types?
A: u8 buffer[512]; u32 block_number = 0; bool success = ca_sdio_read_block(block_number, buffer); if (success) { // Process the data in buffer } else { // Handle read failure }

Parameters conf:0.90 **Q:** What does the block parameter in ca_sdio_read_block represent, and what are valid values?

A: The block parameter in ca_sdio_read_block represents the 0-based index of the block to read from the SD card. Valid values are integers that correspond to valid block addresses on the SD card. If an invalid block number is passed (e.g., beyond the card's capacity), the function will return false. The block number must be valid for the SDIO device in use.

Return conf:0.95 **Q:** What does a return value of false from ca_sdio_read_block indicate, and what causes it?

A: A return value of false from ca_sdio_read_block indicates that the read operation failed. This can happen due to several reasons: the buffer pointer is NULL, the block number is invalid, or there is a hardware error during the SDIO read operation. The function does not distinguish between these causes in its return value.

`ca_sdio_write_block`

Header: `caos/drivers/inc/sdio.h:22`
Source: `caos/drivers/src/sdio.c:125:129`
GUARD: `EN_SDIO`

```
1 bool ca_sdio_write_block(u32 block, const u8 *buf)
```

Writes a single 512-byte data block to the SD card at the specified logical block address.

Parametri:

Nome	Tipo	Descrizione
`block`	`u32`	The logical block address (LBA) on the SD card where data should be written. Must be within the valid range returned by ca_sdio_get_block_count.
`buf`	`const u8 *`	Pointer to a buffer containing exactly 512 bytes of data. Must not be NULL.

Ritorna:
Returns true if the write operation was successful, false if the input buffer is NULL or if the hardware communication failed.

Note:

- This function is a blocking call, meaning the task will wait until the SD card controller completes the write operation before returning. It is not safe to call this function from an Interrupt Service Routine (ISR) because it relies on blocking mechanisms that are not permitted in interrupt context. Ensure that ca_sdio_init has been successfully called before attempting any read or write operations. A common mistake is passing a buffer that is not aligned or is smaller than the expected block size, which can lead to memory corruption or incomplete data writes.

Uso:

- This function is typically used in file system drivers or raw data logging applications where specific sectors of the SD card need to be updated. It is often paired with ca_sdio_init for setup and ca_sdio_read_block for verification purposes. In a typical sequence, you initialize the SDIO peripheral, perform your write operations, and then verify the integrity of the data by reading it back. It is the fundamental building block for higher-level storage operations.

Esempio:

```c
/* Example: Writing a data block to the SD card */
#define BLOCK_SIZE 512

void write_test_data(u32 lba) {
    u8 *data = (u8*)ca_memory_allocate(BLOCK_SIZE);
    if (!data) return;

    /* Fill buffer with dummy data */
    for (int i = 0; i < BLOCK_SIZE; i++) {
        data[i] = (u8)(i & 0xFF);
    }

    /* Ensure SDIO is initialized before use */
    if (ca_sdio_init()) {
        if (ca_sdio_write_block(lba, data)) {
            ca_printf("Write successful at block %lu\n", (unsigned long)lba);
        } else {
            ca_printf("Write failed at block %lu\n", (unsigned long)lba);
        }
    }

    ca_memory_free(data);
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_sdio_write_block() in a caOS application?
A: You should call ca_sdio_write_block() when you need to write a 512-byte block of data to an SDIO device, such as an SD card. It is a blocking function intended for use in thread contexts, not from interrupt service routines (ISRs). This function is typically used when implementing file system operations or direct block-level storage access.

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_sdio_write_block() with caOS types?
A: u8 buffer[512]; // Fill buffer with data u32 block_number = 0; bool success = ca_sdio_write_block(block_number, buffer); if (!success) { // Handle error }

Parameters conf:0.90 **Q:** What are the valid ranges for the block parameter in ca_sdio_write_block()?
A: The block parameter must be a 32-bit unsigned integer (u32) representing a zero-based block index. It must be less than the total number of blocks returned by ca_sdio_get_block_count(). Passing a block number equal to or greater than the total block count will result in a failure.

Return conf:0.95 **Q:** What does it mean when ca_sdio_write_block() returns false?
A: A return value of false indicates that the write operation failed. This can happen if the buffer pointer passed is NULL, or if there was a hardware-level failure during the write operation to the SDIO device. The function does not distinguish between these two failure modes in its return value.

`ca_sdio_write_blocks`

Header: `caos/drivers/inc/sdio.h:23`
Source: `caos/drivers/src/sdio.c:131:135`
GUARD: `EN_SDIO`

```c
bool ca_sdio_write_blocks(u32 block, const u8 *buf, u32 count)
```

Writes a specified number of data blocks to the SD card at a given starting address.

Parametri:

Nome	Tipo	Descrizione
`block`	`u32`	The starting logical block address (LBA) on the SD card where writing begins.
`buf`	`const u8 *`	Pointer to the memory buffer containing the data to be written. Must not be NULL.
`count`	`u32`	The number of consecutive blocks to write. Must be greater than 0.

Ritorna:
Returns true if the write operation was successful, false if the input buffer is NULL, the count is zero, or the hardware operation failed.

Note:

- This function is typically blocking, meaning the calling task will wait until the SD card controller completes the physical write operation. It is not intended to be called from an ISR because it involves waiting for hardware completion, which can take significant time. Ensure that ca_sdio_init has been successfully called before attempting any write operations. A common beginner mistake is passing a buffer that is not properly aligned or is located in memory regions inaccessible to the DMA controller used by the SDIO peripheral.

Uso:

- This function is primarily used by file system drivers (like FatFs) to commit data to the storage medium. It is typically paired with ca_sdio_init during system startup to prepare the hardware. In a typical sequence, you would allocate a buffer using ca_memory_allocate, fill it with data, call ca_sdio_write_blocks to persist it, and then free the memory using ca_memory_free.

Esempio:

```
/* Example: Writing a data buffer to the SD card */
#include "caos.h"

void save_data_to_sd(u32 start_block, u32 num_blocks) {
    // Allocate memory for the data buffer
    u8 *data_buffer = (u8 *)ca_memory_allocate(num_blocks * 512);

    if (data_buffer != NULL) {
        // Fill buffer with dummy data or actual content
        ca_memset(data_buffer, 0xAA, num_blocks * 512);

        // Perform the write operation
        if (ca_sdio_write_blocks(start_block, data_buffer, num_blocks)) {
            // Success: log or proceed
        } else {
            // Handle write error
        }

        // Always free allocated memory
        ca_memory_free(data_buffer);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_sdio_write_blocks in my caOS application?
A: You should use this function when you need to persist data to an SD card, typically within file system drivers like FatFs. It is intended to be called after ca_sdio_init has successfully initialized the hardware. It is a blocking operation, so it must not be called from an ISR.

Code example conf:0.90 **Q:** Provide a minimal example of how to write a single block to the SD card using caOS types.
A: u8 *buffer = ca_memory_allocate(512); /* Fill buffer with data */ bool success = ca_sdio_write_blocks(0x100, buffer, 1); if (success) { ca_memory_free(buffer); }

Parameters conf:1.00 **Q:** What are the constraints on the 'block' and 'count' parameters for ca_sdio_write_blocks?
A: The 'block' parameter represents the starting logical block address (LBA) on the SD card. The 'count' parameter specifies the number of consecutive blocks to write and must be greater than 0. Additionally, the 'buf' pointer must not be NULL.

Return conf:1.00 **Q:** What does a false return value from ca_sdio_write_blocks indicate?
A: A false return indicates that the write operation failed. This can be caused by passing a NULL buffer, providing a count of zero, or a failure in the underlying SDIO hardware operation.

`ca_secure_pipe_deinit`

Header: `caos/crypto/inc/secure_pipe_service.h:78`
Source: `caos/crypto/src/secure_pipe_service.c:99:110`
GUARD: `EN_SECURE_PIPE_SERVICE`

```
void ca_secure_pipe_deinit(ca_secure_pipe_t *p)
```

Safely shuts down a secure pipe, releasing all associated queues and cryptographic resources.

Parametri:

Nome	Tipo	Descrizione
`p`	`ca_secure_pipe_t *`	Pointer to the secure pipe structure to be deinitialized. Must be a valid, previously initialized handle.

Ritorna:
void

Note:

- This function is not thread-safe and must not be called from an ISR. It performs a cleanup sequence: it marks the pipe as inactive, drains any pending data in the internal queues to prevent memory leaks, destroys the heap-based queues, and finally deinitializes the AES channel. A common mistake is calling this function while other tasks are still attempting to send or receive data through the pipe, which will lead to undefined behavior or system instability. Always ensure the pipe is no longer in use before calling this.

Uso:

- This function is used during the teardown phase of a secure communication lifecycle. It is typically paired with ca_secure_pipe_init to manage the lifecycle of a secure channel. In a typical application, you initialize the pipe at system startup, use it for encrypted data exchange, and call this deinit function when the service is stopped or the connection is closed to reclaim memory and hardware resources.

Esempio:

```
ca_secure_pipe_t s_pipe;

// Initialize the pipe before use
// ca_secure_pipe_init(&s_pipe);

```

```c
void stop_secure_service(void) {
    // Ensure no further operations are performed on the pipe
    if (s_pipe.ready) {
        // Safely release all resources associated with the pipe
        ca_secure_pipe_deinit(&s_pipe);
    }

    // Optional: log the shutdown
    ca_printf("Secure pipe deinitialized successfully.\n");
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_secure_pipe_deinit in my caOS application?
A: You should call this function during the teardown phase of a secure communication lifecycle, such as when a service is stopped or a connection is closed, to reclaim memory and hardware resources.

Code example conf:0.90 **Q:** Provide a minimal C example showing how to deinitialize a secure pipe.
A: ca_secure_pipe_t *my_pipe; /* Assume initialized */ ca_secure_pipe_deinit(my_pipe);

Parameters conf:1.00 **Q:** What are the requirements for the parameter passed to ca_secure_pipe_deinit?
A: The parameter 'p' must be a pointer to a valid, previously initialized ca_secure_pipe_t structure.

Error/Edge case conf:1.00 **Q:** What are the risks of calling ca_secure_pipe_deinit while other tasks are active?
A: Calling this function while other tasks are attempting to send or receive data through the pipe will lead to undefined behavior or system instability. Additionally, it is not thread-safe and must not be called from an ISR.

`ca_secure_pipe_init`

Header: `caos/crypto/inc/secure_pipe_service.h:53`
Source: `caos/crypto/src/secure_pipe_service.c:25:50`
GUARD: `EN_SECURE_PIPE_SERVICE`

```c
i32 ca_secure_pipe_init(ca_secure_pipe_t *p, ca_aes_mode_t mode,
    ca_aes_keysize_t keysize, void *ks_handle, u32 depth)
```

Initializes a secure communication pipe for encrypted data exchange using AES.

Parametri:

Nome	Tipo	Descrizione
`p`	`ca_secure_pipe_t *`	Pointer to the pipe structure. Must be allocated and not already initialized (p->ready must be false).
`mode`	`ca_aes_mode_t`	The AES mode to use (e.g., CA_AES_MODE_GCM).
`keysize`	`ca_aes_keysize_t`	The size of the AES key (e.g., CA_AES_KEY_256).
`ks_handle`	`void *`	Handle to the key storage containing the encryption key.
`depth`	`u32`	The number of packets the internal queues can hold. Must be greater than 0.

Ritorna:

Returns CA_SECURE_PIPE_OK (0) on success, or CA_SECURE_PIPE_ERR (-1) if initialization fails due to invalid parameters, memory allocation failure, or if the pipe is already initialized.

Note:

- This function is not thread-safe and must be called before any attempt to send or receive data through the pipe. It performs internal memory allocation for two queues; if these fail, it cleans up the AES channel before returning. A common mistake is passing an uninitialized or already-active pipe structure, which will trigger an error. This function is not callable from an ISR because it performs heap allocations.

Uso:

- This function is the mandatory first step in the secure communication lifecycle. It is typically used during system startup or when a new secure session is established. It pairs with ca_secure_pipe_send and ca_secure_pipe_recv for data transfer, and finally ca_secure_pipe_deinit to release resources. Ensure the key storage handle is valid and accessible before calling this function.

Esempio:

```
ca_secure_pipe_t s_pipe;
void *s_key_ptr = /* pointer to valid key storage */;

/* Initialize the pipe with GCM mode and 256-bit key */
if (ca_secure_pipe_init(&s_pipe, CA_AES_MODE_GCM, CA_AES_KEY_256, s_key_ptr, 10)
    != 0) {
    /* Handle initialization failure */
    ca_printf("Pipe init failed!\n");
} else {
    /* Pipe is ready for use */
    ca_printf("Pipe initialized successfully.\n");
    /* Proceed to use ca_secure_pipe_send or ca_secure_pipe_recv */
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_secure_pipe_init in my caOS application?
A: You must call ca_secure_pipe_init as the mandatory first step in the secure communication lifecycle, typically during system startup or when establishing a new secure session, before using ca_secure_pipe_send or ca_secure_pipe_recv.

Code example conf:0.90 **Q:** Provide a minimal C example for initializing a secure pipe.
A: ca_secure_pipe_t my_pipe; i32 status = ca_secure_pipe_init(&my_pipe, CA_AES_MODE_GCM, CA_AES_KEY_256, my_ks_handle, 10); if (status == CA_SECURE_PIPE_OK) { /* Pipe ready */ }

Parameters conf:1.00 **Q:** What are the constraints on the 'depth' parameter?
A: The 'depth' parameter defines the number of packets the internal queues can hold and must be a value greater than 0.

Return conf:1.00 **Q:** What do the return values CA_SECURE_PIPE_OK and CA_SECURE_PIPE_ERR signify?
A: CA_SECURE_PIPE_OK (0) indicates successful initialization. CA_SECURE_PIPE_ERR (-1) indicates failure due to invalid parameters, memory allocation failure, or attempting to initialize a pipe that is already active.

Error/Edge case conf:1.00 **Q:** What is a common mistake when calling ca_secure_pipe_init?
A: A common mistake is passing a pipe structure that is already active or not properly initialized (where p->ready is not false), which will trigger a CA_SECURE_PIPE_ERR.

Behavior conf:1.00 **Q:** Is ca_secure_pipe_init safe to call from an interrupt service routine (ISR)?

A: No, this function is not ISR-safe because it performs heap allocations. Additionally, it is not thread-safe and must be called before any data transfer operations.

---

ca_secure_pipe_recv

Header: caos/crypto/inc/secure_pipe_service.h:69
Source: caos/crypto/src/secure_pipe_service.c:71:97
GUARD: EN_SECURE_PIPE_SERVICE

```
i32 ca_secure_pipe_recv(ca_secure_pipe_t *p, ca_secure_pipe_end_t at,
    void *buf, u32 cap, u32 *out_len, u32 timeout_ms)
```

Receives and decrypts a secure packet from a specific end of a secure pipe.

Parametri:

Nome	Tipo	Descrizione
p	ca_secure_pipe_t *	Pointer to the initialized secure pipe handle. Must not be NULL.
at	ca_secure_pipe_end_t	The pipe endpoint identifier (e.g., CA_SECURE_PIPE_A or CA_SECURE_PIPE_B).
buf	void *	Pointer to the destination buffer where the decrypted data will be stored.
cap	u32	Capacity of the destination buffer in bytes. Used to prevent buffer overflows.
out_len	u32 *	Pointer to a variable that will store the actual number of decrypted bytes received.
timeout_ms	u32	Maximum time in milliseconds to wait for a packet. Use 0 for non-blocking or a large value for indefinite wait.

Ritorna:
CA_SECURE_PIPE_OK (0) on success. CA_SECURE_PIPE_TIMEOUT if no data arrives within the timeout. CA_SECURE_PIPE_ERR for invalid arguments or uninitialized pipes. CA_SECURE_PIPE_DECRYPT if decryption fails (e.g., wrong key or corrupted data). CA_SECURE_PIPE_SIZE if the decrypted data exceeds the provided buffer capacity.

Note:

- This function is blocking; it will suspend the calling task until a packet is received or the timeout expires. It is thread-safe, provided the pipe handle is correctly initialized and managed. It must NOT be called from an ISR because it performs blocking operations and memory management. A common beginner mistake is providing a buffer that is too small for the expected payload, which results in a CA_SECURE_PIPE_SIZE error. Ensure the pipe is fully initialized via the appropriate setup functions before calling this.

Uso:

- This function is typically used in a producer-consumer pattern where one task waits for encrypted data from a remote source. It is usually paired with ca_secure_pipe_init during system startup and ca_secure_pipe_send in the corresponding sender task. After receiving, the application should check the return value to handle potential decryption errors or timeouts gracefully before processing the data in the buffer.

Esempio:

```c
ca_secure_pipe_t s_pipe;
/* Assume s_pipe is already initialized */
char rx_buffer[128];
u32 received_bytes = 0;

/* Wait up to 1000ms for a secure packet */
i32 status = ca_secure_pipe_recv(&s_pipe, CA_SECURE_PIPE_A, rx_buffer, sizeof(
    rx_buffer), &received_bytes, 1000);

if (status == CA_SECURE_PIPE_OK) {
    /* Process decrypted data */
    ca_printf("Received %u bytes: %s\n", received_bytes, rx_buffer);
} else if (status == CA_SECURE_PIPE_TIMEOUT) {
    ca_printf("No data received within timeout.\n");
} else {
    /* Handle decryption or size errors */
    ca_printf("Secure pipe error: %d\n", status);
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_secure_pipe_recv in my caOS application?
A: You should use this function in a producer-consumer pattern when a task needs to wait for and decrypt data from a remote source. It is typically called after initializing the pipe with ca_secure_pipe_init and is intended to be paired with ca_secure_pipe_send in a corresponding sender task.

Code example conf:1.00 **Q:** Provide a minimal C example of how to call ca_secure_pipe_recv.
A: u8 buffer[64]; u32 received = 0; i32 status = ca_secure_pipe_recv(&my_pipe, CA_SECURE_PIPE_A, buffer, 64, &received, 1000); if (status == CA_SECURE_PIPE_OK) { /* process data */ }

Parameters conf:1.00 **Q:** What is the purpose of the 'cap' parameter in ca_secure_pipe_recv?
A: The 'cap' parameter specifies the capacity of the destination buffer in bytes. It is used by the function to prevent buffer overflows by ensuring the decrypted data does not exceed the allocated memory space.

Return conf:1.00 **Q:** What does a return value of CA_SECURE_PIPE_DECRYPT indicate?
A: A return value of CA_SECURE_PIPE_DECRYPT indicates that the decryption process failed. This typically occurs due to an incorrect key or corrupted data within the received packet.

Error/Edge case conf:1.00 **Q:** What is a common mistake when using ca_secure_pipe_recv that leads to a CA_SECURE_PIPE_SIZE error?
A: A common beginner mistake is providing a destination buffer that is too small to hold the expected payload, which causes the function to return CA_SECURE_PIPE_SIZE.

Behavior conf:1.00 **Q:** Is it safe to call ca_secure_pipe_recv from an interrupt service routine (ISR)?
A: No, it must not be called from an ISR. The function performs blocking operations and memory management, which are not suitable for execution within an interrupt context.

`ca_secure_pipe_send`

Header: `caos/crypto/inc/secure_pipe_service.h:61`
Source: `caos/crypto/src/secure_pipe_service.c:52:69`
GUARD: `EN_SECURE_PIPE_SERVICE`

```c
i32 ca_secure_pipe_send(ca_secure_pipe_t *p, ca_secure_pipe_end_t from,
    const void *data, u32 len, u32 timeout_ms)
```

Encrypts and sends a data packet through a secure pipe to the opposite end.

Parametri:

Nome	Tipo	Descrizione
`p`	`ca_secure_pipe_t *`	Pointer to the initialized secure pipe structure. Must not be NULL.
`from`	`ca_secure_pipe_end_t`	The source end identifier (CA_SECURE_PIPE_A or CA_SECURE_PIPE_B). Must be valid.
`data`	`const void *`	Pointer to the raw data buffer to be sent. Must not be NULL.
`len`	`u32`	Length of data in bytes. Must be > 0 and <= CA_SECURE_PIPE_MAX_MSG.
`timeout_ms`	`u32`	Maximum time in milliseconds to wait for the queue to accept the packet.

Ritorna:
Returns CA_SECURE_PIPE_OK (0) on success. Returns CA_SECURE_PIPE_ERR if parameters are invalid or encryption fails. Returns CA_SECURE_PIPE_TIMEOUT if the queue is full and the timeout expires.

Note:

- This function is blocking; it will suspend the calling task if the internal queue is full until space becomes available or the timeout expires. It is thread-safe, assuming the underlying pipe structure is properly initialized and protected by the OS. It must NOT be called from an ISR because it performs blocking operations. A common mistake is passing a length that exceeds the pipe's maximum message size, which will cause an immediate error return.

Uso:

- This function is used in inter-task communication where data integrity and confidentiality are required. It is typically used after calling ca_secure_pipe_init to set up the pipe and before calling ca_secure_pipe_deinit when the communication is finished. It is often paired with ca_secure_pipe_recv on the receiving end to complete the data exchange cycle.

Esempio:

```
/* Assuming s_pipe is a global or shared ca_secure_pipe_t instance */
char *msg = "Secure Data Payload";
u32 len = (u32)ca_strlen(msg);

/* Attempt to send data with a 1000ms timeout */
i32 result = ca_secure_pipe_send(&s_pipe, CA_SECURE_PIPE_A, msg, len, 1000);

if (result == CA_SECURE_PIPE_OK) {
    ca_printf("Data sent successfully!\n");
} else if (result == CA_SECURE_PIPE_TIMEOUT) {
    ca_printf("Failed to send: Pipe full or timeout reached.\n");
} else {
    ca_printf("Failed to send: Invalid parameters or encryption error.\n");
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_secure_pipe_send function in my caOS application?
A: You should use this function for inter-task communication when you require data integrity and confidentiality. It should be called after the secure pipe has been initialized with ca_secure_pipe_init and before the pipe is deinitialized with ca_secure_pipe_deinit.

Code example conf:0.90 **Q:** Provide a minimal C example of how to send data using ca_secure_pipe_send.
A: ca_secure_pipe_t pipe; u8 buffer[] = {0x01, 0x02}; i32 status = ca_secure_pipe_send(&pipe, CA_SECURE_PIPE_A, buffer, 2, 100); if (status == CA_SECURE_PIPE_OK) { /* Success */ }

Parameters conf:1.00 **Q:** What are the constraints on the 'len' parameter in ca_secure_pipe_send?
A: The 'len' parameter must be greater than 0 and must not exceed the constant CA_SECURE_PIPE_MAX_MSG. Passing a length outside this range will result in an immediate error return.

Return conf:1.00 **Q:** What do the different return values of ca_secure_pipe_send signify?
A: CA_SECURE_PIPE_OK (0) indicates success. CA_SECURE_PIPE_ERR is returned if parameters are invalid or encryption fails. CA_SECURE_PIPE_TIMEOUT is returned if the internal queue is full and the specified timeout_ms expires.

Error/Edge case conf:1.00 **Q:** What is a common mistake developers make when using ca_secure_pipe_send?
A: A common mistake is passing a data length that exceeds the pipe's maximum message size (CA_SECURE_PIPE_MAX_MSG), which causes the function to return an error immediately.

Behavior conf:1.00 **Q:** Is ca_secure_pipe_send safe to call from an Interrupt Service Routine (ISR)?
A: No, it must not be called from an ISR because it is a blocking function that may suspend the calling task if the internal queue is full.

`ca_sem_getvalue`

Header: `caos/inc/sync.h:213`
Source: `caos/src/sync.c:507:512`
GUARD: `EN_SYNC`

```
u32 ca_sem_getvalue(ca_semaphore_t *sem)
```

Returns the current count value of a semaphore without modifying it.

Parametri:

Nome	Tipo	Descrizione
`sem`	`ca_semaphore_t *`	Pointer to the initialized semaphore structure. Must not be NULL.

Ritorna:
The current unsigned integer value (count) of the semaphore. A value of 0 indicates the semaphore is currently unavailable (locked).

Note:

- This function is non-blocking and thread-safe, as it uses an internal spinlock to ensure the count is read atomically. It is safe to call from both tasks and ISRs because it uses an interrupt-safe spinlock mechanism. The most common mistake is assuming the returned value remains valid for long; in a multi-tasking environment, another task might call ca_sem_wait or ca_sem_post immediately after this function returns, making the value stale. Always treat the returned value as a snapshot in time rather than a guarantee of future availability.

Uso:

- This function is typically used for diagnostic purposes, such as monitoring resource availability or debugging synchronization issues in a system. It is often paired with ca_sem_init during setup and used alongside ca_sem_wait or ca_sem_post during runtime to log the

state of a resource pool. It is not intended for flow control (i.e., do not use it to check if a semaphore is available before waiting, as this creates a race condition known as 'check-then-act').

Esempio:

```c
#include "caos.h"

// Global semaphore for resource management
caSemaphore resource_semaphore;

void monitor_task(void *pvParameters) {
    // Initialize the semaphore with 5 available resources
    ca_sem_init(&resource_semaphore, 5);

    while (1) {
        // Get the current count of available resources
        u32 current_val = ca_sem_getvalue(&resource_semaphore);

        // Log the status using the system stdout
        ca_printf_n("Current available resources: %u", current_val);

        // Wait for a resource to become available
        if (ca_sem_wait(&resource_semaphore) == 0) {
            // Perform work with the resource
            // ...

            // Release the resource back to the pool
            ca_sem_post(&resource_semaphore);
        }

        ca_os_delay_ms(1000);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_sem_getvalue() in a caOS application?
A: You should use ca_sem_getvalue() when you need to monitor or debug the current state of a semaphore, such as checking how many resources are available. It is useful for logging, diagnostics, or ensuring proper resource management without blocking execution.

Code example conf:0.90 **Q:** Can you provide a minimal example of how to use ca_sem_getvalue() with caOS types?
A: ca_semaphore_t sem; ca_sem_init(&sem, 3); u32 value = ca_sem_getvalue(&sem); // value will be 3, indicating 3 resources are available.

Parameters conf:0.95 **Q:** What happens if I pass a NULL pointer to the sem parameter of ca_sem_getvalue()?
A: Passing a NULL pointer to the sem parameter will cause a crash because the function does not perform null checks and directly accesses the semaphore structure.

Return conf:0.90 **Q:** What does a return value of 0 from ca_sem_getvalue() indicate?
A: A return value of 0 indicates that no resources are currently available in the semaphore, meaning all resources have been acquired by other threads or tasks.

`ca_sem_init`

Header: `caos/inc/sync.h:176`
Source: `caos/src/sync.c:392:399`
GUARD: `EN_SYNC`

```
void ca_sem_init(ca_semaphore_t *sem, const char *name,u32 initial_count
    , u32 max_count)
```

Initializes a semaphore object for task synchronization or resource management.

Parametri:

Nome	Tipo	Descrizione
sem	ca_semaphore_t *	Pointer to the semaphore structure to be initialized. Must be allocated in memory (static or heap).
name	const char *	A short string (max 7 chars + null terminator) used for debugging and identification in system dumps.
initial_count	u32	The starting value of the semaphore. Use 0 for binary semaphores used as signals, or >0 for counting resources.
max_count	u32	The maximum value the semaphore can reach. If set to 0, it defaults to 0xFFFFFFFF.

Ritorna:
This function returns void and does not provide a status code; it assumes the provided pointer is valid.

Note:

- This function is not thread-safe and must be called during the initialization phase of your application, before any tasks or interrupts attempt to use the semaphore. It is not intended to be called from an ISR. A common mistake is to re-initialize a semaphore that is already in use by other tasks, which will corrupt the wait queue and lead to unpredictable system behavior or kernel panics. Always ensure the semaphore structure is properly allocated and not currently being accessed by the scheduler.

Uso:

- Use this function to prepare synchronization primitives before starting the RTOS scheduler. It is typically paired with ca_sem_wait or ca_sem_post to manage access to shared hardware resources (like DMA channels or Flash memory) or to signal completion of asynchronous tasks. In a typical driver lifecycle, you call this during the driver's init function, then use the semaphore throughout the application's runtime to block tasks until a hardware event occurs.

Esempio:

```
ca_semaphore_t g_flash_sem;

void init_system_resources(void) {
    // Initialize the semaphore with 0 count (blocking) and 1 max (binary)
    // This prepares the semaphore for use with ca_flash_erase_sector_irq
    ca_sem_init(&g_flash_sem, "S_FLASH", 0U, 1U);
}

void task_flash_manager(void) {
    // Wait for the semaphore to be posted by the flash ISR
    // ca_sem_wait blocks the current task until the resource is available
    ca_sem_wait(&g_flash_sem);

    ca_printf("Flash operation completed, resource released.\n");
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_sem_init() in my caOS application?
A: You should call ca_sem_init() before using any semaphore in your application. It is typically called during system initialization or when setting up a driver or service that requires synchronization. The function must be called once per semaphore before any task or ISR attempts to use it.

Code example conf:0.90 **Q:** Can you show a minimal example of initializing a semaphore with ca_sem_init()?
A: ca_semaphore_t my_sem; cstr sem_name = "TX_SEM"; ca_sem_init(&my_sem, sem_name, 1, 1); // Binary semaphore

Parameters conf:0.90 **Q:** What is the significance of the initial_count and max_count parameters in ca_sem_init()?
A: The initial_count sets the starting value of the semaphore. If 0, the semaphore starts in a blocked state. The max_count defines the maximum value the semaphore can reach. If max_count is 0, the semaphore becomes an unlimited counter. Both values must be non-negative, and initial_count must not exceed max_count.

Return conf:0.85 **Q:** Does ca_sem_init() return any value, and what does it indicate?
A: The function returns void, meaning it does not return any value. According to the documentation, the function does not fail, but incorrect parameters may lead to undefined behavior.

`ca_sem_post`

Header: `caos/inc/sync.h:205`
Source: `caos/src/sync.c:478:505`
GUARD: `EN_SYNC`

```
1 void ca_sem_post(ca_semaphore_t *sem)
```

Increments a semaphore's count or wakes a waiting task if one is blocked on it.

Parametri:

Nome	Tipo	Descrizione
`sem`	`ca_semaphore_t *`	Pointer to the semaphore object. Must be initialized via ca_sem_init before use.

Ritorna:
void. If the semaphore count exceeds its maximum capacity, the system triggers a CA_SAFEGUARD error.

Note:

- This function is thread-safe and can be called from both task context and ISRs. It is non-blocking, meaning it returns immediately after updating the semaphore state or signaling a waiting task. A common beginner mistake is calling this on an uninitialized semaphore or failing to account for the semaphore's maximum count, which leads to a system-wide safeguard halt. Always ensure the semaphore is properly initialized with a valid max_count before posting. If you expect the count to frequently hit the maximum and do not want a system halt, consider using ca_sem_post_capped instead.

Uso:

- This function is the standard mechanism for signaling that a resource is available or an event has occurred. It is typically paired with ca_sem_wait or ca_sem_timedwait in a producer-consumer pattern. In driver development, it is frequently used inside an ISR to

notify a background task that hardware processing (like DMA transfer or Flash writing) is complete. It is also used in service modules to synchronize state changes between different threads, ensuring that a task waiting for a specific event (like a DHCP lease or ARP resolution) is resumed immediately upon completion.

Esempio:

```c
/* Example: Signaling a task from an ISR when a DMA transfer completes */

caSemaphore dma_sem;

void DMA_IRQHandler(void) {
    /* Clear hardware interrupt flags here */

    /* Notify the waiting task that the DMA buffer is ready */
    ca_sem_post(&dma_sem);
}

void worker_task(void) {
    /* Initialize semaphore with max count 1 */
    ca_sem_init(&dma_sem, 0, 1);

    while (1) {
        /* Wait for the ISR to signal that data is ready */
        if (ca_sem_timedwait(&dma_sem, 1000)) {
            /* Process the data now that the semaphore is released */
        } else {
            /* Handle timeout error */
        }
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_sem_post() in a DMA transfer completion ISR?
A: You should call ca_sem_post() in a DMA transfer completion ISR to signal that the transfer has completed, allowing a waiting task to resume execution. This is commonly used in drivers to notify tasks that hardware operations are done.

Code example conf:0.90 **Q:** Can you show a minimal example of using ca_sem_post() with ca_semaphore_t?
A: ca_semaphore_t sem; ca_sem_init(&sem, 0); // In ISR or task: ca_sem_post(&sem);

Parameters conf:0.95 **Q:** What happens if I pass NULL to ca_sem_post()?
A: Passing NULL to ca_sem_post() causes a crash because the function requires a valid pointer to a ca_semaphore_t structure that has been initialized.

Return conf:0.90 **Q:** Does ca_sem_post() return any value, and what does it mean?
A: ca_sem_post() does not return any value. It is a void function and does not fail under normal conditions, though it may log an overflow error if the semaphore counter exceeds its maximum.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_sem_post()?
A: Common mistakes include calling ca_sem_post() with an uninitialized semaphore, passing NULL as the parameter, or calling it from an unsafe context. Also, if the semaphore counter overflows, it will be logged but not cause an immediate crash.

Behavior conf:0.90 **Q:** Is ca_sem_post() safe to call from an ISR, and is it blocking?
A: Yes, ca_sem_post() is safe to call from an ISR and is non-blocking. It is also thread-safe and can be called from any context that is safe to use with caOS primitives, but not from unsafe contexts.

ca_sem_post_capped

Header: caos/inc/sync.h:206
Source: caos/src/sync.c:465:476
GUARD: EN_SYNC

```
void ca_sem_post_capped(ca_semaphore_t *sem); /* ISR notification:
   silently caps at max */
```

Increments a semaphore count up to a maximum limit, designed for safe use in ISRs.

Parametri:

Nome	Tipo	Descrizione
sem	ca_semaphore_t *	Pointer to the semaphore object. Must be initialized via ca_sem_init before use.

Ritorna:
void. This function does not return a status code because it is designed to be fire-and-forget.

Note:

- This function is thread-safe and specifically designed to be callable from an Interrupt Service Routine (ISR). It uses a spinlock to protect the internal semaphore state, ensuring that concurrent access from different tasks or interrupts does not corrupt the count. Unlike standard post functions, this will silently ignore requests if the semaphore has already reached its 'max_count', preventing overflow. The most common mistake is passing an uninitialized semaphore pointer or failing to define a reasonable 'max_count' during initialization, which can lead to unexpected behavior in the task waiting for the signal.

Uso:

- This function is primarily used in high-frequency event notification patterns, such as Ethernet RX interrupts or packet processing queues, where the exact number of events is less important than ensuring the processing task is woken up. It is typically paired with ca_sem_wait or ca_sem_trywait in the consumer task. You use this when you want to signal a task that data is available without worrying about the overhead of error checking or the risk of overflowing the semaphore count.

Esempio:

```
/* Example: Signaling a processing task from an ISR */
#include "caos.h"

static ca_semaphore_t rx_sem;

void system_init(void) {
    /* Initialize semaphore with a max count of 10 */
    ca_sem_init(&rx_sem, 0, 10);
}

/* ISR triggered by hardware event */
void hardware_rx_isr(void) {
    /* Notify the task that data is ready.
       If the task is already processing, the count increments.
       If count reaches 10, further signals are ignored. */
    ca_sem_post_capped(&rx_sem);
}

void processing_task(void *arg) {
    while (1) {
        /* Wait for the signal from the ISR */
```

```
        ca_sem_wait(&rx_sem);

        /* Process the data here */
        /* ... */
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_sem_post_capped in an ISR, and what makes it different from a regular semaphore post?
A: You should use ca_sem_post_capped in an ISR when you need to notify a waiting thread without blocking, and you want to avoid exceeding the semaphore's maximum count. Unlike a regular semaphore post, ca_sem_post_capped silently caps the count at its maximum, preventing overflow. It is specifically designed for ISR use and is thread-safe.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_sem_post_capped with caOS types?
A: ca_semaphore_t sem; ca_sem_init(&sem, 0, 10); /* Initialize with 0 count, max 10 */ ca_sem_post_capped(&sem); /* Increment without exceeding max */

Parameters conf:0.95 **Q:** What happens if I pass a NULL pointer to ca_sem_post_capped?
A: Passing a NULL pointer to ca_sem_post_capped will cause a crash, as the function assumes the parameter is valid and initialized. The function does not perform any NULL checks.

Return conf:0.95 **Q:** Does ca_sem_post_capped ever fail or return an error?
A: No, ca_sem_post_capped does not return any value and never fails. It will always increment the semaphore count up to its maximum, silently ignoring any attempts to exceed the limit.

Error/Edge case conf:0.85 **Q:** What are the most common mistakes when using ca_sem_post_capped?
A: Common mistakes include using a semaphore that has not been initialized, calling ca_sem_post_capped from a non-ISR context without proper synchronization, and assuming the function performs error checking. The function assumes correct initialization and context, and does not provide safeguards.

`ca_sem_timedwait`

Header: `caos/inc/sync.h:198`
Source: `caos/src/sync.c:439:461`
GUARD: `EN_SYNC`

```
bool ca_sem_timedwait(ca_semaphore_t *sem, u32 timeout_ms)
```

Attempts to acquire a semaphore, blocking the current task until it is available or the timeout expires.

Parametri:

Nome	Tipo	Descrizione
`sem`	`ca_semaphore_t *`	Pointer to the initialized semaphore object. Must not be NULL.
`timeout_ms`	`u32`	Maximum time in milliseconds to wait. Use 0 for non-blocking check, or a positive value for a timed wait.

Ritorna:

Returns true if the semaphore was successfully acquired (decremented). Returns false if the timeout elapsed before the semaphore became available.

Note:

- This function is blocking, meaning the calling task will be suspended by the OS scheduler while waiting, allowing other tasks to run. It is NOT safe to call from an ISR because it performs task-level blocking operations; use non-blocking alternatives or ISR-specific signaling if you need to interact with semaphores from an interrupt. Ensure the semaphore is initialized via ca_sem_init before use. A common mistake is passing an uninitialized semaphore pointer or calling this from an interrupt context, which will trigger a system safeguard failure or kernel panic.

Uso:

- This function is typically used to synchronize tasks, such as waiting for a hardware event, a network response, or a data processing completion. It is often paired with ca_sem_init during system setup and ca_sem_post in the event-generating task. In a typical sequence, a task initiates an asynchronous operation (like a network request), calls ca_sem_timedwait to wait for the result, and then processes the data once the semaphore is released. It is frequently used in service patterns where a task must block until a specific flag or resource becomes available, ensuring efficient CPU usage by yielding control while waiting.

Esempio:

```
/* Initialize semaphore with 0 count (starts locked) */
caSemaphore my_sem;
ca_sem_init(&my_sem, "my_sem", 0, 1);

/* Task waiting for a signal */
void worker_task(void) {
    /* Wait up to 500ms for the signal */
    if (ca_sem_timedwait(&my_sem, 500)) {
        /* Semaphore acquired, proceed with processing */
    } else {
        /* Timeout reached, handle error or retry */
    }
}

/* Signal task */
void signal_task(void) {
    /* Release the semaphore to wake the waiting task */
    ca_sem_post(&my_sem);
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_sem_timedwait in a network service like ARP or DHCP?
A: You should use ca_sem_timedwait when you need to wait for a specific event or resource with a timeout, such as waiting for a network response or a lock to become available. In network services, this is useful to avoid indefinite blocking while ensuring timely operation.

Code example conf:0.95 **Q:** Can you show a minimal example of using ca_sem_timedwait with ca_semaphore_t?
A: ca_semaphore_t sem; ca_sem_init(&sem, 0); bool result = ca_sem_timedwait(&sem, 1000); // Wait up to 1 second if (result) { // Semaphore was signaled } else { // Timeout occurred }

Parameters conf:0.90 **Q:** What happens if I pass NULL as the semaphore or a very large timeout value?

A: Passing NULL as the semaphore parameter will cause a crash. If the timeout_ms value is too large, it may cause an overflow, leading to unexpected behavior or incorrect timeout calculation.

Return conf:0.90 **Q:** What does it mean when ca_sem_timedwait returns false?

A: ca_sem_timedwait returns false if the timeout expires without the semaphore being signaled, or if an internal error occurs. It also returns false immediately if timeout_ms is 0 and the semaphore is not available.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_sem_timedwait?

A: Common mistakes include passing a NULL semaphore pointer, using a timeout value that is too large and causes overflow, calling the function from an ISR, or not initializing the semaphore with ca_sem_init() before use.

Behavior conf:0.90 **Q:** Is ca_sem_timedwait safe to call from an ISR, and is it blocking?

A: ca_sem_timedwait is not safe to call from an ISR, as it is not ISR-safe. It is blocking only when the timeout is greater than 0 and the semaphore is not available, at which point the calling thread will suspend until signaled or timeout.

`ca_sem_trywait`

Header: `caos/inc/sync.h:190`
Source: `caos/src/sync.c:426:437`
GUARD: `EN_SYNC`

```
1 bool ca_sem_trywait(ca_semaphore_t *sem)
```

Attempts to acquire a semaphore immediately without blocking the calling task.

Parametri:

Nome	Tipo	Descrizione
`sem`	`ca_semaphore_t *`	Pointer to the semaphore object. Must be initialized via ca_sem_init before use.

Ritorna:
Returns true if the semaphore was successfully acquired (count was > 0 and decremented). Returns false if the semaphore count was 0, meaning the resource is currently unavailable.

Note:

- This function is non-blocking, making it ideal for polling or clearing stale signals in high-performance loops. It is thread-safe because it uses internal spinlocks to protect the semaphore count during the check-and-decrement operation. It is safe to call from an ISR, but note that it does not wait; if the resource is busy, it returns immediately. A common mistake is assuming this function will wait for a resource to become available; if you need to wait for a specific duration, use ca_sem_timedwait instead.

Uso:

- This function is typically used in two scenarios: first, to 'drain' or clear stale signals from a semaphore before starting a new operation, ensuring that old events do not trigger logic prematurely. Second, it is used in non-blocking state machines where the task must perform other work if a resource is not immediately ready, rather than stalling the entire thread. It is often paired with ca_sem_post to release the resource and ca_sem_init to set up the semaphore initially. In network services, it is frequently used to reset the state of a synchronization primitive before initiating a new request-response cycle.

Esempio:

```c
/* Example: Clearing stale signals before starting a network operation */
caSemaphore my_sem;

void service_init(void) {
    /* Initialize semaphore with 0 count */
    ca_sem_init(&my_sem, "my_sem", 0, 1);
}

void perform_network_task(void) {
    /* Drain any pending signals from previous cycles to ensure a clean state */
    while (ca_sem_trywait(&my_sem));

    /* Trigger an operation that will eventually post to the semaphore */
    /* ... trigger network request ... */

    /* Wait for the operation to complete with a timeout */
    if (ca_sem_timedwait(&my_sem, 100)) {
        /* Success: resource acquired */
    } else {
        /* Timeout: resource not ready, handle error */
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_sem_trywait() instead of ca_sem_wait()?
A: You should use ca_sem_trywait() when you want to attempt to acquire a semaphore without blocking the calling thread. It is useful in scenarios where you need non-blocking synchronization, such as in interrupt service routines or when checking resource availability without delay. Unlike ca_sem_wait(), which blocks until the semaphore is available, ca_sem_trywait() returns immediately if the semaphore is not available.

Code example conf:0.90 **Q:** Can you show a minimal example of using ca_sem_trywait() with caOS types?
A: ca_semaphore_t sem; ca_sem_init(&sem, 1); bool acquired = ca_sem_trywait(&sem); if (acquired) { // Semaphore was acquired ca_sem_post(&sem); }

Parameters conf:0.95 **Q:** What happens if I pass NULL to the sem parameter of ca_sem_trywait()?
A: Passing NULL to the sem parameter of ca_sem_trywait() will cause a crash because the function does not perform a NULL check before dereferencing the pointer. The parameter must be a valid pointer to an initialized ca_semaphore_t structure.

Return conf:0.95 **Q:** What does it mean when ca_sem_trywait() returns false?
A: When ca_sem_trywait() returns false, it means that the semaphore was not available (its count was zero). The function does not block; it immediately returns false to indicate that the semaphore could not be acquired. The calling thread continues execution without waiting.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_sem_trywait()?
A: Common mistakes include using ca_sem_trywait() on a semaphore that has not been initialized with ca_sem_init(), which leads to undefined behavior or crash. Another mistake is passing a NULL pointer as the sem parameter, which also causes a crash. Additionally, using ca_sem_trywait() for synchronization where blocking is required can lead to inefficient polling behavior.

`ca_sem_wait`

Header: `caos/inc/sync.h:183`
Source: `caos/src/sync.c:401:424`

GUARD: `EN_SYNC`

```
void ca_sem_wait(ca_semaphore_t *sem)
```

Blocks the current task until the specified semaphore becomes available.

Parametri:

Nome	Tipo	Descrizione
sem	ca_semaphore_t *	Pointer to a previously initialized semaphore object. Must not be NULL.

Ritorna:
This function returns void. If the semaphore is available, it decrements the count and returns immediately. If unavailable, it blocks the task until another thread calls ca_sem_post.

Note:

- This function is blocking: the calling task will stop execution and yield to the scheduler if the semaphore count is zero. It is NOT safe to call this function from an Interrupt Service Routine (ISR) because ISRs cannot be blocked. Ensure the semaphore is initialized with ca_sem_init before use. A common mistake is calling this from an ISR or failing to provide a corresponding ca_sem_post, which leads to a permanent task hang (deadlock). Always ensure the task context is valid before calling, as calling this from an invalid context will trigger a system safeguard.

Uso:

- Use this function to synchronize tasks or protect shared resources. It is typically used in a producer-consumer pattern where one task waits for data or a hardware event signaled by another task or an ISR. In a typical sequence, you initialize the semaphore with ca_sem_init, then use ca_sem_wait to acquire access to a resource, perform your work, and finally release it with ca_sem_post. It is frequently used in driver code to wait for hardware completion interrupts, such as waiting for a DMA transfer or a flash memory operation to finish.

Esempio:

```
/* Example: Waiting for a hardware resource to be ready */
caSemaphore my_sem;

void task_function(void *arg) {
    /* Initialize semaphore with 0 count so it starts in a blocked state */
    ca_sem_init(&my_sem, 0);

    while (1) {
        /* Wait for the resource to be signaled by an ISR or another task */
        ca_sem_wait(&my_sem);

        /* Perform critical operation safely */
        ca_printf("Resource acquired, processing data...\n");
    }
}

/* In an ISR or another task, signal the semaphore to unblock the waiter */
void signal_resource(void) {
    ca_sem_post(&my_sem);
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_sem_wait() in a caOS application?
A: You should use ca_sem_wait() when you need to synchronize access to a shared resource or implement conditional waiting. It blocks the current task until the semaphore becomes available, ensuring mutual exclusion or coordination between tasks.

Code example conf:0.90 **Q:** Can you show a minimal example of using ca_sem_wait() with caOS types?
A: ca_semaphore_t sem; ca_sem_init(&sem, 1); ca_sem_wait(&sem); // Critical section ca_sem_post(&sem);

Parameters conf:0.95 **Q:** What is the significance of the 'sem' parameter in ca_sem_wait()?
A: The 'sem' parameter is a pointer to a ca_semaphore_t that must be initialized with ca_sem_init(). It represents the semaphore to wait on. Passing NULL causes undefined behavior or crash.

Return conf:0.95 **Q:** Does ca_sem_wait() return any value, and what does it signify?
A: ca_sem_wait() does not return any value. It blocks the calling task until the semaphore becomes available. If the semaphore is unavailable, the task is suspended until another task calls ca_sem_post().

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_sem_wait()?
A: Common mistakes include calling ca_sem_wait() on an uninitialized semaphore, calling it from an ISR (which is not allowed), or passing a NULL pointer. These can lead to crashes or undefined behavior.

Behavior conf:0.95 **Q:** Is ca_sem_wait() blocking or non-blocking, and can it be called from an ISR?
A: ca_sem_wait() is a blocking function. It will suspend the calling task until the semaphore is available. It is not safe to call from an ISR context and must only be used from task context.

`ca_sha256`

Header: `caos/crypto/inc/sha256.h:84`
Source: `caos/crypto/src/sha256.c:202:208`
GUARD: `EN_SHA256`

```
1 void ca_sha256(const u8 *data, u32 len, u8 *digest)
```

Computes the SHA-256 hash of a data buffer and stores the result in a provided digest array.

Parametri:

Nome	Tipo	Descrizione
`data`	`const u8 *`	Pointer to the input data buffer to be hashed. Must not be NULL.
`len`	`u32`	Length of the input data in bytes.
`digest`	`u8 *`	Pointer to a buffer of at least 32 bytes where the resulting hash will be stored.

Ritorna:
This function returns void. It performs the operation synchronously and assumes the provided pointers are valid.

Note:

- This function is a blocking wrapper that performs the entire SHA-256 process (init, update, final) in one call. It is thread-safe in the sense that it uses a local stack-allocated context, but it is computationally intensive; calling it from an ISR is strongly discouraged as it will block the CPU for the duration of the hashing process. Ensure the 'digest' buffer is at

least 32 bytes long to avoid memory corruption. A common mistake is passing a NULL pointer for 'data' or 'digest', which will lead to a hard fault.

Uso:

- Use this function when you need a quick, one-shot SHA-256 hash of a data block, such as verifying a firmware image or generating a key fingerprint. It is typically used in high-level application logic where the data is already available in memory. If you are processing a stream of data that arrives in chunks, do not use this function; instead, use the sequence ca_sha256_init, ca_sha256_update, and ca_sha256_final to process the data incrementally.

Esempio:

```
u8 my_data[] = "Hello World";
u8 my_digest[32];

// Compute the hash of the data
ca_sha256(my_data, sizeof(my_data) - 1, my_digest);

// Log the result using the system hex dump utility
ca_hexdump_u32(my_digest, 32);
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_sha256 instead of the incremental SHA-256 functions?
A: Use ca_sha256 when you need a quick, one-shot hash of a data block already available in memory, such as verifying a firmware image or generating a key fingerprint. If you are processing a stream of data that arrives in chunks, you should use the incremental sequence of ca_sha256_init, ca_sha256_update, and ca_sha256_final instead.

Code example conf:1.00 **Q:** How do I implement a basic SHA-256 hash calculation using caOS types?
A: You can compute the hash by providing a data buffer and a 32-byte destination buffer as shown: u8 data[] = {0x01, 0x02, 0x03}; u8 digest[32]; ca_sha256(data, 3, digest);

Parameters conf:1.00 **Q:** What are the requirements for the 'digest' parameter in ca_sha256?
A: The 'digest' parameter must be a pointer to a buffer of at least 32 bytes. Providing a buffer smaller than 32 bytes will lead to memory corruption, and passing a NULL pointer will result in a hard fault.

`ca_sha256_final`

Header: `caos/crypto/inc/sha256.h:76`
Source: `caos/crypto/src/sha256.c:171:200`
GUARD: `EN_SHA256`

```
void ca_sha256_final(ca_sha256_ctx_t *ctx, u8 *digest)
```

Completes the SHA256 hashing process, produces the final digest, and clears the context.

Parametri:

Nome	Tipo	Descrizione
`ctx`	`ca_sha256_ctx_t *`	Pointer to the SHA256 context structure. Must be initialized via ca_sha256_init.
`digest`	`u8 *`	Pointer to a buffer of at least 32 bytes where the final hash result will be stored.

Ritorna:

void. If ctx or digest are NULL, the function returns immediately without performing any operation.

Note:

- This function is not thread-safe; you must ensure that no other task or ISR accesses the same context object while this function is running. It is a blocking operation because it performs the final padding and compression cycles required to finalize the hash. It is not safe to call from an ISR due to the potential duration of the final compression. A common beginner mistake is to reuse the context structure for a new hash calculation without calling ca_sha256_init again, or failing to provide a large enough buffer for the 32-byte digest, which leads to memory corruption.

Uso:

- This function is the final step in the hashing sequence: ca_sha256_init -> ca_sha256_update (called one or more times) -> ca_sha256_final. It is used when the entire data stream has been processed and you need the resulting fingerprint. Because it clears the context structure using ca_memset, the context is effectively destroyed and cannot be used for further updates after this call.

Esempio:

```
ca_sha256_ctx_t ctx;
u8 digest[32];
char *data = "Hello World";

// Initialize the context
ca_sha256_init(&ctx);

// Process data in chunks
ca_sha256_update(&ctx, (u8*)data, 11);

// Finalize and get the result
ca_sha256_final(&ctx, digest);

// The context is now zeroed out and safe to discard or re-initialize
ca_printf("Hash computed successfully.\n");
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** At what point in the hashing workflow should I call ca_sha256_final?
A: You should call ca_sha256_final after you have completed all calls to ca_sha256_update. It is the final step in the sequence: ca_sha256_init -> ca_sha256_update -> ca_sha256_final, used to generate the final 32-byte fingerprint of the processed data stream.

Code example conf:0.90 **Q:** Provide a minimal C example showing how to finalize a SHA256 hash in caOS.
A: ca_sha256_ctx_t ctx; u8 digest[32]; ca_sha256_init(&ctx); ca_sha256_update(&ctx, data, len); ca_sha256_final(&ctx, digest);

Parameters conf:1.00 **Q:** What are the requirements for the digest parameter in ca_sha256_final?
A: The digest parameter must be a pointer to a buffer that is at least 32 bytes in size. If the pointer provided is NULL, the function will return immediately without performing any operation.

Error/Edge case conf:1.00 **Q:** What happens if I try to use the context structure again after calling ca_sha256_final?
A: The context is effectively destroyed because ca_sha256_final clears the structure using ca_memset. You must call ca_sha256_init again before reusing the context structure for a new hash calculation.

Behavior conf:1.00 **Q:** Is it safe to call ca_sha256_final from an interrupt service routine (ISR) or concurrently from multiple tasks?
A: No, it is not safe to call from an ISR due to the potential duration of the final compression. Furthermore, it is not thread-safe; you must ensure no other task or ISR accesses the context while this blocking function is executing.

`ca_sha256_init`

Header: `caos/crypto/inc/sha256.h:61`
Source: `caos/crypto/src/sha256.c:112:129`
GUARD: `EN_SHA256`

```
void ca_sha256_init(ca_sha256_ctx_t *ctx)
```

Initializes the SHA-256 hashing context structure with standard constants.

Parametri:

Nome	Tipo	Descrizione
`ctx`	`ca_sha256_ctx_t *`	Pointer to the context structure to be initialized. Must not be NULL.

Ritorna:
void

Note:

- This function is non-blocking and thread-safe as it only performs local memory writes to the provided structure. It is safe to call from an ISR, provided the context pointer is valid. The function must be called before any call to ca_sha256_update or ca_sha256_final to ensure the internal state variables are set to the correct SHA-256 initial hash values. A common mistake is to reuse a context structure without re-initializing it, which will result in corrupted hash calculations.

Uso:

- This function is the mandatory first step in the SHA-256 hashing lifecycle. It is typically used in a sequence: first, allocate or declare the context structure, then call ca_sha256_init to prepare it, followed by one or more calls to ca_sha256_update to process data chunks, and finally ca_sha256_final to retrieve the resulting digest. It is used whenever you need to verify data integrity or generate a cryptographic fingerprint of a data stream.

Esempio:

```
ca_sha256_ctx_t my_ctx;
uint8_t data[] = "Hello caOS";
uint8_t digest[32];

// Initialize the context before processing
ca_sha256_init(&my_ctx);

// Update the context with data
ca_sha256_update(&my_ctx, data, 11);

// Finalize the hash and store result in digest
ca_sha256_final(&my_ctx, digest);

// Print the resulting hash
ca_hexdump(digest, 32);
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_sha256_init in my application?
A: You must call ca_sha256_init as the mandatory first step in the SHA-256 hashing lifecycle, specifically before calling ca_sha256_update or ca_sha256_final, to ensure internal state variables are correctly set.

Code example conf:1.00 **Q:** Provide a minimal C example showing how to initialize the SHA-256 context.
A: ca_sha256_ctx_t ctx; ca_sha256_init(&ctx);

Parameters conf:1.00 **Q:** What are the requirements for the ctx parameter in ca_sha256_init?
A: The ctx parameter must be a pointer to a ca_sha256_ctx_t structure and must not be NULL.

Error/Edge case conf:1.00 **Q:** What is a common mistake when using the SHA-256 context structure?
A: A common mistake is to reuse a context structure without re-initializing it, which will result in corrupted hash calculations.

Behavior conf:1.00 **Q:** Is it safe to call ca_sha256_init from an interrupt service routine (ISR)?
A: Yes, the function is non-blocking and thread-safe as it only performs local memory writes. It is safe to call from an ISR provided the context pointer is valid.

`ca_sha256_selftest`

Header: `caos/crypto/inc/sha256.h:95`
Source: `caos/crypto/src/sha256.c:256:343`
GUARD: `EN_SHA256`

```
u32 ca_sha256_selftest(void)
```

Runs a suite of NIST FIPS 180-4 standard tests to verify the integrity and correctness of the SHA-256 implementation.

Parametri:

Nome	Tipo	Descrizione
`void`	`void`	This function takes no arguments.

Ritorna:
Returns a u32 representing the total number of failed test cases. A return value of 0 indicates that all tests passed successfully.

Note:

- This function is intended for diagnostic and validation purposes during system startup or maintenance. It is not thread-safe and should not be called from an ISR, as it performs significant computation and potentially uses blocking I/O for reporting. Before calling this, ensure the system clock and basic memory management are initialized, as the function relies on stack allocation and internal crypto buffers. A common mistake is calling this function repeatedly in a high-frequency loop; it is designed to be run once during the boot sequence or as a periodic health check, not as part of the main control loop.

Uso:

- Typically used during the system initialization phase to ensure the cryptographic hardware or software library is functioning correctly before enabling secure communication services. It is often paired with other self-test functions like ca_sha512_selftest or ca_aes_ecb_selftest. Developers should call this after the OS scheduler is running but before starting critical tasks that depend on SHA-256 hashing, such as firmware verification or TLS handshake processes.

Esempio:

```c
/* Example: Running SHA-256 self-test during system startup */

void system_startup_check(void) {
    u32 sha_fails = 0;

    /* Run the self-test suite */
    sha_fails = ca_sha256_selftest();

    if (sha_fails > 0) {
        /* If tests fail, log the error and potentially halt or enter a safe state */
        ca_printf_n("CRITICAL: SHA-256 self-test failed with %u errors", sha_fails);
        /* Perform system recovery or enter infinite loop */
        while(1) {
            ca_os_thread_reset_watchdog();
            ca_os_delay_ms(1000);
        }
    } else {
        ca_printf_n("SHA-256 integrity verified.");
    }
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to execute ca_sha256_selftest in my caOS application?
A: It should be called during the system initialization phase, specifically after the OS scheduler is running but before starting critical tasks that rely on SHA-256, such as firmware verification or TLS handshakes.

Code example conf:1.00 **Q:** Provide a minimal C code snippet to perform the SHA-256 self-test.
A: u32 result = ca_sha256_selftest(); if (result == 0) { /* Success */ } else { /* Handle failure */ }

Parameters conf:1.00 **Q:** What parameters does ca_sha256_selftest require?
A: The function takes no arguments (void).

Return conf:1.00 **Q:** How do I interpret the u32 return value from ca_sha256_selftest?
A: The return value represents the total number of failed test cases. A return value of 0 indicates that all NIST FIPS 180-4 standard tests passed successfully.

Error/Edge case conf:1.00 **Q:** What are common mistakes to avoid when using ca_sha256_selftest?
A: A common mistake is calling the function repeatedly in a high-frequency loop. It is designed for one-time execution during boot or as a periodic health check, not for the main control loop.

Behavior conf:1.00 **Q:** Is ca_sha256_selftest safe to call from an ISR or concurrently from multiple threads?
A: No, it is not thread-safe and must not be called from an ISR because it performs significant computation and may use blocking I/O. Ensure the system clock and memory management are initialized before calling it.

`ca_sha256_test_init`

Header: `caos/crypto/inc/sha256.h:101`
Source: `caos/crypto/src/sha256.c:368:375`
GUARD: `EN_SHA256`

```c
void ca_sha256_test_init(void)
```

Registers the SHA256 test command into the system CLI for debugging purposes.

Ritorna:
This function returns nothing (void).

Note:

- This function is not thread-safe and should be called during the system initialization phase, typically before the scheduler starts. It is not intended to be called from an ISR. The function uses a static flag to ensure the CLI node is registered only once, preventing duplicate entries in the command list. A common mistake is calling this function after the CLI has already been accessed or after the system has started processing tasks, which might lead to race conditions if the CLI structure is being traversed.

Uso:

- This function is used to enable interactive testing of the SHA256 cryptographic module via the system's Command Line Interface (CLI). It is typically invoked in the main initialization sequence, often guarded by preprocessor macros like 'EVAL_SHA256_TEST'. It pairs with 'ca_cli_register' internally to link the SHA256 test handler to the CLI system, allowing developers to trigger cryptographic tests manually during runtime.

Esempio:

```
void ca_os_main(void) {
    /* Initialize system hardware and OS services */
    ca_os_tick_init();

    /* Register the SHA256 test command to the CLI */
    ca_sha256_test_init();

    /* Start the OS scheduler */
    ca_os_run(true);
}
```

Q&A — fine-tuning review *2/2 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to call ca_sha256_test_init in my caOS application?
A: You should call ca_sha256_test_init during the system initialization phase, specifically before the scheduler starts. It is not thread-safe and must not be called from an ISR, as it is intended to register the SHA256 test command into the CLI before the system begins processing tasks.

Code example conf:1.00 **Q:** How do I implement the SHA256 test initialization in my main initialization sequence?
A: You should invoke the function within your main initialization routine, typically guarded by a preprocessor macro as shown:
#ifdef EVAL_SHA256_TEST ca_sha256_test_init(); #endif

`ca_sha256_update`

Header: `caos/crypto/inc/sha256.h:69`
Source: `caos/crypto/src/sha256.c:131:169`
GUARD: `EN_SHA256`

```
void ca_sha256_update(ca_sha256_ctx_t *ctx, const u8 *data, u32 len)
```

Processes a chunk of data to update the internal state of a SHA-256 hashing operation.

Parametri:

Nome	Tipo	Descrizione
`ctx`	`ca_sha256_ctx_t *`	Pointer to the SHA-256 context structure. Must be initialized via ca_sha256_init before use.
`data`	`const u8 *`	Pointer to the input data buffer to be hashed. Can be NULL only if len is 0.
`len`	`u32`	Length of the data in bytes. Must be a valid positive integer.

Ritorna:
void. This function does not return a status code; it silently ignores invalid pointers or null data inputs.

Note:

- This function is not thread-safe; you must ensure that only one task accesses the specific context at a time, typically by using a mutex or by keeping the context local to a single task. It is not designed to be called from an ISR because it performs internal memory copies and potential compression operations that may take significant time. The function is non-blocking in the sense that it does not wait for external hardware, but it performs CPU-intensive hashing work. A common beginner mistake is to pass an uninitialized context or a pointer to a context that has already been finalized by ca_sha256_final, which will lead to corrupted hash results.

Uso:

- This function is used in the middle of a hashing sequence: first you initialize the context with ca_sha256_init, then you call ca_sha256_update one or more times to feed the data chunks into the engine, and finally you call ca_sha256_final to retrieve the resulting digest. It is typically used when processing large files or network streams that do not fit into memory all at once, allowing you to hash data in smaller, manageable segments.

Esempio:

```
ca_sha256_ctx_t my_ctx;
u8 buffer[64];

// Initialize the context before starting
ca_sha256_init(&my_ctx);

// Simulate reading data from a source
// Assume 'read_data_from_source' fills the buffer and returns bytes read
int bytes_read = 64;

if (bytes_read > 0) {
    // Update the hash with the current chunk
    ca_sha256_update(&my_ctx, buffer, (u32)bytes_read);
}

// Finalize the hash to get the result
u8 digest[32];
ca_sha256_final(&my_ctx, digest);
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_sha256_update in my hashing workflow?
A: You should use ca_sha256_update after initializing the context with ca_sha256_init and before calling ca_sha256_final. It is intended for processing large files or network streams in smaller, manageable segments that do not fit into memory all at once.

Code example conf:0.90 **Q:** How do I implement a basic SHA-256 update sequence using caOS types?

A: ca_sha256_ctx_t ctx; u8 buffer[] = {0x01, 0x02, 0x03}; ca_sha256_init(&ctx); ca_sha256_update(&ctx, buffer, 3); ca_sha256_final(&ctx, result_buffer);

Parameters conf:1.00 **Q:** What are the constraints for the data and len parameters in ca_sha256_update?

A: The data parameter is a pointer to the input buffer and can only be NULL if len is 0. The len parameter must be a valid positive integer representing the length of the data in bytes.

Error/Edge case conf:1.00 **Q:** What happens if I pass an uninitialized context or a finalized context to ca_sha256_update?

A: Passing an uninitialized context or a pointer to a context that has already been finalized by ca_sha256_final will lead to corrupted hash results.

Behavior conf:1.00 **Q:** Is ca_sha256_update safe to call from an ISR or across multiple tasks?

A: No, it is not thread-safe and must be protected by a mutex or kept local to a single task. It is also not designed to be called from an ISR because it performs CPU-intensive memory copies and compression operations that may take significant time.

`ca_sha512`

Header: `caos/crypto/inc/sha512.h:81`
Source: `caos/crypto/src/sha512.c:212:218`
GUARD: `EN_SHA512`

```
void ca_sha512(const u8 *data, u32 len, u8 *digest)
```

Computes a SHA-512 hash of a data buffer in a single synchronous operation.

Parametri:

Nome	Tipo	Descrizione
`data`	`const u8 *`	Pointer to the input data buffer to be hashed. Must not be NULL.
`len`	`u32`	Length of the input data in bytes.
`digest`	`u8 *`	Pointer to a buffer of at least 64 bytes where the resulting hash will be stored.

Ritorna:
This function returns void. It assumes the operation completes successfully.

Note:

- This is a blocking, synchronous function. It performs the entire hashing process (init, update, final) before returning, meaning the CPU will be occupied for the duration of the calculation. It is thread-safe as it uses a local stack-allocated context, but it should not be called from an ISR if the data buffer is large, as it could cause significant latency. Ensure the 'digest' buffer is large enough to hold 64 bytes to avoid memory corruption.

Uso:

- Use this function when you need a quick, one-shot SHA-512 hash of a complete data block already available in memory. It is commonly used in cryptographic operations, such as generating keys or verifying data integrity, where the entire input is known upfront. It is often paired with memory management functions like ca_memory_allocate if the input data is dynamic, or used directly on static buffers before passing the result to other crypto functions like ca_ed25519_sign.

Esempio:

```
u8 my_data[] = "Hello World";
u8 hash_result[64];
```

```

/* Compute the SHA-512 hash of the data buffer */
ca_sha512(my_data, sizeof(my_data) - 1, hash_result);

/* Output the result using the system log or debug interface */
ca_hexdump_u32(hash_result, 64);
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_sha512 instead of a streaming hashing approach?
A: You should use ca_sha512 when you need a quick, one-shot hash of a complete data block that is already available in memory. It is ideal for cryptographic operations like key generation or data integrity verification where the entire input is known upfront.

Code example conf:1.00 **Q:** How do I implement a basic SHA-512 hash calculation using caOS types?
A: You must provide a source buffer and a destination buffer of at least 64 bytes. Example: u8 data[] = {0x01, 0x02}; u8 hash[64]; ca_sha512(data, 2, hash);

Parameters conf:1.00 **Q:** What are the constraints for the 'digest' parameter in ca_sha512?
A: The 'digest' parameter must be a pointer to a buffer of at least 64 bytes. Providing a buffer smaller than 64 bytes will result in memory corruption.

`ca_sha512_final`

Header: `caos/crypto/inc/sha512.h:73`
Source: `caos/crypto/src/sha512.c:179:210`
GUARD: `EN_SHA512`

```
void ca_sha512_final(ca_sha512_ctx_t *ctx, u8 *digest)
```

Finalizes the SHA-512 hashing process, produces the digest, and clears the sensitive context.

Parametri:

Nome	Tipo	Descrizione
`ctx`	`ca_sha512_ctx_t *`	Pointer to the SHA-512 context structure. Must be initialized via ca_sha512_init before use.
`digest`	`u8 *`	Pointer to a buffer of at least 64 bytes where the final hash result will be stored.

Ritorna:
This function returns void. If ctx or digest are NULL, the function returns immediately without performing any operations.

Note:

- This function is not thread-safe and must not be called from an ISR because it performs heavy computational work (compression) and memory clearing. It is a blocking operation that completes the padding of the message, performs the final compression, and writes the result to the provided buffer. A critical security feature is that it automatically wipes the context structure using ca_memset to zero, preventing sensitive data (like private keys used in Ed25519) from remaining in RAM. A common beginner mistake is attempting to use the context structure again after calling this function; once finalized, the context is invalid and must be re-initialized.

Uso:

- This function is the final step in the standard hashing sequence: ca_sha512_init -> ca_sha512_update (one or more times) -> ca_sha512_final. It is typically used when

the entire message has been processed and the final signature or hash value is required for cryptographic operations. In the caOS codebase, it is frequently paired with Ed25519 signing routines where the hash of a message is needed to derive a nonce or a scalar value.

Esempio:

```
ca_sha512_ctx_t ctx;
u8 digest[64];
u8 message[] = "Hello World";

// Initialize the hashing context
ca_sha512_init(&ctx);

// Process the data
ca_sha512_update(&ctx, message, 11);

// Finalize and get the result
ca_sha512_final(&ctx, digest);

// The context is now zeroed and safe; do not use it again without re-init
// The digest buffer now contains the 64-byte SHA-512 hash
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_sha512_final in my cryptographic workflow?
A: You should call it as the final step in the sequence: ca_sha512_init, followed by one or more calls to ca_sha512_update, and finally ca_sha512_final. It is used when the entire message has been processed and you need the final hash result for operations like Ed25519 signing.

Code example conf:1.00 **Q:** Provide a minimal C example of how to finalize a SHA-512 hash in caOS.
A: ca_sha512_ctx_t ctx; u8 digest[64]; ca_sha512_init(&ctx); ca_sha512_update(&ctx, data, len); ca_sha512_final(&ctx, digest);

Parameters conf:1.00 **Q:** What are the requirements for the digest parameter in ca_sha512_final?
A: The digest parameter must be a pointer to a buffer of at least 64 bytes. If the pointer is NULL, the function returns immediately without performing any operations.

Error/Edge case conf:1.00 **Q:** What happens if I try to use the context structure after calling ca_sha512_final?
A: This is a common mistake; the context is invalid after the function is called because the function automatically wipes the context structure to zero for security. You must re-initialize the context using ca_sha512_init before using it again.

Behavior conf:1.00 **Q:** Is it safe to call ca_sha512_final from an interrupt service routine (ISR)?
A: No, it is not ISR-safe. It is a blocking operation that performs heavy computational work and memory clearing, and it is not thread-safe. It must not be called from an ISR.

`ca_sha512_init`

Header: `caos/crypto/inc/sha512.h:58`
Source: `caos/crypto/src/sha512.c:112:130`
GUARD: `EN_SHA512`

```
void ca_sha512_init(ca_sha512_ctx_t *ctx)
```

Initializes the SHA-512 hashing context structure with standard constants.

Parametri:

Nome	Tipo	Descrizione
ctx	ca_sha512_ctx_t *	Pointer to the context structure to be initialized. Must not be NULL.

Ritorna:
void

Note:

- This function is non-blocking and thread-safe as it only performs local memory writes to the provided structure. It is safe to call from an ISR, provided the context pointer is valid. The function sets the internal state to the SHA-512 initial hash values and resets the bit length and buffer counters. A common mistake is passing a NULL pointer or failing to initialize the context before calling ca_sha512_update, which will lead to undefined behavior or corrupted hash results.

Uso:

- This function is the mandatory first step in any SHA-512 hashing operation. It must be called before any calls to ca_sha512_update or ca_sha512_final. It is typically used in a sequence: init -> update (one or more times) -> final. It is used whenever you need to compute a cryptographic hash for data integrity, digital signatures, or key derivation.

Esempio:

```
ca_sha512_ctx_t ctx;
unsigned char data[] = "Hello World";
unsigned char hash[64];

// Initialize the context before processing data
ca_sha512_init(&ctx);

// Process the data buffer
ca_sha512_update(&ctx, data, 11);

// Finalize the hash and store the result in the hash array
ca_sha512_final(&ctx, hash);

// The hash is now ready for use
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_sha512_init in my hashing workflow?
A: ca_sha512_init is the mandatory first step in any SHA-512 operation. It must be called before any calls to ca_sha512_update or ca_sha512_final to initialize the context structure with standard constants.

Code example conf:1.00 **Q:** Provide a minimal C example of how to initialize a SHA-512 context in caOS.
A: ca_sha512_ctx_t ctx; ca_sha512_init(&ctx);

Parameters conf:1.00 **Q:** What are the requirements for the ctx parameter in ca_sha512_init?
A: The ctx parameter must be a pointer to a ca_sha512_ctx_t structure. It must not be NULL, as passing a NULL pointer will lead to undefined behavior.

Error/Edge case conf:1.00 **Q:** What happens if I forget to call ca_sha512_init before calling ca_sha512_update?
A: Failing to initialize the context before calling ca_sha512_update will lead to undefined behavior or corrupted hash results.

Behavior conf:1.00 **Q:** Is it safe to call ca_sha512_init from an interrupt service routine (ISR)?
A: Yes, the function is non-blocking and thread-safe because it only performs local memory writes. It is safe to call from an ISR, provided the context pointer is valid.

ca_sha512_selftest

Header: caos/crypto/inc/sha512.h:92
Source: caos/crypto/src/sha512.c:271:358
GUARD: EN_SHA512

```
u32 ca_sha512_selftest(void)
```

Executes a suite of NIST FIPS 180-4 known-answer tests to verify the integrity and correctness of the SHA-512 implementation.

Ritorna:
Returns a u32 representing the total number of failed tests. A return value of 0 indicates that all tests passed successfully. Any non-zero value indicates the count of specific test cases that failed to produce the expected hash digest.

Note:

- This function is intended for diagnostic and validation purposes during system startup or maintenance. It is a blocking function that performs intensive CPU calculations; therefore, it should never be called from an ISR or a high-priority real-time task that requires strict timing, as it will cause significant jitter. It is thread-safe in the sense that it uses local stack variables for context, but it is not re-entrant if multiple tasks attempt to run the self-test simultaneously. The most common beginner mistake is calling this function inside a time-critical control loop, which will lead to watchdog resets or missed deadlines.

Uso:

- Typically used during the system initialization phase or as part of a 'Built-In Self-Test' (BIST) routine before enabling network or security services. It is often paired with other cryptographic self-tests (like ca_x25519_selftest) and is usually wrapped in preprocessor macros (e.g., #if EN_SHA512_TEST) to allow disabling it in production builds to save boot time and flash memory. It serves as a gatekeeper to ensure the hardware or software crypto-acceleration is functioning correctly before the application starts processing sensitive data.

Esempio:

```
void system_startup_check(void) {
    u32 total_fails = 0;

    /* Run SHA-512 integrity verification */
    total_fails += ca_sha512_selftest();

    if (total_fails == 0) {
        ca_stdout_printf_n("System crypto integrity: OK");
    } else {
        /* Log failure and enter a safe state */
        ca_stdout_printf_n("System crypto integrity: FAILED (%u)", total_fails);
        /* Perform system halt or reset */
    }
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to execute ca_sha512_selftest in a caOS application?
A: It is typically used during the system initialization phase or as part of a Built-In Self-Test (BIST) routine. It acts as a gatekeeper to verify cryptographic integrity before enabling security services or processing sensitive data.

Code example conf:0.90 **Q:** How do I implement a basic check for the SHA-512 self-test in my startup code?
A: You should check the return value of the function, for example: u32 result = ca_sha512_selftest(); if (result != 0) { /* Handle failure */ }.

Return conf:1.00 **Q:** What does the u32 return value from ca_sha512_selftest signify?
A: The return value represents the total number of failed tests. A return value of 0 indicates that all NIST FIPS 180-4 known-answer tests passed successfully. Any non-zero value indicates the count of specific test cases that failed to produce the expected hash digest.

Error/Edge case conf:1.00 **Q:** What is the most common mistake developers make when using ca_sha512_selftest?
A: The most common mistake is calling this function inside a time-critical control loop. Because it is a blocking function that performs intensive CPU calculations, doing so will cause significant jitter, potentially leading to watchdog resets or missed deadlines.

Behavior conf:1.00 **Q:** Is ca_sha512_selftest safe to call from an ISR or concurrently from multiple tasks?
A: No, it should never be called from an ISR due to its blocking nature and intensive CPU usage. While it is thread-safe in that it uses local stack variables, it is not re-entrant, meaning multiple tasks should not attempt to run the self-test simultaneously.

`ca_sha512_test_init`

Header: `caos/crypto/inc/sha512.h:98`
Source: `caos/crypto/src/sha512.c:383:390`
GUARD: `EN_SHA512`

```
1 void ca_sha512_test_init(void)
```

Registers the SHA512 test command into the system CLI interface.

Ritorna:
This function returns nothing (void).

Note:

- This function is a wrapper that ensures the SHA512 diagnostic command is added to the Command Line Interface (CLI) exactly once. It is not thread-safe in the sense that it relies on a static flag to prevent multiple registrations, so it should be called during the system initialization phase before the scheduler starts. It is not intended to be called from an Interrupt Service Routine (ISR) as it performs memory registration operations. The most common mistake is calling this function after the CLI has already started processing commands or assuming it initializes the SHA512 hardware engine itself, which it does not.

Uso:

- This function is typically used during the system boot sequence, specifically within the main initialization routine before the RTOS scheduler is started via ca_os_run. It is often paired with other test initialization functions like ca_ed25519_test_init to populate the CLI with diagnostic tools. By wrapping the call in a conditional macro (like EVAL_SHA512_TEST), developers ensure that the test code is only included in builds where CLI debugging is enabled, keeping the production firmware footprint minimal.

Esempio:

```
1 void ca_os_main(void) {
2     /* Initialize system hardware and OS services */
```

```
    ca_os_tick_init();

    /* Register CLI commands for diagnostic testing */
    ca_sha512_test_init();
    ca_ed25519_test_init();

    /* Start the main application task */
    ca_os_job("main_task", CA_TH_LEVEL3, my_app_task, 0, STACK_LARGE, CA_JOB_STD
    , CA_FAULT_ASSERT);

    /* Enter the OS scheduler */
    ca_os_run(true);
}
```

Q&A — fine-tuning review *2/2 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_sha512_test_init and why is it important to call it before the scheduler starts?
A: You should call ca_sha512_test_init during the system boot sequence, specifically within the main initialization routine before the RTOS scheduler is started via ca_os_run. This is necessary because the function is not thread-safe and relies on a static flag to ensure the command is registered exactly once. Calling it early prevents issues with memory registration operations and ensures the CLI is properly populated before it begins processing commands.

Code example conf:0.95 **Q:** How do I correctly initialize the SHA512 test command in my main routine?
A: You should call the function during the system initialization phase. Example:
void main(void) { /* System initialization */ ca_sha512_test_init(); ca_ed25519_test_init(); /* Start the RTOS scheduler */ ca_os_run(); }

---

ca_sha512_update

Header: caos/crypto/inc/sha512.h:66
Source: caos/crypto/src/sha512.c:132:177
GUARD: EN_SHA512

```
void ca_sha512_update(ca_sha512_ctx_t *ctx, const u8 *data, u32 len)
```

Updates the SHA-512 hash context with a new chunk of data.

Parametri:

Nome	Tipo	Descrizione
ctx	ca_sha512_ctx_t *	Pointer to the SHA-512 context structure. Must be initialized via ca_sha512_init before use.
data	const u8 *	Pointer to the input data buffer. Can be NULL only if len is 0.
len	u32	Length of the input data in bytes. Must be a valid positive integer.

Ritorna:
void. This function does not return a status code; it silently ignores invalid inputs (NULL pointers).

Note:

- This function is non-blocking and performs pure computational work, making it safe to call from ISRs or high-priority tasks. It is not inherently thread-safe; if multiple tasks access

the same context, you must protect it with a mutex. The most common mistake is failing to call ca_sha512_init before the first update, which leaves the context in an undefined state, leading to incorrect hash results. Ensure the context structure is not modified by other processes while this function is executing.

Uso:

- This function is used in a three-stage pipeline: initialization, data processing, and finalization. You typically call ca_sha512_init to prepare the context, then call ca_sha512_update one or more times to feed the data stream into the hash engine. Finally, you call ca_sha512_final to retrieve the resulting digest. It is commonly used for streaming data, such as calculating the hash of a large file or a network packet stream where the entire data set is not available in memory at once.

Esempio:

```
ca_sha512_ctx_t ctx;
u8 digest[64];
u8 buffer[128];

// Initialize the context before processing
ca_sha512_init(&ctx);

// Simulate reading data from a source and updating the hash
// Assume 'read_data_from_source' fills the buffer and returns bytes read
int bytes_read = 128; // Example value
ca_sha512_update(&ctx, buffer, bytes_read);

// Finalize the hash calculation
ca_sha512_final(&ctx, digest);

// The digest is now ready in the 'digest' array
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_sha512_update in my caOS application?
A: You use ca_sha512_update as the second stage in the three-stage SHA-512 pipeline. It is intended for streaming data, allowing you to process large files or network packets in chunks when the entire dataset is not available in memory at once.

Code example conf:1.00 **Q:** How do I implement a basic SHA-512 update sequence using caOS types?
A: First, initialize the context, then update it with your data buffer: ca_sha512_ctx_t ctx; u8 data[] = {0x01, 0x02}; ca_sha512_init(&ctx); ca_sha512_update(&ctx, data, 2);

Parameters conf:1.00 **Q:** What are the constraints on the 'data' and 'len' parameters for ca_sha512_update?
A: The 'data' parameter is a pointer to the input buffer and can only be NULL if 'len' is 0. The 'len' parameter must be a valid positive integer representing the length of the data in bytes.

Error/Edge case conf:1.00 **Q:** What happens if I forget to call ca_sha512_init before calling ca_sha512_update?
A: Failing to call ca_sha512_init leaves the context in an undefined state, which will result in incorrect hash calculations. Additionally, the function silently ignores NULL pointers without returning an error code.

Behavior conf:1.00 **Q:** Is ca_sha512_update safe to call from an ISR or a multi-threaded environment?
A: The function is non-blocking and performs pure computational work, making it safe to call from ISRs. However, it is not inherently thread-safe; if multiple tasks access the same context, you must protect it with a mutex.

ca_skip_sp

Header: caos/inc/utils.h:164
GUARD: EN_UTILS

```
static inline const char *ca_skip_sp(const char *s)
```

Advances a string pointer past any leading space characters.

Parametri:

Nome	Tipo	Descrizione
s	const char *	Pointer to the null-terminated string to scan. Must be a valid memory address; passing NULL will cause a segmentation fault.

Ritorna:
A pointer to the first non-space character found in the string. If the string contains only spaces or is empty, it returns a pointer to the null terminator ('\0').

Note:

- This function is non-blocking, thread-safe, and safe to call from an ISR because it performs simple pointer arithmetic without accessing shared resources or OS primitives. It does not modify the original string, as it returns a pointer into the existing buffer. The most common mistake is assuming the returned pointer is always a valid character; if the input string is all spaces, the returned pointer will point to the null terminator, and attempting to read from it will result in reading a zero byte.

Uso:

- This function is primarily used in Command Line Interface (CLI) handlers to tokenize input strings. It is typically paired with ca_strncmp to skip whitespace before checking for command keywords or arguments. By calling it repeatedly, you can effectively parse space-separated parameters from a user-provided command string, ensuring that extra spaces between words do not break your parsing logic.

Esempio:

```
static bool my_cli_handler(c_str *pcmd) {
    c_str cmd = *pcmd;
    // Skip leading spaces to find the start of the command
    const char *p = ca_skip_sp(cmd->data);

    // Check if the user typed 'info'
    if (ca_strncmp(p, "info", 4) == 0) {
        // Use ca_printf to output system info
        ca_printf("System is running normally\n");
        return true;
    }

    // If no command matched, return false
    return false;
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_skip_sp in my caOS application?

A: You should use ca_skip_sp primarily within Command Line Interface (CLI) handlers to tokenize input strings. It is used to skip leading whitespace before checking for command keywords or arguments, allowing your parsing logic to handle extra spaces between words effectively.

Code example conf:0.90 **Q:** How do I implement a simple command check using ca_skip_sp?
A: You can use it to advance the pointer before comparing strings: cstr input = " start"; cstr cmd = ca_skip_sp(input); if (ca_strncmp(cmd, "start", 5) == 0) { /* handle command */ }

Parameters conf:1.00 **Q:** What are the requirements for the 's' parameter in ca_skip_sp?
A: The parameter 's' must be a pointer to a null-terminated string and must be a valid memory address. Passing a NULL pointer is invalid and will cause a segmentation fault.

Return conf:1.00 **Q:** What does the return value of ca_skip_sp represent, and what happens if the string is empty?
A: The function returns a pointer to the first non-space character. If the string is empty or contains only spaces, it returns a pointer to the null terminator ('\0'). Attempting to read from this returned pointer in that case will result in reading a zero byte.

`ca_snprintf`

Header: `caos/inc/utils.h:113`
Source: `caos/src/utils.c:435:441`
GUARD: `EN_UTILS`

```
1 int ca_snprintf( char *buffer, u32 size, const char *format, ...)
```

Formats and writes data into a character buffer with a size limit to prevent memory overflows.

Parametri:

Nome	Tipo	Descrizione
`buffer`	`char *`	Pointer to the destination character array where the formatted string will be stored.
`size`	`u32`	Maximum number of bytes to write, including the null-terminator. Must be greater than 0.
`format`	`const char *`	C-style format string containing text and format specifiers (e.g., %u, %s, %d).
`...`	`...`	Variable arguments corresponding to the format specifiers.

Ritorna:
Returns the number of characters that would have been written if the buffer were large enough (excluding the null-terminator). If the return value is greater than or equal to 'size', the output was truncated.

Note:

- This function is thread-safe and non-blocking, making it safe to use in most contexts. However, it is generally not recommended to call this from an ISR due to the potential CPU overhead of parsing complex format strings. The most common beginner mistake is providing a 'size' that is larger than the actual allocated buffer, which leads to buffer overflows and memory corruption. Always ensure the buffer is large enough to hold the expected string plus the null-terminator, or check the return value to verify if truncation occurred.

Uso:

- Used primarily for generating human-readable logs, constructing network packets, or formatting status messages for CLI interfaces. It is typically used in a sequence where a buffer is allocated or defined, filled with formatted data using ca_snprintf, and then passed to a transmission function like ca_stdout_put or a network send function. It is often paired with ca_vsnprintf if you are building your own custom logging wrappers, or used during initialization to generate unique names for OS objects like semaphores or tasks.

Esempio:

```c
char name_buf[16];
int written;

// Example: Generating a unique name for a semaphore during initialization
written = ca_snprintf(name_buf, sizeof(name_buf), "SEM_%02d", 5);

// Always verify the result to ensure the string was not truncated
if (written > 0 && written < (int)sizeof(name_buf)) {
    // Initialize the semaphore with the formatted name
    // ca_sem_init(handle, name, initial_count, max_count);
    ca_sem_init(&my_sem, name_buf, 0U, 1U);
} else {
    // Handle error: buffer was too small for the generated name
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_snprintf in a caOS project, and what are its main use cases?

A: ca_snprintf is used to format strings into a buffer, which is useful for creating log messages, naming resources, or preparing data to be sent. It is commonly used in driver code and utility functions where formatted output is needed without the overhead of a full printf implementation. It is non-blocking and suitable for use in task contexts but not in ISRs.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_snprintf with caOS types?

A:

```c
u8 buffer[64];
cstr format = "Device ID: %d";
int result = ca_snprintf((char*)buffer, 64, format, 123);
```

Parameters conf:0.90 **Q:** What are the valid and invalid values for the size parameter in ca_snprintf?

A: The size parameter must be greater than 0. If size is zero or less, the function will fail and return a negative value. The buffer must be large enough to hold the formatted string and a null terminator. A size of 0 is invalid and will cause the function to fail.

Return conf:0.85 **Q:** What does a negative return value from ca_snprintf indicate, and what causes it?

A: A negative return value from ca_snprintf indicates an error occurred. This can happen if the buffer is NULL, the size is zero or less, or if the format string is invalid. The function does not specify which error occurred, only that one did.

`ca_sp_link_accept`

Header: `caos/crypto/inc/secure_pipe_service.h:181`
Source: `caos/crypto/src/secure_pipe_service.c:352:395`
GUARD: `EN_SECURE_PIPE_SERVICE`

```c
i32 ca_sp_link_accept(ca_sp_link_t *l, const u8 *in, u32 len)
```

Processes an incoming handshake message to establish or verify a secure communication link.

Parametri:

Nome	Tipo	Descrizione
`l`	`ca_sp_link_t *`	Pointer to the secure pipe link structure. Must be initialized via ca_sp_link_init.
`in`	`const u8 *`	Buffer containing the incoming HELLO or HELLO_R message.
`len`	`u32`	Length of the input buffer. Must match CA_SP_LINK_HELLO_SIZE.

Ritorna:
CA_SECURE_PIPE_OK on success, CA_SECURE_PIPE_REPLY if a response is required, CA_SECURE_PIPE_REPLAY if the message is a duplicate, or CA_SECURE_PIPE_ERR/CA_SECURE_ on failure.

Note:

- This function is thread-safe as it internally locks the link mutex. It is a blocking operation because it performs cryptographic operations and mutex acquisition; therefore, it must NEVER be called from an ISR. Before calling this, ensure the link structure is fully initialized and the 'ready' flag is set. A common mistake is passing an uninitialized buffer or a buffer with incorrect length, which will immediately return CA_SECURE_PIPE_ERR without processing.

Uso:

- This function is the core of the secure handshake process. It is typically used in a task loop where incoming network packets are received via a queue. When a HELLO message is received, this function is called to validate the peer's identity and exchange nonces. If it returns CA_SECURE_PIPE_REPLY, the application must immediately send a response back to the peer using ca_sp_link_hello. It is used in the sequence: init -> loop(receive -> accept -> (if REPLY) send_hello).

Esempio:

```
/* Assuming 'link' is a valid ca_sp_link_t and 'msg_buf' contains incoming data */
i32 rc = ca_sp_link_accept(link, msg_buf, sizeof(msg_buf));

if (rc == CA_SECURE_PIPE_REPLY) {
    u8 resp[CA_SP_LINK_HELLO_SIZE];
    u32 resp_len = 0U;
    /* Generate response to the peer */
    if (ca_sp_link_hello(link, true, resp, sizeof(resp), &resp_len) == CA_SECURE_PIPE_OK) {
        /* Send 'resp' buffer over the network interface */
    }
} else if (rc == CA_SECURE_PIPE_OK) {
    /* Handshake complete, link is now secure */
} else {
    /* Handle error or replay attempt */
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_sp_link_accept in my application logic?
A: You should call it within a task loop when an incoming HELLO or HELLO_R message is received from a network queue. It is used to validate the peer's identity and exchange nonces as part of the secure handshake process.

Code example conf:0.90 **Q:** Provide a minimal C example of how to call ca_sp_link_accept.
A: ca_sp_link_t link; u8 buffer[CA_SP_LINK_HELLO_SIZE]; u32 len = CA_SP_LINK_HELLO_S ca_sp_link_init(&link); i32 res = ca_sp_link_accept(&link, buffer, len); if (res == CA_SECURE_PIPE_REPLY) { ca_sp_link_hello(&link); }

Parameters conf:1.00 **Q:** What are the requirements for the 'len' parameter in ca_sp_link_accept?
A: The 'len' parameter must exactly match the constant CA_SP_LINK_HELLO_SIZE. If the length does not match, the function will return CA_SECURE_PIPE_ERR without processing the buffer.

Return conf:1.00 **Q:** What does it mean if ca_sp_link_accept returns CA_SECURE_PIPE_REPLY?
A: It indicates that the handshake process is ongoing and the application must immediately send a response back to the peer using the ca_sp_link_hello function.

Error/Edge case conf:1.00 **Q:** What are common mistakes that cause ca_sp_link_accept to fail?
A: Common mistakes include passing an uninitialized link structure, providing a buffer with an incorrect length, or failing to ensure the 'ready' flag is set on the link structure before calling the function.

Behavior conf:1.00 **Q:** Is ca_sp_link_accept safe to call from an interrupt service routine (ISR)?
A: No, it must NEVER be called from an ISR. The function is a blocking operation because it performs cryptographic calculations and acquires a mutex; calling it from an ISR would violate these constraints.

`ca_sp_link_deinit`

Header: `caos/crypto/inc/secure_pipe_service.h:199`
Source: `caos/crypto/src/secure_pipe_service.c:514:528`
GUARD: `EN_SECURE_PIPE_SERVICE`

```
1 void ca_sp_link_deinit(ca_sp_link_t *l)
```

Safely tears down a secure pipe link, wipes cryptographic keys, and clears the link structure.

Parametri:

Nome	Tipo	Descrizione
`l`	`ca_sp_link_t *`	Pointer to the secure pipe link structure. Must be a valid, initialized pointer; NULL is ignored.

Ritorna:
void

Note:

- This function is thread-safe as it uses an internal mutex to protect the link state during teardown. It is NOT callable from an ISR because it performs blocking mutex operations. You must ensure that no other tasks are attempting to use the link while this function is running, or you may encounter race conditions or memory corruption. The most common mistake is calling this on a structure that has already been deinitialized or was never properly initialized, which can lead to undefined behavior if the pointer is invalid.

Uso:

- Use this function when a secure communication session needs to be terminated, such as during a graceful shutdown, a re-keying event, or when an error makes the current session invalid. It is typically paired with ca_sp_link_init to start the lifecycle and ca_sp_link_open or ca_sp_link_accept to establish the connection. Always call this before freeing the memory containing the link structure to ensure sensitive cryptographic keys are wiped from RAM.

Esempio:

```
/* Example: Gracefully closing a secure link when a session expires */
void close_session(ca_sp_link_t *link) {
    if (link != NULL) {
        /* Ensure we are not in an ISR before calling deinit */
        if (ca_os_running()) {
            ca_sp_link_deinit(link);
            /* After deinit, the structure is zeroed and safe to free */
            ca_memory_free(link, NULL, true);
        }
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_sp_link_deinit in my caOS application?
A: You should call it when a secure communication session needs to be terminated, such as during a graceful shutdown, a re-keying event, or when an error renders the current session invalid. It must be called before freeing the memory containing the link structure to ensure sensitive cryptographic keys are wiped from RAM.

Code example conf:0.90 **Q:** Provide a minimal C example showing how to deinitialize a secure pipe link.
A: ca_sp_link_t *my_link = get_active_link(); if (my_link != (void*)0) { ca_sp_link_deinit(my_linl
}

Parameters conf:1.00 **Q:** What are the requirements for the ca_sp_link_t pointer passed to ca_sp_link_deinit?
A: The parameter must be a valid, initialized pointer to a ca_sp_link_t structure. If a NULL pointer is passed, the function ignores it. Passing an invalid pointer or a pointer to a structure that was never initialized leads to undefined behavior.

Error/Edge case conf:1.00 **Q:** Can I call ca_sp_link_deinit from an interrupt service routine (ISR)?
A: No, you cannot call it from an ISR. The function performs blocking mutex operations, which makes it unsuitable for ISR context. Additionally, you must ensure no other tasks are using the link during deinitialization to avoid race conditions or memory corruption.

`ca_sp_link_hello`

Header: `caos/crypto/inc/secure_pipe_service.h:176`
Source: `caos/crypto/src/secure_pipe_service.c:319:350`
GUARD: `EN_SECURE_PIPE_SERVICE`

```
i32 ca_sp_link_hello(ca_sp_link_t *l, bool reply, u8 *out, u32 cap, u32
    *out_len)
```

Generates a secure handshake HELLO message to initiate or reply to a secure pipe connection.

Parametri:

Nome	Tipo	Descrizione
`l`	`ca_sp_link_t *`	Pointer to the initialized secure pipe link structure.
`reply`	`bool`	Set to true if responding to a peer's HELLO, false if initiating a new handshake.
`out`	`u8 *`	Buffer to store the generated HELLO packet. Must be at least CA_SP_LINK_HELLO_SIZE bytes.
`cap`	`u32`	Capacity of the 'out' buffer in bytes. Must be >= CA_SP_LINK_HELLO_SIZE.
`out_len`	`u32 *`	Pointer to store the actual number of bytes written to 'out'.

Ritorna:
CA_SECURE_PIPE_OK on success, CA_SECURE_PIPE_ERR if parameters are invalid or internal crypto fails, CA_SECURE_PIPE_SIZE if the buffer capacity is insufficient.

Note:

- This function is blocking because it uses a mutex to protect the link state during the cryptographic operations. It is NOT safe to call from an ISR (Interrupt Service Routine) because mutexes cannot be locked in interrupt context. The link must be fully initialized via ca_sp_link_init before calling this. A common mistake is providing a buffer smaller than CA_SP_LINK_HELLO_SIZE, which will immediately return an error; always ensure your buffer is sized correctly using the provided constant.

Uso:

- This function is the core of the secure handshake process. It is typically used in a sequence: first, the initiator calls it with 'reply' set to false, then the peer calls ca_sp_link_accept to process the packet, followed by the peer calling this function with 'reply' set to true to complete the handshake. It is used in conjunction with ca_net_buf_alloc to prepare the network packet buffer before sending the generated HELLO data over the wire.

Esempio:

```
ca_sp_link_t my_link;
ca_sp_link_init(&my_link); // Assume link is initialized
u8 hello_buf[CA_SP_LINK_HELLO_SIZE];
u32 hello_len = 0;

// Generate HELLO packet to start a handshake
i32 status = ca_sp_link_hello(&my_link, false, hello_buf, sizeof(hello_buf), &
    hello_len);

if (status == CA_SECURE_PIPE_OK) {
    // Use ca_net_buf_alloc to get a buffer and send hello_buf over the network
    // ... network transmission logic ...
} else {
    ca_printf("Handshake initiation failed with error: %d\n", status);
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_sp_link_hello in the secure pipe handshake process?
A: You call it to initiate or reply to a secure pipe connection. The initiator calls it with 'reply' set to false, and the peer calls it with 'reply' set to true after processing the initial packet via ca_sp_link_accept.

Code example conf:1.00 **Q:** How do I call ca_sp_link_hello in C using caOS types?

A: u8 buf[CA_SP_LINK_HELLO_SIZE]; u32 len; i32 res = ca_sp_link_hello(&my_link, false, buf, CA_SP_LINK_HELLO_SIZE, &len); if (res == CA_SECURE_PIPE_OK) { /* send buf */ }

Parameters conf:1.00 **Q:** What are the requirements for the 'out' buffer and the 'cap' parameter?
A: The 'out' buffer must be at least CA_SP_LINK_HELLO_SIZE bytes, and the 'cap' parameter must be greater than or equal to CA_SP_LINK_HELLO_SIZE. If 'cap' is smaller, the function returns CA_SECURE_PIPE_SIZE.

Return conf:1.00 **Q:** What do the different return values of ca_sp_link_hello signify?
A: CA_SECURE_PIPE_OK indicates success. CA_SECURE_PIPE_ERR indicates invalid parameters or an internal cryptographic failure. CA_SECURE_PIPE_SIZE indicates the provided buffer capacity is insufficient.

Error/Edge case conf:1.00 **Q:** What is a common mistake when calling ca_sp_link_hello?
A: A common mistake is providing a buffer smaller than CA_SP_LINK_HELLO_SIZE, which will cause the function to return an error. Always use the provided constant to size your buffers.

Behavior conf:1.00 **Q:** Is ca_sp_link_hello safe to call from an ISR?
A: No, it is not safe to call from an ISR. The function is blocking because it uses a mutex to protect the link state during cryptographic operations, and mutexes cannot be locked in an interrupt context.

`ca_sp_link_init`

Header: `caos/crypto/inc/secure_pipe_service.h:170`
Source: `caos/crypto/src/secure_pipe_service.c:274:317`
GUARD: `EN_SECURE_PIPE_SERVICE`

```
1 i32 ca_sp_link_init(ca_sp_link_t *l, u8 role, const char *password,
     const char *name)
```

Initializes a secure communication link context by deriving cryptographic keys from a password and name.

Parametri:

Nome	Tipo	Descrizione
`l`	`ca_sp_link_t *`	Pointer to the link structure to initialize. Must be allocated and not already initialized.
`role`	`u8`	The role of the device in the link (1 or 2). Determines key derivation direction.
`password`	`const char *`	Secret string used for key derivation. Must be within CA_SP_LINK_SECRET_MIN and CA_SP_LINK_SECRET_MAX length.
`name`	`const char *`	Identifier string used as salt for key derivation. Must be within valid length limits.

Ritorna:
CA_SECURE_PIPE_OK (0) on success, or CA_SECURE_PIPE_ERR (-1) if parameters are invalid, memory allocation fails, or key derivation fails.

Note:

- This function is blocking because it performs heavy cryptographic operations (PBKDF2/HKDF) which consume significant CPU cycles. It is not thread-safe for the same link structure, so ensure no other task accesses 'l' during initialization. It must NOT be called from an ISR because of the blocking nature and memory allocation requirements. A common mistake

is passing a password or name that is too short or too long, which will cause the function to return an error immediately. Always ensure the 'l' structure is zeroed or previously deinitialized before calling this.

Uso:

- This function is the mandatory first step in establishing a secure pipe. It is typically used during system startup or when a new connection is requested. It is paired with ca_sp_link_deinit to clean up resources. In a typical sequence, you allocate the link structure, call ca_sp_link_init, and then proceed to use other functions like ca_sp_link_hello or ca_sp_link_open to establish the actual communication.

Esempio:

```
ca_sp_link_t my_link;
// Ensure the structure is clean before initialization
ca_memzero(&my_link, sizeof(my_link));

// Initialize the link with role 1
i32 status = ca_sp_link_init(&my_link, 1U, "MySecretPassword", "Device_A");

if (status == CA_SECURE_PIPE_OK) {
    ca_printf("Link initialized successfully!\n");
} else {
    ca_printf("Failed to initialize link, check password length or memory.\n");
    // Perform cleanup if necessary
    /* ca_sp_link_deinit(&my_link); */
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_sp_link_init in my caOS application?
A: You should call it as the mandatory first step when establishing a secure pipe, typically during system startup or when a new connection is requested. It must be paired with ca_sp_link_deinit to clean up resources.

Code example conf:0.90 **Q:** Provide a minimal C example for initializing a secure link in caOS.
A: ca_sp_link_t my_link; i32 status = ca_sp_link_init(&my_link, 1, "my_secret_pass", "device_alpha"); if (status == CA_SECURE_PIPE_OK) { /* Proceed to ca_sp_link_hello */ }

Parameters conf:1.00 **Q:** What do the 'role', 'password', and 'name' parameters represent?
A: The 'role' (1 or 2) determines the key derivation direction. The 'password' is a secret string used for key derivation, and the 'name' acts as a salt. Both must fall within specific length limits (CA_SP_LINK_SECRET_MIN to CA_SP_LINK_SECRET_MAX).

Return conf:1.00 **Q:** What do the return values CA_SECURE_PIPE_OK and CA_SECURE_PIPE_ERR signify?
A: CA_SECURE_PIPE_OK (0) indicates success. CA_SECURE_PIPE_ERR (-1) indicates failure, which can be caused by invalid parameters, memory allocation failure, or a failure during the cryptographic key derivation process.

Error/Edge case conf:1.00 **Q:** What are common mistakes that cause ca_sp_link_init to fail?
A: Common mistakes include passing a password or name that is outside the allowed length limits, or failing to ensure the 'l' structure is zeroed or previously deinitialized before the call.

Behavior conf:1.00 **Q:** Is ca_sp_link_init safe to call from an ISR or in a multi-threaded environment?
A: No, it is not ISR-safe due to its blocking nature and memory allocation requirements. It is also not thread-safe for the same link structure, so you must ensure no other task accesses the structure during initialization.

ca_sp_link_is_up

Header: caos/crypto/inc/secure_pipe_service.h:193
Source: caos/crypto/src/secure_pipe_service.c:502:505
GUARD: EN_SECURE_PIPE_SERVICE

```
bool ca_sp_link_is_up(const ca_sp_link_t *l)
```

Checks if a secure pipe link is fully initialized and currently active.

Parametri:

Nome	Tipo	Descrizione
l	const ca_sp_link_t *	Pointer to the secure pipe link structure. Must be a valid, initialized pointer.

Ritorna:
Returns true if the link is both ready and up; returns false if the pointer is NULL, the link is not ready, or the link is down.

Note:

- This function is non-blocking and thread-safe as it performs a simple read of internal status flags. It is safe to call from an ISR because it does not attempt to acquire mutexes or perform I/O operations. The most common mistake is assuming that a 'true' return value guarantees the link will remain up indefinitely; in a multi-threaded environment, the link status could change immediately after the function returns. Always ensure the link structure has been properly initialized via ca_sp_link_init before calling this function to avoid undefined behavior.

Uso:

- This function is typically used as a guard clause before performing secure communication operations like sealing or unsealing data. It is often paired with ca_sp_link_init during the setup phase and ca_sp_link_accept during the handshake process. Developers use it to verify the health of the connection before attempting to send data, preventing errors that would occur if the pipe were disconnected or uninitialized. It is also used in monitoring loops to decide if a re-keying process is necessary via ca_sp_link_need_rekey.

Esempio:

```
ca_sp_link_t my_link;

// Initialize the link structure before use
ca_sp_link_init(&my_link);

// ... perform handshake operations ...

// Check if the link is ready for secure data transmission
if (ca_sp_link_is_up(&my_link)) {
    // Link is active, safe to proceed with secure operations
    // ca_sp_link_seal(&my_link, ...);
} else {
    // Link is down or not ready, handle the error state
    ca_printf_n("Secure pipe link is currently unavailable.");
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_sp_link_is_up in my application logic?
A: You should call it as a guard clause before performing secure communication operations like sealing or unsealing data. It is also used in monitoring loops to determine if a re-keying process is required via ca_sp_link_need_rekey.

Code example conf:0.90 **Q:** Provide a minimal C example showing how to check the link status.
A: ca_sp_link_t *my_link; ca_sp_link_init(my_link); if (ca_sp_link_is_up(my_link)) { /* Proceed with secure communication */ }

Parameters conf:1.00 **Q:** What are the requirements for the 'l' parameter in ca_sp_link_is_up?
A: The parameter 'l' must be a pointer to a valid, initialized ca_sp_link_t structure. Passing a NULL pointer will cause the function to return false.

Return conf:1.00 **Q:** What does a return value of true or false indicate, and can I rely on it for long-term state?
A: A return of true indicates the link is ready and active. A return of false indicates the pointer is NULL, the link is not ready, or the link is down. You cannot assume the link will remain up indefinitely, as the status may change immediately after the function returns.

`ca_sp_link_need_rekey`

Header: `caos/crypto/inc/secure_pipe_service.h:196`
Source: `caos/crypto/src/secure_pipe_service.c:507:512`
GUARD: `EN_SECURE_PIPE_SERVICE`

```
1 bool ca_sp_link_need_rekey(const ca_sp_link_t *l)
```

Checks if a secure pipe link requires a re-keying operation based on message count or session duration.

Parametri:

Nome	Tipo	Descrizione
`l`	`const ca_sp_link_t *`	Pointer to the secure pipe link structure. Must be a valid, initialized pointer; passing NULL will result in a safe return of false.

Ritorna:
Returns true if the link is active and either the message threshold (CA_SP_LINK_REKEY_MSGS) or the time threshold (CA_SP_LINK_REKEY_MS) has been exceeded. Returns false if the link is not up, not ready, or if the thresholds have not been met.

Note:

- This function is non-blocking and thread-safe as it performs a read-only check on the link structure. It is safe to call from an ISR, provided the pointer passed is valid and the link structure is not being modified concurrently by another task. The function relies on the system tick (ca_os_tick_ms), so it assumes the OS clock is correctly initialized. A common mistake is to assume this function performs the re-keying itself; it does not. It is purely a status check used to trigger a re-keying logic flow elsewhere in your application.

Uso:

- This function is typically used in the main loop of a communication task to monitor the health and security status of a secure pipe. It is usually paired with ca_sp_link_is_up to verify the connection state before checking for re-keying requirements. When this function returns true, the application should initiate a new handshake or re-keying sequence (e.g., calling a function to send a new HELLO packet) to maintain secure communication. It is part of the lifecycle management of a secure pipe, following the sequence: ca_sp_link_init -> [data exchange] -> ca_sp_link_need_rekey -> [re-keying logic] -> ca_sp_link_deinit.

Esempio:

```c
/* Check if the secure link needs a re-key to maintain security */
if (ca_sp_link_need_rekey(my_link_ptr)) {
    /* The link is up but requires a new session key due to age or message count */
    /* Perform re-keying handshake logic here */
    /* Example: send_hello(false); // Custom function to initiate handshake */
} else if (!ca_sp_link_is_up(my_link_ptr)) {
    /* Link is down, attempt to reconnect or log error */
    ca_printf("Secure link is down, waiting for reconnection...\n");
}

/* Ensure we do not block the system while waiting for link status */
ca_os_delay_ms(100);
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_sp_link_need_rekey in my application?
A: You should call it in the main loop of your communication task to monitor the security status of a secure pipe. It is typically used after verifying the connection with ca_sp_link_is_up to determine if a new handshake or re-keying sequence is required to maintain secure communication.

Code example conf:1.00 **Q:** Provide a minimal C example showing how to check if a secure pipe needs re-keying.
A: ca_sp_link_t *my_link; if (ca_sp_link_is_up(my_link)) { if (ca_sp_link_need_rekey(my_link)) { /* Trigger re-keying logic here */ } }

Parameters conf:1.00 **Q:** What are the requirements for the 'l' parameter in ca_sp_link_need_rekey?
A: The parameter 'l' must be a pointer to a valid, initialized ca_sp_link_t structure. If a NULL pointer is passed, the function safely returns false.

Return conf:1.00 **Q:** What do the return values of ca_sp_link_need_rekey signify?
A: Returns true if the link is active and either the message count threshold (CA_SP_LINK_REKEY_M or the time threshold (CA_SP_LINK_REKEY_MS) has been exceeded. Returns false if the link is not up, not ready, or if the thresholds have not been met.

`ca_sp_link_open`

Header: `caos/crypto/inc/secure_pipe_service.h:190`
Source: `caos/crypto/src/secure_pipe_service.c:442:500`
GUARD: `EN_SECURE_PIPE_SERVICE`

```c
i32 ca_sp_link_open(ca_sp_link_t *l, const u8 *aad, u32 aad_len, const u8 *in, u32 len, u8 *out, u32 cap, u32 *out_len)
```

Decrypts and verifies an incoming secure pipe packet using AES-GCM.

Parametri:

Nome	Tipo	Descrizione
l	ca_sp_link_t *	Pointer to the initialized secure pipe link structure.
aad	const u8 *	Additional Authenticated Data buffer; must be non-NULL if aad_len > 0.
aad_len	u32	Length of AAD; must not exceed CA_SP_LINK_AAD_MAX.
in	const u8 *	Pointer to the encrypted input buffer (wire format).
len	u32	Total length of the input buffer including header and tag.
out	u8 *	Buffer to store the decrypted plaintext.
cap	u32	Capacity of the output buffer; must be at least the size of the decrypted data.
out_len	u32 *	Pointer to store the actual length of the decrypted data.

Ritorna:
CA_SECURE_PIPE_OK on success. Returns CA_SECURE_PIPE_ERR for invalid parameters or malformed packets, CA_SECURE_PIPE_NOT_UP if the link is not established, CA_SECURE_PIPE_REPLAY if the packet counter is invalid, CA_SECURE_PIPE_DECRYPT if authentication fails, or CA_SECURE_PIPE_SIZE if the output buffer is too small.

Note:

- This function is thread-safe as it uses an internal mutex to protect the link state and replay window. It is a blocking function because it performs cryptographic operations and mutex acquisition; therefore, it must NEVER be called from an ISR. The link must be fully initialized via ca_sp_link_init and established before calling this. A common mistake is providing an output buffer smaller than the actual payload size, which triggers a CA_SECURE_PIPE_SIZE error, or failing to check the return value, which could lead to processing uninitialized or zeroed memory.

Uso:

- Used in secure communication protocols to process incoming encrypted frames. It is typically paired with ca_sp_link_seal on the sender side. The standard sequence is: initialize the link, wait for the connection to be established, and then call this function in a loop to process incoming data packets. It is often used in conjunction with network receive buffers to verify data integrity and authenticity before passing the payload to the application layer.

Esempio:

```
/* Assuming 'link' is an established ca_sp_link_t and 'rx_buf' contains
    encrypted data */
u8 plaintext[128];
u32 decrypted_len = 0;
i32 status = ca_sp_link_open(&link, NULL, 0, rx_buf, rx_len, plaintext, sizeof(
    plaintext), &decrypted_len);

if (status == CA_SECURE_PIPE_OK) {
    /* Process decrypted data safely */
    /* ... */
} else if (status == CA_SECURE_PIPE_DECRYPT) {
    /* Handle authentication failure - potential security breach */
} else {
    /* Handle other errors */
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_sp_link_open in my caOS application?
A: You should call it when processing incoming encrypted frames in a secure communication protocol. It is typically used in a loop after the link has been initialized via ca_sp_link_init and successfully established to verify data integrity and authenticity before passing the payload to the application layer.

Code example conf:0.90 **Q:** Provide a minimal C example of how to call ca_sp_link_open.
A: ca_sp_link_t link; u8 out[128]; u32 out_len; u8 aad[4] = {0}; i32 res = ca_sp_link_open(&link, aad, 4, rx_buf, rx_len, out, 128, &out_len); if (res == CA_SECURE_PIPE_OK) { /* process out */ }

Parameters conf:1.00 **Q:** What are the constraints on the aad and cap parameters?
A: The aad parameter must be non-NULL if aad_len > 0, and aad_len must not exceed CA_SP_LINK_AAD_MAX. The cap parameter represents the capacity of the output buffer and must be at least the size of the decrypted data; otherwise, the function returns CA_SECURE_PIPE_SIZE.

Return conf:1.00 **Q:** What does a return value of CA_SECURE_PIPE_DECRYPT indicate?
A: A return value of CA_SECURE_PIPE_DECRYPT indicates that the authentication of the packet failed, meaning the data was either tampered with or the decryption key/state is mismatched.

Error/Edge case conf:1.00 **Q:** What is a common mistake when using ca_sp_link_open that leads to processing invalid data?
A: A common mistake is failing to check the return value of the function. If the return value is not checked, the application might process uninitialized or zeroed memory in the output buffer if the decryption or authentication failed.

Behavior conf:1.00 **Q:** Is ca_sp_link_open safe to call from an interrupt service routine (ISR)?
A: No, it must NEVER be called from an ISR. The function is blocking because it performs cryptographic operations and acquires an internal mutex to protect the link state and replay window.

`ca_sp_link_seal`

Header: `caos/crypto/inc/secure_pipe_service.h:185`
Source: `caos/crypto/src/secure_pipe_service.c:405:440`
GUARD: `EN_SECURE_PIPE_SERVICE`

```
i32 ca_sp_link_seal(ca_sp_link_t *l, const u8 *aad, u32 aad_len, const
    u8 *plain, u32 len, u8 *out, u32 cap, u32 *out_len)
```

Encrypts and authenticates a plaintext buffer into a secure wire-format packet for transmission.

Parametri:

Nome	Tipo	Descrizione
l	ca_sp_link_t *	Pointer to the initialized secure pipe link handle.
aad	const u8 *	Additional Authenticated Data buffer; can be NULL if aad_len is 0.
aad_len	u32	Length of AAD; must not exceed CA_SP_LINK_AAD_MAX.
plain	const u8 *	Pointer to the plaintext data to be encrypted.
len	u32	Length of plaintext; must be > 0 and <= 0xFFFF.
out	u8 *	Output buffer where the sealed packet is written.
cap	u32	Capacity of the output buffer; must be large enough to hold header, padded data, and tag.
out_len	u32 *	Pointer to store the final size of the generated packet.

Ritorna:
CA_SECURE_PIPE_OK on success. Returns CA_SECURE_PIPE_ERR for invalid parameters or crypto failure, and CA_SECURE_PIPE_NOT_UP if the link is not established. Returns CA_SECURE_PIPE_SIZE if the output buffer capacity is insufficient.

Note:

- This function is thread-safe as it uses an internal mutex to protect the link state and transmission counter. It is a blocking function because it performs cryptographic operations and mutex locking, so it must NOT be called from an ISR. The link must be fully initialized and in an 'up' state before calling this. A common mistake is providing an output buffer that is too small; always ensure the buffer size accounts for the header, padding (to 16-byte alignment), and the AES-GCM tag size.

Uso:

- Used in secure communication protocols to prepare data for transmission over an untrusted medium. It is typically used in a sequence: first, the link is initialized with ca_sp_link_init, then the connection is established, and finally, this function is called to seal data before sending it via a transport layer (like UART or Ethernet). It is the counterpart to ca_sp_link_open, which is used by the receiver to decrypt and verify the packet.

Esempio:

```
ca_sp_link_t my_link;
u8 wire_buf[256];
u32 wire_len = 0;
const u8 my_data[] = "Secret Message";

/* Assuming my_link is already initialized and connected */
if (ca_sp_link_seal(&my_link, NULL, 0, my_data, sizeof(my_data), wire_buf,
    sizeof(wire_buf), &wire_len) == CA_SECURE_PIPE_OK) {
    /* Send wire_buf of size wire_len over the transport interface */
    /* e.g., ca_uart_dma_send(..., wire_buf, wire_len); */
} else {
    /* Handle error: log failure or reset link */
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_sp_link_seal in my caOS application?
A: You should call it after the secure pipe link has been initialized with ca_sp_link_init and the connection is fully established. It is used to encrypt and authenticate plaintext

data into a secure wire-format packet before sending it over an untrusted transport medium.

Code example conf:0.90 **Q:** How do I implement a basic call to ca_sp_link_seal?
A: u8 out_buf[128]; u32 out_len; i32 res = ca_sp_link_seal(&my_link, NULL, 0, plain_data, 16, out_buf, 128, &out_len); if (res == CA_SECURE_PIPE_OK) { /* send out_buf */ }

Parameters conf:1.00 **Q:** What are the constraints on the aad_len and len parameters?
A: The aad_len must not exceed CA_SP_LINK_AAD_MAX. The len (plaintext length) must be greater than 0 and less than or equal to 0xFFFF.

Return conf:1.00 **Q:** What do the different return values from ca_sp_link_seal indicate?
A: CA_SECURE_PIPE_OK indicates success. CA_SECURE_PIPE_ERR indicates invalid parameters or a cryptographic failure. CA_SECURE_PIPE_NOT_UP means the link is not established. CA_SECURE_PIPE_SIZE indicates the provided output buffer capacity is insufficient.

Error/Edge case conf:1.00 **Q:** What is a common mistake when providing the output buffer for ca_sp_link_seal?
A: A common mistake is providing an output buffer that is too small. You must ensure the capacity accounts for the header, the padded data (which is aligned to 16 bytes), and the AES-GCM tag size.

Behavior conf:1.00 **Q:** Is ca_sp_link_seal safe to call from an Interrupt Service Routine (ISR)?
A: No, it must not be called from an ISR. The function is blocking because it performs cryptographic operations and uses an internal mutex to protect the link state and transmission counter.

`ca_spdifrx_channel_status`

Header: `caos/drivers/inc/spdifrx.h:75`
Source: `caos/drivers/src/spdifrx.c:131:171`
GUARD: `EN_SPDIFRX`

```
u32 ca_spdifrx_channel_status(u8 *cs, u32 timeout_ms); /* one whole
   block: CA_SPDIFRX_CS_BYTES, else 0 */
```

Captures a complete S/PDIF Channel Status (CS) block from the incoming stream.

Parametri:

Nome	Tipo	Descrizione
`cs`	`u8 *`	Pointer to a buffer of at least CA_SPDIFRX_CS_BYTES size to store the received status data.
`timeout_ms`	`u32`	Maximum time in milliseconds to wait for a complete, valid block to be received.

Ritorna:
Returns CA_SPDIFRX_CS_BYTES (the number of bytes) if a full block was successfully captured, or 0 if the operation timed out, the input pointer was NULL, or the hardware failed to synchronize.

Note:

- This function is blocking; it will pause the calling task until a full S/PDIF block is received or the timeout expires. It is not safe to call from an ISR because it relies on time-based waiting and hardware state management. The function requires the S/PDIF hardware to be initialized and running. A common mistake is providing a buffer smaller than

CA_SPDIFRX_CS_BYTES, which will cause memory corruption, or setting a timeout too short to capture the full block, which occurs at a specific rate depending on the audio sample frequency.

Uso:

- This function is typically used during the initialization or diagnostic phase of an audio application to verify the source configuration (e.g., sample rate, copyright bits, or audio format). It is usually paired with ca_spdifrx_init to set up the hardware, followed by this function to read the metadata, and finally ca_spdifrx_read_dma to begin streaming the actual audio data. It is essential to clear the buffer before calling this function to ensure that old data does not persist if the capture fails.

Esempio:

```
u8 cs_buffer[CA_SPDIFRX_CS_BYTES];
ca_memzero(cs_buffer, CA_SPDIFRX_CS_BYTES);

/* Attempt to read the channel status block within 500ms */
u32 bytes_read = ca_spdifrx_channel_status(cs_buffer, 500U);

if (bytes_read == CA_SPDIFRX_CS_BYTES) {
    /* Successfully captured the full block, process metadata */
    ca_printf("CS block captured successfully.\n");
} else {
    /* Handle timeout or hardware error */
    ca_printf("Failed to capture CS block: check S/PDIF source.\n");
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_spdifrx_channel_status function in my audio application?
A: This function is typically used during the initialization or diagnostic phase to verify source configuration, such as sample rate, copyright bits, or audio format. It is usually called after ca_spdifrx_init and before starting audio streaming with ca_spdifrx_read_dma.

Code example conf:1.00 **Q:** Provide a minimal code snippet to capture the S/PDIF channel status.
A: u8 buffer[CA_SPDIFRX_CS_BYTES]; u32 result; result = ca_spdifrx_channel_status(buffer, 500); if (result == CA_SPDIFRX_CS_BYTES) { /* Process status data */ }

Parameters conf:1.00 **Q:** What are the requirements for the 'cs' buffer parameter?
A: The 'cs' parameter must be a pointer to a buffer of at least CA_SPDIFRX_CS_BYTES size. Providing a smaller buffer will cause memory corruption.

Return conf:1.00 **Q:** What does a return value of 0 indicate from ca_spdifrx_channel_status?
A: A return value of 0 indicates that the operation failed. This can be caused by a timeout, the input pointer being NULL, or the hardware failing to synchronize.

Error/Edge case conf:1.00 **Q:** What are common mistakes to avoid when using this function?
A: Common mistakes include providing a buffer smaller than CA_SPDIFRX_CS_BYTES, which leads to memory corruption, or setting a timeout value too short to capture the full block, which depends on the audio sample frequency.

Behavior conf:1.00 **Q:** Is it safe to call ca_spdifrx_channel_status from an interrupt service routine (ISR)?
A: No, it is not safe to call this function from an ISR because it is a blocking function that relies on time-based waiting and hardware state management.

---

ca_spdifrx_errors

Header: caos/drivers/inc/spdifrx.h:76
Source: caos/drivers/src/spdifrx.c:101:104
GUARD: EN_SPDIFRX

```
u32 ca_spdifrx_errors(void); /* CA_SPDIFRX_ERR_* */
```

Retrieves the current error status flags from the SPDIF receiver peripheral.

Ritorna:
A 32-bit unsigned integer representing a bitmask of error flags (CA_SPDIFRX_ERR_*). A return value of 0 indicates no errors are currently detected.

Note:

- This function is non-blocking and thread-safe, as it simply reads a cached status variable updated by the SPDIF hardware interrupt service routine (ISR). It can be safely called from both task context and ISRs. The most common mistake is assuming that calling this function clears the errors; it does not. The error flags are typically latched by the hardware or the driver's ISR, and you must ensure your application logic handles these flags immediately after reading them to prevent missing transient error states.

Uso:

- This function is used to monitor the health of the SPDIF digital audio input stream. It is typically called after synchronization functions like ca_spdifrx_sync or data retrieval functions like ca_spdifrx_channel_status to verify that the operation completed without hardware errors (such as parity errors, buffer overflows, or loss of lock). In a typical audio processing loop, you would pair this with your data acquisition logic to decide whether to process the received audio buffer or discard it due to corruption.

Esempio:

```
u32 current_errors;

// Attempt to synchronize with the incoming SPDIF stream with a 100ms timeout
if (ca_spdifrx_sync(100U) != 0U) {
    // Check if any errors occurred during the sync or previous operations
    current_errors = ca_spdifrx_errors();

    if (current_errors != 0U) {
        // Log or handle the specific error bits found in current_errors
        // For example: if (current_errors & CA_SPDIFRX_ERR_PARITY) { ... }
    } else {
        // Proceed with audio data processing
    }
} else {
    // Handle synchronization failure
}
```

Q&A — fine-tuning review

3/3 pairs · gemini-3.1-flash-lite

Usage conf:1.00 **Q:** When should I call ca_spdifrx_errors and why is it important for my audio processing loop?
A: You should call ca_spdifrx_errors after synchronization functions like ca_spdifrx_sync or data retrieval functions like ca_spdifrx_channel_status. It is used to monitor the health of the SPDIF input stream, allowing your application logic to decide whether to process the received audio buffer or discard it due to corruption caused by hardware errors.

Code example conf:1.00 **Q:** Can you provide a minimal C example showing how to check for SPDIF errors using caOS types?
A: u32 err_flags = ca_spdifrx_errors(); if (err_flags != 0) { /* Handle errors such as parity or buffer overflows */ }

Return conf:1.00 **Q:** What does the return value of ca_spdifrx_errors represent, and does calling it clear the error state?
A: The return value is a 32-bit unsigned integer (u32) representing a bitmask of error flags (CA_SPDIFRX_ERR_*). A return value of 0 indicates no errors. Calling this function does not clear the errors; the flags are latched by the hardware or the driver's ISR, so you must handle them immediately to avoid missing transient states.

`ca_spdifrx_fs`

Header: `caos/drivers/inc/spdifrx.h:73`
Source: `caos/drivers/src/spdifrx.c:96:99`
GUARD: `EN_SPDIFRX`

```
u32 ca_spdifrx_fs(void); /* fs measured by the last sync or read, 0 =
    none */
```

Returns the sampling frequency (fs) of the SPDIF receiver measured during the last synchronization or measurement cycle.

Ritorna:
Returns a 32-bit unsigned integer representing the frequency in Hz. A return value of 0 indicates that no valid signal is currently detected or no measurement has been performed yet.

Note:

- This function is non-blocking and thread-safe as it simply returns a cached value stored in a static variable. It is safe to call from an ISR, although the value returned may be stale if the hardware state has changed since the last measurement. It does not trigger a new hardware measurement; it only reports the result of the most recent one. A common mistake is assuming this function will actively wait for a signal or perform a fresh measurement; if you need an up-to-date value, you must ensure a measurement function (like the one that updates the internal state) has been called previously.

Uso:

- This function is typically used in diagnostic CLI commands or monitoring tasks to report the current status of the audio input. It is often paired with functions that perform the actual hardware synchronization and measurement, such as those that wait for a sync lock and then trigger a frequency calculation. In a typical sequence, the system initializes the SPDIF hardware, waits for a valid signal, performs a measurement, and then this function is used to retrieve that result for display or logic decisions.

Esempio:

```
/* Example: Monitoring SPDIF status via CLI */
#include "caos.h"

void monitor_spdif_status(void) {
    u32 current_fs = ca_spdifrx_fs();

    if (current_fs > 0) {
        ca_printf("SPDIF signal detected at %u Hz\n", current_fs);
    } else {
        ca_printf("No valid SPDIF signal detected.\n");
```

```
    }

    /* Check for hardware errors if signal is present */
    if (current_fs > 0) {
        u32 errs = ca_spdifrx_errors();
        if (errs != 0) {
            ca_printf("Warning: SPDIF errors detected: 0x%02X\n", errs);
        }
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_spdifrx_fs() and what is its primary purpose?
A: You should call ca_spdifrx_fs() when you need to retrieve the cached sampling frequency of the SPDIF receiver for diagnostic purposes or monitoring tasks. It is intended to be used after the system has initialized the hardware and performed a measurement, as it does not trigger a new hardware measurement itself.

Code example conf:1.00 **Q:** How do I implement a simple check for the SPDIF sampling frequency in caOS?
A: You can retrieve the frequency into a u32 variable and check if it is non-zero to verify a valid signal:
u32 current_fs = ca_spdifrx_fs(); if (current_fs > 0) { /* Signal detected */ } else { /* No signal or measurement not performed */ }

Return conf:1.00 **Q:** What does a return value of 0 from ca_spdifrx_fs() signify?
A: A return value of 0 indicates that either no valid SPDIF signal is currently detected by the hardware or that no frequency measurement has been performed yet. It does not necessarily mean the hardware is broken, but rather that the cached value is invalid or empty.

`ca_spdifrx_hw_dma_bind`

Header: `caos/drivers/inc/ca_glue_hw.h:413`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_spdifrx_hw_dma_bind(i16 *flat, u16 *req); /* data DMA
    channel (flat, -1 = any) + request */
```

Retrieves the hardware-specific DMA stream and request ID required for SPDIFRX data transfers.

Parametri:

Nome	Tipo	Descrizione
`flat`	`i16 *`	Pointer to store the DMA stream/channel index. Set to -1 if the hardware supports dynamic routing (e.g., via DMAMUX).
`req`	`u16 *`	Pointer to store the hardware-specific DMA request ID required by the peripheral to trigger DMA transfers.

Ritorna:
Returns true if the hardware configuration is valid and parameters were successfully populated. Returns false if DMA is not supported on the current platform or the configuration is unavailable.

Note:

- This function is non-blocking and thread-safe as it only performs a memory write based on compile-time constants or hardware-specific logic. It is safe to call from an ISR, though it is typically used during the driver initialization phase. The most common mistake is assuming that the returned 'flat' index is always a valid DMA stream index; on some platforms (like H7), it returns -1, which indicates that the caller must use a dynamic DMA channel allocation function rather than a hardcoded index. Always check the return value before attempting to use the 'flat' or 'req' values to configure the DMA controller.

Uso:

- This function is used exclusively during the initialization of the SPDIFRX driver to prepare the DMA subsystem. It is typically called before requesting a DMA channel from the system pool. The standard sequence is: call ca_spdifrx_hw_dma_bind to get the requirements, then pass those values to ca_dma_free_channel_index or ca_dma_free_channel to reserve the hardware resource, and finally call ca_dma_set_request to link the peripheral to the DMA controller.

Esempio:

```
i16 flat;
u16 req;

/* Retrieve hardware requirements for SPDIFRX DMA */
if (ca_spdifrx_hw_dma_bind(&flat, &req)) {
    /* Reserve the DMA channel based on the retrieved requirements */
    /* s_ch is a static handle for the SPDIFRX DMA channel */
    if (flat >= 0) {
        s_ch = ca_dma_free_channel_index(flat, &s_ch);
    } else {
        s_ch = ca_dma_free_channel(&s_ch);
    }

    if (s_ch != NULL) {
        /* Bind the peripheral request ID to the reserved DMA channel */
        ca_dma_set_request(s_ch, req);
    } else {
        /* Handle error: DMA channel allocation failed */
    }
} else {
    /* Handle error: DMA not supported or binding failed */
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_spdifrx_hw_dma_bind in my driver initialization sequence?
A: It should be called during the SPDIFRX driver initialization phase, before requesting a DMA channel from the system pool. The standard sequence is to call ca_spdifrx_hw_dma_bind to retrieve the requirements, pass those to ca_dma_free_channel_index or ca_dma_free_channel to reserve the resource, and finally call ca_dma_set_request to link the peripheral to the DMA controller.

Code example conf:1.00 **Q:** How do I correctly call ca_spdifrx_hw_dma_bind in C using caOS types?
A: i16 flat; u16 req; if (ca_spdifrx_hw_dma_bind(&flat, &req)) { /* Proceed to reserve DMA channel using flat and req */ }

Parameters conf:1.00 **Q:** What is the significance of the 'flat' parameter, and why might it be set to -1?
A: The 'flat' parameter is a pointer to store the DMA stream or channel index. It is set to -1 on platforms that support dynamic routing (such as the H7 series using DMAMUX),

indicating that the caller must use a dynamic DMA channel allocation function instead of relying on a hardcoded index.

Return conf:1.00 **Q:** What do the return values of ca_spdifrx_hw_dma_bind indicate?

A: Returning true indicates that the hardware configuration is valid and the parameters were successfully populated. Returning false indicates that DMA is not supported on the current platform or the configuration is unavailable. Always check this return value before using the populated 'flat' or 'req' values.

HW differences conf:1.00 **Q:** How does the behavior of ca_spdifrx_hw_dma_bind differ between the F4/F7 series and the H7 series?

A: On F4xx and F7xx series, the function returns a fixed DMA stream index (e.g., SPDIFRX_DMA_FLAT) and a specific request ID. On the H7xx series, it returns -1 for the 'flat' parameter, signaling that the hardware uses DMAMUX1 for dynamic routing rather than a fixed stream index.

`ca_spdifrx_hw_dr`

Header: `caos/drivers/inc/ca_glue_hw.h:414`
GUARD: `EN_CA_GLUE_HW`

```
extern volatile void *ca_spdifrx_hw_dr(void); /* SPDIFRX_FMT0_DR */
```

Returns the memory-mapped address of the SPDIFRX Data Register (DR) for direct hardware access.

Ritorna:
Returns a volatile pointer to the SPDIFRX->DR register if supported by the hardware, or NULL if the hardware does not support direct access or DMA operations are disabled.

Note:

- This function provides a raw pointer to the hardware register. It is non-blocking and thread-safe in the sense that it only returns a constant memory address. It is safe to call from an ISR, but you must never write to this pointer directly unless you are performing manual register manipulation, which is discouraged in favor of higher-level caOS drivers. The most common beginner mistake is assuming this function will work on all STM32 variants; always check if the returned pointer is NULL before dereferencing it, as many STM32 series do not implement the SPDIFRX peripheral.

Uso:

- This function is primarily used to provide the hardware source or destination address to the DMA controller. In a typical workflow, you initialize the SPDIFRX peripheral, clear any pending data using a loop with ca_spdifrx_hw_get, and then pass the result of ca_spdifrx_hw_dr to ca_dma_copy_target to set up a high-speed data transfer from the peripheral to memory. It is the bridge between the low-level hardware register and the caOS DMA abstraction layer.

Esempio:

```
/* Example: Setting up a DMA transfer from SPDIFRX to a memory buffer */
#include "caos.h"

void start_spdif_capture(u32 *buffer, u16 words) {
    /* Ensure the peripheral is in a known state */
    ca_spdifrx_hw_state(SPDIFRX_EN_IDLE, false);

    /* Clear existing data in the hardware FIFO */
```

```
    u32 junk;
    while (ca_spdifrx_hw_get(&junk)) { }

    /* Get the hardware register address */
    volatile void *dr_addr = ca_spdifrx_hw_dr();

    if (dr_addr != NULL) {
        /* Configure DMA to move data from the DR register to our buffer */
        if (!ca_dma_copy_target(s_ch, buffer, dr_addr, words, CA_DMA_DIR_P2M,
    CA_DMA_WIDTH_32)) {
            /* Handle DMA configuration error */
        }
    } else {
        /* Handle case where hardware does not support SPDIFRX or DMA */
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_spdifrx_hw_dr function in my caOS application?
A: You should use this function primarily to obtain the hardware address of the SPDIFRX Data Register for DMA controller configuration. It acts as a bridge between the low-level hardware and the caOS DMA abstraction layer, allowing you to set up high-speed data transfers from the peripheral to memory.

Code example conf:0.90 **Q:** How do I correctly use ca_spdifrx_hw_dr to set up a DMA transfer?
A: First, ensure the SPDIFRX peripheral is initialized and clear pending data using ca_spdifrx_hw_get. Then, retrieve the register address and pass it to the DMA target function: volatile void *dr_ptr = ca_spdifrx_hw_dr(); if (dr_ptr != NULL) { ca_dma_copy_target(dr_ptr); }

Return conf:1.00 **Q:** What does it mean if ca_spdifrx_hw_dr returns NULL?
A: A NULL return indicates that the hardware does not support direct access to the SPDIFRX Data Register, or that DMA operations are disabled for your specific configuration. You must always check for NULL before dereferencing the pointer to avoid runtime errors.

HW differences conf:1.00 **Q:** Does ca_spdifrx_hw_dr behave the same way across all STM32 families in caOS?
A: No. The function's availability depends on the MCU family and whether DMA is enabled. For example, while F4xx, F7xx, and H7xx implementations return the address of the SPDIFRX->DR register, other families like F1xx, H5xx, and U5xx return NULL, either because the peripheral is not implemented or because DMA support is disabled in the driver configuration.

`ca_spdifrx_hw_flags`

Header: `caos/drivers/inc/ca_glue_hw.h:408`
GUARD: `EN_CA_GLUE_HW`

```
extern u32 ca_spdifrx_hw_flags(void); /* CA_SPDIFRX_ERR_* |
   CA_SPDIFRX_HW_*, the clearable ones cleared */
```

Reads and clears the current SPDIFRX hardware status flags and error conditions.

Ritorna:
A bitmask (u32) representing current hardware states (e.g., CA_SPDIFRX_HW_SYNCD) and error conditions (e.g., CA_SPDIFRX_ERR_PAR). Returns 0 if no flags are set.

Note:

- This function is non-blocking and executes quickly by reading the hardware status register and immediately writing to the interrupt flag clear register. It is thread-safe in the sense that it performs an atomic read-modify-write on the hardware registers, but it should be used carefully in multi-threaded environments to avoid missing transient error flags. It is safe to call from an ISR, as it does not involve any OS primitives or blocking operations. The most common beginner mistake is ignoring the returned flags; you must check these bits to detect stream synchronization issues or data corruption (like parity or overrun errors) that require a peripheral reset.

Uso:

- This function is typically used in a polling loop or an error-handling routine to monitor the health of the SPDIF stream. It is often called after initializing the peripheral with ca_spdifrx_hw_state to verify that the hardware has successfully locked onto the incoming signal. In a typical sequence, you would call it periodically to check for errors, and if a critical error (like a timeout or sync loss) is detected, you would use ca_spdifrx_hw_state to reset the peripheral and attempt to re-synchronize.

Esempio:

```
/* Check SPDIFRX status and handle potential errors */
u32 status = ca_spdifrx_hw_flags();

if (status & CA_SPDIFRX_ERR_OVR) {
    /* Handle buffer overrun: log error and reset peripheral */
    ca_spdifrx_hw_state(0, false); /* Disable peripheral */
    ca_spdifrx_hw_state(1, true);  /* Re-enable peripheral */
} else if (status & CA_SPDIFRX_HW_SYNCD) {
    /* Stream is synchronized, proceed with data processing */
} else {
    /* No sync yet, wait for next cycle */
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_spdifrx_hw_flags in my application?
A: You should call it periodically within a polling loop or an error-handling routine to monitor the health of the SPDIF stream. It is also recommended to call it after initializing the peripheral with ca_spdifrx_hw_state to verify that the hardware has successfully locked onto the incoming signal.

Code example conf:1.00 **Q:** How do I check for SPDIFRX errors in my code?
A: You should capture the return value of ca_spdifrx_hw_flags and check the bits against the defined error constants. Example: u32 flags = ca_spdifrx_hw_flags(); if (flags & CA_SPDIFRX_ERR_PAR) { /* Handle parity error */ }

Return conf:1.00 **Q:** What do the return values of ca_spdifrx_hw_flags represent?
A: The function returns a u32 bitmask. It includes hardware status flags like CA_SPDIFRX_HW_SYN (synchronization detected) and CA_SPDIFRX_HW_SBD (sub-block detected), as well as error conditions such as CA_SPDIFRX_ERR_PAR (parity), CA_SPDIFRX_ERR_OVR (overrun), CA_SPDIFRX_ERR_FRM (frame), CA_SPDIFRX_ERR_SYNC (sync), and CA_SPDIFRX_ERR_TMO (timeout).

Error/Edge case conf:1.00 **Q:** What is a common mistake when using ca_spdifrx_hw_flags?
A: The most common beginner mistake is ignoring the returned flags. You must check these bits to detect stream synchronization issues or data corruption, such as parity or overrun errors, which may require a peripheral reset using ca_spdifrx_hw_state.

HW differences conf:1.00 **Q:** Are there differences in how ca_spdifrx_hw_flags behaves across different STM32 families?
A: Yes. On F4xx, F7xx, and H7xx, the function reads the SPDIFRX->SR register and clears specific flags in the SPDIFRX->IFCR register. Notably, FERR, SERR, and TERR

are only cleared by setting SPDIFRXEN to 0. On other families like F1xx, U5xx, and H5xx, the function is implemented as a stub that returns 0, indicating the peripheral or driver support is not present or active.

---

ca_spdifrx_hw_get

Header: caos/drivers/inc/ca_glue_hw.h:409
GUARD: EN_CA_GLUE_HW

```
extern bool ca_spdifrx_hw_get(u32 *w); /* one data word (FMT0), false
    when RXNE = 0 */
```

Reads a single data word from the SPDIFRX hardware buffer if available.

Parametri:

Nome	Tipo	Descrizione
w	u32 *	Pointer to a 32-bit variable where the received data word will be stored. Must be a valid memory address.

Ritorna:
Returns true if a word was successfully read from the hardware register (RXNE flag was set). Returns false if the hardware buffer is empty (RXNE flag is 0), in which case the value at *w remains unchanged.

Note:

- This function is non-blocking and performs a direct register read. It is thread-safe in the sense that it only accesses the hardware register, but it does not implement any internal locking; if multiple tasks call this simultaneously, you may experience data corruption or race conditions. It is safe to call from an ISR because it does not wait for hardware or use OS primitives. The most common beginner mistake is ignoring the return value; if you do not check the boolean result, you might process 'junk' data or stale values when the hardware has not actually received a new frame.

Uso:

- This function is typically used in a polling loop or within an interrupt service routine to drain the SPDIFRX hardware FIFO. It is often paired with ca_spdifrx_hw_state to manage the peripheral's lifecycle (init -> state(enable) -> loop(get) -> state(disable)). It is essential for clearing the hardware buffer during initialization or after a synchronization error to ensure that subsequent reads start from a clean state.

Esempio:

```
u32 data_word;

// Attempt to read a word from the SPDIFRX hardware
if (ca_spdifrx_hw_get(&data_word)) {
    // Successfully retrieved data, process it here
    ca_printf("Received: %08X\n", data_word);
} else {
    // No data available in the hardware FIFO
    // Perform other tasks or wait for the next interrupt
    ca_os_yield();
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_spdifrx_hw_get and what is its primary purpose?
A: You should call it in a polling loop or an ISR to drain the SPDIFRX hardware FIFO. It is used to read a single data word from the hardware buffer, typically as part of a lifecycle management sequence involving initialization and state enabling.

Code example conf:1.00 **Q:** Provide a minimal example of how to use ca_spdifrx_hw_get to read data.
A: u32 data = 0; if (ca_spdifrx_hw_get(&data)) { /* process data */ }

Parameters conf:1.00 **Q:** What are the requirements for the pointer parameter passed to ca_spdifrx_hw_get?
A: The parameter 'w' must be a valid memory address pointing to a 32-bit variable where the received data word will be stored.

Return conf:1.00 **Q:** What do the boolean return values of ca_spdifrx_hw_get indicate?
A: Returns true if a word was successfully read from the hardware register (RXNE flag was set). Returns false if the hardware buffer is empty (RXNE flag is 0), in which case the value at the provided pointer remains unchanged.

HW differences conf:1.00 **Q:** Are there differences in how ca_spdifrx_hw_get behaves across different STM32 families?
A: Yes. On F4xx, F7xx, and H7xx, the function performs a direct register read from SPDIFRX->DR if the SPDIFRX_SR_RXNE flag is set. On F1xx, H5xx, and U5xx, the function is currently implemented as a stub that always returns false and does not modify the pointer.

`ca_spdifrx_hw_get_cs`

Header: `caos/drivers/inc/ca_glue_hw.h:410`
GUARD: `EN_CA_GLUE_HW`

```
1 extern bool ca_spdifrx_hw_get_cs(u32 *csr); /* raw SPDIFRX_CSR ([24] SOB
    , [23:16] CS, [15:0] user), false when CSRNE = 0 */
```

Reads the raw Channel Status (CS) register from the SPDIFRX peripheral if data is available.

Parametri:

Nome	Tipo	Descrizione
`csr`	`u32 *`	Pointer to a 32-bit variable where the raw register value will be stored. Must be a valid memory address.

Ritorna:
Returns true if the register was read successfully (CSRNE flag was set). Returns false if no new Channel Status data is available (CSRNE = 0).

Note:

- This function is non-blocking and performs a direct hardware register read. It is thread-safe only if the hardware access is not interrupted by other tasks modifying the same peripheral registers. It is safe to call from an ISR, provided the hardware is already initialized. The most common beginner mistake is ignoring the return value; if you call this when the peripheral has no new data, the pointer 'csr' will not be updated, potentially leaving you with stale or uninitialized data.

Uso:

- This function is typically used during the initialization or synchronization phase of the SPDIFRX driver to clear out old, irrelevant data from the hardware buffers. It is often paired with 'ca_spdifrx_hw_get' in a 'while' loop to flush the hardware state before

starting a new stream. It is part of the low-level hardware abstraction layer (HAL) and should be used when you need to inspect the raw Channel Status bits (such as User bits or Channel Status bits) directly from the hardware register.

Esempio:

```
u32 raw_cs_data;

/* Flush any stale Channel Status data from the hardware buffer */
/* before starting the actual SPDIFRX stream processing. */
while (ca_spdifrx_hw_get_cs(&raw_cs_data)) {
    /* Data was present and read into raw_cs_data. */
    /* We continue looping until the hardware buffer is empty. */
}

/* Now the hardware is ready for new incoming data. */
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_spdifrx_hw_get_cs function in my caOS application?
A: This function is primarily used during the initialization or synchronization phase of the SPDIFRX driver. It is intended to flush old or irrelevant data from the hardware buffers, often by being called in a loop alongside ca_spdifrx_hw_get to clear the hardware state before starting a new data stream.

Code example conf:1.00 **Q:** Provide a minimal C example showing how to safely read the Channel Status register.
A: u32 cs_data = 0; if (ca_spdifrx_hw_get_cs(&cs_data)) { /* Process cs_data bits: [24] SOB, [23:16] CS, [15:0] user */ }

Parameters conf:1.00 **Q:** What are the requirements for the 'csr' parameter in ca_spdifrx_hw_get_cs?
A: The 'csr' parameter must be a valid memory address pointing to a 32-bit variable (u32). This variable will be updated with the raw SPDIFRX_CSR register value only if the function returns true.

Return conf:1.00 **Q:** What does the boolean return value indicate, and why might it return false?
A: The function returns true if the register was read successfully, which occurs when the CSRNE (Channel Status Register Not Empty) flag is set. It returns false if no new Channel Status data is available (CSRNE = 0), in which case the provided pointer is not updated.

HW differences conf:1.00 **Q:** Are there differences in how ca_spdifrx_hw_get_cs behaves across different STM32 families?
A: Yes. On F4xx, F7xx, and H7xx families, the function performs a hardware register read of SPDIFRX->CSR gated by the SPDIFRX_SR_CSRNE flag. On F1xx, H5xx, and U5xx families, the function is implemented as a stub that always returns false, indicating that the SPDIFRX peripheral or this specific functionality is not supported on those platforms.

`ca_spdifrx_hw_init`

Header: `caos/drivers/inc/ca_glue_hw.h:406`
GUARD: `EN_CA_GLUE_HW`

```
extern u32 ca_spdifrx_hw_init(void); /* clock, pin, CR set up in
   STATE_IDLE; SPDIFRX_CLK Hz, 0 = no SPDIFRX */
```

Initializes the SPDIFRX hardware peripheral, configures clocks, GPIOs, and sets the receiver to idle state.

Ritorna:
Returns the configured SPDIFRX clock frequency in Hz as a u32. Returns 0 if the hardware is not supported on the current chip, if the clock configuration fails, or if the peripheral is disabled.

Note:

- This function is a low-level hardware abstraction layer (HAL) driver. It is not thread-safe and must be called during the system initialization phase before any SPDIFRX data processing tasks are started. It is not intended to be called from an ISR. A common mistake is calling this function multiple times without proper de-initialization, which may lead to redundant clock configuration or GPIO conflicts. It assumes the system clock tree is already in a state where the required peripheral clocks can be derived.

Uso:

- This function is the mandatory first step in the SPDIFRX driver lifecycle. It is typically called by the higher-level 'ca_spdifrx_init' function to prepare the hardware registers and pins. Once initialized, the application can proceed to use 'ca_spdifrx_sync' to wait for incoming data streams or 'ca_spdifrx_read' to capture audio frames. It should be paired with a corresponding stop or de-init sequence if the peripheral needs to be power-cycled or reconfigured.

Esempio:

```c
/* Initialize the SPDIFRX peripheral and check if it is supported */
u32 spdif_clk = ca_spdifrx_hw_init();

if (spdif_clk > 0U) {
    /* Hardware is ready, proceed to sync with the incoming stream */
    /* Wait up to 500ms for a valid SPDIF signal */
    if (ca_spdifrx_sync(500U) != 0U) {
        /* Signal detected, ready to read data */
    } else {
        /* Timeout or no signal detected */
    }
} else {
    /* SPDIFRX not supported or clock init failed */
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_spdifrx_hw_init in my caOS application?
A: It must be called during the system initialization phase, before starting any SPDIFRX data processing tasks. It is the mandatory first step in the driver lifecycle, typically invoked by the higher-level ca_spdifrx_init function to prepare hardware registers and pins.

Code example conf:1.00 **Q:** Provide a minimal example of how to initialize the SPDIFRX hardware.
A: u32 freq = ca_spdifrx_hw_init(); if (freq > 0U) { /* Hardware ready, proceed to sync or read */ }

Return conf:1.00 **Q:** What do the return values of ca_spdifrx_hw_init indicate?
A: The function returns the configured SPDIFRX clock frequency in Hz as a u32. A return value of 0 indicates that the hardware is not supported on the current chip, the clock configuration failed, or the peripheral is disabled.

Error/Edge case conf:1.00 **Q:** What are the common pitfalls when using ca_spdifrx_hw_init?
A: A common mistake is calling the function multiple times without proper de-initialization, which can lead to redundant clock configuration or GPIO conflicts. Additionally, it assumes the system clock tree is already prepared to derive peripheral clocks.

Behavior conf:1.00 **Q:** Is ca_spdifrx_hw_init safe to call from an ISR or in a multi-threaded environment?
A: No, the function is not thread-safe and is not intended to be called from an ISR. It must be called during the system initialization phase.

HW differences conf:1.00 **Q:** How does the hardware initialization differ between STM32F4xx and STM32H7xx families?
A: On F4xx, the function configures the PLLI2S clock and sets the RCC_DCKCFGR2 register. On H7xx, it configures the HSI kernel clock, modifies the D2CCIP1R register for SPDIFSEL, and uses the APB1LENR register for enabling the peripheral.

`ca_spdifrx_hw_state`

Header: `caos/drivers/inc/ca_glue_hw.h:407`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_spdifrx_hw_state(u32 en, bool dma); /* SPDIFRXEN = en (0
   idle, 1 sync, 3 receive), RXDMAEN = dma */
```

Configures the SPDIFRX hardware peripheral state and DMA mode.

Parametri:

Nome	Tipo	Descrizione
`en`	`u32`	Peripheral state: 0 for IDLE, 1 for SYNC, 3 for RECEIVE.
`dma`	`bool`	True to enable DMA transfer, false to disable it.

Ritorna:
void

Note:

- This function directly modifies the SPDIFRX control register. It is not thread-safe; if multiple tasks access the SPDIFRX peripheral, you must wrap calls in a critical section or mutex. It is not intended to be called from an ISR unless you are certain no other context is modifying the peripheral state simultaneously. A common mistake is calling this while a DMA transfer is active without first stopping the DMA, which can lead to hardware state corruption or unexpected interrupts.

Uso:

- This function is used to transition the SPDIFRX peripheral between operational modes, such as resetting to IDLE before re-initialization or switching to RECEIVE mode after synchronization. It is typically paired with functions that drain the hardware buffers (like reading remaining data) to ensure a clean state before changing the mode. It is part of the low-level driver sequence: init -> sync -> state change -> data processing.

Esempio:

```
/* Example: Resetting the SPDIFRX peripheral to a clean state */
void reset_spdif_interface(void) {
    /* Disable peripheral and DMA to ensure no data is being processed */
    ca_spdifrx_hw_state(0, false);

    /* Clear any pending hardware buffers or flags here */
    /* ... code to drain hardware FIFOs ... */

    /* Re-enable in sync mode to wait for incoming signal */
    ca_spdifrx_hw_state(1, false);
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_spdifrx_hw_state in my application logic?
A: You should call this function to transition the SPDIFRX peripheral between operational modes, such as resetting to IDLE before re-initialization or switching to RECEIVE mode after synchronization. It is typically used as part of the sequence: init -> sync -> state change -> data processing, and you should ensure hardware buffers are drained before changing the state.

Code example conf:1.00 **Q:** Provide a minimal C example to switch the SPDIFRX to receive mode with DMA enabled.
A: u32 state = 3U; bool dma_enabled = true; ca_spdifrx_hw_state(state, dma_enabled);

Parameters conf:1.00 **Q:** What are the valid values for the 'en' parameter in ca_spdifrx_hw_state?
A: The 'en' parameter accepts u32 values representing the peripheral state: 0 for IDLE, 1 for SYNC, and 3 for RECEIVE.

HW differences conf:1.00 **Q:** Are there differences in how ca_spdifrx_hw_state is implemented across STM32 families?
A: Yes. On F4xx, F7xx, and H7xx families, the function directly modifies the SPDIFRX->CR register by clearing and setting the SPDIFEN and RXDMAEN bits. Conversely, on F1xx, H5xx, and U5xx families, the function is implemented as a stub that performs no hardware operations, effectively ignoring the parameters.

`ca_spdifrx_hw_width5`

Header: `caos/drivers/inc/ca_glue_hw.h:411`
GUARD: `EN_CA_GLUE_HW`

```
1 extern u32 ca_spdifrx_hw_width5(void); /* SR WIDTH5: SPDIFRX_CLK periods
      in 5 symbols */
```

Reads the SPDIFRX hardware register to get the measured width of 5 symbols in clock periods.

Ritorna:
Returns a 32-bit unsigned integer representing the number of SPDIFRX_CLK periods measured over 5 symbols. Returns 0 if the hardware is not initialized, not supported on the current chip, or if no signal is currently being detected.

Note:

- This function is non-blocking and executes in constant time as it performs a direct register read from the SPDIFRX peripheral. It is thread-safe in the sense that it only performs a read operation, but it should only be called after the SPDIFRX peripheral has been properly initialized. It is safe to call from an ISR, as it does not involve any OS primitives or blocking mechanisms. The most common beginner mistake is assuming this value is a constant frequency; it is a dynamic measurement that changes based on the incoming audio stream's sample rate, so it must be sampled periodically to calculate the actual frequency.

Uso:

- This function is used in the context of digital audio signal processing to determine the sample rate of an incoming SPDIF stream. It is typically paired with a local clock reference to calculate the actual sampling frequency (Fs) by comparing the measured symbol width against the expected ideal width. Developers usually call this within a periodic task or a monitoring function to detect changes in the audio source's sample rate, often following an initialization sequence involving ca_spdifrx_init.

Esempio:

```c
/* Example: Monitoring the incoming SPDIF sample rate */
#include "caos.h"

#define SPDIFRX_W5_BIAS 2U

void monitor_audio_task(void) {
    /* Get the raw hardware measurement for 5 symbols */
    u32 w5 = ca_spdifrx_hw_width5();

    if (w5 > 0U) {
        /* Calculate width including bias for frequency estimation */
        u32 w = w5 + SPDIFRX_W5_BIAS;

        /*
         * Perform frequency calculation based on system clock.
         * s_clk is assumed to be the known peripheral clock frequency.
         */
        u32 s_fs = (5U * 12288000U + 32U * w) / (64U * w);

        ca_printf("Detected sample rate approx: %lu Hz\n", s_fs);
    } else {
        ca_printf("No valid SPDIF signal detected.\n");
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_spdifrx_hw_width5 and what is its primary purpose?
A: You should call this function within a periodic task or monitoring function after initializing the peripheral with ca_spdifrx_init. Its purpose is to measure the width of 5 SPDIF symbols in clock periods, which allows you to calculate the actual sample rate (Fs) of the incoming audio stream by comparing the measured value against an expected ideal width.

Code example conf:1.00 **Q:** Provide a minimal C example showing how to read the symbol width in caOS.
A: u32 width = ca_spdifrx_hw_width5(); if (width > 0) { /* Process valid measurement */ }

Return conf:1.00 **Q:** What does a return value of 0 from ca_spdifrx_hw_width5 indicate?
A: A return value of 0 indicates that the hardware is not initialized, the feature is not supported on the current chip, or no signal is currently being detected by the SPDIFRX peripheral.

HW differences conf:1.00 **Q:** Does ca_spdifrx_hw_width5 behave the same across all STM32 families in caOS?
A: No. On F4xx, F7xx, and H7xx families, the function performs a direct read of the SPDIFRX->SR register to extract the WIDTH5 bits. On other families like F1xx, H5xx, and U5xx, the function is implemented as a stub that returns 0, indicating that the hardware support for this specific register measurement is not available or implemented for those targets.

`ca_spdifrx_init`

Header: `caos/drivers/inc/spdifrx.h:71`
Source: `caos/drivers/src/spdifrx.c:77:83`
GUARD: `EN_SPDIFRX`

```
u32 ca_spdifrx_init(void); /* SPDIFRX_CLK in Hz, 0 = not possible */
```

Initializes the SPDIFRX hardware peripheral and returns the detected clock frequency in Hz.

Ritorna:
Returns the detected clock frequency as a u32 value in Hz. Returns 0 if the initialization failed or if the clock detection was unsuccessful.

Note:

- This function is a non-blocking wrapper that resets internal state variables and triggers the hardware-level initialization. It is not thread-safe; ensure that only one task manages the SPDIFRX peripheral at a time to avoid race conditions. It is not intended to be called from an ISR, as it may involve hardware setup delays. A common beginner mistake is to ignore the return value; if it returns 0, the peripheral is not ready, and attempting to use it will lead to undefined behavior or system hangs.

Uso:

- This function is typically used during the system startup or peripheral configuration phase. It is usually called after initializing related hardware, such as the SAI (Serial Audio Interface), because the SPDIFRX often shares the same clock source (like the PLLI2S on STM32 F4/F7 series). The typical sequence is: initialize the SAI, call ca_spdifrx_init, verify the returned clock frequency is non-zero, and then proceed to start the audio stream.

Esempio:

```
u32 clk = ca_spdifrx_init();
if (clk == 0U) {
    /* Handle initialization failure: log error or retry */
    return false;
}

/* Check if the clock is sufficient for the required sample rate */
if (clk < 48000U) {
    /* Clock too slow for high-quality audio */
    return false;
}

/* Proceed to start audio processing */
/* ca_sai_stream_start(s_tx, NULL, EX104_BLOCK); */
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_spdifrx_init in my caOS application?
A: It is typically called during system startup or the peripheral configuration phase. You should call it after initializing related hardware, such as the SAI, because the SPDIFRX often shares the same clock source (e.g., PLLI2S on STM32 F4/F7 series).

Code example conf:1.00 **Q:** How do I correctly implement the initialization of the SPDIFRX peripheral using caOS types?
A: You must check the return value to ensure the peripheral is ready. Example: u32 clk = ca_spdifrx_init(); if (clk == 0) { /* Handle error */ } else { /* Proceed with audio stream */ }

Return conf:1.00 **Q:** What does the return value of ca_spdifrx_init indicate, and why might it return 0?
A: The function returns the detected clock frequency in Hz as a u32. A return value of 0 indicates that the initialization failed or the clock detection was unsuccessful. If it returns 0, the peripheral is not ready, and using it will lead to undefined behavior or system hangs.

ca_spdifrx_read

Header: caos/drivers/inc/spdifrx.h:74
Source: caos/drivers/src/spdifrx.c:106:129
GUARD: EN_SPDIFRX

```
u32 ca_spdifrx_read(u32 *w, u32 words, u32 timeout_ms); /* polled, words read */
```

Reads a specified number of words from the SPDIF receiver using a polled approach with a timeout.

Parametri:

Nome	Tipo	Descrizione
w	u32 *	Pointer to the buffer where the received data will be stored. Must not be NULL.
words	u32	Number of 32-bit words to read. Must be greater than 0.
timeout_ms	u32	Maximum time in milliseconds to wait for the requested number of words before aborting.

Ritorna:
Returns the number of words successfully read (0 to 'words'). A return value of 0 indicates either a timeout, an invalid parameter, or a hardware error during synchronization.

Note:

- This function is blocking; it will keep the CPU busy in a loop until the requested data is received or the timeout expires. It is NOT thread-safe and must not be called from an ISR, as it relies on system ticks and hardware state management. Before calling this, ensure the SPDIF hardware has been properly initialized. A common mistake is providing a buffer that is too small for the requested number of words, which will cause memory corruption, or setting a timeout that is too short for the incoming data rate, leading to frequent read failures.

Uso:

- This function is typically used in a polling loop to capture a specific burst of audio data or status frames from the SPDIF interface. It is usually paired with ca_spdifrx_errors() to check for data corruption after the read attempt and ca_spdifrx_init() for initial setup. In a typical sequence, you would initialize the hardware, call this function to fill a buffer, check the error status, and then process the data.

Esempio:

```
u32 buffer[16];
u32 received_count;

/* Initialize SPDIF hardware before use */
/* ca_spdifrx_init(); */

/* Attempt to read 16 words with a 100ms timeout */
received_count = ca_spdifrx_read(buffer, 16, 100U);

if (received_count == 16U) {
    /* Data successfully captured */
} else {
    /* Check for errors if read failed or was incomplete */
    u32 err = ca_spdifrx_errors();
    if (err != 0U) {
```

```
        /* Handle hardware errors */
    }
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_spdifrx_read function in my application?
A: This function is used to capture a specific burst of audio data or status frames from the SPDIF interface. It is typically used in a polling loop, usually following hardware initialization with ca_spdifrx_init and followed by a call to ca_spdifrx_errors to verify data integrity.

Code example conf:1.00 **Q:** Provide a minimal C example of how to read 10 words from the SPDIF receiver using caOS types.
A: u32 buffer[10]; u32 read_count = ca_spdifrx_read(buffer, 10, 100); if (read_count < 10) { /* handle error */ }

Parameters conf:1.00 **Q:** What are the constraints on the parameters for ca_spdifrx_read?
A: The pointer 'w' must not be NULL. The 'words' parameter must be greater than 0. The 'timeout_ms' parameter defines the maximum wait time in milliseconds; setting this too short for the incoming data rate will lead to frequent read failures.

Return conf:1.00 **Q:** What does the return value of ca_spdifrx_read indicate?
A: The function returns the number of 32-bit words successfully read, ranging from 0 to the requested 'words' count. A return value of 0 indicates a timeout, an invalid parameter, or a hardware error during synchronization.

Error/Edge case conf:1.00 **Q:** What are common mistakes that lead to errors when using ca_spdifrx_read?
A: Common mistakes include providing a buffer that is too small for the requested number of words, which causes memory corruption, or setting a timeout that is too short for the incoming data rate, resulting in read failures.

Behavior conf:1.00 **Q:** Is ca_spdifrx_read safe to call from an interrupt service routine (ISR) or in a multi-threaded environment?
A: No, it is not thread-safe and must not be called from an ISR. The function is blocking, meaning it keeps the CPU busy in a loop until the data is received or the timeout expires, and it relies on system ticks and hardware state management.

`ca_spdifrx_read_dma`

Header: `caos/drivers/inc/spdifrx.h:80`
Source: `caos/drivers/src/spdifrx.c:195:220`
GUARD: `EN_SPDIFRX`

```
bool ca_spdifrx_read_dma(u32 *w, u32 words, u32 timeout_ms); /* one
    block by DMA */
```

Reads a block of data from the S/PDIF receiver using DMA with a timeout.

Parametri:

Nome	Tipo	Descrizione
`w`	`u32 *`	Pointer to the destination buffer where the received 32-bit words will be stored. Must be valid and large enough to hold 'words' count.
`words`	`u32`	Number of 32-bit words to read. Must be greater than 0.
`timeout_ms`	`u32`	Maximum time in milliseconds to wait for the data transfer to complete.

Ritorna:
Returns true if the transfer completed successfully within the timeout. Returns false if the input parameters are invalid, the hardware fails to sync, the DMA transfer fails, or the timeout expires.

Note:

- This function is blocking; it will suspend the calling task until the DMA transfer finishes or the timeout occurs. It is not safe to call this function from an ISR because it relies on OS-level timing and synchronization primitives. Before calling this, ensure the S/PDIF hardware is initialized and the DMA channel is properly configured. A common mistake is providing a buffer that is too small for the requested number of words, which will lead to memory corruption, or failing to check the return value, which leaves the application unaware of potential data loss or hardware sync errors.

Uso:

- This function is typically used in a data acquisition loop where high-throughput audio or digital data is expected from an S/PDIF source. It is usually paired with a preceding check of the S/PDIF status or channel status to ensure the signal is valid. After the function returns, the application should verify the data integrity (e.g., checking for errors via ca_spdifrx_errors) and then process the buffer. It is often used in a sequence: initialize hardware, enter a loop to read blocks via DMA, process the data, and finally stop the hardware when the task is complete.

Esempio:

```
/* Example: Reading a block of audio data from S/PDIF */
#define BUFFER_SIZE 128
u32 audio_buffer[BUFFER_SIZE];

void task_audio_process(void) {
    /* Attempt to read a block of data with a 500ms timeout */
    bool success = ca_spdifrx_read_dma(audio_buffer, BUFFER_SIZE, 500U);

    if (success) {
        /* Data is ready in audio_buffer, proceed with processing */
        ca_printf("Successfully read %d words.\n", BUFFER_SIZE);
    } else {
        /* Handle error: check hardware status or log failure */
        u32 err = ca_spdifrx_errors();
        ca_printf("Failed to read S/PDIF data. Error code: %u\n", err);
    }
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_spdifrx_read_dma in my application?
A: This function is used for high-throughput data acquisition, such as reading audio or digital data from an S/PDIF source. It is typically used within a loop after verifying that the S/PDIF hardware is initialized and the signal is valid.

Code example conf:1.00 **Q:** Provide a minimal example of how to call ca_spdifrx_read_dma.
A: u32 buffer[128]; if (ca_spdifrx_read_dma(buffer, 128, 1000)) { /* Process data */ }

Parameters conf:1.00 **Q:** What are the constraints on the 'words' parameter in ca_spdifrx_read_dma?
A: The 'words' parameter represents the number of 32-bit words to read and must be greater than 0. The destination buffer must be large enough to hold this count to avoid memory corruption.

Return conf:1.00 **Q:** What does a return value of false indicate for ca_spdifrx_read_dma?

A: A return value of false indicates that the transfer failed. This can be caused by invalid input parameters, hardware sync failure, DMA transfer failure, or the timeout expiring before the transfer completed.

Error/Edge case conf:1.00 **Q:** What are common mistakes when using ca_spdifrx_read_dma?
A: Common mistakes include providing a buffer that is too small for the requested number of words, which causes memory corruption, or failing to check the return value, which leaves the application unaware of data loss or hardware sync errors.

Behavior conf:1.00 **Q:** Is ca_spdifrx_read_dma safe to call from an ISR?
A: No, it is not safe to call from an ISR because the function is blocking and relies on OS-level timing and synchronization primitives to suspend the calling task.

`ca_spdifrx_stop`

Header: `caos/drivers/inc/spdifrx.h:77`
Source: `caos/drivers/src/spdifrx.c:224:235`
GUARD: `EN_SPDIFRX`

```
void ca_spdifrx_stop(void)
```

Stops the SPDIF receiver hardware and releases associated DMA resources.

Ritorna:
void

Note:

- This function is non-blocking and performs a clean shutdown of the SPDIF peripheral. It is not thread-safe; if multiple tasks access the SPDIF hardware, you must wrap this call in a mutex or scheduler lock to prevent race conditions. It is generally not safe to call this from an ISR because it may trigger DMA abort sequences that require task-level context. The most common mistake is calling this function while a DMA transfer is still being processed by another task, which can lead to inconsistent hardware states or memory corruption if the DMA buffer is freed immediately after.

Uso:

- Use this function when you need to stop audio streaming or reconfigure the hardware pins for a different purpose. It is typically paired with ca_spdifrx_init at the start of a session and ca_spdifrx_read_dma during active operation. In a typical lifecycle, you call this to halt the peripheral, then proceed to reconfigure GPIOs or power down the clock tree to save energy or switch modes.

Esempio:

```
void stop_audio_system(void) {
    // Stop the SPDIF receiver hardware and release DMA
    ca_spdifrx_stop();

    // Reconfigure pins to a safe state (e.g., high impedance or input)
    ca_gpio_configure(CA_GPIO_PORT_D, 7, CA_GPIO_MODE_INPUT,
                      CA_GPIO_OTYPE_PP, CA_GPIO_SPEED_LOW, 0, 0);

    // Log the event using standard caOS output
    ca_printf("SPDIF receiver stopped successfully.\n");
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_spdifrx_stop in my application lifecycle?
A: You should call ca_spdifrx_stop when you need to halt audio streaming or reconfigure the SPDIF hardware pins for a different purpose. It is typically used to shut down the peripheral before reconfiguring GPIOs or powering down the clock tree to save energy.

Code example conf:0.90 **Q:** How do I properly stop the SPDIF receiver in my caOS application?
A: Since the function is not thread-safe, you must ensure access is protected if multiple tasks are involved. Example: { ca_mutex_lock(&spdif_mutex); ca_spdifrx_stop(); ca_mutex_unlock(&spdif_mutex); }

Error/Edge case conf:1.00 **Q:** What are the risks of calling ca_spdifrx_stop while a DMA transfer is active?
A: Calling this function while a DMA transfer is still being processed can lead to inconsistent hardware states or memory corruption, especially if the DMA buffer is freed immediately after the call. Additionally, it is not safe to call from an ISR because it may trigger DMA abort sequences requiring task-level context.

`ca_spdifrx_sync`

Header: `caos/drivers/inc/spdifrx.h:72`
Source: `caos/drivers/src/spdifrx.c:85:94`
GUARD: `EN_SPDIFRX`

```
u32 ca_spdifrx_sync(u32 timeout_ms); /* measured fs, 0 = no stream */
```

Synchronizes the SPDIF receiver with an incoming audio stream and measures the sample rate.

Parametri:

Nome	Tipo	Descrizione
`timeout_ms`	`u32`	Maximum time in milliseconds to wait for a valid SPDIF signal lock. A value of 0 means no wait (immediate return).

Ritorna:
Returns the measured sample rate (fs) in Hz as a u32. Returns 0 if no valid stream is detected within the timeout period or if the hardware is not initialized.

Note:

- This function is a blocking call; it will pause the execution of the calling task until the signal is locked or the timeout expires. It is not thread-safe and must not be called from an ISR, as it relies on OS timing mechanisms. Before calling this, ensure the SPDIF hardware has been initialized via the appropriate hardware setup function. A common beginner mistake is setting the timeout too low for the incoming signal to stabilize, resulting in a false 0 return even when a valid cable is connected.

Uso:

- This function is typically used in audio processing tasks to verify the presence and quality of an incoming digital audio signal before starting a data stream. It is usually paired with ca_spdifrx_errors to check for signal integrity and ca_spdifrx_channel_status to retrieve metadata. The typical sequence is: initialize hardware, call ca_spdifrx_sync to lock the clock, check for errors, and then proceed to read data or configure downstream audio peripherals.

Esempio:

```c
u32 fs_val;
/* Wait up to 500ms for the SPDIF signal to lock */
fs_val = ca_spdifrx_sync(500U);

if (fs_val > 0U) {
    /* Signal locked, check for any hardware errors */
    if (ca_spdifrx_errors() == 0U) {
        /* Proceed with audio processing */
        ca_printf("Stream locked at %u Hz\n", fs_val);
    } else {
        ca_printf("Stream locked but errors detected\n");
    }
} else {
    /* Timeout reached or no signal present */
    ca_printf("Failed to sync with SPDIF stream\n");
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_spdifrx_sync in my audio processing application?
A: You should call it after initializing the SPDIF hardware to verify the presence and quality of an incoming digital audio signal. It is typically used to lock the clock before checking for errors with ca_spdifrx_errors or retrieving metadata via ca_spdifrx_channel_status.

Code example conf:1.00 **Q:** Provide a minimal C code snippet showing how to use ca_spdifrx_sync.
A: u32 fs = 0; u32 timeout = 500; fs = ca_spdifrx_sync(timeout); if (fs > 0) { /* Proceed with audio stream */ }

Parameters conf:1.00 **Q:** What is the significance of the timeout_ms parameter in ca_spdifrx_sync?
A: The timeout_ms parameter defines the maximum time in milliseconds the function will block while waiting for a valid SPDIF signal lock. A value of 0 results in an immediate return without waiting. Setting this value too low is a common mistake that causes the function to return 0 even when a valid cable is connected, as the signal needs time to stabilize.

Return conf:1.00 **Q:** What does a return value of 0 indicate from ca_spdifrx_sync?
A: A return value of 0 indicates that no valid stream was detected within the specified timeout period or that the SPDIF hardware was not initialized. It signifies a failure to lock onto the incoming audio signal.

`ca_spi_cs_assert`

Header: `caos/drivers/inc/spi.h:75`
Source: `caos/drivers/src/spi.c:90:96`
GUARD: `EN_SPI`

```c
void ca_spi_cs_assert(u8 spi_num)
```

Activates the Chip Select (CS) pin for a specific SPI peripheral to initiate communication.

Parametri:

Nome	Tipo	Descrizione
`spi_num`	`u8`	The SPI peripheral index (1-based). Must be within the range [1, SPI_MAX_NUM]. Values outside this range are ignored.

Ritorna:
void. No status is returned; if an invalid index is provided, the function returns silently without performing any action.

Note:

- This function is non-blocking and executes immediately by writing to the GPIO register configured for the CS pin. It is thread-safe in the sense that it performs a single atomic GPIO write, but it does not provide mutual exclusion for the SPI bus itself; if multiple tasks access the same SPI bus, you must use a mutex to prevent data corruption. It is safe to call from an ISR, provided the underlying GPIO driver is also ISR-safe. The most common mistake is calling this function without having first initialized the SPI peripheral and the associated GPIO pin via ca_spi_init, which results in the CS pin remaining in an undefined or inactive state.

Uso:

- This function is used to signal the start of a SPI transaction to a slave device. It is typically called immediately before sending or receiving data (e.g., via ca_spi_dma_xfer or ca_spi_xfer). The standard sequence is: ca_spi_cs_assert() -> [perform data transfer] -> ca_spi_cs_deassert(). It is essential to ensure that the CS pin is correctly configured in the SPI initialization structure so that the driver knows which GPIO port and pin to toggle.

Esempio:

```
/* Initialize SPI peripheral 1 */
ca_spi_init(1U);

/* Prepare to send data to a sensor */
u8 data_to_send[] = {0x9F, 0x00, 0x00};

/* Assert CS to select the slave device */
ca_spi_cs_assert(1U);

/* Perform the SPI transfer (assuming a blocking transfer function exists) */
/* ca_spi_xfer(1U, data_to_send, rx_buffer, 3U); */

/* Deassert CS to release the slave device */
ca_spi_cs_deassert(1U);
```

Q&A — fine-tuning review *2/3 pairs · qwen3-coder:latest*

Usage conf:0.90 **Q:** When should I call ca_spi_cs_assert and why is it important for SPI communication?

A: You should call ca_spi_cs_assert before initiating an SPI transaction to select the slave device by pulling the Chip Select (CS) line low. This function is important because it ensures that the SPI bus is properly configured for communication with a specific peripheral. Since the function is not implemented in the provided code, the actual assertion of the CS line is typically handled by the calling code, not by this function itself.

Parameters conf:0.95 **Q:** What does the spi_num parameter in ca_spi_cs_assert represent, and what are the valid values?

A: The spi_num parameter represents the SPI peripheral number to be used, ranging from 1 to SPI_MAX_NUM. Values less than 1 or greater than SPI_MAX_NUM will cause the function to exit immediately without performing any action. This parameter is essential for identifying which SPI bus to control, as different SPI peripherals are mapped to specific hardware registers on the STM32 microcontroller.

`ca_spi_cs_deassert`

Header: `caos/drivers/inc/spi.h:76`
Source: `caos/drivers/src/spi.c:98:104`
GUARD: `EN_SPI`

```
void ca_spi_cs_deassert(u8 spi_num)
```

Deasserts (sets to high) the Chip Select (CS) pin for a specific SPI peripheral.

Parametri:

Nome	Tipo	Descrizione
spi_num	u8	The SPI peripheral index (1-based). Must be between 1 and SPI_MAX_NUM. Values outside this range are ignored.

Ritorna:
void

Note:

- This function is non-blocking and executes immediately by writing to the GPIO pin configured as CS. It is thread-safe in the sense that it performs a single atomic GPIO write, but it does not provide mutual exclusion for the SPI bus itself; you must ensure no other task is using the bus simultaneously. It is safe to call from an ISR, provided the underlying GPIO driver is also ISR-safe. The most common mistake is calling this before the SPI peripheral has been initialized via ca_spi_init, which results in the function operating on uninitialized or default GPIO settings.

Uso:

- This function is used to signal the end of a SPI transaction to a slave device. It is typically called after a data transfer function (like ca_spi_dma_xfer or ca_spi_xfer) to release the bus. The standard sequence is: ca_spi_cs_assert(n) -> [perform data transfer] -> ca_spi_cs_deassert(n). It is only effective if the SPI peripheral was configured with 'manage_cs' enabled during initialization.

Esempio:

```
/* Assuming SPI peripheral 1 is initialized and managed by caOS */
u8 spi_id = 1U;

/* Start communication */
ca_spi_cs_assert(spi_id);

/* Perform data transfer using DMA */
/* ca_spi_dma_xfer(spi_id, tx_buffer, rx_buffer, length); */

/* End communication to allow other devices to use the bus */
ca_spi_cs_deassert(spi_id);
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_spi_cs_deassert() in my SPI communication code?
A: You should call ca_spi_cs_deassert() after completing an SPI transaction to release the Chip Select (CS) line, indicating that the slave device is no longer selected. This function is typically called after writing or reading data to/from an SPI peripheral.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_spi_cs_deassert() with caOS types?
A: u8 spi_num = 1; ca_spi_cs_deassert(spi_num);

Parameters conf:0.95 **Q:** What does the spi_num parameter in ca_spi_cs_deassert() represent and what are valid values?
A: The spi_num parameter represents the SPI interface number, ranging from 1 to 4. Valid values are 1, 2, 3, and 4. If spi_num is 0 or greater than 4, the function returns immediately without effect.

Return conf:0.90 **Q:** What causes ca_spi_cs_deassert() to fail or not perform any action?
A: The function does not return a value and cannot fail in a strict sense. It will not perform any action if the spi_num parameter is invalid (0 or >4) or if the global flag 'manage_cs' is set to false, indicating that CS management is disabled.

`ca_spi_dma_read`

Header: `caos/drivers/inc/spi.h:111`
Source: `caos/drivers/src/spi.c:309:312`
GUARD: `EN_SPI`

```
u16 ca_spi_dma_read(u8 spi_num, u8 *buf, u16 len)
```

Reads a specified number of bytes from an SPI peripheral using DMA.

Parametri:

Nome	Tipo	Descrizione
`spi_num`	`u8`	The index of the SPI peripheral to use (e.g., 0, 1). Must correspond to a hardware instance initialized via ca_spi_init_dma.
`buf`	`u8 *`	Pointer to the memory buffer where the received data will be stored. Must be large enough to hold 'len' bytes.
`len`	`u16`	Number of bytes to read. Must be greater than 0.

Ritorna:
Returns a status code (u16). Typically, 0 indicates success, while non-zero values indicate an error (e.g., peripheral busy, invalid parameters, or DMA transfer failure).

Note:

- This function is a wrapper around ca_spi_dma_xfer and is non-blocking, meaning it initiates the DMA transfer and returns immediately while the hardware handles the data movement in the background. It is not safe to call from an ISR unless the underlying driver explicitly supports it, and it requires the SPI peripheral to have been previously initialized with ca_spi_init_dma. A common mistake is to modify or free the 'buf' memory before the DMA transfer has completed, which leads to data corruption; always ensure the transfer is finished before accessing the buffer.

Uso:

- This function is used when high-throughput data reception is required without stalling the CPU. It is typically used in a sequence: first, initialize the SPI peripheral with ca_spi_init_dma, then assert the Chip Select line using ca_spi_cs_assert, call ca_spi_dma_read to fetch the data, and finally deassert the Chip Select line using ca_spi_cs_deassert once the transfer is confirmed complete.

Esempio:

```
u8 rx_buffer[16];
// Ensure SPI is initialized for DMA before use
ca_spi_init_dma(0);

// Assert Chip Select to start communication
ca_spi_cs_assert(0);

// Perform non-blocking DMA read
```

```c
if (ca_spi_dma_read(0, rx_buffer, 16) == 0) {
    // Wait for transfer completion (e.g., using a semaphore or event)
    // ca_event_wait(...);
} else {
    // Handle error: SPI busy or invalid parameters
}

// Deassert Chip Select after transfer
ca_spi_cs_deassert(0);
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_spi_dma_read instead of a standard blocking SPI read?
A: You should use ca_spi_dma_read when high-throughput data reception is required without stalling the CPU. Because it is non-blocking, it initiates the DMA transfer and returns immediately, allowing the hardware to handle data movement in the background.

Code example conf:0.90 **Q:** Provide a minimal example of how to perform an SPI read using DMA in caOS.
A: u8 buffer[16]; u16 status; ca_spi_cs_assert(0); status = ca_spi_dma_read(0, buffer, 16); if (status == 0) { /* Wait for completion before accessing buffer */ } ca_spi_cs_deassert(0);

Parameters conf:1.00 **Q:** What are the constraints on the parameters for ca_spi_dma_read?
A: The spi_num must correspond to a hardware instance previously initialized via ca_spi_init_dma. The buf pointer must point to memory large enough to hold len bytes, and len must be greater than 0.

Return conf:1.00 **Q:** How do I interpret the return value of ca_spi_dma_read?
A: The function returns a u16 status code. A return value of 0 indicates success. Non-zero values indicate an error, such as the peripheral being busy, invalid parameters, or a DMA transfer failure.

`ca_spi_dma_write`

Header: `caos/drivers/inc/spi.h:110`
Source: `caos/drivers/src/spi.c:304:307`
GUARD: `EN_SPI`

```c
u16 ca_spi_dma_write(u8 spi_num, const u8 *buf, u16 len)
```

Transmits a buffer of data over SPI using DMA without receiving data.

Parametri:

Nome	Tipo	Descrizione
`spi_num`	`u8`	The index of the SPI peripheral to use (e.g., 0, 1, 2). Must match the hardware configuration.
`buf`	`const u8 *`	Pointer to the data buffer to be sent. Must be a valid memory address and remain unchanged until the transfer completes.
`len`	`u16`	Number of bytes to transmit. A value of 0 will result in no action.

Ritorna:
Returns the number of bytes requested to be sent (len) upon successful initiation. If the SPI peripheral is busy or the DMA channel fails to initialize, it may return 0 or an error code depending on the underlying driver implementation.

Note:

- This function is a wrapper around the DMA transfer mechanism and is typically blocking until the transfer is queued or completed, depending on the driver's internal implementation. It is not safe to call from an ISR unless the driver explicitly supports it, as it may involve waiting for hardware flags. A common beginner mistake is to modify or free the 'buf' memory immediately after calling this function; because DMA operates independently of the CPU, the buffer must remain valid until the hardware has finished shifting all bits out. Ensure the SPI peripheral has been initialized via ca_spi_init and ca_spi_init_dma before calling this function.

Uso:

- This function is used when you need to send command sequences or data packets to an external SPI device (like a display or sensor) efficiently without consuming CPU cycles during the transmission. It is typically used in a sequence: first, assert the Chip Select (CS) line using ca_spi_cs_assert, then call ca_spi_dma_write to send the payload, and finally deassert the CS line using ca_spi_cs_deassert once the transfer is confirmed complete. It is the standard way to perform high-speed SPI output in a multitasking RTOS environment.

Esempio:

```
/* Example: Sending a command to an external SPI sensor */
u8 cmd_buffer[] = {0x9F, 0x00, 0x00}; // Example command sequence

// 1. Select the device
ca_spi_cs_assert(1);

// 2. Send data via DMA
// The function returns the length if successful
if (ca_spi_dma_write(1, cmd_buffer, sizeof(cmd_buffer)) == sizeof(cmd_buffer)) {
    /* Wait for hardware to finish transmission before releasing CS */
    /* In a real scenario, use a semaphore or event to wait for completion */
    ca_os_delay_ms(1);
}

// 3. Deselect the device
ca_spi_cs_deassert(1);
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_spi_dma_write instead of a standard blocking SPI write?
A: You should use ca_spi_dma_write when you need to send data packets or command sequences to external SPI devices efficiently without consuming CPU cycles during the transmission. It is the standard method for high-speed SPI output in a multitasking caOS environment.

Code example conf:0.95 **Q:** How do I correctly implement a DMA SPI write sequence in caOS?
A: You must ensure the SPI is initialized first. Example: u8 data[] = {0x01, 0x02}; ca_spi_cs_assert(0); ca_spi_dma_write(0, data, 2); ca_spi_cs_deassert(0); Note that the buffer must remain valid until the transfer completes.

Parameters conf:1.00 **Q:** What are the constraints on the 'buf' parameter for ca_spi_dma_write?
A: The 'buf' parameter must be a valid memory address. Crucially, the buffer must remain unchanged and valid until the DMA transfer completes, because DMA operates independently of the CPU. Modifying or freeing the buffer immediately after the function call is a common error.

Return conf:1.00 **Q:** What does the return value of ca_spi_dma_write indicate?
A: The function returns the number of bytes requested (len) upon successful initiation. If the SPI peripheral is busy or the DMA channel fails to initialize, it may return 0 or

an error code depending on the driver implementation.

---

ca_spi_dma_xfer

Header: caos/drivers/inc/spi.h:107
Source: caos/drivers/src/spi.c:260:302
GUARD: EN_SPI

```
u16 ca_spi_dma_xfer(u8 spi_num, const u8 *tx, u8 *rx, u16 len)
```

Performs a full-duplex SPI data transfer using DMA, blocking the calling task until completion.

Parametri:

Nome	Tipo	Descrizione
spi_num	u8	The index of the SPI peripheral to use. Must correspond to a valid, initialized SPI instance.
tx	const u8 *	Pointer to the buffer containing data to send. If NULL, the function performs a read-only operation by sending dummy 0xFF bytes.
rx	u8 *	Pointer to the buffer where received data will be stored. If NULL, the function performs a write-only operation.
len	u16	Number of bytes to transfer. Must be greater than 0.

Ritorna:
Returns the number of bytes transferred (len) on success, or 0 if the parameters are invalid or the DMA transfer failed to initialize.

Note:

- This function is blocking; it puts the calling task into a wait state until the DMA hardware signals completion via interrupts. It is NOT safe to call from an ISR because it relies on task-level synchronization (ca_dma_wait). Before calling this, ensure the SPI peripheral has been initialized via ca_spi_init. A common mistake is passing buffers that are not properly aligned or are located in memory regions inaccessible to the DMA controller (e.g., certain stack locations or non-DMA-capable SRAM).

Uso:

- This function is the primary way to perform high-speed SPI communication in caOS. It is typically used in a sequence: first, initialize the SPI peripheral with ca_spi_init, then prepare your data buffers, and finally call ca_spi_dma_xfer to perform the transaction. It is often paired with ca_spi_cs_assert and ca_spi_cs_deassert to manage the Chip Select line manually if the hardware does not handle it automatically.

Esempio:

```
/* Example: Sending a command and reading a response */
#define SPI_ID 1
#define BUF_SIZE 16

u8 tx_buf[BUF_SIZE] = {0x9F, 0x00, 0x00, 0x00}; /* Example command */
u8 rx_buf[BUF_SIZE];

/* Ensure SPI is initialized before use */
ca_spi_init(SPI_ID);
```

```

/* Assert Chip Select manually */
ca_spi_cs_assert(SPI_ID);

/* Perform DMA transfer */
u16 transferred = ca_spi_dma_xfer(SPI_ID, tx_buf, rx_buf, BUF_SIZE);

/* Deassert Chip Select */
ca_spi_cs_deassert(SPI_ID);

if (transferred != BUF_SIZE) {
    /* Handle error: transfer failed */
} else {
    /* Process received data in rx_buf */
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_spi_dma_xfer instead of a standard polling SPI function?
A: You should use ca_spi_dma_xfer for high-speed SPI communication where you want to offload the data transfer to the DMA controller. It is the primary method in caOS for SPI transfers and is typically used after initializing the peripheral with ca_spi_init and managing the Chip Select line manually with ca_spi_cs_assert and ca_spi_cs_deassert.

Code example conf:1.00 **Q:** Provide a minimal example of how to perform a full-duplex SPI transfer using caOS types.
A: u8 tx_buf[] = {0x01, 0x02}; u8 rx_buf[2]; u16 result; ca_spi_cs_assert(1); result = ca_spi_dma_xfer(1, tx_buf, rx_buf, 2); ca_spi_cs_deassert(1);

Parameters conf:1.00 **Q:** What happens if I pass NULL to the tx or rx parameters in ca_spi_dma_xfer?
A: If tx is NULL, the function performs a read-only operation by sending dummy 0xFF bytes. If rx is NULL, the function performs a write-only operation.

Return conf:1.00 **Q:** What does a return value of 0 indicate from ca_spi_dma_xfer?
A: A return value of 0 indicates that the transfer failed. This occurs if the provided parameters are invalid (such as a len of 0) or if the DMA transfer failed to initialize.

Error/Edge case conf:1.00 **Q:** Why might my SPI transfer fail even if my parameters are correct?
A: Common mistakes include passing buffers that are not properly aligned or are located in memory regions inaccessible to the DMA controller, such as certain stack locations or non-DMA-capable SRAM. Additionally, ensure the SPI peripheral was correctly initialized via ca_spi_init first.

Behavior conf:1.00 **Q:** Is ca_spi_dma_xfer safe to call from an interrupt service routine (ISR)?
A: No, it is not safe to call from an ISR. The function is blocking and relies on task-level synchronization (ca_dma_wait) to put the calling task into a wait state until the DMA hardware signals completion.

`ca_spi_hw_dma_bind`

Header: `caos/drivers/inc/ca_glue_hw.h:246`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_spi_hw_dma_bind(u8 spi_num, i16 *rx_flat, u16 *rx_req,
    i16 *tx_flat, u16 *tx_req); /* canale DMA (flat, -1 = qualsiasi) +
    request RX/TX della SPI */
```

Retrieves the hardware-specific DMA stream and request identifiers required to configure a SPI peripheral for DMA transfers.

Parametri:

Nome	Tipo	Descrizione
`spi_num`	`u8`	The SPI peripheral index (e.g., 1, 2, or 3). Must correspond to valid hardware instances.
`rx_flat`	`i16 *`	Pointer to store the RX DMA stream/channel index. Returns -1 if the system allows dynamic allocation.
`rx_req`	`u16 *`	Pointer to store the RX DMA request/channel selection ID required by the DMA controller.
`tx_flat`	`i16 *`	Pointer to store the TX DMA stream/channel index. Returns -1 if the system allows dynamic allocation.
`tx_req`	`u16 *`	Pointer to store the TX DMA request/channel selection ID required by the DMA controller.

Ritorna:
Returns true if the SPI index is valid and parameters were successfully populated. Returns false if the spi_num is invalid or unsupported.

Note:

- This function is non-blocking and thread-safe as it performs simple memory writes based on constant lookup tables. It is safe to call from an ISR, provided the pointers passed are valid. It does not initialize the DMA hardware itself; it only provides the necessary configuration data. The most common mistake is failing to check the boolean return value, which leads to using uninitialized or garbage values for DMA configuration, potentially causing hard faults or peripheral lockups.

Uso:

- This function is typically used during the initialization phase of a SPI driver. It is called before requesting a DMA channel from the OS allocator. The sequence is: 1) Call ca_spi_hw_dma_bind to get the hardware requirements. 2) Use the returned 'flat' values with ca_dma_free_channel or ca_dma_free_channel_index to reserve the hardware resources. 3) Pass the 'req' values to the DMA controller configuration registers. This ensures that the driver respects hardware-specific constraints and avoids conflicts with other peripherals like UARTs.

Esempio:

```
/* Example: Initializing DMA for SPI1 */
u8 spi_id = 1;
i16 rx_flat, tx_flat;
u16 rx_req, tx_req;
ca_dma_ch_t rx_dma, tx_dma;

/* Get hardware requirements */
if (ca_spi_hw_dma_bind(spi_id, &rx_flat, &rx_req, &tx_flat, &tx_req)) {
    /* Allocate DMA channels based on the binding info */
    i16 rx_got = (rx_flat >= 0) ? ca_dma_free_channel_index(rx_flat, &rx_dma) :
    ca_dma_free_channel(&rx_dma);
    i16 tx_got = (tx_flat >= 0) ? ca_dma_free_channel_index(tx_flat, &tx_dma) :
    ca_dma_free_channel(&tx_dma);

    if (rx_got >= 0 && tx_got >= 0) {
        /* Proceed to configure DMA registers using rx_req and tx_req */
        /* ca_dma_set_request(&rx_dma, rx_req); */
    } else {
        /* Handle allocation failure */
    }
```

```
} else {
    /* Handle invalid SPI ID */
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_spi_hw_dma_bind in my SPI driver initialization?
A: You should call it during the initialization phase, specifically before requesting a DMA channel from the OS allocator. The sequence is to first retrieve the hardware requirements via this function, then use the returned 'flat' values to reserve resources, and finally pass the 'req' values to the DMA controller configuration registers.

Code example conf:1.00 **Q:** Provide a minimal C example of how to use ca_spi_hw_dma_bind.
A: u8 spi_idx = 1; i16 rx_f, tx_f; u16 rx_r, tx_r; if (ca_spi_hw_dma_bind(spi_idx, &rx_f, &rx_r, &tx_f, &tx_r)) { /* Proceed to reserve DMA channels using rx_f and tx_f */ }

Parameters conf:1.00 **Q:** What do the 'flat' and 'req' parameters represent in ca_spi_hw_dma_bind?
A: 'flat' represents the DMA stream or channel index, which may return -1 if the system allows dynamic allocation. 'req' represents the DMA request or channel selection ID required by the DMA controller to map the SPI peripheral to the DMA hardware.

Return conf:1.00 **Q:** What does a false return value from ca_spi_hw_dma_bind indicate?
A: A false return value indicates that the provided spi_num is either invalid or unsupported by the current hardware configuration. You should not proceed with DMA configuration if this function returns false.

Error/Edge case conf:1.00 **Q:** What is the most common mistake when using ca_spi_hw_dma_bind?
A: The most common mistake is failing to check the boolean return value. Ignoring this check can lead to using uninitialized or garbage values for DMA configuration, which may result in hard faults or peripheral lockups.

HW differences conf:0.90 **Q:** How do SPI hardware implementations differ across STM32 families in caOS?
A: Implementations vary significantly: F1xx/F4xx use standard DR registers, F7xx requires mandatory 8-bit access to DR to avoid FIFO consumption issues, while H5xx/U5xx utilize newer configuration registers like CFG1, CFG2, and CR2 for infinite mode or specific data sizing.

`ca_spi_hw_dma_finish`

Header: `caos/drivers/inc/ca_glue_hw.h:252`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_spi_hw_dma_finish(u8 spi_num, bool rx_en); /* attende
    fine shift (EOT/BSY), clear flag, ripristina stato idle */
```

Finalizes a DMA-based SPI transfer by waiting for hardware completion and resetting the peripheral to an idle state.

Parametri:

Nome	Tipo	Descrizione
`spi_num`	`u8`	The index of the SPI peripheral to finalize (e.g., 0, 1, 2).
`rx_en`	`bool`	Set to true if RX was enabled during the transfer, false for TX-only. This helps the driver decide whether to clear RX FIFO/OVR flags.

Ritorna:

void

Note:

- This function is blocking; it polls hardware status registers (like EOT or BSY) until the SPI peripheral has finished shifting all bits. It is NOT thread-safe and must not be called from an ISR, as it contains busy-wait loops that could hang the system if the hardware fails to signal completion. A common beginner mistake is calling this function before the DMA transfer has actually finished or while the SPI peripheral is still actively communicating, which can lead to data corruption or unexpected hardware faults (like Mode Faults). Ensure the DMA transfer is fully completed or aborted before invoking this cleanup routine.

Uso:

- This function is used as the final step in a DMA-based SPI communication sequence. It is typically called after the DMA transfer has been triggered and verified, or in an error-handling block if the DMA setup fails. It pairs with functions like ca_spi_dma_xfer or manual DMA configuration to ensure the SPI peripheral is returned to a clean, idle state, preventing the bus from being locked or left in an inconsistent configuration for the next transaction.

Esempio:

```
/* Example: Finalizing a DMA SPI transfer */
void perform_spi_transfer(u8 spi_num, u8 *tx_buf, u8 *rx_buf, u32 len) {
    /* Assume DMA channels are already configured and triggered */
    /* Wait for DMA completion using ca_dma_wait */
    if (ca_dma_wait(s_sdma[spi_num].tx_ch, 100) == 0) {
        /* Cleanup SPI hardware after transfer */
        ca_spi_hw_dma_finish(spi_num, true);
    } else {
        /* Handle timeout or error */
        ca_spi_hw_dma_finish(spi_num, false);
    }
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_spi_hw_dma_finish in my SPI communication sequence?
A: You should call it as the final step after a DMA-based SPI transfer has been triggered and verified. It is used to ensure the SPI peripheral is returned to a clean, idle state, preventing the bus from being locked or left in an inconsistent configuration for the next transaction.

Code example conf:1.00 **Q:** Provide a minimal example of how to finalize a DMA SPI transfer using caOS types.
A: u8 spi_id = 1U; bool rx_enabled = true; /* ... wait for DMA completion ... */ ca_spi_hw_dma_finish(spi_id, rx_enabled);

Parameters conf:1.00 **Q:** What is the purpose of the rx_en parameter in ca_spi_hw_dma_finish?
A: The rx_en parameter is a boolean that indicates if RX was enabled during the transfer. It helps the driver determine whether it needs to clear RX FIFO or OVR (overrun) flags during the finalization process.

Error/Edge case conf:1.00 **Q:** Can I call ca_spi_hw_dma_finish from an interrupt service routine (ISR)?
A: No, you must not call it from an ISR. The function is blocking and contains busy-wait loops that poll hardware status registers; calling it from an ISR could hang the system if the hardware fails to signal completion.

HW differences conf:1.00 **Q:** How do the SPI register interfaces differ across the supported STM32 families?
A: The register access varies based on the architecture: older families like f1xx, f4xx, and f7xx use the DR register for data access, whereas newer families like h5xx, h7xx, and u5xx utilize dedicated RXDR registers for data reception.

`ca_spi_hw_dma_kick`

Header: `caos/drivers/inc/ca_glue_hw.h:251`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_spi_hw_dma_kick(u8 spi_num); /* post-arm: TXDMAEN (+SPE/
    CSTART su IP v3)     il transfer parte qui */
```

Triggers the hardware SPI DMA transfer process by enabling the peripheral and starting the clock.

Parametri:

Nome	Tipo	Descrizione
`spi_num`	`u8`	The index of the SPI peripheral to trigger. Must be a valid index previously initialized by the system.

Ritorna:
void

Note:

- This function is non-blocking and performs direct register manipulation to initiate hardware-level data movement. It is not thread-safe and must not be called from an ISR unless the system design guarantees that no other task is accessing the same SPI peripheral registers simultaneously. The most common mistake is calling this function before the DMA channels and SPI peripheral have been fully configured, which will result in a hardware fault or an incomplete transfer. Ensure that the SPI peripheral is in the correct state (e.g., chip select asserted) before invoking this, as the transfer starts immediately upon execution.

Uso:

- This function is the final step in the SPI DMA transmission sequence. It is typically used after the DMA channels have been configured and the SPI peripheral has been prepared for data exchange. It is almost always followed by a call to ca_dma_wait to block the current task until the hardware transfer completes, ensuring data integrity before the next operation or before deasserting the chip select line.

Esempio:

```
/* Assuming spi_num 1 is already initialized and DMA channels are configured */
ca_spi_cs_assert(1);

/* Trigger the hardware DMA transfer */
ca_spi_hw_dma_kick(1);

/* Wait for the DMA transfer to complete before proceeding */
ca_dma_wait(tx_dma_channel_handle);

ca_spi_cs_deassert(1);
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** At what point in the SPI transmission sequence should I call ca_spi_hw_dma_kick?
A: It should be called as the final step after the DMA channels and SPI peripheral have been fully configured. It is typically followed by ca_dma_wait to block the task until the hardware transfer completes, ensuring data integrity before deasserting the chip select line.

Code example conf:0.90 **Q:** Provide a minimal C example of how to initiate an SPI DMA transfer using caOS.
A: Assuming the SPI and DMA are already configured:
ca_spi_hw_dma_kick(0); ca_dma_wait(dma_channel_index);

Parameters conf:1.00 **Q:** What does the spi_num parameter represent in ca_spi_hw_dma_kick?
A: The spi_num parameter is a u8 representing the index of the SPI peripheral to trigger. It must be a valid index that has been previously initialized by the system.

HW differences conf:0.95 **Q:** Are there architectural differences in how SPI DMA is handled across STM32 families in caOS?
A: Yes. While older families like f1xx, f4xx, and f7xx use the standard DR register for DMA, newer families like h5xx, h7xx, and u5xx utilize SPI v3 architecture, which involves TSIZE and EOT (End of Transfer) mechanisms and requires specific DMA binding via GPDMA or DMAMUX1.

`ca_spi_hw_dma_prepare`

Header: `caos/drivers/inc/ca_glue_hw.h:250`
GUARD: `EN_CA_GLUE_HW`

```
1 extern void ca_spi_hw_dma_prepare(u8 spi_num, u16 len, bool rx_en); /*
    pre-arm: RXDMAEN (+TSIZE=len e SPE=0 su IP v3) */
```

Prepares the SPI hardware for a DMA-based transfer by configuring registers and clearing error flags.

Parametri:

Nome	Tipo	Descrizione
`spi_num`	`u8`	The index of the SPI peripheral to configure (e.g., 0, 1, 2).
`len`	`u16`	The number of data frames to transfer. On newer hardware (v3), this sets the TSIZE register; on older hardware, it is ignored.
`rx_en`	`bool`	Set to true to enable DMA reception (RXDMAEN), false to disable it.

Ritorna:
void

Note:

- This function is not thread-safe and must be called from a single task or protected by a mutex if multiple tasks access the same SPI peripheral. It is not intended to be called from an ISR. The function performs low-level register manipulation to clear error flags like MODF (Mode Fault) and OVR (Overrun), which are critical because if these flags remain set, the SPI peripheral will refuse to act as a Master, causing the DMA transfer to hang indefinitely. A common beginner mistake is calling this function while the SPI peripheral is already enabled (SPE=1); always ensure the peripheral is in a safe state before calling this to avoid configuration errors.

Uso:

- This function is used as the first step in the SPI DMA transfer sequence. It is typically called after initializing the SPI peripheral via ca_spi_init_dma and before starting the actual DMA transfer using ca_dma_copy_target. The standard sequence is: 1) Call ca_spi_cs_assert to select the slave, 2) Call ca_spi_hw_dma_prepare to configure the hardware, 3) Start the DMA transfer, 4) Wait for completion, and 5) Call ca_spi_cs_deassert to release the slave.

Esempio:

```
/* Example: Preparing a SPI DMA read operation */
u8 spi_idx = 0;
u16 data_len = 32;
u8 rx_buffer[32];

/* 1. Select the SPI slave device */
ca_spi_cs_assert(spi_idx);

/* 2. Prepare hardware for DMA: enable RX, set length */
ca_spi_hw_dma_prepare(spi_idx, data_len, true);

/* 3. Start DMA transfer (assuming ca_spi_hw_dma_rx_addr returns the peripheral
    data register address) */
/* ca_dma_copy_target(rx_ch, rx_buffer, ca_spi_hw_dma_rx_addr(spi_idx), data_len
    , CA_DMA_DIR_P2M, CA_DMA_WIDTH_8); */

/* 4. Wait for transfer completion (e.g., using a semaphore or event) */
/* ca_os_wait(...); */

/* 5. Deselect the SPI slave */
ca_spi_cs_deassert(spi_idx);
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** What is the standard sequence for performing a DMA-based SPI transfer in caOS?
A: The standard sequence is: 1) Call ca_spi_cs_assert to select the slave, 2) Call ca_spi_hw_dma_prepare to configure the hardware, 3) Start the DMA transfer, 4) Wait for completion, and 5) Call ca_spi_cs_deassert to release the slave.

Code example conf:0.90 **Q:** Provide a minimal C example of how to prepare an SPI peripheral for a DMA transfer.
A: u8 spi_num = 1; u16 len = 64; bool rx_en = true; ca_spi_cs_assert(spi_num); ca_spi_hw_dma_prepare(spi_num, len, rx_en); /* Proceed to start DMA transfer */

Parameters conf:1.00 **Q:** What is the purpose of the 'len' parameter in ca_spi_hw_dma_prepare?
A: The 'len' parameter specifies the number of data frames to transfer. On newer hardware (v3), this value is used to set the TSIZE register; on older hardware, this parameter is ignored.

Error/Edge case conf:1.00 **Q:** Why might my SPI DMA transfer hang indefinitely?
A: The transfer may hang if SPI error flags like MODF (Mode Fault) or OVR (Overrun) remain set, as these prevent the SPI peripheral from acting as a Master. ca_spi_hw_dma_prepare is designed to clear these flags, but it must be called while the peripheral is in a safe state (SPE=0).

Behavior conf:1.00 **Q:** Is ca_spi_hw_dma_prepare thread-safe or ISR-safe?
A: No, this function is not thread-safe and must be protected by a mutex if accessed by multiple tasks. It is also not intended to be called from an ISR.

HW differences conf:0.95 **Q:** How does the SPI hardware version affect the behavior of ca_spi_hw_dma_prepa
A: The function behavior varies by hardware version: on v3 hardware, it configures the TSIZE register using the 'len' parameter and manages RXDMAEN, whereas on older

hardware, the 'len' parameter is ignored and the function focuses on clearing error flags and configuring DMA enable bits.

---

ca_spi_hw_dma_rx_addr

Header: caos/drivers/inc/ca_glue_hw.h:248
GUARD: EN_CA_GLUE_HW

```
extern volatile void *ca_spi_hw_dma_rx_addr(u8 spi_num); /* indirizzo
    registro dati RX per il DMA (DR/RXDR) */
```

Returns the memory-mapped address of the SPI peripheral's RX data register for DMA operations.

Parametri:

Nome	Tipo	Descrizione
spi_num	u8	The index of the SPI peripheral (e.g., 0 for SPI1, 1 for SPI2). Must be a valid index supported by the hardware configuration.

Ritorna:
A volatile pointer to the hardware RX data register (DR or RXDR). Returns NULL if the spi_num is invalid, though implementation-specific behavior may vary.

Note:

- This function is non-blocking and thread-safe as it simply performs a pointer calculation based on the peripheral base address. It is safe to call from an ISR because it does not perform any I/O or synchronization. The primary purpose is to provide the DMA controller with the source or destination address for hardware-accelerated data transfers. A common mistake is passing an uninitialized or out-of-bounds spi_num, which will result in a pointer to an invalid memory location, likely causing a HardFault when the DMA attempts to access it.

Uso:

- This function is used exclusively in the context of setting up DMA-based SPI transfers. It is typically called immediately before configuring the DMA channel to point to the SPI peripheral's data register. It is almost always paired with ca_dma_copy_target or similar DMA configuration functions to establish the link between the peripheral hardware and system memory. The sequence is: prepare the SPI peripheral for DMA, retrieve the register address, configure the DMA channel, and finally trigger the transfer.

Esempio:

```
/* Example: Setting up a DMA transfer for SPI1 */
u8 spi_num = 0;
u8 *rx_buffer = ca_memory_allocate(128);

/* Prepare SPI for DMA RX */
ca_spi_init_dma(spi_num);

/* Configure DMA to move data from SPI RX register to memory */
/* ca_spi_hw_dma_rx_addr returns the address of the hardware register */
if (ca_dma_copy_target(0, rx_buffer, ca_spi_hw_dma_rx_addr(spi_num), 128, 0, 0))
    {
    /* DMA transfer successfully configured */
} else {
    /* Handle configuration error */
```

```
14     ca_memory_free(rx_buffer);
15 }
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_spi_hw_dma_rx_addr and what is its primary purpose?
A: You should call this function when setting up DMA-based SPI transfers. Its primary purpose is to provide the DMA controller with the source or destination address of the SPI peripheral's RX data register, allowing the DMA to perform hardware-accelerated data transfers.

Code example conf:1.00 **Q:** Provide a minimal C example showing how to use ca_spi_hw_dma_rx_addr in caOS.
A: u8 spi_idx = 0; volatile void *rx_addr = ca_spi_hw_dma_rx_addr(spi_idx); if (rx_addr != NULL) { /* Proceed to configure DMA channel with rx_addr */ }

Parameters conf:1.00 **Q:** What does the spi_num parameter represent and what happens if I pass an invalid value?
A: The spi_num parameter is a u8 representing the index of the SPI peripheral (e.g., 0 for SPI1). Passing an uninitialized or out-of-bounds index will cause the function to return NULL, and if used, will likely result in a HardFault when the DMA attempts to access the invalid memory location.

Return conf:1.00 **Q:** What does the return value of ca_spi_hw_dma_rx_addr signify?
A: The function returns a volatile pointer to the hardware RX data register (DR or RXDR). A valid pointer indicates the memory-mapped address for DMA operations, while a NULL return indicates that the provided spi_num was invalid.

HW differences conf:0.90 **Q:** How does the SPI hardware implementation differ across MCU families regarding the RX data register?
A: The documentation indicates that the function abstracts access to different hardware registers depending on the architecture, specifically referencing DR (Data Register) for older families and RXDR (Receive Data Register) for newer families like the H7 series.

`ca_spi_hw_dma_tx_addr`

Header: `caos/drivers/inc/ca_glue_hw.h:249`
GUARD: `EN_CA_GLUE_HW`

```
1 extern volatile void *ca_spi_hw_dma_tx_addr(u8 spi_num); /* indirizzo
     registro dati TX per il DMA (DR/TXDR) */
```

Returns the memory-mapped address of the SPI peripheral's transmit data register for DMA operations.

Parametri:

Nome	Tipo	Descrizione
`spi_num`	`u8`	The index of the SPI peripheral (e.g., 0, 1, 2). Must be a valid index configured in the system.

Ritorna:
A volatile pointer to the hardware register (e.g., DR, TXDR) used by the DMA controller to push data into the SPI peripheral. Returns NULL if the spi_num is invalid or the peripheral is not initialized.

Note:

- This function is a low-level hardware abstraction layer (HAL) utility. It is non-blocking and thread-safe as it only performs a pointer calculation based on the peripheral base address. It is safe to call from an ISR because it does not perform any complex logic or synchronization. The most common mistake is passing an invalid spi_num, which will lead to a hard fault when the DMA controller attempts to access an invalid memory address. Always ensure the SPI peripheral is initialized before requesting its DMA address.

Uso:

- This function is used exclusively to configure DMA transfers for SPI communication. It is typically called immediately before invoking ca_dma_copy_target to set the destination address for a memory-to-peripheral (M2P) transfer. The standard sequence is: 1) Initialize SPI and DMA, 2) Call ca_spi_hw_dma_tx_addr to get the register address, 3) Pass that address to ca_dma_copy_target, and 4) Trigger the transfer using a hardware-specific kick function.

Esempio:

```
/* Example: Setting up a DMA transfer for SPI 0 */
u8 spi_num = 0;
u8 *data_buffer = /* pointer to data to send */;
size_t len = 16;

/* Get the hardware register address for the DMA controller */
volatile void *tx_reg = ca_spi_hw_dma_tx_addr(spi_num);

if (tx_reg != NULL) {
    /* Configure DMA to copy from memory to the SPI TX register */
    /* Assuming 'tx_ch' is a pre-allocated DMA channel handle */
    if (ca_dma_copy_target(tx_ch, (void *)data_buffer, (void *)tx_reg, len, 1, 0)) {
        /* Trigger the SPI hardware to start the DMA-driven transmission */
        /* ca_spi_hw_dma_kick(spi_num); // This is a hardware-specific trigger */
    }
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_spi_hw_dma_tx_addr in my application?
A: You should call this function exclusively when configuring DMA transfers for SPI communication. It is typically invoked immediately before ca_dma_copy_target to set the destination address for a memory-to-peripheral (M2P) transfer, following the initialization of both the SPI and DMA peripherals.

Code example conf:0.90 **Q:** Provide a minimal C example showing how to use ca_spi_hw_dma_tx_addr.
A: void setup_spi_dma(u8 spi_idx) { volatile void *tx_addr = ca_spi_hw_dma_tx_addr(spi_idx); if (tx_addr != NULL) { ca_dma_copy_target(tx_addr); } }

Parameters conf:1.00 **Q:** What does the u8 spi_num parameter represent and what happens if I pass an invalid value?
A: The spi_num parameter is the index of the SPI peripheral (e.g., 0, 1, 2). It must be a valid index configured in the system. Passing an invalid spi_num will cause the function to return NULL, and if the DMA controller subsequently attempts to access that invalid memory address, it will trigger a hard fault.

Return conf:1.00 **Q:** What does the return value of ca_spi_hw_dma_tx_addr indicate?
A: The function returns a volatile pointer to the hardware register (such as DR or TXDR) used by the DMA controller to push data into the SPI peripheral. It returns NULL if the provided spi_num is invalid or if the peripheral has not been initialized.

HW differences conf:0.80 **Q:** Are there specific hardware differences I should be aware of when using this function across different STM32 families?

A: The function abstracts the underlying hardware register differences (e.g., DR vs TXDR) across various families like f1xx, f4xx, f7xx, h5xx, and h7xx. While the implementation details vary by family, the function provides a consistent interface for the DMA controller to access the transmit data register regardless of the specific MCU architecture.

---

ca_spi_hw_init_periph

Header: caos/drivers/inc/ca_glue_hw.h:239
GUARD: EN_CA_GLUE_HW

```
extern bool ca_spi_hw_init_periph(u8 num, ca_spi_br_t br, u8 cpol, u8
    cpha); /* inizializza il periferico SPI in modalit  master */
```

Initializes the SPI peripheral hardware in Master mode.

Parametri:

Nome	Tipo	Descrizione
num	u8	The SPI peripheral index (e.g., 1, 2, 3). Must correspond to a valid hardware instance.
br	ca_spi_br_t	Baud rate prescaler value. Must be a valid enum or integer supported by the hardware.
cpol	u8	Clock Polarity: 0 for low level when idle, 1 for high level when idle.
cpha	u8	Clock Phase: 0 for first clock transition capture, 1 for second clock transition capture.

Ritorna:
Returns true if the peripheral was successfully initialized, false otherwise (e.g., invalid peripheral index).

Note:

- This function is non-blocking and performs low-level register configuration. It is NOT thread-safe and must not be called from an ISR. Before calling this, you must ensure the peripheral clock and GPIO pins are already configured using the appropriate hardware initialization functions. A common mistake is failing to clear the Mode Fault (MODF) flag on newer STM32 series (H7/H5/U5), which prevents the SPI from enabling; this function handles that sequence internally, so do not attempt to re-initialize registers manually after calling this.

Uso:

- This function is typically called during the system startup phase or when reconfiguring a communication bus. It is part of the initialization sequence: first, initialize the system clocks and GPIOs, then call this function to set up the SPI hardware, and finally proceed to use functions like ca_spi_xfer or ca_spi_dma_xfer for data transmission. It is paired with the driver-level configuration logic to ensure the SPI bus is ready for master-mode operations.

Esempio:

```
/* Assuming clock and GPIO for SPI1 are already initialized */
#define SPI_NUM 1

void setup_spi_bus(void) {
    /* Initialize SPI1 as Master, Baud rate 0, CPOL 0, CPHA 0 */
    bool success = ca_spi_hw_init_periph(SPI_NUM, 0, 0, 0);
```

```
7
8       if (success) {
9           /* SPI is ready for use */
10      } else {
11          /* Handle initialization failure */
12      }
13  }
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_spi_hw_init_periph() in my embedded application?
A: You should call ca_spi_hw_init_periph() once during system initialization to configure an SPI peripheral in master mode. It sets up the baud rate, clock polarity (CPOL), and clock phase (CPHA) for communication with SPI slaves. This function is typically called after initializing clocks and GPIOs using ca_clock_hw_spi_init() and ca_gpio_hw_spi_init().

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_spi_hw_init_periph() with caOS types?
A:

```
1  u8 spi_num = 1;
2  ca_spi_br_t baud_rate = 3;
3  u8 cpol = 0;
4  u8 cpha = 1;
5  bool success = ca_spi_hw_init_periph(spi_num, baud_rate, cpol, cpha);
6  /* success is always true per documentation, no error handling needed */
```

Parameters conf:0.95 **Q:** What do the parameters 'br', 'cpol', and 'cpha' control in ca_spi_hw_init_periph()?
A: The parameter 'br' (baud rate) controls the SPI clock speed, with values 0-7 where 0 is maximum speed and 7 is minimum. 'cpol' sets the clock polarity: 0 means idle low, 1 means idle high. 'cpha' sets the clock phase: 0 means data is sampled on the first edge, 1 means on the second edge.

Return conf:0.85 **Q:** What does ca_spi_hw_init_periph() return, and what causes it to fail?
A: The function returns true if initialization succeeds. According to documentation, it does not explicitly return false on failure. However, it will not return if an invalid SPI number is passed, causing undefined behavior due to incorrect memory access. The function is designed to not return on error, so failure is not handled explicitly.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_spi_hw_init_periph()?
A: Common mistakes include passing an invalid SPI number (0 or >3), which leads to incorrect memory access and undefined behavior. Another mistake is calling this function without first initializing clocks and GPIOs using ca_clock_hw_spi_init() and ca_gpio_hw_spi_init(). Additionally, calling this function from multiple threads or ISRs without proper synchronization can cause race conditions.

Behavior conf:0.90 **Q:** Is ca_spi_hw_init_periph() thread-safe and ISR-safe?
A: No, ca_spi_hw_init_periph() is not thread-safe. If multiple threads access the same SPI peripheral, manual synchronization is required. It is ISR-safe only if no other thread is using the same SPI. It is not a blocking function but directly modifies hardware registers without waiting or timeouts.

HW differences conf:0.95 **Q:** How does ca_spi_hw_init_periph() differ between STM32F1xx and STM32H7xx families?
A: On STM32F1xx, it directly configures CR1 with BR, CPOL, CPHA, MSTR, SSM, and SSI bits. On STM32H7xx, it follows a more complex MODF-clear sequence involving reading SR, writing IFCR.MODFC, configuring CFG1 and CFG2 registers, and setting CR2 to 0. The H7xx version also avoids setting CSTART in this function to prevent MODF during idle periods.

ca_spi_hw_init_periph_slave

Header: caos/drivers/inc/ca_glue_hw.h:240
GUARD: EN_CA_GLUE_HW

```
extern bool ca_spi_hw_init_periph_slave(u8 num, u8 cpol, u8 cpha, u16
    frame_size, ca_spi_slave_cb_t cb); /* inizializza il periferico SPI
    in modalit  slave con callback */
```

Initializes an SPI peripheral in slave mode with interrupt-driven data reception.

Parametri:

Nome	Tipo	Descrizione
num	u8	The SPI peripheral index (typically 1, 2, or 3 depending on the MCU).
cpol	u8	Clock polarity: 0 for low level when idle, 1 for high level when idle.
cpha	u8	Clock phase: 0 for first clock transition capture, 1 for second.
frame_size	u16	Expected packet size in bytes. Values > 0 and <= SPI_DEF_PKT_SIZE are accepted.
cb	ca_spi_slave_cb_t	Callback function pointer triggered when data is received. Pass NULL to disable interrupts.

Ritorna:
Returns true if the peripheral was successfully initialized, false if the index is invalid or hardware configuration failed.

Note:

- This function is non-blocking and configures the hardware registers to act as a passive slave. It is not thread-safe; ensure it is called during system initialization before any SPI communication occurs. It is not intended to be called from an ISR. A common mistake is passing an invalid SPI index or failing to provide a valid callback, which leaves the peripheral in a state where it cannot signal the application about incoming data. On some older hardware (F4xx), the function performs a specific sequence of reading registers to clear stale flags; failing to initialize correctly can lead to interrupt loops that starve other system tasks like USB.

Uso:

- This function is used when the device acts as a peripheral (slave) to an external master (e.g., a sensor or another MCU). It is typically called during the system boot sequence or driver initialization phase. It is often paired with GPIO configuration functions to set up the pins (MOSI, MISO, SCK, NSS) and is the prerequisite for the hardware to start responding to the master's clock signals.

Esempio:

```
/* Initialize SPI1 as slave, CPOL=0, CPHA=0, 32-byte frames, with a callback */
#include "caos.h"

void my_spi_callback(u8 *data, u16 len) {
    /* Handle received data here */
}

void init_system(void) {
    /* Configure GPIO pins for SPI1 before initializing the peripheral */
    ca_gpio_init_port(1);
```

```
11
12      /* Initialize SPI1 in slave mode */
13      bool success = ca_spi_hw_init_periph_slave(1, 0, 0, 32, my_spi_callback);
14
15      if (!success) {
16          /* Handle initialization error */
17      }
18  }
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_spi_hw_init_periph_slave() in my embedded project?
A: You should call ca_spi_hw_init_periph_slave() when you need to configure an SPI peripheral in slave mode to receive data from a master device. This function is used to initialize the SPI hardware for slave operation with an optional callback function that will be invoked upon data reception.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_spi_hw_init_periph_slave() with caOS types?
A:

```
u8 spi_num = 1;
u8 cpol = 0;
u8 cpha = 0;
u16 frame_size = 4;
ca_spi_slave_cb_t callback = NULL;
bool result = ca_spi_hw_init_periph_slave(spi_num, cpol, cpha, frame_size,
    callback);
```

Parameters conf:0.95 **Q:** What do the cpol and cpha parameters control in ca_spi_hw_init_periph_slave()?
A: The cpol parameter sets the clock polarity: 0 means the clock is low when idle, and 1 means it's high. The cpha parameter sets the clock phase: 0 means data is sampled on the first clock edge, and 1 means it's sampled on the second clock edge.

Return conf:0.90 **Q:** What does it mean if ca_spi_hw_init_periph_slave() returns false?
A: A return value of false indicates that the initialization failed, typically due to invalid parameters such as an invalid SPI number (not 1, 2, or 3), or an unsupported SPI peripheral on the current MCU (e.g., SPI2 on F103).

Error/Edge case conf:0.85 **Q:** What are common mistakes when calling ca_spi_hw_init_periph_slave()?
A: Common mistakes include passing an invalid SPI number (not 1, 2, or 3), using SPI2 on an F103 chip which doesn't support it, providing a frame_size outside the valid range (0 < frame_size <= 255), or not setting up the callback function properly if data reception is expected.

Behavior conf:0.90 **Q:** Is ca_spi_hw_init_periph_slave() thread-safe and can it be called from an ISR?
A: No, ca_spi_hw_init_periph_slave() is not thread-safe and should not be called from an ISR. It modifies peripheral registers and is not designed to be reentrant. If used from multiple threads, it must be protected with a mutex.

HW differences conf:0.80 **Q:** How does ca_spi_hw_init_periph_slave() differ between F1xx and F7xx MCUs in terms of hardware configuration?
A: On F1xx MCUs, SPI2 is not available on the F103 chip, and the configuration uses CR1 and CR2 registers with basic SPI control bits. On F7xx MCUs, the function configures CFG2 register with FRXTH bit for FIFO threshold control and uses a different register layout for setting up the slave mode with more detailed control over data size and FIFO behavior.

---

ca_spi_hw_recv_handler

Header: caos/drivers/inc/ca_glue_hw.h:243
GUARD: EN_CA_GLUE_HW

```
extern void ca_spi_hw_recv_handler(u8 num); /* gestore IRQ SPI slave:
   legge il byte ricevuto e chiama la callback */
```

Low-level SPI interrupt handler for slave mode to process incoming bytes and trigger callbacks.

Parametri:

Nome	Tipo	Descrizione
num	u8	The SPI peripheral index (e.g., 1 for SPI1, 2 for SPI2). Must be within the valid range defined by the hardware abstraction layer.

Ritorna:
void

Note:

- This function is designed to be called exclusively from the hardware SPI interrupt service routine (ISR). It is not thread-safe and must never be called from a standard task context. Because it runs in an ISR, it must be extremely fast; avoid heavy processing inside the callback triggered by this function. A common mistake is failing to ensure the SPI peripheral is properly initialized via ca_spi_init before the interrupt is enabled, which will lead to accessing null or unconfigured hardware registers.

Uso:

- This function is the backbone of slave-mode SPI communication in caOS. It is typically invoked inside the system's SPIx_IRQHandler. It automatically manages the internal receive buffer and, once a full frame (as defined by the system configuration) is received, it executes a registered callback function to notify the application layer that data is ready for processing.

Esempio:

```
/* Example: SPI1 interrupt handler implementation */
void SPI1_IRQHandler(void)
{
    /* Call the low-level handler to process the incoming byte */
    ca_spi_hw_recv_handler(1);

    /* Note: If an error occurred, you might need to clear pending flags here */
    /* but the handler already manages RXNE and OVR flags internally. */
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_spi_hw_recv_handler in my SPI slave application?
A: You should call ca_spi_hw_recv_handler from within an SPI interrupt service routine (ISR) when operating in slave mode. It is designed to handle incoming SPI data by reading bytes from the SPI data register and invoking a registered callback when a complete packet is received.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_spi_hw_recv_handle with caOS types?
A:

```
u8 spi_num = 1;
ca_spi_hw_recv_handler(spi_num);
```

Parameters conf:0.95 **Q:** What does the 'num' parameter in ca_spi_hw_recv_handler represent and what are valid values?
A: The 'num' parameter specifies the SPI peripheral number. Valid values are 1 or 2. If a value outside this range is passed, the function returns immediately without performing any action.

Return conf:0.90 **Q:** Does ca_spi_hw_recv_handler return any value and what does it signify?
A: The function does not return any value. It is void. It cannot fail in a strict sense, but it may silently ignore invalid inputs or not complete operations if parameters are incorrect.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_spi_hw_recv_handler?
A: Common mistakes include calling this function from a non-ISR context, which can lead to unpredictable behavior. Also, passing an invalid 'num' parameter (not 1 or 2) will cause the function to return early without processing. Additionally, since it's not thread-safe, concurrent access without synchronization mechanisms can cause data corruption.

Behavior conf:0.90 **Q:** Is ca_spi_hw_recv_handler blocking or non-blocking, and is it safe to use in ISR contexts?
A: The function is non-blocking, only taking time to read a byte and update counters. It is designed to be called exclusively from an ISR and is not thread-safe. It modifies global variables and should not be used in non-ISR contexts due to potential side effects.

HW differences conf:0.85 **Q:** How does ca_spi_hw_recv_handler differ between STM32F1xx and STM32H7xx families in terms of hardware implementation?
A: In F1xx and F4xx families, the function reads one byte at a time from the SPI data register (DR) and checks for overrun errors. In H5xx and H7xx families, it processes all available bytes in the receive FIFO (RXDR) in a loop until no more data is present, using different register names and access methods.

`ca_spi_hw_slave_sync`

Header: `caos/drivers/inc/ca_glue_hw.h:261`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_spi_hw_slave_sync(u8 num)
```

Resets and re-aligns the SPI slave hardware to recover from framing errors or overrun conditions.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The SPI peripheral index (typically 1 to 3). Values outside this range are ignored.

Ritorna:
void

Note:

- This function is non-blocking and performs a low-level hardware reset of the SPI peripheral by toggling the SPE (SPI Enable) bit. It is designed to be called when the slave detects a desynchronization, such as missing a clock edge or encountering an overrun error that halts communication. It is not thread-safe and should be used with caution; it is typically called from an interrupt context (like an EXTI pin handler) or a high-priority task when the SPI state machine is known to be stalled. A common beginner mistake is calling this

during an active transfer, which will immediately abort the transaction and potentially corrupt the data stream.

Uso:

- This function is used in slave-mode SPI configurations to ensure the peripheral is ready for the next frame. It is typically paired with ca_spi_init during system startup and is invoked inside an interrupt service routine triggered by the Chip Select (CS) pin falling edge. By calling this, the driver ensures that any 'phantom' clock pulses or previous errors are cleared, effectively 're-arming' the slave to start fresh on the next incoming byte.

Esempio:

```
/* Example: Synchronizing SPI slave on CS pin falling edge interrupt */
void EXTI_CS_Handler(u8 spi_num) {
    /* Reset the SPI hardware to ensure we are aligned for the new frame */
    ca_spi_hw_slave_sync(spi_num);

    /* Now the SPI is ready to receive data from the master */
    /* Proceed to signal the main task that the SPI is ready */
    ca_os_signal_handle(spi_task_handle);
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_spi_hw_slave_sync in my caOS application?
A: You should call this function in slave-mode SPI configurations when the peripheral detects a desynchronization, such as a framing error, overrun condition, or when the SPI state machine is stalled. It is typically invoked inside an interrupt service routine triggered by the falling edge of the Chip Select (CS) pin to ensure the peripheral is re-armed and ready for the next frame.

Code example conf:1.00 **Q:** Provide a minimal example of how to invoke the SPI slave sync function.
A: Assuming you are using SPI peripheral 1, the call is: ca_spi_hw_slave_sync(1U);. This should be placed within an appropriate context like an EXTI handler for the CS pin.

Parameters conf:1.00 **Q:** What are the valid values for the 'num' parameter in ca_spi_hw_slave_sync?
A: The 'num' parameter represents the SPI peripheral index. For F1xx devices, valid values are 1 to 2. For F4xx, F7xx, H5xx, H7xx, and U5xx devices, valid values are 1 to 3. Values outside these ranges are ignored by the function.

Error/Edge case conf:1.00 **Q:** What is a common mistake when using ca_spi_hw_slave_sync?
A: A common beginner mistake is calling this function during an active SPI transfer. Doing so will immediately abort the transaction and potentially corrupt the data stream.

Behavior conf:1.00 **Q:** Is ca_spi_hw_slave_sync thread-safe and blocking?
A: The function is non-blocking and is not thread-safe. It is designed to be called from an interrupt context or a high-priority task when the SPI state machine is known to be stalled.

HW differences conf:1.00 **Q:** How does the implementation of ca_spi_hw_slave_sync differ between F4xx and H7xx families?
A: On F4xx devices, the function clears errors by reading the DR and SR registers and toggling the SPE bit. On H7xx devices, the function additionally writes to the IFCR register (0xFF8U) to clear specific flags like EOT, TXTF, UDR, OVR, CRCE, TIFRE, MODF, and SUSP, and it uses the TXDR register for refilling the MISO answer instead of the DR register.

---

ca_spi_hw_slave_sync_pending

Header: caos/drivers/inc/ca_glue_hw.h:262
GUARD: EN_CA_GLUE_HW

```
extern u8 ca_spi_hw_slave_sync_pending(u8 num); /* 1 if a byte was
    already waiting at the last sync */ /* slave: frame start (CS) - drop
     partial frame, clear overrun */
```

Checks if a byte was already waiting in the SPI slave hardware buffer at the start of the last synchronization.

Parametri:

Nome	Tipo	Descrizione
num	u8	The SPI peripheral index (typically 1 to 3 depending on the STM32 series).

Ritorna:
Returns 1 if a byte was pending (indicating a potential overrun or late synchronization), and 0 otherwise.

Note:

- This function is non-blocking and thread-safe as it simply reads a status flag maintained by the driver. It is intended to be used in the context of SPI slave communication where timing is critical. The most common beginner mistake is ignoring the return value; if this returns 1, it means the master sent data before the slave was ready, which usually implies the slave's processing loop is too slow or the SPI interrupt priority is too low. It does not perform any hardware interaction itself, so it can be safely called from both tasks and ISRs.

Uso:

- This function is typically used immediately after calling ca_spi_slave_sync() to verify the integrity of the incoming data frame. In a high-speed slave implementation, you pair it with ca_spi_slave_sync() to detect if the master started transmitting before the slave had finished clearing the previous frame. If it returns 1, the system should increment an error counter or trigger a re-synchronization logic to handle the data overrun.

Esempio:

```
/* Example: Checking for late SPI slave synchronization */
#define MY_SPI_INDEX 1

void process_spi_frame(void) {
    /* Synchronize the slave hardware to the incoming frame */
    ca_spi_slave_sync(MY_SPI_INDEX);

    /* Check if the master was already sending data before we were ready */
    if (ca_spi_hw_slave_sync_pending(MY_SPI_INDEX)) {
        /* Log the error: the master is faster than our processing loop */
        ca_printf("SPI Slave overrun detected on bus %d!\n", MY_SPI_INDEX);
    } else {
        /* Proceed with normal data processing */
        /* ca_spi_read_buf(...) would be called here */
    }
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_spi_hw_slave_sync_pending in my SPI slave application?
A: You should call it immediately after calling ca_spi_slave_sync(). It is used to verify the integrity of the incoming data frame by checking if the master started transmitting before the slave was ready, which helps detect potential overruns or late synchronization.

Code example conf:1.00 **Q:** Provide a minimal example of how to check for a pending SPI slave sync in caOS.
A: u8 spi_num = 1; ca_spi_slave_sync(spi_num); if (ca_spi_hw_slave_sync_pending(spi_num)) { /* Handle overrun or re-synchronize */ }

Parameters conf:1.00 **Q:** What does the 'num' parameter represent in ca_spi_hw_slave_sync_pending and what are the valid values?
A: The 'num' parameter is a u8 representing the SPI peripheral index. Valid values depend on the STM32 series: for f1xx, it is 1 to 2; for f4xx, f7xx, h5xx, h7xx, and u5xx, it is 1 to 3.

Return conf:1.00 **Q:** What do the return values 0 and 1 signify for this function?
A: A return value of 1 indicates that a byte was already waiting in the SPI slave hardware buffer at the start of the last synchronization, suggesting a data overrun or late synchronization. A return value of 0 indicates no byte was pending.

HW differences conf:1.00 **Q:** Are there differences in how ca_spi_hw_slave_sync_pending behaves across different STM32 families?
A: The function logic is consistent across all supported families, but the valid range for the 'num' parameter differs. The f1xx family supports SPI peripherals 1 through 2, while the f4xx, f7xx, h5xx, h7xx, and u5xx families support SPI peripherals 1 through 3.

`ca_spi_hw_slave_tx_fill`

Header: `caos/drivers/inc/ca_glue_hw.h:260`
GUARD: `EN_CA_GLUE_HW`

```
1 extern void ca_spi_hw_slave_tx_fill(u8 num, u8 fill); /* slave: byte
     answered on MISO for every byte received, 0 = none */
```

Preloads the SPI slave transmit FIFO with a specific byte to be sent back to the master upon reception.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The SPI peripheral index (typically 1 to 3 depending on the MCU model). Values outside this range are ignored.
`fill`	`u8`	The byte to transmit on MISO. If 0, no data is preloaded, effectively disabling the auto-response behavior.

Ritorna:
void

Note:

- This function is non-blocking and performs direct register access to the SPI peripheral. It is not thread-safe; if multiple tasks access the same SPI peripheral, you must ensure synchronization using other caOS primitives. It is safe to call from an ISR, but it is primarily intended for initialization or configuration phases. A common mistake is calling this function while the SPI peripheral is actively transferring data, which may lead to

unexpected data being sent or FIFO corruption. Always ensure the SPI peripheral is properly initialized via ca_spi_init before calling this function.

Uso:

- This function is used when the device acts as an SPI slave and needs to provide a constant 'dummy' or 'status' byte back to the master for every byte the master sends. It is typically called during the SPI configuration sequence, immediately after ca_spi_init, to ensure the hardware is ready to respond to the very first clock pulse from the master. It is often paired with an interrupt-driven mechanism where the IRQ handler continues to top up the FIFO after the initial byte is consumed.

Esempio:

```
/* Initialize SPI peripheral 1 as a slave */
ca_spi_init(1, /* mode: slave */ 0, /* frame_size: 8 */ 8);

/* Preload the slave response with 0xAA (e.g., a status byte) */
/* This ensures the master receives 0xAA if it clocks the bus immediately */
ca_spi_hw_slave_tx_fill(1, 0xAA);

/* If we need to stop the auto-response later, we pass 0 */
ca_spi_hw_slave_tx_fill(1, 0);
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_spi_hw_slave_tx_fill in my caOS application?
A: You should call it during the SPI configuration sequence, immediately after ca_spi_init. It is used when the device acts as an SPI slave and needs to provide a constant dummy or status byte back to the master for every byte received. It is typically paired with an interrupt-driven mechanism to top up the FIFO after the initial byte is consumed.

Code example conf:1.00 **Q:** How do I initialize SPI1 to respond with a 0xFF byte as a slave?
A: After initializing the peripheral, call the function as follows: ca_spi_init(1); ca_spi_hw_slave_tx_fi 0xFF); Ensure that you do not call this while the SPI peripheral is actively transferring data to avoid FIFO corruption.

Parameters conf:1.00 **Q:** What are the valid ranges for the 'num' and 'fill' parameters?
A: The 'num' parameter represents the SPI peripheral index. For F1xx models, valid values are 1 to 2; for F4xx, F7xx, H5xx, H7xx, and U5xx models, valid values are 1 to 3. Values outside these ranges are ignored. The 'fill' parameter is the byte to transmit on MISO; if set to 0, the auto-response behavior is effectively disabled.

HW differences conf:1.00 **Q:** Are there differences in how ca_spi_hw_slave_tx_fill interacts with hardware registers across STM32 families?
A: Yes. Older families like F1xx, F4xx, and F7xx write to the SPI_DR register and check the SPI_SR_TXE flag. Newer families like H5xx, H7xx, and U5xx write to the SPI_TXDR register and check the SPI_SR_TXP flag, often using a while loop to fill the FIFO.

`ca_spi_hw_stop`

Header: `caos/drivers/inc/ca_glue_hw.h:246`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_spi_hw_stop(u8 num); /* disables the SPI (SPE=0) and its
   interrupt, draining any transfer in flight */
```

Safely disables an SPI peripheral by stopping transfers, clearing interrupts, and draining the FIFO.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The SPI peripheral index (e.g., 1, 2, 3). Must correspond to a valid, initialized SPI hardware instance.

Ritorna:
Returns true if the peripheral was successfully stopped within the timeout period, false if the operation timed out or the peripheral index is invalid.

Note:

- This function is not thread-safe and should not be called from an ISR. It performs a 'graceful' shutdown by suspending active transfers and draining the hardware FIFO to prevent stale data from triggering interrupts after the peripheral is disabled. A common mistake is calling this while a DMA transfer is still active without waiting for the DMA to finish, which can lead to unpredictable hardware states. Always ensure the SPI is not in use by other tasks before calling this, as it forces the peripheral into a disabled state (SPE=0).

Uso:

- Use this function when you need to reconfigure the SPI bus, enter a low-power mode, or shut down a communication channel. It is typically used in the teardown sequence of a driver: first, stop the peripheral with this function, then update any internal state tracking (like a global 'active' bitmask), and finally perform any necessary GPIO re-configuration. It is the mandatory counterpart to the initialization sequence.

Esempio:

```c
/* Example: Safely shutting down SPI 1 */
#include "caos.h"

void shutdown_spi_bus(void) {
    /* 1. Ensure no chip select is active before stopping the hardware */
    ca_spi_cs_deassert(1);

    /* 2. Stop the SPI hardware and drain buffers */
    if (!ca_spi_hw_stop(1)) {
        /* Handle error: hardware failed to stop, possibly stuck bus */
        ca_printf("Error: SPI 1 failed to stop cleanly!\n");
    } else {
        ca_printf("SPI 1 stopped successfully.\n");
    }

    /* 3. Perform additional cleanup, e.g., power down or re-init */
    /* ca_gpio_configure(...) to set pins to analog/input to save power */
}
```

Q&A — fine-tuning review *7/7 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_spi_hw_stop in my application?
A: You should call it when you need to reconfigure the SPI bus, enter a low-power mode, or shut down a communication channel. It is the mandatory teardown counterpart to the initialization sequence.

Code example conf:1.00 **Q:** Provide a minimal example of how to safely shut down SPI peripheral 2.
A: u8 spi_idx = 2U; if (ca_spi_hw_stop(spi_idx)) { /* Success: proceed to update state or reconfigure GPIO */ } else { /* Failure: handle timeout or invalid index */ }

Parameters conf:1.00 **Q:** What does the 'num' parameter represent and what are its constraints?

A: The 'num' parameter is a u8 representing the SPI peripheral index (e.g., 1, 2, 3). It must correspond to a valid, initialized SPI hardware instance.

Return conf:1.00 **Q:** What do the boolean return values of ca_spi_hw_stop indicate?

A: It returns true if the peripheral was successfully stopped within the timeout period. It returns false if the operation timed out or if the provided peripheral index is invalid.

Error/Edge case conf:1.00 **Q:** What is a common mistake when calling ca_spi_hw_stop?

A: A common mistake is calling this function while a DMA transfer is still active without waiting for the DMA to finish, which can lead to unpredictable hardware states.

Behavior conf:1.00 **Q:** Is ca_spi_hw_stop thread-safe or safe to call from an ISR?

A: No, this function is not thread-safe and should not be called from an ISR. It is a blocking operation that performs a graceful shutdown by suspending transfers and draining the hardware FIFO.

HW differences conf:1.00 **Q:** How does the shutdown procedure differ between F1xx/F4xx/F7xx and H5xx/H7xx/U5xx families?

A: F1xx/F4xx/F7xx families wait for TXE and BSY flags to clear before disabling SPE. H5xx/H7xx/U5xx families follow a specific sequence: they use CSUSP to suspend transfers, wait for CSTART to clear, and explicitly drain the RxFIFO (up to 16 bytes) before clearing SPE and resetting flags via the IFCR register.

`ca_spi_hw_xfer`

Header: `caos/drivers/inc/ca_glue_hw.h:263`
GUARD: `EN_CA_GLUE_HW`

```
1 extern bool ca_spi_hw_xfer(u8 num, u8 data, u8 *rx); /* full-duplex byte
    ; false if not done within CA_SPI_XFER_TIMEOUT_MS */
```

Performs a full-duplex SPI transfer of a single byte with a built-in timeout mechanism.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The index of the SPI peripheral to use. Must correspond to a valid, initialized SPI hardware instance.
`data`	`u8`	The byte to transmit over the MOSI line.
`rx`	`u8 *`	Pointer to a memory location where the received byte from the MISO line will be stored. Must be a valid, non-NULL pointer.

Ritorna:
Returns true if the transfer completed successfully within the timeout period. Returns false if the operation timed out, if a hardware error (like MODF) occurred, or if the SPI peripheral was disabled.

Note:

- This function is blocking, meaning the calling task will wait in a loop until the hardware completes the transfer or the timeout expires. It is NOT safe to call from an ISR because it relies on system ticks and blocking loops. Before calling this, ensure the SPI peripheral has been properly initialized via the hardware abstraction layer. A common beginner mistake is to call this function without first asserting the Chip Select (CS) pin, which will result in the peripheral waiting indefinitely or receiving garbage data.

Uso:

- This function is the low-level building block for all SPI communication in caOS. It is typically used by higher-level drivers to implement byte-by-byte transmission. In a typical sequence, you would first call a function to assert the CS pin, then call ca_spi_hw_xfer one or more times to exchange data, and finally call a function to deassert the CS pin. It is often paired with ca_os_delay_ms if you need to implement a specific protocol timing requirement between bytes.

Esempio:

```c
/* Example: Sending a command byte and reading a response */
#include "caos.h"

void read_sensor_data(u8 spi_num, u8 reg_addr) {
    u8 rx_byte = 0;

    /* 1. Assert Chip Select (CS) - using a hypothetical GPIO function */
    /* ca_gpio_write(SENSOR_CS_PIN, 0); */

    /* 2. Send register address and receive data */
    if (ca_spi_hw_xfer(spi_num, reg_addr, &rx_byte)) {
        ca_printf("Received: 0x%02X\n", rx_byte);
    } else {
        ca_printf("SPI Transfer failed or timed out!\n");
    }

    /* 3. Deassert Chip Select (CS) */
    /* ca_gpio_write(SENSOR_CS_PIN, 1); */
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_spi_hw_xfer() in my embedded application?
A: You should use ca_spi_hw_xfer() when you need to perform a full-duplex SPI transaction, sending one byte and receiving another. It's suitable for communicating with SPI slave devices like sensors, memory chips, or display controllers where you need to send a command and read a response in a single operation.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_spi_hw_xfer()?
A: u8 spi_data = ca_spi_hw_xfer(0, 0x55); // Send 0x55 on SPI1 and receive byte u8 received = ca_spi_hw_xfer(0, 0xAA); // Send 0xAA and receive another byte

Parameters conf:0.90 **Q:** What does the 'num' parameter in ca_spi_hw_xfer represent and what are valid values?
A: The 'num' parameter identifies the SPI peripheral to use (0 for SPI1, 1 for SPI2, etc.). Valid values depend on the hardware configuration but must match the number of SPI peripherals available on the microcontroller. Invalid values can cause memory access violations or crashes.

Return conf:0.90 **Q:** What does ca_spi_hw_xfer() return and what does it mean if it returns 0?
A: The function returns the byte received from the SPI slave device. If it returns 0, it typically indicates an error such as MODF (mode fault) or that the SPI peripheral is disabled, and the function attempts to recover by re-enabling master mode.

Error/Edge case conf:0.85 **Q:** What are common error conditions or mistakes when using ca_spi_hw_xfer()?
A: Common issues include calling it from an ISR (which is not allowed), using invalid SPI peripheral numbers, or calling it from multiple threads simultaneously (not thread-safe). Also, not checking for errors in the return value may lead to processing invalid data if MODF occurs.

Behavior conf:0.95 **Q:** Is ca_spi_hw_xfer() blocking or non-blocking, and can it be called from an ISR?

A: It is completely blocking and must not be called from an ISR. It waits for transmission to complete before returning. It is also not thread-safe, so concurrent access from multiple threads will cause undefined behavior.

HW differences conf:0.85 **Q:** How does ca_spi_hw_xfer() implementation differ between STM32F7xx and STM32H7xx families?

A: The STM32F7xx uses standard SPI registers like DR, SR, and CR1 for data transfer and status checks. In contrast, the STM32H7xx uses more advanced registers like TXDR, RXDR, and requires setting CSTART in CR1 for proper operation, along with handling MODF and SPE flags differently for recovery.

`ca_spi_init`

Header: `caos/drivers/inc/spi.h:72`
Source: `caos/drivers/src/spi.c:38:66`
GUARD: `EN_SPI`

```
bool ca_spi_init(const ca_spi_config_t *cfg)
```

Initializes the SPI hardware peripheral and configures its GPIO pins and operating mode.

Parametri:

Nome	Tipo	Descrizione
`cfg`	`const ca_spi_config_t *`	Pointer to a configuration structure containing SPI number (1-3), master/slave mode, clock polarity/phase, and GPIO settings. Must not be NULL.

Ritorna:
Returns true if the initialization was successful, false if the configuration pointer is invalid, the SPI number is out of range, or the hardware peripheral failed to initialize.

Note:

- This function is not thread-safe and should be called during the system initialization phase before any SPI communication tasks are started. It is not intended to be called from an ISR. The function handles clock enabling, GPIO pin multiplexing, and peripheral register setup. A common mistake is passing a NULL pointer or an uninitialized configuration structure, which will cause the function to return false immediately. If 'manage_cs' is set to true, the function also configures the Chip Select pin as an output and sets it to a high idle state.

Uso:

- This function is the mandatory first step for any SPI communication. It is typically called once during system startup or task initialization. After calling ca_spi_init, you can proceed to use other SPI functions like ca_spi_xfer or ca_spi_dma_write. If you are using the CLI, this function also registers the SPI interface for debugging purposes if the feature is enabled.

Esempio:

```
ca_spi_config_t spi_cfg = {0};
spi_cfg.spi_num = 1;
spi_cfg.master = true;
spi_cfg.cpol = 0;
spi_cfg.cpha = 0;
spi_cfg.manage_cs = true;
/* Configure GPIO ports for SPI1 */
```

```
spi_cfg.cs_port = 0; /* Assuming 0 represents a valid port index */
spi_cfg.cs_pin = 5;

if (!ca_spi_init(&spi_cfg)) {
    /* Handle initialization failure */
    ca_printf("SPI1 init failed!\n");
} else {
    /* Proceed to use SPI1 for data transfer */
    ca_printf("SPI1 ready.\n");
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_spi_init() in a caOS application?
A: You should call ca_spi_init() during the system initialization phase, typically before any SPI communication is required. It is meant to set up the SPI peripheral as either a master or slave, configure the clock, mode, and chip select settings. It is not intended to be called during runtime or in response to events.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_spi_init() with caOS types?
A: ca_spi_config_t config; config.spi_num = 0; config.master = true; config.br = 1000000; config.cpol = false; config.cpha = false; bool result = ca_spi_init(&config);

Parameters conf:0.90 **Q:** What does the spi_num parameter in ca_spi_init() represent and what are valid values?
A: The spi_num parameter specifies which SPI peripheral to initialize. Valid values are 0, 1, or 2, corresponding to SPI1, SPI2, and SPI3 on STM32 microcontrollers. Using any other value will result in an invalid configuration.

Return conf:0.95 **Q:** What does ca_spi_init() return when it fails to initialize an SPI peripheral?
A: The function returns false when initialization fails. This can happen due to invalid configuration parameters, unsupported settings, or hardware initialization errors. A return value of true indicates successful initialization.

Error/Edge case conf:0.85 **Q:** What are common mistakes when calling ca_spi_init()?
A: Common mistakes include passing a NULL pointer to cfg, using an invalid spi_num (not 0, 1, or 2), providing unsupported baudrate or clock polarity/phase settings, or failing to initialize required fields like br, cpol, and cpha when in master mode.

Behavior conf:0.90 **Q:** Is ca_spi_init() safe to call from an ISR or in a multi-threaded environment?
A: No, ca_spi_init() is not safe to call from an ISR and is not thread-safe. It is a blocking function and should only be used during system initialization. If used in a multi-threaded context, it must be protected with a mutex.

`ca_spi_init_dma`

Header: `caos/drivers/inc/spi.h:101`
Source: `caos/drivers/src/spi.c:222:258`
GUARD: `EN_SPI`

```
bool ca_spi_init_dma(const ca_spi_config_t *cfg)
```

Initializes a SPI peripheral for DMA-based data transfers.

Parametri:

Nome	Tipo	Descrizione
`cfg`	`const ca_spi_config_t *`	Pointer to a configuration structure containing SPI parameters (port number, master/slave mode, etc.). Must not be NULL and must specify master mode.

Ritorna:
Returns true if the SPI peripheral was successfully initialized and DMA channels were bound and configured. Returns false if the configuration is invalid, the SPI port is already in use, no free DMA slots are available, or hardware binding fails.

Note:

- This function is not thread-safe and should be called during system initialization before starting tasks. It is not callable from an ISR because it performs memory and resource allocation. It requires that the underlying SPI hardware is already clocked and that the DMA controller is available. A common beginner mistake is attempting to call this function multiple times for the same SPI port without checking if it is already initialized, or failing to verify the return value, which leads to silent failures in communication.

Uso:

- This function is used to prepare a SPI peripheral for high-performance, non-blocking data transfers. It is typically called once during the system startup phase, immediately after defining the configuration structure. It must be paired with subsequent calls to ca_spi_dma_xfer or ca_spi_dma_write/read to actually perform data operations. It is essential to ensure that the SPI port is not already registered in the system's internal DMA table before calling this function.

Esempio:

```
ca_spi_config_t spi_cfg = { .spi_num = 1, .master = true, .cpol = 0, .cpha = 0
    };

// Initialize SPI with DMA support
if (!ca_spi_init_dma(&spi_cfg)) {
    // Handle initialization failure
    ca_printf("SPI DMA init failed!\n");
} else {
    // SPI is ready for DMA transfers
    // Perform a transfer using ca_spi_dma_xfer(1, tx_buf, rx_buf, len);
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_spi_init_dma in my caOS application?
A: You should call it once during the system initialization phase, before starting any tasks, to prepare the SPI peripheral for high-performance, non-blocking data transfers.

Code example conf:0.90 **Q:** How do I initialize a SPI port using ca_spi_init_dma?
A: ca_spi_config_t cfg = { .port = 1, .mode = MASTER_MODE }; if (ca_spi_init_dma(&cfg)) { /* Success */ }

Parameters conf:1.00 **Q:** What are the requirements for the ca_spi_config_t parameter?
A: The pointer must not be NULL, and the configuration structure must specify master mode for the SPI peripheral.

Return conf:1.00 **Q:** What does a return value of false indicate when calling ca_spi_init_dma?
A: It indicates failure due to an invalid configuration, the SPI port being already in use, no free DMA slots available, or a failure in hardware binding.

Error/Edge case conf:1.00 **Q:** What is a common mistake developers make when using ca_spi_init_dma?

A: A common mistake is attempting to call the function multiple times for the same SPI port without checking if it is already initialized, or failing to verify the return value, which leads to silent communication failures.

Behavior conf:1.00 **Q:** Is ca_spi_init_dma safe to call from an ISR or in a multi-threaded environment?

A: No, it is not thread-safe and must not be called from an ISR because it performs memory and resource allocation. It must be called during system initialization.

`ca_spi_read_buf`

Header: `caos/drivers/inc/spi.h:83`
Source: `caos/drivers/src/spi.c:79:83`
GUARD: `EN_SPI`

```
void ca_spi_read_buf(u8 spi_num, u8 *buf, u16 len)
```

Reads a specified number of bytes from an SPI peripheral by performing a dummy write for each byte.

Parametri:

Nome	Tipo	Descrizione
`spi_num`	`u8`	The index of the SPI peripheral to read from. Must correspond to a previously initialized SPI port.
`buf`	`u8 *`	Pointer to the buffer where the received data will be stored. Must have enough allocated space for 'len' bytes.
`len`	`u16`	Number of bytes to read. A value of 0 results in no operation.

Ritorna:
void. This function does not return status codes; it assumes the hardware transfer succeeds.

Note:

- This function is blocking, meaning the CPU will wait for the SPI hardware to complete the transfer of every single byte before returning. It is not thread-safe by itself; if multiple tasks access the same SPI peripheral, you must protect this call with a mutex. It is not safe to call from an ISR because the blocking nature will stall the system interrupt latency. A common beginner mistake is providing a buffer pointer that is too small for the requested length, which will cause a memory corruption (buffer overflow).

Uso:

- This function is typically used in a master-slave communication sequence where the master needs to pull data from a slave device. It is usually paired with ca_spi_cs_assert to select the slave, followed by ca_spi_read_buf to perform the transaction, and finally ca_spi_cs_deassert to release the bus. It is often used in polling-based communication loops where the application logic waits for a specific signal or state before initiating the read.

Esempio:

```
/* Example: Reading 8 bytes from an SPI slave device */
#define SPI_PORT 1
#define DATA_LEN 8

```

```c
void task_read_sensor(void *pvParameters) {
    u8 rx_buffer[DATA_LEN];

    /* Ensure SPI is initialized before use */
    ca_spi_init(SPI_PORT);

    while (1) {
        /* Select the slave device */
        ca_spi_cs_assert(SPI_PORT);

        /* Perform blocking read of 8 bytes */
        ca_spi_read_buf(SPI_PORT, rx_buffer, DATA_LEN);

        /* Release the slave device */
        ca_spi_cs_deassert(SPI_PORT);

        /* Process data or wait for next cycle */
        ca_os_delay_ms(100);
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_spi_read_buf in a caOS project, and what is the expected behavior regarding blocking and thread safety?
A: You should use ca_spi_read_buf when you need to receive a known number of bytes from an SPI slave device in a blocking manner. The function will block the calling thread until all bytes are received. It is not thread-safe if used concurrently by multiple threads without synchronization mechanisms. The function assumes the SPI peripheral is already configured and that a master is actively sending data; otherwise, it may block indefinitely.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_spi_read_buf with caOS types?
A: u8 spi_data[10]; ca_spi_read_buf(1, spi_data, 10);

Parameters conf:0.90 **Q:** What are the valid ranges for the spi_num parameter in ca_spi_read_buf, and what happens if an invalid value is passed?
A: The spi_num parameter must be a value between 1 and SPI_MAX_NUM inclusive. Values outside this range are ignored, meaning the function will not perform any SPI operation for invalid spi_num values.

`ca_spi_recv_handler`

Header: `caos/drivers/inc/spi.h:86`
Source: `caos/drivers/src/spi.c:85:88`
GUARD: `EN_SPI`

```c
void ca_spi_recv_handler(u8 spi_num)
```

Handles incoming SPI data processing, typically triggered by an interrupt service routine (ISR).

Parametri:

Nome	Tipo	Descrizione
`spi_num`	`u8`	The index of the SPI peripheral to handle. Must be a valid index within the range [1, SPI_MAX_NUM].

Ritorna:
void. This function does not return a value.

Note:

- This function is designed to be called from within an Interrupt Service Routine (ISR) context to process hardware-level SPI events. It is not thread-safe in the sense that it should only be invoked by the specific hardware interrupt associated with the SPI peripheral. A common mistake is calling this function from a standard task context instead of the ISR, which can lead to race conditions or improper handling of hardware flags. Ensure the SPI peripheral has been properly initialized using ca_spi_init before this handler is ever triggered.

Uso:

- This function is the bridge between the hardware SPI interrupt and the caOS driver layer. It is typically registered as the callback for the SPI interrupt vector. When the hardware signals that data has been received, the ISR calls this function to clear flags and move data into internal buffers. It is often used in conjunction with ca_spi_read_buf or ca_spi_dma_read to retrieve the processed data after the handler has performed the necessary low-level maintenance.

Esempio:

```c
/* Example: SPI Interrupt Service Routine */
void SPI1_IRQHandler(void)
{
    /* Notify the system that an SPI event occurred on peripheral 1 */
    ca_spi_recv_handler(1U);

    /*
     * After the handler processes the hardware flags,
     * we can safely read the data into our application buffer.
     */
    u8 my_buffer[16];
    ca_spi_read_buf(1U, my_buffer, 16U);
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_spi_recv_handler and in what context is it appropriate?
A: You should call ca_spi_recv_handler when you need to handle incoming data from an SPI bus, typically within an interrupt service routine (ISR). It is designed to be used in contexts where data reception from SPI is expected, and it is not thread-safe but can be invoked from ISR. The function is blocking and will not return until the data reception is complete.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_spi_recv_handler with caOS types?
A: u8 spi_num = 1; ca_spi_recv_handler(spi_num);

Parameters conf:0.90 **Q:** What is the meaning of the spi_num parameter in ca_spi_recv_handler, and what are the valid values?
A: The spi_num parameter specifies the number of the SPI peripheral to handle. It must be a value between 1 and SPI_MAX_NUM inclusive. If spi_num is outside this range, the function will not perform any action. The function does not validate the SPI peripheral itself, only the numerical range.

ca_spi_slave_sync

Header: caos/drivers/inc/spi.h:85
Source: caos/drivers/src/spi.c:112:116
GUARD: EN_SPI

```
void ca_spi_slave_sync(u8 spi_num); /* slave: call at frame start (CS
    edge): drops a partial frame, clears overrun */
```

Resets the SPI slave state machine to ensure frame alignment at the start of a transmission.

Parametri:

Nome	Tipo	Descrizione
spi_num	u8	The SPI peripheral index (1 to SPI_MAX_NUM). Values outside this range are ignored.

Ritorna:
None (void).

Note:

- This function is designed to be called from an EXTI (External Interrupt) handler triggered by the SPI Chip Select (CS) pin falling edge. It is not thread-safe in the sense that it must be called in the context of the hardware event to be effective. If you fail to call this, the SPI slave may remain misaligned if the master starts transmitting mid-frame, or it may remain locked in an error state (like an overrun) on certain hardware versions. The most common mistake is calling this from a standard task instead of the CS interrupt, which misses the critical timing window required to reset the hardware state before the first clock pulse arrives.

Uso:

- This function is used exclusively in SPI slave mode implementations. It is typically paired with an EXTI configuration that detects the CS pin state change. The sequence is: 1) Initialize the SPI peripheral using ca_spi_init, 2) Configure the CS pin as an input with an EXTI interrupt using ca_exti_init, 3) Inside the EXTI ISR, call ca_spi_slave_sync to clear the hardware state, and 4) Proceed to handle the data transfer using ca_spi_recv_handler or similar.

Esempio:

```
/* Example: Handling SPI Slave CS edge in an ISR */
void EXTI_CS_Pin_Handler(void)
{
    /* 1. Notify the SPI driver that a new frame is starting */
    /* This clears any pending overruns and resets the shift register */
    ca_spi_slave_sync(1);

    /* 2. Check if the sync was late (optional diagnostic) */
    /* ca_spi_hw_slave_sync_pending is not in the provided list,
       so we rely on the sync function to handle the hardware state */

    /* 3. Proceed with data reception logic */
    ca_spi_recv_handler(1);
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When and why should I call ca_spi_slave_sync in my caOS project?

A: You should call ca_spi_slave_sync inside an EXTI interrupt handler triggered by the SPI Chip Select (CS) falling edge. It is necessary to reset the SPI slave state machine to ensure frame alignment and clear any overrun errors that might occur if the master starts transmitting mid-frame.

Code example conf:0.95 **Q:** How do I implement the EXTI handler for SPI slave synchronization using caOS types?
A: You should define your ISR to call the sync function as follows: void EXTI_Handler(void) { u8 spi_id = 1; ca_spi_slave_sync(spi_id); } This ensures the hardware state is reset before the first clock pulse arrives.

Parameters conf:1.00 **Q:** What are the valid values for the spi_num parameter in ca_spi_slave_sync?
A: The parameter spi_num must be a u8 representing the SPI peripheral index, ranging from 1 to SPI_MAX_NUM. Any value provided outside of this range will be ignored by the function.

`ca_spi_stop`

Header: `caos/drivers/inc/spi.h:79`
Source: `caos/drivers/src/spi.c:68:80`
GUARD: `EN_SPI`

```
bool ca_spi_stop(u8 spi_num)
```

Disables a specific SPI peripheral and clears its internal active status in the OS.

Parametri:

Nome	Tipo	Descrizione
`spi_num`	`u8`	The SPI peripheral index to stop. Valid values are 1, 2, or 3. Values outside this range will cause the function to return false immediately.

Ritorna:
Returns true if the SPI peripheral was successfully stopped and the internal state updated. Returns false if the provided spi_num is invalid or if the underlying hardware stop operation fails.

Note:

- This function is not thread-safe and should not be called from an ISR. It performs a hard stop on the SPI peripheral, meaning any ongoing DMA transfers or communication will be abruptly terminated. You must call ca_spi_init() again before attempting to use the SPI peripheral after calling this function, as the internal configuration state is cleared. A common mistake is calling this function while a DMA transfer is still in progress, which can leave the hardware in an inconsistent state.

Uso:

- This function is typically used during system shutdown, power-saving modes, or when reconfiguring the SPI bus for a different protocol. It is usually paired with ca_spi_init() for setup and is often called after ensuring all SPI-related tasks have been parked or suspended using ca_os_sleep() or similar synchronization primitives. It ensures that the CLI and DMA drivers no longer attempt to access the peripheral, preventing potential system crashes or memory corruption.

Esempio:

```
/* Example: Safely shutting down the SPI bus before entering a low-power state
   */
```

```c
#define MASTER_SPI 1

void shutdown_spi_bus(void) {
    /* 1. Ensure no tasks are using the SPI bus */
    ca_os_sleep(100);

    /* 2. Stop the SPI peripheral */
    if (ca_spi_stop(MASTER_SPI)) {
        /* Successfully stopped, safe to proceed */
    } else {
        /* Handle error: SPI might have been busy or invalid index */
        ca_printf("Error: Failed to stop SPI %u\n", MASTER_SPI);
    }

    /* 3. Peripheral is now inactive; ca_spi_init() is required to restart */
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_spi_stop and what is its primary purpose?
A: You should use ca_spi_stop during system shutdown, power-saving modes, or when reconfiguring the SPI bus. Its purpose is to disable the SPI peripheral and clear its internal active status in the OS, ensuring that CLI and DMA drivers no longer attempt to access the peripheral.

Code example conf:1.00 **Q:** Provide a minimal example of how to stop SPI peripheral 2 in caOS.
A: u8 spi_id = 2; if (ca_spi_stop(spi_id)) { /* SPI 2 successfully stopped */ }

Parameters conf:1.00 **Q:** What are the valid values for the spi_num parameter in ca_spi_stop?
A: The spi_num parameter must be 1, 2, or 3. Providing any value outside this range will cause the function to return false immediately.

Return conf:1.00 **Q:** What do the return values of ca_spi_stop indicate?
A: A return value of true indicates the SPI peripheral was successfully stopped and the internal state was updated. A return value of false indicates that the provided spi_num was invalid or the underlying hardware stop operation failed.

Error/Edge case conf:1.00 **Q:** What are the common mistakes or risks when calling ca_spi_stop?
A: A common mistake is calling this function while a DMA transfer is in progress, which can leave the hardware in an inconsistent state. Additionally, this function is not thread-safe, must not be called from an ISR, and requires a subsequent call to ca_spi_init() before the peripheral can be used again.

`ca_spi_write_buf`

Header: `caos/drivers/inc/spi.h:82`
Source: `caos/drivers/src/spi.c:73:77`
GUARD: `EN_SPI`

```c
void ca_spi_write_buf(u8 spi_num, const u8 *buf, u16 len)
```

Transmits a buffer of data over a specified SPI peripheral.

Parametri:

Nome	Tipo	Descrizione
`spi_num`	`u8`	The index of the SPI peripheral to use (e.g., 0, 1). Must correspond to a peripheral initialized via ca_spi_init.
`buf`	`const u8 *`	Pointer to the memory buffer containing the data to be sent. Must not be NULL.
`len`	`u16`	Number of bytes to transmit from the buffer. A value of 0 results in no action.

Ritorna:
void

Note:

- This function is a blocking call; it will not return until every byte in the buffer has been processed by the hardware. It is not thread-safe by itself, meaning if multiple tasks access the same SPI peripheral, you must protect this call with a mutex or semaphore. It is generally not safe to call this from an ISR because the blocking nature will stall the system interrupt latency. Ensure the SPI peripheral is properly initialized with ca_spi_init before calling this function, or the system may hang or trigger a hardware fault.

Uso:

- This function is typically used to send command sequences or data frames to external SPI peripherals like sensors, displays, or flash memory. The standard workflow involves: 1) Initializing the SPI bus with ca_spi_init, 2) Asserting the Chip Select (CS) line using ca_spi_cs_assert, 3) Calling ca_spi_write_buf to send the payload, and 4) Deasserting the CS line with ca_spi_cs_deassert. It is often paired with ca_spi_read_buf when full-duplex communication is required.

Esempio:

```c
/* Example: Sending a configuration frame to an external device */
#define MY_SPI_BUS 1

void task_send_config(void) {
    u8 config_data[] = {0xAA, 0xBB, 0xCC, 0xDD};

    /* Ensure SPI is initialized before use */
    ca_spi_init(MY_SPI_BUS);

    /* Select the target device */
    ca_spi_cs_assert(MY_SPI_BUS);

    /* Send the buffer synchronously */
    ca_spi_write_buf(MY_SPI_BUS, config_data, 4);

    /* Release the target device */
    ca_spi_cs_deassert(MY_SPI_BUS);

    ca_os_sleep(100);
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_spi_write_buf() in a caOS project, and what is the main reason for calling it?
A: You should use ca_spi_write_buf() when you need to send a buffer of data over a specific SPI bus in a caOS project. The main reason is to perform a blocking write operation to an SPI peripheral, sending each byte in the buffer one at a time using

hardware transfer functions. It is typically used after initializing the SPI bus and ensuring it is available for communication.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_spi_write_buf() with caOS types?
A: u8 spi_data[] = {0x01, 0x02, 0x03, 0x04}; ca_spi_write_buf(0, spi_data, 4);

Parameters conf:0.90 **Q:** What is the meaning of the spi_num parameter in ca_spi_write_buf(), and what are valid and invalid values for it?
A: The spi_num parameter specifies which SPI bus to use, such as MASTER_SPI = 0 or SPI1 = 1. Valid values must be defined within the system and correspond to initialized SPI buses. Invalid values, such as an undefined bus number, result in undefined behavior and may cause a crash. The buf parameter must not be NULL, or the system will crash.

`ca_spi_xfer`

Header: `caos/drivers/inc/spi.h:79`
Source: `caos/drivers/src/spi.c:68:71`
GUARD: `EN_SPI`

```
u8 ca_spi_xfer(u8 spi_num, u8 data)
```

Performs a full-duplex SPI transfer by sending one byte and receiving one byte simultaneously.

Parametri:

Nome	Tipo	Descrizione
`spi_num`	`u8`	The index of the SPI peripheral to use (e.g., 0, 1, 2). Must match the hardware configuration initialized by ca_spi_init.
`data`	`u8`	The byte of data to transmit over the MOSI line.

Ritorna:
Returns the byte received from the MISO line during the transfer. There is no error code returned; the function assumes the hardware is ready.

Note:

- This function is a blocking call, meaning the CPU will wait until the entire byte has been shifted out and the corresponding byte has been shifted in before returning. It is generally not safe to call this from an ISR unless the underlying hardware driver explicitly supports it, as blocking in an ISR can crash the system. Ensure that ca_spi_init has been called for the specific spi_num before use. A common beginner mistake is to call this function without asserting the Chip Select (CS) pin first using ca_spi_cs_assert, which will result in the peripheral ignoring the data.

Uso:

- This function is the fundamental building block for SPI communication. It is typically used in a sequence: first, call ca_spi_cs_assert to select the target device, then call ca_spi_xfer one or more times to exchange data, and finally call ca_spi_cs_deassert to release the bus. For larger data transfers, developers often wrap this in a loop or use ca_spi_write_buf, but ca_spi_xfer remains the core mechanism for synchronous byte-level interaction.

Esempio:

```
/* Example: Reading a register from an external SPI sensor */
#define SENSOR_SPI 1
#define READ_CMD 0x80
```

```

void read_sensor_data(void) {
    u8 rx_val;

    /* 1. Select the device */
    ca_spi_cs_assert(SENSOR_SPI);

    /* 2. Send command and receive response byte */
    /* We send the command byte and get the status/data back */
    rx_val = ca_spi_xfer(SENSOR_SPI, READ_CMD);

    /* 3. Deselect the device to end communication */
    ca_spi_cs_deassert(SENSOR_SPI);

    /* Log the result using caOS stdout */
    ca_stdout_printf_n("Sensor returned: 0x%02X", rx_val);
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use the ca_spi_xfer function in a caOS project?
A: You should use the ca_spi_xfer function when you need to perform a full-duplex SPI transfer in master mode, sending one byte and receiving one byte simultaneously. It is ideal for communicating with SPI devices such as sensors, flash memory, or display drivers where you need to send a command and read a response in a single operation.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_spi_xfer with caOS types?
A: u8 spi_data = 0x55; u8 received = ca_spi_xfer(0, spi_data);

Parameters conf:0.85 **Q:** What is the valid range for the spi_num parameter in ca_spi_xfer, and what happens if I pass an invalid value?
A: The spi_num parameter must be a valid SPI peripheral number, ranging from 0 up to the maximum number of SPI peripherals available on the STM32 device. Passing an invalid value (e.g., greater than the maximum SPI count) may lead to undefined behavior or a crash, as the function assumes the SPI is already initialized and enabled.

Return conf:0.90 **Q:** What does the return value of ca_spi_xfer indicate, and what causes it to fail?
A: The return value is the byte received from the SPI slave device. If the function fails, it may return an incorrect value or a default value like 0xFF. Failures can occur if the SPI peripheral is not initialized, the slave device does not respond, or if the function is called without proper preconditions, potentially causing indefinite blocking or system instability.

`ca_spinlock_cooperative_init`

Header: `caos/inc/atomic.h:108`
Source: `caos/src/atomic.c:204:209`
GUARD: `EN_ATOMIC`

```
void ca_spinlock_cooperative_init(ca_spinlock_t *lock)
```

Initializes a cooperative spinlock structure to the free state.

Parametri:

Nome	Tipo	Descrizione
`lock`	`ca_spinlock_t *`	Pointer to the spinlock structure to initialize. Must not be NULL.

Ritorna:
void

Note:

- This function is non-blocking and performs a simple memory write followed by a Data Memory Barrier (___DMB). It is safe to call from any context, but it is typically called during system startup. The most common beginner mistake is failing to initialize the lock before passing it to lock or trylock functions, which leads to undefined behavior as the 'key' will contain random garbage memory. Always ensure the memory address provided is valid and not currently being accessed by other tasks during the initialization phase.

Uso:

- This function is the mandatory first step when using a cooperative spinlock. It should be called once during the system initialization phase or before the spinlock is first used by any task. It is typically paired with ca_spinlock_lock_cooperative or ca_spinlock_trylock_cooperative to manage access to shared resources. Once initialized, the lock remains valid until the system is reset or the memory containing the lock is deallocated.

Esempio:

```
ca_spinlock_t my_shared_resource_lock;

void system_init(void) {
    // Initialize the lock before any tasks attempt to use it
    ca_spinlock_cooperative_init(&my_shared_resource_lock);
}

void task_worker(void *param) {
    // Attempt to acquire the lock before accessing shared data
    ca_spinlock_lock_cooperative(&my_shared_resource_lock);

    /* Perform critical section operations here */

    // Always release the lock after the critical section
    ca_spinlock_unlock_cooperative(&my_shared_resource_lock);
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_spinlock_cooperative_init and why is it important for my embedded application?
A: You should call ca_spinlock_cooperative_init to initialize a cooperative spinlock before using it to protect critical sections in your embedded application. This function is important because it prepares the spinlock structure for use in scenarios involving multiple threads or interrupts, ensuring that the lock starts in a known free state. It is specifically designed for long critical sections and must not be called from an ISR.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_spinlock_cooperative_init with caOS types?
A: ca_spinlock_t my_lock; ca_spinlock_cooperative_init(&my_lock);

Parameters conf:0.90 **Q:** What does the 'lock' parameter represent in ca_spinlock_cooperative_init, and what happens if it is NULL?
A: The 'lock' parameter is a pointer to a ca_spinlock_t structure that needs to be initialized. If the pointer is NULL, the function does nothing and does not initialize anything. The function does not return any value, and it is not thread-safe in itself, but it prepares the structure for safe usage in multi-threaded or interrupt contexts.

ca_spinlock_get

Header: caos/inc/atomic.h:152
Source: caos/src/atomic.c:334:336
GUARD: EN_ATOMIC

```
u32 ca_spinlock_get(volatile u32 *key)
```

Reads the current value of a spinlock variable.

Parametri:

Nome	Tipo	Descrizione
key	volatile u32 *	Pointer to the spinlock variable. Must be a valid, non-NULL address pointing to a memory location initialized as a spinlock.

Ritorna:
The current u32 value stored at the memory address pointed to by key. This value typically represents the lock state (e.g., 0 for unlocked, 1 for locked).

Note:

- This function is a simple, non-blocking read operation. It is thread-safe in the sense that it performs a single atomic-like read of a volatile variable, but it does not perform any synchronization or memory barriers itself. It is safe to call from an ISR because it does not block or trigger a context switch. The most common mistake is assuming this function 'acquires' the lock; it only inspects the current state. To actually acquire a lock, you must use functions like ca_spinlock_lock_irq or ca_spinlock_lock_cooperative.

Uso:

- This function is used when you need to check the status of a resource without waiting for it. It is typically used in diagnostic routines, logging, or before attempting a non-blocking operation to see if a resource is currently busy. It is often paired with other spinlock functions to implement custom polling logic or to verify the state of a system component before performing a critical operation.

Esempio:

```
/* Example: Checking if a shared resource is busy before proceeding */
#include "caos.h"

static volatile u32 my_resource_lock = 0;

void check_resource_status(void) {
    /* Read the current state of the lock */
    u32 status = ca_spinlock_get(&my_resource_lock);

    if (status == 0) {
        /* Resource is free, safe to attempt acquisition */
        ca_spinlock_lock_irq(&my_resource_lock);
        /* Perform critical section work here */
        ca_spinlock_unlock_irq(&my_resource_lock);
    } else {
        /* Resource is busy, log the event or perform other tasks */
        ca_printf("Resource is currently locked, skipping operation.\n");
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_spinlock_get() in a caOS application?
A: You should use ca_spinlock_get() when you need to check the status of a spinlock without blocking, typically to determine if a resource is currently occupied or available. It's useful in scenarios where you want to avoid blocking threads or ISRs, and instead make a decision based on whether the lock is already acquired by another thread or interrupt handler.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_spinlock_get() with caOS types?
A: volatile u32 lock = 0;
u32 result = ca_spinlock_get(&lock); if (result == 0) { // Lock is free, resource can be accessed } else { // Lock is acquired by another thread or ISR }

Parameters conf:0.95 **Q:** What does the key parameter in ca_spinlock_get() represent, and what are valid values?
A: The key parameter is a pointer to a volatile u32 variable that represents the state of a spinlock. The variable should be initialized to 0 to indicate a free lock, and any non-zero value to indicate an acquired lock. The pointer itself must not be NULL, as passing NULL leads to undefined behavior and potential crashes.

Return conf:0.90 **Q:** What do the return values of ca_spinlock_get() mean, and what causes failures?
A: A return value of 0 means the lock is free and the resource is available. Any non-zero value indicates the lock is acquired by another thread or ISR. Failures occur if the key parameter is NULL, which leads to undefined behavior, including possible memory corruption or system crashes.

`ca_spinlock_irq_init`

Header: `caos/inc/atomic.h:77`
Source: `caos/src/atomic.c:107:111`
GUARD: `EN_ATOMIC`

```
1 void ca_spinlock_irq_init(ca_spinlock_irq_t *lock)
```

Initializes a spinlock structure designed to protect short critical sections from both task-level and ISR-level interference.

Parametri:

Nome	Tipo	Descrizione
`lock`	`ca_spinlock_irq_t *`	Pointer to the spinlock control structure. Must be allocated in memory accessible by both the tasks and the ISRs that will use it.

Ritorna:
void

Note:

- This function is not blocking; it simply resets the internal state of the lock to 'free' and ensures memory visibility via a Data Memory Barrier (___DMB). It is mandatory to call this before using the lock in any other function like ca_spinlock_lock_irq or ca_spinlock_unlock_irq. A common mistake is to declare the lock structure as a local variable inside a function that exits, which would leave the lock pointing to invalid stack memory. This function is thread-safe in the sense that it sets up the initial state, but it must be called exactly once during the initialization phase of the resource it protects.

Uso:

- This function is used during the system setup phase, typically inside an init function for a driver or a shared resource. It is paired with ca_spinlock_lock_irq and ca_spinlock_unlock_irq to create atomic sections that prevent race conditions between the main thread and interrupt service routines. You should call this once when the hardware or software module is first initialized, ensuring that the lock is in a known 'free' state before any concurrent access attempts occur.

Esempio:

```c
/* Define the lock globally or in a persistent scope */
caSpinlockIrq my_resource_lock;

void my_module_init(void) {
    /* Initialize the lock before any potential use */
    ca_spinlock_irq_init(&my_resource_lock);
}

void my_critical_task(void) {
    /* Acquire the lock to protect shared data */
    ca_spinlock_lock_irq(&my_resource_lock);

    /* Perform short, time-critical operations here */

    /* Release the lock to allow other tasks or ISRs to proceed */
    ca_spinlock_unlock_irq(&my_resource_lock);
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_spinlock_irq_init() and why is it important for my embedded application?
A: You should call ca_spinlock_irq_init() when you need to initialize a spinlock that will be used to protect critical sections in your code, especially in contexts that can be interrupted. This function is important because it ensures that the spinlock is properly set up to be IRQ-safe, allowing it to be used both in regular threads and from interrupt service routines (ISRs). It's particularly useful in drivers or modules that handle concurrent access to shared resources, such as DMA transfers or network services.

Code example conf:0.90 **Q:** Can you provide a minimal working C example using ca_spinlock_irq_init() with caOS types?
A: ca_spinlock_irq_t my_lock; ca_spinlock_irq_init(&my_lock); // Now my_lock is initialized and can be used with ca_spinlock_irq_lock() and ca_spinlock_irq_unlock().

Parameters conf:0.90 **Q:** What is the significance of the 'lock' parameter in ca_spinlock_irq_init(), and what happens if I pass an invalid value?
A: The 'lock' parameter is a pointer to a ca_spinlock_irq_t structure that must be allocated (either on the stack or heap) and not NULL. If you pass a NULL pointer or a structure that is not properly allocated, the behavior is undefined. The function does not fail or return an error code; it simply assumes the parameter is valid and proceeds to initialize the spinlock. It's crucial that the structure remains valid for the entire duration of its use.

`ca_spinlock_lock_adaptive`

Header: `caos/inc/atomic.h:141`
Source: `caos/src/atomic.c:292:328`
GUARD: `EN_ATOMIC`

```
void ca_spinlock_lock_adaptive(ca_spinlock_t *lock)
```

Acquires a spinlock using an adaptive strategy: fast busy-waiting followed by cooperative yielding.

Parametri:

Nome	Tipo	Descrizione
`lock`	`ca_spinlock_t *`	Pointer to the spinlock structure. Must be initialized via ca_spinlock_cooperative_init before use. Passing NULL will result in a silent return.

Ritorna:
void. The function blocks until the lock is successfully acquired.

Note:

- This function is designed for thread-level synchronization and is NOT safe to call from an ISR (Interrupt Service Routine). It uses a two-phase approach: it first performs a short 'busy-wait' loop to avoid the overhead of a context switch if the lock is released quickly. If the lock remains held, it switches to a cooperative mode using ca_os_yield, which allows other tasks to run and prevents CPU starvation. A common mistake is using this in an ISR; since ISRs cannot yield, this will cause undefined behavior or system instability. Always ensure the lock is initialized before the first call.

Uso:

- Use this function when you need to protect a shared resource (like a global variable or a peripheral) accessed by multiple tasks. It is typically paired with ca_spinlock_cooperative_init during system setup and ca_spinlock_unlock_cooperative to release the resource. It is ideal for scenarios where the critical section is very short, as the adaptive spin avoids the latency of putting a task to sleep and waking it up again.

Esempio:

```
/* Global spinlock instance */
caSpinlock my_lock;

void task_function(void *arg) {
    /* Initialize the lock before any task attempts to use it */
    ca_spinlock_cooperative_init(&my_lock);

    while (1) {
        /* Acquire the lock using the adaptive strategy */
        ca_spinlock_lock_adaptive(&my_lock);

        /* Critical section: access shared resource safely */
        // Perform protected operations here

        /* Release the lock so other tasks can proceed */
        ca_spinlock_unlock_cooperative(&my_lock);

        /* Yield to allow other tasks to execute */
        ca_os_yield();
    }
}
```

Q&A — fine-tuning review

6/6 pairs · qwen3-coder:latest

Usage conf:0.95 **Q:** When should I use ca_spinlock_lock_adaptive() in my caOS application?
A: You should use ca_spinlock_lock_adaptive() when you need to protect a critical section with a lock that adapts its behavior based on how long the lock is held. It's

particularly useful in scenarios where you want to avoid unnecessary context switches but still ensure that threads don't busy-wait indefinitely. It's ideal for protecting shared resources in a multi-threaded environment where performance is critical.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_spinlock_lock_adaptive() with caOS types?
A: ca_spinlock_t my_lock; ca_spinlock_init(&my_lock); ca_spinlock_lock_adaptive(&my_lock); // Critical section ca_spinlock_unlock(&my_lock);

Parameters conf:0.95 **Q:** What is the significance of the 'lock' parameter in ca_spinlock_lock_adaptive()?
A: The 'lock' parameter is a pointer to a ca_spinlock_t structure that must be initialized before calling this function. If the pointer is NULL, the function does nothing. It is essential that the lock is properly initialized using ca_spinlock_init() before being passed to this function.

Return conf:0.95 **Q:** Does ca_spinlock_lock_adaptive() return any value, and what does it mean?
A: The function does not return any value. It is a void function. If the lock parameter is NULL, the function silently does nothing. Otherwise, it blocks until the lock is acquired.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_spinlock_lock_adaptive
A: Common mistakes include calling the function with a NULL pointer, which results in no action. Another mistake is using the function from an ISR context, which can lead to deadlock. Additionally, failing to properly initialize the spinlock with ca_spinlock_init() before use can cause undefined behavior.

Behavior conf:0.90 **Q:** Is ca_spinlock_lock_adaptive() blocking or non-blocking, and is it safe to use from an ISR?
A: The function is blocking; it will wait until the lock is acquired. It is not safe to use from an ISR context, as it can cause a deadlock. It is also not thread-safe if used from an ISR, and should not be called from interrupt contexts.

`ca_spinlock_lock_cooperative`

Header: `caos/inc/atomic.h:116`
Source: `caos/src/atomic.c:211:239`
GUARD: `EN_ATOMIC`

```
1 void ca_spinlock_lock_cooperative(ca_spinlock_t *lock)
```

Acquires a spinlock by yielding to the scheduler if the lock is currently held by another thread.

Parametri:

Nome	Tipo	Descrizione
`lock`	`ca_spinlock_t *`	Pointer to the spinlock structure. Must be initialized via ca_spinlock_cooperative_init before use. Passing NULL will result in a silent return.

Ritorna:
void. The function does not return until the lock has been successfully acquired.

Note:

- This function is cooperative, meaning it is non-blocking in the sense that it allows other tasks to run while waiting, but it is blocking for the calling thread. It is strictly forbidden to call this from an ISR (Interrupt Service Routine) because ISRs cannot yield to the scheduler; the DEBUG build includes a safeguard to catch this. It is thread-safe, but it

relies on the assumption that the thread currently holding the lock will eventually release it, otherwise, the system will experience a permanent hang. The most common beginner mistake is attempting to use this in an interrupt context or failing to initialize the lock structure before passing it to this function.

Uso:

- This function is used when you need to protect a shared resource (like a global buffer or a peripheral) across multiple threads in a cooperative multitasking environment. It is typically used in a sequence: first, initialize the lock with ca_spinlock_cooperative_init, then call ca_spinlock_lock_cooperative before accessing the shared resource, and finally call ca_spinlock_unlock_cooperative to release it. It is preferred over non-cooperative locks when you want to ensure that the CPU remains available for other tasks while waiting for the resource to become free.

Esempio:

```c
/* Define a global spinlock structure */
caSpinlock my_shared_lock;

void task_access_resource(void) {
    /* Initialize the lock before first use */
    ca_spinlock_cooperative_init(&my_shared_lock);

    while (1) {
        /* Wait for the lock, yielding to other tasks if busy */
        ca_spinlock_lock_cooperative(&my_shared_lock);

        /* Critical section: access shared resource safely */
        /* ... perform operations ... */

        /* Release the lock so other tasks can acquire it */
        ca_spinlock_unlock_cooperative(&my_shared_lock);

        /* Give other tasks a chance to run */
        ca_os_yield();
    }
}
```

Q&A — fine-tuning review

6/6 pairs · qwen3-coder:latest

Usage conf:0.95 **Q:** When should I use ca_spinlock_lock_cooperative() in a caOS application?
A: You should use ca_spinlock_lock_cooperative() when you need to protect critical sections of code that are shared between threads, ensuring mutual exclusion. It is particularly useful in multi-threaded contexts where you want to avoid blocking other threads while waiting for a lock, as it uses cooperative yielding to allow the scheduler to run other tasks.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_spinlock_lock_cooper
with caOS types?
A: ca_spinlock_t lock = SPINLOCK_FREE; ca_spinlock_lock_cooperative(&lock); // Critical section ca_spinlock_unlock(&lock);

Parameters conf:0.95 **Q:** What is the significance of the lock parameter in ca_spinlock_lock_cooperative()?
A: The lock parameter is a pointer to a ca_spinlock_t structure that must be initialized with SPINLOCK_FREE before use. If the pointer is NULL, the function does nothing. It is essential that the lock is properly initialized to avoid undefined behavior or deadlock.

Return conf:0.95 **Q:** Does ca_spinlock_lock_cooperative() return any value, and what does it signify?
A: The function does not return any value. It is a void function, and it will never fail. However, it may block indefinitely if the lock is never released, leading to a potential

deadlock.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_spinlock_lock_cooperative()?
A: Common mistakes include calling the function from an ISR, using an uninitialized lock pointer, or failing to release the lock after acquiring it. These can lead to undefined behavior or deadlocks.

Behavior conf:0.95 **Q:** Is ca_spinlock_lock_cooperative() blocking or non-blocking, and is it safe to call from an ISR?
A: The function is blocking, as it will not return until the lock is acquired. It is not safe to call from an ISR, as it is not ISR-safe. It must only be used in thread contexts, not in interrupt handlers.

`ca_spinlock_lock_irq`

Header: `caos/inc/atomic.h:85`
Source: `caos/src/atomic.c:113:146`
GUARD: `EN_ATOMIC`

```
void ca_spinlock_lock_irq(ca_spinlock_irq_t *lock)
```

Acquires a spinlock while disabling interrupts to ensure exclusive access to shared resources.

Parametri:

Nome	Tipo	Descrizione
`lock`	`ca_spinlock_irq_t *`	Pointer to the spinlock structure. Must be initialized via ca_spinlock_irq_init before use.

Ritorna:
void

Note:

- This function is strictly non-blocking in the sense that it disables interrupts to prevent preemption, effectively guaranteeing the lock is acquired immediately. It is NOT safe to call from an Interrupt Service Routine (ISR) because disabling interrupts inside an ISR can lead to system instability or deadlocks. The function is thread-safe for task-level code, but you must ensure that every call to this function is paired with a corresponding ca_spinlock_unlock_irq call. A common beginner mistake is forgetting to unlock the spinlock, which will leave the system with interrupts permanently disabled, causing the entire RTOS to hang.

Uso:

- Use this function when you need to protect a shared data structure (like a linked list, a memory pool, or a global counter) from being accessed simultaneously by multiple tasks or by a task and an interrupt. Because it disables interrupts, it is the most 'aggressive' form of synchronization; use it only for very short code sections to avoid increasing system interrupt latency. It is typically used in a sequence: call ca_spinlock_lock_irq, perform the quick data modification, and immediately call ca_spinlock_unlock_irq.

Esempio:

```
static ca_spinlock_irq_t pool_lock;

void update_pool_data(ca_block_pool_t *bp) {
    // Ensure the lock is initialized before use
    ca_spinlock_irq_init(&pool_lock);

    // Enter critical section to protect the block pool
```

```
    ca_spinlock_lock_irq(&pool_lock);

    // Perform a quick operation on the shared resource
    void *ptr = ca_block_alloc(bp);
    if (ptr != NULL) {
        /* Process the allocated block */
    }

    // Always release the lock immediately after the critical section
    ca_spinlock_unlock_irq(&pool_lock);
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_spinlock_lock_irq() in a caOS project?
A: You should use ca_spinlock_lock_irq() when you need to protect a critical section of code from concurrent access in a multi-threaded or interrupt-driven environment. It is particularly useful when you want to ensure mutual exclusion while disabling interrupts to prevent race conditions. It is typically used in device drivers or low-level system components where atomicity is required.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_spinlock_lock_irq() with caOS types?
A: ca_spinlock_irq_t lock; ca_spinlock_init_irq(&lock); ca_spinlock_lock_irq(&lock); // Critical section ca_spinlock_unlock_irq(&lock);

Parameters conf:0.95 **Q:** What is the significance of the 'lock' parameter in ca_spinlock_lock_irq()?
A: The 'lock' parameter is a pointer to a ca_spinlock_irq_t structure that represents the spinlock to be acquired. It must be initialized using ca_spinlock_init_irq() before calling this function. If NULL is passed, the function returns immediately without performing any action.

Return conf:0.95 **Q:** Does ca_spinlock_lock_irq() return any value, and what does it mean?
A: The function does not return any value. It is void. It never fails in a traditional sense, but it will not perform any action if the input parameter is NULL.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_spinlock_lock_irq()?
A: Common mistakes include calling this function from an ISR, which disables interrupts and can cause system instability. Another mistake is failing to call ca_spinlock_unlock_irq() after locking, which leads to deadlock. Also, passing a NULL pointer will result in no action being taken, which may be unintended.

Behavior conf:0.90 **Q:** Is ca_spinlock_lock_irq() blocking or non-blocking, and is it safe to call from an ISR?
A: The function is blocking, meaning it will wait indefinitely until the lock becomes available. It is not safe to call from an ISR because it disables interrupts, which can lead to system instability or missed interrupts. It is also not thread-safe as it is not designed for concurrent calls from multiple threads.

`ca_spinlock_trylock_cooperative`

Header: `caos/inc/atomic.h:129`
Source: `caos/src/atomic.c:264:281`
GUARD: `EN_ATOMIC`

```
bool ca_spinlock_trylock_cooperative(ca_spinlock_t *lock)
```

Attempts to acquire a cooperative spinlock without blocking the current thread.

Parametri:

Nome	Tipo	Descrizione
lock	ca_spinlock_t *	Pointer to the spinlock structure. Must be initialized via ca_spinlock_cooperative_init before use. Passing NULL will return false.

Ritorna:
Returns true if the lock was successfully acquired (the thread now owns the resource). Returns false if the lock is already held by another thread, in which case the function returns immediately without waiting.

Note:

- This function is non-blocking, meaning it is ideal for scenarios where you cannot afford to wait for a resource. It is NOT thread-safe to call from an ISR (Interrupt Service Routine) because it is designed for cooperative task synchronization; the internal safeguard checks for this. The most common beginner mistake is assuming that a failed trylock means the system is broken; in reality, it simply means the resource is busy, and your code should handle this by either performing an alternative task or retrying later.

Uso:

- Use this function when you need to access a shared resource but want to maintain high system responsiveness by avoiding thread suspension. It is typically used in a loop or a conditional check where the thread can perform other work if the lock is unavailable. It is always paired with ca_spinlock_cooperative_init for setup and ca_spinlock_unlock_cooperative to release the resource once the critical section is complete.

Esempio:

```
ca_spinlock_t my_lock;

// Initialize the lock before use
ca_spinlock_cooperative_init(&my_lock);

void task_function(void) {
    // Attempt to acquire the lock without blocking
    if (ca_spinlock_trylock_cooperative(&my_lock)) {
        // Critical section: safely access shared resource
        // ...
        ca_spinlock_unlock_cooperative(&my_lock);
    } else {
        // Lock was busy, perform other work instead of waiting
        ca_os_yield();
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_spinlock_trylock_cooperative instead of a regular spinlock acquire function?
A: You should use ca_spinlock_trylock_cooperative when you need to acquire a spinlock without blocking the calling thread. It's useful in scenarios where you want to avoid potential delays or deadlocks, such as in time-critical sections or when you're already in a context where blocking is not allowed.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_spinlock_trylock_cooperative in C with caOS types?
A: ca_spinlock_t lock; ca_spinlock_init(&lock); if (ca_spinlock_trylock_cooperative(&lock)) { // Lock acquired, do critical section ca_spinlock_unlock(&lock); } else { // Lock was busy, handle accordingly }

Parameters conf:0.95 **Q:** What is the significance of the 'lock' parameter in ca_spinlock_trylock_cooperative, and what happens if it is NULL?
A: The 'lock' parameter is a pointer to the spinlock structure to be acquired. If it is NULL, the function returns false immediately, as it cannot operate on an invalid memory location.

Return conf:0.95 **Q:** What does it mean when ca_spinlock_trylock_cooperative returns false?
A: When ca_spinlock_trylock_cooperative returns false, it means that the spinlock was already occupied by another thread, and the function did not block to wait for it to become available. The lock was not acquired.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_spinlock_trylock_cooperative?
A: Common mistakes include calling this function from an ISR, which is not allowed, and passing a NULL pointer to the lock parameter. Additionally, failing to release the lock after acquiring it can lead to deadlocks or undefined behavior.

Behavior conf:0.95 **Q:** Is ca_spinlock_trylock_cooperative blocking or non-blocking, and can it be called from an ISR?
A: ca_spinlock_trylock_cooperative is a non-blocking function. It does not wait for the lock to become available and returns immediately. It cannot be called from an ISR, as it may cause context or deadlock issues, and is verified to ensure IPSR is 0.

`ca_spinlock_trylock_irq`

Header: `caos/inc/atomic.h:98`
Source: `caos/src/atomic.c:172:198`
GUARD: `EN_ATOMIC`

```
1 bool ca_spinlock_trylock_irq(ca_spinlock_irq_t *lock)
```

Attempts to acquire a spinlock while disabling interrupts to prevent deadlocks.

Parametri:

Nome	Tipo	Descrizione
`lock`	`ca_spinlock_irq_t *`	Pointer to the spinlock structure. Must be initialized via ca_spinlock_irq_init before use. Passing NULL will return false.

Ritorna:
Returns true if the lock was successfully acquired (interrupts remain disabled). Returns false if the lock was already held by another task (interrupts are restored to their previous state).

Note:

- This function is non-blocking; it returns immediately if the lock is unavailable. It is thread-safe and designed to be used in critical sections where you must prevent preemption by interrupts. It is NOT callable from an ISR because it manipulates the CPU's interrupt mask (PRIMASK) to ensure atomicity. A common beginner mistake is forgetting to call the corresponding unlock function, which will leave interrupts permanently disabled and effectively freeze the entire system.

Uso:

- Use this when you need to protect a shared resource (like a global variable or hardware peripheral) that is accessed by both main-loop tasks and interrupt service routines. It is typically used in a 'try-lock' pattern: if the function returns true, you perform your atomic operation and then release the lock; if it returns false, you either skip the operation or retry later. Always pair this with the appropriate unlock function to restore the system state.

Esempio:

```
ca_spinlock_irq_t my_lock;

// Initialize the lock before use
ca_spinlock_irq_init(&my_lock);

// Attempt to access a shared resource
if (ca_spinlock_trylock_irq(&my_lock)) {
    /* Perform critical operation on shared data here */

    // Release the lock and restore interrupts
    ca_spinlock_unlock_irq(&my_lock);
} else {
    // Lock was busy, handle the contention (e.g., yield or retry)
    ca_os_yield();
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_spinlock_trylock_irq instead of a regular spinlock?
A: You should use ca_spinlock_trylock_irq when you need to acquire a spinlock in an interrupt context or when you want to avoid blocking. It is ideal for protecting short critical sections in ISR routines where waiting is not acceptable.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_spinlock_trylock_irq in a C function?
A: ca_spinlock_irq_t lock; ca_spinlock_init_irq(&lock); if (ca_spinlock_trylock_irq(&lock)) { // Critical section ca_spinlock_unlock_irq(&lock); } else { // Lock was not acquired }

Parameters conf:0.95 **Q:** What happens if I pass a NULL pointer to ca_spinlock_trylock_irq?
A: If the 'lock' parameter is NULL, the function will return false immediately without attempting to acquire the lock. This is a safety check to prevent undefined behavior.

Return conf:0.95 **Q:** What does it mean when ca_spinlock_trylock_irq returns false?
A: Returning false means that the spinlock was already in use and could not be acquired immediately. The function does not block; it simply returns false to indicate failure.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_spinlock_trylock_irq?
A: Common mistakes include calling this function from within an ISR context where it's already in an interrupt handler, or using an uninitialized spinlock. Also, calling it without proper initialization of the lock structure can lead to undefined behavior.

Behavior conf:0.90 **Q:** Is ca_spinlock_trylock_irq safe to call from an ISR and does it block?
A: Yes, ca_spinlock_trylock_irq is safe to call from an ISR and it is non-blocking. It uses atomic operations and is thread-safe, making it suitable for use in interrupt contexts without causing delays.

`ca_spinlock_unlock_cooperative`

Header: `caos/inc/atomic.h:122`
Source: `caos/src/atomic.c:241:262`
GUARD: `EN_ATOMIC`

```
void ca_spinlock_unlock_cooperative(ca_spinlock_t *lock)
```

Releases a cooperative spinlock, allowing other threads to acquire it.

Parametri:

Nome	Tipo	Descrizione
`lock`	`ca_spinlock_t *`	Pointer to the spinlock structure. Must be initialized via ca_spinlock_cooperative_init and currently held by the calling thread.

Ritorna:
void

Note:

- This function is strictly for thread-level use and must NEVER be called from an Interrupt Service Routine (ISR). It is not thread-safe if called on an uninitialized or null pointer, and the debug build will trigger a safeguard if the lock is not currently held. It performs a memory barrier to ensure all previous memory operations are completed before the lock is released. The most common beginner mistake is attempting to unlock a spinlock that was never successfully acquired or calling this from an interrupt context, which violates the cooperative nature of the lock.

Uso:

- This function is used to exit a critical section that was protected by a cooperative spinlock. It is typically paired with ca_spinlock_lock_cooperative at the start of a shared resource access. In a standard sequence, you initialize the lock once, call the lock function, perform your thread-safe operations on shared data, and finally call this unlock function to allow other threads to proceed. It is essential for maintaining data integrity when multiple threads access the same memory or peripheral.

Esempio:

```
ca_spinlock_t my_lock;

// Initialize the lock before use
ca_spinlock_cooperative_init(&my_lock);

void update_shared_resource(void) {
    // Acquire the lock before accessing shared data
    ca_spinlock_lock_cooperative(&my_lock);

    /* Perform critical operations on shared memory or hardware */

    // Release the lock so other threads can access the resource
    ca_spinlock_unlock_cooperative(&my_lock);
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_spinlock_unlock_cooperative() in my embedded application?
A: You should call ca_spinlock_unlock_cooperative() after acquiring a cooperative spinlock to release it and allow other threads to acquire it. It is typically used in critical sections where you need to protect shared resources from concurrent access in a cooperative multitasking environment.

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_spinlock_unlock_cooperative() with caOS types?
A: ca_spinlock_t my_lock; ca_spinlock_init_cooperative(&my_lock); ca_spinlock_lock_cooperative // Critical section code here ca_spinlock_unlock_cooperative(&my_lock);

Parameters conf:0.90 **Q:** What is the significance of the 'lock' parameter in ca_spinlock_unlock_cooperative()?
A: The 'lock' parameter is a pointer to a ca_spinlock_t structure that must have been previously initialized. If NULL, the function does nothing. It must point to a valid spinlock structure initialized by ca_spinlock_init_cooperative().

Return conf:0.95 **Q:** Does ca_spinlock_unlock_cooperative() return any value, and what does it indicate?
A: The function returns nothing (void). It cannot fail in a traditional sense, but it may ignore invalid inputs such as a NULL pointer. It does not provide any error indication.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_spinlock_unlock_cooperative()?
A: Common mistakes include calling this function without first acquiring the lock, calling it from an ISR, or passing a NULL pointer. Calling it from an ISR will cause a debug assert, and calling it without acquiring the lock leads to undefined behavior.

Behavior conf:0.95 **Q:** Is ca_spinlock_unlock_cooperative() blocking or non-blocking, and is it safe to call from an ISR?
A: The function is non-blocking and does not wait. It is not safe to call from an ISR because it violates cooperative multitasking rules. It must only be called from a thread context that has previously acquired the lock.

`ca_spinlock_unlock_irq`

Header: `caos/inc/atomic.h:91`
Source: `caos/src/atomic.c:148:170`
GUARD: `EN_ATOMIC`

```
1 void ca_spinlock_unlock_irq(ca_spinlock_irq_t *lock)
```

Releases a spinlock and restores the CPU interrupt state to what it was before the lock was acquired.

Parametri:

Nome	Tipo	Descrizione
`lock`	`ca_spinlock_irq_t *`	Pointer to the spinlock structure. Must be initialized via ca_spinlock_irq_init before use.

Ritorna:
void

Note:

- This function is non-blocking and is intended to be called from thread mode. It is NOT safe to call this from an ISR because it restores the PRIMASK register, which could inadvertently re-enable interrupts that were meant to remain disabled. The function is thread-safe regarding the lock state, but it assumes the caller is the current owner of the lock. A common beginner mistake is to call this function without having previously called ca_spinlock_lock_irq, or to call it from an ISR, which will trigger a safeguard failure in debug builds. Always ensure the lock pointer is valid, as passing a NULL pointer will result in an immediate return without releasing any resources, potentially leaving the system in a deadlocked state.

Uso:

- This function is used to exit a critical section that was protected by a spinlock which also disabled interrupts. It is typically used in a pair: first, you call ca_spinlock_lock_irq to enter the protected area, perform your atomic operations on shared resources (like a memory pool or a hardware peripheral), and then call ca_spinlock_unlock_irq to release the lock and restore the previous interrupt mask. This sequence is essential when modifying shared global variables or hardware registers that could be accessed by other tasks, ensuring that the operation completes without being interrupted by higher-priority threads or context switches.

Esempio:

```
/* Example: Protecting a shared block pool access */
caSpinlockIrq my_lock;

void init_system(void) {
    ca_spinlock_irq_init(&my_lock);
}

void safe_free_operation(ca_block_pool_t *pool, void *ptr) {
    /* Enter critical section: lock and disable interrupts */
    ca_spinlock_lock_irq(&my_lock);

    /* Perform the atomic operation on the pool */
    ca_block_free(pool, ptr);

    /* Exit critical section: release lock and restore interrupts */
    ca_spinlock_unlock_irq(&my_lock);
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_spinlock_unlock_irq and why is it important?
A: You should call ca_spinlock_unlock_irq after acquiring a spinlock with ca_spinlock_lock_irq to release the lock and restore the interrupt state. It's important because it ensures that critical sections are properly protected and that interrupts are restored to their previous state, which is essential for maintaining system stability and correct interrupt handling.

Code example conf:0.90 **Q:** Can you show a minimal example of using ca_spinlock_unlock_irq in a caOS application?
A: ca_spinlock_irq_t lock; ca_spinlock_init_irq(&lock); ca_spinlock_lock_irq(&lock); // Critical section code here ca_spinlock_unlock_irq(&lock);

Parameters conf:0.90 **Q:** What does the lock parameter in ca_spinlock_unlock_irq represent and what happens if it's NULL?
A: The lock parameter is a pointer to a ca_spinlock_irq_t structure that was previously initialized and locked with ca_spinlock_lock_irq. If NULL is passed, the function does nothing in normal operation but may generate an error in DEBUG mode.

Return conf:0.95 **Q:** Does ca_spinlock_unlock_irq return anything, and can it fail?
A: The function returns nothing (void). It cannot fail in the traditional sense, but it may ignore invalid inputs such as a NULL pointer in DEBUG mode.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_spinlock_unlock_irq?
A: Common mistakes include calling this function without first calling ca_spinlock_lock_irq on the same lock, passing a NULL pointer, or calling it from a context where the lock was not acquired (e.g., from a thread when it was locked from an ISR). These can lead to undefined behavior or system instability.

Behavior conf:0.90 **Q:** Is ca_spinlock_unlock_irq blocking, ISR-safe, or thread-safe?
A: The function is blocking only if the lock is already held by another thread or ISR. It is ISR-safe if the lock was acquired from an ISR. It is not thread-safe because it's not designed for concurrent access from multiple threads.

`ca_ssh_bpp_selftest`

Header: `caos/net/inc/ssh_service.h:64`
Source: `caos/net/src/ssh_bpp.c:193:338`
GUARD: `EN_SSH_SERVICE`

```
u32 ca_ssh_bpp_selftest(void)
```

Performs a comprehensive suite of unit tests on the SSH Binary Packet Protocol (BPP) framing and wire-buffer logic.

Ritorna:
Returns a u32 representing the total count of failed test cases. A return value of 0 indicates that all tests passed successfully.

Note:

- This function is a diagnostic tool designed to verify the integrity of the SSH framing implementation before the network stack is fully operational. It is non-blocking and executes synchronously, meaning it will hold the CPU until all tests are completed. It is not thread-safe and should not be called from an ISR, as it performs extensive stack operations and uses standard output for reporting. The most common mistake is calling this function after the network stack has already started processing real traffic, which can lead to race conditions or unexpected behavior if the internal buffers are being accessed simultaneously.

Uso:

- This function is typically used during the system initialization phase, specifically before opening network ports or starting the SSH service. It is often paired with ca_ssh_test_init() to prepare the test environment. Developers use it to ensure that the framing logic (packing/unpacking) and wire-buffer management (reading/writing) are functioning correctly on the specific hardware architecture, preventing runtime errors in the SSH transport layer.

Esempio:

```
/* Perform SSH framing sanity check before enabling network services */
#if EN_SSH_TEST
    ca_ssh_test_init();
    u32 test_fails = ca_ssh_bpp_selftest();
    if (test_fails > 0) {
        /* Log failure and prevent system startup if critical */
        ca_printf_n("SSH BPP Selftest failed with %u errors", test_fails);
        /* Halt or enter safe state */
    } else {
        ca_printf_n("SSH BPP Selftest passed");
    }
#endif
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to execute ca_ssh_bpp_selftest in the caOS boot sequence?
A: It should be called during the system initialization phase, specifically before opening network ports or starting the SSH service. It is typically paired with ca_ssh_test_init() to prepare the test environment.

Code example conf:0.90 **Q:** How do I implement a basic check for the SSH BPP logic using caOS types?
A: u32 result; ca_ssh_test_init(); result = ca_ssh_bpp_selftest(); if (result != 0) { /* Handle failure */ }

Return conf:1.00 **Q:** What does the u32 return value of ca_ssh_bpp_selftest signify?
A: The return value represents the total count of failed test cases. A return value of 0 indicates that all tests passed successfully.

Error/Edge case conf:1.00 **Q:** What is the most common mistake developers make when using ca_ssh_bpp_selftest?
A: The most common mistake is calling this function after the network stack has already started processing real traffic, which can lead to race conditions or unexpected behavior if internal buffers are being accessed simultaneously.

Behavior conf:1.00 **Q:** Is ca_ssh_bpp_selftest safe to call from an ISR or a multi-threaded environment?
A: No, it is not thread-safe and should not be called from an ISR. It is a synchronous, non-blocking function that performs extensive stack operations and uses standard output for reporting, which would interfere with ISR execution.

`ca_ssh_service_init`

Header: `caos/net/inc/ssh_service.h:46`
Source: `caos/net/src/ssh_transport.c:638:665`
GUARD: `EN_SSH_SERVICE`

```
bool ca_ssh_service_init(u16 port)
```

Initializes and starts the SSH server task to listen for incoming secure shell connections.

Parametri:

Nome	Tipo	Descrizione
port	u16	The TCP port to listen on. If 0 is passed, the system defaults to the standard SSH port (usually 22).

Ritorna:
Returns true if the service was successfully initialized or was already running. Returns false if critical hardware dependencies (like the RNG) failed to initialize.

Note:

- This function is not thread-safe and should be called during system initialization before the scheduler is fully saturated with user tasks. It is not callable from an ISR because it performs memory operations and task creation. It relies on ca_hash_init, ca_keys_store_init, and ca_rng_init being functional; if the hardware RNG fails, the service will refuse to start to prevent insecure key generation. A common mistake is calling this before the network stack is ready, which may lead to the SSH task failing to bind to the network interface.

Uso:

- This function is typically called once during the system boot sequence after the network stack (ca_net_proto_init) and TCP service (ca_tcp_service_start) have been brought up. It pairs with ca_ssh_service_stop if the service needs to be decommissioned at runtime. It is essential to ensure that the underlying hardware peripherals required for cryptography are initialized before this function is invoked.

Esempio:

```
/* Initialize the network stack first */
ca_net_proto_init(NULL);

/* Start the TCP service to handle socket connections */
ca_tcp_service_start();

/* Initialize the SSH server on the default port */
if (!ca_ssh_service_init(0)) {
    ca_stdout_printf_n("[ERROR] SSH service failed to start");
} else {
    ca_stdout_printf_n("[INFO] SSH service is now listening");
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to call ca_ssh_service_init during the boot sequence?
A: It should be called during system initialization, specifically after the network stack (ca_net_proto_init) and the TCP service (ca_tcp_service_start) have been brought up, but before the scheduler is fully saturated with user tasks.

Code example conf:1.00 **Q:** Provide a minimal C code snippet to initialize the SSH service on the default port.
A: u16 port = 0; if (!ca_ssh_service_init(port)) { /* Handle initialization failure */ }

Parameters conf:1.00 **Q:** What does the port parameter represent and what happens if I pass 0?
A: The port parameter specifies the TCP port for the SSH server. Passing 0 instructs the system to use the standard SSH port, which is typically 22.

Return conf:1.00 **Q:** What do the boolean return values of ca_ssh_service_init indicate?
A: Returning true indicates the service was successfully initialized or was already running. Returning false indicates that critical hardware dependencies, such as the RNG, failed to initialize.

Error/Edge case conf:1.00 **Q:** Why might ca_ssh_service_init fail to start even if the code is correct?
A: Common causes include failure of the hardware RNG, which prevents secure key generation, or calling the function before the network stack is ready, which prevents the task from binding to the network interface.

Behavior conf:1.00 **Q:** Is ca_ssh_service_init safe to call from an ISR or concurrently from multiple threads?
A: No, it is not thread-safe and must not be called from an ISR because it performs memory operations and task creation. It must be called during the initial system boot sequence.

`ca_ssh_service_stop`

Header: `caos/net/inc/ssh_service.h:51`
Source: `caos/net/src/ssh_transport.c:667:680`
GUARD: `EN_SSH_SERVICE`

```
1 void ca_ssh_service_stop(void)
```

Gracefully shuts down the SSH service by closing active connections and stopping the listener.

Ritorna:
void

Note:

- This function is not thread-safe and should be called from a controlled context, typically the main application thread or a management task. It performs a clean teardown by first unlistening the TCP port to prevent new connections, then closing any existing active session. It is not intended to be called from an ISR because it performs network stack operations that may involve blocking or complex resource management. A common mistake is calling this function while the SSH task is still actively processing data, which could lead to race conditions if the task is not properly synchronized or terminated first.

Uso:

- This function is used to stop the SSH server component during system shutdown, reconfiguration, or when entering a low-power state. It is typically paired with ca_ssh_service_init,

which sets up the service, and is called when the application no longer requires remote shell access. In a typical lifecycle, you would call this to release network resources (TCP handles) before re-initializing the service or putting the network interface into a standby mode.

Esempio:

```
/* Example of stopping the SSH service during a system shutdown sequence */

void shutdown_system_services(void) {
    /* Stop the SSH service to free up TCP resources */
    ca_ssh_service_stop();

    /* Perform other cleanup tasks */
    /* ca_net_link_down(); // Hypothetical call to disable network link */

    ca_printf("SSH service stopped successfully.\n");
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_ssh_service_stop in my caOS application?
A: You should call this function during system shutdown, when reconfiguring the system, or when entering a low-power state. It is used to release network resources like TCP handles when remote shell access is no longer required, typically paired with ca_ssh_service_init.

Code example conf:0.90 **Q:** How do I implement a clean shutdown of the SSH service in my main loop?
A: Ensure the SSH task is synchronized or terminated first, then invoke the function: void shutdown_system(void) { /* Ensure SSH task is not processing data */ ca_ssh_service_stop(); }

Error/Edge case conf:1.00 **Q:** Can I call ca_ssh_service_stop from an interrupt service routine (ISR)?
A: No, you must not call this function from an ISR. It performs network stack operations that may involve blocking or complex resource management. Additionally, it is not thread-safe and should be called from a controlled context like the main application thread or a management task.

`ca_ssh_test_init`

Header: `caos/net/inc/ssh_service.h:58`
Source: `caos/net/src/ssh_transport.c:626:634`
GUARD: `EN_SSH_SERVICE`

```
void ca_ssh_test_init(void)
```

Registers the SSH test command-line interface (CLI) node into the system.

Ritorna:
void

Note:

- This function is a wrapper that ensures the SSH test CLI node is registered exactly once. It uses a static boolean flag to prevent multiple registrations, which would corrupt the internal CLI linked list. It is not thread-safe by itself, so it should be called during the system initialization phase before the scheduler starts or within a controlled startup

sequence. It is not intended to be called from an ISR. The most common mistake is calling it multiple times or assuming it initializes the entire SSH stack; it only registers the test commands for the CLI.

Uso:

- This function is used during the system boot-up sequence to enable diagnostic commands for SSH. It is typically paired with other initialization functions like ca_usb_dev_mng_init or network stack initializations. It is only active if the preprocessor macros EN_CLI_SSH, EN_SSH_TEST, and ENABLE_CLI are defined, making it a conditional feature for debugging and testing purposes.

Esempio:

```
#if EN_SSH_TEST
/* Initialize the CLI system first */
ca_cli_init();

/* Register the SSH test commands into the CLI */
ca_ssh_test_init();

/* Proceed with other system initializations */
ca_stdout_printf_n("System initialized with SSH test support");
#endif
```

Q&A — fine-tuning review *2/2 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_ssh_test_init and what are the risks of calling it multiple times?
A: You should call ca_ssh_test_init during the system initialization phase, specifically before the scheduler starts or within a controlled startup sequence. It is not thread-safe and must not be called from an ISR. Calling it multiple times is a common mistake that will corrupt the internal CLI linked list, as the function uses a static boolean flag to ensure it is registered exactly once.

Code example conf:0.95 **Q:** How do I correctly initialize the SSH test CLI in my caOS startup sequence?
A: Ensure the preprocessor macros EN_CLI_SSH, EN_SSH_TEST, and ENABLE_CLI are defined, then call the function during boot-up:
void system_init(void) { /* Initialize other stacks first */ ca_usb_dev_mng_init(); /* Register SSH test CLI node */ ca_ssh_test_init(); }

`ca_stdout_create_thread`

Header: `caos/inc/stdout.h:168`
Source: `caos/src/stdout.c:472:486`
GUARD: `EN_STDOUT`

```
u32 ca_stdout_create_thread(void)
```

Spawns a dedicated background thread to handle asynchronous stdout output processing.

Ritorna:
Returns a unique job ID (u32) on success, or 0xFFFFFFFF if the stdout system is disabled or the thread creation fails.

Note:

- This function is non-blocking as it merely registers a new task with the caOS scheduler. It is not thread-safe to call this from an ISR; it must be called from a task context during system initialization. Before calling this, you must ensure that ca_stdout_init() and ca_stdout_set_sender() have been executed, otherwise the thread will have no destination to send data to. A common mistake is calling this before the stdout system is initialized, which will result in the function returning 0xFFFFFFFF and no output being processed.

Uso:

- This function is typically used during the system boot sequence or application startup phase. It is part of the standard initialization triplet: first, initialize the hardware/buffer with ca_stdout_init(), then define the output destination with ca_stdout_set_sender(), and finally launch the background processing thread with ca_stdout_create_thread(). This allows the application to perform non-blocking logging or printing while the background thread handles the actual transmission to the peripheral.

Esempio:

```
/* Initialize the stdout system before creating the thread */
ca_stdout_init();

/* Set the output destination (e.g., to a UART or ITM) */
ca_stdout_set_sender(STDOUT);

/* Create the background thread to process stdout messages */
u32 thread_id = ca_stdout_create_thread();

if (thread_id == 0xFFFFFFFF) {
    /* Handle error: stdout thread could not be created */
    // Log error to a secondary system or enter a safe state
} else {
    /* Thread is running, safe to use ca_stdout_printf */
    ca_stdout_printf("System initialized successfully.\n");
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_stdout_create_thread() in my caOS application?
A: You should call ca_stdout_create_thread() after initializing stdout with ca_stdout_init() and setting the sender with ca_stdout_set_sender(). This function creates a dedicated thread for handling stdout output, which is only effective if g_stdout_enabled is true. It's typically used in applications that require asynchronous logging or output management on STM32 microcontrollers.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_stdout_create_thread() with caOS types?
A: u32 thread_id = ca_stdout_create_thread(); if (thread_id != 0xFFFFFFFF) { /* Thread created successfully */ } else { /* Failed to create thread */ }

Return conf:0.95 **Q:** What do the return values of ca_stdout_create_thread() indicate?
A: If ca_stdout_create_thread() returns a value other than 0xFFFFFFFF, it indicates successful thread creation and returns the thread ID. If it returns 0xFFFFFFFF, it means either stdout is not enabled (g_stdout_enabled is false) or an error occurred during thread creation, as the function internally calls ca_os_job() which may fail.

Error/Edge case conf:0.85 **Q:** What are common error conditions or edge cases when using ca_stdout_create_thread()?
A: The function fails immediately if g_stdout_enabled is false. It also fails if ca_os_job() returns an error during thread creation. Additionally, calling this function from an ISR is not allowed, and it must be called after initializing stdout with ca_stdout_init() and setting the sender with ca_stdout_set_sender().

ca_stdout_echo

Header: caos/inc/stdout.h:134
Source: caos/src/stdout.c:293:313
GUARD: EN_STDOUT

```
bool ca_stdout_echo(const char *buf, u32 len)
```

Sends a string to the stdout echo queue for logging or output processing.

Parametri:

Nome	Tipo	Descrizione
buf	const char *	Pointer to the character buffer containing data to send. Must not be NULL.
len	u32	Number of characters to send from the buffer. Must be > 0 and <= buffer size.

Ritorna:
true: Successfully sent all characters to the echo queue. false: Failed to send data due to drain not running or no stdout sender configured.

Note:

- This function is NOT blocking; it attempts to send data immediately.
- It can be called from an ISR context (using ISR-specific queue send function).
- If the queue is full, it drops the remaining characters and stops sending.
- It wakes up the stdout drain thread to process the queued data.
- This function is thread-safe when used correctly.
- Precondition: g_drain_running must be true and g_stdout_sender must not be NULL.

Uso:

- Used in CLI command handlers to echo input strings to stdout.
- Called from ISRs to forward log messages or debug data.
- Paired with stdout_thread which processes the queued data.
- Used in error handling to fall back to direct stdout output if echo fails.

Esempio:

```
/* Example: Echo a CLI command input */
char input[] = "help";
if (!ca_stdout_echo(input, strlen(input))) {
    /* Fall back to direct output if echo fails */
    STDOUT(input);
}
/* Example: ISR sends debug string */
void UART_IRQHandler(void) {
    char debug_msg[] = "UART interrupt";
    ca_stdout_echo(debug_msg, sizeof(debug_msg) - 1);
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_stdout_echo in my application?
A: You should use ca_stdout_echo when you need to send logging or CLI output to a secondary drain thread without blocking your main execution flow. It is ideal for high-speed producers that need to offload data to a slower consumer like a UART or network socket.

Code example conf:1.00 **Q:** Provide a minimal C example of how to call ca_stdout_echo.
A: const char *msg = "System Ready"; u32 len = 12; bool success = ca_stdout_echo(msg, len);

Parameters conf:1.00 **Q:** What are the constraints for the buf and len parameters in ca_stdout_echo?
A: The buf parameter must be a valid memory address pointing to the data buffer. The len parameter specifies the number of bytes to echo; if len is 0, the function performs no action.

Return conf:1.00 **Q:** What do the boolean return values of ca_stdout_echo indicate?
A: A return value of true indicates the data was successfully queued or partially queued. A return value of false indicates the system is not in a state to accept echo data, such as when the drain thread is not running or no sender is configured.

Error/Edge case conf:1.00 **Q:** Why might ca_stdout_echo return false immediately after system boot?
A: A common mistake is calling ca_stdout_echo before the stdout subsystem is fully initialized. Ensure you have called ca_stdout_init during system startup to set up the queue and global state flags.

Behavior conf:1.00 **Q:** Is ca_stdout_echo safe to call from an ISR, and does it block?
A: Yes, it is designed to be called from ISRs or high-priority contexts because it uses non-blocking queue operations. It is thread-safe regarding queue access, though it relies on global state flags like g_drain_running.

`ca_stdout_echo_dropped`

Header: `caos/inc/stdout.h:139`
Source: `caos/src/stdout.c:315:317`
GUARD: `EN_STDOUT`

```
u32 ca_stdout_echo_dropped(void)
```

Returns the total count of characters dropped by the standard output system due to buffer overflows.

Ritorna:
Returns a u32 value representing the cumulative number of characters that could not be sent to the output device because the internal buffers were full.

Note:

- This function is non-blocking and thread-safe, as it simply reads a global counter. It is safe to call from both standard tasks and Interrupt Service Routines (ISRs). It does not require any specific initialization beyond the standard caOS stdout subsystem startup. A common mistake is assuming this value is a real-time indicator of current congestion; it is a cumulative statistic, so it only tells you that drops occurred at some point in the past, not necessarily that the system is currently failing.

Uso:

- This function is primarily used for system diagnostics and health monitoring within a Command Line Interface (CLI) or a telemetry task. It is typically paired with other diagnostic functions like ca_stdout_stats or ca_memory_info to provide a snapshot of system performance. Developers use it to determine if the UART or logging output baud rate is too slow for the volume of data being generated, which helps in tuning buffer sizes or reducing log verbosity.

Esempio:

```c
/* Diagnostic task to monitor system health */
void diagnostic_task(void *pvParameters) {
    while (1) {
        u32 dropped = ca_stdout_echo_dropped();

        /* If we detect dropped characters, report it to the user */
        if (dropped > 0) {
            ca_printf_n("Warning: stdout buffer overflow! Dropped: %u", dropped);
        }

        /* Wait for 5 seconds before checking again */
        ca_os_delay_ms(5000);
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_stdout_echo_dropped in my application?
A: You should use this function for system diagnostics and health monitoring, typically within a CLI or telemetry task. It is useful for determining if your UART baud rate is too slow for the volume of data being generated, allowing you to tune buffer sizes or reduce log verbosity.

Code example conf:1.00 **Q:** How do I implement a simple diagnostic check using ca_stdout_echo_dropped?
A: You can call the function directly to retrieve the cumulative drop count. Example: { u32 dropped_count = ca_stdout_echo_dropped(); if (dropped_count > 0) { /* handle diagnostic logic */ } }

Return conf:1.00 **Q:** What does the u32 value returned by ca_stdout_echo_dropped represent, and what causes it to increment?
A: The returned u32 value is a cumulative count of characters that were dropped by the standard output system. It increments when the internal stdout buffers are full and cannot accept new data, preventing the system from sending those characters to the output device.

`ca_stdout_flush`

Header: `caos/inc/stdout.h:119`
Source: `caos/src/stdout.c:345:347`
GUARD: `EN_STDOUT`

```c
void ca_stdout_flush(void)
```

Clears all pending output in the stdout queue without waiting, used to discard buffered console output before sending new data.

Ritorna:
None. This function does not return any value.

Note:

- This function is NOT blocking - it immediately resets the queue. It is NOT thread-safe if multiple threads access stdout concurrently. It is NOT callable from an ISR. It assumes the stdout queue has been initialized before calling. Common mistake: calling this without ensuring the queue is properly set up, which may lead to undefined behavior or crashes.

Uso:

- Used before writing new console output to ensure no stale or buffered data interferes with new messages. Typically called before printing critical startup messages or when switching output destinations (e.g., from UART to USB CDC). Pairs with functions that write to stdout like ca_stdout_put_n() or ca_stdout_printf_n().

Esempio:

```c
/* Example: Clearing stdout buffer before printing a banner message */
void print_banner(void) {
    ca_stdout_flush(); // Clear any pending output
    ca_stdout_printf_n("Starting system..."); // Print new message
    ca_stdout_put_n('\n');
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_stdout_flush() and what is the risk of calling it during active output?
A: You should call ca_stdout_flush() to reset the console output queue and free up buffer space, typically when you want to discard pending messages. However, calling it during an active output stream may cause data loss because the function resets the queue, discarding all messages still in the buffer. It is also blocking if another thread is using the queue, and it must not be called from an ISR.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_stdout_flush() with caOS types?
A: void example_function(void) { ca_stdout_flush(); }

Parameters conf:0.90 **Q:** Does ca_stdout_flush() have any parameters, and what happens if I pass invalid values?
A: ca_stdout_flush() takes no parameters. Since there are no parameters, there are no invalid values to consider. The function operates on the global console output queue and does not accept any input.

`ca_stdout_get_repeater`

Header: `caos/inc/stdout.h:161`
Source: `caos/src/stdout.c:336:338`
GUARD: `EN_STDOUT`

```c
sender ca_stdout_get_repeater(void)
```

Retrieves the current stdout repeater handle used to broadcast system output to multiple destinations.

Ritorna:
Returns a 'sender' handle (a pointer-like object) representing the active repeater. Returns NULL if no repeater has been configured, meaning stdout data is not being duplicated to secondary outputs.

Note:

- This function is non-blocking and thread-safe as it simply returns a stored pointer. It is safe to call from an ISR, though it is rarely necessary to do so. The 'repeater' is a mechanism that allows the system to send stdout logs to multiple interfaces simultaneously (e.g., UART and a network socket). A common mistake is assuming that a non-NULL return value guarantees that the destination is currently connected or functional; it only confirms that a repeater object is registered in the system. You must ensure ca_stdout_init has been called during system startup before relying on this handle.

Uso:

- This function is typically used by diagnostic or logging modules that need to inspect or manually inject data into the stdout stream. It is often paired with ca_stdout_set_repeater to configure the output routing during the initialization phase of the application. Developers use this to verify if a repeater is active before attempting to perform complex logging operations that might otherwise be dropped if no output sink is configured.

Esempio:

```c
/* Example: Checking if a repeater is active before logging */
#include "caos.h"

void check_and_log(void) {
    sender my_repeater = ca_stdout_get_repeater();

    if (my_repeater != NULL) {
        /* Repeater is configured, safe to proceed with logging */
        ca_stdout_printf("System status: Repeater is active.\n");
    } else {
        /* No repeater, perhaps log to a local buffer instead */
        /* Perform local memory logging using ca_queue_heap_send */
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_stdout_get_repeater in my application?
A: You should call this function when you need to inspect or interact with the active output stream, such as in system diagnostic tasks or custom logging drivers. It is commonly used to verify if a logger is attached before performing complex formatting or to dynamically change routing when paired with ca_stdout_set_repeater.

Code example conf:1.00 **Q:** How do I check if the stdout repeater is active using caOS types?
A: You can retrieve the handle and verify it is not NULL as shown: sender my_repeater = ca_stdout_get_repeater(); if (my_repeater != NULL) { /* Repeater is configured */ }

Return conf:1.00 **Q:** What does the return value of ca_stdout_get_repeater indicate, and why might it return NULL?
A: The function returns a 'sender' handle representing the active repeater. It returns NULL if no repeater is currently configured. If no repeater or sender is configured, the stdout system may disable itself to prevent memory waste, which causes logs to vanish.

`ca_stdout_get_sender`

Header: `caos/inc/stdout.h:57`
Source: `caos/src/stdout.c:108:110`
GUARD: `EN_STDOUT`

```c
sender ca_stdout_get_sender(void)
```

Retrieves the current sender object responsible for handling standard output (stdout) data.

Ritorna:
Returns a 'sender' handle (a pointer or identifier) representing the current output destination. Returns NULL if no sender is currently configured or if stdout is disabled.

Note:

- This function is non-blocking and thread-safe, as it simply returns the value of a global variable. It is safe to call from both tasks and Interrupt Service Routines (ISRs). The primary precondition is that the stdout system must have been initialized via ca_stdout_init before this function can return a meaningful sender. A common mistake is assuming the returned sender is always valid; always check if the result is NULL before attempting to use it in functions like ca_stdout_printf or other communication routines.

Uso:

- This function is typically used when you need to temporarily redirect or capture stdout, such as when implementing a CLI capture feature or logging system. You use it to save the current sender state, perform your task (like redirecting output to a buffer), and then restore the original sender using ca_stdout_set_sender. It is frequently paired with ca_stdout_set_sender to manage output routing dynamically during runtime.

Esempio:

```c
/* Example: Temporarily capturing stdout to a buffer */
#include "caos.h"

void capture_stdout_example(void) {
    /* Save the current stdout sender */
    sender original_sender = ca_stdout_get_sender();

    /* Set a new sender (e.g., a custom buffer sender) */
    /* Note: ca_stdout_set_sender(new_sender) would be called here */

    ca_stdout_printf("This message is captured!\n");

    /* Restore the original sender to resume normal operation */
    ca_stdout_set_sender(original_sender);

    /* Verify the sender was restored */
    if (ca_stdout_get_sender() == original_sender) {
        ca_stdout_printf("Stdout restored successfully.\n");
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_stdout_get_sender in my caOS application?
A: You should use this function when you need to dynamically manage or redirect stdout, such as when implementing a CLI capture feature or a custom logging system. It is typically used to save the current sender state before redirecting output, allowing you to restore the original configuration later using ca_stdout_set_sender.

Code example conf:1.00 **Q:** How do I safely retrieve and store the current stdout sender in C?
A: You should retrieve the sender and verify it is not NULL before use. Example: sender current_sender = ca_stdout_get_sender(); if (current_sender != NULL) { /* proceed with operations */ }

Return conf:1.00 **Q:** What does the return value of ca_stdout_get_sender represent, and why might it return NULL?
A: The function returns a 'sender' handle representing the current output destination. It returns NULL if the stdout system has not been initialized via ca_stdout_init, if stdout is disabled, or if no sender is currently configured.

`ca_stdout_init`

Header: `caos/inc/stdout.h:45`
Source: `caos/src/stdout.c:97:101`

GUARD: `EN_STDOUT`

```
void ca_stdout_init(void)
```

Initializes the internal stdout queue and resets the global output state.

Ritorna:
This function returns void and does not provide a status code.

Note:

- This function is not thread-safe and must be called during the system initialization phase, typically before the RTOS scheduler starts or before any tasks attempt to print. It performs a static allocation of the internal queue, meaning it prepares the memory buffers required for buffering log messages. A common mistake is calling this function multiple times or after the stdout thread has already been created, which could lead to memory corruption or inconsistent state in the queue. It is not designed to be called from an ISR.

Uso:

- This function is the mandatory first step in the stdout subsystem lifecycle. It is typically followed by ca_stdout_set_sender to define the physical output interface (like UART or ITM) and ca_stdout_create_thread to spawn the background task that processes the queue. You should call this once during your system configuration sequence, usually inside the ca_os_configure function, to ensure the logging infrastructure is ready to receive data.

Esempio:

```
void ca_os_configure(void) {
    /* Initialize the stdout subsystem before creating its thread */
    ca_stdout_init();

    /* Set the output destination to the predefined STDOUT interface */
    ca_stdout_set_sender(STDOUT);

    /* Create the background thread that handles queue processing */
    ca_stdout_create_thread();

    /* Now it is safe to use ca_printf or other logging functions */
}
```

Q&A — fine-tuning review *2/2 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_stdout_init() in my caOS application, and why is it important to call it before creating the stdout thread?
A: You should call ca_stdout_init() early in your caOS application, typically before creating the stdout thread. This function initializes the standard output queue for logging and message printing, preparing it for use in an RTOS environment with STM32. It must be called before creating the stdout thread because the function initializes a static queue and resets counters, but does not require the stdout thread to exist yet. Calling it after the stdout thread is created may lead to undefined behavior.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_stdout_init() with caOS types that demonstrates its usage?
A: void example(void) { ca_stdout_init(); // At this point, the stdout queue is initialized and ready for use. // The stdout thread should be created after this initialization. }

`ca_stdout_printf`

Header: `caos/inc/stdout.h:67`
Source: `caos/src/stdout.c:334:352`

GUARD: `EN_STDOUT`

```
u16 ca_stdout_printf(const char *fmt, ...)
```

Formats and sends a string to the system standard output queue for logging or debugging.

Parametri:

Nome	Tipo	Descrizione
`fmt`	`const char *`	A standard C format string (like printf). Must not be NULL.
`...`	`variadic`	Variable arguments corresponding to the format specifiers in fmt.

Ritorna:
Returns the number of bytes written to the queue (u16). Returns 0 if stdout is disabled, if the scheduler is not running (or if the fallback sender is triggered), or if the formatting fails.

Note:

- This function is thread-safe and uses an internal buffer to format the string before sending it to the stdout queue. It is NOT safe to call from an ISR because it relies on the scheduler context to manage the queue; if called from an ISR, it will attempt to use a fallback sender or fail silently. The function is blocking in the sense that it waits for the queue to accept the message, but it does not block the CPU indefinitely. A common mistake is to pass a format string that exceeds the internal buffer size (DEFAULT_MSG_SIZE), which may result in truncated output. Always ensure the stdout system is enabled before calling, otherwise, the function will return 0 immediately.

Uso:

- Used primarily for system logging, debugging, and status reporting. It is typically used after calling ca_stdout_init to set up the output channel. In a typical application, you initialize the system, create a stdout thread using ca_stdout_create_thread, and then use ca_stdout_printf throughout your tasks to log events. It is often paired with ca_stdout_flush if you need to ensure all pending logs are processed before a system reset or critical operation. It is frequently used to report task-specific metrics or initialization results.

Esempio:

```
/* Initialize the stdout system before use */
ca_stdout_init();

/* Create a dedicated thread to process the stdout queue */
ca_stdout_create_thread();

void task_monitor(void *arg) {
    u32 heap_free = ca_memory_get_avail_memory();

    /* Log system status using formatted output */
    ca_stdout_printf("[MONITOR] Available heap: %u bytes\n", heap_free);

    if (heap_free < 1024) {
        ca_stdout_printf("[WARNING] Low memory condition detected!\n");
        /* Ensure logs are sent before potential system halt */
        ca_stdout_flush();
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_stdout_printf in a caOS application?
A: You should use ca_stdout_printf for debugging purposes in a caOS application, especially when you need to output formatted messages to stdout. It is useful in contexts where you want to log information from tasks or during system initialization, but it should not be used in interrupt service routines (ISRs) due to potential deadlock risks.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_stdout_printf with caOS types?
A:

```
u16 len = ca_stdout_printf("Task ID: %d, Counter: %u\n", 123, 456);
if (len == 0) {
    // Handle error or full queue
}
```

Parameters conf:0.90 **Q:** What are the valid and invalid values for the fmt parameter in ca_stdout_printf?
A: The fmt parameter must be a constant string literal, not NULL. It should contain valid format specifiers like %d, %s, %x, etc. Invalid values include NULL pointers or non-constant strings, which may lead to undefined behavior or crashes.

Return conf:0.95 **Q:** What does ca_stdout_printf return, and what causes it to return 0?
A: ca_stdout_printf returns a u16 indicating the number of characters written to the output queue. It returns 0 if stdout is disabled, the scheduler context is invalid, or if the output queue is full and the function cannot enqueue the message.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_stdout_printf?
A: Common mistakes include passing incorrect argument types that do not match format specifiers, using non-constant strings for fmt, or calling the function in ISR contexts. These can lead to crashes, corrupted output, or undefined behavior.

Behavior conf:0.90 **Q:** Is ca_stdout_printf blocking or non-blocking, and can it be called from an ISR?
A: ca_stdout_printf is non-blocking. It is not safe to call it from an ISR, as doing so may cause deadlock or undefined behavior. It is thread-safe only if the underlying RTOS and output queue are properly configured for thread safety.

`ca_stdout_printf_n`

Header: `caos/inc/stdout.h:74`
Source: `caos/src/stdout.c:354:377`
GUARD: `EN_STDOUT`

```
u16 ca_stdout_printf_n(const char *fmt, ...)
```

Formats a string and sends it to the standard output stream with an automatic newline appended.

Parametri:

Nome	Tipo	Descrizione
`fmt`	`const char *`	Format string following standard printf conventions. Must not be NULL.
`...`	`variadic`	Variable arguments corresponding to the format specifiers in fmt.

Ritorna:
u16 representing the total number of bytes written to the output buffer, including the appended newline characters. Returns 0 if stdout is disabled or if the function is called from an invalid context (e.g., before the scheduler is running).

Note:

- This function is thread-safe and uses a local stack buffer to format the string before dispatching it to the stdout handler. It is NOT safe to call from an ISR because it checks the scheduler context; calling it from an interrupt will trigger a safety mechanism (PUT_N) and return 0. A common beginner mistake is to call this function before the RTOS scheduler has started or while the system is in a critical section where the scheduler is suspended. Ensure ca_stdout_init has been called during system startup, otherwise, output will be silently dropped. The function automatically appends '\r\n' to the formatted string, provided the resulting length does not exceed the internal buffer capacity.

Uso:

- Typically used for logging, debugging, or reporting test results in a human-readable format. It is often paired with ca_stdout_init during system initialization to prepare the output channel. In test suites, it is frequently used to report pass/fail status of cryptographic self-tests like ca_ed25519_selftest or ca_sha512_selftest to the console. It is designed for task-level code where formatted output is required for monitoring system health or verifying algorithm integrity.

Esempio:

```c
/* Initialize stdout before use */
ca_stdout_init();

/* Perform a self-test and report results */
u32 fails = 0;
/* Assuming ca_sha512_selftest returns 0 on success */
if (ca_sha512_selftest() != 0) {
    fails++;
}

/* Log the result to the console using the _n variant for automatic newline */
ca_stdout_printf_n("SHA512 Self-test status: %s", (fails == 0) ? "PASS" : "FAIL");

/* If an error occurred, log additional details */
if (fails > 0) {
    ca_stdout_printf_n("Critical error: %d tests failed", fails);
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_stdout_printf_n in a caOS application?
A: You should use ca_stdout_printf_n when you need to print formatted debug or log messages to stdout. It is suitable for both task contexts (where it will block) and ISR contexts (where it will not block), provided stdout is configured for thread safety. It is commonly used in network services like DHCP, TCP, and iperf for logging.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_stdout_printf_n with caOS types?
A: u16 len = ca_stdout_printf_n("Task ID: %d, Value: %x\n", 123, 0xABCD);

Parameters conf:0.90 **Q:** What is the meaning of the fmt parameter in ca_stdout_printf_n, and what are valid values?
A: The fmt parameter is a format string similar to printf, containing specifiers like %d, %s, %x. It must be a valid, non-NULL string. Invalid values include NULL pointers or malformed format strings that do not match the provided variadic arguments.

Return conf:0.95 **Q:** What does ca_stdout_printf_n return, and what causes it to return 0?
A: It returns the number of characters written to stdout. It returns 0 if stdout is disabled (g_stdout_enabled is false), or if the function fails due to invalid context or other internal errors.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_stdout_printf_n?
A: Common mistakes include using format specifiers that do not match the argument types, which can cause incorrect output or crashes. Another mistake is calling the function from an ISR without ensuring stdout is configured for thread safety, which may lead to undefined behavior.

Behavior conf:0.90 **Q:** How does ca_stdout_printf_n behave in terms of blocking, ISR safety, and thread safety?
A: It is blocking when called from a task because it uses ca_stdout_send(), which may block. It is non-blocking when called from an ISR, but only if stdout is configured for thread safety. It is not thread-safe because it uses a local buffer and accesses shared stdout resources.

`ca_stdout_put`

Header: `caos/inc/stdout.h:78`
Source: `caos/src/stdout.c:166:185`
GUARD: `EN_STDOUT`

```
u16 ca_stdout_put(const char *msg)
```

Sends a string to the system standard output (stdout) for logging or debugging purposes.

Parametri:

Nome	Tipo	Descrizione
`msg`	`const char *`	Pointer to the null-terminated string to be sent. Must not be NULL.

Ritorna:
Returns the number of characters successfully queued for transmission (u16). Returns 0 if stdout is disabled, the input string is NULL, or if called from an invalid context (e.g., before the scheduler is running).

Note:

- This function is thread-safe and intended for use within tasks. It is NOT safe to call from an Interrupt Service Routine (ISR) because it relies on the scheduler context; calling it from an ISR will trigger an internal error handler. The function is non-blocking in the sense that it queues the message and returns immediately, but it relies on the underlying stdout thread to process the queue. A common beginner mistake is attempting to log messages before the system has initialized the stdout subsystem or calling it from an ISR context, which will cause the system to report an error.

Uso:

- Used primarily for system diagnostics, status reporting, and debugging. It is typically used after the system has initialized the stdout thread via ca_stdout_init. In power management scenarios, it is often paired with ca_os_sleep to ensure the output queue is flushed before entering low-power modes, preventing data loss during sleep transitions.

Esempio:

```
/* Ensure stdout is initialized before use */
ca_stdout_init();

/* Standard usage for logging status */
if (ca_os_running()) {
    u16 sent = ca_stdout_put("System status: OK\r\n");
    if (sent == 0) {
```

```
        /* Handle error: message not queued */
    }
}

/* Example of flushing before power down */
ca_stdout_put("Entering low power mode...\r\n");
ca_os_sleep(100); /* Allow time for the stdout thread to empty the queue */
/* Call function to enter stop mode */
ca_pwr_enter(0, 0, 0); /* Parameters depend on specific hardware requirements */
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_stdout_put() in my caOS application?
A: You should use ca_stdout_put() to send debug messages or logs to the console in a non-blocking way. It is especially useful in task contexts or ISRs where you want to output information without blocking execution. It is ideal for debugging or logging in real-time systems.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_stdout_put() in a caOS task?
A: cstr msg = "Hello from caOS task!\n"; u16 chars_sent = ca_stdout_put(msg); if (chars_sent > 0) { // Message sent successfully } else { // Failed to send message }

Parameters conf:0.95 **Q:** What is the meaning of the msg parameter in ca_stdout_put() and what are valid values?
A: The msg parameter is a pointer to a null-terminated string that will be sent to stdout. It must not be NULL. If NULL is passed, the function returns 0 immediately without sending anything.

Return conf:0.90 **Q:** What does ca_stdout_put() return and what causes it to return 0?
A: The function returns 0 if stdout is disabled (g_stdout_enabled = false), if msg is NULL, or if the function is called in an invalid context (e.g., scheduler not active). It returns a u16 > 0 indicating the number of characters sent if successful.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_stdout_put()?
A: Common mistakes include calling the function with a NULL msg parameter, or calling it before the RTOS is initialized. Calling it in an invalid context (e.g., outside of a task or ISR with no valid scheduler state) will also cause it to return 0 and may invoke PUT_N(&priv_e1).

Behavior conf:0.90 **Q:** Is ca_stdout_put() blocking or non-blocking, and can it be called from an ISR?
A: ca_stdout_put() is non-blocking. It sends the message asynchronously via an internal buffer and returns immediately. It can be called from an ISR, but only if the scheduler context is valid. If not, it returns 0 and signals an error via PUT_N(&priv_e1).

`ca_stdout_put_n`

Header: `caos/inc/stdout.h:85`
Source: `caos/src/stdout.c:187:208`
GUARD: `EN_STDOUT`

```
u16 ca_stdout_put_n(const char *msg)
```

Sends a null-terminated string to the standard output stream with an appended newline.

Parametri:

Nome	Tipo	Descrizione
`msg`	`const char *`	Pointer to the null-terminated string to be sent. Must not be NULL.

Ritorna:
Returns the number of bytes sent (including the newline characters) as a u16. Returns 0 if the message is NULL, stdout is disabled, or if the function is called before the OS scheduler has started.

Note:

- This function is thread-safe and designed for use within tasks. It is NOT safe to call from an ISR (Interrupt Service Routine) because it uses local stack buffers and relies on the scheduler context; calling it from an ISR will trigger a fallback mechanism (PUT_N) which may not be thread-safe or reentrant. The function is blocking in the sense that it prepares the message and hands it to the stdout driver, but it does not wait for the physical transmission to complete. A common beginner mistake is passing a pointer to a string that is about to go out of scope or is located in volatile memory that might change before the driver processes the buffer.

Uso:

- This function is primarily used for logging, debugging, and reporting test results during system operation. It is typically paired with initialization functions like ca_stdout_init or ca_stdout_create_thread to set up the output channel. In a typical sequence, you would initialize the stdout system at boot, then use ca_stdout_put_n throughout your application tasks to output status updates or error messages. It is often wrapped in macros like EVAL_STDOUT to ensure that debug prints are only compiled into the binary when debugging is enabled.

Esempio:

```c
/* Example: Logging a test result in a task */
#include "caos.h"

void run_system_test(void) {
    /* Ensure stdout is initialized before use */
    ca_stdout_init();

    /* Perform a self-test */
    u32 fails = 0;
    ca_stdout_put_n("Starting system self-test...");

    /* Perform SHA512 test */
    if (ca_sha512_selftest() != 0) {
        ca_stdout_put_n("SHA512 test failed!");
        fails++;
    } else {
        ca_stdout_put_n("SHA512 test passed.");
    }

    /* Final report */
    if (fails == 0) {
        ca_stdout_put_n("System test completed successfully.");
    } else {
        ca_stdout_put_n("System test failed with errors.");
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_stdout_put_n() in a caOS application?

A: You should use ca_stdout_put_n() to send a string message to the standard output (e.g., serial console or log) in a caOS application. It is typically used for debugging or logging information from tasks or ISRs. It is especially useful in network services like DHCP where status messages need to be output.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_stdout_put_n() with caOS types?
A: cstr msg = "Hello, caOS!\r\n"; u16 count = ca_stdout_put_n(msg); // count will be the number of characters sent including \r\n

Parameters conf:0.95 **Q:** What does the msg parameter in ca_stdout_put_n() represent, and what are valid values?
A: The msg parameter is a pointer to a null-terminated string that will be sent to stdout. It must be a valid pointer (not NULL). If NULL is passed, the function returns 0 immediately without sending anything.

Return conf:0.95 **Q:** What do the return values of ca_stdout_put_n() mean, and what causes it to return 0?
A: A return value of 0 means the function failed. This can happen if msg is NULL, stdout is not enabled (g_stdout_enabled is false), or if the function is called from an invalid context (e.g., ISR without proper context). Otherwise, it returns the number of characters sent, including \r\n.

Error/Edge case conf:0.90 **Q:** What are common mistakes or edge cases when using ca_stdout_put_n()?
A: Common mistakes include passing NULL as the msg parameter, calling the function before initializing stdout, or using it from an ISR without ensuring a valid context. Also, calling it from a task in a way that causes unexpected blocking if ca_stdout_send() is not designed for task blocking.

Behavior conf:0.90 **Q:** How does ca_stdout_put_n() behave in terms of blocking, ISR safety, and thread safety?
A: If called from a task, the function may block depending on the internal implementation of ca_stdout_send(). If called from an ISR, it must not block. It is thread-safe only if ca_stdout_send() is thread-safe. It requires g_stdout_enabled to be true and msg to be a valid pointer. If these preconditions are not met, it returns 0.

`ca_stdout_put_str`

Header: `caos/inc/stdout.h:95`
Source: `caos/src/stdout.c:439:447`
GUARD: `EN_STDOUT`

```
u16 ca_stdout_put_str(p_str str)
```

Sends a string to the system standard output (stdout) for logging or debugging purposes.

Parametri:

Nome	Tipo	Descrizione
`str`	`p_str`	A pointer to a string structure containing the data to be sent. Must not be NULL.

Ritorna:
Returns the size of the string (u16) if successfully queued for transmission, or 0 if the stdout system is disabled, the input pointer is NULL, or the function is called from an invalid context (e.g., before the scheduler is running).

Note:

- This function is a wrapper for stdout operations and is intended for use within task contexts. It checks if the scheduler is currently running; if called too early (before the OS is fully initialized), it will fail safely by returning 0. It is not designed to be called directly from an Interrupt Service Routine (ISR) because it relies on the scheduler context to manage the transmission queue. A common mistake is attempting to log messages during

system startup before calling ca_stdout_init or before the scheduler has started, which will result in silent failures. If the scheduler is not running, it attempts to use a fallback mechanism (priv_e1_put_str) to ensure critical startup messages are not lost.

Uso:

- This function is typically used for logging application status, debugging variable values, or reporting errors to a console or terminal. It is usually paired with ca_stdout_init during system startup to configure the output interface. In a typical workflow, you would allocate a string buffer, populate it with data, call ca_stdout_put_str to send it, and then potentially free the buffer if it was dynamically allocated. It is the standard way to output formatted text from within a running task.

Esempio:

```c
/* Initialize stdout before use */
ca_stdout_init();

/* Assume 'my_str' is a valid p_str structure containing 'Hello World' */
if (ca_stdout_put_str(&my_str) == 0) {
    /* Handle error: stdout might be disabled or context is invalid */
    /* Use a fallback mechanism or log to internal system memory */
}

/* Example of usage within a task loop */
void my_task(void *pvParameters) {
    while(1) {
        /* Perform some work */
        ca_os_delay_ms(1000);

        /* Send status update */
        if (ca_stdout_put_str(&status_msg) == 0) {
            /* Log failure to send string */
        }
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_stdout_put_str in a caOS application?
A: You should use ca_stdout_put_str when you need to output a string to the standard output in a blocking manner, provided that the scheduler is active and stdout is enabled. It is intended for debugging or logging purposes in a thread-safe environment where the scheduler is running.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_stdout_put_str with caOS types?
A:

```c
p_str msg;
msg.str = "Hello, caOS!";
msg.size = 12;
ca_stdout_put_str(&msg);
```

Parameters conf:0.90 **Q:** What does the 'str' parameter represent, and what are valid values for it?
A: The 'str' parameter is a pointer to a structure containing the string to be sent. It must not be NULL, and the structure must be properly initialized with the 'size' field set to the length of the string. Invalid values include NULL pointers or improperly initialized structures.

Return conf:0.95 **Q:** What do the return values of ca_stdout_put_str indicate, and what causes it to return 0?

A: The function returns 0 if there is an error, which occurs when stdout is not enabled, the 'str' pointer is NULL, or if the function is called from an ISR or in a non-scheduler context. It returns the string length if the operation succeeds.

`ca_stdout_put_str_n`

Header: `caos/inc/stdout.h:99`
Source: `caos/src/stdout.c:210:224`
GUARD: `EN_STDOUT`

```
u16 ca_stdout_put_str_n(p_str str)
```

Appends a newline sequence (\r\n) to a string buffer and transmits it via the standard output interface.

Parametri:

Nome	Tipo	Descrizione
`str`	`p_str`	Pointer to a structure containing the string data and its current size. Must be non-NULL and have sufficient buffer space to accommodate two additional characters (\r and \n).

Ritorna:
Returns the new size of the string (original size + 2) on success, or 0 if the stdout system is disabled, the pointer is NULL, or the system is in an invalid context.

Note:

- This function is not thread-safe if multiple tasks attempt to modify the same string buffer simultaneously. It is designed to be called from task context; if called when the scheduler is not running (e.g., during early boot), it attempts a fallback output mechanism. A common beginner mistake is passing a string buffer that is already full, which causes a buffer overflow when the function appends the \r\n characters. Always ensure your buffer size is at least 2 bytes larger than the maximum expected string length.

Uso:

- This function is typically used for logging or debugging purposes where a formatted line needs to be sent to the console. It is often paired with string manipulation functions to build a message, followed by this function to finalize the line with a carriage return and newline. In a typical sequence, you would initialize the stdout system with ca_stdout_init, prepare your data in a buffer, and then call this function to transmit the result. It is frequently used in diagnostic routines, such as printing memory dumps or status reports.

Esempio:

```c
/* Example: Logging a memory status report */
#include "caos.h"

void log_memory_status(void) {
    p_str my_str = /* Assume this is a valid pointer to a string buffer */;

    /* Prepare a string with memory info */
    /* Use ca_snprintf to format the string into my_str->data */
    /* Assume my_str->size is updated after formatting */

    if (my_str != NULL) {
        /* Append \r\n and send to stdout */
        u16 new_size = ca_stdout_put_str_n(my_str);
```

```
14
15          if (new_size == 0) {
16              /* Handle error: stdout might be disabled or buffer overflowed */
17          }
18      }
19  }
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_stdout_put_str_n() function in a caOS application?
A: You should use ca_stdout_put_str_n() when you need to send a string with a newline and carriage return to the standard output. It is suitable for logging, debugging, or printing messages from tasks. The function is blocking when called from a task context, which ensures proper synchronization of output. It is not recommended to call it from an ISR unless you are certain that the scheduler context is valid.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_stdout_put_str_n() with caOS types?
A:

```
1  p_str my_str;
2  my_str.str = "Hello, caOS!";
3  my_str.size = 13;
4  ca_stdout_put_str_n(&my_str);
```

Parameters conf:0.90 **Q:** What does the 'str' parameter in ca_stdout_put_str_n() represent and what are valid values?
A: The 'str' parameter is a pointer to a structure containing the string to be sent. It must be a valid and allocated structure. The function modifies the 'size' field of the structure and appends '\r\n\0' to the string. Passing a NULL pointer will cause the function to return 0 immediately without sending anything.

Return conf:0.95 **Q:** What do the return values of ca_stdout_put_str_n() indicate and what causes a return value of 0?
A: The function returns 0 if stdout is disabled, if the 'str' parameter is NULL, or if the function is called from an invalid context (e.g., ISR without valid scheduler context). It returns the total size of the string sent, including '\r\n\0', if the operation is successful. A return value of 0 indicates either a failure condition or that the function was not executed due to invalid parameters or context.

Error/Edge case conf:0.90 **Q:** What are common mistakes or edge cases when using ca_stdout_put_str_n()?
A: Common mistakes include passing a NULL pointer to the 'str' parameter, which results in an immediate return of 0. Not properly allocating or initializing the 'str' structure can lead to crashes or undefined behavior. Calling the function from an ISR without ensuring a valid scheduler context will also return 0 and may invoke PUT_N(&priv_e1).

`ca_stdout_set_repeater`

Header: `caos/inc/stdout.h:130`
Source: `caos/src/stdout.c:186:192`
GUARD: `EN_STDOUT`

```
1  void ca_stdout_set_repeater(sender hook)
```

Registers a callback function (repeater) to intercept and process all stdout output streams.

Parametri:

Nome	Tipo	Descrizione
`hook`	`sender`	A function pointer to the repeater logic. Pass NULL to disable the repeater.

Ritorna:
void

Note:

- This function is non-blocking and thread-safe. It is used to keep the stdout system active even when no primary sender (like a UART or USB console) is connected, preventing the system from discarding output data. A common mistake is to pass a function pointer that performs heavy processing or blocking operations; since this hook is called during the stdout write path, it will block the task that is calling ca_printf. Ensure the repeater function is lightweight to avoid system-wide latency.

Uso:

- This function is typically used in logging systems or headless applications where output needs to be redirected to a file, a memory buffer, or a network socket. It is paired with ca_stdout_init during system startup to define the output behavior. When stopping a logging session, you must call this function with NULL to safely detach the repeater before closing the underlying storage or file system to prevent race conditions.

Esempio:

```c
/* Example: Redirecting stdout to a custom logging function */

void my_custom_repeater(const char *data) {
    /* This function is called whenever ca_printf is invoked */
    /* Perform custom processing, e.g., writing to a buffer */
}

void start_logging(void) {
    /* Register the repeater to start capturing logs */
    ca_stdout_set_repeater(my_custom_repeater);
}

void stop_logging(void) {
    /* Unregister the repeater before closing resources */
    ca_stdout_set_repeater(NULL);
    /* Proceed to close file or storage handles safely */
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_stdout_set_repeater and what is its primary purpose?
A: You should use this function during system startup, paired with ca_stdout_init, to intercept stdout streams. Its primary purpose is to keep the stdout system active in headless applications or logging systems, preventing data loss when no primary sender like UART or USB is connected.

Code example conf:0.95 **Q:** How do I implement a simple repeater function and register it in caOS?
A: Define a function that matches the sender hook signature and pass it to ca_stdout_set_repeater. Example: void my_repeater(cstr data) { /* process data */ } ... ca_stdout_set_repeater(my_repeater) Ensure the function is lightweight to avoid blocking the calling task.

Parameters conf:1.00 **Q:** What are the valid values for the 'sender hook' parameter and how do I disable the repeater?
A: The 'sender hook' parameter expects a function pointer to your repeater logic. To disable the repeater and safely detach it, you must pass NULL as the argument.

ca_stdout_set_sender

Header: caos/inc/stdout.h:51
Source: caos/src/stdout.c:103:106
GUARD: EN_STDOUT

```
void ca_stdout_set_sender(sender out)
```

Configures the output destination (sender) for the system standard output (stdout) stream.

Parametri:

Nome	Tipo	Descrizione
out	sender	A function pointer or handle representing the output interface (e.g., UART, ITM, or a custom buffer). Passing NULL disables stdout output.

Ritorna:
void

Note:

- This function is not thread-safe and should typically be called during system initialization before the scheduler starts or while the stdout thread is suspended. It is not intended to be called from an ISR. If you pass a NULL pointer, the system will effectively mute all subsequent calls to stdout functions, which is a common way to temporarily disable logging or debug output. Ensure the 'sender' provided is fully initialized and ready to handle data before calling this function, otherwise, the system may crash or drop data when the stdout thread attempts to use the invalid sender.

Uso:

- This function is used to route system messages to a specific hardware interface or memory buffer. It is typically called after ca_stdout_init() and before ca_stdout_create_thread(). In complex applications, it can be used to dynamically switch output targets, such as redirecting logs from a UART port to a memory capture buffer during a CLI session.

Esempio:

```
/* Initialize the stdout subsystem */
ca_stdout_init();

/* Set the output destination to the default system sender */
ca_stdout_set_sender(STDOUT);

/* Create the background thread that processes the stdout queue */
ca_stdout_create_thread();

/* Later, if we need to capture output to a buffer instead of UART */
/* ca_stdout_set_sender(capture_sender); */

/* Send a message to the configured output */
ca_printf("System initialized successfully.\n");
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_stdout_set_sender() in a caOS application?
A: You should call ca_stdout_set_sender() during the initialization phase of your caOS application, typically after setting up the hardware peripherals like UART. This function enables or disables the standard output functionality, allowing you to route stdout messages to a specific output handler such as a UART driver. It's useful when you want to redirect debug or log messages to a serial interface.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_stdout_set_sender() with caOS types?
A: void uart_send_byte(u8 byte) { /* UART transmit register */ } void my_sender(u8 byte) { uart_send_byte(byte); } int main(void) { ca_stdout_set_sender(my_sender); /* Output is now sent via UART */ return 0; }

Parameters conf:0.95 **Q:** What does the 'out' parameter in ca_stdout_set_sender() represent, and what are valid values?
A: The 'out' parameter is a pointer to a function that handles the actual sending of output data. Valid values are pointers to functions that match the signature void sender(u8), such as a UART send function. If NULL is passed, stdout is disabled, effectively stopping all output from being sent.

Return conf:0.85 **Q:** Does ca_stdout_set_sender() return any value, and what could cause it to fail?
A: The function does not return any value. It modifies an internal global variable to enable or disable stdout output. Failures are not explicitly defined, but improper usage such as calling it from an ISR or without proper initialization of the output handler may lead to undefined behavior or no output being sent.

`ca_stdout_stats`

Header: `caos/inc/stdout.h:106`
Source: `caos/src/stdout.c:112:115`
GUARD: `EN_STDOUT`

```
1 void ca_stdout_stats(u32 *queued, u32 *dropped)
```

Retrieves the current number of pending messages and dropped messages in the stdout system queue.

Parametri:

Nome	Tipo	Descrizione
`queued`	`u32 *`	Pointer to a variable where the current number of messages waiting to be sent will be stored. Pass NULL if you do not need this value.
`dropped`	`u32 *`	Pointer to a variable where the total count of messages lost due to queue overflow will be stored. Pass NULL if you do not need this value.

Ritorna:
This function returns nothing (void). The results are returned via the provided pointers.

Note:

- This function is non-blocking and thread-safe, as it performs a simple read of internal global variables. It is safe to call from an ISR, although it is typically used in application-level monitoring tasks. The most common mistake is assuming that the queue count remains constant; because other threads may be calling ca_stdout_printf simultaneously, the value retrieved is only a snapshot in time. Ensure the pointers passed are valid memory addresses to avoid hard faults.

Uso:

- This function is primarily used for system diagnostics and health monitoring of the stdout logging subsystem. It is often paired with ca_stdout_flush to clear the queue if the 'dropped' count indicates a buffer overflow, or used in a polling loop to wait for logs to be fully transmitted before performing a system shutdown or power-off sequence.

Esempio:

```
void monitor_stdout_health(void) {
    u32 pending_logs = 0;
    u32 lost_logs = 0;

    // Retrieve current status of the stdout queue
    ca_stdout_stats(&pending_logs, &lost_logs);

    if (lost_logs > 0) {
        // Log a warning about data loss if the system allows
        // ca_stdout_printf("Warning: %u logs were dropped!\n", lost_logs);
    }

    if (pending_logs > 0) {
        // Wait for the queue to drain before proceeding
        ca_os_sleep(10);
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_stdout_stats() and why would I need to know the number of queued and dropped messages?
A: You should call ca_stdout_stats() to monitor the status of the standard output queue, particularly to detect if messages are being dropped due to buffer overflow. This is useful for debugging or performance tuning of logging systems. The function is non-blocking and not thread-safe, so it's best used in a controlled environment where you can ensure no other thread modifies the stdout queue during the call.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_stdout_stats() with caOS types?
A: u32 queued, dropped; ca_stdout_stats(&queued, &dropped); // queued now contains the number of messages in the stdout queue // dropped contains the number of messages lost due to overflow

Parameters conf:0.95 **Q:** What happens if I pass NULL for one or both parameters in ca_stdout_stats()?
A: If you pass NULL for the queued parameter, the function will not write to that location and will ignore the number of queued messages. Similarly, if you pass NULL for the dropped parameter, it will not write to that location and will ignore the number of dropped messages. Passing NULL for either parameter is valid and will cause the function to only update the non-NULL pointer.

`ca_strcmp`

Header: `caos/inc/memaux.h:112`
Source: `caos/src/memaux.c:147:154`
GUARD: `EN_MEMAUX`

```
i32 ca_strcmp(const char *s1, const char *s2)
```

Compares two null-terminated strings lexicographically.

Parametri:

Nome	Tipo	Descrizione
`s1`	`const char *`	Pointer to the first null-terminated string. Must not be NULL.
`s2`	`const char *`	Pointer to the second null-terminated string. Must not be NULL.

Ritorna:
Returns 0 if strings are identical. Returns a value less than 0 if s1 is lexicographically less than s2, or greater than 0 if s1 is greater than s2.

Note:

- This function is non-blocking and thread-safe as it only performs read operations on the provided memory buffers. It is safe to call from an ISR because it does not perform any OS-level synchronization or blocking operations. The most common mistake is passing a NULL pointer, which will cause a hard fault when the function attempts to dereference it. Ensure both strings are properly null-terminated, otherwise, the function will read past the buffer until it encounters a zero byte, potentially causing a memory access violation.

Uso:

- This function is typically used for command parsing, configuration validation, or routing logic where string identifiers must be matched against a known set of values. It is frequently paired with ca_skip_sp to strip leading whitespace from user input before comparison. In a typical workflow, you receive a command string, use ca_skip_sp to find the start of the actual text, and then use ca_strcmp to branch your logic based on the command name.

Esempio:

```c
void process_command(const char *input) {
    // Skip leading spaces to find the start of the command
    const char *cmd = ca_skip_sp(input);
    if (!cmd) return;

    // Compare input against known commands
    if (ca_strcmp(cmd, "status") == 0) {
        ca_stdout_put_n("System is running normally.");
    } else if (ca_strcmp(cmd, "reset") == 0) {
        // Perform system reset logic
        ca_os_thread_reset_watchdog();
    } else {
        ca_stdout_put_n("Unknown command.");
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_strcmp in a caOS project, and what is its primary purpose?
A: You should use ca_strcmp to compare two null-terminated strings for equality or ordering. It is primarily used in network services and memory management modules where string comparison is needed, such as in HTTP or TCP service handling. It is not suitable for ISR contexts due to being blocking and non-thread-safe.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_strcmp with caOS types?
A: cstr str1 = "hello"; cstr str2 = "world"; i32 result = ca_strcmp(str1, str2); if (result == 0) { /* strings are equal */ } else if (result > 0) { /* str1 > str2 */ } else { /* str1 < str2 */ }

Parameters conf:0.95 **Q:** What are the valid and invalid values for the parameters of ca_strcmp?
A: Both parameters must be valid pointers to null-terminated strings. Passing NULL to either parameter causes undefined behavior and will likely result in a crash. The strings must be properly allocated and null-terminated for correct operation.

Return conf:0.90 **Q:** What do the return values of ca_strcmp indicate, and what causes failures?
A: ca_strcmp returns 0 if the strings are equal, a positive value if the first differing character in s1 is greater than in s2, and a negative value if it is smaller. Failures occur when NULL pointers are passed, leading to undefined behavior and potential crashes.

ca_strcpy

Header: caos/inc/memaux.h:137
Source: caos/src/memaux.c:181:190
GUARD: EN_MEMAUX

```
char* ca_strcpy(char *dest, const char *src)
```

Copies a null-terminated string from a source buffer to a destination buffer.

Parametri:

Nome	Tipo	Descrizione
dest	char*	Pointer to the destination buffer where the string will be copied. Must have enough allocated space to hold the source string plus the null terminator.
src	const char*	Pointer to the null-terminated source string to be copied.

Ritorna:
Returns the original pointer to the destination buffer (dest).

Note:

- This function is not thread-safe and does not perform bounds checking. If the destination buffer is smaller than the source string, a buffer overflow will occur, leading to memory corruption and unpredictable system behavior. It is not safe to call from an ISR if the destination memory is shared with other tasks without proper synchronization. Always ensure the destination buffer is large enough before calling this function to avoid overwriting adjacent memory regions.

Uso:

- Used for duplicating strings into buffers allocated via ca_memory_allocate or static arrays. It is typically used when preparing messages for logging, CLI output, or network packets. It is often paired with ca_strlen to verify buffer sizes before copying, or used after ca_memory_allocate to initialize a string buffer. Always ensure the destination memory is valid and accessible before invoking this function.

Esempio:

```
void process_task_name(void) {
    char *task_name = ca_memory_allocate(32);
    if (task_name != NULL) {
        // Copy a string into the allocated memory
        ca_strcpy(task_name, "SensorTask");

        // Use the string for logging or system identification
        ca_printf("Task initialized: %s\n", task_name);

        // Always free memory when done to prevent leaks
        ca_memory_free(task_name);
    } else {
        // Handle allocation failure
        ca_printf("Failed to allocate memory for task name\n");
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_strcpy in a caOS application, and what is the primary benefit of using it over manual string copying?
A: You should use ca_strcpy when you need to copy a null-terminated string from a source buffer to a destination buffer in a caOS application. The primary benefit is that it handles the copying process safely and returns a pointer to the destination, allowing for function chaining. It is typically used in memory management or string manipulation tasks, such as in caos/src/memaux.c where string operations are needed.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_strcpy with caOS types?
A: cstr dest = "Hello, World!"; cstr src = "caOS is great"; cstr result = ca_strcpy(dest, src); // result now points to dest, which contains "caOS is great".

Parameters conf:0.95 **Q:** What are the valid and invalid values for the parameters of ca_strcpy, and what happens if I pass NULL?
A: The 'dest' parameter must point to a valid buffer that is large enough to hold the source string plus the null terminator '\0'. The 'src' parameter must point to a valid null-terminated string. Passing NULL for either parameter will cause a crash, as the function does not handle NULL inputs gracefully.

Return conf:0.85 **Q:** What does ca_strcpy return, and what causes it to return NULL?
A: ca_strcpy returns a pointer to the destination buffer 'dest' if the copy is successful. It returns NULL only if one of the input parameters ('dest' or 'src') is NULL, which leads to a crash according to the documentation, but the return value is specified as NULL in case of invalid input.

`ca_stream_bytes_available`

Header: `caos/inc/stream.h:125`
Source: `caos/src/stream.c:247:250`
GUARD: `EN_STREAM`

```
u32 ca_stream_bytes_available(ca_stream_buffer_t *sb)
```

Returns the number of bytes currently stored and ready to be read in a stream buffer.

Parametri:

Nome	Tipo	Descrizione
`sb`	`ca_stream_buffer_t *`	Pointer to the initialized stream buffer handle. Must not be NULL.

Ritorna:
Returns the count of bytes (u32) currently in the buffer. Returns 0 if the buffer is empty or if the provided pointer is NULL.

Note:

- This function is non-blocking and thread-safe as it performs a simple read of the internal counter. It is safe to call from an ISR, as it does not involve any complex synchronization or waiting. The most common beginner mistake is assuming that the returned value will remain constant; in a multi-threaded environment, another task or an ISR could add data to the stream immediately after this function returns, making the value stale. Always use the returned value as a snapshot for your current logic, but do not rely on it for long-term state tracking.

Uso:

- This function is typically used in a producer-consumer pattern to check if there is enough data to justify a read operation. It is usually paired with ca_stream_receive to avoid

blocking the task unnecessarily if the buffer is empty. The standard sequence is: initialize the stream with ca_stream_init, poll the availability with ca_stream_bytes_available, and if the result is greater than zero, call ca_stream_receive to process the data.

Esempio:

```
ca_stream_buffer_t data_stream;
u8 rx_buffer[64];

// Initialize the stream before use
ca_stream_init(&data_stream, ...);

// Inside a task loop
while (1) {
    u32 available = ca_stream_bytes_available(&data_stream);

    // Only attempt to read if there is data available
    if (available > 0) {
        // Limit read to buffer size
        u32 to_read = (available > sizeof(rx_buffer)) ? sizeof(rx_buffer) :
    available;

        // Receive data with a 100ms timeout
        u32 received = ca_stream_receive(&data_stream, rx_buffer, to_read, 100);

        // Process received data here
    }

    ca_os_delay_ms(10);
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_stream_bytes_available() in my embedded application?
A: You should call ca_stream_bytes_available() when you need to check how many bytes are available for reading from a stream buffer without blocking. This is useful for determining if you can read data immediately or if you need to wait, especially in scenarios where you want to avoid blocking operations in your RTOS tasks.

Code example conf:0.90 **Q:** Can you provide a minimal example of using ca_stream_bytes_available() with caOS types?
A: ca_stream_buffer_t sb; // Assume sb is initialized u32 bytes = ca_stream_bytes_available(&sb); if (bytes > 0) { // Safe to read bytes from the stream buffer }

Parameters conf:0.90 **Q:** What is the significance of the ca_stream_buffer_t *sb parameter, and what happens if it's invalid?
A: The parameter sb is a pointer to a stream buffer. If sb is NULL or points to an uninitialized buffer, the function returns 0. This is not a fatal error, but it indicates that no valid data can be read from the buffer. The caller must ensure the buffer is properly initialized before passing it to this function.

Return conf:0.95 **Q:** What do the return values of ca_stream_bytes_available() indicate, and what causes a return value of 0?
A: The function returns 0 if the input pointer is NULL or the buffer is not initialized. It also returns 0 if the buffer is empty. Otherwise, it returns the number of bytes available for reading. A return value of 0 does not indicate an error, but rather that no data is currently available to be read.

ca_stream_destroy

Header: caos/inc/stream.h:68
Source: caos/src/stream.c:46:57
GUARD: EN_STREAM

```
void ca_stream_destroy(ca_stream_buffer_t *sb)
```

Safely deallocates the internal buffer of a stream and resets its control structure.

Parametri:

Nome	Tipo	Descrizione
sb	ca_stream_buffer_t *	Pointer to the stream buffer handle to be destroyed. Must be a valid, previously initialized pointer or NULL.

Ritorna:
void. No value is returned.

Note:

- This function is thread-safe as it uses a spinlock to protect the internal state during cleanup. It is non-blocking, but it must not be called from an ISR because it performs memory deallocation, which is generally not permitted in interrupt context. The function handles NULL pointers gracefully by returning immediately. The most common mistake is attempting to use the stream buffer handle after calling this function, which will lead to undefined behavior or memory access faults.

Uso:

- This function is used during the teardown phase of a module or task lifecycle to release dynamically allocated memory. It is typically paired with ca_stream_init, which performs the initial allocation. In a typical sequence, you would stop any tasks using the stream, call ca_stream_destroy to free the heap memory, and then set your pointer to NULL to prevent dangling references.

Esempio:

```
/* Assuming 'my_stream' was previously initialized with ca_stream_init */

void cleanup_task(ca_stream_buffer_t *my_stream) {
    if (my_stream != NULL) {
        /* Safely destroy the stream and free its internal buffer */
        ca_stream_destroy(my_stream);

        /* Important: prevent further use of the destroyed handle */
        my_stream = NULL;
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_stream_destroy() and why is it important to use it?
A: You should call ca_stream_destroy() when you are done using a stream buffer and want to free the memory it allocated. It is important because it ensures proper cleanup of resources and prevents memory leaks. The function resets the buffer's parameters and frees the memory, making it safe to reuse or discard the buffer.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_stream_destroy() with caOS types?

A: ca_stream_buffer_t *sb = ca_stream_create(1024); // Create a stream buffer // ... use the stream buffer ... ca_stream_destroy(sb); // Destroy the stream buffer sb = NULL; // Set pointer to NULL after destruction

Parameters conf:0.90 **Q:** What is the significance of the ca_stream_buffer_t *sb parameter in ca_stream_destroy() and what happens if it's invalid?
A: The parameter sb is a pointer to the stream buffer that needs to be destroyed. If sb is NULL, the function terminates immediately without performing any action. If sb points to an invalid or unallocated buffer, the behavior is undefined because the function assumes the buffer was allocated with ca_memory_alloc().

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_stream_destroy()?
A: Common mistakes include calling ca_stream_destroy() on a buffer that was not allocated with ca_memory_alloc(), calling it on a buffer that has already been destroyed, or calling it from an ISR. Additionally, calling it from multiple threads without external synchronization can lead to undefined behavior. The function uses a spinlock, which can cause deadlocks if called from an ISR.

`ca_stream_init`

Header: `caos/inc/stream.h:63`
Source: `caos/src/stream.c:29:44`
GUARD: `EN_STREAM`

```
1 bool ca_stream_init(ca_stream_buffer_t *sb, u32 size)
```

Initializes a dynamic stream buffer for byte-oriented data communication between tasks.

Parametri:

Nome	Tipo	Descrizione
`sb`	`ca_stream_buffer_t *`	Pointer to the stream buffer control structure. Must not be NULL.
`size`	`u32`	The total number of bytes to allocate for the buffer. Must be greater than 0.

Ritorna:
Returns true if the buffer was successfully allocated and initialized, false otherwise (e.g., if memory allocation fails or input parameters are invalid).

Note:

- This function performs dynamic memory allocation using ca_memory_allocate, so it must be called before any other stream operations. It is not thread-safe during the initialization phase, so ensure it is called during system startup or from a single task before the buffer is shared. It is not safe to call this from an ISR because it performs heap allocation. A common mistake is failing to check the return value; if the system is low on memory, this function will return false, and attempting to use the uninitialized buffer will cause a system crash or undefined behavior.

Uso:

- This function is the mandatory first step in the lifecycle of a dynamic stream buffer. It is typically used during the initialization phase of a driver or a communication task to prepare a data pipe. After successful initialization, the buffer is used with functions like ca_stream_send and ca_stream_receive to exchange data. Once the stream is no longer needed, it must be cleaned up using ca_stream_destroy to prevent memory leaks.

Esempio:

```c
ca_stream_buffer_t my_stream;

// Initialize a stream buffer with 1024 bytes
if (ca_stream_init(&my_stream, 1024)) {
    // Stream is ready for use
    // ... perform data operations ...

    // Cleanup when finished
    ca_stream_destroy(&my_stream);
} else {
    // Handle initialization failure (e.g., log error or retry)
    ca_printf("Failed to initialize stream buffer!\n");
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_stream_init in a caOS application?
A: You should call ca_stream_init when you need to set up a stream buffer for inter-task or interrupt communication. It is typically called during system initialization or when a task is created that requires such a buffer. The function prepares the buffer for use but does not allocate memory in an ISR context.

Code example conf:0.95 **Q:** Can you show a minimal example of how to use ca_stream_init with caOS types?
A: ca_stream_buffer_t sb; bool result = ca_stream_init(&sb, 256); if (result) { // Buffer initialized successfully } else { // Handle initialization failure }

Parameters conf:0.95 **Q:** What is the significance of the 'size' parameter in ca_stream_init, and what values are valid?
A: The 'size' parameter specifies the number of bytes to allocate for the stream buffer. It must be greater than zero; otherwise, the function will return false. A size of zero or less will cause initialization to fail immediately.

Return conf:0.95 **Q:** What does it mean if ca_stream_init returns false?
A: If ca_stream_init returns false, it indicates that initialization failed. This can happen if the 'sb' parameter is NULL, the 'size' is zero, or if memory allocation fails during the initialization process.

Error/Edge case conf:0.90 **Q:** What are common mistakes developers make when using ca_stream_init?
A: Common mistakes include passing a NULL pointer for 'sb', using a size of zero, or not checking the return value of the function. Not checking the return value can lead to using an uninitialized buffer, which may cause undefined behavior.

Behavior conf:0.90 **Q:** Is ca_stream_init blocking or non-blocking, and can it be called from an ISR?
A: ca_stream_init is non-blocking in terms of execution time, but it may take time to allocate memory. It is not recommended to call it from an ISR because memory allocation can cause issues in RTOS environments and is generally discouraged in interrupt contexts.

`ca_stream_init_static`

Header: `caos/inc/stream.h:76`
Source: `caos/src/stream.c:59:70`
GUARD: `EN_STREAM`

```c
void ca_stream_init_static(ca_stream_buffer_t *sb, u8 *buffer, u32 size)
```

Initializes a stream buffer using a pre-allocated memory block for data storage.

Parametri:

Nome	Tipo	Descrizione
sb	ca_stream_buffer_t *	Pointer to the stream buffer control structure. Must be a valid, non-NULL memory address.
buffer	u8 *	Pointer to the memory array that will hold the stream data. Must be non-NULL and large enough for the intended use.
size	u32	The total capacity of the buffer in bytes. Must be greater than 0.

Ritorna:
void. If parameters are invalid (NULL or size 0), the function returns immediately without performing any initialization.

Note:

- This function is non-blocking and thread-safe in the sense that it initializes the internal spinlock, but it must be called before any other stream operations. It is intended for static memory allocation, meaning the buffer memory must persist for the entire lifetime of the stream. A common mistake is passing a local stack-allocated array to this function; if the function where the array is defined returns, the stream buffer will point to invalid memory, leading to system crashes or data corruption. This function is not intended to be called from an ISR, as it performs initialization tasks that should be done during system setup.

Uso:

- This function is typically used during the system initialization phase to prepare communication channels or data pipes. It is the first step in the lifecycle of a static stream: first, you define a buffer array and a control structure, then you call ca_stream_init_static, and finally, you use functions like ca_stream_send or ca_stream_receive to exchange data. It is often paired with ca_stream_set_trigger to define when tasks should be woken up upon data arrival.

Esempio:

```
/* Define a static buffer and control structure */
static ca_stream_buffer_t my_stream;
static u8 my_data_storage[128];

void setup_communication(void) {
    /* Initialize the stream with the static buffer */
    ca_stream_init_static(&my_stream, my_data_storage, 128);

    /* Set a trigger level so we know when data is ready */
    ca_stream_set_trigger(&my_stream, 10);

    /* Now the stream is ready for use */
    if (!ca_stream_is_full(&my_stream)) {
        /* Send data into the stream */
        // ca_stream_send(&my_stream, data_ptr, length);
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_stream_init_static instead of a dynamic stream buffer allocation?
A: You should use ca_stream_init_static when you want to avoid dynamic memory allocation, typically in bare-metal environments where heap usage is restricted or undesirable. It is ideal for initializing stream buffers with pre-allocated static memory, such as global or static local variables.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_stream_init_static with caOS types?
A: u8 my_buffer[256]; caStreamBuffer sb; ca_stream_init_static(&sb, my_buffer, 256);

Parameters conf:0.90 **Q:** What are the valid and invalid values for the 'size' parameter in ca_stream_init_static?
A: The 'size' parameter must be greater than zero. A value of zero is invalid and will cause the function to return immediately without performing any action. Any positive value is valid, but the buffer size should be sufficient for the expected data throughput.

Return conf:0.95 **Q:** Does ca_stream_init_static return any value, and what does it indicate?
A: The function does not return any value. It is void. It will not fail in a strict sense, but it returns immediately if any of the parameters are invalid (e.g., NULL pointers or zero size).

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_stream_init_static?
A: Common mistakes include passing NULL pointers for 'sb' or 'buffer', or a size of zero. Additionally, if the buffer size is too small for the data being written, it may lead to overwrites. The function does not initialize the data buffer itself, so care must be taken to ensure proper data handling.

`ca_stream_is_empty`

Header: `caos/inc/stream.h:135`
Source: `caos/src/stream.c:257:260`
GUARD: `EN_STREAM`

```
1 bool ca_stream_is_empty(ca_stream_buffer_t *sb)
```

Checks if the specified stream buffer contains no data.

Parametri:

Nome	Tipo	Descrizione
`sb`	`ca_stream_buffer_t *`	Pointer to the stream buffer handle. Must be a valid, initialized pointer. If NULL, the function safely returns true.

Ritorna:
Returns true if the buffer is empty (count is 0) or if the pointer is NULL. Returns false if there is at least one byte of data stored in the buffer.

Note:

- This function is non-blocking and executes in constant time, making it safe to call from both standard tasks and Interrupt Service Routines (ISRs). It is thread-safe in the sense that it performs a simple read of the internal count variable, but it does not provide atomicity for the entire stream state; if another task or ISR writes to the stream immediately after this check, the result may become stale. The most common mistake is assuming that a 'true' result guarantees the buffer will remain empty for subsequent operations, which is false in multi-threaded environments.

Uso:

- This function is typically used in a polling loop or a conditional check before attempting to read data from a stream. It is often paired with ca_stream_bytes_available to determine if processing is required, or used as a guard condition to prevent underflow errors when calling receive functions. In a typical lifecycle, you initialize the stream with ca_stream_init, perform data processing based on the status returned by this function, and eventually clean up with ca_stream_destroy.

Esempio:

```
ca_stream_buffer_t my_stream;

// Initialize the stream buffer before use
ca_stream_init(&my_stream, buffer_memory, STREAM_SIZE);

// Check if there is data to process
if (ca_stream_is_empty(&my_stream)) {
    // No data available, perform other tasks or sleep
    ca_os_delay_ms(10);
} else {
    // Data is present, safely proceed to read
    u8 data;
    // Perform read operation using ca_stream_receive(&my_stream, &data, 1);
    ca_stdout_printf_n("Processing data from stream...");
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_stream_is_empty() in my embedded application?
A: You should call ca_stream_is_empty() to check if a stream buffer is empty before attempting to read from it. This is useful to avoid reading invalid or stale data, especially in interrupt service routines or when implementing non-blocking data handling logic. It helps prevent unnecessary operations and ensures data integrity.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_stream_is_empty() with caOS types?
A: ca_stream_buffer_t myBuffer; bool isEmpty; // Assume myBuffer is initialized elsewhere isEmpty = ca_stream_is_empty(&myBuffer); if (isEmpty) { // Handle empty buffer case }

Parameters conf:0.85 **Q:** What does the 'sb' parameter in ca_stream_is_empty() represent, and what are valid values?
A: The 'sb' parameter is a pointer to a ca_stream_buffer_t structure that must be valid and properly initialized. If sb is NULL, the function returns true by default. Passing an uninitialized or corrupted buffer pointer may lead to unpredictable behavior, although the function itself does not modify the buffer state.

Return conf:0.80 **Q:** What do the return values of ca_stream_is_empty() indicate, and what causes it to return true?
A: The function returns true if the buffer is empty (no data present), and false if data is available. If the input pointer 'sb' is NULL, it returns true as a default value. Failures or unexpected behavior are not explicitly defined in the documentation, but passing an invalid pointer may cause undefined results.

`ca_stream_is_full`

Header: `caos/inc/stream.h:140`
Source: `caos/src/stream.c:262:265`
GUARD: `EN_STREAM`

```
bool ca_stream_is_full(ca_stream_buffer_t *sb)
```

Checks if a stream buffer has reached its maximum capacity.

Parametri:

Nome	Tipo	Descrizione
sb	ca_stream_buffer_t *	Pointer to the stream buffer structure. Must be a valid, initialized pointer. If NULL, the function returns true.

Ritorna:
Returns true if the buffer is full (count >= size) or if the pointer is NULL. Returns false if there is still space available in the buffer.

Note:

- This function is non-blocking and executes in constant time, making it safe to call from both tasks and Interrupt Service Routines (ISRs). It is thread-safe in the sense that it performs a simple read operation on the buffer's internal counter; however, it does not provide atomicity for subsequent operations. A common beginner mistake is to check if the buffer is full and then proceed to write without considering that another task or ISR might have filled the buffer in the microsecond between the check and the write operation. Always ensure your logic handles the case where the buffer might become full immediately after your check.

Uso:

- This function is typically used in a producer-consumer pattern to prevent buffer overflows. Before attempting to push data into a stream using ca_stream_send, you should call ca_stream_is_full to verify that there is sufficient capacity. It is often paired with ca_stream_bytes_available or ca_stream_space_available to provide detailed telemetry or to implement flow control, such as putting a task to sleep or dropping packets when the buffer is saturated.

Esempio:

```
ca_stream_buffer_t my_stream;

// Initialize the stream buffer before use
ca_stream_init(&my_stream, buffer_memory, STREAM_SIZE);

// Inside a producer task loop
if (!ca_stream_is_full(&my_stream)) {
    // Safe to send data as the buffer is not full
    ca_stream_send(&my_stream, data_ptr, data_len);
} else {
    // Buffer is full, handle overflow (e.g., wait or drop data)
    ca_stdout_printf_n("Stream full, dropping data");
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_stream_is_full() in my embedded application?
A: You should call ca_stream_is_full() to check if a stream buffer is full before attempting to write data into it. This helps prevent overwriting data that hasn't been read yet, ensuring safe and predictable stream handling. It is especially useful in scenarios where you are managing data flow between interrupt service routines and task contexts.

Code example conf:0.90 **Q:** Can you show a minimal example of using ca_stream_is_full() with caOS types?
A: ca_stream_buffer_t buffer; ca_stream_init(&buffer, 1024); if (ca_stream_is_full(&buffer)) { // Handle buffer full condition } else { // Safe to write data }

Parameters conf:0.90 **Q:** What does the 'sb' parameter in ca_stream_is_full() represent, and what are valid values?
A: The 'sb' parameter is a pointer to a ca_stream_buffer_t structure that must be initialized before calling the function. If the pointer is NULL or the buffer is not properly

initialized, the function will return true. Valid values are pointers to correctly initialized stream buffers created using functions like ca_stream_init().

Return conf:0.90 **Q:** What do the return values of ca_stream_is_full() mean, and what causes it to return true?

A: The function returns true if the buffer is full or if the pointer passed is NULL. This can happen when the buffer is full, or if the buffer was not initialized correctly. It returns false only when the buffer is not full and the pointer is valid. A return value of true may indicate a logic error if not handled properly.

`ca_stream_receive`

Header: `caos/inc/stream.h:108`
Source: `caos/src/stream.c:165:208`
GUARD: `EN_STREAM`

```
u32 ca_stream_receive(ca_stream_buffer_t *sb, u8 *data, u32 max_len, u32
    timeout_ms)
```

Reads data from a stream buffer, blocking the current task until data is available or the timeout expires.

Parametri:

Nome	Tipo	Descrizione
`sb`	`ca_stream_buffer_t *`	Pointer to the initialized stream buffer handle. Must not be NULL.
`data`	`u8 *`	Pointer to the destination buffer where received bytes will be stored. Must not be NULL.
`max_len`	`u32`	Maximum number of bytes to read. Must be greater than 0.
`timeout_ms`	`u32`	Wait time in milliseconds. Use 0 for non-blocking (return immediately if empty) or 0xFFFFFFFF for infinite wait.

Ritorna:
Returns the number of bytes actually read (0 to max_len). Returns 0 if the buffer is empty and the timeout expires, or if input parameters are invalid.

Note:

- This function is blocking, meaning the calling task will be suspended by the RTOS scheduler if no data is available, allowing other tasks to run. It is thread-safe due to internal spinlock usage, but it must NEVER be called from an ISR (Interrupt Service Routine) because it performs blocking operations. The most common beginner mistake is passing a timeout of 0 while expecting the task to wait for data, or failing to check the return value, which leads to processing uninitialized or stale data in the destination buffer.

Uso:

- This function is typically used in a producer-consumer pattern where a task waits for incoming data from a peripheral (like UART or network). It is usually paired with ca_stream_init or ca_stream_init_static during system setup. In a typical loop, you call this function to fetch data, process it, and then repeat. If you need to check if data is available without blocking, use ca_stream_bytes_available first.

Esempio:

```c
/* Assuming data_stream is already initialized via ca_stream_init_static */
u8 rx_buffer[64];
u32 bytes_read = ca_stream_receive(&data_stream, rx_buffer, 64, 1000);

if (bytes_read > 0) {
    /* Successfully received data, process it */
    ca_printf("Received %d bytes\n", bytes_read);
} else {
    /* Timeout occurred or stream is empty */
    ca_printf("No data received within 1 second\n");
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_stream_receive to read data from a stream buffer?
A: You should use ca_stream_receive when you need to read data from a stream buffer that was initialized with ca_stream_init(). It is typically used in tasks to retrieve data produced by other tasks or ISRs. The function supports both blocking and non-blocking behavior depending on the timeout value.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_stream_receive in a task?
A: ca_stream_buffer_t sb; ca_stream_init(&sb, 256); // Assume data is written to sb by another task or ISR u8 buffer[64]; u32 bytes_read = ca_stream_receive(&sb, buffer, 64, 1000); // Wait up to 1 second if (bytes_read > 0) { // Process received data }

Parameters conf:0.95 **Q:** What does the timeout_ms parameter control in ca_stream_receive?
A: The timeout_ms parameter controls whether the function blocks or not. If timeout_ms is 0, the function is non-blocking and returns immediately. If timeout_ms is 0xFFFFFFFF, the function blocks indefinitely until data is available. Any other value specifies a maximum wait time in milliseconds.

Return conf:0.90 **Q:** What does a return value of 0 from ca_stream_receive indicate?
A: A return value of 0 indicates either an error or a timeout. If timeout_ms is 0, it means no data was available. If timeout_ms is non-zero and not 0xFFFFFFFF, it means the timeout expired before data was available.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_stream_receive?
A: Common mistakes include calling ca_stream_receive with timeout_ms = 0xFFFFFFFF from an ISR context, which can cause deadlock. Also, passing NULL pointers for sb, data, or max_len = 0 will cause an immediate return of 0 without reading any data.

Behavior conf:0.90 **Q:** Is ca_stream_receive thread-safe and can it be called from an ISR?
A: Yes, ca_stream_receive is thread-safe and uses spinlock for concurrent access protection. However, it is not safe to call from an ISR context. It should only be used from task contexts to avoid potential deadlocks or race conditions.

`ca_stream_receive_isr`

Header: `caos/inc/stream.h:120`
Source: `caos/src/stream.c:233:241`
GUARD: `EN_STREAM`

```c
u32 ca_stream_receive_isr(ca_stream_buffer_t *sb, u8 *data, u32 max_len)
```

Reads data from a stream buffer specifically designed for use within Interrupt Service Routines (ISRs).

Parametri:

Nome	Tipo	Descrizione
sb	ca_stream_buffer_t *	Pointer to the initialized stream buffer handle. Must not be NULL.
data	u8 *	Pointer to the destination buffer where the read data will be stored. Must not be NULL.
max_len	u32	Maximum number of bytes to read. Must be greater than 0.

Ritorna:
Returns the actual number of bytes copied into the destination buffer. Returns 0 if the input parameters are invalid (NULL pointers or zero length) or if the stream buffer is currently empty.

Note:

- This function is specifically designed to be safe for calling from an ISR because it uses interrupt-safe locking mechanisms (ca_spinlock_lock_irq). It is non-blocking, meaning it will return immediately with whatever data is currently available in the buffer rather than waiting for more data to arrive. A common beginner mistake is calling this function from a standard task context where a blocking read might be preferred, or failing to check the return value, which leads to processing incomplete or empty data packets.

Uso:

- This function is typically used in high-speed data acquisition scenarios where an interrupt (e.g., UART or DMA completion) triggers the need to extract data from a stream buffer immediately. It is usually paired with ca_stream_init or ca_stream_init_static during system setup. In the typical lifecycle, the system initializes the stream, the ISR calls ca_stream_receive_isr to drain the buffer into local memory, and the main application logic processes that data.

Esempio:

```
/* Example: Reading data from a stream buffer inside a UART RX interrupt */
void UART_RX_IRQHandler(void) {
    ca_stream_buffer_t *my_stream = get_stream_handle(); // Assume this
   retrieves the initialized handle
    u8 rx_buffer[16];
    u32 bytes_read;

    /* Attempt to read up to 16 bytes from the stream buffer */
    bytes_read = ca_stream_receive_isr(my_stream, rx_buffer, 16);

    if (bytes_read > 0) {
        /* Process the received data */
        // process_data(rx_buffer, bytes_read);
    } else {
        /* Handle case where no data was available */
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_stream_receive_isr() in my embedded application?
A: You should use ca_stream_receive_isr() when you need to read data from a stream buffer inside an ISR (Interrupt Service Routine). This function is designed specifically for non-blocking data retrieval from a stream buffer in interrupt context, avoiding the overhead of task scheduling or blocking operations.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_stream_receive_isr() in an ISR?

A: u8 buffer[10]; caStreamBuffer uart_rx_buffer; // Assume uart_rx_buffer is initialized u32 bytes_read = ca_stream_receive_isr(&uart_rx_buffer, buffer, 10); if (bytes_read > 0) { // Process received bytes }

Parameters conf:0.90 **Q:** What are the valid and invalid values for the max_len parameter in ca_stream_receive_isr()?

A: The max_len parameter must be greater than 0. If max_len is 0 or any parameter is NULL, the function returns 0 indicating an error. The max_len specifies the maximum number of bytes that can be read into the data buffer, and it must be a positive integer.

Return conf:0.95 **Q:** What does a return value of 0 from ca_stream_receive_isr() indicate?

A: A return value of 0 indicates an error, which occurs when one or more parameters are NULL or when max_len is 0. It does not indicate the absence of data in the buffer, as the function is non-blocking and returns immediately with available data.

`ca_stream_reset`

Header: `caos/inc/stream.h:81`
Source: `caos/src/stream.c:72:80`
GUARD: `EN_STREAM`

```
void ca_stream_reset(ca_stream_buffer_t *sb)
```

Resets a stream buffer by clearing its internal pointers and byte count.

Parametri:

Nome	Tipo	Descrizione
sb	`ca_stream_buffer_t *`	Pointer to the stream buffer structure to be reset. Must be a valid, initialized pointer.

Ritorna:
void

Note:

- This function is thread-safe because it uses a spinlock to protect the internal state during the reset operation. It is non-blocking in terms of logic, but it briefly disables interrupts to ensure atomicity. It is safe to call from an ISR, but you must ensure the pointer passed is valid. The most common mistake is passing a NULL pointer or a pointer to a buffer that has not been initialized via ca_stream_init or ca_stream_init_static, which will lead to a system crash or undefined behavior.

Uso:

- Use this function when you need to clear all pending data in a stream buffer, for example, when a communication protocol detects a framing error or when a user requests a manual reset via a CLI command. It is typically used in the 'use' phase of the buffer lifecycle, after initialization and before destruction. It is often paired with ca_stream_init for setup and ca_stream_destroy for cleanup, and it is frequently called when re-synchronizing a data stream after a timeout or a connection drop.

Esempio:

```
/* Assuming data_stream is a globally initialized ca_stream_buffer_t */
#include "caos.h"

void handle_reset_command(void) {
    /* Check if the buffer is valid before resetting */
    if (&data_stream != NULL) {
        /* Reset the stream buffer to clear old, stale data */
```

```
        ca_stream_reset(&data_stream);

        /* Notify the user that the buffer is now empty */
        ca_printf_n("Stream buffer has been cleared.");
    } else {
        ca_printf_n("Error: Stream buffer not initialized.");
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_stream_reset() in a caOS application?
A: You should call ca_stream_reset() when you need to clear the state of a stream buffer, for example after an error condition or when resetting a data flow. It is useful for restoring the buffer to an empty state without deallocating memory or changing other configuration parameters like trigger_level or waiting_task.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_stream_reset() with caOS types?
A: ca_stream_buffer_t my_buffer; ca_stream_init(&my_buffer, 1024); // ... use the buffer ... ca_stream_reset(&my_buffer);

Parameters conf:0.90 **Q:** What is the significance of the 'sb' parameter in ca_stream_reset() and what happens if it is invalid?
A: The 'sb' parameter is a pointer to the ca_stream_buffer_t that needs to be reset. If the pointer is NULL, the function does nothing. The buffer must be valid and initialized; otherwise, calling this function can lead to undefined behavior.

Return conf:0.95 **Q:** Does ca_stream_reset() return any value, and what does it indicate?
A: The function does not return any value. It is void. It cannot fail in a strict sense, but if the input parameter 'sb' is NULL, the function performs no operation.

`ca_stream_send`

Header: `caos/inc/stream.h:98`
Source: `caos/src/stream.c:128:163`
GUARD: `EN_STREAM`

```
u32 ca_stream_send(ca_stream_buffer_t *sb, const u8 *data, u32 len, u32
    timeout_ms)
```

Sends a block of data into a stream buffer, blocking the task until all data is sent or the timeout expires.

Parametri:

Nome	Tipo	Descrizione
`sb`	`ca_stream_buffer_t *`	Pointer to the initialized stream buffer. Must not be NULL.
`data`	`const u8 *`	Pointer to the source data buffer. Must not be NULL.
`len`	`u32`	Number of bytes to send. Must be greater than 0.
`timeout_ms`	`u32`	Maximum time to wait in milliseconds. Use 0 to return immediately if buffer is full, or 0xFFFFFFFF to block indefinitely.

Ritorna:

Returns the total number of bytes successfully written to the stream. Returns 0 if parameters are invalid, the stream is full (with 0 timeout), or the operation times out before sending any data.

Note:

- This function is blocking and must only be called from a task context, never from an ISR. It is thread-safe due to internal spinlock usage, but it relies on the stream buffer being properly initialized via ca_stream_init or ca_stream_init_static beforehand. A common beginner mistake is passing a timeout of 0 and failing to check the return value, leading to silent data loss when the buffer is full. Because it uses ca_os_sleep(1) internally, it yields control to the scheduler, allowing other tasks to run while waiting for space to become available in the stream.

Uso:

- Used for inter-task communication where one task produces data and another consumes it via a stream. It is typically used in a producer-consumer pattern: initialize the stream with ca_stream_init, then call ca_stream_send to push data. It is often paired with ca_stream_receive in the consumer task. If the producer needs to ensure data delivery, it should check the return value against the requested length and implement a retry mechanism or error handling if the returned value is less than expected.

Esempio:

```
/* Assuming 'data_stream' is a global ca_stream_buffer_t */

void producer_task(void *pvParameters) {
    u8 my_data[] = "Hello caOS";
    u32 len = 10;
    u32 timeout = 1000; // 1 second timeout

    // Attempt to send data to the stream
    u32 sent = ca_stream_send(&data_stream, my_data, len, timeout);

    if (sent < len) {
        // Handle partial send or timeout error
        // Log error using ca_printf or similar
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_stream_send to transmit data in caOS?
A: You should use ca_stream_send when you need to send data to a stream buffer in a blocking manner, ensuring that the data is written or the timeout expires. It is ideal for scenarios where you want to guarantee data delivery or handle transmission delays gracefully.

Code example conf:0.95 **Q:** Can you show a minimal example of how to use ca_stream_send with caOS types?
A: ca_stream_buffer_t sb; ca_stream_init(&sb, 1024); const u8 data[] = {0x01, 0x02, 0x03}; u32 sent = ca_stream_send(&sb, data, sizeof(data), 1000); if (sent == sizeof(data)) { // All data sent successfully }

Parameters conf:0.95 **Q:** What does the timeout_ms parameter control in ca_stream_send?
A: The timeout_ms parameter controls how long ca_stream_send will wait for space in the buffer. If set to 0, the function does not block and returns immediately. If set to 0xFFFFFFFF, it blocks indefinitely until space is available. Any other value specifies the maximum time in milliseconds to wait before returning.

Return conf:0.95 **Q:** What does ca_stream_send return when it fails to send all data?

A: ca_stream_send returns a value between 0 and len (inclusive). If it returns 0, an error occurred (e.g., invalid parameters). If it returns a value less than len, only part of the data was sent, typically due to timeout or insufficient buffer space.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_stream_send?

A: Common mistakes include passing a NULL buffer or data pointer, using a buffer that hasn't been initialized, setting timeout_ms to 0 and expecting blocking behavior, or calling the function from an ISR, which is not safe due to use of ca_os_sleep() and scheduler functions.

Behavior conf:0.95 **Q:** Is ca_stream_send thread-safe and can it be called from an ISR?

A: Yes, ca_stream_send is thread-safe as it uses a spinlock to protect access to the stream buffer. However, it is not safe to call from an ISR because it uses ca_os_sleep() and ca_scheduler_signal_wake_up(), which are not ISR-compatible.

`ca_stream_send_isr`

Header: `caos/inc/stream.h:114`
Source: `caos/src/stream.c:214:231`
GUARD: `EN_STREAM`

```
u32 ca_stream_send_isr(ca_stream_buffer_t *sb, const u8 *data, u32 len)
```

Writes data to a stream buffer from within an Interrupt Service Routine (ISR) and signals a waiting task if the trigger level is met.

Parametri:

Nome	Tipo	Descrizione
`sb`	`ca_stream_buffer_t *`	Pointer to the initialized stream buffer handle. Must not be NULL.
`data`	`const u8 *`	Pointer to the source buffer containing the data to be written. Must not be NULL.
`len`	`u32`	Number of bytes to write. Must be greater than 0.

Ritorna:
Returns the number of bytes successfully written to the buffer. Returns 0 if the input parameters are invalid (NULL pointers or zero length).

Note:

- This function is specifically designed for use inside ISRs. It is non-blocking, meaning it will not wait for space to become available in the buffer; it only writes what fits immediately. It uses a spinlock with interrupt disabling to ensure thread safety when accessed from both ISRs and standard tasks. A common beginner mistake is calling this function with a pointer to a buffer that has not been initialized via ca_stream_init or ca_stream_init_static, which will lead to a system crash or memory corruption. Always ensure the stream buffer is properly configured before triggering interrupts that call this function.

Uso:

- This function is typically used in hardware interrupt handlers (e.g., UART RX or DMA completion interrupts) to push incoming data into a stream buffer for later processing by a background task. It is usually paired with ca_stream_init or ca_stream_init_static during system setup. In the processing task, you would typically use ca_stream_receive to consume the data. If the data volume exceeds the buffer capacity, this function will only write what is available, so ensure your buffer size is sufficient for your expected burst traffic.

Esempio:

```c
/* Global stream buffer handle */
caStreamBuffer my_stream;

/* UART interrupt handler example */
void UART_IRQHandler(void) {
    u8 rx_byte;
    /* Assume hardware register read operation here */
    rx_byte = /* Read from UART hardware register */;

    /* Push the byte into the stream buffer from the ISR */
    u32 written = ca_stream_send_isr(&my_stream, &rx_byte, 1);

    if (written == 0) {
        /* Handle buffer full or error condition */
    }
}

/* Initialization in main or setup task */
void setup(void) {
    /* Initialize the stream buffer with a specific trigger level */
    ca_stream_init(&my_stream, 128);
    ca_stream_set_trigger(&my_stream, 64);
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_stream_send_isr() in an ISR context?
A: You should use ca_stream_send_isr() when you need to send data from an ISR to a stream buffer without blocking. It is specifically designed for ISR usage, allowing fast, non-blocking transmission of data such as received UART bytes or timer events into a buffer for later processing by a task.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_stream_send_isr() in an ISR?
A: ca_stream_buffer_t sb; ca_stream_init(&sb, 256); void USART1_IRQHandler(void) { u8 data = USART1->DR; ca_stream_send_isr(&sb, &data, 1); }

Parameters conf:0.95 **Q:** What are the valid and invalid values for the 'len' parameter in ca_stream_send_isr()?
A: The 'len' parameter must be greater than 0. If len is 0, the function returns 0, indicating an error. The parameter cannot be negative or zero, as it represents the number of bytes to send.

Return conf:0.95 **Q:** What does a return value of 0 from ca_stream_send_isr() indicate?
A: A return value of 0 indicates an error, typically caused by one or more invalid parameters such as sb being NULL, data being NULL, or len being 0.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_stream_send_isr()?
A: Common mistakes include calling the function from a non-ISR context, which may cause synchronization issues, and assuming that a return value equal to 'len' guarantees all data was sent, since the function is non-blocking and may return fewer bytes than requested if buffer space is insufficient.

Behavior conf:0.95 **Q:** Is ca_stream_send_isr() thread-safe and blocking?
A: Yes, ca_stream_send_isr() is thread-safe due to its use of a spinlock for protection. It is also non-blocking, meaning it does not wait for buffer space to become available and returns immediately, even if not all data can be written.

ca_stream_set_trigger

Header: caos/inc/stream.h:88
Source: caos/src/stream.c:82:87
GUARD: EN_STREAM

```
void ca_stream_set_trigger(ca_stream_buffer_t *sb, u32 level)
```

Configures the minimum number of bytes required in a stream buffer to trigger a wake-up event.

Parametri:

Nome	Tipo	Descrizione
sb	ca_stream_buffer_t *	Pointer to the initialized stream buffer object. Must not be NULL.
level	u32	The threshold in bytes. Values are clamped between 1 and the buffer's total size.

Ritorna:
void

Note:

- This function is non-blocking and thread-safe as it performs a simple assignment to the internal structure. It is safe to call from an ISR, although it is typically used during the initialization phase of a task. The most common mistake is assuming this function automatically enables interrupts or signals; it only sets the internal threshold value that other stream functions (like ca_stream_send) use to decide when to trigger a notification. Ensure the stream buffer has been properly initialized with ca_stream_init or ca_stream_init_static before calling this function, otherwise, the behavior is undefined.

Uso:

- This function is used to optimize task wake-ups by preventing a task from being signaled for every single byte received. It is typically called immediately after initializing the stream buffer and before starting the producer/consumer tasks. By setting a trigger level (e.g., 8 or 16 bytes), you ensure the consumer task only wakes up when a meaningful chunk of data is available, reducing context switching overhead and improving system efficiency. It is often paired with ca_stream_init and ca_os_job.

Esempio:

```
ca_stream_buffer_t data_stream;

/* Initialize the stream buffer with a fixed memory area */
ca_stream_init(&data_stream, buffer_memory, 128);

/* Set trigger to 16 bytes: the reader task will only be signaled
   when at least 16 bytes are available in the stream */
ca_stream_set_trigger(&data_stream, 16);

/* Proceed to launch tasks that will use this stream */
ca_os_job("reader", CA_TH_LEVEL2, task_reader, 0, STACK_MEDIUM, CA_JOB_STD,
    CA_FAULT_ASSERT);
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_stream_set_trigger and why is it important for task synchronization?
A: You should call ca_stream_set_trigger when you want to configure the number of bytes that must be available in a stream buffer before a waiting task is woken up. This

is important for optimizing task synchronization and avoiding unnecessary wake-ups. It allows fine-grained control over when tasks are resumed based on data availability in the buffer.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_stream_set_trigger with caOS types?
A: ca_stream_buffer_t sb; ca_stream_init(&sb, 1024); ca_stream_set_trigger(&sb, 256);

Parameters conf:0.95 **Q:** What is the meaning of the 'level' parameter in ca_stream_set_trigger, and what happens if it's set to 0 or greater than the buffer size?
A: The 'level' parameter specifies the number of bytes that must be available in the stream buffer before a waiting task is woken up. If level is 0, it is automatically set to 1. If level exceeds the buffer size, it is capped at the buffer's maximum size.

`ca_stream_space_available`

Header: `caos/inc/stream.h:130`
Source: `caos/src/stream.c:252:255`
GUARD: `EN_STREAM`

```
u32 ca_stream_space_available(ca_stream_buffer_t *sb)
```

Returns the number of free bytes currently available for writing in a stream buffer.

Parametri:

Nome	Tipo	Descrizione
`sb`	`ca_stream_buffer_t *`	Pointer to the initialized stream buffer structure. Must not be NULL.

Ritorna:
Returns the count of free bytes (u32). Returns 0 if the buffer is full or if the provided pointer is NULL.

Note:

- This function is non-blocking and thread-safe for reading the current state of the buffer. It is safe to call from an ISR, as it performs a simple arithmetic operation on the buffer's internal counters without waiting for locks or resources. The most common mistake is assuming that the returned value remains valid indefinitely; in a multi-threaded environment, another task or an ISR might write to the buffer immediately after you check the space, making the returned value stale. Always ensure the stream buffer has been properly initialized with ca_stream_init or ca_stream_init_static before calling this function.

Uso:

- This function is typically used in a producer-consumer pattern to prevent buffer overflows. Before attempting to write data into a stream using ca_stream_send, you should call this function to verify that enough space exists to accommodate your data packet. It is often paired with ca_stream_bytes_available to monitor the overall health and throughput of the data stream, or used in a loop to wait until sufficient space is available before proceeding with a write operation.

Esempio:

```
/* Example: Checking space before writing to a stream */
#include "caos.h"

#define STREAM_SIZE 1024
```

```c
static ca_stream_buffer_t data_stream;
static u8 buffer_memory[STREAM_SIZE];

void task_producer(void *pvParameters) {
    /* Initialize the stream buffer before use */
    ca_stream_init_static(&data_stream, buffer_memory, STREAM_SIZE);

    u8 my_data[] = "Hello World";
    u32 data_len = 11;

    /* Check if there is enough space to write the data */
    if (ca_stream_space_available(&data_stream) >= data_len) {
        /* Safe to send data */
        ca_stream_send(&data_stream, my_data, data_len);
    } else {
        /* Handle buffer full condition */
        ca_stdout_printf_n("Stream full, cannot write data");
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_stream_space_available() in my embedded application?
A: You should call ca_stream_space_available() before writing data to a stream buffer to determine how much space is available for writing. This helps prevent overwriting data in the buffer and ensures safe operation, especially when dealing with interrupt-driven data sources or variable-length writes.

Code example conf:0.90 **Q:** Can you show a minimal example of using ca_stream_space_available() with caOS types?
A: ca_stream_buffer_t my_buffer; ca_stream_init(&my_buffer, 256); // Check available space before writing u32 space = ca_stream_space_available(&my_buffer); if (space > 0) { // Safe to write up to 'space' bytes u8 data[10] = {0}; ca_stream_write(&my_buffer, data, 10); }

Parameters conf:0.90 **Q:** What is the significance of the 'sb' parameter in ca_stream_space_available() and what happens if it's invalid?
A: The 'sb' parameter is a pointer to a ca_stream_buffer_t structure that must be valid and initialized. If sb is NULL or points to an uninitialized buffer, the function returns 0, indicating no space is available. Passing an invalid pointer can cause a crash, so it's essential to validate the buffer before calling the function.

Return conf:0.95 **Q:** What does a return value of 0 from ca_stream_space_available() indicate, and what causes it?
A: A return value of 0 indicates either that the buffer pointer 'sb' is NULL or that the buffer is not valid. It is not a fatal error but signals that no space is available for writing. This can happen if the buffer was not initialized or if a NULL pointer was passed to the function.

`ca_stricmp`

Header: `caos/inc/memaux.h:120`
Source: `caos/src/memaux.c:156:163`
GUARD: `EN_MEMAUX`

```c
i32 ca_stricmp(const char *s1, const char *s2)
```

Compares two strings character by character, ignoring case differences.

Parametri:

Nome	Tipo	Descrizione
s1	const char *	Pointer to the first null-terminated string. Must not be NULL.
s2	const char *	Pointer to the second null-terminated string. Must not be NULL.

Ritorna:
Returns 0 if strings are equal (ignoring case). Returns a value < 0 if the first differing character in s1 is less than in s2, and > 0 if greater.

Note:

- This function is non-blocking and thread-safe as it only performs read operations on the provided memory buffers. It is safe to call from an ISR, provided the pointers point to valid, accessible memory. The most common mistake is passing a NULL pointer, which will cause a hard fault or memory access violation. Ensure both strings are null-terminated; otherwise, the function will continue reading memory until it hits a zero byte, potentially leading to buffer overreads.

Uso:

- Use this function when you need to compare user input, configuration keys, or command strings where case sensitivity should not matter (e.g., 'START' vs 'start'). It is typically used in command-line interfaces or configuration parsers to validate input against expected keywords. It is often paired with ca_cli_register or similar input-processing logic to determine which action to trigger based on a string command.

Esempio:

```c
/* Example: Checking a command string from a CLI input */
#include "caos.h"

void process_command(const char *input) {
    /* Compare input against a known command, ignoring case */
    if (ca_stricmp(input, "RESET") == 0) {
        /* Perform system reset logic */
        /* ca_os_thread_reset_watchdog(); */
    } else if (ca_stricmp(input, "STATUS") == 0) {
        /* Print system memory info */
        ca_memory_info();
    } else {
        ca_printf("Unknown command: %s\n", input);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_stricmp in a caOS project, and what makes it different from a regular string comparison?
A: You should use ca_stricmp when you need to compare two strings in a case-insensitive manner. It is useful in scenarios like configuration parsing or user input handling where the case of characters should not affect equality. Unlike a standard string comparison function, ca_stricmp ignores the difference between uppercase and lowercase letters, making it suitable for more flexible matching.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_stricmp with caOS types?
A: cstr str1 = "Hello"; cstr str2 = "HELLO"; i32 result = ca_stricmp(str1, str2); // result will be 0 because the strings are equal ignoring case.

Parameters conf:0.95 **Q:** What happens if I pass a NULL pointer to ca_stricmp, and what are the valid inputs?
A: Passing a NULL pointer to either s1 or s2 parameter causes undefined behavior and may lead to a crash. Both parameters must be valid, non-NULL pointers to null-terminated strings. Valid inputs are pointers to valid C strings that are not NULL.

Return conf:0.90 **Q:** What do the return values of ca_stricmp indicate, and what causes the function to fail?
A: The function returns 0 if the strings are equal (case-insensitive), a positive value if s1 is lexicographically greater than s2, and a negative value if s1 is less than s2. Failure occurs if either parameter is NULL, which leads to undefined behavior and potential crash.

`ca_strlen`

Header: `caos/inc/memaux.h:154`
Source: `caos/src/memaux.c:203:212`
GUARD: `EN_MEMAUX`

```
u32 ca_strlen(const char *str)
```

Calculates the number of characters in a null-terminated string.

Parametri:

Nome	Tipo	Descrizione
`str`	`const char *`	Pointer to the null-terminated string. Must not be NULL. If the string is not null-terminated, the function will read past the buffer until it hits a zero byte, causing a memory access violation.

Ritorna:
Returns the number of characters (u32) excluding the null terminator ('\0').

Note:

- This function is non-blocking and thread-safe as it only performs read operations on the provided memory. It is safe to call from an ISR, provided the pointer points to valid, accessible memory. The most common mistake is passing a non-null-terminated buffer or a NULL pointer, which will lead to a hard fault or undefined behavior. It does not perform any bounds checking, so ensure the input string is properly terminated.

Uso:

- Used extensively in networking and communication stacks to determine the length of dynamic strings before copying them into buffers or sending them over a transport layer. It is typically paired with functions like ca_memcpy or custom buffer-writing functions that require an explicit length parameter. It is essential for calculating packet sizes, such as in HTTP headers or MQTT payloads, where the exact byte count must be known before transmission.

Esempio:

```
void send_log_message(const char *msg) {
    if (msg == NULL) return;

    // Calculate length to prepare buffer
    u32 len = ca_strlen(msg);

    // Ensure we do not exceed a hypothetical buffer size of 128
```

```
    if (len > 128U) len = 128U;

    // Use ca_printf to output the string safely
    // ca_printf is a standard caOS logging function
    ca_printf("Log length: %u, Content: %s\n", len, msg);
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_strlen in a caOS project, and why is it preferred over other string length functions?
A: You should use ca_strlen when you need to determine the length of a null-terminated string in a caOS environment. It is preferred because it is designed for bare-metal systems without HAL, and it's efficient for string handling in RTOS contexts. It's used in network services like http_service.c and mqtt_service.c where string lengths are needed for processing data.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_strlen with caOS types?
A: u32 length = ca_strlen("Hello"); cstr test_str = "World"; u32 len2 = ca_strlen(test_str);

Parameters conf:0.95 **Q:** What happens if I pass NULL to ca_strlen, and is it safe to do so?
A: Passing NULL to ca_strlen causes undefined behavior, which may result in a crash or system failure. The function assumes the input is a valid pointer to a null-terminated string. It is not safe to call ca_strlen with a NULL parameter.

Return conf:0.90 **Q:** What does ca_strlen return when given a valid string, and what causes it to return 0?
A: ca_strlen returns the number of characters in the string, excluding the null terminator. It returns 0 if the input string is NULL, which is undefined behavior. The function does not return 0 for valid strings; it returns the actual character count.

`ca_strncmp`

Header: `caos/inc/memaux.h:129`
Source: `caos/src/memaux.c:165:179`
GUARD: `EN_MEMAUX`

```
i32 ca_strncmp(const char *s1, const char *s2, u32 max)
```

Compares up to a specified number of characters between two strings.

Parametri:

Nome	Tipo	Descrizione
`s1`	`const char *`	Pointer to the first string to compare. Must be null-terminated.
`s2`	`const char *`	Pointer to the second string to compare. Must be null-terminated.
`max`	`u32`	Maximum number of characters to compare. If 0, the function returns 0 immediately.

Ritorna:
Returns 0 if strings are equal up to 'max' characters or until a null terminator is reached. Returns a non-zero integer (the difference between the first non-matching characters) if they differ.

Note:

- This function is non-blocking and thread-safe as it performs a simple read-only memory comparison. It is safe to call from an ISR because it does not perform any OS-level

synchronization or blocking operations. The most common mistake is passing a NULL pointer or a non-null-terminated string, which will cause the function to read past the intended memory buffer, potentially leading to a hard fault or undefined behavior.

Uso:

- It is primarily used in command-line interface (CLI) handlers to parse user input by comparing incoming strings against known command keywords. It is typically paired with ca_skip_sp to ignore leading whitespace before performing the comparison. Developers use it to safely check prefixes of strings without needing to know the full length of the input buffer, ensuring that the comparison never exceeds the provided 'max' limit.

Esempio:

```c
/* Example: CLI command handler for a system service */
static bool my_service_handler(c_str *pcmd) {
    /* Get the raw string data from the CLI structure */
    const char *p = ca_skip_sp((*pcmd)->data);

    /* Check if the command starts with 'info' */
    if (ca_strncmp(p, "info", 4) == 0) {
        /* Perform action if command matches */
        ca_printf("Service is running normally.\n");
        return true;
    }

    /* Check if the command starts with 'stop' */
    if (ca_strncmp(p, "stop", 4) == 0) {
        /* Logic to stop the service */
        return true;
    }

    return false; /* Command not recognized */
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_strncmp in an embedded application, and why is it useful for comparing strings?
A: ca_strncmp is useful when comparing two null-terminated strings up to a maximum number of characters, ignoring case differences. It's commonly used in embedded systems for configuration parsing, command-line handling, or comparing device identifiers where case-insensitive comparison is required. It's particularly helpful in driver code like ADC handling where string-based configuration or identification may be needed.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_strncmp with caOS types?
A: cstr str1 = "ADC0"; cstr str2 = "adc0"; u32 max_len = 4; i32 result = ca_strncmp(str1, str2, max_len); // result will be 0 since the strings are equal ignoring case.

Parameters conf:0.90 **Q:** What is the significance of the 'max' parameter in ca_strncmp, and what happens if it's set to 0 or a large value?
A: The 'max' parameter limits how many characters are compared between the two strings. If set to 0, the function returns 0 without comparing anything. If set to a large value, it may cause invalid memory access if the strings are shorter than max, leading to undefined behavior or crashes.

Return conf:0.95 **Q:** What do the different return values of ca_strncmp indicate, and what causes a negative or positive result?
A: A return value of 0 indicates that the first 'max' characters of both strings are equal (case-insensitive). A positive value means the first differing character in s1 is greater than in s2 (based on ASCII values). A negative value means the first differing character

in s1 is less than in s2. Failures or errors do not produce specific return codes; invalid inputs like NULL pointers cause crashes.

Error/Edge case conf:0.90 **Q:** What are the common mistakes or edge cases when using ca_strncmp that could lead to system crashes or undefined behavior?
A: Common mistakes include passing NULL pointers for s1 or s2, which will cause a crash. Also, using a very large 'max' value without ensuring the strings are long enough can lead to invalid memory access. Additionally, if the strings are not null-terminated, behavior is undefined and may cause crashes or incorrect comparisons.

`ca_strncpy`

Header: `caos/inc/memaux.h:147`
Source: `caos/src/memaux.c:276:285`
GUARD: `EN_MEMAUX`

```
char* ca_strncpy(char *dest, const char *src, u32 max)
```

Safely copies a string from a source to a destination buffer with a maximum length limit.

Parametri:

Nome	Tipo	Descrizione
`dest`	`char*`	Pointer to the destination buffer where the string will be copied.
`src`	`const char*`	Pointer to the source string to be copied.
`max`	`u32`	The maximum number of bytes to write to dest, including the null terminator.

Ritorna:
Returns the pointer to the destination buffer (dest).

Note:

- This function is non-blocking and thread-safe as it performs a simple memory copy without OS primitives. It is safe to call from an ISR because it does not interact with the scheduler or wait for resources. The most common beginner mistake is passing a 'max' value larger than the actual size of the destination buffer, which causes a buffer overflow. Always ensure 'max' is equal to or less than the allocated size of 'dest' to prevent memory corruption.

Uso:

- Use this function whenever you need to copy strings into fixed-size buffers, such as task names, CLI input buffers, or configuration strings. It is typically used during task initialization to store a name in the task control block or when processing user input from a CLI. It is often paired with sizeof() to ensure the destination buffer is not overrun, providing a safer alternative to standard string copying functions.

Esempio:

```
/* Example: Storing a task name safely during initialization */
#define TASK_NAME_LEN 16

void setup_my_task(void) {
    char my_task_name[TASK_NAME_LEN];
    const char *input_name = "SensorTask";

    /* Copy the name safely, ensuring we do not exceed the buffer size */
    ca_strncpy(my_task_name, input_name, TASK_NAME_LEN);

```

```
    /* Now the buffer is guaranteed to be null-terminated */
    ca_printf_n("Task initialized with name: %s\r\n", my_task_name);

    /* If we needed to allocate memory for a dynamic structure, we would use: */
    /* char *ptr = ca_memory_allocate(TASK_NAME_LEN); */
    /* if (ptr != NULL) { ca_strncpy(ptr, input_name, TASK_NAME_LEN); } */
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.90 **Q:** When should I use ca_strncpy instead of regular strncpy in caOS?
A: You should use ca_strncpy when you need to copy strings in a bare-metal environment without HAL, especially in contexts where you must avoid undefined behavior due to invalid parameters. It is ideal for CLI input handling, memory management utilities, and scheduler string operations where safety and deterministic behavior are required.

Code example conf:0.95 **Q:** Can you show a minimal example of using ca_strncpy with caOS types?
A: cstr dest = ""; cstr src = "Hello"; u32 max = 10; ca_strncpy(dest, src, max); // dest now contains "Hello" with null termination

Parameters conf:0.95 **Q:** What happens if I pass max=0 to ca_strncpy?
A: Passing max=0 results in undefined behavior. The function assumes max $>$ 0 and does not handle this case gracefully. It is the caller's responsibility to ensure max is greater than zero to avoid unpredictable outcomes.

Return conf:0.90 **Q:** What does ca_strncpy return, and what causes it to fail?
A: ca_strncpy returns a pointer to the destination string (dest). It does not return NULL under normal conditions. Failure occurs if parameters are invalid (e.g., dest or src are NULL, or max is 0), leading to undefined behavior rather than a specific error code.

`ca_strtol`

Header: `caos/inc/utils.h:160`
GUARD: `EN_UTILS`

```
static inline i32 ca_strtol(const char *buff, u16 base)
```

Converts a string representation of a number into a signed 32-bit integer based on a specified radix.

Parametri:

Nome	Tipo	Descrizione
`buff`	`const char *`	Pointer to the null-terminated string to parse. Must not be NULL.
`base`	`u16`	The numerical base for conversion (e.g., 10 for decimal, 16 for hexadecimal). Valid values are typically 2 through 36.

Ritorna:
Returns the parsed integer value as an i32. If the string is invalid or empty, it returns 0.

Note:

- This function is a non-blocking, thread-safe utility that performs no heap allocation and relies solely on stack variables. It is safe to call from an ISR, provided the input string buffer is valid and accessible. The most common beginner mistake is assuming the function returns an error code for invalid input; it does not. If the string does not start with a valid digit for the given base, it simply returns 0. Always validate your input string format or range before or after parsing if strict error checking is required.

Uso:

- This function is primarily used in CLI command parsers to extract numerical arguments from user input strings. It is typically paired with ca_skip_sp to advance the pointer past whitespace before parsing, and ca_strncmp to identify the command keyword. It is essential for converting raw text input from UART or other communication interfaces into usable configuration parameters for drivers like ADC, DAC, or CAN.

Esempio:

```c
/* Example: Parsing a command 'set <channel> <value>' from a CLI buffer */
void handle_cli_command(const char *p) {
    if (ca_strncmp(p, "set", 3) == 0) {
        /* Skip 'set' and leading spaces */
        p = ca_skip_sp(p + 3);

        /* Parse channel number (decimal) */
        u8 ch = (u8)ca_strtol(p, 10);

        /* Move pointer to the next argument */
        while (*p && *p != ' ') p++;

        /* Parse value (decimal) */
        u32 val = (u32)ca_strtol(ca_skip_sp(p), 10);

        /* Validate range before hardware interaction */
        if (ch >= 1 && ch <= 2 && val <= 4095U) {
            ca_dac_write(ch, (u16)val);
        } else {
            ca_printf_n("Invalid parameters");
        }
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_strtol in my caOS application?
A: You should use ca_strtol primarily in CLI command parsers to convert raw text input from communication interfaces like UART into numerical configuration parameters for drivers such as ADC, DAC, or CAN. It is often used after calling ca_skip_sp to handle whitespace.

Code example conf:1.00 **Q:** How do I implement a simple conversion of a decimal string to an i32 using caOS types?
A: You can call the function directly with a cstr and the base:
const char *input = "1024"; i32 value = ca_strtol(input, 10);

Parameters conf:1.00 **Q:** What are the constraints for the 'base' parameter in ca_strtol?
A: The 'base' parameter defines the numerical radix for the conversion. While valid values are typically 2 through 36, the function uses this to interpret the string representation of the number.

Return conf:1.00 **Q:** What does it mean if ca_strtol returns 0?
A: A return value of 0 indicates that the function either successfully parsed the value 0, or that the input string was invalid, empty, or did not start with a valid digit. Because it does not indicate an error, you should validate your input string format before parsing if strict error checking is required.

`ca_strtol_index`

Header: `caos/inc/utils.h:158`
Source: `caos/src/utils.c:510:568`

GUARD: `EN_UTILS`

```
i32 ca_strtol_index(const char *buff, u16 base,u16 *index)
```

Converts a string representation of a number into a 32-bit integer, tracking how many characters were processed.

Parametri:

Nome	Tipo	Descrizione
`buff`	`const char *`	Pointer to the null-terminated string to convert. Must not be NULL.
`base`	`u16`	The numerical base (e.g., 10 for decimal, 16 for hexadecimal). Valid range is typically 2-36.
`index`	`u16 *`	Pointer to a variable where the function will store the number of characters consumed during conversion.

Ritorna:
The converted 32-bit signed integer. Returns 0 if the input string is NULL or if no valid digits are found at the start of the string.

Note:

- This function is non-blocking and thread-safe as it does not access shared global state or hardware peripherals. It is safe to call from an ISR because it performs simple arithmetic and memory reads without any OS synchronization primitives. The most common mistake is passing a NULL pointer for the 'index' parameter, which the function handles safely, but it prevents the caller from knowing how many characters were consumed, making it impossible to parse subsequent tokens in a command string.

Uso:

- This function is primarily used in CLI (Command Line Interface) parsing to extract numerical parameters from user input strings. It is typically used in a sequence: first, locate the start of a token using ca_skip_sp, then call ca_strtol_index to extract the value, and finally use the returned 'index' to advance the pointer for the next token. It is often paired with ca_strncmp to identify commands and ca_stdout_printf_n to report errors if the conversion fails or the value is out of range.

Esempio:

```
/* Example: Parsing a command like 'set_prio 10 5' */
const char *input = "10 5";
u16 consumed = 0;

/* Extract the first number (task ID) */
i32 task_id = ca_strtol_index(input, 10, &consumed);

/* Advance pointer based on consumed characters and skip spaces */
const char *next_part = ca_skip_sp(input + consumed);

/* Extract the second number (priority) */
u16 consumed_prio = 0;
i32 priority = ca_strtol_index(next_part, 10, &consumed_prio);

if (consumed > 0 && consumed_prio > 0) {
    /* Apply the priority using the OS handle */
    const ca_task_handle_t h = ca_os_handle_from_thid((u16)task_id);
    if (h != CA_TASK_HANDLE_NONE) {
        ca_os_set_priority_handle(h, (u8)priority);
    }
```

```
} else {
    ca_stdout_printf_n("Error: Invalid numeric format");
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_strtol_index instead of standard library functions like strtol?
A: You should use ca_strtol_index when working in a bare-metal environment without standard library support, such as in caOS. It is ideal for parsing user input from CLI or configuration strings where you need both the converted integer and the length of the parsed portion. It's particularly useful in embedded systems where memory and performance are constrained.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_strtol_index to parse a hexadecimal string?
A: cstr input = "1A3F"; u16 index = 0; i32 result = ca_strtol_index(input, 16, &index); // result = 6719, index = 4

Parameters conf:0.95 **Q:** What are the valid values for the base parameter in ca_strtol_index?
A: The base parameter must be between 2 and 36 inclusive. Valid bases include 2 for binary, 8 for octal, 10 for decimal, and 16 for hexadecimal. Using a base outside this range leads to undefined behavior.

Return conf:0.90 **Q:** What does ca_strtol_index return when the input string is empty or contains invalid characters?
A: The function returns 0 when the input string is empty or contains characters that are not valid for the specified base. It also returns 0 if the conversion fails due to invalid input, and the index value will reflect how much of the string was successfully parsed.

Error/Edge case conf:0.85 **Q:** What are the most common mistakes when calling ca_strtol_index?
A: Common mistakes include passing a NULL pointer for the 'buff' parameter, using an invalid base (less than 2 or greater than 36), and not checking if the returned value is 0, which could indicate either a valid zero or a parsing error. Also, passing a NULL pointer for the 'index' parameter is valid but may lead to missing useful parsing information.

Behavior conf:0.90 **Q:** Is ca_strtol_index safe to call from an ISR in caOS?
A: Yes, ca_strtol_index is safe to call from an ISR because it is non-blocking, does not use global variables, and does not require OS services. It is also thread-safe as it does not rely on shared state.

`ca_sw_timer_count_active`

Header: `caos/inc/sw_timer.h:157`
Source: `caos/src/sw_timer.c:282:291`
GUARD: `EN_SW_TIMER`

```
u8 ca_sw_timer_count_active(void)
```

Returns the total number of software timers currently in the RUNNING state.

Ritorna:
An unsigned 8-bit integer (u8) representing the count of active timers. Returns 0 if no timers are currently running or if the timer system is not initialized.

Note:

- This function is non-blocking and executes in constant time relative to the maximum number of timers defined in the system (CA_TIMER_MAX). It is thread-safe as it performs a read-only iteration over the global timer array. It can be safely called from an ISR, though

it is primarily intended for monitoring tasks. A common mistake is assuming this returns the total number of created timers; it specifically filters for those that are both 'active' and in the 'CA_TIMER_RUNNING' state, ignoring stopped or expired timers.

Uso:

- This function is typically used in diagnostic or monitoring tasks to provide system health feedback. It is often paired with ca_sw_timer_get_state or ca_sw_timer_get_remaining to provide a detailed overview of the system's background activity. Developers use it to verify that periodic tasks are correctly scheduled and that one-shot timers have not unexpectedly stopped, often printing the result via ca_stdout_printf_n for debugging purposes.

Esempio:

```c
/* Example: Monitoring active timers in a background task */
#include "caos.h"

void monitor_task(void *p) {
    while (1) {
        /* Count how many timers are currently running */
        u8 running_count = ca_sw_timer_count_active();

        /* Report the status to the console */
        ca_stdout_printf_n("System status: %u timers are currently active.", running_count);

        /* Check if we have too many timers running, which might indicate a logic error */
        if (running_count > 5) {
            ca_stdout_printf_n("Warning: High timer load detected!");
        }

        /* Wait before the next check to avoid flooding the output */
        ca_os_delay_ms(5000);
    }
}
```

Q&A — fine-tuning review

3/3 pairs · qwen3-coder:latest

Usage conf:0.95 **Q:** When should I call ca_sw_timer_count_active() and why would I need to know the number of active software timers?
A: You should call ca_sw_timer_count_active() when you need to monitor or debug the state of software timers in your caOS system. It is useful for verifying that timers are running as expected, especially in applications where timer activity is critical. The function helps ensure that timer resources are being used correctly and can be used to detect potential issues with timer initialization or execution.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_sw_timer_count_active() with caOS types?
A: u8 active_timers = ca_sw_timer_count_active(); if (active_timers > 0) { // Process active timers // Example: log or handle active timer count } else { // No active timers }

Return conf:0.90 **Q:** What do the return values of ca_sw_timer_count_active() mean, and what could cause incorrect results?
A: The function returns a u8 value representing the number of software timers currently in the 'CA_TIMER_RUNNING' state. A return value of 0 indicates no active timers. The function does not return errors, but incorrect results may occur if the timer system has not been properly initialized. Since the function accesses a global structure (g_timers), improper initialization or concurrent access in a multi-threaded context

without protection can lead to inaccurate counts.

---

ca_sw_timer_create

Header: caos/inc/sw_timer.h:100
Source: caos/src/sw_timer.c:136:162
GUARD: EN_SW_TIMER

```
ca_sw_timer_handle_t ca_sw_timer_create(u32 period_ms,
    ca_sw_timer_mode_t mode, ca_timer_callback_t callback, void *arg)
```

Allocates and initializes a software timer object for periodic or one-shot tasks.

Parametri:

Nome	Tipo	Descrizione
period_ms	u32	Timer duration in milliseconds. Must be greater than 0.
mode	ca_sw_timer_mode_t	Operation mode: CA_TIMER_PERIODIC (restarts automatically) or CA_TIMER_ONESHOT (runs once).
callback	ca_timer_callback_t	Function pointer to be executed when the timer expires.
arg	void *	Optional pointer to data passed to the callback function.

Ritorna:
Returns a valid ca_sw_timer_handle_t (index + 1) on success, or CA_TIMER_INVALID if the system is not initialized, the period is 0, or no free timer slots are available.

Note:

- This function is thread-safe as it uses a spinlock to protect the internal timer array during allocation. It is non-blocking, meaning it returns immediately after finding a slot. You must call ca_sw_timer_init() before using this function, otherwise it will always return CA_TIMER_INVALID. A common mistake is forgetting to check if the returned handle is CA_TIMER_INVALID before passing it to ca_sw_timer_start(), which can lead to undefined behavior or system crashes.

Uso:

- This function is typically used during the initialization phase of a module or task to reserve a timer resource. It is part of a lifecycle: first, create the timer with this function, then start it using ca_sw_timer_start(), and finally, if the timer is no longer needed, release the slot using ca_sw_timer_delete(). It is commonly used for background tasks like LED blinking, periodic sensor polling, or implementing timeouts for communication protocols.

Esempio:

```
ca_sw_timer_handle_t my_timer;

/* Ensure the timer system is ready */
ca_sw_timer_init();

/* Create a periodic timer that triggers every 100ms */
my_timer = ca_sw_timer_create(100, CA_TIMER_PERIODIC, my_callback_function, NULL
    );

if (my_timer != CA_TIMER_INVALID) {
    /* Start the timer to begin execution */
```

```
    ca_sw_timer_start(my_timer);
} else {
    ca_stdout_printf("Failed to create timer!\n");
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_sw_timer_create function in my caOS project?
A: You should use it during the initialization phase of a module or task to reserve a timer resource. It is intended for background tasks such as LED blinking, periodic sensor polling, or implementing communication protocol timeouts.

Code example conf:0.90 **Q:** Provide a minimal C example of how to create a periodic timer in caOS.
A: void my_callback(void *arg) { /* Task logic */ } void init(void) { ca_sw_timer_handle_t h = ca_sw_timer_create(1000, CA_TIMER_PERIODIC, my_callback, NULL); if (h != CA_TIMER_INVALID) { ca_sw_timer_start(h); } }

Parameters conf:1.00 **Q:** What are the constraints on the period_ms parameter?
A: The period_ms parameter must be a u32 value greater than 0. If it is set to 0, the function will return CA_TIMER_INVALID.

Return conf:1.00 **Q:** What does the return value of ca_sw_timer_create represent and when does it fail?
A: It returns a valid ca_sw_timer_handle_t (index + 1) on success. It returns CA_TIMER_INVALID if the system is not initialized, the period is 0, or there are no free timer slots available.

Error/Edge case conf:1.00 **Q:** What is a common mistake developers make when using this function?
A: A common mistake is failing to check if the returned handle is CA_TIMER_INVALID before passing it to ca_sw_timer_start(), which can lead to undefined behavior or system crashes.

Behavior conf:1.00 **Q:** Is ca_sw_timer_create blocking and what are the preconditions for calling it?
A: It is non-blocking and returns immediately after finding a slot. It is thread-safe due to an internal spinlock, but you must call ca_sw_timer_init() before using this function, otherwise it will always return CA_TIMER_INVALID.

`ca_sw_timer_delete`

Header: `caos/inc/sw_timer.h:137`
Source: `caos/src/sw_timer.c:237:254`
GUARD: `EN_SW_TIMER`

```
bool ca_sw_timer_delete(ca_sw_timer_handle_t handle)
```

Permanently removes a software timer from the system, freeing its slot for future use.

Parametri:

Nome	Tipo	Descrizione
`handle`	`ca_sw_timer_handle_t`	The unique identifier returned by ca_sw_timer_create. Must be a valid handle; CA_TIMER_INVALID or out-of-bounds values will cause the function to return false.

Ritorna:
Returns true if the timer was successfully deactivated and removed. Returns false if the handle is invalid, the timer was already inactive, or the software timer system was not initialized.

Note:

- This function is thread-safe as it uses internal spinlocks to protect the timer array during modification. It is non-blocking, meaning it will return immediately after updating the timer's internal state. You must ensure that the timer handle is not used again after calling this function, as it will no longer point to a valid, active timer. A common mistake is attempting to delete a timer that has already been deleted or was never created, which will result in a false return value.

Uso:

- Use this function when a periodic or one-shot timer is no longer needed to prevent unnecessary processing overhead. It is typically used in the cleanup phase of a task or when a specific application state (e.g., a timeout monitor) is exited. It is usually paired with ca_sw_timer_create at the start of a process and called before the associated task or module is destroyed or re-initialized.

Esempio:

```
ca_sw_timer_handle_t my_timer = ca_sw_timer_create(my_callback_func, 1000); // Create a 1s timer

if (my_timer != CA_TIMER_INVALID) {
    // ... timer is running ...

    // When the task is finished or the timer is no longer required:
    if (ca_sw_timer_delete(my_timer)) {
        // Timer successfully removed
        my_timer = CA_TIMER_INVALID;
    } else {
        // Handle error: timer might have already been deleted
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_sw_timer_delete() in my embedded application?
A: You should call ca_sw_timer_delete() when you want to free the resources associated with a previously created software timer. This is typically done when the timer is no longer needed, such as when a module or task that uses the timer is shutting down or when dynamically managing timer resources.

Code example conf:0.90 **Q:** Can you show a minimal example of how to create and then delete a software timer using caOS?
A: ca_sw_timer_handle_t handle = ca_sw_timer_create(1000, true, NULL); // Create a periodic timer bool result = ca_sw_timer_delete(handle); // Delete the timer // handle must be a valid handle between 1 and CA_TIMER_MAX

Parameters conf:0.95 **Q:** What does the handle parameter in ca_sw_timer_delete() represent, and what are valid values?
A: The handle parameter is the identifier of the software timer to delete. It must be a valid value between 1 and CA_TIMER_MAX. If handle is CA_TIMER_INVALID or greater than CA_TIMER_MAX, the function will return false.

Return conf:0.95 **Q:** What does it mean when ca_sw_timer_delete() returns false?
A: A return value of false indicates that the deletion failed. This can happen if the timer was not initialized (g_timer_init = false), if the handle is invalid (CA_TIMER_INVALID or > CA_TIMER_MAX), or if the timer is not active (t->active = false).

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_sw_timer_delete()?
A: Common mistakes include calling the function with an invalid handle, attempting to delete a timer that was never initialized, or trying to delete a timer that has already

been deleted. Also, calling this function on a timer that is currently running may lead to failure.

Behavior conf:0.90 **Q:** Is ca_sw_timer_delete() blocking or non-blocking, and is it safe to call from an ISR?
A: The function is non-blocking and returns immediately. It is thread-safe due to the use of a spinlock g_timer_lock. It can be called from an ISR, but care must be taken to ensure that the lock g_timer_lock is compatible with the ISR environment to avoid deadlock.

---

ca_sw_timer_get_remaining

Header: `caos/inc/sw_timer.h:144`
Source: `caos/src/sw_timer.c:256:267`
GUARD: `EN_SW_TIMER`

```
u32 ca_sw_timer_get_remaining(ca_sw_timer_handle_t handle)
```

Retrieves the remaining time in milliseconds for a specific software timer.

Parametri:

Nome	Tipo	Descrizione
`handle`	`ca_sw_timer_handle_t`	The handle of the timer to query. Must be a valid handle returned by ca_sw_timer_create. Passing CA_TIMER_INVALID or an out-of-bounds value will result in a return of 0.

Ritorna:
Returns the remaining time in milliseconds if the timer is active. Returns 0 if the timer is inactive, not initialized, or if the provided handle is invalid.

Note:

- This function is non-blocking and thread-safe, as it performs a simple read operation on the internal timer structure. It can be safely called from an ISR, provided the timer system has been initialized via ca_sw_timer_init. A common mistake is assuming that a return value of 0 implies the timer has expired; it could also mean the timer was never started or has been stopped. Always verify the timer state using ca_sw_timer_get_state if you need to distinguish between an expired timer and an inactive one.

Uso:

- This function is typically used for monitoring or diagnostic purposes within a task loop. It is often paired with ca_sw_timer_get_state to provide a full status report of the system's background tasks. Developers use it to check how much time is left before a periodic or one-shot event triggers, allowing for dynamic adjustments to application logic or UI updates based on the remaining duration.

Esempio:

```
ca_sw_timer_handle_t my_timer;

// Initialize the timer system before use
ca_sw_timer_init();

// Create a timer (assuming a callback function exists)
my_timer = ca_sw_timer_create(my_callback_func);

// Start the timer for 5000ms
```

```
ca_sw_timer_start(my_timer, 5000);

// Periodically check the remaining time
u32 remaining = ca_sw_timer_get_remaining(my_timer);
if (remaining > 0) {
    ca_stdout_printf_n("Timer active, %u ms remaining", remaining);
} else {
    ca_stdout_printf_n("Timer is stopped or expired");
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_sw_timer_get_remaining() in my embedded application?
A: You should use ca_sw_timer_get_remaining() when you need to monitor or debug the status of a software timer, particularly to check how much time is left before it expires. It is useful for implementing dynamic timeout adjustments or logging timer behavior.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_sw_timer_get_remaining() with caOS types?
A: ca_sw_timer_handle_t handle = 1; u32 remaining = ca_sw_timer_get_remaining(handle); if (remaining > 0) { /* Timer is active and has remaining time */ } else { /* Timer is inactive or invalid */ }

Parameters conf:0.95 **Q:** What is the significance of the handle parameter in ca_sw_timer_get_remaining()?
A: The handle parameter identifies a specific software timer. It must be a valid value between 1 and CA_TIMER_MAX. If the handle is outside this range, the function returns 0, indicating an invalid timer.

Return conf:0.90 **Q:** What does a return value of 0 from ca_sw_timer_get_remaining() indicate?
A: A return value of 0 indicates that either the timer is not initialized, the handle is invalid, or the timer is not currently active. It does not mean the timer has expired, but rather that the timer state cannot be queried.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_sw_timer_get_remaining()?
A: Common mistakes include calling the function with an invalid handle outside the range [1, CA_TIMER_MAX], using it after the timer has been closed, or calling it from an ISR, which may cause concurrency issues. Also, not ensuring that the timer system is initialized (g_timer_init=true) can lead to incorrect behavior.

`ca_sw_timer_get_state`

Header: `caos/inc/sw_timer.h:151`
Source: `caos/src/sw_timer.c:269:280`
GUARD: `EN_SW_TIMER`

```
ca_sw_timer_state_t ca_sw_timer_get_state(ca_sw_timer_handle_t handle)
```

Retrieves the current operational state of a software timer.

Parametri:

Nome	Tipo	Descrizione
`handle`	`ca_sw_timer_handle_t`	The unique identifier returned by ca_sw_timer_create. Must be a valid handle; CA_TIMER_INVALID or values exceeding CA_TIMER_MAX will result in an error state.

Ritorna:
Returns a ca_sw_timer_state_t enum value. CA_TIMER_RUNNING indicates the timer is active and counting down. CA_TIMER_STOPPED indicates the timer is either inactive, has expired, or the handle provided was invalid.

Note:

- This function is non-blocking and thread-safe, making it safe to call from multiple tasks. It is not intended for use inside Interrupt Service Routines (ISRs) unless explicitly documented as such by the OS architecture, though it is generally safe as it performs a simple memory read. The most common beginner mistake is failing to check if the timer handle is valid before calling this function, or assuming that a 'STOPPED' state implies the timer was never started rather than having simply finished its cycle.

Uso:

- This function is typically used in monitoring loops or diagnostic tasks to check the health or progress of background processes. It is usually paired with ca_sw_timer_create during system initialization, and then used periodically within a while(1) loop alongside ca_sw_timer_get_remaining to display system status or trigger conditional logic based on whether a specific task's timer is still active.

Esempio:

```
/* Assuming htim_led was created via ca_sw_timer_create */
caTimerState state = ca_sw_timer_get_state(htim_led);

if (state == CA_TIMER_RUNNING) {
    /* Timer is active, we can safely check remaining time */
    u32 remaining = ca_sw_timer_get_remaining(htim_led);
    ca_stdout_printf_n("LED timer running, %u ms left", remaining);
} else {
    /* Timer is stopped or expired */
    ca_stdout_printf_n("LED timer is currently stopped");
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_sw_timer_get_state in my application?
A: This function is typically used in monitoring loops or diagnostic tasks to check the health or progress of background processes. It is often used within a while(1) loop, potentially alongside ca_sw_timer_get_remaining, to trigger conditional logic or display system status based on whether a timer is still active.

Code example conf:0.90 **Q:** How do I check the state of a software timer in C?
A: You can check the state by passing the handle to the function: ca_sw_timer_handle_t my_timer = ca_sw_timer_create(...); if (ca_sw_timer_get_state(my_timer) == CA_TIMER_RUN { /* Timer is active */ }

Parameters conf:1.00 **Q:** What are the requirements for the 'handle' parameter?
A: The 'handle' must be a valid ca_sw_timer_handle_t returned by ca_sw_timer_create. Providing CA_TIMER_INVALID or any value exceeding CA_TIMER_MAX will result in an error state.

Return conf:1.00 **Q:** What do the different return values of ca_sw_timer_get_state signify?
A: The function returns CA_TIMER_RUNNING if the timer is active and counting down. It returns CA_TIMER_STOPPED if the timer is inactive, has expired, or if the provided handle was invalid.

Error/Edge case conf:1.00 **Q:** What is a common mistake when using this function?
A: A common mistake is failing to verify if the timer handle is valid before calling the function, or assuming that a 'CA_TIMER_STOPPED' state implies the timer was never started, when it could simply mean the timer has finished its cycle.

ca_sw_timer_init

Header: caos/inc/sw_timer.h:90
Source: caos/src/sw_timer.c:100:134
GUARD: EN_SW_TIMER

```
bool ca_sw_timer_init(void)
```

Initializes the software timer system and spawns the background daemon task.

Ritorna:
Returns true if the system was successfully initialized or was already initialized. Returns false if the internal daemon task could not be created.

Note:

- This function is not thread-safe and must be called once during system startup before any other timer operations are performed. It initializes the internal timer structures and creates a dedicated daemon task that manages the timing logic; if this task fails to start, the timer system will not function. It is not callable from an ISR. A common mistake is attempting to create or start timers before calling this initialization function, which will lead to undefined behavior or system instability.

Uso:

- This function is the mandatory entry point for the software timer subsystem. It is typically called once in the main initialization sequence of the application, before any calls to ca_sw_timer_create or ca_sw_timer_start. It pairs with the rest of the timer API to ensure that the background daemon is running to process timer expirations and execute callbacks.

Esempio:

```
/* Initialize the software timer system before using any timer functions */
if (ca_sw_timer_init()) {
    /* Create a periodic timer that triggers every 500ms */
    /* cb_led is a user-defined callback function */
    u16 timer_handle = ca_sw_timer_create(500, CA_TIMER_PERIODIC, cb_led, NULL);

    if (timer_handle != 0xFFFF) {
        ca_sw_timer_start(timer_handle);
    } else {
        /* Handle error: timer creation failed */
    }
} else {
    /* Handle error: timer system failed to initialize */
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_sw_timer_init() in my caOS application?
A: You should call ca_sw_timer_init() at the beginning of your application, before creating any software timers. It initializes the software timer system and creates a daemon task to manage timer activations. This function must be called from a thread context, not from an ISR.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_sw_timer_init() in a caOS application?
A: bool success = ca_sw_timer_init(); if (success) { // Software timer system is ready } else { // Initialization failed }

Return conf:0.90 **Q:** What does ca_sw_timer_init() return and what causes it to return false?
A: The function returns true if initialization succeeds or if it was already initialized. It returns false if there's an error during daemon task creation or other internal failures. A return value of false indicates that the software timer system could not be initialized properly.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_sw_timer_init()?
A: Common mistakes include calling this function from an ISR context, which is not allowed. Also, calling it from multiple threads simultaneously without synchronization can lead to undefined behavior. Additionally, if the function is called after timers have already been created, it will not reinitialize the system but will return true immediately.

Behavior conf:0.90 **Q:** Is ca_sw_timer_init() blocking or non-blocking, and is it safe to call from an ISR or multiple threads?
A: ca_sw_timer_init() is a non-blocking function. It is not safe to call from an ISR context, as it must be called from a thread context. It is also not thread-safe if called from multiple threads simultaneously, as it modifies internal state without synchronization.

`ca_sw_timer_reset`

Header: `caos/inc/sw_timer.h:122`
Source: `caos/src/sw_timer.c:199:215`
GUARD: `EN_SW_TIMER`

```
bool ca_sw_timer_reset(ca_sw_timer_handle_t handle)
```

Resets a software timer's remaining time to its original period without changing its running or stopped state.

Parametri:

Nome	Tipo	Descrizione
`handle`	`ca_sw_timer_handle_t`	The unique identifier of the timer to reset. Must be a valid handle returned by ca_sw_timer_create. Passing CA_TIMER_INVALID or a value out of range will cause the function to return false.

Ritorna:
Returns true if the timer was successfully reset. Returns false if the timer system is not initialized, the handle is invalid, or the timer is not currently active (not created/already deleted).

Note:

- This function is thread-safe as it uses internal spinlocks to protect the timer structure during the update. It is non-blocking, meaning it will not put the calling task into a wait state. While it can technically be called from an ISR, it is generally intended for task-level logic. The most common mistake is assuming this function starts a stopped timer; it only resets the countdown value. If the timer is currently stopped, it will remain stopped with the period reloaded, and if it is running, it will continue running from the full period duration.

Uso:

- Use this function when you need to restart a timeout period for an active timer, such as a watchdog-style heartbeat or a command-timeout mechanism. It is typically used in a sequence: first, create the timer with ca_sw_timer_create, then start it with ca_sw_timer_start,

and use ca_sw_timer_reset whenever the event associated with the timer occurs to prevent it from expiring. It is often paired with ca_sw_timer_stop if the monitoring logic needs to be suspended entirely.

Esempio:

```c
/* Example: Resetting a communication timeout timer */
caTimerHandle comm_timer;

void task_init(void) {
    /* Create a 5000ms timer */
    comm_timer = ca_sw_timer_create(5000);
    if (comm_timer != CA_TIMER_INVALID) {
        ca_sw_timer_start(comm_timer);
    }
}

void on_data_received(void) {
    /* Reset the timer every time new data arrives to extend the timeout */
    if (!ca_sw_timer_reset(comm_timer)) {
        /* Handle error: timer might have been deleted or not initialized */
        ca_printf("Failed to reset comm_timer!\n");
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_sw_timer_reset() and what is its purpose in a real-time application?
A: You should call ca_sw_timer_reset() when you need to restart the countdown of a software timer without stopping or reinitializing it. This is useful in applications where a timer is used to manage periodic events or timeouts, and you want to reset the timer's countdown to its initial value while keeping it active.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_sw_timer_reset() with caOS types?
A: ca_sw_timer_handle_t handle = ca_sw_timer_create(1000, true); // Create a 1000ms periodic timer bool result = ca_sw_timer_reset(handle); // result will be true if reset was successful

Parameters conf:0.95 **Q:** What does the handle parameter in ca_sw_timer_reset() represent, and what are valid values?
A: The handle parameter is the identifier of the timer to reset. It must be a valid handle between 1 and CA_TIMER_MAX, and must not be CA_TIMER_INVALID. If the handle is NULL or outside this range, the function will fail.

Return conf:0.95 **Q:** What do the return values of ca_sw_timer_reset() indicate, and what causes the function to return false?
A: The function returns true if the timer was successfully reset. It returns false if the timer is not initialized, the handle is invalid, or the timer is not active. These conditions prevent resetting a timer that is not properly set up or currently running.

Error/Edge case conf:0.90 **Q:** What are common mistakes or edge cases when using ca_sw_timer_reset()?
A: Common mistakes include calling ca_sw_timer_reset() on a timer that is not initialized or not active, which will cause the function to return false. Also, passing an invalid handle (e.g., NULL or outside the range 1 to CA_TIMER_MAX) will lead to failure. Avoid resetting non-active timers to prevent unexpected behavior.

Behavior conf:0.90 **Q:** Is ca_sw_timer_reset() blocking or non-blocking, and is it safe to call from an ISR?
A: The function is non-blocking and returns immediately. It is thread-safe due to the use of a spinlock to protect concurrent access. It can be called from an ISR, but only if

the spinlock is compatible with the ISR context to avoid potential deadlocks.

---

ca_sw_timer_set_period

Header: caos/inc/sw_timer.h:130
Source: caos/src/sw_timer.c:217:235
GUARD: EN_SW_TIMER

```
bool ca_sw_timer_set_period(ca_sw_timer_handle_t handle, u32 period_ms)
```

Updates the interval period of an existing software timer without resetting its current countdown.

Parametri:

Nome	Tipo	Descrizione
handle	ca_sw_timer_handle_t	The unique identifier of the timer to modify. Must be a valid handle returned by ca_sw_timer_create and not CA_TIMER_INVALID.
period_ms	u32	The new duration in milliseconds. Must be greater than 0.

Ritorna:
Returns true if the period was successfully updated, false if the handle is invalid, the timer is inactive, the period is 0, or the timer system is not initialized.

Note:

- This function is thread-safe as it uses internal spinlocks to protect the timer structure during the update. It is non-blocking, meaning it returns immediately after updating the internal configuration. Crucially, it does not reset the 'remaining_ms' counter; the new period will only take effect starting from the next cycle of the timer. A common mistake is assuming the timer will immediately restart or reset its current countdown upon calling this function, which is not the case.

Uso:

- This function is typically used in dynamic applications where the frequency of a periodic task needs to change based on system state (e.g., increasing a polling rate when a sensor detects activity). It is used after the timer has been created with ca_sw_timer_create and started with ca_sw_timer_start. It is often paired with ca_sw_timer_get_state to ensure the timer is active before attempting to modify its period.

Esempio:

```
ca_sw_timer_handle_t my_timer;

// Initialize the timer system first
ca_sw_timer_init();

// Create a timer with an initial 1000ms period
my_timer = ca_sw_timer_create(1000);

if (my_timer != CA_TIMER_INVALID) {
    // Start the timer
    ca_sw_timer_start(my_timer);

    // Later, change the period to 500ms dynamically
    if (ca_sw_timer_set_period(my_timer, 500)) {
```

```
        // Period updated successfully
    } else {
        // Handle error: timer might be inactive or invalid
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_sw_timer_set_period to modify a software timer?
A: You should use ca_sw_timer_set_period when you need to dynamically adjust the period of an already active software timer without stopping or restarting it. This is useful in applications where timing requirements change at runtime, such as adjusting LED blink rates or communication timeouts.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_sw_timer_set_period with caOS types?
A: ca_sw_timer_handle_t timer = ca_sw_timer_create(1000); // Create a 1000ms timer bool success = ca_sw_timer_set_period(timer, 500); // Change period to 500ms if (success) { // Timer period updated successfully }

Parameters conf:0.95 **Q:** What are the valid ranges for the handle and period_ms parameters in ca_sw_timer_set_period?
A: The handle must be a valid timer identifier between 1 and CA_TIMER_MAX, and must not be CA_TIMER_INVALID. The period_ms must be greater than 0; otherwise, the function returns false. Values that are too large may cause overflow or unexpected behavior.

Return conf:0.95 **Q:** What does it mean when ca_sw_timer_set_period returns false?
A: A return value of false indicates that the operation failed. This can happen if the timer is not initialized, the handle is invalid, the timer is not active, or if the period_ms is zero.

Error/Edge case conf:0.90 **Q:** What are common mistakes when calling ca_sw_timer_set_period?
A: Common mistakes include passing an invalid handle (e.g., CA_TIMER_INVALID or out of range), passing a period of zero, or calling the function on a timer that is not active. Not checking the return value can also lead to unexpected behavior.

Behavior conf:0.90 **Q:** Is ca_sw_timer_set_period blocking or thread-safe, and can it be called from an ISR?
A: ca_sw_timer_set_period is non-blocking and thread-safe, using a spinlock to protect access to timer data. It can be called from an ISR, but only if the spinlock is compatible with the ISR context to avoid potential deadlocks.

`ca_sw_timer_start`

Header: `caos/inc/sw_timer.h:108`
Source: `caos/src/sw_timer.c:164:180`
GUARD: `EN_SW_TIMER`

```
bool ca_sw_timer_start(ca_sw_timer_handle_t handle)
```

Starts a previously created software timer, enabling it to begin counting down its period.

Parametri:

Nome	Tipo	Descrizione
`handle`	`ca_sw_timer_handle_t`	The unique identifier returned by ca_sw_timer_create. Must be a valid handle (not CA_TIMER_INVALID) and within the range [1, CA_TIMER_MAX].

Ritorna:
Returns true if the timer was successfully started. Returns false if the system is not initialized, the handle is invalid, or the timer is not in an active state.

Note:

- This function is thread-safe as it uses internal spinlocks to protect the timer state during the update. It is non-blocking, meaning it returns immediately after updating the timer's internal state. You must ensure the timer has been created via ca_sw_timer_create before calling this; attempting to start an uninitialized or deleted timer will fail. A common mistake is forgetting to check the return value, which can hide errors if the timer handle was already invalidated or if the system timer module failed to initialize. Note that this function is not intended to be called from an ISR, as it uses spinlocks that may conflict with system-level locking mechanisms.

Uso:

- This function is typically used to trigger periodic tasks or one-shot delays in the background. It is usually paired with ca_sw_timer_create during system initialization to allocate the timer, and ca_sw_timer_stop or ca_sw_timer_delete when the task is no longer needed. In a typical workflow, you create the timer once, then call ca_sw_timer_start whenever you need to restart the countdown, such as resetting a watchdog-style timer or re-triggering a periodic sensor sampling task.

Esempio:

```
ca_sw_timer_handle_t my_timer = CA_TIMER_INVALID;

// Initialize the timer once
my_timer = ca_sw_timer_create(1000, 0, my_callback_function, NULL);

if (my_timer != CA_TIMER_INVALID) {
    // Start the timer to begin the 1000ms countdown
    if (ca_sw_timer_start(my_timer)) {
        ca_printf_n("Timer started successfully");
    } else {
        // Log error using standard caOS output
        ca_printf_n("[ERROR] Failed to start timer");
    }
} else {
    ca_printf_n("[ERROR] Timer creation failed");
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_sw_timer_start() in my embedded application?
A: You should call ca_sw_timer_start() when you want to begin counting time on a previously created and stopped software timer. This function is typically used after initializing a timer with ca_sw_timer_create() and stopping it with ca_sw_timer_stop(). It's useful for implementing timeouts, delays, or periodic events in your application.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_sw_timer_start()?
A: ca_sw_timer_handle_t timer = ca_sw_timer_create(1000); // Create a 1000ms timer bool result = ca_sw_timer_start(timer); // Start the timer if (result) { // Timer started successfully } else { // Handle error }

Parameters conf:0.95 **Q:** What does the handle parameter in ca_sw_timer_start() represent and what are valid values?
A: The handle parameter is an identifier for a software timer created with ca_sw_timer_create(). Valid values are integers between 1 and CA_TIMER_MAX inclusive. Any value outside this range will cause the function to return false.

Return conf:0.95 **Q:** What do the return values of ca_sw_timer_start() indicate?
A: The function returns true if the timer was successfully started. It returns false if the handle is invalid, the timer is not initialized, or the timer is not in an active state. These failures usually indicate incorrect usage or system misconfiguration.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_sw_timer_start()?
A: Common mistakes include using an invalid handle (outside the range 1 to CA_TIMER_MAX), calling the function on a timer that was never created, or not verifying that the timer system is initialized before calling this function. Also, failing to check the return value can lead to unhandled errors.

Behavior conf:0.85 **Q:** Is ca_sw_timer_start() blocking or non-blocking, and is it safe to call from an ISR?
A: The function is non-blocking and returns immediately. It is thread-safe, using a spinlock to protect shared data. It can be called from an ISR, but care must be taken to ensure that the spinlock mechanism is compatible with the ISR environment.

`ca_sw_timer_stop`

Header: `caos/inc/sw_timer.h:115`
Source: `caos/src/sw_timer.c:182:197`
GUARD: `EN_SW_TIMER`

```
1 bool ca_sw_timer_stop(ca_sw_timer_handle_t handle)
```

Stops a running software timer, preventing its callback from executing.

Parametri:

Nome	Tipo	Descrizione
`handle`	`ca_sw_timer_handle_t`	The unique identifier of the timer to stop. Must be a valid handle returned by ca_sw_timer_create. Passing CA_TIMER_INVALID or an out-of-bounds index will result in failure.

Ritorna:
Returns true if the timer was successfully stopped. Returns false if the handle is invalid, the timer was not active, or the timer system is not initialized.

Note:

- This function is thread-safe as it uses a spinlock to protect the internal timer state during the update. It is non-blocking, meaning it returns immediately after updating the state. While it is technically safe to call from an ISR due to the spinlock implementation, it is generally recommended to use it from task context to manage application-level logic. A common mistake is attempting to stop a timer that has already expired or was never started, which will return false; always check the return value if your logic depends on the timer being successfully halted.

Uso:

- This function is used to cancel pending periodic or one-shot events before they trigger. It is typically paired with ca_sw_timer_start to manage state machines or timeout mechanisms. In a typical lifecycle, you create a timer with ca_sw_timer_create, start it when an operation begins, and call ca_sw_timer_stop if the operation completes early or is aborted by the user, preventing the callback from firing unexpectedly.

Esempio:

```
ca_sw_timer_handle_t my_timer;

// Initialize the timer system first
if (ca_sw_timer_init()) {
    my_timer = ca_sw_timer_create(5000, my_callback_func);
}

// Start the timer
ca_sw_timer_start(my_timer);

// ... later, if an event occurs that requires stopping the timer ...
if (ca_sw_timer_stop(my_timer)) {
    ca_printf_n("Timer stopped successfully");
} else {
    ca_printf_n("Failed to stop timer: already stopped or invalid handle");
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_sw_timer_stop() in a real-time embedded application?
A: You should call ca_sw_timer_stop() when you need to disable a software timer that was previously started with ca_sw_timer_start(). This is useful for stopping periodic tasks or one-shot timers that are no longer needed, such as when a system state changes or a resource is released.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_sw_timer_stop() with caOS types?
A: ca_sw_timer_handle_t handle = ca_sw_timer_start(1000, true, my_callback); // Start a periodic timer bool result = ca_sw_timer_stop(handle); // Stop the timer if (result) { // Timer stopped successfully }

Parameters conf:0.95 **Q:** What does the handle parameter in ca_sw_timer_stop() represent and what are valid values?
A: The handle parameter is the identifier of the timer to stop. It must be a valid value between 1 and CA_TIMER_MAX. If the handle is CA_TIMER_INVALID or outside this range, the function will return false.

Return conf:0.95 **Q:** What do the return values of ca_sw_timer_stop() indicate?
A: The function returns true if the timer was successfully stopped. It returns false if the function failed, which can happen due to an invalid handle, a timer that is not initialized, or a timer that is already stopped.

Error/Edge case conf:0.90 **Q:** What are common mistakes or edge cases when using ca_sw_timer_stop()?
A: Common mistakes include calling ca_sw_timer_stop() on a timer that was never initialized or is already stopped, or not checking the return value. Calling the function with an invalid handle (CA_TIMER_INVALID or out of range) will also cause it to return false.

Behavior conf:0.85 **Q:** Is ca_sw_timer_stop() blocking or non-blocking, and is it safe to call from an ISR?
A: The function is non-blocking and returns immediately. It is thread-safe due to the use of a spinlock g_timer_lock. It can be called from an ISR, but only if the spinlock g_timer_lock is also safe to use in ISR context, which must be verified in the implementation.

`ca_syslog_dropped`

Header: `caos/net/inc/syslog_service.h:93`
Source: `caos/net/src/syslog_service.c:182:185`

GUARD: `EN_SYSLOG_SERVICE`

```
u32 ca_syslog_dropped(void)
```

Returns the total count of syslog messages that failed to be queued due to a full buffer.

Ritorna:
Returns a u32 value representing the cumulative number of dropped syslog messages since system boot or last reset.

Note:

- This function is non-blocking and thread-safe as it simply returns the current value of a static counter. It is safe to call from both tasks and ISRs. The counter increments only when the syslog queue is full and a 'trysend' operation fails. A high or rapidly increasing value indicates that the syslog service is unable to process logs as fast as they are being generated, suggesting a need to increase the queue size or optimize the logging frequency.

Uso:

- This function is typically used for system health monitoring and diagnostics. It is often called within a periodic status report task or a CLI command handler to provide visibility into system performance. Developers use it to verify if the logging subsystem is keeping up with the application's demand, often pairing it with other diagnostic functions like ca_net_stats or ca_memory_get_task_usage to get a complete picture of system stability.

Esempio:

```
/* Example: Periodic system health check */
void task_health_monitor(void *pvParameters) {
    while (1) {
        u32 dropped = ca_syslog_dropped();

        /* Report dropped logs if any occurred */
        if (dropped > 0) {
            ca_stdout_printf_n("Warning: Syslog dropped %u messages", dropped);
        }

        /* Wait for 5 seconds before next check */
        ca_os_delay_ms(5000);
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should a developer call ca_syslog_dropped() and why is it important to monitor its return value?
A: A developer should call ca_syslog_dropped() to monitor the number of log messages that were dropped due to a full logging queue. This function is important because it provides visibility into system congestion or logging performance issues. The return value indicates whether any messages were lost, which can help in tuning logging verbosity or identifying potential bottlenecks in the system. It can be called from any context, including ISRs, making it suitable for real-time monitoring.

Code example conf:0.90 **Q:** Can you provide a minimal C code example that uses ca_syslog_dropped() with caOS types?
A: u32 dropped = ca_syslog_dropped(); if (dropped > 0) { // Handle dropped messages, e.g., log or alert // Note: This is a simplified example using caOS types }

Return conf:0.95 **Q:** What do the return values of ca_syslog_dropped() indicate, and what causes the function to return a non-zero value?

A: The function returns a u32 value representing the total number of dropped log messages. A return value of 0 means no messages were dropped, indicating the logging queue had sufficient capacity. A non-zero return value indicates that at least one message was dropped because the queue was full when an attempt to send a log message failed. This failure occurs specifically when ca_queue_static_trysend is used and cannot enqueue a message due to lack of space.

---

ca_syslog_send

Header: caos/net/inc/syslog_service.h:88
Source: caos/net/src/syslog_service.c:168:180
GUARD: EN_SYSLOG_SERVICE

```
void ca_syslog_send(u8 severity, const char *msg)
```

Sends a log message to the system logging service queue for remote transmission.

Parametri:

Nome	Tipo	Descrizione
severity	u8	The priority level of the log (e.g., SYSLOG_SEV_INFO, SYSLOG_SEV_ERROR). Must be a valid severity constant defined in the system headers.
msg	const char *	Pointer to the null-terminated string to be logged. If NULL, the function returns immediately without action.

Ritorna:
void. This function does not return a status code; if the internal queue is full, the message is silently dropped and the internal drop counter is incremented.

Note:

- This function is non-blocking and thread-safe, as it uses a try-send mechanism on the internal queue. It is safe to call from tasks, but be cautious when calling from high-frequency ISRs as it performs string copying which consumes CPU cycles. The function requires the syslog service to be initialized via ca_syslog_service_init before use. A common mistake is assuming the message is delivered immediately; because it uses a queue, the message is only buffered and will be sent later by the syslog background task.

Uso:

- Used to report system events, errors, or status updates to a remote syslog server. It is typically used after the network stack is ready (ca_syslog_service_net_ready). The standard lifecycle involves initializing the service at startup, setting the server address with ca_syslog_service_set_server, and then calling ca_syslog_send whenever an event occurs. If you notice logs are missing, check ca_syslog_dropped to see if the queue is overflowing.

Esempio:

```
/* Initialize the syslog service during system startup */
ca_syslog_service_init();
ca_syslog_service_set_server("192.168.1.100");

/* Later, when the network is ready */
ca_syslog_service_net_ready();

```

```c
/* Example of logging an event */
void log_event(void) {
    char buffer[64];
    /* Format the message into a local buffer */
    ca_snprintf(buffer, sizeof(buffer), "System event occurred at tick %u", (unsigned int)ca_os_tick_64());

    /* Send the log to the queue */
    ca_syslog_send(SYSLOG_SEV_INFO, buffer);

    /* Check if any logs were dropped due to queue overflow */
    if (ca_syslog_dropped() > 0) {
        /* Handle overflow: perhaps increase queue size or reduce logging frequency */
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_syslog_send in a caOS application?
A: You should use ca_syslog_send to log messages with a specified severity level. It is typically used for debugging, monitoring system state, or reporting errors in a non-blocking way. It is not suitable for ISR contexts due to potential blocking behavior.

Code example conf:0.90 **Q:** Can you provide a minimal example of how to use ca_syslog_send with caOS types?
A: u8 severity = 2; cstr msg = "System initialized successfully"; ca_syslog_send(severity, msg);

Parameters conf:0.85 **Q:** What does the severity parameter in ca_syslog_send represent and what are valid values?
A: The severity parameter indicates the importance or level of the log message. Valid values are typically 0-7, though exact values depend on the implementation. The parameter is of type u8 and must be a valid integer within the expected range.

Return conf:0.95 **Q:** Does ca_syslog_send return any value, and what does it mean?
A: The function does not return any value. It is non-failing, meaning it will never produce an error or return a failure code. If the internal queue is full, it silently drops the message and increments a drop counter.

Error/Edge case conf:0.90 **Q:** What are common mistakes or edge cases when using ca_syslog_send?
A: A common mistake is passing a NULL pointer for the message, which will cause the function to return without doing anything. Another edge case is calling this function from an ISR, which is not allowed because it may block. Additionally, if the queue is full, messages are silently dropped, which can lead to loss of important debug information.

`ca_syslog_service_init`

Header: `caos/net/inc/syslog_service.h:60`
Source: `caos/net/src/syslog_service.c:135:147`
GUARD: `EN_SYSLOG_SERVICE`

```c
void ca_syslog_service_init(ca_ipv4_t server_ip, ca_mac_t local_mac, u8 facility)
```

Initializes the internal configuration for the Syslog logging service.

Parametri:

Nome	Tipo	Descrizione
`server_ip`	`ca_ipv4_t`	The IPv4 address of the remote Syslog server. Must be a valid structure containing the server's IP.
`local_mac`	`ca_mac_t`	The MAC address of the local network interface. Used for identifying the source of logs.
`facility`	`u8`	The Syslog facility code (e.g., SYSLOG_FAC_LOCAL0) defining the category of the application logs.

Ritorna:
This function returns void and does not provide a status code.

Note:

- This function is non-blocking and performs internal variable initialization, including setting up a mutex for configuration thread-safety. It is not intended to be called from an ISR, as it performs mutex initialization which is not ISR-safe. The most common mistake is calling this function before the network stack or the OS scheduler is fully prepared, or attempting to send logs before the service has been properly initialized. Ensure that the provided IP and MAC structures are correctly populated before passing them to this function.

Uso:

- This function is typically called during the system startup phase, immediately after network hardware initialization but before the main application loop begins. It is part of the standard initialization sequence: ca_dhcp_service_init -> ca_syslog_service_init -> ca_syslog_service_net_ready. It must be called once to prepare the internal state before any calls to ca_syslog_send are made. If the system requires dynamic reconfiguration of the Syslog server, this function may be called again, provided the application logic handles the potential race conditions.

Esempio:

```c
/* Example of initializing the Syslog service during system startup */
#include "caos.h"

void app_init(void) {
    ca_ipv4_t syslog_srv = { .v = {192, 168, 1, 100} };
    ca_mac_t my_mac = { .v = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55} };

    /* Initialize the service with server IP, local MAC, and facility code */
    ca_syslog_service_init(syslog_srv, my_mac, 16); /* 16 is typically LOCAL0 */

    /* After init, the service waits for network readiness before sending */
    if (ca_syslog_service_net_ready()) {
        /* Proceed to send logs */
        ca_syslog_send("System initialized successfully");
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_syslog_service_init in my caOS application?
A: You should call ca_syslog_service_init after initializing the network stack in your caOS application. This function sets up the syslog service to send logs to a remote server via UDP, so it must be called after network initialization but before attempting to send any syslog messages.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_syslog_service_init with caOS types?

A: ca_ipv4_t server_ip = {192, 168, 1, 100}; mac_t local_mac = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55}; ca_syslog_service_init(server_ip, local_mac, 16);

Parameters conf:0.95 **Q:** What are the valid ranges for the facility parameter in ca_syslog_service_init?

A: The facility parameter must be a value between 0 and 23 inclusive. Values outside this range will cause undefined behavior according to the syslog standard.

Return conf:0.90 **Q:** Does ca_syslog_service_init return any value, and what does it mean?

A: The function does not return any value. It is declared as void, meaning it cannot fail in a way that would allow error checking. The function is designed to always succeed if called with valid parameters.

Error/Edge case conf:0.85 **Q:** What happens if I pass an invalid server IP or MAC address to ca_syslog_service_init?

A: Passing an invalid server IP (like 0.0.0.0) or an invalid MAC address (like an array with fewer than 6 bytes) will cause the syslog service to not function properly. The function does not validate these inputs and will not fail, but the service may not send logs correctly.

---

ca_syslog_service_net_ready

Header: `caos/net/inc/syslog_service.h:78`
Source: `caos/net/src/syslog_service.c:163:166`
GUARD: `EN_SYSLOG_SERVICE`

```
void ca_syslog_service_net_ready(void)
```

Signals to the syslog service that the network interface is initialized and ready for transmission.

Ritorna:
void

Note:

- This function is a non-blocking, thread-safe flag setter. It is designed to be called once the network stack has successfully acquired an IP address or established a link. It is safe to call from an ISR, as it only performs a simple boolean assignment. The most common mistake is calling this before the network stack is actually ready, which may cause the syslog task to attempt sending packets over an uninitialized interface, leading to dropped logs or system instability.

Uso:

- This function is used in the initialization sequence of network-dependent services. It is typically called after the network stack (e.g., DHCP or static IP configuration) has confirmed a valid connection. It acts as a gatekeeper, ensuring that the syslog task does not attempt to transmit data until the underlying network hardware and protocol stack are fully operational. It is usually paired with ca_syslog_service_start to initialize the task and then this function to enable its network-dependent operations.

Esempio:

```
/* Initialize the syslog service task */
ca_syslog_service_start();

/* ... network configuration logic ... */

/* Once the IP is assigned and the network is ready */
ca_syslog_service_net_ready();
```

```

/* Now it is safe to send logs */
ca_syslog_send(0, "System network is now online and ready for logging.");
```

Q&A — fine-tuning review *2/2 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_syslog_service_net_ready() and why is it important for the syslog task?
A: You should call ca_syslog_service_net_ready() when the network interface is ready to send log messages. This function sets a global flag 's_net_ready' which is used as a condition to unblock the syslog task. It allows the syslog task to start sending logs once the network is operational, ensuring that logging messages are not lost during system initialization. The function can be called from any context including ISRs and is non-blocking.

Code example conf:0.85 **Q:** Can you provide a minimal C example showing how to use ca_syslog_service_net_r with caOS types?
A: void example_syslog_net_ready(void) { // Simulate network initialization completion ca_syslog_service_net_ready(); // At this point, syslog task can begin sending logs }

`ca_syslog_service_set_server`

Header: `caos/net/inc/syslog_service.h:66`
Source: `caos/net/src/syslog_service.c:149:155`
GUARD: `EN_SYSLOG_SERVICE`

```
void ca_syslog_service_set_server(ca_ipv4_t server_ip)
```

Updates the destination IPv4 address for the Syslog service and resets the cached MAC address.

Parametri:

Nome	Tipo	Descrizione
`server_ip`	`ca_ipv4_t`	The target IPv4 address of the Syslog server. Ensure this is a valid network-byte-order address.

Ritorna:
void

Note:

- This function is thread-safe because it uses a mutex to protect the internal configuration variables. It is a blocking function because it waits to acquire the mutex, so it must NEVER be called from an ISR (Interrupt Service Routine). A common mistake is calling this function before the Syslog service has been initialized, which may lead to undefined behavior or race conditions if the service is already attempting to send logs. Always ensure the network stack is ready before updating the server address.

Uso:

- This function is typically used during system configuration or when a user changes the Syslog server destination via a CLI command. It is usually paired with ca_syslog_service_init to set up the service, and ca_syslog_send to transmit logs. By setting the internal MAC cache to false, it forces the network stack to perform a new ARP resolution for the new IP address, ensuring that logs are routed correctly to the updated server.

Esempio:

```c
/* Example: Updating the Syslog server address via CLI */
#include "caos.h"

void update_syslog_config(void) {
    ca_ipv4_t new_server = {192, 168, 1, 50}; /* Define target IP */

    /* Ensure the service is initialized before setting the server */
    ca_syslog_service_init();

    /* Update the server address; this will trigger a new ARP resolution */
    ca_syslog_service_set_server(new_server);

    /* Now the service is ready to send logs to the new destination */
    ca_syslog_send("Syslog server updated successfully");
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_syslog_service_set_server and why is it important for syslog functionality?
A: You should call ca_syslog_service_set_server to configure the destination IP address of the syslog server that your caOS application will send log messages to. This function is essential for enabling syslog logging capabilities in your embedded application. It must be called before any syslog messages are sent, and it's typically called during system initialization or configuration setup.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_syslog_service_set_server with caOS types?
A: ca_ipv4_t server_ip = 0x0A000001; /* 10.0.0.1 */ ca_syslog_service_set_server(server_ip);

Parameters conf:0.90 **Q:** What is the meaning of the server_ip parameter in ca_syslog_service_set_server, and what are valid and invalid values?
A: The server_ip parameter specifies the IPv4 address of the syslog server in network byte order. Valid values are any non-zero IPv4 address represented as a u32 value (e.g., 0x0A000001 for 10.0.0.1). Invalid values include 0x00000000 (null IP) and any NULL value, as these will cause undefined behavior or potential system crashes.

`ca_syslog_service_start`

Header: `caos/net/inc/syslog_service.h:72`
Source: `caos/net/src/syslog_service.c:157:161`
GUARD: `EN_SYSLOG_SERVICE`

```c
void ca_syslog_service_start(void)
```

Initializes and launches the background task responsible for sending system logs to a remote server.

Ritorna:
This function returns void and does not provide a status code; it assumes the underlying task creation will succeed.

Note:

- This function is non-blocking as it merely registers a new task with the RTOS scheduler. It is not thread-safe to call this from an ISR, as it involves memory allocation and task list management within the kernel. Before calling this, you must ensure the system memory and the syslog service have been initialized via ca_syslog_service_init. A common

beginner mistake is calling this function multiple times during runtime, which may lead to multiple instances of the syslog task competing for resources or causing memory leaks.

Uso:

- This function is typically called during the system startup phase inside ca_os_main, alongside other network service starters like ca_arp_service_start and ca_dhcp_service_start. It is part of the initialization sequence that prepares the device for network communication. Once started, the syslog task runs independently in the background, waiting for log messages to be sent via ca_syslog_send. It is rarely called again unless the service needs to be explicitly restarted after a critical failure.

Esempio:

```c
void ca_os_main(void) {
    /* Initialize memory and basic system services first */
    ca_memory_init();
    ca_syslog_service_init();

    /* Start the syslog service task */
    ca_syslog_service_start();

    /* Start other network services */
    ca_arp_service_start();
    ca_dhcp_service_start();

    /* Start the OS scheduler to begin task execution */
    ca_os_run();
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should ca_syslog_service_start() be called in a caOS application, and why is timing important?
A: The function should be called after the network resources are ready, as it starts a non-blocking RTOS task to send system logs to a remote syslog server. It must not be called from an ISR and requires that the RTOS context is already initialized. Calling it before the network is ready can cause a crash or undefined behavior.

Code example conf:0.85 **Q:** Can you provide a minimal C code example using caOS types that demonstrates how to start the syslog service?
A: u32 result = ca_syslog_service_start(); // No return value check is needed as the function fails silently. // The function is non-blocking and must be called after network initialization.

Return conf:0.90 **Q:** What happens if ca_syslog_service_start() fails to create the syslog task, and what causes such a failure?
A: The function does not return any value and fails silently if the task cannot be created. The most common cause of failure is calling the function before the network system is ready, which can lead to a crash or undefined behavior.

`ca_system_banner`

Header: `caos/inc/caos.h:151`
Source: `caos/src/caos.c:37:73`
GUARD: `EN_CAOS`

```c
extern u32 ca_system_banner(void *arg)
```

Prints the system identification banner and hardware status information to the configured stdout.

Parametri:

Nome	Tipo	Descrizione
`arg`	`void *`	Reserved for future use. Pass NULL as it is currently ignored by the implementation.

Ritorna:
Always returns 0. The return value is currently unused.

Note:

- This function is not thread-safe and should not be called from an ISR. It performs blocking operations (via ca_os_sleep) when using UART to ensure the transmission buffer is not overwhelmed. A common mistake is calling this function before the stdout subsystem (UART or USB CDC) has been fully initialized, which will result in silent failure or garbled output. When using USB CDC, the function checks for a connection; if the host has not opened the terminal, it exits immediately to prevent data loss, relying on the CDC event handler to trigger a reprint later.

Uso:

- Used primarily during system boot to display firmware version, MCU details, and hardware health metrics. It is also invoked dynamically by the USB CDC event handler when a terminal connection is established to ensure the user sees the banner upon opening the serial port. It pairs with stdout initialization functions and relies on system status functions like ca_os_clock_get_freq and ca_memory_get_total_memory to gather its report data.

Esempio:

```c
/* Example: Triggering the system banner after ensuring stdout is ready */
#include "caos.h"

void display_startup_info(void) {
    /* Ensure the system is ready before printing */
    if (ca_os_running()) {
        /* Print the banner to the default stdout (UART or USB) */
        ca_system_banner(NULL);
    }
}

/* Example: USB CDC connection callback */
void on_usb_connect_event(u32 control) {
    /* If DTR is asserted (bit 0), the terminal is now open */
    if (control & 0x01U) {
        ca_system_banner(NULL);
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_system_banner() in my caOS application?
A: You should call ca_system_banner() during system initialization, typically after the console and USB CDC are initialized. It is used to print system information such as build details, MCU type, clock speed, and memory usage to the console. It is especially useful for debugging and confirming the system configuration at boot time.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_system_banner() with caOS types?
A: u32 banner_result = ca_system_banner(0U, 0U); // Call with dummy parameters // The function always returns 0U and does not indicate errors.

Parameters conf:0.95 **Q:** What do the parameters idx and p1 in ca_system_banner() represent?
A: Both parameters are unused and serve only for compatibility with callback interfaces. Any u32 value can be passed for idx and p1, as they are cast to void and ignored internally.

Return conf:0.95 **Q:** What does ca_system_banner() return, and can it fail?
A: The function always returns 0U. It never fails or indicates errors. Even if the USB CDC terminal is not connected, it returns 0 without printing anything or blocking.

Error/Edge case conf:0.90 **Q:** What are common mistakes when using ca_system_banner()?
A: Common mistakes include calling the function before console or USB CDC initialization, which may result in no output or potential crashes. Also, calling it from an ISR is not allowed, as it uses blocking I/O and ca_os_sleep().

Behavior conf:0.90 **Q:** Is ca_system_banner() blocking or non-blocking, and is it safe to use in ISR or multi-threaded contexts?
A: ca_system_banner() is a blocking function. It uses ca_os_sleep() on UART and performs blocking I/O. It is not ISR-safe and not thread-safe. It must be called during initialization or in a single-threaded context. Precondition: console and USB CDC must be initialized.

`ca_task_set_emergency_exec`

Header: `caos/inc/scheduler.h:367`
Source: `caos/src/scheduler.c:2081:2094`
GUARD: `EN_SCHEDULER`

```
bool ca_task_set_emergency_exec(ca_task_handle_t h, ca_emergency_fn_t
    func, u32 emerg_param)
```

Registers a callback function to be executed when a task encounters a critical failure or emergency state.

Parametri:

Nome	Tipo	Descrizione
`h`	`ca_task_handle_t`	The handle of the target task. Must be a valid handle obtained from ca_os_task_handle or similar.
`func`	`ca_emergency_fn_t`	A pointer to the function to execute. Must match the required signature for emergency handlers.
`emerg_param`	`u32`	A user-defined 32-bit value passed to the emergency function when triggered.

Ritorna:
Returns true if the emergency handler was successfully registered, false otherwise (e.g., invalid handle, task not found, or task memory corrupted).

Note:

- This function is thread-safe and uses internal interrupt disabling to ensure atomic updates to the task control block. It is not intended to be called from an ISR, although the internal implementation handles the critical section safely. A common mistake is passing an uninitialized or NULL handle, which will cause the function to return false. Ensure the task is fully initialized before attempting to set its emergency handler, as the function checks for a valid task canary to prevent operating on corrupted memory.

Uso:

- This function is typically used during the system initialization phase or when a task is dynamically created to define a 'last resort' recovery routine. It is often paired with task creation logic to ensure that if a task crashes or hits a watchdog timeout, the system can execute a cleanup routine (like closing files or resetting hardware peripherals) before the system performs a full reset. It is part of the robust error-handling strategy in caOS.

Esempio:

```c
/* Define the emergency handler function */
void my_emergency_handler(u32 param) {
    /* Perform safe cleanup, e.g., turn off critical hardware */
    ca_gpio_write(1, 0);
}

void setup_task_safety(ca_task_handle_t task_h) {
    /* Register the emergency handler for the specific task */
    bool success = ca_task_set_emergency_exec(task_h, my_emergency_handler, 0
    xDEADBEEF);

    if (!success) {
        /* Handle registration failure: log error or enter safe state */
        ca_printf("Failed to set emergency handler!\n");
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_task_set_emergency_exec and what is its purpose in a caOS application?
A: You should call ca_task_set_emergency_exec when you want to define a specific emergency handler function for a task in case of a critical error. This function allows you to set up a custom recovery or logging mechanism that will be invoked if the task encounters an error. It's useful for implementing robust error handling in embedded systems where a task failure might require special actions like system reset, logging, or switching to a safe state.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_task_set_emergency_exec with caOS types?
A: void emergency_handler(void) { // Custom emergency logic here } int main(void) { u32 task_idx = 0; bool result = ca_task_set_emergency_exec(task_idx, emergency_handler, 0x12345678); return result ? 0 : 1; }

Parameters conf:0.90 **Q:** What are the valid and invalid values for the idx parameter in ca_task_set_emergency
A: The idx parameter must be a valid index between 0 and the maximum number of tasks minus one. Values that are negative or greater than or equal to the maximum number of tasks will cause the function to fail and return false. The index must refer to an initialized and valid task, as verified by the 'digest' field.

Return conf:0.85 **Q:** What does a return value of false from ca_task_set_emergency_exec indicate, and what can cause it?
A: A return value of false indicates that the operation failed. This can happen if the task index is invalid, the task is not initialized or is corrupted, or if the function is called in an invalid context such as from an ISR. It also fails if the task is not valid or if the function pointer is NULL and no valid emergency function is provided.

`ca_task_swap_param`

Header: `caos/inc/scheduler.h:343`
Source: `caos/src/scheduler.c:1593:1600`

GUARD: `EN_SCHEDULER`

```
void ca_task_swap_param(void)
```

Swaps the current task's primary parameter (p1) with its emergency parameter (emerg_param).

Ritorna:
This function returns nothing (void). If no task is currently running, the function performs no operation.

Note:

- This function is non-blocking and executes in constant time. It is thread-safe in the sense that it only modifies the context of the currently executing task, but it should not be called from an ISR as it relies on the global 'g_current_task' pointer which may be inconsistent during context switches. The primary purpose is to allow a task to temporarily switch its operational context or data pointer during an emergency or error-handling routine. A common mistake is calling this function without having previously configured an emergency parameter using ca_task_set_emergency_exec, which would result in swapping the primary parameter with an uninitialized or zero value.

Uso:

- This function is typically used within a task's internal logic or an emergency callback function to switch between a 'normal' operating mode and an 'emergency' mode. By swapping the parameters, the task can quickly redirect its processing logic to use a different data buffer or configuration set without needing to re-initialize its entire state. It is often paired with ca_task_set_emergency_exec during the task initialization phase to define the fallback parameter, and then invoked when the task detects a fault or a specific system event that requires a change in behavior.

Esempio:

```c
/* Define a task that uses an emergency parameter for error recovery */
void my_task_function(void *arg) {
    /* Set the emergency parameter to a specific error-handling buffer */
    ca_task_set_emergency_exec(NULL, (void*)0xDEADBEEF);

    while (1) {
        /* Normal processing logic */
        if (/* error condition detected */) {
            /* Swap p1 with the emergency parameter to switch context */
            ca_task_swap_param();

            /* Perform recovery logic using the new parameter */
            /* ... */

            /* Swap back to restore normal operation */
            ca_task_swap_param();
        }
        ca_os_delay_ms(100);
    }
}
```

Q&A — fine-tuning review *2/2 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_task_swap_param() and what is the intended purpose of swapping p1 and emerg_param?
A: You should call ca_task_swap_param() when you need to pass data between execution contexts of the same task without losing information. It is useful for scenarios where a task needs to preserve its parameter values during a context switch or when

transitioning between different execution modes. The function swaps the values of p1 and emerg_param of the current task, allowing data to be retained across these transitions. However, it is important to note that this function is not thread-safe and cannot be called from an ISR.

Code example conf:0.85 **Q:** Can you provide a minimal C code example showing how to use ca_task_swap_param() with caOS types?
A: void example_task(void) { g_current_task->p1 = 42; g_current_task->emerg_param = 100; ca_task_swap_param(); // After swap, p1 and emerg_param have been exchanged // p1 now contains 100, emerg_param contains 42 }

`ca_tcp_abort`

Header: `caos/net/inc/tcp_service.h:107`
Source: `caos/net/src/tcp_service.c:1081:1089`
GUARD: `EN_TCP_SERVICE`

```
void ca_tcp_abort(u8 handle)
```

Immediately terminates a TCP connection by sending a RST packet and freeing the associated resources.

Parametri:

Nome	Tipo	Descrizione
`handle`	`u8`	The connection identifier (1 to TCP_MAX_CONN). Passing 0 or an invalid handle results in no action.

Ritorna:
void

Note:

- This function is non-blocking and thread-safe, as it uses an internal mutex to protect the connection state. It is NOT safe to call from an ISR because it attempts to acquire a mutex. Unlike ca_tcp_close(), which performs a graceful shutdown (FIN handshake), ca_tcp_abort() forces an immediate closure by sending a TCP RST (Reset) packet. This is essential when a connection is stuck in a non-established state (like SYN_SENT) where a graceful close would be ignored by the stack.

Uso:

- Use this function when you need to forcefully reset a connection, such as during error recovery, service restarts, or when a peer becomes unresponsive. It is typically used in the cleanup phase of a service lifecycle, often paired with a memset to clear the service structure after the connection is aborted. It is the standard way to ensure that a stale PCB (Protocol Control Block) does not linger in the system memory or on the network wire.

Esempio:

```
/* Example: Forcefully abort a connection before re-initializing a service */
void reset_tcp_service(u8 *handle) {
    if (*handle != 0U) {
        /* Force immediate termination of the connection */
        ca_tcp_abort(*handle);

        /* Reset the handle to indicate no active connection */
        *handle = 0U;
```

```
    }

    /* Proceed to re-initialize service parameters */
    /* ca_memset(&service_data, 0, sizeof(service_data)); */
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I choose ca_tcp_abort() over ca_tcp_close()?
A: You should use ca_tcp_abort() when you need to forcefully reset a connection, such as during error recovery, service restarts, or when a peer is unresponsive. Unlike ca_tcp_close(), which performs a graceful FIN handshake, ca_tcp_abort() sends a TCP RST packet, making it essential for clearing connections stuck in non-established states like SYN_SENT.

Code example conf:0.90 **Q:** How do I implement a forceful connection cleanup in caOS?
A: To forcefully terminate a connection, call the abort function using the connection handle. Example: { u8 conn_id = 5; ca_tcp_abort(conn_id); }

Parameters conf:1.00 **Q:** What are the valid values for the handle parameter in ca_tcp_abort()?
A: The handle parameter must be a u8 value between 1 and TCP_MAX_CONN. Passing 0 or any value outside this valid range results in no action being taken by the function.

`ca_tcp_accept`

Header: `caos/net/inc/tcp_service.h:140`
Source: `caos/net/src/tcp_service.c:1096:1106`
GUARD: `EN_TCP_SERVICE`

```
u8 ca_tcp_accept(u8 listen_handle, u32 timeout_ms)
```

Blocks the calling task until a new TCP connection request arrives on the specified listening handle.

Parametri:

Nome	Tipo	Descrizione
`listen_handle`	`u8`	The handle returned by a previous call to ca_tcp_listen. Must be in the range [1, TCP_MAX_LISTEN].
`timeout_ms`	`u32`	Maximum time in milliseconds to wait for a connection. Use 0 for no wait or a large value for indefinite blocking.

Ritorna:
Returns a valid connection handle (non-zero) if a connection is accepted. Returns 0 if the timeout expires, the handle is invalid, or the listener is not in blocking mode.

Note:

- This function is blocking; it puts the calling task into a waiting state until a connection is received or the timeout expires. It is NOT thread-safe to call from an ISR, as it relies on semaphore waiting mechanisms that require task context. The most common beginner mistake is passing an invalid handle or failing to check the return value for 0, which leads to attempting operations on a non-existent connection. Ensure the listener has been successfully initialized via ca_tcp_listen before calling this function.

Uso:

- Typically used in a dedicated server task loop. The sequence is: 1) Initialize the listener with ca_tcp_listen, 2) Enter a loop where ca_tcp_accept waits for incoming clients, 3) Process the returned connection handle (e.g., reading/writing data), and 4) Close the connection with ca_tcp_close when finished. It is the standard way to implement TCP servers that handle multiple clients sequentially or by spawning new tasks.

Esempio:

```
u8 lh = ca_tcp_listen(&lcfg); // Initialize listener
if (lh != 0U) {
    while (1) {
        // Wait for a client connection for up to 5 seconds
        u8 conn = ca_tcp_accept(lh, 5000U);
        if (conn != 0U) {
            ca_stdout_printf_n("New connection: %u", (u32)conn);
            // Perform network communication here
            // ...
            ca_tcp_close(conn); // Always close the connection when done
        } else {
            ca_stdout_printf_n("Accept timeout or error");
        }
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_tcp_accept() in a TCP server implementation?
A: You should call ca_tcp_accept() when you want to wait for and accept new incoming TCP connections on a listening socket. It is typically used after creating a listening socket with ca_tcp_listen() to handle multiple client connections sequentially.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_tcp_accept() in a caOS application?
A: u8 listen_sock = ca_tcp_listen(8080, 10); // Create listening socket u8 client_sock = ca_tcp_accept(listen_sock, 5000); // Wait up to 5 seconds for a connection if (client_sock != 0) { // Handle client connection }

Parameters conf:0.95 **Q:** What is the meaning of the listen_handle parameter in ca_tcp_accept() and what are valid values?
A: The listen_handle parameter is the socket handle of an active listening socket created with ca_tcp_listen(). Valid values are integers between 1 and TCP_MAX_LISTEN. If the handle is outside this range, the function fails and returns 0.

Return conf:0.90 **Q:** What does ca_tcp_accept() return when it successfully accepts a connection?
A: If a connection is successfully accepted, ca_tcp_accept() returns a value between 1 and TCP_MAX_CONN, which represents the handle of the newly accepted client socket. If the function fails due to an invalid handle, inactive socket, or timeout, it returns 0.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_tcp_accept()?
A: Common mistakes include using an invalid listen_handle (outside the range 1 to TCP_MAX_LISTEN), calling the function on a non-blocking socket (where on_accept is not NULL), or calling it from an ISR. Additionally, using a timeout value that is too short may cause premature timeouts.

`ca_tcp_close`

Header: `caos/net/inc/tcp_service.h:96`
Source: `caos/net/src/tcp_service.c:1018:1048`

GUARD: `EN_TCP_SERVICE`

```
void ca_tcp_close(u8 handle)
```

Gracefully terminates an established TCP connection by sending a FIN packet.

Parametri:

Nome	Tipo	Descrizione
`handle`	`u8`	The connection identifier returned by ca_tcp_connect or ca_tcp_accept. Valid range is 1 to TCP_MAX_CONN. Passing 0 or an invalid handle results in an immediate return.

Ritorna:
void. This function does not provide feedback on whether the FIN packet was successfully transmitted or acknowledged by the peer.

Note:

- This function is thread-safe as it uses an internal mutex to protect the connection state. It is NOT callable from an ISR because it performs blocking operations (mutex locking). The function initiates a graceful shutdown by transitioning the state to TCP_STATE_FIN_WAIT_1; it does not force an immediate abort. A common mistake is assuming the connection is closed immediately upon return; the TCP stack still needs to process the handshake completion in the background. Ensure the handle is valid and the connection is in an established state before calling, otherwise, the function will silently exit.

Uso:

- Use this function when you are finished with a TCP session to allow the remote peer to close its side cleanly. It is typically called after detecting a connection error, a timeout during reception, or when the application protocol (like HTTP or MQTT) signals the end of a transaction. It is often paired with ca_tcp_connect or ca_tcp_accept during the cleanup phase of a service task. Always ensure that any buffers associated with the connection are handled or freed if your application logic requires it before calling this function.

Esempio:

```
u8 conn = ca_tcp_connect(&cfg);
if (conn != 0xFFU) {
    /* Perform network communication */
    u16 rx_len = 0U;
    /* Attempt to receive data with a timeout */
    if (!ca_tcp_recv(conn, buffer, sizeof(buffer), &rx_len, 1000U)) {
        /* Connection lost or timed out, close it gracefully */
        ca_tcp_close(conn);
    } else {
        /* Process data and close when finished */
        ca_tcp_close(conn);
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_tcp_close() in my TCP application?
A: You should call ca_tcp_close() when you want to properly close a TCP connection identified by a handle. It is typically used after completing data transfer or when the application no longer needs the connection. The function sends a FIN packet to gracefully terminate the connection.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_tcp_close() with caOS types?
A: u8 tcp_handle = 1; ca_tcp_close(tcp_handle);

Parameters conf:0.95 **Q:** What is the valid range for the handle parameter in ca_tcp_close()?
A: The handle must be an unsigned 8-bit integer between 1 and TCP_MAX_CONN (exclusive). If the handle is 0 or greater than or equal to TCP_MAX_CONN, the function will do nothing.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_tcp_close()?
A: Common issues include calling ca_tcp_close() with an invalid handle (0 or >= TCP_MAX_CONN), which results in no action. Additionally, if the buffer for sending the FIN packet is not available, the close operation may silently fail. Also, calling this function from an ISR is not thread-safe and may cause unexpected behavior.

Behavior conf:0.90 **Q:** Is ca_tcp_close() blocking or non-blocking, and is it safe to call from an ISR?
A: ca_tcp_close() is a blocking function due to its use of a mutex. It is not safe to call from an ISR because it is not thread-safe in that context. The function sends a FIN packet and transitions the connection to FIN_WAIT_1 state, but does not guarantee immediate closure.

`ca_tcp_connect`

Header: `caos/net/inc/tcp_service.h:69`
Source: `caos/net/src/tcp_service.c:821:898`
GUARD: `EN_TCP_SERVICE`

```
1 u8 ca_tcp_connect(const ca_tcp_connect_cfg_t *cfg)
```

Initiates a TCP connection to a remote host and returns a handle for subsequent communication.

Parametri:

Nome	Tipo	Descrizione
`cfg`	`const ca_tcp_connect_cfg_t *`	Pointer to a configuration structure containing local/remote IP, ports, and optional callback functions. Must not be NULL.

Ritorna:
Returns a non-zero handle (1 to TCP_MAX_CONN) on success. Returns 0U if the connection fails, the configuration is invalid, or no connection slots are available.

Note:

- This function is thread-safe as it uses internal mutexes to manage connection slots. It is NOT callable from an ISR because it performs blocking operations (like ARP resolution and semaphore waiting). If 'on_connected' is NULL in the configuration, the function will block the calling thread until the connection is established or the timeout expires. A common mistake is failing to check the returned handle for 0U, which leads to passing an invalid handle to subsequent functions like ca_tcp_send or ca_tcp_close.

Uso:

- Typically used in network service tasks (e.g., MQTT, Modbus) to establish a client-side connection. The sequence starts by populating the configuration struct, calling ca_tcp_connect, and checking the returned handle. If successful, the handle is stored and used for data exchange; finally, the connection is terminated using ca_tcp_close when the service shuts down or encounters a fatal error.

Esempio:

```
ca_tcp_connect_cfg_t cfg;
ca_memzero(&cfg, sizeof(cfg));
cfg.local_ip = 0x0A000005; // 10.0.0.5
cfg.remote_ip = 0x0A000001; // 10.0.0.1
cfg.remote_port = 8080;
cfg.connect_timeout_ms = 5000;

// Attempt to connect
u8 handle = ca_tcp_connect(&cfg);

if (handle != 0U) {
    // Connection successful, handle is valid
    // Proceed to send data using ca_tcp_send(handle, ...)
} else {
    // Connection failed, handle is 0
    ca_printf("Failed to connect to server\n");
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_tcp_connect() in a caOS application?
A: You should call ca_tcp_connect() when you need to establish a TCP connection to a remote server or device. It is typically used after initializing the network stack with pcb_init_once and when you have a valid configuration for the connection. It's commonly used in network services like MQTT, Modbus, or TCP-based clients.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_tcp_connect() with caOS types?
A: ca_tcp_connect_cfg_t cfg; cfg.ip = 0xC0A80101; // 192.168.1.1 cfg.port = 80; cfg.timeout = 5000; cfg.on_connected = NULL;
u8 handle = ca_tcp_connect(&cfg); if (handle == 0U || handle == 0xFFU) { // Handle error }

Parameters conf:0.90 **Q:** What does the cfg parameter in ca_tcp_connect() control, and what are valid values?
A: The cfg parameter is a pointer to a ca_tcp_connect_cfg_t structure that defines connection parameters. It must be a valid pointer. The structure includes ip (destination IP address), port (destination port), timeout (connection timeout in milliseconds), and on_connected (callback function). Passing NULL for cfg will cause the function to return 0U.

Return conf:0.95 **Q:** What do the return values of ca_tcp_connect() mean, and what causes each failure?
A: 0U means an error occurred: invalid configuration, allocation failure, timeout, or SYN transmission failure. 0xFFU indicates no connection available or inability to complete the connection. A valid handle (1-254) is returned on success, which can be used for further TCP operations like sending/receiving data.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_tcp_connect()?
A: Common mistakes include passing a NULL pointer for cfg, not initializing the network stack with pcb_init_once before calling the function, not handling timeouts properly, or using the returned handle after closing the connection. Also, calling this function from an ISR is not allowed and may cause undefined behavior.

Behavior conf:0.90 **Q:** Is ca_tcp_connect() blocking or non-blocking, and is it safe to call from an ISR?
A: ca_tcp_connect() is blocking if the 'on_connected' callback in the configuration is NULL. It is not safe to call from an ISR (Interrupt Service Routine) as it's not ISR-safe. However, it is thread-safe due to internal mutex usage for connection management.

ca_tcp_is_connected

Header: caos/net/inc/tcp_service.h:101
Source: caos/net/src/tcp_service.c:1050:1055
GUARD: EN_TCP_SERVICE

```
bool ca_tcp_is_connected(u8 handle)
```

Checks if a specific TCP connection handle is currently active and in the ESTABLISHED state.

Parametri:

Nome	Tipo	Descrizione
handle	u8	The connection identifier returned by ca_tcp_connect or ca_tcp_accept. Valid range is 1 to TCP_MAX_CONN. Passing 0 or an invalid handle returns false.

Ritorna:
Returns true if the connection is fully established and active; returns false if the handle is invalid, the connection is closed, or the state is in transition (e.g., SYN_SENT, FIN_WAIT).

Note:

- This function is non-blocking and thread-safe as it performs a simple read of the internal connection state structure. It is safe to call from an ISR, though typically used in task loops. The most common beginner mistake is assuming that a non-zero handle implies a connected state; you must always verify the connection status with this function before attempting to send or receive data. If this function returns false, the connection is effectively dead and should be cleaned up using ca_tcp_close.

Uso:

- This function is the primary guard for TCP communication loops. It is typically used in a 'while' loop condition to keep a task alive as long as the remote peer maintains the connection. It is used in conjunction with ca_tcp_connect or ca_tcp_accept to verify the handshake completion, and paired with ca_tcp_recv or ca_tcp_send to ensure data operations are performed only on valid sockets. When the function returns false, the application should exit the communication loop and call ca_tcp_close to release resources.

Esempio:

```
u8 h = ca_tcp_connect(&cfg);
if (h != 0xFFU) {
    /* Wait for connection to be established */
    ca_os_delay_ms(100);

    if (ca_tcp_is_connected(h)) {
        ca_stdout_printf_n("Connected! Sending data...");
        /* Perform data exchange */
        while (ca_tcp_is_connected(h)) {
            /* Attempt to receive data with a timeout */
            u16 got = 0U;
            bool ok = ca_tcp_recv(h, rx_buf, 64U, &got, 1000U);
            if (!ok) break;
        }
    }
    /* Always close the handle when finished or disconnected */
    ca_tcp_close(h);
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I check if a TCP connection is established using ca_tcp_is_connected?
A: You should call ca_tcp_is_connected to verify that a TCP socket is in the established state before attempting to send or receive data. This function is useful in scenarios where you need to ensure that a connection is stable before proceeding with network operations, such as in TCP service implementations or in applications that rely on reliable communication.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_tcp_is_connected with caOS types?
A: u8 tcp_handle = 1; bool is_connected = ca_tcp_is_connected(tcp_handle); if (is_connected) { // Proceed with TCP operations } else { // Handle disconnected state }

Parameters conf:0.95 **Q:** What is the significance of the handle parameter in ca_tcp_is_connected, and what values are valid?
A: The handle parameter identifies a specific TCP connection. It must be a value between 1 and TCP_MAX_CONN (inclusive). Values of 0 or greater than TCP_MAX_CONN are considered invalid and will cause the function to return false. The handle must correspond to an initialized TCP connection structure.

Return conf:0.90 **Q:** What does it mean when ca_tcp_is_connected returns false, and what can cause this?
A: ca_tcp_is_connected returns false when the handle is invalid (0 or > TCP_MAX_CONN) or when the connection is not in the TCP_STATE_ESTABLISHED state. This can happen if the connection has not yet been established, has been closed, or if the handle refers to an uninitialized or improperly configured connection.

`ca_tcp_listen`

Header: `caos/net/inc/tcp_service.h:127`
Source: `caos/net/src/tcp_service.c:1061:1086`
GUARD: `EN_TCP_SERVICE`

```
1 u8 ca_tcp_listen(const ca_tcp_listen_cfg_t *cfg)
```

Initializes a TCP passive socket to listen for incoming connection requests on a specific port.

Parametri:

Nome	Tipo	Descrizione
`cfg`	`const ca_tcp_listen_cfg_t *`	Pointer to a configuration structure containing the local IP, port, and optional callback functions. Must not be NULL.

Ritorna:
Returns a non-zero handle (1 to TCP_MAX_LISTEN) on success, which is used to identify this listener in subsequent calls. Returns 0 if no free slots are available or if the input configuration is invalid.

Note:

- This function is thread-safe as it uses an internal spinlock to manage access to the listener table. It is non-blocking and does not perform network I/O itself; it merely registers the intent to listen. It is NOT safe to call from an ISR because it performs internal initialization and locking. A common mistake is failing to check the return value for 0, which leads to passing an invalid handle to ca_tcp_accept, causing the application to hang or fail silently.

Uso:

- This function is the first step in creating a TCP server. It is typically called once during service initialization to reserve a port. After obtaining a valid handle, the application enters a loop calling ca_tcp_accept to wait for incoming clients. When the service is no longer needed, the handle should be released using ca_tcp_unlisten (if available) or by closing the associated connections.

Esempio:

```
ca_tcp_listen_cfg_t lcfg = { .local_port = 8080, .local_ip = 0, .on_accept =
    NULL };
u8 listen_handle = ca_tcp_listen(&lcfg);
if (listen_handle == 0U) {
    ca_stdout_printf_n("Failed to start TCP listener");
} else {
    while (1) {
        /* Wait for a client connection with a 1000ms timeout */
        u8 conn = ca_tcp_accept(listen_handle, 1000U);
        if (conn != 0U) {
            /* Process the connection here */
            ca_tcp_close(conn);
        }
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_tcp_listen in a caOS application?
A: You should call ca_tcp_listen after the system is initialized, specifically after pcb_init_once() has been invoked. It is used to start a TCP server that listens on a specified port for incoming connections. This function is typically called during application initialization to set up network services like HTTP or TCP-based protocols.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_tcp_listen with caOS types?
A: ca_tcp_listen_cfg_t cfg; cfg.port = 80; cfg.ip_addr = 0xC0A80101; // 192.168.1.1 cfg.mac_addr = 0x001122334455; cfg.accept_cb = my_accept_callback; cfg.recv_cb = my_recv_callback; cfg.close_cb = my_close_callback; u8 handle = ca_tcp_listen(&cfg); if (handle == 0) { // Handle error }

Parameters conf:0.90 **Q:** What does the cfg parameter in ca_tcp_listen represent and what are valid values?
A: The cfg parameter is a pointer to a ca_tcp_listen_cfg_t structure that defines the listening configuration including port, IP address, MAC address, and callback functions. It must be a valid, non-NULL pointer. The port should be a valid TCP port number, ip_addr and mac_addr must be properly formatted values, and all callback pointers must be valid function addresses.

Return conf:0.95 **Q:** What do the return values of ca_tcp_listen indicate?
A: A return value of 0U indicates failure, either because cfg is NULL or no listener slots are available. Return values from 1 to TCP_MAX_LISTEN+1 indicate success, with each value representing a unique handle for the created listener. The handle can be used later to manage the listener.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_tcp_listen?
A: Common mistakes include passing NULL as the cfg parameter, which returns 0 and causes subsequent errors. Not checking the return value can lead to using an invalid handle. Also, calling ca_tcp_listen before pcb_init_once() is invoked will likely fail. Additionally, calling this function from an ISR is unsafe due to lock usage and shared memory access.

Behavior conf:0.90 **Q:** Is ca_tcp_listen blocking or thread-safe, and can it be called from an ISR?
A: ca_tcp_listen is not blocking; it acquires a spinlock for mutual exclusion but does not block the calling thread for long periods. It is thread-safe due to the spinlock protecting shared data structures. However, it is not safe to call from an ISR because it uses locks and accesses shared memory, which could lead to deadlocks or corruption.

`ca_tcp_recv`

Header: `caos/net/inc/tcp_service.h:89`
Source: `caos/net/src/tcp_service.c:984:1016`
GUARD: `EN_TCP_SERVICE`

```
1 bool ca_tcp_recv(u8 handle, u8 *buf, u16 buf_len, u16 *out_len, u32
     timeout_ms)
```

Reads incoming TCP data from a specific connection handle into a provided buffer with a timeout.

Parametri:

Nome	Tipo	Descrizione
`handle`	`u8`	The connection identifier returned by ca_tcp_accept or ca_tcp_connect. Must be within [1, TCP_MAX_CONN].
`buf`	`u8 *`	Pointer to the destination buffer where received data will be stored. Must not be NULL.
`buf_len`	`u16`	Size of the destination buffer in bytes.
`out_len`	`u16 *`	Pointer to a variable that will store the actual number of bytes copied. Can be NULL if not needed.
`timeout_ms`	`u32`	Maximum time in milliseconds to wait for data to arrive. Use 0 for no wait or a large value for indefinite blocking.

Ritorna:
Returns true if data was successfully received and copied, false otherwise. A false return indicates a timeout, an invalid handle, a closed connection, or a NULL buffer pointer.

Note:

- This function is blocking; it will suspend the calling task until data arrives or the timeout expires. It is NOT callable from an ISR because it uses semaphores and mutexes that may cause the task to sleep. Ensure the TCP connection is active and established before calling. A common mistake is passing a buffer that is too small for the incoming packet, which results in truncated data, or failing to check the return value, which leads to processing stale or uninitialized data when the connection has actually closed.

Uso:

- Typically used in a dedicated communication task loop. It is usually called after a connection is established via ca_tcp_accept or ca_tcp_connect. It is often paired with ca_tcp_close to terminate the session if the receive operation fails or returns zero bytes, indicating the remote peer has disconnected.

Esempio:

```c
u8 rx_buffer[128];
u16 received_bytes = 0;
// Assume 'conn_handle' is a valid active TCP connection
if (ca_tcp_recv(conn_handle, rx_buffer, sizeof(rx_buffer), &received_bytes,
    1000)) {
    if (received_bytes > 0) {
        // Process the received data
        ca_printf("Received %u bytes\n", received_bytes);
    }
} else {
    // Handle timeout or connection error
    ca_tcp_close(conn_handle);
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_tcp_recv in a TCP client application?
A: You should use ca_tcp_recv when you need to read data from an active TCP connection. It is typically called after establishing a TCP connection and when you expect to receive data from a remote server or peer. The function blocks until data is available or the timeout expires.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_tcp_recv with proper caOS types?
A: u8 handle = 1; u8 buffer[1024]; u16 bytes_received; bool result = ca_tcp_recv(handle, buffer, sizeof(buffer), &bytes_received, 5000); if (result) { /* Process received data */ }

Parameters conf:0.90 **Q:** What are the valid ranges for the handle and buf_len parameters in ca_tcp_recv?
A: The handle must be between 1 and 255 (inclusive), and cannot be 0 or greater than TCP_MAX_CONN. The buf_len must be greater than 0, as a zero-length buffer is invalid for receiving data.

Return conf:0.95 **Q:** What does it mean when ca_tcp_recv returns false?
A: ca_tcp_recv returns false if there was an error, such as an invalid handle, a closed or inactive connection, a NULL buffer, or if the timeout expired without data being received.

Error/Edge case conf:0.90 **Q:** What are common mistakes when calling ca_tcp_recv?
A: Common mistakes include passing an invalid handle (0 or > TCP_MAX_CONN), passing a NULL buffer, calling the function on a closed or inactive connection, or using a handle that was not previously opened with a successful TCP connection.

Behavior conf:0.90 **Q:** Is ca_tcp_recv a blocking or non-blocking function, and can it be called from an ISR?
A: ca_tcp_recv is a blocking function that waits for data or until the timeout expires. It is not safe to call from an ISR, as it is not ISR-safe. It is thread-safe only if the connection state is not modified by multiple threads simultaneously.

`ca_tcp_send`

Header: `caos/net/inc/tcp_service.h:76`
Source: `caos/net/src/tcp_service.c:900:974`
GUARD: `EN_TCP_SERVICE`

```c
bool ca_tcp_send(u8 handle, const u8 *data, u16 len)
```

Sends a buffer of data over an established TCP connection handle.

Parametri:

Nome	Tipo	Descrizione
`handle`	`u8`	The TCP connection identifier returned by ca_tcp_accept or ca_tcp_connect. Must be within [1, TCP_MAX_CONN].
`data`	`const u8 *`	Pointer to the buffer containing the data to be transmitted. Must not be NULL.
`len`	`u16`	Number of bytes to send. Must be greater than 0.

Ritorna:
Returns true if the data was successfully queued for transmission, false if the connection is not established, the handle is invalid, or the internal network buffers are full.

Note:

- This function is thread-safe as it uses an internal mutex to protect the connection state and transmission buffers. It is a blocking operation in the sense that it waits for the mutex, but it does not block until the data is acknowledged by the remote peer; it only queues the data for transmission. It must NOT be called from an ISR because it attempts to lock a mutex. A common mistake is to call this function without checking if the connection is still active or writable, which can lead to silent failures or buffer exhaustion if the network stack is congested.

Uso:

- This function is typically used in a task loop after a connection has been established via ca_tcp_connect or ca_tcp_accept. It is often paired with ca_tcp_wait_writable to ensure the transmission window is open before attempting to send large amounts of data. In a typical sequence, you initialize the connection, wait for the state to be established, send data using ca_tcp_send, and finally call ca_tcp_close when the communication is finished.

Esempio:

```
u8 conn_handle = /* Assume valid handle obtained from ca_tcp_accept */;
const u8 my_data[] = "Hello, World!";

/* Ensure the connection is ready to accept data */
if (ca_tcp_wait_writable(conn_handle, 1000U)) {
    if (ca_tcp_send(conn_handle, my_data, (u16)sizeof(my_data))) {
        /* Data successfully queued for transmission */
    } else {
        /* Handle error: connection might have dropped or buffer full */
    }
} else {
    /* Handle timeout: connection not writable within 1 second */
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_tcp_send in a caOS network application?
A: You should use ca_tcp_send when you need to transmit data over an established TCP connection in a caOS application. It is typically used in network services like HTTP, MQTT, or Modbus to send response data or messages to connected clients. The function is designed for reliable, blocking transmission of data.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_tcp_send with caOS types?
A: u8 handle = 1; u8 buffer[] = {0x01, 0x02, 0x03}; bool result = ca_tcp_send(handle, buffer, 3); if (!result) { /* handle error */ }

Parameters conf:0.90 **Q:** What are the valid ranges for the handle and len parameters in ca_tcp_send?
A: The handle must be between 1 and TCP_MAX_CONN (inclusive), where 0 and values greater than TCP_MAX_CONN are invalid. The len parameter must be greater than 0; a value of 0 will cause the function to return false.

Return conf:0.95 **Q:** What does it mean when ca_tcp_send returns false?
A: ca_tcp_send returns false when the send operation fails. Possible causes include an invalid handle, a connection that is not in ESTABLISHED state, a full buffer, or allocation errors. It also returns false if the handle is 0 or exceeds TCP_MAX_CONN.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_tcp_send?
A: Common mistakes include calling ca_tcp_send without verifying that the connection is in ESTABLISHED state, sending data on a closed or invalid connection, using an invalid handle (0 or > TCP_MAX_CONN), or not checking the return value for errors.

Behavior conf:0.90 **Q:** Is ca_tcp_send thread-safe and can it be called from an ISR?
A: Yes, ca_tcp_send is thread-safe as it uses a mutex to protect internal data structures, allowing it to be safely called from multiple tasks. However, it is not ISR-safe and should not be called from interrupt service routines.

`ca_tcp_service_start`

Header: `caos/net/inc/tcp_service.h:62`
Source: `caos/net/src/tcp_service.c:807:819`
GUARD: `EN_TCP_SERVICE`

```
void ca_tcp_service_start(void)
```

Initializes the TCP/IP stack services, registers the TCP timer task, and optionally enables the CLI interface.

Ritorna:
void

Note:

- This function is not thread-safe and must be called during the system initialization phase, typically inside ca_os_main before starting other network-dependent tasks. It performs internal memory and protocol binding, so calling it multiple times or after the network stack is already active may lead to undefined behavior or memory leaks. It is not callable from an ISR because it spawns OS tasks and performs heap allocations. The most common mistake is calling this function before the underlying network hardware or memory manager has been initialized.

Uso:

- It is used as a mandatory entry point for any application requiring TCP communication. It is typically paired with ca_arp_service_start and ca_dhcp_service_start to ensure the full network stack is ready. Once called, the system is prepared to handle TCP connections, and the developer can proceed to use functions like ca_tcp_listen or ca_tcp_connect to establish communication channels.

Esempio:

```
void ca_os_main(void) {
    /* Initialize memory and basic network services first */
    ca_memory_init();
    ca_arp_service_start();
    ca_dhcp_service_start();
```

```
6
7       /* Start the TCP service to enable socket operations */
8       ca_tcp_service_start();
9
10      /* Now safe to proceed with network-dependent tasks */
11      ca_os_job("app_task", CA_TH_LEVEL3, my_app_task, 0, STACK_SMALL, CA_JOB_STD,
        CA_FAULT_RESTART_THREAD);
12
13      /* Start the OS scheduler */
14      ca_os_run();
15  }
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_tcp_service_start() in my caOS application, and why is it important?
A: You should call ca_tcp_service_start() once during the system startup phase, typically after initializing other network components. It initializes the TCP protocol module, registers the receive handler, and starts the TCP timer task. This function must be called from a task context, not from an ISR, and it's not thread-safe. Calling it more than once may lead to undefined behavior. It's essential for enabling TCP functionality in your caOS-based application.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_tcp_service_start() with caOS types?
A: void example_task(void* arg) { ca_tcp_service_start(); // Other TCP-related operations... while(1) { ca_task_delay(1000); } }

Error/Edge case conf:0.85 **Q:** What are the common mistakes or edge cases when using ca_tcp_service_start()
A: Common mistakes include calling ca_tcp_service_start() from an ISR, which is not allowed, or calling it multiple times during runtime, which could cause undefined behavior. It must be called only once during system initialization from a task context. Since it's not thread-safe, concurrent calls from multiple tasks should be avoided. The function does not return any error codes, so developers must ensure correct usage to prevent system instability.

`ca_tcp_unlisten`

Header: `caos/net/inc/tcp_service.h:133`
Source: `caos/net/src/tcp_service.c:1088:1094`
GUARD: `EN_TCP_SERVICE`

```
1 void ca_tcp_unlisten(u8 listen_handle)
```

Stops a TCP listening socket and releases the associated resources.

Parametri:

Nome	Tipo	Descrizione
`listen_handle`	`u8`	The handle returned by a previous call to ca_tcp_listen. Valid range is 1 to TCP_MAX_LISTEN. Passing 0 or an invalid handle results in no action.

Ritorna:
void

Note:

- This function is thread-safe as it uses a spinlock to protect the internal listen table. It is non-blocking, meaning it will return immediately after updating the internal state. It is safe to call from an ISR because it uses an IRQ-safe spinlock (ca_spinlock_lock_irq). The most common mistake is failing to call this function after a connection is accepted or when a service is shutting down, which leads to resource leaks where the TCP port remains occupied and unavailable for future use.

Uso:

- This function is used to clean up after a TCP server has finished its task or when a listening socket needs to be reconfigured. It is typically paired with ca_tcp_listen during initialization and called before the application exits or when the network configuration changes. In a standard server loop, you call ca_tcp_listen to start, ca_tcp_accept to wait for a client, and then ca_tcp_unlisten to stop accepting new connections once the service is no longer needed.

Esempio:

```
u8 lh = ca_tcp_listen(&lcfg); // Start listening
if (lh != 0U) {
    u8 h = ca_tcp_accept(lh, 5000U); // Wait for client
    ca_tcp_unlisten(lh); // Stop listening once we have a connection or timeout
    if (h != 0U) {
        // Perform data exchange
        ca_tcp_close(h); // Close the active connection
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_tcp_unlisten() and what is its purpose in a TCP server implementation?
A: You should call ca_tcp_unlisten() when you want to stop accepting new TCP connections on a specific port. This function removes an active TCP listening handle, preventing new connections from being established on that port. It is useful for gracefully shutting down a TCP service or dynamically changing listening ports without affecting already established connections.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_tcp_unlisten() with caOS types?
A: u8 listen_handle = 3; ca_tcp_unlisten(listen_handle);

Parameters conf:0.90 **Q:** What does the listen_handle parameter in ca_tcp_unlisten() represent, and what are the valid values?
A: The listen_handle parameter identifies a specific TCP listening instance. Valid values are integers between 1 and TCP_MAX_LISTEN (inclusive). If the value is 0 or greater than TCP_MAX_LISTEN, the function does nothing and ignores the invalid input.

Return conf:0.95 **Q:** Does ca_tcp_unlisten() return any value, and can it fail under any circumstances?
A: The function does not return any value. It is designed to never fail, even with invalid inputs. If an invalid handle is passed (0 or greater than TCP_MAX_LISTEN), the function simply ignores it and performs no action.

`ca_tcp_wait_writable`

Header: `caos/net/inc/tcp_service.h:82`
Source: `caos/net/src/tcp_service.c:976:982`
GUARD: `EN_TCP_SERVICE`

```
bool ca_tcp_wait_writable(u8 handle, u32 timeout_ms)
```

Blocks the calling task until the TCP connection buffer has space to send data or the timeout expires.

Parametri:

Nome	Tipo	Descrizione
`handle`	`u8`	The connection identifier returned by a previous TCP operation. Must be between 1 and TCP_MAX_CONN.
`timeout_ms`	`u32`	Maximum time in milliseconds to wait. Use 0 for no wait or a specific value for a timed block.

Ritorna:
Returns true if the connection is writable within the timeout period, false if the timeout expired, the handle is invalid, or the connection is not active.

Note:

- This function is blocking; it puts the calling task into a waiting state until the network stack signals that the transmit buffer is ready or the timeout occurs. It is NOT thread-safe to call from an ISR, as it performs a blocking wait on a semaphore. A common beginner mistake is to ignore the return value; if the function returns false, the connection might be closed or the buffer full, and attempting to send data immediately after will likely fail or cause undefined behavior.

Uso:

- This function is typically used in a loop before calling a send function to ensure the network stack can accept more data. It is part of a standard lifecycle: first, establish a connection, then enter a loop where you wait for writability, check if the connection is still alive, and finally perform the data transmission. It is essential for flow control to prevent overwhelming the network buffer.

Esempio:

```c
u8 h = /* Assume handle obtained from a previous connection call */;
u32 timeout = 1000U;

// Wait up to 1 second for the TCP buffer to be ready for new data
if (ca_tcp_wait_writable(h, timeout)) {
    if (ca_tcp_is_connected(h)) {
        // Proceed to send data using the network stack
        // ca_tcp_send(h, buffer, size);
    }
} else {
    // Handle timeout or connection error
    ca_tcp_close(h);
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should a developer use ca_tcp_wait_writable() in a TCP communication context?

A: A developer should call ca_tcp_wait_writable() when they need to ensure that a TCP connection is ready to send data. This function is useful in scenarios where the application wants to avoid sending data when the connection is not ready, preventing potential errors or data loss. It is typically used before attempting to write data to a TCP socket to ensure proper flow control.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_tcp_wait_writable() with caOS types?
A: u8 tcp_handle = 1; bool result = ca_tcp_wait_writable(tcp_handle, 1000); if (result) { // Proceed with sending data } else { // Handle timeout or invalid connection }

Parameters conf:0.90 **Q:** What are the valid ranges for the handle and timeout_ms parameters in ca_tcp_wait_writable()?
A: The handle parameter must be between 1 and TCP_MAX_CONN inclusive, otherwise the function will fail. The timeout_ms parameter can be any u32 value, with 0 meaning no wait (non-blocking), and larger values specifying the maximum time to wait in milliseconds. The maximum value for timeout_ms depends on the system implementation.

Return conf:0.95 **Q:** What does it mean when ca_tcp_wait_writable() returns false, and what causes this failure?
A: When ca_tcp_wait_writable() returns false, it indicates either that the TCP connection handle is invalid (outside the range 1 to TCP_MAX_CONN) or that the connection was not ready to send data within the specified timeout period. This function may also fail if the mutex s_mtx has not been acquired before calling it, leading to undefined behavior.

`ca_temp_dts_present`

Header: `caos/drivers/inc/temp_sensor.h:35`
Source: `caos/drivers/src/temp_sensor.c:84:87`
GUARD: `EN_TEMP_SENSOR`

```
bool ca_temp_dts_present(void)
```

Checks if the internal Digital Temperature Sensor (DTS) is available on the current hardware.

Ritorna:
Returns true if the DTS is present and initialized, false otherwise.

Note:

- This function is a non-blocking, thread-safe status check that simply returns a cached boolean value. It is safe to call from any context, including ISRs, because it does not perform hardware I/O or wait for sensor measurements. The most common mistake is assuming that this function triggers a measurement; it does not. You must call this before attempting to read the DTS to avoid reading invalid or uninitialized data on hardware variants that lack the sensor.

Uso:

- This function is typically used as a guard clause in initialization routines or CLI commands to ensure hardware compatibility. It is usually paired with ca_temp_read_dts() in a sequence where you first verify presence, then perform the actual measurement. By checking this first, you prevent your application from attempting to access registers or memory regions that do not exist on specific STM32 chip variants.

Esempio:

```
/* Check if DTS is available before attempting to read it */
if (ca_temp_dts_present()) {
    /* DTS is present, safe to read */
    i32 temp = ca_temp_read_dts();
    ca_printf("DTS Temperature: %d\n", temp);
```

```
} else {
    /* DTS not available, fallback to standard ADC temperature sensor */
    i32 temp = ca_temp_read();
    ca_printf("DTS not found, using ADC: %d\n", temp);
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_temp_dts_present and why is it necessary?
A: You should call this function as a guard clause before attempting to read the internal Digital Temperature Sensor. It is necessary because different STM32 chip variants may lack the DTS hardware; calling it prevents your application from attempting to access non-existent registers or memory regions.

Code example conf:0.90 **Q:** How do I safely check for the DTS and read it in caOS?
A: You should verify the sensor's presence before reading it: if (ca_temp_dts_present()) { u32 temp = ca_temp_read_dts(); }

Return conf:1.00 **Q:** What do the return values of ca_temp_dts_present indicate?
A: The function returns true if the DTS is present and initialized on the current hardware, and false otherwise. A false return indicates that the sensor is either missing from the specific STM32 variant or has not been initialized, and you should not attempt to read from it.

`ca_temp_hw_convert`

Header: `caos/drivers/inc/ca_glue_hw.h:261`
GUARD: `EN_CA_GLUE_HW`

```
extern i32 ca_temp_hw_convert(u16 ts_raw, u16 vrefint_raw); /* converte
   i valori grezzi in gradi Celsius (compensato) */
```

Converts raw ADC values from the internal temperature sensor and VREFINT into a calibrated temperature in degrees Celsius.

Parametri:

Nome	Tipo	Descrizione
`ts_raw`	`u16`	The raw 12-bit or 16-bit ADC value read from the internal temperature sensor channel.
`vrefint_raw`	`u16`	The raw 12-bit or 16-bit ADC value read from the internal VREFINT channel, used to compensate for supply voltage fluctuations.

Ritorna:
Returns an i32 representing the temperature in degrees Celsius. If the hardware calibration data is missing or the ADC readings are invalid (e.g., zero), the result may be mathematically incorrect or represent a default error state depending on the specific MCU implementation.

Note:

- This function is a pure mathematical calculation and is non-blocking, thread-safe, and safe to call from an ISR. It relies on factory-programmed calibration values stored in the MCU's system memory (e.g., TS_CAL1_ADDR). A common beginner mistake is passing zero or uninitialized ADC values to this function, which will lead to division-by-zero errors or nonsensical temperature outputs. Always ensure the ADC peripheral is fully initialized and the conversion has completed before reading the raw values.

Uso:

- This function is typically used as the final step in a temperature acquisition sequence. It is paired with ca_temp_read_raw() (or direct ADC reads) to obtain the raw data, which is then passed to this function to produce a human-readable value. It is usually called within a periodic task or a sensor polling loop after the ADC has finished sampling both the temperature and VREFINT channels.

Esempio:

```c
/* Example: Reading and converting internal temperature */
#include "caos.h"

void task_temp_monitor(void *pvParameters) {
    /* Ensure the temperature sensor driver is initialized */
    ca_temp_init();

    while (1) {
        /* Read raw values from the hardware */
        u16 ts_raw = ca_temp_read_raw();
        u16 vref_raw = ca_temp_hw_read_vrefint(); /* Note:
    ca_temp_hw_read_vrefint is the internal driver function */

        if (ts_raw > 0 && vref_raw > 0) {
            i32 temp_c = ca_temp_hw_convert(ts_raw, vref_raw);
            ca_printf("Current Temperature: %d C\n", temp_c);
        }

        /* Wait for 1 second before next reading */
        ca_os_delay_ms(1000);
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_temp_hw_convert in my embedded application?
A: You should call ca_temp_hw_convert after reading raw temperature and Vrefint values from the ADC. It's typically used in temperature monitoring tasks where you need to convert ADC readings into actual Celsius temperatures for further processing or logging.

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_temp_hw_convert with caOS types?
A: u16 ts_raw = 2048; u16 vrefint_raw = 1800; i32 temp_celsius = ca_temp_hw_convert(ts_raw, vrefint_raw); // temp_celsius will be in tenths of degrees Celsius

Parameters conf:0.95 **Q:** What do the ts_raw and vrefint_raw parameters represent, and what are valid ranges?
A: ts_raw is the raw ADC reading from the internal temperature sensor (0-4095). vrefint_raw is the raw ADC reading from the internal reference voltage (Vrefint) (0-4095). Values outside these ranges are considered invalid and may cause incorrect results.

Return conf:0.85 **Q:** What does a negative return value from ca_temp_hw_convert indicate?
A: A negative return value indicates either an error (such as invalid input parameters) or a temperature below -40°C. The function returns negative values for errors, and the specific meaning depends on the implementation and input validation.

Error/Edge case conf:0.80 **Q:** What are common mistakes when using ca_temp_hw_convert?
A: Common mistakes include passing invalid values outside the 0-4095 range for ts_raw or vrefint_raw, not ensuring ADC readings are valid before calling the function, and ignoring that vrefint_raw is ignored in F1xx implementations but used in others.

HW differences conf:0.90 **Q:** How does ca_temp_hw_convert differ between F1xx and F4xx families?
A: In F1xx, the vrefint_raw parameter is ignored and the function uses fixed calibration

values (V25_MV, AVG_SLOPE_X10). In F4xx and later families, both ts_raw and vrefint_raw are used to compute VDDA and perform compensation for more accurate temperature readings.

---

ca_temp_hw_dts_alarm

Header: caos/drivers/inc/ca_glue_hw.h:295
GUARD: EN_CA_GLUE_HW

```
extern void ca_temp_hw_dts_alarm(u32 raw_hot, u32 raw_cold, bool hot_on,
    bool cold_on, ca_temp_alarm_cb_t cb); /* thresholds + IRQ, cb NULL =
    off */
```

Configures hardware temperature thresholds and an interrupt callback for the Die Temperature Sensor (DTS).

Parametri:

Nome	Tipo	Descrizione
raw_hot	u32	Raw sensor count threshold for the hot alarm (triggers when temperature is at or below this value).
raw_cold	u32	Raw sensor count threshold for the cold alarm (triggers when temperature is at or above this value).
hot_on	bool	Enable the hot alarm interrupt.
cold_on	bool	Enable the cold alarm interrupt.
cb	ca_temp_alarm_cb_t	Function pointer to be called when an alarm triggers. Pass NULL to disable all alarms.

Ritorna:
void

Note:

- This function is non-blocking and interacts directly with hardware registers. It is not thread-safe; if multiple tasks attempt to configure the alarm simultaneously, the behavior is undefined. It is intended to be called from a task context, but it internally manages NVIC settings to ensure the interrupt is configured correctly. A common mistake is passing invalid raw values that exceed the hardware register capacity, or failing to realize that the callback executes within the context of the DTS interrupt service routine (ISR). Keep the callback execution time extremely short to avoid blocking other system interrupts.

Uso:

- This function is used to monitor die temperature without polling. It is typically paired with a conversion function like ca_temp_hw_dts_raw() to translate human-readable temperature values into hardware-specific counts. The sequence is usually: calculate thresholds, call this function to arm the hardware, and then handle the resulting event in the provided callback. To stop monitoring, call this function again with cb set to NULL.

Esempio:

```
/* Example: Setting a temperature alarm for a specific range */
#include "caos.h"

void my_temp_callback(void) {
    /* This runs in ISR context when the temperature crosses the threshold */
    /* Perform emergency shutdown or logging here */
```

```
}

void setup_thermal_protection(void) {
    u32 hot_limit = 0x0500; /* Example raw value for high temp */
    u32 cold_limit = 0x0100; /* Example raw value for low temp */

    /* Configure the hardware alarm with the callback */
    ca_temp_hw_dts_alarm(hot_limit, cold_limit, true, true, my_temp_callback);
}

void disable_thermal_protection(void) {
    /* Passing NULL disables the alarm and stops the DTS hardware */
    ca_temp_hw_dts_alarm(0, 0, false, false, NULL);
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_temp_hw_dts_alarm function in my caOS application?
A: You should use this function when you need to monitor die temperature without the overhead of polling. It is typically used by calculating thresholds, calling this function to arm the hardware, and then handling the resulting event within the provided callback function.

Code example conf:1.00 **Q:** Provide a minimal example of how to disable the DTS alarm using caOS types.
A: To disable the DTS alarm, you simply pass NULL as the callback argument: ca_temp_hw_dts_alarm(0U, false, false, NULL);

Parameters conf:1.00 **Q:** What do the raw_hot and raw_cold parameters represent in ca_temp_hw_dts_alarm
A: raw_hot is the raw sensor count threshold that triggers the hot alarm when the temperature is at or below this value. raw_cold is the raw sensor count threshold that triggers the cold alarm when the temperature is at or above this value.

Error/Edge case conf:1.00 **Q:** What are some common mistakes to avoid when using ca_temp_hw_dts_alarm?
A: Common mistakes include passing invalid raw values that exceed hardware register capacity, failing to realize that the callback executes within the context of the DTS interrupt service routine (ISR), and not keeping the callback execution time extremely short to avoid blocking other system interrupts.

Behavior conf:1.00 **Q:** Is the ca_temp_hw_dts_alarm function thread-safe?
A: No, the function is not thread-safe. If multiple tasks attempt to configure the alarm simultaneously, the behavior is undefined. It is intended to be called from a task context and is non-blocking.

HW differences conf:1.00 **Q:** How does the implementation of ca_temp_hw_dts_alarm differ across STM32 families?
A: The function is only actively implemented for the H5xx family, where it interacts with registers like DTS->ITENR, DTS->ICIFR, and DTS->CFGR1. For F1xx, F4xx, F7xx, H7xx, and U5xx families, the function is currently a stub that performs no operations.

`ca_temp_hw_dts_centi`

Header: `caos/drivers/inc/ca_glue_hw.h:293`
GUARD: `EN_CA_GLUE_HW`

```
extern i32 ca_temp_hw_dts_centi(u32 raw); /* raw count -> centi-degC at
    the current clock */
```

Converts raw digital temperature sensor (DTS) counts into temperature in centi-degrees Celsius.

Parametri:

Nome	Tipo	Descrizione
`raw`	`u32`	The raw frequency count obtained from the DTS hardware. A value of 0 is invalid and indicates a sensor error or uninitialized state.

Ritorna:
Returns the temperature in centi-degrees Celsius (e.g., 2500 for 25.00°C) as an i32. Returns CA_TEMP_NO_DTS if the input is 0, if the sensor hardware is missing, or if calibration data (ramp/offset) is invalid.

Note:

- This function is non-blocking and thread-safe as it performs pure mathematical calculations based on hardware registers. It is safe to call from an ISR, provided the hardware registers are accessible. It is mandatory that the system clock is initialized before calling this, as the calculation depends on the current CPU frequency. A common beginner mistake is passing a raw value of 0 without checking, which will result in an error return code rather than a valid temperature.

Uso:

- This function is typically used in a periodic task or a monitoring loop to convert raw sensor data into human-readable units. It is usually paired with a sampling function that reads the hardware register, followed by an averaging filter to reduce noise. The sequence is: sample the raw value, validate the result, pass it to this function, and then use the resulting centi-degree value for thermal management or logging.

Esempio:

```c
/* Assuming ca_temp_hw_dts_sample() exists to get raw data */
#define CA_TEMP_NO_DTS (-1)

void monitor_temperature(void) {
    u32 raw_val = ca_temp_hw_dts_sample();

    if (raw_val == 0U) {
        /* Handle sensor error */
        return;
    }

    i32 temp_centi = ca_temp_hw_dts_centi(raw_val);

    if (temp_centi != CA_TEMP_NO_DTS) {
        /* Log temperature in Celsius */
        ca_printf("Current Temp: %d.%02d C\n", temp_centi / 100, temp_centi %
    100);
    } else {
        ca_printf("DTS sensor failure or missing\n");
    }
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_temp_hw_dts_centi in my application?
A: You should call it within a periodic task or monitoring loop after sampling the raw DTS value. It is intended to convert raw hardware counts into human-readable centi-degrees Celsius for thermal management or logging purposes.

Code example conf:1.00 **Q:** How do I implement a basic temperature reading loop using caOS types?
A: You should sample the sensor, validate the result, and pass it to the conversion function: u32 raw = ca_temp_hw_dts_sample(); if (raw != 0U) { i32 temp = ca_temp_hw_dts_centi(raw) }

Parameters conf:1.00 **Q:** What does the 'raw' parameter represent and what happens if I pass 0?
A: The 'raw' parameter is the frequency count from the DTS hardware. Passing 0 is invalid and indicates a sensor error or uninitialized state; the function will return CA_TEMP_NO_DTS in this case.

Return conf:1.00 **Q:** What does the return value of ca_temp_hw_dts_centi signify?
A: It returns the temperature in centi-degrees Celsius (e.g., 2500 for 25.00°C) as an i32. It returns CA_TEMP_NO_DTS if the input is 0, the hardware is missing, or if calibration data like ramp or offset is invalid.

HW differences conf:1.00 **Q:** Are there differences in how ca_temp_hw_dts_centi behaves across STM32 families?
A: Yes. On the H5xx family, the function performs a calculation using the DTS peripheral registers (DTS->T0VALR1 and DTS->RAMPVALR). On the F1xx, F4xx, F7xx, H7xx, and U5xx families, the function is currently unimplemented and returns CA_TEMP_NO_DTS regardless of the input.

`ca_temp_hw_dts_init`

Header: `caos/drivers/inc/ca_glue_hw.h:291`
GUARD: `EN_CA_GLUE_HW`

```
1 extern bool ca_temp_hw_dts_init(void); /* DTS on, false when the part
     has none */
```

Initializes the internal Digital Temperature Sensor (DTS) hardware if available on the current STM32 chip.

Ritorna:
Returns true if the DTS was successfully initialized and is ready for use. Returns false if the hardware does not support a DTS or if the initialization sequence failed.

Note:

- This function is hardware-dependent and acts as a wrapper for chip-specific implementations. It is not thread-safe and should be called during the system initialization phase before any temperature reading tasks are started. It is not intended to be called from an ISR. A common mistake is assuming that this function will always succeed; you must always check the return value, as many STM32 families (like F1, F4, F7) do not have a DTS and will always return false.

Uso:

- This function is typically called once during the driver initialization sequence for the temperature sensor subsystem. It is usually paired with a generic initialization function (e.g., ca_temp_init) to ensure the hardware abstraction layer is ready. If it returns false, the application should handle the absence of the sensor gracefully, perhaps by disabling temperature-dependent features or logging a warning.

Esempio:

```c
bool status;

/* Attempt to initialize the hardware DTS peripheral */
status = ca_temp_hw_dts_init();

if (status) {
    ca_printf("DTS initialized successfully.\n");
} else {
    /* Log that the sensor is missing or failed to start */
    ca_printf("DTS not available or failed to init.\n");
    /* Proceed with system startup, but do not rely on DTS data */
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_temp_hw_dts_init in my caOS application?
A: It should be called once during the system initialization phase, before any temperature reading tasks are started. It is not thread-safe and must not be called from an ISR.

Code example conf:1.00 **Q:** How do I implement a safe initialization check for the DTS in caOS?
A: You should check the return value of the function to handle hardware availability: if (ca_temp_hw_dts_init()) { /* DTS ready */ } else { /* Handle absence or failure */ }

Return conf:1.00 **Q:** What do the return values of ca_temp_hw_dts_init signify?
A: Returning true indicates the DTS was successfully initialized and is ready. Returning false means the hardware does not support a DTS, or the initialization sequence failed.

HW differences conf:1.00 **Q:** Why does ca_temp_hw_dts_init return false on my F4xx chip but not on an H5xx?
A: The function is hardware-dependent. On F1xx, F4xx, F7xx, H7xx, and U5xx families, the DTS is not present, so the function returns false. On the H5xx family, it actively configures the DTS by enabling the clock via RCC->APB1HENR and setting the DTS->CFGR1 register.

`ca_temp_hw_dts_raw`

Header: `caos/drivers/inc/ca_glue_hw.h:294`
GUARD: `EN_CA_GLUE_HW`

```c
extern u32 ca_temp_hw_dts_raw(i32 centi); /* centi-degC -> raw count at
    the current clock */
```

Converts a temperature in centi-degrees Celsius to the raw hardware counter value required by the DTS peripheral.

Parametri:

Nome	Tipo	Descrizione
`centi`	`i32`	The target temperature in centi-degrees Celsius (e.g., 2500 for 25.00°C). Valid range depends on the specific hardware DTS calibration limits.

Ritorna:
A u32 representing the raw counter value for the DTS hardware. Returns 0xFFFFU if the calculated value is out of range or if the hardware does not support DTS.

Note:

- This function is non-blocking and thread-safe as it performs a mathematical calculation based on hardware-specific calibration constants. It is safe to call from an ISR because

it does not perform I/O or wait for locks. The function relies on the DTS peripheral being correctly calibrated at the factory; if the hardware does not support DTS (e.g., F1xx, F4xx, F7xx), it will return 0U or a dummy value. A common mistake is passing a temperature value that results in a negative frequency calculation, which the function handles by returning 0xFFFFU as an error indicator.

Uso:

- This function is primarily used to configure temperature alarm thresholds for the DTS peripheral. It is typically paired with ca_temp_set_alarm, where the raw values calculated by this function are passed as the 'hot' and 'cold' thresholds. Developers use this to translate human-readable temperature limits into the specific register-level counts the hardware expects for triggering interrupts.

Esempio:

```c
/* Example: Setting a temperature alarm for 80 degrees Celsius */
#include "caos.h"

void configure_thermal_protection(void)
{
    /* Convert 80.00 degrees Celsius to raw hardware units */
    i32 hot_threshold_centi = 8000;
    u32 raw_limit = ca_temp_hw_dts_raw(hot_threshold_centi);

    if (raw_limit != 0xFFFFU) {
        /* Use the calculated raw value to set the hardware alarm */
        /* ca_temp_set_alarm(raw_limit, ...); // Function from ca_temp_* API */
    } else {
        /* Handle error: temperature out of valid calibration range */
    }
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_temp_hw_dts_raw function in my caOS application?
A: You should use this function when you need to configure temperature alarm thresholds for the DTS peripheral. It is typically used to translate human-readable temperature limits (in centi-degrees Celsius) into the raw counter values required by the hardware registers, often in preparation for calling ca_temp_set_alarm.

Code example conf:1.00 **Q:** Provide a minimal example of how to calculate a raw DTS value for 30 degrees Celsius.
A: i32 target_temp = 3000; u32 raw_val = ca_temp_hw_dts_raw(target_temp);

Parameters conf:1.00 **Q:** What does the 'centi' parameter represent and what are its constraints?
A: The 'centi' parameter represents the target temperature in centi-degrees Celsius (e.g., 2500 for 25.00°C). The valid range is dependent on the specific hardware DTS calibration limits; passing a value that results in a negative frequency calculation will cause the function to return 0xFFFFU.

Return conf:1.00 **Q:** What does a return value of 0xFFFFU indicate when calling ca_temp_hw_dts_raw?
A: A return value of 0xFFFFU indicates an error. This occurs if the calculated raw value is out of range, if the hardware does not support DTS, or if the input temperature results in a negative frequency calculation.

HW differences conf:1.00 **Q:** How does the implementation of ca_temp_hw_dts_raw differ across STM32 families?
A: On families like F1xx, F4xx, F7xx, H7xx, and U5xx, the function is a stub that returns 0U because these families do not support the DTS peripheral. In contrast, the H5xx implementation performs a complex calculation using the DTS->T0VALR1 and

DTS->RAMPVALR registers, incorporating the system clock frequency to derive the raw counter value.

---

ca_temp_hw_dts_sample

Header: caos/drivers/inc/ca_glue_hw.h:292
GUARD: EN_CA_GLUE_HW

```
extern u32 ca_temp_hw_dts_sample(void); /* one fresh DTS measurement (
    raw count), 0 on timeout */
```

Performs a single, fresh raw measurement from the internal Digital Temperature Sensor (DTS).

Ritorna:
Returns the raw 32-bit sensor count on success. Returns 0U if the sensor fails to signal readiness within the internal timeout period or if the hardware does not support a DTS.

Note:

- This function is blocking; it waits for the hardware to complete a conversion cycle, which may take several milliseconds. It is NOT safe to call from an ISR because it relies on polling the hardware status register. The function handles the complex state machine required to trigger a one-shot measurement even if the sensor was previously configured for continuous operation. A common mistake is calling this function too frequently in a tight loop without considering the sensor's conversion time, which can lead to unnecessary CPU overhead or timeout errors.

Uso:

- This function is typically used when you need a precise, non-averaged temperature reading. It is often paired with a loop to collect multiple samples for software-based averaging to improve signal stability. Before calling this, ensure the system has verified that the DTS hardware is present and initialized. It is usually called within a dedicated monitoring task that runs periodically, rather than in high-priority real-time control loops.

Esempio:

```
/* Example: Collect an average of 8 samples from the DTS */
#define SAMPLES_COUNT 8

void task_temp_monitor(void *arg) {
    u32 total = 0;
    for (u32 i = 0; i < SAMPLES_COUNT; i++) {
        u32 raw = ca_temp_hw_dts_sample();
        if (raw == 0U) {
            /* Handle error: sensor timeout or hardware failure */
            return;
        }
        total += raw;
        /* Wait briefly between samples to allow hardware stabilization */
        ca_os_delay_ms(10);
    }
    u32 average = total / SAMPLES_COUNT;
    ca_printf("Average raw temp: %u\n", average);
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_temp_hw_dts_sample in my caOS application?
A: Use this function when you require a precise, non-averaged raw temperature reading from the internal DTS. It is best suited for periodic monitoring tasks rather than high-priority real-time control loops, and can be used in a loop to collect multiple samples for software-based averaging.

Code example conf:1.00 **Q:** Provide a minimal C example for reading the DTS using caOS types.
A: u32 raw_val = ca_temp_hw_dts_sample(); if (raw_val != 0U) { /* process raw_val */ }

Return conf:1.00 **Q:** What do the return values of ca_temp_hw_dts_sample indicate?
A: The function returns the raw 32-bit sensor count on success. It returns 0U if the sensor fails to signal readiness within the internal timeout period or if the hardware does not support a DTS.

Error/Edge case conf:1.00 **Q:** What are common mistakes when calling ca_temp_hw_dts_sample?
A: A common mistake is calling the function too frequently in a tight loop without accounting for the sensor's conversion time, which can cause unnecessary CPU overhead or trigger timeout errors.

Behavior conf:1.00 **Q:** Is ca_temp_hw_dts_sample safe to call from an ISR?
A: No, it is not safe to call from an ISR. The function is blocking and relies on polling the hardware status register, which can take several milliseconds to complete a conversion cycle.

HW differences conf:1.00 **Q:** How does the implementation of ca_temp_hw_dts_sample differ across STM32 families?
A: The DTS is not present on the F1xx, F4xx, F7xx, H7xx, and U5xx families, where the function simply returns 0U. On the H5xx family, the function actively manages the DTS_CFGR1 register, specifically the DTS_CFGR1_TS1_START bit, to perform a one-shot measurement even if the sensor was previously configured for continuous operation.

`ca_temp_hw_init`

Header: `caos/drivers/inc/ca_glue_hw.h:258`
GUARD: `EN_CA_GLUE_HW`

```
1 extern bool ca_temp_hw_init(void); /* abilita il sensore di temperatura
     interno e il canale Vrefint */
```

Low-level hardware initialization for the internal temperature sensor and reference voltage channel.

Ritorna:
Returns true if the hardware was successfully configured and the ADC is ready; returns false if the initialization sequence failed (e.g., timeout waiting for ADC ready or LDO ready).

Note:

- This function performs direct register manipulation to enable clocks, power up the ADC, calibrate it, and configure internal channels. It is NOT thread-safe and must be called before any attempt to read the temperature sensor. It is not intended to be called from an ISR as it contains blocking 'while' loops waiting for hardware flags. A common mistake is calling this function multiple times without checking the initialization state, which may trigger redundant calibration or reset the ADC state machine.

Uso:

- This function is the mandatory hardware-level setup step for the temperature sensing subsystem. It is typically called once during system startup or the first time a temperature reading is requested. It is designed to be wrapped by a higher-level function (like ca_temp_init) that manages a static initialization flag to ensure the hardware is configured only once. After this returns true, the system can safely use ca_temp_read or ca_temp_read_raw to retrieve sensor data.

Esempio:

```c
/* Example of safe initialization wrapper */
static bool g_temp_initialized = false;

bool ca_temp_init(void) {
    if (g_temp_initialized) {
        return true;
    }

    /* Perform low-level hardware setup */
    if (!ca_temp_hw_init()) {
        /* Log error using system audit if available */
        return false;
    }

    g_temp_initialized = true;
    return true;
}

/* Usage in a task */
void sensor_task(void *pv) {
    if (ca_temp_init()) {
        /* Now safe to read the sensor */
        /* int16_t temp = ca_temp_read(); */
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_temp_hw_init() in my caOS application?
A: You should call ca_temp_hw_init() once during system initialization, before using any temperature sensor functions. It initializes the ADC hardware and enables the internal temperature sensor and Vrefint channel. This function must be called before any other temperature sensor functions and should only be called once to avoid undefined behavior.

Code example conf:0.95 **Q:** Can you show a minimal example of how to use ca_temp_hw_init() in a caOS application?
A: bool success = ca_temp_hw_init(); if (success) { // Temperature sensor is now ready to use // Proceed with temperature readings } else { // Handle initialization failure // This is rare but possible if hardware doesn't respond }

Return conf:0.90 **Q:** What does ca_temp_hw_init() return and what causes it to return false?
A: ca_temp_hw_init() returns true on successful initialization or false if initialization fails. Failures are rare but can occur if the hardware doesn't respond properly or if there are clock configuration issues. The function is designed to be robust and typically succeeds unless there's a hardware or clock problem.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_temp_hw_init()?
A: Common mistakes include calling ca_temp_hw_init() multiple times, which may cause undefined behavior; calling it from an ISR (which is not safe); or calling it in a multi-threaded RTOS environment without synchronization. The function is not thread-safe and must be called once and only once before using other temperature sensor functions.

Behavior conf:0.90 **Q:** Is ca_temp_hw_init() blocking or non-blocking, and is it safe to call from an ISR or in a multi-threaded environment?
A: ca_temp_hw_init() is a blocking function that includes software delays (NOP loops) for sensor startup. It is not safe to call from an ISR and is not thread-safe. It must be called once during system startup before any other temperature sensor functions. In RTOS environments, it should be called from a single task without concurrent access.

HW differences conf:0.85 **Q:** How does ca_temp_hw_init() differ between STM32F1xx and STM32H7xx families?
A: The main differences are in ADC configuration and startup procedures. F1xx uses ADC1 with specific prescaler settings (RCC_CFGR_ADCPRE_DIV6) and simple software delays. H7xx uses ADC3 with more complex initialization including BOOST mode (ADC_CR_BOOST), deep power down exit, voltage regulator enable, and calibration steps. H7xx also uses different registers like ADC3_COMMON->CCR for enabling sensors and different sample time configurations in SMPR2.

`ca_temp_hw_read_raw`

Header: `caos/drivers/inc/ca_glue_hw.h:265`
GUARD: `EN_CA_GLUE_HW`

```
extern u16 ca_temp_hw_read_raw(void); /* reads the raw ADC value of the
    internal temperature sensor */
```

Reads the raw ADC value from the internal MCU temperature sensor.

Ritorna:
A 16-bit unsigned integer representing the raw ADC conversion result. The value range depends on the specific STM32 ADC resolution (typically 12-bit, 0-4095).

Note:

- This function is a low-level hardware abstraction layer (HAL) call. It is not thread-safe by itself and assumes the ADC peripheral has been correctly initialized and configured for the internal temperature channel. It is generally non-blocking, as it triggers or reads the ADC register directly, but it should not be called from an ISR unless the underlying ADC driver is specifically designed to handle concurrent access. The most common beginner mistake is calling this function before the temperature sensor or ADC peripheral has been initialized, which may result in reading garbage data or triggering a hardware fault.

Uso:

- This function is used as the primary data acquisition step for temperature monitoring. It is typically paired with ca_temp_init to ensure the hardware is ready, and then used in conjunction with ca_temp_hw_read_vrefint to perform a calibrated conversion. In a typical application, a task will call this function periodically, pass the raw value to a conversion function, and then log or act upon the resulting temperature value.

Esempio:

```
/* Ensure the temperature sensor driver is initialized before reading */
if (ca_temp_init()) {
    /* Read the raw ADC value from the internal sensor */
    u16 raw_temp = ca_temp_hw_read_raw();

    /* Read the internal reference voltage to calibrate the reading */
    u16 raw_vref = ca_temp_hw_read_vrefint();

```

```
    /* Perform calculation using the raw values (conversion logic omitted) */
    /* float temperature = calculate_temp(raw_temp, raw_vref); */

    ca_printf("Raw Temp ADC: %u, Raw Vref: %u\n", raw_temp, raw_vref);
} else {
    ca_printf("Failed to initialize temperature sensor!\n");
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_temp_hw_read_raw() and what is its purpose in a temperature monitoring system?
A: You should call ca_temp_hw_read_raw() to read the raw ADC value from the internal temperature sensor of an STM32 microcontroller. It is used in temperature monitoring systems where you need to acquire raw sensor data for further processing or conversion to actual temperature values. The function must only be called after initializing the temperature sensor driver with ca_temp_init().

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_temp_hw_read_raw() with caOS types?
A: u16 raw_temp = ca_temp_hw_read_raw(); if (raw_temp != -999) { // Process the raw temperature value // ... } else { // Handle initialization error // ... }

Return conf:0.90 **Q:** What do the return values of ca_temp_hw_read_raw() indicate and what causes the function to return -999?
A: The function returns a 12-bit ADC value (0-4095) when the temperature sensor driver has been properly initialized. It returns -999 if the driver has not been initialized with ca_temp_init(), indicating a failure to read the sensor. This typically happens when the function is called before proper initialization or if the initialization process failed.

HW differences conf:0.85 **Q:** What are the hardware differences in how ca_temp_hw_read_raw() is implemented across different STM32 families?
A: The implementation varies by MCU family: F1xx, F4xx, and F7xx families use ADC1 with ADC_CHANNEL_TEMP; H5xx family also uses ADC1 with ADC_CHANNEL_TEMP; H7xx family uses ADC3 with ADC_CHANNEL_TEMP instead of ADC1. This reflects differences in ADC peripheral availability and configuration across STM32 families.

`ca_temp_hw_read_vrefint`

Header: `caos/drivers/inc/ca_glue_hw.h:260`
GUARD: `EN_CA_GLUE_HW`

```
extern u16 ca_temp_hw_read_vrefint(void); /* legge il valore grezzo ADC
   del riferimento interno Vrefint */
```

Reads the raw ADC value of the internal reference voltage (Vrefint) for temperature calculation.

Ritorna:
Returns a 16-bit unsigned integer representing the raw ADC conversion result of the internal reference channel. A value of 0 typically indicates an error or that the ADC peripheral is not properly initialized.

Note:

- This function is a low-level hardware abstraction layer (HAL) call. It is not thread-safe by itself and should only be called after the ADC peripheral has been initialized via the appropriate hardware driver. It is generally non-blocking, as it triggers a direct register read or a simple polling sequence depending on the MCU architecture. Do not

call this from an ISR unless the underlying ADC driver is specifically designed to be re-entrant or protected by a spinlock, as concurrent access to the ADC hardware will cause data corruption or system instability. The most common beginner mistake is calling this function before the system clock or the ADC peripheral has been fully initialized, which will result in reading invalid or zero values.

Uso:

- This function is used exclusively by the temperature sensor driver to perform calibration. It is typically paired with ca_temp_read_raw() to obtain both the sensor reading and the reference voltage reading simultaneously. These two raw values are then passed to a conversion function to calculate the actual temperature in degrees Celsius. The typical sequence is: ca_temp_init() -> ca_temp_read_raw() -> ca_temp_hw_read_vrefint() -> calculation. It is essential to use these values together because the internal reference voltage changes with the supply voltage, and using the raw sensor value alone without Vrefint compensation will lead to inaccurate temperature readings.

Esempio:

```
/* Ensure the temperature sensor driver is initialized before reading */
if (ca_temp_init() == 0) {
    u16 raw_temp = ca_temp_read_raw();
    /* Read Vrefint to compensate for supply voltage fluctuations */
    u16 raw_vref = ca_temp_hw_read_vrefint();

    if (raw_temp != 0 && raw_vref != 0) {
        /* Perform calculation using raw_temp and raw_vref */
        /* Use a custom function to convert raw values to Celsius */
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_temp_hw_read_vrefint() and why is it important for temperature sensing?
A: You should call ca_temp_hw_read_vrefint() when you need to read the raw ADC value from the internal reference voltage (Vrefint) to calculate ambient temperature. This function is essential for temperature sensing because it provides the baseline measurement required to compute temperature using other functions in the caOS framework. It's typically used in temperature sensor implementations across different STM32 families.

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_temp_hw_read_vrefint() with caOS types?
A: u16 raw_value = ca_temp_hw_read_vrefint(); if (raw_value != 0) { // Process the raw ADC value for temperature calculation // ... temperature computation logic here } else { // Handle initialization error }

Return conf:0.95 **Q:** What do the return values of ca_temp_hw_read_vrefint() mean and what causes it to return 0?
A: The function returns a raw ADC value between 0-4095 if successful, representing the voltage level of the internal reference. It returns 0 if either the ADC module or the Vrefint sensor has not been initialized, indicating a failure to read valid data. This is a common error that developers must check for before using the returned value.

HW differences conf:0.85 **Q:** How does ca_temp_hw_read_vrefint() differ between STM32 H7xx and other STM32 families like F4xx?
A: The function implementation differs between STM32 families in terms of which ADC peripheral is used. For F4xx, F7xx, and H5xx families, it uses adc1_read_channel(ADC_CHANNEL_V whereas for H7xx family, it uses adc3_read_channel(ADC_CHANNEL_VREF). This

reflects differences in ADC peripheral architecture and channel mapping across STM32 families.

---

ca_temp_init

Header: caos/drivers/inc/temp_sensor.h:24
Source: caos/drivers/src/temp_sensor.c:29:40
GUARD: EN_TEMP_SENSOR

```
bool ca_temp_init(void)
```

Initializes the temperature sensor hardware and ensures it is ready for use.

Ritorna:
Returns true if the sensor is successfully initialized or was already initialized. Returns false if the hardware initialization fails.

Note:

- This function is a wrapper that ensures the temperature sensor hardware is only initialized once, preventing re-initialization conflicts. It is not thread-safe by itself, so it should be called during the system configuration phase before the RTOS scheduler starts. It is not intended to be called from an ISR. A common mistake is calling this function multiple times from different tasks without ensuring the system is in a stable state, or assuming it will re-initialize hardware if a previous attempt failed.

Uso:

- This function is typically called once during the system startup sequence, specifically within the ca_os_configure() function. It acts as a prerequisite for any subsequent calls to ca_temp_read() or ca_temp_read_raw(). By placing it in the configuration phase, you ensure that the sensor hardware is ready before any application tasks attempt to access temperature data.

Esempio:

```
void ca_os_configure(void) {
    /* Initialize the temperature sensor hardware */
    if (!ca_temp_init()) {
        /* Handle initialization failure: log error or enter safe state */
        ca_printf("Critical: Temperature sensor failed to init!\n");
    } else {
        ca_printf("Temperature sensor initialized successfully.\n");
    }

    /* Proceed with other system initializations */
    ca_stdout_init();
    ca_stdout_create_thread();
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_temp_init() in my embedded application?
A: You should call ca_temp_init() once during system initialization, before using any other temperature sensor functions. It must be called before any other sensor-related functions and cannot be called from an ISR. The function is meant to be a one-time setup for the temperature sensor hardware.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_temp_init() with caOS types?

A: bool success = ca_temp_init(); if (success) { // Temperature sensor initialized successfully } else { // Handle initialization failure }

Return conf:0.95 **Q:** What does ca_temp_init() return and what causes it to return false?
A: ca_temp_init() returns true if initialization is successful, and false if it fails. Failures can occur due to hardware unavailability or configuration errors. The function is designed to be blocking and must be called only once before using other sensor functions.

Error/Edge case conf:0.90 **Q:** What are common mistakes or edge cases when using ca_temp_init()?
A: Common mistakes include not checking the return value of ca_temp_init() and calling it multiple times, which may cause unpredictable behavior. It should not be called from an ISR, and it's not thread-safe. The function must be called once before using other sensor functions, otherwise the sensor may not work as expected.

`ca_temp_read`

Header: `caos/drivers/inc/temp_sensor.h:25`
Source: `caos/drivers/src/temp_sensor.c:42:50`
GUARD: `EN_TEMP_SENSOR`

```
i32 ca_temp_read(void); /* temperature in integer  C , VREFINT
    compensated */
```

Reads the internal MCU temperature sensor and returns the value in degrees Celsius.

Ritorna:
Returns the temperature as an i32 integer in degrees Celsius. Returns -999 if the sensor has not been initialized via ca_temp_init().

Note:

- This function is non-blocking and performs a quick hardware read of the internal ADC channels for the temperature sensor and VREFINT. It is thread-safe as it only performs read operations on hardware registers, but it relies on the global initialization flag being set. It is generally safe to call from an ISR, provided the underlying ADC hardware is not being used by another high-priority task. The most common beginner mistake is calling this function before the system has completed the ca_temp_init() sequence, which will result in the error code -999.

Uso:

- This function is typically used in periodic monitoring tasks or CLI commands to report system health. It is often paired with ca_temp_init() during system startup to prepare the hardware. In a typical application, you would call it inside a loop, perhaps combined with ca_os_sleep() to regulate the sampling frequency, and use the returned value to trigger safety logic or thermal throttling.

Esempio:

```
i32 current_temp;

/* Initialize the sensor before reading */
ca_temp_init();

while (1) {
    /* Read the core temperature */
    current_temp = ca_temp_read();

    /* Check for initialization error */
    if (current_temp != -999) {
```

```
        ca_printf("System Temp: %d C\n", current_temp);
    } else {
        ca_printf("Error: Sensor not initialized\n");
    }

    /* Wait 1000 ticks before next reading */
    ca_os_sleep(1000);
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_temp_read() and why is it important to check its return value?
A: You should call ca_temp_read() to get the current CPU core temperature in degrees Celsius, compensated using VREFINT. It's important to check the return value because if the sensor has not been initialized (e.g., via ca_temp_init()), the function returns -999, indicating an invalid temperature reading. Ignoring this negative return value can lead to incorrect temperature usage in your application.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_temp_read() with caOS types?
A: i32 temp = ca_temp_read(); if (temp != -999) { // Use valid temperature value cstr msg = "Temperature: "; // Example of how to process the value } else { // Handle uninitialized sensor case }

Return conf:0.95 **Q:** What do the return values of ca_temp_read() mean and what causes the function to return -999?
A: The function returns -999 when the temperature sensor has not been initialized, typically because ca_temp_init() was not called beforehand. In all other cases, it returns the temperature in integer degrees Celsius. A return value of -999 indicates that the sensor is not ready for use and should not be used as a valid temperature measurement.

`ca_temp_read_dts`

Header: `caos/drivers/inc/temp_sensor.h:36`
Source: `caos/drivers/src/temp_sensor.c:89:99`
GUARD: `EN_TEMP_SENSOR`

```
i32 ca_temp_read_dts(void); /* centi-degC, averaged */
```

Reads the internal Digital Temperature Sensor (DTS) and returns the averaged temperature in centi-degrees Celsius.

Ritorna:
Returns the temperature as a signed 32-bit integer (centi-degrees Celsius, e.g., 2550 for 25.50°C). Returns CA_TEMP_NO_DTS if the sensor is not present, not initialized, or if the hardware sampling fails.

Note:

- This function is not thread-safe if called concurrently from multiple tasks without external synchronization, as it performs multiple hardware samples. It is not intended to be called from an ISR because it performs a loop of hardware samples which may take significant time, potentially violating real-time constraints. Before calling this, ensure the system has initialized the temperature hardware. A common mistake is ignoring the return value CA_TEMP_NO_DTS, which leads to using invalid temperature data in control logic.

Uso:

- This function is typically used in monitoring tasks or CLI interfaces to provide high-precision thermal data from the MCU core. It is often paired with ca_temp_dts_present() to verify hardware availability before attempting a read. In a typical lifecycle, the application calls ca_temp_init() during system startup, then periodically calls ca_temp_read_dts() to log thermal status or adjust cooling/performance parameters.

Esempio:

```c
/* Check if DTS is available before reading to avoid invalid data */
if (ca_temp_dts_present()) {
    i32 temp_centi = ca_temp_read_dts();
    if (temp_centi != CA_TEMP_NO_DTS) {
        /* Convert centi-degrees to degrees for display */
        ca_printf_n("Core Temp: %d.%02d C", temp_centi / 100, (temp_centi < 0 ? -temp_centi : temp_centi) % 100);
    } else {
        ca_printf_n("Error: Failed to sample DTS hardware");
    }
} else {
    ca_printf_n("DTS not supported on this hardware");
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_temp_read_dts and what is its primary purpose?
A: You should call it in monitoring tasks or CLI interfaces to obtain high-precision thermal data from the MCU core. It is typically used to log thermal status or adjust cooling and performance parameters after the system has been initialized via ca_temp_init.

Code example conf:1.00 **Q:** Provide a minimal C example showing how to safely read the temperature using caOS types.
A: i32 temp = 0; if (ca_temp_dts_present()) { temp = ca_temp_read_dts(); }

Return conf:1.00 **Q:** What does the return value represent and what does CA_TEMP_NO_DTS signify?
A: The function returns a signed 32-bit integer representing temperature in centi-degrees Celsius (e.g., 2550 for 25.50°C). CA_TEMP_NO_DTS is returned if the sensor is missing, not initialized, or if the hardware sampling process fails.

Error/Edge case conf:1.00 **Q:** Are there any risks or common mistakes when using ca_temp_read_dts?
A: A common mistake is ignoring the CA_TEMP_NO_DTS return value, which leads to using invalid data. Additionally, the function is not thread-safe for concurrent calls and should not be called from an ISR because the hardware sampling loop may take significant time and violate real-time constraints.

`ca_temp_read_raw`

Header: `caos/drivers/inc/temp_sensor.h:26`
Source: `caos/drivers/src/temp_sensor.c:52:58`
GUARD: `EN_TEMP_SENSOR`

```c
u16 ca_temp_read_raw(void); /* raw ADC value (12-bit) */
```

Reads the raw 12-bit ADC value from the internal temperature sensor.

Ritorna:
Returns a 12-bit unsigned integer (0-4095) representing the raw ADC conversion. Returns 0 if the temperature sensor driver has not been initialized via ca_temp_init.

Note:

- This function is a non-blocking, thread-safe wrapper that interacts with the hardware ADC peripheral. It is safe to call from both tasks and ISRs, provided the driver has been initialized. The most common mistake is calling this function before invoking ca_temp_init, which will result in a return value of 0, potentially leading to incorrect temperature calculations if the caller does not check for this error condition.

Uso:

- This function is typically used in a periodic monitoring task to fetch raw data before processing it into a human-readable format. The standard sequence is: first, call ca_temp_init during system startup to configure the ADC and internal references; then, periodically call ca_temp_read_raw within a loop or timer-triggered task; finally, use the returned value to perform calculations or pass it to other conversion functions.

Esempio:

```c
/* Initialize the temperature sensor driver before use */
ca_temp_init();

/* Periodically read the raw sensor value */
u16 raw_val = ca_temp_read_raw();

if (raw_val != 0) {
    /* Perform conversion or logging using the raw data */
    ca_printf("Raw Temp ADC: %u\n", raw_val);
} else {
    /* Handle error: sensor not initialized or hardware failure */
    ca_printf("Error: Temperature sensor not ready.\n");
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_temp_read_raw() and why is it important to check the return value?
A: You should call ca_temp_read_raw() when you need to read the raw temperature value from the sensor, but only after ensuring the sensor has been properly initialized. It's important to check the return value because if the sensor is not initialized or there's a hardware error, the function returns 0, which could lead to incorrect temperature readings if not handled.

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_temp_read_raw() with caOS types?
A: u16 raw_temp = ca_temp_read_raw(); if (raw_temp != 0) { // Process valid temperature reading } else { // Handle sensor initialization or hardware error }

Return conf:0.90 **Q:** What do the return values of ca_temp_read_raw() mean and what causes the function to return 0?
A: The function returns a 12-bit raw ADC value representing the temperature if the sensor is properly initialized and there's no hardware error. It returns 0 if either the sensor has not been initialized correctly or if there is a hardware error, indicating that the reading should not be used.

`ca_temp_retune`

Header: `caos/drivers/inc/temp_sensor.h:38`
Source: `caos/drivers/src/temp_sensor.c:124:127`
GUARD: `EN_TEMP_SENSOR`

```
void ca_temp_retune(void); /* after a core clock change (see
    ca_pwr_set_freq) */
```

Recalculates temperature alarm thresholds after a change in the system core clock frequency.

Ritorna:
void

Note:

- This function is not thread-safe and must be called only when the system scheduler is in a known state, typically immediately after a frequency change. It is not intended to be called from an ISR because it performs internal logic that may rely on system state consistency. The primary precondition is that the temperature sensor driver must have been initialized via ca_temp_init. A common beginner mistake is forgetting to call this after changing the clock frequency; if omitted, the temperature alarm thresholds will be mathematically incorrect (based on the old clock speed), leading to false positives or, worse, failing to trigger an alarm when the chip is actually overheating.

Uso:

- This function is used exclusively in the context of power management and clock scaling. Whenever the system frequency is modified using ca_pwr_set_freq or similar clock-adjusting functions, the hardware-dependent timing values (like DTS alarm thresholds) become invalid. You must call this function as part of the 'retuning' sequence, which typically includes calling ca_os_tick_retune, ca_uart_retune, and ca_can_retune to ensure all time-dependent peripherals remain synchronized with the new core clock.

Esempio:

```
/* Example: Adjusting system frequency and updating dependent peripherals */
void update_system_performance(bool high_perf) {
    if (high_perf) {
        /* Assume this function sets the core clock to a higher frequency */
        /* ca_pwr_set_freq(HIGH_FREQ); */
    } else {
        (void)ca_os_clock_reduce_freq(50);
    }

    /* After changing the clock, we must retune all time-sensitive drivers */
    ca_os_tick_retune();
    ca_uart_retune();
    ca_temp_retune(); /* Recalculate DTS thresholds for the new PCLK speed */
}
```

Q&A — fine-tuning review *2/2 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When exactly should I call ca_temp_retune and what happens if I skip it?
A: You must call ca_temp_retune immediately after changing the system core clock frequency using ca_pwr_set_freq. If you omit this call, the temperature alarm thresholds will remain calculated based on the old clock speed, which makes them mathematically incorrect. This can lead to false positives or, more dangerously, a failure to trigger an alarm when the chip is actually overheating.

Code example conf:0.95 **Q:** How do I correctly update the system frequency and retune the temperature sensor in caOS?
A: After updating the clock frequency, you must perform a retuning sequence. Example: void update_system_clock(u32 new_freq) { ca_pwr_set_freq(new_freq); ca_temp_retune(); ca_os_tick_retune(); ca_uart_retune(); ca_can_retune(); }

ca_temp_set_alarm

Header: caos/drivers/inc/temp_sensor.h:37
Source: caos/drivers/src/temp_sensor.c:112:120
GUARD: EN_TEMP_SENSOR

```
bool ca_temp_set_alarm(i32 cold_c, i32 hot_c, ca_temp_alarm_cb_t cb)
```

Configures a temperature threshold alarm that triggers a callback when the DTS sensor crosses defined limits.

Parametri:

Nome	Tipo	Descrizione
cold_c	i32	The temperature threshold in Celsius for the cold alarm. Must be strictly less than hot_c.
hot_c	i32	The temperature threshold in Celsius for the hot alarm. Must be strictly greater than cold_c.
cb	ca_temp_alarm_cb_t	A function pointer to the callback to execute when a threshold is crossed. Pass NULL to disable the alarm.

Ritorna:
Returns true if the alarm was successfully configured. Returns false if the DTS sensor is not initialized, if cold_c is not less than hot_c, or if the configuration failed.

Note:

- This function is non-blocking and thread-safe. The callback provided runs directly within the context of the DTS (Digital Temperature Sensor) interrupt service routine (ISR). Because it runs in an ISR, the callback must be extremely fast, must not perform blocking operations (like waiting for a mutex or sleeping), and should avoid complex logic. A common mistake is to perform heavy processing or logging inside the callback, which can cause system instability or missed interrupts. Ensure the DTS hardware is initialized via ca_temp_init before calling this function.

Uso:

- This function is used to monitor thermal conditions without polling the sensor continuously in a loop. It is typically used in a system initialization sequence after ca_temp_init is called. You define a callback function that handles the thermal event (e.g., triggering a fan or reducing CPU frequency via ca_os_clock_reduce_freq) and pass it to this function. Once a threshold is crossed, the callback fires once; to re-arm the alarm, you must call ca_temp_set_alarm again.

Esempio:

```
/* Callback function to be executed in ISR context */
void my_thermal_callback(void)
{
    /* Perform non-blocking action, e.g., signal a task to handle overheating */
    /* ca_os_signal_handle(thermal_task_handle); */
}

void setup_thermal_monitoring(void)
{
    /* Ensure DTS is ready before setting alarms */
    if (ca_temp_dts_present()) {
        /* Set cold alarm at 10C and hot alarm at 80C */
        if (!ca_temp_set_alarm(10, 80, my_thermal_callback)) {
            /* Handle configuration error */
```

```
            ca_printf("Failed to set thermal alarm!\n");
        }
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_temp_set_alarm in my caOS application?
A: You should call it during the system initialization sequence, specifically after the DTS hardware has been initialized using ca_temp_init. It is used to monitor thermal conditions via interrupts rather than continuous polling.

Code example conf:0.90 **Q:** How do I implement a simple temperature alarm callback in caOS?
A: void my_alarm_cb(void) { /* Handle thermal event */ } ... if (ca_temp_set_alarm(20, 80, my_alarm_cb)) { /* Success */ }

Parameters conf:1.00 **Q:** What are the constraints for the cold_c and hot_c parameters?
A: The cold_c parameter must be strictly less than hot_c, and hot_c must be strictly greater than cold_c. If these conditions are not met, the function will return false.

Return conf:1.00 **Q:** Why would ca_temp_set_alarm return false?
A: It returns false if the DTS sensor is not initialized, if the cold_c and hot_c values violate the threshold constraints (cold_c must be less than hot_c), or if the configuration process fails.

`ca_tftp_get`

Header: `caos/net/inc/tftp_service.h:110`
Source: `caos/net/src/tftp_service.c:312:414`
GUARD: `EN_TFTP_SERVICE`

```
bool ca_tftp_get(const ca_tftp_cfg_t *cfg, ca_tftp_rx_block_t on_block,
    void *arg)
```

Downloads a file from a remote TFTP server using a callback-based mechanism.

Parametri:

Nome	Tipo	Descrizione
`cfg`	`const ca_tftp_cfg_t *`	Pointer to configuration structure containing server IP, filename, timeout, and retry settings. Must not be NULL.
`on_block`	`ca_tftp_rx_block_t`	Callback function invoked for every received data block. Must return true to continue or false to abort. Must not be NULL.
`arg`	`void *`	User-defined pointer passed directly to the on_block callback, useful for context (e.g., file handles or buffers).

Ritorna:
Returns true if the file was successfully received in its entirety. Returns false if the transfer failed, timed out, was aborted by the callback, or if parameters were invalid.

Note:

- This function is blocking and will suspend the calling task until the transfer completes or fails. It is NOT thread-safe for concurrent TFTP operations; it uses an internal mutex to ensure only one transfer occurs at a time. It must NOT be called from an ISR because it performs blocking network operations and mutex locking. A common mistake is providing

a callback that takes too long to process, which can cause the TFTP block timeout to trigger and lead to unnecessary retransmissions.

Uso:

- Typically used in a dedicated background task to fetch configuration files or firmware updates from a server. It is usually paired with ca_tftp_service_init to ensure the network stack is ready. The sequence is: 1) Initialize network/TFTP service, 2) Define a callback function to handle incoming data (e.g., writing to flash), 3) Call ca_tftp_get with the configuration, 4) Check the boolean return value to verify success.

Esempio:

```
/* Callback to process incoming data blocks */
bool my_rx_callback(u16 block_num, const u8 *data, u16 len, void *arg) {
    /* Logic to write 'data' to storage using 'arg' as a file handle */
    return true; /* Continue receiving */
}

void task_tftp_download(void *p) {
    ca_tftp_cfg_t cfg = { .filename = "config.bin", .block_timeout_ms = 2000 };

    /* Ensure TFTP service is ready */
    ca_tftp_service_init();

    /* Perform the download */
    if (ca_tftp_get(&cfg, my_rx_callback, NULL)) {
        /* Success */
    } else {
        /* Handle error */
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_tftp_get in a caOS application?
A: You should use ca_tftp_get when you need to download a file from a TFTP server in a blocking manner. It is ideal for scenarios where the application must wait for the file transfer to complete before proceeding, such as fetching firmware updates or configuration files during system initialization.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_tftp_get with caOS types?
A: ca_tftp_cfg_t cfg = { .file = "config.txt", .timeout = 1000, .retries = 3 }; bool on_block(u16 block_num, const u8 *data, u16 len, void *arg) { /* Process block */ return true; } bool success = ca_tftp_get(&cfg, on_block, NULL);

Parameters conf:0.90 **Q:** What are the valid values for the cfg parameter in ca_tftp_get?
A: The cfg parameter must be a valid pointer to a ca_tftp_cfg_t structure. It cannot be NULL. The structure must contain a valid file name, a timeout value in milliseconds, and a retry count. Invalid values include NULL pointer or uninitialized fields.

Return conf:0.95 **Q:** What does ca_tftp_get return when it succeeds or fails?
A: It returns true if the file was downloaded successfully. It returns false if there was an error during initialization, configuration, or during the transfer, such as timeout, server error, or if the callback returns false.

Error/Edge case conf:0.90 **Q:** What are common mistakes when calling ca_tftp_get?
A: Common mistakes include passing NULL to cfg or on_block, which causes an immediate return of false. Another error is having a callback that returns false, which stops the transfer. Also, calling this function before the network system is initialized or when s_xfer_mtx is not available can lead to failures.

Behavior conf:0.90 **Q:** Is ca_tftp_get thread-safe and can it be called from an ISR?
A: Yes, ca_tftp_get is thread-safe because it uses mutexes to protect shared resources. However, it is not ISR-safe and must not be called from an ISR because it performs blocking operations and uses mutexes.

---

ca_tftp_put

Header: `caos/net/inc/tftp_service.h:120`
Source: `caos/net/src/tftp_service.c:419:512`
GUARD: `EN_TFTP_SERVICE`

```
bool ca_tftp_put(const ca_tftp_cfg_t *cfg, const u8 *data, u32 total_len
    )
```

Uploads a data buffer to a remote TFTP server using the TFTP protocol.

Parametri:

Nome	Tipo	Descrizione
`cfg`	`const ca_tftp_cfg_t *`	Pointer to a configuration structure containing server IP, filename, timeout, and retry settings. Must not be NULL.
`data`	`const u8 *`	Pointer to the memory buffer containing the data to be uploaded. Must not be NULL.
`total_len`	`u32`	Total size of the data buffer in bytes.

Ritorna:
Returns true if the file was successfully acknowledged by the server, false otherwise (e.g., invalid parameters, network timeout, or server error).

Note:

- This function is blocking; it will suspend the calling task until the entire transfer completes or the retry limit is reached. It is NOT safe to call from an ISR because it uses mutexes for thread safety and performs network I/O. Before calling this, ensure the network stack is initialized and the TFTP service is ready. A common mistake is passing a pointer to a buffer that is modified by another task while the transfer is in progress, which will corrupt the uploaded data.

Uso:

- This function is typically used to push configuration files or firmware updates to a remote server. It is usually called after verifying network connectivity. It pairs with ca_tftp_service_init to ensure the underlying network buffers and state machines are ready. The sequence is: initialize the network, prepare the configuration struct, call ca_tftp_put, and then handle the boolean result to determine if the operation succeeded.

Esempio:

```
ca_tftp_cfg_t cfg;
ca_memzero(&cfg, sizeof(cfg));
cfg.filename = "config.bin";
// Assume data_buffer is already populated with valid data
if (ca_tftp_put(&cfg, data_buffer, data_size)) {
    // Transfer successful
} else {
    // Handle error: log failure or retry
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_tftp_put to send a file via TFTP?
A: You should use ca_tftp_put when you need to send a file to a TFTP server in PUT mode. It is designed for use within tasks, not ISRs, and is a blocking operation that waits until the transfer completes or fails.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_tftp_put with caOS types?
A: ca_tftp_cfg_t cfg = { .file_name = "test.txt", .server_ip = 0xC0A80101, .timeout_ms = 1000, .retry_count = 3 }; u8 data[1024] = {0}; bool result = ca_tftp_put(&cfg, data, 1024);

Parameters conf:0.95 **Q:** What are the valid values for the data parameter in ca_tftp_put?
A: The data parameter must be a valid pointer to a buffer containing the data to be sent. It cannot be NULL. The buffer must contain at least total_len bytes of data to be transmitted.

Return conf:0.95 **Q:** What does ca_tftp_put return when the transfer is successful?
A: It returns true when the TFTP transfer is completed successfully. This indicates that all data was sent and acknowledged by the server.

Error/Edge case conf:0.90 **Q:** What causes ca_tftp_put to return false?
A: It returns false if the configuration is invalid, if there is a timeout, network error, or if the server fails to acknowledge the transfer. Passing NULL for cfg or data also causes an immediate return of false.

Behavior conf:0.90 **Q:** Is ca_tftp_put safe to call from an ISR or is it thread-safe?
A: No, ca_tftp_put is not safe to call from an ISR because it is a blocking function that uses RTOS primitives like mutexes and timeouts. However, it is thread-safe due to the use of a mutex (s_xfer_mtx) to protect the transfer operations.

`ca_tftp_receive`

Header: `caos/net/inc/tftp_service.h:164`
Source: `caos/net/src/tftp_service.c:666:734`
GUARD: `EN_TFTP_SERVICE`

```
bool ca_tftp_receive(const ca_tftp_srv_cfg_t *cfg, ca_tftp_wrq_t on_wrq,
    ca_tftp_rx_block_t on_block, void *arg, u32 wait_ms)
```

Starts a blocking TFTP server listener to receive a single file transfer (WRQ) over UDP.

Parametri:

Nome	Tipo	Descrizione
`cfg`	`const ca_tftp_srv_cfg_t *`	Configuration structure containing local MAC, optional port, and error reporting pointer. Must not be NULL.
`on_wrq`	`ca_tftp_wrq_t`	Callback function invoked when a Write Request (WRQ) is received. Return 0 to accept, or a TFTP error code to refuse.
`on_block`	`ca_tftp_rx_block_t`	Callback function invoked for every received data block. Must not be NULL.
`arg`	`void *`	User-defined pointer passed to the callbacks for context (e.g., file handles or status structures).
`wait_ms`	`u32`	Timeout in milliseconds to wait for an incoming WRQ. Use a large value or a specific constant for indefinite waiting.

Ritorna:

Returns true if the file was received successfully, false if a timeout occurred, the request was refused, or an error occurred during transfer.

Note:

- This function is blocking and must be called from a task context, never from an ISR. It is thread-safe regarding the internal service state, but only one task can listen for a WRQ at a time; subsequent calls will return false if the service is already waiting. The function handles the entire lifecycle of the TFTP transfer: it binds the UDP port, waits for the initial request, executes the transfer loop, and automatically unbinds the port upon completion or error. A common mistake is providing a NULL pointer for the mandatory configuration or block callback, which will cause an immediate return of false.

Uso:

- Used primarily for firmware updates or file uploads where the device acts as a TFTP server. It is typically called in a loop or a dedicated update task. The sequence is: 1) Prepare a configuration structure, 2) Call ca_tftp_receive, 3) Process the result (e.g., verify file integrity or flash the received data). It is often paired with custom callback functions that handle the actual writing of data to flash memory or storage.

Esempio:

```
/* Example: Waiting for a firmware update file */
ca_tftp_err_t err_info;
ca_tftp_srv_cfg_t cfg = { .local_mac = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55}, .
    err = &err_info };

/* User-defined context for the transfer */
struct { u32 bytes; } ctx = { 0 };

/* Wait for a WRQ for up to 60 seconds */
if (ca_tftp_receive(&cfg, NULL, my_on_block_callback, &ctx, 60000U)) {
    ca_printf_n("Transfer complete: %u bytes received", ctx.bytes);
} else {
    ca_printf_n("Transfer failed or timed out. Error code: %u", err_info.code);
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_tftp_receive function in my caOS application?
A: You should use this function when your device needs to act as a TFTP server to receive files, such as during firmware updates or file uploads. It is typically called within a dedicated update task or a loop.

Code example conf:0.90 **Q:** Provide a minimal C example of how to call ca_tftp_receive.
A: ca_tftp_srv_cfg_t cfg = { .mac = {0x02, 0x00, 0x00, 0x00, 0x00, 0x01}, .port = 69 }; bool success = ca_tftp_receive(&cfg, my_wrq_cb, my_block_cb, NULL, 5000);

Parameters conf:1.00 **Q:** What are the requirements for the 'cfg' and 'on_block' parameters?
A: The 'cfg' parameter must be a pointer to a valid ca_tftp_srv_cfg_t structure and cannot be NULL. The 'on_block' parameter is a mandatory callback function for processing data blocks and also cannot be NULL.

Return conf:1.00 **Q:** What does a return value of false indicate when calling ca_tftp_receive?
A: A return value of false indicates that the file transfer failed. This can be caused by a timeout, the WRQ being refused by the callback, an error occurring during the transfer, or the service being already in use by another task.

Error/Edge case conf:1.00 **Q:** What is a common mistake that causes ca_tftp_receive to return false immediately?
A: A common mistake is providing a NULL pointer for the mandatory configuration structure or the required block callback function, which results in an immediate return

of false.

Behavior conf:1.00 **Q:** Is ca_tftp_receive safe to call from an ISR or multiple tasks simultaneously?
A: No, it is a blocking function and must never be called from an ISR. While it is thread-safe regarding internal state, only one task can listen for a WRQ at a time; subsequent calls from other tasks will return false if the service is already waiting.

`ca_tftp_service_init`

Header: `caos/net/inc/tftp_service.h:128`
Source: `caos/net/src/tftp_service.c:670:677`
GUARD: `EN_TFTP_SERVICE`

```
void ca_tftp_service_init(ca_mac_t local_mac)
```

Initializes the TFTP service and registers its CLI interface.

Parametri:

Nome	Tipo	Descrizione
`local_mac`	`ca_mac_t`	The MAC address assigned to the network interface that will handle TFTP traffic. Must be a valid, initialized ca_mac_t structure.

Ritorna:
void

Note:

- This function is not thread-safe and must be called during the system initialization phase, typically before the RTOS scheduler starts. It is not intended to be called from an Interrupt Service Routine (ISR). The function performs internal memory and state initialization; calling it multiple times after the service has already started may lead to undefined behavior or memory leaks. Ensure the network stack is ready before invoking this, as it relies on underlying network structures.

Uso:

- This function is part of the network service initialization sequence. It is typically called in the main configuration block alongside other services like ca_arp_service_init or ca_dhcp_service_init. It must be called before the corresponding service start function (if applicable) to ensure the internal state is prepared. It is often used in systems where a Command Line Interface (CLI) is enabled to allow remote file transfers via TFTP.

Esempio:

```
/* Define the local MAC address */
mac_t s_mac = { .addr = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55} };

void system_init(void) {
    /* Initialize network services before starting the OS */
    ca_arp_service_init(s_mac, (ca_ipv4_t){0});

    /* Initialize TFTP service to enable file transfer capabilities */
    ca_tftp_service_init(s_mac);

    /* Proceed to start the OS scheduler */
    ca_os_run();
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should ca_tftp_service_init() be called during system initialization, and why is it important?
A: ca_tftp_service_init() should be called once during the system initialization phase, before any network operations involving TFTP. It initializes the TFTP service, registers CLI functionality if enabled, and configures the local MAC address. This ensures that the TFTP service is ready to handle requests and that communication can proceed correctly.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_tftp_service_init() with caOS types?
A: u8 local_mac[6] = {0x00, 0x08, 0x9B, 0x00, 0x00, 0x01}; ca_tftp_service_init(local_mac);

Parameters conf:0.90 **Q:** What is the significance of the local_mac parameter in ca_tftp_service_init(), and what happens if it is invalid?
A: The local_mac parameter specifies the MAC address of the local device. It must be a valid 6-byte array; otherwise, the behavior is undefined. An invalid MAC address can lead to communication errors or unexpected behavior in network operations involving TFTP.

`ca_timer_capture_event`

Header: `caos/drivers/inc/timer.h:93`
Source: `caos/drivers/src/timer.c:200:208`
GUARD: `EN_TIMER`

```
void ca_timer_capture_event(u8 num, u32 ccr, u32 arr); /* hw/: one
   capture, arr = counter wrap mask */
```

Processes a hardware timer capture event to calculate the time interval between consecutive pulses.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The timer index (1-based). Must be between 1 and TIMER_MAX.
`ccr`	`u32`	The raw counter value captured from the hardware timer register at the moment of the event.
`arr`	`u32`	The counter wrap mask (e.g., 0xFFFF for a 16-bit timer) used to handle counter overflow correctly.

Ritorna:
void

Note:

- This function is designed to be called directly from an Interrupt Service Routine (ISR). It is not thread-safe in the sense that it relies on the hardware timer state being consistent; if called from outside an ISR, you must ensure the timer hardware is not currently updating the capture registers. The function uses the 'arr' mask to perform modular arithmetic, which is critical for calculating the delta between captures even if the timer counter has wrapped around since the last event. A common mistake is passing an incorrect 'arr' mask, which will result in garbage time-interval calculations when the timer overflows.

Uso:

- This function is used in high-precision timing applications, such as measuring the frequency or pulse width of an incoming signal. It is typically paired with ca_timer_init_capture,

which sets up the timer hardware and registers the callback function (ic_cb). Once initialized, the hardware triggers the interrupt, and the ISR calls this function to process the raw capture value and trigger the user-defined callback with the calculated delta.

Esempio:

```c
/* Example: Handling a timer capture event in an ISR */
void TIM1_IRQHandler(void) {
    /* Assume hardware provides the captured value and the timer wrap mask */
    u32 captured_value = 0; /* Read from hardware register */
    u32 wrap_mask = 0xFFFF; /* 16-bit timer mask */

    /* Process the capture event using caOS timer logic */
    ca_timer_capture_event(1, captured_value, wrap_mask);

    /* Clear hardware interrupt pending bit here */
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_timer_capture_event and why is it important to call it from an ISR?
A: You should call ca_timer_capture_event within an Interrupt Service Routine (ISR) triggered by a hardware timer capture event. It is designed for high-precision timing, such as measuring signal frequency or pulse width. Calling it from an ISR ensures the hardware state is consistent; if called outside an ISR, you must manually ensure the timer hardware is not currently updating the capture registers to avoid race conditions.

Code example conf:0.90 **Q:** How do I implement a basic ISR handler using ca_timer_capture_event?
A: void TIM2_IRQHandler(void) { u8 timer_idx = 2; u32 ccr_val = TIM2->CCR1; u32 arr_mask = 0xFFFF; ca_timer_capture_event(timer_idx, ccr_val, arr_mask); }

Parameters conf:1.00 **Q:** What is the purpose of the 'arr' parameter and what happens if I provide an incorrect value?
A: The 'arr' parameter is the counter wrap mask (e.g., 0xFFFF for a 16-bit timer) used to handle counter overflow. It allows the function to perform modular arithmetic to correctly calculate the delta between captures even if the timer counter has wrapped around. Providing an incorrect mask will result in garbage time-interval calculations.

`ca_timer_clock_hz`

Header: `caos/drivers/inc/timer.h:76`
Source: `caos/drivers/src/timer.c:193:197`
GUARD: `EN_TIMER`

```c
u32 ca_timer_clock_hz (u8 num); /* counting rate, ticks per second */
```

Returns the input clock frequency in Hz for a specific hardware timer.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The hardware timer index. Must be between 1 and TIMER_MAX. Passing an invalid index returns 0.

Ritorna:
The clock frequency in Hz as a u32. Returns 0 if the timer index is out of range or if the timer is not initialized.

Note:

- This function is non-blocking and thread-safe, as it simply reads a pre-configured value from the timer's hardware abstraction layer. It is safe to call from an ISR because it does not perform any complex operations or resource locking. The most common mistake is assuming the returned value is the actual counting rate (ticks per second) without considering the prescaler; this function returns the raw input clock frequency of the timer peripheral, not the frequency after the prescaler division.

Uso:

- This function is typically used in calibration or measurement tasks where you need to convert raw timer tick counts into physical time units (like seconds or milliseconds). It is usually paired with ca_timer_init_capture or ca_timer_init_counter to set up the hardware, followed by reading the counter, and finally using this function to calculate the frequency or period of an external signal. It is essential for converting raw tick deltas into meaningful engineering units.

Esempio:

```c
/* Example: Measuring the frequency of an external signal */
#define IC_TIM 1
#define IC_DIV 1

void measure_signal(void) {
    /* Initialize timer for capture mode */
    if (ca_timer_init_capture(IC_TIM, 1, 0, 0)) {
        ca_timer_start(IC_TIM);

        /* Wait for some time to collect samples */
        ca_os_sleep(1000);

        /* Get the base clock frequency of the timer */
        u32 clk_hz = ca_timer_clock_hz(IC_TIM);

        if (clk_hz > 0) {
            /* Perform calculations using the clock frequency */
            /* ... logic to read capture values and compute frequency ... */
        }

        ca_timer_stop(IC_TIM);
        ca_timer_deinit(IC_TIM);
    }
}
```

Q&A — fine-tuning review

4/4 pairs · gemini-3.1-flash-lite

Usage conf:1.00 **Q:** When should I use ca_timer_clock_hz in my application?
A: You should use this function during calibration or measurement tasks where you need to convert raw timer tick counts into physical time units like seconds or milliseconds. It is typically used after initializing a timer with ca_timer_init_capture or ca_timer_init_counter to calculate the frequency or period of an external signal.

Code example conf:1.00 **Q:** How do I retrieve the clock frequency for timer 1 and store it in a variable?
A: You can retrieve the frequency using the following code: u8 timer_idx = 1; u32 freq = ca_timer_clock_hz(timer_idx);

Parameters conf:1.00 **Q:** What are the constraints for the 'num' parameter in ca_timer_clock_hz?
A: The 'num' parameter represents the hardware timer index. It must be a value between 1 and TIMER_MAX. If you pass an index outside of this range, the function will return 0.

Return conf:1.00 **Q:** What does a return value of 0 indicate when calling ca_timer_clock_hz?

A: A return value of 0 indicates that the function failed. This occurs if the provided timer index is out of range (not between 1 and TIMER_MAX) or if the specified timer has not been initialized.

`ca_timer_counter_read`

Header: `caos/drivers/inc/timer.h:67`
Source: `caos/drivers/src/timer.c:203:212`
GUARD: `EN_TIMER`

```
void ca_timer_counter_read(u8 num); /* aggiorna *count += delta */
```

Updates a software counter by calculating the delta from the hardware timer since the last read.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The timer slot index (1-based). Must be between 1 and TIMER_MAX. Invalid values result in an immediate return.

Ritorna:
void. No value is returned, but the internal counter variable passed during initialization is updated.

Note:

- This function is non-blocking and performs a simple register read and arithmetic update. It is not thread-safe; if multiple tasks access the same timer slot, you must protect it with a mutex. It is safe to call from an ISR, provided the timer slot was initialized correctly. The most common mistake is calling this on a timer that was not initialized with ca_timer_init_counter, which will cause the function to return early without doing anything.

Uso:

- Use this function periodically in a task loop or an ISR to accumulate hardware timer ticks into a larger 64-bit software counter. It is typically used after calling ca_timer_init_counter to track long-term pulse counts or frequency-based events. You should pair this with a periodic task that calls this function at a fixed rate to ensure the hardware timer does not overflow between reads.

Esempio:

```
/* Example: Tracking pulses from a sensor */
u64 total_pulses = 0;

void task_pulse_counter(void *arg) {
    /* Initialize timer 1 in counter mode, linking it to our variable */
    ca_timer_init_counter(1, &total_pulses);

    while (1) {
        /* Update the total_pulses variable with new hardware ticks */
        ca_timer_counter_read(1);

        /* Log the current count every 100ms */
        ca_printf("Total pulses: %llu\n", total_pulses);
        ca_os_delay_ms(100);
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_timer_counter_read() and what is its purpose in a real-time embedded system?
A: You should call ca_timer_counter_read() when you need to update a counter value based on the elapsed time measured by a hardware timer configured in counter mode. It is useful for tracking time intervals or measuring durations in real-time systems where precise timing is required. The function is non-blocking and can be safely invoked from both task contexts and interrupt service routines (ISRs).

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_timer_counter_read() with caOS types?
A: u8 timer_num = 1; ca_timer_counter_read(timer_num);

Parameters conf:0.90 **Q:** What does the 'num' parameter in ca_timer_counter_read() represent, and what happens if it's outside the valid range?
A: The 'num' parameter represents the timer number to read, ranging from 1 to TIMER_MAX. If the value is outside this valid range, the function returns immediately without performing any action or updating the counter.

`ca_timer_deinit`

Header: `caos/drivers/inc/timer.h:64`
Source: `caos/drivers/src/timer.c:188:201`
GUARD: `EN_TIMER`

```
bool ca_timer_deinit(u8 num)
```

Safely stops and releases a hardware timer resource for reuse.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The timer identifier. Must be between 1 and TIMER_MAX. Values outside this range will cause the function to return false.

Ritorna:
Returns true if the timer was successfully deinitialized, or false if the provided timer number is invalid.

Note:

- This function is not thread-safe and should not be called from an ISR. It performs a hard stop on the hardware timer and clears all associated callback pointers and configuration data in the internal slot structure. A common mistake is attempting to use the timer handle after calling this function without re-initializing it, which will lead to undefined behavior or null pointer dereferences if the callback is triggered. Always ensure that no other tasks are currently waiting on or using this specific timer before calling deinit.

Uso:

- Use this function when you are finished with a specific timer peripheral to free it up for other parts of your application. It is typically used in the cleanup phase of a driver or a task lifecycle. The standard sequence is: ca_timer_init (or specific init functions like ca_timer_init_periodic) -> use the timer -> ca_timer_stop -> ca_timer_deinit. This ensures the hardware is in a clean state and the software slot is ready to be re-allocated.

Esempio:

```c
/* Example: Cleaning up a timer after a task completes its work */
u8 my_timer_id = 1;

// ... timer was previously initialized and used ...

// Stop the timer to ensure no further interrupts occur
if (ca_timer_stop(my_timer_id)) {
    // Successfully stopped, now release the resource
    if (ca_timer_deinit(my_timer_id)) {
        // Timer is now fully deinitialized and available for other tasks
    } else {
        // Handle error: invalid timer ID
    }
} else {
    // Handle error: timer was not busy or failed to stop
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_timer_deinit() in my caOS application?
A: You should call ca_timer_deinit() when you want to stop and release the hardware resources of a specific timer that was previously initialized. This is typically done when the timer is no longer needed, such as during system shutdown, module deinitialization, or when dynamically managing timer resources.

Code example conf:0.90 **Q:** Can you show a minimal example of using ca_timer_deinit() with caOS types?
A: u8 timer_num = 3; bool result = ca_timer_deinit(timer_num); if (result) { // Timer successfully deinitialized } else { // Failed to deinitialize timer }

Parameters conf:0.95 **Q:** What does the 'num' parameter in ca_timer_deinit() represent and what are valid values?
A: The 'num' parameter represents the hardware timer number to be deinitialized. Valid values are integers from 1 to TIMER_MAX inclusive. Any value outside this range will cause the function to return false without performing any action.

Return conf:0.90 **Q:** What do the return values of ca_timer_deinit() indicate and what causes it to return false?
A: The function returns true if the timer was successfully deinitialized and its resources were released. It returns false if the input parameter 'num' is invalid (less than 1 or greater than TIMER_MAX) or if the timer is already inactive (ca_timer_is_busy returns false).

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_timer_deinit()?
A: Common mistakes include calling ca_timer_deinit() on a timer number that is out of range (less than 1 or greater than TIMER_MAX), or calling it on a timer that was never initialized or is already inactive. Additionally, calling this function from multiple threads simultaneously may cause race conditions, and it's not recommended to call it from an ISR due to potential synchronization issues.

`ca_timer_dispatch`

Header: `caos/drivers/inc/timer.h:72`
Source: `caos/drivers/src/timer.c:227:239`
GUARD: `EN_TIMER`

```c
void ca_timer_dispatch (u8 num); /* chiamare da TIMx_IRQHandler */
```

Dispatches timer events by executing the associated callback function when a hardware timer interrupt occurs.

Parametri:

Nome	Tipo	Descrizione
num	u8	The timer slot index (1-based). Must be between 1 and TIMER_MAX. Values outside this range are ignored.

Ritorna:
void

Note:

- This function is designed to be called exclusively from within a hardware timer interrupt service routine (ISR). It is not thread-safe and must not be called from standard tasks, as it assumes the system is already in an interrupt context. Because it executes user-defined callbacks, keep the callback logic extremely short and fast to avoid blocking the CPU and causing jitter in other interrupts. A common mistake is performing heavy operations (like memory allocation or complex I/O) inside the callback, which can lead to system instability or missed interrupts.

Uso:

- This function is the bridge between the hardware timer peripheral and the caOS timer management system. It is typically invoked inside the STM32 TIMx_IRQHandler. The standard lifecycle involves initializing a timer with ca_timer_init_periodic or ca_timer_init_oneshot, starting it with ca_timer_start, and then calling ca_timer_dispatch inside the corresponding ISR to trigger the registered callback function.

Esempio:

```c
/* Example: Handling a timer interrupt in the STM32 vector table */
void TIM2_IRQHandler(void) {
    /* Check if the hardware timer flag is set */
    /* If flag is set, clear it to acknowledge the interrupt */

    /* Dispatch the caOS timer event for slot 1 */
    ca_timer_dispatch(1);
}

/* Example: Initializing a periodic timer */
void setup_timer(void) {
    /* Initialize timer 1 as periodic with a specific callback */
    ca_timer_init_periodic(1, my_callback_function);
    ca_timer_start(1);
}

void my_callback_function(void) {
    /* Keep this code extremely fast */
    ca_gpio_toggle(1, 5); /* Toggle LED on Port 1, Pin 5 */
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_timer_dispatch() and why is it important for timer interrupt handling?
A: You should call ca_timer_dispatch() from within a timer interrupt handler (ISR), specifically from TIMx_IRQHandler. It is important because it processes the timer's callback function based on the timer's mode (ONESHOT, PERIODIC, or COUNTER_IRQ). This ensures that timer events are properly handled and that the correct callback is executed for each timer instance.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_timer_dispatch() in an ISR?

A: void TIM1_IRQHandler(void) { ca_timer_dispatch(1); }

Parameters conf:0.90 **Q:** What does the 'num' parameter in ca_timer_dispatch() represent, and what are the valid values?

A: The 'num' parameter represents the timer number (1-16). It must be between 1 and TIMER_MAX, otherwise the function does nothing. Valid values are integers from 1 up to the maximum number of timers supported by the hardware platform.

Return conf:0.95 **Q:** Does ca_timer_dispatch() return any value, and what does it mean if it does?

A: The function does not return any value. It is void. Since it cannot fail, there are no return values to indicate success or failure. It simply processes the timer interrupt and executes the associated callback if the timer number is valid.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_timer_dispatch()?

A: A common mistake is calling ca_timer_dispatch() from outside an ISR context, which violates its intended usage. Another edge case is passing an invalid timer number (less than 1 or greater than TIMER_MAX), which causes the function to do nothing. It is also not thread-safe, so it must only be called from ISR context.

`ca_timer_encoder_read`

Header: `caos/drivers/inc/timer.h:68`
Source: `caos/drivers/src/timer.c:214:223`
GUARD: `EN_TIMER`

```
void ca_timer_encoder_read(u8 num); /* aggiorna *pos += delta signed */
```

Updates an encoder's accumulated position by reading the hardware timer delta.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The timer index (1-based). Must be between 1 and TIMER_MAX. Invalid values result in an immediate return.

Ritorna:
None (void).

Note:

- This function is not thread-safe and should not be called from an ISR. It relies on the internal state of the timer slot, specifically the 'last_cnt' value, which is updated every time this function is called. If you call this function too infrequently, the hardware timer might wrap around multiple times, leading to incorrect delta calculations. Ensure the timer is initialized in encoder mode using the appropriate initialization function before calling this.

Uso:

- This function is typically used in a periodic task or a main loop to poll the encoder's movement. It is paired with ca_timer_init_encoder during the system setup phase to configure the hardware timer for quadrature decoding. After initialization, you call this function periodically to sync the hardware counter with your software-tracked position variable.

Esempio:

```
/* Global variable to track encoder position */
i64 g_encoder_pos = 0;
```

```
3
4  void task_encoder_monitor(void *arg) {
5      /* Initialize timer 1 as an encoder, linking it to our variable */
6      /* ca_timer_init_encoder(1, &g_encoder_pos); */
7
8      while (1) {
9          /* Update the software counter based on hardware timer 1 */
10         ca_timer_encoder_read(1);
11
12         /* Log the current position */
13         ca_printf("Current encoder position: %lld\n", g_encoder_pos);
14
15         /* Wait 100ms before next poll */
16         ca_os_delay_ms(100);
17     }
18 }
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_timer_encoder_read() and what is its purpose in an embedded application?
A: You should call ca_timer_encoder_read() to update the position value of an optical encoder by reading the timer counter and calculating the difference from the previous value. It is useful for monitoring the movement of an encoder in real-time, especially in applications involving motor control or position tracking. The function is non-blocking and can be safely called from both task context and interrupt service routines (ISR).

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_timer_encoder_read() with caOS types?
A:

```
1 u8 timer_num = 1;
2 ca_timer_encoder_read(timer_num);
```

Parameters conf:0.90 **Q:** What does the 'num' parameter in ca_timer_encoder_read() represent, and what are valid values for it?
A: The 'num' parameter represents the timer number to use for reading the encoder, ranging from 1 to TIMER_MAX. If the value is outside this range, the function returns immediately without effect. Valid values are integers within this inclusive range, and invalid values (less than 1 or greater than TIMER_MAX) cause the function to do nothing.

`ca_timer_hw_clock_hz`

Header: `caos/drivers/inc/ca_glue_hw.h:314`
GUARD: `EN_CA_GLUE_HW`

```
1 extern u32 ca_timer_hw_clock_hz (u8 num); /* counting rate of timer num
    (PSC = 0) */
```

Returns the base input frequency in Hz of a specific hardware timer when its prescaler is set to zero.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The hardware timer index (typically 1-based). Values outside the valid range defined by the hardware abstraction layer will result in an invalid operation.

Ritorna:
The frequency in Hz as a u32. Returns 0 if the timer index is invalid or if the hardware clock configuration for that timer is not initialized.

Note:

- This function is non-blocking and thread-safe as it performs a simple register read or lookup. It is safe to call from an ISR because it does not involve any OS primitives, mutexes, or blocking delays. The primary precondition is that the system clock and the specific timer peripheral clock must be initialized by the board support package before calling this. A common beginner mistake is assuming this returns the current counting frequency after a prescaler has been applied; it specifically returns the clock rate at PSC=0, which is the raw input frequency to the timer's prescaler stage.

Uso:

- This function is used when you need to calculate precise timing intervals, such as converting a desired microsecond delay into a timer counter value (ARR/CCR registers). It is typically used during the initialization phase of a driver or a capture/compare task to determine the timebase resolution. You will often see it paired with ca_timer_init_counter or ca_timer_init_pwm to ensure the timer is configured to the correct frequency for the application's timing requirements.

Esempio:

```c
/* Example: Calculate the number of ticks required for a 1ms delay */
#include "caos.h"

void setup_timer_delay(u8 timer_id) {
    u32 clock_hz = ca_timer_clock_hz(timer_id);

    if (clock_hz > 0) {
        /* Calculate ticks for 1ms: (Clock / 1000) */
        u32 ticks_for_1ms = clock_hz / 1000U;

        /* Initialize the timer as a periodic counter with the calculated period */
        /* Note: ca_timer_init_periodic is the standard way to set up the timer base */
        ca_timer_init_periodic(timer_id, ticks_for_1ms);
        ca_timer_start(timer_id);
    } else {
        /* Handle error: invalid timer or clock not configured */
        ca_printf("Error: Could not determine clock for timer %d\n", timer_id);
    }
}
```

Q&A — fine-tuning review *4/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_timer_hw_clock_hz in my caOS application?
A: You should call this function during the initialization phase of a driver or a capture/-compare task. It is used to determine the raw timebase resolution of a timer, which allows you to calculate the correct ARR or CCR values needed to achieve specific timing intervals or PWM frequencies.

Code example conf:0.90 **Q:** How do I use ca_timer_hw_clock_hz to calculate a timer period in C?
A: You can use it to determine the input frequency before setting your timer parameters: u8 timer_id = 1; u32 clock_hz = ca_timer_hw_clock_hz(timer_id); if (clock_hz > 0) { /* Use clock_hz to calculate ARR for desired frequency */ }

Parameters conf:1.00 **Q:** What does the 'num' parameter represent and what happens if I pass an invalid value?

A: The 'num' parameter is a u8 representing the hardware timer index, which is typically 1-based. If you provide a value outside the valid range defined by the hardware abstraction layer, the function will return 0, indicating an invalid operation.

Return conf:1.00 **Q:** What does a return value of 0 from ca_timer_hw_clock_hz indicate?

A: A return value of 0 indicates that either the provided timer index is invalid, or the hardware clock configuration for that specific timer has not been initialized by the board support package.

`ca_timer_hw_deinit`

Header: `caos/drivers/inc/ca_glue_hw.h:276`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_timer_hw_deinit (u8 num); /* ferma e disabilita il timer
   rilasciando le risorse */
```

Low-level hardware cleanup for a specific timer peripheral.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The timer index (1 to TIMER_MAX). Passing an invalid index will result in the function returning early without performing any action.

Ritorna:
void

Note:

- This function performs direct register manipulation to disable the timer's clock enable bit (CEN), disable update interrupts (UIE), and clear the status register (SR). It is a non-blocking, low-level hardware operation that is not thread-safe; it assumes the caller has already ensured no other task is accessing this specific timer. It is safe to call from an ISR, but it is typically intended for use during system shutdown or resource reallocation. A common mistake is calling this function while the timer is still being used by an active PWM or encoder task, which will cause immediate hardware failure or unexpected behavior in the application logic.

Uso:

- This function is part of the hardware abstraction layer (HAL) for timers. It is typically called by the higher-level ca_timer_deinit function as the final step in the teardown sequence. The standard usage pattern is: first stop the timer using ca_timer_stop, then call ca_timer_deinit to clean up software structures, and finally, this function is invoked to ensure the hardware peripheral is completely silenced and reset to a safe state before the resource is released for other uses.

Esempio:

```
/* Example: Safely shutting down a timer peripheral */
void shutdown_timer_resource(u8 timer_id) {
    /* 1. Stop the timer operation first */
    ca_timer_stop(timer_id);

    /* 2. Perform hardware-level deinitialization to clear registers */
    ca_timer_hw_deinit(timer_id);

    /* 3. Clear software state and release the timer slot */
```

```
    /* Note: 'timer_release' is not in the catalog, so we assume the
       higher-level ca_timer_deinit handles the slot cleanup */
    ca_timer_deinit(timer_id);
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should ca_timer_hw_deinit be called, and what is its purpose in a caOS application?
A: ca_timer_hw_deinit should be called to disable and stop a hardware timer, releasing its associated resources. It is typically used after stopping the timer with ca_timer_hw_stop to fully shut down the timer peripheral. This function ensures that the timer is no longer generating interrupts and that its status register is cleared.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_timer_hw_deinit with caOS types?
A:

```
u8 timer_num = 2;
ca_timer_hw_deinit(timer_num);
```

Parameters conf:0.90 **Q:** What is the meaning of the 'num' parameter in ca_timer_hw_deinit, and what are the valid values?
A: The 'num' parameter specifies the number of the timer to be deinitialized. Valid values are integers from 1 to TIMER_MAX. If an invalid value (less than 1 or greater than TIMER_MAX) is passed, the function returns immediately without performing any action.

HW differences conf:0.85 **Q:** Are there any differences in how ca_timer_hw_deinit is implemented across different STM32 MCU families?
A: No, the implementation of ca_timer_hw_deinit is consistent across all supported STM32 families (f1xx, f4xx, f7xx, h5xx, h7xx). It performs the same operations on the timer peripheral: disabling the timer counter (CR1), clearing the update interrupt enable bit (DIER), and resetting the status register (SR).

`ca_timer_hw_get_cnt`

Header: `caos/drivers/inc/ca_glue_hw.h:277`
GUARD: `EN_CA_GLUE_HW`

```
extern u32 ca_timer_hw_get_cnt (u8 num); /* legge il valore corrente del
    contatore del timer */
```

Reads the current raw value of the hardware timer counter register.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The identifier of the hardware timer to read. Must be a valid index previously initialized by the system.

Ritorna:
The current 32-bit value of the timer's CNT register. Returns 0U if the provided timer index is invalid or not initialized.

Note:

- This function is non-blocking and executes in constant time, making it safe to call from both tasks and Interrupt Service Routines (ISRs). It provides direct access to the hardware

register, meaning it does not account for timer overflows or software-level abstractions. A common mistake is assuming this value represents elapsed time in milliseconds; it is a raw hardware counter value that depends on the timer's configured clock frequency and prescaler. Ensure the timer has been initialized via ca_timer_init_counter before calling this function to avoid reading from an unconfigured peripheral.

Uso:

- This function is typically used for high-resolution time measurement, profiling, or calculating the delta between two events. It is often paired with ca_timer_init_counter to set up the hardware, and then called periodically to track elapsed cycles. In the caOS driver layer, it is used internally by functions like ca_timer_start to capture the initial timestamp of a timer slot before triggering the hardware start.

Esempio:

```
u32 start_time, end_time;

// Initialize timer 1 as a simple counter
ca_timer_init_counter(1);

// Capture the start value
start_time = ca_timer_hw_get_cnt(1);

// Perform a time-sensitive operation
// ...

// Capture the end value
end_time = ca_timer_hw_get_cnt(1);

// Calculate elapsed cycles
// Note: This does not handle counter wrap-around (overflow)
u32 elapsed = end_time - start_time;
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_timer_hw_get_cnt() in my embedded application?
A: You should use ca_timer_hw_get_cnt() when you need to read the current value of a hardware timer counter, for example to measure elapsed time, synchronize operations, or implement precise timing delays. It's particularly useful in performance-critical applications where direct hardware access is required without HAL overhead.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_timer_hw_get_cnt()?
A: u32 cnt1 = ca_timer_hw_get_cnt(1); u32 cnt2 = ca_timer_hw_get_cnt(2); if (cnt1 && cnt2) { /* Both timers are valid and initialized */ }

Parameters conf:0.95 **Q:** What does the 'num' parameter in ca_timer_hw_get_cnt() represent and what are valid values?
A: The 'num' parameter specifies which hardware timer to read. Valid values are from 1 to 17, corresponding to the available timers on the STM32 family. Using 0 or any number greater than 17 will cause the function to return 0, indicating an invalid timer number.

Return conf:0.90 **Q:** What do the return values of ca_timer_hw_get_cnt() mean and when does it return 0?
A: The function returns the current 32-bit counter value of the specified timer. It returns 0 in case of error, which occurs when the timer number is invalid (0 or > 17) or when the timer has not been initialized properly.

HW differences conf:0.85 **Q:** Are there any differences in how ca_timer_hw_get_cnt() is implemented across different STM32 families?
A: No, the implementation is consistent across all supported STM32 families (f1xx, f4xx, f7xx, h5xx, h7xx). Each implementation uses the same register access pattern: it retrieves

the timer base address via timer_hw_get() and reads the CNT register directly from the TIM_TypeDef structure.

---

ca_timer_hw_init_capture

Header: caos/drivers/inc/ca_glue_hw.h:313
GUARD: EN_CA_GLUE_HW

```
extern bool ca_timer_hw_init_capture (u8 num, u8 ch, u32 src, u32 psc);
   /* input capture on ch: src 0 pin / 1 LSE / 2 LSI, psc = ICxPSC;
   false if unsupported */
```

Configures a hardware timer channel to capture external signal events (like pulse width or frequency) and triggers an interrupt.

Parametri:

Nome	Tipo	Descrizione
num	u8	Timer instance number (e.g., 2, 3, 4). Must be a valid timer supported by the hardware; TIM1 and TIM8 are explicitly unsupported.
ch	u8	Timer channel number (1 to 4). Must be within the range supported by the specific timer instance.
src	u32	Input source: 0 for physical pin, 1 for LSE (Low Speed External), 2 for LSI (Low Speed Internal).
psc	u32	Prescaler for the capture event. The capture occurs every 2^psc edges. Valid values are typically 0-3.

Ritorna:
Returns true if the timer was successfully configured and the interrupt enabled. Returns false if the timer number is invalid, the channel is out of range, or the hardware routing fails.

Note:

- This function is not thread-safe and should be called during system initialization before the scheduler starts or within a critical section. It configures the hardware registers and enables the NVIC interrupt for the specific timer; it does not handle the ISR logic itself, which must be implemented elsewhere. A common beginner mistake is attempting to use TIM1 or TIM8, which are reserved for advanced control and have different interrupt vector requirements not covered by this driver. The function assumes the timer clock has already been enabled by the system clock configuration.

Uso:

- This function is used to set up hardware-level input capture for measuring external signals. It is typically called during the initialization phase of a driver or application task. It is often paired with a subsequent call to start the timer or enable the specific capture interrupt. Once initialized, the hardware will automatically capture the timer counter value into the CCR register on every rising edge (or every 2^psc edges), triggering an interrupt that the application must handle to read the captured value.

Esempio:

```
/* Initialize timer 2, channel 1 to capture from a physical pin with no
   prescaler */
if (ca_timer_hw_init_capture(2, 1, 0, 0)) {
```

```
    /* Successfully configured, now the hardware will trigger interrupts on
       capture */
} else {
    /* Handle error: invalid timer, channel, or routing failure */
    ca_printf("Failed to initialize capture timer\n");
}
```

Q&A — fine-tuning review *7/7 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_timer_hw_init_capture in my application?
A: You should call this function during the system initialization phase, before the scheduler starts, or within a critical section. It is used to configure hardware-level input capture for measuring external signals like pulse width or frequency.

Code example conf:1.00 **Q:** Provide a minimal example of how to initialize input capture on timer 3, channel 1 using a physical pin.
A: u8 timer_num = 3; u8 channel = 1; u32 source = 0; u32 prescaler = 0; if (ca_timer_hw_init_capture channel, source, prescaler)) { /* Success */ }

Parameters conf:1.00 **Q:** What do the 'src' and 'psc' parameters represent in ca_timer_hw_init_capture?
A: 'src' defines the input source: 0 for a physical pin, 1 for LSE, or 2 for LSI. 'psc' is the prescaler for the capture event, where the capture occurs every 2^psc edges; valid values are typically 0-3.

Return conf:1.00 **Q:** What does a return value of false indicate?
A: A return value of false indicates that the timer number is invalid, the requested channel is out of range for that timer, or the hardware routing failed.

Error/Edge case conf:1.00 **Q:** Are there any specific timers I should avoid using with this function?
A: Yes, TIM1 and TIM8 are explicitly unsupported. Attempting to use them is a common beginner mistake because they are reserved for advanced control and require different interrupt vector handling not covered by this driver.

Behavior conf:1.00 **Q:** Is ca_timer_hw_init_capture thread-safe or ISR-safe?
A: The function is not thread-safe. It must be called during system initialization or within a critical section. It configures hardware registers and enables the NVIC interrupt, but it does not handle the ISR logic itself, which must be implemented by the user.

HW differences conf:0.90 **Q:** Are there differences in how this function is implemented across the supported STM32 families?
A: The implementation is consistent across the f1xx, f4xx, f7xx, h5xx, h7xx, and u5xx families. All versions perform the same register configuration (CR1, DIER, PSC, ARR, CCER, CCMR1/2) and use the same logic to enable the NVIC interrupt, confirming that the hardware abstraction for timer capture is uniform across these architectures.

`ca_timer_hw_init_counter`

Header: `caos/drivers/inc/ca_glue_hw.h:269`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_timer_hw_init_counter (u8 num); /* configura il timer
    come contatore di eventi esterni */
```

Configures a hardware timer to act as an external event counter using the ETR (External Trigger) input.

Parametri:

Nome	Tipo	Descrizione
num	u8	The hardware timer index. Valid values depend on the MCU architecture; invalid or unsupported timers (like basic timers 6/7 or high-index timers on specific chips) will cause the function to return false.

Ritorna:
Returns true if the timer was successfully configured as an external counter, false if the timer index is invalid, unsupported, or if the hardware resource is unavailable.

Note:

- This function is non-blocking and performs direct register manipulation to set the timer into External Clock Mode 2. It is not thread-safe and should be called during the initialization phase of your application before the timer is started. It is not intended to be called from an ISR. A common beginner mistake is attempting to use basic timers (like TIM6 or TIM7) for external counting, which lack the necessary slave mode hardware; always check your specific MCU datasheet for ETR pin availability.

Uso:

- This function is used when you need to count external pulses (e.g., from a sensor) without consuming CPU cycles. It is typically used in a sequence: first, ensure the timer is available, then call this function to configure the hardware, and finally use ca_timer_counter_read to retrieve the pulse count. It is often paired with ca_timer_deinit when the counting task is finished to release the hardware resource.

Esempio:

```c
/* Example: Configure Timer 2 to count external pulses */
#include "caos.h"

void setup_external_counter(void) {
    u8 timer_num = 2U;

    /* 1. Ensure the timer is initialized/acquired */
    /* Note: 'timer_acquire' is a placeholder for your system's resource management */
    if (timer_acquire(timer_num)) {
        /* 2. Configure the hardware timer as an external event counter */
        if (ca_timer_hw_init_counter(timer_num)) {
            /* 3. Start the timer to begin counting */
            ca_timer_start(timer_num);
        } else {
            /* Handle configuration failure */
            ca_timer_deinit(timer_num);
        }
    }
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_timer_hw_init_counter() in my embedded project?
A: You should use ca_timer_hw_init_counter() when you need to configure an STM32 timer as an external event counter, such as for measuring encoder pulses or button events. It initializes the timer in external clock mode 2 (ETR pin), which allows it to count external events rather than internal clock ticks.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_timer_hw_init_counter() with caOS types?

A: u8 timer_num = 2; bool success = ca_timer_hw_init_counter(timer_num); if (success) { // Timer initialized successfully as external counter } else { // Handle initialization failure }

Parameters conf:0.95 **Q:** What does the 'num' parameter represent and what are valid values?
A: The 'num' parameter specifies the timer number to initialize as an external counter. Valid values are from 0 to 13 for STM32F4. Values less than 0 or greater than 13 will cause the function to return false.

Return conf:0.90 **Q:** What do the return values of ca_timer_hw_init_counter() indicate?
A: The function returns true if the timer was successfully initialized as an external counter. It returns false if the timer number is invalid, the timer pointer is NULL, or the timer is not supported (e.g., TIM6/7 or TIM15+).

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_timer_hw_init_counter()?
A: Common mistakes include calling this function from an ISR context, not acquiring the timer beforehand (which can cause hardware conflicts), and using unsupported timers like TIM6/7 or TIM15+. Also, calling it in a blocking RTOS context may cause system issues.

Behavior conf:0.90 **Q:** Is ca_timer_hw_init_counter() safe to use in ISR or thread contexts?
A: No, ca_timer_hw_init_counter() is not safe for ISR use and is not thread-safe. It is a blocking function that should not be called from high-priority contexts or RTOS tasks where blocking is undesirable. It must be protected with mutexes or semaphores if used in RTOS environments.

HW differences conf:0.80 **Q:** How does ca_timer_hw_init_counter() differ between STM32F1xx and STM32H7xx families?
A: The main difference is in supported timers. STM32F1xx supports all timers from 0 to 13, while STM32H7xx excludes timers 6, 7, and 15-17 due to hardware limitations. Additionally, F1xx uses a simpler register configuration compared to H7xx, which has more complex timer peripherals.

`ca_timer_hw_init_counter_irq`

Header: `caos/drivers/inc/ca_glue_hw.h:271`
GUARD: `EN_CA_GLUE_HW`

```
1 extern bool ca_timer_hw_init_counter_irq(u8 num, u32 preset); /*
    configura il timer come contatore con interrupt al raggiungimento del
     preset */
```

Configures a hardware timer to act as an external event counter that triggers an interrupt when a preset value is reached.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The hardware timer index (e.g., 1, 2, 3). Note that some chips restrict specific timers (e.g., basic timers 6/7 on F4/H5/H7/U5 cannot be used).
`preset`	`u32`	The target count value. The timer will count down from this value to zero, triggering an interrupt upon reaching the threshold.

Ritorna:
Returns true if the timer was successfully configured and the interrupt enabled. Returns false if the timer index is invalid, unsupported, or if the hardware resource is unavailable.

Note:

- This function is non-blocking and configures the hardware registers directly. It is not thread-safe; ensure no other task is reconfiguring the same timer simultaneously. It is generally not intended to be called from an ISR, as it performs NVIC configuration which should be done during system initialization. A common beginner mistake is assuming the timer starts counting immediately; this function only performs the setup, and you must ensure the hardware clock source is active and the timer is enabled via other control functions.

Uso:

- This function is used when you need to count external pulses (e.g., from a rotary encoder or a sensor) and perform an action via an interrupt service routine once a specific number of events have occurred. It is typically paired with ca_timer_deinit to release the hardware resource when the task is finished. In the caOS driver architecture, it is often called during the initialization phase of a driver that requires precise event counting.

Esempio:

```c
/* Example: Configure Timer 2 to count external pulses and trigger an interrupt
    at 1000 events */
#include "caos.h"

void setup_pulse_counter(void) {
    u8 timer_idx = 2U;
    u32 threshold = 1000U;

    /* Initialize the timer as an external counter with interrupt */
    if (ca_timer_hw_init_counter_irq(timer_idx, threshold)) {
        /* Timer is ready. The hardware will now trigger an interrupt
           when the external signal reaches the preset count. */
    } else {
        /* Handle error: timer index invalid or hardware busy */
    }
}

/* To stop the counter later, use the deinit function */
void stop_pulse_counter(void) {
    /* ca_timer_deinit(2U); // Release the timer resource */
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_timer_hw_init_counter_irq() in my embedded application?
A: You should use ca_timer_hw_init_counter_irq() when you need to configure an STM32 timer as a counter that generates an interrupt upon reaching a specific preset value. This is useful for time measurements, synchronization tasks, or triggering events at precise intervals.

Code example conf:0.95 **Q:** Can you show a minimal C example using ca_timer_hw_init_counter_irq() with caOS types?
A:

```c
u8 timer_num = 2;
u32 preset_value = 1000000;
bool success = ca_timer_hw_init_counter_irq(timer_num, preset_value);
if (!success) {
    /* handle error */
}
```

Parameters conf:0.95 **Q:** What does the 'num' parameter represent and what are valid values?

A: The 'num' parameter specifies the timer number to initialize. Valid values are integers from 1 up to the maximum number of timers available on the chip. Values 0 or greater than the maximum timer count will cause the function to return false. Additionally, timers 6 and 7 are basic timers and not supported, and timers >= 15 are not supported on H5xx and H7xx families.

Return conf:0.95 **Q:** What do the return values of ca_timer_hw_init_counter_irq() indicate?
A: The function returns true if the timer was successfully configured and is ready for use. It returns false if the timer number is invalid, the timer is not available (e.g., pointer is NULL), or if the timer is not supported (e.g., basic timers 6 and 7, or timers >= 15 on H5xx/H7xx).

Error/Edge case conf:0.90 **Q:** What are common mistakes or edge cases when calling ca_timer_hw_init_count
A: Common mistakes include using a timer that is already in use, passing a preset value that is too large and may cause overflow, or failing to enable the interrupt in the NVIC if you do not handle the interrupt. Also, calling this function from an ISR is not allowed and can cause issues.

Behavior conf:0.95 **Q:** Is ca_timer_hw_init_counter_irq() blocking or non-blocking, and is it safe to call from an ISR or multiple threads?
A: The function is non-blocking and returns immediately after configuration. It is not thread-safe, so concurrent calls from multiple threads without protection may lead to unpredictable behavior. It is also not safe to call from an ISR, as it may cause context or deadlock issues.

HW differences conf:0.85 **Q:** How does ca_timer_hw_init_counter_irq() differ between F1xx and H7xx families?
A: Both F1xx and H7xx families support the same basic timer configuration. However, H7xx and H5xx families have additional restrictions: timers 6 and 7 are not supported, and timers 15 and above are not supported on H5xx and H7xx chips. The register-level configuration remains consistent across these families.

`ca_timer_hw_init_encoder`

Header: `caos/drivers/inc/ca_glue_hw.h:270`
GUARD: `EN_CA_GLUE_HW`

```
1 extern bool ca_timer_hw_init_encoder (u8 num); /* configura il timer in
     modalit  encoder quadratura */
```

Configures a hardware timer in quadrature encoder mode to track rotary position.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The timer index (e.g., 1, 2, 3, 4, 5, 8). Values 6, 7, and others outside the supported range for the specific MCU architecture will return false.

Ritorna:
Returns true if the timer was successfully configured as an encoder, false if the timer index is invalid or the hardware resource is unavailable.

Note:

- This function is a low-level hardware configuration routine and is not thread-safe. It must be called before any attempt to read the encoder value. It is not intended to be called from an ISR, as it performs direct register manipulation that assumes exclusive access to the timer hardware. A common mistake is attempting to use a basic timer (like TIM6 or

TIM7 on STM32) which lacks encoder interface capabilities; always verify the timer index supports quadrature mode in the datasheet.

Uso:

- This function is typically used during the initialization phase of a motor control or user interface module. It is usually paired with a higher-level initialization function that manages the timer's state structure. The sequence is: 1) Call the driver-specific init, 2) If successful, proceed to use ca_timer_encoder_read to retrieve the current position, and 3) Call ca_timer_deinit when the resource is no longer needed.

Esempio:

```c
/* Example: Initializing a timer as an encoder for a rotary knob */
#include "caos.h"

void setup_encoder(u8 timer_id) {
    /* Configure the hardware timer for quadrature mode */
    if (ca_timer_hw_init_encoder(timer_id)) {
        /* Successfully initialized, now we can read the position */
        u32 position = ca_timer_encoder_read(timer_id);
        ca_printf("Encoder initialized. Current position: %u\n", position);
    } else {
        /* Handle error: timer index invalid or hardware busy */
        ca_printf("Failed to initialize encoder on timer %u\n", timer_id);
    }
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_timer_hw_init_encoder() in my embedded project?
A: You should call ca_timer_hw_init_encoder() when you need to configure a timer in STM32 to read position data from an encoder or rotary potentiometer. It initializes the timer in quadrature encoder mode, which is essential for capturing rotation direction and count.

Code example conf:0.95 **Q:** Can you show a minimal C example using ca_timer_hw_init_encoder()?
A:

```c
u8 timer_num = 2;
bool success = ca_timer_hw_init_encoder(timer_num);
if (success) {
    // Timer configured successfully
} else {
    // Handle error
}
```

Parameters conf:0.95 **Q:** What does the 'num' parameter represent and what are valid values?
A: The 'num' parameter specifies the timer number to initialize in encoder mode. Valid values are from 1 to the maximum number of timers available on the specific STM32 chip. For example, on F1xx, valid values are 1-2; on F4xx/F7xx/H7xx, valid values are 1-8, excluding 6 and 7. Values outside this range or invalid timer numbers will cause the function to return false.

Return conf:0.95 **Q:** What do the return values of ca_timer_hw_init_encoder() indicate?
A: The function returns true if the timer was successfully configured in encoder mode. It returns false if the timer number is invalid, the timer is not available (not acquired), or the hardware configuration fails due to unsupported timer or incorrect setup.

Error/Edge case conf:0.95 **Q:** What are common mistakes when calling ca_timer_hw_init_encoder()?
A: Common mistakes include calling the function without first acquiring the timer using timer_acquire(), passing an invalid timer number (e.g., 0 or greater than the maximum supported), or using a timer that does not support encoder mode (like timers 6 and 7 on F4xx/F7xx/H7xx). These will result in a false return value.

Behavior conf:0.95 **Q:** Is ca_timer_hw_init_encoder() blocking or thread-safe?
A: The function is non-blocking and returns immediately after hardware configuration. It is not thread-safe, so access should be protected with mutexes if used in a multi-threaded environment. It is also not recommended to call it from an ISR due to potential hardware register conflicts.

HW differences conf:0.90 **Q:** How does ca_timer_hw_init_encoder() differ between STM32F1xx and F4xx families?
A: On F1xx, all timers from 1 to 2 can be used for encoder mode. On F4xx, F7xx, and H7xx, timers 1-8 are supported for encoder mode, but timers 6 and 7 are excluded. Additionally, F4xx/F7xx/H7xx implementations check for valid timer numbers and return false for unsupported timers, whereas F1xx does not perform this validation.

`ca_timer_hw_init_oneshot`

Header: `caos/drivers/inc/ca_glue_hw.h:268`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_timer_hw_init_oneshot (u8 num, u32 delay_us); /*
    configura il timer in modalit  one-shot con ritardo in microsecondi
    */
```

Configures a hardware timer to trigger a single interrupt after a specified microsecond delay.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The hardware timer index (e.g., 1, 2, 3). Must be a valid timer supported by the specific STM32 hardware.
`delay_us`	`u32`	The desired delay in microseconds. The value must be within the range supported by the timer's prescaler and auto-reload register limits.

Ritorna:
Returns true if the timer was successfully configured and the interrupt enabled. Returns false if the timer index is invalid or the hardware resource is unavailable.

Note:

- This function is non-blocking and configures the timer in One-Pulse Mode (OPM), meaning it will count once and stop automatically. It is not thread-safe; ensure that timer initialization is performed during system setup or within a protected critical section if called dynamically. It is not intended to be called from an ISR. A common beginner mistake is failing to ensure the timer hardware is actually available or already acquired by another driver, which will cause this function to return false. Always verify the return value before assuming the timer is ready for use.

Uso:

- This function is typically used to set up precise, hardware-level one-shot delays for time-sensitive tasks like triggering a sensor read or a pulse-width signal. It is usually paired with a timer dispatch mechanism or an interrupt service routine that handles the event once the timer expires. The typical sequence is: 1) Acquire the timer resource, 2) Call this function to configure the delay, 3) Start the timer, and 4) Deinitialize or release the timer once the task is complete to free the hardware for other uses.

Esempio:

```c
/* Example: Configure Timer 2 for a 500 microsecond one-shot delay */
#include "caos.h"

void setup_oneshot_timer(void) {
    u8 timer_id = 2;
    u32 delay = 500;

    /* Attempt to configure the timer for a one-shot event */
    if (ca_timer_hw_init_oneshot(timer_id, delay)) {
        /* Timer is configured; now start the timer hardware */
        /* Note: ca_timer_start(timer_id) would be called here if available */
    } else {
        /* Handle error: timer index invalid or hardware busy */
        ca_printf("Failed to initialize timer %d\n", timer_id);
    }
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_timer_hw_init_oneshot() in my embedded application?

A: You should use ca_timer_hw_init_oneshot() when you need precise one-shot timing delays in microsecond resolution. It's ideal for implementing timeouts, debouncing, or any scenario where a single, accurate delay is required without continuous polling.

Code example conf:0.95 **Q:** Can you show a minimal example of how to use ca_timer_hw_init_oneshot()?

A:

```c
u8 timer_num = 2;
u32 delay = 500000; // 500 ms
if (ca_timer_hw_init_oneshot(timer_num, delay)) {
    // Timer initialized successfully
} else {
    // Initialization failed
}
```

Parameters conf:0.95 **Q:** What does the 'num' parameter represent and what are valid values?

A: The 'num' parameter specifies the hardware timer number to initialize (0-15). Values outside this range will cause the function to return false. Each number maps to a specific timer peripheral on the STM32 device.

Return conf:0.95 **Q:** What do the return values of ca_timer_hw_init_oneshot() indicate?

A: It returns true if the timer was successfully initialized with the specified delay. It returns false if initialization failed, typically due to an invalid timer number (>15), a timer already in use, or unavailable hardware resources.

Error/Edge case conf:0.90 **Q:** What are common mistakes when calling ca_timer_hw_init_oneshot()?

A: Common mistakes include using a timer number outside the valid range (0-15), calling the function on a timer that is already in use, or not ensuring the timer is acquired beforehand. Also, passing a delay_us that is too large may cause overflow or timeout issues.

Behavior conf:0.95 **Q:** Is ca_timer_hw_init_oneshot() thread-safe and can it be called from an ISR?

A: No, ca_timer_hw_init_oneshot() is not thread-safe and should not be called from an ISR. It modifies hardware registers and may cause race conditions or unexpected behavior if multiple threads or ISRs attempt to access the same timer simultaneously.

HW differences conf:0.85 **Q:** Are there any differences in how ca_timer_hw_init_oneshot() is implemented across different STM32 families?

A: The implementation is consistent across all supported families (f1xx, f4xx, f7xx, h5xx, h7xx). All use the same register configuration: setting CR1 to OPM, configuring PSC and ARR, enabling the timer interrupt via DIER, and setting NVIC priority and enable. The only differences are in the underlying timer hardware peripherals and clock sources.

ca_timer_hw_init_periodic

Header: caos/drivers/inc/ca_glue_hw.h:267
GUARD: EN_CA_GLUE_HW

```
extern bool ca_timer_hw_init_periodic (u8 num, u32 period_us); /*
    configura il timer in modalit  periodica con periodo in microsecondi
     */
```

Configures a hardware timer to trigger an interrupt at a fixed, repeating interval.

Parametri:

Nome	Tipo	Descrizione
num	u8	The hardware timer index (e.g., 1, 2, 3). Must correspond to a valid timer peripheral on the specific STM32 chip.
period_us	u32	The desired interval in microseconds. Must be greater than zero; values too small for the timer's clock frequency may cause configuration errors.

Ritorna:
Returns true if the timer was successfully configured and the interrupt enabled; returns false if the timer index is invalid or the hardware could not be initialized.

Note:

- This function performs low-level hardware configuration, including setting the prescaler (PSC) and auto-reload register (ARR) to match the requested period. It is not thread-safe and should be called during system initialization or from a single configuration task. It is not intended to be called from an ISR. A common mistake is passing an invalid timer index or a period that exceeds the timer's maximum count capacity, which will result in incorrect timing or failure to trigger interrupts.

Uso:

- This function is typically used to set up the base hardware for periodic tasks or high-frequency sampling. It is usually paired with a corresponding interrupt service routine (ISR) that handles the timer event. The standard sequence is: first, ensure the timer is acquired (if required by the system), then call this function to configure the hardware, and finally, use other caOS functions to start the timer or manage the associated callback.

Esempio:

```
/* Configure Timer 2 to trigger an interrupt every 1000 microseconds (1ms) */
#define TIMER_NUM 2
#define PERIOD_US 1000

if (ca_timer_hw_init_periodic(TIMER_NUM, PERIOD_US)) {
    /* Timer hardware is ready. Now we would typically enable the timer
    peripheral */
    /* or register a callback to be executed when the interrupt fires. */
} else {
    /* Handle error: invalid timer index or hardware failure */
    ca_printf("Failed to initialize timer %d\n", TIMER_NUM);
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_timer_hw_init_periodic in my embedded application?
A: You should call ca_timer_hw_init_periodic during system initialization or when setting up a periodic timer for timekeeping, scheduling, or interrupt-based tasks. It's typically used to configure a hardware timer in periodic mode to generate interrupts at regular intervals.

Code example conf:0.95 **Q:** Can you show a minimal example of how to use ca_timer_hw_init_periodic?
A: u8 timer_num = 2; u32 period = 1000000; /* 1 second */ bool success = ca_timer_hw_init_periodic
period); if (!success) { /* Handle error */ }

Parameters conf:0.95 **Q:** What do the parameters 'num' and 'period_us' represent in ca_timer_hw_init_perioc
A: The parameter 'num' specifies the hardware timer number to initialize (0-7). The parameter 'period_us' specifies the desired period in microseconds. Valid timer numbers are 0 through 7; values outside this range cause the function to return false. A period of 0 is invalid and may lead to undefined behavior.

Return conf:0.95 **Q:** What does it mean when ca_timer_hw_init_periodic returns false?
A: A return value of false indicates that the timer initialization failed. This can happen if the timer number is invalid (greater than 7), the timer is not available, the clock for the timer is not enabled, or there are insufficient resources to configure the timer.

Error/Edge case conf:0.95 **Q:** What are common mistakes when using ca_timer_hw_init_periodic?
A: Common mistakes include passing an invalid timer number (e.g., >7), passing a period of 0, not checking the return value of the function, and calling this function from an ISR. Not verifying the return value can lead to unexpected behavior if the timer fails to initialize.

Behavior conf:0.95 **Q:** Is ca_timer_hw_init_periodic blocking or thread-safe?
A: The function is not blocking, but it may take some time to configure the timer registers. It is not thread-safe, so if used in a multi-threaded environment, it must be protected with synchronization mechanisms like mutexes. It should not be called from an ISR.

HW differences conf:0.85 **Q:** Are there any differences in how ca_timer_hw_init_periodic is implemented across different STM32 families?
A: The implementation is consistent across all supported families (f1xx, f4xx, f7xx, h5xx, h7xx). All use the same register configuration approach: setting CR1, PSC, ARR, CNT, EGR, SR, and DIER registers, and enabling the timer interrupt via NVIC. The only differences are in the specific timer hardware peripherals and clock sources, which are abstracted by helper functions.

`ca_timer_hw_init_pwm`

Header: `caos/drivers/inc/ca_glue_hw.h:272`
GUARD: `EN_CA_GLUE_HW`

```
1 extern bool ca_timer_hw_init_pwm (u8 num, u8 ch, u32 freq_hz); /*
    configura il canale ch (1-4) in PWM mode 1 con preload; false se il
    canale non esiste */
```

Configures a specific hardware timer channel for PWM output mode.

Parametri:

Nome	Tipo	Descrizione
num	u8	The hardware timer index (typically 1-17 depending on the STM32 series).
ch	u8	The specific channel number (1-4) to configure for PWM.
freq_hz	u32	The desired PWM frequency in Hertz. Must be high enough to allow a valid prescaler/ARR calculation.

Ritorna:
Returns true if the timer and channel were successfully configured, false if the timer/channel is invalid or the requested frequency cannot be achieved with the current clock settings.

Note:

- This function is non-blocking and performs direct register manipulation on the hardware timer peripheral. It is not thread-safe; you must ensure that no other task or interrupt is modifying the same timer's registers simultaneously. It is generally not recommended to call this from an ISR due to the complexity of the calculations involved. Before calling this, the timer must be acquired (e.g., via timer_acquire) and the peripheral clock must be enabled. A common mistake is attempting to set a frequency that is too low for the system clock, resulting in a division error or an invalid ARR value.

Uso:

- This function is used to set up hardware-level PWM generation. It is typically called after initializing the timer peripheral and before starting the timer. In the caOS driver architecture, it is often paired with timer_acquire to reserve the hardware resource, followed by ca_timer_set_pwm to actually control the duty cycle of the configured channel.

Esempio:

```
/* Example: Initialize Timer 2, Channel 1 for 1kHz PWM */
#define TIMER_NUM 2U
#define CHANNEL 1U
#define FREQ 1000U

if (timer_acquire(TIMER_NUM)) {
    /* Initialize the timer clock before configuring PWM */
    if (ca_clock_hw_timer_init(TIMER_NUM)) {
        if (ca_timer_hw_init_pwm(TIMER_NUM, CHANNEL, FREQ)) {
            /* PWM configured successfully, now set duty cycle */
            ca_timer_set_pwm(TIMER_NUM, CHANNEL, 500U); /* 50% duty cycle */
            ca_timer_start(TIMER_NUM);
        } else {
            /* Handle configuration failure */
            timer_release(TIMER_NUM);
        }
    } else {
        timer_release(TIMER_NUM);
    }
}
```

Q&A — fine-tuning review *7/7 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_timer_hw_init_pwm in my application?
A: You should call this function after you have successfully acquired the timer using timer_acquire and ensured the peripheral clock is enabled. It is used to configure the hardware timer for PWM output mode before you start the timer and begin controlling the duty cycle.

Code example conf:1.00 **Q:** Provide a minimal example of how to initialize a PWM channel using caOS types.
A: u8 timer_num = 1U; u8 channel = 1U; u32 frequency = 20000U; if (ca_timer_hw_init_pwm(timer_ channel, frequency)) { /* PWM initialized successfully */ }

Parameters conf:1.00 **Q:** What are the constraints on the freq_hz parameter?
A: The freq_hz parameter must be high enough to allow a valid prescaler (PSC) and auto-reload register (ARR) calculation. If the requested frequency is too low relative to the system clock, the calculation will fail, resulting in a division error or an invalid ARR value.

Return conf:1.00 **Q:** What does a return value of false indicate?
A: A return value of false indicates that the configuration failed. This happens if the specified timer index or channel is invalid, or if the requested frequency cannot be achieved given the current system clock settings.

Error/Edge case conf:1.00 **Q:** What is a common mistake when configuring PWM frequency?
A: A common mistake is attempting to set a frequency that is too low for the system clock, which leads to a division error or an invalid ARR value. Additionally, failing to acquire the timer or enable the peripheral clock before calling the function will result in failure.

Behavior conf:1.00 **Q:** Is ca_timer_hw_init_pwm thread-safe or safe to call from an ISR?
A: The function is non-blocking but is not thread-safe; you must ensure no other task or interrupt modifies the timer's registers simultaneously. It is generally not recommended to call this from an ISR due to the complexity of the internal calculations.

HW differences conf:1.00 **Q:** How does the implementation of ca_timer_hw_init_pwm differ across STM32 families regarding advanced timer features?
A: The implementation varies in how it handles the Main Output Enable (MOE) bit in the BDTR register. For f1xx, it only enables MOE for timer 1. For f4xx and f7xx, it enables it for timers 1 and 8. For h5xx, h7xx, and u5xx, it enables MOE for timers 1, 8, and any timer index 15 or greater.

`ca_timer_hw_set_pwm`

Header: `caos/drivers/inc/ca_glue_hw.h:279`
GUARD: `EN_CA_GLUE_HW`

```
1 extern bool ca_timer_hw_set_pwm (u8 num, u8 ch, u16 duty_pm); /* sets
    the channel duty cycle in permille 0-1000 (CCR preload, glitch-free)
    */
```

Configures the duty cycle of a hardware timer PWM channel in permille (0-1000).

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The hardware timer index (e.g., 1, 2, 8). Must correspond to a timer initialized for PWM.
`ch`	`u8`	The specific channel index (1-4) on the selected timer.
`duty_pm`	`u16`	Duty cycle in permille. 0 means 0% (always low), 1000 means 100% (always high).

Ritorna:
Returns true if the duty cycle was successfully updated, false if the timer or channel is invalid or not in PWM mode.

Note:

- This function is a low-level hardware driver call and is not thread-safe; it should be called only after the timer has been properly initialized via higher-level caOS timer functions. It is non-blocking and performs direct register manipulation, making it extremely fast, but it does not perform bounds checking on the timer index itself. A common beginner mistake is calling this function before the timer hardware has been enabled or configured for PWM mode, which will result in no signal output or potential system instability if the timer registers are accessed while the peripheral clock is disabled.

Uso:

- This function is typically used in the application layer to dynamically adjust motor speeds, LED brightness, or control signals. It is usually paired with ca_timer_init_pwm to set up the hardware, followed by repeated calls to this function to update the duty cycle during runtime. It should not be called directly if the caOS timer abstraction layer is already managing the timer, as that layer handles validation and state tracking.

Esempio:

```c
/* Example: Setting up a PWM channel and updating its duty cycle */
#include "caos.h"

void update_motor_speed(u8 timer_num, u8 channel, u16 speed_pm) {
    /* Ensure the duty cycle is within the valid 0-1000 range */
    if (speed_pm > 1000U) {
        speed_pm = 1000U;
    }

    /* Update the hardware PWM duty cycle directly */
    /* Note: This assumes the timer was already initialized via
    ca_timer_init_pwm */
    bool success = ca_timer_hw_set_pwm(timer_num, channel, speed_pm);

    if (!success) {
        /* Log error: Failed to set PWM, check if timer_num/channel are valid */
        ca_printf("Error: Failed to set PWM on timer %d, channel %d\n",
    timer_num, channel);
    }
}
```

Q&A — fine-tuning review

6/6 pairs · gemini-3.1-flash-lite

Usage conf:1.00 **Q:** When should I use the ca_timer_hw_set_pwm function in my application?
A: This function is used to dynamically adjust PWM duty cycles for tasks like motor speed control, LED brightness, or signal modulation. It should be called after the timer has been initialized with ca_timer_init_pwm and started with ca_timer_start.

Code example conf:1.00 **Q:** Provide a minimal C example to set a 50% duty cycle on timer 2, channel 1.
A: u8 timer_num = 2U; u8 channel = 1U; u16 duty = 500U; if (ca_timer_hw_set_pwm(timer_num, channel, duty)) { /* Success */ }

Parameters conf:1.00 **Q:** What is the valid range for the duty_pm parameter and how are values outside this range handled?
A: The duty_pm parameter represents the duty cycle in permille, with a valid range of 0 to 1000 (where 0 is 0% and 1000 is 100%). Any value provided that is greater than 1000 is treated as 100%.

Return conf:1.00 **Q:** What does a false return value from ca_timer_hw_set_pwm indicate?
A: A false return indicates that the update failed. This occurs if the timer index is invalid, the requested channel is out of the supported range for that specific timer, or the timer has not been initialized.

Error/Edge case conf:1.00 **Q:** Why might ca_timer_hw_set_pwm return false even if I am using a valid timer index?
A: A common mistake is calling this function before the timer has been initialized via ca_timer_init_pwm. If initialization has not occurred, the internal hardware handle will be NULL, causing the function to return false.

HW differences conf:1.00 **Q:** Are there significant differences in how ca_timer_hw_set_pwm is implemented across the supported STM32 families?
A: No, the implementation is consistent across the f1xx, f4xx, f7xx, h5xx, h7xx, and u5xx families. All versions use the same logic: they retrieve the TIM_TypeDef pointer, validate the channel against timer_hw_pwm_nch, and update the corresponding CCR register based on the ARR value.

`ca_timer_hw_start`

Header: `caos/drivers/inc/ca_glue_hw.h:274`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_timer_hw_start (u8 num); /* avvia il timer; false se il
   timer non    stato inizializzato */
```

Enables the hardware timer peripheral by setting the Counter Enable (CEN) bit.

Parametri:

Nome	Tipo	Descrizione
num	u8	The index of the hardware timer to start. Must correspond to a valid, previously initialized timer index.

Ritorna:
Returns true if the timer was successfully started, or false if the timer index is invalid or the peripheral was not initialized.

Note:

- This function is a low-level hardware abstraction and is non-blocking. It is thread-safe provided that the underlying hardware register access is atomic, but it is generally intended to be called from the main application thread or initialization routines. It is not designed to be called from an ISR unless the specific hardware architecture guarantees atomic register modification. The most common mistake is calling this function before the timer has been properly configured via ca_timer_init or equivalent, which will result in a false return value because the internal handle lookup will fail.

Uso:

- This function is used to trigger the actual counting process of a hardware timer after it has been configured. It is typically paired with ca_timer_init to set up the peripheral parameters and ca_timer_stop to halt the counting. In the caOS driver layer, it is often invoked by higher-level functions like ca_timer_start, which perform safety checks (like verifying if the timer is busy) before committing the hardware start command.

Esempio:

```
/* Example: Initialize and start a hardware timer */
#include "caos.h"

void start_my_timer(u8 timer_id) {
    /* First, ensure the timer is configured for periodic operation */
    if (ca_timer_init_periodic(timer_id, 1000)) {
```

```
        /* Attempt to start the hardware timer */
        if (!ca_timer_hw_start(timer_id)) {
            /* Handle error: timer could not be started */
            ca_printf("Failed to start timer %d\n", timer_id);
        } else {
            ca_printf("Timer %d started successfully\n", timer_id);
        }
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_timer_hw_start() in my embedded application?
A: You should call ca_timer_hw_start() after initializing a timer with ca_timer_hw_init() to actually start the timer hardware. This function is typically used when you want to begin counting or generate timer events, such as periodic interrupts or PWM signals.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_timer_hw_start() with caOS types?
A:

```
u8 timer_num = 2;
bool success = ca_timer_hw_start(timer_num);
if (!success) {
    // Handle error: timer not initialized or invalid
}
```

Parameters conf:0.90 **Q:** What does the 'num' parameter in ca_timer_hw_start() represent, and what are valid values?
A: The 'num' parameter represents the hardware timer number to start. Valid values are integers from 1 up to the maximum number of timers available on the specific STM32 family. Values of 0 or greater than the maximum number of timers will cause the function to return false.

Return conf:0.95 **Q:** What do the return values of ca_timer_hw_start() indicate, and what causes it to return false?
A: The function returns true if the timer was successfully started. It returns false if the timer number is invalid (0 or exceeds maximum) or if the timer was not initialized with ca_timer_hw_init() before calling this function.

HW differences conf:0.85 **Q:** Are there any differences in how ca_timer_hw_start() is implemented across different STM32 families?
A: No, the implementation of ca_timer_hw_start() is consistent across all STM32 families (f1xx, f4xx, f7xx, h5xx, h7xx). It uses the same register access pattern: retrieving the timer peripheral pointer and setting the TIM_CR1_CEN bit to enable the timer. All implementations follow the same logic and register structure.

`ca_timer_hw_stop`

Header: `caos/drivers/inc/ca_glue_hw.h:275`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_timer_hw_stop (u8 num); /* ferma il timer senza resettare
    il contatore */
```

Stops a hardware timer by clearing the counter enable bit without resetting the current count value.

Parametri:

Nome	Tipo	Descrizione
num	u8	The index of the hardware timer to stop. Must be a valid timer ID previously initialized by the system.

Ritorna:
Returns true if the timer was successfully stopped, or false if the timer index is invalid or the hardware peripheral could not be accessed.

Note:

- This function is non-blocking and performs a direct register write to the timer's Control Register (CR1). It is thread-safe in the sense that it performs an atomic bitwise operation, but it does not provide synchronization if multiple tasks attempt to control the same timer simultaneously. It is safe to call from an ISR because it does not involve OS primitives or blocking operations. The most common mistake is calling this function on a timer index that has not been initialized, which will result in a NULL pointer dereference inside the internal hardware lookup function.

Uso:

- This function is typically used to pause a timer operation temporarily without losing the current progress of the counter. It is often paired with ca_timer_start to resume the operation later, or called during a shutdown sequence via ca_timer_deinit to ensure the peripheral is silenced before releasing resources. It is also used internally by the driver layer to implement higher-level timer management functions that check the timer's status before attempting to stop it.

Esempio:

```c
/* Example: Stopping a timer safely before re-configuration */
#define MY_TIMER_ID 1

void stop_my_timer(void) {
    /* Check if the timer is currently active to avoid unnecessary register
    writes */
    if (ca_timer_is_busy(MY_TIMER_ID)) {
        /* Stop the timer hardware immediately */
        if (!ca_timer_hw_stop(MY_TIMER_ID)) {
            /* Handle error: timer index might be invalid or hardware failed */
            ca_printf("Error: Failed to stop timer %d\n", MY_TIMER_ID);
        } else {
            ca_printf("Timer %d stopped successfully.\n", MY_TIMER_ID);
        }
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_timer_hw_stop() instead of resetting the timer counter?
A: You should use ca_timer_hw_stop() when you want to pause a timer without losing track of the current count. This is useful for temporarily halting time-sensitive operations while preserving the elapsed time, such as pausing a measurement or delaying a task without starting over.

Code example conf:0.95 **Q:** Can you show a minimal example of how to use ca_timer_hw_stop() with caOS types?
A:

```c
u8 timer_num = 2;
bool success = ca_timer_hw_stop(timer_num);
```

```
if (!success) {
    // Handle error: invalid timer number or uninitialized timer
}
```

Parameters conf:0.95 **Q:** What does the 'num' parameter in ca_timer_hw_stop() represent and what are valid values?
A: The 'num' parameter represents the hardware timer number to stop, ranging from 1 to TIMER_MAX. Values outside this range will cause the function to return false. The timer must also be initialized using timer_hw_get() for the function to succeed.

Return conf:0.95 **Q:** What do the return values of ca_timer_hw_stop() indicate and what causes it to return false?
A: The function returns true if the timer was successfully stopped. It returns false if the timer number is invalid (outside 1 to TIMER_MAX) or if the timer was not initialized (i.e., the pointer returned by timer_hw_get() is NULL).

HW differences conf:0.90 **Q:** Are there any differences in how ca_timer_hw_stop() is implemented across different STM32 families?
A: No, the implementation is consistent across all supported families (f1xx, f4xx, f7xx, h5xx, h7xx). All use the same register manipulation: clearing the CEN bit in the CR1 register of the corresponding TIM_TypeDef peripheral.

`ca_timer_init`

Header: `caos/drivers/inc/timer.h:75`
Source: `caos/drivers/src/timer.c:377:383`
GUARD: `EN_TIMER`

```
void ca_timer_init(void)
```

Initializes the timer subsystem and registers the timer CLI command if enabled.

Ritorna:
void

Note:

- This function is non-blocking and is intended to be called during system startup. It is not thread-safe in the sense that it uses a static flag to ensure the CLI registration happens only once; calling it from multiple threads simultaneously is technically safe due to the static flag, but it should only be invoked during the initialization phase. It is not intended to be called from an ISR. The most common mistake is forgetting to call this before attempting to use timer-related CLI commands, which will result in the 'timer' command being missing from the CLI interface.

Uso:

- This function is typically called once inside the ca_os_main function before the scheduler is started via ca_os_run. It serves as a mandatory setup step for the timer module. By registering the CLI node, it allows developers to interact with or debug timer states via the command line interface during runtime. It is usually paired with other initialization functions like ca_os_tick_init or hardware-specific peripheral initializations.

Esempio:

```
void ca_os_main(void) {
    /* Initialize the timer subsystem for CLI support */
    ca_timer_init();

    /* Initialize system tick for RTOS timing */
```

```
    ca_os_tick_init();

    /* Create a monitoring task */
    ca_os_job("monitor", CA_TH_LEVEL3, task_monitor, 0, STACK_SMALL, CA_JOB_STD,
     CA_FAULT_RESTART_THREAD);

    /* Start the OS scheduler */
    ca_os_run(true);
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should ca_timer_init() be called in a caOS application, and why is it important?
A: ca_timer_init() must be called exactly once at the start of the system, before any other RTOS functions. It initializes the system timer used for timekeeping and scheduling. Calling it after ca_os_run() or multiple times will lead to undefined behavior. It is essential because it sets up the core timing mechanism that all other RTOS features depend on.

Code example conf:0.90 **Q:** Can you provide a minimal C example using caOS types that demonstrates proper usage of ca_timer_init()?
A: u32 main(void) { ca_timer_init(); ca_os_run(); return 0; }

Parameters conf:0.85 **Q:** What are the parameters of ca_timer_init() and what happens if invalid values are passed?
A: ca_timer_init() takes no parameters. Since there are no parameters, there are no invalid values to consider. The function is designed to be called with no arguments and performs initialization of the system timer hardware.

`ca_timer_init_capture`

Header: `caos/drivers/inc/timer.h:75`
Source: `caos/drivers/src/timer.c:168:191`
GUARD: `EN_TIMER`

```
bool ca_timer_init_capture (u8 num, u8 ch, ca_timer_ic_src_t src, u8 div
   , ca_timer_ic_cb_t cb); /* div 1/2/4/8 */
```

Initializes a hardware timer channel in input capture mode to measure external signal timing or frequency.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	Timer peripheral index (e.g., 1, 2, 3).
`ch`	`u8`	Timer channel number (1 to 4).
`src`	`ca_timer_ic_src_t`	Input capture source selection (e.g., internal clock, LSE, or external pin).
`div`	`u8`	Prescaler for capture events: 1 (every edge), 2, 4, or 8 (every 8th edge).
`cb`	`ca_timer_ic_cb_t`	Callback function pointer executed when a capture event occurs.

Ritorna:
Returns true if the timer was successfully acquired and configured, false otherwise (e.g., invalid channel, invalid divisor, or hardware resource already in use).

Note:

- This function is non-blocking and performs hardware-level configuration of the STM32 timer registers. It is not thread-safe; ensure that no other task attempts to initialize or use the same timer peripheral simultaneously. It must not be called from an ISR. A common mistake is passing a NULL callback or an invalid divisor value, which will cause the function to return false immediately. Before calling this, ensure the system clock is configured and the timer hardware is available.

Uso:

- Use this function when you need to measure the period, frequency, or pulse width of an external signal. The typical sequence is: 1) Call ca_timer_init_capture to set up the hardware and register the callback. 2) Call ca_timer_start to enable the timer counter. 3) The callback will be triggered automatically by the hardware upon signal edges. 4) Call ca_timer_stop when measurements are complete to save power and release the hardware.

Esempio:

```c
/* Define the callback function to handle capture events */
void my_capture_callback(u8 num, u8 ch, u32 value) {
    /* Process the captured timer value here */
}

/* Initialize timer 2, channel 1 to capture on every edge */
if (ca_timer_init_capture(2, 1, CA_TIMER_IC_LSE, 1, my_capture_callback)) {
    /* Start the timer to begin capturing */
    (void)ca_timer_start(2);
} else {
    /* Handle initialization failure */
    ca_printf("Timer init failed!\n");
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_timer_init_capture function in my caOS project?
A: You should use this function when you need to measure the period, frequency, or pulse width of an external signal. The typical workflow involves initializing the capture, starting the timer, handling the signal via the callback, and stopping the timer when finished.

Code example conf:0.90 **Q:** Provide a minimal example of how to initialize a timer channel for input capture.
A: void my_cb(u32 val) { /* Handle capture */ } ... bool success = ca_timer_init_capture(2, 1, CA_TIMER_IC_SRC_PIN, 1, my_cb);

Parameters conf:1.00 **Q:** What are the valid values for the 'div' parameter in ca_timer_init_capture?
A: The 'div' parameter represents the prescaler for capture events and must be one of the following values: 1 (every edge), 2, 4, or 8 (every 8th edge).

Return conf:1.00 **Q:** What does a return value of false indicate when calling ca_timer_init_capture?
A: A return value of false indicates that the timer was not successfully configured. This can be caused by an invalid channel, an invalid divisor value, or because the hardware resource is already in use.

Error/Edge case conf:1.00 **Q:** What are some common mistakes that cause ca_timer_init_capture to fail?
A: Common mistakes include passing a NULL callback function or providing an invalid divisor value. Both of these will cause the function to return false immediately.

Behavior conf:1.00 **Q:** Is ca_timer_init_capture thread-safe or safe to call from an ISR?
A: No, the function is not thread-safe, and it must not be called from an ISR. You must ensure no other task attempts to initialize or use the same timer peripheral simultaneously.

ca_timer_init_counter

Header: caos/drivers/inc/timer.h:53
Source: caos/drivers/src/timer.c:90:104
GUARD: EN_TIMER

```
bool ca_timer_init_counter (u8 num, u64 *count)
```

Initializes a hardware timer in counter mode to track events and store the count in a provided variable.

Parametri:

Nome	Tipo	Descrizione
num	u8	The hardware timer index to initialize. Must be a valid timer ID supported by the hardware.
count	u64 *	Pointer to a 64-bit variable where the timer will store the current count. Must not be NULL.

Ritorna:
Returns true if the timer was successfully acquired and configured, false otherwise (e.g., if count is NULL, timer is already in use, or hardware initialization fails).

Note:

- This function is not thread-safe and should not be called from an ISR. It performs hardware resource acquisition, so you must ensure the timer is not already in use by another part of the system. A common mistake is passing a NULL pointer for the count variable or failing to check the return value, which leads to silent failures where the timer never starts. Always ensure the timer is released using ca_timer_deinit when it is no longer needed to prevent resource exhaustion.

Uso:

- Use this function when you need to count external pulses or events using a hardware timer. It is typically used in the initialization phase of a driver or application module. The standard sequence is: call ca_timer_init_counter to set up the hardware, then use ca_timer_start to begin counting, and finally use ca_timer_counter_read to retrieve the current value. Always pair this with ca_timer_deinit during system shutdown or module cleanup.

Esempio:

```
u64 event_count = 0;
// Initialize timer 1 in counter mode
if (ca_timer_init_counter(1, &event_count)) {
    // Start the timer to begin counting events
    ca_timer_start(1);

    // Later, read the current value
    u64 current = ca_timer_counter_read(1);

    // Clean up when finished
    ca_timer_stop(1);
    ca_timer_deinit(1);
} else {
    // Handle initialization error
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_timer_init_counter and what is its purpose in a real-time embedded system?
A: You should call ca_timer_init_counter when you need to initialize a hardware timer as a counter to measure elapsed time. It is typically used in real-time systems where precise timing is required, such as for task scheduling, delay functions, or measuring execution time of code sections. The function prepares a timer to be used for counting, and the result is stored in a provided 64-bit variable.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_timer_init_counter with caOS types?
A: u64 counter_value; bool success = ca_timer_init_counter(1, &counter_value); if (success) { // Timer 1 is now initialized as a counter // counter_value holds the elapsed time }

Parameters conf:0.85 **Q:** What does the 'num' parameter represent, and what are valid values for it?
A: The 'num' parameter specifies the timer number to initialize, and it must be a value between 1 and the maximum number of available timers. Values outside this range will cause the function to return false. The function does not validate the timer number against hardware capabilities beyond this range check.

Return conf:0.90 **Q:** What does it mean when ca_timer_init_counter returns false, and what causes this failure?
A: When ca_timer_init_counter returns false, it indicates that the initialization failed. This can happen due to several reasons: the 'count' pointer is NULL, the hardware timer acquisition fails, or the hardware initialization itself fails. The function is blocking, so it may also wait for a timer to become available if it's already in use.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_timer_init_counter?
A: Common mistakes include passing a NULL pointer for the 'count' parameter, using an invalid timer number (outside the 1 to max timer range), or calling the function without ensuring that the timer clock has been initialized. Additionally, calling this function from an ISR is not safe, as it modifies global data structures and is not thread-safe.

`ca_timer_init_counter_irq`

Header: `caos/drivers/inc/timer.h:55`
Source: `caos/drivers/src/timer.c:122:135`
GUARD: `EN_TIMER`

```
bool ca_timer_init_counter_irq (u8 num, u32 preset, ca_timer_cb_t cb);
   /* down, ECE, UIE ogni preset impulsi */
```

Configures a hardware timer to trigger a callback function after a specific number of external pulses.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The hardware timer index (e.g., 1, 2, 3). Must be a valid timer supported by the hardware.
`preset`	`u32`	The number of pulses to count before triggering the callback. Must be greater than 0.
`cb`	`ca_timer_cb_t`	The function pointer to be executed when the counter reaches the preset value.

Ritorna:

Returns true if the timer was successfully acquired and configured. Returns false if the preset is 0, the timer is already in use, or hardware initialization fails.

Note:

- This function is non-blocking and configures the hardware timer to operate in an interrupt-driven mode. It is not thread-safe; ensure that timer initialization is performed during the system setup phase before starting the scheduler or other tasks. The callback function (cb) will be executed within an Interrupt Service Routine (ISR) context, so it must be extremely fast, avoid blocking calls, and not use functions that require waiting (like mutexes or delays). A common mistake is passing a NULL callback or a very small preset value that causes the interrupt to trigger too frequently, potentially starving the CPU.

Uso:

- This function is typically used for counting external events, such as pulses from a rotary encoder or a flow sensor, where an action must be triggered after a fixed number of occurrences. The standard workflow is: 1) Call ca_timer_init_counter_irq to set the target count and the callback, 2) Call ca_timer_start to enable the hardware, 3) The system automatically executes the callback when the count is reached. When the task is finished, always call ca_timer_deinit to release the hardware resource for other parts of the application.

Esempio:

```c
/* Define the callback function to be executed by the ISR */
void my_pulse_callback(void) {
    /* Keep this code extremely short and fast */
    /* Do not use ca_os_delay or blocking primitives here */
}

void setup_timer(void) {
    u8 timer_id = 2;
    u32 pulse_target = 100;

    /* Initialize the timer to trigger after 100 pulses */
    if (ca_timer_init_counter_irq(timer_id, pulse_target, my_pulse_callback)) {
        /* Start the timer hardware */
        ca_timer_start(timer_id);
    } else {
        /* Handle initialization error */
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_timer_init_counter_irq and what is its primary purpose?
A: You should use ca_timer_init_counter_irq to initialize an STM32 timer in counter mode with interrupts triggered after every N pulses. This is useful for measuring time intervals or counting events. The function sets up the timer to generate an interrupt after a specified number of input pulses, making it ideal for precise timing or event counting applications.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_timer_init_counter_irq with caOS types?
A: void my_callback(void) { // Handle interrupt }
int main(void) { bool success = ca_timer_init_counter_irq(3, 1000, my_callback); if (!success) { // Handle error } return 0; }

Parameters conf:0.95 **Q:** What does the 'preset' parameter control in ca_timer_init_counter_irq and what are valid values?

A: The 'preset' parameter specifies how many pulses must occur before an interrupt is generated. It must be greater than zero; otherwise, the function will fail. Valid values are any positive 32-bit integer (u32). A value of zero causes initialization to fail.

Return conf:0.90 **Q:** What do the return values of ca_timer_init_counter_irq indicate and what causes it to return false?

A: The function returns true if initialization completes successfully. It returns false if preset is zero, if acquiring the timer fails, or if hardware initialization fails. These failure conditions prevent the timer from being properly configured for counter mode with interrupt generation.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_timer_init_counter_irq

A: Common mistakes include passing preset = 0, which causes the function to fail; calling the function without first acquiring the timer; or using an unsupported timer number. Additionally, calling this function from an ISR is not allowed, as it's not designed for ISR use and may lead to undefined behavior.

`ca_timer_init_encoder`

Header: `caos/drivers/inc/timer.h:54`
Source: `caos/drivers/src/timer.c:106:120`
GUARD: `EN_TIMER`

```
bool ca_timer_init_encoder (u8 num, i64 *pos)
```

Initializes a hardware timer in encoder mode to track rotation position.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The hardware timer index (e.g., 1, 2, 8). Must correspond to a valid timer peripheral on the STM32.
`pos`	`i64 *`	Pointer to a 64-bit signed integer variable where the encoder count will be stored. Must not be NULL.

Ritorna:
Returns true if the timer was successfully acquired and configured as an encoder. Returns false if the pointer is NULL, the timer is already in use, or hardware initialization fails.

Note:

- This function is not thread-safe and must be called during the system initialization phase before starting the timer. It is not callable from an ISR because it performs resource acquisition and hardware configuration. A common mistake is passing a NULL pointer or an invalid timer index, which will cause the function to return false immediately. Ensure the memory pointed to by 'pos' remains valid for the entire lifetime of the timer operation.

Uso:

- Use this function when you need to interface with a rotary encoder to track position or speed. It is typically used in the main initialization sequence: first, you define a variable to hold the position, then call this function to bind the hardware timer to that variable, and finally call ca_timer_start to begin counting. It is often paired with ca_timer_encoder_read to retrieve the current value or ca_timer_deinit to release the hardware resource when finished.

Esempio:

```
i64 encoder_pos_accum = 0;

// Initialize timer 8 as an encoder
if (ca_timer_init_encoder(8, &encoder_pos_accum)) {
    // Start the timer to begin tracking pulses
    ca_timer_start(8);
} else {
    // Handle initialization failure (e.g., log error or retry)
    ca_printf("Failed to init encoder on timer 8\n");
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_timer_init_encoder and for what purpose?
A: You should call ca_timer_init_encoder when you need to initialize an STM32 timer in encoder mode to read incremental code counts. This is typically used with rotary encoders or other position sensors that output a quadrature signal. The function sets up the timer hardware and prepares it for reading position data.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_timer_init_encoder with caOS types?
A: i64 position = 0; bool success = ca_timer_init_encoder(2, &position); if (success) { // Timer initialized successfully, position can now be read } else { // Initialization failed }

Parameters conf:0.90 **Q:** What does the 'num' parameter represent and what are valid values?
A: The 'num' parameter specifies the STM32 timer number to initialize in encoder mode. For STM32F4, valid values are between 1 and 14. Using any other value will cause the function to return false. The timer must be available and not already in use by another thread.

Return conf:0.95 **Q:** What do the return values of ca_timer_init_encoder indicate and what causes failure?
A: The function returns true if initialization completes successfully. It returns false if initialization fails, which can happen due to: the pos pointer being NULL, failure to acquire access to the timer hardware, or hardware initialization failure. The function is not blocking but may fail if the timer is already in use.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_timer_init_encoder?
A: Common mistakes include passing NULL as the pos parameter, using an invalid timer number (outside 1-14 for STM32F4), or calling the function without having initialized the timer's clock. Additionally, calling this function from an ISR is not allowed, and it's not thread-safe, so concurrent access from multiple threads must be avoided.

`ca_timer_init_oneshot`

Header: `caos/drivers/inc/timer.h:52`
Source: `caos/drivers/src/timer.c:76:88`
GUARD: `EN_TIMER`

```
bool ca_timer_init_oneshot (u8 num, u32 delay_us, ca_timer_cb_t cb)
```

Configures a hardware timer to trigger a single callback function after a specified delay.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The hardware timer index to use. Must be a valid timer ID supported by the hardware abstraction layer.
`delay_us`	`u32`	The delay duration in microseconds before the callback is executed.
`cb`	`ca_timer_cb_t`	The function pointer to be executed once the timer expires. Must be a valid function address.

Ritorna:
Returns true if the timer was successfully acquired and configured. Returns false if the timer is already in use, hardware initialization fails, or invalid parameters are provided.

Note:

- This function is not thread-safe and should be called during system initialization or from a controlled task context. It is not intended for use inside an ISR, as it performs resource acquisition and hardware setup. A common mistake is failing to check the return value; if the timer is already locked by another process, the function will return false, and subsequent attempts to start the timer will fail. Ensure the timer hardware is properly clocked before calling this function.

Uso:

- This function is used to schedule one-time events, such as delayed sensor sampling or timeout mechanisms for communication protocols. It is typically paired with ca_timer_start to begin the countdown and ca_timer_deinit to release the hardware resource when the task is finished. It is often used in conjunction with ca_os_delay or event-based synchronization to manage asynchronous hardware operations.

Esempio:

```c
/* Define the callback function to be executed */
void my_timer_callback(void) {
    /* Perform action after delay */
}

void setup_delayed_action(void) {
    u8 timer_id = 1;
    /* Initialize timer for a 5000us (5ms) one-shot delay */
    if (ca_timer_init_oneshot(timer_id, 5000, my_timer_callback)) {
        /* Start the timer hardware */
        ca_timer_start(timer_id);
    } else {
        /* Handle error: timer could not be initialized */
    }
    /* Later, when finished, clean up the timer resource */
    ca_timer_deinit(timer_id);
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_timer_init_oneshot and what is its main purpose?
A: You should use ca_timer_init_oneshot to set up a one-shot timer that triggers a callback after a specified delay. It's useful for implementing delays, timeouts, or time-based synchronization in embedded applications. The timer fires only once and is ideal for non-periodic timing events.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_timer_init_oneshot with caOS types?
A:

```
u8 timer_num = 1;
u32 delay = 1000000; // 1 second
ca_timer_cb_t callback = my_callback;
bool result = ca_timer_init_oneshot(timer_num, delay, callback);
if (result) {
    // Timer initialized successfully
} else {
    // Initialization failed
}
```

Parameters conf:0.90 **Q:** What are the valid ranges for the timer number and delay parameters in ca_timer_init_oneshot?
A: The timer number (num) must be between 1 and 8, inclusive. The delay_us must be greater than 0, otherwise the timer will not activate correctly. Passing NULL for the callback (cb) will also cause the function to return false.

Return conf:0.95 **Q:** What does it mean when ca_timer_init_oneshot returns false?
A: A return value of false indicates that the timer initialization failed. This can happen if the timer number is invalid or already in use, if the delay is zero or invalid, or if the callback function pointer is NULL or invalid.

Error/Edge case conf:0.90 **Q:** What are the most common mistakes when calling ca_timer_init_oneshot?
A: Common mistakes include passing a timer number outside the valid range (1-8), setting delay_us to zero, or passing a NULL callback function. These errors will cause the function to return false and the timer will not be initialized properly.

`ca_timer_init_periodic`

Header: `caos/drivers/inc/timer.h:51`
Source: `caos/drivers/src/timer.c:62:74`
GUARD: `EN_TIMER`

```
bool ca_timer_init_periodic (u8 num, u32 period_us, ca_timer_cb_t cb)
```

Initializes a hardware timer to trigger a callback function repeatedly at a fixed interval.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The hardware timer index (e.g., 1, 2, 3). Must be between 1 and TIMER_MAX.
`period_us`	`u32`	The interval in microseconds between callback executions. Must be greater than 0.
`cb`	`ca_timer_cb_t`	The function pointer to be executed when the timer expires. Must be a valid function address.

Ritorna:
Returns true if the timer was successfully acquired and configured. Returns false if the timer index is invalid, already in use, or if hardware initialization fails.

Note:

- This function is not thread-safe and should be called during the system initialization phase before the scheduler starts. It is a blocking-style configuration function that interacts with hardware registers; it must not be called from an ISR. A common mistake is failing to check the return value, which leads to silent failures if the timer is already occupied by another task or driver. Ensure the timer index is not being used by other peripherals or PWM functions before calling this.

Uso:

- This function is typically used to set up periodic tasks like sensor polling, LED blinking, or system heartbeat generation. It is part of a standard sequence: first, call ca_timer_init_periodic to configure the hardware and assign the callback, then call ca_timer_start to begin the counting process. If the timer is no longer needed, it should be released using ca_timer_deinit to allow other parts of the application to reuse the hardware resource.

Esempio:

```c
/* Define the callback function to be executed periodically */
void my_periodic_task(void) {
    /* Toggle an LED to indicate activity */
    ca_gpio_toggle(CA_GPIO_PORT_A, CA_GPIO_PIN_5);
}

void setup_system(void) {
    u8 timer_id = 1;
    u32 interval_us = 100000; /* 100ms period */

    /* Initialize the timer and check for success */
    if (ca_timer_init_periodic(timer_id, interval_us, my_periodic_task)) {
        /* Start the timer after successful initialization */
        ca_timer_start(timer_id);
    } else {
        /* Handle error: timer might be busy or invalid */
        /* Log error using ca_printf or similar */
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_timer_init_periodic and what is its purpose in a real-time system?

A: You should use ca_timer_init_periodic to set up a periodic timer that generates interrupts at regular intervals. This is useful for implementing time-based tasks, such as periodic sensor readings, LED blinking, or scheduling periodic operations in a real-time embedded system. The function initializes a hardware timer to generate interrupts based on the specified period in microseconds.

Code example conf:0.90 **Q:** Can you show a minimal example of how to initialize a periodic timer using ca_timer_init_periodic?

A:

```c
void my_timer_callback(void) {
    // Timer interrupt handler
}

int main(void) {
    bool success = ca_timer_init_periodic(2, 1000, my_timer_callback);
    if (success) {
        // Timer initialized successfully
    }
    while (1) {
        // Main loop
    }
}
```

Parameters conf:0.95 **Q:** What are the valid ranges for the timer number and period parameters in ca_timer_init_periodic?

A: The timer number (num) must be between 1 and 4 for STM32 microcontrollers. Values 0 or greater than 4 are invalid. The period_us parameter must be a positive value greater than zero; a value of zero is invalid. Passing NULL as the callback function is also invalid.

Return conf:0.90 **Q:** What does it mean when ca_timer_init_periodic returns false?

A: A return value of false indicates that the initialization failed. This can happen if the timer is already in use, the timer number is invalid, or there is a hardware error during initialization. The function is blocking, so it will not return until initialization completes or an error occurs.

Error/Edge case conf:0.85 **Q:** What are the common mistakes or edge cases that could cause ca_timer_init_periodic to fail?

A: Common mistakes include passing a timer number outside the valid range (1-4), using a period of zero, or passing a NULL callback function. Additionally, attempting to initialize a timer that is already in use, or calling the function when the hardware is not properly initialized, will cause the function to return false. The function must not be called from an ISR.

`ca_timer_init_pwm`

Header: `caos/drivers/inc/timer.h:56`
Source: `caos/drivers/src/timer.c:137:162`
GUARD: `EN_TIMER`

```
bool ca_timer_init_pwm (u8 num, u8 ch, u32 freq_hz); /* ch 1-4;
   chiamabile pi  volte (canali) stessa freq */
```

Initializes a hardware timer channel for PWM output, supporting multiple channels on the same timer.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The hardware timer index (e.g., 1, 2, 3). Must be a valid timer supported by the hardware.
`ch`	`u8`	The specific channel index (1 to 4). Invalid values (outside 1-4) will cause the function to return false.
`freq_hz`	`u32`	The desired PWM frequency in Hertz. Must be greater than 0.

Ritorna:
Returns true if the timer channel was successfully configured, false otherwise. Failure occurs if parameters are invalid, the timer is already in use by a different mode, or if the requested frequency conflicts with an existing PWM configuration on the same timer.

Note:

- This function is not thread-safe and should be called during the initialization phase of your application. It is not intended to be called from an ISR. If you attempt to initialize multiple channels on the same timer, they must all share the exact same frequency; the function will reject any attempt to set a different frequency on an already active PWM timer. A common mistake is forgetting to call ca_timer_start after initialization, which leaves the timer hardware idle.

Uso:

- Use this function to set up hardware-generated PWM signals for tasks like LED dimming or motor control. The typical sequence is: 1) Call ca_gpio_configure to set the pin to the appropriate alternate function mode, 2) Call ca_timer_init_pwm to configure the timer hardware, 3) Call ca_timer_set_pwm to define the duty cycle, and 4) Call ca_timer_start to begin signal generation. If you need to change the duty cycle later, use ca_timer_set_pwm; if you need to stop the signal, use ca_timer_stop.

Esempio:

```
/* Configure GPIO pin for PWM output */
ca_gpio_configure(GPIOA, 0, GPIO_MODE_AF, CA_GPIO_OTYPE_PP, GPIO_SPEED_HIGH,
    GPIO_PULL_NONE, 0);

/* Initialize PWM on Timer 1, Channel 1 at 1kHz */
if (ca_timer_init_pwm(1, 1, 1000)) {
    /* Set duty cycle to 50% using internal driver function */
    /* ca_timer_set_pwm(1, 1, 500); */

    /* Start the timer to output the signal */
    ca_timer_start(1);
} else {
    ca_stdout_printf_n("PWM initialization failed");
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_timer_init_pwm in my application?
A: You should use this function during the initialization phase of your application to configure hardware-generated PWM signals for tasks like motor control or LED dimming. The typical sequence involves configuring the GPIO alternate function, initializing the timer, setting the duty cycle, and finally starting the timer.

Code example conf:1.00 **Q:** How do I initialize a PWM channel on timer 2, channel 1 at 1000Hz?
A: u8 timer_num = 2; u8 channel = 1; u32 freq = 1000; bool success = ca_timer_init_pwm(timer_num channel, freq);

Parameters conf:1.00 **Q:** What are the valid ranges for the parameters in ca_timer_init_pwm?
A: The 'num' parameter must be a valid hardware timer index supported by the device. The 'ch' parameter must be between 1 and 4 inclusive. The 'freq_hz' parameter must be a value greater than 0.

Return conf:1.00 **Q:** What does a return value of false indicate for ca_timer_init_pwm?
A: A return value of false indicates that the configuration failed. This occurs if the parameters are invalid, the timer is already in use by a different mode, or if you attempt to set a frequency that conflicts with an existing PWM configuration on the same timer.

Error/Edge case conf:1.00 **Q:** What is a common mistake when using ca_timer_init_pwm?
A: A common mistake is forgetting to call ca_timer_start after initialization, which results in the timer hardware remaining idle. Additionally, users must ensure that if multiple channels are initialized on the same timer, they all share the exact same frequency.

Behavior conf:1.00 **Q:** Is ca_timer_init_pwm safe to call from an interrupt service routine or in a multi-threaded environment?
A: No, this function is not thread-safe and is not intended to be called from an ISR. It should only be called during the application's initialization phase.

`ca_timer_is_busy`

Header: `caos/drivers/inc/timer.h:71`
Source: `caos/drivers/src/timer.c:56:60`
GUARD: `EN_TIMER`

```
bool ca_timer_is_busy (u8 num)
```

Checks if a specific hardware timer is currently allocated and in use.

Parametri:

Nome	Tipo	Descrizione
num	u8	The hardware timer index to check. Valid range is 1 to TIMER_MAX. Values outside this range will return true (considered busy/invalid).

Ritorna:
Returns true if the timer is currently in use or if the index is invalid. Returns false if the timer is available for allocation.

Note:

- This function is non-blocking and thread-safe, as it performs a simple bitwise check on the internal state variable. It is safe to call from an ISR, as it does not involve any complex synchronization primitives or blocking operations. The most common mistake is assuming that a 'false' return guarantees the timer will remain available; in a multi-threaded environment, another task could call an initialization function immediately after your check. Always ensure you have exclusive access or use this check as a guard before calling initialization functions.

Uso:

- This function is typically used as a safety guard before attempting to initialize a hardware timer. In a typical lifecycle, you would call ca_timer_is_busy(num) to verify availability, then proceed to call ca_timer_init_periodic() or ca_timer_init_oneshot() if the timer is free. It is also useful during system diagnostics to report which hardware resources are currently claimed by the application.

Esempio:

```c
/* Check if timer 1 is available before initializing it */
u8 timer_id = 1U;

if (!ca_timer_is_busy(timer_id)) {
    /* Timer is free, proceed with initialization */
    if (ca_timer_init_periodic(timer_id, 1000U, my_callback_function)) {
        ca_timer_start(timer_id);
    } else {
        /* Handle initialization failure */
    }
} else {
    /* Timer is already in use, log error or wait */
    ca_printf("Timer %d is busy!\n", timer_id);
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_timer_is_busy() in my embedded application?
A: You should call ca_timer_is_busy() before attempting to initialize or configure a timer to ensure it's not already in use. This helps prevent conflicts when multiple parts of your application might try to use the same timer resource.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_timer_is_busy() with caOS types?
A: u8 timer_num = 2; if (!ca_timer_is_busy(timer_num)) { // Timer is free, proceed with initialization ca_timer_init(timer_num, 1000); // Example init } else { // Timer is busy, handle accordingly // Error handling code here }

Parameters conf:0.90 **Q:** What does the 'num' parameter in ca_timer_is_busy() represent and what are valid values?
A: The 'num' parameter represents the timer number to check, which must be between 1 and TIMER_MAX inclusive. Using a value outside this range will cause the function to return true, indicating an error condition rather than a busy state.

Return conf:0.95 **Q:** What do the return values of ca_timer_is_busy() mean and what causes it to return true?
A: The function returns true if the timer is busy (in use) or if the input parameter is invalid (outside the range 1 to TIMER_MAX). It returns false only when the timer is free and the parameter is valid. Invalid parameters cause true to be returned to signal an error condition.

`ca_timer_set_pwm`

Header: `caos/drivers/inc/timer.h:59`
Source: `caos/drivers/src/timer.c:164:173`
GUARD: `EN_TIMER`

```
bool ca_timer_set_pwm (u8 num, u8 ch, u16 duty_pm); /* duty 0-1000
    permille, glitch-free */
```

Updates the duty cycle of an active PWM timer channel without causing signal glitches.

Parametri:

Nome	Tipo	Descrizione
`num`	`u8`	The timer peripheral index (1-based). Must be a timer previously initialized with ca_timer_init_pwm.
`ch`	`u8`	The specific channel index (1-4). Must be within the range configured during initialization.
`duty_pm`	`u16`	Duty cycle in permille (0-1000). Values > 1000 are automatically clamped to 1000.

Ritorna:
Returns true if the duty cycle was successfully updated, false if the timer is not busy, the channel is invalid, or the timer is not in PWM mode.

Note:

- This function is non-blocking and thread-safe for updating hardware registers. It is designed to be called from any task context, but it is not intended for use inside an ISR unless explicitly documented as such by the hardware abstraction layer. The most common mistake is calling this function before the timer has been successfully initialized with ca_timer_init_pwm, which will cause the function to return false immediately. Ensure the timer 'num' is active and in 'CA_TIMER_MODE_PWM' before calling, otherwise the request will be ignored.

Uso:

- This function is used in real-time control loops to adjust output power, such as dimming LEDs or controlling motor speeds. It is typically used after a successful ca_timer_init_pwm call. In a typical application lifecycle, you initialize the timer once during system startup, then repeatedly call ca_timer_set_pwm inside a control task or in response to user input events to change the duty cycle dynamically.

Esempio:

```
/* Example: Adjusting LED brightness based on a control value */
#define PWM_TIMER_ID 1
#define PWM_CHANNEL  1

void update_led_brightness(u16 new_duty_permille) {
    /* Ensure the timer is initialized before setting duty */
```

```
    /* ca_timer_init_pwm(PWM_TIMER_ID, PWM_CHANNEL, 1000); // Assume already
    done */

    if (ca_timer_set_pwm(PWM_TIMER_ID, PWM_CHANNEL, new_duty_permille)) {
        /* Successfully updated hardware registers */
    } else {
        /* Handle error: Timer not busy or invalid channel */
        /* Log error using ca_printf or similar */
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_timer_set_pwm in my application?
A: You should call it after a successful ca_timer_init_pwm call. It is typically used in real-time control loops or in response to user input to dynamically adjust output power, such as for motor speed control or LED dimming.

Code example conf:1.00 **Q:** Provide a minimal example of how to update a PWM duty cycle to 50% using caOS types.
A: u8 timer_num = 1; u8 channel = 1; u16 duty = 500; if (ca_timer_set_pwm(timer_num, channel, duty)) { /* Success */ }

Parameters conf:1.00 **Q:** What are the constraints for the duty_pm parameter?
A: The duty_pm parameter represents the duty cycle in permille, ranging from 0 to 1000. If you provide a value greater than 1000, the function will automatically clamp it to 1000.

Return conf:1.00 **Q:** Why might ca_timer_set_pwm return false?
A: It returns false if the timer is not currently busy, the specified channel is invalid, or the timer was not initialized in PWM mode. A common cause is calling the function before ca_timer_init_pwm has been successfully executed.

`ca_timer_start`

Header: `caos/drivers/inc/timer.h:62`
Source: `caos/drivers/src/timer.c:175:180`
GUARD: `EN_TIMER`

```
bool ca_timer_start (u8 num)
```

Activates a previously initialized hardware timer to begin counting or generating signals.

Parametri:

Nome	Tipo	Descrizione
num	u8	The identifier of the timer to start. Must correspond to a valid timer index previously configured via an init function.

Ritorna:
Returns true if the timer was successfully started, false if the timer is not busy (not initialized) or if the hardware failed to engage.

Note:

- This function is non-blocking and interacts directly with the STM32 hardware registers. It is not inherently thread-safe if multiple tasks attempt to start the same timer simultaneously, so ensure your application logic prevents race conditions. It is mandatory to call an initialization function (e.g., ca_timer_init_periodic or ca_timer_init_pwm) before calling this; otherwise, the function will return false because the timer is not marked as

'busy'. A common beginner mistake is calling this function before the hardware clock for the timer has been enabled or before the timer has been properly configured, leading to silent failures or unexpected behavior.

Uso:

- This function is the final step in the timer setup sequence: init -> start. It is typically used in the main initialization phase of a task or system to trigger periodic tasks, PWM signal generation, or encoder monitoring. It is often paired with ca_timer_stop to pause operations during low-power states or when a specific process is complete. In complex systems, it is used to synchronize the start of multiple hardware peripherals after all configurations are verified.

Esempio:

```c
/* Initialize a periodic timer to trigger every 100ms */
#define TIM_ID 1
#define PERIOD_MS 100

void setup_timer(void) {
    /* Configure the timer before starting it */
    if (ca_timer_init_periodic(TIM_ID, PERIOD_MS, my_callback_function)) {
        /* Successfully initialized, now start the hardware */
        if (!ca_timer_start(TIM_ID)) {
            /* Handle error: timer failed to start */
            ca_printf("Error: Could not start timer %d\n", TIM_ID);
        }
    } else {
        /* Handle initialization failure */
        ca_printf("Error: Initialization failed\n");
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_timer_start() in my embedded application?
A: You should call ca_timer_start() after initializing a timer with a function like ca_timer_init_periodic, ca_timer_init_counter_irq(), or ca_timer_init_encoder(). It is used to activate the timer hardware so it can begin counting, generating signals, or operating in encoder mode. This function is typically called once after setup, not repeatedly during runtime.

Code example conf:0.90 **Q:** Can you show a minimal example of using ca_timer_start() with caOS types?
A: u8 timer_num = 3; bool success = ca_timer_start(timer_num); if (!success) { // Handle error: invalid timer number or timer not initialized }

Parameters conf:0.90 **Q:** What does the 'num' parameter in ca_timer_start() represent and what values are valid?
A: The 'num' parameter represents the hardware timer number to start, ranging from 1 to TIMER_MAX. Values outside this range (less than 1 or greater than TIMER_MAX) will cause the function to return false. The timer must have been previously initialized using one of the ca_timer_init_* functions before calling this function.

Return conf:0.95 **Q:** What do the return values of ca_timer_start() indicate and what causes it to return false?
A: ca_timer_start() returns true if the timer was successfully started after being initialized. It returns false if the timer number is invalid (outside the range 1 to TIMER_MAX) or if the timer was not initialized beforehand. Common causes of failure include passing an invalid timer number or calling the function without prior initialization.

ca_timer_stop

Header: caos/drivers/inc/timer.h:63
Source: caos/drivers/src/timer.c:182:186
GUARD: EN_TIMER

```
bool ca_timer_stop (u8 num)
```

Stops a hardware timer that is currently running.

Parametri:

Nome	Tipo	Descrizione
num	u8	The identifier of the hardware timer to stop. Must be a valid index previously initialized via ca_timer_init.

Ritorna:
Returns true if the timer was successfully stopped, or false if the timer was not busy or the index is invalid.

Note:

- This function is a wrapper around hardware-specific timer control. It is not inherently thread-safe if multiple tasks access the same timer index simultaneously, so ensure proper synchronization if shared. It is non-blocking, meaning it returns immediately after triggering the hardware stop command. A common mistake is calling this function on a timer that has not been initialized, which may lead to undefined behavior or accessing invalid memory slots.

Uso:

- Use this function to halt a periodic or one-shot hardware timer when its task is complete or when a system state change requires the timer to stop. It is typically used in a lifecycle sequence: first, initialize the timer with ca_timer_init, start it with ca_timer_start, and finally call ca_timer_stop when the timing operation is no longer needed. Always check the return value to ensure the timer was actually in a state that allowed it to be stopped.

Esempio:

```c
/* Example: Stopping a timer after a specific event */
#include "caos.h"

void stop_my_timer(u8 timer_id) {
    /* Check if the timer is currently active before attempting to stop */
    if (ca_timer_is_busy(timer_id)) {
        /* Attempt to stop the hardware timer */
        if (ca_timer_stop(timer_id)) {
            /* Successfully stopped */
        } else {
            /* Handle error: timer could not be stopped */
        }
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_timer_stop() in my embedded application?
A: You should call ca_timer_stop() when you need to halt an already started STM32 timer, typically to stop a periodic task or prevent further timer interrupts. It's useful for dynamic control of timer operations, such as stopping a timer when a condition is met or during system shutdown.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_timer_stop() with caOS types?
A: u8 timer_num = 5; bool success = ca_timer_stop(timer_num); if (success) { // Timer was successfully stopped } else { // Timer was not running or invalid }

Parameters conf:0.90 **Q:** What is the valid range for the 'num' parameter in ca_timer_stop()?
A: The 'num' parameter must be an 8-bit unsigned integer between 1 and 16 inclusive. A value of 0 is invalid and will not cause an error, but the function will return false because no timer with number 0 exists in the system.

Return conf:0.95 **Q:** What does ca_timer_stop() return when it fails to stop a timer?
A: The function returns false when it fails, which happens if the timer specified by 'num' was not already running. This can occur if the timer was never started, or if it was already stopped before calling this function. It does not return an error code; it only indicates success or failure.

`ca_tls_accept`

Header: `caos/net/inc/tls.h:137`
Source: `caos/net/src/tls.c:611:837`
GUARD: `EN_TLS`

```
bool ca_tls_accept(ca_tls_t *tls, const ca_tls_config_t *cfg, const
    ca_tls_io_t *io)
```

Performs a TLS 1.3 server-side handshake to establish an encrypted connection.

Parametri:

Nome	Tipo	Descrizione
`tls`	`ca_tls_t *`	Pointer to the TLS context structure. Must be allocated and valid.
`cfg`	`const ca_tls_config_t *`	Configuration containing certificates and private keys. Must not be NULL.
`io`	`const ca_tls_io_t *`	Structure containing function pointers for network send/receive operations.

Ritorna:
Returns true if the handshake completed successfully, false if it failed (e.g., invalid handshake, network error, or certificate mismatch).

Note:

- This function is blocking; it will not return until the handshake is finished or an error occurs. It is NOT thread-safe for the same 'tls' object, meaning you cannot call it from multiple tasks simultaneously on the same handle. It must NOT be called from an ISR because it performs heavy cryptographic operations and network I/O. A common mistake is failing to initialize the 'tls' structure or providing an 'io' structure with NULL function pointers, which will cause an immediate return of false.

Uso:

- This function is the entry point for securing a raw TCP connection. It is typically called after a TCP socket has been accepted but before any application data is exchanged. You must pair it with 'ca_tls_close' once the communication is finished to free resources. It is often used in conjunction with 'ca_keys_store_temp_enable_access' to ensure the private key is accessible only during the handshake process.

Esempio:

```
ca_tls_t my_tls;
ca_tls_io_t my_io = { .send = my_tcp_send, .recv = my_tcp_recv };

// Ensure keys are accessible before starting the handshake
ca_keys_store_temp_enable_access(my_key_handle, 1);

// Perform the TLS 1.3 handshake
if (ca_tls_accept(&my_tls, &my_config, &my_io)) {
    // Handshake successful, proceed to read/write data
    ca_tls_read(&my_tls, buffer, sizeof(buffer));
} else {
    // Handle handshake failure
}

// Lock keys again for security
ca_keys_store_lock_completely(my_key_handle);

// Always close the TLS session when done
ca_tls_close(&my_tls);
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** At what point in the network communication lifecycle should I call ca_tls_accept?
A: You should call ca_tls_accept after a raw TCP socket has been accepted but before any application data is exchanged to secure the connection.

Code example conf:0.90 **Q:** Provide a minimal C example of how to invoke ca_tls_accept using caOS types.
A: ca_tls_t tls; ca_tls_config_t cfg; ca_tls_io_t io; bool success = ca_tls_accept(&tls, &cfg, &io);

Parameters conf:1.00 **Q:** What are the requirements for the parameters passed to ca_tls_accept?
A: The 'tls' pointer must be allocated and valid. The 'cfg' pointer must not be NULL and must contain certificates and private keys. The 'io' structure must contain valid function pointers for network operations; if these are NULL, the function will return false.

Return conf:1.00 **Q:** What do the return values of ca_tls_accept indicate?
A: It returns true if the TLS 1.3 handshake completed successfully. It returns false if the handshake failed due to reasons such as an invalid handshake, a network error, or a certificate mismatch.

Error/Edge case conf:1.00 **Q:** What are common mistakes that cause ca_tls_accept to fail immediately?
A: Common mistakes include failing to initialize the 'tls' structure or providing an 'io' structure that contains NULL function pointers for network operations.

Behavior conf:1.00 **Q:** Is ca_tls_accept safe to call from an ISR or multiple tasks?
A: No, it is not ISR-safe because it performs heavy cryptographic operations and network I/O. It is also not thread-safe for the same 'tls' object, meaning you cannot call it from multiple tasks simultaneously on the same handle.

`ca_tls_close`

Header: `caos/net/inc/tls.h:155`
Source: `caos/net/src/tls.c:942:954`
GUARD: `EN_TLS`

```
void ca_tls_close(ca_tls_t *tls)
```

Gracefully terminates a TLS connection by sending a close notification and clearing internal buffers.

Parametri:

Nome	Tipo	Descrizione
`tls`	`ca_tls_t *`	Pointer to the TLS session handle. Must be a valid, initialized pointer; passing NULL is safe and results in no action.

Ritorna:
void

Note:

- This function is not thread-safe and must not be called from an ISR. It performs a graceful shutdown by sending an 'AL_CLOSE_NOTIFY' alert to the peer if the connection is still established, ensuring the remote side knows the session is ending. After sending the alert, it clears all internal TLS state buffers (read, write, and handshake) to prevent data leakage or reuse of stale session keys. A common mistake is to call this function while another thread is still attempting to read or write to the same TLS handle, which will lead to undefined behavior or memory corruption.

Uso:

- Use this function whenever a secure session needs to be terminated, typically after finishing data exchange or when an error occurs during communication. It is usually the final step in a connection lifecycle, paired with the underlying TCP closure. In a typical sequence, you would call ca_http_serve_connection to process the request, then call ca_tls_close to tear down the security layer, and finally call ca_tcp_close to release the underlying network socket.

Esempio:

```
ca_tls_t s_tls;
ca_tcp_handle_t s_handle;

// ... assume s_tls and s_handle are initialized and connected ...

// Process the secure HTTP request
ca_http_serve_connection(&s_conn, &http_io);

// Gracefully close the TLS session
ca_tls_close(&s_tls);

// Close the underlying TCP connection
ca_tcp_close(s_handle);
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_tls_close in my network application lifecycle?
A: You should call ca_tls_close after finishing data exchange or when an error occurs during communication. It is the final step in the security layer lifecycle, typically called after processing requests (e.g., via ca_http_serve_connection) and before calling ca_tcp_close to release the underlying network socket.

Code example conf:0.90 **Q:** Provide a minimal C example showing how to properly terminate a TLS session using caOS types.
A: ca_tls_t *my_tls_handle = get_active_tls_session(); if (my_tls_handle != (void*)0) { ca_tls_close(my_tls_handle); }

Parameters conf:1.00 **Q:** What are the requirements for the ca_tls_t pointer passed to ca_tls_close?
A: The pointer must be a valid, initialized pointer to a TLS session handle. Passing NULL is safe and results in no action being taken by the function.

Error/Edge case conf:1.00 **Q:** What are the common mistakes or risks when calling ca_tls_close?
A: A common mistake is calling this function while another thread is simultaneously attempting to read or write to the same TLS handle, which leads to undefined behavior or memory corruption. Additionally, this function is not thread-safe and must never be called from an ISR.

---

ca_tls_read

Header: caos/net/inc/tls.h:143
Source: caos/net/src/tls.c:882:933
GUARD: EN_TLS

```
i32 ca_tls_read(ca_tls_t *tls, u8 *buf, u32 len, u32 timeout_ms)
```

Reads decrypted application data from an established TLS session.

Parametri:

Nome	Tipo	Descrizione
tls	ca_tls_t *	Pointer to the initialized and established TLS session handle.
buf	u8 *	Buffer where the decrypted data will be copied.
len	u32	Maximum number of bytes to read into the buffer.
timeout_ms	u32	Time in milliseconds to wait for data before returning a timeout error.

Ritorna:
Returns the number of bytes read (positive integer) on success. Returns TLS_READ_TIMEOUT if the operation expires, TLS_READ_CLOSED if the connection is terminated, or TLS_READ_ERROR for internal failures or invalid parameters.

Note:

- This function is blocking; it will suspend the calling task until data is available or the timeout expires. It is NOT safe to call from an ISR because it performs complex protocol processing and potential memory operations. The TLS handle must be fully established via a handshake before calling this function. A common mistake is passing a NULL pointer or a zero-length buffer, which will immediately return an error. Ensure the TLS session is properly initialized and the underlying network connection is active before attempting to read.

Uso:

- This function is used in the application layer to receive secure data over a network. It is typically called within a loop inside a dedicated task, often paired with ca_https_service_start or similar network services. The sequence is: initialize the TLS session, perform the handshake, call ca_tls_read to retrieve data, and finally close the session when communication is complete. It is the primary interface for consuming decrypted payloads from a secure stream.

Esempio:

```
/* Assuming 'tls_handle' is a valid ca_tls_t pointer */
u8 rx_buffer[256];
i32 bytes_received;

/* Attempt to read up to 256 bytes with a 5000ms timeout */
bytes_received = ca_tls_read(tls_handle, rx_buffer, sizeof(rx_buffer), 5000);
```

```

if (bytes_received > 0) {
    /* Data successfully received, process it */
    ca_printf("Received %d bytes\n", bytes_received);
} else if (bytes_received == TLS_READ_TIMEOUT) {
    /* Handle timeout: maybe retry or check connection status */
} else {
    /* Handle error or closed connection */
    ca_printf("TLS read failed or connection closed\n");
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_tls_read function in my application?
A: You should use ca_tls_read in the application layer to receive decrypted data from an established secure network stream. It is typically called within a loop inside a dedicated task after the TLS handshake has been successfully completed.

Code example conf:1.00 **Q:** Provide a minimal C example of how to read data using ca_tls_read.
A: u8 buffer[128]; u32 bytes_read = ca_tls_read(my_tls_handle, buffer, 128, 5000); if (bytes_read > 0) { /* Process data */ }

Parameters conf:1.00 **Q:** What do the 'len' and 'timeout_ms' parameters control?
A: The 'len' parameter specifies the maximum number of bytes to copy into the provided buffer. The 'timeout_ms' parameter defines the duration in milliseconds the function will wait for incoming data before returning a TLS_READ_TIMEOUT error.

Return conf:1.00 **Q:** What are the possible return values for ca_tls_read and what do they signify?
A: A positive integer indicates the number of bytes successfully read. TLS_READ_TIMEOUT indicates the wait time expired, TLS_READ_CLOSED indicates the connection was terminated, and TLS_READ_ERROR indicates internal failures or invalid parameters.

Error/Edge case conf:1.00 **Q:** What are some common mistakes that cause ca_tls_read to fail immediately?
A: Common mistakes include passing a NULL pointer for the TLS handle or providing a zero-length buffer, both of which will result in an immediate error return.

Behavior conf:1.00 **Q:** Is ca_tls_read safe to call from an Interrupt Service Routine (ISR)?
A: No, it is not safe to call from an ISR. The function is blocking and performs complex protocol processing and potential memory operations, which are unsuitable for interrupt context.

`ca_tls_write`

Header: `caos/net/inc/tls.h:149`
Source: `caos/net/src/tls.c:935:940`
GUARD: `EN_TLS`

```
bool ca_tls_write(ca_tls_t *tls, const u8 *data, u32 len)
```

Sends encrypted application data over an established TLS connection.

Parametri:

Nome	Tipo	Descrizione
`tls`	`ca_tls_t *`	Pointer to the TLS session handle. Must be a valid, initialized, and established session.
`data`	`const u8 *`	Pointer to the buffer containing the raw data to be encrypted and sent.
`len`	`u32`	Number of bytes to send. If 0, the function will return false if data is NULL.

Ritorna:
Returns true if the data was successfully queued for transmission, false otherwise (e.g., session not established, connection closed, or invalid parameters).

Note:

- This function is not thread-safe if multiple tasks attempt to write to the same TLS handle simultaneously; ensure access is serialized via a mutex if necessary. It is a blocking-style operation that relies on the underlying network stack to process the encrypted packets. Do not call this function from an ISR, as it may involve complex cryptographic operations and memory management that are not ISR-safe. A common mistake is attempting to write to a TLS handle before the handshake is fully completed or after the connection has been closed by the peer.

Uso:

- This function is typically used in network service tasks (like HTTPS or secure MQTT) to transmit application-layer payloads. It is usually called after a successful connection establishment (e.g., via ca_tcp_connect and the subsequent TLS handshake). In a typical lifecycle, you initialize the TLS context, perform the handshake, use ca_tls_write to send requests, and finally call ca_tls_close to clean up resources when the communication is finished.

Esempio:

```c
/* Assuming 'tls_handle' is a valid ca_tls_t* already connected */
const u8 *message = (const u8 *)"GET /index.html HTTP/1.1\r\n\r\n";
u32 msg_len = 20;

if (ca_tls_write(tls_handle, message, msg_len)) {
    /* Data successfully passed to the TLS layer for encryption */
} else {
    /* Handle error: check if connection is still established */
    /* Use ca_tls_close(tls_handle) to clean up if necessary */
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_tls_write in my application lifecycle?
A: You should call ca_tls_write after the TLS handshake is fully completed. It is typically used in network service tasks to transmit application-layer payloads, such as HTTPS or secure MQTT requests, following a successful connection establishment via ca_tcp_connect.

Code example conf:0.90 **Q:** Provide a minimal example of how to use ca_tls_write to send a buffer.
A: ca_tls_t *my_tls; u8 buffer[] = "Hello"; u32 len = 5; if (ca_tls_write(my_tls, buffer, len)) { /* Data queued successfully */ }

Parameters conf:1.00 **Q:** What are the requirements for the 'tls' and 'len' parameters?
A: The 'tls' parameter must be a pointer to a valid, initialized, and established TLS session handle. The 'len' parameter specifies the number of bytes to send; if 'len' is 0 and the 'data' pointer is NULL, the function will return false.

Return conf:1.00 **Q:** What does a return value of false indicate when calling ca_tls_write?
A: A return value of false indicates that the data was not queued for transmission. This can be caused by an invalid parameter, the TLS session not being established, or the connection having been closed by the peer.

`ca_top`

Header: `caos/inc/utils.h:147`
GUARD: `EN_UTILS`

```
static inline void ca_top(u16 mode)
```

Triggers a system-wide display or update of task information based on the provided mode.

Parametri:

Nome	Tipo	Descrizione
`mode`	`u16`	An enumeration value defining the level of detail or specific action for the task information dump. Must be a valid member of the ca_scheduler_tasks_info_mode enum.

Ritorna:
void

Note:

- This function is a wrapper for the scheduler's internal task reporting mechanism. It is not thread-safe and should generally be called from a CLI context or a diagnostic task, as it may temporarily suspend or iterate through task lists, potentially causing jitter in time-critical threads. It is strictly forbidden to call this from an Interrupt Service Routine (ISR) because it may perform blocking operations or access shared resources that are not protected for ISR context. The most common mistake is passing an undefined or out-of-range mode value, which can lead to undefined behavior or system crashes if the internal scheduler logic does not validate the input.

Uso:

- This function is typically used during system debugging or maintenance to inspect the state of the RTOS. It is often paired with ca_scheduler_update_all_stack_usage to ensure that the stack depth information is current before printing the report. In a CLI environment, it is used to provide the user with a snapshot of task health, priority, and resource consumption, helping to identify stack overflows or hung threads.

Esempio:

```
/* Example: Diagnostic command to update stack metrics and print task info */
void cli_cmd_task_info(u16 mode) {
    /* Ensure stack usage data is fresh before displaying */
    ca_scheduler_update_all_stack_usage();

    /* Trigger the scheduler to output task information based on the requested
    mode */
    ca_top(mode);

    /* Log the action to the system output */
    ca_printf("Task info dump requested with mode: %d\n", mode);
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_top and what are the risks associated with it?
A: ca_top is intended for system debugging or maintenance to inspect task health, priority, and resource consumption. It should be called from a CLI context or a diagnostic task. It is not thread-safe and may cause jitter in time-critical threads due to task list iteration. It must never be called from an ISR as it may perform blocking operations.

Code example conf:0.90 **Q:** How do I implement a simple CLI command to display task information using ca_top?
A: To ensure accurate data, you should update stack usage before calling the report function. Example: { ca_scheduler_update_all_stack_usage(); ca_top((u16)0); }

Parameters conf:1.00 **Q:** What does the mode parameter in ca_top represent and what happens if I pass an invalid value?
A: The mode parameter is a u16 value that must be a valid member of the ca_scheduler_tasks_info_mod enumeration, defining the level of detail or specific action for the report. Passing an undefined or out-of-range value can lead to undefined behavior or system crashes if the internal scheduler logic does not validate the input.

`ca_uart_configure_gpio`

Header: `caos/drivers/inc/uart.h:59`
Source: `caos/drivers/src/uart.c:87:90`
GUARD: `EN_UART`

```
1 bool ca_uart_configure_gpio(u32 uart_num)
```

Configures the physical GPIO pins (TX/RX) required for a specific UART peripheral.

Parametri:

Nome	Tipo	Descrizione
`uart_num`	`u32`	The index of the UART peripheral to configure (e.g., 0, 1, 2). Must be within the valid range defined by the hardware abstraction layer.

Ritorna:
Returns true if the GPIO configuration was successful, false if the uart_num is invalid or the hardware configuration failed.

Note:

- This function is a low-level hardware setup routine. It is generally non-blocking and performs direct register manipulation to set the pin modes (Alternate Function) and pull-up/pull-down resistors. It is not thread-safe in the sense that it should only be called once during the initialization phase of the UART peripheral. Do not call this from an ISR, as it is intended for system startup or driver initialization sequences. A common mistake is calling this function multiple times or while the UART is already active, which can cause glitches on the communication lines or unexpected hardware behavior.

Uso:

- This function is used as the first step in the UART initialization sequence. It is typically called before ca_uart_init to ensure the physical pins are ready to handle signals. Once the GPIOs are configured, the developer proceeds to initialize the UART logic, set the baudrate, and register the RX callback. It is part of the standard driver setup flow: configure GPIO -> initialize UART -> start communication.

Esempio:

```c
/* Initialize UART 1 hardware pins before starting the driver */
u32 uart_id = 1;

if (ca_uart_configure_gpio(uart_id)) {
    /* GPIOs are ready, now initialize the UART logic with a baudrate of 115200 */
    /* The third parameter is a pointer to the RX callback function */
    if (ca_uart_init(uart_id, 115200, my_rx_callback_function)) {
        ca_printf("UART 1 initialized successfully.\n");
    } else {
        ca_printf("Failed to initialize UART 1 logic.\n");
    }
} else {
    ca_printf("Failed to configure GPIO for UART 1.\n");
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_uart_configure_gpio() in my embedded application?
A: You should call ca_uart_configure_gpio() after initializing the GPIO system but before initializing the UART with ca_uart_init(). This function prepares the necessary GPIO pins for UART communication, ensuring that the hardware is correctly configured for serial data transmission and reception.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_uart_configure_gpio() with caOS types?
A: u32 uart_num = 2; bool success = ca_uart_configure_gpio(uart_num); if (success) { /* GPIO configured successfully */ } else { /* Handle configuration failure */ }

Parameters conf:0.90 **Q:** What does the uart_num parameter in ca_uart_configure_gpio() represent, and what are valid values?
A: The uart_num parameter specifies which UART peripheral to configure. Valid values are integers from 1 to CA_UART_MAX-1. Passing 0 or any value greater than or equal to CA_UART_MAX will cause the function to return false, indicating an invalid UART number.

Return conf:0.95 **Q:** What do the return values of ca_uart_configure_gpio() indicate, and what causes it to return false?
A: The function returns true if the GPIO pins for the specified UART were successfully configured. It returns false if the uart_num is invalid (outside the range 1 to CA_UART_MAX-1) or if there is a hardware error during the configuration process, such as unavailable pins or clock initialization issues.

`ca_uart_dbg_put`

Header: `caos/drivers/inc/uart.h:88`
Source: `caos/drivers/src/uart.c:180:187`
GUARD: `EN_UART`

```c
u16 ca_uart_dbg_put(p_str message)
```

Sends a null-terminated string to the system debug UART interface.

Parametri:

Nome	Tipo	Descrizione
`message`	`p_str`	A pointer to the null-terminated string to be transmitted. Must be a valid memory address containing a string.

Ritorna:
Returns a u16 value representing the number of bytes successfully queued or sent. Returns 0 if the debug UART is disabled (STDOUT_ON_UART is 0) or if the transmission failed.

Note:

- This function acts as a wrapper for the system's standard output (STDOUT) via UART. It is generally thread-safe, but its blocking behavior depends on the underlying driver implementation; if the UART buffer is full, it may block the calling task until space is available. It is not intended to be called from an ISR (Interrupt Service Routine) unless the underlying driver explicitly supports it. A common mistake is passing a NULL pointer or a string that is not null-terminated, which will cause the function to read invalid memory and potentially crash the system.

Uso:

- This function is used for logging, debugging, and printing status messages to a terminal connected to the debug UART. It is typically used throughout the application lifecycle, from initialization to main loop execution. It is often paired with system initialization functions to verify that hardware is ready, or used inside tasks to report state changes or error conditions. It serves as the primary tool for 'printf-style' debugging in embedded environments where a debugger is not attached.

Esempio:

```c
#include "caos.h"

void task_monitor(void *pvParameters) {
    p_str status_msg = "System monitor active.\n";

    // Send status message to debug UART
    u16 sent = ca_uart_dbg_put(status_msg);

    if (sent == 0) {
        // Handle error: UART might be disabled or busy
        // Log error to internal system audit if available
    }

    while (1) {
        // Perform periodic tasks
        ca_os_delay_ms(1000);

        // Report memory usage to debug UART
        u32 free_mem = ca_memory_get_avail_memory();
        // Use a buffer to format the string before sending
        // ca_snprintf(buffer, sizeof(buffer), "Free memory: %lu\n", free_mem);
        // ca_uart_dbg_put(buffer);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_uart_dbg_put() and what is its purpose in a caOS project?
A: You should use ca_uart_dbg_put() to send debug messages via UART, but only if STDOUT_ON_UART is enabled. It's intended for debugging purposes during development and is useful when you need to output status information or log messages from your embedded application. The function is not suitable for real-time or interrupt contexts due to its blocking behavior.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_uart_dbg_put() with caOS types?

A: cstr debug_msg = "System initialized\n"; u16 bytes_sent = ca_uart_dbg_put(debug_msg); if (bytes_sent > 0) { // Message sent successfully } else { // Either STDOUT_ON_UART is disabled or message was NULL }

Parameters conf:0.95 **Q:** What is the significance of the 'message' parameter in ca_uart_dbg_put(), and what happens if it is NULL?
A: The 'message' parameter is a pointer to a null-terminated string that will be sent via UART. If this parameter is NULL, the function will return 0 without sending anything, as per the documentation. Passing a valid pointer to a string is required for the function to perform any action.

Return conf:0.85 **Q:** What do the return values of ca_uart_dbg_put() indicate, and what causes the function to return 0?
A: The function returns the number of bytes sent if successful. It returns 0 if STDOUT_ON_UART is disabled, or if the message parameter is NULL. Other failure conditions such as UART errors are not explicitly mentioned, so 0 implies either configuration or input error.

`ca_uart_dma_abort`

Header: `caos/drivers/inc/uart.h:134`
Source: `caos/drivers/src/uart.c:335:340`
GUARD: `EN_UART`

```
1 void ca_uart_dma_abort(u32 uart_num)
```

Immediately stops an ongoing UART DMA reception process for a specific UART instance.

Parametri:

Nome	Tipo	Descrizione
`uart_num`	`u32`	The identifier of the UART peripheral to abort. Must be a valid index corresponding to an initialized UART instance.

Ritorna:
void

Note:

- This function is non-blocking and performs an immediate hardware-level shutdown of the DMA RX channel associated with the UART. It is thread-safe in the sense that it safely accesses internal driver structures, but it does not provide synchronization for the data already in the buffer. It is generally safe to call from an ISR, but be aware that it abruptly terminates the transfer, meaning any partial data currently being moved by the DMA will be lost or incomplete. A common mistake is calling this function without checking if a transfer is actually active, though the driver handles invalid indices gracefully by doing nothing.

Uso:

- Use this function when you need to cancel a pending UART reception, such as when a timeout occurs, a protocol error is detected, or the user manually requests a reset of the communication state machine. It is typically paired with ca_uart_dma_recv to start a new transfer after the abort. In a typical sequence, you might call ca_uart_dma_remaining to check if data is still expected, and if the system decides to abandon the current frame, you call ca_uart_dma_abort to clear the hardware state before re-initializing the buffer for the next packet.

Esempio:

```
/* Example: Aborting a UART reception due to a timeout or user command */
#define MY_UART_ID 1

void handle_uart_error(u32 uart_num) {
    /* Check if there is still data pending in the DMA buffer */
    u32 remaining = ca_uart_dma_remaining(uart_num);

    if (remaining > 0) {
        /* Stop the DMA transfer immediately to prevent further processing */
        ca_uart_dma_abort(uart_num);

        /* Log the event using the system debug output */
        ca_printf("UART %u: DMA aborted, %u bytes were pending\n", uart_num,
    remaining);
    }

    /* Reset the state machine or prepare for a new reception */
    /* ... logic to re-trigger ca_uart_dma_recv ... */
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_uart_dma_abort in my caOS application?
A: You should call ca_uart_dma_abort when you need to cancel a pending UART reception, such as during a timeout, a protocol error, or a manual reset of the communication state machine. It is typically used to clear the hardware state before re-initializing the buffer for a new packet, often following a check with ca_uart_dma_remaining.

Code example conf:0.90 **Q:** How do I implement a timeout-based abort using ca_uart_dma_abort?
A: If a timeout occurs, you can stop the DMA transfer as follows: u32 uart_id = 1; if (ca_uart_dma_remaining(uart_id) > 0) { ca_uart_dma_abort(uart_id); }

Parameters conf:1.00 **Q:** What are the requirements for the uart_num parameter in ca_uart_dma_abort?
A: The uart_num parameter is a u32 representing the identifier of the UART peripheral. It must be a valid index corresponding to an initialized UART instance. If an invalid index is provided, the driver handles it gracefully by doing nothing.

`ca_uart_dma_recv`

Header: `caos/drivers/inc/uart.h:116`
Source: `caos/drivers/src/uart.c:307:320`
GUARD: `EN_UART`

```
bool ca_uart_dma_recv(u32 uart_num, u8 *buf, u16 len)
```

Initiates a non-blocking DMA-based reception for a specific UART peripheral.

Parametri:

Nome	Tipo	Descrizione
`uart_num`	`u32`	The index of the UART peripheral to use. Must correspond to a valid, initialized UART instance.
`buf`	`u8 *`	Pointer to the destination buffer where received data will be stored. Must be a valid memory address.
`len`	`u16`	Number of bytes to receive. Must be greater than 0.

Ritorna:
Returns true if the DMA transfer was successfully configured and started. Returns false if the UART is not initialized, the buffer pointer is NULL, the length is zero, or the DMA channel configuration fails.

Note:

- This function is non-blocking; it configures the DMA controller to transfer data from the UART hardware register to your buffer in the background. It is not thread-safe if multiple tasks attempt to initiate a DMA transfer on the same UART channel simultaneously. It must not be called from an ISR unless specifically designed for it, as it relies on internal state management. A common beginner mistake is to assume the data is ready immediately after calling this function; you must wait for the DMA transfer to complete (e.g., via an event or semaphore) before accessing the buffer contents.

Uso:

- This function is typically used in communication protocols where you know the expected length of an incoming frame. The standard sequence is: first, initialize the UART and DMA using ca_uart_init and ca_uart_init_dma; second, call ca_uart_dma_recv to prepare the buffer; third, wait for the transfer to finish using a synchronization primitive like ca_event_wait; finally, process the received data. It is often paired with ca_uart_dma_send for full-duplex communication.

Esempio:

```
/* Example: Receiving a fixed-length frame via DMA */
#define UART_ID 1
#define FRAME_SIZE 16

static u8 rx_buffer[FRAME_SIZE];

void task_uart_handler(void) {
    /* Start DMA reception into our buffer */
    if (ca_uart_dma_recv(UART_ID, rx_buffer, FRAME_SIZE)) {
        /* Wait for the DMA transfer to complete (event signaled by ISR) */
        /* ca_event_wait(uart_dma_event, 1000); */

        /* Once signaled, the buffer contains the data */
        ca_hexdump_u32((u32)rx_buffer, FRAME_SIZE);
    } else {
        /* Handle error: UART not ready or invalid parameters */
    }
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_uart_dma_recv instead of a standard blocking UART read?
A: You should use ca_uart_dma_recv when you need non-blocking, background data reception, particularly for communication protocols where the expected length of an incoming frame is known. It allows the CPU to perform other tasks while the DMA controller handles the data transfer from the UART hardware register to your buffer.

Code example conf:1.00 **Q:** Provide a minimal example of how to initiate a DMA reception for 16 bytes on UART 1.
A: u8 buffer[16]; if (ca_uart_dma_recv(1, buffer, 16)) { /* DMA transfer started successfully */ }

Parameters conf:1.00 **Q:** What are the constraints on the 'len' parameter for ca_uart_dma_recv?
A: The 'len' parameter represents the number of bytes to receive and must be greater than 0. If 'len' is 0, the function will return false.

Return conf:1.00 **Q:** What does a return value of false indicate when calling ca_uart_dma_recv?
A: A return value of false indicates that the DMA transfer failed to start. This occurs if the UART is not initialized, the provided buffer pointer is NULL, the length is zero, or the DMA channel configuration fails.

Error/Edge case conf:1.00 **Q:** Is it safe to access the buffer immediately after calling ca_uart_dma_recv?
A: No, that is a common beginner mistake. Because the function is non-blocking, the data is not ready immediately. You must wait for the DMA transfer to complete using a synchronization primitive, such as ca_event_wait, before accessing the buffer contents.

`ca_uart_dma_remaining`

Header: `caos/drivers/inc/uart.h:128`
Source: `caos/drivers/src/uart.c:329:333`
GUARD: `EN_UART`

```
u32 ca_uart_dma_remaining(u32 uart_num)
```

Returns the number of bytes remaining to be received by the UART DMA controller.

Parametri:

Nome	Tipo	Descrizione
`uart_num`	`u32`	The identifier of the UART peripheral to query. Must be a valid UART index previously initialized in the system.

Ritorna:
Returns the count of bytes still pending in the DMA transfer buffer. Returns 0 if the UART index is invalid or if no DMA transfer is currently active.

Note:

- This function is non-blocking and thread-safe, as it simply reads the current state of the DMA hardware registers. It is safe to call from an ISR, though it is primarily intended for monitoring transfer progress from a task. A common beginner mistake is assuming this value is static; because DMA operates independently of the CPU, this value changes constantly while data is being received. Always ensure the UART peripheral has been properly initialized with DMA support before calling this, otherwise, it will return 0.

Uso:

- This function is typically used in communication protocols to verify if a full packet has arrived or to monitor the progress of a large data transfer. It is often paired with ca_uart_dma_wait to block until a transfer completes, or used in a polling loop to check if the expected number of bytes has been received. In CLI or diagnostic tasks, it is used to report the health and status of the communication interface to the user.

Esempio:

```c
/* Check if we have received enough bytes for a full packet */
#define MY_UART_ID 1
#define EXPECTED_BYTES 16

void check_uart_status(void) {
    u32 remaining = ca_uart_dma_remaining(MY_UART_ID);

    if (remaining == 0) {
        /* All bytes received, safe to process buffer */
        ca_printf_n("Transfer complete.");
```

```
    } else {
        /* Still waiting for data */
        ca_printf_n("Waiting for %u more bytes...", remaining);
    }
}

/* Example of aborting a stuck transfer */
void force_reset_uart(void) {
    if (ca_uart_dma_remaining(MY_UART_ID) > 0) {
        ca_uart_dma_abort(MY_UART_ID);
        ca_printf_n("DMA transfer aborted by user.");
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_uart_dma_remaining in my application?
A: You should use this function to monitor the progress of large data transfers or to verify if a full packet has arrived in communication protocols. It is often used in polling loops to check for received bytes or in diagnostic tasks to report the health of the UART interface.

Code example conf:1.00 **Q:** How do I implement a simple check for pending UART DMA bytes using caOS types?
A: u32 uart_id = 1; u32 pending = ca_uart_dma_remaining(uart_id); if (pending > 0) { /* Handle pending data */ }

Parameters conf:1.00 **Q:** What does the uart_num parameter represent and what are the requirements for it?
A: The uart_num parameter is the identifier of the UART peripheral you wish to query. It must be a valid UART index that has been previously initialized in the system with DMA support; otherwise, the function will return 0.

Return conf:1.00 **Q:** What does a return value of 0 indicate from ca_uart_dma_remaining?
A: A return value of 0 indicates that either no DMA transfer is currently active, the provided UART index is invalid, or the UART peripheral was not properly initialized with DMA support.

`ca_uart_dma_send`

Header: `caos/drivers/inc/uart.h:144`
Source: `caos/drivers/src/uart.c:344:357`
GUARD: `EN_UART`

```
u16 ca_uart_dma_send(u32 uart_num, const u8 *buf, u16 len)
```

Transmits a buffer over UART using DMA, blocking the calling task until the transfer completes.

Parametri:

Nome	Tipo	Descrizione
`uart_num`	`u32`	The index of the UART peripheral to use. Must correspond to a valid, initialized UART instance.
`buf`	`const u8 *`	Pointer to the data buffer to send. Must not be NULL and must remain valid until the function returns.
`len`	`u16`	Number of bytes to transmit. Must be greater than 0.

Ritorna:
Returns the number of bytes sent (len) on success, or 0 if the parameters are invalid or the DMA transfer failed to initiate.

Note:

- This function is blocking: the calling task will be suspended by the RTOS until the DMA controller signals that the entire buffer has been transmitted. It is thread-safe, but it must NOT be called from an ISR because it performs a blocking wait. Before calling this, ensure the UART peripheral is properly initialized and the DMA channel is available. A common beginner mistake is passing a pointer to a local (stack-allocated) buffer that goes out of scope or is modified while the DMA is still reading from it, leading to data corruption.

Uso:

- This function is typically used in communication tasks where high throughput is required without consuming CPU cycles during the transmission. It is often paired with buffer preparation functions like ca_memcpy to assemble packets before sending. The typical sequence is: 1) Prepare data in a buffer, 2) Call ca_uart_dma_send to transmit, 3) The task automatically yields to other tasks while the hardware handles the transfer, 4) The task resumes once the transmission is complete.

Esempio:

```c
/* Example: Sending a prepared packet over UART */
#define UART_ID 1
#define BUF_SIZE 16

void task_send_data(void) {
    u8 tx_buffer[BUF_SIZE];

    /* Prepare data */
    ca_memset(tx_buffer, 0, BUF_SIZE);
    tx_buffer[0] = 0xAA; /* Header */
    tx_buffer[1] = 0x05; /* Payload */

    /* Send via DMA; task will block here until transmission is finished */
    u16 sent = ca_uart_dma_send(UART_ID, tx_buffer, 2);

    if (sent == 0) {
        /* Handle error: DMA failed or invalid params */
    }
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I choose ca_uart_dma_send over a standard polling UART transmit function?
A: You should use ca_uart_dma_send in communication tasks where high throughput is required. It allows the CPU to perform other tasks while the hardware handles the transmission, as the calling task is suspended until the DMA transfer completes.

Code example conf:0.90 **Q:** How do I implement a simple UART DMA transmission in caOS?
A: Ensure the UART is initialized, then call the function: u32 uart_id = 1; u8 data[] = {0x01, 0x02, 0x03}; u16 sent = ca_uart_dma_send(uart_id, data, 3);

Parameters conf:1.00 **Q:** What are the constraints on the 'len' parameter for ca_uart_dma_send?
A: The 'len' parameter must be greater than 0. If it is 0 or if other parameters like the buffer pointer are invalid, the function will return 0.

Return conf:1.00 **Q:** What does a return value of 0 indicate when calling ca_uart_dma_send?
A: A return value of 0 indicates that the function failed to initiate the DMA transfer or that the provided parameters were invalid.

Error/Edge case conf:1.00 **Q:** Why is it dangerous to pass a stack-allocated buffer to ca_uart_dma_send?
A: Because the function is blocking and relies on DMA, the buffer must remain valid until the transfer completes. If the buffer is local to a function that returns or goes out of scope while the DMA is still reading from it, data corruption will occur.

`ca_uart_dma_wait`

Header: `caos/drivers/inc/uart.h:155`
Source: `caos/drivers/src/uart.c:359:364`
GUARD: `EN_UART`

```
bool ca_uart_dma_wait(u32 uart_num, u32 timeout_ms)
```

Blocks the current task until a UART DMA reception is complete or the timeout expires.

Parametri:

Nome	Tipo	Descrizione
`uart_num`	`u32`	The identifier of the UART peripheral to monitor.
`timeout_ms`	`u32`	Maximum time to wait in milliseconds. Use CA_WAIT_FOREVER to block indefinitely.

Ritorna:
Returns true if the DMA transfer finished successfully, false if the timeout occurred or the UART index is invalid.

Note:

- This function is blocking, meaning the calling task will be suspended by the RTOS scheduler until the DMA hardware signals completion or the timeout is reached. It is NOT safe to call this from an Interrupt Service Routine (ISR) because it relies on task-level blocking mechanisms. Ensure the UART and DMA channels are properly initialized via ca_uart_init_dma before calling this. A common mistake is passing an incorrect UART index or a timeout that is too short, causing the function to return false prematurely while the hardware is still processing data.

Uso:

- This function is typically used in a producer-consumer pattern where a task waits for a complete data frame to arrive via UART. It is usually paired with ca_uart_init_dma to set up the transfer and then called in a loop to process incoming data packets. After this function returns true, the application can safely access the buffer where the DMA has placed the received data. If it returns false, the application should implement error handling or recovery logic, such as resetting the UART peripheral.

Esempio:

```
/* Assuming UART1 is initialized for DMA reception */
#define MY_UART 1

void uart_task(void *pvParameters) {
    while (1) {
        /* Wait for incoming data indefinitely */
        if (ca_uart_dma_wait(MY_UART, CA_WAIT_FOREVER)) {
            /* Data is now ready in the buffer */
            ca_printf_n("Data received successfully");
            /* Perform data processing here */
        } else {
            /* Handle timeout or error */
```

```
            ca_printf_n("UART DMA wait failed or timed out");
            /* Optional: ca_uart_dma_abort(MY_UART); */
        }
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_uart_dma_wait and why is it necessary?
A: You should use ca_uart_dma_wait when you need to synchronize your task with a UART DMA reception. It is necessary because it blocks the current task until the DMA controller signals that the transfer is complete, ensuring the data buffer is ready for processing. It is typically used in a producer-consumer pattern after calling ca_uart_dma_recv.

Code example conf:0.90 **Q:** How do I implement a basic UART DMA receive operation using caOS types?
A: You must first initiate the transfer and then wait for completion. Example: u32 uart_id = 1; ca_uart_dma_recv(uart_id, buffer, length); ca_uart_dma_wait(uart_id);

Parameters conf:1.00 **Q:** What does the uart_num parameter represent and are there any restrictions on its usage?
A: The uart_num parameter is a u32 identifier for the specific UART peripheral (e.g., 1, 2, or 3). It is critical that you do not call this function from an ISR, as it relies on the OS scheduler to suspend the task. Additionally, you must ensure a DMA transfer was previously initiated via ca_uart_dma_recv, otherwise, the task may wait indefinitely.

`ca_uart_hw_configure_gpio`

Header: `caos/drivers/inc/ca_glue_hw.h:307`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_uart_hw_configure_gpio(u32 uart_num, u8 alternate); /*
    configures TX/RX pins in alternate function mode; alternate selects
    the pin pair (0 = default) */
```

Configures the physical GPIO pins for a specific UART peripheral to operate in Alternate Function mode.

Parametri:

Nome	Tipo	Descrizione
`uart_num`	`u32`	The hardware index of the UART peripheral (e.g., 1 for USART1). Valid values depend on the specific STM32 chip series; invalid values return false.
`alternate`	`u8`	Selects the specific pin mapping for the UART. 0 is the default mapping. Other values select alternative pin pairs if supported by the hardware; otherwise, it falls back to default.

Ritorna:
Returns true if the GPIO pins were successfully configured for the requested UART, or false if the uart_num is unsupported or invalid.

Note:

- This function is non-blocking and performs direct register manipulation to set up the GPIO hardware. It is not thread-safe and should only be called during the system initialization phase, before the UART peripheral is actively used by tasks. It is not intended to be

called from an ISR. The most common beginner mistake is attempting to reconfigure pins while the UART is already transmitting or receiving data, which causes immediate signal corruption and potential hardware bus contention.

Uso:

- This function is a low-level driver helper used during the hardware abstraction layer (HAL) initialization sequence. It is typically invoked by the higher-level ca_uart_init function to ensure the physical pins are routed to the UART peripheral before the peripheral clock is enabled and registers are configured. You should never call this function manually in your application code; always rely on the standard driver initialization sequence to ensure the entire peripheral stack is set up in the correct order.

Esempio:

```c
/* Example of initializing a UART peripheral */
#define MY_UART_ID 1
#define MY_BAUD_RATE 115200

void setup_my_serial(void) {
    /* Configure the physical pins for the UART before enabling the peripheral */
    if (ca_uart_hw_configure_gpio(MY_UART_ID, 0)) {
        /* If pins are ready, proceed to initialize the UART driver */
        /* Call the driver initialization function here */
    } else {
        /* Handle error: invalid UART ID or hardware failure */
        ca_printf("Failed to configure GPIO for UART %d\n", MY_UART_ID);
    }
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_uart_hw_configure_gpio() in my UART initialization sequence?
A: You should call ca_uart_hw_configure_gpio() before initializing the UART with ca_uart_init(). This function configures the GPIO pins for alternate function mode, which is required for UART communication. Without this step, the UART will not be able to transmit or receive data correctly.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_uart_hw_configure_g
A: u32 result = ca_uart_hw_configure_gpio(1); if (result != 0) { /* Handle error */ }

Parameters conf:0.85 **Q:** What does the uart_num parameter in ca_uart_hw_configure_gpio() represent, and what are valid values?
A: The uart_num parameter specifies which UART instance to configure. Valid values are integers from 1 to 8, depending on the MCU family. For example, in H7xx and H5xx families, UARTs 1-8 are supported, while in F7xx, only 1-3 are typically available.

Return conf:0.90 **Q:** What does ca_uart_hw_configure_gpio() return, and what causes it to return a non-zero value?
A: The function returns 0 on success and a non-zero value on failure. It returns a non-zero value when the uart_num parameter is outside the valid range (e.g., less than 1 or greater than 8), indicating an invalid UART instance number.

Error/Edge case conf:0.80 **Q:** What are common mistakes when using ca_uart_hw_configure_gpio()?
A: Common mistakes include calling ca_uart_hw_configure_gpio() after ca_uart_init(), which will not work correctly, or passing an invalid uart_num value such as 0 or 9. Another mistake is assuming that the function handles all GPIO configuration, when in fact it only sets the alternate function mode for specific pins.

Behavior conf:0.85 **Q:** Is ca_uart_hw_configure_gpio() blocking or non-blocking, and is it safe to call from an ISR?

A: The function is non-blocking. However, it is not safe to call it from an ISR because it is not marked as ISR-safe and may not be reentrant. Additionally, it is not thread-safe, so concurrent calls from multiple threads should be avoided.

HW differences conf:0.75 **Q:** How do the GPIO configurations differ between H7xx and H5xx MCU families for UARTs?

A: In H7xx, UARTs 1-8 are supported with specific pin assignments such as PA9/PA10 for UART1, PD5/PD6 for UART2, and PE8/PE7 for UART7. In H5xx, UARTs 1-6 are supported, with UART2 using PD5/PD6 instead of PD5/PD6, and UART6 using PC6/PC7 instead of PC6/PC7. The main difference lies in the pin mapping and AF selection for each UART instance.

`ca_uart_hw_dma_bind`

Header: `caos/drivers/inc/ca_glue_hw.h:307`
GUARD: `EN_CA_GLUE_HW`

```
1 extern bool ca_uart_hw_dma_bind(u32 uart_num, i16 *rx_flat, u16 *rx_req,
      i16 *tx_flat, u16 *tx_req); /* canale DMA (flat, -1 = qualsiasi) +
    request RX/TX della uart */
```

Retrieves the hardware-specific DMA channel and request mapping for a given UART peripheral.

Parametri:

Nome	Tipo	Descrizione
`uart_num`	`u32`	The UART peripheral index (e.g., 1 for USART1). Must be within the valid range for the specific MCU.
`rx_flat`	`i16 *`	Pointer to store the calculated RX DMA channel index. Returns -1 if the mapping is not applicable.
`rx_req`	`u16 *`	Pointer to store the DMA request/channel selection value required by the hardware.
`tx_flat`	`i16 *`	Pointer to store the calculated TX DMA channel index.
`tx_req`	`u16 *`	Pointer to store the DMA request/channel selection value for TX.

Ritorna:
Returns true if the UART number is valid and the mapping was successfully populated; returns false if the uart_num is invalid or unsupported.

Note:

- This function is non-blocking and thread-safe as it performs simple arithmetic and memory writes based on constant tables. It is safe to call from an ISR, provided the pointers passed are valid. It does not initialize any hardware; it only provides the configuration data required by the DMA controller. A common mistake is assuming this function enables the DMA or the UART; it is strictly a helper to resolve hardware-specific routing before calling initialization functions like ca_uart_init_dma.

Uso:

- This function is used during the initialization phase of a UART peripheral that requires DMA support. It is typically called immediately before requesting a DMA channel from the

system. The sequence is: 1) Call ca_uart_hw_dma_bind to get the required channel indices and request IDs. 2) Use the returned 'flat' indices to call ca_dma_free_channel_index to reserve the hardware resources. 3) Pass the resulting channel handles and request IDs to ca_uart_init_dma to complete the setup.

Esempio:

```c
void setup_uart_dma(u32 uart_id) {
    i16 rx_flat, tx_flat;
    u16 rx_req, tx_req;
    ca_dma_ch_t rx_ch, tx_ch;

    // Get the hardware mapping for the specific UART
    if (!ca_uart_hw_dma_bind(uart_id, &rx_flat, &rx_req, &tx_flat, &tx_req)) {
        return; // Invalid UART ID
    }

    // Reserve the DMA channels based on the binding
    if (ca_dma_free_channel_index(rx_flat, &rx_ch) < 0) return;
    if (ca_dma_free_channel_index(tx_flat, &tx_ch) < 0) return;

    // Configure the DMA request mapping for the reserved channels
    ca_dma_set_request(&rx_ch, rx_req);
    ca_dma_set_request(&tx_ch, tx_req);

    // Initialize the UART with the configured DMA channels
    // ca_uart_init_dma(uart_id, &rx_ch, &tx_ch);
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** At what stage in the initialization process should I call ca_uart_hw_dma_bind?
A: You should call it during the UART initialization phase, specifically before requesting DMA resources. The sequence is: first call ca_uart_hw_dma_bind to resolve hardware routing, then use the returned 'flat' indices to call ca_dma_free_channel_index to reserve the resources, and finally pass the resulting handles and request IDs to ca_uart_init_dma.

Code example conf:1.00 **Q:** Provide a minimal C example showing how to retrieve DMA binding information for USART1.
A: u32 uart = 1; i16 rx_f, tx_f; u16 rx_r, tx_r; if (ca_uart_hw_dma_bind(uart, &rx_f, &rx_r, &tx_f, &tx_r)) { /* Proceed to reserve channels using rx_f and tx_f */ }

Parameters conf:1.00 **Q:** What do the 'flat' and 'req' parameters represent in ca_uart_hw_dma_bind?
A: The 'flat' parameters (rx_flat, tx_flat) represent the calculated DMA channel index used by the system to manage resources. The 'req' parameters (rx_req, tx_req) represent the hardware-specific DMA request or channel selection value required by the MCU's DMA controller to route the UART peripheral signals correctly.

Return conf:1.00 **Q:** What does a return value of false indicate when calling ca_uart_hw_dma_bind?
A: A return value of false indicates that the provided uart_num is either invalid for the specific MCU or unsupported by the current hardware implementation. In such cases, the output pointers for the DMA channels and requests will not be populated with valid configuration data.

Error/Edge case conf:1.00 **Q:** What is a common mistake developers make when using ca_uart_hw_dma_bind
A: A common mistake is assuming that calling this function enables the DMA or the UART peripheral. This function is strictly a helper to resolve hardware-specific routing; it does not perform any hardware initialization. You must still call functions like ca_uart_init_dma to complete the setup.

HW differences conf:1.00 **Q:** How does the DMA binding logic differ between the F1xx and H7xx families?

A: In the F1xx family, DMA channels are fixed in hardware and the request values are ignored (no selection register). In the H7xx family, the function uses a DMAMUX-based approach where request IDs are explicitly mapped (e.g., 41U for USART1 RX), and it specifically excludes BDMA channels (16-23) because they lack USART requests.

---

ca_uart_hw_dma_init

Header: `caos/drivers/inc/ca_glue_hw.h:311`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_uart_hw_dma_init(USART_TypeDef *inst); /* abilita CR3.
    DMAT; RX resta in modalit  header (RXNEIE) */
```

Enables DMA transmission mode for a specific UART peripheral.

Parametri:

Nome	Tipo	Descrizione
`inst`	`USART_TypeDef *`	Pointer to the STM32 UART hardware instance (e.g., USART1, USART2). Must be a valid, initialized peripheral base address.

Ritorna:
void

Note:

- This function is non-blocking and performs a direct register write to the CR3 register to set the DMAT bit. It is thread-safe in the sense that it only modifies a specific hardware register, but it must be called only after the UART peripheral has been properly clocked and configured. It is safe to call from an ISR, but typically it is called during the initialization phase of the UART driver. A common mistake is calling this before the UART peripheral clock is enabled in the RCC, which will result in a hard fault or ignored configuration.

Uso:

- This function is used to transition a UART peripheral from standard interrupt-driven transmission to DMA-assisted transmission. It is typically called during the driver initialization sequence, immediately after configuring GPIO pins and setting the baud rate. It is usually paired with ca_uart_init to set up the hardware, followed by ca_uart_dma_send or ca_uart_dma_recv to perform actual data transfers. It does not affect the RX path, which remains in standard interrupt mode (RXNEIE).

Esempio:

```
/* Initialize UART1 for DMA usage */
USART_TypeDef *uart_inst = USART1;

/* 1. Configure GPIO pins for UART1 (e.g., TX/RX pins) */
ca_uart_configure_gpio(uart_inst);

/* 2. Enable DMA transmission mode in the UART hardware */
ca_uart_hw_dma_init(uart_inst);

/* 3. Now the UART is ready to use DMA for sending data */
/* ca_uart_dma_send(uart_inst, data_buffer, length); */
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_uart_hw_dma_init in my caOS application?
A: You should call this function during the UART driver initialization sequence, specifically after configuring GPIO pins and setting the baud rate, but only after the UART peripheral clock has been enabled in the RCC. It is used to transition the UART from standard interrupt-driven transmission to DMA-assisted transmission.

Code example conf:0.90 **Q:** Provide a minimal C example of how to initialize DMA for USART2 using caOS types.
A: void init_uart_dma(void) { USART_TypeDef *uart_inst = USART2; ca_uart_hw_dma_init(uart
}

Parameters conf:1.00 **Q:** What does the inst parameter represent and what are the requirements for a valid input?
A: The inst parameter is a pointer to the STM32 UART hardware instance (e.g., USART1, USART2). It must be a valid, initialized peripheral base address. Passing an uninitialized or invalid address will result in undefined behavior.

HW differences conf:1.00 **Q:** Are there differences in how ca_uart_hw_dma_init is implemented across the supported STM32 families?
A: No, the implementation is consistent across all listed families (f1xx, f4xx, f7xx, h5xx, h7xx, u5xx). Every implementation performs a direct register write to the CR3 register to set the USART_CR3_DMAT bit.

`ca_uart_hw_dma_rx_addr`

Header: `caos/drivers/inc/ca_glue_hw.h:309`
GUARD: `EN_CA_GLUE_HW`

```
1 extern volatile void *ca_uart_hw_dma_rx_addr(USART_TypeDef *inst); /*
    indirizzo registro dati RX per il DMA (DR/RDR) */
```

Returns the memory-mapped address of the UART receive data register for DMA operations.

Parametri:

Nome	Tipo	Descrizione
`inst`	`USART_TypeDef *`	Pointer to the hardware UART instance (e.g., USART1, USART2). Must be a valid, initialized peripheral base address.

Ritorna:
A volatile void pointer to the hardware register (RDR or DR) where received data is stored. Never returns NULL for a valid peripheral instance.

Note:

- This function is non-blocking and thread-safe as it performs a simple pointer arithmetic operation on the peripheral base address. It is safe to call from an ISR. The primary purpose is to provide the DMA controller with the exact memory address it must read from when performing peripheral-to-memory transfers. A common mistake is passing a NULL or uninitialized peripheral pointer, which will cause a hard fault when the DMA controller attempts to access the invalid address.

Uso:

- This function is used exclusively in the context of setting up DMA-based UART reception. It is typically called during the initialization phase of a UART driver or when preparing a new DMA transfer buffer. It is almost always paired with ca_dma_copy_target, where it serves as the 'source' address for the DMA transfer, and followed by a call to enable the UART's DMA request bit (e.g., via a hardware-specific register write).

Esempio:

```
/* Example: Setting up a DMA transfer for UART1 */
USART_TypeDef *uart_inst = USART1;
u8 rx_buffer[64];

/* Get the hardware RX register address for the DMA controller */
volatile void *rx_reg_addr = ca_uart_hw_dma_rx_addr(uart_inst);

/* Configure DMA to copy data from the UART RX register to our buffer */
/* We use the channel index 0 for this example */
if (ca_dma_copy_target(0, rx_buffer, rx_reg_addr, 64, CA_DMA_DIR_P2M,
    CA_DMA_WIDTH_8)) {
    /* DMA transfer successfully configured */
    /* Now enable the UART DMA hardware request bit */
    /* (Hardware-specific register access: uart_inst->CR3 |= USART_CR3_DMAR;) */
} else {
    /* Handle configuration error */
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_uart_hw_dma_rx_addr in my caOS project?
A: You should call this function during the initialization phase of a UART driver or when preparing a new DMA transfer buffer. Its primary purpose is to provide the DMA controller with the correct memory-mapped address of the UART receive data register to enable peripheral-to-memory transfers.

Code example conf:0.90 **Q:** How do I use ca_uart_hw_dma_rx_addr to configure a DMA transfer?
A: You typically use it to set the source address for a DMA transfer. Example: void *src = (void *)ca_uart_hw_dma_rx_addr(USART1); ca_dma_copy_target(dma_handle, src, buffer_ptr, size);

Parameters conf:1.00 **Q:** What are the requirements for the 'inst' parameter in ca_uart_hw_dma_rx_addr?
A: The 'inst' parameter must be a pointer to a valid, initialized hardware UART peripheral base address (e.g., USART1, USART2). Passing a NULL or uninitialized pointer is invalid and will cause a hard fault when the DMA controller attempts to access the resulting address.

HW differences conf:1.00 **Q:** Are there differences in how ca_uart_hw_dma_rx_addr handles different STM32 families?
A: Yes, the function abstracts hardware-specific register naming. For F1xx and F4xx families, it returns the address of the 'DR' register. For F7xx, H5xx, H7xx, and U5xx families, it returns the address of the 'RDR' register.

`ca_uart_hw_dma_rx_off`

Header: `caos/drivers/inc/ca_glue_hw.h:313`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_uart_hw_dma_rx_off(USART_TypeDef *inst); /* fine frame:
    CR3.DMAR off, RXNEIE on */
```

Disables UART DMA reception and re-enables the RX interrupt to switch from DMA-based to interrupt-based reception.

Parametri:

Nome	Tipo	Descrizione
inst	USART_TypeDef *	Pointer to the STM32 UART peripheral instance (e.g., USART1, USART2). Must be a valid, initialized peripheral base address.

Ritorna:
void

Note:

- This function is non-blocking and executes directly on the hardware registers. It is safe to call from an ISR because it performs simple bitwise operations on the UART control registers without using OS primitives like mutexes or semaphores. The primary purpose is to transition the UART state machine: by clearing the DMAR bit, the DMA controller stops requesting data transfers, and by setting the RXNEIE bit, the CPU is notified via interrupt whenever a new byte arrives. A common mistake is calling this while a DMA transfer is still active without ensuring the DMA channel is properly stopped or synchronized, which can lead to missed data or unexpected interrupt behavior.

Uso:

- This function is typically used in the UART driver's DMA completion callback or when a timeout occurs during a reception frame. It is part of the 'header-to-payload' transition logic: the system uses DMA to efficiently capture a burst of data, and once the frame is complete, it switches to interrupt mode to handle individual bytes or to wait for the next start-of-frame. It is usually paired with functions that configure the UART hardware and DMA channels, such as ca_uart_init_dma or internal DMA management routines.

Esempio:

```
/* Example: Handling the end of a DMA reception frame in an ISR context */
void my_uart_rx_isr_callback(void *arg)
{
    USART_TypeDef *uart_inst = (USART_TypeDef *)arg;

    /* 1. Stop DMA hardware to prevent further automatic transfers */
    ca_uart_hw_dma_rx_off(uart_inst);

    /* 2. Signal the processing task that a frame has been received */
    /* Assuming 'rx_sem' is a semaphore initialized elsewhere */
    ca_sem_post(&rx_sem);

    /* 3. The task will later re-enable DMA using the appropriate driver
    function */
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_uart_hw_dma_rx_off in my caOS application?
A: You should call this function when transitioning from DMA-based reception to interrupt-based reception, typically in a DMA completion callback or upon a reception timeout. It is used to stop the DMA controller from requesting transfers and to enable the RX interrupt to handle individual bytes.

Code example conf:0.90 **Q:** How do I implement a transition from DMA to interrupt mode using caOS types?
A: You can use the following snippet: void handle_frame_end(USART_TypeDef *uart_inst) { ca_uart_hw_dma_rx_off(uart_inst); }

Parameters conf:1.00 **Q:** What are the requirements for the 'inst' parameter in ca_uart_hw_dma_rx_off?
A: The 'inst' parameter must be a pointer to a valid, initialized STM32 UART peripheral instance (e.g., USART1, USART2). It must be a valid peripheral base address.

HW differences conf:1.00 **Q:** Are there any differences in how ca_uart_hw_dma_rx_off is implemented across different STM32 families?
A: Yes, the implementation varies based on the register definitions for the RX interrupt enable bit. While F1xx, F4xx, and F7xx use the USART_CR1_RXNEIE bit, newer families like H5xx, H7xx, and U5xx use the USART_CR1_RXNEIE_RXFNEIE bit to enable the interrupt.

---

ca_uart_hw_dma_rx_on

Header: caos/drivers/inc/ca_glue_hw.h:312
GUARD: EN_CA_GLUE_HW

```
extern void ca_uart_hw_dma_rx_on(USART_TypeDef *inst); /* fase payload:
    RXNEIE off, CR3.DMAR on */
```

Enables UART DMA reception by disabling interrupt-based RX and activating the DMA request signal.

Parametri:

Nome	Tipo	Descrizione
inst	USART_TypeDef *	Pointer to the STM32 UART peripheral instance (e.g., USART1, USART2). Must be a valid, initialized hardware base address.

Ritorna:
void

Note:

- This function is non-blocking and executes immediately by modifying hardware registers. It is thread-safe only if the UART peripheral is not being accessed by other tasks or ISRs simultaneously. It is safe to call from an ISR, but it is typically used in the setup phase of a DMA transfer. The most common mistake is calling this function before the DMA channel has been properly configured and enabled, which can lead to immediate DMA errors or missed data. Ensure the UART peripheral clock is enabled and the GPIO pins are configured before calling this.

Uso:

- This function is used to transition the UART from a standard interrupt-driven mode to a high-performance DMA-driven mode. It is typically called immediately after configuring the DMA channel with ca_dma_copy_target to ensure the hardware is ready to stream incoming bytes directly into memory. It is part of the standard initialization sequence: 1) Configure DMA, 2) Call ca_uart_hw_dma_rx_on, 3) Wait for completion using ca_uart_dma_wait.

Esempio:

```
/* Example: Setting up a DMA buffer for UART reception */
#define RX_BUFFER_SIZE 64
u8 rx_buf[RX_BUFFER_SIZE];

void start_uart_dma_reception(USART_TypeDef *inst, u32 uart_num) {
    /* 1. Configure the DMA channel to move data from UART to memory */
    /* Note: ca_uart_hw_dma_rx_addr(inst) is a helper to get the data register
    address */
    if (ca_dma_copy_target(1, rx_buf, (void*)&inst->RDR, RX_BUFFER_SIZE, 0, 0))
    {
```

```
        /* 2. Switch UART to DMA mode: disable RX interrupts and enable DMA
    request */
        ca_uart_hw_dma_rx_on(inst);
    } else {
        /* Handle error: DMA channel busy or invalid */
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_uart_hw_dma_rx_on in my initialization sequence?
A: You should call it after configuring the DMA channel (e.g., using ca_dma_copy_target) but before waiting for completion with ca_uart_dma_wait. It is used to transition the UART from standard interrupt-driven mode to high-performance DMA-driven mode.

Code example conf:1.00 **Q:** Provide a minimal example of how to enable DMA reception for USART1.
A: void setup_uart_dma(void) { /* Assume DMA is already configured */ ca_uart_hw_dma_rx_on(U
}

Parameters conf:1.00 **Q:** What does the inst parameter represent and what are the requirements for it?
A: The inst parameter is a pointer to the STM32 UART peripheral instance (e.g., USART1, USART2). It must be a valid, initialized hardware base address. You must ensure the UART peripheral clock is enabled and the GPIO pins are configured before passing the pointer to this function.

HW differences conf:1.00 **Q:** Are there any differences in how ca_uart_hw_dma_rx_on handles register bits across different STM32 families?
A: Yes. For F1xx, F4xx, and F7xx families, the function clears the USART_CR1_RXNEIE bit. However, for H5xx, H7xx, and U5xx families, the function clears the USART_CR1_RXNEIE_RXFI bit instead, reflecting differences in the register definitions for those newer architectures.

`ca_uart_hw_dma_tx_addr`

Header: `caos/drivers/inc/ca_glue_hw.h:310`
GUARD: `EN_CA_GLUE_HW`

```
extern volatile void *ca_uart_hw_dma_tx_addr(USART_TypeDef *inst); /*
    indirizzo registro dati TX per il DMA (DR/TDR) */
```

Returns the memory address of the UART Transmit Data Register (TDR/DR) for DMA operations.

Parametri:

Nome	Tipo	Descrizione
`inst`	`USART_TypeDef *`	Pointer to the specific UART hardware instance (e.g., USART1, USART2). Must be a valid, initialized peripheral pointer.

Ritorna:
A volatile void pointer to the hardware register where data must be written for transmission. Returns NULL if the provided instance pointer is invalid.

Note:

- This function is a low-level hardware abstraction layer (HAL) helper. It is non-blocking, thread-safe, and can be called from an ISR because it performs a simple pointer arithmetic operation based on the architecture. It is strictly intended to provide the destination

address for DMA controllers. The most common mistake is passing a NULL or uninitialized peripheral pointer, which will cause a hard fault when the DMA controller attempts to access the address.

Uso:

- This function is used exclusively when configuring DMA transfers for UART transmission. It is typically paired with ca_dma_copy_target to link a memory buffer to the UART hardware peripheral. The sequence is: first, identify the UART instance; second, retrieve the TDR address using this function; third, pass that address to the DMA configuration function to initiate the hardware-managed data transfer.

Esempio:

```c
/* Example: Setting up a DMA transfer for UART1 */
#include "caos.h"

void send_data_via_dma(USART_TypeDef *uart_inst, ca_dma_ch_t tx_channel, uint8_t *data, uint16_t size) {
    /* Get the hardware register address for the UART TDR */
    volatile void *tx_reg_addr = ca_uart_hw_dma_tx_addr(uart_inst);

    if (tx_reg_addr != NULL) {
        /* Configure DMA to copy data from memory to the UART TDR register */
        if (ca_dma_copy_target(tx_channel, (void *)data, (void *)tx_reg_addr, size, CA_DMA_DIR_M2P, CA_DMA_WIDTH_8)) {
            /* Wait for the DMA transfer to complete before proceeding */
            ca_dma_wait(tx_channel);
        }
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_uart_hw_dma_tx_addr function in my caOS application?
A: You should use this function exclusively when configuring DMA transfers for UART transmission. It is intended to provide the destination address for the DMA controller, typically by being paired with ca_dma_copy_target to link a memory buffer to the UART peripheral.

Code example conf:1.00 **Q:** How do I retrieve the UART transmit register address for a DMA transfer?
A: You pass the UART instance pointer to the function as shown: volatile void *tx_addr = ca_uart_hw_dma_tx_addr(USART1);. Ensure the instance pointer is initialized and valid to avoid a hard fault.

Parameters conf:1.00 **Q:** What are the requirements for the 'inst' parameter in ca_uart_hw_dma_tx_addr?
A: The 'inst' parameter must be a pointer to a valid, initialized USART_TypeDef peripheral instance (e.g., USART1, USART2). Passing a NULL or uninitialized pointer is a common mistake that will cause a hard fault when the DMA controller attempts to access the returned address.

HW differences conf:1.00 **Q:** Are there differences in how this function handles register access across different STM32 families?
A: Yes, the function abstracts hardware-specific register naming. For F1xx and F4xx families, the function returns the address of the 'DR' register. For F7xx, H5xx, H7xx, and U5xx families, the function returns the address of the 'TDR' register.

ca_uart_hw_init

Header: caos/drivers/inc/ca_glue_hw.h:302
GUARD: EN_CA_GLUE_HW

```
extern bool ca_uart_hw_init(u32 uart_num, u32 baudrate, USART_TypeDef **
    _instance); /* inizializza il periferico UART al baudrate indicato */
```

Initializes a specific UART hardware peripheral, configures its baud rate, and enables its RX interrupt.

Parametri:

Nome	Tipo	Descrizione
uart_num	u32	The hardware index of the UART peripheral (e.g., 1 for USART1, 2 for USART2). Must match the hardware mapping of the specific STM32 series.
baudrate	u32	The desired communication speed in bits per second (e.g., 115200).
_instance	USART_TypeDef**	A pointer to a pointer that will be updated with the base address of the initialized hardware register block. Must not be NULL.

Ritorna:
Returns true if the peripheral was successfully identified, clocked, and configured. Returns false if the uart_num is invalid or the hardware instance cannot be resolved.

Note:

- This function is not thread-safe and should be called during system initialization before starting the OS scheduler. It is not intended to be called from an ISR. It performs low-level register manipulation, including enabling peripheral clocks in the RCC and configuring the NVIC for RX interrupts. A common mistake is passing an uninitialized pointer for _instance or attempting to re-initialize an already active UART without first ensuring no other tasks are using it. Note that this function does not configure GPIO pins; you must call ca_uart_configure_gpio separately to map the pins to the UART alternate function.

Uso:

- This function is the primary entry point for setting up serial communication. It is typically used in the board support package (BSP) initialization sequence. It is often paired with ca_uart_configure_gpio to set up the physical pins and ca_uart_retune if the system clock frequency changes dynamically. Once initialized, the returned instance pointer is used by other ca_uart_* functions to perform data transfers.

Esempio:

```
/* Example: Initializing USART2 for serial communication */
USART_TypeDef *my_uart = NULL;

// 1. Configure the physical pins for USART2 (e.g., PA2/PA3)
ca_uart_configure_gpio(2);

// 2. Initialize the hardware peripheral
if (ca_uart_hw_init(2, 115200, &my_uart)) {
    // Success: my_uart now points to the USART2 register base
    ca_printf("UART2 initialized successfully.\n");
} else {
    // Handle error: invalid UART number or hardware failure
    ca_printf("Failed to initialize UART2.\n");
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_uart_hw_init() and why is it necessary?
A: You should call ca_uart_hw_init() during system initialization to configure a UART peripheral for communication. It's necessary because it sets up the hardware registers, enables the clock, configures the baud rate, and prepares the UART for data transmission and reception.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_uart_hw_init() with caOS types?
A:

```
USART_TypeDef* uart;
bool success = ca_uart_hw_init(1, 115200, &uart);
if (success) {
    // UART initialized successfully
    uart->TDR = 'A'; // Send a character
}
```

Parameters conf:0.85 **Q:** What does the 'uart_num' parameter represent and what are valid values?
A: The 'uart_num' parameter specifies which UART peripheral to initialize. Valid values are typically 1 through 8 depending on the MCU family. For example, on H7xx, valid values include 1, 2, 3, 4, 5, 6, 7, and 8, while on F7xx it may be limited to 1-6.

Return conf:0.90 **Q:** What does ca_uart_hw_init() return and what causes it to fail?
A: The function returns true on success and false on failure. Failures can occur if the uart_num is invalid, the hardware registers cannot be configured, or if the clock enabling fails.

Error/Edge case conf:0.80 **Q:** What are common mistakes or edge cases when using ca_uart_hw_init()?
A: Common mistakes include passing an invalid uart_num, not ensuring the system clock is properly configured before calling the function, or forgetting to enable the appropriate RCC clocks. Edge cases include attempting to initialize a UART that doesn't exist on the specific MCU or trying to initialize a UART that is already in use.

Behavior conf:0.85 **Q:** Is ca_uart_hw_init() blocking or non-blocking, and is it safe to call from an ISR?
A: The function is blocking as it performs hardware register writes and waits for configuration to complete. It is generally not ISR-safe because it modifies global hardware state and may involve system-level operations. It should be called from thread context during initialization.

HW differences conf:0.90 **Q:** How do the UART initialization functions differ between MCU families like F7xx, H7xx, and H7xx?
A: On F7xx, the function uses APB1ENR/APB2ENR for clock enabling and accesses standard USART registers. On H7xx, it uses APB1LENR/APB2ENR and includes a PRESC register for clock prescaling. Additionally, H7xx uses USART_CR1_RXNEIE_RXFNEIE for enabling RX interrupts, whereas F7xx uses USART_CR1_RXNEIE.

`ca_uart_hw_put`

Header: `caos/drivers/inc/ca_glue_hw.h:303`
GUARD: `EN_CA_GLUE_HW`

```
extern u16 ca_uart_hw_put(USART_TypeDef **_instance, p_str message); /*
    trasmette una stringa via UART in polling */
```

Transmits a string via UART using blocking polling mode.

Parametri:

Nome	Tipo	Descrizione
_instance	USART_TypeDef **	A pointer to the UART peripheral handle. Must not be NULL, and the dereferenced pointer must point to a valid initialized UART peripheral.
message	p_str	A pointer to a structure containing the data buffer and its size. Must not be NULL.

Ritorna:
Returns the number of bytes successfully sent (equal to message->size on success). Returns 0 if any input parameter is NULL or invalid.

Note:

- This function is strictly blocking; it waits in a 'while' loop for the hardware registers to be ready for each byte and for the final transmission to complete. Because it uses busy-waiting, it is NOT thread-safe if multiple tasks attempt to write to the same UART simultaneously, and it must NEVER be called from an ISR as it will cause a system hang. Ensure the UART peripheral has been initialized via ca_uart_init before calling this. A common mistake is passing a message structure with an uninitialized or zero-length buffer, which results in no data being sent.

Uso:

- This function is typically used for low-level debugging or simple serial communication where DMA is not required or available. It is often paired with ca_uart_configure_gpio to set up the pins before the first transmission. In the caOS architecture, it serves as the fallback transmission method when DMA resources are exhausted or when the message size is too small to justify the overhead of a DMA transfer.

Esempio:

```c
/* Assuming a message structure is already defined and populated */
extern USART_TypeDef *g_uart_instances[];

void send_debug_info(void) {
    p_str my_msg;
    /* Initialize message structure with data */
    my_msg.data = (u8 *)"System Ready\n";
    my_msg.size = 13;

    /* Ensure UART is configured before use */
    ca_uart_configure_gpio(1);

    /* Send the message via UART 1 in blocking mode */
    u16 sent = ca_uart_hw_put(&g_uart_instances[1], &my_msg);

    if (sent == 0) {
        /* Handle error: transmission failed or invalid parameters */
    }
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_uart_hw_put for UART transmission?
A: Use ca_uart_hw_put when you need to send a string via UART in a polling mode, especially during debugging or system configuration where DMA or interrupt-based transmission is not feasible. It is suitable for non-critical timing scenarios where blocking behavior is acceptable.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_uart_hw_put?
A:

```
USART_TypeDef* uart_instance = USART1;
char msg[] = "Hello UART\n";
p_str message = { .data = (u8*)msg, .size = 12 };
ca_uart_hw_put(&uart_instance, &message);
```

Parameters conf:0.95 **Q:** What are the valid and invalid values for the parameters of ca_uart_hw_put?
A: The first parameter _instance must be a valid pointer to a USART_TypeDef pointer, and the USART instance must not be NULL. The second parameter message must be a valid p_str structure with a non-NULL data field and a positive size. Passing NULL for either parameter results in a return value of 0.

Return conf:0.95 **Q:** What does ca_uart_hw_put return in case of success and failure?
A: On success, ca_uart_hw_put returns the number of characters sent (message->size). On failure, it returns 0, which occurs when either _instance or message is NULL, or when the message structure is invalid.

Error/Edge case conf:0.90 **Q:** What are the common mistakes or edge cases when using ca_uart_hw_put?
A: Common errors include passing NULL pointers for _instance or message, not properly initializing the UART peripheral before calling the function, and attempting to call this function from an ISR, which is not allowed. Additionally, incorrect handling of buffer sizes or timeouts may lead to unexpected behavior.

Behavior conf:0.95 **Q:** Is ca_uart_hw_put blocking or non-blocking, and can it be called from an ISR?
A: ca_uart_hw_put is a blocking function that waits until all characters are transmitted. It is not safe to call from an ISR because it is blocking and could cause deadlocks or excessive delays in interrupt contexts.

HW differences conf:0.85 **Q:** How does ca_uart_hw_put differ between F7xx and H7xx microcontrollers?
A: On F7xx, the function uses ISR register bits like USART_ISR_TXE and USART_ISR_TC, and writes to TDR register. On H7xx, it also uses ISR register bits like USART_ISR_TXE_TXFNF and USART_ISR_TC, but with different naming conventions and writes to TDR register. The register access methods are consistent across families but differ in naming and bit positions.

`ca_uart_hw_recv_handler`

Header: `caos/drivers/inc/ca_glue_hw.h:304`
GUARD: `EN_CA_GLUE_HW`

```
extern u32 ca_uart_hw_recv_handler(USART_TypeDef **_instance, u32 *
    rx_data); /* gestore IRQ UART: legge il byte ricevuto dal registro DR
     */
```

Low-level hardware handler to read a single byte from the UART receive register.

Parametri:

Nome	Tipo	Descrizione
`_instance`	`USART_TypeDef **`	Pointer to the UART peripheral instance handle. Must not be NULL and must point to a valid initialized UART peripheral.
`rx_data`	`u32 *`	Pointer to a memory location where the received byte will be stored. Must not be NULL.

Ritorna:
Returns 0 if a byte was successfully read from the hardware register. Returns 1 if no data is available (RXNE flag not set) or if input pointers are invalid.

Note:

- This function is designed to be called directly from an Interrupt Service Routine (ISR) or a high-priority polling loop. It is non-blocking and performs a direct register read, making it extremely fast but also dangerous if called without checking the hardware status first. It is not thread-safe by itself; it assumes the caller has already verified that the UART peripheral is initialized and that an interrupt or status flag indicates data is ready. A common mistake is calling this function without checking the UART status flags, which may result in reading stale or invalid data if the hardware buffer is empty.

Uso:

- This function is the lowest level of the UART receive chain. It is typically invoked by the UART interrupt handler (the ISR) when the hardware signals that a byte has arrived. After reading the byte, the ISR usually passes the data to higher-level functions like ca_cli_rx_byte for command processing or a queue-based buffer system to store the data for later consumption by a task. It is never used in isolation; it is always part of a sequence: check status -> call ca_uart_hw_recv_handler -> process byte.

Esempio:

```c
void USART1_IRQHandler(void) {
    u32 rx_byte = 0;
    /* Check if the UART instance is valid and attempt to read the byte */
    if (ca_uart_hw_recv_handler(&g_uart_instances[0], &rx_byte) == 0) {
        /* Successfully read, pass the byte to the CLI system */
        ca_cli_rx_byte((u8)rx_byte);
    } else {
        /* Handle error or spurious interrupt if necessary */
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_uart_hw_recv_handler in my UART interrupt service routine?

A: You should call ca_uart_hw_recv_handler from your UART ISR when you are certain that the RXNE (Receive Data Register Not Empty) flag is set. This function is designed to safely read a byte from the UART data register (DR/RDR) and is intended for use in interrupt contexts where hardware-driven UART reception occurs.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_uart_hw_recv_handler in a UART ISR?

A:

```c
u32 data;
USART_TypeDef* uart_instance = USART1;
if (ca_uart_hw_recv_handler(&uart_instance, &data) == 0) {
    // Process received byte in 'data'
}
```

Parameters conf:0.95 **Q:** What are the valid and invalid values for the _instance and rx_data parameters?

A: The _instance parameter must be a pointer to a valid USART_TypeDef*; passing NULL or a pointer to NULL will cause the function to return 1. The rx_data parameter must be a valid pointer to a u32 where the received byte will be stored; passing NULL will cause a crash.

Return conf:0.90 **Q:** What does it mean when ca_uart_hw_recv_handler returns 1?

A: A return value of 1 indicates either that no byte is available for reading (RXNE flag not set), or that one or more parameters passed to the function were invalid (e.g., _instance is NULL or points to NULL).

HW differences conf:0.85 **Q:** How does ca_uart_hw_recv_handler differ between F7xx and H7xx MCUs in terms of register usage?
A: On F7xx MCUs, the function checks the ISR register for the USART_ISR_RXNE flag and reads from the RDR register. On H7xx MCUs, it also checks the ISR register but uses the USART_ISR_RXNE_RXFNE flag and reads from the RDR register. The main difference is in the flag used to check for data availability.

`ca_uart_init`

Header: `caos/drivers/inc/uart.h:73`
Source: `caos/drivers/src/uart.c:108:190`
GUARD: `EN_UART`

```
bool ca_uart_init(u32 uart_num, u32 baudrate, ca_uart_rx_cb_t on_rx, u8
    alternate)
```

Initializes a hardware UART peripheral, configures its GPIO pins, and registers an optional RX callback.

Parametri:

Nome	Tipo	Descrizione
`uart_num`	`u32`	The hardware index of the UART peripheral (e.g., 1 for USART1). Must be within the range [1, CA_UART_MAX).
`baudrate`	`u32`	The desired communication speed in bits per second (e.g., 115200).
`on_rx`	`ca_uart_rx_cb_t`	A function pointer to be called when data is received. Pass NULL if no callback is required.
`alternate`	`u8`	The alternate function mapping index for the GPIO pins. This determines which physical pins are assigned to the UART.

Ritorna:
Returns true if the UART and GPIO were successfully initialized, false otherwise (e.g., invalid uart_num or hardware configuration failure).

Note:

- This function is not thread-safe and should be called during system initialization before starting the OS scheduler. It performs hardware-level configuration, including GPIO setup, which is blocking and relies on underlying hardware registers. Do not call this function from an ISR, as it may perform operations that are not ISR-safe. A common mistake is passing an invalid uart_num or failing to ensure the peripheral clock is enabled, which will result in a false return value. Additionally, the 'alternate' parameter is critical; providing the wrong value can cause the UART to drive pins that are already in use by other board peripherals, potentially causing electrical conflicts.

Uso:

- Use this function during the early boot phase to prepare communication interfaces. It is typically paired with other initialization functions like ca_os_tick_init. Once initialized, the UART can be used for data transmission or command-line interfaces. It is common to call this once at startup to set the hardware state, after which the driver maintains the configuration for subsequent use with functions like ca_uart_put or ca_uart_dma_send.

Esempio:

```c
void system_init(void) {
    /* Define a callback for UART7 RX events */
    void on_rx_uart7(u8 data) {
        /* Handle incoming byte */
    }

    /* Initialize UART7 at 115200 baud with the RX callback and alternate pin map 1 */
    if (ca_uart_init(7, 115200, on_rx_uart7, 1)) {
        /* UART initialized successfully */
    } else {
        /* Handle initialization error */
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_uart_init in a caOS project?
A: You should call ca_uart_init during system initialization, typically before starting any tasks that rely on UART communication. It is used to configure a specific UART peripheral with a given baudrate and an optional receive callback function.

Code example conf:0.95 **Q:** Can you show a minimal example of how to use ca_uart_init with caOS types?
A: u32 baudrate = 115200; bool success = ca_uart_init(1, baudrate, NULL); if (success) { // UART initialized successfully } else { // Handle initialization failure }

Parameters conf:0.90 **Q:** What are the valid values for uart_num in ca_uart_init?
A: The uart_num must be a value between 1 and CA_UART_MAX-1. Values 0 or greater than or equal to CA_UART_MAX will cause the function to fail. For example, if CA_UART_MAX is 4, valid values are 1, 2, and 3.

Return conf:0.95 **Q:** What does ca_uart_init return when it fails?
A: It returns false when initialization fails. This can happen if uart_num is invalid, or if there is a hardware initialization error.

Error/Edge case conf:0.90 **Q:** What are common mistakes when calling ca_uart_init?
A: Common mistakes include passing an invalid uart_num (0 or >= CA_UART_MAX), not configuring GPIOs with ca_uart_configure_gpio() before calling ca_uart_init, and using a baudrate that is not supported by the STM32 chip.

Behavior conf:0.90 **Q:** Is ca_uart_init thread-safe and can it be called from an ISR?
A: No, ca_uart_init is not thread-safe and must not be called from an ISR. It is a blocking function that should only be called from a task or main thread, and requires synchronization if called from multiple threads.

`ca_uart_init_dma`

Header: `caos/drivers/inc/uart.h:134`
Source: `caos/drivers/src/uart.c:295:335`
GUARD: `EN_UART`

```c
bool ca_uart_init_dma(u32 uart_num, u32 baudrate, ca_uart_rx_cb_t on_rx, u8 alternate)
```

Initializes a UART peripheral with DMA support for efficient, non-blocking data transmission and reception.

Parametri:

Nome	Tipo	Descrizione
`uart_num`	`u32`	The hardware identifier for the UART peripheral (e.g., 1, 2, 3). Must not be 0.
`baudrate`	`u32`	The communication speed in bits per second (e.g., 115200).
`on_rx`	`ca_uart_rx_cb_t`	A pointer to the callback function executed when data is received via DMA.
`alternate`	`u8`	Selects the alternate GPIO pin mapping for the UART peripheral. Use 0 for default mapping.

Ritorna:
Returns true if the UART and DMA channels were successfully initialized and bound; returns false if the UART is already in use, no DMA channels are available, or hardware initialization fails.

Note:

- This function is not thread-safe and should be called during system initialization before starting the scheduler. It is not callable from an ISR because it performs resource allocation and hardware configuration. A common mistake is failing to ensure that the DMA controller has been initialized via ca_dma_init() before calling this function, which will cause the internal binding process to fail. The function handles the complex task of mapping UART hardware requests to available DMA channels, so you do not need to manually configure DMA registers.

Uso:

- Use this function when you need high-performance serial communication that does not block the CPU during data transfers. It is typically called once during the system startup phase, immediately after ca_dma_init(). It is often paired with other DMA-based UART functions for actual data movement, and the provided callback will be triggered by the DMA interrupt once a transfer completes. Ensure the UART peripheral is not already initialized or in use by another task before calling this function.

Esempio:

```
void system_init(void) {
    // Initialize DMA controller first
    ca_dma_init();

    // Define a callback for received data
    // void my_rx_callback(void *data, u32 len) { ... }

    // Initialize UART1 with DMA at 115200 baud using default pins
    if (!ca_uart_init_dma(1, 115200, my_rx_callback, 0U)) {
        // Handle initialization error
        ca_printf("UART DMA init failed!\n");
    }
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_uart_init_dma instead of standard UART functions?
A: You should use ca_uart_init_dma when you require high-performance serial communication that does not block the CPU during data transfers. It is designed for non-blocking operations and is typically called once during the system startup phase.

Code example conf:0.90 **Q:** How do I initialize UART1 with DMA in caOS?
A: void my_rx_callback(u8* data, u32 len) { /* Handle data */ } void init(void) { ca_dma_init(); ca_uart_init_dma(1, 115200, my_rx_callback); }

Parameters conf:1.00 **Q:** What are the constraints on the uart_num parameter?
A: The uart_num parameter represents the hardware identifier for the UART peripheral. It must be a value greater than 0 (e.g., 1, 2, or 3).

Return conf:1.00 **Q:** What does a return value of false indicate for ca_uart_init_dma?
A: A return value of false indicates that the initialization failed. This occurs if the UART is already in use, no DMA channels are available, or the hardware initialization fails.

Error/Edge case conf:1.00 **Q:** What is a common mistake that causes ca_uart_init_dma to fail?
A: A common mistake is failing to ensure that the DMA controller has been initialized via ca_dma_init() before calling ca_uart_init_dma(), which causes the internal binding process to fail.

Behavior conf:1.00 **Q:** Is ca_uart_init_dma safe to call from an interrupt service routine or multiple threads?
A: No, this function is not thread-safe and is not callable from an ISR because it performs resource allocation and hardware configuration. It must be called during system initialization before starting the scheduler.

`ca_uart_put`

Header: `caos/drivers/inc/uart.h:81`
Source: `caos/drivers/src/uart.c:189:204`
GUARD: `EN_UART`

```
u16 ca_uart_put(u32 uart_num, p_str message)
```

Transmits a message buffer over a specified UART peripheral.

Parametri:

Nome	Tipo	Descrizione
`uart_num`	`u32`	The index of the UART peripheral to use. Must be a valid index within the range [1, CA_UART_MAX - 1]. Passing 0 or an index out of bounds will result in an immediate return of 0.
`message`	`p_str`	A pointer to the message structure containing the data to send. Must not be NULL. The structure must have a valid size field.

Ritorna:
Returns a u16 value representing the status or number of bytes processed. Returns 0 if the UART index is invalid, the UART instance is not initialized, or the message pointer is NULL.

Note:

- This function acts as a high-level abstraction layer that automatically chooses between DMA-based transmission (if EN_UART_DMA is enabled and a DMA channel is found) or standard hardware-based transmission. It is not explicitly documented as thread-safe for concurrent access to the same UART instance, so ensure that multiple tasks do not attempt to write to the same UART simultaneously without external synchronization. It is generally not recommended to call this from an ISR unless the underlying driver implementation explicitly supports it, as it may involve blocking operations or resource contention. The most common beginner mistake is passing an uninitialized UART index or a NULL message pointer, which causes the function to fail silently by returning 0.

Uso:

- This function is typically used for sending formatted data, logs, or communication packets to external devices or debug consoles. In a standard workflow, you would first initialize the UART peripheral using ca_uart_init, then prepare your data in a buffer, and finally call ca_uart_put to transmit it. It is often paired with ca_uart_dbg_put when you want to send data specifically to the system's designated debug UART. It is frequently used in communication loops where a message is constructed, its size is set, and then it is passed to this function for transmission.

Esempio:

```c
/* Example: Sending a status message over UART */
#include "caos.h"

void send_status_update(void) {
    p_str msg;
    /* Allocate memory for the message structure */
    msg = ca_memory_allocate(sizeof(struct_p_str));
    if (msg == NULL) return;

    /* Prepare the message content */
    /* Assuming 'data_buffer' is a pre-filled array */
    msg->data = data_buffer;
    msg->size = 10;

    /* Send the message via UART 1 */
    u16 result = ca_uart_put(1, msg);

    if (result == 0) {
        /* Handle error: transmission failed or invalid UART */
    }

    /* Free the allocated message structure */
    ca_memory_free(msg);
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use the ca_uart_put function in a caOS application?
A: You should use ca_uart_put to send a string message via a specific UART interface in a caOS application. It is typically used for debugging, logging, or communication with external devices. The function is blocking, so it waits until the entire message is sent, making it suitable for scenarios where you need to ensure message delivery before proceeding.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_uart_put with caOS types?
A: u16 result = ca_uart_put(1, "Hello, caOS!\n"); if (result == 0) { /* Handle error */ } else { /* Success: sent result bytes */ }

Parameters conf:0.90 **Q:** What are the valid values for uart_num and what happens if I pass an invalid value?
A: The uart_num must be greater than 0 and less than CA_UART_MAX. If uart_num is invalid or if g_uart_instances[uart_num] is NULL, the function will return 0. The message parameter must not be NULL; otherwise, it also returns 0.

Return conf:0.95 **Q:** What does the return value of ca_uart_put indicate, and what causes it to return 0?
A: The function returns the number of bytes successfully sent if successful. It returns 0 in case of error, which occurs when uart_num is invalid, g_uart_instances[uart_num] is NULL, or when message is NULL.

ca_uart_recv_handler

Header: caos/drivers/inc/uart.h:94
Source: caos/drivers/src/uart.c:206:223
GUARD: EN_UART

```
void ca_uart_recv_handler(u32 uart_num)
```

Processes incoming UART data by routing it to the CLI or a registered callback function.

Parametri:

Nome	Tipo	Descrizione
uart_num	u32	The index of the UART peripheral. Must be a valid index within [1, CA_UART_MAX-1]. Values outside this range or 0 are ignored.

Ritorna:
void

Note:

- This function is designed to be called from within a UART hardware interrupt service routine (ISR). It is not thread-safe in the sense that it relies on the hardware state being consistent at the time of the interrupt. The function performs a non-blocking check on the hardware buffer; if data is available, it either passes it to the CLI (if the UART is configured for debugging) or triggers a user-defined callback hook. A common mistake is failing to initialize the UART instance or the callback hook before the interrupt fires, which results in the function silently returning without processing data.

Uso:

- This function is the core of the UART reception pipeline in caOS. It is typically invoked by the low-level hardware interrupt vector when a 'RXNE' (Receive Register Not Empty) event occurs. It pairs with ca_uart_init to set up the hardware and ca_cli_set_hook or a custom hook registration to define what happens to the received bytes. It acts as the bridge between raw hardware interrupts and the application-level logic.

Esempio:

```
/* Example of setting up a custom callback for UART 1 */

void my_rx_callback(u32 data) {
    /* Process the received byte */
    ca_printf("Received: %c\n", (char)data);
}

void setup_uart_processing(void) {
    /* Initialize UART hardware first */
    /* ca_uart_init(1, ...); */

    /* Register the callback to be triggered by ca_uart_recv_handler */
    /* hook_on_rx[1] = my_rx_callback; */
}

/* Inside the actual hardware interrupt handler for UART 1 */
void USART1_IRQHandler(void) {
    /* This function is called by the hardware when data arrives */
    ca_uart_recv_handler(1);
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_uart_recv_handler in my embedded application?
A: You should call ca_uart_recv_handler only from an interrupt service routine (ISR) when a byte is received on a UART. It is not meant to be called directly from a thread or main application code. The function is designed to inject received bytes into either a CLI or a custom user function, depending on how hook_on_rx is initialized.

Code example conf:0.90 **Q:** Can you provide a minimal example of how to use ca_uart_recv_handler with caOS types?
A:

```
u32 uart_num = 0;
ca_uart_recv_handler(uart_num);
```

Parameters conf:0.90 **Q:** What is the meaning of the uart_num parameter, and what are valid values?
A: The uart_num parameter specifies the UART number from which the byte was received. It must be a valid value between 0 and the maximum number of UARTs available. Using an invalid value may lead to undefined behavior such as memory access violations or incorrect handling of the received byte.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_uart_recv_handler?
A: Common mistakes include calling this function directly from a thread instead of an ISR, which can cause undefined behavior. Additionally, if hook_on_rx is not properly initialized, the function may ignore the received byte. Using an invalid uart_num value can also lead to memory access violations or unpredictable behavior.

Behavior conf:0.90 **Q:** Is ca_uart_recv_handler blocking or non-blocking, and is it safe to use in a thread context?
A: ca_uart_recv_handler is non-blocking and is designed to be called from an ISR. It is not thread-safe because it modifies global variables and should not be invoked directly from a thread. It is intended to be fast and avoid any blocking operations.

`ca_uart_retune`

Header: `caos/drivers/inc/uart.h:73`
Source: `caos/drivers/src/uart.c:171:177`
GUARD: `EN_UART`

```
void ca_uart_retune(void)
```

Re-initializes all active UART peripherals to match the current system clock frequency.

Ritorna:
This function returns void and does not provide a status code.

Note:

- This function is intended to be called whenever the system clock configuration changes, such as during power-saving frequency scaling. It iterates through all registered UART instances and re-applies their baud rate settings using the current clock parameters. It is not thread-safe and should not be called from an ISR, as it performs hardware initialization which may involve blocking operations or shared resource access. A common mistake is calling this function before the UART hardware has been initialized at least once, which may lead to undefined behavior if the internal instance pointers are NULL.

Uso:

- This function is typically used within power management routines, specifically when the CPU frequency is scaled up or down to save energy. It is usually paired with ca_os_clock_reduce_freq

or ca_os_clock_restore_freq to ensure that serial communication remains stable after the peripheral clock source changes. In a typical sequence, you would call ca_os_clock_reduce_freq to change the system speed, followed by ca_uart_retune to update the UART baud rate generators, and finally ca_os_tick_retune to adjust the OS heartbeat.

Esempio:

```
void adjust_system_performance(u16 scale_factor) {
    /* Reduce system clock frequency to save power */
    ca_os_clock_reduce_freq(scale_factor);

    /* Update OS tick timing to match new clock */
    ca_os_tick_retune();

    /* Re-calculate and apply baud rates for all active UARTs */
    ca_uart_retune();

    /* If CAN bus is used, update its timing as well */
    ca_can_retune();
}
```

Q&A — fine-tuning review *2/2 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_uart_retune in my application?
A: You should call ca_uart_retune whenever the system clock configuration changes, such as during power-saving frequency scaling. It is typically used in power management routines after calling ca_os_clock_reduce_freq or ca_os_clock_restore_freq to ensure serial communication remains stable after the peripheral clock source changes.

Code example conf:0.95 **Q:** How do I implement a frequency scaling routine using ca_uart_retune?
A: To scale the system frequency, you must update the clock and then re-tune the peripherals. Example: { ca_os_clock_reduce_freq(); ca_uart_retune(); ca_os_tick_retune(); } Note that this function is not thread-safe and must not be called from an ISR.

`ca_udp_client_connect`

Header: `caos/net/inc/udp_client.h:80`
Source: `caos/net/src/udp_client.c:183:229`
GUARD: `EN_UDP_CLIENT`

```
u8 ca_udp_client_connect(const ca_udp_client_cfg_t *cfg)
```

Initializes and registers a new UDP client instance to enable network communication.

Parametri:

Nome	Tipo	Descrizione
`cfg`	`const ca_udp_client_cfg_t *`	Pointer to a configuration structure containing server IP, ports, and callback functions. Must not be NULL.

Ritorna:
Returns a 1-based handle (1 to UDPCLI_MAX_HANDLES) on success, or 0 if the connection fails or no slots are available.

Note:

- This function is thread-safe as it uses an internal mutex to manage slot allocation. It is NOT callable from an ISR because it performs blocking operations like mutex locking and potential network resolution. You must ensure the network stack is initialized before

calling this. A common mistake is failing to check the return value; if it returns 0, the client was not created, and subsequent calls to send or receive using that handle will fail.

Uso:

- This function is the entry point for UDP communication. It is typically called during system initialization or when a task needs to establish a connection to a remote server. It is used in a sequence: first, populate the configuration struct, then call this function to get a handle, use the handle for ca_udp_client_send and ca_udp_client_recv, and finally call ca_udp_client_disconnect to release the slot when finished.

Esempio:

```
ca_udp_client_cfg_t cfg;
ca_udp_client_cfg_t *my_client;

/* Initialize configuration struct */
ca_memzero(&cfg, sizeof(cfg));
cfg.server_ip = 0x0A000001; // Example IP: 10.0.0.1
cfg.server_port = 8080;
cfg.local_port = 0; // Let system assign ephemeral port

/* Attempt to connect */
u8 handle = ca_udp_client_connect(&cfg);

if (handle > 0) {
    // Connection successful, use 'handle' for communication
    // ca_udp_client_send(handle, data, len);
} else {
    // Handle error: log failure or retry
    ca_printf("Failed to connect UDP client\n");
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_udp_client_connect and what is its purpose in a caOS application?
A: You should call ca_udp_client_connect after initializing the network stack with cli_init_once to create a UDP client that connects to a remote server. It is used to establish a UDP communication channel with a server by providing configuration details such as server IP, port, and local network settings. The function returns an handle that can be used to manage the client.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_udp_client_connect with caOS types?
A: ca_udp_client_cfg_t cfg; cfg.server_ip = 0xC0A80101U; // 192.168.1.1 cfg.server_port = 12345U; cfg.local_ip = 0xC0A80102U; // 192.168.1.2 cfg.local_mac = 0x123456789ABCULL; cfg.arp_timeout_ms = 1000U; cfg.rx_callback = NULL; u8 handle = ca_udp_client_connect(&cfg); if (handle > 0U) { // Client created successfully with handle = handle } else { // Failed to create client }

Parameters conf:0.90 **Q:** What does the cfg parameter in ca_udp_client_connect represent and what are valid values?
A: The cfg parameter is a pointer to a ca_udp_client_cfg_t structure that holds configuration details for the UDP client. It must not be NULL. Valid values include a valid server IP address, a valid server port, correct local IP and MAC addresses, and a valid ARP timeout value. The structure must be initialized before calling the function.

Return conf:0.95 **Q:** What do the return values of ca_udp_client_connect mean and what causes a failure?
A: A return value of 0U indicates failure due to NULL config pointer, no free slots, bind failure, or ARP resolution failure. Return values from 1U to UDPCLI_MAX_HANDLES

indicate success, with the value being the handle of the created UDP client. The handle is 1-based, so the first client has handle = 1.

Error/Edge case conf:0.90 **Q:** What are common error conditions or mistakes when using ca_udp_client_connect?

A: Common errors include passing a NULL pointer to cfg, not having available slots for a new client, ARP resolution timeout, bind failure, or incorrect IP/mac address configuration. These can lead to a return value of 0U, indicating failure to create the UDP client.

Behavior conf:0.95 **Q:** Is ca_udp_client_connect blocking or non-blocking, and is it safe to call from an ISR?

A: ca_udp_client_connect is a blocking function because it waits for mutex acquisition and ARP resolution (if enabled). It is not safe to call from an ISR because it is not ISR-safe and uses mutexes, which are not suitable for interrupt context.

`ca_udp_client_disconnect`

Header: `caos/net/inc/udp_client.h:100`
Source: `caos/net/src/udp_client.c:275:290`
GUARD: `EN_UDP_CLIENT`

```
1 void ca_udp_client_disconnect(u8 handle)
```

Terminates an active UDP client session, releases associated network buffers, and unbinds the local port.

Parametri:

Nome	Tipo	Descrizione
`handle`	`u8`	The unique identifier returned by ca_udp_client_connect. Valid range is 1 to UDPCLI_MAX_HANDLES. Passing 0 or an out-of-bounds value will result in a silent failure.

Ritorna:
void

Note:

- This function is thread-safe as it uses an internal mutex to protect the client slot state. It is a blocking operation because it must acquire the mutex and potentially iterate through pending network buffers to free them. It must NOT be called from an ISR (Interrupt Service Routine) because blocking calls are forbidden in interrupt context. A common mistake is to call this function without having previously initialized the UDP client system or using an invalid handle, which leads to memory leaks if buffers are not properly cleaned up or undefined behavior if the handle points to an uninitialized slot.

Uso:

- This function is used to gracefully shut down a UDP client when communication is no longer needed or when a connection error occurs. It is typically the final step in the lifecycle of a UDP client: ca_udp_client_connect -> ca_udp_client_recv/send -> ca_udp_client_disconnect. It ensures that the local port is released back to the system and that any memory allocated for queued packets is returned to the heap, preventing memory exhaustion.

Esempio:

```c
u8 my_udp_handle = 0;

// Initialize connection
my_udp_handle = ca_udp_client_connect(12345, 0x0A000001, 8080);

if (my_udp_handle != 0) {
    // ... perform network operations ...

    // Disconnect and clean up resources
    ca_udp_client_disconnect(my_udp_handle);
    my_udp_handle = 0;
} else {
    // Handle connection failure
    ca_printf("Failed to connect UDP client\n");
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_udp_client_disconnect() in my UDP client application?
A: You should call ca_udp_client_disconnect() when you want to close a UDP client connection and free up the associated local port. This is typically done when the client is no longer needed, such as after completing a communication session or when shutting down the application.

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_udp_client_disconnect() with caOS types?
A: u8 handle = 1; ca_udp_client_disconnect(handle);

Parameters conf:0.90 **Q:** What is the valid range for the handle parameter in ca_udp_client_disconnect()?
A: The handle parameter must be an unsigned 8-bit integer (u8) in the range [1, UDPCLI_MAX_HANDLES], inclusive. If handle is 0 or greater than UDPCLI_MAX_HANDLES, the function does nothing.

Error/Edge case conf:0.95 **Q:** What happens if I call ca_udp_client_disconnect() with an invalid handle value?
A: If the handle is 0 or greater than UDPCLI_MAX_HANDLES, the function will ignore the input and perform no operation. It does not return an error code or throw an exception.

Behavior conf:0.85 **Q:** Is ca_udp_client_disconnect() thread-safe and ISR-safe?
A: The function is not thread-safe if called from an ISR. It is also blocking, as it may wait to acquire a mutex. It is not safe to call this function from an ISR, and it should only be used when the handle has been correctly allocated.

`ca_udp_client_recv`

Header: `caos/net/inc/udp_client.h:93`
Source: `caos/net/src/udp_client.c:244:273`
GUARD: `EN_UDP_CLIENT`

```c
bool ca_udp_client_recv(u8 handle, u8 *buf, u16 buf_len, u16 *out_len,
    u32 timeout_ms)
```

Blocks the calling task until a UDP packet is received on the specified client handle or the timeout expires.

Parametri:

Nome	Tipo	Descrizione
handle	u8	The unique ID of the UDP client session. Must be between 1 and UDPCLI_MAX_HANDLES.
buf	u8 *	Pointer to the destination buffer where the received payload will be copied.
buf_len	u16	Size of the destination buffer in bytes.
out_len	u16 *	Pointer to store the actual number of bytes copied. Can be NULL if not needed.
timeout_ms	u32	Maximum time to wait in milliseconds. Use 0 for no wait or a large value for indefinite blocking.

Ritorna:
Returns true if a packet was successfully received and copied to the buffer. Returns false if the handle is invalid, the client is inactive, the operation timed out, or the internal buffer is invalid.

Note:

- This function is blocking; it puts the calling task into a waiting state until data arrives or the timeout occurs. It is NOT safe to call from an ISR because it performs blocking operations (semaphores and mutexes). Ensure the UDP client has been initialized and connected via ca_udp_client_connect before calling this. A common mistake is passing a buffer that is too small, which will result in truncated data without an error flag, or failing to check the return value, leading to processing uninitialized data.

Uso:

- This function is typically used in a dedicated network task loop to continuously poll for incoming UDP data. It is usually paired with ca_udp_client_connect to establish the session and ca_udp_client_send to transmit data. After receiving, the application processes the buffer and then waits for the next packet. It is the standard way to implement a request-response pattern where the client waits for a server reply.

Esempio:

```
u8 my_handle = 1; // Assume handle 1 is already connected
u8 rx_buffer[128];
u16 received_bytes = 0;

// Wait up to 500ms for a response
if (ca_udp_client_recv(my_handle, rx_buffer, sizeof(rx_buffer), &received_bytes,
    500)) {
    // Successfully received data
    ca_printf("Received %d bytes\n", received_bytes);
} else {
    // Handle timeout or error
    ca_printf("No data received or error occurred\n");
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_udp_client_recv in a caOS application?
A: You should use ca_udp_client_recv when you need to receive data from a UDP client in a caOS application. It is designed for blocking reception of UDP data with configurable timeout, making it suitable for applications where you want to wait for incoming packets without busy-waiting.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_udp_client_recv?
A: u8 handle = 1; u8 buffer[1024]; u16 received_len; bool result = ca_udp_client_recv(handle, buffer, sizeof(buffer), &received_len, 1000); if (result) { /* Process received data */ }

Parameters conf:0.95 **Q:** What are the valid ranges for the handle parameter in ca_udp_client_recv?

A: The handle must be between 1 and UDPCLI_MAX_HANDLES inclusive. If handle is 0 or greater than UDPCLI_MAX_HANDLES, the function returns false.

Return conf:0.90 **Q:** What does it mean when ca_udp_client_recv returns false?
A: ca_udp_client_recv returns false if there's an error such as an invalid handle, a NULL buffer, a timeout occurred, or if the client is not active. It also returns false if the client is in 'on_rx' mode or if the function is called from an ISR.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_udp_client_recv?
A: Common mistakes include passing a handle of 0 or greater than UDPCLI_MAX_HANDLES, passing a NULL buffer, not checking the return value when timeout_ms is 0, and calling the function from an ISR. Also, not handling the case where out_len is NULL can lead to errors.

Behavior conf:0.90 **Q:** Is ca_udp_client_recv thread-safe and can it be called from an ISR?
A: Yes, ca_udp_client_recv is thread-safe because it uses mutexes to protect shared data structures. However, it is NOT safe to call from an ISR because it uses semaphores and mutexes that may cause issues in interrupt contexts.

`ca_udp_client_refresh_mac`

Header: `caos/net/inc/udp_client.h:106`
Source: `caos/net/src/udp_client.c:292:315`
GUARD: `EN_UDP_CLIENT`

```
bool ca_udp_client_refresh_mac(u8 handle, u32 arp_timeout_ms)
```

Forces an update of the cached MAC address for a specific UDP client handle by performing an ARP resolution.

Parametri:

Nome	Tipo	Descrizione
`handle`	`u8`	The identifier of the UDP client slot. Must be between 1 and UDPCLI_MAX_HANDLES. Values outside this range will cause the function to return false.
`arp_timeout_ms`	`u32`	Maximum time in milliseconds to wait for the ARP resolution to complete. If the network is slow or the target is unreachable, the function will block for up to this duration.

Ritorna:
Returns true if the MAC address was successfully resolved and updated in the client slot. Returns false if the handle is invalid, the client is not active, or the ARP resolution failed (e.g., timeout reached).

Note:

- This function is blocking because it waits for the network stack to resolve the IP address via ARP. It is thread-safe as it uses internal mutexes to protect the client slot data, but it must NOT be called from an ISR (Interrupt Service Routine) because blocking operations are forbidden in ISR context. A common mistake is to call this function with an extremely short timeout, which will cause it to fail frequently on busy networks. Ensure the UDP client has been initialized and is in an active state before calling this.

Uso:

- This function is typically used when a UDP client detects that communication with the server has failed or when the network topology has changed, requiring a fresh ARP lookup.

It is usually called after a failed 'ca_udp_client_send' attempt. The typical sequence is: 1) Check if the client is active, 2) Call 'ca_udp_client_refresh_mac' to update the ARP cache, 3) Retry the network operation if the refresh succeeds.

Esempio:

```
u8 my_handle = 1; // Assume handle 1 is already initialized
u32 timeout = 500; // 500ms timeout

// Attempt to refresh the MAC address before sending data
if (ca_udp_client_refresh_mac(my_handle, timeout)) {
    // MAC updated successfully, safe to proceed with communication
    // ca_udp_client_send(my_handle, data, len);
} else {
    // Handle error: log failure or notify system
    ca_printf("Failed to refresh MAC for client %d\n", my_handle);
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_udp_client_refresh_mac() in my UDP client application?
A: You should call ca_udp_client_refresh_mac() when you need to update the MAC address of a UDP server, especially after a network reconnection or when verifying the server's presence on the network. It is useful in scenarios where the server's MAC address might have changed or become stale.

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_udp_client_refresh_mac()?
A: u8 handle = 1; bool success = ca_udp_client_refresh_mac(handle, 1000); if (success) { // MAC address updated successfully } else { // Handle failure }

Parameters conf:0.90 **Q:** What does the arp_timeout_ms parameter control in ca_udp_client_refresh_mac()?
A: The arp_timeout_ms parameter sets the timeout in milliseconds for the ARP request. If set to 0, the function uses a default timeout value. Valid values are any positive integer, including 0, but the function will fail if the handle is invalid or the client is inactive.

Return conf:0.95 **Q:** What does it mean when ca_udp_client_refresh_mac() returns false?
A: A return value of false indicates that the MAC address update failed. This can happen due to an invalid handle, an inactive client, or a failed ARP request. If EN_ARP_SERVICE is disabled, the function will also return false if ARP fails.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_udp_client_refresh_mac()?
A: Common mistakes include passing an invalid handle (less than 1 or greater than UDPCLI_MAX_HANDLES), calling the function on an uninitialized or inactive client, or calling it from an ISR. Additionally, not ensuring that EN_ARP_SERVICE is enabled if ARP is required will cause the function to return false.

Behavior conf:0.90 **Q:** Is ca_udp_client_refresh_mac() thread-safe and ISR-safe?
A: Yes, the function is thread-safe as it uses a mutex to protect shared data structures. However, it is not ISR-safe and must not be called from an ISR because it uses mutexes and network functions that could cause deadlocks or undefined behavior.

`ca_udp_client_send`

Header: `caos/net/inc/udp_client.h:86`
Source: `caos/net/src/udp_client.c:231:242`
GUARD: `EN_UDP_CLIENT`

```
bool ca_udp_client_send(u8 handle, const u8 *payload, u16 len)
```

Sends a UDP payload using a previously initialized and connected client handle.

Parametri:

Nome	Tipo	Descrizione
`handle`	`u8`	The 1-based identifier returned by a successful connection function. Must be within the range [1, UDPCLI_MAX_HANDLES].
`payload`	`const u8 *`	Pointer to the buffer containing the data to be transmitted.
`len`	`u16`	The number of bytes to send from the payload buffer.

Ritorna:
Returns true if the packet was successfully queued for transmission, false if the handle is invalid, the client is not active, or the transmission failed.

Note:

- This function is thread-safe as it uses a mutex to check the client's active status. It is NOT callable from an ISR because it performs a mutex lock operation, which can cause the calling task to block. The function assumes the client has been properly initialized and connected; calling this with an uninitialized handle will result in a false return. A common mistake is passing a null pointer or an incorrect length, which may lead to memory access violations or truncated packets.

Uso:

- This function is used in the data transmission phase of a network application. It is typically paired with ca_udp_client_connect to establish the session and ca_udp_client_disconnect to clean up resources. In a typical workflow, the application initializes the client, connects to a remote endpoint, enters a loop to send data using this function, and finally disconnects when the communication is complete.

Esempio:

```
/* Assuming handle was obtained via a connection function */
u8 my_handle = 1;
const u8 data[] = "Hello, caOS!";

/* Check if the client is ready and send data */
if (ca_udp_client_send(my_handle, data, sizeof(data))) {
    /* Transmission successful */
} else {
    /* Handle error: log or retry */
    ca_printf("Failed to send UDP packet\n");
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_udp_client_send in a caOS-based embedded application?
A: You should call ca_udp_client_send when you need to transmit UDP packets from a pre-configured UDP client in your caOS application. It is typically used after initializing and activating a UDP client with ca_udp_client_init and ca_udp_client_activate, and is suitable for sending data over a network connection in a non-blocking manner.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_udp_client_send with caOS types?
A: u8 handle = 1; const u8 payload[] = {0x01, 0x02, 0x03}; u16 len = 3; bool success = ca_udp_client_send(handle, payload, len); if (success) { // Packet sent successfully } else { // Failed to send packet }

Parameters conf:0.95 **Q:** What are the valid ranges for the handle parameter in ca_udp_client_send?
A: The handle parameter must be a value between 1 and UDPCLI_MAX_HANDLES (inclusive). Using a value of 0 or any number greater than UDPCLI_MAX_HANDLES will cause the function to return false. The handle identifies a specific UDP client instance that must have been initialized and activated before calling this function.

Return conf:0.90 **Q:** What does it mean when ca_udp_client_send returns false?
A: A return value of false indicates that the packet transmission failed. This can happen due to invalid handle, a client that is not active, a NULL payload pointer, or a zero length. It does not necessarily mean the network is down, but rather that the function could not proceed with sending the data based on the provided parameters.

Error/Edge case conf:0.85 **Q:** What are the most common mistakes developers make when calling ca_udp_client_send?
A: Common mistakes include passing an invalid handle (0 or greater than UDPCLI_MAX_HANDLES), using a NULL payload pointer, calling the function on a UDP client that has not been activated, or attempting to call this function from an ISR. These errors will result in the function returning false and failing to send the packet.

`ca_udp_server_bind`

Header: `caos/net/inc/udp_server.h:95`
Source: `caos/net/src/udp_server.c:331:335`
GUARD: `EN_UDP_SERVER`

```
bool ca_udp_server_bind(void)
```

Registers the UDP server port and assigns a callback function to handle incoming network packets.

Ritorna:
Returns true if the port was successfully bound to the UDP handler, or false if the port configuration is invalid (e.g., port 0) or the binding process failed.

Note:

- This function is not thread-safe and must be called during the system initialization phase before the network stack starts processing traffic. It is not intended to be called from an Interrupt Service Routine (ISR). The function relies on a pre-configured static structure (s_srv_cfg) which must be initialized before calling this; failing to do so will result in a return value of false. A common beginner mistake is attempting to call this function before the network stack has been initialized via ca_net_proto_init, which will cause the binding to fail silently or return an error.

Uso:

- This function is typically used in the main initialization sequence of a network-enabled task. It is usually paired with ca_net_proto_init to prepare the stack, followed by ca_udp_server_bind to open the listening port. After binding, developers often call ca_eth_link_poll to ensure the physical link is active and ca_arp_send_garp to announce the device's presence on the network. It is essential to ensure the port is defined in the configuration before invocation.

Esempio:

```
/* Initialize the network stack first */
ca_net_proto_init(&net_cfg);

```

```
/* Attempt to bind the UDP server to the configured port */
if (ca_udp_server_bind()) {
    ca_stdout_printf_n("UDP server successfully bound");
} else {
    ca_stdout_printf_n("Failed to bind UDP server");
}

/* Ensure the link is polled to process incoming packets */
ca_eth_link_poll();
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_udp_server_bind() in my UDP server implementation?
A: You should call ca_udp_server_bind() after you have configured the UDP server port, typically after setting up the server's IP address and port number. This function binds the specified port for receiving UDP packets and is essential for the server to start listening for incoming data.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_udp_server_bind() with caOS types?
A: u16 port = 12345; // Assume UDP server is already configured with IP and port bool result = ca_udp_server_bind(); if (result) { // Binding successful } else { // Binding failed }

Return conf:0.90 **Q:** What does it mean when ca_udp_server_bind() returns false?
A: The function returns false if the port is 0 or if the binding operation fails. A return value of false indicates that the UDP server was not able to bind to the specified port, which could be due to incorrect configuration or a system-level failure during the binding process.

`ca_udp_server_client_count`

Header: `caos/net/inc/udp_server.h:120`
Source: `caos/net/src/udp_server.c:369:378`
GUARD: `EN_UDP_SERVER`

```
u8 ca_udp_server_client_count(void)
```

Returns the current number of active UDP server clients.

Ritorna:
Returns an unsigned 8-bit integer (u8) representing the count of currently assigned UDP clients. The value will always be between 0 and UDPSRV_MAX_CLIENTS.

Note:

- This function is thread-safe because it uses a mutex to protect the internal client list while iterating. It is a blocking call because it must wait for the mutex to be available; therefore, it must NEVER be called from an ISR (Interrupt Service Routine) or a critical section, as this would cause a system deadlock or crash. The UDP server must be initialized before calling this function. A common mistake is assuming the count remains constant after the function returns; in a multi-threaded environment, a client could disconnect or connect immediately after the function returns, so treat the result as a snapshot in time.

Uso:

- This function is typically used for system monitoring, diagnostics, or telemetry reporting. It is often paired with CLI commands to display server status or included in periodic system

health logs to track network load. Developers use it to verify if the server is reaching its maximum capacity or to detect if clients are failing to disconnect properly, which might indicate a memory or resource leak.

Esempio:

```c
/* Example: Periodic diagnostic task to monitor UDP server load */
void monitor_task(void *pvParameters) {
    while (1) {
        u8 active_clients = ca_udp_server_client_count();

        /* Log the current number of clients to the system console */
        ca_stdout_printf_n("System Status: Active UDP clients = %u", (u32)
    active_clients);

        /* Check if we are nearing capacity */
        if (active_clients > 10) {
            ca_stdout_printf_n("Warning: High UDP client load detected!");
        }

        /* Wait for 5 seconds before the next check */
        ca_os_delay_ms(5000);
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_udp_server_client_count() and why is it important for managing UDP connections?
A: You should call ca_udp_server_client_count() when you need to monitor or control the number of active UDP clients connected to your server. This function is important because it provides a thread-safe way to retrieve the current client count, protecting access to internal data structures using a mutex. It helps in managing resources and understanding connection load, especially in applications where connection limits or dynamic behavior based on client count is required.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_udp_server_client_count() with caOS types?
A: u8 client_count; client_count = ca_udp_server_client_count(); if (client_count == 0) { // No clients connected } else { // Process client count }

Return conf:0.90 **Q:** What does the return value of ca_udp_server_client_count() indicate, and what can cause the function to return 0?
A: The return value of ca_udp_server_client_count() indicates the number of currently connected UDP clients, represented as a u8 value from 0 to 255. A return value of 0 can occur if the mutex protecting the internal data structure cannot be acquired, which may lead to a blocking condition or an error if the mutex is already locked by another thread. The function does not return explicit error codes, so improper use or mutex contention can result in a return value of 0 without indicating a specific failure reason.

`ca_udp_server_disconnect`

Header: `caos/net/inc/udp_server.h:115`
Source: `caos/net/src/udp_server.c:354:367`
GUARD: `EN_UDP_SERVER`

```c
void ca_udp_server_disconnect(u8 client_id)
```

Forcefully disconnects a specific UDP client and clears its associated network buffers.

Parametri:

Nome	Tipo	Descrizione
`client_id`	`u8`	The index of the client to disconnect. Must be less than UDPSRV_MAX_CLIENTS.

Ritorna:
void

Note:

- This function is thread-safe as it uses a mutex to protect the client state during modification. It is a blocking operation because it must acquire the server mutex and iterate through the client's ring buffer to free pending packets. It must NOT be called from an ISR (Interrupt Service Routine) because mutex locking is prohibited in interrupt context. A common mistake is passing an invalid client_id, which the function handles by returning early, but developers should always validate the ID before calling to ensure the intended client is actually the one being disconnected.

Uso:

- This function is typically used in network management tasks, such as a CLI command to 'kick' a misbehaving client or during a system shutdown sequence to clean up resources. It is often paired with ca_udp_server_get_client to verify the client exists before attempting to disconnect it. By clearing the ring buffer, it ensures that no stale data remains in memory, preventing potential memory leaks or processing of old packets if the same client slot is reused later.

Esempio:

```c
/* Example: Kicking a client via CLI command */
u8 id = 2; // Assume we want to disconnect client index 2
ca_udp_srv_client_info_t info;

// First, check if the client is actually connected
if (ca_udp_server_get_client(id, &info)) {
    // Disconnect the client and free its buffers
    ca_udp_server_disconnect(id);
    ca_printf_n("Client %u disconnected successfully.", (u32)id);
} else {
    ca_printf_n("Client %u is not active.", (u32)id);
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_udp_server_disconnect() and what is its purpose in a UDP server implementation?
A: You should call ca_udp_server_disconnect() when you want to explicitly disconnect a specific UDP client from the server. Its purpose is to free up resources associated with that client, such as buffers and connection state. This function is typically used when a client session ends or needs to be terminated for any reason, ensuring proper cleanup of allocated memory and preventing resource leaks.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_udp_server_disconnect() with caOS types?
A: u8 client_id = 0; ca_udp_server_disconnect(client_id);

Parameters conf:0.95 **Q:** What is the significance of the client_id parameter in ca_udp_server_disconnect(), and what are the valid ranges for this value?
A: The client_id parameter identifies which specific UDP client to disconnect. It must be a value between 0 and UDPSRV_MAX_CLIENTS-1. If client_id is greater than or equal to UDPSRV_MAX_CLIENTS, the function will simply return without taking

any action. This ensures that only valid client indices are processed, preventing access to invalid memory locations.

Error/Edge case conf:0.90 **Q:** What are the potential issues or mistakes when using ca_udp_server_disconnect and what edge cases should be considered?
A: A common mistake is calling ca_udp_server_disconnect() with an invalid client_id (i.e., greater than or equal to UDPSRV_MAX_CLIENTS), which will cause the function to silently return without performing any action. Additionally, calling this function from an ISR is not allowed since it's blocking due to mutex usage. It's also important to ensure that the client was previously allocated and registered; otherwise, calling this function may lead to undefined behavior due to accessing uninitialized memory.

`ca_udp_server_get_client`

Header: `caos/net/inc/udp_server.h:133`
Source: `caos/net/src/udp_server.c:380:391`
GUARD: `EN_UDP_SERVER`

```
bool ca_udp_server_get_client(u8 client_id, ca_udp_srv_client_info_t *
    info)
```

Retrieves connection details for a specific UDP server client slot.

Parametri:

Nome	Tipo	Descrizione
`client_id`	`u8`	The index of the client slot to query. Must be less than UDPSRV_MAX_CLIENTS.
`info`	`ca_udp_srv_client_info_t *`	Pointer to a structure where the client data will be copied. Must not be NULL.

Ritorna:
Returns true if the client slot is currently assigned (connected), false otherwise. Also returns false if the provided pointer is NULL or the client_id is out of range.

Note:

- This function is thread-safe as it uses an internal mutex to protect the client table during the copy operation. It is a blocking function because it waits for the mutex; therefore, it must NEVER be called from an ISR (Interrupt Service Routine). A common mistake is to assume the client remains connected after the function returns; since the network state can change asynchronously, always check the 'connected' field in the returned structure before using the IP or port data.

Uso:

- This function is typically used in diagnostic tasks or CLI commands to iterate through all available client slots to display network status. It is usually paired with a loop that checks every index from 0 to UDPSRV_MAX_CLIENTS. It is the standard way to safely extract connection metadata (IP, port, and buffer usage) from the UDP server's internal state without causing race conditions.

Esempio:

```
void print_active_udp_clients(void) {
    ca_udp_srv_client_info_t info;
    for (u8 i = 0; i < 8; i++) {
        /* Attempt to retrieve client info for slot i */
        if (ca_udp_server_get_client(i, &info)) {
            /* Use ca_printf to display the client's IP address */
```

```
            ca_printf("Client %u connected: %u.%u.%u.%u\n",
                      (u32)i,
                      info.src_ip.v[0], info.src_ip.v[1],
                      info.src_ip.v[2], info.src_ip.v[3]);
        } else {
            /* Slot is empty or invalid */
            ca_printf("Client %u is inactive.\n", (u32)i);
        }
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_udp_server_get_client in a caOS-based UDP server application?
A: You should use ca_udp_server_get_client when you need to retrieve information about a specific connected UDP client, such as its IP address or port, for monitoring or management purposes. It is typically called after the UDP server has been initialized and clients have connected.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_udp_server_get_client with caOS types?
A: u8 client_id = 0; ca_udp_srv_client_info_t client_info; bool result = ca_udp_server_get_client(c &client_info); if (result) { // Use client_info }

Parameters conf:0.95 **Q:** What are the valid ranges for the client_id parameter in ca_udp_server_get_client?
A: The client_id must be a value between 0 and UDPSRV_MAX_CLIENTS-1. Values outside this range will cause the function to return false.

Return conf:0.95 **Q:** What does it mean when ca_udp_server_get_client returns false?
A: A return value of false indicates either that the client_id is out of range (greater than or equal to UDPSRV_MAX_CLIENTS) or that the info pointer passed is NULL.

Error/Edge case conf:0.85 **Q:** What are the most common mistakes developers make when calling ca_udp_server_get_client?
A: Common mistakes include passing a client_id that is greater than or equal to UDPSRV_MAX_CLIENTS, or passing a NULL pointer for the info parameter. Both will result in the function returning false.

`ca_udp_server_send`

Header: `caos/net/inc/udp_server.h:102`
Source: `caos/net/src/udp_server.c:337:352`
GUARD: `EN_UDP_SERVER`

```
bool ca_udp_server_send(u8 client_id, const u8 *payload, u16 len)
```

Sends a UDP packet to a specific client previously registered with the UDP server.

Parametri:

Nome	Tipo	Descrizione
`client_id`	`u8`	The index of the client in the server's internal client table. Must be less than UDPSRV_MAX_CLIENTS.
`payload`	`const u8 *`	Pointer to the data buffer to be sent.
`len`	`u16`	Length of the data in bytes.

Ritorna:
Returns true if the packet was successfully queued for transmission, false if the client_id is invalid, the client is not assigned, or the transmission failed.

Note:

- This function is thread-safe as it uses a mutex to protect access to the client table. It is NOT safe to call from an ISR because it performs a mutex lock, which can cause the system to hang or trigger a kernel panic if called from an interrupt context. Ensure the UDP server has been initialized and bound using ca_udp_server_bind before attempting to send data. A common mistake is passing an uninitialized or out-of-bounds client_id, which will immediately return false.

Uso:

- This function is used to respond to clients that have previously sent data to the server. It is typically used in a request-response pattern where the server receives a packet, processes it, and then uses this function to send the reply back to the specific client identified by the index. It is usually paired with ca_udp_server_bind to set up the server and is called within the application logic after processing incoming data.

Esempio:

```
/* Assuming the server is already bound and a client is active */
void send_response_to_client(u8 cid, const u8 *data, u16 len) {
    /* Check if the client ID is within valid range before sending */
    if (cid < 10) {
        bool success = ca_udp_server_send(cid, data, len);
        if (!success) {
            /* Log error if transmission failed */
            ca_printf("Failed to send UDP packet to client %u\n", cid);
        }
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_udp_server_send to send data to a UDP client?
A: You should use ca_udp_server_send when you need to send a message to a specific UDP client that has been registered with the UDP server. It is typically called after the UDP server has been initialized and a client has connected and been assigned an ID.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_udp_server_send in C with caOS types?
A: u8 client_id = 0; const u8 payload[] = {0x01, 0x02, 0x03}; u16 len = 3; bool success = ca_udp_server_send(client_id, payload, len); if (success) { // Message sent successfully } else { // Handle send failure }

Parameters conf:0.95 **Q:** What are the valid ranges for client_id and payload in ca_udp_server_send?
A: The client_id must be between 0 and UDPSRV_MAX_CLIENTS-1. Values outside this range will cause the function to return false. The payload pointer must not be NULL, and the len parameter must be greater than 0 for a successful send.

Return conf:0.95 **Q:** What does it mean when ca_udp_server_send returns false?
A: A return value of false indicates that the message was not sent. This can happen if the client_id is invalid, the client is not registered or active, or if the UDP server has not been initialized with a valid port.

Error/Edge case conf:0.90 **Q:** What are common mistakes when calling ca_udp_server_send?
A: Common mistakes include passing a client_id outside the valid range, passing a NULL payload pointer, calling the function before initializing the UDP server, or attempting to send a message with a length of 0.

Behavior conf:0.90 **Q:** Is ca_udp_server_send blocking or thread-safe, and can it be called from an ISR?
A: ca_udp_server_send is not strictly blocking but uses a mutex, so it may wait for access if another thread is using the server. It is thread-safe and can be called from

multiple threads. However, it is not ISR-safe and should not be called from an ISR.

---

ca_udp_server_set_local_ip

Header: caos/net/inc/udp_server.h:117
GUARD: EN_UDP_SERVER

```
static inline void ca_udp_server_set_local_ip(ca_ipv4_t ip)
```

Updates the local IP address configuration for the UDP server module.

Parametri:

Nome	Tipo	Descrizione
ip	ca_ipv4_t	A structure containing the new IPv4 address to be assigned to the UDP server. Ensure the address is valid and formatted correctly before passing.

Ritorna:
void

Note:

- This function is a non-blocking, inline operation that directly updates the internal configuration structure of the UDP server. It is not thread-safe by itself; if multiple tasks attempt to modify network settings simultaneously, you must ensure external synchronization. It is safe to call from an ISR if the underlying memory access is atomic, but generally, network configuration changes should be performed in a task context. A common mistake is calling this before the UDP server has been properly initialized, which may lead to the configuration being overwritten or ignored.

Uso:

- This function is typically used during the network initialization sequence, specifically after a DHCP lease has been successfully acquired or a static IP has been assigned. It is usually paired with ca_net_proto_set_local_ip to update the global network stack and ca_arp_service_update_local_ip to ensure the ARP table reflects the new address. It is often wrapped in an EVAL_UDP_SRV macro to ensure the operation only executes if the UDP server module is enabled in the project configuration.

Esempio:

```
/* Assuming 'ip' is a valid ca_ipv4_t structure obtained from DHCP */

// Update the global network protocol stack with the new IP
ca_net_proto_set_local_ip(ip);

// Update the UDP server configuration specifically
ca_udp_server_set_local_ip(ip);

// Update the ARP service to handle requests for the new IP
ca_arp_service_update_local_ip(ip);

// Notify the user via the system console
ca_stdout_printf_n("UDP Server IP updated to: %u.%u.%u.%u",
    ip.v[0], ip.v[1], ip.v[2], ip.v[3]);
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to call ca_udp_server_set_local_ip in my network initialization sequence?
A: This function should be called after the UDP server has been properly initialized. It is typically used after a DHCP lease is acquired or a static IP is assigned, and it is recommended to pair it with ca_net_proto_set_local_ip and ca_arp_service_update_local_ip to ensure consistency across the network stack.

Code example conf:0.95 **Q:** How do I implement a basic update of the UDP server IP address using caOS types?
A: You should wrap the call in the EVAL_UDP_SRV macro to ensure the module is enabled. Example: { ca_ipv4_t new_ip; new_ip.addr = 0xC0A8000A; EVAL_UDP_SRV(ca_udp_serve }

Parameters conf:1.00 **Q:** What are the requirements for the ca_ipv4_t ip parameter passed to ca_udp_server_set_local_ip?
A: The parameter must be an ca_ipv4_t structure containing the new IPv4 address. You must ensure the address is valid and formatted correctly before passing it, as the function performs a direct, non-blocking update to the internal configuration structure.

`ca_udp_server_start`

Header: `caos/net/inc/udp_server.h:89`
Source: `caos/net/src/udp_server.c:301:329`
GUARD: `EN_UDP_SERVER`

```
1 bool ca_udp_server_start(const ca_udp_server_cfg_t *cfg)
```

Initializes and launches the UDP server background tasks based on the provided configuration.

Parametri:

Nome	Tipo	Descrizione
`cfg`	`const ca_udp_server_cfg_t *`	Pointer to a configuration structure. Must contain a valid callback function (on_rx) and a non-zero number of clients not exceeding UDPSRV_MAX_CLIENTS.

Ritorna:
Returns true if the server was successfully initialized and tasks were created; returns false if the configuration is invalid or NULL.

Note:

- This function is not thread-safe and must be called during system initialization, typically within ca_os_main, before the scheduler starts running tasks. It performs internal memory initialization for mutexes and semaphores for each client slot; therefore, it must not be called more than once. It is not callable from an ISR. A common mistake is passing a NULL pointer or a configuration where max_clients exceeds the hardcoded system limit, which will cause the function to fail silently by returning false.

Uso:

- This function is used to set up the network infrastructure for UDP communication. It is typically called in the main initialization sequence alongside other network services like ca_arp_service_start or ca_dhcp_service_start. Once called, it spawns internal worker tasks that handle incoming UDP packets asynchronously. It should be invoked before any network-dependent tasks are created via ca_os_job to ensure the server is ready to process traffic.

Esempio:

```
/* Define the server configuration structure */
ca_udp_server_cfg_t srv_cfg = {
    .on_rx = my_udp_rx_callback,
    .max_clients = 2
};

void ca_os_main(void) {
    /* Initialize network services before starting tasks */
    ca_arp_service_start();

    /* Start the UDP server; check return value for safety */
    if (ca_udp_server_start(&srv_cfg)) {
        /* Server started successfully, proceed to launch application tasks */
        ca_os_job("app_task", CA_TH_LEVEL3, my_app_task, 0, STACK_SMALL,
    CA_JOB_STD, CA_FAULT_RESTART_THREAD);
    } else {
        /* Handle initialization failure */
        while(1) { /* Error: could not start UDP server */ }
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_udp_server_start() in my caOS application?
A: You should call ca_udp_server_start() after initializing the caOS RTOS and setting up a valid UDP server configuration. It is typically used when you need to receive UDP packets from remote clients and process them using a callback function. The function is not blocking, so it returns immediately after initializing the server threads.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_udp_server_start() with caOS types?
A:

```
ca_udp_server_cfg_t cfg;
cfg.on_rx = my_udp_callback;
cfg.max_clients = 5;
bool success = ca_udp_server_start(&cfg);
```

This example initializes a UDP server configuration with a callback function and a maximum of 5 clients, then starts the server.

Parameters conf:0.90 **Q:** What does the max_clients parameter in ca_udp_server_start() control and what are valid values?
A: The max_clients parameter specifies the maximum number of concurrent UDP clients the server can handle. Valid values are integers between 1 and UDPSRV_MAX_CLIENTS (inclusive). If set to 0 or greater than UDPSRV_MAX_CLIENTS, the function will return false.

Return conf:0.95 **Q:** What does it mean when ca_udp_server_start() returns false?
A: A return value of false indicates that the UDP server failed to start. This happens when the configuration is invalid, such as passing a NULL pointer for cfg, having on_rx set to NULL, or setting max_clients to 0 or a value exceeding UDPSRV_MAX_CLIENTS.

Error/Edge case conf:0.90 **Q:** What are common mistakes when calling ca_udp_server_start()?
A: Common mistakes include passing a NULL pointer for the cfg parameter, not initializing the on_rx callback function, setting max_clients to 0 or a value greater than UDPSRV_MAX_CLIENTS, and calling the function before the RTOS is initialized. These errors will cause the function to return false.

Behavior conf:0.90 **Q:** Is ca_udp_server_start() thread-safe and can it be called from an ISR?
A: No, ca_udp_server_start() is not thread-safe and must not be called from multiple threads simultaneously. It also cannot be called from an ISR (Interrupt Service Rou-

tine). The function is not blocking and returns immediately after initializing the server structures and creating worker threads.

---

ca_usb_cdc_connected

Header: caos/usb/inc/usb_cdc.h:77
Source: caos/usb/src/usb_cdc.c:274:277
GUARD: EN_USB_CDC

```
bool ca_usb_cdc_connected(void)
```

Checks if the USB CDC virtual serial port is currently connected to a host.

Ritorna:
Returns true if the DTR (Data Terminal Ready) signal is asserted by the host, indicating a terminal or application is listening; returns false otherwise.

Note:

- This function is non-blocking and thread-safe as it simply reads a status flag updated by the USB interrupt service routine. It is safe to call from any context, including ISRs, because it performs a simple memory read. The most common mistake is assuming that 'connected' means the USB cable is physically plugged in; it specifically checks if the host software has opened the virtual COM port and set the DTR line. If you attempt to send data via ca_usb_cdc_write while this returns false, the data may be silently dropped or lost, so always check this status before initiating a transmission.

Uso:

- This function is typically used as a guard clause before attempting to send logs or data over USB to avoid wasting CPU cycles or memory on data that has nowhere to go. It is often paired with ca_usb_cdc_write to ensure the transmission is valid, or with ca_usb_cdc_wait_connected when the application must block until a user opens the terminal. In a typical sequence, you would initialize the USB stack, wait for connection, and then periodically check this status to handle unexpected disconnections gracefully.

Esempio:

```
/* Check if the USB CDC terminal is ready before sending data */
if (ca_usb_cdc_connected()) {
    /* Prepare a message buffer using ca_memory_allocate */
    /* Assume 'msg' is a valid pointer to a message structure */
    if (!ca_usb_cdc_write(msg)) {
        /* Handle write failure if necessary */
    }
} else {
    /* Terminal is not connected, skip transmission to save resources */
    /* Optionally wait for connection if the task is not time-critical */
    /* ca_usb_cdc_wait_connected(1000); // Wait up to 1 second */
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_usb_cdc_connected() in my embedded application?
A: You should call ca_usb_cdc_connected() before attempting to send data over USB CDC to ensure the host is connected. This function is useful for avoiding data loss or undefined behavior that can occur when trying to transmit data without an active USB connection.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_usb_cdc_connected() with caOS types?
A:

```c
u8 send_data_if_connected(void) {
    if (ca_usb_cdc_connected()) {
        // Send data here
        return 1;
    }
    return 0;
}
```

Return conf:0.90 **Q:** What does it mean when ca_usb_cdc_connected() returns false?
A: When ca_usb_cdc_connected() returns false, it means the USB CDC device is not connected to a host computer. This typically happens when the USB cable is unplugged or the host has not yet enumerated the device. Attempting to send data in this state may result in data loss or undefined behavior.

HW differences conf:0.85 **Q:** Are there any differences in how ca_usb_cdc_connected() is implemented across different STM32 families?
A: No, there are no differences in the implementation of ca_usb_cdc_connected() across STM32 families (f1xx, f4xx, f7xx, h5xx, h7xx). All implementations simply call tud_cdc_connected() which is a TinyUSB function, indicating that the underlying USB stack is consistent across these families.

`ca_usb_cdc_get_line`

Header: `caos/usb/inc/usb_cdc.h:82`
Source: `caos/usb/src/usb_cdc.c:290:293`
GUARD: `EN_USB_CDC`

```c
const ca_usb_cdc_line_t *ca_usb_cdc_get_line(void)
```

Retrieves a pointer to the static USB CDC line configuration structure.

Ritorna:
Returns a constant pointer to the internal ca_usb_cdc_line_t structure. This pointer is never NULL as it points to a statically allocated object.

Note:

- This function is non-blocking and thread-safe because it simply returns the address of a pre-allocated static variable. It is safe to call from both tasks and ISRs. The primary precondition is that the USB CDC subsystem must have been initialized, otherwise the returned structure may contain default or uninitialized values. A common mistake is attempting to modify the fields of the returned structure directly; since it is a 'const' pointer, you should treat the configuration as read-only to avoid undefined behavior in the USB stack.

Uso:

- This function is typically used to inspect the current state of the USB CDC line (such as baud rate, parity, or stop bits) after a connection has been established. It is often paired with ca_usb_cdc_connected or ca_usb_cdc_wait_connected to ensure the line is active before reading its parameters. Developers use this to synchronize application-level serial settings with the USB host's requirements.

Esempio:

```c
/* Check if USB CDC is connected before accessing line settings */
if (ca_usb_cdc_connected()) {
    const ca_usb_cdc_line_t *line = ca_usb_cdc_get_line();

    /* Safely read the current line configuration */
    ca_printf("Current Baudrate: %lu\n", line->dwDTERate);
} else {
    /* Wait for connection if not ready */
    if (ca_usb_cdc_wait_connected()) {
        const ca_usb_cdc_line_t *line = ca_usb_cdc_get_line();
        ca_printf("Connected! Baudrate: %lu\n", line->dwDTERate);
    } else {
        ca_printf("USB CDC connection timeout.\n");
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should a developer call ca_usb_cdc_get_line() and why would they need to read the CDC configuration parameters?
A: A developer should call ca_usb_cdc_get_line() when they need to read the current USB CDC serial communication settings such as baudrate, parity, and stop bits. This function is useful for debugging or dynamically adapting application behavior based on the host's CDC configuration. Since it is non-blocking and thread-safe, it can be safely called from ISRs or task contexts without risk of data corruption or delays.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_usb_cdc_get_line() with caOS types?
A: u32 baudrate; const ca_usb_cdc_line_t *line = ca_usb_cdc_get_line(); if (line != NULL) { baudrate = line->baudrate; // Use baudrate for further processing } else { // Handle error: CDC not initialized or not available }

Return conf:0.90 **Q:** What does it mean when ca_usb_cdc_get_line() returns NULL, and what could cause this failure?
A: When ca_usb_cdc_get_line() returns NULL, it indicates that the USB CDC module is either not initialized or not active. This can happen if the USB CDC driver has not been set up properly or if the USB peripheral is not enabled. The function does not indicate a runtime error but rather a configuration or initialization issue. The returned pointer points to a global shared variable, so it should not be modified directly.

`ca_usb_cdc_wait_connected`

Header: `caos/usb/inc/usb_cdc.h:79`
Source: `caos/usb/src/usb_cdc.c:279:288`
GUARD: `EN_USB_CDC`

```c
bool ca_usb_cdc_wait_connected(u32 timeout_ms)
```

Blocks the current task until a USB CDC connection is established or the timeout expires.

Parametri:

Nome	Tipo	Descrizione
`timeout_ms`	`u32`	Maximum time in milliseconds to wait for the DTR signal. Use 0 for no timeout (infinite wait) or a specific duration.

Ritorna:
Returns true if the USB CDC connection (DTR signal) is detected within the timeout period, otherwise returns false.

Note:

- This function is a blocking call that yields execution to other tasks via ca_os_sleep(50) while waiting. It is NOT thread-safe for concurrent calls on the same resource and must NEVER be called from an ISR (Interrupt Service Routine) because it contains blocking logic. The most common beginner mistake is calling this function inside a high-priority task without considering that it will halt that task's execution for up to the specified timeout duration, potentially starving other processes.

Uso:

- This function is typically used at the start of a task that requires a USB terminal connection to function, such as a logging or network configuration task. It is often paired with ca_usb_cdc_write to send data only after confirming the host is ready to receive. By waiting for the connection, you prevent data loss that occurs if the device attempts to transmit before the USB stack is fully enumerated and the terminal application has asserted the DTR line.

Esempio:

```
static u32 task_usb_logger(u32 arg, u32 arg2) {
    (void)arg; (void)arg2;

    // Wait up to 5 seconds for the PC to open the serial terminal
    if (ca_usb_cdc_wait_connected(5000)) {
        ca_stdout_printf_n("USB CDC connected, starting logger...");
        ca_usb_cdc_write("System Ready\n", 13);
    } else {
        // Handle case where user never connected the USB cable
        ca_stdout_printf_n("USB connection timeout.");
    }

    while (1) {
        // Perform periodic tasks
        ca_os_sleep(1000);
    }
}
```

Q&A — fine-tuning review

5/5 pairs · qwen3-coder:latest

Usage conf:0.95 **Q:** When should I call ca_usb_cdc_wait_connected() in my caOS application?
A: You should call ca_usb_cdc_wait_connected() after initializing the USB CDC driver to ensure the host (e.g., PC) is connected before attempting to send serial data. This is useful for synchronizing serial output with the host connection, preventing data loss or undefined behavior.

Code example conf:0.90 **Q:** Can you provide a minimal C example using ca_usb_cdc_wait_connected() with caOS types?
A:

```
u32 timeout = 5000;
if (ca_usb_cdc_wait_connected(timeout)) {
    ca_usb_cdc_printf("USB connected\n");
} else {
    ca_usb_cdc_printf("USB connection timeout\n");
}
```

Parameters conf:0.90 **Q:** What does the timeout_ms parameter in ca_usb_cdc_wait_connected() control, and what are valid values?
A: The timeout_ms parameter specifies the maximum time in milliseconds to wait for USB CDC connection. Valid values are any positive u32 value, or 0 to indicate non-blocking behavior (though the function will still block for at least 10ms per iteration).

Return conf:0.95 **Q:** What do the return values of ca_usb_cdc_wait_connected() indicate, and what causes it to return false?
A: The function returns true if the USB CDC device is connected, and false if the timeout expires without a connection. A false return is caused by the timeout_ms elapsing before tud_cdc_connected() returns true.

HW differences conf:0.85 **Q:** Are there any hardware differences in how ca_usb_cdc_wait_connected() is implemented across different STM32 families?
A: No, the implementation of ca_usb_cdc_wait_connected() is consistent across all supported STM32 families (f1xx, f4xx, f7xx, h5xx, h7xx). Each uses the same logic involving tud_task() and ca_os_sleep(10), checking tud_cdc_connected() in a loop until either connection occurs or timeout expires.

`ca_usb_cdc_write`

Header: `caos/usb/inc/usb_cdc.h:74`
Source: `caos/usb/src/usb_cdc.c:268:272`
GUARD: `EN_USB_CDC`

```
bool ca_usb_cdc_write(p_str message)
```

Sends a data buffer over the USB CDC virtual serial port.

Parametri:

Nome	Tipo	Descrizione
`message`	`p_str`	A pointer to a structure containing the data buffer (data) and its length (size). Must be a valid, non-NULL pointer.

Ritorna:
Returns true if the data was successfully queued for transmission, false otherwise (e.g., USB not connected, buffer overflow, or hardware error).

Note:

- This function is a wrapper that handles the low-level USB endpoint addressing for CDC communication. It is generally non-blocking, meaning it queues the data for the USB peripheral to send and returns immediately. It is not intended to be called from an ISR (Interrupt Service Routine) unless explicitly documented as thread-safe in your specific hardware abstraction layer. A common beginner mistake is calling this function before the USB host has initialized the connection, which will result in a false return value; always verify the connection status first.

Uso:

- This function is used to send telemetry, logs, or command responses to a connected PC terminal. It is typically used in a sequence: first, check if the USB CDC interface is active using ca_usb_cdc_connected(), then prepare the data structure, and finally call ca_usb_cdc_write(). It is often paired with a buffer management strategy to ensure that data is not lost if the host is not reading fast enough.

Esempio:

```
/* Example: Sending a status message to the USB host */
#include "ca_os.h"

void send_status_update(void) {
    /* Define a static message structure */
```

```
    static const char my_msg[] = "System Ready\r\n";
    p_str msg_ptr;
    msg_ptr.data = (u8*)my_msg;
    msg_ptr.size = sizeof(my_msg) - 1;

    /* Check if the USB host has opened the terminal */
    if (ca_usb_cdc_connected()) {
        /* Attempt to send the message */
        if (!ca_usb_cdc_write(&msg_ptr)) {
            /* Handle error: log to internal system or retry later */
            /* Use ca_log_log if available or handle locally */
        }
    } else {
        /* USB not connected, do nothing or wait */
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_usb_cdc_write() in my caOS application?

A: You should use ca_usb_cdc_write() when you need to send data over USB CDC (Communication Device Class) interface in a blocking manner. It's typically used for logging, debugging, or communication with a host PC. The function is ideal when you require guaranteed delivery of the message before proceeding, as it blocks until the data is accepted for transmission or a timeout occurs.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_usb_cdc_write() with caOS types?

A:

```
#include "caos.h"

void send_hello(void) {
    cstr msg;
    u8 data[] = "Hello USB!";
    msg.data = data;
    msg.size = 10;
    u16 bytes_written = ca_usb_cdc_write(&msg);
    if (bytes_written == 0) {
        // Handle error
    }
}
```

Parameters conf:0.95 **Q:** What does the msg parameter in ca_usb_cdc_write() represent and what are valid values?

A: The msg parameter is a pointer to a cstr structure containing the data to be sent. It must be a valid pointer to a cstr structure, not NULL. The cstr structure has a data field pointing to the actual data and a size field indicating the number of bytes to send. If msg is NULL or msg->size is 0, the function returns 0 immediately.

Return conf:0.90 **Q:** What do the return values of ca_usb_cdc_write() mean and what causes it to return 0?

A: The function returns a positive number indicating the number of bytes successfully written to the USB transmission queue. It returns 0 if the USB CDC is not connected, if the msg parameter is NULL, if msg->size is 0, or if the write operation fails. This can happen if the USB device is not properly initialized or if the underlying USB driver fails.

HW differences conf:0.85 **Q:** Are there any hardware-specific differences in how ca_usb_cdc_write() is implemented across different STM32 families?

A: No, the implementation of ca_usb_cdc_write() is consistent across all supported STM32 families (f1xx, f4xx, f7xx, h5xx, h7xx). All implementations use the same underlying TinyUSB functions tud_cdc_connected(), tud_cdc_write(), and tud_cdc_write_flush(). The function relies on the USB CDC driver provided by TinyUSB, which abstracts hard-

ware differences, so there are no register or peripheral-specific variations in the implementation.

---

ca_usb_connect

Header: caos/drivers/inc/usb.h:108
Source: caos/drivers/src/usb.c:79:83
GUARD: EN_USB

```
void ca_usb_connect(u32 usb_num)
```

Connects the specified USB peripheral to the host by enabling the D+ pull-up resistor.

Parametri:

Nome	Tipo	Descrizione
usb_num	u32	The index of the USB peripheral to connect. Must be less than USB_MAX_CTRL. Values outside this range are ignored.

Ritorna:
None (void).

Note:

- This function is non-blocking and performs a low-level hardware operation to signal the USB host that a device is present. It is not thread-safe in the sense that it does not use internal mutexes, so it should be called from a single management task or protected by higher-level logic. It is not intended to be called directly from an ISR, as it interacts with driver structures that may require consistent state. The most common mistake is calling this before the USB driver has been properly initialized, which will result in a silent failure because the internal operation pointer (s_ops) will be NULL.

Uso:

- This function is typically used during the USB device lifecycle management, specifically when transitioning from a disconnected or halted state to an active state. It is often paired with ca_usb_disconnect to manage the physical connection status. In a typical application, you would call this after initializing the USB stack and descriptors to allow the host to perform enumeration. It is also used in CLI commands to 'resume' USB communication after a halt or suspend event, effectively re-triggering the host's enumeration process.

Esempio:

```
/* Example: Resuming USB communication via CLI command */
#define USB_NUM 0

void handle_usb_resume_command(void) {
    /* Check if the USB peripheral is currently in a halted state */
    if (s_dev_state == USB_ST_HALTED) {
        /* Connect the USB device to the host to trigger re-enumeration */
        ca_usb_connect(USB_NUM);

        /* Update local state machine to reflect the active status */
        s_dev_state = USB_ST_IDLE;

        /* Notify the user via the system console */
        ca_printf("USB peripheral %d connected and re-enumerating...\n", USB_NUM
    );
    } else {
        ca_printf("USB is not in a halted state, cannot resume.\n");
```

```
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_usb_connect() and what is its purpose in a caOS application?
A: You should call ca_usb_connect() after initializing the USB controller to connect a USB device. It is used to enable the USB hardware driver for a specific USB controller (e.g., USB0 or USB1) so that it can communicate with connected USB devices. This function is essential for USB communication and must be called only after the USB controller has been properly initialized.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_usb_connect() with caOS types?
A: u32 usb_num = 0; ca_usb_connect(usb_num);

Parameters conf:0.90 **Q:** What does the usb_num parameter in ca_usb_connect() represent, and what are valid values?
A: The usb_num parameter specifies which USB controller to connect. Valid values are integers from 0 up to the maximum number of USB controllers available on the STM32 chip. For example, if the chip has two USB controllers, valid values are 0 and 1. Passing an invalid value (e.g., greater than the maximum number of controllers) may lead to undefined behavior.

Return conf:0.85 **Q:** Does ca_usb_connect() return any value, and what causes it to fail?
A: ca_usb_connect() does not return any value. It is a void function. However, it can cause errors or undefined behavior if called before the USB controller is initialized or with an invalid usb_num value. The function is not designed to explicitly report failure, but improper usage may lead to system crashes or malfunction.

`ca_usb_dbg_fs_reset`

Header: `caos/drivers/inc/usb.h:171`
GUARD: `EN_USB`

```
void ca_usb_dbg_fs_reset(void)
```

Resets the internal USB Full-Speed (FS) debugging statistics and diagnostic counters to zero.

Ritorna:
void

Note:

- This function is a non-blocking utility used to clear internal diagnostic structures, such as interrupt counters, latency measurements, and setup logs. It is not thread-safe by design, meaning it should not be called while the USB stack is actively processing interrupts or updating these statistics, as it could lead to race conditions or inconsistent data. It is generally safe to call from an ISR if the hardware implementation allows, but it is primarily intended for system initialization or manual diagnostic resets. A common beginner mistake is calling this function while the USB stack is in a high-traffic state, which may cause the diagnostic data to appear corrupted or incomplete for the current session.

Uso:

- This function is typically used during the USB subsystem initialization sequence or when a user requests a manual reset of performance metrics via a CLI command. It is often paired

with 'ca_usb_dbg_fs_stat' to clear old data before starting a new measurement period. In a typical application, you would call this during the system boot phase or after a USB re-enumeration event to ensure that the statistics reflect only the current connection cycle.

Esempio:

```
/* Reset USB FS debug counters before starting a new measurement session */
ca_usb_dbg_fs_reset();

/* Perform some USB operations... */

/* Retrieve and display the fresh statistics */
ca_usb_dbg_t stats;
ca_usb_dbg_fs_stat(&stats, 0); /* Assuming index 0 for the FS controller */
ca_printf("USB FS Reset complete. Current ISR count: %u\n", stats.isr_rx_cnt);
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to call ca_usb_dbg_fs_reset in a caOS application?

A: It is typically called during the system boot phase or after a USB re-enumeration event to ensure statistics reflect the current connection cycle. It is also used when a user requests a manual reset of performance metrics via a CLI command, often paired with ca_usb_dbg_fs_stat to clear old data before starting a new measurement period.

Code example conf:1.00 **Q:** Provide a minimal example of how to reset USB debug statistics in caOS.

A: To reset the statistics, simply call the function directly. Example: void reset_usb_metrics(void) { ca_usb_dbg_fs_reset(); }

Error/Edge case conf:1.00 **Q:** What is a common mistake when using ca_usb_dbg_fs_reset and why should it be avoided?

A: A common mistake is calling this function while the USB stack is in a high-traffic state. Because the function is not thread-safe, calling it while the USB stack is actively processing interrupts or updating statistics can lead to race conditions, resulting in corrupted or incomplete diagnostic data for the current session.

HW differences conf:1.00 **Q:** How does the implementation of ca_usb_dbg_fs_reset differ across STM32 families?

A: Implementations vary significantly: the f1xx family provides an empty function if EN_USB_DBG is enabled; f4xx, f7xx, h5xx, and u5xx families reset a global usb_dbg_t structure (and s_setup_cnt for f4xx); the h7xx family performs a complex reset of multiple arrays including latency, interrupt timing, and endpoint-specific diagnostic counters for two USB instances.

`ca_usb_dbg_fs_stat`

Header: `caos/drivers/inc/usb.h:170`
GUARD: `EN_USB`

```
void ca_usb_dbg_fs_stat(ca_usb_dbg_t *out, u32 usb_num)
```

Captures a snapshot of internal USB Full-Speed peripheral diagnostic counters and status flags.

Parametri:

Nome	Tipo	Descrizione
out	ca_usb_dbg_t *	Pointer to a structure where the diagnostic data will be copied. Must not be NULL.
usb_num	u32	The index of the USB peripheral to query. Valid values depend on the MCU (usually 0 or 1).

Ritorna:
None (void). If 'out' is NULL or 'usb_num' is out of range, the function returns immediately without modifying memory.

Note:

- This function is non-blocking and thread-safe in the sense that it performs a read-only snapshot of volatile diagnostic structures. It is safe to call from an ISR, as it does not perform any complex synchronization or blocking operations. The primary purpose is to provide visibility into USB stack performance, such as endpoint transfer counts, latency, and error flags like buffer overflows. A common beginner mistake is to assume this function resets the counters; it does not. To clear the statistics, you must explicitly call ca_usb_dbg_fs_reset() before or after the snapshot.

Uso:

- This function is typically used in diagnostic tasks or CLI commands to monitor USB health. It is often paired with ca_usb_dbg_fs_reset() to measure statistics over a specific time interval. Developers use this to debug enumeration issues, NAK-heavy traffic, or dropped packets by comparing the 'out' structure values against expected traffic patterns.

Esempio:

```
ca_usb_dbg_t usb_stats;

// Reset counters to start a fresh measurement window
ca_usb_dbg_fs_reset();

// Perform some USB operations...

// Capture the current state of USB peripheral 0
ca_usb_dbg_fs_stat(&usb_stats, 0U);

// Print the number of setup packets received for debugging
ca_printf("Setup packets received: %u\n", usb_stats.setup);
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_usb_dbg_fs_stat in my caOS application?
A: You should use it in diagnostic tasks or CLI commands to monitor USB health. It is particularly useful for debugging enumeration issues, NAK-heavy traffic, or dropped packets by comparing the captured statistics against expected traffic patterns.

Code example conf:1.00 **Q:** Provide a minimal C example to capture USB statistics.
A: ca_usb_dbg_t stats; ca_usb_dbg_fs_stat(&stats, 0U);

Parameters conf:1.00 **Q:** What are the requirements for the 'out' and 'usb_num' parameters?
A: 'out' must be a non-NULL pointer to a ca_usb_dbg_t structure where data will be copied. 'usb_num' is the index of the USB peripheral; valid values depend on the MCU (e.g., 0 or 1 for H7xx). If 'out' is NULL or 'usb_num' is out of range, the function returns immediately without modifying memory.

Error/Edge case conf:1.00 **Q:** Does calling ca_usb_dbg_fs_stat reset the USB diagnostic counters?
A: No, a common beginner mistake is to assume this function resets the counters. It only performs a read-only snapshot. To clear the statistics, you must explicitly call ca_usb_dbg_fs_reset() before or after the snapshot.

Behavior conf:1.00 **Q:** Is it safe to call ca_usb_dbg_fs_stat from an interrupt service routine (ISR)?
A: Yes, the function is non-blocking and thread-safe in the sense that it performs a read-only snapshot of volatile diagnostic structures. It does not perform complex synchronization, making it safe to call from an ISR.

HW differences conf:1.00 **Q:** How does the implementation of ca_usb_dbg_fs_stat differ across MCU families?
A: Implementations vary significantly: H7xx supports two USB peripherals and tracks latency/ISR timing metrics; H5xx includes live snapshots of CHEP registers and RX/TX queue pointers; F1xx provides a minimal implementation; F7xx and H5xx include specific fields like suspension/wakeup counters, while U5xx focuses on RX buffer and write-full status.

`ca_usb_dbg_hs_reset`

Header: `caos/drivers/inc/usb.h:175`
GUARD: `EN_USB`

```
void ca_usb_dbg_hs_reset(void)
```

Resets the internal high-speed USB debug statistics and counters to zero.

Ritorna:
void

Note:

- This function is non-blocking and performs a direct memory reset of the USB debug tracking variables. It is not thread-safe; if called while the USB interrupt service routine (ISR) is actively updating these statistics, you may experience race conditions or inconsistent data. It is intended for use during system initialization or when a manual reset of the debug telemetry is required. Note that this function is only implemented for specific hardware targets (like H7xx); on other platforms, it is an empty stub, so do not rely on it to clear state across different MCU architectures.

Uso:

- This function is typically used in a diagnostic context to clear performance metrics before starting a new measurement session. It is often paired with ‘ca_usb_dbg_print‘ to ensure that the statistics displayed represent only the activity occurring after the reset. Developers should call this during the system setup phase or via a CLI command to ensure that stale data from the boot process does not pollute the runtime performance analysis.

Esempio:

```
/* Reset USB debug stats before starting a new performance test */
ca_usb_dbg_hs_reset();

/* Perform some USB operations here */

/* Print the collected statistics to the console */
ca_usb_dbg_print(0U);
```

Q&A — fine-tuning review

5/5 pairs · gemini-3.1-flash-lite

Usage conf:1.00 **Q:** When should I call ca_usb_dbg_hs_reset in my application?
A: You should call it during the system setup phase or via a CLI command before starting a new measurement session. This ensures that stale data from the boot process does not pollute your runtime performance analysis.

Code example conf:0.90 **Q:** How do I implement a simple reset and print sequence using caOS types?
A: You can reset the statistics and then print them using the following pattern: ca_usb_dbg_hs_reset(); ca_usb_dbg_print();

Error/Edge case conf:1.00 **Q:** What happens if I call ca_usb_dbg_hs_reset while the USB ISR is running?
A: The function is not thread-safe. If called while the USB interrupt service routine is actively updating statistics, you may experience race conditions or inconsistent data.

Behavior conf:1.00 **Q:** Is ca_usb_dbg_hs_reset a blocking function?
A: No, it is non-blocking and performs a direct memory reset of the USB debug tracking variables.

HW differences conf:1.00 **Q:** Does ca_usb_dbg_hs_reset work the same way on all STM32 families supported by caOS?
A: No. It is only implemented for specific hardware targets like the H7xx. On other platforms like F1xx, F7xx, H5xx, and U5xx, the function is an empty stub and will not clear any state.

`ca_usb_dbg_hs_stat`

Header: `caos/drivers/inc/usb.h:174`
GUARD: `EN_USB`

```
1 void ca_usb_dbg_hs_stat(ca_usb_dbg_t *out)
```

Retrieves internal USB High-Speed controller diagnostic statistics and performance metrics.

Parametri:

Nome	Tipo	Descrizione
`out`	`ca_usb_dbg_t *`	Pointer to a structure where the USB statistics will be copied. Must not be NULL.

Ritorna:
None. If the pointer is NULL, the function returns immediately without performing any operations.

Note:

- This function is non-blocking and thread-safe in the sense that it performs a memory copy of internal diagnostic buffers. It is safe to call from an ISR, though it is primarily intended for diagnostic tasks or CLI commands. The function relies on internal USB debug counters that must be enabled via the EN_USB_DBG configuration flag; if disabled, the function will simply zero out the provided structure. A common mistake is assuming this function resets the counters; it only reads them. To reset the statistics, you must call ca_usb_dbg_hs_reset separately.

Uso:

- This function is used to monitor the health and performance of the USB High-Speed peripheral, specifically for debugging enumeration issues or throughput bottlenecks. It is typically called within a diagnostic task or a CLI command handler to dump the state of the USB bus, such as setup packet counts, endpoint transfer counts, and latency metrics. It is usually paired with ca_usb_dbg_hs_reset to clear counters before starting a new measurement period, and it is often wrapped by the generic ca_usb_dbg_stat function to provide a unified interface across different USB hardware implementations.

Esempio:

```c
ca_usb_dbg_t usb_stats;

// Clear existing stats before starting a new measurement
ca_usb_dbg_hs_reset();

// Perform some USB operations...

// Retrieve the current statistics
ca_usb_dbg_hs_stat(&usb_stats);

// Print the number of SETUP packets received on EP0
ca_printf("Setup packets: %u\n", usb_stats.setup);

// Check if the device is currently suspended
if (usb_stats.susp) {
    ca_printf("USB is in suspend mode\n");
}
```

Q&A — fine-tuning review

7/7 pairs · gemini-3.1-flash-lite

Usage conf:1.00 **Q:** When should I use ca_usb_dbg_hs_stat in my application?

A: You should use this function to monitor the health and performance of the USB High-Speed peripheral. It is typically called within a diagnostic task or a CLI command handler to retrieve metrics like setup packet counts, endpoint transfer counts, and latency, which helps in debugging enumeration issues or throughput bottlenecks.

Code example conf:1.00 **Q:** How do I implement a simple diagnostic call to retrieve USB statistics?

A: You can retrieve the statistics by declaring a ca_usb_dbg_t structure and passing its address to the function: ca_usb_dbg_t stats; ca_usb_dbg_hs_stat(&stats);

Parameters conf:1.00 **Q:** What are the requirements for the 'out' parameter in ca_usb_dbg_hs_stat?

A: The 'out' parameter must be a pointer to a ca_usb_dbg_t structure. It must not be NULL. If a NULL pointer is passed, the function returns immediately without performing any operations.

Return conf:1.00 **Q:** What does the function return, and how do I check for errors?

A: The function returns void. There is no explicit error return value; however, if the EN_USB_DBG configuration flag is disabled, the function will simply zero out the provided structure instead of populating it with diagnostic data.

Error/Edge case conf:1.00 **Q:** What is a common mistake when using ca_usb_dbg_hs_stat to track USB performance?

A: A common mistake is assuming that calling ca_usb_dbg_hs_stat resets the internal counters. It only reads them. To clear the statistics for a new measurement period, you must explicitly call ca_usb_dbg_hs_reset.

Behavior conf:1.00 **Q:** Is ca_usb_dbg_hs_stat safe to call from an interrupt service routine?

A: Yes, the function is non-blocking and thread-safe in the sense that it performs a memory copy of internal diagnostic buffers, making it safe to call from an ISR.

HW differences conf:1.00 **Q:** Are there differences in how ca_usb_dbg_hs_stat behaves across different STM32 families?

A: Yes. On the H7xx family, the function performs a detailed copy of internal event buffers (g_evt) and latency metrics (lat_sum, lat_max, etc.). On the F1xx, F7xx, H5xx, and U5xx families, the function is currently implemented to simply zero out the provided structure if EN_USB_DBG is enabled.

ca_usb_dbg_print

Header: caos/drivers/inc/usb.h:181
Source: caos/usb/src/usb_mgmt.c:433:453
GUARD: EN_USB

```
void ca_usb_dbg_print(u32 usb_num)
```

Prints a detailed diagnostic report of the USB peripheral state to the system console.

Parametri:

Nome	Tipo	Descrizione
usb_num	u32	The index of the USB peripheral to diagnose. Must be a valid peripheral ID supported by the hardware (e.g., 0 for FS, 1 for HS).

Ritorna:
void

Note:

- This function is a diagnostic tool and is not intended for high-frequency use in production loops. It is not thread-safe if multiple tasks attempt to print to the console simultaneously, though it relies on ca_stdout_printf_n which handles output buffering. It must not be called from an ISR because it performs significant string formatting and console output operations, which are blocking and time-consuming. Ensure the USB peripheral is initialized before calling this, otherwise, the internal status structure will contain garbage data.

Uso:

- Used primarily during development or troubleshooting to verify USB enumeration, endpoint activity, and interrupt performance. It is typically paired with CLI handlers to allow developers to dump the state of the USB stack on demand. In power management scenarios, it is used to verify that the USB peripheral has correctly resumed or re-enumerated after a low-power state transition by comparing the output before and after the sleep period.

Esempio:

```
/* Example: CLI command handler to dump USB status */
static bool usb_dbg_cli_handler(c_str *pcmd) {
    const char *p = ca_skip_sp((*pcmd)->data);
    /* Check if user typed 'usbdbg' */
    if (ca_strncmp(p, "usbdbg", 6) == 0) {
        /* Get the active USB peripheral number */
        u32 usb_id = ca_usb_dev_mng_get_num();
        /* Print diagnostic info to console */
        ca_usb_dbg_print(usb_id);
        return true;
    }
    return false;
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_usb_dbg_print function in my caOS project?
A: It is primarily used during development or troubleshooting to verify USB enumeration, endpoint activity, and interrupt performance. It is also useful in power management scenarios to verify that the USB peripheral has correctly resumed or re-enumerated after a low-power state transition.

Code example conf:1.00 **Q:** Provide a minimal example of how to call ca_usb_dbg_print for the FS USB peripheral.
A: u32 usb_id = 0; ca_usb_dbg_print(usb_id);

Parameters conf:1.00 **Q:** What does the usb_num parameter represent and what are valid values?
A: The usb_num parameter is the index of the USB peripheral to diagnose. It must be a valid peripheral ID supported by the hardware, such as 0 for FS (Full Speed) or 1 for HS (High Speed).

Error/Edge case conf:1.00 **Q:** What happens if I call ca_usb_dbg_print before initializing the USB peripheral?
A: If the USB peripheral is not initialized before calling this function, the internal status structure will contain garbage data, leading to an inaccurate diagnostic report.

Behavior conf:1.00 **Q:** Is ca_usb_dbg_print safe to call from an Interrupt Service Routine (ISR) or in a multi-threaded environment?
A: No, it must not be called from an ISR because it performs significant, time-consuming, and blocking string formatting and console output operations. Additionally, it is not thread-safe if multiple tasks attempt to print to the console simultaneously.

`ca_usb_dbg_reset`

Header: `caos/drivers/inc/usb.h:179`
Source: `caos/usb/src/usb_mgmt.c:423:431`
GUARD: `EN_USB`

```
void ca_usb_dbg_reset(void)
```

Resets the internal USB debug counters and statistics for all enabled USB controllers.

Ritorna:
void

Note:

- This function is a wrapper that triggers a reset of the internal diagnostic counters for High-Speed (HS) and Full-Speed (FS) USB controllers, depending on the build configuration. It is not thread-safe in the sense that it modifies global state; if called while another task is reading statistics, the data might be inconsistent. It is generally safe to call from a task context, but you should avoid calling it from an ISR unless the underlying hardware-specific reset functions are explicitly documented as ISR-safe. The most common mistake is assuming this resets the USB hardware connection itself; it only clears the software-level debug counters used for performance monitoring.

Uso:

- This function is typically used during system maintenance or debugging sessions to clear accumulated statistics, such as interrupt counts or error flags, before starting a new test cycle. It is often paired with 'ca_usb_dbg_stat' to capture a baseline, followed by a period of USB activity, and then another 'ca_usb_dbg_stat' to analyze the delta. In a CLI environment, it is triggered by a user command to reset the performance metrics without requiring a full system reboot.

Esempio:

```
/* Example: Resetting USB debug counters via a CLI command */
#include "ca_os.h"

```

```c
void handle_usb_debug_command(char *p) {
    /* Check if the user requested a reset of the debug counters */
    if (ca_strncmp(p, "dbg reset", 9) == 0) {
        ca_usb_dbg_reset();
        /* Notify the user that the counters have been cleared */
        ca_printf("USB debug counters have been reset to zero.\n");
    } else if (ca_strncmp(p, "dbg stat", 8) == 0) {
        /* Retrieve and display current stats using ca_usb_dbg_stat */
        /* Note: ca_usb_dbg_stat is the standard way to read the counters */
        ca_printf("Displaying current USB statistics...\n");
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_usb_dbg_reset() and what is its purpose in a caOS project?
A: You should call ca_usb_dbg_reset() during debugging sessions to reset USB debug counters to zero. It is used to clear performance and activity statistics related to USB operations, which helps in analyzing USB behavior during development. The function is intended for debugging only and must not be used in production code.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_usb_dbg_reset() with caOS types?
A:

```c
#include "caos.h"

void example_function(void)
{
    ca_usb_dbg_reset();
}
```

Error/Edge case conf:0.90 **Q:** What are the common mistakes or edge cases when using ca_usb_dbg_reset()?
A: A common mistake is calling ca_usb_dbg_reset() from an ISR if the debug counters are shared, as it may cause concurrency issues or data corruption. Additionally, the function is not thread-safe and should only be called when the system is in a stable state. It is also not meant for use in production code and should only be used during debugging.

Behavior conf:0.90 **Q:** Is ca_usb_dbg_reset() blocking or non-blocking, and is it safe to call from an ISR or in a multi-threaded environment?
A: ca_usb_dbg_reset() is a blocking function and is not thread-safe. It should not be called from an ISR if the debug counters are shared, as it could lead to race conditions or corrupted data. It must be called only when the system is in a stable state and is intended for single-threaded debugging scenarios.

HW differences conf:0.85 **Q:** How does ca_usb_dbg_reset() differ between F4xx, H7xx, and U5xx MCU families in caOS?
A: In F4xx, ca_usb_dbg_reset() resets only the global debug structure and a setup counter. In H7xx, it resets multiple debug structures and various counters for both USB instances (0 and 1), including ISR RX, IAT, and latency counters per endpoint. U5xx uses a simpler reset of the global debug structure similar to F4xx but without additional counters.

`ca_usb_dbg_stat`

Header: `caos/drivers/inc/usb.h:178`
Source: `caos/usb/src/usb_mgmt.c:411:421`
GUARD: `EN_USB`

```
void ca_usb_dbg_stat(ca_usb_dbg_t *out, u32 usb_num)
```

Retrieves real-time diagnostic statistics from the specified USB controller hardware.

Parametri:

Nome	Tipo	Descrizione
out	ca_usb_dbg_t *	Pointer to a structure where the USB diagnostic data will be stored. Must not be NULL.
usb_num	u32	The index of the USB controller to query (0 for primary, 1 for secondary). Valid values depend on hardware configuration.

Ritorna:
This function returns void. If the provided pointer is NULL, the function safely does nothing (or clears the structure if supported).

Note:

- This function is a dispatcher that routes the request to the appropriate low-level driver (HS or FS) based on your system configuration. It is non-blocking and executes quickly, making it safe to call from most contexts, though it is primarily intended for CLI or monitoring tasks. It is not thread-safe in the sense that it does not lock the USB hardware; if the USB controller is undergoing a reset or heavy re-initialization, the data returned might be inconsistent. The most common beginner mistake is passing an uninitialized pointer or assuming the data is updated in real-time without considering the overhead of the USB interrupt activity it measures.

Uso:

- This function is used for system health monitoring and debugging USB communication issues. It is typically called within a CLI command handler or a background monitoring task to inspect endpoint status, interrupt counts, and latency metrics. You would typically pair this with other diagnostic tools to print the gathered data to the console, allowing you to verify if endpoints are stalled or if the interrupt service routine is processing data as expected.

Esempio:

```
void usb_monitor_task(void) {
    ca_usb_dbg_t usb_stats;
    // Retrieve stats for the first USB controller
    ca_usb_dbg_stat(&usb_stats, 0U);

    // Check if the controller has processed any RX interrupts
    if (usb_stats.isr_rx_cnt > 0U) {
        ca_printf_n("USB Controller 0 active. RX count: %lu", usb_stats.
    isr_rx_cnt);
    } else {
        ca_printf_n("USB Controller 0 idle or inactive.");
    }

    // Example of checking endpoint 1 status
    // The chep array contains bit-packed status information
    u32 ep1_status = usb_stats.chep[1];
    ca_printf_n("EP1 raw status: 0x%08lx", ep1_status);
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_usb_dbg_stat() and what is its purpose in debugging USB issues?

A: You should call ca_usb_dbg_stat() during debugging to print detailed statistics about the USB state, including endpoint transfer counts, buffer status, and packet loss information. It helps analyze endpoint behavior and identify issues like buffer overflows or dropped packets.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_usb_dbg_stat() with caOS types?
A:

```c
void example_usb_debug(void)
{
    // Ensure USB is initialized before calling
    ca_usb_dbg_stat(); // Prints USB debug info to stdout
}
```

Error/Edge case conf:0.90 **Q:** What are the common mistakes or error conditions when using ca_usb_dbg_stat()?
A: Common mistakes include calling the function without initializing the USB subsystem, calling it during active data transfers which may result in stale data, or ignoring the debug output that could indicate buffering issues or packet loss.

Behavior conf:0.95 **Q:** Is ca_usb_dbg_stat() blocking, ISR-safe, or thread-safe, and what are its preconditions?
A: ca_usb_dbg_stat() is a blocking function that does not return until printing is complete. It is not ISR-safe and not thread-safe. It requires the USB system to be initialized and functioning. It must not be called from an ISR.

`ca_usb_dev_mng_add`

Header: `caos/usb/inc/usb_mgmt.h:132`
Source: `caos/usb/src/usb_mgmt.c:363:376`
GUARD: `EN_USB_MGMT`

```c
bool ca_usb_dev_mng_add (const ca_usb_device_type_t *cfg)
```

Registers a new USB device configuration into the USB management system.

Parametri:

Nome	Tipo	Descrizione
`cfg`	`const ca_usb_device_type_t *`	Pointer to the USB device configuration structure. Must not be NULL, must contain valid operations, and must have a valid device type index.

Ritorna:
Returns true if the device was successfully registered, false if the configuration is invalid, the device type is already registered, or if there is an endpoint resource collision.

Note:

- This function is not thread-safe and is intended to be called during the system initialization phase before the USB stack is started. It performs a static check for endpoint (EP) collisions; if two different USB device types attempt to use the same hardware endpoint, the function will reject the second one. It is not callable from an ISR. A common mistake is attempting to add a device after calling ca_usb_dev_mng_start, which may lead to inconsistent USB enumeration states.

Uso:

- This function is part of the USB initialization sequence. It is typically used after ca_usb_dev_mng_init and before ca_usb_dev_mng_start. You use it to register specific USB classes (like CDC or custom HID) into the management array. If you need to change device configurations at runtime, you must first call ca_usb_dev_mng_remove to free the endpoint resources before adding a new configuration.

Esempio:

```c
/* Initialize USB management first */
if (ca_usb_dev_mng_init(CA_USB_SPEED_FS, 1U)) {
    /* Define and add a custom USB device configuration */
    static const ca_usb_device_type_t my_usb_cfg = { /* ... configuration data ... */ };

    if (ca_usb_dev_mng_add(&my_usb_cfg)) {
        ca_stdout_put_n("[USB] Device added successfully");
        /* Start the USB stack to allow host enumeration */
        ca_usb_dev_mng_start();
    } else {
        ca_stdout_put_n("[USB] Failed to add device: check for EP collisions");
    }
} else {
    ca_stdout_put_n("[USB] Manager init failed");
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to call ca_usb_dev_mng_add in the caOS initialization sequence?
A: It should be called during the system initialization phase, specifically after ca_usb_dev_mng_init and before ca_usb_dev_mng_start. It is intended to register specific USB classes like CDC or HID into the management system.

Code example conf:0.90 **Q:** How do I register a USB device configuration using caOS types?
A: You pass a pointer to a ca_usb_device_type_t structure to the function: ca_usb_device_type_t my_dev; bool status = ca_usb_dev_mng_add(&my_dev);

Parameters conf:1.00 **Q:** What are the requirements for the cfg parameter in ca_usb_dev_mng_add?
A: The cfg parameter must be a non-NULL pointer to a ca_usb_device_type_t structure. It must contain valid operations and a valid device type index.

Return conf:1.00 **Q:** What does a false return value from ca_usb_dev_mng_add indicate?
A: A false return indicates that the configuration is invalid, the device type is already registered, or there is an endpoint resource collision where two device types attempt to use the same hardware endpoint.

Error/Edge case conf:1.00 **Q:** What is a common mistake when using ca_usb_dev_mng_add?
A: A common mistake is attempting to add a device after calling ca_usb_dev_mng_start, which may lead to inconsistent USB enumeration states. Additionally, the function is not thread-safe and cannot be called from an ISR.

`ca_usb_dev_mng_cdc_cli`

Header: `caos/usb/inc/usb_cdc.h:91`
Source: `caos/usb/src/usb_cdc.c:343:358`
GUARD: `EN_USB_CDC`

```c
bool ca_usb_dev_mng_cdc_cli(void)
```

Initializes and starts the USB CDC device manager for CLI communication.

Ritorna:
Returns true if the USB device manager was successfully initialized, configured, and started; returns false if any step (init, add, or start) fails.

Note:

- This function is intended to be called during system startup, specifically within ca_os_main before the RTOS scheduler is fully operational. It is not thread-safe and must never be called from an ISR. The function performs a blocking delay (ca_os_delay_ms) to allow the USB hardware and clock tree to stabilize after the D+ pull-up is enabled. A common mistake is calling this function after the scheduler has started or inside a task, which can lead to race conditions during USB enumeration or system instability due to the blocking nature of the internal delays.

Uso:

- This function is typically used in the main initialization sequence of an embedded application to enable a USB-based Command Line Interface (CLI). It is usually paired with ca_cli_create_thread and ca_cli_set_sender to establish the full CLI environment. The sequence is: 1) Initialize system peripherals, 2) Call ca_usb_dev_mng_cdc_cli to bring up the USB stack, 3) Configure the CLI hooks, and 4) Start the RTOS scheduler.

Esempio:

```
void ca_os_main(void) {
    /* Initialize the USB CDC CLI interface */
    if (ca_usb_dev_mng_cdc_cli()) {
        /* Configure the CLI to use the USB CDC as the output stream */
        ca_cli_set_sender(STDOUT);
        /* Create the CLI task to process user commands */
        ca_cli_create_thread();
    } else {
        /* Handle initialization failure, e.g., log error or enter safe mode */
        /* ca_printf("USB CLI init failed!\r\n"); */
    }
    /* Start the RTOS scheduler */
    ca_os_run();
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to call ca_usb_dev_mng_cdc_cli in my application?
A: It should be called during the system startup sequence, specifically within ca_os_main before the RTOS scheduler is fully operational. It is typically used to enable a USB-based CLI before starting the scheduler.

Code example conf:0.90 **Q:** How do I implement the initialization sequence for the USB CLI in caOS?
A: You should initialize peripherals, call ca_usb_dev_mng_cdc_cli, configure CLI hooks, and then start the scheduler. Example: { bool status = ca_usb_dev_mng_cdc_cli(); if(status) { ca_cli_create_thread(); ca_cli_set_sender(my_sender_func); } }

Return conf:1.00 **Q:** What do the boolean return values of ca_usb_dev_mng_cdc_cli indicate?
A: The function returns true if the USB device manager was successfully initialized, configured, and started. It returns false if any of these steps (init, add, or start) fail.

Error/Edge case conf:1.00 **Q:** What are the common mistakes or risks when calling ca_usb_dev_mng_cdc_cli
A: A common mistake is calling this function after the scheduler has started or inside a task, which can lead to race conditions during USB enumeration or system instability due to the internal blocking delays.

Behavior conf:1.00 **Q:** Is ca_usb_dev_mng_cdc_cli thread-safe or ISR-safe?
A: No, it is not thread-safe and must never be called from an ISR. It performs a blocking delay (ca_os_delay_ms) to allow the USB hardware and clock tree to stabilize.

---

ca_usb_dev_mng_get_num

Header: `caos/usb/inc/usb_mgmt.h:144`
Source: `caos/usb/src/usb_mgmt.c:402:405`
GUARD: `EN_USB_MGMT`

```
u32 ca_usb_dev_mng_get_num(void)
```

Retrieves the internal identifier (index) of the currently active USB device.

Ritorna:
Returns a u32 representing the USB device index. This value is typically used as a handle for subsequent USB endpoint operations.

Note:

- This function is a simple getter for a static internal variable and is non-blocking. It is thread-safe in the sense that it performs a read operation on a variable, but it does not provide synchronization guarantees if the USB device is being reconfigured or removed simultaneously. It is safe to call from an ISR, as it does not involve any OS primitives or blocking logic. The most common mistake is assuming the returned index is valid even when the USB stack is not initialized or has been stopped; always ensure the USB subsystem is in a valid state before using this index to open endpoints.

Uso:

- This function is used as a prerequisite for almost all USB-related operations that require a device handle. In a typical lifecycle, it is called during the device configuration phase (e.g., inside an 'on_configure' callback) to obtain the index needed to open endpoints via 'ca_usb_ep_open'. It is also used in diagnostic contexts to identify which USB instance is being referenced when printing debug information or performing status checks.

Esempio:

```
/* Example: Initializing endpoints for a custom USB device */
static void custom_on_configure(const ca_usb_device_type_t *cfg) {
    /* Retrieve the current USB device index */
    u32 usb_idx = ca_usb_dev_mng_get_num();

    /* Use the index to open endpoints for data transfer */
    /* Assuming d->ep_data is a valid endpoint address from config */
    ca_usb_ep_open(usb_idx, 0x81U, 0x02U, 64U); /* Open IN endpoint */
    ca_usb_ep_open(usb_idx, 0x01U, 0x02U, 64U); /* Open OUT endpoint */

    /* Log the initialization for debugging */
    ca_printf("USB device %lu configured successfully\n", usb_idx);
}
```

Q&A — fine-tuning review

3/3 pairs · gemini-3.1-flash-lite

Usage conf:1.00 **Q:** When should I call ca_usb_dev_mng_get_num and why is it necessary?
A: You should call this function during the device configuration phase, such as within an 'on_configure' callback. It is necessary because it retrieves the internal identifier (index) required as a handle for subsequent USB operations, such as opening endpoints with ca_usb_ep_open.

Code example conf:0.90 **Q:** Provide a minimal C example showing how to retrieve the USB device index and use it to open an endpoint.
A: u32 dev_idx = ca_usb_dev_mng_get_num(); if (dev_idx != 0xFFFFFFFF) { ca_usb_ep_open(dev_idx, 0x81); }

Return conf:1.00 **Q:** What does the return value of ca_usb_dev_mng_get_num represent, and are there any risks if the USB stack is not initialized?
A: The function returns a u32 representing the index of the currently active USB device. A common mistake is assuming this index is valid when the USB stack is stopped or uninitialized; if the USB subsystem is not in a valid state, the returned index may not be usable for subsequent operations.

`ca_usb_dev_mng_init`

Header: `caos/usb/inc/usb_mgmt.h:129`
Source: `caos/usb/src/usb_mgmt.c:321:335`
GUARD: `EN_USB_MGMT`

```
bool ca_usb_dev_mng_init (ca_usb_speed_t speed, u32 usb_num)
```

Initializes the USB device management layer and prepares the hardware interface for a specific USB controller.

Parametri:

Nome	Tipo	Descrizione
`speed`	`ca_usb_speed_t`	The desired USB operating speed (e.g., CA_USB_SPEED_FS for Full Speed). Must be a valid enum value supported by the hardware.
`usb_num`	`u32`	The index of the USB peripheral to initialize (e.g., 0 or 1). Must be less than 8; values >= 8 are invalid and will cause initialization failure.

Ritorna:
Returns true if the USB management layer was successfully initialized. Returns false if the specified usb_num is already in use, if the hardware initialization fails, or if invalid parameters are provided.

Note:

- This function is not thread-safe and must be called during the system initialization phase before starting the RTOS scheduler or spawning USB-related tasks. It is strictly non-blocking and must NOT be called from an ISR, as it performs internal memory resets and hardware configuration that require a stable thread context. A common beginner mistake is calling this function multiple times without first calling a de-initialization or stop function, which will trigger the 'already in use' error. Ensure that the system clock and GPIOs associated with the USB peripheral are configured correctly before invoking this, as the underlying hardware init will fail otherwise.

Uso:

- This function is the mandatory entry point for any USB-based application in caOS. It is typically used in the following sequence: first, call ca_usb_dev_mng_init to set up the management layer; second, call ca_usb_dev_mng_add to register specific USB device classes (like CDC or HID); and finally, call ca_usb_dev_mng_start to begin the enumeration process. It is paired with ca_usb_dev_mng_add to build the device descriptor tree before the host is allowed to connect.

Esempio:

```c
/* Initialize USB peripheral 0 at Full Speed */
if (!ca_usb_dev_mng_init(CA_USB_SPEED_FS, 0U)) {
    /* Handle initialization error: check clocks or GPIO configuration */
    ca_stdout_put_n("USB Init Failed!\n");
} else {
    /* Add a device class header to the management list */
    if (!ca_usb_dev_mng_add(&my_device_header)) {
        ca_stdout_put_n("Failed to add USB device class\n");
    } else {
        /* Start the USB management process to allow host enumeration */
        ca_usb_dev_mng_start();
    }
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_usb_dev_mng_init in my caOS application?
A: This function must be called during the system initialization phase, specifically before starting the RTOS scheduler or spawning USB-related tasks. It serves as the mandatory entry point for USB applications, followed by ca_usb_dev_mng_add and ca_usb_dev_mng_start.

Code example conf:0.90 **Q:** Provide a minimal code snippet to initialize the USB management layer.
A: u32 usb_index = 0; bool status = ca_usb_dev_mng_init(CA_USB_SPEED_FS, usb_index); if (status) { /* Proceed to add device classes */ }

Parameters conf:1.00 **Q:** What are the constraints for the usb_num parameter?
A: The usb_num parameter represents the index of the USB peripheral. It must be a value less than 8. Any value of 8 or greater is considered invalid and will result in an initialization failure.

Return conf:1.00 **Q:** What does a false return value from ca_usb_dev_mng_init indicate?
A: A false return indicates that the initialization failed. This occurs if the specified usb_num is already in use, if the hardware initialization fails, or if invalid parameters were provided.

Error/Edge case conf:1.00 **Q:** Why does my code trigger an 'already in use' error when calling ca_usb_dev_mng_init?
A: This error occurs if you call the function multiple times without first calling a de-initialization or stop function. Additionally, ensure that system clocks and GPIOs associated with the USB peripheral are configured correctly before calling this function, as hardware initialization will fail otherwise.

`ca_usb_dev_mng_remove`

Header: `caos/usb/inc/usb_mgmt.h:135`
Source: `caos/usb/src/usb_mgmt.c:378:385`
GUARD: `EN_USB_MGMT`

```c
bool ca_usb_dev_mng_remove(ca_usb_type_t type)
```

Removes a registered USB device from the management system and frees its associated endpoint resources.

Parametri:

Nome	Tipo	Descrizione
type	ca_usb_type_t	The identifier of the USB device type to remove. Must be a valid enum value within the range (0, CA_USB_TYPE___MAX).

Ritorna:
Returns true if the device was found and successfully removed. Returns false if the type is invalid, out of range, or if no device is currently registered for that type.

Note:

- This function is not thread-safe and should be called within a controlled context where the USB management state is not being concurrently modified. It is not intended for use inside an ISR, as it performs internal management logic that assumes a stable environment. The most common mistake is attempting to remove a device that was never successfully added or has already been removed, which will result in a false return value. Ensure that any hardware-level USB activity associated with this device type is stopped or quiesced before calling this function to avoid dangling endpoint references.

Uso:

- This function is typically used during the teardown phase of a USB device lifecycle or when reconfiguring the USB stack at runtime. It is usually paired with ca_usb_dev_mng_add (not shown here but implied) to manage dynamic device registration. In a CLI or configuration task, it is used to safely unregister a specific USB class (like DFU or a custom interface) so that the endpoint mask can be updated and the resources reclaimed for other purposes.

Esempio:

```
/* Example: Removing a custom USB device via CLI command */
#include "ca_os.h"

void handle_usb_remove_command(char *p) {
    /* Check if the user requested to remove the custom USB device */
    if (ca_strncmp(p, "remove", 6) == 0) {
        /* Attempt to remove the device from the management table */
        bool success = ca_usb_dev_mng_remove(CA_USB_TYPE_CUSTOM);

        if (success) {
            ca_printf_n("[USB] Custom device removed successfully.");
        } else {
            /* This happens if the device was not present or type was invalid */
            ca_printf_n("[USB] Error: Could not remove custom device (not
   present).");
        }
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_usb_dev_mng_remove and what is its primary purpose?
A: This function is used during the teardown phase of a USB device lifecycle or when reconfiguring the USB stack at runtime. Its purpose is to remove a registered USB device from the management system and free its associated endpoint resources, allowing the endpoint mask to be updated and resources reclaimed.

Code example conf:0.90 **Q:** How do I implement a safe removal of a USB device in caOS?
A: Ensure hardware-level activity is quiesced first, then call the function: bool success = ca_usb_dev_mng_remove(CA_USB_TYPE_DFU); if (!success) { /* handle error */ }

Parameters conf:1.00 **Q:** What are the constraints for the ca_usb_type_t parameter?
A: The parameter must be a valid enum value within the range (0, CA_USB_TYPE___MAX). Providing a value outside this range will result in the function returning false.

Return conf:1.00 **Q:** What does a false return value from ca_usb_dev_mng_remove indicate?
A: A false return indicates that the removal failed. This occurs if the provided type is invalid or out of range, or if no device is currently registered for that type. A common mistake is attempting to remove a device that was never added or has already been removed.

`ca_usb_dev_mng_start`

Header: `caos/usb/inc/usb_mgmt.h:138`
Source: `caos/usb/src/usb_mgmt.c:387:393`
GUARD: `EN_USB_MGMT`

```
bool ca_usb_dev_mng_start (void)
```

Initializes USB descriptors and enables the D+ pull-up resistor to signal device presence to the host.

Ritorna:
Always returns true, indicating the USB stack has been instructed to start.

Note:

- This function is not thread-safe and should be called during the system initialization phase or from a dedicated management task. It is strictly non-blocking, meaning it triggers the hardware connection and returns immediately; however, the actual USB enumeration by the host happens asynchronously afterward. A common beginner mistake is to attempt to send data immediately after calling this function; you must wait for the host to complete enumeration (e.g., using ca_usb_cdc_wait_connected) before attempting any communication. Ensure that all USB device descriptors and endpoints have been configured via ca_usb_dev_mng_add before invoking this start function.

Uso:

- This function is the final step in the USB initialization sequence. It is typically used after calling ca_usb_dev_mng_add to register specific device classes (like CDC or HID). Once started, the device pulls the D+ line high, which alerts the USB host that a device is connected. It is often paired with ca_usb_dev_mng_stop when the device needs to be logically disconnected from the bus, such as during a system reset or power-saving mode.

Esempio:

```
/* Initialize USB CDC device and start the stack */
if (ca_usb_dev_mng_add(&my_cdc_header)) {
    /* Start the USB stack to enable D+ pull-up */
    ca_usb_dev_mng_start();

    /* Wait for the host to enumerate the device before sending data */
    if (ca_usb_cdc_wait_connected()) {
        ca_stdout_put_n("USB Host connected and enumerated");
    } else {
        ca_stdout_put_n("USB connection timeout");
    }
} else {
    ca_stdout_put_n("Failed to add USB device");
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_usb_dev_mng_start in my initialization sequence?
A: You should call it as the final step of the USB initialization sequence, specifically after all device descriptors and endpoints have been configured using ca_usb_dev_mng_add. It is used to signal the host that the device is present by enabling the D+ pull-up resistor.

Code example conf:0.90 **Q:** How do I correctly initialize the USB stack using caOS types?
A: You must first register your classes and then start the management stack. Example: { ca_usb_dev_mng_add(descriptor_ptr); bool status = ca_usb_dev_mng_start(); if (status) { ca_usb_cdc_wait_connected(); } }

Return conf:1.00 **Q:** What does the return value of ca_usb_dev_mng_start indicate, and what causes it to fail?
A: The function always returns true, which indicates that the USB stack has been successfully instructed to start. There are no documented failure conditions that result in a false return value.

`ca_usb_dev_mng_stop`

Header: `caos/usb/inc/usb_mgmt.h:141`
Source: `caos/usb/src/usb_mgmt.c:395:400`
GUARD: `EN_USB_MGMT`

```
bool ca_usb_dev_mng_stop (void)
```

Stops the USB device management service and disconnects the device from the host.

Ritorna:
Always returns true, indicating the stop command has been processed.

Note:

- This function is not thread-safe and should be called from a single control task to avoid race conditions with the USB state machine. It is a non-blocking operation that immediately triggers a hardware disconnect. It must only be called after the USB management system has been successfully initialized via ca_usb_dev_mng_init. A common mistake is calling this function while a data transfer is still in progress, which may lead to unexpected behavior or incomplete packets on the host side.

Uso:

- This function is typically used in command-line interfaces or system state machines to gracefully shut down USB communication. It is the counterpart to ca_usb_dev_mng_start and is often paired with it to toggle USB connectivity. In a typical lifecycle, you would call this to release the USB bus before entering a low-power mode or when reconfiguring the USB device descriptors.

Esempio:

```
/* Example: Stopping the USB service via a CLI command */
#include "ca_os.h"

void handle_usb_stop_command(char *cmd) {
    if (ca_strncmp(cmd, "stop", 4) == 0) {
        /* Stop the USB management service */
        bool success = ca_usb_dev_mng_stop();

        if (success) {
            ca_printf("USB service stopped successfully.\n");
        } else {
```

```
            ca_printf("Failed to stop USB service.\n");
        }
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_usb_dev_mng_stop in my caOS application?
A: You should call this function to gracefully shut down USB communication, such as when preparing to enter a low-power mode or when reconfiguring USB device descriptors. It is typically used in command-line interfaces or system state machines to release the USB bus.

Code example conf:0.90 **Q:** How do I implement a simple toggle for the USB service using caOS functions?
A: You can manage the USB lifecycle by calling the start and stop functions. Example: void toggle_usb(bool active) { if (active) { ca_usb_dev_mng_start(); } else { ca_usb_dev_mng_stop(); } }

Return conf:1.00 **Q:** What does the boolean return value of ca_usb_dev_mng_stop indicate, and can it fail?
A: The function always returns true, which indicates that the stop command has been processed. There are no documented failure states for the return value itself, though the function must only be called after a successful ca_usb_dev_mng_init.

`ca_usb_dfu_poll`

Header: `caos/usb/inc/usb_dfu.h:89`
Source: `caos/usb/src/usb_dfu.c:167:189`
GUARD: `EN_USB_DFU`

```
bool ca_usb_dfu_poll(u32 wait_ms)
```

Processes pending USB DFU (Device Firmware Upgrade) requests by waiting for a semaphore signal.

Parametri:

Nome	Tipo	Descrizione
`wait_ms`	`u32`	Time in milliseconds to wait for a DFU job. Use CA_WAIT_FOREVER to block indefinitely until a job arrives.

Ritorna:
Returns true if a DFU job was successfully processed, false if the timeout expired or the job type was invalid.

Note:

- This function is blocking; it uses ca_sem_timedwait internally to suspend the calling task until the USB interrupt handler signals that a new DFU block or manifest command has arrived. It is NOT thread-safe for concurrent calls and must be called from a single dedicated task. It is not callable from an ISR. A common beginner mistake is calling this function from multiple tasks or failing to provide a valid configuration pointer (s_cfg) before the USB host starts sending data, which will cause the function to return false.

Uso:

- This function is the core of the DFU update loop. It is typically used in a dedicated background task that runs in an infinite loop. The sequence is: 1) Initialize the system, 2)

Enter a while(1) loop, 3) Call ca_usb_dfu_poll with CA_WAIT_FOREVER, 4) Check the result and perform system actions like resetting the device via ca_os_reset once the update is complete. It pairs with the USB stack's interrupt-driven job posting mechanism.

Esempio:

```c
/* Dedicated task for handling DFU updates */
void dfu_task(void *arg) {
    while (1) {
        /* Wait for a DFU job from the USB host */
        if (ca_usb_dfu_poll(CA_WAIT_FOREVER)) {
            /* Job processed successfully */
            ca_printf("DFU block processed\n");
        } else {
            /* Handle error or timeout */
            ca_printf("DFU poll failed or timed out\n");
        }

        /* Check if we need to reboot after update */
        if (/* check if s_dfu_reset flag is set */ 1) {
            ca_os_sleep(1000);
            ca_os_reset();
        }
    }
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When and why should I call ca_usb_dfu_poll in my application?
A: You should call ca_usb_dfu_poll within a dedicated background task's infinite loop. It is used to process pending USB DFU requests, allowing the system to handle firmware update blocks or manifest commands sent by a USB host.

Code example conf:0.95 **Q:** Provide a minimal C code example for implementing the DFU update loop using caOS types.
A: void dfu_task(void *p) { while(1) { if(ca_usb_dfu_poll(CA_WAIT_FOREVER)) { /* Processed successfully */ } else { /* Handle timeout or error */ } } }

Parameters conf:1.00 **Q:** What does the wait_ms parameter control and what is the significance of CA_WAIT_FOREVER?
A: The wait_ms parameter specifies the time in milliseconds to wait for a DFU job. Using CA_WAIT_FOREVER allows the function to block indefinitely until a job arrives, which is the standard approach for a dedicated DFU background task.

Return conf:1.00 **Q:** What do the return values of ca_usb_dfu_poll indicate?
A: It returns true if a DFU job was successfully processed. It returns false if the specified timeout expired or if the received job type was invalid.

Error/Edge case conf:1.00 **Q:** What are the common mistakes or error conditions that cause ca_usb_dfu_poll to return false?
A: Common mistakes include failing to provide a valid configuration pointer (s_cfg) before the USB host starts sending data, or calling the function from multiple tasks simultaneously, which violates its thread-safety requirements.

Behavior conf:1.00 **Q:** Is ca_usb_dfu_poll safe to call from an ISR or multiple tasks?
A: No, it is not ISR-safe. Furthermore, it is not thread-safe for concurrent calls and must be called from a single dedicated task because it uses ca_sem_timedwait internally to suspend the calling task.

ca_usb_disconnect

Header: caos/drivers/inc/usb.h:109
Source: caos/drivers/src/usb.c:85:89
GUARD: EN_USB

```
void ca_usb_disconnect(u32 usb_num)
```

Performs a soft disconnect of the USB device by signaling the host to drop the connection.

Parametri:

Nome	Tipo	Descrizione
usb_num	u32	The index of the USB controller to disconnect. Must be less than USB_MAX_CTRL and correspond to an initialized controller.

Ritorna:
void. The function does not return a status code; it silently ignores invalid controller indices.

Note:

- This function is non-blocking and performs a 'soft' disconnect, meaning it pulls the D+ line low (or equivalent) to simulate a physical unplug event to the host. It is not inherently thread-safe if multiple tasks attempt to manipulate the same USB controller state simultaneously, so ensure proper synchronization if your application architecture allows concurrent access. It is generally safe to call from most contexts, but avoid calling it from high-priority ISRs if the underlying driver implementation performs complex operations or memory management. The most common beginner mistake is assuming this function physically powers down the USB peripheral; it only signals the host to stop communication, leaving the peripheral hardware in its current power state.

Uso:

- This function is typically used during system shutdown, device re-enumeration, or when the user explicitly requests a 'halt' or 'soft disconnect' via a CLI command. It is usually paired with ca_usb_connect to restore the link after a maintenance task or configuration change. In a typical lifecycle, you would call it as part of a teardown sequence (e.g., stopping the USB management service) to ensure the host does not attempt to send data to a device that is about to enter a low-power or idle state.

Esempio:

```c
/* Example: Handling a user command to halt the USB interface */
#define USB_NUM 0

void handle_usb_halt_command(void) {
    /* Check if the USB controller is currently active */
    if (ca_usb_cdc_connected(USB_NUM)) {
        /* Signal the host that the device is disconnecting */
        ca_usb_disconnect(USB_NUM);

        /* Log the action to the system console */
        ca_printf("USB controller %d has been soft-disconnected.\n", USB_NUM);
    } else {
        ca_printf("USB controller %d is already disconnected.\n", USB_NUM);
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_usb_disconnect and what is its purpose in a caOS application?
A: You should call ca_usb_disconnect when you need to physically disconnect a specific USB interface in your caOS application. This function is useful for implementing soft USB device control, such as simulating a USB unplugging event. It is intended for use in main control contexts or RTOS tasks, not from ISR, as it is a blocking and non-thread-safe function.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_usb_disconnect with caOS types?
A: u32 usb_num = 0; ca_usb_disconnect(usb_num);

Parameters conf:0.90 **Q:** What does the usb_num parameter in ca_usb_disconnect represent and what are valid values?
A: The usb_num parameter represents the identification number of the USB interface to disconnect. It must be a valid value between 0 and the maximum number of USB interfaces available on the device. Values that are negative or greater than the maximum number of USB interfaces will cause undefined behavior.

`ca_usb_ep_clear_stall`

Header: `caos/drivers/inc/usb.h:123`
Source: `caos/drivers/src/usb.c:103:107`
GUARD: `EN_USB`

```
bool ca_usb_ep_clear_stall(u32 usb_num, u8 ep_addr)
```

Clears the STALL condition on a specific USB endpoint to resume normal data communication.

Parametri:

Nome	Tipo	Descrizione
`usb_num`	`u32`	The index of the USB controller (e.g., 0 for FS, 1 for HS). Must be less than USB_MAX_CTRL.
`ep_addr`	`u8`	The hardware address of the endpoint (e.g., 0x81 for IN endpoint 1, 0x02 for OUT endpoint 2).

Ritorna:
Returns true if the stall was successfully cleared, false if the controller index is invalid or the operation failed.

Note:

- This function is typically non-blocking and interacts directly with the USB hardware registers. It is not guaranteed to be thread-safe if called simultaneously from multiple tasks, so it should ideally be called from the USB management task or the USB ISR context. A common beginner mistake is calling this function without first verifying if the endpoint is actually stalled using ca_usb_ep_is_stalled, or failing to handle the subsequent USB protocol requirements (like sending a Zero Length Packet) after clearing the stall. Ensure the USB controller is initialized before calling this.

Uso:

- This function is used during USB enumeration or runtime when the host sends a CLEAR_FEATURE request to an endpoint. It is part of the standard USB control transfer sequence: the firmware receives a setup packet, identifies the request, calls ca_usb_ep_clear_stall to reset the endpoint's data toggle and stall bit, and then typically calls ca_usb_ep_write

to send a status response (ZLP) back to the host. It is often paired with ca_usb_ep_stall when the device needs to signal an error to the host.

Esempio:

```c
/* Example: Handling a USB CLEAR_FEATURE request for an endpoint */
void handle_usb_request(u32 usb_num, u8 *setup_packet) {
    u8 request = setup_packet[1];
    u8 ep_addr = setup_packet[4];

    /* Check if the host is requesting to clear a stall condition */
    if (request == 0x01U) { /* USB_REQ_CLEAR_FEATURE */
        if (ca_usb_ep_clear_stall(usb_num, ep_addr)) {
            /* Successfully cleared stall, now send ZLP to acknowledge status */
            ca_usb_ep_write(usb_num, 0x80U, NULL, 0U);
        } else {
            /* Log error or handle failure to clear stall */
            /* ca_printf("Failed to clear stall on EP 0x%02X\n", ep_addr); */
        }
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_usb_ep_clear_stall and why is it important in USB communication?
A: You should call ca_usb_ep_clear_stall when an endpoint is in a STALL state, typically after an error condition such as a USB transfer error or a protocol violation. This function is important because it allows the endpoint to recover from the STALL state and resume normal USB communication. It is usually invoked after detecting an error condition and before attempting to re-initiate a USB transfer.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_usb_ep_clear_stall with caOS types?
A: bool result = ca_usb_ep_clear_stall(0, 0x81); if (result) { // Endpoint cleared successfully } else { // Handle error }

Parameters conf:0.90 **Q:** What are the valid ranges for usb_num and ep_addr parameters in ca_usb_ep_clear_stall, and what happens if they are out of range?
A: The usb_num parameter must be a valid USB controller number between 0 and the maximum number of available USB controllers. An invalid value may cause a crash or undefined behavior. The ep_addr parameter must be a valid endpoint address: 0x00-0x0F for IN endpoints and 0x80-0x8F for OUT endpoints. Using an invalid endpoint address may result in hardware errors or unexpected behavior.

Return conf:0.90 **Q:** What does the return value of ca_usb_ep_clear_stall indicate, and what causes it to return false?
A: The function returns true if the endpoint was successfully cleared from the STALL state. It returns false if the operation failed, which can be caused by an invalid endpoint address, an unavailable USB controller, or a hardware error. The function also may return false if the endpoint was not in a STALL state to begin with.

`ca_usb_ep_close`

Header: `caos/drivers/inc/usb.h:119`
Source: `caos/drivers/src/usb.c:97:101`
GUARD: `EN_USB`

```c
bool ca_usb_ep_close(u32 usb_num, u8 ep_addr)
```

Closes a specific USB endpoint to stop data transmission and release associated resources.

Parametri:

Nome	Tipo	Descrizione
`usb_num`	`u32`	The index of the USB controller (e.g., 0 for FS, 1 for HS). Must be less than USB_MAX_CTRL.
`ep_addr`	`u8`	The address of the endpoint to close (e.g., 0x81 for IN endpoint 1, 0x02 for OUT endpoint 2).

Ritorna:
Returns true if the endpoint was successfully closed, or false if the controller index is invalid or the operation failed.

Note:

- This function is not thread-safe and should not be called from an ISR. It is a blocking operation that interacts with the low-level USB driver to disable the hardware endpoint. A common mistake is calling this function while a transfer is still active or without checking if the USB controller is actually initialized, which can lead to undefined behavior or system instability. Always ensure the USB stack is in a state where closing the endpoint is safe, such as during a device disconnect or a configuration change.

Uso:

- This function is typically used during USB device state transitions, such as moving from an 'configured' state to an 'idle' or 'disconnected' state. It is often paired with ca_usb_ep_open to manage dynamic endpoint allocation. In a typical sequence, you would call ca_usb_ep_close when the host requests a configuration change or when the device is unplugged, ensuring that no further data is processed on that specific endpoint.

Esempio:

```c
/* Example: Closing USB endpoints during a device reset or disconnect event */
#define USB_NUM 0
#define EP_IN_ADDR 0x81
#define EP_OUT_ADDR 0x01

void handle_usb_disconnect(void) {
    /* Attempt to close the bulk endpoints to stop communication */
    bool closed_in = ca_usb_ep_close(USB_NUM, EP_IN_ADDR);
    bool closed_out = ca_usb_ep_close(USB_NUM, EP_OUT_ADDR);

    if (closed_in && closed_out) {
        ca_printf_n("[USB] Endpoints closed successfully.");
    } else {
        /* Log an error if the hardware failed to release the endpoints */
        ca_printf_n("[USB] Error: Failed to close one or more endpoints.");
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_usb_ep_close and why is it important?
A: You should call ca_usb_ep_close when you want to close a specific USB endpoint that was previously opened with ca_usb_ep_open(). This function is important for resource management, especially after completing a USB transfer or when shutting down USB communication. It helps free up hardware resources and ensures proper cleanup of endpoint state.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_usb_ep_close in C with caOS types?

A: u32 usb_num = 0; u8 ep_addr = 0x81; bool result = ca_usb_ep_close(usb_num, ep_addr); if (!result) { // Handle error }

Parameters conf:0.90 **Q:** What do the parameters usb_num and ep_addr represent in ca_usb_ep_close, and what are valid values?

A: The parameter usb_num specifies the USB controller number (e.g., 0 for USB0), and must be a valid index between 0 and the maximum number of USB controllers available. The ep_addr specifies the endpoint address, where the MSB indicates direction (0 = OUT, 1 = IN). Valid endpoint addresses are typically 0x00 to 0x0F, with values outside this range potentially causing undefined behavior.

Return conf:0.95 **Q:** What does ca_usb_ep_close return, and what causes it to return false?

A: ca_usb_ep_close returns true if the endpoint was successfully closed. It returns false if the operation failed, which can happen if the endpoint was not previously opened, if the usb_num is invalid, or if a hardware error occurs during the closing process.

`ca_usb_ep_is_stalled`

Header: `caos/drivers/inc/usb.h:126`
Source: `caos/drivers/src/usb.c:109:113`
GUARD: `EN_USB`

```
bool ca_usb_ep_is_stalled(u32 usb_num, u8 ep_addr)
```

Checks if a specific USB endpoint is currently in a stalled state.

Parametri:

Nome	Tipo	Descrizione
`usb_num`	`u32`	The index of the USB controller. Must be less than USB_MAX_CTRL.
`ep_addr`	`u8`	The address of the endpoint to query (e.g., 0x81 for IN endpoint 1).

Ritorna:
Returns true if the endpoint is stalled, false otherwise. Returns false if the usb_num is invalid or the driver is not initialized.

Note:

- This function is non-blocking and performs a direct read of the endpoint status register. It is generally thread-safe depending on the underlying driver implementation, but it is not intended to be called from an ISR unless the driver explicitly supports it. The most common mistake is passing an invalid endpoint address or an uninitialized USB controller index, which will silently return false without indicating an error.

Uso:

- This function is primarily used during USB enumeration or control transfers when the host requests the status of an endpoint via the GET_STATUS request. It is typically paired with ca_usb_ep_write to send the status back to the host. Developers use this to verify if a previous error condition (stall) has been cleared or to report the current state of the communication pipe to the USB host.

Esempio:

```
/* Example: Handling a USB GET_STATUS request for an endpoint */
void handle_usb_get_status(u32 usb_num, u8 *setup, u16 wln) {
    u8 ep_addr = setup[4];
    /* Check if the requested endpoint is stalled */
```

```
    bool stalled = ca_usb_ep_is_stalled(usb_num, ep_addr);

    /* Prepare status response: bit 0 is stall status */
    u8 status[2] = { stalled ? 0x01U : 0x00U, 0x00U };

    /* Send the status back to the host using the control endpoint */
    ca_usb_ep_write(usb_num, 0x80U, status, (wln < 2U) ? wln : 2U);
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_usb_ep_is_stalled and why is it useful in USB communication?
A: You should call ca_usb_ep_is_stalled to check if a USB endpoint is currently in a stalled state, which indicates a communication error. This is useful for error handling and recovery in USB protocols, especially when implementing robust USB device drivers. It allows the application to detect and respond to endpoint stalls without blocking execution.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_usb_ep_is_stalled with caOS types?
A: u32 usb_num = 0; u8 ep_addr = 0x81; bool is_stalled = ca_usb_ep_is_stalled(usb_num, ep_addr); if (is_stalled) { // Handle stall condition }

Parameters conf:0.90 **Q:** What are the valid ranges for usb_num and ep_addr parameters in ca_usb_ep_is_stalled?
A: The usb_num parameter must be a valid USB controller number between 0 and the maximum number of available USB controllers. The ep_addr parameter must be between 0x00-0x0F for IN endpoints or 0x80-0x8F for OUT endpoints. Passing invalid values may cause undefined behavior or errors.

Return conf:0.85 **Q:** What does the return value of ca_usb_ep_is_stalled indicate and what causes it to fail?
A: The function returns true if the endpoint is currently in a stalled state, and false if it is not. Failures can occur if the USB controller is not initialized, the endpoint is not configured correctly, or invalid parameters are passed such as an unconfigured endpoint or invalid USB number.

`ca_usb_ep_open`

Header: `caos/drivers/inc/usb.h:90`
Source: `caos/drivers/src/usb.c:43:47`
GUARD: `EN_USB`

```
bool ca_usb_ep_open(u32 usb_num, u8 ep_addr, ca_usb_ep_type_t type, u16
    mps)
```

Initializes and enables a specific USB endpoint for data transmission or reception.

Parametri:

Nome	Tipo	Descrizione
usb_num	u32	The index of the USB controller (e.g., 0 for FS, 1 for HS). Must be less than USB_MAX_CTRL.
ep_addr	u8	The endpoint address (e.g., 0x81 for IN endpoint 1, 0x02 for OUT endpoint 2). Bit 7 is set for IN endpoints.
type	ca_usb_ep_type_t	The transfer type: CA_USB_EP_TYPE_BULK, CA_USB_EP_TYPE_INTR, or CA_USB_EP_TYPE_ISOC.
mps	u16	Maximum Packet Size in bytes. Must match the descriptor requirements for the chosen endpoint type.

Ritorna:
Returns true if the endpoint was successfully opened, false if the controller index is invalid or the hardware failed to configure the endpoint.

Note:

- This function is not thread-safe and must be called during the USB configuration phase, typically triggered by the host's SET_CONFIGURATION request. It is not intended to be called from an ISR. The USB controller must be fully initialized via the driver's init sequence before calling this. A common mistake is providing an incorrect MPS value that does not align with the USB specification for the specific endpoint type, which will cause the host to reject the device.

Uso:

- This function is used during the USB device enumeration process to prepare endpoints for data exchange. It is typically called inside a callback function (like an 'on_configure' handler) after the USB device has been assigned an address by the host. It is usually paired with ca_usb_ep_write or ca_usb_ep_read to perform actual data transfers once the endpoints are open.

Esempio:

```
/* Example: Opening endpoints for a custom USB device */
void custom_on_configure(const ca_usb_device_type_t *cfg) {
    u32 n = ca_usb_dev_mng_get_num();
    u16 mps = 64; // Standard Full-Speed Bulk MPS

    // Open Bulk IN endpoint (0x81)
    if (!ca_usb_ep_open(n, 0x81, CA_USB_EP_TYPE_BULK, mps)) {
        // Handle error: log failure or trigger system reset
        return;
    }

    // Open Bulk OUT endpoint (0x01)
    if (!ca_usb_ep_open(n, 0x01, CA_USB_EP_TYPE_BULK, mps)) {
        // Handle error: log failure
        return;
    }

    // Endpoints are now ready for ca_usb_ep_write or ca_usb_ep_read
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_usb_ep_open in a caOS application?

A: You should call ca_usb_ep_open after initializing the USB hardware with ca_usb_hw_init and configuring the GPIOs appropriately. It is used to open a specific USB endpoint for data transfer, typically before starting any USB communication such as sending or receiving data on that endpoint.

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_usb_ep_open with caOS types?
A: u32 usb_num = 0; u8 ep_addr = 0x81; ca_usb_ep_type_t type = CA_USB_EP_TYPE_BULK; u16 mps = 64; bool result = ca_usb_ep_open(usb_num, ep_addr, type, mps);

Parameters conf:0.90 **Q:** What are the valid values for the endpoint address and type parameters in ca_usb_ep_open?
A: The endpoint address (ep_addr) must be between 0x00-0x0F for IN endpoints or 0x80-0x8F for OUT endpoints. The type parameter must be one of CA_USB_EP_TYPE_CTRL, USB_EP_TYPE_ISO, CA_USB_EP_TYPE_BULK, or USB_EP_TYPE_INT. Using invalid values may cause hardware errors or unpredictable behavior.

Return conf:0.90 **Q:** What does it mean when ca_usb_ep_open returns false?
A: A return value of false indicates that the endpoint could not be opened successfully. This can happen due to invalid parameters, insufficient resources, or hardware errors. It is important to check the parameters passed to the function and ensure the USB hardware is properly initialized before calling this function.

`ca_usb_ep_read`

Header: `caos/drivers/inc/usb.h:96`
Source: `caos/drivers/src/usb.c:55:59`
GUARD: `EN_USB`

```
u16 ca_usb_ep_read(u32 usb_num, u8 ep_addr, u8 *buf, u16 max_len)
```

Reads data from a specific USB endpoint into a provided buffer.

Parametri:

Nome	Tipo	Descrizione
`usb_num`	`u32`	The index of the USB controller. Must be less than USB_MAX_CTRL.
`ep_addr`	`u8`	The endpoint address (e.g., 0x00 for EP0, 0x81 for IN, 0x01 for OUT).
`buf`	`u8 *`	Pointer to the memory buffer where received data will be stored.
`max_len`	`u16`	The maximum number of bytes to read. Must not exceed the buffer size.

Ritorna:
Returns the number of bytes actually read (u16). Returns 0 if the USB controller index is invalid, the controller is not initialized, or no data is available.

Note:

- This function is a wrapper that delegates to the specific hardware driver implementation. It is generally non-blocking in the sense that it returns immediately with the data currently available in the endpoint buffer. It is not inherently thread-safe if multiple tasks access the same endpoint simultaneously, so ensure proper synchronization or task-exclusive access. Do not call this from an ISR unless the underlying driver implementation explicitly supports it, as it may involve complex buffer management or synchronization primitives that are unsafe in interrupt context. A common mistake is providing a buffer smaller than the actual incoming packet size, which can lead to memory corruption or truncated data.

Uso:

- This function is typically used within the USB stack's data processing loop or inside a callback triggered by a USB event. It is used to retrieve control requests (like SET_LINE_CODING in CDC) or bulk data (like CBW in MSC). It is usually paired with ca_usb_ep_write to send responses or status packets back to the host. In a typical sequence, you wait for a USB event, call this function to pull the data into a local buffer, process the data, and then potentially send an acknowledgment or the requested data back.

Esempio:

```c
/* Example: Reading a setup packet from EP0 */
u32 n = ca_usb_dev_mng_get_num();
u8 rx_buffer[8];

/* Attempt to read up to 8 bytes from Endpoint 0 */
u16 bytes_received = ca_usb_ep_read(n, 0x00U, rx_buffer, sizeof(rx_buffer));

if (bytes_received >= 7U) {
    /* Process the received setup data */
    ca_printf("Received %d bytes from host\n", bytes_received);
} else {
    /* Handle error or empty buffer case */
    ca_printf("No data or incomplete packet received\n");
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_usb_ep_read in a caOS application?
A: You should call ca_usb_ep_read when you need to receive data from a USB endpoint in a blocking manner. It is typically used after configuring and activating a USB endpoint, and it will wait until data arrives or a hardware timeout occurs. This function is not suitable for use inside an ISR because it is blocking and may cause deadlocks.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_usb_ep_read with caOS types?
A: u8 buffer[64]; u16 bytes_read = ca_usb_ep_read(0, 0x81, buffer, 64); if (bytes_read != 0 && bytes_read != 0xFFFF) { // Process received data }

Parameters conf:0.90 **Q:** What are the valid ranges for the usb_num and ep_addr parameters in ca_usb_ep_read?
A: The usb_num parameter must be either 0 or 1, representing USB0 or USB1 interfaces respectively. The ep_addr parameter must be between 0x00 and 0x0F, indicating the endpoint address. Values outside these ranges are considered invalid and may lead to errors.

Return conf:0.95 **Q:** What does a return value of 0xFFFF from ca_usb_ep_read indicate?
A: A return value of 0xFFFF indicates a critical error, such as an invalid buffer pointer or an invalid endpoint address. This means the function could not proceed due to a fundamental issue with the input parameters.

`ca_usb_ep_stall`

Header: `caos/drivers/inc/usb.h:112`
Source: `caos/drivers/src/usb.c:91:95`
GUARD: `EN_USB`

```c
void ca_usb_ep_stall(u32 usb_num, u8 ep_addr)
```

Forces a USB endpoint into a STALL state to signal a protocol error or unsupported request to the host.

Parametri:

Nome	Tipo	Descrizione
usb_num	u32	The index of the USB controller (e.g., 0 for FS, 1 for HS). Must be less than USB_MAX_CTRL.
ep_addr	u8	The address of the endpoint to stall (e.g., 0x80 for IN endpoint 0, 0x01 for OUT endpoint 1).

Ritorna:
void. If the parameters are invalid, the function returns silently without performing any action.

Note:

- This function is non-blocking and typically executes immediately by setting the STALL bit in the endpoint control register. It is generally thread-safe depending on the underlying hardware driver implementation, but it is not intended to be called from an ISR unless the driver explicitly supports it. The most common beginner mistake is calling this on an endpoint that is not yet initialized or already closed, which may lead to undefined behavior in the USB stack. Always ensure the USB controller is initialized and the endpoint is active before attempting to stall it.

Uso:

- This function is used during USB enumeration or data transfer to handle protocol violations, such as receiving an invalid Command Block Wrapper (CBW) in Mass Storage Class or an unsupported request during SETUP phase. It is typically paired with ca_usb_ep_open during initialization and ca_usb_ep_clear_stall when the error condition is resolved. In a typical workflow, if a host sends a request that the device cannot fulfill, the device calls this function to inform the host that the endpoint is halted, effectively forcing the host to perform error recovery.

Esempio:

```c
/* Example: Handling an unsupported USB request */
void handle_usb_setup(u32 usb_num, u8 *setup_packet) {
    /* Check if the request type is supported */
    if (setup_packet[1] != 0x01) {
        /* Unsupported request: Stall the control endpoint (0x80) */
        ca_usb_ep_stall(usb_num, 0x80U);
    } else {
        /* Proceed with valid request handling */
        // ... logic to process request ...
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_usb_ep_stall() in a USB communication context?
A: You should call ca_usb_ep_stall() when you need to temporarily halt data transmission on a specific USB endpoint, typically to signal an error or to indicate that the endpoint is not ready to receive more data. It is a low-level function used for direct hardware control and must be used carefully in control contexts.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_usb_ep_stall() with caOS types?
A: u32 usb_num = 0; u8 ep_addr = 0x80; ca_usb_ep_stall(usb_num, ep_addr);

Parameters conf:0.90 **Q:** What do the parameters usb_num and ep_addr represent in ca_usb_ep_stall(), and what are valid values for them?

A: The parameter usb_num specifies the USB interface number (e.g., 0 for USB0), and must be a valid interface number defined in the system. The parameter ep_addr specifies the USB endpoint address (e.g., 0x80 for an IN endpoint, 0x00 for a control endpoint). Invalid values for either parameter may cause crashes or undefined behavior.

---

ca_usb_ep_write

Header: caos/drivers/inc/usb.h:93
Source: caos/drivers/src/usb.c:49:53
GUARD: EN_USB

```
bool ca_usb_ep_write(u32 usb_num, u8 ep_addr, const u8 *buf, u16 len)
```

Transmits data from the device to the USB host via a specific endpoint.

Parametri:

Nome	Tipo	Descrizione
usb_num	u32	The index of the USB controller instance. Must be less than USB_MAX_CTRL.
ep_addr	u8	The endpoint address (e.g., 0x80 for IN endpoint 0). Must be an endpoint previously opened with ca_usb_ep_open.
buf	const u8 *	Pointer to the data buffer to send. Can be NULL if len is 0 (used for ZLP - Zero Length Packet).
len	u16	Number of bytes to transmit. Must not exceed the maximum packet size configured for the endpoint.

Ritorna:
Returns true if the write request was successfully queued, false if the controller is invalid, the endpoint is not open, or the hardware is busy.

Note:

- This function is non-blocking and typically queues the data for the USB hardware to transmit. It is not inherently thread-safe for concurrent access to the same endpoint, so ensure only one task or ISR manages a specific endpoint at a time. It is a mandatory precondition that the USB controller is initialized and the specific endpoint has been opened via ca_usb_ep_open. A common beginner mistake is passing a buffer that goes out of scope (e.g., a local stack variable) before the USB hardware has finished transmitting the data, which leads to memory corruption on the wire.

Uso:

- This function is used in the USB stack to respond to host requests or send asynchronous data. It is typically paired with ca_usb_ep_open during initialization to prepare the endpoint. In control transfers, it is frequently used to send status packets or data descriptors, often with a NULL buffer and 0 length to signal a Zero Length Packet (ZLP) to the host, indicating the end of a control transaction.

Esempio:

```
/* Example: Sending a status response to the USB host */
#define USB_CTRL_IDX 0
#define EP_IN_0 0x80U

```

```c
void send_usb_status(void) {
    /* Send a Zero Length Packet (ZLP) to acknowledge a control request */
    bool success = ca_usb_ep_write(USB_CTRL_IDX, EP_IN_0, NULL, 0U);

    if (!success) {
        /* Log error: failed to queue USB packet */
        ca_printf("USB write failed!\n");
    }
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_usb_ep_write() in a caOS application?
A: You should call ca_usb_ep_write() when you need to send data from your application to a USB host via a specific USB endpoint. It is typically used after opening an endpoint with ca_usb_ep_open() and is essential for USB communication in device mode.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_usb_ep_write() with caOS types?
A: u8 data[64] = {0x01, 0x02, 0x03}; bool success = ca_usb_ep_write(0, 0x81, data, 3);

Parameters conf:0.90 **Q:** What are the valid values for the usb_num and ep_addr parameters in ca_usb_ep_write()?
A: The usb_num parameter must be 0 or 1, corresponding to USB1 and USB2 controllers respectively. The ep_addr parameter must be an IN endpoint address between 0x80 and 0x8F, where bit 7 indicates the direction (1 for IN). Using invalid values may cause undefined behavior or errors.

Return conf:0.95 **Q:** What does ca_usb_ep_write() return, and what causes it to return false?
A: The function returns true if the data was successfully written to the USB endpoint. It returns false if the operation failed, which can happen due to an unopened endpoint, a NULL buffer, or if the endpoint is busy or unavailable.

`ca_usb_fs_dev_clock_init`

Header: `caos/drivers/inc/ca_glue_hw.h:31`
GUARD: `EN_CA_GLUE_HW`

```c
extern void ca_usb_fs_dev_clock_init(u32 usb_num)
```

Initializes the hardware clock and power supply requirements for the USB Full-Speed peripheral.

Parametri:

Nome	Tipo	Descrizione
`usb_num`	`u32`	The index of the USB peripheral to initialize (e.g., 0 for USB1, 1 for USB2). Value depends on the specific STM32 series hardware capabilities.

Ritorna:
void

Note:

- This function is hardware-specific and performs low-level register manipulation to enable the HSI48 oscillator, configure clock selection (CCIPR), and power up the USB transceiver. It is not thread-safe and must be called during the system initialization phase before any USB communication functions are invoked. It is strictly forbidden to call this from an ISR,

as it contains blocking 'while' loops waiting for hardware ready flags (e.g., HSI48RDY). A common mistake is calling this function multiple times or after the USB stack has already started, which can cause clock instability or peripheral lockups.

Uso:

- This function is the mandatory first step in the USB initialization sequence. It is typically called during the board support package (BSP) setup or early system boot. It must be followed by GPIO configuration (e.g., ca_usb_fs_dev_gpio_init) and the USB stack initialization (e.g., ca_usb_fs_dev_init) to ensure the peripheral is ready for enumeration.

Esempio:

```
/* Initialize the USB clock before starting the USB stack */
ca_usb_fs_dev_clock_init(0U);

/* Configure the physical pins for USB communication */
ca_usb_fs_dev_gpio_init(0U);

/* Initialize the USB peripheral logic */
ca_usb_fs_dev_init(0U);

/* Wait for the USB connection to be established */
if (ca_usb_cdc_wait_connected(1000U)) {
    /* USB is ready for data transmission */
    ca_usb_cdc_write((u8 *)"Hello World", 11U);
} else {
    /* Handle timeout or connection failure */
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** At what point in the system boot sequence should I call ca_usb_fs_dev_clock_init?
A: It must be called during the system initialization phase, typically within the board support package (BSP) setup or early boot. It is the mandatory first step in the USB initialization sequence, which must be followed by GPIO configuration (ca_usb_fs_dev_gpio_init) and USB stack initialization (ca_usb_fs_dev_init).

Code example conf:0.90 **Q:** Provide a minimal example of how to initialize the USB clock using caOS types.
A: void system_init(void) { u32 usb_index = 0; ca_usb_fs_dev_clock_init(usb_index); ca_usb_fs_dev_gpio_init(usb_index); ca_usb_fs_dev_init(usb_index); }

Parameters conf:1.00 **Q:** What does the u32 usb_num parameter represent?
A: It represents the index of the USB peripheral to initialize (e.g., 0 for USB1, 1 for USB2). The valid values depend on the specific STM32 series hardware capabilities.

Error/Edge case conf:1.00 **Q:** What are the common pitfalls when calling ca_usb_fs_dev_clock_init?
A: Common mistakes include calling the function multiple times or invoking it after the USB stack has already started, which can lead to clock instability or peripheral lockups. Additionally, it must not be called from an ISR due to blocking loops.

Behavior conf:1.00 **Q:** Is ca_usb_fs_dev_clock_init thread-safe or ISR-safe?
A: The function is not thread-safe and is strictly forbidden from being called within an ISR. It contains blocking 'while' loops that wait for hardware ready flags, such as HSI48RDY, making it unsuitable for interrupt context.

HW differences conf:1.00 **Q:** How does the implementation of ca_usb_fs_dev_clock_init differ across STM32 families?
A: Implementations vary significantly: F1xx/F4xx/F7xx often rely on existing PLLQ configurations and simple AHB/APB clock enabling, while H5xx, H7xx, and U5xx require explicit HSI48 oscillator activation, CRS (Clock Recovery System) synchronization to

USB SOF, and specific power supply management (e.g., PWR->USBSCR for H5xx, PWR->CR3 for H7xx, or PWR->SVMCR for U5xx).

---

ca_usb_fs_dev_connect

Header: caos/drivers/inc/ca_glue_hw.h:40
GUARD: EN_CA_GLUE_HW

```
extern void ca_usb_fs_dev_connect(u32 usb_num)
```

Enables the USB device pull-up resistor to signal presence to the host.

Parametri:

Nome	Tipo	Descrizione
usb_num	u32	The index of the USB peripheral to connect. Must be a valid peripheral ID supported by the hardware.

Ritorna:
void

Note:

- This function is non-blocking and performs a direct register write to the USB controller to release the 'Soft Disconnect' (SDIS) bit or enable the D+ pull-up. It is not thread-safe in the sense that it should only be called once the USB stack is fully initialized and ready to handle enumeration requests from the host. Calling this too early will cause the host to attempt enumeration before the device is ready, leading to failed connections or 'Device Descriptor Request Failed' errors. It is generally not intended to be called from an ISR, as it is part of the high-level device management sequence.

Uso:

- This function is the final step in the USB device initialization sequence. It is typically called by the USB management layer (e.g., ca_usb_dev_mng_start) only after the hardware has been clocked, GPIOs configured, and all required endpoints have been opened via ca_usb_fs_dev_ep_open. By keeping the pull-up disabled until this point, you ensure the host does not see the device until the firmware is prepared to respond to standard USB requests.

Esempio:

```
/* Initialize USB peripheral and endpoints before connecting */
ca_usb_fs_dev_init(0);

/* Open control endpoint 0 for enumeration */
ca_usb_fs_dev_ep_open(0, 0x00, USB_EP_TYPE_CONTROL, 64);

/* Now that we are ready to handle host requests, signal presence */
ca_usb_fs_dev_connect(0);

/* The host will now detect the device and start enumeration */
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the correct time to call ca_usb_fs_dev_connect in the USB initialization sequence?
A: It should be the final step in the initialization sequence, called only after the hardware is clocked, GPIOs are configured, and all required endpoints have been opened via

ca_usb_fs_dev_ep_open. This ensures the host does not attempt enumeration before the device is fully prepared.

Code example conf:0.90 **Q:** Provide a minimal code snippet showing how to connect the USB device using caOS types.
A: u32 usb_id = 0; /* Assuming peripheral index 0 */ ca_usb_fs_dev_ep_open(usb_id, 0x81, CA_USB_EP_TYPE_BULK, 64); ca_usb_fs_dev_connect(usb_id);

Parameters conf:1.00 **Q:** What does the u32 usb_num parameter represent in ca_usb_fs_dev_connect?
A: It represents the index of the USB peripheral to connect. The value must correspond to a valid peripheral ID supported by the specific hardware implementation.

Error/Edge case conf:1.00 **Q:** What happens if I call ca_usb_fs_dev_connect before initializing endpoints?
A: Calling this function too early causes the host to attempt enumeration before the device is ready to handle requests, which typically leads to failed connections or 'Device Descriptor Request Failed' errors.

Behavior conf:1.00 **Q:** Is ca_usb_fs_dev_connect thread-safe or ISR-safe?
A: The function is non-blocking and performs a direct register write. It is not thread-safe and is not intended to be called from an ISR, as it is part of the high-level device management sequence.

HW differences conf:1.00 **Q:** How does the implementation of ca_usb_fs_dev_connect differ across STM32 families?
A: Most families (f4xx, f7xx, h7xx, u5xx) implement this by clearing the SDIS (Soft Disconnect) bit in the DCTL register of the DWC2 controller. In contrast, the h5xx family uses the BCDR register to set the DPPU (D+ pull-up) bit.

`ca_usb_fs_dev_disconnect`

Header: `caos/drivers/inc/ca_glue_hw.h:41`
GUARD: `EN_CA_GLUE_HW`

```
1 extern void ca_usb_fs_dev_disconnect(u32 usb_num)
```

Disconnects the USB Full-Speed device from the host by pulling the D+ line low or setting the disconnect bit.

Parametri:

Nome	Tipo	Descrizione
`usb_num`	`u32`	The index of the USB peripheral to disconnect. Must be a valid peripheral index supported by the hardware (e.g., 0 or 1).

Ritorna:
void

Note:

- This function is non-blocking and performs a direct register write to the USB peripheral's control register. It is generally thread-safe as it performs an atomic register modification, but it should not be called from an ISR unless the specific hardware implementation guarantees atomic access. The primary purpose is to simulate a physical cable unplug event to the host computer, forcing the host to re-enumerate the device. A common beginner mistake is calling this function while the USB stack is actively transmitting data, which can lead to undefined behavior or data loss in the USB buffers.

Uso:

- This function is typically used during device initialization sequences or when implementing a software-controlled USB reset. It is often paired with ca_usb_fs_dev_connect to toggle the connection state, allowing the device to 're-plug' itself to the host without physical intervention. In a typical lifecycle, you might call this during a system reset or when the application detects a critical USB error that requires a full re-enumeration.

Esempio:

```c
/* Example: Force a USB re-enumeration by disconnecting and reconnecting */
void reset_usb_connection(u32 usb_id) {
    /* Disconnect the device from the host */
    ca_usb_fs_dev_disconnect(usb_id);

    /* Wait for a short period to ensure the host registers the disconnect */
    ca_os_delay_ms(500);

    /* Re-connect the device to trigger re-enumeration */
    ca_usb_fs_dev_connect(usb_id);
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_usb_fs_dev_disconnect function in my caOS application?
A: You should use this function during device initialization sequences, when implementing a software-controlled USB reset, or when the application detects a critical USB error that requires a full re-enumeration. It simulates a physical cable unplug event to force the host to re-enumerate the device.

Code example conf:0.90 **Q:** How do I implement a software-controlled USB reset using caOS functions?
A: You can toggle the connection state by calling the disconnect and connect functions. Example: void reset_usb(u32 usb_idx) { ca_usb_fs_dev_disconnect(usb_idx); /* Add delay here if needed */ ca_usb_fs_dev_connect(usb_idx); }

Parameters conf:1.00 **Q:** What does the u32 usb_num parameter represent and what are the valid values?
A: The usb_num parameter is the index of the USB peripheral to disconnect. It must be a valid peripheral index supported by the hardware, such as 0 or 1.

HW differences conf:1.00 **Q:** How does the implementation of ca_usb_fs_dev_disconnect differ across STM32 families?
A: Implementations vary by register and bit usage: F4, F7, H7, and U5 families use the DCTL register and the USB_OTG_DCTL_SDIS bit. The H5 family uses the BCDR register and the USB_BCDR_DPPU bit. The F1 family implementation is currently a stub that performs no action.

`ca_usb_fs_dev_ep_clear_stall`

Header: `caos/drivers/inc/ca_glue_hw.h:44`
GUARD: `EN_CA_GLUE_HW`

```c
extern bool ca_usb_fs_dev_ep_clear_stall(u32 usb_num, u8 ep_addr)
```

Clears the STALL condition on a specific USB endpoint to resume normal communication.

Parametri:

Nome	Tipo	Descrizione
usb_num	u32	The index of the USB peripheral (e.g., 0 for FS, 1 for HS). Must be a valid peripheral index for the target MCU.
ep_addr	u8	The endpoint address (e.g., 0x81 for IN endpoint 1, 0x02 for OUT endpoint 2). Must be a valid endpoint index.

Ritorna:
Returns true if the stall was successfully cleared, false if the endpoint address is invalid or the operation is not supported on the current hardware.

Note:

- This function is non-blocking and performs direct register manipulation on the USB OTG peripheral. It is generally not thread-safe if called concurrently from multiple tasks for the same endpoint, so ensure synchronization if necessary. It is typically safe to call from an ISR, but check specific hardware implementation constraints. A common beginner mistake is calling this function on an endpoint that has not been opened or initialized, which may lead to undefined behavior or system instability. Always ensure the endpoint is in a stalled state before attempting to clear it, as clearing a non-stalled endpoint may disrupt active data transfers.

Uso:

- This function is used during USB enumeration or error recovery when the host or device needs to reset the data toggle and clear the stall bit after a protocol error. It is typically paired with ca_usb_fs_dev_ep_stall to manage endpoint flow control. In a standard sequence, you would detect a stall condition, perform necessary cleanup or buffer resets, call this function to clear the hardware stall, and then resume data transmission or reception.

Esempio:

```
/* Example: Clearing a stall on IN endpoint 1 after a protocol error */
u8 ep_addr = 0x81; // Endpoint 1 IN
u32 usb_idx = 0;

// Check if the endpoint is actually stalled before clearing
if (ca_usb_fs_dev_ep_is_stalled(usb_idx, ep_addr)) {
    // Clear the stall condition to allow the host to resume communication
    if (ca_usb_fs_dev_ep_clear_stall(usb_idx, ep_addr)) {
        // Successfully cleared, proceed to re-enable data flow
    } else {
        // Handle error: failed to clear stall
    }
}
```

Q&A — fine-tuning review *7/7 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_usb_fs_dev_ep_clear_stall in my USB stack?
A: You should call this function during USB enumeration or error recovery when the host or device needs to reset the data toggle and clear the stall bit after a protocol error. It is typically used to resume communication after a stall condition has been detected and handled.

Code example conf:1.00 **Q:** Provide a minimal example of how to clear a stall on endpoint 1 (IN).
A: u32 usb_idx = 0; u8 ep_addr = 0x81; if (ca_usb_fs_dev_ep_clear_stall(usb_idx, ep_addr)) { /* Stall cleared successfully */ }

Parameters conf:1.00 **Q:** What do the usb_num and ep_addr parameters represent?

A: usb_num is the index of the USB peripheral (e.g., 0 for FS, 1 for HS). ep_addr is the endpoint address, such as 0x81 for IN endpoint 1 or 0x02 for OUT endpoint 2. Both must be valid for the target MCU.

Return conf:1.00 **Q:** What does the boolean return value indicate?

A: It returns true if the stall was successfully cleared. It returns false if the endpoint address is invalid or if the operation is not supported on the current hardware.

Error/Edge case conf:1.00 **Q:** What is a common mistake when using this function?

A: A common beginner mistake is calling this function on an endpoint that has not been opened or initialized, which may lead to undefined behavior or system instability. Additionally, clearing a non-stalled endpoint may disrupt active data transfers.

Behavior conf:1.00 **Q:** Is this function thread-safe or blocking?

A: The function is non-blocking and performs direct register manipulation. It is generally not thread-safe if called concurrently from multiple tasks for the same endpoint, so synchronization is required. It is typically safe to call from an ISR, but you should check specific hardware implementation constraints.

HW differences conf:1.00 **Q:** Are there differences in how this function is implemented across MCU families?

A: Yes. On F1xx and F7xx, the function is currently a stub that returns false. On F4xx, H7xx, and U5xx, it manipulates DWC2 registers like DIEPCTL and DOEPCTL. On H5xx, it interacts with R_CHEP registers and manages DTOG bits directly.

`ca_usb_fs_dev_ep_close`

Header: `caos/drivers/inc/ca_glue_hw.h:43`
GUARD: `EN_CA_GLUE_HW`

```
1 extern bool ca_usb_fs_dev_ep_close(u32 usb_num, u8 ep_addr)
```

Safely disables and resets a specific USB device endpoint hardware configuration.

Parametri:

Nome	Tipo	Descrizione
`usb_num`	`u32`	The index of the USB peripheral (e.g., 0 for FS, 1 for HS). Must be a valid peripheral index.
`ep_addr`	`u8`	The endpoint address (e.g., 0x81 for IN endpoint 1, 0x02 for OUT endpoint 2). EP0 is not allowed.

Ritorna:
Returns true if the endpoint was successfully closed, false if the endpoint was already closed, invalid (EP0), or if the peripheral index is out of range.

Note:

- This function is blocking because it waits for hardware handshake signals (NAK/EPDISD) to ensure the endpoint is safely disabled before clearing registers. It is not thread-safe and must not be called from an ISR, as it performs busy-waiting loops that could hang the system if the hardware is unresponsive. A common mistake is attempting to close EP0, which is reserved for control transfers and managed by the stack; the function will return false immediately. Ensure the endpoint is no longer being accessed by the USB stack before calling this, as it clears internal state buffers and hardware masks.

Uso:

- This function is typically used during USB device re-enumeration or when a specific interface is being disabled to free up resources. It is the final step in the teardown sequence: first, stop data transmission/reception, then call this function to reset the hardware state, and finally update the application-level endpoint status. It is often paired with ca_usb_fs_dev_ep_open to re-initialize the endpoint with new parameters after a configuration change.

Esempio:

```c
/* Example: Safely closing an IN endpoint before re-configuration */
u8 ep_addr = 0x81; // IN endpoint 1
u32 usb_idx = 0;

if (ca_usb_fs_dev_ep_close(usb_idx, ep_addr)) {
    /* Endpoint hardware is now disabled and buffers are flushed */
    ca_printf("Endpoint 0x%02X closed successfully.\n", ep_addr);
} else {
    /* Endpoint was already closed or invalid */
    ca_printf("Failed to close endpoint 0x%02X.\n", ep_addr);
}
```

Q&A — fine-tuning review *7/7 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_usb_fs_dev_ep_close in my USB application?
A: This function is used during USB device re-enumeration or when disabling a specific interface to free resources. It should be the final step in a teardown sequence: first stop data transmission/reception, then call this function to reset hardware, and finally update application-level status.

Code example conf:1.00 **Q:** Provide a minimal example of how to close an IN endpoint using caOS types.
A: u32 usb_idx = 0; u8 ep_addr = 0x81; if (ca_usb_fs_dev_ep_close(usb_idx, ep_addr)) { /* Endpoint closed successfully */ }

Parameters conf:1.00 **Q:** What are the constraints on the ep_addr parameter?
A: The ep_addr must be a valid endpoint address (e.g., 0x81 for IN, 0x02 for OUT). EP0 is strictly reserved for control transfers and is not allowed; attempting to close it will cause the function to return false.

Return conf:1.00 **Q:** What does a return value of false indicate?
A: A false return indicates that the endpoint was already closed, the provided endpoint address was EP0, the peripheral index was out of range, or the operation failed to complete.

Error/Edge case conf:1.00 **Q:** What are the common mistakes when using ca_usb_fs_dev_ep_close?
A: A common mistake is attempting to close EP0, which is managed by the stack. Additionally, failing to stop data transmission/reception before calling this function can lead to issues, as the function clears internal state buffers and hardware masks.

Behavior conf:1.00 **Q:** Is ca_usb_fs_dev_ep_close safe to call from an interrupt service routine (ISR)?
A: No, it is not ISR-safe. The function is blocking and uses busy-waiting loops to wait for hardware handshake signals (NAK/EPDISD). Calling it from an ISR could hang the system if the hardware is unresponsive.

HW differences conf:1.00 **Q:** How does the implementation of ca_usb_fs_dev_ep_close differ across STM32 families?
A: Implementations vary significantly: F1xx and F7xx provide stubs that return false, while F4xx, H7xx, and U5xx perform complex register-level operations (e.g., DWC2_GLB, DWC2_DEV, DIEPCTL, DOEPCTL) to disable endpoints and flush FIFOs. H5xx uses a simplified abstraction with ep_set_stat_tx/rx functions.

ca_usb_fs_dev_ep_is_stalled

Header: caos/drivers/inc/ca_glue_hw.h:45
GUARD: EN_CA_GLUE_HW

```
extern bool ca_usb_fs_dev_ep_is_stalled(u32 usb_num, u8 ep_addr)
```

Checks if a specific USB endpoint is currently in a stalled state.

Parametri:

Nome	Tipo	Descrizione
usb_num	u32	The index of the USB peripheral (e.g., 0 for USB_FS). Must be within the hardware-supported range.
ep_addr	u8	The endpoint address (e.g., 0x81 for IN endpoint 1, 0x02 for OUT endpoint 2). Must be a valid endpoint index.

Ritorna:
Returns true if the endpoint is stalled, false otherwise. Returns false if the endpoint address is invalid or the peripheral is not initialized.

Note:

- This function is non-blocking and performs a direct register read from the USB hardware controller. It is thread-safe in the sense that it performs a read-only operation, but it does not provide synchronization against concurrent modifications of the endpoint state. It is safe to call from an ISR, as it does not involve OS primitives or blocking calls. A common mistake is passing an invalid endpoint address or failing to check if the USB peripheral is actually initialized before querying its status.

Uso:

- This function is typically used during USB error handling or protocol recovery sequences. It is often paired with ca_usb_fs_dev_ep_clear_stall when a host requests a clear feature on a stalled endpoint. Developers use it to verify the current state of the hardware before attempting to send or receive data, ensuring that the endpoint is not in a 'halted' state that would cause data transfers to fail.

Esempio:

```
/* Check if endpoint 1 (IN) is stalled before attempting to send data */
#define EP_IN_1 0x81

void check_and_recover_ep(u32 usb_idx) {
    if (ca_usb_fs_dev_ep_is_stalled(usb_idx, EP_IN_1)) {
        /* Endpoint is stalled, clear the stall condition to resume communication */
        /* Note: ca_usb_fs_dev_ep_clear_stall is the function to reset the stall bit */
        ca_usb_fs_dev_ep_clear_stall(usb_idx, EP_IN_1);
    } else {
        /* Endpoint is healthy, proceed with data transmission */
        // ca_usb_fs_dev_ep_write(usb_idx, EP_IN_1, buffer, size);
    }
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_usb_fs_dev_ep_is_stalled in my USB stack?

A: It is typically used during USB error handling or protocol recovery sequences. It is often paired with ca_usb_fs_dev_ep_clear_stall when a host requests a clear feature on a stalled endpoint to verify the hardware state before attempting further data transfers.

Code example conf:1.00 **Q:** Provide a minimal example of checking if endpoint 1 IN is stalled.

A: u32 usb_idx = 0; u8 ep_addr = 0x81; if (ca_usb_fs_dev_ep_is_stalled(usb_idx, ep_addr)) { /* Handle stalled endpoint */ }

Parameters conf:1.00 **Q:** What are the requirements for the ep_addr parameter?

A: The ep_addr must be a valid endpoint index (e.g., 0x81 for IN endpoint 1, 0x02 for OUT endpoint 2). Passing an invalid endpoint address will cause the function to return false.

Return conf:1.00 **Q:** What does a return value of false indicate?

A: A return value of false indicates that the endpoint is not stalled, the endpoint address provided is invalid, or the USB peripheral is not initialized.

HW differences conf:1.00 **Q:** How does the implementation of ca_usb_fs_dev_ep_is_stalled differ across MCU families?

A: Implementations vary significantly: F4xx, H7xx, and U5xx use DWC2 registers (DIEPCTL/DOEPCTL), while H5xx uses R_CHEP registers (USB_CHEP_TX_STTX/USB_CHEP_ Notably, F1xx and F7xx implementations are currently stubs that return false.

`ca_usb_fs_dev_ep_open`

Header: `caos/drivers/inc/ca_glue_hw.h:34`
GUARD: `EN_CA_GLUE_HW`

```
1 extern bool ca_usb_fs_dev_ep_open(u32 usb_num, u8 ep_addr,
    ca_usb_ep_type_t type, u16 mps)
```

Initializes and enables a specific USB endpoint for data transmission or reception.

Parametri:

Nome	Tipo	Descrizione
`usb_num`	`u32`	The index of the USB peripheral (e.g., 0 for FS, 1 for HS). Must be within valid hardware range.
`ep_addr`	`u8`	The endpoint address (e.g., 0x81 for IN endpoint 1, 0x02 for OUT endpoint 2). Bit 7 indicates direction.
`type`	`ca_usb_ep_type_t`	The transfer type (Control, Bulk, Interrupt, or Isochronous). Must match the descriptor.
`mps`	`u16`	Maximum Packet Size in bytes. Must not exceed the hardware buffer capacity.

Ritorna:
Returns true if the endpoint was successfully configured and enabled, false if the endpoint index is invalid or the hardware failed to initialize.

Note:

- This function is not thread-safe and must be called during the USB enumeration process or when reconfiguring an interface. It is typically not callable from an ISR as it performs direct hardware register manipulation. A common beginner mistake is providing an incorrect MPS value that exceeds the physical buffer size, which can lead to memory corruption or silent data loss. Ensure the USB peripheral clock and GPIOs are initialized via ca_usb_fs_dev_clock_init and ca_usb_fs_dev_gpio_init before calling this.

Uso:

- This function is used during the USB device setup phase, specifically after receiving a SET_CONFIGURATION or SET_INTERFACE request from the host. It is part of a sequence: first, initialize the USB core with ca_usb_fs_dev_init, then open the required endpoints using this function, and finally enable the device connection via ca_usb_fs_dev_connect. It is paired with ca_usb_fs_dev_ep_close when the interface is disabled or the device is disconnected.

Esempio:

```c
/* Example: Opening Bulk IN and OUT endpoints for a custom USB interface */
#define EP_BULK_IN  0x81
#define EP_BULK_OUT 0x01

void setup_usb_endpoints(u32 usb_idx) {
    /* Configure Bulk IN endpoint with 64 bytes MPS */
    if (!ca_usb_fs_dev_ep_open(usb_idx, EP_BULK_IN, CA_USB_EP_TYPE_BULK, 64)) {
        /* Handle error: log or trigger system reset */
    }

    /* Configure Bulk OUT endpoint with 64 bytes MPS */
    if (!ca_usb_fs_dev_ep_open(usb_idx, EP_BULK_OUT, CA_USB_EP_TYPE_BULK, 64)) {
        /* Handle error: log or trigger system reset */
    }

    /* Now the endpoints are ready to receive/send data via
    ca_usb_fs_dev_ep_read/write */
}
```

Q&A — fine-tuning review *7/7 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** At what point in the USB device lifecycle should I call ca_usb_fs_dev_ep_open?
A: This function should be called during the USB device setup phase, specifically after the host sends a SET_CONFIGURATION or SET_INTERFACE request. It must be executed after initializing the USB core with ca_usb_fs_dev_init and before enabling the connection via ca_usb_fs_dev_connect.

Code example conf:0.90 **Q:** Provide a minimal example of how to open an IN endpoint using caOS types.
A: u32 usb_idx = 0; u8 ep_addr = 0x81; ca_usb_ep_type_t type = CA_USB_EP_BULK; u16 mps = 64; bool success = ca_usb_fs_dev_ep_open(usb_idx, ep_addr, type, mps);

Parameters conf:1.00 **Q:** How do I correctly specify the ep_addr parameter?
A: The ep_addr is a u8 where the lower bits represent the endpoint number and bit 7 indicates the direction. For example, 0x81 represents IN endpoint 1, while 0x02 represents OUT endpoint 2.

Return conf:1.00 **Q:** What does a false return value from ca_usb_fs_dev_ep_open indicate?
A: A return value of false indicates that the endpoint index is invalid (e.g., exceeds the hardware limit) or that the hardware failed to initialize the endpoint configuration.

Error/Edge case conf:1.00 **Q:** What is a common mistake when configuring the Maximum Packet Size (MPS)?
A: A common beginner mistake is providing an MPS value that exceeds the physical buffer capacity of the hardware. This can lead to memory corruption or silent data loss.

Behavior conf:1.00 **Q:** Is ca_usb_fs_dev_ep_open safe to call from an Interrupt Service Routine (ISR)?
A: No, this function is not thread-safe and is typically not callable from an ISR because it performs direct hardware register manipulation. It must be called during the USB enumeration process or interface reconfiguration.

HW differences conf:1.00 **Q:** How does the implementation of ca_usb_fs_dev_ep_open differ between the H5xx family and the F4xx/F7xx families?

A: The H5xx implementation uses a packet memory area (PMA) approach with registers like R_CHEP and R_BTABLE, whereas the F4xx, F7xx, and H7xx families utilize the DWC2 OTG controller, manipulating registers like DIEPCTL and DOEPCTL.

---

ca_usb_fs_dev_ep_read

Header: caos/drivers/inc/ca_glue_hw.h:36
GUARD: EN_CA_GLUE_HW

```
extern u16 ca_usb_fs_dev_ep_read(u32 usb_num, u8 ep_addr, u8 *buf, u16
    max_len)
```

Reads data from a specific USB device endpoint into a provided buffer.

Parametri:

Nome	Tipo	Descrizione
usb_num	u32	The index of the USB peripheral (usually 0). Must be within valid hardware range.
ep_addr	u8	The endpoint address (e.g., 0x01 for EP1 OUT). Must be an OUT endpoint; IN endpoints will return 0.
buf	u8 *	Pointer to the destination buffer. Must not be NULL.
max_len	u16	Maximum number of bytes to copy. If the received packet is larger, only this amount is copied.

Ritorna:
Returns the number of bytes actually copied into the buffer. Returns 0 if the endpoint is empty, invalid, or if an error occurs (e.g., NULL buffer or IN endpoint).

Note:

- This function is non-blocking and retrieves data already present in the internal USB receive queues. It is generally thread-safe regarding the queue management, but it interacts with hardware registers that may be modified by the USB ISR; therefore, it uses critical sections internally to prevent race conditions during endpoint re-arming. A common beginner mistake is calling this function in a tight loop without checking if data is actually available, which wastes CPU cycles; instead, wait for a signal or event indicating that data has arrived. Ensure the endpoint has been successfully opened with ca_usb_fs_dev_ep_open before attempting to read.

Uso:

- This function is typically used in a dedicated USB task or a callback triggered by a data-ready event. The standard sequence is: 1) Initialize the USB peripheral, 2) Open the desired endpoint using ca_usb_fs_dev_ep_open, 3) Wait for a notification (e.g., via ca_event_wait) that data is ready, 4) Call ca_usb_fs_dev_ep_read to copy the data into a local buffer, and 5) Process the data. It is often paired with ca_usb_fs_dev_ep_write to implement bidirectional communication.

Esempio:

```
/* Assuming USB is initialized and EP1 OUT is open */
u8 rx_buffer[64];
u16 bytes_received;

```

```c
/* Wait for an event indicating data arrival */
ca_event_wait(usb_data_event, 1000);

/* Attempt to read up to 64 bytes from EP1 OUT */
bytes_received = ca_usb_fs_dev_ep_read(0, 0x01, rx_buffer, 64);

if (bytes_received > 0) {
    /* Process the received data */
    ca_printf("Received %d bytes\n", bytes_received);
} else {
    /* Handle empty queue or error */
    ca_printf("No data available or read error\n");
}
```

Q&A — fine-tuning review *7/7 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to call ca_usb_fs_dev_ep_read in a caOS application?
A: It should be called after initializing the USB peripheral and opening the endpoint with ca_usb_fs_dev_ep_open. You should wait for a notification, such as via ca_event_wait, indicating that data is ready, rather than polling in a tight loop.

Code example conf:1.00 **Q:** Provide a minimal example of reading data from USB endpoint 1 using caOS types.
A: u8 buffer[64]; u16 received = ca_usb_fs_dev_ep_read(0, 0x01, buffer, 64); if (received > 0) { /* process data */ }

Parameters conf:1.00 **Q:** What are the constraints on the ep_addr parameter for ca_usb_fs_dev_ep_read?
A: The ep_addr must represent an OUT endpoint. If an IN endpoint address is provided, the function will return 0. The endpoint must also be within the valid hardware range defined for the specific MCU.

Return conf:1.00 **Q:** What does a return value of 0 from ca_usb_fs_dev_ep_read signify?
A: A return value of 0 indicates that the endpoint is empty, the endpoint address is invalid (e.g., an IN endpoint), the buffer pointer is NULL, or an error occurred during the read operation.

Error/Edge case conf:1.00 **Q:** What is a common mistake when using ca_usb_fs_dev_ep_read?
A: A common beginner mistake is calling the function in a tight loop without checking if data is actually available, which wastes CPU cycles. It is better to wait for a signal or event indicating data arrival.

Behavior conf:1.00 **Q:** Is ca_usb_fs_dev_ep_read thread-safe and blocking?
A: The function is non-blocking and retrieves data already present in internal queues. It is generally thread-safe regarding queue management and uses internal critical sections to prevent race conditions with the USB ISR during endpoint re-arming.

HW differences conf:1.00 **Q:** How does the implementation of ca_usb_fs_dev_ep_read differ between the F7xx and H5xx families?
A: The F7xx implementation uses DMA (dma_rx) and directly manipulates DOEPDMA and DOEPTSIZ registers to re-arm the endpoint. The H5xx implementation manages RX rings and uses R_CHEP registers to check status and set RX_VALID/NAK states, specifically avoiding RXBD rewrites to prevent ghost length-0 packets.

`ca_usb_fs_dev_ep_stall`

Header: `caos/drivers/inc/ca_glue_hw.h:42`
GUARD: `EN_CA_GLUE_HW`

```c
extern void ca_usb_fs_dev_ep_stall(u32 usb_num, u8 ep_addr)
```

Forces a USB endpoint into the STALL state to signal a protocol error or lack of resources to the host.

Parametri:

Nome	Tipo	Descrizione
`usb_num`	`u32`	The index of the USB peripheral (e.g., 0 or 1). On single-controller chips, this is often ignored.
`ep_addr`	`u8`	The endpoint address. Bit 7 is the direction (1 for IN, 0 for OUT). Bits 0-3 define the endpoint number.

Ritorna:
void

Note:

- This function is non-blocking and executes immediately by modifying hardware registers. It is generally safe to call from an ISR, as it performs simple register writes without waiting for external events. The most common mistake is passing an invalid endpoint address or forgetting that stalling an endpoint stops all communication on that pipe until the host sends a CLEAR_FEATURE request. Ensure the USB peripheral is properly initialized before calling this, otherwise, the register access may cause a hard fault.

Uso:

- Use this function when your USB stack detects an invalid request or a buffer overflow condition that requires the host to stop sending data. It is typically paired with ca_usb_fs_dev_ep_clear_stall, which is used once the error condition is resolved and the host requests to resume communication. In a standard USB device implementation, you would call this inside your control transfer handler when a setup packet contains unsupported parameters.

Esempio:

```
/* Example: Stalling an endpoint due to an invalid data request */
void handle_usb_request(u8 ep_addr, u32 length) {
    if (length > MAX_BUFFER_SIZE) {
        /* Signal the host that this endpoint cannot process the request */
        ca_usb_fs_dev_ep_stall(0, ep_addr);
    } else {
        /* Proceed with normal data processing */
        // ca_usb_fs_dev_ep_read(0, ep_addr, buffer, length);
    }
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_usb_fs_dev_ep_stall function in my USB device implementation?
A: You should call this function when your USB stack detects an invalid request or a buffer overflow condition that requires the host to stop sending data. It is typically used in the control transfer handler when a setup packet contains unsupported parameters.

Code example conf:1.00 **Q:** Provide a minimal C example showing how to stall endpoint 1 IN using caOS types.
A: u32 usb_idx = 0; u8 ep_addr = 0x81; ca_usb_fs_dev_ep_stall(usb_idx, ep_addr);

Parameters conf:1.00 **Q:** How do I correctly format the ep_addr parameter for ca_usb_fs_dev_ep_stall?
A: The ep_addr parameter is a u8 where bits 0-3 define the endpoint number and bit 7 indicates the direction (1 for IN, 0 for OUT).

Error/Edge case conf:1.00 **Q:** What are the common pitfalls when calling ca_usb_fs_dev_ep_stall?

A: Common mistakes include passing an invalid endpoint address, failing to initialize the USB peripheral before calling (which may cause a hard fault), or forgetting that stalling stops all communication on that pipe until the host sends a CLEAR_FEATURE request.

HW differences conf:1.00 **Q:** How does the implementation of ca_usb_fs_dev_ep_stall differ across STM32 families?

A: Implementations vary significantly: F1xx provides a dummy implementation, F4xx/F7xx/U5xx directly manipulate DWC2_EPIN/EPOUT registers and call ep0_out_arm for endpoint 0, H7xx includes bounds checking for usb_num, and H5xx uses abstraction functions ep_set_stat_tx/rx with internal bounds checking against USB_EP_MAX.

---

ca_usb_fs_dev_ep_write

Header: `caos/drivers/inc/ca_glue_hw.h:35`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_usb_fs_dev_ep_write(u32 usb_num, u8 ep_addr, const u8 *
    buf, u16 len)
```

Writes data to a specific USB device endpoint by queuing it for transmission.

Parametri:

Nome	Tipo	Descrizione
`usb_num`	`u32`	The index of the USB peripheral (usually 0).
`ep_addr`	`u8`	The endpoint address (e.g., 0x81 for IN endpoint 1). Must be an IN endpoint.
`buf`	`const u8 *`	Pointer to the data buffer to be sent. Can be NULL if len is 0.
`len`	`u16`	Number of bytes to write. Must not exceed the endpoint's buffer capacity.

Ritorna:
Returns true if the data was successfully queued for transmission, false otherwise (e.g., invalid endpoint, buffer full, or hardware busy).

Note:

- This function is thread-safe and uses internal critical sections to protect the transmission ring buffer from concurrent access by multiple tasks or ISRs. It is non-blocking; if the internal queue is full, it returns false immediately, allowing the caller to decide whether to retry or yield. A common mistake is assuming the data is sent immediately; the function only copies data into the driver's internal queue. Ensure the endpoint has been properly opened with ca_usb_fs_dev_ep_open before calling this function. Misuse, such as passing an OUT endpoint address, will result in an immediate false return.

Uso:

- Used in USB device applications to send data to the host. It is typically called after the USB connection is established and the endpoint is configured. In a standard workflow, the application initializes the USB stack, opens the required endpoints, and then calls this function whenever data is ready to be transmitted. If the function returns false due to a full queue, the application should use ca_os_yield or a small delay before attempting to resend the data.

Esempio:

```
/* Example: Sending a status message over a USB IN endpoint */
#define EP_IN_ADDR 0x81
```

```

void send_usb_status(const u8 *data, u16 length) {
    /* Attempt to write data to the USB endpoint */
    if (!ca_usb_fs_dev_ep_write(0, EP_IN_ADDR, data, length)) {
        /* If queue is full, wait briefly and try again */
        ca_os_delay_ms(10);
        /* Note: In a real application, implement a retry counter or buffer management */
    }
}
```

Q&A — fine-tuning review *7/7 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_usb_fs_dev_ep_write in my application?
A: You should call this function after the USB connection is established and the target endpoint has been successfully initialized using ca_usb_fs_dev_ep_open. It is used to queue data for transmission to the host whenever your application has data ready to send.

Code example conf:1.00 **Q:** Provide a minimal example of how to send data using ca_usb_fs_dev_ep_write.
A: u8 data[] = {0x01, 0x02, 0x03}; u32 usb_idx = 0; u8 ep = 0x81; if (!ca_usb_fs_dev_ep_write(usb_idx, ep, data, 3)) { ca_os_yield(); }

Parameters conf:1.00 **Q:** What are the constraints on the ep_addr and len parameters?
A: The ep_addr must represent an IN endpoint (e.g., 0x81). Passing an OUT endpoint address will result in an immediate false return. The len parameter must not exceed the endpoint's configured buffer capacity.

Return conf:1.00 **Q:** What does a false return value from ca_usb_fs_dev_ep_write indicate?
A: A false return indicates the data was not queued. This can happen if the endpoint address is invalid, the internal transmission queue is full, the hardware is busy, or the endpoint was not properly opened.

Error/Edge case conf:1.00 **Q:** What is a common mistake when using ca_usb_fs_dev_ep_write?
A: A common mistake is assuming the data is sent immediately. The function only copies data into the driver's internal queue. If the queue is full, the function returns false, and the application should handle this by yielding or retrying later.

Behavior conf:1.00 **Q:** Is ca_usb_fs_dev_ep_write safe to call from multiple tasks or ISRs?
A: Yes, the function is thread-safe. It uses internal critical sections to protect the transmission ring buffer from concurrent access by multiple tasks or ISRs, preventing data corruption.

HW differences conf:1.00 **Q:** How does the implementation of ca_usb_fs_dev_ep_write differ between F4xx and H5xx families?
A: On F4xx, the driver uses a ring buffer and checks for free slots in the queue. On H5xx, the driver manages PMA (Packet Memory Area) and must explicitly handle ZLP (Zero Length Packet) logic for multi-packet transfers that are exact multiples of the MPS, as PMA does not packetize in hardware.

`ca_usb_fs_dev_get_speed`

Header: `caos/drivers/inc/ca_glue_hw.h:38`
GUARD: `EN_CA_GLUE_HW`

```
extern ca_usb_speed_t ca_usb_fs_dev_get_speed(u32 usb_num)
```

Retrieves the current operational speed of the specified USB Full-Speed device.

Parametri:

Nome	Tipo	Descrizione
`usb_num`	`u32`	The index of the USB peripheral to query. Valid values depend on the hardware configuration (usually 0 or 1 for multi-USB systems).

Ritorna:
Returns a ca_usb_speed_t enum value (e.g., CA_USB_SPEED_FS). If the hardware does not support speed detection or the index is invalid, it typically defaults to CA_USB_SPEED_FS.

Note:

- This function is non-blocking and thread-safe as it typically reads from a cached state variable maintained by the USB driver. It is safe to call from an ISR because it performs a simple memory read without acquiring mutexes or waiting for hardware events. The most common mistake is assuming the speed can change dynamically without the USB stack re-enumerating; always ensure the USB peripheral has been initialized via the appropriate driver init function before calling this.

Uso:

- This function is used during the USB enumeration process or when debugging connection issues to verify if the device has negotiated the expected speed. It is typically called after the USB stack has been initialized and the device is connected to a host. Developers often pair this with ca_usb_cdc_wait_connected to ensure the link is active before checking the speed, or use it in diagnostic logs to confirm hardware capabilities.

Esempio:

```c
/* Check the speed of the first USB peripheral */
ca_usb_speed_t speed = ca_usb_fs_dev_get_speed(0);

/* Log the detected speed for debugging purposes */
if (speed == CA_USB_SPEED_FS) {
    /* USB is operating at Full Speed (12 Mbps) */
    ca_printf("USB device 0 is running at Full Speed\n");
} else {
    /* Handle unexpected speed or error states */
    ca_printf("USB device 0 speed status: %d\n", (int)speed);
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to call ca_usb_fs_dev_get_speed in my application?
A: You should call this function after the USB stack has been initialized and the device is connected to a host. It is typically used during the USB enumeration process or for diagnostic logging to verify the negotiated speed. It is recommended to pair this with ca_usb_cdc_wait_connected to ensure the link is active before checking.

Code example conf:1.00 **Q:** Provide a minimal example of how to check the USB speed in caOS.
A: u32 usb_idx = 0; ca_usb_speed_t speed = ca_usb_fs_dev_get_speed(usb_idx); if (speed == CA_USB_SPEED_FS) { /* Handle Full-Speed logic */ }

Parameters conf:0.90 **Q:** What does the usb_num parameter represent and what are its valid values?
A: The usb_num parameter is the index of the USB peripheral to query. Valid values depend on the hardware configuration, typically 0 or 1 for multi-USB systems. Note that some implementations ignore this parameter if the hardware only supports a single USB instance.

Return conf:0.95 **Q:** What does the return value of ca_usb_fs_dev_get_speed indicate, and what happens if it fails?

A: It returns a ca_usb_speed_t enum value, such as CA_USB_SPEED_FS. If the hardware does not support speed detection or if an invalid index is provided, the function typically defaults to returning CA_USB_SPEED_FS.

HW differences conf:1.00 **Q:** How does the implementation of ca_usb_fs_dev_get_speed vary across different STM32 families?

A: Implementations vary: F1xx and H5xx return a hardcoded CA_USB_SPEED_FS. F4xx, F7xx, and U5xx return a cached speed value from a global state (g_st.speed) and ignore the usb_num parameter. H7xx is unique in that it checks the index against a global array (g_usb[usb_num].speed) to support multiple peripherals.

---

ca_usb_fs_dev_gpio_init

Header: `caos/drivers/inc/ca_glue_hw.h:32`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_usb_fs_dev_gpio_init(u32 usb_num)
```

Configures the physical GPIO pins required for USB Full-Speed peripheral operation.

Parametri:

Nome	Tipo	Descrizione
usb_num	u32	The index of the USB peripheral to initialize (e.g., 0 for USB_OTG_FS1). Values depend on the specific MCU architecture.

Ritorna:
void

Note:

- This function is non-blocking and performs direct register manipulation to set up GPIO pins in Alternate Function (AF) mode. It is not thread-safe and must be called during the system initialization phase before the USB peripheral is enabled. It is generally not intended to be called from an ISR. A common mistake is calling this function after the USB peripheral has already started, which can cause electrical contention or signal integrity issues on the data lines.

Uso:

- This function is a mandatory step in the USB initialization sequence. It is typically called immediately after clock configuration and before the USB peripheral core is initialized via ca_usb_fs_dev_init. It ensures that the physical pins (D+, D-, and others like VBUS or ID) are correctly routed to the USB hardware block rather than acting as standard GPIOs.

Esempio:

```
/* Initialize USB Full-Speed GPIOs for the first USB peripheral */
ca_usb_fs_dev_gpio_init(0U);

/* Proceed to initialize the USB peripheral core */
/* ca_usb_fs_dev_init(0U); */

/* Connect the device to the USB bus */
ca_usb_connect();
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_usb_fs_dev_gpio_init in my initialization sequence?
A: It must be called during the system initialization phase, specifically after clock configuration and before initializing the USB peripheral core via ca_usb_fs_dev_init.

Code example conf:1.00 **Q:** Provide a minimal C example for initializing USB GPIOs using caOS types.
A: u32 usb_index = 0U; ca_usb_fs_dev_gpio_init(usb_index);

Parameters conf:1.00 **Q:** What does the usb_num parameter represent in ca_usb_fs_dev_gpio_init?
A: It is the index of the USB peripheral to initialize (e.g., 0 for USB_OTG_FS1). The valid values depend on the specific MCU architecture being used.

Error/Edge case conf:1.00 **Q:** What happens if I call ca_usb_fs_dev_gpio_init after the USB peripheral has already started?
A: Calling it after the peripheral has started can cause electrical contention or signal integrity issues on the data lines.

Behavior conf:1.00 **Q:** Is ca_usb_fs_dev_gpio_init thread-safe or ISR-safe?
A: The function is non-blocking, but it is not thread-safe and is generally not intended to be called from an ISR.

HW differences conf:1.00 **Q:** How does the implementation of ca_usb_fs_dev_gpio_init differ between STM32 H5xx and F4xx families?
A: On H5xx, the USB pins are dedicated and require no GPIO AF configuration, leaving them in reset state. On F4xx, the function performs explicit GPIO configuration and includes a specific workaround for boards with external D+ pull-ups by driving PA12 low to force a re-enumeration.

`ca_usb_fs_dev_init`

Header: `caos/drivers/inc/ca_glue_hw.h:33`
GUARD: `EN_CA_GLUE_HW`

```
1 extern bool ca_usb_fs_dev_init(u32 usb_num, const ca_usb_cb_t *cb)
```

Initializes the USB Full-Speed hardware peripheral and registers the application callback structure.

Parametri:

Nome	Tipo	Descrizione
`usb_num`	`u32`	The index of the USB peripheral to initialize (e.g., 0 for OTG_FS). Must be a valid hardware index for the specific MCU.
`cb`	`const ca_usb_cb_t *`	Pointer to a structure containing function pointers for USB events (e.g., reset, setup, data). Must not be NULL.

Ritorna:
Returns true if the hardware was successfully reset and configured, false if the initialization failed (e.g., invalid parameters or hardware timeout).

Note:

- This function performs low-level hardware initialization, including clock configuration, GPIO setup, and core reset. It is NOT thread-safe and must be called during the system startup phase before any USB communication begins. It is not intended to be called from an ISR. A common mistake is calling this function while the USB cable is already connected or before the required clocks are enabled, which can lead to hardware lockups or failed enumeration. Note that this function leaves the USB device in a disconnected state

(D+ pull-up disabled); you must call ca_usb_fs_dev_connect later to signal presence to the host.

Uso:

- This is the first step in the USB device lifecycle. It is typically called during system initialization after memory and clocks are configured. The sequence is: 1) ca_usb_fs_dev_init to prepare the hardware, 2) ca_usb_dev_mng_add to register endpoints, and 3) ca_usb_fs_dev_connect to physically signal the host that the device is ready for enumeration. It is paired with the callback structure that handles the USB protocol logic.

Esempio:

```c
/* Define the callback structure for USB events */
static const ca_usb_cb_t my_usb_callbacks = {
    /* .reset = my_reset_handler, .setup = my_setup_handler, ... */
};

void app_usb_init(void) {
    /* Initialize the USB FS peripheral (index 0) */
    if (ca_usb_fs_dev_init(0U, &my_usb_callbacks)) {
        /* Hardware ready, proceed to add endpoints and connect */
        /* ca_usb_dev_mng_add(...) */
        ca_usb_fs_dev_connect(0U);
    } else {
        ca_stdout_put_n("USB Init failed!\n");
    }
}
```

Q&A — fine-tuning review *7/7 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** What is the correct sequence to initialize the USB device in caOS?
A: The USB device lifecycle starts with ca_usb_fs_dev_init to configure hardware, followed by ca_usb_dev_mng_add to register endpoints, and finally ca_usb_fs_dev_connect to signal the host that the device is ready for enumeration.

Code example conf:0.90 **Q:** Provide a minimal C example for initializing the USB peripheral.
A: ca_usb_cb_t my_cbs = { .reset = my_reset_fn, .setup = my_setup_fn }; if (ca_usb_fs_dev_init(0, &my_cbs)) { /* proceed to register endpoints */ }

Parameters conf:1.00 **Q:** What are the requirements for the 'cb' parameter in ca_usb_fs_dev_init?
A: The 'cb' parameter must be a pointer to a ca_usb_cb_t structure containing function pointers for USB events. It must not be NULL, as the function will return false if it is.

Return conf:1.00 **Q:** What does a return value of false indicate from ca_usb_fs_dev_init?
A: A false return indicates that initialization failed. This can be caused by passing invalid parameters (like a NULL callback), or a hardware timeout during the core reset process (e.g., AHB idle or CSRST timeout).

Error/Edge case conf:1.00 **Q:** What is a common mistake that leads to hardware lockups during USB initialization?
A: A common mistake is calling ca_usb_fs_dev_init while the USB cable is already connected or before the required clocks are enabled, which can lead to hardware lockups or failed enumeration.

Behavior conf:1.00 **Q:** Is ca_usb_fs_dev_init thread-safe or ISR-safe?
A: The function is not thread-safe and must be called during the system startup phase. It is not intended to be called from an ISR.

HW differences conf:1.00 **Q:** How does the USB initialization differ between H5xx and F4xx/F7xx families?
A: The H5xx family uses the USB_DRD_FS peripheral and manages power-down via the CNTR register (PDWN/USBRST bits), whereas F4xx, F7xx, and U5xx families use

the DWC2 OTG_FS core, which requires managing registers like GRSTCTL for core reset and GUSBCFG for PHY selection.

---

ca_usb_fs_dev_isr

Header: caos/drivers/inc/ca_glue_hw.h:39
GUARD: EN_CA_GLUE_HW

```
extern void ca_usb_fs_dev_isr(u32 usb_num)
```

Interrupt Service Routine (ISR) for the USB Full-Speed device peripheral.

Parametri:

Nome	Tipo	Descrizione
usb_num	u32	The index of the USB peripheral to service. Use 0 for single-USB devices; for multi-USB devices, refer to the specific hardware manual.

Ritorna:
void

Note:

- This function is an Interrupt Service Routine (ISR) and must be called exclusively from the hardware interrupt vector table associated with the USB peripheral. It is not thread-safe and must never be called from a standard task or thread context. Because it runs in interrupt context, it must execute as quickly as possible; avoid performing heavy processing or blocking operations inside this function. The most common mistake is attempting to use blocking caOS functions (like ca_os_delay or ca_mutex_lock) inside this ISR, which will cause system instability or deadlocks.

Uso:

- This function is the entry point for the USB hardware interrupt. It is typically registered in the startup vector table or the NVIC configuration. It handles low-level USB events such as bus resets, enumeration completion, and data packet reception by dispatching them to internal handler functions. It is the core of the USB stack's responsiveness; without it, the device will fail to communicate with the host or respond to enumeration requests.

Esempio:

```
/* Example of how the USB interrupt is linked to the system */

void USB_OTG_FS_IRQHandler(void)
{
    /* This is the hardware vector entry point */
    /* Call the caOS USB ISR handler for the first USB peripheral */
    ca_usb_fs_dev_isr(0);
}

/* Note: The internal handlers like handle_reset() or handle_rxflvl()
   are called internally by ca_usb_fs_dev_isr and are not part of the
   public caOS API. */
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call the ca_usb_fs_dev_isr function in my application?
A: You should not call this function manually from your application code. It is an Interrupt Service Routine (ISR) that must be registered in the hardware interrupt vector table or NVIC configuration to be triggered automatically by the USB peripheral hardware.

Code example conf:0.90 **Q:** How do I register the USB ISR in my startup code?
A: While the exact registration depends on your specific startup file, the function signature is: extern void ca_usb_fs_dev_isr(u32 usb_num);. For a single-USB device, you would typically map the USB interrupt vector to call ca_usb_fs_dev_isr(0);.

Parameters conf:1.00 **Q:** What does the u32 usb_num parameter represent?
A: The usb_num parameter is the index of the USB peripheral to service. Use 0 for single-USB devices. For multi-USB devices, you must refer to your specific hardware manual to determine the correct index.

Error/Edge case conf:1.00 **Q:** What happens if I call ca_os_delay inside ca_usb_fs_dev_isr?
A: Calling blocking caOS functions like ca_os_delay or ca_mutex_lock inside this ISR is a common mistake that will cause system instability or deadlocks, as the function must execute as quickly as possible and is not thread-safe.

Behavior conf:1.00 **Q:** Is ca_usb_fs_dev_isr thread-safe?
A: No, the function is not thread-safe. It is designed to run exclusively in an interrupt context and must never be called from a standard task or thread context.

HW differences conf:1.00 **Q:** How does the implementation of ca_usb_fs_dev_isr differ between the F1xx and F4xx families?
A: In the F1xx family, the implementation is essentially a stub that does nothing. In the F4xx family, the implementation is complex and directly interacts with the DWC2 USB_OTG_GlobalTypeDef registers (such as GINTSTS and GINTMSK) to handle specific events like bus resets, enumeration, and endpoint interrupts.

`ca_usb_fs_dev_set_address`

Header: `caos/drivers/inc/ca_glue_hw.h:37`
GUARD: `EN_CA_GLUE_HW`

```
1 extern void ca_usb_fs_dev_set_address(u32 usb_num, u8 addr)
```

Updates the USB device address in the hardware configuration register.

Parametri:

Nome	Tipo	Descrizione
`usb_num`	`u32`	The index of the USB peripheral (e.g., 0 for FS, 1 for HS). Must be a valid peripheral index for the specific MCU.
`addr`	`u8`	The 7-bit USB address assigned by the host during enumeration. Valid range is 0 to 127.

Ritorna:
void

Note:

- This function is a low-level hardware abstraction layer (HAL) call. It is not thread-safe and should only be called by the USB stack's control transfer handler. On some architectures (like H5xx), the address update is deferred until the status stage of the control transfer is complete; calling this manually outside the stack's state machine will likely break USB enumeration. Do not call this from an ISR unless the specific hardware implementation explicitly supports it, as it modifies shared peripheral registers.

Uso:

- This function is used exclusively during the USB enumeration process when the host sends a 'Set Address' request. It is typically called by the USB stack after the device has

successfully received the address from the host and is ready to switch to the new address. It is usually paired with other USB stack functions that manage the control endpoint (EP0) state machine to ensure the address change happens at the correct protocol timing.

Esempio:

```
/* Example: Handling a Set Address request from the USB host */
void handle_usb_set_address(u32 usb_num, u8 new_addr)
{
    /* The USB stack has received the SET_ADDRESS request.
       We apply the address to the hardware peripheral. */
    ca_usb_fs_dev_set_address(usb_num, new_addr);

    /* After setting the address, we must ensure the status stage
       of the control transfer is acknowledged. */
    // ca_usb_fs_dev_ep_write(usb_num, 0, NULL, 0);
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_usb_fs_dev_set_address in my USB stack implementation?
A: This function should be called exclusively during the USB enumeration process when the host sends a 'Set Address' request. It must be invoked by the USB stack's control transfer handler after the device has received the address and is ready to switch, ensuring it aligns with the protocol timing of the control endpoint (EP0) state machine.

Code example conf:1.00 **Q:** How do I correctly invoke the address update function in caOS?
A: You should call the function using the peripheral index and the 7-bit address provided by the host. Example: u32 usb_idx = 0; u8 new_addr = 0x05; ca_usb_fs_dev_set_address(usb_idx, new_addr);

Parameters conf:1.00 **Q:** What are the constraints on the 'addr' parameter for ca_usb_fs_dev_set_address?
A: The 'addr' parameter is a u8 representing the 7-bit USB address assigned by the host. The valid range is 0 to 127. Providing a value outside this range is not supported by the 7-bit addressing specification.

HW differences conf:1.00 **Q:** Are there architectural differences in how ca_usb_fs_dev_set_address handles the address update across MCU families?
A: Yes. On most families like F4xx, F7xx, H7xx, and U5xx, the function directly modifies the DCFG register of the DWC2 peripheral. However, on the H5xx family, the address update is deferred and stored in a state structure (g_st.pending_addr) to be applied only after the status stage of the control transfer is complete.

`ca_usb_get_speed`

Header: `caos/drivers/inc/usb.h:102`
Source: `caos/drivers/src/usb.c:67:71`
GUARD: `EN_USB`

```
ca_usb_speed_t ca_usb_get_speed(u32 usb_num)
```

Retrieves the current operational speed of the specified USB controller.

Parametri:

Nome	Tipo	Descrizione
`usb_num`	`u32`	The index of the USB controller to query. Must be less than USB_MAX_CTRL.

Ritorna:

Returns a ca_usb_speed_t enum value (e.g., CA_USB_SPEED_FS, CA_USB_SPEED_HS). Returns CA_USB_SPEED_FS as a default fallback if the provided index is invalid or the controller is not initialized.

Note:

- This function is non-blocking and thread-safe, as it performs a simple lookup in the internal driver table. It is safe to call from an ISR because it does not perform any complex operations or wait for hardware responses. The most common mistake is passing an uninitialized or out-of-bounds index, which will silently return the default Full-Speed (FS) value rather than an error code, potentially leading to incorrect timing assumptions in your USB stack logic.

Uso:

- This function is typically used during the USB enumeration process or when configuring endpoint buffers to determine if the hardware is operating at High-Speed (HS) or Full-Speed (FS). It is usually called after ca_usb_init and before opening endpoints with ca_usb_ep_open. Developers use it to dynamically adjust packet sizes or polling intervals based on the negotiated bus speed.

Esempio:

```c
/* Check USB speed before configuring endpoints */
u32 my_usb_id = 0;
ca_usb_speed_t speed = ca_usb_get_speed(my_usb_id);

if (speed == CA_USB_SPEED_HS) {
    /* High-speed logic: larger buffers might be needed */
    ca_printf("USB controller %u is running in High-Speed mode\n", my_usb_id);
} else {
    /* Full-speed logic */
    ca_printf("USB controller %u is running in Full-Speed mode\n", my_usb_id);
}

/* Proceed to open endpoints based on the detected speed */
// ca_usb_ep_open(my_usb_id, ...);
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_usb_get_speed() and why would I need to know the USB speed?
A: You should call ca_usb_get_speed() when you need to determine the current speed of a USB connection, such as to optimize data transfer rates or to configure USB endpoints appropriately. Knowing the speed (Full-Speed or Low-Speed) helps in managing USB communication efficiently, especially when handling different device types or when debugging connection issues.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_usb_get_speed() with caOS types?
A: ca_usb_speed_t speed = ca_usb_get_speed(0); if (speed == CA_USB_SPEED_FULL) { // Handle Full-Speed USB connection } else if (speed == CA_USB_SPEED_LOW) { // Handle Low-Speed USB connection } else { // Handle unknown or disconnected USB }

Parameters conf:0.90 **Q:** What does the usb_num parameter represent, and what are valid values for it?
A: The usb_num parameter represents the identifier of the USB controller, where 0 typically refers to USB0 and 1 to USB1. Valid values are integers from 0 up to the maximum number of USB controllers available on the specific STM32 chip. Using negative values or values exceeding the maximum number of controllers can lead to undefined behavior.

Return conf:0.95 **Q:** What do the return values of ca_usb_get_speed() mean, and what causes each one?
A: CA_USB_SPEED_FULL indicates the USB connection is operating at 12 Mbps. CA_USB_SPEED_LOW indicates a 1.5 Mbps connection. CA_USB_SPEED_UNKNOWN means the speed has not been determined yet or the connection is absent, which can happen if the USB controller is not initialized or no device is connected.

`ca_usb_hs_dev_clock_init`

Header: `caos/drivers/inc/ca_glue_hw.h:48`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_usb_hs_dev_clock_init(u32 usb_num)
```

Initializes the hardware clock and power configuration for the High-Speed USB peripheral.

Parametri:

Nome	Tipo	Descrizione
`usb_num`	`u32`	The index of the USB peripheral to initialize (e.g., 0 for USB_OTG_HS, 1 for USB_OTG_FS). Valid values are 0 or 1 depending on the specific STM32 hardware variant.

Ritorna:
void

Note:

- This function performs low-level register manipulation to enable the HSI48 oscillator and configure the USB clock source. It is a blocking function because it waits for the HSI48 oscillator to stabilize before returning. It is NOT thread-safe and must not be called from an ISR, as it performs busy-waiting loops. The most common mistake is calling this function before the system clock or power domains are properly initialized, which can lead to a system hang during the oscillator stabilization loop.

Uso:

- This function is typically the first step in the USB initialization sequence, called during the system boot phase or when a USB device is being brought online. It should be paired with subsequent hardware-specific initialization functions like ca_usb_hs_dev_gpio_init and ca_usb_hs_dev_init. It is used to ensure the peripheral has a valid clock signal before any USB communication or register access is attempted.

Esempio:

```
/* Initialize the USB High-Speed clock for the first USB peripheral */
ca_usb_hs_dev_clock_init(0U);

/* Proceed with GPIO and peripheral initialization */
/* ca_usb_hs_dev_gpio_init(0U); */
/* ca_usb_hs_dev_init(0U); */

/* Wait for USB connection to be established */
if (ca_usb_cdc_wait_connected(1000U)) {
    /* USB is ready for data transmission */
    ca_usb_cdc_write((u8 *)"USB Ready\r\n", 11U);
} else {
    /* Handle timeout or connection failure */
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_usb_hs_dev_clock_init in my application?
A: It should be called during the system boot phase or when bringing a USB device online. It is the first step in the USB initialization sequence and must be called before any USB communication or register access, followed by functions like ca_usb_hs_dev_gpio_init and ca_usb_hs_dev_init.

Code example conf:1.00 **Q:** Provide a minimal example of how to initialize the USB clock using caOS types.
A: void init_usb(void) { u32 usb_index = 0; ca_usb_hs_dev_clock_init(usb_index); }

Parameters conf:1.00 **Q:** What does the usb_num parameter represent and what are the valid values?
A: The usb_num parameter is a u32 representing the index of the USB peripheral to initialize. Valid values are 0 (typically for USB_OTG_HS) or 1 (typically for USB_OTG_FS), depending on the specific STM32 hardware variant.

Error/Edge case conf:1.00 **Q:** Why might my system hang when calling ca_usb_hs_dev_clock_init?
A: The function performs a busy-waiting loop to wait for the HSI48 oscillator to stabilize. A common mistake is calling this function before the system clock or power domains are properly initialized, which causes the stabilization loop to hang.

HW differences conf:1.00 **Q:** How does the implementation of ca_usb_hs_dev_clock_init differ across STM32 families?
A: On the H7xx family, the function performs active register manipulation on RCC (e.g., RCC_CR_HSI48ON, RCC_D2CCIP2R) and PWR (PWR_CR3_USB33DEN) peripherals. In contrast, for the F1xx, F4xx, F7xx, H5xx, and U5xx families, the function is currently implemented as a stub that performs no operations, indicated by the (void)n cast.

`ca_usb_hs_dev_connect`

Header: `caos/drivers/inc/ca_glue_hw.h:57`
GUARD: `EN_CA_GLUE_HW`

```
1 extern void ca_usb_hs_dev_connect(u32 usb_num)
```

Connects the USB High-Speed device to the host by enabling the D+ pull-up resistor.

Parametri:

Nome	Tipo	Descrizione
`usb_num`	`u32`	The index of the USB peripheral (e.g., 0 for USB_OTG_HS). Must be a valid index supported by the hardware.

Ritorna:
void

Note:

- This function is non-blocking and performs a direct register write to the USB controller's Device Control Register (DCTL) to clear the Soft Disconnect (SDIS) bit. It is not thread-safe in the sense that it should only be called once the USB device stack is fully initialized and ready to handle enumeration requests from the host. Calling this too early will cause the host to detect the device before the firmware is ready, leading to enumeration failures or 'Device Descriptor Request Failed' errors. It is not intended to be called from an ISR, as it is part of the high-level device management sequence.

Uso:

- This function is the final step in the USB device initialization sequence. It is typically called after the USB hardware has been clocked, the GPIOs configured, and all endpoints have been opened using ca_usb_hs_dev_ep_open. It is usually invoked by the higher-level management function ca_usb_dev_mng_start to signal to the host that the device is ready to begin the enumeration process. Do not call this function until your application is prepared to handle the subsequent setup packets from the host.

Esempio:

```c
/* Example of initializing a USB device and connecting it to the host */
void start_usb_device(u32 usb_num) {
    /* 1. Initialize hardware and clocks (platform specific) */
    ca_usb_hs_dev_clock_init(usb_num);
    ca_usb_hs_dev_gpio_init(usb_num);

    /* 2. Open necessary endpoints for communication */
    /* Endpoint 0 is mandatory for control transfers */
    ca_usb_hs_dev_ep_open(usb_num, 0x00, USB_EP_TYPE_CONTROL, 64);
    ca_usb_hs_dev_ep_open(usb_num, 0x81, CA_USB_EP_TYPE_BULK, 512);

    /* 3. Finalize setup and connect to host */
    /* This enables the D+ pull-up, triggering host enumeration */
    ca_usb_hs_dev_connect(usb_num);

    /* Now the device is visible to the host */
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** At what point in the USB initialization sequence should I call ca_usb_hs_dev_connect?
A: It should be the final step in the initialization sequence, called only after the USB hardware is clocked, GPIOs are configured, and all endpoints have been opened via ca_usb_hs_dev_ep_open. It is typically invoked by ca_usb_dev_mng_start to signal the host that the device is ready for enumeration.

Code example conf:1.00 **Q:** Provide a minimal C example showing how to connect the USB device using caOS types.
A: u32 usb_idx = 0U; /* Initialize hardware and endpoints first */ ca_usb_hs_dev_connect(usb_idx);

Parameters conf:0.90 **Q:** What does the usb_num parameter represent and what are valid values?
A: The usb_num parameter is the index of the USB peripheral (e.g., 0 for USB_OTG_HS). It must be a valid index supported by the hardware; for example, in the H7 implementation, the index must be less than 2.

Error/Edge case conf:1.00 **Q:** What happens if I call ca_usb_hs_dev_connect before the firmware is fully ready?
A: Calling it too early causes the host to detect the device before the firmware is prepared to handle setup packets, which leads to enumeration failures or 'Device Descriptor Request Failed' errors.

Behavior conf:1.00 **Q:** Is ca_usb_hs_dev_connect thread-safe or ISR-safe?
A: The function is non-blocking and performs a direct register write. It is not thread-safe and is not intended to be called from an ISR, as it is part of the high-level device management sequence.

HW differences conf:0.90 **Q:** How does the implementation of ca_usb_hs_dev_connect differ across MCU families?
A: In the H7xx family, the function explicitly clears the SDIS bit in the DCTL register of the DWC2 peripheral. In other families like F1xx, F4xx, F7xx, H5xx, and U5xx, the provided implementation is currently a stub that performs no register operations.

ca_usb_hs_dev_disconnect

Header: caos/drivers/inc/ca_glue_hw.h:58
GUARD: EN_CA_GLUE_HW

```
extern void ca_usb_hs_dev_disconnect(u32 usb_num)
```

Forces a USB High-Speed device to disconnect by setting the Soft Disconnect bit in the hardware controller.

Parametri:

Nome	Tipo	Descrizione
usb_num	u32	The index of the USB High-Speed peripheral (e.g., 0 or 1). Values outside the supported hardware range (typically >= 2) are ignored.

Ritorna:
void

Note:

- This function is non-blocking and performs a direct register write to the USB controller's DCTL register. It is thread-safe in the sense that it performs an atomic register modification, but it does not provide synchronization with the USB stack state. It is safe to call from an ISR, as it does not involve any OS primitives or blocking waits. The most common mistake is calling this function while the USB stack is actively transmitting data, which can lead to unexpected behavior or incomplete transfers; ensure the stack is in a quiescent state before disconnecting.

Uso:

- This function is used to programmatically simulate a physical cable unplug event, which is useful for forcing a re-enumeration of the USB device by the host. It is typically paired with ca_usb_hs_dev_connect to reset the connection after a configuration change or a firmware-level reset. In a typical lifecycle, you would call this to drop the connection, perform necessary hardware re-initialization, and then call the connect function to signal the host to re-enumerate the device.

Esempio:

```
/* Example: Force a USB re-enumeration */

void reset_usb_connection(u32 usb_id) {
    /* Disconnect the device from the host */
    ca_usb_hs_dev_disconnect(usb_id);

    /* Wait for a short period to ensure the host detects the disconnect */
    ca_os_delay_ms(100);

    /* Re-connect the device to trigger re-enumeration */
    ca_usb_hs_dev_connect(usb_id);
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_usb_hs_dev_disconnect and what is its primary purpose?
A: You should use this function to programmatically simulate a physical USB cable unplug event. It is primarily used to force a re-enumeration of the USB device by

the host, often paired with ca_usb_hs_dev_connect after a configuration change or firmware-level reset. Ensure the USB stack is in a quiescent state before calling it to avoid incomplete transfers.

Code example conf:1.00 **Q:** How do I implement a soft reset of the USB connection in caOS?
A: You can perform a reset by disconnecting and then reconnecting the device: ca_usb_hs_dev_disconn /* Perform re-init */ ca_usb_hs_dev_connect(0);

Parameters conf:1.00 **Q:** What are the valid values for the usb_num parameter in ca_usb_hs_dev_disconnect?
A: The usb_num parameter represents the index of the USB High-Speed peripheral. Valid values are typically 0 or 1. Any value outside the supported hardware range (typically >= 2) is ignored by the function.

HW differences conf:1.00 **Q:** Are there differences in how ca_usb_hs_dev_disconnect is implemented across different STM32 families?
A: Yes. For the H7xx family, the function performs a direct register write to the DCTL register (setting the USB_OTG_DCTL_SDIS bit) for valid indices. In contrast, for the F1xx, F4xx, F7xx, H5xx, and U5xx families, the provided implementation is a stub that performs no operation, meaning the function does not currently control hardware on those specific platforms.

`ca_usb_hs_dev_ep_clear_stall`

Header: `caos/drivers/inc/ca_glue_hw.h:61`
GUARD: `EN_CA_GLUE_HW`

```
1 extern bool ca_usb_hs_dev_ep_clear_stall(u32 usb_num, u8 ep_addr)
```

Clears the STALL condition on a specific USB High-Speed endpoint and resets its data toggle.

Parametri:

Nome	Tipo	Descrizione
`usb_num`	`u32`	The index of the USB peripheral (e.g., 0 for USB_OTG_HS). Must be less than 2.
`ep_addr`	`u8`	The endpoint address (e.g., 0x81 for IN endpoint 1, 0x02 for OUT endpoint 2). Must be a valid endpoint index.

Ritorna:
Returns true if the stall was successfully cleared, false if the parameters were invalid or the hardware does not support this operation.

Note:

- This function is non-blocking and executes immediately by modifying hardware registers. It is thread-safe regarding the USB state structure, but it should be used carefully as it resets the data toggle (PID) to DATA0, which can cause synchronization issues if the host is not expecting a reset. It is not intended to be called from an ISR unless the specific hardware implementation guarantees atomicity, which is generally not the case for complex USB register sequences. The most common beginner mistake is calling this function when the endpoint is not actually stalled, which may force an unnecessary data toggle reset and lead to packet sequence errors.

Uso:

- This function is typically used in the USB device stack error recovery sequence. When a host sends a request that the device cannot process, the device may respond with a

STALL; once the error condition is resolved, the host sends a 'Clear Feature' request, which triggers this function. It is usually paired with ca_usb_hs_dev_ep_stall to manage endpoint states and is called after the USB stack has successfully processed the control transfer that requested the stall clearance.

Esempio:

```c
/* Assuming we need to recover endpoint 1 after a protocol error */
u8 ep_addr = 0x81; // Endpoint 1 IN
u32 usb_idx = 0;

/* Clear the stall condition on the hardware */
if (ca_usb_hs_dev_ep_clear_stall(usb_idx, ep_addr)) {
    /* Successfully cleared, now we can resume normal data transmission */
    /* ca_usb_hs_dev_ep_write(usb_idx, ep_addr, buffer, size); */
} else {
    /* Handle error: invalid endpoint or hardware failure */
    ca_printf("Failed to clear stall on EP 0x%02X\n", ep_addr);
}
```

Q&A — fine-tuning review *7/7 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_usb_hs_dev_ep_clear_stall in my USB device stack?
A: This function is typically used during the USB device stack error recovery sequence. It should be called after the host sends a 'Clear Feature' request to resolve an error condition that previously caused the device to issue a STALL.

Code example conf:1.00 **Q:** Provide a minimal example of clearing a stall on endpoint 1 IN.
A: u32 usb_idx = 0; u8 ep_addr = 0x81; if (ca_usb_hs_dev_ep_clear_stall(usb_idx, ep_addr)) { /* Success */ }

Parameters conf:1.00 **Q:** What are the constraints for the usb_num and ep_addr parameters?
A: usb_num must be less than 2 (e.g., 0 for USB_OTG_HS). ep_addr must be a valid endpoint address, such as 0x81 for IN endpoint 1 or 0x02 for OUT endpoint 2.

Return conf:1.00 **Q:** What do the return values of ca_usb_hs_dev_ep_clear_stall indicate?
A: It returns true if the stall was successfully cleared. It returns false if the parameters provided are invalid (e.g., usb_num >= 2 or invalid endpoint index) or if the hardware does not support this operation.

Error/Edge case conf:1.00 **Q:** What is a common mistake when using this function?
A: The most common beginner mistake is calling this function when the endpoint is not actually stalled. This forces an unnecessary data toggle reset to DATA0, which can lead to packet sequence errors if the host is not expecting it.

Behavior conf:1.00 **Q:** Is ca_usb_hs_dev_ep_clear_stall safe to call from an ISR or concurrently?
A: The function is non-blocking and thread-safe regarding the USB state structure. However, it is not intended to be called from an ISR because the complex USB register sequences involved are generally not atomic.

HW differences conf:1.00 **Q:** How does the implementation of ca_usb_hs_dev_ep_clear_stall vary across STM32 families?
A: For F1xx, F4xx, F7xx, H5xx, and U5xx families, the function is a stub that returns false and performs no action. Only the H7xx implementation contains the logic to modify DWC2 registers (DIEPCTL/DOEPCTL) and reset data toggles for BULK/INTR endpoints.

`ca_usb_hs_dev_ep_close`

Header: `caos/drivers/inc/ca_glue_hw.h:60`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_usb_hs_dev_ep_close(u32 usb_num, u8 ep_addr)
```

Closes a specific USB High-Speed device endpoint and resets its internal driver state.

Parametri:

Nome	Tipo	Descrizione
usb_num	u32	The index of the USB peripheral (0 or 1). Values >= 2 are invalid.
ep_addr	u8	The endpoint address (e.g., 0x81 for IN endpoint 1). Endpoint 0 cannot be closed.

Ritorna:

Returns true if the endpoint was successfully closed, false if the endpoint was already closed, invalid, or if the index is out of range.

Note:

- This function is blocking because it performs hardware-level handshaking (SNAK/EPDIS) and waits for hardware status bits to clear, which can take time depending on bus traffic. It is NOT safe to call from an ISR because it uses busy-wait loops (DWC2_WAIT). You must ensure the endpoint was previously opened via ca_usb_hs_dev_ep_open before calling this. A common mistake is attempting to close Endpoint 0, which is reserved for control transfers and managed by the stack, or calling this while the USB bus is under heavy load, which might trigger the internal timeout.

Uso:

- This function is typically used during USB device reconfiguration, such as when a host sends a 'Set Configuration' request that changes the active interface or alternate setting. It is part of the cleanup sequence: first, you stop data transmission, then you call ca_usb_hs_dev_ep_close to release the hardware resources, and finally, you may re-open the endpoint with different parameters if needed. It is the counterpart to ca_usb_hs_dev_ep_open.

Esempio:

```
/* Example: Closing an endpoint during a configuration change */
#define EP_IN_1 0x81

void handle_usb_reconfig(u32 usb_idx) {
    /* Ensure we are not currently transmitting before closing */
    /* Assume a hypothetical check function exists */
    if (/* check if EP_IN_1 is busy */ false) {
        /* Wait for completion or abort */
    }

    /* Close the endpoint to free hardware resources */
    if (ca_usb_hs_dev_ep_close(usb_idx, EP_IN_1)) {
        /* Successfully closed, now we can re-initialize or stay idle */
    } else {
        /* Handle error: endpoint was already closed or invalid */
    }
}
```

Q&A — fine-tuning review *7/7 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_usb_hs_dev_ep_close in my USB stack implementation?

A: You should call it during USB device reconfiguration, such as when the host sends a 'Set Configuration' request that changes the active interface or alternate setting. It is used to release hardware resources after stopping data transmission, often as a cleanup step before re-opening the endpoint with different parameters.

Code example conf:1.00 **Q:** Provide a minimal example of how to close an IN endpoint using caOS types.
A: u32 usb_idx = 0; u8 ep_addr = 0x81; if (ca_usb_hs_dev_ep_close(usb_idx, ep_addr)) { /* Endpoint closed successfully */ }

Parameters conf:1.00 **Q:** What are the constraints on the ep_addr parameter?
A: The ep_addr represents the endpoint address (e.g., 0x81 for IN endpoint 1). Endpoint 0 is reserved for control transfers and cannot be closed using this function. Additionally, the endpoint must have been previously opened via ca_usb_hs_dev_ep_open.

Return conf:1.00 **Q:** What does a return value of false indicate?
A: A return value of false indicates that the endpoint was already closed, the provided endpoint address is invalid, the USB peripheral index is out of range (>= 2), or an attempt was made to close the reserved Endpoint 0.

Error/Edge case conf:1.00 **Q:** Why might ca_usb_hs_dev_ep_close fail during heavy bus traffic?
A: The function performs hardware-level handshaking (SNAK/EPDIS) and uses busy-wait loops (DWC2_WAIT) to monitor hardware status bits. If the USB bus is under heavy load, the hardware may not respond within the internal timeout period, causing the operation to fail.

Behavior conf:1.00 **Q:** Is it safe to call ca_usb_hs_dev_ep_close from an interrupt service routine (ISR)?
A: No, it is not safe. The function is blocking and utilizes busy-wait loops (DWC2_WAIT) to perform hardware handshaking, which can take significant time depending on bus traffic.

HW differences conf:1.00 **Q:** Are there differences in the implementation of ca_usb_hs_dev_ep_close across STM32 families?
A: Yes. While the function signature is consistent, the H7xx family provides a full implementation using DWC2 registers (e.g., DIEPCTL, DOEPCTL, GINTSTS, DCTL) to manage hardware handshaking and FIFO flushing. Conversely, the F1xx, F4xx, F7xx, H5xx, and U5xx families currently provide stub implementations that return false and perform no operations.

`ca_usb_hs_dev_ep_is_stalled`

Header: `caos/drivers/inc/ca_glue_hw.h:62`
GUARD: `EN_CA_GLUE_HW`

```
1 extern bool ca_usb_hs_dev_ep_is_stalled(u32 usb_num, u8 ep_addr)
```

Checks if a specific USB High-Speed device endpoint is currently in a stalled state.

Parametri:

Nome	Tipo	Descrizione
`usb_num`	`u32`	The index of the USB peripheral (e.g., 0 or 1). Must be within the valid range for the specific MCU.
`ep_addr`	`u8`	The endpoint address (e.g., 0x81 for IN endpoint 1, 0x02 for OUT endpoint 2). Must be a valid endpoint number for the hardware.

Ritorna:
Returns true if the endpoint is stalled, false if it is not stalled or if the parameters are invalid (e.g., out-of-bounds index).

Note:

- This function is non-blocking and thread-safe as it performs a direct register read. It is safe to call from an ISR because it does not involve OS primitives or waiting. The function relies on the underlying hardware registers (DWC2 on H7xx); on many other STM32 series, this function is currently unimplemented and will always return false. A common mistake is assuming this function will work on all STM32 platforms; always check the specific hardware implementation file for your target.

Uso:

- This function is typically used during USB error recovery or protocol debugging to determine why data transfers are failing. It is often paired with ca_usb_hs_dev_ep_clear_stall to reset an endpoint after a protocol error has been resolved. Developers use this to verify the state of the USB stack after receiving a stall condition from the host or after a local error event.

Esempio:

```c
/* Check if endpoint 1 IN is stalled before attempting to clear it */
#define EP_IN_1 0x81

void check_and_recover_usb(u32 usb_idx) {
    if (ca_usb_hs_dev_ep_is_stalled(usb_idx, EP_IN_1)) {
        /* Endpoint is stalled, attempt to clear the stall condition */
        /* Call the function to clear the stall bit in the hardware register */
        ca_usb_hs_dev_ep_clear_stall(usb_idx, EP_IN_1);
    } else {
        /* Endpoint is healthy, proceed with normal data transmission */
    }
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_usb_hs_dev_ep_is_stalled in my application?
A: This function is typically used during USB error recovery or protocol debugging to investigate why data transfers are failing. It is often paired with ca_usb_hs_dev_ep_clear_stall to reset an endpoint after a protocol error has been resolved or to verify the endpoint state after receiving a stall condition from the host.

Code example conf:1.00 **Q:** How do I check if endpoint 0x81 is stalled on USB peripheral 0?
A: You can call the function directly as shown: bool stalled = ca_usb_hs_dev_ep_is_stalled(0U, 0x81U); if (stalled) { /* handle stall */ }

Parameters conf:1.00 **Q:** What are the requirements for the ep_addr parameter?
A: The ep_addr parameter must be a valid endpoint address for the hardware, such as 0x81 for IN endpoint 1 or 0x02 for OUT endpoint 2. The function internally extracts the endpoint number and direction; providing an invalid endpoint number will result in the function returning false.

Return conf:1.00 **Q:** What does a return value of false indicate?
A: A return value of false indicates that the endpoint is not currently in a stalled state, or that the parameters provided (such as usb_num or ep_addr) were invalid/out-of-bounds. On many STM32 series where this function is unimplemented, it will also consistently return false.

HW differences conf:1.00 **Q:** Why does ca_usb_hs_dev_ep_is_stalled return false on my F4xx device?
A: The function relies on DWC2 hardware registers (specifically DIEPCTL and DOEPCTL) which are implemented for the H7xx family. For many other STM32 series, including F1xx, F4xx, F7xx, H5xx, and U5xx, the function is currently unimplemented and explicitly returns false regardless of the actual hardware state.

ca_usb_hs_dev_ep_open

Header: caos/drivers/inc/ca_glue_hw.h:51
GUARD: EN_CA_GLUE_HW

```
extern bool ca_usb_hs_dev_ep_open(u32 usb_num, u8 ep_addr,
    ca_usb_ep_type_t type, u16 mps)
```

Configures and enables a specific USB High-Speed device endpoint for data transmission or reception.

Parametri:

Nome	Tipo	Descrizione
usb_num	u32	The index of the USB peripheral (0 or 1). Values >= 2 are invalid.
ep_addr	u8	The endpoint address (e.g., 0x81 for IN, 0x02 for OUT). Must be within valid hardware limits.
type	ca_usb_ep_type_t	The transfer type (Control, Bulk, Interrupt, or Isochronous).
mps	u16	Maximum Packet Size in bytes. Must match the descriptor requirements.

Ritorna:
Returns true if the endpoint was successfully configured, false if the parameters are invalid or the hardware is not supported.

Note:

- This function is non-blocking and performs direct register manipulation on the USB OTG peripheral. It is not thread-safe; ensure that endpoint configuration is performed during the initialization phase or within a protected context to avoid race conditions. It is not intended to be called from an ISR. A common mistake is providing an endpoint address that exceeds the hardware's maximum endpoint count or failing to ensure the USB peripheral clock is initialized before calling this function.

Uso:

- This function is typically used during the USB device enumeration process after the USB peripheral has been initialized via ca_usb_hs_dev_init. It is called for each endpoint defined in the device's configuration descriptor to prepare the hardware registers for data flow. It is usually paired with ca_usb_hs_dev_ep_write or ca_usb_hs_dev_ep_read to handle actual data transfers once the host has configured the device.

Esempio:

```
/* Example: Opening a Bulk IN endpoint (0x81) with 512 bytes MPS */
#define EP_BULK_IN 0x81

void setup_usb_endpoints(u32 usb_idx) {
    /* Configure endpoint 1 as Bulk IN with 512 bytes MPS */
    bool success = ca_usb_hs_dev_ep_open(usb_idx, EP_BULK_IN,
    CA_USB_EP_TYPE_BULK, 512);

    if (!success) {
        /* Handle configuration error: log or enter safe state */
        ca_printf("Failed to open USB endpoint!\n");
    } else {
        ca_printf("Endpoint 0x81 opened successfully.\n");
    }
}
```

Q&A — fine-tuning review *7/7 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_usb_hs_dev_ep_open in my application?
A: This function is typically called during the USB device enumeration process, after the USB peripheral has been initialized via ca_usb_hs_dev_init. It should be invoked for each endpoint defined in the device's configuration descriptor to prepare the hardware registers for data flow.

Code example conf:0.90 **Q:** Provide a minimal example of how to open a Bulk IN endpoint using caOS types.
A: u32 usb_idx = 0; u8 ep_addr = 0x81; ca_usb_ep_type_t type = CA_USB_EP_BULK; u16 mps = 64; bool success = ca_usb_hs_dev_ep_open(usb_idx, ep_addr, type, mps);

Parameters conf:1.00 **Q:** What are the constraints on the usb_num and ep_addr parameters?
A: The usb_num parameter must be 0 or 1; values >= 2 are invalid. The ep_addr must be within valid hardware limits for the specific MCU, and the endpoint index derived from it must be less than USB_EP_MAX.

Return conf:1.00 **Q:** What do the return values of ca_usb_hs_dev_ep_open indicate?
A: It returns true if the endpoint was successfully configured. It returns false if the parameters are invalid (e.g., invalid usb_num or ep_addr) or if the hardware is not supported.

Error/Edge case conf:1.00 **Q:** What are some common mistakes when using ca_usb_hs_dev_ep_open?
A: Common mistakes include providing an endpoint address that exceeds the hardware's maximum endpoint count, or failing to ensure the USB peripheral clock is initialized before calling the function.

Behavior conf:1.00 **Q:** Is ca_usb_hs_dev_ep_open thread-safe or ISR-safe?
A: The function is non-blocking but not thread-safe; it must be called during initialization or within a protected context to avoid race conditions. It is not intended to be called from an ISR.

HW differences conf:1.00 **Q:** How does the implementation of ca_usb_hs_dev_ep_open differ across MCU families?
A: For the H7xx family, the function performs direct register manipulation on the DWC2 peripheral (e.g., DIEPCTL, DOEPCTL, and DAINTMSK). In contrast, for F1xx, F4xx, F7xx, H5xx, and U5xx families, the provided implementation is a stub that returns false, indicating the driver is not yet implemented for those specific families.

`ca_usb_hs_dev_ep_read`

Header: `caos/drivers/inc/ca_glue_hw.h:53`
GUARD: `EN_CA_GLUE_HW`

```
extern u16 ca_usb_hs_dev_ep_read(u32 usb_num, u8 ep_addr, u8 *buf, u16
    max_len)
```

Reads received data from a specific USB High-Speed device endpoint into a provided buffer.

Parametri:

Nome	Tipo	Descrizione
usb_num	u32	Index of the USB peripheral (0 or 1). Must be valid for the hardware configuration.
ep_addr	u8	Endpoint address (e.g., 0x01 for EP1 OUT). Must be an OUT endpoint; IN endpoints will return 0.
buf	u8 *	Pointer to the destination buffer where data will be copied. Must not be NULL.
max_len	u16	Maximum number of bytes to copy. If the received packet is larger, only this amount is copied.

Ritorna:
Returns the number of bytes actually copied into the buffer. Returns 0 if the endpoint is empty, the parameters are invalid, or the endpoint is an IN type.

Note:

- This function is non-blocking and thread-safe for EP1+ as it uses a ring buffer mechanism. It is NOT safe to call from an ISR because it manages critical sections and internal state updates that must be synchronized with the main task context. A common mistake is passing an IN endpoint address or a NULL buffer, which will immediately return 0. Ensure the endpoint has been successfully opened via ca_usb_hs_dev_ep_open before attempting to read, otherwise the internal state will not be ready to receive data.

Uso:

- This function is typically used within a task loop or a callback triggered by a USB event. The standard sequence is: 1) Initialize the USB peripheral, 2) Open the desired endpoint using ca_usb_hs_dev_ep_open, 3) Periodically poll or wait for a signal indicating data availability, 4) Call ca_usb_hs_dev_ep_read to retrieve the data into a local buffer, and 5) Process the data. It is often paired with ca_usb_hs_dev_ep_write to send responses back to the host.

Esempio:

```
u8 rx_buffer[64];
u16 bytes_received;

/* Assuming USB peripheral 0 and EP1 OUT (0x01) are already initialized and
    opened */
bytes_received = ca_usb_hs_dev_ep_read(0, 0x01, rx_buffer, sizeof(rx_buffer));

if (bytes_received > 0) {
    /* Data successfully copied to rx_buffer, process it here */
    ca_printf("Received %d bytes\n", bytes_received);
} else {
    /* No data available or error occurred */
}
```

Q&A — fine-tuning review *7/7 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** What is the standard workflow for reading data from a USB endpoint in caOS?
A: The standard sequence is: 1) Initialize the USB peripheral, 2) Open the desired endpoint using ca_usb_hs_dev_ep_open, 3) Periodically poll or wait for a signal indicating data availability, 4) Call ca_usb_hs_dev_ep_read to retrieve the data into a local buffer, and 5) Process the data.

Code example conf:1.00 **Q:** Provide a minimal C example of reading data from USB endpoint 1.

A: u8 buffer[64]; u16 bytes_read = ca_usb_hs_dev_ep_read(0, 0x01, buffer, 64); if (bytes_read > 0) { /* Process data */ }

Parameters conf:1.00 **Q:** What are the constraints on the ep_addr and buf parameters?

A: ep_addr must be an OUT endpoint; IN endpoints will return 0. The buf parameter must be a non-NULL pointer to a destination buffer.

Return conf:1.00 **Q:** What does the return value of ca_usb_hs_dev_ep_read represent?

A: It returns the number of bytes actually copied into the buffer. It returns 0 if the endpoint is empty, the parameters are invalid, or the endpoint is an IN type.

Error/Edge case conf:1.00 **Q:** What are common mistakes that cause ca_usb_hs_dev_ep_read to return 0?

A: Common mistakes include passing an IN endpoint address, providing a NULL buffer, or attempting to read from an endpoint that has not been successfully opened via ca_usb_hs_dev_ep_open.

Behavior conf:1.00 **Q:** Is ca_usb_hs_dev_ep_read safe to call from an ISR?

A: No, it is not safe to call from an ISR because it manages critical sections and internal state updates that must be synchronized with the main task context.

HW differences conf:1.00 **Q:** How does the H7 implementation handle EP0 versus EP1+?

A: On H7, EP0 uses legacy single staging where data is read from a DMA buffer, while EP1+ uses a dedicated RX ring buffer mechanism that allows for non-blocking, thread-safe access by the consumer task.

`ca_usb_hs_dev_ep_stall`

Header: `caos/drivers/inc/ca_glue_hw.h:59`
GUARD: `EN_CA_GLUE_HW`

```
1 extern void ca_usb_hs_dev_ep_stall(u32 usb_num, u8 ep_addr)
```

Forces a USB endpoint into a STALL state to signal a protocol error or unsupported request to the host.

Parametri:

Nome	Tipo	Descrizione
`usb_num`	`u32`	The index of the USB peripheral (e.g., 0 for USB_OTG_HS). Must be a valid peripheral index supported by the hardware.
`ep_addr`	`u8`	The endpoint address. Bit 7 is the direction (1 for IN, 0 for OUT). Bits 0-3 represent the endpoint number.

Ritorna:
void

Note:

- This function is non-blocking and executes immediately by modifying the hardware control registers. It is thread-safe in the sense that it performs atomic register writes, but it should be used carefully to avoid race conditions with the USB interrupt service routine. It is safe to call from an ISR, but typically it is called from the main USB stack logic when a request cannot be fulfilled. A common mistake is passing an invalid endpoint address or an endpoint number that has not been opened, which may lead to undefined behavior or ignored commands by the USB controller.

Uso:

- This function is used during USB enumeration or data transfer phases when the device receives a request it cannot handle (e.g., an unsupported standard request or a buffer overflow condition). It is typically paired with 'ca_usb_hs_dev_ep_open' during initialization and 'ca_usb_hs_dev_ep_clear_stall' when the host clears the stall condition. In a standard USB stack flow, you would call this when the control transfer state machine enters an error state, signaling the host that the specific endpoint is temporarily unable to process further traffic.

Esempio:

```
/* Example: Handling an unsupported request on Endpoint 1 IN */
#define EP1_IN_ADDR 0x81U

void handle_usb_request(u8 request_type) {
    if (request_type == 0xFFU) {
        /* Unsupported request: Stall the endpoint to notify the host */
        ca_usb_hs_dev_ep_stall(0, EP1_IN_ADDR);
    } else {
        /* Proceed with normal data processing */
        // ca_usb_hs_dev_ep_write(0, EP1_IN_ADDR, buffer, size);
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_usb_hs_dev_ep_stall in my USB stack?
A: You should call this function when the device receives a request it cannot handle, such as an unsupported standard request or a buffer overflow condition. It is typically used during USB enumeration or data transfer phases to signal to the host that a specific endpoint is temporarily unable to process further traffic.

Code example conf:1.00 **Q:** How do I implement a stall on endpoint 1 IN using caOS types?
A: You can trigger the stall by calling the function with the peripheral index and the endpoint address (where bit 7 is set for IN). Example: u32 usb_idx = 0; u8 ep_addr = 0x81; ca_usb_hs_dev_ep_stall(usb_idx, ep_addr);

Parameters conf:1.00 **Q:** How is the ep_addr parameter formatted for ca_usb_hs_dev_ep_stall?
A: The ep_addr parameter is an u8 where bits 0-3 represent the endpoint number, and bit 7 indicates the direction (1 for IN, 0 for OUT). Passing an invalid endpoint address or an endpoint that has not been opened may lead to undefined behavior or ignored commands.

HW differences conf:1.00 **Q:** Are there differences in how ca_usb_hs_dev_ep_stall is implemented across STM32 families?
A: Yes. For the H7xx family, the function actively modifies hardware registers (DWC2_EPIN/DWC2_E DIEPCTL/DOEPCTL bits) and includes logic to re-arm endpoint 0. In contrast, for the F1xx, F4xx, F7xx, H5xx, and U5xx families, the provided implementation is a stub that performs no action.

`ca_usb_hs_dev_ep_write`

Header: `caos/drivers/inc/ca_glue_hw.h:52`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_usb_hs_dev_ep_write(u32 usb_num, u8 ep_addr, const u8 *
    buf, u16 len)
```

Queues data for transmission on a specific USB High-Speed device endpoint.

Parametri:

Nome	Tipo	Descrizione
usb_num	u32	The index of the USB peripheral (usually 0 or 1).
ep_addr	u8	The endpoint address (e.g., 0x81 for IN endpoint 1). Must be an IN endpoint.
buf	const u8 *	Pointer to the data buffer to send. Must be valid for the duration of the transfer.
len	u16	Number of bytes to send. Must not exceed the endpoint's buffer capacity.

Ritorna:
Returns true if the data was successfully queued for transmission, false otherwise (e.g., invalid endpoint, buffer full, or hardware busy).

Note:

- This function is non-blocking; it copies data into an internal ring buffer and returns immediately. It is thread-safe for EP1+ because it manages its own internal queue, but it is NOT safe to call from an ISR. A common mistake is assuming the data has been physically sent the moment the function returns; the actual transmission happens asynchronously via the USB hardware engine. Ensure the endpoint is properly opened with ca_usb_hs_dev_ep_open before calling this function.

Uso:

- This function is used in the main data transmission loop of a USB device application. It is typically paired with ca_usb_hs_dev_ep_open to initialize the endpoint and ca_usb_hs_dev_connect to start the USB communication. In a typical sequence, you initialize the USB stack, open the required endpoints, and then call this function whenever you have data ready to be sent to the host.

Esempio:

```c
/* Example: Sending data over USB High-Speed Endpoint 1 */
#define EP1_IN 0x81

void send_usb_data(const u8 *data, u16 length) {
    /* Ensure the USB endpoint is ready before writing */
    if (ca_usb_hs_dev_ep_write(0, EP1_IN, data, length)) {
        /* Data successfully queued in the internal ring buffer */
    } else {
        /* Handle error: queue full or invalid endpoint */
        ca_printf("USB write failed!\n");
    }
}
```

Q&A — fine-tuning review *7/7 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_usb_hs_dev_ep_write in my application?
A: You should call this function in your main data transmission loop whenever you have data ready to be sent to the host. It must be called after the USB stack is initialized, the specific endpoint is opened with ca_usb_hs_dev_ep_open, and the USB connection is established via ca_usb_hs_dev_connect.

Code example conf:1.00 **Q:** Provide a minimal example of how to send data using ca_usb_hs_dev_ep_write.
A: u8 data[] = {0x01, 0x02, 0x03}; u32 usb_idx = 0; u8 ep = 0x81; if (ca_usb_hs_dev_ep_write(usb_i
ep, data, 3)) { /* Data queued successfully */ }

Parameters conf:1.00 **Q:** What are the requirements for the ep_addr and len parameters?
A: The ep_addr must represent an IN endpoint (e.g., 0x81). The len parameter specifies the number of bytes to send and must not exceed the capacity of the endpoint's buffer.

Return conf:1.00 **Q:** What does a return value of false indicate when calling ca_usb_hs_dev_ep_write?
A: A return value of false indicates that the data could not be queued. This can happen if the endpoint address is invalid, the internal buffer is full, the hardware is busy, or the endpoint was not properly opened.

Error/Edge case conf:1.00 **Q:** What is a common mistake developers make regarding the timing of data transmission?
A: A common mistake is assuming the data has been physically sent to the host the moment the function returns. The function is non-blocking and only queues the data; the actual transmission occurs asynchronously via the USB hardware engine.

Behavior conf:1.00 **Q:** Is ca_usb_hs_dev_ep_write safe to call from an Interrupt Service Routine (ISR)?
A: No, it is not safe to call this function from an ISR. While it is thread-safe for endpoints EP1 and higher due to internal queue management, it is strictly non-blocking and relies on mechanisms not suitable for ISR context.

HW differences conf:1.00 **Q:** Are there differences in the implementation of ca_usb_hs_dev_ep_write across STM32 families?
A: Yes. While the function signature is consistent, the H7xx implementation contains specific logic for managing TX ring buffers, DMA, and DWT latency tracking, whereas the F1xx, F4xx, F7xx, H5xx, and U5xx implementations provided in the documentation are currently stubs that return false.

`ca_usb_hs_dev_get_speed`

Header: `caos/drivers/inc/ca_glue_hw.h:55`
GUARD: `EN_CA_GLUE_HW`

```
1 extern ca_usb_speed_t ca_usb_hs_dev_get_speed(u32 usb_num)
```

Retrieves the current operational speed of the specified High-Speed USB device controller.

Parametri:

Nome	Tipo	Descrizione
`usb_num`	`u32`	The index of the USB peripheral to query. Valid values are typically 0 or 1 depending on the specific STM32 hardware configuration.

Ritorna:
Returns a ca_usb_speed_t enum value (e.g., CA_USB_SPEED_HS, CA_USB_SPEED_FS). If the index is invalid, it may return a default value like CA_USB_SPEED_FS depending on the specific hardware implementation.

Note:

- This function is non-blocking and thread-safe as it typically performs a simple register read or returns a cached value from the USB driver state. It is safe to call from an ISR, provided the hardware index is valid. The primary precondition is that the USB peripheral must have been initialized via the appropriate driver init function. A common mistake is passing an out-of-bounds index, which can lead to undefined behavior or return a default speed value that does not match the actual hardware capability.

Uso:

- This function is used during the USB enumeration process or when diagnosing connection issues to verify if the device has successfully negotiated High-Speed (HS) or Full-Speed (FS) mode. It is typically called after the USB stack has been initialized and the device

has been connected to a host. It is often paired with ca_usb_hs_dev_init to ensure the peripheral is ready before querying its status.

Esempio:

```c
/* Check the speed of the first USB HS controller */
u32 usb_index = 0;
ca_usb_speed_t speed = ca_usb_hs_dev_get_speed(usb_index);

if (speed == CA_USB_SPEED_HS) {
    /* Device is operating at High Speed */
    ca_printf("USB device %u is running at High Speed\n", usb_index);
} else {
    /* Device is operating at Full Speed or lower */
    ca_printf("USB device %u is running at Full Speed\n", usb_index);
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_usb_hs_dev_get_speed in my application?
A: You should call this function during the USB enumeration process or when diagnosing connection issues to verify if the device has successfully negotiated High-Speed (HS) or Full-Speed (FS) mode. It is typically called after the USB stack has been initialized and the device has been connected to a host, often paired with ca_usb_hs_dev_init.

Code example conf:1.00 **Q:** Provide a minimal example of how to check the USB speed using caOS types.
A: u32 usb_index = 0; ca_usb_speed_t speed = ca_usb_hs_dev_get_speed(usb_index); if (speed == CA_USB_SPEED_HS) { /* Handle High-Speed logic */ }

Parameters conf:1.00 **Q:** What does the usb_num parameter represent and what are the valid values?
A: The usb_num parameter is the index of the USB peripheral to query. Valid values are typically 0 or 1, depending on the specific STM32 hardware configuration. Passing an out-of-bounds index can lead to undefined behavior or return a default speed value.

Return conf:1.00 **Q:** What do the return values of ca_usb_hs_dev_get_speed indicate and what happens on failure?
A: The function returns a ca_usb_speed_t enum value, such as CA_USB_SPEED_HS or CA_USB_SPEED_FS. If an invalid index is provided, the function may return a default value like CA_USB_SPEED_FS, depending on the specific hardware implementation.

HW differences conf:1.00 **Q:** How does the implementation of ca_usb_hs_dev_get_speed differ between the H7xx family and other families like F4xx?
A: In the H7xx family, the function performs a bounds check (usb_num < 2U) and returns the speed from a global state array (g_usb[usb_num].speed), defaulting to CA_USB_SPEED_FS. In contrast, families like F1xx, F4xx, F7xx, H5xx, and U5xx currently implement the function as a stub that ignores the input index and always returns CA_USB_SPEED_HS.

`ca_usb_hs_dev_gpio_init`

Header: `caos/drivers/inc/ca_glue_hw.h:49`
GUARD: `EN_CA_GLUE_HW`

```c
extern void ca_usb_hs_dev_gpio_init(u32 usb_num)
```

Configures the physical GPIO pins required for the High-Speed USB peripheral interface.

Parametri:

Nome	Tipo	Descrizione
`usb_num`	`u32`	The index of the USB peripheral to configure. Typically 0 for the primary HS controller. Invalid values may result in no pins being configured or undefined behavior depending on the specific MCU implementation.

Ritorna:
void

Note:

- This function is a low-level hardware abstraction layer (HAL) call. It is non-blocking and executes synchronously to set up the GPIO registers (mode, speed, pull-up/down). It is not thread-safe and must be called during the system initialization phase before the USB peripheral is enabled. It is not intended to be called from an ISR. A common mistake is calling this function after the USB peripheral has already been initialized or while it is active, which can cause electrical contention or hardware faults.

Uso:

- This function is used exclusively during the board bring-up or peripheral initialization sequence. It is typically paired with ca_usb_hs_dev_clock_init to prepare the hardware environment before calling ca_usb_hs_dev_init. In a standard application, you would call this once at startup to map the physical pins to the USB controller's alternate function mode.

Esempio:

```c
/* Initialize the USB HS hardware pins before starting the USB stack */
void system_usb_setup(void) {
    /* Configure GPIOs for USB HS controller 0 */
    ca_usb_hs_dev_gpio_init(0U);

    /* Initialize the clock for the USB HS peripheral */
    ca_usb_hs_dev_clock_init(0U);

    /* Proceed to initialize the USB device stack */
    /* ca_usb_hs_dev_init(0U); // Function to initialize the USB controller */
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_usb_hs_dev_gpio_init in my application startup sequence?
A: This function should be called during the system initialization phase, specifically during board bring-up. It must be executed before the USB peripheral is enabled, typically paired with ca_usb_hs_dev_clock_init, and before calling ca_usb_hs_dev_init.

Code example conf:0.90 **Q:** Provide a minimal C example for initializing the primary USB HS controller.
A: u32 usb_index = 0U; ca_usb_hs_dev_clock_init(usb_index); ca_usb_hs_dev_gpio_init(usb_ind
ca_usb_hs_dev_init(usb_index);

Parameters conf:1.00 **Q:** What does the usb_num parameter represent and what are the consequences of passing an invalid value?
A: The usb_num parameter is the index of the USB peripheral to configure, where 0 is typically used for the primary HS controller. Passing invalid values may result in no pins being configured or undefined behavior depending on the specific MCU implementation.

Error/Edge case conf:1.00 **Q:** What is a common mistake when calling ca_usb_hs_dev_gpio_init?
A: A common mistake is calling this function after the USB peripheral has already been initialized or while it is active. This can cause electrical contention or hardware faults.

Behavior conf:1.00 **Q:** Is ca_usb_hs_dev_gpio_init thread-safe or ISR-safe?
A: The function is not thread-safe and is not intended to be called from an ISR. It is a non-blocking, synchronous function that configures GPIO registers.

HW differences conf:1.00 **Q:** How does the implementation of ca_usb_hs_dev_gpio_init differ across MCU families?
A: For the H7xx family, the function explicitly configures specific GPIO ports (A, B, C, H, I) with alternate functions and high-speed settings. In contrast, for the F1xx, F4xx, F7xx, H5xx, and U5xx families, the function is currently implemented as a stub that performs no operations, as indicated by the (void)n cast.

`ca_usb_hs_dev_init`

Header: `caos/drivers/inc/ca_glue_hw.h:50`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_usb_hs_dev_init(u32 usb_num, const ca_usb_cb_t *cb)
```

Initializes the High-Speed USB peripheral hardware and registers the callback structure for device operations.

Parametri:

Nome	Tipo	Descrizione
`usb_num`	`u32`	The index of the USB peripheral (0 or 1). Values >= 2 are invalid and will cause the function to return false.
`cb`	`const ca_usb_cb_t *`	Pointer to a structure containing callback functions for USB events. Must not be NULL.

Ritorna:
Returns true if initialization was successful, false if parameters were invalid or if the hardware failed to reset/initialize within the timeout period.

Note:

- This function performs low-level hardware configuration, including clock gating, DMA setup, and FIFO allocation. It is not thread-safe and must be called during the system initialization phase before any USB communication begins. It is not intended to be called from an ISR. A common mistake is calling this function before the necessary GPIO and clock configuration functions have been executed, which will lead to a timeout during the hardware reset phase.

Uso:

- This function is the first step in the USB device initialization sequence. It is typically followed by ca_usb_dev_mng_add to register specific USB interfaces (like CDC or HID) and finally ca_usb_dev_mng_start to enable the pull-up resistor and allow the host to enumerate the device. It should only be called once per peripheral instance during the application startup.

Esempio:

```
/* Define the callback structure for USB events */
const ca_usb_cb_t my_usb_callbacks = { /* ... callback function pointers ... */
    };

void usb_setup_task(void) {
```

```
    u32 usb_instance = 0;

    /* Perform hardware-specific clock and GPIO setup before initializing the
    driver */
    ca_usb_hs_dev_clock_init(usb_instance);
    ca_usb_hs_dev_gpio_init(usb_instance);

    /* Initialize the USB HS peripheral */
    if (ca_usb_hs_dev_init(usb_instance, &my_usb_callbacks)) {
        /* Proceed to add device classes and start the USB manager */
        /* ca_usb_dev_mng_add(...) */
        /* ca_usb_dev_mng_start(...) */
    } else {
        /* Handle initialization failure */
        ca_printf("USB HS Init Failed!\n");
    }
}
```

Q&A — fine-tuning review *7/7 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** At what point in the application lifecycle should I call ca_usb_hs_dev_init?
A: It must be called during the system initialization phase, before any USB communication begins. It is the first step in the USB device initialization sequence, typically followed by ca_usb_dev_mng_add and ca_usb_dev_mng_start.

Code example conf:0.90 **Q:** Provide a minimal C example for initializing the USB peripheral.
A: ca_usb_cb_t my_cbs = {0}; u32 usb_idx = 0U; if (ca_usb_hs_dev_init(usb_idx, &my_cbs)) { /* Success */ }

Parameters conf:1.00 **Q:** What are the constraints for the usb_num and cb parameters?
A: usb_num must be 0 or 1; values >= 2 are invalid. The cb parameter is a pointer to a ca_usb_cb_t structure and must not be NULL.

Return conf:1.00 **Q:** What does a false return value from ca_usb_hs_dev_init indicate?
A: It indicates that the initialization failed. This occurs if the input parameters are invalid (e.g., usb_num >= 2 or NULL callback) or if the hardware failed to reset/initialize within the specified timeout period.

Error/Edge case conf:1.00 **Q:** What is a common mistake that leads to a timeout during the hardware reset phase?
A: A common mistake is calling ca_usb_hs_dev_init before the necessary GPIO and clock configuration functions have been executed.

Behavior conf:1.00 **Q:** Is ca_usb_hs_dev_init thread-safe or ISR-safe?
A: The function is not thread-safe and is not intended to be called from an ISR. It performs low-level hardware configuration including clock gating, DMA setup, and FIFO allocation.

HW differences conf:1.00 **Q:** How does the implementation of ca_usb_hs_dev_init differ across STM32 families?
A: For the H7xx family, the function performs complex DWC2 peripheral register configuration (e.g., GAHBCFG, GUSBCFG, GRSTCTL). In contrast, the provided implementations for f1xx, f4xx, f7xx, h5xx, and u5xx are stubs that return false and perform no operations.

`ca_usb_hs_dev_isr`

Header: `caos/drivers/inc/ca_glue_hw.h:56`
GUARD: `EN_CA_GLUE_HW`

```
extern void ca_usb_hs_dev_isr(u32 usb_num)
```

Interrupt Service Routine (ISR) for handling High-Speed USB device events.

Parametri:

Nome	Tipo	Descrizione
`usb_num`	`u32`	The index of the USB peripheral (e.g., 0 or 1 for STM32H7). Must be a valid index supported by the hardware.

Ritorna:
void

Note:

- This function is an Interrupt Service Routine (ISR) and must be called directly from the hardware interrupt vector table for the USB peripheral. It is not thread-safe and must never be called from a standard task or thread context. Because it runs in an interrupt context, it must execute as quickly as possible; avoid performing heavy processing or blocking operations inside this function. The function handles low-level USB events like bus resets, enumeration, and endpoint interrupts by clearing hardware flags and triggering internal state machine updates.

Uso:

- This function is the backbone of the USB device stack. It is typically registered in the startup code or the NVIC configuration to be triggered automatically by the hardware whenever a USB event occurs. It works in conjunction with the USB initialization sequence (e.g., ca_usb_hs_dev_init) and the endpoint management functions. You do not call this manually; the hardware triggers it, and it in turn calls internal handlers to manage the USB protocol state machine.

Esempio:

```c
/* Example of how the USB interrupt is linked to the hardware vector */
void OTG_HS_IRQHandler(void) {
    /* Call the caOS USB ISR handler for the first USB peripheral */
    ca_usb_hs_dev_isr(0);
}

/* Typical initialization sequence in a task */
void usb_task(void *p) {
    /* Initialize the USB hardware peripheral */
    /* ca_usb_hs_dev_init(0); */

    while(1) {
        /* Wait for USB events or process data */
        ca_os_delay_ms(100);
    }
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I manually call ca_usb_hs_dev_isr in my application code?
A: You should never call ca_usb_hs_dev_isr manually. It is an Interrupt Service Routine (ISR) designed to be triggered automatically by the hardware. It must be registered in the startup code or NVIC configuration to handle USB events like resets and enumeration directly from the hardware interrupt vector table.

Code example conf:0.90 **Q:** How do I register the ISR for a USB peripheral in caOS?
A: The function is typically registered in the startup code or NVIC configuration. While the specific NVIC registration code depends on the architecture, the ISR signature is: void ca_usb_hs_dev_isr(u32 usb_num);. For example, to handle USB peripheral 0, the hardware vector table would point to this function.

Parameters conf:1.00 **Q:** What does the u32 usb_num parameter represent?
A: The usb_num parameter is the index of the USB peripheral, such as 0 or 1 for STM32H7 devices. It must be a valid index supported by the specific hardware implementation to ensure the function accesses the correct USB_OTG_GlobalTypeDef registers.

Error/Edge case conf:1.00 **Q:** Can I perform heavy data processing or blocking operations inside ca_usb_hs_dev_isr?
A: No. Because this function runs in an interrupt context, it must execute as quickly as possible. Performing heavy processing or blocking operations is prohibited as it will interfere with the USB protocol state machine and system stability.

HW differences conf:1.00 **Q:** How does the implementation of ca_usb_hs_dev_isr differ between the STM32H7 and other families like F1xx or F4xx?
A: In the H7xx family, the function actively manages the DWC2 peripheral by reading GINTSTS and GINTMSK registers to handle specific events like USBRST, ENUMDNE, and endpoint interrupts. In contrast, the implementations for F1xx, F4xx, F7xx, H5xx, and U5xx provided in the documentation are currently empty stubs that perform no operations.

`ca_usb_hs_dev_set_address`

Header: `caos/drivers/inc/ca_glue_hw.h:54`
GUARD: `EN_CA_GLUE_HW`

```
1 extern void ca_usb_hs_dev_set_address(u32 usb_num, u8 addr)
```

Sets the USB device address in the hardware register during the enumeration process.

Parametri:

Nome	Tipo	Descrizione
`usb_num`	`u32`	The index of the USB peripheral (e.g., 0 for USB_OTG_HS). Must be a valid hardware index.
`addr`	`u8`	The 7-bit address assigned by the USB host during the SET_ADDRESS request. Valid range is 0 to 127.

Ritorna:
void

Note:

- This function is a low-level hardware abstraction that directly modifies the USB peripheral's device configuration register. It is not thread-safe and is typically called from the USB interrupt service routine (ISR) when the host sends a SET_ADDRESS request. It is non-blocking and executes immediately. A common mistake is calling this function before the USB peripheral has been properly initialized or while the device is in a state where address changes are prohibited by the USB specification.

Uso:

- This function is used exclusively during the USB enumeration phase. It is typically called after the device receives a SETUP packet containing the SET_ADDRESS request from the host. It is usually paired with other low-level USB functions that handle endpoint management and status reporting. Once the address is set, the hardware will respond to the new address for subsequent transactions.

Esempio:

```
/* Example: Handling a USB SET_ADDRESS request in the USB ISR */
void usb_handle_setup_packet(u32 usb_num, u8 new_addr) {
    /* The host has requested a new address.
       We must update the hardware register to match the assigned address. */
    ca_usb_hs_dev_set_address(usb_num, new_addr);

    /* After setting the address, we typically send a zero-length status packet
       to acknowledge the request to the host. */
    /* ca_usb_hs_dev_ep_write(usb_num, 0, NULL, 0); // Send status stage */
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_usb_hs_dev_set_address in my USB stack implementation?
A: This function should be called exclusively during the USB enumeration phase, specifically after the device receives a SETUP packet containing the SET_ADDRESS request from the host. It is typically invoked from within the USB interrupt service routine (ISR).

Code example conf:1.00 **Q:** Provide a minimal example of how to use ca_usb_hs_dev_set_address.
A: void handle_set_address(u32 usb_idx, u8 new_addr) { ca_usb_hs_dev_set_address(usb_idx, new_addr); }

Parameters conf:1.00 **Q:** What are the constraints for the addr parameter in ca_usb_hs_dev_set_address?
A: The addr parameter represents the 7-bit address assigned by the USB host. The valid range is 0 to 127. Passing a value outside this range is invalid as it must conform to the USB specification for device addressing.

HW differences conf:1.00 **Q:** How does the implementation of ca_usb_hs_dev_set_address differ across STM32 families?
A: For the H7xx family, the function directly manipulates the DWC2_DEV(usb_num)->DCFG register by masking and setting the USB_OTG_DCFG_DAD bits. In contrast, for the F1xx, F4xx, F7xx, H5xx, and U5xx families, the provided implementation is a stub that performs no hardware operations, as indicated by the empty function bodies.

`ca_usb_init`

Header: `caos/drivers/inc/usb.h:86`
Source: `caos/drivers/src/usb.c:23:41`
GUARD: `EN_USB`

```
bool ca_usb_init(ca_usb_speed_t speed, u32 usb_num, const ca_usb_cb_t *
    cb)
```

Initializes the USB hardware peripheral, clocks, and GPIOs for a specific USB controller.

Parametri:

Nome	Tipo	Descrizione
speed	ca_usb_speed_t	The desired USB speed (e.g., CA_USB_SPEED_FS for Full Speed, CA_USB_SPEED_HS for High Speed). Must be supported by the hardware.
usb_num	u32	The index of the USB controller to initialize. Must be less than USB_MAX_CTRL (usually 0 or 1).
cb	const ca_usb_cb_t *	Pointer to a structure containing callback functions for USB events (e.g., reset, setup, data received). Must not be NULL.

Ritorna:
Returns true if initialization succeeded, false if the controller index is invalid or the underlying hardware driver failed to initialize.

Note:

- This function is not thread-safe and must be called during the system initialization phase before any USB communication occurs. It is not intended to be called from an ISR. The function performs hardware-level configuration (clocks and GPIOs), so it is a mandatory precondition for any other USB operations. A common mistake is calling this function multiple times for the same controller without proper deinitialization, or passing an invalid usb_num, which will immediately return false and prevent the USB peripheral from functioning.

Uso:

- This function is the entry point for the USB stack. It is typically called once at startup to prepare the hardware. After successful initialization, the developer usually calls ca_usb_connect to physically enable the D+ pull-up resistor and allow the host to enumerate the device. It is often paired with ca_usb_dev_mng_start if using the device management layer, or used directly in custom implementations where the developer manages endpoints manually.

Esempio:

```
ca_usb_cb_t cb = { .on_reset = my_reset_handler, .on_setup = my_setup_handler };

// Initialize USB controller 0 at Full Speed
if (!ca_usb_init(CA_USB_SPEED_FS, 0U, &cb)) {
    // Log error using standard output
    ca_stdout_put_n("[USB] Initialization failed!");
} else {
    // Hardware is ready, now signal the host that we are present
    ca_usb_connect(0U);
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_usb_init and why is it important to call it before other USB functions?
A: You should call ca_usb_init before any other USB-related functions to properly configure the USB hardware controller, including setting up clocks and GPIOs. It is important because it initializes the USB controller and prepares it for operation, ensuring that the hardware is correctly set up for communication. The function must be called once before using any other USB functions in the system.

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_usb_init with caOS types?

A: ca_usb_cb_t cb = { .on_tx_done = NULL, .on_rx_done = NULL }; bool success = ca_usb_init(0, &cb);

Parameters conf:0.90 **Q:** What does the usb_num parameter in ca_usb_init represent, and what are valid values for it?

A: The usb_num parameter specifies which USB controller to initialize (e.g., 0 for USB0, 1 for USB1). Valid values depend on the specific STM32 device being used, and invalid values may cause hardware errors or undefined behavior. The parameter must correspond to a supported USB controller on the target microcontroller.

Return conf:0.95 **Q:** What does ca_usb_init return, and what causes it to return false?

A: ca_usb_init returns true if initialization is successful, and false if it fails. Failures can be caused by an invalid usb_num, a NULL callback pointer (cb), or hardware errors such as incorrect clock or GPIO configuration. The function may also fail if the USB controller is not enabled or supported by the device.

`ca_usb_isr`

Header: `caos/drivers/inc/usb.h:105`
Source: `caos/drivers/src/usb.c:73:77`
GUARD: `EN_USB`

```
void ca_usb_isr(u32 usb_num)
```

Dispatches USB hardware interrupts to the registered driver callback for a specific USB controller.

Parametri:

Nome	Tipo	Descrizione
`usb_num`	`u32`	The index of the USB controller (e.g., 0 for USB_OTG_FS). Must be less than USB_MAX_CTRL.

Ritorna:
void

Note:

- This function is designed to be called directly from the STM32 hardware interrupt vector (ISR). It is non-blocking and executes in interrupt context, meaning it must be extremely fast to avoid starving the system. It is thread-safe only if the underlying driver callback is re-entrant or protected by appropriate hardware-level locking. The most common beginner mistake is performing heavy processing or blocking operations (like waiting for a semaphore) inside the callback triggered by this ISR, which will cause system instability or missed USB events.

Uso:

- This function is the entry point for the USB peripheral's interrupt service routine. It is typically called from the STM32 startup file's vector table (e.g., OTG_FS_IRQHandler). It pairs with the driver initialization sequence where a specific callback is registered via the internal driver structure. Once the hardware triggers an interrupt, this function checks if a valid driver is registered for that controller and executes the corresponding handler to process USB packets or state changes.

Esempio:

```
/* Example of an ISR implementation in the vector table */
void OTG_FS_IRQHandler(void)
```

```c
{
    /* Dispatch the interrupt to the caOS USB driver layer */
    /* usb_num 0 corresponds to the first USB controller */
    ca_usb_isr(0);
}

/* Example of initializing the USB subsystem */
void setup_usb(void)
{
    /* Initialize the USB hardware and register callbacks */
    ca_usb_init(0);

    /* Connect the device to the bus */
    ca_usb_connect(0);

    /* Wait for connection before starting data transfer */
    if (ca_usb_cdc_wait_connected(0)) {
        /* Proceed with data transmission */
        ca_usb_cdc_write(0, "Hello USB", 9);
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should ca_usb_isr() be called, and why is it important for USB interrupt handling?
A: ca_usb_isr() should be called from an Interrupt Service Routine (ISR) to handle USB interrupts. It is important because it processes USB events by calling the corresponding hardware interrupt handler, ensuring proper communication with USB devices. It must only be invoked from ISR context and not from RTOS tasks or non-interrupt code.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_usb_isr() with caOS types?
A: void USB0_IRQHandler(void) { ca_usb_isr(0); }
void USB1_IRQHandler(void) { ca_usb_isr(1); }

Parameters conf:0.90 **Q:** What does the usb_num parameter in ca_usb_isr() represent, and what are the valid values?
A: The usb_num parameter specifies which USB controller to handle interrupts for. Valid values are 0 or 1, corresponding to USB0 and USB1 respectively. Using any other value (e.g., 2 or more) may lead to crashes or undefined behavior because ca_usb_hw_isr might access invalid hardware registers.

`ca_usb_ncm_is_up`

Header: `caos/usb/inc/usb_ncm.h:88`
Source: `caos/usb/src/usb_ncm.c:433:436`
GUARD: `EN_USB_NCM`

```c
bool ca_usb_ncm_is_up(void)
```

Returns the current connection status of the USB NCM (Network Control Model) interface.

Ritorna:
Returns true if the USB NCM link is established and ready for data transfer, false otherwise.

Note:

- This function is non-blocking and thread-safe, as it simply reads a static internal state variable maintained by the USB stack. It is safe to call from both tasks and ISRs. The most

common mistake is assuming that a 'true' return value guarantees that the host has already assigned an IP address or that the network stack is fully configured; it only indicates that the USB physical and logical link (NCM) is active. Ensure the USB subsystem has been initialized via the appropriate stack startup functions before relying on this status.

Uso:

- This function is typically used in monitoring tasks or network management loops to detect when the USB cable is plugged in or the host has initialized the NCM driver. It is often paired with ca_os_sleep to poll the link status periodically without starving other tasks of CPU time. Developers use it to gate network-dependent operations, such as starting DHCP or sending data, ensuring that the application does not attempt to transmit frames over a disconnected USB interface.

Esempio:

```c
/* Monitor USB NCM link status and report changes */
void usb_monitor_task(void *arg) {
    bool prev_up = false;
    while (1) {
        bool current_up = ca_usb_ncm_is_up();

        if (current_up != prev_up) {
            ca_stdout_printf_n("[USB] Link status changed to: %s", current_up ? "UP" : "DOWN");
            prev_up = current_up;
        }

        /* Wait 500ms before checking again to save CPU cycles */
        ca_os_sleep(500U);
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_usb_ncm_is_up() in my embedded application?
A: You should call ca_usb_ncm_is_up() to check the current state of the USB NCM connection without blocking the calling thread. It is useful for periodic status checks or conditional logic in your application where you need to know if the USB network connection is active. It is not meant to be called from an ISR since it's not designed for interrupt context.

Code example conf:0.90 **Q:** Can you show a minimal C code example using ca_usb_ncm_is_up() with caOS types?
A: bool is_up = ca_usb_ncm_is_up(); if (is_up) { // USB NCM is active } else { // USB NCM is not active }

Return conf:0.95 **Q:** What do the return values of ca_usb_ncm_is_up() indicate and what could cause it to fail?
A: The function returns true if the USB NCM connection is active (UP) and false if it is not active (DOWN). It cannot fail because it only reads a global variable and does not perform any operations that could result in an error. There are no preconditions or failure modes associated with this function.

`ca_usb_ncm_send_frame`

Header: `caos/usb/inc/usb_ncm.h:85`
Source: `caos/usb/src/usb_ncm.c:393:431`
GUARD: `EN_USB_NCM`

```
bool ca_usb_ncm_send_frame(const u8 *eth, u16 len)
```

Encapsulates an Ethernet frame into a USB NCM NTB (Network Transfer Block) and sends it over the USB bulk endpoint.

Parametri:

Nome	Tipo	Descrizione
`eth`	`const u8 *`	Pointer to the Ethernet frame buffer. Must not be NULL.
`len`	`u16`	Length of the Ethernet frame in bytes. Must be greater than 0 and less than or equal to USB_NCM_MAX_FRAME.

Ritorna:
Returns true if the frame was successfully encapsulated and queued for transmission, false otherwise (e.g., USB not initialized, invalid parameters, or frame too large).

Note:

- This function is not thread-safe and should not be called from an ISR. It performs memory operations and interacts with the USB stack, which requires a stable, initialized USB state. The function is blocking in the sense that it performs the encapsulation and triggers the USB transfer, but it does not wait for the hardware to finish the physical transmission. A common mistake is passing a buffer that is not properly aligned or is larger than the defined USB_NCM_MAX_FRAME, which will cause the function to return false immediately.

Uso:

- This function is typically used in the network interface layer to transmit outgoing Ethernet packets over a USB NCM connection. It is usually called after a network buffer has been prepared by the stack. In a typical sequence, you would receive a buffer from the network stack, call this function to send it, and if it returns true, you must free the network buffer using ca_net_buf_free to prevent memory leaks. It is the primary bridge between the generic Ethernet frame format and the USB-specific NCM protocol encapsulation.

Esempio:

```
/* Example: Sending a network buffer over NCM */
#include "caos.h"

void send_packet_example(u16 buf_idx) {
    /* Retrieve the network buffer pointer */
    ca_net_buf_t *b = ca_net_buf_ptr(buf_idx);

    if (b != NULL) {
        /* Attempt to send the frame via NCM */
        bool success = ca_usb_ncm_send_frame(b->data + b->data_off, b->data_len)
    ;

        if (success) {
            /* If sent successfully, release the buffer back to the pool */
            ca_net_buf_free(buf_idx);
        } else {
            /* Handle error: log or drop the packet */
            ca_printf("Failed to send NCM frame\n");
        }
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_usb_ncm_send_frame in a caOS application?
A: You should call ca_usb_ncm_send_frame when you need to transmit an Ethernet frame over USB-NCM interface. It is typically used in network applications where data needs to be sent from the microcontroller to a host PC via USB. The function encapsulates the Ethernet frame into a USB NTB and sends it via bulk-in endpoint.

Code example conf:0.90 **Q:** Can you provide a minimal C example showing how to use ca_usb_ncm_send_fram
A: u8 eth_frame[64] = {0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08}; bool success = ca_usb_ncm_send_frame(eth_frame, 64); if (!success) { // Handle error }

Parameters conf:0.95 **Q:** What are the valid ranges for the eth buffer pointer and len parameter in ca_usb_ncm_send_frame?
A: The eth parameter must point to a valid, non-NULL buffer containing the Ethernet frame. The len parameter must be greater than 0 and less than or equal to USB_NCM_MAX_FRAME (typically 1514 bytes). Values outside this range will cause the function to return false immediately.

Return conf:0.95 **Q:** What does it mean when ca_usb_ncm_send_frame returns false?
A: A return value of false indicates that the frame was not sent successfully. This can happen due to invalid parameters (like NULL buffer or len out of range), an inactive NCM interface (s_up is false), NTB size exceeding limits, or errors during USB transfer.

Error/Edge case conf:0.85 **Q:** What are common mistakes when using ca_usb_ncm_send_frame?
A: Common mistakes include calling the function with a NULL eth buffer pointer, passing a len parameter that is zero or exceeds USB_NCM_MAX_FRAME, calling the function from an ISR context (not thread-safe), and not ensuring the NCM interface is active (s_up must be true). Also, the caller must free the input buffer after successful transmission.

Behavior conf:0.90 **Q:** Is ca_usb_ncm_send_frame blocking or non-blocking, and is it safe to call from an ISR?
A: The function is blocking only if ca_usb_ep_write is blocking (which depends on the underlying USB driver). It is not ISR-safe and must be called from a task context. It is also not thread-safe by itself and requires protection in a multitasking environment.

`ca_usb_set_address`

Header: `caos/drivers/inc/usb.h:99`
Source: `caos/drivers/src/usb.c:61:65`
GUARD: `EN_USB`

```
1 void ca_usb_set_address(u32 usb_num, u8 addr)
```

Updates the USB device address assigned by the host during the enumeration process.

Parametri:

Nome	Tipo	Descrizione
`usb_num`	`u32`	The index of the USB controller instance. Must be less than USB_MAX_CTRL.
`addr`	`u8`	The 7-bit address assigned by the USB host. Values above 127 are invalid.

Ritorna:
void. This function does not return a status code.

Note:

- This function is a low-level driver call used specifically during the USB enumeration phase. It is not thread-safe and must be called only when the USB controller is initialized and

active. It is typically called in response to a SET_ADDRESS request from the host. A common mistake is calling this function before the host has actually requested the address change or using an invalid address range, which will cause the device to drop off the USB bus.

Uso:

- This function is used during the USB device enumeration sequence. It is typically paired with ca_usb_ep_write to send a zero-length status packet back to the host after the address has been updated. It is called after the device receives a setup packet containing the USB_REQ_SET_ADDRESS request, ensuring the hardware is ready to respond to the new address for subsequent transactions.

Esempio:

```
/* Example: Handling a SET_ADDRESS request from the USB host */
void handle_usb_setup_request(u32 usb_num, u8 request, u16 value) {
    if (request == 0x05U) { /* USB_REQ_SET_ADDRESS */
        u8 new_addr = (u8)(value & 0x7FU);

        /* Apply the new address to the USB hardware */
        ca_usb_set_address(usb_num, new_addr);

        /* Send a zero-length status packet to acknowledge the request */
        /* ca_usb_ep_write(usb_num, 0x80U, NULL, 0U); */
    }
}
```

Q&A — fine-tuning review *3/3 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_usb_set_address and why is it important in USB communication?
A: You should call ca_usb_set_address after receiving a SET_ADDRESS request on endpoint 0 (EP0) during the USB configuration process. This function is important because it assigns a unique address to the USB device, which is required for subsequent communication with the host. It must be called in a non-ISR context and is typically used in the USB driver's configuration handling logic.

Code example conf:0.90 **Q:** Can you provide a minimal C code example showing how to use ca_usb_set_address with caOS types?
A: u32 usb_num = 0; u8 addr = 5; ca_usb_set_address(usb_num, addr);

Parameters conf:0.90 **Q:** What are the valid ranges for the parameters in ca_usb_set_address and what happens if invalid values are passed?
A: The usb_num parameter must be a valid USB interface number, typically 0 or 1 depending on the chip's capabilities. The addr parameter must be between 1 and 127, as 0 is reserved for the default address. Passing invalid values such as 0 or values greater than 127 may lead to unpredictable behavior or hardware errors, as the function does not validate these inputs.

`ca_valid_pointer`

Header: `caos/inc/utils.h:77`
Source: `caos/src/utils.c:171:180`
GUARD: `EN_UTILS`

```
bool ca_valid_pointer(const void *ptr)
```

Checks if a memory address falls within the valid, defined memory regions of the system.

Parametri:

Nome	Tipo	Descrizione
`ptr`	`const void *`	The memory address to validate. Can be any pointer; NULL is treated as invalid.

Ritorna:
Returns true if the address is within a valid memory range (e.g., RAM, Flash), false otherwise.

Note:

- This function is non-blocking and thread-safe, making it safe to call from both standard tasks and Interrupt Service Routines (ISRs). It relies on a compile-time memory map, so it is extremely fast and does not require any dynamic initialization. The most common mistake is assuming that a 'true' return means the memory is currently allocated or safe to write to; it only confirms the address is within the hardware's valid address space. Always validate pointers before dereferencing them if they originate from external sources or untrusted buffers.

Uso:

- Use this function as a defensive programming measure when receiving pointers from external interfaces, such as communication buffers or user-provided CLI arguments. It is typically used before passing a pointer to functions like ca_memcpy or when processing data structures that might contain corrupted addresses. By checking the pointer validity first, you prevent HardFault exceptions caused by accessing invalid memory regions or peripheral registers that are not mapped in the current context.

Esempio:

```
void process_external_data(void *data_ptr, u32 size) {
    // Check if the provided pointer is within valid system memory before usage
    if (ca_valid_pointer(data_ptr)) {
        // Safe to proceed with memory operations
        // Using ca_memcpy to copy data into a local buffer
        // ca_memcpy(local_buffer, data_ptr, size);
    } else {
        // Log an error or handle the invalid pointer case
        ca_printf("Error: Received invalid memory pointer!\n");
    }
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_valid_pointer in my caOS application?
A: You should use it as a defensive programming measure when receiving pointers from external interfaces, such as communication buffers or user-provided CLI arguments. It is essential to call it before dereferencing pointers or passing them to functions like ca_memcpy to prevent HardFault exceptions caused by accessing unmapped memory regions.

Code example conf:0.90 **Q:** Provide a minimal example of how to validate a pointer before processing it.
A: void process_buffer(void *ptr) { if (ca_valid_pointer(ptr)) { u32 *data = (u32 *)ptr; /* Safe to proceed */ } }

Parameters conf:1.00 **Q:** What are the constraints for the 'ptr' parameter in ca_valid_pointer?
A: The 'ptr' parameter can be any memory address. The function treats NULL as an invalid address. It checks if the provided address falls within the system's defined memory regions, such as RAM or Flash, based on the compile-time memory map.

Return conf:1.00 **Q:** Does a 'true' return value from ca_valid_pointer guarantee that the memory is safe to write to?

A: No. A 'true' return only confirms that the address is within the hardware's valid address space (e.g., RAM or Flash). It does not guarantee that the memory is currently allocated, initialized, or safe to write to. You must still ensure the memory is logically valid for your specific operation.

---

ca_vsnprintf

Header: caos/inc/utils.h:57
Source: caos/src/utils.c:214:371
GUARD: EN_UTILS

```
int ca_vsnprintf( char *buffer, u32 size, const char *format, va_list
    args)
```

Formats a variable argument list into a provided buffer, similar to standard vsnprintf.

Parametri:

Nome	Tipo	Descrizione
buffer	char *	Pointer to the destination character array where the formatted string will be stored.
size	u32	Maximum number of bytes to write, including the null terminator. Must be greater than 0.
format	const char *	Format string containing text and specifiers (e.g., %d, %s, %x).
args	va_list	The variable argument list initialized via va_start.

Ritorna:
Returns the number of characters written (excluding the null terminator) on success. Returns -1 if the buffer is NULL, format is NULL, or size is 0.

Note:

- This function is non-blocking and thread-safe as it operates entirely on the provided stack-allocated or static buffer. It is safe to call from an ISR because it does not perform any OS-level blocking operations or resource locking. The most common beginner mistake is providing an insufficient buffer size, which will result in truncation of the output string; always ensure the buffer is large enough for the expected output. Note that this implementation does not support floating-point specifiers (like %f).

Uso:

- This function is typically used to build formatted log messages or debug strings before sending them to a peripheral or a queue. It is almost always paired with va_start and va_end macros to handle the variable argument list. In a typical workflow, you initialize a buffer, call ca_vsnprintf to populate it, and then pass that buffer to a transmission function like ca_queue_static_send or ca_stdout_printf_n.

Esempio:

```
#include <stdarg.h>

void log_message(const char *fmt, ...) {
    char buffer[64];
    va_list args;

    va_start(args, fmt);
    // Format the string into the local buffer
```

```
    int len = ca_vsnprintf(buffer, sizeof(buffer), fmt, args);
    va_end(args);

    if (len > 0) {
        // Send the formatted string to the standard output system
        ca_stdout_printf_n("%s", buffer);
    }
}
```

Q&A — fine-tuning review *6/6 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I use ca_vsnprintf in an embedded system?

A: You should use ca_vsnprintf when you need to format a string with variable arguments in an embedded environment, especially in logging, debugging, or output functions. It's ideal for generating human-readable messages from structured data, such as printing sensor values or status information to a console or log buffer.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_vsnprintf with caOS types?

A:

```
u8 buffer[64];
va_list args;
va_start(args, format);
int len = ca_vsnprintf((char*)buffer, 64, "Value: %d, String: %s", args);
va_end(args);
```

Parameters conf:0.95 **Q:** What are the valid and invalid values for the 'size' parameter in ca_vsnprintf?

A: The 'size' parameter must be greater than 0. If size is 0, the function will return -1 indicating an error. The buffer must be large enough to hold the formatted string, including the null terminator. A size of 0 is invalid and causes failure.

Return conf:0.95 **Q:** What does a return value of -1 from ca_vsnprintf indicate?

A: A return value of -1 indicates an error occurred, typically due to one or more parameters being NULL or size being zero. It signifies that the function could not process the input due to invalid arguments.

Error/Edge case conf:0.90 **Q:** What are common mistakes developers make when using ca_vsnprintf?

A: Common mistakes include passing NULL to buffer or format, not calling va_start() or va_end() correctly, or using a buffer that is too small for the formatted output. These can lead to undefined behavior or incorrect string formatting.

Behavior conf:0.90 **Q:** Is ca_vsnprintf safe to call from an ISR or thread-safe?

A: ca_vsnprintf is not safe to call from an ISR because it may use functions not allowed in ISR context or take time to execute. However, it is thread-safe if the buffer and arguments are not modified by other threads during execution.

`ca_wait_queue_dequeue`

Header: `caos/inc/sync.h:382`
Source: `caos/src/sync.c:47:67`
GUARD: `EN_SYNC`

```
ca_thread_context_t* ca_wait_queue_dequeue(ca_wait_queue_t *wq)
```

Removes and returns the first task waiting in a specific wait queue.

Parametri:

Nome	Tipo	Descrizione
`wq`	`ca_wait_queue_t*`	Pointer to the wait queue structure. Must be initialized before use.

Ritorna:
Returns a pointer to the ca_thread_context_t of the dequeued task, or NULL if the queue is empty.

Note:

- This function is not thread-safe on its own; it must be called while holding the appropriate spinlock (e.g., using ca_spinlock_lock_irq) to prevent race conditions when multiple threads access the queue. It is non-blocking and can be called from an ISR, provided the caller has already acquired the necessary synchronization primitives. A common mistake is forgetting to lock the queue before calling this function, which leads to memory corruption or inconsistent queue states. Ensure the queue is properly initialized with ca_wait_queue_init before attempting to dequeue.

Uso:

- This function is typically used in synchronization primitives or resource managers to wake up tasks that are blocked waiting for an event. It is usually paired with ca_wait_queue_enqueue to add tasks to the list and ca_scheduler_wake_waiter to notify the scheduler that the task is ready to run again. In a typical workflow, you lock the queue, dequeue the task, unlock the queue, and then call the scheduler to perform the context switch or mark the task as ready.

Esempio:

```c
/* Example: Waking up a task waiting for a resource */
ca_wait_queue_t *my_queue = &resource_wait_list;

/* Always lock the queue before modifying it */
ca_spinlock_lock_irq(&my_queue->lock);

ca_thread_context_t *task = ca_wait_queue_dequeue(my_queue);

ca_spinlock_unlock_irq(&my_queue->lock);

if (task != NULL) {
    /* Notify the scheduler that the task is no longer waiting */
    ca_scheduler_wake_waiter(task);
} else {
    /* No tasks were waiting in the queue */
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use the ca_wait_queue_dequeue function in my caOS application?
A: You should use this function when implementing synchronization primitives or resource managers to wake up tasks that are blocked waiting for a specific event. It is typically used to remove a task from a wait queue so it can be marked as ready to run.

Code example conf:0.95 **Q:** Provide a minimal C example showing how to safely dequeue a task from a wait queue.
A: ca_thread_context_t *task; ca_spinlock_lock_irq(&my_lock); task = ca_wait_queue_dequeue(& ca_spinlock_unlock_irq(&my_lock); if (task != NULL) { /* proceed to wake the task */ }

Parameters conf:1.00 **Q:** What is the requirement for the wq parameter passed to ca_wait_queue_dequeue?
A: The wq parameter must be a pointer to a ca_wait_queue_t structure that has been properly initialized using ca_wait_queue_init before the dequeue operation is attempted.

Return conf:1.00 **Q:** What does the return value of ca_wait_queue_dequeue indicate?

A: It returns a pointer to the ca_thread_context_t of the task that was removed from the front of the queue. If the queue is empty, it returns NULL.

Error/Edge case conf:1.00 **Q:** What is a common mistake developers make when using ca_wait_queue_dequeu
A: A common mistake is forgetting to lock the queue using a spinlock before calling the function. This leads to race conditions, memory corruption, or inconsistent queue states when multiple threads access the queue simultaneously.

Behavior conf:1.00 **Q:** Is ca_wait_queue_dequeue thread-safe and can it be called from an ISR?
A: The function is not thread-safe on its own and requires the caller to hold a spinlock. It is non-blocking and can be called from an ISR, provided the caller has already acquired the necessary synchronization primitives.

`ca_wait_queue_enqueue`

Header: `caos/inc/sync.h:381`
Source: `caos/src/sync.c:27:42`
GUARD: `EN_SYNC`

```
void ca_wait_queue_enqueue(ca_wait_queue_t *wq, ca_thread_context_t *
    task)
```

Adds a task to a wait queue, marking it as waiting for a specific resource or event.

Parametri:

Nome	Tipo	Descrizione
`wq`	`ca_wait_queue_t *`	Pointer to the wait queue structure. Must be initialized before use.
`task`	`ca_thread_context_t *`	Pointer to the thread context of the task to be queued. Must be a valid, currently running task.

Ritorna:
void

Note:

- This function is not thread-safe on its own; it must be called while holding the appropriate spinlock associated with the wait queue to prevent race conditions. It is non-blocking, as it only performs pointer manipulation to link the task into the list. It is not intended to be called directly from an ISR, as it modifies task states that should be managed via the scheduler. The most common mistake is failing to acquire the associated spinlock before calling this function, which can lead to queue corruption or lost wake-up signals.

Uso:

- This function is used when a task needs to wait for an event (like data availability or a signal) and must be removed from the ready-to-run list. It is typically paired with ca_os_wait_prepare() or ca_os_sleep_prepare() to actually transition the task to a blocked state. The standard sequence is: lock the resource, check if the condition is met, call ca_wait_queue_enqueue() if not, prepare the wait, unlock the resource, and finally call ca_os_yield() to switch to another task.

Esempio:

```
/* Example: Waiting for a custom resource */
ca_wait_queue_t my_wq;
ca_spinlock_t my_lock;

void task_wait_for_resource(void) {
    ca_thread_context_t *ctx = ca_scheduler_get_current_context();
```

```

    ca_spinlock_lock_irq(&my_lock);

    /* Check if resource is available */
    if (resource_is_empty()) {
        /* Enqueue the current task to the wait queue */
        ca_wait_queue_enqueue(&my_wq, ctx);

        /* Prepare to block and release the lock */
        ca_os_wait_prepare();
        ca_spinlock_unlock_irq(&my_lock);

        /* Yield CPU to other tasks until signaled */
        ca_os_yield();
    } else {
        ca_spinlock_unlock_irq(&my_lock);
    }
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_wait_queue_enqueue in my caOS application?
A: You should use it when a task needs to wait for a specific event or resource, such as data availability or a signal, and needs to be removed from the ready-to-run list. It is typically used in a sequence where you lock a resource, check a condition, enqueue the task if the condition is not met, prepare the wait, unlock the resource, and finally call ca_os_yield().

Code example conf:0.90 **Q:** Provide a minimal C example showing how to use ca_wait_queue_enqueue.
A: ca_wait_queue_t my_wq; ca_thread_context_t *current_task = ca_os_get_current_task(); ca_spinlock_lock(&my_wq.lock); ca_wait_queue_enqueue(&my_wq, current_task); ca_os_wait_pr ca_spinlock_unlock(&my_wq.lock); ca_os_yield();

Parameters conf:1.00 **Q:** What are the requirements for the parameters passed to ca_wait_queue_enqueue?
A: The 'wq' parameter must be a pointer to a properly initialized ca_wait_queue_t structure. The 'task' parameter must be a pointer to a valid, currently running ca_thread_context_t.

Error/Edge case conf:1.00 **Q:** What is the most common mistake when calling ca_wait_queue_enqueue?
A: The most common mistake is failing to acquire the spinlock associated with the wait queue before calling the function. This can lead to queue corruption or lost wake-up signals.

Behavior conf:1.00 **Q:** Is ca_wait_queue_enqueue thread-safe and can I call it from an ISR?
A: The function is not thread-safe on its own; it requires the caller to hold the associated spinlock. It is non-blocking as it only performs pointer manipulation. It is not intended to be called directly from an ISR because it modifies task states that should be managed by the scheduler.

`ca_wait_queue_init`

Header: `caos/inc/sync.h:380`
Source: `caos/src/sync.c:19:22`
GUARD: `EN_SYNC`

```
void ca_wait_queue_init(ca_wait_queue_t *wq)
```

Initializes a wait queue structure by setting its head and tail pointers to NULL.

Parametri:

Nome	Tipo	Descrizione
wq	ca_wait_queue_t *	Pointer to the wait queue structure to be initialized. Must be a valid, non-NULL memory address.

Ritorna:
This function returns void and does not provide a status code.

Note:

- This function is a low-level initialization routine. It is not thread-safe on its own, so it must be called before the queue is accessed by any tasks or ISRs. It is typically used during the setup phase of a synchronization primitive (like a mutex, event group, or queue). A common mistake is to call this function on a queue that is currently being used by other tasks, which will immediately corrupt the queue state and likely cause a system crash or deadlock.

Uso:

- This function is used as part of the initialization sequence for higher-level synchronization objects. You will typically see it called inside an 'init' function (e.g., ca_event_init or ca_queue_static_init) alongside the initialization of spinlocks or other metadata. It prepares the internal list structure so that subsequent calls to enqueue or dequeue tasks can safely manage the wait list.

Esempio:

```c
/* Define a custom structure containing a wait queue */
typedef struct {
    ca_wait_queue_t wait_recv;
    ca_spinlock_t lock;
} my_sync_obj_t;

void my_sync_init(my_sync_obj_t *obj) {
    /* Initialize the wait queue before any task attempts to wait on it */
    ca_wait_queue_init(&obj->wait_recv);

    /* Initialize the spinlock to protect the queue during runtime */
    ca_spinlock_irq_init(&obj->lock);
}

/* Usage in a task context */
void task_function(void *arg) {
    my_sync_obj_t my_obj;
    my_sync_init(&my_obj);

    /* Now the queue is ready for use with ca_wait_queue_enqueue or similar */
}
```

Q&A — fine-tuning review

3/3 pairs · gemini-3.1-flash-lite

Usage conf:1.00 **Q:** When should I call ca_wait_queue_init and what happens if I call it on an active queue?
A: You should call this function during the setup phase of a synchronization primitive, before any tasks or ISRs access the queue. Calling it on a queue currently in use is a common mistake that will corrupt the queue state, likely resulting in a system crash or deadlock.

Code example conf:1.00 **Q:** Provide a minimal C example showing how to initialize a wait queue in caOS.
A: ca_wait_queue_t my_wq;
void init_my_sync_primitive(void) { ca_wait_queue_init(&my_wq); }

Parameters conf:1.00 **Q:** What are the requirements for the wq parameter in ca_wait_queue_init?
A: The wq parameter must be a pointer to a ca_wait_queue_t structure and must be a valid, non-NULL memory address. Passing a NULL pointer is invalid.

`ca_wait_queue_is_empty`

Header: `caos/inc/sync.h:384`
Source: `caos/src/sync.c:103:105`
GUARD: `EN_SYNC`

```
bool ca_wait_queue_is_empty(ca_wait_queue_t *wq)
```

Checks if a wait queue has no tasks currently waiting for a resource.

Parametri:

Nome	Tipo	Descrizione
`wq`	`ca_wait_queue_t *`	Pointer to the wait queue structure to check. Must be a valid, initialized queue handle.

Ritorna:
Returns true if the queue is empty (no tasks waiting), false otherwise.

Note:

- This function is non-blocking and executes in constant time, making it safe to call from both thread context and Interrupt Service Routines (ISRs). It does not perform any locking internally, so it assumes the caller has already ensured the queue structure is valid and not being concurrently modified in a way that would cause a race condition. The most common mistake is calling this on an uninitialized or NULL pointer, which will lead to a hard fault. Always ensure the queue has been initialized via the appropriate setup function before checking its status.

Uso:

- This function is typically used as a guard condition before attempting to dequeue tasks or when cleaning up synchronization primitives. In a typical lifecycle, you might use it to verify if any tasks are blocked on a resource (like a mutex or semaphore) before deciding to wake them up or destroy the object. It is frequently paired with ca_wait_queue_dequeue to drain a queue of waiting tasks during system shutdown or resource deallocation sequences.

Esempio:

```
/* Example: Checking if a custom synchronization object has waiters before
    cleanup */

void cleanup_resource(ca_wait_queue_t *wq) {
    /* Check if any tasks are still waiting on this resource */
    if (!ca_wait_queue_is_empty(wq)) {
        /* If not empty, we must wake up all waiting tasks before destroying */
        while (!ca_wait_queue_is_empty(wq)) {
            /* Dequeue the task and wake it up */
            ca_thread_context_t *task = ca_wait_queue_dequeue(wq);
            if (task != NULL) {
                ca_scheduler_wake_waiter(task);
            }
        }
    }
    /* Now it is safe to proceed with resource deallocation */
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_wait_queue_is_empty in my application logic?
A: You should use it as a guard condition before attempting to dequeue tasks or when cleaning up synchronization primitives. It is useful for verifying if any tasks are blocked on a resource, such as a mutex or semaphore, before deciding to wake them up or destroy the object.

Code example conf:0.90 **Q:** Provide a minimal C example showing how to check if a wait queue is empty.
A: ca_wait_queue_t *my_queue; if (ca_wait_queue_is_empty(my_queue)) { /* No tasks waiting, safe to proceed */ }

Parameters conf:1.00 **Q:** What are the requirements for the wq parameter in ca_wait_queue_is_empty?
A: The wq parameter must be a pointer to a valid, initialized ca_wait_queue_t structure. Passing an uninitialized or NULL pointer is a common mistake that will lead to a hard fault.

Return conf:1.00 **Q:** What do the return values of ca_wait_queue_is_empty indicate?
A: The function returns true if the queue is empty, meaning no tasks are currently waiting for the resource. It returns false if there are tasks waiting. Failure to provide a valid, initialized queue pointer will result in a hard fault rather than a specific return value.

`ca_wait_queue_remove`

Header: `caos/inc/sync.h:383`
Source: `caos/src/sync.c:72:101`
GUARD: `EN_SYNC`

```
1 bool ca_wait_queue_remove(ca_wait_queue_t *wq, ca_thread_context_t *task
     )
```

Removes a specific task from a wait queue, typically used when a task times out or is cancelled.

Parametri:

Nome	Tipo	Descrizione
`wq`	`ca_wait_queue_t *`	Pointer to the wait queue structure from which the task should be removed.
`task`	`ca_thread_context_t *`	Pointer to the task context to be removed. Must be a valid task currently waiting in the queue.

Ritorna:
Returns true if the task was found and successfully removed from the queue. Returns false if the task was not found in the specified queue.

Note:

- This function is not thread-safe by itself; it must be called while holding the appropriate synchronization primitive (usually a spinlock) that protects the wait queue. It is safe to call from an ISR if the spinlock is acquired using an IRQ-safe method (e.g., ca_spinlock_lock_irq). A common mistake is to attempt to remove a task from a queue without first locking the queue, which leads to race conditions where the queue pointers become corrupted. Always ensure the task is actually associated with the queue before calling this, as the function performs a linear search through the list.

Uso:

- This function is used in scenarios where a task is waiting for an event (like a notification or a message) but needs to be removed prematurely, such as when a timeout occurs or the task is terminated. It is typically paired with ca_spinlock_lock_irq to ensure atomic access to the queue structure. In the typical lifecycle, a task is added to a queue via an enqueue function, and this function is called to 'clean up' the queue if the task is no longer waiting, preventing the scheduler from attempting to wake a task that is no longer in a valid wait state.

Esempio:

```c
/* Example: Removing a task from a wait queue due to a timeout */
void handle_task_timeout(ca_wait_queue_t *wq, ca_thread_context_t *task,
    ca_spinlock_t *lock) {
    /* Acquire lock to safely modify the wait queue */
    ca_spinlock_lock_irq(lock);

    /* Attempt to remove the task from the queue */
    if (ca_wait_queue_remove(wq, task)) {
        /* Task was in the queue, now removed. Perform cleanup if necessary */
        /* e.g., task->result = TIMEOUT_ERROR; */
    }

    /* Release the lock to allow other tasks to access the queue */
    ca_spinlock_unlock_irq(lock);
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_wait_queue_remove in my caOS application?
A: You should use this function when a task needs to be removed from a wait queue prematurely, such as when a task times out while waiting for an event or when a task is terminated, to prevent the scheduler from attempting to wake a task that is no longer in a valid wait state.

Code example conf:1.00 **Q:** Provide a minimal C example showing how to safely remove a task from a wait queue.
A: ca_spinlock_lock_irq(&my_lock); bool removed = ca_wait_queue_remove(&my_wq, my_task_ptr); ca_spinlock_unlock_irq(&my_lock);

Parameters conf:1.00 **Q:** What are the requirements for the task parameter in ca_wait_queue_remove?
A: The task parameter must be a valid pointer to a ca_thread_context_t structure that is currently waiting in the specified wait queue. The function performs a linear search, so the task must be associated with the queue for the operation to succeed.

Return conf:1.00 **Q:** What do the return values of ca_wait_queue_remove indicate?
A: It returns true if the task was found and successfully removed from the queue. It returns false if the task was not found in the specified wait queue.

Error/Edge case conf:1.00 **Q:** What is a common mistake when calling ca_wait_queue_remove?
A: A common mistake is attempting to remove a task from a queue without first acquiring the appropriate synchronization primitive (like a spinlock). This leads to race conditions that can corrupt the queue pointers.

Behavior conf:1.00 **Q:** Is ca_wait_queue_remove thread-safe and can it be called from an ISR?
A: The function is not thread-safe by itself and must be called while holding the appropriate synchronization primitive. It is safe to call from an ISR provided that the spinlock is acquired using an IRQ-safe method, such as ca_spinlock_lock_irq.

---

ca_wdt_early_supported

Header: caos/drivers/inc/wdt.h:81
Source: caos/drivers/src/wdt.c:158:161
GUARD: EN_WDT

```
bool ca_wdt_early_supported(ca_wdt_type_t type)
```

Checks if the specified hardware watchdog timer supports an early warning interrupt.

Parametri:

Nome	Tipo	Descrizione
type	ca_wdt_type_t	The watchdog peripheral type (e.g., CA_WDT_TYPE_IWDG or CA_WDT_TYPE_WWDG). Must be a valid enum value defined by the hardware abstraction layer.

Ritorna:
Returns true if the hardware supports an early warning interrupt for the given type, false otherwise.

Note:

- This function is non-blocking and thread-safe, as it performs a simple read of the hardware capability configuration. It is safe to call from an ISR, though typically it is used during system initialization or task setup. The most common mistake is assuming that all watchdog types support early interrupts; failing to check this before calling ca_wdt_on_early can lead to silent failures or unexpected behavior depending on the underlying hardware implementation. Always verify support before attempting to register an early warning callback.

Uso:

- This function is used to perform a feature-check on the watchdog peripheral before attempting to configure advanced features like early warning interrupts. It is typically paired with ca_wdt_init to determine the configuration strategy: if the hardware supports early warnings, the application can register a callback using ca_wdt_on_early; if not, the application must rely on standard watchdog refreshing to prevent a system reset. This pattern ensures that the code remains portable across different STM32 microcontrollers that may have varying watchdog capabilities.

Esempio:

```
/* Check if the Independent Watchdog supports early warning before configuring
    it */
if (ca_wdt_early_supported(CA_WDT_TYPE_IWDG)) {
    /* Register a callback to be executed before the watchdog triggers a reset
    */
    ca_wdt_on_early(my_watchdog_callback, NULL);
    ca_wdt_init(1000, CA_WDT_MODE_INTERRUPT, CA_WDT_TYPE_IWDG);
} else {
    /* Fallback: standard watchdog initialization without early warning */
    ca_wdt_init(1000, CA_WDT_MODE_NORMAL, CA_WDT_TYPE_IWDG);
}

/* Periodically reload the watchdog to prevent system reset */
ca_wdt_reload();
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_wdt_early_supported and why is it necessary?
A: You should call this function during system initialization or task setup to verify if the hardware supports early warning interrupts. It is necessary because not all watchdog types support this feature; checking beforehand prevents silent failures or unexpected behavior when attempting to register a callback with ca_wdt_on_early.

Code example conf:0.90 **Q:** How do I check for early warning support in my code?
A: You can check the capability as follows: bool supported = ca_wdt_early_supported(CA_WDT_TY
if (supported) { /* proceed to register callback */ }

Parameters conf:1.00 **Q:** What does the ca_wdt_type_t parameter represent and what are valid values?
A: The parameter represents the watchdog peripheral type. Valid values are enum constants defined by the hardware abstraction layer, such as CA_WDT_TYPE_IWDG or CA_WDT_TYPE_WWDG.

Return conf:1.00 **Q:** What do the boolean return values indicate?
A: A return value of true indicates that the specific hardware watchdog peripheral supports an early warning interrupt. A return value of false indicates that the feature is not supported by that hardware, meaning the application must rely on standard watchdog refreshing instead.

`ca_wdt_enable`

Header: `caos/drivers/inc/wdt.h:40`
Source: `caos/drivers/src/wdt.c:87:108`
GUARD: `EN_WDT`

```
bool ca_wdt_enable(void)
```

Activates the hardware Watchdog Timer (WDT) to monitor system health and trigger a reset if the system hangs.

Ritorna:
Returns true if the WDT was successfully enabled or was already running. Returns false if the WDT was not initialized via ca_wdt_init or if the hardware-specific enable operation failed.

Note:

- This function is not thread-safe and should be called during the system initialization phase or from a controlled task context. It is not intended to be called from an Interrupt Service Routine (ISR). Before calling this, you must have successfully called ca_wdt_init; otherwise, the function will fail immediately. A common beginner mistake is to enable the watchdog without having a strategy to periodically call ca_wdt_reload, which will cause the system to enter an infinite reset loop.

Uso:

- The watchdog is typically enabled as the final step of the system boot sequence, once all critical peripherals and tasks are ready. It is paired with ca_wdt_init to configure the timeout and mode, and subsequently with ca_wdt_reload inside the main application loop or a dedicated monitor task. If the system fails to reach the reload point within the configured timeout, the hardware will force a system reset.

Esempio:

```
/* Initialize the WDT with a 1000ms timeout in reset mode */
if (ca_wdt_init(1000U, CA_WDT_MODE_RESET, CA_WDT_TYPE_IWDG)) {
    /* Enable the watchdog to start the countdown */
```

```
    if (ca_wdt_enable()) {
        /* System is now protected */
        while (1) {
            /* Periodically reload the watchdog to prevent reset */
            ca_wdt_reload();
            ca_os_delay_ms(500);
        }
    } else {
        /* Handle failure to enable hardware */
        ca_printf("WDT enable failed!\n");
    }
} else {
    /* Handle initialization failure */
    ca_printf("WDT init failed!\n");
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to call ca_wdt_enable in the caOS boot sequence?
A: It should be called as the final step of the system boot sequence, once all critical peripherals and tasks are ready for operation.

Code example conf:0.90 **Q:** Provide a minimal C example showing how to initialize and enable the watchdog using caOS types.
A: void system_init(void) { if (ca_wdt_init()) { bool status = ca_wdt_enable(); if (status) { /* WDT active */ } } }

Return conf:1.00 **Q:** What do the return values of ca_wdt_enable signify?
A: Returning true indicates the WDT was successfully enabled or was already running. Returning false indicates the WDT was not initialized via ca_wdt_init or the hardware-specific enable operation failed.

Error/Edge case conf:1.00 **Q:** What is a common mistake developers make when using ca_wdt_enable?
A: A common mistake is enabling the watchdog without implementing a strategy to periodically call ca_wdt_reload, which results in the system entering an infinite reset loop.

Behavior conf:1.00 **Q:** Is ca_wdt_enable thread-safe or safe to call from an ISR?
A: No, the function is not thread-safe and is not intended to be called from an Interrupt Service Routine (ISR). It should be called during system initialization or from a controlled task context.

`ca_wdt_get_timeout_ms`

Header: `caos/drivers/inc/wdt.h:42`
Source: `caos/drivers/src/wdt.c:120:134`
GUARD: `EN_WDT`

```
u32 ca_wdt_get_timeout_ms(void)
```

Retrieves the current Watchdog Timer (WDT) timeout period in milliseconds.

Ritorna:
Returns the timeout in milliseconds as a u32. Returns 0 if the WDT service has not been initialized or is in an invalid state.

Note:

- This function is thread-safe and non-blocking, making it safe to call from both tasks and Interrupt Service Routines (ISRs). It prioritizes reading the actual hardware register value

if the hardware driver supports it; otherwise, it falls back to the software-cached value. A common mistake is assuming the WDT is active before calling this; always ensure the WDT subsystem has been initialized via the appropriate init function, otherwise, you will receive a 0, which might be misinterpreted as an immediate timeout. It does not perform any hardware reset or interaction that would cause a system hang.

Uso:

- This function is typically used for system health monitoring or diagnostic logging to verify that the watchdog is configured with the expected timing parameters. It is often paired with ca_wdt_init during the system startup phase to confirm the configuration, and with ca_wdt_reload during normal operation to ensure the system remains responsive. Developers use it to dynamically adjust application logic or logging frequency based on the watchdog's strictness.

Esempio:

```c
/* Check if the watchdog is configured correctly before starting a critical task */
u32 current_timeout = ca_wdt_get_timeout_ms();

if (current_timeout == 0) {
    /* WDT not initialized, handle error or log it */
    ca_printf("Error: WDT not initialized!\n");
} else if (current_timeout < 1000) {
    /* If timeout is too short, log a warning */
    ca_printf("Warning: WDT timeout is very aggressive: %lu ms\n",
    current_timeout);
}

/* Periodically reload the watchdog to prevent system reset */
ca_wdt_reload();
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_wdt_get_timeout_ms in my application?
A: You should use this function for system health monitoring or diagnostic logging to verify the watchdog is configured with the expected timing parameters. It is typically used during system startup after ca_wdt_init to confirm configuration, or during normal operation to dynamically adjust application logic or logging frequency based on the watchdog's strictness.

Code example conf:1.00 **Q:** Provide a minimal C example showing how to check the WDT timeout.
A: u32 timeout = ca_wdt_get_timeout_ms(); if (timeout > 0) { /* Log or process the valid timeout value */ }

Return conf:1.00 **Q:** What do the return values of ca_wdt_get_timeout_ms represent?
A: The function returns the current WDT timeout period in milliseconds as a u32. A return value of 0 indicates that the WDT service has not been initialized or is currently in an invalid state.

Error/Edge case conf:1.00 **Q:** What is a common mistake when calling ca_wdt_get_timeout_ms?
A: A common mistake is assuming the WDT is active before calling the function. If the WDT subsystem has not been initialized via the appropriate init function, the call will return 0, which could be misinterpreted as an immediate timeout rather than an uninitialized state.

ca_wdt_hw_early_supported

Header: caos/drivers/inc/ca_glue_hw.h:369
GUARD: EN_CA_GLUE_HW

```
extern bool ca_wdt_hw_early_supported(ca_wdt_type_t type); /* whether
    that watchdog has an early warning interrupt here */
```

Checks if the specified watchdog hardware supports an early warning interrupt (EWI).

Parametri:

Nome	Tipo	Descrizione
type	ca_wdt_type_t	The watchdog type to query (e.g., CA_WDT_TYPE_WWDG or CA_WDT_TYPE_IWDG). Must be a valid enum value defined in the system headers.

Ritorna:
Returns true if the hardware supports an early warning interrupt for the given type, false otherwise.

Note:

- This function is non-blocking and thread-safe as it performs a simple hardware capability check. It is safe to call from an ISR because it does not interact with the OS scheduler or perform complex operations. The primary purpose is to allow drivers to validate configuration requests before attempting to initialize hardware that lacks specific features. A common mistake is assuming all watchdog types support early warnings on all STM32 families; always use this function to verify support before requesting an interrupt-based watchdog mode.

Uso:

- This function is typically used during the initialization phase of a watchdog service to validate user configuration. It is used to prevent the system from entering an invalid state where a user requests an interrupt-based watchdog on hardware that only supports a hard reset. It is usually paired with ca_wdt_init to ensure that the requested mode is compatible with the underlying MCU hardware capabilities.

Esempio:

```
/* Check if we can use an early warning interrupt for the IWDG */
ca_wdt_type_t my_wdt = CA_WDT_TYPE_IWDG;

if (ca_wdt_hw_early_supported(my_wdt)) {
    /* Hardware supports it, proceed with interrupt-based watchdog init */
    /* ca_wdt_init(timeout_ms, CA_WDT_MODE_INTERRUPT, my_wdt); */
} else {
    /* Fallback to standard reset mode or log an error */
    ca_printf("Early warning not supported on this hardware, using reset mode.\n
    ");
    /* ca_wdt_init(timeout_ms, CA_WDT_MODE_RESET, my_wdt); */
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_wdt_hw_early_supported and why is it important?
A: You should call this function during the initialization phase of your watchdog service. It is used to validate user configuration before calling ca_wdt_init, ensuring you do not attempt to enable an interrupt-based watchdog mode on hardware that only supports a hard reset.

Code example conf:1.00 **Q:** Provide a minimal C example showing how to check for early warning support before initialization.
A: if (ca_wdt_hw_early_supported(CA_WDT_TYPE_IWDG)) { /* Proceed with interrupt-based IWDG init */ } else { /* Fallback to standard reset mode */ }

Parameters conf:1.00 **Q:** What are the valid values for the 'type' parameter in ca_wdt_hw_early_supported?
A: The 'type' parameter must be a valid enum value of type ca_wdt_type_t, such as CA_WDT_TYPE_WWDG or CA_WDT_TYPE_IWDG, as defined in the system headers.

Return conf:1.00 **Q:** What does the boolean return value indicate, and can this function fail?
A: The function returns true if the hardware supports an early warning interrupt for the specified watchdog type, and false otherwise. It is non-blocking and thread-safe, so it does not fail in the traditional sense; it simply performs a hardware capability check.

HW differences conf:1.00 **Q:** How does hardware support for early warning interrupts differ across STM32 families in caOS?
A: On F1xx, F4xx, F7xx, and H7xx families, only the WWDG supports early interrupts, while the IWDG does not. Conversely, on the H5xx and U5xx families, the function returns true for both types, as these newer MCUs support both WWDG EWI and IWDG EWCR (early wakeup).

`ca_wdt_hw_enable`

Header: `caos/drivers/inc/ca_glue_hw.h:319`
GUARD: `EN_CA_GLUE_HW`

```
1 extern bool ca_wdt_hw_enable(void); /* avvia il watchdog; dopo questa
    chiamata va ricaricato periodicamente */
```

Activates the hardware Watchdog timer (IWDG or WWDG) to monitor system health.

Ritorna:
Returns true if the hardware watchdog was successfully enabled, false otherwise. A false return usually indicates that the watchdog hardware is not initialized or the system is in an invalid state.

Note:

- This function is non-blocking and performs a direct register write to the STM32 hardware watchdog peripheral. It is not thread-safe in the sense that it should be called during the system initialization phase before the scheduler starts or within a controlled startup sequence. It is generally not intended to be called from an ISR. The most common beginner mistake is calling this function without having previously configured the watchdog peripheral, which may lead to undefined behavior or immediate system resets depending on the hardware state. Once enabled, you must ensure that the watchdog is 'kicked' (reset) periodically using the appropriate OS functions to prevent a system reset.

Uso:

- This function is used during the system boot-up sequence to arm the safety mechanism that resets the MCU if the software hangs. It is typically paired with ca_os_thread_set_watchdog to register a task for monitoring and ca_os_thread_reset_watchdog to perform the periodic 'kick'. You should call this only after the hardware clocks and watchdog peripheral have been properly initialized by the system startup code.

Esempio:

```c
/* Initialize the system and enable the watchdog */
void system_startup(void) {
    /* Assume hardware watchdog peripheral is already configured */
    if (ca_wdt_hw_enable()) {
        /* Watchdog is now active; tasks must now reset it periodically */
        ca_os_thread_set_watchdog(1000); /* Set 1000ms timeout for current task */
    } else {
        /* Handle failure: log error or enter safe state */
        ca_printf("Failed to enable WDT!\n");
    }
}

/* Inside a periodic task */
void watchdog_task(void *arg) {
    while (1) {
        /* Reset the watchdog to prevent system reset */
        ca_os_thread_reset_watchdog();
        ca_os_delay_ms(500);
    }
}
```

Q&A — fine-tuning review

5/5 pairs · qwen3-coder:latest

Usage conf:0.95 **Q:** When should I call ca_wdt_hw_enable() in my caOS application?

A: You should call ca_wdt_hw_enable() once during system initialization, typically at the start of your main function or early in the boot sequence. This function enables the hardware watchdog (IWDG or WWDG) to prevent system crashes. It must be called only once, and after this call, you must periodically reload the watchdog using ca_wdt_reload() to avoid a system reset.

Code example conf:0.90 **Q:** Can you show a minimal example of how to use ca_wdt_hw_enable() in a caOS application?

A: bool success = ca_wdt_hw_enable(); if (success) { // Watchdog is enabled, now reload it periodically ca_wdt_reload(); } else { // Handle error, watchdog type may be invalid or already enabled }

Return conf:0.95 **Q:** What does ca_wdt_hw_enable() return, and what causes it to return false?

A: The function returns true if the watchdog was successfully enabled. It returns false if the watchdog type (IWDG or WWDG) is invalid or not properly configured, or if the watchdog is already enabled. The function does not perform a reset; it only enables the watchdog hardware.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_wdt_hw_enable()?

A: Common mistakes include calling ca_wdt_hw_enable() more than once, which may cause undefined behavior. It is also not thread-safe and must not be called from an ISR. Additionally, if the watchdog type is not correctly set in the global variable g_wdt_type, the function will return false. After enabling, you must ensure to periodically call ca_wdt_reload() to prevent system reset.

HW differences conf:0.80 **Q:** How does ca_wdt_hw_enable() differ between STM32F7xx and STM32H7xx MCUs?

A: The main difference is in the watchdog peripheral registers. On STM32F7xx, the function uses IWDG->KR and WWDG->CR registers, while on STM32H7xx, it uses IWDG1->KR and WWDG1->CR registers. This reflects the different watchdog peripherals used in these families, with H7xx having a dedicated IWDG1 instance.

---

ca_wdt_hw_get_timeout_ms

Header: caos/drivers/inc/ca_glue_hw.h:321
GUARD: EN_CA_GLUE_HW

```
extern u32 ca_wdt_hw_get_timeout_ms(void); /* ritorna il timeout
   effettivo in millisecondi dopo arrotondamento HW */
```

Returns the actual watchdog timeout in milliseconds after hardware-specific rounding.

Ritorna:
Returns a u32 value representing the effective timeout in milliseconds. If the hardware watchdog is not initialized or the value is not set, it may return 0.

Note:

- This function is non-blocking and thread-safe as it simply reads a global variable updated during hardware initialization. It is safe to call from an ISR, although it is rarely necessary to do so. The primary purpose is to provide the actual timing value used by the hardware peripheral, which may differ from the requested value due to clock prescalers and register limitations. A common mistake is assuming this function returns the 'requested' timeout rather than the 'actual' hardware-rounded timeout.

Uso:

- This function is typically used by the system watchdog management layer to verify the real-time constraints of the hardware. It is often called during system initialization or diagnostic routines to report the watchdog status. It is frequently paired with ca_os_thread_set_watchdog to ensure that the task's monitoring period is aligned with the actual hardware reset interval.

Esempio:

```
/* Check if the hardware watchdog is configured and report its actual timing */
u32 current_timeout = ca_wdt_hw_get_timeout_ms();

if (current_timeout > 0) {
    /* Log the actual hardware timeout value */
    ca_printf("Watchdog active with actual timeout: %lu ms\n", current_timeout);
} else {
    /* Handle case where watchdog is not initialized */
    ca_printf("Watchdog is not currently active or timeout is unknown.\n");
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_wdt_hw_get_timeout_ms() and why is it useful in a real embedded application?
A: You should call ca_wdt_hw_get_timeout_ms() when you need to verify or configure the current watchdog timeout value in your embedded application. It's useful for debugging, logging, or ensuring that the watchdog is set to an appropriate timeout value. Since it's non-blocking and can be called from any context, it's safe to use in both threads and ISRs.

Code example conf:0.90 **Q:** Can you provide a minimal C code example using ca_wdt_hw_get_timeout_ms() with caOS types?
A: u32 timeout = ca_wdt_hw_get_timeout_ms(); if (timeout == 0) { // Watchdog not initialized or invalid } else { // Use the timeout value }

Return conf:0.90 **Q:** What do the return values of ca_wdt_hw_get_timeout_ms() mean, and what causes it to return 0?

A: The function returns the current watchdog timeout in milliseconds as a u32. If it returns 0, it indicates that the watchdog is either not initialized or not available. This can happen if the watchdog hardware has not been properly configured or if the underlying hardware does not support the watchdog feature. The function does not modify the watchdog state and is not thread-safe.

HW differences conf:0.85 **Q:** Are there any differences in how ca_wdt_hw_get_timeout_ms() is implemented across different STM32 MCU families?
A: No, the implementation of ca_wdt_hw_get_timeout_ms() is consistent across all supported STM32 families (f1xx, f4xx, f7xx, h5xx, h7xx). Each family simply returns the global variable g_wdt_actual_timeout_ms, which is set during watchdog initialization. There are no differences in registers or peripherals used, as the function abstracts away hardware-specific details.

`ca_wdt_hw_init`

Header: `caos/drivers/inc/ca_glue_hw.h:330`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_wdt_hw_init(u32 timeout_ms, ca_wdt_mode_t mode,
    ca_wdt_type_t type); /* initializes the watchdog with the indicated
    timeout */
```

Initializes the hardware Watchdog timer (IWDG or WWDG) with a specific timeout and operational mode.

Parametri:

Nome	Tipo	Descrizione
`timeout_ms`	`u32`	The desired watchdog timeout period in milliseconds. Must be within the hardware-specific range supported by the MCU clock configuration.
`mode`	`ca_wdt_mode_t`	Operational mode: CA_WDT_MODE_INTERRUPT (triggers an interrupt before reset) or standard reset mode.
`type`	`ca_wdt_type_t`	The watchdog peripheral type: CA_WDT_TYPE_IWDG (Independent) or CA_WDT_TYPE_WWDG (Windowed).

Ritorna:
Returns true if the hardware was successfully configured, false if the timeout is out of range or the hardware failed to initialize.

Note:

- This function is not thread-safe and must be called during the system initialization phase before the RTOS scheduler starts. It is not intended to be called from an ISR. The function performs hardware-level register writes to configure clocks and prescalers; if the provided timeout is too short or too long for the LSI/APB clock, the calculation will fail and return false. A common mistake is calling this function after the watchdog has already been started or without ensuring the LSI clock is stable. Note that on some architectures like F1xx, the interrupt mode is not supported for IWDG.

Uso:

- This function is typically used in the board support package (BSP) or system startup sequence to ensure the system can recover from hangs. It is usually paired with ca_os_thread_set_watchdog to register a task for periodic kicking. Once initialized, the application must ensure the watchdog is serviced regularly to prevent an unwanted system reset. It should be called once during boot, as re-initialization may trigger an immediate reset or cause undefined behavior in the watchdog state machine.

Esempio:

```c
/* Initialize the Independent Watchdog for a 1000ms timeout */
#define WDT_TIMEOUT_MS 1000

void system_init(void) {
    /* Attempt to initialize the hardware watchdog */
    if (!ca_wdt_hw_init(WDT_TIMEOUT_MS, CA_WDT_MODE_INTERRUPT, CA_WDT_TYPE_IWDG)) {
        /* Log error using standard output if hardware init fails */
        ca_stdout_printf_n("Watchdog initialization failed!");
    } else {
        /* Register the current task to kick the watchdog periodically */
        ca_os_thread_set_watchdog(WDT_TIMEOUT_MS / 2);
    }
}
```

Q&A — fine-tuning review *7/7 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_wdt_hw_init() in my embedded application?
A: You should call ca_wdt_hw_init() during system initialization, typically right after the hardware is configured and before the main application logic begins. It's used to enable the watchdog timer with a specified timeout to protect against system hangs or unexpected behavior.

Code example conf:0.95 **Q:** Can you show a minimal example of how to use ca_wdt_hw_init()?
A: bool success = ca_wdt_hw_init(1000, CA_WDT_MODE_RESET, CA_WDT_TYPE_IWDG); if (!success) { /* handle error */ }

Parameters conf:0.95 **Q:** What are the valid values for the timeout_ms parameter and what happens if I pass an invalid value?
A: The timeout_ms parameter must be between 1 and 32767 milliseconds. Values less than or equal to 0 or greater than 32767 will cause the function to return false, indicating initialization failure.

Return conf:0.95 **Q:** What does it mean when ca_wdt_hw_init() returns false?
A: A return value of false indicates that the watchdog initialization failed. This can happen due to an invalid timeout_ms value, an unsupported watchdog type, or hardware-related issues such as the LSI clock not being enabled for IWDG or the APB1 clock not being enabled for WWDG.

Error/Edge case conf:0.90 **Q:** What are the most common mistakes when calling ca_wdt_hw_init()?
A: Common mistakes include passing a timeout_ms value outside the valid range (1-32767), using an unsupported watchdog type, not enabling the required clocks (LSI for IWDG or APB1 for WWDG) before calling the function, and calling the function from an ISR or in a multi-threaded context without proper synchronization.

Behavior conf:0.95 **Q:** Is ca_wdt_hw_init() safe to call from an ISR or in a multi-threaded environment?
A: No, ca_wdt_hw_init() is neither ISR-safe nor thread-safe. It modifies hardware registers and is a blocking function, making it unsuitable for use in ISRs or in contexts where multiple threads might call it concurrently without proper mutual exclusion.

HW differences conf:0.85 **Q:** How does ca_wdt_hw_init() differ between F1xx and H7xx MCUs in terms of hardware implementation?

A: On F1xx, IWDG prescaler uses 3 bits (0x07) and WWDG uses a fixed register layout without interrupt support. On H7xx, IWDG prescaler uses 4 bits (0x0F) and supports interrupt mode via WINR register, while WWDG uses RCC_APB3ENR for enabling and supports EWI bit for interrupt mode.

`ca_wdt_hw_is_enabled`

Header: `caos/drivers/inc/ca_glue_hw.h:322`
GUARD: `EN_CA_GLUE_HW`

```
extern bool ca_wdt_hw_is_enabled(void); /* ritorna true se il watchdog
      attivo e non pu  pi  essere disabilitato */
```

Checks if the hardware watchdog timer is currently active and locked.

Ritorna:
Returns true if the hardware watchdog is enabled and cannot be disabled, false otherwise.

Note:

- This function is non-blocking and thread-safe as it performs a direct register read or checks a global state variable. It is safe to call from an ISR, as it does not involve any OS primitives or blocking operations. The primary precondition is that the watchdog driver must have been initialized during system startup. A common mistake is assuming this function returns the status of the software-level watchdog management; it specifically targets the hardware peripheral state, which may differ if the software watchdog has been initialized but the hardware peripheral has not yet been triggered or configured.

Uso:

- This function is typically used by system monitoring tasks or diagnostic routines to verify if the safety watchdog is active before performing critical operations. It is often paired with other watchdog management functions to ensure the system is in a safe state. Developers use it to decide whether to proceed with a system reset or to report a fault if the watchdog is unexpectedly disabled in a production environment. It is part of the standard sequence: init -> check status -> reset/service.

Esempio:

```
/* Check if the hardware watchdog is active before proceeding with a critical
    task */
if (ca_wdt_hw_is_enabled()) {
    /* Watchdog is active, safe to proceed with critical operation */
    /* Perform critical system task here */
} else {
    /* Watchdog is not enabled, log a warning or trigger a system fault */
    ca_printf("Warning: Hardware watchdog is not enabled!\n");
}
```

Q&A — fine-tuning review *4/4 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_wdt_hw_is_enabled() in my embedded application?
A: You should call ca_wdt_hw_is_enabled() when you need to verify if the hardware watchdog is already active and cannot be disabled. This is particularly useful before attempting to reconfigure or disable the watchdog, as attempting to disable an active watchdog will fail. It's also helpful for debugging or logging purposes to confirm the watchdog state.

Code example conf:0.90 **Q:** Can you show a minimal C example using ca_wdt_hw_is_enabled() with caOS types?
A: bool check_wdt_status(void) { bool is_enabled = ca_wdt_hw_is_enabled(); if (is_enabled) { // Watchdog is active, cannot be disabled return true; } else { // Watchdog is not active return false; } }

Return conf:0.90 **Q:** What does it mean when ca_wdt_hw_is_enabled() returns true or false?
A: When ca_wdt_hw_is_enabled() returns true, it means the watchdog is active and cannot be disabled via software. When it returns false, the watchdog is either disabled or not supported on the current hardware. A return value of true typically occurs when the WWDG peripheral is enabled (e.g., WWDG->CR & WDGA bit is set) or when g_wdt_enabled is true for non-WWDG watchdog types.

HW differences conf:0.85 **Q:** How does ca_wdt_hw_is_enabled() differ between STM32F1xx and STM32H7xx families?
A: The function implementation is mostly consistent across families, but differs in register access. For STM32F1xx, STM32F4xx, and STM32F7xx, it checks WWDG->CR for WWDG type, while for STM32H7xx it checks WWDG1->CR. The non-WWDG watchdogs use a global variable g_wdt_enabled in all families. There are no significant differences in functionality or limitations between these families for this function.

`ca_wdt_hw_max_timeout_ms`

Header: `caos/drivers/inc/ca_glue_hw.h:366`
GUARD: `EN_CA_GLUE_HW`

```
1 extern u32 ca_wdt_hw_max_timeout_ms(ca_wdt_type_t type); /* longest
     timeout the family supports for that type, in ms */
```

Returns the maximum possible watchdog timeout in milliseconds for a specific hardware watchdog type.

Parametri:

Nome	Tipo	Descrizione
`type`	`ca_wdt_type_t`	The watchdog type to query, typically CA_WDT_TYPE_IWDG (Independent) or CA_WDT_TYPE_WWDG (Windowed).

Ritorna:
A 32-bit unsigned integer representing the maximum timeout in milliseconds supported by the current hardware configuration. This value is calculated based on the hardware clock frequency and the maximum prescaler settings.

Note:

- This function is non-blocking and thread-safe as it performs a simple mathematical calculation based on hardware constants. It is safe to call from an ISR because it does not interact with OS primitives or perform I/O. The primary purpose is to allow the application to validate user-provided timeout values before attempting to initialize the watchdog. A common mistake is assuming this value is constant across different STM32 families; always use this function to query the limit rather than hardcoding a value, as the underlying clock frequencies (LSI or PCLK) vary significantly between chips.

Uso:

- This function is typically used during the system initialization phase or when reconfiguring the watchdog. It is almost always paired with ca_wdt_init to perform input validation.

By checking the requested timeout against this maximum, you prevent the driver from attempting to set a watchdog period that exceeds the hardware's physical capability, which would otherwise lead to an invalid configuration or a watchdog reset loop.

Esempio:

```c
/* Check if the requested watchdog timeout is within hardware limits before
    initializing */
#include "caos.h"

void setup_watchdog(u32 requested_ms) {
    ca_wdt_type_t type = CA_WDT_TYPE_IWDG;
    u32 max_ms = ca_wdt_hw_max_timeout_ms(type);

    if (requested_ms > max_ms) {
        /* Log error: requested timeout exceeds hardware capacity */
        ca_printf("Watchdog timeout too long, capping to %lu ms\n", max_ms);
        requested_ms = max_ms;
    }

    if (!ca_wdt_init(requested_ms, CA_WDT_MODE_NORMAL, type)) {
        /* Handle initialization failure */
        ca_printf("Failed to initialize watchdog\n");
    }
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_wdt_hw_max_timeout_ms in my application?
A: You should call this function during system initialization or when reconfiguring the watchdog. It is primarily used to validate user-provided timeout values before calling ca_wdt_init to ensure the requested duration does not exceed the hardware's physical capabilities, which prevents configuration errors or reset loops.

Code example conf:1.00 **Q:** How do I use ca_wdt_hw_max_timeout_ms to validate a timeout before initialization?
A: You can compare your desired timeout against the function's return value: u32 max_ms = ca_wdt_hw_max_timeout_ms(CA_WDT_TYPE_IWDG); if (requested_ms <= max_ms) { ca_wdt_init(CA_WDT_TYPE_IWDG, requested_ms); }

Parameters conf:1.00 **Q:** What are the valid inputs for the type parameter in ca_wdt_hw_max_timeout_ms?
A: The type parameter accepts ca_wdt_type_t values, specifically CA_WDT_TYPE_IWDG for the Independent Watchdog and CA_WDT_TYPE_WWDG for the Windowed Watchdog.

Return conf:1.00 **Q:** What does the return value of ca_wdt_hw_max_timeout_ms represent and can it fail?
A: It returns a 32-bit unsigned integer representing the maximum supported watchdog timeout in milliseconds. The function is non-blocking and thread-safe, performing a mathematical calculation based on hardware constants; it does not interact with OS primitives, so it does not 'fail' in the traditional sense.

HW differences conf:1.00 **Q:** Why is it important to use ca_wdt_hw_max_timeout_ms instead of a hardcoded constant across different STM32 families?
A: The maximum timeout depends on hardware-specific clock frequencies like LSI (for IWDG) or PCLK1 (for WWDG), which vary significantly between STM32 families. The function calculates the limit dynamically based on family-specific constants like IWDG_LSI_KHZ, PCLK1_FREQ_HZ, and prescaler registers (IWDG_PR_N, WWDG_PR_N).

ca_wdt_hw_on_early

Header: caos/drivers/inc/ca_glue_hw.h:368
GUARD: EN_CA_GLUE_HW

```
extern void ca_wdt_hw_on_early(ca_wdt_early_cb_t cb, void *arg); /*
    callback of the early warning interrupt (ISR context) */
```

Registers a callback function to be executed when the Watchdog (WDT) early warning interrupt triggers.

Parametri:

Nome	Tipo	Descrizione
cb	ca_wdt_early_cb_t	Pointer to the function to execute. Must be a valid function pointer or NULL to disable.
arg	void *	User-defined pointer passed to the callback function when it executes.

Ritorna:
void

Note:

- This function is designed to be called from a thread context to configure the hardware-level interrupt handler. The callback itself will execute in an ISR (Interrupt Service Routine) context, meaning it must be extremely fast, non-blocking, and must not use any OS primitives that could cause a context switch (like mutexes or blocking queues). A common mistake is performing heavy operations like logging or complex calculations inside the callback, which can lead to system instability or missed deadlines. The implementation uses volatile variables to ensure the callback and its argument are updated atomically relative to the hardware interrupt trigger.

Uso:

- This function is used to implement 'pre-watchdog' logic, such as saving critical system state or performing a clean shutdown before the hardware watchdog forces a hard reset. It is typically called during the system initialization phase after the watchdog has been initialized with ca_wdt_init. It pairs with the watchdog driver's internal ISR, which will invoke the registered callback when the WDT counter reaches the early warning threshold.

Esempio:

```
/* Define the callback function to be executed in ISR context */
void my_wdt_warning_handler(void *arg) {
    /* Perform minimal, non-blocking emergency actions */
    /* Example: signal a task to save state or log a critical error */
    /* Note: Do not use blocking functions here */
}

void system_setup(void) {
    /* Initialize the watchdog first */
    /* ca_wdt_init(timeout_ms); */

    /* Register the early warning callback */
    ca_wdt_on_early(my_wdt_warning_handler, NULL);

    /* Enable the watchdog */
    /* ca_wdt_enable(); */
}
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_wdt_hw_on_early and what is its primary purpose?
A: You should call this function during the system initialization phase, typically after ca_wdt_init. Its purpose is to register a callback that executes when the watchdog early warning interrupt triggers, allowing you to perform critical tasks like saving system state or executing a clean shutdown before the hardware watchdog forces a hard reset.

Code example conf:1.00 **Q:** How do I register an early warning callback in caOS?
A: You pass a function pointer and an optional argument to ca_wdt_hw_on_early. Example: void my_callback(void *arg) { /* fast, non-blocking code */ } ... ca_wdt_hw_on_early(my_call NULL);

Parameters conf:1.00 **Q:** What are the constraints on the 'cb' parameter in ca_wdt_hw_on_early?
A: The 'cb' parameter must be a valid function pointer of type ca_wdt_early_cb_t or NULL. If you pass NULL, the early warning interrupt functionality is disabled. Because the callback executes in an ISR context, it must be extremely fast, non-blocking, and must not use OS primitives like mutexes or blocking queues.

HW differences conf:1.00 **Q:** Are there differences in how ca_wdt_hw_on_early is implemented across the supported STM32 families?
A: Based on the provided documentation, the implementation of ca_wdt_hw_on_early is identical across the f1xx, f4xx, f7xx, h5xx, h7xx, and u5xx families. Each implementation uses a static volatile pointer to ensure the callback and its argument are updated atomically relative to the hardware interrupt trigger.

`ca_wdt_hw_reload`

Header: `caos/drivers/inc/ca_glue_hw.h:320`
GUARD: `EN_CA_GLUE_HW`

```
1 extern void ca_wdt_hw_reload(void); /* ricarica il watchdog (kick) per
     prevenire il reset */
```

Resets the hardware watchdog timer to prevent a system reset.

Ritorna:
void

Note:

- This function is non-blocking and executes immediately by writing to the hardware registers of the watchdog peripheral. It is thread-safe because the underlying register access is atomic, meaning it can be called from multiple threads or even from an ISR without requiring additional mutexes. The primary precondition is that the watchdog must have been previously initialized by the system startup code. A common beginner mistake is to call this function too infrequently, causing the watchdog to expire and trigger a system reset, or to call it inside a tight loop that prevents other tasks from running, effectively starving the system.

Uso:

- This function is used to 'kick' or 'pet' the watchdog timer, signaling to the hardware that the software is still running correctly. It is typically called periodically within a main loop or a dedicated high-priority task to ensure the system has not hung. In a robust design, you should pair this with ca_os_thread_set_watchdog to monitor specific task health, ensuring that if a task stops responding, the watchdog is no longer reloaded and the system safely reboots.

Esempio:

```c
/* Example: Periodically reloading the watchdog in a task loop */
void watchdog_task(void *pvParameters) {
    while (1) {
        /* Perform critical system checks here */

        /* Reload the hardware watchdog to prevent reset */
        ca_wdt_hw_reload();

        /* Sleep for a duration shorter than the watchdog timeout */
        ca_os_delay_ms(500);
    }
}
```

Q&A — fine-tuning review *5/5 pairs · qwen3-coder:latest*

Usage conf:0.95 **Q:** When should I call ca_wdt_hw_reload() in my embedded application?
A: You should call ca_wdt_hw_reload() periodically to prevent a system reset caused by the watchdog timer. It is typically called in main loops or periodic tasks to 'kick' or refresh the watchdog, ensuring that the system does not reset due to inactivity. This function must only be called after the watchdog has been successfully initialized.

Code example conf:0.90 **Q:** Can you provide a minimal C example using caOS types that demonstrates how to use ca_wdt_hw_reload()?
A:

```c
u32 i;
for (i = 0; i < 1000000; i++) {
    if (i % 100000 == 0) {
        ca_wdt_hw_reload();
    }
}
```

Return conf:0.90 **Q:** What does ca_wdt_hw_reload() return, and what causes it to fail?
A: The function does not return any value. It cannot fail in a traditional sense, but it may not have any effect if the watchdog has not been initialized. The function is designed to be a no-op if the watchdog is not initialized, rather than causing an error.

Error/Edge case conf:0.85 **Q:** What are common mistakes or edge cases when using ca_wdt_hw_reload()?
A: A common mistake is calling ca_wdt_hw_reload() before initializing the watchdog, which will result in no action being taken. Another issue is calling it from an ISR when atomicity is required, as the function is not thread-safe and modifies hardware registers directly. Additionally, if the watchdog is not properly reloaded in time, the system will reset.

HW differences conf:0.80 **Q:** How does ca_wdt_hw_reload() differ between STM32 F7xx and H7xx microcontrollers?
A: The main difference lies in the watchdog peripheral registers used. For F7xx, both IWDG and WWDG registers are accessed directly from the standard IWDG and WWDG peripherals. For H7xx, IWDG1 and WWDG1 are used, indicating a different peripheral mapping or dual watchdog configuration in the H7xx family compared to F7xx.

`ca_wdt_hw_reset_cause`

Header: `caos/drivers/inc/ca_glue_hw.h:367`
GUARD: `EN_CA_GLUE_HW`

```c
extern ca_wdt_reset_t ca_wdt_hw_reset_cause(bool clear); /* watchdog
   that caused the last reset; clear = clear ALL the RCC reset flags */
```

Retrieves the hardware reset source to determine if the system was restarted by a Watchdog timer.

Parametri:

Nome	Tipo	Descrizione
`clear`	`bool`	If true, clears the internal RCC reset flags after reading them. If false, the flags remain set for subsequent reads.

Ritorna:
Returns a ca_wdt_reset_t enum value: CA_WDT_RESET_IWDG if the Independent Watchdog caused the reset, CA_WDT_RESET_WWDG if the Window Watchdog caused it, or CA_WDT_RESET_NONE if the reset was caused by another source (e.g., power-on, software reset, or pin reset).

Note:

- This function is non-blocking and thread-safe as it performs a direct read of the hardware RCC registers. It is safe to call from an ISR, although it is typically called during system initialization before the application starts. The most common beginner mistake is to call this function with 'clear = true' in multiple places; if you clear the flags early in the boot process, subsequent calls will return CA_WDT_RESET_NONE, causing you to lose the diagnostic information about why the system crashed.

Uso:

- This function is used during the early stages of system startup to perform post-mortem diagnostics. Developers typically call it once at the beginning of the application to log the reset cause to a persistent storage or a debug console. It is often paired with other system initialization functions to decide whether to enter a recovery mode or a safe state if a watchdog reset is detected.

Esempio:

```c
/* Check why the system restarted */
ca_wdt_reset_t cause = ca_wdt_reset_cause(true);

if (cause == CA_WDT_RESET_IWDG) {
    /* Log the IWDG reset event to the system console */
    ca_printf("System reset by Independent Watchdog!\n");
} else if (cause == CA_WDT_RESET_WWDG) {
    /* Log the WWDG reset event */
    ca_printf("System reset by Window Watchdog!\n");
} else {
    /* Normal power-on or other reset source */
    ca_printf("System started normally.\n");
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the best time to call ca_wdt_hw_reset_cause in my application?
A: It is typically called during the early stages of system startup, before the main application logic begins. This allows you to log the reset cause to persistent storage or a debug console to perform post-mortem diagnostics, or to decide whether to enter a recovery mode.

Code example conf:1.00 **Q:** How do I check the reset cause and store it in a variable using caOS types?
A: You should call the function once and store the result. Example: ca_wdt_reset_t cause = ca_wdt_hw_reset_cause(false); if (cause == CA_WDT_RESET_IWDG) { /* handle IWDG reset */ }

Parameters conf:1.00 **Q:** What happens if I set the 'clear' parameter to true in multiple places in my code?

A: This is a common beginner mistake. If you set 'clear' to true, the internal RCC reset flags are cleared. Subsequent calls to the function will return CA_WDT_RESET_NONE, causing you to lose the diagnostic information about the actual cause of the reset.

Return conf:1.00 **Q:** What do the different return values of ca_wdt_hw_reset_cause signify?
A: The function returns CA_WDT_RESET_IWDG if the Independent Watchdog caused the reset, CA_WDT_RESET_WWDG if the Window Watchdog caused it, and CA_WDT_RESET_N if the reset was caused by another source, such as a power-on, software reset, or pin reset.

HW differences conf:1.00 **Q:** Are there differences in how ca_wdt_hw_reset_cause interacts with hardware registers across different STM32 families?
A: Yes. Most families (f1xx, f4xx, f7xx, u5xx) read from the RCC->CSR register and use the RCC_CSR_RMVF flag to clear it. However, the h5xx family reads from the RCC->RSR register using RCC_RSR_RMVF, and the h7xx family also uses the RCC->RSR register but specifically checks for IWDG1RSTF and WWDG1RSTF flags.

`ca_wdt_init`

Header: `caos/drivers/inc/wdt.h:39`
Source: `caos/drivers/src/wdt.c:54:85`
GUARD: `EN_WDT`

```
1 bool ca_wdt_init(u32 timeout_ms, ca_wdt_mode_t mode, ca_wdt_type_t type)
```

Initializes the hardware Watchdog Timer (WDT) with a specific timeout and operational mode.

Parametri:

Nome	Tipo	Descrizione
`timeout_ms`	`u32`	Watchdog timeout duration in milliseconds. Must be between 50 and 10000 inclusive.
`mode`	`ca_wdt_mode_t`	Operational mode (e.g., CA_WDT_MODE_RESET to trigger a system reset on timeout).
`type`	`ca_wdt_type_t`	Hardware type (e.g., CA_WDT_TYPE_IWDG for Independent Watchdog).

Ritorna:
Returns true if the WDT was successfully initialized and configured. Returns false if parameters are out of range, if the WDT is already enabled, or if the hardware initialization fails.

Note:

- This function is not thread-safe and should be called during system initialization before the scheduler starts or in a controlled environment. It is not intended to be called from an ISR. The most common beginner mistake is attempting to re-initialize the WDT without checking if it is already enabled, or providing a timeout value outside the allowed 50ms-10000ms range. Once initialized, the WDT must be started using ca_wdt_enable() and periodically serviced using ca_wdt_reload() to prevent a system reset.

Uso:

- This function is the mandatory first step in the WDT lifecycle. It is typically called during the system boot sequence or within a task responsible for hardware configuration. It is always paired with ca_wdt_enable() to activate the timer. In advanced scenarios, it is used during emergency shutdown routines to force a hardware reset if the system becomes unresponsive.

Esempio:

```c
/* Initialize WDT for a 2-second timeout in reset mode */
#define WDT_TIMEOUT_MS 2000U

if (ca_wdt_init(WDT_TIMEOUT_MS, CA_WDT_MODE_RESET, CA_WDT_TYPE_IWDG)) {
    /* Successfully configured, now activate the hardware */
    if (ca_wdt_enable()) {
        /* WDT is now running; must call ca_wdt_reload() periodically */
    } else {
        /* Handle hardware enable failure */
    }
} else {
    /* Handle initialization failure (e.g., invalid timeout) */
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_wdt_init in my caOS project?
A: It should be called during the system boot sequence or within a task responsible for hardware configuration, ideally before the scheduler starts.

Code example conf:0.90 **Q:** How do I initialize the watchdog timer using caOS types?
A: u32 timeout = 1000; if (ca_wdt_init(timeout, CA_WDT_MODE_RESET, CA_WDT_TYPE_IW
{ ca_wdt_enable(); }

Parameters conf:1.00 **Q:** What are the constraints for the timeout_ms parameter?
A: The timeout_ms parameter must be a u32 value between 50 and 10000 inclusive.

Return conf:1.00 **Q:** What does a false return value from ca_wdt_init indicate?
A: It indicates that the initialization failed, which occurs if parameters are out of range, the WDT is already enabled, or the hardware initialization fails.

Error/Edge case conf:1.00 **Q:** What is a common mistake developers make when using ca_wdt_init?
A: The most common mistake is attempting to re-initialize the WDT without checking if it is already enabled, or providing a timeout value outside the allowed 50ms-10000ms range.

Behavior conf:1.00 **Q:** Is ca_wdt_init thread-safe or safe to call from an ISR?
A: No, this function is not thread-safe and is not intended to be called from an ISR. It should be called in a controlled environment or during system initialization.

`ca_wdt_is_enabled`

Header: `caos/drivers/inc/wdt.h:43`
Source: `caos/drivers/src/wdt.c:136:145`
GUARD: `EN_WDT`

```c
bool ca_wdt_is_enabled(void)
```

Checks if the system watchdog timer is currently active.

Ritorna:
Returns true if the watchdog is enabled (either in hardware or software state), false otherwise.

Note:

- This function is non-blocking and thread-safe, as it performs a simple read of the internal watchdog state. It can be safely called from an ISR, as it does not involve any complex synchronization or blocking mechanisms. The function relies on the internal state initialized during the system boot sequence; if the watchdog was never initialized, it will return the default software state. A common mistake is assuming that this function resets

the watchdog timer; it only reports the status, while ca_os_thread_reset_watchdog or ca_wdt_reload must be used to prevent a system reset.

Uso:

- This function is typically used in diagnostic routines or before performing critical operations where you need to verify if the system is being monitored by the hardware watchdog. It is often paired with ca_wdt_init during the system startup phase to verify that the watchdog configuration was applied successfully. In a typical application, you might check this status before entering a long-running task to ensure the watchdog is active, or use it to decide whether to perform a manual reload.

Esempio:

```c
/* Check if the watchdog is active before starting a critical task */
if (ca_wdt_is_enabled()) {
    /* Watchdog is active, proceed with the task */
    /* Perform critical operation here */

    /* Reset the watchdog to prevent a system reset */
    ca_os_thread_reset_watchdog();
} else {
    /* Watchdog is disabled, log a warning or re-initialize */
    ca_printf("Warning: Watchdog is not enabled!\n");
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_wdt_is_enabled in my application logic?
A: You should use it during diagnostic routines or before critical operations to verify if the system is being monitored. It is also useful during startup to confirm that the configuration applied by ca_wdt_init was successful, or to decide if a manual reload is necessary before starting a long-running task.

Code example conf:0.90 **Q:** How do I check the watchdog status and perform a reload if it is active?
A: You can check the status using the boolean return value and then call the reload function:
if (ca_wdt_is_enabled()) { ca_wdt_reload(); }

Return conf:0.95 **Q:** What do the return values of ca_wdt_is_enabled indicate, and can it fail?
A: The function returns true if the watchdog is enabled (hardware or software state) and false otherwise. It does not fail in the traditional sense as it is non-blocking and thread-safe; however, if the watchdog was never initialized, it will return the default software state.

`ca_wdt_max_timeout_ms`

Header: `caos/drivers/inc/wdt.h:66`
Source: `caos/drivers/src/wdt.c:143:146`
GUARD: `EN_WDT`

```c
u32 ca_wdt_max_timeout_ms(ca_wdt_type_t type)
```

Returns the maximum supported timeout duration in milliseconds for a specific watchdog hardware type.

Parametri:

Nome	Tipo	Descrizione
type	ca_wdt_type_t	The watchdog hardware identifier (e.g., CA_WDT_TYPE_IWDG for Independent Watchdog or CA_WDT_TYPE_WWDG for Window Watchdog).

Ritorna:
A u32 value representing the maximum timeout in milliseconds. If the hardware type is invalid or unsupported, the function typically returns 0 or a default fallback value defined by the driver.

Note:

- This function is non-blocking and thread-safe, as it performs a simple read of the hardware capability configuration. It is safe to call from an ISR, although it is rarely necessary to do so. The primary purpose is to allow software to dynamically query hardware limits rather than hardcoding magic numbers. A common beginner mistake is assuming that the returned value is the current timeout; this function only returns the theoretical maximum limit, not the currently configured watchdog period.

Uso:

- This function is typically used during the initialization phase of a task or system service to validate user-provided watchdog timeout settings before calling ca_wdt_init. By comparing the requested timeout against the result of ca_wdt_max_timeout_ms, developers can implement robust error handling to reject invalid configurations that exceed hardware capabilities. It is often paired with ca_wdt_init to ensure the system remains within safe operational bounds.

Esempio:

```c
/* Example: Validating watchdog configuration before initialization */
#include "ca_os.h"

void task_watchdog_setup(void) {
    const ca_wdt_type_t wdt_type = CA_WDT_TYPE_IWDG;
    const u32 requested_ms = 5000U;

    /* Query the hardware limit for the IWDG */
    u32 max_limit = ca_wdt_max_timeout_ms(wdt_type);

    if (requested_ms > 0U && requested_ms <= max_limit) {
        /* Configuration is within hardware limits, proceed to init */
        bool success = ca_wdt_init(requested_ms, CA_WDT_MODE_RESET, wdt_type);
        if (success) {
            ca_stdout_put_n("Watchdog initialized successfully");
        }
    } else {
        /* Log error: requested timeout is out of hardware range */
        ca_stdout_put_n("Error: Requested WDT timeout exceeds hardware limit");
    }
}
```

Q&A — fine-tuning review

4/4 pairs · gemini-3.1-flash-lite

Usage conf:1.00 **Q:** When should I use ca_wdt_max_timeout_ms in my application?
A: You should use this function during the initialization phase of a task or system service. It is intended to validate user-provided watchdog timeout settings against hardware limits before calling ca_wdt_init, allowing you to reject invalid configurations dynamically instead of using hardcoded magic numbers.

Code example conf:0.90 **Q:** How do I check if a requested timeout is valid using caOS types?

A: You can compare the requested duration against the hardware limit like this: u32 max_ms = ca_wdt_max_timeout_ms(CA_WDT_TYPE_IWDG); if (requested_ms > max_ms) { /* handle error */ }.

Parameters conf:1.00 **Q:** What does the ca_wdt_type_t parameter represent?

A: The parameter represents the watchdog hardware identifier. Valid examples include CA_WDT_TYPE_IWDG for the Independent Watchdog and CA_WDT_TYPE_WWDG for the Window Watchdog.

Return conf:1.00 **Q:** What does a return value of 0 indicate from ca_wdt_max_timeout_ms?

A: A return value of 0 typically indicates that the provided hardware type is invalid or unsupported by the driver. It is important to note that this function returns the theoretical maximum limit of the hardware, not the currently configured watchdog period.

`ca_wdt_on_early`

Header: `caos/drivers/inc/wdt.h:79`
Source: `caos/drivers/src/wdt.c:153:156`
GUARD: `EN_WDT`

```
void ca_wdt_on_early(ca_wdt_early_cb_t cb, void *arg)
```

Registers a callback function to be executed just before the Watchdog timer triggers a system reset.

Parametri:

Nome	Tipo	Descrizione
`cb`	`ca_wdt_early_cb_t`	A pointer to the function to be called. Must be a valid function pointer or NULL to disable the callback.
`arg`	`void *`	A user-defined pointer passed to the callback function. Can be NULL if no context is needed.

Ritorna:
void

Note:

- This function is non-blocking and is typically called during system initialization. The callback function executes in an interrupt context (ISR), meaning it must be extremely fast, must not perform blocking operations (like waiting for semaphores or I/O), and should avoid complex memory allocations. A common beginner mistake is performing heavy logging or complex logic inside this callback, which can lead to system instability or missed reset timing. Ensure the watchdog hardware actually supports early warning interrupts before relying on this feature.

Uso:

- This function is used to perform 'last-gasp' operations, such as saving critical system state to backup registers or logging a final error code to non-volatile memory before the hardware resets. It is typically called before ca_wdt_init and ca_wdt_enable. It is often paired with ca_bkp_write to store diagnostic information that can be retrieved after the system reboots to identify why the watchdog was triggered.

Esempio:

```
/* Define the early warning callback */
void my_early_warning_cb(void *arg) {
    /* Save the error type to backup registers before reset */
```

```
    ca_bkp_write(0U, (uint32_t)arg);
}

void setup_watchdog(void) {
    /* Register the callback to run before the WDT resets the system */
    ca_wdt_on_early(my_early_warning_cb, (void *)0xDEADBEEF);

    /* Initialize WDT with interrupt mode to allow the callback to run */
    if (ca_wdt_init(1000U, CA_WDT_MODE_INTERRUPT, 0U)) {
        ca_wdt_enable();
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_wdt_on_early and what is its primary purpose?
A: You should call ca_wdt_on_early during system initialization, typically before calling ca_wdt_init and ca_wdt_enable. Its purpose is to register a 'last-gasp' callback that executes just before a system reset, allowing you to perform critical operations like saving system state to backup registers or logging error codes.

Code example conf:0.90 **Q:** How do I implement a minimal callback for ca_wdt_on_early using caOS types?
A: The callback must be fast and non-blocking as it runs in an ISR. Example: void my_callback(void *arg) { u32 *err_code = (u32 *)arg; ca_bkp_write(0, *err_code); } ... ca_wdt_on_early(my_callback, &my_error_var);

Parameters conf:1.00 **Q:** What are the requirements for the 'cb' parameter in ca_wdt_on_early?
A: The 'cb' parameter must be a valid function pointer of type ca_wdt_early_cb_t, or it can be set to NULL to disable the callback. Because it executes in an interrupt context, the function must be extremely fast and avoid blocking operations or complex memory allocations.

`ca_wdt_reload`

Header: `caos/drivers/inc/wdt.h:41`
Source: `caos/drivers/src/wdt.c:110:118`
GUARD: `EN_WDT`

```
void ca_wdt_reload(void)
```

Resets the hardware watchdog timer counter to prevent a system reset.

Ritorna:
This function returns nothing (void).

Note:

- This function is non-blocking and thread-safe because the underlying hardware reload operation is atomic. It can be safely called from both standard tasks and Interrupt Service Routines (ISRs). The function will silently do nothing if the watchdog has not been previously enabled via ca_wdt_init. The most common mistake for beginners is to place this call inside a loop that can block for longer than the watchdog timeout period, which will cause the system to reset unexpectedly.

Uso:

- This function is used in the main loop or critical task cycles to signal that the system is still responsive and healthy. It is typically paired with ca_wdt_init during system startup to configure the timeout period. In a robust design, you should call this function at a

frequency significantly higher than the watchdog timeout to ensure that even if a task is slightly delayed, the system does not trigger a hardware reset.

Esempio:

```
/* Initialize the watchdog with a specific timeout before starting the loop */
ca_wdt_init();

while (1) {
    /* Perform critical system tasks */
    do_sensor_readings();

    /*
     * Reset the watchdog timer to prevent a system reset.
     * If this code is not reached within the timeout period,
     * the hardware will force a system reboot.
     */
    ca_wdt_reload();

    /* Sleep to allow other tasks to run */
    ca_os_sleep(100);
}
```

Q&A — fine-tuning review *2/2 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_wdt_reload and why is it important for system stability?
A: You should call ca_wdt_reload in your main loop or critical task cycles to signal that the system is responsive. It is important to call it at a frequency significantly higher than the watchdog timeout period to prevent unexpected system resets caused by task delays or blocking code.

Code example conf:0.90 **Q:** How do I implement a basic watchdog reload in a caOS task loop?
A: You can call the function directly within your task loop. Ensure the watchdog has been initialized first. Example: void task_loop(void) { while(1) { ca_wdt_reload(); /* perform task logic */ } }

`ca_wdt_reset_cause`

Header: `caos/drivers/inc/wdt.h:70`
Source: `caos/drivers/src/wdt.c:148:151`
GUARD: `EN_WDT`

```
ca_wdt_reset_t ca_wdt_reset_cause(bool clear)
```

Retrieves the reason for the last system reset, specifically identifying if it was triggered by the Watchdog Timer.

Parametri:

Nome	Tipo	Descrizione
`clear`	`bool`	If true, the internal hardware reset flag is cleared after reading. If false, the flag remains set for subsequent reads.

Ritorna:
Returns a ca_wdt_reset_t enumeration value representing the reset source (e.g., power-on, software reset, or watchdog timeout).

Note:

- This function is non-blocking and thread-safe, as it typically reads a hardware register directly. It is safe to call from an ISR, though it is usually called during system initialization to log the cause of a reboot. A common beginner mistake is to pass 'false' to the clear parameter during the initial boot check, which causes the system to report a watchdog reset on every subsequent boot until the flag is manually cleared.

Uso:

- This function is typically used at the very beginning of the application startup sequence, often before the main RTOS scheduler is started. It is used to determine if the system crashed due to a watchdog timeout, allowing the developer to log the event to non-volatile memory (like backup registers) for post-mortem analysis. It is often paired with ca_bkp_read and ca_bkp_write to store diagnostic information across reboots.

Esempio:

```
ca_wdt_reset_t reset_reason;

/* Check why the system rebooted */
reset_reason = ca_wdt_reset_cause(true);

/* If the reset was caused by the watchdog, log it to backup registers */
if (reset_reason == CA_WDT_RESET_TIMEOUT) {
    /* Assume bkp_mark_idx() returns a valid index for backup storage */
    ca_bkp_write(bkp_mark_idx(), 0xDEADBEEF);
}

/* Proceed with system initialization */
ca_os_run();
```

Q&A — fine-tuning review *4/4 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the best time to call ca_wdt_reset_cause in my caOS application?
A: It is typically called at the very beginning of the application startup sequence, often before the main RTOS scheduler is started. This allows the developer to determine if the system crashed due to a watchdog timeout and log the event to non-volatile memory for post-mortem analysis.

Code example conf:0.90 **Q:** Provide a minimal example of how to check the reset cause during initialization.
A: ca_wdt_reset_t cause = ca_wdt_reset_cause(true); if (cause == CA_WDT_TIMEOUT) { /* Log to backup registers */ }

Parameters conf:1.00 **Q:** What happens if I pass 'false' to the clear parameter in ca_wdt_reset_cause?
A: If you pass false, the internal hardware reset flag remains set. A common mistake is to do this during the initial boot check, which causes the system to report a watchdog reset on every subsequent boot until the flag is manually cleared.

Return conf:1.00 **Q:** What does the return value of ca_wdt_reset_cause represent?
A: It returns a ca_wdt_reset_t enumeration value representing the reset source, such as power-on, software reset, or watchdog timeout. The function is non-blocking and thread-safe as it reads the hardware register directly.

`ca_x25519`

Header: `caos/crypto/inc/x25519.h:70`
Source: `caos/crypto/src/x25519.c:435:511`
GUARD: `EN_X25519`

```
bool ca_x25519(u8 *out, const u8 *scalar, const u8 *point)
```

Performs X25519 scalar multiplication to compute a shared secret or public key.

Parametri:

Nome	Tipo	Descrizione
`out`	`u8 *`	Pointer to a 32-byte buffer where the resulting public key or shared secret will be stored.
`scalar`	`const u8 *`	Pointer to a 32-byte private scalar (clamped internally per RFC 7748).
`point`	`const u8 *`	Pointer to a 32-byte public key or base point.

Ritorna:
Returns true if the operation succeeded and the result is non-zero (valid point). Returns false if any input pointer is NULL or if the resulting point is the point at infinity (invalid result).

Note:

- This function is computationally intensive and performs a fixed-time Montgomery ladder to prevent side-channel attacks. It is NOT thread-safe if the same buffers are accessed concurrently by other tasks. It is NOT callable from an ISR due to its long execution time and stack usage. Ensure that the 'out', 'scalar', and 'point' buffers are properly aligned and allocated before calling. A common mistake is passing uninitialized or random data as the 'scalar' without ensuring it is a valid private key, or failing to check the boolean return value, which leads to using an all-zero shared secret.

Uso:

- Used primarily in cryptographic key exchange protocols (like SSH or TLS) to derive a shared secret between two parties. It is typically used in a sequence: first, generate a private scalar using ca_rng_fill, then compute the public key using ca_x25519_base, and finally compute the shared secret by calling ca_x25519 with your private scalar and the peer's public key. Always verify the return value to ensure the resulting secret is valid before proceeding with encrypted communication.

Esempio:

```
u8 my_priv[32];
u8 peer_pub[32];
u8 shared_secret[32];

/* Initialize private key with random bytes */
if (ca_rng_fill(my_priv, 32)) {
    /* Compute shared secret using peer's public key */
    if (ca_x25519(shared_secret, my_priv, peer_pub)) {
        /* Proceed to use shared_secret for encryption */
    } else {
        /* Handle error: invalid peer key or point at infinity */
    }
} else {
    /* Handle RNG failure */
}
```

Q&A — fine-tuning review *6/6 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** In what scenario should I use the ca_x25519 function?
A: It is used in cryptographic key exchange protocols like SSH or TLS to derive a shared secret. Typically, you use it after generating a private scalar with ca_rng_fill and computing your public key with ca_x25519_base, then calling ca_x25519 with your private scalar and the peer's public key.

Code example conf:1.00 **Q:** Provide a minimal C example for computing a shared secret using caOS types.

A: u8 my_secret[32]; u8 my_private[32]; u8 peer_public[32]; if (ca_x25519(my_secret, my_private, peer_public)) { /* Proceed with secure communication */ }

Parameters conf:1.00 **Q:** What are the requirements for the 'scalar' parameter in ca_x25519?

A: The 'scalar' must be a pointer to a 32-byte buffer containing the private scalar. The function internally clamps this value per RFC 7748, but you must ensure the buffer is properly allocated and initialized before calling.

Return conf:1.00 **Q:** What does a return value of false indicate for ca_x25519?

A: A return value of false indicates that the operation failed. This occurs if any input pointer is NULL or if the resulting point is the point at infinity, which is an invalid result.

Error/Edge case conf:1.00 **Q:** What is a common mistake when using ca_x25519 that leads to security issues?

A: A common mistake is failing to check the boolean return value, which can lead to using an all-zero shared secret if the operation fails. Another is passing uninitialized or random data as the scalar without ensuring it is a valid private key.

Behavior conf:1.00 **Q:** Is ca_x25519 safe to call from an interrupt service routine (ISR)?

A: No, it is not callable from an ISR due to its long execution time and stack usage. Additionally, it is not thread-safe if the same buffers are accessed concurrently by other tasks.

`ca_x25519_base`

Header: `caos/crypto/inc/x25519.h:77`
Source: `caos/crypto/src/x25519.c:513:517`
GUARD: `EN_X25519`

```
1 void ca_x25519_base(u8 *out, const u8 *scalar)
```

Computes the X25519 public key by performing scalar multiplication on the standard base point (u=9).

Parametri:

Nome	Tipo	Descrizione
`out`	`u8 *`	Pointer to a 32-byte buffer where the resulting public key will be stored. Must be allocated and valid.
`scalar`	`const u8 *`	Pointer to a 32-byte buffer containing the private scalar (secret key). Must be 32 bytes long.

Ritorna:
void. This function does not return a status code; it assumes the provided buffers are valid and of sufficient size.

Note:

- This function is a wrapper around the core X25519 algorithm, specifically hardcoded to use the RFC 7748 base point (u=9). It is computationally intensive and performs heavy mathematical operations; therefore, it should not be called from an ISR (Interrupt Service Routine) as it will block the CPU for a significant duration. It is thread-safe provided that the 'out' and 'scalar' buffers are not accessed by other tasks simultaneously. A common beginner mistake is passing uninitialized or incorrectly sized buffers, which will lead to memory corruption or invalid cryptographic keys.

Uso:

- This function is typically used during the key exchange phase of secure communication protocols (like SSH or TLS). The standard sequence involves: first, generating a random

private scalar using ca_rng_fill; second, calling ca_x25519_base to derive the corresponding public key; and finally, using the resulting public key in a handshake or key exchange process with a remote peer. It is often paired with ca_x25519 to perform the actual Diffie-Hellman shared secret calculation once the peer's public key is received.

Esempio:

```c
/* Example: Generating a local X25519 key pair for a secure session */
#define X25519_KEY_SIZE 32

void generate_keys(void) {
    u8 private_key[X25519_KEY_SIZE];
    u8 public_key[X25519_KEY_SIZE];

    /* 1. Fill private key with random bytes */
    if (ca_rng_fill(private_key, X25519_KEY_SIZE)) {
        /* 2. Derive public key from the base point (u=9) */
        ca_x25519_base(public_key, private_key);

        /* Now public_key is ready to be sent to the peer */
    } else {
        /* Handle RNG failure: log error or abort */
    }
}
```

Q&A — fine-tuning review *3/3 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I use ca_x25519_base in my caOS application?
A: You should use this function during the key exchange phase of secure communication protocols like SSH or TLS. The typical workflow is to first generate a random private scalar using ca_rng_fill, then call ca_x25519_base to derive the public key, and finally use that public key to perform a handshake with a remote peer.

Code example conf:1.00 **Q:** How do I implement a basic public key generation using ca_x25519_base?
A: You must provide two 32-byte buffers. Example: u8 private_key[32]; u8 public_key[32]; ca_rng_fill(private_key, 32); ca_x25519_base(public_key, private_key);

Parameters conf:1.00 **Q:** What are the requirements for the 'out' and 'scalar' parameters in ca_x25519_base?
A: Both 'out' and 'scalar' must be pointers to 32-byte buffers. The 'scalar' buffer must contain the private secret key, and the 'out' buffer must be allocated and valid to store the resulting public key. Passing incorrectly sized buffers will result in memory corruption or invalid cryptographic keys.

`ca_x25519_selftest`

Header: `caos/crypto/inc/x25519.h:119`
Source: `caos/crypto/src/x25519.c:571:645`
GUARD: `EN_X25519`

```c
u32 ca_x25519_selftest(void)
```

Executes a comprehensive suite of cryptographic self-tests for the X25519 algorithm to verify implementation integrity.

Ritorna:
Returns a u32 representing the total number of failed tests. A return value of 0 indicates that all tests passed successfully.

Note:

- This function is intended for diagnostic and validation purposes during system startup or maintenance. It is not thread-safe and should not be called from an ISR, as it performs significant computation and potentially uses blocking I/O for reporting. Before calling this, ensure the system clock is initialized and the cryptographic hardware/software environment is ready. A common mistake is calling this function in a production loop; it should only be executed once during the initialization phase to verify that the X25519 implementation is functioning correctly on the specific hardware platform.

Uso:

- This function is typically invoked during the system boot sequence or within a dedicated diagnostic task. It is often paired with other self-test functions like ca_ed25519_selftest or ca_sha512_selftest to ensure the entire cryptographic stack is operational. Developers use the returned value to decide whether to proceed with system initialization or to enter a safe/error state if the hardware or software implementation fails to meet the expected cryptographic standards.

Esempio:

```c
/* Example: Running X25519 self-tests during system initialization */
#include "caos.h"

void system_init_diagnostics(void) {
    u32 total_fails = 0;

    /* Initialize the X25519 test environment if required */
    ca_x25519_test_init();

    /* Execute the self-test suite */
    total_fails = ca_x25519_selftest();

    if (total_fails > 0) {
        ca_stdout_printf_n("CRITICAL: X25519 self-test failed with %u errors",
    total_fails);
        /* Enter a safe state or halt system if crypto is unreliable */
        while(1) { /* Halt */ }
    } else {
        ca_stdout_put_n("X25519 self-test passed successfully.");
    }
}
```

Q&A — fine-tuning review *5/5 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When is the appropriate time to invoke ca_x25519_selftest in a caOS project?
A: It should be invoked during the system boot sequence or within a dedicated diagnostic task. It is intended for diagnostic and validation purposes during system startup or maintenance to ensure the cryptographic stack is operational before proceeding with system initialization.

Code example conf:1.00 **Q:** Provide a minimal C example showing how to check the result of the X25519 self-test.
A: u32 result = ca_x25519_selftest(); if (result != 0) { /* Enter safe or error state */ }

Return conf:1.00 **Q:** What does the u32 return value of ca_x25519_selftest signify?
A: The return value represents the total number of failed tests. A return value of 0 indicates that all tests passed successfully, while any non-zero value indicates the number of failures encountered during the integrity verification.

Error/Edge case conf:1.00 **Q:** What is a common mistake developers make when using ca_x25519_selftest?
A: A common mistake is calling this function in a production loop. It should only be executed once during the initialization phase to verify that the X25519 implementation is functioning correctly on the specific hardware platform.

Behavior conf:1.00 **Q:** Is ca_x25519_selftest safe to call from an ISR or in a multi-threaded environment?
A: No, the function is not thread-safe and should not be called from an ISR. It performs significant computation and may use blocking I/O, so it must be executed in a context where such operations are permitted, typically during system startup.

`ca_x25519_test_init`

Header: `caos/crypto/inc/x25519.h:124`
Source: `caos/crypto/src/x25519.c:670:677`
GUARD: `EN_X25519`

```
void ca_x25519_test_init(void)
```

Registers the X25519 cryptographic test command into the Command Line Interface (CLI) system.

Ritorna:
void

Note:

- This function is a wrapper that ensures the X25519 test command is registered exactly once in the CLI system. It uses a static boolean flag to prevent duplicate registrations, which would otherwise corrupt the CLI linked list. It is not thread-safe by itself, so it must be called during the system initialization phase before the scheduler starts or within a critical section if called later. It is not intended to be called from an ISR. The most common mistake is calling this function multiple times from different tasks without ensuring the underlying CLI registration mechanism is protected, or calling it before the CLI system itself has been initialized.

Uso:

- This function is typically called during the system startup sequence, specifically within the 'ca_os_main' function or an equivalent initialization task. It is used to expose cryptographic self-tests to the user via the serial console. It is usually paired with other test initialization functions like 'ca_sha512_test_init' and 'ca_ed25519_test_init' to populate the CLI with available diagnostic commands before 'ca_os_run' is invoked.

Esempio:

```
void ca_os_main(void) {
    /* Initialize the CLI system first (assumed existing) */
    ca_cli_init();

    /* Register cryptographic test commands for CLI access */
    ca_sha512_test_init();
    ca_x25519_test_init();
    ca_ed25519_test_init();

    /* Start the OS scheduler */
    ca_os_run(true);
}
```

Q&A — fine-tuning review *2/2 pairs · gemini-3.1-flash-lite*

Usage conf:1.00 **Q:** When should I call ca_x25519_test_init and why is it important to do so during initialization?

A: You should call ca_x25519_test_init during the system startup sequence, such as within 'ca_os_main' or an initialization task, before the scheduler starts. This is critical because the function is not thread-safe and uses a static flag to prevent duplicate registrations in the CLI linked list, which would otherwise cause corruption.

Code example conf:0.95 **Q:** How do I correctly initialize the X25519 test command in my caOS startup sequence?

A: You should invoke the function during the system initialization phase before calling ca_os_run. Example: void ca_os_main(void) { ca_x25519_test_init(); ca_sha512_test_init(); ca_ed25519_test_init(); ca_os_run(); }

Chapter 115

Reference Tipi di dato caOS

Strutture, enumerazioni e callback in ordine alfabetico. Per ogni voce: definizione C, descrizione, membri, note d'uso.

ca_adc_config_t

Tipo: Struttura
File: caos/drivers/inc/adc.h:29:32

```c
typedef struct {
u8 adc_num;      /* 1, 2, 3 */
u8 sample_time;  /* 0..7 (SMPx bits, family-specific encoding) */
} ca_adc_config_t;
```

Configuration structure used to define the hardware parameters for an ADC peripheral instance in caOS.

Membri:

Nome	Tipo	Descrizione
adc_num	u8	The identifier of the ADC peripheral to be used. Valid values are 1, 2, or 3 depending on the specific STM32 chip variant. Providing an invalid number will result in a driver initialization error.
sample_time	u8	The sampling time configuration, mapped to the SMPx bits of the ADC registers. Valid range is 0 to 7. Higher values increase sampling duration, which is necessary for high-impedance signal sources to ensure accurate readings.

Note:
This structure is typically passed by value to initialization functions. It does not require dynamic memory allocation; it should be declared as a local variable or a static constant. The user is responsible for ensuring the selected adc_num is physically present on the target MCU.

Uso:
Use this struct to prepare the hardware settings before calling the ADC initialization routine. It acts as a parameter block to configure the ADC peripheral's timing and identity.

Esempio:

```c
ca_adc_config_t my_adc_cfg = { .adc_num = 1, .sample_time = 3 };

// Pass the configuration to the driver initialization function
ca_adc_init(&my_adc_cfg);
```

---

ca_adc_isr_cb_t

Tipo: Callback
File: caos/drivers/inc/adc.h:46:46

```
typedef void (*ca_adc_isr_cb_t)(u8 adc_num, u16 value)
```

A function pointer type used to define a callback function that executes when an ADC conversion completes.

Membri:

Nome	Tipo	Descrizione
adc_num	u8	The identifier of the ADC peripheral that triggered the interrupt. Must match the hardware index defined in the system configuration.
value	u16	The raw digital value captured from the ADC conversion. Range is typically 0 to 4095 for 12-bit resolution.

Note:
This callback is executed within the context of an Interrupt Service Routine (ISR). Keep the implementation extremely short and avoid blocking operations, mutexes, or heavy processing. The user is responsible for registering this callback via the driver's configuration interface.

Uso:
Use this type when you need to process ADC data asynchronously without polling. It is typically passed to the ADC driver initialization or configuration structure to handle data arrival events.

Esempio:

```
void my_adc_handler(u8 adc_num, u16 value) {
    if (adc_num == 1) {
        process_sensor_data(value);
    }
}

// Usage in configuration
ca_adc_config_t config = {
    .isr_cb = my_adc_handler
};
```

---

ca_aes_channel_t

Tipo: Struttura
File: caos/crypto/inc/aes_channel_service.h:36:46

```
typedef struct {
ca_aes_mode_t            mode;
ca_aes_keysize_t         keysize;
void                    *ks_handle;     /* keys_store pointer (
    ca_keys_store_allocate) */
ca_aes_iv_gen_cb_t       iv_gen_cb;     /* NULL     ca_rng_fill() */
ca_aes_key_ratchet_cb_t  ratchet_cb;    /* NULL     rotate disabled */
void                    *cb_user;       /* opaque for both callbacks */
ca_aes_ctx_t            *red_ctx;       /* encrypt context (heap) */
```

```
ca_aes_ctx_t             *black_ctx;     /* decrypt context (heap) */
caMutex                   mutex;
} ca_aes_channel_t;
```

A configuration and state container for managing secure AES encryption/decryption channels in caOS.

Membri:

Nome	Tipo	Descrizione
mode	ca_aes_mode_t	Defines the AES operation mode (e.g., CBC, GCM). Must match the hardware/software capability.
keysize	ca_aes_keysize_t	Specifies the AES key length (128, 192, or 256 bits). Must be compatible with the key stored in ks_handle.
ks_handle	void *	Pointer to the key store object allocated via ca_keys_store_allocate. Must not be NULL.
iv_gen_cb	ca_aes_iv_gen_cb_t	Optional callback for IV generation. If NULL, the system defaults to ca_rng_fill().
ratchet_cb	ca_aes_key_ratchet_cb_t	Optional callback for key rotation. If NULL, key ratcheting is disabled.
cb_user	void *	Opaque pointer passed to iv_gen_cb and ratchet_cb for context-specific data.
red_ctx	ca_aes_ctx_t *	Pointer to the heap-allocated encryption context. Managed by the service.
black_ctx	ca_aes_ctx_t *	Pointer to the heap-allocated decryption context. Managed by the service.
mutex	ca_mutex_t	Internal synchronization primitive to ensure thread-safe access to the channel.

Note:
The struct must be initialized to zero before use. The red_ctx and black_ctx pointers are managed by the caOS crypto service; do not manually free these pointers. The mutex is automatically initialized during channel registration. Ensure the key store handle is valid for the lifetime of the channel.

Uso:
Used to initialize and maintain a secure cryptographic channel. It is passed to ca_aes_channel_init() to prepare the contexts and mutex before performing any encryption or decryption operations.

Esempio:

```
ca_aes_channel_t my_channel = {0};
my_channel.mode = CA_AES_MODE_GCM;
my_channel.keysize = CA_AES_KEYSIZE_256;
my_channel.ks_handle = my_key_store_ptr;

if (ca_aes_channel_init(&my_channel) == CA_SUCCESS) {
    /* Channel is ready for use */
}
```

---

ca_aes_ctx_t

Tipo: Struttura
File: caos/drivers/inc/aes.h:39:45

```
typedef struct {
ca_aes_mode_t      mode;
ca_aes_keysize_t   keysize;
u32 key[8];       /* max 256-bit = 8 words */
u32 iv[4];        /* IV = 4 words (CBC/CTR) */
bool configured;
} ca_aes_ctx_t;
```

A configuration structure used to store the state and parameters for AES hardware encryption/decryption operations.

Membri:

Nome	Tipo	Descrizione
`mode`	`ca_aes_mode_t`	Defines the AES operation mode (e.g., ECB, CBC). Must be set before calling initialization functions.
`keysize`	`ca_aes_keysize_t`	Specifies the bit-length of the key (128, 192, or 256 bits). Must match the data provided in the key array.
`key`	`u32[8]`	The secret key buffer. Only the required number of words based on keysize are used.
`iv`	`u32[4]`	Initialization Vector for modes like CBC or CTR. Ignored for ECB mode.
`configured`	`bool`	Internal flag used by the driver to verify if the context has been properly initialized. Do not modify manually.

Note:
The structure is typically allocated by the user on the stack or as a global variable. It must be passed to the AES driver initialization function before any encryption or decryption calls. The driver does not own the memory; the user is responsible for its lifecycle.

Uso:
Use this struct to define the cryptographic parameters for a session. Pass a pointer to this struct to the AES driver's setup and processing functions to perform secure data transformations.

Esempio:

```
ca_aes_ctx_t aes_ctx = {0};
aes_ctx.mode = CA_AES_MODE_CBC;
aes_ctx.keysize = CA_AES_KEYSIZE_128;
memcpy(aes_ctx.key, my_secret_key, 16);
memcpy(aes_ctx.iv, my_iv, 16);
ca_aes_init(&aes_ctx);
```

---

ca_aes_iv_gen_cb_t

Tipo: Callback
File: `caos/crypto/inc/aes_channel_service.h:27:27`

```
typedef void (*ca_aes_iv_gen_cb_t)(u8 iv[AES_CH_IV_SIZE], void *user)
```

A callback function type used to generate a unique Initialization Vector (IV) for AES encryption operations.

Membri:

Nome	Tipo	Descrizione
iv	u8[AES_CH_IV_SIZE]	A buffer of size AES_CH_IV_SIZE (typically 16 bytes) where the generated IV must be written. Must not be null.
user	void *	A pointer to user-defined context data passed through from the registration function. Can be NULL if no context is needed.

Note:
This callback is invoked by the AES channel service whenever a new IV is required for a cryptographic operation. The implementation must ensure the IV is cryptographically strong (e.g., using a hardware random number generator). The caller of the AES service owns the memory pointed to by the 'user' parameter. The callback must be thread-safe if the AES service is accessed from multiple tasks.

Uso:
Used when configuring an AES channel via the service API. You provide a pointer to a function matching this signature to allow the system to request a fresh IV for each encryption block or session.

Esempio:

```
static void my_iv_generator(u8 iv[AES_CH_IV_SIZE], void *user) {
    // Fill the IV buffer with random data from the hardware RNG
    for (int i = 0; i < AES_CH_IV_SIZE; i++) {
        iv[i] = (u8)(rand() & 0xFF);
    }
}

// Usage in configuration
ca_aes_config_t config = {
    .iv_gen_cb = my_iv_generator,
    .user_data = NULL
};
```

---

ca_aes_key_ratchet_cb_t

Tipo: Callback
File: caos/crypto/inc/aes_channel_service.h:34:34

```
typedef bool (*ca_aes_key_ratchet_cb_t)(u8 *key, u32 key_len, void *user
    )
```

A callback function type used to update or rotate an AES encryption key dynamically during a secure session.

Membri:

Nome	Tipo	Descrizione
key	u8 *	Pointer to the buffer where the new key material should be written. Must not be NULL.
key_len	u32	The size of the key buffer in bytes. Ensure the written key does not exceed this length to prevent buffer overflows.
user	void *	A user-defined context pointer passed through from the registration function, typically used to access session state or configuration.

Note:
This callback is invoked by the AES service when a key rotation event is triggered. The function must return 'true' if the key was successfully updated, or 'false' to indicate a failure, which will likely halt the cryptographic operation. Ensure the implementation is thread-safe if the AES service is accessed from multiple tasks.

Uso:
Used when configuring an AES channel that requires periodic key updates (ratcheting) for forward secrecy. It is registered during the initialization of the AES channel service.

Esempio:

```
static bool my_key_ratchet_handler(u8 *key, u32 key_len, void *user) {
    // Example: Derive a new key using a simple XOR or KDF
    for (u32 i = 0; i < key_len; i++) {
        key[i] ^= 0xAA;
    }
    return true;
}

// Usage in service configuration
ca_aes_channel_cfg_t cfg = {
    .ratchet_cb = my_key_ratchet_handler,
    .user_data = NULL
};
```

`ca_aes_keysize_t`

Tipo: Enumerazione
File: `caos/drivers/inc/aes.h:32:36`

```
typedef enum {
AES_KEY_128 = 0,    /* KEYSIZE = 00 */
AES_KEY_192 = 1,    /* KEYSIZE = 01 */
AES_KEY_256 = 2,    /* KEYSIZE = 10 */
} ca_aes_keysize_t;
```

An enumeration defining the supported bit-lengths for AES encryption keys in the caOS hardware abstraction layer.

Membri:

Nome	Tipo	Descrizione
`CA_AES_KEY_128`	enum constant	Configures the AES engine for 128-bit keys. Valid value: 0.
`CA_AES_KEY_192`	enum constant	Configures the AES engine for 192-bit keys. Valid value: 1.
`CA_AES_KEY_256`	enum constant	Configures the AES engine for 256-bit keys. Valid value: 2.

Note:
This enum is used to program the hardware registers of the STM32 AES peripheral. Ensure the key buffer provided to the driver matches the size selected here; providing a buffer smaller than the selected key size will lead to memory corruption or undefined hardware behavior.

Uso:
Used as a parameter in AES initialization or configuration functions to specify the security strength of the encryption operation.

Esempio:

```c
ca_aes_keysize_t my_key_size = CA_AES_KEY_256;

// Example usage in a hypothetical driver configuration
void setup_encryption(void) {
    ca_aes_config_t config;
    config.key_size = my_key_size;
    // Proceed to initialize the AES peripheral with the selected size
}
```

---

ca_aes_mode_t

Tipo: Enumerazione
File: `caos/drivers/inc/aes.h:25:30`

```c
typedef enum {
AES_MODE_ECB = 0,
AES_MODE_CBC = 1,
AES_MODE_CTR = 2,
AES_MODE_GCM = 3,
} ca_aes_mode_t;
```

An enumeration defining the cryptographic operation modes available for the hardware AES peripheral.

Membri:

Nome	Tipo	Descrizione
`CA_AES_MODE_ECB`	`enum constant`	Electronic Codebook mode. Simplest mode, encrypts blocks independently. Not recommended for long messages due to pattern leakage.
`CA_AES_MODE_CBC`	`enum constant`	Cipher Block Chaining mode. Uses an Initialization Vector (IV) to chain blocks. Requires padding for data not aligned to block size.
`CA_AES_MODE_CTR`	`enum constant`	Counter mode. Turns a block cipher into a stream cipher. Does not require padding.
`CA_AES_MODE_GCM`	`enum constant`	Galois/Counter Mode. Provides both encryption and authentication (AEAD). Recommended for secure communication.

Note:
This enum is used to configure the AES hardware engine. Ensure the selected mode is supported by the specific STM32 hardware variant being used. Incorrect mode selection will result in decryption failure or invalid authentication tags.

Uso:
Pass this value to the AES initialization or configuration functions to set the operational mode before starting a cryptographic operation.

Esempio:

```c
ca_aes_config_t config;
config.mode = CA_AES_MODE_GCM;
config.key_size = CA_AES_KEY_128;
// Proceed to initialize the AES peripheral with the selected mode
ca_aes_init(&config);
```

---

ca_arp_cfg_t

Tipo: Struttura
File: caos/net/inc/net_cfg.h:83:86

```
typedef struct {
u32 renew_ms;   /* ARP cache entry lifetime in ms (0 = no cache) */
u8  cache_size; /* number of entries in static ARP cache (0 = disabled)
   */
} ca_arp_cfg_t;
```

Configuration structure for the Address Resolution Protocol (ARP) cache settings in the caOS network stack.

Membri:

Nome	Tipo	Descrizione
renew_ms	u32	The duration in milliseconds before an ARP cache entry is considered stale and requires renewal. Set to 0 to disable caching entirely.
cache_size	u8	The maximum number of static ARP entries the system can store. Set to 0 to disable the ARP cache feature.

Note:
This structure is typically passed as part of a larger network configuration object during system initialization. It is usually allocated in static memory or on the stack during the boot sequence. The user is responsible for ensuring these values are set before the network stack is started.

Uso:
Use this struct when configuring the network interface parameters in the caOS stack. It defines how the system manages IP-to-MAC address mappings to optimize network traffic.

Esempio:

```
ca_arp_cfg_t arp_config = { .renew_ms = 60000, .cache_size = 10 };
```

---

ca_arp_frame_t

Tipo: Struttura
File: caos/net/inc/net_proto.h:91:101

```
typedef struct {
u16    htype;   /* hardware type = ARP_HTYPE_ETH big-endian */
u16    ptype;   /* protocol type = ARP_PTYPE_IP  big-endian */
u8     hlen;    /* hardware address length = 6 */
u8     plen;    /* protocol address length = 4 */
u16    oper;    /* ARP_OP_REQUEST / ARP_OP_REPLY big-endian */
ca_mac_t  sha;     /* sender hardware address */
ca_ipv4_t spa;     /* sender protocol address */
ca_mac_t  tha;     /* target hardware address */
ca_ipv4_t tpa;     /* target protocol address */
} __attribute__((packed)) ca_arp_frame_t;
```

Represents the structure of an ARP (Address Resolution Protocol) packet used for mapping hardware addresses to network addresses.

Membri:

Nome	Tipo	Descrizione
`htype`	`u16`	Hardware type. Must be set to ARP_HTYPE_ETH (0x0001) in big-endian format.
`ptype`	`u16`	Protocol type. Must be set to ARP_PTYPE_IP (0x0800) in big-endian format.
`hlen`	`u8`	Hardware address length. For Ethernet, this is always 6.
`plen`	`u8`	Protocol address length. For IPv4, this is always 4.
`oper`	`u16`	Operation code. Use ARP_OP_REQUEST (1) or ARP_OP_REPLY (2) in big-endian format.
`sha`	`ca_mac_t`	Sender hardware address (MAC address of the device sending the packet).
`spa`	`ca_ipv4_t`	Sender protocol address (IP address of the device sending the packet).
`tha`	`ca_mac_t`	Target hardware address (MAC address of the destination device).
`tpa`	`ca_ipv4_t`	Target protocol address (IP address of the destination device).

Note:
The struct is marked with ___attribute___((packed)) to ensure no padding bytes are inserted by the compiler, which is critical for network protocol compliance. All multi-byte fields must be stored in big-endian (network byte order).

Uso:
Used when manually constructing or parsing raw ARP packets in the network stack layer. Typically populated before passing the buffer to the Ethernet driver's transmit function.

Esempio:

```
ca_arp_frame_t arp_pkt;

arp_pkt.htype = htons(ARP_HTYPE_ETH);
arp_pkt.ptype = htons(ARP_PTYPE_IP);
arp_pkt.hlen = 6;
arp_pkt.plen = 4;
arp_pkt.oper = htons(ARP_OP_REQUEST);

// Populate addresses
memcpy(arp_pkt.sha, my_mac, 6);
memcpy(arp_pkt.spa, my_ip, 4);
memset(arp_pkt.tha, 0, 6); // Target MAC unknown in request
memcpy(arp_pkt.tpa, target_ip, 4);
```

`ca_barrier_t`

Tipo: Struttura
File: `caos/inc/sync.h:352:358`

```
typedef struct {
volatile u32  count;          /* number of tasks to synchronize */
volatile u32  arrived;        /* tasks arrived at the barrier in this
   phase */
volatile u32  phase;          /* current phase (auto-incremented) */
ca_wait_queue_t   wait_queue;
```

```
ca_spinlock_irq_t internal_lock;
} ca_barrier_t;
```

A synchronization primitive that forces a group of tasks to wait until a specific number of tasks have reached the barrier point.

Membri:

Nome	Tipo	Descrizione
`count`	`u32`	The total number of tasks required to release the barrier. Must be greater than zero.
`arrived`	`u32`	Internal counter tracking how many tasks have currently reached the barrier. Do not modify manually.
`phase`	`u32`	Internal counter used to distinguish between different synchronization cycles. Do not modify manually.
`wait_queue`	`ca_wait_queue_t`	The internal queue where tasks are blocked while waiting for the barrier to be satisfied.
`internal_lock`	`ca_spinlock_irq_t`	A spinlock used to ensure atomic access to the barrier's internal state during synchronization.

Note:
The barrier must be initialized before use. It is typically allocated in the global data segment or on the heap by the application. Ownership is held by the application, but the RTOS kernel manages the internal state during wait/signal operations. Ensure 'count' is set correctly during initialization, as an incorrect value can lead to permanent task blocking (deadlock).

Uso:
Use this when you need to coordinate multiple parallel tasks to reach a common execution point before any of them can proceed to the next stage of processing.

Esempio:

```
ca_barrier_t my_barrier;

void init_sync(void) {
    my_barrier.count = 3;
    my_barrier.arrived = 0;
    my_barrier.phase = 0;
    ca_wait_queue_init(&my_barrier.wait_queue);
    ca_spinlock_irq_init(&my_barrier.internal_lock);
}

void task_worker(void *arg) {
    /* Perform work */
    ca_barrier_wait(&my_barrier);
    /* Proceed only after 3 tasks have reached this point */
}
```

`ca_block_pool_t`

Tipo: Struttura
File: `caos/inc/block_pool.h:38:44`

```
typedef struct {
void          *free_list;   /* first free block (in-place linked list) */
u32            block_size;  /* size of each block (>= 4, multiple of 4)
   */
```

```
u32            total;       /* total number of blocks */
volatile u32   free_count;  /* currently free blocks */
ca_spinlock_irq_t lock;        /* ISR-safe spinlock */
} ca_block_pool_t;
```

A fixed-size memory block allocator that provides O(1) allocation and deallocation for objects of identical size.

Membri:

Nome	Tipo	Descrizione
`free_list`	`void *`	Pointer to the first available memory block. Managed internally as a linked list.
`block_size`	`u32`	Size of each individual block in bytes. Must be at least 4 and a multiple of 4.
`total`	`u32`	Total number of blocks available in the pool.
`free_count`	`volatile u32`	Current number of available blocks. Updated atomically.
`lock`	`ca_spinlock_irq_t`	Spinlock used to ensure thread-safe and ISR-safe access to the pool.

Note:
The pool must be initialized using ca_block_pool_init before use. The memory buffer provided to the pool must be aligned to 4 bytes. The structure itself is typically allocated in static memory or as part of a larger control block. Ownership of the memory remains with the pool; users must return blocks using ca_block_free to prevent leaks.

Uso:
Use this when you need to frequently allocate and free objects of the same size (e.g., network packets or message buffers) to avoid heap fragmentation. Use ca_block_alloc to retrieve a block and ca_block_free to return it.

Esempio:

```
#include "caos/inc/block_pool.h"

#define POOL_SIZE 10
#define BLOCK_SIZE 64

static ca_block_pool_t my_pool;
static u8 pool_buffer[POOL_SIZE * BLOCK_SIZE];

void init_system(void) {
    /* Initialize the pool with the buffer and parameters */
    ca_block_pool_init(&my_pool, pool_buffer, BLOCK_SIZE, POOL_SIZE);
}

void task_process(void) {
    void *ptr = ca_block_alloc(&my_pool);
    if (ptr != NULL) {
        /* Use the block for data processing */
        ca_memzero(ptr, BLOCK_SIZE);

        /* Return the block to the pool */
        ca_block_free(&my_pool, ptr);
    }
}
```

ca_can_cfg_t

Tipo: Struttura
File: caos/drivers/inc/can.h:24:29

```
typedef struct {
u8  can_num;    /* 1 = CAN1/FDCAN1, 2 = CAN2/FDCAN2 */
u32 baudrate;   /* 125000 | 250000 | 500000 | 1000000 */
bool loopback;  /* true = test loopback interno */
bool silent;    /* true = listen-only (bus monitoring) */
} ca_can_cfg_t;
```

Configuration structure used to initialize a CAN peripheral interface in caOS.

Membri:

Nome	Tipo	Descrizione
can_num	u8	The hardware peripheral index. Use 1 for CAN1/FDCAN1 or 2 for CAN2/FDCAN2. Invalid values may cause hardware initialization failure.
baudrate	u32	The communication speed in bits per second. Supported values are 125000, 250000, 500000, or 1000000. Incorrect values will result in bus errors.
loopback	bool	Enables internal loopback mode for testing. If true, transmitted frames are received internally and not sent to the physical bus.
silent	bool	Enables listen-only mode. If true, the controller receives messages but does not acknowledge them or transmit, useful for bus monitoring.

Note:
This struct is typically passed by pointer to the driver initialization function. It is usually allocated on the stack before calling the init function. Ensure all fields are explicitly set to avoid undefined behavior from uninitialized memory.

Uso:
Use this struct to define the operating parameters of a CAN bus before calling the driver's initialization function. It is the standard way to configure hardware settings without hardcoding values in the driver logic.

Esempio:

```
ca_can_cfg_t can_config = {
    .can_num = 1,
    .baudrate = 500000,
    .loopback = false,
    .silent = false
};

/* Pass the address of the struct to the driver init function */
ca_can_init(&can_config);
```

---

ca_can_frame_t

Tipo: Struttura
File: caos/drivers/inc/can.h:31:37

```c
typedef struct {
u32  id;       /* 11-bit (ide=false) or 29-bit (ide=true) */
bool ide;      /* false = standard frame, true = extended */
bool rtr;      /* true = remote frame */
u8   dlc;      /* number of valid bytes in data[], 0-8 */
u8   data[8];
} ca_can_frame_t;
```

Represents a single CAN bus data frame used for transmitting or receiving messages.

Membri:

Nome	Tipo	Descrizione
`id`	`u32`	The CAN message identifier. Use 11 bits for standard frames or 29 bits for extended frames.
`ide`	`bool`	Identifier Extension bit. Set to false for 11-bit standard IDs, true for 29-bit extended IDs.
`rtr`	`bool`	Remote Transmission Request. Set to true to request data from another node, false for standard data frames.
`dlc`	`u8`	Data Length Code. Specifies the number of valid bytes in the data array. Must be between 0 and 8.
`data`	`u8[8]`	The payload buffer containing up to 8 bytes of data.

Note:
This struct is typically allocated on the stack by the application. It does not require dynamic memory allocation. Ensure dlc is set correctly, as the hardware will ignore bytes beyond the specified length.

Uso:
Used as a parameter in CAN driver functions to define the content of a message to be sent or to store a received message.

Esempio:

```c
ca_can_frame_t frame = {0};
frame.id = 0x123;
frame.ide = false;
frame.rtr = false;
frame.dlc = 2;
frame.data[0] = 0xAA;
frame.data[1] = 0xBB;
ca_can_send(CAN1, &frame);
```

`ca_caos_config_status`

Tipo: Struttura
File: `caos/inc/caos_info.h:9:175`

```c
typedef struct {
/* --- I/O subsystems (os_config.h) --- */
u8  itm;
u8  dbg_uart;
u8  uart;
/* USB */
u8  usb;
u8  usb_cdc;
u8  usb_msc;
```

```
u8  usb_ncm;
u8  usb_dfu;
/* stdout */
u8  stdout_en;
u8  stdout_on_uart;
u8  stdout_on_usb_cdc;
u8  stdout_on_itm;
/* CLI */
u8  cli;
u8  cli_capture;
u8  stdin_on_uart;
u8  stdin_on_usb_cdc;
/* log / sensor */
u8  log;
u8  sensor;
/* --- Peripherals (os_config.h) --- */
u8  spi;
u8  i2c;
u8  adc;
u8  dma;
u8  exti;
u8  dac;
u8  flash;
u8  qspi;
u8  sdio;
u8  timer;
u8  wdt;
u8  can;
u8  eth;
u8  eth_chk_hw;
u8  rtc;
/* --- NET stack (os_config.h) --- */
u8  net;
u8  net_dhcp;
u8  net_ntp;
u8  net_syslog;
u8  net_dns;
u8  net_mdns;
u8  net_mqtt;
u8  net_modbus;
u8  net_iperf;
u8  net_http;
u8  packet_tracer;
u8  arp_service;
u8  route_service;
u8  udp_server;
u8  udp_client;
u8  dhcp_service;
u8  ntp_service;
u8  dns_service;
u8  mdns_service;
u8  syslog_service;
u8  tcp_service;
u8  mqtt_service;
u8  modbus_service;
u8  iperf_service;
u8  http_service;
/* --- Crypto / compute (os_config.h) --- */
```

```
u8  crc;
u8  rng;
u8  hash;
u8  aes;
/* --- OS primitives (os_config.h) --- */
u8  sw_timer;
u8  event_grp;
u8  notify;
u8  message;
u8  stream;
u8  measure;
u8  keys_store;
u8  pool;
u8  block_pool;
u8  rwlock;
u8  barrier;
/* --- Endian (os_config.h) --- */
u8  big_endian;
u8  little_endian;
/* --- CLI sub-flags (os_config.h) --- */
u8  cli_arp;
u8  cli_dhcp;
u8  cli_ntp;
u8  cli_dns;
u8  cli_mdns;
u8  cli_syslog;
u8  cli_tcp;
u8  cli_mqtt;
u8  cli_modbus;
u8  cli_iperf;
u8  cli_http;
u8  cli_trace;
u8  cli_udp_srv;
u8  cli_udp_cli;
u8  cli_rtc;
u8  cli_eth;
u8  cli_net;
u8  cli_route;
u8  cli_gpio;
u8  cli_uart;
u8  cli_adc;
u8  cli_dac;
u8  cli_i2c;
u8  cli_flash;
u8  cli_timer;
u8  cli_dma;
u8  cli_spi;
u8  cli_wdt;
u8  cli_rng;
u8  cli_crc;
u8  cli_kstore;
u8  cli_sdio;
u8  cli_qspi;
u8  cli_can;
/* --- Numeric: clocks (objmap.h) --- */
u32 sysclk_hz;
u32 hclk_hz;
u32 pclk1_hz;
```

```
126 u32 pclk2_hz;
127 /* --- Numeric: scheduler (objmap.h) --- */
128 u8  max_task;
129 u8  max_timer;
130 u8  dump_buffer_size;
131 u32 os_tick_us;
132 /* --- Numeric: stack sizes (objmap.h) --- */
133 u32 stack_min;
134 u32 stack_small;
135 u32 stack_medium;
136 u32 stack_large;
137 u32 stack_xlarge;
138 /* --- Numeric: UART debug (os_config.h) --- */
139 u8  uart_dbg_num;
140 u32 uart_dbg_baudrate;
141 /* --- Numeric: net sizes / counts (os_config.h) --- */
142 u8  packet_tracer_ring_size;
143 u8  arp_cache_size;
144 u8  net_route_max;
145 u8  pool_queue_size;
146 u8  udpsrv_max_clients;
147 u8  udpsrv_ring_depth;
148 u8  udpcli_max_handles;
149 u8  udpcli_ring_depth;
150 u8  tcp_max_conn;
151 u8  tcp_max_listen;
152 u8  tcp_max_inflight;
153 u8  tcp_rx_ring_depth;
154 u8  ntp_server_n;
155 u32 ntp_poll_ms;
156 u8  dns_pending_max;
157 u8  syslog_queue_size;
158 u8  syslog_msg_max;
159 u16 mqtt_rx_buf_size;
160 u16 mqtt_keepalive_sec;
161 u32 modbus_timeout_ms;
162 u16 modbus_rx_buf_size;
163 u16 iperf_default_port;
164 u16 http_default_port;
165 u16 http_req_buf_size;
166 u16 http_resp_buf_size;
167 } ca_caos_config_status;
```

A structure containing the current configuration and status flags of the caOS RTOS environment.

Membri:

Nome	Tipo	Descrizione
`u8 fields (e.g., itm, usb, spi, net)`	`u8`	Boolean-like subsystem, p current build
`u32/u16/u8 numeric fields (e.g., sysclk_hz, max_task, stack_min)`	`u32/u16/u8`	Configuration clock frequen

Note:
This struct is typically populated by the system during initialization. It is read-only for the application layer; modifying these values at runtime will not change the actual hardware or OS configuration, as they reflect the static settings defined in os_config.h and objmap.h.

Uso:
Use this struct to inspect the system's capabilities or current configuration at runtime, especially when writing diagnostic tools, CLI commands, or generic drivers that need to adapt based on enabled features.

Esempio:

```
ca_caos_config_status config;

// Retrieve the current system configuration
if (caos_get_config(&config) == CA_OK) {
    if (config.usb_cdc) {
        // USB CDC is enabled, safe to initialize USB-related logging
        printf("USB CDC is active at %lu Hz\n", config.sysclk_hz);
    }
}
```

---

ca_cli_hook_t

Tipo: Callback
File: caos/inc/cli.h:189:189

```
typedef u16 (*ca_cli_hook_t)(c_str cmd)
```

A function pointer type used to define custom command handlers for the caOS Command Line Interface (CLI).

Membri:

Nome	Tipo	Descrizione
cmd	c_str	A pointer to the string containing the command arguments passed by the user. Must not be modified.
return value	u16	An integer status code. Typically 0 indicates success, while non-zero values represent specific error codes defined by the application.

Note:
The callback is executed within the context of the CLI task. Ensure the implementation is non-blocking and does not perform long-running operations to avoid starving other system tasks. The memory for the command string is managed by the CLI module and is only valid for the duration of the callback execution.

Uso:
Use this type when registering new commands in the CLI system. You define a function matching this signature and pass it to the CLI registration API to extend the shell functionality.

Esempio:

```
static u16 my_custom_cmd(c_str cmd) {
    if (cmd == NULL) return 1;
    // Process the command logic here
    return 0;
}

// Registration example
ca_cli_register_cmd("status", my_custom_cmd);
```

---

ca_cli_node_t

Tipo: Struttura
File: caos/inc/cli.h:59:62

```
typedef struct ca_cli_node {
bool (*handler)(c_str *cmd);  /* true = comando gestito */
struct ca_cli_node *next;
} ca_cli_node_t;
```

A linked-list node representing a single command entry in the caOS Command Line Interface (CLI) system.

Membri:

Nome	Tipo	Descrizione
handler	bool (*)(c_str *cmd)	Function pointer to the command logic. Must return true if the command was recognized and executed, false otherwise. If it returns false, the CLI engine will attempt to match the next node in the list.
next	ca_cli_node_t *	Pointer to the next command node in the linked list. Must be set to NULL for the last element to avoid memory access violations during traversal.

Note:
The structure is typically allocated statically in the application code to ensure it persists for the lifetime of the CLI. The developer is responsible for linking these nodes together to form a chain. The CLI engine traverses this list sequentially, so place frequently used commands at the beginning of the list for better performance.

Uso:
Used to register custom commands to the caOS CLI. You define a handler function, assign it to a node, and link it into the system's command list so the CLI can parse user input.

Esempio:

```
static bool my_cmd_handler(c_str *cmd) { if (c_str_cmp(cmd, "help") == 0) { /*
    handle help */ return true; } return false; } static ca_cli_node_t my_node =
     { .handler = my_cmd_handler, .next = NULL };
```

---

ca_condvar_t

Tipo: Struttura
File: caos/inc/sync.h:237:240

```
typedef struct {
ca_wait_queue_t wait_queue;       // Task waiting
ca_spinlock_irq_t  internal_lock; // Protects internal structure
} ca_condvar_t;
```

A condition variable used to block a task until a specific condition is met or a signal is received from another task.

Membri:

Nome	Tipo	Descrizione
`wait_queue`	`ca_wait_queue_t`	A list of tasks currently blocked on this condition variable. Do not modify this field directly.
`internal_lock`	`ca_spinlock_irq_t`	A spinlock used to ensure thread-safe access to the wait queue. Must be held during internal operations.

Note:
The condition variable must be initialized before use. It does not store state; it only provides a mechanism for tasks to wait for a signal. It is typically used in conjunction with a mutex to prevent race conditions when checking the condition.

Uso:
Use this when a task needs to wait for a specific event or data availability. It is used with ca_cond_wait() to block and ca_cond_signal() or ca_cond_broadcast() to wake up waiting tasks.

Esempio:

```
ca_condvar_t my_cond;

void init_system() {
    ca_cond_init(&my_cond);
}

void task_waiter() {
    ca_mutex_lock(&my_mutex);
    while (!data_ready) {
        ca_cond_wait(&my_cond, &my_mutex);
    }
    // Process data
    ca_mutex_unlock(&my_mutex);
}

void task_signaler() {
    ca_mutex_lock(&my_mutex);
    data_ready = 1;
    ca_cond_signal(&my_cond);
    ca_mutex_unlock(&my_mutex);
}
```

`ca_cordic_func_t`

Tipo: Enumerazione
File: `caos/drivers/inc/cordic.h:35:46`

```
typedef enum {
CA_CORDIC_COS = 0,
CA_CORDIC_SIN,
CA_CORDIC_PHASE,
CA_CORDIC_MODULUS,
CA_CORDIC_ATAN,
CA_CORDIC_COSH,
CA_CORDIC_SINH,
CA_CORDIC_ATANH,
CA_CORDIC_LN,
CA_CORDIC_SQRT
} ca_cordic_func_t;
```

Enumeration defining the mathematical operations supported by the hardware CORDIC co-processor.

Membri:

Nome	Tipo	Descrizione
CA_CORDIC_COS	enum	Calculates the cosine of the input angle.
CA_CORDIC_SIN	enum	Calculates the sine of the input angle.
CA_CORDIC_PHASE	enum	Calculates the phase (arctangent) of a complex number (x, y).
CA_CORDIC_MODULUS	enum	Calculates the magnitude (modulus) of a complex number (x, y).
CA_CORDIC_ATAN	enum	Calculates the arctangent of the input value.
CA_CORDIC_COSH	enum	Calculates the hyperbolic cosine.
CA_CORDIC_SINH	enum	Calculates the hyperbolic sine.
CA_CORDIC_ATANH	enum	Calculates the hyperbolic arctangent.
CA_CORDIC_LN	enum	Calculates the natural logarithm.
CA_CORDIC_SQRT	enum	Calculates the square root.

Note:
This enum is used to configure the CORDIC hardware peripheral. Ensure the CORDIC unit is initialized via ca_cordic_init before performing calculations. The hardware expects specific input formats (typically Q1.31 fixed-point) depending on the selected function.

Uso:
Used as a parameter in the ca_cordic_calc function to specify which mathematical operation the hardware should execute on the provided input data.

Esempio:

```c
#include "caos/drivers/inc/cordic.h"

void perform_math(void) {
    int32_t input = 1073741824; // Example Q1.31 value
    int32_t result = 0;

    // Initialize the CORDIC hardware
    ca_cordic_init();

    // Perform a square root calculation using the CORDIC unit
    // ca_cordic_calc(function, input_value, &output_pointer)
    ca_cordic_calc(CA_CORDIC_SQRT, input, &result);
}
```

---

ca_dac_comp_hyst_t

Tipo: Enumerazione
File: caos/drivers/inc/dac.h:46:51

```c
typedef enum {
CA_DAC_COMP_HYST_NONE = 0,
CA_DAC_COMP_HYST_LOW,
CA_DAC_COMP_HYST_MEDIUM,
CA_DAC_COMP_HYST_HIGH
} ca_dac_comp_hyst_t;
```

Defines the hysteresis level for the DAC internal comparator, used to prevent signal noise from triggering false state changes.

Membri:

Nome	Tipo	Descrizione
`CA_DAC_COMP_HYST_NONE`	`enum value`	No hysteresis applied. The comparator is highly sensitive to noise.
`CA_DAC_COMP_HYST_LOW`	`enum value`	Low hysteresis level. Suitable for signals with minimal noise.
`CA_DAC_COMP_HYST_MEDIUM`	`enum value`	Medium hysteresis level. Balanced setting for general use.
`CA_DAC_COMP_HYST_HIGH`	`enum value`	High hysteresis level. Best for noisy environments to ensure stable switching.

Note:
This enum is passed to the DAC configuration functions to tune the hardware comparator. It is a simple integer-based enum, so no memory allocation or lifecycle management is required by the user.

Uso:
Use this type when configuring the DAC comparator via ca_dac_comp_init or updating the level dynamically using ca_dac_comp_level.

Esempio:

```c
#include "caos/drivers/inc/dac.h"

void setup_dac_comparator(void) {
    /* Initialize the DAC comparator with medium hysteresis */
    ca_dac_comp_init(CA_DAC_COMP_HYST_MEDIUM);

    /* Later, if noise increases, update the hysteresis level */
    ca_dac_comp_level(CA_DAC_COMP_HYST_HIGH);
}
```

`ca_dac_config_t`

Tipo: Struttura
File: `caos/drivers/inc/dac.h:24:26`

```c
typedef struct {
u8 channel;   /* 1 = CH1 (PA4), 2 = CH2 (PA5) */
} ca_dac_config_t;
```

Configuration structure used to specify which hardware DAC channel to initialize on the STM32.

Membri:

Nome	Tipo	Descrizione
`channel`	`u8`	The DAC channel index. Use 1 for CH1 (mapped to PA4) or 2 for CH2 (mapped to PA5). Providing any other value may result in undefined behavior or initialization failure.

Note:
This structure is typically passed by value or pointer to the DAC initialization function. It does not require dynamic allocation and should be defined in the scope where the driver is being configured.

Uso:

Use this struct when preparing to initialize the DAC peripheral. It acts as a parameter for the driver setup function to inform the system which physical pin and internal DAC register set to enable.

Esempio:

```
ca_dac_config_t dac_cfg;
dac_cfg.channel = 1;
ca_dac_init(&dac_cfg);
```

---

ca_dhcp_bound_cb_t

Tipo: Callback
File: caos/net/inc/dhcp_service.h:81:81

```
typedef void (*ca_dhcp_bound_cb_t)(ca_ipv4_t ip, ca_ipv4_t mask,
```

A callback function type triggered by the DHCP service when an IP address lease is successfully obtained or renewed.

Membri:

Nome	Tipo	Descrizione
ip	ca_ipv4_t	The IPv4 address assigned to the device by the DHCP server. Must be treated as a read-only value.
mask	ca_ipv4_t	The subnet mask associated with the assigned IP address. Used to define the local network boundary.

Note:
This callback is executed within the context of the DHCP service task. It must be kept short and non-blocking to avoid delaying network stack processing. The user is responsible for storing these values if they are needed outside the callback scope, as the parameters are passed by value.

Uso:
Used when registering a DHCP client instance. You assign a function pointer of this type to the DHCP configuration structure to receive notification of network configuration changes.

Esempio:

```
void my_dhcp_callback(ca_ipv4_t ip, ca_ipv4_t mask) {
    printf("DHCP Bound! IP: %u.%u.%u.%u\n", ip.addr[0], ip.addr[1], ip.addr[2],
    ip.addr[3]);
}

// Usage in configuration
ca_dhcp_config_t config = {
    .bound_cb = my_dhcp_callback
};
```

---

ca_dhcp_bound_t

Tipo: Callback
File: caos/net/inc/net_cfg.h:101:101

```
typedef void (*ca_dhcp_bound_t)(ca_ipv4_t ip, ca_ipv4_t mask, ca_ipv4_t
    gw, ca_ipv4_t dns)
```

A callback function type triggered by the DHCP client when an IP address lease is successfully obtained or renewed.

Membri:

Nome	Tipo	Descrizione
ip	ca_ipv4_t	The IPv4 address assigned to the device by the DHCP server.
mask	ca_ipv4_t	The subnet mask provided by the DHCP server to define the local network range.
gw	ca_ipv4_t	The default gateway (router) IP address for external network communication.
dns	ca_ipv4_t	The primary DNS server IP address provided by the DHCP server.

Note:
This callback is executed within the context of the network stack task. It must be non-blocking and should not perform heavy processing or wait on semaphores/mutexes to avoid stalling network traffic. The user is responsible for storing these values if they need to be accessed outside the callback scope.

Uso:
Used during the initialization of the DHCP client module. You assign a function pointer of this type to the DHCP configuration structure to receive notification once the network interface is fully configured.

Esempio:

```
void my_dhcp_callback(ca_ipv4_t ip, ca_ipv4_t mask, ca_ipv4_t gw, ca_ipv4_t dns)
    {
    printf("Network configured! IP: %u.%u.%u.%u\n", ip[0], ip[1], ip[2], ip[3]);
}

// In your network initialization code:
ca_dhcp_config_t config;
config.on_bound = my_dhcp_callback;
ca_dhcp_start(&config);
```

---

ca_dhcp_cfg_t

Tipo: Struttura
File: caos/net/inc/net_cfg.h:102:105

```
typedef struct {
u32             timeout_ms;  /* discover timeout (0 = 5000 ms default)
   */
ca_dhcp_bound_t on_bound;    /* called when IP is assigned (may be NULL)
    */
} ca_dhcp_cfg_t;
```

Configuration structure used to define the behavior and callback for the caOS DHCP client.

Membri:

Nome	Tipo	Descrizione
timeout_ms	u32	Time in milliseconds to wait for a DHCP offer. If set to 0, the system defaults to 5000ms.
on_bound	ca_dhcp_bound_t	A function pointer to a callback executed once the DHCP server assigns an IP address. Can be set to NULL if no notification is required.

Note:
The structure is typically passed by value or pointer to the DHCP initialization function. It does not require dynamic allocation; it can be declared on the stack or as a global configuration object. Ensure the callback function is thread-safe as it may be executed from the network stack context.

Uso:
Use this structure when initializing the network interface to enable dynamic IP assignment. It is passed as an argument to the DHCP configuration API to customize the handshake timeout and define the application-level logic that triggers once the network is ready.

Esempio:

```
void my_on_bound_callback(u32 ip_addr) { /* Handle IP assignment */ }

ca_dhcp_cfg_t dhcp_cfg = {
    .timeout_ms = 3000,
    .on_bound = my_on_bound_callback
};

ca_net_dhcp_start(&dhcp_cfg);
```

`ca_dhcp_expire_cb_t`

Tipo: Callback
File: `caos/net/inc/dhcp_service.h:88:88`

```
typedef void (*ca_dhcp_expire_cb_t)(void)
```

A callback function type triggered when a DHCP lease expires, used to notify the application that the IP address is no longer valid.

Membri:

Nome	Tipo	Descrizione
`void`	`void`	The callback takes no arguments and returns nothing.

Note:
This callback is executed within the context of the DHCP service task. It must be kept short and non-blocking to avoid delaying network stack operations. The user is responsible for implementing logic to handle the loss of network connectivity, such as re-initializing the network interface or stopping dependent services.

Uso:
Use this type when configuring the DHCP service to monitor lease status. It is typically assigned during the initialization phase of the network stack to ensure the application can react gracefully when the assigned IP address is revoked.

Esempio:

```
void my_dhcp_expire_handler(void) { /* Log the event or signal the main loop to
    stop network tasks */ } ... ca_dhcp_config_t config = { .expire_cb =
    my_dhcp_expire_handler };
```

---

ca_dhcp_offer_t

Tipo: Struttura
File: caos/net/inc/net_proto.h:358:370

```
typedef struct {
ca_ipv4_t offered_ip;   /* yiaddr     IP offered by server */
ca_ipv4_t server_ip;    /* siaddr     next-server IP (may be 0) */
ca_ipv4_t server_id;    /* option 54     Server Identifier (use for
   REQUEST) */
ca_ipv4_t subnet_mask;  /* option 1 */
ca_ipv4_t gateway;      /* option 3 (first router) */
ca_ipv4_t dns;          /* option 6 (first DNS) */
ca_ipv4_t ntp_server;   /* option 42 (first NTP server); 0 if not
   present */
u32    lease_time;   /* option 51, seconds */
u32    t1;           /* option 58, seconds; 0 = use lease/2 */
u32    t2;           /* option 59, seconds; 0 = use lease*7/8 */
u8     msg_type;     /* DHCP_MSG_OFFER / DHCP_MSG_ACK / DHCP_MSG_NAK */
} ca_dhcp_offer_t;
```

A structure containing the network configuration parameters received from a DHCP server during the DORA process.

Membri:

Nome	Tipo	Descrizione
offered_ip	ca_ipv4_t	The IP address assigned to this client by the DHCP server.
server_ip	ca_ipv4_t	The IP address of the next server to use in bootstrap; often 0 if not applicable.
server_id	ca_ipv4_t	The unique identifier of the DHCP server (Option 54). Must be included in the subsequent DHCP REQUEST packet.
subnet_mask	ca_ipv4_t	The subnet mask for the network (Option 1).
gateway	ca_ipv4_t	The default gateway IP address (Option 3).
dns	ca_ipv4_t	The primary DNS server IP address (Option 6).
ntp_server	ca_ipv4_t	The primary NTP server IP address (Option 42). Set to 0 if the server did not provide one.
lease_time	u32	The duration of the IP lease in seconds (Option 51).
t1	u32	Renewal time (T1) in seconds. If 0, the stack defaults to lease_time / 2.
t2	u32	Rebinding time (T2) in seconds. If 0, the stack defaults to lease_time * 7 / 8.
msg_type	u8	The DHCP message type (e.g., DHCP_MSG_OFFER, DHCP_MSG_ACK, DHCP_MSG_NAK).

Note:
This structure is typically populated by the caOS network stack when a DHCP response is parsed. The application should treat this as read-only data provided by the stack. Ensure the memory is not modified while the DHCP state machine is processing the offer.

Uso:
Used by the application to inspect the network configuration offered by a DHCP server before accepting it or to verify the parameters after a successful DHCP ACK.

Esempio:

```
ca_dhcp_offer_t my_offer;

// Assuming the stack has populated the offer structure
if (my_offer.msg_type == DHCP_MSG_OFFER) {
    printf("Received IP: %d.%d.%d.%d\n",
           (my_offer.offered_ip >> 24) & 0xFF,
           (my_offer.offered_ip >> 16) & 0xFF,
           (my_offer.offered_ip >> 8) & 0xFF,
           my_offer.offered_ip & 0xFF);
}
```

---

ca_dma2d_am_t

Tipo: Enumerazione
File: caos/drivers/inc/dma2d.h:66:70

```
typedef enum {
CA_DMA2D_ALPHA_KEEP = 0,     /* the pixel's own (0xFF without an alpha
    channel) */
CA_DMA2D_ALPHA_REPLACE,      /* `alpha` */
CA_DMA2D_ALPHA_MULT          /* pixel alpha * `alpha` / 255 */
} ca_dma2d_am_t;
```

Defines how the DMA2D engine handles pixel alpha blending during image copy or conversion operations.

Membri:

Nome	Tipo	Descrizione
CA_DMA2D_ALPHA_KEEP	enum value	Uses the pixel's original alpha value (or 0xFF if no alpha channel exists). No blending is applied.
CA_DMA2D_ALPHA_REPLACE	enum value	Replaces the pixel's alpha value with a constant alpha value provided in the configuration.
CA_DMA2D_ALPHA_MULT	enum value	Calculates the new alpha as (pixel_alpha * constant_alpha) / 255. Used for fading or transparency effects.

Note:
This enum is used as a parameter in DMA2D configuration structures. It determines the mathematical operation performed on the alpha channel of pixels during hardware-accelerated transfers. Ensure the destination format supports alpha if using MULT or REPLACE modes.

Uso:
Used when configuring DMA2D blending parameters, typically passed to functions like ca_dma2d_blend or ca_dma2d_copy to define how source pixels interact with the destination buffer.

Esempio:

```
ca_dma2d_am_t alpha_mode = CA_DMA2D_ALPHA_MULT;
/* Assuming a buffer structure is prepared for a blend operation */
/* The following function call uses the alpha mode to blend two images */
ca_dma2d_blend(src_addr, dst_addr, width, height, alpha_mode,
    constant_alpha_value);
```

---

ca_dma2d_dst_t

Tipo: Struttura
File: caos/drivers/inc/dma2d.h:89:94

```
typedef struct {
void            *addr;        /* first pixel of the rectangle */
u32              stride;      /* pixels per line of the whole image */
ca_dma2d_fmt_t   fmt;         /* CA_DMA2D_ARGB8888..ARGB4444 */
u8               flags;       /* CA_DMA2D_AI | _RBS */
} ca_dma2d_dst_t;
```

Defines the destination buffer properties for DMA2D graphics operations, such as memory address, layout, and pixel format.

Membri:

Nome	Tipo	Descrizione
addr	void *	Pointer to the memory location of the first pixel of the destination rectangle. Must be aligned according to hardware requirements.
stride	u32	The number of pixels per line for the entire image buffer, not just the rectangle being drawn. Used to calculate the memory offset for the next row.
fmt	ca_dma2d_fmt_t	The pixel format of the destination buffer (e.g., ARGB8888, RGB565). Must match the hardware configuration of the display or buffer.
flags	u8	Configuration flags for the operation. CA_DMA2D_AI (Alpha Inversion) or CA_DMA2D_RBS (Red-Blue Swap) can be applied.

Note:
This struct is typically allocated on the stack by the caller before passing it to DMA2D functions. The user is responsible for ensuring the memory pointed to by 'addr' is valid and large enough to hold the operation's output. The 'stride' must be correctly set to the full width of the image buffer to avoid memory corruption during multi-line operations.

Uso:
Used as an input parameter in ca_dma2d_copy, ca_dma2d_fill, or ca_dma2d_blend functions to define where the processed pixel data should be written.

Esempio:

```
ca_dma2d_dst_t dst = {0};
dst.addr = (void *)0xD0000000; // Framebuffer start address
dst.stride = 800;              // Full width of the display
dst.fmt = CA_DMA2D_ARGB8888;
dst.flags = 0;

// Perform a fill operation using the defined destination
// ca_dma2d_fill(..., &dst, ...);
```

---

ca_dma2d_fmt_t

Tipo: Enumerazione
File: caos/drivers/inc/dma2d.h:49:61

```
typedef enum {
CA_DMA2D_ARGB8888 = 0,   /* input and output */
CA_DMA2D_RGB888,         /* input and output, 3 bytes */
CA_DMA2D_RGB565,         /* input and output */
CA_DMA2D_ARGB1555,       /* input and output */
CA_DMA2D_ARGB4444,       /* input and output */
CA_DMA2D_L8,             /* input: CLUT index */
CA_DMA2D_AL44,           /* input: alpha 4 + index 4 */
CA_DMA2D_AL88,           /* input: alpha 8 + index 8 */
CA_DMA2D_L4,             /* input: CLUT index, two per byte */
CA_DMA2D_A8,             /* input: alpha only, color fixed */
CA_DMA2D_A4              /* input: alpha only, two per byte */
} ca_dma2d_fmt_t;
```

Defines the pixel format for DMA2D (Chrom-ART) operations, specifying how color data is interpreted by the hardware.

Membri:

Nome	Tipo	Descrizione
CA_DMA2D_ARGB8888	enum	32-bit color: 8 bits each for Alpha, Red, Green, Blue.
CA_DMA2D_RGB888	enum	24-bit color: 8 bits each for Red, Green, Blue.
CA_DMA2D_RGB565	enum	16-bit color: 5 bits Red, 6 bits Green, 5 bits Blue.
CA_DMA2D_ARGB1555	enum	16-bit color: 1 bit Alpha, 5 bits each for R, G, B.
CA_DMA2D_ARGB4444	enum	16-bit color: 4 bits each for A, R, G, B.
CA_DMA2D_L8	enum	8-bit indexed color (CLUT).
CA_DMA2D_AL44	enum	8-bit: 4 bits Alpha, 4 bits index.
CA_DMA2D_AL88	enum	16-bit: 8 bits Alpha, 8 bits index.
CA_DMA2D_L4	enum	4-bit indexed color (two pixels per byte).
CA_DMA2D_A8	enum	8-bit Alpha channel only.
CA_DMA2D_A4	enum	4-bit Alpha channel only (two pixels per byte).

Note:
This enum is used to configure the DMA2D hardware registers. Ensure the format matches the source/destination buffer memory layout. Incorrect formats will result in corrupted graphical output or memory access violations.

Uso:
Used in ca_dma2d_init, ca_dma2d_copy, and ca_dma2d_fill functions to inform the DMA2D engine how to interpret the pixel data in the provided memory buffers.

Esempio:

```
#include "caos/drivers/inc/dma2d.h"

void setup_display_buffer(void) {
    /* Define the format for the frame buffer */
    ca_dma2d_fmt_t format = CA_DMA2D_RGB565;

    /* Initialize DMA2D hardware with the chosen format */
    ca_dma2d_init(format);

    /* Perform a copy operation using the configured format */
    /* ca_dma2d_copy(src, dst, x, y, w, h); */
}
```

ca_dma2d_src_t

Tipo: Struttura
File: caos/drivers/inc/dma2d.h:77:87

```
typedef struct {
const void     *addr;       /* first pixel of the rectangle */
u32             stride;     /* pixels per line of the whole image */
ca_dma2d_fmt_t  fmt;
ca_dma2d_am_t   am;
u8              alpha;      /* for REPLACE and MULT */
u8              flags;      /* CA_DMA2D_AI | _RBS | _CLUT_CPU */
u32             color;      /* 0xRRGGBB for A8/A4 */
const u32      *clut;       /* ARGB8888 entries for L8/L4/AL44/AL88,
   NULL = keep the loaded one */
u16             clut_n;     /* entries, 1..256 */
} ca_dma2d_src_t;
```

Defines the source image configuration for DMA2D hardware acceleration operations like blending or copying.

Membri:

Nome	Tipo	Descrizione
addr	const void *	Pointer to the memory address of the first pixel of the source rectangle.
stride	u32	Number of pixels per line in the source image buffer (including padding).
fmt	ca_dma2d_fmt_t	Pixel format of the source data (e.g., RGB565, ARGB8888).
am	ca_dma2d_am_t	Alpha mode: defines how the alpha channel is handled (e.g., constant, pixel-based, or combined).
alpha	u8	Constant alpha value used when the alpha mode is set to REPLACE or MULT.
flags	u8	Bitmask for hardware options: CA_DMA2D_AI (Alpha Inversion), _RBS (Red/Blue Swap), or _CLUT_CPU.
color	u32	Default color value (0xRRGGBB) used for A8/A4 formats.
clut	const u32 *	Pointer to the Color Look-Up Table (ARGB8888 entries). Set to NULL to reuse the previously loaded CLUT.
clut_n	u16	Number of entries in the CLUT, valid range is 1 to 256.

Note:
The struct is typically allocated on the stack by the caller. The DMA2D hardware reads directly from the provided 'addr'. Ensure the memory pointed to by 'addr' and 'clut' remains valid until the DMA2D operation is completed (use ca_dma2d_wait to ensure synchronization).

Uso:
Used as an input parameter in ca_dma2d_copy and ca_dma2d_blend functions to define the source image properties before triggering the hardware transfer.

Esempio:

```
ca_dma2d_src_t src = {0};
src.addr = my_image_buffer;
```

```
src.stride = 320;
src.fmt = CA_DMA2D_FMT_RGB565;
src.am = CA_DMA2D_AM_CONSTANT;
src.alpha = 255;

/* Perform a copy operation using the configured source */
ca_dma2d_copy(&src, &dst, 0, 0, 320, 240);
ca_dma2d_wait();
```

---

ca_dma_cb_t

Tipo: Callback
File: caos/drivers/inc/dma.h:131:131

```
typedef void (*ca_dma_cb_t)(void *arg)
```

A function pointer type used to define a callback function that executes when a DMA transfer completes.

Membri:

Nome	Tipo	Descrizione
arg	void *	A pointer to user-defined data or context. This is passed back to the callback function when the DMA interrupt triggers. If no context is needed, pass NULL.

Note:
The callback is executed within the context of the DMA interrupt service routine (ISR). Therefore, the function must be kept short, must not block, and should not call any RTOS functions that are not 'from-ISR' safe. The user is responsible for ensuring the memory pointed to by 'arg' remains valid until the callback is executed.

Uso:
Used when configuring a DMA channel via the caOS driver API. You assign a function matching this signature to the DMA configuration structure to handle asynchronous completion events, such as notifying a task that a buffer is ready for processing.

Esempio:

```
void my_dma_complete_handler(void *arg) { BaseType_t xHigherPriorityTaskWoken =
    pdFALSE; xSemaphoreGiveFromISR((SemaphoreHandle_t)arg, &
    xHigherPriorityTaskWoken); portYIELD_FROM_ISR(xHigherPriorityTaskWoken); }
    // Usage in setup: config.callback = my_dma_complete_handler; config.cb_arg
    = my_semaphore;
```

---

ca_dma_ch_t

Tipo: Struttura
File: caos/drivers/inc/dma.h:43:46

```
typedef struct {
u8 controller;  /* 1=DMA1, 2=DMA2, 3=BDMA (H7) / LPDMA (H5) */
u8 stream;      /* stream/channel index (0-based) */
} ca_dma_ch_t;
```

A structure representing a specific DMA hardware channel identifier used to configure and request DMA transfers.

Membri:

Nome	Tipo	Descrizione
`controller`	`u8`	The DMA controller index. Valid values: 1 for DMA1, 2 for DMA2, 3 for BDMA (H7) or LPDMA (H5). Incorrect values will cause hardware access faults.
`stream`	`u8`	The specific stream or channel index within the selected controller (0-based). Must match the hardware datasheet for the specific peripheral.

Note:
This struct is a simple descriptor and does not contain state information. It is typically passed by value to driver functions. The user is responsible for ensuring the controller/stream combination is valid for the specific STM32 chip being used.

Uso:
Used as an input parameter for DMA peripheral initialization and transfer request functions to specify which hardware resource the driver should claim.

Esempio:

```
ca_dma_ch_t my_dma_channel = { .controller = 1, .stream = 0 };

/* Example usage in a hypothetical driver setup */
ca_dma_init(my_dma_channel, DMA_PRIORITY_HIGH);
```

`ca_dma_dir_t`

Tipo: Enumerazione
File: `caos/drivers/inc/dma.h:54:57`

```
typedef enum {
DMA_DIR_M2P = 0,  /* memory     peripheral (TX) */
DMA_DIR_P2M = 1,  /* peripheral     memory  (RX) */
} ca_dma_dir_t;
```

An enumeration defining the data transfer direction for DMA operations in caOS.

Membri:

Nome	Tipo	Descrizione
`CA_DMA_DIR_M2P`	`enum constant`	Memory to Peripheral transfer. Used for transmission (TX) operations where data is read from RAM and sent to a hardware peripheral.
`CA_DMA_DIR_P2M`	`enum constant`	Peripheral to Memory transfer. Used for reception (RX) operations where data is read from a hardware peripheral and stored into RAM.

Note:
This enum is used to configure the DMA controller's data flow direction. Ensure the direction matches the hardware peripheral's capability (e.g., a UART TX register only supports M2P). Incorrect configuration will result in data not being transferred or potential bus errors.

Uso:

Used as a parameter in DMA configuration structures or initialization functions to specify the source and destination relationship for the DMA stream.

Esempio:

```
ca_dma_config_t config;
config.direction = CA_DMA_DIR_M2P;
config.buffer = my_tx_buffer;
config.size = sizeof(my_tx_buffer);
ca_dma_init(DMA1_STREAM0, &config);
```

---

ca_dma_width_t

Tipo: Enumerazione
File: caos/drivers/inc/dma.h:60:64

```
typedef enum {
DMA_WIDTH_8  = 0,  /* 8-bit      size in byte      */
DMA_WIDTH_16 = 1,  /* 16-bit     size in halfword   */
DMA_WIDTH_32 = 2,  /* 32-bit     size in word       */
} ca_dma_width_t;
```

An enumeration defining the data transfer size for DMA operations on STM32 peripherals.

Membri:

Nome	Tipo	Descrizione
CA_DMA_WIDTH_8	enum constant	Configures the DMA to transfer data in 8-bit (byte) increments.
CA_DMA_WIDTH_16	enum constant	Configures the DMA to transfer data in 16-bit (halfword) increments.
CA_DMA_WIDTH_32	enum constant	Configures the DMA to transfer data in 32-bit (word) increments.

Note:
This enum must match the data alignment of the source and destination buffers. Using an incorrect width (e.g., 32-bit width for an 8-bit peripheral register) will result in data corruption or hardware bus errors.

Uso:
Used within DMA configuration structures to define the 'data width' parameter for both source and destination transfers. It ensures the DMA controller correctly increments its internal address pointers based on the peripheral's data register size.

Esempio:

```
ca_dma_config_t dma_cfg = { .src_width = CA_DMA_WIDTH_8, .dst_width =
    CA_DMA_WIDTH_8, .mode = DMA_MODE_NORMAL };
```

---

ca_dns_hdr_t

Tipo: Struttura
File: caos/net/inc/net_proto.h:527:534

```
typedef struct {
u16 id;       /* transaction ID */
u16 flags;    /* QR|Opcode|AA|TC|RD|RA|Z|RCODE (big-endian) */
```

```
u16 qdcount;  /* question count */
u16 ancount;  /* answer count */
u16 nscount;  /* authority RR count */
u16 arcount;  /* additional RR count */
} __attribute__((packed)) ca_dns_hdr_t;
```

Represents the standard 12-byte DNS header structure used for parsing and constructing DNS network packets.

Membri:

Nome	Tipo	Descrizione
`id`	`u16`	Unique transaction identifier. Must be matched in the response to ensure the reply corresponds to the sent query.
`flags`	`u16`	Bitfield containing query/response type, opcode, and status codes. Must be handled in network byte order (big-endian).
`qdcount`	`u16`	Number of entries in the question section.
`ancount`	`u16`	Number of resource records in the answer section.
`nscount`	`u16`	Number of name server resource records in the authority section.
`arcount`	`u16`	Number of resource records in the additional section.

Note:
The struct is marked with ___attribute___((packed)) to prevent compiler padding, ensuring it maps exactly to the 12-byte wire format. Fields are in big-endian format; use htons() or ntohs() when accessing these fields on little-endian STM32 architectures.

Uso:
Used when manually crafting DNS packets or parsing incoming DNS responses from a socket buffer. Typically cast directly onto the start of a network buffer.

Esempio:

```
ca_dns_hdr_t dns_hdr;
dns_hdr.id = htons(0x1234);
dns_hdr.flags = htons(0x0100); // Standard query with recursion desired
dns_hdr.qdcount = htons(1);
dns_hdr.ancount = 0;
dns_hdr.nscount = 0;
dns_hdr.arcount = 0;
memcpy(buffer, &dns_hdr, sizeof(ca_dns_hdr_t));
```

`ca_dns_qtail_t`

Tipo: Struttura
File: `caos/net/inc/net_proto.h:537:540`

```
typedef struct {
u16 qtype;    /* DNS_TYPE_A = 1 */
u16 qclass;   /* DNS_CLASS_IN = 1 */
} __attribute__((packed)) ca_dns_qtail_t;
```

Represents the tail portion of a DNS query, specifying the resource record type and class.

Membri:

Nome	Tipo	Descrizione
qtype	u16	The type of DNS query (e.g., 1 for A record). Must be in network byte order.
qclass	u16	The class of the DNS query (e.g., 1 for IN/Internet). Must be in network byte order.

Note:
This struct is packed to ensure it maps directly to the wire format of a DNS packet. It does not contain pointers, so no dynamic memory allocation is required. The user is responsible for ensuring endianness conversion (e.g., using htons) before transmission.

Uso:
Used when constructing manual DNS query packets or parsing incoming DNS query responses to identify the requested resource type and class.

Esempio:

```
ca_dns_qtail_t query_tail;
query_tail.qtype = htons(1); // DNS_TYPE_A
query_tail.qclass = htons(1); // DNS_CLASS_IN
// Append query_tail to the DNS packet buffer before sending
```

---

ca_dns_response_t

Tipo: Struttura
File: caos/net/inc/net_proto.h:501:505

```
typedef struct {
u16     id;     /* transaction ID (matches query) */
u8      rcode; /* DNS_RCODE_* */
ipv4_t ip;     /* resolved IPv4 (first A record); zero if not found */
} ca_dns_response_t;
```

A structure representing the result of a DNS resolution request, containing the transaction ID, status code, and resolved IP address.

Membri:

Nome	Tipo	Descrizione
id	u16	The transaction ID used to match the response with the original DNS query. Must match the ID sent in the request.
rcode	u8	The DNS response code (DNS_RCODE_*). Indicates success (0) or specific error types like NXDOMAIN or Server Failure.
ip	ca_ipv4_t	The resolved IPv4 address extracted from the first A record. Set to 0.0.0.0 if the resolution failed or no record was found.

Note:
This structure is typically populated by the caOS network stack upon completion of a DNS lookup. The memory is usually managed by the stack or provided by the caller as a buffer; it does not require manual heap allocation by the user.

Uso:
Used as an output parameter in DNS resolution APIs to retrieve the resolved IP address after a query has been processed by the network stack.

Esempio:

```c
ca_dns_response_t dns_res;
if (ca_dns_resolve("example.com", &dns_res) == CA_OK) {
    if (dns_res.rcode == DNS_RCODE_NO_ERROR) {
        printf("Resolved IP: %d.%d.%d.%d\n",
               dns_res.ip.addr[0], dns_res.ip.addr[1],
               dns_res.ip.addr[2], dns_res.ip.addr[3]);
    }
}
```

---

ca_dns_rr_hdr_t

Tipo: Struttura
File: caos/net/inc/net_proto.h:543:548

```c
typedef struct {
u16 type;     /* record type */
u16 class_;   /* record class */
u32 ttl;      /* seconds to cache */
u16 rdlength; /* RDATA length in bytes; RDATA follows immediately */
} __attribute__((packed)) ca_dns_rr_hdr_t;
```

Represents the fixed-length header of a DNS Resource Record (RR) used when parsing or constructing DNS responses.

Membri:

Nome	Tipo	Descrizione
type	u16	The type of the resource record (e.g., A, AAAA, CNAME). Must be in network byte order.
class_	u16	The class of the resource record (usually 1 for IN-/Internet). Must be in network byte order.
ttl	u32	Time-to-live value in seconds, indicating how long the record can be cached. Must be in network byte order.
rdlength	u16	The length of the following RDATA field in bytes. Must be in network byte order.

Note:
This struct is packed to match the exact binary layout of DNS packets on the wire. It does not include the variable-length name field that precedes it in a real DNS packet. Fields are typically in big-endian (network) byte order, so use ntohs/ntohl when accessing them on little-endian STM32 hardware.

Uso:
Used when manually parsing a DNS response buffer or constructing a custom DNS packet. It is typically cast onto a pointer offset within a raw network buffer after skipping the variable-length domain name field.

Esempio:

```c
ca_dns_rr_hdr_t *rr_hdr = (ca_dns_rr_hdr_t *)(buffer + offset);
u16 type = ntohs(rr_hdr->type);
u32 ttl = ntohl(rr_hdr->ttl);
if (type == 1) { /* Handle A record */ }
```

---

ca_emergency_fn_t

Tipo: Callback
File: caos/inc/scheduler.h:142:142

```
typedef void (*ca_emergency_fn_t)(u32 task_idx, u32 emerg_param)
```

A function pointer type used to define a callback triggered during critical system emergency events.

Membri:

Nome	Tipo	Descrizione
task_idx	u32	The unique index of the task that was running when the emergency occurred. Used to identify the context of the failure.
emerg_param	u32	An error code or status parameter providing specific details about the nature of the emergency.

Note:
This callback is executed in a high-priority, restricted context. It must be kept extremely short, must not block, and should avoid calling complex OS services. It is typically registered during system initialization.

Uso:
Use this type when you need to define a custom handler for system-level emergencies, such as stack overflows or hardware faults, to perform emergency logging or safe state transitions.

Esempio:

```
void my_emergency_handler(u32 task_idx, u32 emerg_param) {
    /* Log the error to a non-volatile memory or toggle a safety LED */
    GPIO_Set(LED_RED_PIN);
}

/* Registering the handler */
ca_emergency_fn_t my_handler = my_emergency_handler;
```

---

ca_eth_cfg_t

Tipo: Struttura
File: caos/drivers/inc/eth.h:70:78

```
typedef struct {
u8                  phy_addr;    /* MDIO PHY address 0-31 */
bool                rmii;        /* true = RMII, false = MII */
mac_t               mac;         /* MAC address */
ca_eth_link_cb_t    link_cb;     /* task ctx: link UP/DOWN; NULL =
    ignore */
ca_eth_rx_cb_t      rx_cb;       /* ISR ctx: RX frame available; NULL =
    polling */
ca_eth_tx_done_cb_t tx_done_cb;  /* ISR ctx: TX complete; NULL = ignore
    */
ca_eth_error_cb_t   error_cb;    /* ISR ctx: DMA error; NULL = ignore */
} ca_eth_cfg_t;
```

Configuration structure used to initialize the Ethernet peripheral driver in caOS.

Membri:

Nome	Tipo	Descrizione
phy_addr	u8	The MDIO address of the external PHY chip. Valid range is 0-31. Incorrect values will prevent the driver from communicating with the PHY.
rmii	bool	Interface mode selection. Set true for RMII (Reduced Media Independent Interface) or false for MII. Must match the hardware PCB design.
mac	ca_mac_t	The 6-byte hardware MAC address assigned to the interface.
link_cb	ca_eth_link_cb_t	Callback function triggered when the link status changes. Executed in task context. Set to NULL if not needed.
rx_cb	ca_eth_rx_cb_t	Callback function triggered when a new frame is received. Executed in ISR context. If NULL, the driver assumes polling mode.
tx_done_cb	ca_eth_tx_done_cb_t	Callback function triggered after a frame transmission completes. Executed in ISR context. Set to NULL if not needed.
error_cb	ca_eth_error_cb_t	Callback function triggered on DMA or hardware errors. Executed in ISR context. Set to NULL if not needed.

Note:
The structure is typically passed by value or pointer to the driver initialization function. Ensure all callback functions are lightweight as they may execute within an ISR context. The user is responsible for the memory allocation of this struct, usually as a static or global variable.

Uso:
Use this structure to define the hardware interface parameters and event handlers before calling the Ethernet driver initialization routine.

Esempio:

```
ca_eth_cfg_t eth_config = { .phy_addr = 0, .rmii = true, .mac = {0x00, 0x80, 0
    xE1, 0x00, 0x00, 0x01}, .link_cb = my_link_handler, .rx_cb = my_rx_handler,
    .tx_done_cb = NULL, .error_cb = my_err_handler }; ca_eth_init(&eth_config);
```

---

ca_eth_error_cb_t

Tipo: Callback
File: caos/drivers/inc/eth.h:60:60

```
typedef void (*ca_eth_error_cb_t)(u32 flags)
```

A callback function pointer type triggered by the Ethernet driver when a hardware or protocol error occurs.

Membri:

Nome	Tipo	Descrizione
flags	u32	A bitmask representing the specific error conditions encountered (e.g., buffer overflow, link loss, or DMA error). Values are defined by the driver's error bitmask constants.

Note:

The callback is executed within the context of the Ethernet interrupt service routine (ISR) or a high-priority system task. It must be kept extremely short and non-blocking. Do not perform heavy processing or call blocking RTOS primitives inside this function. The user is responsible for registering this callback via the driver's configuration structure during initialization.

Uso:
Used to implement custom error handling or logging when the Ethernet peripheral encounters a fault. Assign a function pointer of this type to the error_cb field in the Ethernet configuration struct before calling the driver initialization function.

Esempio:

```
void my_eth_error_handler(u32 flags) {
    if (flags & ETH_ERR_DMA_MASK) {
        // Log DMA failure
    }
}

// During system setup:
eth_config_t config = {
    .error_cb = my_eth_error_handler
};
ca_eth_init(&config);
```

---

ca_eth_link_cb_t

Tipo: Callback
File: caos/drivers/inc/eth.h:48:48

```
typedef void (*ca_eth_link_cb_t)(bool up)
```

A callback function type triggered by the Ethernet driver whenever the physical link status changes.

Membri:

Nome	Tipo	Descrizione
up	bool	The new state of the Ethernet link. 'true' indicates the cable is connected and the link is active; 'false' indicates the link is down.

Note:
This callback is executed within the context of the Ethernet driver's interrupt service routine or a high-priority system task. Keep the implementation extremely short; do not perform blocking operations, heavy calculations, or memory allocation inside this function. The user is responsible for registering this callback via the driver's configuration structure during system initialization.

Uso:
Use this callback to monitor network connectivity in real-time. It is typically used to trigger application-level logic, such as restarting a DHCP client, notifying the TCP/IP stack to re-initialize, or updating an LED status indicator on the board.

Esempio:

```
void my_link_status_handler(bool up) {
    if (up) {
        printf("Ethernet link is UP!\n");
    } else {
        printf("Ethernet link is DOWN!\n");
    }
```

```
}

// Configuration during system setup
ca_eth_config_t eth_cfg = {
    .link_cb = my_link_status_handler
};
ca_eth_init(&eth_cfg);
```

---

ca_eth_rx_cb_t

Tipo: Callback
File: caos/drivers/inc/eth.h:52:52

```
typedef void (*ca_eth_rx_cb_t)(void)
```

A function pointer type used as a callback to notify the application when a new Ethernet frame has been received.

Membri:

Nome	Tipo	Descrizione
ca_eth_rx_cb_t	void (*)(void)	A pointer to a function that takes no arguments and returns nothing. This function is executed by the Ethernet driver's interrupt service routine or task context upon packet arrival.

Note:
This callback is executed in a high-priority context (often an ISR). Keep the implementation extremely short; do not perform blocking operations, heavy calculations, or long-running loops inside this function. If you need to process data, use the callback to signal a task (e.g., via a semaphore or queue) instead of processing the packet directly.

Uso:
Use this type when configuring the Ethernet driver to register a custom handler. It is typically passed to the driver initialization or configuration function to allow the application to react to incoming network traffic.

Esempio:

```
void my_network_handler(void) { /* Signal a task to process the packet */
    osSemaphoreRelease(eth_data_sem); } int main(void) { ca_eth_config_t config;
     config.rx_callback = my_network_handler; ca_eth_init(&config); }
```

---

ca_eth_tx_done_cb_t

Tipo: Callback
File: caos/drivers/inc/eth.h:56:56

```
typedef void (*ca_eth_tx_done_cb_t)(void)
```

A callback function pointer type triggered by the Ethernet driver when a packet transmission is successfully completed.

Membri:

Nome	Tipo	Descrizione
void	void	The callback takes no arguments and returns no value.

Note:
This callback is executed within the context of the Ethernet interrupt service routine (ISR) or a high-priority deferred task. It must be kept extremely short to avoid blocking the network stack. Do not perform blocking operations or heavy processing inside this function.

Uso:
Used to notify the application layer that a previously sent buffer is now free to be reused or deallocated. It is typically registered during the Ethernet driver initialization or configuration phase.

Esempio:

```
void my_tx_complete_handler(void) {
    // Signal a task that the buffer is free
    osSemaphoreRelease(tx_buffer_ready_sem);
}

// Registering the callback
ca_eth_config_t config = {
    .tx_done_cb = my_tx_complete_handler
};
ca_eth_init(&config);
```

---

ca_event_group_t

Tipo: Struttura
File: `caos/inc/event.h:71:76`

```
typedef struct {
volatile u32 bits;                      /**< 32 bit flags */
ca_wait_queue_t wait_queue;                 /**< Tasks waiting */
ca_spinlock_irq_t lock;                     /**< Concurrent access
    protection */
ca_event_waiter_meta_t waiters[MAX_TASK];    /**< Metadata for task
    index */
} ca_event_group_t;
```

A synchronization primitive used to manage and signal multiple event flags across different tasks.

Membri:

Nome	Tipo	Descrizione
`bits`	`volatile u32`	A bitmask where each bit represents a specific event state. Set bits indicate the event has occurred.
`wait_queue`	`ca_wait_queue_t`	Internal queue tracking tasks currently blocked waiting for specific bits to be set.
`lock`	`ca_spinlock_irq_t`	Spinlock used to ensure atomic access to the event group structure during concurrent operations.
`waiters`	`ca_event_waiter_meta_t[MAX_TASK]`	Array of metadata for each task waiting on this group, indexed by task ID.

Note:
The structure must be initialized before use. It is typically allocated statically in global memory or dynamically by the system. Ownership is managed by the kernel once passed to event API functions. Do not modify members directly; always use the provided ca_event_* API functions to ensure thread safety.

Uso:

Use this type when you need to synchronize multiple tasks based on one or more conditions. It is passed to functions like ca_event_wait() to block a task until specific bits are set, or ca_event_set() to notify waiting tasks.

Esempio:

```
ca_event_group_t my_event_group;

void init_system(void) {
    ca_event_init(&my_event_group);
}

void task_a(void *p) {
    // Wait for bit 0 or bit 1 to be set
    ca_event_wait(&my_event_group, 0x03, CA_EVENT_WAIT_ANY, CA_TIMEOUT_FOREVER);
}

void task_b(void *p) {
    // Signal that bit 0 has occurred
    ca_event_set(&my_event_group, 0x01);
}
```

---

ca_event_waiter_meta_t

Tipo: Struttura
File: caos/inc/event.h:60:65

```
typedef struct {
u32 wait_bits;        /**< Bit mask to wait for */
u8  wait_mode;        /**< CA_EVENT_WAIT_ANY or CA_EVENT_WAIT_ALL */
u8  auto_clear;       /**< If true, clear bits after match */
volatile u32 result; /**< Matched bits, set by wakeup_waiters; 0 = not
    signaled */
} ca_event_waiter_meta_t;
```

A configuration structure used to define the criteria and storage for a task waiting on an event group.

Membri:

Nome	Tipo	Descrizione
wait_bits	u32	A bitmask representing the specific events the task is interested in. Each bit corresponds to a flag in the event group.
wait_mode	u8	Determines logic: CA_EVENT_WAIT_ANY (wakes if any bit matches) or CA_EVENT_WAIT_ALL (wakes only if all bits match). Invalid values may cause undefined behavior.
auto_clear	u8	If set to 1, the matched bits are automatically cleared from the event group upon successful wake-up. If 0, bits remain set.
result	volatile u32	Output field populated by the kernel when the task is unblocked. Contains the actual bits that triggered the wake-up. 0 indicates no signal.

Note:

This structure is typically allocated on the stack of the waiting task. The 'result' field is volatile because it is modified by the kernel's scheduler/event handler context. Ensure the structure remains in scope until the task is unblocked.

Uso:
Used as a parameter in event waiting functions to specify the blocking condition. It acts as a bridge between the task's requirements and the kernel's event notification system.

Esempio:

```
ca_event_waiter_meta_t waiter = { .wait_bits = 0x03, .wait_mode =
    CA_EVENT_WAIT_ALL, .auto_clear = 1, .result = 0 };

if (ca_event_wait(my_event_group, &waiter, CA_TIMEOUT_FOREVER) == CA_OK) {
    if (waiter.result & 0x01) {
        /* Handle event bit 0 */
    }
}
```

---

ca_exti_cb_t

Tipo: Callback
File: caos/drivers/inc/exti.h:36:36

```
typedef void (*ca_exti_cb_t)(void)
```

A function pointer type used to define the callback routine executed when an external interrupt (EXTI) occurs.

Membri:

Nome	Tipo	Descrizione
void	void	The callback function takes no arguments and returns no value. It must be a standard C function or a static function within your driver/module.

Note:
The callback is executed within the context of the EXTI Interrupt Service Routine (ISR). Therefore, it must be kept extremely short and efficient. Do not perform blocking operations, heavy calculations, or use RTOS primitives that could block the system inside this callback.

Uso:
Used when registering an interrupt handler for a specific GPIO pin via the caOS EXTI driver. You assign a function pointer of this type to the configuration structure before calling the registration function.

Esempio:

```
static void my_button_handler(void) { /* Toggle an LED or set a flag */ } void
    init_exti(void) { ca_exti_config_t config = { .pin = GPIO_PIN_0, .callback =
     my_button_handler }; ca_exti_init(&config); }
```

---

ca_exti_edge_t

Tipo: Enumerazione
File: caos/drivers/inc/exti.h:29:33

```
typedef enum {
EXTI_EDGE_RISING  = 0,
EXTI_EDGE_FALLING = 1,
EXTI_EDGE_BOTH    = 2,
} ca_exti_edge_t;
```

Defines the signal transition type that triggers an external interrupt (EXTI) on an STM32 GPIO pin.

Membri:

Nome	Tipo	Descrizione
`CA_EXTI_EDGE_RISING`	`enum constant`	Interrupt triggers when the signal goes from Low to High (0 to 1).
`CA_EXTI_EDGE_FALLING`	`enum constant`	Interrupt triggers when the signal goes from High to Low (1 to 0).
`CA_EXTI_EDGE_BOTH`	`enum constant`	Interrupt triggers on both rising and falling edges.

Note:
This enum is used to configure the hardware trigger sensitivity of the EXTI controller. It does not allocate memory as it is a simple integer-based enumeration. Ensure the selected edge matches the physical signal characteristics of your connected sensor or button to avoid missed interrupts or false triggers.

Uso:
Used as a parameter in EXTI configuration functions to specify which signal edge should generate an interrupt request (IRQ) to the CPU.

Esempio:

```
ca_exti_edge_t edge_config = CA_EXTI_EDGE_FALLING;

// Example usage in a hypothetical driver configuration function
// Assuming a driver function exists to set the edge sensitivity
ca_exti_set_edge(CA_GPIO_PIN_0, edge_config);
```

`ca_fat_event_cb_t`

Tipo: Callback
File: `caos/drivers/inc/fat_service.h:39:39`

```
typedef void (*ca_fat_event_cb_t)(ca_fat_event_t ev, const char *what)
```

A callback function pointer type used to handle asynchronous file system events in the caOS FAT service.

Membri:

Nome	Tipo	Descrizione
`ev`	`ca_fat_event_t`	The specific event type that occurred (e.g., file creation, deletion, or error). Must be a valid member of the ca_fat_event_t enum.
`what`	`const char *`	A string pointer representing the file path or resource name associated with the event. Must be a null-terminated string.

Note:
The callback is executed within the context of the FAT service task. It must be non-blocking and should not perform heavy computations or call blocking RTOS primitives to avoid starving

the file system driver. The pointer 'what' is only guaranteed to be valid during the execution of the callback.

Uso:
Used to register an event listener with the FAT service. You assign a function matching this signature to the service configuration to monitor file system activity or handle errors in real-time.

Esempio:

```c
void my_fat_handler(ca_fat_event_t ev, const char *what) {
    if (ev == CA_FAT_EV_FILE_CREATED) {
        printf("New file detected: %s\n", what);
    }
}

// Usage in configuration
ca_fat_config_t config = {
    .event_cb = my_fat_handler
};
```

---

ca_fat_event_t

Tipo: Enumerazione
File: caos/drivers/inc/fat_service.h:33:37

```c
typedef enum {
CA_FAT_EV_MOUNTED = 0,    /* volume just mounted
    */
CA_FAT_EV_UNMOUNTING,     /* volume about to be unmounted: close files
    */
CA_FAT_EV_DESTRUCTIVE     /* fdisk/mkfs about to rewrite the card
    */
} ca_fat_event_t;
```

An enumeration representing lifecycle events of a FAT filesystem volume in caOS.

Membri:

Nome	Tipo	Descrizione
CA_FAT_EV_MOUNTED	enum constant	Triggered immediately after a filesystem volume is successfully mounted. Use this to initialize application-level file handles or logs.
CA_FAT_EV_UNMOUNTING	enum constant	Triggered before the volume is unmounted. You must close all open file handles immediately to prevent data corruption.
CA_FAT_EV_DESTRUCTIVE	enum constant	Triggered before a format or partition operation. All data on the volume will be lost; ensure no critical files are open.

Note:
This enum is typically passed as an argument to a callback function registered with the FAT service. The application does not allocate this; it is provided by the OS kernel as a notification of state changes.

Uso:
Use this type in your FAT event callback function to react to hardware changes (e.g., SD card insertion/removal) or administrative commands (e.g., formatting).

Esempio:

```c
void my_fat_callback(ca_fat_event_t event) {
    switch(event) {
        case CA_FAT_EV_MOUNTED:
            // Initialize file system tasks
            break;
        case CA_FAT_EV_UNMOUNTING:
            // Close all open files
            break;
        case CA_FAT_EV_DESTRUCTIVE:
            // Stop all file operations
            break;
    }
}
```

---

ca_fat_read_cb_t

Tipo: Callback
File: caos/drivers/inc/fat_service.h:97:97

```c
typedef bool (*ca_fat_read_cb_t)(const u8 *data, u32 len, void *user)
```

A callback function type used to process data chunks read from a file in the FAT filesystem.

Membri:

Nome	Tipo	Descrizione
data	const u8 *	Pointer to the buffer containing the data chunk read from the file. Do not modify this memory.
len	u32	The number of bytes available in the data buffer.
user	void *	User-defined context pointer passed through the FAT read function, useful for maintaining state.

Note:
The callback is invoked by the filesystem driver during read operations. It must return true to continue reading or false to abort the operation. The memory pointed to by 'data' is only valid for the duration of the callback execution.

Uso:
Used when reading files via ca_fat_read_enc to process file content in chunks without loading the entire file into memory.

Esempio:

```c
static bool my_read_handler(const u8 *data, u32 len, void *user) {
    /* Process the chunk of data here */
    ca_printf("Read %u bytes\n", len);
    return true; /* Continue reading */
}

void process_file(void) {
    /* Assuming file is already opened or handled by path */
    /* ca_fat_read_enc(path, offset, size, callback, user_ptr) */
    ca_fat_read_enc("test.txt", 0, 1024, my_read_handler, NULL);
}
```

---

ca_flash_key_block_hdr_t

Tipo: Struttura
File: caos/crypto/inc/keystore_service.h:48:55

```
typedef struct {
u32 size;
u8  IV[KSTORE_SVC_IV_SIZE];
u32 n_evolve;
u8  p1[KSTORE_SVC_P1_SIZE];
u16 n_key;
u8  gcm_nonce[KSTORE_SVC_NONCE_SIZE];
} ca_flash_key_block_hdr_t;    /* 4+16+4+16+2+12 = 54 byte */
```

A header structure used to manage the metadata and cryptographic context of a key block stored in non-volatile flash memory.

Membri:

Nome	Tipo	Descrizione
`size`	`u32`	Total size of the key block including the header and payload. Must match the actual stored data length to prevent buffer overflows.
`IV`	`u8[16]`	Initialization Vector used for AES-GCM decryption. Must be unique for every encryption operation to maintain security.
`n_evolve`	`u32`	Evolution counter used for key derivation or anti-rollback protection. Incremented during key updates.
`p1`	`u8[16]`	Padding or reserved field for future cryptographic parameters. Should be initialized to zero if unused.
`n_key`	`u16`	Identifier or index of the key stored within this block. Used to retrieve specific keys from the keystore.
`gcm_nonce`	`u8[12]`	GCM mode nonce. Must be unique per key block to ensure the integrity and confidentiality of the encrypted data.

Note:
This structure is typically mapped directly onto flash memory. It is not dynamically allocated by the RTOS; it is usually declared as a static struct or part of a larger flash-mapped structure. Ensure the struct is packed if the compiler adds padding, as flash layouts must be byte-aligned to the hardware specification.

Uso:
Used by the Keystore Service to verify and decrypt key blocks retrieved from flash. It is passed as a pointer to internal keystore functions when reading or writing encrypted key material.

Esempio:

```
ca_flash_key_block_hdr_t my_key_hdr = { .size = 128, .n_key = 0x01 };
// Initialize IV and nonce with secure random values before writing to flash
memcpy(my_key_hdr.IV, random_iv, KSTORE_SVC_IV_SIZE);
memcpy(my_key_hdr.gcm_nonce, random_nonce, KSTORE_SVC_NONCE_SIZE);
```

`ca_flash_key_entry_t`

Tipo: Struttura
File: `caos/crypto/inc/keystore_service.h:39:45`

```
typedef struct {
u16 key_id;
```

```
u8  label[KSTORE_SVC_LABEL_SIZE];
u8  key[KSTORE_SVC_KEY_SIZE];
u32 start;      /* KSTORE_DATE_PACK validity start (0 = no limit) */
u32 stop;       /* KSTORE_DATE_PACK validity stop  (0 = no limit) */
} ca_flash_key_entry_t;    /* 2+6+32+4+4 = 48 byte */
```

A structure representing a single cryptographic key entry stored in the non-volatile flash memory.

Membri:

Nome	Tipo	Descrizione
`key_id`	`u16`	Unique identifier for the key. Must be unique within the keystore to avoid collisions.
`label`	`u8[KSTORE_SVC_LABEL_SIZE]`	Human-readable identifier or alias for the key. Must be null-terminated if treated as a string.
`key`	`u8[KSTORE_SVC_KEY_SIZE]`	The raw cryptographic key material. Ensure this buffer is populated with secure, high-entropy data.
`start`	`u32`	Validity start timestamp in KSTORE_DATE_PACK format. Set to 0 to indicate no start restriction.
`stop`	`u32`	Validity end timestamp in KSTORE_DATE_PACK format. Set to 0 to indicate no expiration.

Note:
This structure is typically used for direct flash mapping. Ensure the struct is packed if the compiler alignment settings differ from the expected 48-byte layout. The application is responsible for allocating this memory, usually on the stack or as a static global before passing it to the keystore service functions.

Uso:
Use this type when interacting with the keystore service to add, update, or retrieve cryptographic keys from the device's secure flash storage.

Esempio:

```
ca_flash_key_entry_t my_key = { .key_id = 0x01, .label = "AES_KEY_01", .key = {0
    x01, 0x02, 0x03}, .start = 0, .stop = 0 };
```

`ca_fmac_filter_t`

Tipo: Struttura
File: `caos/drivers/inc/fmac.h:54:61`

```
typedef struct {
const i16 *b;       /* feed-forward coefficients b[0..nb-1] */
u32        nb;
const i16 *a;       /* feedback a[1..na] (a[0] is not passed), NULL for a
    FIR */
u32        na;
u32        gain;    /* output x 2^gain, 0..7 */
bool       clip;    /* saturate the output instead of wrapping */
} ca_fmac_filter_t;
```

Configuration structure for the hardware Filter-MAC (FMAC) unit, used to define filter coefficients and processing behavior.

Membri:

Nome	Tipo	Descrizione
b	const i16 *	Pointer to the feed-forward coefficients array. Must be valid for the duration of the filter operation.
nb	u32	Number of feed-forward coefficients (length of array b).
a	const i16 *	Pointer to feedback coefficients array. Set to NULL for FIR filters; required for IIR filters.
na	u32	Number of feedback coefficients. Ignored if a is NULL.
gain	u32	Output scaling factor applied as x * 2^gain. Valid range is 0 to 7.
clip	bool	If true, saturates the output at the maximum/minimum value instead of allowing integer overflow/wrapping.

Note:
The structure is passed by value to the initialization function. The coefficient arrays (b and a) must reside in memory accessible by the FMAC peripheral (typically static or global memory) and must not be modified while the filter is running.

Uso:
Used to configure the FMAC hardware before processing data streams. Pass this struct to ca_fmac_init to set up the filter parameters.

Esempio:

```
static const i16 my_b_coeffs[] = {100, 200, 100};
ca_fmac_filter_t my_filter = {
    .b = my_b_coeffs,
    .nb = 3,
    .a = NULL,
    .na = 0,
    .gain = 1,
    .clip = true
};

// Initialize the FMAC hardware with the defined filter
ca_fmac_init(&my_filter);
```

---

ca_gpio_mode_t

Tipo: Enumerazione
File: caos/drivers/inc/gpio_driver.h:64:69

```
typedef enum {
CA_GPIO_MODE_INPUT = 0,
CA_GPIO_MODE_OUTPUT,
GPIO_MODE_AF,
CA_GPIO_MODE_ANALOG,
} ca_gpio_mode_t;
```

Defines the electrical operating mode for a GPIO pin (e.g., input, output, analog, or alternate function).

Membri:

Nome	Tipo	Descrizione
CA_GPIO_MODE_INPUT	enum value	Configures the pin as a digital input. Used to read external signals.
CA_GPIO_MODE_OUTPUT	enum value	Configures the pin as a digital output. Used to drive external components like LEDs.
GPIO_MODE_AF	enum value	Configures the pin for Alternate Function (e.g., UART, SPI, or PWM peripheral usage).
CA_GPIO_MODE_ANALOG	enum value	Configures the pin for analog mode. Required for ADC or DAC operations.

Note:
This enum is passed to the ca_gpio_configure function. Ensure the pin is initialized via ca_gpio_init_port before attempting to change its mode. Incorrect mode selection may cause electrical conflicts if the pin is connected to an active external signal.

Uso:
Use this type when configuring GPIO pins during system startup or peripheral setup. It is the primary argument for the mode parameter in the ca_gpio_configure function.

Esempio:

```c
#include "caos/drivers/inc/gpio_driver.h"

void setup_led(void) {
    /* Initialize the GPIO port first */
    ca_gpio_init_port(0);

    /* Configure pin 5 on port 0 as an output to drive an LED */
    ca_gpio_configure(0, 5, CA_GPIO_MODE_OUTPUT);

    /* Turn the LED on */
    ca_gpio_write(0, 5, 1);
}
```

---

ca_gpio_otype_t

Tipo: Enumerazione
File: caos/drivers/inc/gpio_driver.h:74:77

```c
typedef enum {
CA_GPIO_OTYPE_PP = 0,   /*!< Push-pull */
CA_GPIO_OTYPE_OD,       /*!< Open-drain */
} ca_gpio_otype_t;
```

Defines the output configuration mode for a GPIO pin, determining how the pin drives the electrical signal.

Membri:

Nome	Tipo	Descrizione
CA_GPIO_OTYPE_PP	enum constant	Push-pull configuration. The pin actively drives both high (VCC) and low (GND). Use for standard digital outputs.
CA_GPIO_OTYPE_OD	enum constant	Open-drain configuration. The pin can only drive low (GND) or float (high impedance). Requires an external pull-up resistor to drive high.

Note:

This enum is typically passed as a member of a GPIO configuration structure. It does not require manual allocation as it is a simple enumeration type. Ensure the hardware circuit matches the selected mode, especially for Open-drain, which requires an external pull-up resistor to function correctly as a high-level output.

Uso:
Used during the initialization phase of a GPIO peripheral to configure the output driver type. It is passed to the GPIO configuration function to set the hardware registers of the STM32 GPIO port.

Esempio:

```
ca_gpio_config_t my_pin_config = {
    .pin = CA_GPIO_PIN_5,
    .mode = CA_GPIO_MODE_OUTPUT,
    .otype = CA_GPIO_OTYPE_PP,
    .speed = CA_GPIO_SPEED_HIGH
};
ca_gpio_init(GPIOA, &my_pin_config);
```

---

ca_gpio_pin_t

Tipo: Enumerazione
File: `caos/drivers/inc/gpio_driver.h:42:59`

```
typedef enum {
CA_GPIO_PIN_0 = 0,
CA_GPIO_PIN_1,
GPIO_PIN_2,
CA_GPIO_PIN_3,
GPIO_PIN_4,
CA_GPIO_PIN_5,
GPIO_PIN_6,
CA_GPIO_PIN_7,
GPIO_PIN_8,
CA_GPIO_PIN_9,
GPIO_PIN_10,
CA_GPIO_PIN_11,
GPIO_PIN_12,
CA_GPIO_PIN_13,
GPIO_PIN_14,
CA_GPIO_PIN_15,
} ca_gpio_pin_t;
```

An enumeration representing the 16 individual pins available on a single STM32 GPIO port.

Membri:

Nome	Tipo	Descrizione
`CA_GPIO_PIN_0 to CA_GPIO_PIN_15`	`enum constant`	Represents the specific pin index (0-15) on a GPIO port. Using an invalid value outside this range will result in undefined behavior in driver functions.

Note:
This enum is used to identify a specific pin within a port. Note that while some constants are prefixed with 'CA_', others are not; ensure you use the provided constants consistently. These values are typically passed as arguments to GPIO configuration and state-setting functions.

Uso:

Use this type when configuring pin modes (input/output/alternate function) or when reading/writing the digital state of a specific pin on a port.

Esempio:

```
ca_gpio_pin_t my_pin = CA_GPIO_PIN_5;

// Example usage in a hypothetical driver function
ca_gpio_set_mode(GPIOA, my_pin, CA_GPIO_MODE_OUTPUT);
ca_gpio_write(GPIOA, my_pin, CA_GPIO_STATE_HIGH);
```

---

ca_gpio_port_t

Tipo: Enumerazione
File: `caos/drivers/inc/gpio_driver.h:25:37`

```
typedef enum {
CA_GPIO_PORT_A = 0,
CA_GPIO_PORT_B,
GPIO_PORT_C,
CA_GPIO_PORT_D,
GPIO_PORT_E,
CA_GPIO_PORT_F,
GPIO_PORT_G,
CA_GPIO_PORT_H,
GPIO_PORT_I,
CA_GPIO_PORT_J,
GPIO_PORT_K,
} ca_gpio_port_t;
```

An enumeration representing the available physical GPIO ports on the STM32 microcontroller.

Membri:

Nome	Tipo	Descrizione
`CA_GPIO_PORT_A through CA_GPIO_PORT_K`	`enum constant`	Represents the specific hardware port GPIOA, GPIOB). Ensure the selected port e on your specific STM32 chip variant to avoic defined behavior.

Note:
This enum is used to abstract hardware-specific port identifiers. It is typically passed by value to driver functions. Note that some ports may not be physically bonded or available on smaller STM32 packages; verify your datasheet before use.

Uso:
Used as a parameter in GPIO configuration and control functions to specify which port the operation should target. It is the primary way to identify a port in the caOS GPIO driver API.

Esempio:

```
ca_gpio_port_t my_port = CA_GPIO_PORT_A;

// Example usage in a hypothetical driver function
ca_gpio_init(my_port, 5, CA_GPIO_MODE_OUTPUT);
```

---

ca_gpio_pull_t

Tipo: Enumerazione
File: caos/drivers/inc/gpio_driver.h:92:96

```
typedef enum {
CA_GPIO_NOPULL = 0,
CA_GPIO_PULLUP,
GPIO_PULLDOWN,
} ca_gpio_pull_t;
```

An enumeration defining the internal resistor configuration for an STM32 GPIO pin.

Membri:

Nome	Tipo	Descrizione
CA_GPIO_NOPULL	enum constant	Disables both pull-up and pull-down resistors. The pin is in a high-impedance state.
CA_GPIO_PULLUP	enum constant	Connects an internal pull-up resistor to VDD. Used to keep the line high when no external device is driving it.
GPIO_PULLDOWN	enum constant	Connects an internal pull-down resistor to VSS. Used to keep the line low when no external device is driving it.

Note:
This enum is used to configure the electrical state of a GPIO pin during initialization. Ensure the chosen pull configuration matches your hardware schematic to avoid floating inputs or short circuits.

Uso:
Pass this value to the GPIO configuration structure (typically passed to a driver initialization function) to define the default state of a pin when it is not being actively driven by an external source.

Esempio:

```
ca_gpio_config_t my_pin_config = { .pin = 5, .mode = CA_GPIO_MODE_INPUT, .pull =
    CA_GPIO_PULLUP };
```

---

ca_gpio_speed_t

Tipo: Enumerazione
File: caos/drivers/inc/gpio_driver.h:82:87

```
typedef enum {
CA_GPIO_SPEED_LOW = 0,
CA_GPIO_SPEED_MED,
GPIO_SPEED_HIGH,
CA_GPIO_SPEED_VERY_HIGH,
} ca_gpio_speed_t;
```

An enumeration defining the output slew rate (speed) of an STM32 GPIO pin.

Membri:

Nome	Tipo	Descrizione
CA_GPIO_SPEED_LOW	enum constant	Lowest slew rate. Reduces electromagnetic interference (EMI) and power consumption. Use for low-frequency signals like status LEDs or buttons.
CA_GPIO_SPEED_MED	enum constant	Medium slew rate. A balance between signal integrity and power consumption.
GPIO_SPEED_HIGH	enum constant	High slew rate. Used for communication interfaces like SPI or UART where faster rise/fall times are required.
CA_GPIO_SPEED_VERY_HIGH	enum constant	Maximum slew rate. Use only for high-speed peripherals. Note: High speeds can cause signal ringing and increased noise.

Note:
This enum is used to configure the GPIO_OSPEEDR register in the STM32 hardware. Choosing a speed higher than necessary can increase noise on the PCB; always select the lowest speed that meets your signal timing requirements.

Uso:
Used within the GPIO configuration structure when initializing a pin via the driver's setup functions to define how fast the pin can toggle.

Esempio:

```
ca_gpio_cfg_t my_pin_cfg = {
    .mode = CA_GPIO_MODE_OUTPUT,
    .speed = CA_GPIO_SPEED_MED,
    .pull = CA_GPIO_PULL_NONE
};
ca_gpio_init(GPIOA, GPIO_PIN_5, &my_pin_cfg);
```

---

ca_hash_algo_t

Tipo: Enumerazione
File: caos/drivers/inc/hash.h:26:29

```
typedef enum {
HASH_ALGO_SHA1   = 0,
HASH_ALGO_SHA256 = 1,
} ca_hash_algo_t;
```

An enumeration defining the supported cryptographic hashing algorithms available in the caOS hardware abstraction layer.

Membri:

Nome	Tipo	Descrizione
CA_HASH_ALGO_SHA1	enum constant	Selects the SHA-1 hashing algorithm. Note: SHA-1 is considered cryptographically insecure for modern security applications.
CA_HASH_ALGO_SHA256	enum constant	Selects the SHA-256 hashing algorithm. This is the recommended standard for most data integrity and security tasks.

Note:
This enum is used to configure the hardware hashing engine. Ensure the selected algorithm is supported by the specific STM32 hardware variant being used, as some older chips may not support SHA-256.

Uso:
Pass this type as an argument to the initialization or configuration functions of the caOS HASH driver to specify which algorithm the hardware peripheral should execute.

Esempio:

```
ca_hash_algo_t selected_algo = CA_HASH_ALGO_SHA256;

// Configure the hash peripheral with the chosen algorithm
ca_hash_init(selected_algo);

// Proceed to process data buffers
ca_hash_update(data_buffer, buffer_size);
```

---

ca_hash_ctx_t

Tipo: Struttura
File: `caos/drivers/inc/hash.h:45:52`

```
typedef struct {
ca_hash_algo_t algo;
bool started;               /* an update has begun buffering
    */
bool error;                 /* allocation failed -> final() fails
    */
u32  len;                   /* bytes buffered so far
    */
u32  cap;                   /* current buf capacity
    */
u8  *buf;                   /* heap buffer (ca_memory_allocate)
    */
} ca_hash_ctx_t;
```

Context structure used to manage incremental hashing operations (e.g., SHA256) in caOS.

Membri:

Nome	Tipo	Descrizione
`algo`	`ca_hash_algo_t`	The specific hashing algorithm selected for the operation.
`started`	`bool`	Flag indicating if the hashing process has been initialized and is currently buffering data.
`error`	`bool`	Error flag set if internal heap allocation fails; if true, subsequent calls to final() will fail.
`len`	`u32`	Number of bytes currently stored in the internal buffer.
`cap`	`u32`	Total capacity of the allocated buffer.
`buf`	`u8 *`	Pointer to the heap-allocated memory buffer used to store data chunks before hashing.

Note:
The context must be initialized via ca_hash_ctx_init before use. The buffer is dynamically allocated on the heap using ca_memory_allocate; therefore, the user must call ca_hash_ctx_free to prevent memory leaks once the hashing operation is complete.

Uso:
Used for hashing large data streams that do not fit into memory at once. The typical pattern is: initialize context, call update repeatedly with data chunks, and finally call final to retrieve the digest.

Esempio:

```
ca_hash_ctx_t ctx;
u8 data_chunk[] = {0x01, 0x02, 0x03};
u8 digest[32];

/* Initialize the hashing context */
ca_hash_ctx_init(&ctx, CA_HASH_SHA256);

/* Feed data into the context */
ca_hash_ctx_update(&ctx, data_chunk, sizeof(data_chunk));

/* Finalize the hash and store result in digest */
ca_hash_ctx_final(&ctx, digest);

/* Clean up allocated memory */
ca_hash_ctx_free(&ctx);
```

---

ca_hmac_sha256_ctx_t

Tipo: Struttura
File: caos/crypto/inc/kdf.h:36:39

```
typedef struct {
ca_sha256_ctx_t inner;
ca_sha256_ctx_t outer;      /* already keyed with K ^ opad */
} ca_hmac_sha256_ctx_t;
```

Context structure for performing HMAC-SHA256 operations, holding the internal state for the inner and outer hash calculations.

Membri:

Nome	Tipo	Descrizione
inner	ca_sha256_ctx_t	Context for the inner SHA256 hash calculation, initialized with the key XORed with the ipad.
outer	ca_sha256_ctx_t	Context for the outer SHA256 hash calculation, initialized with the key XORed with the opad.

Note:
This structure is used by the HMAC-SHA256 API. It is typically allocated on the stack by the caller. The user must initialize it using ca_hmac_sha256_init before passing it to update or final functions. It does not require explicit freeing as it contains no dynamic pointers.

Uso:
Use this structure when you need to compute an HMAC-SHA256 signature over a stream of data. It is passed as the first argument to ca_hmac_sha256_init, ca_hmac_sha256_update, and ca_hmac_sha256_final.

Esempio:

```
#include "caos/crypto/inc/kdf.h"

void compute_hmac(uint8_t *key, size_t key_len, uint8_t *data, size_t data_len)
    {
    ca_hmac_sha256_ctx_t ctx;
    uint8_t digest[32];

    /* Initialize the HMAC context with the provided key */
    ca_hmac_sha256_init(&ctx, key, key_len);

```

```
    /* Process the data stream */
    ca_hmac_sha256_update(&ctx, data, data_len);

    /* Finalize the HMAC and store the result in digest */
    ca_hmac_sha256_final(&ctx, digest);
}
```

---

ca_http_conn_t

Tipo: Struttura
File: caos/net/inc/http_service.h:134:138

```
typedef struct {
u8                  rx[HTTP_REQ_BUF_SIZE];
u8                  tx[HTTP_RESP_BUF_SIZE];
ca_http_response_t  resp;
} ca_http_conn_t;
```

Represents a single HTTP connection state, including buffers for incoming requests and outgoing responses.

Membri:

Nome	Tipo	Descrizione
rx	u8[HTTP_REQ_BUF_SIZE]	Buffer for incoming HTTP request data. Must be large enough to hold the full request header.
tx	u8[HTTP_RESP_BUF_SIZE]	Buffer for outgoing HTTP response data. Used by ca_http_resp_* functions to build the response.
resp	ca_http_response_t	Structure tracking the current response state (e.g., status code, headers, chunking state).

Note:
This structure is typically allocated by the application or the HTTP server task. It is passed to ca_http_serve_connection to process a specific client request. Ensure the buffers are not accessed by other tasks while the HTTP service is actively using the connection.

Uso:
Used when implementing a custom HTTP server or handling specific HTTP requests. It is passed as a pointer to ca_http_serve_connection to handle the lifecycle of a single HTTP transaction.

Esempio:

```
ca_http_conn_t my_conn;

void handle_client(void) {
    /* Initialize connection buffers to zero before use */
    ca_memzero(&my_conn, sizeof(ca_http_conn_t));

    /* Process the connection using the caOS HTTP service */
    /* Note: 'socket_fd' would be obtained from a TCP accept call */
    ca_http_serve_connection(socket_fd, &my_conn);
}
```

---

ca_http_handler_fn

Tipo: Callback
File: caos/net/inc/http_service.h:108:108

```
typedef void (*ca_http_handler_fn)(const ca_http_request_t *req,
```

A callback function type used to process incoming HTTP requests in the caOS web server.

Membri:

Nome	Tipo	Descrizione
`req`	`const ca_http_request_t *`	Pointer to the structure containing the HTTP request details (method, URI, headers, body). Must not be modified.

Note:
This callback is invoked by the HTTP service thread when a valid request is received. The implementation must be thread-safe and should return quickly to avoid blocking the HTTP service. You are responsible for using ca_http_resp_* functions to send the response back to the client.

Uso:
Used when implementing custom HTTP endpoints. You define a function matching this signature and register it with the HTTP service (if the service provides a registration mechanism) to handle specific URI paths.

Esempio:

```
void my_http_handler(const ca_http_request_t *req) {
    /* Check the request and send a response */
    ca_http_resp_begin(req, 200);
    ca_http_resp_puts(req, "Hello from caOS!");
    ca_http_resp_flush(req);
}
```

`ca_http_io_t`

Tipo: Struttura
File: `caos/net/inc/http_service.h:52:56`

```
typedef struct {
bool  (*send)(void *io, const u8 *data, u16 len);
i32   (*recv)(void *io, u8 *buf, u16 len, u32 timeout_ms);
void   *io;
} ca_http_io_t;
```

An abstraction layer (interface) used by the HTTP service to perform read/write operations on a generic I/O stream.

Membri:

Nome	Tipo	Descrizione
send	bool (*)(void *io, const u8 *data, u16 len)	Function pointer to send data. Returns true on success, false on failure. Must handle the provided length.
recv	i32 (*)(void *io, u8 *buf, u16 len, u32 timeout_ms)	Function pointer to receive data. Returns the number of bytes read, or a negative value on error/timeout.
io	void *	Opaque pointer to the underlying hardware or software context (e.g., a socket handle or UART instance) passed to the callbacks.

Note:
This struct is not managed by caOS; the user must allocate it (usually on the stack or as a static global) and populate the function pointers. The 'io' member acts as the 'this' pointer for the callbacks. Ensure the callbacks are thread-safe if the HTTP service is accessed by multiple tasks.

Uso:
Used when configuring or initiating an HTTP connection or service where the underlying transport is not fixed (e.g., switching between UART, TCP, or TLS). It allows the HTTP logic to remain agnostic of the physical transport layer.

Esempio:

```c
/* Example of defining a custom I/O interface for a hypothetical stream */

bool my_send(void *io, const u8 *data, u16 len) {
    /* Implementation using ca_uart_dma_send or similar */
    return true;
}

i32 my_recv(void *io, u8 *buf, u16 len, u32 timeout_ms) {
    /* Implementation using ca_uart_dma_recv or similar */
    return (i32)len;
}

void setup_http_io(void) {
    ca_http_io_t my_io_interface;
    my_io_interface.send = my_send;
    my_io_interface.recv = my_recv;
    my_io_interface.io = NULL; /* Pointer to specific hardware context */

    /* The interface is now ready to be passed to ca_http service functions */
}
```

---

ca_http_request_t

Tipo: Struttura
File: caos/net/inc/http_service.h:37:43

```c
typedef struct {
u8   method;                    /* HTTP_METHOD_* */
char uri[HTTP_URI_MAX];         /* path, es. "/status" */
```

```
char query[HTTP_QUERY_MAX];      /* query string dopo '?', es. "state=on" */
const u8 *body;                  /* puntatore nel rx buffer (POST body) */
u16  body_len;
} ca_http_request_t;
```

Represents an incoming HTTP request structure used by the caOS HTTP service to pass request details to handlers.

Membri:

Nome	Tipo	Descrizione
`method`	`u8`	The HTTP verb (e.g., GET, POST). Must be one of the HTTP_METHOD_* constants. Invalid values will cause the handler to fail to identify the request type.
`uri`	`char[]`	The target path of the request (e.g., '/api/data'). Must be null-terminated. Exceeding HTTP_URI_MAX will cause buffer overflow or truncation.
`query`	`char[]`	The query string parameters following the '?' character. Must be null-terminated. If empty, should be an empty string.
`body`	`const u8 *`	A pointer to the raw request payload located in the system RX buffer. Do not modify this memory as it is managed by the network stack.
`body_len`	`u16`	The length of the data pointed to by body in bytes. Must match the actual payload size to prevent reading out-of-bounds memory.

Note:
This struct is typically populated by the caOS HTTP service layer before calling a registered callback. The memory for the 'body' pointer is owned by the network stack and is only valid for the duration of the callback execution. Do not store this pointer for asynchronous processing.

Uso:
Used in HTTP callback functions registered with the caOS HTTP service. When a client sends a request, the service fills this struct and passes it to your handler to determine the requested resource and process any provided data.

Esempio:

```
void my_http_handler(ca_http_request_t *req) {
    if (req->method == HTTP_METHOD_GET) {
        if (strcmp(req->uri, "/status") == 0) {
            // Handle status request
        }
    } else if (req->method == HTTP_METHOD_POST) {
        // Process req->body of length req->body_len
    }
}
```

`ca_http_response_t`

Tipo: Struttura
File: `caos/net/inc/http_service.h:46:62`

```c
typedef struct {
u8  *buf;           /* output buffer (provided by the service) */
u16  size;          /* buffer size */
u16  len;           /* bytes written so far */
u16  cl_pos;        /* offset of the Content-Length placeholder in the
   buf (0 = not used) */
u16  hdr_end;       /* offset where the body starts (after \r\n\r\n) */
bool head_only;     /* true if HEAD method: skip body */
/*        chunked transfer (RFC 7230  4 .1)


* Set by ca_http_resp_begin_chunked(). In this mode the body is NOT
* accumulated: every flush leaves the buffer as a self-contained chunk
   on
* the wire, so the response is not bounded by `size`. Content-Length is
* absent by definition -- which is the whole point: it cannot be patched
* afterwards if the body was never held in memory. */
bool chunked;       /* true = emit chunks instead of one buffered body
   */
u8   conn;          /* TCP handle used to flush chunks (0 = none) */
bool failed;        /* a flush failed: stop writing, close the
   connection */
} ca_http_response_t;
```

A structure representing the state and buffer management for an outgoing HTTP response in the caOS network stack.

Membri:

Nome	Tipo	Descrizione
`buf`	`u8*`	Pointer to the memory buffer where the HTTP response headers and body are stored. Managed by the service.
`size`	`u16`	Total capacity of the buffer in bytes. Must not be exceeded to prevent memory corruption.
`len`	`u16`	Current number of bytes written into the buffer.
`cl_pos`	`u16`	Offset index for the Content-Length header value. If 0, the header is not being tracked.
`hdr_end`	`u16`	Offset index marking the end of the HTTP header section (after the double CRLF).
`head_only`	`bool`	If true, the service will omit the response body, typically used for HEAD requests.
`chunked`	`bool`	If true, enables chunked transfer encoding, allowing streaming of data without a pre-defined Content-Length.
`conn`	`u8`	The TCP connection handle associated with this response. 0 indicates no active connection.
`failed`	`bool`	Flag indicating a transmission error occurred. If true, the response process should be aborted.

Note:
This structure is typically allocated by the caOS HTTP service layer. Users should not manually modify the internal offsets (cl_pos, hdr_end) unless using specific service helper functions. When chunked mode is enabled, the buffer is flushed incrementally, meaning the total response size is not limited by the buffer size.

Uso:

Used by HTTP handler callbacks to construct and send responses to clients. It is passed as a parameter to functions that write headers or body content to the network.

Esempio:

```
ca_http_response_t resp;
resp.buf = my_buffer;
resp.size = 512;
resp.len = 0;
resp.chunked = false;
resp.head_only = false;
resp.failed = false;
// Proceed to populate headers and body using service-provided functions
```

---

ca_http_route_t

Tipo: Struttura
File: caos/net/inc/http_service.h:78:82

```
typedef struct {
const char         *uri;      /* es. "/status"     confronto esatto */
u8                  methods;  /* bitmask HTTP_METHOD_* accettati */
ca_http_handler_fn handler;
} ca_http_route_t;
```

Defines a mapping between a specific URI path and a callback function to handle HTTP requests.

Membri:

Nome	Tipo	Descrizione
uri	const char *	The exact string path to match (e.g., '/api/data'). Must be a null-terminated string.
methods	u8	A bitmask of allowed HTTP methods (e.g., HTTP_METHOD_GET | HTTP_METHOD_POST). If set to 0, the route will not trigger.
handler	ca_http_handler_fn	The function pointer to be executed when the URI and method match. Must match the signature defined by ca_http_handler_fn.

Note:
The struct is typically defined as a static constant array to save RAM. The HTTP service iterates through these routes to find a match; ensure the URI strings are unique to avoid shadowing.

Uso:
Used to register endpoints with the caOS HTTP server. You pass an array of these structs to the HTTP service initialization or registration function to define the API surface of your device.

Esempio:

```
static void my_handler(ca_http_req_t *req) { /* Handle request */ } static const
    ca_http_route_t my_routes[] = { { .uri = "/status", .methods =
   HTTP_METHOD_GET, .handler = my_handler } };
```

---

ca_i2c_config_t

Tipo: Struttura
File: caos/drivers/inc/i2c.h:28:32

```
typedef struct {
u8              i2c_num;    /* 1, 2, 3 */
ca_i2c_speed_t  speed;
u16             irq_req;    /* I2C_EN_IRQ to enable EV IRQ+NVIC, 0 =
   polling only */
} ca_i2c_config_t;
```

Configuration structure used to initialize an I2C peripheral instance in caOS.

Membri:

Nome	Tipo	Descrizione
i2c_num	u8	The hardware peripheral index (e.g., 1, 2, or 3). Must match the physical STM32 I2C peripheral mapping.
speed	ca_i2c_speed_t	The desired bus clock frequency (e.g., standard mode 100kHz or fast mode 400kHz). Ensure the value is supported by the hardware clock configuration.
irq_req	u16	Interrupt request flags. Set to I2C_EN_IRQ to enable event-driven interrupts and NVIC configuration, or 0 to force blocking polling mode.

Note:
This structure is typically passed by value to the initialization function. It does not require dynamic allocation; it is usually declared on the stack or as a static constant. The user is responsible for ensuring the i2c_num corresponds to a valid, non-conflicting peripheral on the specific STM32 target.

Uso:
Use this structure as an argument when calling the driver initialization function to configure the I2C bus parameters before performing any read or write operations.

Esempio:

```
ca_i2c_config_t i2c_cfg = { .i2c_num = 1, .speed = I2C_SPEED_400K, .irq_req =
   I2C_EN_IRQ }; ca_i2c_init(&i2c_cfg);
```

---

ca_i2c_slave_ctx_t

Tipo: Struttura
File: caos/drivers/inc/i2c.h:38:47

```
typedef struct {
u8                  addr;
u8                  rx_buf[64];
u8                  rx_idx;
u8                  tx_buf[64];
u8                  tx_len;
u8                  tx_idx;
bool                is_slave;
ca_i2c_slave_rx_cb  rx_cb;
} ca_i2c_slave_ctx_t;
```

A configuration and state structure used to manage an I2C slave peripheral instance in caOS.

Membri:

Nome	Tipo	Descrizione
`addr`	`u8`	The 7-bit I2C slave address. Must be unique on the bus to avoid collisions.
`rx_buf`	`u8[64]`	Internal buffer for incoming data. Fixed size of 64 bytes; do not overflow.
`rx_idx`	`u8`	Current write index for the receive buffer. Managed by the driver.
`tx_buf`	`u8[64]`	Buffer containing data to be sent to the master. Fixed size of 64 bytes.
`tx_len`	`u8`	Total number of bytes currently available in tx_buf to be transmitted.
`tx_idx`	`u8`	Current read index for the transmit buffer. Managed by the driver.
`is_slave`	`bool`	Flag to identify the operating mode. Must be set to true for slave operation.
`rx_cb`	`ca_i2c_slave_rx_cb`	Callback function pointer triggered when data is received. Can be NULL if not needed.

Note:
The structure must be zero-initialized before use. The user is responsible for allocating this struct, typically as a static global variable to ensure it persists for the lifetime of the driver. The driver manages the indices (rx_idx, tx_idx) internally during I2C interrupts.

Uso:
Used to initialize an I2C peripheral in slave mode. Pass a pointer to this struct to the driver initialization function to register the slave address and the callback for data processing.

Esempio:

```
static ca_i2c_slave_ctx_t my_i2c_slave = {0};

void my_rx_handler(u8 *data, u8 len) {
    // Process received data
}

void setup_slave(void) {
    my_i2c_slave.addr = 0x42;
    my_i2c_slave.is_slave = true;
    my_i2c_slave.rx_cb = my_rx_handler;
    ca_i2c_init(&my_i2c_slave);
}
```

`ca_i2c_slave_rx_cb`

Tipo: Callback
File: `caos/drivers/inc/i2c.h:35:35`

```
typedef void (*ca_i2c_slave_rx_cb)(u8 num, const u8 *buf, u8 len)
```

A callback function pointer type triggered by the I2C driver when the MCU receives data as a slave device.

Membri:

Nome	Tipo	Descrizione
num	u8	The index or ID of the I2C peripheral instance that received the data. Must match the hardware peripheral index.
buf	const u8 *	Pointer to the buffer containing the received data. This memory is managed by the driver and is read-only.
len	u8	The number of bytes received in the current transaction. Valid range is 0 to the maximum buffer size configured for the I2C slave.

Note:
This callback is executed in the context of the I2C interrupt service routine. Keep the logic inside this function extremely brief; do not perform blocking operations or heavy processing. The buffer pointer is only guaranteed to be valid for the duration of the callback execution.

Uso:
Used to register a custom handler with the I2C slave driver. You assign a function of this type to the driver's configuration structure before initializing the I2C peripheral to process incoming data packets asynchronously.

Esempio:

```
void my_i2c_rx_handler(u8 num, const u8 *buf, u8 len) {
    if (len > 0) {
        process_data(buf[0]);
    }
}

// During initialization
ca_i2c_config_t config = {
    .slave_rx_cb = my_i2c_rx_handler
};
ca_i2c_init(0, &config);
```

---

ca_i2c_speed_t

Tipo: Enumerazione
File: caos/drivers/inc/i2c.h:23:26

```
typedef enum __attribute__ ((packed)) {
I2C_SPEED_STANDARD = 0,   /* 100 kHz */
I2C_SPEED_FAST     = 1,   /* 400 kHz */
} ca_i2c_speed_t;
```

An enumeration defining the clock frequency modes for I2C communication.

Membri:

Nome	Tipo	Descrizione
CA_I2C_SPEED_STANDARD	enum constant	Sets the I2C bus speed to 100 kHz. Use this for standard-mode devices or when long wires are used.
CA_I2C_SPEED_FAST	enum constant	Sets the I2C bus speed to 400 kHz. Use this for high-speed peripherals to improve data throughput.

Note:

This enum is packed to ensure a 1-byte memory footprint. It is typically passed by value to driver configuration structures. Ensure the hardware peripheral and the connected slave device both support the selected speed, otherwise, communication will fail.

Uso:
Used during the initialization phase of an I2C peripheral to configure the bus timing parameters before the bus is enabled.

Esempio:

```
ca_i2c_config_t config;
config.speed = CA_I2C_SPEED_FAST;
ca_i2c_init(I2C1, &config);
```

---

ca_icmp_cfg_t

Tipo: Struttura
File: caos/net/inc/net_cfg.h:92:94

```
typedef struct {
bool echo_reply; /* true = respond to ICMP echo request (ping) */
} ca_icmp_cfg_t;
```

Configuration structure for the ICMP (Internet Control Message Protocol) stack behavior.

Membri:

Nome	Tipo	Descrizione
echo_reply	bool	Enables or disables the device's response to ICMP Echo Request packets (ping). Set to true to allow the device to respond to pings, false to ignore them.

Note:
This structure is typically passed as part of a larger network configuration object during the initialization phase of the network stack. It does not require dynamic memory allocation and is usually defined as a static constant or local variable before being passed to the network configuration API.

Uso:
Use this struct when configuring the network stack parameters to define whether the device should be discoverable via ping requests. It is passed to the network stack initialization function to set the ICMP protocol behavior.

Esempio:

```
ca_icmp_cfg_t icmp_config = { .echo_reply = true };

// Pass the configuration to the network stack initialization
ca_net_init_cfg_t net_cfg = {
    .icmp = &icmp_config,
    // ... other configuration fields
};
ca_net_init(&net_cfg);
```

---

ca_icmp_echo_t

Tipo: Struttura
File: caos/net/inc/net_proto.h:126:132

```
typedef struct {
u8      type;         /* 8 = echo request, 0 = echo reply */
u8      code;         /* 0 */
u16     checksum;     /* big-endian */
u16     id;           /* identifier big-endian */
u16     seq;          /* sequence number big-endian */
} __attribute__((packed)) ca_icmp_echo_t;
```

Represents the header structure of an ICMP Echo message (ping) used for network connectivity testing.

Membri:

Nome	Tipo	Descrizione
type	u8	ICMP message type. Use 8 for Echo Request and 0 for Echo Reply. Incorrect values will cause the packet to be ignored by the stack.
code	u8	ICMP code field. Must be 0 for Echo messages.
checksum	u16	16-bit one's complement of the one's complement sum of the ICMP message. Must be calculated in big-endian format.
id	u16	Identifier to match requests with replies. Must be provided in big-endian format.
seq	u16	Sequence number to track packets. Must be provided in big-endian format.

Note:
The struct is marked with ___attribute___((packed)) to ensure no padding bytes are added by the compiler, matching the exact wire format of the ICMP protocol. Fields requiring multi-byte values (u16) must be converted to network byte order (big-endian) before transmission.

Uso:
Used when manually constructing ICMP packets for network diagnostics or implementing custom ping utilities within the caOS network stack.

Esempio:

```
ca_icmp_echo_t ping_pkt;
ping_pkt.type = 8;
ping_pkt.code = 0;
ping_pkt.id = htons(0x1234);
ping_pkt.seq = htons(1);
ping_pkt.checksum = 0; // Calculate checksum here
// Proceed to send packet via network interface
```

---

ca_ip4_frame_t

Tipo: Struttura
File: caos/net/inc/net_proto.h:104:115

```
typedef struct {
u8      ver_ihl;      /* version[7:4]=4 | IHL[3:0] in 32-bit words */
u8      dscp_ecn;     /* DSCP[7:2] | ECN[1:0] */
u16     total_len;    /* total length incl. header big-endian */
u16     id;           /* identification big-endian */
u16     flags_frag;   /* flags[15:13] | fragment offset[12:0] big-endian
    */
u8      ttl;          /* time to live */
```

```
u8     proto;        /* IP_PROTO_* */
u16    cksum;        /* header checksum big-endian */
ca_ipv4_t src;          /* source IPv4 */
ca_ipv4_t dst;          /* destination IPv4 */
} __attribute__((packed)) ca_ip4_frame_t;
```

Represents the standard IPv4 header structure used for parsing and constructing network packets in the caOS networking stack.

Membri:

Nome	Tipo	Descrizione
`ver_ihl`	`u8`	Version (upper 4 bits, must be 4) and Internet Header Length (lower 4 bits, number of 32-bit words). Incorrect values will cause the packet to be dropped by the network stack.
`dscp_ecn`	`u8`	Differentiated Services Code Point and Explicit Congestion Notification bits. Usually set to 0 for standard traffic.
`total_len`	`u16`	Total length of the IP packet including header and data in bytes. Must be in big-endian format.
`id`	`u16`	Identification field for packet fragmentation. Must be in big-endian format.
`flags_frag`	`u16`	Flags (3 bits) and fragment offset (13 bits). Must be in big-endian format.
`ttl`	`u8`	Time to Live: decremented by each router. If it reaches 0, the packet is discarded.
`proto`	`u8`	Protocol identifier (e.g., 6 for TCP, 17 for UDP). Defines the payload type.
`cksum`	`u16`	Header checksum. Must be calculated correctly or the packet will be rejected by the receiver.
`src`	`ca_ipv4_t`	Source IPv4 address.
`dst`	`ca_ipv4_t`	Destination IPv4 address.

Note:
This struct is marked with ___attribute___((packed)) to ensure no padding bytes are inserted by the compiler, matching the exact wire format of an IPv4 header. Fields requiring multi-byte values (u16) must be converted to network byte order (big-endian) before transmission.

Uso:
Used when manually crafting raw IP packets or parsing incoming network buffers. It is typically cast onto a pointer of a received network buffer to access header fields directly.

Esempio:

```
ca_ip4_frame_t *ip_hdr = (ca_ip4_frame_t *)buffer_ptr;
if (ip_hdr->ver_ihl >> 4 == 4) {
    ca_ipv4_t source = ip_hdr->src;
    // Process packet header fields
}
```

`ca_iperf_result_t`

Tipo: Struttura
File: `caos/net/inc/iperf_service.h:11:15`

```
typedef struct {
u32 bytes_total;
u32 duration_ms;
u32 bps;            /* bits per second */
} ca_iperf_result_t;
```

A structure used to store the performance metrics collected after an iPerf network test session.

Membri:

Nome	Tipo	Descrizione
bytes_total	u32	The total number of bytes successfully transferred during the test. Must be non-negative.
duration_ms	u32	The total duration of the test in milliseconds. Used to calculate throughput.
bps	u32	The calculated throughput in bits per second. Derived from bytes_total and duration_ms.

Note:
This structure is typically populated by the caOS iPerf service upon completion of a measurement. It is a plain data structure (POD) and does not require dynamic memory allocation or explicit cleanup. The values are usually read-only for the application layer once the test finishes.

Uso:
Use this type to retrieve and display network performance statistics after calling the iPerf service functions. It allows the application to log or report the bandwidth efficiency of the current network configuration.

Esempio:

```
ca_iperf_result_t result;

/* Assuming iperf_get_last_result is the API to retrieve the data */
if (iperf_get_last_result(&result) == CA_OK) {
    printf("Test complete: %u bytes in %u ms\n", result.bytes_total, result.
    duration_ms);
    printf("Throughput: %u bps\n", result.bps);
}
```

---

ca_ipv4_t

Tipo: union
File: caos/net/inc/net_cfg.h:38:41

```
typedef union {
u32 addr;   /* comparison: a.addr == b.addr  |  assignment: dst = src */
u8  v[4];   /* octet access: ip.v[0] .. ip.v[3]                        */
} ca_ipv4_t;
```

A union representing an IPv4 address, allowing access either as a single 32-bit integer or as an array of four 8-bit octets.

Membri:

Nome	Tipo	Descrizione
addr	u32	The IPv4 address stored as a 32-bit unsigned integer. Useful for direct comparisons or copying the entire address at once.
v	u8[4]	The IPv4 address stored as an array of four bytes (octets). Useful for accessing specific segments of the IP address (e.g., v[0] for the first octet).

Note:
This is a union, meaning both members share the same memory location. Modifying one member will change the value of the other. It is typically allocated on the stack or as part of a larger network configuration structure. No dynamic memory allocation is required.

Uso:
Use this type whenever you need to store or manipulate an IPv4 address in network-related configurations or packet headers. It is designed to provide flexibility between treating the address as a single numerical value or as individual bytes.

Esempio:

```
ca_ipv4_t my_ip; my_ip.v[0] = 192; my_ip.v[1] = 168; my_ip.v[2] = 1; my_ip.v[3] = 10; if (my_ip.addr == 0x0A01A8C0) { /* Logic for specific IP */ }
```

---

ca_job_type

Tipo: Enumerazione
File: caos/inc/scheduler.h:53:60

```
typedef enum ca_job_type_t{
CA_JOB_STD,
JOB_SAFE,
#if __FPU_PRESENT
JOB_FLOAT_POINT,
CA_JOB_SAFE_FLOAT_POINT,
#endif
}ca_job_type;
```

An enumeration defining the execution context and hardware requirements for a task (job) in the caOS scheduler.

Membri:

Nome	Tipo	Descrizione
CA_JOB_STD	enum constant	Standard task execution. Uses integer registers only. Most efficient for general logic.
JOB_SAFE	enum constant	Task with stack overflow protection enabled. Use this for critical tasks where memory corruption must be detected.
JOB_FLOAT_POINT	enum constant	Task that performs floating-point calculations. Requires the FPU context to be saved/restored during context switches.
CA_JOB_SAFE_FLOAT_POINT	enum constant	Combines both stack protection and FPU context support. Use for critical tasks performing math.

Note:
The availability of floating-point types depends on the ___FPU_PRESENT macro defined in the CMSIS device header. If you select a float-enabled type on a device without an FPU, the

behavior is undefined. Always ensure the stack size is sufficient when using 'SAFE' variants, as they may add overhead for monitoring.

Uso:
This type is passed as an argument during task creation (e.g., when calling the scheduler's task spawn functions) to inform the kernel how to configure the CPU registers and memory protection unit for that specific thread.

Esempio:

```
ca_job_type my_task_type = JOB_FLOAT_POINT;

// Example of passing the type to a task creation function
ca_task_create(my_task_function, "TaskName", 256, NULL, 1, my_task_type);
```

---

ca_jpeg_cfg_t

Tipo: Struttura
File: caos/drivers/inc/jpeg.h:52:57

```
typedef struct {
u16            width;
u16            height;
ca_jpeg_css_t  css;
u8             quality;      /* 1..100, encoding only */
} ca_jpeg_cfg_t;
```

Configuration structure for JPEG image processing, defining dimensions, color space, and compression quality.

Membri:

Nome	Tipo	Descrizione
width	u16	Image width in pixels. Must be a multiple of 8 or 16 depending on the color space.
height	u16	Image height in pixels. Must be a multiple of 8 or 16 depending on the color space.
css	ca_jpeg_css_t	Chroma Subsampling format (e.g., 4:2:0, 4:2:2, 4:4:4). Incorrect values will cause encoding/decoding errors.
quality	u8	Compression quality factor, ranging from 1 (lowest quality, highest compression) to 100 (highest quality, lowest compression). Used only during encoding.

Note:
This struct is passed by value or pointer to ca_jpeg functions. It does not require dynamic allocation; it is typically declared on the stack by the user before calling initialization or processing functions.

Uso:
Used to configure the JPEG hardware peripheral before starting an encode or decode operation. Pass this struct to ca_jpeg_init to set up the hardware context.

Esempio:

```
#include "caos/drivers/inc/jpeg.h"

void setup_jpeg_task(void) {
```

```
    ca_jpeg_cfg_t config;

    /* Initialize configuration for a 640x480 image */
    config.width = 640;
    config.height = 480;
    config.css = 0; /* Assuming 0 maps to a valid ca_jpeg_css_t enum value */
    config.quality = 85;

    /* Apply configuration to the JPEG peripheral */
    ca_jpeg_init(&config);
}
```

---

ca_jpeg_css_t

Tipo: Enumerazione
File: caos/drivers/inc/jpeg.h:45:50

```
typedef enum {
CA_JPEG_GRAY = 0,
CA_JPEG_444,
CA_JPEG_422,
CA_JPEG_420
} ca_jpeg_css_t;
```

Defines the Chroma Subsampling (CSS) format used for JPEG image encoding and decoding.

Membri:

Nome	Tipo	Descrizione
CA_JPEG_GRAY	enum value	Grayscale (no chroma subsampling). Uses only the luminance channel.
CA_JPEG_444	enum value	4:4:4 subsampling. No chroma reduction; highest quality, largest file size.
CA_JPEG_422	enum value	4:2:2 subsampling. Horizontal chroma reduction by factor of 2.
CA_JPEG_420	enum value	4:2:0 subsampling. Horizontal and vertical chroma reduction by factor of 2. Most common for standard JPEG compression.

Note:
This enum is passed to JPEG driver functions to configure the color space conversion logic. Ensure the selected mode matches the source image buffer format and the desired compression ratio.

Uso:
Used as a parameter in ca_jpeg_encode or ca_jpeg_decode functions to specify how the MCU (Minimum Coded Unit) data is interpreted or generated.

Esempio:

```
#include "caos/drivers/inc/jpeg.h"

void process_image(void) {
    /* Define the subsampling format for the JPEG encoder */
    ca_jpeg_css_t format = CA_JPEG_420;

    /* Initialize the JPEG hardware peripheral */
    ca_jpeg_init();

```

```
    /*
     * Perform encoding operation.
     * Note: ca_jpeg_encode is the hypothetical function used to process
     * the image buffer using the specified format.
     */
    // ca_jpeg_encode(input_buffer, output_buffer, format);
}
```

---

ca_jpeg_info_t

Tipo: Struttura
File: caos/drivers/inc/jpeg.h:59:65

```
typedef struct {
u16            width;
u16            height;
ca_jpeg_css_t  css;
u32            mcus;          /* MCUs of the image */
u32            mcu_bytes;     /* total MCU data, mcus x bytes per MCU */
} ca_jpeg_info_t;
```

Structure containing metadata about a JPEG image, such as dimensions and MCU (Minimum Coded Unit) layout.

Membri:

Nome	Tipo	Descrizione
width	u16	Image width in pixels.
height	u16	Image height in pixels.
css	ca_jpeg_css_t	Chroma Subsampling format (e.g., 4:2:0, 4:2:2).
mcus	u32	Total number of MCUs in the image.
mcu_bytes	u32	Total size of MCU data in bytes (calculated as mcus * bytes_per_mcu).

Note:
This structure is typically populated by the ca_jpeg_geometry function. It is used by the application to allocate appropriate buffers before calling decoding or encoding functions. The memory for this struct is usually provided by the caller on the stack or as a static variable.

Uso:
Use this struct to retrieve image properties from a JPEG stream or file header using ca_jpeg_geometry. The resulting values are essential for calculating buffer sizes required for ca_jpeg_decode or ca_jpeg_encode operations.

Esempio:

```
#include "caos/drivers/inc/jpeg.h"

void process_jpeg_info(void *jpeg_data, u32 size) {
    ca_jpeg_info_t info;

    /* Retrieve geometry information from the raw JPEG data */
    if (ca_jpeg_geometry(jpeg_data, size, &info) == 0) {
        ca_printf("Image dimensions: %u x %u\n", info.width, info.height);
        ca_printf("Total MCUs: %u, Total bytes: %u\n", info.mcus, info.mcu_bytes
    );

        /* Proceed to decode using the retrieved info */
        /* ca_jpeg_decode(jpeg_data, size, ...); */
    }
```

```
}
```

---

ca_keys_store_access_mode_t

Tipo: Enumerazione
File: caos/crypto/inc/keys_store.h:42:47

```
typedef enum {
CA_KEYS_STORE_NO_ACCESS = 0,        /* No access allowed (region disabled
    ) */
CA_KEYS_STORE_READ_ONLY = 1,        /* Read-only allowed */
CA_KEYS_STORE_READ_WRITE = 2,       /* Read and write allowed */
CA_KEYS_STORE_EXEC_ONLY = 3         /* Execute-only allowed (rarely used)
     */
} ca_keys_store_access_mode_t;
```

Defines the permission levels for accessing a specific key storage region in the caOS crypto subsystem.

Membri:

Nome	Tipo	Descrizione
CA_KEYS_STORE_NO_ACCESS	enum value	Region is disabled; any attempt to access keys will fail.
CA_KEYS_STORE_READ_ONLY	enum value	Only read operations are permitted on the keys in this region.
CA_KEYS_STORE_READ_WRITE	enum value	Both read and write operations are permitted.
CA_KEYS_STORE_EXEC_ONLY	enum value	Only execution (e.g., cryptographic operations) is permitted; rarely used.

Note:
This enum is used to configure the hardware or software protection level of key storage regions. It is typically passed to ca_keys_store_set_access_mode to update the security policy of a specific key slot. Ensure the mode is set correctly before performing cryptographic operations to avoid access violations.

Uso:
Use this type when managing key storage security policies via the ca_keys_store_* API family, specifically when initializing or dynamically updating the access permissions of a key storage region.

Esempio:

```
#include "caos/crypto/inc/keys_store.h"

void update_key_security(void) {
    /* Define the desired access mode for a key storage region */
    ca_keys_store_access_mode_t new_mode = CA_KEYS_STORE_READ_ONLY;

    /* Apply the access mode to the key store */
    /* Note: 'key_id' is a placeholder for the specific key index being managed
    */
    ca_keys_store_set_access_mode(key_id, new_mode);
}
```

---

ca_l2_frame_t

Tipo: Struttura
File: caos/net/inc/net_proto.h:75:79

```
typedef struct {
ca_mac_t  dst;          /* destination MAC */
ca_mac_t  src;          /* source MAC */
u16    ethertype;   /* EtherType big-endian: ETH_P_IP / ETH_P_ARP /
   ETH_P_VLAN */
} __attribute__((packed)) ca_l2_frame_t;
```

Represents the standard Ethernet Layer 2 header structure used for packet transmission and reception.

Membri:

Nome	Tipo	Descrizione
dst	ca_mac_t	The 6-byte destination MAC address. Must be a valid unicast, multicast, or broadcast address.
src	ca_mac_t	The 6-byte source MAC address of the local network interface.
ethertype	u16	The protocol identifier (e.g., 0x0800 for IPv4). Must be provided in Big-Endian format (network byte order).

Note:
The struct is marked with ___attribute___((packed)) to ensure no padding bytes are inserted by the compiler, matching the exact wire format of an Ethernet frame. Users are responsible for ensuring the ethertype is converted to network byte order using appropriate macros if the CPU is Little-Endian.

Uso:
Used when manually constructing raw Ethernet frames for network drivers or packet injection. It is typically cast to the start of a buffer before passing the pointer to the low-level transmission function.

Esempio:

```
ca_l2_frame_t frame;
memcpy(frame.dst.addr, target_mac, 6);
memcpy(frame.src.addr, local_mac, 6);
frame.ethertype = htons(ETH_P_IP);

/* Proceed to copy payload data immediately after the header */
u8 *packet_ptr = (u8*)&frame;
ca_net_send(packet_ptr, sizeof(ca_l2_frame_t) + payload_len);
```

---

ca_l2_result_t

Tipo: Enumerazione
File: caos/net/inc/net_proto.h:181:186

```
typedef enum {
CA_L2_ACCEPT_UNICAST   = 0,
CA_L2_ACCEPT_BROADCAST = 1,
CA_L2_ACCEPT_MULTICAST = 2,
CA_L2_DROP             = 3,
} ca_l2_result_t;
```

An enumeration used by Layer 2 network callbacks to instruct the stack on how to handle an incoming data frame.

Membri:

Nome	Tipo	Descrizione
`CA_L2_ACCEPT_UNICAST`	`enum constant`	Accepts the frame because it is addressed specifically to this device. Processing continues.
`CA_L2_ACCEPT_BROADCAST`	`enum constant`	Accepts the frame as it is intended for all devices on the network segment.
`CA_L2_ACCEPT_MULTICAST`	`enum constant`	Accepts the frame because the device is subscribed to the specific multicast group.
`CA_L2_DROP`	`enum constant`	Rejects the frame. The stack will discard the packet immediately, saving memory and processing time.

Note:
This enum is returned by user-defined filter callbacks registered within the caOS network stack. The stack relies on this return value to decide whether to allocate buffer resources for the incoming packet. Returning an invalid value may lead to undefined behavior in the packet processing pipeline.

Uso:
Use this type as the return value for your L2 frame filtering functions. It is typically used when implementing custom MAC address filtering or protocol-specific packet inspection to prevent unnecessary processing of irrelevant traffic.

Esempio:

```
ca_l2_result_t my_frame_filter(const uint8_t *mac_addr) {
    if (is_my_mac(mac_addr)) {
        return CA_L2_ACCEPT_UNICAST;
    } else if (is_broadcast(mac_addr)) {
        return CA_L2_ACCEPT_BROADCAST;
    }
    return CA_L2_DROP;
}
```

`ca_l2_vlan_frame_t`

Tipo: Struttura
File: `caos/net/inc/net_proto.h:82:88`

```
typedef struct {
ca_mac_t  dst;          /* destination MAC */
ca_mac_t  src;          /* source MAC */
u16     tpid;         /* 0x8100 big-endian */
u16     tci;          /* PCP[15:13] | DEI[12] | VID[11:0] big-endian */
u16     ethertype;    /* inner EtherType big-endian */
} __attribute__((packed)) ca_l2_vlan_frame_t;
```

Represents an IEEE 802.1Q tagged Ethernet frame header used for VLAN-aware network communication.

Membri:

Nome	Tipo	Descrizione
dst	ca_mac_t	The 6-byte destination MAC address. Must be a valid unicast, multicast, or broadcast address.
src	ca_mac_t	The 6-byte source MAC address of the sending interface.
tpid	u16	Tag Protocol Identifier. Must be set to 0x8100 (in big-endian) to indicate a VLAN-tagged frame.
tci	u16	Tag Control Information. Contains 3 bits for Priority (PCP), 1 bit for Drop Eligible Indicator (DEI), and 12 bits for the VLAN ID (VID). Must be provided in big-endian format.
ethertype	u16	The protocol type of the encapsulated payload (e.g., 0x0800 for IPv4). Must be in big-endian.

Note:
This struct is marked with ___attribute___((packed)) to ensure no padding bytes are inserted by the compiler, matching the exact wire format of an Ethernet frame. Always use network byte order (big-endian) for the 16-bit fields to ensure compatibility with network hardware.

Uso:
Used when constructing or parsing raw Ethernet frames in the caOS network stack. It is typically cast over a buffer received from the NIC driver or used to populate a buffer before transmission.

Esempio:

```
ca_l2_vlan_frame_t frame;
memset(&frame, 0, sizeof(ca_l2_vlan_frame_t));
frame.tpid = htons(0x8100);
frame.tci = htons(0x000A); // VLAN ID 10
frame.ethertype = htons(0x0800); // IPv4
memcpy(frame.dst.addr, dest_mac, 6);
memcpy(frame.src.addr, my_mac, 6);
```

---

ca_lptim_cb_t

Tipo: Callback
File: caos/drivers/inc/lptim.h:43:43

```
typedef void (*ca_lptim_cb_t)(void)
```

A function pointer type used to define a callback function triggered by the Low Power Timer (LPTIM) interrupt.

Membri:

Nome	Tipo	Descrizione
void	void	The callback function takes no arguments and returns no value.

Note:
The callback is executed within the context of the LPTIM interrupt service routine (ISR). Keep the implementation extremely short and avoid blocking operations or heavy processing. Ensure the function is defined with a void return type and no parameters to match the signature.

Uso:
Use this type when configuring the LPTIM driver to specify a custom action that should occur upon a timer event (e.g., overflow or compare match). It is typically passed to the LPTIM initialization or configuration function provided by the caOS driver API.

Esempio:

```
void my_lptim_handler(void) {
    /* Toggle an LED or set a flag for the main loop */
    HAL_GPIO_TogglePin(GPIOA, CA_GPIO_PIN_5);
}

/* Registering the callback */
ca_lptim_cb_t my_callback = my_lptim_handler;
ca_lptim_set_callback(my_callback);
```

---

ca_lptim_mode_t

Tipo: Enumerazione
File: caos/drivers/inc/lptim.h:45:48

```
typedef enum {
LPTIM_MODE_PERIODIC = 0,
LPTIM_MODE_ONESHOT,
} ca_lptim_mode_t;
```

Defines the operational behavior of the Low Power Timer (LPTIM) peripheral regarding signal generation.

Membri:

Nome	Tipo	Descrizione
CA_LPTIM_MODE_PERIODIC	enum constant	The timer automatically reloads and restarts after reaching the compare value, creating a continuous pulse train.
CA_LPTIM_MODE_ONESHOT	enum constant	The timer stops automatically after the first match event occurs; requires manual restart to trigger again.

Note:
This enum is used during the LPTIM configuration phase. It determines whether the hardware peripheral will generate a continuous signal or a single event. Ensure the mode matches your application's timing requirements to avoid unexpected interrupt flooding in periodic mode.

Uso:
Passed as a parameter to the LPTIM initialization or configuration functions to set the hardware operating mode before starting the timer.

Esempio:

```
ca_lptim_config_t config;
config.mode = CA_LPTIM_MODE_ONESHOT;
config.prescaler = LPTIM_PRESCALER_DIV1;
ca_lptim_init(LPTIM1, &config);
```

---

ca_lpuart_rx_cb_t

Tipo: Callback
File: caos/drivers/inc/lpuart.h:42:42

```
typedef void (*ca_lpuart_rx_cb_t)(u8 byte)
```

A function pointer type used as a callback to handle individual bytes received by the LPUART peripheral.

Membri:

Nome	Tipo	Descrizione
byte	u8	The 8-bit data byte received from the LPUART hardware buffer. Valid range is 0x00 to 0xFF.

Note:
This callback is executed within the LPUART Interrupt Service Routine (ISR) context. It must be kept extremely short and efficient to avoid blocking other system interrupts. Do not perform blocking operations, heavy calculations, or complex RTOS primitives (like long-duration mutex locks) inside this function.

Uso:
Used when configuring the LPUART driver to register a custom handler for incoming data. It is typically passed to the driver initialization or configuration function to enable asynchronous byte-by-byte processing.

Esempio:

```
void my_rx_handler(u8 byte) {
    if (byte == '\n') {
        // Signal a task that a full message has arrived
    }
}

// During driver setup:
ca_lpuart_config_t config = {
    .rx_cb = my_rx_handler,
    .baudrate = 115200
};
ca_lpuart_init(LPUART1, &config);
```

---

ca_mac_t

Tipo: union
File: caos/net/inc/net_cfg.h:44:50

```
typedef union {
struct {
u32 l;  // First 4 bytes of the MAC (0, 1, 2, 3)
u16 h; // Last 2 bytes of the MAC (4, 5)
} __attribute__((packed)) raw;
u8  v[6];      // For access to individual bytes
} ca_mac_t;
```

A union representing a 6-byte MAC address, allowing access either as an array of bytes or as split 32-bit and 16-bit integers.

Membri:

Nome	Tipo	Descrizione
raw	struct	Provides access to the MAC address split into a 32-bit integer (first 4 bytes) and a 16-bit integer (last 2 bytes). Useful for hardware register mapping.
v	u8[6]	Provides direct access to the 6 individual bytes of the MAC address. Use this for standard array-based iteration or network packet construction.

Note:
The union is packed to ensure no padding bytes are inserted by the compiler, maintaining a strict 6-byte footprint. It is typically allocated on the stack or as part of a configuration struct. The user is responsible for the byte order (endianness) when assigning values to the 'raw' struct members.

Uso:
Used in network configuration structures and Ethernet driver APIs to store or pass MAC addresses. Use the 'v' array for easy indexing (e.g., v[0] to v[5]) when parsing or printing MAC addresses.

Esempio:

```
ca_mac_t my_mac = {.v = {0x00, 0x80, 0xE1, 0x00, 0x00, 0x01}};

// Accessing via array
printf("MAC: %02X:%02X:%02X:%02X:%02X:%02X\n",
       my_mac.v[0], my_mac.v[1], my_mac.v[2],
       my_mac.v[3], my_mac.v[4], my_mac.v[5]);

// Accessing via raw struct
u32 first_part = my_mac.raw.l;
u16 last_part = my_mac.raw.h;
```

---

ca_mem_range

Tipo: Struttura
File: caos/inc/memory.h:171:175

```
typedef struct {
u32  addr;               /**< base address */
u32  size;               /**< size in BYTES */
char name[12];           /**< "RAM1".."RAMn" by ascending address, "FLASH
   " */
} ca_mem_range;
```

Defines a contiguous block of memory within the system, used by the kernel to manage memory regions.

Membri:

Nome	Tipo	Descrizione
addr	u32	The starting physical address of the memory block. Must be aligned according to hardware requirements.
size	u32	The total size of the memory block in bytes. Ensure this does not exceed the actual physical memory boundaries.
name	char[12]	A human-readable identifier for the memory region (e.g., 'RAM1', 'FLASH'). Limited to 11 characters plus the null terminator.

Note:
This structure is typically used in static configuration arrays. The memory it describes must be physically present and accessible by the CPU. The kernel does not allocate this struct; it is usually defined as a global constant in the board support package (BSP).

Uso:

Used during system initialization to inform the caOS memory manager about available memory segments. It is passed to kernel memory registration functions to define the heap or memory pools.

Esempio:

```
const ca_mem_range my_ram_region = { .addr = 0x20000000, .size = 0x20000, .name = "RAM1" };
```

---

ca_memory_check_t

Tipo: Struttura
File: caos/inc/memory.h:143:157

```
typedef struct {
u32  blocks;        /**< blocks walked, START and END included */
u32  busy;          /**< BUSY blocks */
u32  busy_bytes;    /**< payload of the BUSY blocks */
u32  free;          /**< FREE blocks */
u32  free_bytes;    /**< payload of the FREE blocks */
u32  largest_free;  /**< largest FREE payload: the biggest allocation that can succeed */
u32  unmerged;      /**< FREE blocks followed by another FREE one (missed merge) */
u32  avail;         /**< g_avail_mem as the allocator holds it */
u32  avail_expect;  /**< what the chain says it should be */
u32  bad_addr;      /**< first block that failed a check, 0 if none */
cstr bad_what;      /**< which check it failed, NULL if none */
bool chain_ok;      /**< every header valid, linked both ways, contiguous */
bool canary_ok;     /**< ca_memory_check_canary() */
} ca_memory_check_t;
```

A diagnostic structure used to report the health and fragmentation status of the caOS heap memory allocator.

Membri:

Nome	Tipo	Descrizione
`blocks`	`u32`	Total number of memory blocks traversed during the heap walk.
`busy`	`u32`	Number of blocks currently allocated to the application.
`busy_bytes`	`u32`	Total bytes currently in use by allocated blocks.
`free`	`u32`	Number of blocks currently available for allocation.
`free_bytes`	`u32`	Total bytes available across all free blocks.
`largest_free`	`u32`	The size of the largest contiguous free block; represents the maximum possible single allocation.
`unmerged`	`u32`	Count of adjacent free blocks that should have been merged; indicates potential heap fragmentation issues.
`avail`	`u32`	The total available memory reported by the allocator's internal counter.
`avail_expect`	`u32`	The total available memory calculated by summing the free blocks in the chain; should match 'avail'.
`bad_addr`	`u32`	The memory address where a corruption or inconsistency was first detected; 0 if the heap is healthy.
`bad_what`	`cstr`	A string description of the specific validation check that failed; NULL if the heap is healthy.
`chain_ok`	`bool`	True if the linked list of memory blocks is structurally sound (pointers are valid and contiguous).
`canary_ok`	`bool`	True if the memory guard patterns (canaries) surrounding blocks remain intact, indicating no buffer overflows.

Note:
This structure is populated by the system during a heap integrity check. It is typically allocated on the stack by the caller. The fields are read-only for the user; the system fills them to provide a snapshot of heap health.

Uso:
Use this structure when debugging memory-related crashes, heap corruption, or to monitor fragmentation levels in your application during development.

Esempio:

```c
ca_memory_check_t heap_status;

// Perform the check and populate the structure
if (ca_memory_check(&heap_status)) {
    if (!heap_status.chain_ok || !heap_status.canary_ok) {
        // Handle heap corruption error
        printf("Heap corruption at: %p\n", (void*)heap_status.bad_addr);
    } else {
        printf("Largest free block: %lu bytes\n", heap_status.largest_free);
    }
}
```

`ca_message_buffer_t`

Tipo: Struttura
File: `caos/inc/message.h:34:43`

```
typedef struct {
u8 *buffer;          /**< Data buffer */
u32 size;            /**< Total buffer size */
volatile u32 head;   /**< Write index */
volatile u32 tail;   /**< Read index */
volatile u32 used;   /**< Used bytes (header + payload) */
volatile u16 msg_count; /**< Number of messages in buffer */
ca_spinlock_irq_t lock; /**< IRQ-safe lock */
u32 waiting_task;    /**< Waiter HANDLE (ca_task_handle_t),
    CA_TASK_HANDLE_NONE = none */
} ca_message_buffer_t;
```

A thread-safe, circular buffer structure used for inter-task communication and message queuing in caOS.

Membri:

Nome	Tipo	Descrizione
`buffer`	`u8 *`	Pointer to the raw memory array allocated for storing message data. Must be large enough to hold all queued messages.
`size`	`u32`	The total capacity of the buffer in bytes. Must match the allocated size of the buffer pointer.
`head`	`volatile u32`	Current write index. Incremented by the producer; should not be modified manually by the user.
`tail`	`volatile u32`	Current read index. Incremented by the consumer; should not be modified manually by the user.
`used`	`volatile u32`	Current number of bytes occupied in the buffer. Used to track overflow conditions.
`msg_count`	`volatile u16`	The number of discrete messages currently stored in the buffer.
`lock`	`ca_spinlock_irq_t`	Internal spinlock used to ensure atomic access to the buffer across different task contexts and ISRs.
`waiting_task`	`u32`	Stores the handle of a task blocked waiting for data. Set to CA_TASK_HANDLE_NONE if no task is waiting.

Note:
The structure must be initialized before use. The 'buffer' memory must be allocated statically or globally to ensure it persists for the lifetime of the message buffer. The 'volatile' qualifiers are critical as they prevent the compiler from optimizing away reads/writes that occur across different execution contexts (tasks vs ISRs).

Uso:
Used as the primary data structure for message passing APIs in caOS. It is passed by reference to functions that push data into the queue or pull data out, ensuring thread-safe data exchange.

Esempio:

```
static u8 raw_data[256];
ca_message_buffer_t my_msg_queue = {
    .buffer = raw_data,
    .size = 256,
    .head = 0,
    .tail = 0,
    .used = 0,
    .msg_count = 0,
    .waiting_task = CA_TASK_HANDLE_NONE
};
```

```

/* The queue is now ready to be passed to caOS message API functions */
```

---

ca_mqtt_connected_cb_t

Tipo: Callback
File: caos/net/inc/mqtt_service.h:51:51

```
typedef void (*ca_mqtt_connected_cb_t)(void)
```

A callback function pointer type triggered by the MQTT service when a successful connection to the broker is established.

Membri:

Nome	Tipo	Descrizione
void	void	This callback takes no arguments and returns no value.

Note:
This callback is executed within the context of the MQTT service task. It must be non-blocking and should not perform heavy processing. Ensure that any shared resources accessed within this callback are protected by appropriate synchronization primitives like mutexes.

Uso:
Use this callback to trigger application-level logic immediately after the MQTT client successfully connects, such as subscribing to specific topics or publishing initial status messages.

Esempio:

```
void my_on_connect_handler(void) {
    // Logic to execute upon connection
    ca_mqtt_subscribe("sensors/data", 1);
}

// Registering the callback during MQTT configuration
ca_mqtt_config_t config = {
    .on_connected = my_on_connect_handler,
    // ... other config fields
};
ca_mqtt_init(&config);
```

---

ca_mqtt_disconnected_cb_t

Tipo: Callback
File: caos/net/inc/mqtt_service.h:62:62

```
typedef void (*ca_mqtt_disconnected_cb_t)(void)
```

A callback function type triggered by the MQTT service when the connection to the broker is lost.

Membri:

Nome	Tipo	Descrizione
void	void	This callback takes no arguments and returns nothing.

Note:

This callback is executed within the context of the MQTT service task. It must be kept short and non-blocking to avoid stalling the network stack. It is typically registered during the MQTT client initialization phase.

Uso:
Use this to implement reconnection logic, update UI status indicators, or trigger system-wide alerts when the MQTT connection drops.

Esempio:

```
void my_on_disconnect(void) {
    printf("MQTT connection lost! Attempting to reconnect...\n");
    // Trigger a flag for the main loop to handle reconnection
}

// During initialization:
ca_mqtt_config_t config = {
    .disconnected_cb = my_on_disconnect
};
```

---

ca_mqtt_message_cb_t

Tipo: Callback
File: caos/net/inc/mqtt_service.h:58:58

```
typedef void (*ca_mqtt_message_cb_t)(const char *topic,  u16 topic_len,
```

A callback function pointer type used to handle incoming MQTT messages received by the caOS MQTT client.

Membri:

Nome	Tipo	Descrizione
topic	const char *	Pointer to the string containing the MQTT topic. Note: this string is not guaranteed to be null-terminated.
topic_len	u16	The length of the topic string. Always use this value to process the topic safely.
payload	const uint8_t *	Pointer to the raw data buffer received in the MQTT message.
payload_len	u16	The size of the payload in bytes.

Note:
The callback is executed within the context of the MQTT service task. Keep the implementation short and non-blocking to avoid delaying other network operations. The memory pointed to by topic and payload is managed by the MQTT stack and is only valid for the duration of the callback execution.

Uso:
Used when subscribing to MQTT topics. You pass a function pointer of this type to the subscription API to define the logic that executes whenever a message arrives on a specific topic.

Esempio:

```
void my_mqtt_handler(const char *topic, u16 topic_len, const uint8_t *payload,
    u16 payload_len) { if (payload_len > 0) { printf("Received message on topic:
     %.*s\n", topic_len, topic); } } // Usage: ca_mqtt_subscribe(client, "
    sensors/temp", my_mqtt_handler);
```

---

ca_mutex_t

Tipo: Struttura
File: caos/inc/sync.h:51:58

```
typedef struct ca_mutex_s {
volatile u32 owner;          // owner's ca_task_handle_t while locked (
    not a
// thid: a position moves when a task below ends)
volatile u8 locked;          // 0 = free, 1 = locked
ca_wait_queue_t wait_queue;      // Tasks waiting
ca_spinlock_irq_t internal_lock; // Protects internal structure
struct ca_mutex_s *pi_next;   // next mutex in the owner's pi_held list
} ca_mutex_t;
```

A mutual exclusion primitive used to protect shared resources from concurrent access by multiple tasks.

Membri:

Nome	Tipo	Descrizione
owner	u32	Stores the handle of the task currently holding the mutex. Do not modify manually.
locked	u8	Status flag: 0 means the mutex is available, 1 means it is currently held by a task.
wait_queue	ca_wait_queue_t	Internal list of tasks blocked while waiting for this mutex to become available.
internal_lock	ca_spinlock_irq_t	A low-level spinlock used to ensure atomic operations on the mutex structure itself.
pi_next	struct ca_mutex_s *	Pointer used for Priority Inheritance (PI) tracking to link multiple mutexes held by the same task.

Note:
The mutex must be initialized before use. It is typically allocated in the global data segment or as part of a larger control block. Ownership is managed by the RTOS kernel; never attempt to manually set the 'owner' or 'locked' fields, as this will corrupt the scheduler's synchronization logic.

Uso:
Use this type to prevent race conditions when multiple tasks access the same peripheral or global variable. It is passed to synchronization APIs to lock and unlock critical sections.

Esempio:

```
ca_mutex_t my_mutex;

void task_function(void *arg) {
    ca_mutex_init(&my_mutex);

    if (ca_mutex_lock(&my_mutex, CA_WAIT_FOREVER) == CA_OK) {
        // Critical section: access shared resource here
        ca_mutex_unlock(&my_mutex);
    }
}
```

---

ca_net_buf_meta_t

Tipo: union
File: caos/net/inc/net_buf.h:106:110

```
typedef union {
ca_net_buf_rx_t rx;   /* RX path: filled by   L  2  L3L4   decode
       */
ca_net_buf_tx_t tx;   /* TX path: filled by upper layer
    */
u8              raw[52];
} ca_net_buf_meta_t;                                          /* 52
   bytes */
```

A union structure representing the metadata associated with a network buffer, used to store either RX or TX specific protocol information.

Membri:

Nome	Tipo	Descrizione
rx	ca_net_buf_rx_t	Metadata structure for incoming packets. Contains fields like protocol type, source/destination ports, and checksum status. Only valid when the buffer is in RX mode.
tx	ca_net_buf_tx_t	Metadata structure for outgoing packets. Contains fields like destination address and transmission flags. Only valid when the buffer is in TX mode.
raw	u8[52]	A raw byte array of 52 bytes used to access the memory directly or to ensure the union size is fixed for memory alignment and buffer pool management.

Note:
This union is part of the ca_net_buf_t structure. It does not require manual allocation as it is embedded within the buffer object. The developer must ensure they only access the member corresponding to the current direction of the packet (RX or TX) to avoid data corruption.

Uso:
Used by the network stack to track packet state as it moves through the OSI layers. You will typically access this when writing custom protocol handlers or inspecting packet headers in a callback.

Esempio:

```
ca_net_buf_t *buf = ca_net_buf_alloc();
if (buf != NULL) {
    // Accessing RX metadata for an incoming packet
    buf->meta.rx.proto = CA_NET_PROTO_UDP;
    buf->meta.rx.port = 8080;
    ca_net_process(buf);
}
```

---

ca_net_buf_pressure_t

Tipo: Struttura
File: caos/net/inc/net_buf.h:171:177

```
typedef struct {
u16 total;        /* NET_BUF_COUNT
    */
```

```
u16 free_now;     /* free buffers at the time of the call
    */
u16 free_min;     /* fewest ever free since the last reset  <-- the
   number */
u32 alloc_calls; /* allocations attempted since the last reset
    */
u32 alloc_fail;  /* of those, refused: the pool was empty
    */
} ca_net_buf_pressure_t;
```

A structure used to monitor the health and memory usage of the network buffer pool.

Membri:

Nome	Tipo	Descrizione
`total`	`u16`	The total number of network buffers available in the system pool.
`free_now`	`u16`	The number of buffers currently available for allocation.
`free_min`	`u16`	The lowest number of free buffers recorded since the last system reset; useful for detecting memory pressure spikes.
`alloc_calls`	`u32`	Total number of allocation requests made to the pool since the last reset.
`alloc_fail`	`u32`	Number of allocation requests that failed because the pool was empty.

Note:
This structure is populated by the system. It does not require manual allocation or initialization by the user. It is typically used to diagnose network stability issues or to tune the buffer pool size.

Uso:
Use this structure with ca_net_buf_pressure to retrieve current statistics. If the alloc_fail count is increasing, your system is running out of network buffers, which may lead to dropped packets or connection failures.

Esempio:

```
ca_net_buf_pressure_t stats;

/* Retrieve current network buffer statistics */
ca_net_buf_pressure(&stats);

/* Log the findings using the system printf */
ca_printf("Net Buffers: Total=%u, Free=%u, Min=%u, Failures=%lu\n",
          stats.total, stats.free_now, stats.free_min, stats.alloc_fail);

/* Optional: Reset the statistics counters if needed */
ca_net_buf_pressure_reset();
```

`ca_net_buf_rx_t`

Tipo: Struttura
File: `caos/net/inc/net_buf.h:68:78`

```
typedef struct {
u16    l3_off;   /* frame offset to L3 header: 14 no-VLAN, 18 with 802.1
   Q */
```

```
u16    l4_off;   /* frame offset to L4 header: l3_off + IP IHL; 0 if
    none */
ipv4_t src_ip;   /* IPv4 src (spa for ARP, 0 if non-IP)
     */
ipv4_t dst_ip;   /* IPv4 dst (tpa for ARP, 0 if non-IP)
     */
u16    src_port; /* UDP/TCP source port (0 if non UDP/TCP)
     */
u16    dst_port; /* UDP/TCP dest   port (0 if non UDP/TCP)
     */
u16    vlan_id;  /* 802.1Q VID (0 = untagged)
     */
u8     vlan_pcp; /* 802.1Q PCP (0-7)
     */
u8     ip_proto; /* IP_PROTO_ICMP=1 / UDP=17 / TCP=6; 0 if non-IP
    */
} ca_net_buf_rx_t;                                              /* 20
    bytes */
```

A metadata structure containing parsed network header offsets and addresses for an incoming packet buffer.

Membri:

Nome	Tipo	Descrizione
`l3_off`	`u16`	Byte offset from the start of the buffer to the Layer 3 (IP/ARP) header. Typically 14 for Ethernet II or 18 if 802.1Q VLAN tagged.
`l4_off`	`u16`	Byte offset to the Layer 4 (TCP/UDP) header. Set to 0 if the packet is not a transport layer protocol.
`src_ip`	`ca_ipv4_t`	Source IPv4 address extracted from the packet header. Set to 0 for non-IP packets.
`dst_ip`	`ca_ipv4_t`	Destination IPv4 address extracted from the packet header. Set to 0 for non-IP packets.
`src_port`	`u16`	Source port for TCP/UDP packets. Set to 0 if the protocol is not TCP or UDP.
`dst_port`	`u16`	Destination port for TCP/UDP packets. Set to 0 if the protocol is not TCP or UDP.
`vlan_id`	`u16`	The 802.1Q VLAN ID (VID). Value 0 indicates the packet is untagged.
`vlan_pcp`	`u8`	The 802.1Q Priority Code Point (PCP). Valid range is 0-7.
`ip_proto`	`u8`	The IP protocol number (e.g., 1 for ICMP, 6 for TCP, 17 for UDP). Set to 0 if not an IP packet.

Note:
This structure is typically populated by the caOS network stack driver during packet reception. It is read-only for the application layer. Do not manually modify these fields as they are used by the stack to route packets to the correct socket or handler.

Uso:
Used by network callback functions or packet processing tasks to inspect packet headers without manually parsing raw bytes. It allows for efficient filtering of incoming traffic based on IP or port.

Esempio:

```
void my_packet_handler(ca_net_buf_t *buf, ca_net_buf_rx_t *meta) {
    if (meta->ip_proto == 17 && meta->dst_port == 8080) {
```

```
        /* Process UDP packet destined for port 8080 */
        uint8_t *payload = (uint8_t *)buf->data + meta->l4_off + 8; /* Skip UDP
    header */
    }
}
```

---

ca_net_buf_t

Tipo: Struttura
File: caos/net/inc/net_buf.h:94:105

```
typedef struct {
/* ---- 64 byte metadata ---- */
u16                 next;      /* chained buffer index, NET_BUF_INVALID=
    end */
u16                 data_off;  /* byte offset from data[0] to payload
    start */
u16                 data_len;  /* payload length in bytes
     */
u8                  flags;     /* NET_BUF_F_*
     */
u8                  proto;     /* NET_PROTO_* set by L2 dispatcher
     */
u32                 timestamp; /* ca_os_tick() at frame reception
     */
ca_net_buf_meta_t meta;      /* L2/L3/L4 parse cache + TX routing info
     */
/* ---- 1984 byte data area ---- */
u8                  data[NET_BUF_DATA_SIZE];
} ca_net_buf_t;
```

A network buffer structure used to store packet data and metadata during transmission or reception in the caOS network stack.

Membri:

Nome	Tipo	Descrizione
next	u16	Index of the next buffer in a chain. Use NET_BUF_INVALID if this is the last buffer.
data_off	u16	Offset in bytes from the start of the data array to the actual payload. Must be within [0, NET_BUF_DATA_SIZE].
data_len	u16	Length of the payload in bytes. Must not exceed the remaining space in the data array.
flags	u8	Bitmask of NET_BUF_F_* flags indicating buffer state or packet properties.
proto	u8	Protocol identifier (NET_PROTO_*) assigned by the L2 dispatcher.
timestamp	u32	System tick value captured at the moment of frame reception.
meta	ca_net_buf_meta_t	Internal cache for L2/L3/L4 parsing results and routing information.
data	u8[NET_BUF_DATA_SIZE]	The raw memory buffer where packet data is stored.

Note:

Buffers are typically managed by the caOS network pool. Do not manually allocate these on the stack as they are large (2KB). Ownership is transferred to the network stack once passed to transmission functions.

Uso:
Used by network drivers and protocol layers to pass packet data through the stack. Always initialize the buffer via the network pool API before use.

Esempio:

```
ca_net_buf_t *buf = net_buf_alloc();
if (buf != NULL) {
    buf->data_off = 0;
    buf->data_len = 10;
    memcpy(&buf->data[buf->data_off], "HelloNet", 8);
    buf->next = NET_BUF_INVALID;
    net_send(buf);
}
```

---

ca_net_buf_tx_t

Tipo: Struttura
File: caos/net/inc/net_buf.h:81:86

```
typedef struct {
ipv4_t dst_ip;   /* IPv4 destination address
     */
u16    dst_port; /* UDP destination port
     */
u16    src_port; /* UDP source port
     */
mac_t  dst_mac;  /* resolved destination MAC
     */
} ca_net_buf_tx_t;                                    /* 14
   bytes */
```

A structure defining the network layer routing information required to transmit a UDP packet.

Membri:

Nome	Tipo	Descrizione
dst_ip	ca_ipv4_t	The target IPv4 address. Must be a valid unicast or broadcast address.
dst_port	u16	The destination UDP port number. Values 0-65535 are valid.
src_port	u16	The source UDP port number. Usually assigned by the application.
dst_mac	ca_mac_t	The resolved hardware MAC address of the destination. Must be populated via ARP resolution before transmission.

Note:
This structure is typically allocated on the stack by the application layer. It does not own any memory buffers; it acts as a header descriptor for the network stack to encapsulate data into an Ethernet frame. Ensure the dst_mac is correctly resolved via the ARP table before passing this struct to the transmission API.

Uso:

Used as a parameter in network transmission functions to specify the routing metadata for outgoing UDP datagrams.

Esempio:

```
ca_net_buf_tx_t tx_info = { .dst_ip = {192, 168, 1, 10}, .dst_port = 8080, .
    src_port = 5000, .dst_mac = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55} };
```

---

ca_net_cfg_t

Tipo: Struttura
File: caos/net/inc/net_cfg.h:186:217

```
typedef struct {
ipv4_t         ip;     /* local IPv4 address */
ipv4_t         mask;   /* local subnet mask (0 = unknown, set by DHCP) */
mac_t          mac;    /* local MAC address (used for L2 filter) */
u16            mtu;            /* max TX payload bytes (0 = default 1500)
    */
u8             phy_nrst_port;  /* GPIO port index for PHY nRST (0xFF =
   not wired) */
u8             phy_nrst_pin;   /* GPIO pin 0..15 */
ca_arp_cfg_t   arp;
ca_udp_cfg_t   udp;
ca_icmp_cfg_t  icmp;
#if EN_NET_DHCP
ca_dhcp_cfg_t dhcp;
#endif
#if EN_NET_NTP
ca_ntp_cfg_t   ntp;
#endif
#if EN_NET_SYSLOG
ca_syslog_cfg_t syslog;
#endif
#if EN_NET_DNS
ca_dns_cfg_t   dns;
#endif
#if EN_NET_MDNS
ca_mdns_cfg_t mdns;
#endif
#if EN_NET_MQTT
ca_mqtt_cfg_t mqtt;
#endif
#if EN_NET_MODBUS
ca_modbus_cfg_t modbus;
#endif
} ca_net_cfg_t;
```

Configuration structure used to initialize the caOS network stack with hardware and protocol settings.

Membri:

Nome	Tipo	Descrizione
ip	ca_ipv4_t	The static IPv4 address for the device. If using DHCP, this may be ignored or overwritten.
mask	ca_ipv4_t	The subnet mask. Set to 0 if the network configuration is handled dynamically via DHCP.
mac	ca_mac_t	The unique 6-byte hardware MAC address for the Ethernet interface.
mtu	u16	Maximum Transmission Unit. Set to 0 to use the default value of 1500 bytes.
phy_nrst_port	u8	GPIO port index for the PHY reset pin. Use 0xFF if the PHY reset is not connected to a GPIO.
phy_nrst_pin	u8	GPIO pin number (0-15) for the PHY reset. Only relevant if phy_nrst_port is not 0xFF.
arp	ca_arp_cfg_t	Configuration parameters for the ARP protocol layer.
udp	ca_udp_cfg_t	Configuration parameters for the UDP protocol layer.
icmp	ca_icmp_cfg_t	Configuration parameters for the ICMP (ping) protocol layer.

Note:
This structure must be populated before calling the network initialization function. It is typically declared as a static constant or global variable to ensure it persists throughout the application lifecycle. Fields dependent on preprocessor macros (like DHCP or MQTT) are only available if those features are enabled in the project configuration.

Uso:
Use this struct to define the network identity and protocol behavior of your device. Pass a pointer to this structure to the network stack initialization function during system startup.

Esempio:

```
ca_net_cfg_t my_net_cfg = { .ip = {192, 168, 1, 10}, .mask = {255, 255, 255, 0}, .mac = {0x00, 0x08, 0xDC, 0x11, 0x22, 0x33}, .mtu = 1500, .phy_nrst_port = 0xFF }; ca_net_init(&my_net_cfg);
```

---

ca_net_filter_fn_t

Tipo: Callback
File: caos/net/inc/net.h:146:146

```
typedef void (*ca_net_filter_fn_t)(u16 buf_idx)
```

A callback function type used to inspect or modify network packets before they are processed by the stack.

Membri:

Nome	Tipo	Descrizione
buf_idx	u16	The index of the network buffer in the internal pool. Valid range is 0 to (MAX_NET_BUFFERS - 1). Passing an invalid index will cause an out-of-bounds access.

Note:

This callback is executed within the context of the network task. It must be kept short and non-blocking to avoid dropping incoming packets. The user does not allocate the buffer; they only receive the index to perform operations via the net_buf_get/set APIs.

Uso:
Used to implement custom packet filtering, firewalling, or packet inspection. It is registered with the network stack during initialization to intercept incoming traffic.

Esempio:

```
void my_packet_filter(u16 buf_idx) {
    u8 *data = net_buf_get_ptr(buf_idx);
    if (data[0] == 0xDE) {
        net_buf_drop(buf_idx);
    }
}

// Registration
ca_net_set_filter(my_packet_filter);
```

---

ca_net_handler_t

Tipo: Callback
File: caos/net/inc/net.h:61:61

```
typedef void (*ca_net_handler_t)(u16 buf_idx)
```

A callback function pointer type used by the network stack to process incoming network buffers.

Membri:

Nome	Tipo	Descrizione
buf_idx	u16	The index of the network buffer in the internal pool. Use this index with ca_net_buf_ptr to access the data.

Note:
This callback is typically invoked by the network stack when a packet is received. The handler is responsible for processing the data and must eventually call ca_net_buf_free to release the buffer back to the pool. Failure to free the buffer will lead to a memory leak in the network buffer pool.

Uso:
Used when registering custom protocol handlers or packet processors via ca_net_register. It allows the application to intercept specific network traffic.

Esempio:

```
void my_packet_processor(u16 buf_idx) {
    /* Get pointer to the buffer data using the provided index */
    void *data = ca_net_buf_ptr(buf_idx);

    /* Process the packet data here */
    ca_printf("Received packet at index %d\n", buf_idx);

    /* IMPORTANT: Always free the buffer after processing */
    ca_net_buf_free(buf_idx);
}

void setup_network(void) {
```

```
    /* Register the handler to process incoming packets */
    ca_net_register(my_packet_processor);
}
```

---

ca_net_mac_filter_fn_t

Tipo: Callback
File: caos/net/inc/net.h:182:182

```
typedef bool (*ca_net_mac_filter_fn_t)(u16 buf_idx, u8 iface)
```

A callback function type used to filter incoming network packets based on their MAC address or interface.

Membri:

Nome	Tipo	Descrizione
buf_idx	u16	The index of the network buffer containing the packet to be inspected.
iface	u8	The interface identifier where the packet was received.

Note:
The callback must return true to accept the packet or false to drop it. It is typically registered via ca_net_filter_set. Ensure the logic inside is fast, as it runs within the network processing context.

Uso:
Use this to implement custom firewall rules or packet filtering logic at the link layer. Register the function using ca_net_filter_set to enable the filtering mechanism.

Esempio:

```
bool my_mac_filter(u16 buf_idx, u8 iface) {
    /* Check if the packet at buf_idx should be allowed */
    /* ca_net_buf_ptr(buf_idx) could be used to inspect the header */
    return true;
}

void setup_filter(void) {
    /* Register the custom filter callback */
    ca_net_filter_set(my_mac_filter);
}
```

---

ca_net_queue_t

Tipo: Struttura
File: caos/net/inc/net_queue.h:27:31

```
typedef struct {
u16           buf[NET_QUEUE_DEPTH];
volatile u16 head;    /* consumer (net task) advances head */
volatile u16 tail;    /* producer (ISR)      advances tail */
} ca_net_queue_t;
```

A circular buffer structure used for thread-safe communication of network packets between an ISR and the network task.

Membri:

Nome	Tipo	Descrizione
`buf`	`u16[NET_QUEUE_DEPTH]`	The underlying storage array for packet descriptors or indices. Must be sized to NET_QUEUE_DEPTH.
`head`	`volatile u16`	The read index. Only the consumer (network task) should modify this value.
`tail`	`volatile u16`	The write index. Only the producer (ISR) should modify this value.

Note:
This structure implements a lock-free single-producer single-consumer (SPSC) queue. It relies on the volatile keyword to prevent compiler optimization of index access. The user is responsible for ensuring that the queue is not overflowed by the producer, as there is no built-in bounds checking in the struct itself.

Uso:
Use this type when you need to pass data from a hardware interrupt (e.g., Ethernet RX interrupt) to a background processing task without using heavy synchronization primitives like mutexes.

Esempio:

```c
ca_net_queue_t rx_queue;

void init_queue(void) {
    rx_queue.head = 0;
    rx_queue.tail = 0;
}

/* Inside ISR */
void HAL_ETH_RxCpltCallback(void) {
    rx_queue.buf[rx_queue.tail] = new_packet_id;
    rx_queue.tail = (rx_queue.tail + 1) % NET_QUEUE_DEPTH;
}

/* Inside Network Task */
void net_task(void *p) {
    while(1) {
        if(rx_queue.head != rx_queue.tail) {
            u16 packet = rx_queue.buf[rx_queue.head];
            rx_queue.head = (rx_queue.head + 1) % NET_QUEUE_DEPTH;
            process_packet(packet);
        }
    }
}
```

`ca_net_stats_t`

Tipo: Struttura
File: `caos/net/inc/net.h:68:78`

```c
typedef struct {
u32 rx_frames;         /* frames received */
u32 rx_dropped_pool;   /* dropped: pool exhausted */
u32 rx_dropped_small;  /* dropped: frame < 14 bytes */
u32 rx_no_handler;     /* received but no handler registered */
u32 tx_frames;         /* frames sent */
u32 tx_failed;         /* iface->send() returned false */
```

```
u32 err_bus;           /* CA_ETH_ERR_BUS events */
u32 err_rbu;           /* CA_ETH_ERR_RBU events */
u32 process_calls;     /* how many times ca_net_process() was called */
} ca_net_stats_t;
```

A structure containing diagnostic counters for the network stack, used to monitor traffic and identify packet loss or hardware errors.

Membri:

Nome	Tipo	Descrizione
rx_frames	u32	Total number of Ethernet frames successfully received.
rx_dropped_pool	u32	Frames dropped because the network buffer pool was exhausted.
rx_dropped_small	u32	Frames dropped because they were smaller than the minimum required size (14 bytes).
rx_no_handler	u32	Frames received for which no protocol handler was registered.
tx_frames	u32	Total number of Ethernet frames successfully sent.
tx_failed	u32	Frames that failed to send because the interface driver returned an error.
err_bus	u32	Count of CA_ETH_ERR_BUS hardware events.
err_rbu	u32	Count of CA_ETH_ERR_RBU hardware events.
process_calls	u32	Number of times the network processing loop was executed.

Note:
The structure is populated by the network stack. The user is responsible for providing a valid pointer to a ca_net_stats_t variable to the ca_net_stats() function, which will fill it with current values. It does not require manual allocation if declared on the stack.

Uso:
Use this type to perform periodic health checks on the network interface. It is typically called within a monitoring task to log network performance or detect connectivity issues.

Esempio:

```
ca_net_stats_t stats;

// Retrieve current network statistics
ca_net_stats(&stats);

// Log the number of received frames to the system console
ca_printf("Network RX frames: %u\n", stats.rx_frames);

// Check if there are any dropped frames due to pool exhaustion
if (stats.rx_dropped_pool > 0) {
    ca_printf("Warning: Dropped %u frames due to pool exhaustion!\n", stats.
    rx_dropped_pool);
}
```

---

ca_ntp_cfg_t

Tipo: Struttura
File: caos/net/inc/net_cfg.h:114:118

```c
typedef struct {
ca_ipv4_t           server_ip;    /* NTP server IPv4 */
u32              poll_ms;      /* poll interval in ms (0 = 64000 default)
    */
ca_ntp_synced_t on_synced;    /* called when time is updated (may be NULL
    ) */
} ca_ntp_cfg_t;
```

Configuration structure used to define the parameters for the NTP (Network Time Protocol) client service.

Membri:

Nome	Tipo	Descrizione
`server_ip`	`ca_ipv4_t`	The IPv4 address of the NTP server to synchronize with. Must be a valid network-byte-order address.
`poll_ms`	`u32`	The interval in milliseconds between synchronization attempts. If set to 0, the system defaults to 64000ms.
`on_synced`	`ca_ntp_synced_t`	A callback function pointer triggered whenever the system time is successfully updated. Can be set to NULL if no notification is required.

Note:
The structure is typically passed by value or pointer to the NTP initialization function. The user is responsible for allocating this struct, usually in static memory or on the stack before starting the network stack. Ensure the server_ip is reachable via the current network configuration.

Uso:
Use this structure when configuring the caOS network stack to enable automatic time synchronization. It is passed to the NTP client initialization routine to define the server target and the frequency of updates.

Esempio:

```c
void my_sync_callback(void) { /* Handle time update */ }

ca_ntp_cfg_t ntp_config = {
    .server_ip = {192, 168, 1, 10},
    .poll_ms = 30000,
    .on_synced = my_sync_callback
};

/* Pass ntp_config to the relevant caOS network initialization API */
```

`ca_ntp_fail_cb_t`

Tipo: Callback
File: `caos/net/inc/ntp_service.h:69:69`

```c
typedef void (*ca_ntp_fail_cb_t)(void)
```

A callback function type triggered by the NTP service when a time synchronization attempt fails.

Membri:

Nome	Tipo	Descrizione
`void`	`void`	The callback takes no arguments and returns no value.

Note:
This callback is executed within the context of the NTP service task. It must be kept short and non-blocking to avoid delaying network synchronization tasks. It is typically registered during the initialization of the NTP service.

Uso:
Use this to implement custom error handling or logging when the device fails to reach an NTP server, such as triggering a system status LED or attempting a network interface reset.

Esempio:

```
void my_ntp_error_handler(void) {
    log_error("NTP sync failed, retrying in 1 hour");
    status_led_blink(RED_COLOR);
}

// During system init:
ca_ntp_config_t config = {
    .fail_cb = my_ntp_error_handler
};
ca_ntp_init(&config);
```

---

ca_ntp_pkt_t

Tipo: Struttura
File: caos/net/inc/net_proto.h:424:448

```
typedef struct {
u8  li_vn_mode;      /* Leap Indicator, Version, Mode */
u8  stratum;         /* Stratum */
u8  poll;            /* Poll interval */
i8  precision;       /* Precision */
u32 root_delay;      /* Root delay (32-bit) */
u32 root_dispersion; /* Root dispersion (32-bit) */
u32 ref_id;          /* Reference ID */

/* Reference timestamp (64-bit) */
u32 ref_ts_s;        /* Seconds */
u32 ref_ts_f;        /* Fraction */

/* Origin Timestamp (64-bit) - Here the server copies back your XMT */
u32 org_ts_s;
u32 org_ts_f;

/* Receive Timestamp (64-bit) - When the server received the packet */
u32 rec_ts_s;
u32 rec_ts_f;

/* Transmit Timestamp (64-bit) - When the server replied (THE TRUE TIME)
    */
u32 xmt_ts_s;
u32 xmt_ts_f;
} __attribute__((packed)) ca_ntp_pkt_t;
```

Represents a standard NTP (Network Time Protocol) packet structure used for network time synchronization.

Membri:

Nome	Tipo	Descrizione
`li_vn_mode`	`u8`	Bitfield containing Leap Indicator (2 bits), Version Number (3 bits), and Mode (3 bits).
`stratum`	`u8`	Indicates the distance from the reference clock (0 = unspecified, 1 = primary reference).
`poll`	`u8`	Maximum interval between successive messages, in log2 seconds.
`precision`	`i8`	Precision of the local clock, in log2 seconds.
`root_delay`	`u32`	Total round-trip delay to the reference clock, in 16.16 fixed-point format.
`root_dispersion`	`u32`	Maximum error relative to the reference clock, in 16.16 fixed-point format.
`ref_id`	`u32`	Reference clock identifier.
`ref_ts_s`	`u32`	Reference timestamp seconds part.
`ref_ts_f`	`u32`	Reference timestamp fraction part.
`org_ts_s`	`u32`	Origin timestamp seconds part (client transmit time).
`org_ts_f`	`u32`	Origin timestamp fraction part.
`rec_ts_s`	`u32`	Receive timestamp seconds part (server receive time).
`rec_ts_f`	`u32`	Receive timestamp fraction part.
`xmt_ts_s`	`u32`	Transmit timestamp seconds part (server transmit time).
`xmt_ts_f`	`u32`	Transmit timestamp fraction part.

Note:
The struct is marked with ___attribute___((packed)) to ensure no padding bytes are inserted by the compiler, which is critical for network protocol compliance. All multi-byte fields are typically expected in Network Byte Order (Big Endian).

Uso:
Used when constructing NTP request packets to be sent to a time server or when parsing the response packet received from a server to calculate network latency and clock offset.

Esempio:

```
ca_ntp_pkt_t ntp_req = {0};
ntp_req.li_vn_mode = 0x1B; // Version 3, Client mode
// ... populate other fields ...
// Send ntp_req via socket
// Receive response into another ca_ntp_pkt_t instance
```

`ca_ntp_sync_cb_t`

Tipo: Callback
File: `caos/net/inc/ntp_service.h:62:62`

```
typedef void (*ca_ntp_sync_cb_t)(u32 unix_time)
```

A callback function type triggered by the NTP service when a time synchronization event completes.

Membri:

Nome	Tipo	Descrizione
unix_time	u32	The current time in seconds since the Unix Epoch (January 1, 1970). If the value is 0, it indicates a synchronization failure.

Note:
This callback is executed within the context of the NTP service task. It must be kept short and non-blocking to avoid delaying network stack operations. The user does not allocate this; it is registered via the NTP service configuration.

Uso:
Use this callback to update the system RTC or application-level timestamps immediately after the device successfully synchronizes with an NTP server.

Esempio:

```
void my_ntp_callback(u32 unix_time) {
    if (unix_time > 0) {
        rtc_set_time(unix_time);
    }
}

// Registration
ca_ntp_config_t config = {
    .sync_cb = my_ntp_callback
};
ca_ntp_init(&config);
```

---

ca_ntp_synced_t

Tipo: Callback
File: caos/net/inc/net_cfg.h:113:113

```
typedef void (*ca_ntp_synced_t)(u32 unix_time);  /* called on sync */
```

A callback function pointer triggered by the caOS network stack when an NTP time synchronization event completes successfully.

Membri:

Nome	Tipo	Descrizione
unix_time	u32	The current time in seconds since the Unix Epoch (January 1, 1970). This value is provided by the NTP server and represents the synchronized system time.

Note:
This callback is executed within the context of the network task. It must be kept short and non-blocking to avoid delaying network packet processing. The user is responsible for assigning a valid function pointer to this type in the network configuration structure before the network stack is initialized.

Uso:
Used during the initialization of the caOS network stack. It allows the application to be notified immediately when the system clock has been updated via NTP, which is useful for logging, scheduling, or updating local RTC hardware.

Esempio:

```c
void my_ntp_callback(u32 unix_time) { printf("Time synchronized: %lu\n",
    unix_time); }

// In your network configuration setup:
ca_net_cfg_t config;
config.on_ntp_synced = my_ntp_callback;
ca_net_init(&config);
```

---

ca_opamp_input_t

Tipo: Enumerazione
File: caos/drivers/inc/opamp.h:30:33

```c
typedef enum {
CA_OPAMP_IN_PIN = 0,      /* OPAMPx_VINP pin */
CA_OPAMP_IN_DAC           /* DAC channel x, internal */
} ca_opamp_input_t;
```

Defines the signal source for the OPAMP non-inverting input, used during hardware configuration.

Membri:

Nome	Tipo	Descrizione
CA_OPAMP_IN_PIN	enum constant	Selects the physical external OPAMPx_VINP pin as the input source.
CA_OPAMP_IN_DAC	enum constant	Selects the internal DAC channel x output as the input source.

Note:
This enum is used to configure the internal routing of the STM32 OPAMP peripheral. It does not require allocation or freeing as it is a simple enumeration passed by value to initialization functions.

Uso:
Used in the ca_opamp_init function to specify whether the operational amplifier should sample an external voltage from a pin or an internal voltage generated by the DAC.

Esempio:

```c
#include "caos/drivers/inc/opamp.h"

void setup_opamp(void) {
    /* Configure OPAMP 1 to use the internal DAC as its input source */
    ca_opamp_input_t input_source = CA_OPAMP_IN_DAC;

    /* Initialize the OPAMP peripheral with the selected input */
    /* Note: ca_opamp_init is the standard function for this setup */
    ca_opamp_init(1, input_source);
}
```

---

ca_pool_cfg_t

Tipo: Struttura
File: caos/inc/pool.h:29:33

```
typedef struct {
u8            n_threads;   /**< number of worker threads (max 99) */
ca_stack_size_t stack_size;  /**< stack for each worker */
u16           priority;    /**< worker priority and one-shot fallback */
} ca_pool_cfg_t;
```

Configuration structure used to define the parameters for creating a thread pool in caOS.

Membri:

Nome	Tipo	Descrizione
`n_threads`	`u8`	The number of worker threads to spawn in the pool. Must be between 1 and 99. Setting this too high may lead to stack overflow or memory exhaustion.
`stack_size`	`ca_stack_size_t`	The size of the stack allocated for each individual worker thread. Ensure this is large enough for the tasks the threads will perform.
`priority`	`u16`	The scheduling priority assigned to the worker threads. Also acts as a fallback configuration for one-shot task execution.

Note:
This structure is typically passed by value to pool initialization functions. It does not own any memory itself; it is a template used by the kernel to allocate resources. Ensure the values are within system limits to avoid initialization failure.

Uso:
Use this structure when you need to initialize a thread pool to manage concurrent tasks. It is passed as an argument to the pool creation API to define the pool's footprint and scheduling behavior.

Esempio:

```
ca_pool_cfg_t my_pool_cfg = { .n_threads = 4, .stack_size = 512, .priority = 10
    }; ca_pool_init(&my_pool, &my_pool_cfg);
```

---

ca_pwr_info_t

Tipo: Struttura
File: `caos/drivers/inc/pwr.h:47:54`

```
typedef struct {
u32  sysclk_hz;        /*!< SYSCLK corrente */
u32  sysclk_max_hz;    /*!< SYSCLK nominale della scheda */
u8   vos;              /*!< voltage scaling corrente; 0xFF = non
   disponibile (F1) */
bool standby_resume;   /*!< true se il boot proviene da Standby */
u16  periph_on;        /*!< periferiche con clock RCC attivo */
u16  periph_total;     /*!< periferiche censite per la famiglia */
} ca_pwr_info_t;
```

A structure containing the current power and clock configuration status of the STM32 system.

Membri:

Nome	Tipo	Descrizione
sysclk_hz	u32	Current system clock frequency in Hertz. Used to calculate peripheral timings.
sysclk_max_hz	u32	Maximum rated frequency for the specific board. Used to validate overclocking limits.
vos	u8	Voltage scaling range index. Set to 0xFF if the hardware does not support dynamic voltage scaling (e.g., STM32F1).
standby_resume	bool	Flag indicating if the current execution started after waking up from Standby mode.
periph_on	u16	Bitmask or count of peripherals currently enabled in the RCC (Reset and Clock Control) registers.
periph_total	u16	Total number of peripherals available on the current MCU family.

Note:
This structure is typically populated by the caOS power driver. It is read-only for the application layer. Do not manually modify these fields as they reflect the hardware state managed by the kernel.

Uso:
Use this structure to perform runtime checks on system performance or to verify if the system is operating within safe power limits before enabling high-consumption peripherals.

Esempio:

```
ca_pwr_info_t pwr_status;

if (ca_pwr_get_info(&pwr_status) == CA_OK) {
    if (pwr_status.sysclk_hz < pwr_status.sysclk_max_hz) {
        // System is running underclocked
    }
    if (pwr_status.standby_resume) {
        // Perform specific wake-up initialization
    }
}
```

---

ca_pwr_mode_t

Tipo: Enumerazione
File: caos/drivers/inc/pwr.h:28:32

```
typedef enum {
PWR_MODE_SLEEP = 0,     /*!< WFI semplice, clock attivi; wake da
    qualsiasi IRQ */
PWR_MODE_STOP,          /*!< deepsleep, regolatore LP; RAM/periferiche
    ferme, wake      prosegue */
PWR_MODE_STANDBY,       /*!< RAM persa; wake      reset (vedi
    standby_resume in ca_pwr_info_t) */
} ca_pwr_mode_t;
```

An enumeration defining the low-power states available for the STM32 microcontroller in caOS.

Membri:

Nome	Tipo	Descrizione
CA_PWR_MODE_SLEEP	enum constant	Executes WFI (Wait For Interrupt) with clocks active. The CPU stops, but peripherals continue running. Wakeup occurs on any interrupt.
CA_PWR_MODE_STOP	enum constant	Enters deepsleep mode with the low-power regulator. RAM and peripheral states are preserved, but clocks are stopped. Wakeup resumes execution from the point of entry.
PWR_MODE_STANDBY	enum constant	Enters the lowest power state. RAM content is lost. Wakeup triggers a system reset, effectively restarting the application.

Note:
This enum is used to configure the power management unit. Be aware that PWR_MODE_STANDBY causes a full system reset, meaning all volatile data will be lost unless specifically handled in the boot sequence.

Uso:
Used as an argument in power management API functions to transition the MCU into a specific low-power state to conserve battery life.

Esempio:

```
ca_pwr_mode_t target_mode = CA_PWR_MODE_STOP;

// Transition the system to STOP mode
ca_pwr_set_mode(target_mode);
```

---

ca_pwr_periph_t

Tipo: Struttura
File: caos/drivers/inc/pwr.h:40:45

```
typedef struct {
const char   *name;     /*!< nome periferica, es. "USART1" */
const char   *bus;      /*!< bus RCC, es. "APB2" */
volatile u32 *enr;      /*!< registro RCC xxxENR */
u32           mask;     /*!< bit di clock enable */
} ca_pwr_periph_t;
```

Defines the hardware configuration required to enable or disable the clock for a specific STM32 peripheral.

Membri:

Nome	Tipo	Descrizione
name	const char *	Human-readable identifier for the peripheral (e.g., "USART1"). Used primarily for debugging and logging.
bus	const char *	The RCC bus name (e.g., "APB1", "APB2", "AHB1"). Used by the driver to identify the clock domain.
enr	volatile u32 *	Pointer to the specific RCC Enable Register (e.g., &RCC->APB2ENR). Must be a valid volatile memory address.
mask	u32	The specific bit position within the ENR register that controls the peripheral clock. Must be a single bit mask.

Note:
This structure is typically defined as a constant global array in the board support package (BSP). It does not require dynamic allocation. The driver treats these as read-only configuration descriptors. Ensure the mask matches the STM32 reference manual for the specific MCU series.

Uso:
Used by the caOS power management driver to perform low-level clock gating. It is passed to internal driver functions to safely toggle peripheral clocks during initialization or power-saving transitions.

Esempio:

```
static const ca_pwr_periph_t my_uart_cfg = {
    .name = "USART1",
    .bus = "APB2",
    .enr = (volatile u32 *)&RCC->APB2ENR,
    .mask = RCC_APB2ENR_USART1EN
};

/* The driver uses this struct to enable the clock */
ca_pwr_enable(&my_uart_cfg);
```

---

ca_qspi_cfg_t

Tipo: Struttura
File: caos/drivers/inc/qspi.h:36:47

```
typedef struct {
u8  clk_div;       /*!< Prescaler: periph_clk / (clk_div + 1)  */
u8  flash_size;    /*!< log2(size_bytes) - 1, e.g. 23 per 16MB */
u8  addr_bytes;    /*!< Byte di indirizzo: 3 (24-bit) o 4 (32-bit) */
u8  dummy_cycles;  /*!< Cicli dummy per il comando di lettura    */
u8  data_lines;    /*!< Linee dati: 1=single, 2=dual, 4=quad    */
u8  cmd_read;      /*!< Comando lettura veloce, es. 0xEB o 0x6B */
u8  cmd_page_prog; /*!< Page program, es. 0x32 o 0x02           */
u8  cmd_sector_er; /*!< Sector erase, es. 0x20                  */
u8  cmd_wren;      /*!< Write enable, 0x06                       */
u8  cmd_rdsr;      /*!< Read status reg, 0x05                    */
} ca_qspi_cfg_t;
```

Configuration structure used to define the communication parameters and command set for an external QSPI flash memory.

Membri:

Nome	Tipo	Descrizione
clk_div	u8	Clock prescaler value. The resulting frequency is periph_clk / (clk_div + 1). Ensure this does not exceed the flash chip's maximum rated frequency.
flash_size	u8	Log2 of the total flash size in bytes minus 1. Example: 23 for 16MB (2^24 bytes).
addr_bytes	u8	Number of address bytes used for memory access. Valid values are 3 (24-bit addressing) or 4 (32-bit addressing).
dummy_cycles	u8	Number of dummy cycles required by the flash chip for read operations. Incorrect values will result in corrupted data reads.
data_lines	u8	Number of data lines used for transfer. 1 for Single SPI, 2 for Dual SPI, 4 for Quad SPI.
cmd_read	u8	Opcode for the fast read command (e.g., 0xEB for Quad I/O Read).
cmd_page_prog	u8	Opcode for the page program command (e.g., 0x32 for Quad Page Program).
cmd_sector_er	u8	Opcode for the sector erase command (e.g., 0x20 for 4KB sector erase).
cmd_wren	u8	Opcode for the Write Enable command, typically 0x06.
cmd_rdsr	u8	Opcode for reading the flash status register, typically 0x05.

Note:
This struct is passed by value or pointer to the QSPI driver initialization function. It must be populated based on the specific datasheet of the external flash chip connected to the STM32. The user is responsible for ensuring the values match the hardware capabilities.

Uso:
Used during the initialization phase of the QSPI peripheral to configure the communication protocol and command set required to interface with external flash memory.

Esempio:

```
ca_qspi_cfg_t my_flash_cfg = {
    .clk_div = 3,
    .flash_size = 23,
    .addr_bytes = 3,
    .dummy_cycles = 6,
    .data_lines = 4,
    .cmd_read = 0xEB,
    .cmd_page_prog = 0x32,
    .cmd_sector_er = 0x20,
    .cmd_wren = 0x06,
    .cmd_rdsr = 0x05
};

ca_qspi_init(&my_flash_cfg);
```

---

ca_qspi_pin_t

Tipo: Enumerazione
File: caos/drivers/inc/qspi.h:145:148

```
typedef enum {
CA_QSPI_PIN_CLK = 0, CA_QSPI_PIN_NCS, CA_QSPI_PIN_IO0,
CA_QSPI_PIN_IO1, CA_QSPI_PIN_IO2, CA_QSPI_PIN_IO3, CA_QSPI_PIN_COUNT
} ca_qspi_pin_t;
```

An enumeration representing the specific physical pins used by the QSPI peripheral interface.

Membri:

Nome	Tipo	Descrizione
CA_QSPI_PIN_CLK	enum constant	The Serial Clock pin.
CA_QSPI_PIN_NCS	enum constant	The Chip Select (active low) pin.
CA_QSPI_PIN_IO0	enum constant	Data line 0 (often MOSI in single-line mode).
CA_QSPI_PIN_IO1	enum constant	Data line 1 (often MISO in single-line mode).
CA_QSPI_PIN_IO2	enum constant	Data line 2.
CA_QSPI_PIN_IO3	enum constant	Data line 3.
CA_QSPI_PIN_COUNT	enum constant	Helper value representing the total number of pins; do not use as a valid pin index.

Note:
This enum is used to index into configuration arrays or pin mapping tables. CA_QSPI_PIN_COUNT is provided to allow for easy array sizing and bounds checking. Ensure that the hardware design matches the pin mapping expected by the driver.

Uso:
Used primarily in driver configuration structures or initialization functions where specific GPIO pins must be assigned to the QSPI peripheral functionality.

Esempio:

```
ca_qspi_pin_t pin = CA_QSPI_PIN_CLK; if (pin < CA_QSPI_PIN_COUNT) { /* Proceed
    with pin configuration */ }
```

---

ca_qspi_xfer_t

Tipo: Struttura
File: caos/drivers/inc/qspi.h:123:135

```
typedef struct {
u8         cmd;
u8         cmd_lines;     /* 0/1 = single, 2, 4 (QPI devices)  */
u8         addr_bytes;    /* 0 = cfg.addr_bytes                 */
u8         addr_lines;
u32        addr;
u8         dummy;
u8         data_lines;
bool       read;          /* data phase direction              */
u8        *rx;            /* read: destination                 */
const u8 *tx;             /* write: source                     */
u32        len;           /* data bytes (0 with data_lines 0) */
} ca_qspi_xfer_t;
```

A structure defining the parameters for a single QSPI bus transaction, used to configure command, address, and data phases.

Membri:

Nome	Tipo	Descrizione
`cmd`	`u8`	The 8-bit instruction code to send to the QSPI device.
`cmd_lines`	`u8`	Number of data lines used for the command phase. Valid: 0 or 1 (Single), 2, or 4 (QPI).
`addr_bytes`	`u8`	Number of address bytes to transmit. Set to 0 to use the default configured value.
`addr_lines`	`u8`	Number of data lines used for the address phase.
`addr`	`u32`	The memory address on the external device to access.
`dummy`	`u8`	Number of dummy cycles required by the device before data transfer.
`data_lines`	`u8`	Number of data lines used for the data phase. Set to 0 if no data phase is required.
`read`	`bool`	Direction of the data phase. True for read (device to MCU), false for write (MCU to device).
`rx`	`u8 *`	Pointer to the buffer where received data will be stored. Only used if read is true.
`tx`	`const u8 *`	Pointer to the buffer containing data to be sent. Only used if read is false.
`len`	`u32`	Number of bytes to transfer during the data phase.

Note:
The structure must be fully initialized before passing it to the driver. Ensure that the buffers pointed to by rx or tx remain valid for the duration of the transfer. This struct is typically allocated on the stack by the caller.

Uso:
Used as an argument for QSPI transfer functions to perform read/write operations on external flash or memory-mapped devices.

Esempio:

```
ca_qspi_xfer_t xfer = {0};
xfer.cmd = 0x03; // Read command
xfer.cmd_lines = 1;
xfer.addr = 0x000000;
xfer.addr_bytes = 3;
xfer.addr_lines = 1;
xfer.data_lines = 1;
xfer.read = true;
xfer.rx = my_buffer;
xfer.len = 256;
ca_qspi_transfer(&xfer);
```

`ca_queue_static_t`

Tipo: Struttura
File: `caos/inc/queue.h:59:69`

```
typedef struct {
u32 item_size;                // Single item size (bytes)
```

```
u32 max_items;                // Maximum capacity
volatile u32 count;           // Number of items present
volatile u32 head;            // Read index (consumer)
volatile u32 tail;            // Write index (producer)
ca_wait_queue_t wait_send;        // Task waiting for send (queue full)
ca_wait_queue_t wait_recv;        // Task waiting for receive (queue
    empty)
ca_spinlock_irq_t lock;           // Concurrent access protection
u8 buffer[];                  // Flexible array member (inline buffer)
} ca_queue_static_t;
```

A statically allocated FIFO queue structure used for thread-safe data exchange between tasks.

Membri:

Nome	Tipo	Descrizione
item_size	u32	Size of a single element in the queue in bytes.
max_items	u32	Maximum number of items the queue can hold.
count	volatile u32	Current number of items stored in the queue.
head	volatile u32	Index for the next read operation.
tail	volatile u32	Index for the next write operation.
wait_send	ca_wait_queue_t	Internal wait queue for tasks blocked while trying to send to a full queue.
wait_recv	ca_wait_queue_t	Internal wait queue for tasks blocked while trying to receive from an empty queue.
lock	ca_spinlock_irq_t	Spinlock used to ensure atomic access to the queue structure.
buffer	u8[]	Flexible array member acting as the storage area for queue items.

Note:
The structure must be declared with enough space for the buffer. Since it uses a flexible array member, you typically define a wrapper struct containing the queue and a static array. It is initialized via ca_queue_static_init. Ownership is usually shared between producer and consumer tasks.

Uso:
Use this when you need a fixed-size, thread-safe FIFO buffer. It is ideal for passing data between ISRs and tasks or between two tasks without dynamic memory allocation.

Esempio:

```
#define QUEUE_SIZE 10
#define ITEM_SIZE sizeof(u32)

struct {
    ca_queue_static_t q;
    u8 storage[QUEUE_SIZE * ITEM_SIZE];
} my_queue_wrapper;

void setup_queue(void) {
    // Initialize the queue with the buffer address and capacity
    ca_queue_static_init(&my_queue_wrapper.q, my_queue_wrapper.storage,
    QUEUE_SIZE, ITEM_SIZE);
}

void task_producer(void) {
    u32 data = 42;
    // Send data to the queue
    ca_queue_static_send(&my_queue_wrapper.q, &data);
```

```c
}

void task_consumer(void) {
    u32 data;
    // Receive data from the queue
    ca_queue_static_receive(&my_queue_wrapper.q, &data);
}
```

---

ca_queue_t

Tipo: Struttura
File: caos/inc/queue.h:43:53

```c
typedef struct {
u8 *buffer;                    // Buffer items (allocated on heap)
u32 item_size;                 // Single item size (bytes)
u32 max_items;                 // Maximum capacity
volatile u32 count;            // Number of items present
volatile u32 head;             // Read index (consumer)
volatile u32 tail;             // Write index (producer)
ca_wait_queue_t wait_send;        // Task waiting for send (queue full)
ca_wait_queue_t wait_recv;        // Task waiting for receive (queue
    empty)
ca_spinlock_irq_t lock;           // Concurrent access protection
} ca_queue_t;
```

A thread-safe FIFO queue structure used for inter-task communication and data buffering.

Membri:

Nome	Tipo	Descrizione
buffer	u8 *	Pointer to the memory area storing the queue items. Must be allocated before initialization.
item_size	u32	Size of a single element in bytes.
max_items	u32	Maximum number of items the queue can hold.
count	volatile u32	Current number of items in the queue.
head	volatile u32	Index for the next read operation.
tail	volatile u32	Index for the next write operation.
wait_send	ca_wait_queue_t	Internal wait queue for tasks blocked on a full queue.
wait_recv	ca_wait_queue_t	Internal wait queue for tasks blocked on an empty queue.
lock	ca_spinlock_irq_t	Spinlock used to ensure atomic access to the queue structure.

Note:
The structure is typically initialized via ca_queue_heap_init or ca_queue_static_init. The user is responsible for providing the memory buffer if using static initialization. The internal spinlock handles concurrency, making it safe to use between tasks and ISRs.

Uso:
Use this type to pass data between producer and consumer tasks. It is the core structure for the ca_queue_* API family, which provides thread-safe push and pop operations.

Esempio:

```c
#include "caos/inc/queue.h"

```

```c
#define QUEUE_SIZE 10
#define ITEM_SIZE sizeof(u32)

static ca_queue_t my_queue;
static u8 queue_buffer[QUEUE_SIZE * ITEM_SIZE];

void setup_queue(void) {
    /* Initialize the queue with a static buffer */
    ca_queue_static_init(&my_queue, queue_buffer, ITEM_SIZE, QUEUE_SIZE);
}

void producer_task(void) {
    u32 data = 42;
    /* Send data to the queue */
    ca_queue_static_send(&my_queue, &data);
}

void consumer_task(void) {
    u32 received_data;
    /* Receive data from the queue */
    ca_queue_static_receive(&my_queue, &received_data);
}
```

---

ca_recursive_mutex_t

Tipo: Struttura
File: caos/inc/sync.h:114:119

```c
typedef struct {
volatile u32 owner;          // owner's ca_task_handle_t while
    lock_count > 0
volatile u16 lock_count;     // Reentrancy counter (0 = free)
ca_wait_queue_t wait_queue;      // Tasks waiting
ca_spinlock_irq_t internal_lock; // Protects internal structure
} ca_recursive_mutex_t;
```

A synchronization primitive that allows the same task to acquire the lock multiple times without deadlocking itself.

Membri:

Nome	Tipo	Descrizione
owner	volatile u32	Stores the handle of the task currently holding the mutex. Only valid if lock_count > 0.
lock_count	volatile u16	Tracks the nesting level of the lock. Incremented on lock, decremented on unlock. 0 means the mutex is available.
wait_queue	ca_wait_queue_t	Internal list of tasks blocked waiting for this mutex.
internal_lock	ca_spinlock_irq_t	Low-level spinlock used to ensure atomic access to the structure members during state changes.

Note:
The recursive mutex must be initialized using ca_rmutex_init before use. It is typically allocated in the global data segment or as part of a larger control structure. The owner must be the same task that performs the unlock; attempting to unlock a mutex held by another task is undefined behavior.

Uso:

Use this when a task needs to call multiple functions that all require the same resource, and each function independently attempts to lock the resource. It prevents the task from blocking on its own held lock.

Esempio:

```c
#include "caos/inc/sync.h"

ca_recursive_mutex_t my_mutex;

void task_function(void) {
    ca_rmutex_init(&my_mutex);

    // First acquisition
    ca_rmutex_lock(&my_mutex);

    // Nested acquisition (allowed because it is recursive)
    ca_rmutex_lock(&my_mutex);

    // Perform protected operations here

    // Must unlock as many times as it was locked
    ca_rmutex_unlock(&my_mutex);
    ca_rmutex_unlock(&my_mutex);
}
```

`ca_route_entry_t`

Tipo: Struttura
File: `caos/net/inc/route_service.h:34:39`

```c
typedef struct {
ca_ipv4_t dst;
ca_ipv4_t mask;
ca_ipv4_t gw;
bool   active;
} ca_route_entry_t;
```

Represents a single entry in the system routing table used to determine the next hop for outgoing network packets.

Membri:

Nome	Tipo	Descrizione
`dst`	`ca_ipv4_t`	The destination IPv4 address or network prefix. Must be a valid IPv4 structure.
`mask`	`ca_ipv4_t`	The subnet mask associated with the destination. Defines the range of the network.
`gw`	`ca_ipv4_t`	The gateway (next hop) IP address. If the destination is local, this may be set to 0.0.0.0.
`active`	`bool`	Status flag. Set to true to enable the route, false to ignore it during lookup.

Note:
The routing table is typically managed by the network stack. Ensure that the mask is correctly aligned with the destination address to avoid routing loops or unreachable destinations. This struct is usually allocated in static memory or as part of a configuration array.

Uso:

Used when configuring static routes in the network stack or when querying the routing table to determine how to forward packets to specific subnets.

Esempio:

```
ca_route_entry_t my_route = {
    .dst = {192, 168, 1, 0},
    .mask = {255, 255, 255, 0},
    .gw = {192, 168, 1, 1},
    .active = true
};

// Add the route to the system table
ca_route_add(&my_route);
```

---

ca_rtc_alarm_cb_t

Tipo: Callback
File: `caos/drivers/inc/rtc.h:64:64`

```
typedef void (*ca_rtc_alarm_cb_t)(u8 alarm_id)
```

A function pointer type used to define a callback function that executes when a specific RTC alarm triggers.

Membri:

Nome	Tipo	Descrizione
`alarm_id`	`u8`	The identifier of the alarm that triggered the event. Valid values are typically 0 or 1 depending on the hardware configuration. Providing an invalid ID may lead to logic errors in the handler.

Note:
This callback is executed within the context of the RTC interrupt service routine (ISR). Therefore, the function must be kept extremely short, must not perform blocking operations (like waiting for semaphores or mutexes), and should avoid heavy processing. The user is responsible for defining the function and passing its address to the RTC driver configuration.

Uso:
Use this type when you need to perform an action (e.g., setting a flag, waking a task) immediately upon an RTC alarm event. It is typically passed as an argument during the RTC initialization or alarm configuration phase.

Esempio:

```
void my_alarm_handler(u8 alarm_id) {
    if (alarm_id == 0) {
        // Signal a task that the alarm has fired
        os_event_set(alarm_event_handle);
    }
}

// Usage during configuration
ca_rtc_config_alarm(0, &my_alarm_handler);
```

---

ca_rtc_alarm_t

Tipo: Struttura
File: caos/drivers/inc/rtc.h:52:61

```
typedef struct {
u8   sec;          /* 0  59    */
u8   min;          /* 0  59    */
u8   hour;         /* 0  23    */
u8   day;          /* 1  31    */
bool match_sec;    /* true = confronta secondi   */
bool match_min;    /* true = confronta minuti    */
bool match_hour;   /* true = confronta ore       */
bool match_day;    /* true = confronta giorno    */
} ca_rtc_alarm_t;
```

A configuration structure used to define the trigger conditions for a Real-Time Clock (RTC) alarm event.

Membri:

Nome	Tipo	Descrizione
sec	u8	Seconds value for the alarm. Valid range: 0-59. Ignored if match_sec is false.
min	u8	Minutes value for the alarm. Valid range: 0-59. Ignored if match_min is false.
hour	u8	Hours value for the alarm (24h format). Valid range: 0-23. Ignored if match_hour is false.
day	u8	Day of the month for the alarm. Valid range: 1-31. Ignored if match_day is false.
match_sec	bool	If true, the RTC will trigger the alarm when the current seconds match the 'sec' field.
match_min	bool	If true, the RTC will trigger the alarm when the current minutes match the 'min' field.
match_hour	bool	If true, the RTC will trigger the alarm when the current hours match the 'hour' field.
match_day	bool	If true, the RTC will trigger the alarm when the current day matches the 'day' field.

Note:
The structure is typically allocated on the stack by the user. The RTC hardware only checks fields where the corresponding match flag is set to true. If all match flags are false, the alarm behavior is undefined or may never trigger.

Uso:
Use this structure to configure an RTC alarm via the caOS driver API. Populate the fields, set the desired match flags, and pass the structure pointer to the RTC alarm configuration function.

Esempio:

```
ca_rtc_alarm_t my_alarm = { .sec = 30, .match_sec = true, .match_min = false, .
    match_hour = false, .match_day = false }; ca_rtc_set_alarm(&my_alarm);
```

---

ca_rtc_clk_src_t

Tipo: Enumerazione
File: caos/drivers/inc/rtc.h:30:34

```c
typedef enum {
CA_RTC_CLK_LSE = 1,    /* 32.768 kHz cristallo esterno */
CA_RTC_CLK_LSI = 2,    /* ~32 kHz RC interno (no cristallo) */
CA_RTC_CLK_HSE = 3,    /* HSE/32 */
} ca_rtc_clk_src_t;
```

Defines the clock source selection for the Real-Time Clock (RTC) peripheral.

Membri:

Nome	Tipo	Descrizione
`CA_RTC_CLK_LSE`	`enum value`	External 32.768 kHz crystal. Provides the highest accuracy for timekeeping.
`CA_RTC_CLK_LSI`	`enum value`	Internal Low-Speed RC oscillator. Does not require external components but is less accurate and temperature-dependent.
`CA_RTC_CLK_HSE`	`enum value`	High-Speed External clock divided by 32. Used when an external crystal is available but LSE is not populated.

Note:
The clock source must be configured before initializing the RTC peripheral. Changing the clock source while the RTC is running may cause unpredictable behavior or time drift. Ensure the hardware board actually supports the selected source (e.g., verify the presence of an LSE crystal).

Uso:
Used during the RTC initialization phase to tell the driver which hardware oscillator to use as the timebase for the RTC counter.

Esempio:

```c
ca_rtc_config_t rtc_cfg;
rtc_cfg.clk_src = CA_RTC_CLK_LSE;
rtc_cfg.format = CA_RTC_FORMAT_BIN;
ca_rtc_init(&rtc_cfg);
```

---

ca_rtc_dt_t

Tipo: Struttura
File: `caos/drivers/inc/rtc.h:37:45`

```c
typedef struct {
u8  sec;     /* 0  59    */
u8  min;     /* 0  59    */
u8  hour;    /* 0  23    */
u8  wday;    /* 1  7     (1=luned ) */
u8  day;     /* 1  31    */
u8  month;   /* 1  12    */
u8  year;    /* 0  99    (relativo al 2000) */
} ca_rtc_dt_t;
```

A structure representing a calendar date and time for the RTC peripheral.

Membri:

Nome	Tipo	Descrizione
sec	u8	Seconds, range 0-59. Values outside this range may cause hardware errors.
min	u8	Minutes, range 0-59.
hour	u8	Hours in 24-hour format, range 0-23.
wday	u8	Day of the week, range 1-7 where 1 is Monday.
day	u8	Day of the month, range 1-31.
month	u8	Month of the year, range 1-12.
year	u8	Year offset from 2000, range 0-99 (e.g., 24 represents 2024).

Note:
This struct is used to interface with the RTC hardware. It does not perform validation; ensure values are within the specified ranges before passing to driver functions to avoid undefined behavior in the RTC peripheral.

Uso:
Used as a parameter for RTC driver functions to set or retrieve the current system time and date.

Esempio:

```
ca_rtc_dt_t my_time = { .sec = 0, .min = 30, .hour = 14, .wday = 1, .day = 15, .month = 5, .year = 24 };
ca_rtc_set_time(&my_time);
```

---

ca_rwlock_t

Tipo: Struttura
File: caos/inc/sync.h:309:315

```
typedef struct {
volatile u16  readers;       /* active readers */
volatile u8   writer_active; /* writer in progress */
ca_wait_queue_t   read_queue;    /* waiting readers */
ca_wait_queue_t   write_queue;   /* waiting writers */
ca_spinlock_irq_t internal_lock;
} ca_rwlock_t;
```

A synchronization primitive that allows multiple concurrent readers or a single exclusive writer to access a shared resource.

Membri:

Nome	Tipo	Descrizione
readers	volatile u16	Counter of currently active readers. Incremented when a reader acquires the lock, decremented when released.
writer_active	volatile u8	Flag indicating if a writer currently holds the lock. 1 if active, 0 otherwise.
read_queue	ca_wait_queue_t	Queue of tasks blocked waiting to acquire the lock for reading.
write_queue	ca_wait_queue_t	Queue of tasks blocked waiting to acquire the lock for writing.
internal_lock	ca_spinlock_irq_t	Internal spinlock used to ensure atomic access to the rwlock structure members during state changes.

Note:
The structure must be initialized before use. It is typically allocated in static memory or as part of a larger control block. Ownership is shared among tasks accessing the resource. Do not modify members directly; use the provided ca_rwlock API functions to ensure thread safety.

Uso:
Use this when you have a data structure that is read frequently but updated rarely. It improves performance over a standard mutex by allowing multiple readers to access the data simultaneously while ensuring exclusive access for writers.

Esempio:

```
ca_rwlock_t my_data_lock;

void init_system(void) {
    ca_rwlock_init(&my_data_lock);
}

void task_reader(void) {
    ca_rwlock_read_lock(&my_data_lock);
    /* Perform read operations */
    ca_rwlock_read_unlock(&my_data_lock);
}

void task_writer(void) {
    ca_rwlock_write_lock(&my_data_lock);
    /* Perform write operations */
    ca_rwlock_write_unlock(&my_data_lock);
}
```

---

ca_sai_config_t

Tipo: Struttura
File: caos/drivers/inc/sai.h:55:59

```
typedef struct {
u32             fs_hz;    /* requested sample rate */
u8              bits;     /* 16, 24 or 32 data bits */
ca_sai_proto_t  proto;
} ca_sai_config_t;
```

Configuration structure for the Serial Audio Interface (SAI) peripheral, defining sample rate, bit depth, and protocol.

Membri:

Nome	Tipo	Descrizione
fs_hz	u32	The desired audio sample rate in Hertz (e.g., 44100, 48000). Must be supported by the hardware clock configuration.
bits	u8	Data word length in bits. Valid values are 16, 24, or 32.
proto	ca_sai_proto_t	The audio protocol mode (e.g., I2S, Left-Justified, Right-Justified). Must be a valid enum value from ca_sai_proto_t.

Note:
This struct is passed by value to the initialization function. It does not require dynamic allocation; it is typically declared on the stack before calling the initialization routine. Ensure the hardware clock is configured to support the requested fs_hz before calling ca_sai_init.

Uso:
Used as an input parameter for the ca_sai_init function to configure the SAI peripheral hardware registers before starting audio streaming.

Esempio:

```c
#include "caos/drivers/inc/sai.h"

void setup_audio(void) {
    ca_sai_config_t config;

    config.fs_hz = 48000;
    config.bits = 16;
    config.proto = CA_SAI_PROTO_I2S; // Assuming CA_SAI_PROTO_I2S is a valid enum member

    // Initialize the SAI peripheral with the defined configuration
    ca_sai_init(0, &config);
}
```

---

ca_sai_proto_t

Tipo: Enumerazione
File: caos/drivers/inc/sai.h:50:53

```c
typedef enum {
CA_SAI_I2S = 0,      /* Philips I2S: FS low = left, data one bit after FS */
CA_SAI_MSB           /* MSB-justified: FS high = left, data on the FS edge */
} ca_sai_proto_t;
```

Defines the digital audio protocol format (I2S or MSB-justified) used for SAI peripheral communication.

Membri:

Nome	Tipo	Descrizione
CA_SAI_I2S	enum value	Standard Philips I2S format. Frame Sync (FS) is low for the left channel; data is delayed by one bit clock cycle relative to FS.
CA_SAI_MSB	enum value	MSB-justified format. Frame Sync (FS) is high for the left channel; data is aligned exactly with the FS edge.

Note:
This enum is used to configure the hardware interface behavior. Ensure the selected protocol matches the requirements of your external audio codec or DAC/ADC chip. Incorrect selection will result in bit-shifted or garbled audio data.

Uso:
Used during the initialization of the SAI peripheral to specify the data alignment and framing protocol.

Esempio:

```c
#include "caos/drivers/inc/sai.h"

void setup_audio(void) {
    /* Define the protocol configuration */
```

```
    ca_sai_proto_t my_protocol = CA_SAI_I2S;

    /* Initialize the SAI peripheral with the chosen protocol */
    /* Note: ca_sai_init is the standard function to apply these settings */
    ca_sai_init(my_protocol);
}
```

---

ca_scheduler_tasks_info_mode

Tipo: Enumerazione
File: caos/inc/scheduler.h:419:425

```
typedef enum ca_scheduler_tasks_info_mode_tag{
CA_TH_INFO_DEFAULT=0,
CA_TH_INFO_FLAG,
TH_INFO_RUNTIME,
CA_TH_INFO_MEM,
TH_INFO_ASSERT
} ca_scheduler_tasks_info_mode;
```

An enumeration used to specify the level of detail or the specific category of information to retrieve when querying task statistics.

Membri:

Nome	Tipo	Descrizione
CA_TH_INFO_DEFAULT	enum constant	Retrieves standard task information. Use this for general monitoring.
CA_TH_INFO_FLAG	enum constant	Retrieves task flag status. Useful for debugging task state transitions.
TH_INFO_RUNTIME	enum constant	Retrieves execution time statistics. Essential for profiling and identifying bottlenecks.
CA_TH_INFO_MEM	enum constant	Retrieves stack usage and memory allocation data. Critical for preventing stack overflows.
TH_INFO_ASSERT	enum constant	Retrieves assertion-related diagnostic data. Used primarily for error tracking.

Note:
This enum is passed as an argument to scheduler diagnostic functions. It does not require manual allocation as it is a simple integer-based enumeration. Ensure the mode passed is supported by the specific diagnostic function being called to avoid undefined behavior.

Uso:
Use this type when calling scheduler diagnostic APIs to filter the output data. It allows you to request only the specific metrics needed, reducing overhead and keeping logs clean.

Esempio:

```
ca_scheduler_tasks_info_mode mode = CA_TH_INFO_MEM;
ca_scheduler_get_task_info(task_handle, mode, &buffer);
```

---

ca_secure_key_entry_t

Tipo: Struttura
File: caos/crypto/inc/keys_store.h:49:57

```
typedef struct {
u32 base_address;               /* Base address of the key within the
    common area */
u32 size;                       /* Key size */
u32 validity_start;             /* Packed validity start date (0 = no
    limit) */
u32 validity_stop;              /* Packed validity end date (0 = no
    limit) */
u8 is_allocated;                /* Flag: key allocated */
u8 access_mode;                 /* Current access mode */
u16 key_id;                     /* Unique key ID */
} ca_secure_key_entry_t;
```

Represents a metadata entry for a cryptographic key stored in the secure common area.

Membri:

Nome	Tipo	Descrizione
base_address	u32	The starting memory address of the key in the secure storage area. Must be aligned according to hardware requirements.
size	u32	The size of the key in bytes. Must match the actual key data length to prevent buffer overflows.
validity_start	u32	Packed date format representing when the key becomes active. Set to 0 to disable this check.
validity_stop	u32	Packed date format representing when the key expires. Set to 0 to disable this check.
is_allocated	u8	Boolean flag indicating if the entry is currently in use. 1 for allocated, 0 for free.
access_mode	u8	Bitmask defining read/write/execute permissions for the key. Incorrect values may cause hardware access faults.
key_id	u16	A unique identifier used to reference the key during cryptographic operations.

Note:
This structure is typically managed by the caOS key store manager. The application should treat this as read-only unless using specific management APIs. Ensure the memory region pointed to by base_address is initialized before setting is_allocated to 1.

Uso:
Used when querying the key store for metadata or when registering a new key into the secure storage system via the caOS crypto subsystem.

Esempio:

```
ca_secure_key_entry_t my_key = { .base_address = 0x08010000, .size = 32, .
    validity_start = 0, .validity_stop = 0, .is_allocated = 1, .access_mode = 0
    x01, .key_id = 0x0001 };
```

---

ca_secure_pipe_end_t

Tipo: Enumerazione
File: caos/crypto/inc/secure_pipe_service.h:37:40

```
typedef enum {
CA_SECURE_PIPE_A = 0U,
```

```
CA_SECURE_PIPE_B = 1U,
} ca_secure_pipe_end_t;
```

Identifies the specific endpoint (A or B) of a secure communication pipe.

Membri:

Nome	Tipo	Descrizione
`CA_SECURE_PIPE_A`	`enum constant`	Represents the first endpoint of the secure pipe. Must be used consistently with the peer configuration.
`CA_SECURE_PIPE_B`	`enum constant`	Represents the second endpoint of the secure pipe. Must be used consistently with the peer configuration.

Note:
This enum is used to distinguish between the two sides of a secure pipe connection. It is typically passed to initialization or communication functions to ensure data is routed to the correct logical channel. Ensure that the local and remote ends are configured to use opposite identifiers if required by the protocol.

Uso:
Used as a parameter in ca_secure_pipe_init, ca_secure_pipe_send, and ca_secure_pipe_recv to specify which pipe end is being accessed.

Esempio:

```c
#include "caos/crypto/inc/secure_pipe_service.h"

void setup_pipe(void) {
    ca_secure_pipe_end_t my_end = CA_SECURE_PIPE_A;

    /* Initialize the secure pipe service for endpoint A */
    ca_secure_pipe_init(my_end);

    uint8_t data[] = {0x01, 0x02, 0x03};
    /* Send data through the initialized pipe end */
    ca_secure_pipe_send(my_end, data, sizeof(data));
}
```

`ca_secure_pipe_t`

Tipo: Struttura
File: `caos/crypto/inc/secure_pipe_service.h:42:46`

```c
typedef struct {
ca_aes_channel_t ch;
ca_queue_t       q[2];    /* q[e]: ciphertext waiting to be read by end
    e */
bool             ready;
} ca_secure_pipe_t;
```

Represents a secure, encrypted communication channel between two endpoints using AES encryption.

Membri:

Nome	Tipo	Descrizione
ch	ca_aes_channel_t	The underlying AES cryptographic channel used for encrypting and decrypting data.
q	ca_queue_t[2]	Two internal queues: q[0] holds ciphertext for endpoint 0, q[1] for endpoint 1. Do not manipulate these directly.
ready	bool	Flag indicating if the pipe is fully initialized and ready for secure data transmission.

Note:
The structure must be initialized via ca_secure_pipe_init. The user is responsible for allocating the memory for the struct, but the internal queues and AES channel are managed by the ca_secure_pipe service. Ensure the pipe is deinitialized with ca_secure_pipe_deinit before freeing the memory to prevent resource leaks.

Uso:
Used to establish a secure, encrypted data stream between two tasks or system components. Use ca_secure_pipe_send to push data into the pipe and ca_secure_pipe_recv to retrieve decrypted data from the other end.

Esempio:

```c
#include "caos/crypto/inc/secure_pipe_service.h"

ca_secure_pipe_t my_pipe;

void setup_secure_channel(void) {
    /* Initialize the secure pipe structure */
    ca_secure_pipe_init(&my_pipe);
}

void send_data(const uint8_t *data, size_t len) {
    if (my_pipe.ready) {
        /* Send encrypted data through the pipe */
        ca_secure_pipe_send(&my_pipe, data, len);
    }
}

void receive_data(uint8_t *buffer, size_t max_len) {
    /* Retrieve and decrypt data from the pipe */
    ca_secure_pipe_recv(&my_pipe, buffer, max_len);
}
```

---

ca_sem_name_t

Tipo: union
File: caos/inc/sync.h:174:177

```c
typedef union ca_sem_name_u{
u8 name_c[8];
u64 name_l;
} ca_sem_name_t;
```

A union used to provide a unique identifier for a semaphore, allowing it to be represented as either an 8-character string or a 64-bit integer.

Membri:

Nome	Tipo	Descrizione
name_c	u8[8]	An array of 8 characters. Used for human-readable names. If the string is shorter than 8 bytes, ensure the remaining bytes are null-terminated.
name_l	u64	A 64-bit integer representation of the name. Useful for faster comparisons or when a numeric ID is preferred over a string.

Note:
This is a union, meaning both members share the same memory location. Writing to name_c will overwrite name_l and vice versa. It is typically passed by value to semaphore creation functions. No dynamic allocation is required as it is usually declared on the stack or as a static member.

Uso:
Used when initializing a semaphore to give it a unique name for debugging or tracking purposes within the RTOS kernel. It is passed as an argument to semaphore initialization functions to identify the resource.

Esempio:

```
ca_sem_name_t my_sem_name; my_sem_name.name_c[0] = 'U'; my_sem_name.name_c[1] = 'A'; my_sem_name.name_c[2] = 'R'; my_sem_name.name_c[3] = 'T'; my_sem_name.name_c[4] = '\0'; // Initialize semaphore with this name
```

---

ca_semaphore_t

Tipo: Struttura
File: caos/inc/sync.h:180:186

```
typedef struct __attribute__((aligned(4))) {
volatile u32 count;           // Available resource counter
u32 max_count;                // Maximum value
ca_wait_queue_t wait_queue;       // Tasks waiting
ca_spinlock_irq_t internal_lock; // Protects internal structure
ca_sem_name_t name;           // added for debug
} ca_semaphore_t;
```

A counting semaphore used to manage access to a limited number of shared resources or to synchronize tasks.

Membri:

Nome	Tipo	Descrizione
count	volatile u32	Current number of available resources. Decremented by wait, incremented by post.
max_count	u32	The upper limit for the semaphore count. Prevents overflow if set correctly.
wait_queue	ca_wait_queue_t	Internal list of tasks currently blocked waiting for this semaphore.
internal_lock	ca_spinlock_irq_t	Spinlock used to ensure atomic access to the semaphore structure members.
name	ca_sem_name_t	Optional string identifier for debugging purposes.

Note:
The semaphore must be initialized using ca_sem_init before use. It is typically allocated in the global data segment or as part of a larger control structure. Ownership is usually shared among

tasks that need to synchronize. Do not modify members directly; always use the ca_sem_* API functions.

Uso:
Use this type when you need to limit the number of tasks accessing a resource (e.g., a pool of buffers) or to signal between tasks. Use ca_sem_init to set the initial and maximum count, ca_sem_wait to acquire a resource, and ca_sem_post to release it.

Esempio:

```c
#include "caos/inc/sync.h"

ca_semaphore_t my_sem;

void task_init(void) {
    /* Initialize semaphore with 0 initial count and 5 max count */
    ca_sem_init(&my_sem, 0, 5);
}

void producer_task(void) {
    /* Signal that a resource is available */
    ca_sem_post(&my_sem);
}

void consumer_task(void) {
    /* Wait for a resource to become available */
    if (ca_sem_wait(&my_sem) == 0) {
        /* Resource acquired, perform work */
    }
}
```

---

ca_sha256_ctx_t

Tipo: Struttura
File: caos/crypto/inc/sha256.h:43:51

```c
typedef struct {
u32 state[8];                   /* Stato interno H0..H7 */
u64 bitlen;                     /* Lunghezza messaggio in bit (64 bit)
    */
u8  buffer[SHA256_BLOCK_SIZE];  /* Blocco parziale in accumulo */
u32 buflen;                     /* Byte validi in buffer */
u32 W[64];                      /* Message schedule     nel ctx, non su
    stack:
* la compressione SW    profonda e sui target
* senza HASH HW (F407) graverebbe lo stack. */
} ca_sha256_ctx_t;
```

A structure holding the internal state and working memory for the SHA-256 hashing algorithm.

Membri:

Nome	Tipo	Descrizione
`state`	`u32[8]`	The internal hash state (H0-H7). Initialized by ca_sha256_init and updated during processing.
`bitlen`	`u64`	Total length of the message processed so far in bits. Must be updated correctly to ensure final hash integrity.
`buffer`	`u8[64]`	Temporary storage for incoming data chunks that are not yet a full 64-byte block.
`buflen`	`u32`	Number of valid bytes currently stored in the buffer. Must be less than 64.
`W`	`u32[64]`	Message schedule array. Stored in the context to prevent stack overflow on memory-constrained targets like STM32F407.

Note:
The context must be initialized using ca_sha256_init before any update or finalization. It is typically allocated by the caller (on the stack or as a static/global variable). The user is responsible for the memory lifecycle of this struct. Do not manually modify these fields; use the provided API functions.

Uso:
Use this type to maintain the state of a SHA-256 calculation across multiple data chunks. It is passed as a pointer to ca_sha256_init, ca_sha256_update, and ca_sha256_final.

Esempio:

```
ca_sha256_ctx_t ctx; uint8_t hash[32]; ca_sha256_init(&ctx); ca_sha256_update(&ctx, (uint8_t*)"hello", 5); ca_sha256_final(&ctx, hash);
```

`ca_sha512_ctx_t`

Tipo: Struttura
File: `caos/crypto/inc/sha512.h:42:51`

```
typedef struct {
u64 state[8];                   /* Stato interno H0..H7 */
u64 bitlen_lo;                  /* Lunghezza messaggio in bit (parte bassa) */
u64 bitlen_hi;                  /* Lunghezza messaggio in bit (parte alta) */
u8  buffer[SHA512_BLOCK_SIZE];  /* Blocco parziale in accumulo */
u32 buflen;                     /* Byte validi in buffer */
u64 W[80];                      /* Message schedule     nel ctx, non su stack:
* 640B che graverebbero lo stack del task
* (Ed25519 chiama SHA-512 in profondit ). */
} ca_sha512_ctx_t;
```

A context structure used to maintain the internal state of a SHA-512 hashing operation across multiple data chunks.

Membri:

Nome	Tipo	Descrizione
state	u64[8]	The internal hash state (H0-H7). Initialized by the hash start function.
bitlen_lo	u64	Lower 64 bits of the total message length in bits. Used for final padding.
bitlen_hi	u64	Upper 64 bits of the total message length in bits.
buffer	u8[128]	Temporary storage for incoming data that does not yet fill a full 128-byte block.
buflen	u32	Number of valid bytes currently stored in the buffer. Must be less than 128.
W	u64[80]	Message schedule array. Stored here to prevent stack overflow in deep call stacks.

Note:
The structure must be initialized before use. It is typically allocated by the caller on the stack or as a static variable. It does not require dynamic memory allocation (malloc). Do not manually modify the fields after initialization.

Uso:
Use this structure when you need to hash data that arrives in fragments or is too large to fit in memory at once. Pass a pointer to this struct to the initialization, update, and finalization functions of the SHA-512 API.

Esempio:

```
ca_sha512_ctx_t ctx;
u8 hash_result[64];

ca_sha512_init(&ctx);
ca_sha512_update(&ctx, (u8*)"Hello ", 6);
ca_sha512_update(&ctx, (u8*)"World", 5);
ca_sha512_final(&ctx, hash_result);
```

---

ca_sp_link_stats_t

Tipo: Struttura
File: caos/crypto/inc/secure_pipe_service.h:135:142

```
typedef struct {
u32 hello_ok, hello_bad;        /* handshakes accepted / rejected
    */
u32 sessions;                   /* session keys derived
    */
u32 sealed, opened;
u32 tag_fail;                   /* forged, tampered or wrong key
    */
u32 replay;                     /* duplicate or out of the window
    */
u32 bad_format;                 /* wrong type, length, size
    */
} ca_sp_link_stats_t;
```

A structure containing diagnostic counters for a secure pipe link, used to monitor connection health and security events.

Membri:

Nome	Tipo	Descrizione
hello_ok	u32	Number of successful handshake attempts.
hello_bad	u32	Number of failed handshake attempts.
sessions	u32	Total number of session keys successfully derived.
sealed	u32	Number of packets successfully encrypted/sealed.
opened	u32	Number of packets successfully decrypted/opened.
tag_fail	u32	Count of authentication failures (forged or tampered data).
replay	u32	Count of packets rejected due to replay protection (duplicate or out-of-window).
bad_format	u32	Count of packets rejected due to malformed headers or invalid lengths.

Note:
This structure is typically populated by the secure pipe service. It is a read-only snapshot for the user. The application does not need to allocate memory for it; it is usually passed as a pointer to be filled by the underlying secure pipe driver.

Uso:
Use this to monitor the security integrity of a communication channel. If 'tag_fail' or 'replay' counters increase rapidly, it may indicate an active attack or severe synchronization issues between peers.

Esempio:

```c
#include "caos/crypto/inc/secure_pipe_service.h"

void monitor_link(void *pipe_handle) {
    ca_sp_link_stats_t stats;

    /* Clear the stats structure before use */
    ca_memzero(&stats, sizeof(ca_sp_link_stats_t));

    /*
     * Retrieve current statistics from the secure pipe.
     * Note: Assuming a hypothetical function to fetch stats exists
     * or the stats are retrieved via a specific service call.
     */
    if (stats.tag_fail > 0) {
        ca_printf("Security Alert: %u authentication failures detected!\n",
    stats.tag_fail);
    }
}
```

---

ca_sp_link_t

Tipo: Struttura
File: caos/crypto/inc/secure_pipe_service.h:144:163

```c
typedef struct {
u8              role;               /* 1 or 2
    */
bool            ready;              /* init done
    */
bool            up;                 /* session keys derived
    */
bool            my_fresh;           /* my_nonce sent, not paired yet
    */
```

```
bool           peer_pending;     /* peer nonce waiting for my hello()
    */
u8             my_nonce[16];
u8             peer_nonce[16];
void          *k_tx, *k_rx;      /* static direction keys (keys_store)
    */
void          *s_tx, *s_rx;      /* session AES contexts (keys_store),
   set
* up once per session, only the nonce
* changes per message                    */
ca_aes_ctx_t *ctx;               /* heap, set up per message
    */
ca_mutex_t     mutex;
u64            tx_ctr;           /* last counter sent
    */
u64            rx_top;           /* highest counter accepted
    */
u64            rx_win;           /* bit i = rx_top - i already accepted
    */
u32            session_ms;       /* ca_os_tick_ms() at derivation
    */
ca_sp_link_stats_t st;
} ca_sp_link_t;
```

Represents the state and cryptographic context of a secure communication pipe between two nodes.

Membri:

Nome	Tipo	Descrizione
role	u8	Defines the node role (1 or 2). Used to determine key derivation order.
ready	bool	Indicates if the structure has been successfully initialized.
up	bool	Indicates if session keys are derived and the pipe is ready for encrypted traffic.
my_fresh	bool	Flag indicating if the local nonce has been sent but pairing is incomplete.
peer_pending	bool	Flag indicating a peer nonce is waiting for a local hello response.
my_nonce	u8[16]	Local random nonce used for session key derivation.
peer_nonce	u8[16]	Remote peer nonce used for session key derivation.
k_tx, k_rx	void*	Pointers to static direction keys stored in the keys_store.
s_tx, s_rx	void*	Pointers to session AES contexts; initialized once per session.
ctx	ca_aes_ctx_t*	Pointer to heap-allocated AES context, re-initialized per message.
mutex	ca_mutex_t	Mutex to ensure thread-safe access to the pipe state.
tx_ctr	u64	Monotonic counter for outgoing messages to prevent replay attacks.
rx_top	u64	Highest sequence number accepted from the peer.
rx_win	u64	Sliding window bitmask for tracking accepted incoming sequence numbers.
session_ms	u32	Timestamp (ca_os_tick_ms) when the session was established.
st	ca_sp_link_stats_t	Statistics structure for tracking pipe performance and errors.

Note:
The structure must be initialized via ca_sp_link_init. The 'ctx' member is heap-allocated and must be managed carefully to avoid leaks. The mutex must be locked before accessing any fields during transmission or reception. Ownership of the link structure is typically held by the task managing the secure connection.

Uso:
Used by the secure pipe service to maintain session state. It is passed as a handle to functions like ca_sp_link_hello, ca_sp_link_accept, and ca_sp_link_seal to perform cryptographic operations and manage the secure channel lifecycle.

Esempio:

```
ca_sp_link_t my_pipe;
ca_mutex_t pipe_mutex;

/* Initialize the mutex before the pipe */
ca_mutex_init(&pipe_mutex);

/* Initialize the secure pipe link structure */
ca_sp_link_init(&my_pipe, 1, &pipe_mutex);

/* Check if the pipe is ready for communication */
if (ca_sp_link_is_up(&my_pipe)) {
    /* Proceed with secure data transmission */
    ca_sp_link_seal(&my_pipe, buffer, length);
```

```
} else {
    /* Initiate handshake */
    ca_sp_link_hello(&my_pipe);
}
```

---

ca_spi_br_t

Tipo: Enumerazione
File: caos/drivers/inc/spi.h:43:52

```
typedef enum {
SPI_BR_DIV2   = 0,   /* FPCLK / 2   */
SPI_BR_DIV4   = 1,   /* FPCLK / 4   */
SPI_BR_DIV8   = 2,   /* FPCLK / 8   */
SPI_BR_DIV16  = 3,   /* FPCLK / 16  */
SPI_BR_DIV32  = 4,   /* FPCLK / 32  */
SPI_BR_DIV64  = 5,   /* FPCLK / 64  */
SPI_BR_DIV128 = 6,   /* FPCLK / 128 */
SPI_BR_DIV256 = 7,   /* FPCLK / 256 */
} ca_spi_br_t;
```

Defines the baud rate prescaler for the SPI peripheral, determining the communication speed relative to the peripheral clock.

Membri:

Nome	Tipo	Descrizione
CA_SPI_BR_DIV2 to CA_SPI_BR_DIV256	enum constant	Sets the divisor for the SPI peripheral clock (FP CLK). Higher divisors result in slower communica tion speeds. Ensure the resulting frequency doe not exceed the maximum rating of the connecte slave device.

Note:
This enum is used to configure the hardware clock divider. The actual SPI frequency is calculated as FPCLK / (2^n), where n is the value of the enum. It must be set during the SPI initialization phase before the peripheral is enabled.

Uso:
Used within the SPI configuration structure passed to the initialization function to set the desired bit rate for synchronous serial communication.

Esempio:

```
ca_spi_cfg_t my_spi_cfg = { .baud_rate = CA_SPI_BR_DIV16, .mode =
    SPI_MODE_MASTER, .data_size = SPI_DATASIZE_8BIT };
```

---

ca_spi_config_t

Tipo: Struttura
File: caos/drivers/inc/spi.h:57:69

```
typedef struct {
u8                spi_num;     /* 1=SPI1, 2=SPI2, 3=SPI3          */
ca_spi_br_t       br;          /* clock prescaler (master only)    */
u8                cpol;        /* 0=idle low,  1=idle high         */
```

```
u8                  cpha;         /* 0=1st edge,  1=2nd edge          */
bool                master;       /* true=master, false=slave         */
u16                 frame_size;   /* byte per frame (slave IRQ mode)  */
ca_spi_slave_cb_t   on_rx;        /* NULL=polling, non-NULL=IRQ slave */
bool                alternate_gpio; /* true=mappatura pin alternativa   */
bool                manage_cs;    /* true=driver gestisce CS via GPIO */
gpio_port_t         cs_port;      /* porta GPIO del CS                */
gpio_pin_t          cs_pin;       /* pin GPIO del CS                  */
} ca_spi_config_t;
```

Configuration structure used to define the hardware parameters and operating mode of an SPI peripheral.

Membri:

Nome	Tipo	Descrizione
spi_num	u8	The hardware SPI instance index. Valid values are 1, 2, or 3. Incorrect values will cause driver initialization to fail.
br	ca_spi_br_t	Baud rate prescaler for master mode. Determines the SPI clock speed. Must be set according to the peripheral's maximum clock frequency.
cpol	u8	Clock polarity: 0 for idle low, 1 for idle high. Must match the slave device's requirements.
cpha	u8	Clock phase: 0 for first edge capture, 1 for second edge capture. Must match the slave device's requirements.
master	bool	Sets the SPI role. True for master (generates clock), false for slave (receives clock).
frame_size	u16	Expected number of bytes per frame when operating in slave IRQ mode.
on_rx	ca_spi_slave_cb_t	Callback function pointer for slave IRQ mode. If set to NULL, the driver operates in polling mode.
alternate_gpio	bool	If true, uses the alternate pin mapping for the SPI peripheral pins.
manage_cs	bool	If true, the driver automatically toggles the Chip Select pin defined by cs_port and cs_pin.
cs_port	ca_gpio_port_t	GPIO port identifier for the Chip Select signal.
cs_pin	ca_gpio_pin_t	GPIO pin number for the Chip Select signal.

Note:
This structure is typically allocated on the stack before calling the SPI initialization function. Ensure all fields are initialized to avoid undefined behavior. If manage_cs is true, ensure the GPIO port and pin are correctly configured as outputs before SPI initialization.

Uso:
Used as an input parameter for the SPI driver initialization function to configure the peripheral hardware settings before any data transfer occurs.

Esempio:

```
ca_spi_config_t spi_cfg = { .spi_num = 1, .br = CA_SPI_BR_DIV8, .cpol = 0, .cpha
    = 0, .master = true, .manage_cs = true, .cs_port = GPIOA, .cs_pin =
   GPIO_PIN_4 }; ca_spi_init(&spi_cfg);
```

ca_spi_slave_cb_t

Tipo: Callback
File: caos/drivers/inc/spi.h:55:55

```
typedef void (*ca_spi_slave_cb_t)(u8 spi_num, const u8 *data, u16 len)
```

A function pointer type used as a callback to handle incoming data when the STM32 is configured as an SPI slave.

Membri:

Nome	Tipo	Descrizione
spi_num	u8	The index of the SPI peripheral that received the data (e.g., 1 for SPI1). Must match the hardware peripheral ID.
data	const u8 *	Pointer to the buffer containing the received bytes. This memory is managed by the driver and should be treated as read-only.
len	u16	The number of bytes received in the current transaction. Must be used to avoid buffer overflows when processing data.

Note:
This callback is executed in the context of the SPI interrupt service routine. Keep the logic inside this function extremely brief to avoid blocking other system tasks. Do not perform blocking operations or complex calculations here. The pointer 'data' is only valid for the duration of the callback execution.

Uso:
Used when initializing an SPI slave interface via the caOS driver. You assign a function matching this signature to the driver configuration structure so the system knows where to route incoming SPI data.

Esempio:

```
void my_spi_handler(u8 spi_num, const u8 *data, u16 len) {
    if (len > 0) {
        process_received_packet(data, len);
    }
}

// During initialization
ca_spi_config_t config = {
    .slave_callback = my_spi_handler,
    // ... other config fields
};
ca_spi_init(1, &config);
```

---

ca_spinlock_irq_t

Tipo: Struttura
File: caos/inc/atomic.h:48:51

```
typedef struct ca_spinlock_irq_s{
volatile u32 key;
u32 irq_state;
}   ca_spinlock_irq_t;
```

A spinlock structure that automatically disables interrupts to ensure thread safety in both task and ISR contexts.

Membri:

Nome	Tipo	Descrizione
`key`	`volatile u32`	Internal flag used to track the lock state. Must be initialized to 0.
`irq_state`	`u32`	Storage for the CPU interrupt mask state saved during lock acquisition to restore it correctly upon release.

Note:
This structure must be initialized using ca_spinlock_irq_init before use. It is typically allocated as a static variable or within a global control block. It is designed to prevent deadlocks when a resource is shared between tasks and interrupt service routines (ISRs).

Uso:
Use this type when you need to protect a shared resource (like a global variable or hardware peripheral) that is accessed by both normal tasks and ISRs. Use ca_spinlock_lock_irq to acquire and ca_spinlock_unlock_irq to release.

Esempio:

```c
static ca_spinlock_irq_t my_lock;

void init_resource(void) {
    ca_spinlock_irq_init(&my_lock);
}

void update_shared_data(void) {
    ca_spinlock_lock_irq(&my_lock);
    /* Perform critical section operations here */
    ca_spinlock_unlock_irq(&my_lock);
}
```

`ca_spinlock_t`

Tipo: Struttura
File: `caos/inc/atomic.h:40:42`

```c
typedef struct  ca_spinlock_s{
volatile u32 key;
}  ca_spinlock_t;
```

A lightweight synchronization primitive used to protect shared resources by busy-waiting.

Membri:

Nome	Tipo	Descrizione
`key`	`volatile u32`	Internal state of the spinlock. Must be treated as opaque; do not modify directly.

Note:
Spinlocks are intended for very short critical sections. They do not put the task to sleep, meaning the CPU will spin in a loop until the lock is acquired. Using them for long operations will starve other tasks and degrade system performance. Always initialize before use.

Uso:

Use when you need to protect data accessed by both ISRs and tasks, or between tasks where the critical section is extremely short. Use ca_spinlock_lock_irq and ca_spinlock_unlock_irq for ISR-safe protection.

Esempio:

```
#include "caos/inc/atomic.h"

static ca_spinlock_t my_lock;
static int shared_resource = 0;

void update_resource(int value) {
    /* Initialize the spinlock before first use */
    ca_spinlock_irq_init(&my_lock);

    /* Acquire lock and disable interrupts */
    ca_spinlock_lock_irq(&my_lock);

    shared_resource = value;

    /* Release lock and restore interrupts */
    ca_spinlock_unlock_irq(&my_lock);
}
```

---

ca_stream_buffer_t

Tipo: Struttura
File: caos/inc/stream.h:30:39

```
typedef struct {
u8 *buffer;          /**< Data buffer */
u32 size;            /**< Buffer size */
volatile u32 head;   /**< Write index */
volatile u32 tail;   /**< Read index */
volatile u32 count; /**< Bytes in buffer */
ca_spinlock_irq_t lock; /**< IRQ-safe lock */
u32 waiting_task;    /**< Waiter HANDLE (ca_task_handle_t),
   CA_TASK_HANDLE_NONE = none */
u32 trigger_level;   /**< Minimum bytes to wake receiver */
} ca_stream_buffer_t;
```

A thread-safe, circular FIFO buffer used for streaming byte-oriented data between tasks or ISRs.

Membri:

Nome	Tipo	Descrizione
buffer	u8 *	Pointer to the allocated memory block used for storage.
size	u32	Total capacity of the buffer in bytes.
head	volatile u32	Index where the next byte will be written.
tail	volatile u32	Index where the next byte will be read.
count	volatile u32	Current number of bytes stored in the buffer.
lock	ca_spinlock_irq_t	Internal spinlock to ensure atomic access during concurrent operations.
waiting_task	u32	Handle of the task waiting for data; set to CA_TASK_HANDLE_NONE if no task is waiting.
trigger_level	u32	Threshold of bytes required to trigger a task wake-up event.

Note:
The structure must be initialized via ca_stream_init before use. The memory for the buffer must be allocated by the user (e.g., via ca_memory_allocate) and must persist for the lifetime of the stream. The 'volatile' fields ensure that the compiler does not optimize away memory accesses, which is critical for ISR-to-task communication.

Uso:
Used for buffering data streams (e.g., UART RX/TX, network data). Use ca_stream_send/ca_stream_receive for data transfer, and ca_stream_set_trigger to define when a blocked task should be unblocked.

Esempio:

```
ca_stream_buffer_t my_stream;
u8 my_raw_buffer[128];

void setup_stream(void) {
    /* Initialize the stream with a 128-byte buffer */
    ca_stream_init(&my_stream, my_raw_buffer, 128);
    /* Wake up task when at least 10 bytes are available */
    ca_stream_set_trigger(&my_stream, 10);
}

void task_producer(void) {
    u8 data = 0xAA;
    ca_stream_send(&my_stream, &data, 1);
}

void task_consumer(void) {
    u8 rx_byte;
    /* Wait for data and receive it */
    if (ca_stream_receive(&my_stream, &rx_byte, 1) > 0) {
        /* Process byte */
    }
}
```

---

ca_sw_timer_handle_t

Tipo: simple
File: caos/inc/sw_timer.h:77:77

```
typedef u8 ca_sw_timer_handle_t
```

A unique identifier (handle) used to reference and manage a software timer within the caOS RTOS.

Membri:

Nome	Tipo	Descrizione
`ca_sw_timer_handle_t`	`u8`	An 8-bit unsigned integer acting as an index or ID for a specific software timer. Valid values range from 0 to the maximum number of timers configured in the system.

Note:
This type is a simple alias for an unsigned 8-bit integer. It does not represent the timer object itself, but rather a reference to it. The handle is typically returned by the timer creation function and must be stored by the user to perform subsequent operations like starting, stopping, or deleting the timer. It is not dynamically allocated; it is a stack or global variable.

Uso:
Use this type whenever you need to interact with a software timer after it has been created. It is passed as the first argument to functions such as ca_sw_timer_start, ca_sw_timer_stop, and ca_sw_timer_delete to specify which timer instance to target.

Esempio:

```
ca_sw_timer_handle_t my_timer_id;

void setup_timer(void) {
    // Create a timer and store the returned handle
    my_timer_id = ca_sw_timer_create(1000, CA_TIMER_PERIODIC, my_callback_func);

    if (my_timer_id != CA_INVALID_TIMER_HANDLE) {
        ca_sw_timer_start(my_timer_id);
    }
}
```

`ca_sw_timer_mode_t`

Tipo: Enumerazione
File: `caos/inc/sw_timer.h:53:56`

```
typedef enum {
CA_TIMER_ONE_SHOT = 0,   /* Single execution */
CA_TIMER_PERIODIC = 1    /* Automatic repetition */
} ca_sw_timer_mode_t;
```

An enumeration defining the execution behavior of a software timer in caOS.

Membri:

Nome	Tipo	Descrizione
`CA_TIMER_ONE_SHOT`	`enum constant`	The timer callback will execute exactly once after the timeout period expires. The timer stops automatically after the first trigger.
`CA_TIMER_PERIODIC`	`enum constant`	The timer callback will execute repeatedly at the specified interval until explicitly stopped by the user.

Note:

This enum is used as a configuration parameter during timer creation. It does not require manual memory management as it is a simple integer-based enumeration. Ensure the mode is selected based on whether you need a single event or a recurring task.

Uso:
Used when initializing a software timer structure to determine if the timer should restart itself automatically after expiration. Pass this value to the timer creation API to set the operational mode.

Esempio:

```
ca_sw_timer_t my_timer;
ca_sw_timer_init(&my_timer, CA_TIMER_PERIODIC, my_callback_function, NULL);
ca_sw_timer_start(&my_timer, 1000); // Starts a periodic timer with 1000ms
    interval
```

---

ca_sw_timer_state_t

Tipo: Enumerazione
File: caos/inc/sw_timer.h:59:63

```
typedef enum {
CA_TIMER_STOPPED = 0,
CA_TIMER_RUNNING = 1,
CA_TIMER_EXPIRED = 2
} ca_sw_timer_state_t;
```

Represents the current operational status of a software timer within the caOS RTOS.

Membri:

Nome	Tipo	Descrizione
CA_TIMER_STOPPED	enum constant	The timer is inactive and not counting. It will not trigger any callbacks.
CA_TIMER_RUNNING	enum constant	The timer is currently active and counting down towards its expiration time.
CA_TIMER_EXPIRED	enum constant	The timer has reached zero and the associated callback has been executed or is pending execution.

Note:
This enum is typically returned by status-check functions to allow the application to verify if a timer is safe to reconfigure or restart. It is a read-only state indicator managed internally by the caOS kernel.

Uso:
Use this type when you need to poll the status of a software timer before performing operations like starting, stopping, or deleting the timer instance to avoid race conditions or invalid state transitions.

Esempio:

```
ca_sw_timer_state_t status = ca_sw_timer_get_state(my_timer_handle);

if (status == CA_TIMER_STOPPED) {
    ca_sw_timer_start(my_timer_handle);
} else if (status == CA_TIMER_RUNNING) {
    // Timer is busy, wait or perform other tasks
}
```

---

ca_sw_timer_t

Tipo: Struttura
File: caos/inc/sw_timer.h:66:74

```
typedef struct {
u32 period_ms;              /* Period in ms */
u32 remaining_ms;           /* Remaining time */
ca_timer_callback_t cb;     /* Callback */
void *cb_arg;               /* Callback argument */
ca_sw_timer_mode_t mode;          /* One-shot or periodic */
ca_sw_timer_state_t state;        /* Current state */
u8 active;                  /* 1 = slot used, 0 = free */
} ca_sw_timer_t;
```

A software timer structure used to schedule periodic or one-shot tasks without blocking the CPU.

Membri:

Nome	Tipo	Descrizione
period_ms	u32	The timer interval in milliseconds. Must be greater than 0.
remaining_ms	u32	Internal counter tracking time left until expiration. Do not modify manually.
cb	ca_timer_callback_t	Function pointer to the code executed when the timer expires.
cb_arg	void *	Pointer to data passed to the callback function. Can be NULL.
mode	ca_sw_timer_mode_t	Determines if the timer repeats (CA_TIMER_PERIODIC) or runs once (CA_TIMER_ONESHOT).
state	ca_sw_timer_state_t	Current operational state (e.g., running, stopped). Managed by the RTOS kernel.
active	u8	Flag indicating if the timer slot is allocated (1) or available (0).

Note:
The structure should be treated as opaque by the application. Always initialize the struct to zero before use. The RTOS kernel manages the timing logic; manual modification of 'remaining_ms' or 'state' will lead to undefined behavior.

Uso:
Use this type to create non-blocking delays or periodic background tasks. It is typically passed to timer management functions to register, start, or stop the timer instance.

Esempio:

```
void my_callback(void *arg) { /* Task logic here */ }

ca_sw_timer_t my_timer = {0};
my_timer.period_ms = 1000;
my_timer.cb = my_callback;
my_timer.mode = CA_TIMER_PERIODIC;
ca_timer_start(&my_timer);
```

ca_syslog_cfg_t

Tipo: Struttura
File: caos/net/inc/net_cfg.h:126:130

```
typedef struct {
ca_ipv4_t server_ip;   /* syslog server IPv4 */
u16     server_port; /* syslog UDP port (0 = 514 default) */
u8      facility;    /* syslog facility (0-23, 1=user) */
} ca_syslog_cfg_t;
```

Configuration structure used to define the destination and protocol parameters for the caOS system logging service.

Membri:

Nome	Tipo	Descrizione
server_ip	ca_ipv4_t	The IPv4 address of the remote syslog server. Must be a valid network-byte-order address.
server_port	u16	The UDP port number for the syslog server. Set to 0 to automatically use the standard syslog port 514.
facility	u8	The syslog facility code (0-23). Defines the type of program logging the message. Use 1 for general user-level messages.

Note:
This structure is typically passed by value or pointer to the system initialization routines. It does not require dynamic allocation; it is usually declared as a static configuration object in the application layer. Ensure the IP address is correctly formatted before passing to the network stack.

Uso:
Use this structure when configuring the system logging subsystem during the board initialization phase to enable remote log forwarding over UDP.

Esempio:

```
ca_syslog_cfg_t syslog_cfg = { .server_ip = {192, 168, 1, 50}, .server_port =
    514, .facility = 1 };
```

---

ca_task_fn_t

Tipo: Callback
File: caos/inc/scheduler.h:139:139

```
typedef u32 (*ca_task_fn_t)(u32 idx, u32 p1)
```

A function pointer type defining the entry point for a task in the caOS scheduler.

Membri:

Nome	Tipo	Descrizione
idx	u32	The unique index or ID assigned to the task instance. Used to identify which task is currently executing if multiple tasks share the same function.
p1	u32	A user-defined parameter passed to the task during creation. Typically used for configuration data or pointers to task-specific structures.

Note:
The function must be non-blocking and should ideally return 0 upon successful execution. It is invoked by the caOS scheduler context switch mechanism. Ensure the function is re-entrant if the task logic accesses shared resources.

Uso:
Used when defining the main logic of a task. You pass a pointer to a function matching this signature to the task creation API to register the task with the scheduler.

Esempio:

```
u32 my_task_logic(u32 idx, u32 p1) {
    // Task logic goes here
    return 0;
}

// Usage in task creation
ca_task_create(my_task_logic, 0, 1024, 1, NULL);
```

`ca_task_handle_t`

Tipo: simple
File: `caos/inc/scheduler.h:254:254`

```
typedef u32 ca_task_handle_t
```

An opaque identifier used by the kernel to uniquely reference and manage a specific task within the system.

Membri:

Nome	Tipo	Descrizione
`ca_task_handle_t`	`u32`	A numeric handle representing a task. It is used as an input parameter for task control functions like suspension, resumption, or priority modification.

Note:
The handle is typically obtained via ca_os_self_handle() for the current task or returned by task creation routines. It is managed by the scheduler; users should treat it as an opaque value and never attempt to modify its contents or perform arithmetic on it.

Uso:
Use this type whenever you need to perform operations on a specific task, such as changing its priority, suspending it, or waking it up from a blocked state.

Esempio:

```
#include "caos/inc/scheduler.h"

void task_control_example(void) {
    /* Get the handle of the currently running task */
    ca_task_handle_t my_handle = ca_os_self_handle();

    /* Suspend the current task */
    ca_os_suspend_handle(my_handle);

    /* Resume the task later */
    ca_os_resume_handle(my_handle);

    /* Change the priority of the task */
    ca_os_set_priority_handle(my_handle, 10);
}
```

ca_task_list_t

Tipo: Struttura
File: caos/inc/scheduler.h:235:240

```
typedef struct {
ca_thread_context_t **items;  /**< Pointer buffer */
u32 size;                     /**< Number of active elements */
u32 capacity;                 /**< Maximum capacity */
bool good;                    /**< Last operation validity flag */
} ca_task_list_t;
```

A dynamic container structure used by the scheduler to manage a list of task control blocks (ca_thread_context_t).

Membri:

Nome	Tipo	Descrizione
items	ca_thread_context_t **	A pointer to an array of pointers, where each pointer references a task's context. Must be allocated with sufficient memory to hold 'capacity' elements.
size	u32	The current number of tasks stored in the list. Must always be less than or equal to 'capacity'.
capacity	u32	The maximum number of tasks the list can hold. Defines the size of the memory buffer pointed to by 'items'.
good	bool	A status flag indicating if the last operation on the list was successful. True if successful, false if an error (e.g., overflow) occurred.

Note:
The structure itself does not manage the memory for the 'items' buffer; the user or the system allocator must provide a buffer of size (capacity * sizeof(ca_thread_context_t*)). The 'good' flag is updated by caOS internal functions to signal if an insertion or removal was valid.

Uso:
Used primarily by the scheduler to maintain queues of ready or blocked tasks. You will encounter this when inspecting the internal state of the scheduler or when implementing custom task management logic that requires grouping tasks.

Esempio:

```
ca_thread_context_t *task_buffer[10];
ca_task_list_t my_task_list = {
    .items = task_buffer,
    .size = 0,
    .capacity = 10,
    .good = true
};

// The list is now ready to be passed to internal caOS scheduler functions
// that accept a pointer to a ca_task_list_t structure.
```

ca_tcp_close_t

Tipo: Callback
File: caos/net/inc/tcp_service.h:38:38

```
typedef void (*ca_tcp_close_t)(u8 handle);     /* called on close/error */
```

A callback function pointer triggered by the TCP stack when a connection is closed or encounters a fatal error.

Membri:

Nome	Tipo	Descrizione
handle	u8	The unique identifier of the TCP connection that has been closed. Must be a valid handle previously opened by the stack.

Note:
This callback is executed within the context of the TCP service task. It is intended for cleanup operations, such as freeing application-level buffers or notifying other tasks. Do not perform blocking operations or long-running tasks inside this callback to avoid stalling the network stack.

Uso:
Used when registering a TCP service or connection configuration. Assign this function pointer to the 'on_close' field of a connection structure to handle asynchronous connection termination events.

Esempio:

```
void my_tcp_cleanup(u8 handle) {
    // Perform cleanup for the specific connection handle
    printf("Connection %d closed or failed.\n", handle);
}

// Usage in a configuration struct
ca_tcp_config_t config = {
    .on_close = my_tcp_cleanup,
    // ... other fields
};
```

---

ca_tcp_conn_t

Tipo: Callback
File: caos/net/inc/tcp_service.h:37:37

```
typedef void (*ca_tcp_conn_t)(u8 handle);     /* called when ESTABLISHED */
```

A callback function pointer triggered by the TCP stack when a new connection is successfully established.

Membri:

Nome	Tipo	Descrizione
handle	u8	The unique identifier (index) assigned to the new TCP connection. Must be used in subsequent API calls to identify this specific stream.

Note:

This callback is executed within the context of the TCP service task. It must be non-blocking and should not perform heavy processing. The user is responsible for storing the handle if they intend to send or receive data on this connection later.

Uso:
Used during the initialization of a TCP server or client to notify the application layer that the three-way handshake is complete and the socket is ready for data transmission.

Esempio:

```
void my_connection_handler(u8 handle) {
    printf("Connection established with handle: %d\n", handle);
}

// Registering the callback during service setup
ca_tcp_init(my_connection_handler);
```

---

ca_tcp_connect_cfg_t

Tipo: Struttura
File: caos/net/inc/tcp_service.h:43:53

```
typedef struct {
ipv4_t          remote_ip;
u16             remote_port;
ipv4_t          local_ip;
mac_t           local_mac;
u32             arp_timeout_ms;     /* 0     TCP_ARP_TIMEOUT_MS default */
u32             connect_timeout_ms; /* 0     TCP_CONNECT_TIMEOUT_MS default */
ca_tcp_rx_t     on_rx;              /* NULL = blocking ca_tcp_recv() mode */
ca_tcp_conn_t   on_connected;       /* NULL = blocking ca_tcp_connect() */
ca_tcp_close_t  on_closed;
} ca_tcp_connect_cfg_t;
```

Configuration structure used to define parameters and callback handlers for establishing a TCP connection.

Membri:

Nome	Tipo	Descrizione
remote_ip	ca_ipv4_t	The destination IPv4 address. Must be a valid network-order address.
remote_port	u16	The destination TCP port number.
local_ip	ca_ipv4_t	The local IPv4 address to bind to. Use 0.0.0.0 for default interface.
local_mac	ca_mac_t	The local MAC address associated with the interface.
arp_timeout_ms	u32	Time to wait for ARP resolution. Set to 0 to use the system default.
connect_timeout_ms	u32	Time to wait for the TCP handshake to complete. Set to 0 to use the system default.
on_rx	ca_tcp_rx_t	Callback function for incoming data. If NULL, the socket operates in blocking mode.
on_connected	ca_tcp_conn_t	Callback triggered upon successful connection. If NULL, the connect function blocks until completion.
on_closed	ca_tcp_close_t	Callback triggered when the connection is closed by either party.

Note:
The structure is typically allocated on the stack before calling the connection function. If callbacks are provided, the socket operates asynchronously; otherwise, it operates synchronously (blocking). Ensure all fields are zero-initialized if you intend to use default values for timeouts.

Uso:
Used as an argument in the TCP connection API to specify target parameters and event handling behavior. It is essential for non-blocking network applications where the CPU must perform other tasks while waiting for network events.

Esempio:

```
ca_tcp_connect_cfg_t cfg = {0};
cfg.remote_ip = ip_addr_make(192, 168, 1, 100);
cfg.remote_port = 80;
cfg.on_rx = my_rx_callback;
cfg.on_closed = my_close_callback;
ca_tcp_connect(socket_handle, &cfg);
```

---

ca_tcp_frame_t

Tipo: Struttura
File: caos/net/inc/net_proto.h:135:145

```
typedef struct {
u16     src_port;  /* source port big-endian */
u16     dst_port;  /* destination port big-endian */
u32     seq;       /* sequence number big-endian */
u32     ack_seq;   /* acknowledgement number big-endian */
u8      data_off;  /* [7:4]=header length in 32-bit words, [3:0]=reserved
    */
u8      flags;     /* TCP_FLAG_* */
u16     window;    /* receive window big-endian */
u16     checksum;  /* pseudo-header checksum big-endian */
u16     urg_ptr;   /* urgent pointer big-endian (unused) */
} __attribute__((packed)) ca_tcp_frame_t;
```

Represents the raw TCP header structure used for packet parsing and construction in the caOS network stack.

Membri:

Nome	Tipo	Descrizione
`src_port`	`u16`	Source port number in network byte order (big-endian). Must be converted from host byte order using htons() if necessary.
`dst_port`	`u16`	Destination port number in network byte order (big-endian).
`seq`	`u32`	TCP sequence number in network byte order. Used to track the order of data segments.
`ack_seq`	`u32`	Acknowledgment number in network byte order. Indicates the next sequence number expected.
`data_off`	`u8`	High 4 bits represent the header length in 32-bit words. Low 4 bits are reserved and should be 0.
`flags`	`u8`	TCP control flags (e.g., SYN, ACK, FIN). Use defined TCP_FLAG_* constants.
`window`	`u16`	Receive window size in network byte order, indicating available buffer space.
`checksum`	`u16`	16-bit one's complement of the one's complement sum of the TCP header and data, plus a pseudo-header.
`urg_ptr`	`u16`	Urgent pointer field, typically unused in standard caOS implementations.

Note:
The struct is marked with ___attribute___((packed)) to ensure no padding bytes are inserted by the compiler, matching the exact wire format of a TCP header. Always ensure multi-byte fields are in big-endian format before transmission.

Uso:
Used when manually crafting TCP packets or parsing incoming raw network buffers within the caOS network layer. It is typically cast onto a pointer of a received network buffer.

Esempio:

```
ca_tcp_frame_t *tcp_hdr = (ca_tcp_frame_t *)(buffer + ip_header_len);
tcp_hdr->src_port = htons(80);
tcp_hdr->dst_port = htons(1234);
tcp_hdr->flags = TCP_FLAG_ACK | TCP_FLAG_PSH;
tcp_hdr->seq = htonl(current_seq_num);
```

`ca_tcp_listen_cfg_t`

Tipo: Struttura
File: `caos/net/inc/tcp_service.h:113:120`

```
typedef struct {
u16            local_port;
ipv4_t         local_ip;
mac_t          local_mac;
ca_tcp_conn_t  on_accept;   /* NULL = blocking ca_tcp_accept() mode */
ca_tcp_rx_t    on_rx;       /* NULL = blocking ca_tcp_recv() on data
   conn */
```

```
ca_tcp_close_t on_closed;
} ca_tcp_listen_cfg_t;
```

Configuration structure used to define the parameters and event callbacks for a TCP listening socket.

Membri:

Nome	Tipo	Descrizione
`local_port`	`u16`	The TCP port number to bind the listener to. Must be a valid non-zero port.
`local_ip`	`ca_ipv4_t`	The local IPv4 address to bind. Use 0.0.0.0 to listen on all available network interfaces.
`local_mac`	`ca_mac_t`	The MAC address associated with the interface. Usually retrieved from the network stack configuration.
`on_accept`	`ca_tcp_conn_t`	Callback function triggered when a new connection is established. If NULL, the system defaults to blocking mode.
`on_rx`	`ca_tcp_rx_t`	Callback function triggered when data is received on the connection. If NULL, the system defaults to blocking mode.
`on_closed`	`ca_tcp_close_t`	Callback function triggered when the connection is closed by either party.

Note:
This structure is typically passed by value or pointer to the TCP initialization API. The user is responsible for allocating this struct, usually on the stack or as a static global. Ensure all callback pointers are either valid functions or explicitly set to NULL to avoid undefined behavior.

Uso:
Use this struct when setting up a TCP server listener. It defines the network identity of the server and the event-driven behavior for handling incoming connections and data streams.

Esempio:

```
ca_tcp_listen_cfg_t cfg = {0};
cfg.local_port = 8080;
cfg.local_ip = IPV4_ADDR(192, 168, 1, 10);
cfg.on_accept = my_accept_callback;
cfg.on_rx = my_rx_callback;
cfg.on_closed = my_close_callback;
ca_tcp_listen(&cfg);
```

`ca_tcp_rx_raw_t`

Tipo: Callback
File: `caos/net/inc/net_proto.h:222:222`

```
typedef void (*ca_tcp_rx_raw_t)(u16 buf_idx)
```

A callback function type used by the TCP stack to notify the application when raw data has been received.

Membri:

Nome	Tipo	Descrizione
`buf_idx`	`u16`	An index representing the internal network buffer containing the received TCP payload. This index is used to retrieve or process the data.

Note:
This callback is executed within the context of the network stack. It must be kept short and efficient to avoid blocking network processing. The buffer associated with buf_idx is typically managed by the stack; ensure you call the appropriate buffer release function if the stack expects the user to free it.

Uso:
Used when registering a TCP listener or connection handler via the network stack. It allows the application to react immediately to incoming TCP segments.

Esempio:

```c
void my_tcp_rx_handler(u16 buf_idx) {
    /* Retrieve the pointer to the received data buffer */
    void *data = ca_net_buf_ptr(buf_idx);

    /* Process the data here */
    ca_printf("Received TCP data at index %d\n", buf_idx);

    /* Release the buffer back to the network pool after processing */
    ca_net_buf_free(buf_idx);
}

/* Usage in initialization */
void init_network(void) {
    /* Register the callback to handle incoming raw TCP data */
    ca_net_register(my_tcp_rx_handler);
}
```

---

ca_tcp_rx_t

Tipo: Callback
File: caos/net/inc/tcp_service.h:36:36

```c
typedef void (*ca_tcp_rx_t)(u8 handle, const u8 *data, u16 len)
```

A callback function type used to handle incoming TCP data packets received by the caOS network stack.

Membri:

Nome	Tipo	Descrizione
handle	u8	The unique identifier of the TCP connection that received the data. Must be a valid handle returned by a previous connection request.
data	const u8 *	Pointer to the buffer containing the received payload. This memory is managed by the stack and is only valid for the duration of the callback.
len	u16	The number of bytes available in the data buffer. Must be checked before accessing the buffer to prevent out-of-bounds reads.

Note:
This callback is executed within the context of the network task. It must be non-blocking and should not perform heavy processing. The data pointer is read-only; modifying it will corrupt the internal network buffer. The user does not allocate or free the data buffer.

Uso:

Used when registering a TCP service or connection. You assign a function matching this signature to the rx_callback field of a TCP configuration structure to process incoming network traffic.

Esempio:

```
void my_tcp_handler(u8 handle, const u8 *data, u16 len) {
    if (len > 0) {
        // Process incoming data here
        process_packet(data, len);
    }
}

// Usage in configuration
ca_tcp_config_t config = {
    .rx_callback = my_tcp_handler,
    // ... other fields
};
```

---

ca_temp_alarm_cb_t

Tipo: Callback
File: caos/drivers/inc/temp_sensor.h:22:22

```
typedef void (*ca_temp_alarm_cb_t)(bool hot);    /* called from the DTS
    interrupt */
```

A callback function type triggered by the internal temperature sensor (DTS) interrupt when a temperature threshold is crossed.

Membri:

Nome	Tipo	Descrizione
hot	bool	Indicates the alarm state. 'true' means the temperature has exceeded the high threshold; 'false' means it has returned to a safe level.

Note:
This callback is executed directly from the DTS interrupt service routine (ISR). It must be extremely fast, non-blocking, and must not perform any operations that require waiting (like mutex locks or heavy processing). It is registered via the ca_temp_set_alarm function.

Uso:
Use this to implement safety shutdowns or cooling fan control when the MCU temperature exceeds safe operating limits.

Esempio:

```
/* Callback implementation */
void my_temp_handler(bool hot) {
    if (hot) {
        /* Trigger emergency cooling or system shutdown */
        ca_led_on(0);
    } else {
        ca_led_off(0);
    }
}

/* Registration */
void setup_temp_monitoring(void) {
    /* Register the callback to the temperature sensor driver */
```

```
    ca_temp_set_alarm(my_temp_handler);
}
```

---

ca_tftp_cfg_t

Tipo: Struttura
File: caos/net/inc/tftp_service.h:88:97

```
typedef struct {
ipv4_t      server_ip;
const char *filename;
mac_t       local_mac;
ipv4_t      local_ip;
u16         local_port;         /* 0 = auto ephemeral */
u32         arp_timeout_ms;     /* 0 = 2000 ms */
u32         block_timeout_ms;   /* 0 = TFTP_BLOCK_TIMEOUT_MS */
u8          retry_n;            /* 0 = TFTP_RETRY_N */
} ca_tftp_cfg_t;
```

Configuration structure used to define the parameters for a TFTP client transfer session.

Membri:

Nome	Tipo	Descrizione
server_ip	ca_ipv4_t	The IPv4 address of the remote TFTP server. Must be a valid network-byte-order address.
filename	const char *	Pointer to a null-terminated string representing the path of the file to request or send.
local_mac	ca_mac_t	The MAC address of the local network interface to be used for the transfer.
local_ip	ca_ipv4_t	The local IPv4 address assigned to the interface.
local_port	u16	The local UDP port. Set to 0 to allow the stack to assign an automatic ephemeral port.
arp_timeout_ms	u32	Timeout in milliseconds for ARP resolution. Set to 0 to use the default 2000ms.
block_timeout_ms	u32	Timeout in milliseconds for waiting for a data block. Set to 0 to use the system default.
retry_n	u8	Maximum number of retries before aborting the transfer. Set to 0 to use the system default.

Note:
The structure is passed by value or reference to TFTP client functions. The caller is responsible for ensuring the 'filename' string remains valid for the duration of the transfer. Fields set to 0 trigger the use of internal caOS default values.

Uso:
Use this structure to initialize a TFTP transfer session. It is typically passed as an argument to the client-side TFTP initiation functions to configure network routing and timing parameters.

Esempio:

```
ca_tftp_cfg_t tftp_cfg = { .server_ip = {192, 168, 1, 10}, .filename = "firmware
    .bin", .local_mac = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55}, .local_ip = {192,
    168, 1, 50}, .local_port = 0, .arp_timeout_ms = 0, .block_timeout_ms = 5000,
     .retry_n = 3 };
```

---

ca_tftp_err_t

Tipo: Struttura
File: caos/net/inc/tftp_service.h:74:77

```
typedef struct {
u16  code;      /* TFTP_ERR_*, or CA_TFTP_NO_ERROR */
char msg[48];   /* its text, printable ASCII */
} ca_tftp_err_t;
```

A structure used to report TFTP protocol errors, containing a numeric error code and a descriptive message string.

Membri:

Nome	Tipo	Descrizione
code	u16	The specific TFTP error code (e.g., TFTP_ERR_FILE_NOT_FOUND) or CA_TFTP_NO_ERROR if the operation succeeded.
msg	char[48]	A human-readable string describing the error. Must be null-terminated and fit within 48 bytes.

Note:
This struct is typically populated by the caOS TFTP service layer when a transfer fails. It is stack-allocated by the caller or the service handler. Ensure the 'msg' buffer is not overflowed if manually constructing this struct.

Uso:
Used as a return type or output parameter in TFTP client functions like ca_tftp_get or ca_tftp_put to provide diagnostic information when a network transfer fails.

Esempio:

```
ca_tftp_err_t err;
/* Attempt to retrieve a file via TFTP */
/* Assuming a hypothetical buffer and address setup */
err = ca_tftp_get(remote_ip, "config.bin", local_buffer);

if (err.code != CA_TFTP_NO_ERROR) {
    /* Log the error message to the system console */
    ca_printf("TFTP Error %d: %s\n", err.code, err.msg);
}
```

---

ca_tftp_rx_block_t

Tipo: Callback
File: caos/net/inc/tftp_service.h:80:80

```
typedef bool (*ca_tftp_rx_block_t)(u16 block_num,
```

A callback function type triggered by the TFTP service whenever a new data block is successfully received.

Membri:

Nome	Tipo	Descrizione
block_num	u16	The sequence number of the TFTP block currently being received. Starts at 1 for the first data block.
data	const u8 *	Pointer to the buffer containing the raw payload of the received block.
len	u16	The number of bytes contained in the data buffer. Valid values are 0 to 512.

Note:
The callback must return true to acknowledge the block and continue the transfer, or false to abort the transfer (e.g., if a write error occurs). The memory pointed to by 'data' is only valid for the duration of the callback execution; do not store the pointer for later use.

Uso:
Used when configuring a TFTP server or client instance to process incoming file data. You assign a function matching this signature to the 'rx_callback' field in the TFTP configuration structure before starting the service.

Esempio:

```
static bool my_tftp_handler(u16 block_num, const u8 *data, u16 len) { if (len > 0) { flash_write_page(block_num, data, len); return true; } return false; }
// Usage in config: .rx_callback = my_tftp_handler;
```

---

ca_tftp_srv_cfg_t

Tipo: Struttura
File: caos/net/inc/tftp_service.h:140:147

```
typedef struct {
ca_mac_t    local_mac;
ca_ipv4_t   local_ip;           /* 0 = the current address of the stack */
u16         port;               /* listening port, 0 = TFTP_PORT */
u32         block_timeout_ms;   /* 0 = TFTP_BLOCK_TIMEOUT_MS */
u8          retry_n;            /* 0 = TFTP_RETRY_N */
ca_tftp_err_t *err;             /* NULL, or filled with the peer's ERROR packet */
} ca_tftp_srv_cfg_t;
```

Configuration structure for initializing and managing a TFTP server instance in caOS.

Membri:

Nome	Tipo	Descrizione
local_mac	ca_mac_t	The MAC address to bind the TFTP server to. If zero, the stack's default interface is used.
local_ip	ca_ipv4_t	The IPv4 address to bind the TFTP server to. Set to 0 to use the current stack IP address.
port	u16	UDP port for listening. Set to 0 to use the default TFTP port (69).
block_timeout_ms	u32	Timeout in milliseconds for waiting on data blocks. Set to 0 to use the system default.
retry_n	u8	Number of retries before aborting a transfer. Set to 0 to use the system default.
err	ca_tftp_err_t *	Pointer to a structure that will be populated if the peer sends a TFTP ERROR packet. Can be NULL.

Note:
This structure is passed to the TFTP service initialization functions. The user is responsible for allocating this struct (usually on the stack or as a global). The fields set to 0 act as 'use default' flags, allowing for easy partial configuration.

Uso:
Use this structure when setting up a TFTP server instance to define network binding and timeout behavior before calling the service initialization functions.

Esempio:

```
ca_tftp_srv_cfg_t tftp_cfg;
ca_tftp_err_t last_err;

/* Initialize configuration with defaults */
ca_memzero(&tftp_cfg, sizeof(tftp_cfg));
tftp_cfg.port = 69;
tftp_cfg.err = &last_err;

/* Pass the configuration to the TFTP service initialization */
ca_tftp_service_init(&tftp_cfg);
```

`ca_tftp_wrq_t`

Tipo: Callback
File: `caos/net/inc/tftp_service.h:154:154`

```
typedef u16 (*ca_tftp_wrq_t)(const char *filename, ca_ipv4_t client,
    void *arg)
```

Callback function type used to handle incoming TFTP Write Request (WRQ) packets.

Membri:

Nome	Tipo	Descrizione
`filename`	`const char *`	The name of the file requested by the client to be written to the server.
`client`	`ca_ipv4_t`	The IPv4 address of the remote client initiating the write request.
`arg`	`void *`	User-defined context pointer passed during the TFTP service registration.

Note:
The function must return a 16-bit status code. A return value of 0 typically indicates success, while non-zero values represent specific TFTP error codes. This callback is executed within the context of the TFTP service thread; therefore, it should be non-blocking and efficient to avoid stalling network processing.

Uso:
Used when configuring the TFTP service to define custom logic for handling file uploads. You implement this function to validate the filename or check permissions before the server accepts the incoming data stream.

Esempio:

```
u16 my_tftp_wrq_handler(const char *filename, ca_ipv4_t client, void *arg) {
    /* Check if the filename is allowed */
    if (ca_strcmp(filename, "config.bin") != 0) {
        return 1; /* Return TFTP error code 1: File not found */
    }
```

```
    /* Log the request using system printf */
    ca_printf("TFTP WRQ received for file: %s\n", filename);
    return 0; /* Success */
}
```

---

ca_thread_context_t

Tipo: Struttura
File: caos/inc/scheduler.h:152:205

```
typedef struct __attribute__((packed, aligned(4))) ca_thread_context {
/* Hardware context (Cortex-M4) */
u32 digest;             /* 0 0xCACAFEFE*/
/* Registers saved automatically by hardware (read from stack) */
u32 r[13];              /**< 4+52 R0-R12 */
u32 lr;                 /**< 56+4 Link Register */
u32 pc;                 /**< 60+4 Program Counter */
u32 xpsr;               /**< 64+4 xPSR */
u32 psp;                /**< 68+4 Process Stack Pointer */
/* Task metadata */
u32 stack_start;        /**< 72+4 Start of stack area */
u32 stack_end;          /**< 76+4 End of stack area (with guard) */
/* Scheduling */
u16 index;              /**< 80+2 Index in taskList array */
u16 priority;           /**< 82+2 EFFECTIVE priority (the scheduler's
   weight):
prio_base raised by priority inheritance */
u16 num_alloc;          /**< 84+2 Number of allocated heap blocks */
volatile u16 status;    /**< 86+2 Task status (ISR-shared: volatile
prevents compiler reordering/caching) */
/* Statistics + Notification */
volatile u32 notify_value; /**< 88+4 Task notification value (ISR-shared
   ) */
u32 nswitch;            /**< 92+4 Number of context switches */
u32 running;            /**< 96+4 Total task lifetime */
u32 sleep;              /**< 100+4 remaining clocks if in sleep */
void *wait_list;        /**< 104+4 ca_wait_queue_t the task is linked in
   , NULL if none
(was 'blocked', a refresh counter no one used) */
u32 result;             /**< 108+4 Task return value */

/* Debug / Stack monitoring */
struct ca_thread_context *next_task_wait;  /**< 112+4 the next task in
   wait state */
/* Emergency handler (fault-safe) */
ca_task_fn_t thread_exec;     /**< 116+4 the thread entry function */
u32 p1;                 /**< 120+4 entry parameter */
u32 emerg_param;        /**< 124+4 parameter for emergency handler */
ca_emergency_fn_t emergency_exec;  /**< 128+4  the function for
   emergency management*/
u32 runtime_enter;      /**< 132+4 last CYCCNT task entry */
u32 runtime_peak_max;   /**< 136+4 maximum duration (WCET) in clock
   cycles */
u32 runtime_total;      /**< 140+4 total cumulative runtime in clock
   cycles */
u32 runtime_wdt;        /**< 144+4 WDT threshold in clock cycles (0=
   disabled) */
```

```
40 u16 n_assert;            /**< 148+2 number of asserts for this thread  */
41 u16 n_restart;           /**< 150+2 number of restarts for this thread */
42 u16 n_memfault;          /**< 152+2 number of memfaults for this thread
   */
43 u16 prio_base;           /**< 154+2 priority given at creation /
   set_priority */
44 u16 stack_usage_pct;     /**< 156+2 0-1000 (0.0%-100.0%) */
45 u16 fault_type;          /**< 158+2 recovery policy: CA_ONLY_ASSERT/
   RESTART/EMERGENCY */
46
47 /* Priority inheritance (sync.c): effective priority = max(prio_base,
   the
48 * waiters of every mutex in pi_held), propagated along pi_wait owners */
49 struct ca_mutex_s *pi_held; /**< 160+4 mutexes this task owns (list) */
50 struct ca_mutex_s *pi_wait; /**< 164+4 mutex this task is blocked on */
51
52 char name[TASK_NAME_LEN];          /**< 168+12 thread name*/
53 /* 180 size END*/
54 } ca_thread_context_t;
```

Represents the complete execution state, metadata, and scheduling information for a single task in the caOS RTOS.

Membri:

Nome	Tipo	Descrizione
`digest`	`u32`	Magic number 0xCACAFEFE used for stack integrity validation.
`r[13], lr, pc, xpsr`	`u32[]`	Hardware registers saved during context switches to allow task resumption.
`psp`	`u32`	Process Stack Pointer; tracks the current stack location for this specific task.
`stack_start, stack_end`	`u32`	Memory boundaries of the task stack; used for overflow detection.
`priority`	`u16`	The current effective priority used by the scheduler, potentially boosted by priority inheritance.
`status`	`u16`	Current task state (e.g., READY, BLOCKED, RUNNING). Marked volatile as it is modified by ISRs.
`notify_value`	`u32`	A 32-bit value used for inter-task signaling and synchronization.
`thread_exec`	`ca_task_fn_t`	Pointer to the function that serves as the entry point for the task.
`name`	`char[]`	Human-readable identifier for the task, useful for debugging.

Note:
This structure is managed internally by the caOS kernel. Users should never manually modify these fields. The structure is packed and aligned to 4 bytes to match the Cortex-M4 stack frame requirements. Memory for this struct is typically allocated by the kernel during task creation.

Uso:
Used by the kernel scheduler to manage multitasking. While you rarely interact with this struct directly, it is referenced when inspecting task lists, debugging stack usage, or monitoring task statistics via kernel diagnostic tools.

Esempio:

```c
/* Example of accessing task metadata for debugging purposes */
void print_task_info(ca_thread_context_t *task) {
    if (task != NULL) {
        printf("Task Name: %s\n", task->name);
        printf("Current Priority: %u\n", task->priority);
        printf("Stack Usage: %u.%u%%\n", task->stack_usage_pct / 10, task->
    stack_usage_pct % 10);
    }
}
```

---

ca_tick_speed_t

Tipo: Enumerazione
File: caos/inc/systick.h:33:44

```c
typedef enum __attribute__((packed)) tag_speed {
CA_TICK_SPEED_100US = 100,
CA_TICK_SPEED_200US = 200,
CA_TICK_SPEED_250US = 250,
CA_TICK_SPEED_400US = 400,
CA_TICK_SPEED_500US = 500,
CA_TICK_SPEED_1MS = 1000,
CA_TICK_SPEED_2MS = 2000,
CA_TICK_SPEED_4MS = 4000,
CA_TICK_SPEED_5MS = 5000,
CA_TICK_SPEED_10MS = 10000,
} ca_tick_speed_t;
```

An enumeration defining the duration of a single system tick in microseconds, used to configure the RTOS heartbeat frequency.

Membri:

Nome	Tipo	Descrizione
CA_TICK_SPEED_100US to CA_TICK_SPEED_10MS	enum constant	Represents the tick period in microsecond values result in lower CPU overhead bu timing resolution for tasks and delays.

Note:
This enum is used to configure the hardware timer responsible for the RTOS system tick. It must be set during the system initialization phase before the scheduler starts. The values are packed to save memory, and the integer value corresponds directly to the period in microseconds.

Uso:
Used as an argument when initializing the system clock or RTOS kernel to define the base time unit for all kernel-level timing operations like delays and timeouts.

Esempio:

```c
ca_tick_speed_t my_tick_rate = CA_TICK_SPEED_1MS;

void system_init(void) {
    /* Configure the system tick to trigger every 1ms */
    caos_systick_init(my_tick_rate);
}
```

---

ca_timer_callback_t

Tipo: Callback
File: caos/inc/sw_timer.h:50:50

```
typedef void (*ca_timer_callback_t)(void *arg)
```

A function pointer type used to define the callback function executed when a software timer expires.

Membri:

Nome	Tipo	Descrizione
arg	void *	A user-defined pointer passed to the callback function. It allows the callback to access context-specific data (e.g., a struct or a peripheral handle). Must be a valid memory address or NULL.

Note:
The callback function is executed within the context of the caOS timer service task. Therefore, the callback must be kept short and efficient. Blocking operations, such as infinite loops or long delays, must be avoided to prevent system-wide timing jitter or starvation of other tasks.

Uso:
Used when creating a software timer via the caOS timer API. You define a function matching this signature and pass its address during timer initialization or configuration to handle periodic or one-shot events.

Esempio:

```
void my_timer_handler(void *arg) { uint32_t *counter = (uint32_t *)arg; (*
    counter)++; } ... ca_timer_t my_timer; uint32_t event_count = 0;
    ca_timer_init(&my_timer, my_timer_handler, &event_count, 100,
    CA_TIMER_PERIODIC);
```

---

ca_timer_cb_t

Tipo: Callback
File: caos/drivers/inc/timer.h:39:39

```
typedef void (*ca_timer_cb_t)(void)
```

A function pointer type used to define the callback routine executed when a caOS software timer expires.

Membri:

Nome	Tipo	Descrizione
ca_timer_cb_t	void (*)(void)	A pointer to a function that takes no arguments and returns nothing. This function is invoked by the system timer task upon timeout.

Note:
The callback function runs in the context of the timer task or an ISR depending on the RTOS configuration. It must be kept short and non-blocking to avoid starving other system tasks. Do not perform heavy processing or infinite loops inside this function.

Uso:

Used when registering a software timer via the caOS timer API. Assign a function name to a variable of this type and pass it to the timer initialization function to define the action taken upon expiration.

Esempio:

```
void my_timer_handler(void) { /* Toggle an LED or set a flag */ }

/* Usage in timer setup */
ca_timer_cb_t my_callback = my_timer_handler;
ca_timer_create(my_callback, 1000);
```

---

ca_timer_ic_cb_t

Tipo: Callback
File: caos/drivers/inc/timer.h:66:66

```
typedef void (*ca_timer_ic_cb_t)(u32 ticks)
```

A callback function type used to handle timer input capture events in caOS.

Membri:

Nome	Tipo	Descrizione
ticks	u32	The timer counter value captured at the moment the event occurred. Represents the timestamp of the input signal.

Note:
This callback is executed within the context of the timer interrupt service routine (ISR). Keep the implementation extremely short and avoid blocking operations, mutexes, or heavy processing. If complex logic is required, use the callback to signal a task (e.g., via ca_notify_give_isr) and process the data there.

Uso:
Used when configuring hardware timers for input capture mode. You pass a function pointer of this type to the ca_timer_capture_event function to register your custom logic to be triggered whenever the timer hardware detects an input signal edge.

Esempio:

```
void my_capture_handler(u32 ticks) {
    /* Signal a task that a capture event occurred */
    ca_notify_give_isr(capture_task_handle);
}

void setup_timer(void) {
    /* Register the callback for the timer capture event */
    ca_timer_capture_event(TIMER_ID, my_capture_handler);
}
```

---

ca_timer_ic_src_t

Tipo: Enumerazione
File: caos/drivers/inc/timer.h:58:62

```
typedef enum {
CA_TIMER_IC_PIN = 0,
CA_TIMER_IC_LSE,
CA_TIMER_IC_LSI
} ca_timer_ic_src_t;
```

Defines the input source for a hardware timer's input capture or clocking mechanism.

Membri:

Nome	Tipo	Descrizione
CA_TIMER_IC_PIN	enum value	Uses an external physical GPIO pin as the input source.
CA_TIMER_IC_LSE	enum value	Uses the Low Speed External (LSE) crystal oscillator as the input source.
CA_TIMER_IC_LSI	enum value	Uses the Low Speed Internal (LSI) RC oscillator as the input source.

Note:
This enum is used to configure the hardware timer peripheral. It does not require manual allocation as it is a simple enumeration passed by value to initialization functions. Ensure the selected source is physically available and enabled on the STM32 hardware before calling the initialization function.

Uso:
Used primarily in the ca_timer_init_capture function to specify which signal should trigger the capture event or drive the timer counter.

Esempio:

```
#include "caos/drivers/inc/timer.h"

void setup_timer_capture(void) {
    /* Configure timer 2 to capture events from an external GPIO pin */
    /* The function ca_timer_init_capture takes the timer ID, the source, and a
    callback */
    ca_timer_init_capture(2, CA_TIMER_IC_PIN, NULL);

    /* Start the timer to begin monitoring the input source */
    ca_timer_start(2);
}
```

---

ca_timer_mode_t

Tipo: Enumerazione
File: caos/drivers/inc/timer.h:41:48

```
typedef enum {
TIMER_MODE_PERIODIC = 0,
TIMER_MODE_ONESHOT,
TIMER_MODE_COUNTER,
TIMER_MODE_ENCODER,
TIMER_MODE_COUNTER_IRQ,  /* down counter su clock ETR, IRQ ogni preset
    impulsi */
TIMER_MODE_PWM,          /* output PWM, fino a 4 canali per timer */
} ca_timer_mode_t;
```

Defines the operational behavior of a hardware timer peripheral in caOS.

Membri:

Nome	Tipo	Descrizione
CA_TIMER_MODE_PERIODIC	enum constant	Timer triggers an interrupt or event at a fixed, repeating time interval.
CA_TIMER_MODE_ONESHOT	enum constant	Timer triggers once after a delay and then stops automatically.
TIMER_MODE_COUNTER	enum constant	Timer increments or decrements based on external clock pulses.
CA_TIMER_MODE_ENCODER	enum constant	Configures the timer to decode quadrature signals from a rotary encoder.
TIMER_MODE_COUNTER_IRQ	enum constant	Counts external ETR pulses and triggers an interrupt when a preset threshold is reached.
CA_TIMER_MODE_PWM	enum constant	Configures the timer to generate Pulse Width Modulation signals on up to 4 output channels.

Note:
This enum is used during the hardware initialization phase of a timer. It determines which internal STM32 registers are configured. Ensure the selected mode is physically supported by the specific timer instance (e.g., not all timers support PWM or Encoder modes).

Uso:
Pass this value to the timer configuration structure before calling the driver's initialization function to define the timer's functional role in your application.

Esempio:

```
ca_timer_config_t my_timer_cfg = { .mode = CA_TIMER_MODE_PWM, .period = 1000, .
    prescaler = 84 }; ca_timer_init(TIMER2, &my_timer_cfg);
```

---

ca_tls_config_t

Tipo: Struttura
File: caos/net/inc/tls.h:90:96

```
typedef struct {
const u8   *cert_der;
u32         cert_len;
const u8   *private_key;            /* 32 bytes, P-256 */
const char *alpn;                   /* e.g. "http/1.1", NULL = none */
u32         handshake_timeout_ms;   /* each read during the handshake */
} ca_tls_config_t;
```

Configuration structure for initializing a TLS connection, containing identity credentials and handshake parameters.

Membri:

Nome	Tipo	Descrizione
cert_der	const u8 *	Pointer to the device certificate in DER format. Must remain valid for the duration of the TLS session.
cert_len	u32	Length of the certificate in bytes.
private_key	const u8 *	Pointer to the 32-byte P-256 private key. Must be kept secure and valid during the session.
alpn	const char *	Application-Layer Protocol Negotiation string (e.g., "http/1.1"). Set to NULL if not required.
handshake_timeout_ms	u32	Timeout in milliseconds for each read operation during the TLS handshake process.

Note:
The structure does not own the memory pointed to by cert_der or private_key; the caller must ensure these buffers persist as long as the TLS connection is active. This struct is typically passed by value or reference to TLS initialization routines.

Uso:
Used to define the security identity and handshake behavior before initiating a secure connection via ca_tls_* functions.

Esempio:

```c
#include "caos/net/inc/tls.h"

void setup_secure_connection(void) {
    static const u8 my_cert[] = {0x30, 0x82, ...}; // DER data
    static const u8 my_key[] = {0x01, 0x02, ...};  // 32 bytes

    ca_tls_config_t tls_cfg = {
        .cert_der = my_cert,
        .cert_len = sizeof(my_cert),
        .private_key = my_key,
        .alpn = "http/1.1",
        .handshake_timeout_ms = 5000
    };

    /* Proceed to use tls_cfg with ca_tls_* functions to establish connection */
}
```

---

ca_tls_dir_t

Tipo: Struttura
File: `caos/net/inc/tls.h:99:104`

```c
typedef struct {
ca_aes_ctx_t aes;
u8           iv[12];
u64          seq;
u8           secret[32];            /* traffic secret, for KeyUpdate */
} ca_tls_dir_t;
```

Represents the cryptographic state (AES context, IV, sequence number, and secret) for one direction of a TLS connection.

Membri:

Nome	Tipo	Descrizione
`aes`	`ca_aes_ctx_t`	The AES context used for encryption or decryption operations.
`iv`	`u8[12]`	The Initialization Vector (IV) used for the current record, typically 12 bytes for AES-GCM.
`seq`	`u64`	The 64-bit sequence number used to prevent replay attacks; must be incremented for every record.
`secret`	`u8[32]`	The current traffic secret, used for key derivation during KeyUpdate procedures.

Note:
This struct is typically part of a larger TLS session object. It is not automatically managed by the OS; the application or the TLS stack must ensure it is zeroed out using ca_memzero before use and after the connection is closed to prevent sensitive data leakage.

Uso:
Used internally by the TLS stack to maintain the state of the encryption/decryption stream. You will encounter this when implementing custom TLS record processing or debugging low-level secure communication layers.

Esempio:

```
ca_tls_dir_t tls_dir;

/* Initialize the structure to ensure no garbage data is present */
ca_memzero(&tls_dir, sizeof(ca_tls_dir_t));

/* Setup the AES context within the TLS direction struct */
ca_aes_ctx_setup(&tls_dir.aes, /* key pointer */ NULL, 32);

/* Increment sequence number before sending a new record */
tls_dir.seq++;
```

---

ca_tls_io_t

Tipo: Struttura
File: caos/net/inc/tls.h:77:81

```
typedef struct {
i32  (*send)(void *io, const u8 *data, u32 len);
i32  (*recv)(void *io, u8 *buf, u32 len, u32 timeout_ms);
void  *io;
} ca_tls_io_t;
```

An abstraction layer (I/O interface) used by the TLS stack to perform read/write operations on custom transport mediums.

Membri:

Nome	Tipo	Descrizione
send	i32 (*)(void *io, const u8 *data, u32 len)	Function pointer to the custom send implementation. Must return the number of bytes sent or a negative value on error.
recv	i32 (*)(void *io, u8 *buf, u32 len, u32 timeout_ms)	Function pointer to the custom receive implementation. Must return the number of bytes received or a negative value on error.
io	void *	Opaque pointer to the underlying transport object (e.g., a socket handle or a stream pointer) passed as the first argument to send/recv.

Note:
The struct is typically allocated by the user and passed to TLS initialization functions. The user is responsible for ensuring the 'io' pointer remains valid for the entire duration of the TLS session. The callbacks must be thread-safe if the TLS session is accessed from multiple tasks.

Uso:
Used to decouple the TLS library from specific hardware or network drivers. You define this struct to wrap your specific communication channel (like a TCP socket or a UART stream) so the TLS stack can perform encrypted communication over it.

Esempio:

```
static i32 my_send(void *io, const u8 *data, u32 len) { /* Use ca_tcp_send or similar to transmit data */ return (i32)len; } static i32 my_recv(void *io, u8 *buf, u32 len, u32 timeout_ms) { /* Use ca_tcp_recv or similar to receive data */ return 0; } void setup_tls(void *my_socket) { ca_tls_io_t tls_io; tls_io.send = my_send; tls_io.recv = my_recv; tls_io.io = my_socket; /* Now pass tls_io to the TLS service initialization */ }
```

---

ca_tls_t

Tipo: Struttura
File: caos/net/inc/tls.h:106:126

```
typedef struct {
ca_tls_io_t              io;
const ca_tls_config_t   *cfg;
ca_sha256_ctx_t          transcript;
ca_tls_dir_t             rd;
ca_tls_dir_t             wr;
bool                     rd_encrypted;
bool                     wr_encrypted;
bool                     established;
bool                     closed;
u8                       alert;          /* last alert sent or received */

/* Application data received but not yet handed to ca_tls_read */
u32                      app_off;
u32                      app_len;

u8                       rx[TLS_RX_BUF_SIZE];
u8                       tx[TLS_TX_BUF_SIZE];
u8                       hs[TLS_HS_BUF_SIZE];
u32                      hs_len;
} ca_tls_t;
```

Represents the state and buffers for a TLS connection session in caOS.

Membri:

Nome	Tipo	Descrizione
io	ca_tls_io_t	Abstraction for underlying transport (e.g., TCP socket).
cfg	const ca_tls_config_t *	Pointer to TLS configuration (certificates, keys). Must remain valid for the session duration.
transcript	ca_sha256_ctx_t	Internal context for handshake integrity verification.
rd	ca_tls_dir_t	Read direction state (keys, IVs).
wr	ca_tls_dir_t	Write direction state (keys, IVs).
rd_encrypted	bool	True if incoming data is currently encrypted.
wr_encrypted	bool	True if outgoing data is currently encrypted.
established	bool	True if the TLS handshake is complete.
closed	bool	True if the connection is terminated.
alert	u8	Stores the last TLS alert code received or sent.
app_off	u32	Offset of unread application data in the RX buffer.
app_len	u32	Length of unread application data in the RX buffer.
rx	u8[]	Buffer for incoming encrypted/decrypted data.
tx	u8[]	Buffer for outgoing encrypted/decrypted data.
hs	u8[]	Buffer for handshake messages.
hs_len	u32	Current length of data in the handshake buffer.

Note:
The ca_tls_t structure is typically allocated by the application. It must be zero-initialized before use. The structure maintains its own internal buffers (rx, tx, hs) defined by macros. Ownership remains with the caller, but it is passed by pointer to ca_tls_* functions which modify its internal state.

Uso:
Used to manage a TLS session. Initialize the struct, then pass it to ca_tls_accept (for servers) or use it with ca_tls_read and ca_tls_write to perform secure communication over a transport layer.

Esempio:

```
ca_tls_t my_tls_session;
ca_memset(&my_tls_session, 0, sizeof(ca_tls_t));

/* Configure the session with a pre-defined config */
my_tls_session.cfg = &my_app_tls_config;

/* Perform TLS handshake as a server */
if (ca_tls_accept(&my_tls_session) == 0) {
    /* Connection established, ready to read/write */
    u8 buffer[64];
    ca_tls_read(&my_tls_session, buffer, sizeof(buffer));
}

/* Close the session when finished */
ca_tls_close(&my_tls_session);
```

---

ca_uart_baud_rate_t

Tipo: Enumerazione
File: caos/drivers/inc/uart.h:28:43

```
typedef enum {
CA_UART_BAUD_9600    = 9600,      /*!< 9600 bps */
CA_UART_BAUD_19200   = 19200,     /*!< 19200 bps */
CA_UART_BAUD_38400   = 38400,     /*!< 38400 bps */
CA_UART_BAUD_57600   = 57600,     /*!< 57600 bps */
CA_UART_BAUD_115200  = 115200,    /*!< 115200 bps */
CA_UART_BAUD_230400  = 230400,    /*!< 230400 bps */
CA_UART_BAUD_460800  = 460800,    /*!< 460800 bps */
CA_UART_BAUD_921600  = 921600,    /*!< 921600 bps */
CA_UART_BAUD_1000000 = 1000000,   /*!< 1 Mbps */
CA_UART_BAUD_2000000 = 2000000,   /*!< 2 Mbps */
CA_UART_BAUD_3000000 = 3000000,   /*!< 3 Mbps */
CA_UART_BAUD_4000000 = 4000000,   /*!< 4 Mbps - Theoretical maximum for
    some STM32 */
CA_UART_BAUD_4500000 = 4500000,   /*!< 4.5 Mbps - Practical maximum for
    some STM32 */
CA_UART_BAUD_5000000 = 5000000,   /*!< 5 Mbps - Theoretical maximum for
    some high-performance STM32 */
} ca_uart_baud_rate_t;
```

An enumeration defining the supported communication speeds (baud rates) for UART peripheral configuration.

Membri:

Nome	Tipo	Descrizione
`CA_UART_BAUD_9600 to CA_UART_BAUD_5000000`	`enum constant`	Represents the target baud rate in bits p (bps). Selecting a value unsupported by ware clock configuration may result in co tion errors.

Note:
This enum is used to map human-readable baud rates to the integer values required by the hardware abstraction layer. Ensure the selected baud rate is compatible with the STM32 peripheral clock frequency (PCLK) to avoid high bit-error rates.

Uso:
Used as a parameter in UART initialization structures or configuration functions to set the transmission speed of the serial interface.

Esempio:

```
ca_uart_config_t uart_cfg = { .baud_rate = CA_UART_BAUD_115200, .parity =
    CA_UART_PARITY_NONE, .stop_bits = CA_UART_STOP_1 };
```

`ca_uart_rx_cb_t`

Tipo: Callback
File: `caos/drivers/inc/uart.h:46:46`

```
typedef void (*ca_uart_rx_cb_t)(u32 rx_data)
```

A function pointer type used to define a callback function that executes when a byte is received via UART.

Membri:

Nome	Tipo	Descrizione
`rx_data`	`u32`	The 32-bit value containing the received byte. Only the lowest 8 bits are typically valid data; higher bits should be ignored unless specified by the driver.

Note:
This callback is executed within the UART interrupt service routine (ISR) context. It must be kept extremely short and efficient to avoid blocking other system interrupts. Do not perform heavy processing or blocking operations inside this function.

Uso:
Used when configuring a UART peripheral to handle incoming data asynchronously. You assign a function matching this signature to the UART configuration structure before initializing the driver.

Esempio:

```
void my_uart_handler(u32 rx_data) {
    uint8_t byte = (uint8_t)(rx_data & 0xFF);
    // Process the received byte
}

// Usage in configuration
ca_uart_cfg_t config = {
    .rx_cb = my_uart_handler,
    // ... other config fields
};
```

---

ca_udp_binding_t

Tipo: Struttura
File: caos/net/inc/net_cfg.h:69:72

```
typedef struct {
u16          port;           /* UDP port number (0 = unused slot) */
ca_udp_rx_t handler;         /* callback, NULL = port open but no handler
    */
} ca_udp_binding_t;
```

A structure defining a UDP port binding, mapping a specific port number to a callback function for incoming packet processing.

Membri:

Nome	Tipo	Descrizione
port	u16	The UDP port number to listen on. Set to 0 to indicate an unused or disabled slot in the binding table.
handler	ca_udp_rx_t	A function pointer to the callback executed when a packet arrives on this port. If NULL, the port is considered open but packets will be dropped.

Note:
This structure is typically used as an element within a static array defined in the network configuration. The system iterates through this array to register listeners. Ensure the array is large enough to accommodate all required services; otherwise, new bindings will fail to register.

Uso:
Use this type when configuring the network stack's UDP listener table. It is essential for routing incoming UDP traffic to specific application-level logic.

Esempio:

```
static void my_udp_handler(u16 port, const u8 *data, u16 len) { /* Process
    packet */ } static const ca_udp_binding_t my_bindings[] = { { .port = 5000,
    .handler = my_udp_handler }, { .port = 0, .handler = NULL } };
```

---

ca_udp_cfg_t

Tipo: Struttura
File: caos/net/inc/net_cfg.h:74:77

```
typedef struct {
ca_udp_binding_t bindings[CA_UDP_MAX_PORTS];
u16              count;      /* number of active bindings */
} ca_udp_cfg_t;
```

Configuration structure used to define and register multiple UDP port bindings for the network stack.

Membri:

Nome	Tipo	Descrizione
bindings	ca_udp_binding_t[]	An array of UDP binding structures defining local ports and their associated callback functions. Must not exceed CA_UDP_MAX_PORTS.
count	u16	The number of valid entries currently populated in the bindings array. Must be less than or equal to CA_UDP_MAX_PORTS.

Note:
This structure is typically passed to the network initialization routine. The user is responsible for allocating this structure, usually as a static global variable to ensure it persists throughout the application lifecycle. Do not modify the contents after the network stack has been initialized.

Uso:
Use this struct when configuring the UDP layer of the caOS network stack. It allows you to batch-register multiple UDP listeners (ports) during system startup.

Esempio:

```
ca_udp_cfg_t udp_config = {
    .bindings = {
        { .port = 8080, .callback = my_udp_handler },
        { .port = 9090, .callback = my_other_handler }
    },
    .count = 2
};

/* Pass this configuration to the network stack initialization */
ca_net_init(&udp_config);
```

---

ca_udp_cli_rx_t

Tipo: Callback
File: caos/net/inc/udp_client.h:53:53

```
typedef void (*ca_udp_cli_rx_t)(u8 handle,
```

A callback function type triggered by the caOS UDP client stack whenever a new packet is received.

Membri:

Nome	Tipo	Descrizione
`handle`	`u8`	The unique identifier of the UDP client instance that received the data. Must be a valid handle returned by the initialization function.
`data`	`u8*`	Pointer to the buffer containing the received payload. Valid only for the duration of the callback execution.
`len`	`u16`	The length of the received data in bytes. Must be checked against buffer limits to prevent overflows.

Note:
This callback is executed within the context of the network task. It must be kept short and non-blocking to avoid starving the network stack. The memory pointed to by 'data' is managed by the stack and should not be freed or modified by the user.

Uso:
Used during the registration of a UDP client. You assign a function of this type to the client configuration structure so the system knows where to route incoming UDP traffic.

Esempio:

```
void my_udp_rx_handler(u8 handle, u8 *data, u16 len) { if (len > 0) {
    process_packet(data, len); } } ... ca_udp_cli_cfg_t cfg = { .rx_cb =
    my_udp_rx_handler };
```

`ca_udp_client_cfg_t`

Tipo: Struttura
File: `caos/net/inc/udp_client.h:61:69`

```
typedef struct {
ca_ipv4_t          server_ip;
u16              server_port;
u16              local_port;          /* 0 = ephemeral (49152 + slot) */
u32              arp_timeout_ms;
ca_mac_t           local_mac;           /* our MAC (for TX frames) */
ca_ipv4_t          local_ip;            /* our IP  (for TX frames) */
ca_udp_cli_rx_t on_rx;              /* NULL = use blocking recv */
} ca_udp_client_cfg_t;
```

Configuration structure used to define the network parameters and callback behavior for a caOS UDP client instance.

Membri:

Nome	Tipo	Descrizione
server_ip	ca_ipv4_t	The destination IPv4 address of the remote server. Must be a valid unicast or broadcast address.
server_port	u16	The destination UDP port number on the remote server.
local_port	u16	The local UDP port to bind. Set to 0 to allow the system to assign an ephemeral port automatically.
arp_timeout_ms	u32	Timeout in milliseconds to wait for an ARP resolution before failing the transmission.
local_mac	ca_mac_t	The hardware MAC address assigned to the local network interface.
local_ip	ca_ipv4_t	The local IPv4 address assigned to the network interface.
on_rx	ca_udp_cli_rx_t	Callback function pointer triggered upon receiving data. If set to NULL, the client operates in blocking mode.

Note:
This structure is typically passed by value or reference to the client initialization function. The user is responsible for allocating this struct, usually on the stack or as a static global. Ensure all fields are initialized to zero before setting specific values to avoid undefined behavior from uninitialized memory.

Uso:
Use this struct when initializing a new UDP client instance to define its network identity and communication target. It is passed as an argument to the client creation API to configure the socket before it starts processing packets.

Esempio:

```
ca_udp_client_cfg_t cfg = {0};
cfg.server_ip = CA_IPV4(192, 168, 1, 100);
cfg.server_port = 5000;
cfg.local_port = 0;
cfg.arp_timeout_ms = 1000;
cfg.local_mac = my_mac_addr;
cfg.local_ip = my_ip_addr;
cfg.on_rx = my_rx_callback;

ca_udp_client_init(&client_handle, &cfg);
```

---

ca_udp_frame_t

Tipo: Struttura
File: caos/net/inc/net_proto.h:118:123

```
typedef struct {
u16    src_port;    /* source port big-endian */
u16    dst_port;    /* destination port big-endian */
u16    length;      /* header + data big-endian */
u16    checksum;    /* 0 = not present (RFC 768) big-endian */
} __attribute__((packed)) ca_udp_frame_t;
```

Represents the standard UDP header structure used for network packet construction and parsing.

Membri:

Nome	Tipo	Descrizione
src_port	u16	The source port number in big-endian format. Must be set to the sender's port.
dst_port	u16	The destination port number in big-endian format. Must be set to the receiver's port.
length	u16	Total length of the UDP header plus the payload in bytes, in big-endian format.
checksum	u16	UDP checksum in big-endian format. Set to 0 to indicate that no checksum is provided, as per RFC 768.

Note:
This struct is marked with ___attribute___((packed)) to ensure no padding bytes are inserted by the compiler, matching the exact binary layout of a network UDP header. All multi-byte fields must be converted to network byte order (big-endian) using functions like htons() before transmission.

Uso:
Used when manually crafting raw UDP packets or parsing incoming network buffers. It is typically cast onto a byte array pointer at the start of the UDP payload section of an IP packet.

Esempio:

```
ca_udp_frame_t *udp_hdr = (ca_udp_frame_t *)buffer_ptr;
udp_hdr->src_port = htons(8080);
udp_hdr->dst_port = htons(9090);
udp_hdr->length = htons(sizeof(ca_udp_frame_t) + data_len);
udp_hdr->checksum = 0;
```

---

ca_udp_rx_t

Tipo: Callback
File: caos/net/inc/net_cfg.h:65:65

```
typedef void (*ca_udp_rx_t)(u16 buf_idx)
```

A callback function type triggered by the network stack when a new UDP packet is received.

Membri:

Nome	Tipo	Descrizione
buf_idx	u16	The index of the internal network buffer containing the received packet data. Must be used with network retrieval functions to access the payload.

Note:
This callback is executed within the context of the network task or an ISR depending on the driver implementation. It must be kept short to avoid blocking network traffic. The buffer index is managed by the network stack; do not attempt to free or manually modify the buffer index directly.

Uso:
Used to register a custom handler in the network configuration structure. Assign a function pointer of this type to the UDP receive hook to process incoming data packets asynchronously.

Esempio:

```c
void my_udp_handler(u16 buf_idx) {
    uint8_t *data = net_get_buffer_ptr(buf_idx);
    uint16_t len = net_get_buffer_len(buf_idx);
    /* Process packet data here */
    net_release_buffer(buf_idx);
}

/* In initialization code */
ca_udp_rx_t my_callback = my_udp_handler;
```

---

ca_udp_server_cfg_t

Tipo: Struttura
File: caos/net/inc/udp_server.h:68:77

```c
typedef struct {
u16                 port;               /* UDP port to listen on */
u8                  max_clients;        /* <= UDPSRV_MAX_CLIENTS */
u32                 client_timeout_ms;  /* inactivity timeout */
mac_t               local_mac;          /* our MAC (for TX frames) */
ipv4_t              local_ip;           /* our IP  (for TX frames) */
ca_udp_srv_rx_t     on_rx;              /* mandatory */
ca_udp_srv_event_t  on_connect;         /* NULL ok */
ca_udp_srv_event_t  on_disconnect;      /* NULL ok */
} ca_udp_server_cfg_t;
```

Configuration structure used to initialize and define the behavior of a UDP server instance in caOS.

Membri:

Nome	Tipo	Descrizione
port	u16	The UDP port number the server will bind to. Must be a valid non-reserved port.
max_clients	u8	Maximum number of concurrent clients allowed. Must be less than or equal to UDPSRV_MAX_CLIENTS.
client_timeout_ms	u32	Inactivity duration in milliseconds before a client is automatically disconnected.
local_mac	ca_mac_t	The local MAC address used for outgoing UDP frames.
local_ip	ca_ipv4_t	The local IPv4 address assigned to the server interface.
on_rx	ca_udp_srv_rx_t	Mandatory callback function triggered when data is received from a client.
on_connect	ca_udp_srv_event_t	Optional callback triggered when a new client connects. Can be set to NULL.
on_disconnect	ca_udp_srv_event_t	Optional callback triggered when a client disconnects. Can be set to NULL.

Note:
The structure is typically allocated on the stack before passing it to the server initialization function. The user is responsible for ensuring the memory remains valid during the initialization call. The 'on_rx' callback is mandatory; failing to provide it will result in undefined behavior or initialization failure.

Uso:

Use this struct to configure a UDP server before calling the server start or initialization functions. It acts as the primary configuration object for the network stack's UDP layer.

Esempio:

```
ca_udp_server_cfg_t cfg = {0};
cfg.port = 8080;
cfg.max_clients = 5;
cfg.client_timeout_ms = 30000;
cfg.local_mac = my_mac_addr;
cfg.local_ip = my_ip_addr;
cfg.on_rx = my_rx_callback;
cfg.on_connect = NULL;
cfg.on_disconnect = NULL;
ca_udp_server_init(&cfg);
```

---

ca_udp_srv_client_info_t

Tipo: Struttura
File: caos/net/inc/udp_server.h:136:141

```
typedef struct {
bool    connected;
ca_ipv4_t src_ip;
u16     src_port;
u8    ring_used;    /* packets queued in ring */
} ca_udp_srv_client_info_t;
```

Represents the current connection state and metadata of a remote client interacting with a caOS UDP server.

Membri:

Nome	Tipo	Descrizione
connected	bool	Indicates if the client is currently tracked by the server. Set to true when a packet is received from a new or existing client.
src_ip	ca_ipv4_t	The IPv4 address of the remote client. Must be a valid ca_ipv4_t structure.
src_port	u16	The UDP source port of the client. Valid range is 1 to 65535.
ring_used	u8	The number of packets currently buffered in the client's specific ring buffer. If this reaches the maximum capacity, new packets may be dropped.

Note:
This structure is typically managed internally by the caOS UDP server stack. It is usually passed as a pointer to callback functions or status retrieval APIs. Do not manually modify the 'ring_used' field as it is managed by the network interrupt service routine.

Uso:
Use this structure when implementing a UDP server callback to identify the sender of a packet or to monitor the traffic load of specific clients.

Esempio:

```
void my_udp_callback(ca_udp_srv_client_info_t *client_info) {
    if (client_info->connected) {
        printf("Packet from: %d.%d.%d.%d:%d\n",
```

```
                client_info->src_ip.addr[0], client_info->src_ip.addr[1],
                client_info->src_ip.addr[2], client_info->src_ip.addr[3],
                client_info->src_port);
        if (client_info->ring_used > 5) {
            /* Handle potential congestion */
        }
    }
}
```

---

ca_udp_srv_event_t

Tipo: Callback
File: caos/net/inc/udp_server.h:61:61

```
typedef void (*ca_udp_srv_event_t)(u8 client_id,
```

A callback function type triggered by the caOS UDP server when a new packet is received from a client.

Membri:

Nome	Tipo	Descrizione
client_id	u8	A unique identifier assigned to the remote client. Valid range is 0-255. If invalid, the server cannot track the client session.
data	u8*	Pointer to the buffer containing the received payload. Must not be modified outside the callback scope.
len	u16	The length of the received data in bytes. Ensure this does not exceed your local processing buffer size.

Note:
This callback is executed within the context of the UDP server task. Keep the implementation short to avoid blocking the network stack. Do not perform long-running operations or blocking calls inside this function.

Uso:
Used when initializing a UDP server instance via the ca_udp_srv_init function to handle incoming network traffic asynchronously.

Esempio:

```
void my_udp_handler(u8 client_id, u8 *data, u16 len) { if (len > 0) { /* Process
    incoming data */ } }

ca_udp_srv_cfg_t config = { .port = 8080, .on_event = my_udp_handler };
ca_udp_srv_init(&config);
```

---

ca_udp_srv_rx_t

Tipo: Callback
File: caos/net/inc/udp_server.h:56:56

```
typedef void (*ca_udp_srv_rx_t)(u8 client_id,
```

A callback function type triggered by the caOS UDP server whenever a new packet is received from a client.

Membri:

Nome	Tipo	Descrizione
`client_id`	`u8`	The unique identifier for the client session. Valid range is 0-255. If invalid, the server cannot route the response back to the correct socket.
`data`	`const u8 *`	Pointer to the buffer containing the received payload. Must not be modified or freed by the callback.
`len`	`u16`	The length of the received data in bytes. Ensure bounds checking before accessing the data pointer.

Note:
This callback is executed within the context of the UDP server task. It must be non-blocking and should not perform heavy computations or long delays to avoid starving the network stack. The memory pointed to by 'data' is managed by the caOS network stack and is only guaranteed to be valid for the duration of the callback execution.

Uso:
Used when initializing a UDP server instance. You assign a function pointer of this type to the 'rx_callback' field in the server configuration structure to handle incoming network traffic.

Esempio:

```
void my_udp_handler(u8 client_id, const u8 *data, u16 len) { if (len > 0) { /* Process incoming data */ } } ... ca_udp_srv_cfg_t cfg = { .rx_callback = my_udp_handler, /* other fields */ };
```

`ca_usb_cb_t`

Tipo: Struttura
File: `caos/drivers/inc/usb.h:56:63`

```
typedef struct {
ca_usb_reset_cb_t     on_reset;
ca_usb_mounted_cb_t   on_mounted;
ca_usb_unmounted_cb_t on_unmounted;
ca_usb_setup_cb_t     on_setup;
ca_usb_rx_cb_t        on_rx;
ca_usb_tx_done_cb_t   on_tx_done;
} ca_usb_cb_t;
```

A structure containing function pointers (callbacks) to handle USB events and data transfers in the caOS USB stack.

Membri:

Nome	Tipo	Descrizione
on_reset	ca_usb_reset_cb_t	Triggered when the USB bus reset signal is detected. Used to re-initialize endpoints.
on_mounted	ca_usb_mounted_cb_t	Triggered when the device is successfully enumerated by the host.
on_unmounted	ca_usb_unmounted_cb_t	Triggered when the device is disconnected or the host resets the port.
on_setup	ca_usb_setup_cb_t	Triggered when a SETUP packet is received on endpoint 0. Essential for handling standard USB requests.
on_rx	ca_usb_rx_cb_t	Triggered when data is received on a non-control endpoint.
on_tx_done	ca_usb_tx_done_cb_t	Triggered when a previously initiated data transmission has completed successfully.

Note:
This structure acts as an interface between the hardware driver and your application logic. You must provide a pointer to an instance of this struct when initializing the USB peripheral. Ensure all pointers are either valid functions or NULL if the event does not need handling. The struct must remain in memory (e.g., static or global) as long as the USB peripheral is active.

Uso:
Use this struct to define the behavior of your USB device. Pass it to the USB initialization function to register your custom logic for handling host communication.

Esempio:

```
static void my_usb_reset(void) { /* Handle reset */ } static const ca_usb_cb_t
    my_usb_callbacks = { .on_reset = my_usb_reset, .on_mounted = NULL, .
    on_unmounted = NULL, .on_setup = NULL, .on_rx = NULL, .on_tx_done = NULL };
```

---

ca_usb_cdc_cfg_t

Tipo: Struttura
File: caos/usb/inc/usb_cdc.h:59:64

```
typedef struct {
ca_usb_device_type_t header;        /* tag + ops (D19)     first field
    */
u16  mps;                            /* Data bulk MPS (64 FS, 512 HS); 0
    = 64 */
void (*on_rx)  (const u8 *buf, u16 len);          /* data from host */
void (*on_line)(const ca_usb_cdc_line_t *line, u8 control);  /* opt. */
} __attribute__((aligned(4))) ca_usb_cdc_cfg_t;
```

Configuration structure for initializing a USB CDC (Communication Device Class) interface in caOS.

Membri:

Nome	Tipo	Descrizione
header	ca_usb_device_type_t	Must be the first field. Contains the device tag and operation pointers required by the caOS USB stack to identify the driver type.
mps	u16	Maximum Packet Size for bulk endpoints. Use 64 for Full Speed (FS) or 512 for High Speed (HS). Setting this to 0 defaults to 64 bytes.
on_rx	void (*)(const u8 *buf, u16 len)	Callback function triggered when data is received from the USB host. 'buf' points to the received data, 'len' is the number of bytes.
on_line	void (*)(const ca_usb_cdc_line_t *line, u8 control)	Optional callback for line coding changes (baud rate, parity) or control signals (DTR/RTS). Can be set to NULL if not required.

Note:
The structure must be 4-byte aligned as per the attribute. The user is responsible for allocating this structure, typically as a static global variable, as it must persist for the lifetime of the USB connection. The 'header' field must be correctly initialized with the appropriate caOS USB device tag before passing the struct to the registration function.

Uso:
Used during the initialization phase of the USB stack to register a CDC device. It defines how the application handles incoming data and line state changes from the host.

Esempio:

```
static void my_rx_handler(const u8 *buf, u16 len) { /* Process data */ };

ca_usb_cdc_cfg_t my_cdc_cfg = {
    .header = CA_USB_CDC_TAG,
    .mps = 64,
    .on_rx = my_rx_handler,
    .on_line = NULL
};

/* Pass the address of this struct to the caOS USB registration API */
ca_usb_register((ca_usb_device_type_t *)&my_cdc_cfg);
```

---

ca_usb_cdc_line_t

Tipo: Struttura
File: caos/usb/inc/usb_cdc.h:54:59

```
typedef struct {
u32 baud;         /* dwDTERate */
u8  stop_bits;    /* 0=1, 1=1.5, 2=2 */
u8  parity;       /* 0=none,1=odd,2=even,3=mark,4=space */
u8  data_bits;    /* 5,6,7,8,16 */
} ca_usb_cdc_line_t;
```

Represents the communication parameters (line coding) for a USB CDC virtual COM port.

Membri:

Nome	Tipo	Descrizione
baud	u32	The communication speed in bits per second (e.g., 115200). Must be a standard value supported by the hardware.
stop_bits	u8	Number of stop bits: 0 for 1 bit, 1 for 1.5 bits, 2 for 2 bits.
parity	u8	Parity mode: 0=none, 1=odd, 2=even, 3=mark, 4=space.
data_bits	u8	Number of data bits per frame. Valid values are 5, 6, 7, 8, or 16.

Note:
This structure is typically populated by the USB stack when the host sends a SET_LINE_CODING request. The application should treat this as a read-only configuration snapshot unless implementing a custom USB class driver.

Uso:
Used to configure the UART peripheral or software buffer settings when the USB host changes the virtual serial port parameters.

Esempio:

```
ca_usb_cdc_line_t line_config = { .baud = 115200, .stop_bits = 0, .parity = 0, .data_bits = 8 };

void apply_usb_settings(ca_usb_cdc_line_t *config) {
    // Example logic to update hardware UART based on USB host request
    UART_SetBaudrate(config->baud);
    UART_SetFrameFormat(config->data_bits, config->parity, config->stop_bits);
}
```

---

ca_usb_class_ops_t

Tipo: Struttura
File: caos/usb/inc/usb_mgmt.h:57:85

```
typedef struct ca_usb_class_ops_s {
/* Appende interface(+IAD)+endpoint descriptors al buffer; ritorna byte appesi.
*iface_num_inout    incrementato dell'iface count usato. */
u16  (*build_iface_desc)(const ca_usb_device_type_t *cfg,
u8 *buf, u16 max, u8 *iface_num_inout);
/* Contributo PID composito (D14): PID_BASE_x + USB_x_EP_DATA, o 0. */
u16  (*pid_contribution)(const ca_usb_device_type_t *cfg);
/* Notifiche driver (NULL-safe, fan-out a tutti i servizi attivi). */
void (*on_reset)    (const ca_usb_device_type_t *cfg);
void (*on_mounted)  (const ca_usb_device_type_t *cfg, ca_usb_speed_t speed);
void (*on_unmounted)(const ca_usb_device_type_t *cfg);
/* Apre/chiude gli EP del servizio (SET_CONFIGURATION). */
void (*on_configure)(const ca_usb_device_type_t *cfg);
/* SET_INTERFACE: il mgmt la invoca su OGNI servizio attivo passando iface
e alt richiesti. Il servizio confronta iface col proprio range e, se di
sua competenza, apre/chiude i bulk EP (NCM alt0=idle, alt1=active).
Ritorna true se iface e' di sua competenza (ferma l'iterazione). */
```

```
18 bool (*on_set_interface)(const ca_usb_device_type_t *cfg, u8 iface, u8
       alt);
19 /* Class SETUP (bmRequestType.Type=1). Ritorna true se gestito, false
        STALL. */
20 bool (*on_setup)    (const ca_usb_device_type_t *cfg, const u8 setup[8])
       ;
21 /* Fan-out EP I/O: il servizio filtra ep_addr (pu  essere ep_owns);
       ritorna
22 true se l'EP    suo (ferma l'iterazione). */
23 bool (*on_rx)       (const ca_usb_device_type_t *cfg, u8 ep_addr, u16
       len);
24 bool (*on_tx_done)  (const ca_usb_device_type_t *cfg, u8 ep_addr);
25 /* String descriptor per index > 3 (iMACAddress NCM, iInterface, ...).
       Scrive
26 in buf un USB string descriptor completo (bLength, bDescriptorType=0x03,
27 UTF-16LE) e ritorna la lunghezza; 0 se l'index non e' suo. */
28 u16  (*get_string)  (const ca_usb_device_type_t *cfg, u8 idx, u8 *buf,
       u16 max);
29 } ca_usb_class_ops_t;
```

A structure containing function pointers that define the behavior and interface of a specific USB class driver in caOS.

Membri:

Nome	Tipo	Descrizione
build_iface_desc	u16 (*)(const ca_usb_device_type_t *, u8 *, u16, u8 *)	Callback to append USB descriptors to the provided buffer. Must increment iface_num_inout by the number of interfaces used.
pid_contribution	u16 (*)(const ca_usb_device_type_t *)	Returns the PID offset contribution for composite devices. Return 0 if not applicable.
on_reset	void (*)(const ca_usb_device_type_t *)	Called when a USB bus reset occurs. Used to reset internal state.
on_mounted	void (*)(const ca_usb_device_type_t *, ca_usb_speed_t)	Called when the device is successfully enumerated and mounted.
on_unmounted	void (*)(const ca_usb_device_type_t *)	Called when the device is disconnected or unmounted.
on_configure	void (*)(const ca_usb_device_type_t *)	Called during SET_CONFIGURATION to initialize endpoints.
on_set_interface	bool (*)(const ca_usb_device_type_t *, u8, u8)	Handles SET_INTERFACE requests. Return true if the interface index belongs to this class.
on_setup	bool (*)(const ca_usb_device_type_t *, const u8[8])	Handles class-specific SETUP requests. Return true if handled, false to trigger a STALL.
on_rx	bool (*)(const ca_usb_device_type_t *, u8, u16)	Called when data is received on an endpoint. Return true if the endpoint belongs to this class.
on_tx_done	bool (*)(const ca_usb_device_type_t *, u8)	Called when a transmission is complete on an endpoint.
get_string	u16 (*)(const ca_usb_device_type_t *, u8, u8 *, u16)	Provides string descriptors for indices > 3. Returns length of written descriptor or 0 if index is unknown.

Note:
This structure is typically defined as a constant global variable in your driver file. The caOS

USB management layer uses these pointers to perform 'fan-out' calls to all registered services. Ensure all pointers are set to NULL if not implemented to avoid crashes.

Uso:
Used when implementing a custom USB class (e.g., CDC, HID, or vendor-specific). You register this structure with the caOS USB stack during system initialization to link your driver logic to the USB hardware events.

Esempio:

```
static u16 my_class_build_iface(const ca_usb_device_type_t *cfg, u8 *buf, u16
    max, u8 *iface_num) { /* Implementation */ return 0; } static const
    ca_usb_class_ops_t my_usb_ops = { .build_iface_desc = my_class_build_iface,
    .on_reset = NULL, .on_mounted = NULL, .on_unmounted = NULL, .on_configure =
    NULL, .on_set_interface = NULL, .on_setup = NULL, .on_rx = NULL, .on_tx_done
     = NULL, .get_string = NULL };
```

---

ca_usb_custom_cfg_t

Tipo: Struttura
File: caos/usb/inc/usb_custom.h:36:44

```
typedef struct {
ca_usb_device_type_t header;        /* tag + ops (D19) */
u8   ep_data;                        /* data EP number (1..N), IN+OUT
    bulk */
u16  mps;                            /* bulk MPS (64 FS, 512 HS) */
void (*on_rx)     (u8 ep_addr, u16 len);
void (*on_tx_done)(u8 ep_addr);
void (*on_mounted)(void);            /* opt.: host has enumerated the
    device */
void (*on_unmounted)(void);          /* opt.: cable disconnected / host
    suspended */
} __attribute__((aligned(4))) ca_usb_custom_cfg_t;
```

Configuration structure used to define the behavior and callbacks for a custom USB device interface in caOS.

Membri:

Nome	Tipo	Descrizione
header	ca_usb_device_type_t	Contains the device tag and operation pointers. Must be initialized to identify the device class.
ep_data	u8	The endpoint number for bulk data transfer. Valid range is 1 to N, representing both IN and OUT directions.
mps	u16	Maximum Packet Size. Use 64 for Full Speed (FS) or 512 for High Speed (HS) operation.
on_rx	void (*)(u8, u16)	Callback triggered when data is received. Receives the endpoint address and length of data.
on_tx_done	void (*)(u8)	Callback triggered after a transmission is completed on the specified endpoint.
on_mounted	void (*)(void)	Optional callback executed when the USB host successfully enumerates the device.
on_unmounted	void (*)(void)	Optional callback executed when the USB cable is disconnected or the host suspends the device.

Note:

The structure must be 4-byte aligned as per the attribute. The user is responsible for allocating this structure, typically in static memory or global scope, as it must persist for the lifetime of the USB connection. Do not free this memory while the USB stack is active.

Uso:
Used during the initialization phase of a custom USB device to register the endpoint configuration and event handlers with the caOS USB stack.

Esempio:

```
static void my_rx_handler(u8 ep, u16 len) { /* Handle data */ } static void
    my_tx_handler(u8 ep) { /* Handle completion */ } ca_usb_custom_cfg_t
    my_usb_cfg = { .header = CA_USB_TYPE_CUSTOM, .ep_data = 1, .mps = 64, .on_rx
     = my_rx_handler, .on_tx_done = my_tx_handler, .on_mounted = NULL, .
    on_unmounted = NULL };
```

---

ca_usb_dbg_t

Tipo: Struttura
File: caos/drivers/inc/usb.h:131:167

```
typedef struct {
/* event counters (all families) */
u32 rst;
u32 enumdne;
u32 susp;
u32 wkup;
u32 setup;
u32 rxflvl;
u32 rx_push;
u32 rx_drop_full;
u32 oep_xfrc[USB_EP_MAX];
u32 iep_xfrc[USB_EP_MAX];
u32 ep_read [USB_EP_MAX];
u32 ep_write[USB_EP_MAX];
u32 wr_full [USB_EP_MAX];
/* EP state snapshot (H5xx USB_DRD_FS; zero on other families) */
u32 chep    [USB_EP_MAX];   /* Raw R_CHEP: STAT_RX[13:12], STAT_TX[5:4],
    DTOG, VTRX/VTTX */
u8  rxq_h   [USB_EP_MAX];   /* ring RX head (ISR) / tail (ep_read) */
u8  rxq_t   [USB_EP_MAX];
u8  txq_h   [USB_EP_MAX];   /* ring TX head (ep_write) / tail (ISR) */
u8  txq_t   [USB_EP_MAX];
u8  tx_act  [USB_EP_MAX];   /* an IN packet is armed in PMA */
u32 rx_drop [USB_EP_MAX];   /* OUT ACKed by HW but RX ring full -> lost
   */
u32 rx_cnt0 [USB_EP_MAX];   /* OUT received by ISR with cnt==0 */
u32 rx_len0 [USB_EP_MAX];   /* rxq entry with len==0 consumed by ep_read
    */
/* DWT cycle counter measurements (H7xx     zero on other families) */
u32 isr_rx_sum;
u32 isr_rx_max;
u32 isr_rx_cnt;
u32 iat_sum;
u32 iat_max;
u32 iat_min;
u32 iat_cnt;
```

```
u32 lat_sum[USB_EP_MAX];
u32 lat_max[USB_EP_MAX];
u32 lat_cnt[USB_EP_MAX];
} ca_usb_dbg_t;
```

A structure containing diagnostic counters and state snapshots for USB peripheral debugging.

Membri:

Nome	Tipo	Descrizione
rst	u32	Number of USB reset events detected.
enumdne	u32	Number of enumeration done events.
susp	u32	Number of suspend events.
wkup	u32	Number of wakeup events.
setup	u32	Number of SETUP packet events.
rxflvl	u32	Number of RX FIFO level interrupts.
rx_push	u32	Total packets pushed to RX ring buffer.
rx_drop_full	u32	Total packets dropped due to full RX ring buffer.
oep_xfrc	u32[]	Transfer complete events for OUT endpoints.
iep_xfrc	u32[]	Transfer complete events for IN endpoints.
ep_read	u32[]	Number of read operations per endpoint.
ep_write	u32[]	Number of write operations per endpoint.
wr_full	u32[]	Number of times write buffer was full per endpoint.
chep	u32[]	Raw hardware endpoint status register snapshot.
rxq_h	u8[]	RX ring buffer head index per endpoint.
rxq_t	u8[]	RX ring buffer tail index per endpoint.
txq_h	u8[]	TX ring buffer head index per endpoint.
txq_t	u8[]	TX ring buffer tail index per endpoint.
tx_act	u8[]	Flag indicating if an IN packet is currently armed in PMA.
rx_drop	u32[]	Packets dropped because RX ring was full.
rx_cnt0	u32[]	Count of zero-length packets received by ISR.
rx_len0	u32[]	Count of zero-length entries consumed by ep_read.
isr_rx_sum	u32	Sum of DWT cycles for RX ISR processing (H7xx only).
isr_rx_max	u32	Maximum DWT cycles for RX ISR processing (H7xx only).
isr_rx_cnt	u32	Count of RX ISR measurements (H7xx only).
iat_sum	u32	Sum of inter-arrival times (H7xx only).
iat_max	u32	Maximum inter-arrival time (H7xx only).
iat_min	u32	Minimum inter-arrival time (H7xx only).
iat_cnt	u32	Count of inter-arrival measurements (H7xx only).
lat_sum	u32[]	Sum of latency measurements per endpoint (H7xx only).
lat_max	u32[]	Maximum latency per endpoint (H7xx only).
lat_cnt	u32[]	Count of latency measurements per endpoint (H7xx only).

Note:
This structure is populated by the USB driver's internal diagnostic functions. It is typically allocated by the caller on the stack or as a static variable. The fields are updated by the USB ISR and driver functions. On non-H7xx/H5xx families, many fields (like DWT measurements or raw EP snapshots) will remain zero.

Uso:

Use this structure to retrieve and inspect the internal state of the USB peripheral for debugging communication issues, packet loss, or performance bottlenecks. Pass a pointer to this struct to the appropriate ca_usb_dbg_* functions.

Esempio:

```c
#include "caos/drivers/inc/usb.h"

void print_usb_stats(void) {
    ca_usb_dbg_t stats;

    /* Clear the structure before filling it */
    ca_memzero(&stats, sizeof(ca_usb_dbg_t));

    /* Retrieve current USB FS statistics */
    ca_usb_dbg_fs_stat(&stats);

    /* Print the number of reset events */
    ca_printf("USB Resets: %u\n", stats.rst);
}
```

---

ca_usb_dev_ops_t

Tipo: Struttura
File: `caos/drivers/inc/usb.h:66:81`

```c
typedef struct {
void            (*clock_init)    (u32 usb_num);
void            (*gpio_init)     (u32 usb_num);
bool            (*init)          (u32 usb_num, const ca_usb_cb_t *cb);
bool            (*ep_open)       (u32 usb_num, u8 ep_addr,
    ca_usb_ep_type_t type, u16 mps);
bool            (*ep_write)      (u32 usb_num, u8 ep_addr, const u8 *buf,
     u16 len);
u16             (*ep_read)       (u32 usb_num, u8 ep_addr, u8 *buf, u16
    max_len);
void            (*set_address)   (u32 usb_num, u8 addr);
ca_usb_speed_t  (*get_speed)     (u32 usb_num);
void            (*connect)       (u32 usb_num);
void            (*disconnect)    (u32 usb_num);
void            (*ep_stall)      (u32 usb_num, u8 ep_addr);
bool            (*ep_close)      (u32 usb_num, u8 ep_addr);
bool            (*ep_clear_stall)(u32 usb_num, u8 ep_addr);
bool            (*ep_is_stalled) (u32 usb_num, u8 ep_addr);
} ca_usb_dev_ops_t;
```

A structure containing function pointers that define the hardware-specific operations for a USB controller.

Membri:

Nome	Tipo	Descrizione
clock_init	void (*)(u32)	Configures the peripheral clock. Must be called before any other USB operation.
gpio_init	void (*)(u32)	Configures the physical pins (D+/D-) for USB communication.
init	bool (*)(u32, const ca_usb_cb_t *)	Initializes the USB controller and registers the callback structure. Returns true on success.
ep_open	bool (*)(u32, u8, ca_usb_ep_type_t, u16)	Configures a specific endpoint. ep_addr is the hardware address, type defines the transfer mode, mps is the max packet size.
ep_write	bool (*)(u32, u8, const u8 *, u16)	Queues data to be sent from the specified endpoint. Returns true if the transfer was initiated.
ep_read	u16 (*)(u32, u8, u8 *, u16)	Reads received data from the endpoint buffer into the provided memory. Returns the number of bytes read.
set_address	void (*)(u32, u8)	Sets the USB device address assigned by the host during enumeration.
get_speed	ca_usb_speed_t (*)(u32)	Returns the current negotiated connection speed (e.g., Full Speed, High Speed).
connect	void (*)(u32)	Enables the pull-up resistor on the D+ line to signal presence to the host.
disconnect	void (*)(u32)	Disables the pull-up resistor to simulate a physical unplug event.
ep_stall	void (*)(u32, u8)	Forces the endpoint into a STALL state, indicating a protocol error or lack of resources.
ep_close	bool (*)(u32, u8)	Disables the specified endpoint and releases associated hardware resources.
ep_clear_stall	bool (*)(u32, u8)	Clears the STALL condition on an endpoint to resume normal operation.
ep_is_stalled	bool (*)(u32, u8)	Checks if the specified endpoint is currently in a STALL state.

Note:
This struct acts as a Hardware Abstraction Layer (HAL). It is typically defined as a 'const' global variable in a driver file (e.g., usb_stm32.c) and passed to the caOS USB stack during initialization. The user does not allocate this; it is provided by the board support package.

Uso:
Use this to implement or interface with low-level USB hardware drivers. It allows the caOS USB stack to remain hardware-agnostic by calling these function pointers instead of direct register manipulation.

Esempio:

```
static const ca_usb_dev_ops_t my_usb_ops = {
    .clock_init = stm32_usb_clock_init,
    .gpio_init = stm32_usb_gpio_init,
    .init = stm32_usb_init,
    .ep_open = stm32_usb_ep_open,
    .ep_write = stm32_usb_ep_write,
    .ep_read = stm32_usb_ep_read,
    .set_address = stm32_usb_set_address,
    .get_speed = stm32_usb_get_speed,
    .connect = stm32_usb_connect,
    .disconnect = stm32_usb_disconnect,
    .ep_stall = stm32_usb_ep_stall,
    .ep_close = stm32_usb_ep_close,
    .ep_clear_stall = stm32_usb_ep_clear_stall,
    .ep_is_stalled = stm32_usb_ep_is_stalled
};
```

---

ca_usb_dfu_cfg_t

Tipo: Struttura
File: caos/usb/inc/usb_dfu.h:54:63

```
typedef struct {
ca_usb_device_type_t header;        /* tag + ops (D19) */
u16  detach_timeout_ms;             /* wDetachTimeOut in the functional
    desc */
u16  transfer_size;                 /* wTransferSize in the functional
    desc */
u8   attributes;                    /* bmAttributes (USB_DFU_ATTR_*); 0
       WILL_DETACH */
const char *iface_str;              /* iInterface (e.g. "caOS DFU");
    NULL     none */
/* Host requested DFU_DETACH: the app must switch to DFU mode (e.g. flag
no-init + reset to bootloader). NULL     DETACH ignored (ACK only). */
void (*on_detach)(void);
} __attribute__((aligned(4))) ca_usb_dfu_cfg_t;
```

Configuration structure for the USB Device Firmware Upgrade (DFU) interface in caOS.

Membri:

Nome	Tipo	Descrizione
header	ca_usb_device_type_t	Standard USB device header containing the interface tag and operation pointers. Must be initialized correctly to link the DFU interface to the USB stack.
detach_timeout_ms	u16	Time in milliseconds the host waits for the device to detach and re-enumerate in DFU mode. Typical values range from 100 to 1000.
transfer_size	u16	The size of the data blocks transferred during firmware upload/download. Must match the hardware buffer constraints, typically 64, 128, or 256 bytes.
attributes	u8	Bitmask defining DFU capabilities (e.g., can download, can upload). Set to 0 for default behavior (WILL_DETACH).
iface_str	const char *	Optional string descriptor for the DFU interface shown in the device manager. Set to NULL if no string is required.
on_detach	void (*)(void)	Callback function triggered when the host sends a DFU_DETACH request. The application should use this to set a bootloader flag and perform a system reset.

Note:
The structure must be aligned to 4 bytes as per the attribute. It is typically allocated in static memory or global scope as it must persist for the lifetime of the USB interface. The user is responsible for providing a valid callback if they wish to support automatic transition to the bootloader.

Uso:
Used when initializing the USB stack to register the DFU class. Pass this structure to the USB interface registration function to enable firmware updates over USB.

Esempio:

```
static void my_detach_handler(void) { /* Set bootloader flag and reset */
    NVIC_SystemReset(); } static const ca_usb_dfu_cfg_t my_dfu_cfg = { .header =
     { .tag = CA_USB_TAG_DFU }, .detach_timeout_ms = 250, .transfer_size = 128,
    .attributes = 0, .iface_str = "caOS DFU", .on_detach = my_detach_handler };
```

---

ca_usb_ep_type_t

Tipo: Enumerazione
File: caos/drivers/inc/usb.h:35:40

```
typedef enum {
USB_EP_TYPE_CTRL = 0,
USB_EP_TYPE_ISOC = 1,
USB_EP_TYPE_BULK = 2,
USB_EP_TYPE_INTR = 3,
} ca_usb_ep_type_t;
```

Defines the transfer type for a USB endpoint, determining how data is scheduled and handled by the USB controller.

Membri:

Nome	Tipo	Descrizione
`CA_USB_EP_TYPE_CTRL`	`enum value`	Control transfer type. Used for device enumeration and configuration. Mandatory for Endpoint 0.
`CA_USB_EP_TYPE_ISOC`	`enum value`	Isochronous transfer type. Used for time-sensitive data like audio/video where delivery speed is prioritized over error correction.
`CA_USB_EP_TYPE_BULK`	`enum value`	Bulk transfer type. Used for large data transfers where reliability is critical, such as file transfers or mass storage.
`CA_USB_EP_TYPE_INTR`	`enum value`	Interrupt transfer type. Used for small, low-latency data packets, such as mouse movements or keyboard presses.

Note:
This enum is used to configure the hardware registers of the STM32 USB peripheral. It must be set correctly during the endpoint initialization phase. Incorrect assignment will result in the USB host rejecting the device descriptor or failing to communicate with the specific endpoint.

Uso:
Used when defining the endpoint configuration structure before calling the USB driver initialization functions to register endpoints with the USB stack.

Esempio:

```
ca_usb_ep_type_t my_ep_type = CA_USB_EP_TYPE_BULK;

// Example of assigning the type to an endpoint configuration struct
struct usb_ep_config {
    uint8_t ep_addr;
    ca_usb_ep_type_t type;
    uint16_t max_packet_size;
};

struct usb_ep_config bulk_in = {
    .ep_addr = 0x81,
    .type = my_ep_type,
    .max_packet_size = 64
};
```

`ca_usb_mounted_cb_t`

Tipo: Callback
File: `caos/drivers/inc/usb.h:50:50`

```
typedef void (*ca_usb_mounted_cb_t)(ca_usb_speed_t speed)
```

A callback function type triggered by the USB driver when the device successfully completes enumeration and is mounted by the host.

Membri:

Nome	Tipo	Descrizione
`speed`	`ca_usb_speed_t`	Indicates the negotiated USB speed (e.g., Full Speed or High Speed). Incorrect handling of speed-specific buffers or endpoints may lead to communication errors.

Note:

This callback is executed within the context of the USB interrupt or the USB task, depending on the driver implementation. It must be kept short and non-blocking to avoid stalling the USB stack. The user is responsible for registering this callback via the driver configuration structure before initializing the USB peripheral.

Uso:
Use this callback to perform application-level initialization that requires the USB bus to be active, such as enabling specific USB classes (CDC, HID) or notifying the main application state machine that the device is ready for data transfer.

Esempio:

```
void my_usb_mounted_handler(ca_usb_speed_t speed) {
    if (speed == CA_USB_SPEED_FULL) {
        log_info("USB mounted at Full Speed");
    }
    app_state = APP_READY;
}

ca_usb_config_t usb_cfg = {
    .on_mounted = my_usb_mounted_handler
};
ca_usb_init(&usb_cfg);
```

---

ca_usb_msc_cfg_t

Tipo: Struttura
File: caos/usb/inc/usb_msc.h:56:63

```
typedef struct {
ca_usb_device_type_t header;        /* tag + ops (D19)     first field */
u16  mps;                            /* Bulk MPS (64 FS, 512 HS); 0 = 64 */
u32  block_size;                     /* typically 512 */
u32  block_count;                    /* total number of blocks (capacity) */
bool (*read) (u32 lba, u8 *buf, u32 count);
bool (*write)(u32 lba, const u8 *buf, u32 count);
} __attribute__((aligned(4))) ca_usb_msc_cfg_t;
```

Configuration structure for defining a USB Mass Storage Class (MSC) device interface in caOS.

Membri:

Nome	Tipo	Descrizione
header	ca_usb_device_type_t	Must be the first member; contains the device tag and operation pointers required by the caOS USB stack.
mps	u16	Maximum Packet Size for Bulk endpoints. Use 64 for Full Speed (FS) or 512 for High Speed (HS). Set to 0 to default to 64.
block_size	u32	Size of a single logical block in bytes, typically 512 for standard storage media.
block_count	u32	Total number of addressable blocks available on the storage medium.
read	bool (*)(u32, u8 *, u32)	Callback function pointer for reading data from the storage medium. Returns true on success, false on failure.
write	bool (*)(u32, const u8 *, u32)	Callback function pointer for writing data to the storage medium. Returns true on success, false on failure.

Note:
The structure must be 4-byte aligned as per the attribute. The user is responsible for allocating this structure, typically in static memory or global scope, as it must persist for the lifetime of the USB MSC service. The callbacks must be thread-safe if the underlying storage driver is accessed by other tasks.

Uso:
Used to register a custom storage backend (like an SD card or Flash memory) with the caOS USB MSC driver. Pass a pointer to this struct when initializing the USB MSC peripheral.

Esempio:

```
static bool my_read(u32 lba, u8 *buf, u32 count) { /* Read logic */ return true; }
static bool my_write(u32 lba, const u8 *buf, u32 count) { /* Write logic */ return true; }

ca_usb_msc_cfg_t my_msc_cfg = {
    .header = { .tag = CA_USB_MSC_TAG },
    .mps = 512,
    .block_size = 512,
    .block_count = 1024,
    .read = my_read,
    .write = my_write
};
```

---

ca_usb_ncm_cfg_t

Tipo: Struttura
File: caos/usb/inc/usb_ncm.h:71:76

```
typedef struct {
ca_usb_device_type_t header;     /* tag + ops (D19)      first field */
u16  mps;                         /* Bulk MPS (64 FS); 0 = 64 */
u8   mac[6];                      /* MAC exposed to the host (iMACAddress) */
ca_usb_ncm_rx_cb_t on_rx;         /* received Ethernet datagram */
} __attribute__((aligned(4))) ca_usb_ncm_cfg_t;
```

Configuration structure for initializing a USB Network Control Model (NCM) device interface.

Membri:

Nome	Tipo	Descrizione
`header`	`ca_usb_device_t`	Standard USB device header containing the device tag and operation table. Must be initialized with the appropriate NCM device type constant.
`mps`	`u16`	Maximum Packet Size for Bulk endpoints. Set to 0 to default to 64 bytes (Full Speed). Incorrect values may cause USB enumeration failures.
`mac`	`u8[6]`	The 6-byte hardware MAC address reported to the host via the iMACAddress descriptor. Must be unique on the network.
`on_rx`	`ca_usb_ncm_rx_cb_t`	Callback function pointer triggered when an Ethernet datagram is received from the host. Must not be NULL.

Note:
The structure must be 4-byte aligned as per the attribute. The user is responsible for allocating this struct, typically in static memory or global scope, as it must persist for the lifetime of the USB interface. Do not modify fields after the USB stack has been initialized.

Uso:
Used as the configuration parameter when registering or initializing an NCM USB class instance in the caOS USB stack.

Esempio:

```
static void my_rx_handler(const void *data, size_t len) { /* Process packet */ }
    static const ca_usb_ncm_cfg_t ncm_cfg = { .header = { .tag = CA_USB_DEV_NCM
    }, .mps = 512, .mac = {0x02, 0x00, 0x00, 0x00, 0x00, 0x01}, .on_rx =
   my_rx_handler };
```

`ca_usb_ncm_rx_cb_t`

Tipo: Callback
File: `caos/usb/inc/usb_ncm.h:71:71`

```
typedef void (*ca_usb_ncm_rx_cb_t)(const u8 *eth, u16 len)
```

A callback function type triggered by the caOS USB NCM stack whenever a new Ethernet frame is received.

Membri:

Nome	Tipo	Descrizione
`eth`	`const u8 *`	Pointer to the start of the received Ethernet frame buffer. Must be treated as read-only.
`len`	`u16`	The length of the received Ethernet frame in bytes.

Note:
This callback is executed within the context of the USB interrupt or the USB task. It must be kept as short as possible to avoid blocking the USB stack. The memory pointed to by 'eth' is owned by the USB driver and is only valid for the duration of the callback; do not store this pointer for later use.

Uso:

Used to register a custom handler for incoming network traffic. You assign a function of this type to the NCM configuration structure before initializing the USB NCM interface.

Esempio:

```
void my_ncm_rx_handler(const u8 *eth, u16 len) {
    if (len > 0) {
        process_packet(eth, len);
    }
}

// During initialization
ca_usb_ncm_config_t config;
config.rx_cb = my_ncm_rx_handler;
ca_usb_ncm_init(&config);
```

`ca_usb_reset_cb_t`

Tipo: Callback
File: `caos/drivers/inc/usb.h:49:49`

```
typedef void (*ca_usb_reset_cb_t)(void)
```

A function pointer type used to define a callback that executes when the USB bus receives a reset signal.

Membri:

Nome	Tipo	Descrizione
`void`	`void`	The callback takes no arguments and returns no value.

Note:
This callback is triggered by the USB hardware interrupt service routine. It must be kept extremely short and non-blocking to avoid delaying USB stack processing. It is typically registered during the USB driver initialization phase.

Uso:
Use this type when you need to perform specific hardware or software resets (e.g., clearing buffers or resetting state machines) immediately after the USB host initiates a bus reset.

Esempio:

```
void my_usb_reset_handler(void) {
    // Reset application-specific USB state
    my_app_buffer_index = 0;
}

// Registering the callback
ca_usb_set_reset_callback(my_usb_reset_handler);
```

`ca_usb_rx_cb_t`

Tipo: Callback
File: `caos/drivers/inc/usb.h:53:53`

```
typedef void (*ca_usb_rx_cb_t)(u8 ep_addr, u16 len)
```

A function pointer type used to define a callback that executes when the USB peripheral receives data on a specific endpoint.

Membri:

Nome	Tipo	Descrizione
`ep_addr`	`u8`	The address of the USB endpoint that received the data. Must match the endpoint configured during initialization.
`len`	`u16`	The number of bytes received in the current packet. Value ranges from 0 up to the maximum packet size defined for the endpoint.

Note:
This callback is triggered within the USB interrupt context. Keep the implementation short and non-blocking. Do not perform heavy processing or blocking RTOS calls here; instead, signal a task or use a queue to process the data.

Uso:
Used when registering a USB driver configuration. You assign a function matching this signature to the driver's receive callback field so the system knows which code to run when data arrives.

Esempio:

```
void my_usb_rx_handler(u8 ep_addr, u16 len) { if (ep_addr == 0x01) { /* Signal a task to read data */ } } ca_usb_config_t config = { .rx_cb = my_usb_rx_handler };
```

`ca_usb_setup_cb_t`

Tipo: Callback
File: `caos/drivers/inc/usb.h:52:52`

```
typedef void (*ca_usb_setup_cb_t)(const u8 setup[8])
```

A callback function type triggered by the USB driver when a SETUP packet is received from the host.

Membri:

Nome	Tipo	Descrizione
`setup`	`const u8[8]`	An 8-byte array containing the raw USB SETUP request data (bmRequestType, bRequest, wValue, wIndex, wLength). Must be treated as read-only.

Note:
This callback is executed within the USB interrupt context or a high-priority system task. Keep the implementation extremely short; do not perform blocking operations or heavy processing here. The memory for the setup array is managed by the USB hardware peripheral and is only valid for the duration of the callback execution.

Uso:
Used to handle custom USB control requests (e.g., vendor-specific commands or class-specific requests). You register this callback during the USB device initialization phase to allow your application to parse and respond to incoming control transfers.

Esempio:

```c
void my_usb_setup_handler(const u8 setup[8]) {
    if (setup[0] == 0xC0 && setup[1] == 0x01) {
        // Handle custom vendor request 0x01
    }
}

// During initialization:
ca_usb_set_setup_callback(my_usb_setup_handler);
```

---

ca_usb_speed_t

Tipo: Enumerazione
File: caos/drivers/inc/usb.h:42:45

```c
typedef enum {
USB_SPEED_FS = 0,    /* Full Speed 12 Mbit/s  */
USB_SPEED_HS = 1,    /* High Speed 480 Mbit/s */
} ca_usb_speed_t;
```

An enumeration defining the operational data rate for the USB peripheral.

Membri:

Nome	Tipo	Descrizione
CA_USB_SPEED_FS	enum constant	Full Speed mode, operating at 12 Mbit/s. Use this for standard USB 1.1/2.0 compatibility.
CA_USB_SPEED_HS	enum constant	High Speed mode, operating at 480 Mbit/s. Requires hardware support and appropriate PHY configuration.

Note:
This enum is used to configure the USB controller's clock and physical layer settings. Ensure the selected speed matches the capabilities of your hardware design and the connected device requirements.

Uso:
Used during the initialization phase of the USB driver to specify the desired bus speed before the USB stack is started.

Esempio:

```c
ca_usb_speed_t my_speed = CA_USB_SPEED_FS;

/* Configure the USB peripheral with the selected speed */
if (usb_init(my_speed) != CA_OK) {
    /* Handle initialization error */
}
```

---

ca_usb_tx_done_cb_t

Tipo: Callback
File: caos/drivers/inc/usb.h:54:54

```c
typedef void (*ca_usb_tx_done_cb_t)(u8 ep_addr)
```

A callback function type triggered by the USB driver when a data transmission (TX) operation is successfully completed.

Membri:

Nome	Tipo	Descrizione
`ep_addr`	`u8`	The address of the USB endpoint that finished sending data. Valid values are standard USB endpoint addresses (e.g., 0x81 for IN endpoint 1). Incorrect handling may lead to logic errors if the application tracks multiple endpoints.

Note:
This callback is executed within the context of the USB interrupt service routine (ISR). It must be kept extremely short and non-blocking. Do not perform heavy processing or call blocking RTOS primitives inside this function. The user is responsible for providing the function pointer, while the caOS USB driver manages the invocation.

Uso:
Used when registering a USB device configuration or initiating a transmission. It allows the application to know exactly when the hardware buffer is free to accept new data, preventing data loss or buffer overwrites.

Esempio:

```
void my_tx_complete_handler(u8 ep_addr) {
    if (ep_addr == 0x81) {
        // Signal a task that the buffer is ready for more data
        os_sem_post(&tx_ready_sem);
    }
}

// Usage in driver configuration
ca_usb_config_t config = {
    .tx_done_cb = my_tx_complete_handler
};
```

`ca_usb_type_t`

Tipo: Enumerazione
File: `caos/usb/inc/usb_mgmt.h:41:49`

```
typedef enum {
CA_USB_TYPE_NONE  = 0,
CA_USB_TYPE_CDC   = 1,
CA_USB_TYPE_MSC   = 2,
CA_USB_TYPE_NCM   = 3,
CA_USB_TYPE_DFU   = 4,
CA_USB_TYPE_CUSTOM = 5,   /* servizio vendor custom (step 6) */
CA_USB_TYPE__MAX
} ca_usb_type_t;
```

An enumeration defining the specific USB class or protocol mode supported by the caOS USB stack.

Membri:

Nome	Tipo	Descrizione
CA_USB_TYPE_NONE	enum value	USB interface is disabled or not initialized. Default state.
CA_USB_TYPE_CDC	enum value	Communication Device Class, typically used for virtual COM ports.
CA_USB_TYPE_MSC	enum value	Mass Storage Class, used for file system access (e.g., USB flash drive emulation).
CA_USB_TYPE_NCM	enum value	Network Control Model, used for Ethernet-over-USB networking.
CA_USB_TYPE_DFU	enum value	Device Firmware Upgrade, used for bootloader and firmware update procedures.
CA_USB_TYPE_CUSTOM	enum value	Vendor-specific protocol implementation.
CA_USB_TYPE__MAX	enum value	Boundary marker for validation; do not use as a valid configuration type.

Note:
This enum is used to configure the USB peripheral driver behavior. It must be set before the USB stack is initialized. The value CA_USB_TYPE___MAX is intended for bounds checking and should not be assigned to a configuration variable.

Uso:
Use this type when configuring the USB subsystem in your application initialization code to tell the caOS stack which USB class driver to load and manage.

Esempio:

```
ca_usb_type_t my_usb_mode = CA_USB_TYPE_CDC;

void setup_usb(void) {
    /* Configure the USB stack to operate as a Virtual COM port */
    usb_mgmt_init(my_usb_mode);
}
```

---

ca_usb_unmounted_cb_t

Tipo: Callback
File: caos/drivers/inc/usb.h:51:51

```
typedef void (*ca_usb_unmounted_cb_t)(void)
```

A callback function pointer type triggered by the USB driver when the device is physically disconnected or logically unmounted.

Membri:

Nome	Tipo	Descrizione
void	void	The callback takes no arguments and returns nothing.

Note:
This callback is executed within the context of the USB interrupt or the USB management task. It must be kept short and non-blocking to avoid stalling the USB stack. The user is responsible for assigning a valid function address to the driver configuration structure before initializing the USB peripheral.

Uso:

Used during USB driver initialization to register a custom cleanup routine. It is typically used to update the application state machine, stop data processing tasks, or notify the user interface that the USB connection is no longer active.

Esempio:

```
void my_usb_cleanup_handler(void) { /* Reset application state */ }

/* Inside initialization code */
ca_usb_config_t config;
config.on_unmounted = my_usb_cleanup_handler;
ca_usb_init(&config);
```

---

ca_wait_queue_t

Tipo: Struttura
File: caos/inc/sync.h:34:37

```
typedef struct {
ca_thread_context_t *head;  // Points to the first task in the chain
ca_thread_context_t *tail;  // Points to the last task in the chain
} ca_wait_queue_t;
```

A wait queue structure used to manage a list of threads that are blocked waiting for a specific resource or event.

Membri:

Nome	Tipo	Descrizione
head	ca_thread_context_t *	Pointer to the first thread in the waiting list. If NULL, the queue is empty.
tail	ca_thread_context_t *	Pointer to the last thread in the waiting list. Used to efficiently append new threads to the end of the queue.

Note:
This structure must be initialized to {NULL, NULL} before use. It is typically managed by the kernel's synchronization primitives (like semaphores or mutexes). Do not manually modify these pointers unless you are implementing custom synchronization logic, as it may lead to kernel instability or thread starvation.

Uso:
Used internally by caOS synchronization objects to track threads that are in the BLOCKED state. You will encounter this when inspecting kernel objects or implementing custom blocking mechanisms.

Esempio:

```
ca_wait_queue_t my_queue = {NULL, NULL};

// Example of checking if any threads are waiting
if (my_queue.head != NULL) {
    // Threads are currently blocked on this queue
} else {
    // Queue is empty
}
```

---

ca_wdt_early_cb_t

Tipo: Callback
File: caos/drivers/inc/wdt.h:38:38

```
typedef void (*ca_wdt_early_cb_t)(void *arg)
```

A callback function type triggered by the Watchdog Timer (WDT) shortly before the system reset occurs.

Membri:

Nome	Tipo	Descrizione
arg	void *	A user-defined pointer passed to the callback when it is executed. It is typically used to pass context data or state information.

Note:
This callback is executed in an interrupt context (ISR) or a high-priority system context depending on the hardware implementation. It must be extremely fast, non-blocking, and must not perform complex operations like memory allocation or heavy I/O. The callback is registered using ca_wdt_on_early.

Uso:
Use this to perform emergency logging, save critical state to non-volatile memory, or signal other tasks that a system reset is imminent due to a watchdog timeout.

Esempio:

```
#include "caos/drivers/inc/wdt.h"

/* Emergency callback function */
void my_wdt_emergency_handler(void *arg) {
    /* Perform minimal emergency actions here */
    /* Example: Log that a WDT reset is about to happen */
    ca_printf("WDT early warning triggered!\n");
}

void setup_watchdog(void) {
    /* Register the callback to be called before WDT reset */
    /* Note: Ensure the WDT is initialized before setting the callback */
    ca_wdt_on_early(my_wdt_emergency_handler, NULL);
}
```

---

ca_wdt_mode_t

Tipo: Enumerazione
File: caos/drivers/inc/wdt.h:30:33

```
typedef enum {
CA_WDT_MODE_RESET = 0,     /*!< MCU reset only (all families) */
CA_WDT_MODE_INTERRUPT,     /*!< Early warning interrupt + reset (if
    supported) */
} ca_wdt_mode_t;
```

Defines the behavior of the Watchdog Timer (WDT) when a timeout occurs.

Membri:

Nome	Tipo	Descrizione
CA_WDT_MODE_RESET	enum constant	The system will trigger a hardware MCU reset upon timeout. Supported by all STM32 families.
CA_WDT_MODE_INTERRUPT	enum constant	The system triggers an early warning interrupt before the reset. If the interrupt is not cleared or the WDT is not refreshed, a reset follows. Only available on specific hardware.

Note:
This enum is used during the initialization phase of the WDT driver. Ensure the hardware supports the interrupt mode before selecting CA_WDT_MODE_INTERRUPT, otherwise, the driver may default to reset or return an error.

Uso:
Pass this value to the WDT initialization function to configure the watchdog's recovery strategy. Use RESET for critical safety applications and INTERRUPT for debugging or logging purposes.

Esempio:

```
ca_wdt_config_t config = { .timeout_ms = 1000, .mode = CA_WDT_MODE_RESET };
    ca_wdt_init(&config);
```

---

ca_wdt_reset_t

Tipo: Enumerazione
File: caos/drivers/inc/wdt.h:44:48

```
typedef enum {
CA_WDT_RESET_NONE = 0,     /*!< the last reset was not a watchdog one */
CA_WDT_RESET_IWDG,         /*!< Independent Watchdog reset */
CA_WDT_RESET_WWDG,         /*!< Window Watchdog reset */
} ca_wdt_reset_t;
```

An enumeration representing the cause of the last system reset, specifically identifying if a watchdog triggered it.

Membri:

Nome	Tipo	Descrizione
CA_WDT_RESET_NONE	enum value	The system did not reset due to a watchdog; it was likely a power-on, manual reset, or software reset.
CA_WDT_RESET_IWDG	enum value	The system reset was triggered by the Independent Watchdog (IWDG).
CA_WDT_RESET_WWDG	enum value	The system reset was triggered by the Window Watchdog (WWDG).

Note:
This value is typically populated by the hardware abstraction layer during the boot process. It is read-only for the application. You should check this value early in your main() function to log or handle unexpected watchdog resets.

Uso:
Use this enum with the ca_wdt_reset_cause() function to determine why the system restarted after a crash or hang.

Esempio:

```c
#include "caos/drivers/inc/wdt.h"

void check_reset_reason(void) {
    ca_wdt_reset_t reason = ca_wdt_reset_cause();

    if (reason == CA_WDT_RESET_IWDG) {
        ca_printf("System recovered from IWDG reset!\n");
    } else if (reason == CA_WDT_RESET_WWDG) {
        ca_printf("System recovered from WWDG reset!\n");
    } else {
        ca_printf("Normal boot or non-watchdog reset.\n");
    }
}
```

---

ca_wdt_type_t

Tipo: Enumerazione
File: caos/drivers/inc/wdt.h:21:24

```c
typedef enum {
CA_WDT_TYPE_IWDG = 0,      /*!< Independent Watchdog (LSI clock) */
CA_WDT_TYPE_WWDG,          /*!< Window Watchdog (APB clock) */
} ca_wdt_type_t;
```

An enumeration defining the hardware watchdog timer peripheral type available on the STM32.

Membri:

Nome	Tipo	Descrizione
CA_WDT_TYPE_IWDG	enum constant	Independent Watchdog. Uses the internal LSI clock. Remains active even if the main system clock fails.
CA_WDT_TYPE_WWDG	enum constant	Window Watchdog. Uses the APB clock. Requires the application to refresh within a specific time window.

Note:
This enum is used to select the hardware peripheral during the initialization phase of the watchdog driver. It does not require manual allocation as it is a simple integer-based enumeration.

Uso:
Used as a parameter in the ca_wdt_init() function to specify which hardware watchdog peripheral the driver should configure and manage.

Esempio:

```c
ca_wdt_config_t config = { .type = CA_WDT_TYPE_IWDG, .timeout_ms = 1000 };
ca_wdt_init(&config);
```

Chapter 116

Reference Enumerazioni caOS

Enumerazioni globali in ordine alfabetico. Per ogni tipo: file sorgente e tabella completa degli enumeratori con il relativo valore.

`ca_aes_keysize_t`

Tipo: Enumerazione
File: `caos/crypto/inc/aes.h:34:38`

Defines the supported AES key lengths for cryptographic operations in the ca_aes_* API.

Enumeratore	Valore
`CA_AES_KEY_128`	0
`CA_AES_KEY_192`	1
`CA_AES_KEY_256`	2

Note:
The chosen key size must match the length of the key buffer provided to the ca_aes_ctx_setup function. Using an incorrect size will lead to undefined cryptographic behavior or hardware errors in the STM32 AES peripheral.

Uso:
This enum is passed as an argument to ca_aes_ctx_setup to configure the AES context before calling ca_aes_encrypt or ca_aes_decrypt.

`ca_aes_mode_t`

Tipo: Enumerazione
File: `caos/crypto/inc/aes.h:27:32`

Enumeratore	Valore
`CA_AES_MODE_ECB`	0
`CA_AES_MODE_CBC`	1
`CA_AES_MODE_CTR`	2
`CA_AES_MODE_GCM`	3

`ca_cordic_func_t`

Tipo: Enumerazione
File: `caos/drivers/inc/cordic.h:35:46`

Enumeratore	Valore
CA_CORDIC_COS	0
CA_CORDIC_SIN	*(auto)*
CA_CORDIC_PHASE	*(auto)*
CA_CORDIC_MODULUS	*(auto)*
CA_CORDIC_ATAN	*(auto)*
CA_CORDIC_COSH	*(auto)*
CA_CORDIC_SINH	*(auto)*
CA_CORDIC_ATANH	*(auto)*
CA_CORDIC_LN	*(auto)*
CA_CORDIC_SQRT	*(auto)*

---

ca_dac_comp_hyst_t

Tipo: Enumerazione
File: caos/drivers/inc/dac.h:46:51

Enumeratore	Valore
CA_DAC_COMP_HYST_NONE	0
CA_DAC_COMP_HYST_LOW	*(auto)*
CA_DAC_COMP_HYST_MEDIUM	*(auto)*
CA_DAC_COMP_HYST_HIGH	*(auto)*

---

ca_dma2d_am_t

Tipo: Enumerazione
File: caos/drivers/inc/dma2d.h:66:70

Enumeratore	Valore
CA_DMA2D_ALPHA_KEEP	0
CA_DMA2D_ALPHA_REPLACE	*(auto)*
CA_DMA2D_ALPHA_MULT	*(auto)*

---

ca_dma2d_fmt_t

Tipo: Enumerazione
File: caos/drivers/inc/dma2d.h:49:61

Enumeratore	Valore
CA_DMA2D_ARGB8888	0
CA_DMA2D_RGB888	*(auto)*
CA_DMA2D_RGB565	*(auto)*
CA_DMA2D_ARGB1555	*(auto)*
CA_DMA2D_ARGB4444	*(auto)*
CA_DMA2D_L8	*(auto)*
CA_DMA2D_AL44	*(auto)*
CA_DMA2D_AL88	*(auto)*
CA_DMA2D_L4	*(auto)*
CA_DMA2D_A8	*(auto)*
CA_DMA2D_A4	*(auto)*

`ca_dma_dir_t`

Tipo: Enumerazione
File: `caos/drivers/inc/dma.h:56:59`

Defines the direction of data transfer for DMA operations in caOS drivers.

Enumeratore	Valore
`CA_DMA_DIR_M2P`	0
`CA_DMA_DIR_P2M`	1

Note:
This enum is used to configure the DMA controller's flow direction. Choosing the wrong direction will result in data not being transferred or hardware errors. Ensure the memory address provided is valid for the chosen direction (e.g., source buffer for M2P, destination buffer for P2M).

Uso:
This type is used in ca_dma_set_request and related DMA initialization functions to define the data flow path for a specific DMA channel.

`ca_dma_width_t`

Tipo: Enumerazione
File: `caos/drivers/inc/dma.h:62:66`

Defines the data width for DMA transfer operations, used in caOS DMA configuration structures.

Enumeratore	Valore
`CA_DMA_WIDTH_8`	0
`CA_DMA_WIDTH_16`	1
`CA_DMA_WIDTH_32`	2

Note:
The width must match the destination/source register size. Mismatched widths between source and destination can lead to data corruption or hardware-level alignment faults. Ensure the memory address is aligned to the chosen width (e.g., 4-byte alignment for 32-bit transfers).

Uso:
Used in the ca_dma_config_t struct passed to ca_dma_init(). Choose the value that matches the data type of the buffer or the peripheral register being accessed.

`ca_exti_edge_t`

Tipo: Enumerazione
File: `caos/drivers/inc/exti.h:31:35`

Defines the signal transition trigger condition for external interrupts (EXTI) in the caOS driver API.

Enumeratore	Valore
`CA_EXTI_EDGE_RISING`	0
`CA_EXTI_EDGE_FALLING`	1
`CA_EXTI_EDGE_BOTH`	2

Note:

The hardware configuration depends on the specific STM32 EXTI line capabilities. Ensure the GPIO pin is correctly configured with internal pull-up or pull-down resistors if no external hardware resistor is present, otherwise the pin will float and trigger spurious interrupts.

Uso:
This enum is passed to the 'ca_exti_config' function during peripheral initialization. Choose the value based on the electrical characteristics of your input signal and the desired logic state change.

`ca_fat_event_t`

Tipo: Enumerazione
File: `caos/drivers/inc/fat_service.h:35:39`

Enumeratore	Valore
`CA_FAT_EV_MOUNTED`	`0`
`CA_FAT_EV_UNMOUNTING`	*(auto)*
`CA_FAT_EV_DESTRUCTIVE`	*(auto)*

`ca_gpio_mode_t`

Tipo: Enumerazione
File: `caos/drivers/inc/gpio_driver.h:66:71`

Defines the operational mode of an STM32 GPIO pin, used in the caOS GPIO configuration API.

Enumeratore	Valore
`CA_GPIO_MODE_INPUT`	`0`
`CA_GPIO_MODE_OUTPUT`	*(auto)*
`CA_GPIO_MODE_AF`	*(auto)*
`CA_GPIO_MODE_ANALOG`	*(auto)*

Note:
The default value is typically CA_GPIO_MODE_INPUT to ensure safety upon reset. Ensure the pin is not configured as an output if it is connected to an external voltage source to avoid short circuits. On some STM32 families, AF mode requires an additional register configuration (AFR) to select the specific peripheral mapping.

Uso:
This enum is passed to the 'gpio_init()' function. Choose the mode based on the hardware schematic: use INPUT for sensors, OUTPUT for actuators, AF for communication peripherals, and ANALOG for voltage measurement.

`ca_gpio_otype_t`

Tipo: Enumerazione
File: `caos/drivers/inc/gpio_driver.h:76:79`

Defines the output type configuration for STM32 GPIO pins, used in the GPIO initialization API.

Enumeratore	Valore
`CA_GPIO_OTYPE_PP`	`0`
`CA_GPIO_OTYPE_OD`	*(auto)*

Note:

CA_GPIO_OTYPE_PP is the default state for most GPIO pins after reset. Choosing Open-Drain without an external pull-up will result in the signal floating, which can cause unpredictable logic levels. Ensure the hardware schematic supports the chosen mode, especially for communication protocols.

Uso:
This enum is passed to the 'gpio_init' function within the GPIO configuration structure. Choose PP for general-purpose outputs and OD for communication buses or scenarios requiring wired-OR logic.

`ca_gpio_pin_t`

Tipo: Enumerazione
File: `caos/drivers/inc/gpio_driver.h:44:61`

Represents the specific physical pin index (0-15) on a GPIO port, used in all caOS GPIO configuration and control APIs.

Enumeratore	Valore
`CA_GPIO_PIN_0`	`0`
`CA_GPIO_PIN_1`	*(auto)*
`CA_GPIO_PIN_2`	*(auto)*
`CA_GPIO_PIN_3`	*(auto)*
`CA_GPIO_PIN_4`	*(auto)*
`CA_GPIO_PIN_5`	*(auto)*
`CA_GPIO_PIN_6`	*(auto)*
`CA_GPIO_PIN_7`	*(auto)*
`CA_GPIO_PIN_8`	*(auto)*
`CA_GPIO_PIN_9`	*(auto)*
`CA_GPIO_PIN_10`	*(auto)*
`CA_GPIO_PIN_11`	*(auto)*
`CA_GPIO_PIN_12`	*(auto)*
`CA_GPIO_PIN_13`	*(auto)*
`CA_GPIO_PIN_14`	*(auto)*
`CA_GPIO_PIN_15`	*(auto)*

Note:
These values correspond directly to the bit position in the STM32 GPIO IDR/ODR registers. Ensure the physical pin is not being used by the debugger (e.g., PA13/PA14) before assigning it to general logic, or you may lose connection to the board.

Uso:
Used in functions like gpio_init(), gpio_write(), and gpio_read(). Select the value based on the schematic pinout of your specific STM32 target board.

`ca_gpio_port_t`

Tipo: Enumerazione
File: `caos/drivers/inc/gpio_driver.h:27:39`

Represents the physical GPIO port identifier on the STM32 microcontroller, used in all caOS GPIO configuration and control functions.

Enumeratore	Valore
CA_GPIO_PORT_A	0
CA_GPIO_PORT_B	*(auto)*
CA_GPIO_PORT_C	*(auto)*
CA_GPIO_PORT_D	*(auto)*
CA_GPIO_PORT_E	*(auto)*
CA_GPIO_PORT_F	*(auto)*
CA_GPIO_PORT_G	*(auto)*
CA_GPIO_PORT_H	*(auto)*
CA_GPIO_PORT_I	*(auto)*
CA_GPIO_PORT_J	*(auto)*
CA_GPIO_PORT_K	*(auto)*

Note:
Not all ports are available on every STM32 chip. Attempting to configure a port that does not exist on your specific MCU will result in undefined behavior or a hard fault. Always consult the datasheet for your specific part number to confirm port availability.

Uso:
Used as the first argument in functions like caos_gpio_init(), caos_gpio_write(), and caos_gpio_read(). Choose the port based on the physical pin label found on the schematic (e.g., PA5 corresponds to CA_GPIO_PORT_A).

---

ca_gpio_pull_t

Tipo: Enumerazione
File: caos/drivers/inc/gpio_driver.h:94:98

Defines the internal resistor configuration for STM32 GPIO pins, used in GPIO initialization and configuration APIs.

Enumeratore	Valore
CA_GPIO_NOPULL	0
CA_GPIO_PULLUP	*(auto)*
CA_GPIO_PULLDOWN	*(auto)*

Note:
The internal pull resistors are typically weak (approx 40k ohms). They are not a substitute for strong external resistors in high-speed communication or noisy environments. Do not enable both pull-up and pull-down simultaneously on the same pin as it may cause unexpected behavior or increased power consumption.

Uso:
This enum is passed to the 'caos_gpio_init' function within the 'GPIO_InitTypeDef' structure. Choose 'CA_GPIO_NOPULL' for standard outputs, 'CA_GPIO_PULLUP' for active-low inputs (e.g., buttons connected to GND), and 'GPIO_PULLDOWN' for active-high inputs.

---

ca_gpio_speed_t

Tipo: Enumerazione
File: caos/drivers/inc/gpio_driver.h:84:89

Defines the output slew rate of a GPIO pin, used in the GPIO configuration API to balance signal integrity against electromagnetic interference (EMI).

Enumeratore	Valore
CA_GPIO_SPEED_LOW	0
CA_GPIO_SPEED_MED	*(auto)*
CA_GPIO_SPEED_HIGH	*(auto)*
CA_GPIO_SPEED_VERY_HIGH	*(auto)*

Note:
Higher speeds increase the risk of signal ringing and ground bounce. Always choose the lowest speed that meets your timing requirements to ensure system stability and compliance with EMC standards. The default value is typically CA_GPIO_SPEED_LOW.

Uso:
Used in the 'gpio_init_t' structure passed to 'gpio_init()'. Choose the speed based on the maximum toggle frequency of the specific pin function.

---

ca_hash_algo_t

Tipo: Enumerazione
File: caos/drivers/inc/hash.h:28:31

Defines the cryptographic hashing algorithm to be used by the caOS hardware abstraction layer for the STM32 HASH peripheral.

Enumeratore	Valore
CA_HASH_ALGO_SHA1	0
CA_HASH_ALGO_SHA256	1

Note:
The hardware peripheral must be initialized before selecting an algorithm. Switching algorithms mid-operation without resetting the HASH peripheral will result in undefined behavior. SHA-256 is significantly more computationally intensive than SHA-1, though the STM32 hardware accelerator minimizes this impact.

Uso:
This enum is passed as an argument to 'ca_hash_init()' to configure the hardware engine. Choose CA_HASH_ALGO_SHA256 for security-critical applications or CA_HASH_ALGO_SHA1 only when interfacing with existing legacy systems.

---

ca_i2c_speed_t

Tipo: Enumerazione
File: caos/drivers/inc/i2c.h:25:28

Defines the operating clock frequency mode for the I2C peripheral in the caOS I2C driver API.

Enumeratore	Valore
CA_I2C_SPEED_STANDARD	0
CA_I2C_SPEED_FAST	1

Note:
The actual timing parameters (rise/fall times) are calculated by the driver based on the peripheral clock frequency. Ensure your pull-up resistors are sized correctly; 400 kHz requires lower resistance values than 100 kHz to avoid signal rounding.

Uso:

This enum is passed as a parameter to the ca_i2c_init() function during peripheral configuration. Choose STANDARD for general-purpose communication and FAST when the slave device supports it and bus speed is a bottleneck.

---

ca_job_type

Tipo: Enumerazione
File: caos/inc/scheduler.h:55:62

Defines the execution context and hardware resource requirements for a task, used in the ca_scheduler_add_job API.

Enumeratore	Valore
CA_JOB_STD	*(auto)*
JOB_SAFE	*(auto)*
JOB_FLOAT_POINT	*(auto)*
CA_JOB_SAFE_FLOAT_POINT	*(auto)*

Note:
Choosing a FLOAT_POINT type on an MCU without a hardware FPU will trigger a software emulation overhead, significantly slowing down the task. JOB_SAFE types consume more RAM due to the inclusion of stack guard zones. Do not use CA_JOB_STD if the task uses float variables, as the FPU registers will not be saved, leading to non-deterministic behavior.

Uso:
This enum is passed as the second argument to ca_scheduler_add_job. Choose CA_JOB_STD by default for performance, and upgrade to JOB_FLOAT_POINT only if the compiler flags indicate floating-point usage.

---

ca_jpeg_css_t

Tipo: Enumerazione
File: caos/drivers/inc/jpeg.h:45:50

Enumeratore	Valore
CA_JPEG_GRAY	0
CA_JPEG_444	*(auto)*
CA_JPEG_422	*(auto)*
CA_JPEG_420	*(auto)*

---

ca_keys_store_access_mode_t

Tipo: Enumerazione
File: caos/crypto/inc/keys_store.h:44:49

Defines the permission levels for accessing cryptographic keys stored in the caOS secure key store, used in key management APIs.

Enumeratore	Valore
CA_KEYS_STORE_NO_ACCESS	0
CA_KEYS_STORE_READ_ONLY	1
CA_KEYS_STORE_READ_WRITE	2
CA_KEYS_STORE_EXEC_ONLY	3

Note:

The default value is typically CA_KEYS_STORE_NO_ACCESS for security reasons. Once a key is set to CA_KEYS_STORE_EXEC_ONLY, it cannot be read back, meaning you must keep a secure backup if the key needs to be restored later. Changing access modes may require a system reset or specific privilege levels depending on the STM32 hardware security module (HSM) configuration.

Uso:
This enum is passed to 'keys_store_set_access()' and 'keys_store_init()' functions. Choose the most restrictive mode possible (Principle of Least Privilege) to minimize the impact of potential software vulnerabilities.

`ca_l2_result_t`

Tipo: Enumerazione
File: `caos/net/inc/net_proto.h:192:197`

Defines the packet filtering or acceptance policy for the network layer (L2) in caOS, used primarily in ca_net_proto_l2.

Enumeratore	Valore
`CA_L2_ACCEPT_UNICAST`	`0`
`CA_L2_ACCEPT_BROADCAST`	`1`
`CA_L2_ACCEPT_MULTICAST`	`2`
`CA_L2_DROP`	`3`

Note:
The default behavior depends on the network interface configuration. These values are typically used to configure hardware filters or software hooks in the network stack. Choosing CA_L2_DROP is useful for security or to reduce CPU load by ignoring irrelevant traffic early in the processing pipeline.

Uso:
This enum is used within the ca_net_proto_l2 function to determine how the network stack should handle incoming frames. When implementing a custom network filter or configuring the Ethernet controller, pass one of these values to define the acceptance policy.

`ca_lptim_mode_t`

Tipo: Enumerazione
File: `caos/drivers/inc/lptim.h:47:50`

Defines the operational behavior of the Low Power Timer (LPTIM) peripheral within the caOS hardware abstraction layer.

Enumeratore	Valore
`CA_LPTIM_MODE_PERIODIC`	`0`
`CA_LPTIM_MODE_ONESHOT`	*(auto)*

Note:
The default value is CA_LPTIM_MODE_PERIODIC. Ensure the LPTIM clock source is configured correctly in the RCC before initialization, as some modes may not be supported depending on the specific STM32 series (e.g., L0 vs U5). A common mistake is using ONESHOT mode when a continuous timebase is required for RTOS scheduling.

Uso:

This enum is passed to the ca_lptim_init() function during peripheral configuration. Choose PERIODIC for continuous background tasks and ONESHOT for transient, event-driven operations.

---

ca_opamp_input_t

Tipo: Enumerazione
File: caos/drivers/inc/opamp.h:30:33

Enumeratore	Valore
CA_OPAMP_IN_PIN	0
CA_OPAMP_IN_DAC	*(auto)*

---

ca_pwr_mode_t

Tipo: Enumerazione
File: caos/drivers/inc/pwr.h:30:34

Defines the power consumption state for the MCU, used in the ca_pwr_enter API.

Enumeratore	Valore
CA_PWR_MODE_SLEEP	0
CA_PWR_MODE_STOP	*(auto)*
PWR_MODE_STANDBY	*(auto)*

Note:
The power modes are mutually exclusive. Choosing the wrong mode can lead to unexpected system resets (Standby) or failure to wake up if the interrupt source is disabled in that specific mode. Always ensure the required wakeup interrupt is configured before entering low power.

Uso:
This enum is passed to the ca_pwr_enter(ca_pwr_mode_t mode) function. Select the mode based on the required balance between power consumption and wakeup latency.

---

ca_qspi_pin_t

Tipo: Enumerazione
File: caos/drivers/inc/qspi.h:147:150

Enumeratore	Valore
CA_QSPI_PIN_CLK	0, CA_QSPI_PIN_NCS, CA_QSPI_PIN_IO0

---

ca_rtc_clk_src_t

Tipo: Enumerazione
File: caos/drivers/inc/rtc.h:32:37

Defines the clock source for the RTC peripheral in the caOS RTC driver API.

Enumeratore	Valore
CA_RTC_CLK_AUTO	0
CA_RTC_CLK_LSE	1
CA_RTC_CLK_LSI	2
CA_RTC_CLK_HSE	3

Note:

The clock source must be configured before the RTC is initialized. Changing the clock source while the RTC is running may lead to undefined behavior or loss of time data. Ensure the corresponding oscillator is enabled in the system clock configuration before selecting it here.

Uso:
This enum is passed to the ca_rtc_init() function. Choose CA_RTC_CLK_LSE for production devices requiring accurate timekeeping, and CA_RTC_CLK_LSI for low-cost prototypes without an external crystal.

`ca_sai_proto_t`

Tipo: Enumerazione
File: `caos/drivers/inc/sai.h:60:64`

Enumeratore	Valore
`CA_SAI_I2S`	`0`
`CA_SAI_MSB`	*(auto)*
`CA_SAI_SPDIF`	*(auto)*

`ca_scheduler_tasks_info_mode`

Tipo: Enumerazione
File: `caos/inc/scheduler.h:427:433`

Defines the level of detail and data source for task information retrieval in the caOS scheduler API.

Enumeratore	Valore
`CA_TH_INFO_DEFAULT`	`0`
`CA_TH_INFO_FLAG`	*(auto)*
`TH_INFO_RUNTIME`	*(auto)*
`CA_TH_INFO_MEM`	*(auto)*
`TH_INFO_ASSERT`	*(auto)*

Note:
CA_TH_INFO_DEFAULT is the fallback mode if an invalid value is passed. These modes are mutually exclusive; you can only request one type of information per function call. Note that TH_INFO_RUNTIME requires the scheduler's runtime statistics feature to be enabled in the configuration file, otherwise it may return zeroed data.

Uso:
This enum is used as an argument in the 'ca_scheduler_get_task_info' function. Choose the mode based on the specific diagnostic data required: use DEFAULT for general status, MEM for stack analysis, or RUNTIME for performance tuning.

`ca_secure_pipe_end_t`

Tipo: Enumerazione
File: `caos/crypto/inc/secure_pipe_service.h:39:42`

Enumeratore	Valore
`CA_SECURE_PIPE_A`	`0U`
`CA_SECURE_PIPE_B`	`1U`

---

ca_spi_br_t

Tipo: Enumerazione
File: caos/drivers/inc/spi.h:45:54

Defines the SPI baud rate prescaler relative to the peripheral clock, used in ca_spi_init() to configure communication speed.

Enumeratore	Valore
CA_SPI_BR_DIV2	0
CA_SPI_BR_DIV4	1
CA_SPI_BR_DIV8	2
CA_SPI_BR_DIV16	3
CA_SPI_BR_DIV32	4
CA_SPI_BR_DIV64	5
CA_SPI_BR_DIV128	6
CA_SPI_BR_DIV256	7

Note:
The actual frequency depends on the clock source of the specific SPI bus (e.g., APB1 or APB2). Ensure the resulting frequency does not exceed the maximum clock speed specified in the datasheet of the connected slave device. Choosing a value that is too high will result in data corruption or failed communication.

Uso:
This enum is passed as a member of the configuration structure to the ca_spi_init() function. Select the highest value that remains within the slave device's maximum clock frequency specification.

---

ca_stack_size_t

Tipo: Enumerazione
File: hw/f1xx/f103/objmap.h:51:58

Defines predefined memory allocation sizes for task stacks within the caOS RTOS, used during task creation APIs.

Enumeratore	Valore
STACK_SMALL	(STACK_MIN*2)
STACK_MEDIUM	(STACK_SMALL*2)
STACK_LARGE	(STACK_MEDIUM)
STACK_XLARGE	(STACK_MEDIUM)
STACK_KERNEL	(STACK_MEDIUM)

Note:
On the STM32F103, stack sizes are defined in bytes. Note that STACK_LARGE, STACK_KERNEL, and STACK_XLARGE are currently aliases for STACK_MEDIUM. Choosing a size too small will result in a stack overflow, which typically triggers a HardFault exception. Always monitor stack usage using the caOS debug tools.

Uso:
Used as an argument in task creation functions like 'caos_task_create()'. Choose based on the task's complexity: start with STACK_MEDIUM and reduce to STACK_SMALL if memory is tight, or increase if you encounter stack overflow errors.

`ca_sw_timer_mode_t`

Tipo: Enumerazione
File: `caos/inc/sw_timer.h:55:58`

Defines the execution behavior of a software timer within the caOS RTOS timer management API.

Enumeratore	Valore
`CA_TIMER_ONE_SHOT`	`0`
`CA_TIMER_PERIODIC`	`1`

Note:
The timer mode is set during the creation phase and cannot be changed while the timer is running. Choosing CA_TIMER_PERIODIC requires careful management of the callback execution time to ensure it does not exceed the timer period, which could lead to missed intervals or system instability.

Uso:
This enum is used as an argument in the ca_sw_timer_create() function. Choose CA_TIMER_ONE_SHOT for tasks like delayed power-off or debouncing, and CA_TIMER_PERIODIC for recurring tasks like status polling or heartbeat signals.

`ca_sw_timer_state_t`

Tipo: Enumerazione
File: `caos/inc/sw_timer.h:61:65`

Represents the current operational state of a software timer within the caOS RTOS timer management API.

Enumeratore	Valore
`CA_TIMER_STOPPED`	`0`
`CA_TIMER_RUNNING`	`1`
`CA_TIMER_EXPIRED`	`2`

Note:
The state is managed internally by the caOS kernel. Manually forcing these values via pointers is discouraged as it may lead to race conditions or inconsistent timer queue processing. CA_TIMER_STOPPED is the default state upon timer initialization.

Uso:
Used in ca_sw_timer_get_state() to query the status of a timer handle and internally by the scheduler to determine if a timer callback should be executed.

`ca_tick_speed_t`

Tipo: Enumerazione
File: `caos/inc/systick.h:35:46`

Defines the system tick interval in microseconds, used in the caOS initialization API to configure the SysTick timer frequency.

Enumeratore	Valore
CA_TICK_SPEED_100US	100
CA_TICK_SPEED_200US	200
CA_TICK_SPEED_250US	250
CA_TICK_SPEED_400US	400
CA_TICK_SPEED_500US	500
CA_TICK_SPEED_1MS	1000
CA_TICK_SPEED_2MS	2000
CA_TICK_SPEED_4MS	4000
CA_TICK_SPEED_5MS	5000
CA_TICK_SPEED_10MS	10000

Note:
The tick speed determines the resolution of all OS delays and timeouts. Choosing a value that is too small (e.g., 100us) on low-clock-speed MCUs can lead to 'tick starvation' where the CPU spends most of its time in the SysTick ISR. This value must be set once during system boot and cannot be changed at runtime.

Uso:
Passed as an argument to the 'caos_init()' or 'caos_systick_config()' functions. Choose the largest value that still satisfies the timing requirements of your most time-critical task.

`ca_timer_ic_src_t`

Tipo: Enumerazione
File: `caos/drivers/inc/timer.h:58:62`

Enumeratore	Valore
CA_TIMER_IC_PIN	0
CA_TIMER_IC_LSE	*(auto)*
CA_TIMER_IC_LSI	*(auto)*

`ca_timer_mode_t`

Tipo: Enumerazione
File: `caos/drivers/inc/timer.h:43:51`

Defines the operational mode of a hardware timer peripheral within the caOS timer driver API.

Enumeratore	Valore
CA_TIMER_MODE_PERIODIC	0
CA_TIMER_MODE_ONESHOT	*(auto)*
TIMER_MODE_COUNTER	*(auto)*
CA_TIMER_MODE_ENCODER	*(auto)*
TIMER_MODE_COUNTER_IRQ	*(auto)*
CA_TIMER_MODE_PWM	*(auto)*
CA_TIMER_MODE_CAPTURE	*(auto)*

Note:
CA_TIMER_MODE_PERIODIC is the default value. Ensure the timer peripheral supports the selected mode, as some basic timers (e.g., TIM6/TIM7 on STM32) do not support PWM or Encoder modes. Avoid changing modes while the timer is running; always stop the timer before reconfiguring.

Uso:

Used in the ca_timer_init() function. Choose the mode based on your hardware requirement: use PERIODIC for timing, PWM for power control, and ENCODER for motion feedback.

---

ca_uart_baud_rate_t

Tipo: Enumerazione
File: caos/drivers/inc/uart.h:30:45

Defines the supported communication speeds for UART peripherals in the caOS driver API.

Enumeratore	Valore
CA_UART_BAUD_9600	9600
CA_UART_BAUD_19200	19200
CA_UART_BAUD_38400	38400
CA_UART_BAUD_57600	57600
CA_UART_BAUD_115200	115200
CA_UART_BAUD_230400	230400
CA_UART_BAUD_460800	460800
CA_UART_BAUD_921600	921600
CA_UART_BAUD_1000000	1000000
CA_UART_BAUD_2000000	2000000
CA_UART_BAUD_3000000	3000000
CA_UART_BAUD_4000000	4000000
CA_UART_BAUD_4500000	4500000
CA_UART_BAUD_5000000	5000000

Note:
The actual baud rate error depends on the STM32 peripheral clock frequency. Always check the calculated baud rate error in the datasheet for your specific MCU. High baud rates (>1 Mbps) are highly sensitive to cable length and capacitance. DMA is strongly recommended for baud rates above 115200 to avoid CPU starvation.

Uso:
Used in the uart_init() function to configure the hardware baud rate generator. Select the value based on the requirements of the connected peripheral device.

---

ca_usb_ep_type_t

Tipo: Enumerazione
File: caos/drivers/inc/usb.h:37:42

Enumeratore	Valore
CA_USB_EP_TYPE_CTRL	0
CA_USB_EP_TYPE_ISOC	1
CA_USB_EP_TYPE_BULK	2
CA_USB_EP_TYPE_INTR	3

---

ca_usb_speed_t

Tipo: Enumerazione
File: caos/drivers/inc/usb.h:44:47

Defines the operational speed mode for the USB peripheral, used in the caOS USB initialization and configuration API.

Enumeratore	Valore
CA_USB_SPEED_FS	0
CA_USB_SPEED_HS	1

Note:
The default value is typically CA_USB_SPEED_FS. Not all STM32 MCUs support High-Speed; attempting to set CA_USB_SPEED_HS on a device without an integrated or external HS PHY will result in initialization failure. Ensure the system clock configuration (HSE/PLL) is sufficient to support the chosen speed, especially for HS.

Uso:
This enum is passed to the ca_usb_init() function. Choose CA_USB_SPEED_FS for general-purpose connectivity or CA_USB_SPEED_HS only if your hardware design explicitly includes a High-Speed PHY and you require higher throughput.

---

ca_usb_type_t

Tipo: Enumerazione
File: caos/usb/inc/usb_mgmt.h:43:51

Defines the USB device class configuration for the caOS USB management stack, used during initialization to determine the device's functional role.

Enumeratore	Valore
CA_USB_TYPE_NONE	0
CA_USB_TYPE_CDC	1
CA_USB_TYPE_MSC	2
CA_USB_TYPE_NCM	3
CA_USB_TYPE_DFU	4
CA_USB_TYPE_CUSTOM	5
CA_USB_TYPE__MAX	*(auto)*

Note:
The default value is CA_USB_TYPE_NONE. These values are mutually exclusive; only one USB class can be active per instance. Ensure the hardware descriptors match the selected type, as selecting an incorrect type will cause enumeration failure on the host PC.

Uso:
This enum is passed to the ca_usb_init() function. Choose the value based on the desired communication protocol required by your project specifications.

---

ca_wdt_mode_t

Tipo: Enumerazione
File: caos/drivers/inc/wdt.h:32:35

Defines the watchdog timer behavior upon timeout, used in the caOS WDT driver initialization API.

Enumeratore	Valore
CA_WDT_MODE_RESET	0
CA_WDT_MODE_INTERRUPT	*(auto)*

Note:
CA_WDT_MODE_RESET is the default and safest choice for production firmware. CA_WDT_MODE_INT should be used with caution as it does not guarantee system recovery if the CPU is stuck in a

hard fault or critical error state. Ensure the watchdog is refreshed (kicked) frequently enough to avoid unintended resets.

Uso:
This enum is passed to the 'ca_wdt_init()' function. Choose CA_WDT_MODE_RESET for standard watchdog functionality. Choose CA_WDT_MODE_INTERRUPT only during development phases when you need to inspect the system state via a breakpoint or log before the watchdog forces a reset.

`ca_wdt_reset_t`

Tipo: Enumerazione
File: `caos/drivers/inc/wdt.h:44:48`

Enumeratore	Valore
`CA_WDT_RESET_NONE`	`0`
`CA_WDT_RESET_IWDG`	*(auto)*
`CA_WDT_RESET_WWDG`	*(auto)*

`ca_wdt_type_t`

Tipo: Enumerazione
File: `caos/drivers/inc/wdt.h:23:26`

Defines the hardware watchdog timer peripheral type used in the caOS WDT driver API.

Enumeratore	Valore
`CA_WDT_TYPE_IWDG`	`0`
`CA_WDT_TYPE_WWDG`	*(auto)*

Note:
IWDG is independent of the system clock and is the safest choice for general system hang recovery. WWDG is more precise but will stop if the system clock fails or enters certain low-power modes. Do not attempt to refresh a WWDG too early, as it will trigger a reset if the counter is above the window threshold.

Uso:
Used in 'ca_wdt_init()' to configure the hardware peripheral. Choose IWDG for long-term system stability and WWDG for strict timing-sensitive task monitoring.

Chapter 117

Reference Definizioni caOS

Costanti e flag di configurazione in ordine alfabetico. Per ogni voce: valore per MCU, descrizione, note d'uso.

`AES_CH_HDR_SIZE`

File: `caos/crypto/inc/aes_channel_service.h`

Defines the total size of the header used in AES channel communication packets.

Valore: `(AES_CH_IV_SIZE + 2U)`

Note:
This constant represents the sum of the Initialization Vector (IV) size and an additional 2-byte overhead. It is used to calculate buffer offsets for encrypted data streams. It depends directly on AES_CH_IV_SIZE. Changing this value will break compatibility with existing encrypted communication protocols if not updated on both sender and receiver sides.

Uso:
Do not modify this value unless you are changing the underlying cryptographic protocol specification. If you encounter buffer overflow errors during decryption or encryption, verify that your allocated buffers account for this header size plus the payload size.

`AES_CH_IV_SIZE`

File: `caos/crypto/inc/aes_channel_service.h`

Defines the fixed size in bytes for the Initialization Vector (IV) used in AES channel encryption.

Valore: `12U`

Note:
This constant sets the buffer length for the IV required by the AES-GCM or AES-CTR modes. Changing this value affects the memory allocation of the AES channel control blocks. It must match the hardware-accelerated AES peripheral requirements of the STM32 series being used; typical values are 12 bytes for GCM or 16 bytes for CBC/CTR modes.

Uso:
Modify this only if the underlying cryptographic standard or hardware security module (HSM) requirements change. Setting an incorrect value will cause encryption/decryption failures or buffer overflows. Ensure all communicating nodes are configured with the same IV size to maintain protocol compatibility.

AES_CH_OVERHEAD

File: caos/crypto/inc/aes_channel_service.h

Defines the total byte overhead added to each encrypted data packet for metadata storage.

Valore: (AES_CH_IV_SIZE + 2U + AES_CH_TAG_SIZE)

Note:
This constant represents the sum of the Initialization Vector (IV), a 2-byte header, and the authentication tag size. Increasing this value consumes more RAM per channel buffer and reduces the effective payload capacity of each transmission. It must be consistent across all communicating nodes to prevent decryption failures.

Uso:
Adjust this value only if the underlying AES security parameters (like IV length or tag size) are modified in the system configuration. Incorrect values will lead to buffer overflows or data corruption during packet parsing. Ensure that the resulting total overhead does not exceed the maximum MTU of your communication interface.

AES_CH_TAG_SIZE

File: caos/crypto/inc/aes_channel_service.h

Defines the byte length of the authentication tag used for AES-GCM cryptographic operations.

Valore: AES_GCM_TAG_SIZE

Note:
This constant sets the size of the integrity check value appended to encrypted data. It must match the tag size expected by the receiver. Typical values are 16 bytes (128 bits) for maximum security, though 8 or 12 bytes are sometimes used for bandwidth-constrained applications. It depends on the underlying hardware crypto peripheral capabilities and the specific GCM implementation settings.

Uso:
Change this value only if your security protocol or communication partner requires a non-standard tag length. Using a smaller tag size reduces packet overhead but significantly increases the risk of forgery attacks. Ensure that the value is consistent across all communicating nodes, as a mismatch will cause authentication failures and dropped packets.

AES_GCM_TAG_SIZE

File: caos/drivers/inc/aes.h

Defines the length in bytes of the authentication tag generated or verified by the AES-GCM cryptographic algorithm.

Valore: 16

Note:
The value 16 corresponds to 128 bits, which is the standard and maximum tag size for AES-GCM. Reducing this value decreases security by increasing the probability of tag forgery. This constant must match the tag size expected by the remote peer or the stored encrypted data; otherwise, decryption will fail with an authentication error.

Uso:
Keep this set to 16 for all standard security applications. Only modify this value if you are interfacing with a legacy system or a specific protocol that explicitly requires truncated tags (e.g., 8 or 12 bytes). Changing this value requires a full recompile of all modules utilizing the AES driver.

`ARP_CACHE_SIZE`

File: `os_config.h`

Defines the maximum number of IP-to-MAC address mappings stored in the ARP cache table.

Valore: `8U`

Note:
Increasing this value allows the system to track more network devices simultaneously but consumes more RAM. Each entry typically occupies a fixed structure size. If the cache is full, the oldest entry is usually overwritten. Valid range is typically 4 to 32 depending on available heap memory.

Uso:
Increase this value if your device communicates with many different IP addresses on the local network to prevent frequent ARP requests. Decrease it to save RAM if your device only communicates with a single gateway or a very small number of static peers.

`ARP_ENTRY_TTL`

File: `os_config.h`

Defines the Time-To-Live (TTL) duration in milliseconds for entries stored in the ARP cache.

Valore: `30000U`

Note:
This constant determines how long an IP-to-MAC address mapping remains valid before being discarded or refreshed. The value is expressed in milliseconds; 30000U equals 30 seconds. Increasing this value reduces network traffic by minimizing ARP requests but may lead to stale entries if devices change their MAC addresses frequently. Decreasing it ensures higher accuracy but increases broadcast traffic on the local network.

Uso:
Adjust this value if your device operates in a dynamic network environment where IP/MAC mappings change often, or if you need to optimize bandwidth usage. Set a lower value (e.g., 5000U) for highly dynamic networks or a higher value (e.g., 60000U) for static, low-power environments. Ensure the value is not set too low to avoid excessive ARP request storms.

`ARP_HTYPE_ETH`

File: `caos/net/inc/net_proto.h`

Defines the hardware type identifier for Ethernet networks within the ARP protocol stack.

Valore: `1U`

Note:

This constant is set to 1U, which is the standard IANA-assigned value for Ethernet (10Mb). It is used in the hardware type field of ARP packets. Changing this value will cause the network stack to fail to communicate with standard Ethernet hardware, as the remote host will reject the ARP request as an unsupported protocol type.

Uso:
This value should generally not be changed for standard STM32 Ethernet implementations. It is used by the caOS network driver to identify the physical medium. Only modify this if you are implementing a custom non-Ethernet link layer that still requires ARP-like address resolution.

`ARP_OP_REPLY`

File: `caos/net/inc/net_proto.h`

Defines the ARP operation code for an ARP Reply packet.

Valore: `2U`

Note:
This constant represents the integer value 2, which identifies an incoming or outgoing packet as an ARP Reply in the Ethernet frame header. It is a fixed protocol value defined by RFC 826 and must not be modified. It is typically used in the network stack's packet parsing logic to distinguish replies from requests (ARP_OP_REQUEST, value 1).

Uso:
Use this constant when writing custom packet filtering or network diagnostic tools to identify ARP replies. Do not change this value, as it is strictly defined by the ARP protocol standard; changing it will cause the network stack to fail to recognize legitimate ARP responses, leading to a loss of connectivity.

`ARP_OP_REQUEST`

File: `caos/net/inc/net_proto.h`

Defines the operation code for an ARP Request packet in the network stack.

Valore: `1U`

Note:
This constant represents the ARP opcode 1, which signals a request for a hardware address. It is used by the network stack to construct outgoing ARP packets. It has no dependencies on other flags but must match the standard ARP protocol specification (RFC 826).

Uso:
This value should generally not be changed as it is a protocol-defined constant. It is used when the system needs to resolve an IP address to a MAC address. Modifying this value will cause the network stack to send invalid ARP packets, resulting in a failure to communicate with other devices on the local network.

`ARP_PTYPE_IP`

File: `caos/net/inc/net_proto.h`

Defines the Ethernet frame type identifier for the IPv4 protocol.

Valore: `0x0800U`

Note:
This constant represents the hexadecimal value 0x0800, which is the standard EtherType for IPv4 packets as defined by IEEE 802.3. It is used by the caOS network stack to filter incoming Ethernet frames and identify which packets should be passed to the IP layer. Changing this value will cause the device to ignore all standard IPv4 traffic.

Uso:
This value should generally remain unchanged as it follows the global networking standard. You would only modify this if you are implementing a custom, non-standard network protocol or working in a specialized environment that does not use standard IPv4 encapsulation. Ensure this matches the hardware-level frame filtering settings if your STM32 Ethernet peripheral is configured to perform hardware-based packet filtering.

`ARP_RETRY_MS`

File: `os_config.h`

Defines the timeout duration in milliseconds for the ARP protocol to wait before retransmitting an address resolution request.

Valore: `500U`

Note:
This value directly impacts the latency of network communication when a MAC address is unknown. Increasing this value reduces network traffic but slows down connection establishment; decreasing it speeds up recovery but may cause network congestion. It must be a positive integer, typically ranging from 100 to 2000 depending on the network stability.

Uso:
Adjust this value if your device experiences connection timeouts on busy or high-latency networks. If you notice the device failing to resolve IP addresses, try increasing this value. Ensure it is not set too low, as frequent retries can overwhelm the network stack and lead to packet loss.

`ARP_RETRY_N`

File: `os_config.h`

Defines the maximum number of times the ARP protocol will retransmit an address resolution request before giving up.

Valore: `3U`

Note:
When enabled, the system will attempt to resolve a MAC address for a given IP up to this count. If the target device does not respond after these attempts, the network stack will return an error. Typical values range from 1 to 5. Increasing this value improves reliability on lossy networks but increases latency during connection failures.

Uso:
Increase this value if you experience intermittent connection drops or if the network environment has high packet loss. Decrease it if you need the system to fail fast and report network errors quickly to the application layer. Setting this to 0 may prevent the system from ever resolving addresses.

ARRAY_SIZE

File: caos/inc/common.h

A macro that calculates the total number of elements in a static array.

Valore: (sizeof(arr) / sizeof((arr)[0]))

Note:
This macro divides the total memory size of the array by the size of a single element. It only works on arrays defined in the current scope; it will fail if passed a pointer to an array, as the pointer size will be used instead of the array size. It is a compile-time calculation with no runtime overhead.

Uso:
Use this whenever you need to iterate through an array or define buffer limits to avoid hardcoding magic numbers. Always ensure the input is a true array and not a pointer to prevent incorrect size calculations and potential buffer overflows.

CAOS_ENDIAN_BIG

File: caos/inc/endian.h

Defines the byte order (endianness) of the processor architecture, where 0 indicates Little-Endian.

Valore: 0

Note:
When set to 0, the system treats the least significant byte as the first byte in memory. STM32 microcontrollers (ARM Cortex-M) are natively Little-Endian, so this must remain 0 for standard operation. Setting this to 1 on an STM32 will cause data corruption during memory access and network communication.

Uso:
Keep this value at 0 for all STM32-based projects. Only change this if porting caOS to a Big-Endian architecture (e.g., certain PowerPC or older legacy processors). Incorrect configuration will result in swapped bytes in multi-byte variables like integers and pointers.

CAOS_ENDIAN_LITTLE

File: caos/inc/endian.h

Defines the byte order (endianness) of the processor architecture as Little-Endian.

Valore: 1

Note:
When set to 1, the system treats the least significant byte as the first byte in memory. This flag is typically required for ARM Cortex-M processors used in STM32 devices. Disabling this (setting to 0) will cause data corruption during memory access, network packet parsing, and peripheral register operations.

Uso:
This value should almost always remain set to 1 for STM32 development. Only change this if you are porting caOS to a specific Big-Endian architecture, which is rare in the STM32 ecosystem. Incorrect configuration will lead to unpredictable system behavior and invalid data processing.

CA_BOOT_BKP_MSG

File: caos/inc/boot_image.h

Defines the specific index or ID used for storing boot-related status messages in the backup register/SRAM area.

Valore: 17U

Note:
This constant acts as an offset or identifier for the ca_bkp_* API functions. It determines which memory slot in the non-volatile backup domain is reserved for boot diagnostic messages. It is typically used during the bootloader sequence to persist error codes or status flags across system resets. The value 17U is a fixed hardware-specific mapping.

Uso:
Do not change this value unless the memory map of the backup domain is redefined. It is used as an argument for ca_bkp_write or ca_bkp_read to ensure the bootloader and the application are accessing the same memory location for status reporting. Incorrect values will lead to reading garbage data or overwriting critical system configuration stored in other backup registers.

CA_BOOT_BKP_STATE

File: caos/inc/boot_image.h

Defines the specific backup register index used by the bootloader to store the system state.

Valore: 16U

Note:
This constant specifies the hardware backup register (BKP) index on the STM32. It is used by the bootloader to persist state information across system resets. Changing this value requires ensuring the index does not conflict with other system-level data stored in the backup domain. Valid range is typically 0 to 41, depending on the specific STM32 series.

Uso:
Change this value only if the current backup register index conflicts with other firmware components or hardware peripherals using the backup domain. Ensure that the new index is correctly mapped in the hardware documentation for your specific STM32 chip. Incorrect values may lead to boot loops or failure to recover the system state after a reset.

CA_BOOT_CONFIRMED

File: caos/inc/boot_image.h

A magic constant used to mark a firmware update image as successfully verified and ready for permanent execution.

Valore: 0xFFU

Note:
This value is typically written to a specific flash memory location or metadata header. When the bootloader checks this location, it compares the stored value against CA_BOOT_CONFIRMED (0xFFU). If they match, the system considers the update 'confirmed'. If the value is different (e.g., 0x00U), the system may trigger a rollback or enter a recovery mode. It is a fundamental part of the caOS A/B update mechanism.

Uso:
Use this constant when calling ca_boot_confirm() to finalize a firmware update process. You should only call this function after your application has performed its own self-tests (e.g., checking network connectivity or sensor data) to ensure the new firmware is stable. Warning: Never write this value to the confirmation slot before the application is fully verified, as doing so prevents the system from automatically rolling back to the previous known-good firmware in case of a crash.

`CA_BOOT_HDR_SIZE`

File: `caos/inc/boot_image.h`

Defines the fixed size in bytes reserved for the bootloader header at the start of a firmware image.

Valore: `0x400U`

Note:
This constant determines the offset where the actual application code begins within the flash memory slot. It must match the value used by the bootloader to parse the image metadata. Changing this value requires re-linking the application and updating the bootloader configuration. The value 0x400U (1024 bytes) is the standard alignment for caOS boot images.

Uso:
Change this value only if the bootloader header structure is modified to include more metadata (e.g., larger digital signatures or additional configuration fields). Warning: If this value does not match the bootloader's expectation, the system will fail to verify or boot the application, potentially resulting in a bricked state requiring a manual flash via JTAG/SWD.

`CA_BOOT_IWDG_MS`

File: `caos/inc/boot_image.h`

Defines the Independent Watchdog (IWDG) timeout duration in milliseconds during the boot sequence.

Valore: `8000U`

Note:
This constant sets the hardware watchdog period before the RTOS scheduler takes over. If the system does not reach the application initialization phase and reload the watchdog within this time, the STM32 will trigger a hardware reset. The value is in milliseconds (8000U = 8 seconds). It depends on the hardware IWDG clock configuration.

Uso:
Increase this value if the boot process (e.g., flash integrity checks, network initialization, or filesystem mounting) takes longer than 8 seconds to avoid spurious resets. Decrease it if you require a faster system recovery time in case of a hang during the early boot stage. Ensure the value does not exceed the maximum hardware timeout supported by the STM32 IWDG peripheral.

CA_BOOT_LOADER_LEN

File: caos/inc/boot_image.h

Defines the total memory size reserved for the bootloader image in the flash memory.

Valore: 0x00080000U

Note:
This constant sets the boundary for the bootloader section. It is typically used by the memory map and the bootloader update logic to ensure that application images do not overwrite the bootloader code. The value 0x00080000U represents 512 KB. Changing this requires a corresponding update to the linker script (.ld file) to ensure the memory regions are aligned with this constant.

Uso:
Change this value only if you are modifying the flash memory layout or if the bootloader code size exceeds the current allocation. Incorrect values will lead to memory corruption, boot failures, or the application overwriting critical bootloader code. Always verify the flash sector boundaries of your specific STM32 microcontroller before adjusting this value.

CA_BOOT_MAGIC

File: caos/inc/boot_image.h

A unique 32-bit identifier used to verify the integrity and validity of a firmware boot image.

Valore: 0xCAB70000U

Note:
This constant is typically placed at the beginning of a firmware binary or a specific header structure. The bootloader checks this magic value before attempting to execute the image. If the value does not match 0xCAB70000U, the bootloader will reject the image as corrupted or incompatible. It is a static constant and should not be modified at runtime.

Uso:
Use this constant when defining the header structure for your firmware images to ensure the bootloader only runs valid caOS-compatible code. If you change this value, you must update both the bootloader's verification logic and the build script that generates the firmware header, otherwise, the device will fail to boot after an update.

CA_BOOT_MAGIC_MASK

File: caos/inc/boot_image.h

A bitmask used to validate the integrity and authenticity of a firmware image header during the boot process.

Valore: 0xFFFF0000U

Note:
This constant defines the upper 16 bits of a 32-bit magic word. When the bootloader inspects a firmware image, it performs a bitwise AND operation with this mask to verify that the header contains the expected signature. If the result does not match the expected magic value, the bootloader will reject the image as invalid or corrupted. It is typically used in conjunction with a lower 16-bit version or status field.

Uso:
Change this value only if you are modifying the bootloader's image verification protocol or creating a new firmware format. Incorrect values will cause the system to fail to boot any firmware, potentially bricking the device until a valid image is flashed via a hardware debug interface (e.g., JTAG/SWD). Ensure this matches the value defined in your firmware build tools.

`CA_BOOT_MAX_TRIES`

File: `caos/inc/boot_image.h`

Defines the maximum number of boot attempts allowed for a firmware image before marking it as invalid.

Valore: `3U`

Note:
This constant controls the retry logic during the boot sequence. If the system fails to boot (e.g., due to a crash or watchdog reset) after this many attempts, the bootloader will typically revert to a previous stable image or enter a recovery state. It is a critical safety mechanism to prevent boot loops. Typical values range from 1 to 5.

Uso:
Increase this value if your application has a complex initialization sequence that might occasionally trigger a watchdog reset during legitimate startup. Decrease it if you want the system to fail over to a recovery image more aggressively. Do not set this to 0, as it would prevent any boot attempts.

`CA_BOOT_MSG`

File: `caos/inc/boot_image.h`

Constructs a unique boot message identifier for the caOS bootloader system.

Valore: `(CA_BOOT_MAGIC | ((u32)(u8)(cmd) « 8) | (u32)(u8)(slot))`

Note:
This macro combines a magic constant (CA_BOOT_MAGIC), a command code, and a target slot index into a single 32-bit value. It is used to communicate boot instructions between the application and the bootloader. The command is shifted by 8 bits, and the slot index occupies the lowest 8 bits. Ensure that 'cmd' and 'slot' are cast to u8 to prevent overflow into the magic constant bits.

Uso:
Use this macro when calling ca_boot_request_update or similar bootloader interface functions to specify which firmware slot to target and what action to perform. Do not modify the structure of this macro, as the bootloader expects this exact bit-packed format to validate the request.

`CA_BOOT_MSG_UPDATE`

File: `caos/inc/boot_image.h`

Defines the character identifier used in the bootloader to signal that a firmware update is available.

Valore: `'U'`

Note:
This constant is typically used as a command byte or status flag within the bootloader communication protocol. When the bootloader receives this specific character, it triggers the update sequence. It is a single-byte constant ('U'). Changing this value requires updating both the sender (e.g., a PC-side tool or another MCU) and the receiver (the caOS bootloader) to ensure they use the same handshake protocol.

Uso:
Change this value only if you need to modify the communication protocol between the bootloader and the host. Ensure that the host software is updated to send the new character, otherwise, the bootloader will fail to recognize the update request. Incorrect values will result in the system ignoring update commands, effectively locking the device in its current firmware version.

`CA_BOOT_SLOT_A_BASE`

File: `caos/inc/boot_image.h`

Defines the starting memory address for the primary application firmware slot (Slot A) in Flash.

Valore: `(0x08000000U + CA_BOOT_LOADER_LEN)`

Note:
This constant calculates the absolute memory address by adding the bootloader size (CA_BOOT_LOADER_L
to the base Flash address (0x08000000U). It is used by the bootloader and the application update system to locate where the executable code for Slot A begins. Changing this value requires a corresponding update to the linker script (.ld file) to ensure the application is compiled for the correct memory offset.

Uso:
Modify this value only when changing the size of the bootloader or repartitioning the Flash memory layout. Incorrect values will cause the system to jump to an invalid memory location, resulting in a hard fault or a failure to boot the application. Always ensure that the resulting address is aligned with the Flash sector boundaries of the specific STM32 microcontroller being used.

`CA_BOOT_SLOT_B_BASE`

File: `caos/inc/boot_image.h`

Defines the starting memory address for the secondary firmware image (Slot B) in the flash memory.

Valore: `0x08100000U`

Note:
This constant represents the absolute physical address in the STM32 internal flash where the secondary firmware image is stored. It is used by the bootloader to locate and verify the update image. It must be aligned with the flash sector boundaries defined in the hardware datasheet. Changing this value requires a corresponding update to the linker script (.ld file) to ensure the application is compiled for the correct memory region.

Uso:

Modify this value only when changing the flash memory layout or partition scheme. Incorrect values will cause the bootloader to fail to locate or boot the secondary image, potentially bricking the device if the primary image is also corrupted. Always ensure the address does not overlap with the primary image (Slot A) or the bootloader code itself.

`CA_BOOT_SLOT_LEN`

File: `caos/inc/boot_image.h`

Defines the fixed size in bytes allocated for each firmware update slot in the flash memory.

Valore: `0x00080000U`

Note:
This constant determines the maximum capacity of a single firmware image slot. It must be a multiple of the flash sector size to ensure proper erasure. If the application binary exceeds this size, the bootloader will reject the update. Typical values are powers of two (e.g., 0x80000 for 512KB).

Uso:
Change this value only when modifying the flash memory map or when the application binary size grows beyond the current limit. Ensure that the total flash size of the STM32 device can accommodate the number of slots defined in the system (e.g., Slot A + Slot B + Bootloader) without overlapping.

`CA_BOOT_STATE`

File: `caos/inc/boot_image.h`

A packed 32-bit value representing the boot configuration, including the target firmware slot and retry count.

Valore: `(CA_BOOT_MAGIC | ((u32)(u8)(slot) « 8) | ((u32)(tries) & 0xFFU))`

Note:
The constant is composed of a magic number (bits 31-16), the target firmware slot index (bits 15-8), and the remaining boot retry attempts (bits 7-0). It is typically stored in non-volatile memory (e.g., backup registers) to inform the bootloader which firmware image to execute after a reset. It depends on the CA_BOOT_MAGIC constant being defined in the system configuration.

Uso:
Use this when implementing custom bootloader logic or firmware update recovery mechanisms. You should update this value before triggering a system reset (using ca_os_reset) to ensure the bootloader selects the correct image. Warning: Providing an invalid slot index or an incorrect magic number will cause the bootloader to fail or fall back to a safe state, potentially bricking the device if no valid image is found.

`CA_BOOT_SUPPORTED`

File: `caos/inc/boot_image.h`

Enables or disables the bootloader support features within the caOS firmware.

Valore: `1`

Note:
When set to 1, the system includes the necessary code to handle firmware updates, image verification, and boot slot management. This flag is a prerequisite for using the ca_boot_* API functions. If disabled (0), the bootloader logic is excluded from the build, reducing the binary size but preventing remote or local firmware updates.

Uso:
Set this to 1 when your project requires field-upgradable firmware or dual-bank bootloader support. You must ensure that your memory map is configured to accommodate the bootloader and application slots. If you are developing a simple application that does not require updates, you may set this to 0 to save flash memory.

`CA_BOOT_TRIAL_MS`

File: `caos/inc/boot_image.h`

Defines the duration in milliseconds for the trial period of a new firmware image.

Valore: `60000U`

Note:
This constant determines how long the system waits for a confirmation after a new firmware update is applied. If ca_boot_confirm() is not called within this time, the system assumes the update is faulty and reverts to the previous stable image. The value is in milliseconds; 60000U equals 60 seconds. This is a system-wide configuration and should be consistent with the application's startup time requirements.

Uso:
Increase this value if your application performs complex initialization or network connectivity checks that might delay the call to ca_boot_confirm(). Decrease it if you want the system to revert faster in case of a boot loop or hang. Ensure the value is long enough to allow the application to reach the confirmation point under worst-case conditions.

`CA_CONFIG_HASH_LEN`

File: `caos/inc/caos_info.h`

Defines the fixed length in bytes for the internal task and resource hash identifiers used by the caOS kernel.

Valore: `32u`

Note:
This constant determines the memory footprint of the hash table entries. Increasing this value improves collision resistance for large task sets but consumes more RAM. It must be a power of two or a multiple of the system word size for optimal performance. Typical values range from 16 to 64 depending on the number of registered system objects.

Uso:
Adjust this value if the system reports hash collisions or if you are scaling the application to include a significantly higher number of tasks and semaphores. Do not change this value unless you are re-compiling the entire kernel, as it affects the binary structure of the task control blocks.

CA_DMA2D_AI

File: caos/drivers/inc/dma2d.h

Enables the DMA2D Abstraction Interface (AI) for hardware-accelerated graphics operations.

Valore: 0x01U

Note:
When set to 0x01U, the caOS DMA2D driver uses the abstraction layer to interface with the STM32 hardware. Disabling this (0x00U) will cause ca_dma2d_* functions to return errors or fail to initialize. This flag is a prerequisite for using hardware-accelerated pixel copying, blending, and filling operations.

Uso:
Enable this flag when your project requires high-performance GUI rendering or image processing using the STM32 DMA2D peripheral. If your hardware does not have a DMA2D controller or you are performing software-only rendering, set this to 0x00U to save code space and avoid peripheral initialization errors.

CA_DMA2D_CLUT_CPU

File: caos/drivers/inc/dma2d.h

Flag defining the CPU-based Color Look-Up Table (CLUT) processing mode for the DMA2D peripheral.

Valore: 0x04U

Note:
This constant is used as a bitmask or configuration parameter within the ca_dma2d_init or related configuration functions. When enabled, it instructs the DMA2D driver to handle CLUT color conversion via CPU-assisted routines rather than relying solely on hardware-accelerated DMA2D registers. It is typically used when the hardware DMA2D engine lacks specific support for certain pixel formats or when custom color mapping is required. It does not have direct dependencies on other flags but should be used in conjunction with valid CLUT memory buffers.

Uso:
Change this value only if you are implementing custom graphics drivers or if the standard hardware-accelerated DMA2D operations fail to render specific indexed color formats correctly. Warning: Enabling CPU-based processing will significantly increase CPU load during graphics operations, potentially impacting the real-time performance of other tasks. Use only when hardware-only rendering is insufficient for your specific display requirements.

CA_DMA2D_ERR_ARG

File: caos/drivers/inc/dma2d.h

Error flag indicating an invalid argument was passed to a DMA2D function.

Valore: 0x10U

Note:
This constant is used by the ca_dma2d_errors() function to report that a parameter provided to a DMA2D operation (such as an invalid buffer address, incorrect color format, or out-of-bounds

coordinate) is invalid. It is a bitmask value (0x10U) that can be combined with other error flags. It does not depend on other flags but is triggered by improper usage of the ca_dma2d_* API.

Uso:
Check for this flag when a DMA2D operation fails to return success. If this error occurs, verify that all pointers, dimensions, and color format constants passed to functions like ca_dma2d_copy or ca_dma2d_fill are within valid ranges and properly aligned as required by the STM32 hardware.

`CA_DMA2D_ERR_CAE`

File: `caos/drivers/inc/dma2d.h`

Configuration flag representing a Configuration Access Error (CAE) in the DMA2D peripheral.

Valore: `0x04U`

Note:
This constant is used as a bitmask or error code within the ca_dma2d_errors() function. When the DMA2D hardware detects an invalid configuration (such as incorrect pixel format or invalid memory address alignment), it sets this error flag. It is typically checked after a DMA2D operation fails to determine the root cause. It is a read-only status flag defined by the hardware driver layer.

Uso:
Use this constant when debugging DMA2D operations. If ca_dma2d_errors() returns a value where (result & CA_DMA2D_ERR_CAE) is true, verify your input parameters passed to ca_dma2d_copy() or ca_dma2d_blend(). Ensure that source and destination buffers are correctly aligned and that the pixel format matches the hardware capabilities.

`CA_DMA2D_ERR_CE`

File: `caos/drivers/inc/dma2d.h`

Configuration flag representing a Configuration Error (CE) in the DMA2D peripheral.

Valore: `0x02U`

Note:
This constant is a bitmask value (0x02U) used within the ca_dma2d_errors() function to identify if a configuration error occurred during a DMA2D transfer. It is typically checked alongside other error flags. When this bit is set in the error register, it indicates that the DMA2D hardware was provided with invalid parameters (e.g., unsupported color format or invalid memory address).

Uso:
Use this flag when debugging DMA2D transfer failures. If ca_dma2d_errors() returns a value where (result & CA_DMA2D_ERR_CE) is true, verify your input parameters such as pixel format, buffer alignment, or memory addresses passed to ca_dma2d_copy() or ca_dma2d_blend(). Do not modify this constant value as it is mapped directly to the hardware status register.

`CA_DMA2D_ERR_TE`

File: `caos/drivers/inc/dma2d.h`

Flag representing a Transfer Error (TE) interrupt or status bit for the DMA2D peripheral.

Valore: `0x01U`

Note:
This constant is used to identify or mask the Transfer Error interrupt flag in the DMA2D hardware. When enabled in the interrupt configuration, the system will trigger an error callback if the DMA2D hardware encounters a bus error or configuration issue during a memory transfer. It is typically used in conjunction with ca_dma2d_errors to check the status of the peripheral.

Uso:
Use this flag when configuring DMA2D interrupts or checking error status registers to detect hardware transfer failures. If your graphics operations are failing silently, check if this error bit is set. Do not modify this value as it corresponds to the specific hardware register bit defined in the STM32 reference manual.

`CA_DMA2D_ERR_TMO`

File: `caos/drivers/inc/dma2d.h`

Defines the timeout value for DMA2D error handling operations.

Valore: `0x08U`

Note:
This constant sets the maximum number of cycles or ticks the system waits for the DMA2D peripheral to recover from an error state. If the DMA2D hardware fails to clear its error flags within this threshold, the driver will report a failure. It is typically used in conjunction with ca_dma2d_errors. Increasing this value may be necessary if the system bus is heavily loaded, while decreasing it makes the system more responsive to hardware hangs.

Uso:
Adjust this value if you encounter frequent 'DMA2D timeout' errors during heavy graphical operations or when the system bus is congested. If the value is too low, the system may falsely report hardware errors; if too high, the system may hang for an extended period before detecting a genuine hardware failure.

`CA_DMA2D_HW_TC`

File: `caos/drivers/inc/dma2d.h`

Defines the bitmask for the DMA2D Transfer Complete (TC) interrupt flag.

Valore: `0x100U`

Note:
This constant represents the hardware-specific bit position for the Transfer Complete interrupt in the DMA2D Interrupt Status Register. It is used internally by the ca_dma2d_* driver functions to check or clear the interrupt status. It should not be modified unless the underlying STM32 hardware register mapping changes.

Uso:

This value is used by the caOS DMA2D driver to identify when a hardware graphics operation has finished. You do not need to change this value for standard application development. If you are porting caOS to a new STM32 chip, verify this value against the specific Reference Manual for the DMA2D peripheral.

`CA_DMA2D_OUT_FMTS`

File: `caos/drivers/inc/dma2d.h`

Defines the default output pixel format for the DMA2D (Chrom-ART) peripheral.

Valore: `5U`

Note:
This constant sets the hardware register configuration for the DMA2D output color mode. A value of 5U typically corresponds to ARGB8888 format in STM32 hardware. Changing this affects how the DMA2D engine interprets and writes pixel data to the frame buffer. It must be compatible with the display driver and memory layout used in your project.

Uso:
Change this value only if you are switching to a different color depth or pixel format (e.g., RGB565 or RGB888) for your display output. Ensure the hardware supports the new format before modifying. Incorrect values will result in corrupted graphics, color inversion, or display artifacts.

`CA_DMA2D_RBS`

File: `caos/drivers/inc/dma2d.h`

Configures the DMA2D Request Burst Size (RBS) for memory-to-memory transfers.

Valore: `0x02U`

Note:
This constant defines the number of beats in a burst transfer for the DMA2D controller. A value of 0x02U typically corresponds to a 4-beat burst. Increasing this value can improve throughput for large memory copies by reducing bus arbitration overhead, but it may increase bus latency for other peripherals. It is hardware-dependent and must align with the STM32 DMA2D peripheral specifications.

Uso:
Modify this value only if you encounter performance bottlenecks during heavy graphical operations or memory copies. If the system experiences bus contention or graphical artifacts, verify that the burst size is supported by the specific STM32 chip's DMA2D implementation. Incorrect values may lead to DMA2D transfer errors or system instability.

`CA_ETH_ERR_BUS`

File: `caos/drivers/inc/eth.h`

A bitmask flag indicating that the Ethernet peripheral has encountered a bus-level communication error.

Valore: `(1U ≪ 0)`

Note:

This flag is set in the Ethernet status register when the DMA fails to access the system bus (e.g., AHB bus error). It is typically used within the Ethernet interrupt service routine (ISR) to trigger a driver reset or error recovery sequence. It is a bit-field constant, so it must be used with bitwise operators (e.g., status & CA_ETH_ERR_BUS).

Uso:
Use this flag when implementing custom error handling logic in the Ethernet driver. You should check this flag if the network interface stops transmitting or receiving packets unexpectedly. Do not modify the value itself, as it corresponds to a specific hardware register bit defined by the STM32 Ethernet peripheral architecture.

`CA_ETH_ERR_RBU`

File: `caos/drivers/inc/eth.h`

Error flag indicating that the Ethernet Receive Buffer is unavailable (RBU).

Valore: `(1U « 1)`

Note:
When this bit is set in the status register, it means the DMA has run out of descriptors to store incoming packets. This flag is typically checked within the Ethernet ISR or the driver's polling loop. It depends on the CA_ETH_ENABLE_INTERRUPTS flag being active to trigger an error callback.

Uso:
Use this flag to diagnose packet loss during high network traffic. If this error occurs frequently, increase the number of RX descriptors in the Ethernet configuration or optimize the task processing the incoming data to prevent buffer exhaustion.

`CA_ETH_ERR_RPS`

File: `caos/drivers/inc/eth.h`

A bitmask flag representing a Receive Packet Stall (RPS) error condition in the Ethernet driver.

Valore: `(1U « 3)`

Note:
When enabled, this flag indicates that the Ethernet DMA has stopped receiving packets due to a buffer exhaustion or descriptor ring stall. It is part of the CA_ETH_ERR_MASK and is typically checked within the Ethernet ISR. It depends on the hardware DMA status registers being correctly mapped to the caOS driver layer.

Uso:
Use this flag when debugging network throughput drops or intermittent connection losses. If this bit is frequently set in the error status register, it indicates that the application is not processing incoming packets fast enough, requiring an increase in the RX descriptor count or a higher priority for the Ethernet task.

`CA_ETH_ERR_TPS`

File: `caos/drivers/inc/eth.h`

A bitmask flag representing a Transmit Packet Stall (TPS) error condition in the Ethernet driver.

Valore: `(1U « 2)`

Note:
When this bit is set in the Ethernet error status register, it indicates that the DMA engine has stalled during packet transmission. This flag is part of the CA_ETH_ERR_MASK and is typically checked within the Ethernet ISR. It depends on the hardware DMA state machine being active and the Ethernet peripheral being initialized.

Uso:
Use this flag when debugging network transmission timeouts or 'stuck' Ethernet interfaces. If this error occurs frequently, it usually indicates a buffer descriptor chain issue or a hardware-level DMA hang. Do not modify the bit position unless updating the entire driver's error handling logic to match the hardware register map.

`CA_ETH_SEND_ERR_LINK`

File: `caos/drivers/inc/eth.h`

Error code indicating that an Ethernet packet transmission failed because the network link is currently down.

Valore: `(-102)`

Note:
This constant is returned by the Ethernet driver's send function when the physical link status (PHY) is disconnected. It is a negative integer value (-102) used for error handling in the application layer. It depends on the PHY status register being correctly polled or updated via interrupt.

Uso:
Use this value in your error-checking logic after calling the Ethernet send function. If you receive this code, you should stop attempting to send data and wait for the link-up event before retrying. Do not modify this value as it is a reserved system error code.

`CA_ETH_SEND_ERR_TX`

File: `caos/drivers/inc/eth.h`

Error code returned by the Ethernet driver when a transmission buffer fails to send.

Valore: `(-101)`

Note:
This constant represents a specific negative integer error state (-101) indicating a hardware or DMA failure during the packet transmission process. It is typically checked by the network stack after calling the low-level transmit function. It does not have configurable ranges as it is a fixed return value, but it depends on the underlying HAL_ETH_Transmit status.

Uso:

Use this constant in your application code to handle Ethernet transmission failures gracefully. If you encounter this error frequently, check the Ethernet cable connection, verify that the DMA descriptors are correctly initialized, and ensure that the network buffer pool is not exhausted. Do not modify this value as it is defined by the caOS driver architecture.

`CA_ETH_SEND_OK`

File: `caos/drivers/inc/eth.h`

Defines the return status code indicating a successful Ethernet packet transmission.

Valore: `0`

Note:
This constant is used by the Ethernet driver's transmit function to signal that the buffer was successfully handed off to the DMA controller. It is typically compared against the return value of the 'eth_send' function. It must remain 0 to maintain compatibility with standard C error handling conventions where 0 represents success.

Uso:
Do not change this value unless you are porting the driver to a new hardware abstraction layer that uses a different convention for success codes. Changing this value will break all existing driver logic that checks for successful packet transmission, likely causing the network stack to report false errors.

`CA_EVENT_BIT`

File: `caos/inc/event.h`

Defines a specific bit position within an event group to represent a unique system event.

Valore: `(1U « (n))`

Note:
This macro creates a bitmask where 'n' represents the bit index (0-31). Enabling a specific bit allows a task to wait for or signal a unique event flag. It depends on the event group size defined in the RTOS kernel. Valid values for 'n' are typically 0 to 31 for a 32-bit event register.

Uso:
Use this when you need to signal or synchronize tasks based on specific conditions (e.g., data ready, error occurred). Change 'n' to assign a unique identifier to each event. Warning: Ensure 'n' does not exceed the maximum bit width of the event group, or you will trigger undefined behavior or silent failures in event signaling.

`CA_EVENT_WAIT_ALL`

File: `caos/inc/event.h`

A flag used in event group operations to require all specified bits to be set before a task is unblocked.

Valore: `1`

Note:
When enabled (set to 1), the task will wait for the logical AND of all requested event bits. If disabled (set to 0), the task will unblock if any one of the requested bits is set (logical OR). This

flag is typically passed as an argument to the ca_event_wait() function. It has no dependencies on other system flags but directly dictates the synchronization logic of the event group.

Uso:
Use this flag when your task needs to synchronize with multiple independent conditions simultaneously, such as waiting for both a sensor data ready signal and a timer interrupt. Do not use this if you only need one of several events to trigger the task, as it will cause the task to hang indefinitely if one of the required events never occurs.

`CA_EVENT_WAIT_ANY`

File: `caos/inc/event.h`

A flag used in event synchronization to trigger a task wake-up if any one of the specified events occurs.

Valore: `0`

Note:
When enabled (set to 0), the task will unblock as soon as at least one bit in the event mask is set. This is the logical OR operation for event waiting. It is the default behavior for event synchronization in caOS. It does not require additional memory but changes the scheduler's condition check from a bitwise AND to a bitwise OR.

Uso:
Use this flag when a task needs to respond to multiple independent triggers, such as 'Data Ready' OR 'Error Occurred'. Do not use this if the task requires all specified events to be present before proceeding; for that, use the CA_EVENT_WAIT_ALL flag instead. Setting this incorrectly may cause tasks to wake up prematurely, leading to race conditions if the logic expects a complete set of data.

`CA_FAULT_ASSERT`

File: `caos/inc/scheduler.h`

A global configuration flag that determines whether the system triggers a hard fault or enters an infinite loop when a kernel assertion fails.

Valore: `0x00`

Note:
When set to 1, the system executes a software breakpoint or triggers a hard fault upon an assertion failure, which is useful for debugging. When set to 0, the system enters a silent infinite loop to prevent undefined behavior in production. This flag depends on the CA_DEBUG_MODE macro; if CA_DEBUG_MODE is disabled, this flag is ignored.

Uso:
Set this to 1 during the development and testing phase to catch logic errors immediately via the debugger. Set this to 0 for production firmware to ensure the system remains in a controlled state if an unexpected condition occurs, preventing erratic behavior in the field.

CA_FAULT_CHECK

File: `caos/inc/scheduler.h`

Configures the level of hardware fault detection and stack overflow monitoring performed by the caOS kernel.

Valore: `0x03`

Note:
The value 0x03 acts as a bitmask enabling both stack boundary checking and CPU exception trapping. Disabling this (setting to 0x00) reduces CPU overhead but makes debugging memory corruption or illegal instruction errors significantly harder. It depends on the hardware MPU (Memory Protection Unit) being correctly initialized in the system startup code.

Uso:
Enable this during development and integration testing to catch buffer overflows and invalid pointer dereferences early. You should only consider disabling it in final production builds if you have strictly verified memory safety and require absolute minimum interrupt latency, though it is generally recommended to keep it enabled for system stability.

CA_FAULT_EXEC_EMERGENCY

File: `caos/inc/scheduler.h`

Defines the emergency execution mode flag used by the scheduler during critical system fault recovery.

Valore: `0x02`

Note:
When this flag is set, the scheduler bypasses standard task switching logic to prioritize the execution of emergency recovery routines. It depends on the CA_FAULT_HANDLER_ENABLED flag being active. The value 0x02 is a bitmask; ensure it does not conflict with other scheduler state flags in the bitfield.

Uso:
Change this value only if you are modifying the core scheduler architecture or porting to a new STM32 series with different fault handling requirements. Setting this incorrectly can lead to a system hang or an infinite loop during a fault condition, preventing the watchdog from resetting the device.

CA_FAULT_RESTART_THREAD

File: `caos/inc/scheduler.h`

Determines if the scheduler should attempt to restart a thread after a fault occurs.

Valore: `0x01`

Note:
When enabled (0x01), the kernel will attempt to re-initialize the thread's stack and context upon a fault exception. If disabled, the thread remains in a suspended or terminated state. This flag depends on the system having enough memory to re-allocate the thread control block and stack. Typical values are 0x01 (enabled) or 0x00 (disabled).

Uso:

Enable this flag during development to keep the system running after non-critical thread crashes. Disable it in production if you prefer the system to enter a safe state or trigger a full watchdog reset upon encountering a fault, as restarting a corrupted thread may lead to unpredictable system behavior.

CA_FMAC_FIR_MAX

File: caos/drivers/inc/fmac.h

Defines the maximum number of taps (coefficients) supported by the FMAC FIR filter engine.

Valore: 127U

Note:
This constant sets the upper limit for the FIR filter length in the FMAC driver. Increasing this value allows for more complex filtering but consumes more memory for coefficient storage. The value 127 is the hardware-imposed limit for the STM32 FMAC peripheral. Ensure your filter design does not exceed this count, or the driver will fail to initialize the filter parameters.

Uso:
Change this value only if you need to reduce memory footprint and are certain your application will never require a filter with more than the new, lower limit. Do not increase it beyond 127, as it will exceed the hardware capability of the FMAC peripheral. If your filter design requires more than 127 taps, you must implement a multi-stage filtering approach or use a different processing method.

CA_FMAC_IIR_MAX

File: caos/drivers/inc/fmac.h

Defines the maximum number of coefficients supported by the FMAC (Filter Math Accelerator) IIR filter implementation.

Valore: 64U

Note:
This constant sets the upper limit for the IIR filter order or coefficient array size. Increasing this value consumes more static memory (RAM) for the filter state buffers. It must be large enough to accommodate the highest order filter used in your application. Typical values are powers of 2, such as 64, 128, or 256, depending on the available heap and the complexity of the signal processing tasks.

Uso:
Change this value if you encounter errors when initializing complex IIR filters or if you need to optimize memory usage by reducing it to the minimum required for your specific filter design. If you attempt to initialize a filter with more coefficients than this constant allows, the ca_fmac_init function will likely fail or return an error code.

CA_FMAC_OVFL

File: caos/drivers/inc/fmac.h

Flag representing the FMAC (Filter Math Accelerator) overflow error condition.

Valore: 0x02U

Note:
This constant is used as a bitmask within the ca_fmac_flags() function to check if the FMAC hardware has encountered an arithmetic overflow during calculation. When this bit is set in the status register, it indicates that the result of the filter operation exceeded the representable range of the output format. It is typically used in conjunction with other status flags to validate the integrity of signal processing data.

Uso:
Use this flag when polling or checking the FMAC status after a computation to ensure the output data is valid. If this flag is detected, the current filter output should be considered unreliable. You should check this value if your application performs high-gain filtering or processes signals with high dynamic range that might cause saturation.

`CA_FMAC_SAT`

File: `caos/drivers/inc/fmac.h`

Enables hardware saturation logic for the FMAC (Filter Math Accelerator) peripheral.

Valore: `0x01U`

Note:
When set to 0x01U, the FMAC hardware automatically clamps output results to the maximum or minimum representable values if an overflow occurs during calculation. If disabled (0x00U), the result will wrap around, which can lead to significant signal distortion in DSP applications. This flag is typically used in conjunction with ca_fmac_init to configure the peripheral's operational mode.

Uso:
Enable this flag when performing filtering operations (like FIR or IIR) where numerical stability is critical and you want to prevent wrap-around artifacts. Disable it only if your specific algorithm requires modular arithmetic or if you are manually handling overflow protection in software to save clock cycles.

`CA_FMAC_UNFL`

File: `caos/drivers/inc/fmac.h`

Flag to enable underflow detection in the FMAC (Filter Math Accelerator) peripheral.

Valore: `0x04U`

Note:
When enabled (0x04U), the FMAC hardware will trigger an interrupt or error state if a calculation result underflows the representable range. This is typically used in conjunction with other FMAC flags to configure the peripheral's behavior during signal processing. It depends on the hardware support for FMAC error reporting on the specific STM32 series being used.

Uso:
Use this flag when performing high-precision digital filtering where you need to detect if signal values become too small to be represented accurately by the fixed-point format. If you notice unexpected signal clipping or loss of precision in your filter output, enable this flag to debug potential underflow conditions. Warning: enabling this may cause the FMAC to stop processing or trigger an error handler if an underflow occurs, so ensure your application has a proper error recovery strategy using ca_fmac_init or similar.

CA_ICMP_PAYLOAD

File: caos/net/inc/net_proto.h

A pointer offset macro used to locate the start of the data payload within an ICMP echo packet.

Valore: ((u8 *)(icmp) + sizeof(ca_icmp_echo_t))

Note:
This constant calculates the memory address immediately following the ICMP header structure (ca_icmp_echo_t). It is used by the network stack to read or write the payload section of a ping request or reply. It depends on the structure alignment of ca_icmp_echo_t; if the header size changes, this macro must remain synchronized to avoid memory corruption or data misalignment.

Uso:
Use this macro when you need to access the raw data buffer of an ICMP packet for inspection or modification. Do not manually calculate this offset using hardcoded integers, as it will break if the header structure is modified. Ensure that the packet buffer is large enough to hold the payload before accessing this address to prevent buffer overflows.

CA_IP4_IHL

File: caos/net/inc/net_proto.h

Calculates the length of the IPv4 header in bytes based on the IHL field.

Valore: (((ip4)->ver_ihl & 0x0Fu) « 2)

Note:
The IHL (Internet Header Length) field represents the number of 32-bit words in the header. Multiplying this value by 4 (via the left shift by 2) converts the count into the total byte size. This value is dynamic and depends on the presence of IP options. It must be used only when the IP header is valid.

Uso:
Use this macro when parsing incoming network packets to determine where the payload begins relative to the start of the IP header. Do not modify the macro definition itself; incorrect usage will lead to buffer overflows or memory corruption if the calculated offset points outside the packet buffer.

CA_IP4_PAYLOAD

File: caos/net/inc/net_proto.h

A macro that calculates the memory address where the actual data payload begins within an IPv4 packet.

Valore: ((u8 *)(ip4) + CA_IP4_IHL(ip4))

Note:
This macro adds the length of the IPv4 header (IHL) to the base pointer of the IP packet. It is essential for accessing the transport layer data (like TCP or UDP segments). It depends on the CA_IP4_IHL macro, which parses the header length field. The result is a pointer to the first byte of the payload.

Uso:
Use this whenever you need to extract or process the data carried by an IP packet. Do not modify this macro unless you are porting the stack to a non-standard network architecture. Incorrect usage will lead to memory access violations or data corruption if the pointer calculation points outside the valid packet buffer.

`CA_JPEG_ERR_ARG`

File: `caos/drivers/inc/jpeg.h`

Error code indicating an invalid argument was passed to a JPEG driver function.

Valore: `0x01U`

Note:
This constant is returned by functions in the ca_jpeg module when input parameters (such as pointers, buffer sizes, or configuration structures) fail validation checks. It is a specific error flag used within the ca_jpeg_errors enumeration. It does not depend on other flags but is part of the standard error handling flow for the JPEG hardware peripheral.

Uso:
Check for this value when calling ca_jpeg_decode or ca_jpeg_encode. If a function returns this error, verify that your input buffers are correctly aligned, pointers are not null, and the image dimensions provided to ca_jpeg_geometry are within hardware limits. Example: if (ca_jpeg_decode(data, size) == CA_JPEG_ERR_ARG) { /* handle invalid input */ }.

`CA_JPEG_ERR_HDR`

File: `caos/drivers/inc/jpeg.h`

Error flag indicating a corrupted or invalid JPEG header detected during decoding.

Valore: `0x08U`

Note:
This constant is used as a bitmask within the ca_jpeg_errors() function. When the JPEG decoder encounters a malformed header (e.g., missing SOI marker or invalid quantization tables), it sets this bit in the error status register. It is a read-only status flag and does not require configuration; it is strictly for diagnostic purposes when ca_jpeg_decode() returns a non-zero value.

Uso:
Check this flag when ca_jpeg_decode() fails to determine if the issue is due to a bad input file rather than a hardware or memory issue. If this flag is set, verify the integrity of the JPEG source data buffer before passing it to the decoder again. Do not attempt to modify this value.

`CA_JPEG_ERR_OVF`

File: `caos/drivers/inc/jpeg.h`

JPEG hardware decoder overflow error flag.

Valore: `0x04U`

Note:

This constant represents a bitmask (0x04U) indicating that the JPEG hardware peripheral has encountered an overflow condition during data processing. It is typically used within the ca_jpeg_errors() function to check the status of the hardware decoder. When this flag is set, it means the input buffer or the internal FIFO has been overwhelmed, usually due to data arriving faster than the decoder can process it or an incorrectly formatted JPEG stream.

Uso:
Check this flag when ca_jpeg_errors() returns a non-zero value to determine if the failure was caused by an overflow. If this error occurs, verify the input data stream integrity and ensure the DMA or interrupt service routine feeding the JPEG peripheral is not being starved by higher-priority tasks. Do not attempt to continue decoding if this flag is raised; reset the JPEG peripheral using ca_jpeg_init() before attempting a new operation.

`CA_JPEG_ERR_TMO`

File: `caos/drivers/inc/jpeg.h`

Defines the timeout error flag for JPEG hardware operations.

Valore: `0x02U`

Note:
This constant represents a specific bitmask or error code returned by the JPEG driver when a hardware operation exceeds the expected time limit. It is typically used in conjunction with ca_jpeg_errors to identify if a failure was caused by a timeout rather than a data corruption or configuration issue. The value 0x02U is a bit-field constant.

Uso:
Use this value when checking the return status of ca_jpeg_decode or ca_jpeg_encode. If the driver returns an error, perform a bitwise AND with CA_JPEG_ERR_TMO to determine if the hardware hung. If this error occurs frequently, verify the clock configuration of the JPEG peripheral or ensure the input data stream is not stalling the DMA transfer.

`CA_JPEG_HW_EOC`

File: `caos/drivers/inc/jpeg.h`

Flag representing the End of Conversion (EOC) interrupt bit for the hardware JPEG peripheral.

Valore: `0x400U`

Note:
This constant is a bitmask (0x400U) used to identify or clear the hardware interrupt status register for the JPEG codec. It is typically used within the ca_jpeg_* driver functions to verify if a hardware-accelerated encoding or decoding operation has finished. It depends on the underlying STM32 JPEG peripheral register map.

Uso:
Do not modify this value unless you are porting the driver to a different STM32 series with a different register layout. It is used internally by the ca_jpeg driver to handle hardware synchronization. If you are implementing a custom interrupt handler or polling mechanism for JPEG operations, use this flag to check the status register via the hardware abstraction layer.

`CA_JPEG_HW_HDR`

File: `caos/drivers/inc/jpeg.h`

Enables hardware-accelerated High Dynamic Range (HDR) processing for JPEG operations.

Valore: `0x800U`

Note:
When enabled (0x800U), the ca_jpeg driver utilizes dedicated STM32 hardware peripherals to perform HDR-related color space transformations or tone mapping during JPEG encoding/decoding. This flag is typically used in conjunction with other JPEG configuration bits. If the specific STM32 hardware variant does not support hardware HDR, enabling this flag may result in undefined behavior or driver initialization errors.

Uso:
Enable this flag only when the application requires high-performance HDR image processing and the underlying hardware supports it. If you encounter image artifacts or driver crashes during JPEG operations, verify if your hardware supports this feature before enabling. For standard JPEG tasks without HDR requirements, keep this flag disabled to save system resources and ensure compatibility.

`CA_JPEG_HW_IN_FREE`

File: `caos/drivers/inc/jpeg.h`

Flag indicating that the JPEG hardware peripheral is available and initialized in the system.

Valore: `0x100U`

Note:
When enabled (0x100U), the ca_jpeg_* API functions will attempt to utilize the STM32 hardware JPEG codec. If disabled or not defined, the system may fall back to software-based processing or return an error if hardware acceleration is required. This constant is typically used in the board support package (BSP) configuration to define hardware capabilities.

Uso:
Change this value only if you are porting the code to a different STM32 variant that lacks a hardware JPEG peripheral. If you are working on a board that does not have a JPEG hardware block, ensure this flag is not set to avoid attempting to access non-existent memory-mapped registers, which would trigger a hard fault.

`CA_JPEG_HW_OUT_DATA`

File: `caos/drivers/inc/jpeg.h`

Defines the hardware output data register offset for the JPEG peripheral.

Valore: `0x200U`

Note:
This constant represents the memory-mapped offset used by the JPEG hardware accelerator to output processed pixel data. It is typically used internally by the ca_jpeg_* API functions. Changing this value will break communication with the STM32 JPEG hardware block, leading to decoding or encoding failures. It is a fixed hardware-dependent constant and should not be modified by the application layer.

Uso:
Do not change this value. It is used by the driver to correctly interface with the STM32 JPEG peripheral registers. If you encounter JPEG processing errors, verify that the hardware initialization via ca_jpeg_init was successful rather than modifying this constant.

`CA_L2_PAYLOAD`

File: `caos/net/inc/net_proto.h`

Calculates the memory address where the actual data payload begins within a Layer 2 network frame.

Valore: `((u8 *)(l2) + sizeof(ca_l2_frame_t))`

Note:
This macro adds the size of the L2 header structure to the base address of the frame buffer. It is a pointer arithmetic operation. It depends on the definition of 'ca_l2_frame_t' and must be used only after the frame header has been correctly initialized. The resulting address must be aligned according to the CPU architecture requirements to avoid bus faults.

Uso:
Use this constant whenever you need to access or copy the data portion of a received packet. Change this only if you modify the structure of the L2 header. Warning: Incorrect usage will lead to memory corruption or reading garbage data, as it will point to the wrong offset in the packet buffer.

`CA_L2_VLAN_PAYLOAD`

File: `caos/net/inc/net_proto.h`

Calculates the memory offset to the start of the payload data within an 802.1Q VLAN-tagged Ethernet frame.

Valore: `((u8 *)(l2v) + sizeof(ca_l2_vlan_frame_t))`

Note:
This macro adds the size of the VLAN header structure to the base pointer of the Layer 2 frame. It is strictly dependent on the ca_l2_vlan_frame_t structure definition. If the VLAN header size changes or padding is added by the compiler, this macro must be updated to prevent memory misalignment or buffer overflows.

Uso:
Use this when parsing incoming network packets to correctly locate the encapsulated IP or ARP data. Do not modify this value manually unless you are changing the underlying network frame structure; incorrect offsets will lead to corrupted packet processing or hard faults when accessing the payload buffer.

`CA_LED_COUNT`

File: `hw/f1xx/f103/objmap.h`
File: `hw/f4xx/f407/objmap.h`
File: `hw/f4xx/f429/objmap.h`
File: `hw/f4xx/f446/objmap.h`
File: `hw/f7xx/f746/objmap.h`

File: `hw/h5xx/h563/objmap.h`
File: `hw/h7xx/h747/objmap.h`
File: `hw/h7xx/h753/objmap.h`

Defines the total number of physical status LEDs available on the specific STM32 hardware target.

Valori per MCU:

MCU	Valore
F103	1U
F407	1U
F429	3U
F446	3U
F746	3U
H563	3U
H747	3U
H753	3U

Note:
This constant determines the size of the internal LED control array. Enabling this allows the caOS HAL to manage multiple status indicators (e.g., system heartbeat, error, and activity). It must match the hardware schematic of your specific board. Values range from 1 to 3 depending on the MCU series; setting a value higher than the physical hardware will result in out-of-bounds memory access during LED initialization.

Uso:
Change this value only when porting caOS to a new custom PCB or a different STM32 development board. If you are using a standard supported board, keep the default value provided in the board configuration file. Incorrect values will cause the system to hang during the boot sequence or trigger a HardFault when attempting to toggle non-existent GPIO pins.

`CA_LED_TABLE`

File: `hw/f1xx/f103/objmap.h`
File: `hw/f4xx/f407/objmap.h`
File: `hw/f4xx/f429/objmap.h`
File: `hw/f4xx/f446/objmap.h`
File: `hw/f7xx/f746/objmap.h`
File: `hw/h5xx/h563/objmap.h`
File: `hw/h7xx/h747/objmap.h`
File: `hw/h7xx/h753/objmap.h`

Defines the hardware mapping table for system status LEDs across different STM32 MCU architectures.

Valori per MCU:

MCU	Valore
F103	\
F407	\
F429	\
F446	\
F746	\
H563	\
H747	\
H753	\

Note:
This constant acts as a lookup table or pointer to the GPIO port and pin definitions for the board's diagnostic LEDs. Enabling or modifying this allows the RTOS to signal system states (like heartbeat or error codes) via physical LEDs. It is hardware-dependent; selecting the wrong table for your specific MCU (e.g., F407 vs H747) will result in incorrect GPIO toggling or potential hardware faults if pins are misconfigured.

Uso:
Change this value only when porting the caOS RTOS to a new hardware target or when changing the physical LED wiring on a custom PCB. Ensure the selected table matches the specific MCU variant defined in your project settings. Incorrect values will lead to silent failures in system diagnostics, making it difficult to debug RTOS kernel panics or initialization errors.

`CA_MEMMAP_COUNT`

File: `hw/f1xx/f103/objmap.h`
File: `hw/f4xx/f407/objmap.h`
File: `hw/f4xx/f429/objmap.h`
File: `hw/f4xx/f446/objmap.h`
File: `hw/f7xx/f746/objmap.h`
File: `hw/h5xx/h563/objmap.h`
File: `hw/h7xx/h747/objmap.h`
File: `hw/h7xx/h753/objmap.h`

Defines the total number of memory regions or memory banks managed by the caOS memory protection unit (MPU) or memory map manager.

Valori per MCU:

MCU	Valore
F103	2U
F407	3U
F429	3U
F446	3U
F746	4U
H563	2U
H747	4U
H753	5U

Note:
This constant dictates how many memory segments the RTOS kernel tracks for access control and allocation. Increasing this value allows for more granular memory partitioning but consumes more RAM for the internal management structures. It must match the hardware capabilities of the specific STM32 series (e.g., 2 for F103, 5 for H753). Setting this value incorrectly can lead to memory access faults or kernel panics during initialization.

Uso:
Change this value only when porting caOS to a new STM32 microcontroller family or when reconfiguring the memory layout of the current MCU. Ensure the value matches the specific hardware datasheet requirements for the target chip. Do not arbitrarily increase this value, as it directly impacts the kernel's memory footprint and stack usage.

`CA_MEMMAP_MAX`

File: `caos/inc/memory.h`

Defines the maximum number of memory regions or segments that the caOS memory manager can track simultaneously.

Valore: `16U`

Note:
This constant sets the size of the internal static array used for memory mapping. Increasing this value allows for more granular memory protection or segment definitions but directly increases the static RAM footprint of the kernel. It must be a positive integer, typically ranging from 8 to 32 depending on the complexity of the application's memory layout.

Uso:
Increase this value if the system fails to initialize or reports a memory mapping error when adding new peripheral memory regions or complex task stacks. Decrease it if you need to optimize RAM usage and your application only requires a few distinct memory segments.

`CA_MEMMAP_TABLE`

File: `hw/f1xx/f103/objmap.h`
File: `hw/f4xx/f407/objmap.h`
File: `hw/f4xx/f429/objmap.h`
File: `hw/f4xx/f446/objmap.h`
File: `hw/f7xx/f746/objmap.h`
File: `hw/h5xx/h563/objmap.h`
File: `hw/h7xx/h747/objmap.h`
File: `hw/h7xx/h753/objmap.h`

Defines the memory layout and peripheral base address mapping for the specific STM32 target MCU.

Valori per MCU:

MCU	Valore
`F103`	`\`
`F407`	`\`
`F429`	`\`
`F446`	`\`
`F746`	`\`
`H563`	`\`
`H747`	`\`
`H753`	`\`

Note:
This constant acts as a pointer to the hardware-specific memory map header file. Enabling or selecting a specific MCU variant ensures that the RTOS correctly maps registers, SRAM, and

Flash addresses. It is a mandatory dependency for the HAL and kernel memory management. Valid values are restricted to the supported MCU series (F103, F407, F429, F446, F746, H563, H747, H753).

Uso:
Change this value only when porting the project to a different STM32 hardware target. Setting an incorrect value will lead to immediate hard faults, memory access violations, or peripheral initialization failures. Ensure the selected value matches the physical silicon on your development board exactly.

`CA_MSG_HEADER_SIZE`

File: `caos/inc/message.h`

Defines the number of bytes reserved for the internal header in every caOS message structure.

Valore: `2`

Note:
This value determines the memory offset where the actual message payload begins. Increasing this value consumes more RAM per message buffer. It must be consistent across all compiled modules to prevent memory corruption or misaligned data access. Typical values are small integers representing the size of internal metadata like message IDs or priority tags.

Uso:
Change this value only if you are modifying the core caOS message handling logic or adding new metadata fields to the message header. Setting this value incorrectly will cause the system to read message data from the wrong memory address, leading to unpredictable behavior or hard faults.

`CA_NOTIFY_ACTION_CLEAR_BITS`

File: `caos/inc/notify.h`

Defines the notification action to clear specific bits in the task's notification register upon receipt.

Valore: `2`

Note:
When this flag is used, the bits specified in the notification value are cleared in the target task's internal state after the notification is processed. This is typically used in conjunction with bitwise operations to manage event flags. It is a mutually exclusive option with CA_NOTIFY_ACTION_SET_BITS.

Uso:
Use this flag when you need to implement a 'consume-on-read' mechanism for event flags. It is ideal for clearing specific event triggers once the task has finished handling them. Warning: If you clear bits that other parts of your code are still waiting on, you may cause race conditions or missed events.

`CA_NOTIFY_ACTION_INCREMENT`

File: `caos/inc/notify.h`

Defines the notification action type where the internal counter of a task notification is incremented.

Valore: `4`

Note:
When this flag is used, the target task's notification value is increased by one instead of being overwritten. This is typically used for counting events or signals. It depends on the task notification system being enabled in the kernel configuration. Valid only when passed as an action parameter to the notify API functions.

Uso:
Use this when you need to track the number of occurrences of an event (e.g., counting incoming data packets) rather than just signaling that an event occurred. Do not use this if you need to pass specific bitmasks or data values, as those require the 'overwrite' or 'set bits' actions instead.

`CA_NOTIFY_ACTION_NONE`

File: `caos/inc/notify.h`

A sentinel value indicating that no specific action or callback should be triggered upon a notification event.

Valore: `0`

Note:
When set to this value, the notification system performs no operation, effectively acting as a 'no-op'. It is the default state for notification structures that are initialized but not yet assigned a specific handler. It has no dependencies on other flags and is strictly a constant representing a null-action state.

Uso:
Use this value when initializing a notification structure where you do not want any function to execute or signal to be sent when the event occurs. It is useful for placeholders or disabling specific notification channels without removing the structure from the system. Setting this incorrectly may lead to silent failures where an event occurs but the system appears unresponsive because no action was defined.

`CA_NOTIFY_ACTION_OVERWRITE`

File: `caos/inc/notify.h`

Defines the behavior where a new notification overwrites an existing, unread notification in the queue.

Valore: `3`

Note:
When enabled, the system discards the oldest pending notification if the queue is full, replacing it with the incoming one. This flag is typically used in conjunction with notification handling macros. It requires the notification system to be initialized and is mutually exclusive with blocking notification modes.

Uso:

Use this when you only care about the most recent state or data update and want to prevent the system from stalling due to a full notification buffer. Avoid using this if every notification is critical and must be processed sequentially, as enabling this will lead to data loss.

`CA_NOTIFY_ACTION_SET_BITS`

File: `caos/inc/notify.h`

A notification mode flag that performs a bitwise OR operation on the target task's notification value.

Valore: `1`

Note:
When enabled, the notification value is updated using 'current_value | new_value'. This allows multiple events to be signaled simultaneously without overwriting previous bits. It requires the task to be in a state capable of receiving notifications and is typically used alongside CA_NOTIFY_ACTION_OVERWRITE.

Uso:
Use this when you need to signal multiple independent events to a single task (e.g., flags for UART data, timer expiry, and button press). Avoid using this if you need to pass a specific counter or data value, as bitwise OR will corrupt the data; use CA_NOTIFY_ACTION_OVERWRITE for data passing instead.

`CA_NOTIFY_ACTION_SET_IF_EMPTY`

File: `caos/inc/notify.h`

Flag to set notification value only if no pending value exists

Valore: `5 /**< Write value only if none is pending (value 0),`

Note:
When enabled (value 5), the notification system will only write a new value to the notification structure if there is currently no pending value (indicated by value 0). If a value is already pending, the new value will be ignored. This prevents overwriting important pending notifications. The flag depends on proper initialization of notification structures and requires the notification system to be properly configured. Valid range is 0 (disabled) to 5 (enabled with specific behavior).

Uso:
Enable this flag when you want to prevent notification value overwrites in critical applications where pending notifications must not be lost. Use only when the notification system is properly initialized and when you need to ensure that new values don't overwrite existing pending ones. Disable if you want to allow immediate value updates regardless of pending state. Warning: Setting this flag without proper notification structure initialization may cause unexpected behavior or missed notifications.

`CA_NO_WAIT_TIME`

File: `caos/inc/common.h`

A configuration constant that disables wait time functionality in the caOS RTOS kernel

Valore: `0U`

Note:
When CA_NO_WAIT_TIME is set to 0U (default), the RTOS kernel will not enforce wait time constraints for tasks. This flag typically depends on other timing-related configuration flags like CA_ENABLE_TIMERS. Setting this to 0U removes all wait time limitations, which may cause tasks to block indefinitely. Valid range is typically 0U (disabled) or 1U (enabled), though 0U is the standard default value. When enabled, it may affect task scheduling and system responsiveness.

Uso:
Change this value to 1U only when you need to enable wait time functionality for specific task management requirements. Setting it to 0U (default) is recommended for most applications as it provides maximum flexibility in task scheduling. Warning: Enabling this flag without proper timer configuration may cause system instability or tasks that never complete execution. This flag should be used carefully in real-time applications where deterministic behavior is required.

`CA_PI_MAX_CHAIN`

File: `caos/inc/sync.h`

Defines the maximum number of nested priority inheritance locks allowed for a single task.

Valore: `4U`

Note:
This constant sets the depth of the internal stack used to track priority inheritance chains. Increasing this value consumes more RAM per task control block. It must be at least 1, but typical values range from 2 to 8 depending on the complexity of resource nesting in the application. If a task attempts to acquire more locks than this limit, the system may trigger a kernel panic or fail to grant the mutex.

Uso:
Increase this value if your application architecture involves tasks that hold multiple mutexes simultaneously in a nested fashion (e.g., Task A locks Mutex 1, then Mutex 2, then Mutex 3). If you encounter system instability or assertion failures during complex resource contention, verify if the nesting depth exceeds this constant. Keep it as low as possible to conserve memory.

`CA_PWR_WAKE_LPTIM`

File: `caos/drivers/inc/pwr.h`

Enables the Low Power Timer (LPTIM) as a valid wake-up source for the system from low-power sleep modes.

Valore: `(1U << 2)`

Note:
When enabled, the LPTIM peripheral is configured to trigger an interrupt that exits the CPU from Stop or Standby modes. This flag must be included in the wake-up mask passed to the power management API. It depends on the LPTIM peripheral clock being active during sleep, typically requiring the LSI or LSE clock source to be enabled.

Uso:

Enable this flag when your application requires periodic wake-ups or time-based events while the system is in a low-power state. Do not enable this if the LPTIM is not initialized, as it may cause the system to fail to wake up or trigger spurious interrupts.

CA_PWR_WAKE_LPUART

File: `caos/drivers/inc/pwr.h`

Enables the LPUART peripheral as a valid wake-up source for the system from low-power sleep modes.

Valore: `(1U « 3)`

Note:
When enabled, the LPUART hardware is configured to trigger an interrupt that wakes the CPU from Stop or Standby modes upon receiving data. This flag must be used in conjunction with the system power management driver. If disabled, the LPUART will remain inactive during sleep, preventing serial communication from waking the device. The value is a bitmask intended for use with the CA_PWR_EnableWakeupSource() function.

Uso:
Enable this flag when your application requires the device to remain in a low-power state while waiting for incoming serial commands or data. Ensure the LPUART clock is configured to remain active during sleep in the RCC settings, otherwise, the wake-up signal will not be detected. Do not enable this if the LPUART is not physically connected to an external device, as noise on the RX line could cause unintended wake-ups.

CA_PWR_WAKE_PIN

File: `caos/drivers/inc/pwr.h`

Defines the specific GPIO pin index used to trigger a system wake-up event from low-power sleep modes.

Valore: `(1U « 1)`

Note:
This constant represents a bitmask (1U « 1), targeting Pin 1 of the designated wake-up port. Enabling this flag allows the hardware to transition from standby or stop mode to active mode upon a signal change on this pin. It depends on the CA_PWR_WAKE_PORT configuration being correctly initialized. Valid values are bit-shifted integers corresponding to valid GPIO pins (0-15).

Uso:
Change this value when the hardware design requires a different physical pin to wake the MCU from sleep. Use this to solve issues where the system fails to resume operation after entering low-power states. Warning: Ensure the selected pin is physically connected to the wake-up source; using an incorrect pin will result in the system remaining permanently stuck in sleep mode.

CA_PWR_WAKE_RTC

File: `caos/drivers/inc/pwr.h`

Enables the Real-Time Clock (RTC) alarm as a valid wake-up source for the system from low-power sleep modes.

Valore: `(1U « 0)`

Note:
When enabled, the RTC peripheral is configured to trigger an interrupt that exits the CPU from sleep or stop modes. This flag must be used in conjunction with the system's power management API. If disabled, the RTC will continue to run but will not be capable of waking the processor, potentially causing the system to remain stuck in a low-power state indefinitely if no other wake-up sources are active.

Uso:
Enable this flag when your application requires periodic tasks or scheduled wake-ups based on time (e.g., waking up every hour to send sensor data). Ensure the RTC clock source (LSE or LSI) is correctly initialized before enabling this flag, otherwise, the system may fail to wake up as expected.

`CA_QSPI_DEV_NOR`

File: `caos/drivers/inc/qspi.h`

A configuration flag that enables or disables support for NOR-type flash memory devices on the QSPI peripheral.

Valore: `0U`

Note:
When set to 1U, the driver initializes the specific command sets and timing parameters required for NOR flash chips. When set to 0U, this support is excluded from the build to save code space. This flag depends on the QSPI peripheral being enabled in the system configuration. Valid values are 0U (disabled) or 1U (enabled).

Uso:
Change this value to 1U only if your hardware design includes a NOR flash memory connected to the QSPI bus. If you are using a different type of memory or no QSPI device at all, keep this at 0U to prevent compilation errors or unnecessary memory usage. Setting this to 1U without a physical NOR flash chip present may cause the driver initialization to hang or return errors.

`CA_QSPI_DEV_RAM`

File: `caos/drivers/inc/qspi.h`

Enables or disables the use of internal RAM as a buffer for QSPI flash operations.

Valore: `1U`

Note:
When set to 1U, the driver allocates a dedicated memory buffer in internal SRAM for QSPI data transfers. Disabling this (0U) forces the driver to use direct memory-mapped access or external buffers, which may increase latency. This flag depends on having sufficient free SRAM; if enabled, ensure the linker script provides enough heap or static memory to accommodate the buffer size.

Uso:

Enable this when you need to improve read/write performance for QSPI flash operations by reducing bus contention. Change this to 0U if your application is running low on internal SRAM or if you are using a custom DMA implementation that manages its own memory buffers. Incorrectly enabling this on memory-constrained devices may lead to stack overflows or heap exhaustion.

`CA_QSPI_DEV_RAW`

File: `caos/drivers/inc/qspi.h`

Defines the operating mode of the QSPI peripheral to use raw memory-mapped access instead of a file system or abstraction layer.

Valore: `2U`

Note:
When set to 2U, the QSPI driver bypasses high-level storage drivers and exposes the flash memory as a raw byte-addressable array. This flag is mutually exclusive with CA_QSPI_DEV_FS. Valid values are typically 0U (disabled), 1U (buffered), and 2U (raw).

Uso:
Change this to 2U when you need direct, low-latency access to the flash memory for custom data structures or bootloader operations. Warning: Using raw mode disables built-in wear leveling and file system safety features; ensure your application handles data integrity manually.

`CA_QSPI_MMAP_BASE`

File: `caos/drivers/inc/qspi.h`

Defines the starting memory address where the external QSPI flash is mapped into the STM32 memory space.

Valore: `0x90000000UL`

Note:
This constant sets the base address for memory-mapped mode (XIP). Changing this value must strictly match the memory map defined in the STM32 reference manual for the specific QSPI peripheral instance. It is typically set to 0x90000000 for the first QSPI bank on most STM32 series. Incorrect values will result in hard faults or data corruption when attempting to read from flash memory.

Uso:
Change this value only if you are porting the driver to a different STM32 microcontroller or if the hardware design uses a different memory bank mapping. Ensure that the linker script (.ld file) is updated to reflect this address if you intend to execute code or store constants directly from the QSPI flash.

`CA_QSPI_SSHIFT_AUTO`

File: `caos/drivers/inc/qspi.h`

Enables or disables the automatic shifting of data samples for the Quad-SPI interface.

Valore: `0U`

Note:

When set to 1, the hardware automatically adjusts the sampling clock phase to compensate for signal propagation delays. When set to 0, the sampling phase is fixed. This flag depends on the specific QSPI peripheral clock frequency and PCB trace length. Valid values are 0 (disabled) or 1 (enabled).

Uso:
Enable this if you experience data corruption or CRC errors during high-speed QSPI read operations. Disable it if your hardware design uses a custom clock-delay circuit or if the peripheral is running at low frequencies where timing margins are naturally sufficient. Incorrect configuration may lead to intermittent read failures or complete communication loss.

`CA_QSPI_SSHIFT_AUTO_HZ`

File: `caos/drivers/inc/qspi.h`

Defines the frequency threshold in Hz at which the QSPI driver automatically enables data sampling shift.

Valore: `40000000U`

Note:
When the QSPI clock frequency exceeds this value, the driver enables the internal delay line to compensate for signal propagation delays. If set too low, the system may experience data corruption at high speeds; if set too high, the system may fail to sample data correctly at lower speeds. The default value is 40MHz, which is standard for most QSPI flash memories on STM32 platforms.

Uso:
Adjust this value if you are using a specific QSPI flash chip that requires a different timing margin or if you are operating the QSPI bus at non-standard frequencies. Ensure this value is lower than the maximum clock frequency supported by your external flash memory to prevent read errors.

`CA_QSPI_SSHIFT_OFF`

File: `caos/drivers/inc/qspi.h`

Defines the clock phase shift offset for the Quad-SPI (QSPI) peripheral interface.

Valore: `2U`

Note:
This constant sets the internal delay cycles for the QSPI clock signal relative to the data sampling edge. Increasing this value shifts the clock phase forward, which is critical for maintaining signal integrity at high clock frequencies. It depends on the hardware trace length and the specific flash memory chip's timing requirements. Valid range is typically 0U to 3U, depending on the STM32 hardware revision.

Uso:
Adjust this value if you encounter data corruption or CRC errors during high-speed QSPI read operations. If the system fails to read from external flash reliably, incrementing this value can compensate for propagation delays on the PCB. Warning: setting an incorrect value will cause communication failure or intermittent data bit-flips; always verify with an oscilloscope or logic analyzer when tuning.

CA_QSPI_SSHIFT_ON

File: caos/drivers/inc/qspi.h

Enables the data sampling shift feature for the Quad-SPI peripheral to improve signal integrity.

Valore: 1U

Note:
When set to 1U, the QSPI controller adds a half-cycle delay to the data sampling clock. This is critical for high-frequency QSPI operations where signal propagation delay might cause data to be sampled incorrectly. It depends on the hardware layout and the specific flash memory chip's timing requirements. Valid values are 0U (disabled) or 1U (enabled).

Uso:
Enable this flag if you encounter data corruption or CRC errors during QSPI read operations at high clock speeds. If the system fails to communicate with the external flash after enabling this, verify if your specific hardware design requires the shift or if the clock frequency is too high for the current trace length.

CA_RTC_ALARM_A

File: caos/drivers/inc/rtc.h

Enables the RTC Alarm A interrupt functionality within the caOS driver layer.

Valore: 1u

Note:
When set to 1u, the system initializes and enables the hardware interrupt for Alarm A. If set to 0u, the alarm hardware is ignored, and any associated callback functions will never trigger. This flag depends on the RTC peripheral clock being enabled in the system configuration.

Uso:
Set this to 1u when your application needs to trigger periodic tasks or wake the MCU from low-power modes at a specific time. Disable it (0u) to save code space if your project does not utilize RTC alarms, or to prevent interrupt conflicts if you are manually managing the RTC registers.

CA_RTC_ALARM_B

File: caos/drivers/inc/rtc.h

Defines the hardware alarm channel index used for the RTC alarm interrupt.

Valore: 2u

Note:
This constant specifies which of the available RTC alarm registers (e.g., Alarm A or Alarm B) the system driver will utilize. Setting this to 2u typically maps to Alarm B on STM32 hardware. It must be consistent with the hardware peripheral configuration and should not conflict with other drivers using the same alarm channel. Valid values are typically 1u or 2u depending on the specific STM32 series.

Uso:

Change this value if the system design requires a specific RTC alarm channel for task scheduling or periodic wake-ups. Use this when Alarm A is already reserved by another system component or peripheral. Incorrect values will result in the RTC interrupt failing to trigger, causing the RTOS to miss scheduled events or fail to wake from low-power modes.

`CA_SAFEGUARD`

File: `caos/inc/safeguard.h`

A macro-based dispatcher that validates the number of arguments passed to system-critical RTOS functions.

Valore: `_SAFEGUARD_DISPATCH_(_SAFEGUARD_NARG(__VA_ARGS__))(__VA_ARGS__)`

Note:
This constant acts as a compile-time safety layer. When enabled, it ensures that the correct number of parameters is provided to kernel primitives. If the argument count is mismatched, the compiler will throw an error, preventing undefined behavior or stack corruption. It has no runtime overhead as it is resolved during the pre-processing stage.

Uso:
You should keep this enabled during development and debugging to catch API misuse early. It is typically disabled only in extreme memory-constrained production environments where every byte of pre-processed code size matters, though this is rarely recommended. Do not attempt to redefine this macro unless you are modifying the core kernel dispatch logic.

`CA_SAFE_FAIL`

File: `caos/inc/safeguard.h`

A macro used to trigger a system-wide safety halt or error recovery routine when a critical condition is not met.

Valore: `CA_SAFEGUARD(0, msg)`

Note:
When triggered, the system typically enters a locked state or resets to prevent undefined behavior. It depends on the CA_ENABLE_SAFEGUARD flag being set to 1. The first parameter is the error code, and the second is a descriptive string for debugging purposes.

Uso:
Use this to wrap critical assertions or hardware state checks. Change this when you need to define custom error handling for specific modules. Warning: triggering this in a production environment will stop the application execution; ensure it is only used for truly unrecoverable system states.

`CA_SAI_ERR_AFS`

File: `caos/drivers/inc/sai.h`

Error flag indicating an Audio Frame Synchronization (AFS) error in the SAI peripheral.

Valore: `0x04U`

Note:

This constant is a bitmask used within the ca_sai_errors() function to identify specific hardware-level synchronization failures. When this bit is set in the error status register, it means the SAI peripheral failed to align the audio frame correctly with the incoming clock signal. It is typically used in conjunction with other error flags in the ca_sai driver. The value 0x04U is specific to the SAI hardware abstraction layer in caOS.

Uso:
Check this flag when debugging audio glitches, pops, or silence in your application. If ca_sai_errors() returns a value where (status & CA_SAI_ERR_AFS) is true, verify your external audio clock source (MCLK/BCLK) stability and ensure the SAI configuration matches the peripheral's expected frame format. Do not modify this value as it is defined by the hardware driver interface.

`CA_SAI_ERR_LFS`

File: `caos/drivers/inc/sai.h`

Error flag indicating a Low-Level Frame Synchronization (LFS) error in the SAI peripheral.

Valore: `0x08U`

Note:
This constant represents a bitmask value (0x08U) used within the SAI error handling logic. When the SAI hardware detects a frame synchronization error, this bit is set in the error status register. It is typically checked by the ca_sai_errors function to determine if the audio stream has lost synchronization with the external codec or I2S device.

Uso:
Use this constant when implementing custom error recovery routines or logging within the SAI interrupt service routine. If this error occurs frequently, check the physical clock signals (MCLK, SCK, WS) for noise or timing violations. Do not modify this value as it is mapped directly to the hardware status register bit.

`CA_SAI_ERR_OVR`

File: `caos/drivers/inc/sai.h`

Flag indicating a Serial Audio Interface (SAI) Overrun error.

Valore: `0x02U`

Note:
This constant represents the bitmask for an overrun condition in the SAI peripheral, where the hardware receives data faster than the software can process it. It is typically used within the ca_sai_errors function to check the status register. When this bit is set, it indicates data loss in the audio stream. It is a hardware-level flag defined in the STM32 SAI peripheral registers.

Uso:
Use this flag when debugging audio glitches or stuttering in your application. If ca_sai_errors returns a value containing CA_SAI_ERR_OVR, it means your task is not reading the SAI buffer fast enough. To solve this, ensure your audio processing task has a higher priority or optimize the code inside your audio callback to reduce execution time.

CA_SAI_ERR_UDR

File: caos/drivers/inc/sai.h

Flag representing a Serial Audio Interface (SAI) underrun error.

Valore: 0x01U

Note:
This constant is a bitmask value (0x01U) used within the ca_sai_errors function to identify if the SAI peripheral has failed to provide data to the audio buffer in time. When this bit is set in the error status, it indicates that the audio stream has been interrupted, typically resulting in audible clicks or pops. It is a hardware-level status flag dependent on the STM32 SAI peripheral configuration.

Uso:
Use this flag when checking the return value of ca_sai_errors to diagnose audio playback issues. If this error occurs frequently, it suggests that the system is under heavy CPU load or that the DMA transfer is not keeping up with the audio sample rate. You should investigate task priorities or increase the audio buffer size if this flag is consistently detected.

CA_SAI_SPDIF_C

File: caos/drivers/inc/sai.h

Configuration flag to enable SPDIF input/output mode for the SAI peripheral.

Valore: (1UL « 26)

Note:
This constant is a bitmask (1UL « 26) used within the ca_sai_init function to configure the SAI hardware interface for SPDIF protocol support. When enabled, the SAI peripheral switches from standard I2S/PCM framing to SPDIF mode. It is mutually exclusive with standard I2S configurations and requires the hardware pins to be physically connected to an SPDIF transceiver or optical module.

Uso:
Use this flag when initializing the SAI peripheral specifically for digital audio transmission or reception via SPDIF. Do not enable this if you are using standard I2S or TDM audio protocols, as it will cause incorrect clocking and data framing, resulting in audio noise or complete signal loss.

CA_SAI_SPDIF_U

File: caos/drivers/inc/sai.h

A bitmask flag used to enable SPDIF input functionality on the SAI peripheral.

Valore: (1UL « 25)

Note:
This constant represents the 25th bit of a configuration register. When enabled, it configures the Serial Audio Interface (SAI) to operate in SPDIF receiver mode rather than standard I2S/TDM mode. It is typically passed to the ca_sai_init function. Ensure the hardware pins are physically connected to an SPDIF source before enabling this flag.

Uso:

Use this flag when your application needs to receive digital audio data via an SPDIF interface. Do not enable this if you are using the SAI peripheral for standard I2S communication, as it will misconfigure the clock and data framing logic, leading to invalid audio data or hardware errors.

CA_SAI_SPDIF_V

File: `caos/drivers/inc/sai.h`

A bitmask flag used to configure the SAI peripheral for S/PDIF protocol mode.

Valore: `(1UL « 24)`

Note:
When enabled, this flag modifies the SAI peripheral's internal register configuration to support the S/PDIF (Sony/Philips Digital Interface) standard instead of standard I2S/PCM audio formats. It is typically passed as an argument to the ca_sai_init function. It depends on the underlying STM32 hardware supporting S/PDIF mode within the SAI block.

Uso:
Use this flag when your application needs to interface with digital audio equipment via S/PDIF. Do not use this flag if you are connecting to standard I2S DACs or ADCs, as it will cause incorrect clocking and data framing. Ensure the hardware pins are correctly routed for S/PDIF signals before enabling this.

CA_SDIO_READ_TIMEOUT_MS

File: `caos/drivers/inc/sdio.h`

SDIO read timeout value in milliseconds for caOS RTOS

Valore: `100U`

Note:
Controls how long the SDIO driver will wait for a read operation to complete before timing out. When enabled, this timeout prevents the system from hanging indefinitely during SD card operations. The value must be greater than 0 and typically ranges from 50-500ms. This flag is independent of other SDIO configuration parameters but affects the reliability of SD card communication. A value of 100ms is the default safe setting.

Uso:
Change this value when experiencing SD card read timeouts or slow performance. Increase to 200-500ms if SD card operations are failing due to slow card response, or decrease to 50-100ms if you need faster response times and know the SD card is performing well. Setting this value too low (below 20ms) may cause frequent timeouts, while setting it too high (above 1000ms) may cause unnecessary delays in error detection. The default value of 100ms provides a good balance between reliability and performance for most SD card applications.

CA_SDIO_WRITE_TIMEOUT_MS

File: `caos/drivers/inc/sdio.h`

Configures the timeout duration for SDIO write operations in milliseconds

Valore: `500U`

Note:
This constant determines how long the system will wait for an SDIO write operation to complete before timing out. When increased, it allows more time for slow SD card operations or noisy electrical environments. When decreased, it makes the system more responsive but may cause timeouts on slower cards. This timeout directly affects the SDIO driver's reliability and performance. The value must be greater than 0 and typically ranges from 100-2000ms depending on SD card type and system requirements.

Uso:
Increase this value when experiencing SD card write timeouts, especially with slower SD cards or in noisy electrical environments. Decrease only when you need faster error detection and are certain about card performance. Setting it too low (below 100ms) may cause false timeouts, while setting it too high (above 2000ms) may delay system response times. Typical value of 500ms is suitable for most standard SD cards in normal operating conditions.

`CA_SECRET_AREA_BASE`

File: `hw/h5xx/h563/objmap.h`
File: `hw/u5xx/u575/objmap.h`

Defines the starting memory address for the caOS secure storage area.

Valori per MCU:

MCU	Valore
H563	0x20040000UL
U575	0x20030000UL

Note:
This constant sets the base address in SRAM where sensitive data (like keys or secrets) is reserved. The value is hardware-dependent: 0x20040000UL for STM32H563 and 0x20030000UL for STM32U575. It is used by the ca_memory_reserve_secret function to ensure specific memory regions are protected from general allocation.

Uso:
Do not change this value manually unless porting to a new MCU variant. Ensure the address aligns with the specific SRAM bank layout defined in the MCU datasheet to avoid memory corruption or overlapping with the stack/heap. If you need to store sensitive data, use ca_memory_reserve_secret instead of accessing this address directly.

`CA_SECRET_AREA_SIZE`

File: `hw/h5xx/h563/objmap.h`
File: `hw/u5xx/u575/objmap.h`

Defines the size of the protected memory region reserved for sensitive data storage.

Valori per MCU:

MCU	Valore
H563	(64UL * 1024UL)
U575	(64UL * 1024UL)

Note:
This constant allocates a fixed block of memory (typically 64KB) used by the caOS security subsystem. It is used by functions like ca_memory_reserve_secret to isolate critical data from

general application access. Increasing this value consumes more RAM, while decreasing it may cause allocation failures for secure objects. It must be aligned with the hardware-specific memory map of the STM32 H563 or U575.

Uso:
Adjust this value only if your application requires more or less space for secure keys, certificates, or encrypted buffers. If you encounter errors during ca_memory_reserve_secret, verify if this area is exhausted. Ensure the value is a multiple of the memory page size for your specific MCU to avoid alignment issues.

`CA_SECURE_PIPE_DECRYPT`

File: `caos/crypto/inc/secure_pipe_service.h`

Defines the error code returned by the secure pipe service when a decryption operation fails.

Valore: `-3`

Note:
This constant is a negative integer (-3) used as a return value in the ca_secure_pipe_recv function. It indicates that the received data could not be decrypted, likely due to an invalid key, corrupted payload, or authentication failure. It is a specific error state within the secure pipe communication layer.

Uso:
Use this constant to check the return value of ca_secure_pipe_recv. If the function returns CA_SECURE_PIPE_DECRYPT, the application should discard the received buffer and potentially trigger a re-keying process or log a security event. Do not modify this value unless you are updating the entire error code mapping of the caOS crypto stack.

`CA_SECURE_PIPE_ERR`

File: `caos/crypto/inc/secure_pipe_service.h`

Error code returned by the secure pipe service when an operation fails.

Valore: `-1`

Note:
This constant is defined as -1. It is used as a return value for functions like ca_secure_pipe_send or ca_secure_pipe_recv to indicate that the secure communication channel encountered an internal error, such as a timeout, decryption failure, or invalid packet structure. It does not depend on other flags but is specific to the secure pipe subsystem.

Uso:
Use this constant to check the return status of secure pipe operations. If a function returns CA_SECURE_PIPE_ERR, you should handle the failure by logging the event or attempting to re-initialize the pipe using ca_secure_pipe_deinit and ca_secure_pipe_init. Do not attempt to process data buffers if this error code is returned, as the integrity of the pipe is no longer guaranteed.

`CA_SECURE_PIPE_MAX_MSG`

File: `caos/crypto/inc/secure_pipe_service.h`

Defines the maximum number of messages that can be queued in a secure pipe.

Valore: `0xFFFFU`

Note:
This constant sets the upper limit for the message buffer capacity within the secure pipe service. Increasing this value allows for higher throughput and better handling of burst traffic but consumes more RAM. The value 0xFFFFU (65535) is the maximum allowed, representing the total number of pending messages the system can track before rejecting new ones.

Uso:
Adjust this value if you encounter 'buffer full' errors or dropped messages during high-frequency secure communication. If your system has limited RAM, consider reducing this value to save memory. Ensure that the memory allocated for the pipe is sufficient to hold the number of messages defined by this constant.

`CA_SECURE_PIPE_NOT_UP`

File: `caos/crypto/inc/secure_pipe_service.h`

Error code indicating that the secure communication pipe is not currently established.

Valore: `-6`

Note:
This constant is returned by secure pipe functions (e.g., ca_secure_pipe_send or ca_secure_pipe_recv) when the underlying link is not in an 'up' state. It is a negative integer (-6), consistent with other caOS error codes. It typically occurs if the handshake process has not completed or if the connection was dropped.

Uso:
Check for this return value when calling secure pipe functions. If received, the application should stop attempting to send or receive data and trigger the connection logic (e.g., ca_sp_link_open) to re-establish the secure channel. Do not ignore this error, as it indicates that data transmission is currently impossible.

`CA_SECURE_PIPE_OK`

File: `caos/crypto/inc/secure_pipe_service.h`

Enables or disables the secure pipe communication service in caOS.

Valore: `0`

Note:
When set to 1, the system includes the secure pipe service logic, allowing for encrypted data transmission channels. When set to 0 (default), the service is disabled, reducing the binary footprint and removing dependencies on secure pipe internal buffers. This flag is a compile-time constant; changing it requires a full project rebuild.

Uso:
Change this value to 1 only if your application requires secure, encrypted inter-process or inter-device communication using ca_secure_pipe_init and related functions. If your project does

not use secure pipes, keep it at 0 to save memory and processing overhead. Ensure that if enabled, you have sufficient heap memory allocated for the secure pipe buffers.

CA_SECURE_PIPE_REPLAY

File: caos/crypto/inc/secure_pipe_service.h

Defines the maximum number of allowed replay attempts or the error code for replay detection in the secure pipe service.

Valore: -5

Note:
This constant is used by the secure pipe service to handle packet replay attacks. A value of -5 typically indicates a specific error state returned by the crypto layer when a sequence number mismatch or a replayed packet is detected. It is a signed integer; negative values are reserved for error signaling, while positive values would represent a threshold count.

Uso:
Do not modify this value unless you are performing low-level debugging of the secure communication protocol. If you encounter this error code during development, it suggests that the device is receiving duplicate or out-of-order encrypted packets, which may indicate a network issue or an active security threat. Ensure that sequence numbers are correctly managed by the sender using ca_secure_pipe_send.

CA_SECURE_PIPE_REPLY

File: caos/crypto/inc/secure_pipe_service.h

Enables automatic reply generation for secure pipe communication.

Valore: 1

Note:
When set to 1, the caOS secure pipe service automatically handles acknowledgement or reply packets for incoming requests. Disabling this (0) requires the application layer to manually manage all response transmissions using ca_secure_pipe_send. This flag is a dependency for automated secure handshake protocols.

Uso:
Enable this when you want to reduce code complexity in your communication tasks by offloading reply logic to the OS. Disable it if your application requires custom, non-standard response timing or specific packet formatting that the automated service cannot provide. Ensure that if this is enabled, your task logic does not conflict with the automatic replies to avoid duplicate or malformed data on the pipe.

CA_SECURE_PIPE_SIZE

File: caos/crypto/inc/secure_pipe_service.h

Defines the internal buffer size for the secure communication pipe service.

Valore: -4

Note:

This constant determines the memory allocation for the secure pipe's internal data buffer. A value of -4 is a special configuration flag indicating that the buffer size is dynamically managed or derived from a specific hardware-dependent memory pool rather than a fixed static allocation. Modifying this value directly may cause memory corruption or initialization failures in the ca_secure_pipe_init service.

Uso:
Do not change this value unless instructed by the system architecture team. It is used to balance memory footprint against throughput for encrypted data streams. If you encounter 'buffer overflow' errors in secure communication, verify the memory pool availability using ca_memory_get_avail_memory before attempting to adjust system-wide buffer configurations.

`CA_SECURE_PIPE_TIMEOUT`

File: `caos/crypto/inc/secure_pipe_service.h`

Defines the default timeout duration for secure pipe communication operations.

Valore: `-2`

Note:
This constant sets the wait time for secure pipe operations. A value of -2 typically indicates an infinite wait or a specific blocking behavior defined by the caOS secure pipe service. Changing this affects how long tasks block when calling ca_secure_pipe_send or ca_secure_pipe_recv. Ensure that the system architecture can handle blocking calls if this is set to a high value or infinite.

Uso:
Modify this value if secure communication operations are timing out prematurely in high-latency environments or if you need to enforce stricter real-time constraints. Use caution: setting this too low may cause frequent communication failures, while setting it to infinite (if supported) may lead to task starvation if the remote peer becomes unresponsive.

`CA_SLICE_MIN_TICKS`

File: `hw/f1xx/f103/objmap.h`
File: `hw/f4xx/f407/objmap.h`
File: `hw/f4xx/f429/objmap.h`
File: `hw/f4xx/f446/objmap.h`
File: `hw/f7xx/f746/objmap.h`
File: `hw/f7xx/f767/objmap.h`
File: `hw/h5xx/h563/objmap.h`
File: `hw/h7xx/h747/objmap.h`
File: `hw/h7xx/h753/objmap.h`
File: `hw/u5xx/u575/objmap.h`

Defines the minimum number of system ticks a task is guaranteed to run before being preempted by the scheduler.

Valori per MCU:

MCU	Valore
F103	2U
F407	2U
F429	2U
F446	2U
F746	2U
F767	2U
H563	2U
H747	2U
H753	2U
U575	2U

Note:
This constant controls the granularity of the round-robin time-slicing mechanism. Increasing this value reduces the frequency of context switches, which lowers CPU overhead but increases latency for task responsiveness. It is typically set to 2U across all supported STM32 platforms to balance throughput and real-time performance. It depends on the system tick frequency configured via ca_os_tick_init.

Uso:
Modify this value if you need to optimize for high-throughput background processing (increase the value) or if you require faster task switching for time-critical event handling (decrease the value). Warning: setting this to 0 or 1 may cause excessive context switching, leading to 'thrashing' where the CPU spends more time saving/restoring registers than executing application code.

`CA_SPDIFRX_C`

File: `caos/drivers/inc/spdifrx.h`

A bitmask flag used to enable the SPDIFRX peripheral in caOS.

Valore: `(1UL « 27)`

Note:
This constant represents the 27th bit of a configuration register. When enabled, it allows the caOS driver to initialize and interact with the hardware SPDIF receiver. It is typically used as an argument in the ca_spdifrx_init function. Ensure the hardware pins are correctly configured before setting this flag.

Uso:
Use this flag when you need to initialize the SPDIFRX peripheral for digital audio input. If the peripheral is not required, keep this bit unset to save power and avoid unnecessary hardware initialization. Do not modify the bit position (1UL « 27) as it is hardcoded to match the STM32 hardware register mapping.

`CA_SPDIFRX_CS_BYTES`

File: `caos/drivers/inc/spdifrx.h`

Defines the size in bytes of the S/PDIF Channel Status block buffer.

Valore: `24U`

Note:

This constant sets the memory allocation size for storing the S/PDIF Channel Status data. The S/PDIF specification defines the Channel Status block as 192 bits (24 bytes). Changing this value will affect the memory footprint of the ca_spdifrx driver. It must be set to 24U to comply with the standard S/PDIF protocol.

Uso:
Do not change this value unless you are implementing a non-standard S/PDIF protocol variant that uses a different Channel Status block size. Incorrect values will cause the ca_spdifrx_channel_status function to return incomplete or corrupted data, potentially leading to synchronization errors or misinterpretation of the audio stream metadata.

`CA_SPDIFRX_ERR_FRM`

File: `caos/drivers/inc/spdifrx.h`

Flag indicating a framing error in the SPDIFRX peripheral.

Valore: `0x04U`

Note:
This constant represents the 0x04 bit in the SPDIFRX error status register. It is triggered when the incoming SPDIF stream fails to adhere to the expected subframe structure. It is typically checked via the ca_spdifrx_errors() function. It does not have dependencies on other flags but is part of the broader error bitmask returned by the driver.

Uso:
Use this flag when debugging audio signal integrity issues or synchronization problems. If ca_spdifrx_errors() returns a value where (errors & CA_SPDIFRX_ERR_FRM) is true, it indicates that the physical connection or the source clock is unstable. If this error persists, check the physical cable quality or the source device's sample rate stability.

`CA_SPDIFRX_ERR_OVR`

File: `caos/drivers/inc/spdifrx.h`

Flag indicating an Overrun Error in the SPDIFRX peripheral.

Valore: `0x02U`

Note:
This constant represents a bitmask (0x02U) used to identify when the SPDIFRX hardware buffer has been overwritten before the CPU could read the previous data. It is typically checked within the ca_spdifrx_errors function. If this bit is set, it means the application is not processing incoming audio data fast enough to keep up with the hardware sample rate.

Uso:
Use this flag when debugging audio dropouts or glitches. If ca_spdifrx_errors returns a value containing this bit, you must optimize your data processing loop or increase the priority of the task handling the audio stream. Do not modify this value, as it is defined by the hardware register mapping.

`CA_SPDIFRX_ERR_PAR`

File: `caos/drivers/inc/spdifrx.h`

Bitmask flag representing a parity error in the SPDIFRX peripheral.

Valore: `0x01U`

Note:
This constant is used as a bitmask within the ca_spdifrx_errors() function. When the SPDIFRX hardware detects a parity error in the incoming stream, this bit is set in the error status register. It is typically checked alongside other error flags (like framing or overrun errors) to determine the health of the digital audio input signal.

Uso:
Use this constant when implementing custom error handling logic for the SPDIFRX driver. If ca_spdifrx_errors() returns a value where (result & CA_SPDIFRX_ERR_PAR) is true, it indicates that the received audio data contains parity violations, which may result in audible noise or signal dropouts. Do not modify this value as it maps directly to the hardware register definition.

`CA_SPDIFRX_ERR_SYNC`

File: `caos/drivers/inc/spdifrx.h`

Flag indicating a synchronization error in the S/PDIF receiver peripheral.

Valore: `0x08U`

Note:
This constant represents a bitmask (0x08U) used within the ca_spdifrx_errors() function to identify if the S/PDIF hardware has lost synchronization with the incoming audio stream. When this bit is set in the error status register, the receiver is unable to lock onto the incoming clock or data frame. It is typically checked alongside other error flags returned by the driver.

Uso:
Use this flag when implementing a callback or polling loop for the S/PDIF interface to detect signal loss. If ca_spdifrx_errors() returns a value where (result & CA_SPDIFRX_ERR_SYNC) is true, the application should trigger a re-synchronization routine or notify the user that the audio source is disconnected or unstable. Do not modify this value as it is mapped directly to the hardware register bit definition.

`CA_SPDIFRX_ERR_TMO`

File: `caos/drivers/inc/spdifrx.h`

Defines the timeout threshold for SPDIFRX error detection.

Valore: `0x10U`

Note:
This constant represents the number of cycles or internal ticks the SPDIFRX peripheral waits before flagging an error state. It is used by the ca_spdifrx driver to handle signal instability or synchronization loss. Typical values are small integers; 0x10U is the default configuration. Increasing this value makes the system more tolerant to jitter but slower to report actual signal loss.

Uso:
Adjust this value if you experience frequent, false-positive error interrupts during SPDIF signal reception. If the signal is noisy, increasing the timeout can prevent unnecessary driver resets. If the system fails to detect a disconnected cable or a dead signal source promptly, consider decreasing this value.

`CA_SPDIFRX_HW_SBD`

File: `caos/drivers/inc/spdifrx.h`

Defines the hardware synchronization bit mask for the SPDIFRX peripheral.

Valore: `0x200U`

Note:
This constant represents the bit field used to identify the Synchronization Block Detected (SBD) status within the SPDIFRX hardware registers. It is typically used by the ca_spdifrx_sync function to verify if the incoming audio stream has achieved frame and block synchronization. It is a hardware-specific constant and should not be modified unless the underlying STM32 hardware revision changes.

Uso:
Use this constant when checking the status of the SPDIFRX peripheral to ensure a valid audio signal is present before attempting to read data. If ca_spdifrx_sync returns a value that does not match this mask, the audio stream is not yet locked, and reading data will result in invalid samples or noise.

`CA_SPDIFRX_HW_STOP`

File: `caos/drivers/inc/spdifrx.h`

Defines the set of SPDIFRX hardware error conditions that trigger an automatic stop of the receiver.

Valore: `(CA_SPDIFRX_ERR_FRM | CA_SPDIFRX_ERR_SYNC | CA_SPDIFRX_ERR_TMO)`

Note:
This constant acts as a bitmask. When the hardware detects any of the included error flags (Frame error, Synchronization error, or Timeout error), the ca_spdifrx_stop() function is automatically invoked to halt the peripheral. It depends on the underlying STM32 SPDIFRX peripheral error registers. Typical values are a combination of CA_SPDIFRX_ERR_* flags.

Uso:
Modify this value if you need to change the error-handling policy of the SPDIFRX interface. For example, if you want the system to ignore specific errors (like minor frame errors) to maintain audio stream continuity, remove the corresponding flag from the mask. Warning: removing critical error flags may lead to the processing of corrupted data or system instability if the hardware enters an undefined state.

`CA_SPDIFRX_HW_SYNCD`

File: `caos/drivers/inc/spdifrx.h`

Flag to enable hardware-level synchronization for the SPDIFRX peripheral.

Valore: `0x100U`

Note:
When enabled (0x100U), the driver forces the hardware to wait for a valid synchronization signal before reporting the stream as ready. If disabled, the driver may attempt to read data before the clock or frame alignment is stable. This flag is typically used in conjunction with ca_spdifrx_init and ca_spdifrx_sync. It is a bitmask value intended for use in the configuration structure of the SPDIFRX driver.

Uso:
Enable this flag if you experience noise, corrupted audio frames, or 'garbage' data at the start of an SPDIF stream. It solves synchronization issues caused by the peripheral starting before the external transmitter has fully locked onto the signal. Warning: Enabling this may increase the latency between calling ca_spdifrx_init and the first valid data read, as the system must wait for the hardware to achieve lock.

`CA_SPDIFRX_PE`

File: `caos/drivers/inc/spdifrx.h`

Enables the Parity Error (PE) interrupt for the SPDIFRX peripheral.

Valore: `(1UL « 24)`

Note:
When this bit is set, the SPDIFRX hardware will trigger an interrupt if a parity error is detected in the incoming data stream. This flag is typically used in conjunction with the ca_spdifrx_init function. It is a bitmask value (1UL « 24) intended to be ORed with other configuration flags.

Uso:
Enable this flag when you need to monitor the integrity of the incoming SPDIF signal. If your application experiences data corruption or synchronization issues, enabling this interrupt allows you to catch parity errors in real-time via the ca_spdifrx_errors function. Do not enable this if your hardware source does not support parity or if you want to ignore transmission errors to prioritize continuous data flow.

`CA_SPDIFRX_PT`

File: `caos/drivers/inc/spdifrx.h`

Extracts the 2-bit S/PDIF channel status information from a raw 32-bit register value.

Valore: `(((u32)(w) » 28) & 3U)`

Note:
This macro performs a bitwise shift and mask operation on a 32-bit input word (w). It isolates bits 28 and 29, which typically represent specific channel status flags in the STM32 S/PDIF-RX peripheral. It is a low-level helper used internally by ca_spdifrx_channel_status.

Uso:
Use this macro when you need to parse raw register data from the S/PDIF-RX peripheral to determine channel status. Do not modify the shift or mask values unless the underlying hardware register mapping changes, as incorrect values will result in reading invalid status bits.

`CA_SPDIFRX_PT_B`

File: `caos/drivers/inc/spdifrx.h`

Configures the SPDIFRX peripheral to use Port B for input signals.

Valore: `1U`

Note:
When set to 1U, the driver routes the SPDIFRX input pins to the STM32 Port B GPIO bank. If set to 0U, it typically defaults to Port A or another hardware-specific mapping depending on the MCU variant. This flag must be set before calling ca_spdifrx_init to ensure the hardware abstraction layer configures the correct GPIO alternate functions.

Uso:
Change this value if your hardware design connects the SPDIF input signal to pins located on Port B instead of the default port. Incorrect configuration will result in the peripheral failing to detect a valid signal, causing ca_spdifrx_sync to return an error or timeout.

`CA_SPDIFRX_PT_M`

File: `caos/drivers/inc/spdifrx.h`

Defines the number of SPDIFRX peripheral channels available for concurrent use.

Valore: `2U`

Note:
This constant sets the hardware abstraction layer limit for the SPDIFRX driver. Increasing this value allows more simultaneous SPDIFRX instances to be initialized, but consumes more system memory for driver control blocks. It must be consistent with the actual number of SPDIFRX peripherals present on the specific STM32 MCU variant.

Uso:
Change this value only if the hardware design utilizes multiple SPDIFRX peripherals simultaneously. If you encounter initialization errors when attempting to open a second SPDIFRX channel, verify that this constant is set to at least 2. Do not set this value higher than the number of physical SPDIFRX interfaces supported by your target MCU.

`CA_SPDIFRX_PT_W`

File: `caos/drivers/inc/spdifrx.h`

Configures the SPDIFRX peripheral input pin selection (multiplexing).

Valore: `3U`

Note:
This constant determines which physical GPIO pin is mapped to the SPDIFRX input. The value 3U corresponds to a specific hardware-defined input path on the STM32. Changing this value requires verifying the hardware schematic to ensure the correct pin is connected to the SPDIF source. It is a low-level hardware configuration constant and should not be modified unless the PCB layout changes.

Uso:

Change this value only if you are porting the code to a different hardware revision where the SPDIFRX signal is routed to a different pin. Using an incorrect value will result in the peripheral failing to receive any data (sync errors), as it will be listening to the wrong physical pin.

`CA_SPDIFRX_SAMPLE`

File: `caos/drivers/inc/spdifrx.h`

A macro that performs sign-extension on a 24-bit SPDIF sample to a 32-bit signed integer.

Valore: `((i32)((u32)(w) « 8) » 8)`

Note:
The macro shifts the input word left by 8 bits and then performs an arithmetic right shift by 8 bits. This effectively treats the 24-bit audio data as a signed value, preserving the sign bit (bit 23) across the upper 8 bits of the 32-bit container. It is intended for use with raw data retrieved from the SPDIFRX peripheral.

Uso:
Use this macro when processing raw audio samples from the SPDIFRX driver to ensure that negative values are correctly interpreted by the CPU. Do not use this on data that is already 32-bit or unsigned, as it will corrupt the bit pattern.

`CA_SPDIFRX_U`

File: `caos/drivers/inc/spdifrx.h`

A bitmask flag used to enable the SPDIFRX peripheral in the caOS hardware abstraction layer.

Valore: `(1UL « 26)`

Note:
This constant represents the 26th bit of a configuration register. When enabled, it allows the caOS driver to initialize and access the SPDIFRX hardware. It is typically used as an argument in hardware initialization functions. It has no direct dependencies on other flags but must be used only on STM32 microcontrollers that physically possess an SPDIFRX peripheral.

Uso:
Use this flag when calling initialization routines for the SPDIFRX interface. If you are not using the SPDIFRX peripheral, do not include this flag to save system resources and avoid potential hardware conflicts. Warning: Enabling this on a device without the physical peripheral will result in undefined behavior or a system fault.

`CA_SPDIFRX_V`

File: `caos/drivers/inc/spdifrx.h`

A bitmask flag used to identify or enable the SPDIFRX peripheral in caOS driver configurations.

Valore: `(1UL « 25)`

Note:
This constant represents the 25th bit (value 0x02000000). It is typically used in bitwise operations to select the SPDIFRX hardware instance within the caOS driver initialization or resource

management functions. It does not have dependencies on other flags but must be used in accordance with the specific STM32 hardware support for SPDIFRX.

Uso:
Use this flag when calling ca_spdifrx_init or similar driver-level functions that require a peripheral identifier. Do not modify the bit position unless you are porting the driver to a different hardware architecture where the peripheral mapping has changed.

`CA_SPINLOCK_COOP_INIT`

File: `caos/inc/atomic.h`

Initializes a cooperative spinlock structure to the 'free' state.

Valore: `{ .key = SPINLOCK_FREE }`

Note:
This constant sets the internal key of a spinlock to SPINLOCK_FREE, indicating that the resource is currently available. It is a static initializer used during the declaration of a spinlock variable. It has no dependencies on other flags but must be used only with variables of type ca_spinlock_t.

Uso:
Use this when declaring a new spinlock in your global or static scope to ensure it starts in an unlocked state. Failure to initialize a spinlock before use will lead to undefined behavior, likely causing the system to hang or deadlock immediately upon the first attempt to acquire the lock.

`CA_SPINLOCK_IRQ_INIT`

File: `caos/inc/atomic.h`

Initializes a spinlock structure to a safe, unlocked state with interrupts enabled.

Valore: `{ .key = SPINLOCK_FREE, .irq_state = 0 }`

Note:
This constant sets the spinlock key to 'SPINLOCK_FREE' and resets the interrupt state tracker to 0. It is a static initializer used to define spinlock variables at compile time. It has no dependencies on other flags but must be used only for variables of type 'ca_spinlock_t'.

Uso:
Use this when declaring a new spinlock variable globally or within a struct. You should never manually assign these values at runtime; always use this initializer during declaration to ensure the lock starts in a known, valid state. Failure to initialize correctly will result in undefined behavior, likely causing system deadlocks or race conditions during the first attempt to acquire the lock.

`CA_SPI_XFER_TIMEOUT_MS`

File: `caos/drivers/inc/spi.h`

SPI transfer timeout value in milliseconds for the caOS RTOS

Valore: `1000U`

Note:
This constant defines how long the SPI driver will wait for a transfer operation to complete before timing out. When enabled, it prevents the system from hanging indefinitely during SPI communication. The timeout value directly affects system responsiveness during SPI errors. Typical range is 100-10000ms, with 1000ms being the default. This flag is independent of other SPI configuration flags but requires proper error handling in the calling code.

Uso:
Change this value when experiencing SPI communication timeouts or when working with slow SPI devices. Increase to 2000-5000ms for devices with high latency or when debugging communication issues. Decrease to 100-500ms for time-critical applications or when using fast SPI peripherals. Setting this value too low (below 100ms) may cause false timeouts, while setting it too high (above 10000ms) may delay error detection and system recovery.

`CA_SP_LINK_AAD_MAX`

File: `caos/crypto/inc/secure_pipe_service.h`

Defines the maximum size in bytes for the Additional Authenticated Data (AAD) buffer in the secure pipe service.

Valore: `48U`

Note:
This constant sets the upper limit for the AAD field used during cryptographic operations in the secure pipe. Increasing this value allows for larger metadata headers in encrypted packets but consumes more static memory per secure pipe instance. It must be consistent across all communicating nodes to ensure successful packet decryption.

Uso:
Adjust this value only if you need to include larger authentication tags or metadata in your secure communication protocol. If you encounter 'buffer too small' errors during secure pipe transmission, verify if your AAD payload exceeds 48 bytes. Ensure that the memory allocated for the secure pipe context is sufficient to accommodate this buffer size.

`CA_SP_LINK_DATA_HDR`

File: `caos/crypto/inc/secure_pipe_service.h`

Defines the total size in bytes of the secure pipe link data header.

Valore: `(1U + 8U + 2U)`

Note:
The value (1U + 8U + 2U) = 11 bytes represents the fixed overhead for secure communication packets. This includes fields for protocol versioning, sequence numbers, and integrity metadata. It is a static constant used by the ca_secure_pipe_service to calculate buffer offsets and payload alignment. Modifying this value without updating the corresponding protocol implementation in the secure pipe service will cause packet corruption and communication failure.

Uso:
Do not change this value unless you are modifying the underlying secure pipe protocol specification. It is used internally by ca_secure_pipe_service to ensure that buffers allocated for transmission have enough space for the header. If you encounter 'buffer too small' errors during

secure transmission, ensure your application-level buffers account for this header size plus the actual payload.

`CA_SP_LINK_HELLO_SIZE`

File: `caos/crypto/inc/secure_pipe_service.h`

Defines the fixed byte size of the handshake 'HELLO' message in the caOS secure pipe service.

Valore: `(1U + 1U + 12U + 32U + 16U)`

Note:
The value is calculated as the sum of protocol fields: 1 (version) + 1 (flags) + 12 (nonce) + 32 (public key) + 16 (authentication tag). This constant determines the buffer allocation size for secure pipe link initialization. It must match the peer's expected message length exactly to prevent communication failures.

Uso:
Do not modify this value unless the secure pipe protocol specification is updated. Changing this value without updating the corresponding peer implementation will cause handshake rejection and connection failure. If you need to adjust the buffer size for custom extensions, ensure the peer is updated simultaneously.

`CA_SP_LINK_OVERHEAD_MAX`

File: `caos/crypto/inc/secure_pipe_service.h`

Defines the maximum buffer overhead required for secure pipe link communication.

Valore: `(CA_SP_LINK_DATA_HDR + 15U + 16U)`

Note:
This constant represents the total byte overhead added to the payload, consisting of the link data header (CA_SP_LINK_DATA_HDR), a 15-byte padding/alignment buffer, and a 16-byte authentication tag. It is used to calculate buffer sizes for secure packet transmission. Changing this value affects memory allocation for secure pipe buffers; it must be large enough to accommodate the underlying cryptographic protocol requirements.

Uso:
Adjust this value only if the underlying cryptographic protocol (e.g., AES-GCM or similar) changes its authentication tag size or if the link header structure is modified. Incorrect values will lead to buffer overflows or decryption failures during secure pipe communication. Ensure that any buffer allocated using this constant is large enough to hold the payload plus this overhead.

`CA_SP_LINK_PBKDF2_ITER`

File: `caos/crypto/inc/secure_pipe_service.h`

Defines the number of iterations for the PBKDF2 key derivation function used in the secure pipe service.

Valore: `100000U`

Note:

This constant controls the computational cost of deriving a cryptographic key from a password. Increasing this value significantly improves resistance against brute-force attacks but increases the time required to establish a secure connection. It is a fixed integer value, typically set to 100,000. It is used internally by the ca_secure_pipe_service during the handshake process.

Uso:
Increase this value if security requirements demand higher resistance to offline password cracking, provided the target STM32 hardware has sufficient CPU performance to handle the increased latency. Decrease it only if the connection handshake time exceeds the system's real-time constraints. Warning: setting this too low makes the system vulnerable to brute-force attacks; setting it too high may cause watchdog timeouts during the key derivation process.

`CA_SP_LINK_REKEY_MS`

File: `caos/crypto/inc/secure_pipe_service.h`

Defines the time interval in milliseconds after which the secure pipe session keys are automatically rotated.

Valore: `(60UL * 60UL * 1000UL)`

Note:
This constant determines the frequency of the rekeying process for secure communication channels. It is defined as 3,600,000 ms (1 hour). Reducing this value increases security by limiting the amount of data encrypted with the same key, but increases CPU overhead due to more frequent cryptographic operations. It is used by the secure pipe service to trigger key updates.

Uso:
Adjust this value if your security policy requires more frequent key rotation (e.g., for high-security environments) or less frequent rotation (e.g., to save power or reduce processing load on low-power STM32 devices). Ensure the value is set in milliseconds. Example: #define CA_SP_LINK_REKEY_MS (30UL * 60UL * 1000UL) // Rotate every 30 minutes.

`CA_SP_LINK_REKEY_MSGS`

File: `caos/crypto/inc/secure_pipe_service.h`

Defines the threshold for the number of messages sent over a secure pipe before triggering a key re-negotiation.

Valore: `(1UL « 20)`

Note:
This constant sets the maximum message count limit for a secure session. Once the number of messages sent exceeds this value, the system forces a re-keying process to maintain forward secrecy. The value (1UL « 20) equals 1,048,576 messages. Lowering this value increases security by rotating keys more frequently but introduces higher CPU overhead due to more frequent cryptographic handshakes.

Uso:
Adjust this value if your application requires stricter security policies or if you need to reduce the frequency of re-keying to save power/CPU cycles. Ensure the value is a power of two for optimal performance. If you notice performance drops during long-running sessions, verify if the re-keying frequency is too high for your specific throughput requirements.

CA_SP_LINK_SECRET_MAX

File: caos/crypto/inc/secure_pipe_service.h

Defines the maximum length in bytes allowed for the secret key used in the secure pipe service.

Valore: 16U

Note:
This constant sets the buffer size for cryptographic secrets. Increasing this value consumes more RAM per secure pipe instance. It must be large enough to accommodate the specific key exchange algorithm requirements (e.g., X25519). Typical value is 16U, which is sufficient for standard symmetric keys.

Uso:
Change this value only if you are implementing a custom cryptographic protocol that requires longer keys. If you set this value too low, secure pipe connections will fail to initialize or authenticate. Ensure that the value is consistent across both ends of the communication link.

CA_SP_LINK_SECRET_MIN

File: caos/crypto/inc/secure_pipe_service.h

Defines the minimum required length in bytes for the secret key used in the secure pipe link.

Valore: 8U

Note:
This constant sets the security floor for key exchange operations. Increasing this value enhances security but requires both communication endpoints to support longer keys. It is used during the initialization of the secure pipe service. The default value is 8 bytes. Ensure that any custom key generation logic complies with this minimum to prevent connection rejection.

Uso:
Modify this value only if your security policy requires a higher minimum entropy for link secrets. If you encounter 'link_init' failures, verify that the provided secret meets this length requirement. Do not set this value lower than 8, as it may weaken the cryptographic integrity of the secure pipe.

CA_ST_TH_BLOCKED

File: caos/inc/scheduler.h

A status flag indicating that a specific task is currently in the BLOCKED state.

Valore: 0x01

Note:
This flag is used by the scheduler to identify tasks waiting for an event, semaphore, or timer. When a task is in this state, it is removed from the ready list and will not be scheduled for execution until the blocking condition is resolved. It is a bitmask value typically used in conjunction with other status flags in the task control block (TCB).

Uso:
Do not manually modify this constant value as it is managed internally by the caOS kernel. Use this flag only when writing custom debugging tools or diagnostic functions that need to inspect

the task state via the scheduler's API. Incorrectly interpreting this flag in custom code may lead to false assumptions about task availability.

`CA_ST_TH_DIGEST_ERROR`

File: `caos/inc/scheduler.h`

A specific error flag indicating that a thread's stack or control block digest verification has failed.

Valore: `0x20`

Note:
This flag is set by the kernel's integrity monitor when a memory corruption or stack overflow is detected in a thread's metadata. It is typically used in conjunction with CA_ST_TH_ERROR. Enabling system-wide integrity checks will cause this flag to be raised if the calculated checksum of the thread structure does not match the stored value.

Uso:
Use this flag when debugging system crashes or unexpected task terminations to determine if the cause was memory corruption. Do not manually set this flag in application code; it is a read-only status indicator for the kernel. If you see this flag frequently, investigate your stack sizes and check for buffer overflows in your task functions.

`CA_ST_TH_INIT`

File: `caos/inc/scheduler.h`

Defines the initial stack pattern value used to fill task stacks during initialization.

Valore: `0x80`

Note:
This constant sets the byte value written to the entire stack memory area when a task is created. It is primarily used by the stack overflow detection mechanism to identify how much of the stack has been utilized. Typical values are 0x80 or 0xCC. Changing this does not affect system performance, but it is critical for debugging stack usage via the caOS stack watermarking tools.

Uso:
Change this value if you need to distinguish between uninitialized stack memory and actual data during debugging. If you notice that your stack monitoring tools are reporting incorrect usage, ensure this value matches the pattern expected by your specific debugging configuration. Do not set this to 0x00, as it makes it impossible to differentiate between empty stack space and valid null-terminated strings or zero-initialized variables.

`CA_ST_TH_REMOVE`

File: `caos/inc/scheduler.h`

A bitmask flag used to identify and remove a specific thread from the scheduler's active task list.

Valore: `0x08`

Note:

This flag is typically used within the internal task control block (TCB) status field. When set, the scheduler ignores the thread during context switching. It is often used in conjunction with other status flags like CA_ST_TH_SUSPEND. It must not be manually toggled by application code unless using the provided caOS API functions to avoid memory corruption or scheduler deadlocks.

Uso:
Change or use this flag when implementing custom thread management or debugging tools that require dynamic removal of tasks from the execution cycle. Incorrect usage can lead to a system hang if the scheduler attempts to context switch into a thread that has been improperly flagged as removed.

`CA_ST_TH_RUN`

File: `caos/inc/scheduler.h`

A bitmask flag representing the 'Running' state of a thread within the caOS scheduler.

Valore: `0x40`

Note:
This constant is used by the kernel to identify which thread is currently occupying the CPU. When this bit is set in a thread's status register, the scheduler recognizes it as the active task. It is typically used in conjunction with other status flags (like Ready, Blocked, or Suspended) and should not be manually modified by application code to avoid corrupting the scheduler's internal state.

Uso:
This value is primarily used for debugging purposes or when implementing custom system monitors to check if a specific thread is currently executing. Do not change this value unless you are modifying the core kernel source code, as it is hardcoded into the scheduler's state machine logic.

`CA_ST_TH_RUNNING`

File: `caos/inc/scheduler.h`

A bitmask constant representing the internal state of a task when it is currently executing on the CPU.

Valore: `0xC0`

Note:
This value is used by the caOS kernel scheduler to identify active tasks in the task control block (TCB). It is a read-only status flag for the system. Modifying this value manually will corrupt the scheduler's ability to track task execution, leading to kernel panics or system hangs. It is typically used in debugging tools or status monitoring functions to filter tasks that are currently running.

Uso:
Do not modify this value. It is intended for internal kernel use only. If you are writing a custom task monitor or a system diagnostic tool, you may use this constant to compare against the task state field to determine if a specific task is currently occupying the processor.

`CA_ST_TH_SIG_WDT`

File: `caos/inc/scheduler.h`

Defines the bitmask flag used to signal that a thread has triggered a watchdog timeout event.

Valore: `0x200`

Note:
This constant represents a specific bit in the thread status register. When enabled, the scheduler identifies that the thread has exceeded its execution time budget. It depends on the system watchdog timer being active and the scheduler's monitoring task being enabled. The value 0x200 corresponds to the 9th bit position in the status word.

Uso:
Use this flag when debugging thread starvation or infinite loops within a specific task. You should change this value only if you are remapping the internal status register bitmask for the entire RTOS kernel. Warning: modifying this value incorrectly will cause the scheduler to misinterpret thread states, leading to system instability or silent task failures.

`CA_ST_TH_SLEEP`

File: `caos/inc/scheduler.h`

A bitmask flag representing the thread state where a task is currently blocked or waiting for a timer/event.

Valore: `0x04`

Note:
This flag is used by the scheduler to identify threads that are not ready to run. When a thread is in this state, the scheduler skips it during the context switch process. It is a core internal state bit and should not be manually modified by application code. It is typically used in conjunction with other state flags like CA_ST_TH_READY.

Uso:
Do not change this value as it is a hardcoded internal state identifier for the caOS kernel. If you are debugging and see this flag in the thread control block (TCB), it confirms the thread is correctly suspended and waiting for a resource or delay to expire.

`CA_ST_TH_STACK_OVERFLOW`

File: `caos/inc/scheduler.h`

A bitmask flag used to identify if a specific task has exceeded its allocated stack memory limit.

Valore: `0x10`

Note:
When this flag is set in the task status register, the kernel detects a stack overflow condition. It is typically checked by the kernel's memory protection unit (MPU) or software stack-check hooks. This flag is dependent on CA_CFG_STACK_CHECK being enabled. It is a read-only status bit for the user, managed internally by the scheduler.

Uso:

Use this flag during debugging to verify if your task stack sizes are sufficient. If you encounter hard faults or erratic behavior, check if this bit is set in the task control block (TCB). Do not manually set this bit; it is intended for diagnostic purposes to identify memory corruption issues before they crash the system.

`CA_ST_TH_STOP`

File: `caos/inc/scheduler.h`

A status flag indicating that a specific thread has been explicitly stopped or suspended by the scheduler.

Valore: `0x02`

Note:
When this bit is set in the thread control block (TCB), the scheduler will skip this thread during context switching. It is typically used in conjunction with other status flags like CA_ST_TH_READY. If this flag is cleared, the thread becomes eligible for scheduling again, provided it is not blocked by other conditions.

Uso:
Use this flag when you need to temporarily halt a task's execution without deleting it from memory. This is useful for power management or pausing non-essential background tasks. Do not manually modify this flag while the scheduler is running; always use the provided caOS API functions to suspend or resume threads to avoid race conditions or system instability.

`CA_ST_TH_SUSPENDED`

File: `caos/inc/scheduler.h`

A bitmask flag representing the suspended state of a task within the caOS scheduler.

Valore: `0x100`

Note:
When this bit is set in a task's status register, the scheduler will ignore the task during context switching, effectively pausing its execution. This flag is independent of the task's priority level. It is typically used in conjunction with other state flags (like CA_ST_TH_READY) to determine the task's lifecycle. Valid values are strictly defined by the bitmask 0x100.

Uso:
Use this flag when you need to manually pause a background task (e.g., a data logger or communication handler) without deleting it from memory. You should change this value only when interacting directly with the task control block (TCB) status field. Warning: improperly setting this flag on a critical system task can lead to system deadlocks or watchdog resets if the task is responsible for clearing hardware interrupts.

`CA_ST_TH_UNKNOW`

File: `caos/inc/scheduler.h`

Represents the initial state of a task that has been created but not yet assigned a valid execution state by the scheduler.

Valore: `0x00`

Note:
This constant is used as a placeholder in the Task Control Block (TCB). When a task is initialized, its state is set to CA_ST_TH_UNKNOW until the scheduler performs the first context switch or state transition. It is a reserved internal value and should not be manually assigned to a task state variable by the application code.

Uso:
This value is used internally by the caOS kernel during the task creation process. Do not modify this value or use it in your application logic, as it is intended for kernel-level state tracking. Changing this value will likely cause the scheduler to crash or behave unpredictably during task initialization.

`CA_TASK_HANDLE_NONE`

File: `caos/inc/scheduler.h`

A sentinel value representing an invalid or non-existent task handle.

Valore: `0xFFFFFFFFU`

Note:
This constant is used to initialize task handle variables before a task is created or to indicate that a lookup operation failed. It is defined as 0xFFFFFFFFU, which is an unsigned 32-bit integer representing all bits set to high. It is a system-wide constant and should not be modified, as internal kernel functions rely on this specific value to perform null-pointer-like checks for task objects.

Uso:
Use this value to initialize your task handle variables (e.g., TaskHandle_t myTask = CA_TASK_HANDLE_NO to ensure they do not point to valid memory before the task is actually created. Always compare your task handles against this constant after calling task creation functions to verify if the task was successfully initialized. Never assign this value to a running task, as it will cause the scheduler to lose track of the task's control block.

`CA_TCP_GONE`

File: `caos/net/inc/tcp_service.h`

A sentinel value representing an invalid or closed TCP socket handle within the caOS network stack.

Valore: `(-1)`

Note:
This constant is typically used as a return value for functions that fail to open a socket or as a state indicator for a disconnected connection. It is defined as -1 to distinguish it from valid positive integer file descriptors. It does not have direct dependencies on other flags but is used throughout the TCP service layer to validate socket integrity.

Uso:
Use this value when checking the return status of socket creation or connection functions to verify if the operation succeeded. Do not modify this value, as it is a standard convention for error signaling in the caOS network stack. Comparing a socket handle against CA_TCP_GONE is essential to prevent null pointer dereferences or operations on uninitialized network resources.

CA_TCP_HDR_LEN

File: caos/net/inc/net_proto.h

Calculates the length of the TCP header in bytes based on the data offset field.

Valore: ((u16)(((tcp)->data_off » 4U) « 2U))

Note:
This macro extracts the 4-bit data offset field from the TCP header, which represents the number of 32-bit words in the header, and multiplies it by 4 to convert it into bytes. It depends on the 'tcp' pointer being valid and pointing to a structure containing the 'data_off' field. The result is typically between 20 and 60 bytes.

Uso:
Use this macro when parsing incoming TCP packets to determine where the actual payload data begins. Do not modify the calculation logic unless the underlying TCP structure definition changes, as this follows standard RFC 793 header formatting.

CA_TCP_PAYLOAD

File: caos/net/inc/net_proto.h

A macro that calculates the memory address where the TCP payload data begins within a received packet buffer.

Valore: ((u8 *)(tcp) + CA_TCP_HDR_LEN(tcp))

Note:
This constant performs pointer arithmetic by adding the TCP header length to the base address of the TCP segment. It depends on the CA_TCP_HDR_LEN macro, which parses the Data Offset field in the TCP header. It is used exclusively for accessing the application-layer data following the TCP header.

Uso:
Use this macro whenever you need to extract or process the payload of a TCP packet. Do not manually calculate offsets, as TCP headers have variable lengths due to optional fields. Incorrect usage or passing a null pointer to this macro will result in a memory access violation or a hard fault on the STM32.

CA_TCP_RETRY

File: caos/net/inc/tcp_service.h

Defines the maximum number of retransmission attempts for a TCP packet before the connection is considered lost.

Valore: (0)

Note:
When enabled (value > 0), the TCP stack will attempt to resend unacknowledged segments up to the specified count. If set to 0, the stack performs no retries, effectively making the connection highly sensitive to packet loss. This flag depends on the TCP timeout configuration (CA_TCP_TIMEOUT_MS); if the timeout is too short, retries may trigger prematurely.

Uso:

Increase this value if your application experiences frequent connection drops in unstable network environments (e.g., high-latency Wi-Fi or noisy industrial buses). Set to 0 only for low-latency, high-reliability wired connections where you prefer to handle errors at the application layer immediately rather than waiting for the stack to retry.

`CA_TCP_SENT`

File: `caos/net/inc/tcp_service.h`

A flag that enables or disables the transmission confirmation callback mechanism for the TCP stack.

Valore: `( 1)`

Note:
When set to 1, the system triggers a callback function whenever a TCP packet is successfully acknowledged by the remote host. Disabling this (setting to 0) reduces CPU overhead but prevents the application from tracking packet delivery status. This flag depends on the TCP stack being initialized and requires sufficient heap memory for the callback queue.

Uso:
Enable this flag when your application needs to implement flow control or requires confirmation that data has reached the destination. Change this to 0 if you are experiencing high CPU load and do not require delivery verification. Warning: setting this to 0 while the application logic expects delivery confirmation will cause the system to hang or time out.

`CA_TEMP_NO_DTS`

File: `caos/drivers/inc/temp_sensor.h`

A sentinel value representing that the Digital Temperature Sensor (DTS) is not available or failed to provide a valid reading.

Valore: `(-99900)`

Note:
This constant is used by the ca_temp driver to indicate an error state or absence of hardware. It is set to -99900 (representing -999.00 degrees, assuming a fixed-point format with two decimal places). When ca_temp_read returns this value, it signifies that the sensor data is invalid or the sensor is disconnected. It is typically checked against the return value of ca_temp_read or ca_temp_read_dts.

Uso:
Use this constant to validate temperature readings in your application logic. If your code calls ca_temp_read, always compare the result against CA_TEMP_NO_DTS before using the value for control loops or logging. Example: int16_t temp = ca_temp_read(); if (temp == CA_TEMP_NO_DTS) { /* Handle sensor error */ }.

`CA_TFTP_NO_ERROR`

File: `caos/net/inc/tftp_service.h`

A sentinel value representing the absence of a TFTP error condition.

Valore: `0xFFFFU`

Note:
This constant is used within the TFTP service layer to indicate that a packet or operation was processed successfully. It is typically returned by internal parsing or state-machine functions when no protocol-level error (like file not found or access violation) has occurred. It is not a standard TFTP error code defined by RFC 1350, but rather an internal caOS status flag.

Uso:
Use this constant when checking the return status of TFTP operations to verify success. Do not modify this value, as it is hardcoded to 0xFFFFU to avoid collision with standard TFTP error codes (0-7). If a function returns this value, it confirms the operation proceeded without protocol errors.

`CA_THID_API`

File: `caos/inc/scheduler.h`

A legacy identifier for tasks that is now deprecated in favor of task handles.

Valore: `__attribute__((deprecated("a thid is a position: hold a ca_task_handle_t and use the _handle form")))`

Note:
This constant represents an integer-based task ID. Enabling or using this flag is discouraged as the system is transitioning to object-oriented handles (ca_task_handle_t). It has no functional dependency on other flags but is marked for removal in future versions of caOS.

Uso:
Do not use this in new code. If you encounter it in legacy projects, refactor the code to use the corresponding '_handle' API functions instead. Using this constant will trigger compiler warnings, indicating that your code is using an outdated and potentially unsafe method for task identification.

`CA_TH_LEVEL0`

File: `caos/inc/scheduler.h`

Defines the priority level for the lowest-priority background task in the caOS scheduler.

Valore: `1`

Note:
This constant sets the base priority index for system tasks. Increasing this value shifts the priority floor of the scheduler. It must be compatible with the CA_MAX_PRIORITY constant; if set higher than the maximum allowed priority, the scheduler will fail to initialize. Typical values are 0 or 1 depending on whether the system reserves level 0 for the idle task.

Uso:
Change this value when you need to adjust the task hierarchy or reserve specific priority slots for critical system processes. Use caution: setting this too high may starve lower-priority background tasks or cause priority inversion if not carefully balanced with other task priorities.

`CA_TH_LEVEL1`

File: `caos/inc/scheduler.h`

Defines the priority level for the highest-tier real-time tasks in the caOS scheduler.

Valore: `2`

Note:
This constant sets the numerical priority value for Level 1 tasks. Lower numerical values typically represent higher urgency in caOS. Changing this affects task preemption behavior. It must be configured consistently across all task definitions to avoid priority inversion or scheduling conflicts. Valid range is typically 0 to (CA_MAX_PRIORITY - 1).

Uso:
Modify this value when you need to adjust the relative importance of critical system tasks compared to background processes. Increase this value if you need to create more priority tiers above it, or decrease it to reserve higher priority slots for kernel-level operations. Warning: setting this too low may cause critical tasks to be preempted by lower-priority threads, leading to system instability or missed deadlines.

`CA_TH_LEVEL2`

File: `caos/inc/scheduler.h`

Defines the maximum number of priority levels available for tasks in the caOS scheduler.

Valore: `4`

Note:
This constant sets the depth of the priority bitmask. Increasing this value allows for more granular task scheduling but consumes additional RAM for the ready-list structures. It must be a power of 2 or a multiple of 4 depending on the architecture implementation. Typical values range from 4 to 32.

Uso:
Increase this value if your application requires more than 4 distinct priority levels to manage complex task preemption. Decrease it to save memory if your system only uses a few tasks. Setting this value too high without sufficient RAM will cause heap allocation failures or stack overflows during scheduler initialization.

`CA_TH_LEVEL3`

File: `caos/inc/scheduler.h`

Defines the maximum number of priority levels available for tasks in the caOS scheduler.

Valore: `8`

Note:
This constant sets the size of the ready-list array. Increasing this value allows for more granular task prioritization but consumes more RAM for the scheduler's internal structures. It must be a positive integer; typical values range from 4 to 16 depending on the application complexity. Changing this requires a full project rebuild.

Uso:

Increase this value if your application requires more than 8 distinct priority levels to manage task execution order. Decrease it to save memory if your system uses fewer than 8 priorities. Ensure that all task priority assignments remain within the range of 0 to (CA_TH_LEVEL3 - 1) to avoid scheduler errors.

`CA_TH_LEVEL4`

File: `caos/inc/scheduler.h`

Defines the maximum number of priority levels available for tasks in the caOS scheduler.

Valore: `16`

Note:
This constant sets the size of the ready-list array. Increasing this value allows for more granular task prioritization but consumes more RAM for the scheduler's internal structures. It must be a positive integer; typical values range from 8 to 32 depending on the application complexity. Ensure that any task priority assigned in the code does not exceed (CA_TH_LEVEL4 - 1).

Uso:
Increase this value if your application requires more than 16 distinct priority levels to manage complex task scheduling. Decrease it to save memory if your project uses fewer than 16 priority levels. Setting this value too low will cause the scheduler to ignore or fault on tasks assigned higher priority indices.

`CA_TH_LEVEL5`

File: `caos/inc/scheduler.h`

Defines the maximum number of priority levels available for tasks in the caOS scheduler.

Valore: `32`

Note:
This constant sets the upper bound for task priority assignment. Increasing this value expands the scheduler's granularity but consumes more RAM for the ready-list structures. It must be a positive integer, typically a power of two or a multiple of 8 depending on the underlying bitmask implementation. Ensure this value is consistent across all modules to prevent scheduler overflow errors.

Uso:
Increase this value if your application requires more than 32 distinct priority levels to manage complex task preemption. Decrease it to save memory if your system uses fewer than 32 tasks. Warning: setting this value too high without sufficient system memory can lead to heap exhaustion or stack corruption during scheduler initialization.

`CA_TH_LEVEL6`

File: `caos/inc/scheduler.h`

Defines the maximum number of concurrent threads allowed in the caOS scheduler.

Valore: `64`

Note:

This constant sets the size of the internal thread control block (TCB) array. Increasing this value allows for more tasks but consumes more static RAM. It must be greater than or equal to the total number of tasks defined in your application. If set too low, the system will fail to register additional tasks during initialization.

Uso:
Adjust this value when your application requires more or fewer tasks than the current limit. Ensure that the total RAM usage remains within the STM32 memory constraints. Do not set this value excessively high to avoid wasting precious SRAM on unused TCB slots.

`CA_TIMER_INVALID`

File: `caos/inc/sw_timer.h`

A sentinel value representing an uninitialized or invalid software timer handle.

Valore: `0`

Note:
This constant is used as a null-pointer equivalent for timer objects. When a timer is not active or has failed to initialize, the system assigns this value to the handle. It is typically defined as 0, and any function attempting to operate on a timer with this value will return an error code or trigger a safety assertion.

Uso:
Use this constant to check if a timer has been successfully created before attempting to start, stop, or delete it. Always initialize your timer handle variables to CA_TIMER_INVALID upon declaration to prevent accidental operations on uninitialized memory. Do not manually assign 0 to a timer handle; always use this constant for code readability and maintainability.

`CA_TIMER_MAX`

File: `caos/inc/sw_timer.h`

Defines the maximum number of software timers that can be active simultaneously in the system.

Valore: `16`

Note:
This constant determines the size of the static memory pool allocated for software timer control blocks. Increasing this value consumes more RAM (sizeof(ca_timer_t) * CA_TIMER_MAX). It must be set to a value greater than or equal to the total number of timers initialized in your application. If the system attempts to create more timers than this limit, the creation function will return an error code.

Uso:
Increase this value if your application requires more concurrent software timers than the current limit. Decrease it to save RAM if your application uses fewer timers than the default. Ensure that the value is sufficient for all modules in your project, as setting it too low will cause timer initialization failures at runtime.

CA_TIMER_TASK_PRIO

File: caos/inc/sw_timer.h

Defines the priority level for the internal software timer management task in caOS.

Valore: 2

Note:
This constant sets the RTOS task priority for the timer service. A lower numerical value typically indicates a lower priority in caOS. It depends on the total number of available priority levels defined in the kernel. Valid range is usually 0 to (CA_MAX_PRIO - 1).

Uso:
Increase this value if timer callbacks are being delayed by other tasks, or decrease it if timer processing is starving critical system tasks. Ensure this priority is higher than non-critical background tasks but lower than hardware interrupt-driven tasks to maintain system stability.

CA_TIMER_TASK_STACK

File: caos/inc/sw_timer.h

Defines the stack size in bytes allocated for the software timer management task.

Valore: 512

Note:
This constant determines the memory footprint of the internal timer service thread. If set too low, the system will trigger a stack overflow exception during timer callback execution. If set too high, it wastes precious RAM. The default value of 512 bytes is typically sufficient for basic timer operations, but must be increased if timer callbacks perform complex logic, use large local variables, or call nested functions.

Uso:
Increase this value if you encounter hard fault exceptions or stack overflow errors when the software timer task is active. You should only change this if your application-specific timer callbacks are complex or require significant stack space. Always monitor stack usage using the RTOS debug tools after modifying this value to ensure it remains within safe limits.

CA_UDP_MAX_PORTS

File: caos/net/inc/net_cfg.h

Defines the maximum number of concurrent UDP sockets allowed in the system.

Valore: 8U

Note:
This constant determines the size of the internal UDP port allocation table. Increasing this value consumes more RAM for socket control blocks. It must be greater than 0 and is typically constrained by the available heap memory. If this limit is reached, subsequent attempts to open a UDP socket will return an error code.

Uso:
Increase this value if your application requires more simultaneous UDP communication channels (e.g., multiple network services or data streams). Decrease it to save RAM if your application

only uses a few ports. Ensure that the total number of ports does not exceed the system's memory capacity to avoid allocation failures.

`CA_UDP_PAYLOAD`

File: `caos/net/inc/net_proto.h`

A pointer calculation macro that identifies the starting memory address of the UDP payload data within a received frame.

Valore: `((u8 *)(udp) + sizeof(ca_udp_frame_t))`

Note:
This constant calculates the offset by adding the size of the ca_udp_frame_t header structure to the base address of the UDP packet. It is a read-only pointer arithmetic operation. It depends entirely on the structure definition of ca_udp_frame_t; if the header size changes, this macro automatically adjusts. It is used exclusively within the network stack's packet processing layer.

Uso:
Use this macro whenever you need to access the actual data (payload) contained in a UDP packet after the header has been parsed. Do not modify this definition unless you are modifying the underlying network protocol header structure. Incorrect usage or manual pointer arithmetic instead of using this macro can lead to memory corruption or buffer overflows if the offset is calculated incorrectly.

`CA_VLAN_PCP`

File: `caos/net/inc/net_proto.h`

Extracts the 3-bit Priority Code Point (PCP) field from an 802.1Q VLAN Tag Control Information (TCI) field.

Valore: `((ca_ntohs(tci) » 13U) & 0x07U)`

Note:
This macro performs a bitwise shift and mask operation on a 16-bit TCI value to isolate the priority bits (0-7). It depends on the ca_ntohs macro for network-to-host byte order conversion. The result is always an integer between 0 and 7, where 0 is best-effort and 7 is highest priority.

Uso:
Use this when implementing Quality of Service (QoS) logic to determine the traffic class of an incoming Ethernet frame. Change this only if the underlying VLAN header structure changes or if you need to mask different bits. Warning: Ensure the input 'tci' is a valid 16-bit VLAN TCI value, otherwise the extracted priority will be garbage data.

`CA_VLAN_VID`

File: `caos/net/inc/net_proto.h`

Extracts the 12-bit Virtual Local Area Network Identifier (VLAN ID) from a 16-bit Tag Control Information (TCI) field.

Valore: `(ca_ntohs(tci) & 0x0FFFU)`

Note:

This constant performs a bitwise AND operation with 0x0FFF to mask out the priority and drop-eligible bits, leaving only the VID. It depends on the ca_ntohs macro for network-to-host byte order conversion. The valid range is 0 to 4095, where 0 is typically reserved for priority-tagged frames and 4095 is reserved for implementation use.

Uso:
Use this when parsing Ethernet frames that contain 802.1Q VLAN tags. Change this only if you need to access the priority bits (PCP) or the Drop Eligible Indicator (DEI) instead of the VID. Warning: Incorrectly modifying the mask will result in corrupted VLAN identification, causing the network stack to drop valid packets or misroute traffic to the wrong virtual network.

`CA_WAIT_FOREVER`

File: `caos/inc/common.h`

Constant representing infinite wait time for caOS kernel operations

Valore: `0xFFFFFFFFU`

Note:
When used as a timeout parameter, CA_WAIT_FOREVER causes threads to wait indefinitely until the requested resource becomes available. This value is a 32-bit unsigned integer set to maximum value (4294967295). It has no dependencies on other flags but requires proper thread management to avoid system deadlocks. Valid range is 0 to 0xFFFFFFFFU, where 0 means no wait and 0xFFFFFFFFU means infinite wait.

Uso:
Use CA_WAIT_FOREVER when you want a thread to block indefinitely until a condition is met, such as waiting for a semaphore, mutex, or message queue. Common in critical sections where timing is not critical. Warning: excessive use can cause system deadlocks if not properly managed, and may prevent proper system shutdown sequences. Avoid using in time-critical applications where timeout behavior is required.

`CA_WDT_TIMEOUT_MIN_MS`

File: `caos/drivers/inc/wdt.h`

Defines the minimum allowed watchdog timeout duration in milliseconds.

Valore: `1U`

Note:
This constant acts as a safety floor for the watchdog timer configuration. If a requested timeout is lower than this value, the driver will enforce this minimum to prevent accidental system resets due to extremely short watchdog windows. It is typically set to 1ms. It is used by the ca_wdt_init function during hardware initialization.

Uso:
Change this value only if your specific hardware or application requirements demand a higher safety margin for the watchdog timer. Increasing this value prevents the system from being configured with a watchdog timeout that is too aggressive, which could lead to spurious resets if the CPU is momentarily busy. Do not set this to 0, as it may cause undefined behavior in the watchdog reload logic.

`CH_CMD`

File: `caos/inc/cli.h`

Enables the command-line interface (CLI) module within the caOS RTOS.

Valore: `0x01U`

Note:
When set to 0x01U, the system initializes the CLI task and command parser. Disabling it (0x00U) removes the CLI functionality, reducing flash memory usage and freeing up CPU cycles. This flag depends on the UART driver being correctly configured in the hardware abstraction layer.

Uso:
Enable this during the development and debugging phase to interact with the system via a serial terminal. Disable it in production builds to save memory and prevent unauthorized access to system commands. Ensure the UART baud rate and buffer sizes are configured correctly before enabling.

`CH_ERASE`

File: `caos/inc/cli.h`

A bitmask flag used by the CLI module to enable the automatic clearing of the terminal screen buffer upon command execution.

Valore: `0x04U`

Note:
When enabled, the system sends ANSI escape sequences to clear the console before printing output. It depends on the terminal emulator supporting standard ANSI codes. The value 0x04U is a bitmask, meaning it can be combined with other CLI configuration flags using the bitwise OR operator.

Uso:
Enable this flag if you want a clean terminal interface for each command response to improve readability. Disable it if you need to maintain a scrollable history of previous command outputs. Ensure your serial terminal software (e.g., PuTTY, Tera Term) is configured to interpret ANSI escape sequences, otherwise, you will see raw characters like '[2J' printed on your screen.

`CH_ESC`

File: `caos/inc/cli.h`

Defines the ASCII character code for the Escape key used to terminate or cancel command-line interface (CLI) operations.

Valore: `0x08U`

Note:
This constant sets the byte value (0x08U, which corresponds to Backspace/Control-H in standard ASCII) that the CLI parser interprets as an interrupt or exit signal. Changing this value requires the terminal emulator sending the commands to be configured to send the same byte. It is a dependency for the CLI input buffer clearing logic.

Uso:

Modify this value only if your specific terminal emulator or serial console uses a different control character for the 'Escape' or 'Cancel' function. Setting this to an incorrect value will cause the CLI to ignore exit commands, potentially leaving the system stuck in an active command state.

`CH_NONE`

File: `caos/inc/cli.h`

Disables the Command Line Interface (CLI) module in the caOS system.

Valore: `0x00U`

Note:
When set to 0x00U, the CLI functionality is active. If set to a non-zero value, the CLI code is excluded from the build, reducing flash memory usage. This flag is independent of the RTOS kernel but may be required by other debug modules.

Uso:
Set this to 0x01U (or any non-zero value) if your final production firmware does not require serial console interaction to save memory. Keep it at 0x00U during development to allow for system monitoring and runtime debugging via the serial port.

`CH_SUBMIT`

File: `caos/inc/cli.h`

A bitmask flag used by the CLI module to enable the submission of commands via the serial interface.

Valore: `0x02U`

Note:
When this bit is set in the CLI configuration register, the system processes incoming character buffers as executable commands. If disabled, the CLI will ignore input strings. It depends on the CLI_ENABLED flag being set to 1. The value 0x02U specifically targets the submission sub-routine within the command parser.

Uso:
Enable this flag when you need to allow users to trigger system functions or configuration changes via the terminal. Disable it in production builds to prevent unauthorized command execution or to save processing cycles if the CLI is only used for monitoring.

`CH_TYPO`

File: `caos/inc/cli.h`

Defines the ASCII character code used by the CLI to represent a detected input error or mistyped command.

Valore: `0x10U /* printable, but no command uses it:  a mistyped`

Note:
This constant sets the internal error character for the command line interface. It must be a valid, printable ASCII value. If set to a value already used by a valid command, the CLI may trigger false error reports. Typical values are in the range 0x00 to 0x7F.

Uso:
Change this value if the default 0x10 conflicts with your specific terminal emulator or if you need to reserve 0x10 for a functional command. Ensure the value is not a control character that could disrupt the serial communication stream.

`CLI_HISTORY_SIZE`

File: `caos/inc/cli.h`

Defines the maximum number of previous commands stored in the CLI buffer for recall.

Valore: `16U`

Note:
This constant determines the depth of the command history stack. Increasing this value consumes more RAM, as each entry occupies a fixed-size buffer. It is independent of the command string length but directly impacts the number of 'up-arrow' key presses supported. Typical values range from 8 to 32 depending on available heap memory.

Uso:
Increase this value if you need to navigate back through a longer sequence of executed commands during debugging. Decrease it if the system is running low on RAM. Ensure the value is a positive integer; setting it to 0 will disable the history feature entirely.

`CLI_PRINTF_N`

File: `caos/inc/cli.h`

A macro wrapper for the Command Line Interface (CLI) that directs formatted output to the system's standard output stream.

Valore: `ca_stdout_printf_n(__VA_ARGS__)`

Note:
This constant maps the CLI_PRINTF_N macro to the internal ca_stdout_printf_n function. Enabling this allows the CLI to send strings to the UART or console buffer. It depends on the underlying hardware abstraction layer (HAL) being initialized for UART/USB-CDC. If this is undefined or points to a null function, CLI commands will produce no visible output.

Uso:
Change this value if you need to redirect CLI output to a different peripheral (e.g., switching from UART1 to UART2) or if you need to implement a custom logging wrapper for debugging. Warning: Ensure the underlying print function is thread-safe, as multiple tasks may attempt to write to the CLI simultaneously, which could lead to garbled text or system crashes if not protected by a mutex.

`CLI_PUT_N`

File: `caos/inc/cli.h`

A macro that defines the output function used by the caOS Command Line Interface to send strings to the serial console.

Valore: `ca_stdout_put_n(MSG)`

Note:

This constant maps the CLI output to a specific hardware abstraction layer function. Enabling or changing this allows the CLI to redirect its output to different peripherals like UART, ITM, or a memory buffer. It depends on the availability of the underlying 'ca_stdout_put_n' function implementation in your board support package.

Uso:
Change this value when porting the CLI to a new hardware target or when you need to redirect debug logs to a different communication interface. Ensure the function assigned is non-blocking or properly synchronized to prevent system hangs during high-frequency CLI output.

`CLI_QUEUE_DEPTH`

File: `hw/f1xx/f103/objmap.h`
File: `hw/f4xx/f407/objmap.h`
File: `hw/f4xx/f429/objmap.h`
File: `hw/f4xx/f446/objmap.h`
File: `hw/f7xx/f746/objmap.h`
File: `hw/f7xx/f767/objmap.h`
File: `hw/h5xx/h563/objmap.h`
File: `hw/h7xx/h747/objmap.h`
File: `hw/h7xx/h753/objmap.h`

Defines the maximum number of pending command strings the CLI task can buffer before dropping new input.

Valori per MCU:

MCU	Valore
F103	8U
F407	32U
F429	32U
F446	16U
F746	32U
F767	32U
H563	32U
H747	32U
H753	32U

Note:
This constant sets the size of the internal message queue for the Command Line Interface. Increasing this value consumes more RAM (Queue Size * Max Command Length). It is dependent on the available heap memory and the expected burst rate of incoming commands. Typical values range from 8 to 32 depending on the MCU's RAM capacity.

Uso:
Increase this value if you notice 'CLI Buffer Overflow' errors or dropped commands during high-frequency serial communication. Decrease it if the system is running low on RAM. Ensure the value is set according to the specific MCU target to avoid memory exhaustion.

`DEEP_GUARD_MAX`

File: `hw/f4xx/f407/objmap.h`
File: `hw/f4xx/f429/objmap.h`
File: `hw/f4xx/f446/objmap.h`

File: `hw/f7xx/f746/objmap.h`
File: `hw/h7xx/h747/objmap.h`
File: `hw/h7xx/h753/objmap.h`

Defines the maximum size in bytes for the stack overflow protection guard zone.

Valori per MCU:

MCU	Valore
F407	0x2000U
F429	0x2000U
F446	0x2000U
F746	0x2000U
H747	0x8000U
H753	0x8000U

Note:
This constant sets the memory boundary used by the RTOS to detect stack corruption. Increasing this value provides a larger safety buffer for detecting overflows but consumes more RAM. It is hardware-dependent: F4 series MCUs typically use 0x2000U, while high-performance H7 series require 0x8000U due to larger cache and memory alignment requirements.

Uso:
Adjust this value if you encounter HardFaults related to stack overflows or if the system reports memory corruption in task stacks. Do not decrease this value below the manufacturer-recommended default for your specific MCU, as it will disable the RTOS's ability to detect stack violations, potentially leading to silent system crashes.

`DHCP_BIND_TIMEOUT_MS`

File: `os_config.h`

Defines the maximum time in milliseconds the system waits for a DHCP server to assign an IP address.

Valore: `60000U`

Note:
When enabled, the network stack will attempt to acquire an IP for this duration before timing out. If set too low, the device may fail to connect on slow networks. If set too high, the system boot time increases significantly during network failures. Typical values range from 5000 to 60000.

Uso:
Increase this value if the device frequently fails to obtain an IP address in environments with high network latency or slow DHCP servers. Decrease it if you need the device to fall back to a static IP or error state faster during startup.

`DHCP_CLIENT_PORT`

File: `caos/net/inc/net_proto.h`

Defines the UDP port number used by the caOS DHCP client to listen for server responses.

Valore: `68U`

Note:

This constant sets the destination port for DHCP discovery and request packets. It must be set to 68U to comply with the standard DHCP protocol (RFC 2131). Changing this value will prevent the device from communicating with standard DHCP servers, as they are hardcoded to send replies to port 68.

Uso:
This value should generally remain at 68U. Change this only if you are implementing a custom DHCP server or a specialized network simulation environment where the server is configured to listen and respond on a non-standard port.

`DHCP_DISCOVER_RETRY_N`

File: `os_config.h`

Defines the maximum number of DHCP DISCOVER packet retransmission attempts before the network stack reports a failure.

Valore: `3U`

Note:
When enabled, the system will attempt to request an IP address from a DHCP server up to this count. If the server does not respond within the timeout period, the stack stops trying. Increasing this value improves reliability on unstable networks but increases the total time taken to detect a network failure. Typical values range from 3 to 10.

Uso:
Increase this value if your device frequently fails to obtain an IP address in environments with high packet loss or slow DHCP servers. Decrease it if you need the system to fail fast and proceed to a fallback static IP configuration or error state more quickly.

`DHCP_MSG_ACK`

File: `caos/net/inc/net_proto.h`

Defines the numeric identifier for the DHCP Acknowledge (DHCPACK) message type.

Valore: `5U`

Note:
This constant represents the specific opcode used in the DHCP protocol to confirm a client's request for an IP address. It is used by the network stack to filter incoming UDP packets. It must remain set to 5U to comply with the standard DHCP protocol (RFC 2131). Changing this value will cause the network stack to fail to recognize server acknowledgments, resulting in failed IP address assignment.

Uso:
Do not modify this value under normal circumstances. It should only be changed if you are implementing a custom, non-standard DHCP-like protocol or if the network stack is being ported to a system that uses a proprietary message mapping. Incorrect values will prevent the device from completing the network handshake.

`DHCP_MSG_DISCOVER`

File: `caos/net/inc/net_proto.h`

Defines the message type identifier for a DHCP Discover packet used during the initial network address acquisition phase.

Valore: `1U`

Note:
This constant represents the specific opcode (1) sent by the client to broadcast a request for available DHCP servers. It is a fixed protocol value defined by RFC 2131. Changing this value will cause the DHCP server to ignore the client's discovery requests, effectively breaking the network initialization process.

Uso:
This value should never be changed during standard application development. It is used internally by the caOS network stack to construct outgoing DHCP packets. If you are debugging network traffic with a packet sniffer like Wireshark, use this constant to verify that the client is correctly identifying its broadcast messages as 'Discover' packets.

`DHCP_MSG_NAK`

File: `caos/net/inc/net_proto.h`

Defines the DHCP Negative Acknowledgment (NAK) message type identifier used in network communication.

Valore: `6U`

Note:
This constant represents the specific numeric code (6) sent by a DHCP server to inform a client that its requested IP address is invalid or no longer available. It is a protocol-level identifier required for the DHCP state machine to correctly parse incoming UDP packets. It has no configurable range as it is a fixed protocol constant defined by RFC 2131.

Uso:
Do not modify this value unless you are implementing a custom or non-standard DHCP protocol extension. Changing this will cause the caOS network stack to fail to recognize NAK messages from standard DHCP servers, potentially leading to infinite loops or connection timeouts during the IP address acquisition phase.

`DHCP_MSG_OFFER`

File: `caos/net/inc/net_proto.h`

Defines the numeric identifier for a DHCP Offer message type within the network stack.

Valore: `2U`

Note:
This constant represents the specific message type field in a DHCP packet header. It is used by the network parser to distinguish an 'Offer' from other DHCP messages like 'Discover' or 'Request'. It must match the standard DHCP protocol specification (RFC 2131) and should not be modified unless the underlying network stack implementation is being ported to a non-standard protocol.

Uso:
This value is used internally by the caOS network driver to process incoming DHCP packets. You should never change this value during standard application development. Modifying this constant will cause the device to fail to negotiate an IP address with DHCP servers, effectively breaking network connectivity.

`DHCP_MSG_REQUEST`

File: `caos/net/inc/net_proto.h`

Defines the DHCP Request message type identifier used in the network stack.

Valore: `3U`

Note:
This constant represents the numeric code (3) sent in the DHCP message type option field. It is a fixed protocol value defined by RFC 2131. Changing this value will cause the DHCP server to ignore or reject the client's request, effectively breaking the IP address assignment process.

Uso:
This value should never be changed by the developer. It is used internally by the caOS network stack to identify and construct DHCP Request packets. If you are debugging network traffic with a packet sniffer like Wireshark, you can use this constant to verify that the device is correctly identifying its own outgoing request packets.

`DHCP_OFFER_TIMEOUT_MS`

File: `os_config.h`

Defines the maximum time in milliseconds the system waits to receive a DHCP offer from a server.

Valore: `4000U`

Note:
This constant sets the duration of the blocking wait state during the network initialization phase. If the DHCP server does not respond within this window, the request will time out. It depends on the network stack initialization sequence and should be set higher than the typical round-trip time of your local network infrastructure. Valid range is typically 1000U to 10000U.

Uso:
Increase this value if your device is failing to obtain an IP address on slow or congested networks. Decrease it if you need the system to fail fast and proceed to a fallback static IP configuration or error state more quickly. Setting this value too low may cause intermittent connection failures in environments with high network latency.

`DHCP_RENEW_TIMEOUT_MS`

File: `os_config.h`

Defines the duration in milliseconds the system waits for a DHCP server response before attempting a renewal retry.

Valore: `10000U`

Note:

This constant directly impacts the network stack's responsiveness during IP lease renewal. If set too low, the system may trigger unnecessary retries on slow networks; if set too high, it delays recovery from network interruptions. It depends on the underlying network driver's ability to handle asynchronous timeouts. Typical values range from 5000U to 30000U depending on network latency requirements.

Uso:
Increase this value if the device frequently loses its IP address on congested or high-latency networks. Decrease it only if the application requires faster detection of DHCP server failures. Ensure the value is not set lower than the round-trip time of your network infrastructure to avoid constant, failed renewal loops.

`DHCP_REQUEST_TIMEOUT_MS`

File: `os_config.h`

Defines the maximum time in milliseconds the system waits for a DHCP server to respond to a request.

Valore: `4000U`

Note:
This constant sets the duration of the blocking wait state during the network initialization phase. If the DHCP server does not respond within this window, the request is considered failed. It depends on the underlying network stack's timer resolution. Typical values range from 2000 to 10000 depending on network latency and reliability.

Uso:
Increase this value if the device fails to obtain an IP address on slow or congested networks. Decrease it if you need the system to fail fast and proceed to a fallback static IP configuration or error state. Setting this too low may cause intermittent connection failures in high-traffic environments.

`DHCP_RETRY_BACKOFF_MS`

File: `os_config.h`

Defines the time interval in milliseconds to wait before retrying a failed DHCP request.

Valore: `5000U`

Note:
This constant controls the delay between consecutive DHCP discovery attempts. Increasing this value reduces network traffic and power consumption during network outages, while decreasing it speeds up reconnection times. It is dependent on the network stack initialization and typically ranges from 1000U to 10000U.

Uso:
Adjust this value if your device takes too long to reconnect after a network drop or if the DHCP server is overloaded. Use a higher value if you are operating on a battery-powered device to minimize radio activity, or a lower value if rapid network availability is critical for your application.

`DHCP_SERVER_PORT`

File: `caos/net/inc/net_proto.h`

Defines the UDP port number used by the caOS DHCP server to listen for client requests.

Valore: `67U`

Note:
This constant sets the destination port for incoming DHCP discovery and request packets. It must be set to 67U to comply with the standard DHCP protocol (RFC 2131). Changing this value will prevent standard DHCP clients from discovering the server, as they are hardcoded to send requests to port 67.

Uso:
Keep this value at 67U for standard network operations. Only change this if you are implementing a custom, non-standard DHCP-like protocol or if you are running a secondary DHCP server instance on a non-standard port for testing purposes. Incorrect values will result in the server failing to receive any client configuration requests.

`DHCP_TASK_STACK_MIN`

File: `caos/net/inc/dhcp_service.h`

Defines the minimum stack size in bytes allocated for the DHCP client background task.

Valore: `2048U`

Note:
This value sets the memory footprint for the DHCP task. If set too low, the system will trigger a stack overflow exception during network negotiation. It depends on the underlying network stack's buffer requirements and the complexity of the DHCP options being processed. Typical values range from 1024 to 4096 bytes depending on the RTOS heap configuration.

Uso:
Increase this value if the system crashes or exhibits undefined behavior specifically when the device attempts to acquire an IP address. Decrease it only if you are extremely constrained by RAM and have verified via stack watermarking that the task uses significantly less than 2048 bytes.

`DMA_CH_INVALID`

File: `caos/drivers/inc/dma.h`

A sentinel value representing an uninitialized or invalid DMA channel handle.

Valore: `((ca_dma_ch_t){0xFFU, 0xFFU})`

Note:
This constant acts as a null-pointer equivalent for DMA channel structures. It is used to initialize variables before a valid channel is assigned. It has no functional impact on hardware but is critical for logic checks in driver functions. It is a constant structure of type ca_dma_ch_t with both stream and channel fields set to 0xFF.

Uso:
Use this value to initialize DMA channel variables or to reset a channel handle after a transfer is completed or aborted. Always compare your DMA channel variables against this constant

before attempting to start a transfer to prevent accessing invalid hardware registers. Never attempt to pass this value directly to hardware configuration functions, as it will trigger an error or undefined behavior.

`DMA_MAX_CHANNELS`

File: `caos/drivers/inc/dma.h`

Defines the total number of available DMA channels supported by the hardware abstraction layer.

Valore: `24U`

Note:
This constant sets the upper bound for the DMA channel array size in the driver. If set higher than the physical hardware capability, it may lead to out-of-bounds memory access or invalid register addressing. If set lower, additional DMA channels provided by the STM32 silicon will remain inaccessible to the RTOS.

Uso:
Adjust this value only when porting the caOS driver to a different STM32 microcontroller series with a different number of DMA streams or channels. Ensure the value matches the specific datasheet for your MCU. Incorrect values will cause silent failures or hard faults when attempting to initialize or trigger DMA transfers on higher-indexed channels.

`DMA_REQ_NONE`

File: `caos/drivers/inc/dma.h`

A sentinel value representing the absence of a DMA request channel assignment.

Valore: `0xFFFFU`

Note:
When set to 0xFFFFU, the driver interprets this as 'no DMA channel requested'. It is used as a default value in peripheral configuration structures to prevent accidental activation of DMA streams. It has no dependencies on other flags but must be checked by initialization functions to skip DMA setup logic.

Uso:
Use this value when initializing a peripheral structure that supports DMA but does not require it for the current application. Do not use this value if you intend to perform data transfers, as it will cause the driver to ignore the DMA configuration and potentially fall back to polling mode or return an error.

`DNS_CLASS_IN`

File: `caos/net/inc/net_proto.h`

Defines the Internet (IN) class identifier for DNS queries as specified in RFC 1035.

Valore: `1U`

Note:
This constant represents the standard class value for IP-based networks. It is used in the DNS header structure to indicate that the query pertains to the Internet protocol family. It is a fixed

protocol constant and should not be modified, as changing it will cause DNS servers to reject requests as malformed or unsupported.

Uso:
Use this constant when manually constructing DNS packets or filtering incoming DNS traffic. Do not change this value unless you are implementing a non-standard network protocol that specifically requires a different DNS class identifier.

`DNS_FLAGS_QR`

File: `caos/net/inc/net_proto.h`

A bitmask flag used to identify if a DNS packet is a Query (0) or a Response (1).

Valore: `0x8000U`

Note:
When this bit is set (0x8000), the DNS header indicates the packet is a response from a server. When cleared, it indicates a client query. This flag is typically used in conjunction with bitwise AND operations to parse incoming network buffers. It is a standard field defined by RFC 1035 and must not be modified.

Uso:
Do not change this value as it is defined by the DNS protocol specification. Use this constant when writing custom packet parsers or debugging network traffic to verify if a received packet is a valid response to a previous request.

`DNS_FLAGS_RCODE`

File: `caos/net/inc/net_proto.h`

A bitmask used to extract the 4-bit Response Code (RCODE) field from a DNS message header.

Valore: `0x000FU`

Note:
This constant represents the mask 0x000F, which isolates the lower 4 bits of the DNS flags field. It is used to identify the status of a DNS query (e.g., No Error, Format Error, Server Failure). It depends on the DNS header structure defined in the network stack and is typically used in conjunction with bitwise AND operations on the flags field.

Uso:
Use this when parsing incoming DNS responses to determine if the server successfully resolved the domain or returned an error. Do not modify this value unless the underlying DNS protocol implementation changes, as it is strictly defined by RFC 1035. Incorrect values will result in the system misinterpreting DNS server error codes.

`DNS_FLAGS_RD`

File: `caos/net/inc/net_proto.h`

The Recursion Desired (RD) flag bit for DNS query headers.

Valore: `0x0100U`

Note:
When set to 0x0100U, this bit instructs the DNS server to perform a recursive search if it does not have the requested record in its cache. If disabled (0x0000U), the server will only return what it already knows or provide a referral to another server. This flag is typically ORed with other header flags to construct the full DNS request packet.

Uso:
Enable this flag when your device needs to resolve domain names through a standard recursive DNS server (like those provided by ISPs or public DNS providers). Disable it only if you are communicating with an authoritative-only server or if you want to minimize network traffic by preventing the server from performing additional lookups on your behalf.

`DNS_PENDING_MAX`

File: `os_config.h`

Defines the maximum number of concurrent DNS resolution requests allowed in the system queue.

Valore: `4U`

Note:
This constant determines the size of the internal buffer for pending DNS queries. Increasing this value allows more simultaneous lookups but consumes more RAM in the system heap. It is dependent on the network stack being initialized and typically ranges from 1 to 16 depending on available memory.

Uso:
Increase this value if your application frequently experiences 'DNS_QUEUE_FULL' errors when performing multiple network requests simultaneously. Decrease it if the system is running low on memory. Setting this value too high may lead to heap exhaustion during peak network activity.

`DNS_PORT`

File: `caos/net/inc/net_proto.h`

Defines the standard UDP port number used for Domain Name System (DNS) queries.

Valore: `53U`

Note:
This constant sets the destination port for outgoing DNS requests. It is a 16-bit unsigned integer. Changing this value will prevent the device from resolving hostnames unless the remote DNS server is also configured to listen on the new port. It is a dependency for the caOS network stack's resolver module.

Uso:
Change this value only if you are operating within a non-standard network environment or a private network that uses a custom DNS port. Do not change this for standard internet connectivity, as 53 is the globally recognized IANA standard for DNS.

DNS_RCODE_NXDOMAIN

File: caos/net/inc/net_proto.h

A DNS response code indicating that the domain name requested does not exist.

Valore: 3U

Note:
This constant represents the numeric value 3, which is returned by a DNS server when a query fails because the requested hostname cannot be resolved. It is used within the network stack's DNS client module to interpret server responses. It has no configurable range as it is a fixed protocol-defined constant.

Uso:
Use this constant when writing conditional logic to handle DNS lookup failures. Compare the result of your DNS query function against this flag to distinguish between a network timeout and a legitimate 'domain not found' error. Do not modify this value, as it must strictly adhere to the DNS protocol specification (RFC 1035).

DNS_RCODE_OK

File: caos/net/inc/net_proto.h

Represents the success status code for a DNS query response.

Valore: 0U

Note:
This constant is defined as 0U, which is the standard value for 'No Error' in the DNS protocol header. It is used by the caOS network stack to validate incoming DNS packets. If a received packet's RCODE field matches this constant, the system proceeds with parsing the resource records; otherwise, it triggers an error handling routine.

Uso:
This value should not be changed as it is defined by the RFC 1035 standard. It is primarily used in conditional checks when processing network responses. If your application is receiving unexpected DNS errors, check this constant against the RCODE field in the received packet header to debug network configuration or server-side issues.

DNS_RESPONSE_TIMEOUT_MS

File: os_config.h

Defines the maximum time in milliseconds the system waits for a DNS server to respond to a hostname resolution request.

Valore: 3000U

Note:
When enabled, the network stack will block or pend the calling task for up to this duration. If no response is received within this window, the DNS query fails. This value is dependent on the network stack's tick rate and should be set higher than the expected round-trip time of your network environment. Typical values range from 1000ms to 5000ms.

Uso:

Increase this value if your device is operating on high-latency networks (e.g., cellular or congested Wi-Fi) to prevent premature timeouts. Decrease it if you need the application to fail fast and recover quickly when the network is unreachable. Setting this value too low may cause frequent DNS resolution failures, while setting it too high may cause the application to hang unnecessarily during network outages.

DNS_RETRY_N

File: os_config.h

Defines the maximum number of times the DNS client will attempt to resolve a hostname before returning an error.

Valore: 2U

Note:
This constant controls the retry logic in the network stack. Increasing this value allows for more attempts in unstable network environments but increases the total blocking time for the calling task. It depends on the underlying network driver's timeout settings. Typical values range from 1 to 5.

Uso:
Increase this value if your device frequently fails to resolve hostnames due to high packet loss or slow network response times. Decrease it if you need to fail fast to maintain system responsiveness. Setting this too high may cause task starvation or watchdog timeouts if the network is completely unreachable.

DNS_TYPE_A

File: caos/net/inc/net_proto.h

Defines the protocol identifier for IPv4 address resolution requests in the caOS networking stack.

Valore: 1U

Note:
This constant represents the 'A' record type in DNS queries. It is a mandatory value for the DNS client module to resolve hostnames into 32-bit IPv4 addresses. It must be set to 1U to comply with RFC 1035 standards. Changing this value will cause the DNS resolver to request incorrect record types, leading to resolution failures.

Uso:
Use this constant when calling the DNS query function to specify that you are looking for an IPv4 address. Do not modify this value unless you are intentionally implementing a different DNS record type (e.g., AAAA for IPv6 or MX for mail servers) and the underlying network stack supports it.

DUMP_BUFFER_SIZE

File: hw/f4xx/f407/objmap.h
File: hw/f4xx/f429/objmap.h
File: hw/f4xx/f446/objmap.h
File: hw/f7xx/f746/objmap.h

File: `hw/h5xx/h563/objmap.h`
File: `hw/h7xx/h747/objmap.h`
File: `hw/h7xx/h753/objmap.h`

Defines the fixed size in bytes of the internal memory buffer used for system diagnostic dumps and error logging.

Valori per MCU:

MCU	Valore
F407	128
F429	128
F446	128
F746	128
H563	128
H747	128
H753	128

Note:
This constant allocates a static memory block for capturing system state information during a crash or debug event. Increasing this value allows for more detailed log entries but consumes more RAM. It is currently set to 128 bytes across all supported STM32 platforms (F4, F7, H5, H7) to ensure consistent diagnostic output capacity regardless of the specific MCU architecture.

Uso:
Modify this value if you need to capture larger diagnostic payloads or if you are running out of RAM and need to reclaim space. If set too small, diagnostic logs may be truncated, leading to incomplete information during debugging. Ensure the value is a multiple of the word size (4 bytes) for optimal memory alignment on ARM Cortex-M cores.

`DUMP_TASK_ADDR`

File: `caos/inc/scheduler.h`

Enables the inclusion of task entry point addresses in the system dump logs for debugging purposes.

Valore: `0x02`

Note:
When enabled (0x02), the scheduler appends the memory address of each task's function to the diagnostic output. This flag depends on the system's logging subsystem being initialized. It is typically used in conjunction with other diagnostic flags and consumes a small amount of additional memory in the dump buffer.

Uso:
Enable this flag when you need to identify which specific task function is associated with a stack overflow or a task crash during post-mortem analysis. Do not enable this in production builds if memory constraints are extremely tight, as it increases the size of the diagnostic log entries.

`DUMP_TASK_ALL`

File: `caos/inc/scheduler.h`

A bitmask flag used to enable the logging or diagnostic dumping of all active task control blocks (TCBs) within the scheduler.

Valore: `0xFF`

Note:
When enabled, the system will iterate through all registered tasks and output their current state, stack usage, and priority to the debug interface. This flag depends on the system's debug UART or ITM port being initialized. It is typically used as a bitwise mask; setting it to 0xFF ensures all task-related diagnostic modules are active. Disabling it reduces CPU overhead and memory footprint during runtime.

Uso:
Change this value when you need to perform a system-wide health check or debug a task-related crash. Use it during the development and integration phase to monitor task starvation or stack overflows. Warning: Enabling this in production code can significantly degrade real-time performance and increase interrupt latency due to the high volume of serial data transmission.

`DUMP_TASK_HDR`

File: `caos/inc/scheduler.h`

Enables or disables the inclusion of task control block headers in system memory dumps.

Valore: `0x01`

Note:
When set to 0x01, the scheduler appends task metadata (such as stack pointers and state) to crash dumps. Disabling this (0x00) reduces the size of the dump file but makes debugging task-related crashes significantly harder. It depends on the system's logging or debug interface being initialized.

Uso:
Enable this flag during the development and testing phases to facilitate post-mortem debugging of task stack overflows or deadlocks. Disable it in production builds if memory constraints are extremely tight or if you need to minimize the footprint of diagnostic data.

`DUMP_TASK_REG`

File: `caos/inc/scheduler.h`

Enables the inclusion of CPU register snapshots in task dump reports during system crashes.

Valore: `0x04`

Note:
When enabled (non-zero), the scheduler saves the current CPU register state to the task control block upon a fault. This flag requires sufficient stack space to store the register context. It is typically used in conjunction with a debugger or a serial logging utility to perform post-mortem analysis.

Uso:
Enable this flag during the development and debugging phase to identify the exact instruction pointer and register values at the moment of a task crash. Disable it in production builds to save RAM and reduce the overhead of context switching and exception handling.

`DUMP_TASK_STACK`

File: `caos/inc/scheduler.h`

Enables the automatic dumping of task stack memory contents to the debug console upon a task crash or stack overflow event.

Valore: `0x08`

Note:
When enabled (0x08), the kernel triggers a memory dump of the task's stack area to the UART/ITM interface during a fault. This flag depends on the system having a configured debug output stream. Typical values are 0x00 (disabled) or 0x08 (enabled). Enabling this increases the size of the fault handler code and may slightly increase the time spent in the error recovery routine.

Uso:
Enable this flag during the development and debugging phase to identify the root cause of stack overflows or memory corruption. Disable it in production builds to reduce binary size and prevent sensitive data from being printed to the debug interface.

`DUMP_TASK_STATS`

File: `caos/inc/scheduler.h`

Enables the collection and reporting of CPU usage and stack depth statistics for individual tasks.

Valore: `0x10`

Note:
When enabled (0x10), the scheduler tracks execution time and stack high-water marks for each task. This increases memory overhead per task control block (TCB) and adds a small CPU cycle penalty to context switches. It requires the system heap to be initialized and is typically used in conjunction with a debug console or telemetry module.

Uso:
Enable this flag during the development and profiling phase to identify tasks that are starving the CPU or nearing stack overflow. Disable it in production builds to save RAM and improve real-time performance. Ensure that your task stack sizes are large enough to accommodate the extra overhead if this flag is enabled.

`ECDSA_P256_HASH_SIZE`

File: `caos/crypto/inc/ecdsa_p256.h`

Defines the fixed byte length of the hash used in ECDSA P-256 signature operations.

Valore: `32`

Note:
This constant is set to 32 bytes, which corresponds to the 256-bit output size of the SHA-256 algorithm. It is a hard requirement for the ca_ecdsa_p256_sign_hash and ca_ecdsa_p256_verify_hash functions. Changing this value will cause cryptographic mismatches and likely lead to runtime errors or failed signature verifications.

Uso:

Do not modify this value. It is fixed by the ECDSA P-256 standard (FIPS 186-4). If you are implementing a custom signing flow, ensure your input buffer matches this size exactly before calling ca_ecdsa_p256_sign_hash.

ECDSA_P256_PRIVATE_SIZE

File: `caos/crypto/inc/ecdsa_p256.h`

Defines the fixed byte length of a P-256 elliptic curve private key.

Valore: `32`

Note:
This constant is set to 32 bytes, which corresponds to 256 bits. It is a fundamental requirement for the ca_ecdsa_p256_* family of functions. Changing this value will cause buffer overflows or cryptographic failures in any function expecting a standard NIST P-256 key, as the underlying math relies on this specific size.

Uso:
This value should not be changed. It is used to allocate memory buffers for private keys when calling ca_ecdsa_p256_keygen or when passing private keys to signing functions. If you are handling raw key data, use this constant to ensure your buffers are correctly sized: uint8_t my_key[ECDSA_P256_PRIVATE_SIZE];

ECDSA_P256_PUBLIC_SIZE

File: `caos/crypto/inc/ecdsa_p256.h`

Defines the fixed byte length of an ECDSA P-256 public key in the caOS crypto stack.

Valore: `64`

Note:
This constant is set to 64 bytes, representing the concatenation of the X and Y coordinates (32 bytes each) of the elliptic curve point. It is used by ca_ecdsa_p256_verify and related functions to validate buffer sizes. Changing this value will cause buffer overflows or validation failures in the cryptographic subsystem, as the underlying hardware/software implementation expects exactly 64 bytes for uncompressed P-256 public keys.

Uso:
Do not change this value. It is a protocol-defined constant for the NIST P-256 curve. If you are handling public keys, ensure your buffers are allocated with this size using ca_memory_allocate(ECDSA_P256_ to prevent memory corruption.

ECDSA_P256_SIGNATURE_SIZE

File: `caos/crypto/inc/ecdsa_p256.h`

Defines the fixed byte length of an ECDSA P-256 digital signature.

Valore: `64`

Note:
The value 64 represents the concatenation of two 32-byte integers (r and s) that constitute a standard ECDSA signature on the P-256 curve. This constant is used for buffer allocation and

validation checks in cryptographic operations. It is a fixed protocol requirement and should not be modified.

Uso:
Use this constant when allocating buffers for signature storage or when passing length arguments to functions like ca_ecdsa_p256_sign or ca_ecdsa_p256_verify. Do not change this value, as it is mathematically tied to the P-256 curve specification; changing it will cause cryptographic failures.

`ED25519_PUBLIC_SIZE`

File: `caos/crypto/inc/ed25519.h`

Defines the fixed byte length of an Ed25519 public key in the caOS cryptographic library.

Valore: `32`

Note:
This constant is set to 32 bytes, which is the standard size for Ed25519 public keys as per RFC 8032. Changing this value will cause buffer overflows or cryptographic verification failures in any function expecting a standard Ed25519 key. It is a hard-coded dependency for all signing and verification routines within the caOS crypto stack.

Uso:
Do not modify this value unless you are implementing a non-standard variant of the Ed25519 algorithm. It is used to allocate memory for key storage and to define the bounds for array copies. If you encounter 'invalid key size' errors, ensure your input buffer matches this constant exactly.

`ED25519_SEED_SIZE`

File: `caos/crypto/inc/ed25519.h`

Defines the fixed byte length of the seed used for generating Ed25519 cryptographic keys.

Valore: `32`

Note:
This constant is set to 32 bytes (256 bits), which is the standard requirement for the Ed25519 algorithm. Changing this value will cause cryptographic functions to fail or produce invalid keys, as the underlying library expects exactly 32 bytes of entropy. It has no dependencies on other RTOS flags but is critical for all secure communication modules.

Uso:
Do not change this value under normal circumstances. It should only be modified if you are implementing a custom cryptographic wrapper that requires a different input size, though this is highly discouraged as it breaks compatibility with standard Ed25519 implementations. Ensure any buffer allocated for the seed is exactly this size to prevent memory overflows.

`ED25519_SIGNATURE_SIZE`

File: `caos/crypto/inc/ed25519.h`

Defines the fixed byte length of an Ed25519 digital signature.

Valore: `64`

Note:
This constant is set to 64 bytes, which is the standard size for Ed25519 signatures as per RFC 8032. Changing this value will cause cryptographic verification functions to fail or trigger buffer overflows if the underlying memory buffers are not resized accordingly. It is a hard-coded requirement for the Ed25519 algorithm implementation in caOS.

Uso:
Do not modify this value unless you are porting a non-standard variant of the Ed25519 algorithm. It is used to allocate stack or heap memory for signature storage. Ensure that any buffers passed to signing or verification functions are at least this size to prevent memory corruption.

`ENABLE_CLI`

File: `os_config.h`

Enables the Command Line Interface (CLI) module for real-time system debugging and interaction via UART.

Valore: `1`

Note:
When set to 1, the system initializes the CLI task and allocates memory for command buffers. Disabling it (0) reduces flash memory usage and frees up CPU cycles. This flag depends on the UART driver being initialized in the hardware abstraction layer.

Uso:
Enable this during development to inspect task states, memory usage, and system variables in real-time. Disable it in production builds to save memory and prevent unauthorized access to system commands.

`ENABLE_CLI_STDOUT`

File: `caos/inc/cli.h`

Enables or disables the redirection of the Command Line Interface (CLI) output to the standard output (STDOUT) stream.

Valore: `1`

Note:
When set to 1, all CLI responses are routed to the system's default serial output (usually UART). If set to 0, the CLI will not print any feedback to the console. This flag depends on the hardware abstraction layer (HAL) being initialized for UART communication. Valid values are 0 (disabled) or 1 (enabled).

Uso:
Enable this during development to debug system commands and view task status via a serial terminal. Disable it in production builds to reduce CPU overhead and prevent sensitive system information from being exposed through the serial port.

`ENABLE_DBG_UART`

File: `hw/f1xx/f103/objmap.h`
File: `hw/f4xx/f407/objmap.h`
File: `hw/f4xx/f429/objmap.h`

File: `hw/f4xx/f446/objmap.h`
File: `hw/f7xx/f746/objmap.h`
File: `hw/h5xx/h563/objmap.h`
File: `hw/h7xx/h747/objmap.h`
File: `hw/h7xx/h753/objmap.h`

Enables the dedicated debug UART peripheral for system logging and RTOS kernel diagnostics.

Valori per MCU:

MCU	Valore
F103	1
F407	1
F429	1
F446	1
F746	1
H563	1
H747	1
H753	1

Note:
When enabled, the RTOS redirects kernel trace messages and system assertions to the hardware UART port. Disabling this flag saves CPU cycles and reduces power consumption by stopping the UART peripheral and associated DMA/interrupt overhead. It requires the corresponding UART pins to be physically connected to a serial-to-USB adapter on your development board.

Uso:
Enable this during the development and debugging phase to monitor system health and catch runtime errors via a serial terminal. Disable this flag in production builds to minimize binary size and prevent potential security leaks of internal system states through the debug port.

`ENABLE_ITM`

File: `os_config.h`

Enables or disables the Instrumentation Trace Macrocell (ITM) for real-time debugging output.

Valore: `0`

Note:
When set to 1, the RTOS redirects system logs and trace data to the ITM port, allowing high-speed debugging via an SWO pin. When set to 0, this feature is disabled, saving CPU cycles and memory. This flag requires the hardware debugger to support SWO tracing and the corresponding pin to be configured in the MCU.

Uso:
Enable this (set to 1) when you need to monitor task switching or system events in real-time without stopping the CPU. Disable it (set to 0) for production builds to reduce code size and prevent potential timing issues caused by trace overhead. Ensure your debugger is connected to the SWO pin before enabling.

`ENABLE_SENSOR`

File: `os_config.h`

Global master switch to enable or disable the sensor driver subsystem within the caOS kernel.

Valore: `1`

Note:
When set to 1, the kernel initializes sensor-related tasks and hardware abstraction layers. When set to 0, all sensor-related code is excluded from the build, reducing flash memory usage. This flag is a prerequisite for other sensor-specific configurations like SENSOR_POLLING_RATE and SENSOR_I2C_ADDRESS.

Uso:
Set to 1 during standard development to enable sensor data acquisition. Set to 0 if your specific hardware variant does not include sensors or if you need to free up memory space. Ensure this is enabled before attempting to call any sensor API functions, otherwise, the linker will report undefined references.

`ENABLE_STDOUT`

File: `os_config.h`

Enables or disables the redirection of standard output (printf) to the UART peripheral.

Valore: `1`

Note:
When set to 1, the RTOS initializes the hardware UART and routes stdout to it. When set to 0, all printf calls are ignored or redirected to a null device. This flag depends on the UART_PORT_CONFIG being defined in the hardware abstraction layer. Typical values are 0 (production) or 1 (debugging).

Uso:
Enable this during development to view system logs and debug messages via a serial terminal. Disable it before deploying to production to save CPU cycles, reduce power consumption, and prevent potential blocking issues if the UART buffer becomes full.

`ENABLE_UART`

File: `os_config.h`

Enables or disables the hardware abstraction layer for UART communication within the RTOS.

Valore: `0`

Note:
When set to 1, the RTOS initializes UART peripherals and includes driver code in the build. When set to 0, all UART-related functions are excluded, reducing the binary footprint. This flag must be enabled if your application uses serial debugging, logging, or external UART-based sensors.

Uso:

Set to 1 if your project requires serial communication or console logging. Set to 0 to save flash memory and RAM if UART is not used. Warning: disabling this will cause compilation errors if your application code calls any UART-related API functions.

EN_ADC

File: os_config.h

Enables the Analog-to-Digital Converter (ADC) driver subsystem within the caOS RTOS.

Valore: 1

Note:
When set to 1, the kernel initializes ADC hardware abstraction layers and allocates necessary memory for ADC peripheral management. If set to 0, all ADC-related API calls will return an error code or trigger a hard fault. This flag depends on the hardware abstraction layer (HAL) being correctly configured in the STM32CubeMX project settings.

Uso:
Set this to 1 when your application requires reading analog sensors or voltage levels. Disable it (set to 0) to save flash memory and RAM if your project does not use any ADC channels. Ensure that the corresponding ADC clock is enabled in the system clock configuration before enabling this flag.

EN_AES

File: os_config.h

Enables hardware-accelerated AES encryption support within the caOS kernel and middleware.

Valore: 1

Note:
When set to 1, the system initializes the STM32 hardware cryptographic peripheral and links the AES driver to the OS task scheduler. Disabling this (0) reduces the binary footprint and saves RAM by excluding the crypto-library stack. This flag depends on the target MCU having a hardware AES engine; if enabled on a device without one, the system will trigger a hard fault during initialization.

Uso:
Enable this flag when your application requires secure data transmission (e.g., TLS, encrypted storage, or secure boot). Change this to 0 if your project does not use encryption to optimize memory usage. Warning: Ensure the STM32 model supports hardware AES before setting this to 1, otherwise the kernel will fail to boot.

EN_AES_CHANNEL_SERVICE

File: os_config.h

Enables the hardware-accelerated AES encryption service within the caOS kernel.

Valore: 1

Note:

When set to 1, the RTOS initializes the STM32 hardware cryptographic peripheral and exposes the AES API for task-level encryption. Disabling this (0) reduces the kernel memory footprint and removes the dependency on the hardware crypto driver. This flag requires the STM32 hardware to have a built-in AES peripheral. The AES channel service also enables secure key management and hardware-accelerated cryptographic operations that significantly improve performance compared to software implementations.

Uso:
Enable this when your application requires secure data transmission or storage (e.g., TLS, encrypted logging, secure boot). Change this to 0 if you are running on a low-power device without crypto hardware or if you need to save flash memory space. Ensure that the corresponding hardware clock is enabled in the system initialization if this flag is set to 1. Warning: Setting this to 1 on devices without AES hardware will cause kernel initialization failure. Typical use cases include IoT devices requiring secure communication, embedded systems with security requirements, and applications handling sensitive data where performance is critical.

`EN_AES_GCM_SW_TEST`

File: `os_config.h`

Enables the software-based implementation of the AES-GCM cryptographic algorithm for testing purposes.

Valore: `1`

Note:
When set to 1, the system includes the software library for AES-GCM encryption and decryption. If set to 0, the software implementation is excluded to save flash memory. This flag is independent of hardware-accelerated AES modules but may conflict if hardware drivers are improperly initialized. Valid values are 0 (disabled) or 1 (enabled).

Uso:
Enable this flag during the development and validation phase to verify cryptographic logic without relying on specific STM32 hardware peripherals. Disable it in production builds to reduce binary size and memory footprint, especially if hardware acceleration is available or if AES-GCM is not required by the application.

`EN_ARP_SERVICE`

File: `os_config.h`

Enables or disables the Address Resolution Protocol (ARP) service within the caOS network stack.

Valore: `1`

Note:
When set to 1, the RTOS handles ARP requests and responses to map IP addresses to MAC addresses. Disabling this (0) will prevent the device from communicating over Ethernet unless static ARP entries are manually managed. This flag depends on the underlying Ethernet driver being initialized and the TCP/IP stack being active.

Uso:

Enable this (1) for standard network communication where the device needs to discover or respond to other nodes on the local network. Disable it (0) only in highly specialized, static-network environments where you want to minimize memory footprint or prevent the device from responding to network discovery traffic.

`EN_BARRIER`

File: `os_config.h`

Enables or disables the synchronization barrier primitive within the caOS RTOS kernel.

Valore: `1`

Note:
When set to 1, the kernel includes code for barrier objects, allowing multiple tasks to wait until a specified number of threads have reached a synchronization point. Disabling this (0) reduces the total flash memory footprint by removing the barrier management logic. This flag has no direct dependencies on other kernel objects but is required if your application uses barrier-based task coordination.

Uso:
Enable this flag if your application architecture requires multiple tasks to synchronize at a specific execution gate before proceeding. You should disable it if your project is memory-constrained and does not utilize barriers to save code space. Ensure that all tasks intended to use barriers are initialized correctly after enabling this flag to avoid null pointer dereferences.

`EN_BIG_E`

File: `os_config.h`

Configures the system to use Big-Endian byte order for data storage and memory access.

Valore: `1`

Note:
When enabled (1), the system stores the most significant byte at the lowest memory address. When disabled (0), it defaults to Little-Endian. This flag must match the hardware architecture and the endianness of any external communication protocols (like network packets or specific sensors) to prevent data corruption.

Uso:
Change this value only if your specific STM32 hardware configuration or external peripheral communication requires Big-Endian format. Incorrectly setting this flag will cause multi-byte variables (like uint32_t) to be read or written in reverse, leading to critical logic errors and system instability.

`EN_BLOCK_POOL`

File: `os_config.h`

Enables the memory block pool allocator for fixed-size memory management.

Valore: `1`

Note:

When set to 1, the kernel includes the block pool API, allowing for deterministic, fragmentation-free memory allocation. Disabling it (0) removes this functionality to save flash and RAM. It requires the system heap to be initialized if dynamic pool creation is used.

Uso:
Enable this when your application requires high-speed, predictable memory allocation for tasks or message buffers. Use this instead of standard malloc() to prevent heap fragmentation. Ensure sufficient RAM is allocated for the pool size defined in your task configuration.

`EN_BOOT_IMAGE`

File: `os_config.h`

Enables or disables the bootloader image verification and update management features.

Valore: `0`

Note:
When set to 1, the system includes support for bootloader image slots and update verification via the ca_boot_* API. When set to 0, these features are excluded from the build, reducing the binary size. This flag is typically used in conjunction with flash memory partitioning. If enabled, ensure the memory map is correctly defined to accommodate multiple image slots.

Uso:
Set this to 1 only if your application requires remote firmware updates or a secure bootloader mechanism. If you are developing a simple application that does not require field updates, keep this at 0 to save flash space and simplify the boot process. Warning: Enabling this without proper flash layout configuration will cause the system to fail during the boot sequence.

`EN_CAN`

File: `os_config.h`

Enables or disables the Controller Area Network (CAN) peripheral driver support within the RTOS.

Valore: `0`

Note:
When set to 1, the RTOS initializes CAN hardware abstraction layers and allocates memory for CAN message queues. When set to 0, the CAN driver is excluded from the build, reducing flash memory usage. This flag must be enabled if your application uses CAN communication; otherwise, calling CAN-related API functions will result in linker errors or undefined behavior.

Uso:
Set this to 1 only if your hardware design includes a CAN transceiver and your application requires CAN bus communication. Keep it at 0 for projects that do not use CAN to save system resources and minimize the binary footprint. Ensure that the corresponding hardware clock configuration is enabled in the system initialization if this flag is set to 1.

`EN_CLI_ADC`

File: `os_config.h`

Enables the Command Line Interface (CLI) integration for real-time ADC peripheral monitoring and debugging.

Valore: `1`

Note:
When set to 1, the system initializes the ADC command parser, allowing users to read raw values or status via the serial console. This requires the UART driver to be active and consumes additional stack space in the CLI task. If set to 0, the ADC CLI commands are excluded from the binary, reducing flash memory usage.

Uso:
Enable this during the development and testing phase to verify sensor data or troubleshoot ADC hardware issues without needing an external debugger. Disable it in production builds to save memory and prevent unauthorized access to hardware registers via the console.

`EN_CLI_ARP`

File: `os_config.h`

Enables the Command Line Interface (CLI) support for Address Resolution Protocol (ARP) commands.

Valore: `1`

Note:
When set to 1, the system includes code to inspect and manipulate the ARP table via the serial console. Disabling this (0) reduces the firmware binary size and memory footprint. It depends on the network stack being initialized and the CLI module being active.

Uso:
Enable this during the development and debugging phase to verify network connectivity and troubleshoot IP-to-MAC address mapping issues. Disable it in production builds to save flash memory and prevent unauthorized network table manipulation.

`EN_CLI_CALLTO`

File: `os_config.h`

Enables the Command Line Interface (CLI) call-to-function feature for interactive system debugging.

Valore: `1`

Note:
When set to 1, the RTOS includes the command parser module, allowing users to trigger specific system functions via a serial terminal. This increases the binary size (Flash usage) and requires a configured UART peripheral. If set to 0, the CLI code is excluded from the build, saving memory but disabling interactive runtime control.

Uso:
Enable this during the development and integration phase to inspect task states, memory usage, or trigger manual tests without recompiling. Disable it in production builds to save Flash space and prevent unauthorized access to system functions via the serial port.

EN_CLI_CAN

File: os_config.h

Enables the Command Line Interface (CLI) functionality over the CAN bus.

Valore: 1

Note:
When set to 1, the RTOS initializes the CAN peripheral to listen for and respond to diagnostic or control commands. This requires the CAN driver to be configured in the hardware abstraction layer. If set to 0, the CAN-based CLI task is excluded from the build, reducing flash memory usage and freeing up CAN message buffers.

Uso:
Set to 1 during development or field maintenance to allow remote debugging and parameter tuning via CAN. Set to 0 in production firmware if the CAN bus is reserved exclusively for application-specific communication or to prevent unauthorized access to system commands.

EN_CLI_CAPTURE

File: os_config.h

Enables the Command Line Interface (CLI) input capture feature for real-time system debugging.

Valore: 1

Note:
When set to 1, the RTOS allocates a dedicated buffer and interrupt handler to capture serial input commands. Disabling it (0) saves RAM and reduces CPU overhead. This feature depends on the UART driver being initialized and requires sufficient heap space for the command buffer.

Uso:
Enable this during the development and integration phase to allow runtime inspection of system variables and task states via a terminal. Disable it in production builds to minimize memory footprint and prevent unauthorized access to system commands.

EN_CLI_CONFIG

File: os_config.h

Enables or disables the Command Line Interface (CLI) module within the caOS RTOS.

Valore: 1

Note:
When set to 1, the system initializes the CLI task and serial communication buffers, allowing runtime debugging and command execution. Disabling it (0) reduces flash memory usage and RAM overhead by removing the command parser and associated buffers. This flag depends on the UART driver being correctly initialized in the hardware abstraction layer.

Uso:
Set to 1 during development to interact with the system via a terminal for real-time monitoring and debugging. Set to 0 for production builds to save system resources and prevent unauthorized access to system commands. Ensure that the UART baud rate is configured correctly in the hardware settings before enabling this feature.

`EN_CLI_CRC`

File: `os_config.h`

Enables or disables CRC verification for Command Line Interface (CLI) data packets.

Valore: `1`

Note:
When set to 1, the system calculates and validates a checksum for incoming CLI commands to ensure data integrity. If set to 0, the CRC check is bypassed, reducing CPU overhead but increasing the risk of executing corrupted commands. This flag requires the hardware CRC peripheral to be initialized in the system clock configuration.

Uso:
Enable this (1) in production environments or noisy electrical conditions to prevent command corruption. Disable it (0) only during initial development or if the communication link is guaranteed to be error-free, as it saves a small amount of processing time per command.

`EN_CLI_DAC`

File: `os_config.h`

Enables the Command Line Interface (CLI) support for the Digital-to-Analog Converter (DAC) peripheral.

Valore: `1`

Note:
When set to 1, the system initializes the DAC driver and registers CLI commands to control analog output levels. This flag depends on EN_CLI being enabled and requires the hardware DAC peripheral to be configured in the STM32 CubeMX project. Valid values are 0 (disabled) or 1 (enabled).

Uso:
Enable this flag when you need to perform real-time testing or calibration of analog signals via the serial terminal. Change this to 0 if you need to save flash memory or if the DAC pins are being used for other purposes to avoid resource conflicts.

`EN_CLI_DHCP`

File: `os_config.h`

Enables or disables the Command Line Interface (CLI) support for dynamic IP address assignment via DHCP.

Valore: `1`

Note:
When set to 1, the system attempts to acquire an IP address from a network server during startup. If set to 0, the system ignores DHCP and relies on static IP configurations. This flag requires the network stack (LwIP) to be initialized and enabled in the project configuration.

Uso:
Enable this (1) when the device needs to connect to a network with a dynamic router. Disable it (0) for industrial or static environments where a fixed IP is mandatory to prevent connection delays or address conflicts.

EN_CLI_DMA

File: os_config.h

Enables Direct Memory Access (DMA) for the Command Line Interface (CLI) peripheral communication.

Valore: 1

Note:
When set to 1, the CLI uses DMA to handle data transmission and reception, offloading the CPU from byte-by-byte processing. When set to 0, the system reverts to interrupt-driven or polling mode. This flag requires the underlying hardware UART/USART peripheral to have DMA channels configured and available in the STM32 CubeMX project.

Uso:
Enable this when the CLI is experiencing high latency or causing CPU bottlenecks during heavy logging or data transfer. Change to 0 only if you encounter DMA configuration conflicts or if the specific hardware peripheral does not support DMA. Ensure that the DMA buffer sizes are correctly defined in the system memory map to avoid memory corruption.

EN_CLI_DNS

File: os_config.h

Enables or disables the Command Line Interface (CLI) DNS resolution module within the caOS RTOS.

Valore: 1

Note:
When set to 1, the system includes code to resolve hostnames to IP addresses via the CLI. Enabling this increases the firmware binary size and requires a configured network stack (e.g., LwIP). If set to 0, DNS commands will be unavailable, saving flash memory and reducing stack usage.

Uso:
Enable this flag when your application requires the ability to ping or connect to remote servers using domain names (e.g., 'google.com') directly from the serial console. Disable it if you are working on a memory-constrained device or if your application only uses static IP addresses to save resources.

EN_CLI_ECDSA_P256

File: os_config.h

Enables or disables ECDSA P-256 cryptographic support within the Command Line Interface (CLI).

Valore: 0

Note:
When set to 1, the system includes code for ECDSA P-256 signature verification or generation via CLI commands. Setting this to 0 (default) excludes this functionality, reducing the binary size (Flash usage). This flag depends on the availability of the ca_ecdsa_* API set. Typical values are 0 (disabled) or 1 (enabled).

Uso:
Change this to 1 only if your application requires performing ECDSA P-256 cryptographic operations directly from the CLI (e.g., for remote device authentication or signature verification during field maintenance). If your application does not use these specific security features, keep it at 0 to save memory.

`EN_CLI_ED25519`

File: `os_config.h`

Enables the ED25519 cryptographic signature verification module within the caOS Command Line Interface.

Valore: `1`

Note:
When set to 1, the system includes the ED25519 library to verify signed commands or firmware updates. Enabling this increases the binary size by approximately 4KB and requires additional stack space for cryptographic calculations. It depends on the hardware abstraction layer providing a secure random number generator (RNG).

Uso:
Enable this flag when your application requires secure, authenticated remote command execution or signed bootloader verification. Disable it if you are constrained by flash memory or if your project does not require cryptographic security features to save system resources.

`EN_CLI_ETH`

File: `os_config.h`

Enables the Command Line Interface (CLI) over Ethernet for remote system debugging and configuration.

Valore: `1`

Note:
When set to 1, the RTOS initializes the TCP/IP stack and binds a telnet-like server to the CLI task. This requires the network stack (LwIP) to be enabled and configured. Disabling this (0) saves significant RAM and Flash memory by excluding the network driver and CLI processing buffers.

Uso:
Enable this during development to allow remote access to system logs and shell commands without a physical serial cable. Disable it in production builds to reduce the attack surface and free up system resources.

`EN_CLI_FAT`

File: `os_config.h`

Enables the Command Line Interface (CLI) integration with the FAT filesystem module.

Valore: `1`

Note:

When set to 1, the system includes commands to interact with SD cards or flash storage (e.g., ls, cat, rm). Enabling this requires the FATFS library to be initialized and linked in the project. If disabled, the CLI will not recognize file-system-related commands, reducing the binary size.

Uso:
Enable this flag when you need to perform file operations via the serial console for debugging or data logging. Ensure that the hardware abstraction layer for your storage medium (e.g., SPI/SDIO) is correctly configured before enabling, otherwise, the CLI commands will return error codes.

`EN_CLI_FLASH`

File: `os_config.h`

Enables the Command Line Interface (CLI) functionality stored in the device's internal Flash memory.

Valore: `1`

Note:
When set to 1, the system initializes the CLI module and allocates memory for command parsing; when 0, the CLI is disabled, reducing the binary size and RAM usage. This flag depends on the hardware abstraction layer (HAL) flash driver being correctly configured in the project.

Uso:
Enable this during the development and debugging phase to interact with the system via a serial terminal. Disable it in production builds to save memory and prevent unauthorized access to system commands.

`EN_CLI_GPIO`

File: `os_config.h`

Enables the Command Line Interface (CLI) commands for real-time GPIO pin manipulation and status monitoring.

Valore: `1`

Note:
When enabled, the system allocates memory for CLI command handlers related to hardware pins. Disabling this flag removes these commands from the firmware image, reducing the binary size and freeing up memory. It depends on the core CLI module being enabled in the OS configuration.

Uso:
Enable this during the development and debugging phase to toggle pins or check logic levels directly via a serial terminal without recompiling code. Disable it in production builds to save flash memory and prevent unauthorized hardware manipulation via the console.

`EN_CLI_HTTP`

File: `os_config.h`

Enables the Command Line Interface (CLI) access over the HTTP protocol.

Valore: `1`

Note:
When set to 1, the RTOS initializes a web-based terminal interface; when 0, the HTTP server component is excluded from the build to save flash memory and RAM. This flag requires the network stack (EN_NET_STACK) to be enabled. Typical values are 0 or 1.

Uso:
Enable this when you need to debug or configure the system remotely via a web browser. Disable it in production builds to reduce the attack surface and free up system resources. Ensure your network stack is correctly configured before enabling this, otherwise, the system may fail to initialize the network interface.

`EN_CLI_HTTPS`

File: `os_config.h`

Enables or disables the HTTPS support within the command line interface (CLI) service.

Valore: `1`

Note:
When set to 1, the system includes the necessary code to handle HTTPS requests via the CLI. This flag depends on the underlying TLS stack being initialized. Disabling this (0) reduces the firmware binary size and memory footprint by excluding secure HTTP parsing logic. Typical value is 1 for secure remote management or 0 for resource-constrained devices.

Uso:
Change this value to 1 when you need to perform secure remote configuration or firmware updates over the CLI. Set to 0 if you are running out of flash memory or if the device does not require encrypted network communication. Warning: Disabling this will cause any code calling ca_https_service_start to fail or result in a compilation error if the linker removes the associated symbols.

`EN_CLI_I2C`

File: `os_config.h`

Enables the Command Line Interface (CLI) commands for debugging and interacting with I2C peripherals.

Valore: `1`

Note:
When set to 1, the system includes code to scan, read, and write to I2C devices via the serial console. This increases the binary size and requires the I2C driver stack to be initialized. If set to 0, these commands are excluded from the build to save flash memory.

Uso:
Enable this during the development and hardware bring-up phase to verify I2C sensor connectivity without writing custom test code. Disable it in production builds to reduce memory footprint and improve security by preventing unauthorized hardware access via the CLI.

`EN_CLI_IPERF`

File: `os_config.h`

Enables the built-in Command Line Interface (CLI) module for network performance testing using iPerf.

Valore: `1`

Note:
When set to 1, the system includes the iPerf server/client command set in the CLI. This requires the TCP/IP stack (LwIP) to be initialized and active. Disabling it (0) reduces the firmware binary size and frees up memory by removing the network testing overhead.

Uso:
Enable this during the development and network debugging phase to measure throughput and latency. Disable it in production builds to save flash memory and prevent unauthorized network testing commands from being executed.

`EN_CLI_KSTORE`

File: `os_config.h`

Enables the persistent key-value storage module within the caOS Command Line Interface.

Valore: `1`

Note:
When set to 1, the system allocates memory for the KSTORE subsystem and initializes the flash-backed storage driver. Disabling this (0) removes the KSTORE commands from the CLI and frees up flash/RAM resources. This flag depends on the hardware abstraction layer (HAL) flash driver being correctly configured in the board support package.

Uso:
Enable this when you need to save system configuration parameters (like network settings or calibration offsets) that must persist across power cycles. Disable it if your application is memory-constrained or does not require non-volatile storage to save space. Warning: Ensure the flash sector addresses defined in the linker script do not overlap with the application code area when this is enabled.

`EN_CLI_KSVC`

File: `os_config.h`

Enables the Command Line Interface (CLI) kernel service for real-time system debugging and monitoring.

Valore: `1`

Note:
When set to 1, the RTOS initializes a dedicated task to process serial input commands. This requires an active UART peripheral configured in the hardware abstraction layer. Disabling this (0) saves approximately 2KB of Flash and 512 bytes of RAM, but removes the ability to inspect task states or system metrics at runtime.

Uso:

Enable this during the development and integration phases to monitor system health and task execution. Disable it in production builds to reduce memory footprint and eliminate potential security vulnerabilities associated with an exposed command interface.

`EN_CLI_LED`

File: `os_config.h`

Enables or disables the visual feedback LED associated with the Command Line Interface (CLI) activity.

Valore: `1`

Note:
When set to 1, the system toggles a dedicated GPIO pin whenever a command is received or processed via the CLI. This flag depends on the hardware abstraction layer (HAL) having a valid LED pin definition in the board support package. Valid values are 1 (enabled) or 0 (disabled).

Uso:
Enable this during the initial bring-up phase to verify that the serial communication and CLI task are responding to input. Disable it in production builds to reduce CPU overhead and prevent unnecessary GPIO toggling if the LED is not physically present on the custom PCB.

`EN_CLI_LPTIM`

File: `os_config.h`

Enables the Command Line Interface (CLI) functionality using the Low Power Timer (LPTIM) peripheral.

Valore: `1`

Note:
When set to 1, the system initializes the LPTIM to handle CLI timing and input processing. Disabling this (0) removes CLI support, reducing flash usage and power consumption. This flag depends on the hardware LPTIM peripheral being available and configured in the system clock tree.

Uso:
Enable this when you need to debug the system via a serial console while in low-power modes. Change this to 0 if you are running out of memory or if the application does not require a user interface. Ensure the LPTIM clock source is correctly defined in the system initialization, otherwise, the CLI will remain unresponsive.

`EN_CLI_MDNS`

File: `os_config.h`

Enables the Command Line Interface (CLI) support for mDNS service discovery.

Valore: `1`

Note:
When set to 1, the system initializes the mDNS responder module, allowing the device to be discovered on the network via hostname. This requires the network stack (LwIP) to be active and

configured. Disabling this (0) reduces flash memory usage and removes the mDNS background task.

Uso:
Enable this when you need to access the device via a friendly hostname (e.g., 'device.local') instead of a static IP address. Ensure that the network interface is initialized before this flag is enabled, otherwise, the system may hang during the mDNS startup sequence.

`EN_CLI_MODBUS`

File: `os_config.h`

Enables the built-in Command Line Interface (CLI) support for Modbus protocol debugging and configuration.

Valore: `1`

Note:
When set to 1, the RTOS compiles the Modbus command parser into the system image. This increases Flash memory usage and requires an active UART or console peripheral. It depends on EN_CLI being enabled. Valid values are 0 (disabled) or 1 (enabled).

Uso:
Enable this flag during the development and integration phase to monitor Modbus registers and troubleshoot communication issues via the serial console. Disable it in production builds to save Flash memory and improve system security by preventing unauthorized register access.

`EN_CLI_MQTT`

File: `os_config.h`

Enables the Command Line Interface (CLI) integration for MQTT protocol debugging and control.

Valore: `1`

Note:
When set to 1, the system initializes the MQTT command parser, allowing users to send MQTT-related commands via the serial console. This requires the network stack and MQTT client modules to be enabled. Disabling this (0) saves approximately 2KB of Flash memory and reduces RAM usage by removing the command buffer.

Uso:
Enable this during the development and integration phase to monitor MQTT connection status, publish test messages, or subscribe to topics manually. Disable this in production firmware to reduce the attack surface and free up system resources.

`EN_CLI_NET`

File: `os_config.h`

Enables the Command Line Interface (CLI) over network protocols like Telnet or TCP.

Valore: `1`

Note:

When set to 1, the RTOS initializes the network stack and binds a CLI listener to a specific port. Disabling this (0) saves significant RAM and Flash memory by excluding network buffers and command parsing logic. It requires the network stack (e.g., LwIP) to be enabled in the project configuration.

Uso:
Enable this during development to allow remote debugging and system monitoring via a network terminal. Disable it in production firmware to reduce the attack surface and free up system resources.

`EN_CLI_NTP`

File: `os_config.h`

Enables the Command Line Interface (CLI) support for Network Time Protocol (NTP) synchronization commands.

Valore: `1`

Note:
When set to 1, the system includes CLI handlers to manually trigger or query NTP time synchronization. This flag depends on the network stack being initialized and the NTP client module being enabled in the build. Typical values are 0 (disabled) or 1 (enabled).

Uso:
Enable this during the development and debugging phase to verify time synchronization status or force a time update via the serial console. Disable it in production builds to save flash memory and reduce the attack surface of the device.

`EN_CLI_PWR`

File: `os_config.h`

Enables or disables the Command Line Interface (CLI) power management features.

Valore: `1`

Note:
When set to 1, the system allows the CLI to enter low-power sleep modes during idle periods. When set to 0, the CLI remains fully active, preventing the MCU from entering deep sleep states. This flag depends on the hardware abstraction layer (HAL) power management module being initialized.

Uso:
Enable this (1) when developing battery-powered devices to extend runtime. Disable it (0) during initial debugging or if you experience issues with the CLI becoming unresponsive after the system enters sleep mode.

`EN_CLI_QSPI`

File: `os_config.h`

Enables the Command Line Interface (CLI) functionality over the QSPI peripheral.

Valore: `1`

Note:
When set to 1, the system initializes the QSPI driver to handle serial communication commands. This flag depends on the hardware abstraction layer (HAL) being configured for QSPI and sufficient heap memory allocation for the CLI buffer. Typical values are 0 (disabled) or 1 (enabled).

Uso:
Enable this flag when you need to debug or interact with the system via a QSPI-connected terminal. Change this to 0 if you need to save flash memory or if the QSPI pins are required for external memory storage instead of communication. Ensure the baud rate and pin mapping are correctly defined in the board support package before enabling.

`EN_CLI_RNG`

File: `os_config.h`

Enables the Command Line Interface (CLI) random number generator utility functions.

Valore: `1`

Note:
When set to 1, the system includes additional CLI commands to generate and display random values. This requires the hardware random number generator (RNG) peripheral to be initialized in the system clock configuration. If set to 0, the associated CLI commands are excluded from the binary, reducing flash memory usage.

Uso:
Enable this flag during the development and debugging phase if you need to test entropy-based features or generate random test data via the serial console. Disable it in production builds to save memory and prevent unauthorized access to system entropy sources.

`EN_CLI_ROUTE`

File: `os_config.h`

Enables the command-line interface (CLI) routing engine for processing system commands.

Valore: `1`

Note:
When set to 1, the system initializes the CLI parser and routes incoming serial data to registered command handlers. Disabling this (0) removes the CLI task from the scheduler, saving RAM and CPU cycles. It depends on the UART driver being initialized and requires sufficient stack space for the CLI task.

Uso:
Enable this during development to interact with the system via a terminal for debugging, runtime monitoring, or manual task control. Disable it in production builds to reduce the memory footprint and prevent unauthorized access to system commands.

`EN_CLI_RTC`

File: `os_config.h`

Enables the Command Line Interface (CLI) integration for the Real-Time Clock (RTC) peripheral.

Valore: `1`

Note:
When set to 1, the system registers RTC-specific commands (e.g., date/time setting, alarm configuration) into the shell. This requires the RTC peripheral to be initialized in the hardware abstraction layer. If set to 0, the RTC commands are excluded from the binary, reducing flash memory usage.

Uso:
Enable this flag when you need to debug or manually adjust system time via the serial console. Disable it in production builds if the CLI is not required or to save memory. Ensure that the underlying RTC driver is enabled in the system configuration before setting this to 1, otherwise, the CLI commands will return errors.

`EN_CLI_SDIO`

File: `os_config.h`

Enables the Command Line Interface (CLI) communication over the SDIO peripheral.

Valore: `1`

Note:
When set to 1, the RTOS initializes the SDIO hardware driver to route CLI input/output commands. This flag depends on the hardware abstraction layer (HAL) being configured for SDIO and requires sufficient heap memory for the SDIO buffer. Typical values are 0 (disabled) or 1 (enabled).

Uso:
Enable this flag when you need to debug or interact with the system via an SD card interface or high-speed SDIO-based terminal. Change this to 0 if you are not using SDIO to save flash memory and RAM. Warning: Ensure the SDIO pins are not being used by other peripherals before enabling, as this will cause hardware conflicts.

`EN_CLI_SHA256`

File: `os_config.h`

Enables or disables the SHA-256 cryptographic hashing module within the caOS Command Line Interface.

Valore: `1`

Note:
When set to 1, the system includes the SHA-256 algorithm library, increasing the binary size and memory footprint. This flag is a prerequisite for any CLI commands that require file integrity verification or secure password hashing. It depends on the availability of sufficient flash memory and may conflict with memory-constrained builds if not managed alongside other cryptographic modules.

Uso:
Enable this flag when your application requires secure verification of firmware updates, configuration files, or user authentication via the CLI. Disable it if your project does not perform

cryptographic operations to save flash space and reduce the overall memory overhead of the RTOS kernel.

`EN_CLI_SHA512`

File: `os_config.h`

Enables SHA-512 cryptographic hashing support within the caOS Command Line Interface.

Valore: `1`

Note:
When set to 1, the system includes the SHA-512 algorithm library, increasing the binary footprint by approximately 4KB. This flag depends on the core crypto module being enabled. Valid values are 0 (disabled) or 1 (enabled).

Uso:
Enable this flag when the CLI requires secure file integrity verification or password hashing. Disable it if your STM32 device has limited Flash memory and SHA-512 is not required for your specific application to save space.

`EN_CLI_SPI`

File: `os_config.h`

Enables the Command Line Interface (CLI) support for SPI peripheral debugging and configuration.

Valore: `1`

Note:
When set to 1, the system initializes the SPI driver hooks for the CLI module. This requires the main CLI module (EN_CLI) to be enabled. If set to 0, all SPI-related diagnostic commands are excluded from the binary, reducing flash memory usage.

Uso:
Enable this flag during the development and integration phase to allow real-time monitoring and manual testing of SPI bus transactions via the serial console. Disable it in production builds to save memory and prevent unauthorized access to hardware registers.

`EN_CLI_SSH`

File: `os_config.h`

Enables or disables the integrated SSH server for the Command Line Interface (CLI) over network.

Valore: `1`

Note:
When set to 1, the RTOS initializes the SSH daemon and binds it to the network stack; when 0, the service is excluded from the build to save flash memory and RAM. This flag requires the network stack (e.g., lwIP) to be enabled and configured in the project settings.

Uso:

Set to 1 during development or remote maintenance to allow secure shell access to the system console. Set to 0 for production builds where network security is a concern or to reduce the binary footprint if the CLI is only accessed via UART.

`EN_CLI_SYSLOG`

File: `os_config.h`

Enables the Command Line Interface (CLI) system logging feature for real-time diagnostic output.

Valore: `1`

Note:
When set to 1, the RTOS kernel routes internal system events and error logs to the registered CLI output stream. Disabling this (0) reduces CPU overhead and memory usage by preventing string formatting and buffer allocation for logs. This flag depends on the CLI module being initialized in the system startup sequence.

Uso:
Enable this during the development and debugging phase to monitor task switching, stack overflows, or kernel errors via a serial terminal. Disable it in production builds to save flash memory and improve execution performance, as logging operations can introduce latency in time-critical tasks.

`EN_CLI_TCP`

File: `os_config.h`

Enables the Command Line Interface (CLI) over a TCP/IP network connection.

Valore: `1`

Note:
When set to 1, the RTOS initializes a TCP server socket that allows remote access to the system shell. This requires the network stack (LwIP) to be initialized and configured. Disabling this (0) saves memory and reduces CPU overhead by removing the TCP listener task.

Uso:
Enable this during development to debug the system remotely without a physical serial cable. Disable it in production builds to reduce the attack surface and free up system resources.

`EN_CLI_TFTP`

File: `os_config.h`

Enables or disables the built-in TFTP server functionality within the caOS Command Line Interface.

Valore: `1`

Note:
When set to 1, the system initializes the TFTP stack to allow remote file transfers over the network. This flag depends on the network stack being active (e.g., LwIP). Disabling it (0) reduces the firmware binary size and frees up RAM used by the TFTP buffer. Valid values are 0 (disabled) or 1 (enabled).

Uso:
Enable this flag when you need to perform remote firmware updates or configuration file uploads/downloads via TFTP. Disable it in production builds if the feature is not required to save memory and reduce the attack surface. Ensure your network interface is correctly initialized before attempting to use the TFTP service.

`EN_CLI_TIMER`

File: `os_config.h`

Enables the internal system timer used for measuring command execution time within the Command Line Interface.

Valore: `1`

Note:
When set to 1, the CLI will track and display the duration of executed commands. This requires the system tick timer to be active. If set to 0, the timing feature is disabled, reducing the memory footprint of the CLI module slightly.

Uso:
Enable this flag during the development and debugging phase to profile command performance or identify bottlenecks in CLI-based tasks. Disable it in production builds to save minor CPU cycles and memory if command timing is not required by the end user.

`EN_CLI_TRACE`

File: `os_config.h`

Enables the Command Line Interface (CLI) diagnostic tracing feature for real-time system monitoring.

Valore: `1`

Note:
When set to 1, the RTOS outputs internal kernel events and task state transitions to the serial console. Enabling this increases CPU overhead and memory usage for the trace buffer. It requires the UART peripheral to be initialized and is typically disabled in production builds to save resources.

Uso:
Enable this flag during the debugging phase to track task switching, interrupt latency, or kernel errors. Disable it before final deployment to prevent performance degradation and potential buffer overflows in the trace log.

`EN_CLI_UART`

File: `os_config.h`

Enables the Command Line Interface (CLI) functionality over the UART peripheral.

Valore: `1`

Note:
When set to 1, the RTOS initializes the UART driver and spawns a background task to process serial commands. Disabling this (0) saves RAM and CPU cycles by removing the CLI task and

UART interrupt handlers. This flag depends on the UART hardware abstraction layer being correctly configured in the board support package.

Uso:
Enable this during development to interact with the system, debug tasks, or monitor variables in real-time via a terminal. Disable it in production builds to reduce the memory footprint and improve security by preventing unauthorized access to system commands.

`EN_CLI_UDP_SRV`

File: `os_config.h`

Enables or disables the built-in UDP Command Line Interface (CLI) server for remote debugging.

Valore: `1`

Note:
When set to 1, the RTOS initializes a UDP socket listener on a predefined port to accept remote commands. This requires the network stack (LwIP) to be initialized and active. If set to 0, the UDP server task is not created, saving RAM and CPU cycles. It depends on EN_NETWORK_STACK being enabled.

Uso:
Set to 1 during development to allow remote system monitoring and command execution via network tools like Netcat or custom scripts. Set to 0 in production builds to reduce the attack surface and free up system resources. Ensure the network configuration is valid before enabling, otherwise the system may hang during initialization.

`EN_CLI_WDT`

File: `os_config.h`

Enables or disables the Command Line Interface (CLI) watchdog timer monitoring.

Valore: `1`

Note:
When set to 1, the system monitors the CLI task for responsiveness; if the CLI hangs, the watchdog triggers a system reset. This flag depends on the global watchdog driver being initialized. Valid values are 0 (disabled) or 1 (enabled).

Uso:
Enable this during development to detect if CLI commands are causing task starvation or deadlocks. Disable it during production if the CLI is not required or if you need to perform long-running debug operations that might otherwise trigger a false watchdog reset.

`EN_CLI_X25519`

File: `os_config.h`

Enables the X25519 Elliptic Curve Diffie-Hellman key exchange protocol within the caOS CLI module.

Valore: `1`

Note:
When set to 1, the system includes the cryptographic library required for secure key exchange. Enabling this increases the flash memory footprint by approximately 4KB and requires the system to have sufficient heap space for temporary buffers during the handshake. It depends on the underlying hardware acceleration or software math library being configured in the build system.

Uso:
Enable this flag when the device needs to establish encrypted communication channels or perform secure authentication via the command line interface. Disable it if the device does not require secure remote access or if flash memory constraints are critical. Ensure that the system clock is stable, as cryptographic operations are sensitive to timing.

`EN_CRC`

File: `os_config.h`

Enables hardware-based Cyclic Redundancy Check (CRC) verification for system memory and task stacks.

Valore: `1`

Note:
When set to 1, the RTOS performs periodic integrity checks on critical memory areas. This requires the STM32 hardware CRC peripheral to be available and initialized. Disabling this (0) reduces CPU overhead and power consumption but removes the ability to detect silent memory corruption.

Uso:
Enable this during development and in safety-critical applications to detect stack overflows or memory corruption caused by hardware faults or pointer errors. Disable only if you are strictly constrained by CPU cycles or if the hardware CRC unit is reserved for other high-priority application tasks.

`EN_DAC`

File: `os_config.h`

Enables the Digital-to-Analog Converter (DAC) driver support within the caOS RTOS.

Valore: `1`

Note:
When set to 1, the OS initializes the DAC peripheral hardware abstraction layer. If set to 0, the DAC driver is excluded from the build, reducing binary size. This flag depends on the STM32 hardware having a physical DAC peripheral; enabling it on devices without a DAC will cause compilation or runtime errors.

Uso:
Set this to 1 only if your application requires analog signal generation. Disable it (0) to save flash memory and RAM if the DAC is not used in your project. Ensure the corresponding GPIO pins are not being used for other functions when this is enabled.

`EN_DHCP_SERVICE`

File: `os_config.h`

Enables or disables the automatic Dynamic Host Configuration Protocol (DHCP) client service for network connectivity.

Valore: `1`

Note:
When set to 1, the RTOS stack attempts to acquire an IP address from a network router upon startup. If set to 0, the system will rely on a static IP configuration defined in the network settings. This flag requires the TCP/IP stack module to be enabled in the project configuration.

Uso:
Set to 1 when the device needs to operate in a dynamic network environment (like a standard office or home router). Set to 0 if the device requires a fixed IP address for industrial applications or direct peer-to-peer communication. Warning: Disabling this without providing a valid static IP will result in the device being unreachable on the network.

`EN_DMA`

File: `os_config.h`

Enables or disables the Direct Memory Access (DMA) controller support within the caOS RTOS kernel.

Valore: `1`

Note:
When set to 1, the kernel initializes and manages DMA channels for peripheral data transfers. Disabling this (0) forces the CPU to handle all data transfers via interrupts or polling, which increases CPU load. This flag must be enabled if any drivers (like UART, SPI, or ADC) are configured to use DMA mode in the hardware abstraction layer.

Uso:
Enable this (1) to offload data-heavy tasks from the CPU, improving overall system performance and responsiveness. Disable it (0) only for debugging purposes to isolate potential DMA-related memory corruption or interrupt conflicts, or if the specific STM32 hardware variant lacks DMA support.

`EN_DMA2D`

File: `os_config.h`

Enables or disables the hardware acceleration support for the STM32 DMA2D (Chrom-ART) peripheral.

Valore: `0`

Note:
When set to 1, the ca_dma2d_* API functions are compiled and available for use. When set to 0, these functions are disabled to save code space. This flag is a prerequisite for any graphical operations or memory-to-memory copy tasks that utilize the DMA2D hardware. Ensure your specific STM32 target actually contains a DMA2D peripheral before enabling this.

Uso:

Set this to 1 if your application requires high-performance image blending, color format conversion, or fast memory filling. If your project does not perform GUI rendering or heavy image processing, keep it at 0 to reduce the binary footprint. If you attempt to call ca_dma2d_init or other related functions while this is 0, the code will fail to compile.

EN_DNS_SERVICE

File: `os_config.h`

Enables or disables the built-in Domain Name System (DNS) client service within the RTOS network stack.

Valore: `1`

Note:
When set to 1, the system includes the code and memory buffers required to resolve hostnames (e.g., 'example.com') into IP addresses. Disabling this (0) reduces the firmware binary size and saves RAM, but requires the application to use hardcoded IP addresses for all network connections. This flag depends on the network stack being initialized and requires a valid gateway/DNS server configuration in the network settings.

Uso:
Enable this flag when your application needs to connect to external servers or cloud services using domain names rather than static IPs. Disable it if your device operates in a closed network with static addressing to conserve system resources. Warning: If disabled, any function call to a DNS resolution service will return an error or fail to compile.

EN_ECDSA_P256

File: `os_config.h`

Enables support for ECDSA P-256 cryptographic operations in the caOS library.

Valore: `1`

Note:
When set to 1, the system includes the code for ECDSA P-256 signing, verification, and key generation. This increases the binary size (Flash usage) and requires additional stack space during cryptographic operations. It is a prerequisite for secure boot or TLS features that rely on P-256 certificates. If set to 0, all ca_ecdsa_p256_* functions will be unavailable or return an error.

Uso:
Enable this flag when your application needs to perform digital signature verification (e.g., verifying firmware updates or secure network handshakes). Disable it if you are constrained by Flash memory and your project does not require P-256 cryptography, or if you are using a different algorithm like Ed25519.

EN_ECDSA_P256_TEST

File: `os_config.h`

Enables or disables the inclusion of ECDSA P-256 cryptographic test vectors and self-test routines.

Valore: `0`

Note:
When set to 1, the system includes additional code for verifying ECDSA P-256 operations using predefined test vectors. This increases the binary size (Flash usage). It is typically used during development or factory validation. It depends on the availability of the ECDSA P-256 module in the caOS build.

Uso:
Set to 1 only during the debugging or validation phase of the security stack to ensure the ECDSA implementation is functioning correctly. Set to 0 for production builds to save Flash memory and prevent potential side-channel exposure of test routines. Use ca_ecdsa_p256_selftest to trigger the verification if enabled.

`EN_ED25519`

File: `os_config.h`

Enables hardware-accelerated or software-based ED25519 digital signature verification within the RTOS kernel.

Valore: `1`

Note:
When set to 1, the system includes the ED25519 cryptographic library in the build, increasing the binary size and RAM usage. It requires a sufficient stack size for the task performing the verification. If set to 0, all related cryptographic functions will be unavailable, and calls to them will result in a compilation error or a linker failure.

Uso:
Enable this flag when your application needs to verify secure boot signatures, firmware updates, or encrypted communication packets. Only enable it if your STM32 device has enough Flash and RAM to accommodate the library; otherwise, you may encounter stack overflows or memory allocation failures.

`EN_ED25519_TEST`

File: `os_config.h`

Enables the inclusion of the ED25519 cryptographic test suite within the caOS kernel build.

Valore: `1`

Note:
When set to 1, the system compiles and links the ED25519 verification unit tests. This increases the total flash memory footprint significantly. It depends on the core cryptographic library being enabled. Typical values are 0 for production builds and 1 for development or hardware validation phases.

Uso:
Enable this flag only during the bring-up phase or when performing security regression testing on new hardware. Do not enable this in production firmware, as it consumes unnecessary flash space and exposes test entry points that are not required for standard application operation.

`EN_ETH`

File: `os_config.h`

Enables or disables the Ethernet peripheral driver stack within the caOS RTOS.

Valore: `1`

Note:
When set to 1, the system initializes the Ethernet MAC/PHY hardware and includes the TCP/IP stack tasks. When set to 0, the Ethernet driver is excluded from the build, reducing flash and RAM usage. This flag requires the hardware abstraction layer (HAL) for the specific STM32 Ethernet peripheral to be correctly configured in the project settings.

Uso:
Set this to 1 if your application requires network connectivity (e.g., LwIP, web server, or socket communication). Set to 0 if the project does not use networking to save memory resources. Warning: Disabling this while other tasks attempt to call network-related APIs will result in compilation errors or runtime crashes.

`EN_ETH_CHK_HW`

File: `os_config.h`

Enables hardware-level integrity checks for the Ethernet peripheral controller.

Valore: `1`

Note:
When set to 1, the RTOS performs periodic register validation and status monitoring of the Ethernet MAC/DMA hardware. Disabling this (0) reduces CPU overhead but removes automatic detection of hardware hangs or peripheral clock failures. It depends on the Ethernet driver being initialized and requires the hardware abstraction layer (HAL) to support status polling.

Uso:
Enable this during development and production to ensure the network interface remains responsive. Change to 0 only if you are experiencing extreme CPU constraints and have implemented custom, lightweight monitoring, or if the hardware design does not support status register polling.

`EN_ETH_DBG`

File: `os_config.h`

Enables or disables the Ethernet peripheral debugging and diagnostic logging features within the caOS kernel.

Valore: `1`

Note:
When set to 1, the system routes Ethernet driver status messages and packet statistics to the configured debug UART or ITM port. Enabling this increases CPU overhead and memory usage for the logging buffer. It depends on the global 'EN_DEBUG' flag being enabled. Typical values are 0 (disabled) or 1 (enabled).

Uso:

Enable this flag during the development phase when troubleshooting network connectivity issues, dropped packets, or PHY initialization failures. Disable it in production builds to save flash memory, reduce RAM usage, and prevent potential timing issues caused by logging latency.

EN_EVENT

File: `os_config.h`

Enables the event flag management system within the caOS kernel.

Valore: `1`

Note:
When set to 1, the kernel includes code for event group synchronization primitives. Disabling it (0) reduces the kernel binary size and RAM usage by removing event-related structures. This flag is a prerequisite for using os_event_wait and os_event_set functions.

Uso:
Enable this flag if your application requires tasks to synchronize based on multiple event signals or flags. Keep it disabled if your project only uses simple semaphores or mutexes to save memory on resource-constrained STM32 devices.

EN_EXTI

File: `os_config.h`

Enables the caOS external interrupt (EXTI) management layer for handling hardware signals.

Valore: `1`

Note:
When set to 1, the RTOS hooks into the STM32 EXTI controller to manage ISRs; when 0, the RTOS ignores external interrupts, requiring manual implementation. This flag depends on the HAL_EXTI_MODULE_ENABLED definition in the STM32 HAL configuration.

Uso:
Enable this when your application needs to trigger tasks or semaphores based on physical pin transitions (e.g., buttons, sensors). Disable it only if you are implementing custom interrupt handlers outside the RTOS framework to save memory or avoid conflicts with legacy code.

EN_FAT

File: `os_config.h`

Enables the integrated FAT file system support within the caOS kernel.

Valore: `1`

Note:
When set to 1, the kernel includes the FAT driver stack, increasing Flash and RAM usage. It requires the hardware abstraction layer for SD/MMC or SPI-Flash to be configured. If set to 0, all file system related APIs will be disabled, reducing the binary footprint.

Uso:

Enable this flag only when your application needs to read or write files to external storage media. Ensure that the underlying peripheral drivers are initialized before calling any file system functions to avoid hard faults.

`EN_FAT_CRYPT`

File: `os_config.h`

Enables or disables the integrated hardware-accelerated encryption layer for the FAT file system.

Valore: `1`

Note:
When set to 1, the RTOS automatically encrypts/decrypts data blocks during SD card or Flash read/write operations. Enabling this requires the 'EN_CRYPTO_LIB' flag to be active and increases CPU overhead by approximately 5-8%. It is incompatible with standard FAT32 drivers that do not support encrypted sectors.

Uso:
Set to 1 when storing sensitive user data or configuration files on external storage media to ensure data privacy. Disable it (0) if you are experiencing performance bottlenecks during heavy file I/O or if the storage medium is not intended to hold confidential information.

`EN_FAT_SERVICE`

File: `os_config.h`

Enables or disables the integrated FAT file system service layer within caOS.

Valore: `1`

Note:
When set to 1, the RTOS initializes the FAT file system driver and allocates the necessary memory buffers for file operations. Disabling this (0) reduces the kernel's memory footprint and removes dependencies on SD card or Flash storage drivers. This flag must be enabled if your application uses the 'ca_fs_open' or 'ca_fs_write' APIs.

Uso:
Enable this flag only if your project requires reading or writing files to external storage media. If your application is strictly memory-based or uses raw peripheral access, keep this disabled to save RAM and Flash space. Ensure that the hardware abstraction layer (HAL) for your storage interface (e.g., SPI or SDMMC) is correctly configured before enabling this service.

`EN_FLASH`

File: `os_config.h`

Enables or disables the internal Flash memory driver and management services within caOS.

Valore: `1`

Note:
When set to 1, the RTOS initializes the Flash abstraction layer, allowing tasks to perform read/write operations on non-volatile memory. Disabling this (0) reduces the kernel's memory

footprint and removes dependencies on the STM32 HAL Flash drivers. This flag must be enabled if your application uses the caOS File System or persistent configuration storage.

Uso:
Enable this flag when your application requires saving persistent data or logs to the internal Flash. Disable it if you are optimizing for minimal code size or if your hardware design does not require Flash access, as this prevents potential bus contention or accidental memory corruption during runtime.

`EN_FMAC`

File: `os_config.h`

Enables the hardware support for the STM32 Filter Math Accelerator (FMAC).

Valore: `1`

Note:
When set to 1, the caOS kernel initializes and includes the driver for the FMAC peripheral. This allows the system to offload complex mathematical operations like FIR/IIR filtering to dedicated hardware. Disabling this (0) saves code space and memory but forces all filtering operations to be performed by the CPU, which may significantly increase processing time. This flag requires the target STM32 MCU to have an integrated FMAC peripheral.

Uso:
Enable this flag when your application requires high-performance digital signal processing (DSP) or audio filtering. If your project does not perform real-time signal processing, keep this disabled to minimize the memory footprint. Ensure your hardware supports the FMAC peripheral before enabling, otherwise, the system may trigger a hardware fault during initialization.

`EN_HASH`

File: `os_config.h`

Enables the internal hash-based integrity check for task control blocks and system structures.

Valore: `1`

Note:
When set to 1, the kernel calculates a checksum for critical memory regions to detect corruption. Enabling this increases CPU overhead per context switch and requires additional RAM for storage. It depends on the system's memory protection unit (MPU) configuration being active.

Uso:
Enable this during the development and debugging phase to catch stack overflows or illegal memory writes. Disable it in final production builds if the CPU cycles are strictly limited and the system has passed extensive stability testing, as it adds a slight latency to task scheduling.

`EN_HTTPS_SERVICE`

File: `os_config.h`

Enables or disables the HTTPS server functionality in the caOS network stack.

Valore: `1`

Note:
When set to 1, the system compiles and includes the HTTPS service code, allowing secure web communication. This flag depends on the underlying TLS/SSL library support within caOS. Typical values are 0 (disabled) or 1 (enabled). Enabling this increases the binary size and memory footprint significantly due to cryptographic overhead.

Uso:
Set this to 1 only if your application requires secure web services (HTTPS). If your device only uses plain HTTP or does not act as a web server, set this to 0 to save flash memory and RAM. Ensure that your hardware has sufficient resources to handle the computational load of TLS handshakes.

`EN_HTTP_SERVICE`

File: `os_config.h`

Enables or disables the built-in HTTP server service within the caOS RTOS.

Valore: `1`

Note:
When set to 1, the RTOS initializes the network stack and starts the HTTP server task. When set to 0, the service is excluded from the build, reducing flash and RAM usage. This flag requires the TCP/IP stack (e.g., LwIP) to be enabled in the project configuration.

Uso:
Set to 1 if your application requires a web interface for remote monitoring or configuration. Set to 0 if the device does not need network connectivity or if you are optimizing for minimal memory footprint. Ensure that the network interface hardware is initialized before this service starts to avoid runtime errors.

`EN_I2C`

File: `os_config.h`

Enables or disables the I2C communication driver stack within the caOS RTOS.

Valore: `1`

Note:
When set to 1, the I2C peripheral drivers and associated interrupt handlers are compiled into the kernel. When set to 0, the I2C stack is excluded, reducing the total flash memory footprint. This flag is a prerequisite for any I2C-based peripheral drivers (e.g., sensors, EEPROMs) to function.

Uso:
Set this to 1 if your project requires communication with I2C devices. Set to 0 if your hardware design does not use I2C to save memory. Ensure that the corresponding hardware pins are correctly configured in the STM32CubeMX project before enabling this flag.

`EN_IPERF_SERVICE`

File: `os_config.h`

Enables the integrated network performance testing service (iPerf) within the RTOS stack.

Valore: `1`

Note:
When set to 1, the system allocates a dedicated task and memory buffer for iPerf server functionality. This flag depends on the TCP/IP stack being initialized and active. Disabling it (0) saves approximately 4KB of RAM and reduces CPU overhead during network idle states.

Uso:
Enable this during the development and QA phases to benchmark network throughput and verify stack stability. It must be set to 0 in production firmware to prevent unauthorized network testing and to minimize the memory footprint.

`EN_IP_DEFRAG`

File: `os_config.h`

Enables the built-in IP packet fragmentation reassembly module in the caOS network stack.

Valore: `1`

Note:
When enabled (1), the system allocates a buffer pool to reconstruct fragmented IP packets before passing them to the application layer. Disabling (0) saves significant RAM but causes the stack to drop any incoming fragmented packets. This flag depends on the network stack being initialized and requires sufficient heap memory to hold the reassembly buffers.

Uso:
Enable this if your device communicates over networks where MTU sizes vary or packets are frequently fragmented. Disable it to conserve RAM in memory-constrained systems if you are certain that all incoming traffic will arrive as single, non-fragmented packets.

`EN_JPEG`

File: `os_config.h`

Enables or disables the JPEG hardware acceleration and processing library support.

Valore: `0`

Note:
When set to 1, the ca_jpeg_* API functions are compiled into the binary. When set to 0, these functions are excluded, reducing the total flash memory footprint. This flag has no direct dependencies on other OS flags but is required if your application intends to use the ca_jpeg_decode or ca_jpeg_encode functions.

Uso:
Change this value to 1 only if your project requires image processing or compression/decompression of JPEG streams. If your application does not perform image manipulation, keep this at 0 to save valuable flash space. Warning: Enabling this without sufficient memory or hardware support on the target STM32 chip may lead to compilation errors or runtime faults.

`EN_KEYSTORE_SERVICE`

File: `os_config.h`

Enables the secure keystore service for managing cryptographic keys and sensitive credentials within the RTOS.

Valore: `1`

Note:
When set to 1, the system initializes the secure storage driver and allocates memory for the keystore manager. This flag depends on the hardware abstraction layer (HAL) supporting secure flash or EEPROM access. If disabled (0), any calls to keystore APIs will return an error code or trigger a hard fault.

Uso:
Enable this flag when your application requires encrypted storage for keys, certificates, or sensitive configuration data. Change this to 0 only if your project does not use cryptography or if you are optimizing for minimal memory footprint on devices without secure storage hardware. Ensure that the underlying hardware memory protection unit (MPU) is configured correctly before enabling this service.

`EN_KEYS_STORE`

File: `os_config.h`

Enables the internal persistent storage mechanism for system-wide key-value pairs.

Valore: `1`

Note:
When set to 1, the RTOS allocates a dedicated memory block for storing configuration keys. Disabling this (0) removes the storage API from the build, reducing flash and RAM usage. It depends on the underlying HAL flash driver being initialized. Valid values are 0 (disabled) or 1 (enabled).

Uso:
Enable this when your application needs to save settings like calibration data, network credentials, or user preferences across power cycles. Change to 0 only if your application does not require persistent storage to save memory resources. Warning: Ensure the flash memory sector defined for this store is not being used by your application code or bootloader to prevent data corruption.

`EN_LED`

File: `os_config.h`

Enables or disables the built-in LED heartbeat and status indicator functionality within the RTOS.

Valore: `1`

Note:
When set to 1, the RTOS kernel will periodically toggle the designated system LED to indicate that the scheduler is running. If set to 0, the LED driver code is excluded from the build,

reducing flash memory usage. This flag requires the hardware abstraction layer (HAL) to have a valid GPIO pin defined for the system LED in the board support package.

Uso:
Enable this during initial board bring-up or debugging to verify that the RTOS kernel is alive and not stuck in a hard fault. Disable it in production builds to save power and free up the GPIO pin for other application-specific tasks.

`EN_LITTLE_E`

File: `os_config.h`

Defines the system endianness as Little-Endian when set to 1.

Valore: `1`

Note:
When enabled (1), the processor stores the least significant byte at the lowest memory address. This is the native mode for ARM Cortex-M processors used in STM32. Disabling this (0) would force Big-Endian mode, which is generally not supported by standard STM32 hardware and would cause severe data corruption and system crashes.

Uso:
This flag should almost always remain set to 1 for STM32 development. Change this only if you are porting the RTOS to a specific architecture that requires Big-Endian byte ordering. Incorrectly setting this to 0 will result in incorrect integer arithmetic, pointer errors, and failure of communication protocols like SPI or I2C.

`EN_LPTIM`

File: `os_config.h`

Enables the Low-Power Timer (LPTIM) peripheral support within the caOS kernel.

Valore: `1`

Note:
When set to 1, the RTOS utilizes the LPTIM hardware for low-power tick generation and sleep mode wake-ups. Disabling this (0) forces the system to use standard timers, which may increase power consumption during idle states. This flag requires the STM32 LPTIM peripheral to be physically present on the MCU and correctly initialized in the hardware abstraction layer.

Uso:
Enable this flag when designing battery-powered applications that require the system to remain in Stop mode while maintaining timekeeping. Change to 0 only if you are debugging power issues or if the specific STM32 variant lacks an LPTIM peripheral. Incorrect configuration may lead to system hangs or failure to wake from low-power modes.

`EN_LPUART`

File: `os_config.h`

Enables or disables the Low Power UART (LPUART) peripheral driver support within the caOS kernel.

Valore: `0`

Note:
When set to 1, the kernel initializes and manages the LPUART hardware for low-power serial communication. When set to 0, the driver is excluded from the build, reducing flash memory usage. This flag depends on the STM32 hardware having an LPUART peripheral available; enabling it on chips without LPUART will cause compilation errors.

Uso:
Set to 1 if your application requires serial communication while the MCU is in Stop mode to wake up the system. Set to 0 if you are using standard UART peripherals or do not require serial communication, as this saves memory and prevents unnecessary hardware initialization.

`EN_MDNS_SERVICE`

File: `os_config.h`

Enables or disables the built-in mDNS (Multicast DNS) service for network device discovery.

Valore: `1`

Note:
When set to 1, the RTOS initializes a background task to broadcast the device's hostname and IP address on the local network. This requires the TCP/IP stack (e.g., LwIP) to be active and configured. Disabling this (0) reduces memory usage and CPU overhead by removing the mDNS responder task.

Uso:
Enable this when you want your device to be reachable via a friendly hostname (e.g., 'device.local') instead of a static IP address. Disable it if you are operating in a strictly controlled network environment where multicast traffic is prohibited or if you need to save RAM/Flash resources.

`EN_MEASURE`

File: `os_config.h`

Enables the internal CPU load and execution time measurement module for system tasks.

Valore: `1`

Note:
When set to 1, the RTOS tracks task execution time and idle percentage. This requires additional RAM for storage and slightly increases context switch overhead. It depends on the hardware timer configured in the HAL layer. Valid values are 0 (disabled) or 1 (enabled).

Uso:
Enable this during the development and profiling phase to identify bottlenecks or verify task timing requirements. Disable it in the final production firmware to save CPU cycles and reduce memory footprint.

`EN_MESSAGE`

File: `os_config.h`

Enables the inter-task messaging system (mailboxes/queues) within the caOS kernel.

Valore: `1`

Note:
When set to 1, the kernel allocates memory and initializes structures for message passing. Disabling this (0) reduces the static RAM footprint and code size significantly. This flag is a prerequisite for using any OS_Send or OS_Receive functions; if disabled, calling these functions will result in a compile-time error or a kernel panic.

Uso:
Enable this flag when your application requires tasks to exchange data or synchronize via messages. Disable it only in extremely memory-constrained systems where tasks operate in complete isolation to save precious SRAM and Flash space.

`EN_MODBUS_SERVICE`

File: `os_config.h`

Enables or disables the integrated Modbus RTU/TCP communication service within the caOS kernel.

Valore: `1`

Note:
When set to 1, the RTOS allocates memory for the Modbus task and initializes the protocol stack. When set to 0, the service is excluded from the build, reducing Flash and RAM usage. This flag depends on the hardware abstraction layer (HAL) having the necessary UART or Ethernet drivers configured.

Uso:
Set to 1 if your application requires Modbus communication for industrial sensors or PLC integration. Set to 0 to save system resources if the project does not use Modbus. Ensure that the associated communication peripheral is not being used by another task if this flag is enabled to avoid resource conflicts.

`EN_MQTT_SERVICE`

File: `os_config.h`

Enables or disables the integrated MQTT client service within the caOS RTOS stack.

Valore: `1`

Note:
When set to 1, the MQTT task and associated network buffers are initialized during system boot. Disabling this (0) reduces the overall RAM footprint and flash usage by excluding the MQTT protocol stack. This flag depends on the network stack being enabled (e.g., EN_LWIP_STACK must be 1).

Uso:
Set to 1 if your application requires remote telemetry or cloud connectivity via MQTT. Set to 0 if the device operates in a standalone mode or uses a different communication protocol to save system resources. Ensure your heap size is sufficient before enabling, as the MQTT client requires dynamic memory allocation for message buffers.

`EN_NET`

File: `os_config.h`

Enables the integrated networking stack and TCP/IP protocol support within the caOS RTOS.

Valore: `1`

Note:
When set to 1, the kernel initializes the network interface drivers and allocates memory for the socket buffer pool. This flag depends on the hardware abstraction layer (HAL) having Ethernet or Wi-Fi peripherals configured. If set to 0, all network-related tasks and memory allocations are excluded from the build, reducing the total flash and RAM footprint.

Uso:
Enable this flag when your application requires network connectivity (e.g., MQTT, HTTP, or socket communication). Disable it for low-power or resource-constrained applications that do not require connectivity to save memory. Ensure that the network buffer size is also configured in the corresponding header file if this flag is enabled, otherwise, the system may trigger a hard fault during initialization.

`EN_NET_DHCP`

File: `os_config.h`

Enables the Dynamic Host Configuration Protocol (DHCP) client module for automatic IP address assignment.

Valore: `1`

Note:
When set to 1, the RTOS stack attempts to negotiate network settings with a DHCP server upon initialization. If set to 0, the system defaults to a static IP configuration defined in the network settings. This flag requires the TCP/IP stack (e.g., LwIP) to be enabled in the project configuration.

Uso:
Enable this when the device needs to connect to networks where IP addresses are managed by a router or server. Disable it for industrial or point-to-point applications where a fixed, predictable IP address is required to prevent connection delays during boot-up.

`EN_NET_DNS`

File: `os_config.h`

Enables or disables the Domain Name System (DNS) client functionality within the caOS networking stack.

Valore: `1`

Note:
When set to 1, the system includes code to resolve human-readable hostnames (e.g., 'example.com') into IP addresses. When set to 0, the DNS client is excluded from the build, reducing flash memory usage. This flag depends on the TCP/IP stack being enabled; if the network stack is disabled, this flag must also be 0.

Uso:
Enable this (1) if your application needs to connect to remote servers using URLs instead of hardcoded IP addresses. Disable it (0) to save memory if your device only communicates via static IP addresses or if you are performing manual IP resolution.

`EN_NET_FILTER`

File: `os_config.h`

Enables hardware-level packet filtering for network traffic

Valore: `1`

Note:
When set to 1, incoming network packets are filtered at hardware level before reaching the OS network stack, reducing CPU overhead. When set to 0, all packets bypass hardware filtering and go directly to software processing. This flag requires the network hardware to support filtering and depends on proper MAC address configuration. Hardware filtering is typically only available on certain STM32 Ethernet peripherals.

Uso:
Set to 1 when implementing security policies or reducing network processing overhead in production systems. Set to 0 for debugging network issues or when hardware doesn't support filtering. Warning: Setting to 1 on unsupported hardware may cause network initialization failures or packet loss. Only enable if your STM32 device has hardware filtering capabilities.

`EN_NET_MDNS`

File: `os_config.h`

Enables or disables the multicast DNS (mDNS) service discovery protocol within the caOS network stack.

Valore: `1`

Note:
When set to 1, the system announces its hostname and services on the local network, allowing other devices to resolve its IP address via 'hostname.local'. Disabling this (0) reduces memory footprint and CPU overhead by removing the mDNS responder task and associated UDP socket handling. This flag requires the TCP/IP stack to be initialized and typically depends on the EN_NET_UDP flag being enabled.

Uso:
Enable this when you want your device to be discoverable by other network clients without needing a static IP or a central DNS server. Disable it in resource-constrained applications where network discovery is not required or to minimize the attack surface of the device on the local network.

`EN_NET_NTP`

File: `os_config.h`

Enables or disables the Network Time Protocol (NTP) client service within the caOS RTOS stack.

Valore: `1`

Note:
When set to 1, the system initializes the NTP client task to synchronize the internal RTC with external time servers. This flag depends on the network stack being active (EN_NET_STACK must be 1). If disabled, the system will not attempt to fetch network time, and the RTC will rely solely on manual setting or hardware battery backup.

Uso:
Enable this when your application requires accurate, synchronized timestamps for logging or scheduling. Change to 0 if your device operates in an isolated network or if you want to reduce memory footprint and network traffic. Warning: Ensure your network gateway and DNS settings are correctly configured before enabling, otherwise the task may hang waiting for a response.

`EN_NET_SYSLOG`

File: `os_config.h`

Enables or disables the system-wide logging of network events to a remote syslog server.

Valore: `1`

Note:
When set to 1, the RTOS will forward internal network stack events and error messages to a configured UDP syslog server. This feature requires the network stack (TCP/IP) to be initialized and active. Disabling this (0) reduces CPU overhead and memory usage by removing the logging task and buffer allocations.

Uso:
Enable this during the development and debugging phase to monitor network connectivity issues or stack errors in real-time. Disable it in production builds to save system resources and prevent unnecessary network traffic. Ensure the syslog server IP is correctly configured in the network settings before enabling this flag.

`EN_NOTIFY`

File: `os_config.h`

Enables the Task Notification feature, allowing tasks to send lightweight signals or data directly to other tasks.

Valore: `1`

Note:
When set to 1, the OS allocates a notification value and state variable for every task control block. This increases RAM usage per task. It is a prerequisite for using functions like os_task_notify() and os_task_wait_notify(). If set to 0, these functions will be disabled, and the kernel will ignore any notification-related calls.

Uso:
Enable this when you need a fast, low-overhead alternative to semaphores or queues for simple task synchronization or event signaling. Change this to 0 only if you are extremely constrained on RAM and are certain your application does not require direct task-to-task signaling.

`EN_NTP_SERVICE`

File: `os_config.h`

Enables or disables the built-in Network Time Protocol (NTP) client service for automatic system clock synchronization.

Valore: `1`

Note:
When set to 1, the RTOS initializes a background task to periodically query a time server; when set to 0, the service is excluded from the build, reducing flash and RAM usage. This flag requires the TCP/IP stack (e.g., LwIP) to be enabled and a valid network interface to be active.

Uso:
Enable this when your application requires accurate UTC timestamps or logging with real-world time. Disable it if your device operates in an isolated network without internet access or if you are using an external Hardware Real-Time Clock (RTC) module to save system resources.

`EN_PACKET_TRACER`

File: `os_config.h`

Enables the internal diagnostic logging system for tracking inter-task communication packets.

Valore: `1`

Note:
When set to 1, the kernel records metadata for every message queue and mailbox transaction. Enabling this increases CPU overhead and RAM usage for the trace buffer. It depends on the OS_DEBUG_LEVEL being set to at least 1. Typical values are 0 (disabled) or 1 (enabled).

Uso:
Enable this flag during the development and debugging phase to trace message flow and identify race conditions or lost packets. Disable it in production builds to save memory and improve system performance, as the tracing mechanism consumes significant clock cycles.

`EN_POOL`

File: `os_config.h`

Enables the dynamic memory pool management system within caOS.

Valore: `1`

Note:
When set to 1, the kernel allocates memory from pre-defined fixed-size pools instead of using the standard heap. This prevents memory fragmentation and ensures deterministic allocation times. Disabling this (0) forces the system to rely on standard heap allocation, which may be slower and prone to fragmentation. This flag must be enabled if your application uses message queues or dynamic task creation.

Uso:
Enable this flag when your application requires frequent creation and deletion of objects like tasks or messages. Change this to 0 only if your system is strictly static and you need to save the small amount of RAM overhead required for the pool management structures. Ensure that

the pool sizes defined in the configuration are sufficient for your peak load to avoid allocation failures.

`EN_PWR`

File: `os_config.h`

Enables the power management module for the caOS RTOS.

Valore: `1`

Note:
When set to 1, the kernel enters low-power sleep modes during idle time. When set to 0, the CPU remains in active mode continuously. This flag requires the hardware abstraction layer (HAL) to have tickless idle support configured. Typical values are 0 for debugging and 1 for battery-powered applications.

Uso:
Enable this flag when your device needs to minimize power consumption during periods of inactivity. Disable it if you experience issues with peripheral wake-up latency or if you are using a debugger that loses connection when the CPU enters sleep mode.

`EN_PWR_DBG`

File: `os_config.h`

Enables or disables the power management debug logging features within the RTOS kernel.

Valore: `0`

Note:
When set to 1, the system outputs detailed power state transitions and sleep mode entry/exit logs to the debug console. Disabling it (0) reduces CPU overhead and memory footprint. This flag depends on the global debug logging system being active; if the main logging module is disabled, this flag has no effect.

Uso:
Set to 1 during the development phase when troubleshooting battery life issues or unexpected wake-up events. Set to 0 for production builds to minimize power consumption and prevent UART/ITM buffer overflows caused by excessive logging.

`EN_QSPI`

File: `os_config.h`

Enables or disables the Quad-SPI (QSPI) peripheral driver support within the caOS kernel.

Valore: `0`

Note:
When set to 1, the OS initializes the QSPI controller and includes the necessary HAL drivers for external flash memory access. When set to 0, the QSPI peripheral is ignored, reducing the kernel binary size and freeing up memory. This flag must be enabled if your hardware design uses an external QSPI flash chip for data storage or XIP (Execute-in-Place) operations.

Uso:

Set this to 1 only if your specific STM32 board has an external QSPI flash memory connected. If you attempt to use QSPI functions while this is set to 0, the system will trigger a hard fault or return null pointers. Keep it at 0 for projects that do not require external memory to save precious RAM and Flash resources.

`EN_RNG`

File: `os_config.h`

Enables the hardware random number generator (RNG) peripheral driver within the caOS kernel.

Valore: `1`

Note:
When set to 1, the kernel initializes the STM32 RNG peripheral and provides a thread-safe API for entropy generation. Disabling this (0) reduces the memory footprint and removes the dependency on the RNG hardware clock. This flag requires the STM32 RNG peripheral to be present on the specific MCU model.

Uso:
Enable this when your application requires cryptographically secure random numbers, unique session IDs, or jitter for network protocols. Change to 0 if you need to save power or if the RNG peripheral is not available on your target hardware. Warning: Ensure the RNG clock is enabled in the system clock configuration, otherwise the kernel may hang during initialization.

`EN_ROUTE_SERVICE`

File: `os_config.h`

Enables the internal routing service for inter-task communication and message passing.

Valore: `1`

Note:
When set to 1, the kernel initializes the routing table and message buffers. Disabling this (0) reduces RAM usage by approximately 2KB but prevents tasks from using the OS_Route_Send() and OS_Route_Receive() APIs. This flag must be enabled if any system modules rely on asynchronous event signaling.

Uso:
Enable this flag when your application requires decoupled communication between tasks. Disable it only in extremely memory-constrained systems where all task synchronization is handled strictly via binary semaphores or direct task notifications to save stack and heap space.

`EN_RTC`

File: `os_config.h`

Enables the Real-Time Clock (RTC) driver integration within the caOS kernel.

Valore: `1`

Note:

When set to 1, the OS initializes the hardware RTC peripheral and provides system-wide timekeeping services. If set to 0, all RTC-related API calls will return an error or remain unimplemented. This flag depends on the underlying STM32 HAL RTC configuration being correctly defined in the project's hardware abstraction layer.

Uso:
Set to 1 if your application requires timestamping, scheduling based on wall-clock time, or low-power wake-up alarms. Disable it (0) to save flash memory and reduce initialization time if the application does not require time-tracking features.

`EN_RWLOCK`

File: `os_config.h`

Enables or disables the Read-Write Lock (RWLock) synchronization primitive in the caOS kernel.

Valore: `1`

Note:
When set to 1, the kernel includes code for RWLock objects, allowing multiple readers or a single writer to access a resource. Disabling this (0) reduces the kernel binary size and memory footprint. This flag has no dependencies on other synchronization primitives but is required if any application task attempts to initialize or use an RWLock handle.

Uso:
Enable this flag when your application requires high-concurrency access to shared resources where read operations significantly outnumber write operations. Change to 0 if your project is memory-constrained and does not utilize RWLocks to save flash and RAM. Warning: disabling this while the application code still references RWLock functions will result in compilation errors.

`EN_SAI`

File: `os_config.h`

Enables the Serial Audio Interface (SAI) peripheral driver support in caOS.

Valore: `1`

Note:
When set to 1, the caOS SAI driver module is compiled and initialized, allowing the use of ca_sai_* API functions. This flag consumes additional flash memory and requires the corresponding hardware peripheral to be available on the STM32 chip. If set to 0, any attempt to call ca_sai_* functions will result in a compilation error or linker failure.

Uso:
Enable this flag only when your application requires digital audio streaming (e.g., I2S, TDM, or PDM audio interfaces). If your project does not use audio peripherals, keep this set to 0 to save system resources and reduce the binary footprint.

`EN_SDIO`

File: `os_config.h`

Enables the SDIO peripheral driver for SD card communication within the caOS RTOS.

Valore: `1`

Note:
When set to 1, the OS initializes the SDIO hardware abstraction layer and includes the SD card file system support. Disabling this (0) reduces the binary footprint and frees up CPU cycles if no SD card is used. This flag depends on the hardware having an SDIO interface connected; if enabled without physical hardware, the system may hang during the initialization phase.

Uso:
Set this to 1 when your application requires logging data to an SD card or loading configuration files from external storage. Change to 0 if you are using a different storage medium (like SPI Flash) or if the SDIO pins are needed for other GPIO functions to prevent hardware conflicts.

`EN_SHA256`

File: `os_config.h`

Enables the SHA-256 cryptographic hash function implementation in the RTOS kernel

Valore: `1`

Note:
When set to 1, the RTOS includes SHA-256 hashing capabilities which increases binary size by approximately 2-4KB and requires cryptographic hardware support on STM32 devices. Setting to 0 removes all SHA-256 functionality and will cause linker errors if any application code attempts to use these functions. The flag depends on EN_CRYPTO being enabled for hardware acceleration support.

Uso:
Enable this flag when implementing secure communication protocols, firmware verification, or digital signature validation. Disable only when memory constraints are critical and no cryptographic hashing is required. Warning: Disabling this flag while code references SHA-256 functions will cause compilation failures

---

`EN_SHA256_TEST`

File: `os_config.h`

Enables or disables the built-in SHA-256 cryptographic integrity verification suite for system binaries.

Valore: `1`

Note:
When set to 1, the RTOS performs a checksum validation of the application image during the boot sequence. This requires the hardware CRC or cryptographic acceleration unit to be initialized. If set to 0, the boot time is reduced, but the system loses the ability to detect unauthorized firmware modifications or memory corruption at startup.

Uso:

Enable this flag during production builds to ensure firmware authenticity and integrity. Disable it only during early development stages if you need to speed up the flashing and boot cycle, or if you are using a custom bootloader that handles verification externally. Ensure that your linker script reserves space for the checksum signature if this is enabled.

`EN_SHA512`

File: `os_config.h`

Enables or disables the hardware-accelerated SHA-512 cryptographic hashing module within the RTOS kernel.

Valore: `1`

Note:
When set to 1, the kernel includes SHA-512 support, increasing the binary size and memory footprint. It requires the underlying STM32 hardware to have a functional HASH peripheral. If set to 0, any calls to SHA-512 functions will return an error or trigger a linker error if the code is not wrapped in preprocessor checks.

Uso:
Enable this flag only if your application requires secure hashing, data integrity checks, or digital signature verification. Disable it to save flash memory and RAM if your project does not utilize SHA-512, as it consumes significant system resources.

`EN_SHA512_TEST`

File: `os_config.h`

Enables the built-in SHA-512 cryptographic integrity verification test suite during system startup.

Valore: `1`

Note:
When set to 1, the RTOS executes a self-test routine on the SHA-512 hardware accelerator or software library before the scheduler starts. If set to 0, the test is skipped, reducing boot time. This flag depends on the availability of the cryptographic module in the current STM32 target and requires sufficient stack space for the test execution.

Uso:
Enable this during the development and production validation phases to ensure the integrity of the hashing engine. Disable it only in final production builds if boot time requirements are extremely strict and the hardware has been verified to be reliable across all units.

`EN_SPDIFRX`

File: `os_config.h`

Enables or disables the SPDIFRX peripheral driver support in caOS.

Valore: `0`

Note:
When set to 1, the caOS SPDIFRX driver (ca_spdifrx_*) is compiled into the binary, allowing the application to interface with the hardware SPDIF receiver. When set to 0, the driver is

excluded, reducing flash memory usage. This flag does not have direct dependencies on other peripherals but requires the target STM32 MCU to have a physical SPDIFRX peripheral.

Uso:
Set this to 1 only if your application needs to receive digital audio data via the SPDIF interface. If your project does not use SPDIFRX, keep it at 0 to save code space and avoid unnecessary peripheral initialization code.

`EN_SPI`

File: `os_config.h`

Enables or disables the SPI peripheral driver support within the caOS RTOS kernel.

Valore: `0`

Note:
When set to 1, the system initializes SPI hardware abstraction layers and allocates memory for SPI-related structures. When set to 0, all SPI-related code is excluded from the build, reducing the final binary size. This flag is a prerequisite for any SPI-based communication tasks; if disabled, calls to SPI functions will result in compilation errors or undefined behavior.

Uso:
Set this to 1 only if your application requires SPI communication (e.g., interfacing with sensors, displays, or external memory). Keep it at 0 if your project does not use SPI to save flash memory and RAM. Ensure that the corresponding hardware pins are correctly configured in your STM32CubeMX project if you enable this flag.

`EN_SPI_DMA`

File: `os_config.h`

Enables Direct Memory Access (DMA) support for SPI peripheral communication.

Valore: `1`

Note:
When set to 1, SPI data transfers bypass the CPU, reducing overhead. When set to 0, the system uses standard interrupt-driven or polling methods. This flag requires the corresponding DMA controller clock to be enabled in the system initialization code. Valid values are 0 (disabled) or 1 (enabled).

Uso:
Enable this when your application requires high-speed SPI data transfers or when the CPU is heavily loaded with other tasks. Change to 0 only if you are debugging SPI timing issues or if the specific STM32 hardware configuration lacks DMA support for the chosen SPI instance.

`EN_SSH_SERVICE`

File: `os_config.h`

Enables or disables the integrated SSH server service within the caOS RTOS kernel.

Valore: `1`

Note:

When set to 1, the SSH service task is initialized during system boot, consuming additional RAM for the stack and heap. It requires the TCP/IP stack (e.g., lwIP) to be enabled and configured. If set to 0, the service is excluded from the build, reducing the binary size and freeing up system resources.

Uso:
Enable this flag when remote secure shell access is required for debugging, configuration, or file management over Ethernet. Disable it in production environments where security is a concern or when memory constraints are critical. Ensure that the network interface is properly initialized before enabling this service to prevent boot-time crashes.

`EN_SSH_TEST`

File: `os_config.h`

Enables the Secure Shell (SSH) diagnostic interface for remote system debugging.

Valore: `1`

Note:
When set to 1, the RTOS initializes the SSH server task and binds it to the configured network interface. This flag depends on the network stack being enabled (e.g., LWIP_ENABLED). Disabling this (0) saves significant RAM and CPU cycles by removing the SSH daemon and associated encryption buffers.

Uso:
Enable this during the development and integration phase to allow remote access to the system shell for monitoring and troubleshooting. Disable it in production builds to reduce the attack surface and free up memory resources. Ensure that the network stack is fully initialized before this flag is set to 1 to prevent system crashes during boot.

`EN_SSH_TRACE`

File: `os_config.h`

Enables the Secure Shell (SSH) diagnostic tracing mechanism for remote system monitoring.

Valore: `1`

Note:
When set to 1, the RTOS routes kernel diagnostic logs to the SSH interface. Disabling this (0) reduces CPU overhead and memory usage. This flag depends on the network stack being initialized and requires the SSH_PORT configuration to be defined.

Uso:
Enable this during development or field debugging to monitor task states and system health remotely. Disable it in production builds to save flash memory and prevent potential security vulnerabilities associated with exposed diagnostic ports.

`EN_STLINK_WAIT_READY`

File: `os_config.h`

Enables a mandatory delay during system startup to allow the ST-LINK debugger to initialize and attach.

Valore: `1`

Note:
When set to 1, the system executes a blocking wait loop at the very beginning of main() before the RTOS scheduler starts. If set to 0, the system boots immediately. This flag is independent of other RTOS features but is critical for debugging early-boot code. Typical values are 1 (enabled) for development and 0 (disabled) for production firmware.

Uso:
Enable this (1) if you are experiencing hard faults or erratic behavior immediately after power-on or reset while a debugger is connected. It ensures the IDE has time to halt the CPU before the application code runs. Disable this (0) for final production builds to ensure the device starts up instantly without unnecessary delays.

`EN_STREAM`

File: `os_config.h`

Enables the internal stream processing module for asynchronous data handling.

Valore: `1`

Note:
When set to 1, the kernel allocates memory for stream buffers and enables stream-related API functions. Disabling this (0) reduces the kernel's static RAM footprint. This flag is a prerequisite for using the caOS communication drivers and must be enabled if any peripheral stream tasks are defined in the project.

Uso:
Enable this flag when your application requires non-blocking data transmission or reception via UART, SPI, or I2C. If your project does not use streaming features, keep it disabled to save memory. Ensure that the heap size is sufficient to accommodate the buffers created when this flag is active.

`EN_SW_TIMER`

File: `os_config.h`

Enables the software timer management module within the caOS kernel.

Valore: `1`

Note:
When set to 1, the kernel initializes the software timer service, allowing tasks to schedule callbacks without hardware timer constraints. Disabling this (0) reduces the kernel's RAM footprint and CPU overhead. This flag must be enabled if any application-level timers or periodic software tasks are utilized.

Uso:
Enable this flag when your application requires non-blocking delays or periodic execution of functions that do not justify a dedicated hardware timer. If you disable this, ensure no part of your code calls timer-related API functions, as this will lead to linker errors or runtime crashes.

`EN_SYSLOG_SERVICE`

File: `os_config.h`

Enables or disables the internal system logging service for debugging and event tracking.

Valore: `1`

Note:
When set to 1, the RTOS records system events to a circular buffer or UART output. Disabling it (0) reduces CPU overhead and saves RAM by removing the logging task and buffer allocation. This flag depends on the availability of a configured output peripheral, such as UART or ITM.

Uso:
Enable this during the development and integration phase to monitor task switching, error codes, and system health. Disable it in production builds to optimize performance and reduce power consumption. Ensure your logging output peripheral is initialized before the scheduler starts if this is enabled.

`EN_TAMP`

File: `os_config.h`

Enables or disables the hardware tamper detection feature for the system.

Valore: `0`

Note:
When set to 1, the system initializes and monitors hardware tamper pins (via ca_exti_tamper_enable). When set to 0, this functionality is disabled, saving memory and CPU cycles. This flag depends on the underlying STM32 hardware support for tamper detection. Typical values are 0 (disabled) or 1 (enabled).

Uso:
Change this value to 1 if your application requires physical security, such as detecting chassis intrusion or unauthorized access to the board. If your hardware design does not include tamper switches or if you do not need this security feature, keep it at 0 to avoid unnecessary interrupt overhead and potential false triggers.

`EN_TCP_SERVICE`

File: `os_config.h`

Enables or disables the integrated TCP/IP networking stack service within the caOS kernel.

Valore: `1`

Note:
When set to 1, the RTOS initializes the TCP/IP stack and allocates necessary memory buffers for network communication. When set to 0, the stack is excluded from the build, reducing flash and RAM usage. This flag requires the hardware abstraction layer (HAL) for the Ethernet or Wi-Fi peripheral to be correctly configured in the project.

Uso:
Set to 1 if your application requires network connectivity (e.g., web server, MQTT, or socket communication). Set to 0 if the device is standalone to save system resources. Warning: Disabling this will cause compilation errors if your application code contains calls to network-related APIs.

EN_TCP_TRACE

File: `os_config.h`

Enables or disables the internal diagnostic logging for the TCP/IP stack.

Valore: `0`

Note:
When set to 1, the system outputs detailed packet processing and connection state logs to the debug console. When set to 0, tracing is disabled, reducing CPU overhead and memory usage. This flag depends on the underlying UART or ITM debug interface being initialized.

Uso:
Set to 1 only during development or when troubleshooting network connectivity issues. Always set to 0 in production builds to prevent performance degradation and excessive log data flooding the serial port.

EN_TFTP_SERVICE

File: `os_config.h`

Enables or disables the built-in Trivial File Transfer Protocol (TFTP) server service within the RTOS.

Valore: `1`

Note:
When set to 1, the system initializes the TFTP stack and binds it to the network interface. This requires the TCP/IP stack (e.g., LwIP) to be enabled and configured. Disabling this (0) reduces the memory footprint and removes the network port dependency.

Uso:
Enable this flag when you need to perform remote firmware updates or configuration file transfers over Ethernet. Ensure that the network stack is fully initialized before this service starts; setting this to 1 without a valid network interface configuration may cause the system to hang during the boot sequence.

EN_TIMER

File: `os_config.h`

Enables the internal software timer management system within the caOS kernel.

Valore: `1`

Note:
When set to 1, the kernel initializes the timer task and processes software timer callbacks. If set to 0, all software timer functionality is disabled, reducing RAM and CPU overhead. This flag is a prerequisite for using any os_timer_create or os_timer_start functions. It requires a defined OS_TIMER_TASK_PRIORITY and OS_TIMER_STACK_SIZE in the configuration file.

Uso:
Enable this flag when your application requires periodic tasks or delayed execution events. Disable it if your system is strictly event-driven or resource-constrained to save memory. Warning:

Disabling this while application code attempts to start a timer will result in a kernel assertion or undefined behavior.

`EN_TLS`

File: `os_config.h`

Enables or disables the Transport Layer Security (TLS) stack support within the caOS RTOS.

Valore: `1`

Note:
When set to 1, the system includes the code and memory structures required for TLS communication (e.g., ca_tls_* functions). Disabling this (set to 0) reduces the firmware binary size and RAM usage by excluding cryptographic overhead. This flag is a prerequisite for using secure network services like HTTPS or secure MQTT.

Uso:
Enable this flag when your application needs to establish encrypted network connections. If your project does not require secure communication, set this to 0 to save significant flash and heap memory. Ensure that your hardware has sufficient resources, as TLS operations are computationally intensive and require additional memory buffers.

`EN_UART_DMA`

File: `os_config.h`

Enables or disables Direct Memory Access (DMA) support for UART communication peripherals.

Valore: `0`

Note:
When set to 1, UART data transfers are offloaded to the DMA controller, reducing CPU overhead. When set to 0, the system uses standard interrupt-driven or polling-based I/O. Enabling this requires proper DMA stream/channel configuration in the STM32 CubeMX or hardware abstraction layer. It is typically a boolean flag (0 or 1).

Uso:
Set to 1 if your application handles high-speed serial communication or requires low CPU utilization during data transmission. Set to 0 if you are debugging UART issues, have limited DMA resources, or if the application has very low throughput requirements where the overhead of DMA setup is not justified.

`EN_UDP_CLIENT`

File: `os_config.h`

Enables or disables the UDP client functionality within the caOS networking stack.

Valore: `0`

Note:

When set to 1, the system allocates memory buffers and initializes the UDP socket interface. When set to 0, the UDP client code is excluded from the build, reducing the overall flash memory footprint. This flag depends on the global network stack being enabled in the configuration.

Uso:
Set this to 1 only if your application needs to send or receive data via UDP protocols. If your project only uses TCP or does not require network communication, keep this at 0 to save system resources and avoid unnecessary memory allocation.

`EN_UDP_SERVER`

File: `os_config.h`

Enables or disables the built-in UDP server functionality within the caOS network stack.

Valore: `1`

Note:
When set to 1, the RTOS initializes the UDP socket listener and allocates memory for the UDP buffer. If set to 0, the UDP server code is excluded from the build, reducing flash memory usage. This flag depends on the network stack being enabled in the project configuration.

Uso:
Set to 1 if your application needs to receive or send data via UDP packets. Set to 0 if your device only uses TCP or does not require network communication to save system resources. Ensure that the UDP buffer size is also configured correctly if this flag is enabled.

`EN_USB`

File: `os_config.h`

Enables the USB stack and hardware peripheral drivers within the caOS RTOS environment.

Valore: `1`

Note:
When set to 1, the kernel initializes USB-related memory pools and interrupt handlers. Disabling this (0) reduces the binary footprint and frees up RAM. This flag depends on the hardware abstraction layer (HAL) being configured for USB support in the STM32 CubeMX project.

Uso:
Set to 1 when your application requires USB communication (e.g., Virtual COM Port, HID, or Mass Storage). Disable it if your project does not use USB to save system resources and avoid potential conflicts with USB interrupt vectors.

`EN_USB_CDC`

File: `os_config.h`

Enables or disables the USB Communications Device Class (CDC) driver stack within the RTOS.

Valore: `0`

Note:

When set to 1, the system initializes the USB peripheral and allocates memory for the CDC buffer and interrupt handlers. When set to 0, the USB stack is completely excluded from the build, reducing flash and RAM usage. This flag depends on the hardware having a USB-capable STM32 peripheral and requires the USB clock configuration to be active in the system clock tree.

Uso:
Set to 1 if your application requires a virtual COM port for logging, debugging, or data communication with a PC. Set to 0 if the USB peripheral is not used or if you need to save memory resources. Ensure that if this is enabled, the corresponding USB interrupt priority is correctly configured in the NVIC to avoid blocking the RTOS scheduler.

`EN_USB_CUSTOM`

File: `os_config.h`

Enables or disables the custom USB stack implementation within the caOS RTOS.

Valore: `0`

Note:
When set to 1, the system uses the custom caOS USB driver; when set to 0, the system defaults to the standard STM32 HAL USB library. This flag requires the USB peripheral clock to be configured in the system clock tree. Disabling this (0) is recommended for standard HID or CDC applications to ensure compatibility with ST-provided drivers.

Uso:
Change this to 1 only if you are implementing a proprietary USB class or require low-level control over USB descriptors not supported by the standard HAL. Keep this at 0 for most projects to avoid unnecessary code bloat and potential stability issues with standard USB host controllers.

`EN_USB_DBG`

File: `os_config.h`

Enables or disables the USB-based debug interface for the caOS RTOS.

Valore: `0`

Note:
When set to 1, the system initializes the USB peripheral to stream debug logs and telemetry to a host PC. When set to 0, the USB peripheral is disabled, saving CPU cycles and memory. This flag requires the hardware USB pins to be correctly configured in the board support package (BSP).

Uso:
Set to 1 during development to monitor system state and task execution via a serial terminal. Set to 0 for production builds to reduce power consumption and prevent unauthorized access to system logs. Ensure the USB clock is configured in the system clock tree before enabling this flag.

`EN_USB_DFU`

File: `os_config.h`

Enables or disables the USB Device Firmware Upgrade (DFU) bootloader functionality within the RTOS.

Valore: `0`

Note:
When set to 1, the system initializes the USB DFU stack, allowing firmware updates over USB. When set to 0, this feature is disabled, saving flash memory and RAM. This flag requires the USB hardware peripheral to be physically connected and configured in the hardware abstraction layer.

Uso:
Set to 1 during the production phase or when field updates are required. Set to 0 during standard development to save resources or if the USB interface is not used. Warning: Disabling this while the device is in the field will prevent remote firmware updates via USB.

`EN_USB_FS`

File: `os_config.h`

Enables the Full-Speed USB peripheral driver within the caOS RTOS stack.

Valore: `1`

Note:
When set to 1, the RTOS initializes the USB hardware abstraction layer and allocates memory for USB buffers. This flag depends on the hardware having a physical USB FS controller (e.g., STM32F103 or STM32F407). If set to 0, all USB-related tasks and interrupt service routines are excluded from the build, reducing the binary footprint.

Uso:
Set this to 1 when your application requires USB communication (CDC, HID, or MSC). You must ensure that the system clock is configured to 48MHz for the USB peripheral to function correctly. Disable this flag if you are not using USB to save RAM and Flash memory, or if you are using a different communication interface like UART or CAN.

`EN_USB_HS`

File: `os_config.h`

Enables or disables the High-Speed (HS) USB peripheral controller driver in the caOS kernel.

Valore: `0`

Note:
When set to 1, the kernel initializes the USB HS hardware abstraction layer and allocates memory for its associated buffers. When set to 0, the peripheral is disabled, saving RAM and flash memory. This flag depends on the hardware having an external ULPI PHY or an integrated HS PHY. Valid values are 0 (disabled) or 1 (enabled).

Uso:
Set this to 1 only if your specific STM32 board design includes a High-Speed USB PHY and you intend to use USB 2.0 HS communication. If you are using Full-Speed (FS) USB or no USB

at all, keep this at 0 to prevent hard faults caused by accessing uninitialized or non-existent hardware registers.

`EN_USB_MSC`

File: `os_config.h`

Enables or disables the USB Mass Storage Class (MSC) driver stack within the RTOS.

Valore: `0`

Note:
When set to 1, the system initializes the USB peripheral as a storage device (like a flash drive). When set to 0, the USB MSC stack is excluded from the build, reducing flash memory usage and disabling USB storage functionality. This flag depends on the hardware having a USB-capable STM32 peripheral and requires the USB hardware abstraction layer (HAL) to be configured.

Uso:
Set this to 1 if your application needs to expose internal memory or an SD card as a drive to a PC. Set to 0 if you are not using USB storage to save code space and avoid unnecessary interrupt overhead. Ensure that the system clock is configured for USB operation (48MHz) before enabling this flag.

`EN_USB_NCM`

File: `os_config.h`

Enables the Network Control Model (NCM) class support for the USB stack.

Valore: `1`

Note:
When set to 1, the RTOS initializes the USB NCM driver, allowing the device to appear as a virtual Ethernet adapter over USB. This requires the USB peripheral to be configured in Device mode and depends on the underlying USB hardware abstraction layer (HAL) being active. Disabling this (0) removes the NCM stack from the build, reducing flash memory usage. The NCM implementation requires additional RAM for USB buffer management and will increase the overall memory footprint of the application.

Uso:
Enable this flag when you need to implement USB-to-Ethernet bridging or RNDIS-like connectivity for network debugging. Change this value only if your application requires USB networking; ensure that the USB clock and GPIO pins are correctly initialized in your board support package before enabling. Warning: Do not enable this flag if your USB peripheral is not properly configured for device mode, as it will cause runtime errors. Typical use cases include firmware update over USB, network monitoring, and remote debugging scenarios.

`EN_WDT`

File: `os_config.h`

Enables the internal Watchdog Timer (WDT) supervisor for the caOS kernel.

Valore: `1`

Note:

When set to 1, the kernel periodically resets the hardware watchdog timer. If the system hangs or enters an infinite loop, the WDT will trigger a hardware reset. This flag depends on the hardware-specific WDT peripheral initialization being completed in the board support package. Typical values are 1 (enabled) or 0 (disabled).

Uso:
Enable this in production builds to ensure system recovery from unexpected software freezes. Disable it only during debugging sessions when using breakpoints, as the WDT will otherwise trigger a reset while the CPU is halted.

`EN_X25519`

File: `os_config.h`

Enables the X25519 Elliptic Curve Diffie-Hellman (ECDH) key exchange algorithm within the caOS cryptographic library.

Valore: `1`

Note:
When set to 1, the system includes the X25519 implementation, increasing the flash memory footprint by approximately 4KB. It requires the hardware random number generator (RNG) peripheral to be initialized. If set to 0, any attempt to call secure handshake functions will result in a compile-time error or a runtime 'feature not supported' exception.

Uso:
Enable this flag when your application requires secure key exchange for encrypted communication (e.g., TLS or custom secure protocols). Disable it if you are strictly optimizing for code space and your project does not perform asymmetric key exchanges. Ensure your STM32 chip has sufficient flash capacity before enabling.

`EN_X25519_TEST`

File: `os_config.h`

Enables the built-in self-test suite for the X25519 elliptic curve cryptography module.

Valore: `1`

Note:
When set to 1, the system executes a series of known-answer tests (KAT) during the kernel initialization phase. If set to 0, these tests are skipped, reducing boot time and flash memory footprint. This flag depends on the presence of the cryptographic library module in the build configuration.

Uso:
Enable this flag during the development and integration phase to verify that the X25519 implementation is functioning correctly on the target STM32 hardware. It is recommended to disable this (set to 0) in production firmware to minimize startup latency and prevent potential security side-channel exposure during the test execution.

`ETH_MPU_REGION`

File: `hw/f4xx/f407/objmap.h`
File: `hw/f4xx/f429/objmap.h`
File: `hw/f7xx/f746/objmap.h`
File: `hw/h5xx/h563/objmap.h`
File: `hw/h7xx/h747/objmap.h`
File: `hw/h7xx/h753/objmap.h`

Defines the Memory Protection Unit (MPU) region index reserved for the Ethernet DMA descriptor memory space.

Valori per MCU:

MCU	Valore
F407	5U
F429	5U
F746	5U
H563	5U
H747	5U
H753	5U

Note:
This constant assigns a specific MPU slot (0-7) to the Ethernet peripheral's memory buffers. Enabling this ensures the MPU correctly marks the Ethernet memory as non-cacheable or shareable to prevent data coherency issues. It is strictly dependent on the hardware architecture; all supported STM32 variants currently require index 5. Changing this value without updating the corresponding MPU configuration table will lead to memory access faults or Ethernet driver failure.

Uso:
This value should generally remain at 5U for all supported STM32 targets. You should only modify this if you are porting caOS to a new MCU architecture where region 5 is already occupied by another critical system peripheral or if the hardware memory map requires a different slot allocation to avoid overlapping protection regions.

`ETH_P_ARP`

File: `caos/net/inc/net.h`

The Ethernet protocol identifier for the Address Resolution Protocol (ARP).

Valore: `0x0806U`

Note:
This constant represents the EtherType field value (0x0806) used in Ethernet frames to indicate that the payload contains an ARP packet. It is a fixed standard defined by IEEE 802.3. Changing this value will cause the network stack to ignore all incoming ARP requests and replies, effectively breaking IPv4 address resolution.

Uso:
This value should never be changed in standard network applications. It is used by the caOS network driver to filter incoming Ethernet frames and identify which packets should be passed to the ARP handler. If you are debugging raw Ethernet frames and notice that ARP packets are not being processed, verify that the hardware filter or the driver's receive logic is correctly comparing the incoming EtherType against this constant.

ETH_P_IP

File: `caos/net/inc/net.h`

The Ethernet protocol identifier for the Internet Protocol (IPv4) packet type.

Valore: `0x0800U`

Note:
This constant represents the EtherType field value 0x0800, which signals that the payload of an Ethernet frame is an IPv4 packet. It is a standard value defined by IEEE 802.3. Changing this value will cause the network stack to ignore all incoming IPv4 traffic, as the hardware or driver will fail to match the frame type.

Uso:
This value should generally remain unchanged as it is a fixed networking standard. You would only reference this constant when writing custom packet filters, raw socket handlers, or low-level Ethernet driver logic to identify or construct IPv4 frames. Do not modify this value unless you are implementing a non-standard protocol stack.

ETH_P_IPV6

File: `caos/net/inc/net.h`

Defines the Ethernet protocol identifier for IPv6 packets.

Valore: `0x86DDU`

Note:
This constant represents the EtherType value 0x86DD used in the Ethernet frame header to indicate that the payload is an IPv6 packet. It is a fixed standard defined by IEEE 802.3. Changing this value will cause the network stack to fail to recognize or process any incoming IPv6 traffic, as the hardware/driver will no longer match the frame type correctly.

Uso:
This value should generally never be changed as it is defined by international networking standards. You would only reference this constant when writing custom packet filters, raw socket handlers, or debugging the network stack to verify that the Ethernet layer is correctly identifying IPv6 frames.

ETH_P_VLAN

File: `caos/net/inc/net.h`

Defines the standard IEEE 802.1Q EtherType value used to identify VLAN-tagged Ethernet frames.

Valore: `0x8100U`

Note:
This constant represents the hex value 0x8100, which is the global standard for VLAN tagging. When enabled in the network stack, the system will parse incoming frames for this specific header to extract VLAN IDs. It is a read-only configuration for the protocol parser and should not be modified unless working with non-standard or proprietary network encapsulation.

Uso:

Use this value when configuring the Ethernet MAC filter or the packet inspection layer to ensure the RTOS correctly identifies and strips VLAN tags. Do not change this value for standard network communication, as doing so will cause the stack to ignore all standard VLAN-tagged traffic.

FAT_CRYPT_CHUNK

File: caos/drivers/inc/fat_service.h

Defines the size in bytes of the data buffer used for cryptographic operations within the FAT file system driver.

Valore: 512U

Note:
This constant determines the memory footprint of the encryption/decryption buffer. It must be a multiple of the underlying storage sector size (typically 512 bytes). Increasing this value can improve throughput for large file operations but will consume more RAM from the heap or static pool. It is strictly dependent on the hardware's DMA alignment requirements.

Uso:
Adjust this value if you encounter performance bottlenecks during file I/O or if you need to optimize RAM usage on memory-constrained STM32 devices. Ensure the value is a power of two to maintain compatibility with standard FAT sector alignment. Setting this too low may increase CPU overhead due to frequent context switching during encryption tasks.

HASH_CTX_INIT_CAP

File: caos/drivers/inc/hash.h

Defines the initial memory allocation size in bytes for the hashing context buffer.

Valore: 256U

Note:
This constant determines the static memory footprint of the hash driver's internal state. Increasing this value allows for larger internal buffers or more complex hashing algorithms, while decreasing it saves RAM. It must be large enough to accommodate the context structure of the specific hardware peripheral or software algorithm being used; otherwise, the driver will trigger a buffer overflow or memory corruption.

Uso:
Adjust this value if you encounter memory allocation errors during hash initialization or if you switch to a hashing algorithm that requires a larger state structure. Do not set this value lower than the size of the largest context structure required by your enabled algorithms, as this will lead to undefined behavior or system crashes.

HASH_MAX_DIGEST_SIZE

File: caos/drivers/inc/hash.h

Defines the maximum buffer size in bytes allocated for storing cryptographic hash results.

Valore: 32

Note:

This constant sets the static memory allocation for hash output buffers. If set too low, hashing algorithms requiring larger outputs (like SHA-256) will trigger buffer overflows or truncation errors. It must be at least 32 to support SHA-256. Increasing this value consumes more RAM in the driver's internal state structures.

Uso:
Change this value only if you need to support newer or stronger hashing algorithms that produce digests larger than 32 bytes (e.g., SHA-512). Ensure that all application-level buffers receiving hash outputs are sized according to this constant to prevent memory corruption.

`HASH_SHA1_DIGEST_SIZE`

File: `caos/drivers/inc/hash.h`

Defines the fixed output length in bytes for the SHA-1 cryptographic hash algorithm.

Valore: `20`

Note:
The value is set to 20 bytes (160 bits) as per the SHA-1 standard specification. This constant is used to allocate buffers for hash results. Changing this value will cause the SHA-1 hardware peripheral or software implementation to produce incorrect results or trigger buffer overflows in dependent code.

Uso:
This value should not be changed as it is strictly defined by the SHA-1 algorithm. It is used primarily for static memory allocation in security modules and communication protocols that require SHA-1 integrity checks. If you need a different digest size, you must switch to a different algorithm (e.g., SHA-256) rather than modifying this constant.

`HASH_SHA256_DIGEST_SIZE`

File: `caos/drivers/inc/hash.h`

Defines the fixed output length in bytes for the SHA-256 cryptographic hash algorithm.

Valore: `32`

Note:
This constant is set to 32 bytes (256 bits) as per the SHA-256 standard. Changing this value will break compatibility with standard cryptographic libraries and hardware acceleration drivers. It is a read-only configuration used to allocate buffers for hash results.

Uso:
Do not modify this value unless you are implementing a custom variant of the algorithm. It is used to size arrays that store the final digest; setting it incorrectly will lead to buffer overflows or truncated hash outputs.

`HCLK_FREQ_HZ`

File: `hw/f4xx/f407/objmap.h`
File: `hw/f4xx/f429/objmap.h`
File: `hw/f4xx/f446/objmap.h`
File: `hw/f7xx/f746/objmap.h`
File: `hw/h5xx/h563/objmap.h`

File: `hw/h7xx/h747/objmap.h`
File: `hw/h7xx/h753/objmap.h`

Defines the AHB bus clock frequency (HCLK) in Hertz for the system timer and peripheral timing calculations.

Valori per MCU:

MCU	Valore
F407	168000000UL
F429	168000000UL
F446	168000000UL
F746	216000000UL
H563	250000000UL
H747	200000000UL
H753	240000000UL

Note:
This constant is used by the RTOS kernel to calibrate the system tick timer (SysTick) and ensure accurate task scheduling. If this value does not match the actual hardware clock configuration set in the RCC registers, timing-dependent functions like vTaskDelay() or hardware timeouts will run at the wrong speed. It must be set to the exact frequency resulting from your PLL and prescaler settings.

Uso:
Change this value whenever you modify the system clock tree configuration (e.g., changing the crystal frequency or PLL multipliers). If your tasks are running faster or slower than expected, or if communication peripherals like UART/I2C have baud rate errors, verify that this constant matches the actual HCLK frequency output by the MCU clock tree.

`HKDF_SHA256_MAX_OUT`

File: `caos/crypto/inc/kdf.h`

Defines the maximum output length in bytes for the HKDF-SHA256 key derivation function.

Valore: `(255U * SHA256_DIGEST_SIZE)`

Note:
This constant sets the upper limit for the key material generated by the HKDF process. It is calculated as 255 times the SHA256 digest size (32 bytes), resulting in a maximum of 8160 bytes. Increasing this value consumes more stack or heap memory during the derivation process, while decreasing it limits the amount of key material you can extract in a single call.

Uso:
Modify this value only if your application requires a specific, smaller buffer size to save RAM, or if you need to generate larger amounts of key material than the default allows. Ensure that the buffer passed to ca_hkdf_sha256 does not exceed this limit to prevent buffer overflows.

`HMAC_SHA256_SIZE`

File: `caos/crypto/inc/kdf.h`

Defines the fixed byte length of the HMAC-SHA256 output digest.

Valore: `SHA256_DIGEST_SIZE`

Note:
This constant is set to SHA256_DIGEST_SIZE (32 bytes). It determines the buffer size required for storing HMAC-SHA256 results. Changing this value will break compatibility with standard HMAC-SHA256 implementations and cause buffer overflows or truncated data if the underlying ca_hmac_sha256 functions expect the standard 32-byte output.

Uso:
Do not change this value under normal circumstances. It is used by the ca_hmac_sha256 family of functions to allocate memory for cryptographic operations. If you are implementing a custom protocol that requires a truncated HMAC, perform the truncation manually after the calculation rather than modifying this constant.

`HTTPS_DEFAULT_PORT`

File: `os_config.h`

Defines the default TCP port number used for HTTPS communication services.

Valore: `443U`

Note:
This constant sets the listening or connection port for the ca_https service. The value 443U is the IANA standard for HTTPS. Changing this value will prevent the device from connecting to standard web servers unless they are specifically configured to listen on a non-standard port. It is a 16-bit unsigned integer.

Uso:
Modify this value only if your application must communicate with an HTTPS server running on a non-standard port (e.g., 8443). Do not change this for standard web traffic. Ensure the server side is configured to match this port, otherwise, the connection will be rejected by the remote host.

`HTTP_DEFAULT_PORT`

File: `os_config.h`

Defines the default TCP port number used by the caOS HTTP server when no specific port is provided.

Valore: `80U`

Note:
This constant sets the listening port for the HTTP stack. Changing this value affects all incoming HTTP connections. It must be a valid 16-bit unsigned integer (1-65535). Ensure that the chosen port does not conflict with other network services running on the STM32, such as DHCP or MQTT.

Uso:
Change this value if your application needs to run on a non-standard port (e.g., 8080 for development or 443 for HTTPS). Note that ports below 1024 are typically reserved for system services; using them may require specific network privileges or configurations.

`HTTP_METHOD_GET`

File: `caos/net/inc/http_service.h`

A bitmask flag representing the HTTP GET request method used by the caOS network service.

Valore: `0x01U`

Note:
This constant is used to identify or filter HTTP GET requests within the service stack. It is typically used in conjunction with other method flags (e.g., POST, PUT) as part of a bitwise field. The value 0x01U is a fixed identifier and should not be modified to ensure compatibility with the internal HTTP parser.

Uso:
Use this flag when configuring request handlers or defining allowed methods for a specific endpoint. Do not change this value, as it is hardcoded to match the HTTP protocol specification expected by the caOS network stack. If you are implementing a custom callback, check the incoming request method against this constant using a bitwise AND operation.

`HTTP_METHOD_HEAD`

File: `caos/net/inc/http_service.h`

A constant representing the HTTP HEAD method identifier used by the caOS network stack.

Valore: `0x04U`

Note:
This value is used internally by the HTTP service to identify requests that ask for header information only, without the response body. It must match the value expected by the server-side parser. It is a static identifier and does not affect system memory or task scheduling.

Uso:
Use this constant when implementing custom HTTP request handlers or filtering incoming traffic. Do not modify this value unless you are updating the entire network stack to a different protocol specification, as changing it will cause the HTTP service to fail to recognize HEAD requests.

`HTTP_METHOD_OPTIONS`

File: `caos/net/inc/http_service.h`

A bitmask flag used to enable or disable support for the HTTP OPTIONS method in the caOS web server.

Valore: `0x08U`

Note:
When enabled, the HTTP service will process and respond to OPTIONS requests, typically used for CORS preflight checks. If disabled, the server will return a 405 Method Not Allowed error for these requests. This flag is part of a bitmask configuration; it must be ORed with other method flags (e.g., GET, POST) to define the full set of supported HTTP verbs.

Uso:

Enable this flag if your web application requires CORS support or if a client needs to query the server's capabilities. Disable it to reduce the memory footprint and attack surface if your application only performs simple data retrieval or submission.

HTTP_METHOD_POST

File: caos/net/inc/http_service.h

A constant representing the HTTP POST request method identifier used by the caOS networking stack.

Valore: 0x02U

Note:
This value is used internally by the HTTP service to distinguish POST requests from other methods like GET (0x01) or PUT (0x03). It is typically passed as an argument to the http_send_request function. It has no direct impact on system memory or scheduling, but must match the protocol definition expected by the remote server.

Uso:
Use this constant when you need to send data to a server, such as submitting a form or uploading sensor readings. Do not change this value unless you are modifying the core HTTP protocol implementation, as changing it will cause the server to reject the request or misinterpret the data payload.

HTTP_QUERY_MAX

File: caos/net/inc/http_service.h

Defines the maximum character length allowed for an incoming HTTP query string.

Valore: 128U

Note:
This constant sets the buffer size for parsing URL parameters. If the incoming query exceeds 128 bytes, the HTTP service will truncate the data or return a 414 Request-URI Too Long error. It must be large enough to accommodate your longest expected URL parameters but should be kept small to conserve RAM in the heap or stack.

Uso:
Increase this value if your application requires processing long URL query strings or complex API parameters. Decrease it if you are running low on system memory and your application only uses short, simple query strings. Ensure that the buffer allocated in the HTTP task is sufficient to hold this value plus any necessary headers.

HTTP_RECV_TIMEOUT_MS

File: caos/net/inc/http_service.h

Defines the maximum time in milliseconds the HTTP service waits for incoming data packets before aborting the request.

Valore: 1500U

Note:

Increasing this value allows for slower network connections but consumes more memory and blocks the task for longer periods. Decreasing it improves responsiveness but may cause premature timeouts on high-latency networks. It is dependent on the underlying TCP stack timeout settings and should generally be greater than the round-trip time of the network.

Uso:
Adjust this value if you encounter frequent 'HTTP Timeout' errors during data reception. Increase it if the device is operating on a slow or congested network. Decrease it if you need the system to fail fast to maintain real-time performance. Ensure the value is not set too low, or valid data transfers will be interrupted.

`HTTP_REQ_BUF_SIZE`

File: `os_config.h`

Defines the maximum size in bytes for the internal buffer used to store incoming HTTP request data.

Valore: `2048U`

Note:
Increasing this value allows the system to handle larger HTTP headers or payloads but directly consumes more RAM from the heap. It must be large enough to accommodate the longest expected request header; if the request exceeds this size, the parser will return an error. Typical values range from 512 to 4096 depending on available memory.

Uso:
Increase this value if you encounter 'HTTP 414 Request-URI Too Long' or '400 Bad Request' errors when sending complex requests. Decrease it if the system is running low on heap memory and you are only handling small, simple API calls.

`HTTP_RESP_BUF_SIZE`

File: `os_config.h`

Defines the maximum size in bytes of the buffer used to store incoming HTTP response payloads.

Valore: `4096U`

Note:
This constant allocates a static memory block for HTTP data processing. Increasing this value allows the system to handle larger HTTP responses but directly increases the static RAM footprint of the application. It must be large enough to hold the expected response body; if the response exceeds this size, the HTTP client will truncate the data or return an overflow error. Ensure that the total RAM usage does not exceed the STM32 device's available heap and stack limits.

Uso:
Adjust this value when your application needs to fetch larger JSON objects or files from a web server. If you encounter 'Buffer Overflow' or 'Incomplete Data' errors during HTTP GET requests, increase this value. Conversely, if your device is running low on RAM, decrease this value to the smallest size that still accommodates your largest expected response.

HTTP_URI_MAX

File: caos/net/inc/http_service.h

Defines the maximum character length allowed for an HTTP request URI string.

Valore: 128U

Note:
This constant sets the size of the static buffer used to store incoming URI paths. If the incoming URI exceeds 128 bytes, the HTTP service will return a 414 Request-URI Too Long error. It directly impacts the memory footprint of the HTTP task stack or heap allocation. Valid range is typically 32 to 1024 depending on available RAM.

Uso:
Increase this value if your application requires complex REST API endpoints with long query parameters or deep directory structures. Decrease it to save RAM if your device only handles simple, short resource requests. Ensure the value is large enough to accommodate the longest expected URI plus the null terminator.

I2C_EN_IRQ

File: caos/drivers/inc/i2c.h

Enables interrupt-driven mode for the I2C peripheral driver.

Valore: ((u16)0x0102U)

Note:
When enabled, the driver uses hardware interrupts to handle data transfer instead of blocking the CPU in a polling loop. This requires the corresponding I2C interrupt vector to be enabled in the NVIC. If disabled, the driver defaults to a blocking polling mechanism. The value 0x0102 acts as a bitmask configuration for the I2C peripheral control register.

Uso:
Enable this flag to improve system efficiency and allow the CPU to perform other tasks while waiting for I2C data transfers to complete. Change this to disabled only if you are debugging timing-sensitive issues or if the system lacks sufficient interrupt priority resources. Ensure that the I2C interrupt handler is correctly implemented in your project before enabling this flag to prevent system hangs.

IPERF_DEFAULT_DURATION_MS

File: os_config.h

Defines the default execution time in milliseconds for the built-in iPerf network performance test.

Valore: 10000U

Note:
This constant sets the duration for which the network stack will transmit or receive data during a test session. It is used as the fallback value when no specific duration is provided by the user command. The value must be a positive integer representing milliseconds; typical values range from 1000U to 60000U depending on the desired test granularity.

Uso:

Adjust this value if your network performance tests are too short to provide a stable average or too long, causing system unresponsiveness. Increase this value if you need to observe long-term network stability or throughput trends. Be aware that setting an excessively high value may block other low-priority tasks if the iPerf task is not properly preempted.

`IPERF_DEFAULT_PORT`

File: `os_config.h`

Defines the default TCP/UDP port number used by the caOS built-in iPerf network performance testing utility.

Valore: `5001U`

Note:
This constant sets the listening port for the iPerf server task. It must be a valid 16-bit unsigned integer (1-65535). Changing this value requires a recompile of the firmware. Ensure the chosen port does not conflict with other network services like HTTP (80) or MQTT (1883).

Uso:
Change this value if your network environment or firewall policies restrict the use of port 5001, or if you need to run multiple iPerf instances simultaneously on different ports. Always ensure the client-side testing tool is configured to match this port number, otherwise, the connection will be refused.

`IPERF_IDLE_TIMEOUT_MS`

File: `os_config.h`

Defines the maximum duration in milliseconds the system waits for network activity before entering an idle state during iPerf testing.

Valore: `3000U`

Note:
When enabled, the system monitors incoming packets; if no data is received within this window, the test session is terminated. This value is dependent on the network stack's tick rate. Typical values range from 1000U to 10000U depending on the expected latency of the network environment.

Uso:
Increase this value if you experience premature test termination due to high network jitter or slow link initialization. Decrease it if you need the system to recover faster from dropped connections. Setting this value too low may cause valid tests to fail, while setting it too high may prevent the system from freeing up resources after a genuine connection loss.

`IP_DEFRAG_MAX_PAYLOAD`

File: `os_config.h`

Defines the maximum size in bytes of the reassembled IP packet payload that the system can buffer.

Valore: `1480U`

Note:

This constant sets the memory limit for the IP fragmentation reassembly buffer. If set too low, fragmented packets larger than this value will be dropped. If set too high, it increases static RAM consumption. The value must be a multiple of the MTU and is typically set to 1480 to accommodate standard Ethernet frames minus the IP header.

Uso:
Increase this value if your application receives large fragmented IP packets that are currently being discarded. Decrease it to save RAM if your network environment guarantees that packets will not exceed a smaller size. Ensure the value does not exceed the total available heap or static buffer allocation defined in the network stack configuration.

`IP_DEFRAG_MAX_SESSIONS`

File: `os_config.h`
File: `caos/net/inc/ip_defrag.h`

Defines the maximum number of concurrent IP fragments the system can track for reassembly.

Valore: `8U`

Note:
This constant determines the size of the internal buffer pool for fragmented packets. Increasing this value consumes more RAM but allows the system to handle more simultaneous fragmented network streams. It must be set based on available heap memory and expected network traffic patterns. Typical values range from 4 to 16 depending on the STM32 memory footprint.

Uso:
Increase this value if your application experiences dropped packets or network communication failures when receiving large data frames over UDP or TCP. Decrease it if the system is running low on RAM. Setting this value too high may lead to heap exhaustion, while setting it too low will cause the network stack to discard incoming fragments.

`IP_DEFRAG_TIMEOUT_MS`

File: `os_config.h`

Defines the maximum time in milliseconds to wait for all fragments of an IP packet to arrive before discarding them.

Valore: `15000U`

Note:
When enabled, the system buffers incoming IP fragments until the full packet is reassembled or this timer expires. If set too low, fragmented packets will be dropped, causing network communication failures. If set too high, it consumes more system memory by holding incomplete packets longer. Typical values range from 5000 to 30000 depending on network latency.

Uso:
Increase this value if you observe packet loss or incomplete data reception in environments with high network latency or congested links. Decrease it if you are running low on heap memory and need to free up buffers faster. Ensure this value is consistent with the expected round-trip time of your network infrastructure.

`IP_FLAG_DF`

File: `caos/net/inc/net_proto.h`

A bitmask flag representing the 'Don't Fragment' (DF) option in the IPv4 header.

Valore: `0x4000U`

Note:
When this bit is set in the IP header flags field, it instructs routers not to fragment the packet. If the packet size exceeds the Maximum Transmission Unit (MTU) of the next hop, the router will drop the packet and send an ICMP 'Destination Unreachable' message. This flag is typically used in Path MTU Discovery (PMTUD) mechanisms. It has no dependencies on other flags but requires the network stack to handle ICMP error responses correctly.

Uso:
Enable this flag when you need to prevent packet fragmentation to improve network efficiency or to perform Path MTU Discovery. Use it with caution: if the path between your STM32 device and the destination contains a link with a smaller MTU than your packet size, the communication will fail. Ensure your application layer is prepared to handle dropped packets or reduced payload sizes if this flag is active.

`IP_FLAG_MF`

File: `caos/net/inc/net_proto.h`

A bitmask flag representing the 'More Fragments' bit in an IPv4 header.

Valore: `0x2000U`

Note:
When this bit is set (1), it indicates that the current IP packet is not the last fragment of a larger datagram. When cleared (0), it signifies the last fragment. This flag is used by the network stack's reassembly logic. It is a standard protocol constant defined by RFC 791 and should not be modified manually.

Uso:
Do not change this value as it is defined by the IPv4 protocol specification. It is used internally by the caOS network stack when parsing incoming fragmented packets or constructing outgoing ones. Modifying this will break network compatibility and cause packet reassembly failures.

`IP_FRAG_OFF_MASK`

File: `caos/net/inc/net_proto.h`

A bitmask used to extract the 13-bit fragment offset field from an IPv4 header.

Valore: `0x1FFFU`

Note:
The value 0x1FFF represents the 13 bits allocated for the fragment offset in the IP header. This constant is used during packet reassembly to identify the position of a data fragment within the original datagram. It is a fixed protocol constant defined by RFC 791 and should not be modified unless implementing a non-standard IP stack.

Uso:

Use this constant when parsing incoming IP packets to isolate the offset value from the flags field. Do not change this value; modifying it will cause the network stack to incorrectly calculate fragment positions, leading to corrupted data reassembly and dropped packets.

`IP_PROTO_ICMP`

File: `caos/net/inc/net_proto.h`

Enables or disables the Internet Control Message Protocol (ICMP) stack within the caOS networking layer.

Valore: `1U`

Note:
When set to 1U, the system includes code to process ICMP packets, such as Echo Requests (ping). Disabling it (0U) reduces the firmware binary size and memory footprint by removing ICMP handling logic. This flag is a prerequisite for network diagnostic tools and requires the base IP stack to be enabled.

Uso:
Set this to 1U if your device needs to respond to 'ping' requests for network connectivity testing. Disable it (0U) in production environments where you want to minimize the attack surface or save flash memory on resource-constrained STM32 devices.

`IP_PROTO_TCP`

File: `caos/net/inc/net_proto.h`

Defines the protocol number for TCP as specified in the IPv4 header.

Valore: `6U`

Note:
This constant is set to 6, which is the IANA-assigned protocol number for TCP. It is used by the network stack to identify incoming packets as TCP segments. Changing this value will cause the stack to fail to recognize TCP traffic, effectively disabling all TCP-based communication.

Uso:
This value should generally not be changed as it is defined by the Internet Protocol standard. It is used when constructing raw IP packets or filtering incoming traffic. Ensure this matches the standard value of 6 to maintain compatibility with external network devices and standard protocol stacks.

`IP_PROTO_UDP`

File: `caos/net/inc/net_proto.h`

Defines the protocol identifier for the User Datagram Protocol (UDP) within the caOS network stack.

Valore: `17U`

Note:
This constant is set to 17U, which is the standard IANA-assigned protocol number for UDP. It is used by the network layer to identify incoming packets and to populate the protocol field in

outgoing IP headers. It has no dependencies on other flags but must match the value expected by the underlying hardware abstraction layer (HAL) for packet filtering.

Uso:
You should never change this value as it is defined by the IPv4 protocol standard. It is used primarily when creating new socket structures or debugging packet headers in the network stack. If you are implementing a custom protocol, do not modify this constant; instead, define a new constant for your specific protocol ID.

`IP_TX_MTU_DEFAULT`

File: `caos/net/inc/net_proto.h`

Defines the default Maximum Transmission Unit (MTU) in bytes for outgoing IP packets.

Valore: `1500U`

Note:
This value sets the upper limit for the payload size of an IP packet before fragmentation occurs. It must be less than or equal to the physical interface's maximum frame size (e.g., 1500 for standard Ethernet). Reducing this value can decrease memory usage per buffer but may increase CPU overhead due to packet fragmentation.

Uso:
Adjust this value if your network hardware or specific protocol requirements (such as VPN tunnels or specific link-layer constraints) mandate a smaller MTU. Ensure that the system's memory pool for network buffers is large enough to accommodate this size plus protocol headers; setting this too high without sufficient RAM will lead to buffer overflows or allocation failures.

`ITM_BAUDRATE`

File: `caos/inc/itm.h`

Defines the transmission speed in bits per second for the Instrumentation Trace Macrocell (ITM) debug output.

Valore: `2000000UL`

Note:
This value sets the clock frequency for the SWO (Serial Wire Output) pin. It must match the baud rate configured in your debugger software (e.g., STM32CubeIDE or ST-Link Utility). If the value is too high for your hardware probe or trace clock settings, you will see garbled text or no output in the console. Typical values range from 115200 to 2000000.

Uso:
Adjust this value if you are experiencing data corruption in your debug logs or if your specific hardware probe requires a lower speed for stability. Ensure that the system core clock (HCLK) is high enough to support the requested baud rate, as the ITM clock is derived from the CPU frequency.

`KSTORE_DATE_DAY`

File: `caos/crypto/inc/keys_store.h`

Extracts the day component from a packed date value used in the key storage system.

Valore: `((v) & 0xFFU)`

Note:
This macro performs a bitwise AND operation with 0xFF to isolate the lower 8 bits of a provided date variable. It assumes the date is stored in a packed format where the day occupies the least significant byte. It has no dependencies on other RTOS flags but relies on the input 'v' being a valid packed date integer.

Uso:
Use this macro when retrieving or parsing key metadata from the secure storage. Change this only if the underlying packed date structure changes its bit-field layout. Incorrect usage will result in corrupted date values being returned, potentially causing key expiration logic to fail.

`KSTORE_DATE_MONTH`

File: `caos/crypto/inc/keys_store.h`

Extracts the month component from a packed 16-bit date value stored in the key management system.

Valore: `(((v) » 8) & 0xFFU)`

Note:
This macro performs a bitwise right shift by 8 bits followed by a mask of 0xFF to isolate the month field. It assumes the input 'v' is a 16-bit integer where the month is stored in the second byte. It has no dependencies on other flags but requires the input data to follow the specific caOS packed date format (typically bits 8-15).

Uso:
Use this macro when retrieving or validating the expiration date of cryptographic keys stored in the system. Do not modify this macro unless the underlying packed date structure in the key store is redefined. Providing an incorrectly formatted input will result in an invalid month value, potentially causing key validation logic to fail.

`KSTORE_DATE_PACK`

File: `caos/crypto/inc/keys_store.h`

A macro used to pack a calendar date (year, month, day) into a single 32-bit unsigned integer for efficient storage.

Valore: `(((u32)(y) « 16) | ((u32)(m) « 8) | (u32)(d))`

Note:
The macro performs bit-shifting to place the year in the upper 16 bits, the month in the middle 8 bits, and the day in the lower 8 bits. It requires three arguments (y, m, d). It is typically used in the key management system to timestamp cryptographic keys. Ensure the year value does not exceed 65535 to avoid overflow into other bit fields.

Uso:
Use this macro when you need to store or compare key expiration dates within the caOS key store. It is essential for maintaining a compact memory footprint in the key database. Do not use this for general-purpose date arithmetic, as it is strictly a formatting tool for storage.

KSTORE_DATE_YEAR

File: caos/crypto/inc/keys_store.h

Extracts the year component from a packed 32-bit date representation used in the key storage system.

Valore: (((v) » 16) & 0xFFU)

Note:
This macro performs a bitwise right shift by 16 bits followed by a mask of 0xFF to isolate the year byte. It assumes the input 'v' is a 32-bit integer where the year is stored in bits 16-23. It is a dependency for any function that validates key expiration dates or performs time-based key rotation logic.

Uso:
Use this macro when you need to read or verify the year of a key stored in the secure vault. Do not modify the shift or mask values unless the underlying data structure of the key metadata is updated, as incorrect values will result in corrupted date parsing and potential security validation failures.

KSTORE_SVC_IV_SIZE

File: caos/crypto/inc/keystore_service.h

Defines the fixed byte size of the Initialization Vector (IV) used for cryptographic operations within the Keystore Service.

Valore: 16

Note:
This constant sets the buffer size for IVs in AES-based encryption/decryption routines. Changing this value requires a full recompile of the crypto module and all dependent services. It must match the requirements of the underlying hardware acceleration (e.g., STM32 AES peripheral) or software library implementation. Typical value is 16 bytes for AES-128/256 in CBC or GCM modes.

Uso:
Modify this only if the security protocol or the underlying cryptographic algorithm requires a different IV length. Setting an incorrect value will cause cryptographic operations to fail with invalid parameter errors or result in corrupted data during decryption. Ensure that the hardware crypto engine supports the new length before changing this constant.

KSTORE_SVC_KEY_SIZE

File: caos/crypto/inc/keystore_service.h

Defines the fixed byte length for cryptographic keys managed by the caOS Keystore Service.

Valore: 32

Note:
This constant sets the buffer size for key storage operations. Enabling this ensures memory alignment for AES-256 keys. It must be consistent across all modules accessing the keystore; changing this value requires a full system recompile and may invalidate existing stored keys in non-volatile memory.

Uso:
Change this value only if your security requirements dictate a different key length (e.g., 16 for AES-128). Ensure the hardware crypto peripheral supports the new size. Setting this too low will cause truncation errors, while setting it too high will waste precious SRAM and Flash resources.

`KSTORE_SVC_LABEL_SIZE`

File: `caos/crypto/inc/keystore_service.h`

Defines the maximum character length allowed for labels assigned to cryptographic keys in the caOS keystore.

Valore: `6`

Note:
This constant sets the fixed-size buffer allocation for key identifiers. Increasing this value will increase the memory footprint of every keystore entry. It must be consistent across all modules that interface with the keystore service to prevent buffer overflows or truncation errors. The default value is 6 bytes.

Uso:
Increase this value if your application requires more descriptive or longer unique names for cryptographic keys. Decrease it only if you are extremely constrained by RAM and your naming convention allows for shorter identifiers. Ensure that any external storage or communication protocols expecting this specific size are updated accordingly to avoid data corruption.

`KSTORE_SVC_NONCE_SIZE`

File: `caos/crypto/inc/keystore_service.h`

Defines the fixed byte length of the cryptographic nonce used by the Keystore Service.

Valore: `12`

Note:
This constant sets the size of the initialization vector (IV) or nonce for encryption operations. Changing this value affects the memory allocation of internal buffers and the compatibility with external cryptographic standards. It must match the requirements of the underlying cipher suite (e.g., AES-GCM). Typical values are 12 bytes for GCM mode, though it may vary based on the specific security protocol implemented.

Uso:
Modify this value only if the security protocol or the hardware-accelerated cryptographic engine requires a different nonce length. Setting an incorrect value will cause cryptographic operations to fail or result in invalid ciphertext. Ensure that any change is synchronized with the peer devices or backend systems that expect a specific nonce size.

`KSTORE_SVC_P1_SIZE`

File: `caos/crypto/inc/keystore_service.h`

Defines the size in bytes of the primary partition buffer used by the keystore service.

Valore: `16`

Note:
This constant determines the memory allocation for the primary keystore storage slot. Increasing this value allows for larger cryptographic keys or more metadata but directly increases the static RAM footprint of the keystore service. It must be a multiple of the underlying flash page size if the service performs direct hardware writes. Typical values range from 16 to 256 depending on the security module requirements.

Uso:
Adjust this value when you need to store larger keys (e.g., moving from AES-128 to RSA-2048) or when the keystore service reports buffer overflow errors. Warning: setting this value too low will cause key storage operations to fail, while setting it too high may lead to stack or heap exhaustion in memory-constrained STM32 devices.

`KSTORE_SVC_P2_SIZE`

File: `caos/crypto/inc/keystore_service.h`

Defines the size of the secondary buffer (P2) used by the keystore service for cryptographic operations.

Valore: `16`

Note:
This constant sets the memory allocation in bytes for the secondary keystore processing stage. Increasing this value allows for larger cryptographic payloads or more complex key derivation functions but directly consumes more RAM from the system heap. It must be large enough to accommodate the maximum expected output size of the configured hash or encryption algorithms. Typical values range from 16 to 128 depending on the security requirements.

Uso:
Adjust this value if you encounter buffer overflow errors or 'insufficient memory' faults during cryptographic operations. Increase it if you switch to stronger encryption standards that require larger intermediate buffers. Do not set this value unnecessarily high to avoid wasting precious RAM on resource-constrained STM32 devices.

`KSTORE_SVC_TAG_SIZE`

File: `caos/crypto/inc/keystore_service.h`

Defines the fixed byte size for the authentication tag used in the keystore service cryptographic operations.

Valore: `16`

Note:
This constant sets the length of the Message Authentication Code (MAC) or tag appended to encrypted data. Changing this value directly affects the memory layout of the keystore storage structures. It must match the tag size expected by the underlying hardware cryptographic peripheral or software library. Typical values are 16 bytes for AES-GCM or similar standards; values other than 16 may cause decryption failures or buffer overflows if the storage buffers are not sized accordingly.

Uso:
Change this value only if the security protocol or the cryptographic algorithm used by the keystore service is updated to require a different tag length. Ensure that all existing stored keys

are re-encrypted or cleared before changing this value, as a mismatch will render stored data unreadable. Incorrect values will lead to integrity check failures during key retrieval.

`LPUART_STD_BAUD`

File: `os_config.h`

Defines the default baud rate for the Low Power UART (LPUART) peripheral in the caOS RTOS.

Valore: `115200U`

Note:
This constant sets the communication speed in bits per second. It directly affects the clock prescaler calculation for the LPUART hardware. Ensure this value matches the baud rate configured on your terminal emulator (e.g., PuTTY or Tera Term). It is typically set to 115200U, but valid ranges depend on the specific STM32 LPUART clock source frequency.

Uso:
Change this value if your hardware interface requires a different communication speed (e.g., 9600 or 921600). If you set this incorrectly, you will see garbage characters or no output at all on your serial console. Always verify that the system clock configuration supports the requested baud rate to avoid framing errors.

`LPUART_STD_WAKE`

File: `os_config.h`

Enables the Low-Power UART (LPUART) wake-up capability for the system.

Valore: `1`

Note:
When set to 1, the LPUART peripheral is configured to trigger an interrupt that wakes the MCU from Stop or Standby power modes upon receiving data. This requires the LPUART clock source to be set to HSI or LSE to remain active during low-power states. If set to 0, the LPUART will not wake the system, potentially causing data loss if the MCU is in a deep sleep state.

Uso:
Enable this flag when your application needs to remain in a low-power state while waiting for incoming serial commands. Change this to 0 if you are not using LPUART or if you want to prevent serial traffic from waking the device to save power. Ensure your hardware design supports LPUART wake-up pins before enabling.

`MAX`

File: `caos/inc/common.h`

A macro that returns the greater of two provided values.

Valore: `(((a) > (b)) ? (a) : (b))`

Note:
This is a standard ternary-based comparison macro. It evaluates both arguments, so avoid passing expressions with side effects (e.g., 'MAX(x++, y)') as they will be executed twice. It

is type-agnostic but relies on the compiler to handle implicit type promotion between the two arguments.

Uso:
Use this for calculating buffer sizes, timeout durations, or threshold comparisons. Always ensure the arguments are of compatible types to prevent unexpected compiler warnings or truncation errors. Avoid using this with function calls that modify global state.

`MAX_SECURE_KEYS`

File: `caos/crypto/inc/keys_store.h`

Defines the maximum number of cryptographic keys that can be stored in the caOS secure key vault.

Valore: `20`

Note:
This constant determines the size of the static array allocated in RAM for key management. Increasing this value consumes more memory (sizeof(key_t) * MAX_SECURE_KEYS). It must be greater than or equal to the total number of unique keys required by your application tasks. If set too low, key registration functions will return an error code (ERR_KEY_STORAGE_FULL).

Uso:
Increase this value if your application needs to handle more concurrent encrypted connections or secure storage slots than the default 20. Ensure that your STM32 device has sufficient RAM before increasing this limit, as it directly impacts the heap/static memory footprint.

`MAX_TASK`

File: `hw/f1xx/f103/objmap.h`
File: `hw/f4xx/f407/objmap.h`
File: `hw/f4xx/f429/objmap.h`
File: `hw/f4xx/f446/objmap.h`
File: `hw/f7xx/f746/objmap.h`
File: `hw/h5xx/h563/objmap.h`
File: `hw/h7xx/h747/objmap.h`
File: `hw/h7xx/h753/objmap.h`

Defines the maximum number of concurrent tasks allowed in the caOS scheduler.

Valori per MCU:

MCU	Valore
F103	5
F407	23
F429	23
F446	23
F746	90
H563	90
H747	93
H753	93

Note:
This constant determines the size of the internal Task Control Block (TCB) array. Increasing this value consumes more static RAM, as each task requires a dedicated memory slot regardless

of whether it is currently active. It must be set to a value greater than or equal to the total number of tasks created in your application. If you attempt to create more tasks than this limit, the system will trigger an assertion or return an error code during initialization.

Uso:
Adjust this value when adding or removing tasks from your project. Set it to the exact number of tasks you intend to run plus a small buffer for future expansion. Do not set this value excessively high, as it will waste precious RAM on your STM32 microcontroller. Always verify that your total RAM usage remains within the limits of your specific MCU model after increasing this constant.

`MAX_TIMER`

File: `hw/f1xx/f103/objmap.h`
File: `hw/f4xx/f407/objmap.h`
File: `hw/f4xx/f429/objmap.h`
File: `hw/f4xx/f446/objmap.h`
File: `hw/f7xx/f746/objmap.h`
File: `hw/h5xx/h563/objmap.h`
File: `hw/h7xx/h747/objmap.h`
File: `hw/h7xx/h753/objmap.h`

Defines the maximum number of software timers that can be active simultaneously in the caOS system.

Valori per MCU:

MCU	Valore
F103	5
F407	14
F429	14
F446	14
F746	32
H563	32
H747	32
H753	32

Note:
This constant allocates a static array in memory to track timer control blocks. Increasing this value consumes more RAM (sizeof(Timer_t) * MAX_TIMER). It is hardware-dependent; higher-performance MCUs like the H7 series support more timers due to larger memory availability. Ensure this value is greater than or equal to the total number of unique software timers initialized in your application code.

Uso:
Increase this value if your application requires more concurrent software timers and you encounter 'Timer Allocation Failed' errors during runtime. Decrease it if you need to optimize RAM usage on memory-constrained devices like the F103. Do not set this value arbitrarily high, as it wastes static memory.

`MAX_TOTAL_KEY_STORAGE`

File: `caos/crypto/inc/keys_store.h`

Defines the maximum total memory in bytes allocated for storing cryptographic keys within the system.

Valore: 2048

Note:
This constant determines the static buffer size for the key management module. Increasing this value consumes more RAM from the heap or static data segment. It must be large enough to accommodate the sum of all individual key sizes plus internal metadata overhead. If set too low, key registration functions will return an 'out of memory' error.

Uso:
Adjust this value when your application needs to store more keys or larger key types (e.g., switching from AES-128 to RSA-2048). Ensure the total size does not exceed the available RAM on your STM32 target. Always verify that the system heap remains sufficient for other RTOS tasks after increasing this constant.

MCU_HAS_ETH

File: hw/f1xx/f103/objmap.h
File: hw/f4xx/f407/objmap.h
File: hw/f4xx/f429/objmap.h
File: hw/f4xx/f446/objmap.h
File: hw/f7xx/f746/objmap.h
File: hw/h5xx/h563/objmap.h
File: hw/h7xx/h747/objmap.h
File: hw/h7xx/h753/objmap.h

A boolean flag indicating whether the target microcontroller hardware includes an integrated Ethernet MAC peripheral.

Valori per MCU:

MCU	Valore
F103	0U
F407	1U
F429	1U
F446	0U
F746	1U
H563	1U
H747	1U
H753	1U

Note:
When set to 1U, the caOS network stack and hardware abstraction layer (HAL) will initialize Ethernet-related registers and interrupt vectors. When set to 0U, these features are disabled to save memory and prevent illegal memory access. This flag must match the specific STM32 part number; enabling it on a chip without an Ethernet controller will cause a hard fault during system initialization.

Uso:
Change this value only when porting the project to a different STM32 MCU variant. If your application requires network connectivity, ensure this is set to 1U for supported chips (e.g., F407, H7 series). If you are using a chip without Ethernet (e.g., F103, F446), this must remain 0U, or the build will fail during the linking stage due to missing peripheral definitions.

`MCU_HAS_GPDMA`

File: `hw/f1xx/f103/objmap.h`
File: `hw/f4xx/f407/objmap.h`
File: `hw/f4xx/f429/objmap.h`
File: `hw/f4xx/f446/objmap.h`
File: `hw/f7xx/f746/objmap.h`
File: `hw/h5xx/h563/objmap.h`
File: `hw/h7xx/h747/objmap.h`
File: `hw/h7xx/h753/objmap.h`

A boolean flag indicating if the target microcontroller includes a General Purpose Direct Memory Access (GPDMA) controller.

Valori per MCU:

MCU	Valore
F103	0U
F407	0U
F429	0U
F446	0U
F746	0U
H563	1U
H747	0U
H753	0U

Note:
When set to 1U, the caOS kernel enables support for the GPDMA peripheral, which is distinct from the older DMA controllers found in F4/F7 series. Enabling this requires the hardware to support the GPDMA architecture (e.g., STM32H5 series). If set to 0U, the system will use standard DMA or polling methods. This flag is hardware-dependent and must match the specific MCU datasheet.

Uso:
Change this value only when porting caOS to a new STM32 series. Set to 1U if your MCU features the newer GPDMA block to allow the OS to offload data transfers efficiently. Setting this to 1U on an MCU that lacks GPDMA will result in compilation errors or hard faults during peripheral initialization.

`MCU_HAS_QSPI`

File: `hw/f1xx/f103/objmap.h`
File: `hw/f4xx/f407/objmap.h`
File: `hw/f4xx/f429/objmap.h`
File: `hw/f4xx/f446/objmap.h`
File: `hw/f7xx/f746/objmap.h`
File: `hw/f7xx/f767/objmap.h`
File: `hw/h5xx/h563/objmap.h`
File: `hw/h7xx/h747/objmap.h`
File: `hw/h7xx/h753/objmap.h`

A boolean flag indicating if the target STM32 microcontroller features a hardware Quad-SPI (QSPI) peripheral.

Valori per MCU:

MCU	Valore
F103	0U
F407	0U
F429	0U
F446	1U
F746	1U
F767	1U
H563	1U
H747	1U
H753	1U

Note:
When set to 1U, the caOS kernel enables drivers and memory-mapped access routines for external QSPI flash memory. When set to 0U, these features are disabled to save code space and prevent compilation errors on MCUs lacking the peripheral. This flag must match the specific hardware capabilities of the selected MCU; enabling it on an unsupported chip will cause linker errors or hard faults.

Uso:
Set this to 1U only if your specific STM32 model includes a QSPI controller and you intend to interface with external flash memory. If you are using an F103 or F407 series chip, this must remain 0U. Always verify your MCU datasheet's peripheral list before changing this value to avoid undefined behavior.

`MCU_HAS_RNG`

File: `hw/f1xx/f103/objmap.h`
File: `hw/f4xx/f407/objmap.h`
File: `hw/f4xx/f429/objmap.h`
File: `hw/f4xx/f446/objmap.h`
File: `hw/f7xx/f746/objmap.h`
File: `hw/h5xx/h563/objmap.h`
File: `hw/h7xx/h747/objmap.h`
File: `hw/h7xx/h753/objmap.h`

A boolean flag indicating whether the target STM32 microcontroller has a built-in hardware Random Number Generator (RNG) peripheral.

Valori per MCU:

MCU	Valore
F103	0U
F407	1U
F429	1U
F446	0U
F746	1U
H563	1U
H747	1U
H753	1U

Note:
When set to 1U, the caOS kernel will utilize the hardware RNG for entropy-based features such as secure task stack canaries or cryptographic seeding. If set to 0U, the system will fall back to a software-based pseudo-random number generator or disable RNG-dependent features. This flag must match the specific hardware capabilities of the MCU defined in the datasheet; enabling it

on an MCU without an RNG peripheral will cause a hard fault or undefined behavior during peripheral initialization.

Uso:
Set this value based on the specific STM32 series being used. You should change this only when porting the project to a different MCU model. If you are unsure, check the 'Reference Manual' for your specific STM32 part under the 'RNG' section. Incorrect configuration will lead to initialization errors or failure of security-related features.

`MCU_HAS_USB`

File: `hw/f1xx/f103/objmap.h`
File: `hw/f4xx/f407/objmap.h`
File: `hw/f4xx/f429/objmap.h`
File: `hw/f4xx/f446/objmap.h`
File: `hw/f7xx/f746/objmap.h`
File: `hw/h5xx/h563/objmap.h`
File: `hw/h7xx/h747/objmap.h`
File: `hw/h7xx/h753/objmap.h`

A hardware abstraction flag that indicates whether the target STM32 microcontroller includes a built-in USB peripheral.

Valori per MCU:

MCU	Valore
F103	0U
F407	1U
F429	1U
F446	1U
F746	1U
H563	1U
H747	1U
H753	1U

Note:
When set to 1U, the caOS kernel enables USB-related drivers and memory allocation for USB stacks. When set to 0U, all USB-related code is excluded from the build to save flash memory. This flag is strictly dependent on the specific MCU part number; enabling it on an MCU without USB hardware will cause compilation errors or hard faults during peripheral initialization.

Uso:
Change this value only when porting caOS to a new STM32 chip. You must set this to 1U if your application requires USB communication (e.g., CDC, HID, or MSC). If you are working on a chip like the F103, ensure this is set to 0U to prevent the compiler from attempting to access non-existent USB registers.

`MCU_NO_ETH`

File: `hw/f1xx/f103/objmap.h`
File: `hw/f4xx/f446/objmap.h`

A configuration flag that disables the Ethernet peripheral driver and stack within the caOS RTOS.

Valori per MCU:

MCU	Valore
F103	1
F446	1

Note:
When set to 1, the Ethernet hardware abstraction layer and associated network buffers are excluded from the build. This reduces the memory footprint (RAM and Flash) and prevents the RTOS from attempting to initialize non-existent or unused network hardware. It is independent of other peripherals but must be set to 1 for STM32 models like F103 and F446 that lack integrated Ethernet MAC controllers.

Uso:
Set this to 1 for any STM32 target that does not have a built-in Ethernet peripheral to avoid compilation errors or runtime faults. You should only set this to 0 if you are using a specific MCU variant that supports Ethernet and you intend to use the networking stack. Setting this to 0 on an unsupported MCU will lead to undefined behavior or hard faults during system initialization.

`MDNS_CACHE_FLUSH`

File: `caos/net/inc/net_proto.h`

Defines the bitmask flag used to trigger an immediate flush of the mDNS resource record cache.

Valore: `0x8000U`

Note:
When this flag is set in the mDNS control register, the system clears all cached mDNS entries to force a re-discovery of network services. It is a bitwise constant (0x8000U) and should be used in conjunction with the mDNS command interface. It has no dependencies on other flags but requires the mDNS stack to be initialized.

Uso:
Use this flag when you need to resolve stale network service information or after a significant network topology change. Do not set this flag continuously in a loop, as it will cause excessive network traffic and CPU overhead due to constant re-querying of the local network.

`MDNS_DST_IP`

File: `caos/net/inc/net_proto.h`

Defines the IPv4 multicast destination address used for mDNS (Multicast DNS) service discovery packets.

Valore: `{224U,0U,0U,251U}`

Note:
The value represents the standard mDNS multicast address 224.0.0.251 in an array format. Modifying this will cause the device to stop responding to or sending mDNS queries on the standard network, effectively isolating it from local service discovery. It must remain as {224U, 0U, 0U, 251U} for standard network compatibility.

Uso:

Change this only if you are operating on a non-standard, isolated network that requires a custom multicast group for service discovery. Incorrect values will result in the device being invisible to other network nodes using standard mDNS protocols.

`MDNS_DST_MAC`

File: `caos/net/inc/net_proto.h`

Defines the standard IPv4 multicast MAC address used for mDNS discovery packets.

Valore: `{0x01U,0x00U,0x5EU,0x00U,0x00U,0xFBU}`

Note:
This constant represents the IEEE 802.3 multicast address 01:00:5E:00:00:FB. It is required by the network stack to filter incoming mDNS traffic at the hardware level. Changing this value will cause the device to stop receiving mDNS queries and announcements, effectively making the device invisible to network discovery services.

Uso:
Do not modify this value unless you are implementing a custom or non-standard multicast protocol. It is used by the caOS network driver to configure the Ethernet MAC address filter. Ensure that the hardware peripheral is initialized to accept this multicast group, otherwise, the stack will not process mDNS packets.

`MDNS_PORT`

File: `caos/net/inc/net_proto.h`

Defines the UDP port number used for mDNS (Multicast DNS) service discovery.

Valore: `5353U`

Note:
This constant sets the destination port for mDNS packets. It must be set to 5353 to comply with the RFC 6762 standard. Changing this value will prevent the device from being discovered by standard mDNS clients like Bonjour or Avahi. It is a dependency for the caOS network stack's zero-configuration networking module.

Uso:
Keep this value at 5353U for standard network interoperability. Only change this value if you are operating in a private, isolated network environment that requires a non-standard port for custom service discovery protocols. Incorrect values will result in the device becoming invisible to network discovery tools.

`MDNS_TTL`

File: `caos/net/inc/net_proto.h`

Defines the Time-To-Live (TTL) value in seconds for mDNS resource records.

Valore: `120U`

Note:
This constant determines how long network peers should cache the mDNS service information before re-querying. The value is an unsigned integer, typically set to 120 seconds. Lowering this

value increases network traffic due to more frequent queries, while increasing it reduces traffic but may cause stale service information if the device IP or status changes.

Uso:
Adjust this value if your device frequently changes its network configuration or if you notice other devices on the network failing to discover your service after a reconnection. A value of 120 is a standard balance for most local networks; avoid setting this to extremely low values (e.g., < 10) to prevent network congestion.

`MIN`

File: `caos/inc/common.h`

A macro that returns the smaller of two provided values.

Valore: `(((a) < (b)) ? (a) : (b))`

Note:
This is a functional macro that performs a comparison between two inputs. It is side-effect prone if the arguments contain increment or decrement operators (e.g., MIN(a++, b)), as the chosen argument will be evaluated twice. It has no dependencies on other RTOS flags and works with any comparable data type.

Uso:
Use this macro to safely determine the minimum of two variables without writing manual if-else blocks. Warning: Avoid passing expressions with side effects as arguments to prevent unexpected behavior, such as double-incrementing a counter.

`MIN_KEY_SIZE`

File: `caos/crypto/inc/keys_store.h`

Defines the minimum allowed length in bytes for cryptographic keys stored in the system.

Valore: `16`

Note:
This constant enforces a security floor for key generation and storage. Setting this value higher increases security but may cause key generation functions to fail if the requested entropy or algorithm requires smaller keys. It is typically set to 16 (128 bits) to comply with AES standards. Ensure that all hardware-accelerated crypto peripherals support the chosen minimum size.

Uso:
Increase this value if your security policy requires stronger encryption (e.g., 32 for 256-bit keys). Decrease it only if you are using legacy algorithms that explicitly require shorter keys. Warning: setting this too low may expose the system to brute-force attacks, while setting it too high may cause runtime errors in modules that expect smaller key buffers.

`MODBUS_RX_BUF_SIZE`

File: `os_config.h`

Defines the maximum size in bytes for the Modbus reception buffer.

Valore: `270U`

Note:
This constant determines the memory allocation for incoming Modbus frames. The value must be large enough to accommodate the maximum expected PDU size plus any protocol overhead. It is typically set to 270 to support the Modbus standard maximum frame size of 256 bytes plus additional framing bytes. Increasing this value consumes more RAM, while setting it too low will cause frame truncation and CRC errors.

Uso:
Adjust this value if your application needs to handle larger custom Modbus frames or if you need to optimize RAM usage on memory-constrained devices. Ensure the value is at least 256. If you encounter 'Buffer Overflow' or 'Invalid Frame' errors during communication, verify that this size is sufficient for the specific Modbus function codes being used.

`MODBUS_TIMEOUT_MS`

File: `os_config.h`

Defines the maximum duration in milliseconds to wait for a response from a Modbus slave device.

Valore: `1000U`

Note:
This constant sets the timeout threshold for the Modbus master state machine. If no valid response is received within this period, the transaction is marked as failed. It depends on the underlying UART baud rate and the physical latency of the bus. Typical values range from 100ms to 2000ms depending on the network topology.

Uso:
Increase this value if you experience frequent 'timeout' errors on long or noisy communication lines. Decrease it if you need faster error recovery in high-speed, reliable networks. Setting this value too low will cause valid responses to be discarded, while setting it too high will make the system unresponsive during communication failures.

`MPU_ACCESS_NONE`

File: `caos/inc/mpu.h`

A configuration constant representing a memory region with no read, write, or execute permissions.

Valore: `0`

Note:
When applied to an MPU region, this setting triggers a MemManage fault if the CPU attempts to access the memory address. It is the default state for uninitialized or restricted memory segments. It has no dependencies on other flags but acts as the base security level for the MPU configuration.

Uso:
Use this value when defining MPU regions that should be completely inaccessible to tasks to prevent buffer overflows or unauthorized memory access. Do not use this for memory regions where your application code or data resides, as it will cause the system to crash immediately upon access.

`MPU_ACCESS_RDONLY`

File: `caos/inc/mpu.h`

Defines the memory protection unit (MPU) attribute for read-only memory regions.

Valore: `1`

Note:
When enabled, this flag configures the hardware MPU to trigger a memory management fault if a write operation is attempted on the associated memory region. It is typically used in conjunction with memory region definition structures. It depends on the hardware support for MPU on the specific STM32 series being used.

Uso:
Use this flag when defining memory regions for constants, flash-based code, or read-only lookup tables to prevent accidental data corruption. Ensure that the memory region is not intended to be modified at runtime; setting this on a RAM region that requires updates will cause the system to crash with a MemManage fault.

`MPU_ACCESS_RDWR`

File: `caos/inc/mpu.h`

Defines the Memory Protection Unit (MPU) access permission level for Read and Write operations.

Valore: `2`

Note:
When set to 2, this constant configures the MPU region to allow both read and write access to the associated memory block. It is typically used in conjunction with other MPU configuration flags to define memory attributes. Disabling or changing this value incorrectly will trigger a MemManage fault if the CPU attempts to write to a region restricted to read-only or no-access.

Uso:
Change this value when you need to grant or restrict write permissions to specific memory regions, such as shared buffers or peripheral registers. Use this when you encounter hard faults during memory access operations. Ensure that the memory region is properly aligned and sized according to the STM32 MPU hardware requirements before applying this flag.

`MPU_CANARY_REGION_MSP`

File: `caos/inc/mpu.h`

Enables or disables the MPU memory protection region specifically for the Main Stack Pointer (MSP) stack overflow detection.

Valore: `0`

Note:
When set to 1, the MPU configures a hardware-enforced guard zone at the end of the MSP stack; any write access to this region triggers a MemManage fault. When set to 0, this hardware protection is disabled. This flag depends on MPU_ENABLED being set to 1. Valid values are 0 (disabled) or 1 (enabled).

Uso:

Enable this (set to 1) during development and debugging to catch stack overflows immediately via hardware exceptions. Disable it (set to 0) only if you need to reclaim the MPU region for other memory mapping purposes or if you are running on hardware that does not support sufficient MPU regions. Warning: disabling this removes a critical safety mechanism that prevents silent memory corruption.

`MPU_CANARY_REGION_PSP_END`

File: `caos/inc/mpu.h`

Defines the MPU region index used to protect the end of the Process Stack Pointer (PSP) memory area.

Valore: `2`

Note:
This constant sets the specific hardware MPU slot (0-7) reserved for stack overflow detection. Enabling this requires the MPU to be active and the stack memory to be aligned to the MPU region size requirements. If set to an index already used by another memory region, it will cause a configuration conflict or system fault.

Uso:
Change this value only if the hardware memory map or the MPU region allocation scheme is modified. It is used to solve stack overflow issues by triggering a MemManage fault when the stack pointer exceeds the defined boundary. Ensure the chosen index does not overlap with Flash, RAM, or peripheral regions defined elsewhere in the system.

`MPU_CANARY_REGION_PSP_START`

File: `caos/inc/mpu.h`

Enables the MPU memory protection region for the Process Stack Pointer (PSP) start address to detect stack overflows.

Valore: `1`

Note:
When enabled, the MPU configures a hardware-enforced boundary at the beginning of the task stack. If the stack pointer moves below this address, a MemManage fault is triggered. This flag depends on MPU_ENABLED being set to 1. Typical value is 1 for production builds and 0 only for specific low-memory debugging scenarios.

Uso:
Enable this to prevent silent memory corruption caused by stack overflows. Change this to 0 only if you are performing advanced memory mapping that conflicts with the default stack placement, though this is strongly discouraged as it removes a critical safety mechanism.

`MPU_REGION_COUNT`

File: `hw/f1xx/f103/objmap.h`
File: `hw/f4xx/f407/objmap.h`
File: `hw/f4xx/f429/objmap.h`
File: `hw/f4xx/f446/objmap.h`
File: `hw/f7xx/f746/objmap.h`

File: `hw/h5xx/h563/objmap.h`
File: `hw/h7xx/h747/objmap.h`
File: `hw/h7xx/h753/objmap.h`

Defines the total number of hardware memory protection regions available on the specific STM32 microcontroller.

Valori per MCU:

MCU	Valore
F103	8U
F407	8U
F429	8U
F446	8U
F746	8U
H563	16U
H747	16U
H753	16U

Note:
This constant sets the limit for the Memory Protection Unit (MPU) configuration. Enabling this allows the RTOS to isolate tasks and protect memory areas. The value is hardware-dependent: F1/F4/F7 series typically support 8 regions, while H5/H7 series support 16. Setting this value higher than the hardware capability will cause MPU configuration errors or hard faults.

Uso:
You should change this value only when porting the RTOS to a different STM32 family. It is used by the kernel to allocate memory protection tables. Ensure the value matches the datasheet specification for your specific MCU part number; an incorrect value will lead to unpredictable system behavior or immediate crashes during kernel initialization.

`MQTT_KEEPALIVE_SEC`

File: `os_config.h`

Defines the interval in seconds at which the MQTT client sends a PINGREQ packet to the broker to maintain the connection.

Valore: `60U`

Note:
This value determines the timeout period for the broker to consider the client disconnected. If set too low, it increases network traffic and power consumption; if set too high, the system may take longer to detect a dropped connection. It must be less than the broker's maximum allowed keep-alive threshold. Typical values range from 30 to 300 seconds.

Uso:
Adjust this value if the broker frequently disconnects the device or if you need to optimize for low-power battery operation. Ensure the value is synchronized with the broker's configuration to prevent unexpected session termination.

`MQTT_RX_BUF_SIZE`

File: `os_config.h`

Defines the maximum size in bytes of the buffer used to store incoming MQTT messages.

Valore: `256U`

Note:
This value determines the memory footprint of the MQTT task. If set too low, incoming packets exceeding this size will be truncated or dropped. It must be large enough to accommodate the largest expected MQTT payload plus protocol headers. Typical values range from 256 to 2048 depending on available RAM and application requirements.

Uso:
Increase this value if you observe incomplete message reception or errors when receiving large MQTT payloads. Decrease it if your system is running low on heap memory and you are only handling small control messages. Ensure the value is a multiple of 4 for optimal memory alignment on STM32 architectures.

`NET_BUF_COUNT`

File: `hw/f1xx/f103/objmap.h`
File: `hw/f4xx/f407/objmap.h`
File: `hw/f4xx/f429/objmap.h`
File: `hw/f4xx/f446/objmap.h`
File: `hw/f7xx/f746/objmap.h`
File: `hw/f7xx/f767/objmap.h`
File: `hw/h5xx/h563/objmap.h`
File: `hw/h7xx/h747/objmap.h`
File: `hw/h7xx/h753/objmap.h`

Defines the total number of network packet buffers available in the system memory pool.

Valori per MCU:

MCU	Valore
F103	0
F407	8
F429	16
F446	8
F746	64
F767	64
H563	64
H747	64
H753	64

Note:
This constant determines the maximum number of concurrent network packets the stack can hold. Increasing this value consumes more RAM (each buffer is typically 1536 bytes). If set to 0, networking features are disabled. Values must be chosen based on available SRAM; for example, F7/H7 series support higher counts due to larger memory capacity, while F103 lacks sufficient resources for network buffering.

Uso:
Increase this value if the system experiences 'packet dropped' errors or fails to handle high-traffic bursts. Decrease it if the application encounters hard faults or memory allocation failures during startup. Ensure the value is sufficient for your application's throughput requirements, but do not exceed available heap space.

NET_BUF_DATA_SIZE

File: caos/net/inc/net_buf.h

Defines the actual payload capacity in bytes for a network buffer by subtracting the header overhead from the total buffer size.

Valore: (NET_BUF_SIZE - NET_BUF_HDR_SIZE)

Note:
This constant is derived from NET_BUF_SIZE and NET_BUF_HDR_SIZE. It represents the maximum amount of application data that can fit into a single network packet. If NET_BUF_SIZE is too small, this value may become negative or insufficient for standard MTU requirements, potentially causing memory corruption or packet fragmentation issues.

Uso:
Adjust this value only if you need to change the total buffer size (NET_BUF_SIZE) to accommodate larger network packets. Ensure that the resulting size is sufficient for your protocol's MTU (e.g., 1500 bytes for Ethernet). Always verify that the total memory allocated for buffers does not exceed the available RAM on the STM32.

NET_BUF_F_IN_USE

File: caos/net/inc/net_buf.h

A bitmask flag indicating that a specific network buffer is currently allocated and being used by the system.

Valore: (1U « 0)

Note:
When this bit is set in the buffer's status field, the memory manager considers the buffer busy and will not reallocate it. If cleared, the buffer is marked as free for reuse. This flag is typically checked by the network stack's allocation functions to prevent memory leaks or data corruption. It is a fundamental flag for the buffer management lifecycle.

Uso:
You should never manually toggle this flag in your application code; it is managed internally by the caOS network stack. If you notice that network buffers are not being freed, check if your code is correctly passing buffers back to the stack after processing. Modifying this flag directly will lead to memory leaks or system crashes due to double-allocation of the same memory block.

NET_BUF_F_RAW

File: caos/net/inc/net_buf.h

Flag to disable automatic checksum calculation for a network buffer.

Valore: (1U « 2) /* send as is: no checksum insertion by

Note:
When this flag is set in the buffer flags, the caOS network stack skips the automatic insertion of checksums (e.g., TCP/UDP/IP checksums) before transmission. This is typically used when the application layer or a hardware offload engine is responsible for calculating the checksum, or when sending raw packets that do not require standard protocol checksums. It is a bitmask value (1U « 2).

Uso:
Use this flag when you are manually constructing a packet and have already calculated the checksum, or when you are sending custom protocol frames that do not follow standard checksum requirements. Warning: If you enable this flag without providing a valid checksum, the receiving device will likely drop the packet as corrupted.

`NET_BUF_F_TX`

File: `caos/net/inc/net_buf.h`

A bitmask flag indicating that the network buffer is currently designated for transmission (TX) operations.

Valore: `(1U ≪ 1)`

Note:
This flag is used by the network stack to distinguish between incoming (RX) and outgoing (TX) buffers. When set, the buffer is managed by the transmission logic and should not be modified by the RX interrupt service routines. It is typically used in conjunction with buffer allocation functions and must be cleared once the transmission is complete to allow the buffer to be recycled.

Uso:
Use this flag when manually allocating or inspecting buffers within the network stack to ensure the buffer is correctly routed to the hardware DMA or MAC peripheral. Do not manually toggle this bit unless you are implementing custom network drivers, as incorrect state management will lead to memory leaks or buffer corruption in the network pool.

`NET_BUF_HDR_SIZE`

File: `caos/net/inc/net_buf.h`

Defines the fixed size in bytes reserved for the network buffer header metadata.

Valore: `64U`

Note:
This value determines the memory offset where the actual payload data begins within a network buffer. Increasing this value consumes more RAM per buffer but allows for larger protocol headers (e.g., adding custom tunneling or encryption headers). It must be large enough to accommodate the stack's internal headers; otherwise, packet corruption will occur. Typical values are powers of 2 to ensure memory alignment.

Uso:
Adjust this value if you are implementing custom network protocols that require additional metadata space before the payload. Do not decrease this value below the default unless you are certain the underlying network stack requires less header space, as doing so will cause buffer overflows and system instability.

`NET_BUF_HEADROOM`

File: `caos/net/inc/net_buf.h`

Defines the number of bytes reserved at the beginning of a network buffer for protocol headers.

Valore: `128U`

Note:
This value sets the offset where the actual payload data begins within a network packet buffer. Increasing this value allows for more layers of encapsulation (e.g., adding VLAN tags or tunneling headers) without reallocating the buffer. It must be large enough to accommodate the maximum expected stack of headers (Ethernet + IP + TCP/UDP + custom headers). Typical values are powers of 2, and it must be less than the total buffer size.

Uso:
Increase this value if you encounter buffer overflow errors or kernel panics when adding new protocol layers or tunneling protocols. Decrease it only if memory is extremely constrained and you are certain the header stack will never exceed the new limit. Setting this too low will cause memory corruption or packet drops when headers are prepended.

`NET_BUF_INVALID`

File: `caos/net/inc/net_buf.h`

A sentinel value representing an uninitialized or invalid network buffer handle.

Valore: `0xFFFFU`

Note:
This constant is used as a null-pointer equivalent for network buffer descriptors. When a function returns this value, it indicates that the requested buffer allocation failed or the buffer reference is no longer valid. It is defined as 0xFFFFU to ensure it falls outside the range of valid memory-mapped buffer indices.

Uso:
Use this constant to check the success of network buffer allocation functions. Always compare the returned handle against NET_BUF_INVALID before attempting to access buffer data to prevent memory corruption or system crashes. Do not modify this value unless the underlying buffer pool architecture is redefined.

`NET_BUF_SIZE`

File: `caos/net/inc/net_buf.h`

Defines the fixed size in bytes for each network packet buffer allocated by the caOS networking stack.

Valore: `2048U`

Note:
This value determines the maximum payload size a single network buffer can hold. Increasing this value consumes more RAM from the heap for every allocated buffer, which can lead to memory exhaustion if the pool size is not adjusted accordingly. It must be large enough to accommodate the Maximum Transmission Unit (MTU) of the underlying network interface, typically 1500 bytes for Ethernet plus protocol headers.

Uso:
Increase this value if you encounter packet fragmentation or if your application needs to handle jumbo frames. Decrease it only if you are extremely constrained by RAM and are certain that your network traffic will never exceed the smaller limit. Setting this value too low will cause incoming packets to be dropped or truncated, leading to communication failures.

`NET_HANDLER_MAX`

File: `caos/net/inc/net.h`

Defines the maximum number of concurrent network socket handlers allowed in the system.

Valore: `8U`

Note:
This constant determines the size of the internal static array used to track active network connections. Increasing this value consumes more RAM in the .bss section. It must be greater than or equal to the number of simultaneous TCP/UDP sockets your application intends to open. If set too low, new connection attempts will be rejected by the network stack.

Uso:
Adjust this value based on your application's memory constraints and communication requirements. Increase it if your application needs to handle more simultaneous connections (e.g., a web server with multiple clients). Decrease it to save RAM if your device only communicates with a single server. Ensure that the total memory usage does not exceed the STM32's available SRAM.

`NET_IFACE_INVALID`

File: `caos/net/inc/net_iface.h`

A sentinel value representing an uninitialized or non-existent network interface handle.

Valore: `0xFFU`

Note:
This constant is used as a null-equivalent for network interface pointers or IDs. When a function returns this value, it indicates that the requested interface could not be found or is not currently active. It is typically used in comparison checks against interface handles returned by the network stack.

Uso:
Use this constant to validate interface handles before performing operations like sending or receiving packets. Always check if your interface handle equals NET_IFACE_INVALID to prevent null pointer dereferences or accessing invalid memory regions. Do not use this value as a valid interface index in your application code.

`NET_IFACE_MAX`

File: `caos/net/inc/net_iface.h`

Defines the maximum number of network interfaces that can be registered simultaneously in the caOS network stack.

Valore: `2U`

Note:
This constant determines the size of the internal interface registry array. Increasing this value consumes more static RAM for the interface control blocks. It must be set to at least 1, and it is typically constrained by the available memory and the number of physical or virtual hardware peripherals (e.g., Ethernet, Wi-Fi, or PPP) present in the system.

Uso:

Increase this value if your application needs to support multiple network connections simultaneously, such as a dual-homed gateway or a system with both Ethernet and Wi-Fi. If you set this value higher than the number of actual hardware drivers initialized, you will waste RAM; if set too low, additional interfaces will fail to register during system startup.

NET_MAC_FILTER_MAX

File: caos/net/inc/net.h

Defines the maximum number of MAC addresses that can be stored in the network hardware filter list.

Valore: 2U

Note:
This constant determines the size of the internal array used by the network stack to manage MAC address filtering. Increasing this value consumes more static RAM. It is typically used in conjunction with ca_net_filter_mac_set to whitelist specific devices. The default value is 2U, which is sufficient for simple point-to-point or gateway configurations.

Uso:
Increase this value if your application needs to filter or whitelist more than two specific MAC addresses simultaneously. If you set this too high, you will waste memory; if you set it too low, calls to ca_net_filter_mac_set will fail once the limit is reached. Ensure you have enough RAM before increasing this constant.

NET_MPU_REGION

File: caos/net/inc/net_buf.h

Defines the specific MPU memory region index reserved for network buffer operations.

Valore: 3U

Note:
This constant sets the hardware MPU slot used to enforce memory protection on network packet buffers. It must be a valid MPU region index supported by the STM32 hardware (typically 0-7). Changing this value requires ensuring that the corresponding MPU configuration in the system startup code is updated to match, otherwise, the system will trigger a MemManage fault when accessing network buffers.

Uso:
Change this value only if the default MPU region conflicts with other system-critical memory protections or if the hardware abstraction layer requires a specific region for DMA-capable network memory. Ensure that the chosen region is not already assigned to the RTOS kernel stack or peripheral memory maps to avoid system instability.

NET_PROTO_ARP

File: caos/net/inc/net_buf.h

Enables or disables the Address Resolution Protocol (ARP) support in the caOS network stack.

Valore: 0x01U

Note:
When set to 0x01U, the system includes the logic to resolve IP addresses to MAC addresses. Disabling this (0x00U) will prevent the device from communicating with other nodes on a local Ethernet network unless static ARP entries are manually configured. This flag is a dependency for IPv4 networking modules.

Uso:
Enable this flag when your application needs to communicate over an Ethernet network using IPv4. Disable it only if you are implementing a custom, non-ARP based protocol or if you are operating in a point-to-point environment where ARP is explicitly not required to save memory and code space.

`NET_PROTO_IP4`

File: `caos/net/inc/net_buf.h`

Enables IPv4 protocol support within the caOS networking stack.

Valore: `0x02U`

Note:
When set to 0x02U, the system includes the IPv4 header processing logic and routing tables. Disabling this flag will remove IPv4 functionality, reducing the binary footprint but preventing communication over standard IPv4 networks. This flag is often a dependency for higher-level protocols like TCP or UDP if they are configured to run over IPv4.

Uso:
Enable this flag when your application needs to communicate over an IPv4 network. Change this value only if you are strictly using IPv6 or a custom non-IP protocol to save memory. Ensure that your network interface driver is compatible with IPv4 before enabling this, otherwise, the stack may attempt to process unsupported packets.

`NET_PROTO_IP6`

File: `caos/net/inc/net_buf.h`

Enables IPv6 protocol support within the caOS network stack.

Valore: `0x03U`

Note:
When set to 0x03U, the system allocates memory structures and enables logic for IPv6 packet processing. This flag is a dependency for any network services requiring IPv6 addressing. It must be consistent across all network modules to prevent memory alignment issues or stack overflows. Valid values are 0x00U (disabled) or 0x03U (enabled).

Uso:
Enable this flag when your application needs to communicate over IPv6 networks. Change this value if you are migrating from an IPv4-only environment to a dual-stack or IPv6-only setup. Warning: Enabling this increases the static memory footprint of the network buffer pool; ensure your heap size is sufficient before activation.

`NET_PROTO_NONE`

File: `caos/net/inc/net_buf.h`

A configuration flag indicating that no network protocol stack is currently initialized or active.

Valore: `0x00U`

Note:
When set to 0x00U, the network buffer management system bypasses all protocol-specific processing. This flag is typically used as a default state or a sentinel value in the network stack initialization sequence. It has no dependencies on other flags but must be checked before attempting to access protocol-specific headers or payload structures.

Uso:
Use this value during system initialization or when the network interface is explicitly disabled to prevent the stack from attempting to process incoming or outgoing packets. Changing this value incorrectly can lead to null pointer dereferences or memory corruption if the system expects a valid protocol handler to be present.

`NET_QUEUE_DEPTH`

File: `caos/net/inc/net_queue.h`

Defines the maximum number of network packets that can be buffered in the internal queue before dropping new incoming data.

Valore: `128U`

Note:
Increasing this value consumes more RAM from the heap, as each slot reserves space for a packet descriptor. It must be a power of two if the underlying implementation uses bitwise masking for circular buffer indexing. Typical values range from 16 to 256 depending on available memory and traffic burst intensity.

Uso:
Increase this value if the system experiences packet loss during high-traffic bursts or when the processing task is temporarily preempted by higher-priority threads. Decrease it if the system is running low on heap memory and network throughput requirements are low.

`NET_ROUTE_MAX`

File: `os_config.h`

Defines the maximum number of entries allowed in the network routing table.

Valore: `8U`

Note:
This constant determines the size of the static array used to store routing paths. Increasing this value consumes more RAM but allows the device to manage more complex network topologies or subnets. It must be a positive integer, typically ranging from 4 to 32 depending on available heap/stack memory.

Uso:

Increase this value if your device needs to communicate with multiple gateways or subnets and you encounter 'route not found' errors. Decrease it to save RAM if your device only communicates with a single local gateway.

`NTP_EPOCH_DELTA`

File: `caos/net/inc/net_proto.h`

The constant representing the number of seconds between the NTP epoch (January 1, 1900) and the Unix epoch (January 1, 1970).

Valore: `2208988800UL`

Note:
This value is used to convert NTP timestamps into standard Unix timestamps used by the system clock. It is a fixed constant of 2208988800 seconds. Changing this value will cause incorrect system time synchronization and will break all time-dependent network protocols like TLS/SSL certificate validation.

Uso:
This value should generally not be changed. It is used internally by the caOS network stack when parsing NTP packets. You only need to verify this value if you are porting the stack to a non-standard time reference or if you encounter significant time offsets during network synchronization.

`NTP_POLL_MS`

File: `os_config.h`

Defines the time interval in milliseconds between consecutive NTP server synchronization requests.

Valore: `64000U`

Note:
This value determines the frequency of network time updates. Increasing this value reduces network traffic and CPU overhead but may lead to larger clock drift between syncs. It depends on the network stack being initialized and the NTP client task being active. The value is an unsigned integer (U), typically set to 64000ms (64 seconds).

Uso:
Adjust this value if your application requires higher time precision (lower the value) or if you need to minimize bandwidth consumption (increase the value). Warning: Setting this value too low may flood the network or cause the NTP server to block your device; setting it too high may cause the system clock to drift significantly beyond acceptable tolerances for your application.

`NTP_PORT`

File: `caos/net/inc/net_proto.h`

Defines the UDP port number used for Network Time Protocol (NTP) synchronization.

Valore: `123U`

Note:

This constant sets the destination port for outgoing time sync requests and the listening port for incoming server responses. It must match the standard NTP port (123) to communicate with public or local time servers. Changing this value will prevent the system from synchronizing with standard NTP servers unless the server is specifically configured to listen on a non-standard port.

Uso:
Change this value only if your network architecture requires NTP traffic to be routed through a custom port or a proxy server. Ensure that your firewall settings and the target NTP server configuration are updated to match this value, otherwise, the system clock will fail to synchronize.

`NTP_RESPONSE_TIMEOUT_MS`

File: `os_config.h`

Defines the maximum duration in milliseconds to wait for an NTP server response before timing out.

Valore: `5000U`

Note:
This constant determines the blocking duration for the network stack during time synchronization. If set too low, the system may fail to sync on high-latency networks. If set too high, it may delay system startup or task execution. The value is an unsigned integer, typically ranging from 1000 to 10000 depending on network stability.

Uso:
Increase this value if the device frequently fails to synchronize time due to poor network conditions or high latency. Decrease it if the system startup time is too slow and you are operating on a reliable, low-latency local network. Ensure the value is sufficient to account for round-trip time (RTT) plus server processing time.

`NTP_RETRY_BACKOFF_MS`

File: `os_config.h`

Defines the base time interval in milliseconds for the exponential backoff mechanism during NTP synchronization failures.

Valore: `10000U`

Note:
This value acts as the multiplier for the retry delay. When NTP requests fail, the system waits for (NTP_RETRY_BACKOFF_MS * 2^n) milliseconds before the next attempt, where 'n' is the number of consecutive failures. It must be a positive unsigned integer. Setting this too low may flood the network or the NTP server, while setting it too high increases the time required to recover from transient network outages.

Uso:
Adjust this value if your device operates on high-latency networks or if the NTP server enforces strict rate-limiting. Increase this value if you observe 'NTP sync failed' errors due to network congestion. Ensure this value is not set to zero to avoid immediate, rapid-fire retry loops that could crash the network stack.

`NTP_RETRY_N`

File: `os_config.h`

Defines the maximum number of attempts the system will make to synchronize time with an NTP server before failing.

Valore: `3U`

Note:
When enabled, the system attempts to reach the NTP server up to this count. If the value is set to 0, the system will not attempt any retries. Increasing this value improves reliability in unstable network environments but increases the total boot or sync time if the server is unreachable. Typical values range from 1 to 5.

Uso:
Increase this value if your device frequently logs 'NTP Sync Failed' errors during startup or periodic updates due to poor network connectivity. Decrease it if you need the system to fail fast and proceed to other tasks without waiting for network timeouts.

`NTP_SERVER_N`

File: `os_config.h`

Defines the maximum number of NTP server addresses stored in the system configuration.

Valore: `2U`

Note:
This constant determines the size of the internal array used to hold NTP server hostnames or IP addresses. Increasing this value allows the system to attempt synchronization with multiple time servers for redundancy. It must be a positive integer. If set to 0, NTP functionality may be disabled or restricted to a hardcoded default.

Uso:
Change this value when you need to add more fallback NTP servers to improve time synchronization reliability. Ensure that the memory allocated for the NTP client task is sufficient to handle the increased array size. Setting this value too high may lead to unnecessary memory consumption in the heap or static data section.

`OS_TICK_US`

File: `hw/f1xx/f103/objmap.h`
File: `hw/f4xx/f407/objmap.h`
File: `hw/f4xx/f429/objmap.h`
File: `hw/f4xx/f446/objmap.h`
File: `hw/f7xx/f746/objmap.h`
File: `hw/h5xx/h563/objmap.h`
File: `hw/h7xx/h747/objmap.h`
File: `hw/h7xx/h753/objmap.h`

Defines the duration of a single system tick in microseconds, determining the OS heartbeat frequency.

Valori per MCU:

MCU	Valore
F103	CA_TICK_SPEED_2MS
F407	CA_TICK_SPEED_250US
F429	CA_TICK_SPEED_250US
F446	CA_TICK_SPEED_250US
F746	CA_TICK_SPEED_200US
H563	CA_TICK_SPEED_200US
H747	CA_TICK_SPEED_100US
H753	CA_TICK_SPEED_100US

Note:
This constant sets the resolution of the RTOS scheduler and software timers. Lower values (e.g., 100US) provide higher precision for task scheduling and timeouts but increase CPU overhead due to more frequent interrupt handling. It must be compatible with the hardware timer configuration. Typical values range from 100 to 2000 microseconds depending on the MCU performance class.

Uso:
Adjust this value to balance timing precision against CPU load. Increase the value if the system is overloaded by too many context switches. Decrease it only if your application requires high-resolution task execution or faster response times for software timers. Ensure all time-based logic in your application is scaled relative to this tick rate.

PACKET_TRACER_RING_SIZE

File: os_config.h

Defines the total number of slots available in the circular buffer used for packet tracing.

Valore: 8U

Note:
This constant determines the depth of the internal trace log. Increasing this value allows for a longer history of captured packets but consumes more RAM. It must be a power of two for optimal performance in the ring buffer logic. Typical values range from 4 to 64 depending on available heap memory.

Uso:
Increase this value if you are losing trace data during high-frequency communication bursts. Decrease it if the system is running low on static RAM. Ensure that the value is not set too high, as it may lead to stack overflow or memory exhaustion in resource-constrained STM32 devices.

PCLK1_FREQ_HZ

File: hw/f1xx/f103/objmap.h
File: hw/f4xx/f407/objmap.h
File: hw/f4xx/f429/objmap.h
File: hw/f4xx/f446/objmap.h
File: hw/f7xx/f746/objmap.h
File: hw/h5xx/h563/objmap.h
File: hw/h7xx/h747/objmap.h
File: hw/h7xx/h753/objmap.h

Defines the operating frequency of the Peripheral Clock 1 (PCLK1) bus in Hertz.

Valori per MCU:

MCU	Valore
F103	36000000UL
F407	42000000UL
F429	42000000UL
F446	42000000UL
F746	54000000UL
H563	250000000UL
H747	100000000UL
H753	120000000UL

Note:
This constant is used by the RTOS kernel and peripheral drivers to calculate timing-dependent parameters such as baud rates, timer prescalers, and watchdog timeouts. It must match the actual hardware clock configuration set in the RCC registers. Incorrect values will cause communication errors (e.g., wrong UART baud rate) or incorrect timing intervals.

Uso:
Update this value whenever the system clock tree configuration (RCC) is modified. It is critical for ensuring that peripherals like UART, SPI, and Timers operate at the intended speeds. Always verify this value against the STM32 CubeMX clock configuration tab for your specific target MCU.

`PCLK2_FREQ_HZ`

File: `hw/f4xx/f407/objmap.h`
File: `hw/f4xx/f429/objmap.h`
File: `hw/f4xx/f446/objmap.h`
File: `hw/f7xx/f746/objmap.h`
File: `hw/h5xx/h563/objmap.h`
File: `hw/h7xx/h747/objmap.h`
File: `hw/h7xx/h753/objmap.h`

Defines the operating frequency of the Peripheral Clock 2 (PCLK2) bus in Hertz.

Valori per MCU:

MCU	Valore
F407	84000000UL
F429	84000000UL
F446	84000000UL
F746	108000000UL
H563	250000000UL
H747	100000000UL
H753	120000000UL

Note:
This constant informs the RTOS kernel and peripheral drivers of the exact clock speed of the APB2 bus. It is critical for calculating baud rates for UART, SPI, and timer prescalers. Values must match the actual hardware clock tree configuration set in the RCC registers. Incorrect values will lead to incorrect peripheral timing, such as garbled serial communication or wrong PWM frequencies.

Uso:

Update this value whenever the system clock tree (RCC) is modified in the hardware abstraction layer or startup files. If you change the PLL settings or the APB2 prescaler, you must recalculate and update this constant. Failure to keep this in sync with the hardware will cause peripheral drivers to malfunction, even if the CPU core itself appears to be running correctly.

`PCLK3_FREQ_HZ`

File: `hw/h5xx/h563/objmap.h`
File: `hw/h7xx/h747/objmap.h`
File: `hw/h7xx/h753/objmap.h`

Defines the operating frequency in Hertz of the Peripheral Clock 3 (PCLK3) bus for the specific STM32 microcontroller.

Valori per MCU:

MCU	Valore
H563	250000000UL
H747	100000000UL
H753	120000000UL

Note:
This constant is used by the RTOS kernel and peripheral drivers to calculate timing-dependent parameters such as baud rates, timer intervals, and watchdog timeouts. Enabling this ensures hardware peripherals attached to the APB3 bus operate at the correct speed. Values must match the actual hardware clock tree configuration defined in the System Clock Configuration (RCC). Typical values range from 50MHz to 250MHz depending on the specific STM32 series and PLL settings.

Uso:
Change this value only when modifying the system clock tree (e.g., changing the PLL multipliers or dividers in the RCC configuration). If this value is incorrect, peripheral communication (like UART, SPI, or I2C) will fail due to incorrect timing calculations, and RTOS software timers may drift significantly. Always verify this value against the STM32CubeMX clock configuration tool output.

`PCLK4_FREQ_HZ`

File: `hw/h7xx/h747/objmap.h`
File: `hw/h7xx/h753/objmap.h`

Defines the operating frequency in Hertz of the Peripheral Clock 4 (PCLK4) bus for the STM32 MCU.

Valori per MCU:

MCU	Valore
H747	100000000UL
H753	120000000UL

Note:
This constant is used by the caOS hardware abstraction layer to calculate baud rates and timing intervals for peripherals connected to the APB4 bus. Enabling or changing this value directly affects the accuracy of communication interfaces like I2C, SPI, or timers mapped to this clock domain. It must match the actual clock tree configuration set in the System Clock Configuration (RCC) registers. Typical values are 100MHz for H747 and 120MHz for H753.

Uso:
Change this value only when modifying the system clock tree (e.g., changing the PLL settings or clock dividers). If this value is incorrect, peripherals on the APB4 bus will operate at the wrong speed, leading to communication errors, frame corruption, or timeout failures. Always verify this value against the STM32CubeMX clock configuration tool output.

`POOL_QUEUE_SIZE`

File: `os_config.h`

Defines the maximum number of message slots available in the system memory pool for inter-task communication.

Valore: `8`

Note:
This constant determines the total capacity of the global message queue. Increasing this value consumes more RAM from the heap. It must be greater than zero and is typically set to a power of two for optimal performance. If the queue fills up, tasks attempting to send messages will be blocked or return an error depending on the API call.

Uso:
Increase this value if your application experiences 'Queue Full' errors during high-traffic periods or when multiple tasks send messages simultaneously. Decrease it if your system is running low on RAM and you have verified that the current queue depth is never fully utilized.

`PRINTF`

File: `os_config.h`

Enables or disables the system-wide formatted output logging functionality via the ca_printf macro.

Valore: `ca_printf(STDOUT,BUFF,LEN,__VA_ARGS__)`

Note:
When enabled, this constant maps the standard printf-style calls to the caOS internal serial output driver. Disabling this flag removes the overhead of string formatting and buffer management, significantly reducing flash memory usage and CPU cycles. It depends on the UART/USART peripheral being initialized in the hardware abstraction layer.

Uso:
Enable this during development and debugging to monitor system state and task execution via a serial terminal. Disable this in production builds to save memory and prevent blocking the CPU with slow serial transmission. Ensure the buffer size (BUFF) is large enough to accommodate your longest log message, otherwise, output will be truncated.

`PRINTF_N`

File: `os_config.h`

Defines the macro used for system-wide formatted output logging.

Valore: `ca_printf_n(STDOUT,BUFF,LEN,__VA_ARGS__)`

Note:

This constant maps the standard printf functionality to the specific caOS output driver. Enabling or modifying this allows redirection of logs to UART, ITM, or memory buffers. It depends on the underlying hardware abstraction layer (HAL) being initialized. Typical values are function pointers or macros that wrap the hardware-specific transmit function.

Uso:
Change this value when porting the RTOS to new hardware or when switching the debug output interface (e.g., from UART to SWO). Ensure the provided function signature matches the expected arguments (STDOUT, buffer, length, and variadic arguments) to prevent memory corruption or hard faults during runtime logging.

`PSP_CHECK_TOP`

File: `caos/inc/mpu.h`

Enables or disables the stack overflow detection mechanism for the Process Stack Pointer (PSP).

Valore: `0`

Note:
When set to 1, the MPU (Memory Protection Unit) monitors the stack limit to trigger a fault if the stack pointer exceeds the defined boundary. When set to 0, this hardware-based protection is disabled. This flag requires the MPU to be initialized and configured in the system startup code. Typical values are 0 (disabled) or 1 (enabled).

Uso:
Enable this flag (set to 1) during development and debugging to catch stack overflow errors early, which otherwise cause silent memory corruption or hard faults. Disable it (set to 0) only if you need to save a negligible amount of CPU cycles or if your specific hardware configuration does not support MPU-based stack monitoring. Warning: disabling this makes the system vulnerable to undetected stack overflows.

`PUT`

File: `os_config.h`

A macro function used to send formatted debug messages or system logs to the standard output (STDOUT) interface.

Valore: `ca_put(STDOUT,MSG)`

Note:
This function acts as a wrapper for the system's serial or logging peripheral. Enabling it allows real-time monitoring of task execution and error reporting. It depends on the hardware abstraction layer (HAL) being initialized for the UART or ITM port. Typical usage involves passing a string literal or a formatted buffer. It is non-blocking in some configurations but may introduce latency if the output buffer is full.

Uso:
Use this during development to trace task transitions, interrupt triggers, or variable states. Change this value when switching between different debug interfaces (e.g., UART to SWO). Warning: Excessive use of ca_put within high-frequency tasks or critical sections can significantly degrade system performance and timing determinism; always remove or wrap in debug-only macros before deploying to production.

`PUT_N`

File: `os_config.h`

A macro used to output a specific message string to a designated system output stream.

Valore: `ca_put_n(STDOUT,MSG)`

Note:
This constant defines the interface for system logging or console output. When enabled, it routes messages to the hardware peripheral defined by STDOUT (e.g., UART or ITM). It depends on the underlying hardware abstraction layer (HAL) being initialized. Typical values are the macro definition itself, which wraps the system's low-level character transmission function.

Uso:
Change this value when redirecting system logs to a different peripheral, such as switching from UART1 to UART2 for debugging. Use this to solve issues where system status messages are not appearing on your terminal. Warning: Ensure the baud rate and hardware pins for the new STDOUT are correctly configured, or the system may hang while waiting for the transmission buffer to clear.

`RO_MPU_REGION`

File: `hw/h5xx/h563/objmap.h`

Defines the total number of hardware Memory Protection Unit (MPU) regions available for the caOS kernel to manage.

Valori per MCU:

MCU	Valore
`H563`	`15U`

Note:
This constant directly maps to the physical MPU hardware capabilities of the STM32H563. Setting this value higher than the hardware limit (15) will cause memory access faults or kernel initialization errors. It is a static configuration that dictates how many memory segments (e.g., Flash, RAM, Peripheral space) the RTOS can protect simultaneously.

Uso:
You should only change this value if you are porting the RTOS to a different STM32 chip with a different MPU architecture. For the H563, it must remain at 15U. Do not modify this value to 'solve' memory access errors; if you encounter a MemManage fault, check your task stack sizes or memory region definitions instead.

`SHA256_BLOCK_SIZE`

File: `caos/crypto/inc/sha256.h`

Defines the fixed byte length of a single data block processed by the SHA-256 hashing algorithm.

Valore: `64`

Note:
This constant is set to 64 bytes (512 bits) as per the FIPS 180-4 standard. It is a hard-coded requirement for the internal buffer size of the SHA-256 state machine. Changing this value

will cause the cryptographic functions to produce incorrect hashes or trigger memory access violations.

Uso:
Do not modify this value under any circumstances. It is defined by the SHA-256 specification. If you are implementing a custom crypto driver or buffer management, use this constant instead of hard-coding the number 64 to ensure code maintainability and compliance with the algorithm's requirements.

`SHA256_DIGEST_SIZE`

File: `caos/crypto/inc/sha256.h`

Defines the fixed output length in bytes for the SHA-256 cryptographic hash algorithm.

Valore: `32`

Note:
This constant is set to 32 bytes (256 bits) as per the SHA-256 standard. Changing this value will break the cryptographic implementation and cause buffer overflows or incorrect hash calculations. It is a read-only constant used for memory allocation in hash context structures.

Uso:
Do not modify this value. It is used to allocate static buffers for storing digest results. If you are implementing a function that handles SHA-256 outputs, always use this constant instead of hardcoding the number 32 to ensure code maintainability and prevent errors if the algorithm is ever swapped.

`SHA512_BLOCK_SIZE`

File: `caos/crypto/inc/sha512.h`

Defines the fixed byte length of a single data block processed by the SHA-512 hashing algorithm.

Valore: `128`

Note:
This constant is set to 128 bytes (1024 bits) as per the FIPS 180-4 standard for SHA-512. It is a hard-coded architectural requirement for the internal buffer size of the hashing engine. Changing this value will cause compilation errors or runtime memory corruption in the crypto stack, as it is tightly coupled with the SHA-512 state machine and padding logic.

Uso:
Do not modify this value. It is defined by the cryptographic standard and is not a tunable parameter. If you are implementing a custom crypto driver or buffer management system, use this constant to ensure your buffers are correctly sized to hold exactly one SHA-512 block.

`SHA512_DIGEST_SIZE`

File: `caos/crypto/inc/sha512.h`

Defines the fixed output length of the SHA-512 hashing algorithm in bytes.

Valore: `64`

Note:
This constant is set to 64 bytes (512 bits). It is a hard-coded requirement of the SHA-512 standard. Modifying this value will cause cryptographic signature verification failures and buffer overflows in functions expecting a 64-byte digest array.

Uso:
Do not change this value. It is used to allocate buffers for hash results. If you are writing code that stores SHA-512 hashes, always use this constant to define your array size (e.g., uint8_t hash[SHA512_DIGEST_SIZE]) to ensure compatibility and prevent memory corruption.

`SPINLOCK_FREE`

File: `caos/inc/atomic.h`

A configuration flag that determines whether the kernel uses hardware-based spinlocks for atomic operations.

Valore: `0`

Note:
When set to 0, the system enforces the use of hardware spinlocks to ensure thread safety during atomic operations. If set to 1, spinlocks are disabled, which may lead to race conditions in multi-core or preemptive environments. This flag is typically dependent on the underlying architecture's support for atomic instructions (e.g., LDREX/STREX on ARM Cortex-M).

Uso:
Set this to 0 in multi-threaded or multi-core applications to prevent data corruption. Only set this to 1 if you are running on a single-core system with interrupts disabled or if you are implementing a custom, non-blocking synchronization mechanism, as disabling it without proper alternatives will cause system instability.

`SPINLOCK_TAKEN`

File: `caos/inc/atomic.h`

A unique magic number used to identify the state of a spinlock as currently held by a processor core.

Valore: `0x08091961`

Note:
This constant is used as a sentinel value in atomic memory operations to verify lock ownership. It must be a unique 32-bit pattern that does not conflict with valid memory addresses or other status flags. Changing this value requires updating all assembly-level atomic primitives and the spinlock acquisition logic across the entire kernel.

Uso:
This value should generally not be changed. It is used for debugging and memory integrity checks during spinlock operations. If you are porting the RTOS to a new architecture or suspect memory corruption in the spinlock state, ensure this constant matches the value expected by the hardware-specific atomic instructions.

SPI_DEF_PKT_SIZE

File: caos/drivers/inc/spi.h

Defines the default size in bytes for SPI data packets when no specific length is provided.

Valore: 128U

Note:
This constant sets the buffer allocation size for SPI transactions. Increasing this value consumes more RAM in the SPI driver's internal buffers. It must be large enough to accommodate the largest expected data frame in your application. Typical values range from 16 to 256 depending on available heap memory.

Uso:
Adjust this value if you encounter buffer overflow errors during SPI transfers or if your peripheral requires larger data chunks. Ensure that the value does not exceed the available RAM allocated for the SPI driver, otherwise, the system may experience memory corruption or hard faults.

SSH_LISTEN_PORT

File: os_config.h

Defines the TCP port number on which the caOS SSH server listens for incoming remote connections.

Valore: 22U

Note:
This constant sets the listening socket port for the SSH daemon. The default value is 22, which is the standard IANA port for SSH. Valid values range from 1 to 65535. Ensure the chosen port does not conflict with other network services running on the STM32, such as HTTP (80) or Telnet (23).

Uso:
Change this value if you need to run the SSH server on a non-standard port for security through obscurity or to avoid port collisions in a specific network environment. Warning: Changing this to a port below 1024 may require specific privileges in some environments, and remote clients must be updated to connect to the new port number.

STACK_CANARY_SIZE

File: caos/inc/mpu.h

Defines the size in bytes of the memory protection guard zone added to each task stack to detect overflows.

Valore: 32

Note:
When enabled, the RTOS places a specific pattern at the end of the stack. If the stack pointer exceeds this boundary, the MPU triggers a fault. Increasing this value provides a larger buffer but consumes more RAM per task. It is typically set to 32 bytes, but must be a multiple of the MPU region alignment requirements (usually 32 or 64 bytes depending on the STM32 architecture).

Uso:

Increase this value if you suspect stack corruption or intermittent hard faults during heavy task execution. Decrease it only if you are extremely constrained by RAM and have verified stack usage via static analysis or runtime monitoring. Setting this to 0 disables stack overflow protection, which is strongly discouraged for production firmware.

`STACK_CANARY_VALUE`

File: `caos/inc/mpu.h`

A unique 64-bit pattern placed at the end of each task stack to detect memory corruption.

Valore: `0xCAFEBABEDEADBEEFULL`

Note:
When enabled, the kernel writes this specific hex value to the stack boundary. If the value is overwritten, the system triggers a stack overflow exception. It requires the MPU (Memory Protection Unit) to be active. The value must be a 64-bit constant to ensure alignment and uniqueness.

Uso:
Change this value only if you suspect a collision with legitimate data patterns in your application memory. Use this to debug 'HardFault' errors caused by stack overflows. Do not disable this in production, as it is a critical safety mechanism for detecting buffer overruns.

`STACK_FILL_PATTERN`

File: `caos/inc/scheduler.h`

The hexadecimal byte pattern used to initialize task stack memory areas upon creation.

Valore: `0xDEADC0DE`

Note:
When a task is created, the RTOS fills its allocated stack space with this specific pattern. This allows the system to track stack usage by checking how much of this pattern has been overwritten by function calls and local variables. It has no functional impact on task execution but is critical for debugging stack overflows. The value must be a 32-bit word.

Uso:
Change this value if you need to distinguish stack memory from other initialized data in your debugger's memory view. It is primarily used during development to identify stack depth; if you see the pattern at the very bottom of the stack, you are at risk of a stack overflow. Do not change this value unless you have a specific reason to track memory patterns differently in your IDE.

`STACK_MIN`

File: `hw/f1xx/f103/objmap.h`
File: `hw/f4xx/f407/objmap.h`
File: `hw/f4xx/f429/objmap.h`
File: `hw/f4xx/f446/objmap.h`
File: `hw/f7xx/f746/objmap.h`
File: `hw/h5xx/h563/objmap.h`
File: `hw/h7xx/h747/objmap.h`

File: `hw/h7xx/h753/objmap.h`

Defines the absolute minimum stack size in bytes allowed for any task created within the caOS RTOS.

Valori per MCU:

MCU	Valore
F103	256
F407	512
F429	512
F446	512
F746	512
H563	2048
H747	2048
H753	2048

Note:
This constant acts as a safety floor for task memory allocation. If a task is created with a stack size smaller than STACK_MIN, the kernel will either reject the task creation or force the size to this value. Higher values are required for architectures with larger register sets or deeper interrupt nesting (e.g., H7 series). It is strictly dependent on the specific MCU architecture's memory footprint and stack frame requirements.

Uso:
Increase this value if you encounter stack overflow errors or hard faults during task initialization. Do not decrease this value below the recommended architecture default, as it will lead to unpredictable system crashes and memory corruption. Ensure that your task stack sizes are always a multiple of 8 bytes to maintain proper alignment on ARM Cortex-M processors.

`STDOUT_ON_ITM`

File: `os_config.h`

Enables or disables the redirection of standard output (printf) to the ARM Cortex-M ITM (Instrumentation Trace Macrocell) peripheral.

Valore: `0`

Note:
When set to 1, the system routes stdout data through the ITM stimulus port 0, allowing real-time logging via an external debugger like ST-LINK. When set to 0, this feature is disabled, and stdout will not produce output via ITM. This requires the debugger's SWO pin to be connected and configured in your IDE's debug settings.

Uso:
Enable this (set to 1) during development when you need to monitor system logs or debug messages without using a physical UART peripheral. Disable it (set to 0) for production builds to save CPU cycles and avoid potential blocking behavior if the debugger is not connected.

`STDOUT_ON_LPUART`

File: `os_config.h`

Enables or disables the redirection of standard output (printf) to the LPUART peripheral.

Valore: `0`

Note:
When set to 1, the system initializes the LPUART hardware and routes all standard output streams to it. When set to 0, standard output is disabled or redirected to a null device. This flag depends on the LPUART peripheral being correctly configured in the hardware abstraction layer. Typical values are 0 (disabled) or 1 (enabled).

Uso:
Set this to 1 during development to enable serial debugging and log messages via the LPUART interface. Set to 0 in production builds to save power and reduce CPU overhead caused by string formatting and UART transmission. Ensure the baud rate and GPIO pins are correctly initialized before enabling this flag to avoid system hangs during initialization.

`STDOUT_ON_UART`

File: `os_config.h`

Enables or disables the redirection of standard output (printf) to the hardware UART peripheral.

Valore: `1`

Note:
When set to 1, the system routes stdout to the configured UART port; when 0, stdout is disabled or redirected to a null device. This flag depends on the UART driver being initialized in the board support package. Typical values are 0 (disabled) or 1 (enabled).

Uso:
Enable this during development to view debug logs via a serial terminal. Disable it in production builds to save CPU cycles and reduce power consumption, or if the UART pins are required for other communication protocols.

`STDOUT_ON_USB_CDC`

File: `os_config.h`

Enables or disables the redirection of standard output (printf) to the USB CDC virtual COM port.

Valore: `0`

Note:
When set to 1, the system initializes the USB peripheral and routes stdout to the host PC via USB. When set to 0, stdout is disabled or routed to the default UART peripheral. This flag requires the USB_DEVICE_ENABLED flag to be set to 1. Valid values are 0 (disabled) or 1 (enabled).

Uso:
Change this to 1 when you need to debug your application using a terminal emulator on your PC via USB. Set it to 0 if you are using UART for logging or if you need to save power by disabling the USB stack. Ensure your USB hardware is connected and the driver is installed on the host PC before enabling.

`STDOUT_QUEUE_DEPTH`

File: `caos/inc/stdout.h`

Defines the maximum number of characters that can be buffered in the STDOUT output queue before blocking or dropping data.

Valore: `64`

Note:
This constant sets the size of the circular buffer used by the system's standard output stream. If the buffer fills up faster than the hardware peripheral (e.g., UART) can transmit, subsequent print calls may block or lose data depending on the driver implementation. It must be a positive integer, typically a power of two for optimal performance. Increasing this value consumes more RAM but reduces the likelihood of output stalls during high-frequency logging.

Uso:
Increase this value if you notice truncated log messages or if your application performs heavy logging that causes the system to hang. Decrease this value if your device is running low on SRAM. Ensure that the value is large enough to hold the longest single log message you intend to print to avoid partial output.

`SVC_OS_END`

File: `caos/inc/safeguard.h`

A magic constant used by the caOS kernel to identify the end of a Supervisor Call (SVC) sequence.

Valore: `0xFD`

Note:
This value is typically used in the context of system call handling or kernel-level state transitions. It acts as a sentinel or marker in the stack frame or register set during an SVC exception. It is a fixed internal constant (0xFD) and should not be modified by application code. It is tightly coupled with the low-level assembly implementation of the caOS context switcher.

Uso:
This constant is for internal kernel use only. Do not change this value. Modifying it will break the kernel's ability to correctly identify and return from system calls, likely resulting in a hard fault or system hang. If you are debugging and see this value in a register or stack dump, it indicates the processor is currently processing an OS-level service request.

`SVC_SAFEGUARD`

File: `caos/inc/safeguard.h`

A magic constant used to verify the integrity of the Supervisor Call (SVC) stack frame during context switches.

Valore: `0xFE`

Note:
When enabled, the kernel checks this value at the base of the stack to detect stack overflows or memory corruption. If the value does not match 0xFE, the system triggers a kernel panic. This

flag must be consistent across all task stack initializations and is dependent on the architecture-specific SVC handler implementation.

Uso:
Change this value only if you are porting caOS to a new architecture or if you need to avoid a collision with specific hardware-reserved memory patterns. Do not modify this value during runtime; incorrect values will cause the system to enter a hard fault or kernel panic immediately upon the first task switch.

`SYSCLK_FREQ_HZ`

File: `hw/f1xx/f103/objmap.h`
File: `hw/f4xx/f407/objmap.h`
File: `hw/f4xx/f429/objmap.h`
File: `hw/f4xx/f446/objmap.h`
File: `hw/f7xx/f746/objmap.h`
File: `hw/h5xx/h563/objmap.h`
File: `hw/h7xx/h747/objmap.h`
File: `hw/h7xx/h753/objmap.h`

Defines the target system clock frequency in Hertz for the MCU core.

Valori per MCU:

MCU	Valore
F103	72000000UL
F407	168000000UL
F429	168000000UL
F446	168000000UL
F746	216000000UL
H563	250000000UL
H747	400000000UL
H753	480000000UL

Note:
This constant is used by the RTOS kernel to calculate timing intervals, tick rates, and peripheral baud rates. It must match the actual hardware clock configuration set in the RCC registers. Setting this value incorrectly will cause all time-dependent functions (like delays, timeouts, and UART communication) to run at the wrong speed. Valid values are strictly determined by the specific STM32 model's maximum operating frequency.

Uso:
Change this value only when modifying the system clock tree (e.g., changing the PLL settings or external crystal frequency). Always ensure this value matches the frequency calculated in your clock configuration code. If your system timing seems 'slow' or 'fast' compared to real-time, verify this constant first.

`SYSLOG_FAC_KERN`

File: `caos/net/inc/net_proto.h`

Defines the facility code for kernel-level messages in the system logging protocol.

Valore: `0U`

Note:

This constant sets the 'facility' field in syslog packets to 0, identifying the source as the kernel. It is used by the network logging stack to categorize messages. Changing this value will cause log aggregators to misclassify kernel events as coming from other system components (e.g., mail or user processes). It must remain 0 to comply with standard RFC 5424 logging conventions for kernel logs.

Uso:
Do not modify this value unless you are implementing a custom logging protocol that requires non-standard facility mapping. Changing this will break log filtering and severity analysis on remote syslog servers.

`SYSLOG_FAC_LOCAL0`

File: `caos/net/inc/net_proto.h`

Defines the syslog facility identifier for local use 0, used to categorize network log messages.

Valore: `16U`

Note:
This constant sets the facility field in the syslog header to 16. It is used by the network stack to tag outgoing log packets. It has no direct impact on system performance but requires the remote syslog server to be configured to accept messages from the 'local0' facility.

Uso:
Change this value only if your network infrastructure requires a specific syslog facility code (e.g., local1 through local7) to route logs to different files or servers. Ensure the value matches the standard syslog facility definitions (0-23) to avoid malformed log headers.

`SYSLOG_FAC_LOCAL7`

File: `caos/net/inc/net_proto.h`

Defines the syslog facility code for local use 7 (value 23) used in network logging protocols.

Valore: `23U`

Note:
This constant maps to the standard Syslog facility 'local7' as defined in RFC 5424. It is used to categorize log messages sent over the network. Changing this value will cause log collectors (like rsyslog or syslog-ng) to misclassify or ignore incoming logs from the device. It has no direct impact on RTOS scheduling or memory, but it must match the configuration of your remote logging server.

Uso:
Change this value only if your network infrastructure requires a specific facility code for log filtering or routing. Use this when you need to separate device logs from other system logs on a centralized server. Ensure the value remains within the valid syslog facility range (0-23) to avoid protocol violations.

`SYSLOG_FAC_USER`

File: `caos/net/inc/net_proto.h`

Defines the facility code for user-level messages in the system logging protocol.

Valore: `1U`

Note:
This constant sets the 'facility' field in a syslog packet to the 'USER' category (value 1). It is used by the network stack to categorize logs generated by application-level tasks. It must be a valid integer defined by the RFC 5424 standard; changing this value may cause log servers to misclassify or ignore incoming messages.

Uso:
Change this value only if your network logging infrastructure requires a specific facility code other than the standard 'USER' category. Ensure that your remote syslog server is configured to accept and process the facility code you assign here, otherwise, your application logs may be filtered out or lost.

`SYSLOG_MSG_MAX`

File: `os_config.h`

Defines the maximum character length allowed for a single system log message.

Valore: `128U`

Note:
This constant sets the buffer size for the internal logging queue. If a message exceeds 128 bytes, it will be truncated. Increasing this value consumes more RAM from the system heap, while decreasing it saves memory but risks losing data in long diagnostic strings.

Uso:
Increase this value if you notice your debug logs are being cut off mid-sentence. Decrease it only if your STM32 device is running extremely low on RAM. Ensure that the buffer size is large enough to accommodate your longest 'printf' style debug statements.

`SYSLOG_PORT`

File: `caos/net/inc/net_proto.h`

Defines the UDP port number used for transmitting system log messages over the network.

Valore: `514U`

Note:
This constant sets the destination port for the caOS logging subsystem. It typically defaults to 514, which is the standard port for the Syslog protocol (RFC 5424). Changing this value requires the receiving log server to be configured to listen on the new port. It is a dependency for the network logging module; if the network stack is disabled, this value is ignored.

Uso:
Change this value only if your network infrastructure requires logs to be sent to a non-standard port or if there is a port conflict with other network services. Ensure the value is a valid 16-bit unsigned integer (1-65535). Setting this to an invalid or occupied port will result in log packets being dropped or rejected by the network stack.

`SYSLOG_QUEUE_SIZE`

File: `os_config.h`

Defines the maximum number of log messages that can be stored in the system log queue before new messages are dropped.

Valore: `8U`

Note:
This value determines the depth of the internal message buffer for the system logger. Increasing this value consumes more RAM but prevents log loss during high-frequency bursts. It must be a positive integer, typically a power of two for optimal performance. If set too low, the system may experience 'log overflow' errors when multiple tasks attempt to write logs simultaneously.

Uso:
Increase this value if you notice missing log entries during high-load scenarios or complex interrupt sequences. Decrease it if your application is running low on heap memory. Ensure that the task processing the log queue is fast enough to keep up with the rate of incoming messages to avoid filling the buffer.

`SYSLOG_SEV_ALERT`

File: `caos/net/inc/net_proto.h`

Defines the severity level for system alert messages within the caOS network logging subsystem.

Valore: `1U`

Note:
This constant represents the numerical priority level for 'Alert' events. It is used by the logging filter to determine if a message should be recorded or transmitted. It typically ranges from 0 to 7, where lower values indicate higher urgency. It depends on the global SYSLOG_LEVEL_THRESHOLD setting; if the system threshold is set higher than 1, these alerts may be suppressed.

Uso:
Change this value only if you are modifying the core network protocol logging architecture. It is used to categorize critical network failures that require immediate attention. Do not change this value to filter logs; instead, adjust the system-wide logging threshold to avoid missing critical system alerts.

`SYSLOG_SEV_CRIT`

File: `caos/net/inc/net_proto.h`

Defines the numerical severity level for critical system log messages within the caOS networking stack.

Valore: `2U`

Note:
This constant sets the threshold for logging; when enabled, only messages with a severity level equal to or greater than 2 are recorded. It is typically used in conjunction with the global logging verbosity filter. Valid values are integers, where lower numbers represent higher urgency.

Uso:
Change this value if you need to filter out non-critical network logs to save memory or reduce UART/buffer traffic. Do not set this value too high during debugging, as you may lose visibility into critical system failures or stack overflows occurring within the network protocol layer.

SYSLOG_SEV_DEBUG

File: `caos/net/inc/net_proto.h`

Sets the minimum severity level for system log messages to include debug-level information.

Valore: `7U`

Note:
When enabled, the system logs all messages with a severity level of 7 or lower. Disabling or increasing this value reduces memory usage and CPU overhead by filtering out verbose debug logs. It depends on the global logging subsystem being initialized and requires sufficient buffer space in the log queue.

Uso:
Change this value during development to troubleshoot network protocol issues by enabling detailed trace logs. Set it to a lower value (e.g., 3 for errors only) in production environments to prevent log buffer overflows and minimize performance impact.

SYSLOG_SEV_EMERG

File: `caos/net/inc/net_proto.h`

Defines the numerical severity level for emergency system logs, representing the highest priority error state.

Valore: `0U`

Note:
This constant is used as a filter or identifier in the caOS logging subsystem. Setting this to 0U indicates that the message is critical and the system is likely unusable. It is typically used in conjunction with a bitmask or threshold check; if the system log level is set to 0, only emergency messages will be recorded.

Uso:
Do not change this value as it is part of the standard syslog protocol definition (RFC 5424). Use this constant when calling the logging API to report catastrophic hardware failures or kernel panics that require immediate attention.

SYSLOG_SEV_ERR

File: `caos/net/inc/net_proto.h`

Defines the severity level threshold for system error logging in the network stack.

Valore: `3U`

Note:
This constant sets the minimum severity level required for a network event to be recorded in the system log. A value of 3U typically corresponds to 'Error' level messages. Decreasing this

value increases log verbosity, while increasing it filters out less critical warnings. It depends on the global logging subsystem being initialized.

Uso:
Change this value during debugging to capture more granular network diagnostic information. Set it to a higher value in production environments to reduce memory usage and CPU overhead caused by excessive logging. Ensure the value remains within the range defined by the system's logging enumeration to avoid undefined behavior.

`SYSLOG_SEV_INFO`

File: `caos/net/inc/net_proto.h`

Defines the severity level threshold for system logging, specifically set to informational messages.

Valore: `6U`

Note:
This constant sets the logging filter level to 6. When enabled, the system will record all messages with a severity level of 6 (INFO) or higher (more critical). Decreasing this value increases the volume of logs, while increasing it filters out less critical information. It depends on the global logging subsystem being initialized.

Uso:
Change this value when you need to debug network protocol behavior without flooding the log buffer with low-level debug data. Use it during integration testing to monitor standard operational events. Warning: setting this value too low in a production environment may cause performance degradation due to excessive UART or flash memory write operations.

`SYSLOG_SEV_NOTICE`

File: `caos/net/inc/net_proto.h`

Defines the severity level for 'Notice' messages in the caOS system logging framework.

Valore: `5U`

Note:
This constant represents a numerical severity level of 5. It is used by the logging subsystem to filter messages; only logs with a severity equal to or higher than the configured system threshold will be processed or transmitted. It is typically used in conjunction with other severity constants like SYSLOG_SEV_ERROR or SYSLOG_SEV_DEBUG.

Uso:
Change this value only if you are modifying the core logging protocol definitions. It is used as a reference point for log filtering logic. Do not change this value if you are simply trying to adjust the verbosity of your application logs; instead, adjust the global system log level threshold in your configuration header.

`SYSLOG_SEV_WARN`

File: `caos/net/inc/net_proto.h`

Defines the severity level threshold for system logging, specifically targeting warning-level events.

Valore: `4U`

Note:
This constant sets the filter level for the caOS logging subsystem. When enabled, the system will record events with a severity of 4 or higher. Lowering this value increases log verbosity, while raising it filters out less critical messages. It depends on the global SYSLOG_LEVEL configuration and typically ranges from 0 (Emergency) to 7 (Debug).

Uso:
Change this value when you need to debug network protocol issues without flooding the log buffer with informational messages. Increase the threshold (e.g., to 5 or 6) in production environments to reduce memory usage and CPU overhead caused by excessive logging, or decrease it during development to capture potential warning signs before they escalate into system errors.

`SYSTICK_ERROR`

File: `caos/inc/systick.h`

A sentinel value representing a failure in the system timer initialization or operation.

Valore: `-1`

Note:
This constant is typically returned by the caOS kernel initialization functions when the hardware timer fails to start. It is not a configurable setting but a status code. It depends on the HAL_SYSTICK_Config function success. The value -1 is used to indicate an error state in signed integer return types.

Uso:
Use this constant to check the return value of system initialization functions. If a function returns SYSTICK_ERROR, the RTOS kernel cannot guarantee timing accuracy, and the system should enter a safe state or trigger a hardware reset. Do not modify this value as it is a reserved error code.

`SYSTICK_OK`

File: `caos/inc/systick.h`

A configuration flag that enables or disables the internal caOS System Tick timer driver.

Valore: `0`

Note:
When set to 1, the RTOS initializes the hardware timer to generate periodic interrupts for task scheduling. When set to 0, the Systick driver is disabled, meaning the kernel will not have a time base for preemptive multitasking or delay functions. This flag must be enabled for any RTOS features requiring time-slicing or blocking delays.

Uso:
Set this to 1 for standard RTOS operation. Set to 0 only if you are implementing a custom time-base driver or if you are running the system in a bare-metal mode where the RTOS scheduler is not required. Disabling this while using RTOS primitives like vTaskDelay will cause the system to hang or behave unpredictably.

`TASK_CANARY_VALUE_SAFE`

File: `caos/inc/scheduler.h`

A unique 32-bit pattern used to detect stack overflows by filling unused stack memory.

Valore: `0xCACAFEFC`

Note:
When enabled, the kernel writes this value to the entire stack area during task creation. The system periodically checks if this value has been overwritten, indicating a stack overflow. It requires sufficient RAM to store the canary pattern and depends on the stack monitoring feature being active in the kernel configuration.

Uso:
Change this value only if it conflicts with specific memory patterns used by your application code. Use this to debug hard faults caused by stack corruption. Warning: Do not set this to 0x00000000 or 0xFFFFFFFF, as these are common values in uninitialized memory and will lead to false negatives in overflow detection.

`TASK_CANARY_VALUE_SAFE_FP`

File: `caos/inc/scheduler.h`

The specific 32-bit pattern used to detect stack overflows in task memory regions.

Valore: `0xCACAFEFD`

Note:
When enabled, the kernel writes this value to the entire stack area of a task upon creation. During context switches, the scheduler checks if the top of the stack still contains this value. If the value is corrupted, the system triggers a stack overflow fault. This constant must be a 32-bit word aligned with the architecture's word size. It has no direct dependencies on other flags but requires sufficient stack space to be allocated to avoid false positives.

Uso:
Change this value only if you suspect memory corruption patterns are conflicting with the default constant. It is primarily used for debugging stack usage; if your system crashes with a stack overflow error, verify that your task stack sizes are large enough. Do not set this to a value that is likely to appear naturally in your application's data, as it will mask potential stack overflows.

`TASK_CANARY_VALUE_STD`

File: `caos/inc/scheduler.h`

The default 32-bit pattern used to fill task stacks to detect stack overflows.

Valore: `0xCACAFEF8`

Note:
When enabled, the kernel writes this specific hex value across the entire stack memory area of a task during creation. If the stack pointer exceeds the allocated bounds, the kernel checks if this pattern has been overwritten. It requires the stack monitoring feature to be enabled in the kernel configuration. The value 0xCACAFEF8 is chosen because it is unlikely to occur naturally in program data.

Uso:
Change this value only if you need to avoid specific memory patterns that might conflict with your application's data or if you are performing advanced debugging. Ensure the value is a 32-bit word aligned with the architecture. If you change this, you must also update the corresponding check logic in the kernel's stack overflow handler.

`TASK_CANARY_VALUE_STD_FP`

File: `caos/inc/scheduler.h`

The default 32-bit pattern used to fill task stacks to detect stack overflows.

Valore: `0xCACAFEF9`

Note:
When enabled, the kernel writes this specific hex value across the entire stack memory area of a task during creation. If the stack pointer exceeds the allocated bounds, the kernel checks if this pattern has been overwritten. This constant must be a 32-bit word aligned value. Changing this value requires a full project rebuild to ensure all stack initialization routines use the new pattern.

Uso:
Change this value if you suspect the default pattern (0xCACAFEF9) is accidentally appearing in your application data, causing false-positive stack overflow detections. Use a unique, non-zero value that is unlikely to occur naturally in your program's variables. Ensure the value is defined as a 32-bit unsigned integer to maintain memory alignment and prevent bus faults during stack initialization.

`TASK_NAME_LEN`

File: `caos/inc/scheduler.h`

Defines the maximum number of characters allowed for a task name string.

Valore: `12`

Note:
This constant sets the fixed size of the character array used to store task identifiers in the Task Control Block (TCB). If set to 12, names longer than 11 characters (plus the null terminator) will be truncated. Increasing this value increases the memory footprint of every task created in the system.

Uso:
Adjust this value if your project requires more descriptive task names for debugging or logging purposes. Ensure that the value is large enough to accommodate your naming convention, but keep it as small as possible to conserve RAM, especially in systems with a high number of concurrent tasks.

`TCP_ARP_TIMEOUT_MS`

File: `os_config.h`

Defines the duration in milliseconds that the system waits for an ARP response before timing out.

Valore: `2000U`

Note:
This constant determines the lifespan of an ARP request packet. If set too low, the system may fail to resolve IP addresses to MAC addresses on busy networks. If set too high, it increases latency when communicating with new devices. The value is typically set between 1000 and 5000 milliseconds depending on network stability.

Uso:
Increase this value if you experience frequent 'Host Unreachable' errors or connection timeouts on high-latency or congested networks. Decrease it only if your application requires rapid failure detection for non-responsive network nodes. Ensure this value is consistent with the overall network stack timeout settings to avoid race conditions.

`TCP_CLOSE_WAIT_MS`

File: `os_config.h`

Defines the timeout duration in milliseconds for a TCP connection to remain in the CLOSE_WAIT state.

Valore: `10000U`

Note:
This constant determines how long the stack waits for the application to acknowledge a connection closure before forcing a reset. It is dependent on the system tick frequency. Typical values range from 5000U to 30000U depending on network latency requirements.

Uso:
Increase this value if you experience premature connection resets during slow application processing. Decrease it if you need to free up socket resources faster in high-traffic environments. Setting this too low may cause active connections to drop unexpectedly.

`TCP_CONNECT_TIMEOUT_MS`

File: `os_config.h`

Defines the maximum time in milliseconds the system waits for a TCP connection to be established.

Valore: `5000U`

Note:
This constant sets the timeout duration for the socket connect operation. If the connection is not completed within this period, the function returns a timeout error. It depends on the underlying network stack's tick rate. Typical values range from 1000U to 10000U depending on network latency requirements.

Uso:
Increase this value if you are connecting to remote servers over high-latency networks or unstable connections to prevent premature timeouts. Decrease it if you need the application to fail fast when a server is unreachable to maintain system responsiveness. Setting this too low may cause connection failures on busy networks.

`TCP_FLAG_ACK`

File: `caos/net/inc/net_proto.h`

A bitmask flag representing the Acknowledgment (ACK) field in a TCP packet header.

Valore: `0x10U`

Note:
When set, this flag indicates that the Acknowledgment Number field in the TCP header is valid. It is a fundamental component of the TCP state machine, used to confirm receipt of data segments. This constant is typically used in conjunction with bitwise OR operations to construct or parse TCP headers. It has no direct dependencies on other flags but must be used in accordance with the TCP protocol specification (RFC 793).

Uso:
Use this flag when manually constructing TCP packets or when inspecting incoming packet headers to determine if the segment is an acknowledgment. Do not modify the value 0x10U, as it is defined by the TCP protocol standard. Incorrect usage or failure to set this flag when required will result in dropped connections or retransmission loops.

`TCP_FLAG_FIN`

File: `caos/net/inc/net_proto.h`

A bitmask flag representing the TCP FIN (Finish) control bit used to signal the graceful termination of a connection.

Valore: `0x01U`

Note:
This flag is set in the TCP header control field. When enabled, it indicates that the sender has no more data to transmit. It is typically used in conjunction with the TCP state machine to transition from ESTABLISHED to FIN_WAIT_1 or CLOSE_WAIT states. It has no dependencies on other flags but must be handled by the TCP stack's state transition logic.

Uso:
Change this value only if you are modifying the low-level TCP/IP stack implementation or debugging packet headers. Do not modify this constant for application-level code. Using an incorrect value will cause the TCP stack to misinterpret connection termination requests, leading to hung sockets or broken network communication.

`TCP_FLAG_PSH`

File: `caos/net/inc/net_proto.h`

Defines the PUSH flag in the TCP header to force immediate data delivery to the application layer.

Valore: `0x08U`

Note:
When set, this flag instructs the receiving TCP stack to push buffered data to the application immediately rather than waiting for the buffer to fill. It is a single-bit flag within the TCP control byte. It has no direct dependencies on other flags but must be used in conjunction with valid sequence numbers.

Uso:
Enable this flag when sending time-sensitive control packets or the final segment of a data stream to reduce latency. Do not set this flag on every single packet in a high-throughput stream, as it can cause excessive interrupt overhead and degrade network performance by preventing efficient packet coalescing.

`TCP_FLAG_RST`

File: `caos/net/inc/net_proto.h`

A control flag used in the TCP header to indicate an immediate connection reset.

Valore: `0x04U`

Note:
When this flag is set in a TCP packet, the receiving stack must immediately abort the connection and discard all associated buffers. It is a bitmask value (0x04U) typically combined with other TCP flags. It has no dependencies on other RTOS flags but requires the network stack to be in a valid state to process the reset signal.

Uso:
Use this flag when you need to force-close a connection due to an error, a timeout, or an invalid packet sequence. Do not set this flag manually unless you are implementing custom low-level socket handling, as incorrect usage will cause unexpected connection drops and potential data loss for the peer.

`TCP_FLAG_SYN`

File: `caos/net/inc/net_proto.h`

A bitmask flag representing the Synchronize sequence number control bit in a TCP packet header.

Valore: `0x02U`

Note:
This flag is used during the TCP three-way handshake to initiate a connection. When set to 0x02, the TCP stack interprets the segment as a connection request. It is typically used in conjunction with other flags like ACK or FIN. It must be defined as 0x02 to comply with RFC 793 standards for TCP header formatting.

Uso:
Change this value only if you are implementing a custom or non-standard TCP stack variant. For standard network communication, this value should remain unchanged. Incorrect values will cause the remote host to reject connection attempts or fail to recognize the packet type, leading to silent connection timeouts.

`TCP_FLAG_URG`

File: `caos/net/inc/net_proto.h`

A bitmask flag representing the Urgent pointer field in a TCP packet header.

Valore: `0x20U`

Note:

When set in the TCP control bits, this flag indicates that the Urgent Pointer field in the TCP header is valid and contains data that should be processed with higher priority. It is typically used in conjunction with the TCP_URG_PTR field. This flag is a standard part of the TCP protocol stack and should not be modified unless implementing custom protocol extensions.

Uso:
Use this flag when inspecting or constructing TCP segments that require out-of-band data signaling. It is primarily used for urgent data transmission, such as interrupting a remote process. Do not manually set this flag unless you are handling specific legacy protocols or specialized network traffic, as incorrect usage can lead to unexpected behavior in standard TCP/IP stacks.

`TCP_MAX_CONN`

File: `os_config.h`

Defines the maximum number of concurrent TCP connections allowed by the caOS network stack.

Valore: `8U`

Note:
This constant determines the size of the internal connection control block (CCB) table. Increasing this value consumes more RAM (each connection requires a fixed memory buffer for state tracking). It must be greater than 0 and is typically limited by the available heap memory. If set too low, new connection requests will be rejected by the stack.

Uso:
Adjust this value based on your application's memory constraints and the expected number of simultaneous clients. Increase it if your device acts as a server handling multiple concurrent requests; decrease it to save RAM if the device only communicates with a single master node.

`TCP_MAX_INFLIGHT`

File: `os_config.h`

Defines the maximum number of unacknowledged TCP segments allowed in the network buffer.

Valore: `16U`

Note:
This constant controls the TCP sliding window size at the transport layer. Increasing this value allows higher throughput on high-latency links but consumes more RAM per socket. It must be balanced against the total available heap memory and the number of concurrent active connections. Typical values range from 4 to 32.

Uso:
Increase this value if you observe slow data transfer speeds or high packet loss on high-bandwidth networks. Decrease this value if the system experiences memory allocation failures (heap exhaustion) during heavy network traffic. Ensure that the total memory allocated for all sockets does not exceed the system's available RAM.

`TCP_MAX_LISTEN`

File: `os_config.h`

Defines the maximum number of pending TCP connection requests that can be queued for a listening socket.

Valore: `2U`

Note:
This constant determines the size of the backlog queue for the TCP stack. Increasing this value allows the system to handle more simultaneous incoming connection attempts before rejecting new ones. It consumes additional RAM for each slot in the queue. Valid range is typically 1 to 16, depending on available heap memory.

Uso:
Increase this value if your application experiences 'Connection Refused' errors during high-traffic periods or when multiple clients attempt to connect simultaneously. Decrease it if the system is running low on memory. Setting this too high may lead to memory exhaustion if many clients attempt to connect at once.

`TCP_MAX_RETRIES`

File: `os_config.h`

Defines the maximum number of times the TCP stack will attempt to retransmit a packet before declaring a connection failure.

Valore: `5U`

Note:
When enabled, the stack will retry sending unacknowledged segments up to this count. Increasing this value improves reliability on lossy networks but increases the time taken to detect a broken connection. It depends on the TCP_RETRANSMIT_TIMEOUT setting. Typical values range from 3 to 10; setting this too high may cause the application to hang while waiting for a timeout.

Uso:
Increase this value if your device experiences frequent packet loss or intermittent connectivity issues that cause premature socket closures. Decrease it if you need the system to detect and recover from network failures faster. Ensure this value is balanced with your application's latency requirements.

`TCP_MSS_DEFAULT`

File: `caos/net/inc/net_proto.h`

Defines the Maximum Segment Size (MSS) in bytes for TCP connections, representing the largest amount of data that can be received in a single TCP segment.

Valore: `1460U`

Note:
This value directly impacts the TCP payload size. Enabling a larger value increases throughput by reducing header overhead, while a smaller value may be required for networks with specific MTU constraints. It is typically set to 1460 to fit within a standard 1500-byte Ethernet MTU

after accounting for IP and TCP headers. It must be less than or equal to the network interface MTU minus 40 bytes.

Uso:
Adjust this value if your device communicates over networks with restricted MTU sizes (e.g., VPNs, tunnels, or specific cellular links) to prevent packet fragmentation. Setting this value too high relative to the network path MTU will cause packet loss or performance degradation due to fragmentation; setting it too low will unnecessarily increase protocol overhead.

`TCP_RTO_INITIAL_MS`

File: `os_config.h`

Defines the initial Retransmission Timeout (RTO) value in milliseconds for new TCP connections.

Valore: `1000U`

Note:
This value sets the duration the stack waits for an acknowledgment before retransmitting the first SYN packet. Increasing this value helps in high-latency networks, while decreasing it speeds up connection failure detection. It must be a positive integer, typically ranging from 200 to 3000 depending on the expected network stability.

Uso:
Adjust this value if you experience frequent connection timeouts on slow or congested networks. If set too low, the system may trigger unnecessary retransmissions, wasting bandwidth; if set too high, the system will be slow to recover from dropped packets during the initial handshake.

`TCP_RTO_MAX_MS`

File: `os_config.h`

Defines the maximum timeout duration in milliseconds for a TCP retransmission attempt.

Valore: `16000U`

Note:
This constant sets the upper bound for the exponential backoff algorithm used during packet loss. If the system fails to receive an ACK within this period, the connection is typically dropped. It depends on the TCP_MAX_RETRIES setting; increasing this value allows for more robust connections on high-latency or unstable networks but increases the time taken to detect a dead link.

Uso:
Increase this value if your device operates on high-latency networks (e.g., satellite or congested cellular links) to prevent premature connection termination. Decrease it only if your application requires rapid detection of broken connections. Setting this value too low may cause frequent, unnecessary connection drops on busy networks.

`TCP_RX_RING_DEPTH`

File: `os_config.h`

Defines the number of packet slots available in the TCP receive buffer ring.

Valore: `4U`

Note:
This value determines how many incoming TCP segments can be queued before the stack begins dropping packets or applying backpressure. Increasing this value consumes more RAM (each slot is fixed-size based on MTU). It must be a power of two for optimal performance in the circular buffer implementation. Typical range is 2 to 16 depending on available heap memory.

Uso:
Increase this value if you observe packet loss or 'TCP receive buffer overflow' errors during high-throughput network bursts. Decrease this value if the system is running low on RAM and the network traffic is low-frequency or strictly request-response based.

`TCP_TCP_ACCEPT_TIMEOUT_MS`

File: `os_config.h`

Defines the maximum time in milliseconds the system waits for an incoming TCP connection request before aborting.

Valore: `10000U`

Note:
This constant controls the blocking duration of the accept() function. If set to 0, the system waits indefinitely. If the value is too low, legitimate clients may fail to connect during network congestion. It depends on the underlying TCP/IP stack's tick rate and memory availability for socket buffers.

Uso:
Increase this value if your application experiences frequent 'connection timeout' errors in high-latency network environments. Decrease it if you need the system to remain responsive and quickly free up resources when a client fails to initiate a handshake. Ensure the value is sufficient to cover the round-trip time of your network infrastructure.

`TCP_TIMER_TICK_MS`

File: `os_config.h`

Defines the time interval in milliseconds for the TCP/IP stack's internal timer heartbeat.

Valore: `100U`

Note:
This constant determines the resolution of TCP retransmissions, keep-alive probes, and connection timeouts. Decreasing this value increases the frequency of stack processing, which improves responsiveness but consumes more CPU cycles. It must be a multiple of the system tick period. Typical values range from 50ms to 500ms.

Uso:
Adjust this value if you experience slow connection recovery or delayed timeouts. Use a smaller value for high-speed networks requiring fast retransmission, or a larger value to reduce CPU

overhead in low-power applications. Setting this too low may cause the stack to starve other tasks of processing time.

`TCP_TIME_WAIT_MS`

File: `os_config.h`

Defines the duration in milliseconds that a TCP connection remains in the TIME_WAIT state after closure.

Valore: `4000U`

Note:
This constant controls how long the socket remains reserved after a connection is closed to ensure delayed packets are handled. Increasing this value consumes more memory resources as the socket control block cannot be reused until the timer expires. The default value is 4000ms (4 seconds).

Uso:
Increase this value if you experience data corruption or connection errors due to late-arriving packets from a previous session. Decrease it only if your system is running out of available sockets and you are certain that network latency is low enough to avoid packet collisions.

`TFTP_BLOCK_SIZE`

File: `caos/net/inc/tftp_service.h`

Defines the size in bytes of a single data packet transferred during a TFTP operation.

Valore: `512U`

Note:
This value directly impacts the payload size of UDP packets sent over the network. While 512 bytes is the standard defined by RFC 1350, increasing this value can improve throughput on high-speed networks, provided the underlying network stack and hardware buffers support larger MTUs. Decreasing it may be necessary for memory-constrained systems or unstable links, but values larger than 512 may cause compatibility issues with standard TFTP clients.

Uso:
Adjust this value if you encounter packet fragmentation issues or if you need to optimize file transfer speeds. Ensure that your network buffer size (defined elsewhere in the stack) is large enough to accommodate this value plus the UDP/IP/Ethernet header overhead. Do not change this unless you have verified that both the client and server support the chosen block size.

`TFTP_BLOCK_TIMEOUT_MS`

File: `os_config.h`

Defines the maximum time in milliseconds the system waits for a response during a TFTP data block transfer.

Valore: `5000U`

Note:
This constant sets the timeout duration for the TFTP client state machine. If no acknowledgment is received from the server within this period, the transfer is aborted. It depends on

the network latency and the reliability of the connection. Typical values range from 1000ms to 10000ms depending on the network environment.

Uso:
Increase this value if you experience frequent 'timeout' errors on high-latency or congested networks. Decrease it if you need the system to fail fast and recover quickly during a network disconnection. Setting this value too low may cause premature termination of valid transfers on slow links.

`TFTP_ERR_ACCESS`

File: `caos/net/inc/tftp_service.h`

An error code indicating that the TFTP server has denied access to the requested file or directory.

Valore: `2U`

Note:
This constant is returned by the TFTP service layer when a client attempts to read or write a file that violates security permissions or is outside the allowed root directory. It is a standard TFTP error code (Error Code 2) defined in RFC 1350. It does not depend on other flags but requires the file system abstraction layer to be correctly initialized.

Uso:
Use this value when implementing custom file access control logic within the TFTP callback functions. Do not modify this value, as it must remain compliant with the TFTP protocol specification to ensure compatibility with standard TFTP clients.

`TFTP_ERR_DISKFULL`

File: `caos/net/inc/tftp_service.h`

Error code indicating that the TFTP server cannot write data because the storage medium is full.

Valore: `3U`

Note:
This constant is returned by the TFTP service layer when a file write operation fails due to insufficient disk space. It is a specific error enumeration used in the TFTP packet response. It depends on the underlying filesystem driver reporting a 'disk full' status to the network stack.

Uso:
Use this value when implementing custom error handling or logging for TFTP file transfers. Do not modify this value as it must remain compliant with the TFTP protocol error code definitions (RFC 1350). If you see this error frequently, check the filesystem capacity or the file size limits of the storage partition.

`TFTP_ERR_EXISTS`

File: `caos/net/inc/tftp_service.h`

Error code indicating that the file requested for creation already exists on the server.

Valore: `6U`

Note:
This constant is returned by the TFTP service layer when a write request (WRQ) is received for a filename that is already present in the file system. It is a standard TFTP error code (Error Code 6) as defined in RFC 1350. It does not require configuration but must be handled by the client-side logic to prevent accidental overwriting of existing data.

Uso:
Use this constant in your application-level error handling switch-case blocks when implementing TFTP file transfer clients. If you receive this error, you should prompt the user or the automated system to either choose a different filename or explicitly delete the existing file before attempting the upload again.

`TFTP_ERR_ILLEGAL`

File: `caos/net/inc/tftp_service.h`

An error code indicating that the TFTP request received by the server is not supported or violates the protocol format.

Valore: `4U`

Note:
This constant represents the 'Illegal TFTP Operation' error defined in RFC 1350. When returned, the TFTP service will send an error packet to the client and terminate the current transfer session. It is a read-only status code used internally by the caOS network stack and should not be modified by the user.

Uso:
This value is used for debugging network communication issues. If you see this error in your logs, it typically means the TFTP client is sending malformed packets, using an unsupported transfer mode (e.g., requesting 'mail' mode instead of 'octet'), or attempting an operation that the caOS TFTP server does not implement.

`TFTP_ERR_NOTFOUND`

File: `caos/net/inc/tftp_service.h`

Error code returned by the TFTP service when a requested file does not exist on the storage medium.

Valore: `1U`

Note:
This constant is used as a return value in the TFTP protocol handler. It indicates that the server successfully processed the request but could not locate the specified filename. It is typically used in conjunction with the TFTP error packet mechanism (Error Code 1).

Uso:
Do not modify this value as it is defined by the TFTP protocol specification (RFC 1350). Use this constant in your application logic when checking the return status of file system read operations to distinguish between 'file not found' and other hardware or permission errors.

`TFTP_ERR_NOUSER`

File: `caos/net/inc/tftp_service.h`

Error code indicating that the requested TFTP user or account does not exist on the server.

Valore: `7U`

Note:
This is a constant value (7U) used within the TFTP protocol stack to signal an authentication or authorization failure. It is returned by the TFTP service layer when a client attempts to access a resource associated with a non-existent user. It does not affect system memory or scheduling, but acts as a return status for error handling logic.

Uso:
This value should not be changed as it is defined by the TFTP protocol specification (RFC 1350). Use this constant in your application code when implementing custom file access control or user validation logic to ensure compatibility with standard TFTP clients. If you receive this error, verify the user credentials or the file path mapping configured in the TFTP service.

`TFTP_ERR_TID`

File: `caos/net/inc/tftp_service.h`

Defines the specific error code returned by the TFTP service when an unexpected Transfer Identifier (TID) is received.

Valore: `5U`

Note:
This constant is used to identify packets originating from an unknown or unauthorized source port during an active TFTP session. When enabled, the stack compares the incoming packet's TID against the session's expected TID. If they do not match, the system triggers this error code. It is a standard protocol requirement for security and session integrity. The value 5U is the protocol-defined error code for 'Unknown transfer ID'.

Uso:
Do not change this value unless you are implementing a custom, non-standard TFTP variant. Changing this constant will cause the TFTP client to fail to recognize the error, potentially leading to session hangs or security vulnerabilities where unauthorized packets could interfere with an active file transfer.

`TFTP_ERR_UNDEF`

File: `caos/net/inc/tftp_service.h`

Represents a generic or undefined error condition within the TFTP service state machine.

Valore: `0U`

Note:
This constant is used as a fallback error code when a specific protocol violation or network failure does not map to a defined TFTP error type. It is typically returned by the TFTP server task when an unexpected packet structure or internal state mismatch occurs. It does not require specific dependencies but should be handled by the client-side error logging mechanism.

Uso:

This value should not be modified as it is part of the standard TFTP error enumeration. It is used by developers to identify 'catch-all' failure scenarios during debugging. If you encounter this error frequently, check for network packet corruption or incomplete TFTP request frames.

`TFTP_MAX_XFER`

File: `os_config.h`

Defines the maximum number of concurrent TFTP transfer sessions allowed by the caOS TFTP service.

Valore: `2U`

Note:
This constant controls the memory allocation for the TFTP server's internal session table. Increasing this value allows more simultaneous file transfers but consumes more RAM. It is typically set to a small integer (e.g., 2U to 4U) to conserve resources on embedded STM32 targets. It must be defined before initializing the TFTP service via ca_tftp_service_init.

Uso:
Increase this value if your application requires multiple simultaneous TFTP clients to upload or download files. Decrease it if you encounter memory allocation failures or if the system has very limited RAM. Ensure that the total memory footprint of all concurrent sessions does not exceed the available heap.

`TFTP_PORT`

File: `caos/net/inc/tftp_service.h`

Defines the UDP port number used by the caOS TFTP server for file transfer operations.

Valore: `69U`

Note:
This constant sets the listening port for incoming TFTP requests. The default value is 69, which is the IANA-registered port for TFTP. Changing this value requires all TFTP clients to be configured to connect to the new port. It must be a valid 16-bit unsigned integer (1-65535).

Uso:
Change this value only if your network environment requires a non-standard port to avoid conflicts with other services or to bypass specific firewall restrictions. Ensure that the new port is not already reserved by other system services or protocols to prevent binding failures.

`TFTP_RETRY_N`

File: `os_config.h`

Defines the maximum number of times the TFTP client will attempt to retransmit a packet before declaring a timeout error.

Valore: `3U`

Note:
Increasing this value allows the system to tolerate more packet loss on unstable networks but increases the total time taken to report a failed transfer. It depends on the TFTP_TIMEOUT_MS

setting; if the timeout is too short, high retry counts may still fail. Typical values range from 3 to 10 depending on network reliability.

Uso:
Increase this value if you experience frequent 'Transfer Failed' errors during firmware updates over lossy or congested Ethernet links. Do not set this value to 0, as it will disable retransmission entirely, causing the transfer to fail on the first dropped packet.

`TLS_HS_BUF_SIZE`

File: `caos/net/inc/tls.h`

Defines the size in bytes of the internal buffer used for TLS handshake operations.

Valore: `4096U`

Note:
This constant allocates a static memory block for the TLS handshake state machine. Increasing this value allows for larger certificate chains or more complex handshake parameters, but consumes more RAM. Decreasing it may cause handshake failures if the peer sends large certificate chains that exceed the buffer capacity. Typical value is 4096 bytes, which is generally sufficient for standard TLS 1.2/1.3 handshakes.

Uso:
Adjust this value if you encounter 'TLS handshake failed' errors when connecting to servers with large certificate chains. Ensure the value is a multiple of the system's memory alignment requirements. If your system is low on RAM, you may attempt to reduce this value, but be aware that it will limit the maximum size of the handshake messages the device can process.

`TLS_MAX_PLAINTEXT`

File: `caos/net/inc/tls.h`

Defines the maximum size in bytes of a single TLS plaintext record fragment.

Valore: `16384U`

Note:
This constant sets the upper limit for the TLS record layer buffer. Increasing this value allows for larger data chunks per TLS packet, potentially improving throughput for large transfers, but it directly increases the RAM footprint of every active TLS connection. The value 16384 is the standard maximum defined by the TLS specification (RFC 5246).

Uso:
Change this value if you are working on a memory-constrained device and need to reduce RAM usage by lowering the maximum TLS record size. Warning: reducing this value below 16384 may cause compatibility issues with some TLS peers that expect the full standard buffer size. Ensure your heap allocation is sufficient to handle the chosen size multiplied by the number of concurrent TLS connections.

`TLS_READ_CLOSED`

File: `caos/net/inc/tls.h`

A constant representing the error state when a TLS connection has been closed by the remote peer.

Valore: `(-1)`

Note:
This value is returned by ca_tls_read when the underlying socket or TLS session has been gracefully terminated. It is a negative integer (-1), which is standard for indicating an error or EOF condition in network APIs. It does not indicate a hardware failure, but rather a protocol-level closure.

Uso:
Use this constant when checking the return value of ca_tls_read. If the function returns TLS_READ_CLOSED, you should stop attempting to read from that connection and proceed to call ca_tls_close to release the associated resources. Do not treat this as a fatal error; it is the expected behavior when a client or server disconnects.

`TLS_READ_ERROR`

File: `caos/net/inc/tls.h`

Error code indicating a failure during a TLS read operation.

Valore: `(-2)`

Note:
This constant is returned by ca_tls_read when the underlying secure socket fails to retrieve data. It typically signifies a connection reset, a protocol violation, or a timeout at the TLS layer. It is a negative integer (-2), consistent with other error codes in the caOS network stack.

Uso:
Use this value to check the return status of ca_tls_read. If the function returns TLS_READ_ERROR, the application should assume the secure session is no longer reliable and should proceed to close the connection using ca_tls_close and potentially attempt a reconnection.

`TLS_READ_TIMEOUT`

File: `caos/net/inc/tls.h`

Defines the default timeout in milliseconds for TLS read operations.

Valore: `0`

Note:
This constant determines how long the system waits for incoming TLS data before returning an error. A value of 0 typically indicates an infinite timeout (blocking indefinitely). This setting affects the behavior of ca_tls_read. It is a global configuration and does not depend on other flags, but it should be balanced against the system's watchdog requirements to prevent task starvation.

Uso:
Change this value if your application experiences unexpected hangs during network communication or if you need to enforce a strict deadline for data reception. If set to 0, ensure that the calling task has a mechanism to handle potential permanent blocks, such as a watchdog or a separate monitoring thread. Use a non-zero value if you want the system to recover gracefully from network drops or unresponsive peers.

`TLS_RX_BUF_SIZE`

File: `caos/net/inc/tls.h`

Defines the total memory buffer size allocated for incoming TLS data packets.

Valore: `(5U + TLS_MAX_PLAINTEXT + 256U)`

Note:
The value is calculated as 5 bytes (TLS record header) + TLS_MAX_PLAINTEXT (maximum payload size) + 256 bytes (overhead for MAC and padding). Increasing this value consumes more RAM per TLS connection. It must be large enough to accommodate the maximum expected TLS record size; otherwise, incoming packets will be truncated or rejected.

Uso:
Adjust this value if you encounter TLS handshake failures or data corruption when receiving large packets. If your application uses small, fixed-size messages, you may decrease this to save RAM, provided it remains larger than the largest expected TLS record. Ensure the system has sufficient heap memory before increasing this constant.

`TLS_TX_BUF_SIZE`

File: `caos/net/inc/tls.h`

Defines the total size of the TLS transmission buffer in bytes.

Valore: `(5U + TLS_TX_RECORD + 1U + 16U)`

Note:
This constant calculates the memory allocation required for outgoing TLS packets. It accounts for the TLS record header (TLS_TX_RECORD), protocol overhead (5U), padding/-MAC (16U), and a safety margin (1U). It is a static memory allocation; increasing this value directly increases the RAM footprint of every TLS connection context.

Uso:
Adjust this value only if you encounter 'buffer too small' errors during TLS transmission or if you need to optimize RAM usage on memory-constrained STM32 devices. Ensure that the resulting size is sufficient to hold the maximum application payload plus the TLS overhead. Incorrectly low values will cause ca_tls_write to fail or truncate data.

`TLS_TX_RECORD`

File: `caos/net/inc/tls.h`

Defines the maximum size in bytes for a single TLS record transmission buffer.

Valore: `2048U`

Note:
This constant allocates memory for the TLS transmission buffer. Increasing this value allows for larger TLS packets, which can improve throughput for large data transfers but increases the static memory footprint of every TLS connection. Decreasing it saves RAM but may cause fragmentation of large application messages into multiple TLS records, increasing overhead. Typical values are powers of 2, such as 1024, 2048, or 4096.

Uso:

Adjust this value if you encounter 'buffer too small' errors during TLS communication or if you need to optimize RAM usage on memory-constrained devices. Ensure the value is sufficient to hold the maximum expected application payload plus TLS header and MAC overhead. Do not set this value excessively high to avoid exhausting the system heap.

`UART_DBG_BAUDRATE`

File: `os_config.h`

Defines the communication speed in bits per second for the system debug UART interface.

Valore: `CA_UART_BAUD_115200`

Note:
This constant sets the baud rate for the hardware peripheral used for logging and shell commands. It must match the baud rate configured on your terminal emulator (e.g., PuTTY, Tera Term). Valid values are standard baud rates like 9600, 57600, or 115200. Ensure the system clock frequency is high enough to support the requested baud rate with minimal error percentage.

Uso:
Change this value if you experience garbled text in your serial console or if you need to increase throughput for high-frequency logging. Always ensure the hardware connection (TX/RX lines) is stable before increasing the baud rate, as higher speeds are more susceptible to signal noise.

`UART_DBG_NUM`

File: `hw/f1xx/f103/objmap.h`
File: `hw/f4xx/f407/objmap.h`
File: `hw/f4xx/f429/objmap.h`
File: `hw/f4xx/f446/objmap.h`
File: `hw/f7xx/f746/objmap.h`
File: `hw/h5xx/h563/objmap.h`
File: `hw/h7xx/h747/objmap.h`
File: `hw/h7xx/h753/objmap.h`

Specifies the hardware UART peripheral index used for system-wide debug logging and console output.

Valori per MCU:

MCU	Valore
F103	1
F407	1
F429	3
F446	3
F746	3
H563	3
H747	1
H747	3
H753	3

Note:
This constant maps the caOS logging subsystem to a specific physical UART instance (e.g., 1 for USART1, 3 for USART3). Enabling this flag routes all system diagnostic messages to the selected peripheral. It depends on the hardware availability of the specific STM32 MCU variant;

selecting an invalid index will result in a hard fault or silent failure during initialization. Typical values range from 1 to 6 depending on the MCU datasheet.

Uso:
Change this value when porting the firmware to a different STM32 board or when the default debug pins are required for other application functions. Ensure the selected UART is correctly initialized in the hardware abstraction layer (HAL) and that the baud rate matches your terminal settings. Warning: setting this to an incorrect index will disable all debug output, making system troubleshooting significantly more difficult.

`UART_DBG_PINMAP`

File: `caos/drivers/inc/uart.h`

Defines the hardware pin mapping configuration for the debug UART interface.

Valore: `0U`

Note:
This constant selects which physical GPIO pins are assigned to the debug UART peripheral. Setting this to 0U typically selects the default hardware mapping (e.g., PA9/PA10 on many STM32 series). Changing this value requires a corresponding update to the hardware schematic and potentially the GPIO initialization code. It has no direct impact on RTOS scheduling but is critical for serial communication stability.

Uso:
Change this value only when moving the debug console to a different set of pins on the MCU to avoid pin conflicts with other peripherals. Ensure the selected pins support the UART alternate function in the datasheet. Incorrect values will result in no serial output or potential electrical shorts if the pins are configured for conflicting functions.

`UART_STDOUT_NUM`

File: `hw/h7xx/h747/objmap.h`

Specifies the hardware UART peripheral index used for system standard output (printf/logging).

Valori per MCU:

MCU	Valore
H747	1
H747	3

Note:
This constant maps the RTOS logging stream to a specific physical UART port on the STM32. For the H747, valid values are typically 1 or 3, corresponding to USART1 or USART3. Enabling this requires the corresponding GPIO pins and UART peripheral to be initialized in the hardware abstraction layer. If set to an invalid index, system logs will be silently dropped or may cause a hard fault if the peripheral clock is not enabled.

Uso:
Change this value when you need to route debug messages to a different physical connector on your development board. Use 1 if you are using the primary ST-Link virtual COM port or 3 if your specific hardware design routes the console to a different header. Ensure the chosen UART

is not being used by another application task to avoid data corruption or transmission conflicts.

`UDPCLI_MAX_HANDLES`

File: `os_config.h`

Defines the maximum number of concurrent UDP client handles allowed in the caOS network stack.

Valore: `4U`

Note:
This constant determines the size of the internal static memory pool allocated for UDP client structures. Increasing this value consumes more RAM (SRAM) but allows more simultaneous UDP connections. It must be greater than zero and is limited by the available heap or static memory defined in the system linker script.

Uso:
Increase this value if your application requires more simultaneous UDP communication channels (e.g., multiple logging servers or telemetry streams). If you set this value too high, the system may fail to compile or trigger a hard fault due to memory exhaustion. Ensure that the total number of handles does not exceed the hardware's network buffer capacity.

`UDPCLI_RING_DEPTH`

File: `os_config.h`

Defines the maximum number of pending UDP packets that can be stored in the input ring buffer before the system starts dropping new incoming data.

Valore: `4U`

Note:
This value determines the memory allocation for the UDP command line interface buffer. Increasing this value allows the system to handle larger bursts of incoming UDP traffic but consumes more RAM. It must be a power of two for optimal performance in most caOS implementations. Typical values range from 2 to 16 depending on available heap memory.

Uso:
Increase this value if you observe packet loss during high-frequency UDP communication or when the system is busy processing other tasks. Decrease this value if the system is running low on RAM. Setting this value too high may lead to heap exhaustion, while setting it too low will cause the UDP interface to become unresponsive under load.

`UDPSRV_MAX_CLIENTS`

File: `os_config.h`

Defines the maximum number of concurrent UDP client connections supported by the caOS network stack.

Valore: `4U`

Note:

This constant determines the size of the internal connection tracking table. Increasing this value consumes more RAM (each client requires a dedicated control block). It must be set to a value greater than 0. Typical values range from 1 to 16 depending on available heap memory and application requirements.

Uso:
Increase this value if your application needs to handle more simultaneous UDP streams or if you receive 'connection table full' errors. Decrease it to save RAM if your system only communicates with a single master controller or a limited number of peers.

`UDPSRV_RING_DEPTH`

File: `os_config.h`

Defines the number of slots in the UDP server's circular buffer for incoming packets.

Valore: `8U`

Note:
Each slot holds one UDP packet structure. Increasing this value consumes more RAM but allows the system to buffer more incoming packets during traffic bursts. It must be a power of two for optimal performance in the ring buffer logic. Typical range is 4 to 32 depending on available heap memory.

Uso:
Increase this value if the system is dropping UDP packets during high-frequency network traffic. Decrease it if the application is running low on RAM. Ensure that the processing task is fast enough to clear the buffer, otherwise, increasing the depth only delays the inevitable overflow.

`UDPSRV_WORKER_STACK_MIN`

File: `caos/net/inc/udp_server.h`

Defines the minimum stack size in bytes allocated for each UDP server worker thread.

Valore: `2048U`

Note:
This value determines the memory footprint of every spawned UDP worker task. If set too low, the system will trigger a stack overflow exception during complex packet processing. It must be a multiple of the architecture's word size (typically 4 or 8 bytes). Increasing this value consumes more RAM from the heap or static pool, while decreasing it saves memory but risks stability.

Uso:
Increase this value if your UDP server handles complex protocols, performs deep function calls, or uses large local buffers within the worker callback. Decrease it only if RAM is extremely limited and you have verified via stack watermarking that the current usage is significantly lower than 2048 bytes. Always perform stress testing after modifying this value to ensure no stack corruption occurs.

`UNUSED`

File: `caos/inc/common.h`

A macro used to explicitly mark a variable as intentionally unused to prevent compiler warnings.

Valore: `((void)(x))`

Note:
When enabled, the compiler ignores the variable during static analysis, preventing 'unused variable' warnings. It has no effect on the final binary size or execution speed as it evaluates to a void cast. It is typically used in callback functions or API implementations where a parameter is required by the function signature but not needed by the logic.

Uso:
Use this macro whenever you have a function parameter that you do not intend to use in your code. This keeps your build logs clean and ensures that legitimate warnings are not hidden by 'noise'. Do not use this on variables that are intended to be used, as it will mask potential logic errors where a variable was forgotten.

`USB_BCD_DEVICE`

File: `caos/usb/inc/usb_mgmt.h`

Defines the Binary Coded Decimal (BCD) version number of the USB device.

Valore: `0x0100U`

Note:
This value represents the USB specification version the device complies with (e.g., 0x0100 for USB 1.0, 0x0200 for USB 2.0). It is reported to the host during the USB enumeration process via the Device Descriptor. Changing this value does not change the physical hardware capabilities, only the version reported to the operating system.

Uso:
Update this value to match the USB standard your hardware implementation is designed to support. If the host OS expects a higher version than what is reported, it may trigger compatibility warnings or fail to load specific drivers. Ensure the value is formatted as a 16-bit BCD integer (e.g., 0x0200 for version 2.0.0).

`USB_CDC_EP_DATA`

File: `caos/usb/inc/usb_cdc.h`
File: `hw/f1xx/f103/objmap.h`
File: `hw/f4xx/f407/objmap.h`
File: `hw/f4xx/f429/objmap.h`
File: `hw/f4xx/f446/objmap.h`
File: `hw/f7xx/f746/objmap.h`
File: `hw/h5xx/h563/objmap.h`
File: `hw/h7xx/h747/objmap.h`
File: `hw/h7xx/h753/objmap.h`

Defines the hardware endpoint index used for USB CDC data transmission and reception.

Valori per MCU:

MCU	Valore
F103	1U
F407	1U
F429	1U
F446	1U
F746	3U
H563	3U
H747	3U
H753	3U

Note:
This constant specifies the physical endpoint number assigned to the CDC data interface. Enabling a higher value allows the USB stack to map the data stream to a specific hardware buffer. It is strictly dependent on the USB peripheral architecture of the STM32 series; for example, F1/F4 series typically use endpoint 1, while H7/H5 series require endpoint 3. Incorrect values will result in USB enumeration failure or silent data loss.

Uso:
Change this value only when porting the caOS USB stack to a different STM32 MCU family. Do not modify this value for standard application development. Setting an invalid endpoint index will cause the USB host to reject the device or fail to open the virtual COM port.

`USB_CDC_EP_NOTIF`

File: `caos/usb/inc/usb_cdc.h`
File: `hw/f1xx/f103/objmap.h`
File: `hw/f4xx/f407/objmap.h`
File: `hw/f4xx/f429/objmap.h`
File: `hw/f4xx/f446/objmap.h`
File: `hw/f7xx/f746/objmap.h`
File: `hw/h5xx/h563/objmap.h`
File: `hw/h7xx/h747/objmap.h`
File: `hw/h7xx/h753/objmap.h`

Defines the endpoint address used for USB CDC interrupt notifications.

Valori per MCU:

MCU	Valore
F103	2U
F407	2U
F429	2U
F446	2U
F746	4U
H563	4U
H747	4U
H753	4U

Note:
This constant specifies the hardware endpoint index for the CDC notification pipe. Enabling this allows the host to receive asynchronous status updates from the device. It is strictly tied to the USB peripheral's hardware capabilities; values must match the specific MCU's USB controller architecture (e.g., 2U for F4 series, 4U for H7 series). Incorrect values will cause USB enumeration failure or silent data transmission errors.

Uso:
Change this value only when porting the caOS USB stack to a new STM32 family or when reconfiguring the USB descriptor table. Ensure the value matches the hardware endpoint allocation defined in the MCU reference manual. Setting an invalid endpoint index will result in the USB stack failing to initialize the notification interface, preventing the host from detecting serial state changes.

`USB_CDC_FIFO_WORDS_DATA`

File: `caos/usb/inc/usb_cdc.h`

Defines the size of the USB CDC receive buffer in 32-bit words.

Valore: `64U`

Note:
This constant determines the memory allocation for the CDC data FIFO. The total buffer size in bytes is calculated as (USB_CDC_FIFO_WORDS_DATA * 4). Increasing this value allows the system to buffer more incoming data during high-traffic periods, reducing the risk of buffer overflows, but it consumes more RAM. It must be a positive integer, typically a power of two for optimal memory alignment.

Uso:
Adjust this value if you experience data loss during high-speed serial communication. If the application processes incoming USB data slowly, increase this value to provide a larger safety margin. Conversely, if your STM32 device is low on RAM, you may decrease this value, provided the data throughput remains manageable.

`USB_CDC_FIFO_WORDS_NOTIF`

File: `caos/usb/inc/usb_cdc.h`

Defines the size of the USB CDC notification FIFO buffer in 32-bit words.

Valore: `8U`

Note:
This constant determines the depth of the internal queue used to store CDC state change notifications (e.g., serial line state, break signals). Increasing this value allows the system to buffer more events during high-traffic periods, preventing data loss. It must be a positive integer, typically a power of two for optimal performance. It depends on the USB stack's event processing speed; if the application processes notifications slower than they arrive, this buffer will overflow.

Uso:
Increase this value if you observe dropped CDC notifications or 'FIFO overflow' errors in the USB debug logs during heavy serial communication. Decrease it only if you are extremely constrained by RAM, as setting this too low can cause the USB stack to miss critical state changes from the host.

`USB_DFU_ATTR_CAN_DNLOAD`

File: `caos/usb/inc/usb_dfu.h`

A bitmask flag indicating that the DFU device supports firmware download operations.

Valore: `0x01U`

Note:
When set to 0x01, the DFU descriptor informs the host that the device is capable of receiving new firmware images. If this bit is cleared, the host will treat the device as read-only, preventing any firmware update attempts. This flag is a mandatory component of the DFU functional descriptor and must be consistent with the actual implementation of the DFU_DNLOAD request handler in the application layer.

Uso:
Enable this flag when you want to allow users to update the device firmware via USB. You should only disable this if the device is intended to be strictly read-only or if the DFU interface is being used solely for status reporting. Incorrectly setting this flag while the backend code lacks download support will result in the host attempting to send data that the device will reject, potentially causing the host-side DFU tool to report an error.

`USB_DFU_ATTR_CAN_UPLOAD`

File: `caos/usb/inc/usb_dfu.h`

A bitmask flag indicating that the DFU device supports uploading (reading) firmware from the device memory to the host.

Valore: `0x02U`

Note:
When this bit is set in the DFU functional descriptor, the host is notified that the device supports the DFU_UPLOAD request. If disabled, the host will typically grey out or disable the 'Read' or 'Extract' functionality in DFU tools like dfu-util. This flag is usually combined with other attributes using a bitwise OR operation.

Uso:
Enable this flag if you want to allow users to back up or verify the firmware currently running on the STM32. Disable it if you need to protect intellectual property or prevent firmware extraction for security reasons. Ensure your DFU implementation actually handles the DFU_UPLOAD state machine requests before setting this flag, otherwise, the host may encounter errors during read attempts.

`USB_DFU_ATTR_MANIFEST_TOL`

File: `caos/usb/inc/usb_dfu.h`

Defines the DFU manifest tolerance attribute for the USB device descriptor.

Valore: `0x04U`

Note:
This flag sets the bit 2 of the DFU attributes field, indicating that the device can communicate during the manifest phase. When enabled, the host expects the device to remain responsive after a download; if disabled, the host assumes the device will reset immediately. It must be consistent with the device's firmware update implementation and the DFU class specification.

Uso:
Change this value if the host-side DFU tool reports timeout errors or fails to verify the firmware after the download phase. Set this to 0x04U if your application requires the device to perform

post-download processing (like internal flash verification) before resetting. Ensure the host-side driver supports this attribute to avoid communication stalls.

`USB_DFU_ATTR_WILL_DETACH`

File: `caos/usb/inc/usb_dfu.h`

A bitmask flag indicating that the USB device will perform a self-detach sequence before entering DFU mode.

Valore: `0x08U`

Note:
When this bit is set in the DFU functional descriptor, the host expects the device to disconnect and reconnect automatically after receiving a DFU_DETACH request. If disabled, the host may wait indefinitely for a disconnect that never happens. This flag is typically ORed with other attributes in the bmAttributes field of the DFU descriptor.

Uso:
Enable this flag when your application firmware needs to trigger a software reset or jump to the bootloader automatically upon a DFU request. If you disable this, you must manually disconnect and reconnect the USB cable or toggle the D+ pull-up resistor to force the host to re-enumerate the device in DFU mode.

`USB_DFU_STR_IFACE`

File: `caos/usb/inc/usb_dfu.h`

Defines the string descriptor index for the USB Device Firmware Upgrade (DFU) interface.

Valore: `4U`

Note:
This constant sets the index value used by the USB host to identify the DFU interface string in the device descriptor table. It must match the index defined in the USB string descriptor array. Changing this value requires a corresponding update in the string descriptor table to avoid enumeration errors or incorrect interface labeling in the OS.

Uso:
Change this value only if you are modifying the USB descriptor structure or if there is a conflict with other USB interface string indices. Ensure the value is unique among all interface descriptors to prevent host-side identification issues. Incorrect values will cause the DFU interface to fail to initialize or appear with an incorrect name in the device manager.

`USB_DFU_XFER_MAX`

File: `caos/usb/inc/usb_dfu.h`

Defines the maximum size in bytes for a single USB Device Firmware Upgrade (DFU) data transfer packet.

Valore: `64U`

Note:
This constant sets the buffer size used during DFU firmware download operations. The value 64U is standard for Full-Speed USB endpoints. Increasing this value requires matching support

in the host-side DFU tool and the underlying USB hardware endpoint configuration. If set too high, it may cause buffer overflows or USB transaction errors.

Uso:
Change this value only if you are optimizing DFU transfer speeds and have verified that the USB hardware endpoint descriptors and the host-side DFU software can handle larger packet sizes. For most standard STM32 implementations, 64 bytes is the optimal and safest value.

`USB_EP0_MPS`

File: `hw/f1xx/f103/objmap.h`
File: `hw/f4xx/f407/objmap.h`
File: `hw/f4xx/f429/objmap.h`
File: `hw/f4xx/f446/objmap.h`
File: `hw/f7xx/f746/objmap.h`
File: `hw/h5xx/h563/objmap.h`
File: `hw/h7xx/h747/objmap.h`
File: `hw/h7xx/h753/objmap.h`

Defines the Maximum Packet Size (MPS) in bytes for the USB Endpoint 0 (Control Endpoint).

Valori per MCU:

MCU	Valore
F103	64U
F407	64U
F429	64U
F446	64U
F746	64U
H563	64U
H747	64U
H753	64U

Note:
This constant sets the buffer size for the initial handshake and enumeration phase of the USB device. For Full-Speed USB, the standard value is 64 bytes. Changing this value affects the hardware peripheral's internal FIFO allocation. It must match the value defined in the USB Device Descriptor to avoid enumeration failures or data corruption.

Uso:
Change this value only if you are implementing a custom USB stack or working with a specific hardware requirement that deviates from the USB 2.0 Full-Speed specification. Setting this to a value other than 64 on standard STM32 hardware will likely cause the host computer to fail to recognize the device during the initial enumeration process.

`USB_EP0_TX_BUF`

File: `hw/h7xx/h747/objmap.h`
File: `hw/h7xx/h753/objmap.h`

Defines the size in bytes of the transmission buffer for USB Endpoint 0.

Valori per MCU:

MCU	Valore
H747	512U
H753	512U

Note:
This constant determines the maximum packet size the device can send during the control transfer phase. For STM32H7 series microcontrollers, the value is typically set to 512 bytes to match the hardware FIFO capacity. Changing this value requires a corresponding update in the USB descriptor tables; if the buffer size is smaller than the requested data length, the USB stack may fail to enumerate or cause data corruption.

Uso:
Adjust this value only if you are modifying the USB hardware abstraction layer or implementing custom USB classes that require specific buffer alignment. For standard HID or CDC implementations on H747/H753, keep this at 512U. Setting this value too low will cause the host to report a device malfunction, while setting it higher than the physical FIFO size will lead to memory overflows.

`USB_EP0_XFR_MAX`

File: `hw/h7xx/h747/objmap.h`
File: `hw/h7xx/h753/objmap.h`

Defines the maximum transfer size in bytes for the USB Control Endpoint 0 (EP0).

Valori per MCU:

MCU	Valore
H747	127U
H753	127U

Note:
This constant sets the buffer limit for EP0 data transactions. For STM32H7 series microcontrollers, the value is fixed at 127U to align with the hardware FIFO constraints and USB specification requirements for control transfers. Disabling or modifying this value outside the recommended range will likely cause USB enumeration failures or data corruption during the setup phase.

Uso:
This value should generally not be changed by the user. It is configured to ensure the USB stack correctly handles standard requests during device enumeration. If you encounter 'USB Device Not Recognized' errors or stalls during the initial handshake, verify that this constant matches the hardware-specific requirement for your target MCU.

`USB_EPBIT`

File: `caos/usb/inc/usb_mgmt.h`

A macro that calculates the bitmask for a specific USB endpoint index.

Valore: `(1U « (n))`

Note:
This constant shifts the value 1 to the left by 'n' positions, where 'n' is the endpoint number (0-7). It is used to create a bitmask for enabling, disabling, or checking the status of specific USB

endpoints in the hardware registers. It depends on the hardware's endpoint count; providing an 'n' greater than the physical endpoint limit will result in an invalid bitmask.

Uso:
Use this macro when configuring USB endpoint interrupts or status registers to ensure code readability and avoid manual bit-shifting errors. Always ensure the index 'n' is within the valid range defined by the STM32 USB peripheral specifications to prevent undefined behavior or register corruption.

`USB_EPMASK_CDC`

File: `caos/usb/inc/usb_mgmt.h`

A bitmask defining which USB endpoints are reserved for the CDC (Communication Device Class) serial emulation interface.

Valore: `(USB_EPBIT(USB_CDC_EP_NOTIF) | USB_EPBIT(USB_CDC_EP_DATA))`

Note:
This constant combines the notification and data endpoints into a single bitmask used by the USB stack to manage resource allocation. Enabling this flag ensures the USB controller correctly routes traffic to the CDC driver. It depends on the definitions of USB_CDC_EP_NOTIF and USB_CDC_EP_DATA. Typical values are derived from the bitwise OR of these two specific endpoint identifiers.

Uso:
Modify this value only if you are changing the hardware endpoint configuration in the USB descriptor. If you add or remove CDC endpoints, you must update this mask to prevent the USB stack from ignoring or mismanaging the traffic. Setting an incorrect mask will cause the USB device to fail enumeration or result in a complete loss of serial communication.

`USB_EPMASK_MSC`

File: `caos/usb/inc/usb_mgmt.h`

Defines the bitmask for the Mass Storage Class (MSC) endpoint within the USB peripheral configuration.

Valore: `(USB_EPBIT(USB_MSC_EP_DATA))`

Note:
This constant acts as a bitwise identifier for the MSC data endpoint. Enabling or modifying this flag directly affects the USB interrupt handler's ability to route data packets to the correct buffer. It depends on the USB_EPBIT macro and the specific endpoint address defined in the hardware abstraction layer. Typical values are restricted to valid endpoint bit positions defined by the STM32 USB hardware registers.

Uso:
Change this value only if you are remapping the USB endpoint configuration in the hardware descriptor table. It is used to filter incoming USB events to ensure the MSC driver processes only relevant traffic. Incorrect values will cause the USB device to fail enumeration or result in data corruption during file transfer operations.

USB_EPMASK_NCM

File: caos/usb/inc/usb_mgmt.h

A bitmask defining the specific USB endpoints used for the Network Control Model (NCM) class.

Valore: (USB_EPBIT(USB_NCM_EP_NOTIF) | USB_EPBIT(USB_NCM_EP_DATA))

Note:
This constant combines the notification endpoint and the data endpoint bits into a single mask. It is used by the caOS USB stack to identify which endpoints are reserved for NCM traffic. It depends on the definitions of USB_NCM_EP_NOTIF and USB_NCM_EP_DATA. If these endpoints are changed in the hardware descriptor, this mask must be updated accordingly to prevent resource conflicts.

Uso:
Modify this value only if you are reconfiguring the USB endpoint layout for the NCM interface. Incorrect values will cause the USB stack to fail to initialize the NCM class or lead to data corruption if endpoints overlap with other USB functions (like HID or CDC). Ensure the bits defined here match the endpoint addresses specified in your device's USB configuration descriptor.

USB_EP_MAX

File: hw/f1xx/f103/objmap.h
File: hw/f4xx/f407/objmap.h
File: hw/f4xx/f429/objmap.h
File: hw/f4xx/f446/objmap.h
File: hw/f7xx/f746/objmap.h
File: hw/h5xx/h563/objmap.h
File: hw/h7xx/h747/objmap.h
File: hw/h7xx/h753/objmap.h

Defines the maximum number of physical USB endpoints supported by the hardware abstraction layer.

Valori per MCU:

MCU	Valore
F103	4U
F407	4U
F429	4U
F446	4U
F746	6U
H563	8U
H747	9U
H753	9U

Note:
This constant sets the size of the internal endpoint descriptor table. Enabling a higher value increases RAM usage for endpoint buffers and control structures. It must match the physical capabilities of the specific STM32 series (e.g., 4 for F103, 9 for H747). Setting this value higher than the hardware limit will cause memory corruption or hard faults during USB initialization.

Uso:

Change this value only when porting the firmware to a different STM32 MCU series or when the specific application requires more endpoints than the default configuration. Ensure the value is exactly equal to or greater than the number of endpoints defined in your USB device descriptor; otherwise, the USB stack will fail to enumerate or crash when accessing undefined endpoints.

`USB_MPU_REGION`

File: `hw/f1xx/f103/objmap.h`
File: `hw/f4xx/f407/objmap.h`
File: `hw/f4xx/f429/objmap.h`
File: `hw/f4xx/f446/objmap.h`
File: `hw/f7xx/f746/objmap.h`
File: `hw/h5xx/h563/objmap.h`
File: `hw/h7xx/h747/objmap.h`
File: `hw/h7xx/h753/objmap.h`

Defines the specific Memory Protection Unit (MPU) region index reserved for USB peripheral memory access.

Valori per MCU:

MCU	Valore
F103	3U
F407	3U
F429	3U
F446	3U
F746	3U
H563	3U
H747	3U
H753	3U

Note:
This constant assigns a hardware MPU slot to the USB controller's memory space. Enabling this ensures the RTOS can enforce memory access permissions for USB buffers. It is currently set to 3U across all supported STM32 families. Changing this value requires ensuring that no other system peripheral or task is assigned to the same MPU region index, as this would cause memory access conflicts or hard faults.

Uso:
You should change this value only if you are porting the RTOS to a new MCU architecture where region 3 is already occupied by critical system memory or another peripheral. If you encounter a MemManage fault during USB data transfers, verify that this region is correctly configured in the MPU initialization sequence and does not overlap with other defined regions.

`USB_MSC_EP_DATA`

File: `caos/usb/inc/usb_msc.h`
File: `hw/f1xx/f103/objmap.h`
File: `hw/f4xx/f407/objmap.h`
File: `hw/f4xx/f429/objmap.h`
File: `hw/f4xx/f446/objmap.h`
File: `hw/f7xx/f746/objmap.h`
File: `hw/h5xx/h563/objmap.h`
File: `hw/h7xx/h747/objmap.h`

File: `hw/h7xx/h753/objmap.h`

Defines the hardware endpoint index used for USB Mass Storage Class (MSC) data transfers.

Valori per MCU:

MCU	Valore
F103	3U
F407	3U
F429	3U
F446	3U
F746	2U
H563	2U
H747	2U
H753	2U

Note:
This constant specifies the physical endpoint number on the USB peripheral dedicated to bulk data transmission. Enabling this allows the MCU to act as a USB flash drive. It is hardware-dependent: F1/F4 series typically use index 3, while F7/H5/H7 series use index 2. Ensure this value matches the hardware abstraction layer (HAL) configuration for the specific STM32 family to avoid enumeration failures.

Uso:
Change this value only when porting the firmware to a different STM32 microcontroller family. Incorrect values will cause the USB device to fail enumeration or result in data corruption during file transfers. Always verify the endpoint availability in the specific STM32 Reference Manual for your target MCU.

`USB_MSC_FIFO_WORDS_DATA`

File: `caos/usb/inc/usb_msc.h`

Defines the size of the USB Mass Storage Class (MSC) data buffer in 32-bit words.

Valore: `64U`

Note:
This constant determines the memory allocated for the USB MSC data FIFO. Increasing this value allows for larger data bursts during USB transfers, potentially improving throughput. It must be a multiple of the USB endpoint packet size (typically 64 bytes/16 words for Full Speed). Setting this too high will consume excessive RAM, while setting it too low may cause buffer underruns or performance degradation.

Uso:
Adjust this value if you experience slow file transfer speeds or USB communication errors during heavy I/O operations. Increase it if the system has sufficient RAM and you need to optimize throughput. Ensure the value does not exceed the available heap or static memory limits defined in your linker script.

`USB_NCM_EP_DATA`

File: `caos/usb/inc/usb_ncm.h`
File: `hw/f1xx/f103/objmap.h`
File: `hw/f4xx/f407/objmap.h`

File: hw/f4xx/f429/objmap.h
File: hw/f4xx/f446/objmap.h
File: hw/f7xx/f746/objmap.h
File: hw/h5xx/h563/objmap.h
File: hw/h7xx/h747/objmap.h
File: hw/h7xx/h753/objmap.h

Defines the endpoint address used for USB Network Control Model (NCM) data transfers.

Valori per MCU:

MCU	Valore
F103	1U
F407	1U
F429	1U
F446	1U
F746	1U
H563	1U
H747	1U
H753	1U

Note:
This constant specifies the hardware endpoint index for the USB NCM data pipe. It must match the endpoint configuration defined in the USB descriptor table. Changing this value requires a corresponding update in the USB device descriptor; otherwise, the host will fail to enumerate the device or communicate with the network interface. The value is typically set to 1 for standard STM32 USB peripheral implementations.

Uso:
Change this value only if you are modifying the USB hardware endpoint mapping or resolving endpoint conflicts with other USB classes (e.g., CDC or HID) sharing the same peripheral. Ensure the value is within the valid range supported by the specific STM32 USB IP core (usually 1 to 5). Incorrect values will result in USB enumeration failure or data transmission errors.

USB_NCM_EP_NOTIF

File: caos/usb/inc/usb_ncm.h
File: hw/f1xx/f103/objmap.h
File: hw/f4xx/f407/objmap.h
File: hw/f4xx/f429/objmap.h
File: hw/f4xx/f446/objmap.h
File: hw/f7xx/f746/objmap.h
File: hw/h5xx/h563/objmap.h
File: hw/h7xx/h747/objmap.h
File: hw/h7xx/h753/objmap.h

Defines the endpoint address used for USB NCM (Network Control Model) status notifications.

Valori per MCU:

MCU	Valore
F103	2U
F407	2U
F429	2U
F446	2U
F746	5U
H563	5U
H747	5U
H753	5U

Note:
This constant specifies the hardware endpoint index for the interrupt-based notification pipe. Values are hardware-dependent based on the USB peripheral controller architecture: F1/F4 series typically use 2U, while high-performance F7/H5/H7 series use 5U. Ensure this value matches the endpoint descriptor defined in your USB stack configuration to avoid enumeration failures.

Uso:
Change this value only when porting the NCM stack to a new STM32 series or if the hardware endpoint mapping is modified in the USB peripheral configuration. Setting an incorrect value will cause the USB host to fail to initialize the network interface, as the notification pipe will be unreachable.

`USB_NCM_FIFO_WORDS_DATA`

File: `caos/usb/inc/usb_ncm.h`

Defines the size of the USB NCM data FIFO buffer in 32-bit words.

Valore: `64U`

Note:
This constant determines the memory allocation for the NCM data packet queue. Increasing this value allows the system to buffer more data packets before dropping them, which improves throughput during high-traffic bursts. It depends on the available RAM in the heap; setting this too high may cause memory allocation failures. Typical values range from 64 to 256 depending on the available SRAM.

Uso:
Increase this value if you observe dropped packets or performance bottlenecks during high-speed USB data transfers. Decrease it if the system is running low on heap memory. Ensure that the value is a multiple of the USB packet size to maintain alignment and efficiency.

`USB_NCM_FIFO_WORDS_NOTIF`

File: `caos/usb/inc/usb_ncm.h`

Defines the size of the notification FIFO buffer in 32-bit words for the USB NCM class.

Valore: `8U`

Note:
This constant sets the depth of the internal queue used to store USB NCM notification packets. Increasing this value allows the system to buffer more pending notifications before they are sent to the host, which helps prevent data loss during high-traffic bursts. It must be a positive

integer, typically a power of two for optimal memory alignment. It depends on the available heap memory and the frequency of the USB polling interval.

Uso:
Adjust this value if you observe 'FIFO overflow' errors or dropped network status notifications in the USB NCM stack. Increase the value if the host is slow to poll the device or if the system generates a high volume of status updates. Be cautious: setting this value too high will unnecessarily consume precious RAM on the STM32 microcontroller.

`USB_NCM_MAX_FRAME`

File: `caos/usb/inc/usb_ncm.h`

Defines the maximum size in bytes of an Ethernet frame supported by the USB NCM class driver.

Valore: `1514U`

Note:
This value sets the buffer size for incoming and outgoing network packets. It must be at least 1514 bytes to accommodate a standard Ethernet frame (1500 bytes payload + 14 bytes header). Increasing this value consumes more RAM in the USB peripheral's memory pool. It is typically set to 1514, but can be increased if jumbo frames are required by the network stack.

Uso:
Change this value if you need to support jumbo frames or if the network stack reports buffer overflow errors during high-traffic periods. Ensure that the allocated USB buffer memory in the linker script or memory pool configuration is large enough to hold this value multiplied by the number of active USB endpoints.

`USB_PID_BASE_CDC`

File: `os_config.h`

Defines the base Product ID (PID) offset for the USB CDC device descriptor.

Valore: `0x0010U`

Note:
This value acts as the starting point for the USB PID enumeration. When enabled, the system adds this offset to the base USB vendor ID configuration. It must be a 16-bit hexadecimal value. Changing this requires a corresponding update in the host-side driver INF/configuration files to ensure the OS correctly identifies the device.

Uso:
Change this value when you need to differentiate between multiple USB device configurations or to avoid PID conflicts with other hardware on the same host. Ensure the value does not conflict with reserved system PIDs. Incorrect values will result in the host failing to recognize the device or loading the wrong driver.

`USB_PID_BASE_CUSTOM`

File: `caos/usb/inc/usb_custom.h`

Defines the base Product ID (PID) offset for custom USB device descriptors in caOS.

Valore: `0x0000U`

Note:
This constant acts as the starting value for the USB PID when the system is configured for custom hardware. It must be a 16-bit unsigned integer. Changing this value directly affects the USB enumeration process on the host PC. It is typically used in conjunction with USB_VID_BASE_CUSTOM to ensure unique identification of custom-designed peripherals.

Uso:
Change this value when you are developing a new custom USB device and need to avoid conflicts with existing hardware IDs on the host machine. Ensure the value matches the PID defined in your host-side driver or INF file. Setting this to an incorrect value will cause the device to fail enumeration or be recognized as the wrong device type by the operating system.

`USB_PID_BASE_DFU`

File: `os_config.h`

Defines the base Product ID (PID) offset used by the caOS USB DFU (Device Firmware Upgrade) bootloader mode.

Valore: `0x0040U`

Note:
This value acts as the base identifier for the USB device descriptor when the system enters DFU mode. It is typically added to a hardware-specific offset to ensure unique identification. Valid range is 0x0000 to 0xFFFF. Changing this requires a corresponding update in the host-side DFU utility (e.g., dfu-util) configuration to ensure the device is correctly recognized during firmware updates.

Uso:
Modify this value only if you are creating a custom hardware variant that requires a unique USB PID to avoid conflicts with other devices on the same host machine. Ensure that the resulting PID does not conflict with the standard application-mode PID defined elsewhere in the project. Incorrect values will cause the host computer to fail to detect the device in DFU mode, preventing firmware flashing.

`USB_PID_BASE_MSC`

File: `os_config.h`

Defines the base Product ID (PID) offset for the Mass Storage Class (MSC) USB device interface.

Valore: `0x0020U`

Note:
This value acts as a base identifier for the USB descriptor. When enabled, the system adds this offset to the device's unique hardware ID to generate the final PID. It must be a 16-bit unsigned integer. Changing this value requires a corresponding update in the host-side driver or INF file to ensure the OS correctly recognizes the device.

Uso:
Modify this value if you are deploying multiple devices with the same hardware configuration on the same host and need to avoid PID collisions. Ensure the resulting PID does not conflict

with other USB classes defined in the system. Incorrect values will cause the host computer to fail to enumerate the device or load the wrong driver.

`USB_PID_BASE_NCM`

File: `os_config.h`

Defines the base Product ID (PID) offset for the USB NCM (Network Control Model) device class.

Valore: `0x0030U`

Note:
This value acts as the starting point for the USB PID enumeration. When enabled, the system adds this offset to the base USB device ID to distinguish the NCM interface from other USB classes. It must be a 16-bit unsigned integer. Changing this value requires a corresponding update in the host-side driver INF or configuration files to ensure the OS correctly identifies the device.

Uso:
Change this value if you encounter USB device conflicts when multiple caOS-based devices are connected to the same host, or if the default PID overlaps with existing hardware IDs in your product line. Ensure the new value does not conflict with other USB classes defined in the same project to prevent enumeration failures.

`USB_STR_MANUFACTURER`

File: `caos/usb/inc/usb_mgmt.h`

Defines the manufacturer string descriptor displayed by the USB device when connected to a host.

Valore: `"caOS"`

Note:
This constant sets the string returned during the USB enumeration process. It must be a valid null-terminated string. Changing this value does not affect system performance or memory usage, but it is required for proper USB device identification by the operating system.

Uso:
Update this value to reflect your organization's name or the specific product branding. Change this if you need to distinguish between different hardware revisions or custom firmware builds in the device manager of the host computer. Ensure the string length does not exceed the maximum descriptor size defined in the USB specification.

`USB_STR_PRODUCT`

File: `caos/usb/inc/usb_mgmt.h`

Defines the product string descriptor displayed by the USB device when connected to a host.

Valore: `"caOS USB Device"`

Note:
This string is sent to the host OS (Windows, Linux, macOS) during the USB enumeration process. It must be a valid UTF-16LE encoded string. Changing this value does not affect

system performance or memory usage, but it is critical for device identification in the Device Manager or system logs.

Uso:
Update this value to reflect your specific product name for branding purposes. Ensure the string is not excessively long, as some host controllers have limits on descriptor lengths. Do not use special characters that might cause encoding issues on specific host operating systems.

`USB_STR_SERIAL`

File: `caos/usb/inc/usb_mgmt.h`

Defines the unique serial number string descriptor for the USB device identification.

Valore: `"0001"`

Note:
This value is sent to the host computer during USB enumeration to uniquely identify the specific hardware instance. It must be a string of characters. If multiple devices with the same serial number are connected to the same host, it may cause driver conflicts or OS-level identification issues.

Uso:
Change this value during production to assign a unique ID to every individual board. Use this when you need to distinguish between multiple identical devices connected to the same PC. Ensure the value remains a valid string format as expected by the USB stack; avoid using non-printable characters.

`USB_VID`

File: `caos/usb/inc/usb_mgmt.h`

Defines the Vendor ID (VID) for the USB device descriptor, identifying the manufacturer to the host computer.

Valore: `0xCA01U`

Note:
This 16-bit hexadecimal value is reported to the USB host during enumeration. It must be unique and registered with the USB-IF for commercial products. Changing this value requires a re-enumeration of the USB device on the host side. It is typically paired with a corresponding USB_PID (Product ID).

Uso:
Change this value only if your organization has a registered Vendor ID from the USB-IF. Using a random or unauthorized ID may cause driver conflicts or identification issues on Windows/Linux/macOS hosts. For internal prototyping, ensure it does not collide with other devices connected to the same host.

`X25519_KEY_SIZE`

File: `caos/crypto/inc/x25519.h`

Defines the fixed byte length of an X25519 public or private key.

Valore: `32`

Note:
This constant is set to 32 bytes, which is the standard size for X25519 keys as defined by RFC 7748. Changing this value will cause cryptographic functions to fail or result in buffer overflows, as the underlying algorithms expect exactly 256 bits of data.

Uso:
This value should never be changed for standard X25519 operations. It is used primarily for static buffer allocation in structures and as a boundary check in cryptographic API calls. If you are porting code or writing unit tests, use this constant instead of hardcoding the number 32 to ensure maintainability.

`_SAFEGUARD_DISPATCH`

File: `caos/inc/safeguard.h`

Enables or disables the runtime safety check for task dispatching to prevent illegal context switches.

Valore: `_SAFEGUARD_##N`

Note:
When enabled, the kernel validates the task state before performing a context switch. If disabled, the system skips these checks, reducing CPU overhead but increasing the risk of kernel panics during invalid task transitions. This flag depends on the system's debug level and should be enabled during development. Valid values are 0 (disabled) or 1 (enabled).

Uso:
Enable this flag during the development and testing phases to catch illegal task state transitions early. Disable it only in final production builds if you have verified system stability and require maximum performance, as disabling it masks potential logic errors in task management.

`_SAFEGUARD_DISPATCH_`

File: `caos/inc/safeguard.h`

Enables a watchdog-style safety check that monitors the execution time of task dispatching cycles.

Valore: `_SAFEGUARD_DISPATCH(N)`

Note:
When enabled, the system tracks the duration of the scheduler's context switch. If the dispatch time exceeds N microseconds, the system triggers a kernel panic or error handler. This flag depends on the hardware timer resolution defined in the BSP. Typical values for N range from 10 to 100, depending on the CPU clock speed and the complexity of the task list.

Uso:
Use this to detect if high-priority interrupts or poorly written kernel hooks are blocking the scheduler for too long. Increase N if you encounter false-positive system resets during heavy interrupt load. Do not set N to an excessively low value, as it may cause the system to crash during normal operation due to jitter.

_SAFEGUARD_NARG

File: caos/inc/safeguard.h

A preprocessor macro used to count the number of arguments passed to a variadic macro.

Valore: _SAFEGUARD_NARG_(__VA_ARGS__, _SAFEGUARD_RSEQ())

Note:
This constant relies on the _SAFEGUARD_RSEQ helper sequence to calculate the argument count at compile-time. It is a fundamental building block for variadic macros in caOS. It has no direct impact on runtime memory or CPU usage, but it must be defined correctly to prevent compilation errors in macros that use variable arguments.

Uso:
Use this when creating custom macros that need to behave differently based on the number of arguments provided (e.g., overloading a function-like macro). Do not modify this value unless you are refactoring the core macro infrastructure of the RTOS, as incorrect implementation will break all variadic macros throughout the system.

_SAFEGUARD_NARG_

File: caos/inc/safeguard.h

A macro used to count the number of arguments passed to a variadic macro.

Valore: _SAFEGUARD_ARG_N(__VA_ARGS__)

Note:
This constant acts as a preprocessor utility that expands to the integer count of arguments provided in a macro call. It relies on the _SAFEGUARD_ARG_N helper macro. It is a compile-time mechanism and has no runtime memory footprint. It is typically used in variadic macro definitions to dispatch to different function implementations based on the number of arguments provided.

Uso:
Use this when creating flexible macros that need to handle a variable number of inputs (e.g., logging functions or initialization routines). Do not modify the underlying implementation unless you are familiar with C preprocessor variadic expansion techniques, as incorrect changes will cause compilation errors or unexpected macro expansion behavior.

_SAFEGUARD_RSEQ

File: caos/inc/safeguard.h

Configures the strictness level of the Restart Sequence (RSEQ) integrity monitor during system boot and task recovery.

Valore: 2, 1, 0

Note:
Setting 0 disables the safeguard; 1 enables basic validation of the restart vector; 2 enables full memory-mapped integrity checks. This flag depends on the hardware watchdog being initialized. Valid range is 0 to 2. Higher values increase boot time due to additional checksum verification.

Uso:

Set to 2 during development and debugging to catch corrupted task stacks or invalid jump addresses early. Set to 1 for production builds to balance safety and boot speed. Avoid setting to 0 unless you are operating in a resource-constrained environment where boot time is critical and memory integrity is guaranteed by external hardware.

`__disable_irq`

File: `caos/inc/common.h`

A low-level assembly instruction that globally disables all maskable hardware interrupts on the STM32 processor.

Valore: `__asm volatile ("cpsid i" : : : "memory")`

Note:
When enabled, the CPU stops responding to peripheral and external interrupts, effectively freezing the scheduler and background tasks. This is a critical section primitive. It does not affect Non-Maskable Interrupts (NMI) or HardFaults. It must always be paired with a corresponding ___enable_irq() call to restore system functionality.

Uso:
Use this when you need to protect a shared resource (like a global variable or hardware register) from being modified by an Interrupt Service Routine (ISR) while the main thread is accessing it. Warning: keeping interrupts disabled for too long will cause the system to miss real-time events, leading to watchdog resets or data loss in communication peripherals.

`__enable_irq`

File: `caos/inc/common.h`

A macro that globally enables all maskable interrupts on the ARM Cortex-M processor.

Valore: `__asm volatile ("cpsie i" : : : "memory")`

Note:
This instruction clears the PRIMASK register, allowing the CPU to respond to hardware interrupts again. It is the counterpart to ___disable_irq. It has no dependencies on other RTOS flags but must be used carefully to avoid race conditions. It is a low-level assembly instruction that affects the CPU core directly.

Uso:
Use this to exit a critical section where you have temporarily disabled interrupts to protect shared resources. You should only call this after you have finished accessing the protected data. Warning: calling this prematurely or in the wrong context can lead to data corruption or system instability if the critical section was not fully completed.

`ca_be16toh`

File: `caos/inc/endian.h`

Converts a 16-bit unsigned integer from big-endian byte order to host-endian byte order.

Valore: `ca_bswap16(x)`

Note:

This macro uses ca_bswap16 to perform a byte-swap operation if the host architecture is little-endian, or returns the value unchanged if the host is big-endian. It is a compile-time or runtime abstraction depending on the compiler's ability to optimize the swap. It has no external dependencies other than the ca_bswap16 implementation.

Uso:
Use this when parsing network packets or binary data structures received from big-endian hardware or protocols. Failure to use this on a little-endian STM32 (ARM Cortex-M) will result in corrupted data values where the high and low bytes are swapped. Ensure the input is exactly 16 bits to avoid unexpected behavior.

`ca_be32toh`

File: `caos/inc/endian.h`

Converts a 32-bit unsigned integer from big-endian byte order to host-endian byte order.

Valore: `ca_bswap32(x)`

Note:
This macro utilizes ca_bswap32 to perform a byte-swap operation if the host architecture is little-endian, or acts as a no-op if the host is already big-endian. It is a compile-time abstraction that ensures network-to-host byte order compatibility. It has no dependencies on other RTOS flags but relies on the underlying hardware's endianness configuration.

Uso:
Use this macro when parsing 32-bit data received from network protocols or external sensors that transmit in big-endian format. Failure to use this on little-endian STM32 microcontrollers will result in corrupted data values. Do not use this on data that is already in host-native format, as it will incorrectly reverse the byte order.

`ca_be64toh`

File: `caos/inc/endian.h`

Converts a 64-bit integer from big-endian byte order to host-endian byte order.

Valore: `ca_bswap64(x)`

Note:
This macro uses ca_bswap64 to perform a byte-level swap. If the host system is already big-endian, this operation typically resolves to a no-op or a direct assignment. It is a dependency for network protocol parsing and file system drivers where data is stored in big-endian format regardless of the STM32 architecture.

Uso:
Use this macro when receiving 64-bit data packets from network interfaces or external sensors that transmit in big-endian format. Failure to use this on a little-endian STM32 will result in corrupted data values. Ensure the input variable is exactly 64 bits wide to avoid unexpected truncation or padding issues.

---

ca_cli_char_class

File: caos/inc/cli.h

Defines the character classification mode for the command line interface (CLI) input processing.

Valore: (0U)

Note:
When set to 0U, the CLI uses a basic ASCII character set for command parsing. Enabling higher values typically enables extended character support or specific input filtering modes. This constant depends on the underlying UART driver configuration and the buffer size allocated for CLI input. Valid range is typically 0U to 2U depending on the specific caOS build version.

Uso:
Change this value if the CLI fails to recognize special characters or if you need to restrict input to alphanumeric characters only. Setting this incorrectly may cause the CLI to ignore valid commands or crash when receiving unexpected input sequences. Ensure that your terminal emulator settings match the character class defined here.

---

ca_cli_exec_capture

File: caos/inc/cli.h

Enables or disables the redirection of command-line interface (CLI) output to a software buffer instead of the default serial port.

Valore: (false)

Note:
When set to true, the system captures CLI execution output into a memory buffer for later retrieval or processing. When false, output is streamed directly to the configured UART/USART peripheral. This flag requires sufficient heap or static memory allocation if enabled, as it depends on the buffer size defined in the CLI configuration module.

Uso:
Set this to true if you need to programmatically inspect CLI responses within your application code or if you are implementing a remote logging feature. Keep it false for standard interactive debugging via terminal to save RAM. Warning: enabling this without sufficient buffer space can lead to data truncation or memory corruption.

---

ca_cli_exec_capture_stream

File: caos/inc/cli.h

Determines if the command line interface (CLI) output should be captured into a buffer instead of being sent directly to the serial port.

Valore: (false)

Note:
When set to true, the system redirects CLI responses to an internal memory buffer, allowing the application to process or store the output programmatically. When false, output is streamed directly to the UART/USB peripheral. Enabling this requires sufficient heap or static memory

allocation to hold the captured stream; otherwise, the system may experience buffer overflows or memory corruption.

Uso:
Change this to true if you need to implement features like logging CLI commands to an SD card, sending command results over a network socket, or parsing output within the application logic. Keep it false for standard interactive debugging to save RAM and avoid unnecessary memory management overhead.

`ca_cli_filtered_count`

File: `caos/inc/cli.h`

Defines the number of command history entries to be filtered out or ignored by the CLI interface.

Valore: `(0U)`

Note:
When set to 0, no commands are filtered from the history buffer. Increasing this value reduces the memory footprint of the CLI history by limiting the number of stored commands. This constant is dependent on the CLI_HISTORY_ENABLED flag; if history is disabled, this value has no effect. Valid range is 0 to the maximum defined CLI_HISTORY_SIZE.

Uso:
Change this value if you need to optimize RAM usage on memory-constrained STM32 devices or if you want to prevent specific sensitive commands from appearing in the CLI history. Setting this too high may result in an empty history buffer, making it difficult for users to recall previous commands.

`ca_cli_hist_count`

File: `caos/inc/cli.h`

Defines the maximum number of command entries stored in the CLI history buffer.

Valore: `(0U)`

Note:
When set to 0, the command history feature is completely disabled, saving RAM. If set to a positive integer, the system allocates a circular buffer to store the last N commands. This value is directly proportional to the memory footprint of the CLI module. Valid range is 0 to 255, depending on available heap/static memory.

Uso:
Increase this value if you want to allow users to scroll through previous commands using arrow keys for faster debugging. Set it to 0 if your target device has extremely limited RAM or if you want to prevent command history persistence for security reasons.

`ca_cli_hist_get`

File: `caos/inc/cli.h`

Defines the default command-line interface (CLI) history buffer pointer.

Valore: `((const char *)0)`

Note:
This constant sets the initial memory address for the CLI command history storage. If set to ((const char *)0), the history feature is effectively disabled or uninitialized. Enabling it requires pointing this to a valid, pre-allocated memory buffer of size (ca_cli_hist_depth * ca_cli_max_cmd_len).

Uso:
Change this value when you need to enable command history recall (using arrow keys). Set it to a valid memory address if you want users to be able to scroll through previous commands. Ensure the buffer is large enough to prevent memory corruption; setting it to NULL is the safe default for systems with very limited RAM.

`ca_cli_typo_count`

File: `caos/inc/cli.h`

Defines the maximum number of allowed character typos or command mismatches in the CLI input buffer.

Valore: `(0U)`

Note:
When set to 0, the CLI performs strict command matching. Increasing this value allows the CLI to tolerate minor input errors or fuzzy matching logic if implemented. It depends on the CLI buffer size; setting this too high relative to the buffer may cause memory overflow or unexpected command execution. Typical values range from 0 to 5.

Uso:
Change this value if you want to make the command line interface more user-friendly by allowing slight typing errors. Set it to 0 for critical systems where only exact command matches are permitted to prevent accidental execution of unintended functions.

`ca_htobe16`

File: `caos/inc/endian.h`

Converts a 16-bit unsigned integer from host byte order to big-endian byte order.

Valore: `ca_bswap16(x)`

Note:
This macro performs a byte swap if the host architecture is little-endian; if the host is already big-endian, it typically resolves to the original value. It depends on the ca_bswap16 implementation. It is a compile-time or inline operation, meaning it adds minimal overhead to the system execution.

Uso:
Use this when preparing 16-bit data for network transmission or communication with big-endian peripherals. Always use this instead of manual bit-shifting to ensure code portability across different STM32 architectures. Failure to use this when required will result in corrupted data packets being sent to external devices.

`ca_htobe32`

File: `caos/inc/endian.h`

Converts a 32-bit unsigned integer from host byte order to big-endian byte order.

Valore: `ca_bswap32(x)`

Note:
This macro performs a byte-swap operation if the host architecture is little-endian (like ARM Cortex-M). If the host is already big-endian, it typically resolves to a no-op. It depends on the ca_bswap32 implementation and is used primarily for network protocol compliance.

Uso:
Use this when preparing 32-bit data packets for transmission over network protocols (e.g., TCP/IP) that require big-endian format. Do not use this for local memory operations or internal calculations, as it will corrupt the data on STM32 devices.

`ca_htobe64`

File: `caos/inc/endian.h`

A macro that converts a 64-bit integer from host byte order to big-endian byte order.

Valore: `ca_bswap64(x)`

Note:
This macro performs a byte-swap operation if the host architecture is little-endian (like ARM Cortex-M), or returns the value unchanged if the host is already big-endian. It depends on the ca_bswap64 implementation. It is typically used for network communication protocols where big-endian is the standard 'network byte order'.

Uso:
Use this when preparing 64-bit data packets to be sent over a network interface or stored in a big-endian file format. Do not use this if the destination device expects little-endian data, as it will corrupt the byte sequence.

`ca_htole16`

File: `caos/inc/endian.h`

Converts a 16-bit unsigned integer from host byte order to little-endian byte order.

Valore: `(x)`

Note:
If the host system is already little-endian, this macro evaluates to the input value directly. If the host is big-endian, it performs a byte swap. This is a compile-time operation if the input is a constant, or a runtime operation if the input is a variable. It has no dependencies on other RTOS flags.

Uso:
Use this macro when serializing data for network protocols, file systems, or hardware peripherals that strictly require little-endian format. Always use this instead of manual bit-shifting to ensure code portability across different CPU architectures (e.g., moving from ARM Cortex-M to other platforms).

`ca_htole32`

File: `caos/inc/endian.h`

Converts a 32-bit unsigned integer from host byte order to little-endian byte order.

Valore: `(x)`

Note:
This macro ensures data consistency when communicating with little-endian peripherals or network protocols. On STM32 (ARM Cortex-M), which is natively little-endian, this macro typically resolves to a no-op or a simple identity function. If the host architecture were ever changed to big-endian, this macro would perform a byte-swap operation. It has no dependencies on other RTOS flags.

Uso:
Use this macro when preparing 32-bit data packets for transmission over buses like SPI, I2C, or Ethernet to ensure the receiving device interprets the bytes correctly. Do not use this if the data is already in little-endian format, as it may cause unnecessary processing overhead or incorrect data transformation if the architecture is ported to a non-little-endian system.

`ca_htole64`

File: `caos/inc/endian.h`

Converts a 64-bit integer from host byte order to little-endian byte order.

Valore: `(x)`

Note:
This macro ensures data consistency across different CPU architectures. If the host is already little-endian, it typically resolves to a no-op. If the host is big-endian, it performs a byte-swap operation. It has no dependencies on other RTOS flags but is essential for network protocols and file system structures.

Uso:
Use this macro whenever you are preparing 64-bit data to be sent over a network or written to non-volatile storage (like an SD card or Flash) to ensure the data is stored in little-endian format regardless of the processor's native endianness. Failure to use this when communicating with little-endian systems will result in corrupted data values.

`ca_htonl`

File: `caos/inc/endian.h`

Converts a 32-bit unsigned integer from host byte order to network byte order (big-endian).

Valore: `ca_bswap32(x)`

Note:
This macro uses ca_bswap32 to swap the byte sequence of a 32-bit value. If the host system is already big-endian, this operation may be optimized out or result in a no-op depending on the architecture definition. It is a dependency for network stack protocols like TCP/IP where big-endian is the standard.

Uso:

Use this whenever you are preparing 32-bit data (like IP addresses or port numbers) to be sent over a network interface. Do not use this for local data processing or storage, as it will incorrectly swap bytes on little-endian STM32 microcontrollers, leading to corrupted data values.

`ca_htonl_signed`

File: `caos/inc/endian.h`

Converts a 32-bit signed integer from host byte order to network byte order (big-endian).

Valore: `ca_bswap32_signed(x)`

Note:
This macro maps directly to ca_bswap32_signed(x). It performs a byte-swap operation on the input value to ensure compatibility with network protocols that require big-endian format. It is independent of the CPU architecture's native endianness, as it forces the conversion regardless of the host state. It is typically used for 32-bit signed data types.

Uso:
Use this macro when sending signed 32-bit integers over a network interface or serial bus to ensure the receiving device interprets the bytes in the correct order. Do not use this on data that is already in network byte order, as it will corrupt the value. Ensure the input variable is exactly 32 bits wide to avoid unexpected behavior.

`ca_htonll`

File: `caos/inc/endian.h`

Converts a 64-bit unsigned integer from host byte order to network byte order (big-endian).

Valore: `ca_bswap64(x)`

Note:
This macro maps to ca_bswap64(x), which performs a byte-level swap of the 64-bit value. If the host system is already big-endian, this operation may be optimized away or result in a no-op depending on the architecture. It is a dependency for network protocols requiring standardized big-endian data transmission.

Uso:
Use this macro whenever you are preparing 64-bit data (like timestamps or large identifiers) to be sent over a network interface. Do not use this if the data is already in big-endian format, as it will corrupt the value. Ensure the input variable is explicitly 64 bits wide to avoid unexpected truncation or padding issues.

`ca_htonll_signed`

File: `caos/inc/endian.h`

Converts a 64-bit signed integer from host byte order to network byte order (big-endian).

Valore: `ca_bswap64_signed(x)`

Note:
This macro maps to ca_bswap64_signed(x). Enabling this ensures data consistency when transmitting 64-bit signed values over a network where the host architecture (e.g., ARM Cortex-M)

differs from the network standard (big-endian). It has no dependencies on other RTOS flags but requires the input to be a 64-bit signed type.

Uso:
Use this whenever you are preparing 64-bit signed data (like timestamps or large counters) to be sent over a network interface. Do not use this if the host is already big-endian, as it will incorrectly swap the bytes. Ensure the input variable is explicitly 64 bits to avoid truncation or unexpected sign extension behavior.

`ca_htons`

File: `caos/inc/endian.h`

Converts a 16-bit unsigned integer from host byte order to network byte order (big-endian).

Valore: `ca_bswap16(x)`

Note:
This macro maps to ca_bswap16(x). If the host system is already big-endian, this operation may be optimized away or result in a no-op depending on the architecture. It is a dependency for network stack protocols that require standardized big-endian data transmission.

Uso:
Use this whenever you are preparing 16-bit data (like port numbers or length fields) to be sent over a network interface. Always use this instead of manual bit-shifting to ensure your code remains portable across different CPU architectures (e.g., moving from Little-Endian STM32 to a Big-Endian processor).

`ca_htons_signed`

File: `caos/inc/endian.h`

Converts a signed 16-bit integer from host byte order to network byte order (big-endian).

Valore: `ca_bswap16_signed(x)`

Note:
This macro wraps ca_bswap16_signed to ensure consistent data representation across different CPU architectures. It is typically used when preparing signed 16-bit data for transmission over network protocols like TCP/IP or CAN bus. It has no dependencies on other RTOS flags but relies on the compiler's ability to perform bitwise swaps.

Uso:
Use this macro whenever you need to send signed 16-bit integers (like sensor readings or signed offsets) to an external device or network interface. Do not use this for unsigned data; use ca_htons instead. Using this on data already in network byte order will corrupt the value by swapping the bytes twice.

`ca_le16toh`

File: `caos/inc/endian.h`

A macro that converts a 16-bit little-endian value to the host system's native endianness.

Valore: `(x)`

Note:
If the host system is already little-endian, this macro typically resolves to a no-op (identity function). If the host is big-endian, it performs a byte swap. It is a compile-time operation when the input is a constant, and a runtime operation when the input is a variable. It depends on the architecture-specific definitions in the caOS port layer.

Uso:
Use this macro when reading 16-bit data from external peripherals, network packets, or file systems that are stored in little-endian format. It ensures your code remains portable across different CPU architectures (e.g., ARM Cortex-M vs. PowerPC). Do not use this on data that is already in the host's native format, as it will corrupt the value.

`ca_le32toh`

File: `caos/inc/endian.h`

A macro that converts a 32-bit unsigned integer from little-endian byte order to the host system's native byte order.

Valore: `(x)`

Note:
This macro performs a no-op on little-endian architectures (like ARM Cortex-M) but performs a byte-swap on big-endian systems. It is a compile-time or runtime operation depending on the architecture. It has no dependencies on other RTOS flags but is essential for cross-platform data compatibility.

Uso:
Use this when parsing binary data received from network packets, file systems, or external sensors that transmit data in little-endian format. Always use this instead of manual bit-shifting to ensure your code remains portable across different CPU architectures.

`ca_le64toh`

File: `caos/inc/endian.h`

A macro that converts a 64-bit unsigned integer from little-endian byte order to the host system's native byte order.

Valore: `(x)`

Note:
This macro performs a no-op on little-endian architectures (like ARM Cortex-M) and a byte-swap on big-endian architectures. It is a compile-time abstraction that ensures data portability when reading 64-bit values from network packets or external storage. It has no dependencies on other RTOS flags but requires the input to be a 64-bit unsigned integer type.

Uso:
Use this macro whenever you are parsing 64-bit data structures received from external hardware or communication protocols (e.g., Modbus, CAN, or binary files) to ensure the code remains platform-independent. Do not use this on data that is already in the host's native format, as it will corrupt the value on big-endian systems.

`ca_mpu_add_region`

File: `caos/inc/mpu.h`

Configures the Memory Protection Unit (MPU) to define a specific memory region for hardware access control.

Valore: `0`

Note:
When set to 0, the MPU region configuration is disabled or bypassed. Enabling this requires defining base addresses, sizes, and access permissions (Read/Write/Execute). It depends on the STM32 hardware MPU peripheral being initialized. Valid values are typically 0 (disabled) or 1 (enabled), though some implementations may allow multiple region indices.

Uso:
Change this value to 1 when you need to isolate specific memory areas, such as protecting kernel memory from user tasks or defining non-cacheable regions for peripherals. Warning: Incorrectly configuring MPU regions will trigger a MemManage fault, causing the system to halt or reset immediately upon accessing the restricted memory.

`ca_mpu_first_free_region`

File: `caos/inc/mpu.h`

Defines the index of the first MPU region available for user-defined memory protection tasks.

Valore: `0`

Note:
This constant sets the starting point for the MPU region allocator. When set to 0, the system assumes the first MPU slot is available for general use. Changing this value shifts the base index, which is critical if lower-indexed regions are reserved for kernel-level memory protection or peripheral access. It must be less than the total number of available MPU regions on the specific STM32 hardware.

Uso:
Adjust this value if your specific application or board support package reserves the initial MPU slots for hardware-specific memory mapping or kernel-critical regions. Incorrectly setting this value can lead to memory access violations or overwriting essential system protection settings, causing a HardFault.

`ca_mpu_get_available_regions`

File: `caos/inc/mpu.h`

Returns the number of hardware MPU regions currently available for allocation.

Valore: `0`

Note:
This function queries the Memory Protection Unit (MPU) controller to determine how many slots remain free. The return value is an integer ranging from 0 to the maximum number of regions supported by the specific STM32 hardware (typically 8 or 16). It depends on the MPU initialization state; if the MPU is disabled, it may return the total hardware capacity.

Uso:

Use this before dynamically creating memory-protected tasks or regions to ensure the hardware has enough slots to enforce the requested security policy. If this returns 0, any attempt to create a new protected region will fail or trigger a system error. Do not assume a fixed number of regions across different STM32 models, as this value varies by chip architecture.

`ca_mpu_get_busy_regions`

File: `caos/inc/mpu.h`

A configuration flag that determines whether the MPU driver retrieves and tracks currently active memory protection regions.

Valore: `0`

Note:
When set to 1, the system queries the hardware MPU registers to identify active regions; when set to 0, this feature is disabled. Enabling this requires the MPU hardware to be initialized and active. It is typically used for debugging memory access violations or dynamic task stack monitoring.

Uso:
Set this to 1 if you need to perform runtime diagnostics on memory protection boundaries or if you are implementing custom memory safety checks. Keep it at 0 for production builds to reduce CPU overhead and code size, as reading MPU registers adds latency to context switches.

`ca_mpu_get_system_regions`

File: `caos/inc/mpu.h`

A configuration flag that determines whether the system memory regions are automatically retrieved and configured by the MPU driver.

Valore: `0`

Note:
When set to 1, the kernel automatically detects and protects critical system memory areas. When set to 0, the MPU will not automatically apply these default system-wide memory protections. This flag depends on the hardware support for the Memory Protection Unit (MPU) on the specific STM32 target. Valid values are 0 (disabled) or 1 (enabled).

Uso:
Set this to 1 if you want the RTOS to handle memory safety for system tasks automatically. Set to 0 only if you are manually defining all MPU regions in your application code or if you are working on a platform where automatic detection is not supported. Warning: Disabling this (0) increases the risk of memory corruption if your application tasks are not strictly isolated.

`ca_mpu_get_total_regions`

File: `caos/inc/mpu.h`

Defines the total number of memory protection regions supported by the MPU hardware.

Valore: `0`

Note:

This value directly corresponds to the hardware capability of the STM32 MPU (e.g., 8 or 16 regions). Setting this to 0 disables MPU-based memory protection features. It is a dependency for any task-level memory isolation or stack overflow protection mechanisms.

Uso:
Change this value only if you are porting the RTOS to a different STM32 chip with a different MPU architecture. Do not set this to 0 if you intend to use memory protection, as it will cause the kernel to bypass all hardware-enforced memory access rules, potentially leading to silent memory corruption.

`ca_mpu_region_covers`

File: `caos/inc/mpu.h`

A boolean flag that determines if the MPU region configuration should automatically cover the entire memory range of a task's stack.

Valore: `false`

Note:
When set to true, the kernel automatically calculates and enforces MPU protection for the task stack boundaries. When false, the MPU must be configured manually for each task. This flag depends on the hardware MPU being enabled in the system configuration. Typical values are true for safety-critical applications and false for systems requiring custom, non-standard memory mapping.

Uso:
Set this to true to simplify memory protection management and prevent stack overflow errors. Change to false only if you are implementing a custom memory management scheme or if the default automatic coverage conflicts with specific hardware-defined memory regions. Warning: disabling this without manual MPU configuration leaves the task stack vulnerable to corruption.

`ca_mpu_update_region`

File: `caos/inc/mpu.h`

A macro interface used to update the configuration of a specific Memory Protection Unit (MPU) region.

Valore: `((void)(mode), 0)`

Note:
This function acts as a wrapper for hardware-specific MPU register updates. When enabled, it enforces memory access permissions (Read/Write/Execute) for defined memory segments. If disabled or set to a null operation, the MPU remains static, potentially leaving memory unprotected. It depends on the hardware architecture supporting MPU features and must be called after any changes to memory map definitions.

Uso:
Use this when you need to dynamically change memory access rights, such as isolating a task's stack or protecting peripheral registers. Change this value if you are porting the RTOS to a new STM32 series or if you need to disable MPU enforcement for debugging purposes. Warning: Incorrect region parameters can trigger a MemManage fault, causing the system to hang or reset.

`ca_ntohl`

File: `caos/inc/endian.h`

Converts a 32-bit unsigned integer from network byte order (big-endian) to host byte order (little-endian).

Valore: `ca_bswap32(x)`

Note:
This macro utilizes the ca_bswap32 function to perform a byte-swap operation. It is essential for STM32 architectures, which are little-endian, when processing data received from network protocols that use big-endian format. It has no dependencies on other RTOS flags but relies on the hardware-specific implementation of ca_bswap32.

Uso:
Use this macro whenever you receive 32-bit data packets from a network interface (e.g., Ethernet or CAN bus) to ensure the byte order matches the processor's native format. Failure to use this when required will result in corrupted data values. Do not use this on data that is already in host byte order, as it will reverse the bytes and cause incorrect calculations.

`ca_ntohl_signed`

File: `caos/inc/endian.h`

Converts a 32-bit signed integer from network byte order (big-endian) to host byte order.

Valore: `ca_bswap32_signed(x)`

Note:
This macro maps directly to ca_bswap32_signed(x). It performs a byte-swap operation on the input value. It is architecture-independent, meaning it will always swap the bytes regardless of whether the host CPU is little-endian or big-endian, ensuring the data is interpreted correctly according to network standards.

Uso:
Use this when receiving 32-bit signed data packets over a network interface (e.g., TCP/IP or CAN bus) where the data is transmitted in big-endian format. Do not use this on data that is already in host byte order, as it will corrupt the value by reversing the byte sequence.

`ca_ntohll`

File: `caos/inc/endian.h`

Converts a 64-bit unsigned integer from network byte order (big-endian) to host byte order (little-endian).

Valore: `ca_bswap64(x)`

Note:
This macro utilizes the ca_bswap64 function to perform a byte-swap operation. It is typically used when receiving 64-bit data packets over a network interface where the protocol specifies big-endian format. If the host architecture is already big-endian, this macro may be optimized to a no-op by the compiler.

Uso:

Use this macro whenever you are parsing 64-bit values (like timestamps or large identifiers) received from external network protocols. Failure to use this on little-endian systems (like most STM32 ARM Cortex-M cores) will result in corrupted data values due to byte-order mismatch.

`ca_ntohll_signed`

File: `caos/inc/endian.h`

Converts a 64-bit signed integer from network byte order (big-endian) to host byte order (little-endian).

Valore: `ca_bswap64_signed(x)`

Note:
This macro wraps ca_bswap64_signed to perform a byte-order swap. It is essential for cross-platform communication where the network protocol uses big-endian format. It has no dependencies on other RTOS flags but assumes the host architecture is little-endian, which is standard for STM32 (ARM Cortex-M).

Uso:
Use this when receiving 64-bit signed data packets from a network interface or external peripheral. Failure to use this on a little-endian system will result in corrupted numerical values. Ensure the input variable is exactly 64 bits wide to avoid unexpected behavior.

`ca_ntohs`

File: `caos/inc/endian.h`

Converts a 16-bit unsigned integer from network byte order (big-endian) to host byte order (little-endian).

Valore: `ca_bswap16(x)`

Note:
This macro maps directly to ca_bswap16(x). On STM32 (ARM Cortex-M), this typically utilizes the REV instruction for high-performance byte swapping. It is a no-op on big-endian architectures. It has no dependencies on other RTOS flags but is essential for cross-platform network communication.

Uso:
Use this whenever receiving 16-bit data packets from a network interface or external peripheral that transmits in big-endian format. Failure to use this on a little-endian STM32 will result in byte-swapped data (e.g., 0x1234 becoming 0x3412), leading to incorrect sensor readings or protocol errors.

`ca_ntohs_signed`

File: `caos/inc/endian.h`

Converts a 16-bit signed integer from network byte order (big-endian) to host byte order (little-endian).

Valore: `ca_bswap16_signed(x)`

Note:

This macro maps directly to ca_bswap16_signed(x). It performs a byte-swap operation on the input value. Since STM32 microcontrollers are little-endian, this function is essential when receiving 16-bit signed data from network protocols or external sensors that transmit in big-endian format. It has no dependencies on other RTOS flags but requires the input to be a 16-bit signed integer type.

Uso:
Use this macro whenever you receive 16-bit signed data (e.g., sensor readings, protocol headers) over a network interface or SPI/I2C bus that uses big-endian byte ordering. Failing to use this will result in corrupted data values due to byte reversal. Ensure the input variable is exactly 16 bits wide to avoid unexpected behavior during the swap.

`ca_stdout_create_thread`

File: `caos/inc/stdout.h`

Defines the priority level for the dedicated caOS stdout logging thread.

Valore: `0xFFFFFFFF`

Note:
This constant sets the RTOS task priority for the background thread responsible for processing stdout output. A value of 0xFFFFFFFF typically indicates that the thread is disabled or not created. Valid ranges depend on the caOS priority configuration, usually 0 (lowest) to 31 (highest). If enabled, this thread must have a priority that does not starve critical system tasks.

Uso:
Change this value if you need to enable asynchronous logging to stdout. Set it to a valid priority level (e.g., 10) to start the thread. Keep it at 0xFFFFFFFF if you want to disable the thread to save memory or if you are using a blocking, synchronous output method. Warning: setting this priority too high can cause system instability by preempting time-critical hardware control tasks.

`ca_stdout_echo`

File: `caos/inc/stdout.h`

Enables or disables the automatic echoing of characters received via the standard output interface.

Valore: `false`

Note:
When set to true, every character sent to the stdout stream is immediately echoed back to the sender. This flag is independent of the UART hardware configuration but relies on the underlying driver's transmit buffer. Typical values are boolean true or false.

Uso:
Set to true during initial development or debugging to verify serial communication and see user input in a terminal. Set to false in production environments to prevent data loops or unnecessary traffic on the communication bus, especially when the interface is connected to a machine-to-machine controller rather than a human-readable console.

---

ca_stdout_echo_dropped

File: caos/inc/stdout.h

Determines if characters dropped by the stdout buffer are echoed to the debug console.

Valore: 0

Note:
When set to 1, any character that fails to enter the stdout queue due to a full buffer is printed to the serial port with a special marker. When set to 0 (default), dropped characters are silently discarded. This flag depends on the UART driver being initialized and the stdout module being enabled.

Uso:
Enable this (set to 1) during debugging if you suspect your application is flooding the serial output and losing data. Keep it disabled (0) in production builds to prevent performance degradation and garbled console output caused by the extra error-reporting characters.

---

ca_stdout_get_repeater

File: caos/inc/stdout.h

A function pointer used to redirect or duplicate standard output (stdout) data to an external interface.

Valore: NULL

Note:
When set to NULL, stdout data is only sent to the default system console. If assigned a valid function pointer, the system will call this function every time a character is printed, allowing for logging to UART, ITM, or custom buffers. It has no dependencies on other flags but requires a function matching the signature void (*)(char) to be defined.

Uso:
Change this value when you need to debug output on a peripheral other than the default one or when you need to log system messages to an external memory device. Ensure the assigned function is non-blocking and fast; otherwise, it will significantly degrade the performance of the entire RTOS.

---

ca_stdout_printf

File: caos/inc/stdout.h

Enables or disables the standard output (printf) redirection functionality in the caOS RTOS.

Valore: 0

Note:
When set to 1, the system initializes the UART/ITM peripheral to support printf debugging. When set to 0, all printf calls are compiled out or redirected to a null handler. This flag depends on the hardware abstraction layer (HAL) being configured for the specific STM32 UART pins. Valid values are 0 (disabled) or 1 (enabled).

Uso:

Set this to 1 during development to view debug logs via a serial terminal. Set this to 0 in production builds to reduce binary size, save CPU cycles, and prevent potential blocking behavior caused by slow UART transmission.

`ca_stdout_printf_n`

File: `caos/inc/stdout.h`

Enables or disables the inclusion of the printf-style formatting library for standard output.

Valore: `0`

Note:
When set to 1, the system includes the code for formatted string output (e.g., %d, %s, %f), which significantly increases the flash memory footprint. When set to 0, the printf functionality is stripped, reducing binary size. This flag depends on the underlying hardware abstraction layer for UART or ITM output.

Uso:
Set to 0 in production builds or memory-constrained applications where formatted logging is not required. Set to 1 during development and debugging to enable human-readable logs. Warning: enabling this on very small STM32 devices may cause linker errors due to flash overflow.

`ca_stdout_put`

File: `caos/inc/stdout.h`

Enables or disables the standard output (stdout) redirection to the hardware serial interface.

Valore: `0`

Note:
When set to 1, the system routes printf-style output to the configured UART peripheral. When set to 0, all stdout calls are ignored or redirected to a null device. This flag depends on the UART driver being initialized in the board support package. Typical values are 0 (disabled) or 1 (enabled).

Uso:
Set this to 1 during development to enable debug logging via serial console. Set to 0 in production builds to reduce CPU overhead and save flash memory by removing string formatting logic. Ensure the UART baud rate is configured correctly before enabling this flag to avoid system hangs during transmission.

`ca_stdout_put_n`

File: `caos/inc/stdout.h`

Configures whether the standard output (stdout) driver supports non-blocking character transmission.

Valore: `0`

Note:
When set to 1, the system uses a non-blocking buffer mechanism for stdout; when set to 0, the system uses a blocking (synchronous) approach. Enabling this requires sufficient heap memory

for the output buffer. It depends on the underlying UART/USART hardware abstraction layer being initialized.

Uso:
Set to 1 if your application needs to print logs without stalling the execution of high-priority tasks. Set to 0 if you are debugging early boot stages where the RTOS scheduler or memory allocator is not yet fully operational, or if you want to ensure every character is physically sent before the function returns.

`ca_stdout_put_str`

File: `caos/inc/stdout.h`

Enables or disables the standard output string transmission functionality in caOS.

Valore: `0`

Note:
When set to 1, the system includes the code for the ca_stdout_put_str function, allowing strings to be sent to the configured serial interface. When set to 0, the function is disabled to save flash memory. This flag depends on the hardware abstraction layer (HAL) being configured for UART/USART output.

Uso:
Set this to 1 if your application requires logging or debugging via a serial terminal. Set it to 0 if you are deploying to production on a memory-constrained device where serial output is not required, as this will reduce the binary size.

`ca_stdout_put_str_n`

File: `caos/inc/stdout.h`

A configuration flag that enables or disables the non-blocking string output function for the system standard output.

Valore: `0`

Note:
When set to 1, the system includes the ca_stdout_put_str_n function, which attempts to write a string to the output buffer without blocking execution. When set to 0, this function is excluded from the build to save flash memory. This flag depends on the underlying UART or peripheral driver being initialized. Valid values are 0 (disabled) or 1 (enabled).

Uso:
Set this to 1 if your application requires asynchronous logging or non-blocking serial communication to prevent the CPU from stalling during slow I/O operations. Set to 0 if you are constrained by flash memory space and do not require non-blocking output. Warning: disabling this will cause compilation errors if your code calls ca_stdout_put_str_n elsewhere.

`ca_usb_dbg_print`

File: `caos/drivers/inc/usb.h`

A macro used to toggle USB-based debug logging output for the caOS system.

Valore: `do { (void)(n); } while (0)`

Note:
When enabled, this macro routes debug messages to the USB serial interface; when disabled (as shown in the default value), it resolves to a no-operation (no-op) to save CPU cycles and memory. It depends on the USB driver being initialized and the USB stack being active. Typical values are either a function call to the logging routine or the provided empty macro.

Uso:
Change this value during development to enable real-time logging of system events over USB. You should disable it (set to the no-op macro) before deploying to production to prevent performance degradation and unnecessary USB bus traffic. Ensure the USB peripheral is configured correctly before enabling this, or the system may hang while attempting to write to an uninitialized buffer.

`ca_usb_dbg_reset`

File: `caos/drivers/inc/usb.h`

A macro used to trigger a hardware reset of the USB peripheral during debugging sessions.

Valore: `do {} while (0)`

Note:
When defined as a functional routine, it forces the USB controller to re-initialize. The current 'do {} while (0)' implementation is a no-op, meaning the reset is currently disabled. It has no dependencies but should be used cautiously as it will drop any active USB enumeration or data transfer.

Uso:
Change this value when you need to force a USB bus reset without power-cycling the board. This is useful for recovering from 'USB Device Not Recognized' errors during development. Do not enable this in production code as it may cause unexpected disconnects if triggered accidentally.

`ca_usb_dbg_stat`

File: `caos/drivers/inc/usb.h`

A macro used to log or output USB driver status information for debugging purposes.

Valore: `do { (void)(out); (void)(n); } while (0)`

Note:
When enabled, this macro typically routes USB state data to a serial console or debugger. In the current definition, it is a 'no-op' (no operation) macro that discards input parameters to save memory and CPU cycles. It depends on the USB driver initialization and is typically used in conjunction with a global debug level flag.

Uso:
Change this value when you need to troubleshoot USB enumeration or communication errors. Replace the empty macro with a function call like 'printf' or a custom logging routine. Warning: enabling verbose logging in this macro can significantly slow down USB interrupt handling, potentially causing timing-sensitive USB protocol timeouts.

---

ca_usb_dev_mng_add

File: caos/usb/inc/usb_mgmt.h

Enables or disables the automatic management and enumeration of USB device endpoints within the caOS USB stack.

Valore: (false)

Note:
When set to true, the system automatically handles endpoint allocation and descriptor management. When false, the developer must manually configure and manage USB endpoints. This flag depends on the USB peripheral clock being enabled in the system configuration. Typical values are boolean true or false.

Uso:
Set this to true if you want the RTOS to handle USB device setup automatically, which is recommended for standard HID or CDC classes. Set to false only if you are implementing a custom USB protocol or require fine-grained control over endpoint memory buffers. Incorrectly setting this to false without providing manual management code will result in the USB device failing to enumerate on the host PC.

---

ca_usb_dev_mng_init

File: caos/usb/inc/usb_mgmt.h

Initializes the USB device management subsystem during the caOS boot sequence.

Valore: (false)

Note:
When set to true, the kernel allocates memory and initializes the USB peripheral driver stack. If set to false, the USB stack remains inactive, saving RAM and CPU cycles. This flag must be enabled if your application requires USB communication (e.g., CDC, HID, or MSC). It depends on the hardware abstraction layer (HAL) being correctly configured for the specific STM32 USB peripheral.

Uso:
Set this to true only if your project requires USB connectivity. If you are developing a headless system or a device that does not use USB, keep it false to reduce the memory footprint and avoid potential hardware initialization errors. Ensure that the system clock configuration is compatible with USB requirements (e.g., 48MHz clock) before enabling this.

---

ca_usb_dev_mng_remove

File: caos/usb/inc/usb_mgmt.h

A configuration flag that determines whether the USB device management task is automatically removed from the scheduler after initialization.

Valore: (false)

Note:
When set to true, the USB management task will terminate itself after the initial enumeration process to free up stack memory. When set to false (default), the task remains active in the

background to handle dynamic re-enumeration or bus events. This flag depends on the underlying RTOS task management capabilities and should be kept false if your application requires hot-plugging support.

Uso:
Set this to true only if your system has extremely limited RAM and you are certain that no USB re-enumeration or error recovery will be required after the initial boot sequence. Changing this to true in a system that requires dynamic USB connectivity will cause the device to stop responding to host requests after the initial connection.

`ca_usb_dev_mng_start`

File: `caos/usb/inc/usb_mgmt.h`

A boolean flag that determines whether the USB device management task is automatically started during system initialization.

Valore: `(false)`

Note:
When set to true, the RTOS spawns the USB management thread at boot. When false, the USB stack remains inactive until manually initialized by the application code. This flag depends on the USB hardware abstraction layer being correctly configured in the board support package.

Uso:
Set this to true if your application requires USB connectivity immediately upon startup. Set it to false if you need to perform complex hardware checks or wait for specific system conditions before initializing the USB peripheral to avoid potential bus enumeration errors or power-up spikes.

`ca_usb_dev_mng_stop`

File: `caos/usb/inc/usb_mgmt.h`

A boolean flag that determines whether the USB device management task is halted or active.

Valore: `(false)`

Note:
When set to true, the USB management thread is suspended, effectively disabling USB communication and power negotiation. When set to false (default), the USB stack runs normally. This flag depends on the USB peripheral being initialized in the system configuration. Valid values are true or false.

Uso:
Change this to true if you need to perform low-power operations or if you need to safely re-initialize the USB hardware without interference from the management task. Ensure this is set to false during normal operation, otherwise, the device will not be recognized by the host.

`ca_usb_ep_clear_stall`

File: `caos/drivers/inc/usb.h`

Clears the stall condition on a specific USB endpoint to allow data transmission to resume.

Valore: `(false)`

Note:
When set to false, the endpoint remains in a stalled state if previously triggered. Enabling this (true) resets the endpoint's data toggle and clears the stall bit in the USB peripheral registers. This function depends on the USB hardware being initialized and the endpoint being correctly configured in the descriptor table.

Uso:
Change this value to true when the host has cleared a stall condition or during error recovery sequences to re-enable communication on a halted endpoint. Do not set this to true prematurely, as it may cause data synchronization errors if the host is not expecting a reset of the data toggle sequence.

`ca_usb_ep_close`

File: `caos/drivers/inc/usb.h`

A configuration flag that determines whether the USB endpoint is automatically closed upon a transmission error or disconnection.

Valore: `(false)`

Note:
When set to true, the system will force the endpoint into a closed state during communication failures, requiring a manual re-initialization. When false, the endpoint remains open, allowing for potential retry attempts without re-configuring the hardware registers. This flag depends on the underlying USB peripheral driver state machine and is typically a boolean value.

Uso:
Set this to true if your application requires strict state management and you want to ensure the endpoint is reset to a clean state after any error. Keep it false if you prefer to handle error recovery at the application layer without re-initializing the USB hardware, which is useful for high-speed data streams where re-initialization latency is unacceptable.

`ca_usb_ep_is_stalled`

File: `caos/drivers/inc/usb.h`

A boolean flag indicating whether a specific USB endpoint is currently in a stalled state.

Valore: `(false)`

Note:
When set to true, the USB controller will respond with a STALL handshake to any host requests on this endpoint, effectively halting data transfer. This flag is typically managed by the USB stack's control transfer logic. It depends on the hardware-specific endpoint status register and should be synchronized with the USB interrupt handler.

Uso:
Change this value to true when the application needs to signal a protocol error or an unsupported request to the USB host. Set it to false to clear the stall condition after the error has been handled. Incorrectly setting this value can cause the USB host to drop the device connection or report a device malfunction.

`ca_usb_ep_open`

File: `caos/drivers/inc/usb.h`

Configures whether the USB endpoint is automatically opened and initialized during the USB stack startup sequence.

Valore: `(false)`

Note:
When set to true, the endpoint is initialized immediately upon USB driver start. When false, the endpoint remains closed until explicitly opened by the application code. This flag depends on the USB peripheral being correctly clocked and configured in the device tree. Typical values are boolean true or false.

Uso:
Change this to false if you need to perform complex hardware initialization or wait for specific application states before the USB endpoint becomes active. Setting this to false prevents premature enumeration or data reception. Warning: If set to false, ensure your application calls the corresponding open function, otherwise, USB communication will fail to initialize.

`ca_usb_ep_read`

File: `caos/drivers/inc/usb.h`

Configures the USB endpoint read mode, determining if data reception is blocking or interrupt-driven.

Valore: `(0)`

Note:
When set to 0, the system uses a polling-based read mechanism. Enabling this (setting to 1) switches to an interrupt-driven buffer management system. This flag depends on the USB peripheral clock being enabled in the system configuration. Valid values are 0 (polling) or 1 (interrupt-driven).

Uso:
Change this to 1 if your application experiences data loss during high-speed USB transfers or if the CPU needs to perform other tasks while waiting for data. Keep it at 0 for simple, low-bandwidth applications to reduce interrupt overhead. Warning: setting this to 1 requires implementing the corresponding interrupt service routine (ISR) in your application code.

`ca_usb_ep_write`

File: `caos/drivers/inc/usb.h`

Configures whether the USB endpoint write operation should be performed in blocking or non-blocking mode.

Valore: `(false)`

Note:
When set to false, the function returns immediately after queuing the data for transmission, preventing the CPU from stalling. If set to true, the function waits until the hardware buffer is ready or the transfer completes. This flag depends on the underlying USB peripheral driver state and should be kept false to maintain high system responsiveness in an RTOS environment.

Uso:
Change this to true only if you are performing synchronous data transfers where the application must guarantee the data has left the buffer before proceeding. Warning: setting this to true in a high-priority task can cause system-wide latency spikes or watchdog timeouts if the USB bus is busy or disconnected.

`ca_usb_get_speed`

File: `caos/drivers/inc/usb.h`

Defines the operational speed mode for the USB peripheral interface.

Valore: `(CA_USB_SPEED_FS)`

Note:
This constant sets the USB controller to Full-Speed (12 Mbps). Disabling or changing this value may cause the USB peripheral to fail enumeration if the hardware or the host controller does not support the selected speed. It is typically dependent on the system clock configuration and the specific USB PHY hardware present on the STM32 chip.

Uso:
Change this value only if the hardware design requires High-Speed (480 Mbps) or Low-Speed (1.5 Mbps) operation. Ensure that the system clock (HCLK) is configured to meet the timing requirements of the selected speed mode, otherwise, the USB stack will fail to initialize or experience frequent data transmission errors.

`ca_usb_init`

File: `caos/drivers/inc/usb.h`

Initializes the USB peripheral hardware and internal driver stack during system startup.

Valore: `(false)`

Note:
When set to true, the RTOS will automatically configure the USB controller, clocks, and GPIO pins during the kernel boot sequence. If set to false, the USB peripheral remains in a low-power reset state, and the developer must manually call the initialization function later in the application code. This flag depends on the hardware abstraction layer (HAL) being correctly configured for the specific STM32 target.

Uso:
Set this to true if your application requires USB connectivity (e.g., CDC, HID, or MSC) immediately upon power-up. Set to false if you need to perform complex pre-initialization tasks, want to save power by delaying USB startup, or if the USB pins are being used for other purposes during the early boot phase. Warning: disabling this without manual initialization will cause USB-related API calls to return errors or trigger hard faults.

`ms_to_ticks`

File: `caos/inc/systick.h`

A macro that converts a time duration in milliseconds into the equivalent number of RTOS system ticks.

Valore: `ca_os_tick_x_ms(ms)`

Note:
This macro relies on the system clock frequency defined by CA_OS_TICK_RATE_HZ. Enabling this allows the scheduler to calculate delays and timeouts based on real-time units rather than raw tick counts. The result is an integer value representing the number of ticks required to span the requested milliseconds. If the requested time is shorter than one tick period, the result may truncate to zero.

Uso:
Use this whenever you need to specify a timeout or delay in milliseconds for RTOS primitives like osDelay() or semaphore timeouts. Change this when you need to adjust the system tick frequency to ensure your timing logic remains consistent. Warning: Ensure that the resulting tick count does not exceed the maximum value allowed by the underlying data type (typically uint32_t) to avoid overflow errors.

`seconds_to_ticks`

File: `caos/inc/systick.h`

A conversion macro that translates a duration in seconds into the equivalent number of system ticks.

Valore: `ca_os_tick_x_ms((s) * 1000)`

Note:
This constant relies on the system tick frequency defined by ca_os_tick_x_ms. Enabling this allows developers to specify timeouts or delays in human-readable seconds rather than raw tick counts. It is mathematically dependent on the OS_TICK_RATE_HZ configuration; if the tick rate changes, this macro automatically scales correctly.

Uso:
Use this when setting task delays, software timer intervals, or timeout parameters in API calls where you prefer to define time in seconds. Change this value only when you need to adjust the granularity of your timing logic. Warning: Ensure the resulting tick count does not exceed the maximum value of the variable type used by the receiving function to prevent integer overflow.

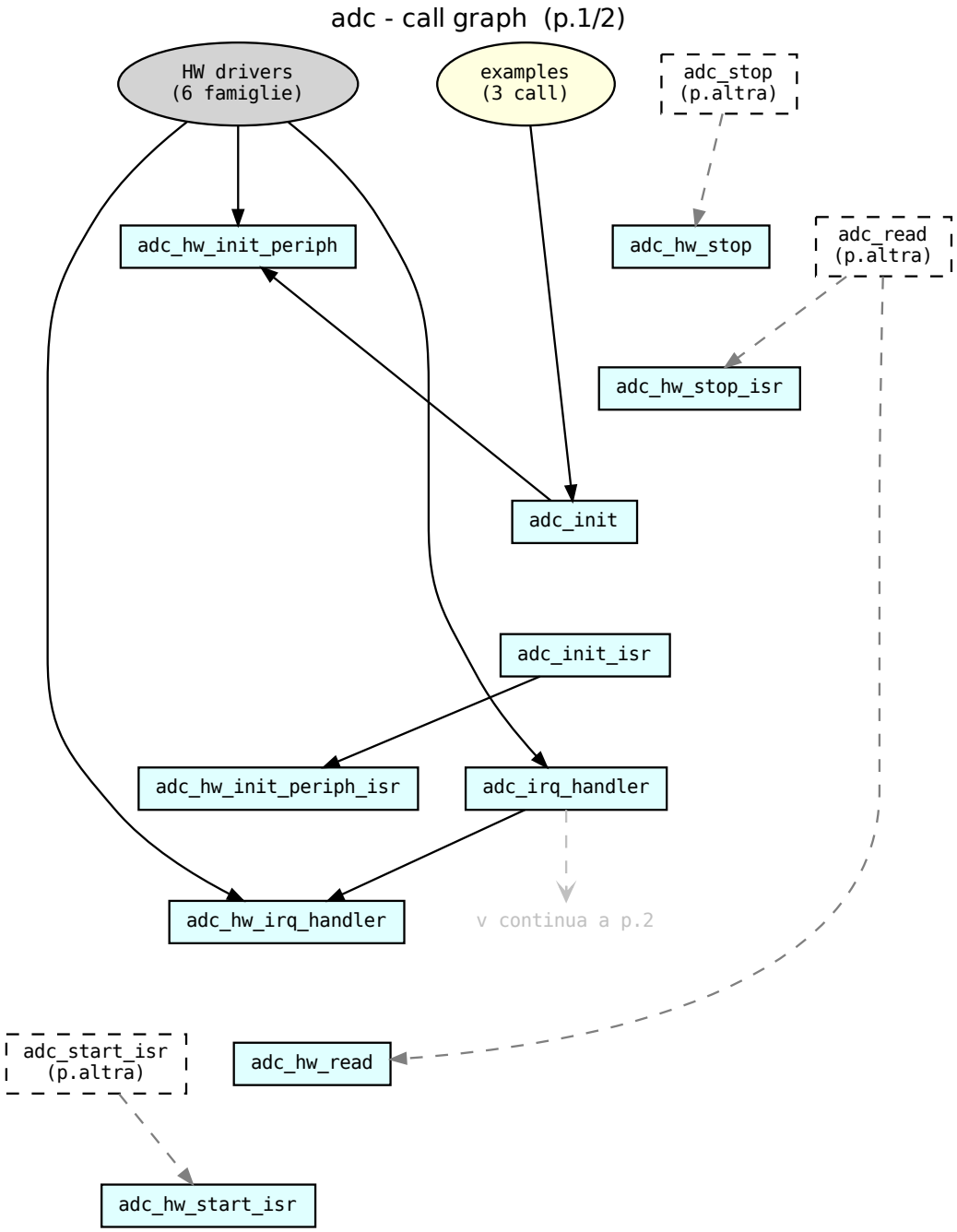
Chapter 118

Call Graph API caOS

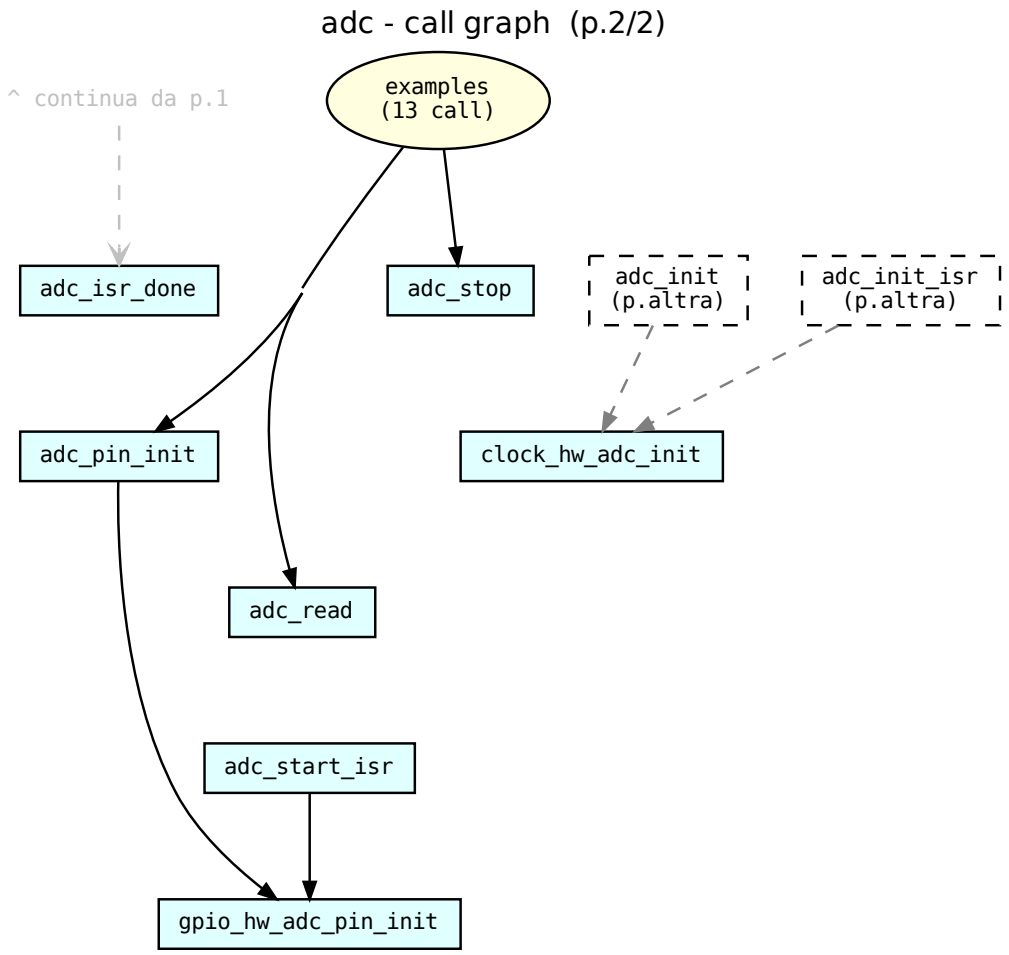
Diagrammi delle relazioni di chiamata tra le funzioni caOS, raggruppati per file sorgente che le implementa (le funzioni HW di famiglia stanno col driver generico). Ogni nodo rappresenta una funzione pubblica; le frecce indicano chi chiama chi. Nodi **gialli**: chiamanti dagli esempi. **Verdi**: applicazioni del workspace. **Rosa**: altri moduli caOS. **Grigi**: driver HW (numero di famiglie).

118.1 adc

Pagina 1/2

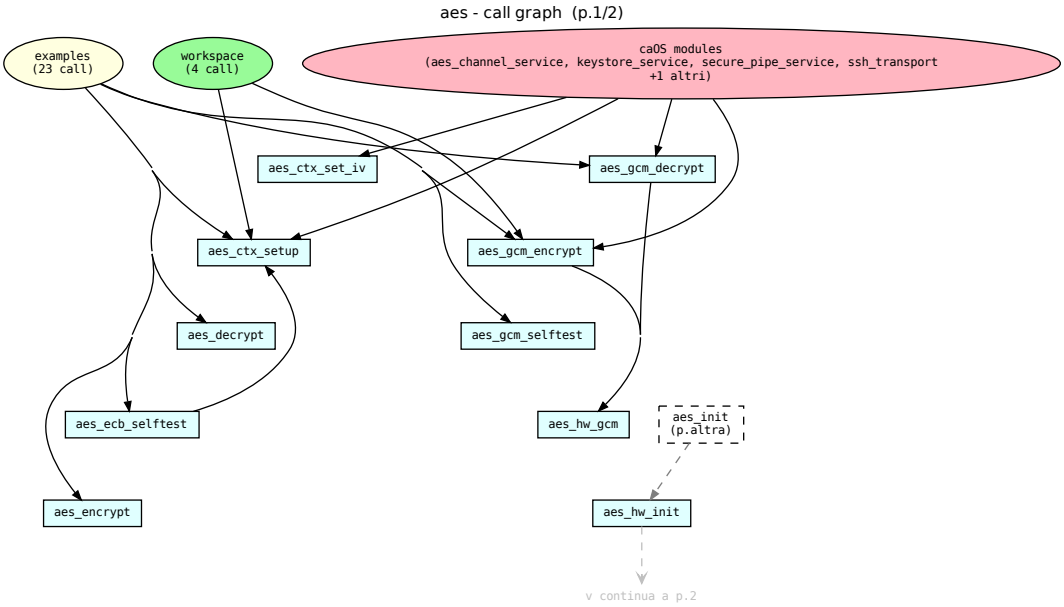


Pagina 2/2

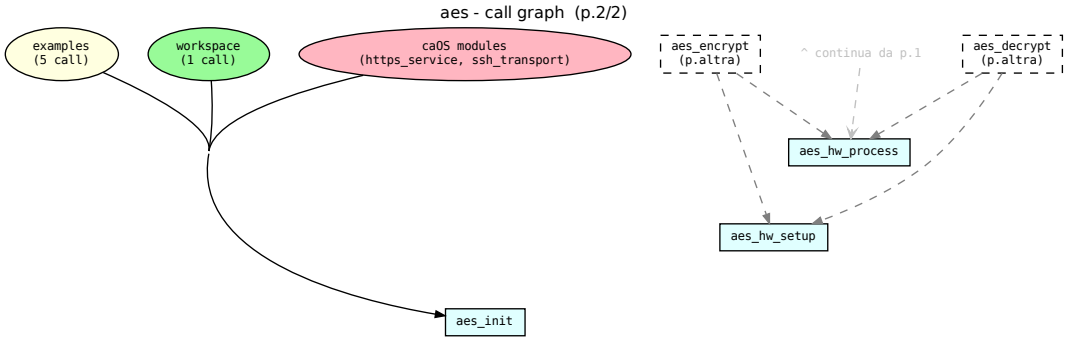

adc - call graph (p.2/2)
^ continua da p.1
examples
(13 call)
adc_isr_done
adc_stop
adc_init
(p.altra)
adc_init_isr
(p.altra)
adc_pin_init
clock_hw_adc_init
adc_read
adc_start_isr
gpio_hw_adc_pin_init

118.2 aes

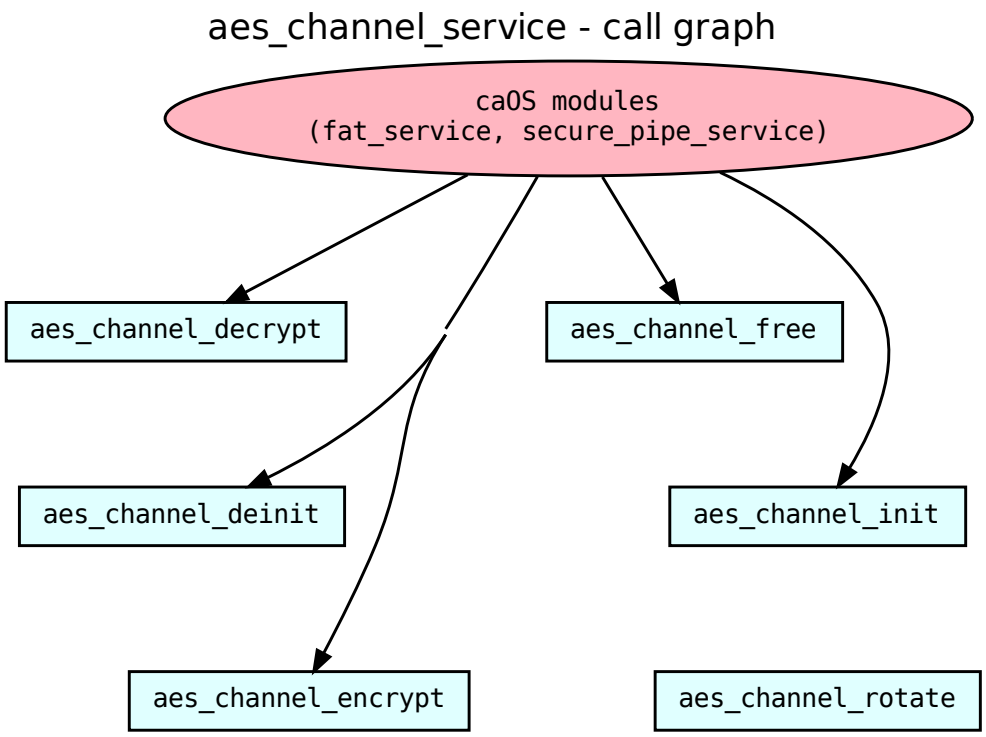
Pagina 1/2



Pagina 2/2

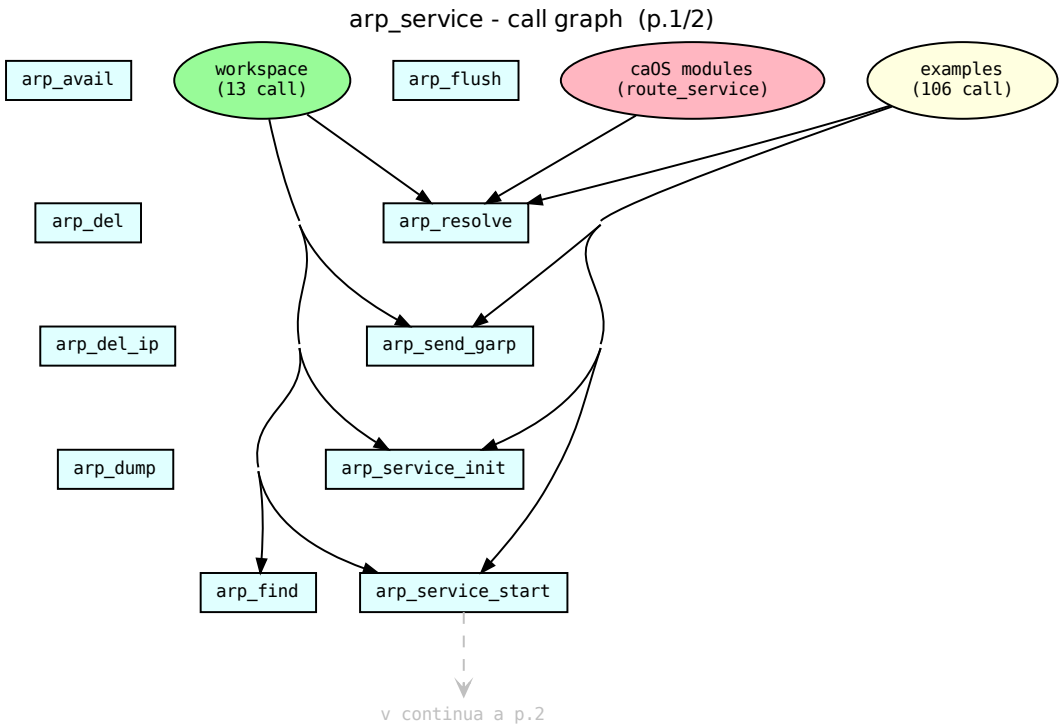

aes - call graph (p.2/2)
examples
(5 call)
workspace
(1 call)
caOS modules
(https_service, ssh_transport)
aes_encrypt
(p.altra)
^ continua da p.1
aes_decrypt
(p.altra)
aes_hw_process
aes_hw_setup
aes_init

118.3 `aes_channel_service`

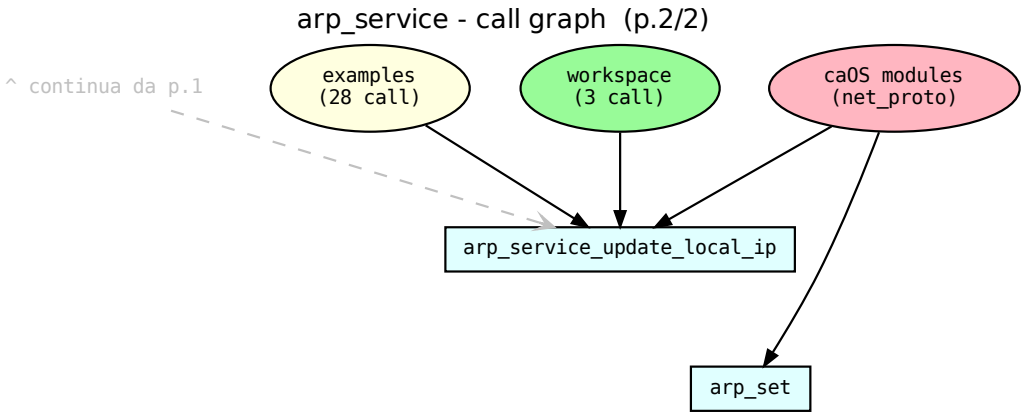


118.4 arp_service

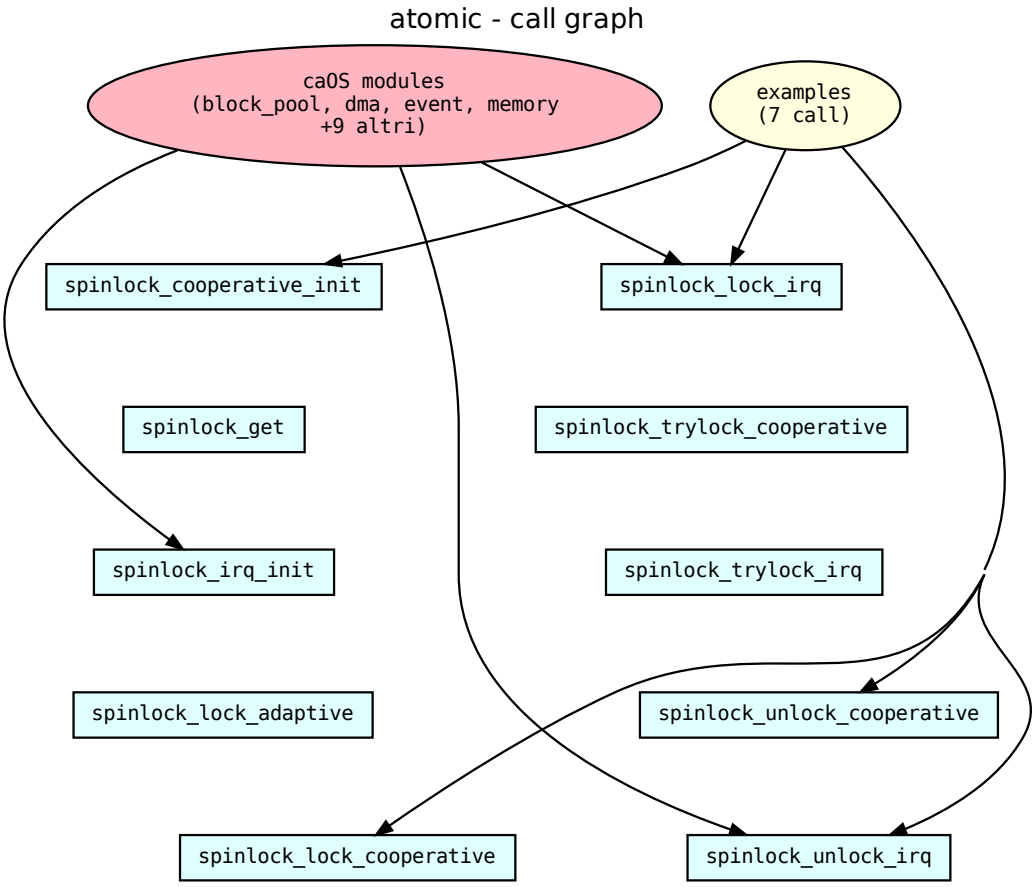
Pagina 1/2



Pagina 2/2

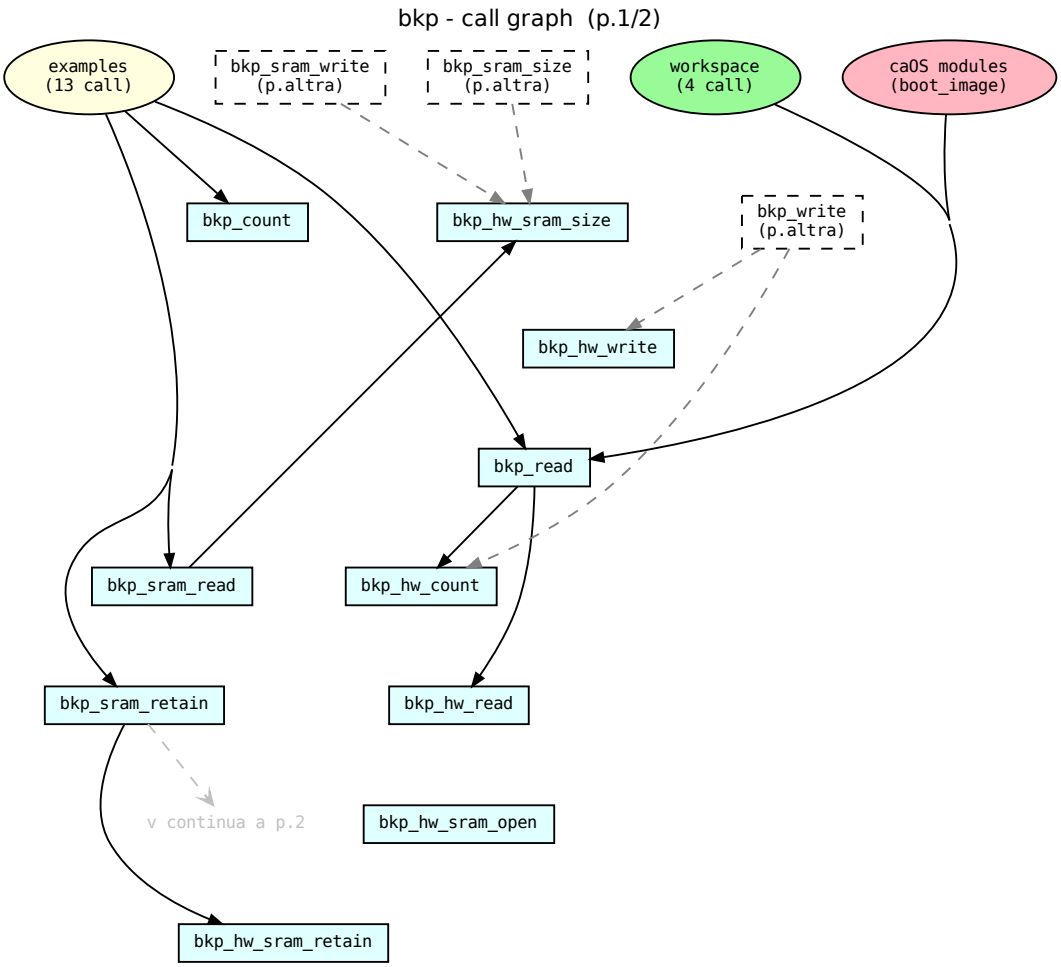

arp_service - call graph (p.2/2)
^ continua da p.1
examples
(28 call)
workspace
(3 call)
caOS modules
(net_proto)
arp_service_update_local_ip
arp_set

118.5 atomic

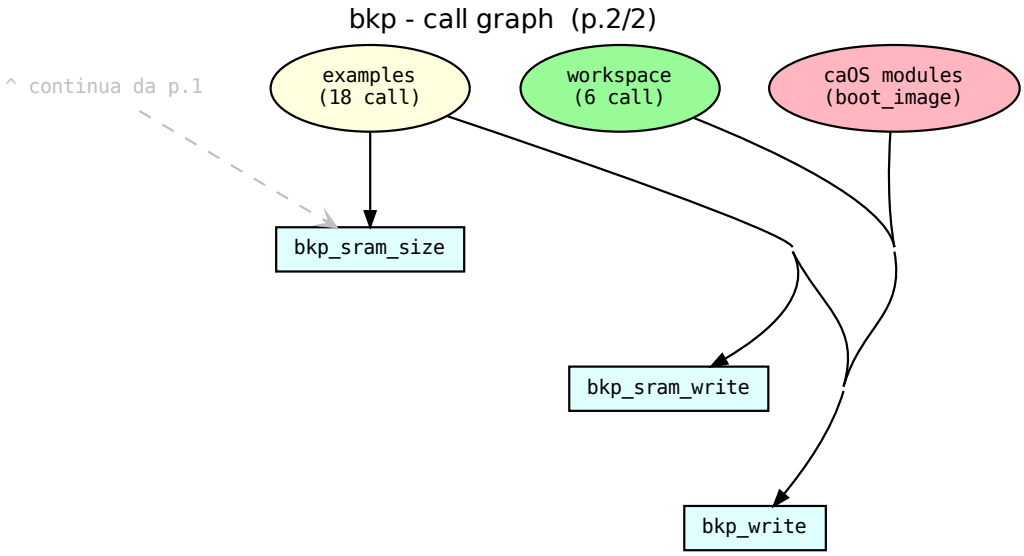

atomic - call graph
caOS modules
(block_pool, dma, event, memory
+9 altri)
examples
(7 call)
spinlock_cooperative_init
spinlock_lock_irq
spinlock_get
spinlock_trylock_cooperative
spinlock_irq_init
spinlock_trylock_irq
spinlock_lock_adaptive
spinlock_unlock_cooperative
spinlock_lock_cooperative
spinlock_unlock_irq

118.6 bkp

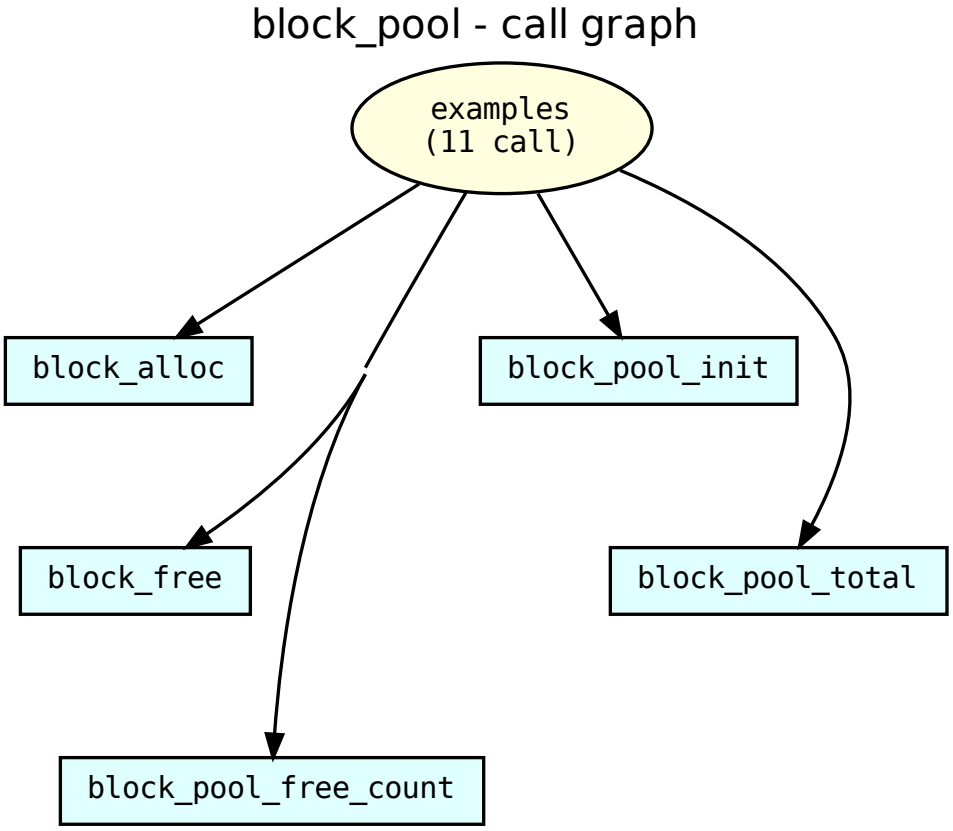
Pagina 1/2



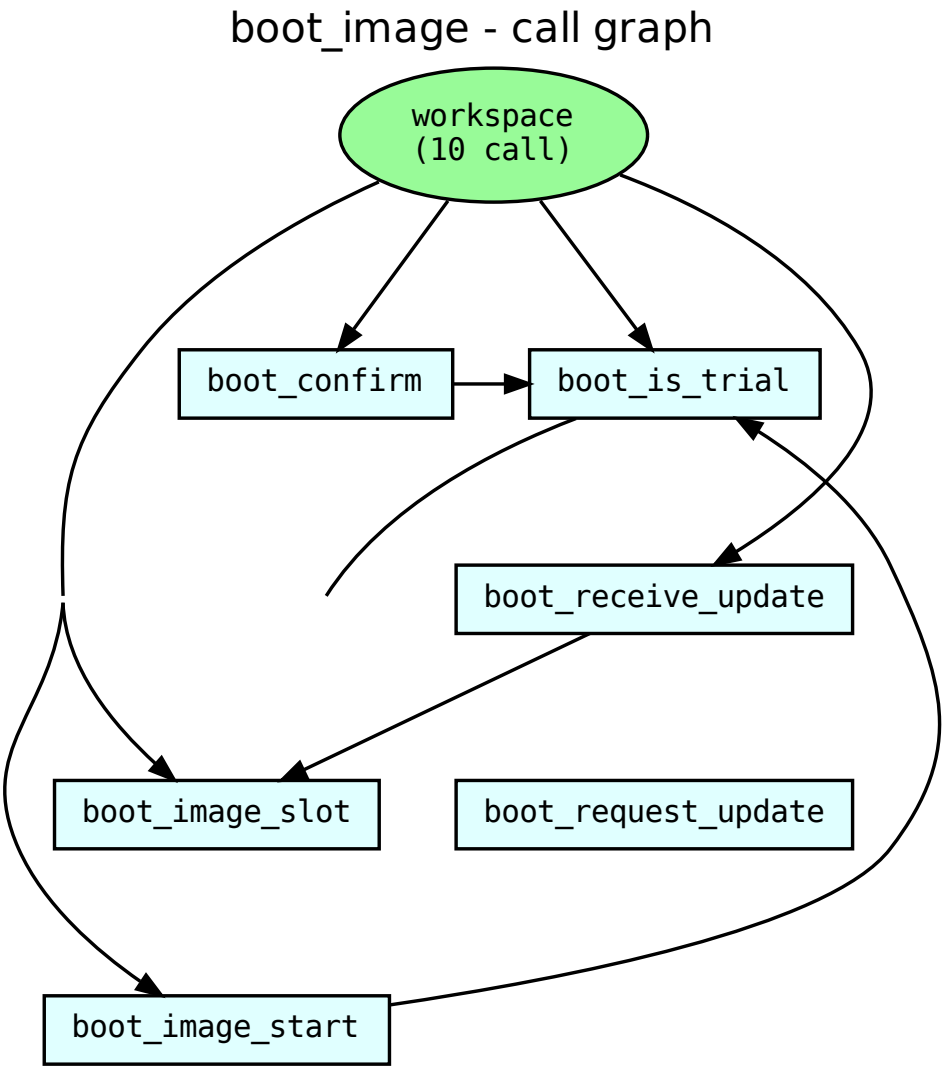
Pagina 2/2


bkp - call graph (p.2/2)
^ continua da p.1
examples
(18 call)
workspace
(6 call)
caOS modules
(boot_image)
bkp_sram_size
bkp_sram_write
bkp_write

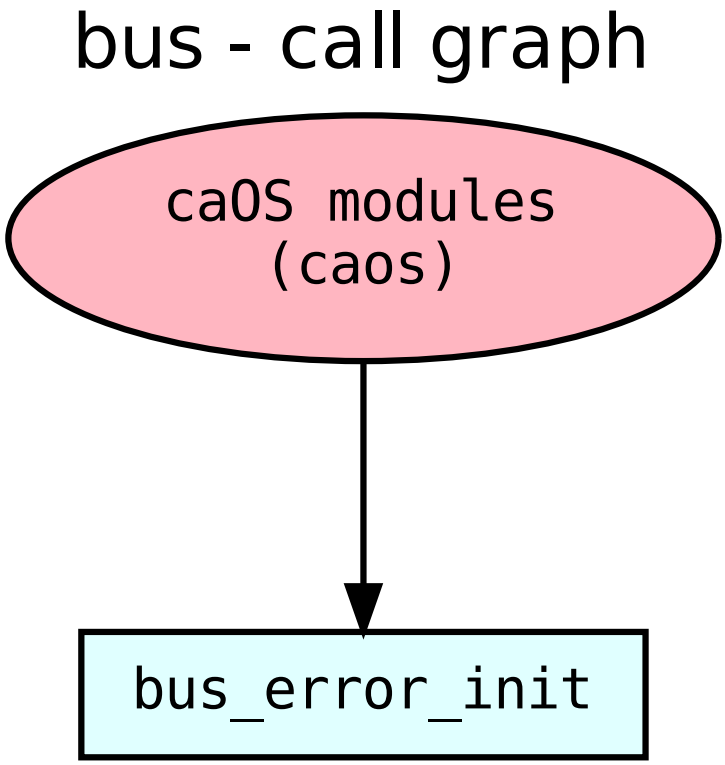
118.7 `block_pool`



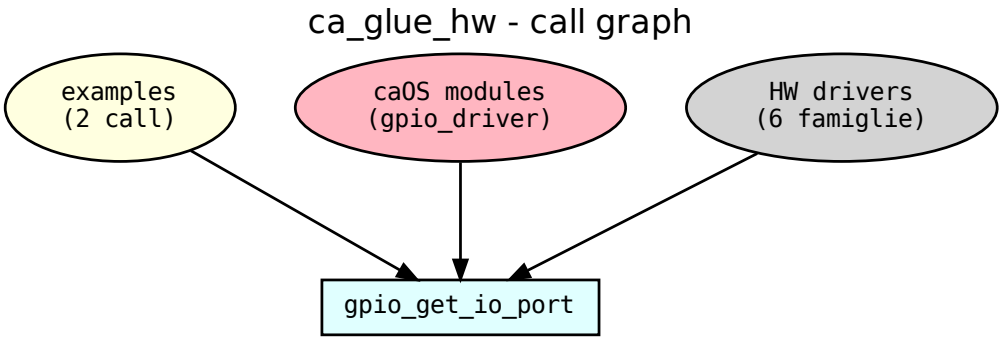
118.8 boot_image



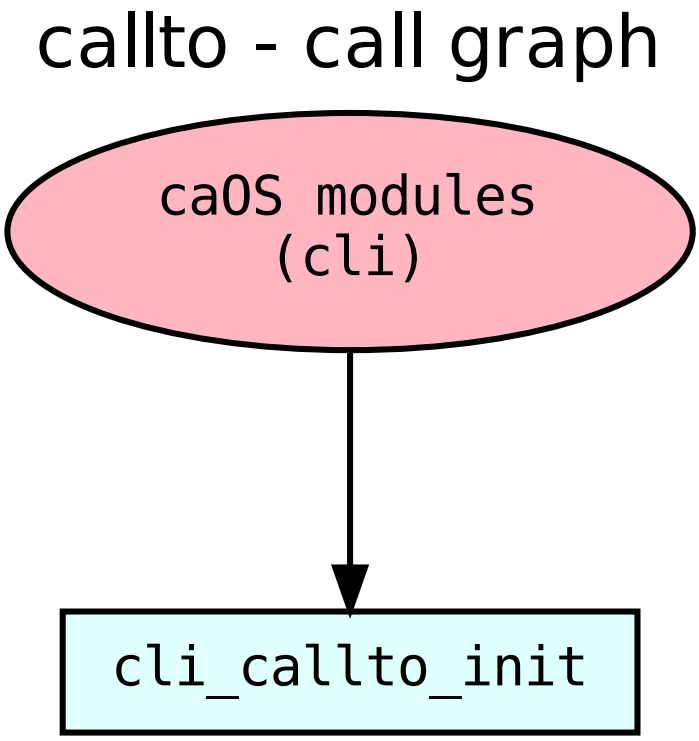
118.9 bus



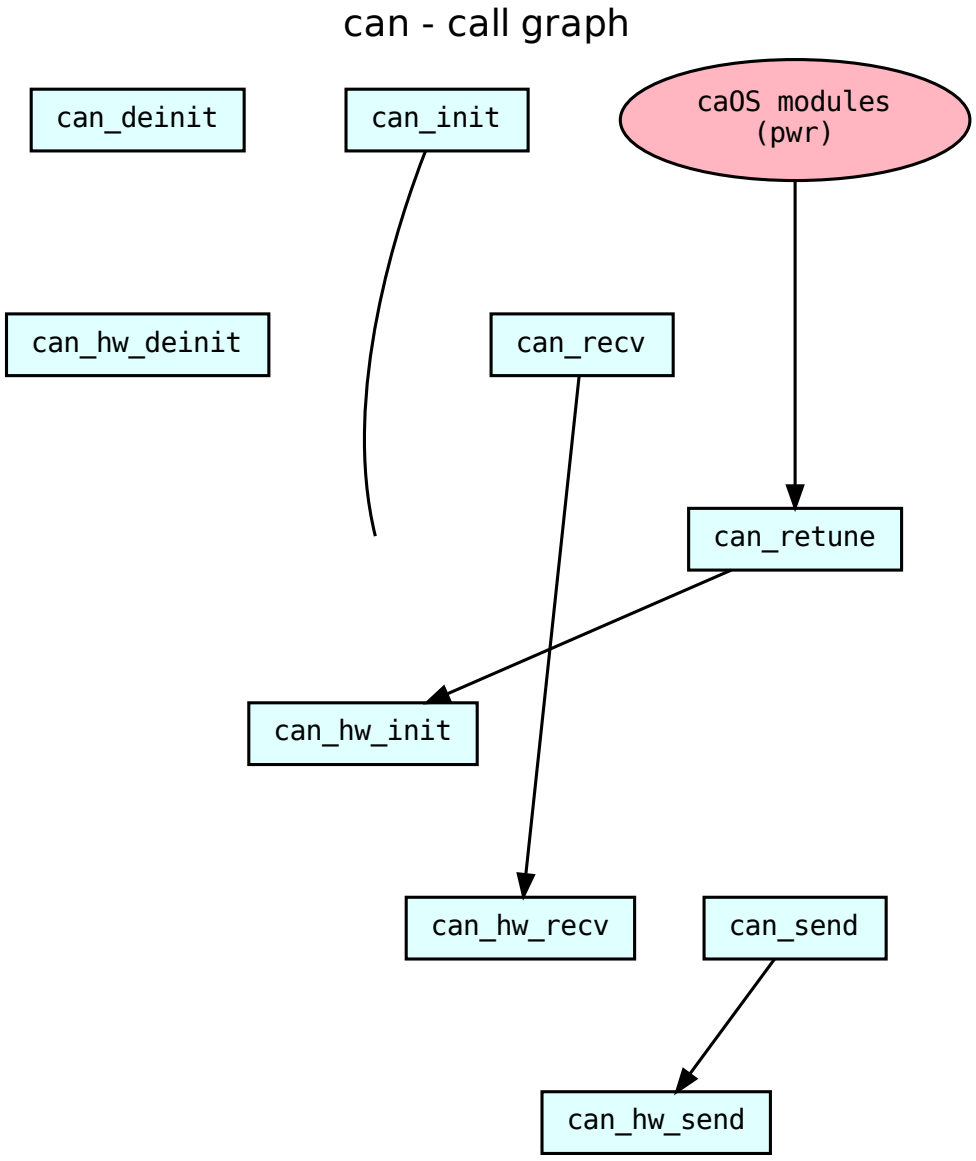
118.10 ca_glue_hw



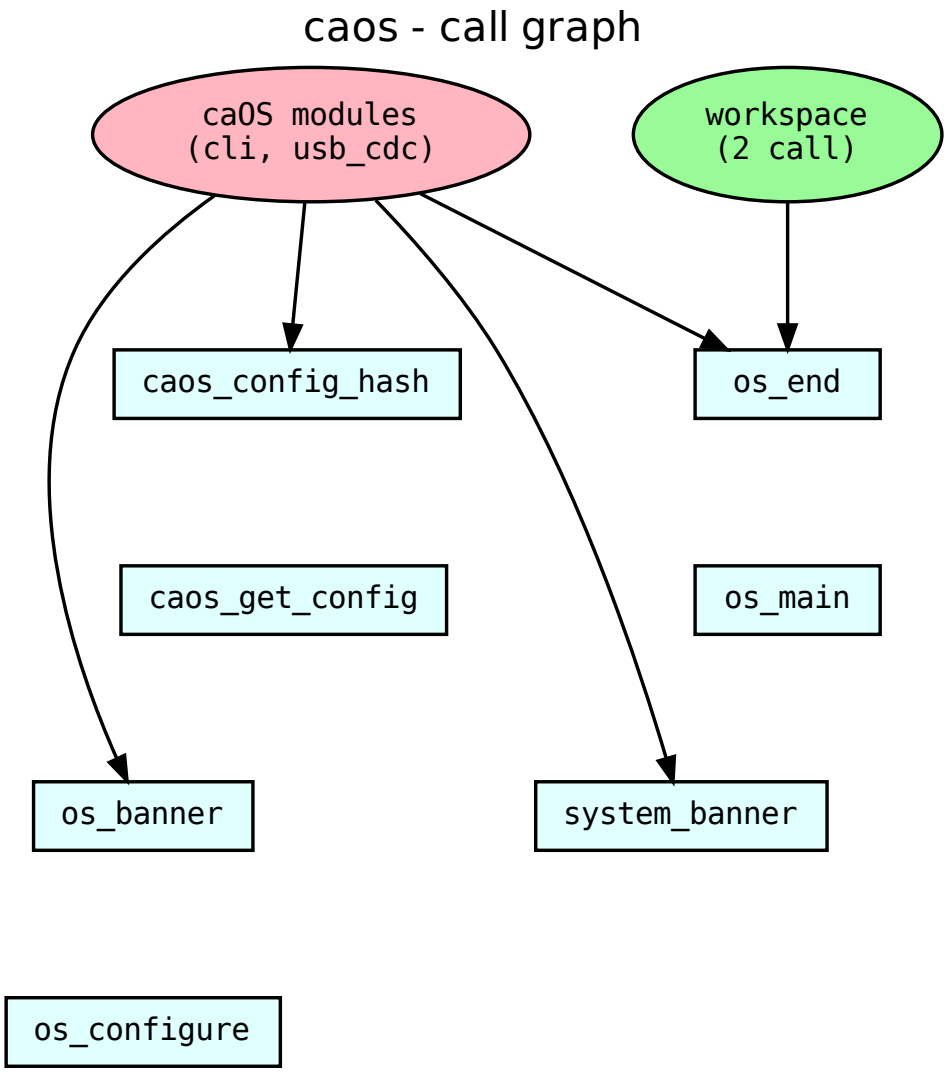
118.11 callto



118.12 can

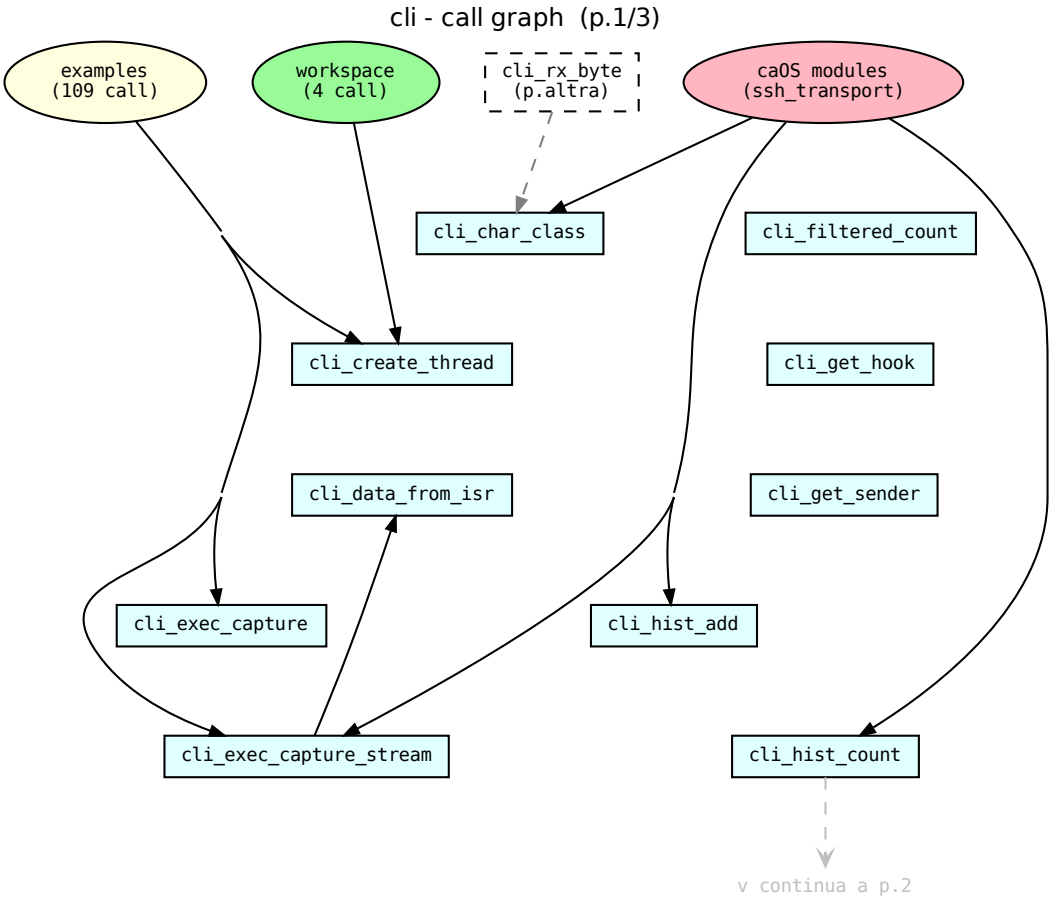

can - call graph
can_deinit
can_init
caOS modules
(pwr)
can_hw_deinit
can_recv
can_retune
can_hw_init
can_hw_recv
can_send
can_hw_send

118.13 caos

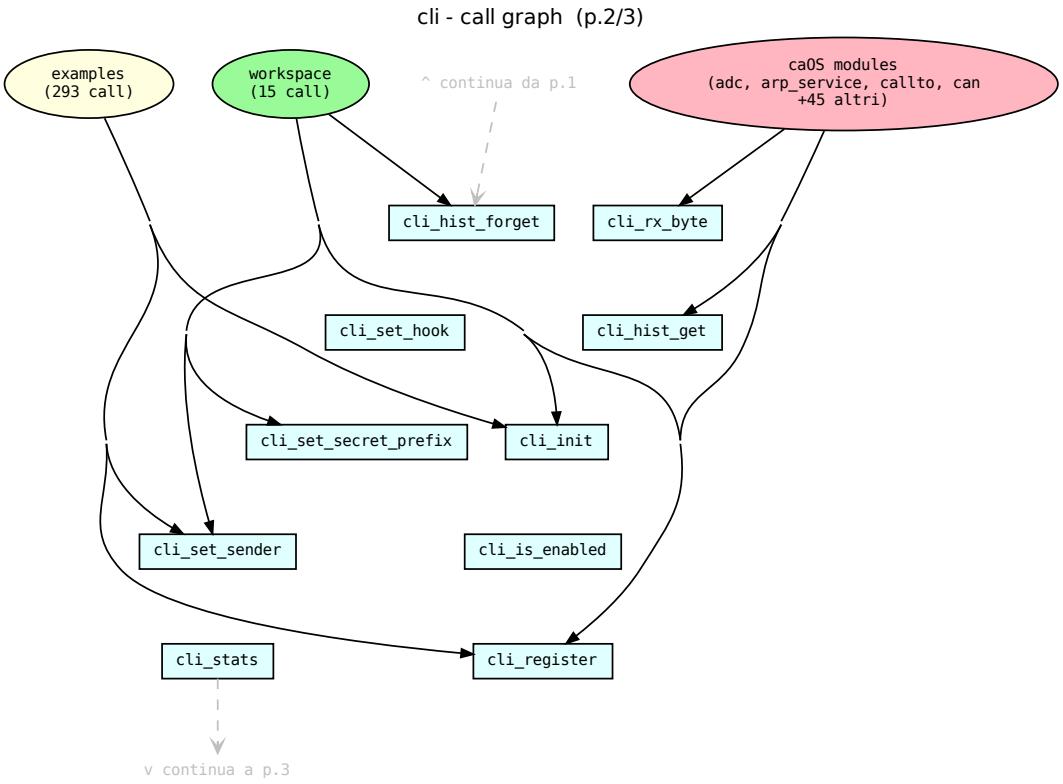

caos - call graph
caOS modules
(cli, usb_cdc)
workspace
(2 call)
caos_config_hash
os_end
caos_get_config
os_main
os_banner
system_banner
os_configure

118.14 `cli`

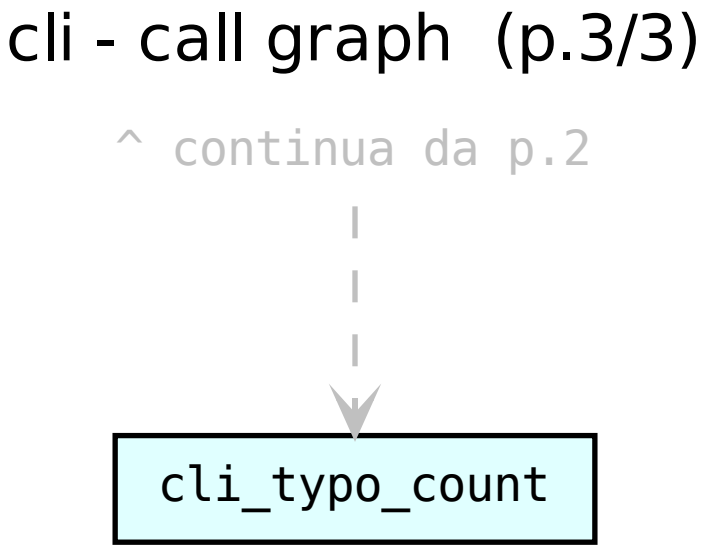
Pagina 1/3



Pagina 2/3

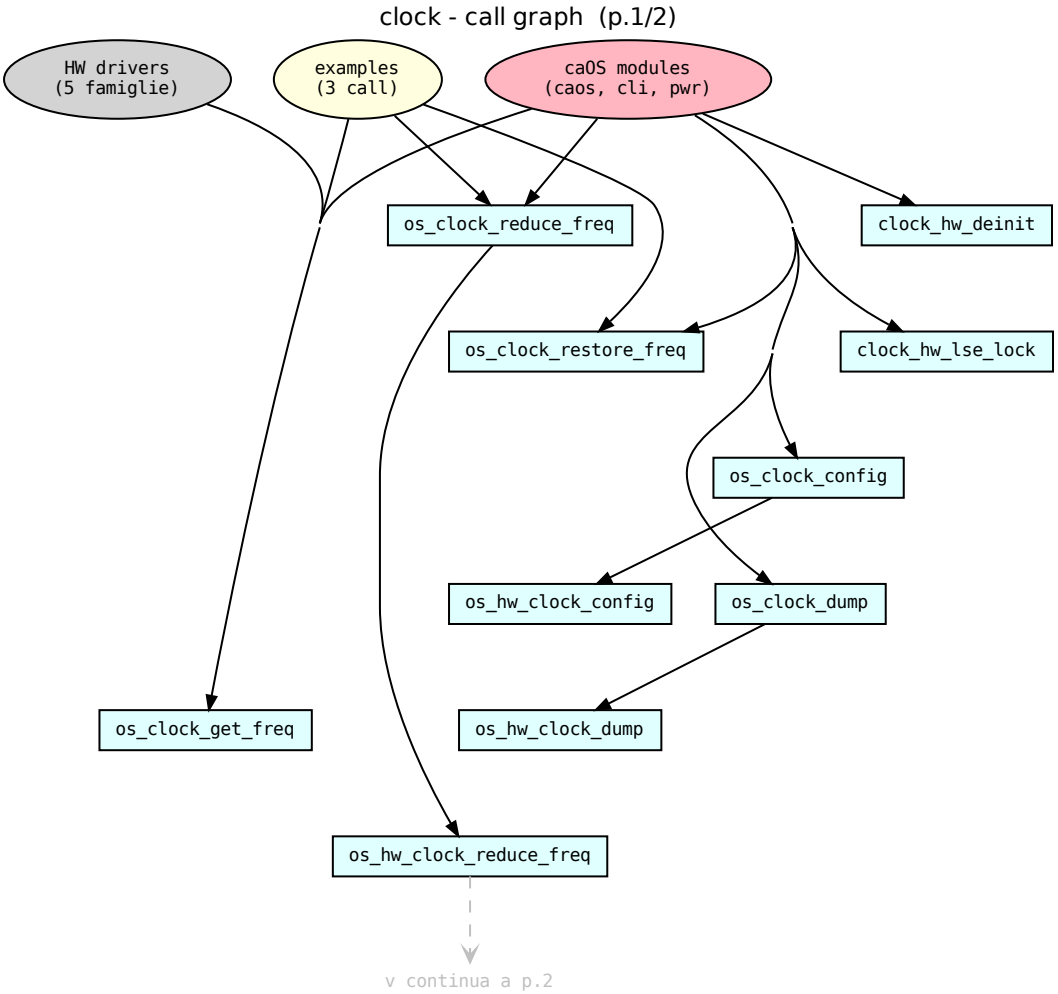

cli - call graph (p.2/3)
examples
(293 call)
workspace
(15 call)
^ continua da p.1
caOS modules
(adc, arp_service, callto, can
+45 altri)
cli_hist_forget
cli_rx_byte
cli_set_hook
cli_hist_get
cli_set_secret_prefix
cli_init
cli_set_sender
cli_is_enabled
cli_stats
cli_register
v continua a p.3

Pagina 3/3

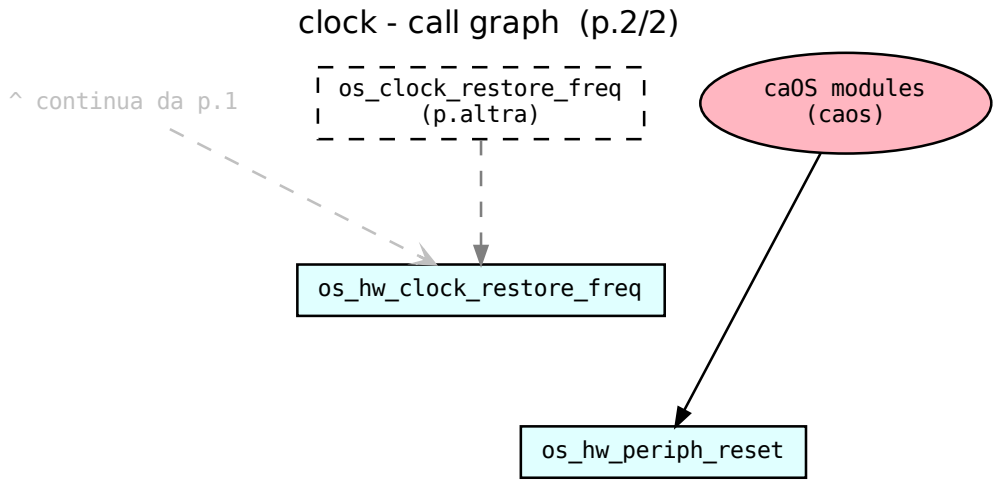


118.15 clock

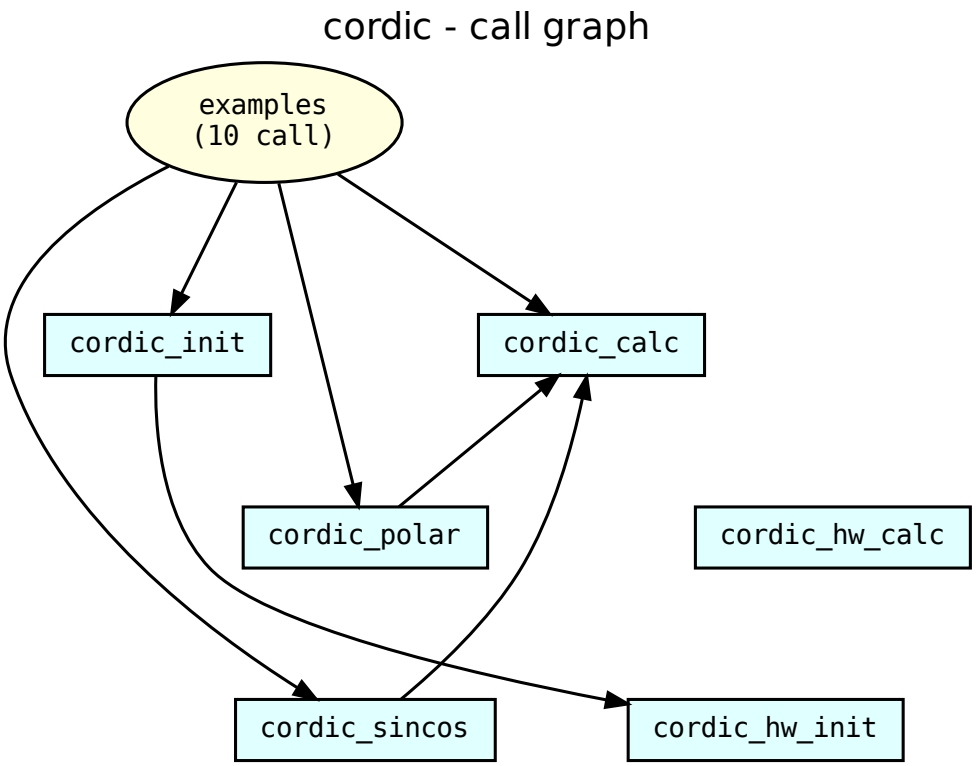
Pagina 1/2



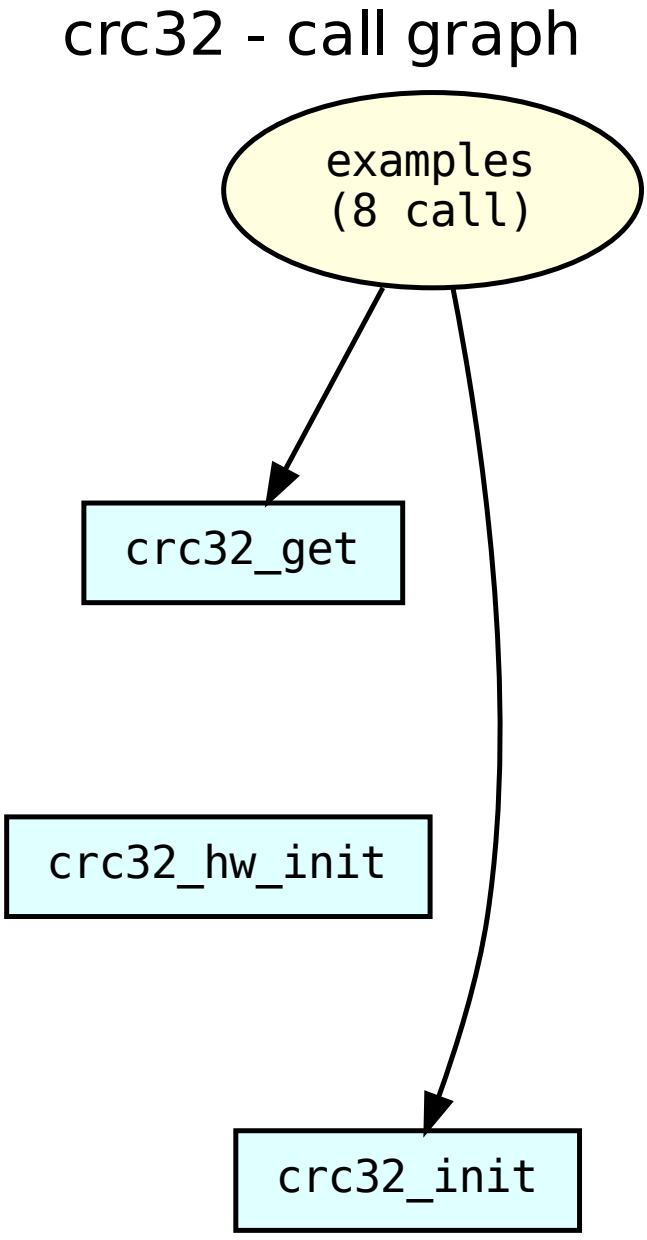
Pagina 2/2


clock - call graph (p.2/2)
^ continua da p.1
os_clock_restore_freq
(p.altra)
caOS modules
(caos)
os_hw_clock_restore_freq
os_hw_periph_reset

118.16 cordic

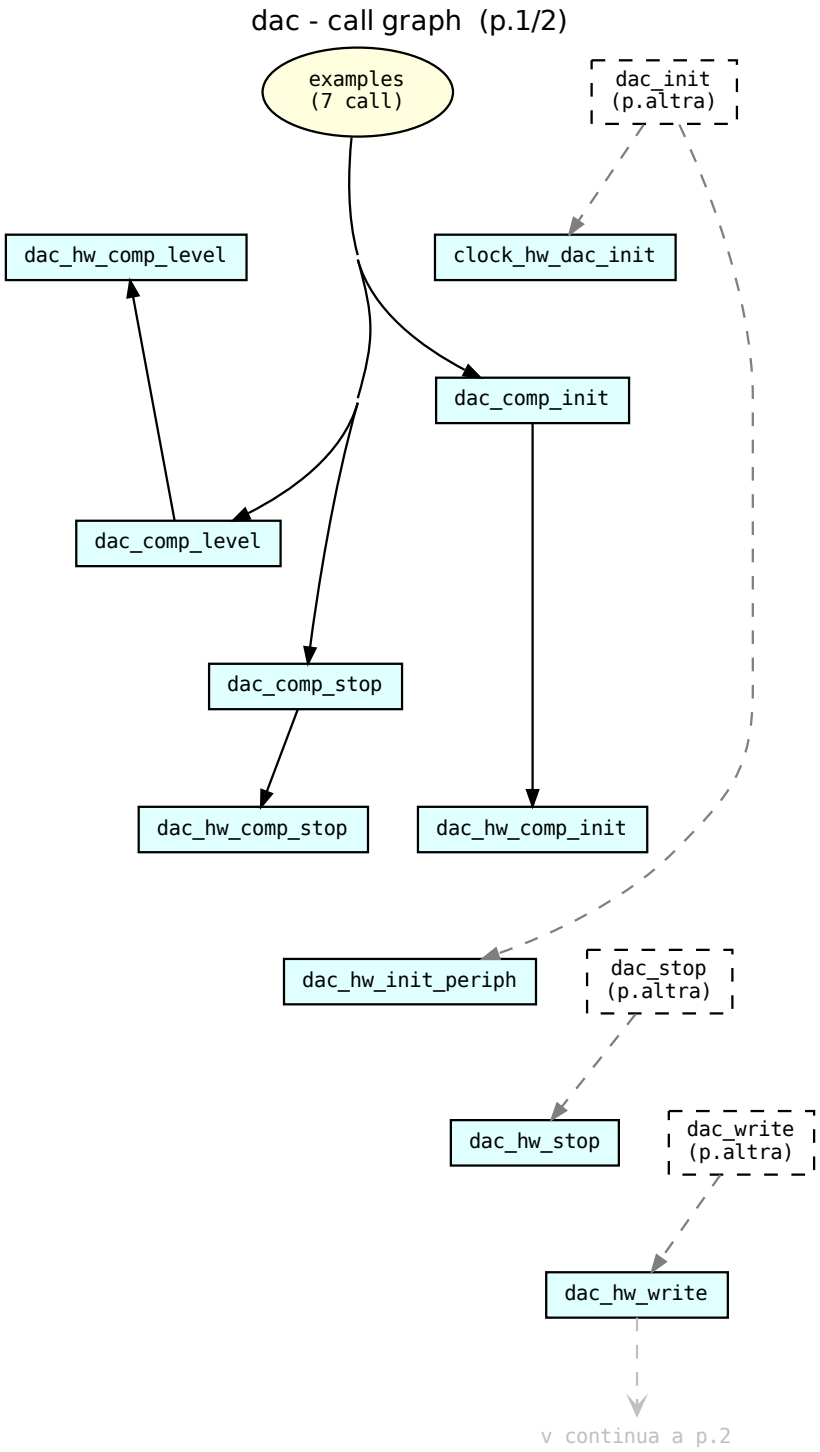


118.17 crc32

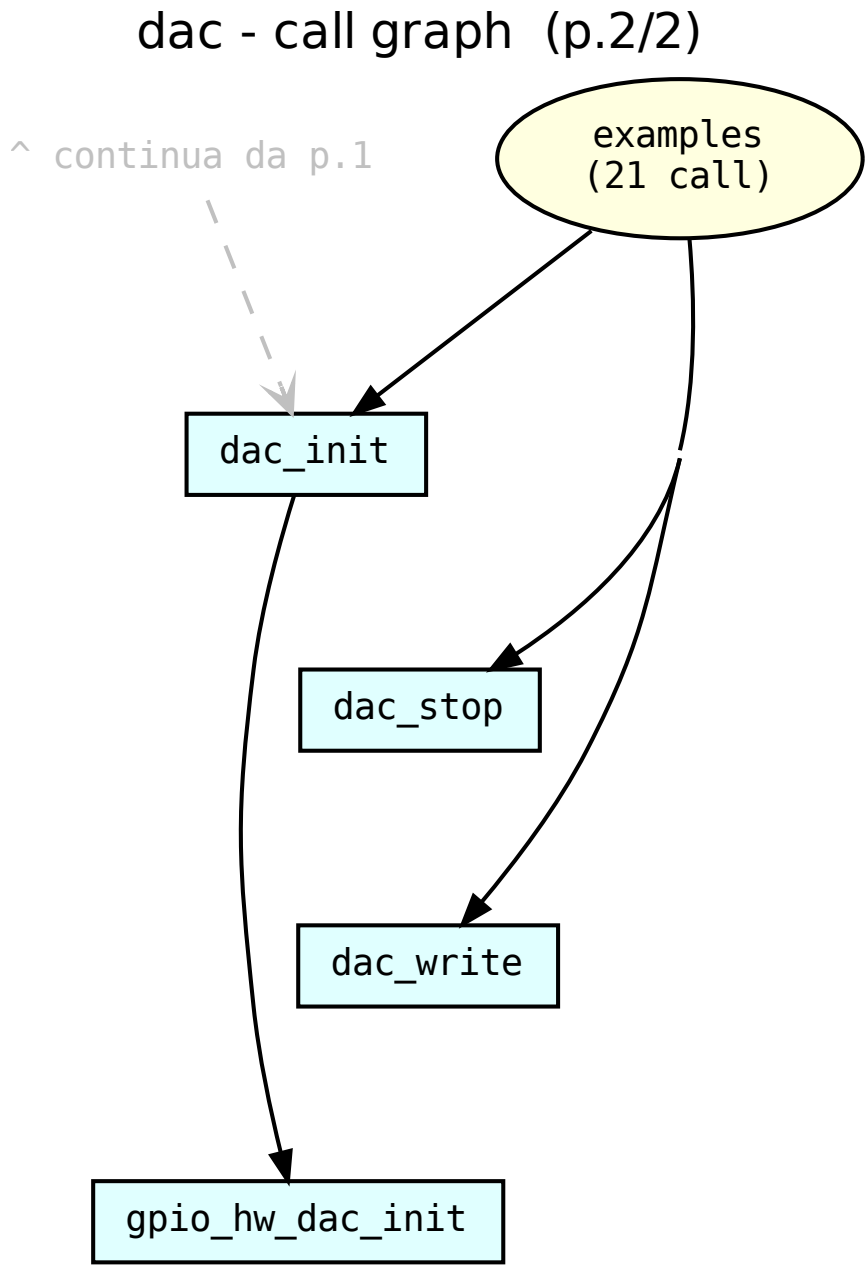

crc32 - call graph
examples
(8 call)
crc32_get
crc32_hw_init
crc32_init

118.18 dac

Pagina 1/2

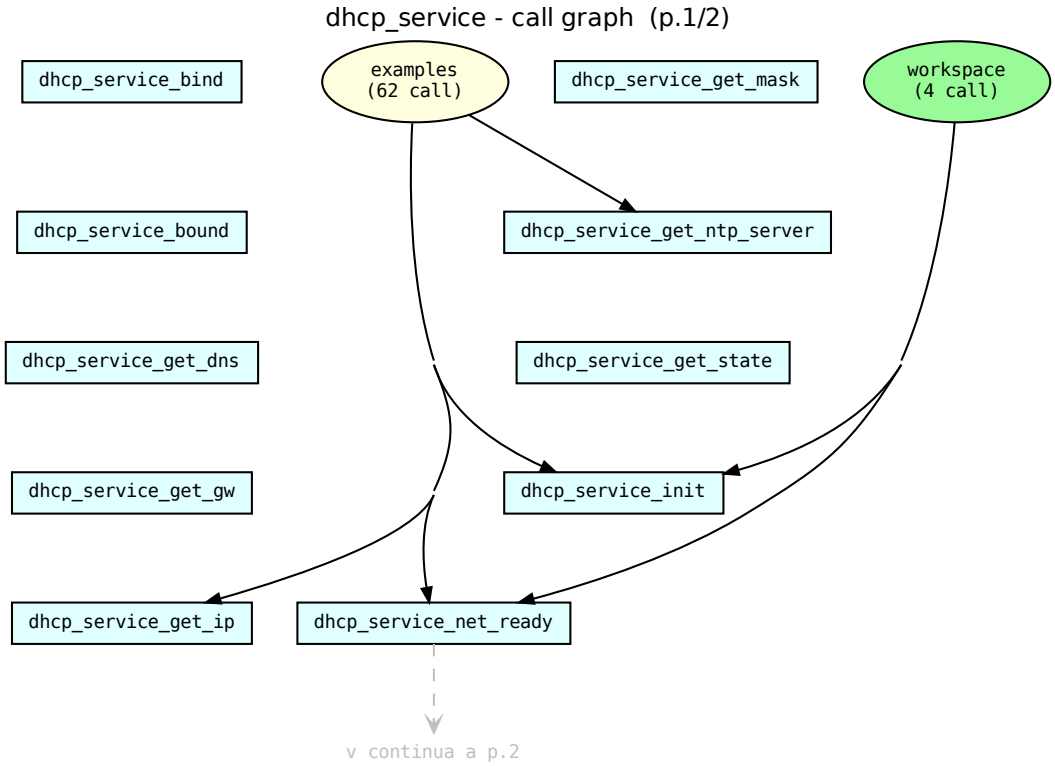


Pagina 2/2

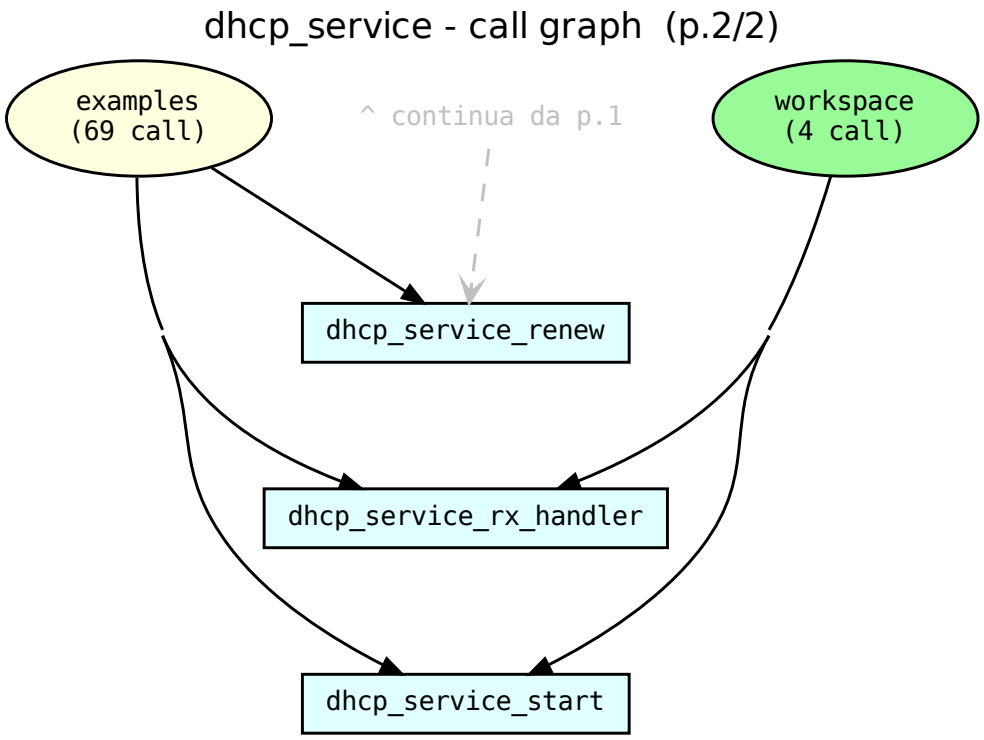

dac - call graph (p.2/2)
^ continua da p.1
examples
(21 call)
dac_init
dac_stop
dac_write
gpio_hw_dac_init

118.19 dhcp_service

Pagina 1/2

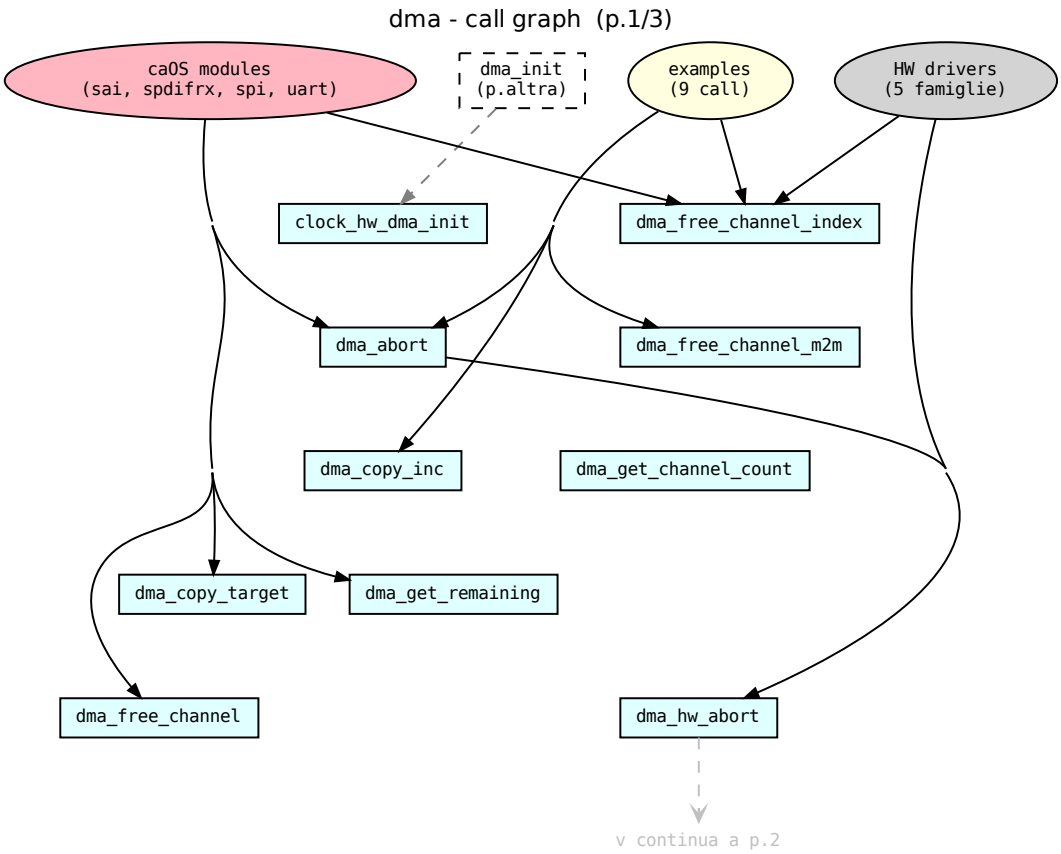


Pagina 2/2

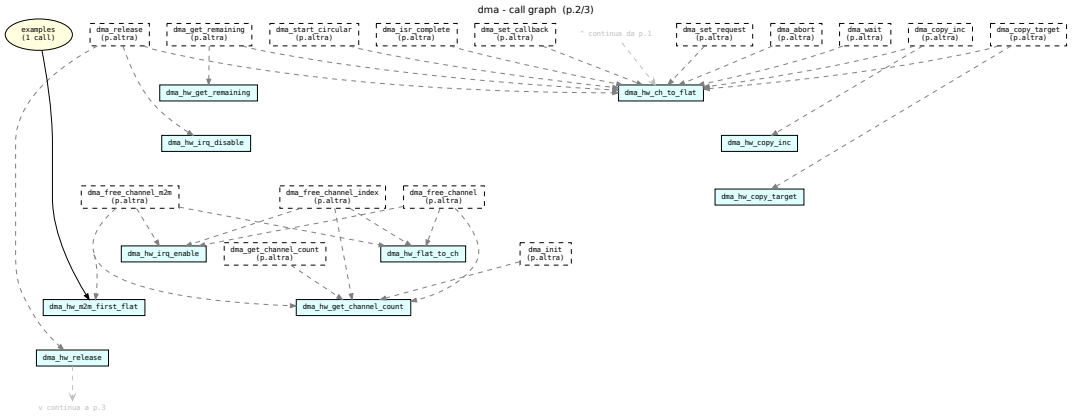

dhcp_service - call graph (p.2/2)
examples
(69 call)
^ continua da p.1
workspace
(4 call)
dhcp_service_renew
dhcp_service_rx_handler
dhcp_service_start

118.20 dma

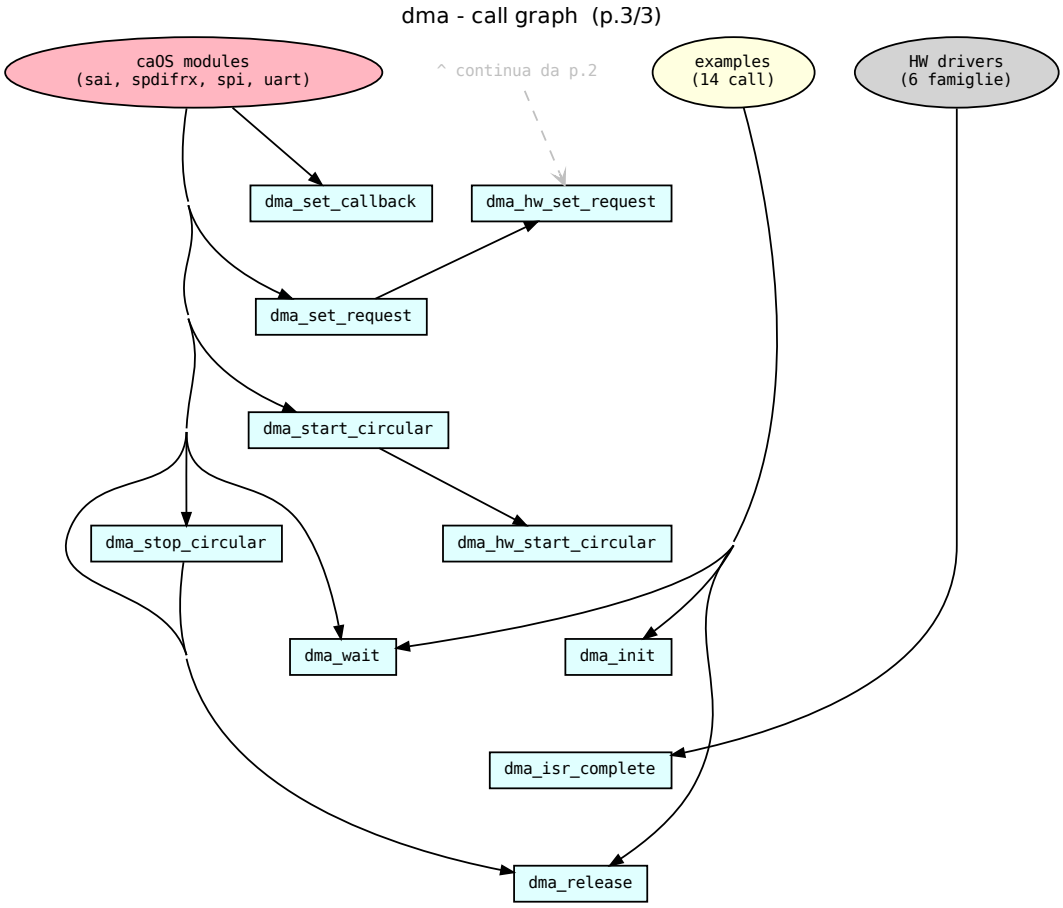
Pagina 1/3



Pagina 2/3

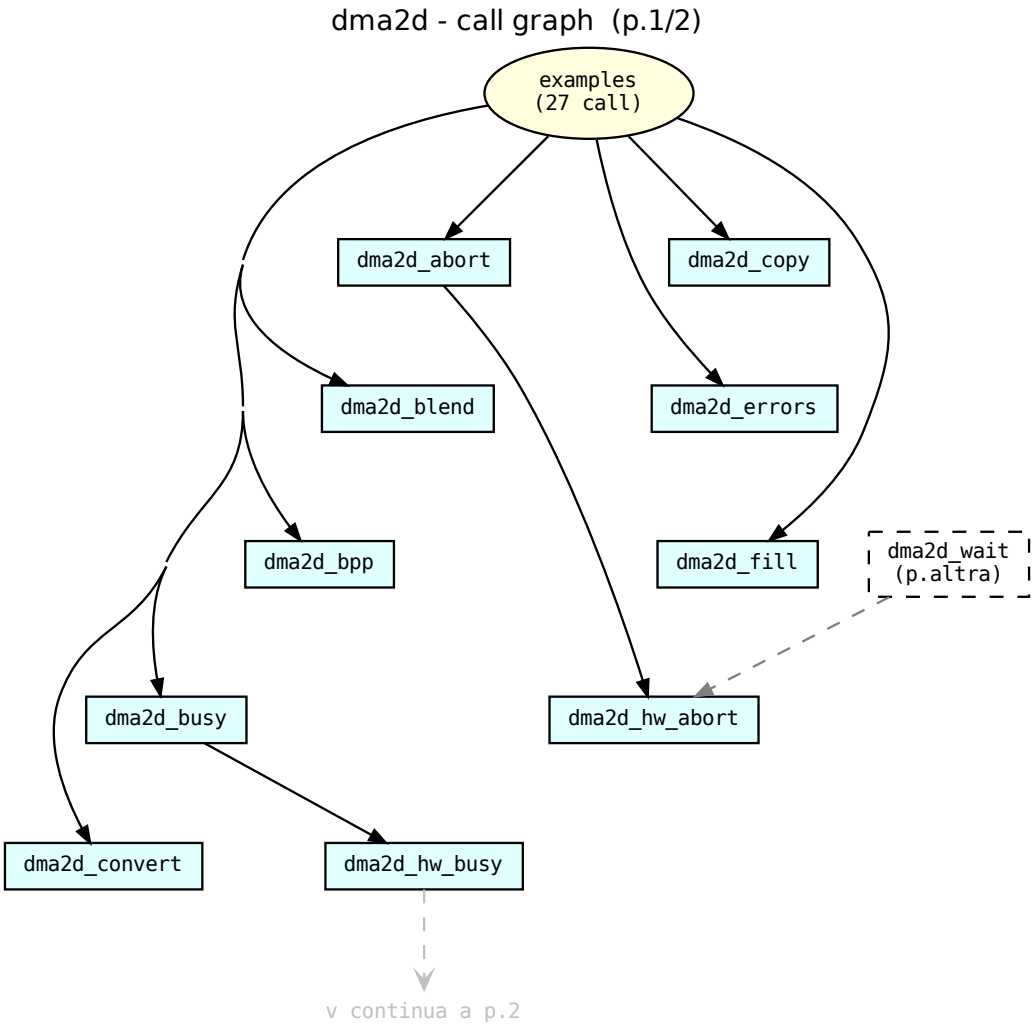

dma - call graph (p.2/3)
examples (1 call)
dma_release (p.altra)
dma_get_remaining (p.altra)
dma_start_circular (p.altra)
dma_isr_complete (p.altra)
dma_set_callback (p.altra)
^ continua da p.1
dma_set_request (p.altra)
dma_abort (p.altra)
dma_wait (p.altra)
dma_copy_inc (p.altra)
dma_copy_target (p.altra)
dma_hw_get_remaining
dma_hw_ch_to_flat
dma_hw_irq_disable
dma_hw_copy_inc
dma_hw_copy_target
dma_free_channel_m2m (p.altra)
dma_free_channel_index (p.altra)
dma_free_channel (p.altra)
dma_hw_irq_enable
dma_get_channel_count (p.altra)
dma_hw_flat_to_ch
dma_init (p.altra)
dma_hw_m2m_first_flat
dma_hw_get_channel_count
dma_hw_release
v continua a p.3

Pagina 3/3

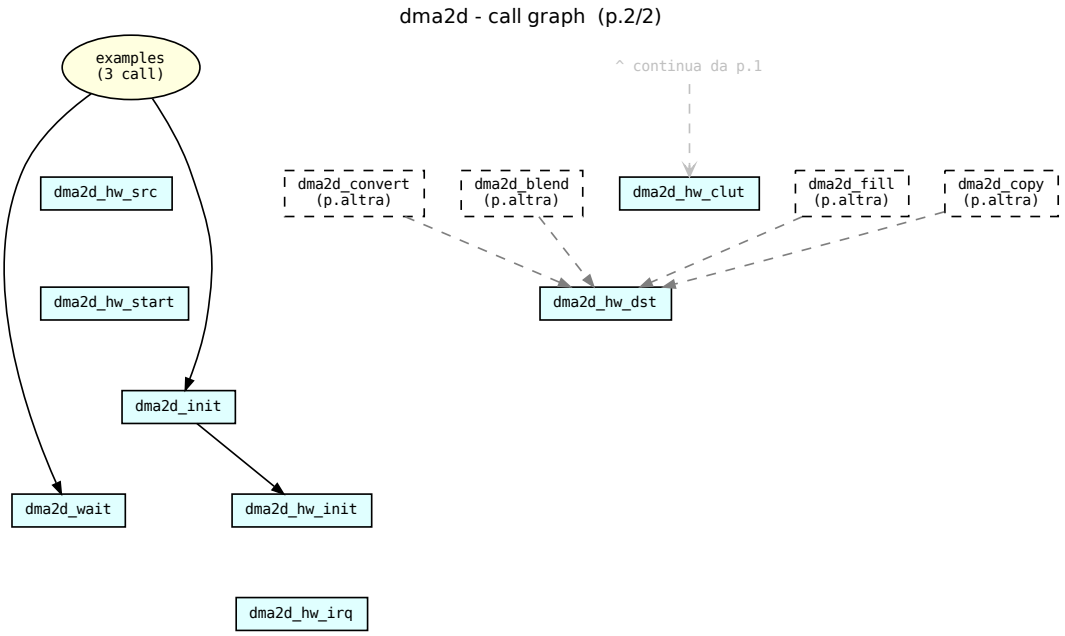

dma - call graph (p.3/3)
caOS modules
(sai, spdifrx, spi, uart)
^ continua da p.2
examples
(14 call)
HW drivers
(6 famiglie)
dma_set_callback
dma_hw_set_request
dma_set_request
dma_start_circular
dma_stop_circular
dma_hw_start_circular
dma_wait
dma_init
dma_isr_complete
dma_release

118.21 dma2d

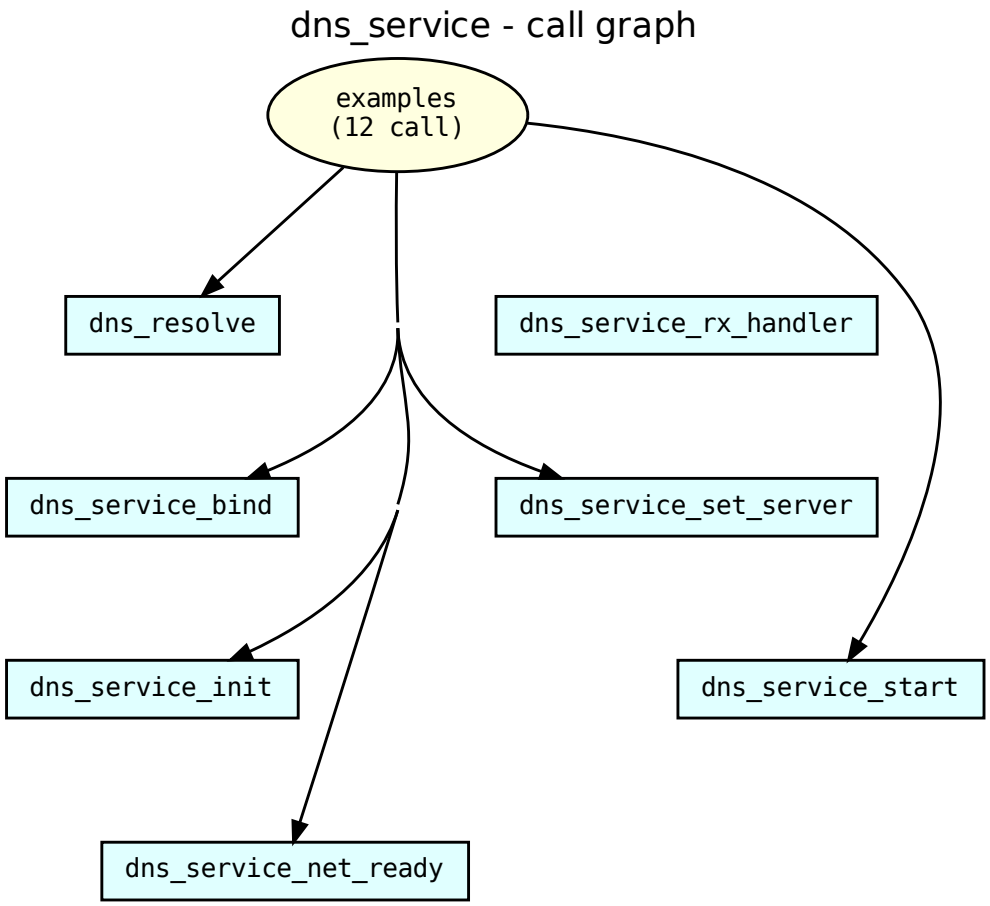
Pagina 1/2



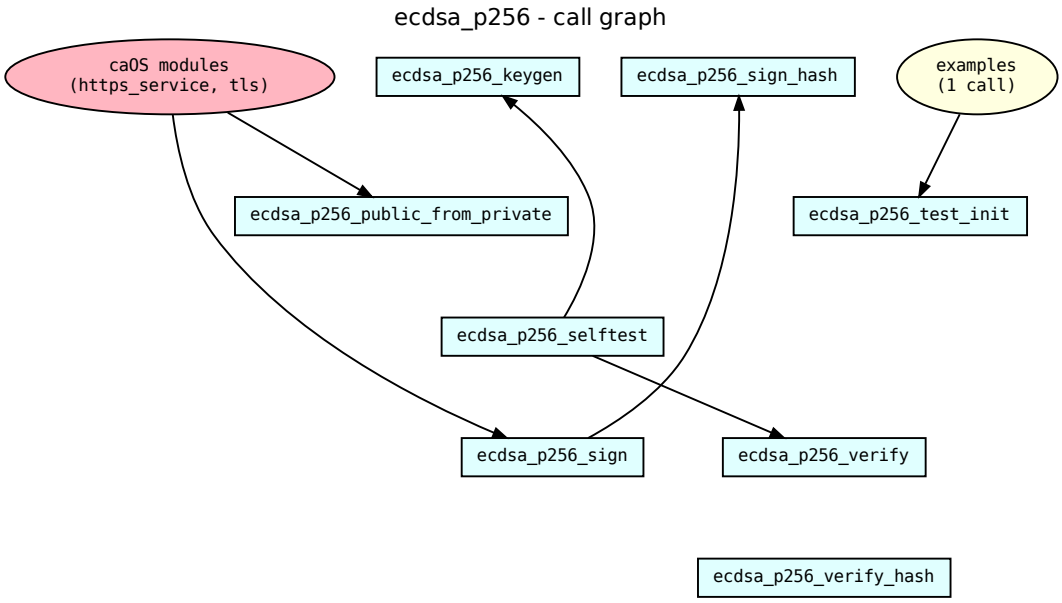
Pagina 2/2


dma2d - call graph (p.2/2)
examples
(3 call)
^ continua da p.1
dma2d_hw_src
dma2d_hw_start
dma2d_init
dma2d_wait
dma2d_hw_init
dma2d_hw_irq
dma2d_convert
(p.altra)
dma2d_blend
(p.altra)
dma2d_hw_clut
dma2d_fill
(p.altra)
dma2d_copy
(p.altra)
dma2d_hw_dst

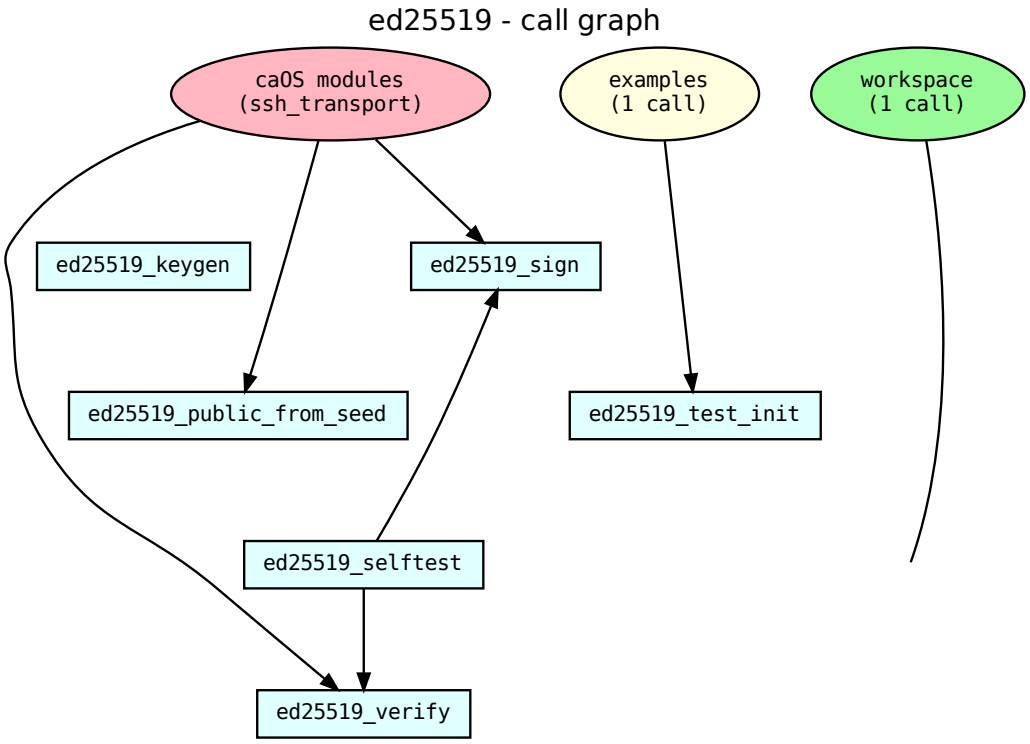
118.22 dns_service



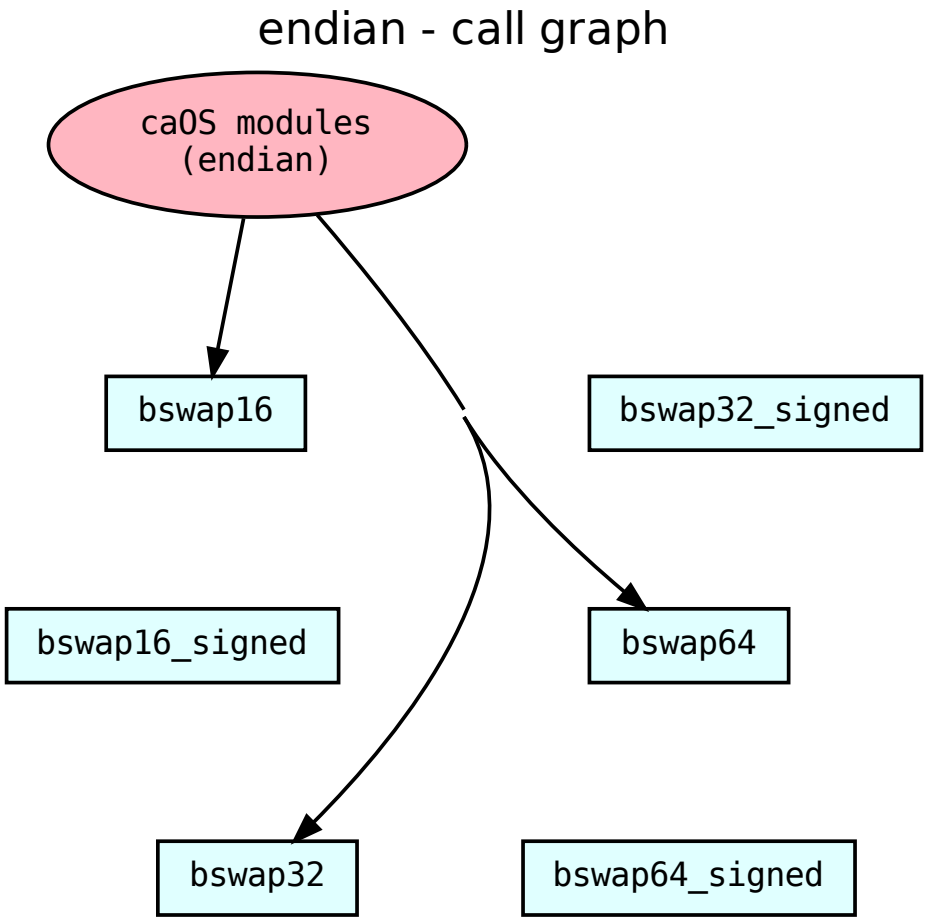
118.23 ecdsa_p256



118.24 ed25519

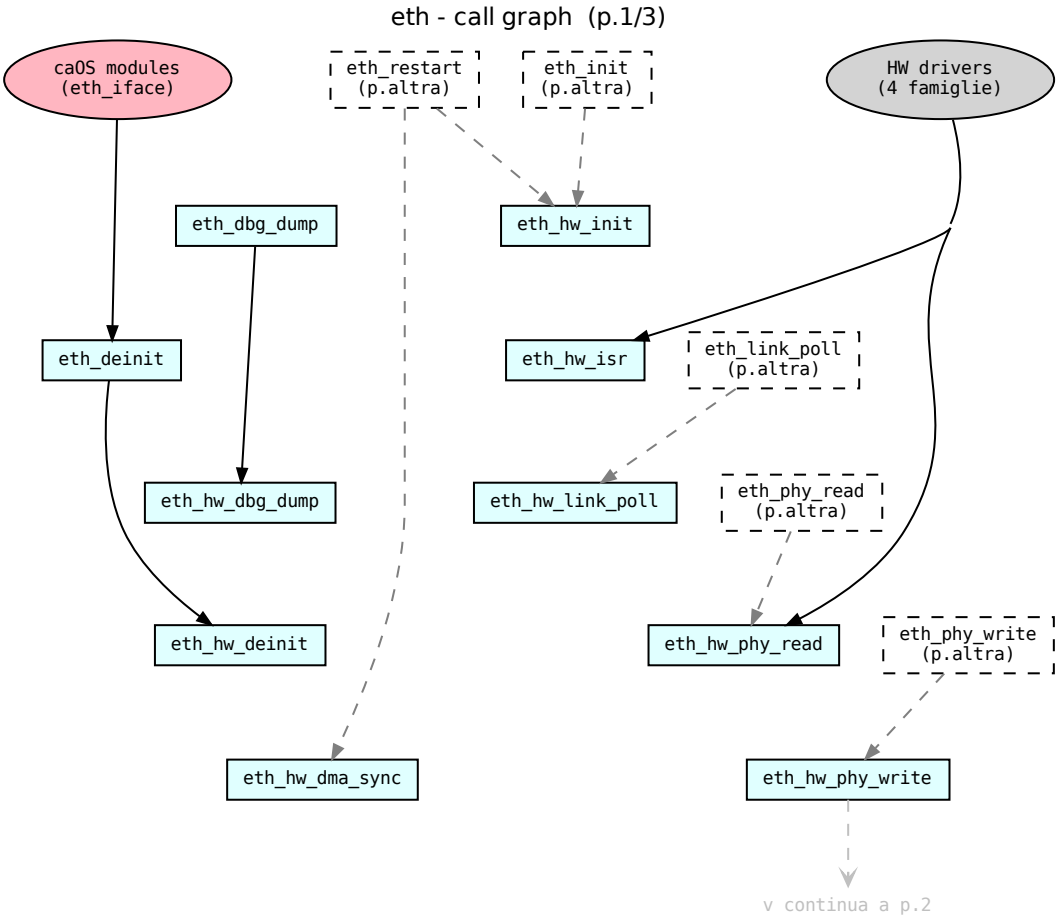


118.25 endian

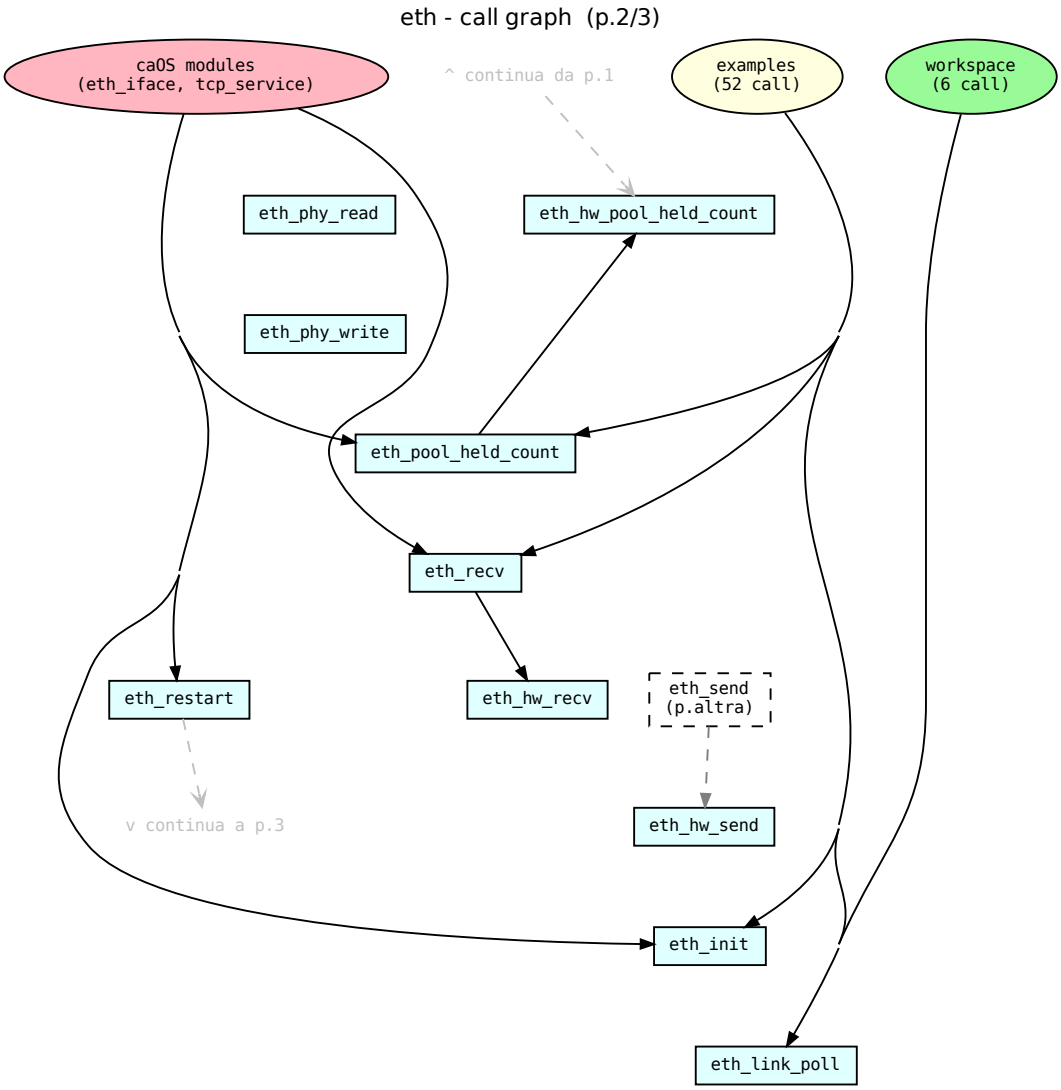


118.26 `eth`

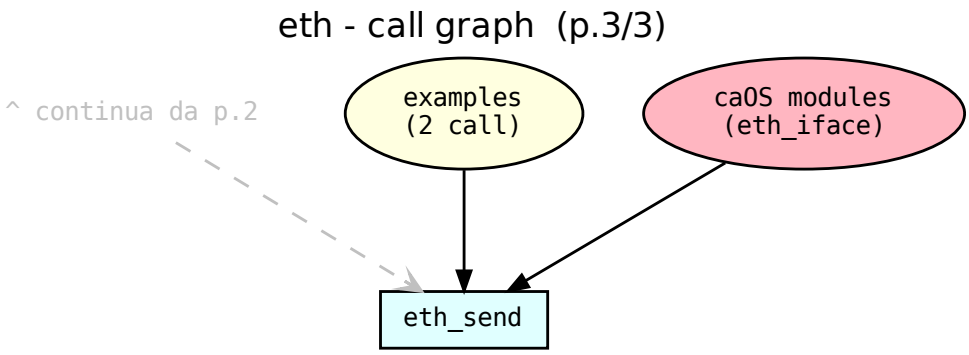
Pagina 1/3



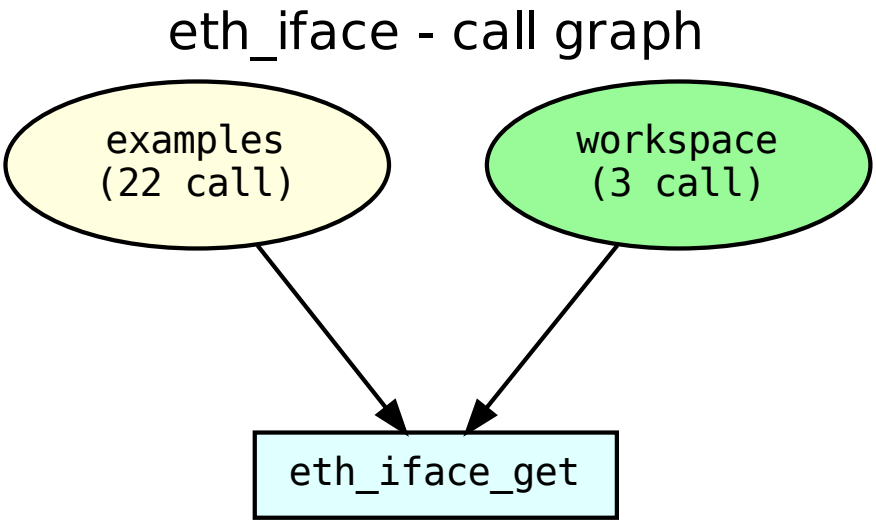
Pagina 2/3


eth - call graph (p.2/3)
caOS modules
(eth_iface, tcp_service)
^ continua da p.1
examples
(52 call)
workspace
(6 call)
eth_phy_read
eth_hw_pool_held_count
eth_phy_write
eth_pool_held_count
eth_recv
eth_restart
eth_hw_recv
eth_send
(p.altra)
v continua a p.3
eth_hw_send
eth_init
eth_link_poll

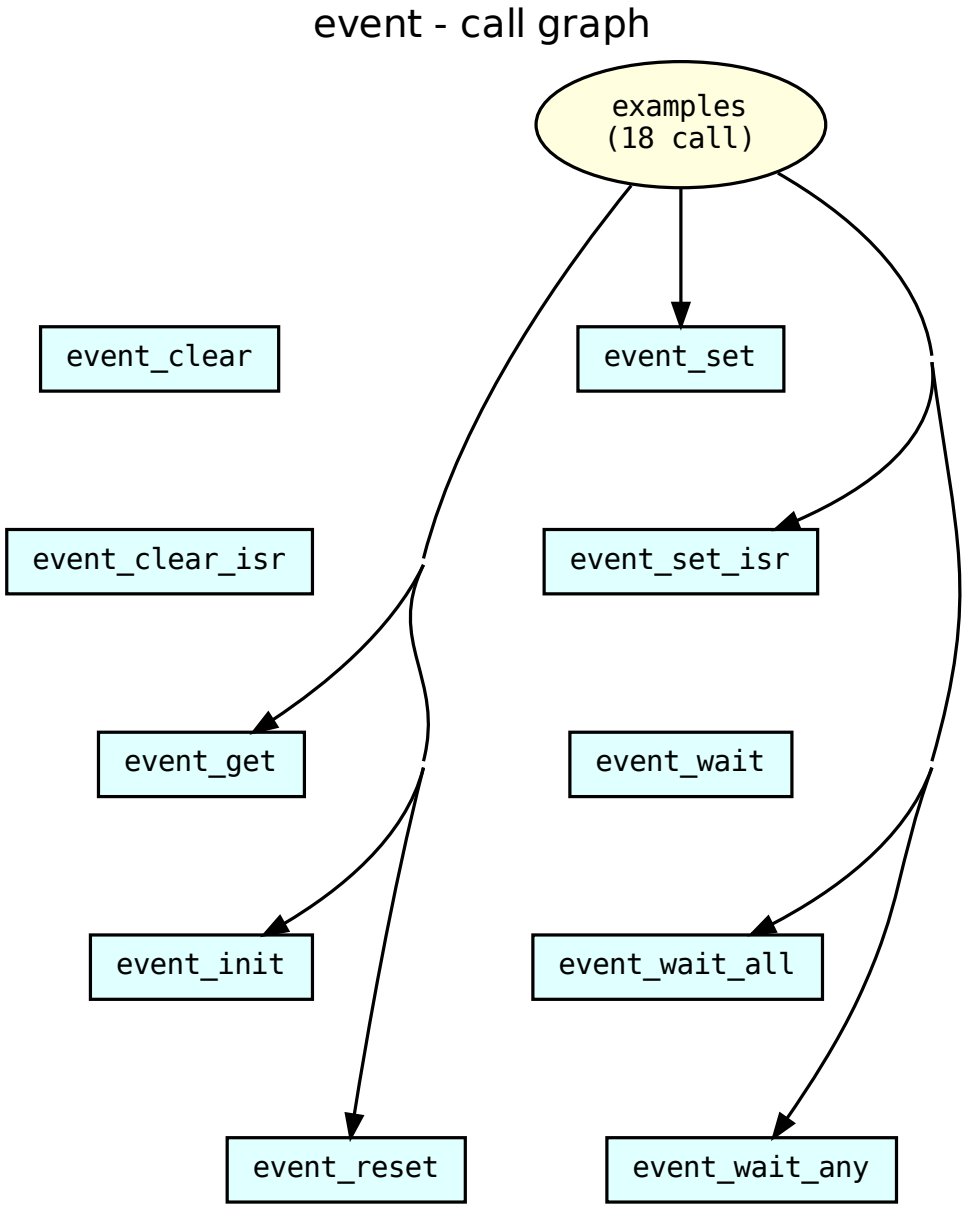
Pagina 3/3


eth - call graph (p.3/3)
^ continua da p.2
examples
(2 call)
caOS modules
(eth_iface)
eth_send

118.27 `eth_iface`

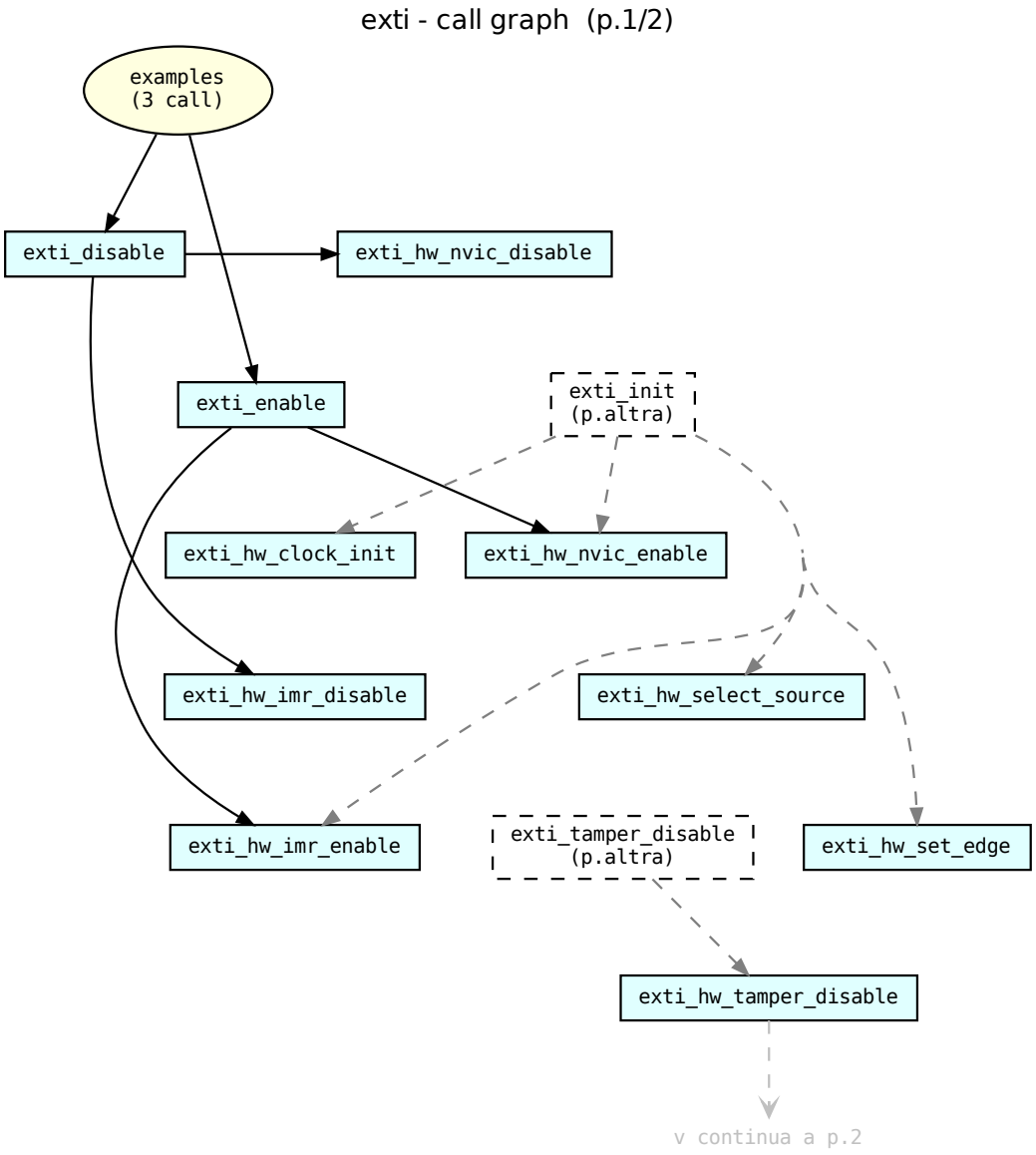


118.28 event

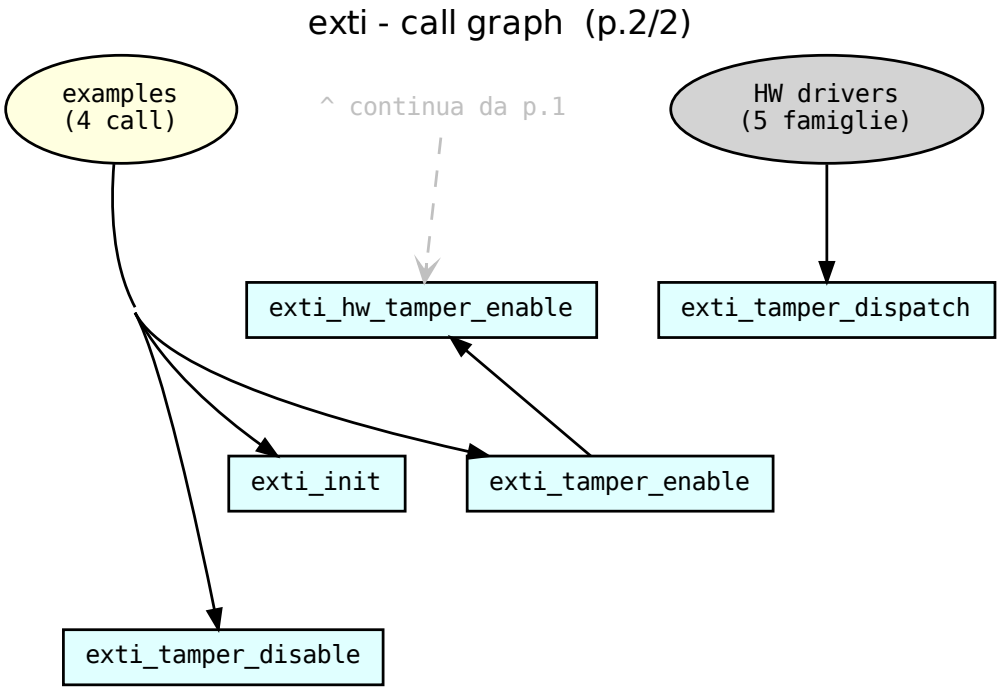

event - call graph
examples
(18 call)
event_clear
event_set
event_clear_isr
event_set_isr
event_get
event_wait
event_init
event_wait_all
event_reset
event_wait_any

118.29 exti

Pagina 1/2

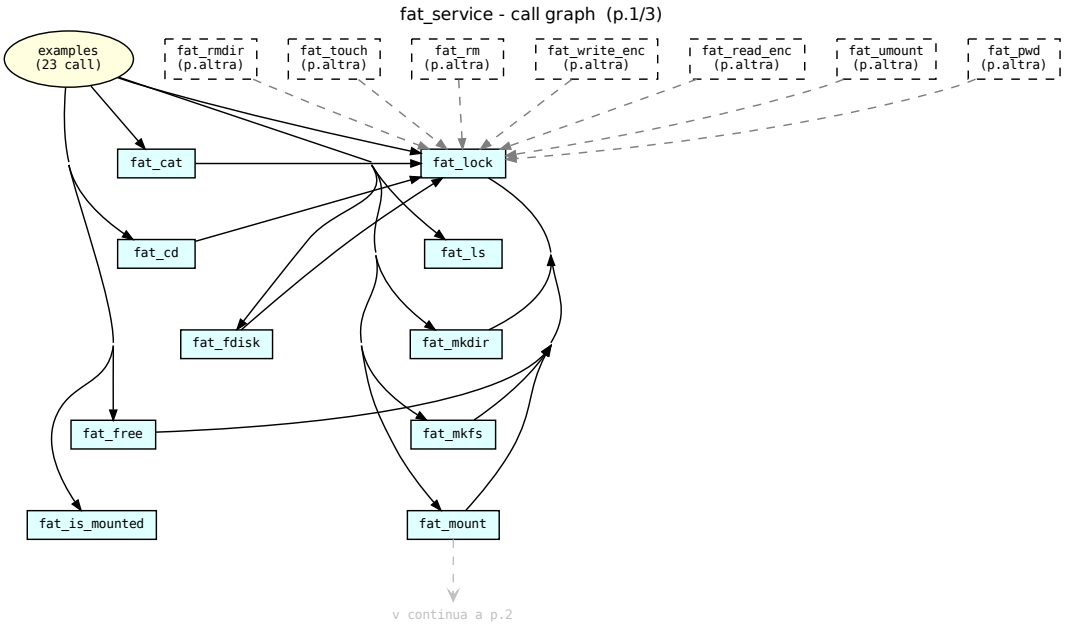


Pagina 2/2

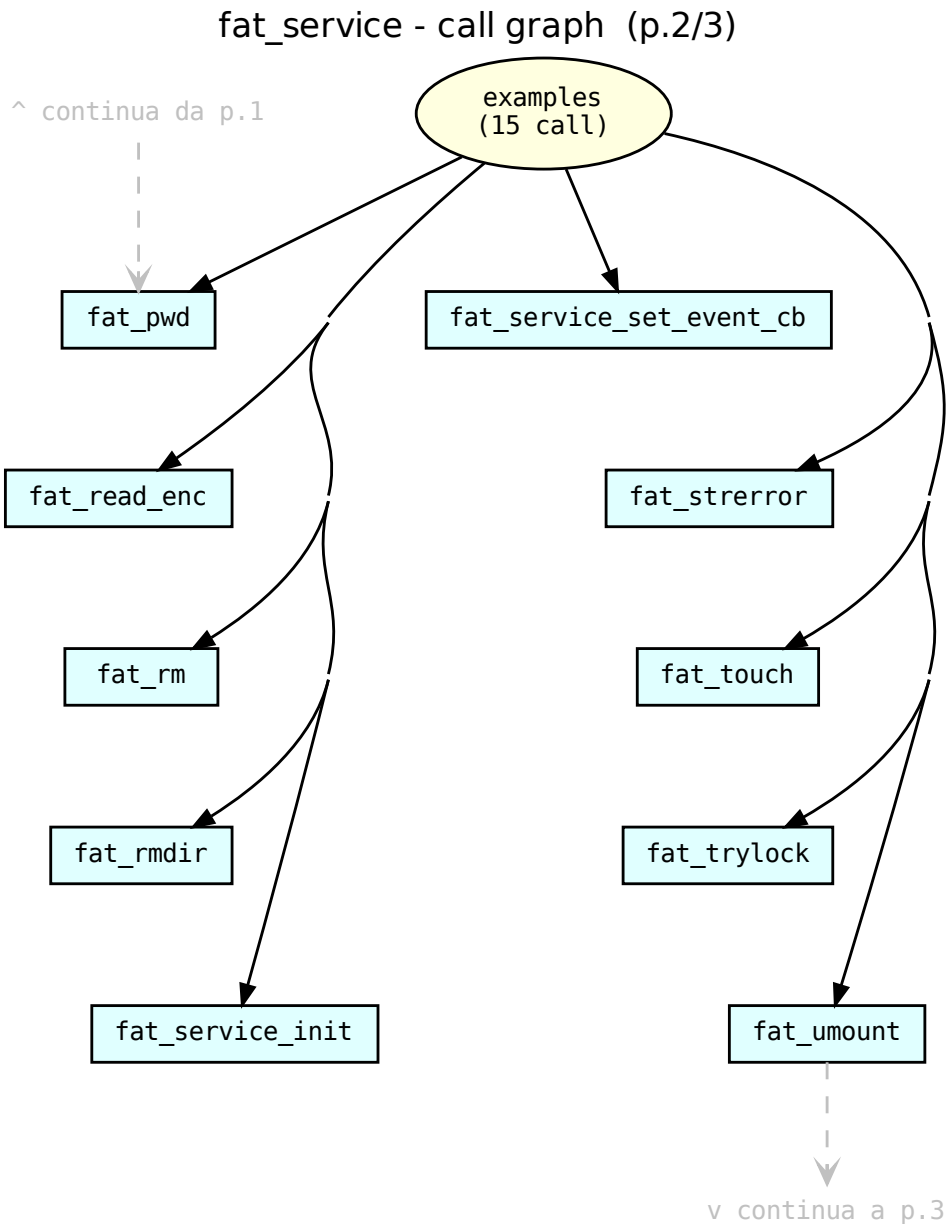

exti - call graph (p.2/2)
examples
(4 call)
^ continua da p.1
HW drivers
(5 famiglie)
exti_hw_tamper_enable
exti_tamper_dispatch
exti_init
exti_tamper_enable
exti_tamper_disable

118.30 `fat_service`

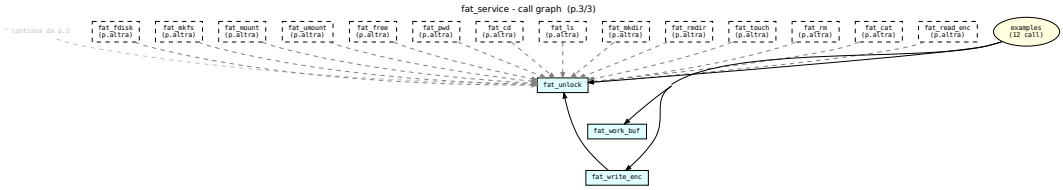
Pagina 1/3



Pagina 2/3

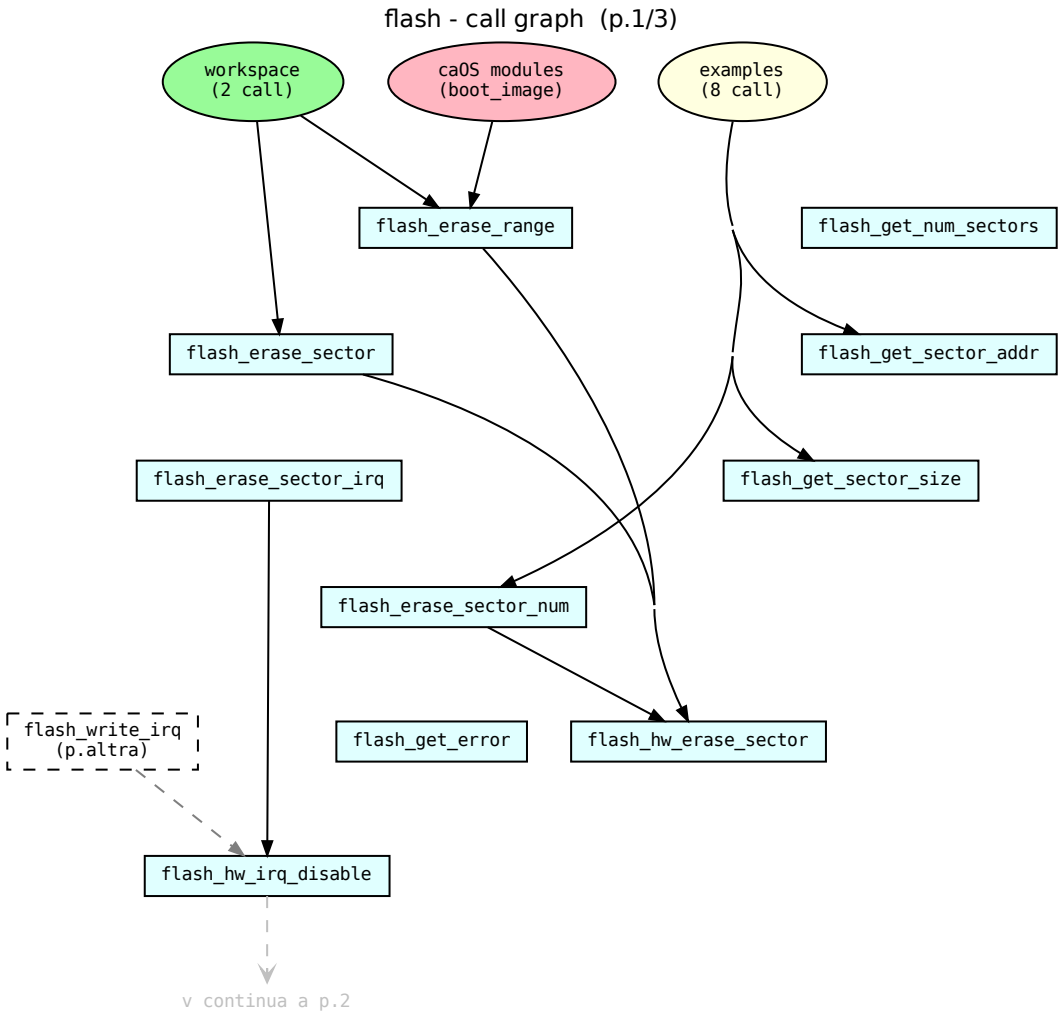

fat_service - call graph (p.2/3)
^ continua da p.1
examples
(15 call)
fat_pwd
fat_service_set_event_cb
fat_read_enc
fat_strerror
fat_rm
fat_touch
fat_rmdir
fat_trylock
fat_service_init
fat_umount
v continua a p.3

Pagina 3/3

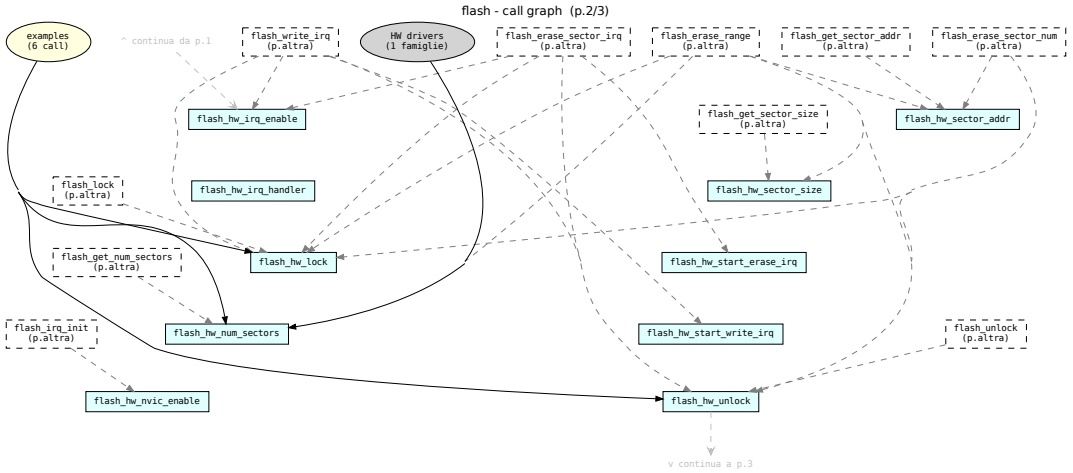

fat_service - call graph (p.3/3)
^ continua da p.2
fat_fdisk (p.altra)
fat_mkfs (p.altra)
fat_mount (p.altra)
fat_umount (p.altra)
fat_free (p.altra)
fat_pwd (p.altra)
fat_cd (p.altra)
fat_ls (p.altra)
fat_mkdir (p.altra)
fat_rmdir (p.altra)
fat_touch (p.altra)
fat_rm (p.altra)
fat_cat (p.altra)
fat_read_enc (p.altra)
examples (12 call)
fat_unlock
fat_work_buf
fat_write_enc

118.31 flash

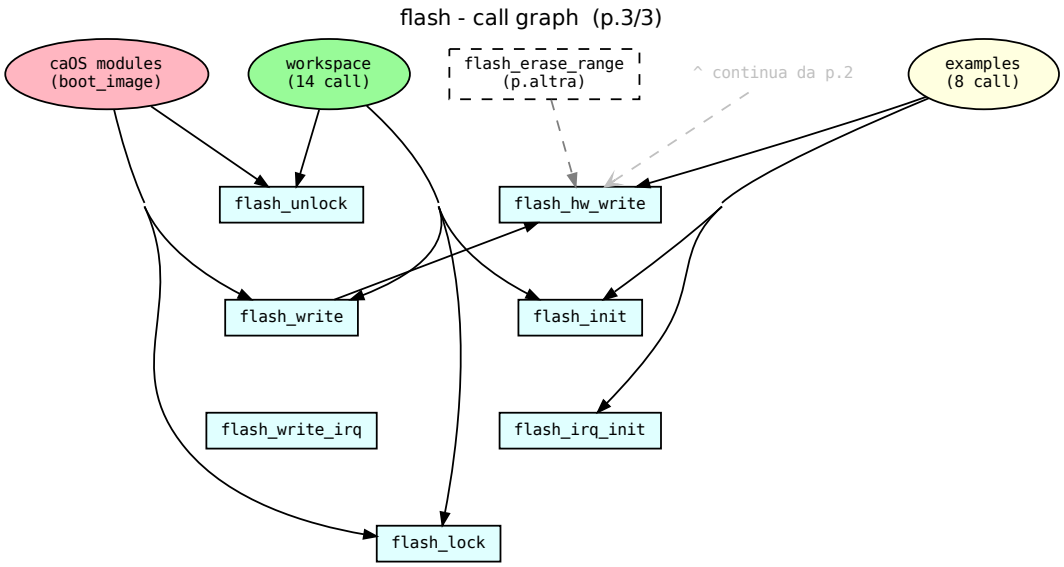
Pagina 1/3



Pagina 2/3

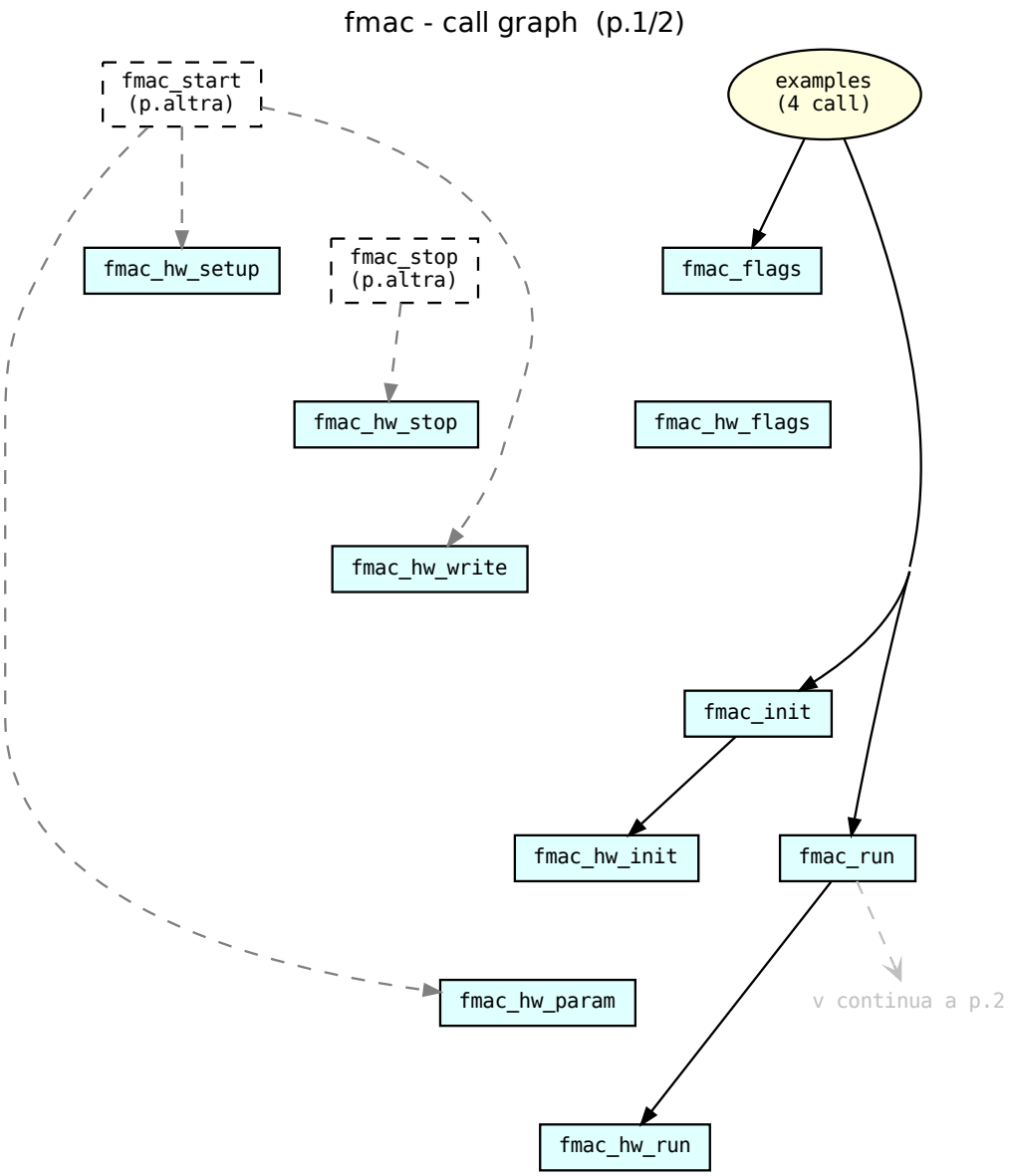

flash - call graph (p.2/3)
examples (6 call)
^ continua da p.1
flash_write_irq (p.altra)
HW drivers (1 famiglie)
flash_erase_sector_irq (p.altra)
flash_erase_range (p.altra)
flash_get_sector_addr (p.altra)
flash_erase_sector_num (p.altra)
flash_hw_irq_enable
flash_get_sector_size (p.altra)
flash_hw_sector_addr
flash_lock (p.altra)
flash_hw_irq_handler
flash_hw_sector_size
flash_get_num_sectors (p.altra)
flash_hw_lock
flash_hw_start_erase_irq
flash_irq_init (p.altra)
flash_hw_num_sectors
flash_hw_start_write_irq
flash_unlock (p.altra)
flash_hw_nvic_enable
flash_hw_unlock
v continua a p.3

Pagina 3/3

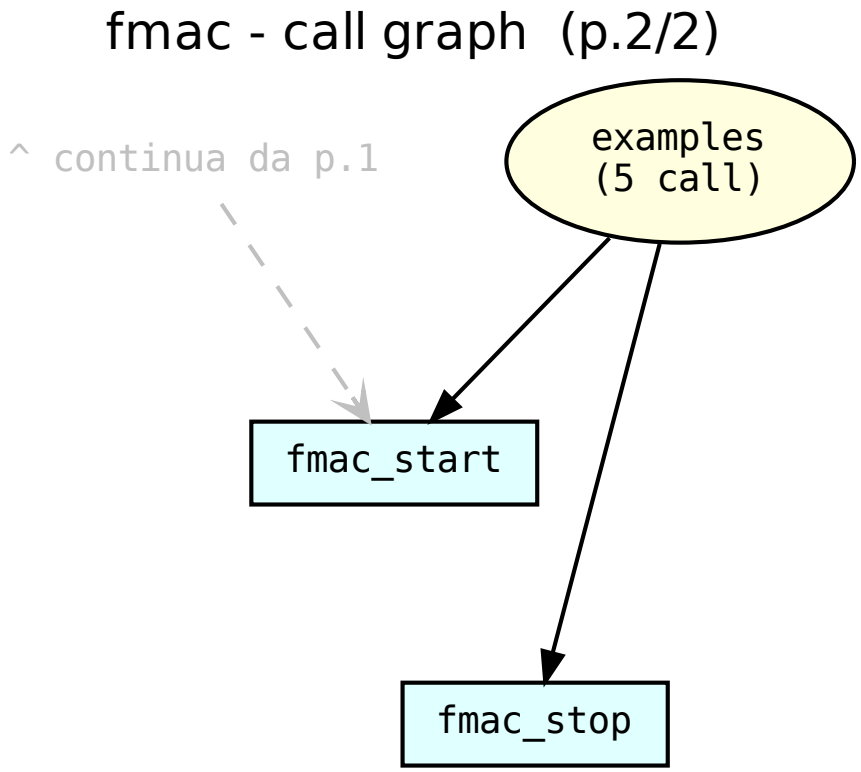

flash - call graph (p.3/3)
caOS modules (boot_image)
workspace (14 call)
flash_erase_range (p.altra)
^ continua da p.2
examples (8 call)
flash_unlock
flash_hw_write
flash_write
flash_init
flash_write_irq
flash_irq_init
flash_lock

118.32 `fmac`

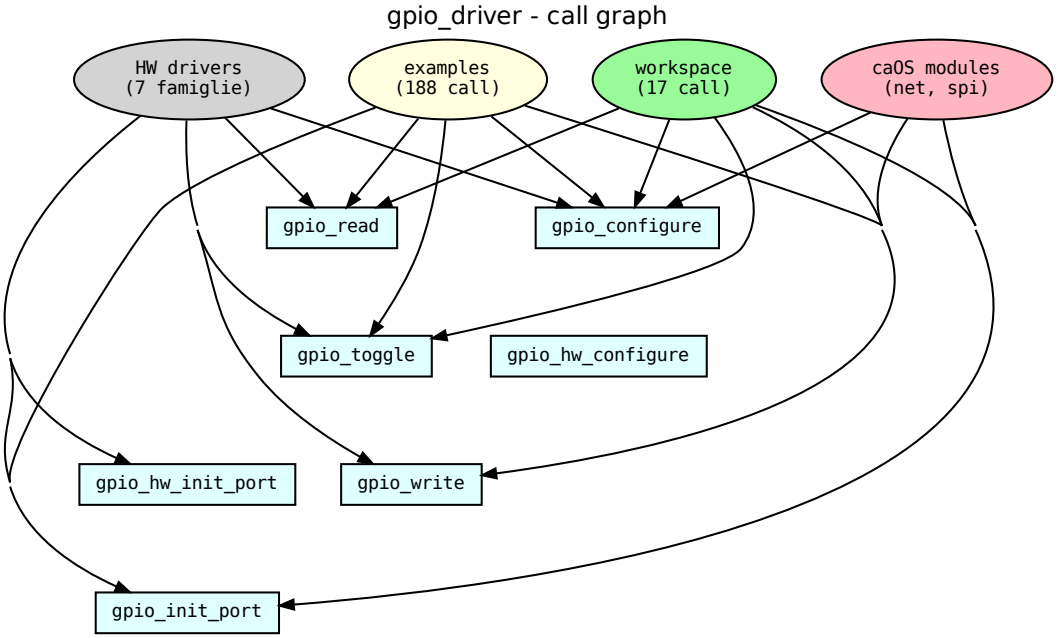
Pagina 1/2



Pagina 2/2

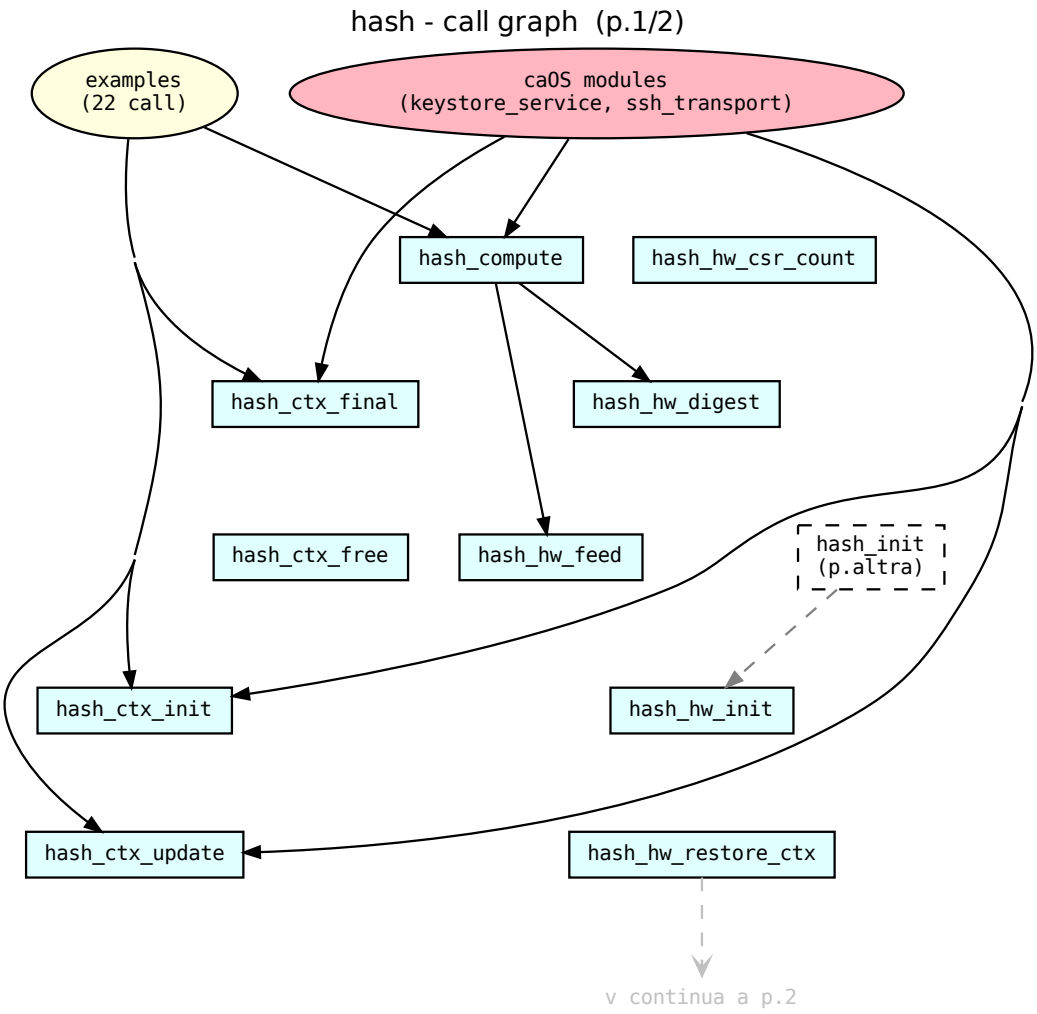

fmac - call graph (p.2/2)
^ continua da p.1
examples
(5 call)
fmac_start
fmac_stop

118.33 gpio_driver

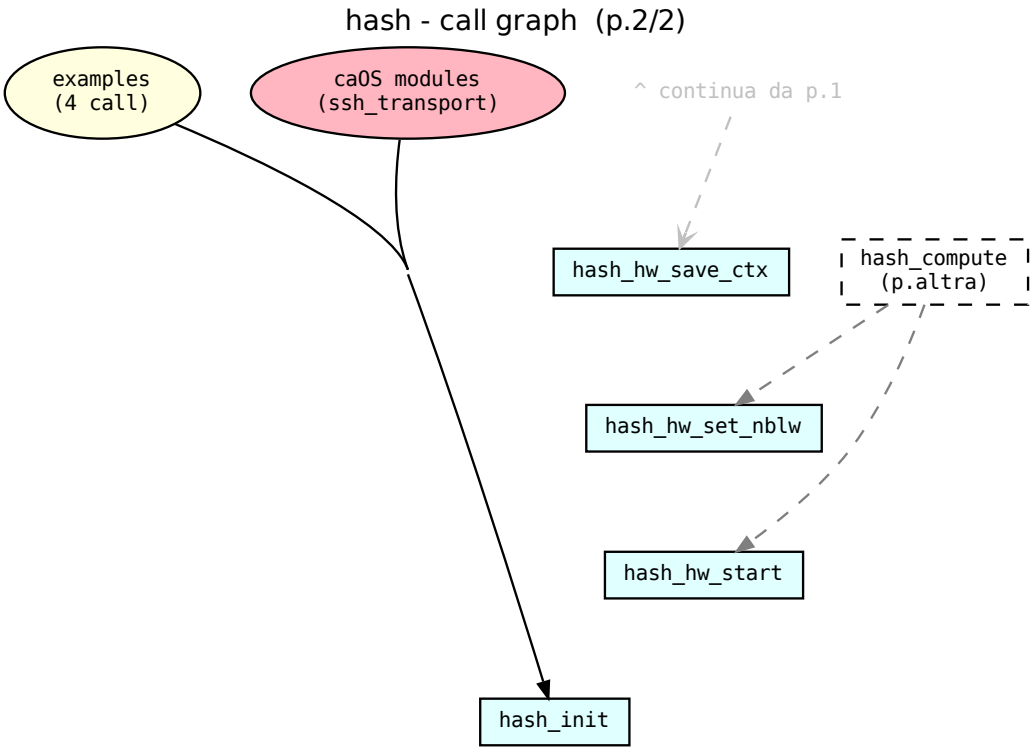


118.34 hash

Pagina 1/2

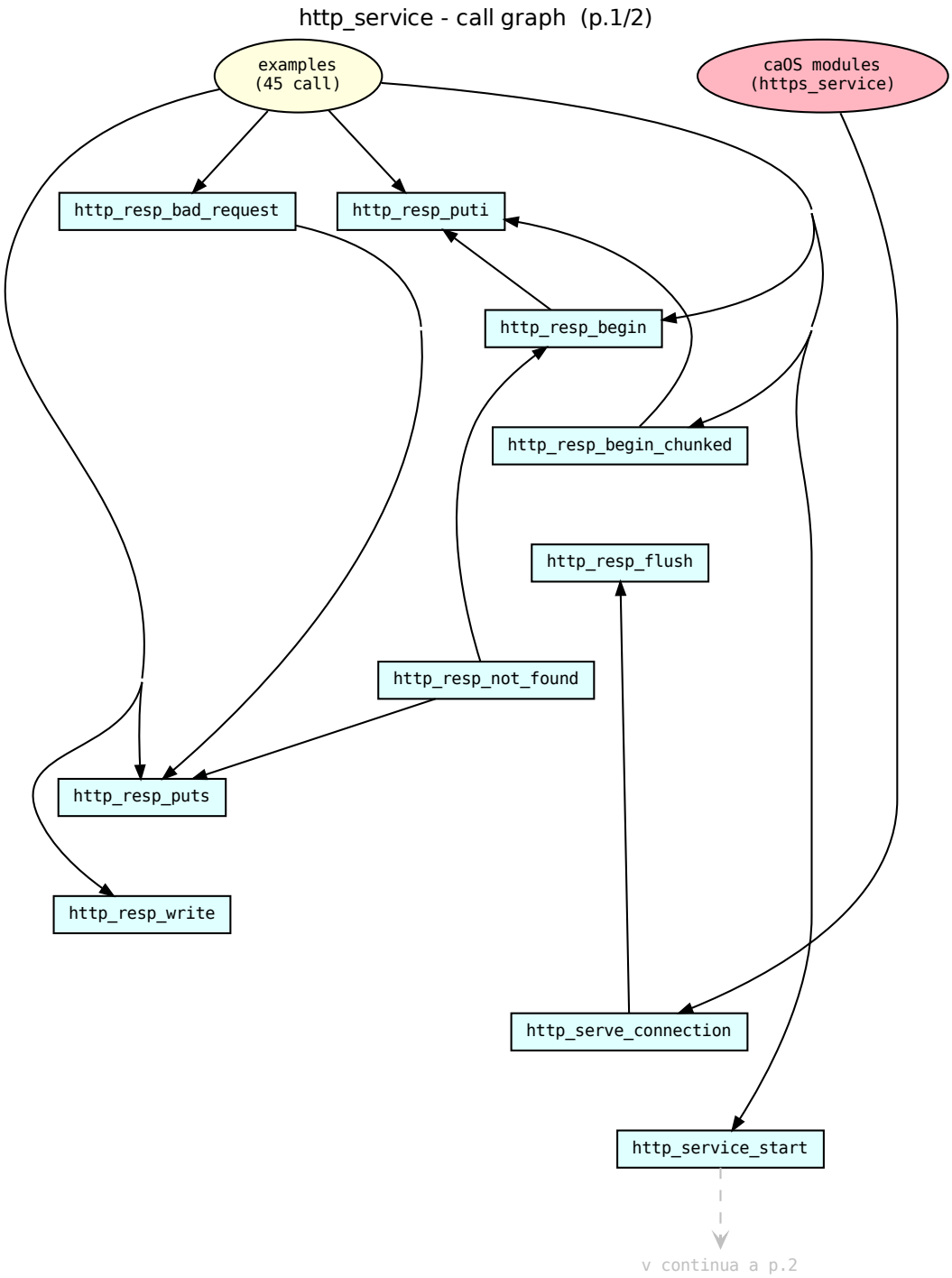


Pagina 2/2

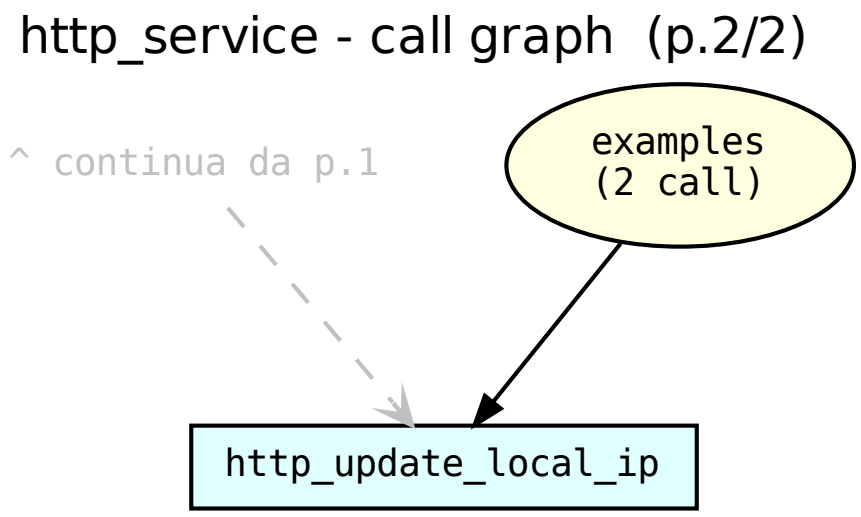

hash - call graph (p.2/2)
examples
(4 call)
caOS modules
(ssh_transport)
^ continua da p.1
hash_hw_save_ctx
hash_compute
(p.altra)
hash_hw_set_nblw
hash_hw_start
hash_init

118.35 http_service

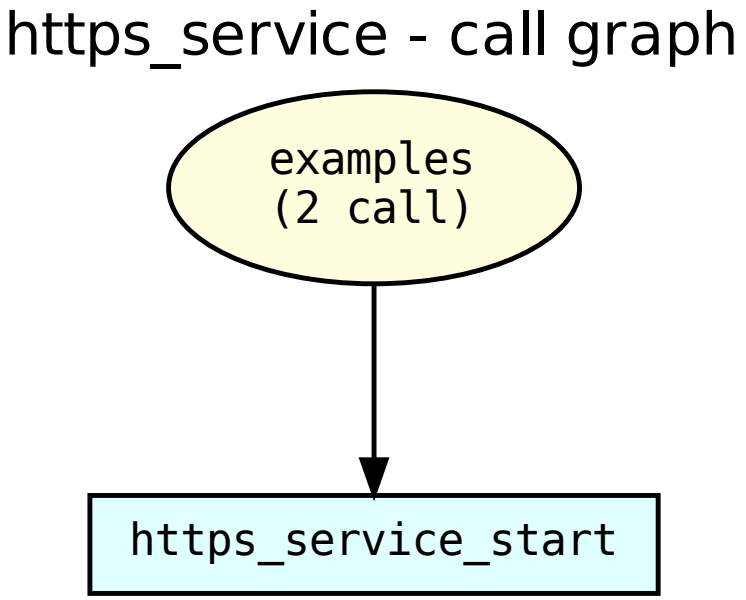
Pagina 1/2



Pagina 2/2

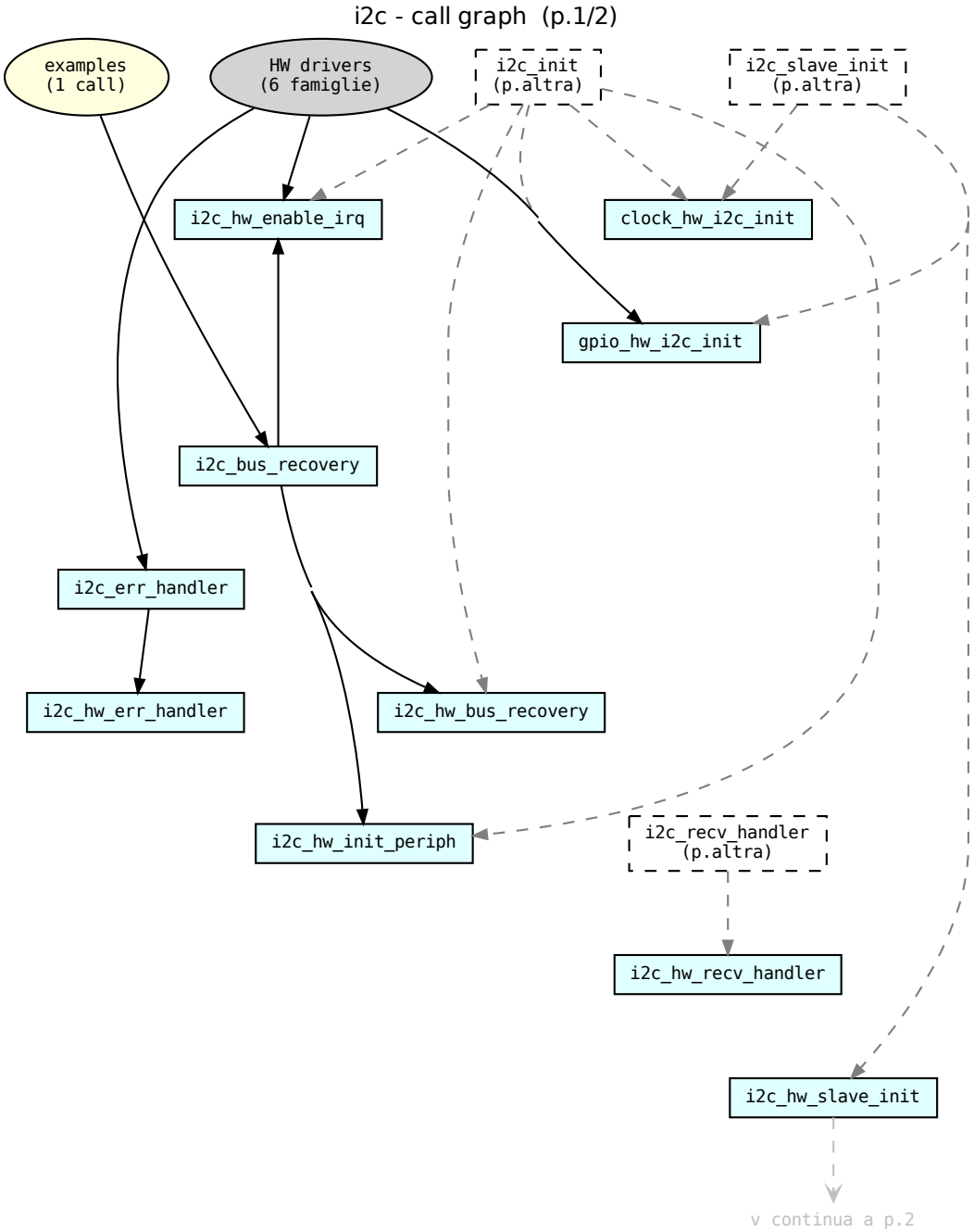

http_service - call graph (p.2/2)
^ continua da p.1
examples
(2 call)
http_update_local_ip

118.36 https_service

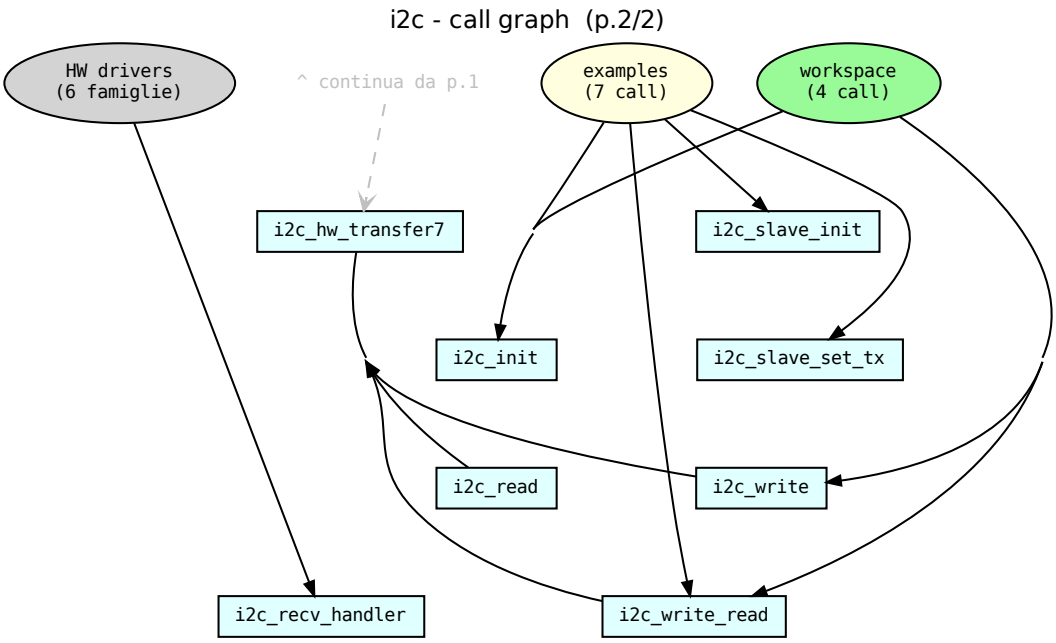


118.37 i2c

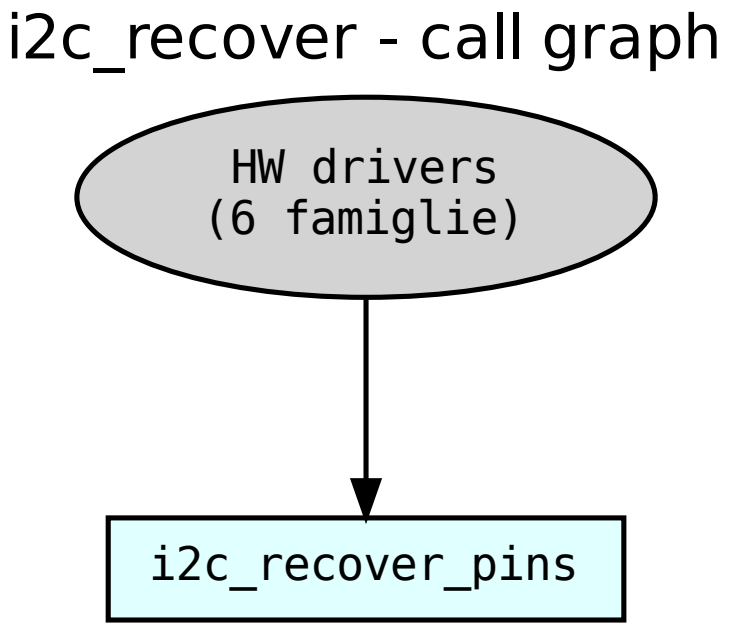
Pagina 1/2



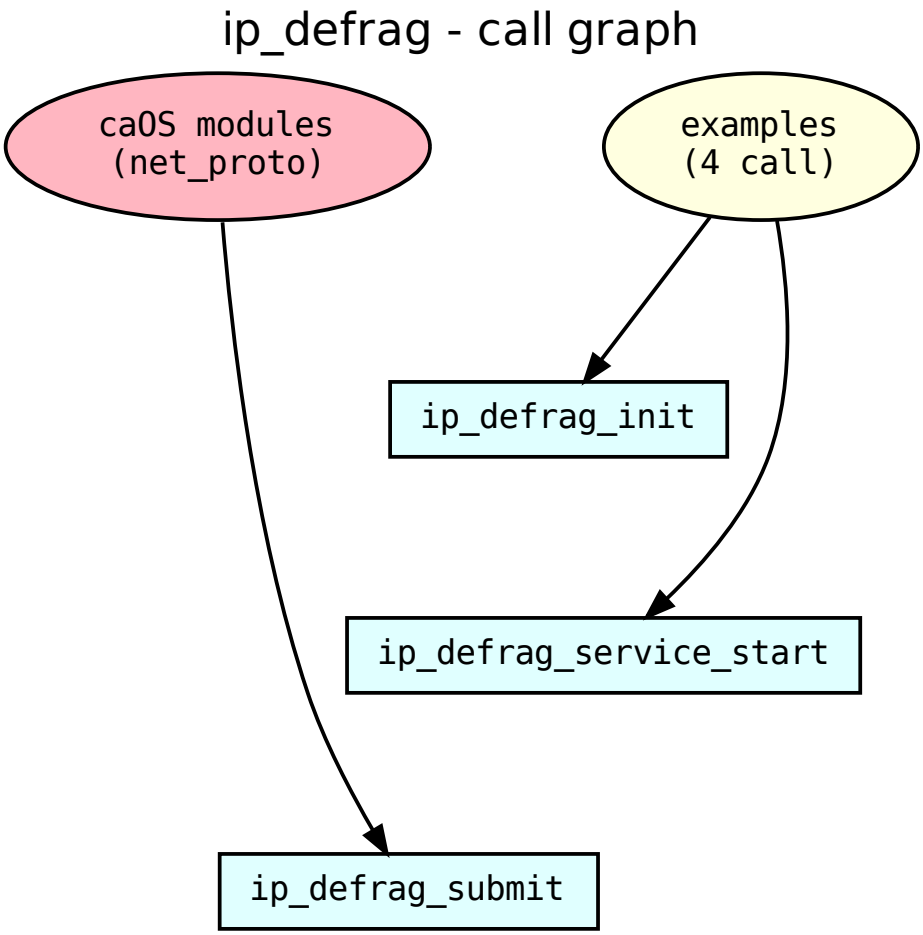
Pagina 2/2


i2c - call graph (p.2/2)
HW drivers
(6 famiglie)
^ continua da p.1
examples
(7 call)
workspace
(4 call)
i2c_hw_transfer7
i2c_slave_init
i2c_init
i2c_slave_set_tx
i2c_read
i2c_write
i2c_recv_handler
i2c_write_read

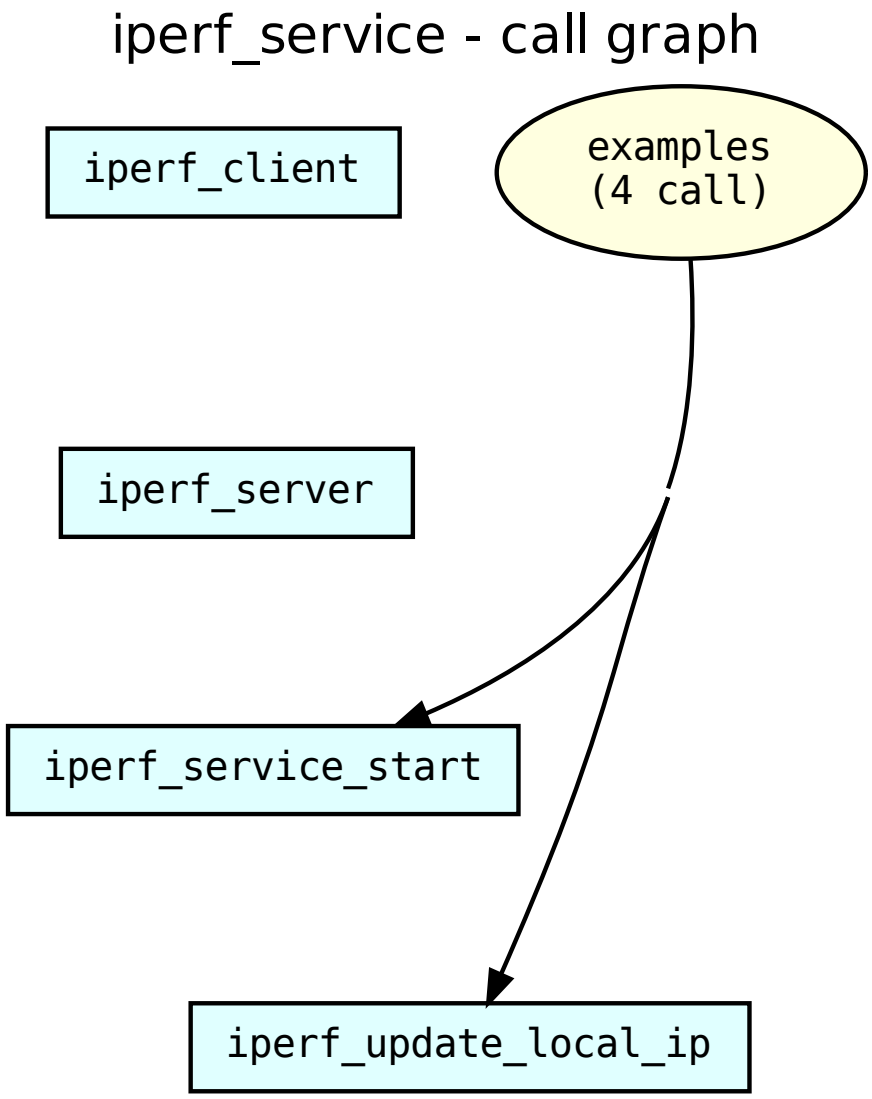
118.38 `i2c_recover`



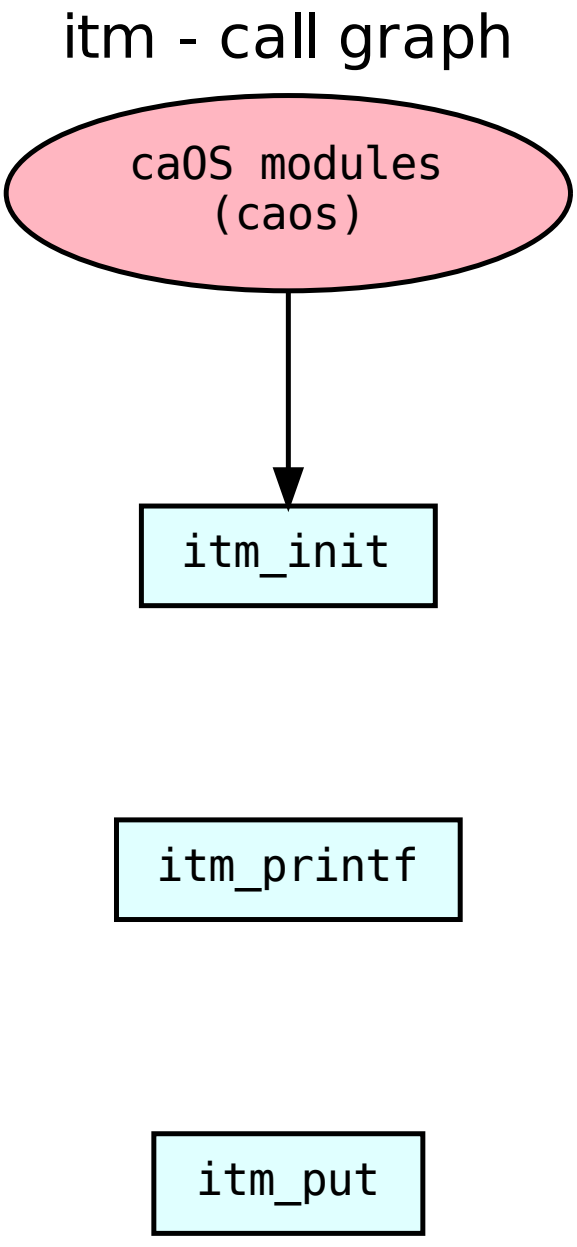
118.39 ip_defrag



118.40 `iperf_service`

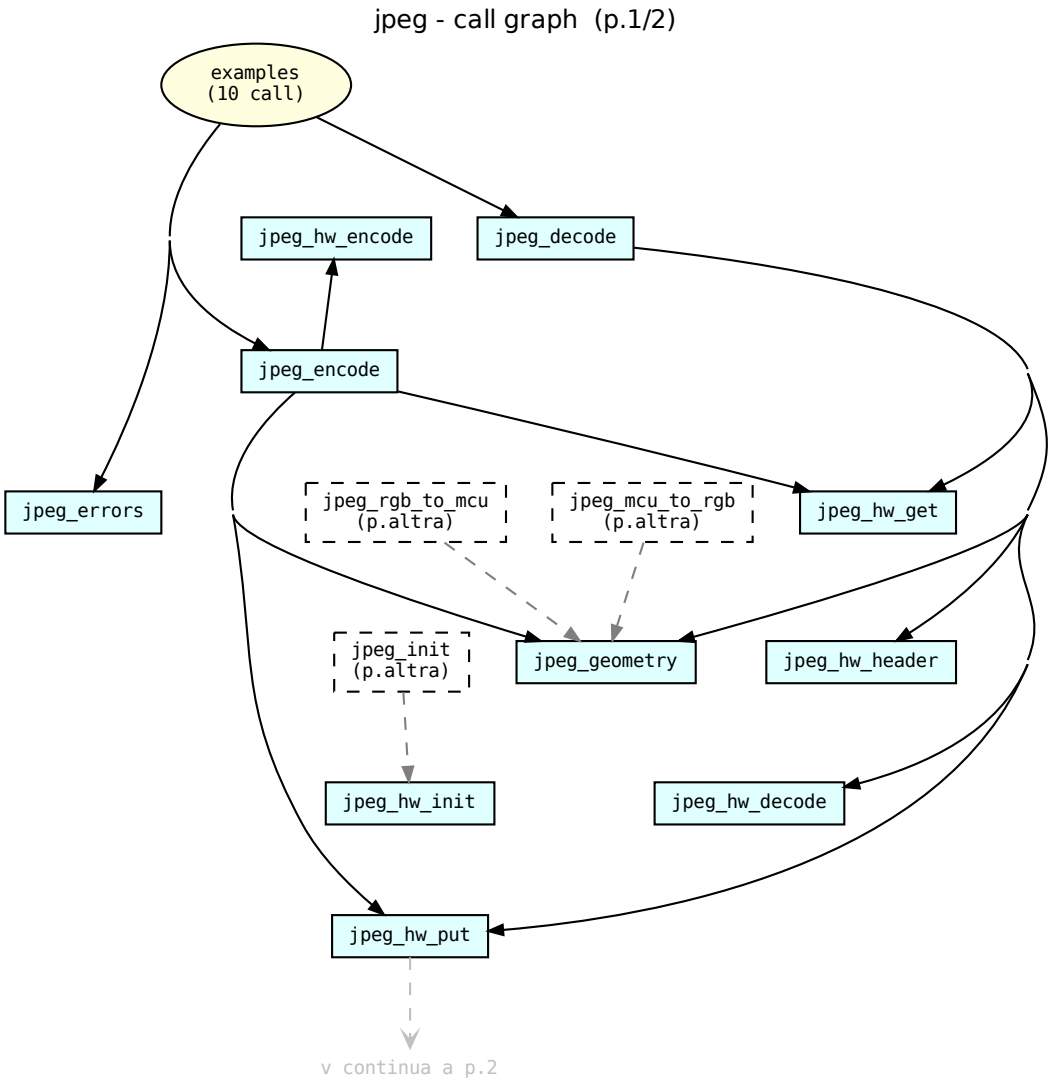


118.41 itm

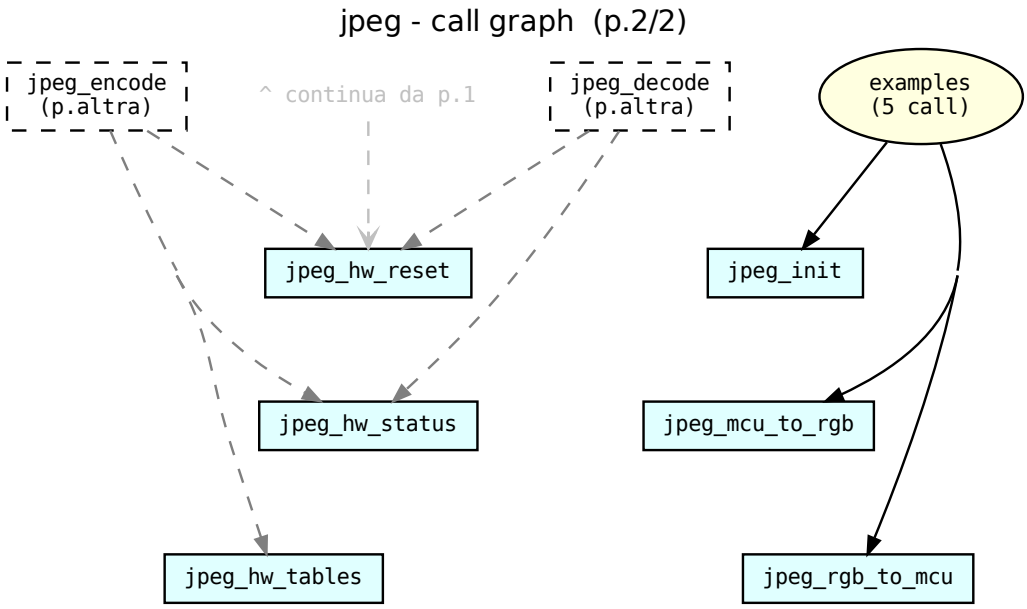


118.42 jpeg

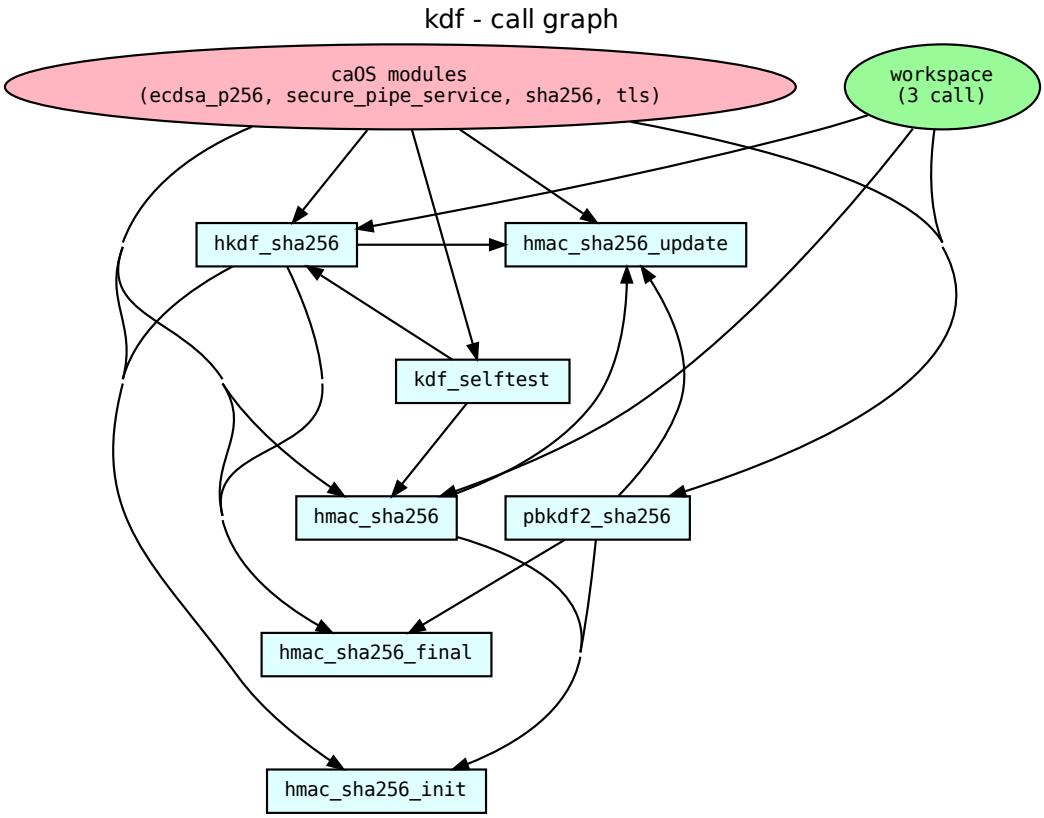
Pagina 1/2



Pagina 2/2

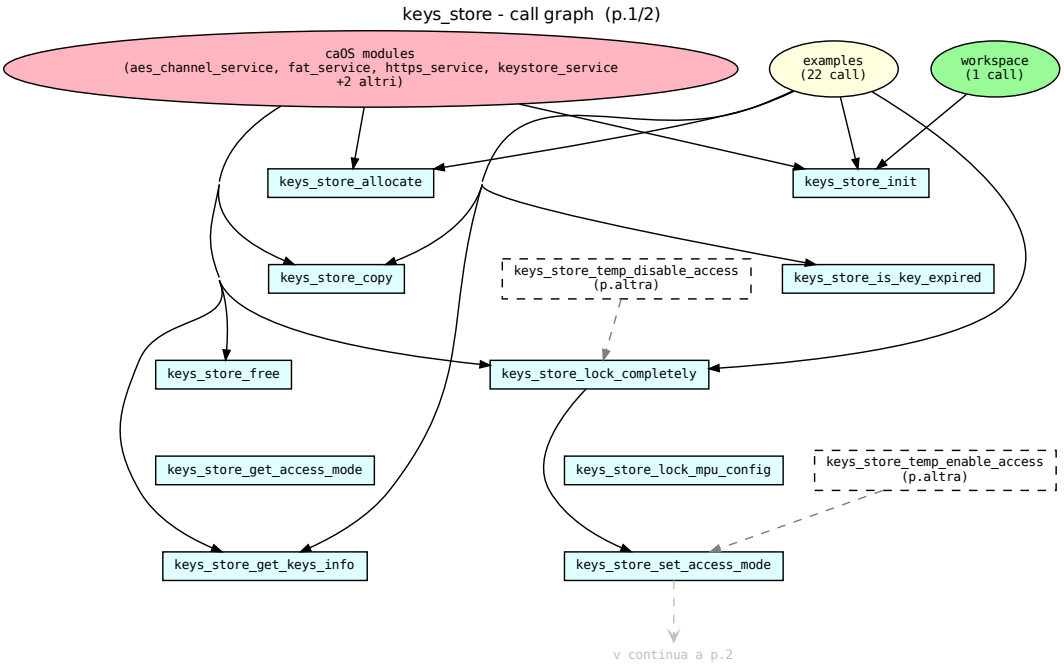

jpeg - call graph (p.2/2)
jpeg_encode
(p.altra)
^ continua da p.1
jpeg_decode
(p.altra)
examples
(5 call)
jpeg_hw_reset
jpeg_init
jpeg_hw_status
jpeg_mcu_to_rgb
jpeg_hw_tables
jpeg_rgb_to_mcu

118.43 `kdf`

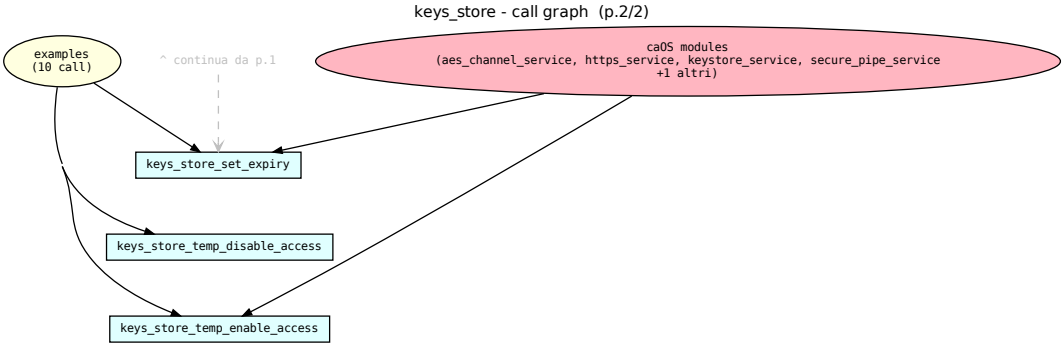

kdf - call graph
caOS modules
(ecdsa_p256, secure_pipe_service, sha256, tls)
workspace
(3 call)
hkdf_sha256
hmac_sha256_update
kdf_selftest
hmac_sha256
pbkdf2_sha256
hmac_sha256_final
hmac_sha256_init

118.44 keys_store

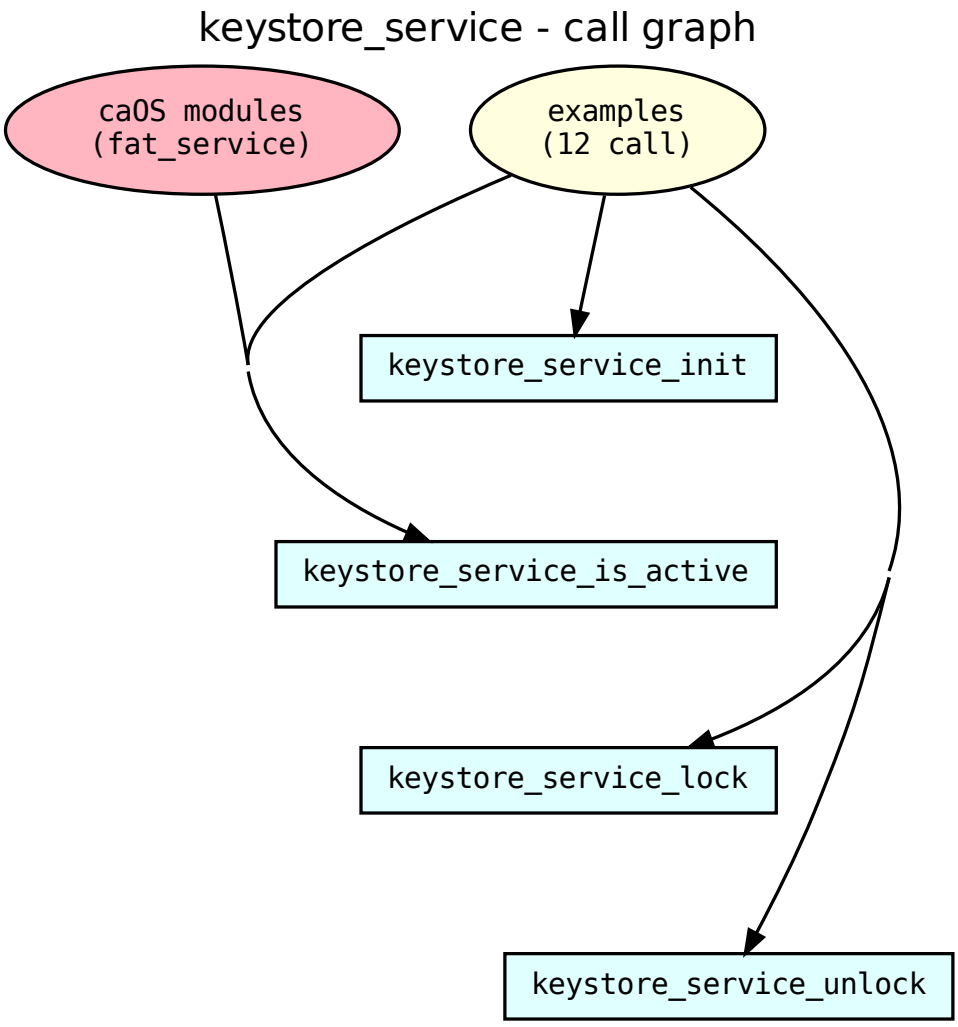
Pagina 1/2



Pagina 2/2

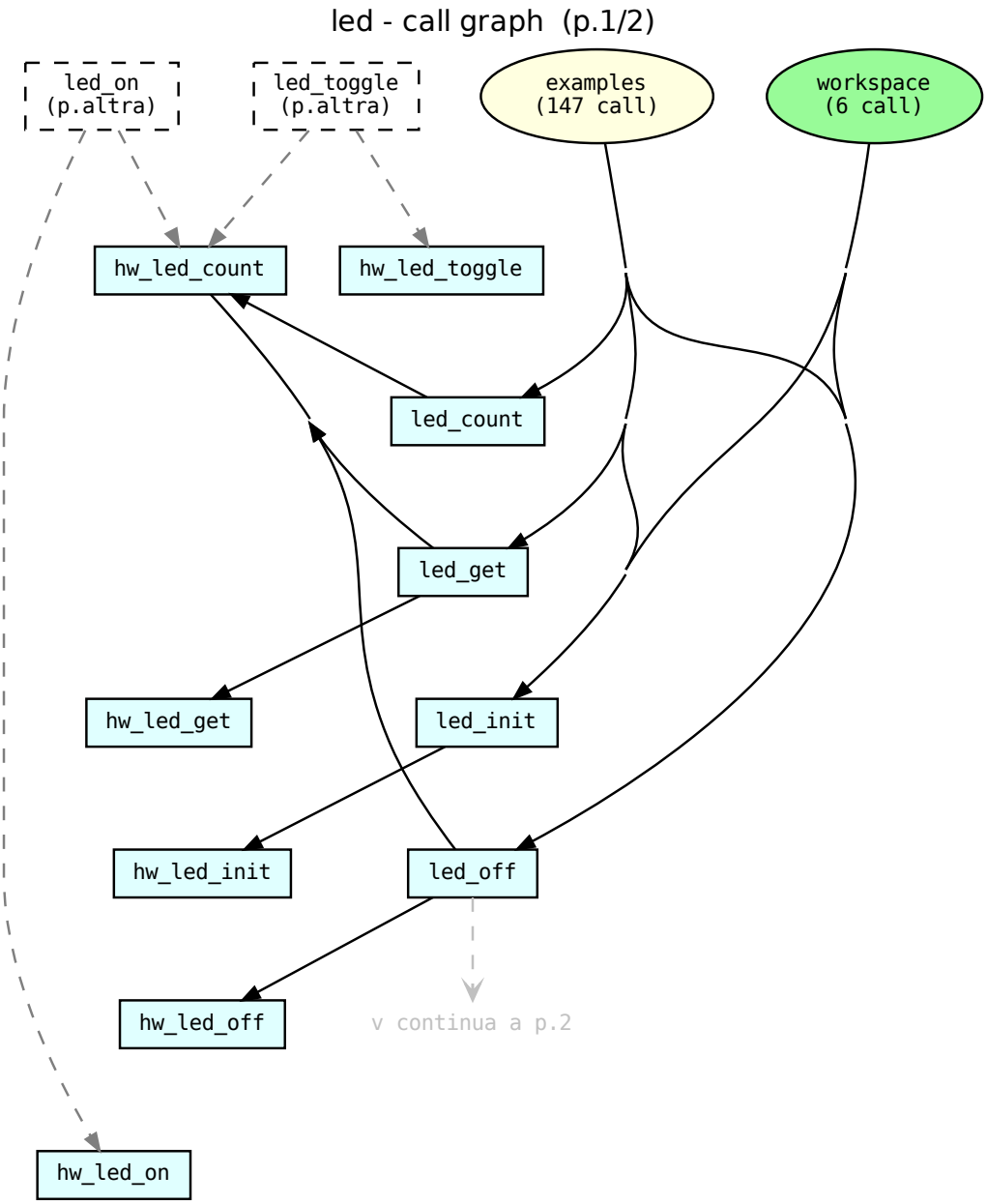

keys_store - call graph (p.2/2)
examples
(10 call)
^ continua da p.1
caOS modules
(aes_channel_service, https_service, keystore_service, secure_pipe_service
+1 altri)
keys_store_set_expiry
keys_store_temp_disable_access
keys_store_temp_enable_access

118.45 keystore_service

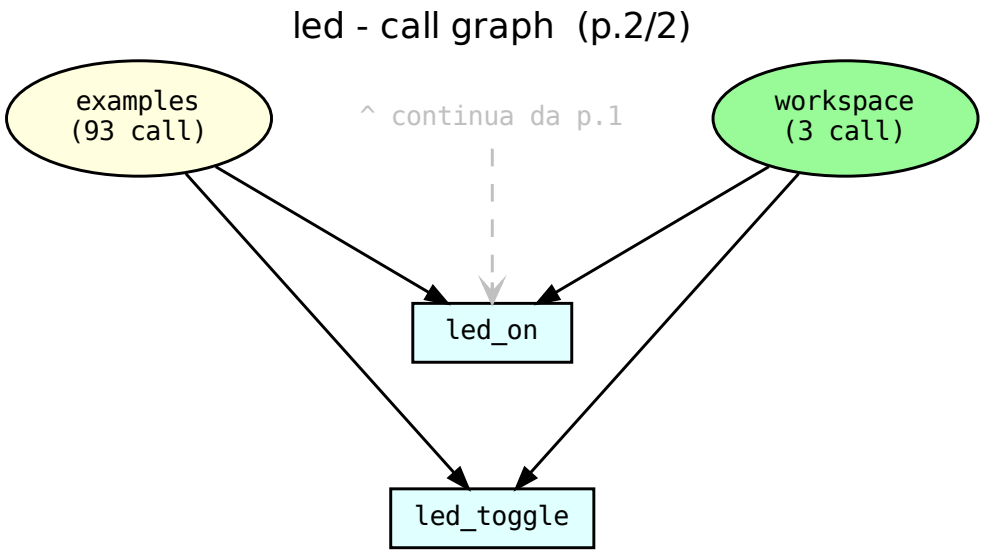


118.46 led

Pagina 1/2

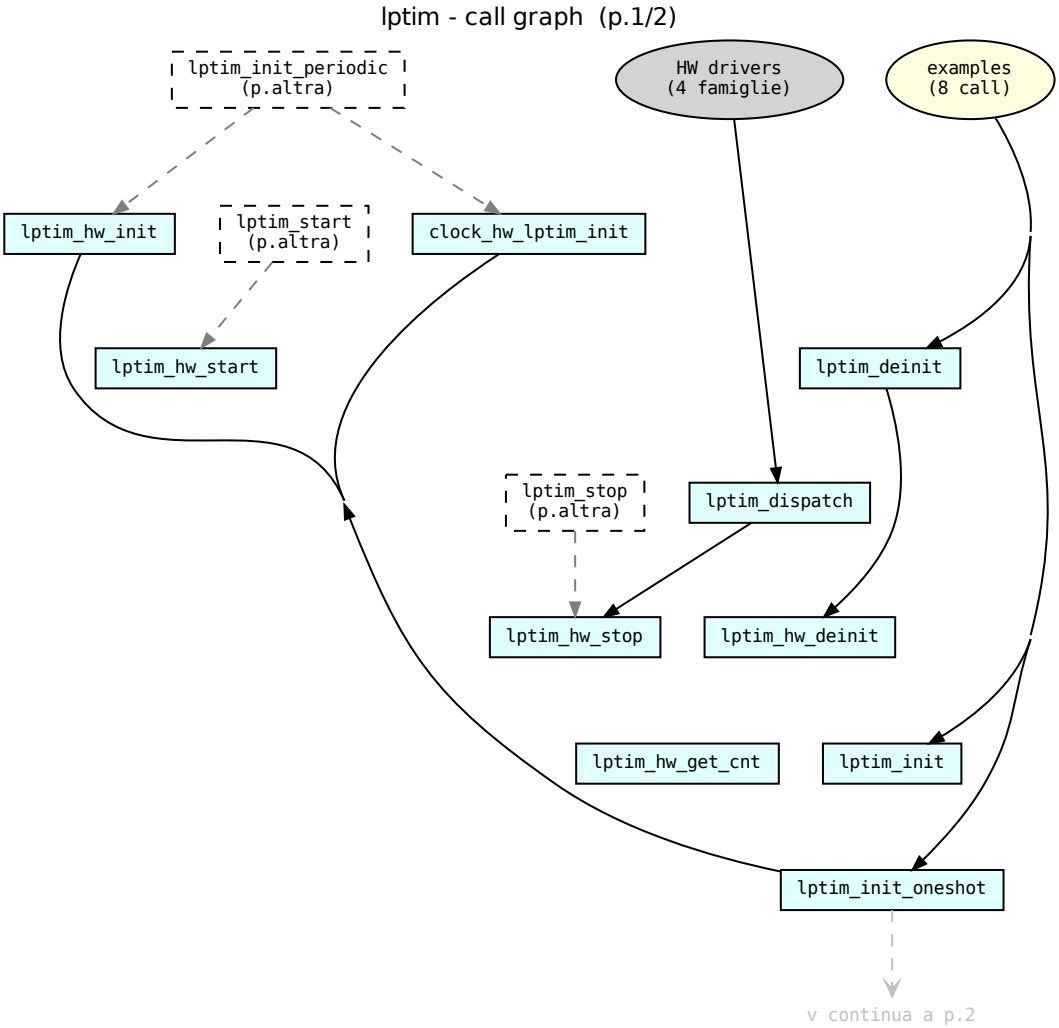


Pagina 2/2

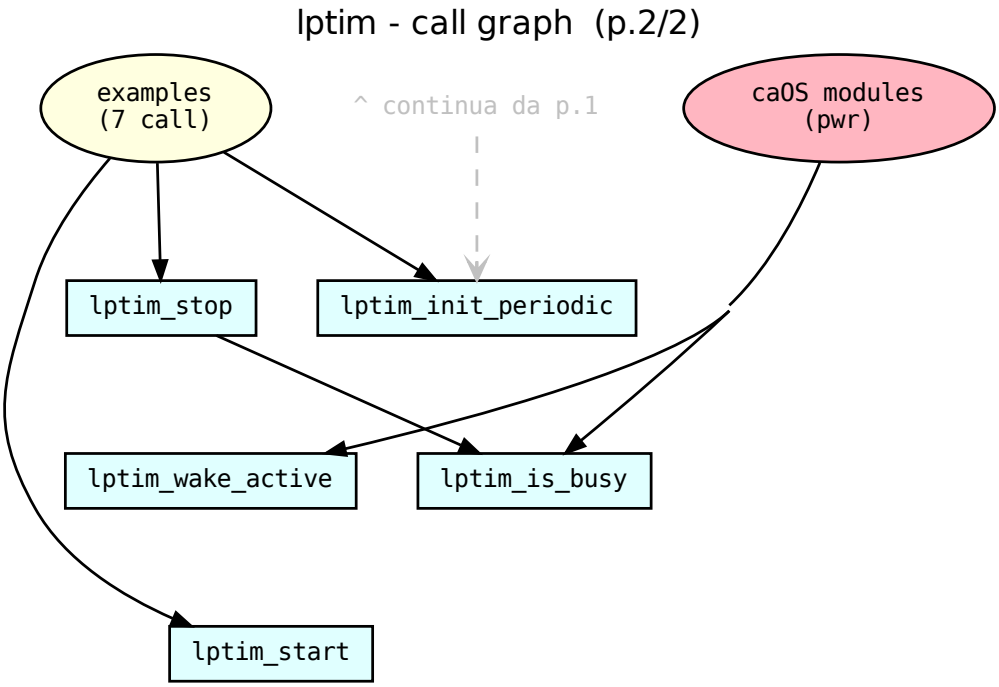

led - call graph (p.2/2)
examples
(93 call)
^ continua da p.1
workspace
(3 call)
led_on
led_toggle

118.47 lptim

Pagina 1/2

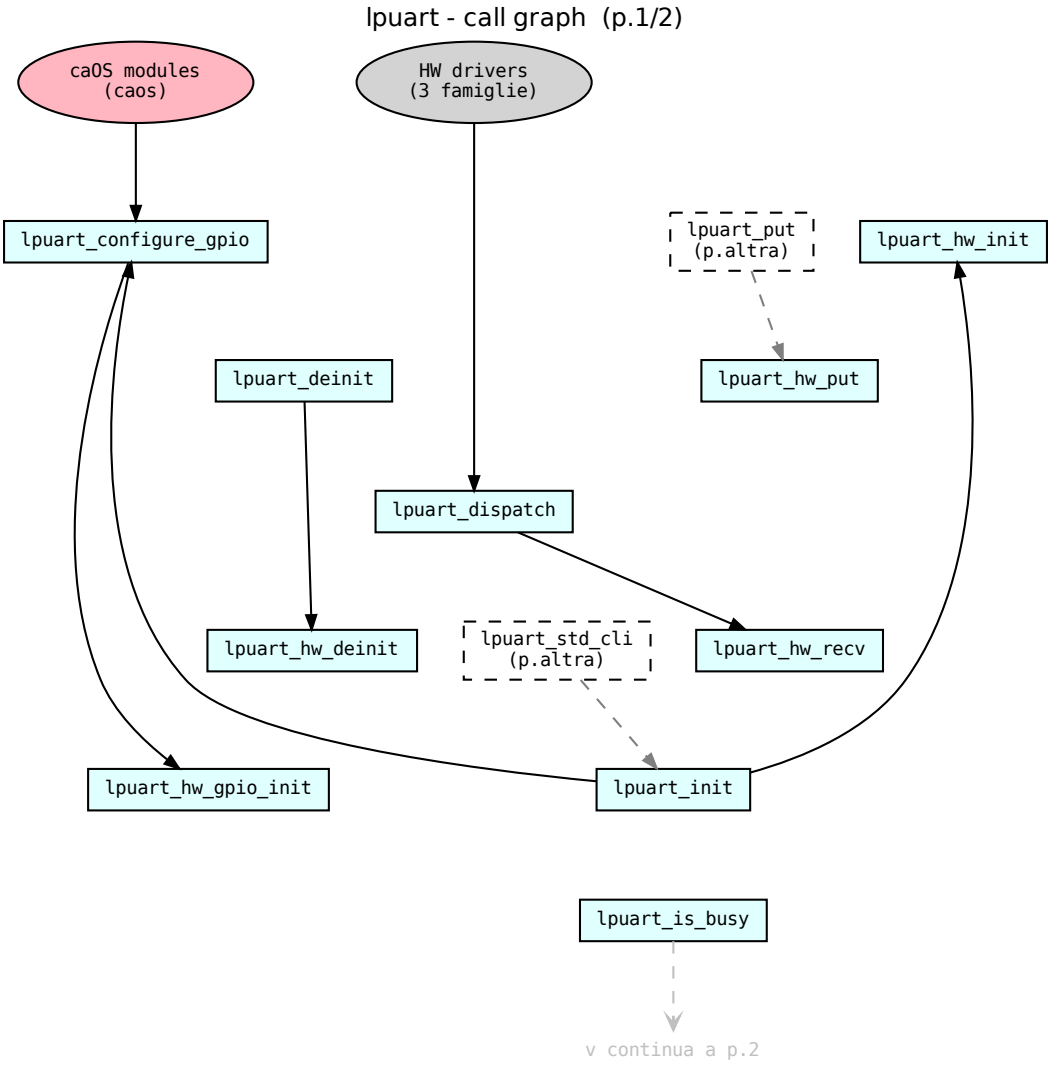


Pagina 2/2

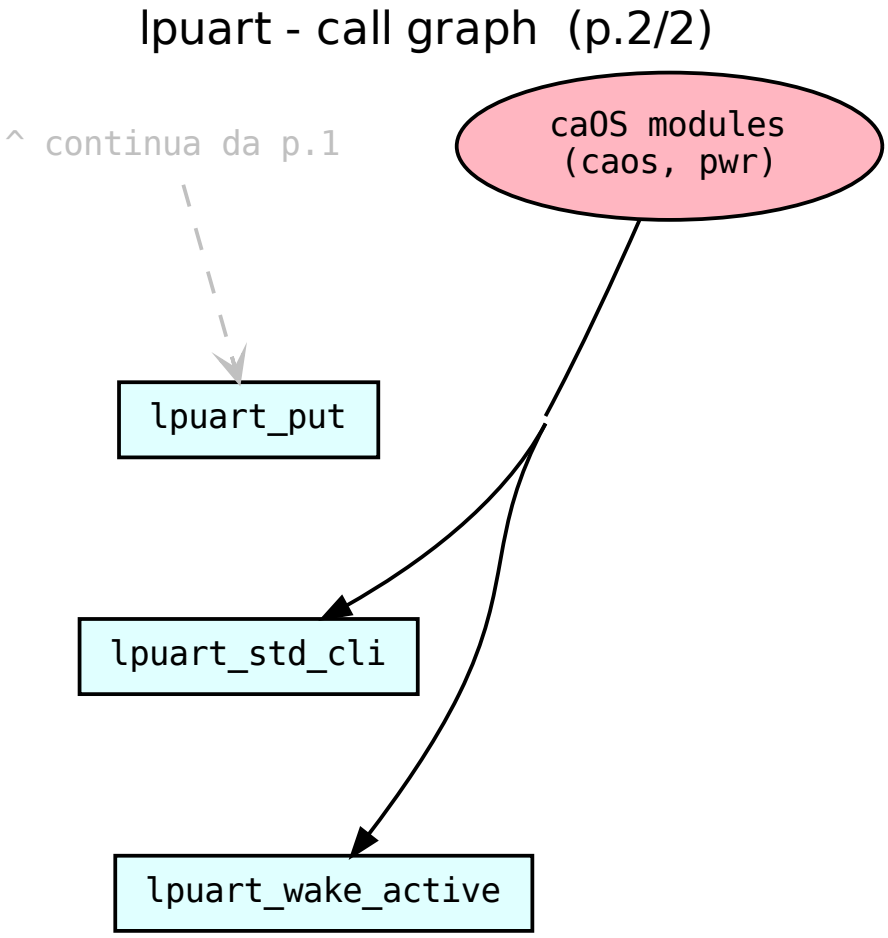

lptim - call graph (p.2/2)
examples
(7 call)
^ continua da p.1
caOS modules
(pwr)
lptim_stop
lptim_init_periodic
lptim_wake_active
lptim_is_busy
lptim_start

118.48 lpuart

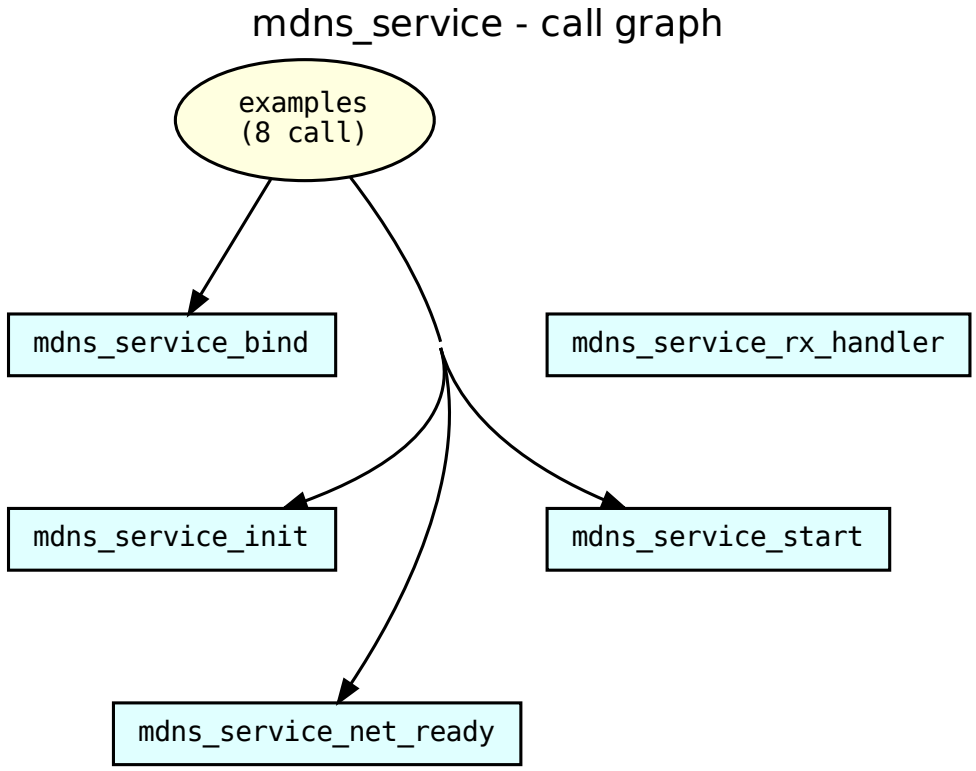
Pagina 1/2



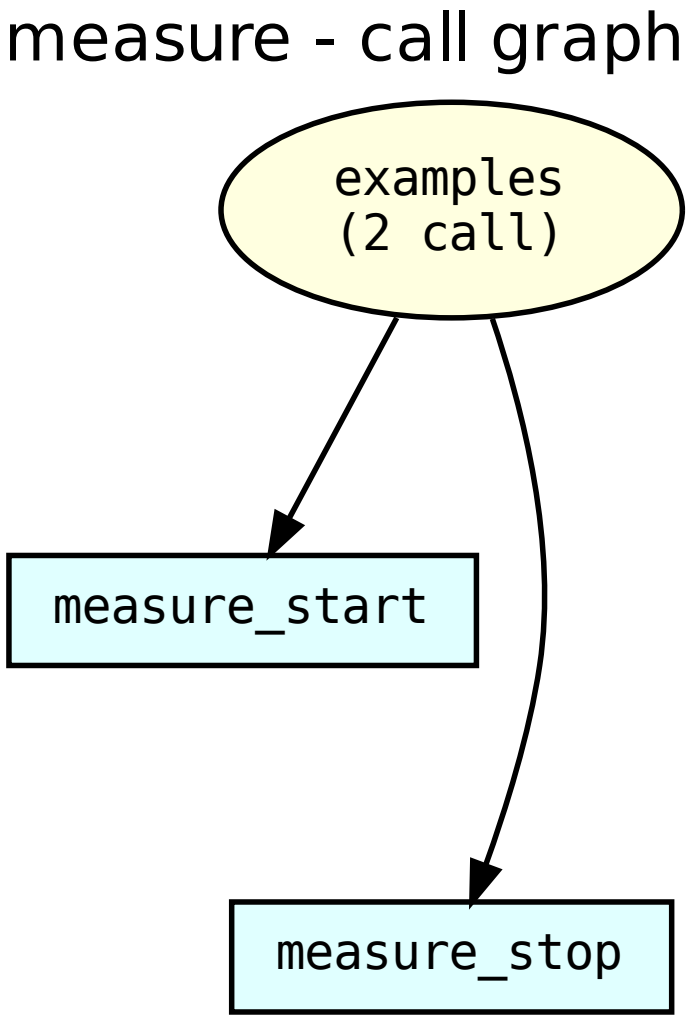
Pagina 2/2


lpuart - call graph (p.2/2)
^ continua da p.1
caOS modules
(caos, pwr)
lpuart_put
lpuart_std_cli
lpuart_wake_active

118.49 mdns_service

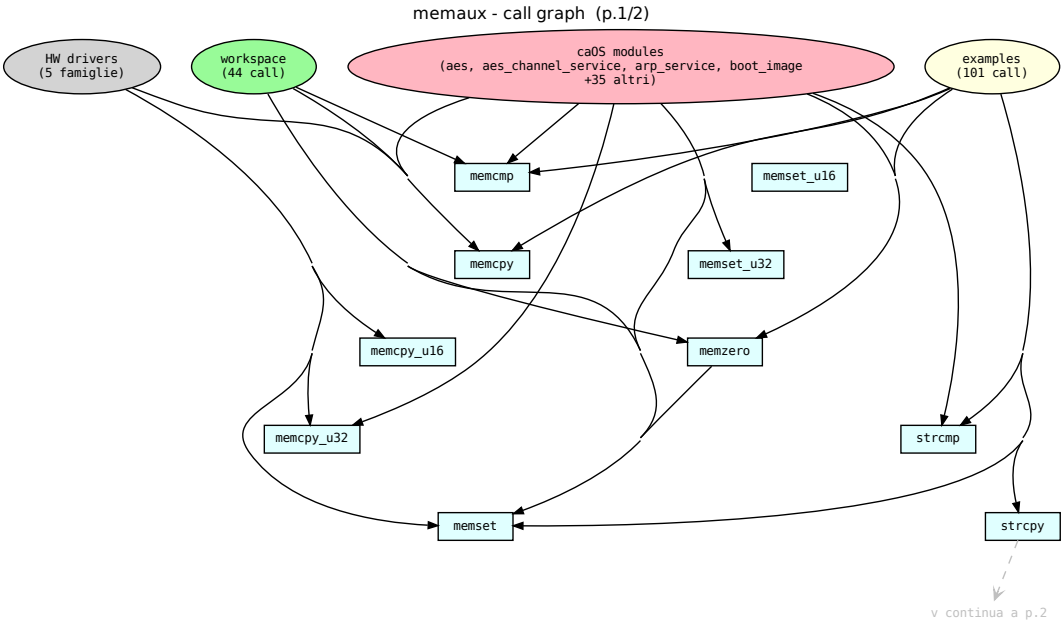


118.50 measure

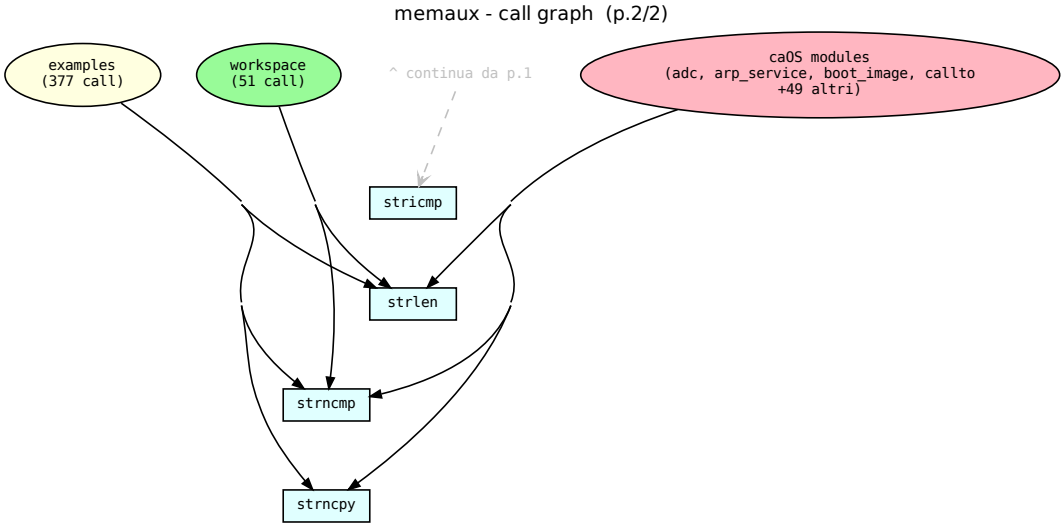


118.51 memaux

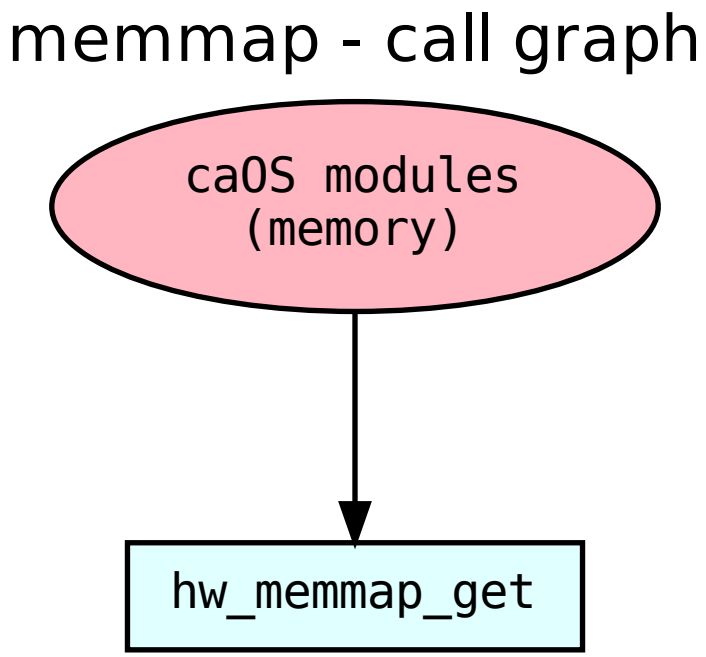
Pagina 1/2



Pagina 2/2

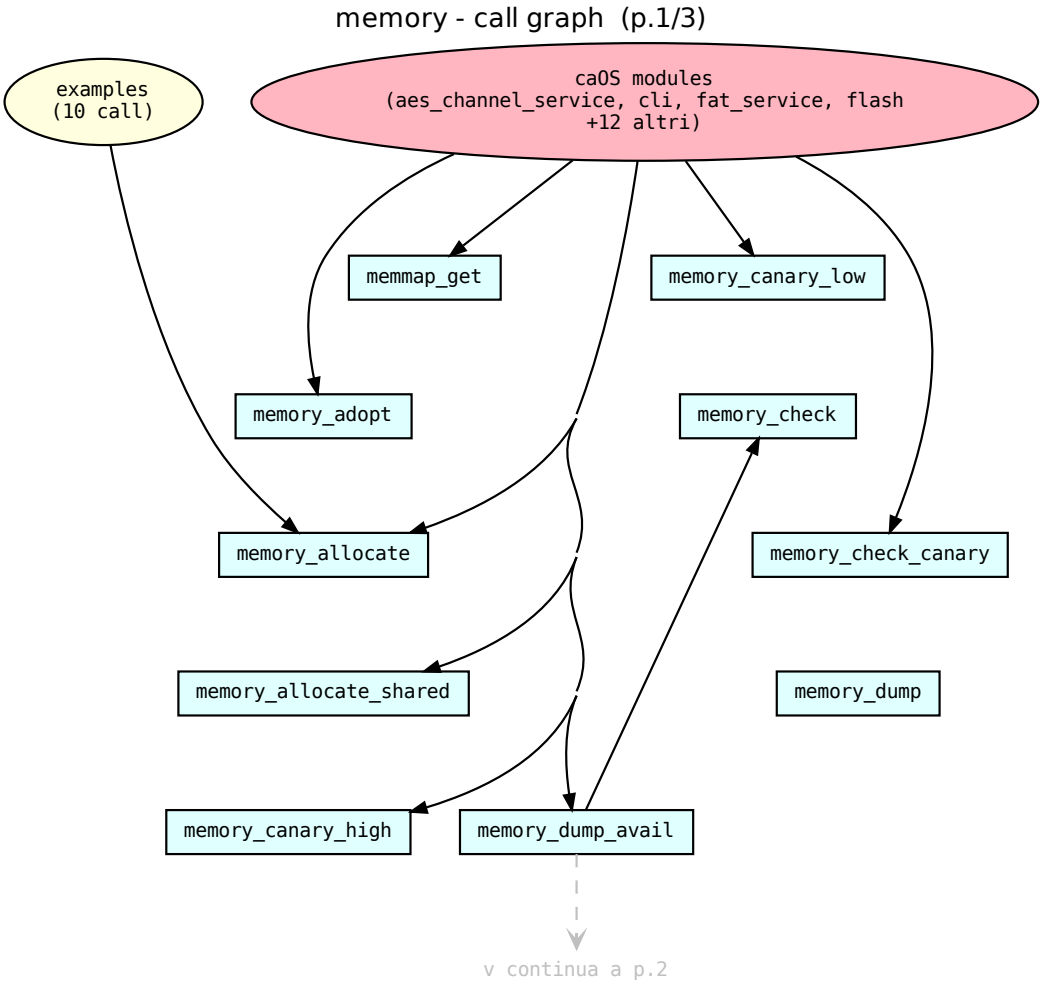

memaux - call graph (p.2/2)
examples
(377 call)
workspace
(51 call)
^ continua da p.1
caOS modules
(adc, arp_service, boot_image, callto
+49 altri)
stricmp
strlen
strncmp
strncpy

118.52 memmap

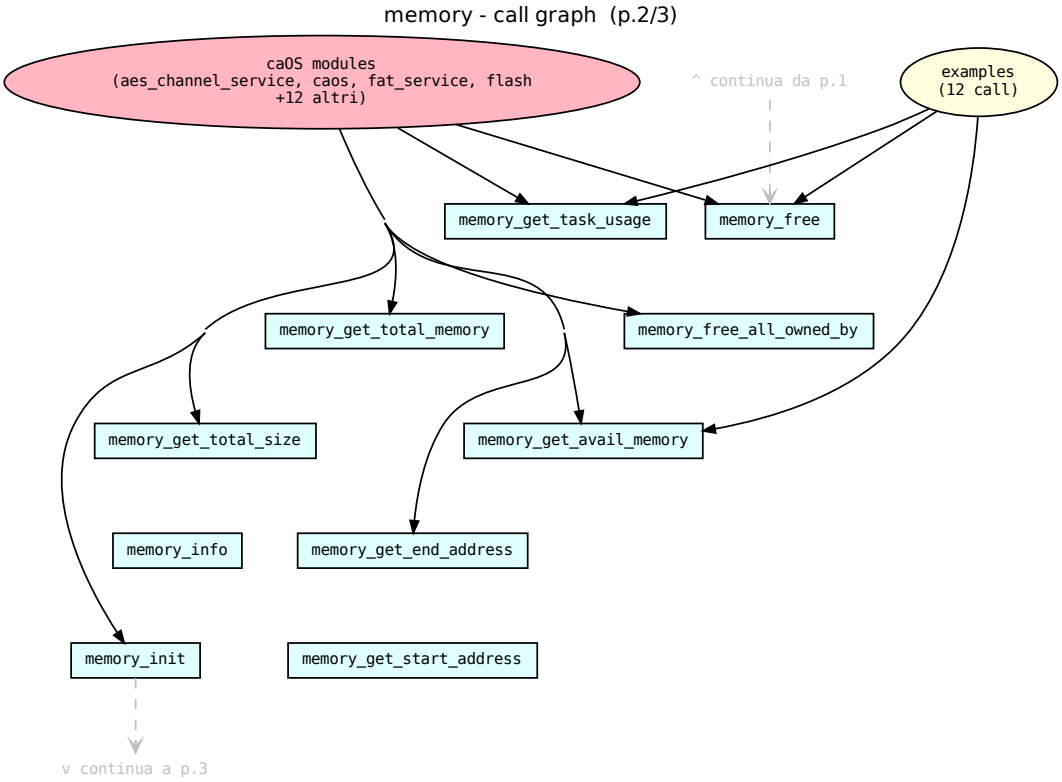


118.53 memory

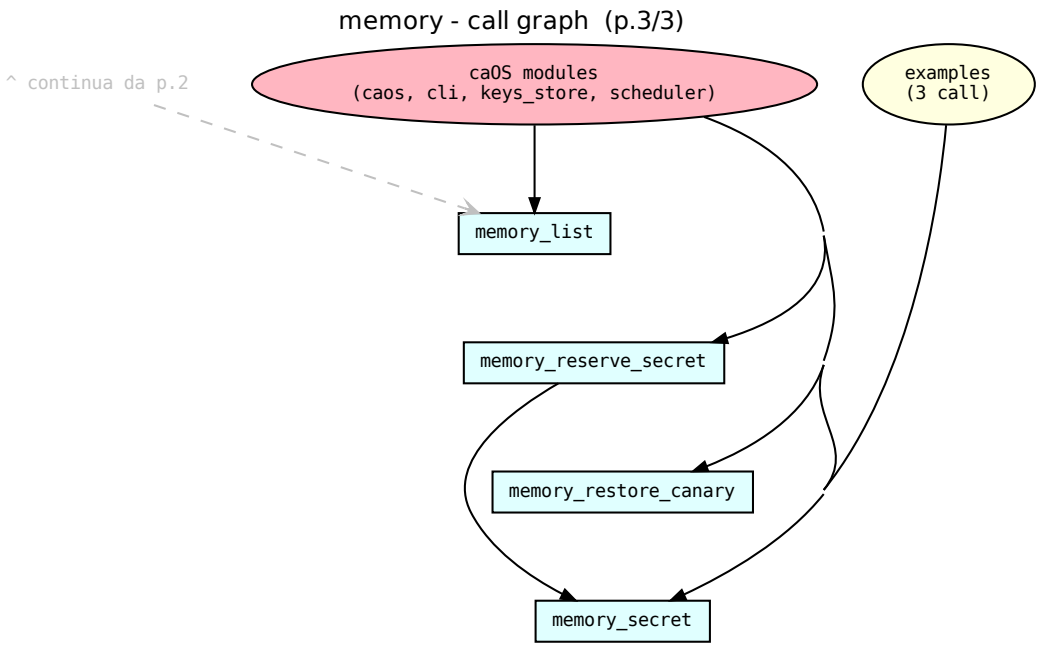
Pagina 1/3



Pagina 2/3

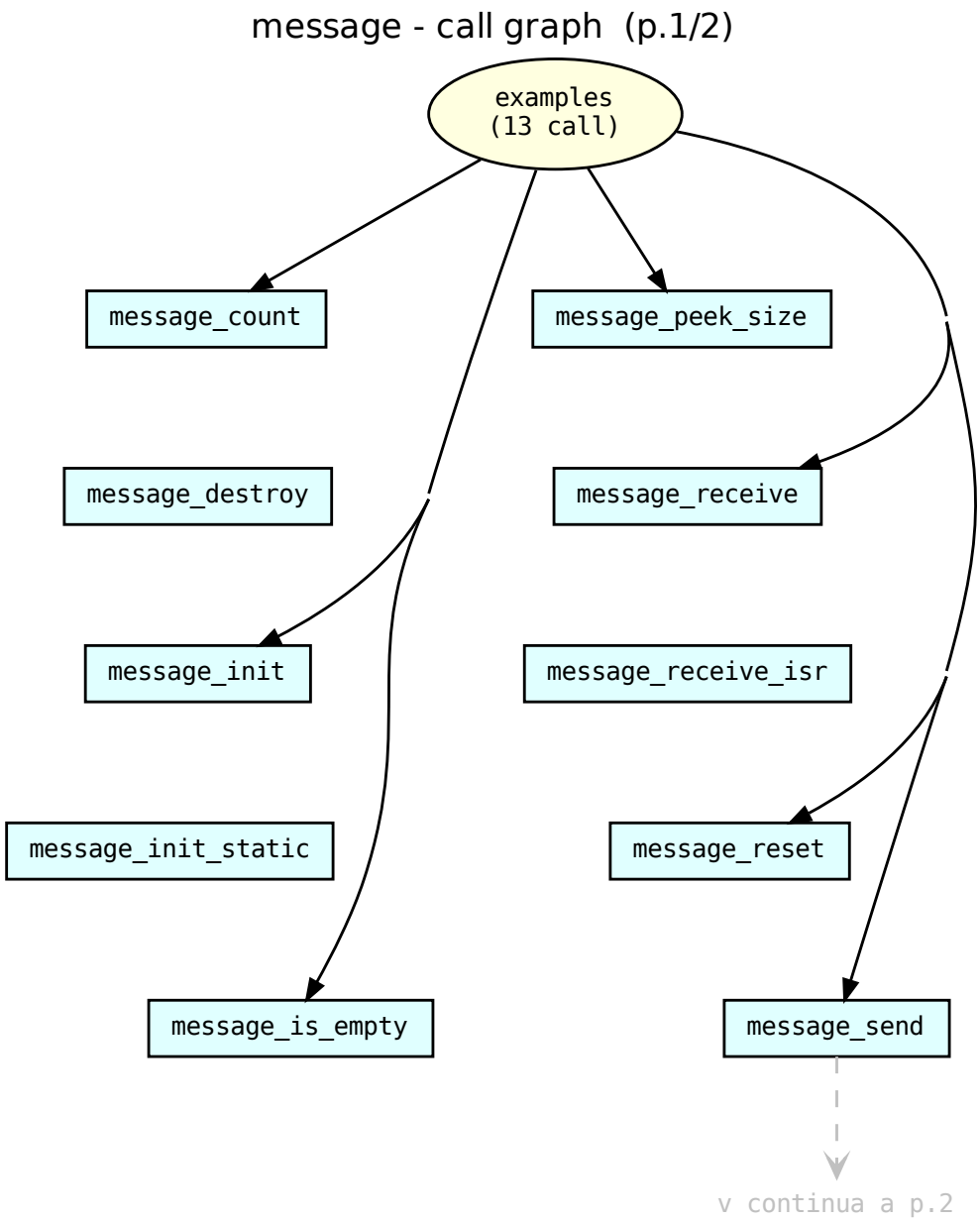

memory - call graph (p.2/3)
caOS modules
(aes_channel_service, caos, fat_service, flash
+12 altri)
^ continua da p.1
examples
(12 call)
memory_get_task_usage
memory_free
memory_get_total_memory
memory_free_all_owned_by
memory_get_total_size
memory_get_avail_memory
memory_info
memory_get_end_address
memory_init
memory_get_start_address
v continua a p.3

Pagina 3/3

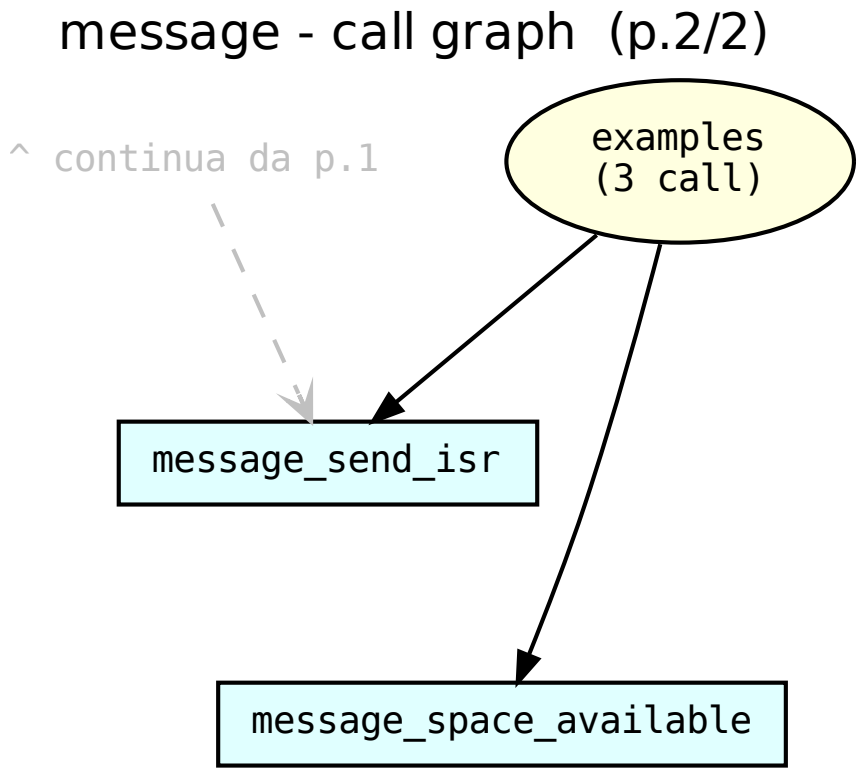

memory - call graph (p.3/3)
^ continua da p.2
caOS modules
(caos, cli, keys_store, scheduler)
examples
(3 call)
memory_list
memory_reserve_secret
memory_restore_canary
memory_secret

118.54 message

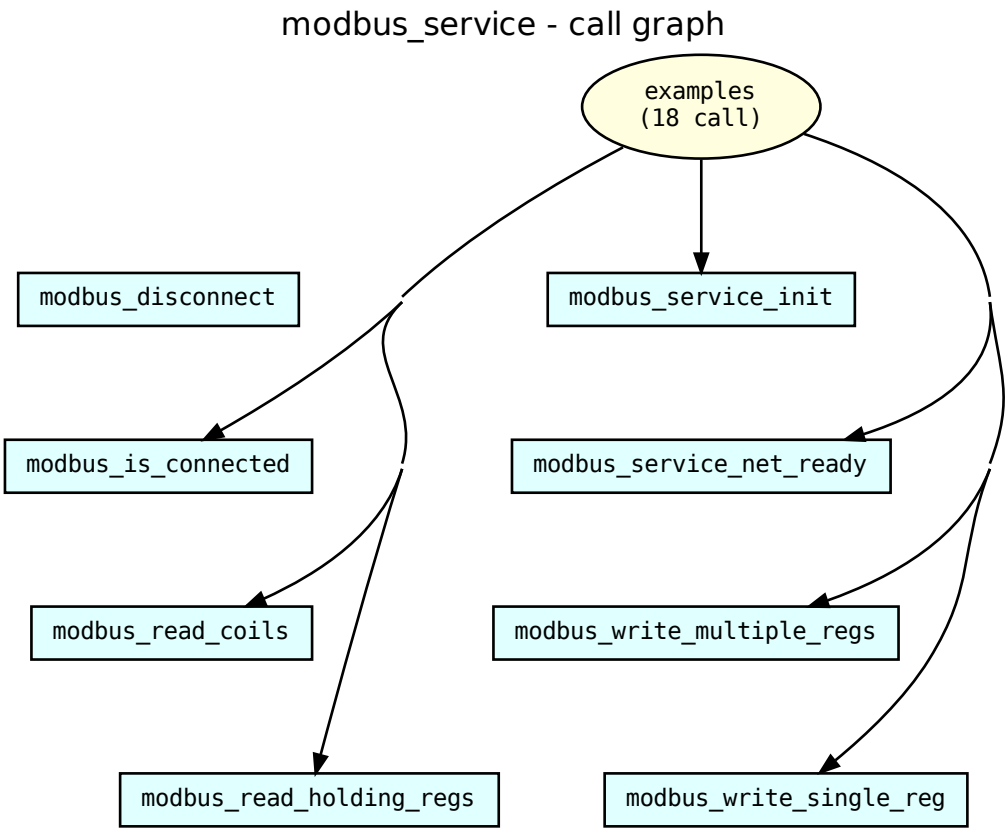
Pagina 1/2



Pagina 2/2

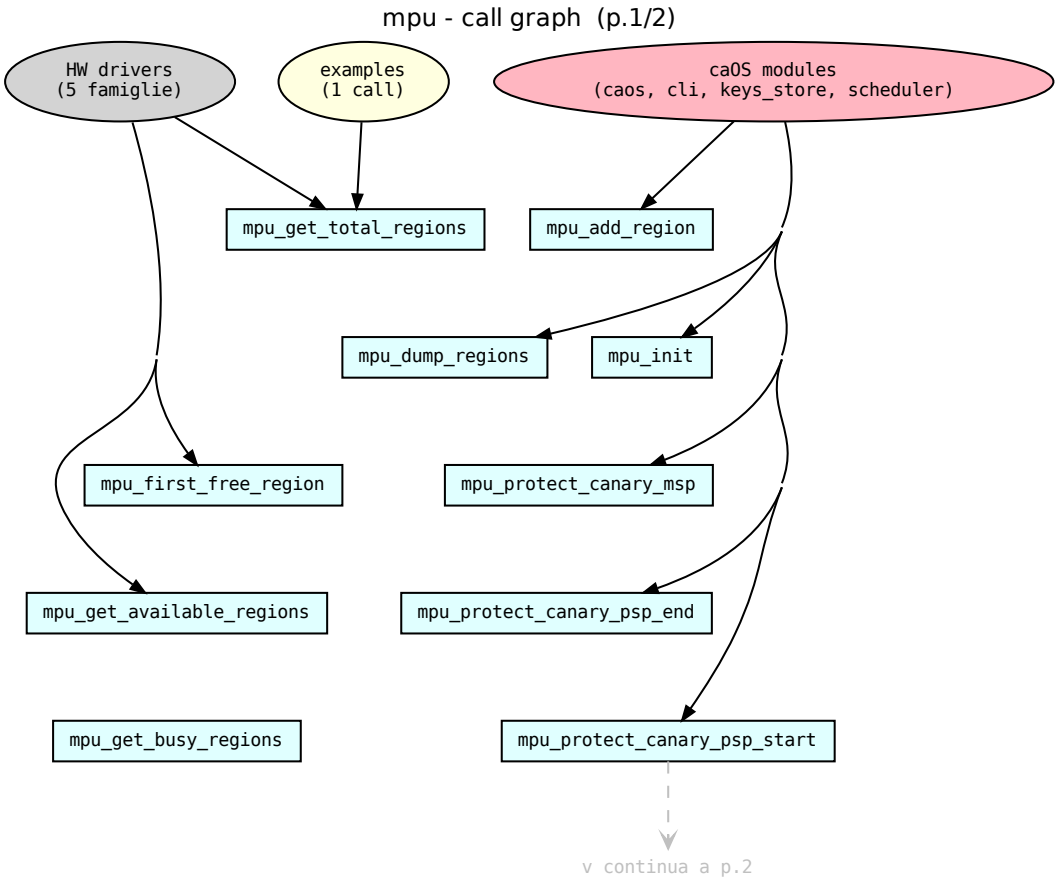

message - call graph (p.2/2)
^ continua da p.1
examples
(3 call)
message_send_isr
message_space_available

118.55 modbus_service

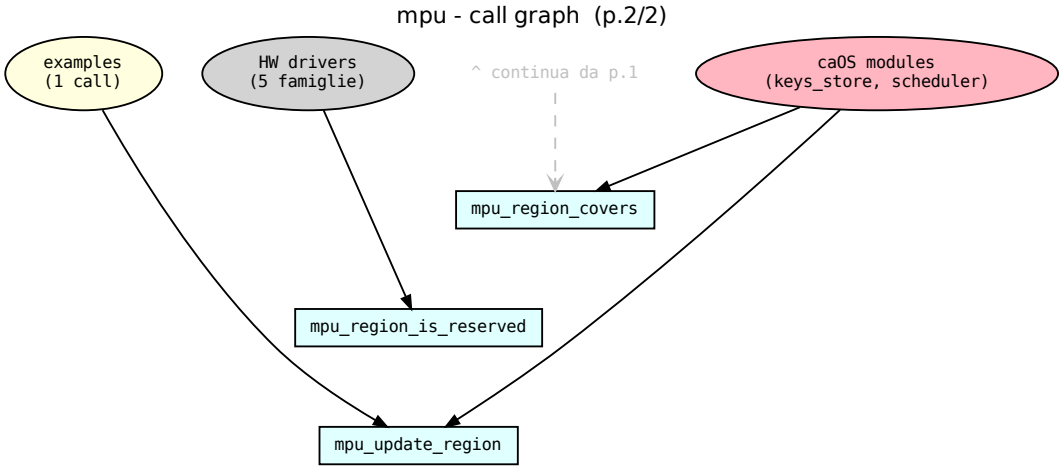


118.56 mpu

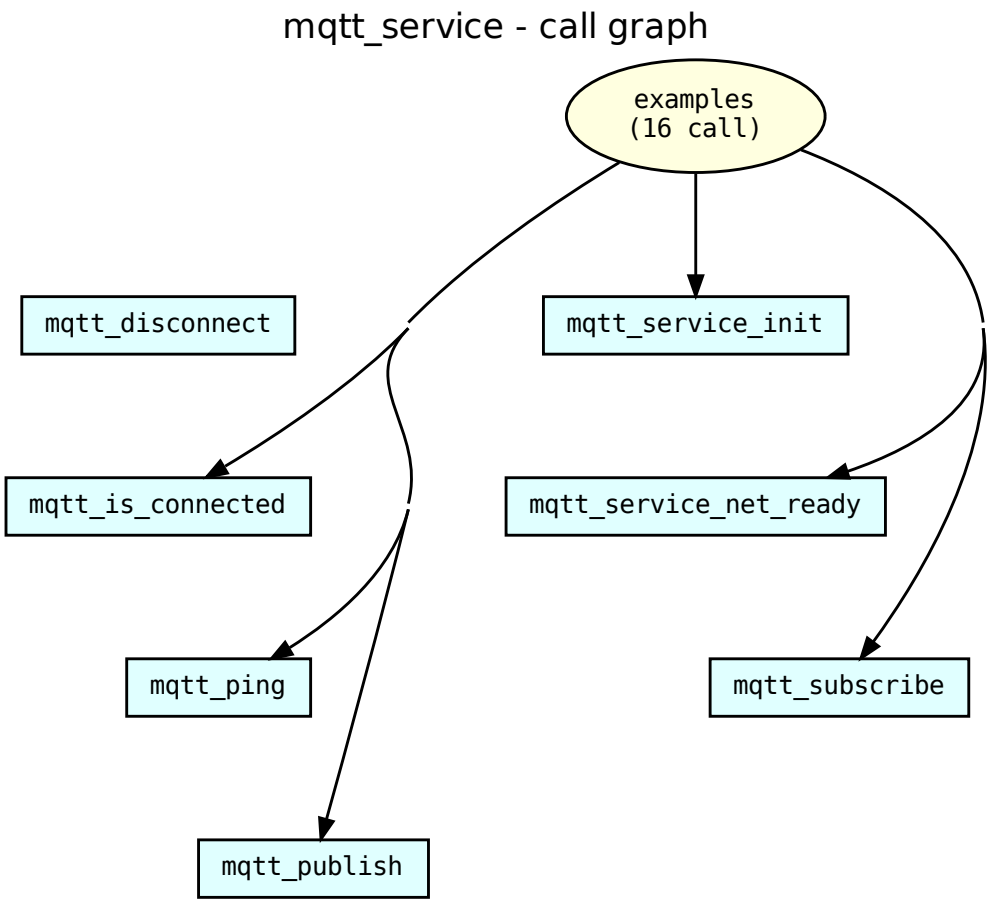
Pagina 1/2



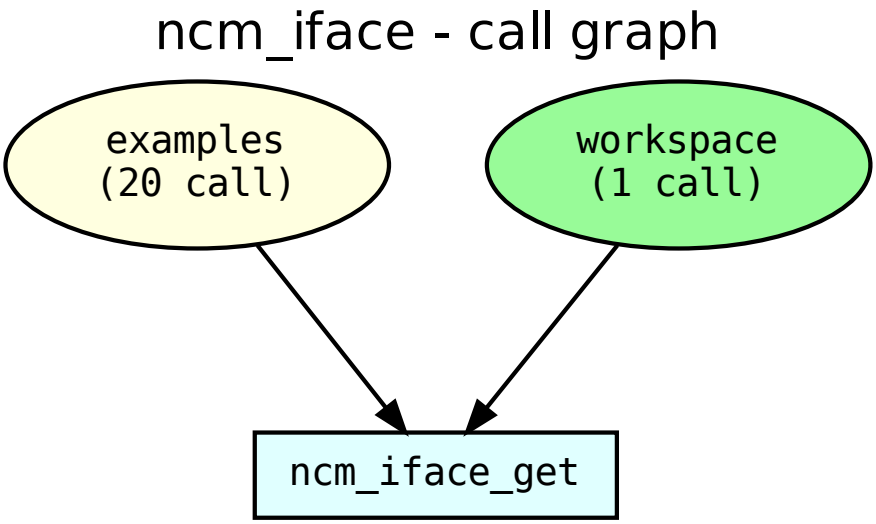
Pagina 2/2


mpu - call graph (p.2/2)
examples
(1 call)
HW drivers
(5 famiglie)
^ continua da p.1
caOS modules
(keys_store, scheduler)
mpu_region_covers
mpu_region_is_reserved
mpu_update_region

118.57 mqtt_service

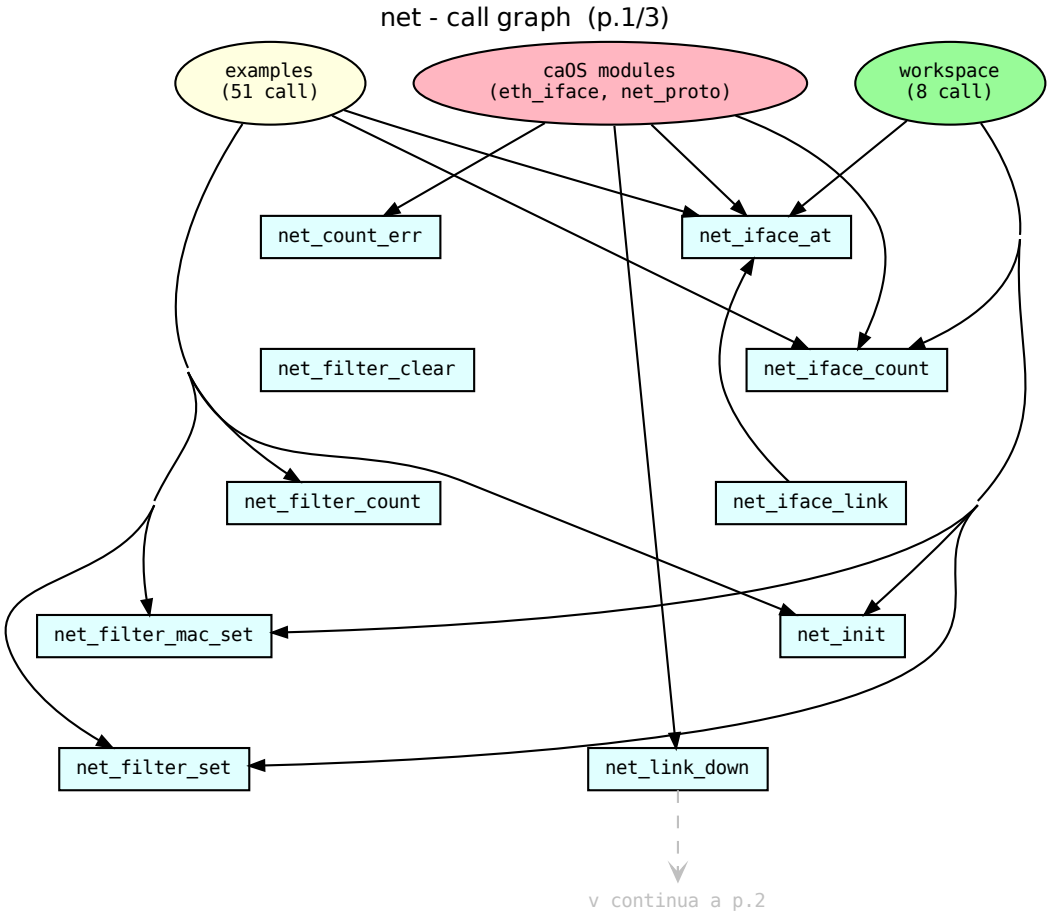


118.58 ncm_iface

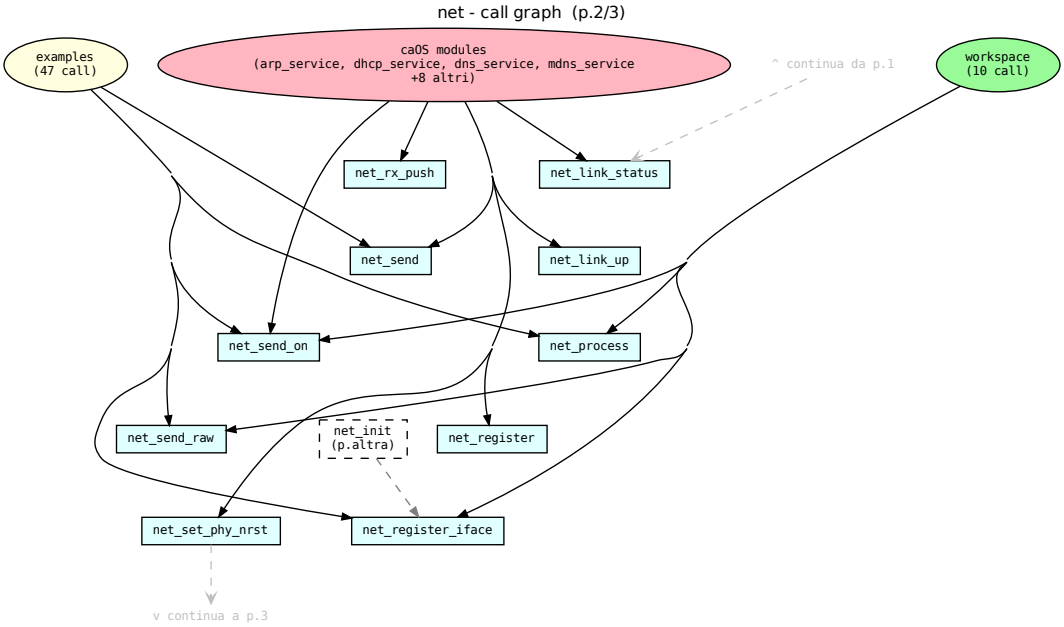


118.59 net

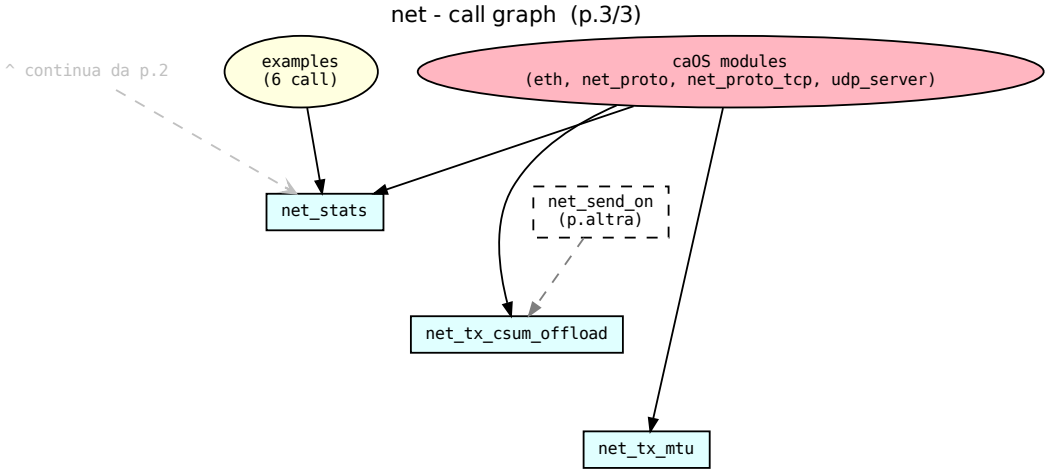
Pagina 1/3



Pagina 2/3

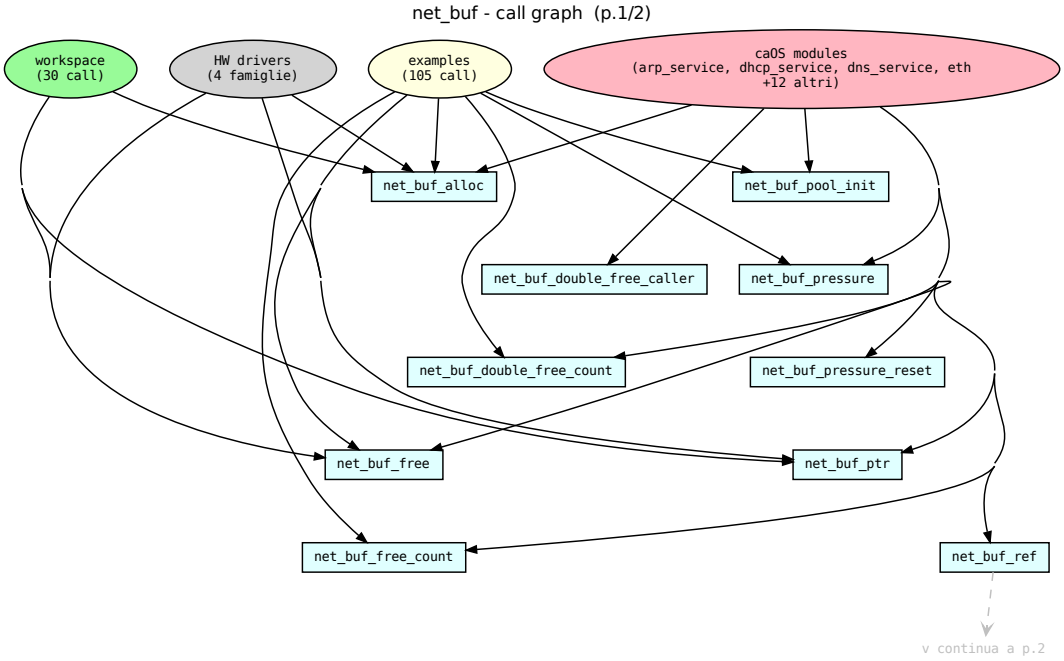

net - call graph (p.2/3)
examples
(47 call)
caOS modules
(arp_service, dhcp_service, dns_service, mdns_service
+8 altri)
^ continua da p.1
workspace
(10 call)
net_rx_push
net_link_status
net_send
net_link_up
net_send_on
net_process
net_send_raw
net_init
(p.altra)
net_register
net_set_phy_nrst
net_register_iface
v continua a p.3

Pagina 3/3

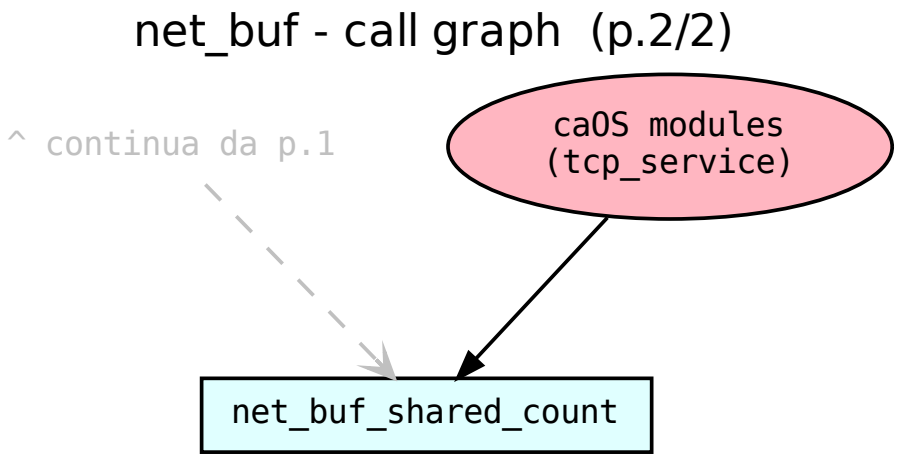

net - call graph (p.3/3)
^ continua da p.2
examples
(6 call)
caOS modules
(eth, net_proto, net_proto_tcp, udp_server)
net_stats
net_send_on
(p.altra)
net_tx_csum_offload
net_tx_mtu

118.60 net_buf

Pagina 1/2

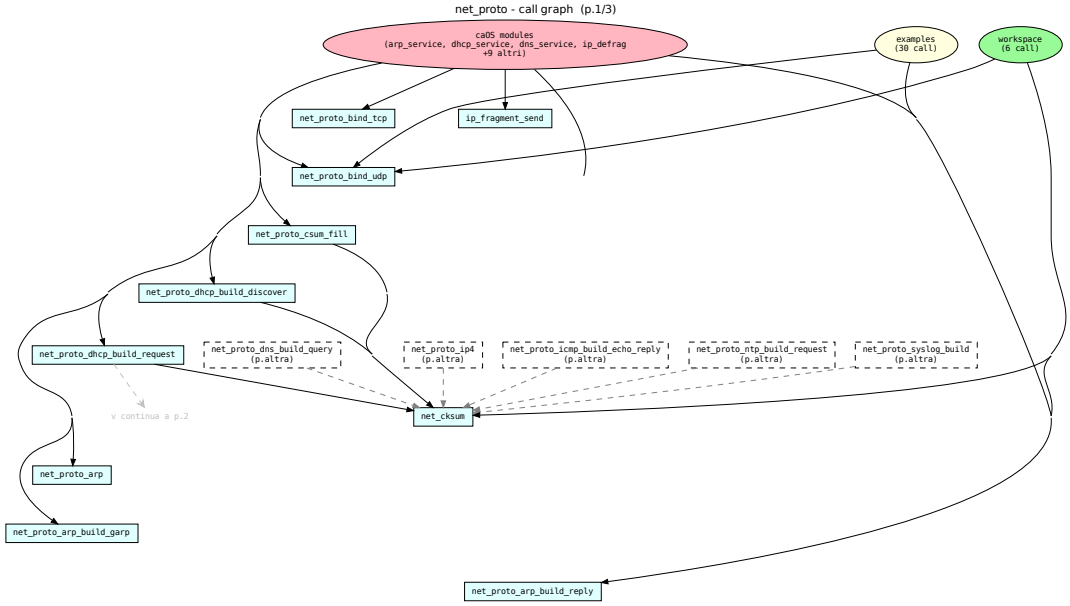


Pagina 2/2

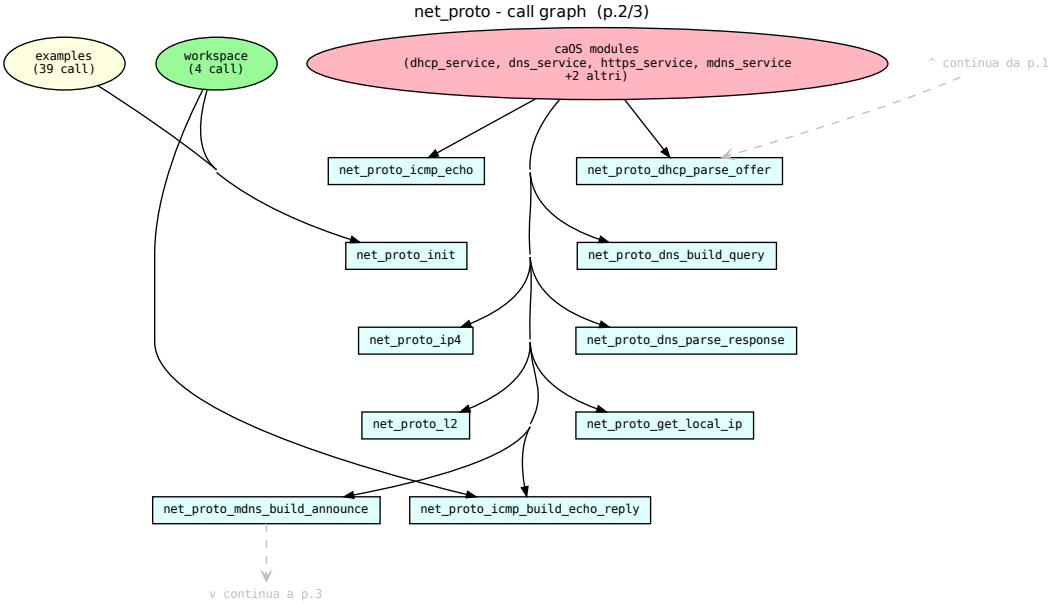

net_buf - call graph (p.2/2)
^ continua da p.1
caOS modules
(tcp_service)
net_buf_shared_count

118.61 net_proto

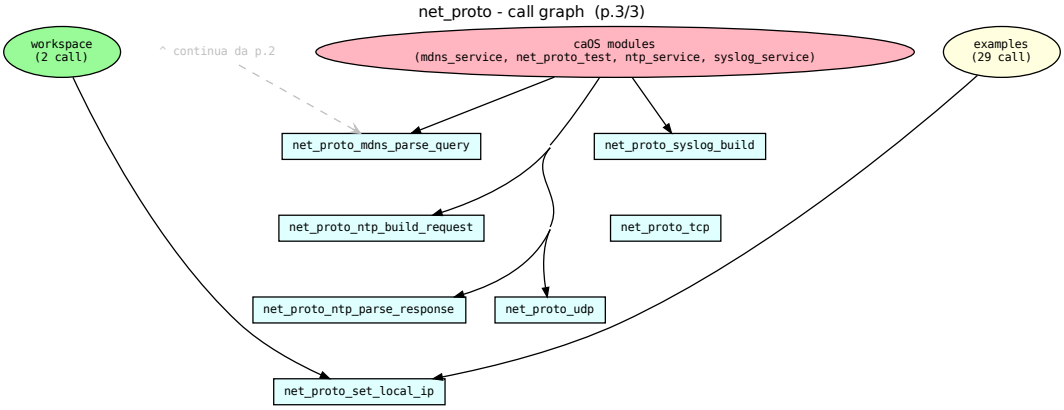
Pagina 1/3



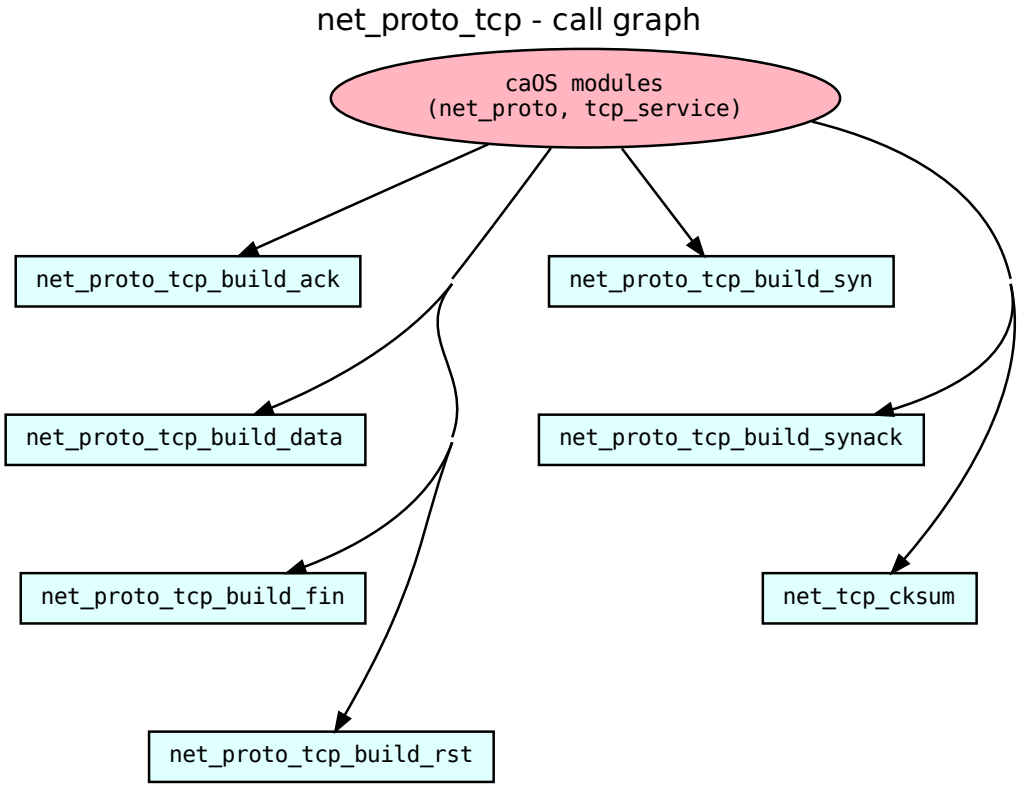
Pagina 2/3


net_proto - call graph (p.2/3)
examples
(39 call)
workspace
(4 call)
caOS modules
(dhcp_service, dns_service, https_service, mdns_service
+2 altri)
^ continua da p.1
net_proto_icmp_echo
net_proto_dhcp_parse_offer
net_proto_init
net_proto_dns_build_query
net_proto_ip4
net_proto_dns_parse_response
net_proto_l2
net_proto_get_local_ip
net_proto_mdns_build_announce
net_proto_icmp_build_echo_reply
v continua a p.3

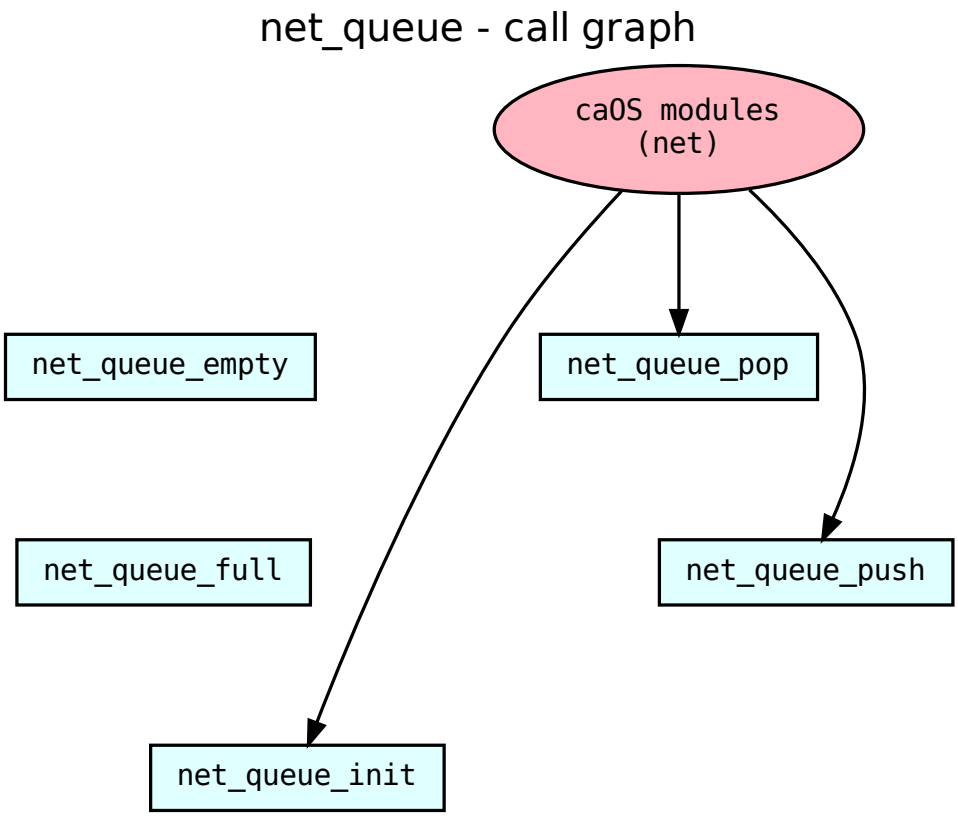
Pagina 3/3


net_proto - call graph (p.3/3)
workspace
(2 call)
^ continua da p.2
caOS modules
(mdns_service, net_proto_test, ntp_service, syslog_service)
examples
(29 call)
net_proto_mdns_parse_query
net_proto_syslog_build
net_proto_ntp_build_request
net_proto_tcp
net_proto_ntp_parse_response
net_proto_udp
net_proto_set_local_ip

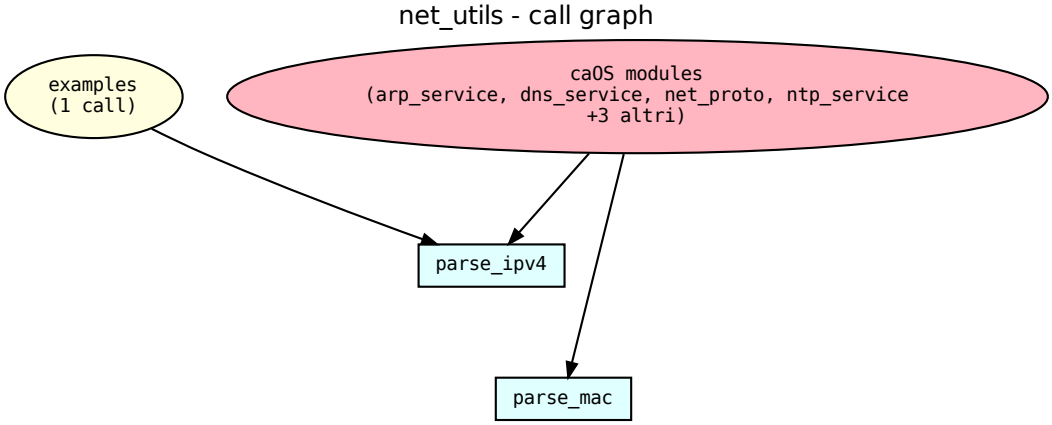
118.62 net_proto_tcp



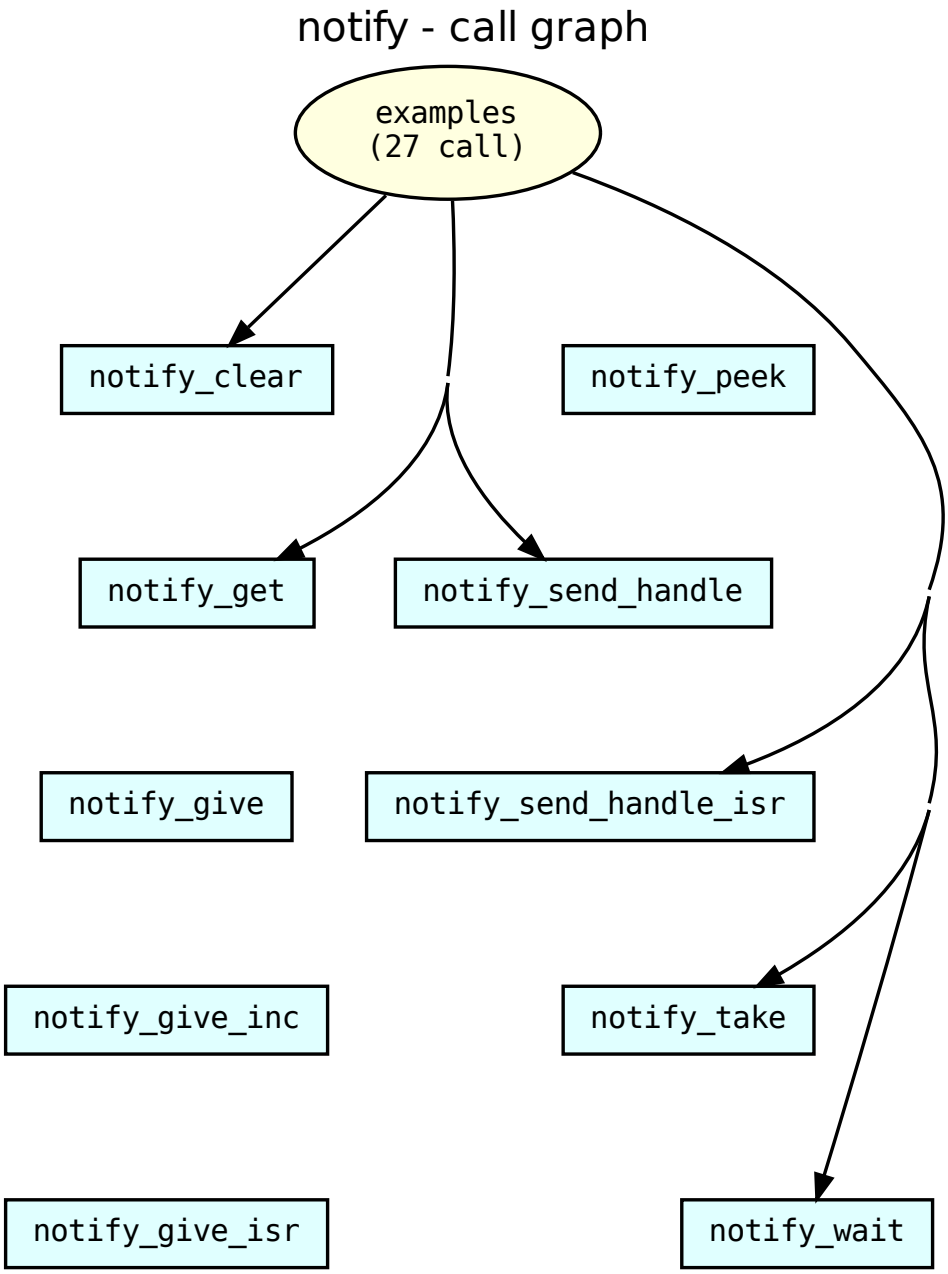
118.63 net_queue



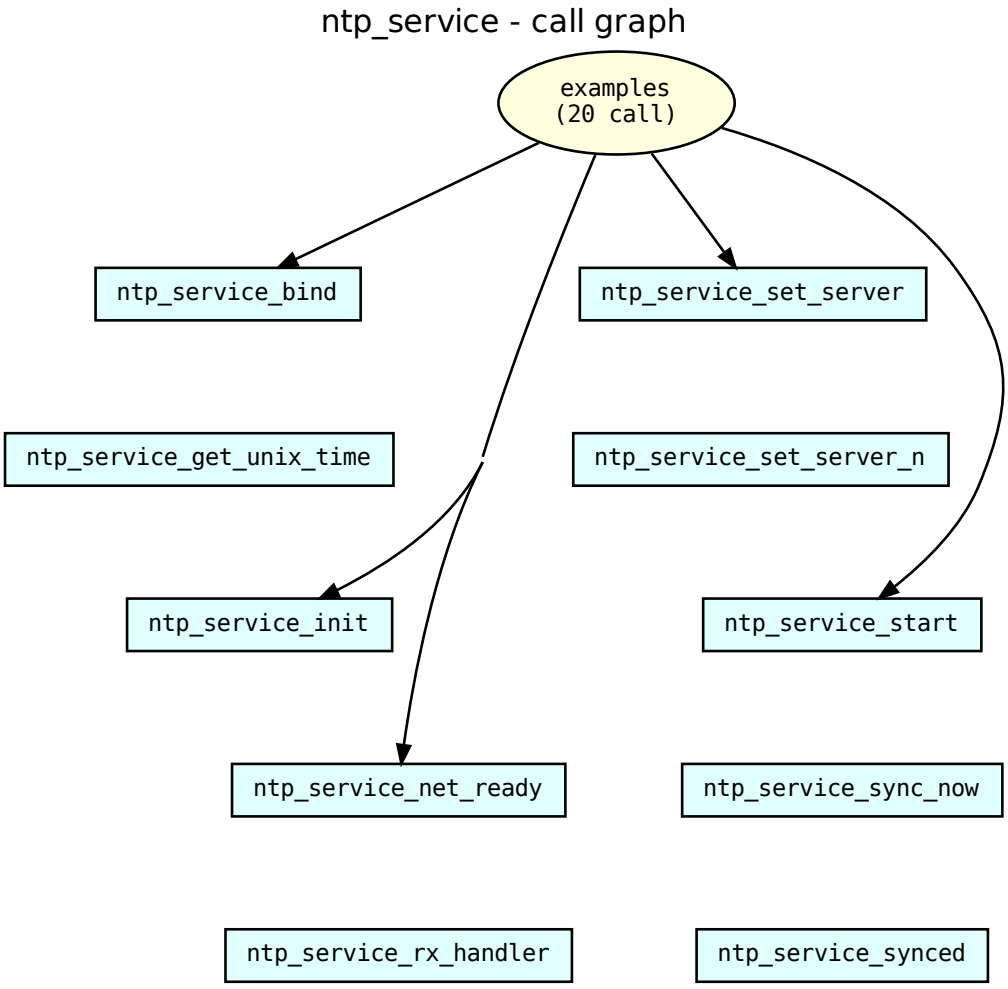
118.64 net_utils



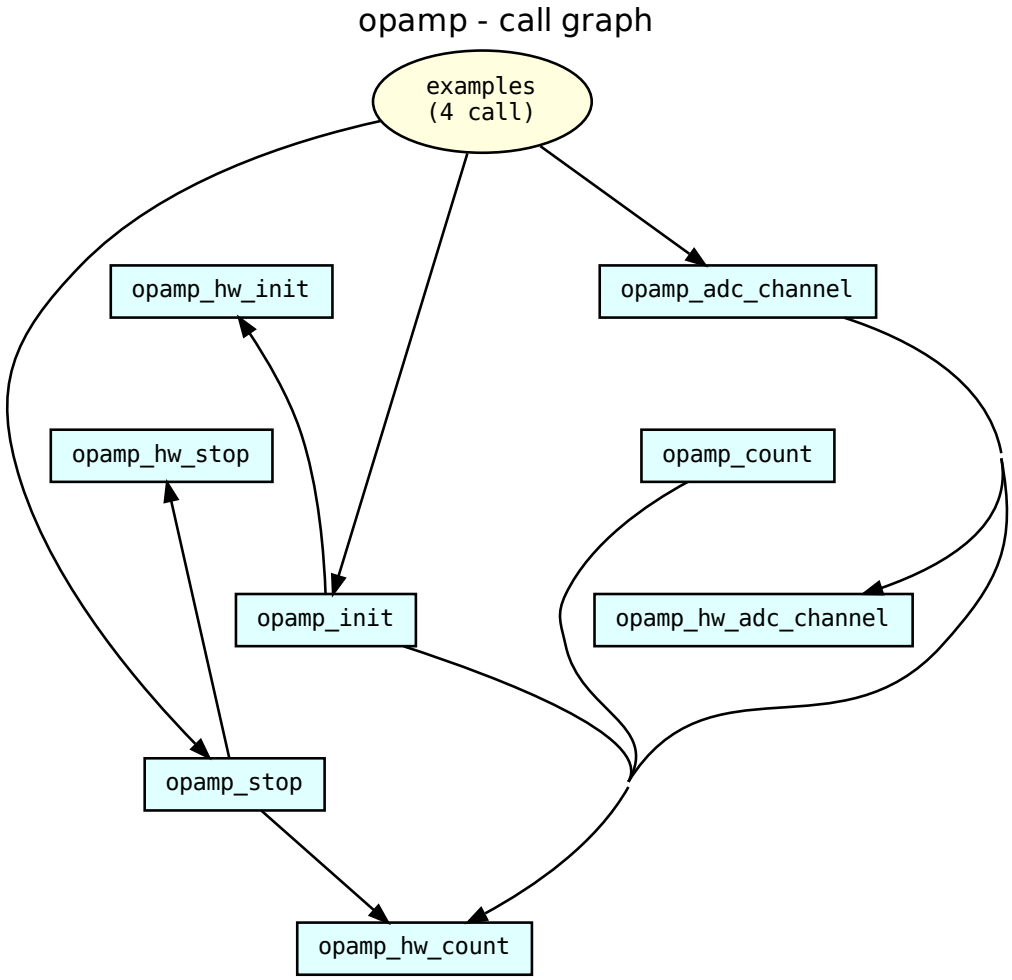
118.65 `notify`



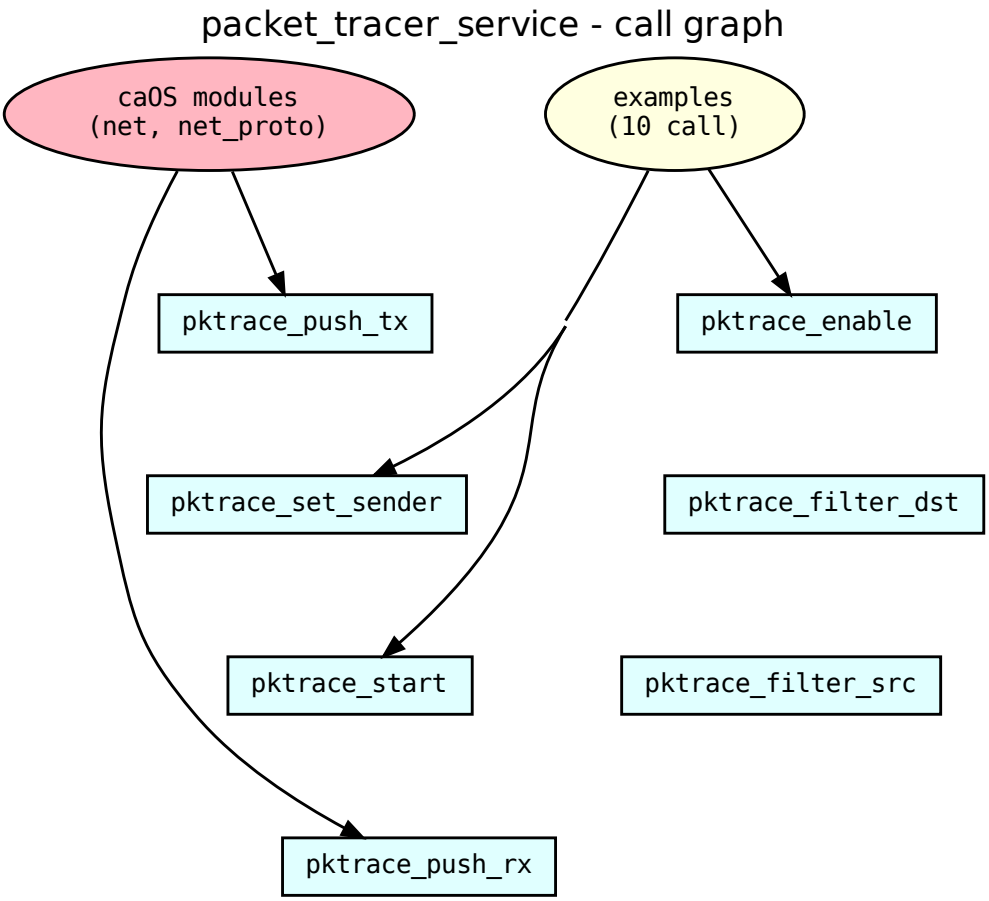
118.66 ntp_service



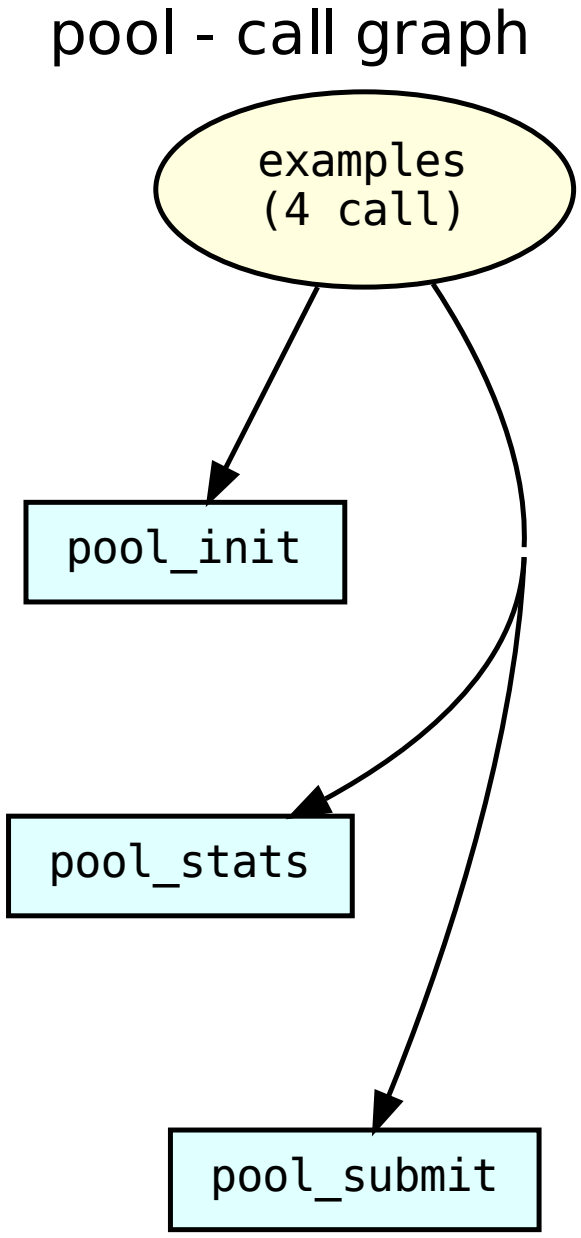
118.67 opamp


opamp - call graph
examples
(4 call)
opamp_hw_init
opamp_adc_channel
opamp_hw_stop
opamp_count
opamp_init
opamp_hw_adc_channel
opamp_stop
opamp_hw_count

118.68 packet_tracer_service

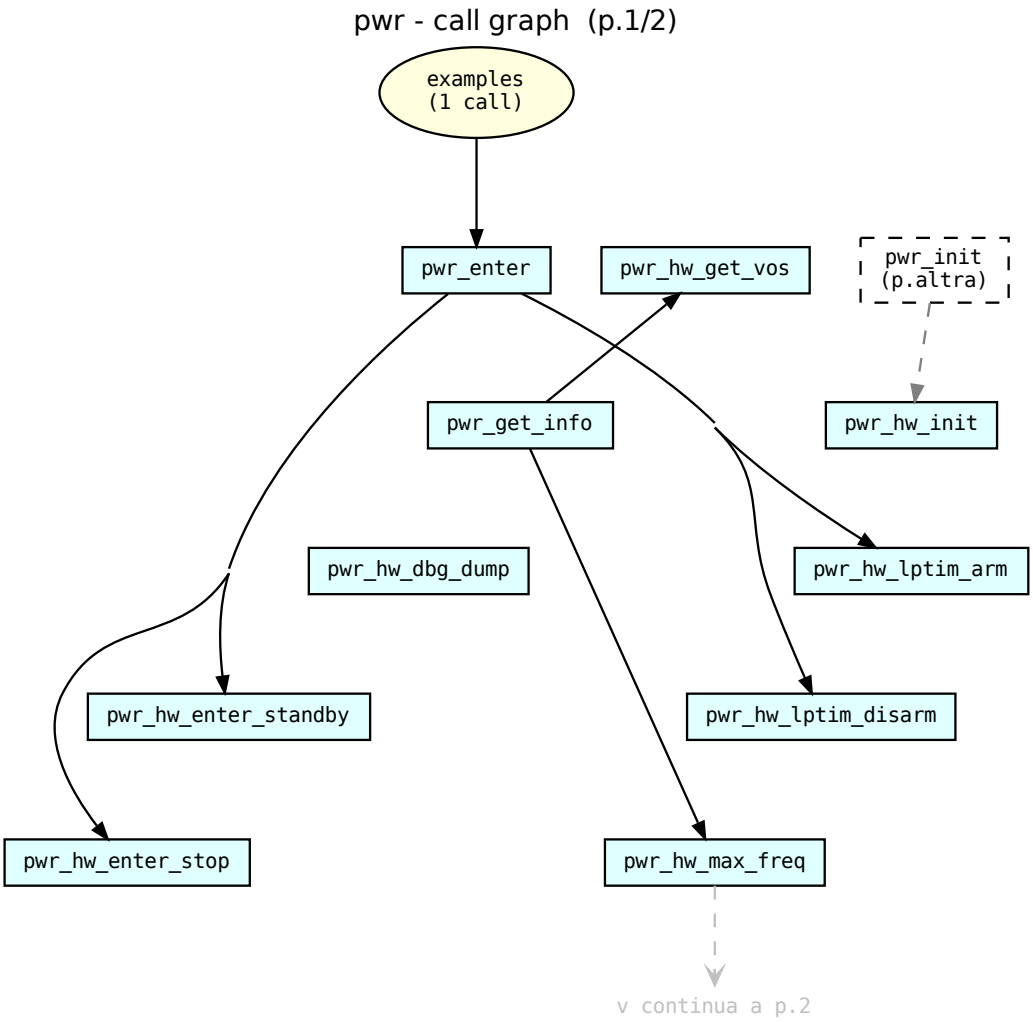


118.69 `pool`

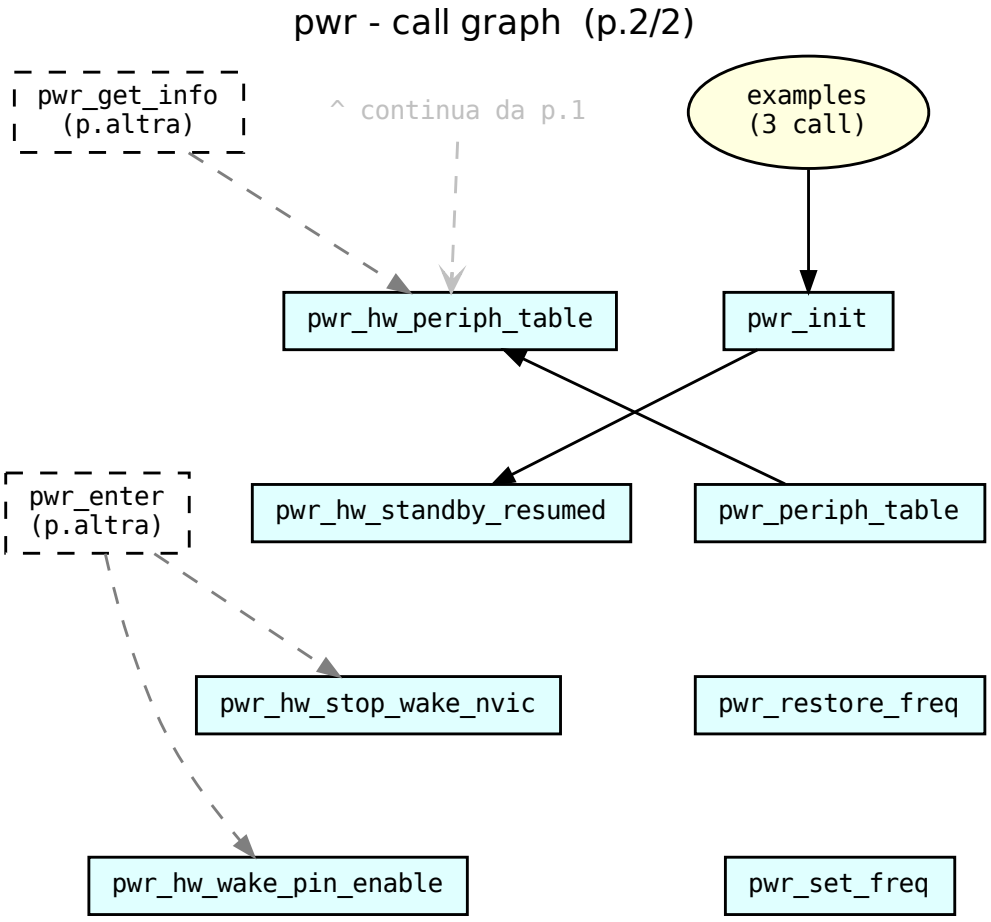

pool - call graph
examples
(4 call)
pool_init
pool_stats
pool_submit

118.70 pwr

Pagina 1/2

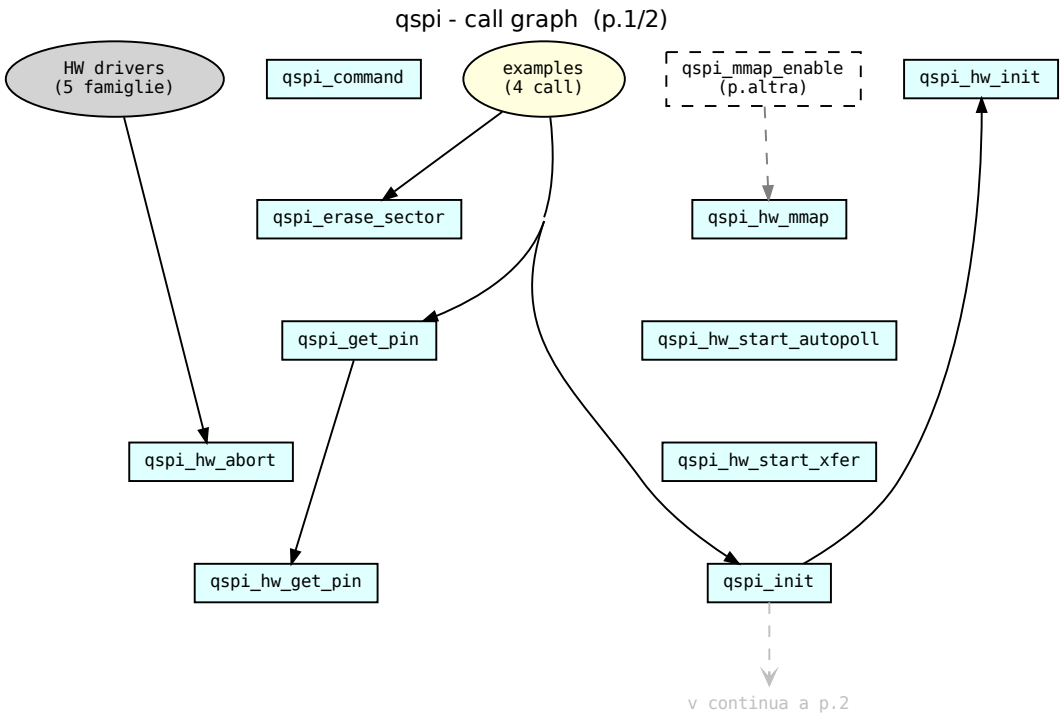


Pagina 2/2

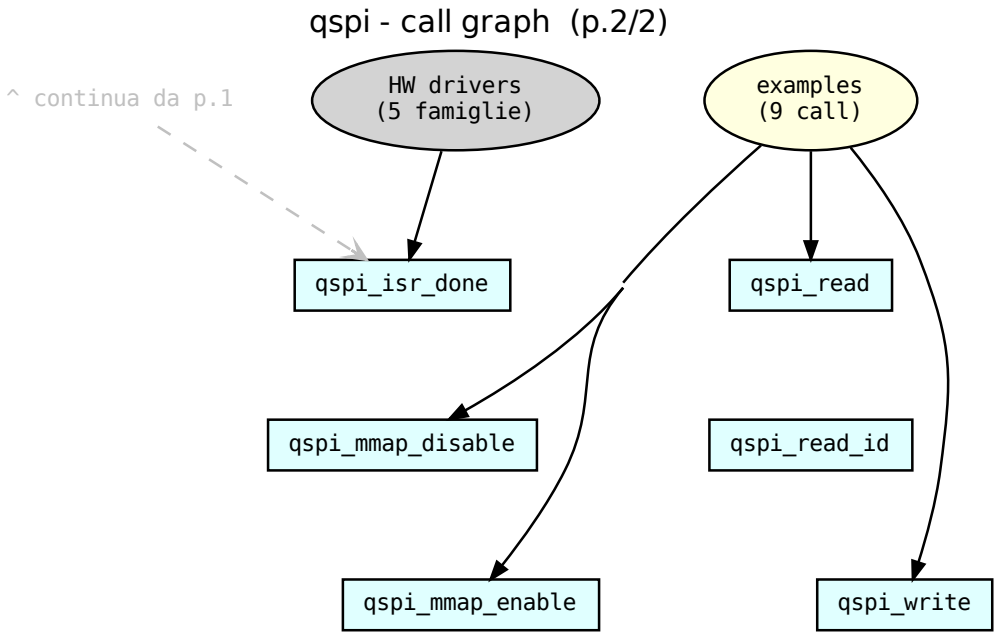

pwr - call graph (p.2/2)
pwr_get_info
(p.altra)
^ continua da p.1
examples
(3 call)
pwr_hw_periph_table
pwr_init
pwr_enter
(p.altra)
pwr_hw_standby_resumed
pwr_periph_table
pwr_hw_stop_wake_nvic
pwr_restore_freq
pwr_hw_wake_pin_enable
pwr_set_freq

118.71 qspi

Pagina 1/2

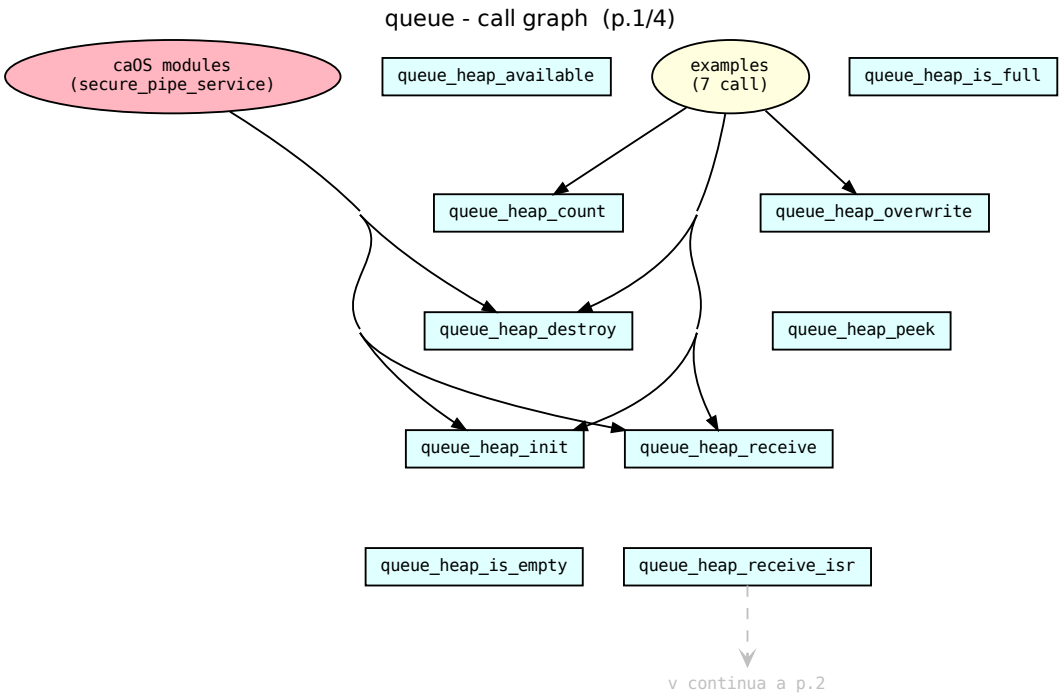


Pagina 2/2

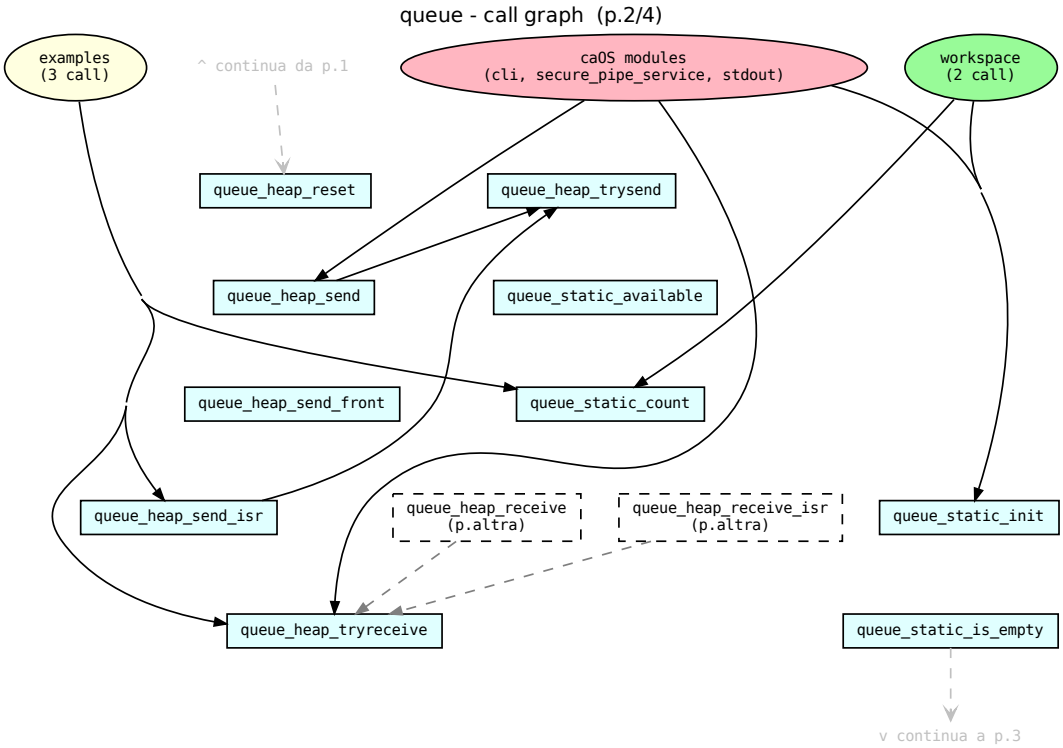

qspi - call graph (p.2/2)
^ continua da p.1
HW drivers
(5 famiglie)
examples
(9 call)
qspi_isr_done
qspi_read
qspi_mmap_disable
qspi_read_id
qspi_mmap_enable
qspi_write

118.72 queue

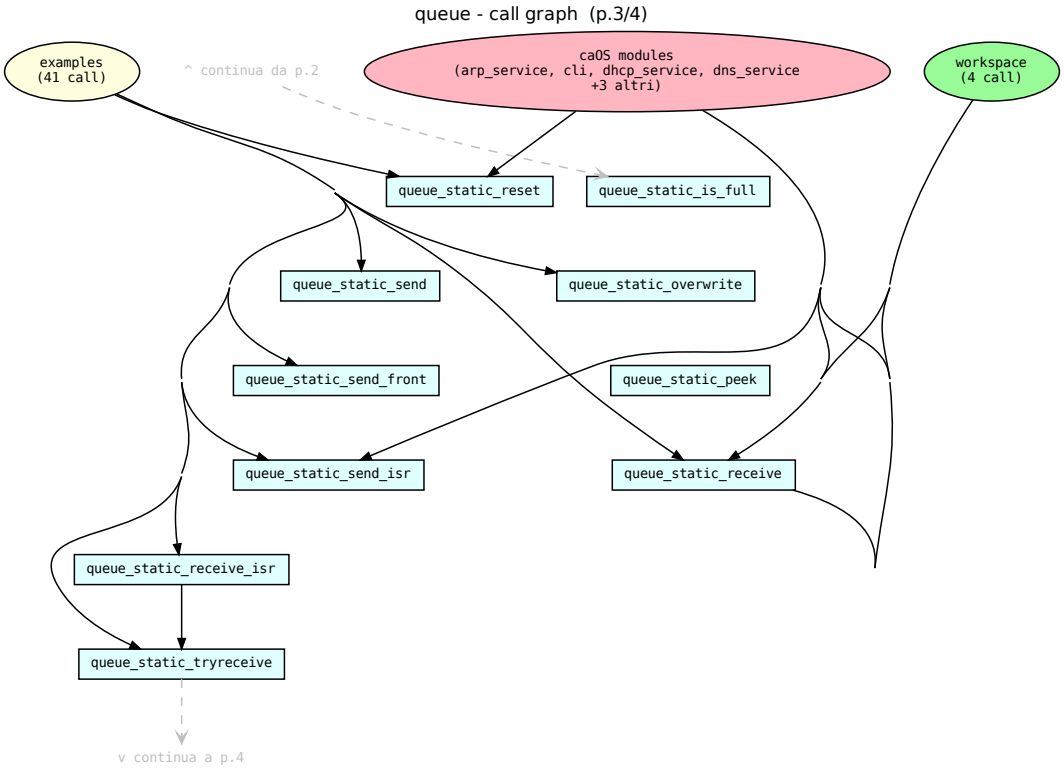
Pagina 1/4



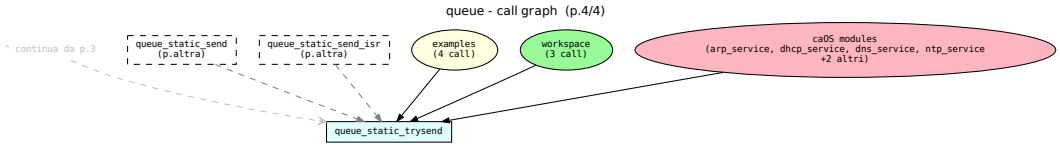
Pagina 2/4


queue - call graph (p.2/4)
examples
(3 call)
^ continua da p.1
caOS modules
(cli, secure_pipe_service, stdout)
workspace
(2 call)
queue_heap_reset
queue_heap_trysend
queue_heap_send
queue_static_available
queue_heap_send_front
queue_static_count
queue_heap_send_isr
queue_heap_receive
(p.altra)
queue_heap_receive_isr
(p.altra)
queue_static_init
queue_heap_tryreceive
queue_static_is_empty
v continua a p.3

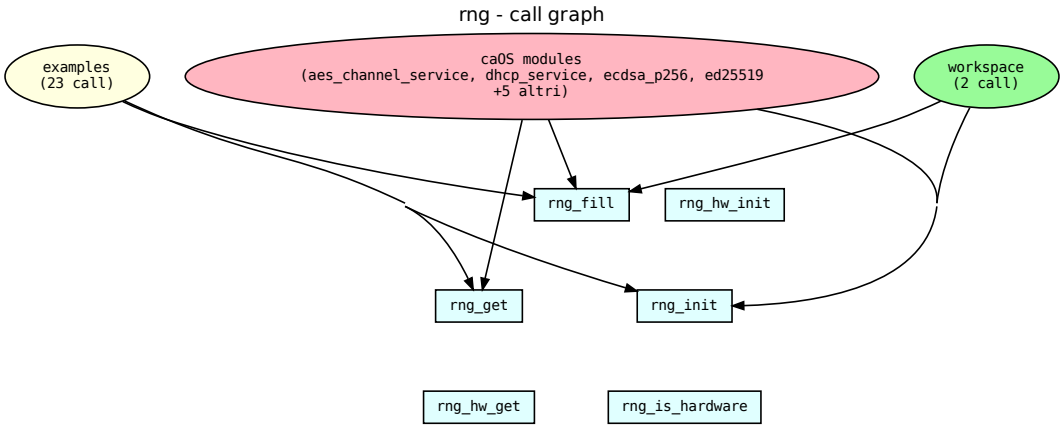
Pagina 3/4


queue - call graph (p.3/4)
examples
(41 call)
^ continua da p.2
caOS modules
(arp_service, cli, dhcp_service, dns_service
+3 altri)
workspace
(4 call)
queue_static_reset
queue_static_is_full
queue_static_send
queue_static_overwrite
queue_static_send_front
queue_static_peek
queue_static_send_isr
queue_static_receive
queue_static_receive_isr
queue_static_tryreceive
v continua a p.4

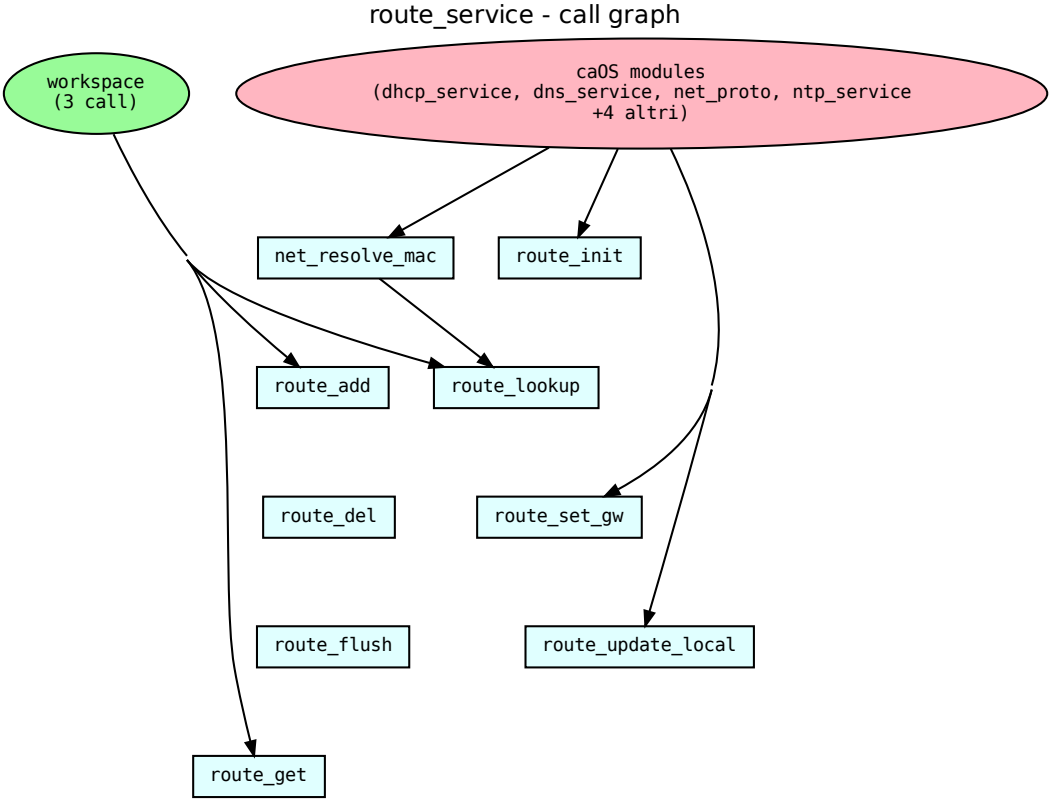
Pagina 4/4


queue - call graph (p.4/4)
^ continua da p.3
queue_static_send
(p.altra)
queue_static_send_isr
(p.altra)
examples
(4 call)
workspace
(3 call)
caOS modules
(arp_service, dhcp_service, dns_service, ntp_service
+2 altri)
queue_static_trysend

118.73 `rng`

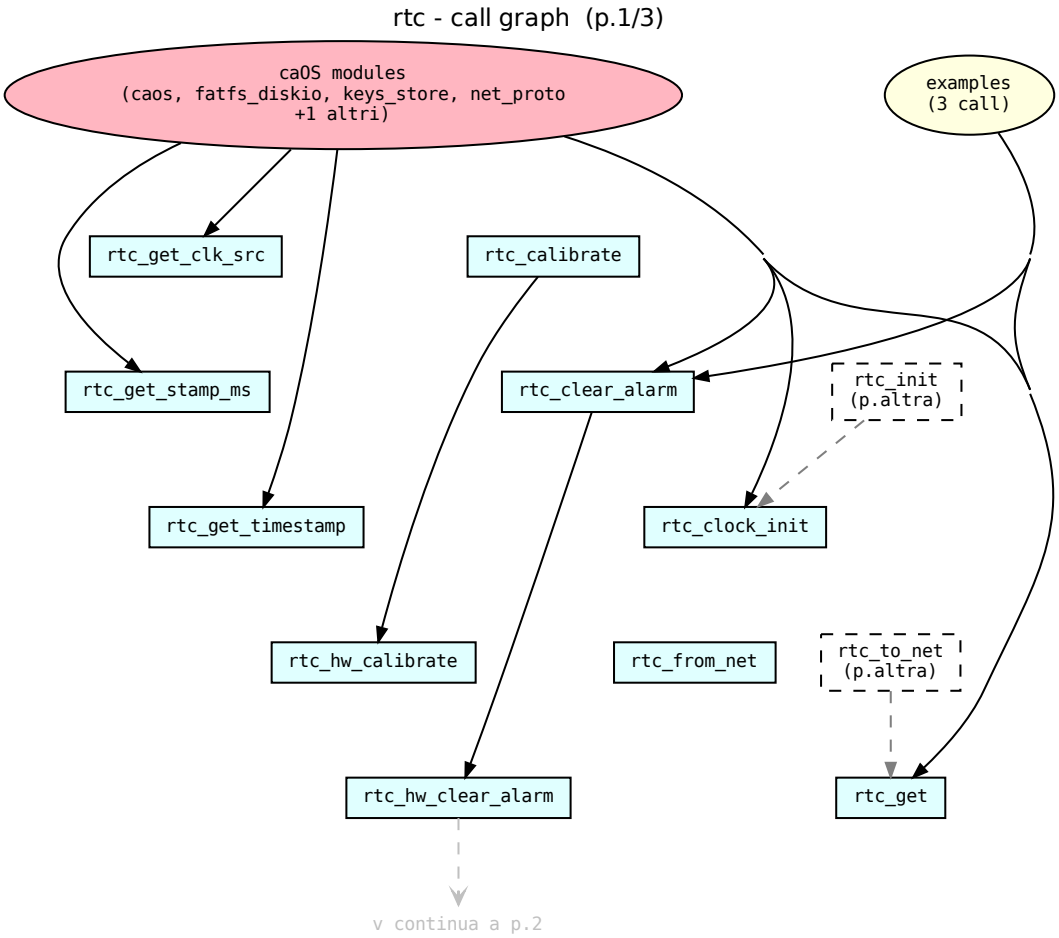

rng - call graph
examples
(23 call)
caOS modules
(aes_channel_service, dhcp_service, ecdsa_p256, ed25519
+5 altri)
workspace
(2 call)
rng_fill
rng_hw_init
rng_get
rng_init
rng_hw_get
rng_is_hardware

118.74 route_service

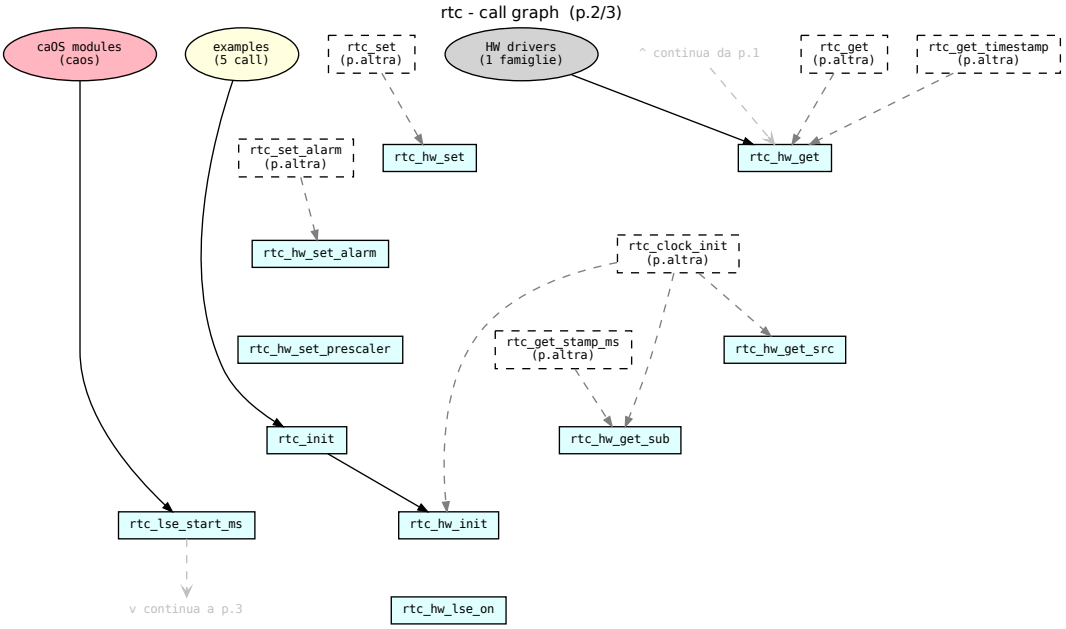


118.75 `rtc`

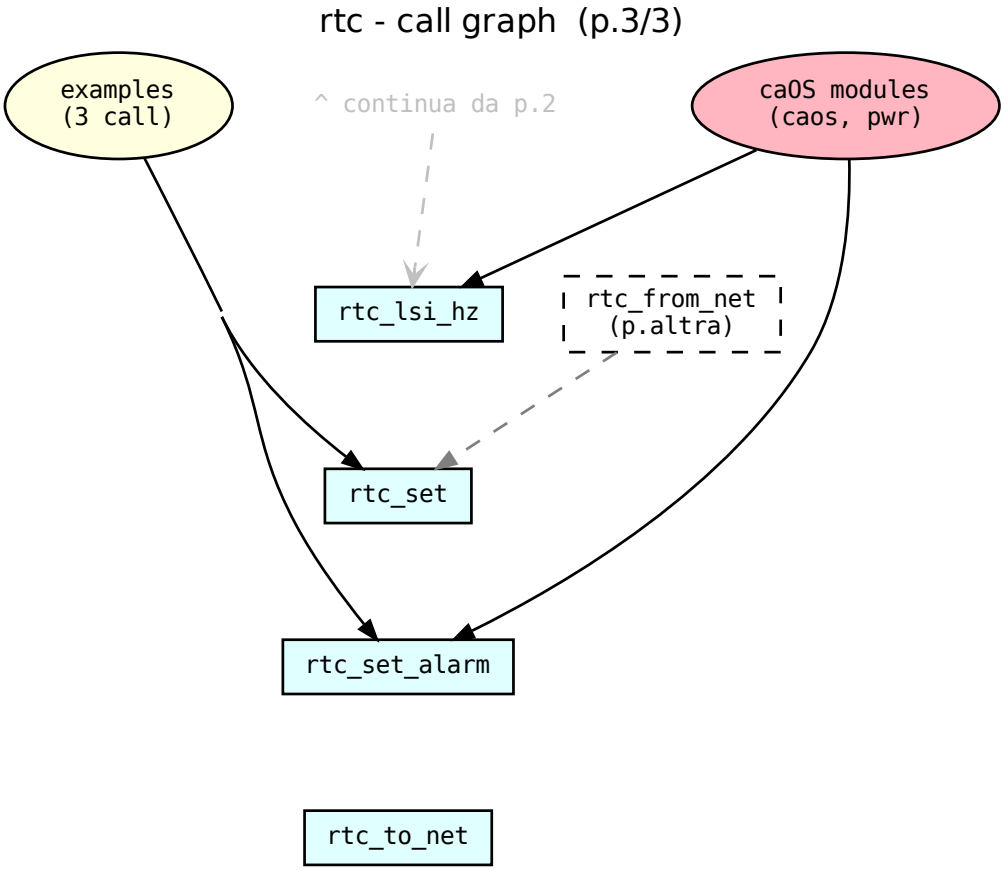
Pagina 1/3



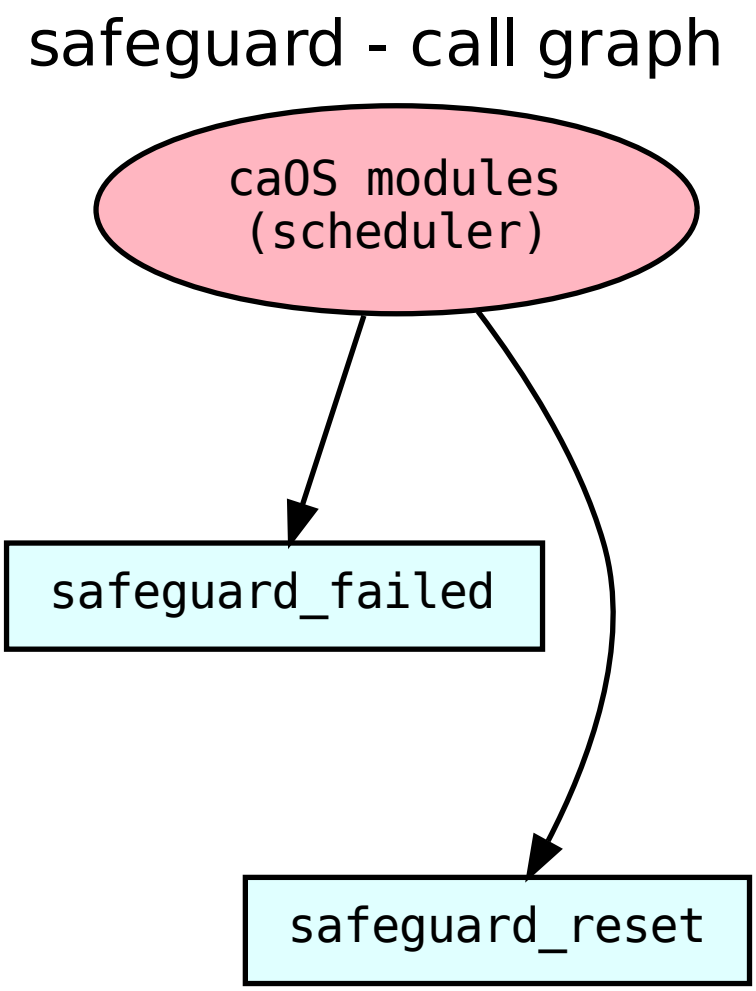
Pagina 2/3


rtc - call graph (p.2/3)
caOS modules (caos)
examples (5 call)
rtc_set (p.altra)
HW drivers (1 famiglie)
^ continua da p.1
rtc_get (p.altra)
rtc_get_timestamp (p.altra)
rtc_set_alarm (p.altra)
rtc_hw_set
rtc_hw_get
rtc_hw_set_alarm
rtc_clock_init (p.altra)
rtc_hw_set_prescaler
rtc_get_stamp_ms (p.altra)
rtc_hw_get_src
rtc_init
rtc_hw_get_sub
rtc_lse_start_ms
rtc_hw_init
v continua a p.3
rtc_hw_lse_on

Pagina 3/3

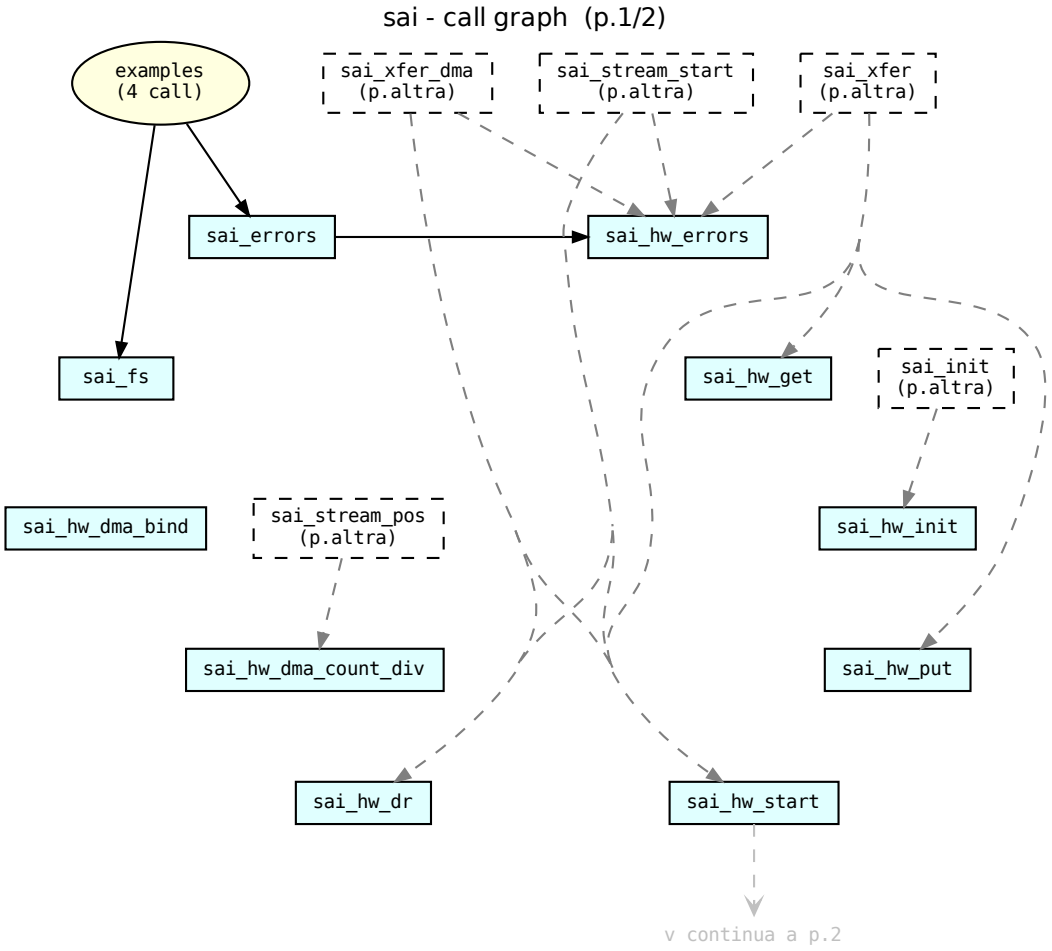

rtc - call graph (p.3/3)
examples
(3 call)
^ continua da p.2
caOS modules
(caos, pwr)
rtc_lsi_hz
rtc_from_net
(p.altra)
rtc_set
rtc_set_alarm
rtc_to_net

118.76 `safeguard`

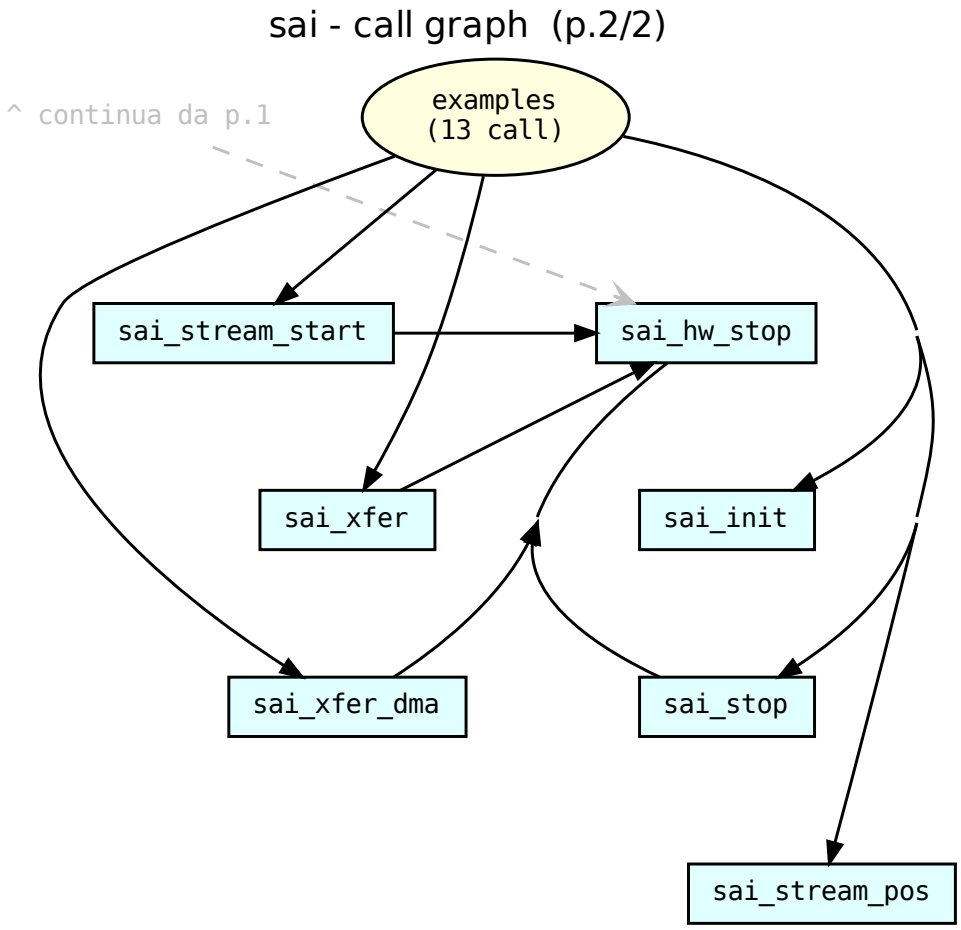


118.77 sai

Pagina 1/2

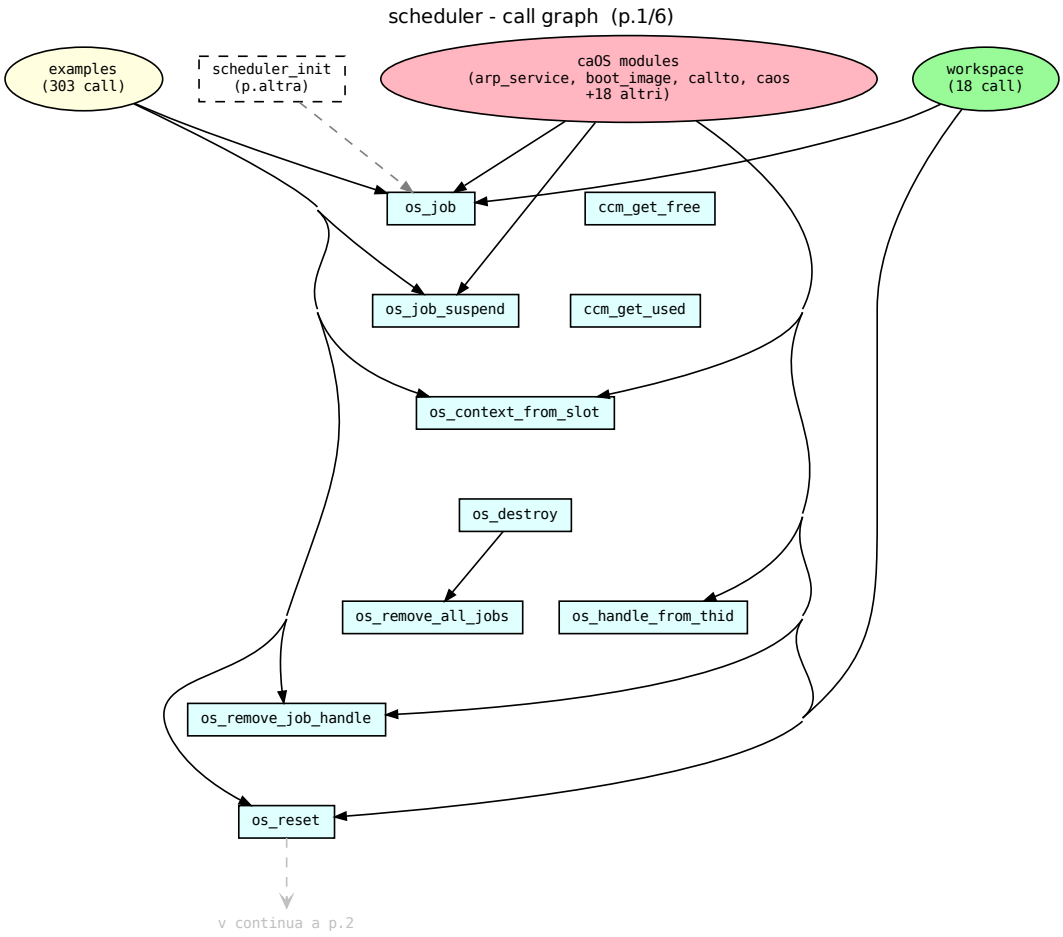


Pagina 2/2

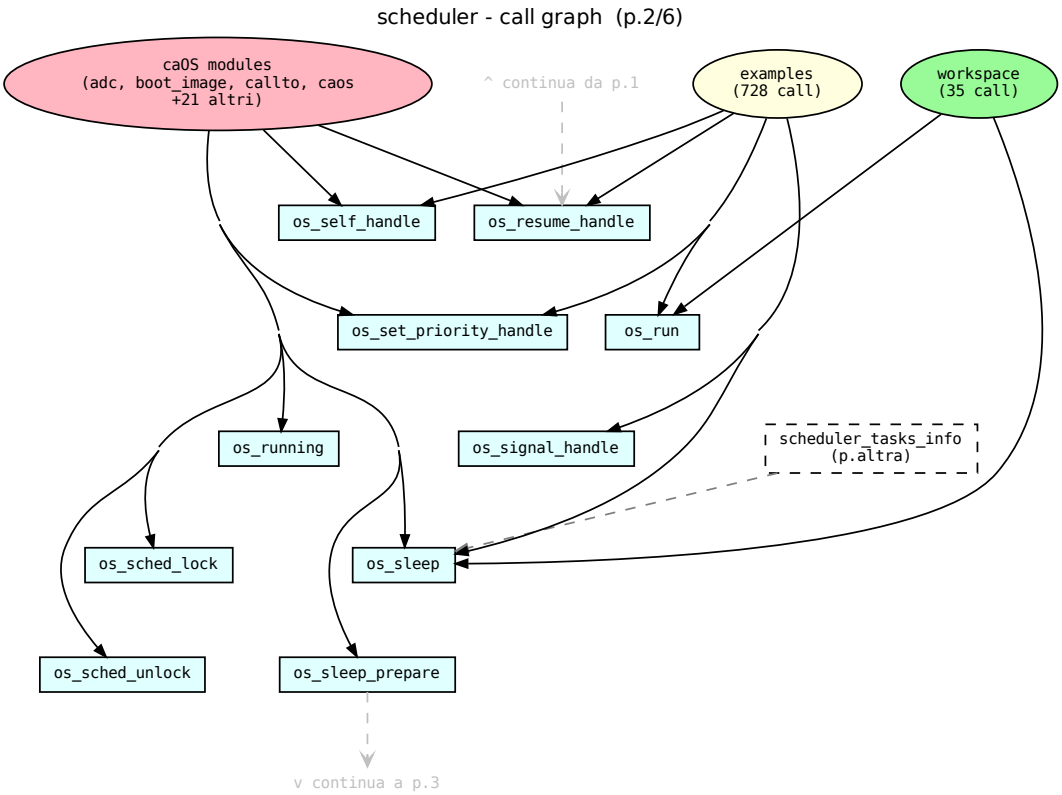

sai - call graph (p.2/2)
^ continua da p.1
examples
(13 call)
sai_stream_start
sai_hw_stop
sai_xfer
sai_init
sai_xfer_dma
sai_stop
sai_stream_pos

118.78 scheduler

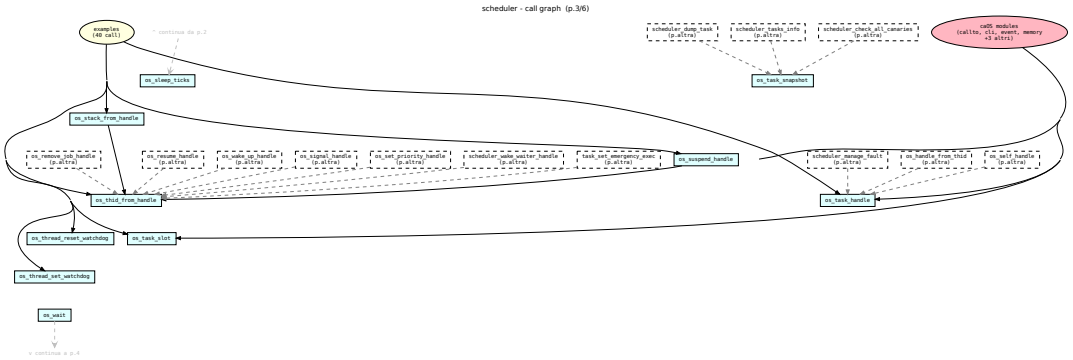
Pagina 1/6



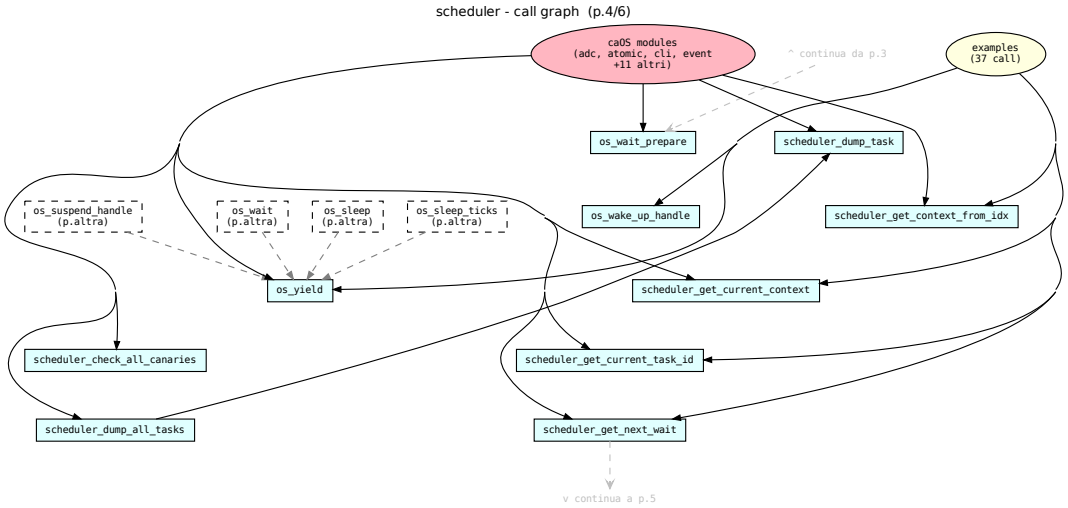
Pagina 2/6


scheduler - call graph (p.2/6)
caOS modules
(adc, boot_image, callto, caos
+21 altri)
^ continua da p.1
examples
(728 call)
workspace
(35 call)
os_self_handle
os_resume_handle
os_set_priority_handle
os_run
os_running
os_signal_handle
scheduler_tasks_info
(p.altra)
os_sched_lock
os_sleep
os_sched_unlock
os_sleep_prepare
v continua a p.3

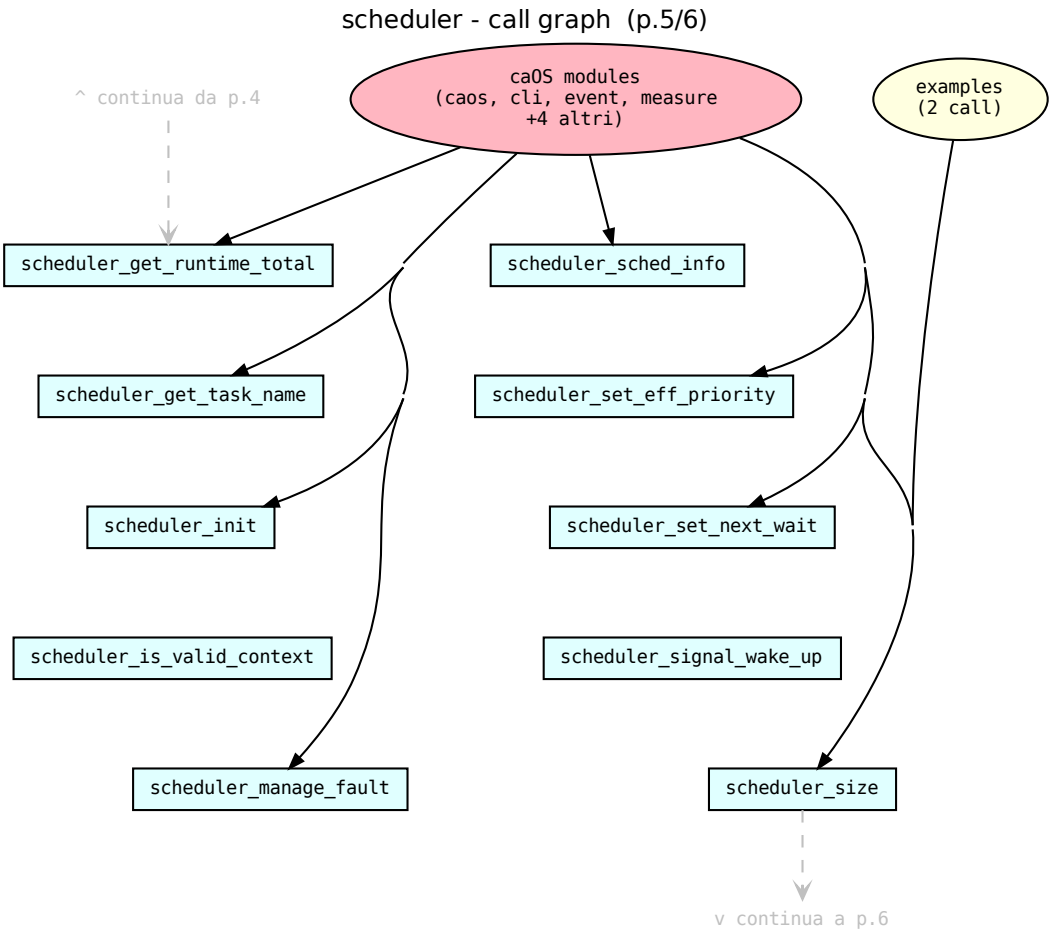
Pagina 3/6


scheduler - call graph (p.3/6)
examples (40 call)
os_sleep_ticks
os_stack_from_handle
os_remove_job_handle (p.a11ra)
os_resume_handle (p.a11ra)
os_wake_up_handle (p.a11ra)
os_signal_handle (p.a11ra)
os_set_priority_handle (p.a11ra)
scheduler_wake_waiter_handle (p.a11ra)
task_set_emergency_exec (p.a11ra)
os_suspend_handle
os_thid_from_handle
os_thread_reset_watchdog
os_task_slot
os_thread_set_watchdog
os_wait
scheduler_dump_task (p.a11ra)
scheduler_tasks_info (p.a11ra)
scheduler_check_all_canaries (p.a11ra)
os_task_snapshot
cat05 modules (calltp, cli, event, memory +3 a11ra)
scheduler_manage_fault (p.a11ra)
os_handle_from_thid (p.a11ra)
os_self_handle (p.a11ra)
os_task_handle

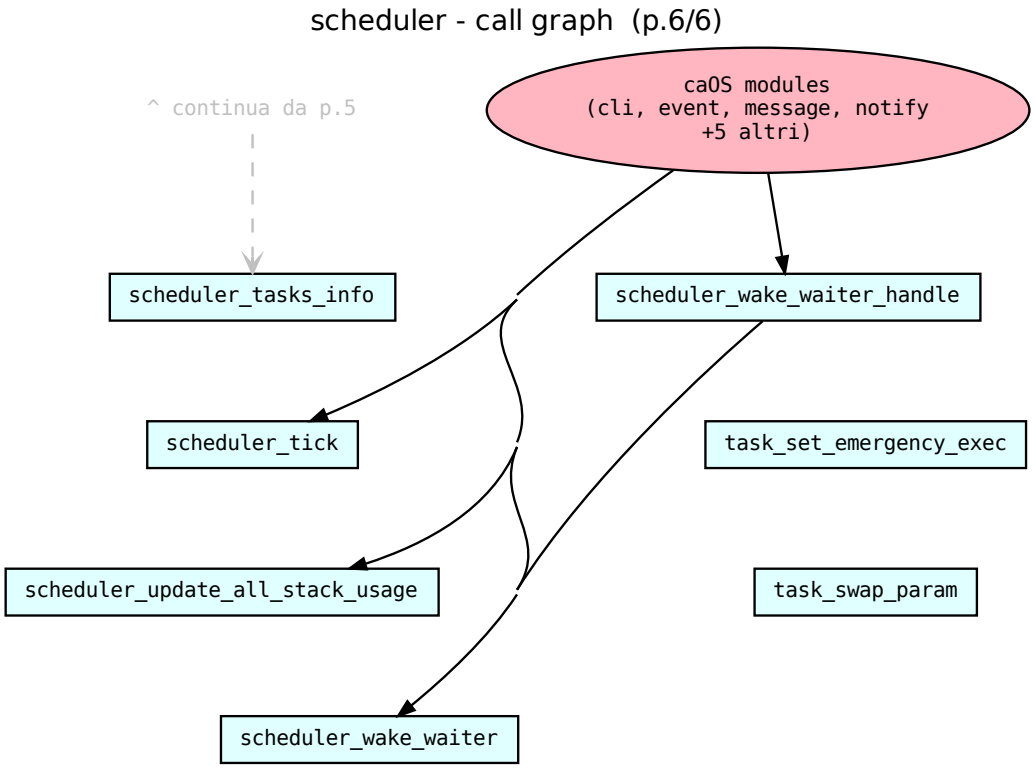
Pagina 4/6


scheduler - call graph (p.4/6)
caOS modules
(adc, atomic, cli, event
+11 altri)
^ continua da p.3
examples
(37 call)
os_wait_prepare
scheduler_dump_task
os_suspend_handle
(p.altra)
os_wait
(p.altra)
os_sleep
(p.altra)
os_sleep_ticks
(p.altra)
os_wake_up_handle
scheduler_get_context_from_idx
os_yield
scheduler_get_current_context
scheduler_check_all_canaries
scheduler_get_current_task_id
scheduler_dump_all_tasks
scheduler_get_next_wait
v continua a p.5

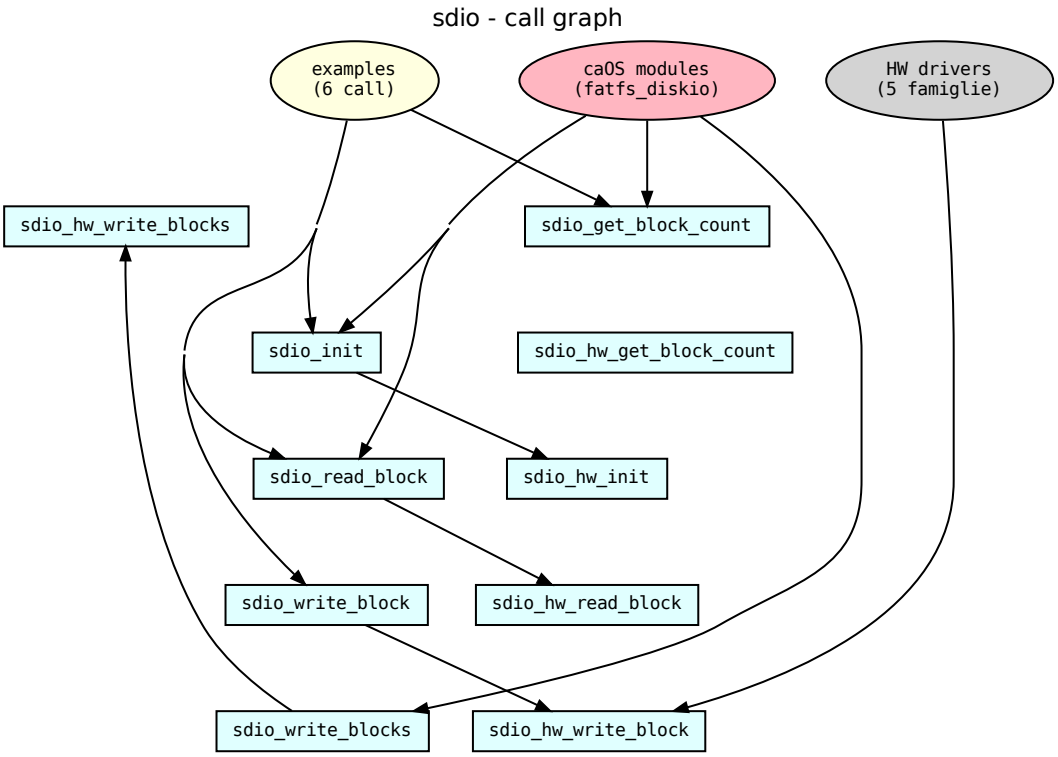
Pagina 5/6


scheduler - call graph (p.5/6)
^ continua da p.4
caOS modules
(caos, cli, event, measure
+4 altri)
examples
(2 call)
scheduler_get_runtime_total
scheduler_sched_info
scheduler_get_task_name
scheduler_set_eff_priority
scheduler_init
scheduler_set_next_wait
scheduler_is_valid_context
scheduler_signal_wake_up
scheduler_manage_fault
scheduler_size
v continua a p.6

Pagina 6/6

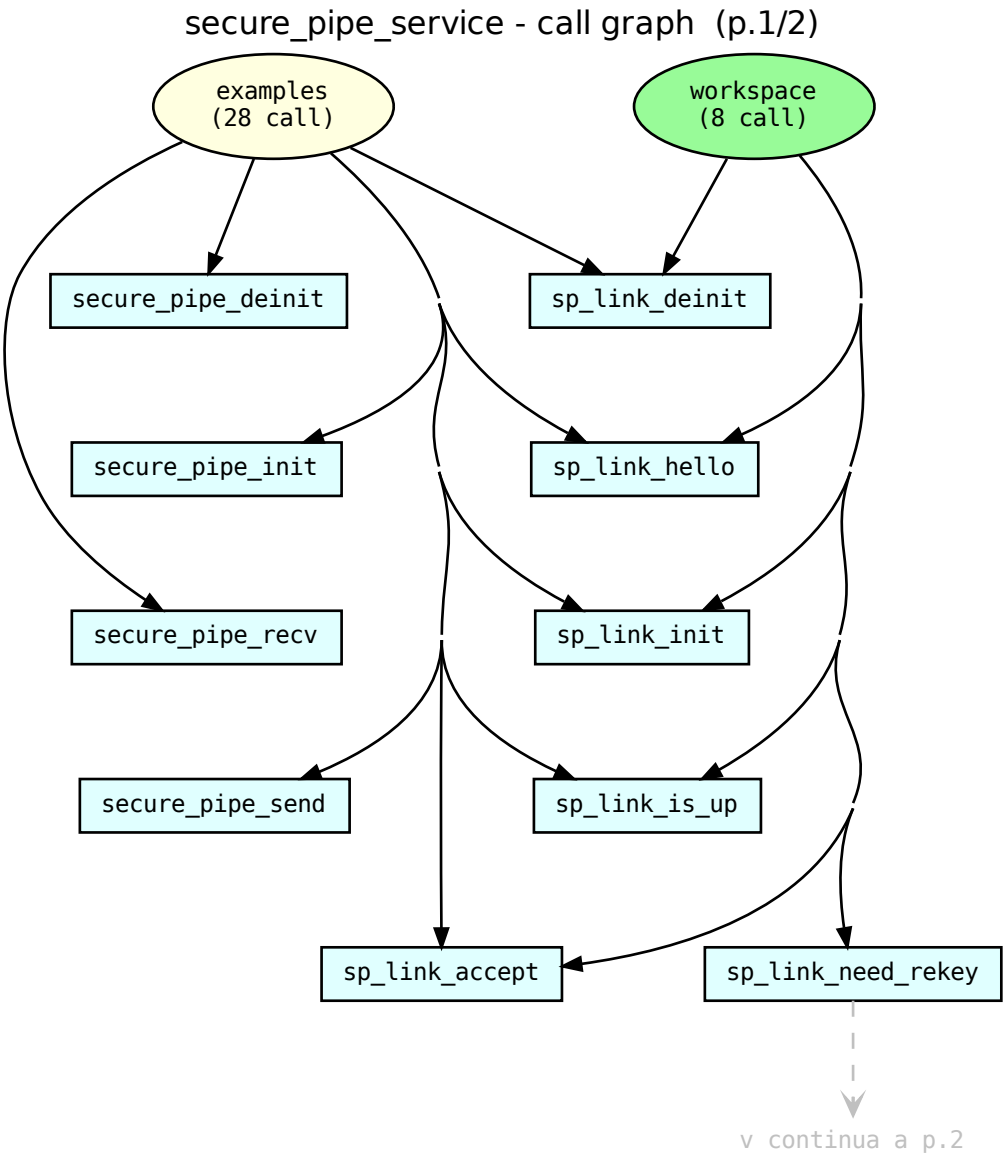

scheduler - call graph (p.6/6)
^ continua da p.5
caOS modules
(cli, event, message, notify
+5 altri)
scheduler_tasks_info
scheduler_wake_waiter_handle
scheduler_tick
task_set_emergency_exec
scheduler_update_all_stack_usage
task_swap_param
scheduler_wake_waiter

118.79 sdio

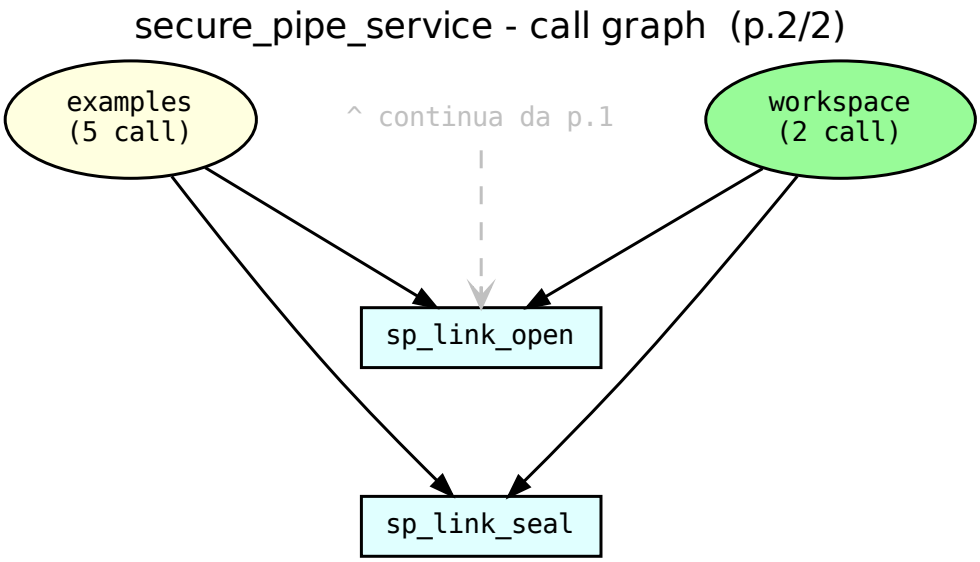

sdio - call graph
examples
(6 call)
caOS modules
(fatfs_diskio)
HW drivers
(5 famiglie)
sdio_hw_write_blocks
sdio_get_block_count
sdio_init
sdio_hw_get_block_count
sdio_read_block
sdio_hw_init
sdio_write_block
sdio_hw_read_block
sdio_write_blocks
sdio_hw_write_block

118.80 secure_pipe_service

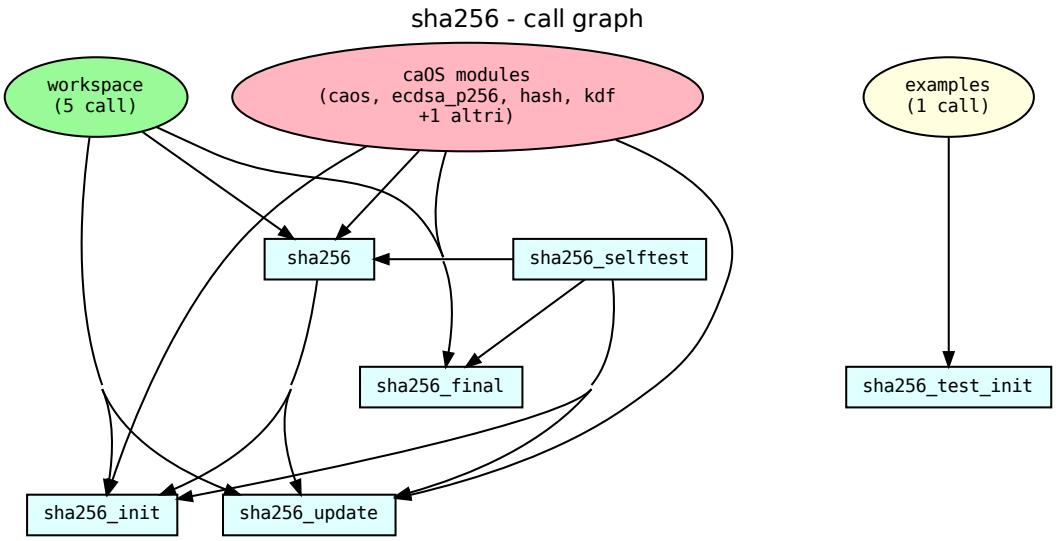
Pagina 1/2



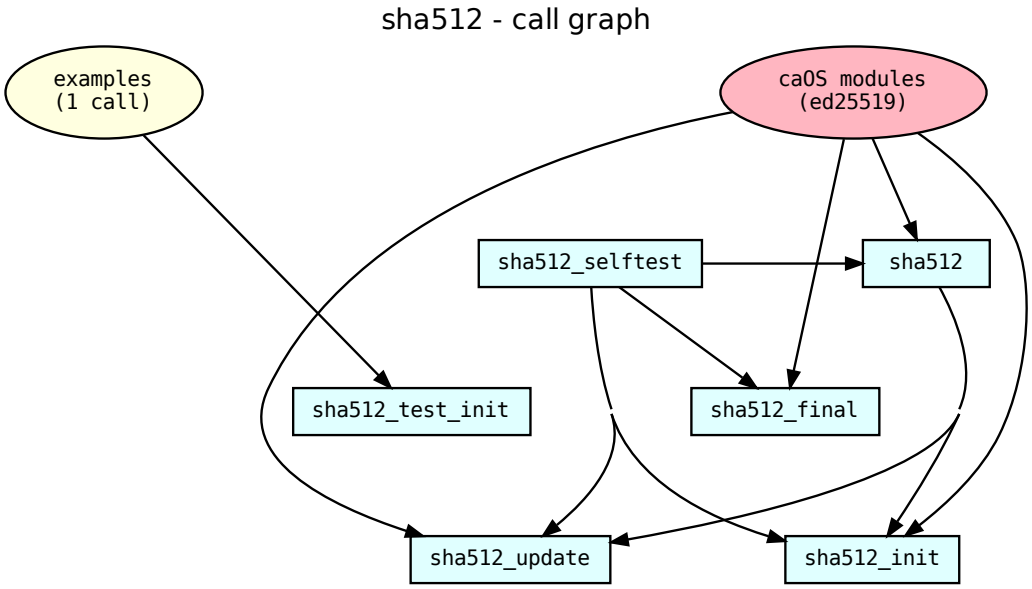
Pagina 2/2


secure_pipe_service - call graph (p.2/2)
examples
(5 call)
^ continua da p.1
workspace
(2 call)
sp_link_open
sp_link_seal

118.81 sha256

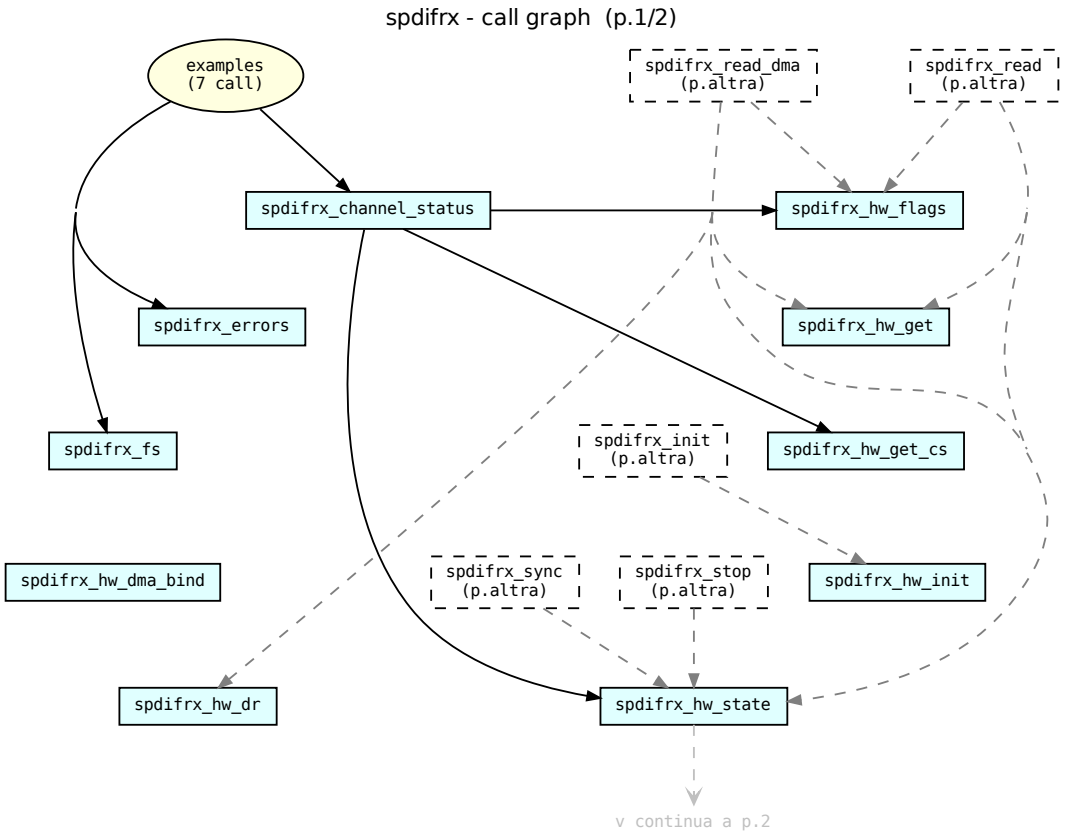


118.82 sha512

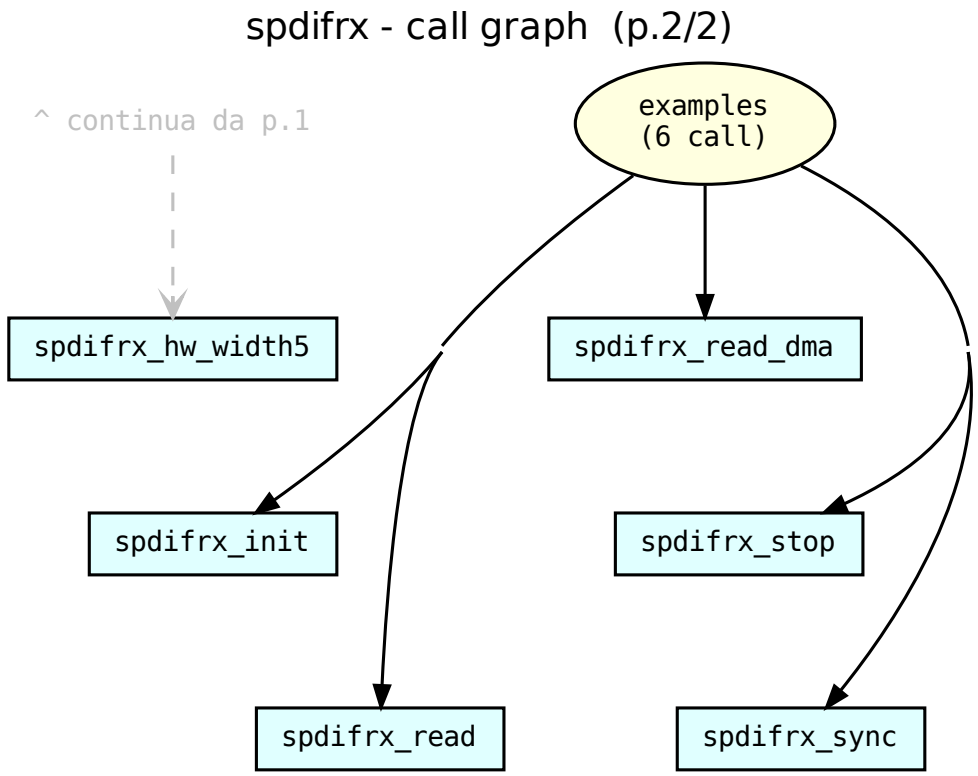


118.83 spdifrx

Pagina 1/2

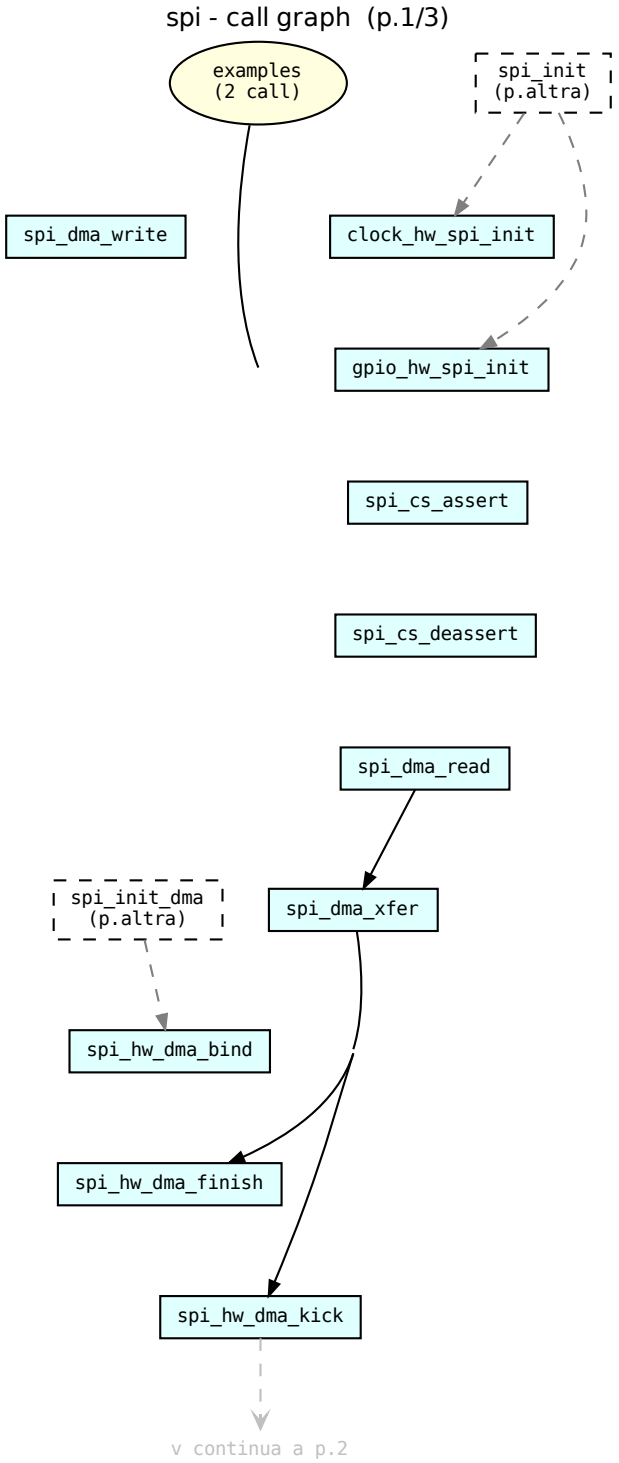


Pagina 2/2

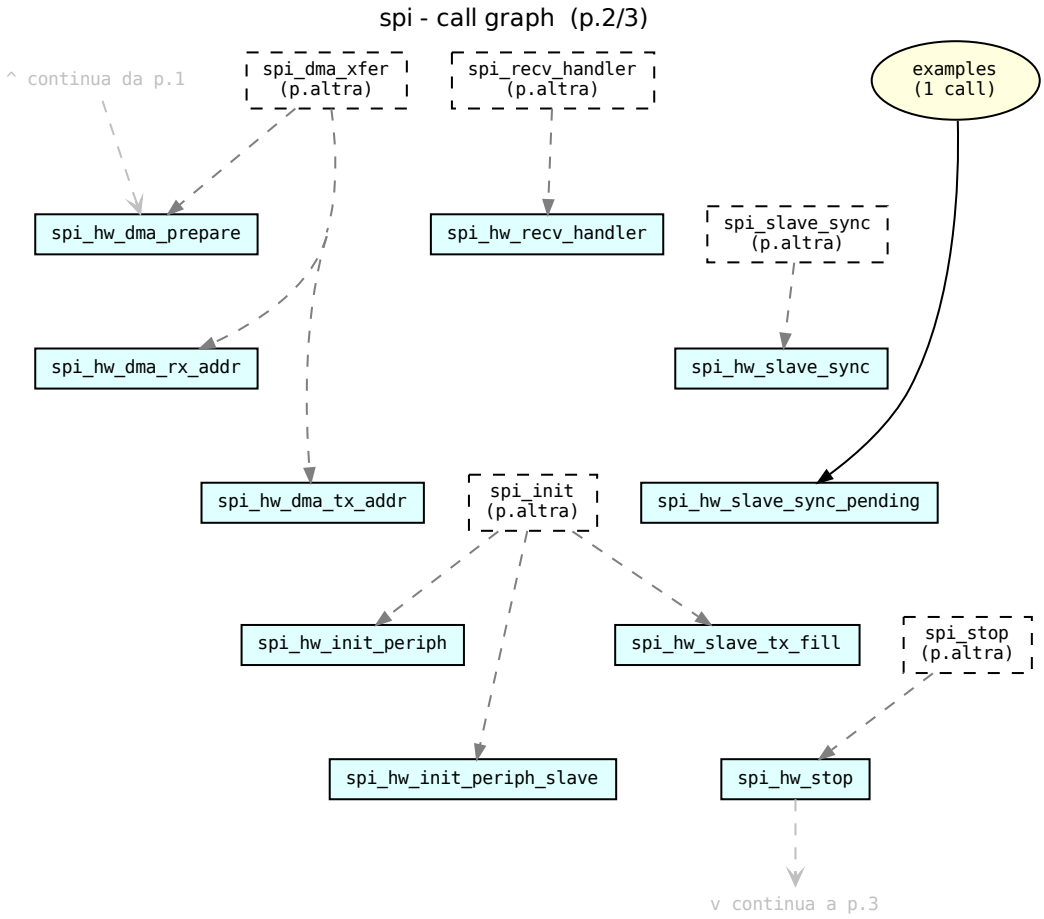

spdifrx - call graph (p.2/2)
^ continua da p.1
examples
(6 call)
spdifrx_hw_width5
spdifrx_read_dma
spdifrx_init
spdifrx_stop
spdifrx_read
spdifrx_sync

118.84 spi

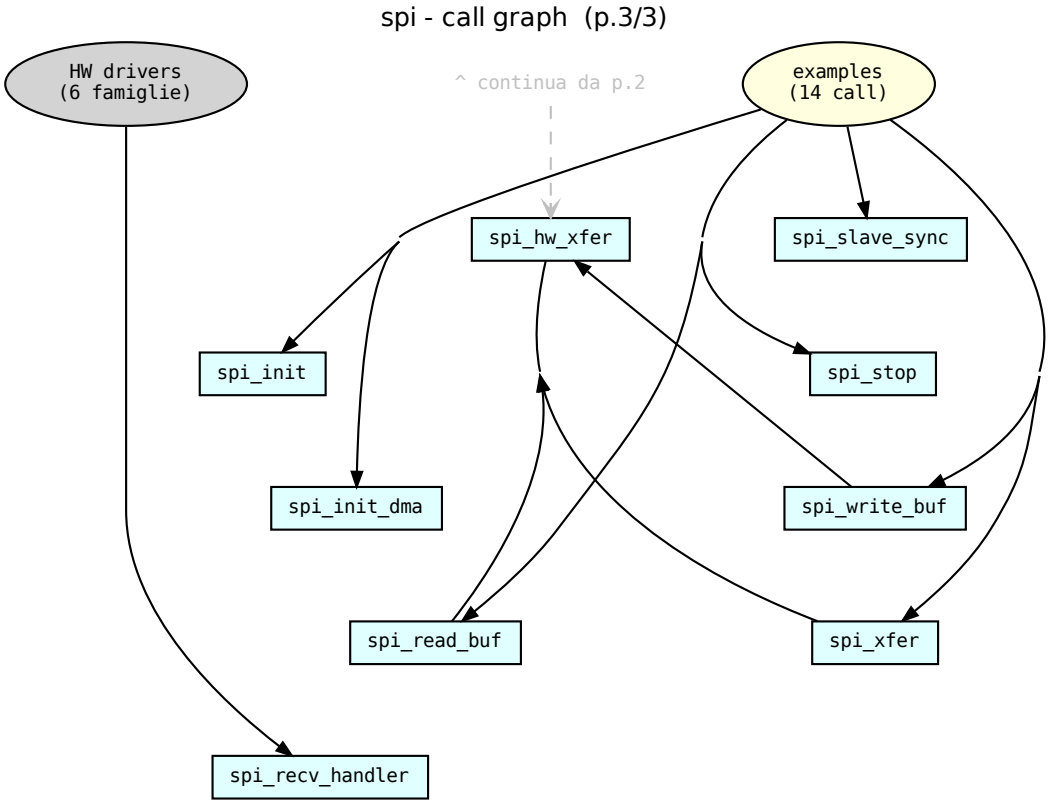
Pagina 1/3



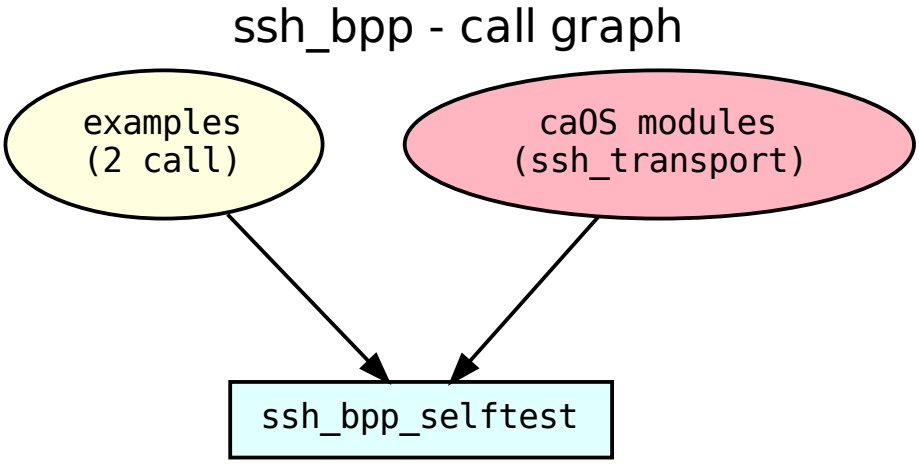
Pagina 2/3


spi - call graph (p.2/3)
^ continua da p.1
spi_dma_xfer (p.altra)
spi_recv_handler (p.altra)
examples (1 call)
spi_hw_dma_prepare
spi_hw_recv_handler
spi_slave_sync (p.altra)
spi_hw_dma_rx_addr
spi_hw_slave_sync
spi_hw_dma_tx_addr
spi_init (p.altra)
spi_hw_slave_sync_pending
spi_hw_init_periph
spi_hw_slave_tx_fill
spi_stop (p.altra)
spi_hw_init_periph_slave
spi_hw_stop
v continua a p.3

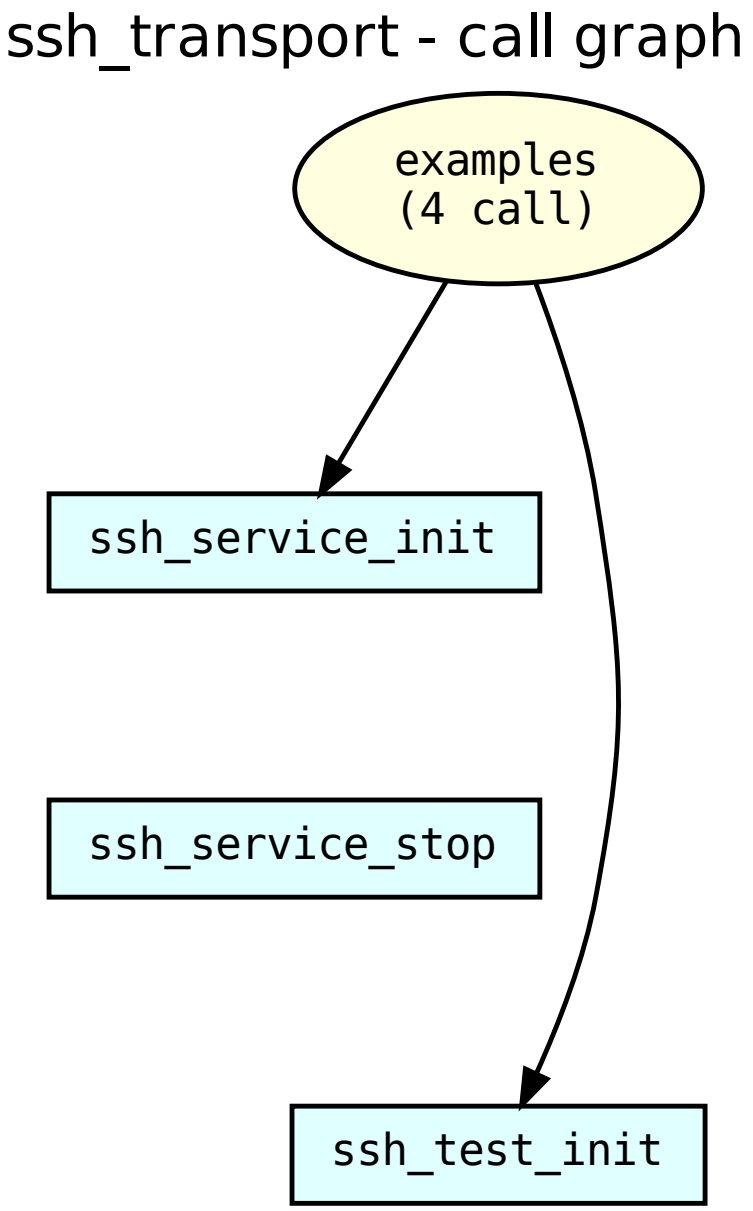
Pagina 3/3


spi - call graph (p.3/3)
HW drivers
(6 famiglie)
^ continua da p.2
examples
(14 call)
spi_hw_xfer
spi_slave_sync
spi_init
spi_stop
spi_init_dma
spi_write_buf
spi_read_buf
spi_xfer
spi_recv_handler

118.85 ssh_bpp

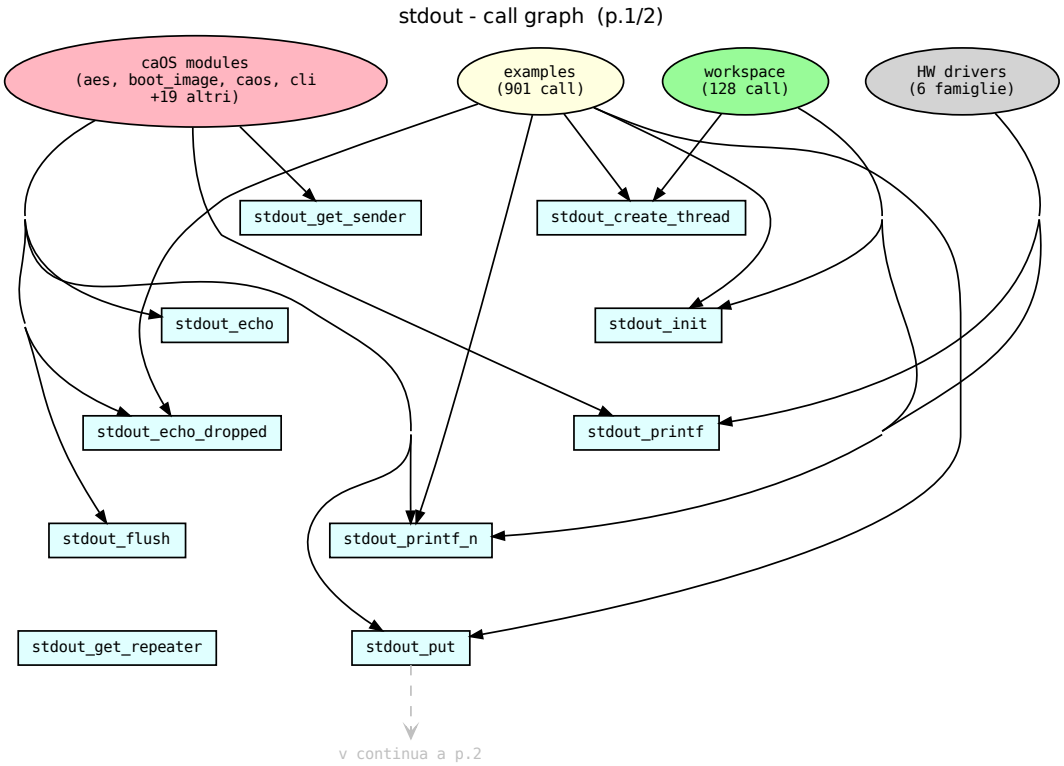


118.86 ssh_transport

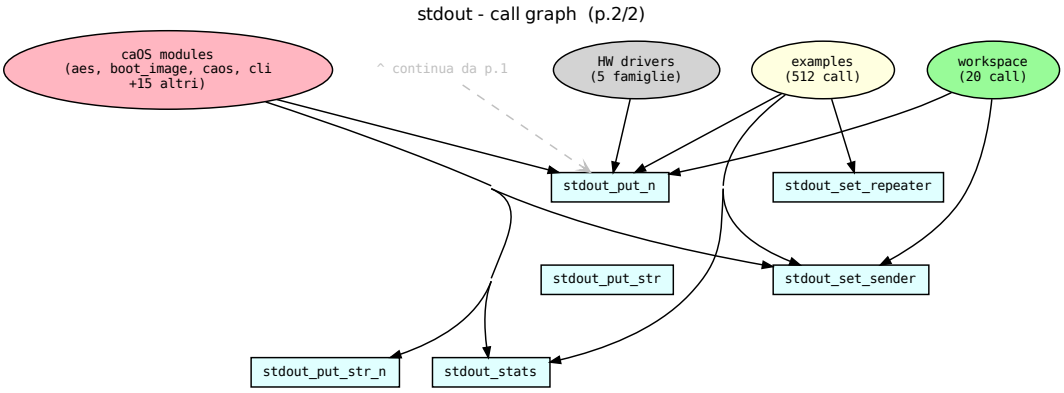


118.87 stdout

Pagina 1/2

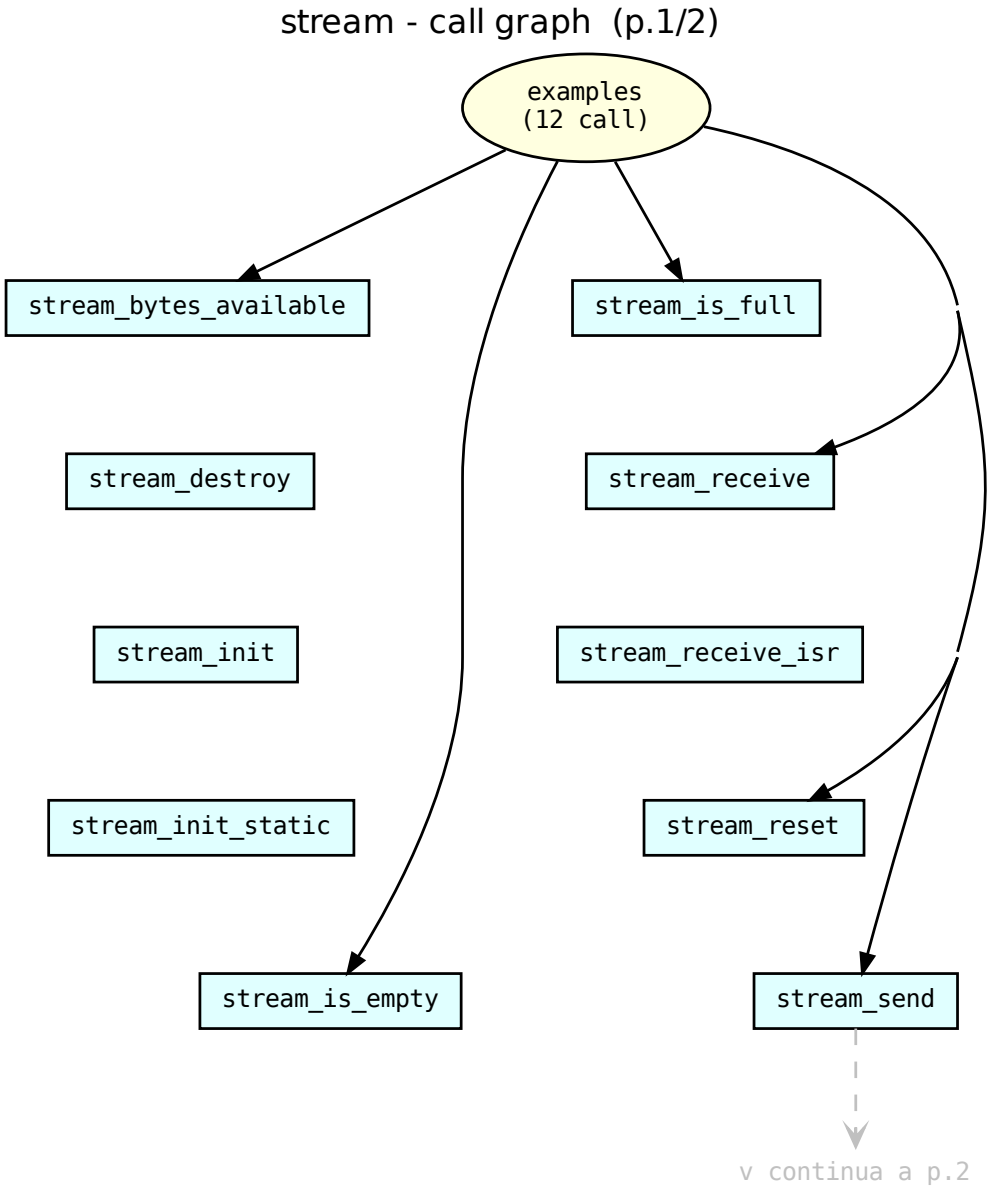


Pagina 2/2

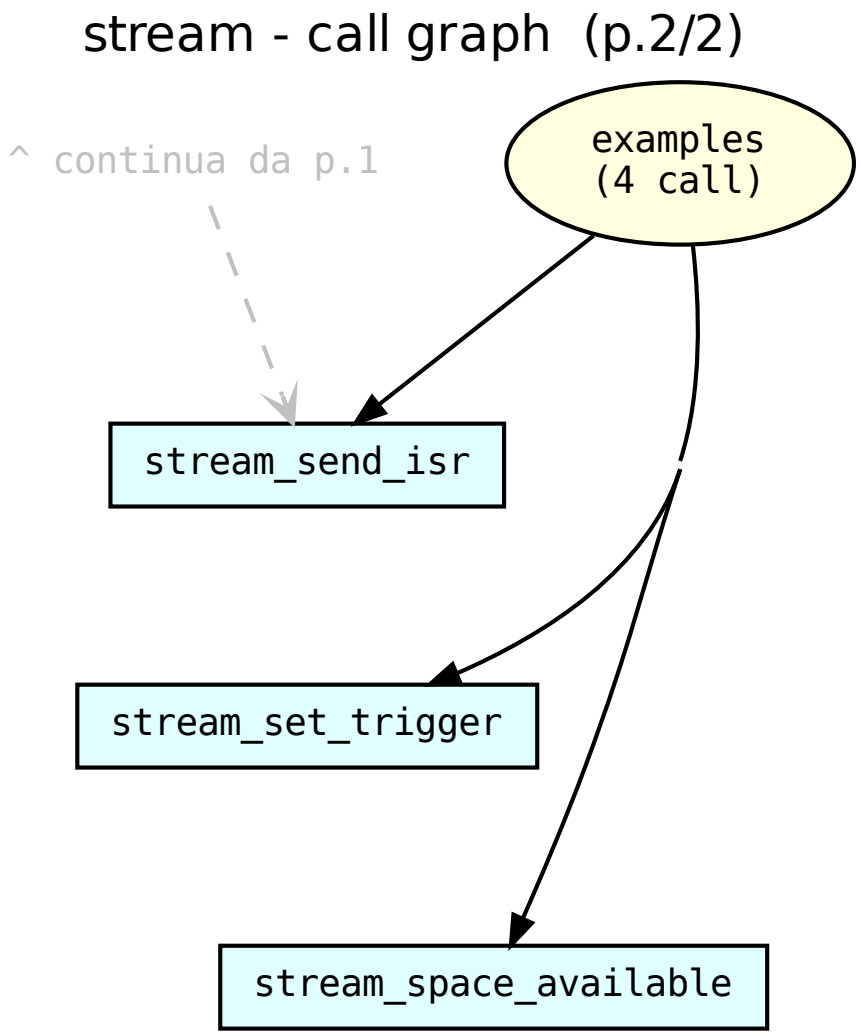

stdout - call graph (p.2/2)
caOS modules
(aes, boot_image, caos, cli
+15 altri)
^ continua da p.1
HW drivers
(5 famiglie)
examples
(512 call)
workspace
(20 call)
stdout_put_n
stdout_set_repeater
stdout_put_str
stdout_set_sender
stdout_put_str_n
stdout_stats

118.88 stream

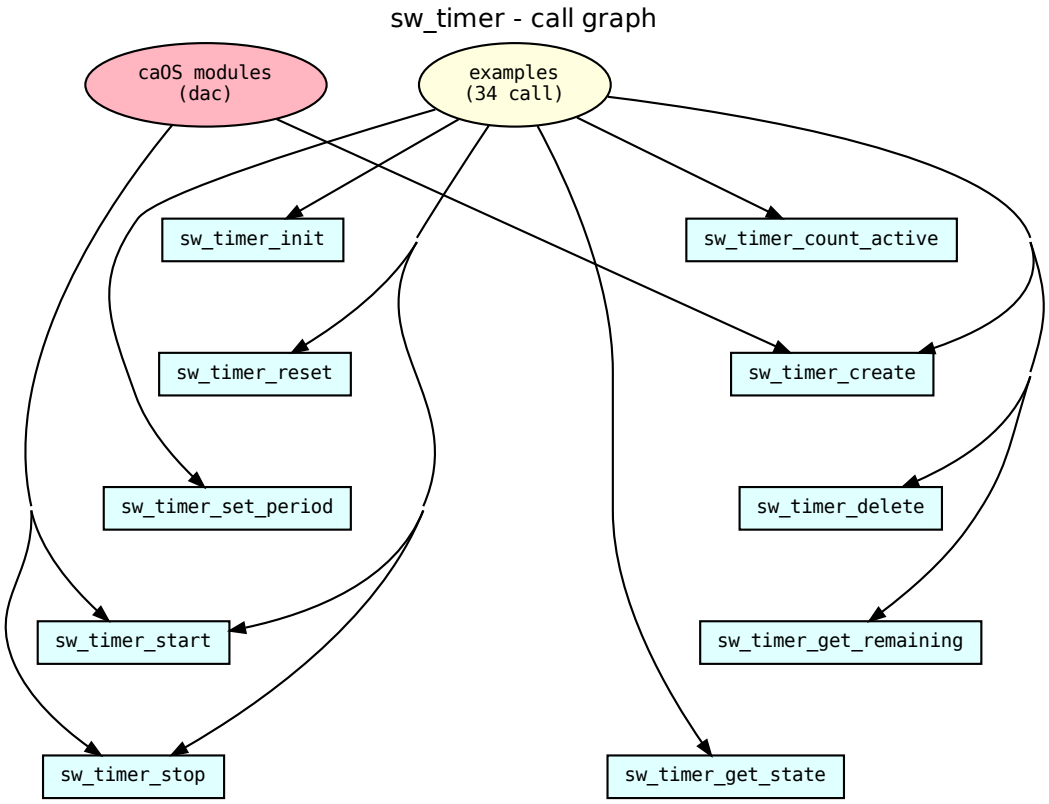
Pagina 1/2



Pagina 2/2

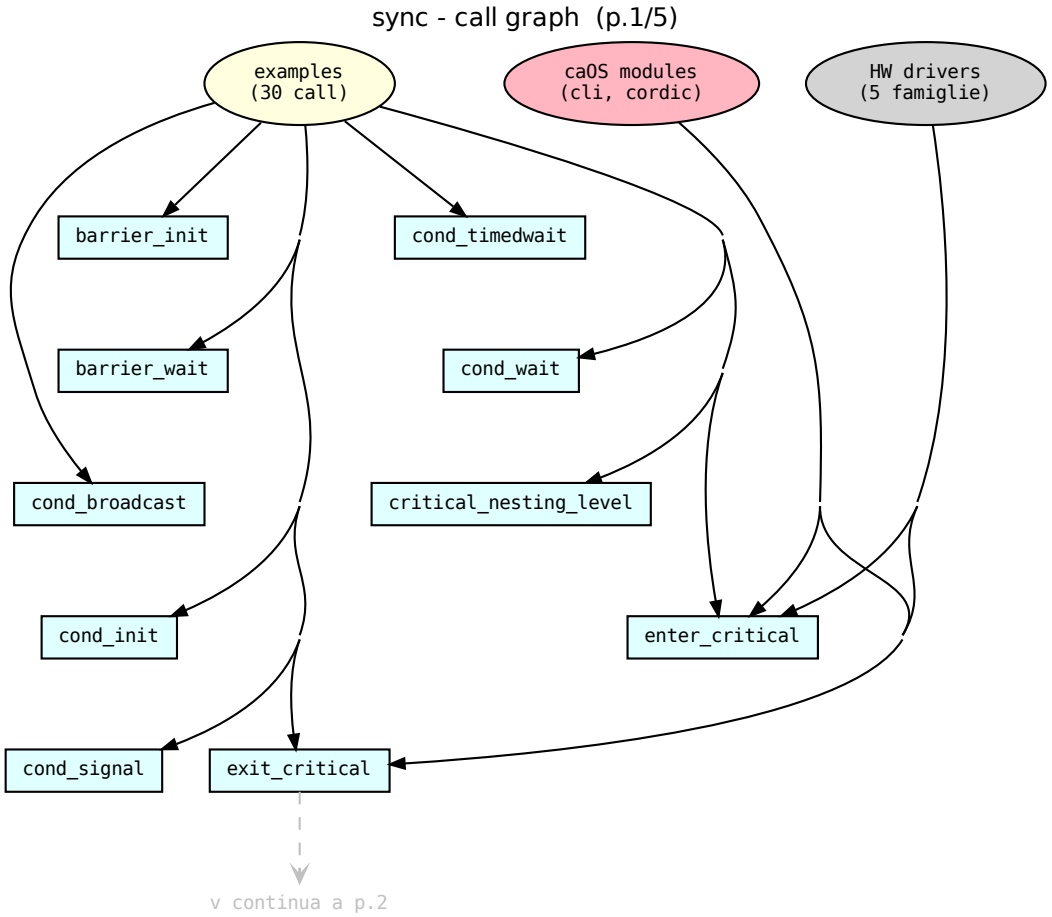

stream - call graph (p.2/2)
^ continua da p.1
examples
(4 call)
stream_send_isr
stream_set_trigger
stream_space_available

118.89 sw_timer

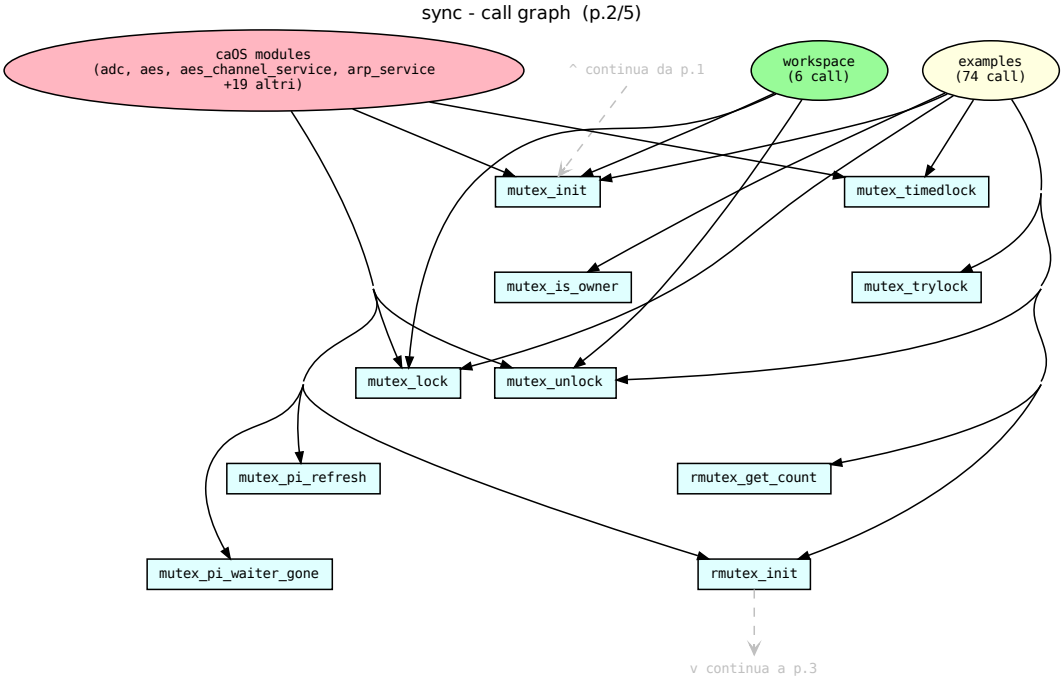


118.90 sync

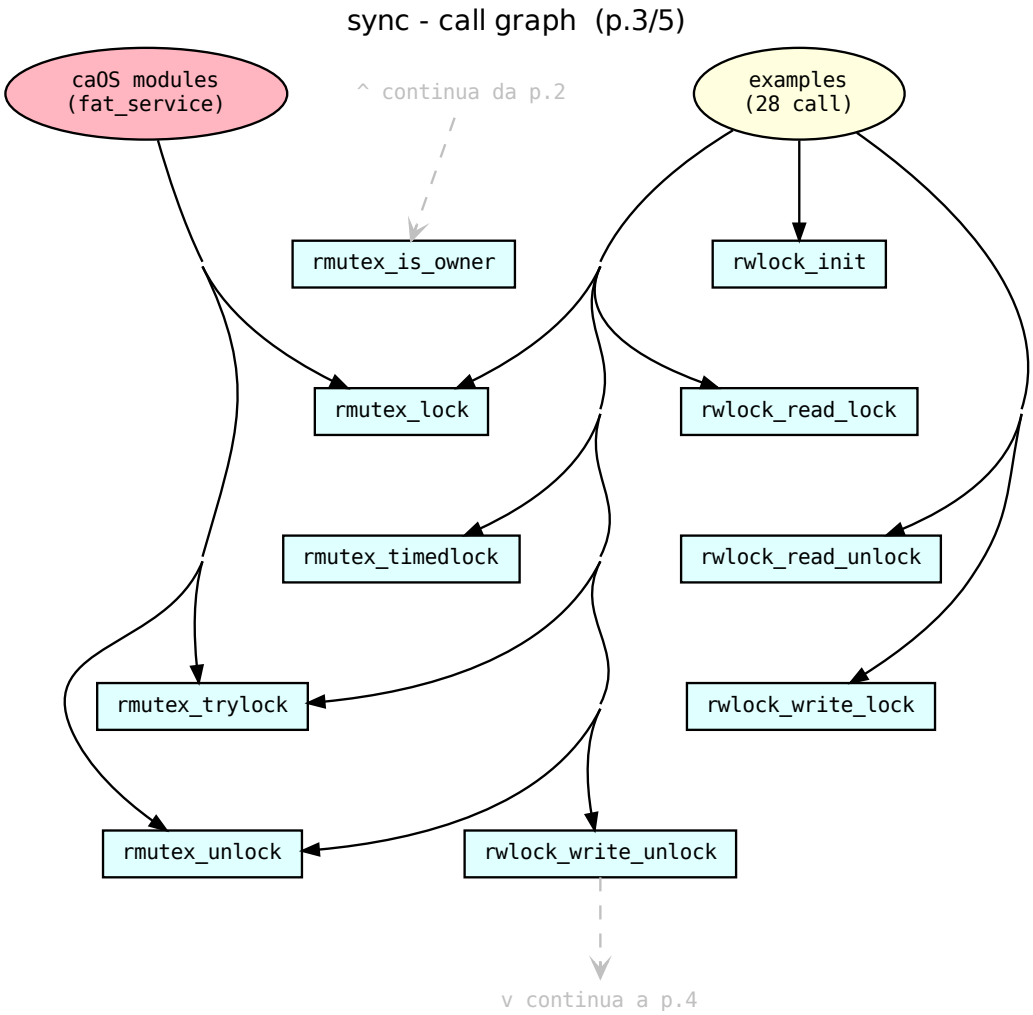
Pagina 1/5



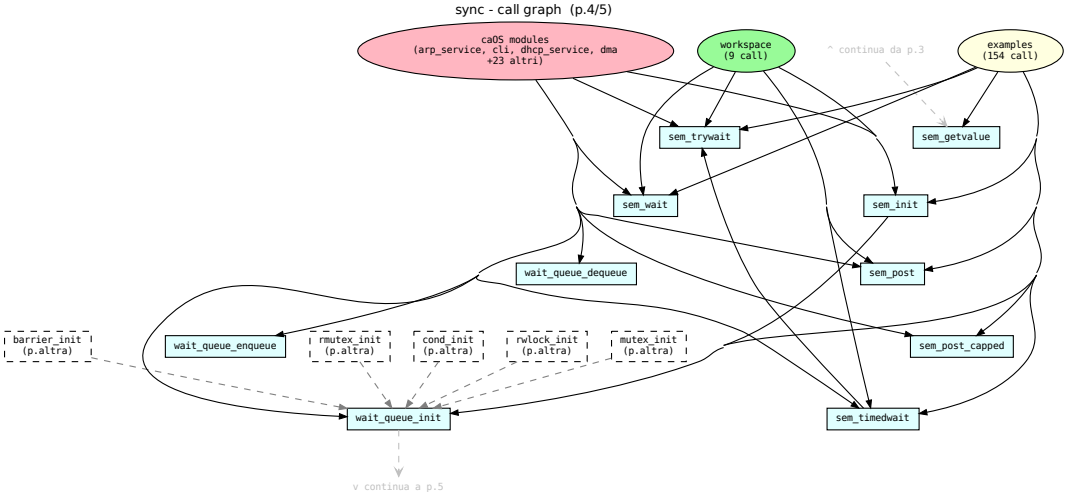
Pagina 2/5


sync - call graph (p.2/5)
caOS modules
(adc, aes, aes_channel_service, arp_service
+19 altri)
^ continua da p.1
workspace
(6 call)
examples
(74 call)
mutex_init
mutex_timedlock
mutex_is_owner
mutex_trylock
mutex_lock
mutex_unlock
mutex_pi_refresh
rmutex_get_count
mutex_pi_waiter_gone
rmutex_init
v continua a p.3

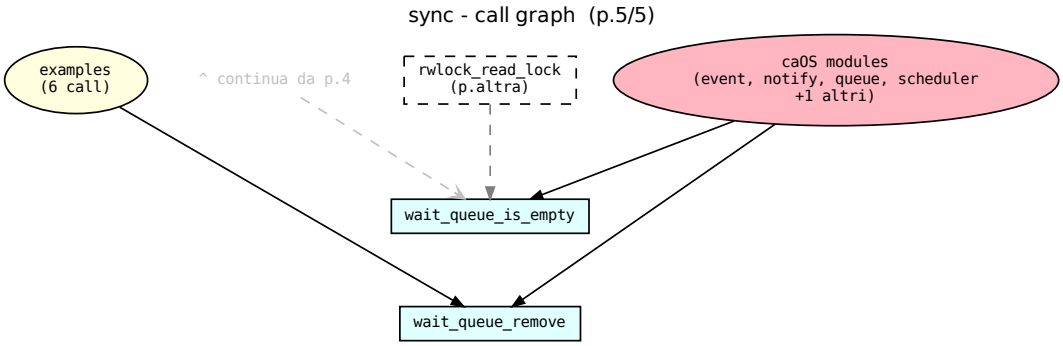
Pagina 3/5


sync - call graph (p.3/5)
caOS modules
(fat_service)
^ continua da p.2
examples
(28 call)
rmutex_is_owner
rwlock_init
rmutex_lock
rwlock_read_lock
rmutex_timedlock
rwlock_read_unlock
rmutex_trylock
rwlock_write_lock
rmutex_unlock
rwlock_write_unlock
v continua a p.4

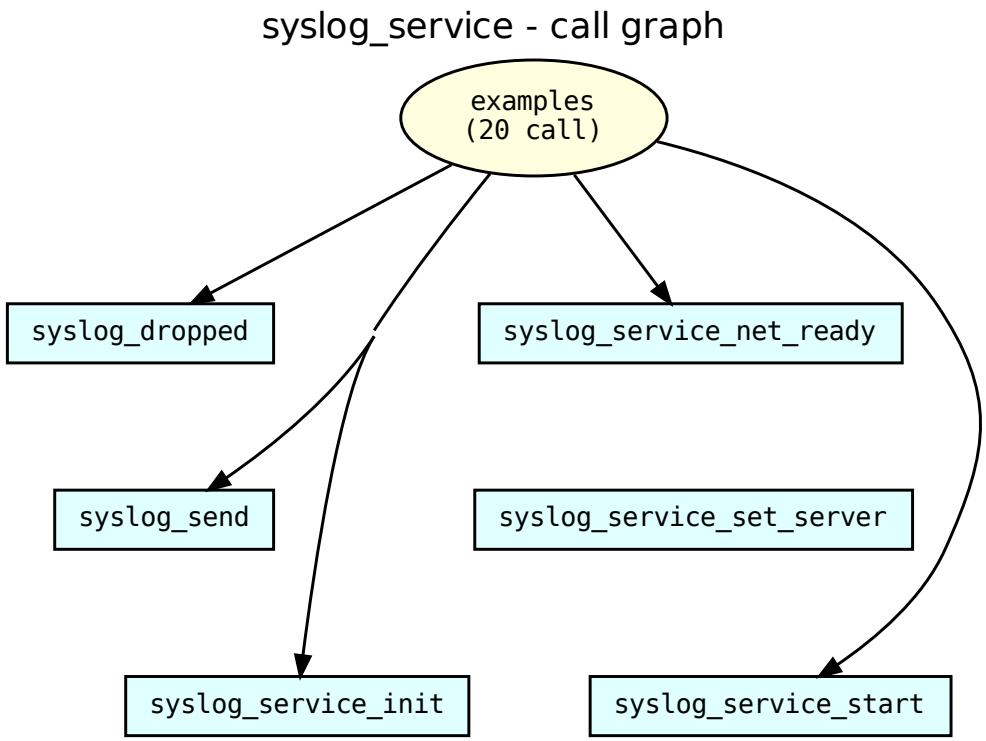
Pagina 4/5


sync - call graph (p.4/5)
caOS modules
(arp_service, cli, dhcp_service, dma
+23 altri)
workspace
(9 call)
^ continua da p.3
examples
(154 call)
sem_trywait
sem_getvalue
sem_wait
sem_init
wait_queue_dequeue
sem_post
barrier_init
(p.altra)
wait_queue_enqueue
rmutex_init
(p.altra)
cond_init
(p.altra)
rwlock_init
(p.altra)
mutex_init
(p.altra)
sem_post_capped
wait_queue_init
sem_timedwait
v continua a p.5

Pagina 5/5

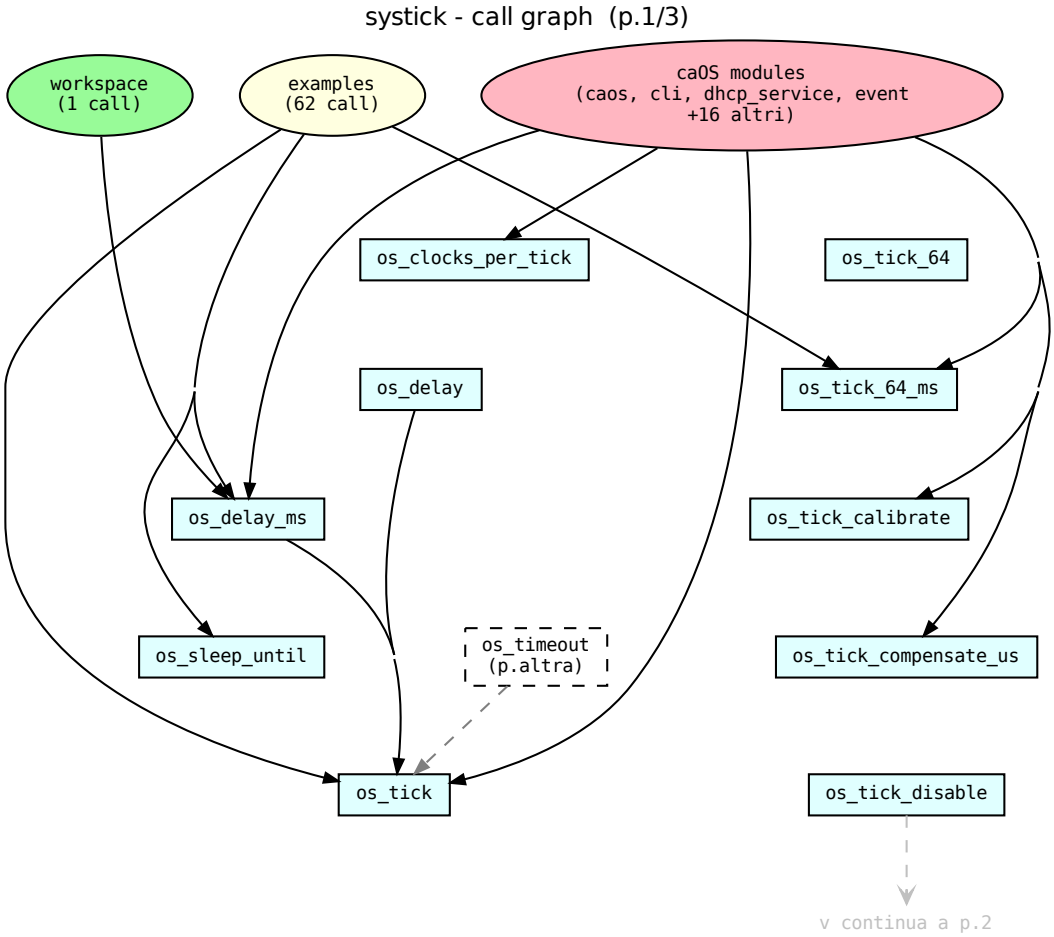

sync - call graph (p.5/5)
examples
(6 call)
^ continua da p.4
rwlock_read_lock
(p.altra)
caOS modules
(event, notify, queue, scheduler
+1 altri)
wait_queue_is_empty
wait_queue_remove

118.91 syslog_service

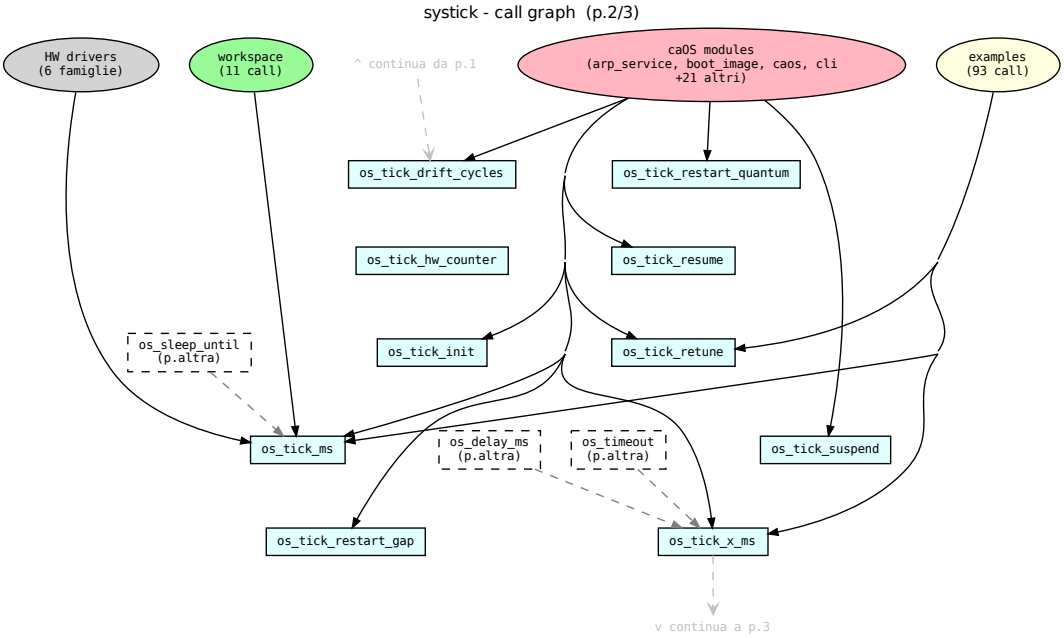


118.92 `systick`

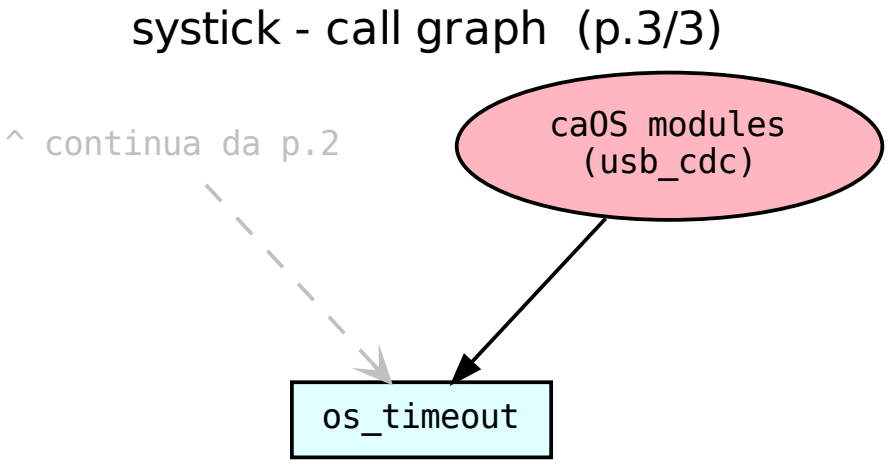
Pagina 1/3



Pagina 2/3

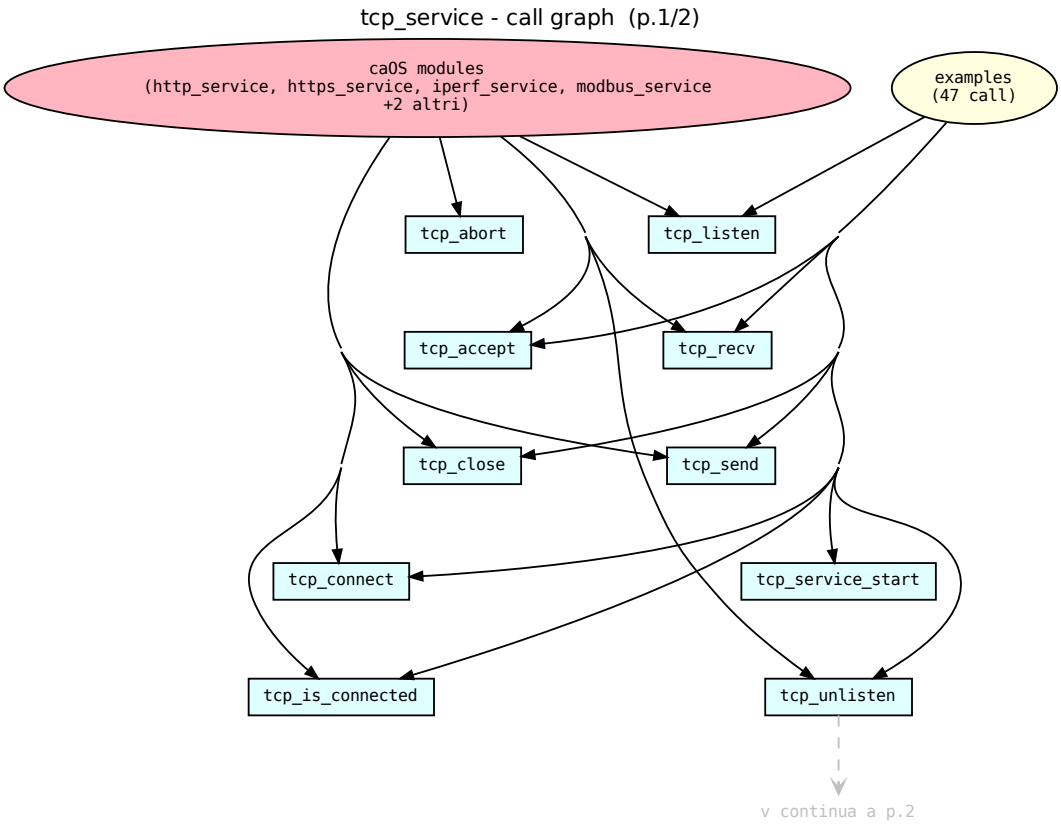

systick - call graph (p.2/3)
HW drivers
(6 famiglie)
workspace
(11 call)
^ continua da p.1
caOS modules
(arp_service, boot_image, caos, cli
+21 altri)
examples
(93 call)
os_tick_drift_cycles
os_tick_restart_quantum
os_tick_hw_counter
os_tick_resume
os_sleep_until
(p.altra)
os_tick_init
os_tick_retune
os_tick_ms
os_delay_ms
(p.altra)
os_timeout
(p.altra)
os_tick_suspend
os_tick_restart_gap
os_tick_x_ms
v continua a p.3

Pagina 3/3

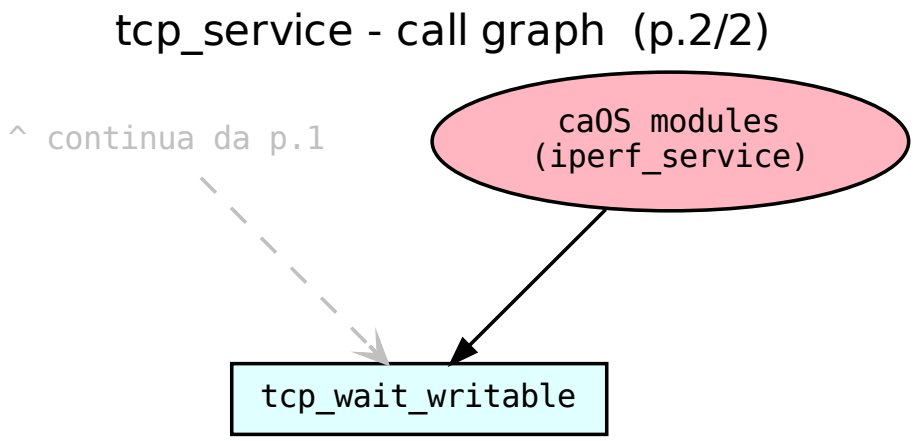

systick - call graph (p.3/3)
^ continua da p.2
caOS modules
(usb_cdc)
os_timeout

118.93 tcp_service

Pagina 1/2

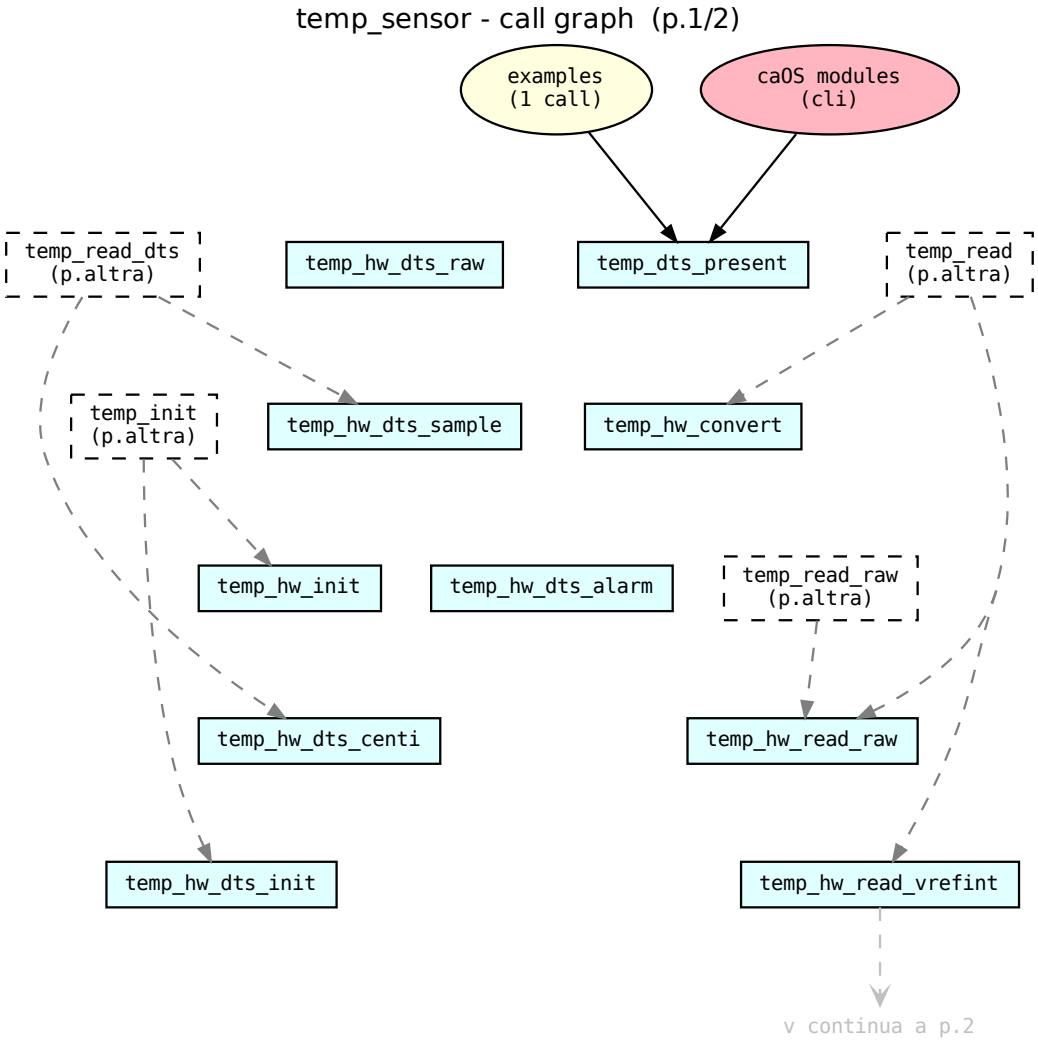


Pagina 2/2

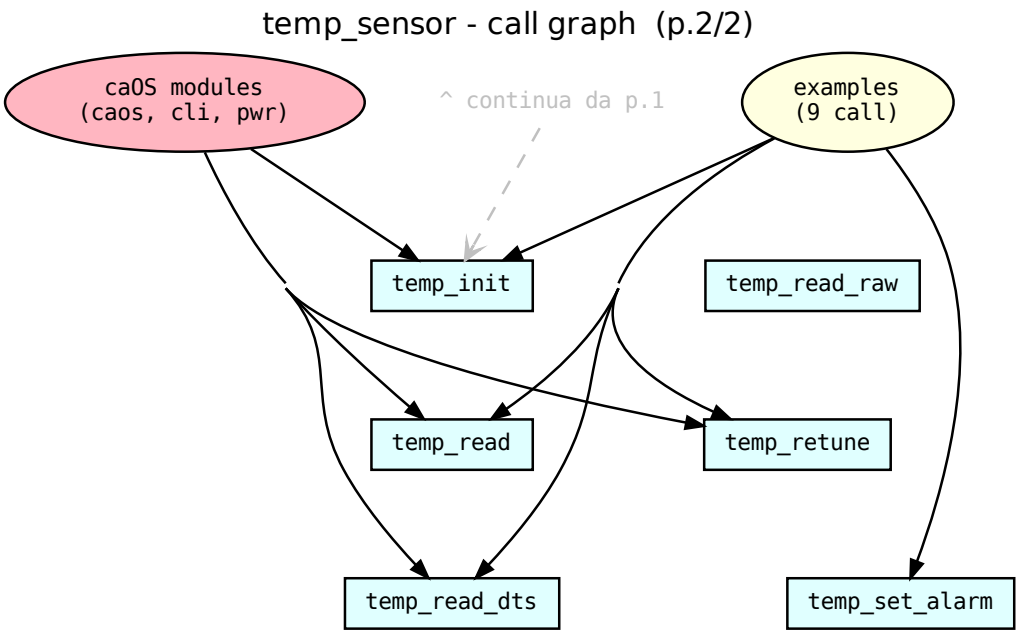

tcp_service - call graph (p.2/2)
^ continua da p.1
caOS modules
(iperf_service)
tcp_wait_writable

118.94 temp_sensor

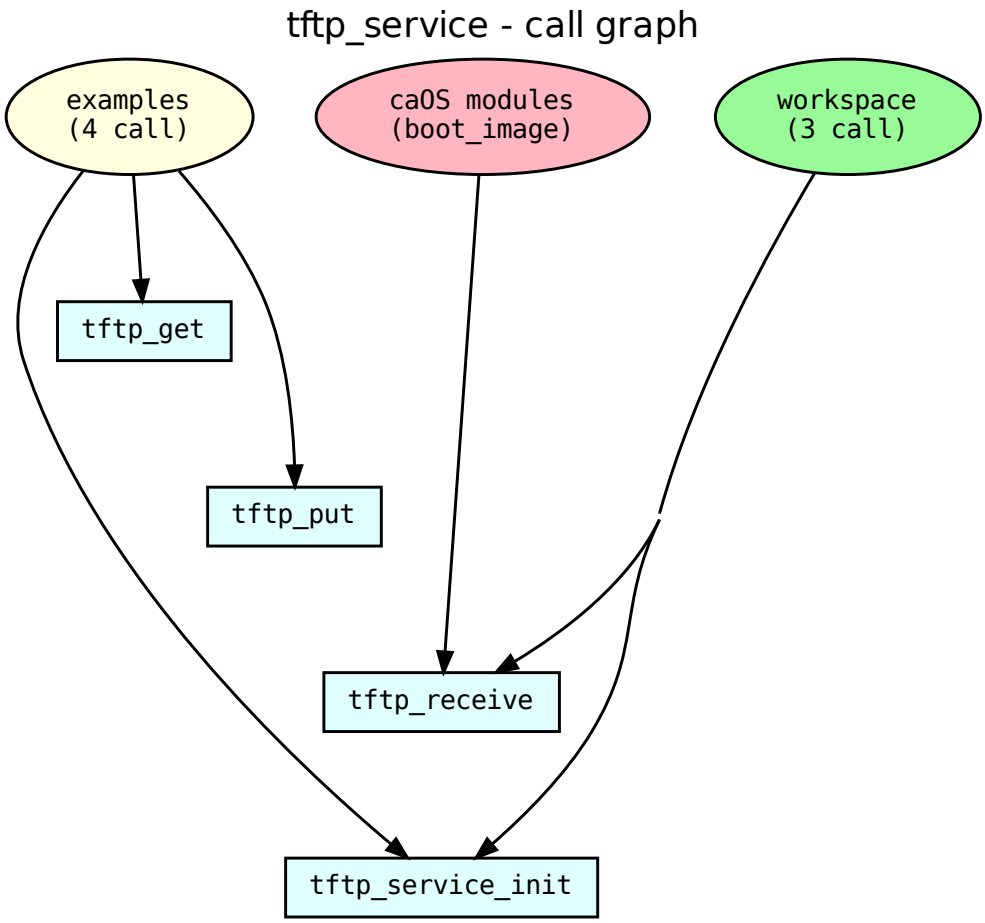
Pagina 1/2



Pagina 2/2

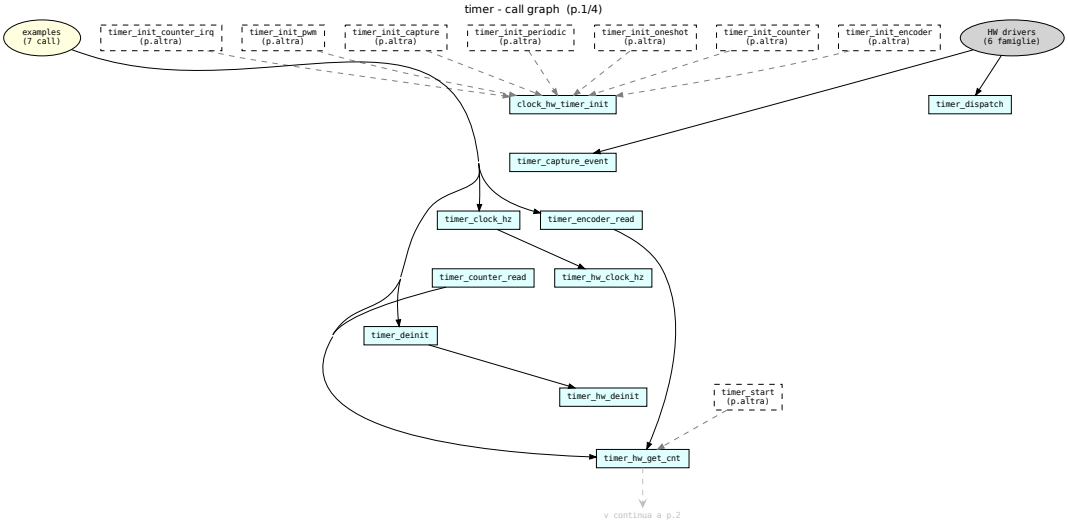

temp_sensor - call graph (p.2/2)
caOS modules
(caos, cli, pwr)
^ continua da p.1
examples
(9 call)
temp_init
temp_read_raw
temp_read
temp_retune
temp_read_dts
temp_set_alarm

118.95 tftp_service

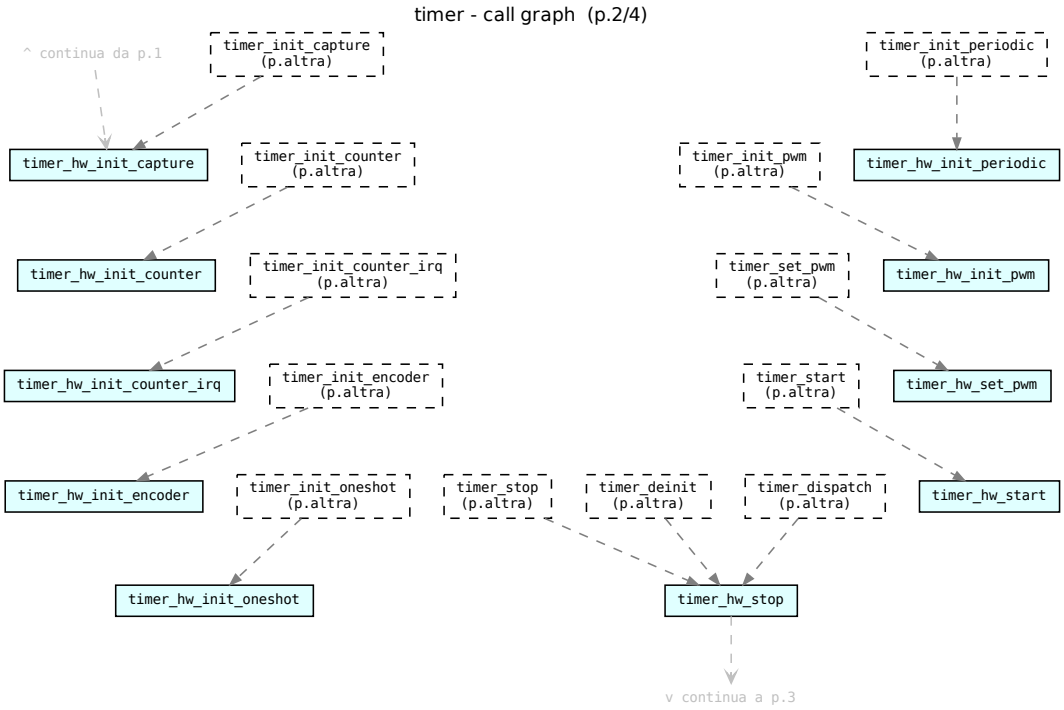


118.96 `timer`

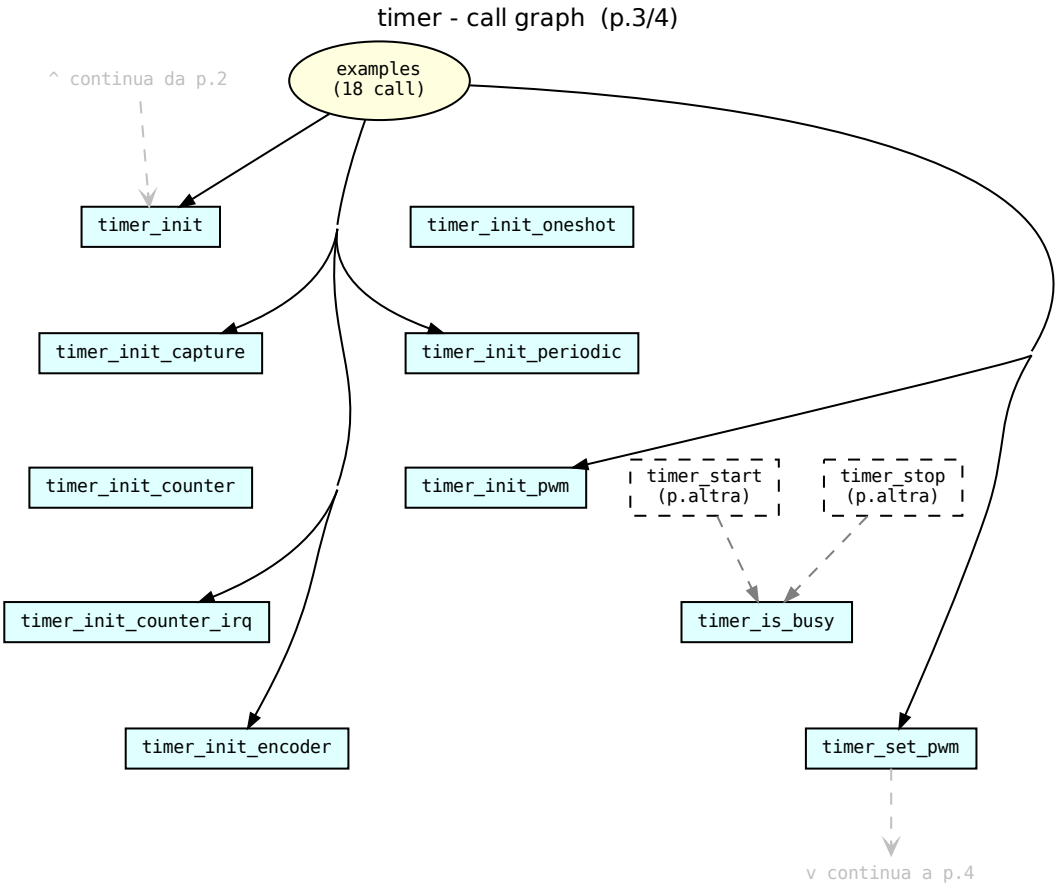
Pagina 1/4



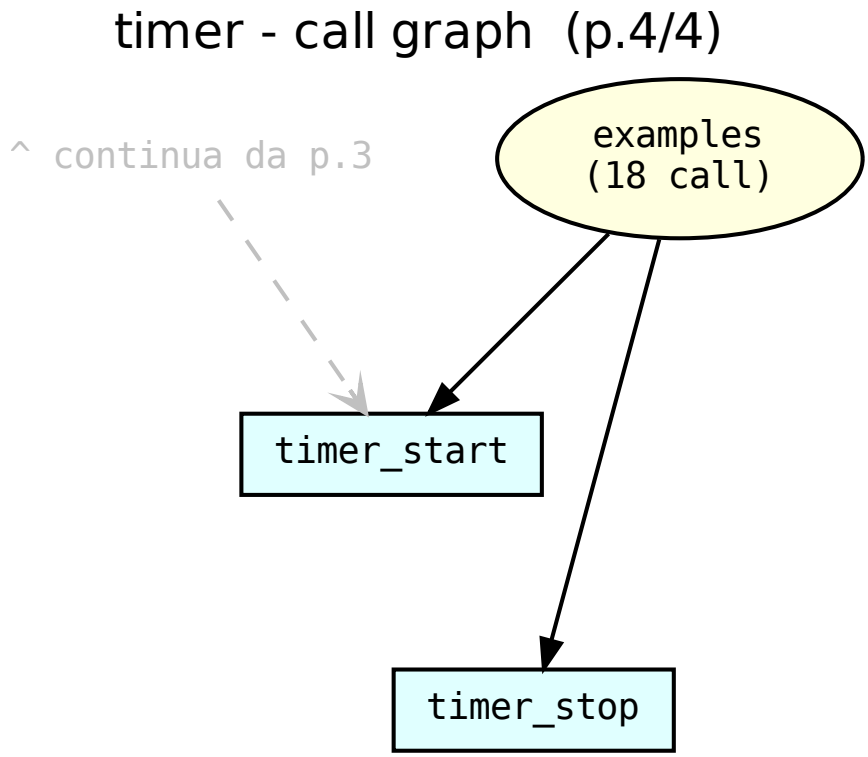
Pagina 2/4


timer - call graph (p.2/4)
^ continua da p.1
timer_init_capture (p.altra)
timer_init_periodic (p.altra)
timer_hw_init_capture
timer_init_counter (p.altra)
timer_init_pwm (p.altra)
timer_hw_init_periodic
timer_hw_init_counter
timer_init_counter_irq (p.altra)
timer_set_pwm (p.altra)
timer_hw_init_pwm
timer_hw_init_counter_irq
timer_init_encoder (p.altra)
timer_start (p.altra)
timer_hw_set_pwm
timer_hw_init_encoder
timer_init_oneshot (p.altra)
timer_stop (p.altra)
timer_deinit (p.altra)
timer_dispatch (p.altra)
timer_hw_start
timer_hw_init_oneshot
timer_hw_stop
v continua a p.3

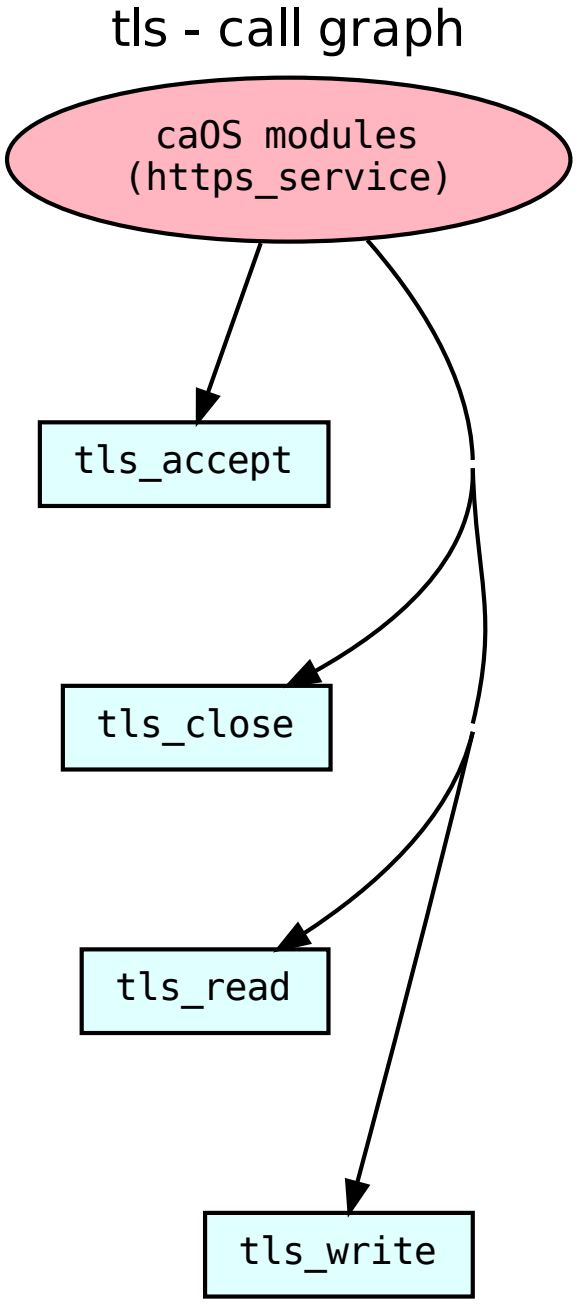
Pagina 3/4


timer - call graph (p.3/4)
^ continua da p.2
examples
(18 call)
timer_init
timer_init_oneshot
timer_init_capture
timer_init_periodic
timer_init_counter
timer_init_pwm
timer_start
(p.altra)
timer_stop
(p.altra)
timer_init_counter_irq
timer_is_busy
timer_init_encoder
timer_set_pwm
v continua a p.4

Pagina 4/4

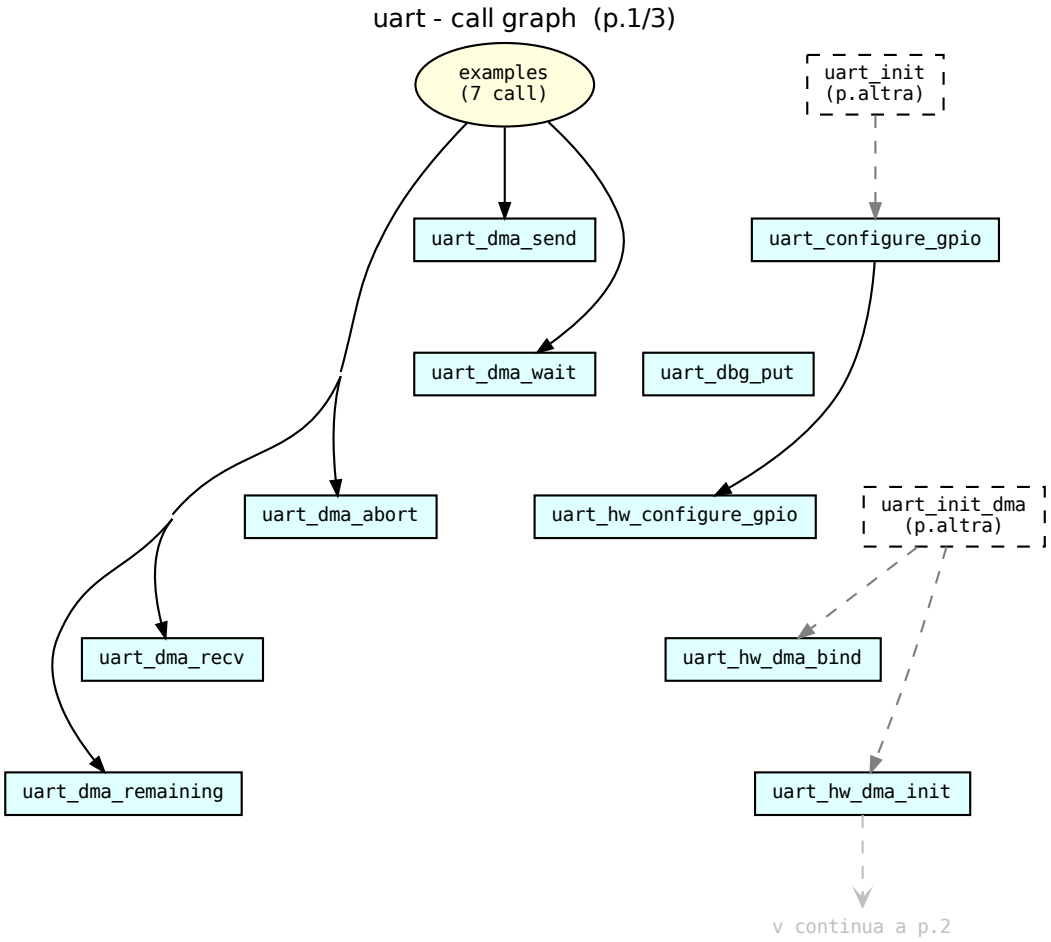

timer - call graph (p.4/4)
^ continua da p.3
examples
(18 call)
timer_start
timer_stop

118.97 `tls`

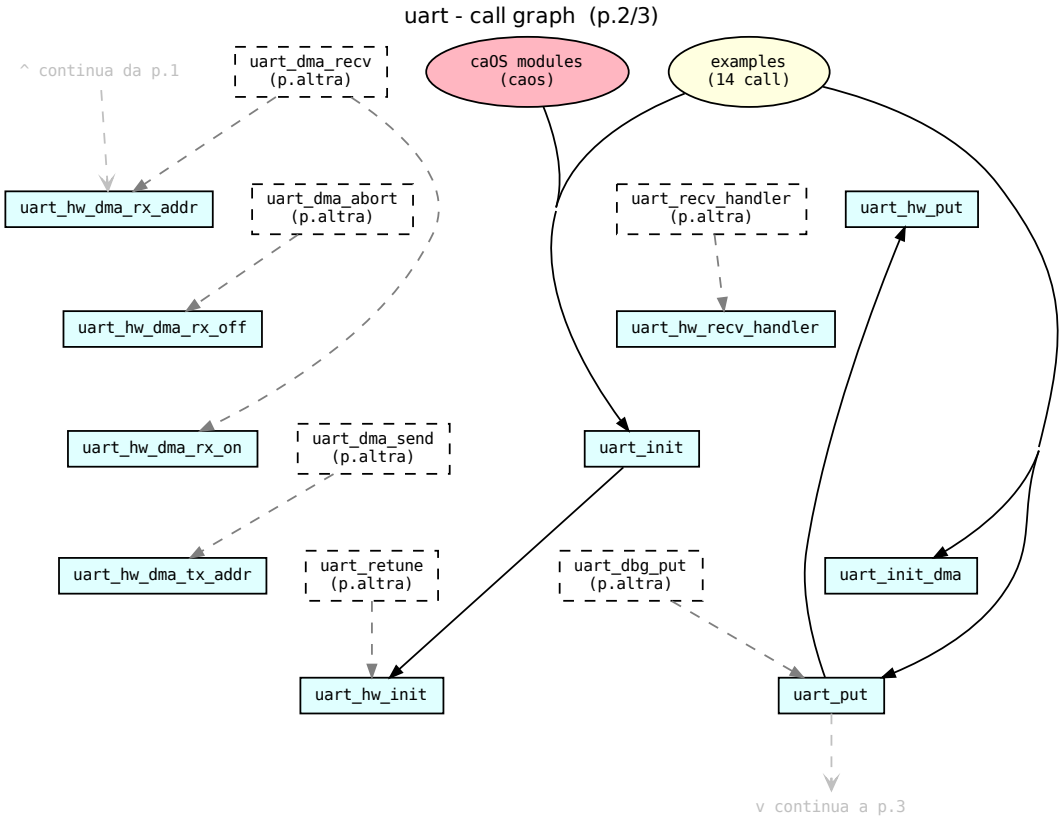

tls - call graph
caOS modules
(https_service)
tls_accept
tls_close
tls_read
tls_write

118.98 uart

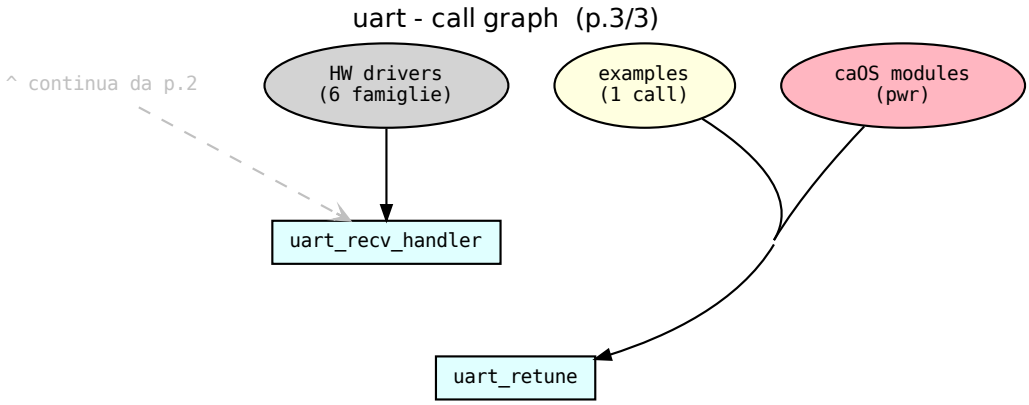
Pagina 1/3



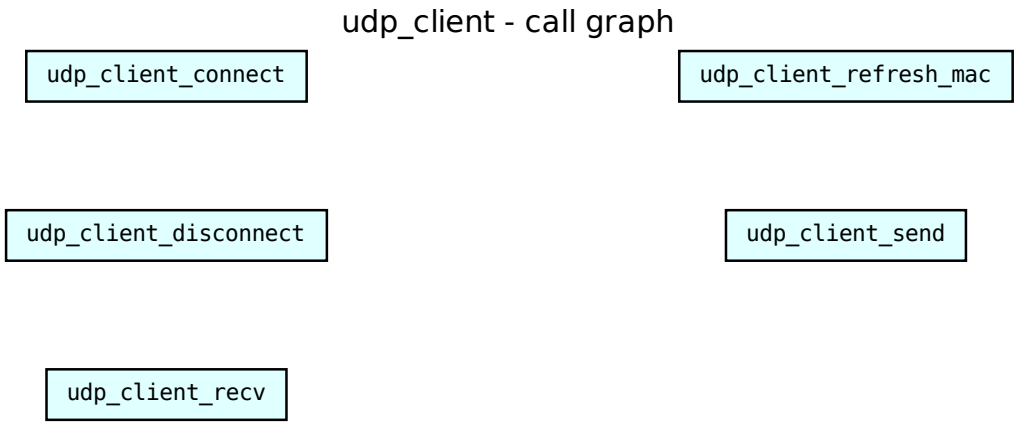
Pagina 2/3


uart - call graph (p.2/3)
^ continua da p.1
uart_dma_recv (p.altra)
caOS modules (caos)
examples (14 call)
uart_hw_dma_rx_addr
uart_dma_abort (p.altra)
uart_recv_handler (p.altra)
uart_hw_put
uart_hw_dma_rx_off
uart_hw_recv_handler
uart_hw_dma_rx_on
uart_dma_send (p.altra)
uart_init
uart_hw_dma_tx_addr
uart_retune (p.altra)
uart_dbg_put (p.altra)
uart_init_dma
uart_hw_init
uart_put
v continua a p.3

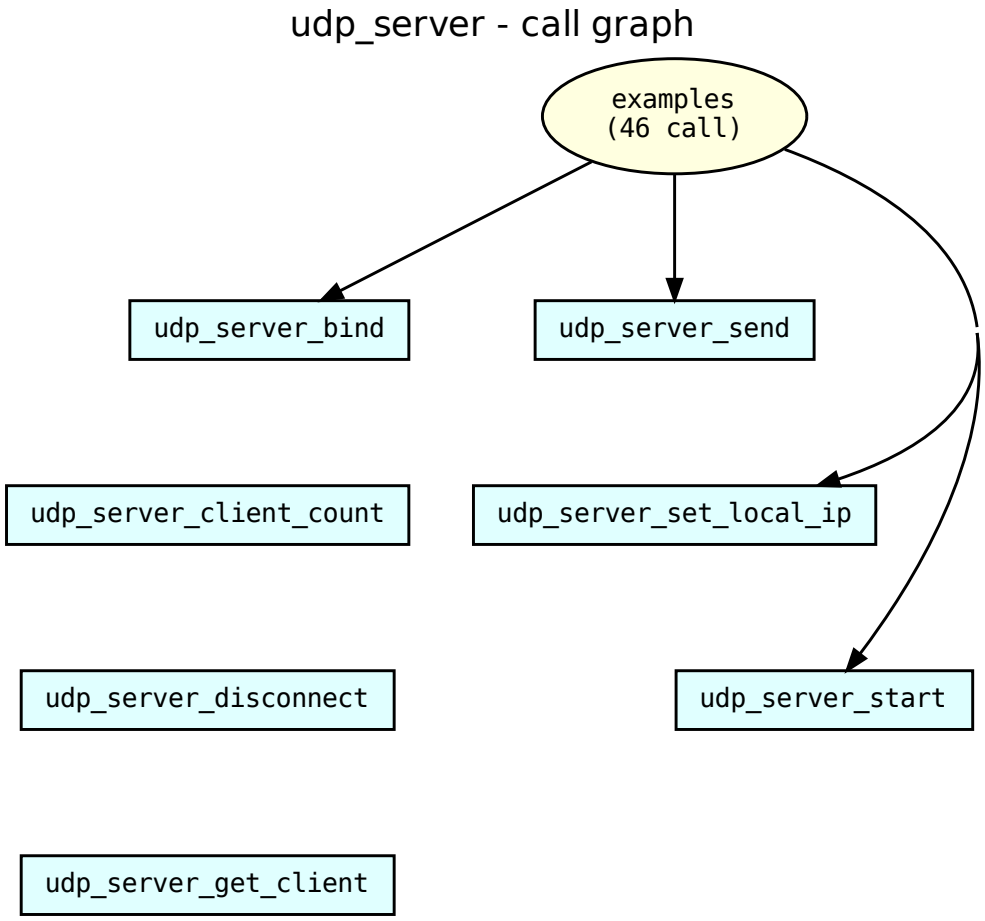
Pagina 3/3


uart - call graph (p.3/3)
^ continua da p.2
HW drivers
(6 famiglie)
examples
(1 call)
caOS modules
(pwr)
uart_recv_handler
uart_retune

118.99 udp_client

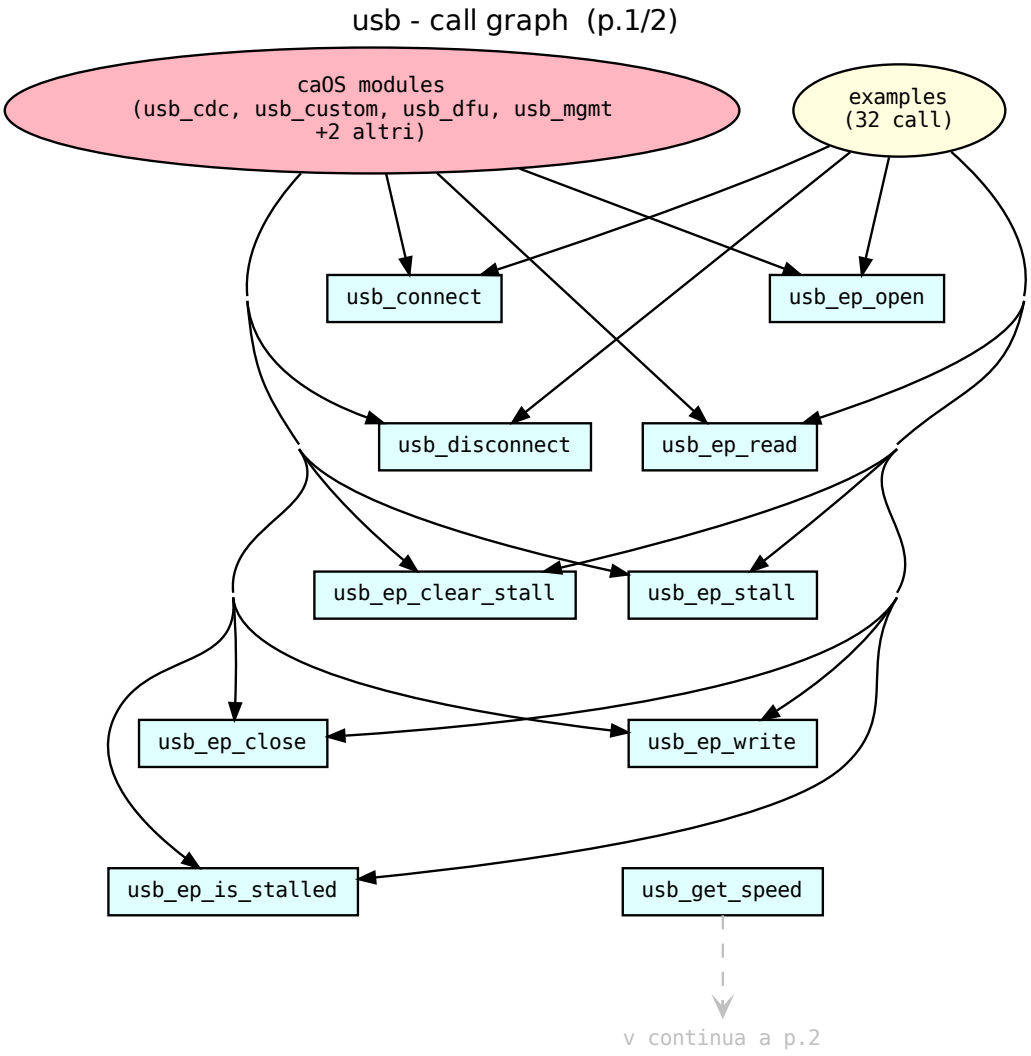


118.100 udp_server

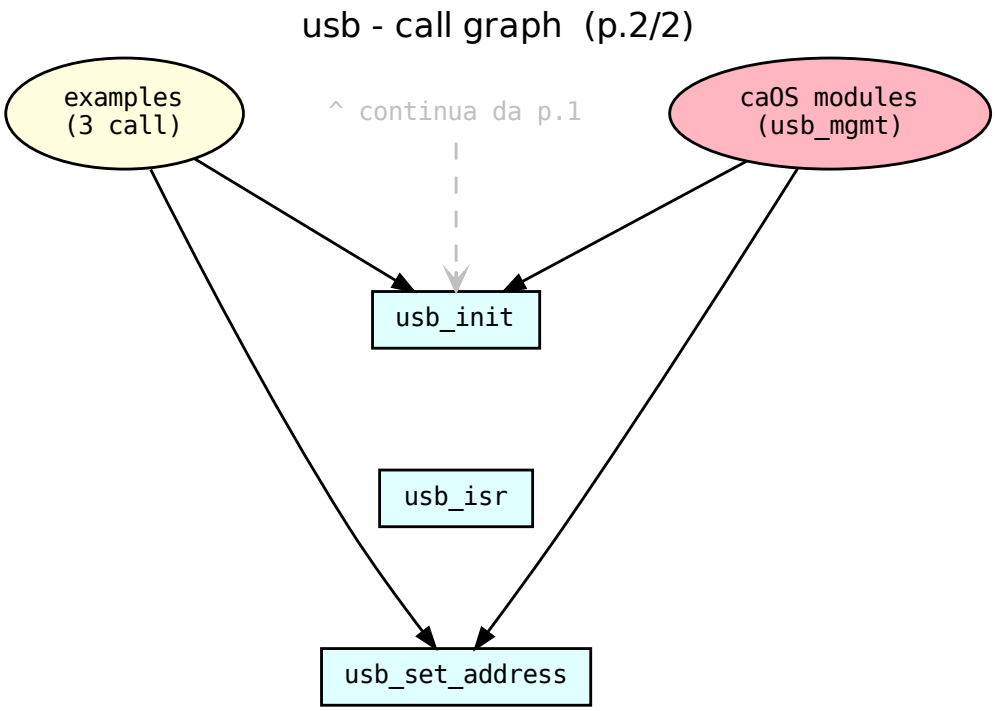


118.101 usb

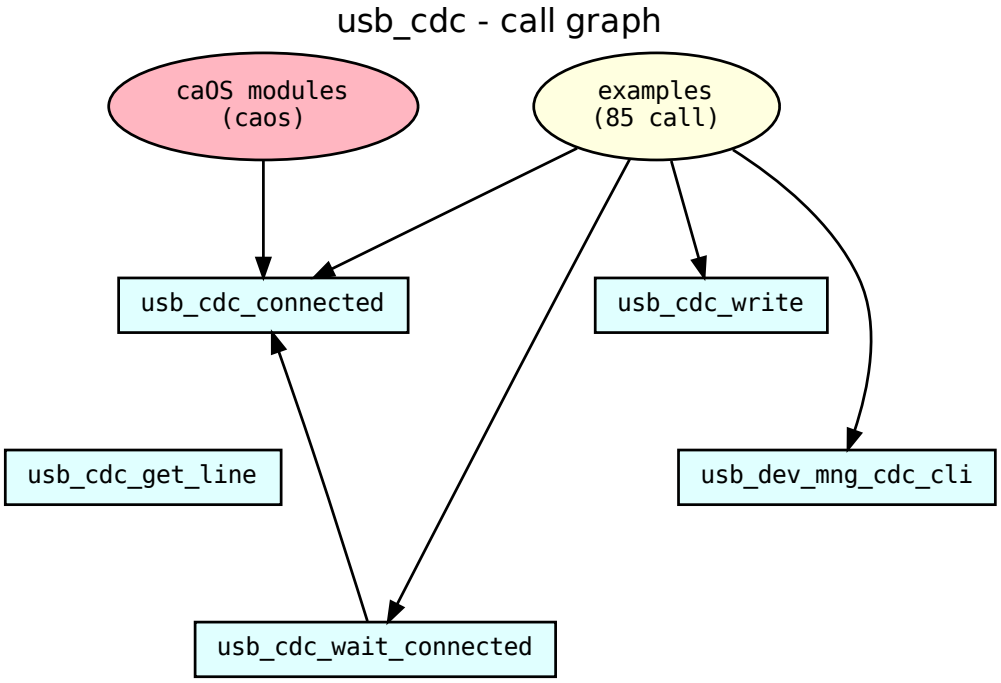
Pagina 1/2



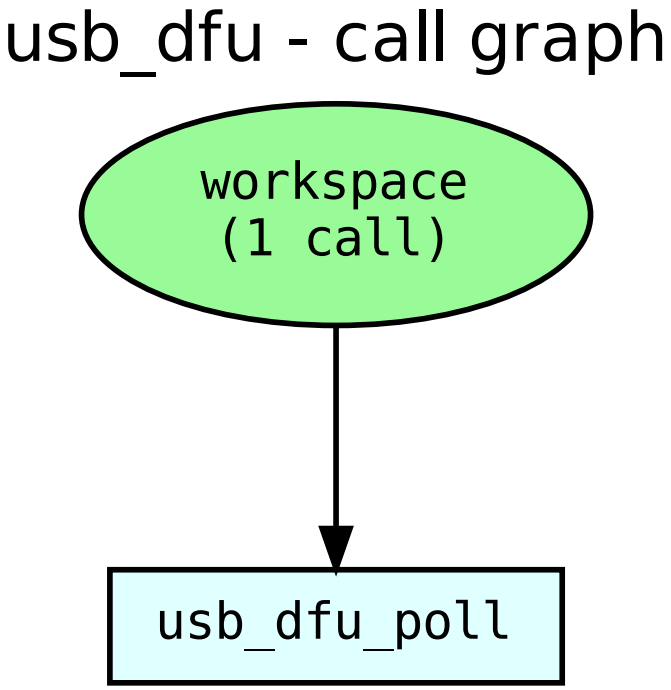
Pagina 2/2


usb - call graph (p.2/2)
examples
(3 call)
^ continua da p.1
caOS modules
(usb_mgmt)
usb_init
usb_isr
usb_set_address

118.102 usb_cdc

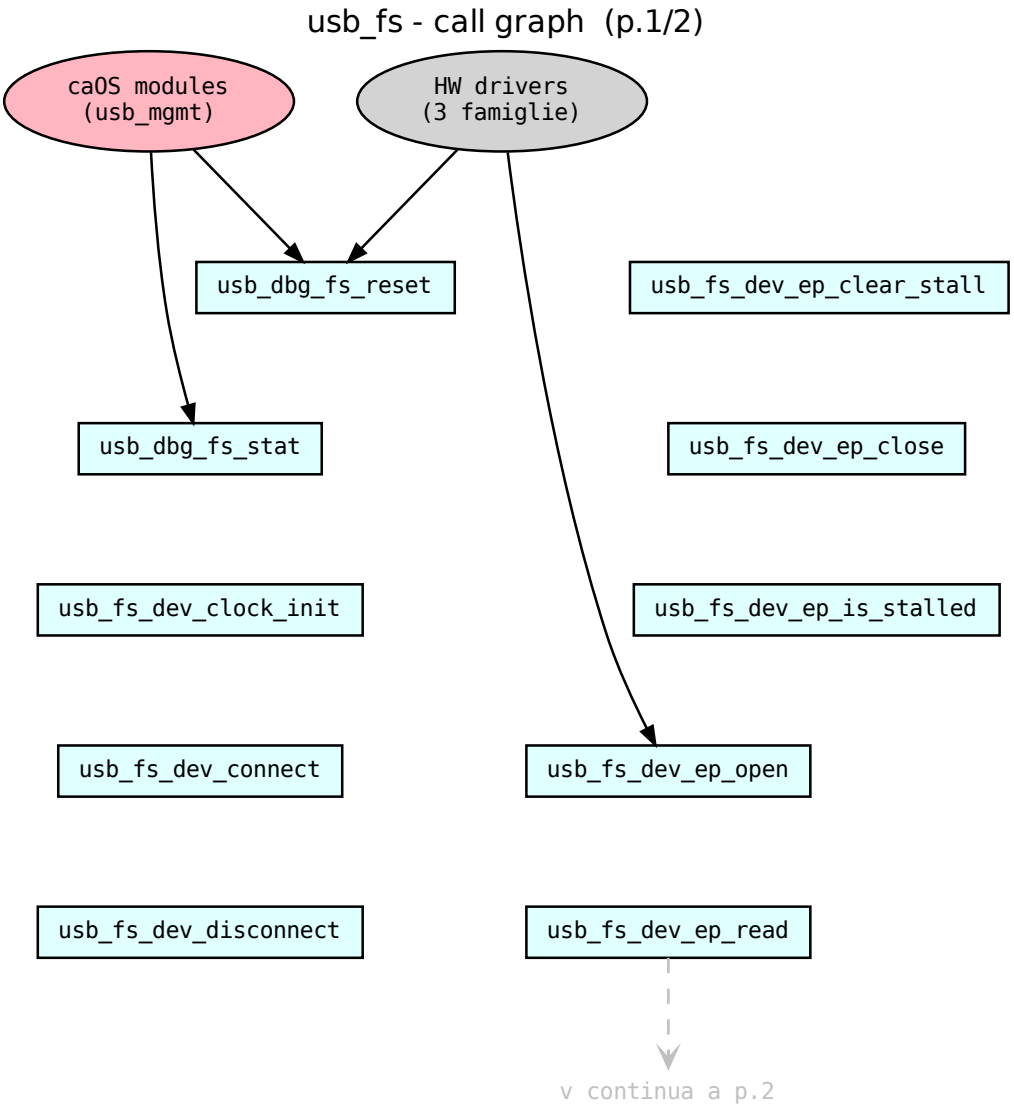


118.103 usb_dfu

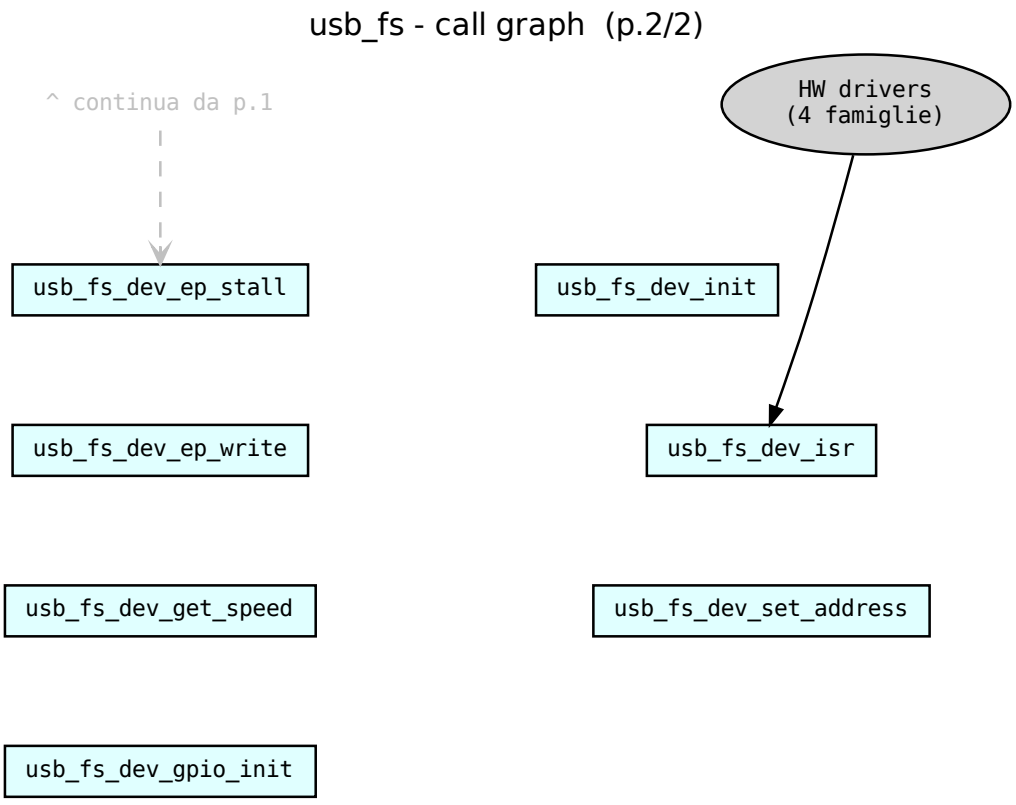


118.104 usb_fs

Pagina 1/2

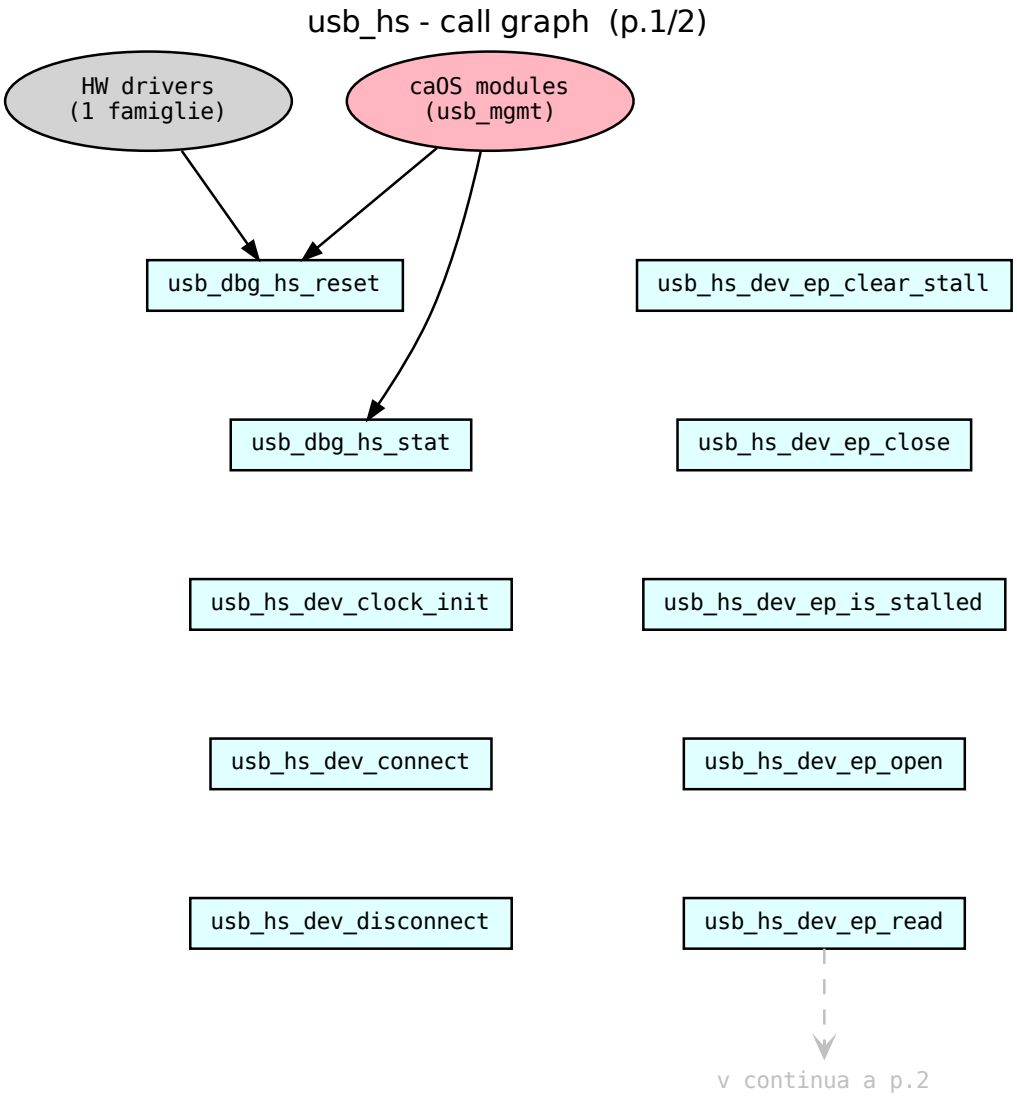


Pagina 2/2

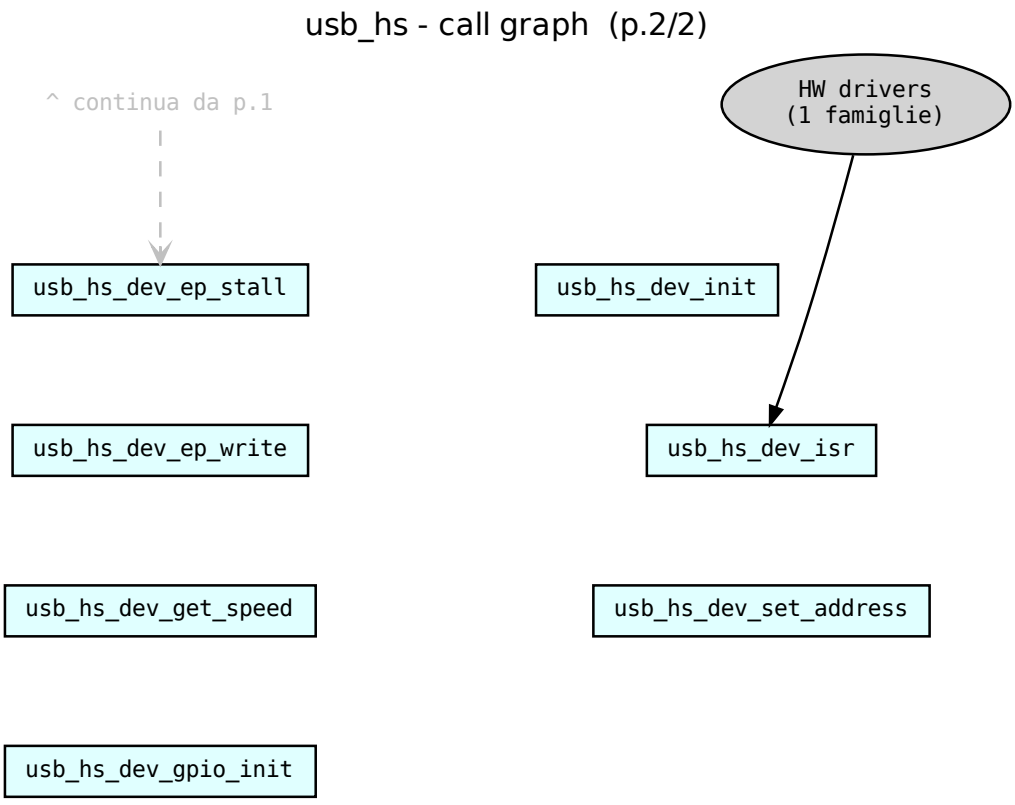

usb_fs - call graph (p.2/2)
^ continua da p.1
HW drivers
(4 famiglie)
usb_fs_dev_ep_stall
usb_fs_dev_init
usb_fs_dev_ep_write
usb_fs_dev_isr
usb_fs_dev_get_speed
usb_fs_dev_set_address
usb_fs_dev_gpio_init

118.105 usb_hs

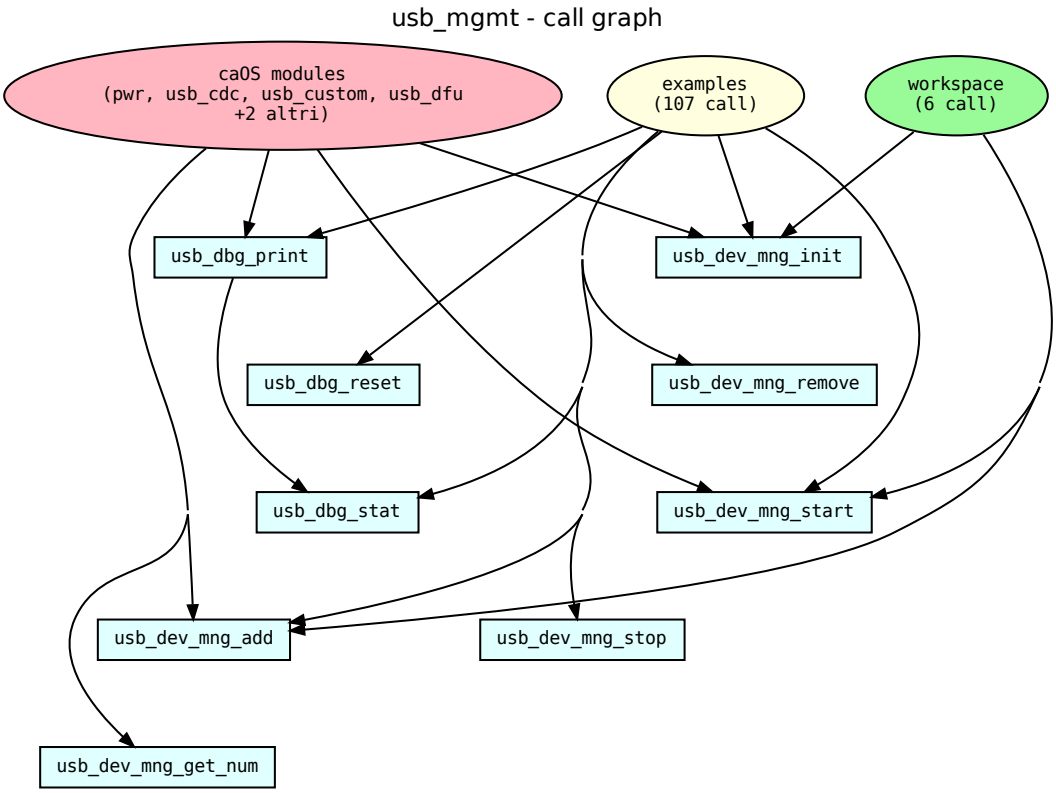
Pagina 1/2



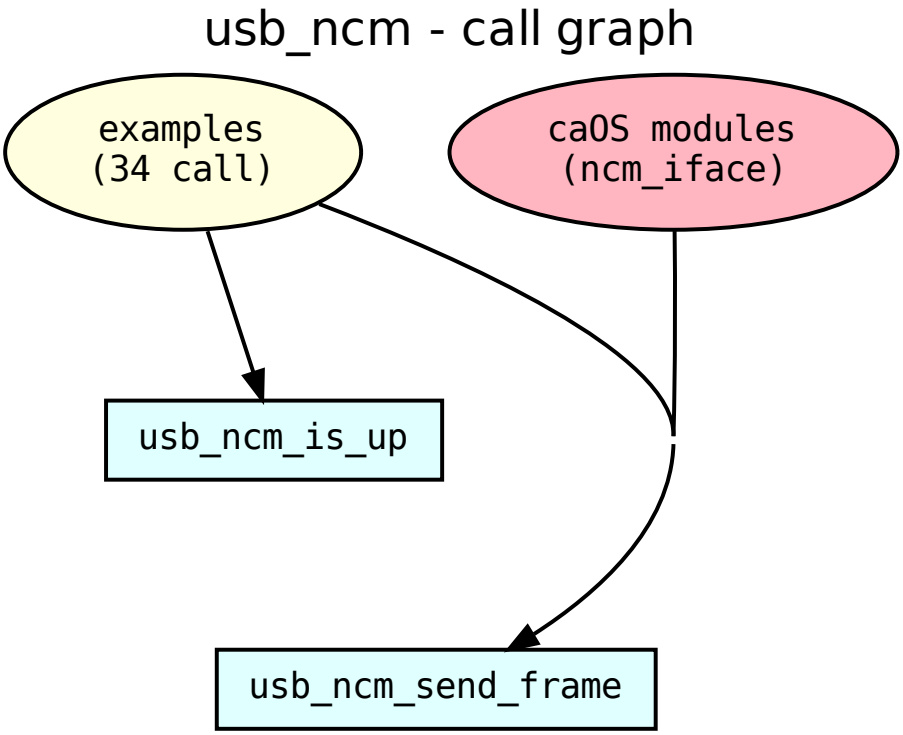
Pagina 2/2


usb_hs - call graph (p.2/2)
^ continua da p.1
HW drivers
(1 famiglie)
usb_hs_dev_ep_stall
usb_hs_dev_init
usb_hs_dev_ep_write
usb_hs_dev_isr
usb_hs_dev_get_speed
usb_hs_dev_set_address
usb_hs_dev_gpio_init

118.106 usb_mgmt

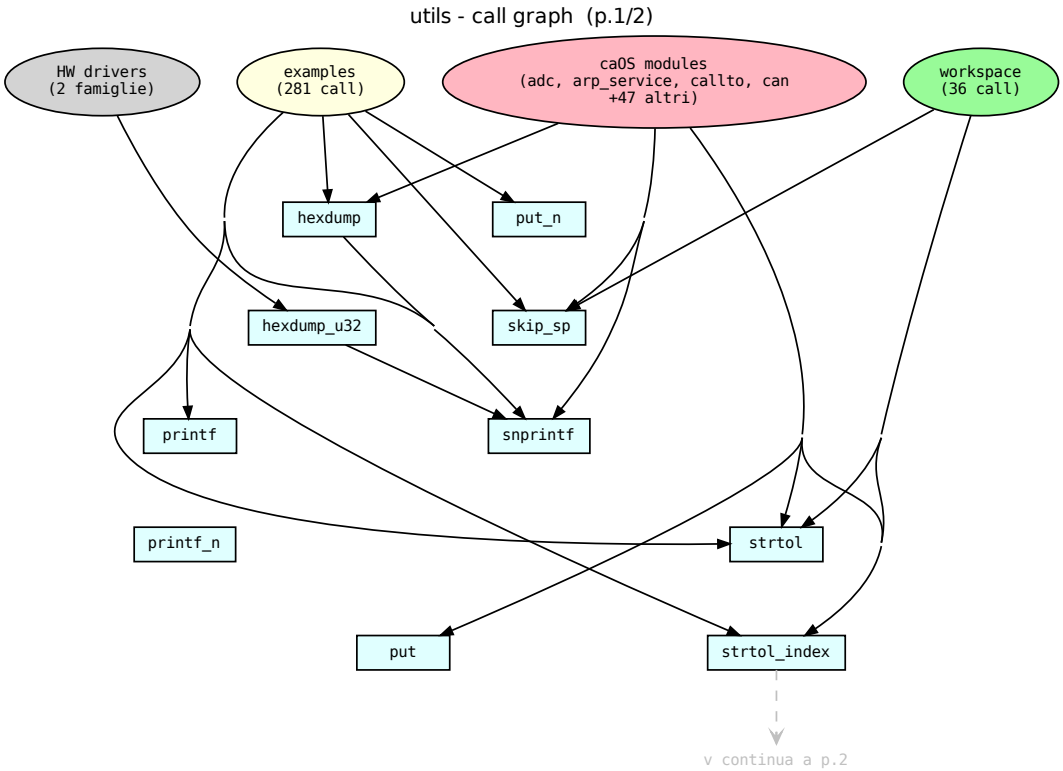


118.107 usb_ncm

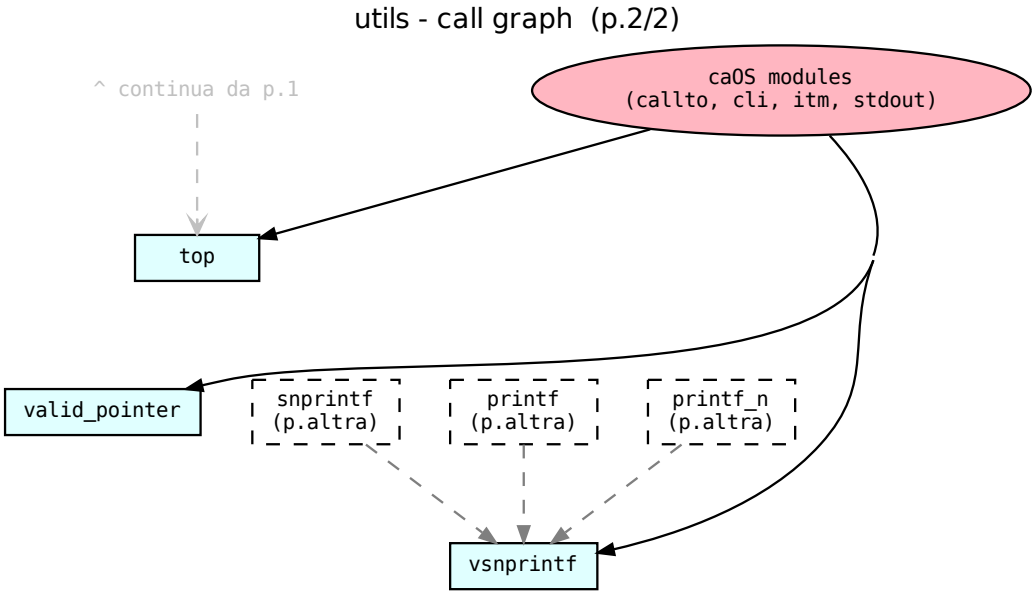


118.108 `utils`

Pagina 1/2

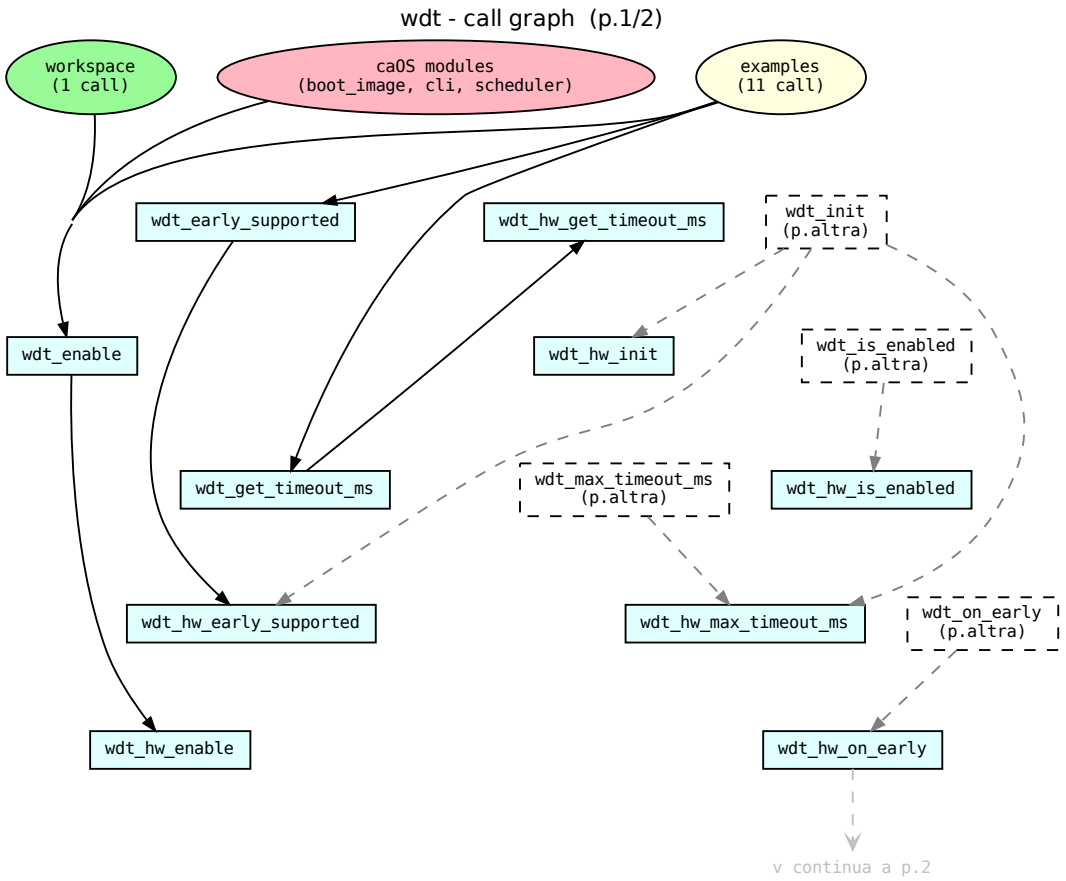


Pagina 2/2

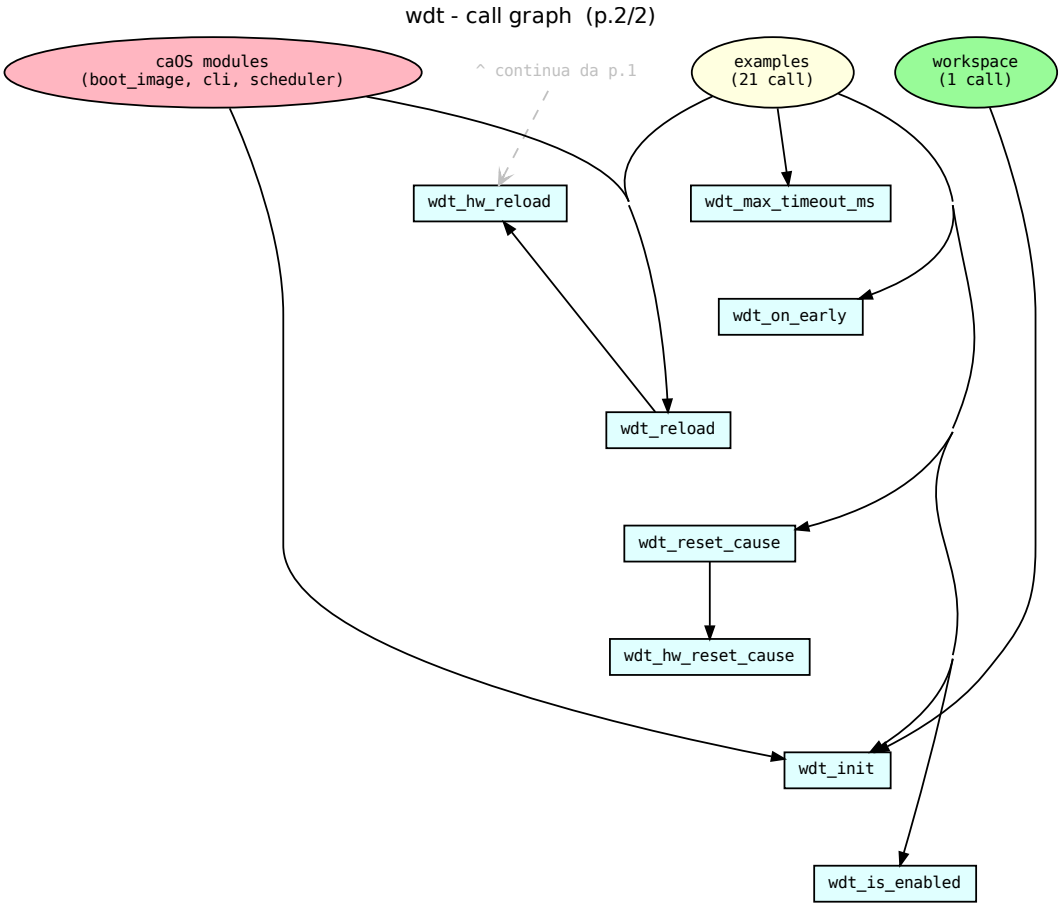

utils - call graph (p.2/2)
^ continua da p.1
caOS modules
(callto, cli, itm, stdout)
top
valid_pointer
snprintf
(p.altra)
printf
(p.altra)
printf_n
(p.altra)
vsnprintf

118.109 `wdt`

Pagina 1/2



Pagina 2/2


wdt - call graph (p.2/2)
caOS modules
(boot_image, cli, scheduler)
^ continua da p.1
examples
(21 call)
workspace
(1 call)
wdt_hw_reload
wdt_max_timeout_ms
wdt_on_early
wdt_reload
wdt_reset_cause
wdt_hw_reset_cause
wdt_init
wdt_is_enabled

118.110 x25519